Universes in Pairs

A Deep-Dive Course on the dirac16complex Fields in the Author's 4+4 Primordial Universe, with a Complete Jupyter Notebook for Every Example

dirac16complex project (with Claude Opus 5.5)

October 2026

Contents

0 How to use this book; installing the software; the honesty ledger 28

0.1 What this book teaches . . . 28
0.2 The request, and what the equations prove and do not prove . . . 30
0.3 The five labels of every statement . . . 34
0.4 How to check a statement yourself . . . 35
0.5 Conventions used everywhere . . . 35
0.6 The map of the book . . . 36
0.7 Every example is a notebook . . . 37
0.8 Installing the software . . . 38
0.9 How the notebooks are built and checked . . . 40
0.10 Example: checking the installation . . . 41
0.11 How to run Notebook 00a . . . 42
0.12 Notebook 00a: complete text . . . 47
0.13 Line-by-line walk-through of Notebook 00a . . . 68
0.14 The eight directions and the author's metric . . . 77
0.15 How to run Notebook 00b . . . 82
0.16 Notebook 00b: complete text . . . 87
0.17 Line-by-line walk-through of Notebook 00b . . . 116
0.18 The honesty ledger, and how a report is tied to its files . . . 130
0.19 How to run Notebook 00c . . . 137

0.20 Notebook 00c: complete text . . . 143
0.21 Line-by-line walk-through of Notebook 00c . . . 185
0.22 Why every run gives the same bytes . . . 212
0.23 How to run Notebook 00d . . . 217
0.24 Notebook 00d: complete text . . . 223
0.25 Line-by-line walk-through of Notebook 00d . . . 260
0.26 What we proved, what we computed, what we assumed . . . 276
0.27 Exercises . . . 278

1 Mathematics from zero I: numbers, complex numbers, vectors, matrices, indices, determinants, the generalized Kronecker delta 280

1.1 What this chapter is for . . . 280
1.2 Whole numbers and fractions, exactly . . . 282
1.3 Real numbers: the square root of 2 is not a fraction . . . 284
1.4 Floating-point numbers, rounding and cancellation . . . 285
1.5 Powers, roots, the exponential function and the logarithm . . . 286
1.6 Example: numbers, rounding and the length factors of the author's metric . . . 288
1.7 How to run Notebook 01f . . . 288
1.8 Notebook 01f: complete text . . . 293
1.9 Line-by-line walk-through of Notebook 01f . . . 327
1.10 Complex numbers: the rules of arithmetic . . . 345
1.11 The exponential series and Euler's formula . . . 347
1.12 Multiplication turns the plane; the real matrix of a complex number . . . 349
1.13 Boosts, hyperbolic functions, oscillation and growth . . . 352
1.14 Example: complex numbers, rotations and boosts . . . 354
1.15 How to run Notebook 01b . . . 354
1.16 Notebook 01b: complete text . . . 359
1.17 Line-by-line walk-through of Notebook 01b . . . 391
1.18 Vectors and matrices; the matrix product . . . 403
1.19 Permutations and their signs . . . 405
1.20 Determinants: the Leibniz formula and its six rules . . . 407
1.21 What a determinant measures, what it costs, and two matrices of the course . . . 408

1.22 Example: matrices, permutations and determinants 410
1.23 How to run Notebook 01a 410
1.24 Notebook 01a: complete text 416
1.25 Line-by-line walk-through of Notebook 01a 450
1.26 Index notation and the summation convention 464
1.27 The frame metric: lowering, raising and squared lengths 465
1.28 Symmetric and antisymmetric arrays; commutators; the Clifford relation 466
1.29 The author's metric in index notation 468
1.30 Example: index notation in the author's spacetime 469
1.31 How to run Notebook 01e 470
1.32 Notebook 01e: complete text 475
1.33 Line-by-line walk-through of Notebook 01e 509
1.34 Eigenvalues and eigenvectors; the characteristic polynomial 523
1.35 The chain matrix of any size; rotations and boosts; conjugate pairs 525
1.36 Symmetric and Hermitian matrices; the signature and Sylvester's law 527
1.37 The eigenvalues of the gamma matrices and of B; power iteration 529
1.38 Example: eigenvalues, eigenvectors and the signature (4,4) 530
1.39 How to run Notebook 01g 531
1.40 Notebook 01g: complete text 536
1.41 Line-by-line walk-through of Notebook 01g 577
1.42 The generalized Kronecker delta: the author's definition and its four cases 596
1.43 Counting the values; nine indices in eight dimensions 598
1.44 Contracting one index; the Lovelock trace factors; the Levi-Civita symbol 599
1.45 Example: the generalized Kronecker delta 601
1.46 How to run Notebook 01c 601
1.47 Notebook 01c: complete text 606
1.48 Line-by-line walk-through of Notebook 01c 645
1.49 Binary numbers, bit operations and a pseudo-random generator 664
1.50 Fingerprints, the exporter's file and the birthday problem 666
1.51 Example: reproducing the record's tests of the generalized delta 667
1.52 How to run Notebook 01d 667

1.53 Notebook 01d: complete text 672
1.54 Line-by-line walk-through of Notebook 01d 706
1.55 What we proved, what we computed, what we assumed 721
1.56 Exercises 724

2 Mathematics from zero II: several variables, ODEs, Euler and RK4, convergence, the shooting method 727

2.1 What this chapter is for 727
2.2 Differential equations 728
2.3 Taylor's theorem and the exponential series 730
2.4 Three step rules: Euler, midpoint and RK4 731
2.5 The amplification factor 733
2.6 Error, order, the log-log plot, rounding and Richardson's rule 734
2.7 The oscillator and the product of the two scale factors 736
2.8 Example: Euler, midpoint and RK4 on the deflating scale factor and the oscillator 739
2.9 How to run Notebook 02a 739
2.10 Notebook 02a: complete text 744
2.11 Line-by-line walk-through of Notebook 02a 781
2.12 Boundary-value problems and the shooting method 802
2.13 The finite square well of quantum mechanics 804
2.14 Example: shooting a string and a quantum well 807
2.15 How to run Notebook 02b 807
2.16 Notebook 02b: complete text 812
2.17 Line-by-line walk-through of Notebook 02b 847
2.18 Functions of several variables and partial derivatives 863
2.19 The chain rule and the derivatives of the author's metric 866
2.20 Example: partial derivatives of a model function and of the author's metric . . . 870
2.21 How to run Notebook 02c 870
2.22 Notebook 02c: complete text 875
2.23 Line-by-line walk-through of Notebook 02c 907
2.24 Two first-order equations, the Prüfer angle and Newton's method 919
2.25 Example: the shooting method of the Revision Kohn-Sham solver, rewritten . . . 926

2.26 How to run Notebook 02d 926
2.27 Notebook 02d: complete text 931
2.28 Line-by-line walk-through of Notebook 02d 972
2.29 What we proved, what we computed, what we assumed 991
2.30 Exercises 993

3 Spacetime geometry and the author's metric; curvature 995

3.1 What this chapter is for 995
3.2 Coordinates, indices and sums 997
3.3 The metric: lengths from coordinate steps 998
3.4 Signature: space-like and time-like directions 999
3.5 The author's metric 1000
3.6 The determinant and proper volumes 1001
3.7 Scale factors, the vielbein and the rates of expansion 1003
3.8 The deflating history: a prescribed background 1004
3.9 The hidden coordinate y and the warped form of the metric 1005
3.10 Example: the author's metric, read and checked 1006
3.11 How to run Notebook 03a 1007
3.12 Notebook 03a: complete text 1012
3.13 Line-by-line walk-through of Notebook 03a 1052
3.14 Vectors, covectors and tensors 1074
3.15 The covariant derivative and the Christoffel symbols 1076
3.16 Parallel transport and geodesics 1080
3.17 Curvature: the Riemann tensor and its contractions 1083
3.18 The curvature of the plane and of the sphere, by hand; geodesic deviation 1085
3.19 Example: curvature where it can be pictured 1087
3.20 How to run Notebook 03c 1087
3.21 Notebook 03c: complete text 1092
3.22 Line-by-line walk-through of Notebook 03c 1124
3.23 The Christoffel symbols of the author's metric 1136
3.24 The curvature of the author's metric, by hand 1139

3.25 Ricci, Einstein and Kretschmann of the author's metric; the contracted Bianchi identity . . . 1143
3.26 What Einstein's equations would ask of the source . . . 1146
3.27 Example: the curvature of the author's metric . . . 1148
3.28 How to run Notebook 03b . . . 1149
3.29 Notebook 03b: complete text . . . 1154
3.30 Line-by-line walk-through of Notebook 03b . . . 1207
3.31 Free fall in the author's metric . . . 1234
3.32 Example: free fall in the author's metric . . . 1238
3.33 How to run Notebook 03d . . . 1238
3.34 Notebook 03d: complete text . . . 1243
3.35 Line-by-line walk-through of Notebook 03d . . . 1279
3.36 What we proved, what we computed, what we assumed . . . 1293
3.37 Exercises . . . 1296

4 Clifford algebra Cl(4,4) and the author's T16 1298

4.1 Why matrices, and why sixteen components . . . 1298
4.2 The matrix tools of this chapter . . . 1300
4.3 The square-root problem and the Clifford relation . . . 1303
4.4 The eight directions of the author's space-time and their signs . . . 1305
4.5 The author's construction of T16, step by step . . . 1306
4.6 How to run Notebook 04a . . . 1311
4.7 Notebook 04a: complete text . . . 1316
4.8 Line-by-line walk-through of Notebook 04a . . . 1360
4.9 Square roots in 4+4 dimensions and the plane waves of the field . . . 1382
4.10 How to run Notebook 04b . . . 1386
4.11 Notebook 04b: complete text . . . 1391
4.12 Line-by-line walk-through of Notebook 04b . . . 1427
4.13 The 256 products: Cl(4,4) is all real 16 by 16 matrices . . . 1441
4.14 How to run Notebook 04c . . . 1444
4.15 Notebook 04c: complete text . . . 1450
4.16 Line-by-line walk-through of Notebook 04c . . . 1487

4.17 A second set of gammas and the one change of basis 1502
4.18 How to run Notebook 04d 1506
4.19 Notebook 04d: complete text 1511
4.20 Line-by-line walk-through of Notebook 04d 1548
4.21 What we proved, what we computed, what we assumed 1561
4.22 Exercises 1564

5 The matrices C, Γ and B, the groups Pin(4,4) and Spin(4,4), and the charge-conjugation matrices 1565

5.1 What this chapter does 1566
5.2 The eight real gamma matrices 1567
5.3 Six rules for products of gammas 1569
5.4 The charge matrix C and the Dirac adjoint 1571
5.5 The chirality and the two halves 1573
5.6 The matrix B and the charge density 1575
5.7 Example: Notebook 05a checks C, Γ and B 1579
5.8 How to run Notebook 05a 1579
5.9 Notebook 05a: complete text 1584
5.10 Line-by-line walk-through of Notebook 05a 1620
5.11 Groups and representations from zero 1637
5.12 Pin(4,4), Spin(4,4) and the generators S^{ab} 1639
5.13 Irreducible under Pin(4,4), two inequivalent halves under Spin(4,4) 1641
5.14 Example: Notebook 05b computes the commutants 1644
5.15 How to run Notebook 05b 1644
5.16 Notebook 05b: complete text 1649
5.17 Line-by-line walk-through of Notebook 05b 1682
5.18 Spin transformations: rotations, boosts and the double cover 1694
5.19 Example: Notebook 05d computes spin transformations 1698
5.20 How to run Notebook 05d 1698
5.21 Notebook 05d: complete text 1703
5.22 Line-by-line walk-through of Notebook 05d 1738
5.23 What do the scaled commutators generate? 1750

5.24 Example: Notebook 05f computes what the scaled commutators generate 1755
5.25 How to run Notebook 05f 1755
5.26 Notebook 05f: complete text 1760
5.27 Line-by-line walk-through of Notebook 05f 1802
5.28 Charge conjugation is a matrix 1820
5.29 The bilinears, the reality conditions and real fields 1826
5.30 Example: Notebook 05c computes the charge-conjugation matrices 1829
5.31 How to run Notebook 05c 1829
5.32 Notebook 05c: complete text 1834
5.33 Line-by-line walk-through of Notebook 05c 1874
5.34 The quantised field: operators, the indefinite form and normal ordering 1892
5.35 Example: Notebook 05e computes the conjugation of the quantised field 1897
5.36 How to run Notebook 05e 1898
5.37 Notebook 05e: complete text 1903
5.38 Line-by-line walk-through of Notebook 05e 1943
5.39 What the charge-conjugation matrices say, and do not say, about matter and antimatter 1961
5.40 What we proved, what we computed, what we assumed 1963
5.41 Exercises 1965

6 Spinors in curved 4+4 space: vielbein, spin connection and $\gamma^\mu\Omega_\mu$ 1967

6.1 What this chapter does 1968
6.2 A frame at every point: the vielbein 1970
6.3 The Christoffel symbols of the author's metric 1973
6.4 The canonical spin connection: the vielbein postulate and its solution 1975
6.5 A warm-up by hand: the flat plane in polar coordinates 1977
6.6 The spin connection of the author's metric: twelve components 1979
6.7 The spinor connection and the covariant derivative of a spinor 1980
6.8 The contraction $\gamma^\mu\Omega_\mu$, direction by direction 1983
6.9 Why a_4 drops out and b_4 does not: the divergence form 1985
6.10 The Dirac operator written out, non-triviality and the negative control 1987
6.11 Example: Notebook 06a 1989

6.12 How to run Notebook 06a 1990
6.13 Notebook 06a: complete text 1995
6.14 Line-by-line walk-through of Notebook 06a 2048
6.15 Changing the frame: local spin covariance 2082
6.16 A boost in the plane of $x4$ and $x0$ 2084
6.17 What no frame can remove: the curvature 2086
6.18 Changing the field variable: the rescaling $\Psi = \sin^{-1/2}(z)\, b_4^{-3/4}\, \chi$ 2090
6.19 Example: Notebook 06b 2092
6.20 How to run Notebook 06b 2092
6.21 Notebook 06b: complete text 2097
6.22 Line-by-line walk-through of Notebook 06b 2136
6.23 Upper and lower frame indices: the mixed contraction 2154
6.24 Why only the value $3H$ is right 2157
6.25 Example: Notebook 06c 2158
6.26 How to run Notebook 06c 2158
6.27 Notebook 06c: complete text 2164
6.28 Line-by-line walk-through of Notebook 06c 2204
6.29 The exact scope of the statement that gravity enters the field equations 2223
6.30 What we proved, what we computed, what we assumed 2225
6.31 Exercises 2227

7 The two fields and their Lagrangians; Grassmann numbers; Euler-Lagrange equations 2229

7.1 What this chapter does 2229
7.2 Paths, the action and the first variation 2231
7.3 From the principle to the Euler-Lagrange equation 2233
7.4 Energy, total derivatives and first-order Lagrangians 2236
7.5 Fields: one derivative term per coordinate, and what an extra time does 2238
7.6 Example: Notebook 07d derives the Euler-Lagrange equation 2240
7.7 How to run Notebook 07d 2240
7.8 Notebook 07d: complete text 2246
7.9 Line-by-line walk-through of Notebook 07d 2286

7.10 Grassmann numbers from zero . . . 2309
7.11 Even and odd elements, conjugation and derivatives . . . 2310
7.12 Bilinear forms, real columns and the powers of S . . . 2311
7.13 Field equations with Grassmann numbers . . . 2314
7.14 Example: Notebook 07a builds a Grassmann algebra by hand . . . 2315
7.15 How to run Notebook 07a . . . 2315
7.16 Notebook 07a: complete text . . . 2320
7.17 Line-by-line walk-through of Notebook 07a . . . 2361
7.18 The geometry the fields live in . . . 2378
7.19 The two fields and their common Lagrangian . . . 2382
7.20 Reality, the total divergence, and the connection that drops out of $\mathcal{L}$ . . . 2383
7.21 The Euler-Lagrange equations of both fields . . . 2385
7.22 The equations in the author's metric: explicit, block and evolution forms . . . 2387
7.23 Non-triviality [1] and [2], their exact scope, and an exact family of solutions . . . 2389
7.24 Example: Notebook 07b derives the field equations in the author's metric . . . 2395
7.25 How to run Notebook 07b . . . 2395
7.26 Notebook 07b: complete text . . . 2400
7.27 Line-by-line walk-through of Notebook 07b . . . 2453
7.28 Real fields and the Majorana-type negative control . . . 2478
7.29 Charge conjugation is a matrix . . . 2480
7.30 Example: Notebook 07c: real fields and the charge-conjugation matrices . . . 2483
7.31 How to run Notebook 07c . . . 2483
7.32 Notebook 07c: complete text . . . 2488
7.33 Line-by-line walk-through of Notebook 07c . . . 2524
7.34 What we proved, what we computed, what we assumed . . . 2538
7.35 Exercises . . . 2542

8 Non-triviality, its exact scope, and well-posedness 2545

8.1 What this chapter does . . . 2545
8.2 The stage: coordinates, metric, frame and the field equation . . . 2548
8.3 Frames, and why a frame is not unique . . . 2550
8.4 The Christoffel symbols and the canonical spin connection of the diagonal frame 2551

8.5 What cancels and what survives: the term $\gamma^\mu \Omega_\mu = 3H\gamma^{(0)}$ 2553
8.6 Non-triviality: the connection vanishes only in flat space 2555
8.7 The deflation is invisible in the term, but not in the curvature 2556
8.8 The connection drops out of the Lagrangian but not out of the field equation . . 2556
8.9 Another frame: the boosted frame and its term 2557
8.10 What no frame removes: the curvature of the connection 2559
8.11 The connection in the energy-momentum tensor 2560
8.12 Example: the term in two frames, computed from the definitions 2561
8.13 How to run Notebook 08c 2561
8.14 Notebook 08c: complete text 2566
8.15 Line-by-line walk-through of Notebook 08c 2602
8.16 Well-posedness: what a good evolution equation must do 2625
8.17 Plane waves with frozen coefficients and the mode matrix 2627
8.18 Growth without bound: the Cauchy problem is not well posed 2628
8.19 The Krein form is conserved; the growing waves carry Krein form zero 2630
8.20 Example: growth rates and Hadamard ill-posedness 2630
8.21 How to run Notebook 08a 2631
8.22 Notebook 08a: complete text 2636
8.23 Line-by-line walk-through of Notebook 08a 2668
8.24 The deflating history pushes every extra-time wave into growth 2681
8.25 Following one wave through the onset: the local-frame model and WKB 2683
8.26 Example: the onset of growth along the deflating history 2685
8.27 How to run Notebook 08b 2686
8.28 Notebook 08b: complete text 2691
8.29 Line-by-line walk-through of Notebook 08b 2722
8.30 Changing the field variables: the rescaling $\Psi = \sin^{-1/2} z\, \chi$ 2734
8.31 The two ends of the hidden direction and an exact family of solutions 2735
8.32 The boundary term at the patch end and the growing modes of the good sector . 2737
8.33 Example: the rescaling and the modes of the hidden direction 2738
8.34 How to run Notebook 08d 2738
8.35 Notebook 08d: complete text 2743

8.36 Line-by-line walk-through of Notebook 08d 2774
8.37 The exact scope of non-triviality and of the evolution, in one table 2786
8.38 What we proved, what we computed, what we assumed 2788
8.39 Exercises 2790

9 The energy-momentum tensor, pressures, equations of state, conservation identities 2792

9.1 What this chapter does 2793
9.2 The stage: coordinates, metric and the field dirac16complex00 2794
9.3 Two exact facts about the spin connection of this metric 2796
9.4 What an energy-momentum tensor is 2798
9.5 The diagonal entries, derived from the definition 2799
9.6 The whole tensor: its formula, its reality and its symmetry 2800
9.7 Energy density, pressures, kinetic and potential parts, equations of state 2802
9.8 The trace, and what the field equation adds 2803
9.9 The covariant divergence of a diagonal tensor 2804
9.10 The x4 identity: energy exchange between 3-space and the extra times 2808
9.11 The x0 identity: the balance along the hidden direction 2809
9.12 Where the spin connection does enter the tensor 2810
9.13 The same tensor for dirac16complex 2811
9.14 Example: the tensor at one point and the two identities 2813
9.15 How to run Notebook 09a 2813
9.16 Notebook 09a: complete text 2818
9.17 Line-by-line walk-through of Notebook 09a 2857
9.18 Condensates: exact solutions that depend on the time only 2876
9.19 The energy density, pressure and equation of state of a condensate 2878
9.20 Where condensates oscillate and where they grow 2880
9.21 The energy of the commuting field has no lower bound 2880
9.22 Example: condensates 2882
9.23 How to run Notebook 09b 2882
9.24 Notebook 09b: complete text 2887
9.25 Line-by-line walk-through of Notebook 09b 2925

9.26 The full tensor of a condensate . . . 2939
9.27 Example: the full tensor of a condensate . . . 2945
9.28 How to run Notebook 09c . . . 2945
9.29 Notebook 09c: complete text . . . 2950
9.30 Line-by-line walk-through of Notebook 09c . . . 2989
9.31 What we proved, what we computed, what we assumed . . . 3006
9.32 Exercises . . . 3008

10 Canonical quantisation in 4+4: Krein space and the good sector 3009

10.1 What this chapter does . . . 3009
10.2 The matrix B and the Krein form . . . 3012
10.3 One plane wave: the mode Hamiltonian . . . 3014
10.4 The square of h, the Krein form, and three kinds of waves . . . 3016
10.5 Krein inertia of the eigenspaces . . . 3018
10.6 The universes of masses $+m$ and $-m$ have the same one-particle spectrum . . . 3020
10.7 Example: Notebook 10a computes the one-particle spectra and their Krein inertia 3021
10.8 How to run Notebook 10a . . . 3021
10.9 Notebook 10a: complete text . . . 3026
10.10 Line-by-line walk-through of Notebook 10a . . . 3064
10.11 Quantum states, operators and fermion modes from zero . . . 3084
10.12 The canonical rule from the Lagrangian . . . 3086
10.13 The canonical conjugate is not the Hilbert adjoint: the Krein space . . . 3089
10.14 Two realisations and the expectation-value rule . . . 3090
10.15 Maps that keep the canonical rule, and the conjugation of the quantised field . . 3091
10.16 Example: Notebook 10b builds the canonical rule on a Fock space . . . 3093
10.17 How to run Notebook 10b . . . 3093
10.18 Notebook 10b: complete text . . . 3098
10.19 Line-by-line walk-through of Notebook 10b . . . 3133
10.20 The good sector: a positive Fock space for one momentum . . . 3148
10.21 The expectation-value rule and the energy-momentum tensor operator . . . 3151
10.22 The commuting field dirac16complex00: an energy unbounded below . . . 3153
10.23 Example: Notebook 10c builds the positive Fock space of one momentum . . . 3154

10.24How to run Notebook 10c 3154
10.25Notebook 10c: complete text 3159
10.26Line-by-line walk-through of Notebook 10c 3191
10.27The curved good sector: symmetric only up to a boundary term 3204
10.28Waves that do not depend on x_0, and where their Krein charge goes 3206
10.29Example: Notebook 10d studies the curved good sector 3208
10.30How to run Notebook 10d 3208
10.31Notebook 10d: complete text 3213
10.32Line-by-line walk-through of Notebook 10d 3242
10.33No invariant charge density is positive 3252
10.34The symmetries that keep the canonical rule 3254
10.35Example: Notebook 10e computes the invariant forms and the generators 3256
10.36How to run Notebook 10e 3256
10.37Notebook 10e: complete text 3261
10.38Line-by-line walk-through of Notebook 10e 3288
10.39The Krein structure along the deflating history 3296
10.40Example: Notebook 10f follows the Krein structure along the deflating history . 3300
10.41How to run Notebook 10f 3300
10.42Notebook 10f: complete text 3305
10.43Line-by-line walk-through of Notebook 10f 3339
10.44The quantum reading of the pairing 3351
10.45What the quantum reading does not establish 3355
10.46Example: Notebook 10g checks the quantum reading of the pairing 3356
10.47How to run Notebook 10g 3356
10.48Notebook 10g: complete text 3361
10.49Line-by-line walk-through of Notebook 10g 3400
10.50What we proved, what we computed, what we assumed 3418
10.51Exercises 3424
10.52Answers to the exercises 3424

11 GKD and the Lovelock tensors **3426**

11.1 What this chapter does, and why 3426

11.2 The words and symbols of this chapter 3427
11.3 The author's generalized Kronecker delta and the rule GKD 3429
11.4 Counting the values; nine labels in eight dimensions; the cost of the determinant 3431
11.5 The author's second route: two Levi-Civita tensors of the curved metric 3433
11.6 Example: the generalized Kronecker delta in Python and in Rust 3434
11.7 How to run Notebook 11a 3435
11.8 Notebook 11a: complete text 3441
11.9 Line-by-line walk-through of Notebook 11a 3482
11.10The curvature that the Lovelock sums need 3504
11.11Lovelock's equation (4.38): the three Lovelock tensors 3507
11.12Organising the sum: only nonzero curvature, no repeated label 3508
11.13Laplace's rule and the expansion along the column of h 3510
11.14Order 1 is Einstein's tensor; the trace identity 3512
11.15Order 2 is the Gauss-Bonnet tensor; order 3 is the cubic Lovelock density 3513
11.16Example: the three Lovelock tensors computed twice 3515
11.17How to run Notebook 11b 3515
11.18Notebook 11b: complete text 3522
11.19Line-by-line walk-through of Notebook 11b 3572
11.20The components of the three tensors for the author's metric 3597
11.21Their structure: equal directions, constraint, evolution factor, mean, sign and weight 3599
11.22The two conservation identities 3602
11.23Along the deflating history; an illustrative test history 3603
11.24Example: the components and identities along the deflating history 3605
11.25How to run Notebook 11c 3605
11.26Notebook 11c: complete text 3610
11.27Line-by-line walk-through of Notebook 11c 3640
11.28What we proved, what we computed, what we assumed 3652
11.29Exercises 3655

12 The field equations for a4: Einstein and Einstein-Lovelock 3658

12.1 What this chapter does, and why 3658
12.2 The words and symbols of this chapter 3659

12.3 The author's metric, its determinant and the volume of seven directions 3661
12.4 The Christoffel symbols of the author's metric 3662
12.5 The curvature computed by hand 3663
12.6 The Ricci tensor, the Ricci scalar and the Einstein tensor by hand 3665
12.7 The Lovelock tensors: the Einstein tensor and its two companions 3667
12.8 The source and the independent field equations 3668
12.9 The constraint, the evolution equation and the algebraic condition 3669
12.10The Bianchi identity and the conservation law 3671
12.11Einstein gravity: no empty universe, and the null energy condition along x0 . . . 3673
12.12The linear member: the extra times deflate by a choice of sign 3674
12.13Example: the field equations derived by computer algebra 3677
12.14How to run Notebook 12a 3677
12.15Notebook 12a: complete text 3682
12.16Line-by-line walk-through of Notebook 12a 3728
12.17Example: the source that the linear member requires 3753
12.18How to run Notebook 12b 3753
12.19Notebook 12b: complete text 3758
12.20Line-by-line walk-through of Notebook 12b 3792
12.21Solving the evolution equation for a prescribed stress 3802
12.22Example: four prescribed histories integrated with RK4 3804
12.23How to run Notebook 12c 3805
12.24Notebook 12c: complete text 3810
12.25Line-by-line walk-through of Notebook 12c 3846
12.26A condensate of dirac16complex00 as an exact source 3859
12.27Example: three exact solutions of the coupled equations 3863
12.28How to run Notebook 12d 3863
12.29Notebook 12d: complete text 3868
12.30Line-by-line walk-through of Notebook 12d 3907
12.31What we proved, what we computed, what we assumed 3924
12.32Exercises 3926

13 Many-body quantum mechanics and DFT from zero **3928**

13.1 What this chapter is for . . . 3928
13.2 One quantum particle: states, operators and the variational principle . . . 3929
13.3 Many identical fermions and Slater determinants . . . 3931
13.4 Creation and annihilation operators and Wick's theorem . . . 3934
13.5 The energy of a determinant: direct and exchange terms . . . 3937
13.6 The Hartree and Hartree-Fock equations . . . 3938
13.7 Two electrons on two sites: exact and Hartree-Fock . . . 3941
13.8 The density decides everything: the Hohenberg-Kohn theorem . . . 3943
13.9 Functionals and functional derivatives . . . 3944
13.10The Kohn-Sham equations . . . 3945
13.11Example: determinants, operators and the two-site model . . . 3946
13.12How to run Notebook 13c . . . 3947
13.13Notebook 13c: complete text . . . 3952
13.14Line-by-line walk-through of Notebook 13c . . . 3986
13.15The uniform gas, the local density approximation and exact local exchange . . . 4005
13.16The exchange of the 16-component field dirac16complex . . . 4009
13.17Example: exchange of a contact interaction . . . 4011
13.18How to run Notebook 13b . . . 4011
13.19Notebook 13b: complete text . . . 4016
13.20Line-by-line walk-through of Notebook 13b . . . 4057
13.21Solving the Kohn-Sham equations: iteration and mixing . . . 4075
13.22Example: charge sloshing . . . 4078
13.23How to run Notebook 13d . . . 4078
13.24Notebook 13d: complete text . . . 4083
13.25Line-by-line walk-through of Notebook 13d . . . 4112
13.26Finite temperature: Mermin's theorem . . . 4122
13.27Excited states: the Kohn-Sham gap, Janak's theorem and Delta-SCF . . . 4127
13.28Example: Mermin thermodynamics and Delta-SCF . . . 4128
13.29How to run Notebook 13e . . . 4128
13.30Notebook 13e: complete text . . . 4133
13.31Line-by-line walk-through of Notebook 13e . . . 4165

13.32A complete Kohn-Sham calculation: eight fermions in a trap 4176
13.33Example: the one-dimensional Kohn-Sham toy 4179
13.34How to run Notebook 13a 4179
13.35Notebook 13a: complete text 4185
13.36Line-by-line walk-through of Notebook 13a 4230
13.37From the toy models to dirac16complex 4250
13.38What we proved, what we computed, what we assumed 4252
13.39Exercises 4254

14 The Kohn-Sham model of dirac16complex in the deflating primordial field 4256
14.1 What this chapter does 4257
14.2 The author's field and the hidden coordinate y 4259
14.3 The Kohn-Sham equation, the good sector and the stationary-slice ansatz 4261
14.4 The spin connection, the term 3H γ(x0) and the factor W^(-3) 4262
14.5 Eight blocks of two components 4265
14.6 The chirality matrix Γ on the blocks, and the rotations of 3-space 4268
14.7 The exact rescaling identity between the slices 4269
14.8 Example: Notebook 14a, the block reduction and the rescaling identity 4270
14.9 How to run Notebook 14a 4270
14.10Notebook 14a: complete text 4275
14.11Line-by-line walk-through of Notebook 14a 4314
14.12The block equation in real form and the boundary conditions 4336
14.13The exact free levels at zero 3-momentum 4340
14.14Shooting with a Pruefer angle: every level has a label 4342
14.15Example: Notebook 14b, the free spectra exact and numerical 4344
14.16How to run Notebook 14b 4345
14.17Notebook 14b: complete text 4350
14.18Line-by-line walk-through of Notebook 14b 4386
14.19The brane band and its exact slope 4405
14.20The band along the deflating history, and the tip 4407
14.21Particles, degeneracies and closed shells 4409
14.22Example: Notebook 14c, the brane band 4410

14.23How to run Notebook 14c 4411
14.24Notebook 14c: complete text 4416
14.25Line-by-line walk-through of Notebook 14c 4450
14.26The exact local exchange of the contact interaction 4467
14.27The Kohn-Sham potentials, the exact Fock exchange of the model's states, and the eight zero modes 4470
14.28Example: Notebook 14d, the exact local exchange 4473
14.29How to run Notebook 14d 4473
14.30Notebook 14d: complete text 4478
14.31Line-by-line walk-through of Notebook 14d 4508
14.32What we proved, what we computed, what we assumed 4519
14.33Exercises 4522

15 Solving the Kohn-Sham equations along the deflating history 4525

15.1 What this chapter does 4525
15.2 The equations the solver must solve 4528
15.3 Finding one level: shooting with the Pruefer angle 4532
15.4 From one level to the whole gas 4536
15.5 Self-consistency, the energy, and the first excited state 4538
15.6 Example: the solver's method by hand (Notebook 15e) 4540
15.7 How to run Notebook 15e 4541
15.8 Notebook 15e: complete text 4546
15.9 Line-by-line walk-through of Notebook 15e 4583
15.10The Rust solver and its canonical matrix 4605
15.11Example: running the canonical matrix (Notebook 15a) 4606
15.12How to run Notebook 15a 4607
15.13Notebook 15a: complete text 4613
15.14Line-by-line walk-through of Notebook 15a 4652
15.15What the canonical matrix shows 4667
15.16The energy-momentum tensor of the gas, and why the gas cannot drive the history 4668
15.17Example: the energy-momentum profiles (Notebook 15b) 4672
15.18How to run Notebook 15b 4672

15.19Notebook 15b: complete text 4678
15.20Line-by-line walk-through of Notebook 15b 4714
15.21Adiabaticity: does the gas follow the moving background? 4726
15.22Example: the adiabaticity measure (Notebook 15c) 4728
15.23How to run Notebook 15c 4728
15.24Notebook 15c: complete text 4733
15.25Line-by-line walk-through of Notebook 15c 4764
15.26The gas at a temperature 4777
15.27Example: thermodynamics (Notebook 15d) 4779
15.28How to run Notebook 15d 4780
15.29Notebook 15d: complete text 4786
15.30Line-by-line walk-through of Notebook 15d 4819
15.31What we proved, what we computed, what we assumed 4832
15.32Exercises 4834

16 Checking the solution independently: the reference solver and the cross-check 4836

16.1 What this chapter does 4837
16.2 Why a second program, and what agreement can show 4839
16.3 The two solvers 4840
16.4 Order of convergence: the ratio test 4841
16.5 Richardson extrapolation and the uncertainty of the reference 4842
16.6 The uncertainty of the Rust solver: canonical and refined runs 4844
16.7 The tolerance rule, fixed in advance 4845
16.8 What the full cross-check found 4847
16.9 Example: the reference solver on a subset and the cross-check (Notebook 16a) . . 4849
16.10How to run Notebook 16a 4849
16.11Notebook 16a: complete text 4856
16.12Line-by-line walk-through of Notebook 16a 4908
16.13Inside the reference solver I: the free problem and its exact levels 4940
16.14Inside the reference solver II: the staggered grid and the matrix 4942
16.15Inside the reference solver III: counting eigenvalues, the zero mode and the brane band 4945

16.16Example: the reference method by hand (Notebook 16b) 4948
16.17How to run Notebook 16b 4949
16.18Notebook 16b: complete text 4954
16.19Line-by-line walk-through of Notebook 16b 4993
16.20How the cross-check caught a rounding error: the chemical potential 5012
16.21Example: the Mermin root (Notebook 16c) 5017
16.22How to run Notebook 16c 5017
16.23Notebook 16c: complete text 5022
16.24Line-by-line walk-through of Notebook 16c 5062
16.25What the cross-check does not test 5081
16.26What we proved, what we computed, what we assumed 5082
16.27Exercises 5084

17 The a4 equations with the Kohn-Sham source: a prescribed background 5087
17.1 What this chapter does, and why 5088
17.2 The words and symbols of this chapter 5090
17.3 The field equations of a4 and their source 5092
17.4 Condition C1: a source must not depend on x0 5094
17.5 Condition C2: the algebraic condition 5095
17.6 Condition C3: what the linear member demands 5095
17.7 The recorded Kohn-Sham states as a candidate source 5097
17.8 Example: the three conditions tested on 75 recorded ground states 5098
17.9 How to run Notebook 17a 5099
17.10Notebook 17a: complete text 5104
17.11Line-by-line walk-through of Notebook 17a 5141
17.12Why the states fail: every source must be conserved 5164
17.13The conservation law in the coordinate y 5165
17.14For a conserved source, C2 says that p0 is flat 5168
17.15The integrated law and the averaged condition 5169
17.16The time direction: energy exchange and the propagation of the constraint . . . 5171
17.17The equations averaged over the hidden direction: what can and cannot be integrated 5173
17.18Example: the conservation laws, the averaged equations, and why the states fail 5177

17.19How to run Notebook 17b 5178
17.20Notebook 17b: complete text 5183
17.21Line-by-line walk-through of Notebook 17b 5230
17.22What a prescribed background is, and what remains open 5258
17.23What we proved, what we computed, what we assumed 5259
17.24Exercises 5262

18 The pairing theorems T1, T2 and Q **5265**

18.1 What this chapter does 5265
18.2 The words of this chapter 5267
18.3 The stage: the author's metric, its frame and its spin connection 5269
18.4 The Lagrangian, the field equation, the energy-momentum tensor and the current 5272
18.5 Four matrix lemmas 5273
18.6 Example: Notebook 18a checks the four lemmas 5278
18.7 How to run Notebook 18a 5278
18.8 Notebook 18a: complete text 5284
18.9 Line-by-line walk-through of Notebook 18a 5321
18.10Theorem T1: the chirality pairing 5341
18.11The proof of T1, line by line 5342
18.12T1 in the block form of the author's field 5343
18.13What T1 says, and what it does not say 5344
18.14Theorem T2: the mirror pairing 5345
18.15The proof of T2 in a general field 5346
18.16T2 in the author's field: the mirror across the brane 5349
18.17Example: Notebook 18b proves T1 and T2 in the author's metric 5350
18.18How to run Notebook 18b 5351
18.19Notebook 18b: complete text 5356
18.20Line-by-line walk-through of Notebook 18b 5398
18.21A family of exact solutions: the fields that depend only on the time 5422
18.22The quantum reading Q 5425
18.23The proofs of Q1 to Q5 5426
18.24Example: Notebook 18c sees the theorems in numbers 5429

18.25How to run Notebook 18c 5430
18.26Notebook 18c: complete text 5435
18.27Line-by-line walk-through of Notebook 18c 5475
18.28What the pairing theorems do not establish 5498
18.29What we proved, what we computed, what we assumed 5500
18.30Exercises 5504
18.31Answers to the exercises 5505

19 T3: the Kohn-Sham universes of mass $+M$ and $-M$ 5507

19.1 What this chapter does 5508
19.2 The words of this chapter 5511
19.3 The Kohn-Sham problem of one orbital 5511
19.4 The Kohn-Sham problem of the whole gas 5514
19.5 The chirality Γ on the blocks 5516
19.6 Step 1 of the proof: an orbital and its image 5518
19.7 Step 2 of the proof: the boundary conditions 5519
19.8 Step 3 of the proof: the densities and the mean field 5520
19.9 Step 4 of the proof: self-consistency, energies and energy-momentum 5522
19.10Theorem T3: the statement, its hypotheses and its records 5524
19.11Why T3 is not T1, and the mirror copy in the Z2 orbifold 5529
19.12The exact levels at zero momentum, and what goes wrong without the transformed tip 5530
19.13Example: the proof of T3 by hand (Notebook 19b) 5533
19.14How to run Notebook 19b 5533
19.15Notebook 19b: complete text 5538
19.16Line-by-line walk-through of Notebook 19b 5577
19.17Watching T3 in the Rust solver: four universes 5600
19.18Example: the universes of mass $+M$ and $-M$ in the Rust solver (Notebook 19a) 5602
19.19How to run Notebook 19a 5603
19.20Notebook 19a: complete text 5609
19.21Line-by-line walk-through of Notebook 19a 5663
19.22What T3 says about pairs of universes, and what it does not 5690

19.23What we proved, what we computed, what we assumed 5692
19.24Exercises 5693

20 Do universes come in pairs? What the equations prove and what they do not 5695

20.1 What this chapter does, and why 5696
20.2 The words of this chapter 5699
20.3 The request and the hypothesis, read word by word 5701
20.4 Three questions: is it allowed, does it happen, how often? 5703
20.5 What is proved (1): theorem T1, line by line 5703
20.6 What is proved (2): theorem T2, the quantum reading Q and theorem T3 5706
20.7 Corollary C1, line by line 5709
20.8 Example: corollary C1 made visible 5712
20.9 How to run Notebook 20a 5712
20.10Notebook 20a: complete text 5718
20.11Line-by-line walk-through of Notebook 20a 5758
20.12One universe without a partner: an exact solution and its two partners 5783
20.13Example: one universe and its two partners, computed 5788
20.14How to run Notebook 20b 5788
20.15Notebook 20b: complete text 5793
20.16Line-by-line walk-through of Notebook 20b 5827
20.17Allowed is not the same as happens: an electron and a positron made from light 5844
20.18Example: the threshold of pair creation, computed 5847
20.19How to run Notebook 20c 5847
20.20Notebook 20c: complete text 5852
20.21Line-by-line walk-through of Notebook 20c 5878
20.22What the conservation laws of this theory allow and forbid 5885
20.23Pairs of universes, matter and antimatter 5889
20.24What is not established 5890
20.25What a calculation of creation would require 5892
20.26The answer to the question of the title 5893
20.27What we proved, what we computed, what we assumed 5894
20.28Exercises 5897

21 Matter and antimatter from zero: what this theory explains and what it does not 5899

21.1 What this chapter does, and the answer first 5899

21.2 The words and symbols of this chapter 5901

21.3 Particles, antiparticles and conserved numbers 5902

21.4 What is observed: the baryon-to-photon ratio, and no antimatter domains 5904

21.5 Sakharov's three conditions, derived in plain words 5905

21.6 Condition 3 in a rate model: decays out of equilibrium 5907

21.7 The theory of this book: two fields, one phase symmetry 5909

21.8 Charge conjugation is a matrix: the condition on the matrix 5912

21.9 The two charge-conjugation matrices 5914

21.10 The reality conditions, the bilinears and real fields 5915

21.11 The conjugations acting on an exact solution in the author's metric 5918

21.12 Example: Notebook 21a solves for every charge-conjugation matrix 5919

21.13 How to run Notebook 21a 5919

21.14 Notebook 21a: complete text 5925

21.15 Line-by-line walk-through of Notebook 21a 5964

21.16 The phase symmetry and Noether's current 5988

21.17 Exact local charge conservation in the author's metric 5989

21.18 The charge of an exact solution: the flux through the brane, a constant charge, an indefinite density 5991

21.19 The pair-level bookkeeping: theorem T1 and the zero total charge 5994

21.20 Example: Notebook 21b follows the charge in the author's metric 5995

21.21 How to run Notebook 21b 5995

21.22 Notebook 21b: complete text 6000

21.23 Line-by-line walk-through of Notebook 21b 6040

21.24 Reflections and the chirality: P, T and Γ 6060

21.25 The quantised field: which conjugation keeps the canonical anticommutator . . . 6062

21.26 Particles and holes; the mass reversed, the spectra equal 6063

21.27 Example: Notebook 21c conjugates the quantised field 6066

21.28 How to run Notebook 21c 6066

21.29 Notebook 21c: complete text 6071

21.30Line-by-line walk-through of Notebook 21c 6104
21.31Sakharov's conditions applied to this theory 6119
21.32What a charge-violating term would look like 6121
21.33Example: Notebook 21d computes the three conditions and the scorecard 6122
21.34How to run Notebook 21d 6123
21.35Notebook 21d: complete text 6128
21.36Line-by-line walk-through of Notebook 21d 6169
21.37The scorecard, pairs of universes as a hypothesis, and what would be needed . . 6188
21.38What we proved, what we computed, what we assumed 6190
21.39Exercises 6193
21.40Answers to the exercises 6194

22 Open problems and how a student could attack them 6195

22.1 What this chapter does 6195
22.2 The words of this chapter 6197
22.3 How to work on an open problem 6199
22.4 Problem 1: the creation question 6201
22.5 Problem 2: beyond the instantaneous Kohn-Sham states 6204
22.6 The amplitude equations, line by line 6206
22.7 What the adiabaticity measure Q measures, and two exact rules 6209
22.8 The sudden limit, two histories and three estimates of the breakdown 6210
22.9 Example: when does the instantaneous picture break down (Notebook 22a) . . . 6212
22.10How to run Notebook 22a 6212
22.11Notebook 22a: complete text 6217
22.12Line-by-line walk-through of Notebook 22a 6260
22.13What Notebook 22a found, and how a student could continue 6291
22.14Problem 3: the back-reaction of the gas on a_4 6293
22.15Problem 4: the Z2 brane and its junction conditions 6296
22.16Problem 5: the dark-sector hypotheses 6298
22.17The observer's density and the equation of state, line by line 6300
22.18dirac16complex: the Kohn-Sham gas, the bulk band, the condensate and the mixtures 6303

22.19dirac16complex00: modes of both energy signs, the populations and the models M1 to M5 . . . 6307
22.20Example: the dark-sector numbers, reproduced from the record (Notebook 22b) . 6311
22.21How to run Notebook 22b . . . 6311
22.22Notebook 22b: complete text . . . 6316
22.23Line-by-line walk-through of Notebook 22b . . . 6347
22.24What Notebook 22b found, and what remains open for the dark sector . . . 6363
22.25Problem 6: matter and antimatter . . . 6365
22.26Smaller open items . . . 6366
22.27What we proved, what we computed, what we assumed . . . 6367
22.28Exercises . . . 6369

23 Reproducing everything 6373

23.1 What this chapter does . . . 6373
23.2 The words of this chapter . . . 6373
23.3 What you need, and the set-up in every new terminal . . . 6374
23.4 The order of everything . . . 6375
23.5 Stage 1: the Revision gate, step by step . . . 6375
23.6 Stage 2: every notebook of the book . . . 6378
23.7 Stages 4 and 5: assembling the book and building its PDF . . . 6379
23.8 What to do when something fails: the general rules . . . 6381
23.9 Example: the bookkeeping of the whole book (Notebook 23a) . . . 6381
23.10How to run Notebook 23a . . . 6382
23.11Notebook 23a: complete text . . . 6387
23.12Line-by-line walk-through of Notebook 23a . . . 6421
23.13The index of the checks of the Revision record . . . 6434
23.14The index of the notebooks . . . 6435
23.15What we proved, what we computed, what we assumed . . . 6435
23.16Exercises with worked answers . . . 6436
23.17Glossary . . . 6437

Abstract

This book teaches, starting from school algebra and the calculus of one variable, the complete theory of two fields in the author's primordial universe, and every computation that the Revision record of the repository has made about them. The universe has eight directions, named as the author names them: three directions of ordinary space, x_1, x_2 and x_3, which inflate; the time x_4; three extra times, x_5, x_6 and x_7, which deflate exponentially; and a hidden space direction x_0. Four of the directions are space-like and four are time-like (signature (4,4)). The fields are dirac16complex, a spinor field with sixteen complex anticommuting (Grassmann) components, and dirac16complex00, its version with sixteen commuting components.

Part I builds the mathematics and the geometry of the author's metric. Part II builds the Clifford algebra Cl(4,4), the facts about Pin(4,4) and Spin(4,4), the charge-conjugation matrices, the Lagrangians, the field equations, the energy-momentum tensors and the canonical quantisation of the two fields. Part III builds the generalized Kronecker delta, the Lovelock tensors and the field equations for the metric function a_4. Part IV introduces density functional theory from zero and solves the Kohn-Sham model of dirac16complex in the deflating field with a Rust program, checked by an independent solver; the Kohn-Sham history of a_4 is a prescribed background, not a solution of the field equations for a_4. Part V proves the pairing theorems and examines pair creation and matter and antimatter.

The book proves exact maps between the solutions of mass $+m$ and those of mass $-m$ (the pairing theorems T1, T2, Q and T3). It does not prove that the big bang creates universes, in pairs or otherwise: no creation process, rate or amplitude follows from these equations. It proves that the charge of the theory obeys an exact local conservation law (inside the patch $0 < z < \pi/2$ of the hidden direction charge can only move: no process creates or destroys it at any point there), shows that the total charge of a universe is constant only if no charge flows through the edge $z = \pi/2$ of the hidden direction (a condition that is assumed, not derived), proves that a pair of partner solutions has total charge zero, and shows that the theory as built does not explain the excess of matter over antimatter. Every statement carries one of the labels PROVED, COMPUTED, ASSUMED, HYPOTHESIS or OPEN. Every worked example is a complete Jupyter notebook, printed in full together with complete instructions for running it on Windows, macOS and Linux, its figures and a line-by-line walk-through; every number in the text is computed by a notebook or taken from a Revision record.

0 How to use this book; installing the software; the honesty ledger

This chapter is the map of the book. It says what the book teaches and, just as important, what the equations of the book prove and what they do not prove. It explains how every statement is labelled, so that you always know whether it is proved, computed, assumed, a hypothesis or an open question. It installs the software on Windows, macOS or Linux, explains how the notebooks of the book are built and checked, and ends with the honesty ledger: the table of the main statements of the book with their labels and the files that verify them. The chapter has four worked examples, each a complete notebook: Notebook 00a checks that your computer is ready; Notebook 00b takes a first look, with numbers and pictures, at the eight directions of the author's universe and at the author's metric; Notebook 00c reads the verifier reports of the Revision record and checks the honesty ledger; Notebook 00d shows why every run of a notebook gives exactly the same bytes.

0.1 What this book teaches

The book is about a model of the very early universe proposed by the author. The universe of the model has eight directions instead of the four of everyday space and time. They are named x_0 to x_7,

as the author names them. The first, x_0, is a **hidden** direction of space. Three of them, x_1, x_2 and x_3, are the directions of ordinary space. One, x_4, is the time in which everything evolves. The last three, x_5, x_6 and x_7, behave like time as well; they are called the **extra times**. The author's **metric** (the rule that gives the length of a small step in each direction; Section 0.14 introduces it from zero) makes ordinary space grow (**inflate**) and the three extra times shrink (**deflate**) exponentially as the time x_4 runs.

Two fields live in this universe. A **field** is a rule that attaches numbers to every point of space and time: the temperature in a room attaches one number to every point, the wind three. The field **dirac16complex** attaches sixteen complex numbers of a special kind (anticommuting numbers, defined in Chapter 7) to every point; the field **dirac16complex00** attaches sixteen ordinary complex numbers. The book sets up the mathematics of both fields from zero, derives their equations and the equations of the gravitational field that they produce, introduces the density-functional (DFT) approximation, and solves the equations as far as present knowledge allows. Then it asks two questions and answers them honestly: do the equations prove that the big bang creates universes in pairs, and does the theory solve the puzzle of matter and antimatter? Section 0.2 gives the short answers; Part V of the book gives the complete proofs of what is proved and a precise list of what is not.

Every formula and every number of the book comes from the **Revision record**, the computations stored in the folder `Revision` of the repository Dirac_claude, or from the book's own notebooks, which reproduce the Revision record wherever the two overlap. (A **repository** is a folder of files whose history is kept by the program Git; Section 0.8 installs it.)

The book is written for a student who knows school algebra and the calculus of one variable (derivatives and integrals of functions of one variable), and nothing else. It is written in a way that we call a **deep dive**:

- every notion is defined, in plain words, before it is used;
- every derivation is written out line by line, and each line is followed by the rule that produced it from the line before; no step is left to "it can be shown";
- every worked example is a complete Jupyter notebook, a file of text and Python code that you run on your own computer, and the book prints it in full;
- just before each notebook the book prints the complete instructions for running it, on Windows, macOS and Linux, without sending you anywhere else;
- just after each notebook the book explains every line of its code in a section "Line-by-line walk-through";
- every number in the text names the notebook cell that computes it and, where it reproduces the Revision record, the record file and the check;
- every chapter ends with the section "What we proved, what we computed, what we assumed" and with at least five exercises with complete worked answers.

Read the chapters in order. Run each notebook when you reach it: reading a computation and doing it are different things, and the notebook lets you change a number and see what happens. Do each exercise before you read its answer.

0.2 The request, and what the equations prove and do not prove

The book answers a request of the author (2026-10-02) for a teaching book that describes every step of the formalism, derives the field equations, introduces and derives approximations of the DFT type, solves the equations as far as possible, and proves "that the 'big bang' creates universes in pairs" and that "this theory solves matter anti-matter mysteries". The first four parts of the request are carried out in Parts I to IV of the book. The last two cannot be delivered as they are worded, because the equations do not prove them, and a book that wrote them as established would teach something false. The book therefore follows one rule above every other, the **honesty rule**: it teaches exactly what the Revision record proves and computes, with every assumption, and it never writes "proved" for a statement that is not proved. The author was told this on 2026-10-02. Here is what the equations give, in plain words. Many words of this section are new. Each is defined from zero in a later chapter or a later section of this chapter; the list below gives each one a one-line meaning and names that place, so that you can follow the story now.

- **gravitational field**: in this book, the author's metric of Section 0.1, the rule for the lengths of small steps; since Einstein, gravity is described by such a rule (Chapter 3);
- a_4, H **and** z: a_4 is the function of the time x_4 in the author's metric that makes ordinary space inflate and the extra times deflate; H is a positive constant of the author; $z = 6Hx_0$ measures the position along the hidden direction, and it lies between $z = 0$ and the edge $z = \pi/2$ (Section 0.14);
- **space-like** and **time-like**: a direction along which a step is measured like a distance, or like a duration; x_0, x_1, x_2 and x_3 are space-like, x_4 to x_7 time-like (Section 0.14);
- **solution**, **boundary condition**: a solution of a field's equations is a field that satisfies them at every point; a boundary condition is a further condition that a solution must satisfy at the edge of the region in which it is defined (Chapters 2 and 14);
- Ψ (the Greek capital letter psi): a field, its sixteen components written as one column of sixteen numbers (Chapter 7);
- **commuting** and **complex conjugation**: ordinary numbers are commuting, the order of a product does not matter ($ab = ba$), while the anticommuting numbers of Chapter 7 have $ab = -ba$; complex conjugation changes the sign of the imaginary part of every complex number, $a + ib \to a - ib$ (Chapter 1);
- **mass** m: a number in the equations of a field; for a field that describes particles it is their mass (Chapter 7);
- **self-coupling** λ: the strength with which a field acts on itself (Chapter 7);
- **Lagrangian**: the function from which the equations of a field follow (Chapter 7);
- **classical field**: a field whose values are numbers, as everywhere in this chapter, and not the operators of quantum theory (Chapter 7);
- **energy-momentum tensor** T: the table of the energy and the momentum that a field carries and of the pressures it exerts (Chapter 9);

- **source**: the energy-momentum tensor of the fields as it enters the equations of gravity (Chapter 12);
- **Einstein's equations**: the field equations of gravity, which say how the source shapes the metric (Chapter 12);
- **charge** Q and **charge current** J: a number carried by a field whose density obeys an exact **local conservation law**, so that the charge of a region changes only by what flows through the boundary of the region (the law is proved at every point with $0 < z < \pi/2$; at the edge $z = \pi/2$ the **no-flux condition**, that no charge flows through the edge, is ASSUMED, not derived, and only under it is the total charge of a universe constant), and the flow of this number through space (Chapters 10, 18 and 21);
- **U(1) symmetry**: the multiplication of every component of a field by one and the same complex number of absolute value 1, which changes no equation; it is the reason why the charge obeys its local conservation law: at no point with $0 < z < \pi/2$ is charge created or destroyed, it can only flow from one place to another (Chapter 21);
- **gamma matrices**: eight fixed 16×16 matrices of the author, one for each direction, from which the equations of both fields are built (Chapter 4);
- **the matrices** C, Γ **and** B: three fixed 16×16 matrices built from gamma matrices: C is the product of the four space-like ones, Γ (the **chirality**) the product of all eight, and B the product of C and the gamma matrix of the time x_4, times $-i$; B enters the quantum theory (Chapter 5);
- **group, Pin(4,4)**: a group is a collection of transformations that can be combined and undone; Pin(4,4) is the group of the transformations of the sixteen components of a field that go with the rotations and reflections of the eight directions, rotations in a wide sense that includes the mixing of space-like and time-like directions (Chapter 5);
- **Z2 mirror**: a reflection across the edge $z = \pi/2$ of the hidden direction, imposed as a boundary condition; Z2 is the group of two elements, doing nothing and reflecting (Chapter 14);
- **quantised**: turned into a quantum theory, in which the field becomes an operator, a rule that turns one state of the system into another (Chapter 10);
- **indefinite (Krein) metric**: a rule for the squared length of a quantum state that can also give negative numbers (Chapter 10);
- **probability amplitude**: the number from which quantum theory computes how likely a process is (Chapter 10);
- **Kohn-Sham universe**: a solution of the equations of the density-functional approximation, in which the field moves in a potential computed from its own density; **self-consistent** means that this potential and this density fit each other; the mass of the field in this model is written M (Chapters 13 to 15);
- **antimatter**: matter made of antiparticles; the antiparticle of a particle has the same mass and the opposite charge, as the positron has for the electron (Chapter 21);

- **the symmetries C and CP**: C exchanges every particle with its antiparticle, P reflects space as in a mirror, and CP does both (Chapter 21);
- **thermal equilibrium**: a state in which every process runs forwards as often as backwards, so that on average nothing changes (Chapter 21);
- **photon**: the particle of light (Chapter 21);
- **discrete symmetry**: a symmetry that is a single operation, such as a reflection, and not a continuous family of operations such as the multiplications of the U(1) symmetry (Chapter 21).

Pairs of universes. Each field has a **mass** m (a number that enters its equations) and a **self-coupling** λ (the strength with which the field acts on itself). The Revision record proves exact maps between the solutions of the equations with mass $+m$ and those with mass $-m$:

- **T1** (both fields, every gravitational field): a fixed 16×16 matrix Γ, the **chirality**, turns every solution with (m, λ) into a solution with $(-m, -\lambda)$; the Lagrangian (the function from which the equations follow) changes as $L_{m,\lambda}[\Gamma\Psi] = -L_{-m,-\lambda}[\Psi]$, the energy-momentum tensor T (the table of the energy and momentum that the field carries) changes into $-T$, and the charge current J into $-J$;
- **T2** (both fields): Γ combined with the reflection of one space-like direction (an element of the group Pin(4,4)) turns (m, λ) into $(-m, \lambda)$, with the same self-coupling; in the author's universe the reflection is the Z2 mirror across the edge $z = \pi/2$ of the hidden direction (a choice of boundary condition, which is ASSUMED, not derived), and the energy-momentum tensor of the image is equal to that of the original (reflected across the mirror), not opposite to it;
- **Q** (dirac16complex): the quantum reading of T1; the image field carries the indefinite (Krein) metric $-B$ and is the same quantum system written in other variables, and there is no cancellation between two universes that are quantised independently of each other;
- **T3** (dirac16complex, the Kohn-Sham level: within the Kohn-Sham approximation, which is ASSUMED (Section 0.18), with the ASSUMED Z2 mirror and the correspondingly transformed boundary conditions): every self-consistent Kohn-Sham universe with (M, λ) has a partner with $(-M, \lambda)$, and the two have equal energies and equal energy-momentum tensors;
- **C1** (a consequence of T1): a T1 pair taken as the complete classical source of the author's metric is a zero source, and Einstein's equations then have no solution for $H > 0$.

These are PROVED, each by two independent verifiers, whose reports are:

- for T1, T2 and Q: `Revision/pairing/reports/wolfram-pairing.json` and `Revision/pairing/reports/python-pairing.json`;
- for T3: `wolfram-t3.json` and `python-t3.json`, with its completion `wolfram-t3-completion.json` and `python-t3-completion.json`, all four in the folder `Revision/pairing/kohn_sham/reports` (the completion closes gaps that a review found in the first proof:

among them, its Wolfram verifier reads the numbers that define the Kohn-Sham potentials from the Kohn-Sham record instead of writing them into the program, and it shows that the rule by which the states are filled carries over to the partner);

- for C1: T1 together with the check `einstein_no_vacuum_solution` of `Revision/field_equations_a4/reports/wolfram-a4-report.json` and the check `einstein_no_vacuum` of `Revision/field_equations_a4/reports/python-a4-report.json`.

They are exact **maps between solution sets**: if a configuration of the field is a solution of the equations, its image under the map is a solution of the equations with the mapped parameters (for T1, of the equations with $(-m, -\lambda)$ in place of (m, λ)). What is NOT proved is that any universe is **created**, in pairs or otherwise: no creation process, no rate, no probability amplitude and no dynamics of a big bang follows from these equations, and a single universe of mass $+m$ is an equally valid solution without its partner. The pairing record says so itself: the report `Revision/pairing/reports/python-pairing.json` holds, under its key `not_established`, a list of twelve things that the theorems do not establish, and Notebook 00c prints all twelve. Chapters 18 to 20 give the complete proofs and the complete list. In the honesty ledger (Section 0.18) the statement "the big bang creates universes in pairs" is therefore labelled OPEN, with the note "not proved".

Matter and antimatter. The universe we observe contains matter but almost no antimatter. In 1967 Sakharov showed that an excess of matter can grow from an equal start only if three conditions hold: there are processes that change the number of baryons (the particles of ordinary matter, such as the proton); the symmetries called C and CP are violated; and the universe departs from thermal equilibrium. Chapter 21 explains all of this from zero, with the measured size of the excess (the baryon-to-photon ratio). What this theory proves is: the charge of each field obeys an exact **local** conservation law (it follows from an exact U(1) symmetry), so no process creates or destroys charge at any point inside a universe off its edge (at every point with $0 < z < \pi/2$, where the law is proved): charge can only flow from one place to another; the total charge Q of a universe is constant only if no charge flows out through the edge $z = \pi/2$ of the hidden direction, and this **no-flux condition** is ASSUMED, not derived (whether a boundary condition at the edge supplies it is OPEN; Chapters 10, 18 and 21 show exact solutions through whose edge charge does flow, so that their total charge changes); the discrete symmetries of the theory are derived exactly (Chapter 21), among them its two charge conjugations, which are matrices, $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$; and a T1 partner (mass $-m$, self-coupling $-\lambda$) carries the opposite charge, so a T1 pair $\{+m, -m\}$ has total charge zero as classical fields (for two universes that are quantised independently of each other there is no such cancellation; this is part of Q). The idea that our universe has a partner of opposite charge (an "anti-universe") belongs to a class of ideas of which one published example is Boyle, Finn and Turok, Phys. Rev. Lett. 121, 251301 (2018). The theory as built does NOT solve the matter-antimatter problem: it has no baryons, no process that changes the number of baryons, no violation of CP, and no computation of a departure from equilibrium. To solve it, the theory would need all of these, and a computation of the excess that agrees with the measured one. That our universe actually has such a partner is a HYPOTHESIS, and every scenario built on it is labelled HYPOTHESIS.

Charge conjugation is a matrix. The author's gamma matrices are real (they contain only real numbers). For a field whose components are real numbers, plain complex conjugation changes nothing at all: it is the identity map, and it cannot be what charge conjugation means here. The charge conjugations of the theory are two matrices, derived in Chapter 5 and Chapter 21: $\mathcal{C}_+ = C$, which keeps the mass, and $\mathcal{C}_- = \Gamma C$, which reverses it. For a real commuting field the charge current J is zero and $\mathcal{C}_+$ acts as the identity (a real field is its own conjugate); for real fields

the map between matter and antimatter is the real matrix Γ together with the reversal of the mass, which is T1. (PROVED: `Revision/lead_checks/reports/charge-conjugation-and-u1.json`, the checks `charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus` and `real_fields_charge_conjugation`; Notebook 00c opens this report, prints the details of these three checks and checks that they state exactly this: Section 0.21, In [3].)

A prescribed background. The Kohn-Sham chapters let a_4 grow in proportion to the time, $a_4 = AHx_4$. This history is ASSUMED, a **prescribed background**: the computed Kohn-Sham states cannot be its source in the field equations (`Revision/field_equations_a4/reports/ks-source-conditions.json`).

0.3 The five labels of every statement

Every statement of the book that matters carries one of five labels, so that you always know how much it is worth:

label	meaning
PROVED	exact; the book gives the complete proof and, where the Revision record contains an exact computer check of the statement, names it (the report file and the name of the check)
COMPUTED	a number from a numerical computation, with its measured uncertainty and the file that holds it
ASSUMED	a starting point that the book does not derive (a convention, a physical input or an approximation); everything that depends on it says so
HYPOTHESIS	an idea that is stated and examined but not established
OPEN	a question that neither the Revision record nor this book answers

A statement can be PROVED in three ways. (1) A statement about finitely many definite objects, such as "these eight matrices of whole numbers satisfy these 64 equations", is proved by an exact computation of every case. (2) A statement in which the computer keeps the unknown quantities as symbols, such as "for every function a_4 the determinant of the metric is $\cos^2 z$", is proved by an exact symbolic computation, which holds for every value of the symbols, like a derivation by hand. (3) A statement about infinitely many cases that the computer cannot hold as symbols, such as "in every gravitational field", is proved by a general derivation in the book; an exact check at a few test points then confirms the derivation (and would catch many errors), but it is not the proof.

Examples from this chapter. "The determinant of the author's metric is $\cos^2 z$" is PROVED (way 2; Section 0.14). "The ground-state energies of the canonical and the refined run of the Revision record's Kohn-Sham solver differ by at most 1.901×10^{-12} (relative)" is COMPUTED (`Revision/kohn_sham/reports/ks-rust-determinism.json`, check `refined_ground_energies`; Notebook 00d reads it, Section 0.22). "The Z2 mirror at the edge of the hidden direction" is ASSUMED. "Our universe has a partner of opposite charge" is a HYPOTHESIS. "The big bang creates universes in

pairs" was proposed by the author as a hypothesis; as a question of this book it is OPEN, and the ledger says why: it is not proved.

0.4 How to check a statement yourself

You can check a statement at three levels.

Level 1, the derivation. Follow it line by line in the book. If a line does not follow from the line before it, you have found either a gap in your understanding or an error in the book; both are worth finding.

Level 2, the report. Every exact verifier of the Revision record writes its results into a **report**, a JSON file (a plain text file of names, numbers and lists that Python reads directly). A report lists its **checks**; each check has a name, a **verdict** (PASS or FAIL) and a detail text that says exactly what was verified. You can read a report yourself. Open a terminal, activate the Python environment of the book and go into the repository folder (Section 0.11, Step 4, gives the commands), type `python` and press Enter. Python shows its prompt, three greater-than signs, at the start of each input line. Type the text after each prompt (the lines that start with three dots are the second line of the loop; press Enter once more on the empty line after them):

```
>>> import json
>>> path = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
>>> report = json.load(open(path, encoding="utf-8"))
>>> report["summary"]["passed"], report["summary"]["total"]
(12, 12)
>>> for check in report["checks"][:3]:
...     print(check["verdict"], check["name"])
...
PASS representation_real
PASS B_imaginary_hermitian
PASS intertwiners_same_mass
>>> exit()
```

The answer `(12, 12)` says that 12 of the 12 checks of this report passed; the loop prints the verdict and the name of its first three checks. Notebook 00c does exactly this, for every report of the Revision record at once.

Level 3, the computation. Run the verifier again and compare its new report with the stored one. The verifiers need Wolfram Mathematica (which the notebooks of this book never need) or Rust, and some run for hours; Chapter 23 lists every command.

0.5 Conventions used everywhere

These conventions hold on every page. They are choices, not results (ASSUMED).

- **The names of the coordinates** are the author's: $x_0, x_1, \ldots, x_7$, always in this order. In Python lists and arrays the entries are counted from 0, so the entry of x_k has the index k: x_0 has the index 0 and x_7 the index 7. The book uses no other names or order of the coordinates.

- **The roles of the coordinates.** x_0: the hidden direction of space, through the variable $z = 6Hx_0$ with $0 < z < \pi/2$, where $H > 0$ is a constant of the author; x_1, x_2, x_3: ordinary space, which inflates; x_4: the time; x_5, x_6, x_7: the three extra times, which deflate exponentially.
- **The signs of the directions** are $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \ldots, x_7$: the directions x_0, x_1, x_2, x_3 are space-like and x_4, x_5, x_6, x_7 time-like; the **signature** is (4,4). (Section 0.14 explains these words; the Revision record states the signs in the file `Revision/algebra/gammas.json`, and its Wolfram verifier checks them: `Revision/algebra/reports/wolfram-algebra.json`, check `eta_in_author_order`.)
- **Units.** The formulas of the Revision record contain no speed of light and no Planck constant: they are written in units in which both are 1, as is usual in this kind of physics. A chapter that computes numbers says in which unit it measures them (for example in units of the mass m).
- **Files** are named by their path in the repository, written with `/` and starting at the repository folder, for example `Revision/README.md`; the book prints such names in typewriter type.

0.6 The map of the book

The book has five parts and an appendix. Each chapter uses only what earlier chapters have built.

Part I, getting ready.

- Chapter 0 (this chapter): how to use the book, the software, how the notebooks are built, the honesty ledger.
- Chapter 1: mathematics from zero, part I: numbers, complex numbers, vectors, matrices, index notation, determinants, the generalized Kronecker delta.
- Chapter 2: mathematics from zero, part II: functions of several variables, partial derivatives, differential equations and how a computer solves them, the shooting method.
- Chapter 3: the geometry of space and time: the metric, the signature (4,4), the author's metric, curvature.

Part II, spinors and fields.

- Chapter 4: the Clifford algebra and the author's sixteen gamma matrices T16.
- Chapter 5: the matrices C, Γ and B, the groups Pin(4,4) and Spin(4,4), and the charge-conjugation matrices.
- Chapter 6: spinors in the curved 4+4 space: the vielbein, the spin connection, the covariant derivative.
- Chapter 7: the two fields and their Lagrangians; Grassmann numbers; the field equations.
- Chapter 8: what non-triviality means exactly, and the growth of the extra-time modes.

- Chapter 9: the energy-momentum tensor: energy density, pressures, equations of state, conservation.
- Chapter 10: canonical quantisation in 4+4 dimensions and the Krein space.

Part III, the field equations of gravity.

- Chapter 11: the generalized Kronecker delta and the Lovelock tensors.
- Chapter 12: the field equations for a_4.

Part IV, density functional theory.

- Chapter 13: many-body quantum mechanics and density functional theory from zero.
- Chapter 14: the Kohn-Sham model of dirac16complex in the deflating field.
- Chapter 15: solving the Kohn-Sham equations with the Rust solver.
- Chapter 16: checking the solution with an independent solver.
- Chapter 17: why the Kohn-Sham history of a_4 is a prescribed background.

Part V, pairs of universes, matter and antimatter.

- Chapter 18: the pairing theorems T1, T2 and Q with complete proofs.
- Chapter 19: T3, the Kohn-Sham universes of mass $+M$ and $-M$.
- Chapter 20: do universes come in pairs? What the equations prove, and what they do not.
- Chapter 21: matter and antimatter from zero, and what this theory does and does not contribute.
- Chapter 22: open problems, and how a student could attack each.

Appendix.

- Chapter 23: reproducing everything; the glossary; the index of the checks and of the notebooks.

0.7 Every example is a notebook

Every worked example of the book is a **Jupyter notebook**: a file (with the ending `.ipynb`) that holds text and Python code in **cells**, together with everything the code printed and drew when it ran. A **markdown cell** holds text; a **code cell** holds Python code. The **kernel** is the running Python program that executes the code cells one after the other. The notebooks are numbered by chapter and letter: Notebook 00a is the first notebook of Chapter 0, Notebook 00b the second, and so on. The book prints every notebook in full, in three parts:

- a section "How to run Notebook NNx" (NN is the number of the chapter and x the letter), which gives the complete instructions for running the notebook on Windows, macOS and Linux;
- a section "Notebook NNx: complete text", which prints every cell in order, with what each code cell printed and every figure it drew;
- a section "Line-by-line walk-through of Notebook NNx", which explains every line of every code cell.

Every notebook has the same numbered sections: (1) what it computes; (2) how to run it, the same complete instructions again; (3) the words it uses; (4) the physical and mathematical situation; then the computation in small steps, each code cell preceded by a text cell that says what it does; and last, what it showed. The first code cell, the **set-up cell**, repeats the instructions as comment lines and prepares the helpers that every notebook uses.

Every computation is followed by **checks**: statements that must be true. When a check is true, the notebook prints a line that starts with PASS and names the check. When a check reproduces a number or a verdict of the Revision record, a second line starting with "reproduces" names the record file and the check in it. When a check is false, the notebook stops with an error that names the check. Lines that start with RESULT print the key numbers. The last line of every notebook says how many checks passed, for example ALL 9 CHECKS PASSED (notebook 00a).

Every figure is saved as a PNG picture file in the folder `Revision/textbook/figures`, named after the notebook, its number and a short name (for example `00a_1_parabola_tangent.png`), and the book prints it with a caption that says what is plotted, what the axes and their units are, and what you should see. The book refers to it as Figure 00a.1.

Next to every notebook lies its **provenance file**, with the same name and the ending `.PROVENANCE.md` (for example `Revision/textbook/notebooks/00a_check_installation.PROVENANCE.md`). It records what the notebook computes and from which Revision records; the complete instructions for running it; the expected output (every PASS line, every RESULT line, every figure with its caption); every file it writes; its run time and peak memory; the computer and the versions of the programs with which it was run; the sha256 fingerprints of the notebook and of every file it writes (Section 0.18 explains fingerprints); and the date and result of its last check.

0.8 Installing the software

The complete installation commands are printed in the section "How to run" before every notebook, for the first time in Section 0.11; you follow them once per computer, and later notebooks need only the steps that start JupyterLab. This section explains what each piece of software is and why it is needed, and adds the installation of Rust, which only seven notebooks need.

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). A command is typed exactly as printed and sent with the Enter key. The commands are printed in framed blocks, one command per line; where they differ between the systems, the block is labelled with the system.

Git is the program that downloads the repository and keeps track of the versions of its files. On

Windows it is installed from https://git-scm.com/download/win; on macOS it comes with the developer tools (the command `xcode-select` with the option install, printed in the instructions); on Linux it is a package of the system.

Python is the programming language of the notebooks. The book needs Python 3.12 or newer; it was built with Python 3.14.5. A **version number** such as 3.14.5 names one release of a program: the first number changes rarely, the second for new features, the third for corrections.

The repository is downloaded with one command,

```
git clone https://github.com/once-ere/Dirac_claude.git
```

which creates the folder Dirac_claude (a few hundred megabytes; the folder then needs up to about 1 GB of disk space). To get a newer version later, run `git pull` inside that folder.

A private Python environment is a folder with its own copy of Python and of the packages, so that installing the packages changes nothing else on your computer. The instructions create it in the folder dirac-book-env inside your home folder and **activate** it: from then on, in this terminal, the commands `python` and `jupyter` are those of the environment, and the prompt starts with (dirac-book-env). An environment must be activated again in every new terminal. On Windows, PowerShell refuses to run the activation script until the command with `Set-ExecutionPolicy` allows it for the one window in which it is typed; it changes nothing permanently.

The nine packages are installed with Python's installer pip at fixed (**pinned**) versions, the versions with which the book was built: numpy 2.4.6 (arrays of numbers, matrices), sympy 1.14.0 (exact algebra and calculus with symbols), mpmath 1.3.0 (numbers with as many digits as we ask for), matplotlib 3.11.0 (plots), and jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6 (the programs that show, store and run notebooks). Another version of a package may print slightly different last digits or draw slightly different pictures; Notebook 00a checks that every package has its pinned version. The pins of these nine packages and of the 93 packages they need in turn are listed in the file `Revision/textbook/requirements.txt`; with the environment active, the command below, run in the repository folder, installs all of them at once at the exact versions of the book (the commands in the run instructions install the nine and let pip choose the others):

```
python -m pip install -r Revision/textbook/requirements.txt
```

JupyterLab shows a notebook in your web browser and runs it: with the environment active, in the folder `Revision/textbook/notebooks`, the command `jupyter lab` followed by the name of the notebook file opens it, and the menu Run > Run All Cells runs every cell from the top. A notebook can also be run without a browser (**headless**) with `jupyter nbconvert`; the run instructions print the exact command.

Rust is a second programming language. The longest computations of the Revision record (the generalized Kronecker delta and the Lovelock tensors, and the Kohn-Sham solver) are Rust programs, and seven notebooks run them: 11a and 11b, 15a, 15b and 15d, 16a and 19a. Their run instructions include the following installation, which you do once per computer. **cargo** is Rust's program that turns the Rust code of a program into a program that the computer can run (it **builds** it). On Windows, download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices;

if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). On macOS and Linux, run the command below and accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then open a new terminal and check the version of cargo; it must print 1.91.1 or a newer version. The two Rust programs are built with these commands, in the repository folder (the same commands on all three systems):

```
cargo --version
cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

A build writes the folder `target` next to the file `Cargo.toml` of its program (git ignores it). The two programs use no package from the internet, so cargo downloads nothing. A notebook that needs a program runs the same build command itself before it uses it.

Removing everything again is simple: delete the folders dirac-book-env and Dirac_claude in your home folder; apart from the programs Git, Python and Rust that you installed, nothing else was changed. Those programs are removed like any other program of your system (a Python that you built from its source code on Debian 12, as a troubleshooting entry of Section 0.11 describes, lies in the folder `/usr/local`); Rust removes itself with the command `rustup self uninstall`.

0.9 How the notebooks are built and checked

You do not need this section to run the notebooks; it explains why you can trust that the printed notebooks are exactly what the code produces.

No notebook of the book is edited by hand. Each one is made by a **builder**, a Python file in the folder `Revision/textbook/notebooks/src` with the same name as the notebook (for example `Revision/textbook/notebooks/src/00a_check_installation.py`), which lists the notebook's cells and its facts: its title, the Revision records it reads, the packages it needs, the files it writes, its expected run time and its last printed lines. The book's tool **nbkit** (the file `Revision/textbook/tools/nbkit.py`) turns a builder into a notebook in two steps.

Build. nbkit inserts the run instructions and the set-up cell, executes the notebook with the kernel python3 in the folder `Revision/textbook/notebooks`, with the hash seed `PYTHONHASHSEED` set to 0 (Section 0.22 explains what it is), checks every output against the rules of the book, removes everything that differs from run to run without meaning anything (time stamps and timing data), and stores the executed notebook, its figures and its captions. A build ends with the check described next, and only when that check passes does it write the provenance file.

Check. nbkit executes the notebook a second time, now with the hash seed 1 and with every file written into a scratch folder instead of the repository (the environment variable `TEXTBOOK_OUTPUT_ROOT` of the set-up cell names that folder), and compares everything byte for byte: the notebook,

every file it wrote, and the provenance file regenerated from its record. Only when everything is identical does the check print PASSED; the result and its date are stored in the provenance file. The rules that a build and a check enforce include: no error and nothing printed on the error channel; every printed line at most 89 characters of plain ASCII text (the width of a page of the book); no memory address and no folder of the build computer in the output; every figure saved at 150 dots per inch, without the name of the program that made it, and at most 1.25 times as high as it is wide; and exactly the files that the builder lists are written.

From the repository folder, with the environment active, the check of one notebook is the command

```
python Revision/textbook/notebooks/src/00a_check_installation.py check
```

(near its end it prints the line `check_00a=PASSED`, and its last line is `nbkit=OK`; the scratch folder, into which the second run writes its files, is a folder in the system's temporary folder). Because the comparison is byte for byte, it is meant for the computer on which the book was built: on another computer a figure may differ in a few bytes (for example through another version of a font library), which is harmless for the student but fails a byte-for-byte comparison. Notebook 00d shows, with small experiments, what else could make two runs differ, and how the book prevents it.

0.10 Example: checking the installation

The first notebook computes no physics. It checks that Python and the packages are installed at the right versions, prints the version of cargo when Rust is installed, does one small computation with each of numpy, sympy and mpmath, and draws two teaching plots. Run it first: if it ends with the line ALL 9 CHECKS PASSED (notebook 00a), every notebook of the book that needs no Rust can run on your computer.

The two plots illustrate two facts that the book uses again and again. We derive both here, line by line.

The tangent line of a parabola. The parabola is $y = x^2$, and we want the straight line that touches it at the point $(1, 1)$. The slope of the parabola at x is the derivative

$$\frac{d}{dx} x^2 = 2x$$

(the power rule $\frac{d}{dx} x^n = n x^{n-1}$ with $n = 2$). At $x = 1$ the slope is $2 \cdot 1 = 2$ (substitution of $x = 1$). The line through the point $(1, 1)$ with slope 2 is

$$y = 1 + 2(x - 1)$$

(the point-slope form $y = y_0 + s(x - x_0)$ of a straight line with $x_0 = 1$, $y_0 = 1$, $s = 2$), that is

$$y = 2x - 1$$

(multiply out the bracket, $2(x - 1) = 2x - 2$, and add $1 - 2 = -1$). The gap between the parabola and the line is

$$x^2 - (2x - 1) = x^2 - 2x + 1$$

(remove the bracket: a minus sign in front of a bracket changes the sign of every term in it), and

$$x^2 - 2x + 1 = (x - 1)^2$$

(the binomial formula $(a - b)^2 = a^2 - 2ab + b^2$ with $a = x$, $b = 1$). A square is never negative and is zero only when the number squared is zero, so the gap is never negative and is zero only at $x = 1$: the line touches the parabola at the single point $(1, 1)$ and lies below it everywhere else. Status: PROVED (above, by school algebra); Notebook 00a confirms it at 401 points (COMPUTED, Figure 00a.1).

A growing and a shrinking exponential. For every number t,

$$e^t \, e^{-t} = e^{t+(-t)}$$

(the law of exponents $e^u e^v = e^{u+v}$), and

$$e^{t+(-t)} = e^0 = 1$$

(the exponent is $t - t = 0$, and $e^0 = 1$ because any nonzero number to the power 0 is 1). So $e^{-t} = 1/e^t$: when e^t grows, e^{-t} shrinks by exactly the same factor. The author's metric has this pattern (Section 0.14): lengths along ordinary space carry the factor e^{a_4} and lengths along the extra times the factor e^{-a_4}, so when a_4 grows, ordinary space inflates while the extra times deflate. Status: PROVED; Notebook 00a confirms it at 401 points (COMPUTED, Figure 00a.2).

0.11 How to run Notebook 00a

Step 1. What this notebook does and what it needs.

Notebook 00a (Checking the installation) is the file `Revision/textbook/notebooks/00a_check_installation.ipynb` of the repository Dirac_claude. It prints the version of Python, of every Python package of the notebooks and of cargo (when cargo is installed), checks that every package has its pinned version, does one small computation with each of numpy, sympy and mpmath, and draws two teaching plots (a parabola with its tangent line, and a growing and a shrinking exponential). It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs

the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is

changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00a_check_installation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00a_check_installation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 00a_check_installation.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/00a.captions.json`

- `Revision/textbook/figures/00a_1_parabola_tangent.png`
- `Revision/textbook/figures/00a_2_growth_and_decay.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 00a_2_growth_and_decay.png exists
ALL 9 CHECKS PASSED (notebook 00a)
```

and the notebook must show 2 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/00a_check_installation.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 00a_check_installation.ipynb
python -m nbconvert --execute --inplace 00a_check_installation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "AssertionError: check failed: every package has its pinned version": the table printed just above the error names each package with the installed and the pinned version; install the pinned versions again with the pip commands of Step 3 (with the environment active) and run the notebook again.
- "AssertionError: check failed: Python is version 3.12 or newer": the environment was made with an older Python; delete the folder dirac-book-env in your home folder and repeat Step 3 with Python 3.12 or newer.
- Debian 12: the version check of Step 2 prints 3.11, and the first of the three commands of Step 2 for a too-low version fails or is not found: those three commands add the package source ppa:deadsnakes, which serves Ubuntu only (and the command add-apt-repository comes with the package software-properties-common, which small installations lack). Debian 13 and newer have Python 3.13, which is new enough: there the two commands of Step 2 suffice. On Debian 12, build Python 3.14.5 from its source code with the commands below, in your home folder (about ten minutes; they add the command `python3.14` and leave the Python of the system unchanged), then type `python3.14` instead of `python3` in the command of Step 3 that creates the environment.

```
sudo apt install build-essential pkg-config wget libssl-dev zlib1g-dev
sudo apt install libbz2-dev libreadline-dev libsqlite3-dev
sudo apt install libffi-dev liblzma-dev libncurses-dev
wget https://www.python.org/ftp/python/3.14.5/Python-3.14.5.tgz
tar -xzf Python-3.14.5.tgz
cd Python-3.14.5
./configure
make -j 4
sudo make altinstall
cd ..
python3.14 --version
```

0.12 Notebook 00a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/00a_check_installation.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell.

A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This is the first notebook of the book. It computes no physics. It checks that your
computer has everything that the notebooks of the book need, and it shows how every
notebook of the book works. It

- prints the version of Python and of every Python package that the notebooks use,
  and checks that each package has exactly the version with which the book was
  built (its *pinned* version);
- prints the version of cargo, the program that builds the Rust programs of the
  repository, when cargo is installed (only a few notebooks need it);
- does one small computation with each of the packages numpy, sympy and mpmath and
  checks the results;
- draws two teaching plots and saves them as picture files;
- prints a line that starts with PASS for every check that passes, and as its last
  line the number of checks that passed.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 00a (Checking the installation) is the file
‘Revision/textbook/notebooks/00a_check_installation.ipynb‘ of the repository
Dirac_claude. It prints the version of Python, of every Python package of the notebooks
and of cargo (when cargo is installed), checks that every package has its pinned version,
does one small computation with each of numpy, sympy and mpmath, and draws two teaching
plots (a parabola with its tangent line, and a growing and a shrinking exponential). It
needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
```

installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00a_check_installation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00a_check_installation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 00a_check_installation.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/00a.captions.json‘
- ‘Revision/textbook/figures/00a_1_parabola_tangent.png‘
- ‘Revision/textbook/figures/00a_2_growth_and_decay.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS the figure file 00a_2_growth_and_decay.png exists
    ALL 9 CHECKS PASSED (notebook 00a)
```

and the notebook must show 2 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/00a_check_installation.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 00a_check_installation.ipynb
      python -m nbconvert --execute --inplace 00a_check_installation.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "AssertionError: check failed: every package has its pinned version": the table printed just above the error names each package with the installed and the pinned version;

install the pinned versions again with the pip commands of Step 3 (with the environment active) and run the notebook again.
- "AssertionError: check failed: Python is version 3.12 or newer": the environment was made with an older Python; delete the folder dirac-book-env in your home folder and repeat Step 3 with Python 3.12 or newer.
- Debian 12: the version check of Step 2 prints 3.11, and the first of the three commands of Step 2 for a too-low version fails or is not found: those three commands add the package source ppa:deadsnakes, which serves Ubuntu only (and the command add-apt-repository comes with the package software-properties-common, which small installations lack). Debian 13 and newer have Python 3.13, which is new enough: there the two commands of Step 2 suffice. On Debian 12, build Python 3.14.5 from its source code with the commands below, in your home folder (about ten minutes; they add the command ‘python3.14‘ and leave the Python of the system unchanged), then type ‘python3.14‘ instead of ‘python3‘ in the command of Step 3 that creates the environment.

```
sudo apt install build-essential pkg-config wget libssl-dev zlib1g-dev
sudo apt install libbz2-dev libreadline-dev libsqlite3-dev
sudo apt install libffi-dev liblzma-dev libncurses-dev
wget https://www.python.org/ftp/python/3.14.5/Python-3.14.5.tgz
tar -xzf Python-3.14.5.tgz
cd Python-3.14.5
./configure
make -j 4
sudo make altinstall
cd ..
python3.14 --version
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 00a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 00a (Checking the installation) is the file
# Revision/textbook/notebooks/00a_check_installation.ipynb of the repository
# Dirac_claude. It prints the version of Python, of every Python package of the notebooks
# and of cargo (when cargo is installed), checks that every package has its pinned
# version, does one small computation with each of numpy, sympy and mpmath, and draws two
# teaching plots (a parabola with its tangent line, and a growing and a shrinking
# exponential). It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
```

```
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
```

```
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
```

```
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 00a_check_installation.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 00a_check_installation.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 00a_check_installation.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/00a.captions.json
# - Revision/textbook/figures/00a_1_parabola_tangent.png
# - Revision/textbook/figures/00a_2_growth_and_decay.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
```

```
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 00a_2_growth_and_decay.png exists
#     ALL 9 CHECKS PASSED (notebook 00a)
#
# and the notebook must show 2 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/00a_check_installation.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 00a_check_installation.ipynb
#     python -m nbconvert --execute --inplace 00a_check_installation.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "AssertionError: check failed: every package has its pinned version": the table
#   printed just above the error names each package with the installed and the pinned
#   version; install the pinned versions again with the pip commands of Step 3 (with the
#   environment active) and run the notebook again.
# - "AssertionError: check failed: Python is version 3.12 or newer": the environment was
#   made with an older Python; delete the folder dirac-book-env in your home folder and
#   repeat Step 3 with Python 3.12 or newer.
```

```
# - Debian 12: the version check of Step 2 prints 3.11, and the first of the three
#   commands of Step 2 for a too-low version fails or is not found: those three commands
#   add the package source ppa:deadsnakes, which serves Ubuntu only (and the command
#   add-apt-repository comes with the package software-properties-common, which small
#   installations lack). Debian 13 and newer have Python 3.13, which is new enough: there
#   the two commands of Step 2 suffice. On Debian 12, build Python 3.14.5 from its source
#   code with the commands below, in your home folder (about ten minutes; they add the
#   command python3.14 and leave the Python of the system unchanged), then type
#   python3.14 instead of python3 in the command of Step 3 that creates the environment.
#     sudo apt install build-essential pkg-config wget libssl-dev zlib1g-dev
#     sudo apt install libbz2-dev libreadline-dev libsqlite3-dev
#     sudo apt install libffi-dev liblzma-dev libncurses-dev
#     wget https://www.python.org/ftp/python/3.14.5/Python-3.14.5.tgz
#     tar -xzf Python-3.14.5.tgz
#     cd Python-3.14.5
#     ./configure
#     make -j 4
#     sudo make altinstall
#     cd ..
#     python3.14 --version
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "00a"  # this notebook: chapter 00, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
```

```
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
```

```
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 00a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Python**: the programming language of the notebooks. A *version number* such as
  3.14.5 names one release of a program: the first number (3) changes rarely, the
  second (14) for new features, the third (5) for corrections.
- **Package**: a collection of Python code that someone else wrote and that we
  *import* (load) to use it: numpy (arrays of numbers and matrices), sympy (exact
  algebra and calculus with symbols), mpmath (numbers with as many digits as we
  want), matplotlib (plots).
- **Pinned version**: the exact version of a package with which the book was built.
  Another version may print slightly different numbers or pictures.
- **Private environment**: a folder with its own copy of Python and of the packages,
```

```
  so that installing them changes nothing else on the computer.
- **Notebook, cell, kernel**: a notebook is a file made of *cells*. A *markdown
  cell* (like this one) holds text; a *code cell* holds Python code. The *kernel* is
  the running Python program that executes the code cells one after the other;
  what a code cell prints appears below it.
- **Check**: a statement that must be true. The helper `check(condition, name)` of
  the set-up cell stops the notebook with an *AssertionError* when the condition is
  false, and prints `PASS name` when it is true.
- **Figure, plot, axis**: a figure is a picture; a plot draws numbers as points or
  lines; the horizontal axis and the vertical axis are the two number lines of the
  plot. **PNG** is the file format in which the figures are saved.
- **Rust, cargo**: Rust is a second programming language used for the longest
  computations of the repository; cargo is the program that turns Rust code into a
  program that the computer can run (it *builds* it).
```

Text cell:

```
## 4. The physical and mathematical situation

There is no physics in this notebook. The two plots are warm-ups for ideas that the
book uses again and again.

The first plot shows the parabola $y = x^2$ and its tangent line at the point
$(1, 1)$. The slope of the tangent line is the derivative of $x^2$ at $x = 1$,
namely $2x = 2$, so the line is $y = 1 + 2(x - 1) = 2x - 1$. The difference
$x^2 - (2x - 1) = (x - 1)^2$ is never negative and is zero only at $x = 1$: the
line touches the parabola at one point and lies below it everywhere else.

The second plot shows a growing exponential $e^{t}$ and a shrinking exponential
$e^{-t}$. Their product is $e^{t} e^{-t} = e^{t - t} = e^{0} = 1$ for every $t$. The
author's metric has this pattern: lengths along the three directions of ordinary
space carry the factor $e^{a_4}$ and lengths along the three extra times the factor
$e^{-a_4}$, so when $a_4$ grows, ordinary space inflates and the extra times
deflate.
```

Text cell:

```
## 5. Python

The next cell reads the version of Python that runs this notebook, prints it, and
checks that it is 3.12 or newer.
```

In [2]:

```
import importlib.metadata  # reads the version numbers of installed packages
import platform  # reads the version number of Python

python_version = platform.python_version()  # a text such as "3.14.5"
say(f"Python version: {python_version}")
# The first two numbers of the version, as whole numbers: "3.14.5" -> 3 and 14.
major, minor = (int(part) for part in python_version.split(".")[:2])
```

```
# Pairs of numbers are compared like words in a dictionary: first the first number,
# then, if they are equal, the second one.
check((major, minor) >= (3, 12), "Python is version 3.12 or newer")
```

Out [2]:

```
Python version: 3.14.5
PASS Python is version 3.12 or newer
```

Text cell:

```
## 6. The packages and their versions

The dictionary ‘PINNED‘ below holds, for each package, the version with which the
book was built. The loop prints the installed and the pinned version of every
package, one line each; then the check compares them.
```

In [3]:

```
PINNED = {  # package name -> the version with which the book was built
    "numpy": "2.4.6",
    "sympy": "1.14.0",
    "mpmath": "1.3.0",
    "matplotlib": "3.11.0",
    "jupyterlab": "4.4.10",
    "nbformat": "5.10.4",
    "nbclient": "0.10.2",
    "ipykernel": "7.1.0",
    "nbconvert": "7.16.6",
}
for package, pinned in PINNED.items():
    installed = importlib.metadata.version(package)  # error if not installed
    # {package:12} pads the name with blanks to 12 characters: aligned columns.
    say(f"{package:12} installed {installed:10} pinned {pinned}")
# The packages whose installed version differs from the pinned one (a list):
different = [p for p, v in PINNED.items() if importlib.metadata.version(p) != v]
check(different == [], "every package has its pinned version")
```

Out [3]:

```
numpy        installed 2.4.6      pinned 2.4.6
sympy        installed 1.14.0     pinned 1.14.0
mpmath       installed 1.3.0      pinned 1.3.0
matplotlib   installed 3.11.0     pinned 3.11.0
jupyterlab   installed 4.4.10     pinned 4.4.10
nbformat     installed 5.10.4     pinned 5.10.4
nbclient     installed 0.10.2     pinned 0.10.2
ipykernel    installed 7.1.0      pinned 7.1.0
nbconvert    installed 7.16.6     pinned 7.16.6
PASS every package has its pinned version
```

Text cell:

```
## 7. Rust (cargo)

The next cell looks for the program cargo. If it is installed, the cell prints its
version; if it is not, the cell says so. This is not a check: only the notebooks
that run the Rust programs of the repository need cargo, and they say so in their
instructions.
```

In [4]:

```
import shutil  # finds a program on the computer
import subprocess  # runs a program and reads what it prints

cargo = shutil.which("cargo")  # where cargo is, or None when it is not installed
if cargo is None:
    say("cargo is not installed; only the notebooks that run the Rust programs of "
        "the repository need it.")
else:
    completed = subprocess.run([cargo, "--version"], capture_output=True, text=True)
    say(f"cargo version: {completed.stdout.strip()}")
```

Out [4]:

```
cargo version: cargo 1.91.1 (ea2d97820 2025-10-10)
```

Text cell:

```
## 8. Three small computations

**numpy.** The determinant of the $2 \times 2$ matrix with rows $(1, 2)$ and
$(3, 4)$ is $1 \cdot 4 - 2 \cdot 3 = -2$. numpy computes it with floating-point
numbers (numbers with about 16 significant digits), so the result may differ from
$-2$ in the last digit; the check therefore allows a difference of $10^{-12}$.
```

In [5]:

```
import numpy as np  # arrays of numbers, matrices, linear algebra

matrix = np.array([[1.0, 2.0], [3.0, 4.0]])  # the rows (1, 2) and (3, 4)
determinant = np.linalg.det(matrix)  # 1*4 - 2*3 = -2, up to rounding
report("determinant of the matrix with rows (1, 2) and (3, 4)", f"{determinant:.12f}")
check(abs(determinant - (-2.0)) < 1e-12, "numpy: the determinant is -2")
```

Out [5]:

```
RESULT determinant of the matrix with rows (1, 2) and (3, 4) = -2.000000000000
PASS numpy: the determinant is -2
```

Text cell:

```
**sympy.** sympy computes exactly, with symbols. The derivative of $\sin^2 x$ is
$2 \sin x \cos x$ by the chain rule, and $2 \sin x \cos x = \sin 2x$ by the
double-angle formula. sympy prints the derivative in its own notation (‘*‘ means
times, ‘**‘ means power), and the check asks sympy to simplify the difference to
zero.
```

In [6]:

```
import sympy as sp  # exact algebra and calculus with symbols

x = sp.symbols("x")  # a symbol: a letter that stands for any number
derivative = sp.diff(sp.sin(x) ** 2, x)  # the derivative of sin(x)^2 with respect to x
say(f"d/dx sin(x)^2 = {derivative}")
check(sp.simplify(derivative - sp.sin(2 * x)) == 0,
      "sympy: the derivative of sin(x)^2 is sin(2x)")
```

Out [6]:

```
d/dx sin(x)^2 = 2*sin(x)*cos(x)
PASS sympy: the derivative of sin(x)^2 is sin(2x)
```

Text cell:

```
**mpmath.** mpmath computes with as many digits as we ask for. The number $\pi$
begins 3.14159265358979323846264338327950288 4. Its 30th significant digit is a 7
and its 31st is a 9; because 9 is 5 or more, rounding to 30 significant digits
raises the 7 to an 8: $\pi$ to 30 significant digits is
3.14159265358979323846264338328.
```

In [7]:

```
import mpmath  # numbers with as many digits as we ask for

mpmath.mp.dps = 30  # dps = decimal places: work with 30 significant digits
pi_text = mpmath.nstr(mpmath.pi, 30)  # pi written with 30 significant digits
say(f"pi to 30 significant digits: {pi_text}")
check(pi_text == "3.14159265358979323846264338328", "mpmath: pi to 30 digits")
```

Out [7]:

```
pi to 30 significant digits: 3.14159265358979323846264338328
PASS mpmath: pi to 30 digits
```

Text cell:

```
## 9. Two teaching plots

The next cell draws the parabola $y = x^2$, its tangent line $y = 2x - 1$ at the
point $(1, 1)$, and the point itself, for $x$ from $-1$ to $3$. `np.linspace(a, b,
n)` makes $n$ equally spaced numbers from $a$ to $b$; arithmetic on such an array
acts on every number of it. The check confirms that the parabola minus the line,
$(x - 1)^2$, is never negative and is zero at $x = 1$.
```

In [8]:

```
x_values = np.linspace(-1.0, 3.0, 401)  # 401 equally spaced numbers from -1 to 3
parabola = x_values ** 2  # y = x^2 at each of these numbers
tangent = 2.0 * x_values - 1.0  # the tangent line y = 1 + 2 (x - 1) = 2x - 1
gap = parabola - tangent  # (x - 1)^2 at each number

fig, ax = plt.subplots()  # a new figure with one pair of axes
ax.plot(x_values, parabola, label="parabola $y = x^2$")
ax.plot(x_values, tangent, "--", label="tangent line $y = 2x - 1$")
ax.plot([1.0], [1.0], "o", color="black", label="the point $(1, 1)$")
ax.set_xlabel("$x$")  # the label of the horizontal axis
ax.set_ylabel("$y$")  # the label of the vertical axis
ax.set_title("A parabola and its tangent line at $x = 1$ (slope 2)")
ax.legend()  # the box that names the three curves
save_figure(fig, "parabola_tangent",
            "The parabola $y = x^2$ (solid line), its tangent line $y = 2x - 1$ at "
            "the point $(1, 1)$ (dashed line) and the point itself (black dot), for "
            "$x$ from $-1$ to $3$; horizontal axis $x$, vertical axis $y$ (pure "
            "numbers, no units). The line touches the parabola only at $(1, 1)$ "
            "and lies below it everywhere else; its slope $2$ is the derivative "
            "$2x$ of $x^2$ at $x = 1$.")
check(gap.min() >= 0.0 and abs(gap[200]) < 1e-15,
      "the tangent line lies below the parabola and touches it at x = 1")
```

Out [8]:

```
Figure 00a.1 saved as Revision/textbook/figures/00a_1_parabola_tangent.png
PASS the tangent line lies below the parabola and touches it at x = 1
```

Text cell:

```
The next cell draws $e^{t}$, $e^{-t}$ and their product for $t$ from $-2$ to $2$,
and checks that the product equals 1 at all 401 points up to rounding (a
difference of at most $10^{-14}$).
```

In [9]:

```
t = np.linspace(-2.0, 2.0, 401)  # 401 equally spaced numbers from -2 to 2
growing = np.exp(t)  # e^t
shrinking = np.exp(-t)  # e^(-t)
```

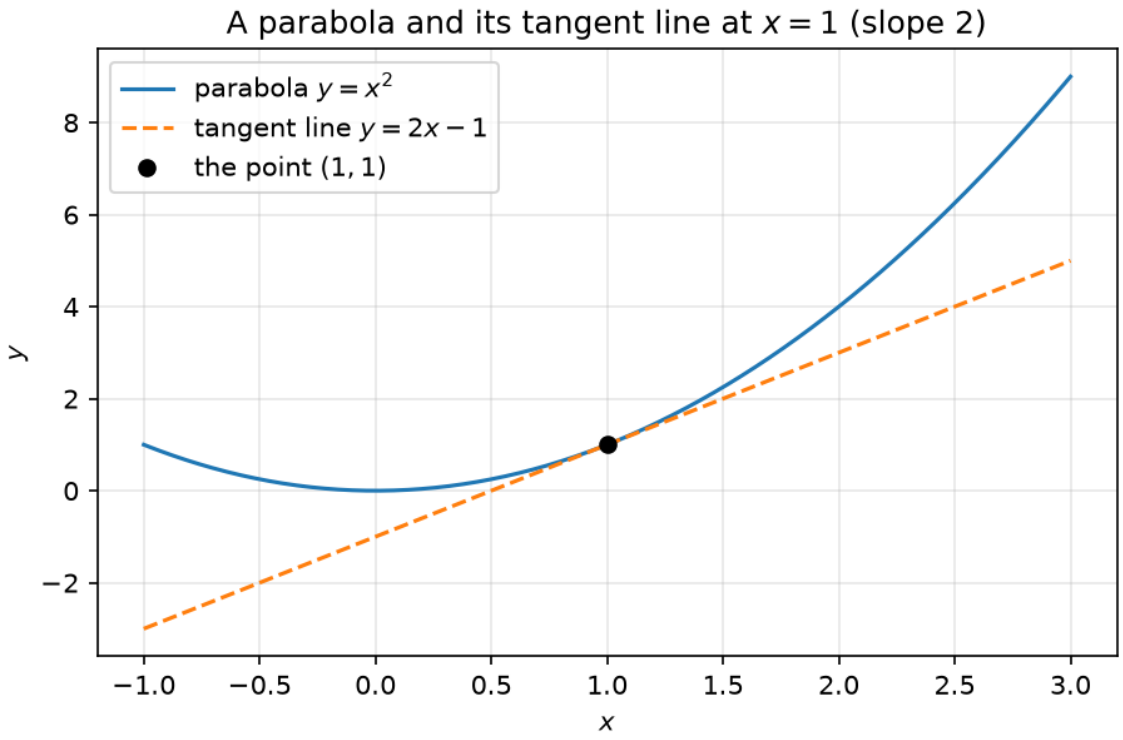

Figure 1: The parabola $y = x^2$ (solid line), its tangent line $y = 2x - 1$ at the point $(1, 1)$ (dashed line) and the point itself (black dot), for x from -1 to 3; horizontal axis x, vertical axis y (pure numbers, no units). The line touches the parabola only at $(1, 1)$ and lies below it everywhere else; its slope 2 is the derivative $2x$ of x^2 at $x = 1$. (Notebook 00a, figure 1.)

```
product = growing * shrinking  # e^t e^(-t), which is 1

fig, ax = plt.subplots()
ax.plot(t, growing, label="$e^{t}$ (grows)")
ax.plot(t, shrinking, "--", label="$e^{-t}$ (shrinks)")
ax.plot(t, product, ":", color="black", label="the product $e^{t} e^{-t} = 1$")
ax.set_xlabel("$t$")
ax.set_ylabel("value")
ax.set_title("A growing and a shrinking exponential")
ax.legend()
save_figure(fig, "growth_and_decay",
            "The growing exponential $e^{t}$ (solid line), the shrinking "
            "exponential $e^{-t}$ (dashed line) and their product (dotted line) "
            "for $t$ from $-2$ to $2$; horizontal axis $t$, vertical axis the value "
            "(pure numbers, no units). Each of the two curves is the mirror image "
            "of the other in the vertical axis $t = 0$, and their product is "
            "exactly $1$ at every $t$: what one factor gains, the other loses.")
check(np.max(np.abs(product - 1.0)) < 1e-14, "e^t times e^(-t) is 1 at every t")
```

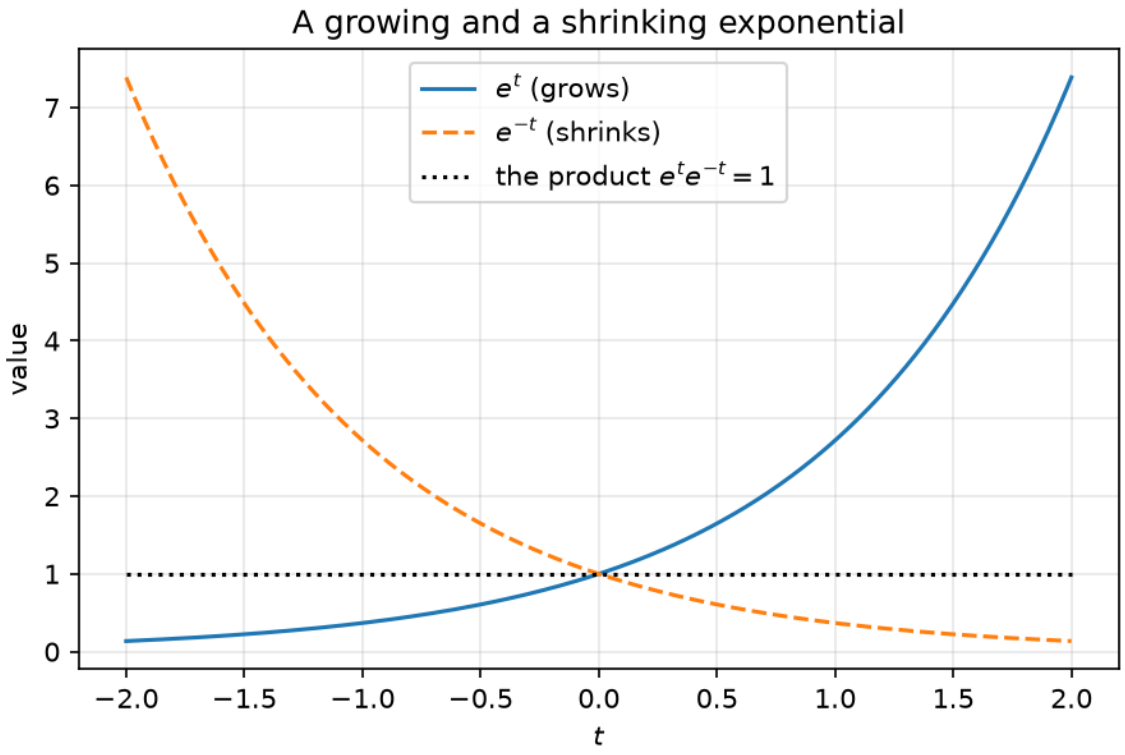

Figure 2: The growing exponential e^t (solid line), the shrinking exponential e^{-t} (dashed line) and their product (dotted line) for t from -2 to 2; horizontal axis t, vertical axis the value (pure numbers, no units). Each of the two curves is the mirror image of the other in the vertical axis $t = 0$, and their product is exactly 1 at every t: what one factor gains, the other loses. (Notebook 00a, figure 2.)

Out [9]:

```
Figure 00a.2 saved as Revision/textbook/figures/00a_2_growth_and_decay.png
PASS e^t times e^(-t) is 1 at every t
```

Text cell:

```
## 10. The last check

The last cell checks that both figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [10]:

```
for name in ("00a_1_parabola_tangent.png", "00a_2_growth_and_decay.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [10]:

```
PASS the figure file 00a_1_parabola_tangent.png exists
PASS the figure file 00a_2_growth_and_decay.png exists
ALL 9 CHECKS PASSED (notebook 00a)
```

Text cell:

```
## 11. What this notebook showed

- Python, the pinned packages and (when it is installed) cargo work on this
  computer: every notebook of the book can run here.
- Every notebook of the book has the same form: numbered sections, a markdown cell
  before every code cell, checks that print PASS lines, figures saved with
  ‘save_figure‘, and a last line that counts the checks.
- A tangent line touches a parabola at one point and its slope is the derivative.
- A growing and a shrinking exponential with opposite exponents have the product 1:
  the pattern of the author’s metric, in which ordinary space inflates with
  $e^{a_4}$ while the three extra times deflate with $e^{-a_4}$.
```

0.13 Line-by-line walk-through of Notebook 00a

The notebook has ten code cells, In [1] to In [10]. This section explains every line of every one of them. (The label In [k] is the number that Jupyter gives a code cell when it runs it; Out [k] is what that cell printed.)

In [1], the set-up cell. Its first part repeats the complete run instructions of Section 0.11 as **comment lines**: every line that starts with `#` is a comment, which Python skips. They are there so that the notebook file carries its own instructions, even when it is copied without the book. The code starts after the line THE SET-UP between two lines of `=` signs. The set-up cell is the same in every notebook of the book, except for the one line that names the notebook.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of a package) so that the code can use it; the text after `#` on each line is a comment. `json`, `os`, `textwrap` and `pathlib` come with Python itself. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python code), which show a picture file below a cell.

```
NOTEBOOK_ID = "00a"  # this notebook: chapter 00, example a
```

A **variable** is a name for a value; this line gives the name `NOTEBOOK_ID` to the text `"00a"` (a text in quotes is called a **string**). It is the only line of the set-up code that differs from notebook to notebook; the figure files and the last printed line are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. The text in triple quotes below the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. `here.parents` lists the parent folder, its parent, and so on up to the top of the disk; `[here, *here.parents]` is the **list** (an ordered collection, written in square brackets) that starts with `here` and continues with all of them. The `for` loop takes the folders of this list one after the other and runs the indented lines for each. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains `Revision/textbook/requirements.txt` is the repository, and `return` hands it back and ends the function. If no folder qualifies, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do (two strings written next to each other are joined into one).

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The first code line calls the function and names its result `REPO`. As the comment says, the notebook never prints `REPO`. The last line chooses the folder below which the notebook writes its files. `os.environ` holds the **environment variables** of the running program (named texts that a program receives from the computer when it starts); `.get(name, default)` returns the value of the variable `TEXTBOOK_OUTPUT_ROOT` if it is set and the default `str(REPO)` (the repository folder as a string) otherwise, and `Path(...)` turns the string into a path. When you run the notebook the variable is not set, so the files go into the repository. The checking tool nbkit sets it to a scratch folder (Section 0.9), so that a check never changes the repository.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions. `repository_file("Revision/...")` gives the full path of a file of the repository, for reading. `output_file("Revision/...")` gives the full path at which to write a file; `path.parent` is the folder that will hold it, and `.mkdir(parents=True, exist_ok=True)` creates that folder together with any missing folder above it, and does nothing if it exists already.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, so that every printed line fits the width of a page of the book. `str(text)` turns any value into a string; `textwrap.fill` breaks it at blanks into lines of at most 89 characters, every line after the first starting with four blanks; `print` writes the result below the cell.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

matplotlib reads personal settings from a file on your computer if you have one. `matplotlib.rcdefaults()` returns to the built-in settings, so that the figures are the same on every computer.

`plt.rcParams.update({...})` then sets four settings for every figure of the notebook: the size of a figure (7.0 inches wide and 4.2 inches high), the size of its letters (10 points), and a faint grid of lines behind the curves (`grid.alpha` 0.3 means 30 per cent opaque). The braces `{...}` make a **dictionary**: pairs of a key and a value, written `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/00a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes the text `{}` and a line end into the captions file: an empty dictionary of captions, which `save_figure` fills. `encoding="utf-8"` fixes how the letters are stored as bytes, and `newline="\n"` stores the same line end on every operating system (Section 0.22 explains why this matters).

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

This function saves a figure and shows it. `setdefault(name, value)` returns the number already stored for this figure name, or, for a new name, stores and returns `len(FIGURE_NUMBERS) + 1` (one more than the number of figures so far); so the figures are numbered 1, 2, 3, ... and a cell that you run twice keeps the number of its figure. The file name is made of the notebook id, the number and the name, for example `00a_1_parabola_tangent.png`. `fig.savefig` writes the picture as a PNG file with 150 dots per inch, cuts away the empty margin and, as the comments say, stores no program name in the file, so that two runs write exactly the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time at the end of the cell. The caption is stored in `CAPTIONS`, and the whole dictionary is written into the captions file: `json.dum`

`ps` turns it into JSON text with one entry per line (`indent=1`) and the keys in alphabetical order (`sort_keys=True`). `display(Image(...))` shows the saved picture below the cell; the `metadata` entry tells the book's tools which file the picture is. The last line prints where the figure was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty list. `check` is the function behind every check of the book. `condition` is a statement that is either true (`True`) or false (`False`). If it is false, `raise AssertionError(...)` stops the notebook with an error that names the check. (Python also has a statement `assert` for this; the docstring says why the book does not use it: Python started with its option O, for "optimise", skips every `assert`.) If the condition is true, the name is appended to `PASSED` and a line `PASS name` is printed. `record=None` makes the third argument optional: a check that reproduces a number of the Revision record passes the record's file and check name, and `check` prints them on a second line that starts with five blanks and the word reproduces.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds a blank and the unit when a unit is given and nothing otherwise (an empty string counts as false). `all_checks_passed` prints the last line of every notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one line of Out [1].

In [2], Python.

```
import importlib.metadata  # reads the version numbers of installed packages
import platform  # reads the version number of Python
```

```
python_version = platform.python_version()  # a text such as "3.14.5"
say(f"Python version: {python_version}")
```

`platform.python_version()` returns the version of the running Python as a string; on the computer that built the book it is `"3.14.5"`, which the next line prints (Out [2], first line).

```
# The first two numbers of the version, as whole numbers: "3.14.5" -> 3 and 14.
major, minor = (int(part) for part in python_version.split(".")[:2])
# Pairs of numbers are compared like words in a dictionary: first the first number,
# then, if they are equal, the second one.
check((major, minor) >= (3, 12), "Python is version 3.12 or newer")
```

`python_version.split(".")` cuts the string at the dots into the list `["3", "14", "5"]`; `[:2]` keeps the first two entries; `int(part)` turns each into a whole number; and `major, minor = ...` names them 3 and 14. Python compares the pairs `(3, 14)` and `(3, 12)` as the comment says: the first numbers are equal, so the second numbers decide, and 14 is at least 12. The condition is true, and the check prints PASS Python is version 3.12 or newer.

In [3], the packages.

```
PINNED = {  # package name -> the version with which the book was built
    "numpy": "2.4.6",
    "sympy": "1.14.0",
    "mpmath": "1.3.0",
    "matplotlib": "3.11.0",
    "jupyterlab": "4.4.10",
    "nbformat": "5.10.4",
    "nbclient": "0.10.2",
    "ipykernel": "7.1.0",
    "nbconvert": "7.16.6",
}
```

The dictionary `PINNED` holds the nine packages of Section 0.8 with their pinned versions.

```
for package, pinned in PINNED.items():
    installed = importlib.metadata.version(package)  # error if not installed
    # {package:12} pads the name with blanks to 12 characters: aligned columns.
    say(f"{package:12} installed {installed:10} pinned {pinned}")
# The packages whose installed version differs from the pinned one (a list):
different = [p for p, v in PINNED.items() if importlib.metadata.version(p) != v]
check(different == [], "every package has its pinned version")
```

`PINNED.items()` gives the pairs (package, pinned version); the loop names them `package` and `pinned`. `importlib.metadata.version(package)` reads the installed version (and stops with an error if the package is not installed). In the f-string, `{package:12}` writes the name padded with blanks to 12 characters and `{installed:10}` the version padded to 10, so that the printed table of Out [3] has straight columns. The line `different = [...]` is a **list comprehension**: it collects

every package `p` whose installed version differs from its pinned version `v` (`!=` means "is not equal to"). The check requires that this list is empty, `[]`.

In [4], cargo.

```
import shutil  # finds a program on the computer
import subprocess  # runs a program and reads what it prints

cargo = shutil.which("cargo")  # where cargo is, or None when it is not installed
if cargo is None:
    say("cargo is not installed; only the notebooks that run the Rust programs of "
        "the repository need it.")
else:
    completed = subprocess.run([cargo, "--version"], capture_output=True, text=True)
    say(f"cargo version: {completed.stdout.strip()}")
```

`shutil.which("cargo")` searches the folders in which the computer looks for programs and returns the path of cargo, or the special value `None` if there is none. `if ... else` runs the first indented block when the condition is true and the second block otherwise. `subprocess.run([...])` runs cargo with the option that asks for its version (the list holds the program and the option, two hyphens followed by the word version); `capture_output=True, text=True` collects what it prints as a string in `completed.stdout`, and `.strip()` removes the line end. On the computer that built the book cargo is installed, and Out [4] shows its version, 1.91.1. This cell prints but checks nothing, because most notebooks do not need cargo.

In [5], numpy.

```
import numpy as np  # arrays of numbers, matrices, linear algebra

matrix = np.array([[1.0, 2.0], [3.0, 4.0]])  # the rows (1, 2) and (3, 4)
determinant = np.linalg.det(matrix)  # 1*4 - 2*3 = -2, up to rounding
report("determinant of the matrix with rows (1, 2) and (3, 4)", f"{determinant:.12f}")
check(abs(determinant - (-2.0)) < 1e-12, "numpy: the determinant is -2")
```

`np.array` makes a matrix from the list of its rows. `np.linalg.det` computes its determinant, $1 \cdot 4 - 2 \cdot 3 = -2$, with floating-point numbers, which keep about 16 significant digits and may be wrong in the last one (Section 0.22). `{determinant:.12f}` prints the number with 12 digits after the decimal point: Out [5] shows -2.000000000000. `abs(...)` is the absolute value, and `1e-12` means 10^{-12}: the check accepts the result when it differs from -2 by less than 10^{-12}.

In [6], sympy.

```
import sympy as sp  # exact algebra and calculus with symbols

x = sp.symbols("x")  # a symbol: a letter that stands for any number
derivative = sp.diff(sp.sin(x) ** 2, x)  # the derivative of sin(x)^2 with respect to x
say(f"d/dx sin(x)^2 = {derivative}")
check(sp.simplify(derivative - sp.sin(2 * x)) == 0,
      "sympy: the derivative of sin(x)^2 is sin(2x)")
```

`sp.symbols("x")` makes the symbol x. In Python `**` means "to the power", so `sp.sin(x) ** 2` is $\sin^2 x$, and `sp.diff(..., x)` differentiates it with respect to x; by the chain rule the result is $2\sin x\cos x$, which sympy prints as `2*sin(x)*cos(x)` (Out [6]). The check asks sympy to simplify $2\sin x\cos x - \sin 2x$; by the double-angle formula $\sin 2x = 2\sin x\cos x$ the result is exactly 0, and `== 0` tests that.

In [7], mpmath.

```
import mpmath  # numbers with as many digits as we ask for

mpmath.mp.dps = 30  # dps = decimal places: work with 30 significant digits
pi_text = mpmath.nstr(mpmath.pi, 30)  # pi written with 30 significant digits
say(f"pi to 30 significant digits: {pi_text}")
check(pi_text == "3.14159265358979323846264338328", "mpmath: pi to 30 digits")
```

`mpmath.mp.dps = 30` tells mpmath to compute with 30 significant decimal digits. `mpmath.nstr(mpmath.pi, 30)` writes π as a string with 30 significant digits, correctly rounded (the 30th digit of π is a 7 and the 31st a 9, so the 7 is rounded up to 8), and the check compares it with the known value character by character.

In [8], the parabola and its tangent line.

```
x_values = np.linspace(-1.0, 3.0, 401)  # 401 equally spaced numbers from -1 to 3
parabola = x_values ** 2  # y = x^2 at each of these numbers
tangent = 2.0 * x_values - 1.0  # the tangent line y = 1 + 2 (x - 1) = 2x - 1
gap = parabola - tangent  # (x - 1)^2 at each number
```

`np.linspace(-1.0, 3.0, 401)` makes an **array** (numpy's list of numbers) of 401 numbers from -1 to 3 with equal steps of $4/400 = 0.01$. Arithmetic on an array acts on every number in it: `x_values ** 2` is the array of the 401 squares, and `2.0 * x_values - 1.0` the array of the 401 values of the tangent line of Section 0.10. Their difference `gap` is $(x-1)^2$ at each point.

```
fig, ax = plt.subplots()  # a new figure with one pair of axes
ax.plot(x_values, parabola, label="parabola $y = x^2$")
ax.plot(x_values, tangent, "--", label="tangent line $y = 2x - 1$")
ax.plot([1.0], [1.0], "o", color="black", label="the point $(1, 1)$")
ax.set_xlabel("$x$")  # the label of the horizontal axis
ax.set_ylabel("$y$")  # the label of the vertical axis
ax.set_title("A parabola and its tangent line at $x = 1$ (slope 2)")
ax.legend()  # the box that names the three curves
```

`plt.subplots()` makes a new figure `fig` with one pair of axes `ax`. `ax.plot(xs, ys)` draws a line through the points with these coordinates; the third argument of the second `ax.plot` line, a string of two hyphens, makes that line dashed, and `"o"` in the third one draws a dot instead of a line; `label=` is the name shown in the legend, and text between dollar signs is typeset as mathematics. The remaining lines label the axes, set the title and draw the legend.

```
save_figure(fig, "parabola_tangent",
            "The parabola $y = x^2$ (solid line), its tangent line $y = 2x - 1$ at "
```

```
            "the point $(1, 1)$ (dashed line) and the point itself (black dot), for "
            "$x$ from $-1$ to $3$; horizontal axis $x$, vertical axis $y$ (pure "
            "numbers, no units). The line touches the parabola only at $(1, 1)$ "
            "and lies below it everywhere else; its slope $2$ is the derivative "
            "$2x$ of $x^2$ at $x = 1$.")
check(gap.min() >= 0.0 and abs(gap[200]) < 1e-15,
      "the tangent line lies below the parabola and touches it at x = 1")
```

`save_figure` saves the figure as `00a_1_parabola_tangent.png` with its caption (Python joins the strings written next to each other into one) and prints Figure 00a.1 saved as ... in Out [8]. The check uses `gap.min()`, the smallest of the 401 numbers of `gap`, which must not be negative, and `gap[200]`, the entry number 200 counted from 0, which belongs to $x = -1 + 200 \cdot 0.01 = 1$ and must be 0 up to 10^{-15}. What Figure 00a.1 shows: the dashed line touches the solid parabola at the black dot and stays below it on both sides, as derived in Section 0.10.

In [9], the two exponentials.

```
t = np.linspace(-2.0, 2.0, 401)  # 401 equally spaced numbers from -2 to 2
growing = np.exp(t)  # e^t
shrinking = np.exp(-t)  # e^(-t)
product = growing * shrinking  # e^t e^(-t), which is 1
```

`np.exp` is the exponential function, applied to every number of the array. The product of two arrays is taken number by number.

```
fig, ax = plt.subplots()
ax.plot(t, growing, label="$e^{t}$ (grows)")
ax.plot(t, shrinking, "--", label="$e^{-t}$ (shrinks)")
ax.plot(t, product, ":", color="black", label="the product $e^{t} e^{-t} = 1$")
ax.set_xlabel("$t$")
ax.set_ylabel("value")
ax.set_title("A growing and a shrinking exponential")
ax.legend()
```

The plotting lines work as in In [8]: a solid line for e^t, a dashed one (two hyphens) for e^{-t}, and a dotted black one (`":"`) for their product.

```
save_figure(fig, "growth_and_decay",
            "The growing exponential $e^{t}$ (solid line), the shrinking "
            "exponential $e^{-t}$ (dashed line) and their product (dotted line) "
            "for $t$ from $-2$ to $2$; horizontal axis $t$, vertical axis the value "
            "(pure numbers, no units). Each of the two curves is the mirror image "
            "of the other in the vertical axis $t = 0$, and their product is "
            "exactly $1$ at every $t$: what one factor gains, the other loses.")
check(np.max(np.abs(product - 1.0)) < 1e-14, "e^t times e^(-t) is 1 at every t")
```

`save_figure` saves the figure as `00a_2_growth_and_decay.png`. The check computes `np.abs(product - 1.0)`, the distance of each of the 401 products from 1, takes the largest with `np.max`,

and requires it to be below 10^{-14}: up to rounding, $e^t e^{-t} = 1$. What Figure 00a.2 shows: the two curves are mirror images of each other in the line $t = 0$, and the dotted product stays flat at 1.

In [10], the last check.

```
for name in ("00a_1_parabola_tangent.png", "00a_2_growth_and_decay.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The loop runs over a **tuple** (a list in round brackets that cannot be changed) of the two figure names and checks that each file exists. `all_checks_passed()` prints the last line, ALL 9 CHECKS PASSED (notebook 00a): one check each in In [2], In [3], In [5], In [6], In [7], In [8] and In [9], and two in In [10].

0.14 The eight directions and the author's metric

Every later chapter lives in the author's universe, so we take a first look at it now, with nothing but school algebra. Chapter 3 returns to it with the full mathematics of curved space; here we need only four facts: which directions are like space and which are like time, how lengths along each direction change as the time runs, why ordinary space inflates while the extra times deflate, and why the volume does not change.

Steps and their sizes. On a flat sheet of paper with the coordinates x and y, a small step dx to the right and dy upwards has a length ds with $ds^2 = dx^2 + dy^2$: this is the theorem of Pythagoras for the right-angled triangle with the sides dx and dy. In ordinary space with the coordinates x_1, x_2, x_3 the same theorem, used twice, gives $ds^2 = dx_1^2 + dx_2^2 + dx_3^2$. A **metric** is a rule of this kind: it turns the small steps $dx_0, dx_1, \ldots, dx_7$ along the eight coordinates into one number ds^2, the squared size of the step. The rules of this book are **diagonal**: each squared step is multiplied by its own number g_{ii}, called the entry of the metric for the direction x_i, and the results are added:

$$ds^2 = g_{00}\, dx_0^2 + g_{11}\, dx_1^2 + \cdots + g_{77}\, dx_7^2.$$

The entries $g_{00}, \ldots, g_{77}$ may change from place to place and from time to time; that is what can make a space curved (Chapter 3).

Space-like and time-like. In the special relativity of Einstein (1905), in the form that Minkowski gave it in 1908, a step in space and time is measured with a minus sign in front of the time step: in units in which the speed of light is 1, the squared size of a step is $dx_1^2 + dx_2^2 + dx_3^2 - dx_4^2$ when x_4 is the time. A direction whose entry is positive is called **space-like** (a step along it is measured like a distance); a direction whose entry is negative is called **time-like** (a step along it is measured like a duration). The list of the signs of the eight directions is called η (the Greek letter eta), and the numbers of plus and minus signs form the **signature**. In the author's universe

$$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$$

in the order $x_0, \ldots, x_7$: the four directions x_0, x_1, x_2 and x_3 are space-like, the four directions x_4, x_5, x_6 and x_7 are time-like, and the signature is (4,4). Here diag(...) is the square table of numbers

(a **matrix**, Chapter 1) that has the listed numbers on its diagonal, from the top left to the bottom right, and zeros everywhere else.

The author's metric. The author gave the metric of their primordial universe as an 8×8 diagonal matrix (`Revision/SPEC.md`, section 1). Its eight entries, in the order $x_0, x_1, \ldots, x_7$, are

$$g_{00} = \cot^2 z, \qquad g_{11} = g_{22} = g_{33} = e^{2a_4} \sin^{1/3} z, \qquad g_{44} = -1,$$

$$g_{55} = g_{66} = g_{77} = -b_4 \, e^{-2a_4} \sin^{1/3} z,$$

where $z = 6Hx_0$, $H > 0$ is a constant of the author, and a_4 and b_4 are functions of the time x_4 alone, the **metric functions** (their field equations are the subject of Chapter 12); b_4 is positive. The function b_4 is the author's change of 2026-10-14: with $b_4 = 1$ the metric is the one used before that date. With the author's own weights every vacuum history has b_4 constant (`Revision/metric_b4/results/vacuum-b4.json`), and a constant b_4 is removed by rescaling x_5, x_6, x_7; the backgrounds of the later chapters therefore take $b_4 = 1$, as the Revision record does, and say so where they do (the record of the change: `Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md`). The hidden coordinate is restricted so that $0 < z < \pi/2$; this interval is called the **patch**. Here $\sin^{1/3} z$ means $(\sin z)^{1/3}$, the cube root of $\sin z$, and $\cot z = \cos z / \sin z$. Status: ASSUMED. The metric is the author's input; the book does not derive it, and Chapter 12 asks which source it would need.

We now derive the four facts. Each line of a derivation is followed, in brackets, by the rule that produced it from the line before.

Fact 1: the signs. Take any point of the patch, $0 < z < \pi/2$, any value of a_4 and any positive value of b_4.

$$\sin z > 0 \quad \text{and} \quad \cos z > 0$$

(the sine and the cosine of an angle between 0 and a right angle are positive),

$$\sin^{1/3} z > 0$$

(the cube root of a positive number is positive),

$$e^{2a_4} > 0 \quad \text{and} \quad e^{-2a_4} > 0$$

(the exponential function e^u is positive for every number u),

$$g_{11} = e^{2a_4} \sin^{1/3} z > 0 \quad \text{and} \quad g_{55} = -b_4 \, e^{-2a_4} \sin^{1/3} z < 0$$

(a product of positive numbers is positive, $b_4 > 0$, and a positive number with a minus sign in front is negative),

$$\cot z = \frac{\cos z}{\sin z} > 0, \quad \text{so} \quad g_{00} = \cot^2 z > 0$$

(a quotient of two positive numbers is positive, and the square of a nonzero number is positive). Together with $g_{44} = -1 < 0$ the signs of the eight entries are $(+,+,+,+,-,-,-,-)$, exactly the list η, at every point of the patch and for every value of a_4 and of $b_4 > 0$. Status: PROVED (by the school algebra above). The Revision record states the signs in the file `Revision/algebra/gammas.json`, and its Wolfram verifier checks them (`Revision/algebra/reports/wolfram-algebra.json`, check `eta_in_author_order`); Notebook 00b confirms the signs of the eight entries at 14457 points (COMPUTED).

Fact 2: the length factors. A step dx_1 along x_1 alone (all other steps zero) has $ds^2 = g_{11}\,dx_1^2$, so its size is $|ds| = \sqrt{|g_{11}|}\,|dx_1|$. The number $f_i = \sqrt{|g_{ii}|}$ is called the **length factor** (or scale factor) of the direction x_i: it converts a step of the coordinate into a length. For x_1:

$$f_1 = \sqrt{e^{2a_4}\sin^{1/3} z}$$

(the definition, with $|g_{11}| = g_{11}$ because $g_{11} > 0$ by Fact 1),

$$f_1 = \sqrt{e^{2a_4}}\,\sqrt{\sin^{1/3} z}$$

(the square root of a product of two nonnegative numbers is the product of their square roots),

$$f_1 = e^{a_4}\sin^{1/6} z$$

(first, $\sqrt{e^{2a_4}} = e^{a_4}$, because e^{a_4} is positive and its square is $e^{a_4}e^{a_4} = e^{2a_4}$; second, $\sqrt{u^{1/3}} = (u^{1/3})^{1/2} = u^{1/6}$ by the power rule $(u^p)^q = u^{pq}$). The same three lines give $f_2 = f_3 = e^{a_4}\sin^{1/6} z$. For the extra times, $|g_{55}| = b_4\,e^{-2a_4}\sin^{1/3} z$ (a minus sign in front of a positive number is removed by the absolute value), and the same steps, with $\sqrt{b_4}$ as one more factor, give

$$f_5 = f_6 = f_7 = b_4^{1/2}\,e^{-a_4}\sin^{1/6} z.$$

Finally $f_4 = \sqrt{|-1|} = 1$, and $f_0 = \sqrt{\cot^2 z} = |\cot z| = \cot z$ (the square root of a square is the absolute value, and $\cot z > 0$ on the patch). Status: PROVED.

Fact 3: ordinary space inflates, the extra times deflate. Fix the hidden coordinate, so that $\sin^{1/6} z$ is a fixed positive number, and let b_4 be constant. When a_4 grows, e^{a_4} grows and $e^{-a_4} = 1/e^{a_4}$ shrinks (Section 0.10): the length factors of x_1, x_2, x_3 grow (ordinary space **inflates**) and those of x_5, x_6, x_7 shrink (the extra times **deflate**). The simplest history is $a_4 = AHx_4$ with a positive constant A, and $b_4 = 1$. It is the history that the Kohn-Sham chapters use, where it is ASSUMED as a **prescribed background** (`Revision/field_equations_a4/reports/ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`), and Chapter 12 shows that the sign of A is a choice: with $-A$ in place of A the roles of growth and shrinking are exchanged. Along this history

$$f_5 = e^{-AHx_4}\sin^{1/6} z$$

(substitution of $a_4 = AHx_4$ and $b_4 = 1$ into Fact 2),

$$\frac{df_5}{dx_4} = -AH\, e^{-AHx_4} \sin^{1/6} z$$

(the chain rule: the derivative of e^u with respect to x_4 is $e^u\, du/dx_4$, here with $u = -AHx_4$ and $du/dx_4 = -AH$; the constant factor $\sin^{1/6} z$ stays in front),

$$\frac{df_5}{dx_4} = -AH\, f_5$$

(the previous line contains f_5 itself, by the first line). A quantity whose rate of change is a fixed negative multiple of itself decreases **exponentially**: in every interval of time of length $1/(AH)$ it is multiplied by the same factor $e^{-1} = 0.3679$. The same steps give $df_1/dx_4 = +AH\, f_1$: ordinary space inflates exponentially. Status: PROVED (for the ASSUMED history). In every history the product of an inflating and a deflating factor contains no a_4:

$$f_1 f_5 = b_4^{1/2}\, e^{a_4} e^{-a_4} \sin^{1/6} z\, \sin^{1/6} z = b_4^{1/2} \sin^{1/3} z$$

(the law of exponents $e^u e^v = e^{u+v}$ gives $e^{a_4 - a_4} = e^0 = 1$, and $u^{1/6} u^{1/6} = u^{1/3}$); it is constant in time whenever b_4 is.

Fact 4: the determinant and the volume factor. The **determinant** is one number computed from a square matrix (Chapter 1 defines it in general); for a diagonal matrix it is the product of the diagonal entries (a fact proved in Chapter 1 and used here). For the author's metric:

$$\det g = \cot^2 z \cdot \left(e^{2a_4} \sin^{1/3} z\right)^3 \cdot (-1) \cdot \left(-b_4\, e^{-2a_4} \sin^{1/3} z\right)^3$$

(the product of the eight entries, the three equal entries of ordinary space and the three equal entries of the extra times collected as cubes),

$$\left(e^{2a_4} \sin^{1/3} z\right)^3 = e^{6a_4} \sin z$$

(the rule $(uv)^3 = u^3 v^3$, then $(e^{2a_4})^3 = e^{6a_4}$ and $(\sin^{1/3} z)^3 = \sin z$, both by the power rule $(u^p)^q = u^{pq}$),

$$\left(-b_4\, e^{-2a_4} \sin^{1/3} z\right)^3 = (-1)^3\, b_4^3\, e^{-6a_4} \sin z = -b_4^3\, e^{-6a_4} \sin z$$

(the same rules, and $(-1)^3 = -1$),

$$\det g = \cot^2 z \cdot e^{6a_4} \sin z \cdot (-1) \cdot \left(-b_4^3\, e^{-6a_4} \sin z\right)$$

(substitution of the two cubes into the first line),

$$\det g = b_4^3 \cdot e^{6a_4} e^{-6a_4} \cdot (-1)(-1) \cdot \sin^2 z \cot^2 z$$

(the factors of a product may be taken in any order),

$$\det g = b_4^3 \sin^2 z \cot^2 z$$

($e^{6a_4} e^{-6a_4} = e^0 = 1$ by the law of exponents, and $(-1)(-1) = 1$),

$$\det g = b_4^3 \sin^2 z \cdot \frac{\cos^2 z}{\sin^2 z} = b_4^3 \cos^2 z$$

(the definition $\cot z = \cos z / \sin z$, then cancelling $\sin^2 z$, which is not zero on the patch). The determinant does not contain a_4. Its square root is

$$\sqrt{|\det g|} = \sqrt{b_4^3 \cos^2 z} = b_4^{3/2} \cos z$$

(the determinant is positive, the square root of a product of nonnegative numbers is the product of their square roots, $\sqrt{b_4^3} = b_4^{3/2}$, and the square root of a square is the absolute value, $|\cos z| = \cos z$ on the patch). This number is the **volume factor**: a small box with the coordinate sides $dx_0, \ldots, dx_7$ has the sides $f_0\,dx_0, \ldots, f_7\,dx_7$ in length, so its volume is $f_0 f_1 \cdots f_7\,dx_0 \cdots dx_7$, and the product of the length factors is $f_0 \cdots f_7 = \sqrt{|g_{00}| \cdots |g_{77}|} = \sqrt{|\det g|}$ (a product of square roots is the square root of the product). Written with the length factors of Fact 2, $f_1 f_2 f_3 f_5 f_6 f_7 = (f_1 f_5)^3 = b_4^{3/2} \sin z$ by Fact 3, and $f_0 f_4 = \cot z$, so the volume factor is $b_4^{3/2} \sin z \cot z = b_4^{3/2} \cos z$ once more. The inflation of the three directions of ordinary space and the deflation of the three extra times cancel exactly in the volume; the volume factor changes in time only through b_4, and at $b_4 = 1$ it is $\cos z$. Status: PROVED, by the derivation above and by exact symbolic computations of the Revision record that hold for every function a_4:

- the Wolfram verifier of the field theory with b_4, in its report `Revision/theory/reports/wolfram-field-theory-b4.json`, check `sqrt_det_g` ($\det g = b_4^3 \cos^2 z$);
- the Wolfram verifier of the field equations for a_4 and b_4, check `sqrt_abs_det_g_is_cos_z` ($\sqrt{|\det g|} = b_4^{3/2} \cos z$), in its report `Revision/field_equations_a4/reports/wolfram-a4-report.json`;
- at $b_4 = 1$, the Wolfram verifier of the field theory, in its report `Revision/theory/reports/wolfram-field-theory.json`, check `sqrt_det_g_is_cos_z`, and the Python verifier of the field theory, in its report `Revision/theory/reports/python-field-theory.json`, check `sqrt_det_g_equals_cos_z`.

Numbers to check by hand. Take $b_4 = 1$. At $z = \pi/4$ we have $\sin(\pi/4) = 1/\sqrt{2} = 2^{-1/2}$, so $\sin^{1/6}(\pi/4) = 2^{-1/12} = 0.9439$, and $\cot(\pi/4) = 1$, $\cos(\pi/4) = 0.7071$. With $a_4 = 0.5$ the factor of ordinary space is $e^{0.5} \cdot 0.9439 = 1.6487 \cdot 0.9439 = 1.5562$ and that of an extra time $e^{-0.5} \cdot 0.9439 = 0.6065 \cdot 0.9439 = 0.5725$; with $a_4 = 1$ they are $2.7183 \cdot 0.9439 = 2.5657$ and $0.3679 \cdot 0.9439 = 0.3472$. In every case the product of the eight factors is $\cos(\pi/4) = 0.7071$. With $b_4 = 2$ every extra-time factor is multiplied by $\sqrt{2}$ and the product by $2^{3/2} = 2.8284$. Notebook 00b prints exactly these numbers.

Example: the eight directions in numbers and pictures. Notebook 00b reads the names and the signs of the eight coordinates from the Revision record, writes the author's metric with the package sympy and compares its eight entries, one by one, with the formulas that the Revision verifiers recorded (the metric with b_4, and the metric at $b_4 = 1$), checks the signs of the entries at 14457 points, computes the determinant exactly, prints the table of the length factors above, and draws four figures: the signs as a picture, the growing and the shrinking factor, the eight length factors as bars, and the volume factor. It checks every fact of this section and ends with ALL 17 CHECKS PASSED (notebook 00b).

0.15 How to run Notebook 00b

Step 1. What this notebook does and what it needs.

Notebook 00b (The eight directions and the author's metric: a first look) is the file `Revision /textbook/notebooks/00b_eight_directions.ipynb` of the repository Dirac_claude. It reads the names and the signs of the eight coordinates x0 to x7 from the Revision record, writes the author's metric (with the functions a4 and b4) with sympy, checks that its eight entries, their signs and its determinant b4^3 cos^2 z agree with what the Revision verifiers recorded, and draws four teaching plots: the signs as a heat map, the growing and the shrinking length factors, the eight length factors as bars, and the volume factor. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00b_eight_directions.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00b_eight_directions.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 00b_eight_directions.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/00b.captions.json`
- `Revision/textbook/figures/00b_1_eta_heat_map.png`
- `Revision/textbook/figures/00b_2_scale_factors.png`
- `Revision/textbook/figures/00b_3_length_factors.png`
- `Revision/textbook/figures/00b_4_volume_factor.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the four figure files of this notebook exist
ALL 17 CHECKS PASSED (notebook 00b)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/00b_eight_directions.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 00b_eight_directions.ipynb
python -m nbconvert --execute --inplace 00b_eight_directions.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

- "FileNotFoundError" or "KeyError" naming a file below the folder Revision: a file of the Revision record is missing or damaged: the repository folder is incomplete. Delete the folder Dirac_claude, download it again with the git clone command of Step 3, and open the notebook from the new folder.

0.16 Notebook 00b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/00b_eight_directions.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook is a first look at the universe of the book. It computes no new
physics; it shows, with numbers and pictures, the facts about the eight directions
and the author's metric that every later chapter uses. It

- reads the names of the eight coordinates $x_0, x_1, \dots, x_7$ and their signs from a
  file of the Revision record (the computations stored in the folder Revision of
  the repository), and prints which directions are like space and which are like
  time;
- writes the author's metric with the package sympy and checks that its eight
  entries are exactly those that the Revision verifiers recorded, that their signs
  are the recorded signs, and that its determinant is $b_4^3 \cos^2 z$;
- shows what happens when the metric function $a_4$ grows: lengths along ordinary
  space grow (inflation) and lengths along the three extra times shrink (deflation),
  while the volume factor stays the same;
- draws four teaching plots and prints a PASS line for every check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 00b (The eight directions and the author's metric: a first look) is the file
`Revision/textbook/notebooks/00b_eight_directions.ipynb` of the repository Dirac_claude.
It reads the names and the signs of the eight coordinates x0 to x7 from the Revision
record, writes the author's metric (with the functions a4 and b4) with sympy, checks that
its eight entries, their signs and its determinant b4^3 cos^2 z agree with what the
Revision verifiers recorded, and draws four teaching plots: the signs as a heat map, the
growing and the shrinking length factors, the eight length factors as bars, and the
volume factor. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
```

2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00b_eight_directions.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00b_eight_directions.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 00b_eight_directions.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/00b.captions.json‘
- ‘Revision/textbook/figures/00b_1_eta_heat_map.png‘
- ‘Revision/textbook/figures/00b_2_scale_factors.png‘
- ‘Revision/textbook/figures/00b_3_length_factors.png‘
- ‘Revision/textbook/figures/00b_4_volume_factor.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the four figure files of this notebook exist
ALL 17 CHECKS PASSED (notebook 00b)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/00b_eight_directions.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command

below does what ‘jupyter lab‘ does, the second what the headless command does:

```
    python -m jupyterlab 00b_eight_directions.ipynb
    python -m nbconvert --execute --inplace 00b_eight_directions.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" or "KeyError" naming a file below the folder Revision: a file of the Revision record is missing or damaged: the repository folder is incomplete. Delete the folder Dirac_claude, download it again with the git clone command of Step 3, and open the notebook from the new folder.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 00b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 00b (The eight directions and the author’s metric: a first look) is the file
# Revision/textbook/notebooks/00b_eight_directions.ipynb of the repository Dirac_claude.
# It reads the names and the signs of the eight coordinates x0 to x7 from the Revision
# record, writes the author’s metric (with the functions a4 and b4) with sympy, checks
# that its eight entries, their signs and its determinant b4^3 cos^2 z agree with what
# the Revision verifiers recorded, and draws four teaching plots: the signs as a heat
# map, the growing and the shrinking length factors, the eight length factors as bars,
# and the volume factor. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
# shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
```

```
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
```

```
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
```

```
#     cd Revision/textbook/notebooks
#     jupyter lab 00b_eight_directions.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 00b_eight_directions.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 00b_eight_directions.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/00b.captions.json
# - Revision/textbook/figures/00b_1_eta_heat_map.png
# - Revision/textbook/figures/00b_2_scale_factors.png
# - Revision/textbook/figures/00b_3_length_factors.png
# - Revision/textbook/figures/00b_4_volume_factor.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the four figure files of this notebook exist
#     ALL 17 CHECKS PASSED (notebook 00b)
```

```
#
# and the notebook must show 4 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/00b_eight_directions.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 00b_eight_directions.ipynb
#     python -m nbconvert --execute --inplace 00b_eight_directions.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" or "KeyError" naming a file below the folder Revision: a file of
#   the Revision record is missing or damaged: the repository folder is incomplete.
#   Delete the folder Dirac_claude, download it again with the git clone command of Step
#   3, and open the notebook from the new folder.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "00b"  # this notebook: chapter 00, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
```

```
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 00b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinate**: a number that says where a point is along one direction. The
  author's universe has eight coordinates, named $x_0, x_1, \dots, x_7$ (the author's
  own names, in the author's order; the hidden direction $x_0$ comes first).
- **Space-like and time-like**: two kinds of direction. Along a space-like direction
  a step is measured like a distance; along a time-like direction it is measured
  like a duration. In the metric a space-like direction has a positive entry and a
  time-like direction a negative entry.
- **Metric**: the rule that turns a small step in the coordinates into its size. For
  the author's metric the rule is
  $ds^2 = g_{00} dx_0^2 + g_{11} dx_1^2 + \dots + g_{77} dx_7^2$, where $dx_i$ is the
  small step along $x_i$ and $g_{ii}$ is the $i$-th entry of the metric.
- **Diagonal matrix**: a square table of numbers that is zero everywhere except on
  the line from the top left to the bottom right (the *diagonal*). The author's
  metric is an $8 \times 8$ diagonal matrix with the entries $g_{00}, \dots, g_{77}$.
- **Determinant**: one number computed from a square matrix. For a diagonal matrix it
  is the product of the diagonal entries.
- **Signature**: the numbers of positive and of negative entries of the metric. Here
  it is (4,4): four space-like and four time-like directions.
- **Length factor**: the number $f_i = \sqrt{|g_{ii}|}$; a small step $dx_i$ along
  $x_i$ has the size $f_i \, |dx_i|$.
- **Inflate, deflate**: grow and shrink. A direction inflates when its length factor
  grows with time and deflates when it shrinks.
- **Exponential**: the function $e^{t}$; $e^{-t} = 1 / e^{t}$.
- **Logarithmic axis**: an axis on which equal steps mean equal *factors* (1, 10,
  100, ...) instead of equal differences; on it $e^{t}$ and $e^{-t}$ are straight
  lines.
- **Heat map**: a picture of a matrix in which every entry is a coloured square.
- **Symbol** (sympy): a letter that stands for any number, so that sympy can compute
  exactly with formulas instead of with decimal numbers.
- **Record, report, JSON**: the Revision folder stores its results as *JSON* files
  (plain text files of names, numbers and lists). A *report* is a JSON file in which
  a verifier program has recorded its checks, each with a name, a *verdict* (PASS or
  FAIL) and a detail text.
```

Text cell:

```
## 4. The physical and mathematical situation
```

The author's universe has eight directions. Their names and roles are:

- x_0: a hidden direction of space; space-like;
- x_1, x_2, x_3: the three directions of ordinary space; space-like; they inflate;
- x_4: the time in which everything evolves; time-like;
- x_5, x_6, x_7: the three **extra times**; time-like; they deflate exponentially.

The author's metric is diagonal, with the entries (in the order $x_0, x_1, \dots, x_7$)

$$g = \mathrm{diag}\big(\cot^2 z,\; e^{2 a_4} s,\; e^{2 a_4} s,\; e^{2 a_4} s,\; -1,\; -b_4 e^{-2 a_4} s,\; -b_4 e^{-2 a_4} s,\; -b_4 e^{-2 a_4} s\big),$$

where $s = \sin^{1/3} z$, $z = 6 H x_0$, $H > 0$ is a constant of the author, and a_4 and $b_4 > 0$ are functions of the time x_4 only. The hidden coordinate is restricted so that $0 < z < \pi/2$; there $\sin z > 0$ and $\cot z > 0$. (The function b_4 is the author's change of 2026-10-14; with $b_4 = 1$ the metric is the one used before. With the author's own weights every vacuum history has b_4 constant, and a constant b_4 is removed by rescaling x_5, x_6, x_7, so the backgrounds of the later chapters take $b_4 = 1$, as the Revision record does, and say so where they do. The record of the change is the file Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md.)

Reading the metric:

1. On $0 < z < \pi/2$ every entry $e^{2 a_4} s$, $b_4 e^{-2 a_4} s$ and $\cot^2 z$ is positive. So the entries of x_0, x_1, x_2, x_3 are positive (space-like) and those of x_4, x_5, x_6, x_7 are negative (time-like): the signs are $(+, +, +, +, -, -, -, -)$, the same for every value of a_4, b_4 and z.
2. The length factor of each of x_1, x_2, x_3 is $\sqrt{e^{2 a_4} s} = e^{a_4} \sin^{1/6} z$, and that of each of x_5, x_6, x_7 is $\sqrt{b_4 e^{-2 a_4} s} = b_4^{1/2} e^{-a_4} \sin^{1/6} z$. When a_4 grows, the first grows and the second shrinks: ordinary space inflates and the extra times deflate. If a_4 grows in proportion to the time, $a_4 = A H x_4$ with a positive number A, and b_4 is constant (the simplest history; later chapters examine what source it needs, and the Kohn-Sham chapters use it as a prescribed background, an assumption), then $e^{-a_4} = e^{-A H x_4}$ shrinks exponentially in time.
3. The determinant of a diagonal matrix is the product of its diagonal entries. The three entries of ordinary space give $(e^{2 a_4} s)^3 = e^{6 a_4} s^3 = e^{6 a_4} \sin z$, because $s^3 = (\sin^{1/3} z)^3 = \sin z$. The three entries of the extra times give $(-b_4 e^{-2 a_4} s)^3 = (-1)^3 b_4^3 e^{-6 a_4} s^3 = -b_4^3 e^{-6 a_4} \sin z$. The entry of the time is -1 and that of the hidden direction is $\cot^2 z$. So $\det g = e^{6 a_4} \sin z \cdot (-b_4^3 e^{-6 a_4} \sin z) \cdot (-1) \cdot \cot^2 z = b_4^3 \sin^2 z \cot^2 z$, because $e^{6 a_4} e^{-6 a_4} = e^{0} = 1$ and $(-1)(-1) = 1$. Since $\cot z = \cos z / \sin z$, this is $b_4^3 \sin^2 z \cdot \cos^2 z / \sin^2 z = b_4^3 \cos^2 z$. It does not depend on a_4: the growth of the three space factors and the shrinking of the three extra-time factors cancel in the volume. On $0 < z < \pi/2$, $\cos z > 0$, so $\sqrt{|\det g|} = b_4^{3/2} \cos z$; it changes in time only if b_4 does, and at $b_4 = 1$ it is $\cos z$.

The notebook checks each of these statements with the computer and compares them

```
with the Revision record.
```

Text cell:

```
## 5. The eight coordinates in the Revision record

The file Revision/algebra/gammas.json of the Revision record holds the gamma
matrices of the book (they appear in later chapters) together with the names of
the coordinates and their signs. The next cell reads the file, prints a table of
the eight directions, and checks the names, the signs and the signature. It also
defines two small functions. ‘recorded_check‘ looks up one named check in a
Revision report; the cell uses it to show what the Wolfram verifier of the
Revision record recorded about the signs. ‘check_reproduces‘ is the helper ‘check‘
for a check that reproduces a Revision record: it first calls ‘sys.stdout.flush()‘,
which sends all printed text that is still waiting, and then calls ‘check‘ with
the record. (Jupyter sends printed text in pieces; flushing first keeps the PASS
line and the line ‘reproduces ...‘ below it in one piece, so that the book’s tools
read them together.)
```

In [2]:

```
import sys  # sys.stdout is the channel through which the notebook prints

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

GAMMAS_FILE = "Revision/algebra/gammas.json"  # coordinates, signs and gammas
gammas_record = json.loads(repository_file(GAMMAS_FILE).read_text(encoding="utf-8"))
coordinates = gammas_record["coordinates"]  # the names x0 ... x7, in order
eta = gammas_record["eta"]  # the sign of each direction: +1 or -1

ROLES = {  # the role of each coordinate in the author’s universe
    "x0": "hidden direction of space",
    "x1": "ordinary space (inflates)",
    "x2": "ordinary space (inflates)",
    "x3": "ordinary space (inflates)",
    "x4": "the time",
    "x5": "extra time (deflates)",
    "x6": "extra time (deflates)",
    "x7": "extra time (deflates)",
}
say("coordinate  sign  kind        role")
for name, sign in zip(coordinates, eta):
    kind = "space-like" if sign > 0 else "time-like"
    # {sign:+d} writes the sign of a whole number (+1 or -1) in front of it.
    say(f"{name:10}  {sign:+d}    {kind:10}  {ROLES[name]}")


def recorded_check(path, name):
    """The verdict (PASS or FAIL) and the detail text that the Revision report
    path recorded for its check called name."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
```

```
    for entry in data["checks"]:  # every check of the report, one after the other
        if entry["name"] == name:
            return entry["verdict"].upper(), entry["detail"]
    raise KeyError(f"{path} has no check {name}")


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), after sending the waiting output."""
    sys.stdout.flush()  # send every printed line that is still waiting
    check(condition, name, record=record)


ALGEBRA_REPORT = "Revision/algebra/reports/wolfram-algebra.json"
verdict, detail = recorded_check(ALGEBRA_REPORT, "eta_in_author_order")
say(f"The Wolfram verifier recorded for eta_in_author_order: {verdict}")

check(coordinates == ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"],
      "the record names the eight coordinates x0 to x7 in this order")
check(eta == [1, 1, 1, 1, -1, -1, -1, -1],
      "the record gives the signs (+,+,+,+,-,-,-,-) in the order x0..x7")
check(eta.count(1) == 4 and eta.count(-1) == 4,
      "four space-like and four time-like directions: the signature is (4,4)")
check(verdict == "PASS",
      "the Wolfram verifier recorded PASS for these signs (eta_in_author_order)")
```

Out [2]:

```
coordinate  sign  kind        role
x0          +1    space-like  hidden direction of space
x1          +1    space-like  ordinary space (inflates)
x2          +1    space-like  ordinary space (inflates)
x3          +1    space-like  ordinary space (inflates)
x4          -1    time-like   the time
x5          -1    time-like   extra time (deflates)
x6          -1    time-like   extra time (deflates)
x7          -1    time-like   extra time (deflates)
The Wolfram verifier recorded for eta_in_author_order: PASS
PASS the record names the eight coordinates x0 to x7 in this order
PASS the record gives the signs (+,+,+,+,-,-,-,-) in the order x0..x7
PASS four space-like and four time-like directions: the signature is (4,4)
PASS the Wolfram verifier recorded PASS for these signs (eta_in_author_order)
```

Text cell:

```
The next cell draws the signs as a heat map: the $8 \times 8$ diagonal matrix
$\eta$ with $\eta_{ii}$ equal to the sign of $x_i$ and zero off the diagonal. A
*colour map* turns each number into a colour: $-1$ blue, $0$ grey, $+1$ red. The
number of each square is also written into it, so that the picture can be read
without colours.
```

In [3]:

```
from matplotlib.colors import ListedColormap  # a colour map made of a few colours
from matplotlib.patches import Patch  # a coloured square for the legend

eta_matrix = np.diag(eta)  # 8 x 8: the signs on the diagonal, 0 elsewhere
BLUE, GREY, RED = "#2a78d6", "#f0efec", "#e34948"
colour_map = ListedColormap([BLUE, GREY, RED])  # three colours for -1, 0, +1

fig, ax = plt.subplots(figsize=(6.4, 5.4))
# vmin and vmax split the numbers into three bands: -1, 0 and +1.
ax.imshow(eta_matrix, cmap=colour_map, vmin=-1.5, vmax=1.5)
for row in range(8):
    for column in range(8):
        value = int(eta_matrix[row, column])
        text = "0" if value == 0 else f"{value:+d}"
        colour = "#52514e" if value == 0 else "white"  # readable on each colour
        ax.text(column, row, text, ha="center", va="center", color=colour,
                fontsize=9)
ax.set_xticks(range(8), labels=coordinates)  # label the columns x0 ... x7
ax.set_yticks(range(8), labels=coordinates)  # label the rows x0 ... x7
ax.grid(False)  # no grid lines across the squares
ax.set_title(r"The signs $\eta$ of the eight directions (signature (4,4))")
legend_squares = [Patch(color=RED, label="+1: space-like"),
                  Patch(color=BLUE, label="-1: time-like"),
                  Patch(color=GREY, label="0: off the diagonal")]
ax.legend(handles=legend_squares, loc="upper left", bbox_to_anchor=(1.02, 1.0))
save_figure(fig, "eta_heat_map",
            r"The signs of the author's eight directions drawn as a heat map: the "
            r"8 by 8 diagonal matrix $\eta$ with the sign of each coordinate on its "
            r"diagonal and 0 everywhere else; rows and columns are labelled by the "
            r"coordinates $x_0$ to $x_7$, and the entries are pure numbers. Red "
            r"squares are $+1$ (space-like: $x_0$, $x_1$, $x_2$, $x_3$), blue "
            r"squares $-1$ (time-like: the time $x_4$ and the extra times $x_5$, "
            r"$x_6$, $x_7$), grey squares 0. Four entries are $+1$ and four are "
            r"$-1$: the signature is (4,4).")
```

Out [3]:

```
Figure 00b.1 saved as Revision/textbook/figures/00b_1_eta_heat_map.png
```

Text cell:

```
## 6. The author's metric, written with sympy

The next cell writes the author's metric as an exact sympy matrix. `sp.symbols`
makes the symbols $x_4$, $x_0$ and $H$ (declared positive), and
`sp.Function("a4")(x4)` and `sp.Function("b4", positive=True)(x4)` make $a_4$
and $b_4$ unknown functions of the time $x_4$ ($b_4$ positive).
`sp.Rational(1, 3)` is the exact fraction $1/3$; the decimal number 0.333... would
not be exact. `sp.diag(...)` makes the diagonal matrix.

Then the cell compares the eight entries with two records:
```

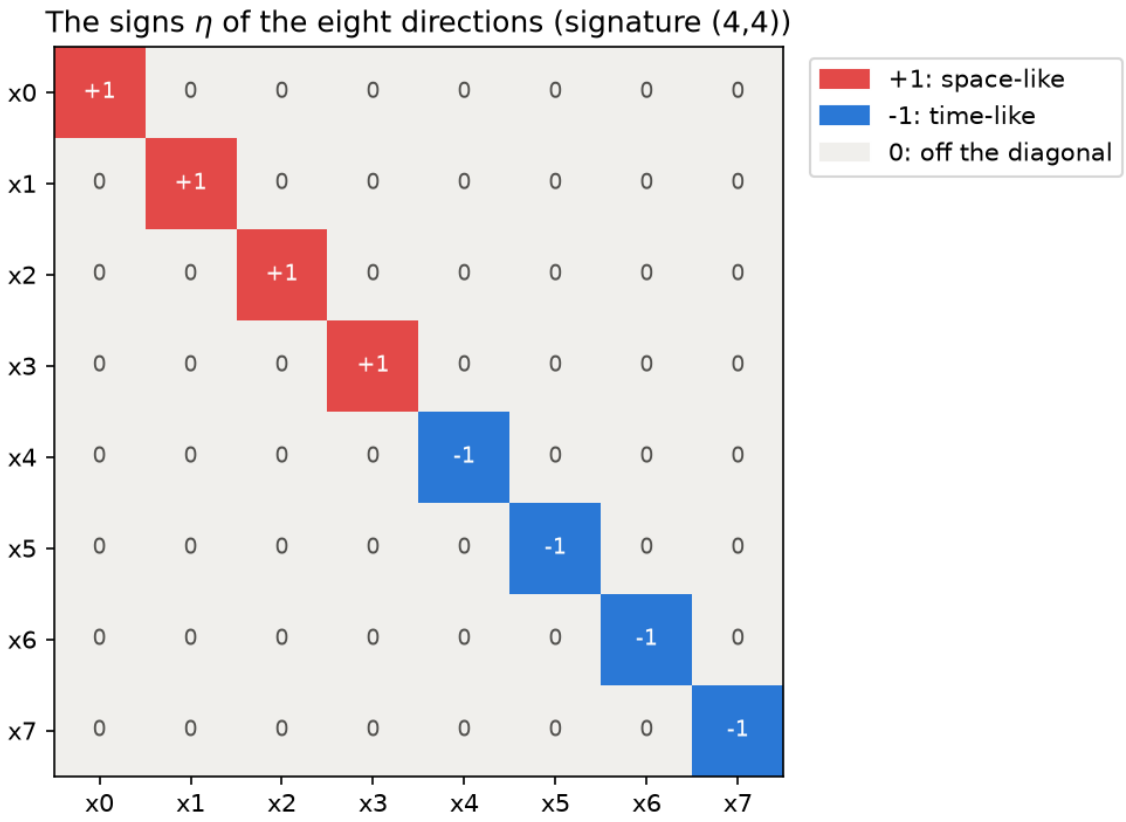

Figure 3: The signs of the author's eight directions drawn as a heat map: the 8 by 8 diagonal matrix η with the sign of each coordinate on its diagonal and 0 everywhere else; rows and columns are labelled by the coordinates x_0 to x_7, and the entries are pure numbers. Red squares are $+1$ (space-like: x_0, x_1, x_2, x_3), blue squares -1 (time-like: the time x_4 and the extra times x_5, x_6, x_7), grey squares 0. Four entries are $+1$ and four are -1: the signature is (4,4). (Notebook 00b, figure 1.)

```
- the key ‘metricDiagonal‘ of the Revision file
  Revision/metric_b4/results/metric-b4.json, the metric with $b_4$ as the
  Wolfram program of the Revision record wrote it in the notation of the program
  Mathematica, for example ‘-((b4[x4]*Sin[6*H*x0]^(1/3))/E^(2*a4[x4]))‘ for
  $-b_4 e^{-2a_4} \sin^{1/3} z$; the Wolfram verifier found it equal to the metric
  stored in the author’s notebook (its check ‘metric_equals_author_Out101‘);
- the check ‘metric_from_vielbein_equals_SPEC‘ of the Revision report
  Revision/theory/reports/python-field-theory.json, which lists the eight entries
  of the metric at $b_4 = 1$ as formulas, for example
  ‘g_x1x1 = exp(2*a4(x4))*sin(6*H*x0)**(1/3)‘.

The cell turns each formula into a sympy expression with ‘sp.sympify‘ and checks
that the eight entries come in the order $x_0, x_1, \dots, x_7$ and that the
difference of each from our entry (with $b_4 = 1$ for the second record) is
exactly zero.
```

In [4]:

```
x4, x0, H = sp.symbols("x4 x0 H", positive=True)  # time, hidden coordinate, H
a4 = sp.Function("a4")(x4)  # the metric function a4, an unknown function of x4
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0 of x4
z = 6 * H * x0  # the combination z = 6 H x0
third = sp.Rational(1, 3)  # the exact fraction 1/3
grow = sp.exp(2 * a4) * sp.sin(z) ** third  # e^(2 a4) sin^(1/3) z
shrink = b4 * sp.exp(-2 * a4) * sp.sin(z) ** third  # b4 e^(-2 a4) sin^(1/3) z
# the author’s order x0, x1, ..., x7: the hidden direction x0 first
g = sp.diag(sp.cot(z) ** 2, grow, grow, grow, -1, -shrink, -shrink, -shrink)

METRIC_B4_FILE = "Revision/metric_b4/results/metric-b4.json"
metric_b4 = json.loads(repository_file(METRIC_B4_FILE).read_text(encoding="utf-8"))
# The Mathematica texts in Python notation: a4[x4] -> a4(x4), ^ -> **, and so on.
MATHEMATICA = [("a4[x4]", "a4(x4)"), ("b4[x4]", "b4(x4)"), ("Sin[", "sin("),
               ("Cot[", "cot("), ("]", ")"), ("^", "**")]
letters = {"a4": sp.Function("a4"), "b4": sp.Function("b4", positive=True),
           "x4": x4, "x0": x0, "H": H, "E": sp.E}
different = []  # the entries whose difference from ours is not zero
for k, text in enumerate(metric_b4["metricDiagonal"]):
    for old, new in MATHEMATICA:
        text = text.replace(old, new)
    if sp.simplify(sp.sympify(text, locals=letters) - g[k, k]) != 0:
        different.append(k)
    say(f"metric-b4.json g_x{k}x{k} = {text}")
verdict, detail = recorded_check(
    "Revision/metric_b4/results/wolfram-metric-b4-report.json",
    "metric_equals_author_Out101")
check_reproduces(
    metric_b4["coordinates"] == coordinates and len(metric_b4["metricDiagonal"]) == 8
    and different == [] and verdict == "PASS",
    "our eight entries equal the metric with b4 of the Revision record, which the "
    "Wolfram verifier found equal to the metric in the author’s notebook",
    "Revision/metric_b4/results/wolfram-metric-b4-report.json, check "
    "metric_equals_author_Out101")
```

```
THEORY_REPORT = "Revision/theory/reports/python-field-theory.json"
verdict, detail = recorded_check(THEORY_REPORT, "metric_from_vielbein_equals_SPEC")
formulas = detail.split("exactly: ", 1)[1].split("; ")  # the 8 texts "g_.. = ..."
g_at_b4_1 = g.subs(b4, 1)  # the metric at b4 = 1
labels = []  # the names g_x0x0, g_x1x1, ... in the order of the record
different = []  # the entries whose difference from ours is not zero
for k, text in enumerate(formulas):
    label, formula = text.split(" = ")  # "g_x0x0" and its formula
    labels.append(label)
    recorded = sp.sympify(formula, locals=letters)  # the text as a sympy formula
    if sp.simplify(recorded - g_at_b4_1[k, k]) != 0:
        different.append(label)
in_order = [f"g_x{k}x{k}" for k in range(8)]  # g_x0x0, ..., g_x7x7
check_reproduces(
    verdict == "PASS" and labels == in_order and different == [],
    "at b4 = 1 our eight entries equal the eight entries recorded by the Revision "
    "verifier of the field theory",
    f"{THEORY_REPORT}, check metric_from_vielbein_equals_SPEC")
```

Out [4]:

```
metric-b4.json g_x0x0 = cot(6*H*x0)**2
metric-b4.json g_x1x1 = E**(2*a4(x4))*sin(6*H*x0)**(1/3)
metric-b4.json g_x2x2 = E**(2*a4(x4))*sin(6*H*x0)**(1/3)
metric-b4.json g_x3x3 = E**(2*a4(x4))*sin(6*H*x0)**(1/3)
metric-b4.json g_x4x4 = -1
metric-b4.json g_x5x5 = -((b4(x4)*sin(6*H*x0)**(1/3))/E**(2*a4(x4)))
metric-b4.json g_x6x6 = -((b4(x4)*sin(6*H*x0)**(1/3))/E**(2*a4(x4)))
metric-b4.json g_x7x7 = -((b4(x4)*sin(6*H*x0)**(1/3))/E**(2*a4(x4)))
PASS our eight entries equal the metric with b4 of the Revision record, which the Wolfram
    verifier found equal to the metric in the author's notebook
     reproduces Revision/metric_b4/results/wolfram-metric-b4-report.json, check
    metric_equals_author_Out101
PASS at b4 = 1 our eight entries equal the eight entries recorded by the Revision
    verifier of the field theory
     reproduces Revision/theory/reports/python-field-theory.json, check
    metric_from_vielbein_equals_SPEC
```

Text cell:

```
The next cell checks the signs of the eight entries numerically. It replaces $a_4$
by a plain number $a$, $b_4$ by a plain positive number $b$ and $6 H x_0$ by a
plain number $\zeta$, turns the eight entries into numpy functions of
$(a, b, \zeta)$ with ‘sp.lambdify‘, and evaluates them on a grid of 61 values of
$a$ from $-3$ to $3$ and 79 values of $\zeta$ from 0.01 to $\pi/2 - 0.01$ (4819
points), for each of the three values $b = 0.5$, $1$ and $2$ (14457 points).
‘np.sign‘ gives $+1$ for a positive number and $-1$ for a negative one. Every entry
must have the sign that the Revision record gives for its direction, at every
point.
```

In [5]:

```
# plain numbers for a4, for b4 (positive) and for 6 H x0
a, zeta = sp.symbols("a zeta", real=True)
b = sp.Symbol("b", positive=True)
# x0 = zeta / (6 H) makes 6 H x0 = zeta; sympy cancels the 6 H automatically.
entries = [g[k, k].subs({a4: a, b4: b}).subs(x0, zeta / (6 * H)) for k in range(8)]
# entry_functions[k](a, b, zeta) computes the entry g_kk with numpy.
entry_functions = [sp.lambdify((a, b, zeta), entry, "numpy") for entry in entries]

# Two arrays of shape 79 x 61: every combination of the 61 values of a and the 79
# values of zeta.
a_grid, zeta_grid = np.meshgrid(np.linspace(-3.0, 3.0, 61),
                                np.linspace(0.01, np.pi / 2 - 0.01, 79))
b_values = (0.5, 1.0, 2.0)  # three values of b4
wrong_signs = 0  # the number of (point, entry) pairs with a wrong sign
for b_value in b_values:
    for k in range(8):
        # np.full_like makes an array of the grid's shape; a constant entry such as
        # -1 is then repeated at every point.
        values = (np.full_like(a_grid, 1.0)
                  * entry_functions[k](a_grid, b_value, zeta_grid))
        wrong_signs += int(np.sum(np.sign(values) != eta[k]))
report("points of the grid", a_grid.size * len(b_values))
report("(point, entry) pairs with a sign different from eta", wrong_signs)
check_reproduces(
    wrong_signs == 0,
    "every entry has the recorded sign at all 14457 points: (+,+,+,+,-,-,-,-)",
    f"{ALGEBRA_REPORT}, check eta_in_author_order")
```

Out [5]:

```
RESULT points of the grid = 14457
RESULT (point, entry) pairs with a sign different from eta = 0
PASS every entry has the recorded sign at all 14457 points: (+,+,+,+,-,-,-,-)
     reproduces Revision/algebra/reports/wolfram-algebra.json, check eta_in_author_order
```

Text cell:

```
The next cell computes the determinant of $g$ with sympy. ‘g.det()‘ multiplies the
eight diagonal entries; sympy cancels $e^{6 a_4} e^{-6 a_4} = 1$ by itself. To
print the result in terms of $z$, the cell replaces $6 H x_0$ by the plain number
$\zeta$ of the previous cell: sympy prints $b_4^3 \sin^2 \zeta \cot^2 \zeta$, and
‘sp.trigsimp‘ (trigonometric simplification) writes this as
$b_4^3 \cos^2 \zeta$. The first check confirms, for the determinant written with
$x_0$ and $H$, that the difference from $b_4^3 \cos^2 z$ is exactly zero, and that
the Wolfram verifier of the field theory with $b_4$ (the report
Revision/theory/reports/wolfram-field-theory-b4.json) recorded PASS for the same
statement, its check ‘sqrt_det_g‘. The second check does the same at $b_4 = 1$,
where the determinant is $\cos^2 z$, against the check ‘sqrt_det_g_is_cos_z‘ of
the earlier report Revision/theory/reports/wolfram-field-theory.json. The third
check confirms that $\det g / b_4^3$ does not change with the time $x_4$: its
derivative with respect to $x_4$ is zero, although $a_4$ depends on $x_4$; the
determinant itself changes in time only through $b_4$. The fourth check reads the
```

```
verdict that the Wolfram verifier of the field equations for $a_4$ and $b_4$ (the
report Revision/field_equations_a4/reports/wolfram-a4-report.json) recorded for its
check ‘metric_is_SPEC_section_1‘: that verifier used this same metric.
```

In [6]:

```
determinant = g.det()  # the product of the eight diagonal entries
determinant_zeta = determinant.subs(x0, zeta / (6 * H))  # 6 H x0 replaced by zeta
say(f"det g = {determinant_zeta}")
say(f"    = {sp.trigsimp(determinant_zeta)}   (zeta stands for z = 6 H x0)")
WOLFRAM_THEORY_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
verdict, detail = recorded_check(WOLFRAM_THEORY_B4, "sqrt_det_g")
say(f"The Wolfram verifier recorded for sqrt_det_g: {verdict}")
check_reproduces(
    sp.simplify(determinant - b4 ** 3 * sp.cos(z) ** 2) == 0 and verdict == "PASS",
    "det g = b4^3 cos^2 z exactly, so sqrt|det g| = b4^(3/2) cos z on 0 < z < pi/2",
    f"{WOLFRAM_THEORY_B4}, check sqrt_det_g")
WOLFRAM_THEORY = "Revision/theory/reports/wolfram-field-theory.json"
verdict, detail = recorded_check(WOLFRAM_THEORY, "sqrt_det_g_is_cos_z")
check_reproduces(
    sp.simplify(determinant.subs(b4, 1) - sp.cos(z) ** 2) == 0 and verdict == "PASS",
    "at b4 = 1: det g = cos^2 z, so sqrt|det g| = cos z",
    f"{WOLFRAM_THEORY}, check sqrt_det_g_is_cos_z")
check(sp.simplify(sp.diff(determinant / b4 ** 3, x4)) == 0,
      "det g / b4^3 does not depend on the time x4: no a4 in the volume factor")
verdict, detail = recorded_check(
    "Revision/field_equations_a4/reports/wolfram-a4-report.json",
    "metric_is_SPEC_section_1")
check(verdict == "PASS",
      "the field-equation verifier recorded PASS for metric_is_SPEC_section_1")
```

Out [6]:

```
det g = b4(x4)**3*sin(zeta)**2*cot(zeta)**2
    = b4(x4)**3*cos(zeta)**2   (zeta stands for z = 6 H x0)
The Wolfram verifier recorded for sqrt_det_g: PASS
PASS det g = b4^3 cos^2 z exactly, so sqrt|det g| = b4^(3/2) cos z on 0 < z < pi/2
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check sqrt_det_g
PASS at b4 = 1: det g = cos^2 z, so sqrt|det g| = cos z
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    sqrt_det_g_is_cos_z
PASS det g / b4^3 does not depend on the time x4: no a4 in the volume factor
PASS the field-equation verifier recorded PASS for metric_is_SPEC_section_1
```

Text cell:

```
## 7. Inflation and deflation

Fix the hidden coordinate (that is, fix $z$). Then the length factor of each
direction of ordinary space is $e^{a_4}$ times the fixed number $\sin^{1/6} z$,
and that of each extra time is $b_4^{1/2} e^{-a_4}$ times the same number. This
```

```
section and the next two take $b_4 = 1$, the background of the Revision record
(a constant $b_4$ only multiplies every extra-time factor by the same number
$b_4^{1/2}$). The next cell draws $e^{a_4}$ and $e^{-a_4}$ for $a_4$ from 0 to 3,
with a logarithmic vertical axis, on which both are straight lines: the logarithm
of $e^{a_4}$ is $a_4$ and that of $e^{-a_4}$ is $-a_4$. It also draws the product
of the three factors of ordinary space and the three factors of the extra times,
$e^{3 a_4} \cdot e^{-3 a_4} = 1$ (for a general $b_4$ it is $b_4^{3/2}$), and
checks both statements.
```

In [7]:

```
a4_values = np.linspace(0.0, 3.0, 301)  # 301 values of a4 from 0 to 3
growing = np.exp(a4_values)  # the 3-space factor e^(a4)
shrinking = np.exp(-a4_values)  # the extra-time factor e^(-a4)
six_factors = growing ** 3 * shrinking ** 3  # three of each: e^(3 a4) e^(-3 a4)

ORANGE = "#eb6834"
fig, ax = plt.subplots()
ax.plot(a4_values, growing, color=BLUE, linewidth=2,
        label=r"$e^{a_4}$: ordinary space $x_1, x_2, x_3$ (inflates)")
ax.plot(a4_values, shrinking, color=ORANGE, linewidth=2, linestyle="--",
        label=r"$e^{-a_4}$: extra times $x_5, x_6, x_7$ (deflate)")
ax.plot(a4_values, six_factors, color="black", linewidth=1.5, linestyle=":",
        label=r"product of all six: $e^{3 a_4} e^{-3 a_4} = 1$")
ax.set_yscale("log")  # a logarithmic vertical axis
ax.set_xlabel(r"the metric function $a_4$ (a pure number)")
ax.set_ylabel(r"length factor divided by $\sin^{1/6} z$")
ax.set_title("As $a_4$ grows, ordinary space inflates and the extra times deflate")
# Direct labels at the right ends of the two lines (the numbers at a4 = 3).
ax.annotate(f"{growing[-1]:.2f}", (3.0, growing[-1]), xytext=(4, 0),
            textcoords="offset points", va="center")
ax.annotate(f"{shrinking[-1]:.4f}", (3.0, shrinking[-1]), xytext=(4, 0),
            textcoords="offset points", va="center")
ax.set_xlim(-0.1, 3.5)  # room on the right for the two numbers
ax.set_ylim(0.02, 150.0)  # room at the top for the legend
ax.legend(loc="upper left")
save_figure(fig, "scale_factors",
            r"The length factors of the author's metric at a fixed hidden "
            r"coordinate and $b_4 = 1$, divided by $\sin^{1/6} z$, against the "
            r"metric function "
            r"$a_4$ from 0 to 3 (horizontal axis, a pure number; vertical axis "
            r"logarithmic, a pure number). Solid blue: $e^{a_4}$, the factor of "
            r"each direction of ordinary space, which grows to $e^3 = 20.09$; "
            r"dashed orange: $e^{-a_4}$, the factor of each extra time, which "
            r"shrinks to $e^{-3} = 0.0498$; dotted black: the product of the three "
            r"space factors and the three extra-time factors, which stays exactly "
            r"1. On the logarithmic axis both exponentials are straight lines.")
report("e^(a4) at a4 = 3", f"{growing[-1]:.4f}")
report("e^(-a4) at a4 = 3", f"{shrinking[-1]:.6f}")
check(np.max(np.abs(six_factors - 1.0)) < 1e-12,
      "the product of the three inflating and three deflating factors is 1")
check(np.max(np.abs(np.log(growing) - a4_values)) < 1e-12
      and np.max(np.abs(np.log(shrinking) + a4_values)) < 1e-12,
```

```
"on a logarithmic axis e^(a4) and e^(-a4) are straight lines of slope +1, -1")
```

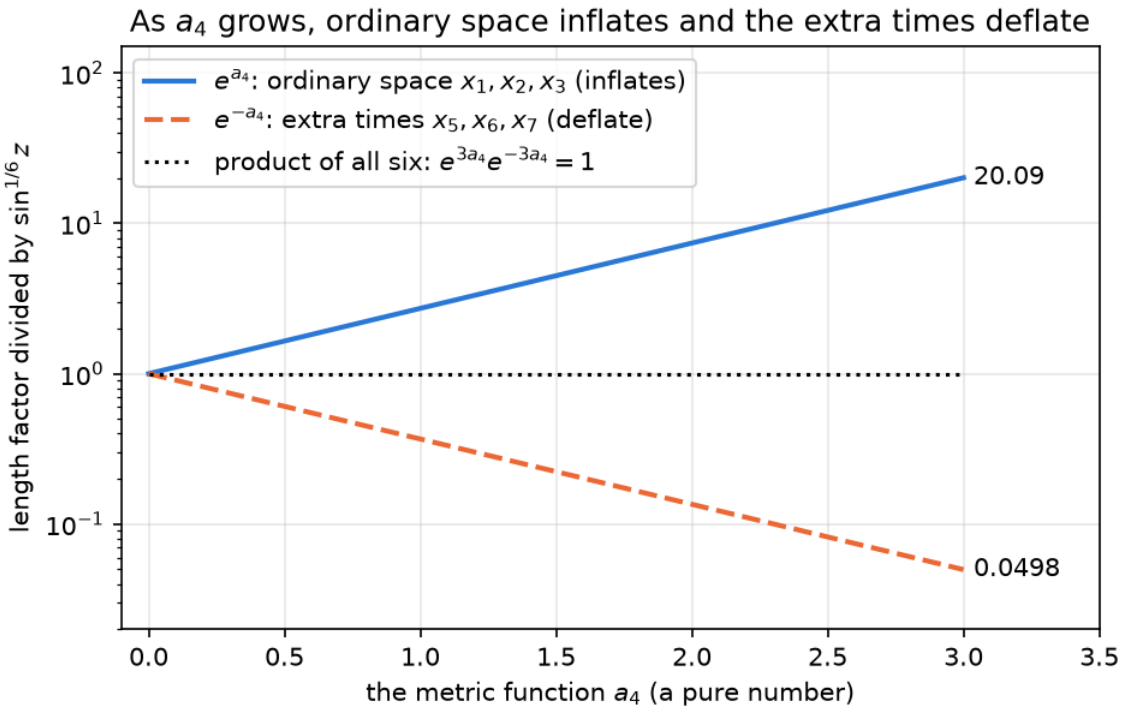

Figure 4: The length factors of the author's metric at a fixed hidden coordinate and $b_4 = 1$, divided by $\sin^{1/6} z$, against the metric function a_4 from 0 to 3 (horizontal axis, a pure number; vertical axis logarithmic, a pure number). Solid blue: e^{a_4}, the factor of each direction of ordinary space, which grows to $e^3 = 20.09$; dashed orange: e^{-a_4}, the factor of each extra time, which shrinks to $e^{-3} = 0.0498$; dotted black: the product of the three space factors and the three extra-time factors, which stays exactly 1. On the logarithmic axis both exponentials are straight lines. (Notebook 00b, figure 2.)

Out [7]:

```
Figure 00b.2 saved as Revision/textbook/figures/00b_2_scale_factors.png
RESULT e^(a4) at a4 = 3 = 20.0855
RESULT e^(-a4) at a4 = 3 = 0.049787
PASS the product of the three inflating and three deflating factors is 1
PASS on a logarithmic axis e^(a4) and e^(-a4) are straight lines of slope +1, -1
```

Text cell:

```
## 8. The eight length factors

The next cell computes all eight length factors $f_i = \sqrt{|g_{ii}|}$ from the
sympy entries made above, at the hidden coordinate $z = \pi/4$ and $b_4 = 1$, for
three values of the metric function: $a_4 = 0$, $0.5$ and $1$. At $z = \pi/4$ we have
$\sin^{1/6}(\pi/4) = 2^{-1/12} = 0.9439$ and $\cot(\pi/4) = 1$. The cell prints
```

```
the factors as a table and draws them as bars: the bars of $x_1, x_2, x_3$ grow
with $a_4$, those of $x_5, x_6, x_7$ shrink, and those of $x_0$ and $x_4$ do not
change. The product of the eight factors is $\sqrt{|\det g|} = \cos(\pi/4) =
0.7071$ for every $a_4$. The check confirms this for the three values of $a_4$
and that the Wolfram verifier of the field equations for $a_4$ recorded PASS for
its check `sqrt_abs_det_g_is_cos_z` (the volume factor is $b_4^{3/2} \cos z$, which
is $\cos z$ at $b_4 = 1$, and does not depend on $a_4$).
```

In [8]:

```
AQUA = "#1baf7a"
z_fixed = np.pi / 4  # the hidden coordinate z = pi/4
a4_choices = [0.0, 0.5, 1.0]
factors = {}  # a4 -> the list of the eight length factors f0 ... f7
for a4_value in a4_choices:
    factors[a4_value] = [float(np.sqrt(abs(entry_functions[k](a4_value, 1.0,
                                                              z_fixed))))
                         for k in range(8)]  # at b4 = 1

say("a4    " + "  ".join(f"{name:>6}" for name in coordinates) + "  product")
for a4_value in a4_choices:
    row = "  ".join(f"{value:6.4f}" for value in factors[a4_value])
    say(f"{a4_value:<4}  {row}  {np.prod(factors[a4_value]):.4f}")

fig, ax = plt.subplots()
positions = np.arange(8)  # one group of bars per direction
width = 0.26  # the width of one bar; three bars side by side in each group
for shift, a4_value, colour in zip((-1, 0, 1), a4_choices, (BLUE, ORANGE, AQUA)):
    # edgecolor="white" with linewidth 1.5 leaves a thin gap between the bars.
    ax.bar(positions + shift * width, factors[a4_value], width, color=colour,
           edgecolor="white", linewidth=1.5, label=f"$a_4 = {a4_value:g}$")
ax.set_xticks(positions, labels=coordinates)
ax.set_xlabel("direction")
ax.set_ylabel(r"length factor $f_i = \sqrt{|g_{ii}|}$ (pure number)")
ax.set_title(r"The eight length factors at $z = \pi/4$, $b_4 = 1$, for three "
             r"values of $a_4$")
ax.legend(loc="upper center")
save_figure(fig, "length_factors",
            r"The eight length factors $f_i = \sqrt{|g_{ii}|}$ of the author's "
            r"metric at the hidden coordinate $z = \pi/4$ and $b_4 = 1$ for $a_4 = 0$ "
            r"(blue), "
            r"$a_4 = 0.5$ (orange) and $a_4 = 1$ (aqua); horizontal axis the "
            r"directions $x_0$ to $x_7$, vertical axis the factor (a pure number). "
            r"The factors of ordinary space $x_1$, $x_2$, $x_3$ grow from 0.94 to "
            r"2.57 as $a_4$ grows from 0 to 1, those of the extra times $x_5$, "
            r"$x_6$, $x_7$ shrink from 0.94 to 0.35, and those of the hidden "
            r"direction $x_0$ and of the time $x_4$ stay 1. The product of the eight "
            r"factors is $\cos(\pi/4) = 0.7071$ in all three cases.")
products = [float(np.prod(factors[a4_value])) for a4_value in a4_choices]
A4_REPORT = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
verdict, detail = recorded_check(A4_REPORT, "sqrt_abs_det_g_is_cos_z")
check_reproduces(
    all(abs(p - np.cos(z_fixed)) < 1e-12 for p in products) and verdict == "PASS",
```

```
    "the product of the eight factors is cos(pi/4) = 0.7071 for a4 = 0, 0.5, 1 "
    "(b4 = 1)",
    f"{A4_REPORT}, check sqrt_abs_det_g_is_cos_z")
```

Out [8]:

```
a4         x0       x1       x2       x3       x4       x5       x6       x7  product
0.0    1.0000   0.9439   0.9439   0.9439   1.0000   0.9439   0.9439   0.9439   0.7071
0.5    1.0000   1.5562   1.5562   1.5562   1.0000   0.5725   0.5725   0.5725   0.7071
1.0    1.0000   2.5657   2.5657   2.5657   1.0000   0.3472   0.3472   0.3472   0.7071
```

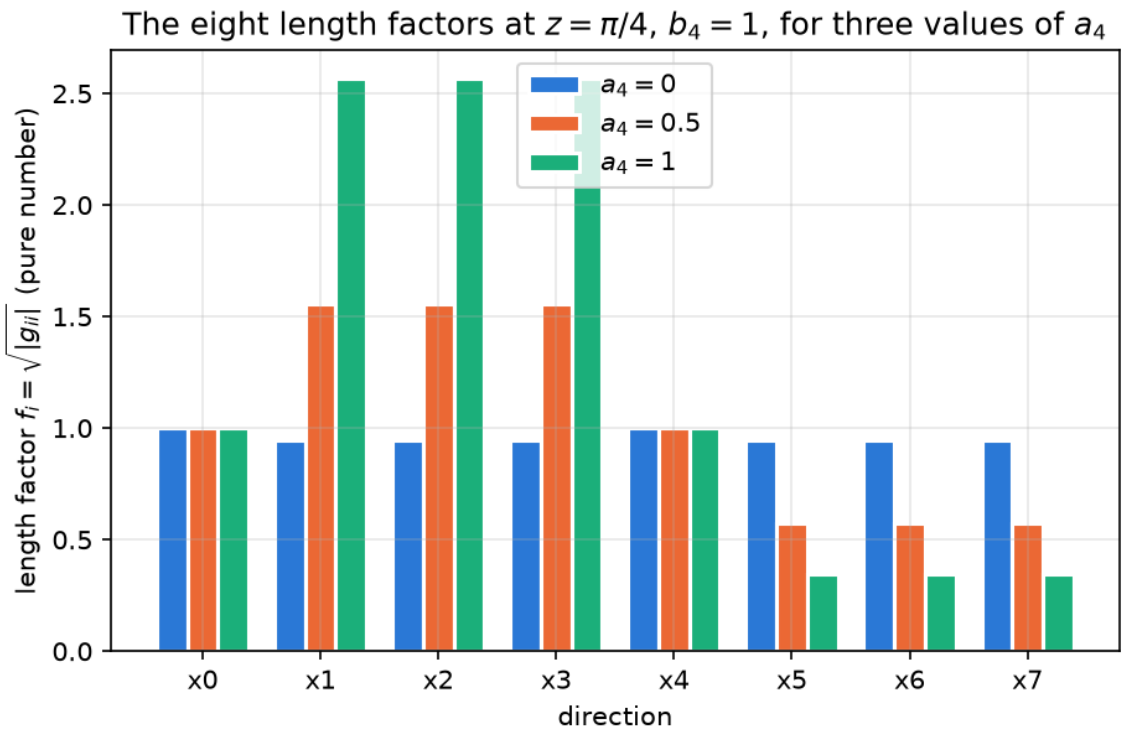

Figure 5: The eight length factors $f_i = \sqrt{|g_{ii}|}$ of the author's metric at the hidden coordinate $z = \pi/4$ and $b_4 = 1$ for $a_4 = 0$ (blue), $a_4 = 0.5$ (orange) and $a_4 = 1$ (aqua); horizontal axis the directions x_0 to x_7, vertical axis the factor (a pure number). The factors of ordinary space x_1, x_2, x_3 grow from 0.94 to 2.57 as a_4 grows from 0 to 1, those of the extra times x_5, x_6, x_7 shrink from 0.94 to 0.35, and those of the hidden direction x_0 and of the time x_4 stay 1. The product of the eight factors is $\cos(\pi/4) = 0.7071$ in all three cases. (Notebook 00b, figure 3.)

Out [8]:

```
Figure 00b.3 saved as Revision/textbook/figures/00b_3_length_factors.png
PASS the product of the eight factors is cos(pi/4) = 0.7071 for a4 = 0, 0.5, 1 (b4 = 1)
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    sqrt_abs_det_g_is_cos_z
```

Text cell:

```
## 9. The volume factor does not depend on $a_4$ (it is $b_4^{3/2} \cos z$)

The product of the eight length factors, $f_0 f_1 \cdots f_7 = \sqrt{|\det g|}$, is
the factor that turns a small coordinate box $dx_0 \, dx_1 \cdots dx_7$ into its
volume. The next cell computes it at $b_4 = 1$ for $a_4 = 0$, $1$ and $2$ at 200
values of $z$ between 0 and $\pi/2$, and draws it together with the curve
$\cos z$. The three sets of points lie on the same curve: the inflation of
ordinary space and the deflation of the extra times cancel in the volume. (To keep
the three sets of points visible, each is drawn only at every 12th value of $z$,
shifted by 4 values from the previous set.) It also computes the volume factor at
$a_4 = 1$, $b_4 = 2$ and draws it with the curve $2^{3/2} \cos z$: $b_4$ enters
only as the factor $b_4^{3/2}$. The first check compares the 600 values at
$b_4 = 1$ with $\cos z$ and confirms that the Python verifier of the field theory
recorded PASS for its check ‘sqrt_det_g_equals_cos_z‘; the second compares the 200
values at $b_4 = 2$ with $2^{3/2} \cos z$ and confirms that the Wolfram verifier of
the field theory with $b_4$ recorded PASS for its check ‘sqrt_det_g‘.
```

In [9]:

```
z_values = np.linspace(0.005, np.pi / 2 - 0.005, 200)  # 200 values inside (0, pi/2)


def volume_factor(a4_value, b4_value):
    """The product of the eight length factors at every z of z_values."""
    product = np.ones_like(z_values)
    for k in range(8):
        values = (np.full_like(z_values, 1.0)
                  * entry_functions[k](a4_value, b4_value, z_values))
        product = product * np.sqrt(np.abs(values))
    return product


volume = {a4_value: volume_factor(a4_value, 1.0) for a4_value in (0.0, 1.0, 2.0)}
volume_b4_2 = volume_factor(1.0, 2.0)  # a4 = 1, b4 = 2

fig, ax = plt.subplots()
ax.plot(z_values, np.cos(z_values), color="black", linewidth=1.5,
        label=r"$\cos z$")
ax.plot(z_values, 2.0 ** 1.5 * np.cos(z_values), color="black", linewidth=1.0,
        linestyle="--", label=r"$2^{3/2} \cos z$")
markers = {0.0: ("o", BLUE), 1.0: ("s", ORANGE), 2.0: ("^", AQUA)}
for start, a4_value in enumerate((0.0, 1.0, 2.0)):
    marker, colour = markers[a4_value]
    chosen = slice(4 * start, None, 12)  # every 12th point, shifted by 4
    ax.plot(z_values[chosen], volume[a4_value][chosen], marker, color=colour,
            markersize=6,
            label=rf"$f_0 \cdots f_7$ for $a_4 = {a4_value:g}$, $b_4 = 1$")
ax.plot(z_values[::12], volume_b4_2[::12], "D", color="#7a4fb0", markersize=5,
        label=r"$f_0 \cdots f_7$ for $a_4 = 1$, $b_4 = 2$")
ax.set_xlabel(r"hidden coordinate $z = 6 H x_0$ (radians)")
ax.set_ylabel(r"volume factor $\sqrt{|\det g|}$ (pure number)")
ax.set_title(r"The volume factor is $b_4^{3/2} \cos z$ for every value of $a_4$")
ax.legend(fontsize=8)
```

```
save_figure(fig, "volume_factor",
            r"The volume factor $\sqrt{|\det g|} = f_0 f_1 \cdots f_7$ of the "
            r"author's metric against the hidden coordinate $z = 6 H x_0$ from 0 to "
            r"$\pi/2$ (horizontal axis, in radians; vertical axis a pure number), "
            r"computed from the eight entries at $b_4 = 1$ for $a_4 = 0$ (blue "
            r"circles), $a_4 = 1$ (orange squares) and $a_4 = 2$ (aqua triangles), "
            r"with the curve $\cos z$ (black line), and at $a_4 = 1$, $b_4 = 2$ "
            r"(purple diamonds) with the curve $2^{3/2} \cos z$ (dashed). The three "
            r"sets at $b_4 = 1$ lie on $\cos z$: the volume factor does not depend "
            r"on $a_4$, because the inflation of ordinary space and the deflation "
            r"of the extra times cancel; it falls from 1 at $z = 0$ to 0 at "
            r"$z = \pi/2$. The set at $b_4 = 2$ lies on $2^{3/2} \cos z$: $b_4$ "
            r"enters only as the factor $b_4^{3/2}$.")
largest = max(float(np.max(np.abs(volume[v] - np.cos(z_values)))) for v in volume)
largest_b4_2 = float(np.max(np.abs(volume_b4_2 - 2.0 ** 1.5 * np.cos(z_values))))
report("largest difference from cos z (b4 = 1)", f"{largest:.1e}")
report("largest difference from 2^(3/2) cos z (b4 = 2)", f"{largest_b4_2:.1e}")
verdict, detail = recorded_check(THEORY_REPORT, "sqrt_det_g_equals_cos_z")
check_reproduces(
    largest < 1e-12 and verdict == "PASS",
    "at b4 = 1 the volume factor equals cos z at 200 points for a4 = 0, 1 and 2",
    f"{THEORY_REPORT}, check sqrt_det_g_equals_cos_z")
verdict, detail = recorded_check(WOLFRAM_THEORY_B4, "sqrt_det_g")
check_reproduces(
    largest_b4_2 < 1e-12 and verdict == "PASS",
    "at a4 = 1, b4 = 2 the volume factor equals 2^(3/2) cos z at 200 points",
    f"{WOLFRAM_THEORY_B4}, check sqrt_det_g")
```

Out [9]:

```
Figure 00b.4 saved as Revision/textbook/figures/00b_4_volume_factor.png
RESULT largest difference from cos z (b4 = 1) = 1.0e-15
RESULT largest difference from 2^(3/2) cos z (b4 = 2) = 2.7e-15
PASS at b4 = 1 the volume factor equals cos z at 200 points for a4 = 0, 1 and 2
     reproduces Revision/theory/reports/python-field-theory.json, check
    sqrt_det_g_equals_cos_z
PASS at a4 = 1, b4 = 2 the volume factor equals 2^(3/2) cos z at 200 points
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check sqrt_det_g
```

Text cell:

```
## 10. The last check

The last cell checks that the four figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [10]:

```
figure_names = ["00b_1_eta_heat_map.png", "00b_2_scale_factors.png",
                "00b_3_length_factors.png", "00b_4_volume_factor.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
```

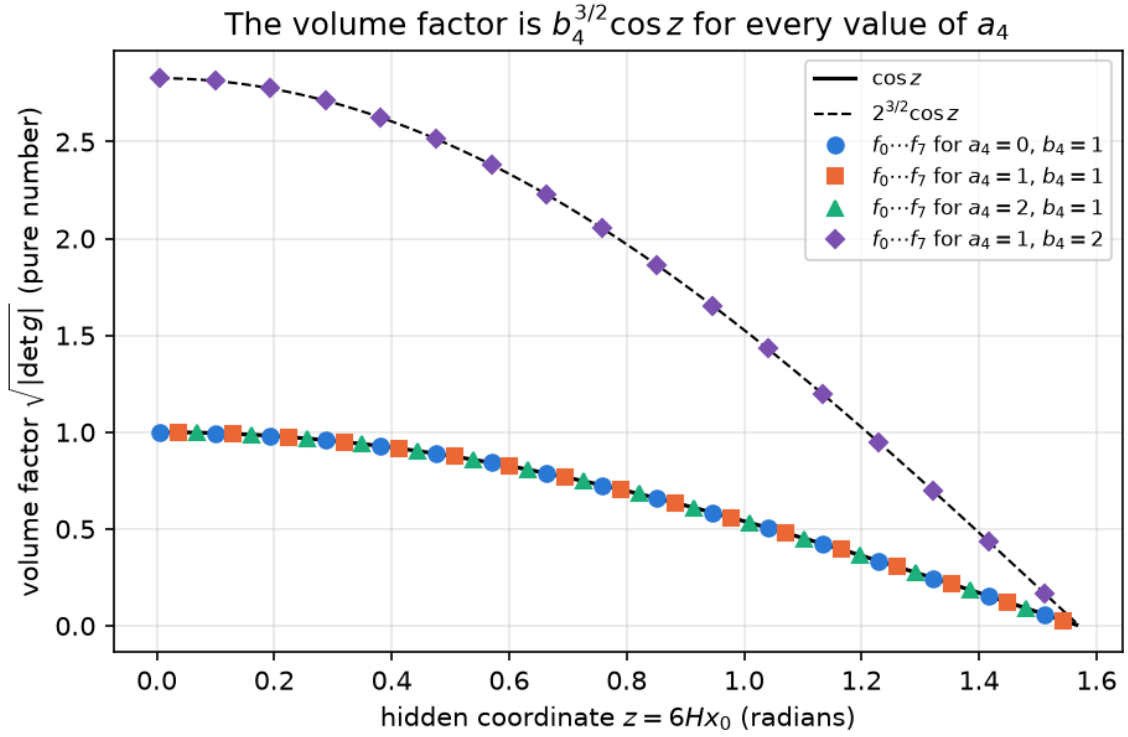

Figure 6: The volume factor $\sqrt{|\det g|} = f_0 f_1 \cdots f_7$ of the author's metric against the hidden coordinate $z = 6Hx_0$ from 0 to $\pi/2$ (horizontal axis, in radians; vertical axis a pure number), computed from the eight entries at $b_4 = 1$ for $a_4 = 0$ (blue circles), $a_4 = 1$ (orange squares) and $a_4 = 2$ (aqua triangles), with the curve $\cos z$ (black line), and at $a_4 = 1$, $b_4 = 2$ (purple diamonds) with the curve $2^{3/2}\cos z$ (dashed). The three sets at $b_4 = 1$ lie on $\cos z$: the volume factor does not depend on a_4, because the inflation of ordinary space and the deflation of the extra times cancel; it falls from 1 at $z = 0$ to 0 at $z = \pi/2$. The set at $b_4 = 2$ lies on $2^{3/2}\cos z$: b_4 enters only as the factor $b_4^{3/2}$. (Notebook 00b, figure 4.)

```
      "the four figure files of this notebook exist")
all_checks_passed()
```

Out [10]:

```
PASS the four figure files of this notebook exist
ALL 17 CHECKS PASSED (notebook 00b)
```

Text cell:

```
## 11. What this notebook showed

- The Revision record names the eight coordinates $x_0, x_1, \dots, x_7$ and gives
  their signs $(+, +, +, +, -, -, -, -)$: the hidden direction $x_0$ and ordinary
  space $x_1, x_2, x_3$ are space-like; the time $x_4$ and the extra times
  $x_5, x_6, x_7$ are time-like; the signature is (4,4).
- The author's metric with the two functions $a_4$ and $b_4$, written with sympy,
  has exactly the eight entries that the Revision verifiers recorded (with $b_4$,
  and at $b_4 = 1$), and every entry has its recorded sign at every tested point.
- When $a_4$ grows, the length factor $e^{a_4} \sin^{1/6} z$ of ordinary space
  grows (inflation) and the length factor $b_4^{1/2} e^{-a_4} \sin^{1/6} z$ of the
  extra times shrinks (deflation); if $a_4$ grows in proportion to the time and
  $b_4$ is constant, the extra times deflate exponentially.
- The determinant is $\det g = b_4^3 \cos^2 z$ exactly (PROVED by the exact
  computation), so the volume factor $\sqrt{|\det g|} = b_4^{3/2} \cos z$ does not
  depend on $a_4$: the inflation and the deflation cancel in the volume; at
  $b_4 = 1$ it is $\cos z$.
```

0.17 Line-by-line walk-through of Notebook 00b

The notebook has ten code cells, In [1] to In [10]. This section explains every line of each of them.

In [1], the set-up cell. It is the set-up cell of Notebook 00a, word for word, except for two things: its comment lines hold the run instructions of Notebook 00b (Section 0.15), and the line `NOTEBOOK_ID = "00b"` names this notebook, so that its figures are called `00b_1_eta_heat_map.png` and so on and its captions file is `Revision/textbook/figures/00b.captions.json`. Every line of its code is explained in Section 0.13. It prints one line, Set-up of notebook 00b complete: repository folder found, helpers defined.

In [2], the eight coordinates in the Revision record.

```
import sys  # sys.stdout is the channel through which the notebook prints

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
```

The module `sys` of Python gives access to the running Python itself; the notebook needs its `sys.stdout`, the channel through which `print` writes. numpy and sympy are loaded under their usual short names `np` and `sp`, as in Notebook 00a.

```
GAMMAS_FILE = "Revision/algebra/gammas.json"  # coordinates, signs and gammas
gammas_record = json.loads(repository_file(GAMMAS_FILE).read_text(encoding="utf-8"))
coordinates = gammas_record["coordinates"]  # the names x0 ... x7, in order
eta = gammas_record["eta"]  # the sign of each direction: +1 or -1
```

The first line names the record file. In the second line, `repository_file(GAMMAS_FILE)` is its full path (Section 0.13), `.read_text(encoding="utf-8")` reads the whole file as one string, and `json.loads` turns that JSON text into Python's dictionaries and lists. The file is one big dictionary; `gammas_record["coordinates"]` takes the value stored under the key `coordinates`, the list of the eight names `["x0", ..., "x7"]`, and `gammas_record["eta"]` the list of the eight signs `[1, 1, 1, 1, -1, -1, -1, -1]`. (The same file also holds the gamma matrices of Chapter 4.)

```
ROLES = {  # the role of each coordinate in the author's universe
    "x0": "hidden direction of space",
    "x1": "ordinary space (inflates)",
    "x2": "ordinary space (inflates)",
    "x3": "ordinary space (inflates)",
    "x4": "the time",
    "x5": "extra time (deflates)",
    "x6": "extra time (deflates)",
    "x7": "extra time (deflates)",
}
```

A dictionary from each coordinate name to a short description of its role, written by the author of the notebook from Section 0.5 (the record holds the names and the signs, not the roles).

```
say("coordinate  sign  kind        role")
for name, sign in zip(coordinates, eta):
    kind = "space-like" if sign > 0 else "time-like"
    # {sign:+d} writes the sign of a whole number (+1 or -1) in front of it.
    say(f"{name:10}  {sign:+d}    {kind:10}  {ROLES[name]}")
```

The first line prints the header of a table. `zip(coordinates, eta)` pairs the two lists entry by entry: `("x0", 1)`, `("x1", 1)`, and so on; the loop names the two parts of each pair `name` and `sign`. The **conditional expression** `"space-like" if sign > 0 else "time-like"` has the first value when the condition holds and the second otherwise. In the f-string, `{name:10}` pads the name to 10 characters, `{sign:+d}` writes the whole number with its sign (`+1` or `-1`), `{kind:10}` pads the kind, and `{ROLES[name]}` looks up the role. The eight lines of the table are the first eight lines of Out [2].

```
def recorded_check(path, name):
    """The verdict (PASS or FAIL) and the detail text that the Revision report
    path recorded for its check called name."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:  # every check of the report, one after the other
        if entry["name"] == name:
            return entry["verdict"].upper(), entry["detail"]
    raise KeyError(f"{path} has no check {name}")
```

A function that looks up one named check in a report of the Revision record. It reads the report as in the lines above; `data["checks"]` is the list of its checks, each a dictionary with the keys `name`,

`verdict` and `detail`. The loop compares each name with the wanted one; at the first match, `return` hands back two values at once, the verdict written in capitals (`.upper()`, because some reports write `pass` in small letters) and the detail text. If no check has that name, `raise KeyError(...)` stops the notebook with a message that names the report and the check.

```
def check_reproduces(condition, name, record):
    """check(condition, name, record=record), after sending the waiting output."""
    sys.stdout.flush()  # send every printed line that is still waiting
    check(condition, name, record=record)
```

The helper for a check that reproduces a Revision record. Python collects printed text and sends it to Jupyter in pieces; `sys.stdout.flush()` sends everything that is still waiting, so that the PASS line and the line reproduces ... that `check` prints just after it arrive together, in one piece of output, and the book's tools read them as one. Then it calls the helper `check` of the set-up cell with the record.

```
ALGEBRA_REPORT = "Revision/algebra/reports/wolfram-algebra.json"
verdict, detail = recorded_check(ALGEBRA_REPORT, "eta_in_author_order")
say(f"The Wolfram verifier recorded for eta_in_author_order: {verdict}")
```

These lines read what the Wolfram verifier of the algebra recorded for its check `eta_in_author_or der` (that the signs are $(+,+,+,+,-,-,-,-)$ in the order $x_0,\ldots,x_7$). The two returned values are named `verdict` and `detail`, and the verdict is printed: PASS (Out [2], ninth line).

```
check(coordinates == ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"],
      "the record names the eight coordinates x0 to x7 in this order")
check(eta == [1, 1, 1, 1, -1, -1, -1, -1],
      "the record gives the signs (+,+,+,+,-,-,-,-) in the order x0..x7")
check(eta.count(1) == 4 and eta.count(-1) == 4,
      "four space-like and four time-like directions: the signature is (4,4)")
check(verdict == "PASS",
      "the Wolfram verifier recorded PASS for these signs (eta_in_author_order)")
```

Four checks. Two lists are equal (`==`) when they have the same entries in the same order; so the first check confirms the names and their order, the second the signs. `eta.count(1)` counts the entries equal to 1; the third check requires four of each sign, the signature (4,4). The fourth requires the recorded verdict PASS. The four PASS lines end Out [2].

In [3], the signs as a heat map.

```
from matplotlib.colors import ListedColormap  # a colour map made of a few colours
from matplotlib.patches import Patch  # a coloured square for the legend
```

Two tools of matplotlib: a **colour map** made of a short list of colours, and a coloured square for a legend.

```
eta_matrix = np.diag(eta)  # 8 x 8: the signs on the diagonal, 0 elsewhere
BLUE, GREY, RED = "#2a78d6", "#f0efec", "#e34948"
colour_map = ListedColormap([BLUE, GREY, RED])  # three colours for -1, 0, +1
```

`np.diag(eta)` makes the 8×8 matrix η of Section 0.14 from the list of its diagonal entries. The colours are written as **hexadecimal colour codes**: after `#`, three pairs of hexadecimal digits give the amounts of red, green and blue (from 00, none, to ff, 255). The colour map has three colours, in the order in which they are used for small, middle and large numbers.

```
fig, ax = plt.subplots(figsize=(6.4, 5.4))
# vmin and vmax split the numbers into three bands: -1, 0 and +1.
ax.imshow(eta_matrix, cmap=colour_map, vmin=-1.5, vmax=1.5)
```

A figure 6.4 inches wide and 5.4 inches high. `ax.imshow` draws the matrix as a grid of coloured squares, row 1 at the top. The colour map divides the interval from `vmin` $= -1.5$ to `vmax` $= 1.5$ into three equal bands, $[-1.5, -0.5]$, $[-0.5, 0.5]$ and $[0.5, 1.5]$, and gives them the three colours in order: -1 is blue, 0 grey and $+1$ red.

```
for row in range(8):
    for column in range(8):
        value = int(eta_matrix[row, column])
        text = "0" if value == 0 else f"{value:+d}"
        colour = "#52514e" if value == 0 else "white"  # readable on each colour
        ax.text(column, row, text, ha="center", va="center", color=colour,
                fontsize=9)
```

Two loops, one inside the other, visit all 64 squares; `range(8)` gives the numbers 0 to 7. `eta_matrix[row, column]` is the entry in that row and column (counted from 0), turned into a whole number with `int`. The text is `0`, or the entry with its sign. Dark grey letters are used on the light grey squares and white letters on the coloured ones. `ax.text(column, row, ...)` writes the text centred (`ha` and `va`: horizontal and vertical alignment) in the square; the horizontal position comes first, so the column comes before the row.

```
ax.set_xticks(range(8), labels=coordinates)  # label the columns x0 ... x7
ax.set_yticks(range(8), labels=coordinates)  # label the rows x0 ... x7
ax.grid(False)  # no grid lines across the squares
ax.set_title(r"The signs $\eta$ of the eight directions (signature (4,4))")
legend_squares = [Patch(color=RED, label="+1: space-like"),
                  Patch(color=BLUE, label="-1: time-like"),
                  Patch(color=GREY, label="0: off the diagonal")]
ax.legend(handles=legend_squares, loc="upper left", bbox_to_anchor=(1.02, 1.0))
```

The columns and rows are labelled with the coordinate names, the grid of the set-up cell is switched off for this picture, and the title is set. A string written with `r` in front (a **raw string**) keeps every backslash as it is, so that matplotlib receives the mathematics `$\eta$`. The legend is made of three coloured squares; `bbox_to_anchor=(1.02, 1.0)` puts its upper left corner just to the right of the picture, so that it covers no square.

```
save_figure(fig, "eta_heat_map",
            r"The signs of the author's eight directions drawn as a heat map: the "
            r"8 by 8 diagonal matrix $\eta$ with the sign of each coordinate on its "
            r"diagonal and 0 everywhere else; rows and columns are labelled by the "
```

```
        r"coordinates $x_0$ to $x_7$, and the entries are pure numbers. Red "
        r"squares are $+1$ (space-like: $x_0$, $x_1$, $x_2$, $x_3$), blue "
        r"squares $-1$ (time-like: the time $x_4$ and the extra times $x_5$, "
        r"$x_6$, $x_7$), grey squares 0. Four entries are $+1$ and four are "
        r"$-1$: the signature is (4,4).")
```

The figure is saved as `00b_1_eta_heat_map.png` with this caption (the raw strings written next to each other are joined into one) and shown; Out [3] is the figure and the line Figure 00b.1 saved as .... What Figure 00b.1 shows: the diagonal of the matrix, from the top left, has four red squares followed by four blue squares, the pattern $(+,+,+,+,-,-,-,-)$ of Fact 1 of Section 0.14.

In [4], the author's metric written with sympy.

```
x4, x0, H = sp.symbols("x4 x0 H", positive=True)  # time, hidden coordinate, H
a4 = sp.Function("a4")(x4)  # the metric function a4, an unknown function of x4
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0 of x4
z = 6 * H * x0  # the combination z = 6 H x0
third = sp.Rational(1, 3)  # the exact fraction 1/3
```

`sp.symbols` makes three symbols at once; `positive=True` tells sympy that they stand for positive numbers, which lets it simplify more. `sp.Function("a4")` makes an unknown function called `a4`, and `(x4)` makes it a function of x_4: sympy knows nothing about it except that it depends on x_4, so every result below holds for every history $a_4(x_4)$. `sp.Function("b4", positive=True)(x4)` makes the second metric function $b_4(x_4)$ in the same way, and tells sympy that its values are positive. `z` is the expression $6Hx_0$. `sp.Rational(1, 3)` is the exact fraction $1/3$; the decimal number 0.3333 would be rounded and not exact.

```
grow = sp.exp(2 * a4) * sp.sin(z) ** third  # e^(2 a4) sin^(1/3) z
shrink = b4 * sp.exp(-2 * a4) * sp.sin(z) ** third  # b4 e^(-2 a4) sin^(1/3) z
# the author's order x0, x1, ..., x7: the hidden direction x0 first
g = sp.diag(sp.cot(z) ** 2, grow, grow, grow, -1, -shrink, -shrink, -shrink)
```

The two kinds of entry of the metric, and the diagonal 8×8 matrix `g` with the eight entries of Section 0.14 in the order $x_0, \ldots, x_7$: the hidden direction first, then ordinary space, the time and the three extra times.

```
METRIC_B4_FILE = "Revision/metric_b4/results/metric-b4.json"
metric_b4 = json.loads(repository_file(METRIC_B4_FILE).read_text(encoding="utf-8"))
```

The file `Revision/metric_b4/results/metric-b4.json` is the record of the metric with b_4, written by the Wolfram program of the Revision record; its key `metricDiagonal` holds the eight diagonal entries as texts in the notation of the program Mathematica, for example `-((b4[x4]*Sin[6*H*x0]^(1/3))/E^(2*a4[x4]))` for $-b_4 e^{-2a_4} \sin^{1/3} z$, and its key `coordinates` the names of the coordinates.

```
# The Mathematica texts in Python notation: a4[x4] -> a4(x4), ^ -> **, and so on.
MATHEMATICA = [("a4[x4]", "a4(x4)"), ("b4[x4]", "b4(x4)"), ("Sin[", "sin("),
```

```
            ("Cot[", "cot("), ("]", ")"), ("^", "**")]
letters = {"a4": sp.Function("a4"), "b4": sp.Function("b4", positive=True),
           "x4": x4, "x0": x0, "H": H, "E": sp.E}
different = []  # the entries whose difference from ours is not zero
```

`MATHEMATICA` is a list of pairs (old text, new text): Mathematica writes a function value with square brackets, `a4[x4]`, and a power with `^`; Python writes `a4(x4)` and `**`. The dictionary `letters` tells sympy what each name in the translated texts means: `a4` and `b4` are the two functions, `x4`, `x0` and `H` our symbols, and `E` the number e (`sp.E`). The empty list `different` will collect the positions of entries that differ from ours.

```
for k, text in enumerate(metric_b4["metricDiagonal"]):
    for old, new in MATHEMATICA:
        text = text.replace(old, new)
    if sp.simplify(sp.sympify(text, locals=letters) - g[k, k]) != 0:
        different.append(k)
    say(f"metric-b4.json g_x{k}x{k} = {text}")
```

`enumerate(...)` gives each recorded text together with its position `k` = 0, 1, ..., 7. The inner loop makes the replacements of `MATHEMATICA` one after the other (`.replace(old, new)` replaces every occurrence of `old`). `sp.sympify` turns the translated text into a sympy expression, reading the names with the dictionary `letters`; `g[k, k]` is our diagonal entry number `k`; `sp.simplify` simplifies the difference of the two, and if it is not exactly zero, the position is noted. The loop prints each translated text: the eight lines metric-b4.json g_x0x0 = cot(6Hx0)**2 to metric-b4.json g_x7x7 = ... of Out [4]. (In the printed formulas ‘ is the power, *the product, and /E(2*a4(x4))‘ divides by e^{2a_4}, which is the same as multiplying by e^{-2a_4}.)

```
verdict, detail = recorded_check(
    "Revision/metric_b4/results/wolfram-metric-b4-report.json",
    "metric_equals_author_Out101")
check_reproduces(
    metric_b4["coordinates"] == coordinates and len(metric_b4["metricDiagonal"]) == 8
    and different == [] and verdict == "PASS",
    "our eight entries equal the metric with b4 of the Revision record, which the "
    "Wolfram verifier found equal to the metric in the author's notebook",
    "Revision/metric_b4/results/wolfram-metric-b4-report.json, check "
    "metric_equals_author_Out101")
```

The Wolfram verifier of the metric with b_4 compared this metric, component by component, with the metric stored in the author's own notebook (the output Out[101] of `Dirac_Einstein-Lovelock_4+4_Nash.nb`) and recorded the verdict of its check `metric_equals_author_Out101`. The check requires the same coordinate names in the same order as in `gammas.json`, eight entries, no difference, and the verdict PASS; Out [4] prints the PASS line and the record it reproduces.

```
THEORY_REPORT = "Revision/theory/reports/python-field-theory.json"
verdict, detail = recorded_check(THEORY_REPORT, "metric_from_vielbein_equals_SPEC")
formulas = detail.split("exactly: ", 1)[1].split("; ")  # the 8 texts "g_.. = ..."
g_at_b4_1 = g.subs(b4, 1)  # the metric at b4 = 1
```

The Python verifier of the field theory works with the metric at $b_4 = 1$, the background of the record. It recorded, in the detail of its check `metric_from_vielbein_equals_SPEC`, the eight entries that it rebuilt from its own construction (the vielbein of Chapter 6). The detail is a sentence that ends with `exactly:` followed by the eight formulas separated by a semicolon and a blank. `detail.split("exactly: ", 1)` cuts the text at the first `exactly:` into two pieces; `[1]` keeps the second piece (Python counts from 0), and `.split("; ")` cuts it into the list of the eight texts such as `g_x1x1 = exp(2*a4(x4))*sin(6*H*x0)**(1/3)`. `g.subs(b4, 1)` is our matrix with the function b_4 replaced by the number 1.

```
labels = []  # the names g_x0x0, g_x1x1, ... in the order of the record
different = []  # the entries whose difference from ours is not zero
for k, text in enumerate(formulas):
    label, formula = text.split(" = ")  # "g_x0x0" and its formula
    labels.append(label)
    recorded = sp.sympify(formula, locals=letters)  # the text as a sympy formula
    if sp.simplify(recorded - g_at_b4_1[k, k]) != 0:
        different.append(label)
```

Two empty lists, and a loop like the one above: each text is cut at `=` into the name and the formula, the name is noted in `labels`, the formula is turned into a sympy expression, and if it differs from our entry at $b_4 = 1$, the name is noted in `different`.

```
in_order = [f"g_x{k}x{k}" for k in range(8)]  # g_x0x0, ..., g_x7x7
check_reproduces(
    verdict == "PASS" and labels == in_order and different == [],
    "at b4 = 1 our eight entries equal the eight entries recorded by the Revision "
    "verifier of the field theory",
    f"{THEORY_REPORT}, check metric_from_vielbein_equals_SPEC")
```

`range(8)` gives 0 to 7, so `in_order` is the list of the names `g_x0x0` to `g_x7x7`. The check requires the recorded verdict PASS, the names in the order $x_0, \ldots, x_7$, and no difference; Out [4] ends with the PASS line and the line reproduces Revision/theory/reports/python-field-theory.json, check metric_from_vielbein_equals_SPEC.

In [5], the signs at 14457 points.

```
# plain numbers for a4, for b4 (positive) and for 6 H x0
a, zeta = sp.symbols("a zeta", real=True)
b = sp.Symbol("b", positive=True)
# x0 = zeta / (6 H) makes 6 H x0 = zeta; sympy cancels the 6 H automatically.
entries = [g[k, k].subs({a4: a, b4: b}).subs(x0, zeta / (6 * H)) for k in range(8)]
# entry_functions[k](a, b, zeta) computes the entry g_kk with numpy.
entry_functions = [sp.lambdify((a, b, zeta), entry, "numpy") for entry in entries]
```

Three new symbols, a for a value of a_4, b for a (positive) value of b_4, and ζ (zeta) for a value of z. `.subs({a4: a, b4: b})` replaces the two functions by the plain symbols, and `.subs(x0, zeta / (6 * H))` replaces x_0 by $\zeta/(6H)$, so that $z = 6Hx_0$ becomes ζ. The list `entries` holds the eight diagonal entries as formulas in a, b and ζ alone. `sp.lambdify((a, b, zeta), entry, "numpy")` turns a formula into an ordinary Python function of three numbers that computes it with numpy, fast and for whole arrays at once.

```
# Two arrays of shape 79 x 61: every combination of the 61 values of a and the 79
# values of zeta.
a_grid, zeta_grid = np.meshgrid(np.linspace(-3.0, 3.0, 61),
                                np.linspace(0.01, np.pi / 2 - 0.01, 79))
b_values = (0.5, 1.0, 2.0)  # three values of b4
wrong_signs = 0  # the number of (point, entry) pairs with a wrong sign
```

`np.linspace(-3.0, 3.0, 61)` gives 61 equally spaced values of a from -3 to 3 (steps of 0.1) and the second `np.linspace` 79 values of ζ inside the patch, from 0.01 to $\pi/2 - 0.01$. `np.meshgrid` makes two tables of 79 rows and 61 columns: in `a_grid` every row is the list of the 61 values of a, in `zeta_grid` every column is the list of the 79 values of ζ; read together, the two tables list all $61 \cdot 79 = 4819$ pairs (a, ζ). The grid is used for each of the three values $b = 0.5$, 1 and 2, so $3 \cdot 4819 = 14457$ points are tested. The counter of wrong signs starts at 0.

```
for b_value in b_values:
    for k in range(8):
        # np.full_like makes an array of the grid's shape; a constant entry such as
        # -1 is then repeated at every point.
        values = (np.full_like(a_grid, 1.0)
                  * entry_functions[k](a_grid, b_value, zeta_grid))
        wrong_signs += int(np.sum(np.sign(values) != eta[k]))
```

For each value of b and each entry k, `entry_functions[k](a_grid, b_value, zeta_grid)` computes the entry at all 4819 points of the grid at once. For the constant entry $g_{44} = -1$ the function returns the single number -1; multiplying by `np.full_like(a_grid, 1.0)`, a table of the grid's shape filled with 1.0, makes every result a table of 4819 values. `np.sign` gives $+1$ for a positive and -1 for a negative number; `!= eta[k]` compares each sign with the recorded sign of the direction, giving a table of `True` (wrong) and `False` (right); `np.sum` counts the `True` entries, and `+=` adds the count to the counter.

```
report("points of the grid", a_grid.size * len(b_values))
report("(point, entry) pairs with a sign different from eta", wrong_signs)
check_reproduces(
    wrong_signs == 0,
    "every entry has the recorded sign at all 14457 points: (+,+,+,+,-,-,-,-)",
    f"{ALGEBRA_REPORT}, check eta_in_author_order")
```

`a_grid.size` is the number of entries of the table, 4819, and `len(b_values)` is 3. Out [5] prints 14457, then 0 wrong signs, then the PASS line, which reproduces the Wolfram check `eta_in_author_order`. This is the COMPUTED confirmation of Fact 1 of Section 0.14.

In [6], the determinant.

```
determinant = g.det()  # the product of the eight diagonal entries
determinant_zeta = determinant.subs(x0, zeta / (6 * H))  # 6 H x0 replaced by zeta
say(f"det g = {determinant_zeta}")
say(f"      = {sp.trigsimp(determinant_zeta)}   (zeta stands for z = 6 H x0)")
```

`g.det()` computes the determinant of the sympy matrix exactly; sympy multiplies the eight entries and cancels $e^{6a_4}e^{-6a_4}$ by itself, as in Fact 4 of Section 0.14. For a short printout $6Hx_0$ is replaced by

ζ; sympy prints `b4(x4)**3*sin(zeta)**2*cot(zeta)**2`, and `sp.trigsimp` (simplification with the rules of trigonometry) turns it into `b4(x4)**3*cos(zeta)**2`. These are the first two lines of Out [6], the last two lines of the derivation of Fact 4.

```
WOLFRAM_THEORY_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
verdict, detail = recorded_check(WOLFRAM_THEORY_B4, "sqrt_det_g")
say(f"The Wolfram verifier recorded for sqrt_det_g: {verdict}")
check_reproduces(
    sp.simplify(determinant - b4 ** 3 * sp.cos(z) ** 2) == 0 and verdict == "PASS",
    "det g = b4^3 cos^2 z exactly, so sqrt|det g| = b4^(3/2) cos z on 0 < z < pi/2",
    f"{WOLFRAM_THEORY_B4}, check sqrt_det_g")
```

The verdict of the Wolfram verifier of the field theory with b_4 for the same statement is read and printed (PASS). The check requires that the difference between our determinant and $b_4^3 \cos^2 z$, written with x_0 and H, simplifies to exactly 0, and that the Wolfram verifier recorded PASS.

```
WOLFRAM_THEORY = "Revision/theory/reports/wolfram-field-theory.json"
verdict, detail = recorded_check(WOLFRAM_THEORY, "sqrt_det_g_is_cos_z")
check_reproduces(
    sp.simplify(determinant.subs(b4, 1) - sp.cos(z) ** 2) == 0 and verdict == "PASS",
    "at b4 = 1: det g = cos^2 z, so sqrt|det g| = cos z",
    f"{WOLFRAM_THEORY}, check sqrt_det_g_is_cos_z")
```

The same at $b_4 = 1$: the determinant with the function b_4 replaced by 1 must be $\cos^2 z$, and the Wolfram verifier of the field theory at $b_4 = 1$ must have recorded PASS for its check `sqrt_det_g_is_cos_z`.

```
check(sp.simplify(sp.diff(determinant / b4 ** 3, x4)) == 0,
      "det g / b4^3 does not depend on the time x4: no a4 in the volume factor")
verdict, detail = recorded_check(
    "Revision/field_equations_a4/reports/wolfram-a4-report.json",
    "metric_is_SPEC_section_1")
check(verdict == "PASS",
      "the field-equation verifier recorded PASS for metric_is_SPEC_section_1")
```

`sp.diff(determinant / b4 ** 3, x4)` is the derivative of $\det g / b_4^3$ with respect to the time; after simplification it is exactly zero, although the entries contain the unknown function $a_4(x_4)$: the determinant changes in time only through b_4. The last check reads the verdict that the Wolfram verifier of the field equations for a_4 and b_4 recorded for its check that it used exactly this metric. Out [6] ends with four PASS lines.

In [7], inflation and deflation.

```
a4_values = np.linspace(0.0, 3.0, 301)  # 301 values of a4 from 0 to 3
growing = np.exp(a4_values)  # the 3-space factor e^(a4)
shrinking = np.exp(-a4_values)  # the extra-time factor e^(-a4)
six_factors = growing ** 3 * shrinking ** 3  # three of each: e^(3 a4) e^(-3 a4)
```

301 values of a_4 from 0 to 3 in steps of 0.01; at each, the factor e^{a_4} of a direction of ordinary space and the factor e^{-a_4} of an extra time at $b_4 = 1$ (both divided by the common number $\sin^{1/6} z$; a constant b_4 would only multiply the second by $b_4^{1/2}$), and the product of three of each.

```
ORANGE = "#eb6834"
fig, ax = plt.subplots()
ax.plot(a4_values, growing, color=BLUE, linewidth=2,
        label=r"$e^{a_4}$: ordinary space $x_1, x_2, x_3$ (inflates)")
ax.plot(a4_values, shrinking, color=ORANGE, linewidth=2, linestyle="--",
        label=r"$e^{-a_4}$: extra times $x_5, x_6, x_7$ (deflate)")
ax.plot(a4_values, six_factors, color="black", linewidth=1.5, linestyle=":",
        label=r"product of all six: $e^{3 a_4} e^{-3 a_4} = 1$")
```

A new colour, a new figure, and three curves: a solid blue line, a dashed orange line (the line style of two hyphens) and a dotted black line; `linewidth` is the thickness of a line in points.

```
ax.set_yscale("log")  # a logarithmic vertical axis
ax.set_xlabel(r"the metric function $a_4$ (a pure number)")
ax.set_ylabel(r"length factor divided by $\sin^{1/6} z$")
ax.set_title("As $a_4$ grows, ordinary space inflates and the extra times deflate")
```

`ax.set_yscale("log")` makes the vertical axis **logarithmic**: equal distances on it mean equal factors (0.1, 1, 10, 100). On such an axis the height of the point of e^{a_4} is proportional to $\ln e^{a_4} = a_4$, so the curve is a straight line rising with slope 1 (per factor e), and e^{-a_4} is a straight line falling with slope -1. The other lines label the axes and set the title.

```
# Direct labels at the right ends of the two lines (the numbers at a4 = 3).
ax.annotate(f"{growing[-1]:.2f}", (3.0, growing[-1]), xytext=(4, 0),
            textcoords="offset points", va="center")
ax.annotate(f"{shrinking[-1]:.4f}", (3.0, shrinking[-1]), xytext=(4, 0),
            textcoords="offset points", va="center")
ax.set_xlim(-0.1, 3.5)  # room on the right for the two numbers
ax.set_ylim(0.02, 150.0)  # room at the top for the legend
ax.legend(loc="upper left")
```

`growing[-1]` is the last entry of the array, the value at $a_4 = 3$ (a negative index counts from the end). `ax.annotate` writes the number next to the point $(3, e^3)$, shifted 4 points to the right (`xytext=(4, 0)` with `textcoords="offset points"`); likewise for e^{-3}. The limits of the axes leave room for these numbers and for the legend.

```
save_figure(fig, "scale_factors",
            r"The length factors of the author's metric at a fixed hidden "
            r"coordinate and $b_4 = 1$, divided by $\sin^{1/6} z$, against the "
            r"metric function "
            r"$a_4$ from 0 to 3 (horizontal axis, a pure number; vertical axis "
            r"logarithmic, a pure number). Solid blue: $e^{a_4}$, the factor of "
            r"each direction of ordinary space, which grows to $e^3 = 20.09$; "
            r"dashed orange: $e^{-a_4}$, the factor of each extra time, which "
```

```
            r"shrinks to $e^{-3} = 0.0498$; dotted black: the product of the three "
            r"space factors and the three extra-time factors, which stays exactly "
            r"1. On the logarithmic axis both exponentials are straight lines.")
report("e^(a4) at a4 = 3", f"{growing[-1]:.4f}")
report("e^(-a4) at a4 = 3", f"{shrinking[-1]:.6f}")
```

The figure is saved as `00b_2_scale_factors.png`, and the two values at $a_4 = 3$ are printed: $e^3 = 20.0855$ and $e^{-3} = 0.049787$ (Out [7]). What Figure 00b.2 shows: two straight lines on the logarithmic axis, one rising and one falling at the same rate, mirror images of each other in the horizontal line at 1, and the flat dotted product at 1. This is Fact 3 of Section 0.14 in a picture: whatever ordinary space gains, the extra times lose.

```
check(np.max(np.abs(six_factors - 1.0)) < 1e-12,
      "the product of the three inflating and three deflating factors is 1")
check(np.max(np.abs(np.log(growing) - a4_values)) < 1e-12
      and np.max(np.abs(np.log(shrinking) + a4_values)) < 1e-12,
      "on a logarithmic axis e^(a4) and e^(-a4) are straight lines of slope +1, -1")
```

The first check requires the product to be 1 at all 301 points up to 10^{-12} (rounding). The second uses `np.log`, the natural logarithm: $\ln e^{a_4} = a_4$ and $\ln e^{-a_4} = -a_4$ at all points, which is exactly the statement that the two curves are straight lines on the logarithmic axis.

In [8], the eight length factors.

```
AQUA = "#1baf7a"
z_fixed = np.pi / 4  # the hidden coordinate z = pi/4
a4_choices = [0.0, 0.5, 1.0]
factors = {}  # a4 -> the list of the eight length factors f0 ... f7
for a4_value in a4_choices:
    factors[a4_value] = [float(np.sqrt(abs(entry_functions[k](a4_value, 1.0,
                                                              z_fixed))))
                         for k in range(8)]  # at b4 = 1
```

A third colour, the fixed hidden coordinate $z = \pi/4$, and three values of a_4. For each value the list of the eight length factors $f_i = \sqrt{|g_{ii}|}$ of Fact 2 is computed at $b_4 = 1$ (the middle argument 1.0) with the numpy functions of In [5] (`abs` is the absolute value, `np.sqrt` the square root, `float` makes a plain Python number) and stored in the dictionary `factors` under the value of a_4.

```
say("a4    " + "  ".join(f"{name:>6}" for name in coordinates) + "  product")
for a4_value in a4_choices:
    row = "  ".join(f"{value:6.4f}" for value in factors[a4_value])
    say(f"{a4_value:<4}  {row}  {np.prod(factors[a4_value]):.4f}")
```

The table of Out [8]. `" ".join(...)` joins texts with two blanks between them; `{name:>6}` writes a name right-aligned in 6 characters, `{value:6.4f}` a number with 4 decimals in 6 characters, and `{a4_value:<4}` the value of a_4 left-aligned in 4 characters. `np.prod` multiplies all entries of a list. The printed rows are the numbers of Section 0.14: the factors of x_0 and x_4 are 1.0000; the other six are 0.9439 at $a_4 = 0$; 1.5562 and 0.5725 at $a_4 = 0.5$; 2.5657 and 0.3472 at $a_4 = 1$; and the product is 0.7071 in each row.

```
fig, ax = plt.subplots()
positions = np.arange(8)  # one group of bars per direction
width = 0.26  # the width of one bar; three bars side by side in each group
for shift, a4_value, colour in zip((-1, 0, 1), a4_choices, (BLUE, ORANGE, AQUA)):
    # edgecolor="white" with linewidth 1.5 leaves a thin gap between the bars.
    ax.bar(positions + shift * width, factors[a4_value], width, color=colour,
           edgecolor="white", linewidth=1.5, label=f"$a_4 = {a4_value:g}$")
```

A bar chart. `np.arange(8)` is the array 0, 1, ..., 7, one position per direction. For each value of a_4 the bars are shifted by -1, 0 or $+1$ bar widths, so that the three bars of a direction stand side by side. `ax.bar(positions, heights, width, ...)` draws one bar per position; `{a4_value:g}` writes the number in its shortest form (0, 0.5, 1).

```
ax.set_xticks(positions, labels=coordinates)
ax.set_xlabel("direction")
ax.set_ylabel(r"length factor $f_i = \sqrt{|g_{ii}|}$ (pure number)")
ax.set_title(r"The eight length factors at $z = \pi/4$, $b_4 = 1$, for three "
             r"values of $a_4$")
ax.legend(loc="upper center")
save_figure(fig, "length_factors",
            r"The eight length factors $f_i = \sqrt{|g_{ii}|}$ of the author's "
            r"metric at the hidden coordinate $z = \pi/4$ and $b_4 = 1$ for $a_4 = 0$ "
            r"(blue), "
            r"$a_4 = 0.5$ (orange) and $a_4 = 1$ (aqua); horizontal axis the "
            r"directions $x_0$ to $x_7$, vertical axis the factor (a pure number). "
            r"The factors of ordinary space $x_1$, $x_2$, $x_3$ grow from 0.94 to "
            r"2.57 as $a_4$ grows from 0 to 1, those of the extra times $x_5$, "
            r"$x_6$, $x_7$ shrink from 0.94 to 0.35, and those of the hidden "
            r"direction $x_0$ and of the time $x_4$ stay 1. The product of the eight "
            r"factors is $\cos(\pi/4) = 0.7071$ in all three cases.")
```

Labels, title, legend, and the figure `00b_3_length_factors.png`. What Figure 00b.3 shows: over x_0 and x_4 all three bars have the height 1; over x_1, x_2, x_3 the bars rise from blue to orange to aqua; over x_5, x_6, x_7 they fall.

```
products = [float(np.prod(factors[a4_value])) for a4_value in a4_choices]
A4_REPORT = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
verdict, detail = recorded_check(A4_REPORT, "sqrt_abs_det_g_is_cos_z")
check_reproduces(
    all(abs(p - np.cos(z_fixed)) < 1e-12 for p in products) and verdict == "PASS",
    "the product of the eight factors is cos(pi/4) = 0.7071 for a4 = 0, 0.5, 1 "
    "(b4 = 1)",
    f"{A4_REPORT}, check sqrt_abs_det_g_is_cos_z")
```

The three products, and the verdict of the Wolfram check `sqrt_abs_det_g_is_cos_z` of the field equations for a_4 and b_4 (it states $\sqrt{|\det g|} = b_4^{3/2}\cos z$, which is $\cos z$ at $b_4 = 1$). `all(...)` is true when the condition holds for every product: each must equal $\cos(\pi/4)$ up to 10^{-12}. Out [8] ends with the PASS line and the record it reproduces.

In [9], the volume factor for three values of a_4 and two values of b_4.

```
z_values = np.linspace(0.005, np.pi / 2 - 0.005, 200)  # 200 values inside (0, pi/2)
```

200 values of z inside the patch.

```
def volume_factor(a4_value, b4_value):
    """The product of the eight length factors at every z of z_values."""
    product = np.ones_like(z_values)
    for k in range(8):
        values = (np.full_like(z_values, 1.0)
                  * entry_functions[k](a4_value, b4_value, z_values))
        product = product * np.sqrt(np.abs(values))
    return product
```

A function of a value of a_4 and a value of b_4. `product` starts as an array of 200 ones (`np.ones_like`) and is multiplied in turn by the eight length factors, computed at all 200 values of z at once (the 1.0-array again makes the constant entry an array). The result is the volume factor $f_0 \cdots f_7$ at every z.

```
volume = {a4_value: volume_factor(a4_value, 1.0) for a4_value in (0.0, 1.0, 2.0)}
volume_b4_2 = volume_factor(1.0, 2.0)  # a4 = 1, b4 = 2
```

A **dictionary comprehension** (a dictionary written with a `for` inside its braces) stores the volume factor at $b_4 = 1$ under each of $a_4 = 0$, 1 and 2; `volume_b4_2` is the volume factor at $a_4 = 1$, $b_4 = 2$.

```
fig, ax = plt.subplots()
ax.plot(z_values, np.cos(z_values), color="black", linewidth=1.5,
        label=r"$\cos z$")
ax.plot(z_values, 2.0 ** 1.5 * np.cos(z_values), color="black", linewidth=1.0,
        linestyle="--", label=r"$2^{3/2} \cos z$")
markers = {0.0: ("o", BLUE), 1.0: ("s", ORANGE), 2.0: ("^", AQUA)}
for start, a4_value in enumerate((0.0, 1.0, 2.0)):
    marker, colour = markers[a4_value]
    chosen = slice(4 * start, None, 12)  # every 12th point, shifted by 4
    ax.plot(z_values[chosen], volume[a4_value][chosen], marker, color=colour,
            markersize=6,
            label=rf"$f_0 \cdots f_7$ for $a_4 = {a4_value:g}$, $b_4 = 1$")
ax.plot(z_values[::12], volume_b4_2[::12], "D", color="#7a4fb0", markersize=5,
        label=r"$f_0 \cdots f_7$ for $a_4 = 1$, $b_4 = 2$")
```

The solid black curve $\cos z$ and the dashed curve $2^{3/2} \cos z$, then the computed points. The dictionary `markers` gives each value of a_4 a marker shape (`"o"` circles, `"s"` squares, `"^"` triangles) and a colour. `slice(4 * start, None, 12)` selects the entries number `4 * start`, `4 * start + 12`, `4 * star t + 24`, and so on to the end: every 12th point, starting at 0, 4 or 8, so that the three sets of markers do not hide each other. A string with both `r` and `f` in front is a raw f-string. The last `ax.plot` draws every 12th point of the volume factor at $b_4 = 2$ as purple diamonds (`"D"`; `[::12]` takes every 12th entry from the first).

```
ax.set_xlabel(r"hidden coordinate $z = 6 H x_0$ (radians)")
ax.set_ylabel(r"volume factor $\sqrt{|\det g|}$ (pure number)")
```

```
ax.set_title(r"The volume factor is $b_4^{3/2} \cos z$ for every value of $a_4$")
ax.legend(fontsize=8)
save_figure(fig, "volume_factor",
            r"The volume factor $\sqrt{|\det g|} = f_0 f_1 \cdots f_7$ of the "
            r"author's metric against the hidden coordinate $z = 6 H x_0$ from 0 to "
            r"$\pi/2$ (horizontal axis, in radians; vertical axis a pure number), "
            r"computed from the eight entries at $b_4 = 1$ for $a_4 = 0$ (blue "
            r"circles), $a_4 = 1$ (orange squares) and $a_4 = 2$ (aqua triangles), "
            r"with the curve $\cos z$ (black line), and at $a_4 = 1$, $b_4 = 2$ "
            r"(purple diamonds) with the curve $2^{3/2} \cos z$ (dashed). The three "
            r"sets at $b_4 = 1$ lie on $\cos z$: the volume factor does not depend "
            r"on $a_4$, because the inflation of ordinary space and the deflation "
            r"of the extra times cancel; it falls from 1 at $z = 0$ to 0 at "
            r"$z = \pi/2$. The set at $b_4 = 2$ lies on $2^{3/2} \cos z$: $b_4$ "
            r"enters only as the factor $b_4^{3/2}$.")
```

Labels, title, legend (in a smaller font, `fontsize=8`) and the figure `00b_4_volume_factor.png`. What Figure 00b.4 shows: circles, squares and triangles all sit on the one black curve $\cos z$, although the three values of a_4 stretch ordinary space by the factors 1, $e = 2.72$ and $e^2 = 7.39$; the diamonds sit on the dashed curve $2^{3/2} \cos z$, 2.83 times higher. This is Fact 4 of Section 0.14.

```
largest = max(float(np.max(np.abs(volume[v] - np.cos(z_values)))) for v in volume)
largest_b4_2 = float(np.max(np.abs(volume_b4_2 - 2.0 ** 1.5 * np.cos(z_values))))
report("largest difference from cos z (b4 = 1)", f"{largest:.1e}")
report("largest difference from 2^(3/2) cos z (b4 = 2)", f"{largest_b4_2:.1e}")
```

For each value of a_4 the largest distance of the 200 computed values from $\cos z$, and `max(...)` takes the largest of the three; likewise the largest distance of the values at $b_4 = 2$ from $2^{3/2} \cos z$. Out [9] prints 1.0×10^{-15} and 2.7×10^{-15}, a few times the machine epsilon of Section 0.22: only rounding.

```
verdict, detail = recorded_check(THEORY_REPORT, "sqrt_det_g_equals_cos_z")
check_reproduces(
    largest < 1e-12 and verdict == "PASS",
    "at b4 = 1 the volume factor equals cos z at 200 points for a4 = 0, 1 and 2",
    f"{THEORY_REPORT}, check sqrt_det_g_equals_cos_z")
verdict, detail = recorded_check(WOLFRAM_THEORY_B4, "sqrt_det_g")
check_reproduces(
    largest_b4_2 < 1e-12 and verdict == "PASS",
    "at a4 = 1, b4 = 2 the volume factor equals 2^(3/2) cos z at 200 points",
    f"{WOLFRAM_THEORY_B4}, check sqrt_det_g")
```

Two checks: at $b_4 = 1$ against the verdict of the Python verifier's check `sqrt_det_g_equals_cos_z`, and at $b_4 = 2$ against the verdict of the Wolfram verifier's check `sqrt_det_g` of the field theory with b_4.

In [10], the last check.

```
figure_names = ["00b_1_eta_heat_map.png", "00b_2_scale_factors.png",
                "00b_3_length_factors.png", "00b_4_volume_factor.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "the four figure files of this notebook exist")
all_checks_passed()
```

The four figure files must exist in the folder of the figures (in a check run of nbkit, in its scratch folder, which `output_file` points to). The last line prints ALL 17 CHECKS PASSED (notebook 00b): four checks in In [2], two in In [4], one in In [5], four in In [6], two in In [7], one in In [8], two in In [9] and one in In [10].

0.18 The honesty ledger, and how a report is tied to its files

The **honesty ledger** is the table of the main statements of the book, each with its label (Section 0.3), a short note where one is needed, and the reports of the Revision record that verify it. It is the book's promise in one page: whatever a later chapter says, its label is the one given here, and every reader can open the named reports and read the verdicts (Section 0.4, level 2). The ledger below has twenty rows. Every one of the 51 verifier reports of the Revision record belongs to exactly one row, and the five rows labelled HYPOTHESIS or OPEN have no report at all, because nothing in the record establishes them. Notebook 00c checks all of this and counts the checks of each row; the ledger describes the record on the date of the notebook's last verified run, which its provenance file records. (Reports in the folder `Revision/kohn_sham/reports` are written below with the short folder name `kohn_sham/reports`, and so on; every path starts in the folder `Revision`.)

statement (with its row number)	**label**	**verified in (folder Revision)**
(1) the gamma matrices, C, Γ, B; Pin(4,4) and Spin(4,4) (Chapters 4 and 5)	PROVED	`algebra/reports/wolfram-algebra.json`, `algebra/reports/python-algebra.json`
(2) the Lagrangians, field equations and energy-momentum tensor of both fields, and the canonical quantisation of dirac16complex (dirac16complex00 is a classical field and is not quantised); the geometry, the Dirac operator and the energy-momentum tensor also for a changing b_4 (Chapters 6, 7, 9 and 10)	PROVED; for a self-coupling $\lambda \neq 0$ the operator form of the energy-momentum tensor is proved only in a finite model of one set of 16 modes, and for the field on a whole slice of space it is OPEN (the field-equations record of Chapter 12 ASSUMES it)	`theory/reports/wolfram-field-theory.json`, `theory/reports/python-field-theory.json`, `theory/fock_quartic/reports/fock-quartic.json`, `theory/reports/wolfram-field-theory-b4.json`, `theory/reports/python-field-theory-b4.json`, `theory/reports/wolfram-field-theory-b4-emt-C.json`, `theory/reports/wolfram-field-theory-b4-emt-G.json`
(3) the exact scope of the non-triviality (Chapter 8)	PROVED	`theory/reports/wolfram-scope.json`, `theory/reports/python-scope.json`

statement (with its row number)	label	verified in (folder Revision)
(4) the conservation identities of the energy-momentum tensor; the spin connection (Chapters 6 and 9)	PROVED	lead_checks/reports/emt-divergence-and-spin-connection.json
(5) the field equations for a_4 and b_4; every vacuum history has b_4 constant and $a_4 = \pm H x_4$ plus a constant (Chapter 12)	PROVED	field_equations_a4/reports/wolfram-a4-report.json, field_equations_a4/reports/python-a4-report.json, field_equations_a4/reports/python-b4-block.json, metric_b4/results/vacuum-b4.json, lead_checks/reports/einstein-gauss-bonnet-a4.json
(6) the generalized Kronecker delta (GKD) and the three Lovelock tensors (Chapter 11)	PROVED	gkd_lovelock/results/lovelock-report.json, gkd_lovelock/results/gkd-selftest.json, gkd_lovelock/results/wolfram-gkd-report.json, gkd_lovelock/results/python-lovelock-report.json
(7) the curvature of the Revision record agrees with the values stored in the author's own notebooks, the earlier one ($b_4 = 1$) and the one with the metric function b_4 (Chapters 3 and 11)	PROVED; 5 of the 78 comparisons with the earlier notebook are NOT-AVAILABLE: that notebook stores no value to compare with	gkd_lovelock/comparison/author-comparison-report.json, metric_b4/results/wolfram-metric-b4-report.json, metric_b4/results/python-metric-b4-report.json
(8) the Kohn-Sham theory: the blocks, the rescaling, the exact exchange; the good-sector problem does not contain b_4 (Chapter 14)	PROVED	kohn_sham/reports/ks-theory-wolfram.json, kohn_sham/reports/ks-theory-python.json, kohn_sham/reports/ks-b4-wolfram.json

statement (with its row number)	**label**	**verified in (folder Revision)**
(9) the Kohn-Sham states along the deflating history (Chapters 15 and 16)	COMPUTED; at the chosen cutoff $L = 3$ of the hidden direction; the limit of large L is not established for every state	kohn_sham/reports/ks-rust-solver.json, kohn_sham/reports/ks-reference.json, kohn_sham/reports/ks-crosscheck.json, kohn_sham/reports/ks-rust-determinism.json, kohn_sham/reports/ks-rust-mermin-roots.json, kohn_sham/tip_convergence/tip-convergence.json
(10) the Kohn-Sham history of a_4 is a prescribed background (Chapter 17)	ASSUMED	field_equations_a4/reports/ks-source-conditions.json, field_equations_a4/ks_source/reports/ks-source-a4.json
(11) the pairing theorems T1, T2 (with the Z2 mirror ASSUMED) and Q, also with b_4 (Chapter 18)	PROVED	pairing/reports/wolfram-pairing.json, pairing/reports/python-pairing.json, metric_b4/results/consequences-b4.json
(12) T3 and its completion (with the Z2 mirror and the Kohn-Sham approximation ASSUMED): the Kohn-Sham universes of mass $+M$ and $-M$ (Chapter 19)	PROVED	pairing/kohn_sham/reports/wolfram-t3.json, pairing/kohn_sham/reports/python-t3.json, pairing/kohn_sham/reports/wolfram-t3-completion.json, pairing/kohn_sham/reports/python-t3-completion.json
(13) T3 shown on computed Kohn-Sham states of mass $+M$ and $-M$ (Chapter 19; demonstrations, not proofs)	COMPUTED	pairing/kohn_sham/reports/t3-rust-demo.json, pairing/kohn_sham/reports/t3-reference-demo.json
(14) the charge-conjugation matrices C and ΓC; the local conservation law of the U(1) charge (Chapters 5, 10 and 21)	PROVED	lead_checks/reports/charge-conjugation-and-u1.json

statement (with its row number)	label	verified in (folder Revision)
(15) the dark-sector investigation: what equations of state each field can give under stated assumptions about the observer (Chapter 22)	COMPUTED	`dark_sector/dirac16complex/reports/derivation-checks.json`, `dark_sector/dirac16complex/reports/ks-history-run.json`, `dark_sector/dirac16complex/reports/eos-checks.json`, `dark_sector/dirac16complex/reports/independent-checks.json`, `dark_sector/dirac16complex00/reports/python-derive-eos.json`, `dark_sector/dirac16complex00/reports/python-independent-numerics.json`
(16) a time-varying dark sector from the fields (Chapter 22)	HYPOTHESIS: investigated (row 15); results stated, not established	no report
(17) our universe has a partner of opposite charge (Chapters 20 and 21)	HYPOTHESIS: the T1 maps exist; that a partner exists is not shown	no report
(18) the total charge Q of one universe is constant (Chapters 18 and 21)	OPEN: holds only with no flux through the brane at $z = \pi/2$ (ASSUMED, not derived)	no report
(19) the big bang creates universes in pairs (Chapter 20)	OPEN: not proved; no creation process, rate or amplitude	no report
(20) what produces the observed excess of matter over antimatter (Chapter 21)	OPEN: the theory as built does not produce it	no report

Twelve remarks make the table exact.

- Rows 11 and 12 are PROVED theorems with an ASSUMED ingredient in their statement: T2 and T3 hold with the Z2 mirror, a choice of boundary condition at the edge of the hidden direction that the equations do not derive. A proved theorem with an assumption means: if the assumption holds, the conclusion holds.
- Rows 8, 9, 12 and 13 rest on the **Kohn-Sham approximation**, which is ASSUMED, not derived. It consists of four simplifications, each explained from zero in Chapters 13 to 15: (i) the problem is solved at one fixed instant x_4 at a time, as if the universe stood still at that instant (the stationary-slice, or adiabatic, ansatz); (ii) the action of the field on itself is replaced

by an average potential computed from the field's own density, made of the Hartree term (the interaction of the field with its own average density) and the exchange term of a uniform gas (a correction that comes from the anticommuting numbers of the field, computed exactly for a gas of the same density everywhere), while everything beyond these two terms, called correlation, is left out; (iii) only the states of such an average potential, called mean-field states, are used; (iv) the condition at the tip, the far end of the hidden direction, is chosen. The record states these hypotheses in `Revision/pairing/kohn_sham/t3-theory.json`, key `hypotheses`, and in `Revision/kohn_sham/ks-theory.json`. Rows 8 and 12 are exact statements about this approximation, and rows 9 and 13 are numbers computed in it: if the approximation is made, they hold, and a statement proved within an approximation says nothing about what lies outside it.

- Row 2 holds one report without a Wolfram counterpart, `theory/fock_quartic/reports/fock-quartic.json` (21 checks, written by one Python program with sympy and its own exact engine for the anticommutation rules of the quantum theory). It asks whether the energy-momentum tensor of dirac16complex with a self-coupling $\lambda \neq 0$ obeys, as an operator of the quantum theory, the exact relations that it obeys before quantisation (among them the relations between its energy density, its pressure and the potential of the self-coupling, Chapter 9). The answer is proved only in a finite model: one set of 16 modes (plane waves of one momentum in a flat frame, with frozen coefficients), whose quantum states number 2^{16}. There, for the solutions of the quantum field equation, the relations hold exactly, on every state, if and only if the operators are written in one particular order (the Wick, or normal, order), and they fail for every other order tested. For the field on a whole slice of space, for the dependence on the hidden direction and for the extra times nothing is proved: there the operator form is OPEN, and the field-equations record of Chapter 12 ASSUMES it; the symmetry and the conservation of this operator are not verified (Chapters 9 and 10).

- Row 9 holds, besides the two solvers and their comparisons, the report `kohn_sham/tip_convergence/tip-convergence.json` (7 checks). The Kohn-Sham states are computed with the hidden direction cut off at the tip $L = 3$, a choice (item (iv) above). This study repeats the computation for L from 3 to 6 and finds that the recorded energies of the states without self-interaction lie within 1.4% of their values at large L, except for one state, while the effects of the self-interaction depend strongly on L, and that the limit of large L is not established for every state (Chapter 14). So the numbers of row 9 are COMPUTED at $L = 3$, and the note of the row says so.

- Row 7 compares the curvature computed in the Revision record with the values that the author's own Mathematica notebooks store. Its first report compares with the earlier notebook, whose metric is the case $b_4 = 1$; 73 of its 78 checks are PASS: four checks of the inputs (one that the file holding the author's stored outputs was taken from exactly their notebook, and three of the metric, the last of which finds the author's metric equal to the Revision's in all 64 components), the Ricci scalar, all 64 components of the Einstein tensor, the first Lovelock tensor and its scalar, which all agree exactly, and two negative controls (deliberate errors that the comparison must detect, and does): $4 + 1 + 64 + 2 + 2 = 73$. For five quantities the author's notebook stores no value (for the second and third Lovelock tensors no stored output was found there, and the Christoffel symbols, the Riemann tensor and the Ricci tensor are computed there but their printing is switched off), so nothing can be compared: their verdict is NOT-AVAILABLE, which is neither PASS nor FAIL. The comparison says nothing about these five quantities; the Revision's own computation of them is checked by the reports of

row 6. Its other two reports, by the Wolfram verifier (27 checks) and the independent Python verifier (21 checks) of the metric with b_4, compare with the author's notebook `Dirac_Einstein-Lovelock_4+4_Nash.nb` of 2026-10-14: the metric, the Ricci tensor and scalar, the Einstein tensor, the three Lovelock tensors and the vacuum equations equal the outputs stored there, and at $b_4 = 1$ they equal the earlier record.

- Row 10 is labelled ASSUMED although 29 checks belong to it. Those checks show that the computed Kohn-Sham states cannot be the source of the history $a_4 = AHx_4$ in the field equations (their energy-momentum tensor depends on x_0, and it violates an algebraic condition of the field equations), and, in the second report, that no recorded Kohn-Sham state is an admissible source, so that nothing about a_4 is derived from the coupled problem. So the history cannot be derived from them and is assumed.
- Row 13 is COMPUTED, next to the PROVED row 12: its two reports solve the Kohn-Sham equations numerically for mass $+M$ and $-M$ (with the Rust solver and with the Python reference solver) and compare the two members as T3 predicts. A numerical demonstration illustrates a theorem; it does not prove it.
- Row 15 is COMPUTED. The investigation derives exact identities (with sympy), but its results, the equations of state that each field can give, are computed numbers, and they depend on an ASSUMED way in which an observer in 3-space normalises over the extra times. It states what each field can and cannot produce; it establishes neither hypothesis, which is why row 16 stays a HYPOTHESIS.
- Corollary C1 of Section 0.2 has no row of its own: it is T1 (row 11) combined with the checks `einstein_no_vacuum_solution` of `Revision/field_equations_a4/reports/wolfram-a4-report.json` and `einstein_no_vacuum` of `Revision/field_equations_a4/reports/python-a4-report.json` (row 5).
- Row 17 is a HYPOTHESIS and not a theorem, although T1 is proved. T1 says: if a universe with mass $+m$ is a solution, its image with mass $-m$ is a solution too, with the opposite charge. It does not say that the image exists alongside ours; a single universe is an equally valid solution without its partner, and for universes quantised independently of each other there is no cancellation of the charges (Q).
- Rows 14 and 18 separate what is proved about the charge from what is not. PROVED is the local law $\partial_\mu(\cos z\, J^\mu) = 0$ on every solution: no charge is made or destroyed at any point. The total charge Q of a slice of space is constant only if no charge flows through its edge, the brane at $z = \pi/2$; this no-flux condition is ASSUMED, not derived, and it fails on the exact homogeneous solutions of Chapter 18. So the constancy of the total charge is OPEN.
- Rows 19 and 20 are OPEN, not hypotheses, because the book examines them as questions and finds that the record does not answer them. Row 19: no equation of the record describes the creation of anything; the pairing record says so itself, in the list of twelve things that it does not establish (Notebook 00c prints the list). Row 20: Section 0.2 says why the theory as built does not produce the excess of matter over antimatter (it has no baryons, no process that changes their number and no violation of CP); what does produce the excess is answered neither by the Revision record nor by this book.

The ledger is the state of the record on the date of the last verified run of Notebook 00c. The Revision record may still grow (verifier reports may be added or gain checks), and the final ledger

of the book is completed when the book is assembled; Notebook 00c is the tool that keeps the table honest, because it fails when a new report appears that no row claims.

Two independent verifiers. For eleven subjects (rows 1 to 3, 5, 6, 8 and 11, the two subjects of row 12, T3 and its completion, the metric with b_4 of row 7, and the geometry and the Dirac operator with b_4 of row 2) the exact statements were checked twice: by a verifier written in the Wolfram Language and by an independent one written in Python with sympy, sharing no code. (The finite model of row 2 is the exception: one Python program checks it.) The two do not check exactly the same list of statements, but where they overlap, an error would have to be made twice, in two languages, by two separate programs, to go unnoticed. The Kohn-Sham numbers (row 9) were computed twice in the same spirit, by the Rust solver and by an independent Python reference solver, and compared with tolerances fixed in advance (Chapter 16); the study of the tip cutoff ran the reference solver for two of its states at two of its values of L, 3 and 4 (28 comparisons, check `reference_solver_at_two_L` of `kohn_sham/tip_convergence/tip-convergence.json`).

Fingerprints. A report is worth something only for the exact files it was computed from. To tie the two together, a report writes down a **fingerprint** of each file it read. The fingerprint used here is **sha256**: a fixed recipe that turns the bytes of a file, however many there are, into a number of 256 binary digits, written as 64 **hexadecimal** characters. Hexadecimal digits are the sixteen symbols 0 to 9 and a to f, standing for the numbers 0 to 15; each one carries 4 binary digits, and $64 \cdot 4 = 256$. The recipe has three properties that matter here:

- it is deterministic: the same bytes always give the same fingerprint, on every computer;
- a change of a single byte changes the fingerprint completely, not just in one place;
- nobody knows how to construct two different files with the same fingerprint, and nobody has ever found such a pair.

The first property is a fact of the recipe. The second is not proved: Notebook 00c measures it for 115 one-character changes of one sentence (COMPUTED, below), and beyond these measured cases the book relies on it, so in general it is ASSUMED. The third is believed by everyone who uses sha256 but is not proved mathematically; the book relies on it, and so it is ASSUMED.

Three words of the theory of chance. The **probability** of an event is the fraction of the equally likely cases in which it happens: a fair die shows a six in 1 of its 6 equally likely cases, so with probability 1/6. An event and its opposite (a six, and no six) have probabilities that add up to 1, because together they cover every case exactly once. Two events are **independent** when knowing whether one of them happened does not change the probability of the other; then the probability that both happen is the product of their probabilities (two dice show two sixes with probability $\frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$, because exactly 1 of the $6 \cdot 6 = 36$ equally likely pairs is two sixes). The **expected number** of events in a list is the average number of them that happen, taken over very many repetitions; it equals the sum of the probabilities of the single events, whether or not they are independent (a rule of the theory of chance that we use without proof). For example, 12 dice show on average $12 \cdot \frac{1}{6} = 2$ sixes.

How much should a fingerprint change? Suppose the new fingerprint behaves like a string of 64 hexadecimal characters chosen at random. One character of it equals the corresponding character of the old fingerprint with probability

$$p_{\text{same}} = \frac{1}{16}$$

(the new character is one of 16 equally likely symbols, and exactly one of them is the old one), so it differs with probability

$$p_{\text{differ}} = 1 - \frac{1}{16} = \frac{15}{16}$$

(the probabilities of an event and of its opposite add up to 1). The expected number of differing characters among the 64 is

$$64 \cdot \frac{15}{16} = 4 \cdot 15 = 60$$

(the expected number of differing characters is the sum, over the 64 characters, of the probability that the character differs; and $64/16 = 4$). Notebook 00c changes each character of a sentence of 115 characters in turn and finds that between 54 and 64 characters of the fingerprint change, 60.03 on average (COMPUTED): close to what a random string gives on average.

A report of the Revision record records the fingerprints of its input files in dedicated keys (the Wolfram check of the Lovelock tensors lists 15 files in its keys `inputSha256` and `sourceSha256`, and several reports have a key `inputs`, a list of files each with its fingerprint), or inside the detail text of a check, written as (sha256 followed by the 64 characters, or only their first 16). When the fingerprint of today's file equals the recorded one, the report was computed from exactly today's file, byte for byte. Notebook 00c compares 24 of the recorded fingerprints with today's files (those of the two reports on the Lovelock tensors in Python and in the Wolfram Language, and seven in the details of five Kohn-Sham reports; it does not compare the others), and Notebook 00d compares 582 more (Section 0.22).

Example: the ledger checked by the computer. Notebook 00c does at level 2 (Section 0.4) what a careful reader would do by hand, but for every report at once. It opens the charge-conjugation report, reads its twelve checks and confirms that three of them state the charge-conjugation matrices of Section 0.2 word for word, and that a fourth proves the local conservation law of the charge and says itself that it does not establish a constant total charge; writes one function that counts the checks of a report in each of the three layouts that occur in the record; searches the folder `Revision` (without the book's own folder and the records of the workflows) for reports and finds exactly the 51 of its list; confirms that no check has the verdict FAIL (all are PASS except five comparisons with the author's own notebook that are NOT-AVAILABLE) and that the counts agree with what the reports say about themselves, with the table of `Revision/README.md`, with the Kohn-Sham cross-check and with the dark-sector summary file; builds the ledger above and checks that every report belongs to exactly one row; prints the twelve sentences of the pairing record about what it does not establish; and compares fingerprints. It draws four figures and ends with ALL 18 CHECKS PASSED (notebook 00c).

0.19 How to run Notebook 00c

Step 1. What this notebook does and what it needs.

Notebook 00c (The honesty ledger: reading and checking the Revision record) is the file `Revision/textbook/notebooks/00c_honesty_ledger.ipynb` of the repository Dirac_claude. It finds the 51 verifier reports of the Revision record, counts their checks and confirms that none has the verdict FAIL (all are PASS except five comparisons with the author's outputs that are NOT-AVAILABLE), compares the counts with the summaries of the reports and with the numbers quoted elsewhere in the record, assigns every report to its row of the honesty ledger with the labels PROVED, COMPUTED, ASSUMED, HYPOTHESIS and OPEN, prints what the pairing record itself says is not established, compares 24 recorded sha256 fingerprints with the files of today, and draws four teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00c_honesty_ledger.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00c_honesty_ledger.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 00c_honesty_ledger.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/00c.captions.json`
- `Revision/textbook/figures/00c_1_checks_by_report.png`
- `Revision/textbook/figures/00c_2_two_verifiers.png`
- `Revision/textbook/figures/00c_3_ledger.png`
- `Revision/textbook/figures/00c_4_fingerprints.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the four figure files of this notebook exist
ALL 18 CHECKS PASSED (notebook 00c)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/00c_honesty_ledger.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 00c_honesty_ledger.ipynb
python -m nbconvert --execute --inplace 00c_honesty_ledger.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" or "KeyError" naming a file below the folder Revision: a file of the Revision record is missing or damaged: the repository folder is incomplete. Delete the folder Dirac_claude, download it again with the git clone command of Step 3, and open the notebook from the new folder.
- "AssertionError: check failed: the 24 recorded fingerprints equal the files of today": a file of the Revision record was changed after its report was written; the table printed just above the error marks it CHANGED. The command below, run in the repository folder, lists every changed file; restore a changed file with the second command (write the file name printed in the table, with the folder Revision in front, instead of FILE).

```
git status
git restore FILE
```

0.20 Notebook 00c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/00c_honesty_ledger.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook checks the honesty ledger of the book: the table that gives every
main statement of the book its label (PROVED, COMPUTED, ASSUMED, HYPOTHESIS or
OPEN) and the place where it is verified. It does what a careful reader would do
by hand, but for every report at once. It

- opens one report of the Revision record and reads its checks one by one;
- writes a function that counts the checks of a report in each of the three
  layouts that occur, searches the whole folder Revision for reports, finds 51,
  and counts the checks of all of them;
- confirms that no check has the verdict FAIL (every check is PASS except five
  comparisons with the author's own outputs that could not be made, verdict
  NOT-AVAILABLE) and that the counts agree with the summaries that the reports
  state themselves, with the table of the file Revision/README.md, with the
  counts quoted by the Kohn-Sham cross-check and with the dark-sector summary;
- assigns every report to its row of the ledger, and prints the list of what the
  pairing record itself says is NOT established;
- computes sha256 fingerprints and compares 24 of the fingerprints that the
  reports recorded for their input files with the files of today;
- draws four teaching plots and prints a PASS line for every check.

It does not run the verifier programs again (they need Wolfram Mathematica or Rust
and minutes to hours); it checks what they recorded.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 00c (The honesty ledger: reading and checking the Revision record) is the file
‘Revision/textbook/notebooks/00c_honesty_ledger.ipynb‘ of the repository Dirac_claude. It
finds the 51 verifier reports of the Revision record, counts their checks and confirms
that none has the verdict FAIL (all are PASS except five comparisons with the author's
outputs that are NOT-AVAILABLE), compares the counts with the summaries of the reports
and with the numbers quoted elsewhere in the record, assigns every report to its row of
the honesty ledger with the labels PROVED, COMPUTED, ASSUMED, HYPOTHESIS and OPEN, prints
what the pairing record itself says is not established, compares 24 recorded sha256
fingerprints with the files of today, and draws four teaching plots. It needs a computer
with Windows 11, macOS or Linux, an internet connection for the installation, the program
```

Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00c_honesty_ledger.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00c_honesty_ledger.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 00c_honesty_ledger.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/00c.captions.json‘
- ‘Revision/textbook/figures/00c_1_checks_by_report.png‘
- ‘Revision/textbook/figures/00c_2_two_verifiers.png‘
- ‘Revision/textbook/figures/00c_3_ledger.png‘
- ‘Revision/textbook/figures/00c_4_fingerprints.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the four figure files of this notebook exist
ALL 18 CHECKS PASSED (notebook 00c)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/00c_honesty_ledger.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command

below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 00c_honesty_ledger.ipynb
python -m nbconvert --execute --inplace 00c_honesty_ledger.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" or "KeyError" naming a file below the folder Revision: a file of the Revision record is missing or damaged: the repository folder is incomplete. Delete the folder Dirac_claude, download it again with the git clone command of Step 3, and open the notebook from the new folder.
- "AssertionError: check failed: the 24 recorded fingerprints equal the files of today": a file of the Revision record was changed after its report was written; the table printed just above the error marks it CHANGED. The command below, run in the repository folder, lists every changed file; restore a changed file with the second command (write the file name printed in the table, with the folder Revision in front, instead of FILE).

```
git status
git restore FILE
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 00c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 00c (The honesty ledger: reading and checking the Revision record) is the file
# Revision/textbook/notebooks/00c_honesty_ledger.ipynb of the repository Dirac_claude. It
# finds the 51 verifier reports of the Revision record, counts their checks and confirms
# that none has the verdict FAIL (all are PASS except five comparisons with the author's
# outputs that are NOT-AVAILABLE), compares the counts with the summaries of the reports
# and with the numbers quoted elsewhere in the record, assigns every report to its row of
# the honesty ledger with the labels PROVED, COMPUTED, ASSUMED, HYPOTHESIS and OPEN,
# prints what the pairing record itself says is not established, compares 24 recorded
# sha256 fingerprints with the files of today, and draws four teaching plots. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
```

```
# and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
```

```
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
```

```
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 00c_honesty_ledger.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 00c_honesty_ledger.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 00c_honesty_ledger.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/00c.captions.json
# - Revision/textbook/figures/00c_1_checks_by_report.png
# - Revision/textbook/figures/00c_2_two_verifiers.png
# - Revision/textbook/figures/00c_3_ledger.png
# - Revision/textbook/figures/00c_4_fingerprints.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
```

```
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the four figure files of this notebook exist
#     ALL 18 CHECKS PASSED (notebook 00c)
#
# and the notebook must show 4 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/00c_honesty_ledger.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 00c_honesty_ledger.ipynb
#     python -m nbconvert --execute --inplace 00c_honesty_ledger.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" or "KeyError" naming a file below the folder Revision: a file of
#   the Revision record is missing or damaged: the repository folder is incomplete.
#   Delete the folder Dirac_claude, download it again with the git clone command of Step
#   3, and open the notebook from the new folder.
```

```
# - "AssertionError: check failed: the 24 recorded fingerprints equal the files of
#   today": a file of the Revision record was changed after its report was written; the
#   table printed just above the error marks it CHANGED. The command below, run in the
#   repository folder, lists every changed file; restore a changed file with the second
#   command (write the file name printed in the table, with the folder Revision in front,
#   instead of FILE).
#     git status
#     git restore FILE
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "00c"  # this notebook: chapter 00, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative
```

```
def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
```

```
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 00c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Revision record**: the computations stored in the folder Revision of the
  repository, done anew for the author's metric. Every number of the book comes
  from it or from a notebook of the book.
- **Verifier, report**: a *verifier* is a program that checks statements; it writes
  its results into a *report*, a JSON file (a plain text file of names, numbers and
  lists).
- **Check, verdict**: a *check* is one statement tested by a verifier; its *verdict*
  is PASS (true) or FAIL (false). Each check also has a name and a detail text.
- **Engine**: the program that did the computation: **Wolfram Language** (the
  language of Mathematica, run with wolframscript), **Python** (with the packages
  sympy, numpy and mpmath), or **Rust** (a fast compiled language). The *lead's
  independent checks* are short Python programs that share no code with the other
  verifiers.
- **Independent**: two verifiers are independent when they share no code; when both
  agree, a programming error in one of them would have to be repeated in the other
  to go unnoticed.
- **Dictionary, list** (Python): a *list* `[a, b, c]` is an ordered collection; a
  *dictionary* `{"name": value, ...}` stores values under names (*keys*). A JSON
  file is read into dictionaries and lists.
- **The five labels**: **PROVED** (exact; the book gives the complete proof and,
  where the Revision record contains an exact computer check of the statement,
  names the report file and the check), **COMPUTED** (a number from a numerical
```

```
  computation, with its measured uncertainty and the file that holds it),
  **ASSUMED** (a starting point that the book does not derive: a convention, a
  physical input or an approximation), **HYPOTHESIS** (an idea that is stated and
  examined but not established), **OPEN** (a question that neither the Revision
  record nor the book answers).
- **Ledger**: the table of the statements, their labels and the reports that
  verify them.
- **Fingerprint (sha256)**: a number of 64 hexadecimal characters computed from the
  bytes of a file by a fixed recipe. The same file always gives the same
  fingerprint. A file that differs in a single byte has, in every case anyone has
  tried, a completely different one; this is not proved, and section 10 measures
  it for 115 changes. **Hexadecimal**: the digits 0 to 9 and the letters a to f,
  sixteen symbols in all.
```

Text cell:

```
## 4. The physical and mathematical situation

The rule above every other rule of the book is honesty: it never writes "proved"
for a statement that is not proved. To make this rule checkable, every statement
carries one of five labels and names the place where it is verified. A statement
can be checked at three levels:

1. **The derivation**: follow it line by line in the book.
2. **The report**: open the report of the verifier that checked it and read the
   verdict of the named check. This notebook works at this level.
3. **The computation**: run the verifier again and compare its new report with the
   stored one (the last chapter of the book lists every command).

Two facts make level 2 trustworthy. First, the exact statements of eleven subjects
were checked by two independent verifiers, one in Wolfram Language and one in
Python, and the Kohn-Sham numbers by two independent solvers, one in Rust and one
in Python.
Second, a report records the sha256 fingerprints of the files it read; when the
fingerprints of today's files are the same, the report belongs to exactly these
files.

A ledger row labelled OPEN or HYPOTHESIS has no report: nothing in the record
establishes it. In particular the record proves exact maps between the solutions
of mass $+m$ and those of mass $-m$ (the pairing theorems T1, T2, Q and T3), but it
does not prove that the big bang creates universes, in pairs or otherwise, and the
theory as built does not explain the excess of matter over antimatter. It proves
that no charge is made or destroyed at any point (a *local* conservation law), but
not that the total charge of a universe stays constant. The pairing record states
itself what it does not establish; the notebook prints that list.
```

Text cell:

```
## 5. One report, opened by hand

The next cell defines the function `read_report`, which reads a JSON file of the
repository into Python dictionaries and lists, and the helper `check_reproduces`.
```

‘check_reproduces‘ is the helper ‘check‘ for a check that reproduces a Revision record: it first calls ‘sys.stdout.flush()‘, which sends all printed text that is still waiting, and then calls ‘check‘ with the record. (Jupyter sends printed text in pieces; flushing first keeps the PASS line and the line ‘reproduces ...‘ below it in one piece, so that the book’s tools read them together.)

Then the cell opens the report of the lead’s independent check of the charge-conjugation matrices and of the local conservation law of the charge, Revision/lead_checks/reports/charge-conjugation-and-u1.json. It prints the program that wrote the report (its *producer*), the verdict and name of each of its 12 checks, and the summary that the report states about itself. The check confirms that our count of the verdicts PASS equals that summary.

In [2]:

```
import hashlib  # computes sha256 fingerprints
import re  # finds patterns in texts ("regular expressions")
import sys  # sys.stdout is the channel through which the notebook prints

import numpy as np  # arrays of numbers


def read_report(path):
    """The content of the JSON file path of the repository (dictionaries, lists)."""
    return json.loads(repository_file(path).read_text(encoding="utf-8"))


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), after sending the waiting output."""
    sys.stdout.flush()  # send every printed line that is still waiting
    check(condition, name, record=record)


CC_REPORT = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
cc = read_report(CC_REPORT)  # a dictionary with the keys producer, summary, checks
producer = cc["producer"]
say(f"producer: {producer}")
for entry in cc["checks"]:  # each entry is a dictionary: name, verdict, detail
    verdict, name = entry["verdict"], entry["name"]
    say(f"  {verdict}  {name}")
stated = cc["summary"]  # what the report says about itself: passed and total
stated_passed, stated_total = stated["passed"], stated["total"]
say(f"the report states: {stated_passed} passed of {stated_total}")
passed = sum(1 for entry in cc["checks"] if entry["verdict"] == "PASS")
number_of_checks = len(cc["checks"])
report("checks with the verdict PASS, counted", f"{passed} of {number_of_checks}")
check_reproduces(passed == stated["passed"] == stated["total"] == 12,
                 "12 of 12 checks of the charge-conjugation report are PASS",
                 f"{CC_REPORT}, its summary")
```

Out [2]:

```
producer: Revision/lead_checks/charge_conjugation_and_u1.py (lead, independent; no
    Revision code imported)
  PASS  representation_real
  PASS  B_imaginary_hermitian
  PASS  intertwiners_same_mass
  PASS  intertwiners_reversed_mass
  PASS  charge_conjugation_matrix_plus
  PASS  charge_conjugation_matrix_minus
  PASS  majorana_conditions_consistent
  PASS  bilinears_under_charge_conjugation
  PASS  quantum_charge_conjugation_unitary_type
  PASS  real_fields_charge_conjugation
  PASS  spinor_connection_real
  PASS  u1_noether_matrix_identity
the report states: 12 passed of 12
RESULT checks with the verdict PASS, counted = 12 of 12
PASS 12 of 12 checks of the charge-conjugation report are PASS
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, its summary
```

Text cell:

```
Every check also has a detail text that says exactly what was verified. The next
cell prints the details of five checks of the same report. They say, in the
report's own words, that the author's gamma matrices are real, so that plain
complex conjugation changes nothing at all in a real field (the charge conjugation
of the theory is a MATRIX, not plain complex conjugation); that the
charge-conjugation matrix which keeps the mass is $\mathcal{C}_+ = C$ and the one
which reverses the mass is $\mathcal{C}_- = \Gamma C$; that for a real commuting
field the charge current is zero and $\mathcal{C}_+$ acts as the identity, while
the real matrix $\Gamma$ with the mass reversed maps solutions to solutions; and
that the charge obeys a *local* conservation law: on every solution
$\partial_\mu(\cos z\, J^\mu) = 0$, so no charge is made or destroyed at any point.
The same detail says what this check does NOT establish: the total charge $Q$ of a
slice of space is constant only if no charge flows through its edge, the brane at
$z = \pi/2$, where the flow is not zero in general. (These statements are derived
from zero later in the book; the book labels the constancy of the total charge
OPEN.) The detail of the last check is long; the cell prints only its last two
parts, after the second-to-last semicolon, which state the result and its limit.
The details are long texts; ‘say‘ breaks each into lines of at most 89
characters. The two checks of the cell confirm that the details contain these
statements word for word.
```

In [3]:

```
details = {entry["name"]: entry["detail"] for entry in cc["checks"]}  # name -> detail
for wanted in ("representation_real", "charge_conjugation_matrix_plus",
               "charge_conjugation_matrix_minus", "real_fields_charge_conjugation",
               "u1_noether_matrix_identity"):
    detail = details[wanted]
    if wanted == "u1_noether_matrix_identity":
        # This detail is long; its last two parts, after the second-to-last "; ",
        # state the result and its limit.
```

```
        detail = "; ".join(detail.split("; ")[-2:])
    say(f"{wanted}:")
    say("    " + detail)  # say breaks it into lines of at most 89 characters

R5_WORDS = [  # (check, words that its detail must contain)
    ("representation_real", "plain complex conjugation is the identity"),
    ("charge_conjugation_matrix_plus", "calC_+ = C (= sigma16)"),
    ("charge_conjugation_matrix_plus", "Psi^c = calC_+ Psibar^T = Psi*"),
    ("charge_conjugation_matrix_minus", "calC_- = Gamma C"),
    ("charge_conjugation_matrix_minus", "Psi^c = calC_- Psibar^T = Gamma Psi*"),
    ("real_fields_charge_conjugation", "J^a = -i Psi^T C gamma^a Psi = 0"),
    ("real_fields_charge_conjugation", "calC_+ acts as the identity"),
    ("real_fields_charge_conjugation", "the matrix Gamma with the mass reversed"),
]
check_reproduces(all(words in details[name] for name, words in R5_WORDS),
                 "the report states calC_+ = C, calC_- = Gamma C and J = 0 for a "
                 "real field",
                 f"{CC_REPORT}, checks charge_conjugation_matrix_plus, "
                 "charge_conjugation_matrix_minus and real_fields_charge_conjugation")
U1_WORDS = ["on shell the LOCAL law d_mu(cos z J^mu) = 0 holds",
            "is constant only if no charge flows through its boundary",
            "constancy of Q is NOT established by this check"]
check_reproduces(all(words in details["u1_noether_matrix_identity"]
                     for words in U1_WORDS),
                 "the report proves the local law and does not establish a "
                 "constant total charge",
                 f"{CC_REPORT}, check u1_noether_matrix_identity")
```

Out [3]:

```
representation_real:
    gamma^a, C (= sigma16, symmetric, C^2 = 1) and S^ab are real: plain complex
    conjugation is the identity on a real field
charge_conjugation_matrix_plus:
    calC_+ = C (= sigma16): calC_+^-1 gamma^a calC_+ = -(gamma^a)^T for every a; real
    symmetric; Psi^c = calC_+ Psibar^T = Psi*
charge_conjugation_matrix_minus:
    calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T for every a; real; Psi^c =
    calC_- Psibar^T = Gamma Psi* (solves the equation with V -> -V)
real_fields_charge_conjugation:
    real commuting Psi: J^a = -i Psi^T C gamma^a Psi = 0 identically and calC_+ acts as
    the identity (a real field is its own conjugate, no U(1) charge); the REAL matrix
    Gamma maps real fields to real fields, keeps S = Psi^T C Psi and reverses every
    kinetic matrix C gamma^a (Gamma^T C gamma^a Gamma = -C gamma^a): with m -> -m (and
    lambda -> -lambda) it maps solutions to solutions (T1) - for real fields the matter
    <-> antimatter map is the matrix Gamma with the mass reversed
u1_noether_matrix_identity:
    on shell the LOCAL law d_mu(cos z J^mu) = 0 holds; the total charge Q = Int cos z
    Psi^dagger B Psi d^7x of a slice is constant only if no charge flows through its
    boundary (at the brane z = pi/2 the flow is not zero in general): constancy of Q is
    NOT established by this check
PASS the report states calC_+ = C, calC_- = Gamma C and J = 0 for a real field
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, checks
```

```
    charge_conjugation_matrix_plus, charge_conjugation_matrix_minus and
    real_fields_charge_conjugation
PASS the report proves the local law and does not establish a constant total charge
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    u1_noether_matrix_identity
```

Text cell:

```
## 6. Three layouts of a report, one counting function

The reports of the Revision record were written by different programs, and their
checks come in three layouts:

- **layout A**: ‘checks‘ is a list of dictionaries, each with a ‘name‘ and a
  ‘verdict‘ (written PASS or pass);
- **layout B**: ‘checks‘ is a dictionary from the name of each check to a
  dictionary with the entry ‘passed‘ (true or false);
- **layout C**: the self-test of the Rust function GKD has no ‘checks‘ but a list
  ‘results‘, one per test, each with the number of ‘mismatches‘ (wrong values).

The reports also state their own totals in different words (‘summary‘, ‘counts‘,
‘checkCount‘), and two of them as a text, such as "49/49 checks pass". The next
cell defines ‘count_checks‘, which counts the passed checks in all three layouts,
and ‘stated_summary‘, which reads the totals that a report states about itself.
It shows one report of each layout.
```

In [4]:

```
def count_checks(data):
    """(number of passed checks, number of checks), counted from the checks."""
    if isinstance(data.get("checks"), list):  # layout A
        verdicts = [entry["verdict"].upper() == "PASS" for entry in data["checks"]]
    elif isinstance(data.get("checks"), dict):  # layout B
        verdicts = [entry["passed"] is True for entry in data["checks"].values()]
    else:  # layout C: a self-test passes when it found no mismatch
        verdicts = [entry["mismatches"] == 0 for entry in data["results"]]
    # True counts as 1 and False as 0 when added.
    return sum(verdicts), len(verdicts)


def stated_summary(data):
    """(passed, total) as the report states them itself; None if it states none."""
    summary = data.get("summary")
    if isinstance(summary, dict):  # {"passed": n, "total": n} and similar names
        passed = summary.get("passed", summary.get("pass", summary.get("PASS")))
        return passed, summary.get("total", summary.get("checks"))
    if isinstance(summary, str):  # a text such as "49/49 checks pass"
        passed, total = re.fullmatch(r"(\d+)/(\d+) checks pass", summary).groups()
        return int(passed), int(total)
    counts = data.get("counts")
    if isinstance(counts, dict) and "pass" in counts:  # {"pass", "fail", "pending"}
        return counts["pass"], counts["pass"] + counts["fail"] + counts["pending"]
```

```
    if "checkCount" in data:  # a total and a number of failed checks
        failed = data.get("failCount", data.get("failedCount",
                                                data.get("failedCheckCount")))
        return data["checkCount"] - failed, data["checkCount"]
    return None


EXAMPLES = [("A", "Revision/pairing/reports/wolfram-pairing.json"),
            ("B", "Revision/gkd_lovelock/results/lovelock-report.json"),
            ("C", "Revision/gkd_lovelock/results/gkd-selftest.json")]
for layout, path in EXAMPLES:
    data = read_report(path)
    passed, total = count_checks(data)
    say(f"layout {layout}: {path}")
    say(f"    counted {passed} of {total}; stated {stated_summary(data)}")
```

Out [4]:

```
layout A: Revision/pairing/reports/wolfram-pairing.json
    counted 101 of 101; stated (101, 101)
layout B: Revision/gkd_lovelock/results/lovelock-report.json
    counted 19 of 19; stated (19, 19)
layout C: Revision/gkd_lovelock/results/gkd-selftest.json
    counted 9 of 9; stated None
```

Text cell:

```
## 7. All 51 reports of the Revision record

The next cell first searches the whole folder Revision for reports: JSON files
whose key ‘checks‘ holds the checks themselves (a list of checks, or a dictionary
whose every entry is a check, that is, a dictionary itself), or, like the GKD
self-test, a list ‘results‘ whose entries count ‘mismatches‘. One JSON file has a
key ‘checks‘ that holds only two numbers, the totals of another report: the
summary file Revision/dark_sector/dirac16complex/outputs/eos-summary.json. It is
not a report, and the search prints its name; the cell after the next compares its
two numbers with our count. The search skips three kinds of folders that are not
part of the record: the folder Revision/textbook of this book, the folder
Revision/workflows (the records of the programs that organised the work, rewritten
while they run; the Revision record says itself that no result depends on them),
and the build folders ‘target‘ of the Rust programs. The cell then lists the 51
verifier reports of the Revision record, grouped by folder, each with its engine
(the language of the program that wrote the report), counts the checks of each with
‘count_checks‘, and prints a table and the totals per engine.

One report has checks whose verdict is neither PASS nor FAIL: the comparison of
the Revision’s curvature with the outputs that the author stored in their own
Mathematica notebook, Revision/gkd_lovelock/comparison/author-comparison-report.json.
For five quantities the author’s notebook holds no stored value (three of them,
the Christoffel symbols, the Riemann tensor and the Ricci tensor, are computed
there with their printing switched off; for the second and third Lovelock tensors
no stored output was found there), so there is nothing to compare; their verdict
```

is NOT-AVAILABLE. Three checks follow:

1. the search finds exactly the 51 reports of the list: no report of the record is
   left out of the count (and so out of the ledger below);
2. no check of the 51 reports has the verdict FAIL: every check is PASS, except
   the five NOT-AVAILABLE comparisons of that one report;
3. for every report that states its own totals, our count equals them (the
   self-test of layout C states only its verdict, SUCCESS, which is checked
   instead).

‘folder.rglob("*.json")‘ lists every JSON file in a folder and in all its
sub-folders; ‘path.as_posix()‘ writes a path with ‘/‘; ‘path.removeprefix("Revision/")‘
removes the folder name Revision/ from the front of a path, to keep the table
narrow. Reading every JSON file of the record (a few hundred files) takes about a
second.

In [5]:

```
def is_report(path):
    """True if the JSON file path (a Path) is a verifier report."""
    data = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(data, dict):
        return False
    checks = data.get("checks")
    listed = isinstance(checks, list) or (  # a list of checks, or a dictionary
        isinstance(checks, dict)  # whose every entry is a check (a dictionary)
        and all(isinstance(entry, dict) for entry in checks.values()))
    results = data.get("results")
    self_test = (isinstance(results, list) and len(results) > 0
                 and isinstance(results[0], dict) and "mismatches" in results[0])
    return listed or self_test


SKIPPED = ("Revision/textbook/", "Revision/workflows/")  # folders that hold no report
found_reports = []  # every report found in the folder Revision, as a relative path
not_reports = []  # JSON files with a key "checks" that holds no checks
for path in sorted(repository_file("Revision").rglob("*.json")):
    relative = path.relative_to(REPO).as_posix()  # e.g. "Revision/algebra/..."
    if relative.startswith(SKIPPED) or "/target/" in relative:
        continue  # the book, the workflow records and the Rust build folders
    if is_report(path):
        found_reports.append(relative)
    elif "checks" in read_report(relative):
        not_reports.append(relative)
report("verifier reports found in the folder Revision", len(found_reports))
say("JSON files with a key checks that holds no checks:")
for relative in not_reports:
    say("    " + relative)

REPORTS = [  # (report, engine)
    # the gammas, C, Gamma, B, Pin(4,4) and Spin(4,4)
    ("Revision/algebra/reports/wolfram-algebra.json", "Wolfram"),
    ("Revision/algebra/reports/python-algebra.json", "Python"),
```

```
# the metric with b4 (2026-10-14): curvature, the author's outputs, vacuum
("Revision/metric_b4/results/wolfram-metric-b4-report.json", "Wolfram"),
("Revision/metric_b4/results/python-metric-b4-report.json", "Python"),
("Revision/metric_b4/results/vacuum-b4.json", "Wolfram"),
("Revision/metric_b4/results/consequences-b4.json", "Wolfram"),
# the Lagrangians, field equations, EMT, quantisation; the scope; a Fock model
("Revision/theory/reports/wolfram-field-theory.json", "Wolfram"),
("Revision/theory/reports/python-field-theory.json", "Python"),
("Revision/theory/reports/wolfram-scope.json", "Wolfram"),
("Revision/theory/reports/python-scope.json", "Python"),
("Revision/theory/fock_quartic/reports/fock-quartic.json", "Python"),
("Revision/theory/reports/wolfram-field-theory-b4.json", "Wolfram"),
("Revision/theory/reports/python-field-theory-b4.json", "Python"),
("Revision/theory/reports/wolfram-field-theory-b4-emt-C.json", "Wolfram"),
("Revision/theory/reports/wolfram-field-theory-b4-emt-G.json", "Wolfram"),
# the field equations for a4 and b4; the Kohn-Sham states as a source
("Revision/field_equations_a4/reports/wolfram-a4-report.json", "Wolfram"),
("Revision/field_equations_a4/reports/python-a4-report.json", "Python"),
("Revision/field_equations_a4/reports/python-b4-block.json", "Python"),
("Revision/field_equations_a4/reports/ks-source-conditions.json", "Python"),
("Revision/field_equations_a4/ks_source/reports/ks-source-a4.json", "Python"),
# GKD and the Lovelock tensors; the comparison with the author's outputs
("Revision/gkd_lovelock/results/lovelock-report.json", "Rust"),
("Revision/gkd_lovelock/results/gkd-selftest.json", "Rust"),
("Revision/gkd_lovelock/results/wolfram-gkd-report.json", "Wolfram"),
("Revision/gkd_lovelock/results/python-lovelock-report.json", "Python"),
("Revision/gkd_lovelock/comparison/author-comparison-report.json", "Python"),
# the Kohn-Sham theory, solvers and comparisons; the tip-cutoff study
("Revision/kohn_sham/reports/ks-theory-wolfram.json", "Wolfram"),
("Revision/kohn_sham/reports/ks-theory-python.json", "Python"),
("Revision/kohn_sham/reports/ks-b4-wolfram.json", "Wolfram"),
("Revision/kohn_sham/reports/ks-rust-solver.json", "Rust"),
("Revision/kohn_sham/reports/ks-rust-determinism.json", "Python"),
("Revision/kohn_sham/reports/ks-rust-mermin-roots.json", "Python"),
("Revision/kohn_sham/reports/ks-reference.json", "Python"),
("Revision/kohn_sham/reports/ks-crosscheck.json", "Python"),
("Revision/kohn_sham/tip_convergence/tip-convergence.json", "Python"),
# the dark-sector hypotheses of the two fields, investigated
("Revision/dark_sector/dirac16complex/reports/derivation-checks.json", "Python"),
("Revision/dark_sector/dirac16complex/reports/ks-history-run.json", "Python"),
("Revision/dark_sector/dirac16complex/reports/eos-checks.json", "Python"),
("Revision/dark_sector/dirac16complex/reports/independent-checks.json", "Python"),
("Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json", "Python"),
("Revision/dark_sector/dirac16complex00/reports/python-independent-numerics.json",
 "Python"),
# the pairing theorems T1, T2, Q and T3; T3 completed and demonstrated
("Revision/pairing/reports/wolfram-pairing.json", "Wolfram"),
("Revision/pairing/reports/python-pairing.json", "Python"),
("Revision/pairing/kohn_sham/reports/wolfram-t3.json", "Wolfram"),
("Revision/pairing/kohn_sham/reports/python-t3.json", "Python"),
("Revision/pairing/kohn_sham/reports/wolfram-t3-completion.json", "Wolfram"),
("Revision/pairing/kohn_sham/reports/python-t3-completion.json", "Python"),
("Revision/pairing/kohn_sham/reports/t3-rust-demo.json", "Python"),
("Revision/pairing/kohn_sham/reports/t3-reference-demo.json", "Python"),
```

```
    # the lead's independent checks
    ("Revision/lead_checks/reports/charge-conjugation-and-u1.json", "lead"),
    ("Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json", "lead"),
    ("Revision/lead_checks/reports/emt-divergence-and-spin-connection.json", "lead"),
]
check(found_reports == sorted(path for path, _ in REPORTS),
      f"the search finds exactly the {len(REPORTS)} reports of the list")
counted = {}  # report -> (passed, total)
header = "report (in the folder Revision)"
say(f"{header:69} engine  pass of all")
for path, engine in REPORTS:
    data = read_report(path)
    counted[path] = count_checks(data)
    passed, total = counted[path]
    short = path.removeprefix("Revision/")
    say(f"{short:69} {engine:7} {passed:4d} of {total:3d}")
all_checks = sum(total for _, total in counted.values())
all_passed = sum(passed for passed, _ in counted.values())
report("reports", len(REPORTS))
report("checks in all reports", all_checks)
report("checks with the verdict PASS", all_passed)
engine_totals = {}  # engine -> the number of its checks
for engine in ("Wolfram", "Python", "Rust", "lead"):
    engine_totals[engine] = sum(counted[path][1] for path, e in REPORTS
                                if e == engine)
    report(f"checks done with the engine {engine}", engine_totals[engine])

COMPARISON = "Revision/gkd_lovelock/comparison/author-comparison-report.json"
not_pass = [path for path, _ in REPORTS if counted[path][0] != counted[path][1]]
other_verdicts = [entry["verdict"] for entry in read_report(COMPARISON)["checks"]
                  if entry["verdict"] != "PASS"]  # the verdicts that are not PASS
not_available = other_verdicts.count("NOT-AVAILABLE")
report("checks with the verdict NOT-AVAILABLE", not_available)
check(not_pass == [COMPARISON] and len(other_verdicts) == not_available == 5
      and all_passed + not_available == all_checks
      and sum(engine_totals.values()) == all_checks,
      f"no check FAILS: {all_passed} PASS, {not_available} NOT-AVAILABLE, "
      f"{all_checks} in all")
disagree = []  # reports whose own summary differs from our count
for path, _ in REPORTS:
    data = read_report(path)
    stated = stated_summary(data)
    if stated is None:  # layout C: the self-test states only its verdict
        stated = counted[path] if data["verdict"] == "SUCCESS" else None
    if stated != counted[path]:
        disagree.append(path)
check(disagree == [], "each report states the same totals that we counted")
```

Out [5]:

```
RESULT verifier reports found in the folder Revision = 51
JSON files with a key checks that holds no checks:
    Revision/dark_sector/dirac16complex/outputs/eos-summary.json
```

```
PASS the search finds exactly the 51 reports of the list
report (in the folder Revision)                                          engine  pass of all
algebra/reports/wolfram-algebra.json                                     Wolfram   45 of  45
algebra/reports/python-algebra.json                                      Python    35 of  35
metric_b4/results/wolfram-metric-b4-report.json                          Wolfram   27 of  27
metric_b4/results/python-metric-b4-report.json                           Python    21 of  21
metric_b4/results/vacuum-b4.json                                         Wolfram    6 of   6
metric_b4/results/consequences-b4.json                                   Wolfram    4 of   4
theory/reports/wolfram-field-theory.json                                 Wolfram   84 of  84
theory/reports/python-field-theory.json                                  Python    70 of  70
theory/reports/wolfram-scope.json                                        Wolfram   15 of  15
theory/reports/python-scope.json                                         Python    14 of  14
theory/fock_quartic/reports/fock-quartic.json                            Python    21 of  21
theory/reports/wolfram-field-theory-b4.json                              Wolfram   19 of  19
theory/reports/python-field-theory-b4.json                               Python     8 of   8
theory/reports/wolfram-field-theory-b4-emt-C.json                        Wolfram    4 of   4
theory/reports/wolfram-field-theory-b4-emt-G.json                        Wolfram    4 of   4
field_equations_a4/reports/wolfram-a4-report.json                        Wolfram   64 of  64
field_equations_a4/reports/python-a4-report.json                         Python    63 of  63
field_equations_a4/reports/python-b4-block.json                          Python    11 of  11
field_equations_a4/reports/ks-source-conditions.json                     Python     6 of   6
field_equations_a4/ks_source/reports/ks-source-a4.json                   Python    23 of  23
gkd_lovelock/results/lovelock-report.json                                Rust      19 of  19
gkd_lovelock/results/gkd-selftest.json                                   Rust       9 of   9
gkd_lovelock/results/wolfram-gkd-report.json                             Wolfram   29 of  29
gkd_lovelock/results/python-lovelock-report.json                         Python    49 of  49
gkd_lovelock/comparison/author-comparison-report.json                    Python    73 of  78
kohn_sham/reports/ks-theory-wolfram.json                                 Wolfram   46 of  46
kohn_sham/reports/ks-theory-python.json                                  Python    58 of  58
kohn_sham/reports/ks-b4-wolfram.json                                     Wolfram    4 of   4
kohn_sham/reports/ks-rust-solver.json                                    Rust      42 of  42
kohn_sham/reports/ks-rust-determinism.json                               Python    14 of  14
kohn_sham/reports/ks-rust-mermin-roots.json                              Python     5 of   5
kohn_sham/reports/ks-reference.json                                      Python    37 of  37
kohn_sham/reports/ks-crosscheck.json                                     Python    31 of  31
kohn_sham/tip_convergence/tip-convergence.json                           Python     7 of   7
dark_sector/dirac16complex/reports/derivation-checks.json                Python    30 of  30
dark_sector/dirac16complex/reports/ks-history-run.json                   Python     5 of   5
dark_sector/dirac16complex/reports/eos-checks.json                       Python    13 of  13
dark_sector/dirac16complex/reports/independent-checks.json               Python     9 of   9
dark_sector/dirac16complex00/reports/python-derive-eos.json              Python    49 of  49
dark_sector/dirac16complex00/reports/python-independent-numerics.json Python    28 of  28
pairing/reports/wolfram-pairing.json                                     Wolfram  101 of 101
pairing/reports/python-pairing.json                                      Python    66 of  66
pairing/kohn_sham/reports/wolfram-t3.json                                Wolfram   10 of  10
pairing/kohn_sham/reports/python-t3.json                                 Python    13 of  13
pairing/kohn_sham/reports/wolfram-t3-completion.json                     Wolfram    3 of   3
pairing/kohn_sham/reports/python-t3-completion.json                      Python     7 of   7
pairing/kohn_sham/reports/t3-rust-demo.json                              Python     7 of   7
pairing/kohn_sham/reports/t3-reference-demo.json                         Python     7 of   7
lead_checks/reports/charge-conjugation-and-u1.json                       lead      12 of  12
lead_checks/reports/einstein-gauss-bonnet-a4.json                        lead      15 of  15
lead_checks/reports/emt-divergence-and-spin-connection.json              lead      10 of  10
RESULT reports = 51
```

```
RESULT checks in all reports = 1357
RESULT checks with the verdict PASS = 1352
RESULT checks done with the engine Wolfram = 465
RESULT checks done with the engine Python = 785
RESULT checks done with the engine Rust = 70
RESULT checks done with the engine lead = 37
RESULT checks with the verdict NOT-AVAILABLE = 5
PASS no check FAILS: 1352 PASS, 5 NOT-AVAILABLE, 1357 in all
PASS each report states the same totals that we counted
```

Text cell:

```
The next cell compares our counts with three other places of the record that quote
them. First, the table of the file Revision/README.md (its lines that start with
a vertical bar and a folder name) prints 49 counts as "19/19", "49/49" and so on;
the counts of the reports on the Lovelock tensors stand there twice, in the row
of their folder and in the row of the documents. The cell finds the counts in the
order in which they stand there and compares them with our counts of the reports
that the table names in that order. The function ‘re.findall‘ of the module ‘re‘
finds every piece of a text that matches a *pattern*: in the pattern
‘(\d+)/(\d+)‘, ‘\d+‘ means one or more digits, and the brackets mark the two
numbers to return. Second, the Kohn-Sham cross-check quotes, in the detail of its
check ‘inputs_all_pass‘, the counts of the three reports it read ("37/37 PASS" and
so on), found with the pattern ‘(\d+)/(\d+) PASS‘. Third, the dark-sector summary
file that the search set aside quotes, under its key ‘checks‘, the totals of the
report eos-checks.json.
```

In [6]:

```
R = "Revision/"
README_ORDER = [  # the reports whose counts the README table quotes, in its order
    R + "gkd_lovelock/results/lovelock-report.json",  # row gkd_lovelock/
    R + "gkd_lovelock/results/python-lovelock-report.json",
    R + "gkd_lovelock/results/wolfram-gkd-report.json",
    R + "metric_b4/results/wolfram-metric-b4-report.json",  # row metric_b4/
    R + "metric_b4/results/python-metric-b4-report.json",
    R + "metric_b4/results/vacuum-b4.json",
    R + "metric_b4/results/consequences-b4.json",
    R + "algebra/reports/wolfram-algebra.json",  # row algebra/
    R + "algebra/reports/python-algebra.json",
    R + "theory/reports/wolfram-field-theory.json",  # row theory/
    R + "theory/reports/python-field-theory.json",
    R + "theory/reports/wolfram-scope.json",
    R + "theory/reports/python-scope.json",
    R + "theory/fock_quartic/reports/fock-quartic.json",
    R + "theory/reports/wolfram-field-theory-b4.json",
    R + "theory/reports/python-field-theory-b4.json",
    R + "theory/reports/wolfram-field-theory-b4-emt-C.json",
    R + "theory/reports/wolfram-field-theory-b4-emt-G.json",
    R + "field_equations_a4/reports/wolfram-a4-report.json",  # row field_equations_a4/
    R + "field_equations_a4/reports/python-a4-report.json",
    R + "field_equations_a4/reports/python-b4-block.json",
```

```
    R + "field_equations_a4/reports/ks-source-conditions.json",
    R + "field_equations_a4/ks_source/reports/ks-source-a4.json",
    R + "kohn_sham/reports/ks-theory-wolfram.json",  # row kohn_sham/
    R + "kohn_sham/reports/ks-theory-python.json",
    R + "kohn_sham/reports/ks-b4-wolfram.json",
    R + "kohn_sham/reports/ks-rust-solver.json",
    R + "kohn_sham/reports/ks-rust-determinism.json",
    R + "kohn_sham/reports/ks-rust-mermin-roots.json",
    R + "kohn_sham/reports/ks-reference.json",
    R + "kohn_sham/reports/ks-crosscheck.json",
    R + "kohn_sham/tip_convergence/tip-convergence.json",
    R + "dark_sector/dirac16complex/reports/derivation-checks.json",  # dark_sector/
    R + "dark_sector/dirac16complex/reports/ks-history-run.json",
    R + "dark_sector/dirac16complex/reports/eos-checks.json",
    R + "dark_sector/dirac16complex/reports/independent-checks.json",
    R + "dark_sector/dirac16complex00/reports/python-derive-eos.json",
    R + "dark_sector/dirac16complex00/reports/python-independent-numerics.json",
    R + "pairing/reports/wolfram-pairing.json",  # row pairing/
    R + "pairing/reports/python-pairing.json",
    R + "pairing/kohn_sham/reports/wolfram-t3.json",
    R + "pairing/kohn_sham/reports/python-t3.json",
    R + "pairing/kohn_sham/reports/wolfram-t3-completion.json",
    R + "pairing/kohn_sham/reports/python-t3-completion.json",
    R + "pairing/kohn_sham/reports/t3-rust-demo.json",
    R + "pairing/kohn_sham/reports/t3-reference-demo.json",
    R + "gkd_lovelock/results/lovelock-report.json",  # row docs/: LOVELOCK_GKD
    R + "gkd_lovelock/results/wolfram-gkd-report.json",
    R + "gkd_lovelock/results/python-lovelock-report.json",
]
readme_lines = repository_file("Revision/README.md").read_text(
    encoding="utf-8").split("\n")
table = "\n".join(line for line in readme_lines if line.startswith("| `"))
quoted_readme = [(int(p), int(t)) for p, t in re.findall(r"(\d+)/(\d+)", table)]
say("the README quotes: " + ", ".join(f"{p}/{t}" for p, t in quoted_readme))
report("counts quoted in the README table", len(quoted_readme))
check_reproduces(quoted_readme == [counted[path] for path in README_ORDER],
                 f"the {len(README_ORDER)} counts quoted in the README equal ours",
                 "Revision/README.md, the table of the folders")

CROSS = "Revision/kohn_sham/reports/ks-crosscheck.json"
detail = next(entry["detail"] for entry in read_report(CROSS)["checks"]
              if entry["name"] == "inputs_all_pass")
quoted = re.findall(r"(\d+)/(\d+) PASS", detail)  # [("37", "37"), ("42", "42"), ...]
say("the cross-check quotes: " + ", ".join(f"{p}/{t}" for p, t in quoted))
ours = [counted["Revision/kohn_sham/reports/ks-reference.json"],
        counted["Revision/kohn_sham/reports/ks-rust-solver.json"],
        counted["Revision/kohn_sham/reports/ks-rust-determinism.json"]]
numbers = ", ".join(str(total) for _, total in ours)  # e.g. "37, 42, 14"
check_reproduces([(int(p), int(t)) for p, t in quoted] == ours,
                 f"the cross-check quotes the counts {numbers} that we counted",
                 f"{CROSS}, check inputs_all_pass")

SUMMARY = "Revision/dark_sector/dirac16complex/outputs/eos-summary.json"
EOS = "Revision/dark_sector/dirac16complex/reports/eos-checks.json"
```

```
summary_numbers = read_report(SUMMARY)["checks"]  # a dictionary of two numbers
say(f"the dark-sector summary quotes: {summary_numbers}")
quoted_eos = (summary_numbers["pass"], summary_numbers["total"])
check_reproduces(not_reports == [SUMMARY] and quoted_eos == counted[EOS],
                 f"the dark-sector summary quotes the {counted[EOS][1]} checks of "
                 "eos-checks.json that we counted",
                 f"{SUMMARY}, key checks")
```

Out [6]:

```
the README quotes: 19/19, 49/49, 29/29, 27/27, 21/21, 6/6, 4/4, 45/45, 35/35, 84/84,
    70/70, 15/15, 14/14, 21/21, 19/19, 8/8, 4/4, 4/4, 64/64, 63/63, 11/11, 6/6, 23/23,
    46/46, 58/58, 4/4, 42/42, 14/14, 5/5, 37/37, 31/31, 7/7, 30/30, 5/5, 13/13, 9/9,
    49/49, 28/28, 101/101, 66/66, 10/10, 13/13, 3/3, 7/7, 7/7, 7/7, 19/19, 29/29, 49/49
RESULT counts quoted in the README table = 49
PASS the 49 counts quoted in the README equal ours
     reproduces Revision/README.md, the table of the folders
the cross-check quotes: 37/37, 42/42, 14/14
PASS the cross-check quotes the counts 37, 42, 14 that we counted
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check inputs_all_pass
the dark-sector summary quotes: {'total': 13, 'pass': 13}
PASS the dark-sector summary quotes the 13 checks of eos-checks.json that we counted
     reproduces Revision/dark_sector/dirac16complex/outputs/eos-summary.json, key checks
```

Text cell:

```
The next cell draws the number of checks of every report as a horizontal bar,
coloured by the engine that did the computation, with the number written at the
end of each bar. The bar of the comparison with the author's outputs counts all
its 78 checks, the five NOT-AVAILABLE ones included.
```

In [7]:

```
from matplotlib.patches import Patch  # a coloured square for the legend

ENGINE_COLOURS = {"Wolfram": "#2a78d6", "Python": "#eb6834", "Rust": "#1baf7a",
                  "lead": "#eda100"}
ENGINE_NAMES = {"Wolfram": "Wolfram Language", "Python": "Python",
                "Rust": "Rust", "lead": "Python, the lead's independent checks"}

# The report with the most checks, and its number of checks.
largest = max((path for path, _ in REPORTS), key=lambda path: counted[path][1])
longest = counted[largest][1]
fig, ax = plt.subplots(figsize=(7.6, 11.6))
rows = np.arange(len(REPORTS))[::-1]  # the first report at the top
for row, (path, engine) in zip(rows, REPORTS):
    total = counted[path][1]
    ax.barh(row, total, height=0.72, color=ENGINE_COLOURS[engine])
    ax.text(total + 0.015 * longest, row, str(total), va="center", fontsize=9)
# The name of each report without its folder and without the ending .json:
names = [path.rsplit("/", 1)[1].removesuffix(".json") for path, _ in REPORTS]
```

```
ax.set_yticks(rows, labels=names, fontsize=8.5)
ax.set_xlim(0, 1.12 * longest)  # room for the longest bar and its number
ax.grid(False, axis="y")  # vertical grid lines only
ax.set_xlabel("number of checks in the report (none has the verdict FAIL)")
ax.set_title(f"The {len(REPORTS)} verifier reports of the Revision record: "
             f"{all_checks} checks")
ax.legend(handles=[Patch(color=ENGINE_COLOURS[e], label=ENGINE_NAMES[e])
                   for e in ENGINE_COLOURS], loc="lower right", fontsize=8)
largest_name = largest.rsplit("/", 1)[1]  # its file name without the folders
n_wolfram, n_python, n_rust, n_lead = (engine_totals[engine] for engine in
                                       ("Wolfram", "Python", "Rust", "lead"))
totals_text = (f"{n_wolfram} Wolfram Language, {n_python} Python, {n_rust} Rust "
               f"and {n_lead} lead checks")
save_figure(fig, "checks_by_report",
            f"The number of checks in each of the {len(REPORTS)} verifier reports "
            r"of the Revision record (horizontal axis, a count; one bar per report, "
            r"named on the vertical axis and grouped by folder: algebra, the metric "
            r"with $b_4$, theory, field equations for $a_4$ and $b_4$, GKD and "
            r"Lovelock with the comparison with the author's outputs, Kohn-Sham, "
            r"dark sector, pairing, lead checks). "
            r"The colour gives the engine: blue Wolfram Language, orange Python, "
            r"aqua Rust, yellow the lead's independent Python checks. Of the "
            f"{all_checks} checks ({totals_text}) {all_passed} have the verdict "
            f"PASS and {not_available} the verdict NOT-AVAILABLE (comparisons with "
            r"values that the author's notebook does not store); none FAILS. The "
            f"largest report is {largest_name} with {longest} checks.")
```

Out [7]:

```
Figure 00c.1 saved as Revision/textbook/figures/00c_1_checks_by_report.png
```

Text cell:

```
## 8. Two independent engines

For eleven subjects the Revision record has two independent verifiers, one in
Wolfram Language and one in Python. The next cell draws, for each subject, the
number of checks of the two verifiers side by side. The two verifiers do not
check exactly the same list of statements (each also checks things the other does
not), so the numbers differ; the point is that two programs, written separately in
two different languages, both find the results of each subject correct.
```

In [8]:

```
SUBJECTS = [  # (subject, Wolfram report, Python report)
    ("the gammas, Pin(4,4), Spin(4,4)", "algebra/reports/wolfram-algebra.json",
     "algebra/reports/python-algebra.json"),
    ("the metric with b4", "metric_b4/results/wolfram-metric-b4-report.json",
     "metric_b4/results/python-metric-b4-report.json"),
    ("geometry and Dirac operator with b4",
     "theory/reports/wolfram-field-theory-b4.json",
```

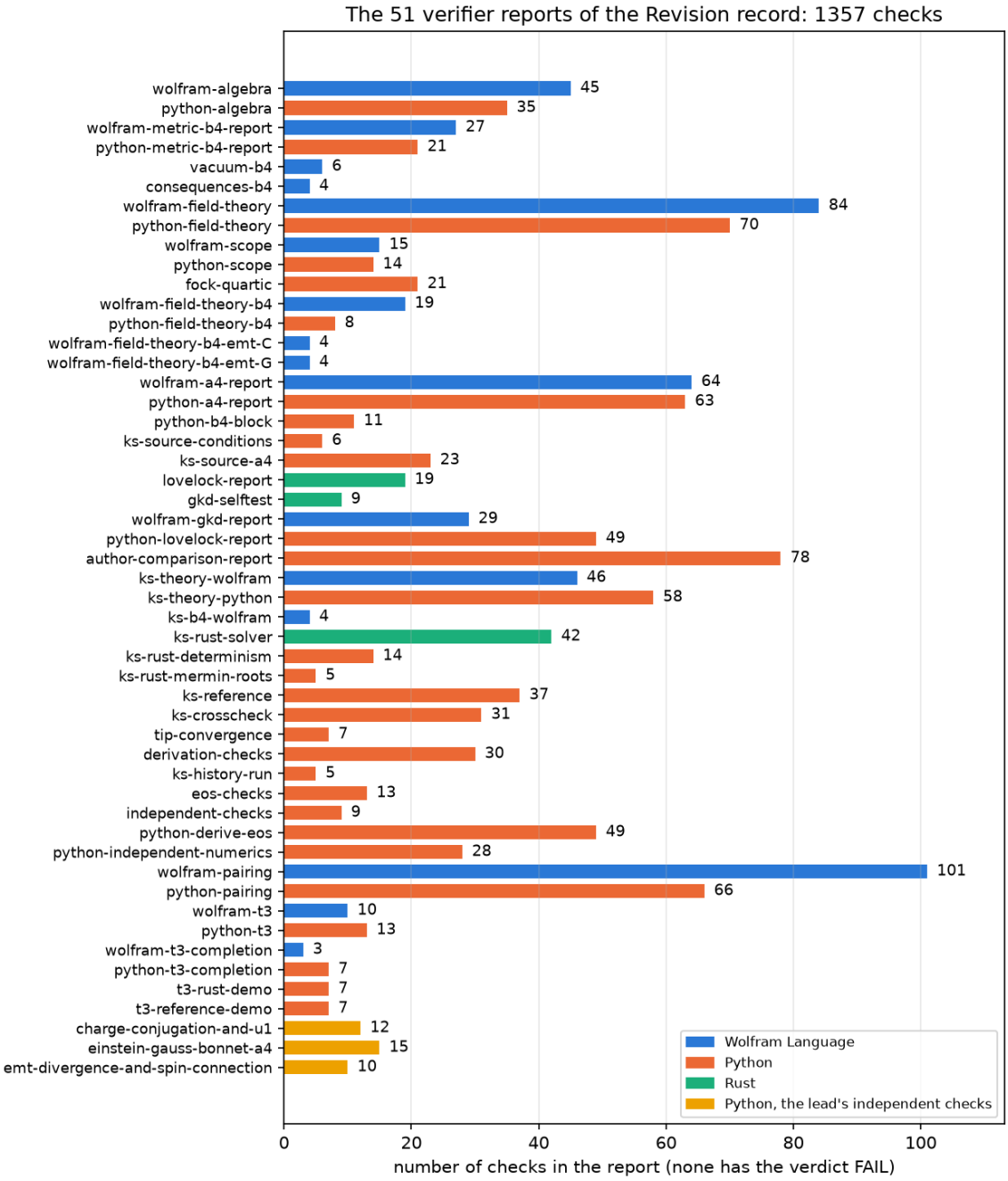

Figure 7: The number of checks in each of the 51 verifier reports of the Revision record (horizontal axis, a count; one bar per report, named on the vertical axis and grouped by folder: algebra, the metric with b_4, theory, field equations for a_4 and b_4, GKD and Lovelock with the comparison with the author's outputs, Kohn-Sham, dark sector, pairing, lead checks). The colour gives the engine: blue Wolfram Language, orange Python, aqua Rust, yellow the lead's independent Python checks. Of the 1357 checks (465 Wolfram Language, 785 Python, 70 Rust and 37 lead checks) 1352 have the verdict PASS and 5 the verdict NOT-AVAILABLE (comparisons with values that the author's notebook does not store); none FAILS. The largest report is wolfram-pairing.json with 101 checks. (Notebook 00c, figure 1.)

```
     "theory/reports/python-field-theory-b4.json"),
    ("Lagrangians, field equations, EMT", "theory/reports/wolfram-field-theory.json",
     "theory/reports/python-field-theory.json"),
    ("scope of the non-triviality", "theory/reports/wolfram-scope.json",
     "theory/reports/python-scope.json"),
    ("field equations for a4", "field_equations_a4/reports/wolfram-a4-report.json",
     "field_equations_a4/reports/python-a4-report.json"),
    ("GKD and the Lovelock tensors", "gkd_lovelock/results/wolfram-gkd-report.json",
     "gkd_lovelock/results/python-lovelock-report.json"),
    ("Kohn-Sham theory", "kohn_sham/reports/ks-theory-wolfram.json",
     "kohn_sham/reports/ks-theory-python.json"),
    ("pairing theorems T1, T2, Q", "pairing/reports/wolfram-pairing.json",
     "pairing/reports/python-pairing.json"),
    ("pairing theorem T3", "pairing/kohn_sham/reports/wolfram-t3.json",
     "pairing/kohn_sham/reports/python-t3.json"),
    ("the completion of T3", "pairing/kohn_sham/reports/wolfram-t3-completion.json",
     "pairing/kohn_sham/reports/python-t3-completion.json"),
]
say("subject                                Wolfram  Python")
wolfram_numbers, python_numbers = [], []
for subject, wolfram, python in SUBJECTS:
    wolfram_numbers.append(counted["Revision/" + wolfram][1])
    python_numbers.append(counted["Revision/" + python][1])
    say(f"{subject:36} {wolfram_numbers[-1]:7d} {python_numbers[-1]:7d}")

widest = max(wolfram_numbers + python_numbers)  # the longest of the 22 bars
fig, ax = plt.subplots(figsize=(7.0, 6.6))
rows = np.arange(len(SUBJECTS))[::-1]
height = 0.38  # two bars in each row
ax.barh(rows + height / 2, wolfram_numbers, height, color="#2a78d6",
        edgecolor="white", linewidth=1.5, label="Wolfram Language verifier")
ax.barh(rows - height / 2, python_numbers, height, color="#eb6834",
        edgecolor="white", linewidth=1.5, label="Python verifier (sympy)")
for row, w, p in zip(rows, wolfram_numbers, python_numbers):
    ax.text(w + 0.015 * widest, row + height / 2, str(w), va="center", fontsize=8)
    ax.text(p + 0.015 * widest, row - height / 2, str(p), va="center", fontsize=8)
ax.set_yticks(rows, labels=[subject for subject, _, _ in SUBJECTS])
ax.set_xlim(0, 1.12 * widest)  # room for the longest bar and its number
ax.grid(False, axis="y")
ax.set_xlabel("number of checks (every one has the verdict PASS)")
ax.set_title("Eleven subjects, each checked by two independent verifiers")
ax.legend(loc="lower right")
save_figure(fig, "two_verifiers",
            r"For each of eleven subjects of the Revision record (vertical axis), "
            r"the number of checks of its Wolfram Language verifier (blue, upper "
            r"bar) and of its independent Python verifier (orange, lower bar); "
            r"horizontal axis a count. The two verifiers share no code; each also "
            r"checks statements that the other does not, so the numbers differ. "
            f"Together they hold {sum(wolfram_numbers)} Wolfram and "
            f"{sum(python_numbers)} Python checks, all PASS.")
report("checks of the Wolfram verifiers of the eleven subjects",
       sum(wolfram_numbers))
report("checks of the Python verifiers of the eleven subjects", sum(python_numbers))
check(all(w > 0 and p > 0 for w, p in zip(wolfram_numbers, python_numbers)),
```

```
"each of the eleven subjects has a Wolfram and a Python verifier")
```

Out [8]:

subject	Wolfram	Python
the gammas, Pin(4,4), Spin(4,4)	45	35
the metric with b4	27	21
geometry and Dirac operator with b4	19	8
Lagrangians, field equations, EMT	84	70
scope of the non-triviality	15	14
field equations for a4	64	63
GKD and the Lovelock tensors	29	49
Kohn-Sham theory	46	58
pairing theorems T1, T2, Q	101	66
pairing theorem T3	10	13
the completion of T3	3	7

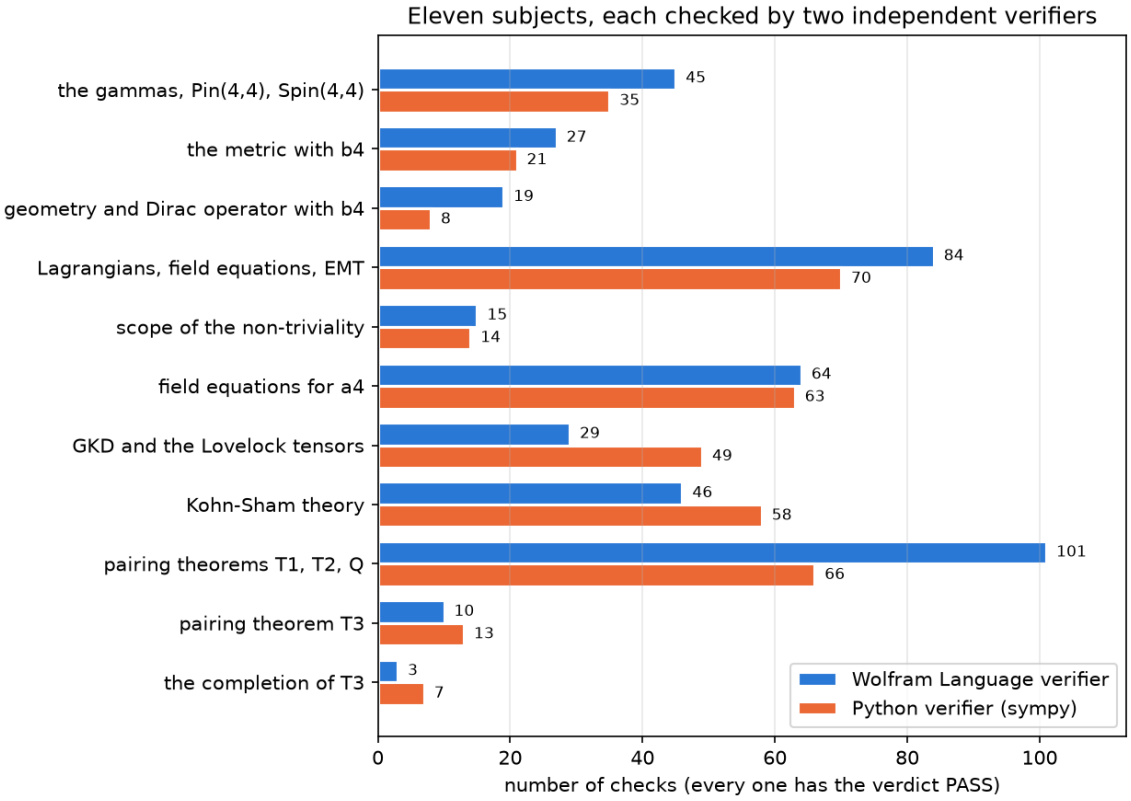

Figure 8: For each of eleven subjects of the Revision record (vertical axis), the number of checks of its Wolfram Language verifier (blue, upper bar) and of its independent Python verifier (orange, lower bar); horizontal axis a count. The two verifiers share no code; each also checks statements that the other does not, so the numbers differ. Together they hold 443 Wolfram and 404 Python checks, all PASS. (Notebook 00c, figure 2.)

Out [8]:

```
Figure 00c.2 saved as Revision/textbook/figures/00c_2_two_verifiers.png
RESULT checks of the Wolfram verifiers of the eleven subjects = 443
RESULT checks of the Python verifiers of the eleven subjects = 404
PASS each of the eleven subjects has a Wolfram and a Python verifier
```

Text cell:

```
## 9. The honesty ledger

The next cell writes the ledger: twenty rows, each with a statement of the book,
its label, a short note where one is needed, and the reports that verify it. Every
one of the 51 reports belongs to exactly one row. The five rows labelled HYPOTHESIS
or OPEN have no report: nothing in the record establishes them, and their notes
say why. Two rows of PROVED theorems name an assumption in their statement: the
pairing theorems T2 and T3 hold with the Z2 mirror (a choice of boundary
condition) ASSUMED. Four rows (8, 9, 12 and 13) rest, in addition, on the
Kohn-Sham approximation, which is ASSUMED: a set of simplifications, stated in the
record and explained from zero later in the book. The equations are solved at one
fixed instant at a time; the action of the field on itself is replaced by an
average potential computed from the field's own density, in a stated approximate
form; and a condition at the far end of the hidden direction is chosen. Rows 8 and
12 are exact statements about this approximation, and rows 9 and 13 are numbers
computed in it. Row 2 says "dirac16complex quantised" because only that field is
quantised; the field dirac16complex00 stays a classical field. The row of the
Kohn-Sham history of $a_4$ is labelled ASSUMED
although it has 29 checks: those checks show that the computed Kohn-Sham states
cannot be the source of that history in the field equations, so the history has
to be assumed (a *prescribed background*). Three rows with reports have a note as
well. Row 2: for a self-coupling $\lambda \neq 0$ the operator form of the
energy-momentum tensor (EMT) of dirac16complex is proved only in a finite model
(one set of 16 modes, checked by one Python program, its third report); for the
field on a whole slice of space it is OPEN: nothing proves it there, and the
record of the field equations for $a_4$ assumes it. Row 7,
the comparisons of the Revision's curvature with the values that the author stored
in their own notebooks (the earlier notebook at $b_4 = 1$, and the notebook with
the metric function $b_4$ of 2026-10-14): five comparisons of the first report
could not be made (NOT-AVAILABLE).
Row 9: the Kohn-Sham states are computed with the far end of the hidden direction
cut off at $L = 3$; a study of larger $L$ (its sixth report) finds that the limit
of large $L$ is not established for every state.

Rows 13 and 15 are labelled COMPUTED as well, because they rest on numerical
computations. Row 13: the theorem T3 is proved (row 12) and, in addition,
demonstrated on computed Kohn-Sham states of mass $+M$ and $-M$; a demonstration
is not a proof.
Row 15: the author's dark-sector hypotheses were investigated; the investigation
derives exact identities, but its results, the equations of state that each field
can give, are computed numbers under stated assumptions about the observer. It
states what each field can and cannot produce; it does not establish the
hypotheses (row 16, HYPOTHESIS).

Several rows concern pairs of universes and matter and antimatter, and their words
are chosen with care. PROVED (rows 11 and 12) are exact maps between the solutions
```

of mass $+m$ and those of mass $-m$; the map T1 also reverses the charge, so a solution and its image carry opposite charges. That our universe actually has such a partner is a HYPOTHESIS (row 17: an idea of the universe and anti-universe kind, not a result). PROVED (row 14) is the local conservation law of the charge: no charge is made or destroyed at any point. That the total charge of one universe stays constant is OPEN (row 18): it holds only if no charge flows through the brane at $z = \pi/2$, a no-flux condition that is ASSUMED, not derived, and that fails on the exact homogeneous solutions found later in the book. That the big bang creates universes in pairs is not proved: no creation process, rate or amplitude follows from the equations (row 19, OPEN). The theory as built has no baryons (the particles of ordinary matter such as the proton), no process that changes their number and no violation of the CP symmetry (the exchange of particles and antiparticles combined with a mirror reflection; it must be violated for matter to win over antimatter), so it does not produce the excess of matter over antimatter that we observe. What does produce it is a question that neither the record nor the book answers (row 20, OPEN).

The cell prints the ledger and the notes, and checks that every report is used exactly once; that the HYPOTHESIS and OPEN rows have a note and no report, and every other row has reports; that only row 7 has checks that are not PASS, its five NOT-AVAILABLE comparisons; and it counts the labels.

In [9]:

```
R = "Revision/"
K = R + "pairing/kohn_sham/reports/"  # the folder of the T3 reports
D = R + "dark_sector/"  # the folder of the dark-sector reports
LEDGER = [  # (statement, label, note, the reports that verify it)
    ("the gammas, C, Gamma, B; Pin(4,4) and Spin(4,4)", "PROVED", "",
     [R + "algebra/reports/wolfram-algebra.json",
      R + "algebra/reports/python-algebra.json"]),
    ("Lagrangians, equations, EMT; dirac16complex quantised", "PROVED",
     "the EMT operator for lambda != 0: proved only in a finite Fock model",
     [R + "theory/reports/wolfram-field-theory.json",
      R + "theory/reports/python-field-theory.json",
      R + "theory/fock_quartic/reports/fock-quartic.json",
      R + "theory/reports/wolfram-field-theory-b4.json",
      R + "theory/reports/python-field-theory-b4.json",
      R + "theory/reports/wolfram-field-theory-b4-emt-C.json",
      R + "theory/reports/wolfram-field-theory-b4-emt-G.json"]),
    ("the exact scope of the non-triviality", "PROVED", "",
     [R + "theory/reports/wolfram-scope.json",
      R + "theory/reports/python-scope.json"]),
    ("EMT conservation identities; the spin connection", "PROVED", "",
     [R + "lead_checks/reports/emt-divergence-and-spin-connection.json"]),
    ("the field equations for a4 and b4; the vacuum", "PROVED", "",
     [R + "field_equations_a4/reports/wolfram-a4-report.json",
      R + "field_equations_a4/reports/python-a4-report.json",
      R + "field_equations_a4/reports/python-b4-block.json",
      R + "metric_b4/results/vacuum-b4.json",
      R + "lead_checks/reports/einstein-gauss-bonnet-a4.json"]),
    ("GKD and the three Lovelock tensors", "PROVED", "",
     [R + "gkd_lovelock/results/lovelock-report.json",
```

```
      R + "gkd_lovelock/results/gkd-selftest.json",
      R + "gkd_lovelock/results/wolfram-gkd-report.json",
      R + "gkd_lovelock/results/python-lovelock-report.json"]),
    ("the curvature agrees with the author's stored outputs", "PROVED",
     "5 of 78 comparisons NOT-AVAILABLE: the author's notebook stores no value",
     [R + "gkd_lovelock/comparison/author-comparison-report.json",
      R + "metric_b4/results/wolfram-metric-b4-report.json",
      R + "metric_b4/results/python-metric-b4-report.json"]),
    ("Kohn-Sham theory: blocks, rescaling, exchange", "PROVED", "",
     [R + "kohn_sham/reports/ks-theory-wolfram.json",
      R + "kohn_sham/reports/ks-theory-python.json",
      R + "kohn_sham/reports/ks-b4-wolfram.json"]),
    ("Kohn-Sham states along the deflating history", "COMPUTED",
     "at the tip cutoff L = 3; the large-L limit not established for every state",
     [R + "kohn_sham/reports/ks-rust-solver.json",
      R + "kohn_sham/reports/ks-reference.json",
      R + "kohn_sham/reports/ks-crosscheck.json",
      R + "kohn_sham/reports/ks-rust-determinism.json",
      R + "kohn_sham/reports/ks-rust-mermin-roots.json",
      R + "kohn_sham/tip_convergence/tip-convergence.json"]),
    ("the Kohn-Sham history of a4 is a prescribed background", "ASSUMED", "",
     [R + "field_equations_a4/reports/ks-source-conditions.json",
      R + "field_equations_a4/ks_source/reports/ks-source-a4.json"]),
    ("pairing T1, T2 (Z2 mirror ASSUMED) and Q", "PROVED", "",
     [R + "pairing/reports/wolfram-pairing.json",
      R + "pairing/reports/python-pairing.json",
      R + "metric_b4/results/consequences-b4.json"]),
    ("T3 and its completion (Z2 mirror ASSUMED): +M and -M", "PROVED", "",
     [K + "wolfram-t3.json", K + "python-t3.json",
      K + "wolfram-t3-completion.json", K + "python-t3-completion.json"]),
    ("T3 shown on computed Kohn-Sham states (not a proof)", "COMPUTED", "",
     [K + "t3-rust-demo.json", K + "t3-reference-demo.json"]),
    ("charge conjugation C, Gamma C; the local U(1) law", "PROVED", "",
     [R + "lead_checks/reports/charge-conjugation-and-u1.json"]),
    ("the dark-sector investigation: what each field gives", "COMPUTED", "",
     [D + "dirac16complex/reports/derivation-checks.json",
      D + "dirac16complex/reports/ks-history-run.json",
      D + "dirac16complex/reports/eos-checks.json",
      D + "dirac16complex/reports/independent-checks.json",
      D + "dirac16complex00/reports/python-derive-eos.json",
      D + "dirac16complex00/reports/python-independent-numerics.json"]),
    ("a time-varying dark sector from the fields", "HYPOTHESIS",
     "investigated (row 15): results stated, not established", []),
    ("our universe has a partner of opposite charge", "HYPOTHESIS",
     "the T1 maps exist; that a partner exists is not shown", []),
    ("the total charge Q of one universe is constant", "OPEN",
     "holds only with no flux through the brane (ASSUMED, not derived)", []),
    ("the big bang creates universes in pairs", "OPEN",
     "not proved: no creation process, rate or amplitude", []),
    ("what produces the excess of matter over antimatter", "OPEN",
     "the theory as built does not produce it", []),
]
say("row label       passed of all  statement")
row_passed, row_totals = [], []
```

```
for number, (statement, label, note, paths) in enumerate(LEDGER, 1):
    passed = sum(counted[path][0] for path in paths)
    total = sum(counted[path][1] for path in paths)
    row_passed.append(passed)
    row_totals.append(total)
    say(f"{number:3d} {label:10} {passed:7d} of {total:3d}  {statement}")
say("The notes of the rows:")
for number, (statement, label, note, paths) in enumerate(LEDGER, 1):
    if note:  # rows 2, 7 and 9, and the HYPOTHESIS and OPEN rows
        say(f"{number:3d} {label:10} {note}")
used = sorted(path for _, _, _, paths in LEDGER for path in paths)
check(used == sorted(path for path, _ in REPORTS),
      f"every one of the {len(REPORTS)} reports belongs to exactly one row")
check(all((label in ("HYPOTHESIS", "OPEN")) == (paths == [] and note != "")
          for _, label, note, paths in LEDGER)
      and all(paths != [] for _, label, _, paths in LEDGER
              if label not in ("HYPOTHESIS", "OPEN")),
      "OPEN and HYPOTHESIS rows have a note and no report; the others have reports")
not_all_pass = [number for number, passed, total
                in zip(range(1, len(LEDGER) + 1), row_passed, row_totals)
                if passed != total]  # the rows with a check that is not PASS
check(not_all_pass == [7] and row_totals[6] - row_passed[6] == not_available
      and "NOT-AVAILABLE" in LEDGER[6][2],
      f"only row 7 has checks that are not PASS: its {not_available} "
      "NOT-AVAILABLE comparisons")
labels = [label for _, label, _, _ in LEDGER]
check([labels.count(name) for name in
       ("PROVED", "COMPUTED", "ASSUMED", "HYPOTHESIS", "OPEN")] == [11, 3, 1, 2, 3],
      "the ledger: 11 PROVED, 3 COMPUTED, 1 ASSUMED, 2 HYPOTHESIS and 3 OPEN rows")
```

Out [9]:

```
row label       passed of all  statement
  1 PROVED          80 of  80  the gammas, C, Gamma, B; Pin(4,4) and Spin(4,4)
  2 PROVED         210 of 210  Lagrangians, equations, EMT; dirac16complex quantised
  3 PROVED          29 of  29  the exact scope of the non-triviality
  4 PROVED          10 of  10  EMT conservation identities; the spin connection
  5 PROVED         159 of 159  the field equations for a4 and b4; the vacuum
  6 PROVED         106 of 106  GKD and the three Lovelock tensors
  7 PROVED         121 of 126  the curvature agrees with the author's stored outputs
  8 PROVED         108 of 108  Kohn-Sham theory: blocks, rescaling, exchange
  9 COMPUTED       136 of 136  Kohn-Sham states along the deflating history
 10 ASSUMED         29 of  29  the Kohn-Sham history of a4 is a prescribed background
 11 PROVED         171 of 171  pairing T1, T2 (Z2 mirror ASSUMED) and Q
 12 PROVED          33 of  33  T3 and its completion (Z2 mirror ASSUMED): +M and -M
 13 COMPUTED        14 of  14  T3 shown on computed Kohn-Sham states (not a proof)
 14 PROVED          12 of  12  charge conjugation C, Gamma C; the local U(1) law
 15 COMPUTED       134 of 134  the dark-sector investigation: what each field gives
 16 HYPOTHESIS       0 of   0  a time-varying dark sector from the fields
 17 HYPOTHESIS       0 of   0  our universe has a partner of opposite charge
 18 OPEN             0 of   0  the total charge Q of one universe is constant
 19 OPEN             0 of   0  the big bang creates universes in pairs
 20 OPEN             0 of   0  what produces the excess of matter over antimatter
```

```
The notes of the rows:
  2 PROVED     the EMT operator for lambda != 0: proved only in a finite Fock model
  7 PROVED     5 of 78 comparisons NOT-AVAILABLE: the author's notebook stores no value
  9 COMPUTED   at the tip cutoff L = 3; the large-L limit not established for every state
 16 HYPOTHESIS investigated (row 15): results stated, not established
 17 HYPOTHESIS the T1 maps exist; that a partner exists is not shown
 18 OPEN       holds only with no flux through the brane (ASSUMED, not derived)
 19 OPEN       not proved: no creation process, rate or amplitude
 20 OPEN       the theory as built does not produce it
PASS every one of the 51 reports belongs to exactly one row
PASS OPEN and HYPOTHESIS rows have a note and no report; the others have reports
PASS only row 7 has checks that are not PASS: its 5 NOT-AVAILABLE comparisons
PASS the ledger: 11 PROVED, 3 COMPUTED, 1 ASSUMED, 2 HYPOTHESIS and 3 OPEN rows
```

Text cell:

```
The next cell draws the ledger: one bar per row, with the row's number and
statement written just above it, its length the number of checks of the row's
reports, its colour the label. The rows labelled HYPOTHESIS and OPEN have no bar;
their label and their note are written where the bar would begin.
```

In [10]:

```
LABEL_COLOURS = {"PROVED": "#2a78d6", "COMPUTED": "#eb6834", "ASSUMED": "#1baf7a"}
widest = max(row_totals)  # the row with the most checks
fig, ax = plt.subplots(figsize=(8.6, 9.0))
rows = np.arange(len(LEDGER))[::-1]  # the first row of the ledger at the top
for number, row, (statement, label, note, paths), passed, total in zip(
        range(1, len(LEDGER) + 1), rows, LEDGER, row_passed, row_totals):
    # The statement is written just above its bar (va="bottom": the text starts
    # at the given height and extends upwards).
    ax.text(0, row + 0.26, f"{number}. {statement}", va="bottom", fontsize=9.5)
    if total > 0:
        ax.barh(row, total, height=0.42, color=LABEL_COLOURS[label])
        if passed == total:
            after = f"{total} checks: {label}"
        else:  # row 7: say how many checks are NOT-AVAILABLE
            after = f"{passed} PASS, {total - passed} NOT-AVAILABLE: {label}"
        ax.text(total + 0.012 * widest, row, after, va="center", fontsize=9)
    else:  # no report: the label and the note, in grey
        ax.text(0, row, f"{label}: {note}", va="center", fontsize=9,
                color="#52514e")
ax.set_yticks([])  # the statements are written above the bars instead
ax.set_xlim(0, 1.39 * widest)  # room for the longest bar and the text after it
ax.set_ylim(-0.6, len(LEDGER) - 0.1)
ax.grid(False, axis="y")
ax.set_xlabel("number of checks in the reports of the row (none FAILS)")
ax.set_title("The honesty ledger at a glance")
ax.legend(handles=[Patch(color=colour, label=label_name)
                   for label_name, colour in LABEL_COLOURS.items()],
          loc="lower right", fontsize=9)
save_figure(fig, "ledger",
```

```
        r"The honesty ledger of the book at a glance: one row per main "
        r"statement (written above its bar), the length of its bar the number of "
        r"checks in the reports that verify it (horizontal axis, a count), the "
        r"colour its label: blue PROVED, orange COMPUTED, aqua ASSUMED. Every "
        r"check is PASS except five NOT-AVAILABLE comparisons of row 7. The "
        f"ASSUMED row has {row_totals[9]} checks, which show why the Kohn-Sham "
        r"history of $a_4$ must be assumed. The last five rows have no bar, "
        r"because no check of the record establishes them: two hypotheses (a "
        r"time-varying dark sector, investigated in row 15 but not established; "
        r"a partner universe of opposite charge) and three open questions (a "
        r"constant total charge of one universe, which needs an assumed "
        r"condition at the brane; the creation of universes in pairs, not "
        r"proved; what produces the excess of matter "
        r"over antimatter, which the theory as built does not produce).")
```

Out [10]:

```
Figure 00c.3 saved as Revision/textbook/figures/00c_3_ledger.png
```

Text cell:

The pairing record states in its own words what the pairing theorems do NOT establish: the key ‘not_established‘ of the report Revision/pairing/reports/python-pairing.json is a list of twelve sentences. The next cell prints all twelve. The first says that nothing in the equations creates a universe or a pair of universes. This is why the ledger labels the statement "the big bang creates universes in pairs" OPEN.

In [11]:

```
PAIRING = "Revision/pairing/reports/python-pairing.json"
not_established = read_report(PAIRING)["not_established"]  # a list of sentences
for number, sentence in enumerate(not_established, 1):
    say(f"{number:2d}. {sentence}")
check_reproduces(len(not_established) == 12
                 and not_established[0].startswith("No creation process"),
                 "the pairing record lists 12 things it does not establish, "
                 "the first: no creation process",
                 f"{PAIRING}, key not_established")
```

Out [11]:

1. No creation process: nothing in these equations produces a universe, a pair of universes or a change of the number of universes; no transition between ’no universe’ and ’two universes’ is described.
2. No rate, probability or amplitude for creating a pair is derived; no wave function of the universe, path integral or tunnelling computation is part of these theorems.
3. No conservation law forces pairing: the theorems are symmetries (maps between solutions); a single universe with mass +m is an equally valid solution of the field

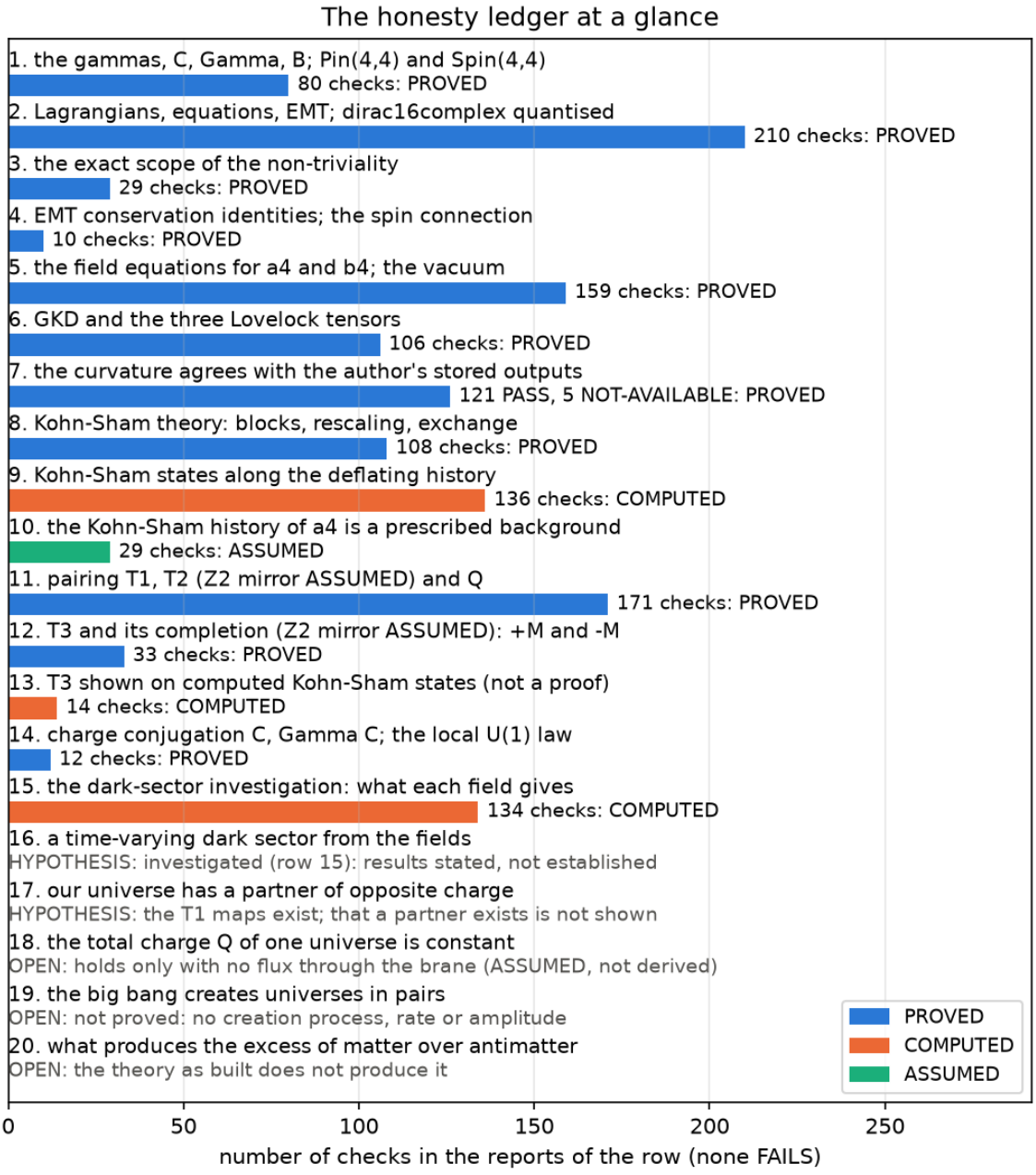

Figure 9: The honesty ledger of the book at a glance: one row per main statement (written above its bar), the length of its bar the number of checks in the reports that verify it (horizontal axis, a count), the colour its label: blue PROVED, orange COMPUTED, aqua ASSUMED. Every check is PASS except five NOT-AVAILABLE comparisons of row 7. The ASSUMED row has 29 checks, which show why the Kohn-Sham history of a_4 must be assumed. The last five rows have no bar, because no check of the record establishes them: two hypotheses (a time-varying dark sector, investigated in row 15 but not established; a partner universe of opposite charge) and three open questions (a constant total charge of one universe, which needs an assumed condition at the brane; the creation of universes in pairs, not proved; what produces the excess of matter over antimatter, which the theory as built does not produce). (Notebook 00c, figure 3.)

```
    equations without its partner.
 4. The vanishing total energy-momentum and charge of a pair (T1) is an identity for a
    field configuration and its image (classical bilinears, or one operator and its
    image); it is not a cancellation between two independently quantised universes
    (Q.no_cancellation_independent_universes).
 5. T2 does not reverse the sign of the energy-momentum tensor: the mirror universe of
    mass -m has the pulled-back T of the original, not -T.
 6. The Z2 brane at z = pi/2 and the mirror construction are ASSUMED (a boundary/orbifold
    choice), not derived from the field equations.
 7. The gravitational back-reaction (the a4 equations of SPEC section 5) is not part of
    the pairing theorems; T1 maps T -> -T, so a pair sourcing ONE common geometry would
    have zero total source, which is a statement about sources, not a derivation that
    such a geometry is created.
 8. No dynamical necessity: nothing forces the partner configuration to exist; the
    theorems are correspondences between solutions of two parameter sets, not a
    mechanism.
 9. T1 pairs (m, lambda) with (-m, -lambda): for lambda != 0 it is not a pure +m / -m
    pairing; the pure pairing (m, lambda) -> (-m, lambda) is T2, at EQUAL (not opposite)
    energy-momentum.
10. Quantum positivity: in flat 4+4 space every real-frequency energy eigenspace of the
    one-particle generator has Krein inertia (4,4) (Q.one_particle_Krein_inertia_proof)
    and every imaginary- or zero-frequency eigenspace (the growing extra-time modes) is
    Krein-neutral (Q.one_particle_complex_frequency_Krein_neutral); a positive-norm Fock
    space for either universe is not established by these theorems.
11. The brane z = pi/2 is a degenerate surface of the metric (g_00 = cot^2 z = 0, sqrt|g|
    = cos z = 0 there, geometry.brane_degenerate); no junction condition, brane tension
    or matching of the field across it is derived.
12. The Kohn-Sham level (T3) is not covered by this checker; it is proved separately in
    Revision/pairing/kohn_sham/ (check_t3.py, with its own hypotheses: ASSUMED Z2 brane,
    instantaneous mean-field Kohn-Sham states).
PASS the pairing record lists 12 things it does not establish, the first: no creation
    process
     reproduces Revision/pairing/reports/python-pairing.json, key not_established
```

Text cell:

```
## 10. Fingerprints

A sha256 fingerprint is computed from the bytes of a file (or of a text) by the
function ‘hashlib.sha256‘; ‘.hexdigest()‘ writes it as 64 hexadecimal characters.
The next cell computes the fingerprints of two texts that differ only in the case
of one letter, and shows that the two fingerprints have nothing in common.

Then it measures this for many changes: it takes one sentence of 115 characters,
changes each character in turn to the next character of the alphabet of the
computer (a to b, a blank to an exclamation mark, and so on), and counts how many
of the 64 characters of the fingerprint change. If the new fingerprint were a
random string, each of its 64 characters would equal the old one with
probability $1/16$, so on average $64 \cdot 15/16 = 60$ characters would change.
The cell draws the 115 counts as a histogram (a bar chart of how often each count
occurs).
```

In [12]:

```
def text_fingerprint(text):
    """The sha256 fingerprint of a text (encoded as UTF-8 bytes)."""
    return hashlib.sha256(text.encode("utf-8")).hexdigest()


for text in ("Universes in Pairs", "Universes in pairs"):
    digest = text_fingerprint(text)
    say(f"{text}:")
    say(f"    {digest}")

SENTENCE = ("The rule above every other rule of this book is honesty: it never "
            "writes proved for a statement that is not proved.")
original = text_fingerprint(SENTENCE)
changed_characters = []  # for each position: how many of the 64 characters change
for position in range(len(SENTENCE)):
    letter = SENTENCE[position]
    altered = SENTENCE[:position] + chr(ord(letter) + 1) + SENTENCE[position + 1:]
    new = text_fingerprint(altered)
    changed_characters.append(sum(1 for a, b in zip(original, new) if a != b))
counts = np.array(changed_characters)
report("characters of the sentence", len(SENTENCE))
report("changed characters of the fingerprint: smallest, mean, largest",
       f"{counts.min()}, {counts.mean():.2f}, {counts.max()}")

values, how_often = np.unique(counts, return_counts=True)  # each count, how often
fig, ax = plt.subplots()
ax.bar(values, how_often, width=0.8, color="#2a78d6", edgecolor="white",
       linewidth=1.5, label="one-character changes of the sentence")
ax.axvline(60, color="black", linewidth=1.2, linestyle=":",
           label="expected for a random fingerprint: 60")
ax.set_xlim(-1, 65)
# "\n" inside a string starts a new line of the text.
ax.text(3, 0.55 * how_often.max(), "a fingerprint that changed only a little\n"
        "would give a bar on this side: none did", color="#52514e", fontsize=9)
ax.set_xlabel("number of the 64 characters of the fingerprint that change")
ax.set_ylabel("number of changed sentences")
ax.set_title("Change one character, and the fingerprint changes completely")
ax.legend(loc="upper left")
save_figure(fig, "fingerprints",
            r"How much a sha256 fingerprint changes when one character of a text "
            r"changes: for each of the 115 characters of one sentence, the "
            r"character was replaced by the next one and the number of the 64 "
            r"hexadecimal characters of the fingerprint that changed was counted; "
            r"horizontal axis that number (0 to 64), vertical axis how many of the "
            r"115 altered sentences gave it. Every change altered between "
            f"{counts.min()} and {counts.max()} of the 64 characters, on average "
            f"{counts.mean():.2f}, as for a random string (dotted line at 60): none "
            f"of the {len(counts)} one-character changes left the fingerprint "
            r"nearly the same.")
check(counts.min() > 32 and abs(counts.mean() - 60.0) < 1.0,
      f"all {len(counts)} one-character changes alter more than half of the "
      "fingerprint")
```

Out [12]:

```
Universes in Pairs:
    3bf948a39ff61a48d1538293b76acbbbd85359fce5b1d824efb5329208185f38
Universes in pairs:
    27d89acd6238771ac751cd99a3578c7f3b65ffbba3cdc2ec0e116ec01ce95fe9
RESULT characters of the sentence = 115
RESULT changed characters of the fingerprint: smallest, mean, largest = 54, 60.03, 64
```

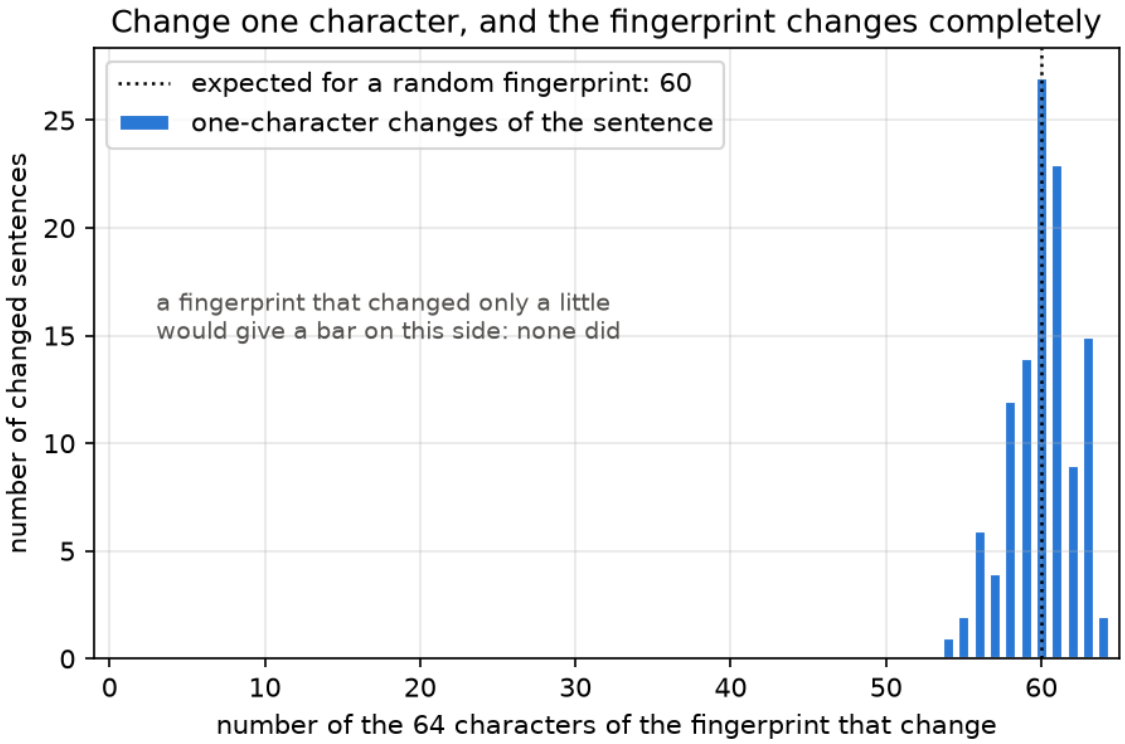

Figure 10: How much a sha256 fingerprint changes when one character of a text changes: for each of the 115 characters of one sentence, the character was replaced by the next one and the number of the 64 hexadecimal characters of the fingerprint that changed was counted; horizontal axis that number (0 to 64), vertical axis how many of the 115 altered sentences gave it. Every change altered between 54 and 64 of the 64 characters, on average 60.03, as for a random string (dotted line at 60): none of the 115 one-character changes left the fingerprint nearly the same. (Notebook 00c, figure 4.)

Out [12]:

```
Figure 00c.4 saved as Revision/textbook/figures/00c_4_fingerprints.png
PASS all 115 one-character changes alter more than half of the fingerprint
```

Text cell:

```
The reports of the Revision record write down the fingerprints of the files they
read. This notebook compares 24 of them. The report of the Wolfram check of GKD and
the Lovelock tensors records 15 (its keys ‘inputSha256‘ and ‘sourceSha256‘: the
result files, the Rust sources and its own Wolfram sources), the Python check of
```

the Lovelock tensors 2 (key ‘inputsSha256‘), and five Kohn-Sham reports write 7
more into the details of their checks, in the form ‘(sha256 95d8cbdd...)‘, either
in full (64 characters) or only the first 16 characters. Other reports record
further fingerprints (for example in a key ‘inputs‘, a list of files each with its
fingerprint); this notebook does not compare those. The next cell collects these
24, computes the fingerprints of today’s files, compares them (a shortened
fingerprint with the beginning of today’s), and prints a table. When all agree,
these reports read exactly today’s versions of these files.

In [13]:

```
def file_fingerprint(path):
    """The sha256 fingerprint of the bytes of the repository file path."""
    return hashlib.sha256(repository_file(path).read_bytes()).hexdigest()


RECORDED = []  # (report, file, recorded fingerprint)
GKD = "Revision/gkd_lovelock/results/wolfram-gkd-report.json"
gkd = read_report(GKD)
for path, value in {**gkd["inputSha256"], **gkd["sourceSha256"]}.items():
    RECORDED.append((GKD, path, value))
LOVELOCK = "Revision/gkd_lovelock/results/python-lovelock-report.json"
for name, value in read_report(LOVELOCK)["inputsSha256"].items():
    RECORDED.append((LOVELOCK, "Revision/gkd_lovelock/results/" + name, value))
IN_DETAILS = [  # (report, check, the file whose fingerprint the detail holds)
    ("ks-theory-wolfram.json", "fixture_input", "algebra/gammas.json"),
    ("ks-theory-python.json", "fixture_input", "algebra/reports/python-gammas.json"),
    ("ks-reference.json", "theory_input_coefficients", "kohn_sham/ks-theory.json"),
    ("ks-rust-solver.json", "theory_input_coefficients", "kohn_sham/ks-theory.json"),
    ("ks-rust-solver.json", "gamma_fixture_numeric", "algebra/gammas.json"),
    ("ks-rust-solver.json", "thermo_mu_vs_40digit_roots",
     "kohn_sham/solver/tools/mermin-roots-40digit.json"),
    ("ks-crosscheck.json", "problem_definition_identical",
     "kohn_sham/ks-theory.json"),
]
for name, check_name, path in IN_DETAILS:
    report_path = "Revision/kohn_sham/reports/" + name
    detail = next(entry["detail"] for entry in read_report(report_path)["checks"]
                  if entry["name"] == check_name)
    value = re.search(r"\(sha256 ([0-9a-f]{16,64})\)", detail).group(1)
    RECORDED.append((report_path, "Revision/" + path, value))

changed = []  # files whose fingerprint today differs from the recorded one
for report_path, path, value in RECORDED:
    same = file_fingerprint(path).startswith(value)  # a full or shortened value
    if not same:
        changed.append(path)
    status = "same" if same else "CHANGED"
    short_path = path.removeprefix("Revision/")  # the path without Revision/
    report_name = report_path.rsplit("/", 1)[1]  # the file name of the report
    say(f"{status:7} {short_path:52} {report_name}")
recorded_gammas = next(value for report_path, path, value in RECORDED
                       if report_path.endswith("ks-theory-wolfram.json"))
```

```
today_gammas = file_fingerprint("Revision/algebra/gammas.json")
say("Revision/algebra/gammas.json, recorded in ks-theory-wolfram.json and today:")
say(f"    {recorded_gammas}")
say(f"    {today_gammas}")
report("recorded fingerprints compared", len(RECORDED))
check_reproduces(len(RECORDED) == 24 and changed == [],
                 "the 24 recorded fingerprints equal the files of today",
                 f"{GKD}, keys inputSha256 and sourceSha256, and five further reports")
```

Out [13]:

```
same    gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md wolfram-gkd-report.json
same    gkd_lovelock/results/curvature.json                   wolfram-gkd-report.json
same    gkd_lovelock/results/lovelock-tensors.json            wolfram-gkd-report.json
same    gkd_lovelock/results/lovelock-report.json             wolfram-gkd-report.json
same    gkd_lovelock/code/Cargo.toml                          wolfram-gkd-report.json
same    gkd_lovelock/code/src/geometry.rs                     wolfram-gkd-report.json
same    gkd_lovelock/code/src/gkd.rs                          wolfram-gkd-report.json
same    gkd_lovelock/code/src/lib.rs                          wolfram-gkd-report.json
same    gkd_lovelock/code/src/lovelock.rs                     wolfram-gkd-report.json
same    gkd_lovelock/code/src/main.rs                         wolfram-gkd-report.json
same    gkd_lovelock/code/src/output.rs                       wolfram-gkd-report.json
same    gkd_lovelock/code/src/poly.rs                         wolfram-gkd-report.json
same    gkd_lovelock/code/src/rational.rs                     wolfram-gkd-report.json
same    gkd_lovelock/verification/LovelockGKDCheck.wl         wolfram-gkd-report.json
same    gkd_lovelock/verification/verify_lovelock_gkd.wls     wolfram-gkd-report.json
same    gkd_lovelock/results/curvature.json                   python-lovelock-report.json
same    gkd_lovelock/results/lovelock-tensors.json            python-lovelock-report.json
same    algebra/gammas.json                                   ks-theory-wolfram.json
same    algebra/reports/python-gammas.json                    ks-theory-python.json
same    kohn_sham/ks-theory.json                              ks-reference.json
same    kohn_sham/ks-theory.json                              ks-rust-solver.json
same    algebra/gammas.json                                   ks-rust-solver.json
same    kohn_sham/solver/tools/mermin-roots-40digit.json      ks-rust-solver.json
same    kohn_sham/ks-theory.json                              ks-crosscheck.json
Revision/algebra/gammas.json, recorded in ks-theory-wolfram.json and today:
    c99277bab4793c425bc26849421995fcb5aa8b2c865c0f505686a43654933c3c
    c99277bab4793c425bc26849421995fcb5aa8b2c865c0f505686a43654933c3c
RESULT recorded fingerprints compared = 24
PASS the 24 recorded fingerprints equal the files of today
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, keys inputSha256
    and sourceSha256, and five further reports
```

Text cell:

```
## 11. The last check

The last cell checks that the four figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [14]:

```
figure_names = ["00c_1_checks_by_report.png", "00c_2_two_verifiers.png",
                "00c_3_ledger.png", "00c_4_fingerprints.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "the four figure files of this notebook exist")
all_checks_passed()
```

Out [14]:

```
PASS the four figure files of this notebook exist
ALL 18 CHECKS PASSED (notebook 00c)
```

Text cell:

```
## 12. What this notebook showed

- A search of the whole folder Revision finds 51 verifier reports, the 51 of our
  list. No check has the verdict FAIL: every check is PASS except five
  comparisons with values that the author's own notebook does not store, whose
  verdict is NOT-AVAILABLE; section 7 prints how many checks there are, in all and
  per engine (Wolfram Language, Python, Rust and the lead's independent Python
  checks).
- Each report states the same totals that we counted, and the counts quoted in
  the file Revision/README.md, in the Kohn-Sham cross-check and in the
  dark-sector summary file are the same as well.
- Eleven subjects are checked by two independent verifiers, one in Wolfram Language
  and one in Python.
- Every report belongs to exactly one row of the honesty ledger. Fifteen rows have
  reports (eleven PROVED, three COMPUTED, one ASSUMED). The charge obeys a local
  conservation law (PROVED); that the total charge of one universe stays constant
  is OPEN, because it needs an assumed condition at the brane. Two rows are
  HYPOTHESIS (a time-varying dark sector, investigated but not established; a
  partner universe of opposite charge) and three are OPEN: the constancy of the
  total charge, whether the big bang creates universes in pairs (not proved), and
  what produces the excess of matter over antimatter (the theory as built does not
  produce it). The pairing record itself lists twelve things it does not
  establish, the first being any creation process.
- Each of the 115 one-character changes of a sentence changed more than half of
  the 64 characters of its sha256 fingerprint, 60.03 on average, as for a random
  string; and 24 fingerprints that the reports recorded for their input files,
  all that this notebook compares, equal those of today's files.
```

0.21 Line-by-line walk-through of Notebook 00c

The notebook has fourteen code cells, In [1] to In [14]. This section explains every line of each of them.

In [1], the set-up cell. It is the set-up cell of Notebook 00a, word for word, except that its comment lines hold the run instructions of Notebook 00c (Section 0.19) and the line `NOTEBOOK_ID = "00c"` names this notebook. Every line of its code is explained in Section 0.13. It prints Set-up of notebook 00c complete: repository folder found, helpers defined.

In [2], one report opened by hand.

```
import hashlib  # computes sha256 fingerprints
import re  # finds patterns in texts ("regular expressions")
import sys  # sys.stdout is the channel through which the notebook prints

import numpy as np  # arrays of numbers
```

Three modules of Python itself: `hashlib` computes fingerprints, `re` finds patterns in texts (In [6] explains them), `sys` gives the printing channel; and numpy.

```
def read_report(path):
    """The content of the JSON file path of the repository (dictionaries, lists)."""
    return json.loads(repository_file(path).read_text(encoding="utf-8"))


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), after sending the waiting output."""
    sys.stdout.flush()  # send every printed line that is still waiting
    check(condition, name, record=record)
```

`read_report` reads a JSON file of the repository into dictionaries and lists, as explained for In [2] of Notebook 00b (Section 0.17). `check_reproduces` is the same helper as there: it sends the waiting printed text first, so that a PASS line and its line reproduces ... arrive together.

```
CC_REPORT = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
cc = read_report(CC_REPORT)  # a dictionary with the keys producer, summary, checks
producer = cc["producer"]
say(f"producer: {producer}")
```

The report of the lead's independent check of the charge-conjugation matrices and of the local conservation law of the charge (row 14 of the ledger) is read. Its key `producer` names the program that wrote it; the first two lines of Out [2] print it: `Revision/lead_checks/charge_conjugation_and_u1.py`, a short independent Python program that imports no other Revision code.

```
for entry in cc["checks"]:  # each entry is a dictionary: name, verdict, detail
    verdict, name = entry["verdict"], entry["name"]
    say(f"  {verdict}  {name}")
```

The loop prints the verdict and the name of each of its checks, the next twelve lines of Out [2]: from `representation_real` to `u1_noether_matrix_identity`, all PASS. (The line `verdict, name = a, b` gives two names at once.)

```
stated = cc["summary"]  # what the report says about itself: passed and total
stated_passed, stated_total = stated["passed"], stated["total"]
say(f"the report states: {stated_passed} passed of {stated_total}")
passed = sum(1 for entry in cc["checks"] if entry["verdict"] == "PASS")
number_of_checks = len(cc["checks"])
report("checks with the verdict PASS, counted", f"{passed} of {number_of_checks}")
check_reproduces(passed == stated["passed"] == stated["total"] == 12,
                 "12 of 12 checks of the charge-conjugation report are PASS",
                 f"{CC_REPORT}, its summary")
```

A report also states its own totals, under the key `summary`; they are printed. Then the notebook counts for itself: `sum(1 for entry in ... if ...)` adds 1 for every check whose verdict is PASS, and `len` is the number of checks. Python allows a chain of comparisons, `a == b == c == 12`, which is true only when every neighbouring pair is equal. The check requires that our count, the stated number of passed checks and the stated total are all 12. This is level 2 of Section 0.4, done by the computer.

In [3], five details of the same report.

```
details = {entry["name"]: entry["detail"] for entry in cc["checks"]}  # name -> detail
```

A **dictionary comprehension**: for every check of the report it stores the detail text under the name of the check, so that, for example, `details["representation_real"]` is the detail of the check `representation_real`.

```
for wanted in ("representation_real", "charge_conjugation_matrix_plus",
               "charge_conjugation_matrix_minus", "real_fields_charge_conjugation",
               "u1_noether_matrix_identity"):
    detail = details[wanted]
    if wanted == "u1_noether_matrix_identity":
        # This detail is long; its last two parts, after the second-to-last "; ",
        # state the result and its limit.
        detail = "; ".join(detail.split("; ")[-2:])
    say(f"{wanted}:")
    say("    " + detail)  # say breaks it into lines of at most 89 characters
```

For five check names the loop looks up the detail and prints the name and, below it and indented by four blanks, the detail; `say` (Section 0.13) breaks a long detail into lines of at most 89 characters and indents every further line by four blanks, so no line of Out [3] is wider than a page of the book. The last detail is long; `detail.split("; ")` cuts it into a list of pieces at every semicolon followed by a blank, `[-2:]` keeps the last two pieces, and `"; ".join(...)` glues them together again with the same separator: the result and its limit. Out [3] prints, in the report's own words (written with plain letters: `calC_+` is $\mathcal{C}_+$, `Psi*` is Ψ^*, `^T` marks a transposed matrix, Chapter 1, and `d_mu` is the derivative ∂_μ): that the gammas, C and S^{ab} are real, so that plain complex conjugation is the identity on a real field; that $\mathcal{C}_+ = C$ satisfies $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$ for every direction a, and that its conjugate field is $\Psi^c = \Psi^*$; that $\mathcal{C}_- = \Gamma C$ satisfies $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = +(\gamma^a)^T$, and that its conjugate field $\Gamma\Psi^*$ solves the field equation in which the term V that holds the mass has the opposite sign (the report writes $V \to -V$); that for a real commuting field the charge current J^a is zero, $\mathcal{C}_+$ acts as the identity, and the real matrix Γ together with the reversal of the mass maps solutions to solutions (this is T1). The last detail says two things. First: "on shell the LOCAL law d_mu(cos z J^mu) = 0 holds", that is, whenever the field equations hold ("on shell"), $\partial_\mu(\cos z\, J^\mu) = 0$: charge can only flow from one place to another; no charge is made or destroyed at any point. Second, the limit: the total charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, d^7x$ of a slice of space (the integral over the seven directions other than the time x_4) is constant only if no charge flows through the boundary of the slice; at the brane $z = \pi/2$, the edge of the hidden direction, the flow is not zero in general, and the report ends: "constancy of Q is NOT established by this check". The book therefore labels the local law PROVED (row 14 of the ledger) and the constancy of the total charge OPEN (row 18): it would need a no-flux condition at the brane, which is ASSUMED, not derived, and Chapter 18 shows exact solutions on which it fails.

```
R5_WORDS = [  # (check, words that its detail must contain)
    ("representation_real", "plain complex conjugation is the identity"),
    ("charge_conjugation_matrix_plus", "calC_+ = C (= sigma16)"),
    ("charge_conjugation_matrix_plus", "Psi^c = calC_+ Psibar^T = Psi*"),
    ("charge_conjugation_matrix_minus", "calC_- = Gamma C"),
    ("charge_conjugation_matrix_minus", "Psi^c = calC_- Psibar^T = Gamma Psi*"),
    ("real_fields_charge_conjugation", "J^a = -i Psi^T C gamma^a Psi = 0"),
    ("real_fields_charge_conjugation", "calC_+ acts as the identity"),
    ("real_fields_charge_conjugation", "the matrix Gamma with the mass reversed"),
]
```

The list `R5_WORDS` holds eight pairs, each a check name and a piece of text that the detail of that check must contain word for word. Together the eight pieces are the statements of Section 0.2 about charge conjugation: plain complex conjugation does nothing to a real field; the two charge-conjugation matrices are $\mathcal{C}_+ = C$ (`sigma16` is the author's name for C) and $\mathcal{C}_- = \Gamma C$, each with its conjugate field; and for a real commuting field the charge current is zero, $\mathcal{C}_+$ acts as the identity, and the map between matter and antimatter is the matrix Γ with the mass reversed.

```
check_reproduces(all(words in details[name] for name, words in R5_WORDS),
                 "the report states calC_+ = C, calC_- = Gamma C and J = 0 for a "
                 "real field",
                 f"{CC_REPORT}, checks charge_conjugation_matrix_plus, "
                 "charge_conjugation_matrix_minus and real_fields_charge_conjugation")
```

`words in text` is true when the string `words` occurs in the string `text`, and `all(...)` is true when this holds for all eight pairs. So the check confirms that the record states exactly the charge-conjugation statements of Section 0.2, and its second line names the three checks of the report in which it found them. Chapters 5 and 21 derive the statements from zero.

```
U1_WORDS = ["on shell the LOCAL law d_mu(cos z J^mu) = 0 holds",
            "is constant only if no charge flows through its boundary",
            "constancy of Q is NOT established by this check"]
check_reproduces(all(words in details["u1_noether_matrix_identity"]
                     for words in U1_WORDS),
                 "the report proves the local law and does not establish a "
                 "constant total charge",
                 f"{CC_REPORT}, check u1_noether_matrix_identity")
```

The list `U1_WORDS` holds three pieces of text that the detail of the check `u1_noether_matrix_identity` must contain word for word: the local law, the condition under which the total charge would be constant, and the report's own statement that this constancy is not established. The check confirms both halves of rows 14 and 18 of the ledger in the record's own words: what is proved, and where the proof stops. Out [3] ends with its PASS line and the line that names the check.

In [4], one counting function for three layouts.

```
def count_checks(data):
    """(number of passed checks, number of checks), counted from the checks."""
    if isinstance(data.get("checks"), list):  # layout A
```

```
        verdicts = [entry["verdict"].upper() == "PASS" for entry in data["checks"]]
    elif isinstance(data.get("checks"), dict):  # layout B
        verdicts = [entry["passed"] is True for entry in data["checks"].values()]
    else:  # layout C: a self-test passes when it found no mismatch
        verdicts = [entry["mismatches"] == 0 for entry in data["results"]]
    # True counts as 1 and False as 0 when added.
    return sum(verdicts), len(verdicts)
```

The reports were written by different programs and store their checks in three layouts. `data.get ("checks")` returns the value under the key `checks`, or `None` if there is none (where `data["ch ecks"]` would stop with an error). `isinstance(value, list)` asks whether the value is a list. In layout A the checks are a list of dictionaries with a verdict; in layout B a dictionary from each name to a dictionary with the entry `passed` (`.values()` gives the inner dictionaries, and `is True` asks for the value `True` itself); in layout C (the self-test of the Rust GKD program) a list `results` of tests, each passing when its count of `mismatches` is 0. In each case `verdicts` is a list of `True` and `False`, and the function returns the number of `True` entries and the length of the list.

```
def stated_summary(data):
    """(passed, total) as the report states them itself; None if it states none."""
    summary = data.get("summary")
    if isinstance(summary, dict):  # {"passed": n, "total": n} and similar names
        passed = summary.get("passed", summary.get("pass", summary.get("PASS")))
        return passed, summary.get("total", summary.get("checks"))
    if isinstance(summary, str):  # a text such as "49/49 checks pass"
        passed, total = re.fullmatch(r"(\d+)/(\d+) checks pass", summary).groups()
        return int(passed), int(total)
    counts = data.get("counts")
    if isinstance(counts, dict) and "pass" in counts:  # {"pass", "fail", "pending"}
        return counts["pass"], counts["pass"] + counts["fail"] + counts["pending"]
    if "checkCount" in data:  # a total and a number of failed checks
        failed = data.get("failCount", data.get("failedCount",
                                                data.get("failedCheckCount")))
        return data["checkCount"] - failed, data["checkCount"]
    return None
```

The totals that a report states about itself are written in different words: a dictionary `summary` with `passed` and `total` (or `pass` or `PASS` for the passed checks, and `checks` for the total); a text `summary` such as "49/49 checks pass" (the two reports of the field dirac16complex00 in the dark sector write it so); a dictionary `counts` with the numbers of passed, failed and pending checks; or a total `checkCount` with a number of failed checks under one of three names. `.get(key, default)` returns the default when the key is missing, so the nested `.get` calls try the names one after the other. `isinstance(summary, str)` asks whether the summary is a text (a **string**). `re.fullmatch(pat tern, text)` succeeds only when the whole text matches the pattern; in the pattern `(\d+)/(\d+) checks pass`, `\d+` stands for one or more digits and the brackets mark the two numbers (In [6] explains patterns once more); `.groups()` returns the two marked pieces as texts, and `int` turns each into a whole number. The function returns the pair (passed, total), or `None` when the report states no totals.

```
EXAMPLES = [("A", "Revision/pairing/reports/wolfram-pairing.json"),
            ("B", "Revision/gkd_lovelock/results/lovelock-report.json"),
```

```
            ("C", "Revision/gkd_lovelock/results/gkd-selftest.json")]
for layout, path in EXAMPLES:
    data = read_report(path)
    passed, total = count_checks(data)
    say(f"layout {layout}: {path}")
    say(f"    counted {passed} of {total}; stated {stated_summary(data)}")
```

One report of each layout, counted and compared with what it states: Out [4] shows the counts of the Wolfram pairing report, of the Rust Lovelock report and of the GKD self-test, which states no totals (`None`) but only its verdict.

In [5], every report of the Revision record.

```
def is_report(path):
    """True if the JSON file path (a Path) is a verifier report."""
    data = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(data, dict):
        return False
    checks = data.get("checks")
    listed = isinstance(checks, list) or (  # a list of checks, or a dictionary
        isinstance(checks, dict)  # whose every entry is a check (a dictionary)
        and all(isinstance(entry, dict) for entry in checks.values()))
    results = data.get("results")
    self_test = (isinstance(results, list) and len(results) > 0
                 and isinstance(results[0], dict) and "mismatches" in results[0])
    return listed or self_test
```

A JSON file is a verifier report when it is a dictionary whose key `checks` holds the checks themselves, or, like the GKD self-test, a non-empty list `results` whose first entry is a dictionary with the key `mismatches`. "Holds the checks themselves" means: `checks` is a list (layout A), or a dictionary each of whose entries is again a dictionary, one per check (layout B). `all(...)` is true when every entry passes the test `isinstance(entry, dict)`. The rule matters for one file of the record: the summary file of the dark sector has a key `checks` whose value is only the two numbers `{"total": 13, "pass": 13}`, the totals of another report; it is not a report, and the cell compares its two numbers in In [6] instead of counting them twice. The conditions joined by `and` are tested from the left and the testing stops at the first false one, so `results[0]` is never read from an empty or missing list; the brackets after `or (` let the condition continue on the next lines.

```
SKIPPED = ("Revision/textbook/", "Revision/workflows/")  # folders that hold no report
found_reports = []  # every report found in the folder Revision, as a relative path
not_reports = []  # JSON files with a key "checks" that holds no checks
for path in sorted(repository_file("Revision").rglob("*.json")):
    relative = path.relative_to(REPO).as_posix()  # e.g. "Revision/algebra/..."
    if relative.startswith(SKIPPED) or "/target/" in relative:
        continue  # the book, the workflow records and the Rust build folders
    if is_report(path):
        found_reports.append(relative)
    elif "checks" in read_report(relative):
        not_reports.append(relative)
report("verifier reports found in the folder Revision", len(found_reports))
say("JSON files with a key checks that holds no checks:")
```

```
for relative in not_reports:
    say("    " + relative)
```

SKIPPED is a **tuple** (a list in round brackets that cannot be changed) of two folder names. rglob("*.json") lists every file ending in .json in the folder Revision and in all its sub-folders, and sorted puts the list in a fixed order. path.relative_to(REPO) is the path from the repository folder on, and .as_posix() writes it with / on every operating system. relative.startswith(SKIPPED) is true when the path starts with either of the two names (startswith accepts a tuple and tries each of its entries). continue skips the rest of the loop for this file. Three kinds of folders are skipped, because they are not part of the record: the book's own folder Revision/textbook; the folder Revision/workflows, which holds the records of the programs that organised the work (they are rewritten while those programs run, so a half-written file could stop the notebook, and the Revision record says itself that no result depends on them); and the build folders target of the Rust programs. Every other report is collected. elif ("else if") collects, in the second list, every file that is not a report but still has a key checks, so that no such file is dropped silently. Out [5] begins with the number found, 51, and the one file of the second list, Revision/dark_sector/dirac16complex/outputs/eos-summary.json.

```
REPORTS = [  # (report, engine)
    # the gammas, C, Gamma, B, Pin(4,4) and Spin(4,4)
    ("Revision/algebra/reports/wolfram-algebra.json", "Wolfram"),
    ("Revision/algebra/reports/python-algebra.json", "Python"),
```

The list REPORTS holds the 51 reports of the record, each as a pair: the path of the report and the **engine** that did its computation, that is, the language of the program that wrote the report: "Wolfram" (the Wolfram Language), "Python", "Rust", or "lead" for the lead's short independent Python checks. A comment line before each group names its subject. The first group is the algebra of Chapters 4 and 5 (the gamma matrices, C, Γ, B and the groups), checked by a Wolfram verifier and by an independent Python verifier.

```
    # the metric with b4 (2026-10-14): curvature, the author's outputs, vacuum
    ("Revision/metric_b4/results/wolfram-metric-b4-report.json", "Wolfram"),
    ("Revision/metric_b4/results/python-metric-b4-report.json", "Python"),
    ("Revision/metric_b4/results/vacuum-b4.json", "Wolfram"),
    ("Revision/metric_b4/results/consequences-b4.json", "Wolfram"),
```

The folder Revision/metric_b4 holds the record of the author's change of 2026-10-14, the metric function b_4 (Section 0.14): a Wolfram and an independent Python verifier compute the curvature and the Lovelock tensors of the metric with b_4 and compare them with the outputs stored in the author's notebook; a Wolfram program solves the vacuum equations (every vacuum history has b_4 constant); and a Wolfram program checks the consequences of b_4 for the pairing theorems, the Kohn-Sham level and the dark sector.

```
    # the Lagrangians, field equations, EMT, quantisation; the scope; a Fock model
    ("Revision/theory/reports/wolfram-field-theory.json", "Wolfram"),
    ("Revision/theory/reports/python-field-theory.json", "Python"),
    ("Revision/theory/reports/wolfram-scope.json", "Wolfram"),
```

```
("Revision/theory/reports/python-scope.json", "Python"),
("Revision/theory/fock_quartic/reports/fock-quartic.json", "Python"),
("Revision/theory/reports/wolfram-field-theory-b4.json", "Wolfram"),
("Revision/theory/reports/python-field-theory-b4.json", "Python"),
("Revision/theory/reports/wolfram-field-theory-b4-emt-C.json", "Wolfram"),
("Revision/theory/reports/wolfram-field-theory-b4-emt-G.json", "Wolfram"),
```

The folder `Revision/theory` holds the field theory of Chapters 7, 9 and 10 (the Lagrangians, the field equations, the energy-momentum tensor, abbreviated EMT, and the quantisation of dirac16complex) and the exact scope of the non-triviality (Chapter 8), each in a Wolfram and in a Python version: four reports. A fifth report, of its sub-folder `fock_quartic`, comes from one Python program only: the energy-momentum tensor operator of dirac16complex with a self-coupling $\lambda \neq 0$ in a finite model of the quantum theory (Section 0.18, the remark on row 2). The last four reports redo the geometry, the spin connection, the Dirac operator and the energy-momentum tensor for a changing b_4: a Wolfram and a Python verifier, and two Wolfram reports on the energy-momentum tensor obtained by varying the vielbein, one for each kind of field (C for dirac16complex, G for dirac16complex00).

```
# the field equations for a4 and b4; the Kohn-Sham states as a source
("Revision/field_equations_a4/reports/wolfram-a4-report.json", "Wolfram"),
("Revision/field_equations_a4/reports/python-a4-report.json", "Python"),
("Revision/field_equations_a4/reports/python-b4-block.json", "Python"),
("Revision/field_equations_a4/reports/ks-source-conditions.json", "Python"),
("Revision/field_equations_a4/ks_source/reports/ks-source-a4.json", "Python"),
```

The field equations for a_4 and b_4 (Chapter 12), in Wolfram and in Python, and a second Python report that checks the equations of b_4 with the author's gamma matrices; the report of a Python program that tests whether the computed Kohn-Sham states can be the source of the history $a_4 = AHx_4$ (Chapter 17; its six checks show that they cannot, for every b_4); and the report of the folder `ks_source`, which writes the equations for a_4 with the Kohn-Sham states as their source (exact identities, the averages over the hidden direction, and the integration of a stated simplified system) and finds that no recorded Kohn-Sham state is an admissible source.

```
# GKD and the Lovelock tensors; the comparison with the author's outputs
("Revision/gkd_lovelock/results/lovelock-report.json", "Rust"),
("Revision/gkd_lovelock/results/gkd-selftest.json", "Rust"),
("Revision/gkd_lovelock/results/wolfram-gkd-report.json", "Wolfram"),
("Revision/gkd_lovelock/results/python-lovelock-report.json", "Python"),
("Revision/gkd_lovelock/comparison/author-comparison-report.json", "Python"),
```

The generalized Kronecker delta and the Lovelock tensors (Chapter 11): the Rust program that computes them writes a report on the tensors and a self-test of its GKD function, and a Wolfram and a Python verifier check its results independently. The fifth report compares the curvature of the Revision record with the values stored in the author's own Mathematica notebook (a Python program with sympy; row 7 of the ledger).

```
# the Kohn-Sham theory, solvers and comparisons; the tip-cutoff study
("Revision/kohn_sham/reports/ks-theory-wolfram.json", "Wolfram"),
("Revision/kohn_sham/reports/ks-theory-python.json", "Python"),
```

```
("Revision/kohn_sham/reports/ks-b4-wolfram.json", "Wolfram"),
("Revision/kohn_sham/reports/ks-rust-solver.json", "Rust"),
("Revision/kohn_sham/reports/ks-rust-determinism.json", "Python"),
("Revision/kohn_sham/reports/ks-rust-mermin-roots.json", "Python"),
("Revision/kohn_sham/reports/ks-reference.json", "Python"),
("Revision/kohn_sham/reports/ks-crosscheck.json", "Python"),
("Revision/kohn_sham/tip_convergence/tip-convergence.json", "Python"),
```

The Kohn-Sham model (Chapters 14 to 16): its theory in Wolfram and in Python; a Wolfram report showing that the Kohn-Sham problem of the good sector does not contain b_4; the Rust solver; two Python programs that examine the results of the Rust solver (the repeated and the refined run of Section 0.22, and the roots of the Mermin equation for the chemical potential, computed with 40 digits; Chapter 15); the independent Python reference solver; the cross-check that compares the two solvers (Chapter 16); and the study of the tip cutoff L of the hidden direction, a Python program that runs the Rust solver (and, for two of its states at $L = 3$ and 4, the reference solver) for L from 3 to 6 (Chapter 14; Section 0.18, the remark on row 9). The determinism report, the report on the roots and the tip-cutoff study concern the Rust solver, but their own checks are computed in Python, so their engine is Python.

```
# the dark-sector hypotheses of the two fields, investigated
("Revision/dark_sector/dirac16complex/reports/derivation-checks.json", "Python"),
("Revision/dark_sector/dirac16complex/reports/ks-history-run.json", "Python"),
("Revision/dark_sector/dirac16complex/reports/eos-checks.json", "Python"),
("Revision/dark_sector/dirac16complex/reports/independent-checks.json", "Python"),
("Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json", "Python"),
("Revision/dark_sector/dirac16complex00/reports/python-independent-numerics.json",
 "Python"),
```

The investigation of the author's two dark-sector hypotheses (Chapter 22), all six reports written by Python programs. For the field dirac16complex: the exact derivation of the identities with sympy; the run of the Rust Kohn-Sham solver along the prescribed history (its checks are computed by the Python program that runs the solver, so its engine is Python); the equations of state computed from that run; and an independent implementation. For the field dirac16complex00: the derivation of its equations of state and of its models, and independent numerics. The last pair is written on two lines, the path on the first and the engine on the second, because one line would be wider than 89 characters.

```
# the pairing theorems T1, T2, Q and T3; T3 completed and demonstrated
("Revision/pairing/reports/wolfram-pairing.json", "Wolfram"),
("Revision/pairing/reports/python-pairing.json", "Python"),
("Revision/pairing/kohn_sham/reports/wolfram-t3.json", "Wolfram"),
("Revision/pairing/kohn_sham/reports/python-t3.json", "Python"),
("Revision/pairing/kohn_sham/reports/wolfram-t3-completion.json", "Wolfram"),
("Revision/pairing/kohn_sham/reports/python-t3-completion.json", "Python"),
("Revision/pairing/kohn_sham/reports/t3-rust-demo.json", "Python"),
("Revision/pairing/kohn_sham/reports/t3-reference-demo.json", "Python"),
```

The pairing theorems (Chapters 18 and 19): T1, T2 and Q in one Wolfram and one Python report; T3 in a second pair of reports; its completion (the Kohn-Sham potentials read from the Kohn-Sham

record, the filling convention carried onto the member of mass $-M$, and the statement for all 16 components) in a third pair; and two numerical demonstrations of T3, one with the Rust solver and one with the Python reference solver, both run by Python programs.

```
    # the lead's independent checks
    ("Revision/lead_checks/reports/charge-conjugation-and-u1.json", "lead"),
    ("Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json", "lead"),
    ("Revision/lead_checks/reports/emt-divergence-and-spin-connection.json", "lead"),
]
```

The three short Python programs of the lead, which import no other Revision code: the charge-conjugation matrices and the local conservation law of the charge (read in In [2] and In [3]); the field equations for a_4 in Einstein's theory and with the Gauss-Bonnet term (the second Lovelock term of Chapter 11), compared with the record of Chapter 12; and the conservation identities of the energy-momentum tensor together with the spin connection (Chapters 6 and 9). The closing bracket ends the list. The table of Out [5] repeats the list with the counts.

```
check(found_reports == sorted(path for path, _ in REPORTS),
      f"the search finds exactly the {len(REPORTS)} reports of the list")
```

The search must find exactly the reports of the list: no report is missing from the list, and so from the ledger. (In `for path, _ in REPORTS` the name `_` takes the engine, which is not needed here.) When a new report is added to the record, this check fails until the new report gets its place in the list and in the ledger.

```
counted = {}  # report -> (passed, total)
header = "report (in the folder Revision)"
say(f"{header:69} engine  pass of all")
for path, engine in REPORTS:
    data = read_report(path)
    counted[path] = count_checks(data)
    passed, total = counted[path]
    short = path.removeprefix("Revision/")
    say(f"{short:69} {engine:7} {passed:4d} of {total:3d}")
```

Every report is read and counted; the pair (passed, total) is stored in the dictionary `counted` under its path. `removeprefix` removes `Revision/` from the front of the path to keep the table narrow; `{short:69}` writes the path filled with blanks to 69 characters (the longest path has 69), and `{passed:4d}` writes a whole number right-aligned in 4 characters, so that a line of the table has at most 89 characters. The 51 lines of the table in Out [5] are printed here. One line shows fewer passed checks than checks: the comparison with the author's notebook, 73 of 78.

```
all_checks = sum(total for _, total in counted.values())
all_passed = sum(passed for passed, _ in counted.values())
report("reports", len(REPORTS))
report("checks in all reports", all_checks)
report("checks with the verdict PASS", all_passed)
engine_totals = {}  # engine -> the number of its checks
```

```
for engine in ("Wolfram", "Python", "Rust", "lead"):
    engine_totals[engine] = sum(counted[path][1] for path, e in REPORTS
                                if e == engine)
    report(f"checks done with the engine {engine}", engine_totals[engine])
```

The totals over all reports, the number of passed checks, and the total per engine: for each engine the numbers of checks of its reports are added (`counted[path][1]` is the total of a report). These are the RESULT lines of Out [5]. The numbers are not written into the notebook: they are counted from the record each time, so they follow the record. In the run printed in Section 0.20 (2026-10-14, the date of the notebook's last verified run, which its provenance file records) they are 1357 checks in all, of which 1352 have the verdict PASS: 465 done with the Wolfram Language, 785 with Python, 70 with Rust and 37 by the lead's checks.

```
COMPARISON = "Revision/gkd_lovelock/comparison/author-comparison-report.json"
not_pass = [path for path, _ in REPORTS if counted[path][0] != counted[path][1]]
other_verdicts = [entry["verdict"] for entry in read_report(COMPARISON)["checks"]
                  if entry["verdict"] != "PASS"]  # the verdicts that are not PASS
not_available = other_verdicts.count("NOT-AVAILABLE")
report("checks with the verdict NOT-AVAILABLE", not_available)
```

`not_pass` lists the reports in which some check is not PASS. `other_verdicts` collects, from the comparison with the author's notebook, every verdict that is not PASS, and `.count("NOT-AVAILABLE")` counts how many of them are the word NOT-AVAILABLE: the comparisons that could not be made, because the author's notebook stores no value of the quantity (Section 0.18, row 7). Out [5] prints the number, 5.

```
check(not_pass == [COMPARISON] and len(other_verdicts) == not_available == 5
      and all_passed + not_available == all_checks
      and sum(engine_totals.values()) == all_checks,
      f"no check FAILS: {all_passed} PASS, {not_available} NOT-AVAILABLE, "
      f"{all_checks} in all")
```

The check requires four things: the only report with a check that is not PASS is the comparison; its checks that are not PASS are exactly five, and all five are NOT-AVAILABLE; the passed and the not-available checks together are all the checks, so not a single check of the 51 reports has the verdict FAIL; and the four engine totals add up to the total (each report has exactly one engine). Its PASS line prints the three numbers: 1352 PASS, 5 NOT-AVAILABLE, 1357 in all.

```
disagree = []  # reports whose own summary differs from our count
for path, _ in REPORTS:
    data = read_report(path)
    stated = stated_summary(data)
    if stated is None:  # layout C: the self-test states only its verdict
        stated = counted[path] if data["verdict"] == "SUCCESS" else None
    if stated != counted[path]:
        disagree.append(path)
check(disagree == [], "each report states the same totals that we counted")
```

For every report our count must equal what the report states about itself (for the comparison with the author's notebook: 73 PASS of 78, as its own summary says). The GKD self-test states no totals; for it the stated verdict SUCCESS is required instead. Out [5] ends with this PASS line.

In [6], three other places that quote the counts.

```
R = "Revision/"
README_ORDER = [  # the reports whose counts the README table quotes, in its order
    R + "gkd_lovelock/results/lovelock-report.json",  # row gkd_lovelock/
    R + "gkd_lovelock/results/python-lovelock-report.json",
    R + "gkd_lovelock/results/wolfram-gkd-report.json",
    R + "metric_b4/results/wolfram-metric-b4-report.json",  # row metric_b4/
    R + "metric_b4/results/python-metric-b4-report.json",
    R + "metric_b4/results/vacuum-b4.json",
    R + "metric_b4/results/consequences-b4.json",
```

`R + "..."` joins two strings. The list `README_ORDER` names, in the order in which they appear in the table of the file `Revision/README.md`, the reports of the 49 counts that table quotes: 46 different reports, three of which stand there twice, as the last lines of the list show. The first row of the table, `gkd_lovelock/`, quotes three counts: that of the Rust report on the Lovelock tensors, then those of the Python and of the Wolfram verifier. The second row, `metric_b4/`, quotes four: the Wolfram and the Python verifier of the metric with b_4, the vacuum and the consequences.

```
    R + "algebra/reports/wolfram-algebra.json",  # row algebra/
    R + "algebra/reports/python-algebra.json",
    R + "theory/reports/wolfram-field-theory.json",  # row theory/
    R + "theory/reports/python-field-theory.json",
    R + "theory/reports/wolfram-scope.json",
    R + "theory/reports/python-scope.json",
    R + "theory/fock_quartic/reports/fock-quartic.json",
    R + "theory/reports/wolfram-field-theory-b4.json",
    R + "theory/reports/python-field-theory-b4.json",
    R + "theory/reports/wolfram-field-theory-b4-emt-C.json",
    R + "theory/reports/wolfram-field-theory-b4-emt-G.json",
```

The rows `algebra/` (two counts, Wolfram then Python) and `theory/` (nine counts: the field theory, then the scope, each Wolfram then Python, the finite Fock model of the folder `theory/fock_quartic`, and the four reports with b_4).

```
    R + "field_equations_a4/reports/wolfram-a4-report.json",  # row field_equations_a4/
    R + "field_equations_a4/reports/python-a4-report.json",
    R + "field_equations_a4/reports/python-b4-block.json",
    R + "field_equations_a4/reports/ks-source-conditions.json",
    R + "field_equations_a4/ks_source/reports/ks-source-a4.json",
    R + "kohn_sham/reports/ks-theory-wolfram.json",  # row kohn_sham/
    R + "kohn_sham/reports/ks-theory-python.json",
    R + "kohn_sham/reports/ks-b4-wolfram.json",
    R + "kohn_sham/reports/ks-rust-solver.json",
    R + "kohn_sham/reports/ks-rust-determinism.json",
    R + "kohn_sham/reports/ks-rust-mermin-roots.json",
```

```
    R + "kohn_sham/reports/ks-reference.json",
    R + "kohn_sham/reports/ks-crosscheck.json",
    R + "kohn_sham/tip_convergence/tip-convergence.json",
```

The rows `field_equations_a4/` (five counts: Wolfram, Python, the block of b_4, the Kohn-Sham source conditions, and the folder `ks_source`) and `kohn_sham/` (nine counts: the theory in Wolfram and in Python, the report on b_4, the Rust solver, its determinism, the Mermin roots, the reference solver, the cross-check and the tip-cutoff study).

```
    R + "dark_sector/dirac16complex/reports/derivation-checks.json",  # dark_sector/
    R + "dark_sector/dirac16complex/reports/ks-history-run.json",
    R + "dark_sector/dirac16complex/reports/eos-checks.json",
    R + "dark_sector/dirac16complex/reports/independent-checks.json",
    R + "dark_sector/dirac16complex00/reports/python-derive-eos.json",
    R + "dark_sector/dirac16complex00/reports/python-independent-numerics.json",
```

The row `dark_sector/` (six counts: four for the field dirac16complex, two for dirac16complex00).

```
    R + "pairing/reports/wolfram-pairing.json",  # row pairing/
    R + "pairing/reports/python-pairing.json",
    R + "pairing/kohn_sham/reports/wolfram-t3.json",
    R + "pairing/kohn_sham/reports/python-t3.json",
    R + "pairing/kohn_sham/reports/wolfram-t3-completion.json",
    R + "pairing/kohn_sham/reports/python-t3-completion.json",
    R + "pairing/kohn_sham/reports/t3-rust-demo.json",
    R + "pairing/kohn_sham/reports/t3-reference-demo.json",
```

The row `pairing/` (eight counts: T1, T2 and Q in Wolfram and in Python; T3 in Wolfram and in Python; its completion in Wolfram and in Python; the two demonstrations, with the Rust solver and with the reference solver).

```
    R + "gkd_lovelock/results/lovelock-report.json",  # row docs/: LOVELOCK_GKD
    R + "gkd_lovelock/results/wolfram-gkd-report.json",
    R + "gkd_lovelock/results/python-lovelock-report.json",
]
```

The row `docs/` names the document on the Lovelock tensors together with the counts of its three reports, in the order Rust, Wolfram, Python; so these three reports stand twice in the list. The reports that the table does not quote (the GKD self-test, the comparison with the author's notebook and the lead's three checks) are not in this list.

```
readme_lines = repository_file("Revision/README.md").read_text(
    encoding="utf-8").split("\n")
table = "\n".join(line for line in readme_lines if line.startswith("| ‘"))
quoted_readme = [(int(p), int(t)) for p, t in re.findall(r"(\d+)/(\d+)", table)]
say("the README quotes: " + ", ".join(f"{p}/{t}" for p, t in quoted_readme))
report("counts quoted in the README table", len(quoted_readme))
```

The file is read and cut into lines; the lines of the table of the folders start with a vertical bar, a blank and a backtick, and they are joined again. A **regular expression** is a pattern that describes a family of texts: in `(\d+)/(\d+)`, `\d` means one digit, `+` means one or more of what stands before it, and the round brackets mark the parts to return. `re.findall` returns every match as a pair of the two marked parts, such as `("19", "19")`; `int` turns them into numbers. Out [6] prints the 49 counts found.

```
check_reproduces(quoted_readme == [counted[path] for path in README_ORDER],
                 f"the {len(README_ORDER)} counts quoted in the README equal ours",
                 "Revision/README.md, the table of the folders")
```

The 49 quoted counts must equal our counts of the 49 entries of the list, in this order.

```
CROSS = "Revision/kohn_sham/reports/ks-crosscheck.json"
detail = next(entry["detail"] for entry in read_report(CROSS)["checks"]
              if entry["name"] == "inputs_all_pass")
quoted = re.findall(r"(\d+)/(\d+) PASS", detail)  # [("37", "37"), ("42", "42"), ...]
say("the cross-check quotes: " + ", ".join(f"{p}/{t}" for p, t in quoted))
ours = [counted["Revision/kohn_sham/reports/ks-reference.json"],
        counted["Revision/kohn_sham/reports/ks-rust-solver.json"],
        counted["Revision/kohn_sham/reports/ks-rust-determinism.json"]]
numbers = ", ".join(str(total) for _, total in ours)  # e.g. "37, 42, 14"
check_reproduces([(int(p), int(t)) for p, t in quoted] == ours,
                 f"the cross-check quotes the counts {numbers} that we counted",
                 f"{CROSS}, check inputs_all_pass")
```

The Kohn-Sham cross-check read three other reports before it compared the two solvers, and quoted their counts in the detail of its check `inputs_all_pass`, in the form `37/37 PASS`. The pattern `(\d+)/(\d+) PASS` finds them; they must equal our counts of the reference report, the Rust solver report and the determinism report, in this order.

```
SUMMARY = "Revision/dark_sector/dirac16complex/outputs/eos-summary.json"
EOS = "Revision/dark_sector/dirac16complex/reports/eos-checks.json"
summary_numbers = read_report(SUMMARY)["checks"]  # a dictionary of two numbers
say(f"the dark-sector summary quotes: {summary_numbers}")
quoted_eos = (summary_numbers["pass"], summary_numbers["total"])
check_reproduces(not_reports == [SUMMARY] and quoted_eos == counted[EOS],
                 f"the dark-sector summary quotes the {counted[EOS][1]} checks of "
                 "eos-checks.json that we counted",
                 f"{SUMMARY}, key checks")
```

The summary file that the search of In [5] set aside holds, under its key `checks`, the dictionary `{'total': 13, 'pass': 13}` (Out [6] prints it as Python writes a dictionary, with single quotes). The program that writes the equations of state of the dark sector writes both this file and the report `eos-checks.json`, and the two numbers are the totals of that report. The check requires that the summary file is the only file set aside by the search and that its two numbers, in the order (passed, total), equal our count of `eos-checks.json`. Out [6] ends with this PASS line.

In [7], a bar for every report.

```
from matplotlib.patches import Patch  # a coloured square for the legend

ENGINE_COLOURS = {"Wolfram": "#2a78d6", "Python": "#eb6834", "Rust": "#1baf7a",
                  "lead": "#eda100"}
ENGINE_NAMES = {"Wolfram": "Wolfram Language", "Python": "Python",
                "Rust": "Rust", "lead": "Python, the lead's independent checks"}
```

The coloured squares of the legend (as in In [3] of Notebook 00b), a colour for each engine and its full name for the legend.

```
# The report with the most checks, and its number of checks.
largest = max((path for path, _ in REPORTS), key=lambda path: counted[path][1])
longest = counted[largest][1]
```

`max(..., key=...)` returns the path whose value under the key function is largest; `lambda p ath: counted[path][1]` is a function without a name that gives the number of checks of a report. `longest` is that largest number of checks (101, of the Wolfram pairing report, in the run printed in Section 0.20).

```
fig, ax = plt.subplots(figsize=(7.6, 11.6))
rows = np.arange(len(REPORTS))[::-1]  # the first report at the top
for row, (path, engine) in zip(rows, REPORTS):
    total = counted[path][1]
    ax.barh(row, total, height=0.72, color=ENGINE_COLOURS[engine])
    ax.text(total + 0.015 * longest, row, str(total), va="center", fontsize=9)
```

A tall figure, 7.6 by 11.6 inches, one row per report. `np.arange(51)` is 0 to 50, and `[::-1]` reverses it (a step of −1), so that the first report gets the highest row and stands at the top. `ax.barh` draws a horizontal bar of the given length in the given row, coloured by the engine; `ax.text` writes the number just after the end of the bar, at a distance of 1.5 per cent of the longest bar.

```
# The name of each report without its folder and without the ending .json:
names = [path.rsplit("/", 1)[1].removesuffix(".json") for path, _ in REPORTS]
ax.set_yticks(rows, labels=names, fontsize=8.5)
ax.set_xlim(0, 1.12 * longest)  # room for the longest bar and its number
ax.grid(False, axis="y")  # vertical grid lines only
ax.set_xlabel("number of checks in the report (none has the verdict FAIL)")
ax.set_title(f"The {len(REPORTS)} verifier reports of the Revision record: "
             f"{all_checks} checks")
ax.legend(handles=[Patch(color=ENGINE_COLOURS[e], label=ENGINE_NAMES[e])
                   for e in ENGINE_COLOURS], loc="lower right", fontsize=8)
```

`rsplit("/", 1)[1]` keeps the part after the last `/`, the file name, and `removesuffix` removes the ending. The names label the rows, in a font of 8.5 points so that 51 names fit. The horizontal axis runs from 0 to 1.12 times the longest bar, which leaves room for the longest bar and its number; because the limit is computed from the counts, the picture stays complete when a report gains checks. Only vertical grid lines are drawn; the axis label says that no check has the verdict FAIL (a bar counts all checks of its report, the five NOT-AVAILABLE ones of the comparison with the author's notebook included); the title contains the total; the legend names the four engines.

```
largest_name = largest.rsplit("/", 1)[1]  # its file name without the folders
n_wolfram, n_python, n_rust, n_lead = (engine_totals[engine] for engine in
                                       ("Wolfram", "Python", "Rust", "lead"))
totals_text = (f"{n_wolfram} Wolfram Language, {n_python} Python, {n_rust} Rust "
               f"and {n_lead} lead checks")
```

The file name of the largest report, for the caption. The four engine totals are unpacked into four names and written into a sentence for the caption, so that the caption always states the counts of the record as it is.

```
save_figure(fig, "checks_by_report",
            f"The number of checks in each of the {len(REPORTS)} verifier reports "
            r"of the Revision record (horizontal axis, a count; one bar per report, "
            r"named on the vertical axis and grouped by folder: algebra, the metric "
            r"with $b_4$, theory, field equations for $a_4$ and $b_4$, GKD and "
            r"Lovelock with the comparison with the author's outputs, Kohn-Sham, "
            r"dark sector, pairing, lead checks). "
            r"The colour gives the engine: blue Wolfram Language, orange Python, "
            r"aqua Rust, yellow the lead's independent Python checks. Of the "
            f"{all_checks} checks ({totals_text}) {all_passed} have the verdict "
            f"PASS and {not_available} the verdict NOT-AVAILABLE (comparisons with "
            r"values that the author's notebook does not store); none FAILS. The "
            f"largest report is {largest_name} with {longest} checks.")
```

The figure `00c_1_checks_by_report.png` with its caption, partly raw strings and partly f-strings (each piece of the joined caption has its own prefix). The caption takes the number of reports, the totals, the number of passed and of NOT-AVAILABLE checks and the largest report from the computation, so it always states the record as it is. What Figure 00c.1 shows: one bar per report, grouped by folder from the algebra at the top to the lead's checks at the bottom; the longest bar belongs to the report that the caption names (the Wolfram pairing report, 101 checks), and every colour occurs, so every engine contributes.

In [8], two independent engines.

```
SUBJECTS = [  # (subject, Wolfram report, Python report)
    ("the gammas, Pin(4,4), Spin(4,4)", "algebra/reports/wolfram-algebra.json",
     "algebra/reports/python-algebra.json"),
    ("the metric with b4", "metric_b4/results/wolfram-metric-b4-report.json",
     "metric_b4/results/python-metric-b4-report.json"),
    ("geometry and Dirac operator with b4",
     "theory/reports/wolfram-field-theory-b4.json",
     "theory/reports/python-field-theory-b4.json"),
    ("Lagrangians, field equations, EMT", "theory/reports/wolfram-field-theory.json",
     "theory/reports/python-field-theory.json"),
    ("scope of the non-triviality", "theory/reports/wolfram-scope.json",
     "theory/reports/python-scope.json"),
    ("field equations for a4", "field_equations_a4/reports/wolfram-a4-report.json",
     "field_equations_a4/reports/python-a4-report.json"),
```

The list `SUBJECTS` names the eleven subjects that have two independent verifiers, each as a group of three: a short name, the Wolfram report and the Python report (paths without `Revision/`). Its

first six entries are the gammas and the groups, the metric with b_4 compared with the author's notebook, the geometry and the Dirac operator with b_4, the field theory (Lagrangians, field equations, energy-momentum tensor), the scope of the non-triviality, and the field equations for a_4.

```
    ("GKD and the Lovelock tensors", "gkd_lovelock/results/wolfram-gkd-report.json",
     "gkd_lovelock/results/python-lovelock-report.json"),
    ("Kohn-Sham theory", "kohn_sham/reports/ks-theory-wolfram.json",
     "kohn_sham/reports/ks-theory-python.json"),
    ("pairing theorems T1, T2, Q", "pairing/reports/wolfram-pairing.json",
     "pairing/reports/python-pairing.json"),
    ("pairing theorem T3", "pairing/kohn_sham/reports/wolfram-t3.json",
     "pairing/kohn_sham/reports/python-t3.json"),
    ("the completion of T3", "pairing/kohn_sham/reports/wolfram-t3-completion.json",
     "pairing/kohn_sham/reports/python-t3-completion.json"),
]
```

The other five: GKD and the Lovelock tensors (the Rust results checked by Wolfram and by Python), the Kohn-Sham theory, the pairing theorems T1, T2 and Q, the theorem T3, and its completion. These are the eleven subjects of the remark "Two independent verifiers" of Section 0.18 (rows 1 to 3, 5, 6, 8 and 11 of the ledger, two subjects of row 12, one of row 7 and one more of row 2).

```
say("subject                                Wolfram  Python")
wolfram_numbers, python_numbers = [], []
for subject, wolfram, python in SUBJECTS:
    wolfram_numbers.append(counted["Revision/" + wolfram][1])
    python_numbers.append(counted["Revision/" + python][1])
    say(f"{subject:36} {wolfram_numbers[-1]:7d} {python_numbers[-1]:7d}")
```

For each subject the totals of its two reports are taken from `counted` and printed: the table of Out [8]. `[-1]` is the entry just appended.

```
widest = max(wolfram_numbers + python_numbers)  # the longest of the 22 bars
fig, ax = plt.subplots(figsize=(7.0, 6.6))
rows = np.arange(len(SUBJECTS))[::-1]
height = 0.38  # two bars in each row
ax.barh(rows + height / 2, wolfram_numbers, height, color="#2a78d6",
        edgecolor="white", linewidth=1.5, label="Wolfram Language verifier")
ax.barh(rows - height / 2, python_numbers, height, color="#eb6834",
        edgecolor="white", linewidth=1.5, label="Python verifier (sympy)")
for row, w, p in zip(rows, wolfram_numbers, python_numbers):
    ax.text(w + 0.015 * widest, row + height / 2, str(w), va="center", fontsize=8)
    ax.text(p + 0.015 * widest, row - height / 2, str(p), va="center", fontsize=8)
```

`wolfram_numbers + python_numbers` joins the two lists of eleven numbers into one list of twenty-two, and `max` finds the longest bar. The figure is 7.0 by 6.6 inches. Two bars in each row, the Wolfram bar half a bar height above the middle of the row and the Python bar half a bar height below it; the numbers are written after the ends of the bars, at a distance of 1.5 per cent of the longest bar.

```
ax.set_yticks(rows, labels=[subject for subject, _, _ in SUBJECTS])
ax.set_xlim(0, 1.12 * widest)  # room for the longest bar and its number
ax.grid(False, axis="y")
ax.set_xlabel("number of checks (every one has the verdict PASS)")
ax.set_title("Eleven subjects, each checked by two independent verifiers")
ax.legend(loc="lower right")
save_figure(fig, "two_verifiers",
            r"For each of eleven subjects of the Revision record (vertical axis), "
            r"the number of checks of its Wolfram Language verifier (blue, upper "
            r"bar) and of its independent Python verifier (orange, lower bar); "
            r"horizontal axis a count. The two verifiers share no code; each also "
            r"checks statements that the other does not, so the numbers differ. "
            f"Together they hold {sum(wolfram_numbers)} Wolfram and "
            f"{sum(python_numbers)} Python checks, all PASS.")
```

Labels, a horizontal axis from 0 to 1.12 times the longest bar (computed from the counts, as in In [7]), title, legend and the figure `00c_2_two_verifiers.png`. What Figure 00c.2 shows: every subject has two bars, so every subject was checked twice, in two languages; the bars differ in length because the two verifiers were written independently and test partly different statements.

```
report("checks of the Wolfram verifiers of the eleven subjects",
       sum(wolfram_numbers))
report("checks of the Python verifiers of the eleven subjects", sum(python_numbers))
check(all(w > 0 and p > 0 for w, p in zip(wolfram_numbers, python_numbers)),
      "each of the eleven subjects has a Wolfram and a Python verifier")
```

The two sums are printed (443 Wolfram and 404 Python checks in the run printed in Section 0.20), and the check requires a nonzero number of checks in both verifiers of every subject.

In [9], the honesty ledger.

```
R = "Revision/"
K = R + "pairing/kohn_sham/reports/"  # the folder of the T3 reports
D = R + "dark_sector/"  # the folder of the dark-sector reports
```

Three short names for the beginnings of paths, so that the lines of the ledger stay narrow: the folder `Revision/`, the folder of the reports on T3, and the folder of the dark sector.

```
LEDGER = [  # (statement, label, note, the reports that verify it)
    ("the gammas, C, Gamma, B; Pin(4,4) and Spin(4,4)", "PROVED", "",
     [R + "algebra/reports/wolfram-algebra.json",
      R + "algebra/reports/python-algebra.json"]),
    ("Lagrangians, equations, EMT; dirac16complex quantised", "PROVED",
     "the EMT operator for lambda != 0: proved only in a finite Fock model",
     [R + "theory/reports/wolfram-field-theory.json",
      R + "theory/reports/python-field-theory.json",
      R + "theory/fock_quartic/reports/fock-quartic.json",
      R + "theory/reports/wolfram-field-theory-b4.json",
      R + "theory/reports/python-field-theory-b4.json",
      R + "theory/reports/wolfram-field-theory-b4-emt-C.json",
```

```
  R + "theory/reports/wolfram-field-theory-b4-emt-G.json"]),
("the exact scope of the non-triviality", "PROVED", "",
 [R + "theory/reports/wolfram-scope.json",
  R + "theory/reports/python-scope.json"]),
```

The list `LEDGER` holds the twenty rows of the table of Section 0.18, each as a group of four: the statement, the label, the note (an empty string `""` for a row that needs none), and the list of the reports that verify the row. Rows 1 to 3 are the exact results of the algebra and of the field theory, each verified by a Wolfram and a Python report. Row 2 names the quantisation of dirac16complex only, because the field dirac16complex00 is a classical field and is not quantised. Row 2 also holds the report of the finite Fock model (`lambda != 0` is $\lambda \neq 0$ written with plain characters: `!=` means "is not equal to"), and its note says how far that report reaches: the energy-momentum tensor (EMT) operator for $\lambda \neq 0$ is proved only in that finite model (Section 0.18); its last four reports are those of the field theory with b_4.

```
("EMT conservation identities; the spin connection", "PROVED", "",
 [R + "lead_checks/reports/emt-divergence-and-spin-connection.json"]),
("the field equations for a4 and b4; the vacuum", "PROVED", "",
 [R + "field_equations_a4/reports/wolfram-a4-report.json",
  R + "field_equations_a4/reports/python-a4-report.json",
  R + "field_equations_a4/reports/python-b4-block.json",
  R + "metric_b4/results/vacuum-b4.json",
  R + "lead_checks/reports/einstein-gauss-bonnet-a4.json"]),
("GKD and the three Lovelock tensors", "PROVED", "",
 [R + "gkd_lovelock/results/lovelock-report.json",
  R + "gkd_lovelock/results/gkd-selftest.json",
  R + "gkd_lovelock/results/wolfram-gkd-report.json",
  R + "gkd_lovelock/results/python-lovelock-report.json"]),
```

Rows 4 to 6, all PROVED: the conservation identities (one report of the lead), the field equations for a_4 and b_4 with the vacuum (the Wolfram and the Python verifier, the block of b_4, the vacuum of the metric with b_4 and the lead's independent check, five reports), and GKD and the Lovelock tensors (the four reports of the folder `gkd_lovelock/results`).

```
("the curvature agrees with the author's stored outputs", "PROVED",
 "5 of 78 comparisons NOT-AVAILABLE: the author's notebook stores no value",
 [R + "gkd_lovelock/comparison/author-comparison-report.json",
  R + "metric_b4/results/wolfram-metric-b4-report.json",
  R + "metric_b4/results/python-metric-b4-report.json"]),
("Kohn-Sham theory: blocks, rescaling, exchange", "PROVED", "",
 [R + "kohn_sham/reports/ks-theory-wolfram.json",
  R + "kohn_sham/reports/ks-theory-python.json",
  R + "kohn_sham/reports/ks-b4-wolfram.json"]),
```

Row 7, PROVED, has both reports and a note, like row 2: the note says that five of the 78 comparisons with the author's earlier notebook could not be made (Section 0.18); its other two reports compare the metric with b_4 with the author's notebook of 2026-10-14. Row 8 holds the exact parts of the Kohn-Sham theory, with the report that the good-sector problem does not contain b_4.

```
("Kohn-Sham states along the deflating history", "COMPUTED",
 "at the tip cutoff L = 3; the large-L limit not established for every state",
 [R + "kohn_sham/reports/ks-rust-solver.json",
  R + "kohn_sham/reports/ks-reference.json",
  R + "kohn_sham/reports/ks-crosscheck.json",
  R + "kohn_sham/reports/ks-rust-determinism.json",
  R + "kohn_sham/reports/ks-rust-mermin-roots.json",
  R + "kohn_sham/tip_convergence/tip-convergence.json"]),
("the Kohn-Sham history of a4 is a prescribed background", "ASSUMED", "",
 [R + "field_equations_a4/reports/ks-source-conditions.json",
  R + "field_equations_a4/ks_source/reports/ks-source-a4.json"]),
```

Row 9 is COMPUTED: the numerical Kohn-Sham states, with the five reports of the two solvers and of their comparisons and, sixth, the study of the tip cutoff. Its note says that the states are computed with the cutoff $L = 3$ and that the limit of large L is not established for every state (Section 0.18). Row 10 is ASSUMED, and its two reports give the reason (Section 0.18).

```
("pairing T1, T2 (Z2 mirror ASSUMED) and Q", "PROVED", "",
 [R + "pairing/reports/wolfram-pairing.json",
  R + "pairing/reports/python-pairing.json",
  R + "metric_b4/results/consequences-b4.json"]),
("T3 and its completion (Z2 mirror ASSUMED): +M and -M", "PROVED", "",
 [K + "wolfram-t3.json", K + "python-t3.json",
  K + "wolfram-t3-completion.json", K + "python-t3-completion.json"]),
("T3 shown on computed Kohn-Sham states (not a proof)", "COMPUTED", "",
 [K + "t3-rust-demo.json", K + "t3-reference-demo.json"]),
("charge conjugation C, Gamma C; the local U(1) law", "PROVED", "",
 [R + "lead_checks/reports/charge-conjugation-and-u1.json"]),
```

Rows 11 to 14. Rows 11 and 12, PROVED: the pairing theorems, whose statements name the ASSUMED Z2 mirror where it enters (row 11 also holds the report that T1 and T2 hold unchanged with b_4); row 12 holds the four reports on T3 and on its completion. Row 13, COMPUTED: the two numerical demonstrations of T3, labelled in their statement as what they are, not a proof. Row 14, PROVED: the charge-conjugation matrices with the local conservation law of the charge; its statement says "the local U(1) law" because only the local law is proved (In [3]).

```
("the dark-sector investigation: what each field gives", "COMPUTED", "",
 [D + "dirac16complex/reports/derivation-checks.json",
  D + "dirac16complex/reports/ks-history-run.json",
  D + "dirac16complex/reports/eos-checks.json",
  D + "dirac16complex/reports/independent-checks.json",
  D + "dirac16complex00/reports/python-derive-eos.json",
  D + "dirac16complex00/reports/python-independent-numerics.json"]),
```

Row 15, COMPUTED: the six reports of the dark-sector investigation. The rows 16 to 20 have an empty list of reports and a note:

```
("a time-varying dark sector from the fields", "HYPOTHESIS",
 "investigated (row 15): results stated, not established", []),
```

```
    ("our universe has a partner of opposite charge", "HYPOTHESIS",
     "the T1 maps exist; that a partner exists is not shown", []),
    ("the total charge Q of one universe is constant", "OPEN",
     "holds only with no flux through the brane (ASSUMED, not derived)", []),
    ("the big bang creates universes in pairs", "OPEN",
     "not proved: no creation process, rate or amplitude", []),
    ("what produces the excess of matter over antimatter", "OPEN",
     "the theory as built does not produce it", []),
]
```

The five entries without reports are the honest answers of Section 0.2, written as data, so that the computer can check them. Row 16 says that the dark-sector hypothesis was investigated (row 15) and is not established. Row 18 is the limit of row 14: the total charge is constant only if no charge flows through the brane, a condition that is ASSUMED and not derived. Row 20 is written as the open question that it is, with the note that the theory as built does not produce the excess.

```
say("row label       passed of all  statement")
row_passed, row_totals = [], []
for number, (statement, label, note, paths) in enumerate(LEDGER, 1):
    passed = sum(counted[path][0] for path in paths)
    total = sum(counted[path][1] for path in paths)
    row_passed.append(passed)
    row_totals.append(total)
    say(f"{number:3d} {label:10} {passed:7d} of {total:3d}  {statement}")
```

`enumerate(LEDGER, 1)` numbers the rows from 1. For each row, the passed checks and the checks of its reports are added (a sum over an empty list is 0), kept in the lists `row_passed` and `row_totals` for the checks below and for the picture of In [10], and a line of the table of Out [9] is printed. Row 7 shows 121 of 126 (the 73 of 78 of the earlier comparison and the 48 checks of the comparison with b_4).

```
say("The notes of the rows:")
for number, (statement, label, note, paths) in enumerate(LEDGER, 1):
    if note:  # rows 2, 7 and 9, and the HYPOTHESIS and OPEN rows
        say(f"{number:3d} {label:10} {note}")
```

The notes of the eight rows that have one are printed below the table (a non-empty string counts as true in `if`): rows 2, 7 and 9 and the five rows without a report.

```
used = sorted(path for _, _, _, paths in LEDGER for path in paths)
check(used == sorted(path for path, _ in REPORTS),
      f"every one of the {len(REPORTS)} reports belongs to exactly one row")
```

`used` is the sorted list of all reports named in all rows (a comprehension with two `for` parts runs through the rows and, inside each row, through its reports). It must equal the sorted list of the 51 reports: every report appears, and none twice (a report named twice would appear twice in `used`).

```
check(all((label in ("HYPOTHESIS", "OPEN")) == (paths == [] and note != "")
          for _, label, note, paths in LEDGER)
```

```
      and all(paths != [] for _, label, _, paths in LEDGER
              if label not in ("HYPOTHESIS", "OPEN")),
      "OPEN and HYPOTHESIS rows have a note and no report; the others have reports")
```

For every row, two statements must be both true or both false: the label is HYPOTHESIS or OPEN; the row has no report and has a note. And every row with another label must have at least one report (`paths != []`; the `if` inside the brackets keeps only those rows). Together: a statement is labelled HYPOTHESIS or OPEN exactly when nothing in the record establishes it, and then the ledger says why.

```
not_all_pass = [number for number, passed, total
                in zip(range(1, len(LEDGER) + 1), row_passed, row_totals)
                if passed != total]  # the rows with a check that is not PASS
check(not_all_pass == [7] and row_totals[6] - row_passed[6] == not_available
      and "NOT-AVAILABLE" in LEDGER[6][2],
      f"only row 7 has checks that are not PASS: its {not_available} "
      "NOT-AVAILABLE comparisons")
```

`not_all_pass` lists the numbers of the rows in which the passed checks are fewer than the checks. It must be the list `[7]`, and the missing checks of row 7 must be exactly the five NOT-AVAILABLE comparisons counted in In [5]; its note (the third entry of the row, `LEDGER[6][2]`, because Python counts the rows from 0) must say so. So no row of the ledger rests on a check that failed.

```
labels = [label for _, label, _, _ in LEDGER]
check([labels.count(name) for name in
       ("PROVED", "COMPUTED", "ASSUMED", "HYPOTHESIS", "OPEN")] == [11, 3, 1, 2, 3],
      "the ledger: 11 PROVED, 3 COMPUTED, 1 ASSUMED, 2 HYPOTHESIS and 3 OPEN rows")
```

The labels are counted: eleven rows PROVED, three COMPUTED, one ASSUMED, two HYPOTHESIS and three OPEN. Out [9] ends with four PASS lines.

In [10], the ledger as a picture.

```
LABEL_COLOURS = {"PROVED": "#2a78d6", "COMPUTED": "#eb6834", "ASSUMED": "#1baf7a"}
widest = max(row_totals)  # the row with the most checks
fig, ax = plt.subplots(figsize=(8.6, 9.0))
rows = np.arange(len(LEDGER))[::-1]  # the first row of the ledger at the top
```

A colour for each label that has reports, the largest number of checks of a row (210, row 2, in the run printed in Section 0.20), a tall figure of 8.6 by 9.0 inches, and the row positions with the first row at the top.

```
for number, row, (statement, label, note, paths), passed, total in zip(
        range(1, len(LEDGER) + 1), rows, LEDGER, row_passed, row_totals):
    # The statement is written just above its bar (va="bottom": the text starts
    # at the given height and extends upwards).
    ax.text(0, row + 0.26, f"{number}. {statement}", va="bottom", fontsize=9.5)
    if total > 0:
```

```
        ax.barh(row, total, height=0.42, color=LABEL_COLOURS[label])
        if passed == total:
            after = f"{total} checks: {label}"
        else:  # row 7: say how many checks are NOT-AVAILABLE
            after = f"{passed} PASS, {total - passed} NOT-AVAILABLE: {label}"
        ax.text(total + 0.012 * widest, row, after, va="center", fontsize=9)
    else:  # no report: the label and the note, in grey
        ax.text(0, row, f"{label}: {note}", va="center", fontsize=9,
                color="#52514e")
```

`zip` runs through five lists together: the row numbers 1 to 20, the positions, the rows of the ledger, their passed checks and their totals. For each row the statement is written just above the place of its bar. A row with checks gets a bar of that length in the colour of its label, with a text after it, at a distance of 1.2 per cent of the longest bar: the number of checks and the label, or, for row 7, the numbers of PASS and of NOT-AVAILABLE checks and the label. A row without checks gets its label and its note in grey instead.

```
ax.set_yticks([])  # the statements are written above the bars instead
ax.set_xlim(0, 1.39 * widest)  # room for the longest bar and the text after it
ax.set_ylim(-0.6, len(LEDGER) - 0.1)
ax.grid(False, axis="y")
ax.set_xlabel("number of checks in the reports of the row (none FAILS)")
ax.set_title("The honesty ledger at a glance")
ax.legend(handles=[Patch(color=colour, label=label_name)
                   for label_name, colour in LABEL_COLOURS.items()],
          loc="lower right", fontsize=9)
```

No labels on the vertical axis (the statements stand above the bars). The horizontal axis runs to 1.39 times the longest bar, computed from the counts, which leaves room for the longest bar and the text after it; the vertical limits leave room for the statement of the top row; the axis label says that no check of the ledger fails; the legend names the three colours.

```
save_figure(fig, "ledger",
            r"The honesty ledger of the book at a glance: one row per main "
            r"statement (written above its bar), the length of its bar the number of "
            r"checks in the reports that verify it (horizontal axis, a count), the "
            r"colour its label: blue PROVED, orange COMPUTED, aqua ASSUMED. Every "
            r"check is PASS except five NOT-AVAILABLE comparisons of row 7. The "
            f"ASSUMED row has {row_totals[9]} checks, which show why the Kohn-Sham "
            r"history of $a_4$ must be assumed. The last five rows have no bar, "
            r"because no check of the record establishes them: two hypotheses (a "
            r"time-varying dark sector, investigated in row 15 but not established; "
            r"a partner universe of opposite charge) and three open questions (a "
            r"constant total charge of one universe, which needs an assumed "
            r"condition at the brane; the creation of universes in pairs, not "
            r"proved; what produces the excess of matter "
            r"over antimatter, which the theory as built does not produce).")
```

The figure `00c_3_ledger.png`; `row_totals[9]` is the number of checks of row 10, the ASSUMED row (28 in the run printed in Section 0.20). What Figure 00c.3 shows: fifteen bars, eleven blue,

three orange (the Kohn-Sham numbers, the demonstrations of T3 and the dark-sector investigation) and one short aqua bar (the prescribed background), and five rows without a bar at the bottom: the two hypotheses and the three open questions. The picture makes the honesty rule visible: a statement without checks is never drawn as if it had them.

In [11], what the pairing record does not establish.

```
PAIRING = "Revision/pairing/reports/python-pairing.json"
not_established = read_report(PAIRING)["not_established"]  # a list of sentences
for number, sentence in enumerate(not_established, 1):
    say(f"{number:2d}. {sentence}")
check_reproduces(len(not_established) == 12
                 and not_established[0].startswith("No creation process"),
                 "the pairing record lists 12 things it does not establish, "
                 "the first: no creation process",
                 f"{PAIRING}, key not_established")
```

The Python pairing report holds, under its key `not_established`, a list of sentences; the loop prints them numbered from 1 (Out [11]). The check requires twelve sentences, the first beginning with No creation process. In plain words, the twelve say: nothing in the equations creates a universe or a pair; no rate, probability or amplitude is derived; no conservation law forces pairing, and a single universe is an equally valid solution; the zero total of a T1 pair is an identity for one field configuration and its image, not a cancellation between two independently quantised universes; T2 does not reverse the energy-momentum tensor; the Z2 mirror is ASSUMED; the back-reaction on gravity is not part of the theorems; nothing forces the partner to exist; T1 pairs (m, λ) with $(-m, -\lambda)$, not with $(-m, \lambda)$; a positive-norm quantum state space is not established by the theorems; the edge $z = \pi/2$ is a degenerate surface of the metric with no derived junction condition; and the Kohn-Sham level T3 is proved separately, with its own hypotheses.

In [12], how a fingerprint reacts to a change.

```
def text_fingerprint(text):
    """The sha256 fingerprint of a text (encoded as UTF-8 bytes)."""
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

`text.encode("utf-8")` turns the text into its bytes, `hashlib.sha256` computes the fingerprint of the bytes, and `.hexdigest()` writes it as 64 hexadecimal characters.

```
for text in ("Universes in Pairs", "Universes in pairs"):
    digest = text_fingerprint(text)
    say(f"{text}:")
    say(f"    {digest}")
```

Two texts that differ only in the capital or small letter p. Out [12] shows their fingerprints: they have nothing visible in common.

```
SENTENCE = ("The rule above every other rule of this book is honesty: it never "
            "writes proved for a statement that is not proved.")
```

```
original = text_fingerprint(SENTENCE)
changed_characters = []  # for each position: how many of the 64 characters change
for position in range(len(SENTENCE)):
    letter = SENTENCE[position]
    altered = SENTENCE[:position] + chr(ord(letter) + 1) + SENTENCE[position + 1:]
    new = text_fingerprint(altered)
    changed_characters.append(sum(1 for a, b in zip(original, new) if a != b))
counts = np.array(changed_characters)
```

A sentence of 115 characters and its fingerprint. For every position, `ord(letter)` is the number of the character in the computer's alphabet (for example 97 for a), `chr(... + 1)` is the next character (b), and `SENTENCE[:position] + ... + SENTENCE[position + 1:]` is the sentence with that one character replaced (`[:position]` is the part before it, `[position + 1:]` the part after it). The fingerprint of the altered sentence is compared with the original character by character (`zip` pairs the two strings), and the number of differing characters is stored. `counts` is the array of the 115 numbers.

```
report("characters of the sentence", len(SENTENCE))
report("changed characters of the fingerprint: smallest, mean, largest",
       f"{counts.min()}, {counts.mean():.2f}, {counts.max()}")
```

Out [12]: 115 characters; the smallest number of changed characters is 54, the mean 60.03, the largest 64. The mean is what Section 0.18 derived for a random string, 60.

```
values, how_often = np.unique(counts, return_counts=True)  # each count, how often
fig, ax = plt.subplots()
ax.bar(values, how_often, width=0.8, color="#2a78d6", edgecolor="white",
       linewidth=1.5, label="one-character changes of the sentence")
ax.axvline(60, color="black", linewidth=1.2, linestyle=":",
           label="expected for a random fingerprint: 60")
ax.set_xlim(-1, 65)
```

`np.unique(..., return_counts=True)` returns the different values that occur and how often each occurs: a **histogram**. The bars are drawn at those values; `ax.axvline(60, ...)` draws a dotted vertical line at 60; the horizontal axis shows the whole range from 0 to 64, so that the empty left part is visible.

```
# "\n" inside a string starts a new line of the text.
ax.text(3, 0.55 * how_often.max(), "a fingerprint that changed only a little\n"
        "would give a bar on this side: none did", color="#52514e", fontsize=9)
ax.set_xlabel("number of the 64 characters of the fingerprint that change")
ax.set_ylabel("number of changed sentences")
ax.set_title("Change one character, and the fingerprint changes completely")
ax.legend(loc="upper left")
```

A note written into the empty left part of the picture, the labels, the title and the legend.

```
save_figure(fig, "fingerprints",
            r"How much a sha256 fingerprint changes when one character of a text "
```

```
            r"changes: for each of the 115 characters of one sentence, the "
            r"character was replaced by the next one and the number of the 64 "
            r"hexadecimal characters of the fingerprint that changed was counted; "
            r"horizontal axis that number (0 to 64), vertical axis how many of the "
            r"115 altered sentences gave it. Every change altered between "
            f"{counts.min()} and {counts.max()} of the 64 characters, on average "
            f"{counts.mean():.2f}, as for a random string (dotted line at 60): none "
            f"of the {len(counts)} one-character changes left the fingerprint "
            r"nearly the same.")
check(counts.min() > 32 and abs(counts.mean() - 60.0) < 1.0,
      f"all {len(counts)} one-character changes alter more than half of the "
      "fingerprint")
```

The figure `00c_4_fingerprints.png`; its caption takes the smallest, the mean and the largest count from the computation (54, 60.03 and 64) and states only what was measured: none of these 115 changes left the fingerprint nearly the same. The check requires that every change alters more than 32 of the 64 characters, that is more than half of them (`>` is "greater than"), and that the mean lies within 1 of 60. What Figure 00c.4 shows: all bars stand close to the dotted line at 60, and the left half of the picture is empty: no one-character change left a fingerprint nearly unchanged.

In [13], the fingerprints recorded by the reports.

```
def file_fingerprint(path):
    """The sha256 fingerprint of the bytes of the repository file path."""
    return hashlib.sha256(repository_file(path).read_bytes()).hexdigest()
```

As `text_fingerprint`, but for the bytes of a file of the repository (`read_bytes` reads them unchanged).

```
RECORDED = []  # (report, file, recorded fingerprint)
GKD = "Revision/gkd_lovelock/results/wolfram-gkd-report.json"
gkd = read_report(GKD)
for path, value in {**gkd["inputSha256"], **gkd["sourceSha256"]}.items():
    RECORDED.append((GKD, path, value))
LOVELOCK = "Revision/gkd_lovelock/results/python-lovelock-report.json"
for name, value in read_report(LOVELOCK)["inputsSha256"].items():
    RECORDED.append((LOVELOCK, "Revision/gkd_lovelock/results/" + name, value))
```

The list `RECORDED` collects groups of three: the report, the file and the fingerprint the report recorded for it. The Wolfram report of GKD and the Lovelock tensors stores two dictionaries from file paths to fingerprints; `{**a, **b}` makes one dictionary of both, and all 15 entries are collected. The Python Lovelock report stores 2 more, under file names without the folder, which is added in front.

```
IN_DETAILS = [  # (report, check, the file whose fingerprint the detail holds)
    ("ks-theory-wolfram.json", "fixture_input", "algebra/gammas.json"),
    ("ks-theory-python.json", "fixture_input", "algebra/reports/python-gammas.json"),
    ("ks-reference.json", "theory_input_coefficients", "kohn_sham/ks-theory.json"),
    ("ks-rust-solver.json", "theory_input_coefficients", "kohn_sham/ks-theory.json"),
    ("ks-rust-solver.json", "gamma_fixture_numeric", "algebra/gammas.json"),
    ("ks-rust-solver.json", "thermo_mu_vs_40digit_roots",
```

```
     "kohn_sham/solver/tools/mermin-roots-40digit.json"),
    ("ks-crosscheck.json", "problem_definition_identical",
     "kohn_sham/ks-theory.json"),
]
```

Seven more fingerprints are written inside the detail texts of checks of five Kohn-Sham reports; this list names, for each, the report, the check and the file.

```
for name, check_name, path in IN_DETAILS:
    report_path = "Revision/kohn_sham/reports/" + name
    detail = next(entry["detail"] for entry in read_report(report_path)["checks"]
                  if entry["name"] == check_name)
    value = re.search(r"\(sha256 ([0-9a-f]{16,64})\)", detail).group(1)
    RECORDED.append((report_path, "Revision/" + path, value))
```

For each, the detail of the named check is read, and the regular expression finds the fingerprint in it: `\(` and `\)` are the round brackets themselves (a backslash makes a special character ordinary), `[0-9a-f]` is one hexadecimal character, and `{16,64}` means 16 to 64 of them. `re.search` finds the first match, and `.group(1)` is the part in the inner brackets, the fingerprint (complete, or only its first 16 characters).

```
changed = []  # files whose fingerprint today differs from the recorded one
for report_path, path, value in RECORDED:
    same = file_fingerprint(path).startswith(value)  # a full or shortened value
    if not same:
        changed.append(path)
    status = "same" if same else "CHANGED"
    short_path = path.removeprefix("Revision/")  # the path without Revision/
    report_name = report_path.rsplit("/", 1)[1]  # the file name of the report
    say(f"{status:7} {short_path:52} {report_name}")
```

For each of the 24, the fingerprint of today's file is computed and must begin with the recorded value (`startswith` accepts a shortened value, and a complete value only if the two are equal). One line per fingerprint is printed: the 24 lines of Out [13], all reading same.

```
recorded_gammas = next(value for report_path, path, value in RECORDED
                       if report_path.endswith("ks-theory-wolfram.json"))
today_gammas = file_fingerprint("Revision/algebra/gammas.json")
say("Revision/algebra/gammas.json, recorded in ks-theory-wolfram.json and today:")
say(f"    {recorded_gammas}")
say(f"    {today_gammas}")
```

One example printed in full: the fingerprint of the file of the gammas as recorded by the Wolfram Kohn-Sham verifier, and as computed today; the two lines of 64 characters in Out [13] are identical.

```
report("recorded fingerprints compared", len(RECORDED))
check_reproduces(len(RECORDED) == 24 and changed == [],
                 "the 24 recorded fingerprints equal the files of today",
                 f"{GKD}, keys inputSha256 and sourceSha256, and five further reports")
```

The check requires 24 recorded fingerprints and no changed file. So these reports read exactly today's versions of these 24 files. (The record holds further fingerprints, for example in the key `inputs` of some reports; this notebook does not compare them.)

In [14], the last check.

```
figure_names = ["00c_1_checks_by_report.png", "00c_2_two_verifiers.png",
                "00c_3_ledger.png", "00c_4_fingerprints.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "the four figure files of this notebook exist")
all_checks_passed()
```

As in Notebook 00b: the four figure files must exist, and the last line prints ALL 18 CHECKS PASSED (notebook 00c): one check each in In [2], In [8], In [11], In [12], In [13] and In [14], two in In [3], three in In [5], three in In [6] and four in In [9].

0.22 Why every run gives the same bytes

Section 0.9 promised that every notebook of the book, run a second time, gives exactly the same bytes: the same printed text, the same pictures, the same files. The Revision record makes the same promise for its long computations. This section explains why such a promise can be kept at all, and what has to be done to keep it. It also explains why two *different* computations of the same number, for example two solvers, never agree exactly, and why their comparison needs a tolerance.

A computer is deterministic. Given the same program and the same input, it performs the same steps and gets the same result. Two runs of a careless notebook can still print different text, for five reasons: (1) it prints the time or the date; (2) it uses random numbers without a fixed starting point; (3) it prints the names of a set, or the files of a folder, in an order that can change from run to run; (4) it adds numbers in an order that depends on the computer, and the order changes the last digits of a sum; (5) it writes files with the line ends of the operating system. The notebooks of the book avoid all five. We now look at each, beginning with the way a computer stores numbers, because reason (4) comes from there.

Binary fractions. We write numbers in the decimal system, with the ten digits 0 to 9 and powers of 10. A computer writes them in the **binary** system, with the two digits 0 and 1 (each one a **bit**) and powers of 2: binary 101 means $1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5$, and binary 0.11 means $1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} = \frac{3}{4}$. A binary fraction with finitely many digits is a whole number divided by a power of 2. The number $0.1 = 1/10$ is not of this kind. Suppose it were, $1/10 = n/2^k$ with whole numbers n and k:

$$2^k = 10\,n$$

(multiply both sides by $10 \cdot 2^k$),

$$2^{k-1} = 5\,n$$

(divide both sides by 2, using $10 = 2 \cdot 5$). Then 5 would divide a power of 2, which is impossible, because the only prime number that divides a power of 2 is 2 itself (every whole number is a product

of prime numbers in exactly one way; this school fact is used here without proof). So the binary expansion of 1/10 never ends, just as the decimal expansion $1/3 = 0.333\ldots$ never ends, and the computer has to cut it off.

Floating-point numbers. Python's numbers with a fraction part (the type `float`, and numpy's `float64`) are **floating-point numbers**: each is a whole number m of exactly 53 binary digits, that is $2^{52} \le m < 2^{53}$, times a power of two, $x = m \cdot 2^e$ (and the same with a minus sign; zero is stored separately). The 53 binary digits correspond to about 16 decimal digits, since $2^{53} = 9007199254740992$ has 16 digits. A number that is not of this form is replaced by the nearest one that is: this is **rounding**. If it lies exactly halfway between two neighbours, the rule is to take the neighbour whose m is even (**round half to even**).

Which power of 2. The size of a stored number fixes its power of 2. Take $x = m \cdot 2^e$ with $2^{52} \le m < 2^{53}$, and multiply both inequalities by 2^e:

$$2^{52+e} \le x < 2^{53+e}$$

(multiplying both sides of an inequality by the positive number 2^e keeps it true, and $2^{52} \cdot 2^e = 2^{52+e}$ by the law of exponents). Now suppose that x lies between 2^k and 2^{k+1} for a whole number k, that is $2^k \le x < 2^{k+1}$. Then

$$52 + e = k, \quad \text{that is} \quad e = k - 52$$

(the intervals from 2^j to 2^{j+1}, one for each whole number j, do not overlap, so x lies in exactly one of them; the line above and the assumption both name that interval, so $52 + e = k$; then subtract 52 on both sides). We call this the **storage rule**: a number between 2^k and 2^{k+1} is stored as $m \cdot 2^{k-52}$ with a 53-digit m.

How 0.1 is stored. The number 0.1 lies between $2^{-4} = 0.0625$ and $2^{-3} = 0.125$; so, by the storage rule with $k = -4$, it must be written as $m \cdot 2^{-4-52} = m \cdot 2^{-56}$ with a 53-digit m:

$$m = 0.1 \cdot 2^{56} = 7205759403792793.6$$

(the condition $x = m \cdot 2^{-56}$ solved for m; $2^{56} = 72057594037927936$),

$$m \approx 7205759403792794$$

(rounding to the nearest whole number; it lies between $2^{52} = 4503599627370496$ and 2^{53}, as it must),

$$0.1 \approx \frac{7205759403792794}{2^{56}} = \frac{3602879701896397}{2^{55}} = 0.1000000000000000055511\ldots$$

(the numerator is even, so numerator and denominator can be divided by 2). The stored number is larger than 1/10 by about 5.6×10^{-18}. Notebook 00d prints exactly this fraction and all its decimal digits.

Why $0.1 + 0.2$ is not 0.3. Doubling changes only the power of 2, so 0.2 is stored exactly twice the stored 0.1:

$$0.2 \approx \frac{3602879701896397}{2^{54}} = \frac{7205759403792794}{2^{55}}$$

(multiply the stored 0.1 by 2, then write it over 2^{55} by multiplying numerator and denominator by 2). The number 0.3 lies between 2^{-2} and 2^{-1}, so, by the storage rule with $k = -2$, it is stored as $m \cdot 2^{-2-52} = m \cdot 2^{-54}$ with $m = 0.3 \cdot 2^{54} = 5404319552844595.2$, rounded:

$$0.3 \approx \frac{5404319552844595}{2^{54}}$$

(the same steps as for 0.1). The sum of the two stored numbers is

$$\frac{3602879701896397}{2^{55}} + \frac{7205759403792794}{2^{55}} = \frac{10808639105689191}{2^{55}}$$

(fractions with the same denominator are added by adding their numerators). This sum lies between 2^{-2} and 2^{-1}, so, by the storage rule with $k = -2$ again, it must be stored as $m \cdot 2^{-54}$, with

$$m = \frac{10808639105689191}{2} = 5404319552844595.5$$

(the sum written over 2^{54}: divide the numerator by 2). This lies exactly halfway between 5404319552844595 and 5404319552844596, and round half to even chooses the even one:

$$0.1 + 0.2 \approx \frac{5404319552844596}{2^{54}}$$

(round half to even). Comparing with the stored 0.3, the numerators differ by 1, so

$$(0.1 + 0.2) - 0.3 = \frac{1}{2^{54}} = 2^{-54} = 5.55 \times 10^{-17}$$

(subtraction of fractions with the same denominator). Python therefore prints $0.1 + 0.2$ as 0.30000000000000004, and the question whether $0.1 + 0.2$ equals 0.3 is answered False. Status: PROVED (by the exact arithmetic above); Notebook 00d checks it.

Machine epsilon and the gaps between numbers. The number 1 is stored with $m = 2^{52}$ and $e = -52$; the next larger stored number has $m = 2^{52} + 1$, so it is $1 + 2^{-52}$. The gap

$$\epsilon = 2^{-52} = 2.22 \times 10^{-16}$$

is called the **machine epsilon**. The number $1 + \epsilon/2 = 1 + 2^{-53}$ lies exactly halfway between 1 and $1 + \epsilon$; round half to even picks 1 (its $m = 2^{52}$ is even), so $1 + 2^{-53}$ is stored as 1. More generally, every stored number x between 2^k and 2^{k+1} (with $2^k \le x < 2^{k+1}$) has the power 2^{k-52} (the storage rule), so the gap to the next stored number, whose m is larger by 1, is 2^{k-52}. Dividing by x:

$$\frac{\text{gap}}{x} = \frac{2^{k-52}}{x} \le \frac{2^{k-52}}{2^k} = 2^{-52} = \epsilon$$

(a fraction becomes larger or stays equal when its denominator is replaced by a smaller or equal one, here $x \geq 2^k$), and

$$\frac{\text{gap}}{x} > \frac{2^{k-52}}{2^{k+1}} = 2^{-53} = \frac{\epsilon}{2}$$

(the same rule with $x < 2^{k+1}$). So every stored number carries the same relative accuracy, between $\epsilon/2$ and ϵ, about 16 significant decimal digits, whatever its size. A difference of one gap is called one **unit in the last place**, one **ulp**.

The order of a sum changes the last digits. Every addition rounds its result. Rounding is deterministic, but it depends on the order of the operations. The smallest example:

$$(1 + 2^{-53}) + 2^{-53} = 1 + 2^{-53} = 1$$

(the bracket is computed first; it is the halfway case above and is rounded to 1; then the same happens again), while

$$1 + (2^{-53} + 2^{-53}) = 1 + 2^{-52}$$

(the bracket is computed first: $2^{-53} + 2^{-53} = 2 \cdot 2^{-53} = 2^{-52}$ exactly, because doubling changes only the power of 2; and $1 + 2^{-52}$ is a stored number). The same three numbers, added in two orders, give two different stored results. In a long sum the effect grows. Notebook 00d adds the million numbers $1/k^2$, $k = 1, \ldots, 10^6$, **forwards** (the largest first) and **backwards** (the smallest first). Forwards, every small term is added to a running total of about 1.6 and is rounded to the gap of that total; backwards, small terms are added to small totals, which loses less. Measured against the exactly rounded sum, the forward sum is wrong by up to 196 ulps and the backward sum by at most 1 ulp; numpy's own way of adding, in pairs and pairs of pairs, by at most 3 ulps (COMPUTED, Notebook 00d). Different computers, or the same computer using several processor cores, may add in different orders; this is why the book always adds in a fixed order.

The error of the method is a different thing. The infinite sum $1 + 1/4 + 1/9 + \cdots$ is $\pi^2/6$ (a theorem of Euler, 1734, used here without proof). Stopping after N terms leaves out the tail $\sum_{k>N} 1/k^2$, and this is not a rounding error but an error of the method. Its size follows from comparing each term with an integral. The function $1/x^2$ decreases, so on the interval from $k-1$ to k it is larger than its value at the right end:

$$\frac{1}{k^2} < \int_{k-1}^{k} \frac{dx}{x^2}$$

(the area under a decreasing curve over an interval of length 1 is larger than the curve's smallest value there, $1/k^2$), and on the interval from k to $k+1$ it is smaller than its value at the left end:

$$\frac{1}{k^2} > \int_{k}^{k+1} \frac{dx}{x^2}$$

(the area is smaller than the largest value, $1/k^2$). Adding these inequalities for $k = N+1, N+2, \ldots$ joins the intervals into one:

$$\int_{N+1}^{\infty} \frac{dx}{x^2} < \sum_{k>N} \frac{1}{k^2} < \int_{N}^{\infty} \frac{dx}{x^2}$$

(sums of inequalities, and integrals over adjacent intervals add up to the integral over their union), and since $\int_a^\infty dx/x^2 = [-1/x]_a^\infty = 1/a$,

$$\frac{1}{N+1} < \frac{\pi^2}{6} - \sum_{k=1}^{N} \frac{1}{k^2} < \frac{1}{N}$$

(the fundamental theorem of calculus with the antiderivative $-1/x$ of $1/x^2$, whose value goes to 0 as x grows). For $N = 10^6$ the method error lies between 9.99999×10^{-7} and 10^{-6}; Notebook 00d measures 9.999995×10^{-7}, more than 10^7 times larger than even the largest rounding difference of the sums (196 ulps of about 2.2×10^{-16} each, that is about 4×10^{-14}).

Tolerances. Two runs of the same program in the same order agree byte for byte. Two *different* computations of the same quantity, such as two solvers, or one solver with two step sizes, agree only up to rounding and up to the errors of their methods. Such a comparison uses a **tolerance**: the largest difference that it accepts as agreement. The tolerance must be fixed *before* the comparison is made; a tolerance chosen after seeing the difference would prove nothing. The Revision record's Kohn-Sham solver was run with its canonical settings and again with refined settings (twice as many integration steps, tighter tolerances of its own, and one equation solved in another, exactly equivalent form that rounds along another path). The tolerances fixed in advance were 10^{-8} for energies, levels and thermodynamic quantities and 10^{-6} for profiles and derivatives. The eight measured largest differences are below their tolerances by factors (**margins**) between 4.7 and 7692 (COMPUTED: `Revision/kohn_sham/reports/ks-rust-determinism.json`; the measured values are quoted by `Revision/kohn_sham/reports/ks-crosscheck.json`, key `rust_matrix_wide_uncertainties`). The eight comparisons are these eight checks of the determinism report:

- `refined_ground_energies`: the ground-state energies;
- `refined_ground_homo_lumo_gap`: the gaps between levels;
- `refined_eigenvalues`: all Kohn-Sham levels;
- `refined_delta_scf`: the excitation energies;
- `refined_profiles`: the profiles;
- `refined_adiabatic_derivatives`: the adiabatic derivatives;
- `refined_thermodynamics`: the thermodynamic quantities;
- `refined_heat_capacity`: the heat capacity.

In the report the check `refined_mermin_root_path` stands between the last two; it is not a comparison with a tolerance but the negative control described next.

A negative control. A **negative control** is a test that must detect a known error; if it does not, the test is useless. The record contains one that teaches a general lesson. An earlier version

of the solver found the chemical potential μ (Chapter 13) with a method that was wrong by up to about 8×10^{-10} in units of the mass m in three thermal states, and it used that method in *both* runs. Both runs made the same rounding error, so their difference was tiny and hid the error. The present refined run finds μ along another rounding path, and its difference from the old result shows the error in full (`Revision/kohn_sham/reports/ks-rust-determinism.json`, check `refined_mermin_root_path`). Two computations that round the same way can agree with each other and both be wrong: a comparison is only as good as the independence of the two computations.

Seeds. A random-number generator of a computer is a formula that produces, step by step, numbers that look random. It starts from a number called the **seed**, and the same seed always gives the same sequence. The notebooks of the book use random numbers only from a generator with a fixed seed, so that every run draws the same numbers.

The order of a set. A Python **set** is an unordered collection of different things, written like `{"x1", "x2"}`. When Python prints a set, the order of its members depends on their **hash values**, numbers computed from the names with a secret key that Python chooses anew every time it starts, unless the **environment variable** `PYTHONHASHSEED` (a named setting that a program receives when it starts) fixes the key. The build of a notebook sets it to 0 and the check run to 1 (Section 0.9), so that a notebook that printed a set in its stored order would fail the check. The notebooks therefore sort every set before printing it.

Line ends. A text file ends each line with special bytes. Linux and macOS use one byte, LF (line feed, the number 10); Windows traditionally uses two, CR LF (carriage return 13, then line feed 10). The same two lines of text written with LF and with CR LF are different bytes, of different lengths and with different fingerprints. The book and the Revision record write every file with LF, on every operating system.

Fingerprints of the results. The two Kohn-Sham solvers of the record wrote, next to their result files, a **manifest**: a file that lists every result file with its sha256 fingerprint (Section 0.18). The record also ran each solver a second time and compared every file of the two runs byte for byte (`Revision/kohn_sham/reports/ks-rust-determinism.json`, check `repeat_byte_identical`; `Revision/kohn_sham/reports/ks-crosscheck.json`, check `reference_repeat_byte_identical`).

Example: what could make two runs differ. Notebook 00d shows each of these facts with a small experiment and reproduces the corresponding checks of the record: it prints how 0.1 is stored and checks the two facts derived above; draws the gaps between stored numbers; adds the million numbers $1/k^2$ in three orders; reads the eight tolerances of the Kohn-Sham solver with the measured differences and the negative control, and draws them; makes random walks with fixed seeds; runs Python twelve times with twelve hash seeds and prints the order of a set each time; shows the bytes of LF and CR LF line ends and confirms that no result file of the record has CR LF line ends; and computes 582 fingerprints and compares them with the two manifests. It draws six figures and ends with ALL 22 CHECKS PASSED (notebook 00d).

0.23 How to run Notebook 00d

Step 1. What this notebook does and what it needs.

Notebook 00d (Why every run gives the same bytes: rounding, order, seeds and line ends) is the file `Revision/textbook/notebooks/00d_reproducible_runs.ipynb` of the repository Dirac_claude.

It shows with small experiments why the notebooks of the textbook print the same numbers and write the same files every time they run: how the computer rounds decimal numbers, why the order of a sum changes its last digits (so that checks allow a tolerance), how a seed fixes random numbers, why names are sorted before they are printed, and why every file is written with the same line ends. It reproduces the tolerance, line-end, fingerprint and repeat checks of the Revision Kohn-Sham record (eight measured differences below their tolerances, a negative control that shows how one shared rounding path hid an error, 584 result files of two solvers without CR LF line ends, 582 recorded sha256 fingerprints equal to the files of today, two repeat runs byte for byte) and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 00d_reproducible_runs.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00d_reproducible_runs.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 00d_reproducible_runs.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/00d.captions.json`
- `Revision/textbook/figures/00d_1_float_spacing.png`
- `Revision/textbook/figures/00d_2_summation_order.png`
- `Revision/textbook/figures/00d_3_tolerances.png`
- `Revision/textbook/figures/00d_4_shared_rounding.png`
- `Revision/textbook/figures/00d_5_seeded_walks.png`
- `Revision/textbook/figures/00d_6_set_orders.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of this notebook exist
ALL 22 CHECKS PASSED (notebook 00d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/00d_reproducible_runs.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 00d_reproducible_runs.ipynb
python -m nbconvert --execute --inplace 00d_reproducible_runs.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "AssertionError: check failed" for a check about the files of the folder Revision/kohn_sham (their number, their line ends or their fingerprints): a file of the Revision record was changed, added or converted after the repository was downloaded. The first command below, run in the repository folder, lists every changed or added file; the second restores a changed file (write the file name that the first command printed instead of FILE); delete a file that the first command lists as untracked.

```
git status
git restore FILE
```

- "CalledProcessError" in the section about the order of a set of names: the notebook could not start Python a second time. Close JupyterLab, activate the environment again, start JupyterLab again from the same terminal and run the notebook again.

0.24 Notebook 00d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/00d_reproducible_runs.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Every notebook of this book is run twice by the book's checking tool, and the
second run must reproduce the first one exactly, byte for byte: the same printed
text, the same pictures, the same files. The Revision record does the same with
its long computations: its solvers of the Kohn-Sham equations were run a second
time and their result files compared byte for byte. This notebook shows, with
small experiments, what could make two runs differ and how the book prevents it.
It

- shows how the computer stores the number 0.1, and why 0.1 + 0.2 is not exactly
  0.3;
- draws the gaps between neighbouring numbers of the computer;
- adds the same numbers in two different orders and draws how the results differ
  in their last digits, which is why checks compare numbers with a tolerance;
- reads the tolerances of the Revision record's Kohn-Sham solver and the
  differences that were measured, and draws them;
- reads the record's negative control, which shows how two runs that round the
  same way hid an error, and draws it;
- shows how a seed makes random numbers repeat exactly;
- runs Python twelve times and shows that the order of a set of names changes from
  run to run, while the sorted order does not;
- shows that line ends change the bytes of a file, counts the result files of the
  Revision record and confirms that none has the line ends of Windows;
- computes the sha256 fingerprints of 582 result files, compares them with the
  fingerprints written down in the record, and reads the record's comparisons of
  two runs of each solver;
- draws six teaching plots and prints a PASS line for every check.
```

Text cell:

```
## 2. How to run this notebook
```

Step 1. What this notebook does and what it needs.

Notebook 00d (Why every run gives the same bytes: rounding, order, seeds and line ends) is the file ‘Revision/textbook/notebooks/00d_reproducible_runs.ipynb‘ of the repository Dirac_claude. It shows with small experiments why the notebooks of the textbook print the same numbers and write the same files every time they run: how the computer rounds decimal numbers, why the order of a sum changes its last digits (so that checks allow a tolerance), how a seed fixes random numbers, why names are sorted before they are printed, and why every file is written with the same line ends. It reproduces the tolerance, line-end, fingerprint and repeat checks of the Revision Kohn-Sham record (eight measured differences below their tolerances, a negative control that shows how one shared rounding path hid an error, 584 result files of two solvers without CR LF line ends, 582 recorded sha256 fingerprints equal to the files of today, two repeat runs byte for byte) and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 00d_reproducible_runs.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 15 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.

**Step 6. Or run the notebook without a browser (headless).**
```

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 00d_reproducible_runs.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 00d_reproducible_runs.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/00d.captions.json‘
- ‘Revision/textbook/figures/00d_1_float_spacing.png‘
- ‘Revision/textbook/figures/00d_2_summation_order.png‘
- ‘Revision/textbook/figures/00d_3_tolerances.png‘
- ‘Revision/textbook/figures/00d_4_shared_rounding.png‘
- ‘Revision/textbook/figures/00d_5_seeded_walks.png‘
- ‘Revision/textbook/figures/00d_6_set_orders.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of this notebook exist
ALL 22 CHECKS PASSED (notebook 00d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/00d_reproducible_runs.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 00d_reproducible_runs.ipynb
python -m nbconvert --execute --inplace 00d_reproducible_runs.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "AssertionError: check failed" for a check about the files of the folder Revision/kohn_sham (their number, their line ends or their fingerprints): a file of the Revision record was changed, added or converted after the repository was downloaded. The first command below, run in the repository folder, lists every changed or added file; the second restores a changed file (write the file name that the first command printed instead of FILE); delete a file that the first command lists as untracked.

```
git status
git restore FILE
```

- "CalledProcessError" in the section about the order of a set of names: the notebook could not start Python a second time. Close JupyterLab, activate the environment again, start JupyterLab again from the same terminal and run the notebook again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 00d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 00d (Why every run gives the same bytes: rounding, order, seeds and line ends)
# is the file Revision/textbook/notebooks/00d_reproducible_runs.ipynb of the repository
# Dirac_claude. It shows with small experiments why the notebooks of the textbook print
# the same numbers and write the same files every time they run: how the computer rounds
# decimal numbers, why the order of a sum changes its last digits (so that checks allow a
# tolerance), how a seed fixes random numbers, why names are sorted before they are
# printed, and why every file is written with the same line ends. It reproduces the
# tolerance, line-end, fingerprint and repeat checks of the Revision Kohn-Sham record
# (eight measured differences below their tolerances, a negative control that shows how
# one shared rounding path hid an error, 584 result files of two solvers without CR LF
# line ends, 582 recorded sha256 fingerprints equal to the files of today, two repeat
# runs byte for byte) and draws six teaching plots. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
```

```
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
```

```
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 00d_reproducible_runs.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 00d_reproducible_runs.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
```

```
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 00d_reproducible_runs.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/00d.captions.json
# - Revision/textbook/figures/00d_1_float_spacing.png
# - Revision/textbook/figures/00d_2_summation_order.png
# - Revision/textbook/figures/00d_3_tolerances.png
# - Revision/textbook/figures/00d_4_shared_rounding.png
# - Revision/textbook/figures/00d_5_seeded_walks.png
# - Revision/textbook/figures/00d_6_set_orders.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the six figure files of this notebook exist
#     ALL 22 CHECKS PASSED (notebook 00d)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/00d_reproducible_runs.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
```

```
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 00d_reproducible_runs.ipynb
#     python -m nbconvert --execute --inplace 00d_reproducible_runs.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "AssertionError: check failed" for a check about the files of the folder
#   Revision/kohn_sham (their number, their line ends or their fingerprints): a file of
#   the Revision record was changed, added or converted after the repository was
#   downloaded. The first command below, run in the repository folder, lists every
#   changed or added file; the second restores a changed file (write the file name that
#   the first command printed instead of FILE); delete a file that the first command
#   lists as untracked.
#     git status
#     git restore FILE
# - "CalledProcessError" in the section about the order of a set of names: the notebook
#   could not start Python a second time. Close JupyterLab, activate the environment
#   again, start JupyterLab again from the same terminal and run the notebook again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "00d"  # this notebook: chapter 00, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
```

```
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
```

```
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 00d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook
```

```
- **Bit, byte**: a bit is one binary digit, 0 or 1; a byte is 8 bits, a whole
  number from 0 to 255. Every file is a sequence of bytes.
- **Binary**: the way of writing numbers with the two digits 0 and 1, in powers of
  2 instead of powers of 10: binary 101 is $4 + 0 + 1 = 5$, and binary 0.11 is
  $1/2 + 1/4 = 3/4$.
- **Floating-point number**: the way the computer stores a number with a fraction
  part: a whole number of at most 53 binary digits times a power of 2. Python's
  `float` and numpy's `float64` are such numbers. They have about 16 significant
  decimal digits.
- **Rounding**: replacing a number by the nearest floating-point number. It happens
  after every addition, multiplication and division.
- **Gap, unit in the last place (ulp)**: the distance from a floating-point number
  to the next larger one. A difference of one ulp is the smallest possible change
  of a stored number.
- **Machine epsilon** $\epsilon$: the gap between 1 and the next larger
  floating-point number, $\epsilon = 2^{-52} \approx 2.22 \times 10^{-16}$.
- **Tolerance**: the largest difference between two numbers that a check accepts
  as agreement.
- **Random-number generator, seed**: a program that produces numbers that look
  random. It starts from a number called the *seed*; the same seed always gives the
  same sequence.
- **Random walk**: a path that takes a step of $+1$ or $-1$ at random, again and
  again.
- **Set**: an unordered collection of different things, written `{"x1", "x2"}` in
  Python. Python keeps the names of a set in an order decided by their *hash
  values*: numbers computed from the names with a secret key that Python chooses
  anew each time it starts, unless the *environment variable* PYTHONHASHSEED fixes
  the key.
- **Environment variable**: a named setting that a program receives from the
  computer when it starts.
- **Line end**: the bytes that end a line of a text file. Linux and macOS use one
  byte, LF (line feed, the number 10); Windows traditionally uses two, CR LF
  (carriage return 13, then line feed 10).
- **Fingerprint (sha256)**: 64 hexadecimal characters computed from the bytes of a
  file; a file that differs in a single byte gives a completely different
  fingerprint. **Hexadecimal**: the digits 0 to 9 and the letters a to f.
- **Manifest**: a file that lists other files with their fingerprints.
- **Byte-identical**: two files with exactly the same bytes.
- **Subprocess**: a program that another program starts and whose printed output it
  reads.
```

Text cell:

```
## 4. The physical and mathematical situation

A computer is deterministic: given the same program and the same input, it does the
same steps and gets the same result. Yet two runs of a careless notebook can print
different text, for five reasons:

1. it prints the time or the date;
2. it uses random numbers without a fixed seed;
3. it prints the names of a set, or of the files of a folder, in an order that can
   change from run to run;
```

```
4. it adds numbers in an order that depends on the computer (for example on the
   number of processor cores working together), and the order changes the last
   digits of a sum;
5. it writes files with the line ends of the operating system.

The notebooks of the book avoid all five: they print no times, use fixed seeds,
sort names before printing them, add in a fixed order, and write every file with
LF line ends. Where two *different* computations of the same quantity are compared,
for example two solvers or two step sizes, the results agree only up to rounding
and up to the errors of the methods; such a comparison uses a tolerance that is
fixed before the comparison is made.

Why does rounding exist at all? A decimal fraction such as $0.1 = 1/10$ has a power
of 10 below the fraction line. The computer stores binary fractions, which have a
power of 2 below the fraction line. Since $10 = 2 \cdot 5$, the factor 5 cannot be
removed, and $1/10$ is not a binary fraction: its binary expansion
$0.000110011001100\ldots$ never ends, just as $1/3 = 0.333\ldots$ never ends in
decimal. The computer keeps 53 significant binary digits and rounds the rest away;
it stores 0.1 as the nearest binary fraction,
$$0.1 \approx \frac{3602879701896397}{2^{55}} = 0.1000000000000000055511\ldots$$
Every further operation rounds its result again to 53 binary digits. Rounding is
deterministic (the same operation on the same numbers always gives the same
result), but it depends on the order of the operations: in general
$(a + b) + c$ and $a + (b + c)$ round differently.
```

Text cell:

```
## 5. How the computer stores 0.1

The next cell imports two modules of Python itself. `Fraction(x)` writes the
floating-point number $x$ as the exact fraction that the computer stores, and
`Decimal(x)` writes all its decimal digits. The cell prints both for 0.1, prints
$0.1 + 0.2$ with every digit that Python needs to identify the stored number
(`!r` inside the braces of an f-string asks for this), and checks two facts: 0.1 is
stored as $3602879701896397 / 2^{55}$, and $0.1 + 0.2$ differs from $0.3$ by
$2^{-54} \approx 5.55 \times 10^{-17}$.

The cell also defines the helper `check_reproduces`, used later for checks that
reproduce a Revision record: it first calls `sys.stdout.flush()`, which sends all
printed text that is still waiting, and then calls `check` with the record.
(Jupyter sends printed text in pieces; flushing first keeps the PASS line and the
line `reproduces ...` below it in one piece, so that the book's tools read them
together.)
```

In [2]:

```
import sys  # sys.stdout is the channel through which the notebook prints
from decimal import Decimal  # the exact decimal digits of a stored number
from fractions import Fraction  # a stored number as an exact fraction

import numpy as np  # arrays of numbers
```

```
def check_reproduces(condition, name, record):
    """check(condition, name, record=record), after sending the waiting output."""
    sys.stdout.flush()  # send every printed line that is still waiting
    check(condition, name, record=record)


stored = Fraction(0.1)  # the fraction that the computer really stores for 0.1
say(f"0.1 is stored as the fraction {stored}")
say(f"    = {Decimal(0.1)}")
say(f"0.1 + 0.2 = {0.1 + 0.2!r}")
say(f"0.1 + 0.2 == 0.3 is {0.1 + 0.2 == 0.3}")  # == asks: exactly equal?
difference = (0.1 + 0.2) - 0.3
report("(0.1 + 0.2) - 0.3", repr(difference))
check(stored == Fraction(3602879701896397, 2 ** 55),
      "0.1 is stored as 3602879701896397 / 2^55, not exactly as 1/10")
check(0.1 + 0.2 != 0.3 and difference == 2.0 ** -54,
      "0.1 + 0.2 differs from 0.3 by 2^-54 = 5.55e-17, one rounding step")
```

Out [2]:

```
0.1 is stored as the fraction 3602879701896397/36028797018963968
    = 0.1000000000000000055511151231257827021181583404541015625
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 is False
RESULT (0.1 + 0.2) - 0.3 = 5.551115123125783e-17
PASS 0.1 is stored as 3602879701896397 / 2^55, not exactly as 1/10
PASS 0.1 + 0.2 differs from 0.3 by 2^-54 = 5.55e-17, one rounding step
```

Text cell:

```
The next cell asks numpy for the machine epsilon $\epsilon$ of 64-bit
floating-point numbers, the gap between 1 and the next larger stored number. It
checks that $\epsilon = 2^{-52}$, that $1 + \epsilon$ is stored as a number larger
than 1, and that $1 + \epsilon/2$, which lies exactly halfway between 1 and
$1 + \epsilon$, is rounded back to 1. (A number exactly halfway is rounded to the
neighbour whose last binary digit is 0, which here is 1.)
```

In [3]:

```
epsilon = float(np.finfo(np.float64).eps)  # the gap between 1 and the next number
report("machine epsilon", repr(epsilon))
say(f"1 + epsilon     == 1 is {1.0 + epsilon == 1.0}")
say(f"1 + epsilon / 2 == 1 is {1.0 + epsilon / 2 == 1.0}")
check(epsilon == 2.0 ** -52 and 1.0 + epsilon > 1.0 and 1.0 + epsilon / 2 == 1.0,
      "machine epsilon is 2^-52: 1 + 2^-52 is stored, 1 + 2^-53 rounds back to 1")
```

Out [3]:

```
RESULT machine epsilon = 2.220446049250313e-16
1 + epsilon     == 1 is False
1 + epsilon / 2 == 1 is True
PASS machine epsilon is 2^-52: 1 + 2^-52 is stored, 1 + 2^-53 rounds back to 1
```

Text cell:

```
## 6. The gaps between neighbouring numbers

The gap between neighbouring floating-point numbers is not the same everywhere: it
grows with the size of the numbers. Between $2^{k}$ and $2^{k+1}$ all stored
numbers have the same gap $2^{k - 52}$, and the gap doubles at every power of 2. So
for every $x$ the gap lies between $\epsilon x / 2$ and $\epsilon x$: every stored
number is accurate to about 16 significant digits, whatever its size.

The next cell computes the gap with `np.spacing(x)` at 3000 values of $x$ from
$10^{-3}$ to $10^{6}$ (`np.logspace(-3, 6, 3000)` makes numbers equally spaced on
a logarithmic axis), draws it on logarithmic axes together with the two lines
$\epsilon x$ and $\epsilon x / 2$, prints the gaps at 1, 1000 and $10^{6}$, and
checks that the gap lies between the two lines at all 3000 points.
```

In [4]:

```
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
x_values = np.logspace(-3.0, 6.0, 3000)  # 3000 numbers from 0.001 to 1000000
gaps = np.spacing(x_values)  # the gap from each number to the next larger one
for x in (1.0, 1000.0, 1.0e6):
    say(f"gap above {x:>9g}: {np.spacing(x):.3e}")

fig, ax = plt.subplots()
# drawstyle="steps-post" draws each value as a flat step until the next x.
ax.plot(x_values, gaps, color=BLUE, linewidth=1.5, drawstyle="steps-post",
        label="gap to the next stored number")
ax.plot(x_values, epsilon * x_values, color="black", linestyle="--",
        linewidth=1.0, label=r"$\epsilon x$")
ax.plot(x_values, epsilon * x_values / 2, color="black", linestyle=":",
        linewidth=1.0, label=r"$\epsilon x / 2$")
ax.set_xscale("log")  # logarithmic horizontal axis
ax.set_yscale("log")  # logarithmic vertical axis
ax.set_xlabel(r"the number $x$")
ax.set_ylabel("gap to the next stored number")
ax.set_title("The gaps between neighbouring floating-point numbers")
ax.legend(loc="upper left")
save_figure(fig, "float_spacing",
            r"The gap between a floating-point number $x$ and the next larger "
            r"stored number (blue staircase) for $x$ from $10^{-3}$ to $10^{6}$, on "
            r"logarithmic axes (both pure numbers), with the lines $\epsilon x$ "
            r"(dashed) and $\epsilon x/2$ (dotted), $\epsilon = 2^{-52}$. The gap is "
            r"constant between two powers of 2 and doubles at each of them, so it "
            r"always lies between the two lines: every stored number carries about "
            r"16 significant digits, whatever its size.")
ratios = gaps / x_values  # the gap relative to the number
```

```
check(bool(np.all(ratios > epsilon / 2) and np.all(ratios <= epsilon))
      and np.spacing(1.0) == epsilon,
      "the gap lies between eps x / 2 and eps x at all 3000 points")
```

Out [4]:

```
gap above         1: 2.220e-16
gap above      1000: 1.137e-13
gap above     1e+06: 1.164e-10
```

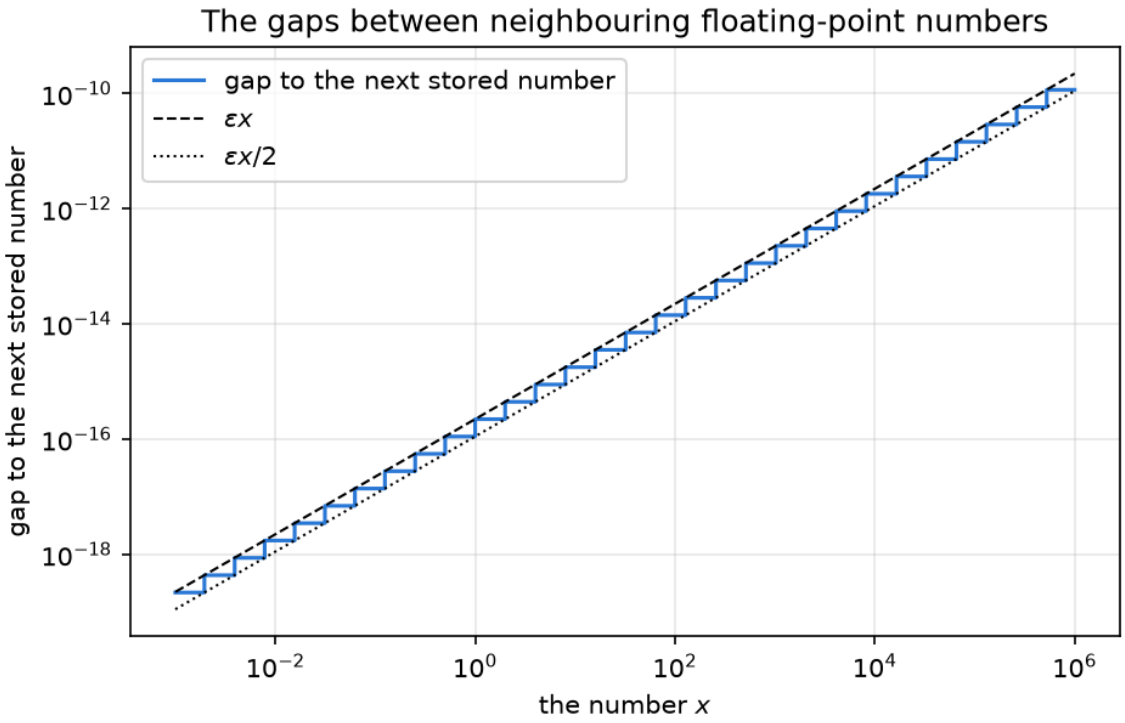

Figure 11: The gap between a floating-point number x and the next larger stored number (blue staircase) for x from 10^{-3} to 10^{6}, on logarithmic axes (both pure numbers), with the lines ϵx (dashed) and $\epsilon x/2$ (dotted), $\epsilon = 2^{-52}$. The gap is constant between two powers of 2 and doubles at each of them, so it always lies between the two lines: every stored number carries about 16 significant digits, whatever its size. (Notebook 00d, figure 1.)

Out [4]:

```
Figure 00d.1 saved as Revision/textbook/figures/00d_1_float_spacing.png
PASS the gap lies between eps x / 2 and eps x at all 3000 points
```

Text cell:

```
## 7. The order of a sum changes the last digits
```

```
We add the numbers $1, 1/4, 1/9, \ldots, 1/N^2$, that is $1/k^2$ for
$k = 1, \ldots, N$, in two orders:

- **forwards**: starting with the largest number 1. The running total is about
  1.6 from the start, so every small number is added to a large total, and each
  addition rounds to the gap of that total;
- **backwards**: starting with the smallest number $1/N^2$. The running total grows
  slowly, and small numbers are added to small totals, which loses less.

The reference is `math.fsum`, which adds the stored numbers exactly and rounds only
once at the end (the *exactly rounded* sum). The difference of each sum from it is
measured in ulps: the difference divided by the gap `np.spacing` of the exact sum.
`np.add.accumulate(terms)` adds the numbers of an array one after the other,
starting with the first, and returns every running total, so `forward[N - 1]` is
the forward sum of the first $N$ numbers. `terms[:N][::-1]` is the first $N$
numbers in reverse order. A third way, numpy's `np.sum`, adds in pairs, then pairs
of pairs, and so on (*pairwise summation*), which also loses little.

The next cell does this for 31 values of $N$ from 10 to $10^{6}$, prints the four
sums for $N = 10^{6}$ with all their digits, and prints how far the sum of $10^{6}$
numbers lies below the limit of the infinite sum, $\pi^2/6$. That last difference,
about $1/N = 10^{-6}$, is not a rounding error: it is the error of the *method*
(stopping after $N$ numbers), and it is more than $10^{7}$ times larger than the
largest rounding difference.
```

In [5]:

```
import math  # math.fsum: the exactly rounded sum; math.pi

N_MAX = 10 ** 6
k = np.arange(1, N_MAX + 1, dtype=np.float64)  # 1, 2, ..., 1000000 as floats
terms = 1.0 / (k * k)  # the numbers 1/k^2
forward = np.add.accumulate(terms)  # all forward running totals
# 31 values of N, equally spaced on a logarithmic axis from 10 to 10^6.
n_values = np.unique(np.round(np.logspace(1.0, 6.0, 31)).astype(int))
ulps = {"forward": [], "backward": [], "pairwise": []}  # the errors in ulps
for n in n_values:
    exact = math.fsum(terms[:n])  # the exactly rounded sum of the first n numbers
    gap = np.spacing(exact)  # one ulp of the exact sum
    backward = np.add.accumulate(terms[:n][::-1])[-1]  # smallest number first
    ulps["forward"].append(float((forward[n - 1] - exact) / gap))
    ulps["backward"].append(float((backward - exact) / gap))
    ulps["pairwise"].append(float((np.sum(terms[:n]) - exact) / gap))

# The three sums of all 10^6 numbers, as plain Python floats (float(...)).
forward_million = float(forward[-1])
backward_million = float(np.add.accumulate(terms[::-1])[-1])
pairwise_million = float(np.sum(terms))
exact_million = math.fsum(terms)
say(f"forward  sum of 10^6 numbers: {forward_million!r}")
say(f"backward sum of 10^6 numbers: {backward_million!r}")
say(f"pairwise sum of 10^6 numbers: {pairwise_million!r}")
say(f"exactly rounded sum         : {exact_million!r}")
```

```
worst_forward = max(abs(u) for u in ulps["forward"])  # the largest size of error
worst_backward = max(abs(u) for u in ulps["backward"])
worst_pairwise = max(abs(u) for u in ulps["pairwise"])
report("largest forward error in ulps", f"{worst_forward:.0f}")
report("largest backward error in ulps", f"{worst_backward:.0f}")
report("largest pairwise error in ulps", f"{worst_pairwise:.0f}")
report("pi^2/6 minus the exact sum of 10^6 numbers",
       f"{math.pi ** 2 / 6 - exact_million:.6e}")
```

Out [5]:

```
forward  sum of 10^6 numbers: 1.64493306684877
backward sum of 10^6 numbers: 1.6449330668487263
pairwise sum of 10^6 numbers: 1.6449330668487263
exactly rounded sum         : 1.6449330668487265
RESULT largest forward error in ulps = 196
RESULT largest backward error in ulps = 1
RESULT largest pairwise error in ulps = 3
RESULT pi^2/6 minus the exact sum of 10^6 numbers = 9.999995e-07
```

Text cell:

```
The next cell draws the three errors in ulps against $N$ and checks three
statements: the forward and the backward sum of the $10^{6}$ numbers are different
stored numbers (the order matters); the backward sum is never more than one ulp
from the exactly rounded sum; and all three ways agree with the exactly rounded sum
to a relative difference below $10^{-13}$, so a check with the tolerance
$10^{-12}$ accepts each of them.
```

In [6]:

```
fig, ax = plt.subplots()
ax.plot(n_values, ulps["forward"], "o-", color=BLUE, markersize=4,
        label="forwards (largest number first)")
ax.plot(n_values, ulps["backward"], "s-", color=ORANGE, markersize=4,
        label="backwards (smallest number first)")
ax.plot(n_values, ulps["pairwise"], "^:", color=AQUA, markersize=4,
        label="pairwise (numpy's np.sum)")
ax.axhline(0.0, color="black", linewidth=0.8)  # the exactly rounded sum
ax.set_xscale("log")
ax.set_xlabel(r"number of terms $N$ of the sum $1 + 1/4 + \cdots + 1/N^2$")
ax.set_ylabel("error in units of the last place (ulps)")
ax.set_title("The same numbers added in different orders")
ax.legend(loc="upper left")
save_figure(fig, "summation_order",
            r"The rounding error of the sum $1 + 1/4 + 1/9 + \cdots + 1/N^2$ in "
            r"units of the last place (vertical axis, ulps, the gap between "
            r"neighbouring stored numbers at the sum) against the number of terms "
            r"$N$ from 10 to $10^{6}$ (horizontal axis, logarithmic), measured from "
            r"the exactly rounded sum: forwards with the largest term first (blue "
            r"circles), backwards with the smallest term first (orange squares), "
```

```
            r"and pairwise as numpy adds (aqua triangles). The forward error grows "
            f"to {worst_forward:.0f} ulps, a relative error of a few times "
            r"$10^{-14}$; the backward error stays within "
            f"{worst_backward:.0f} ulp and the pairwise error within "
            f"{worst_pairwise:.0f} ulps. Same numbers, different order, different "
            r"last digits.")
relative = max(abs(s - exact_million) for s in
               (forward_million, backward_million, pairwise_million)) / exact_million
report("largest relative difference of the three sums of 10^6 numbers",
       f"{relative:.2e}")
check(forward_million != backward_million,
      "forwards and backwards, the sums of the 10^6 numbers differ in the last digits")
check(worst_backward <= 1.0,
      "the backward sum is within one ulp of the exactly rounded sum for every N")
check(relative < 1e-13,
      "the three sums agree to a relative 1e-13: a tolerance of 1e-12 accepts all")
```

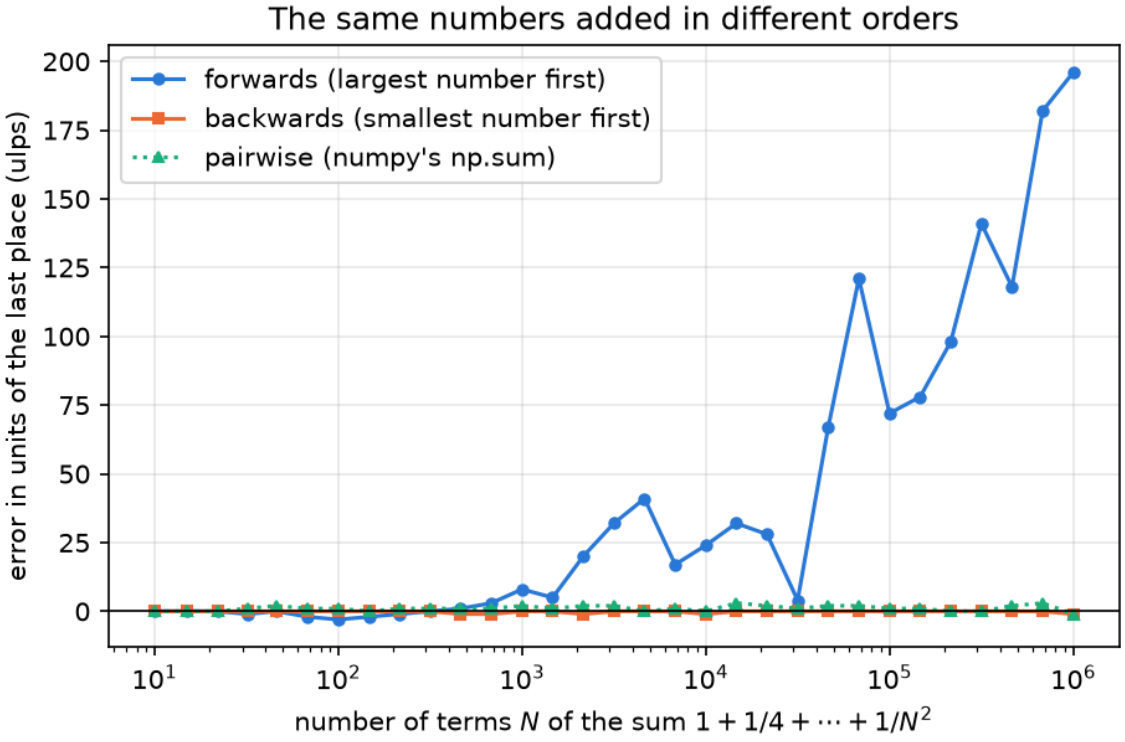

Figure 12: The rounding error of the sum $1 + 1/4 + 1/9 + \cdots + 1/N^2$ in units of the last place (vertical axis, ulps, the gap between neighbouring stored numbers at the sum) against the number of terms N from 10 to 10^6 (horizontal axis, logarithmic), measured from the exactly rounded sum: forwards with the largest term first (blue circles), backwards with the smallest term first (orange squares), and pairwise as numpy adds (aqua triangles). The forward error grows to 196 ulps, a relative error of a few times 10^{-14}; the backward error stays within 1 ulp and the pairwise error within 3 ulps. Same numbers, different order, different last digits. (Notebook 00d, figure 2.)

Out [6]:

```
Figure 00d.2 saved as Revision/textbook/figures/00d_2_summation_order.png
RESULT largest relative difference of the three sums of 10^6 numbers = 2.65e-14
PASS forwards and backwards, the sums of the 10^6 numbers differ in the last digits
PASS the backward sum is within one ulp of the exactly rounded sum for every N
PASS the three sums agree to a relative 1e-13: a tolerance of 1e-12 accepts all
```

Text cell:

```
## 8. Why checks allow a tolerance: the Revision record

The Revision record's Kohn-Sham solver, a Rust program, was run with its canonical
settings and again with *refined* settings: twice as many steps of its
Runge-Kutta integrator, root and self-consistency tolerances ten times smaller, and
one equation (the condition that fixes the chemical potential $\mu$) solved in a
different but exactly equivalent form, which rounds along a different path. The
differences between the two runs measure the error of the canonical run. Before the
comparison, tolerances were fixed: $10^{-8}$ for energies, levels and thermodynamic
quantities, $10^{-6}$ for profiles and derivatives.

The report Revision/kohn_sham/reports/ks-rust-determinism.json records eight such
comparisons, each with its tolerance at the end of its detail text, and the
cross-check report Revision/kohn_sham/reports/ks-crosscheck.json quotes the eight
measured largest differences as numbers under the key
`rust_matrix_wide_uncertainties`. The next cell reads both, finds each measured
number (written with four significant digits) in the detail of its check, reads the
tolerance with the pattern `tolerance (\S+)` of the module `re` (`\S+` means one or
more characters that are not blanks; the brackets mark the part to return), and
prints a table with the *margin*, the tolerance divided by the measured
difference. The names of the eight quantities are explained in the Kohn-Sham
chapters; here only their sizes matter.
```

In [7]:

```
import re  # finds patterns in texts ("regular expressions")


def read_json(path):
    """The content of the JSON file path of the repository (dictionaries, lists)."""
    return json.loads(repository_file(path).read_text(encoding="utf-8"))


DETERMINISM = "Revision/kohn_sham/reports/ks-rust-determinism.json"
CROSS = "Revision/kohn_sham/reports/ks-crosscheck.json"
determinism_checks = {entry["name"]: entry for entry in read_json(DETERMINISM)["checks"]}
measured = read_json(CROSS)["rust_matrix_wide_uncertainties"]  # name -> difference
SHORT_NAMES = {  # the check names of the record -> short names for the table
    "refined_ground_energies": "ground-state energies",
    "refined_ground_homo_lumo_gap": "level gaps",
    "refined_eigenvalues": "all Kohn-Sham levels",
    "refined_delta_scf": "excitation energies",
    "refined_profiles": "profiles",
    "refined_adiabatic_derivatives": "adiabatic derivatives",
```

```
    "refined_thermodynamics": "thermodynamics",
    "refined_heat_capacity": "heat capacity",
}
tolerances, quoted, verdicts = {}, {}, {}
say("quantity                measured   tolerance   margin")
for name, short in SHORT_NAMES.items():
    detail = determinism_checks[name]["detail"]
    verdicts[name] = determinism_checks[name]["verdict"]
    tolerances[name] = float(re.findall(r"tolerance (\S+)", detail)[-1])
    quoted[name] = f"{measured[name]:.3e}" in detail  # the same number in both files
    margin = tolerances[name] / measured[name]
    say(f"{short:22} {measured[name]:9.3e}   {tolerances[name]:9.0e}   {margin:7.1f}")
smallest = min(tolerances[n] / measured[n] for n in SHORT_NAMES)
report("smallest margin (tolerance / measured difference)", f"{smallest:.1f}")
check_reproduces(
    sorted(measured) == sorted(SHORT_NAMES) and all(quoted.values())
    and all(verdicts[n] == "PASS" and measured[n] < tolerances[n] for n in SHORT_NAMES),
    "each of the eight measured differences of the record is below its tolerance",
    f"{DETERMINISM}, checks " + ", ".join(SHORT_NAMES))
```

Out [7]:

```
quantity                measured   tolerance   margin
ground-state energies  1.901e-12       1e-08    5260.4
level gaps             1.300e-12       1e-08    7692.3
all Kohn-Sham levels   2.135e-09       1e-08       4.7
excitation energies    5.815e-11       1e-08     172.0
profiles               2.800e-09       1e-06     357.1
adiabatic derivatives  9.024e-10       1e-06    1108.2
thermodynamics         2.489e-10       1e-08      40.2
heat capacity          5.689e-09       1e-06     175.8
RESULT smallest margin (tolerance / measured difference) = 4.7
PASS each of the eight measured differences of the record is below its tolerance
     reproduces Revision/kohn_sham/reports/ks-rust-determinism.json, checks
    refined_ground_energies, refined_ground_homo_lumo_gap, refined_eigenvalues,
    refined_delta_scf, refined_profiles, refined_adiabatic_derivatives,
    refined_thermodynamics, refined_heat_capacity
```

Text cell:

```
The next cell draws the table: for each of the eight quantities a dot at the
measured difference and a vertical bar at its tolerance, joined by a grey line
whose length on the logarithmic axis is the margin.
```

In [8]:

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
rows = np.arange(len(SHORT_NAMES))[::-1]  # the first quantity at the top
for row, name in zip(rows, SHORT_NAMES):
    ax.plot([measured[name], tolerances[name]], [row, row], color="#b5b3ad",
            linewidth=2.5)  # the margin
```

```
ax.plot([measured[n] for n in SHORT_NAMES], rows, "o", color=BLUE, markersize=7,
        label="largest measured difference")
ax.plot([tolerances[n] for n in SHORT_NAMES], rows, "|", color="black",
        markersize=16, markeredgewidth=2.0, label="tolerance fixed in advance")
ax.set_xscale("log")
ax.set_xlim(1e-13, 1e-4)
ax.set_yticks(rows, labels=list(SHORT_NAMES.values()))
ax.grid(False, axis="y")
ax.set_xlabel("canonical minus refined run (relative, or in units of the mass m)")
ax.set_title("Kohn-Sham solver: measured differences and tolerances")
# The legend below the picture, in two columns, so that it covers no row.
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=2)
margins = [tolerances[n] / measured[n] for n in SHORT_NAMES]  # tolerance / measured
save_figure(fig, "tolerances",
            r"The eight comparisons between the canonical and the refined run of "
            r"the Revision record's Kohn-Sham solver (one row each): the largest "
            r"measured difference (blue dot) and the tolerance fixed before the "
            r"comparison (black bar), on a logarithmic horizontal axis (relative "
            r"differences, or level differences in units of the mass $m$). Every "
            r"dot lies to the left of its bar: the measured differences are "
            f"between {min(margins):.1f} and {max(margins):.0f} times smaller "
            r"than their tolerances. The grey line from a dot to its bar is this "
            r"margin.")
```

Out [8]:

```
Figure 00d.3 saved as Revision/textbook/figures/00d_3_tolerances.png
```

Text cell:

The record also teaches why the refined run had to round along a different path. An earlier version of the solver found μ with a third method, a *direct count*, and used it in BOTH runs. In three thermal states that method was wrong by up to about $8 \times 10^{-10}\,m$, but both runs made the same rounding error, so the difference between the runs (the *former measure*) was tiny and hid the error. The present refined run finds μ along another rounding path, and its difference from the old result (the *present measure*) shows the error in full. The check ‘refined_mermin_root_path‘ of the same report records this as a *negative control* (a test that must detect a known error) in its detail text, one entry per state.

The next cell reads these entries with a regular expression: ‘re.findall‘ returns, for every place where the pattern matches, the parts in brackets (the name of the state and three numbers). It prints them as a table and checks, as the record states, that in every state the former measure stays below half of the error while the present measure is at least half of it. Two computations that make the same rounding error agree with each other and can still both be wrong: a comparison is only as good as the independence of the two computations.

In [9]:

```
root_path = determinism_checks["refined_mermin_root_path"]  # the record's check
CONTROL = (r"(N\w+): direct-count error (\S+) m \(refined run \S+\), "
```

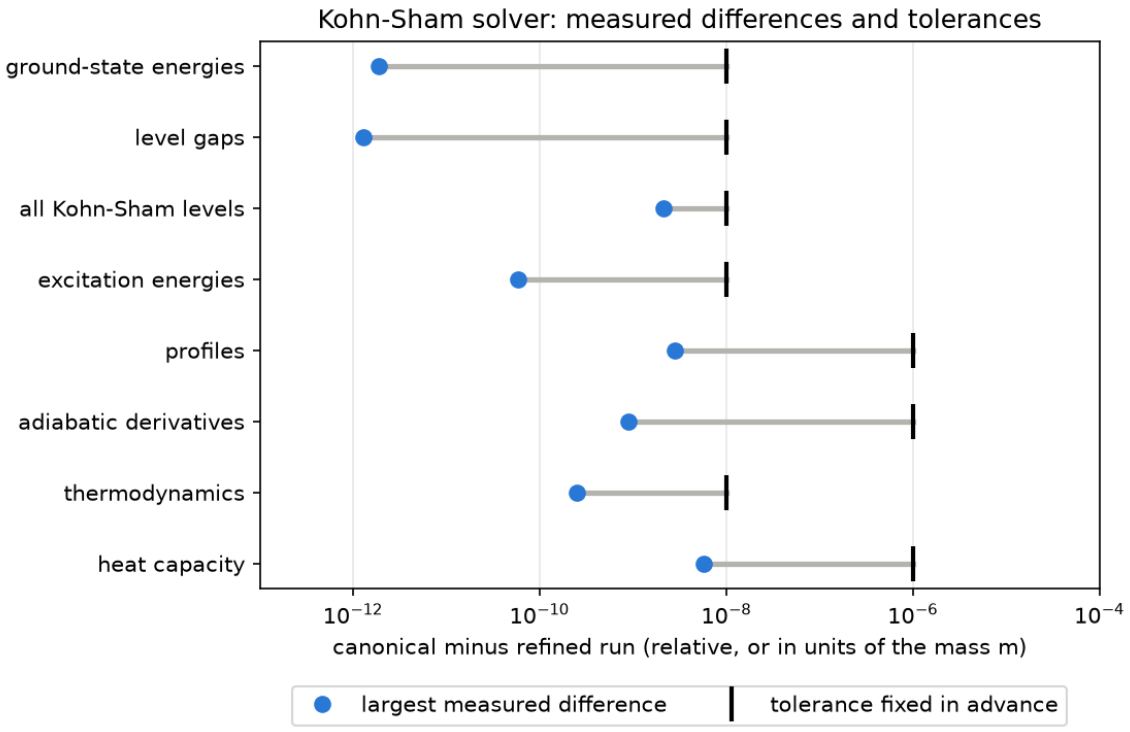

Figure 13: The eight comparisons between the canonical and the refined run of the Revision record's Kohn-Sham solver (one row each): the largest measured difference (blue dot) and the tolerance fixed before the comparison (black bar), on a logarithmic horizontal axis (relative differences, or level differences in units of the mass m). Every dot lies to the left of its bar: the measured differences are between 4.7 and 7692 times smaller than their tolerances. The grey line from a dot to its bar is this margin. (Notebook 00d, figure 3.)

```
          r"former measure (\S+), present measure against it (\S+),")
control = []  # (state, error of the old root, former measure, present measure)
for state, error, former, present in re.findall(CONTROL, root_path["detail"]):
    control.append((state, abs(float(error)), float(former), float(present)))
say("state              old method error   former measure   present measure")
for state, error, former, present in control:
    say(f"{state:17} {error:12.3e} m   {former:14.3e}   {present:15.3e}")
largest_hidden = max(error for _, error, _, _ in control)
report("largest error that one shared rounding path hid", f"{largest_hidden:.3e}", "m")
check_reproduces(
    root_path["verdict"] == "PASS" and len(control) == 3
    and all(former < error / 2 <= present for _, error, former, present in control),
    f"one shared rounding path hid errors up to {largest_hidden:.1e} m; "
    "two paths show them",
    f"{DETERMINISM}, check refined_mermin_root_path")
```

Out [9]:

```
state              old method error   former measure   present measure
N8_lamm1_a00_T10     8.267e-10 m        5.829e-16         8.271e-10
N8_lam0_a00_T10      2.722e-10 m        7.528e-13         2.719e-10
N8_lamp1_a00_T10     6.959e-11 m        7.518e-13         6.921e-11
RESULT largest error that one shared rounding path hid = 8.267e-10 m
PASS one shared rounding path hid errors up to 8.3e-10 m; two paths show them
     reproduces Revision/kohn_sham/reports/ks-rust-determinism.json, check
    refined_mermin_root_path
```

Text cell:

```
The next cell draws the three states of the negative control: for each state, the
error of the old method, the difference that the former comparison saw, and the
difference that the present comparison sees, as bars on a logarithmic axis.
```

In [10]:

```
fig, ax = plt.subplots(figsize=(7.0, 4.0))
rows = np.arange(len(control))[::-1]  # the first state at the top
height = 0.26  # three bars in each row
errors = [error for _, error, _, _ in control]
formers = [former for _, _, former, _ in control]
presents = [present for _, _, _, present in control]
# How many times smaller the former measure is than the error, state by state.
hidden_ratios = [error / former for error, former in zip(errors, formers)]
ax.barh(rows + height, errors, height, color=ORANGE,
        label="error of the old method")
ax.barh(rows, formers, height, color="#b5b3ad",
        label="former comparison (one rounding path in both runs)")
ax.barh(rows - height, presents, height, color=BLUE,
        label="present comparison (two rounding paths)")
ax.set_xscale("log")
ax.set_xlim(1e-17, 1e-8)
```

```
ax.set_yticks(rows, labels=[state for state, _, _, _ in control])
ax.grid(False, axis="y")
ax.set_xlabel(r"size of the difference in $\mu$ (units of the mass m)")
ax.set_title("A comparison that rounds the same way twice sees nothing")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=1)
save_figure(fig, "shared_rounding",
            r"The negative control of the Revision record's Kohn-Sham solver for "
            r"the three thermal states named on the vertical axis: the error of the "
            r"old method for the chemical potential $\mu$ (orange), the difference "
            r"between the two runs when both used that method, and so shared its "
            r"rounding (grey), and the difference seen by the present refined run, "
            r"which rounds along another path (blue); horizontal axis the size of "
            r"the difference in units of the mass $m$, logarithmic. The grey bars "
            f"are between {min(hidden_ratios):.0f} and "
            f"{max(hidden_ratios) / 1e6:.1f} million "
            r"times shorter than the orange ones: two runs with the same rounding "
            f"hid errors up to {largest_hidden:.1e} "
            r"$m$, which the blue bars show in full.")
```

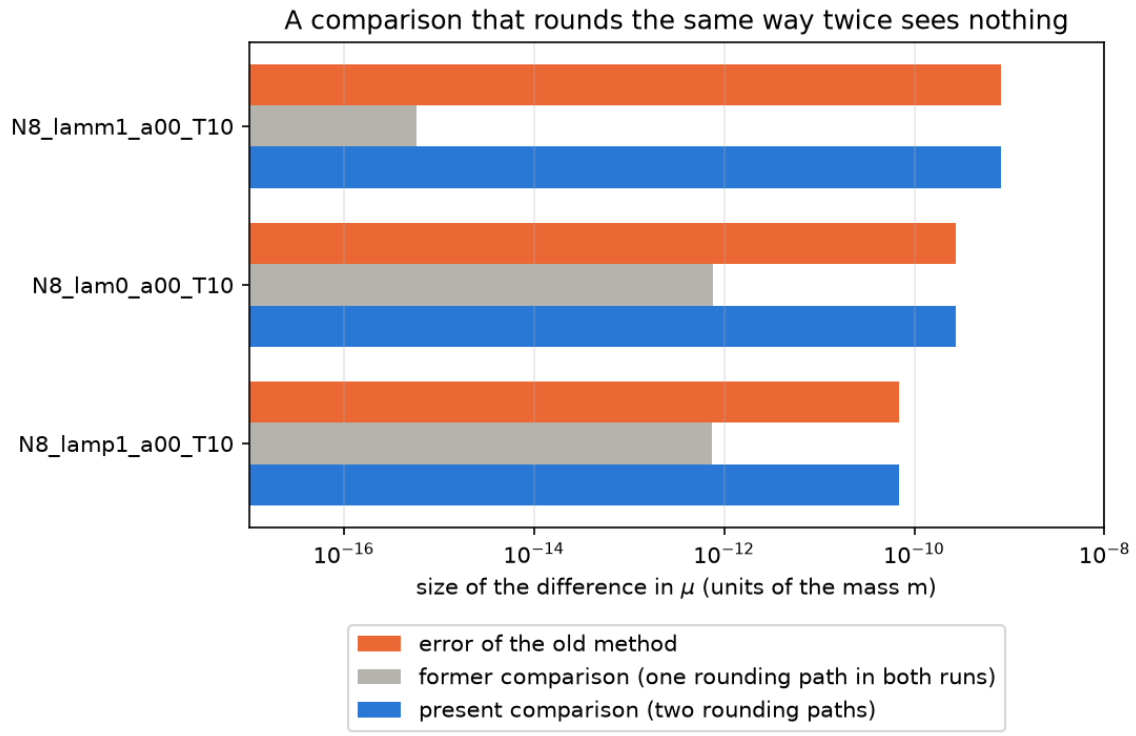

Figure 14: The negative control of the Revision record's Kohn-Sham solver for the three thermal states named on the vertical axis: the error of the old method for the chemical potential μ (orange), the difference between the two runs when both used that method, and so shared its rounding (grey), and the difference seen by the present refined run, which rounds along another path (blue); horizontal axis the size of the difference in units of the mass m, logarithmic. The grey bars are between 93 and 1.4 million times shorter than the orange ones: two runs with the same rounding hid errors up to 8.3e-10 m, which the blue bars show in full. (Notebook 00d, figure 4.)

Out [10]:

```
Figure 00d.4 saved as Revision/textbook/figures/00d_4_shared_rounding.png
```

Text cell:

```
## 9. Random numbers that repeat: seeds

‘np.random.default_rng(seed)‘ makes a random-number generator that starts from the
given seed. The next cell makes three generators, two with the seed 12345 and one
with the seed 2026, lets each produce 400 random steps of $+1$ or $-1$
(‘choice([-1, 1], size=400)‘ picks 400 times from the list $[-1, 1]$), and adds the
steps up into random walks with ‘np.cumsum‘ (the running totals). It prints the
first ten steps of each, draws the three walks, and checks that the two walks with
the same seed are identical while the third differs.
```

In [11]:

```
walks = {}  # a label -> the 401 positions of the walk (it starts at 0)
for label, seed in (("seed 12345, first run", 12345),
                    ("seed 12345, second run", 12345), ("seed 2026", 2026)):
    generator = np.random.default_rng(seed)  # a new generator from this seed
    steps = generator.choice([-1, 1], size=400)  # 400 random steps of +1 or -1
    walks[label] = np.concatenate([[0], np.cumsum(steps)])  # positions 0 ... 400
    first = " ".join(f"{s:+d}" for s in steps[:10])  # e.g. "+1 -1 +1 ..."
    say(f"{label:23} first steps: {first}")

fig, ax = plt.subplots()
step_numbers = np.arange(401)
ax.plot(step_numbers, walks["seed 12345, first run"], color=BLUE, linewidth=3.0,
        label="seed 12345, first run")
ax.plot(step_numbers, walks["seed 12345, second run"], color=YELLOW, linewidth=1.2,
        linestyle="--", label="seed 12345, second run (lies on the first)")
ax.plot(step_numbers, walks["seed 2026"], color=ORANGE, linewidth=1.5,
        label="seed 2026")
ax.set_xlabel("step number")
ax.set_ylabel("position (sum of the steps)")
ax.set_title("Random walks: the same seed gives the same walk")
# The legend below the picture, so that it covers no part of the walks.
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=1)
save_figure(fig, "seeded_walks",
            r"Three random walks of 400 steps of $+1$ or $-1$ (horizontal axis the "
            r"step number, vertical axis the position, the sum of the steps so far; "
            r"pure numbers). The thick blue walk and the dashed yellow walk come from "
            r"two separate generators started with the same seed 12345: they are "
            r"identical, step for step, so the yellow line lies on the blue one. "
            r"The orange walk, from the seed 2026, is different. A fixed seed makes "
            r"random numbers repeat exactly in every run.")
check(np.array_equal(walks["seed 12345, first run"], walks["seed 12345, second run"]),
      "two generators with the same seed give the same 400 steps")
check(not np.array_equal(walks["seed 12345, first run"], walks["seed 2026"]),
      "a generator with another seed gives other steps")
```

Out [11]:

```
seed 12345, first run   first steps: +1 -1 +1 -1 -1 +1 +1 +1 +1 -1
seed 12345, second run  first steps: +1 -1 +1 -1 -1 +1 +1 +1 +1 -1
seed 2026               first steps: +1 -1 -1 +1 -1 -1 -1 -1 +1 -1
```

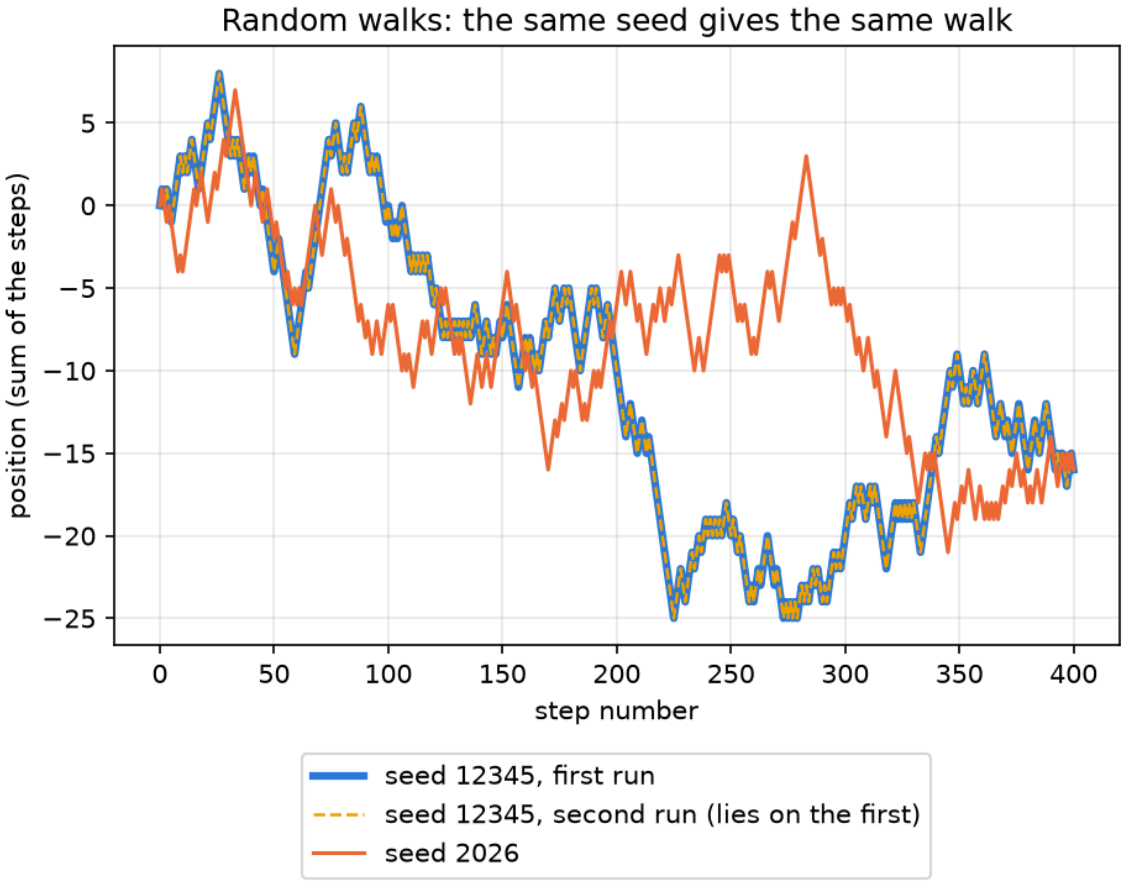

Figure 15: Three random walks of 400 steps of +1 or −1 (horizontal axis the step number, vertical axis the position, the sum of the steps so far; pure numbers). The thick blue walk and the dashed yellow walk come from two separate generators started with the same seed 12345: they are identical, step for step, so the yellow line lies on the blue one. The orange walk, from the seed 2026, is different. A fixed seed makes random numbers repeat exactly in every run. (Notebook 00d, figure 5.)

Out [11]:

```
Figure 00d.5 saved as Revision/textbook/figures/00d_5_seeded_walks.png
PASS two generators with the same seed give the same 400 steps
PASS a generator with another seed gives other steps
```

Text cell:

```
## 10. The order of a set of names

The next cell starts Python twelve times as a subprocess, each time with another
```

```
value 0, 1, ..., 11 of the environment variable PYTHONHASHSEED, and lets it print
the eight names $x_0, \ldots, x_7$ of a set. ‘sys.executable‘ is the Python program
that runs this notebook; ‘-c‘ tells it to run the program text that follows;
the names are given to that program as its arguments ‘sys.argv[1:]‘, and
‘print(*names)‘ prints them separated by blanks. ‘dict(os.environ,
PYTHONHASHSEED=...)‘ is a copy of the environment variables with one of them set.
The cell runs the seed 0 a second time, collects the orders, and checks three
statements: the same seed gives the same order; the twelve seeds give different
orders; and sorted with ‘sorted‘, every order becomes $x_0, \ldots, x_7$.

This is why no notebook of the book prints a set without sorting it first: the
checking tool runs every notebook a second time with another hash seed, and an
unsorted set would print its names in another order.
```

In [12]:

```
import subprocess  # starts another program and reads what it prints

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
PROGRAM = "import sys; print(*set(sys.argv[1:]))"  # print the names of a set


def set_order(seed):
    """The order in which a new Python with PYTHONHASHSEED = seed prints the set."""
    environment = dict(os.environ, PYTHONHASHSEED=str(seed))
    completed = subprocess.run([sys.executable, "-c", PROGRAM, *NAMES],
                               capture_output=True, text=True, env=environment,
                               check=True)  # check=True: stop if Python failed
    return completed.stdout.split()  # the printed names, in their printed order


orders = [set_order(seed) for seed in range(12)]  # the seeds 0, 1, ..., 11
for seed, order in enumerate(orders):
    printed = " ".join(order)  # the eight names separated by blanks
    say(f"PYTHONHASHSEED={seed:<2}  {printed}")
in_order = " ".join(sorted(orders[0]))  # sorted: x0 x1 ... x7
say(f"sorted              {in_order}")
different = len({tuple(order) for order in orders})  # the number of distinct orders
report("distinct orders among the 12 runs", different)
check(set_order(0) == orders[0], "the same hash seed gives the same order")
check(different >= 2, "different hash seeds give different orders of the same set")
check(all(sorted(order) == NAMES for order in orders),
      "sorted, the names come in the same order x0 ... x7 in every run")
```

Out [12]:

```
PYTHONHASHSEED=0   x0 x4 x6 x1 x3 x7 x2 x5
PYTHONHASHSEED=1   x7 x5 x1 x3 x2 x4 x0 x6
PYTHONHASHSEED=2   x1 x0 x7 x2 x5 x6 x4 x3
PYTHONHASHSEED=3   x5 x4 x2 x0 x3 x6 x7 x1
PYTHONHASHSEED=4   x0 x5 x6 x7 x3 x4 x1 x2
PYTHONHASHSEED=5   x1 x2 x3 x5 x7 x4 x6 x0
```

```
PYTHONHASHSEED=6   x0 x2 x7 x4 x3 x5 x1 x6
PYTHONHASHSEED=7   x1 x6 x2 x7 x0 x5 x4 x3
PYTHONHASHSEED=8   x6 x5 x3 x4 x0 x7 x1 x2
PYTHONHASHSEED=9   x3 x0 x6 x7 x1 x4 x5 x2
PYTHONHASHSEED=10  x2 x3 x5 x1 x7 x6 x4 x0
PYTHONHASHSEED=11  x5 x4 x6 x1 x3 x2 x0 x7
sorted              x0 x1 x2 x3 x4 x5 x6 x7
RESULT distinct orders among the 12 runs = 12
PASS the same hash seed gives the same order
PASS different hash seeds give different orders of the same set
PASS sorted, the names come in the same order x0 ... x7 in every run
```

Text cell:

```
The next cell draws the twelve orders as a picture: one row per run, one column
per place in the printed order, each square coloured by the name that stands
there; the last row is the sorted order.
```

In [13]:

```
table = orders + [sorted(orders[0])]  # 13 rows of 8 names: 12 runs and the sorted
# Each name replaced by its number in NAMES: x0 -> 0, x1 -> 1, ..., x7 -> 7.
numbers = np.array([[NAMES.index(name) for name in row] for row in table])
colours = matplotlib.colormaps["viridis"].resampled(8)  # 8 colours, violet to yellow
fig, ax = plt.subplots(figsize=(7.0, 6.0))
# vmin and vmax put each of the numbers 0 ... 7 in the middle of its own colour.
ax.imshow(numbers, cmap=colours, vmin=-0.5, vmax=7.5, aspect="auto")
for row in range(numbers.shape[0]):
    for column in range(8):
        # white letters on the dark colours, black ones on the light colours
        ink = "white" if numbers[row, column] < 5 else "black"
        ax.text(column, row, table[row][column], ha="center", va="center",
                color=ink, fontsize=9)
ax.axhline(11.5, color="white", linewidth=4.0)  # a gap before the sorted row
ax.set_xticks(range(8), labels=[str(place) for place in range(1, 9)])
ax.set_yticks(range(13),
              labels=[f"PYTHONHASHSEED={seed}" for seed in range(12)] + ["sorted"])
ax.grid(False)
ax.set_xlabel("place in the printed order")
ax.set_title("The same set of eight names, printed by twelve runs of Python")
save_figure(fig, "set_orders",
            r"The order in which twelve separate runs of Python print the same set "
            r"of the eight names $x_0, \ldots, x_7$ (one row per run, labelled by "
            r"its hash seed PYTHONHASHSEED from 0 to 11; horizontal axis the place 1 "
            r"to 8 in the printed order; each square coloured by its name, from "
            r"dark violet for $x_0$ to yellow for $x_7$). Every run prints another "
            r"order, so the rows are scrambled differently; the bottom row, the "
            r"sorted order, is the same in every run.")
```

Out [13]:

```
Figure 00d.6 saved as Revision/textbook/figures/00d_6_set_orders.png
```

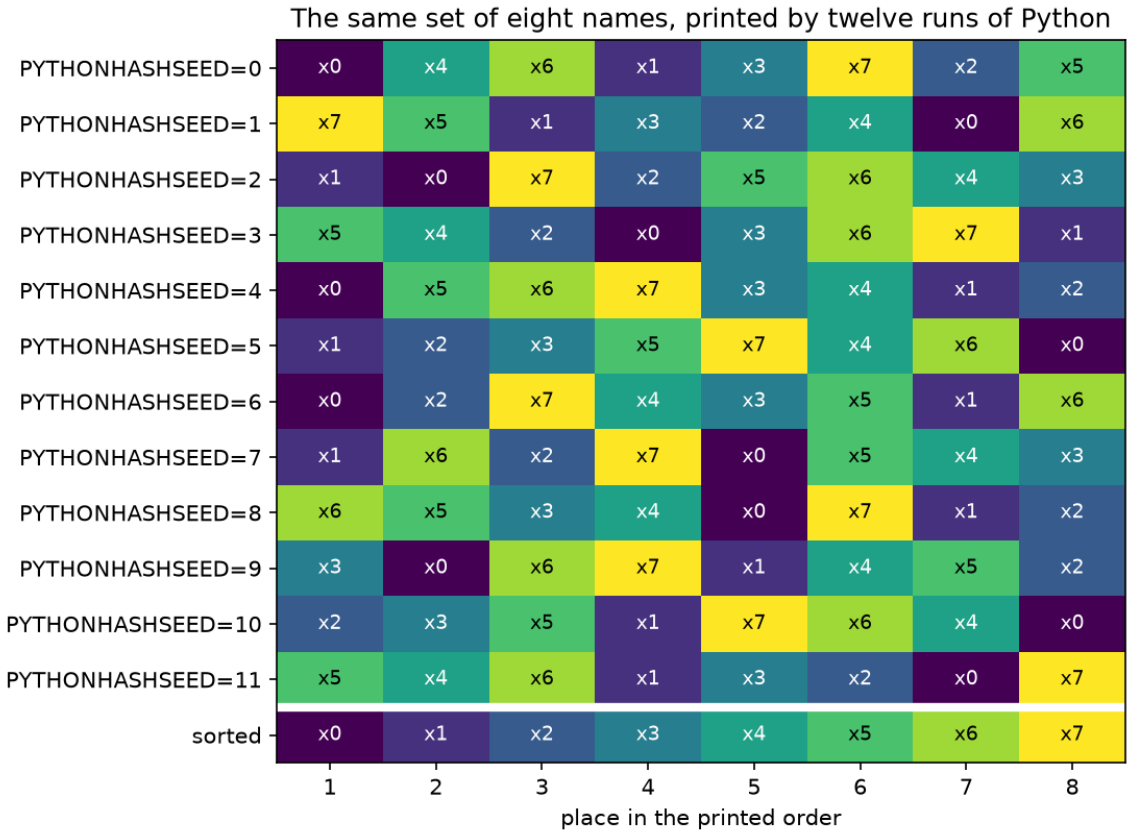

Figure 16: The order in which twelve separate runs of Python print the same set of the eight names $x_0, \ldots, x_7$ (one row per run, labelled by its hash seed PYTHONHASHSEED from 0 to 11; horizontal axis the place 1 to 8 in the printed order; each square coloured by its name, from dark violet for x_0 to yellow for x_7). Every run prints another order, so the rows are scrambled differently; the bottom row, the sorted order, is the same in every run. (Notebook 00d, figure 6.)

Text cell:

```
## 11. Line ends

The same text can be stored with different bytes. The next cell writes the two
lines "x1 x2" and "x3 x4" as bytes, once with LF line ends and once with CR LF line
ends (‘"\n"‘ is LF, ‘"\r\n"‘ is CR LF; ‘.encode("utf-8")‘ turns a text into
bytes), prints the bytes as numbers and their sha256 fingerprints, and checks that
the two versions have different lengths and different fingerprints. A program that
compares files byte for byte therefore needs one fixed kind of line end; the book
and the Revision record use LF everywhere. (The repository’s file .gitattributes
tells git to store and deliver every file unchanged, so that a computer with
Windows receives the same LF line ends.)
```

In [14]:

```
import hashlib  # computes sha256 fingerprints

TEXT = "x1 x2\nx3 x4\n"  # two lines, each ended by a line end
with_lf = TEXT.encode("utf-8")  # LF line ends: the byte 10
with_crlf = TEXT.replace("\n", "\r\n").encode("utf-8")  # CR LF: the bytes 13, 10
for label, data in (("LF   ", with_lf), ("CR LF", with_crlf)):
    say(f"{label} {len(data):2d} bytes: {list(data)}")
    say(f"      sha256 {hashlib.sha256(data).hexdigest()}")
check(len(with_crlf) == len(with_lf) + 2
      and hashlib.sha256(with_lf).digest() != hashlib.sha256(with_crlf).digest(),
      "the same two lines with LF and with CR LF line ends are different bytes")
```

Out [14]:

```
LF    12 bytes: [120, 49, 32, 120, 50, 10, 120, 51, 32, 120, 52, 10]
      sha256 dfcf8f120dfb5a9b8d784c7b038aa4cff5ed0b74c5809b5d51f81f763b30bf59
CR LF 14 bytes: [120, 49, 32, 120, 50, 13, 10, 120, 51, 32, 120, 52, 13, 10]
      sha256 c8b93d4c3f40882e9d91457a270e93ff4ea184e5616806b7e1b6d3ac753b52ab
PASS the same two lines with LF and with CR LF line ends are different bytes
```

Text cell:

```
The next cell looks at the result files of the Revision record’s two Kohn-Sham
solvers: the Rust solver (the folder Revision/kohn_sham/results) and the
independent Python reference solver (the folder Revision/kohn_sham/reference/
results). For each folder it counts the files, adds up their sizes in bytes and
counts the files that contain the two bytes CR LF. The two checks require that no
file has CR LF line ends, as the record’s checks ‘outputs_lf_only‘ (Rust) and
‘reference_outputs_lf_only‘ (reference) found when the solvers ran.
‘folder.rglob("*")‘ lists everything in a folder and in all its sub-folders;
‘sorted‘ puts the list in a fixed order.
```

In [15]:

```
def files_below(folder):
    """The files in the repository folder and its sub-folders, in sorted order."""
    return sorted(p for p in repository_file(folder).rglob("*") if p.is_file())


RUST_RESULTS = "Revision/kohn_sham/results"
REFERENCE_RESULTS = "Revision/kohn_sham/reference/results"
rust_files = files_below(RUST_RESULTS)
rust_crlf = sum(1 for p in rust_files if b"\r\n" in p.read_bytes())
reference_files = files_below(REFERENCE_RESULTS)
reference_crlf = sum(1 for p in reference_files if b"\r\n" in p.read_bytes())
report("Rust result files", len(rust_files))
report("size of the Rust result files", sum(p.stat().st_size for p in rust_files),
       "bytes")
report("Rust result files with CR LF line ends", rust_crlf)
report("reference result files", len(reference_files))
report("size of the reference result files",
       sum(p.stat().st_size for p in reference_files), "bytes")
report("reference result files with CR LF line ends", reference_crlf)

lf_rust = determinism_checks["outputs_lf_only"]  # the record's check (Rust)
check_reproduces(
    rust_crlf == 0 and lf_rust["verdict"] == "PASS"
    and "(offending: none)" in lf_rust["detail"],
    f"none of the {len(rust_files)} Rust result files has CR LF line ends",
    f"{DETERMINISM}, check outputs_lf_only")
cross_checks = {entry["name"]: entry for entry in read_json(CROSS)["checks"]}
lf_reference = cross_checks["reference_outputs_lf_only"]  # the record's check
stated = f"{len(reference_files)} reference result files; files with CRLF: none"
check_reproduces(
    reference_crlf == 0 and lf_reference["verdict"] == "PASS"
    and stated in lf_reference["detail"],
    f"none of the {len(reference_files)} reference result files has CR LF line ends",
    f"{CROSS}, check reference_outputs_lf_only")
```

Out [15]:

```
RESULT Rust result files = 244
RESULT size of the Rust result files = 5857843 bytes
RESULT Rust result files with CR LF line ends = 0
RESULT reference result files = 340
RESULT size of the reference result files = 24125802 bytes
RESULT reference result files with CR LF line ends = 0
PASS none of the 244 Rust result files has CR LF line ends
     reproduces Revision/kohn_sham/reports/ks-rust-determinism.json, check
    outputs_lf_only
PASS none of the 340 reference result files has CR LF line ends
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_outputs_lf_only
```

Text cell:

12. Fingerprints of the result files

Each of the two result folders holds a manifest, a file manifest.json that lists every other file of the folder with its sha256 fingerprint: the Rust solver's manifest gives for each file its path, its size and its fingerprint; the reference solver's manifest gives for each path its fingerprint. The next cell computes the fingerprint of every listed file as it is today, compares it with the listed one, and checks that every file of each folder except the manifest itself is listed. When all agree, the files of today are exactly, byte for byte, the files that the solvers wrote.

The record also ran each solver a second time and compared every file of the two runs: the Rust solver in the check ‘repeat_byte_identical‘ of the report ks-rust-determinism.json ("244 files (... bytes) compared"), the reference solver in the check ‘reference_repeat_byte_identical‘ of the cross-check report. The last two checks of the cell confirm that the record found no differing file and that it compared as many files (and, for the Rust solver, as many bytes) as the folders hold today.

In [16]:

```
def file_fingerprint(path):
    """The sha256 fingerprint of the bytes of the file path."""
    return hashlib.sha256(path.read_bytes()).hexdigest()


def relative_names(files, folder):
    """The paths of files relative to the repository folder, written with /."""
    root = repository_file(folder)
    return sorted(p.relative_to(root).as_posix() for p in files)


# The Rust manifest: a list of entries {"path": ..., "bytes": ..., "sha256": ...}.
rust_manifest = read_json(f"{RUST_RESULTS}/manifest.json")["files"]
rust_listed = {entry["path"]: entry for entry in rust_manifest}  # path -> entry
rust_wrong = []  # the listed files whose size or fingerprint differs today
for path, entry in rust_listed.items():
    today = repository_file(f"{RUST_RESULTS}/{path}")  # the file as it is today
    same_size = today.stat().st_size == entry["bytes"]
    if not same_size or file_fingerprint(today) != entry["sha256"]:
        rust_wrong.append(path)
# The reference manifest: a dictionary path -> fingerprint.
reference_manifest = read_json(f"{REFERENCE_RESULTS}/manifest.json")["files"]
reference_wrong = []
for path, value in reference_manifest.items():
    today = repository_file(f"{REFERENCE_RESULTS}/{path}")
    if file_fingerprint(today) != value:
        reference_wrong.append(path)
report("Rust results: fingerprints compared", len(rust_listed))
report("Rust results: files that differ", len(rust_wrong))
report("reference results: fingerprints compared", len(reference_manifest))
report("reference results: files that differ", len(reference_wrong))
report("fingerprints compared in all", len(rust_listed) + len(reference_manifest))
```

```
rust_others = [n for n in relative_names(rust_files, RUST_RESULTS)
               if n != "manifest.json"]
reference_others = [n for n in relative_names(reference_files, REFERENCE_RESULTS)
                    if n != "manifest.json"]
check(rust_wrong == [] and sorted(rust_listed) == rust_others,
      f"the {len(rust_listed)} fingerprints of the Rust manifest equal today's files")
manifest_check = cross_checks["reference_manifest"]
stated = f"lists {len(reference_manifest)} files with SHA-256"  # the record's words
check_reproduces(
    reference_wrong == [] and sorted(reference_manifest) == reference_others
    and manifest_check["verdict"] == "PASS" and stated in manifest_check["detail"],
    f"the {len(reference_manifest)} fingerprints of the reference manifest equal "
    "today's files",
    f"{CROSS}, check reference_manifest")
rust_repeat = determinism_checks["repeat_byte_identical"]  # the Rust second run
# The part of its detail that counts, e.g. "244 files (5857843 bytes) compared, ...".
found = re.search(r"(\d+) files \((\d+) bytes\) compared, same file set: (\w+), "
                  r"differing files: (\w+)", rust_repeat["detail"])
say(f"the record (Rust): {found.group(0)}")
rust_bytes = sum(p.stat().st_size for p in rust_files)  # the bytes of today
check_reproduces(
    rust_repeat["verdict"] == "PASS"
    and (int(found.group(1)), int(found.group(2))) == (len(rust_files), rust_bytes)
    and found.group(3) == "True" and found.group(4) == "none",
    f"a second run of the Rust solver gave the same {len(rust_files)} files "
    f"({rust_bytes} bytes)",
    f"{DETERMINISM}, check repeat_byte_identical")
repeat = cross_checks["reference_repeat_byte_identical"]  # the reference second run
# The part of its detail that counts the files, e.g. "340 result files, same ...".
found = re.search(r"(\d+) result files, same file set: (\w+), differing files: (\w+)",
                  repeat["detail"])
say(f"the record (reference): {found.group(0)}")
check_reproduces(
    repeat["verdict"] == "PASS" and int(found.group(1)) == len(reference_files)
    and found.group(2) == "True" and found.group(3) == "none",
    f"a second run of the reference solver gave the same {len(reference_files)} "
    "files, byte for byte",
    f"{CROSS}, check reference_repeat_byte_identical")
```

Out [16]:

```
RESULT Rust results: fingerprints compared = 243
RESULT Rust results: files that differ = 0
RESULT reference results: fingerprints compared = 339
RESULT reference results: files that differ = 0
RESULT fingerprints compared in all = 582
PASS the 243 fingerprints of the Rust manifest equal today's files
PASS the 339 fingerprints of the reference manifest equal today's files
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check reference_manifest
the record (Rust): 244 files (5857843 bytes) compared, same file set: True, differing
    files: none
PASS a second run of the Rust solver gave the same 244 files (5857843 bytes)
     reproduces Revision/kohn_sham/reports/ks-rust-determinism.json, check
```

```
    repeat_byte_identical
the record (reference): 340 result files, same file set: True, differing files: none
PASS a second run of the reference solver gave the same 340 files, byte for byte
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
```

Text cell:

```
## 13. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [17]:

```
figure_names = ["00d_1_float_spacing.png", "00d_2_summation_order.png",
                "00d_3_tolerances.png", "00d_4_shared_rounding.png",
                "00d_5_seeded_walks.png", "00d_6_set_orders.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "the six figure files of this notebook exist")
all_checks_passed()
```

Out [17]:

```
PASS the six figure files of this notebook exist
ALL 22 CHECKS PASSED (notebook 00d)
```

Text cell:

```
## 14. What this notebook showed

- The computer stores numbers as binary fractions with 53 significant binary
  digits: 0.1 is stored as $3602879701896397/2^{55}$, and $0.1 + 0.2$ differs from
  $0.3$ by one rounding step, $2^{-54}$. The gap between neighbouring stored
  numbers grows with their size and always lies between $\epsilon x/2$ and
  $\epsilon x$, $\epsilon = 2^{-52}$.
- Rounding is deterministic, but it depends on the order of the operations: the
  same million numbers added forwards and backwards give sums that differ by
  about 200 units in the last place, a relative difference of a few times
  $10^{-14}$. This is why every comparison of two different computations uses a
  tolerance, fixed in advance.
- The Revision record's Kohn-Sham solver was compared with a refined run of itself:
  all eight measured differences lie below the tolerances that were fixed in
  advance (the margins are printed in section 8). The record's negative control
  shows that two runs with the same rounding path hid errors of the chemical
  potential up to about $8 \times 10^{-10}\,m$, which is why the refined run now
  rounds along a different path.
- A fixed seed makes random numbers repeat exactly; the order of a set of names
  changes with the hash seed, while the sorted order does not; LF and CR LF line
```

```
  ends make the same text into different bytes.
- The result files of the record's two Kohn-Sham solvers have only LF line ends,
  and the 582 fingerprints of their two manifests equal the files of today: the
  stored results are exactly the ones the solvers wrote. A second run of each
  solver gave the same files byte for byte.
```

0.25 Line-by-line walk-through of Notebook 00d

The notebook has seventeen code cells, In [1] to In [17]. This section explains every line of each of them.

In [1], the set-up cell. It is the set-up cell of Notebook 00a, word for word, except that its comment lines hold the run instructions of Notebook 00d (Section 0.23) and the line `NOTEBOOK_ID = "00d"` names this notebook. Every line of its code is explained in Section 0.13. It prints Set-up of notebook 00d complete: repository folder found, helpers defined.

In [2], how the computer stores 0.1.

```
import sys  # sys.stdout is the channel through which the notebook prints
from decimal import Decimal  # the exact decimal digits of a stored number
from fractions import Fraction  # a stored number as an exact fraction

import numpy as np  # arrays of numbers
```

Two modules of Python itself that look inside a stored number: `Fraction(x)` gives the exact fraction that the computer stores for the floating-point number `x`, and `Decimal(x)` all its decimal digits. Then `sys` and numpy, as before.

```
def check_reproduces(condition, name, record):
    """check(condition, name, record=record), after sending the waiting output."""
    sys.stdout.flush()  # send every printed line that is still waiting
    check(condition, name, record=record)
```

The helper for checks that reproduce a Revision record, explained in Section 0.17 (In [2] of Notebook 00b).

```
stored = Fraction(0.1)  # the fraction that the computer really stores for 0.1
say(f"0.1 is stored as the fraction {stored}")
say(f"    = {Decimal(0.1)}")
say(f"0.1 + 0.2 = {0.1 + 0.2!r}")
say(f"0.1 + 0.2 == 0.3 is {0.1 + 0.2 == 0.3}")  # == asks: exactly equal?
```

The first two lines of Out [2] print the stored fraction, 3602879701896397/36028797018963968 (the denominator is 2^{55}), and its 55 decimal digits after the point, exactly the result of Section 0.22. `!r` inside the braces prints a number with as many digits as Python needs to identify the stored number uniquely: 0.30000000000000004. The fourth line asks whether $0.1 + 0.2$ is exactly equal to 0.3: False.

```
difference = (0.1 + 0.2) - 0.3
report("(0.1 + 0.2) - 0.3", repr(difference))
check(stored == Fraction(3602879701896397, 2 ** 55),
      "0.1 is stored as 3602879701896397 / 2^55, not exactly as 1/10")
check(0.1 + 0.2 != 0.3 and difference == 2.0 ** -54,
      "0.1 + 0.2 differs from 0.3 by 2^-54 = 5.55e-17, one rounding step")
```

The difference is printed with `repr` (the same as `!r`): 5.551115123125783e-17, which means $5.551115123125783 \times 10^{-17}$. The two checks confirm the two results derived by hand in Section 0.22: the stored 0.1 is $3602879701896397/2^{55}$, and $(0.1 + 0.2) - 0.3$ is exactly 2^{-54} (`!=` means "is not equal to").

In [3], the machine epsilon.

```
epsilon = float(np.finfo(np.float64).eps)  # the gap between 1 and the next number
report("machine epsilon", repr(epsilon))
say(f"1 + epsilon     == 1 is {1.0 + epsilon == 1.0}")
say(f"1 + epsilon / 2 == 1 is {1.0 + epsilon / 2 == 1.0}")
check(epsilon == 2.0 ** -52 and 1.0 + epsilon > 1.0 and 1.0 + epsilon / 2 == 1.0,
      "machine epsilon is 2^-52: 1 + 2^-52 is stored, 1 + 2^-53 rounds back to 1")
```

`np.finfo(np.float64)` describes numpy's 64-bit floating-point numbers, and its `.eps` is the machine epsilon, printed as 2.220446049250313e-16. Then $1 + \epsilon$ is not equal to 1 (False), while $1 + \epsilon/2$ is rounded back to 1 (True), the halfway case of Section 0.22. The check confirms all three facts.

In [4], the gaps between neighbouring numbers.

```
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
x_values = np.logspace(-3.0, 6.0, 3000)  # 3000 numbers from 0.001 to 1000000
gaps = np.spacing(x_values)  # the gap from each number to the next larger one
for x in (1.0, 1000.0, 1.0e6):
    say(f"gap above {x:>9g}: {np.spacing(x):.3e}")
```

Four colours. `np.logspace(-3.0, 6.0, 3000)` makes 3000 numbers from 10^{-3} to 10^{6} whose logarithms are equally spaced, so that they spread evenly over a logarithmic axis. `np.spacing(x)` is the gap from x to the next larger stored number. The loop prints the gaps at 1, 1000 and 10^6: 2.220×10^{-16}, 1.137×10^{-13} and 1.164×10^{-10} (Out [4]). Each is about ϵ times the number: 1000 lies between $2^9 = 512$ and 2^{10}, so its gap is $2^{9-52} = 2^{-43} = 1.137 \times 10^{-13}$.

```
fig, ax = plt.subplots()
# drawstyle="steps-post" draws each value as a flat step until the next x.
ax.plot(x_values, gaps, color=BLUE, linewidth=1.5, drawstyle="steps-post",
        label="gap to the next stored number")
ax.plot(x_values, epsilon * x_values, color="black", linestyle="--",
        linewidth=1.0, label=r"$\epsilon x$")
ax.plot(x_values, epsilon * x_values / 2, color="black", linestyle=":",
        linewidth=1.0, label=r"$\epsilon x / 2$")
ax.set_xscale("log")  # logarithmic horizontal axis
ax.set_yscale("log")  # logarithmic vertical axis
ax.set_xlabel(r"the number $x$")
```

```
ax.set_ylabel("gap to the next stored number")
ax.set_title("The gaps between neighbouring floating-point numbers")
ax.legend(loc="upper left")
```

The gaps are drawn as a staircase (each value as a flat step), together with the two straight lines ϵx (dashed) and $\epsilon x/2$ (dotted) that Section 0.22 derived as the upper and lower bounds; both axes are logarithmic.

```
save_figure(fig, "float_spacing",
            r"The gap between a floating-point number $x$ and the next larger "
            r"stored number (blue staircase) for $x$ from $10^{-3}$ to $10^{6}$, on "
            r"logarithmic axes (both pure numbers), with the lines $\epsilon x$ "
            r"(dashed) and $\epsilon x/2$ (dotted), $\epsilon = 2^{-52}$. The gap is "
            r"constant between two powers of 2 and doubles at each of them, so it "
            r"always lies between the two lines: every stored number carries about "
            r"16 significant digits, whatever its size.")
ratios = gaps / x_values  # the gap relative to the number
check(bool(np.all(ratios > epsilon / 2) and np.all(ratios <= epsilon))
      and np.spacing(1.0) == epsilon,
      "the gap lies between eps x / 2 and eps x at all 3000 points")
```

The figure `00d_1_float_spacing.png`, and the check of the bounds of Section 0.22 at all 3000 points: `np.all` is true when the comparison holds for every entry, and `bool` makes a plain truth value of it. What Figure 00d.1 shows: a staircase that climbs over nine powers of ten while staying between the two parallel lines; each step is flat between two powers of 2 and jumps up by a factor 2 at each of them.

In [5], one sum in three orders.

```
import math  # math.fsum: the exactly rounded sum; math.pi

N_MAX = 10 ** 6
k = np.arange(1, N_MAX + 1, dtype=np.float64)  # 1, 2, ..., 1000000 as floats
terms = 1.0 / (k * k)  # the numbers 1/k^2
forward = np.add.accumulate(terms)  # all forward running totals
```

The module `math` of Python provides `math.fsum`, which adds stored numbers exactly and rounds only once, at the end (the **exactly rounded** sum), and π. `np.arange(1, N_MAX + 1, ...)` is the array 1, 2, ..., 10^6 (the end is not included, hence the +1), as floating-point numbers; `terms` holds the million numbers $1/k^2$. `np.add.accumulate` adds them one after the other, starting with the first and largest, and returns every running total: `forward[n - 1]` is the forward sum of the first `n` terms.

```
# 31 values of N, equally spaced on a logarithmic axis from 10 to 10^6.
n_values = np.unique(np.round(np.logspace(1.0, 6.0, 31)).astype(int))
ulps = {"forward": [], "backward": [], "pairwise": []}  # the errors in ulps
```

31 numbers of terms from 10 to 10^6, rounded to whole numbers (`np.round`, then `.astype(int)`); `np.unique` sorts them and removes repetitions. The dictionary `ulps` will hold the three lists of errors.

```
for n in n_values:
    exact = math.fsum(terms[:n])  # the exactly rounded sum of the first n numbers
    gap = np.spacing(exact)  # one ulp of the exact sum
    backward = np.add.accumulate(terms[:n][::-1])[-1]  # smallest number first
    ulps["forward"].append(float((forward[n - 1] - exact) / gap))
    ulps["backward"].append(float((backward - exact) / gap))
    ulps["pairwise"].append(float((np.sum(terms[:n]) - exact) / gap))
```

For each number of terms `n`: the exactly rounded sum of the first `n` terms (`terms[:n]`); one ulp of it; the backward sum, which adds the reversed list (`[::-1]`) and keeps the last running total (`[-1]`); and the errors of the three sums, each divided by the ulp, so that they are measured in ulps. `np.sum` adds in pairs, then pairs of pairs, and so on (**pairwise summation**).

```
# The three sums of all 10^6 numbers, as plain Python floats (float(...)).
forward_million = float(forward[-1])
backward_million = float(np.add.accumulate(terms[::-1])[-1])
pairwise_million = float(np.sum(terms))
exact_million = math.fsum(terms)
say(f"forward  sum of 10^6 numbers: {forward_million!r}")
say(f"backward sum of 10^6 numbers: {backward_million!r}")
say(f"pairwise sum of 10^6 numbers: {pairwise_million!r}")
say(f"exactly rounded sum         : {exact_million!r}")
```

The four sums of all million terms, printed with every digit that identifies them: the forward sum 1.64493306684877, the backward and the pairwise sum 1.6449330668487263, and the exactly rounded sum 1.6449330668487265 (Out [5]). Forwards and backwards, the same numbers give different last digits.

```
worst_forward = max(abs(u) for u in ulps["forward"])  # the largest size of error
worst_backward = max(abs(u) for u in ulps["backward"])
worst_pairwise = max(abs(u) for u in ulps["pairwise"])
report("largest forward error in ulps", f"{worst_forward:.0f}")
report("largest backward error in ulps", f"{worst_backward:.0f}")
report("largest pairwise error in ulps", f"{worst_pairwise:.0f}")
report("pi^2/6 minus the exact sum of 10^6 numbers",
       f"{math.pi ** 2 / 6 - exact_million:.6e}")
```

The largest size of each error over the 31 values of N: 196 ulps forwards, 1 ulp backwards, 3 ulps pairwise. The last RESULT line is the error of the method, $\pi^2/6$ minus the sum, 9.999995×10^{-7}, inside the bounds $1/(N+1)$ and $1/N$ derived in Section 0.22.

In [6], the three errors drawn.

```
fig, ax = plt.subplots()
ax.plot(n_values, ulps["forward"], "o-", color=BLUE, markersize=4,
        label="forwards (largest number first)")
ax.plot(n_values, ulps["backward"], "s-", color=ORANGE, markersize=4,
        label="backwards (smallest number first)")
ax.plot(n_values, ulps["pairwise"], "^:", color=AQUA, markersize=4,
        label="pairwise (numpy's np.sum)")
```

```
ax.axhline(0.0, color="black", linewidth=0.8)  # the exactly rounded sum
ax.set_xscale("log")
ax.set_xlabel(r"number of terms $N$ of the sum $1 + 1/4 + \cdots + 1/N^2$")
ax.set_ylabel("error in units of the last place (ulps)")
ax.set_title("The same numbers added in different orders")
ax.legend(loc="upper left")
```

The three lists of errors against N on a logarithmic horizontal axis: `"o-"` draws circles joined by lines, `"s-"` squares, `"^:"` triangles joined by a dotted line. `ax.axhline(0.0, ...)` draws a horizontal line at 0, the exactly rounded sum.

```
save_figure(fig, "summation_order",
            r"The rounding error of the sum $1 + 1/4 + 1/9 + \cdots + 1/N^2$ in "
            r"units of the last place (vertical axis, ulps, the gap between "
            r"neighbouring stored numbers at the sum) against the number of terms "
            r"$N$ from 10 to $10^{6}$ (horizontal axis, logarithmic), measured from "
            r"the exactly rounded sum: forwards with the largest term first (blue "
            r"circles), backwards with the smallest term first (orange squares), "
            r"and pairwise as numpy adds (aqua triangles). The forward error grows "
            f"to {worst_forward:.0f} ulps, a relative error of a few times "
            r"$10^{-14}$; the backward error stays within "
            f"{worst_backward:.0f} ulp and the pairwise error within "
            f"{worst_pairwise:.0f} ulps. Same numbers, different order, different "
            r"last digits.")
```

The figure `00d_2_summation_order.png`; its caption takes the three worst errors from the computation. What Figure 00d.2 shows: the blue forward errors wander away from zero as N grows, to almost 200 ulps, while the orange and aqua errors stay on the line at 0 within a few ulps.

```
relative = max(abs(s - exact_million) for s in
               (forward_million, backward_million, pairwise_million)) / exact_million
report("largest relative difference of the three sums of 10^6 numbers",
       f"{relative:.2e}")
check(forward_million != backward_million,
      "forwards and backwards, the sums of the 10^6 numbers differ in the last digits")
check(worst_backward <= 1.0,
      "the backward sum is within one ulp of the exactly rounded sum for every N")
check(relative < 1e-13,
      "the three sums agree to a relative 1e-13: a tolerance of 1e-12 accepts all")
```

The largest distance of the three sums from the exactly rounded sum, divided by the sum (a **relative** difference): 2.65×10^{-14}. Three checks: the forward and the backward sum are different stored numbers; the backward sum is never more than 1 ulp off; and all three agree to a relative 10^{-13}, so a check with a tolerance of 10^{-12} would accept each of them.

In [7], the tolerances of the Revision record.

```
import re  # finds patterns in texts ("regular expressions")
```

```
def read_json(path):
    """The content of the JSON file path of the repository (dictionaries, lists)."""
    return json.loads(repository_file(path).read_text(encoding="utf-8"))
```

The module of regular expressions (explained in Section 0.21, In [6]) and a function that reads a JSON file of the repository.

```
DETERMINISM = "Revision/kohn_sham/reports/ks-rust-determinism.json"
CROSS = "Revision/kohn_sham/reports/ks-crosscheck.json"
determinism_checks = {entry["name"]: entry for entry in read_json(DETERMINISM)["checks"]}
measured = read_json(CROSS)["rust_matrix_wide_uncertainties"]  # name -> difference
```

The determinism report of the Kohn-Sham solver and the cross-check report. `{entry["name"]: entry for entry in ...}` is a **dictionary comprehension**: it makes a dictionary from each check's name to the check, so that a check can be looked up by name. `measured` is a dictionary from the name of each of the eight comparisons to its measured largest difference, as the cross-check quotes it.

```
SHORT_NAMES = {  # the check names of the record -> short names for the table
    "refined_ground_energies": "ground-state energies",
    "refined_ground_homo_lumo_gap": "level gaps",
    "refined_eigenvalues": "all Kohn-Sham levels",
    "refined_delta_scf": "excitation energies",
    "refined_profiles": "profiles",
    "refined_adiabatic_derivatives": "adiabatic derivatives",
    "refined_thermodynamics": "thermodynamics",
    "refined_heat_capacity": "heat capacity",
}
```

The eight checks of the comparison between the canonical and the refined run, each with a short name for the table (the quantities are explained in Chapters 14 and 15; here only their sizes matter).

```
tolerances, quoted, verdicts = {}, {}, {}
say("quantity                measured   tolerance   margin")
for name, short in SHORT_NAMES.items():
    detail = determinism_checks[name]["detail"]
    verdicts[name] = determinism_checks[name]["verdict"]
    tolerances[name] = float(re.findall(r"tolerance (\S+)", detail)[-1])
    quoted[name] = f"{measured[name]:.3e}" in detail  # the same number in both files
    margin = tolerances[name] / measured[name]
    say(f"{short:22} {measured[name]:9.3e}   {tolerances[name]:9.0e}   {margin:7.1f}")
```

Three empty dictionaries, and a table header. For each comparison: the detail text and the verdict of the record's check; the tolerance, found with the pattern `tolerance (\S+)` (`\S+` is one or more characters that are not blanks, so it catches the number after the word tolerance; `[-1]` takes the last match, and `float` turns the text into a number); whether the measured number, written with four significant digits (`.3e`), occurs in the detail text too, so that both files state the same number; and the **margin**, the tolerance divided by the measured difference. The table of Out [7] shows, for example, the ground-state energies: measured 1.901×10^{-12}, tolerance 10^{-8}, margin 5260.4; the smallest margin, 4.7, belongs to all Kohn-Sham levels (2.135×10^{-9} against 10^{-8}).

```
smallest = min(tolerances[n] / measured[n] for n in SHORT_NAMES)
report("smallest margin (tolerance / measured difference)", f"{smallest:.1f}")
check_reproduces(
    sorted(measured) == sorted(SHORT_NAMES) and all(quoted.values())
    and all(verdicts[n] == "PASS" and measured[n] < tolerances[n] for n in SHORT_NAMES),
    "each of the eight measured differences of the record is below its tolerance",
    f"{DETERMINISM}, checks " + ", ".join(SHORT_NAMES))
```

The smallest margin is printed. The check requires that the cross-check quotes exactly these eight comparisons (`sorted` of a dictionary gives its sorted keys), that each measured number appears in its check's detail, and that each check passed with the measured difference below the tolerance. Its record names the eight checks one by one: `", ".join(SHORT_NAMES)` joins the keys of the dictionary, in their order, into one string with a comma and a blank between neighbours. (In the report the check `refined_mermin_root_path` stands between the last two of them; it is not a comparison with a tolerance but the negative control of In [9].) Out [7] ends with the PASS line and the record line, which `say` breaks into four lines.

In [8], the tolerances drawn.

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
rows = np.arange(len(SHORT_NAMES))[::-1]  # the first quantity at the top
for row, name in zip(rows, SHORT_NAMES):
    ax.plot([measured[name], tolerances[name]], [row, row], color="#b5b3ad",
            linewidth=2.5)  # the margin
```

One row per comparison, the first at the top (as in Section 0.21, In [7]). In each row a thick grey line runs from the measured difference to the tolerance: on a logarithmic axis its length is the logarithm of the margin.

```
ax.plot([measured[n] for n in SHORT_NAMES], rows, "o", color=BLUE, markersize=7,
        label="largest measured difference")
ax.plot([tolerances[n] for n in SHORT_NAMES], rows, "|", color="black",
        markersize=16, markeredgewidth=2.0, label="tolerance fixed in advance")
ax.set_xscale("log")
ax.set_xlim(1e-13, 1e-4)
ax.set_yticks(rows, labels=list(SHORT_NAMES.values()))
ax.grid(False, axis="y")
ax.set_xlabel("canonical minus refined run (relative, or in units of the mass m)")
ax.set_title("Kohn-Sham solver: measured differences and tolerances")
# The legend below the picture, in two columns, so that it covers no row.
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=2)
```

A blue dot at each measured difference and a black vertical bar (the marker `"|"`) at each tolerance; a logarithmic axis from 10^{-13} to 10^{-4}; the short names label the rows; the legend is placed below the picture (`bbox_to_anchor=(0.5, -0.16)`, measured in units of the picture's size) in two columns (`ncol=2`).

```
margins = [tolerances[n] / measured[n] for n in SHORT_NAMES]  # tolerance / measured
save_figure(fig, "tolerances",
```

```
        r"The eight comparisons between the canonical and the refined run of "
        r"the Revision record's Kohn-Sham solver (one row each): the largest "
        r"measured difference (blue dot) and the tolerance fixed before the "
        r"comparison (black bar), on a logarithmic horizontal axis (relative "
        r"differences, or level differences in units of the mass $m$). Every "
        r"dot lies to the left of its bar: the measured differences are "
        f"between {min(margins):.1f} and {max(margins):.0f} times smaller "
        r"than their tolerances. The grey line from a dot to its bar is this "
        r"margin.")
```

The figure `00d_3_tolerances.png`, whose caption quotes the smallest and the largest margin, 4.7 and 7692. What Figure 00d.3 shows: every blue dot lies to the left of its black bar, most of them by two to four powers of ten; the closest pair is that of all Kohn-Sham levels.

In [9], the negative control.

```
root_path = determinism_checks["refined_mermin_root_path"]  # the record's check
CONTROL = (r"(N\w+): direct-count error (\S+) m \(refined run \S+\), "
           r"former measure (\S+), present measure against it (\S+),")
```

The record's check `refined_mermin_root_path` and a regular expression that finds its entries for the three states. `\w+` is one or more letters, digits or underscores (the name of a state, such as N8_lam0_a00_T10), `\S+` a number, and `\(` and `\)` are round brackets themselves; the four bracketed parts are the name of the state, the error of the old method, the **former measure** (the difference between the two runs when both used the old method) and the **present measure** (the difference seen by the present refined run).

```
control = []  # (state, error of the old root, former measure, present measure)
for state, error, former, present in re.findall(CONTROL, root_path["detail"]):
    control.append((state, abs(float(error)), float(former), float(present)))
say("state              old method error   former measure   present measure")
for state, error, former, present in control:
    say(f"{state:17} {error:12.3e} m   {former:14.3e}   {present:15.3e}")
```

`re.findall` returns, for every match, the four bracketed parts; the numbers are turned into floating-point numbers (the error with `abs`, its size). The table of Out [9] has three rows: the largest error of the old method, $8.267 \times 10^{-10}\,m$ in the state N8_lamm1_a00_T10, was seen by the former comparison as a difference of only 5.829×10^{-16}, and by the present comparison as 8.271×10^{-10}.

```
largest_hidden = max(error for _, error, _, _ in control)
report("largest error that one shared rounding path hid", f"{largest_hidden:.3e}", "m")
check_reproduces(
    root_path["verdict"] == "PASS" and len(control) == 3
    and all(former < error / 2 <= present for _, error, former, present in control),
    f"one shared rounding path hid errors up to {largest_hidden:.1e} m; "
    "two paths show them",
    f"{DETERMINISM}, check refined_mermin_root_path")
```

The largest hidden error is printed with the unit m. The check states, as the record does, that in each of the three states the former measure is below half of the error (it hid it), while the

present measure is at least half of it (it shows it): a chain comparison `a < b <= c` holds when both comparisons hold.

In [10], the negative control drawn.

```
fig, ax = plt.subplots(figsize=(7.0, 4.0))
rows = np.arange(len(control))[::-1]  # the first state at the top
height = 0.26  # three bars in each row
errors = [error for _, error, _, _ in control]
formers = [former for _, _, former, _ in control]
presents = [present for _, _, _, present in control]
# How many times smaller the former measure is than the error, state by state.
hidden_ratios = [error / former for error, former in zip(errors, formers)]
```

One row per state, three bars in each; the three columns of the table as three lists; and, for the caption, how many times smaller the former measure was than the error in each state.

```
ax.barh(rows + height, errors, height, color=ORANGE,
        label="error of the old method")
ax.barh(rows, formers, height, color="#b5b3ad",
        label="former comparison (one rounding path in both runs)")
ax.barh(rows - height, presents, height, color=BLUE,
        label="present comparison (two rounding paths)")
ax.set_xscale("log")
ax.set_xlim(1e-17, 1e-8)
ax.set_yticks(rows, labels=[state for state, _, _, _ in control])
ax.grid(False, axis="y")
ax.set_xlabel(r"size of the difference in $\mu$ (units of the mass m)")
ax.set_title("A comparison that rounds the same way twice sees nothing")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.18), ncol=1)
```

The three bars of each state, orange above, grey in the middle and blue below, on a logarithmic axis from 10^{-17} to 10^{-8}; the states label the rows, and the legend stands below the picture.

```
save_figure(fig, "shared_rounding",
            r"The negative control of the Revision record's Kohn-Sham solver for "
            r"the three thermal states named on the vertical axis: the error of the "
            r"old method for the chemical potential $\mu$ (orange), the difference "
            r"between the two runs when both used that method, and so shared its "
            r"rounding (grey), and the difference seen by the present refined run, "
            r"which rounds along another path (blue); horizontal axis the size of "
            r"the difference in units of the mass $m$, logarithmic. The grey bars "
            f"are between {min(hidden_ratios):.0f} and "
            f"{max(hidden_ratios) / 1e6:.1f} million "
            r"times shorter than the orange ones: two runs with the same rounding "
            f"hid errors up to {largest_hidden:.1e} "
            r"$m$, which the blue bars show in full.")
```

The figure `00d_4_shared_rounding.png`; its caption computes how much shorter the grey bars are. What Figure 00d.4 shows: in each state the orange and the blue bar have almost the same length

(the present comparison sees the whole error), while the grey bar is between 93 and 1.4 million times shorter, as the caption computes from the table of Out [9] (the former comparison saw almost nothing). This is the lesson of the negative control of Section 0.22.

In [11], random walks with seeds.

```
walks = {}  # a label -> the 401 positions of the walk (it starts at 0)
for label, seed in (("seed 12345, first run", 12345),
                    ("seed 12345, second run", 12345), ("seed 2026", 2026)):
    generator = np.random.default_rng(seed)  # a new generator from this seed
    steps = generator.choice([-1, 1], size=400)  # 400 random steps of +1 or -1
    walks[label] = np.concatenate([[0], np.cumsum(steps)])  # positions 0 ... 400
    first = " ".join(f"{s:+d}" for s in steps[:10])  # e.g. "+1 -1 +1 ..."
    say(f"{label:23} first steps: {first}")
```

Three new random-number generators, two from the seed 12345 and one from the seed 2026. `generator.choice([-1, 1], size=400)` picks 400 times at random from the list $[-1, 1]$: the steps of a **random walk**. `np.cumsum` gives the running totals, the positions after each step, and `np.concatenate([[0], ...])` puts the starting position 0 in front. The first ten steps of each walk are printed (Out [11]): the two walks of the seed 12345 begin with the same ten steps, the third with others.

```
fig, ax = plt.subplots()
step_numbers = np.arange(401)
ax.plot(step_numbers, walks["seed 12345, first run"], color=BLUE, linewidth=3.0,
        label="seed 12345, first run")
ax.plot(step_numbers, walks["seed 12345, second run"], color=YELLOW, linewidth=1.2,
        linestyle="--", label="seed 12345, second run (lies on the first)")
ax.plot(step_numbers, walks["seed 2026"], color=ORANGE, linewidth=1.5,
        label="seed 2026")
ax.set_xlabel("step number")
ax.set_ylabel("position (sum of the steps)")
ax.set_title("Random walks: the same seed gives the same walk")
# The legend below the picture, so that it covers no part of the walks.
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=1)
```

The three walks: the first thick and blue, the second thin, dashed and yellow, so that it is visible on top of the first, the third orange.

```
save_figure(fig, "seeded_walks",
            r"Three random walks of 400 steps of $+1$ or $-1$ (horizontal axis the "
            r"step number, vertical axis the position, the sum of the steps so far; "
            r"pure numbers). The thick blue walk and the dashed yellow walk come from "
            r"two separate generators started with the same seed 12345: they are "
            r"identical, step for step, so the yellow line lies on the blue one. "
            r"The orange walk, from the seed 2026, is different. A fixed seed makes "
            r"random numbers repeat exactly in every run.")
check(np.array_equal(walks["seed 12345, first run"], walks["seed 12345, second run"]),
      "two generators with the same seed give the same 400 steps")
check(not np.array_equal(walks["seed 12345, first run"], walks["seed 2026"]),
      "a generator with another seed gives other steps")
```

The figure `00d_5_seeded_walks.png` and two checks: `np.array_equal` is true when two arrays have the same shape and the same entries. What Figure 00d.5 shows: the yellow dashed walk runs exactly inside the thick blue one for all 400 steps, while the orange walk goes its own way.

In [12], the order of a set.

```
import subprocess  # starts another program and reads what it prints

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
PROGRAM = "import sys; print(*set(sys.argv[1:]))"  # print the names of a set
```

`subprocess` starts other programs. `PROGRAM` is a complete little Python program in one line: it makes a set of the names it receives as its arguments (`sys.argv` is the list of the arguments of a program, and `[1:]` leaves out the first entry, the program's own name) and prints them separated by blanks (`print(*...)` prints the members one after the other).

```
def set_order(seed):
    """The order in which a new Python with PYTHONHASHSEED = seed prints the set."""
    environment = dict(os.environ, PYTHONHASHSEED=str(seed))
    completed = subprocess.run([sys.executable, "-c", PROGRAM, *NAMES],
                               capture_output=True, text=True, env=environment,
                               check=True)  # check=True: stop if Python failed
    return completed.stdout.split()  # the printed names, in their printed order
```

`dict(os.environ, PYTHONHASHSEED=str(seed))` is a copy of the environment variables of the notebook with one of them set to the given hash seed. `subprocess.run` starts a new Python (`sys.executable` is the Python program that runs this notebook) with the option `-c`, which tells it to run the program text that follows, and the eight names as arguments; the new Python receives the copied environment (`env=`). Its printed line is collected and cut at the blanks into the list of the names in their printed order.

```
orders = [set_order(seed) for seed in range(12)]  # the seeds 0, 1, ..., 11
for seed, order in enumerate(orders):
    printed = " ".join(order)  # the eight names separated by blanks
    say(f"PYTHONHASHSEED={seed:<2}  {printed}")
in_order = " ".join(sorted(orders[0]))  # sorted: x0 x1 ... x7
say(f"sorted              {in_order}")
different = len({tuple(order) for order in orders})  # the number of distinct orders
report("distinct orders among the 12 runs", different)
```

Twelve runs with the hash seeds 0 to 11; the order of each is printed (the first twelve lines of Out [12]), then the sorted order. `{tuple(order) for order in orders}` is a set of the twelve orders (a list cannot be a member of a set, a **tuple** can); equal orders count once, so its size is the number of distinct orders: 12, every run printed another order.

```
check(set_order(0) == orders[0], "the same hash seed gives the same order")
check(different >= 2, "different hash seeds give different orders of the same set")
check(all(sorted(order) == NAMES for order in orders),
      "sorted, the names come in the same order x0 ... x7 in every run")
```

Three checks: a thirteenth run with the seed 0 repeats the order of the first; at least two of the orders differ; and every order, sorted, is $x_0, \ldots, x_7$. This is why the notebooks sort before they print.

In [13], the twelve orders as a picture.

```
table = orders + [sorted(orders[0])]  # 13 rows of 8 names: 12 runs and the sorted
# Each name replaced by its number in NAMES: x0 -> 0, x1 -> 1, ..., x7 -> 7.
numbers = np.array([[NAMES.index(name) for name in row] for row in table])
colours = matplotlib.colormaps["viridis"].resampled(8)  # 8 colours, violet to yellow
```

The twelve orders and, as a thirteenth row, the sorted order. `NAMES.index(name)` is the position of a name in the list `NAMES`, so each name becomes a number from 0 to 7, and `numbers` is a table of 13 rows and 8 columns. The colour map viridis, cut into 8 colours from dark violet to yellow, gives each name its colour.

```
fig, ax = plt.subplots(figsize=(7.0, 6.0))
# vmin and vmax put each of the numbers 0 ... 7 in the middle of its own colour.
ax.imshow(numbers, cmap=colours, vmin=-0.5, vmax=7.5, aspect="auto")
for row in range(numbers.shape[0]):
    for column in range(8):
        # white letters on the dark colours, black ones on the light colours
        ink = "white" if numbers[row, column] < 5 else "black"
        ax.text(column, row, table[row][column], ha="center", va="center",
                color=ink, fontsize=9)
```

The table drawn as coloured squares (as in Section 0.17, In [3]); `aspect="auto"` lets the squares stretch to fill the figure, and `numbers.shape[0]` is the number of rows, 13. Each square gets the name written into it.

```
ax.axhline(11.5, color="white", linewidth=4.0)  # a gap before the sorted row
ax.set_xticks(range(8), labels=[str(place) for place in range(1, 9)])
ax.set_yticks(range(13),
              labels=[f"PYTHONHASHSEED={seed}" for seed in range(12)] + ["sorted"])
ax.grid(False)
ax.set_xlabel("place in the printed order")
ax.set_title("The same set of eight names, printed by twelve runs of Python")
```

A thick white line separates the sorted row from the twelve runs; the columns are labelled with the places 1 to 8 and the rows with the hash seeds and the word sorted.

```
save_figure(fig, "set_orders",
            r"The order in which twelve separate runs of Python print the same set "
            r"of the eight names $x_0, \ldots, x_7$ (one row per run, labelled by "
            r"its hash seed PYTHONHASHSEED from 0 to 11; horizontal axis the place 1 "
            r"to 8 in the printed order; each square coloured by its name, from "
            r"dark violet for $x_0$ to yellow for $x_7$). Every run prints another "
            r"order, so the rows are scrambled differently; the bottom row, the "
            r"sorted order, is the same in every run.")
```

The figure `00d_6_set_orders.png`. What Figure 00d.6 shows: twelve rows of scrambled colours, no two alike, above one row in which the colours run in order from violet to yellow.

In [14], line ends.

```
import hashlib  # computes sha256 fingerprints

TEXT = "x1 x2\nx3 x4\n"  # two lines, each ended by a line end
with_lf = TEXT.encode("utf-8")  # LF line ends: the byte 10
with_crlf = TEXT.replace("\n", "\r\n").encode("utf-8")  # CR LF: the bytes 13, 10
```

In a Python string, `\n` is the line end LF and `\r` the carriage return CR. The same two lines are made into bytes once with LF and once with CR LF (`replace` puts `\r\n` for every `\n`).

```
for label, data in (("LF   ", with_lf), ("CR LF", with_crlf)):
    say(f"{label} {len(data):2d} bytes: {list(data)}")
    say(f"      sha256 {hashlib.sha256(data).hexdigest()}")
check(len(with_crlf) == len(with_lf) + 2
      and hashlib.sha256(with_lf).digest() != hashlib.sha256(with_crlf).digest(),
      "the same two lines with LF and with CR LF line ends are different bytes")
```

`list(data)` writes the bytes as numbers: the letter x is 120, the digits 1 to 4 are 49 to 52, the blank is 32, LF is 10 and CR is 13. Out [14] shows 12 bytes with LF and 14 with CR LF, and two completely different fingerprints. The check confirms the two extra bytes and the different fingerprints (`.digest()` is the fingerprint as bytes instead of characters).

In [15], the line ends of the record's result files.

```
def files_below(folder):
    """The files in the repository folder and its sub-folders, in sorted order."""
    return sorted(p for p in repository_file(folder).rglob("*") if p.is_file())
```

All files in a folder and its sub-folders (`rglob("*")` lists everything, `is_file` keeps the files), in sorted order.

```
RUST_RESULTS = "Revision/kohn_sham/results"
REFERENCE_RESULTS = "Revision/kohn_sham/reference/results"
rust_files = files_below(RUST_RESULTS)
rust_crlf = sum(1 for p in rust_files if b"\r\n" in p.read_bytes())
reference_files = files_below(REFERENCE_RESULTS)
reference_crlf = sum(1 for p in reference_files if b"\r\n" in p.read_bytes())
```

The result folders of the Rust solver and of the Python reference solver. For each, the files are listed and the files that contain the two bytes CR LF are counted (`b"\r\n"` is a **bytes** literal, and `in` searches for it in the bytes of the file).

```
report("Rust result files", len(rust_files))
report("size of the Rust result files", sum(p.stat().st_size for p in rust_files),
```

```
       "bytes")
report("Rust result files with CR LF line ends", rust_crlf)
report("reference result files", len(reference_files))
report("size of the reference result files",
       sum(p.stat().st_size for p in reference_files), "bytes")
report("reference result files with CR LF line ends", reference_crlf)
```

The numbers of files, their total sizes (`p.stat().st_size` is the size of a file in bytes) and the numbers of files with CR LF: Out [15] reports 244 Rust result files with 5857843 bytes and 340 reference result files, none with CR LF line ends.

```
lf_rust = determinism_checks["outputs_lf_only"]  # the record's check (Rust)
check_reproduces(
    rust_crlf == 0 and lf_rust["verdict"] == "PASS"
    and "(offending: none)" in lf_rust["detail"],
    f"none of the {len(rust_files)} Rust result files has CR LF line ends",
    f"{DETERMINISM}, check outputs_lf_only")
cross_checks = {entry["name"]: entry for entry in read_json(CROSS)["checks"]}
lf_reference = cross_checks["reference_outputs_lf_only"]  # the record's check
stated = f"{len(reference_files)} reference result files; files with CRLF: none"
check_reproduces(
    reference_crlf == 0 and lf_reference["verdict"] == "PASS"
    and stated in lf_reference["detail"],
    f"none of the {len(reference_files)} reference result files has CR LF line ends",
    f"{CROSS}, check reference_outputs_lf_only")
```

Two checks, one per solver: today no file has CR LF line ends, and the record's own check found the same when the solver ran (its detail says that no file offended, or states the number of reference files and that none had CRLF).

In [16], the fingerprints of the result files.

```
def file_fingerprint(path):
    """The sha256 fingerprint of the bytes of the file path."""
    return hashlib.sha256(path.read_bytes()).hexdigest()


def relative_names(files, folder):
    """The paths of files relative to the repository folder, written with /."""
    root = repository_file(folder)
    return sorted(p.relative_to(root).as_posix() for p in files)
```

The fingerprint of a file (as in Section 0.21, In [13], but for a path that is already complete), and the sorted names of files relative to a folder, written with `/`. (The docstring says "the repository folder"; the names are relative to the folder `folder`, which is what the manifests list.)

```
# The Rust manifest: a list of entries {"path": ..., "bytes": ..., "sha256": ...}.
rust_manifest = read_json(f"{RUST_RESULTS}/manifest.json")["files"]
rust_listed = {entry["path"]: entry for entry in rust_manifest}  # path -> entry
rust_wrong = []  # the listed files whose size or fingerprint differs today
```

```
for path, entry in rust_listed.items():
    today = repository_file(f"{RUST_RESULTS}/{path}")  # the file as it is today
    same_size = today.stat().st_size == entry["bytes"]
    if not same_size or file_fingerprint(today) != entry["sha256"]:
        rust_wrong.append(path)
```

The Rust solver's manifest lists, for every result file, its path, its size and its fingerprint. For each listed file, today's size and today's fingerprint are compared with the listed ones; a file that differs in either is noted.

```
# The reference manifest: a dictionary path -> fingerprint.
reference_manifest = read_json(f"{REFERENCE_RESULTS}/manifest.json")["files"]
reference_wrong = []
for path, value in reference_manifest.items():
    today = repository_file(f"{REFERENCE_RESULTS}/{path}")
    if file_fingerprint(today) != value:
        reference_wrong.append(path)
```

The reference solver's manifest is a dictionary from each path to its fingerprint; the comparison is the same.

```
report("Rust results: fingerprints compared", len(rust_listed))
report("Rust results: files that differ", len(rust_wrong))
report("reference results: fingerprints compared", len(reference_manifest))
report("reference results: files that differ", len(reference_wrong))
report("fingerprints compared in all", len(rust_listed) + len(reference_manifest))
```

Out [16] reports 243 Rust and 339 reference fingerprints compared, 582 in all, and no file that differs. (Each folder holds one more file, its manifest, which cannot list its own fingerprint.)

```
rust_others = [n for n in relative_names(rust_files, RUST_RESULTS)
               if n != "manifest.json"]
reference_others = [n for n in relative_names(reference_files, REFERENCE_RESULTS)
                    if n != "manifest.json"]
check(rust_wrong == [] and sorted(rust_listed) == rust_others,
      f"the {len(rust_listed)} fingerprints of the Rust manifest equal today's files")
```

The files of each folder except the manifest. The check requires that no listed Rust file differs and that the manifest lists exactly the files of the folder: none is missing, none is extra.

```
manifest_check = cross_checks["reference_manifest"]
stated = f"lists {len(reference_manifest)} files with SHA-256"  # the record's words
check_reproduces(
    reference_wrong == [] and sorted(reference_manifest) == reference_others
    and manifest_check["verdict"] == "PASS" and stated in manifest_check["detail"],
    f"the {len(reference_manifest)} fingerprints of the reference manifest equal "
    "today's files",
    f"{CROSS}, check reference_manifest")
```

The same for the reference folder, together with the record's own check `reference_manifest`, whose detail must state the same number of listed files.

```
rust_repeat = determinism_checks["repeat_byte_identical"]  # the Rust second run
# The part of its detail that counts, e.g. "244 files (5857843 bytes) compared, ...".
found = re.search(r"(\d+) files \((\d+) bytes\) compared, same file set: (\w+), "
                  r"differing files: (\w+)", rust_repeat["detail"])
say(f"the record (Rust): {found.group(0)}")
rust_bytes = sum(p.stat().st_size for p in rust_files)  # the bytes of today
check_reproduces(
    rust_repeat["verdict"] == "PASS"
    and (int(found.group(1)), int(found.group(2))) == (len(rust_files), rust_bytes)
    and found.group(3) == "True" and found.group(4) == "none",
    f"a second run of the Rust solver gave the same {len(rust_files)} files "
    f"({rust_bytes} bytes)",
    f"{DETERMINISM}, check repeat_byte_identical")
```

The record's comparison of a second run of the Rust solver with the first. The regular expression picks out four parts of its detail: the number of files, the number of bytes, whether the two runs wrote the same set of files, and the differing files. `found.group(0)` is the whole matched text, printed in Out [16]: 244 files (5857843 bytes) compared, same file set: True, differing files: none. The check requires that the record compared as many files and bytes as the folder holds today, with the same file set and no differing file.

```
repeat = cross_checks["reference_repeat_byte_identical"]  # the reference second run
# The part of its detail that counts the files, e.g. "340 result files, same ...".
found = re.search(r"(\d+) result files, same file set: (\w+), differing files: (\w+)",
                  repeat["detail"])
say(f"the record (reference): {found.group(0)}")
check_reproduces(
    repeat["verdict"] == "PASS" and int(found.group(1)) == len(reference_files)
    and found.group(2) == "True" and found.group(3) == "none",
    f"a second run of the reference solver gave the same {len(reference_files)} "
    "files, byte for byte",
    f"{CROSS}, check reference_repeat_byte_identical")
```

The same for the reference solver: 340 result files, the same file set, no differing file. Out [16] ends with this PASS line and its record.

In [17], the last check.

```
figure_names = ["00d_1_float_spacing.png", "00d_2_summation_order.png",
                "00d_3_tolerances.png", "00d_4_shared_rounding.png",
                "00d_5_seeded_walks.png", "00d_6_set_orders.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "the six figure files of this notebook exist")
all_checks_passed()
```

The six figure files must exist; the last line prints ALL 22 CHECKS PASSED (notebook 00d): two checks in In [2], one in In [3], one in In [4], three in In [6], one in In [7], one in In [9], two in In [11], three in In [12], one in In [14], two in In [15], four in In [16] and one in In [17].

0.26 What we proved, what we computed, what we assumed

This chapter is mostly a map; the few results it derives are elementary, but each is derived line by line and checked by a notebook.

PROVED in this chapter (by school algebra and one-variable calculus):

- the tangent line of $y = x^2$ at $x = 1$ is $y = 2x - 1$, and it lies below the parabola because $x^2 - (2x - 1) = (x - 1)^2 \geq 0$; and $e^t e^{-t} = 1$ for every t (Section 0.10);
- the signs of the eight entries of the author's metric are $(+, +, +, +, -, -, -, -)$ at every point of the patch $0 < z < \pi/2$, for every a_4 and every $b_4 > 0$, so the signature is (4,4) (Section 0.14, Fact 1);
- the length factors are $\cot z$ for x_0, $e^{a_4} \sin^{1/6} z$ for x_1, x_2, x_3, 1 for x_4 and $b_4^{1/2} e^{-a_4} \sin^{1/6} z$ for x_5, x_6, x_7 (Fact 2); along the history $a_4 = AHx_4$, $b_4 = 1$ the extra-time factors obey $df_5/dx_4 = -AHf_5$ (exponential deflation) and the space factors $df_1/dx_4 = +AHf_1$ (exponential inflation), and $f_1 f_5 = b_4^{1/2} \sin^{1/3} z$ contains no a_4 in every history (Fact 3);
- $\det g = b_4^3 \cos^2 z$ and the volume factor $\sqrt{|\det g|} = b_4^{3/2} \cos z$, for every function a_4: the inflation and the deflation cancel in the volume, which changes in time only through b_4 (Fact 4; also by the exact symbolic checks `sqrt_det_g` and `sqrt_abs_det_g_is_cos_z`, and at $b_4 = 1$ `sqrt_det_g_is_cos_z` and `sqrt_det_g_equals_cos_z`, of the Revision reports listed at the end of Fact 4);
- the expected number of changed characters of a random 64-character hexadecimal fingerprint is $64 \cdot 15/16 = 60$ (Section 0.18);
- $1/10$ has no finite binary expansion; the computer stores 0.1 as $3602879701896397/2^{55}$ and computes $0.1 + 0.2$ as $5404319552844596/2^{54}$, which exceeds the stored 0.3 by 2^{-54}; the machine epsilon is 2^{-52}, and the gap between neighbouring stored numbers lies between $\epsilon x/2$ and ϵx; $(1 + 2^{-53}) + 2^{-53} \neq 1 + (2^{-53} + 2^{-53})$ in floating-point arithmetic; and $1/(N + 1) < \pi^2/6 - \sum_{k \leq N} 1/k^2 < 1/N$ (Section 0.22).

The chapter also states, without proof, the theorems that later chapters prove: T1, T2, Q, T3 and C1 (Section 0.2; Chapters 18 to 20) and the exact local conservation law of the U(1) charge (the total charge is constant only under the ASSUMED no-flux condition at the edge $z = \pi/2$) and the charge-conjugation matrices $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$ (Chapters 5 and 21). Their status, PROVED, is that of the Revision record, given in the ledger with the reports that verify them.

COMPUTED by the notebooks (each number in the cell named; where a Revision record is reproduced, the record file and its check):

- Notebook 00a: the installed versions (In [2] to In [4]); the determinant -2.000000000000 (In [5]); the derivative $2 \sin x \cos x$ (In [6]); 30 digits of π (In [7]); the tangent line and the two exponentials at 401 points each (In [8], In [9]).
- Notebook 00b: the names and the signs of the coordinates (In [2]; reproduces `Revision /algebra/reports/wolfram-algebra.json`, check `eta_in_author_order`); the eight entries

of the metric with b_4 (In [4]; reproduces `Revision/metric_b4/results/wolfram-metric-b4-report.json`, check `metric_equals_author_Out101`) and at $b_4 = 1$ (In [4]; reproduces `Revision/theory/reports/python-field-theory.json`, check `metric_from_vielbein_equals_SPEC`); no wrong sign at 14457 points (In [5]); $\det g = b_4^3 \cos^2 z$ (In [6]; reproduces `Revision/theory/reports/wolfram-field-theory-b4.json`, check `sqrt_det_g`) and $\det g = \cos^2 z$ at $b_4 = 1$ (In [6]; reproduces `Revision/theory/reports/wolfram-field-theory.json`, check `sqrt_det_g_is_cos_z`); $e^3 = 20.0855$ and $e^{-3} = 0.049787$ (In [7]); the table of the length factors at $z = \pi/4$, $b_4 = 1$ with the product 0.7071 (In [8]; reproduces `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `sqrt_abs_det_g_is_cos_z`); the volume factor equal to $\cos z$ within 1.0×10^{-15} at 200 points for three values of a_4 at $b_4 = 1$ (In [9]; reproduces `Revision/theory/reports/python-field-theory.json`, check `sqrt_det_g_equals_cos_z`), and equal to $2^{3/2} \cos z$ within 2.7×10^{-15} at $a_4 = 1$, $b_4 = 2$ (In [9]; reproduces `Revision/theory/reports/wolfram-field-theory-b4.json`, check `sqrt_det_g`).

- Notebook 00c: 12 of 12 checks of the charge-conjugation report (In [2]); the statements $\mathcal{C}_+ = C$, $\mathcal{C}_- = \Gamma C$ and $J = 0$ for a real field, word for word in the details of that report (In [3]; checks `charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus` and `real_fields_charge_conjugation`); the local conservation law of the charge, and the report's own statement that a constant total charge is not established (In [3]; check `u1_noether_matrix_identity`); the 51 reports of the record, all found by the search, no check FAIL (1352 PASS and 5 NOT-AVAILABLE of 1357 checks in the run of 2026-10-14; the counts follow the record), and each report's own totals equal to the counted ones (In [5]); the 49 counts of the table of `Revision/README.md`, the 3 counts quoted by `Revision/kohn_sham/reports/ks-crosscheck.json`, check `inputs_all_pass`, and the 13 of 13 quoted by `Revision/dark_sector/dirac16complex/outputs/eos-summary.json`, key `checks` (In [6]); the ledger with 11 PROVED, 3 COMPUTED, 1 ASSUMED, 2 HYPOTHESIS and 3 OPEN rows, every report in exactly one row, and only row 7 with checks that are not PASS, its five NOT-AVAILABLE comparisons (In [9]); the twelve sentences of `Revision/pairing/reports/python-pairing.json`, key `not_established` (In [11]); 54 to 64 changed characters, mean 60.03, for one-character changes (In [12]); 24 recorded fingerprints equal to today's files (In [13]).
- Notebook 00d: the forward, backward and pairwise errors of 196, 1 and 3 ulps, and the method error 9.999995×10^{-7} (In [5]); the relative difference 2.65×10^{-14} of the three sums (In [6]); the eight measured differences of the Kohn-Sham solver below their tolerances with the margins 4.7 to 7692 (In [7]; reproduces `Revision/kohn_sham/reports/ks-rust-determinism.json`, its eight comparison checks named in Section 0.22, from `refined_ground_energies` to `refined_heat_capacity` without the negative control `refined_mermin_root_path`); the negative control with a hidden error up to 8.267×10^{-10} in units of m (In [9]; check `refined_mermin_root_path` of the same report); 12 distinct set orders for 12 hash seeds (In [12]); no CR LF line end in the 244 Rust and the 340 reference result files (In [15]; checks `outputs_lf_only` and `reference_outputs_lf_only`); 582 manifest fingerprints equal to today's files, and the record's repeat runs of both solvers byte-identical (In [16]; checks `reference_manifest`, `repeat_byte_identical` and `reference_repeat_byte_identical`).

ASSUMED (used, not derived here):

- the conventions of Section 0.5: the author's names and roles of the coordinates, the signs η, units with the speed of light and Planck's constant equal to 1;

- the author's metric itself (Section 0.14), and the history $a_4 = AHx_4$ where it is used, a prescribed background (`Revision/field_equations_a4/reports/ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`);
- the Z2 mirror at the edge $z = \pi/2$ of the hidden direction, in T2 and T3;
- the Kohn-Sham approximation (the stationary-slice ansatz, the Hartree term and the exchange of a uniform gas without correlation, mean-field states, the chosen condition at the tip), in T3 and in every Kohn-Sham result (rows 8, 9, 12 and 13 of the ledger, Section 0.18);
- school facts used without proof: the determinant of a diagonal matrix is the product of its diagonal entries (proved in Chapter 1); every whole number is a product of primes in exactly one way; Euler's sum $\pi^2/6$; the rules of the floating-point arithmetic (53 binary digits, round half to even), which are a property of the computer;
- that a change of a single byte changes the sha256 fingerprint completely, beyond the 115 cases that Notebook 00c measures, and that nobody can construct two different files with the same sha256 fingerprint (both believed, not proved; Section 0.18).

HYPOTHESIS and OPEN. The chapter adds no hypothesis of its own; it records the two HYPOTHESIS rows of the ledger (a time-varying dark sector from the fields, investigated but not established; a partner universe of opposite charge) and the three OPEN rows (that the total charge of one universe stays constant, which needs the ASSUMED no-flux condition at the brane; that the big bang creates universes in pairs, which the equations do not prove; and what produces the excess of matter over antimatter, which the theory as built does not produce). The PROVED row 2 names one OPEN part in its note: for a self-coupling $\lambda \neq 0$ the operator form of the energy-momentum tensor for the field on a whole slice of space is OPEN (it is proved only in a finite model of one set of 16 modes), and the field-equations record of Chapter 12 ASSUMES it.

0.27 Exercises

Exercise 1. Give each statement one of the five labels and one sentence of reason. (a) The author's metric has the signature (4,4). (b) The ground-state energies of the canonical and the refined run of the Revision record's Kohn-Sham solver differ by at most 1.901×10^{-12} (relative). (c) The metric function grows in proportion to the time, $a_4 = AHx_4$, in the Kohn-Sham chapters. (d) Our universe has a partner universe of opposite charge. (e) At the big bang, a pair of universes of masses $+M$ and $-M$ was created.

Answer. (a) PROVED: Fact 1 of Section 0.14 derives the signs $(+,+,+,+,-,-,-,-)$ at every point of the patch by school algebra, and the Wolfram verifier checks them (`eta_in_author_order`). (The metric itself is ASSUMED, as the author's input; given the metric, its signature is proved.) (b) COMPUTED: it is a number measured by comparing two numerical runs (`Revision/kohn_sham/reports/ks-rust-determinism.json`, check `refined_ground_energies`), reproducible but not exact. (c) ASSUMED: it is a prescribed background; the computed Kohn-Sham states cannot be its source (row 10 of the ledger). (d) HYPOTHESIS: T1 proves that every solution with mass $+m$ has an image with mass $-m$ and opposite charge, but not that such an image exists alongside our universe (row 17). (e) OPEN as a question of this book: the author proposed it, but no equation of the record describes the creation of anything, so it is not proved (row 19).

Exercise 2. Notebook 00a checks that Python is version 3.12 or newer by comparing the pairs $(3, 12)$ and (major, minor). Would the pair $(4, 0)$ pass? Would $(3, 9)$?

Answer. Pairs are compared like words in a dictionary: first the first numbers, and only when they are equal the second numbers. $(4, 0)$: the first numbers are 4 and 3, and 4 is larger, so $(4, 0) \geq (3, 12)$ and the check passes (the second numbers are not compared). $(3, 9)$: the first numbers are equal, so the second numbers decide; 9 is smaller than 12, so $(3, 9) < (3, 12)$ and the check fails. (Comparing the versions as decimal numbers would be wrong: 3.9 is larger than 3.12 as a decimal number, but version 9 comes before version 12.)

Exercise 3. Compute the eight length factors of the author's metric at the hidden coordinate $z = \pi/6$, at $a_4 = \ln 2$ (so that $e^{a_4} = 2$) and $b_4 = 1$, and show that their product is $\cos(\pi/6) = \sqrt{3}/2$.

Answer. $\sin(\pi/6) = 1/2$, so $\sin^{1/6}(\pi/6) = (1/2)^{1/6} = 2^{-1/6}$; $\cot(\pi/6) = \cos(\pi/6)/\sin(\pi/6) = (\sqrt{3}/2)/(1/2) = \sqrt{3}$. By Fact 2 of Section 0.14 with $b_4 = 1$: $f_0 = \sqrt{3} = 1.7321$; $f_1 = f_2 = f_3 = 2 \cdot 2^{-1/6} = 2^{5/6} = 1.7818$; $f_4 = 1$; $f_5 = f_6 = f_7 = \frac{1}{2} \cdot 2^{-1/6} = 2^{-7/6} = 0.4454$. The product is $\sqrt{3} \cdot (2^{5/6})^3 \cdot 1 \cdot (2^{-7/6})^3 = 2^{5/2} \cdot 2^{-7/2} \cdot \sqrt{3} = 2^{-1}\sqrt{3} = \sqrt{3}/2$ (power rule, then the law of exponents $2^{5/2-7/2} = 2^{-1}$), which is $\cos(\pi/6)$, as Fact 4 says for every a_4.

Exercise 4. Along the history $a_4 = AHx_4$ with $A = 1$, $H = 1$ and $b_4 = 1$, after how much time x_4 has the length factor of an extra time fallen to half its value at $x_4 = 0$? After how much time has it fallen to one thousandth? What has the factor of ordinary space done in the same time?

Answer. By Fact 3, $f_5(x_4) = e^{-x_4} \sin^{1/6} z$, so $f_5(x_4)/f_5(0) = e^{-x_4}$. Half: $e^{-x_4} = 1/2$, so $-x_4 = \ln(1/2) = -\ln 2$ (take the natural logarithm of both sides), and $x_4 = \ln 2 = 0.6931$. One thousandth: $x_4 = \ln 1000 = 6.9078$. In the same times the factor of ordinary space, $e^{x_4} \sin^{1/6} z$, has grown by the factors 2 and 1000, since $e^{x_4} = 1/e^{-x_4}$. Every interval of length $\ln 2$ halves the extra-time factor again: this is what exponential deflation means.

Exercise 5. Is $0.5 + 0.25 == 0.75$ true in Python? Explain with the stored fractions why the answer differs from that for $0.1 + 0.2 == 0.3$.

Answer. True. The numbers $0.5 = 1/2$, $0.25 = 1/4$ and $0.75 = 3/4$ are binary fractions with a few digits (binary 0.1, 0.01 and 0.11), so each is stored exactly, without rounding. Their exact sum, $1/2 + 1/4 = 3/4$, is again a stored number, so the addition does not round either, and the result is exactly the stored 0.75. In Section 0.22, by contrast, 0.1, 0.2 and 0.3 are all rounded when they are stored, and the sum is rounded once more; the roundings do not cancel, and the result misses the stored 0.3 by 2^{-54}.

Exercise 6. Above $2^{53} = 9007199254740992$ the gap between neighbouring stored numbers is 2. In floating-point arithmetic, compute $(2^{53} + 1) + 1$ and $2^{53} + (1 + 1)$.

Answer. 2^{53} is stored with $m = 2^{52}$ and the power 2^1, so the stored numbers just above it are $2^{53} + 2$, $2^{53} + 4$, and so on, with $m = 2^{52} + 1$, $2^{52} + 2$, and so on. First order: $2^{53} + 1$ lies exactly halfway between 2^{53} (with the even $m = 2^{52}$) and $2^{53} + 2$ (with the odd $m = 2^{52} + 1$); round half to even gives 2^{53}. Adding 1 again gives the same halfway case and again $2^{53} = 9007199254740992$. Second order: $1 + 1 = 2$ exactly, and $2^{53} + 2 = 9007199254740994$ is a stored number. The two orders differ by 2, the gap at that size: the same three numbers added in two orders give two different results, as in Section 0.22.

Exercise 7. Notebook 00c compares some fingerprints of which the report recorded only the first 16 hexadecimal characters. If fingerprints behave like random strings, what is the probability that a

different file has a fingerprint that begins with the same 16 characters?

Answer. Each of the 16 characters agrees with probability 1/16, and for a random string the 16 agreements are independent, so the probability is $(1/16)^{16} = 16^{-16} = (2^4)^{-16} = 2^{-64}$ (the power rule). Since $2^{10} = 1024 \approx 10^3$, $2^{-64} = 2^{-4} \cdot 2^{-60} \approx \frac{1}{16} \cdot 10^{-18}$; exactly, $2^{-64} = 5.42 \times 10^{-20}$. Even a shortened fingerprint ties a report to its file beyond any reasonable doubt.

Exercise 8. Theorem T1 is PROVED, and it says that a universe with mass $+m$ and its T1 image with mass $-m$ have opposite charges, so that together they have the total charge zero. (a) Why does this not prove that the big bang creates universes in pairs? (b) Name one calculation that would be needed in addition.

Answer. (a) T1 is a map between solution sets: if a configuration of the field is a solution of the equations with (m, λ), then its image under Γ is a solution of the equations with $(-m, -\lambda)$. It says which configurations are allowed, not which configurations occur, when, or how often; a single universe with mass $+m$ is an equally valid solution without its partner. The zero total holds for classical fields; at the quantum level the image is the same quantum system written in other variables, and two independently quantised universes do not cancel (Q). The same situation is familiar from ordinary physics: the conservation of energy, momentum and charge allows two energetic photons to turn into an electron and a positron, but whether this happens, and how often, is computed from the dynamics of the particles, not from the conservation laws. (b) A dynamical calculation: for example a quantum amplitude, or a probability per unit time, for a transition from a state without universes to a state with the pair, or a solution of the coupled equations of the fields and of gravity that evolves from an initial state into the pair. None of these is derived in the Revision record, which is why the ledger labels the statement OPEN (`Revision/pairing/reports/python-pairing.json`, key `not_established`, sentences 1 and 2).

1 Mathematics from zero I: numbers, complex numbers, vectors, matrices, indices, determinants, the generalized Kronecker delta

This chapter builds, from school algebra, every piece of algebra that the rest of the course uses: exact and rounded numbers, powers and the exponential function, complex numbers, vectors and matrices, determinants, index notation with the summation convention, eigenvalues and the signature of a metric, and the generalized Kronecker delta with which the field equations of gravity are written. Every idea is put to work in a Jupyter notebook, and every notebook is applied to the objects of the author's theory as soon as possible.

1.1 What this chapter is for

Why so much algebra. The theory of this course lives in a universe with eight directions. The author names them $x_0, \ldots, x_7$: a hidden space direction x_0, the three directions x_1, x_2, x_3 of ordinary space, the time x_4, and three further time-like directions x_5, x_6, x_7, called the **extra times**. The geometry of this universe is described by the author's **metric**, a list of eight expressions, one for each direction, which says how long a small step along that direction is (the word is explained fully in Sections 1.5 and 1.29):

$$g = \mathrm{diag}(\,\cot^2 z,\ e^{2a_4}s,\ e^{2a_4}s,\ e^{2a_4}s,\ -1,\ -b_4e^{-2a_4}s,\ -b_4e^{-2a_4}s,\ -b_4e^{-2a_4}s),$$

with $s = \sin^{1/3} z$ and $z = 6Hx_0$. Here H is a positive constant of the author's, z lies between 0 and $\pi/2$, and a_4 and b_4 are functions of the time x_4, b_4 positive (b_4 is the author's change of 2026-10-14; $b_4 = 1$ gives the metric used before, and the backgrounds of the book take $b_4 = 1$, as the Revision record does); as a_4 grows, the entries of space grow like e^{2a_4} and the entries of the three extra times shrink like e^{-2a_4}: space inflates and the extra times **deflate exponentially**. To read this one line you need powers, roots, the exponential function, the trigonometric functions and the idea of a list of numbers with a sign attached to each entry. The fields of the theory have sixteen complex components at every point, and they are acted on by eight real 16×16 matrices, the **gamma matrices**. To compute with them you need complex numbers, matrices and their products, determinants, eigenvalues, and a compact notation, index notation, in which one short line stands for dozens of equations. Finally the field equations of gravity in eight dimensions (Chapter 11) are written with the **generalized Kronecker delta**, a determinant of zeros and ones. This chapter teaches all of it.

The seven notebooks. The chapter has seven worked examples, each a complete Jupyter notebook, taken in this order:

notebook	topic	sections
01f	whole numbers, fractions, real and floating-point numbers, powers, exponential, logarithm	Sections 1.2 to 1.9
01b	complex numbers, Euler's formula, rotations, boosts, oscillation and growth	Sections 1.10 to 1.17
01a	vectors, matrices, permutations, determinants, the gamma matrices and the metric	Sections 1.18 to 1.25
01e	index notation, the summation convention, the frame metric, the Clifford relation	Sections 1.26 to 1.33
01g	eigenvalues, eigenvectors, the signature (4,4), Sylvester's law	Sections 1.34 to 1.41
01c	the generalized Kronecker delta, its proof, its counts, its contractions	Sections 1.42 to 1.48
01d	binary numbers, a pseudo-random generator, the record's tests of the generalized delta reproduced exactly	Sections 1.49 to 1.54

The letters were given to the notebooks when they were made; the chapter takes them in the order in which the ideas build on each other, so that every notion is introduced before a notebook uses it.

Each notebook appears in three parts: a section "How to run Notebook 01x", which gives the complete instructions for running it on Windows, macOS or Linux; a section "Notebook 01x: complete text", which prints every cell, everything it printed and every figure it drew; and a section "Line-by-line walk-through of Notebook 01x", which explains every line of every code cell. The first code cell of every notebook, the **set-up cell**, is the same in all notebooks except for the notebook's name; it is explained line by line once, in Section 1.9, and the later walk-throughs refer back to that explanation.

Counting from 1 and from 0. In mathematics the rows and columns of a table are usually numbered $1, 2, 3, \ldots$; the author numbers the coordinates from 0, x_0 to x_7. The programming language Python numbers the entries of a list $0, 1, 2, \ldots$. So the coordinate x_k is entry k of a Python list (x_0 is entry 0 and x_7 entry 7), while the entry in row 1 and column 1 of a matrix `A` is `A[0, 0]`. Every walk-through says which numbering a line uses.

The labels of every statement. Every result in this chapter carries one of the labels of the book: PROVED (an exact derivation is written out here, line by line, and a notebook checks it), COMPUTED (a number found by a notebook, with the cell that prints it), ASSUMED (a standard fact used without proof here, said so where it is used). When a notebook reproduces a number of the **Revision record** (the folder Revision of the repository, in which every computation of the author's theory is done and checked), the text names the record file and the check, exactly as the notebook's PASS line does. The labels HYPOTHESIS (a scenario that is stated, not derived) and OPEN (a question that no record answers yet), used later in the book, do not occur in this chapter. Nothing in this chapter concerns pair creation or matter and antimatter; those questions are treated, with the exact scope of what is proved, in Chapters 18 to 21.

1.2 Whole numbers and fractions, exactly

Whole numbers. The **whole numbers** (or **integers**) are $\ldots, -2, -1, 0, 1, 2, \ldots$. Python computes with them exactly, however many digits they have: $2^{100} = 1267650600228229401496703205376$ has 31 digits, and every one of them is right. Another large whole number that the course meets is

$$16! = 1 \cdot 2 \cdot 3 \cdots 16 = 20922789888000,$$

the number of terms of the determinant of a 16×16 matrix (Section 1.21). The symbol $n!$, read "n factorial", is the product of the whole numbers from 1 to n, with $0! = 1$.

Fractions. A **fraction** (or **rational number**) is a quotient p/q of two whole numbers with $q \neq 0$. The same fraction has many spellings, $1/2 = 2/4 = 3/6$, because multiplying numerator and denominator by the same number does not change the quotient. The spelling is in **lowest terms** when p and q have no common factor larger than 1. The **greatest common divisor** $\gcd(p, q)$ is the largest whole number that divides both; dividing p and q by it gives the lowest terms. For 84/126:

$$84 = 2 \cdot 42, \qquad 126 = 3 \cdot 42, \qquad \text{so} \qquad \frac{84}{126} = \frac{84/42}{126/42} = \frac{2}{3}$$

(the first two equations factor out 42, which is $\gcd(84, 126)$ because 2 and 3 have no common factor; the third divides numerator and denominator by 42). Two fractions are added by bringing them to

a common denominator:

$$\frac{1}{3}+\frac{1}{6}=\frac{2}{6}+\frac{1}{6}=\frac{3}{6}=\frac{1}{2}$$

(first 1/3 is written with the denominator 6 by multiplying numerator and denominator by 2; then the numerators are added over the common denominator; then the common factor 3 of 3 and 6 is cancelled). Python's module `fractions` computes with fractions exactly in this way, and Notebook 01f, In [2], checks these three examples.

Long division. The decimal digits of a fraction p/q with $0<p<q$ are produced by the school method of long division. Start with the **remainder** $r=p$. At every step multiply the remainder by 10; the whole part of the division of $10r$ by q is the next digit, and what is left over is the new remainder. For 1/7:

$$10=1\cdot 7+3,\quad 30=4\cdot 7+2,\quad 20=2\cdot 7+6,\quad 60=8\cdot 7+4,\quad 40=5\cdot 7+5,\quad 50=7\cdot 7+1.$$

The digits are 1, 4, 2, 8, 5, 7, and the last remainder is 1, the remainder we started with. From here everything repeats, so $1/7=0.142857142857\ldots$, written 0.(142857): the block in brackets repeats for ever. The length of the repeating block is the **period**; for 1/7 it is 6.

Theorem (PROVED). The decimal digits of a fraction p/q either end or, from some place on, repeat a block of at most $q-1$ digits. *Proof.* Every remainder is one of the numbers $0,1,\ldots,q-1$, because a remainder after division by q is smaller than q. If a remainder is 0, the division ends: every further digit is 0. Otherwise every remainder is one of the $q-1$ numbers $1,\ldots,q-1$. Among the first q remainders two must therefore be equal (there are only $q-1$ different values for q remainders). Each step computes the next digit and remainder from the present remainder alone, so as soon as a remainder comes back, the digits from there on repeat the digits after its first appearance. The block between two equal remainders is at most $q-1$ steps long.

From the digits back to the fraction (PROVED). Let $x=0.(c_1\ldots c_m)$ be a repeating block of m digits, and let C be the whole number with the digits $c_1\ldots c_m$. Then

$$10^m x=c_1\ldots c_m.(c_1\ldots c_m)=C+x$$

(multiplying by 10^m moves the decimal point m places to the right, so the first block stands before the point and the same repeating digits follow it),

$$10^m x-x=C,\qquad x=\frac{C}{10^m-1}$$

(subtract x from both sides; then take out the factor x and divide by 10^m-1). For example $0.(142857)=142857/999999=1/7$, because $7\cdot 142857=999999$. If k digits $d_1\ldots d_k$ (the whole number D) come before the repeating block, they are worth $D/10^k$, and the block is shifted k places further right, which divides its value by 10^k:

$$0.d_1\ldots d_k(c_1\ldots c_m)=\frac{D}{10^k}+\frac{C}{10^k(10^m-1)}.$$

For example $5/12 = 0.41(6)$: $D = 41$, $k = 2$, $C = 6$, $m = 1$, and $41/100 + 6/(100 \cdot 9) = 369/900 + 6/900 = 375/900 = 5/12$. Notebook 01f, In [3], writes long division as a Python function and checks this formula on six fractions.

Which fractions end (PROVED, using one ASSUMED fact). A fraction p/q in lowest terms has a decimal expansion that ends exactly when q has no prime factor other than 2 and 5, that is $q = 2^a 5^b$. (A **prime number** is a whole number larger than 1 that only 1 and itself divide.) *Proof.* If $q = 2^a 5^b$ and k is the larger of a and b, then $10^k = 2^k 5^k$ is a multiple of q, say $10^k = qt$, and $p/q = pt/10^k$: a whole number divided by 10^k, which has at most k digits after the point. Conversely, if the digits end after k places, then $p/q = N/10^k$ for a whole number N, so $p \cdot 10^k = Nq$; q divides $p \cdot 10^k$ and has no common factor with p, so it divides $10^k = 2^k 5^k$, and its prime factors are among 2 and 5. The last step uses the uniqueness of the factorisation of a whole number into primes, a standard theorem of arithmetic that we use without proof (ASSUMED). Notebook 01f, In [4], computes the period of $1/q$ for every q from 2 to 300 and checks the theorem and the bound $q - 1$; 23 denominators reach the full period $q - 1$, the first of them 7, 17, 19, 23, 29, 47, 59, 61, 97 (COMPUTED).

1.3 Real numbers: the square root of 2 is not a fraction

The number line contains numbers that are not fractions. The most famous is $\sqrt{2}$, the positive number whose square is 2, the length of the diagonal of a square with side 1.

Theorem (PROVED). No fraction p/q has $(p/q)^2 = 2$. *Proof by contradiction.* Suppose p/q is in lowest terms and $p^2/q^2 = 2$. Then:

- $p^2 = 2q^2$ (multiply both sides by q^2), so p^2 is even;
- p is even, because the square of an odd number is odd: $(2k+1)^2 = 4k^2+4k+1 = 2(2k^2+2k)+1$ (multiply out; then take out the factor 2 from the first two terms); so $p = 2r$ for a whole number r;
- $(2r)^2 = 2q^2$, that is $4r^2 = 2q^2$, that is $q^2 = 2r^2$ (insert $p = 2r$; then divide by 2), so q^2 is even and, by the same argument, q is even;
- so 2 divides both p and q, and p/q was not in lowest terms after all.

The assumption led to a contradiction, so it is false: $\sqrt{2}$ is not a fraction. A number that is not a fraction is called **irrational**; its decimal digits never end and never repeat (by the formula of Section 1.2, a number whose digits end or repeat is a fraction). The **real numbers** are all the numbers of the number line, fractions and irrational numbers together.

Fractions that come close (PROVED). Start with $1/1$ and make new fractions by the rule $p' = p + 2q$, $q' = p + q$: this gives $3/2$, $7/5$, $17/12$, $41/29$, $99/70$, and so on. For each of them $p^2 - 2q^2$ is $+1$ or -1:

$$p'^2 - 2q'^2 = (p + 2q)^2 - 2(p + q)^2 = p^2 + 4pq + 4q^2 - 2p^2 - 4pq - 2q^2 = -(p^2 - 2q^2)$$

(insert the rule; multiply out both squares, $(u+v)^2 = u^2+2uv+v^2$; collect the terms: $4pq-4pq = 0$, $p^2 - 2p^2 = -p^2$, $4q^2 - 2q^2 = 2q^2$). Every step changes the sign, and $1^2 - 2 \cdot 1^2 = -1$, so the values are $-1, +1, -1, \ldots$. How far is p/q from $\sqrt{2}$? Multiply the difference by $p + q\sqrt{2}$:

$$\Big(\frac{p}{q}-\sqrt{2}\Big)(p+q\sqrt{2}) = \frac{p^2+pq\sqrt{2}-pq\sqrt{2}-2q^2}{q} = \frac{p^2-2q^2}{q}, \qquad \text{so} \qquad \frac{p}{q}-\sqrt{2} = \frac{p^2-2q^2}{q(p+q\sqrt{2})}$$

(multiply out the product; the middle terms cancel; then divide by $p+q\sqrt{2}$). With $p^2-2q^2=\pm1$ and $p \approx q\sqrt{2}$ the size of the error is about $1/(q\cdot 2q\sqrt{2}) = 1/(2\sqrt{2}\,q^2) \approx 0.35355/q^2$. Notebook 01f, In [5], computes twelve of these fractions with 50-digit arithmetic: for $19601/13860$ the error is 1.84×10^{-9}, and the error times q^2 is 0.353553 (COMPUTED). It also checks with exact whole numbers that $p^2=2q^2$ has no solution with $q \le 10000$.

Bisection. A real number can be pinned down by fractions alone. Because $1^2<2<2^2$, $\sqrt{2}$ lies between 1 and 2. Take the middle m of the interval: if $m^2<2$, the root lies in the right half, otherwise in the left half. Each step halves the width of the interval while both ends stay exact fractions; after 60 steps the width is $2^{-60} \approx 8.7\times10^{-19}$. This is **bisection**. Notebook 01f, In [6], finds $1.4142135623730950483 < \sqrt{2} < 1.4142135623730950492$ in this way (COMPUTED).

1.4 Floating-point numbers, rounding and cancellation

How a computer stores a real number. A computer does not store real numbers with infinitely many digits. It stores a **floating-point number**: a whole number m of at most 53 binary digits (**bits**, the digits 0 and 1 of the base-2 system) times a power of two, $m\cdot 2^k$. Python's `float` and numpy's arrays use these numbers. A fraction whose denominator is a power of 2, such as $0.5=1/2$ or $0.375=3/8$, is stored exactly. A fraction such as $0.1=1/10$, whose denominator contains the prime factor 5, has an endless expansion in base 2 (by the argument of Section 1.2 with 2 in place of 10), so it is **rounded** to the nearest floating-point number. Notebook 01f, In [7], reads out the stored value: 0.1 is stored as 3602879701896397/36028797018963968, a fraction with the denominator 2^{55}, slightly larger than $1/10$. That is why $0.1+0.2$ prints as 0.30000000000000004 and is not equal to the stored 0.3.

Machine epsilon. The step from 1 to the next floating-point number is

$$\varepsilon = 2^{-52} \approx 2.220446049250313\times10^{-16},$$

called **machine epsilon**. So $1+\varepsilon$ is stored, but $1+\varepsilon/2$ lies halfway and is rounded back to 1. Above $2^{53}=9007199254740992$ the step between neighbouring floating-point numbers is 2, so the odd number $2^{53}+1$ cannot be stored at all: it is rounded to 2^{53}.

The step at every size (PROVED). Write a positive number as $x=f\cdot 2^e$ with $\frac{1}{2}\le f<1$ (every positive number can be written so, by choosing the power of two). The 53 bits of m then cover the factor f, whose last bit is worth 2^{-53}; so the step to the next floating-point number is $2^{-53}\cdot 2^e = 2^{e-53}$. Divided by x:

$$\frac{2^{e-53}}{f\cdot 2^e} = \frac{2^{-53}}{f}, \qquad \text{and} \qquad \tfrac{1}{2}\le f<1 \;\Rightarrow\; 2^{-53} < \frac{2^{-53}}{f} \le 2^{-52}$$

(cancel 2^e; then divide the inequalities for f into 2^{-53}, which reverses them). So the **relative** step always lies between $\varepsilon/2$ and ε: floating-point numbers have about 16 significant decimal digits at

every size, and rounding a number to the nearest one changes it by at most $\varepsilon/2$ of its size. Notebook 01f, In [7], checks the formula 2^{e-53} at 2301 numbers from 10^{-3} to 10^{20}, and In [8] draws the step.

Cancellation. The function $f(x) = (1-\cos x)/x^2$ tends to $\frac{1}{2}$ as x tends to 0 (because $\cos x \approx 1-x^2/2$ for small x, a fact of calculus, the Taylor series of the cosine, ASSUMED here and confirmed by sympy in Notebook 01f, In [9]). Computed as written, it fails for small x. The computer's value of $\cos x$ is a floating-point number near 1, so it is wrong by up to the spacing of the floating-point numbers there, of the size of ε; the subtraction $1-\cos x$ keeps this absolute error, while the true value $x^2/2$ is tiny. The relative error of the result is therefore of the size

$$\frac{\varepsilon}{x^2/2} = \frac{2\varepsilon}{x^2}$$

(absolute error divided by the size of the true value; the dashed line of Figure 01f.4). For $x = 10^{-8}$ the true value of $\cos x$ is $1 - 5\times10^{-17}$, which is nearer to 1 than to the floating-point number just below 1 (which is $1 - 2^{-53} \approx 1 - 1.1\times10^{-16}$), so $\cos x$ is rounded to exactly 1 and the formula returns 0: every digit is lost. This loss of digits when two nearly equal numbers are subtracted is called **cancellation**.

The cure: rewrite the formula (PROVED). From the addition theorem $\cos(2y) = \cos^2 y - \sin^2 y$ and $\cos^2 y = 1 - \sin^2 y$:

$$\cos(2y) = 1 - \sin^2 y - \sin^2 y = 1 - 2\sin^2 y, \qquad \text{so with } y = x/2: \qquad 1 - \cos x = 2\sin^2(x/2)$$

(insert the second identity into the first; collect; then put $y = x/2$ and move terms across). Hence $f(x) = 2\sin^2(x/2)/x^2$, a formula without a subtraction. Notebook 01f, In [9], compares both formulas with 50-digit arithmetic at 361 values of x from 10^{-9} to 1: the rewritten formula has the largest relative error 4.4×10^{-16}, the formula as written 1.0, that is 100 per cent (COMPUTED). Whenever a later notebook of the course rewrites a formula before computing it, this is the reason.

1.5 Powers, roots, the exponential function and the logarithm

The laws of powers. For positive numbers a, b and any real numbers p, q:

$$a^p a^q = a^{p+q}, \qquad (a^p)^q = a^{pq}, \qquad (ab)^p = a^p b^p, \qquad a^{-p} = \frac{1}{a^p}, \qquad \sqrt{a} = a^{1/2}, \qquad a^{1/n} = \sqrt[n]{a}.$$

These are school algebra (ASSUMED here; Notebook 01f, In [10], confirms them with sympy for symbols declared positive). They need positive bases. For a negative number the law $(a^p)^q = a^{pq}$ can fail: $\sqrt{(-2)^2} = \sqrt{4} = 2$, not -2. In general $\sqrt{y^2} = |y|$, the **absolute value** of y (y without its sign).

A worked example (PROVED). For $s > 0$ and real a:

$$\frac{1}{\sqrt{s^{1/3}/e^{2a}}} = \Big(\frac{s^{1/3}}{e^{2a}}\Big)^{-1/2} = \frac{(s^{1/3})^{-1/2}}{(e^{2a})^{-1/2}} = \frac{s^{-1/6}}{e^{-a}} = \frac{e^a}{s^{1/6}}$$

(the root is the power $1/2$ and one over it the power $-1/2$; the power of a quotient is the quotient of the powers; $(a^p)^q = a^{pq}$ twice, with $\frac{1}{3}\cdot(-\frac{1}{2}) = -\frac{1}{6}$ and $2\cdot(-\frac{1}{2}) = -1$; one over e^{-a} is e^a and $s^{-1/6}$ is one over $s^{1/6}$).

The length factors of the author's metric (PROVED). The metric g of Section 1.1 is **diagonal**: it has one entry $g_{\mu\mu}$ for each direction x_μ ($\mu = 0,\dots,7$), and the squared length of a small step dx_μ along that one direction is $g_{\mu\mu}\,dx_\mu^2$. A negative entry marks a **time-like** direction (x_4 to x_7), a positive entry a **space-like** one (x_0, x_1, x_2, x_3). The **length factor** of a direction is $\sqrt{|g_{\mu\mu}|}$: the length of the step is the factor times $|dx_\mu|$. For $0 < z < \pi/2$ both $\sin z$ and $\cos z$ are positive, so the laws of powers apply and give

$$\sqrt{\cot^2 z} = \cot z, \qquad \sqrt{e^{2a_4}\sin^{1/3} z} = e^{a_4}\sin^{1/6} z, \qquad \sqrt{1} = 1,$$
$$\sqrt{b_4 e^{-2a_4}\sin^{1/3} z} = b_4^{1/2} e^{-a_4}\sin^{1/6} z$$

(the root of a product is the product of the roots; $\sqrt{e^{2a_4}} = e^{a_4}$ and $\sqrt{\sin^{1/3} z} = \sin^{1/6} z$ by $(a^p)^q = a^{pq}$; $\sqrt{b_4} = b_4^{1/2}$; $\cot z > 0$). The hidden direction has the factor $\cot z$; the three space directions have $e^{a_4}\sin^{1/6} z$, which grows as a_4 grows; the time has 1; the three extra times have $b_4^{1/2}e^{-a_4}\sin^{1/6} z$, which shrinks exponentially when b_4 is constant. The product of all eight factors is

$$\cot z\cdot(e^{a_4}\sin^{1/6} z)^3\cdot 1\cdot(b_4^{1/2}e^{-a_4}\sin^{1/6} z)^3 = b_4^{3/2}\,e^{3a_4-3a_4}\,\sin^{6/6} z\,\cot z = b_4^{3/2}\sin z\cot z = b_4^{3/2}\cos z$$

(collect the powers of b_4, of e and of $\sin z$ with $a^p a^q = a^{p+q}$: $3\cdot\frac{1}{2} = \frac{3}{2}$, $3a_4 - 3a_4 = 0$ and $6\cdot\frac{1}{6} = 1$; then $\cot z = \cos z/\sin z$). The growth of space and the deflation of the extra times cancel exactly in the product, for every value of a_4; at $b_4 = 1$ the product is $\cos z$. The Revision record stores this product as `sqrtAbsDetG` $= b_4^{3/2}\sin z\cot z$ in `Revision/gkd_lovelock/results/curvature.json` and checks it in `python-lovelock-report.json`, check `sqrt_abs_det_g`; Notebook 01f, In [11], reads the metric from the record and reproduces both. (Section 1.21 shows that this product is the square root of the size of the determinant of g.)

The number e. The number $e = 2.71828\dots$ is the limit of $(1 + 1/n)^n$ for ever larger n: the growth of one unit of money in a year at the interest rate of 100 per cent, paid in n equal parts. How fast is the limit reached? With the natural logarithm ln (defined below) and the series $\ln(1+u) = u - u^2/2 + u^3/3 - \dots$ for small u (a Taylor series of calculus, ASSUMED here):

$$\ln\Big(1+\frac{1}{n}\Big)^n = n\ln\Big(1+\frac{1}{n}\Big) = n\Big(\frac{1}{n} - \frac{1}{2n^2} + \dots\Big) = 1 - \frac{1}{2n} + \dots$$

(the logarithm of a power is the power times the logarithm; insert the series with $u = 1/n$; multiply out), so

$$\Big(1+\frac{1}{n}\Big)^n = e^{1-1/(2n)+\dots} = e\cdot e^{-1/(2n)+\dots} \approx e\Big(1-\frac{1}{2n}\Big), \qquad e - \Big(1+\frac{1}{n}\Big)^n \approx \frac{e}{2n}$$

(undo the logarithm with the exponential; split the power with $a^{p+q} = a^p a^q$; use $e^{-u} \approx 1 - u$ for small u; subtract from e). Notebook 01f, In [12], confirms the law $e/(2n)$ within 1 per cent for every

$n = 10^3, \ldots, 10^{16}$ with 50-digit arithmetic. With floating-point numbers the limit breaks down: $1 + 1/n$ is rounded by up to $\varepsilon/2$ of its size, and raising to the power n multiplies this relative error by about n (because $(1+\delta)^n \approx 1 + n\delta$ for small δ). For $n = 10^9$ the computed error is 2.2×10^{-7} instead of 1.36×10^{-9}, and for $n = 10^{16}$ the number $1 + 10^{-16}$ is rounded to exactly 1, so the result is $1^n = 1$ and the error is $e - 1 = 1.7$ (COMPUTED, Notebook 01f, In [12]; Figure 01f.5).

The exponential function and the logarithm. The **exponential function** e^t is defined for every real t, is always positive and grows; its inverse is the **natural logarithm** $\ln x$, defined for $x > 0$: $\ln(e^t) = t$ and $e^{\ln x} = x$. The logarithm turns products into sums (PROVED):

$$e^{\ln x + \ln y} = e^{\ln x}\, e^{\ln y} = xy \qquad \Rightarrow \qquad \ln(xy) = \ln x + \ln y$$

(the law $a^{p+q} = a^p a^q$ with $a = e$; then take the logarithm of both sides). For the author's metric this means: when a_4 grows by $\ln 2 \approx 0.693$ (and b_4 stays constant), the space factor e^{a_4} doubles and the extra-time factor halves,

$$e^{-(a_4 + \ln 2)} = e^{-a_4}\, e^{-\ln 2} = e^{-a_4} \cdot \frac{1}{2}$$

(the law of powers; then $e^{-\ln 2} = 1/e^{\ln 2} = 1/2$). A growth of a_4 by 1 multiplies e^{a_4} by e, one **e-fold**. To shrink the extra times by a factor 1000, a_4 must grow by $\ln 1000 = 6.9078$ (COMPUTED, Notebook 01f, In [12]). On a **logarithmic axis**, on which each equal step multiplies by the same factor, both e^{a_4} and e^{-a_4} are straight lines, one rising and one falling equally fast (Figure 01f.6).

1.6 Example: numbers, rounding and the length factors of the author's metric

Notebook 01f puts Sections 1.2 to 1.5 to work. It computes exactly with whole numbers and fractions; writes long division as a program and plots the periods of $1/q$; proves and checks that $\sqrt{2}$ is not a fraction and pins it down by bisection and by the fractions p/q with $p^2 - 2q^2 = \pm 1$; shows how floating-point numbers are spaced, why $0.1 + 0.2 \neq 0.3$ in a computer, and how cancellation destroys a formula and how rewriting cures it; checks the laws of powers and applies them to the author's metric as stored in the Revision record; and builds e as a limit and draws the growth e^{a_4} of space and the exponential deflation e^{-a_4} of the extra times. It ends with the line ALL 26 CHECKS PASSED (notebook 01f).

1.7 How to run Notebook 01f

Step 1. What this notebook does and what it needs.

Notebook 01f (Numbers: fractions, real numbers, floating point and powers) is the file `Revision/textbook/notebooks/01f_numbers_and_powers.ipynb` of the repository Dirac_claude. It computes exactly with whole numbers and fractions, turns fractions into repeating decimals by long division, proves and checks that the square root of 2 is not a fraction and pins it down by halving intervals and by its best fractions, shows how a computer stores real numbers as floating-point numbers and what rounding and cancellation do, checks the laws of powers and roots and uses them to turn the author's metric into the length factors of the eight directions, whose product is the square root of the size of its determinant recorded in the Revision record, and builds the exponential

function and the logarithm with the growth of space and the exponential deflation of the three extra times on logarithmic axes. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01f_numbers_and_powers.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01f_numbers_and_powers.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01f_numbers_and_powers.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01f.captions.json`
- `Revision/textbook/figures/01f_1_decimal_periods.png`
- `Revision/textbook/figures/01f_2_square_root_of_two.png`
- `Revision/textbook/figures/01f_3_float_spacing.png`
- `Revision/textbook/figures/01f_4_cancellation.png`
- `Revision/textbook/figures/01f_5_limit_for_e.png`
- `Revision/textbook/figures/01f_6_growth_and_deflation.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 26 CHECKS PASSED (notebook 01f)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01f_numbers_and_powers.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01f_numbers_and_powers.ipynb
python -m nbconvert --execute --inplace 01f_numbers_and_powers.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock or Revision/algebra: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

1.8 Notebook 01f: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01f_numbers_and_powers.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook builds the numbers that the course computes with, from whole
numbers to the floating-point numbers of a computer, and the powers, roots,
exponentials and logarithms that the author's metric is made of. It

- computes exactly with whole numbers and fractions;
- turns fractions into decimals by long division and shows that their digits
  always end or repeat (with a plot of the length of the repeating part);
- proves that $\sqrt 2$ is not a fraction, pins it down between fractions by
  halving an interval, and finds the best fractions near it;
- shows how a computer stores a real number (a *floating-point number*), why
  $0.1 + 0.2$ is not exactly $0.3$ in a computer, how far apart neighbouring
  floating-point numbers are, and how a careless formula loses all its digits
  (*cancellation*);
- checks the laws of powers and roots, and uses them to turn the diagonal of
  the author's metric into the eight length factors, whose product is the square
  root of the size of the determinant stored in the Revision record;
- builds the number $e$ and the exponential function as a limit, and shows the
  growth $e^{a_4}$ of space and the exponential deflation $e^{-a_4}$ of the
  three extra times on ordinary and on logarithmic axes.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01f (Numbers: fractions, real numbers, floating point and powers) is the file
‘Revision/textbook/notebooks/01f_numbers_and_powers.ipynb‘ of the repository
Dirac_claude. It computes exactly with whole numbers and fractions, turns fractions into
repeating decimals by long division, proves and checks that the square root of 2 is not a
fraction and pins it down by halving intervals and by its best fractions, shows how a
computer stores real numbers as floating-point numbers and what rounding and cancellation
do, checks the laws of powers and roots and uses them to turn the author's metric into
the length factors of the eight directions, whose product is the square root of the size
of its determinant recorded in the Revision record, and builds the exponential function
and the logarithm with the growth of space and the exponential deflation of the three
extra times on logarithmic axes. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
notebooks were built with Python 3.14.5) with these packages at exactly these versions:
numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program
that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.
```

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git

pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 01f_numbers_and_powers.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 01f_numbers_and_powers.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01f_numbers_and_powers.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01f.captions.json‘
- ‘Revision/textbook/figures/01f_1_decimal_periods.png‘
- ‘Revision/textbook/figures/01f_2_square_root_of_two.png‘
- ‘Revision/textbook/figures/01f_3_float_spacing.png‘
- ‘Revision/textbook/figures/01f_4_cancellation.png‘
- ‘Revision/textbook/figures/01f_5_limit_for_e.png‘
- ‘Revision/textbook/figures/01f_6_growth_and_deflation.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it

runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 26 CHECKS PASSED (notebook 01f)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01f_numbers_and_powers.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 01f_numbers_and_powers.ipynb
  python -m nbconvert --execute --inplace 01f_numbers_and_powers.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues

and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock or Revision/algebra: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01f (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 01f (Numbers: fractions, real numbers, floating point and powers) is the file
# Revision/textbook/notebooks/01f_numbers_and_powers.ipynb of the repository
# Dirac_claude. It computes exactly with whole numbers and fractions, turns fractions
# into repeating decimals by long division, proves and checks that the square root of 2
# is not a fraction and pins it down by halving intervals and by its best fractions,
# shows how a computer stores real numbers as floating-point numbers and what rounding
# and cancellation do, checks the laws of powers and roots and uses them to turn the
# author's metric into the length factors of the eight directions, whose product is the
# square root of the size of its determinant recorded in the Revision record, and builds
# the exponential function and the logarithm with the growth of space and the exponential
# deflation of the three extra times on logarithmic axes. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
```

```
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01f_numbers_and_powers.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
```

```
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01f_numbers_and_powers.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01f_numbers_and_powers.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01f.captions.json
# - Revision/textbook/figures/01f_1_decimal_periods.png
# - Revision/textbook/figures/01f_2_square_root_of_two.png
# - Revision/textbook/figures/01f_3_float_spacing.png
# - Revision/textbook/figures/01f_4_cancellation.png
# - Revision/textbook/figures/01f_5_limit_for_e.png
# - Revision/textbook/figures/01f_6_growth_and_deflation.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 26 CHECKS PASSED (notebook 01f)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
```

```
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01f_numbers_and_powers.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01f_numbers_and_powers.ipynb
#     python -m nbconvert --execute --inplace 01f_numbers_and_powers.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below Revision/gkd_lovelock or Revision/algebra: the
#   notebook reads Revision records of the repository; your copy of the repository is
#   incomplete. Download it again with git clone and open the notebook inside the new
#   copy.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

```
NOTEBOOK_ID = "01f"  # this notebook: chapter 01, example f


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
```

```
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01f complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Whole numbers** (integers): $\dots, -2, -1, 0, 1, 2, \dots$. Python computes
  with them exactly, however large they are.
- **Fraction** (rational number): $p/q$ with whole numbers $p$ and $q \neq 0$.
  It is in **lowest terms** when $p$ and $q$ have no common factor larger than 1.
  Python's `fractions.Fraction` computes with fractions exactly.
- **Greatest common divisor** $\gcd(p, q)$: the largest whole number that
  divides both $p$ and $q$ (`math.gcd`).
- **Long division**: the school method that produces the decimal digits of
  $p/q$ one after the other from the remainders.
- **Repeating decimal**: a decimal whose digits repeat a block for ever, written
  with the block in brackets: $1/7 = 0.(142857)$. The block's length is the
  **period**.
- **Prime number**: a whole number larger than 1 that only 1 and itself divide.
- **Real number**: a number on the number line; it may need infinitely many
  digits that do not repeat, like $\sqrt 2$ or $\pi$. Such a number is
  **irrational** (not a fraction).
- **Bisection**: halving an interval that contains the number again and again,
  keeping the half that still contains it.
- **Floating-point number**: the way a computer stores a real number: a whole
  number of 53 binary digits (bits) times a power of 2. Neighbouring
  floating-point numbers are a small step apart, so most real numbers are
  **rounded** to the nearest one.
- **Machine epsilon** $\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$: the
  step from 1 to the next floating-point number.
- **Relative error**: $|$computed $-$ exact$|$ divided by $|$exact$|$.
- **Cancellation**: the loss of digits when two nearly equal numbers are
  subtracted.
- **Power** $a^p$ and **root** $a^{1/n}$; the **laws of powers**
  $a^p a^q = a^{p+q}$, $(a^p)^q = a^{pq}$, $(ab)^p = a^p b^p$,
  $a^{-p} = 1/a^p$ for positive $a, b$.
- **Absolute value** $|x|$: $x$ without its sign; $\sqrt{x^2} = |x|$.
- **Exponential function** $e^x$ and **natural logarithm** $\ln x$, which
  undoes it: $\ln(e^x) = x$. The number $e = 2.71828\dots$.
- **e-fold**: a growth by the factor $e$; a growth of $a_4$ by $\ln 2 \approx
  0.693$ doubles $e^{a_4}$ and halves $e^{-a_4}$.
- **Logarithmic axis**: an axis on which each step multiplies by 10, so that
  an exponential function is a straight line.
- **mpmath**: a Python package that computes with as many decimal digits as
  requested (here 50).
```

Text cell:

```
## 4. The physical and mathematical situation
```

```
The author's metric of the primordial gravitational field is the diagonal
matrix

$$g = \mathrm{diag}\big(\cot^2 z, e^{2a_4} s, e^{2a_4} s, e^{2a_4} s, -1,
-b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s\big),\quad
s = \sin^{1/3} z,\ z = 6 H x_0,\ 0 < z < \pi/2,$$

with a function $a_4(x_4)$ and a positive function $b_4(x_4)$ of the time
($b_4 = 1$ in the background of the Revision record). Its entries are built
from powers ($\sin^{1/3} z$ is the cube root of $\sin z$), exponentials
($e^{2a_4}$) and a trigonometric function. A length along a coordinate is the
square root of the size of its metric entry: along $x_1, x_2, x_3$ (space) it is
$e^{a_4}\sin^{1/6} z$, which grows as $a_4$ grows, and along the three extra
times $x_5, x_6, x_7$ it is $b_4^{1/2} e^{-a_4}\sin^{1/6} z$, which shrinks: for
constant $b_4$ the extra times deflate exponentially.
The time $x_4$ has the factor 1 and the hidden direction $x_0$ the factor
$\cot z$. To compute with these expressions correctly one needs the laws of
powers, roots, exponentials and logarithms, and one needs to know when a
computer computes exactly (whole numbers, fractions, sympy's symbols) and when
it rounds (floating-point numbers). Both are built here from zero.
```

Text cell:

```
## 5. Whole numbers and fractions, exactly

Python's whole numbers have no size limit: $2^{100}$ has 31 digits, and every
one of them is right. Fractions are exact too: `fractions.Fraction(1, 3)` is
the number $1/3$ itself, not a rounded decimal. The next cell computes

- $2^{100}$ and $16! = 20{,}922{,}789{,}888{,}000$ exactly;
- $\tfrac13 + \tfrac16 = \tfrac26 + \tfrac16 = \tfrac36 = \tfrac12$ (bring both
  to the common denominator 6, add the numerators, cancel the common factor 3);
- $\tfrac{84}{126}$ in lowest terms: $\gcd(84, 126) = 42$, so
  $\tfrac{84}{126} = \tfrac{84/42}{126/42} = \tfrac23$.
```

In [2]:

```
import fractions  # exact fractions p/q
import math  # factorial, gcd, isqrt, e, log

import mpmath  # numbers with many digits
import numpy as np  # arrays of floating-point numbers
import sympy as sp  # exact algebra with symbols

big = 2 ** 100
say(f"2^100 = {big} ({len(str(big))} digits)")
say(f"16! = {math.factorial(16)}")
check(big == 1267650600228229401496703205376 and len(str(big)) == 31
      and math.factorial(16) == 20922789888000,
      "2^100 and 16! are computed exactly")
third, sixth = fractions.Fraction(1, 3), fractions.Fraction(1, 6)
say(f"1/3 + 1/6 = {third + sixth}")
```

```
check(third + sixth == fractions.Fraction(1, 2), "1/3 + 1/6 = 1/2 exactly")
reduced = fractions.Fraction(84, 126)  # Fraction always cancels common factors
say(f"gcd(84, 126) = {math.gcd(84, 126)}, 84/126 = {reduced}")
check(math.gcd(84, 126) == 42 and reduced == fractions.Fraction(2, 3)
      and (reduced.numerator, reduced.denominator) == (2, 3),
      "84/126 = 2/3 in lowest terms, with gcd(84, 126) = 42")
```

Out [2]:

```
2^100 = 1267650600228229401496703205376 (31 digits)
16! = 20922789888000
PASS 2^100 and 16! are computed exactly
1/3 + 1/6 = 1/2
PASS 1/3 + 1/6 = 1/2 exactly
gcd(84, 126) = 42, 84/126 = 2/3
PASS 84/126 = 2/3 in lowest terms, with gcd(84, 126) = 42
```

Text cell:

```
## 6. Fractions as decimals: the digits end or repeat

Long division of $p$ by $q$ (with $0 < p < q$) works with remainders: multiply
the remainder by 10, the whole part of the division by $q$ is the next digit,
and what is left is the new remainder. For $1/7$: $10 = 1 \cdot 7 + 3$ (digit
1, remainder 3), $30 = 4 \cdot 7 + 2$ (digit 4, remainder 2), then digits
2, 8, 5, 7 with remainders 6, 4, 5, 1, and the remainder 1 is where we
started: from here the digits repeat, $1/7 = 0.(142857)$.

Why the digits must end or repeat: a remainder is one of $0, 1, \dots, q - 1$.
If it becomes 0 the division ends. Otherwise it is one of the $q - 1$ numbers
$1, \dots, q - 1$, so after at most $q - 1$ steps a remainder comes back, and
from then on everything repeats. So the period is at most $q - 1$.

The digits can be turned back into the fraction: if the digits $d_1 \dots d_k$
come first and then the block $c_1 \dots c_m$ repeats, the number is

$$\frac{D}{10^k} + \frac{C}{10^k(10^m - 1)},$$

where $D$ and $C$ are the whole numbers with the digits $d_1 \dots d_k$ and
$c_1 \dots c_m$ (because $0.(c_1 \dots c_m) = C/10^m + C/10^{2m} + \dots =
C/(10^m - 1)$, a geometric series). The next cell defines ‘decimal_digits‘,
prints six examples and checks that the digits give the fractions back.
```

In [3]:

```
def decimal_digits(p, q):
    """The decimal digits of p/q (0 < p < q) by long division: the digits before
    the repeating block, and the repeating block (empty if the division ends)."""
    digits = []
    first_seen = {}  # remainder -> the place of the digit it produced first
    remainder = p
```

```
    while remainder != 0 and remainder not in first_seen:
        first_seen[remainder] = len(digits)
        remainder *= 10
        digits.append(remainder // q)  # the next digit
        remainder %= q  # what is left
    if remainder == 0:
        return digits, []
    start = first_seen[remainder]  # the remainder came back: the digits repeat
    return digits[:start], digits[start:]


def from_digits(head, block):
    """The fraction with the digits head followed by block repeated for ever."""
    k, m = len(head), len(block)
    D = int("".join(map(str, head)) or "0")
    value = fractions.Fraction(D, 10 ** k)
    if block:
        C = int("".join(map(str, block)))
        value += fractions.Fraction(C, 10 ** k * (10 ** m - 1))
    return value


all_back = True
for p, q in [(1, 4), (1, 3), (1, 6), (1, 7), (5, 12), (1, 97)]:
    head, block = decimal_digits(p, q)
    head_text = "".join(map(str, head))
    block_text = "(" + "".join(map(str, block)) + ")" if block else ""
    text = "0." + head_text + block_text
    if len(text) > 40:  # 1/97 has a block of 96 digits: show its length only
        text = f"0.({len(block)} repeating digits)"
    say(f"{p}/{q} = {text}, period {len(block)}")
    all_back &= from_digits(head, block) == fractions.Fraction(p, q)
check(decimal_digits(1, 7) == ([], [1, 4, 2, 8, 5, 7]) and all_back,
      "1/7 = 0.(142857), and the digits of all six examples give the fractions back")
```

Out [3]:

```
1/4 = 0.25, period 0
1/3 = 0.(3), period 1
1/6 = 0.1(6), period 1
1/7 = 0.(142857), period 6
5/12 = 0.41(6), period 1
1/97 = 0.(96 repeating digits), period 96
PASS 1/7 = 0.(142857), and the digits of all six examples give the fractions back
```

Text cell:

```
Which fractions end? $p/q$ in lowest terms ends exactly when $q$ has no prime
factors other than 2 and 5, because only then is $q$ a divisor of a power of
$10 = 2 \cdot 5$. The next cell computes the period of $1/q$ for every
$q = 2, \dots, 300$, checks that it is 0 exactly for these $q$ and never more
than $q - 1$, and plots it; the primes $q$ whose period is the largest
possible, $q - 1$ (such as 7, 17, 19, 23, 29), lie on the line.
```

In [4]:

```
def only_twos_and_fives(q):
    """True if q has no prime factor other than 2 and 5."""
    for factor in (2, 5):
        while q % factor == 0:
            q //= factor
    return q == 1


qs = list(range(2, 301))
periods = [len(decimal_digits(1, q)[1]) for q in qs]
check(all((period == 0) == only_twos_and_fives(q) for q, period in zip(qs, periods))
      and all(period <= q - 1 for q, period in zip(qs, periods)),
      "1/q ends exactly when q = 2^a 5^b, and the period is at most q - 1 "
      "(q = 2 to 300)")
full = [q for q, period in zip(qs, periods) if period == q - 1]
say(f"q with the full period q - 1: {full[:12]} ... ({len(full)} of them)")
fig, ax = plt.subplots(figsize=(8.0, 4.4))
ax.plot(qs, periods, ".", color="tab:blue", label="period of $1/q$")
ax.plot(full, [q - 1 for q in full], "o", color="tab:red", fillstyle="none",
        label="full period $q - 1$")
ax.plot([2, 300], [1, 299], "k:", lw=1, label="the largest possible: $q - 1$")
ax.set_xlabel("denominator $q$")
ax.set_ylabel("length of the repeating block")
ax.set_title("Decimal expansion of $1/q$: the period")
ax.legend(loc="upper left", fontsize=8);
save_figure(fig, "decimal_periods",
            "The length of the repeating block (the period) of the decimal digits "
            "of $1/q$ for every denominator $q = 2$ to 300 (blue dots); horizontal "
            "axis $q$, vertical axis the period (a whole number). Dots at 0 are the "
            "denominators $q = 2^a 5^b$, whose decimals end. No dot lies above the "
            "dotted line $q - 1$, because long division has only $q - 1$ nonzero "
            "remainders; the red circles are the primes that reach it.")
```

Out [4]:

```
PASS 1/q ends exactly when q = 2^a 5^b, and the period is at most q - 1 (q = 2 to 300)
q with the full period q - 1: [7, 17, 19, 23, 29, 47, 59, 61, 97, 109, 113, 131] ... (23
    of them)
```

Out [4]:

```
Figure 01f.1 saved as Revision/textbook/figures/01f_1_decimal_periods.png
```

Text cell:

```
## 7. Real numbers: the square root of 2 is not a fraction

**Claim.** No fraction $p/q$ has $(p/q)^2 = 2$. **Proof** by contradiction,
```

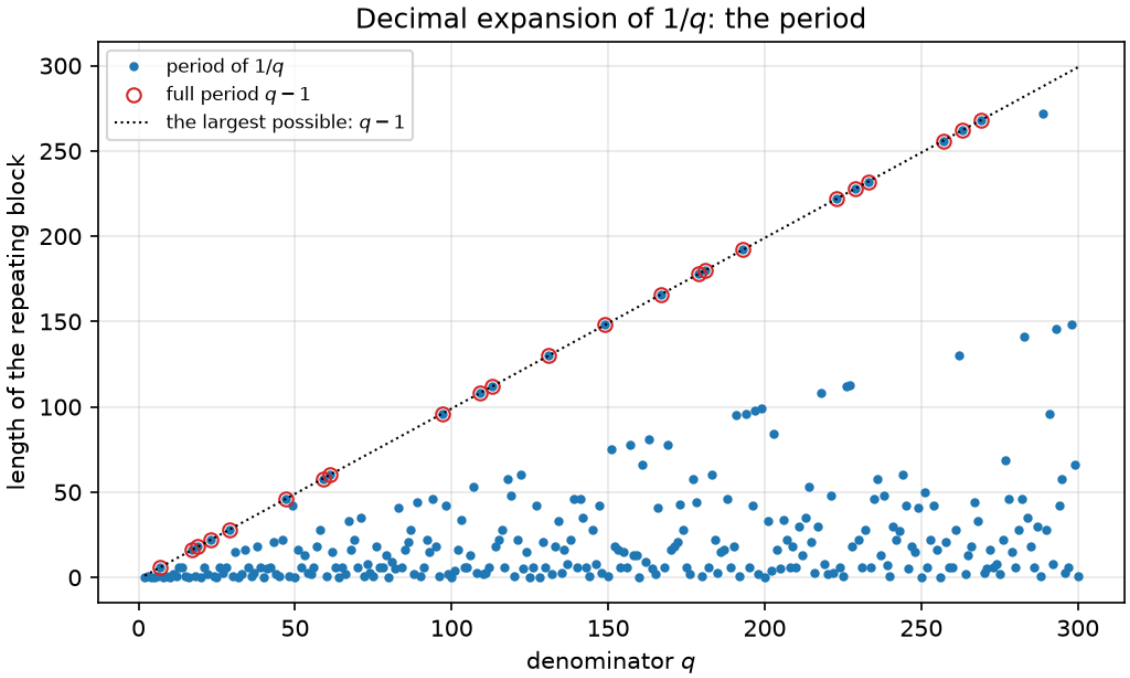

Figure 17: The length of the repeating block (the period) of the decimal digits of $1/q$ for every denominator $q = 2$ to 300 (blue dots); horizontal axis q, vertical axis the period (a whole number). Dots at 0 are the denominators $q = 2^a 5^b$, whose decimals end. No dot lies above the dotted line $q - 1$, because long division has only $q - 1$ nonzero remainders; the red circles are the primes that reach it. (Notebook 01f, figure 1.)

```
line by line. Suppose $p/q$ is in lowest terms and $p^2/q^2 = 2$. Then

- $p^2 = 2q^2$ (multiply by $q^2$), so $p^2$ is even;
- then $p$ is even (the square of an odd number $2k + 1$ is
  $4k^2 + 4k + 1$, which is odd), so $p = 2r$ for a whole number $r$;
- then $4r^2 = 2q^2$, that is $q^2 = 2r^2$, so $q$ is even as well;
- so 2 divides both $p$ and $q$, and $p/q$ was not in lowest terms: a
  contradiction. Hence $\sqrt 2$ is not a fraction: it is irrational.

The best one can do is to come close. The fractions $p/q$ made by the rule
$p' = p + 2q$, $q' = p + q$ from $1/1$ ($3/2$, $7/5$, $17/12$, $41/29$, ...)
have $p^2 - 2q^2 = \pm 1$, alternating in sign: indeed

$$p'^2 - 2q'^2 = (p + 2q)^2 - 2(p + q)^2 = p^2 + 4pq + 4q^2 - 2p^2 - 4pq - 2q^2
= -(p^2 - 2q^2)$$

(multiply out both squares, then collect the terms). So $p^2/q^2 = 2 \pm
1/q^2$ comes ever closer to 2 without reaching it, and
$p/q - \sqrt 2 = (p^2 - 2q^2)/(q(p + q\sqrt 2))$ (multiply numerator and
denominator by $p + q\sqrt 2$), whose size is about $1/(2\sqrt 2\, q^2)$.

The next cell checks with exact whole numbers that $p^2 = 2q^2$ has no
solution with $q \leq 10{,}000$, lists 12 of these fractions with
$p^2 - 2q^2$ and the error (computed with mpmath at 50 digits), and checks
that error $\cdot\, q^2$ approaches $1/(2\sqrt 2) = 0.35355\dots$
```

In [5]:

```
# isqrt(n) is the whole part of the square root of n, computed exactly:
no_solution = all(math.isqrt(2 * q * q) ** 2 != 2 * q * q for q in range(1, 10001))
check(no_solution, "p^2 = 2 q^2 has no solution in whole numbers with q <= 10000")
mpmath.mp.dps = 50  # 50 significant digits
root2 = mpmath.sqrt(2)
convergents = [(1, 1)]
while len(convergents) < 12:
    p, q = convergents[-1]
    convergents.append((p + 2 * q, p + q))  # the rule p' = p + 2q, q' = p + q
for p, q in convergents:
    error = abs(mpmath.mpf(p) / q - root2)
    say(f"{p}/{q}: p^2 - 2 q^2 = {p * p - 2 * q * q:+d}, error "
        f"{mpmath.nstr(error, 3)}, error q^2 = {mpmath.nstr(error * q * q, 6)}")
signs = [p * p - 2 * q * q for p, q in convergents]
check(signs == [(-1) ** (k + 1) for k in range(12)],
      "p^2 - 2 q^2 = -1, +1, -1, ... for the 12 fractions")
p, q = convergents[-1]
limit_value = 1 / (2 * root2)
check(abs(abs(mpmath.mpf(p) / q - root2) * q * q - limit_value) < mpmath.mpf("1e-6"),
      "the error of p/q times q^2 approaches 1/(2 sqrt 2) = 0.35355")
```

Out [5]:

```
PASS p^2 = 2 q^2 has no solution in whole numbers with q <= 10000
1/1: p^2 - 2 q^2 = -1, error 0.414, error q^2 = 0.414214
```

```
3/2: p^2 - 2 q^2 = +1, error 0.0858, error q^2 = 0.343146
7/5: p^2 - 2 q^2 = -1, error 0.0142, error q^2 = 0.355339
17/12: p^2 - 2 q^2 = +1, error 0.00245, error q^2 = 0.353247
41/29: p^2 - 2 q^2 = -1, error 0.00042, error q^2 = 0.353606
99/70: p^2 - 2 q^2 = +1, error 7.22e-5, error q^2 = 0.353544
239/169: p^2 - 2 q^2 = -1, error 1.24e-5, error q^2 = 0.353555
577/408: p^2 - 2 q^2 = +1, error 2.12e-6, error q^2 = 0.353553
1393/985: p^2 - 2 q^2 = -1, error 3.64e-7, error q^2 = 0.353553
3363/2378: p^2 - 2 q^2 = +1, error 6.25e-8, error q^2 = 0.353553
8119/5741: p^2 - 2 q^2 = -1, error 1.07e-8, error q^2 = 0.353553
19601/13860: p^2 - 2 q^2 = +1, error 1.84e-9, error q^2 = 0.353553
PASS p^2 - 2 q^2 = -1, +1, -1, ... for the 12 fractions
PASS the error of p/q times q^2 approaches 1/(2 sqrt 2) = 0.35355
```

Text cell:

```
A real number can also be pinned down by **bisection** with fractions only.
Start with $1 < \sqrt 2 < 2$ (because $1^2 < 2 < 2^2$). Take the middle $m$ of
the interval: if $m^2 < 2$ the root lies in the right half, otherwise in the
left half. Each step halves the width, so after 60 steps the width is
$2^{-60} \approx 8.7 \times 10^{-19}$ while both ends are still exact
fractions. The next cell does this and draws the widths (left) and the errors
of the best fraction with each denominator $q = 1, \dots, 1000$ (right).
```

In [6]:

```
low, high = fractions.Fraction(1), fractions.Fraction(2)  # 1^2 < 2 < 2^2
widths = [high - low]
for step in range(60):
    middle = (low + high) / 2
    if middle * middle < 2:
        low = middle  # the root is in the right half
    else:
        high = middle  # the root is in the left half
    widths.append(high - low)
low_mp = mpmath.mpf(low.numerator) / low.denominator
high_mp = mpmath.mpf(high.numerator) / high.denominator
say(f"after 60 halvings: {mpmath.nstr(low_mp, 20)} < sqrt 2 < "
    f"{mpmath.nstr(high_mp, 20)}")
say(f"sqrt 2 with 30 digits: {mpmath.nstr(root2, 30)}")
check(low * low < 2 < high * high and high - low == fractions.Fraction(1, 2 ** 60)
      and low_mp < root2 < high_mp,
      "60 halvings: exact fractions low < sqrt 2 < high with high - low = 2^(-60)")
denominators = np.arange(1, 1001)
best_p = np.rint(denominators * np.sqrt(2.0))  # the nearest numerator for each q
best_error = np.abs(best_p / denominators - np.sqrt(2.0))
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
left.semilogy(range(61), [float(w) for w in widths], "o-", ms=3)
left.set_xlabel("number of halvings")
left.set_ylabel("width of the interval (logarithmic scale)")
left.set_title("bisection: $\\sqrt{2}$ between two fractions")
right.loglog(denominators, best_error, ".", ms=3, color="0.6",
```

```
            label="best $p/q$ for each $q$")
conv_q = [q for _, q in convergents if q <= 1000]
conv_error = [float(abs(mpmath.mpf(p) / q - root2)) for p, q in convergents
              if q <= 1000]
right.loglog(conv_q, conv_error, "o", color="tab:red",
             label="$1/1, 3/2, 7/5, 17/12, \\dots$")
right.loglog(denominators, 1 / (2 * np.sqrt(2.0) * denominators ** 2.0), "k--",
             lw=1, label="$1/(2\\sqrt{2}\\,q^2)$")
right.set_xlabel("denominator $q$")
right.set_ylabel("error $|p/q - \\sqrt{2}|$")
right.set_title("the best fractions near $\\sqrt{2}$")
right.legend(fontsize=8, loc="lower left")
fig.tight_layout()
save_figure(fig, "square_root_of_two",
            "Left: the width of the interval between two fractions that contains "
            "$\\sqrt{2}$, after each of 60 halvings (bisection), on a logarithmic "
            "vertical axis: a straight line, because each step halves the width, "
            "down to $2^{-60}$. Right: the error $|p/q - \\sqrt{2}|$ of the best "
            "fraction with denominator $q$ for $q = 1$ to 1000 (grey dots), the "
            "fractions $1/1, 3/2, 7/5, 17/12, \\dots$ made by the rule $p + 2q$, "
            "$p + q$ (red) and the line $1/(2\\sqrt{2}\\,q^2)$ (dashed); both axes "
            "logarithmic, pure numbers. No error is 0: $\\sqrt{2}$ is not a "
            "fraction, but the red fractions come as close as $1/q^2$ allows.")
```

Out [6]:

```
after 60 halvings: 1.4142135623730950483 < sqrt 2 < 1.4142135623730950492
sqrt 2 with 30 digits: 1.41421356237309504880168872421
PASS 60 halvings: exact fractions low < sqrt 2 < high with high - low = 2^(-60)
```

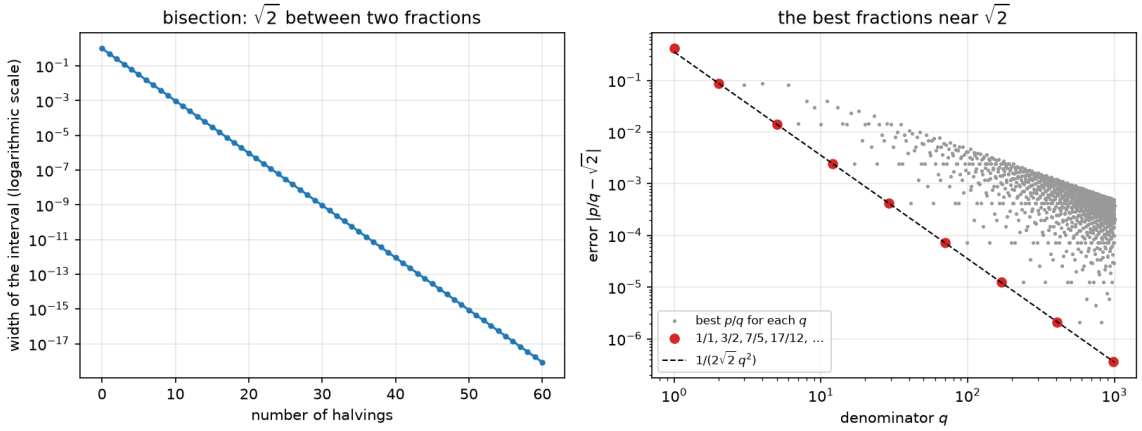

Figure 18: Left: the width of the interval between two fractions that contains $\sqrt{2}$, after each of 60 halvings (bisection), on a logarithmic vertical axis: a straight line, because each step halves the width, down to 2^{-60}. Right: the error $|p/q - \sqrt{2}|$ of the best fraction with denominator q for $q = 1$ to 1000 (grey dots), the fractions $1/1, 3/2, 7/5, 17/12, \dots$ made by the rule $p + 2q$, $p + q$ (red) and the line $1/(2\sqrt{2}\,q^2)$ (dashed); both axes logarithmic, pure numbers. No error is 0: $\sqrt{2}$ is not a fraction, but the red fractions come as close as $1/q^2$ allows. (Notebook 01f, figure 2.)

Out [6]:

```
Figure 01f.2 saved as Revision/textbook/figures/01f_2_square_root_of_two.png
```

Text cell:

```
## 8. Floating-point numbers

A computer stores a real number as a **floating-point number**: a whole number
$m$ of at most 53 bits times a power of two, $m \cdot 2^k$. Fractions whose
denominator is a power of 2, such as $0.5 = 1/2$, are stored exactly; others,
such as $0.1 = 1/10$, are rounded to the nearest such number. That is why
$0.1 + 0.2$ is not exactly $0.3$ in a computer. The next cell shows:

- $0.1 + 0.2$ as Python prints it, and the exact fraction that is stored for
  $0.1$ (`fractions.Fraction(0.1)` reads it out): its denominator is $2^{55}$;
- machine epsilon $\varepsilon = 2^{-52}$: $1 + \varepsilon$ is the next
  floating-point number after 1, and $1 + \varepsilon/2$ is rounded back to 1;
- above $2^{53}$ not even every whole number can be stored:
  $2^{53} + 1$ is rounded to $2^{53}$;
- the step from a number $x$ to the next floating-point number
  (`np.spacing`): writing $x = f \cdot 2^e$ with $\tfrac12 \leq f < 1$
  (`np.frexp` finds $f$ and $e$), the step is exactly $2^{e - 53}$; so it is
  about $\varepsilon x$: the *relative* precision is the same at every size.
```

In [7]:

```
total = 0.1 + 0.2
stored = fractions.Fraction(0.1)  # the exact value of the floating-point number 0.1
say(f"0.1 + 0.2 = {total:.17f} (17 digits), and 0.3 = {0.3:.17f}")
say(f"0.1 is stored as {stored.numerator}/{stored.denominator}")
check(total != 0.3 and stored != fractions.Fraction(1, 10)
      and stored.denominator == 2 ** 55,
      "0.1 + 0.2 is not 0.3 in floating point, and 0.1 is stored as a fraction "
      "with the denominator 2^55")
eps = float(np.finfo(float).eps)  # machine epsilon
say(f"machine epsilon = {eps!r} = 2^-52: {eps == 2.0 ** -52}")
check(eps == 2.0 ** -52 and 1.0 + eps != 1.0 and 1.0 + eps / 2 == 1.0,
      "1 + eps is the next number after 1; 1 + eps/2 is rounded back to 1")
check(float(2 ** 53 + 1) == float(2 ** 53) and float(2 ** 53 - 1) != float(2 ** 53),
      "2^53 + 1 cannot be stored: it is rounded to 2^53")
xs = np.logspace(-3, 20, 2301)  # 2301 numbers from 0.001 to 10^20
fraction_part, exponent = np.frexp(xs)  # xs = fraction_part * 2^exponent
predicted = np.ldexp(1.0, exponent - 53)  # 2^(exponent - 53)
check(bool(np.all(np.spacing(xs) == predicted))
      and bool(np.all(np.spacing(xs) <= eps * xs)),
      "the step after x is 2^(e - 53), never more than eps x (2301 values of x)")
```

Out [7]:

```
0.1 + 0.2 = 0.30000000000000004 (17 digits), and 0.3 = 0.29999999999999999
0.1 is stored as 3602879701896397/36028797018963968
PASS 0.1 + 0.2 is not 0.3 in floating point, and 0.1 is stored as a fraction with the
    denominator 2^55
machine epsilon = 2.220446049250313e-16 = 2^-52: True
PASS 1 + eps is the next number after 1; 1 + eps/2 is rounded back to 1
PASS 2^53 + 1 cannot be stored: it is rounded to 2^53
PASS the step after x is 2^(e - 53), never more than eps x (2301 values of x)
```

Text cell:

```
The next cell draws the step size in two ways. Left: the step itself for $x$
from $2^{48}$ to $2^{57}$ (about $2.8 \times 10^{14}$ to $1.4 \times 10^{17}$),
both axes logarithmic with base 2: a staircase, because the step stays the
same between two powers of 2 and doubles at each power of 2; it is 1 between
$2^{52}$ and $2^{53}$ and 2 above $2^{53}$, where odd whole numbers can no
longer be stored. Right: the *relative* step, step divided by $x$, for $x$ from
$10^{-3}$ to $10^{20}$: it always lies between $\varepsilon/2$ and
$\varepsilon$.
```

In [8]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4))
big_x = np.logspace(48, 57, 2000, base=2.0)  # from 2^48 to 2^57
left.loglog(big_x, np.spacing(big_x), color="tab:blue", lw=1.5, base=2,
            label="step to the next floating-point number")
left.axvline(2.0 ** 53, color="tab:red", lw=1, label="$x = 2^{53}$")
left.axhline(1.0, color="tab:red", ls=":", lw=1, label="step 1")
left.set_xlabel("number $x$ (logarithmic axis, base 2)")
left.set_ylabel("step size (logarithmic axis, base 2)")
left.set_title("the step doubles at every power of 2")
left.legend(fontsize=8, loc="upper left")
right.semilogx(xs, np.spacing(xs) / xs, color="tab:blue", lw=1,
               label="step divided by $x$")
right.axhline(eps, color="black", ls="--", lw=1, label="$\\varepsilon = 2^{-52}$")
right.axhline(eps / 2, color="black", ls=":", lw=1, label="$\\varepsilon/2$")
right.set_ylim(0.0, 1.35 * eps)
right.set_xlabel("number $x$")
right.set_ylabel("relative step")
right.set_title("the relative step is the same at every size")
right.legend(fontsize=8, loc="lower right")
fig.tight_layout()
save_figure(fig, "float_spacing",
            "How far apart floating-point numbers are. Left: the distance from a "
            "number $x$ to the next floating-point number for $x$ from $2^{48}$ to "
            "$2^{57}$, both axes logarithmic with base 2: a staircase that doubles "
            "at every power of 2; right of the red line $x = 2^{53}$ the step is 2, "
            "so odd whole numbers are skipped there. Right: the step divided by "
            "$x$ for $x$ from $10^{-3}$ to $10^{20}$ (logarithmic horizontal axis): "
            "a sawtooth that always stays between $\\varepsilon/2$ (dotted) and "
            "$\\varepsilon = 2^{-52} \\approx 2.2 \\times 10^{-16}$ (dashed), the "
            "relative precision of every floating-point number.")
```

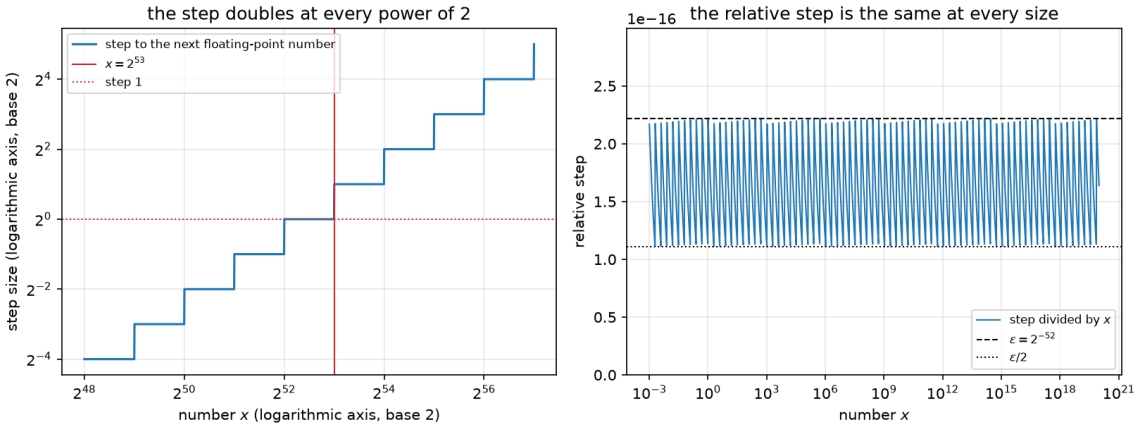

Figure 19: How far apart floating-point numbers are. Left: the distance from a number x to the next floating-point number for x from 2^{48} to 2^{57}, both axes logarithmic with base 2: a staircase that doubles at every power of 2; right of the red line $x = 2^{53}$ the step is 2, so odd whole numbers are skipped there. Right: the step divided by x for x from 10^{-3} to 10^{20} (logarithmic horizontal axis): a sawtooth that always stays between $\varepsilon/2$ (dotted) and $\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$ (dashed), the relative precision of every floating-point number. (Notebook 01f, figure 3.)

Out [8]:

```
Figure 01f.3 saved as Revision/textbook/figures/01f_3_float_spacing.png
```

Text cell:

```
## 9. Rounding and cancellation

The function $f(x) = (1 - \cos x)/x^2$ tends to $\tfrac12$ as $x$ tends to 0.
Computed as written, it fails for small $x$: $\cos x$ is very close to 1, its
rounding error is about $\varepsilon$, and the subtraction $1 - \cos x$ keeps
this error while the true value $1 - \cos x \approx x^2/2$ is tiny. The
relative error is then about $\varepsilon/(x^2/2)$; for $x = 10^{-8}$,
$\cos x$ is rounded to exactly 1 and the result is 0, all digits lost. This
is **cancellation**.

The cure is to rewrite the formula without the subtraction. The half-angle
formula $\cos(2y) = 1 - 2\sin^2 y$ (from $\cos(2y) = \cos^2 y - \sin^2 y$ and
$\cos^2 y = 1 - \sin^2 y$) with $y = x/2$ gives
$1 - \cos x = 2\sin^2(x/2)$, so

$$f(x) = \frac{2\sin^2(x/2)}{x^2},$$

which has no subtraction. The next cell checks the limit and the identity with
sympy, computes both formulas at 361 values of $x$ from $10^{-9}$ to 1,
compares them with mpmath at 50 digits, and plots the relative errors.
```

In [9]:

```
x = sp.Symbol("x", positive=True)
limit_at_zero = sp.limit((1 - sp.cos(x)) / x ** 2, x, 0)
say(f"sympy: the limit of (1 - cos x)/x^2 at x = 0 is {limit_at_zero}")
check(limit_at_zero == sp.Rational(1, 2)
      and sp.simplify(1 - sp.cos(x) - 2 * sp.sin(x / 2) ** 2) == 0,
      "the limit is 1/2, and 1 - cos x = 2 sin(x/2)^2 exactly")
x_values = np.logspace(-9, 0, 361)
naive = (1 - np.cos(x_values)) / x_values ** 2
stable = 2 * np.sin(x_values / 2) ** 2 / x_values ** 2
mpmath.mp.dps = 50
exact = np.array([float((1 - mpmath.cos(mpmath.mpf(v))) / mpmath.mpf(v) ** 2)
                  for v in x_values.tolist()])  # mpf(v) is the stored value of v
error_naive = np.abs(naive - exact) / exact
error_stable = np.abs(stable - exact) / exact
report("largest relative error of 2 sin(x/2)^2/x^2", f"{error_stable.max():.1e}")
report("largest relative error of (1 - cos x)/x^2", f"{error_naive.max():.1e}")
check(float(error_stable.max()) < 1e-15,
      "the rewritten formula is correct to 1e-15 at all 361 values of x")
check((1 - math.cos(1e-8)) / 1e-8 ** 2 == 0.0
      and float(error_naive[x_values <= 1e-4].max()) > 1e-3,
      "the formula as written gives 0 at x = 1e-8: cancellation destroyed every digit")
fig, ax = plt.subplots(figsize=(8.0, 4.6))
floor = 1e-18  # an error 0 is drawn at the bottom of the axis
ax.loglog(x_values, np.maximum(error_naive, floor), ".", ms=3, color="tab:red",
          label="$(1 - \\cos x)/x^2$ as written")
ax.loglog(x_values, np.maximum(error_stable, floor), ".", ms=3, color="tab:blue",
          label="$2\\sin^2(x/2)/x^2$")
ax.loglog(x_values, 2 * eps / x_values ** 2, "k--", lw=1,
          label="$2\\varepsilon/x^2$")
ax.set_ylim(0.5 * floor, 10.0)
ax.set_xlabel("$x$")
ax.set_ylabel("relative error")
ax.set_title("Cancellation and its cure")
ax.legend(fontsize=8, loc="upper right");
save_figure(fig, "cancellation",
            "The relative error of two formulas for $(1 - \\cos x)/x^2$ against $x$ "
            "from $10^{-9}$ to 1, both axes logarithmic, compared with 50-digit "
            "arithmetic. Red: the formula as written loses digits like "
            "$2\\varepsilon/x^2$ (dashed line) and gives an error of 100 per cent "
            "for $x$ below about $10^{-8}$. Blue: the same function written as "
            "$2\\sin^2(x/2)/x^2$ stays correct to about $10^{-16}$ (errors that "
            "are exactly 0 are drawn at the bottom of the axis).")
```

Out [9]:

```
sympy: the limit of (1 - cos x)/x^2 at x = 0 is 1/2
PASS the limit is 1/2, and 1 - cos x = 2 sin(x/2)^2 exactly
RESULT largest relative error of 2 sin(x/2)^2/x^2 = 4.4e-16
RESULT largest relative error of (1 - cos x)/x^2 = 1.0e+00
PASS the rewritten formula is correct to 1e-15 at all 361 values of x
PASS the formula as written gives 0 at x = 1e-8: cancellation destroyed every digit
```

Out [9]:

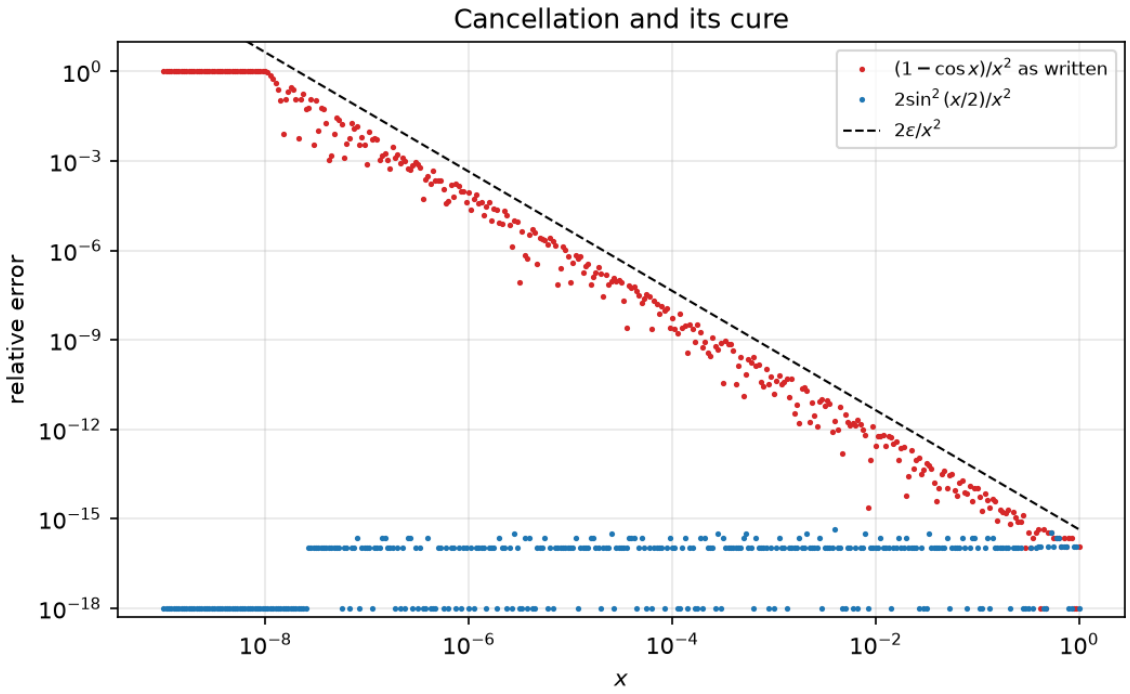

Figure 20: The relative error of two formulas for $(1 - \cos x)/x^2$ against x from 10^{-9} to 1, both axes logarithmic, compared with 50-digit arithmetic. Red: the formula as written loses digits like $2\varepsilon/x^2$ (dashed line) and gives an error of 100 per cent for x below about 10^{-8}. Blue: the same function written as $2\sin^2(x/2)/x^2$ stays correct to about 10^{-16} (errors that are exactly 0 are drawn at the bottom of the axis). (Notebook 01f, figure 4.)

```
Figure 01f.4 saved as Revision/textbook/figures/01f_4_cancellation.png
```

Text cell:

```
## 10. Powers and roots: the length factors of the author's metric

For positive $a, b$ and any real $p, q$ the laws of powers are

$$a^p a^q = a^{p+q},\quad (a^p)^q = a^{pq},\quad (ab)^p = a^p b^p,\quad
a^{-p} = \frac{1}{a^p},\quad \sqrt a = a^{1/2}.$$

They need $a, b > 0$. For a negative number they can fail:
$\sqrt{(-2)^2} = \sqrt 4 = 2$, not $-2$; in general $\sqrt{x^2} = |x|$.

A worked example with $s > 0$ and real $a$:

$$\frac{1}{\sqrt{s^{1/3}/e^{2a}}} = \Big(\frac{s^{1/3}}{e^{2a}}\Big)^{-1/2}
= \frac{(s^{1/3})^{-1/2}}{(e^{2a})^{-1/2}} = \frac{s^{-1/6}}{e^{-a}}
= \frac{e^{a}}{s^{1/6}}$$

(the root is the power $1/2$ and one over it the power $-1/2$; the power of a
quotient is the quotient of the powers; $(a^p)^q = a^{pq}$ twice; one over
$e^{-a}$ is $e^a$).

**The length factors.** The length of a small step $dx$ along one coordinate
is $\sqrt{|g_{\mu\mu}|}\,dx$. For $0 < z < \pi/2$ both $\sin z$ and $\cos z$
are positive, so the laws apply:

$$\sqrt{\cot^2 z} = \cot z,\quad \sqrt{e^{2a_4}\sin^{1/3} z} = e^{a_4}\sin^{1/6} z,
\quad \sqrt{1} = 1,\quad
\sqrt{b_4 e^{-2a_4}\sin^{1/3} z} = b_4^{1/2} e^{-a_4}\sin^{1/6} z,$$

and their product over the eight directions is

$$\cot z \cdot (e^{a_4}\sin^{1/6} z)^3 \cdot 1 \cdot (b_4^{1/2} e^{-a_4}\sin^{1/6} z)^3
= b_4^{3/2}\, e^{3a_4 - 3a_4}\,\sin^{6/6} z\,\cot z = b_4^{3/2} \sin z \cot z
= b_4^{3/2} \cos z,$$

the record's `sqrtAbsDetG` ($b_4^{3/2} \sin z \cot z$; $\cos z$ at $b_4 = 1$).
The next cell checks the laws and the example with sympy (with symbols declared
positive or real), then reads the metric from the record with the symbols
$S = \sin z$ and $C = \cos z$ (both positive, $\cot z = C/S$) and $b_4$
(positive), takes the square root of the size of every entry (the sign of entry
$\mu$ is $\eta_{\mu\mu}$ of the frame metric), and checks the eight factors and
their product.
```

In [10]:

```
a, b = sp.symbols("a b", positive=True)
p_sym, q_sym = sp.symbols("p q", real=True)
laws = [a ** p_sym * a ** q_sym - a ** (p_sym + q_sym),
        (a ** p_sym) ** q_sym - a ** (p_sym * q_sym),
```

```
        (a * b) ** p_sym - a ** p_sym * b ** p_sym,
        a ** (-p_sym) - 1 / a ** p_sym,
        sp.sqrt(a) - a ** sp.Rational(1, 2)]
check(all(sp.simplify(law) == 0 for law in laws),
      "the five laws of powers hold for positive a, b and real p, q")
y = sp.Symbol("y", real=True)  # a real number of either sign
say(f"sympy: sqrt(y^2) = {sp.sqrt(y ** 2)}; numbers: sqrt((-2)^2) = "
    f"{((-2.0) ** 2) ** 0.5}")
check(sp.sqrt(y ** 2) == sp.Abs(y) and ((-2.0) ** 2) ** 0.5 == 2.0,
      "sqrt(y^2) = |y|: for negative numbers the law (a^p)^q = a^(pq) fails")
s_pos, a_real = sp.Symbol("s", positive=True), sp.Symbol("a", real=True)
example = 1 / sp.sqrt(s_pos ** sp.Rational(1, 3) / sp.exp(2 * a_real))
check(sp.simplify(example - sp.exp(a_real) / s_pos ** sp.Rational(1, 6)) == 0,
      "1/sqrt(s^(1/3)/e^(2a)) = e^a/s^(1/6)")
```

Out [10]:

```
PASS the five laws of powers hold for positive a, b and real p, q
sympy: sqrt(y^2) = Abs(y); numbers: sqrt((-2)^2) = 2.0
PASS sqrt(y^2) = |y|: for negative numbers the law (a^p)^q = a^(pq) fails
PASS 1/sqrt(s^(1/3)/e^(2a)) = e^a/s^(1/6)
```

Text cell:

```
The next cell applies the laws to the author's metric as stored in the
Revision record.
```

In [11]:

```
S, C = sp.symbols("S C", positive=True)  # S = sin z, C = cos z, both > 0
a4 = sp.Symbol("a4", real=True)  # the value of a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of b4(x4) at one time


def from_record(text):
    """The record's Mathematica text as sympy, with sin z = S and cot z = C/S."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "S")
    text = text.replace("Cot[6*H*x0]", "(C/S)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "S": S, "C": C, "E": sp.E})


curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))
eta = json.loads(repository_file("Revision/algebra/gammas.json")
                 .read_text(encoding="utf-8"))["eta"]  # +1 or -1 for x0 ... x7
metric = [from_record(text) for text in curvature["metricDiagonal"]]
sizes = [eta[mu] * metric[mu] for mu in range(8)]  # |g_mu mu| = eta_mu mu g_mu mu
check(all(size.is_positive for size in sizes),
      "every entry of the metric has the sign of eta: eta_mu mu g_mu mu > 0")
factors = [sp.sqrt(size) for size in sizes]  # the length factors
```

```
# the author's order x0, x1, ..., x7
expected = ([C / S] + [sp.exp(a4) * S ** sp.Rational(1, 6)] * 3 + [sp.Integer(1)]
            + [sp.sqrt(b4) * sp.exp(-a4) * S ** sp.Rational(1, 6)] * 3)
for mu in range(8):
    say(f"x{mu}: g = {metric[mu]}, length factor {factors[mu]}")
check(len(factors) == 8 and
      all(sp.simplify(f - e) == 0 for f, e in zip(factors, expected)),
      "the length factors: cot z = C/S for x0, e^a4 S^(1/6) for x1 to x3, 1 for x4, "
      "b4^(1/2) e^(-a4) S^(1/6) for x5 to x7")
product = sp.simplify(sp.Mul(*factors))
record_root = sp.simplify(from_record(curvature["sqrtAbsDetG"]))
say(f"product of the eight factors = {product}; the record's sqrtAbsDetG = "
    f"{record_root}")
lovelock_report = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json")
    .read_text(encoding="utf-8"))
root_verdict = {c["name"]: c["verdict"]
                for c in lovelock_report["checks"]}["sqrt_abs_det_g"]
check(sp.simplify(product - b4 ** sp.Rational(3, 2) * C) == 0
      and sp.simplify(record_root - b4 ** sp.Rational(3, 2) * C) == 0
      and root_verdict == "PASS",
      "the product of the length factors is b4^(3/2) cos z, the record's "
      "sqrt|det g| = b4^(3/2) sin z cot z, for every a4",
      record="Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG; "
             "python-lovelock-report.json, check sqrt_abs_det_g")
```

Out [11]:

```
PASS every entry of the metric has the sign of eta: eta_mu mu g_mu mu > 0
x0: g = C**2/S**2, length factor C/S
x1: g = S**(1/3)*exp(2*a4), length factor S**(1/6)*exp(a4)
x2: g = S**(1/3)*exp(2*a4), length factor S**(1/6)*exp(a4)
x3: g = S**(1/3)*exp(2*a4), length factor S**(1/6)*exp(a4)
x4: g = -1, length factor 1
x5: g = -S**(1/3)*b4*exp(-2*a4), length factor S**(1/6)*sqrt(b4)*exp(-a4)
x6: g = -S**(1/3)*b4*exp(-2*a4), length factor S**(1/6)*sqrt(b4)*exp(-a4)
x7: g = -S**(1/3)*b4*exp(-2*a4), length factor S**(1/6)*sqrt(b4)*exp(-a4)
PASS the length factors: cot z = C/S for x0, e^a4 S^(1/6) for x1 to x3, 1 for x4,
    b4^(1/2) e^(-a4) S^(1/6) for x5 to x7
product of the eight factors = C*b4**(3/2); the record's sqrtAbsDetG = C*b4**(3/2)
PASS the product of the length factors is b4^(3/2) cos z, the record's sqrt|det g| =
    b4^(3/2) sin z cot z, for every a4
     reproduces Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG; python-
    lovelock-report.json, check sqrt_abs_det_g
```

Text cell:

```
## 11. The exponential function, the logarithm and logarithmic axes

The number $e$ is the limit of $(1 + 1/n)^n$ for ever larger $n$ (the growth of
1 unit of money at the interest rate 100 per cent per year, paid in $n$ parts).
The error $e - (1 + 1/n)^n$ is about $e/(2n)$ for large $n$. With 50-digit
```

```
arithmetic this is what one sees. With floating-point numbers the formula
breaks down for large $n$: $1 + 1/n$ is rounded (by up to $\varepsilon/2$),
and the power $n$ multiplies that rounding error by about $n$; for
$n = 10^{16}$, $1 + 1/n$ is rounded to exactly 1, and $1^n = 1$.

The logarithm undoes the exponential, $\ln(e^t) = t$, and turns products into
sums, $\ln(xy) = \ln x + \ln y$ for positive $x, y$ (because
$e^{\ln x + \ln y} = e^{\ln x}e^{\ln y} = xy$). In the author's metric the
length factors contain $e^{a_4}$ (space) and $e^{-a_4}$ (extra times). When
$a_4$ grows by $\ln 2 \approx 0.693$, $e^{a_4}$ doubles and
$e^{-a_4}$ halves: $e^{-(a_4 + \ln 2)} = e^{-a_4} e^{-\ln 2} = e^{-a_4}/2$. To
shrink the extra times by a factor 1000, $a_4$ must grow by
$\ln 1000 \approx 6.91$.

The next cell computes the errors for $n = 1, 10, \dots, 10^{16}$ both ways,
checks the $e/(2n)$ law (within 1 per cent for $n \geq 1000$) and the
breakdown of the floating-point version, and checks the laws of the logarithm.
```

In [12]:

```
mpmath.mp.dps = 50
n_values = [10 ** k for k in range(17)]
error_exact = [abs((1 + mpmath.mpf(1) / n) ** n - mpmath.e) for n in n_values]
error_float = [abs((1.0 + 1.0 / n) ** n - math.e) for n in n_values]
for k in (0, 3, 6, 9, 12, 16):
    say(f"n = 10^{k:2d}: 50 digits: error {mpmath.nstr(error_exact[k], 3):>8}; "
        f"floating point: error {error_float[k]:.1e}")
ratios = [error_exact[k] * n_values[k] / (mpmath.e / 2) for k in range(3, 17)]
check(all(abs(r - 1) < mpmath.mpf("0.01") for r in ratios),
      "50 digits: the error of (1 + 1/n)^n is e/(2n) within 1 per cent for n >= 1000")
check(error_float[16] == math.e - 1 and error_float[16] > 1e3 * error_float[6],
      "floating point: for n = 10^16, 1 + 1/n is rounded to 1, and the error grows "
      "to e - 1")
t_sym = sp.Symbol("t", real=True)
u, w = sp.symbols("u w", positive=True)
check(sp.log(sp.exp(t_sym)) == t_sym
      and sp.expand_log(sp.log(u * w)) == sp.log(u) + sp.log(w)
      and sp.simplify(sp.exp(-(a4 + sp.log(2))) - sp.exp(-a4) / 2) == 0,
      "ln(e^t) = t, ln(u w) = ln u + ln w, and e^(-(a4 + ln 2)) = e^(-a4)/2")
report("growth of a4 that shrinks the extra times by a factor 1000",
       f"{math.log(1000):.4f}")
```

Out [12]:

```
n = 10^ 0: 50 digits: error    0.718; floating point: error 7.2e-01
n = 10^ 3: 50 digits: error  0.00136; floating point: error 1.4e-03
n = 10^ 6: 50 digits: error  1.36e-6; floating point: error 1.4e-06
n = 10^ 9: 50 digits: error  1.36e-9; floating point: error 2.2e-07
n = 10^12: 50 digits: error 1.36e-12; floating point: error 2.4e-04
n = 10^16: 50 digits: error 1.36e-16; floating point: error 1.7e+00
PASS 50 digits: the error of (1 + 1/n)^n is e/(2n) within 1 per cent for n >= 1000
PASS floating point: for n = 10^16, 1 + 1/n is rounded to 1, and the error grows to e - 1
```

```
PASS ln(e^t) = t, ln(u w) = ln u + ln w, and e^(-(a4 + ln 2)) = e^(-a4)/2
RESULT growth of a4 that shrinks the extra times by a factor 1000 = 6.9078
```

Text cell:

```
The next cell draws the errors of the limit for $e$ (figure 5) and the two
factors $e^{a_4}$ and $e^{-a_4}$ on an ordinary and on a logarithmic vertical
axis (figure 6), with the points $a_4 = 0, \ln 2, 2\ln 2, \dots$ where they
double and halve.
```

In [13]:

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.loglog(n_values, [float(e) for e in error_exact], "o-", color="tab:blue",
          label="50-digit arithmetic")
ax.loglog(n_values, error_float, "s-", color="tab:red", ms=5,
          label="floating point")
ax.loglog(n_values, [math.e / (2 * n) for n in n_values], "k--", lw=1,
          label="$e/(2n)$")
ax.set_xlabel("$n$")
ax.set_ylabel("error $|(1 + 1/n)^n - e|$")
ax.set_title("The limit that defines $e$")
ax.legend(fontsize=8, loc="lower left");
save_figure(fig, "limit_for_e",
            "The error of $(1 + 1/n)^n$ as an approximation of $e$ for $n = 1$ to "
            "$10^{16}$, both axes logarithmic. Blue: with 50-digit arithmetic the "
            "error falls like $e/(2n)$ (dashed line). Red: with floating-point "
            "numbers it falls only until $n$ is about $10^8$; then the rounding of "
            "$1 + 1/n$, multiplied by the power $n$, takes over, and at $n = "
            "10^{16}$ the result is exactly 1, an error of $e - 1$.")
a4_values = np.linspace(0.0, 4.0, 401)
marks = np.log(2.0) * np.arange(6)  # a4 = 0, ln 2, 2 ln 2, ..., 5 ln 2
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
for ax in (left, right):
    ax.plot(a4_values, np.exp(a4_values), color="tab:red",
            label="$e^{a_4}$: space $x_1, x_2, x_3$")
    ax.plot(a4_values, np.exp(-a4_values), color="tab:blue",
            label="$e^{-a_4}$: extra times $x_5, x_6, x_7$")
    ax.plot(marks, np.exp(marks), "o", color="tab:red", ms=4)
    ax.plot(marks, np.exp(-marks), "o", color="tab:blue", ms=4)
    ax.set_xlabel("$a_4$")
left.set_ylabel("factor (ordinary axis)")
left.set_title("ordinary vertical axis")
left.legend(fontsize=8, loc="upper left")
right.set_yscale("log")
right.set_ylabel("factor (logarithmic axis)")
right.set_title("logarithmic vertical axis: straight lines")
for k in range(1, 6):
    right.text(marks[k], np.exp(-marks[k]) * 0.55, f"$2^{{-{k}}}$", ha="center",
               fontsize=8, color="tab:blue")
fig.tight_layout()
save_figure(fig, "growth_and_deflation",
```

```
"The factors $e^{a_4}$ (red), which multiply the lengths along the three "
"space directions, and $e^{-a_4}$ (blue), which multiply the lengths "
"along the three extra times, for $a_4$ from 0 to 4 (the factor "
"$\\sin^{1/6} z$, which depends on the hidden coordinate $x_0$, is "
"left out); horizontal axis $a_4$, vertical axis "
"the factor (a pure number), ordinary on the left and logarithmic on "
"the right. The dots mark $a_4 = 0, \\ln 2, 2\\ln 2, \\dots$, where "
"$e^{a_4}$ doubles and $e^{-a_4}$ halves. On the logarithmic axis both "
"are straight lines, one rising and one falling equally fast: the "
"extra times deflate exponentially as $a_4$ grows.")
```

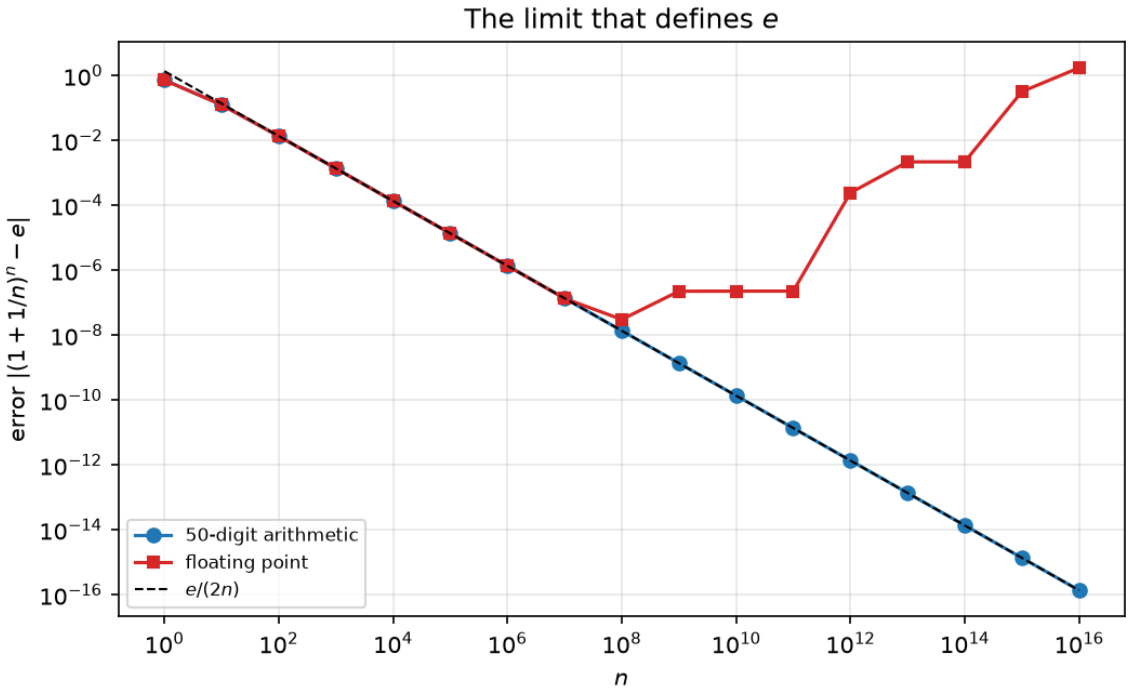

Figure 21: The error of $(1+1/n)^n$ as an approximation of e for $n = 1$ to 10^{16}, both axes logarithmic. Blue: with 50-digit arithmetic the error falls like $e/(2n)$ (dashed line). Red: with floating-point numbers it falls only until n is about 10^8; then the rounding of $1 + 1/n$, multiplied by the power n, takes over, and at $n = 10^{16}$ the result is exactly 1, an error of $e - 1$. (Notebook 01f, figure 5.)

Out [13]:

```
Figure 01f.5 saved as Revision/textbook/figures/01f_5_limit_for_e.png
```

Out [13]:

```
Figure 01f.6 saved as Revision/textbook/figures/01f_6_growth_and_deflation.png
```

Text cell:

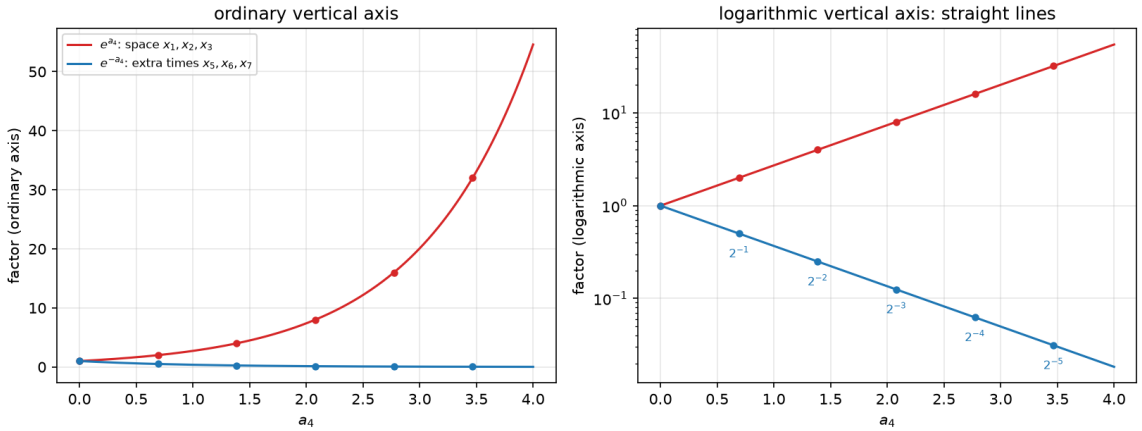

Figure 22: The factors e^{a_4} (red), which multiply the lengths along the three space directions, and e^{-a_4} (blue), which multiply the lengths along the three extra times, for a_4 from 0 to 4 (the factor $\sin^{1/6} z$, which depends on the hidden coordinate x_0, is left out); horizontal axis a_4, vertical axis the factor (a pure number), ordinary on the left and logarithmic on the right. The dots mark $a_4 = 0, \ln 2, 2\ln 2, \ldots$, where e^{a_4} doubles and e^{-a_4} halves. On the logarithmic axis both are straight lines, one rising and one falling equally fast: the extra times deflate exponentially as a_4 grows. (Notebook 01f, figure 6.)

```
## 12. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [14]:

```
figure_names = ["01f_1_decimal_periods.png", "01f_2_square_root_of_two.png",
                "01f_3_float_spacing.png", "01f_4_cancellation.png",
                "01f_5_limit_for_e.png", "01f_6_growth_and_deflation.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

Out [14]:

```
PASS all 6 figure files of this notebook exist
ALL 26 CHECKS PASSED (notebook 01f)
```

Text cell:

```
## 13. What this notebook showed

- Whole numbers and fractions are computed exactly by Python ($2^{100}$, $16!$,
  $1/3 + 1/6 = 1/2$, $84/126 = 2/3$).
- The decimal digits of a fraction end or repeat, with a period of at most
```

```
  $q - 1$; they end exactly when $q = 2^a 5^b$.
- $\sqrt 2$ is not a fraction (proved); bisection pins it between exact
  fractions to $2^{-60}$, and the fractions $1/1, 3/2, 7/5, \dots$ with
  $p^2 - 2q^2 = \pm 1$ approach it with errors close to $1/(2\sqrt2\,q^2)$.
- A computer stores real numbers as floating-point numbers with a relative
  precision of about $\varepsilon = 2^{-52}$: $0.1 + 0.2 \neq 0.3$, and above
  $2^{53}$ whole numbers are skipped. Subtracting nearly equal numbers loses
  digits (cancellation); rewriting $1 - \cos x$ as $2\sin^2(x/2)$ cures it.
- The laws of powers hold for positive bases, and $\sqrt{y^2} = |y|$. Applied
  to the author's metric of the Revision record they give the length factors
  $\cot z$ (hidden direction), $e^{a_4}\sin^{1/6} z$ (space), 1 (time) and
  $b_4^{1/2} e^{-a_4}\sin^{1/6} z$ (the exponentially deflating extra times),
  whose product is $b_4^{3/2} \cos z$, the record's $\sqrt{|\det g|}$, for every
  $a_4$ ($\cos z$ at $b_4 = 1$).
- $e$ is the limit of $(1 + 1/n)^n$, with the error $e/(2n)$; floating point
  breaks this limit for $n$ beyond about $10^8$. A growth of $a_4$ by $\ln 2$
  doubles $e^{a_4}$ and halves $e^{-a_4}$; on a logarithmic axis both are
  straight lines.
```

1.9 Line-by-line walk-through of Notebook 01f

The notebook has fourteen code cells, In [1] to In [14]. This section explains every line of every one of them, in order. A line that starts with `#` is a **comment**: Python skips it; it is there for the reader. A comment may also stand at the end of a line of code, after the sign `#`. Where a quoted cell ends with a long figure caption, the remaining lines of the caption are replaced here by `...`); every caption is printed in full under its figure in Section 1.8.

In [1], the set-up cell. Its first part repeats the complete run instructions of Section 1.7 as comment lines, so that the notebook file carries its own instructions. The code starts below the line of `=` signs that reads THE SET-UP. This code is the same in every notebook of the book except for one line, the notebook's name. It is explained here once; the walk-throughs of the other six notebooks of this chapter refer back to this explanation.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json` reads and writes the text format JSON, in which the Revision record stores its results as names and values; `os` gives access to the computer's environment; `textwrap` breaks long text into lines; `from pathlib import Path` takes the single name `Path` out of the module `pathlib`. A `Path` is the address of a file or folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`,

and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python code), which show a picture file below a cell.

```
NOTEBOOK_ID = "01f"  # this notebook: chapter 01, example f
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"01f"`. It is the only line of the set-up code that differs between notebooks; the figure files and the captions file are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. The text in triple quotes under the `def` line is its **docstring**, a description for the reader. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. `here.parents` is the parent folder, its parent, and so on up to the top of the disk; `[here, *here.parents]` is the list that starts with `here` and continues with all of them (the star unpacks the parents into the list). The `for` loop takes the folders of this list one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If no folder qualifies, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The first code line calls the function and names its result `REPO`. The second chooses the folder below which the notebook writes its files. `os.environ` holds the **environment variables** of the running program (named texts that a program receives from the computer); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and the default `str(REPO)` (the repository folder as a string) otherwise. When you run the notebook the variable is not set, so the files go into the repository; the book's checking tool sets it to a scratch folder, so that a check never changes the repository.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` gives the full path of a file of the repository, for reading. `output_file("Revision/...")` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder that will hold it, together with any missing folder above it, and does nothing if the folder exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, so that every printed line fits the width of a page of the book; `textwrap.fill` breaks the text at blanks, and every line after the first starts with four blanks. `str(text)` turns any value into a string first.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

matplotlib reads personal settings from a file on your computer if you have one; `matplotlib.rcdefaults()` returns to the built-in settings, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets four settings for every figure: its size (7.0 inches wide, 4.2 inches high), the size of its letters (10 points), and a faint grid behind the curves (`grid.alpha` 0.3: 30 per cent opaque). The braces `{...}` make a **dictionary**: pairs of a key and a value, written `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/01f.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes the text `{}` and a line end into the captions file: an empty list of captions, which `save_figure` fills. `encoding="utf-8"` fixes how the letters are stored as bytes.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

This function saves a figure and shows it. `setdefault(name, value)` returns the number already stored for this figure name, or stores and returns `len(FIGURE_NUMBERS) + 1` (one more than the number of figures so far) for a new name; so the figures are numbered 1, 2, 3, ... and a cell run twice keeps the number of its figure. The file name is the notebook's name, the number and the figure's name, for example `01f_1_decimal_periods.png`. `fig.savefig` writes the picture as a PNG file with 150 dots per inch, cuts away the empty margin and stores no program name in the file, so that two runs write exactly the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time at the end of the cell. The caption is stored in `CAPTIONS`, and the whole dictionary is written into the captions file (`json.dumps` turns it into JSON text with one entry per line, the keys sorted). `display(Image(...))` shows the saved picture below the cell; the `metadata` entry tells the book's tools which file the picture is. The last line prints where the figure was saved, for example `Figure 01f.1 saved as Revision/textbook/figures/01f_1_decimal_periods.png`.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection, written with square brackets). `check` is the function behind every check of the book. `condition` is a statement that is either true (`True`) or false (`False`).

If it is false, `raise AssertionError(...)` stops the notebook with an error that names the check. (Python's own statement `assert` would do the same, but Python started with the option that optimises the code skips every `assert`; an `if` statement is never skipped.) If it is true, the name is appended to `PASSED` and the line PASS followed by the name is printed. `record=None` makes the third argument optional: a check that reproduces a number of the Revision record passes the record's file and check name, and `check` prints them on a second line that starts with reproduces.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds a blank and the unit when a unit is given and nothing otherwise. `all_checks_passed` prints the last line of every notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one output line of In [1]: Set-up of notebook 01f complete: repository folder found, helpers defined. (Two strings written next to each other, as here on two lines, are joined by Python into one.)

In [2], whole numbers and fractions.

```
import fractions  # exact fractions p/q
import math  # factorial, gcd, isqrt, e, log

import mpmath  # numbers with many digits
import numpy as np  # arrays of floating-point numbers
import sympy as sp  # exact algebra with symbols
```

The module `fractions` (part of Python) computes with exact fractions; `math` holds functions of single numbers: `math.factorial`, `math.gcd` (the greatest common divisor), `math.isqrt` (the whole part of a square root, computed exactly), the number `math.e` and the logarithm `math.log`. The package mpmath computes with as many decimal digits as requested; numpy (short name `np`) computes with **arrays**, lists of floating-point numbers on which arithmetic acts entry by entry; sympy (short name `sp`) does exact algebra with symbols.

```
big = 2 ** 100
say(f"2^100 = {big} ({len(str(big))} digits)")
say(f"16! = {math.factorial(16)}")
check(big == 1267650600228229401496703205376 and len(str(big)) == 31
      and math.factorial(16) == 20922789888000,
      "2^100 and 16! are computed exactly")
```

In Python ** means "to the power", so 2 ** 100 is 2^{100}. str(big) writes the number as a string of digits and len(...) counts them. The check compares both numbers with their exact values, digit for digit (the operator == tests equality, and requires both conditions). Output: 2^100 = 1267650600228229401496703205376 (31 digits), 16! = 20922789888000, and the PASS line.

```
third, sixth = fractions.Fraction(1, 3), fractions.Fraction(1, 6)
say(f"1/3 + 1/6 = {third + sixth}")
check(third + sixth == fractions.Fraction(1, 2), "1/3 + 1/6 = 1/2 exactly")
```

fractions.Fraction(1, 3) is the exact number 1/3; the first line names two fractions at once (Python assigns the values on the right to the names on the left in order). Their sum is computed exactly as in Section 1.2 and printed as 1/2; the check confirms it.

```
reduced = fractions.Fraction(84, 126)  # Fraction always cancels common factors
say(f"gcd(84, 126) = {math.gcd(84, 126)}, 84/126 = {reduced}")
check(math.gcd(84, 126) == 42 and reduced == fractions.Fraction(2, 3)
      and (reduced.numerator, reduced.denominator) == (2, 3),
      "84/126 = 2/3 in lowest terms, with gcd(84, 126) = 42")
```

A Fraction is always stored in lowest terms, so Fraction(84, 126) becomes 2/3; .numerator and .denominator read its two parts. The check confirms $\gcd(84, 126) = 42$ and the lowest terms of Section 1.2. Output: gcd(84, 126) = 42, 84/126 = 2/3 and the PASS line.

In [3], long division.

```
def decimal_digits(p, q):
    """The decimal digits of p/q (0 < p < q) by long division: the digits before
    the repeating block, and the repeating block (empty if the division ends)."""
    digits = []
    first_seen = {}  # remainder -> the place of the digit it produced first
    remainder = p
    while remainder != 0 and remainder not in first_seen:
        first_seen[remainder] = len(digits)
        remainder *= 10
        digits.append(remainder // q)  # the next digit
        remainder %= q  # what is left
    if remainder == 0:
        return digits, []
    start = first_seen[remainder]  # the remainder came back: the digits repeat
    return digits[:start], digits[start:]
```

This is long division exactly as in Section 1.2. digits collects the digits; the dictionary first_se en remembers, for each remainder met so far, the place of the digit that it produced. The while loop runs as long as its condition is true: the remainder is not 0 (!= means "is not equal to") and has not been seen before. Inside, the present remainder is recorded with its place, multiplied by 10 (remainder *= 10 is short for remainder = remainder * 10), the whole part of the division by q (//, division without the remainder) is appended as the next digit, and remainder %= q keeps what is left (% gives the remainder of a division). When the loop stops, either the remainder is 0 (the division has ended, and the function returns the digits and an empty block []), or a remainder

has come back; then the digits from its first place on repeat. `digits[:start]` is the part of the list before place `start`, `digits[start:]` the part from there on. The function returns both lists.

```
def from_digits(head, block):
    """The fraction with the digits head followed by block repeated for ever."""
    k, m = len(head), len(block)
    D = int("".join(map(str, head)) or "0")
    value = fractions.Fraction(D, 10 ** k)
    if block:
        C = int("".join(map(str, block)))
        value += fractions.Fraction(C, 10 ** k * (10 ** m - 1))
    return value
```

This function turns the digits back into the fraction with the formula $D/10^k + C/(10^k(10^m - 1))$ of Section 1.2. `map(str, head)` turns every digit into a one-letter string, `"".join(...)` glues them into one string, and `int(...)` reads it as the whole number D; when there are no digits before the block, the empty string is replaced by `"0"` (`or` takes its right side when the left side is empty). `if block:` is true when the block is not empty; then the block's whole number C is made the same way and its value is added.

```
all_back = True
for p, q in [(1, 4), (1, 3), (1, 6), (1, 7), (5, 12), (1, 97)]:
    head, block = decimal_digits(p, q)
    head_text = "".join(map(str, head))
    block_text = "(" + "".join(map(str, block)) + ")" if block else ""
    text = "0." + head_text + block_text
    if len(text) > 40:  # 1/97 has a block of 96 digits: show its length only
        text = f"0.({len(block)} repeating digits)"
    say(f"{p}/{q} = {text}, period {len(block)}")
    all_back &= from_digits(head, block) == fractions.Fraction(p, q)
check(decimal_digits(1, 7) == ([], [1, 4, 2, 8, 5, 7]) and all_back,
      "1/7 = 0.(142857), and the digits of all six examples give the fractions back")
```

The loop takes six fractions. For each it computes the digits, writes them as text with the repeating block in brackets (`... if block else ""`: the bracketed block when there is one, nothing otherwise), and prints the result with its period, the length of the block. For 1/97 the text would be too long for a line, so only the length 96 is printed. `all_back &= ...` is short for `all_back = all_back and ...`: it stays true only if every fraction is recovered from its digits. The check also compares the digits of 1/7 with the hand calculation. Output: $1/4 = 0.25$ (period 0), $1/3 = 0.(3)$, $1/6 = 0.1(6)$, $1/7 = 0.(142857)$ (period 6), $5/12 = 0.41(6)$, 1/97 with 96 repeating digits, and the PASS line.

In [4], which fractions end, and the plot of the periods.

```
def only_twos_and_fives(q):
    """True if q has no prime factor other than 2 and 5."""
    for factor in (2, 5):
        while q % factor == 0:
            q //= factor
    return q == 1
```

The function divides q by 2 as long as it can (`q % factor == 0`: the remainder is 0), then by 5 as long as it can; `q //= factor` is short for `q = q // factor`. If 1 is left, q had no other prime factor.

```
qs = list(range(2, 301))
periods = [len(decimal_digits(1, q)[1]) for q in qs]
check(all((period == 0) == only_twos_and_fives(q) for q, period in zip(qs, periods))
      and all(period <= q - 1 for q, period in zip(qs, periods)),
      "1/q ends exactly when q = 2^a 5^b, and the period is at most q - 1 "
      "(q = 2 to 300)")
```

`range(2, 301)` counts from 2 up to 300 (the end 301 is not included), and `list(...)` makes it a list. The second line is a **list comprehension**: for every q it computes the digits of $1/q$, takes the block (entry `[1]` of the pair returned by `decimal_digits`) and its length. `zip(qs, periods)` pairs each q with its period. `all(...)` is true when the condition holds for every pair: the period is 0 exactly when $q = 2^a 5^b$ (the theorem of Section 1.2), and it never exceeds $q - 1$.

```
full = [q for q, period in zip(qs, periods) if period == q - 1]
say(f"q with the full period q - 1: {full[:12]} ... ({len(full)} of them)")
```

`full` collects the denominators with the largest possible period $q - 1$; the line prints the first twelve (`full[:12]`) and their number: 7, 17, 19, 23, 29, 47, 59, 61, 97, 109, 113, 131, ... (23 of them).

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
ax.plot(qs, periods, ".", color="tab:blue", label="period of $1/q$")
ax.plot(full, [q - 1 for q in full], "o", color="tab:red", fillstyle="none",
        label="full period $q - 1$")
ax.plot([2, 300], [1, 299], "k:", lw=1, label="the largest possible: $q - 1$")
ax.set_xlabel("denominator $q$")
ax.set_ylabel("length of the repeating block")
ax.set_title("Decimal expansion of $1/q$: the period")
ax.legend(loc="upper left", fontsize=8);
save_figure(fig, "decimal_periods",
            "The length of the repeating block (the period) of the decimal digits "
            ...)
```

`plt.subplots(figsize=(8.0, 4.4))` makes a new figure `fig`, 8 inches wide and 4.4 high, with one pair of axes `ax`. `ax.plot(xs, ys, style)` draws the points with these coordinates: `"."` small dots (the periods), `"o"` with `fillstyle="none"` open red circles (the full periods), and `"k:"` a black (`k`) dotted (`:`) line of width 1 (`lw=1`) from $(2, 1)$ to $(300, 299)$, the line $q - 1$. `label=` names a curve in the legend; text between dollar signs is typeset as mathematics. The next three lines label the axes and set the title; `ax.legend(...)` draws the legend in the upper left corner with small letters. The semicolon at the end of that line keeps Jupyter from printing the value of the line (a description of the legend with a memory address, which would differ from run to run). `save_figure` saves Figure 01f.1 with its caption. What Figure 01f.1 shows: every dot lies on or below the dotted line, the dots at 0 are the denominators $2^a 5^b$, and the red circles are the primes that reach the bound $q - 1$.

In [5], the square root of 2 is not a fraction; fractions that come close.

```
# isqrt(n) is the whole part of the square root of n, computed exactly:
no_solution = all(math.isqrt(2 * q * q) ** 2 != 2 * q * q for q in range(1, 10001))
check(no_solution, "p^2 = 2 q^2 has no solution in whole numbers with q <= 10000")
```

If $p^2 = 2q^2$ had a solution, p would be the whole number isqrt$(2q^2)$ and its square would equal $2q^2$. The line tests this for every q from 1 to 10000 with exact whole numbers and finds no solution, as the proof of Section 1.3 says.

```
mpmath.mp.dps = 50  # 50 significant digits
root2 = mpmath.sqrt(2)
convergents = [(1, 1)]
while len(convergents) < 12:
    p, q = convergents[-1]
    convergents.append((p + 2 * q, p + q))  # the rule p' = p + 2q, q' = p + q
```

`mpmath.mp.dps = 50` makes mpmath compute with 50 significant decimal digits (`dps`: decimal places), so `root2` is $\sqrt{2}$ to 50 digits. The list `convergents` starts with the pair $(1, 1)$; the loop takes the last pair (`[-1]` is the last entry of a list), applies the rule of Section 1.3 and appends the new pair, until there are 12 pairs.

```
for p, q in convergents:
    error = abs(mpmath.mpf(p) / q - root2)
    say(f"{p}/{q}: p^2 - 2 q^2 = {p * p - 2 * q * q:+d}, error "
        f"{mpmath.nstr(error, 3)}, error q^2 = {mpmath.nstr(error * q * q, 6)}")
```

For each fraction, `mpmath.mpf(p)` makes p an mpmath number, so that p/q is computed to 50 digits; `abs` is the absolute value. The printed line gives $p^2 - 2q^2$ with its sign (`:+d` prints a whole number with a sign), the error with 3 significant digits (`mpmath.nstr(x, 3)`) and the error times q^2 with 6. The output shows the signs $-1, +1, -1, \ldots$ and the error times q^2 settling at 0.353553 from 577/408 on.

```
signs = [p * p - 2 * q * q for p, q in convergents]
check(signs == [(-1) ** (k + 1) for k in range(12)],
      "p^2 - 2 q^2 = -1, +1, -1, ... for the 12 fractions")
p, q = convergents[-1]
limit_value = 1 / (2 * root2)
check(abs(abs(mpmath.mpf(p) / q - root2) * q * q - limit_value) < mpmath.mpf("1e-6"),
      "the error of p/q times q^2 approaches 1/(2 sqrt 2) = 0.35355")
```

The first check compares the list of the values $p^2 - 2q^2$ with $(-1)^{k+1}$ for $k = 0, \ldots, 11$, that is $-1, +1, -1, \ldots$, as derived in Section 1.3. The second takes the last fraction, 19601/13860, and checks that its error times q^2 differs from $1/(2\sqrt{2}) = 0.35355\ldots$ by less than 10^{-6} (`mpmath.mpf("1e-6")` is 10^{-6} as an mpmath number).

In [6], bisection, and the best fractions with each denominator.

```
low, high = fractions.Fraction(1), fractions.Fraction(2)  # 1^2 < 2 < 2^2
widths = [high - low]
```

```
for step in range(60):
    middle = (low + high) / 2
    if middle * middle < 2:
        low = middle  # the root is in the right half
    else:
        high = middle  # the root is in the left half
    widths.append(high - low)
```

This is bisection as described in Section 1.3, with exact fractions: the interval starts as $[1, 2]$; each of the 60 passes computes the middle, keeps the half that contains $\sqrt{2}$ (`if ... else` runs the first block when the condition is true and the second otherwise), and records the new width.

```
low_mp = mpmath.mpf(low.numerator) / low.denominator
high_mp = mpmath.mpf(high.numerator) / high.denominator
say(f"after 60 halvings: {mpmath.nstr(low_mp, 20)} < sqrt 2 < "
    f"{mpmath.nstr(high_mp, 20)}")
say(f"sqrt 2 with 30 digits: {mpmath.nstr(root2, 30)}")
check(low * low < 2 < high * high and high - low == fractions.Fraction(1, 2 ** 60)
      and low_mp < root2 < high_mp,
      "60 halvings: exact fractions low < sqrt 2 < high with high - low = 2^(-60)")
```

The two ends are turned into 50-digit numbers and printed with 20 significant digits, and $\sqrt{2}$ itself with 30 digits:

```
after 60 halvings: 1.4142135623730950483 < sqrt 2 < 1.4142135623730950492
sqrt 2 with 30 digits: 1.41421356237309504880168872421
```

Python allows chained comparisons, `a < b < c`, meaning $a < b$ and $b < c$. The check confirms with exact fractions that $\text{low}^2 < 2 < \text{high}^2$ and that the width is exactly 2^{-60}, and with 50-digit numbers that $\sqrt{2}$ lies between the ends.

```
denominators = np.arange(1, 1001)
best_p = np.rint(denominators * np.sqrt(2.0))  # the nearest numerator for each q
best_error = np.abs(best_p / denominators - np.sqrt(2.0))
```

`np.arange(1, 1001)` is the array of the whole numbers 1 to 1000. For each denominator q the best numerator is the whole number nearest to $q\sqrt{2}$ (`np.rint` rounds to the nearest whole number), and `best_error` is the array of the 1000 errors $|p/q - \sqrt{2}|$, computed in floating point (good enough for a picture).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
left.semilogy(range(61), [float(w) for w in widths], "o-", ms=3)
left.set_xlabel("number of halvings")
left.set_ylabel("width of the interval (logarithmic scale)")
left.set_title("bisection: $\\sqrt{2}$ between two fractions")
```

`plt.subplots(1, 2, ...)` makes a figure with two pairs of axes side by side, named `left` and `right`. `semilogy` draws with a logarithmic vertical axis; the widths (turned into floating-point

numbers with `float`) are drawn against the step number as dots joined by lines (`"o-"`, marker size `ms=3`). In a string in Python a backslash has a special meaning, so the LaTeX command `\sqrt` is written with two backslashes.

```
right.loglog(denominators, best_error, ".", ms=3, color="0.6",
             label="best $p/q$ for each $q$")
conv_q = [q for _, q in convergents if q <= 1000]
conv_error = [float(abs(mpmath.mpf(p) / q - root2)) for p, q in convergents
              if q <= 1000]
right.loglog(conv_q, conv_error, "o", color="tab:red",
             label="$1/1, 3/2, 7/5, 17/12, \\dots$")
right.loglog(denominators, 1 / (2 * np.sqrt(2.0) * denominators ** 2.0), "k--",
             lw=1, label="$1/(2\\sqrt{2}\\,q^2)$")
right.set_xlabel("denominator $q$")
right.set_ylabel("error $|p/q - \\sqrt{2}|$")
right.set_title("the best fractions near $\\sqrt{2}$")
right.legend(fontsize=8, loc="lower left")
fig.tight_layout()
save_figure(fig, "square_root_of_two",
            "Left: the width of the interval between two fractions that contains "
            ...)
```

On the right panel, `loglog` makes both axes logarithmic. The 1000 best errors are drawn as grey dots (`color="0.6"` is a grey of brightness 60 per cent); the fractions of In [5] with $q \le 1000$ (the name `_` marks a value that is not used) as red circles; and the line $1/(2\sqrt{2}\,q^2)$ as a black dashed line (the style string made of the letter k and two hyphens). `fig.tight_layout()` arranges the two panels so that their labels do not overlap. What Figure 01f.2 shows: on the left a straight falling line (each step halves the width); on the right no error is zero, and the red fractions lie on the dashed line, as close to $\sqrt{2}$ as $1/q^2$ allows.

In [7], floating-point numbers.

```
total = 0.1 + 0.2
stored = fractions.Fraction(0.1)  # the exact value of the floating-point number 0.1
say(f"0.1 + 0.2 = {total:.17f} (17 digits), and 0.3 = {0.3:.17f}")
say(f"0.1 is stored as {stored.numerator}/{stored.denominator}")
check(total != 0.3 and stored != fractions.Fraction(1, 10)
      and stored.denominator == 2 ** 55,
      "0.1 + 0.2 is not 0.3 in floating point, and 0.1 is stored as a fraction "
      "with the denominator 2^55")
```

`fractions.Fraction(0.1)` reads the exact value of the floating-point number that Python stores for 0.1. `{total:.17f}` prints a number with 17 digits after the point. Output: 0.1 + 0.2 = 0.30000000000000004 and 0.3 = 0.29999999999999999 (the stored values, as Section 1.4 explains), and 0.1 is stored as 3602879701896397/36028797018963968, whose denominator is 2^{55}. The check confirms all three statements.

```
eps = float(np.finfo(float).eps)  # machine epsilon
say(f"machine epsilon = {eps!r} = 2^-52: {eps == 2.0 ** -52}")
check(eps == 2.0 ** -52 and 1.0 + eps != 1.0 and 1.0 + eps / 2 == 1.0,
```

```
      "1 + eps is the next number after 1; 1 + eps/2 is rounded back to 1")
check(float(2 ** 53 + 1) == float(2 ** 53) and float(2 ** 53 - 1) != float(2 ** 53),
      "2^53 + 1 cannot be stored: it is rounded to 2^53")
```

`np.finfo(float).eps` is machine epsilon as numpy knows it; `{eps!r}` prints it with all its digits, 2.220446049250313e-16 (the letter e followed by a number means "times ten to the power": `e-16` is 10^{-16}), and the comparison with `2.0 ** -52` prints True. The first check is the statement of Section 1.4 about $1 + \varepsilon$ and $1 + \varepsilon/2$. The second converts the exact whole numbers $2^{53} + 1$ and $2^{53} - 1$ to floating point (`float(...)`): the first is rounded to 2^{53}, the second is stored exactly.

```
xs = np.logspace(-3, 20, 2301)  # 2301 numbers from 0.001 to 10^20
fraction_part, exponent = np.frexp(xs)  # xs = fraction_part * 2^exponent
predicted = np.ldexp(1.0, exponent - 53)  # 2^(exponent - 53)
check(bool(np.all(np.spacing(xs) == predicted))
      and bool(np.all(np.spacing(xs) <= eps * xs)),
      "the step after x is 2^(e - 53), never more than eps x (2301 values of x)")
```

`np.logspace(-3, 20, 2301)` makes 2301 numbers from 10^{-3} to 10^{20} that are equally spaced on a logarithmic axis. `np.frexp` writes each as $f \cdot 2^e$ with $\frac{1}{2} \le f < 1$ and returns the arrays of f and e; `np.ldexp(1.0, e - 53)` computes $1 \cdot 2^{e-53}$, the step predicted in Section 1.4. `np.spacing(x)` is the actual distance from x to the next floating-point number. The check requires the two to agree exactly at all 2301 numbers, and the step never to exceed εx. (`np.all` is numpy's version of `all`; `bool(...)` turns its answer into a plain True or False.)

In [8], the picture of the spacing.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4))
big_x = np.logspace(48, 57, 2000, base=2.0)  # from 2^48 to 2^57
left.loglog(big_x, np.spacing(big_x), color="tab:blue", lw=1.5, base=2,
            label="step to the next floating-point number")
left.axvline(2.0 ** 53, color="tab:red", lw=1, label="$x = 2^{53}$")
left.axhline(1.0, color="tab:red", ls=":", lw=1, label="step 1")
left.set_xlabel("number $x$ (logarithmic axis, base 2)")
left.set_ylabel("step size (logarithmic axis, base 2)")
left.set_title("the step doubles at every power of 2")
left.legend(fontsize=8, loc="upper left")
```

The left panel uses 2000 numbers from 2^{48} to 2^{57} (`base=2.0`: equally spaced on a logarithmic axis of base 2) and draws the step against x with both axes logarithmic in base 2. `axvline` draws a vertical line at $x = 2^{53}$, `axhline` a horizontal dotted line (`ls=":"`) at the step 1.

```
right.semilogx(xs, np.spacing(xs) / xs, color="tab:blue", lw=1,
               label="step divided by $x$")
right.axhline(eps, color="black", ls="--", lw=1, label="$\\varepsilon = 2^{-52}$")
right.axhline(eps / 2, color="black", ls=":", lw=1, label="$\\varepsilon/2$")
right.set_ylim(0.0, 1.35 * eps)
right.set_xlabel("number $x$")
right.set_ylabel("relative step")
right.set_title("the relative step is the same at every size")
```

```
right.legend(fontsize=8, loc="lower right")
fig.tight_layout()
save_figure(fig, "float_spacing",
            "How far apart floating-point numbers are. Left: the distance from a "
            ...)
```

The right panel draws the relative step, step divided by x, for the 2301 numbers of In [7] with a logarithmic horizontal axis (`semilogx`), and the two horizontal lines ε (dashed) and $\varepsilon/2$ (dotted). `set_ylim` fixes the vertical range from 0 to $1.35\,\varepsilon$. What Figure 01f.3 shows: on the left a staircase that doubles at every power of 2 and reaches the step 2 right of $x = 2^{53}$; on the right a sawtooth that always stays between $\varepsilon/2$ and ε, the bounds proved in Section 1.4.

In [9], cancellation and its cure.

```
x = sp.Symbol("x", positive=True)
limit_at_zero = sp.limit((1 - sp.cos(x)) / x ** 2, x, 0)
say(f"sympy: the limit of (1 - cos x)/x^2 at x = 0 is {limit_at_zero}")
check(limit_at_zero == sp.Rational(1, 2)
      and sp.simplify(1 - sp.cos(x) - 2 * sp.sin(x / 2) ** 2) == 0,
      "the limit is 1/2, and 1 - cos x = 2 sin(x/2)^2 exactly")
```

`sp.Symbol("x", positive=True)` makes a sympy symbol, a letter that stands for any positive number. `sp.limit(f, x, 0)` computes the limit of the expression as x tends to 0 exactly: $1/2$ (`sp.Rational(1, 2)` is sympy's exact fraction). `sp.simplify(...) == 0` asks sympy to prove that $1 - \cos x - 2\sin^2(x/2)$ is zero for every x: the identity derived in Section 1.4.

```
x_values = np.logspace(-9, 0, 361)
naive = (1 - np.cos(x_values)) / x_values ** 2
stable = 2 * np.sin(x_values / 2) ** 2 / x_values ** 2
mpmath.mp.dps = 50
exact = np.array([float((1 - mpmath.cos(mpmath.mpf(v))) / mpmath.mpf(v) ** 2)
                  for v in x_values.tolist()])  # mpf(v) is the stored value of v
error_naive = np.abs(naive - exact) / exact
error_stable = np.abs(stable - exact) / exact
```

361 values of x from 10^{-9} to 1. `naive` is the formula as written, `stable` the rewritten one, both in floating point. `exact` computes the same function with 50 digits for each value (`x_values.tolist()` turns the array into a plain list; `mpmath.mpf(v)` takes the stored floating-point value exactly, so that all three use the same x) and rounds the result to a floating-point number. The last two lines are the relative errors.

```
report("largest relative error of 2 sin(x/2)^2/x^2", f"{error_stable.max():.1e}")
report("largest relative error of (1 - cos x)/x^2", f"{error_naive.max():.1e}")
check(float(error_stable.max()) < 1e-15,
      "the rewritten formula is correct to 1e-15 at all 361 values of x")
check((1 - math.cos(1e-8)) / 1e-8 ** 2 == 0.0
      and float(error_naive[x_values <= 1e-4].max()) > 1e-3,
      "the formula as written gives 0 at x = 1e-8: cancellation destroyed every digit")
```

The two RESULT lines print the largest relative errors with two significant digits (`.1e`: one digit after the point, in the e-notation): 4.4e-16 for the rewritten formula and 1.0e+00 for the formula as written. The first check requires the rewritten formula to be right to 10^{-15} everywhere. The second confirms that the formula as written gives exactly 0 at $x = 10^{-8}$ (`1e-8` is 10^{-8}) and that its error exceeds 10^{-3} somewhere among the values $x \le 10^{-4}$ (`error_naive[x_values <= 1e-4]` keeps only the errors at these values).

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
floor = 1e-18  # an error 0 is drawn at the bottom of the axis
ax.loglog(x_values, np.maximum(error_naive, floor), ".", ms=3, color="tab:red",
          label="$(1 - \\cos x)/x^2$ as written")
ax.loglog(x_values, np.maximum(error_stable, floor), ".", ms=3, color="tab:blue",
          label="$2\\sin^2(x/2)/x^2$")
ax.loglog(x_values, 2 * eps / x_values ** 2, "k--", lw=1,
          label="$2\\varepsilon/x^2$")
ax.set_ylim(0.5 * floor, 10.0)
ax.set_xlabel("$x$")
ax.set_ylabel("relative error")
ax.set_title("Cancellation and its cure")
ax.legend(fontsize=8, loc="upper right");
save_figure(fig, "cancellation",
            "The relative error of two formulas for $(1 - \\cos x)/x^2$ against $x$ "
            ...)
```

A logarithmic axis cannot show the value 0, so `np.maximum(error, floor)` replaces every error below 10^{-18} by 10^{-18}, which puts exact results at the bottom of the axis. The two error curves and the estimate $2\varepsilon/x^2$ of Section 1.4 are drawn on logarithmic axes. What Figure 01f.4 shows: the red dots follow the dashed line upwards as x shrinks and reach an error of 100 per cent below about $x = 10^{-8}$; the blue dots stay at the rounding level of about 10^{-16}.

In [10], the laws of powers.

```
a, b = sp.symbols("a b", positive=True)
p_sym, q_sym = sp.symbols("p q", real=True)
laws = [a ** p_sym * a ** q_sym - a ** (p_sym + q_sym),
        (a ** p_sym) ** q_sym - a ** (p_sym * q_sym),
        (a * b) ** p_sym - a ** p_sym * b ** p_sym,
        a ** (-p_sym) - 1 / a ** p_sym,
        sp.sqrt(a) - a ** sp.Rational(1, 2)]
check(all(sp.simplify(law) == 0 for law in laws),
      "the five laws of powers hold for positive a, b and real p, q")
```

`sp.symbols("a b", positive=True)` makes two symbols for positive numbers, the next line two for real numbers. Each entry of the list `laws` is the difference of the two sides of one law of Section 1.5; the check asks sympy to simplify each difference to 0. Because sympy knows that a and b are positive, it may use the laws; it would refuse for symbols of unknown sign.

```
y = sp.Symbol("y", real=True)  # a real number of either sign
say(f"sympy: sqrt(y^2) = {sp.sqrt(y ** 2)}; numbers: sqrt((-2)^2) = "
    f"{((-2.0) ** 2) ** 0.5}")
```

```
check(sp.sqrt(y ** 2) == sp.Abs(y) and ((-2.0) ** 2) ** 0.5 == 2.0,
      "sqrt(y^2) = |y|: for negative numbers the law (a^p)^q = a^(pq) fails")
```

For a real symbol of either sign sympy writes $\sqrt{y^2}$ as `Abs(y)`, the absolute value; the number example computes $((-2)^2)^{1/2} = 2$, not $(-2)^{2 \cdot 1/2} = -2$. Output: sympy: sqrt(y^2) = Abs(y); numbers: sqrt((-2)^2) = 2.0, and the PASS line.

```
s_pos, a_real = sp.Symbol("s", positive=True), sp.Symbol("a", real=True)
example = 1 / sp.sqrt(s_pos ** sp.Rational(1, 3) / sp.exp(2 * a_real))
check(sp.simplify(example - sp.exp(a_real) / s_pos ** sp.Rational(1, 6)) == 0,
      "1/sqrt(s^(1/3)/e^(2a)) = e^a/s^(1/6)")
```

The worked example of Section 1.5, with $s > 0$ and real a; `sp.exp` is the exponential function. sympy confirms the result $e^a / s^{1/6}$.

In [11], the length factors of the author's metric, from the record.

```
S, C = sp.symbols("S C", positive=True)  # S = sin z, C = cos z, both > 0
a4 = sp.Symbol("a4", real=True)  # the value of a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of b4(x4) at one time
```

Two positive symbols stand for $\sin z$ and $\cos z$ (both are positive for $0 < z < \pi/2$), a real symbol for the value of a_4 at one time, and a positive symbol for the value of b_4 at one time.

```
def from_record(text):
    """The record's Mathematica text as sympy, with sin z = S and cot z = C/S."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "S")
    text = text.replace("Cot[6*H*x0]", "(C/S)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "S": S, "C": C, "E": sp.E})
```

The Revision record stores the metric as text in the notation of the program Mathematica, for example `E^(2*a4[x4])*Sin[6*H*x0]^(1/3)`. `from_record` translates such a text into a sympy expression: `.replace(old, new)` replaces every occurrence of a piece of text, here the function values `a4[x4]` and `b4[x4]` by the symbol names `a4` and `b4`, $\sin(6Hx_0)$ by `S`, $\cot(6Hx_0)$ by `(C/S)`, and Mathematica's power sign `^` by Python's `**`. `sp.sympify` then reads the text as a sympy expression; `locals` tells it which symbol each name means (`E` is the number e).

```
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))
eta = json.loads(repository_file("Revision/algebra/gammas.json")
                 .read_text(encoding="utf-8"))["eta"]  # +1 or -1 for x0 ... x7
metric = [from_record(text) for text in curvature["metricDiagonal"]]
sizes = [eta[mu] * metric[mu] for mu in range(8)]  # |g_mu mu| = eta_mu mu g_mu mu
check(all(size.is_positive for size in sizes),
      "every entry of the metric has the sign of eta: eta_mu mu g_mu mu > 0")
```

`read_text` reads a file of the repository as text, and `json.loads` turns the JSON text into a Python dictionary. From the record file `curvature.json` the cell takes the list `metricDiagonal` (the eight diagonal entries as texts, in the order $x_0, \ldots, x_7$); from `gammas.json` it takes the list `eta` of the signs $+1$ or -1 of the eight directions (the **frame metric**, Section 1.27). `metric` is the list of the eight entries as sympy expressions. Multiplying each entry by its sign gives its size $|g_{\mu\mu}|$; the check asks sympy whether each size is positive (`.is_positive`), which it can decide because S, C, b_4 and $e^{\pm 2a_4}$ are positive.

```
factors = [sp.sqrt(size) for size in sizes]  # the length factors
# the author's order x0, x1, ..., x7
expected = ([C / S] + [sp.exp(a4) * S ** sp.Rational(1, 6)] * 3 + [sp.Integer(1)]
            + [sp.sqrt(b4) * sp.exp(-a4) * S ** sp.Rational(1, 6)] * 3)
for mu in range(8):
    say(f"x{mu}: g = {metric[mu]}, length factor {factors[mu]}")
check(len(factors) == 8 and
      all(sp.simplify(f - e) == 0 for f, e in zip(factors, expected)),
      "the length factors: cot z = C/S for x0, e^a4 S^(1/6) for x1 to x3, 1 for x4, "
      "b4^(1/2) e^(-a4) S^(1/6) for x5 to x7")
```

`factors` are the eight square roots. `expected` is the list of Section 1.5 in the author's order: $\cot z = C/S$ for x_0, three times the space factor, 1 for the time, three times the extra-time factor (a list multiplied by 3 repeats its entry three times, and `+` joins lists). The loop prints, for each direction x_μ (Python's index `mu` is the author's index, both counting from 0), the entry and its factor; sympy writes e^{a_4} as `exp(a4)`, $S^{1/6}$ as `S**(1/6)` and $b_4^{1/2}$ as `sqrt(b4)`. The check confirms that there are eight factors and that each equals the expected one.

```
product = sp.simplify(sp.Mul(*factors))
record_root = sp.simplify(from_record(curvature["sqrtAbsDetG"]))
say(f"product of the eight factors = {product}; the record's sqrtAbsDetG = "
    f"{record_root}")
lovelock_report = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json")
    .read_text(encoding="utf-8"))
root_verdict = {c["name"]: c["verdict"]
                for c in lovelock_report["checks"]}["sqrt_abs_det_g"]
check(sp.simplify(product - b4 ** sp.Rational(3, 2) * C) == 0
      and sp.simplify(record_root - b4 ** sp.Rational(3, 2) * C) == 0
      and root_verdict == "PASS",
      "the product of the length factors is b4^(3/2) cos z, the record's "
      "sqrt|det g| = b4^(3/2) sin z cot z, for every a4",
      record="Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG; "
             "python-lovelock-report.json, check sqrt_abs_det_g")
```

`sp.Mul(*factors)` multiplies the eight factors (the star hands the list's entries to `Mul` one by one), and `simplify` reduces the product to `C*b4**(3/2)`, that is $b_4^{3/2}\cos z$, as derived in Section 1.5. The record's text `Sin[6*H*x0]*Cot[6*H*x0]*b4[x4]^(3/2)` becomes $S \cdot (C/S) \cdot b_4^{3/2} = b_4^{3/2} C$ as well. The report `python-lovelock-report.json` holds a list of checks, each with a name and a verdict; the **dictionary comprehension** `{c["name"]: c["verdict"] for c in ...}` makes a dictionary from names to verdicts, and `["sqrt_abs_det_g"]` looks up the verdict of the record's

check, PASS. The check requires that both differences from $b_4^{3/2}C$ simplify to 0 and that the verdict is PASS; it reproduces the record and prints the two record names.

In [12], the limit that defines e, and the logarithm.

```
mpmath.mp.dps = 50
n_values = [10 ** k for k in range(17)]
error_exact = [abs((1 + mpmath.mpf(1) / n) ** n - mpmath.e) for n in n_values]
error_float = [abs((1.0 + 1.0 / n) ** n - math.e) for n in n_values]
for k in (0, 3, 6, 9, 12, 16):
    say(f"n = 10^{k:2d}: 50 digits: error {mpmath.nstr(error_exact[k], 3):>8}; "
        f"floating point: error {error_float[k]:.1e}")
```

`n_values` are $1, 10, 100, \ldots, 10^{16}$. For each the error of $(1 + 1/n)^n$ is computed twice: with 50 digits (`mpmath.e` is e to 50 digits) and with floating-point numbers (`math.e` is e as a float). Six of them are printed; `{k:2d}` prints k in two places and `{...:>8}` puts the text at the right end of 8 places, so that the columns line up. Output: with 50 digits the errors 0.718, 0.00136, 1.36e-6, 1.36e-9, 1.36e-12, 1.36e-16; in floating point 7.2e-01, 1.4e-03, 1.4e-06, 2.2e-07, 2.4e-04, 1.7e+00.

```
ratios = [error_exact[k] * n_values[k] / (mpmath.e / 2) for k in range(3, 17)]
check(all(abs(r - 1) < mpmath.mpf("0.01") for r in ratios),
      "50 digits: the error of (1 + 1/n)^n is e/(2n) within 1 per cent for n >= 1000")
check(error_float[16] == math.e - 1 and error_float[16] > 1e3 * error_float[6],
      "floating point: for n = 10^16, 1 + 1/n is rounded to 1, and the error grows "
      "to e - 1")
```

Each ratio divides the error by the estimate $e/(2n)$ of Section 1.5; the first check requires every ratio for $n \geq 1000$ to be within 1 per cent of 1. The second confirms that for $n = 10^{16}$ the floating-point error is exactly $e - 1$ (the result was $1^n = 1$) and that it is more than 1000 times the error at $n = 10^6$.

```
t_sym = sp.Symbol("t", real=True)
u, w = sp.symbols("u w", positive=True)
check(sp.log(sp.exp(t_sym)) == t_sym
      and sp.expand_log(sp.log(u * w)) == sp.log(u) + sp.log(w)
      and sp.simplify(sp.exp(-(a4 + sp.log(2))) - sp.exp(-a4) / 2) == 0,
      "ln(e^t) = t, ln(u w) = ln u + ln w, and e^(-(a4 + ln 2)) = e^(-a4)/2")
report("growth of a4 that shrinks the extra times by a factor 1000",
       f"{math.log(1000):.4f}")
```

`sp.log` is the natural logarithm. The check confirms the three laws of Section 1.5; `sp.expand_log` splits the logarithm of a product of positive symbols into a sum. The RESULT line prints $\ln 1000 = 6.9078$ (`.4f`: four digits after the point).

In [13], the figures of e and of growth and deflation.

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.loglog(n_values, [float(e) for e in error_exact], "o-", color="tab:blue",
          label="50-digit arithmetic")
```

```
ax.loglog(n_values, error_float, "s-", color="tab:red", ms=5,
          label="floating point")
ax.loglog(n_values, [math.e / (2 * n) for n in n_values], "k--", lw=1,
          label="$e/(2n)$")
ax.set_xlabel("$n$")
ax.set_ylabel("error $|(1 + 1/n)^n - e|$")
ax.set_title("The limit that defines $e$")
ax.legend(fontsize=8, loc="lower left");
save_figure(fig, "limit_for_e",
            "The error of $(1 + 1/n)^n$ as an approximation of $e$ for $n = 1$ to "
            ...)
```

The two error lists of In [12] are drawn against n on logarithmic axes, the 50-digit errors as blue circles joined by lines, the floating-point errors as red squares (`"s-"`), together with the estimate $e/(2n)$ as a dashed line. What Figure 01f.5 shows: the blue points lie on the dashed line all the way to 10^{16}; the red points follow it only up to about $n = 10^8$, then rise again, because the rounding error multiplied by n takes over.

```
a4_values = np.linspace(0.0, 4.0, 401)
marks = np.log(2.0) * np.arange(6)  # a4 = 0, ln 2, 2 ln 2, ..., 5 ln 2
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
for ax in (left, right):
    ax.plot(a4_values, np.exp(a4_values), color="tab:red",
            label="$e^{a_4}$: space $x_1, x_2, x_3$")
    ax.plot(a4_values, np.exp(-a4_values), color="tab:blue",
            label="$e^{-a_4}$: extra times $x_5, x_6, x_7$")
    ax.plot(marks, np.exp(marks), "o", color="tab:red", ms=4)
    ax.plot(marks, np.exp(-marks), "o", color="tab:blue", ms=4)
    ax.set_xlabel("$a_4$")
```

`np.linspace(0.0, 4.0, 401)` makes 401 equally spaced values of a_4 from 0 to 4. `marks` are the six values $0, \ln 2, \ldots, 5 \ln 2$ (`np.log` is the natural logarithm). The loop draws the same four things on both panels: the curves e^{a_4} (red, the factor of space) and e^{-a_4} (blue, the factor of the extra times), and dots at the marks, where the factors have doubled or halved once more.

```
left.set_ylabel("factor (ordinary axis)")
left.set_title("ordinary vertical axis")
left.legend(fontsize=8, loc="upper left")
right.set_yscale("log")
right.set_ylabel("factor (logarithmic axis)")
right.set_title("logarithmic vertical axis: straight lines")
for k in range(1, 6):
    right.text(marks[k], np.exp(-marks[k]) * 0.55, f"$2^{{-{k}}}$", ha="center",
               fontsize=8, color="tab:blue")
fig.tight_layout()
save_figure(fig, "growth_and_deflation",
            "The factors $e^{a_4}$ (red), which multiply the lengths along the three "
            ...)
```

The left panel keeps an ordinary vertical axis; the right panel gets a logarithmic one (`set_yscale("log")`). `right.text(x, y, text)` writes a label below each blue dot: $2^{-1}, \ldots, 2^{-5}$ (in an

f-string, double braces `{{` and `}}` stand for single braces in the text, so that LaTeX receives `2^{-1}`; `ha="center"` centres the text on the point). What Figure 01f.6 shows: on the ordinary axis one curve shoots up and the other creeps towards 0; on the logarithmic axis both are straight lines with opposite slopes, and the blue dots halve at every step $\ln 2$: the extra times deflate exponentially as a_4 grows.

In [14], the last check.

```
figure_names = ["01f_1_decimal_periods.png", "01f_2_square_root_of_two.png",
                "01f_3_float_spacing.png", "01f_4_cancellation.png",
                "01f_5_limit_for_e.png", "01f_6_growth_and_deflation.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

The list holds the names of the six figure files; the check confirms that each exists in the folder `Revision/textbook/figures` (below the output folder). `all_checks_passed()` prints the last line, ALL 26 CHECKS PASSED (notebook 01f). The 26 checks are: 3 in In [2], 1 in In [3], 1 in In [4], 3 in In [5], 1 in In [6], 4 in In [7], 3 in In [9], 3 in In [10], 3 in In [11], 3 in In [12] and 1 in In [14].

1.10 Complex numbers: the rules of arithmetic

Why a new kind of number. The equation $x^2 = -1$ has no solution among the real numbers, because the square of a real number is never negative. Yet the course needs such a solution everywhere. The fields of the author's theory have sixteen components at every point, and these components are **complex numbers** (for the field dirac16complex they are complex **anticommuting** quantities: two of them, a and b, obey $ab = -ba$ instead of $ab = ba$; they are introduced in Chapter 7; for the field dirac16complex00 they are ordinary complex numbers). A wave that oscillates in the time x_4 is written with a factor $e^{-i\varepsilon x_4}$, and whether such a wave stays bounded or grows without limit is decided by a complex number. This section builds the complex numbers from school algebra; Sections 1.11 to 1.13 add the exponential function, rotations and boosts.

Definition. We introduce a new number i, the **imaginary unit**, with the single property

$$i^2 = -1,$$

and we compute with it by the ordinary rules of algebra (sums and products may be re-ordered and multiplied out as usual). A **complex number** is a number $z = a + ib$ with two real numbers a and b. The number $a = \operatorname{Re} z$ is its **real part** and $b = \operatorname{Im} z$ its **imaginary part**. Two complex numbers are equal when both their real parts and their imaginary parts are equal. A real number a is the complex number $a + i0$; a number ib with real part 0 is called **purely imaginary**. Python writes the imaginary unit as `1j` (the letter `j` right after a number), and sympy as `I`.

Sum and product (PROVED). Sums are taken part by part: $(a+ib)+(c+id) = (a+c)+i(b+d)$ (collect the real terms and the terms with i). The product follows from multiplying out:

$$(a + ib)(c + id) = ac + iad + ibc + i^2bd$$

(every term of the first bracket times every term of the second: the distributive law),

$$= ac + i(ad + bc) - bd$$

(replace i^2 by -1, and take out the factor i from the two middle terms),

$$= (ac - bd) + i(ad + bc)$$

(group the two real terms). For example

$$(1 + 2i)(3 - i) = 3 - i + 6i - 2i^2 = 3 + 5i + 2 = 5 + 5i$$

(multiply out; collect $-i + 6i = 5i$ and replace $-2i^2$ by $+2$; add $3 + 2$). The formula for the product does not change when (a, b) and (c, d) are exchanged ($ac-bd$ and $ad+bc$ stay the same), so $zw = wz$: complex numbers **commute**. (Matrices, in Section 1.18, will not.)

The conjugate and the modulus (PROVED). The **complex conjugate** of $z = a + ib$ is $z^* = a - ib$: the sign of the imaginary part is changed and nothing else. Then

$$zz^* = (a + ib)(a - ib) = a^2 - iab + iab - i^2b^2 = a^2 + b^2$$

(multiply out; the two middle terms cancel, and $-i^2b^2 = +b^2$). This is a real number and never negative. Its positive square root $|z| = \sqrt{a^2 + b^2}$ is the **modulus** (or size) of z. A number $z \neq 0$ can be divided by: multiplying numerator and denominator of a quotient by the conjugate of the denominator makes the denominator real,

$$\frac{1 + 2i}{3 - i} = \frac{(1 + 2i)(3 + i)}{(3 - i)(3 + i)} = \frac{3 + i + 6i + 2i^2}{9 + 1} = \frac{1 + 7i}{10}$$

(multiply numerator and denominator by $3 + i$, which does not change the quotient; multiply out both, the denominator with $zz^* = a^2 + b^2$; then $3 + 2i^2 = 1$).

Three rules (PROVED). For all complex $z = a + ib$ and $w = c + id$:

1. $(zw)^* = z^*w^*$. Proof: by the product rule $zw = (ac - bd) + i(ad + bc)$, so $(zw)^* = (ac - bd) - i(ad + bc)$; and $z^*w^* = (a - ib)(c - id) = ac - iad - ibc + i^2bd = (ac - bd) - i(ad + bc)$ (multiply out; $i^2 = -1$; group). The two results are equal.
2. $zz^* = a^2 + b^2 = |z|^2$ (shown above).
3. $|zw| = |z|\,|w|$. Proof: $|zw|^2 = (zw)(zw)^*$ (rule 2 for the number zw) $= zw\,z^*w^*$ (rule 1) $= (zz^*)(ww^*)$ (complex numbers commute) $= |z|^2|w|^2$ (rule 2 twice); then take the positive square root of both sides.

Notebook 01b, In [2] and In [3], checks the examples with exact sympy arithmetic and the three rules with sympy symbols that stand for any real numbers a, b, c, d.

The complex plane. The complex number $z = a + ib$ is drawn as the point (a, b) of a plane, or as the arrow from the origin to that point: the horizontal axis carries the real part, the vertical axis the imaginary part. The modulus $|z|$ is the length of the arrow (Pythagoras). Adding two complex numbers adds the arrows: put the arrow of w at the tip of the arrow of z. The conjugate z^* is the mirror image of z in the real axis. Multiplying by i turns an arrow by a right angle:

$$i(a + ib) = ia + i^2 b = -b + ia$$

(multiply out; $i^2 = -1$), so the point (a, b) goes to the point $(-b, a)$, which has the same length $\sqrt{b^2 + a^2}$ and lies a quarter turn further counterclockwise. For $z = 1 + 2i$ this is $iz = -2 + i$. Figure 01b.1 draws these arrows.

1.11 The exponential series and Euler's formula

The exponential series. From calculus we take the series of the exponential function (Chapter 2 derives it from Taylor's theorem; here it is ASSUMED):

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots .$$

A **series** is an endless sum $t_0 + t_1 + t_2 + \cdots$; its **partial sum** $S_N = t_0 + \cdots + t_N$ adds the terms up to number N, and the series **converges** to a number S when the error $|S_N - S|$ becomes as small as we like for large N. In the same way (also ASSUMED from calculus) the sine and the cosine are

$$\cos\theta = 1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \cdots , \qquad \sin\theta = \theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \cdots ,$$

with θ in **radians** (the angle measured as the length of the arc on the circle of radius 1; π radians are 180 degrees).

Euler's formula (PROVED from the assumed series). We define $e^{i\theta}$ for a real θ by putting $x = i\theta$ into the exponential series. The powers of i repeat with period 4:

$$i^0 = 1, \quad i^1 = i, \quad i^2 = -1, \quad i^3 = i^2\, i = -i, \quad i^4 = (i^2)^2 = 1,$$

(each power is the previous one times i, with $i^2 = -1$), and then $1, i, -1, -i$ again. So in $(i\theta)^k/k! = i^k\theta^k/k!$ the even terms $k = 2m$ carry $i^{2m} = (i^2)^m = (-1)^m$ and the odd terms $k = 2m + 1$ carry $i^{2m+1} = i\,(-1)^m$:

$$e^{i\theta} = \Big(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \cdots\Big) + i\Big(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \cdots\Big)$$

(the terms of the series sorted into the even ones and the odd ones, the factor i of the odd ones taken out),

$$e^{i\theta} = \cos\theta + i\sin\theta$$

(the two brackets are the series of the cosine and the sine). This is **Euler's formula**. Notebook 01b, In [5], adds the terms one by one for $\theta = 2$: after 21 terms ($N = 20$) the partial sum differs from $\cos 2 + i \sin 2$ by 4.1×10^{-14} (COMPUTED); In [7] repeats the formula with 50-digit arithmetic.

How fast the series converges (PROVED). For $\theta \geq 0$:

$$|S_N - e^{i\theta}| = \Big| \sum_{k \geq N+1} \frac{(i\theta)^k}{k!} \Big| \leq \sum_{k \geq N+1} \frac{\theta^k}{k!}$$

(the full series minus the partial sum is the sum of the terms left out; the size of a sum is at most the sum of the sizes of its terms, a fact about complex numbers called the triangle inequality, ASSUMED here; and $|i^k| = 1$). Write $k = N + 1 + m$ with $m = 0, 1, 2, \ldots$. Since

$$(N + 1 + m)! = (N + 1)! \cdot (N + 2)(N + 3) \cdots (N + 1 + m) \geq (N + 1)!\, m!$$

(the factorial split after the factor $N + 1$; each of the m remaining factors $N + 1 + j$ is at least j, so their product is at least $1 \cdot 2 \cdots m = m!$), every term obeys

$$\frac{\theta^{N+1+m}}{(N + 1 + m)!} \leq \frac{\theta^{N+1}}{(N + 1)!} \cdot \frac{\theta^m}{m!}$$

(a larger denominator gives a smaller fraction; $\theta^{N+1+m} = \theta^{N+1}\theta^m$). Adding over m and using $\sum_m \theta^m/m! = e^\theta$:

$$|S_N - e^{i\theta}| \leq e^\theta \, \frac{\theta^{N+1}}{(N + 1)!}.$$

For $\theta = 2$ and $N = 20$ the bound is $e^2\, 2^{21}/21! \approx 3.0 \times 10^{-13}$, and the error found, 4.1×10^{-14}, lies below it. For large N the error is about the size of the first term left out, $\theta^{N+1}/(N + 1)!$, because each later term is smaller than the one before by the factor $\theta/(k + 1)$, which is small once k is much larger than θ. Going from N to $N + 1$ the first term left out changes from $\theta^{N+1}/(N + 1)!$ to $\theta^{N+2}/(N + 2)!$, a factor

$$\frac{\theta^{N+2}/(N + 2)!}{\theta^{N+1}/(N + 1)!} = \frac{\theta}{N + 2}$$

(divide the powers and the factorials; $(N + 2)! = (N + 2)\,(N + 1)!$). Notebook 01b, In [6], finds this factor within 10 per cent for $N = 10$ to 19 (COMPUTED; Figure 01b.2).

The law of exponents (PROVED). For real angles α and β:

$$e^{i\alpha} e^{i\beta} = (\cos\alpha + i \sin\alpha)(\cos\beta + i \sin\beta)$$

(Euler's formula twice),

$$= \cos\alpha \cos\beta + i \cos\alpha \sin\beta + i \sin\alpha \cos\beta + i^2 \sin\alpha \sin\beta$$

(multiply out),

$$= (\cos\alpha\cos\beta - \sin\alpha\sin\beta) + i(\sin\alpha\cos\beta + \cos\alpha\sin\beta)$$

($i^2 = -1$; group the real and the imaginary terms),

$$= \cos(\alpha+\beta) + i\sin(\alpha+\beta) = e^{i(\alpha+\beta)}$$

(the addition theorems $\cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$ and $\sin(\alpha+\beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$ of school trigonometry, ASSUMED; then Euler's formula backwards). So the rule $e^p e^q = e^{p+q}$ of real exponents (Section 1.5) holds for imaginary exponents too, which is why the notation $e^{i\theta}$ is justified. For a general complex exponent we define $e^{x+iy} = e^x e^{iy}$ with real x and y; then $e^z e^w = e^{z+w}$ for all complex z and w, by the two laws of exponents. Notebook 01b, In [7], proves the law with sympy for symbols α and β.

Three consequences (PROVED). First, $|e^{i\theta}|^2 = \cos^2\theta + \sin^2\theta = 1$ (the modulus of $a+ib$ with $a = \cos\theta$, $b = \sin\theta$; then $\cos^2 + \sin^2 = 1$): the number $e^{i\theta}$ lies on the circle of radius 1, the **unit circle**, at the angle θ. Second,

$$(e^{i\theta})^* = \cos\theta - i\sin\theta = \cos(-\theta) + i\sin(-\theta) = e^{-i\theta}$$

(conjugate; the cosine does not change and the sine changes sign when the angle changes sign; Euler's formula for the angle $-\theta$). Third, $e^{i\pi} = \cos\pi + i\sin\pi = -1 + 0i = -1$. Notebook 01b, In [7], computes $e^{i\pi}$ with 50 significant digits: the real part is -1 and an imaginary part of size 1.0×10^{-51} is left over, because the number π itself is stored with 50 digits, not because the formula fails (COMPUTED).

The polar form. Every complex number $z \neq 0$ can be written as $z = re^{i\varphi}$ with $r = |z|$ and a real angle φ, its **argument** (or **phase angle**): the point $(a/r, b/r)$ lies on the unit circle, so it is $(\cos\varphi, \sin\varphi)$ for some angle φ (school trigonometry, ASSUMED), and then $z = r(\cos\varphi + i\sin\varphi) = re^{i\varphi}$. Python's `cmath.phase(z)` returns the argument.

1.12 Multiplication turns the plane; the real matrix of a complex number

Multiplication is a turn and a stretch (PROVED). Let $u = re^{i\alpha}$ and let $p = \rho e^{i\varphi}$ be any point of the plane. Then

$$u\,p = r\rho\, e^{i\alpha} e^{i\varphi} = r\rho\, e^{i(\alpha+\varphi)}$$

(re-order the factors, which commute; the law of exponents). So multiplying every point of the plane by u multiplies every length by r and adds α to every angle: it turns the plane about the origin by the angle α and stretches it by r. For $r = 1$ it is a pure **rotation**, which keeps every length. A rotation never mirrors a figure: Figure 01b.3 shows the letter F turned by 60 and by 150 degrees and turned back a quarter while shrunk to 0.6; it is never seen in its mirror image.

Matrices of size 2 by 2. To write the same turn without complex numbers we need a tool that the next part of the chapter develops fully (Sections 1.18 to 1.20), here only for tables of four

numbers. A **2 × 2 matrix** is a table of four numbers in two rows and two columns. It acts on a pair of numbers (x, y), written as a column, by "row times column":

$$\begin{pmatrix} p & q \\ r & s \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} px + qy \\ rx + sy \end{pmatrix}$$

(the first row (p, q) is multiplied entry by entry with the column (x, y) and added, and so is the second row). The **product** MN of two such matrices is the matrix that acts like "first N, then M"; its entry in row i and column k is row i of M times column k of N, entry by entry, added. The **identity matrix** I has the rows $(1, 0)$ and $(0, 1)$ and leaves every pair unchanged. The **determinant** of the matrix with rows (a, b) and (c, d) is the number $ad - bc$. The **transpose** M^T exchanges rows and columns: the rows of M^T are the columns of M.

The matrix of a complex number (PROVED). Multiplying $x + iy$ by $a + ib$ gives, by the product rule of Section 1.10 with $c = x$ and $d = y$,

$$(a + ib)(x + iy) = (ax - by) + i(bx + ay),$$

and written for the pair (x, y) this is

$$\begin{pmatrix} a & -b \\ b & a \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} ax - by \\ bx + ay \end{pmatrix}$$

(row times column: $a \cdot x + (-b) \cdot y$ and $b \cdot x + a \cdot y$). So every complex number $a + ib$ has a real matrix $M(a + ib)$ with the rows $(a, -b)$ and (b, a). Multiplying the matrices is the same as multiplying the numbers:

$$M(a + ib)\, M(c + id) = \begin{pmatrix} a & -b \\ b & a \end{pmatrix} \begin{pmatrix} c & -d \\ d & c \end{pmatrix} = \begin{pmatrix} ac - bd & -ad - bc \\ bc + ad & -bd + ac \end{pmatrix}$$

(row times column for each of the four entries),

$$= \begin{pmatrix} ac - bd & -(ad + bc) \\ ad + bc & ac - bd \end{pmatrix} = M\big((ac - bd) + i(ad + bc)\big) = M\big((a + ib)(c + id)\big)$$

(re-order and take out the sign; this is the pattern of M with real part $ac - bd$ and imaginary part $ad + bc$; the product rule of Section 1.10). The number 1 has the matrix $M(1) = I$, and the number i has the matrix

$$J = M(i) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad JJ = \begin{pmatrix} 0 \cdot 0 + (-1) \cdot 1 & 0 \cdot (-1) + (-1) \cdot 0 \\ 1 \cdot 0 + 0 \cdot 1 & 1 \cdot (-1) + 0 \cdot 0 \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} = -I$$

(row times column). A *real* matrix squares to minus the identity: it plays the role of the number i. This is the first appearance of an idea that runs through the whole course: the author's gamma

matrices are real, and real matrices whose squares are $-I$ do the work that the imaginary unit does elsewhere. The determinant of $M(a+ib)$ is $a\cdot a-(-b)\cdot b=a^2+b^2=|a+ib|^2$.

The rotation matrix (PROVED). The matrix of $e^{i\alpha}=\cos\alpha+i\sin\alpha$ is the **rotation matrix**

$$R(\alpha)=M(e^{i\alpha})=\begin{pmatrix}\cos\alpha & -\sin\alpha\\ \sin\alpha & \cos\alpha\end{pmatrix}.$$

It inherits three properties. (a) $R(\alpha)R(\beta)=M(e^{i\alpha})M(e^{i\beta})=M(e^{i\alpha}e^{i\beta})=M(e^{i(\alpha+\beta)})=R(\alpha+\beta)$ (the product rule of the matrices M; the law of exponents): two turns make one turn by the sum of the angles. (b) $\det R(\alpha)=\cos^2\alpha+\sin^2\alpha=1$ (the determinant of M is the squared modulus, here $|e^{i\alpha}|^2=1$). (c) $R(\alpha)^TR(\alpha)=I$:

$$\begin{pmatrix}\cos\alpha & \sin\alpha\\ -\sin\alpha & \cos\alpha\end{pmatrix}\begin{pmatrix}\cos\alpha & -\sin\alpha\\ \sin\alpha & \cos\alpha\end{pmatrix}=\begin{pmatrix}\cos^2\alpha+\sin^2\alpha & -\cos\alpha\sin\alpha+\sin\alpha\cos\alpha\\ -\sin\alpha\cos\alpha+\cos\alpha\sin\alpha & \sin^2\alpha+\cos^2\alpha\end{pmatrix}=I$$

(the transpose has the columns of R as rows; row times column; then $\cos^2+\sin^2=1$ and the mixed terms cancel). Property (c) says that R keeps the squared length x^2+y^2; directly, with $c=\cos\alpha$ and $s=\sin\alpha$,

$$(cx-sy)^2+(sx+cy)^2=c^2x^2-2csxy+s^2y^2+s^2x^2+2scxy+c^2y^2=(c^2+s^2)(x^2+y^2)=x^2+y^2$$

(the squares of the two components of $R(x,y)$, multiplied out with $(u\pm v)^2=u^2\pm2uv+v^2$; the mixed terms cancel; collect; $c^2+s^2=1$). Notebook 01b, In [9], checks all of this exactly and checks that $R(\pi/3)$ moves the corners of the letter F exactly as multiplication by $e^{i\pi/3}$ does.

The roots of unity (PROVED). For a whole number $n\geq1$ the n numbers

$$z_k=e^{2\pi ik/n},\qquad k=0,1,\ldots,n-1,$$

satisfy $z_k^n=e^{2\pi ik}=\cos(2\pi k)+i\sin(2\pi k)=1$ (the law of exponents applied n times adds the angle $2\pi k/n$ to itself n times; Euler's formula; a whole number of full turns). They are the **roots of unity** of order n, the corners of a regular polygon with n sides on the unit circle. For $n\geq2$ their sum $S=z_0+\cdots+z_{n-1}$ is 0:

$$z_1S=z_1z_0+z_1z_1+\cdots+z_1z_{n-1}=z_1+z_2+\cdots+z_{n-1}+z_0=S$$

(multiply out; $z_1z_k=e^{2\pi i(k+1)/n}=z_{k+1}$ by the law of exponents, and $z_1z_{n-1}=e^{2\pi i}=1=z_0$, so the same numbers appear in another order). Hence $(z_1-1)S=0$, and since $z_1\neq1$ for $n\geq2$, $S=0$ (divide by the nonzero number z_1-1). The eight eighth roots are 1, $\frac{\sqrt2}{2}+\frac{\sqrt2}{2}i$, i, $-\frac{\sqrt2}{2}+\frac{\sqrt2}{2}i$, -1, $-\frac{\sqrt2}{2}-\frac{\sqrt2}{2}i$, $-i$, $\frac{\sqrt2}{2}-\frac{\sqrt2}{2}i$ (from $\cos(\pi/4)=\sin(\pi/4)=\sqrt2/2$). Notebook 01b, In [10], checks this for $n=3$ and $n=8$ and draws the two polygons (Figure 01b.4).

Conjugation and real numbers (PROVED). Conjugation changes the sign of the imaginary part and nothing else. Hence: a real number, whose imaginary part is 0, is its own conjugate,

$x^* = x$; a list of real numbers does not change at all when every entry is conjugated; conjugating twice gives back the original number, $(z^*)^* = z$; and $(e^{i\alpha})^* = e^{-i\alpha}$, so conjugation turns a rotation into the opposite rotation. The first of these facts has a consequence that the course uses again and again. The author's gamma matrices are real, and fields with real components occur throughout the course. On a real field, complex conjugation does nothing at all. So a conjugation that is to change a real field cannot be plain complex conjugation: it must act through a matrix on the sixteen components. Chapter 5 derives the two charge-conjugation matrices of the theory and shows that only one of them changes a real field. Notebook 01b, In [11], checks the four statements.

1.13 Boosts, hyperbolic functions, oscillation and growth

Hyperbolic functions. The **hyperbolic cosine** and **hyperbolic sine** are

$$\cosh\varphi = \frac{e^{\varphi} + e^{-\varphi}}{2}, \qquad \sinh\varphi = \frac{e^{\varphi} - e^{-\varphi}}{2}.$$

They obey (PROVED):

$$\cosh^2\varphi = \frac{e^{2\varphi} + 2 + e^{-2\varphi}}{4}, \qquad \sinh^2\varphi = \frac{e^{2\varphi} - 2 + e^{-2\varphi}}{4}$$

(square the definitions with $(u \pm v)^2 = u^2 \pm 2uv + v^2$, using $e^{\varphi}e^{-\varphi} = e^0 = 1$ and $(e^{\varphi})^2 = e^{2\varphi}$), so

$$\cosh^2\varphi - \sinh^2\varphi = \frac{2 - (-2)}{4} = 1$$

(subtract; the terms $e^{\pm 2\varphi}$ cancel). Adding and subtracting the two definitions gives

$$\cosh\varphi + \sinh\varphi = e^{\varphi}, \qquad \cosh\varphi - \sinh\varphi = e^{-\varphi}.$$

The addition theorems of cosh and sinh (PROVED). With $c_1 = \cosh\varphi_1$ and so on,

$$\cosh\varphi_1\cosh\varphi_2 + \sinh\varphi_1\sinh\varphi_2 = \frac{(e^{\varphi_1} + e^{-\varphi_1})(e^{\varphi_2} + e^{-\varphi_2}) + (e^{\varphi_1} - e^{-\varphi_1})(e^{\varphi_2} - e^{-\varphi_2})}{4}$$

(the definitions inserted; both products have the denominator $2 \cdot 2$),

$$= \frac{2e^{\varphi_1+\varphi_2} + 2e^{-\varphi_1-\varphi_2}}{4} = \cosh(\varphi_1 + \varphi_2)$$

(multiplied out, the mixed terms $e^{\varphi_1-\varphi_2}$ and $e^{-\varphi_1+\varphi_2}$ appear once with $+$ and once with $-$ and cancel, while $e^{\varphi_1+\varphi_2}$ and $e^{-\varphi_1-\varphi_2}$ appear twice; then the definition of cosh). In the same way, $\sinh\varphi_1\cosh\varphi_2 + \cosh\varphi_1\sinh\varphi_2 = \sinh(\varphi_1 + \varphi_2)$: multiplied out, the products give $(e^{\varphi_1+\varphi_2} + e^{\varphi_1-\varphi_2} - e^{-\varphi_1+\varphi_2} - e^{-\varphi_1-\varphi_2}) + (e^{\varphi_1+\varphi_2} - e^{\varphi_1-\varphi_2} + e^{-\varphi_1+\varphi_2} - e^{-\varphi_1-\varphi_2})$ over 4, and the mixed terms cancel again.

Rotations and boosts. In the author's spacetime there are directions of two kinds: four **space-like** ones, x_0, x_1, x_2, x_3, and four **time-like** ones, x_4, x_5, x_6, x_7 (Section 1.5; Section 1.27 makes the words exact). A transformation that mixes two directions of the same kind is a rotation, which keeps $x^2 + y^2$. A transformation that mixes a time-like coordinate t with a space-like coordinate x (for example x_4 with x_0) keeps instead the difference $x^2 - t^2$; it is a **boost**:

$$\begin{pmatrix} t' \\ x' \end{pmatrix} = \Lambda(\varphi) \begin{pmatrix} t \\ x \end{pmatrix}, \qquad \Lambda(\varphi) = \begin{pmatrix} \cosh\varphi & \sinh\varphi \\ \sinh\varphi & \cosh\varphi \end{pmatrix}.$$

The number φ is the **rapidity** of the boost. Three properties (PROVED). (a) It keeps $-t^2 + x^2$: with $c = \cosh\varphi$ and $s = \sinh\varphi$, $t' = ct + sx$ and $x' = st + cx$ (row times column), and

$$-t'^2 + x'^2 = -(c^2t^2 + 2cstx + s^2x^2) + (s^2t^2 + 2sctx + c^2x^2) = -(c^2 - s^2)t^2 + (c^2 - s^2)x^2 = -t^2 + x^2$$

(square with $(u+v)^2 = u^2 + 2uv + v^2$; the mixed terms cancel; collect; $c^2 - s^2 = 1$). With the **frame metric** of this pair, $\eta = \mathrm{diag}(-1, +1)$ (the diagonal matrix with -1 for the time-like t and $+1$ for the space-like x), this statement is written $\Lambda^T \eta \Lambda = \eta$; Section 1.27 explains this way of writing it. Its determinant is $c^2 - s^2 = 1$. (b) Rapidities add: $\Lambda(\varphi_1)\Lambda(\varphi_2) = \Lambda(\varphi_1 + \varphi_2)$; row times column gives the entries $c_1c_2 + s_1s_2$ and $s_1c_2 + c_1s_2$ (each twice), which are $\cosh(\varphi_1 + \varphi_2)$ and $\sinh(\varphi_1 + \varphi_2)$ by the addition theorems. (c) The two **light-like** directions $(t, x) = (1, 1)$ and $(1, -1)$, on which $-t^2 + x^2 = 0$, are only stretched:

$$\Lambda(\varphi) \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} c+s \\ s+c \end{pmatrix} = e^{\varphi} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \Lambda(\varphi) \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} c-s \\ s-c \end{pmatrix} = e^{-\varphi} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

(row times column; then $c \pm s = e^{\pm\varphi}$). A boost stretches one light-like direction by the growing exponential e^{φ} and shrinks the other by the decaying exponential $e^{-\varphi}$; the product of the two factors is 1. The lengths along space and along the extra times in the author's metric carry the factors e^{a_4} and e^{-a_4}, a pattern of the same form; only the form is the same, because a_4 is a function of the time x_4 and not the rapidity of a boost. Where a rotation moves a point around a circle $x^2 + y^2 = 1$, a boost moves it along a **hyperbola** $t^2 - x^2 = 1$ that never crosses the light-like lines $t = \pm x$ (Figure 01b.5). Notebook 01b, In [12] and In [13], checks (a) to (c) exactly and in numbers.

Oscillation and growth (PROVED). For a real number ε, the **frequency**, the wave

$$e^{-i\varepsilon t} = \cos(\varepsilon t) - i\sin(\varepsilon t)$$

(Euler's formula with the angle $-\varepsilon t$; the cosine is even, the sine odd) has the modulus 1 at every time t: its real and imaginary parts oscillate, and it stays bounded. If the frequency is imaginary, $\varepsilon = i\gamma$ with a real γ, the same formula gives

$$e^{-i\varepsilon t} = e^{-i(i\gamma)t} = e^{-i^2\gamma t} = e^{\gamma t}$$

(insert $\varepsilon = i\gamma$; $i \cdot i = i^2$; $-i^2 = 1$): no oscillation at all, but exponential growth for $\gamma > 0$ and exponential decay for $\gamma < 0$. A complex frequency $\varepsilon = 2 + 0.3i$ gives

$$e^{-i(2+0.3i)t} = e^{-2it-0.3i^2t} = e^{0.3t}e^{-2it}$$

(multiply out the exponent; $-0.3i^2 = 0.3$; split the exponential with $e^{z+w} = e^z e^w$): an oscillation whose size $e^{0.3t}$ grows. An imaginary part of a frequency is the mathematical sign of an **instability**; Chapter 8 meets it in the waves along the extra times. Notebook 01b, In [14], checks the three cases at 501 times and draws them (Figure 01b.6).

1.14 Example: complex numbers, rotations and boosts

Notebook 01b puts Sections 1.10 to 1.13 to work. It computes with complex numbers in Python and exactly with sympy and checks the rules of the conjugate and the modulus for all complex numbers; draws complex numbers as arrows; adds up the exponential series at $i\theta$ term by term and measures how fast it reaches $\cos\theta + i\sin\theta$; confirms $e^{i\pi} = -1$ to 50 digits and proves the law of exponents; turns the letter F by multiplication and by the rotation matrix; checks $J^2 = -I$ and the rules of the rotation matrices; draws the roots of unity; checks that conjugation leaves real numbers unchanged; compares rotations, which keep circles, with boosts, which keep hyperbolas; and shows that a real frequency gives an oscillation and an imaginary one exponential growth. Every check is exact (sympy) or numerical with a stated tolerance; nothing in this notebook comes from the Revision record, and it reads no file. It ends with the line ALL 34 CHECKS PASSED (notebook 01b).

1.15 How to run Notebook 01b

Step 1. What this notebook does and what it needs.

Notebook 01b (Complex numbers, rotations and boosts) is the file `Revision/textbook/notebooks/01b_complex_rotations.ipynb` of the repository Dirac_claude. It computes with complex numbers in Python and exactly with sympy, draws them as arrows in the plane, adds up the series of the exponential function to show Euler's formula and how fast the series converges, shows that multiplying by a complex number of modulus 1 rotates the plane and that the same rotation is a real 2 by 2 matrix, draws the roots of unity, shows that conjugation leaves real numbers unchanged, compares rotations, which keep x squared plus y squared, with boosts, which keep x squared minus t squared, and shows that a real frequency gives an oscillation while an imaginary frequency gives exponential growth. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at

the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01b_complex_rotations.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01b_complex_rotations.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01b_complex_rotations.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01b.captions.json`

- `Revision/textbook/figures/01b_1_complex_plane.png`
- `Revision/textbook/figures/01b_2_euler_series.png`
- `Revision/textbook/figures/01b_3_rotation_by_multiplication.png`
- `Revision/textbook/figures/01b_4_roots_of_unity.png`
- `Revision/textbook/figures/01b_5_rotations_and_boosts.png`
- `Revision/textbook/figures/01b_6_oscillation_and_growth.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 34 CHECKS PASSED (notebook 01b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01b_complex_rotations.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01b_complex_rotations.ipynb
python -m nbconvert --execute --inplace 01b_complex_rotations.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

1.16 Notebook 01b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01b_complex_rotations.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook builds the complex numbers from zero and connects them with
rotations of the plane. It

- adds, multiplies and divides complex numbers in Python and exactly with
  sympy, and checks the rules of the modulus and of the conjugate;
- draws complex numbers as arrows in the plane;
- adds up the series $\sum_k (i\theta)^k / k!$ term by term and shows that it
  reaches $\cos\theta + i \sin\theta$ (Euler's formula), with a plot of how fast
  the error shrinks;
- shows that multiplying by $e^{i\alpha}$ rotates every point of the plane by the
  angle $\alpha$, and that the same rotation is a real $2 \times 2$ matrix; the
  real matrix $J$ that represents $i$ satisfies $J^2 = -I$;
- draws the roots of unity, the $n$ numbers with $z^n = 1$;
```

```
- shows that conjugation leaves every real number unchanged;
- compares rotations, which keep $x^2 + y^2$, with *boosts*, which keep
  $x^2 - t^2$ (one time-like and one space-like direction);
- shows that $e^{-i\varepsilon t}$ oscillates with modulus 1 when the frequency
  $\varepsilon$ is real, and grows exponentially when $\varepsilon$ is imaginary.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01b (Complex numbers, rotations and boosts) is the file
‘Revision/textbook/notebooks/01b_complex_rotations.ipynb‘ of the repository Dirac_claude.
It computes with complex numbers in Python and exactly with sympy, draws them as arrows
in the plane, adds up the series of the exponential function to show Euler’s formula and
how fast the series converges, shows that multiplying by a complex number of modulus 1
rotates the plane and that the same rotation is a real 2 by 2 matrix, draws the roots of
unity, shows that conjugation leaves real numbers unchanged, compares rotations, which
keep x squared plus y squared, with boosts, which keep x squared minus t squared, and
shows that a real frequency gives an oscillation while an imaginary frequency gives
exponential growth. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy,
sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and
runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):
```

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01b_complex_rotations.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its

left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01b_complex_rotations.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01b_complex_rotations.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01b.captions.json‘
- ‘Revision/textbook/figures/01b_1_complex_plane.png‘
- ‘Revision/textbook/figures/01b_2_euler_series.png‘
- ‘Revision/textbook/figures/01b_3_rotation_by_multiplication.png‘
- ‘Revision/textbook/figures/01b_4_roots_of_unity.png‘
- ‘Revision/textbook/figures/01b_5_rotations_and_boosts.png‘
- ‘Revision/textbook/figures/01b_6_oscillation_and_growth.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 34 CHECKS PASSED (notebook 01b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01b_complex_rotations.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 01b_complex_rotations.ipynb
      python -m nbconvert --execute --inplace 01b_complex_rotations.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
```

```
#
# Notebook 01b (Complex numbers, rotations and boosts) is the file
# Revision/textbook/notebooks/01b_complex_rotations.ipynb of the repository Dirac_claude.
# It computes with complex numbers in Python and exactly with sympy, draws them as arrows
# in the plane, adds up the series of the exponential function to show Euler's formula
# and how fast the series converges, shows that multiplying by a complex number of
# modulus 1 rotates the plane and that the same rotation is a real 2 by 2 matrix, draws
# the roots of unity, shows that conjugation leaves real numbers unchanged, compares
# rotations, which keep x squared plus y squared, with boosts, which keep x squared minus
# t squared, and shows that a real frequency gives an oscillation while an imaginary
# frequency gives exponential growth. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy, mpmath and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
```

```
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01b_complex_rotations.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01b_complex_rotations.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01b_complex_rotations.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
```

```
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01b.captions.json
# - Revision/textbook/figures/01b_1_complex_plane.png
# - Revision/textbook/figures/01b_2_euler_series.png
# - Revision/textbook/figures/01b_3_rotation_by_multiplication.png
# - Revision/textbook/figures/01b_4_roots_of_unity.png
# - Revision/textbook/figures/01b_5_rotations_and_boosts.png
# - Revision/textbook/figures/01b_6_oscillation_and_growth.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 34 CHECKS PASSED (notebook 01b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01b_complex_rotations.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
```

```
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01b_complex_rotations.ipynb
#     python -m nbconvert --execute --inplace 01b_complex_rotations.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "01b"  # this notebook: chapter 01, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Imaginary unit** $i$: a new number with $i^2 = -1$. Python writes it ‘1j‘,
  sympy writes it ‘I‘.
- **Complex number**: $z = a + ib$ with real numbers $a$ (the **real part**,
  $\mathrm{Re}\,z$) and $b$ (the **imaginary part**, $\mathrm{Im}\,z$).
- **Complex plane**: the plane in which $z = a + ib$ is the point (or the arrow
  from the origin to the point) $(a, b)$; horizontal axis $\mathrm{Re}$,
  vertical axis $\mathrm{Im}$.
- **Modulus** $|z| = \sqrt{a^2 + b^2}$: the length of the arrow.
- **Argument** (or **phase angle**): the angle between the arrow and the positive
  real axis, measured counterclockwise in radians ($\pi$ radians $= 180$
  degrees).
- **Conjugate** $z^* = a - ib$: the mirror image of $z$ in the real axis.
- **Series, partial sum**: an infinite sum $t_0 + t_1 + t_2 + \cdots$; its
  partial sum $S_N$ adds the terms $t_0$ to $t_N$. The series **converges** to a
  number $S$ when the error $|S_N - S|$ becomes as small as we like for large $N$.
- **Factorial** $k! = 1 \cdot 2 \cdots k$, with $0! = 1$.
```

```
- **Euler's formula**: $e^{i\theta} = \cos\theta + i \sin\theta$ for real
  $\theta$.
- **Polar form**: $z = r e^{i\varphi}$ with $r = |z|$ and $\varphi$ the argument.
- **Rotation**: turning the plane about the origin by an angle; it keeps every
  length. The **rotation matrix** of the angle $\alpha$ has the rows
  $(\cos\alpha, -\sin\alpha)$ and $(\sin\alpha, \cos\alpha)$.
- **Hyperbolic functions**: $\cosh\varphi = (e^{\varphi} + e^{-\varphi})/2$ and
  $\sinh\varphi = (e^{\varphi} - e^{-\varphi})/2$; they satisfy
  $\cosh^2\varphi - \sinh^2\varphi = 1$.
- **Boost**: the matrix with rows $(\cosh\varphi, \sinh\varphi)$ and
  $(\sinh\varphi, \cosh\varphi)$ acting on a pair $(t, x)$ of a time-like and a
  space-like coordinate; it keeps $x^2 - t^2$. The number $\varphi$ is its
  **rapidity**.
- **Root of unity**: a complex number $z$ with $z^n = 1$ for a whole number
  $n \geq 1$.
- **Frequency**: the number $\varepsilon$ in $e^{-i\varepsilon t}$; for real
  $\varepsilon$ the real and imaginary parts oscillate like $\cos$ and $\sin$.
- **mpmath**: a Python package that computes with as many decimal digits as
  requested (here 50).
```

Text cell:

```
## 4. The physical and mathematical situation

The fields of the author's theory have 16 *complex* components at every point of
spacetime (for the field dirac16complex they are complex anticommuting quantities,
for which $ab = -ba$ instead of $ab = ba$; for dirac16complex00 they are ordinary
complex numbers), and a wave that oscillates in the
time $x_4$ is written with factors like $e^{-i\varepsilon x_4}$. The author's
gamma matrices, on the other hand, are *real*. This notebook shows
how the two fit together: a real $2 \times 2$ matrix $J$ with $J^2 = -I$ behaves
exactly like the number $i$, and multiplying by a complex number of modulus 1 is
the same as rotating with a real matrix.

Euler's formula $e^{i\theta} = \cos\theta + i\sin\theta$ comes from the series of
the exponential function, $e^x = \sum_{k=0}^\infty x^k/k!$, with $x = i\theta$:
the powers of $i$ repeat as $1, i, -1, -i, 1, \dots$, so the even terms build the
series of $\cos\theta$ and the odd terms $i$ times the series of $\sin\theta$.

The author's spacetime has four space-like directions ($x_0, x_1, x_2, x_3$,
where the frame metric $\eta$ has $+1$) and four time-like ones ($x_4$ to $x_7$,
where $\eta$ has $-1$). A transformation that mixes two directions of the same
kind is a rotation, which keeps $x^2 + y^2$; one that mixes a time-like with a
space-like direction is a boost, which keeps $x^2 - t^2$ and is built from
$\cosh$ and $\sinh$ instead of $\cos$ and $\sin$. A boost stretches one light-like
direction ($x = t$) by $e^{\varphi}$ and shrinks the other ($x = -t$) by
$e^{-\varphi}$: a growing and a shrinking exponential whose product is 1. The
lengths of space and of the extra times in the author's metric carry the factors
$e^{a_4}$ and $e^{-a_4}$, a pattern of the same form (only the form is the
same: $a_4$ is a function of the time, not a rapidity).

Finally, the frequency $\varepsilon$ of a wave decides its fate: if $\varepsilon$
is real, $|e^{-i\varepsilon t}| = 1$ for all times; if $\varepsilon = i\gamma$ is
```

```
imaginary, $e^{-i\varepsilon t} = e^{\gamma t}$ grows (or shrinks) exponentially.
An imaginary frequency is the mathematical sign of an instability.
```

Text cell:

```
## 5. Complex numbers in Python and in sympy

The next cell computes with $z = 1 + 2i$ and $w = 3 - i$ in two ways. Python's
own complex numbers (written `1 + 2j`) are floating-point numbers. sympy's `I` is
the exact imaginary unit, so sympy gives exact results:

$(1 + 2i)(3 - i) = 3 - i + 6i - 2i^2 = 3 + 5i + 2 = 5 + 5i$, and
$\frac{1 + 2i}{3 - i} = \frac{(1 + 2i)(3 + i)}{(3 - i)(3 + i)} =
\frac{3 + i + 6i + 2i^2}{9 + 1} = \frac{1 + 7i}{10}$ (multiplying numerator and
denominator by the conjugate $3 + i$ of the denominator makes the denominator
real).
```

In [2]:

```
import cmath  # functions of complex numbers: phase (the argument), exp
import math  # cos, sin, exp, factorial of real numbers

import mpmath  # numbers with as many digits as we ask for
import numpy as np  # arrays of numbers
import sympy as sp  # exact arithmetic

z = 1 + 2j  # Python writes the imaginary unit as j, right after a number
w = 3 - 1j
say(f"Python: z = {z}, w = {w}, z + w = {z + w}, z w = {z * w}")
I = sp.I  # sympy's exact imaginary unit
z_exact = 1 + 2 * I
w_exact = 3 - I
product = sp.expand(z_exact * w_exact)  # multiply out, with I**2 = -1
quotient = sp.simplify(z_exact / w_exact)
say(f"sympy: z w = {product}, z / w = {quotient}")
check(I ** 2 == -1, "i^2 = -1")
check(product == 5 + 5 * I and complex(product) == z * w, "(1 + 2i)(3 - i) = 5 + 5i")
check(sp.simplify(quotient - (1 + 7 * I) / 10) == 0, "(1 + 2i)/(3 - i) = (1 + 7i)/10")
```

Out [2]:

```
Python: z = (1+2j), w = (3-1j), z + w = (4+1j), z w = (5+5j)
sympy: z w = 5 + 5*I, z / w = 1/10 + 7*I/10
PASS i^2 = -1
PASS (1 + 2i)(3 - i) = 5 + 5i
PASS (1 + 2i)/(3 - i) = (1 + 7i)/10
```

Text cell:

```
The next cell checks the rules of the conjugate and the modulus for *all*
complex numbers $z = a + ib$ and $w = c + id$, with sympy symbols $a, b, c, d$
that stand for any real numbers:

- $(zw)^* = z^* w^*$: conjugating a product conjugates each factor;
- $z z^* = a^2 + b^2 = |z|^2$, a real number that is never negative;
- $|zw|^2 = |z|^2 |w|^2$, hence $|zw| = |z|\,|w|$: lengths multiply.
```

In [3]:

```
a, b, c, d = sp.symbols("a b c d", real=True)  # any real numbers
zz = a + b * I
ww = c + d * I
check(sp.expand(sp.conjugate(zz * ww) - sp.conjugate(zz) * sp.conjugate(ww)) == 0,
      "(z w)* = z* w* for all complex z and w")
check(sp.expand(zz * sp.conjugate(zz)) == a ** 2 + b ** 2,
      "z z* = a^2 + b^2 = |z|^2 for every z = a + i b")
check(sp.expand(zz * ww * sp.conjugate(zz * ww)
                - (a ** 2 + b ** 2) * (c ** 2 + d ** 2)) == 0,
      "|z w|^2 = |z|^2 |w|^2 for all complex z and w")
say(f"|z| = {sp.Abs(z_exact)}, |w| = {sp.Abs(w_exact)}, "
    f"|z w| = {sp.Abs(product)}")
```

Out [3]:

```
PASS (z w)* = z* w* for all complex z and w
PASS z z* = a^2 + b^2 = |z|^2 for every z = a + i b
PASS |z w|^2 = |z|^2 |w|^2 for all complex z and w
|z| = sqrt(5), |w| = sqrt(10), |z w| = 5*sqrt(2)
```

Text cell:

```
## 6. Complex numbers as arrows

The next cell draws $z = 1 + 2i$, $w = 3 - i$, their sum $z + w = 4 + i$ (the
diagonal of the parallelogram spanned by $z$ and $w$: put the arrow $w$ at the
tip of $z$), the conjugate $z^* = 1 - 2i$ (the mirror image in the real axis) and
$iz = -2 + i$, which is $z$ turned by a right angle. The helper ‘arrow‘ draws a
complex number as an arrow from a starting point. The check confirms that $iz$
has the same length as $z$ and an argument larger by $\pi/2$ (‘cmath.phase‘ gives
the argument).
```

In [4]:

```
def arrow(ax, number, color, label, start=0j, style="-"):
    """Draw the complex number as an arrow from start to start + number."""
    tip = start + number
    ax.annotate("", xy=(tip.real, tip.imag), xytext=(start.real, start.imag),
                arrowprops={"arrowstyle": "->", "color": color, "lw": 2,
```

```
                           "linestyle": style})
    ax.text(tip.real + 0.1, tip.imag + 0.1, label, color=color)


fig, ax = plt.subplots(figsize=(7.0, 5.2))
arrow(ax, z, "tab:blue", "$z = 1 + 2i$")
arrow(ax, w, "tab:red", "$w = 3 - i$")
arrow(ax, w, "tab:red", "", start=z, style="--")  # w moved to the tip of z
arrow(ax, z + w, "black", "$z + w$")
arrow(ax, z.conjugate(), "tab:green", "$z^* = 1 - 2i$")
arrow(ax, 1j * z, "tab:purple", "$iz = -2 + i$")
ax.axhline(0, color="0.5", lw=0.8)  # the real axis
ax.axvline(0, color="0.5", lw=0.8)  # the imaginary axis
ax.set_xlim(-2.8, 5.0)
ax.set_ylim(-2.6, 3.4)
ax.set_aspect("equal")
ax.set_xlabel("real part Re")
ax.set_ylabel("imaginary part Im")
ax.set_title("Complex numbers as arrows")
save_figure(fig, "complex_plane",
            "Complex numbers as arrows in the complex plane; horizontal axis the "
            "real part, vertical axis the imaginary part (pure numbers). Blue "
            "$z = 1 + 2i$, red $w = 3 - i$ (dashed: the same arrow moved to the tip "
            "of $z$), black their sum $z + w = 4 + i$, green the conjugate "
            "$z^{\\ast} = 1 - 2i$ (the mirror image of $z$ in the real axis) and "
            "purple $iz = -2 + i$, which is $z$ turned counterclockwise by a right "
            "angle.")
turn = cmath.phase(1j * z) - cmath.phase(z)  # difference of the two arguments
check(abs(abs(1j * z) - abs(z)) < 1e-15 and abs(turn - math.pi / 2) < 1e-12,
      "multiplying by i keeps the length and turns by pi/2")
```

Out [4]:

```
Figure 01b.1 saved as Revision/textbook/figures/01b_1_complex_plane.png
PASS multiplying by i keeps the length and turns by pi/2
```

Text cell:

```
## 7. Euler's formula from the exponential series

The exponential series at $x = i\theta$ is
$\sum_{k=0}^\infty (i\theta)^k / k!$. Its terms can be made one from the other:
the term number $k+1$ is the term number $k$ times $i\theta/(k+1)$. The next cell
adds the terms for $\theta = 2$ one by one and keeps the partial sums
$S_0, S_1, \dots, S_{20}$ and their errors $|S_N - (\cos 2 + i \sin 2)|$.

How big can the error be? The error is the sum of the terms left out after $S_N$,
and the size of a sum is at most the sum of the sizes of its terms. These terms
have the sizes $\theta^k/k!$ for $k \geq N + 1$, and
$\frac{\theta^{N+1+m}}{(N+1+m)!} \leq \frac{\theta^{N+1}}{(N+1)!} \cdot
\frac{\theta^m}{m!}$ (because $(N+1+m)!$ is at least $(N+1)! \, m!$); adding over
$m = 0, 1, 2, \dots$ and using $\sum_m \theta^m/m! = e^\theta$ gives the bound
```

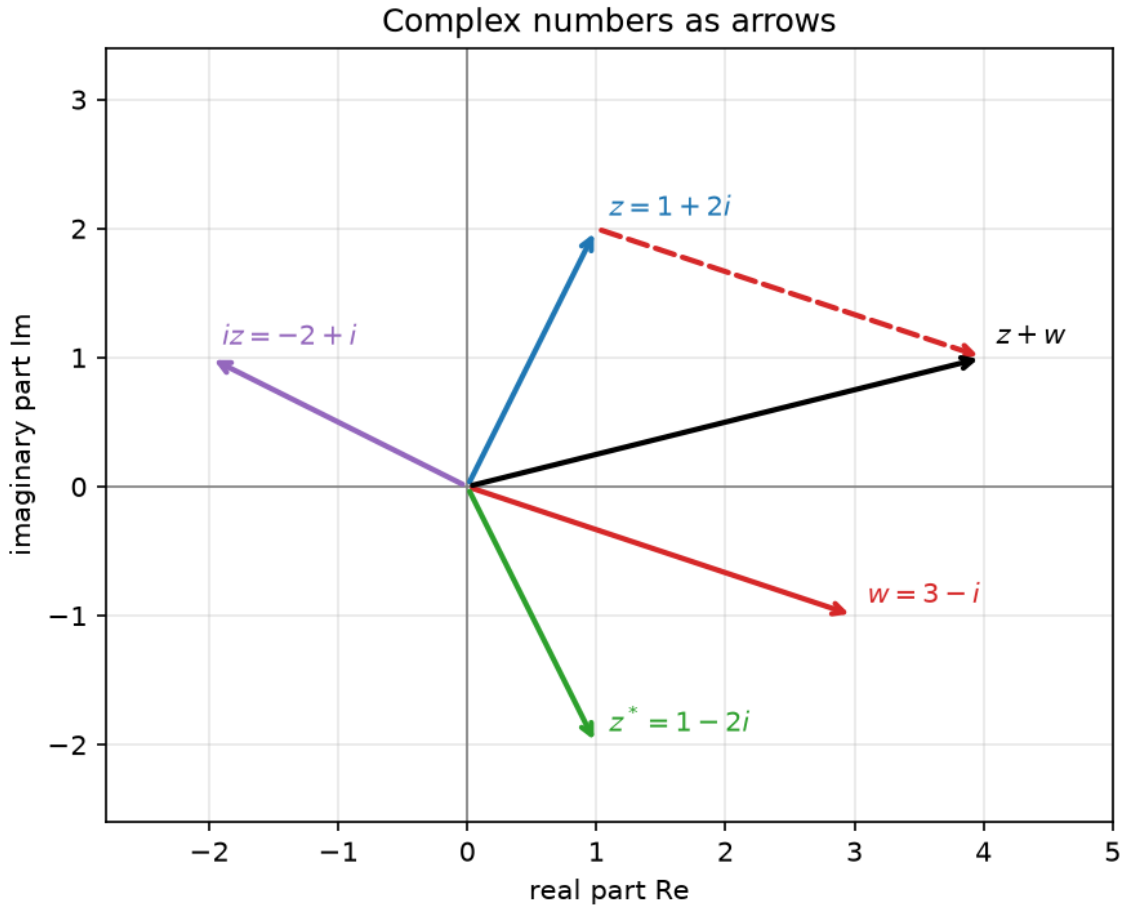

Figure 23: Complex numbers as arrows in the complex plane; horizontal axis the real part, vertical axis the imaginary part (pure numbers). Blue $z = 1 + 2i$, red $w = 3 - i$ (dashed: the same arrow moved to the tip of z), black their sum $z + w = 4 + i$, green the conjugate $z^* = 1 - 2i$ (the mirror image of z in the real axis) and purple $iz = -2 + i$, which is z turned counterclockwise by a right angle. (Notebook 01b, figure 1.)

```
$|S_N - e^{i\theta}| \leq e^\theta \, \theta^{N+1}/(N+1)!$. The cell checks the
error against this bound for every $N$, and checks that after 21 terms the error
is below $10^{-12}$.
```

In [5]:

```
theta = 2.0
target = complex(math.cos(theta), math.sin(theta))  # cos(theta) + i sin(theta)
partial_sums = []
total = 0j
term = 1 + 0j  # the term number 0: (i theta)^0 / 0! = 1
for k in range(21):
    total += term
    partial_sums.append(total)  # S_k
    term = term * 1j * theta / (k + 1)  # the next term
errors = [abs(s - target) for s in partial_sums]
bounds = [math.exp(theta) * theta ** (n + 1) / math.factorial(n + 1)
          for n in range(21)]
for n in (0, 1, 2, 5, 10, 15, 20):
    say(f"N = {n:2d}: S_N = {partial_sums[n].real:+.12f} {partial_sums[n].imag:+.12f} i,"
        f" error {errors[n]:.1e}")
say(f"cos 2 + i sin 2 = {target.real:+.12f} {target.imag:+.12f} i")
check(all(e <= b + 1e-15 for e, b in zip(errors, bounds)),
      "the error of every partial sum is below e^theta theta^(N+1)/(N+1)!")
report("error of the series for e^(2i) after 21 terms", f"{errors[20]:.1e}")
check(errors[20] < 1e-12, "21 terms of the series give cos 2 + i sin 2 to 1e-12")
```

Out [5]:

```
N =  0: S_N = +1.000000000000 +0.000000000000 i, error 1.7e+00
N =  1: S_N = +1.000000000000 +2.000000000000 i, error 1.8e+00
N =  2: S_N = -1.000000000000 +2.000000000000 i, error 1.2e+00
N =  5: S_N = -0.333333333333 +0.933333333333 i, error 8.6e-02
N = 10: S_N = -0.416155202822 +0.909347442681 i, error 5.1e-05
N = 15: S_N = -0.416146839639 +0.909297426461 i, error 3.1e-09
N = 20: S_N = -0.416146836547 +0.909297426826 i, error 4.1e-14
cos 2 + i sin 2 = -0.416146836547 +0.909297426826 i
PASS the error of every partial sum is below e^theta theta^(N+1)/(N+1)!
RESULT error of the series for e^(2i) after 21 terms = 4.1e-14
PASS 21 terms of the series give cos 2 + i sin 2 to 1e-12
```

Text cell:

```
The next cell draws the partial sums $S_0, \dots, S_{10}$ as points of the complex
plane joined in order (they spiral in towards the point $e^{2i}$ on the unit
circle), and on the right the error against $N$ on a logarithmic scale, together
with the bound of the previous cell. For large $N$ the error is about the size of
the first term left out, $\theta^{N+1}/(N+1)!$, so going from $N$ to $N + 1$
multiplies it by about $\theta/(N+2)$: the error shrinks faster and faster. The
cell checks this for $N = 10$ to 19 (agreement within 10 per cent).
```

In [6]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6))
circle = np.exp(1j * np.linspace(0.0, 2.0 * np.pi, 400))  # the unit circle
left.plot(circle.real, circle.imag, color="0.7", lw=1, label="unit circle")
first = np.array(partial_sums[:11])
left.plot(first.real, first.imag, "o-", color="tab:blue",
          label="partial sums $S_0, \\dots, S_{10}$")
for n in range(5):  # number the first five points
    left.text(first[n].real + 0.06, first[n].imag + 0.06, f"$S_{n}$")
left.plot([target.real], [target.imag], "*", color="tab:red", ms=14,
          label="$e^{2i} = \\cos 2 + i \\sin 2$")
left.set_aspect("equal")
left.set_xlabel("real part")
left.set_ylabel("imaginary part")
left.set_title("Partial sums of the series of $e^{2i}$")
left.legend(loc="lower left", fontsize=8)
right.semilogy(range(21), errors, "o-", label="error $|S_N - e^{2i}|$")
right.semilogy(range(21), bounds, "--", label="bound $e^2 \\, 2^{N+1}/(N+1)!$")
right.set_xlabel("number $N$ of the last term")
right.set_ylabel("error (logarithmic scale)")
right.set_title("How fast the series converges")
right.legend()
fig.tight_layout()
save_figure(fig, "euler_series",
            "Left: the partial sums $S_N = \\sum_{k=0}^{N} (2i)^k/k!$ for $N = 0$ to "
            "10 in the complex plane (blue points joined in order, the first five "
            "labelled), the unit circle (grey) and the point $e^{2i} = \\cos 2 + "
            "i \\sin 2$ (red star); axes real and imaginary part. The partial sums "
            "turn around the origin and close in on the star. Right: the error "
            "$|S_N - e^{2i}|$ against $N$ on a logarithmic scale (circles) and the "
            "bound $e^2 2^{N+1}/(N+1)!$ (dashed); the error shrinks faster and "
            "faster, each new term multiplying it by about $2/(N+2)$.")
step_ratios = [errors[n + 1] / errors[n] for n in range(10, 20)]
check(all(abs(ratio - theta / (n + 2)) < 0.1 * theta / (n + 2)
          for ratio, n in zip(step_ratios, range(10, 20))),
      "from N to N + 1 the error is multiplied by about theta/(N + 2) (N = 10 to 19)")
```

Out [6]:

```
Figure 01b.2 saved as Revision/textbook/figures/01b_2_euler_series.png
PASS from N to N + 1 the error is multiplied by about theta/(N + 2) (N = 10 to 19)
```

Text cell:

```
The next cell repeats Euler's formula with mpmath at 50 significant digits, in
particular $e^{i\pi} = -1$ (mpmath stores $\pi$ itself with 50 digits, so a
leftover imaginary part of about $10^{-51}$ appears; it is the rounding of $\pi$,
not an error of the formula), and proves the law of exponents
$e^{i\alpha} e^{i\beta} = e^{i(\alpha + \beta)}$ exactly with sympy: multiplied
out, the left side is
```

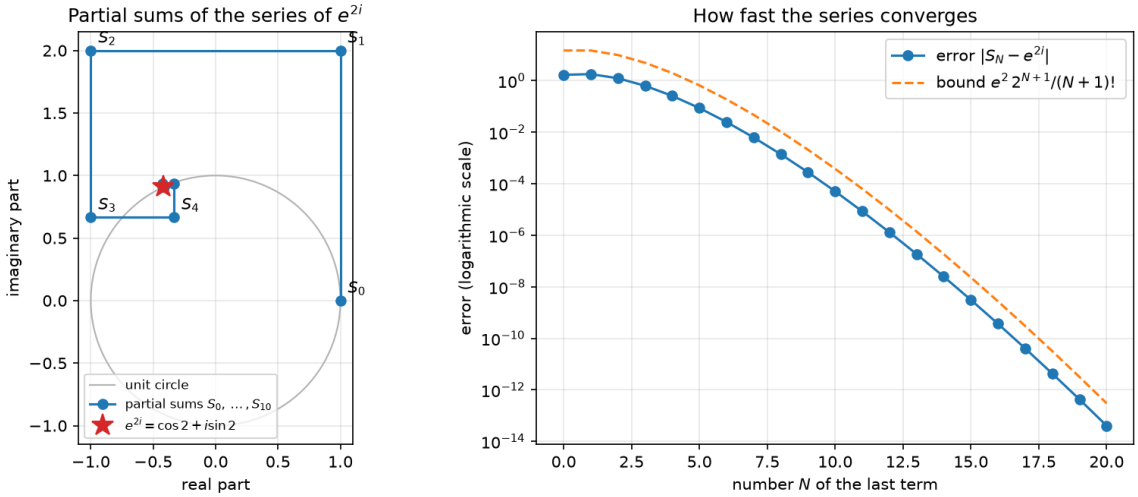

Figure 24: Left: the partial sums $S_N = \sum_{k=0}^{N} (2i)^k/k!$ for $N = 0$ to 10 in the complex plane (blue points joined in order, the first five labelled), the unit circle (grey) and the point $e^{2i} = \cos 2 + i \sin 2$ (red star); axes real and imaginary part. The partial sums turn around the origin and close in on the star. Right: the error $|S_N - e^{2i}|$ against N on a logarithmic scale (circles) and the bound $e^2 2^{N+1}/(N+1)!$ (dashed); the error shrinks faster and faster, each new term multiplying it by about $2/(N+2)$. (Notebook 01b, figure 2.)

```
$(\cos\alpha\cos\beta - \sin\alpha\sin\beta) + i(\sin\alpha\cos\beta +
\cos\alpha\sin\beta)$, which by the addition theorems is
$\cos(\alpha + \beta) + i \sin(\alpha + \beta)$. sympy's 'expand_trig' writes out
the addition theorems.
```

In [7]:

```
mpmath.mp.dps = 50  # work with 50 significant digits
i_mp = mpmath.mpc(0, 1)  # the imaginary unit as an mpmath number
euler_gap = abs(mpmath.exp(i_mp * 2) - (mpmath.cos(2) + i_mp * mpmath.sin(2)))
pi_gap = abs(mpmath.exp(i_mp * mpmath.pi) + 1)
e_i_pi = mpmath.exp(i_mp * mpmath.pi)
# pi itself is stored with 50 digits, so a tiny imaginary part of about 1e-51 is
# left over: the rounding of pi, not a failure of the formula.
say(f"e^(i pi): real part {mpmath.nstr(e_i_pi.real, 30)}, imaginary part of size "
    f"{mpmath.nstr(abs(e_i_pi.imag), 2)}")
check(euler_gap < mpmath.mpf("1e-45") and pi_gap < mpmath.mpf("1e-45"),
      "mpmath, 50 digits: e^(2i) = cos 2 + i sin 2 and e^(i pi) = -1")
alpha, beta = sp.symbols("alpha beta", real=True)
left_side = sp.expand((sp.cos(alpha) + I * sp.sin(alpha))
                      * (sp.cos(beta) + I * sp.sin(beta)))
right_side = sp.expand(sp.expand_trig(sp.cos(alpha + beta) + I * sp.sin(alpha + beta)))
check(sp.expand(left_side - right_side) == 0,
      "sympy: e^(i alpha) e^(i beta) = e^(i (alpha + beta)) for all real angles")
```

Out [7]:

```
e^(i pi): real part -1.0, imaginary part of size 1.0e-51
PASS mpmath, 50 digits: e^(2i) = cos 2 + i sin 2 and e^(i pi) = -1
PASS sympy: e^(i alpha) e^(i beta) = e^(i (alpha + beta)) for all real angles
```

Text cell:

```
## 8. Multiplying by a complex number rotates and stretches

Write $u = r e^{i\alpha}$ and a point of the plane as $p = \rho e^{i\varphi}$.
Then $up = r\rho \, e^{i(\alpha + \varphi)}$ by the law of exponents: the length
is multiplied by $r$ and the angle increased by $\alpha$. For $r = 1$ the
multiplication is a pure rotation.

The next cell draws the outline of the letter F (chosen because its mirror image
looks different, so a mirror would be noticed), and its images under
multiplication by $e^{i\pi/3}$ (60 degrees), $e^{5i\pi/6}$ (150 degrees) and
$0.6\,e^{-i\pi/2}$ (a quarter turn clockwise and a shrinking to 0.6). It checks
that the rotations keep the distance of every corner from the origin and turn
its angle by exactly $\alpha$.
```

In [8]:

```
letter_f = np.array([0, 2j, 1.2 + 2j, 1.2 + 1.6j, 0.4 + 1.6j, 0.4 + 1.1j, 1.0 + 1.1j,
                     1.0 + 0.7j, 0.4 + 0.7j, 0.4, 0]) + (0.3 + 0.2j)  # corners
factors = [(1, "original", "black"),
           (np.exp(1j * np.pi / 3), "times $e^{i\\pi/3}$", "tab:blue"),
           (np.exp(5j * np.pi / 6), "times $e^{5i\\pi/6}$", "tab:red"),
           (0.6 * np.exp(-1j * np.pi / 2), "times $0.6\\,e^{-i\\pi/2}$", "tab:green")]
fig, ax = plt.subplots(figsize=(8.4, 6.0))
for factor, label, color in factors:
    image = factor * letter_f  # every corner multiplied by the factor
    ax.fill(image.real, image.imag, color=color, alpha=0.35, label=label)
    ax.plot(image.real, image.imag, color=color, lw=1)
ax.plot([0], [0], "k+", ms=12)  # the origin, the centre of the rotations
ax.set_aspect("equal")
ax.set_xlabel("real part")
ax.set_ylabel("imaginary part")
ax.set_title("Multiplication by a complex number")
ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), fontsize=9)
save_figure(fig, "rotation_by_multiplication",
            "The letter F (black) and its images after multiplying every corner by "
            "$e^{i\\pi/3}$ (blue, a turn by 60 degrees), by $e^{5i\\pi/6}$ (red, a "
            "turn by 150 degrees) and by $0.6\\,e^{-i\\pi/2}$ (green, a quarter turn "
            "clockwise and a shrinking to 0.6); horizontal axis the real part, "
            "vertical axis the imaginary part, the cross marks the origin. The "
            "letter is turned about the origin and never mirrored.")
rotations_ok = True
for alpha_value in (np.pi / 3, 5 * np.pi / 6):
    image = np.exp(1j * alpha_value) * letter_f
    rotations_ok &= bool(np.max(np.abs(np.abs(image) - np.abs(letter_f))) < 1e-14)
    turned = np.angle(image / letter_f)  # the change of every angle
    rotations_ok &= bool(np.max(np.abs(turned - alpha_value)) < 1e-12)
```

```
check(rotations_ok, "multiplying by e^(i alpha) keeps lengths and adds alpha to "
      "every angle")
```

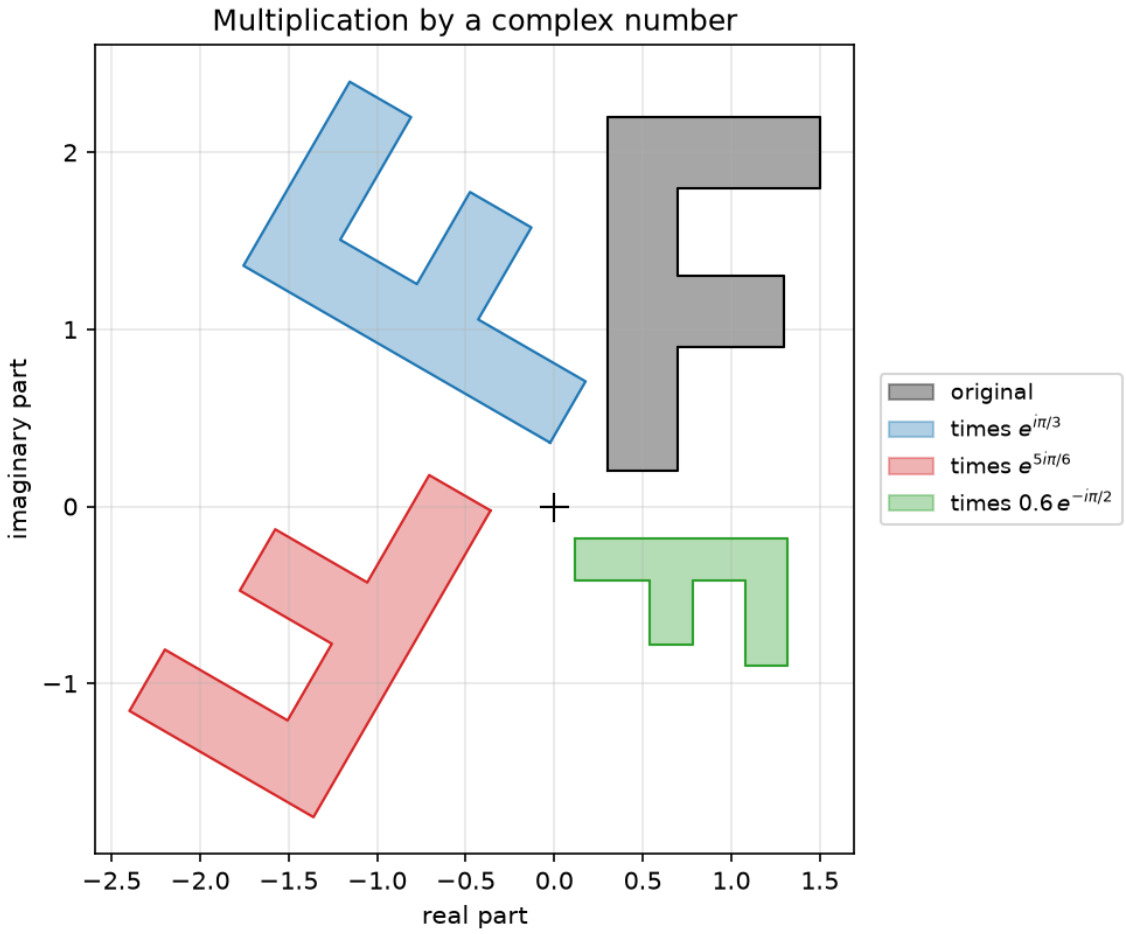

Figure 25: The letter F (black) and its images after multiplying every corner by $e^{i\pi/3}$ (blue, a turn by 60 degrees), by $e^{5i\pi/6}$ (red, a turn by 150 degrees) and by $0.6\,e^{-i\pi/2}$ (green, a quarter turn clockwise and a shrinking to 0.6); horizontal axis the real part, vertical axis the imaginary part, the cross marks the origin. The letter is turned about the origin and never mirrored. (Notebook 01b, figure 3.)

Out [8]:

```
Figure 01b.3 saved as Revision/textbook/figures/01b_3_rotation_by_multiplication.png
PASS multiplying by e^(i alpha) keeps lengths and adds alpha to every angle
```

Text cell:

```
## 9. The same rotation as a real 2 x 2 matrix

Multiplying $x + iy$ by $a + ib$ gives $(ax - by) + i(bx + ay)$. Written for the
pair $(x, y)$ this is the matrix product
```

```
$$\begin{pmatrix} a & -b \\ b & a \end{pmatrix}
\begin{pmatrix} x \\ y \end{pmatrix} =
\begin{pmatrix} ax - by \\ bx + ay \end{pmatrix}.$$

So every complex number $a + ib$ has a real matrix $M(a + ib)$ with the rows
$(a, -b)$ and $(b, a)$. The number 1 has the identity matrix and $i$ has the
matrix $J$ with rows $(0, -1)$ and $(1, 0)$. The next cell checks, exactly with
sympy, that

- $J^2 = -I$: a *real* matrix plays the role of $i$;
- $M(z) M(w) = M(zw)$ for all $z$ and $w$: multiplying matrices is the same as
  multiplying the numbers;
- $\det M(a + ib) = a^2 + b^2 = |z|^2$;
- the rotation matrix $R(\alpha) = M(e^{i\alpha})$ obeys
  $R(\alpha) R(\beta) = R(\alpha + \beta)$, $\det R(\alpha) = 1$ and
  $R(\alpha)^T R(\alpha) = I$ (so it keeps $x^2 + y^2$).

It also checks in numbers that rotating the corners of the letter F with
$R(\pi/3)$ gives the same points as multiplying them by $e^{i\pi/3}$.
```

In [9]:

```
def M(re_part, im_part):
    """The real 2 x 2 matrix of the complex number re_part + i im_part."""
    return sp.Matrix([[re_part, -im_part], [im_part, re_part]])


J = M(0, 1)  # the matrix of i
check(J * J == -sp.eye(2), "J J = -I: the real matrix J behaves like i")
zw_re, zw_im = a * c - b * d, a * d + b * c  # z w = (ac - bd) + i (ad + bc)
check((M(a, b) * M(c, d) - M(zw_re, zw_im)).expand() == sp.zeros(2, 2),
      "M(z) M(w) = M(z w) for all complex z and w")
check(sp.expand(M(a, b).det()) == a ** 2 + b ** 2, "det M(a + i b) = a^2 + b^2")


def R(angle):
    """The rotation matrix of the angle: the matrix of e^(i angle)."""
    return M(sp.cos(angle), sp.sin(angle))


product_rule = (R(alpha) * R(beta) - R(alpha + beta)).applyfunc(sp.expand_trig)
check(product_rule.expand() == sp.zeros(2, 2), "R(alpha) R(beta) = R(alpha + beta)")
check(sp.simplify(R(alpha).det()) == 1 and
      (R(alpha).T * R(alpha)).applyfunc(sp.simplify) == sp.eye(2),
      "det R(alpha) = 1 and R(alpha)^T R(alpha) = I")
rotation = np.array(R(sp.pi / 3).evalf(), dtype=float)  # R(pi/3) as numbers
corners = np.array([letter_f.real, letter_f.imag])  # 2 rows: x and y of each corner
by_matrix = rotation @ corners
by_number = np.exp(1j * np.pi / 3) * letter_f
check(np.max(np.abs(by_matrix[0] + 1j * by_matrix[1] - by_number)) < 1e-14,
      "the rotation matrix R(pi/3) moves the corners exactly like e^(i pi/3)")
```

Out [9]:

```
PASS J J = -I: the real matrix J behaves like i
PASS M(z) M(w) = M(z w) for all complex z and w
PASS det M(a + i b) = a^2 + b^2
PASS R(alpha) R(beta) = R(alpha + beta)
PASS det R(alpha) = 1 and R(alpha)^T R(alpha) = I
PASS the rotation matrix R(pi/3) moves the corners exactly like e^(i pi/3)
```

Text cell:

```
## 10. The roots of unity

The numbers $z_k = e^{2\pi i k/n}$, $k = 0, 1, \dots, n-1$, satisfy
$z_k^n = e^{2\pi i k} = 1$: they are the $n$ roots of unity. They are the corners
of a regular polygon with $n$ sides on the unit circle, and their sum is 0 (the
sum is unchanged when every $z_k$ is multiplied by $z_1$, which only re-orders
them; a number that does not change when multiplied by $z_1 \neq 1$ must be 0).

The next cell checks this in numbers for $n = 3$ and $n = 8$, writes the eight
eighth roots exactly with sympy (‘expand_complex‘ writes a number as real part
plus $i$ times imaginary part), checks their eighth powers and their sum exactly,
and draws both polygons.
```

In [10]:

```
for n in (3, 8):
    roots = np.exp(2j * np.pi * np.arange(n) / n)  # e^(2 pi i k/n), k = 0 ... n-1
    check(np.max(np.abs(roots ** n - 1)) < 1e-12 and abs(roots.sum()) < 1e-12,
          f"the {n} roots of unity of order {n}: z^{n} = 1 and their sum is 0")
eighth = [sp.expand_complex(sp.exp(2 * sp.pi * I * k / 8)) for k in range(8)]
say("the eighth roots of unity, exactly: " + ", ".join(str(r) for r in eighth))
check(all(sp.simplify(r ** 8 - 1) == 0 for r in eighth) and
      sp.simplify(sum(eighth)) == 0,
      "sympy: each exact eighth root has eighth power 1, and they add up to 0")
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.8))
for ax, n in zip(axes, (3, 8)):
    roots = np.exp(2j * np.pi * np.arange(n + 1) / n)  # the first root again at the end
    ax.plot(circle.real, circle.imag, color="0.75", lw=1)
    ax.plot(roots.real, roots.imag, "o-", color="tab:blue")
    for k in range(n):
        arrow(ax, roots[k], "tab:blue", f"$z_{k}$")
    ax.set_aspect("equal")
    ax.set_xlim(-1.45, 1.45)
    ax.set_ylim(-1.35, 1.35)
    ax.set_xlabel("real part")
    ax.set_ylabel("imaginary part")
    ax.set_title(f"the {n} roots of $z^{n} = 1$")
fig.tight_layout()
save_figure(fig, "roots_of_unity",
            "The roots of unity $z_k = e^{2\\pi i k/n}$ for $n = 3$ (left) and "
```

```
"$n = 8$ (right) as arrows from the origin to the unit circle (grey); "
"axes real and imaginary part. They are the corners of a regular "
"triangle and a regular octagon; the arrows of each picture add up to "
"zero.")
```

Out [10]:

```
PASS the 3 roots of unity of order 3: z^3 = 1 and their sum is 0
PASS the 8 roots of unity of order 8: z^8 = 1 and their sum is 0
the eighth roots of unity, exactly: 1, sqrt(2)/2 + sqrt(2)*I/2, I, -sqrt(2)/2 +
    sqrt(2)*I/2, -1, -sqrt(2)/2 - sqrt(2)*I/2, -I, sqrt(2)/2 - sqrt(2)*I/2
PASS sympy: each exact eighth root has eighth power 1, and they add up to 0
```

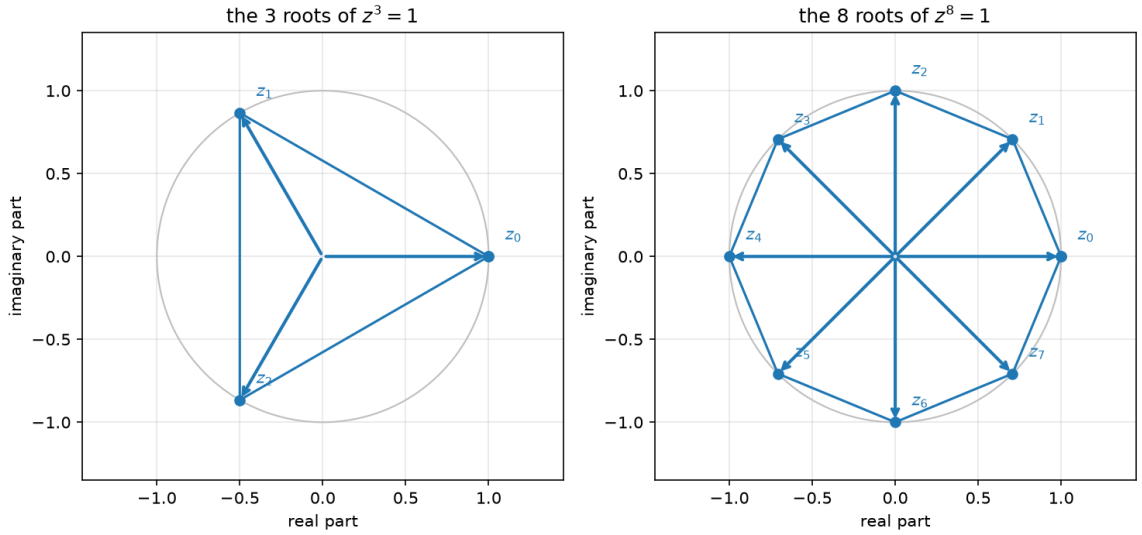

Figure 26: The roots of unity $z_k = e^{2\pi ik/n}$ for $n = 3$ (left) and $n = 8$ (right) as arrows from the origin to the unit circle (grey); axes real and imaginary part. They are the corners of a regular triangle and a regular octagon; the arrows of each picture add up to zero. (Notebook 01b, figure 4.)

Out [10]:

```
Figure 01b.4 saved as Revision/textbook/figures/01b_4_roots_of_unity.png
```

Text cell:

```
## 11. Conjugation and real numbers

Conjugation changes the sign of the imaginary part and nothing else. So a real
number, which has imaginary part 0, is its own conjugate: $x^* = x$; and a list
of real numbers is not changed at all by conjugating every entry. Conjugating
twice gives back the original number, and the conjugate of $e^{i\alpha}$ is
$e^{-i\alpha}$: conjugation turns a rotation into the opposite rotation.
```

```
This simple fact matters later in the course: the author's gamma matrices are
real, and for real quantities conjugation does nothing at all. So a conjugation
that is meant to change a real field cannot be plain complex conjugation: it
must act through a *matrix* on the field's components. The next cell checks the
four statements with numpy (`.conj()` conjugates every entry of an array).
```

In [11]:

```
v_real = np.array([0.5, -2.0, 3.0])  # a list of real numbers
v_complex = np.array([1 + 2j, -1j, 4.0])  # a list with imaginary parts
check((v_real.conj() == v_real).all(),
      "conjugation leaves a list of real numbers unchanged")
check(not (v_complex.conj() == v_complex).all(),
      "conjugation changes a list whose imaginary parts are not all 0")
check((v_complex.conj().conj() == v_complex).all(),
      "conjugating twice gives back the original list")
check(abs(np.exp(0.7j).conjugate() - np.exp(-0.7j)) < 1e-15,
      "the conjugate of e^(i alpha) is e^(-i alpha)")
```

Out [11]:

```
PASS conjugation leaves a list of real numbers unchanged
PASS conjugation changes a list whose imaginary parts are not all 0
PASS conjugating twice gives back the original list
PASS the conjugate of e^(i alpha) is e^(-i alpha)
```

Text cell:

```
## 12. Rotations and boosts

A rotation mixes two directions of the same kind and keeps $x^2 + y^2$. A boost
mixes a time-like coordinate $t$ with a space-like coordinate $x$:

$$\begin{pmatrix} t' \\ x' \end{pmatrix} =
\begin{pmatrix} \cosh\varphi & \sinh\varphi \\ \sinh\varphi & \cosh\varphi
\end{pmatrix} \begin{pmatrix} t \\ x \end{pmatrix}.$$

With the frame metric of this pair, $\eta = \mathrm{diag}(-1, +1)$ ($t$ is
time-like, $x$ space-like, as $x_4$ and $x_0$ in the author's coordinates), the
boost $\Lambda$ satisfies $\Lambda^T \eta \Lambda = \eta$, which says that
$-t'^2 + x'^2 = -t^2 + x^2$; it follows from $\cosh^2 - \sinh^2 = 1$. The next
cell checks exactly with sympy:

- $\Lambda^T \eta \Lambda = \eta$ and $\det \Lambda = 1$;
- rapidities add: $\Lambda(\varphi_1) \Lambda(\varphi_2) = \Lambda(\varphi_1 +
  \varphi_2)$ (the addition theorems of $\cosh$ and $\sinh$);
- the two light-like directions $(t, x) = (1, 1)$ and $(1, -1)$ are only
  stretched: $\Lambda (1, 1) = e^{\varphi} (1, 1)$ and
  $\Lambda (1, -1) = e^{-\varphi} (1, -1)$, because
  $\cosh\varphi \pm \sinh\varphi = e^{\pm\varphi}$.
```

In [12]:

```
phi, phi1, phi2 = sp.symbols("phi phi1 phi2", real=True)


def boost(rapidity):
    """The boost matrix acting on the pair (t, x)."""
    return sp.Matrix([[sp.cosh(rapidity), sp.sinh(rapidity)],
                      [sp.sinh(rapidity), sp.cosh(rapidity)]])


eta_tx = sp.diag(-1, 1)  # t time-like (-1), x space-like (+1)
L = boost(phi)
check((L.T * eta_tx * L - eta_tx).applyfunc(sp.simplify) == sp.zeros(2, 2) and
      sp.simplify(L.det()) == 1,
      "a boost keeps -t^2 + x^2 (Lambda^T eta Lambda = eta) and has det 1")
added = (boost(phi1) * boost(phi2) - boost(phi1 + phi2)).applyfunc(sp.expand_trig)
check(added.expand() == sp.zeros(2, 2), "rapidities add: boost(phi1) boost(phi2) = "
      "boost(phi1 + phi2)")
light_plus = (L * sp.Matrix([1, 1]) - sp.exp(phi) * sp.Matrix([1, 1]))
light_minus = (L * sp.Matrix([1, -1]) - sp.exp(-phi) * sp.Matrix([1, -1]))
# rewrite(sp.exp) writes cosh and sinh with exponentials, then simplify:
check(light_plus.applyfunc(lambda e: sp.simplify(e.rewrite(sp.exp)))
      == sp.zeros(2, 1) and
      light_minus.applyfunc(lambda e: sp.simplify(e.rewrite(sp.exp)))
      == sp.zeros(2, 1),
      "the light-like directions are stretched by e^phi and by e^(-phi)")
```

Out [12]:

```
PASS a boost keeps -t^2 + x^2 (Lambda^T eta Lambda = eta) and has det 1
PASS rapidities add: boost(phi1) boost(phi2) = boost(phi1 + phi2)
PASS the light-like directions are stretched by e^phi and by e^(-phi)
```

Text cell:

```
The next cell draws what the two kinds of transformation do to one point. Left:
the point $(1, 0)$ rotated by angles from 0 to $2\pi$ runs around the circle
$x^2 + y^2 = 1$. Right: the event $(t, x) = (1, 0)$ boosted with rapidities from
$-2$ to $2$ runs along the hyperbola $t^2 - x^2 = 1$ and never crosses the two
light-like lines $t = \pm x$. The points for the angles and rapidities
$-1, -0.5, 0, 0.5, 1$ are marked.
```

In [13]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 5.0))
angles = np.linspace(0.0, 2.0 * np.pi, 400)
left.plot(np.cos(angles), np.sin(angles), color="tab:blue",
          label="rotations of $(1, 0)$")
marks = np.array([-1.0, -0.5, 0.0, 0.5, 1.0])
```

```
left.plot(np.cos(marks), np.sin(marks), "o", color="tab:blue")
for value in marks:
    left.text(1.06 * np.cos(value), 1.06 * np.sin(value), f"{value:+.1f}", fontsize=8)
left.set_aspect("equal")
left.set_xlim(-1.6, 1.6)
left.set_ylim(-1.6, 1.6)
left.set_xlabel("$x$")
left.set_ylabel("$y$")
left.set_title("rotation: $x^2 + y^2$ stays 1")
left.legend(loc="lower left", fontsize=8)
rapidities = np.linspace(-2.0, 2.0, 400)
right.plot(np.sinh(rapidities), np.cosh(rapidities), color="tab:red",
           label="boosts of $(t, x) = (1, 0)$")
right.plot(np.sinh(marks), np.cosh(marks), "o", color="tab:red")
for value in marks:  # labels left of the points on the left half, right otherwise
    shift = 0.12 if value >= 0 else -0.62
    right.text(np.sinh(value) + shift, np.cosh(value) + 0.12, f"{value:+.1f}",
               fontsize=8)
edge = np.linspace(-3.8, 3.8, 2)
right.plot(edge, edge, "k--", lw=1, label="light-like lines $t = \\pm x$")
right.plot(edge, -edge, "k--", lw=1)
right.set_aspect("equal")
right.set_xlim(-3.8, 3.8)
right.set_ylim(-0.5, 3.9)
right.set_xlabel("space-like coordinate $x$")
right.set_ylabel("time-like coordinate $t$")
right.set_title("boost: $t^2 - x^2$ stays 1")
right.legend(loc="lower right", fontsize=8)
fig.tight_layout()
save_figure(fig, "rotations_and_boosts",
            "Left: the point $(1, 0)$ rotated by all angles from 0 to $2\\pi$ "
            "(blue circle $x^2 + y^2 = 1$; the dots are the angles $-1$ to $1$ in "
            "steps of 0.5); axes $x$ and $y$. Right: the event $(t, x) = (1, 0)$ "
            "boosted with rapidities from $-2$ to $2$ (red hyperbola "
            "$t^2 - x^2 = 1$; the dots are the rapidities $-1$ to $1$ in steps of "
            "0.5) and the light-like lines $t = \\pm x$ (dashed); horizontal axis "
            "the space-like $x$, vertical axis the time-like $t$ (pure numbers). "
            "A rotation keeps a circle, a boost keeps a hyperbola.")
on_hyperbola = np.cosh(rapidities) ** 2 - np.sinh(rapidities) ** 2
check(np.max(np.abs(on_hyperbola - 1)) < 1e-12,
      "numbers: cosh^2 - sinh^2 = 1 at 400 rapidities from -2 to 2")
```

Out [13]:

```
Figure 01b.5 saved as Revision/textbook/figures/01b_5_rotations_and_boosts.png
PASS numbers: cosh^2 - sinh^2 = 1 at 400 rapidities from -2 to 2
```

Text cell:

```
## 13. Oscillation and growth

For a real frequency $\varepsilon$ the wave $e^{-i\varepsilon t} =
```

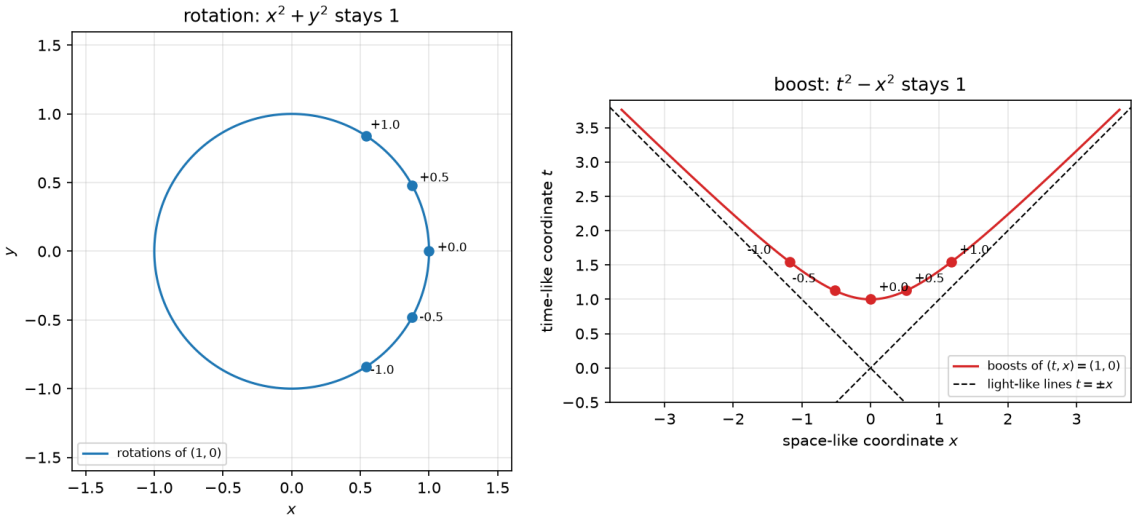

Figure 27: Left: the point $(1, 0)$ rotated by all angles from 0 to 2π (blue circle $x^2 + y^2 = 1$; the dots are the angles -1 to 1 in steps of 0.5); axes x and y. Right: the event $(t, x) = (1, 0)$ boosted with rapidities from -2 to 2 (red hyperbola $t^2 - x^2 = 1$; the dots are the rapidities -1 to 1 in steps of 0.5) and the light-like lines $t = \pm x$ (dashed); horizontal axis the space-like x, vertical axis the time-like t (pure numbers). A rotation keeps a circle, a boost keeps a hyperbola. (Notebook 01b, figure 5.)

```
\cos(\varepsilon t) - i \sin(\varepsilon t)$ has modulus 1 at all times: its real
and imaginary parts oscillate. For an imaginary frequency $\varepsilon = i\gamma$
($\gamma$ real) the same formula gives $e^{-i(i\gamma)t} = e^{\gamma t}$: no
oscillation, but exponential growth for $\gamma > 0$ (and decay for
$\gamma < 0$). A complex frequency $\varepsilon = 2 + 0.3i$ gives
$e^{-i\varepsilon t} = e^{0.3t} e^{-2it}$: an oscillation whose size grows like
$e^{0.3t}$.

The next cell checks these three statements at 501 times from 0 to 10 and draws
them.
```

In [14]:

```
t = np.linspace(0.0, 10.0, 501)
wave = np.exp(-1j * 2.0 * t)  # real frequency 2
growing = np.exp(-1j * (0.3j) * t)  # imaginary frequency 0.3 i
decaying = np.exp(-1j * (-0.3j) * t)  # imaginary frequency -0.3 i
mixed = np.exp(-1j * (2.0 + 0.3j) * t)  # complex frequency 2 + 0.3 i
check(np.max(np.abs(np.abs(wave) - 1.0)) < 1e-14,
      "real frequency: |e^(-i eps t)| = 1 at every time")
check(np.max(np.abs(growing - np.exp(0.3 * t))) < 1e-12 * np.exp(3.0),
      "imaginary frequency 0.3 i: e^(-i eps t) = e^(0.3 t), real and growing")
check(np.max(np.abs(np.abs(mixed) - np.exp(0.3 * t))) < 1e-12 * np.exp(3.0),
      "complex frequency 2 + 0.3 i: the size of the wave is e^(0.3 t)")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4))
left.plot(t, wave.real, label="real part $\\cos 2t$")
left.plot(t, wave.imag, "--", label="imaginary part $-\\sin 2t$")
left.plot(t, np.abs(wave), color="black", lw=2, label="modulus $= 1$")
```

```
left.set_xlabel("time $t$")
left.set_ylabel("value")
left.set_title("real frequency $\\varepsilon = 2$")
left.set_ylim(-1.75, 1.3)  # room below the curves for the legend
left.legend(loc="lower center", ncol=3, fontsize=8)
right.plot(t, growing.real, color="tab:red", lw=2,
           label="$\\varepsilon = 0.3i$: $e^{0.3t}$")
right.plot(t, decaying.real, color="tab:blue", lw=2,
           label="$\\varepsilon = -0.3i$: $e^{-0.3t}$")
right.plot(t, mixed.real, color="tab:green",
           label="$\\varepsilon = 2 + 0.3i$: real part")
right.plot(t, -np.exp(0.3 * t), ":", color="tab:red")  # the lower envelope
right.set_xlabel("time $t$")
right.set_ylabel("value")
right.set_title("imaginary and complex frequencies")
right.legend(loc="upper left", fontsize=8)
fig.tight_layout()
save_figure(fig, "oscillation_and_growth",
            "Left: the wave $e^{-i\\varepsilon t}$ with the real frequency "
            "$\\varepsilon = 2$; its real part (solid) and imaginary part (dashed) "
            "oscillate and its modulus (thick black) stays 1. Right: imaginary "
            "frequencies $\\varepsilon = 0.3i$ (red, growth $e^{0.3t}$) and "
            "$\\varepsilon = -0.3i$ (blue, decay $e^{-0.3t}$), and the real part of "
            "the wave with the complex frequency $2 + 0.3i$ (green), which "
            "oscillates between the curves $\\pm e^{0.3t}$ (red solid and dotted); "
            "horizontal axis the time $t$, vertical axis the value (pure numbers).")
```

Out [14]:

```
PASS real frequency: |e^(-i eps t)| = 1 at every time
PASS imaginary frequency 0.3 i: e^(-i eps t) = e^(0.3 t), real and growing
PASS complex frequency 2 + 0.3 i: the size of the wave is e^(0.3 t)
```

Out [14]:

```
Figure 01b.6 saved as Revision/textbook/figures/01b_6_oscillation_and_growth.png
```

Text cell:

```
## 14. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [15]:

```
figure_names = ["01b_1_complex_plane.png", "01b_2_euler_series.png",
                "01b_3_rotation_by_multiplication.png", "01b_4_roots_of_unity.png",
                "01b_5_rotations_and_boosts.png", "01b_6_oscillation_and_growth.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

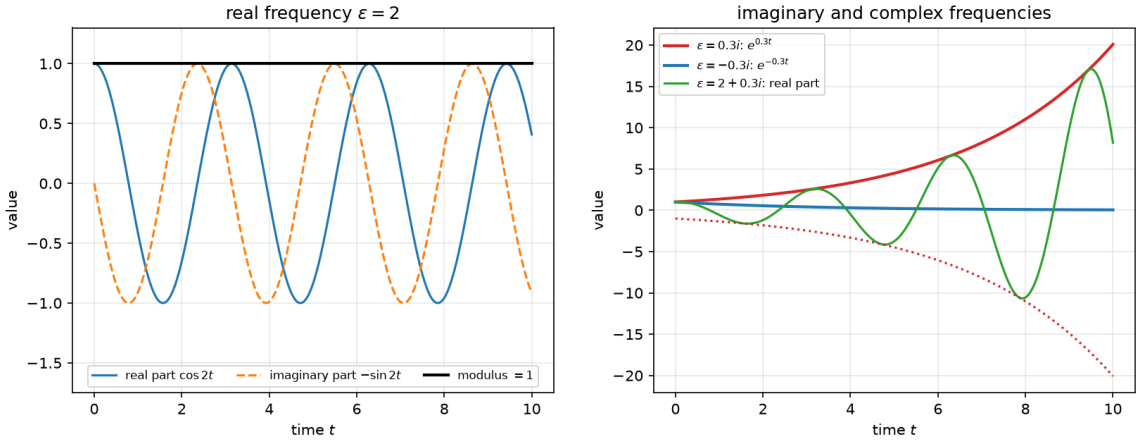

Figure 28: Left: the wave $e^{-i\varepsilon t}$ with the real frequency $\varepsilon = 2$; its real part (solid) and imaginary part (dashed) oscillate and its modulus (thick black) stays 1. Right: imaginary frequencies $\varepsilon = 0.3i$ (red, growth $e^{0.3t}$) and $\varepsilon = -0.3i$ (blue, decay $e^{-0.3t}$), and the real part of the wave with the complex frequency $2 + 0.3i$ (green), which oscillates between the curves $\pm e^{0.3t}$ (red solid and dotted); horizontal axis the time t, vertical axis the value (pure numbers). (Notebook 01b, figure 6.)

Out [15]:

```
PASS all 6 figure files of this notebook exist
ALL 34 CHECKS PASSED (notebook 01b)
```

Text cell:

```
## 15. What this notebook showed

- Complex numbers $a + ib$ obey the ordinary rules of algebra with $i^2 = -1$;
  $(zw)^* = z^* w^*$, $zz^* = |z|^2$ and $|zw| = |z|\,|w|$ hold for all of them
  (proved with sympy symbols).
- The exponential series at $i\theta$ converges to $\cos\theta + i\sin\theta$
  (Euler's formula); after 21 terms the error at $\theta = 2$ is below
  $10^{-12}$, and every error lies below the bound
  $e^\theta \theta^{N+1}/(N+1)!$; mpmath confirms $e^{i\pi} = -1$ to 50 digits.
- Multiplying by $e^{i\alpha}$ rotates the plane by $\alpha$; the same rotation
  is the real matrix $R(\alpha)$, and the real matrix $J$ of $i$ has $J^2 = -I$.
- The $n$ roots of unity are the corners of a regular polygon and add up to 0.
- Conjugation leaves real numbers, and lists of real numbers, unchanged.
- Rotations keep $x^2 + y^2$; boosts, which mix a time-like and a space-like
  direction, keep $x^2 - t^2$, add their rapidities, and stretch the two
  light-like directions by $e^{\varphi}$ and $e^{-\varphi}$.
- A real frequency gives an oscillation of constant size; an imaginary frequency
  gives exponential growth or decay.
```

1.17 Line-by-line walk-through of Notebook 01b

The notebook has fifteen code cells, In [1] to In [15]. As in Section 1.9, a quoted line `...)` stands for the remaining lines of a figure caption, which Section 1.16 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 1.15. Its code is, line for line, the set-up code of Notebook 01f explained in Section 1.9, with the single difference `NOTEBOOK_ID = "01b"`, so that the figures are named `01b_1_complex_plane.png` and so on and the captions file is `Revision/textbook/figures/01b.captions.json`. It prints one line, Set-up of notebook 01b complete: repository folder found, helpers defined.

In [2], complex numbers in Python and in sympy.

```
import cmath  # functions of complex numbers: phase (the argument), exp
import math  # cos, sin, exp, factorial of real numbers

import mpmath  # numbers with as many digits as we ask for
import numpy as np  # arrays of numbers
import sympy as sp  # exact arithmetic
```

The module `cmath` (part of Python) holds the functions of complex numbers, among them `cmath.phase`, the argument of Section 1.11; `math` holds the same functions for real numbers. mpmath, numpy and sympy were introduced in Section 1.9.

```
z = 1 + 2j  # Python writes the imaginary unit as j, right after a number
w = 3 - 1j
say(f"Python: z = {z}, w = {w}, z + w = {z + w}, z w = {z * w}")
```

`2j` is Python's spelling of $2i$, so `z` is $1 + 2i$ and `w` is $3 - i$ (`1j` is i; a lone `j` would be a variable name). These are floating-point complex numbers: two floating-point numbers, the real and the imaginary part. The line prints `Python:  z = (1+2j), w = (3-1j), z + w = (4+1j), z w = (5+5j)`: Python writes a complex number in brackets.

```
I = sp.I  # sympy's exact imaginary unit
z_exact = 1 + 2 * I
w_exact = 3 - I
product = sp.expand(z_exact * w_exact)  # multiply out, with I**2 = -1
quotient = sp.simplify(z_exact / w_exact)
say(f"sympy: z w = {product}, z / w = {quotient}")
```

`sp.I` is sympy's exact imaginary unit; the name `I` is given to it. `z_exact` and `w_exact` are the same two numbers in exact form. `sp.expand` multiplies out a product (and replaces i^2 by -1); `sp.simplify` brings the quotient into its simplest form, which for a complex number is real part plus imaginary part. Output: `sympy:  z w = 5 + 5*I, z / w = 1/10 + 7*I/10`, the results of Section 1.10.

```
check(I ** 2 == -1, "i^2 = -1")
check(product == 5 + 5 * I and complex(product) == z * w, "(1 + 2i)(3 - i) = 5 + 5i")
check(sp.simplify(quotient - (1 + 7 * I) / 10) == 0, "(1 + 2i)/(3 - i) = (1 + 7i)/10")
```

Three checks. The first confirms that sympy's unit squares to -1. The second compares the exact product with $5 + 5i$ and also with Python's floating-point product (`complex(...)` turns the exact number into a Python complex number). The third confirms the quotient by showing that its difference from $(1 + 7i)/10$ simplifies to 0. Three PASS lines.

In [3], the rules of the conjugate and the modulus, for all complex numbers.

```
a, b, c, d = sp.symbols("a b c d", real=True)  # any real numbers
zz = a + b * I
ww = c + d * I
```

Four sympy symbols for real numbers; `zz` $= a + ib$ and `ww` $= c + id$ stand for any two complex numbers.

```
check(sp.expand(sp.conjugate(zz * ww) - sp.conjugate(zz) * sp.conjugate(ww)) == 0,
      "(z w)* = z* w* for all complex z and w")
check(sp.expand(zz * sp.conjugate(zz)) == a ** 2 + b ** 2,
      "z z* = a^2 + b^2 = |z|^2 for every z = a + i b")
check(sp.expand(zz * ww * sp.conjugate(zz * ww)
                - (a ** 2 + b ** 2) * (c ** 2 + d ** 2)) == 0,
      "|z w|^2 = |z|^2 |w|^2 for all complex z and w")
```

`sp.conjugate` forms the conjugate. Each check multiplies out the difference of the two sides of one of the three rules of Section 1.10 and requires 0. Because the symbols stand for all real numbers, these are proofs, not tests of examples. Three PASS lines.

```
say(f"|z| = {sp.Abs(z_exact)}, |w| = {sp.Abs(w_exact)}, "
    f"|z w| = {sp.Abs(product)}")
```

`sp.Abs` is the modulus. Output: `|z| = sqrt(5), |w| = sqrt(10), |z w| = 5*sqrt(2)`, and indeed $\sqrt{5} \cdot \sqrt{10} = \sqrt{50} = 5\sqrt{2}$, rule 3.

In [4], complex numbers as arrows.

```
def arrow(ax, number, color, label, start=0j, style="-"):
    """Draw the complex number as an arrow from start to start + number."""
    tip = start + number
    ax.annotate("", xy=(tip.real, tip.imag), xytext=(start.real, start.imag),
                arrowprops={"arrowstyle": "->", "color": color, "lw": 2,
                            "linestyle": style})
    ax.text(tip.real + 0.1, tip.imag + 0.1, label, color=color)
```

A helper that draws a complex number as an arrow. The arguments after the first ones have default values (`start=0j`: the arrow starts at the origin unless another start is given; `style="-"`: a solid line). `.real` and `.imag` are the real and imaginary parts of a Python complex number. `ax.annotate("", xy=..., xytext=...)` draws an arrow without text from the point `xytext` to the point `xy`; the dictionary `arrowprops` sets its look (an arrow head, the colour, the line width 2 and the line style). `ax.text` writes the label a little to the right of and above the tip.

```
fig, ax = plt.subplots(figsize=(7.0, 5.2))
arrow(ax, z, "tab:blue", "$z = 1 + 2i$")
arrow(ax, w, "tab:red", "$w = 3 - i$")
arrow(ax, w, "tab:red", "", start=z, style="--")  # w moved to the tip of z
arrow(ax, z + w, "black", "$z + w$")
arrow(ax, z.conjugate(), "tab:green", "$z^* = 1 - 2i$")
arrow(ax, 1j * z, "tab:purple", "$iz = -2 + i$")
```

A figure with one pair of axes, then six arrows: z, w, the arrow w again starting at the tip of z (dashed, `style` set to a string of two hyphens), their sum, the conjugate (`.conjugate()` is Python's conjugate) and iz.

```
ax.axhline(0, color="0.5", lw=0.8)  # the real axis
ax.axvline(0, color="0.5", lw=0.8)  # the imaginary axis
ax.set_xlim(-2.8, 5.0)
ax.set_ylim(-2.6, 3.4)
ax.set_aspect("equal")
ax.set_xlabel("real part Re")
ax.set_ylabel("imaginary part Im")
ax.set_title("Complex numbers as arrows")
save_figure(fig, "complex_plane",
            "Complex numbers as arrows in the complex plane; horizontal axis the "
            ...)
```

The two grey lines are the real and the imaginary axis. `set_xlim` and `set_ylim` fix the ranges; `set_aspect("equal")` makes one unit equally long on both axes, so that right angles look right. The labels, the title and `save_figure` follow. What Figure 01b.1 shows: the sum $z + w$ is the diagonal of the parallelogram spanned by z and w; z^* is z mirrored in the real axis; iz is z turned by a right angle.

```
turn = cmath.phase(1j * z) - cmath.phase(z)  # difference of the two arguments
check(abs(abs(1j * z) - abs(z)) < 1e-15 and abs(turn - math.pi / 2) < 1e-12,
      "multiplying by i keeps the length and turns by pi/2")
```

`cmath.phase` gives the argument in radians; `turn` is how much larger the argument of iz is than that of z. `abs` of a Python complex number is its modulus. The check requires the same length to 10^{-15} and a turn of $\pi/2$ to 10^{-12}, the statement of Section 1.10. The output is the figure line and the PASS line.

In [5], Euler's formula from the exponential series.

```
theta = 2.0
target = complex(math.cos(theta), math.sin(theta))  # cos(theta) + i sin(theta)
partial_sums = []
total = 0j
term = 1 + 0j  # the term number 0: (i theta)^0 / 0! = 1
```

The angle $\theta = 2$; `target` is $\cos 2 + i \sin 2$ as a Python complex number (`complex(x, y)` is $x + iy$). The list `partial_sums` will hold $S_0, \ldots, S_{20}$; `total` starts at 0 and `term` at the term number 0, which is 1.

```
for k in range(21):
    total += term
    partial_sums.append(total)  # S_k
    term = term * 1j * theta / (k + 1)  # the next term
errors = [abs(s - target) for s in partial_sums]
```

The loop runs for $k = 0$ to 20: it adds the current term, stores the partial sum S_k, and makes the next term from the current one, $(i\theta)^{k+1}/(k+1)! = \frac{(i\theta)^k}{k!} \cdot \frac{i\theta}{k+1}$ (one more factor $i\theta$ and one more factor $k+1$ in the factorial). Making each term from the previous one avoids computing large powers and factorials. `errors` holds the 21 errors $|S_N - e^{2i}|$.

```
bounds = [math.exp(theta) * theta ** (n + 1) / math.factorial(n + 1)
          for n in range(21)]
for n in (0, 1, 2, 5, 10, 15, 20):
    say(f"N = {n:2d}: S_N = {partial_sums[n].real:+.12f} {partial_sums[n].imag:+.12f} i,"
        f" error {errors[n]:.1e}")
say(f"cos 2 + i sin 2 = {target.real:+.12f} {target.imag:+.12f} i")
```

`bounds` holds the bound $e^\theta \theta^{N+1}/(N+1)!$ of Section 1.11 for $N = 0$ to 20. Seven partial sums are printed with 12 digits after the point and a sign (`:+.12f`), with their errors. The output shows $S_0 = 1$, $S_1 = 1 + 2i$, $S_2 = -1 + 2i$, and then the partial sums settling: $S_{20} = -0.416146836547 + 0.909297426826\,i$, equal in all printed digits to $\cos 2 + i \sin 2$, with the error 4.1×10^{-14}; the errors are 1.7, 1.8, 1.2, 0.086, 5.1×10^{-5}, 3.1×10^{-9}, 4.1×10^{-14}. The first errors do not shrink, because for $k < \theta$ the terms still grow.

```
check(all(e <= b + 1e-15 for e, b in zip(errors, bounds)),
      "the error of every partial sum is below e^theta theta^(N+1)/(N+1)!")
report("error of the series for e^(2i) after 21 terms", f"{errors[20]:.1e}")
check(errors[20] < 1e-12, "21 terms of the series give cos 2 + i sin 2 to 1e-12")
```

The first check compares every error with its bound (the extra 10^{-15} allows for the rounding of the floating-point numbers themselves). The RESULT line prints 4.1×10^{-14}, and the last check requires it to be below 10^{-12}. Two PASS lines.

In [6], the picture of the series.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6))
circle = np.exp(1j * np.linspace(0.0, 2.0 * np.pi, 400))  # the unit circle
left.plot(circle.real, circle.imag, color="0.7", lw=1, label="unit circle")
```

Two panels side by side. `np.linspace(0.0, 2.0 * np.pi, 400)` gives 400 angles from 0 to 2π; `np.exp(1j * ...)` turns them into the points $e^{i\theta}$ of the unit circle (numpy computes complex exponentials entry by entry); `.real` and `.imag` of a numpy array are the arrays of the real and imaginary parts. The circle is drawn in light grey.

```
first = np.array(partial_sums[:11])
left.plot(first.real, first.imag, "o-", color="tab:blue",
          label="partial sums $S_0, \\dots, S_{10}$")
```

```
for n in range(5):  # number the first five points
    left.text(first[n].real + 0.06, first[n].imag + 0.06, f"$S_{n}$")
left.plot([target.real], [target.imag], "*", color="tab:red", ms=14,
          label="$e^{2i} = \\cos 2 + i \\sin 2$")
```

The first eleven partial sums as an array, drawn as points joined in order; the first five are labelled S_0 to S_4; the target e^{2i} is a large red star (`"*"`, marker size 14).

```
left.set_aspect("equal")
left.set_xlabel("real part")
left.set_ylabel("imaginary part")
left.set_title("Partial sums of the series of $e^{2i}$")
left.legend(loc="lower left", fontsize=8)
right.semilogy(range(21), errors, "o-", label="error $|S_N - e^{2i}|$")
right.semilogy(range(21), bounds, "--", label="bound $e^2 \\, 2^{N+1}/(N+1)!$")
right.set_xlabel("number $N$ of the last term")
right.set_ylabel("error (logarithmic scale)")
right.set_title("How fast the series converges")
right.legend()
fig.tight_layout()
save_figure(fig, "euler_series",
            "Left: the partial sums $S_N = \\sum_{k=0}^{N} (2i)^k/k!$ for $N = 0$ to "
            ...)
```

Labels for the left panel; on the right the 21 errors and the 21 bounds against N with a logarithmic vertical axis (`semilogy`), the bound dashed (the style string of two hyphens). What Figure 01b.2 shows: on the left the partial sums turn around the origin and close in on the star; on the right the errors fall ever faster and stay below the dashed bound.

```
step_ratios = [errors[n + 1] / errors[n] for n in range(10, 20)]
check(all(abs(ratio - theta / (n + 2)) < 0.1 * theta / (n + 2)
          for ratio, n in zip(step_ratios, range(10, 20))),
      "from N to N + 1 the error is multiplied by about theta/(N + 2) (N = 10 to 19)")
```

`step_ratios` are the ten ratios of the error at $N+1$ to the error at N for $N = 10$ to 19; the check requires each to lie within 10 per cent of $\theta/(N+2)$, the factor derived in Section 1.11. One PASS line.

In [7], 50 digits and the law of exponents.

```
mpmath.mp.dps = 50  # work with 50 significant digits
i_mp = mpmath.mpc(0, 1)  # the imaginary unit as an mpmath number
euler_gap = abs(mpmath.exp(i_mp * 2) - (mpmath.cos(2) + i_mp * mpmath.sin(2)))
pi_gap = abs(mpmath.exp(i_mp * mpmath.pi) + 1)
e_i_pi = mpmath.exp(i_mp * mpmath.pi)
```

`mpmath.mpc(0, 1)` is the complex number $0 + 1i$ in mpmath, computed with 50 digits. `euler_gap` is the size of $e^{2i} - (\cos 2 + i \sin 2)$, `pi_gap` the size of $e^{i\pi} + 1$, and `e_i_pi` the number $e^{i\pi}$ itself.

```
# pi itself is stored with 50 digits, so a tiny imaginary part of about 1e-51 is
# left over: the rounding of pi, not a failure of the formula.
say(f"e^(i pi): real part {mpmath.nstr(e_i_pi.real, 30)}, imaginary part of size "
    f"{mpmath.nstr(abs(e_i_pi.imag), 2)}")
check(euler_gap < mpmath.mpf("1e-45") and pi_gap < mpmath.mpf("1e-45"),
      "mpmath, 50 digits: e^(2i) = cos 2 + i sin 2 and e^(i pi) = -1")
```

The comment explains the leftover imaginary part. The line prints e^(i pi): real part -1.0, imaginary part of size 1.0e-51 (`mpmath.nstr` writes a number with the given count of significant digits and drops needless zeros, so -1 appears as -1.0). The check requires both gaps to be below 10^{-45}.

```
alpha, beta = sp.symbols("alpha beta", real=True)
left_side = sp.expand((sp.cos(alpha) + I * sp.sin(alpha))
                      * (sp.cos(beta) + I * sp.sin(beta)))
right_side = sp.expand(sp.expand_trig(sp.cos(alpha + beta) + I * sp.sin(alpha + beta)))
check(sp.expand(left_side - right_side) == 0,
      "sympy: e^(i alpha) e^(i beta) = e^(i (alpha + beta)) for all real angles")
```

The proof of the law of exponents of Section 1.11 with sympy: `left_side` is $(\cos\alpha + i\sin\alpha)(\cos\beta + i\sin\beta)$ multiplied out; `sp.expand_trig` writes $\cos(\alpha+\beta)$ and $\sin(\alpha+\beta)$ with the addition theorems, and `right_side` is the result multiplied out. Their difference is 0 for all angles. The cell prints two PASS lines.

In [8], multiplication turns the letter F.

```
letter_f = np.array([0, 2j, 1.2 + 2j, 1.2 + 1.6j, 0.4 + 1.6j, 0.4 + 1.1j, 1.0 + 1.1j,
                     1.0 + 0.7j, 0.4 + 0.7j, 0.4, 0]) + (0.3 + 0.2j)  # corners
```

The eleven corners of the outline of a letter F as complex numbers (the last one repeats the first, to close the outline); adding $0.3 + 0.2i$ to the array shifts every corner a little away from the origin, so that no corner is the origin itself.

```
factors = [(1, "original", "black"),
           (np.exp(1j * np.pi / 3), "times $e^{i\\pi/3}$", "tab:blue"),
           (np.exp(5j * np.pi / 6), "times $e^{5i\\pi/6}$", "tab:red"),
           (0.6 * np.exp(-1j * np.pi / 2), "times $0.6\\,e^{-i\\pi/2}$", "tab:green")]
```

Four factors with their labels and colours: 1 (the original), $e^{i\pi/3}$ (a turn by 60 degrees), $e^{5i\pi/6}$ (150 degrees) and $0.6\,e^{-i\pi/2}$ (a quarter turn clockwise and a shrinking to 0.6).

```
fig, ax = plt.subplots(figsize=(8.4, 6.0))
for factor, label, color in factors:
    image = factor * letter_f  # every corner multiplied by the factor
    ax.fill(image.real, image.imag, color=color, alpha=0.35, label=label)
    ax.plot(image.real, image.imag, color=color, lw=1)
ax.plot([0], [0], "k+", ms=12)  # the origin, the centre of the rotations
```

For each factor every corner is multiplied (numpy multiplies an array entry by entry), and `ax.fill` paints the inside of the outline, 35 per cent opaque (`alpha=0.35`), while `ax.plot` draws its edge. A black cross marks the origin.

```
ax.set_aspect("equal")
ax.set_xlabel("real part")
ax.set_ylabel("imaginary part")
ax.set_title("Multiplication by a complex number")
ax.legend(loc="center left", bbox_to_anchor=(1.02, 0.5), fontsize=9)
save_figure(fig, "rotation_by_multiplication",
            "The letter F (black) and its images after multiplying every corner by "
            ...)
```

The legend is placed outside the axes, to the right (`bbox_to_anchor=(1.02, 0.5)` names a point just right of the axes at half their height). What Figure 01b.3 shows: four copies of the letter, turned about the origin and once shrunk, never mirrored.

```
rotations_ok = True
for alpha_value in (np.pi / 3, 5 * np.pi / 6):
    image = np.exp(1j * alpha_value) * letter_f
    rotations_ok &= bool(np.max(np.abs(np.abs(image) - np.abs(letter_f))) < 1e-14)
    turned = np.angle(image / letter_f)  # the change of every angle
    rotations_ok &= bool(np.max(np.abs(turned - alpha_value)) < 1e-12)
check(rotations_ok, "multiplying by e^(i alpha) keeps lengths and adds alpha to "
      "every angle")
```

For the two pure rotations: `np.abs` of a complex array is the array of the moduli, so the first condition requires every corner to keep its distance from the origin; `np.angle(image / letter_f)` is the argument of the quotient of each image corner and its original, which is the change of its angle, and the second condition requires it to be α. One PASS line.

In [9], the same turns as real 2 × 2 matrices.

```
def M(re_part, im_part):
    """The real 2 x 2 matrix of the complex number re_part + i im_part."""
    return sp.Matrix([[re_part, -im_part], [im_part, re_part]])


J = M(0, 1)  # the matrix of i
check(J * J == -sp.eye(2), "J J = -I: the real matrix J behaves like i")
```

`sp.Matrix([[...], [...]])` makes a sympy matrix from the list of its rows; `M` builds the matrix with the rows $(a, -b)$ and (b, a) of Section 1.12. `J` is the matrix of i; `*` between sympy matrices is the matrix product, and `sp.eye(2)` is the 2×2 identity. The check is $J^2 = -I$.

```
zw_re, zw_im = a * c - b * d, a * d + b * c  # z w = (ac - bd) + i (ad + bc)
check((M(a, b) * M(c, d) - M(zw_re, zw_im)).expand() == sp.zeros(2, 2),
      "M(z) M(w) = M(z w) for all complex z and w")
check(sp.expand(M(a, b).det()) == a ** 2 + b ** 2, "det M(a + i b) = a^2 + b^2")
```

With the real symbols a, b, c, d of In [3]: the real and imaginary parts of zw; then the product rule $M(z)M(w) = M(zw)$ (`sp.zeros(2, 2)` is the 2×2 matrix of zeros) and the determinant $a^2 + b^2$ (`.det()` is sympy's determinant), both for all values of the symbols.

```
def R(angle):
    """The rotation matrix of the angle: the matrix of e^(i angle)."""
    return M(sp.cos(angle), sp.sin(angle))


product_rule = (R(alpha) * R(beta) - R(alpha + beta)).applyfunc(sp.expand_trig)
check(product_rule.expand() == sp.zeros(2, 2), "R(alpha) R(beta) = R(alpha + beta)")
```

`R` builds the rotation matrix. `.applyfunc(f)` applies the function `f` to every entry of a matrix; here `sp.expand_trig` writes the entries of $R(\alpha+\beta)$ with the addition theorems, and after multiplying out every entry of the difference is 0: two turns make one.

```
check(sp.simplify(R(alpha).det()) == 1 and
      (R(alpha).T * R(alpha)).applyfunc(sp.simplify) == sp.eye(2),
      "det R(alpha) = 1 and R(alpha)^T R(alpha) = I")
```

`.T` is the transpose. The check is properties (b) and (c) of Section 1.12.

```
rotation = np.array(R(sp.pi / 3).evalf(), dtype=float)  # R(pi/3) as numbers
corners = np.array([letter_f.real, letter_f.imag])  # 2 rows: x and y of each corner
by_matrix = rotation @ corners
by_number = np.exp(1j * np.pi / 3) * letter_f
check(np.max(np.abs(by_matrix[0] + 1j * by_matrix[1] - by_number)) < 1e-14,
      "the rotation matrix R(pi/3) moves the corners exactly like e^(i pi/3)")
```

`.evalf()` turns the exact matrix $R(\pi/3)$ into decimal numbers, and `np.array(..., dtype=floa t)` into a numpy array of floating-point numbers. `corners` is a table with two rows, the x and the y coordinates of the eleven corners; `rotation @ corners` multiplies the matrix with every column at once (`@` is numpy's matrix product; Section 1.18 treats it in full). `by_matrix[0] + 1j * by_matrix[1]` turns the two rows back into complex numbers, which must agree with the corners multiplied by $e^{i\pi/3}$ to 10^{-14}. The cell prints six PASS lines.

In [10], the roots of unity.

```
for n in (3, 8):
    roots = np.exp(2j * np.pi * np.arange(n) / n)  # e^(2 pi i k/n), k = 0 ... n-1
    check(np.max(np.abs(roots ** n - 1)) < 1e-12 and abs(roots.sum()) < 1e-12,
          f"the {n} roots of unity of order {n}: z^{n} = 1 and their sum is 0")
```

For $n = 3$ and $n = 8$: `np.arange(n)` is $0, \ldots, n-1$, so `roots` holds the n roots of unity; the check requires every n-th power to be 1 and the sum to be 0, both to 10^{-12}. Two PASS lines, whose texts contain the value of `n` (an f-string).

```
eighth = [sp.expand_complex(sp.exp(2 * sp.pi * I * k / 8)) for k in range(8)]
say("the eighth roots of unity, exactly: " + ", ".join(str(r) for r in eighth))
check(all(sp.simplify(r ** 8 - 1) == 0 for r in eighth) and
      sp.simplify(sum(eighth)) == 0,
      "sympy: each exact eighth root has eighth power 1, and they add up to 0")
```

The eight roots exactly: `sp.expand_complex` writes $e^{2\pi ik/8}$ as real part plus i times imaginary part. They are printed (`", ".join(...)` joins the texts with a comma and a blank): `1, sqrt(2)/2 + sqrt(2)*I/2, I`, and so on, the list of Section 1.12. The check proves the two properties exactly.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.8))
for ax, n in zip(axes, (3, 8)):
    roots = np.exp(2j * np.pi * np.arange(n + 1) / n)  # the first root again at the end
    ax.plot(circle.real, circle.imag, color="0.75", lw=1)
    ax.plot(roots.real, roots.imag, "o-", color="tab:blue")
    for k in range(n):
        arrow(ax, roots[k], "tab:blue", f"$z_{k}$")
```

Two panels, one for $n = 3$ and one for $n = 8$. With `np.arange(n + 1)` the list ends with the first root again ($k = n$ gives $e^{2\pi i} = 1$), so that the line through the points closes the polygon. The unit circle of In [6] is drawn in grey, the polygon in blue, and each root as a labelled arrow from the origin, with the helper of In [4].

```
    ax.set_aspect("equal")
    ax.set_xlim(-1.45, 1.45)
    ax.set_ylim(-1.35, 1.35)
    ax.set_xlabel("real part")
    ax.set_ylabel("imaginary part")
    ax.set_title(f"the {n} roots of $z^{n} = 1$")
fig.tight_layout()
save_figure(fig, "roots_of_unity",
            "The roots of unity $z_k = e^{2\\pi i k/n}$ for $n = 3$ (left) and "
            ...)
```

Equal scales, ranges, labels and a title on each panel; then the figure is saved. What Figure 01b.4 shows: a regular triangle and a regular octagon inscribed in the unit circle; the arrows of each picture add up to zero, as proved in Section 1.12. The cell prints three PASS lines, the exact roots and the figure line.

In [11], conjugation and real numbers.

```
v_real = np.array([0.5, -2.0, 3.0])  # a list of real numbers
v_complex = np.array([1 + 2j, -1j, 4.0])  # a list with imaginary parts
check((v_real.conj() == v_real).all(),
      "conjugation leaves a list of real numbers unchanged")
check(not (v_complex.conj() == v_complex).all(),
      "conjugation changes a list whose imaginary parts are not all 0")
check((v_complex.conj().conj() == v_complex).all(),
      "conjugating twice gives back the original list")
check(abs(np.exp(0.7j).conjugate() - np.exp(-0.7j)) < 1e-15,
      "the conjugate of e^(i alpha) is e^(-i alpha)")
```

`.conj()` conjugates every entry of a numpy array; `==` between two arrays compares entry by entry, and `.all()` is true when every comparison is true. The four checks are the four statements of Section 1.12: a real list is unchanged; a list with imaginary parts changes (`not` turns true into false and back); conjugating twice restores the list; and $(e^{0.7i})^* = e^{-0.7i}$. Four PASS lines.

In [12], boosts, exactly.

```
phi, phi1, phi2 = sp.symbols("phi phi1 phi2", real=True)


def boost(rapidity):
    """The boost matrix acting on the pair (t, x)."""
    return sp.Matrix([[sp.cosh(rapidity), sp.sinh(rapidity)],
                      [sp.sinh(rapidity), sp.cosh(rapidity)]])
```

Three real symbols for rapidities, and a function that builds the boost $\Lambda(\varphi)$ of Section 1.13.

```
eta_tx = sp.diag(-1, 1)  # t time-like (-1), x space-like (+1)
L = boost(phi)
check((L.T * eta_tx * L - eta_tx).applyfunc(sp.simplify) == sp.zeros(2, 2) and
      sp.simplify(L.det()) == 1,
      "a boost keeps -t^2 + x^2 (Lambda^T eta Lambda = eta) and has det 1")
```

`sp.diag(-1, 1)` is the diagonal matrix $\eta = \mathrm{diag}(-1, +1)$. The check computes $\Lambda^T \eta \Lambda - \eta$, simplifies every entry (sympy knows $\cosh^2 - \sinh^2 = 1$) and requires the zero matrix, and requires $\det \Lambda = 1$: property (a).

```
added = (boost(phi1) * boost(phi2) - boost(phi1 + phi2)).applyfunc(sp.expand_trig)
check(added.expand() == sp.zeros(2, 2), "rapidities add: boost(phi1) boost(phi2) = "
      "boost(phi1 + phi2)")
```

Property (b): `expand_trig` also knows the addition theorems of cosh and sinh.

```
light_plus = (L * sp.Matrix([1, 1]) - sp.exp(phi) * sp.Matrix([1, 1]))
light_minus = (L * sp.Matrix([1, -1]) - sp.exp(-phi) * sp.Matrix([1, -1]))
# rewrite(sp.exp) writes cosh and sinh with exponentials, then simplify:
check(light_plus.applyfunc(lambda e: sp.simplify(e.rewrite(sp.exp)))
      == sp.zeros(2, 1) and
      light_minus.applyfunc(lambda e: sp.simplify(e.rewrite(sp.exp)))
      == sp.zeros(2, 1),
      "the light-like directions are stretched by e^phi and by e^(-phi)")
```

Property (c): `sp.Matrix([1, 1])` is the column $(1, 1)$. `light_plus` is $\Lambda(1,1) - e^{\varphi}(1,1)$ and `light_minus` is $\Lambda(1,-1) - e^{-\varphi}(1,-1)$. A `lambda e: ...` is a function without a name, written in one line: it takes an entry `e`, rewrites cosh and sinh with exponentials (their definitions) and simplifies. Both differences must be zero columns (`sp.zeros(2, 1)`). The cell prints three PASS lines.

In [13], circles and hyperbolas.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 5.0))
angles = np.linspace(0.0, 2.0 * np.pi, 400)
left.plot(np.cos(angles), np.sin(angles), color="tab:blue",
```

```
          label="rotations of $(1, 0)$")
marks = np.array([-1.0, -0.5, 0.0, 0.5, 1.0])
left.plot(np.cos(marks), np.sin(marks), "o", color="tab:blue")
for value in marks:
    left.text(1.06 * np.cos(value), 1.06 * np.sin(value), f"{value:+.1f}", fontsize=8)
```

The point $(1, 0)$ turned by an angle α is $R(\alpha)(1, 0) = (\cos\alpha, \sin\alpha)$; for 400 angles from 0 to 2π these points make the circle. Five angles $-1, -0.5, 0, 0.5, 1$ are marked with dots and labelled just outside the circle (the factor 1.06 moves the label outwards).

```
left.set_aspect("equal")
left.set_xlim(-1.6, 1.6)
left.set_ylim(-1.6, 1.6)
left.set_xlabel("$x$")
left.set_ylabel("$y$")
left.set_title("rotation: $x^2 + y^2$ stays 1")
left.legend(loc="lower left", fontsize=8)
```

Equal scales, ranges, labels, title and legend of the left panel.

```
rapidities = np.linspace(-2.0, 2.0, 400)
right.plot(np.sinh(rapidities), np.cosh(rapidities), color="tab:red",
           label="boosts of $(t, x) = (1, 0)$")
right.plot(np.sinh(marks), np.cosh(marks), "o", color="tab:red")
```

The event $(t, x) = (1, 0)$ boosted with rapidity φ is $\Lambda(\varphi)(1, 0) = (\cosh\varphi, \sinh\varphi)$; it is drawn with $x = \sinh\varphi$ on the horizontal and $t = \cosh\varphi$ on the vertical axis, for 400 rapidities from -2 to 2, with dots at the five marked rapidities.

```
for value in marks:  # labels left of the points on the left half, right otherwise
    shift = 0.12 if value >= 0 else -0.62
    right.text(np.sinh(value) + shift, np.cosh(value) + 0.12, f"{value:+.1f}",
               fontsize=8)
```

The labels of the dots are put to the right of the points on the right half and to the left on the left half, so that they do not cover the curve (`a if condition else b` takes `a` when the condition holds and `b` otherwise).

```
edge = np.linspace(-3.8, 3.8, 2)
right.plot(edge, edge, "k--", lw=1, label="light-like lines $t = \\pm x$")
right.plot(edge, -edge, "k--", lw=1)
```

`edge` holds the two numbers -3.8 and 3.8; the two dashed black lines through $(-3.8, -3.8)$, $(3.8, 3.8)$ and through $(-3.8, 3.8)$, $(3.8, -3.8)$ are the light-like lines $t = x$ and $t = -x$.

```
right.set_aspect("equal")
right.set_xlim(-3.8, 3.8)
```

```
right.set_ylim(-0.5, 3.9)
right.set_xlabel("space-like coordinate $x$")
right.set_ylabel("time-like coordinate $t$")
right.set_title("boost: $t^2 - x^2$ stays 1")
right.legend(loc="lower right", fontsize=8)
fig.tight_layout()
save_figure(fig, "rotations_and_boosts",
            "Left: the point $(1, 0)$ rotated by all angles from 0 to $2\\pi$ "
            ...)
```

Scales, ranges, labels and legend of the right panel, then the figure. What Figure 01b.5 shows: the rotated point runs around a circle; the boosted event runs along the upper branch of the hyperbola $t^2 - x^2 = 1$, which comes ever closer to the dashed light-like lines but never crosses them.

```
on_hyperbola = np.cosh(rapidities) ** 2 - np.sinh(rapidities) ** 2
check(np.max(np.abs(on_hyperbola - 1)) < 1e-12,
      "numbers: cosh^2 - sinh^2 = 1 at 400 rapidities from -2 to 2")
```

The check computes $\cosh^2\varphi - \sinh^2\varphi$ at the 400 rapidities and requires 1 to 10^{-12}. One PASS line.

In [14], oscillation and growth.

```
t = np.linspace(0.0, 10.0, 501)
wave = np.exp(-1j * 2.0 * t)  # real frequency 2
growing = np.exp(-1j * (0.3j) * t)  # imaginary frequency 0.3 i
decaying = np.exp(-1j * (-0.3j) * t)  # imaginary frequency -0.3 i
mixed = np.exp(-1j * (2.0 + 0.3j) * t)  # complex frequency 2 + 0.3 i
```

501 times from 0 to 10, and the wave $e^{-i\varepsilon t}$ for the four frequencies of Section 1.13: 2, $0.3i$, $-0.3i$ and $2 + 0.3i$.

```
check(np.max(np.abs(np.abs(wave) - 1.0)) < 1e-14,
      "real frequency: |e^(-i eps t)| = 1 at every time")
check(np.max(np.abs(growing - np.exp(0.3 * t))) < 1e-12 * np.exp(3.0),
      "imaginary frequency 0.3 i: e^(-i eps t) = e^(0.3 t), real and growing")
check(np.max(np.abs(np.abs(mixed) - np.exp(0.3 * t))) < 1e-12 * np.exp(3.0),
      "complex frequency 2 + 0.3 i: the size of the wave is e^(0.3 t)")
```

The three statements of Section 1.13: modulus 1 for the real frequency; $e^{0.3t}$ for the imaginary one; size $e^{0.3t}$ for the complex one. The tolerance $10^{-12}e^3$ is relative to the largest value $e^{0.3\cdot 10} = e^3 \approx 20$. Three PASS lines.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4))
left.plot(t, wave.real, label="real part $\\cos 2t$")
left.plot(t, wave.imag, "--", label="imaginary part $-\\sin 2t$")
left.plot(t, np.abs(wave), color="black", lw=2, label="modulus $= 1$")
left.set_xlabel("time $t$")
left.set_ylabel("value")
left.set_title("real frequency $\\varepsilon = 2$")
left.set_ylim(-1.75, 1.3)  # room below the curves for the legend
left.legend(loc="lower center", ncol=3, fontsize=8)
```

The left panel: the real part, the imaginary part (dashed) and the modulus (thick black) of the wave with the real frequency 2. The vertical range leaves room below the curves for a legend in three columns (`ncol=3`).

```
right.plot(t, growing.real, color="tab:red", lw=2,
           label="$\\varepsilon = 0.3i$: $e^{0.3t}$")
right.plot(t, decaying.real, color="tab:blue", lw=2,
           label="$\\varepsilon = -0.3i$: $e^{-0.3t}$")
right.plot(t, mixed.real, color="tab:green",
           label="$\\varepsilon = 2 + 0.3i$: real part")
right.plot(t, -np.exp(0.3 * t), ":", color="tab:red")  # the lower envelope
right.set_xlabel("time $t$")
right.set_ylabel("value")
right.set_title("imaginary and complex frequencies")
right.legend(loc="upper left", fontsize=8)
fig.tight_layout()
save_figure(fig, "oscillation_and_growth",
            "Left: the wave $e^{-i\\varepsilon t}$ with the real frequency "
            ...)
```

The right panel: the growing and the decaying wave (both real), the real part of the wave with the complex frequency, and the dotted red curve $-e^{0.3t}$, which with the solid red curve $e^{0.3t}$ encloses the growing oscillation. What Figure 01b.6 shows: on the left a steady oscillation of constant size; on the right one curve growing, one decaying, and an oscillation whose swings grow between $\pm e^{0.3t}$.

In [15], the last check.

```
figure_names = ["01b_1_complex_plane.png", "01b_2_euler_series.png",
                "01b_3_rotation_by_multiplication.png", "01b_4_roots_of_unity.png",
                "01b_5_rotations_and_boosts.png", "01b_6_oscillation_and_growth.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

As in Notebook 01f: the six figure files must exist, and the last line is printed, ALL 34 CHECKS PASSED (notebook 01b). The 34 checks are: 3 in In [2], 3 in In [3], 1 in In [4], 2 in In [5], 1 in In [6], 2 in In [7], 1 in In [8], 6 in In [9], 3 in In [10], 4 in In [11], 3 in In [12], 1 in In [13], 3 in In [14] and 1 in In [15].

1.18 Vectors and matrices; the matrix product

Vectors. A **vector** is an ordered list of numbers, such as $v = (1, 2, 0)$; the numbers are its **components**, and v has three of them. The fields of the course have sixteen components at every point, so at each point a field is a vector with sixteen components. Vectors with the same number of components are added component by component, and a vector is multiplied by a number c by multiplying every component:

$$(v + w)_i = v_i + w_i, \qquad (cv)_i = c\, v_i.$$

The **dot product** of two vectors with n components is the number

$$v \cdot w = \sum_{i=1}^{n} v_i w_i = v_1 w_1 + v_2 w_2 + \cdots + v_n w_n.$$

The symbol $\sum$ (the capital Greek letter sigma) abbreviates a sum: $\sum_{i=1}^{n} t_i$ means $t_1 + t_2 + \cdots + t_n$, and the letter i under it, the **index** of the sum, runs through the whole numbers from 1 to n. For $v = (1, 2, 0)$ and $w = (0, 1, 1)$: $v + w = (1, 3, 1)$, $3v = (3, 6, 0)$ and $v \cdot w = 1 \cdot 0 + 2 \cdot 1 + 0 \cdot 1 = 2$.

Matrices. A **matrix** with m rows and n columns, an $m \times n$ matrix, is a rectangular table of numbers A_{ij}: the first index $i = 1, \ldots, m$ numbers the row (counted downwards), the second $j = 1, \ldots, n$ the column (counted to the right). It is **square** if $m = n$. Python counts from 0, so the entry A_{23} in row 2 and column 3 is `A[1, 2]` in Python. A vector with n components may be written as a column, an $n \times 1$ matrix.

Matrix times vector. An $m \times n$ matrix A turns a vector v with n components into a vector Av with m components:

$$(Av)_i = \sum_{j=1}^{n} A_{ij} v_j,$$

component i of Av is row i of A times v, entry by entry, added up (the rule of Section 1.12 for 2×2 matrices). For the 3×3 matrix A with the rows $(1, 2, 0)$, $(0, 1, 1)$, $(1, 3, 1)$ and $v = (1, 2, 0)$:

$$Av = \begin{pmatrix} 1 \cdot 1 + 2 \cdot 2 + 0 \cdot 0 \\ 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 0 \\ 1 \cdot 1 + 3 \cdot 2 + 1 \cdot 0 \end{pmatrix} = \begin{pmatrix} 5 \\ 2 \\ 7 \end{pmatrix}$$

(each row times the column v). The map $v \mapsto Av$ is **linear**: $A(v + w) = Av + Aw$ and $A(cv) = c\,Av$, because $\sum_j A_{ij}(v_j + w_j) = \sum_j A_{ij} v_j + \sum_j A_{ij} w_j$ and $\sum_j A_{ij}(c v_j) = c \sum_j A_{ij} v_j$ (multiply out inside the sum; a common factor comes out of a sum).

The matrix product (PROVED). The product of an $m \times n$ matrix A and an $n \times p$ matrix B is the $m \times p$ matrix

$$(AB)_{ik} = \sum_{j=1}^{n} A_{ij} B_{jk},$$

entry (i, k) is row i of A times column k of B. It is made so that $(AB)v = A(Bv)$ for every vector v:

$$\big(A(Bv)\big)_i = \sum_j A_{ij} (Bv)_j = \sum_j A_{ij} \sum_k B_{jk} v_k = \sum_k \Big(\sum_j A_{ij} B_{jk} \Big) v_k = \big((AB)v\big)_i$$

(the rule of matrix times vector, twice; the factor A_{ij} moved into the inner sum and the two sums exchanged, which only re-orders the terms of a finite sum; the definition of AB). With the matrix A above and B with the rows $(0, 1, 0)$, $(1, 0, 2)$, $(0, 1, 1)$:

$$AB = \begin{pmatrix} 2 & 1 & 4 \\ 1 & 1 & 3 \\ 3 & 2 & 7 \end{pmatrix}, \qquad BA = \begin{pmatrix} 0 & 1 & 1 \\ 3 & 8 & 2 \\ 1 & 4 & 2 \end{pmatrix}$$

(for example the entry in row 1, column 3 of AB is $1 \cdot 0 + 2 \cdot 2 + 0 \cdot 1 = 4$, and that of BA is $0 \cdot 0 + 1 \cdot 1 + 0 \cdot 1 = 1$). The two products differ: **matrix multiplication is not commutative**, $AB \neq BA$ in general. The order of the factors matters in every matrix formula of the course. The product is, however, **associative**, $(AB)C = A(BC)$, because both sides have the entries $\sum_j \sum_k A_{ij}B_{jk}C_{kl}$, and **distributive**, $A(B + C) = AB + AC$ (multiply out inside the sum).

The summation convention, a first look. In $(AB)_{ik} = \sum_j A_{ij}B_{jk}$ the index j appears twice in the product and is summed; i and k appear once and are not summed. The course writes such sums without the sign $\sum$: an index that appears twice in one product is summed over all its values. So $A_{ij}B_{jk}$ means $\sum_j A_{ij}B_{jk}$. A summed index is called a **dummy** index (its name does not matter: $A_{ij}B_{jk} = A_{il}B_{lk}$), an index that is not summed a **free** index (the formula holds for each of its values). numpy's function `einsum` reads this convention literally: `np.einsum("ij,jk->ik", A, B)` says "the first factor has the indices i, j, the second j, k; j appears twice and is summed; the result has the indices i, k". Section 1.26 develops the convention fully, with upper and lower indices.

Identity, transpose and trace (PROVED). The **identity matrix** I is the square matrix with 1 on the diagonal and 0 elsewhere; its entries are the **Kronecker delta**, $\delta_{ij} = 1$ if $i = j$ and 0 otherwise, and $IA = AI = A$. The **transpose** A^T has the rows and columns exchanged, $(A^T)_{ij} = A_{ji}$. The transpose of a product is the product of the transposes in the opposite order:

$$\left((AB)^T\right)_{ki} = (AB)_{ik} = \sum_j A_{ij}B_{jk} = \sum_j (B^T)_{kj}(A^T)_{ji} = (B^T A^T)_{ki}$$

(the definition of the transpose; the matrix product; each factor written with its transpose, re-ordered, since numbers commute; the matrix product of B^T and A^T). The **trace** of a square matrix is the sum of its diagonal entries, $\operatorname{tr} A = \sum_i A_{ii}$. Although $AB \neq BA$, the two products have the same trace:

$$\operatorname{tr}(AB) = \sum_i \sum_j A_{ij}B_{ji} = \sum_j \sum_i B_{ji}A_{ij} = \operatorname{tr}(BA)$$

(the diagonal entries of AB, added; the order of the two sums and of the two factors exchanged; the diagonal entries of BA, added). In the example both traces are 10 ($2 + 1 + 7$ and $0 + 8 + 2$). Notebook 01a, In [2] to In [5], computes all of this three ways (with loops, with `einsum`, with numpy's operator `@`) and draws the four matrices as **heat maps**, pictures in which every entry is a coloured square (Figure 01a.1).

1.19 Permutations and their signs

Permutations. A **permutation** of n objects is a re-ordering of them. We take the objects to be the numbers $0, 1, \ldots, n-1$ (as Python does; the book's formulas use $1, \ldots, n$, which changes nothing): a permutation σ is the list $(\sigma(0), \sigma(1), \ldots, \sigma(n-1))$ that contains each number exactly once. There are $n! = 1 \cdot 2 \cdots n$ permutations: the first place can hold any of the n numbers, the

second any of the $n-1$ that are left, and so on, down to 1 choice for the last place, and the numbers of choices multiply.

Inversions and the sign. An **inversion** of a permutation is a pair of places $i < j$ whose entries stand in the wrong order, $\sigma(i) > \sigma(j)$. The **sign** of the permutation is

$$\operatorname{sign}(\sigma) = (-1)^{\text{number of inversions}},$$

$+1$ for an **even** and -1 for an **odd** permutation. The six permutations of $0, 1, 2$:

permutation	**inversions (pairs of entries in the wrong order)**	**number**	**sign**
(0, 1, 2)	none	0	$+1$
(0, 2, 1)	(2, 1)	1	-1
(1, 0, 2)	(1, 0)	1	-1
(1, 2, 0)	(1, 0), (2, 0)	2	$+1$
(2, 0, 1)	(2, 0), (2, 1)	2	$+1$
(2, 1, 0)	(2, 1), (2, 0), (1, 0)	3	-1

Theorem: an exchange flips the sign (PROVED). Exchanging the entries at two places of a permutation changes its sign. *Proof, step 1: neighbouring places.* Exchange the entries at the places k and $k+1$. A pair of places that contains neither k nor $k+1$ keeps its two entries and its order. For a place p outside the two, the two pairs (p, k) and $(p, k+1)$ compare the entry at p with the same two entries before and after the exchange, only in the other order of the pairs, so together they contain the same number of inversions. The pair $(k, k+1)$ itself was an inversion exactly when it is not one after the exchange. So the number of inversions changes by exactly 1, and the sign flips. *Step 2: any two places $i < j$.* Move the entry at place i to the right by $j-i$ exchanges of neighbours, until it stands at place j; the entry that stood at place j is now at place $j-1$. Move that entry to the left by $j-i-1$ exchanges of neighbours, until it stands at place i. Every other entry is back at its place, and the two entries have traded places. That took $(j-i)+(j-i-1) = 2(j-i)-1$ exchanges of neighbours, an odd number; by step 1 each flips the sign, and an odd number of flips is a flip.

Half of the permutations are even (PROVED). For $n \geq 2$, exchanging the first two entries turns every even permutation into an odd one and every odd one into an even one (the theorem), and doing it twice gives back the original permutation. So it pairs the even permutations with the odd ones one to one, and there are $n!/2$ of each. Notebook 01a, In [6] and In [7], lists the six permutations of three objects with their inversions and checks the three statements (the count $n!$, half of them even, every exchange flips the sign) for every permutation of $n = 1$ to 6 objects.

The inverse permutation has the same sign (PROVED). The **inverse** σ^{-1} undoes σ: $\sigma^{-1}(b) = i$ when $\sigma(i) = b$. If the places $i < j$ form an inversion of σ, put $a = \sigma(j)$ and $b = \sigma(i)$; then $a < b$ and $\sigma^{-1}(a) = j > i = \sigma^{-1}(b)$, so (a, b) is an inversion of σ^{-1}; and every inversion of σ^{-1} comes back to one of σ in this way. So both have the same number of inversions and the same sign.

Permutation matrices. The **permutation matrix** P_σ has a 1 in row i and column $\sigma(i)$ for every i, and 0 elsewhere: exactly one 1 in every row and in every column. A **signed permutation**

matrix may have -1 instead of 1 in some of these places. Figure 01a.2 shows the 24 permutation matrices of 4 objects.

1.20 Determinants: the Leibniz formula and its six rules

Definition. The **determinant** of a square $n \times n$ matrix A is the number

$$\det A = \sum_{\sigma} \operatorname{sign}(\sigma)\, A_{1\sigma(1)} A_{2\sigma(2)} \cdots A_{n\sigma(n)},$$

the sum over all $n!$ permutations σ of $1, \ldots, n$ (the **Leibniz formula**). Each term takes exactly one entry from every row i, namely the one in column $\sigma(i)$, so it also takes exactly one entry from every column. For $n = 2$ the permutations $(1, 2)$ (sign $+1$) and $(2, 1)$ (sign -1) give

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc,$$

the formula of Section 1.12. For $n = 3$ the six permutations of the table of Section 1.19 (counted from 1 instead of 0) give

$$\det A = A_{11}A_{22}A_{33} + A_{12}A_{23}A_{31} + A_{13}A_{21}A_{32} - A_{12}A_{21}A_{33} - A_{11}A_{23}A_{32} - A_{13}A_{22}A_{31}.$$

For the matrix A of Section 1.18 the six terms are $1 + 2 + 0 - 0 - 3 - 0 = 0$: the determinant vanishes, and indeed the third row $(1, 3, 1)$ is the sum of the first two.

Three special matrices (PROVED). (a) A **diagonal** matrix, whose entries off the diagonal are 0: every permutation other than the identity takes some entry off the diagonal, so only the term of the identity survives, and $\det \operatorname{diag}(d_1, \ldots, d_n) = d_1 d_2 \cdots d_n$. (b) A permutation matrix P_σ: a term of a permutation $\tau \neq \sigma$ contains an entry $(i, \tau(i))$ with $\tau(i) \neq \sigma(i)$, which is 0; so only the term of σ survives, and it is $\operatorname{sign}(\sigma) \cdot 1 \cdots 1$: $\det P_\sigma = \operatorname{sign}(\sigma)$. (c) A signed permutation matrix with the nonzero entries $s_1, \ldots, s_n = \pm 1$ in the places of σ: by the same argument

$$\det = \operatorname{sign}(\sigma)\, s_1 s_2 \cdots s_n.$$

The six rules of determinants. Write M for an $n \times n$ matrix.

Rule 1. $\det(MN) = \det M \det N$. For 2×2 matrices this is a direct computation (PROVED): with M of rows $(a, b), (c, d)$ and N of rows $(e, f), (g, h)$, MN has the rows $(ae + bg, af + bh)$ and $(ce + dg, cf + dh)$, and

$$\det(MN) = (ae + bg)(cf + dh) - (af + bh)(ce + dg)$$

(the 2×2 formula),

$$= aecf + aedh + bgcf + bgdh - afce - afdg - bhce - bhdg$$

(multiply out), in which $aecf$ cancels $afce$ and $bgdh$ cancels $bhdg$, leaving $adeh+bcfg-adfg-bceh = ad(eh-fg)-bc(eh-fg) = (ad-bc)(eh-fg)$ (re-order the factors; take out the common factors). For every n the rule is a standard theorem of algebra, ASSUMED here; Notebook 01a, In [11], checks it exactly for two 4×4 matrices.

Rule 2. $\det(M^T) = \det M$ (PROVED). The term of σ in $\det M^T$ is $\operatorname{sign}(\sigma)\prod_i (M^T)_{i\sigma(i)} = \operatorname{sign}(\sigma)\prod_i M_{\sigma(i)i}$. Re-ordering the factors by their row $j = \sigma(i)$ writes the product as $\prod_j M_{j\sigma^{-1}(j)}$, the term of σ^{-1} in $\det M$, and $\operatorname{sign}(\sigma^{-1}) = \operatorname{sign}(\sigma)$ (Section 1.19). As σ runs through all permutations, so does σ^{-1}, so the two sums have the same terms.

Rule 3. Exchanging two rows changes the sign (PROVED). Let M' be M with the rows r and s exchanged. The term of σ in $\det M'$ has the factors $M'_{r\sigma(r)} = M_{s\sigma(r)}$ and $M'_{s\sigma(s)} = M_{r\sigma(s)}$, so its product of entries is that of the permutation τ that equals σ with the entries at the places r and s exchanged; and $\operatorname{sign}(\tau) = -\operatorname{sign}(\sigma)$ (the theorem of Section 1.19). As σ runs through all permutations so does τ, so every term of $\det M$ appears in $\det M'$ with the opposite sign: $\det M' = -\det M$.

Rule 4. A matrix with two equal rows has determinant 0 (PROVED): exchanging the two equal rows changes nothing, yet by rule 3 it changes the sign, so $D = -D$, and $D = 0$.

Rule 5. Multiplying one row by a number c multiplies the determinant by c (PROVED): every term contains exactly one entry of that row, so every term is multiplied by c.

Rule 6. Adding c times row s to row r (with $s \neq r$) does not change the determinant (PROVED). Every term contains exactly one entry of row r, which is now $M_{r\sigma(r)} + c\,M_{s\sigma(r)}$; multiplying out, the determinant splits into $\det M$ plus c times the determinant of M with row r replaced by row s; that matrix has two equal rows, so the second part is 0 by rule 4.

Determinants and inverses. A square matrix M is **invertible** if there is a matrix M^{-1} with $MM^{-1} = M^{-1}M = I$; then the equation $Mv = w$ can be undone, $v = M^{-1}w$. For a 2×2 matrix with $ad - bc \neq 0$ the inverse is

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix},$$

as multiplying out shows: the product of the two matrices has the entries $ad - bc$, $-ab + ba = 0$, $cd - dc = 0$ and $-cb + da$, so it is $(ad - bc)I$, and dividing by $ad - bc$ gives I. For every n it is a standard theorem that M is invertible exactly when $\det M \neq 0$ (ASSUMED here; one half follows from rule 1: if M^{-1} exists, $\det M \det M^{-1} = \det I = 1$, so $\det M \neq 0$). The determinant is the one number that says whether a matrix can be undone.

1.21 What a determinant measures, what it costs, and two matrices of the course

A signed area (PROVED). A 2×2 matrix M with rows (a, b) and (c, d) moves the corners $(0,0)$, $(1,0)$, $(1,1)$, $(0,1)$ of the unit square to the corners $(0,0)$, (a,c), $(a+b, c+d)$, (b,d) of a parallelogram: the images $Me_1 = (a,c)$ and $Me_2 = (b,d)$ of $e_1 = (1,0)$ and $e_2 = (0,1)$ are the two columns of M. The **signed area** of a polygon with the corners $(x_1, y_1), \ldots, (x_K, y_K)$, listed in order, is given by the **shoelace formula**

$$\tfrac{1}{2}\sum_{k=1}^{K}\left(x_k y_{k+1} - x_{k+1} y_k\right) \qquad (\text{with } x_{K+1} = x_1,\ y_{K+1} = y_1),$$

which equals the area when the corners are listed counterclockwise and minus the area when they are listed clockwise (a standard fact of plane geometry, ASSUMED here). For the parallelogram the four terms of the sum are

$$0 \cdot c - a \cdot 0 = 0, \quad a(c+d) - (a+b)c = ad - bc, \quad (a+b)d - b(c+d) = ad - bc, \quad b \cdot 0 - 0 \cdot d = 0$$

(insert the corners in order; multiply out; ac and bd cancel), so the signed area is $\frac{1}{2} \cdot 2(ad - bc) = \det M$. The size of the determinant is the factor by which M multiplies areas, and its sign says whether the order of the corners is kept ($\det M > 0$) or reversed, that is, whether the picture is mirrored ($\det M < 0$). A determinant 0 flattens the square onto a line, and such a matrix cannot be undone. Notebook 01a, In [12], checks this for three examples and 1000 random matrices (Figure 01a.3).

What a determinant costs. The Leibniz formula has $n!$ terms, each a product of n entries. For the 16×16 gamma matrices of the course that would be

$$16! = 20922789888000$$

terms. Computers therefore use **elimination**: by rule 6 one may subtract multiples of rows from other rows without changing the determinant, and doing so step by step makes every entry below the diagonal zero, which needs about $n^3/3$ multiplications. For a matrix with zeros below the diagonal (a **triangular** matrix) only the term of the identity survives in the Leibniz formula, because every other permutation σ has some row i with $\sigma(i) < i$ (the numbers $\sigma(i) - i$ add up to 0 and are not all 0, so one of them is negative), and the entry $A_{i\sigma(i)}$ below the diagonal is 0. So the determinant of the triangular matrix is the product of its diagonal. For $n = 16$ elimination needs about $16^3/3 \approx 1365$ steps instead of 2.1×10^{13} terms (Figure 01a.4).

The gamma matrices of the Revision record. The file `Revision/algebra/gammas.json` of the Revision record holds the eight real 16×16 gamma matrices $\gamma^{(x_0)}, \ldots, \gamma^{(x_7)}$ of the author's theory, one for each coordinate, rebuilt in the record from the author's own formulas (Chapter 4 builds them step by step), and the diagonal of the frame metric $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \ldots, x_7$. The record states, in its report `Revision/algebra/reports/python-algebra.json`, check `reality_signed_permutations`, that every gamma matrix is a real signed permutation matrix, and in its check `clifford_relation` that $\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab} I$; with equal indices this says $(\gamma^a)^2 = \eta_{aa} I$: $+I$ for the space-like directions x_0, x_1, x_2, x_3 and $-I$ for the time-like directions $x_4, \ldots, x_7$. Two consequences for the determinant (PROVED): by rule 1, $(\det \gamma^a)^2 = \det\left((\gamma^a)^2\right) = \det(\pm I) = (\pm 1)^{16} = 1$, so $\det \gamma^a = \pm 1$; and by the formula (c) for signed permutation matrices it is $\mathrm{sign}(\sigma)$ times the product of the sixteen signs, a computation of 16 numbers and one permutation instead of 16! terms. Notebook 01a, In [14] and In [15], reads the matrices, reproduces the two record checks and finds $\det \gamma^a = +1$ for all eight (COMPUTED): the permutation of each is even, and each has an even number of entries -1 (eight for $\gamma^{(x_1)}$ to $\gamma^{(x_7)}$, none for $\gamma^{(x_0)}$).

The determinant of the author's metric (PROVED). The author's metric (Section 1.1) is diagonal, so by (a) its determinant is the product of its eight diagonal entries:

$$\det g = \cot^2 z \cdot \big(e^{2a_4} s\big)^3 \cdot (-1) \cdot \big(-\, b_4 e^{-2a_4} s\big)^3$$

(the determinant of a diagonal matrix; the three equal space entries give a cube, and so do the three equal extra-time entries),

$$= \cot^2 z \cdot e^{6a_4} s^3 \cdot (-1) \cdot (-1)\, b_4^3 e^{-6a_4} s^3$$

(the power of a product is the product of the powers, $(e^{2a_4})^3 = e^{6a_4}$, $(b_4 e^{-2a_4})^3 = b_4^3 e^{-6a_4}$, and $(-x)^3 = -x^3$),

$$= b_4^3\, e^{6a_4 - 6a_4}\, s^6\, \cot^2 z = b_4^3 \sin^2 z\, \frac{\cos^2 z}{\sin^2 z} = b_4^3 \cos^2 z$$

(the two signs give $(-1)(-1) = +1$ and the powers of e are collected with $a^p a^q = a^{p+q}$; then $e^0 = 1$, $s^6 = (\sin^{1/3} z)^6 = \sin^2 z$ and $\cot z = \cos z / \sin z$; cancel $\sin^2 z$). The growth e^{6a_4} of the three space directions and the shrinking e^{-6a_4} of the three exponentially deflating extra times cancel exactly, for every value of a_4; only the factor b_4^3 of the extra times is left. The sign is + because the metric has four negative entries, an even number. Since $\cos z > 0$ for $0 < z < \pi/2$ and $b_4 > 0$,

$$\sqrt{|\det g|} = b_4^{3/2} \cos z = b_4^{3/2} \sin z \cot z,$$

and this is the product of the eight length factors of Section 1.5 (the square root of a product of positive numbers is the product of their square roots, and $|\det g| = \prod_\mu |g_{\mu\mu}|$); at $b_4 = 1$ it is $\cos z$. The record stores it as `sqrtAbsDetG` $= b_4^{3/2} \sin z \cot z$ in `Revision/gkd_lovelock/results/curvature.json` and checks it in `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `sqrt_abs_det_g`; Notebook 01a, In [17] and In [18], reproduces both, exactly with sympy and in numbers at 900 points (Figure 01a.6).

1.22 Example: matrices, permutations and determinants

Notebook 01a puts Sections 1.18 to 1.21 to work. It computes the dot product, matrix times vector and the matrix product with loops, with `einsum` and with `@`, and shows with heat maps that $AB \neq BA$; lists the permutations of 3 objects with their inversions and signs, checks the exchange theorem for every permutation of up to 6 objects and draws the 24 permutation matrices of 4 objects; writes the Leibniz formula as a Python function and compares it with sympy and numpy; checks the six rules; draws signed areas; plots the cost of the Leibniz formula against elimination; reads the eight gamma matrices from the Revision record and checks that they are signed permutation matrices with determinant 1 whose squares are $\pm I$; and reads the author's metric from the record and proves that its determinant is $b_4^3 \cos^2 z$ for every a_4. It ends with the line ALL 30 CHECKS PASSED (notebook 01a).

1.23 How to run Notebook 01a

Step 1. What this notebook does and what it needs.

Notebook 01a (Matrices, permutations and determinants) is the file `Revision/textbook/notebooks/01a_matrices_determinants.ipynb` of the repository Dirac_claude. It multiplies matrices in three ways (with loops, with the summation convention through numpy einsum, and with the operator @) and shows with heat maps that the order of the factors matters, lists the permutations of up to four objects with their signs, computes determinants with the Leibniz formula and compares them with numpy and sympy, checks the rules of determinants and draws their meaning as a signed area, reads the real 16 by 16 gamma matrices and the frame metric of the author's coordinates x0 to x7 from the Revision record and checks that every gamma matrix is a signed permutation matrix with determinant 1, and proves with sympy that the determinant of the author's metric is the square of cos(6 H x0) for every value of a4. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01a_matrices_determinants.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01a_matrices_determinants.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01a_matrices_determinants.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01a.captions.json`
- `Revision/textbook/figures/01a_1_matrix_product.png`
- `Revision/textbook/figures/01a_2_permutation_matrices.png`
- `Revision/textbook/figures/01a_3_area_and_determinant.png`
- `Revision/textbook/figures/01a_4_determinant_cost.png`
- `Revision/textbook/figures/01a_5_gamma_matrices.png`
- `Revision/textbook/figures/01a_6_metric_and_determinant.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 01a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01a_matrices_determinants.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01a_matrices_determinants.ipynb
python -m nbconvert --execute --inplace 01a_matrices_determinants.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/algebra or Revision/gkd_lovelock: the notebook reads two Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

1.24 Notebook 01a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01a_matrices_determinants.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook builds, from zero, the tools that every later notebook uses to
handle lists of numbers: vectors, matrices, permutations and determinants. It

- multiplies a matrix with a vector and two matrices with each other in three
  ways (with loops, with the summation convention, with numpy's operator `@`)
  and shows with heat maps that $AB$ and $BA$ are in general different;
- lists the permutations (re-orderings) of 3 and of 4 objects, counts their
  inversions and gives each its sign $+1$ or $-1$;
- computes determinants with the Leibniz formula, written by hand in Python,
  and compares the results with the programs numpy and sympy;
- checks six rules of determinants, and draws what a $2 \times 2$ determinant
  means: the signed area of a parallelogram;
- explains with a plot why computers do not use the Leibniz formula for big
  matrices;
- reads the eight real $16 \times 16$ gamma matrices of the author's theory from
  the Revision record and checks that each one is a *signed permutation matrix*
  whose determinant is 1;
- reads the author's metric from the Revision record and proves with sympy that
  its determinant is $b_4^3 \cos^2 z$ with $z = 6 H x_0$, whatever the value of
  $a_4$.

Every check prints a line that starts with PASS; where a check repeats a result
of the Revision record, a second line names the record file and its check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01a (Matrices, permutations and determinants) is the file
`Revision/textbook/notebooks/01a_matrices_determinants.ipynb` of the repository
Dirac_claude. It multiplies matrices in three ways (with loops, with the summation
convention through numpy einsum, and with the operator @) and shows with heat maps that
the order of the factors matters, lists the permutations of up to four objects with their
signs, computes determinants with the Leibniz formula and compares them with numpy and
sympy, checks the rules of determinants and draws their meaning as a signed area, reads
the real 16 by 16 gamma matrices and the frame metric of the author's coordinates x0 to
x7 from the Revision record and checks that every gamma matrix is a signed permutation
```

matrix with determinant 1, and proves with sympy that the determinant of the author's metric is the square of cos(6 H x0) for every value of a4. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01a_matrices_determinants.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01a_matrices_determinants.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01a_matrices_determinants.ipynb‘ (in the same folder, without running the cells again) to

read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01a.captions.json‘
- ‘Revision/textbook/figures/01a_1_matrix_product.png‘
- ‘Revision/textbook/figures/01a_2_permutation_matrices.png‘
- ‘Revision/textbook/figures/01a_3_area_and_determinant.png‘
- ‘Revision/textbook/figures/01a_4_determinant_cost.png‘
- ‘Revision/textbook/figures/01a_5_gamma_matrices.png‘
- ‘Revision/textbook/figures/01a_6_metric_and_determinant.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 01a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01a_matrices_determinants.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt

  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 01a_matrices_determinants.ipynb
      python -m nbconvert --execute --inplace 01a_matrices_determinants.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/algebra or Revision/gkd_lovelock: the notebook reads two Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 01a (Matrices, permutations and determinants) is the file
# Revision/textbook/notebooks/01a_matrices_determinants.ipynb of the repository
# Dirac_claude. It multiplies matrices in three ways (with loops, with the summation
# convention through numpy einsum, and with the operator @) and shows with heat maps that
# the order of the factors matters, lists the permutations of up to four objects with
# their signs, computes determinants with the Leibniz formula and compares them with
# numpy and sympy, checks the rules of determinants and draws their meaning as a signed
# area, reads the real 16 by 16 gamma matrices and the frame metric of the author's
# coordinates x0 to x7 from the Revision record and checks that every gamma matrix is a
# signed permutation matrix with determinant 1, and proves with sympy that the
# determinant of the author's metric is the square of cos(6 H x0) for every value of a4.
# It needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
```

```
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
```

```
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
```

```
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01a_matrices_determinants.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01a_matrices_determinants.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01a_matrices_determinants.ipynb (in the same folder, without running the cells again)
# to read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01a.captions.json
# - Revision/textbook/figures/01a_1_matrix_product.png
# - Revision/textbook/figures/01a_2_permutation_matrices.png
# - Revision/textbook/figures/01a_3_area_and_determinant.png
# - Revision/textbook/figures/01a_4_determinant_cost.png
# - Revision/textbook/figures/01a_5_gamma_matrices.png
# - Revision/textbook/figures/01a_6_metric_and_determinant.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
```

```
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 30 CHECKS PASSED (notebook 01a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01a_matrices_determinants.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01a_matrices_determinants.ipynb
#     python -m nbconvert --execute --inplace 01a_matrices_determinants.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below Revision/algebra or Revision/gkd_lovelock: the
#   notebook reads two Revision records of the repository; your copy of the repository is
```

```
#   incomplete. Download it again with git clone and open the notebook inside the new
#   copy.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "01a"  # this notebook: chapter 01, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
```

```
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Vector**: an ordered list of numbers, such as $v = (1, 2, 0)$. The numbers
  are its **components**; $v$ has 3 components.
- **Matrix**: a rectangular table of numbers with rows (across) and columns
  (down). $A_{ij}$ is the **entry** in row $i$ and column $j$. A matrix with $m$
  rows and $n$ columns is an $m \times n$ matrix; it is **square** if $m = n$.
- **Index**: a letter such as $i$, $j$ or $k$ that numbers the components of a
  vector or the rows and columns of a matrix. In mathematics rows and columns
  are numbered $1, 2, 3, \dots$; Python numbers them $0, 1, 2, \dots$, so the
  entry $A_{11}$ (row 1, column 1) is `A[0, 0]` in Python.
- **Matrix product**: $(AB)_{ik} = \sum_j A_{ij} B_{jk}$: entry $(i, k)$ of $AB$
  is row $i$ of $A$ times column $k$ of $B$, entry by entry, added up.
- **Summation convention**: an index that appears twice in one product is summed
  over all its values without writing $\sum$; so $A_{ij} B_{jk}$ means
  $\sum_j A_{ij} B_{jk}$. A summed index is a **dummy** index (its name does not
  matter); an index that is not summed is a **free** index. In the physics of the
  course one of the two equal indices is written up and one down, as in
  $v^\mu w_\mu = \sum_\mu v^\mu w_\mu$; for the entries of plain tables of
  numbers, as in this notebook, both are written down.
- **einsum**: numpy's function that evaluates a product written with the
  summation convention, for example `np.einsum("ij,jk->ik", A, B)`.
- **Identity matrix** $I$: the square matrix with 1 on the diagonal and 0
  elsewhere. Its entries are the **Kronecker delta** $\delta_{ij}$, which is 1
  when $i = j$ and 0 otherwise.
- **Transpose** $A^T$: the matrix with rows and columns exchanged,
  $(A^T)_{ij} = A_{ji}$.
- **Permutation**: a re-ordering of the numbers $0, 1, \dots, n-1$ (or of any $n$
  different objects). There are $n! = 1 \cdot 2 \cdots n$ of them.
- **Inversion**: a pair of places in a permutation whose two entries stand in the
```

```
  wrong order (the larger one first).
- **Sign** of a permutation: $+1$ if it has an even number of inversions (an
  **even** permutation), $-1$ if odd.
- **Permutation matrix**: the matrix with exactly one 1 in every row and every
  column and 0 elsewhere; row $i$ has its 1 in column $\sigma(i)$ for a
  permutation $\sigma$. A **signed permutation matrix** may have $-1$ instead of
  $1$ in some of these places.
- **Determinant** $\det A$ of a square matrix (Leibniz formula):
  $\det A = \sum_\sigma \mathrm{sign}(\sigma) A_{1\sigma(1)} A_{2\sigma(2)}
  \cdots A_{n\sigma(n)}$, the sum over all $n!$ permutations $\sigma$.
- **Heat map**: a picture of a matrix in which every entry is a coloured square
  (here red for positive, white for zero, blue for negative entries).
- **Floating-point number**: the way a computer stores a real number, with about
  16 significant digits, so results can be off in the last digits. **Exact**
  arithmetic (sympy) keeps whole numbers, fractions and symbols without rounding.
- **Metric** $g$: the table of numbers that turns small coordinate steps
  $dx_0, \dots, dx_7$ into a squared length,
  $ds^2 = g_{00} dx_0^2 + \dots + g_{77} dx_7^2$ for a diagonal metric; here it is
  a diagonal $8 \times 8$ matrix (only the diagonal entries $g_{00}, \dots,
  g_{77}$ are not zero), and a negative entry marks a time-like direction.
- **Revision record**: the files of the folder Revision of the repository, in
  which every result of the author's theory is computed and checked.
```

Text cell:

```
## 4. The physical and mathematical situation

The theory of this course works with matrices everywhere. Its spacetime has
eight coordinates, which the author calls $x_0, \dots, x_7$: $x_1, x_2, x_3$ are
ordinary space, $x_4$ is the time, $x_5, x_6, x_7$ are three extra times whose
lengths deflate exponentially as $a_4$ grows, and $x_0$ is a hidden space
direction. The author's metric is the diagonal matrix

$$g = \mathrm{diag}\big(\cot^2 z, e^{2a_4} s, e^{2a_4} s, e^{2a_4} s, -1,
-b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s\big),\quad
s = \sin^{1/3} z,\ z = 6 H x_0,$$

with $0 < z < \pi/2$, a positive constant $H$, a function $a_4(x_4)$ of the
time and a positive function $b_4(x_4)$ of the time (the author's metric of
2026-10-14; with $b_4 = 1$ it is the earlier metric). The simpler *frame metric*
$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ keeps only the signs: $+1$
for the space-like directions $x_0, x_1, x_2, x_3$ and $-1$ for the time-like
directions $x_4, x_5, x_6, x_7$. The fields of the theory have 16 components, and
the eight *gamma matrices* that act on them are real $16 \times 16$ matrices.

The determinant is the one number that tells whether a square matrix can be
undone (inverted): it can exactly when $\det A \neq 0$. For a diagonal matrix
only one term of the Leibniz formula survives (all others contain an entry off
the diagonal, which is 0), so the determinant is the product of the diagonal
entries. For the author's metric this product is

$$\det g = \cot^2 z \cdot (e^{2a_4} s)^3 \cdot (-1) \cdot (-b_4 e^{-2a_4} s)^3
= b_4^3 e^{6a_4} e^{-6a_4} s^6 \cot^2 z
```

```
= b_4^3 \sin^2 z \, \frac{\cos^2 z}{\sin^2 z} = b_4^3 \cos^2 z,$$

because $(-1) \cdot (-1)^3 = +1$, $e^{6a_4} e^{-6a_4} = e^0 = 1$,
$s^6 = (\sin^{1/3} z)^6 = \sin^2 z$ and $\cot z = \cos z / \sin z$. The growth
$e^{6a_4}$ of the three space directions is cancelled exactly by the shrinking
$e^{-6a_4}$ of the three extra times; only the factor $b_4^3$ of the three extra
times is left. The last part of the notebook checks this with sympy, starting
from the metric as it is stored in the Revision record.
```

Text cell:

```
## 5. Vectors and matrices in Python

The next cell loads the packages, makes two vectors $v = (1, 2, 0)$ and
$w = (0, 1, 1)$ as numpy arrays, adds them, multiplies $v$ by 3, and computes
the **dot product** $v \cdot w = \sum_i v_i w_i = 1 \cdot 0 + 2 \cdot 1 + 0
\cdot 1 = 2$ twice: once as a written-out sum and once with numpy's `@`.
`tolist()` turns an array into a plain Python list for printing.
```

In [2]:

```
import itertools  # all orderings (permutations) of a list
import math  # factorials

import numpy as np  # arrays of numbers: vectors and matrices
import sympy as sp  # exact arithmetic with whole numbers, fractions and symbols

v = np.array([1, 2, 0])  # a vector with 3 components
w = np.array([0, 1, 1])
say(f"v = {v.tolist()}, w = {w.tolist()}")
say(f"v + w = {(v + w).tolist()}, 3 v = {(3 * v).tolist()}")
# The dot product as a sum over the index i = 0, 1, 2 (Python counts from 0):
dot_by_hand = sum(int(v[i]) * int(w[i]) for i in range(3))
say(f"v . w by the sum = {dot_by_hand}, with numpy's @ = {int(v @ w)}")
check(dot_by_hand == int(v @ w) == 2, "the dot product of v and w is 2")
```

Out [2]:

```
v = [1, 2, 0], w = [0, 1, 1]
v + w = [1, 3, 1], 3 v = [3, 6, 0]
v . w by the sum = 2, with numpy's @ = 2
PASS the dot product of v and w is 2
```

Text cell:

```
The next cell makes the $3 \times 3$ matrix $A$ with rows $(1, 2, 0)$,
$(0, 1, 1)$ and $(1, 3, 1)$ and multiplies it with the vector $v$ by the rule
$(Av)_i = \sum_j A_{ij} v_j$, written as a loop, and with numpy's `A @ v`. Row 1
gives $1 \cdot 1 + 2 \cdot 2 + 0 \cdot 0 = 5$, row 2 gives
$0 + 2 + 0 = 2$, row 3 gives $1 + 6 + 0 = 7$.
```

In [3]:

```
A = np.array([[1, 2, 0],
              [0, 1, 1],
              [1, 3, 1]])  # three rows, each a list of three entries
say(f"A has {A.shape[0]} rows and {A.shape[1]} columns")
# Python counts rows and columns from 0: A[1, 2] is row 2, column 3 of A.
say(f"the entry in row 2, column 3 of A is A[1, 2] = {A[1, 2]}")
# (A v)_i = sum over j of A_ij v_j, one number for each row i:
Av_by_hand = [sum(int(A[i, j]) * int(v[j]) for j in range(3)) for i in range(3)]
say(f"A v by the sum = {Av_by_hand}, with numpy = {(A @ v).tolist()}")
check(Av_by_hand == (A @ v).tolist() == [5, 2, 7],
      "A v computed with the sum rule is (5, 2, 7), the same as numpy's A @ v")
```

Out [3]:

```
A has 3 rows and 3 columns
the entry in row 2, column 3 of A is A[1, 2] = 1
A v by the sum = [5, 2, 7], with numpy = [5, 2, 7]
PASS A v computed with the sum rule is (5, 2, 7), the same as numpy's A @ v
```

Text cell:

```
## 6. The matrix product, three ways

The next cell multiplies $A$ with a second matrix $B$ in three ways and checks
that they agree:

1. three nested loops that add $A_{ij} B_{jk}$ over the dummy index $j$ for every
   pair of free indices $(i, k)$;
2. `np.einsum("ij,jk->ik", A, B)`: the text `"ij,jk->ik"` is the summation
   convention itself. It says: the first factor has the indices $i, j$, the
   second $j, k$; the index $j$ appears twice, so it is summed; the result has
   the indices $i, k$;
3. numpy's operator `A @ B`.

Then it computes $BA$ and checks that it differs from $AB$: the order of the
factors of a matrix product matters. It also checks the transpose rule
$(AB)^T = B^T A^T$ (`.T` is numpy's transpose) and that the trace (the sum of
the diagonal entries, `np.trace`) of $AB$ and of $BA$ is the same.
```

In [4]:

```
B = np.array([[0, 1, 0],
              [1, 0, 2],
              [0, 1, 1]])
n = 3
AB_loops = np.zeros((n, n), dtype=int)  # a 3 x 3 matrix of zeros to fill in
for i in range(n):  # the row of the product (a free index)
    for k in range(n):  # the column of the product (a free index)
```

```
        for j in range(n):  # the summed (dummy) index
            AB_loops[i, k] += A[i, j] * B[j, k]
AB_einsum = np.einsum("ij,jk->ik", A, B)  # j appears twice: it is summed
AB = A @ B
BA = B @ A
say(f"A B = {AB.tolist()}")
say(f"B A = {BA.tolist()}")
check((AB_loops == AB).all() and (AB_einsum == AB).all(),
      "loops, einsum and @ give the same product A B")
check(not (AB == BA).all(), "A B is not equal to B A: the order matters")
check(((A @ B).T == B.T @ A.T).all(), "the transpose rule (A B)^T = B^T A^T")
check(np.trace(AB) == np.trace(BA), "A B and B A have the same trace")
```

Out [4]:

```
A B = [[2, 1, 4], [1, 1, 3], [3, 2, 7]]
B A = [[0, 1, 1], [3, 8, 2], [1, 4, 2]]
PASS loops, einsum and @ give the same product A B
PASS A B is not equal to B A: the order matters
PASS the transpose rule (A B)^T = B^T A^T
PASS A B and B A have the same trace
```

Text cell:

```
The next cell defines a small drawing helper, ‘heat_map‘, used for every picture
of a matrix in this notebook, and draws $A$, $B$, $AB$ and $BA$. Each entry is a
coloured square (red positive, white zero, blue negative) with its value written
in it. ‘plt.subplots(1, 4)‘ makes a figure with 4 drawing areas (*axes*) side by
side; ‘save_figure‘ saves the picture, shows it below the cell and records its
caption.
```

In [5]:

```
def heat_map(ax, matrix, title, labels=None, annotate=True, digits=0,
             skip_zeros=False):
    """Draw matrix on the axes ax: every entry a coloured square, red for
    positive, white for zero, blue for negative entries; when annotate is True
    the value is written in its square with the given number of digits (not
    for the zero entries when skip_zeros is True)."""
    m = np.asarray(matrix, dtype=float)
    limit = max(1.0, float(np.abs(m).max()))  # colours run from -limit to +limit
    image = ax.imshow(m, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.set_title(title)
    ax.grid(False)  # no grid lines across the coloured squares
    if labels is None:  # number rows and columns 1, 2, 3, ... as in mathematics
        labels = [str(k + 1) for k in range(m.shape[0])]
    ax.set_xticks(range(m.shape[1]), labels[:m.shape[1]])
    ax.set_yticks(range(m.shape[0]), labels[:m.shape[0]])
    if annotate:
        for i in range(m.shape[0]):
            for j in range(m.shape[1]):
```

```
                if skip_zeros and m[i, j] == 0:
                    continue
                # white writing on dark squares, black writing on light ones
                ink = "white" if abs(m[i, j]) > 0.6 * limit else "black"
                ax.text(j, i, f"{m[i, j]:.{digits}f}", ha="center", va="center",
                        color=ink)
    return image


fig, axes = plt.subplots(1, 4, figsize=(11.0, 3.2))
for ax, matrix, title in zip(axes, [A, B, AB, BA], ["A", "B", "A B", "B A"]):
    heat_map(ax, matrix, title)
fig.tight_layout()
save_figure(fig, "matrix_product",
            "Heat maps of the $3 \\times 3$ matrices $A$, $B$ and of their products "
            "$AB$ and $BA$; rows are numbered down and columns across from 1 to 3, "
            "and every square shows its entry, red for positive and white for zero "
            "entries. The two products have different entries, for example 4 and 1 "
            "in row 1, column 3: the order of the factors of a matrix product "
            "matters.")
```

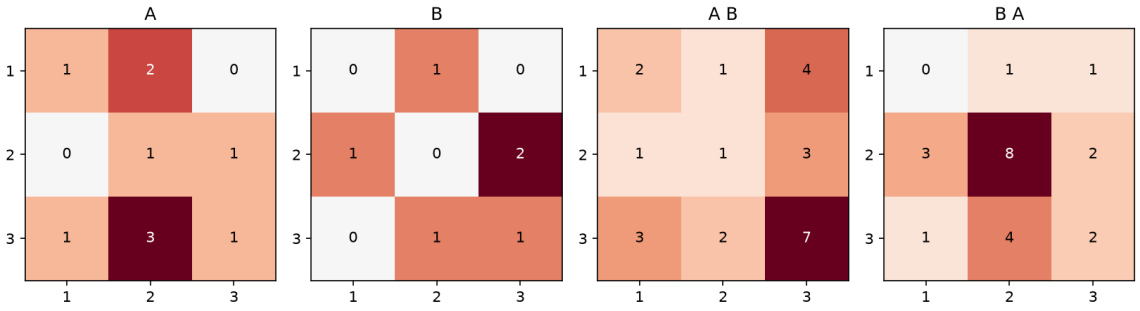

Figure 29: Heat maps of the 3×3 matrices A, B and of their products AB and BA; rows are numbered down and columns across from 1 to 3, and every square shows its entry, red for positive and white for zero entries. The two products have different entries, for example 4 and 1 in row 1, column 3: the order of the factors of a matrix product matters. (Notebook 01a, figure 1.)

Out [5]:

```
Figure 01a.1 saved as Revision/textbook/figures/01a_1_matrix_product.png
```

Text cell:

```
## 7. Permutations and their signs

A permutation of $0, 1, \dots, n-1$ is the same list in another order. An
**inversion** is a pair of places $i < j$ where the entry at place $i$ is larger
than the entry at place $j$. The sign is $(-1)^{\text{number of inversions}}$.

The next cell defines the functions ‘inversions‘ and ‘sign‘ and prints the six
permutations of $0, 1, 2$ (Python’s ‘itertools.permutations‘ lists them) with
```

```
their inversions and signs. For example $(1, 2, 0)$ has the inversions
$(1, 0)$ and $(2, 0)$, two of them, so its sign is $+1$; $(0, 2, 1)$ has one
inversion, $(2, 1)$, so its sign is $-1$.
```

In [6]:

```
def inversions(order):
    """The number of pairs of places i < j whose entries stand in the wrong order."""
    return sum(1 for i in range(len(order)) for j in range(i + 1, len(order))
               if order[i] > order[j])


def sign(order):
    """+1 for an even number of inversions, -1 for an odd number."""
    return 1 if inversions(order) % 2 == 0 else -1


for order in itertools.permutations(range(3)):  # all 6 orderings of 0, 1, 2
    say(f"permutation {order}: number of inversions {inversions(order)}, "
        f"sign {sign(order):+d}")
signs_of_three = [sign(order) for order in itertools.permutations(range(3))]
check(signs_of_three.count(1) == 3 and signs_of_three.count(-1) == 3,
      "of the 6 permutations of 3 objects, 3 are even and 3 are odd")
```

Out [6]:

```
permutation (0, 1, 2): number of inversions 0, sign +1
permutation (0, 2, 1): number of inversions 1, sign -1
permutation (1, 0, 2): number of inversions 1, sign -1
permutation (1, 2, 0): number of inversions 2, sign +1
permutation (2, 0, 1): number of inversions 2, sign +1
permutation (2, 1, 0): number of inversions 3, sign -1
PASS of the 6 permutations of 3 objects, 3 are even and 3 are odd
```

Text cell:

```
The next cell checks three facts for every $n$ from 1 to 6:

- there are $n!$ permutations (`math.factorial(n)`);
- for $n \geq 2$ exactly half of them are even;
- exchanging the entries at two places of a permutation always changes its sign
  (checked for every permutation of $n$ objects and every pair of places).

The third fact is the reason why a determinant changes sign when two rows are
exchanged (section 9 of this notebook).
```

In [7]:

```
swap_rule_holds = True
for n in range(1, 7):
```

```
    orders = list(itertools.permutations(range(n)))
    even = sum(1 for order in orders if sign(order) == 1)
    say(f"n = {n}: n! = {len(orders):3d} permutations; even {even:3d}, "
        f"odd {len(orders) - even:3d}")
    swap_rule_holds &= len(orders) == math.factorial(n)
    swap_rule_holds &= (n == 1 or 2 * even == len(orders))
    for order in orders:
        for i in range(n):
            for j in range(i + 1, n):
                swapped = list(order)
                swapped[i], swapped[j] = swapped[j], swapped[i]  # exchange two
                swap_rule_holds &= sign(swapped) == -sign(order)
check(swap_rule_holds, "n! permutations, half of them even (n >= 2), and every "
      "exchange of two entries flips the sign (n = 1 to 6)")
```

Out [7]:

```
n = 1: n! =   1 permutations; even   1, odd   0
n = 2: n! =   2 permutations; even   1, odd   1
n = 3: n! =   6 permutations; even   3, odd   3
n = 4: n! =  24 permutations; even  12, odd  12
n = 5: n! = 120 permutations; even  60, odd  60
n = 6: n! = 720 permutations; even 360, odd 360
PASS n! permutations, half of them even (n >= 2), and every exchange of two entries flips
    the sign (n = 1 to 6)
```

Text cell:

```
The **permutation matrix** $P_\sigma$ of a permutation $\sigma$ has a 1 in row $i$,
column $\sigma(i)$ and 0 elsewhere. The next cell draws the 24 permutation
matrices of 4 objects, each with the sign of its permutation in the title. Look
at them: the identity (sign $+1$) is the diagonal; exchanging two rows of a
matrix always gives the matrix of a permutation with the opposite sign. The
next section of this notebook proves that $\det P_\sigma = \mathrm{sign}(\sigma)$.
```

In [8]:

```
def permutation_matrix(order):
    """The matrix with a 1 in row i, column order[i], and 0 elsewhere."""
    P = np.zeros((len(order), len(order)), dtype=int)
    for i, column in enumerate(order):
        P[i, column] = 1
    return P


fig, axes = plt.subplots(4, 6, figsize=(10.0, 7.4))
for ax, order in zip(axes.flat, itertools.permutations(range(4))):
    # Grey squares for the 1s; the title gives the permutation and its sign.
    ax.imshow(permutation_matrix(order), cmap="Greys", vmin=0, vmax=1.4)
    ax.set_title(f"{order}  {sign(order):+d}", fontsize=8)
    ax.set_xticks([])
```

```
    ax.set_yticks([])
fig.tight_layout()
save_figure(fig, "permutation_matrices",
            "The 24 permutation matrices of 4 objects; each small picture is a "
            "$4 \\times 4$ matrix whose dark squares are the entries 1 (all other "
            "entries are 0), and the title gives the permutation, listing for rows "
            "1 to 4 the column of the 1 counted from 0, followed by its sign. "
            "Twelve signs are $+1$ and twelve are $-1$; the first picture is the "
            "identity matrix.")
```

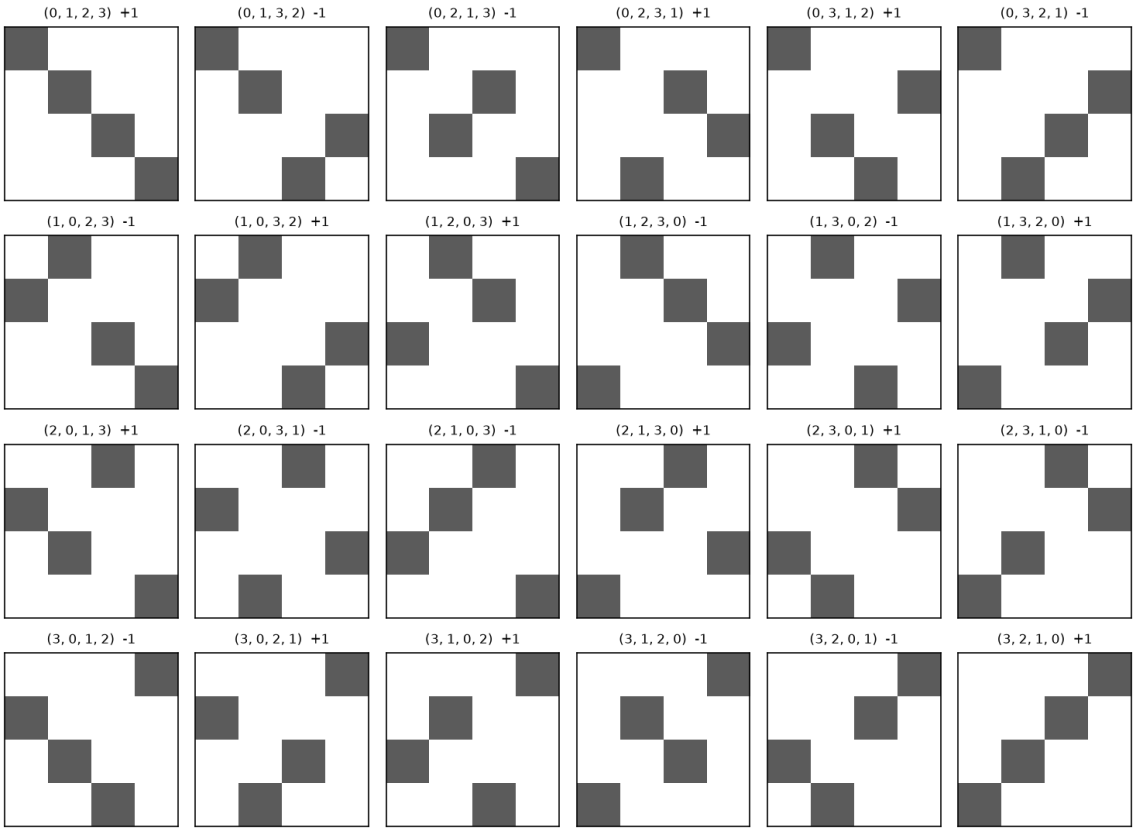

Figure 30: The 24 permutation matrices of 4 objects; each small picture is a 4×4 matrix whose dark squares are the entries 1 (all other entries are 0), and the title gives the permutation, listing for rows 1 to 4 the column of the 1 counted from 0, followed by its sign. Twelve signs are $+1$ and twelve are -1; the first picture is the identity matrix. (Notebook 01a, figure 2.)

Out [8]:

```
Figure 01a.2 saved as Revision/textbook/figures/01a_2_permutation_matrices.png
```

Text cell:

```
## 8. The determinant by the Leibniz formula
```

```
The Leibniz formula takes, for every permutation $\sigma$, one entry from every
row $i$, namely the one in column $\sigma(i)$, multiplies them, attaches the sign
of $\sigma$, and adds the $n!$ results:

$$\det A = \sum_\sigma \mathrm{sign}(\sigma) A_{1\sigma(1)} \cdots
A_{n\sigma(n)}.$$

For $n = 2$ the two permutations $(1, 2)$ (sign $+1$) and $(2, 1)$ (sign $-1$)
give $\det A = A_{11} A_{22} - A_{12} A_{21}$, the familiar $ad - bc$.

The next cell writes this formula as the Python function ‘leibniz_det‘ and
tests it: with sympy symbols for a general $2 \times 2$ matrix (result
$ad - bc$); on the matrix $A$ of section 5 (whose third row is the sum of the
first two, so its determinant must be 0); and on the permutation matrices of 4
objects, whose determinant must be the sign of the permutation (only the term of
$\sigma$ itself is not zero, and it equals $\mathrm{sign}(\sigma) \cdot 1$).
```

In [9]:

```
def leibniz_det(M):
    """The determinant of the square matrix M (a list of rows, a numpy array or a
    sympy Matrix) by the Leibniz formula: for every permutation, the sign times
    one entry from every row, row i giving the entry in column order[i]."""
    rows = [list(row) for row in (M.tolist() if hasattr(M, "tolist") else M)]
    total = 0
    for order in itertools.permutations(range(len(rows))):
        term = sign(order)
        for i in range(len(rows)):
            term = term * rows[i][order[i]]
        total = total + term
    return total


a, b, c, d = sp.symbols("a b c d")  # four symbols: letters for any numbers
det_2 = leibniz_det([[a, b], [c, d]])
say(f"det of the 2 x 2 matrix with rows (a, b) and (c, d) = {det_2}")
check(sp.expand(det_2 - (a * d - b * c)) == 0, "Leibniz for 2 x 2: a d - b c")
say(f"det A = {leibniz_det(A)} (row 3 of A is row 1 plus row 2)")
check(leibniz_det(A) == 0, "det A = 0 because the rows of A are dependent")
check(all(leibniz_det(permutation_matrix(order)) == sign(order)
          for order in itertools.permutations(range(4))),
      "the determinant of every permutation matrix of 4 objects is its sign")
```

Out [9]:

```
det of the 2 x 2 matrix with rows (a, b) and (c, d) = a*d - b*c
PASS Leibniz for 2 x 2: a d - b c
det A = 0 (row 3 of A is row 1 plus row 2)
PASS det A = 0 because the rows of A are dependent
PASS the determinant of every permutation matrix of 4 objects is its sign
```

Text cell:

```
The next cell compares three ways of computing a determinant on matrices of
whole numbers between $-5$ and $5$ of sizes $n = 1$ to $7$, made by numpy's
random-number generator with the fixed *seed* 12345 (the same numbers on every
computer and every run):

- `leibniz_det` (exact: Python adds and multiplies whole numbers without
  rounding);
- sympy's `Matrix.det()` (exact, by another method, elimination);
- numpy's `np.linalg.det` (floating point, by elimination; it may be off in the
  last digits, so it only has to agree to one millionth of the size of the
  exact result).
```

In [10]:

```
generator = np.random.default_rng(12345)  # random numbers with a fixed seed
all_agree = True
for n in range(1, 8):
    M = generator.integers(-5, 6, size=(n, n))  # whole numbers -5 ... 5
    exact = leibniz_det(M)
    by_sympy = sp.Matrix(M.tolist()).det()
    by_numpy = np.linalg.det(M)  # a floating-point number
    say(f"n = {n}: Leibniz {exact}, sympy {by_sympy}, numpy {by_numpy:.6f} "
        f"(n! = {math.factorial(n)} Leibniz terms)")
    all_agree &= exact == by_sympy and abs(by_numpy - exact) < 1e-6 * max(1, abs(exact))
check(all_agree, "Leibniz, sympy and numpy agree on 7 random integer matrices")
```

Out [10]:

```
n = 1: Leibniz 2, sympy 2, numpy 2.000000 (n! = 1 Leibniz terms)
n = 2: Leibniz 15, sympy 15, numpy 15.000000 (n! = 2 Leibniz terms)
n = 3: Leibniz -35, sympy -35, numpy -35.000000 (n! = 6 Leibniz terms)
n = 4: Leibniz -24, sympy -24, numpy -24.000000 (n! = 24 Leibniz terms)
n = 5: Leibniz 730, sympy 730, numpy 730.000000 (n! = 120 Leibniz terms)
n = 6: Leibniz 15445, sympy 15445, numpy 15445.000000 (n! = 720 Leibniz terms)
n = 7: Leibniz 60109, sympy 60109, numpy 60109.000000 (n! = 5040 Leibniz terms)
PASS Leibniz, sympy and numpy agree on 7 random integer matrices
```

Text cell:

```
## 9. The rules of determinants

The next cell checks, exactly with sympy, six rules on two random $4 \times 4$
integer matrices $M$ and $N$:

1. $\det(MN) = \det M \cdot \det N$ (the determinant of a product);
2. $\det(M^T) = \det M$;
3. exchanging two rows changes the sign of the determinant: after the exchange,
   the term of a permutation $\sigma$ has the same product of entries as the term
   of the permutation with the entries $\sigma(r)$ and $\sigma(s)$ of the two
   rows $r, s$ exchanged, whose sign is opposite (section 7); so all products
   come back with opposite signs;
```

```
4. a matrix with two equal rows has determinant 0 (exchanging the two equal rows
   changes nothing, yet by rule 3 changes the sign: $D = -D$, so $D = 0$);
5. multiplying one row by a number $c$ multiplies the determinant by $c$ (every
   term contains exactly one entry of that row);
6. adding $c$ times row $s$ to row $r$ does not change the determinant: every
   term contains exactly one entry of row $r$, so the determinant splits into
   $\det M$ plus $c$ times the determinant of $M$ with row $r$ replaced by row
   $s$, which has two equal rows and is 0 by rule 4.

It also checks that the determinant of a diagonal matrix is the product of its
diagonal entries.
```

In [11]:

```
M = sp.Matrix(generator.integers(-5, 6, size=(4, 4)).tolist())
N = sp.Matrix(generator.integers(-5, 6, size=(4, 4)).tolist())
say(f"det M = {M.det()}, det N = {N.det()}, det(M N) = {(M * N).det()}")
check((M * N).det() == M.det() * N.det(), "rule 1: det(M N) = det M det N")
check(M.T.det() == M.det(), "rule 2: det of the transpose = det")
swapped = M.copy()
swapped.row_swap(0, 2)  # exchange rows 1 and 3
check(swapped.det() == -M.det(), "rule 3: exchanging two rows flips the sign")
twin = M.copy()
twin[3, :] = M[1, :]  # make row 4 equal to row 2
check(twin.det() == 0, "rule 4: two equal rows give determinant 0")
scaled = M.copy()
scaled[1, :] = 7 * M[1, :]  # multiply row 2 by 7
check(scaled.det() == 7 * M.det(), "rule 5: a row times 7 multiplies det by 7")
added = M.copy()
added[0, :] = M[0, :] + 5 * M[3, :]  # row 1 plus 5 times row 4
check(added.det() == M.det(), "rule 6: adding a multiple of a row keeps det")
D = sp.diag(2, -3, 5, 7)
check(D.det() == 2 * (-3) * 5 * 7, "a diagonal matrix: det = product of the diagonal")
```

Out [11]:

```
det M = -192, det N = 128, det(M N) = -24576
PASS rule 1: det(M N) = det M det N
PASS rule 2: det of the transpose = det
PASS rule 3: exchanging two rows flips the sign
PASS rule 4: two equal rows give determinant 0
PASS rule 5: a row times 7 multiplies det by 7
PASS rule 6: adding a multiple of a row keeps det
PASS a diagonal matrix: det = product of the diagonal
```

Text cell:

```
## 10. What a determinant measures: a signed area

A $2 \times 2$ matrix $M$ moves every point $(x, y)$ of the plane to
$M (x, y)$. The unit square with corners $(0,0)$, $(1,0)$, $(1,1)$, $(0,1)$ goes
```

```
to the parallelogram with corners $(0,0)$, $M e_1$, $M e_1 + M e_2$, $M e_2$,
where $e_1 = (1, 0)$, $e_2 = (0, 1)$ and $M e_1$, $M e_2$ are the two columns of
$M$. Its area is $|\det M|$; the sign of $\det M$ says whether the order of the
corners is kept (counterclockwise, $\det M > 0$) or reversed ($\det M < 0$).

The next cell measures the *signed area* of each image with the shoelace formula
$\tfrac12 \sum_k (x_k y_{k+1} - x_{k+1} y_k)$ (the sum over the corners in order,
the last one followed by the first), which is positive for corners in
counterclockwise order, checks that it equals $\det M$ for three example
matrices and for 1000 random ones, and draws the three examples: determinant
$2$ (area doubled), $-1.5$ (area 1.5, orientation reversed) and $0$ (the square
is flattened onto a line).
```

In [12]:

```
def signed_area(corners):
    """The shoelace formula: positive for corners listed counterclockwise."""
    total = 0.0
    for k in range(len(corners)):
        x1, y1 = corners[k]
        x2, y2 = corners[(k + 1) % len(corners)]  # % len: after the last, the first
        total += x1 * y2 - x2 * y1
    return total / 2


square = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], dtype=float)  # counterclockwise
examples = [np.array([[2.0, 1.0], [1.0, 1.5]]),
            np.array([[1.0, 2.0], [1.0, 0.5]]),
            np.array([[1.0, 2.0], [0.5, 1.0]])]
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.9))
for ax, Mx in zip(axes, examples):
    image = square @ Mx.T  # each corner (a row) multiplied by the matrix
    ax.fill(square[:, 0], square[:, 1], color="0.85", label="unit square")
    ax.fill(image[:, 0], image[:, 1], color="tab:orange", alpha=0.5,
            label="its image")
    ax.annotate("", xy=Mx[:, 0], xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": "tab:blue", "lw": 2})
    ax.annotate("", xy=Mx[:, 1], xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": "tab:red", "lw": 2})
    ax.text(*(Mx[:, 0] * 1.05), "$M e_1$", color="tab:blue")
    ax.text(*(Mx[:, 1] * 1.05), "$M e_2$", color="tab:red")
    ax.set_title(f"det M = {np.linalg.det(Mx):+.2f}, area {signed_area(image):+.2f}")
    ax.set_aspect("equal")
    ax.set_xlim(-0.3, 3.4)
    ax.set_ylim(-0.3, 2.9)
    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")
    ax.legend(loc="upper left", fontsize=8)
fig.tight_layout()
save_figure(fig, "area_and_determinant",
            "The unit square (grey) and its image (orange) under three $2 \\times 2$ "
            "matrices $M$; horizontal axis $x$, vertical axis $y$ (pure numbers). The "
            "blue and red arrows are the columns $M e_1$ and $M e_2$. Left: "
```

```
            "$\\det M = 2$, the area doubles. Middle: $\\det M = -1.5$, the area is "
            "1.5 and the arrows have exchanged their order (the picture is "
            "mirrored). Right: $\\det M = 0$, the square is flattened onto a line "
            "and the matrix cannot be undone.")
areas_match = all(abs(signed_area(square @ Mx.T) - np.linalg.det(Mx)) < 1e-12
                  for Mx in examples)
random_matrices = generator.normal(size=(1000, 2, 2))  # 1000 random 2 x 2 matrices
areas_match &= all(abs(signed_area(square @ Mx.T) - np.linalg.det(Mx)) < 1e-12
                   for Mx in random_matrices)
check(areas_match, "the signed area of the image of the unit square is det M "
      "(3 examples and 1000 random matrices)")
```

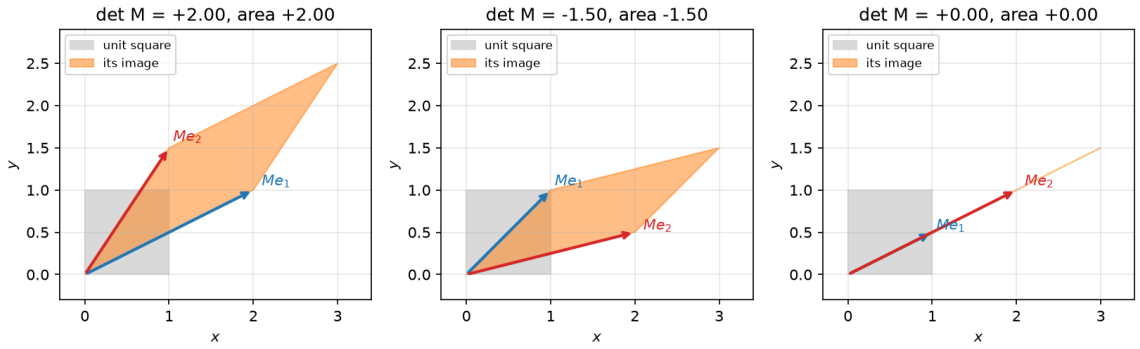

Figure 31: The unit square (grey) and its image (orange) under three 2×2 matrices M; horizontal axis x, vertical axis y (pure numbers). The blue and red arrows are the columns Me_1 and Me_2. Left: $\det M = 2$, the area doubles. Middle: $\det M = -1.5$, the area is 1.5 and the arrows have exchanged their order (the picture is mirrored). Right: $\det M = 0$, the square is flattened onto a line and the matrix cannot be undone. (Notebook 01a, figure 3.)

Out [12]:

```
Figure 01a.3 saved as Revision/textbook/figures/01a_3_area_and_determinant.png
PASS the signed area of the image of the unit square is det M (3 examples and 1000 random
    matrices)
```

Text cell:

```
## 11. Why computers do not use the Leibniz formula for big matrices

The Leibniz formula has $n!$ terms. Elimination (subtracting multiples of rows
until the matrix is triangular, which by rule 6 does not change the
determinant; then the determinant is the product of the diagonal) needs about
$n^3/3$ multiplications. The next cell prints and plots both numbers for
$n = 1$ to $16$. For the $16 \times 16$ gamma matrices of the theory the Leibniz
formula would have $16!$ = 20,922,789,888,000 terms. The vertical axis is
logarithmic: each step up multiplies by 10.
```

In [13]:

```
sizes = np.arange(1, 17)
leibniz_terms = np.array([math.factorial(int(k)) for k in sizes], dtype=float)
elimination_steps = sizes.astype(float) ** 3 / 3
report("number of terms of the Leibniz formula for n = 16", math.factorial(16))
check(math.factorial(16) == 20922789888000, "16! = 20 922 789 888 000")
fig, ax = plt.subplots()
ax.semilogy(sizes, leibniz_terms, "o-", label="Leibniz formula: $n!$ terms")
ax.semilogy(sizes, elimination_steps, "s-", label="elimination: about $n^3/3$ steps")
ax.axvline(16, color="black", ls=":", label="$n = 16$: the gamma matrices")
ax.set_xlabel("size $n$ of the matrix")
ax.set_ylabel("number of terms or steps")
ax.set_title("The cost of a determinant")
ax.legend();
# The two counts for n = 16, rounded, for the caption: 2.1e13 and 1365.
mantissa, exponent = f"{leibniz_terms[-1]:.1e}".split("e")
save_figure(fig, "determinant_cost",
            "The number of terms $n!$ of the Leibniz formula (circles) and the "
            "number of steps $n^3/3$ of elimination (squares) for an $n \\times n$ "
            "matrix, $n = 1$ to $16$; horizontal axis $n$, vertical axis the count "
            "on a logarithmic scale. Up to $n = 4$ the two are similar; for "
            f"$n = 16$ the Leibniz formula has about ${mantissa} \\times "
            f"10^{{{int(exponent)}}}$ terms against about "
            f"{elimination_steps[-1]:.0f} steps of elimination.")
```

Out [13]:

```
RESULT number of terms of the Leibniz formula for n = 16 = 20922789888000
PASS 16! = 20 922 789 888 000
```

Out [13]:

```
Figure 01a.4 saved as Revision/textbook/figures/01a_4_determinant_cost.png
```

Text cell:

```
## 12. Matrices of the course: the gamma matrices of the Revision record

The file Revision/algebra/gammas.json of the Revision record holds the eight real
$16 \times 16$ gamma matrices $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$ of the
author's theory (one for each coordinate, rebuilt in the Revision record from the
author's formulas) and the diagonal of the frame metric $\eta$. The next cell
reads the file (a JSON file: text that holds lists and numbers) and checks what
the record states in its check
`reality_signed_permutations`: every entry is $-1$, $0$ or $1$, and every row and
every column has exactly one entry that is not zero. Such a *signed permutation
matrix* is a permutation matrix with some 1s replaced by $-1$.
```

In [14]:

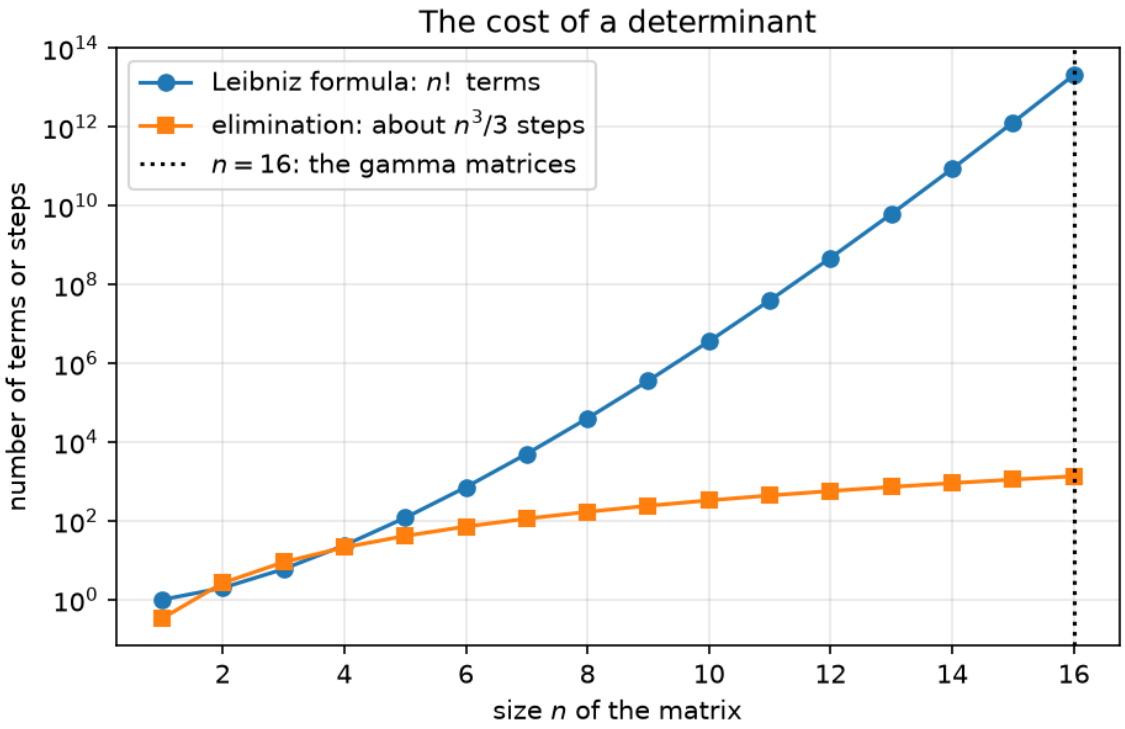

Figure 32: The number of terms $n!$ of the Leibniz formula (circles) and the number of steps $n^3/3$ of elimination (squares) for an $n \times n$ matrix, $n = 1$ to 16; horizontal axis n, vertical axis the count on a logarithmic scale. Up to $n = 4$ the two are similar; for $n = 16$ the Leibniz formula has about 2.1×10^{13} terms against about 1365 steps of elimination. (Notebook 01a, figure 4.)

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
names = record["coordinates"]  # ["x0", "x1", ..., "x7"]
eta = record["eta"]  # the diagonal of the frame metric, in the order x0 ... x7
gamma = [np.array(matrix, dtype=int) for matrix in record["gamma"]]
say(f"coordinates: {names}")
say(f"diagonal of eta: {eta}")
signed_permutation = True
for g_a in gamma:
    signed_permutation &= set(np.unique(g_a).tolist()) <= {-1, 0, 1}
    signed_permutation &= ((g_a != 0).sum(axis=1) == 1).all()  # one per row
    signed_permutation &= ((g_a != 0).sum(axis=0) == 1).all()  # one per column
check(len(gamma) == 8 and all(g_a.shape == (16, 16) for g_a in gamma),
      "the record holds eight 16 x 16 gamma matrices")
check(signed_permutation, "every gamma matrix is a real signed permutation matrix",
      record="Revision/algebra/reports/python-algebra.json, "
             "check reality_signed_permutations")
```

Out [14]:

```
coordinates: ['x0', 'x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7']
diagonal of eta: [1, 1, 1, 1, -1, -1, -1, -1]
PASS the record holds eight 16 x 16 gamma matrices
PASS every gamma matrix is a real signed permutation matrix
     reproduces Revision/algebra/reports/python-algebra.json, check
    reality_signed_permutations
```

Text cell:

```
For a signed permutation matrix only one term of the Leibniz formula is not zero:
the term of the permutation $\sigma$ whose entries $\gamma_{i\sigma(i)}$ are the
nonzero ones. So $\det \gamma = \mathrm{sign}(\sigma) \cdot s_1 s_2 \cdots s_{16}$,
where $s_i = \pm 1$ are the signs of these entries. This needs 16 numbers and one
permutation instead of $16!$ terms.

The next cell computes the determinant of each gamma matrix this way, compares it
with sympy's exact determinant and with numpy's, and checks that each gamma
matrix squared is $\eta_{aa}$ times the $16 \times 16$ identity matrix: $+I$ for
the space-like directions $x_0, x_1, x_2, x_3$ and $-I$ for the time-like
directions $x_4, \dots, x_7$ (this is the part with equal indices of the
relation $\gamma^a \gamma^b + \gamma^b \gamma^a = 2 \eta^{ab} I$ that the gamma
matrices obey).
```

In [15]:

```
identity16 = np.eye(16, dtype=int)  # the 16 x 16 identity matrix
all_det_one = True
squares_right = True
for name, eta_aa, g_a in zip(names, eta, gamma):
    order = [int(np.flatnonzero(row)[0]) for row in g_a]  # column of each nonzero
    signs = [int(g_a[i, order[i]]) for i in range(16)]  # the nonzero entries
```

```
    det_by_structure = sign(order) * math.prod(signs)
    det_exact = sp.Matrix(g_a.tolist()).det()
    det_numpy = np.linalg.det(g_a)
    say(f"gamma^({name}): eta {eta_aa:+d}, {signs.count(-1)} entries -1, "
        f"permutation sign {sign(order):+d}, det {det_by_structure:+d}, "
        f"sympy {det_exact}, numpy {det_numpy:.3f}")
    all_det_one &= det_by_structure == det_exact == 1 and abs(det_numpy - 1) < 1e-9
    squares_right &= (g_a @ g_a == eta_aa * identity16).all()
check(all_det_one, "every gamma matrix has determinant 1 (three ways)")
check(squares_right, "gamma^a gamma^a = eta_aa times the identity for every a",
      record="Revision/algebra/reports/python-algebra.json, check "
             "clifford_relation (its 8 relations with equal indices)")
```

Out [15]:

```
gamma^(x0): eta +1, 0 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x1): eta +1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x2): eta +1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x3): eta +1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x4): eta -1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x5): eta -1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x6): eta -1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
gamma^(x7): eta -1, 8 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000
PASS every gamma matrix has determinant 1 (three ways)
PASS gamma^a gamma^a = eta_aa times the identity for every a
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
    (its 8 relations with equal indices)
```

Text cell:

```
The next cell draws $\gamma^{(x_1)}$ (a space-like direction), $\gamma^{(x_4)}$
(the time) and the product $\gamma^{(x_4)} \gamma^{(x_4)}$, which is $-I$: blue
squares on the diagonal and white elsewhere.
```

In [16]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
ticks = [str(k) for k in range(1, 17)]
pictures = [(gamma[1], "$\\gamma^{(x_1)}$"), (gamma[4], "$\\gamma^{(x_4)}$"),
            (gamma[4] @ gamma[4], "$\\gamma^{(x_4)} \\gamma^{(x_4)} = -I$")]
for ax, (matrix, title) in zip(axes, pictures):
    image = heat_map(ax, matrix, title, labels=ticks, annotate=False)
    ax.tick_params(labelsize=6)
fig.colorbar(image, ax=axes, shrink=0.75, label="entry", ticks=[-1, 0, 1])
save_figure(fig, "gamma_matrices",
            "Heat maps of two of the eight real $16 \\times 16$ gamma matrices of "
            "the Revision record, $\\gamma^{(x_1)}$ (left) and $\\gamma^{(x_4)}$ "
            "(middle), and of the product $\\gamma^{(x_4)}\\gamma^{(x_4)}$ (right); "
            "rows down and columns across numbered 1 to 16, red $+1$, blue $-1$, "
            "white 0. Every row and column of a gamma matrix has exactly one "
            "coloured square, and the square of the time-like $\\gamma^{(x_4)}$ is "
            "minus the identity matrix.")
```

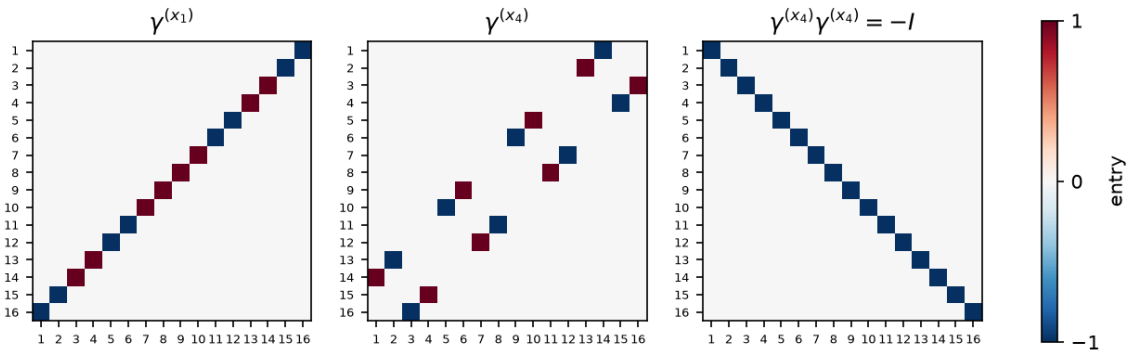

Figure 33: Heat maps of two of the eight real 16×16 gamma matrices of the Revision record, $\gamma^{(x_1)}$ (left) and $\gamma^{(x_4)}$ (middle), and of the product $\gamma^{(x_4)}\gamma^{(x_4)}$ (right); rows down and columns across numbered 1 to 16, red $+1$, blue -1, white 0. Every row and column of a gamma matrix has exactly one coloured square, and the square of the time-like $\gamma^{(x_4)}$ is minus the identity matrix. (Notebook 01a, figure 5.)

Out [16]:

```
Figure 01a.5 saved as Revision/textbook/figures/01a_5_gamma_matrices.png
```

Text cell:

```
## 13. The author's metric and its determinant

The file Revision/gkd_lovelock/results/curvature.json of the Revision record
stores the diagonal of the author's metric as text in the notation of the
program Mathematica, for example 'E^(2*a4[x4])*Sin[6*H*x0]^(1/3)' for
$e^{2a_4(x_4)} \sin^{1/3}(6 H x_0)$ and
'-E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*b4[x4]' for
$-b_4(x_4) e^{-2a_4(x_4)} \sin^{1/3}(6 H x_0)$, and the square root of the size
of its determinant as 'Sin[6*H*x0]*Cot[6*H*x0]*b4[x4]^(3/2)'.

The next cell types the metric exactly as in section 4 of this notebook, with
sympy symbols $a_4$ (the value of $a_4(x_4)$ at one time, any real number),
$b_4$ (the value of $b_4(x_4)$ at one time, a positive number) and
$z = 6 H x_0$ (a positive number), translates the record's text into sympy (the
function 'from_record' replaces the Mathematica names by sympy names), and checks:

- the record's diagonal equals the typed one, entry by entry;
- the product of the diagonal entries, $\det g$, simplifies to $b_4^3 \cos^2 z$
  (no $a_4$ is left);
- the record's $b_4^{3/2} \sin z \cot z$ equals $b_4^{3/2} \cos z$, whose square
  is $\det g$.
```

In [17]:

```
a4 = sp.Symbol("a4", real=True)  # the value of the function a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of the function b4(x4) at one time
```

```
z = sp.Symbol("z", positive=True)  # z = 6 H x0, between 0 and pi/2
s = sp.sin(z) ** sp.Rational(1, 3)  # sin(z) to the power 1/3
g_typed = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * s] * 3 + [sp.Integer(-1)]
           + [-b4 * sp.exp(-2 * a4) * s] * 3)  # the author's order x0, x1, ..., x7
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))


def from_record(text):
    """The record's Mathematica text as a sympy expression."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "sin(z)")
    text = text.replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "z": z, "E": sp.E})


g_record = [from_record(text) for text in curvature["metricDiagonal"]]
check(len(g_record) == 8 and
      all(sp.simplify(x - y) == 0 for x, y in zip(g_record, g_typed)),
      "the record's metric diagonal equals the author's metric typed here")
det_g = sp.Mul(*g_typed)  # the product of the 8 diagonal entries
say(f"product of the diagonal entries, as sympy writes it: {det_g}")
check(sp.simplify(det_g - b4 ** 3 * sp.cos(z) ** 2) == 0,
      "det g = b4^3 cos(z)^2 for every a4: the e^(6 a4) of space and the e^(-6 a4) "
      "of the extra times cancel",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "sqrt_abs_det_g")
root_record = from_record(curvature["sqrtAbsDetG"])
say(f"the record's square root of |det g|: {root_record}")
check(sp.simplify(root_record - b4 ** sp.Rational(3, 2) * sp.cos(z)) == 0 and
      sp.simplify(root_record ** 2 - det_g) == 0,
      "the record's sqrtAbsDetG = b4^(3/2) sin z cot z = b4^(3/2) cos z, and its "
      "square is det g",
      record="Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG")
```

Out [17]:

```
PASS the record's metric diagonal equals the author's metric typed here
product of the diagonal entries, as sympy writes it: b4**3*sin(z)**2*cot(z)**2
PASS det g = b4^3 cos(z)^2 for every a4: the e^(6 a4) of space and the e^(-6 a4) of the
    extra times cancel
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    sqrt_abs_det_g
the record's square root of |det g|: b4**(3/2)*sin(z)*cot(z)
PASS the record's sqrtAbsDetG = b4^(3/2) sin z cot z = b4^(3/2) cos z, and its square is
    det g
     reproduces Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG
```

Text cell:

```
The next cell turns the exact formulas into numbers (sympy's ‘lambdify‘ makes a
numpy function of a formula) and draws two pictures. Left: the heat map of the
```

```
$8 \times 8$ metric at $a_4 = 0.5$, $b_4 = 1$, $z = 0.9$, with the coordinate
names on the axes; only the diagonal is coloured. Right: $\det g / b_4^3$
computed with numpy from the $8 \times 8$ matrix for the three pairs
$(a_4, b_4) = (-1, 1)$, $(0, 0.5)$ and $(1, 2)$ along $0 < z < \pi/2$, together
with $\cos^2 z$: the three curves lie on top of each other. It also checks the
agreement in numbers at 300 values of $z$.
```

In [18]:

```
metric_numbers = sp.lambdify((a4, b4, z), g_typed, "numpy")  # a4, b4, z -> 8 numbers
z_values = np.linspace(0.01, np.pi / 2 - 0.01, 300)  # inside (0, pi/2)
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6))
g_point = np.diag(np.array(metric_numbers(0.5, 1.0, 0.9), dtype=float))
coordinate_labels = [f"$x_{k}$" for k in range(8)]  # x with subscripts 0 ... 7
heat_map(left, g_point, "metric $g$ at $a_4 = 0.5$, $b_4 = 1$, $z = 0.9$",
         labels=coordinate_labels, digits=2, skip_zeros=True)
left.tick_params(labelsize=8)
largest_error = 0.0
for a4_value, b4_value, style in [(-1.0, 1.0, "-"), (0.0, 0.5, "--"), (1.0, 2.0, ":")]:
    dets = np.array([np.linalg.det(np.diag(np.array(metric_numbers(a4_value, b4_value,
                                                                   zv), dtype=float)))
                     for zv in z_values]) / b4_value ** 3
    largest_error = max(largest_error, float(np.max(np.abs(dets - np.cos(z_values)
                                                           ** 2))))
    right.plot(z_values, dets, style, lw=2,
               label=f"det g / $b_4^3$, $a_4 = {a4_value:+.0f}$, $b_4 = {b4_value:g}$")
right.plot(z_values, np.cos(z_values) ** 2, color="black", lw=0.8,
           label="$\\cos^2 z$")
right.set_xlabel("$z = 6 H x_0$")
right.set_ylabel("$\\det g / b_4^3$")
right.set_title("$\\det g / b_4^3$ does not depend on $a_4$")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "metric_and_determinant",
            "Left: heat map of the author's $8 \\times 8$ metric at $a_4 = 0.5$, "
            "$b_4 = 1$ and "
            "$z = 6 H x_0 = 0.9$, rows and columns labelled by the coordinates "
            "$x_0$ to $x_7$; only the diagonal is not zero (the plain white "
            "squares are zeros), positive for $x_0, x_1, x_2, x_3$ and negative "
            "for $x_4$ to $x_7$. Right: the "
            "determinant of the metric divided by $b_4^3$, computed numerically "
            "for $(a_4, b_4) = (-1, 1)$, $(0, 0.5)$ and $(1, 2)$ (three line "
            "styles) against $z$ from 0 to $\\pi/2$, and "
            "$\\cos^2 z$ (thin black line); horizontal axis $z$, vertical axis "
            "$\\det g / b_4^3$ (pure numbers). All curves coincide: the inflation "
            "of space and the deflation of the extra times cancel in the "
            "determinant, and $b_4$ enters only as the factor $b_4^3$.")
report("largest difference between det g / b4^3 and cos(z)^2 at 900 points",
       f"{largest_error:.1e}")
check(largest_error < 1e-12, "numpy: det g = b4^3 cos(z)^2 at 300 values of z for "
      "each of (a4, b4) = (-1, 1), (0, 0.5), (1, 2)")
```

Out [18]:

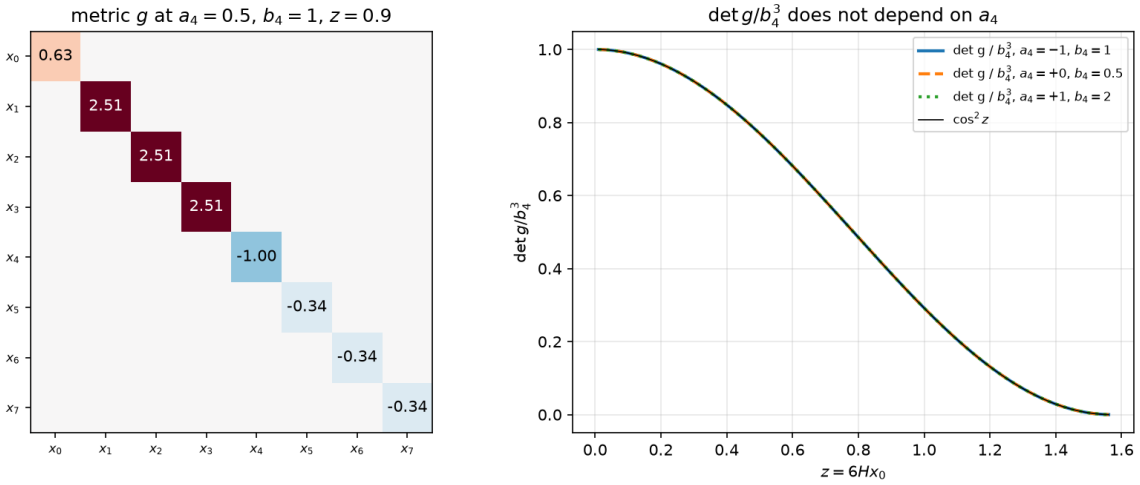

Figure 34: Left: heat map of the author's 8×8 metric at $a_4 = 0.5$, $b_4 = 1$ and $z = 6Hx_0 = 0.9$, rows and columns labelled by the coordinates x_0 to x_7; only the diagonal is not zero (the plain white squares are zeros), positive for x_0, x_1, x_2, x_3 and negative for x_4 to x_7. Right: the determinant of the metric divided by b_4^3, computed numerically for $(a_4, b_4) = (-1, 1)$, $(0, 0.5)$ and $(1, 2)$ (three line styles) against z from 0 to $\pi/2$, and $\cos^2 z$ (thin black line); horizontal axis z, vertical axis $\det g / b_4^3$ (pure numbers). All curves coincide: the inflation of space and the deflation of the extra times cancel in the determinant, and b_4 enters only as the factor b_4^3. (Notebook 01a, figure 6.)

```
Figure 01a.6 saved as Revision/textbook/figures/01a_6_metric_and_determinant.png
RESULT largest difference between det g / b4^3 and cos(z)^2 at 900 points = 2.9e-15
PASS numpy: det g = b4^3 cos(z)^2 at 300 values of z for each of (a4, b4) = (-1, 1), (0,
    0.5), (1, 2)
```

Text cell:

```
## 14. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [19]:

```
figure_names = ["01a_1_matrix_product.png", "01a_2_permutation_matrices.png",
                "01a_3_area_and_determinant.png", "01a_4_determinant_cost.png",
                "01a_5_gamma_matrices.png", "01a_6_metric_and_determinant.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

Out [19]:

```
PASS all 6 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 01a)
```

Text cell:

```
## 15. What this notebook showed

- A matrix product $(AB)_{ik} = A_{ij} B_{jk}$ (summation convention: $j$ is
  summed) can be computed with loops, with ‘einsum‘ or with ‘@‘; the three agree,
  and $AB \neq BA$ in general, while $(AB)^T = B^T A^T$ and the traces of $AB$
  and $BA$ are equal.
- A permutation of $n$ objects has a sign $\pm 1$ given by its number of
  inversions; half of the $n!$ permutations are even ($n \geq 2$), and
  exchanging two entries flips the sign.
- The Leibniz formula, written by hand, gives the same determinants as sympy and
  numpy; determinants obey the six rules of section 9 and measure signed areas.
  For big matrices the Leibniz formula is far too long ($16! \approx 2 \times
  10^{13}$ terms), but for a signed permutation matrix only one term survives.
- The eight gamma matrices of the Revision record are real signed permutation
  matrices with determinant 1; each squares to $+I$ (space-like directions
  $x_0, x_1, x_2, x_3$) or $-I$ (time-like directions $x_4, \dots, x_7$).
- The determinant of the author’s metric is $\det g = b_4^3 \cos^2(6 H x_0)$ for
  every value of $a_4$: the growth $e^{6a_4}$ of the three space directions and
  the shrinking $e^{-6a_4}$ of the three extra times cancel exactly, and $b_4$
  enters only as the factor $b_4^3$. This repeats the Revision record’s check
  ‘sqrt_abs_det_g‘.
```

1.25 Line-by-line walk-through of Notebook 01a

The notebook has nineteen code cells, In [1] to In [19]. As in Section 1.9, a quoted line `...)` stands for the remaining lines of a figure caption, which Section 1.24 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 1.23. Its code is, line for line, the set-up code of Notebook 01f explained in Section 1.9, with the single difference `NOTEBOOK_ID = "01a"`. It prints Set-up of notebook 01a complete: repository folder found, helpers defined.

In [2], vectors.

```
import itertools  # all orderings (permutations) of a list
import math  # factorials

import numpy as np  # arrays of numbers: vectors and matrices
import sympy as sp  # exact arithmetic with whole numbers, fractions and symbols
```

`itertools` (part of Python) produces, among other things, all permutations of a list (`itertools.permutations`); `math` gives the factorial `math.factorial` and the product `math.prod`; numpy and sympy as before.

```
v = np.array([1, 2, 0])  # a vector with 3 components
w = np.array([0, 1, 1])
say(f"v = {v.tolist()}, w = {w.tolist()}")
say(f"v + w = {(v + w).tolist()}, 3 v = {(3 * v).tolist()}")
```

`np.array([1, 2, 0])` makes a numpy array, here of whole numbers. `+` adds two arrays entry by entry and `3 * v` multiplies every entry: exactly the rules of Section 1.18. `.tolist()` turns an array into a plain Python list for printing. Output: v = [1, 2, 0], w = [0, 1, 1] and v + w = [1, 3, 1], 3 v = [3, 6, 0].

```
# The dot product as a sum over the index i = 0, 1, 2 (Python counts from 0):
dot_by_hand = sum(int(v[i]) * int(w[i]) for i in range(3))
say(f"v . w by the sum = {dot_by_hand}, with numpy's @ = {int(v @ w)}")
check(dot_by_hand == int(v @ w) == 2, "the dot product of v and w is 2")
```

`sum(... for i in range(3))` adds the products $v_i w_i$ for $i = 0, 1, 2$ (Python's numbering of the components 1, 2, 3); `int(...)` turns a numpy number into a plain Python whole number. For two vectors, numpy's operator `@` is the dot product. Both give 2, and the check confirms it. Output: v . w by the sum = 2, with numpy's @ = 2, and the PASS line.

In [3], matrix times vector.

```
A = np.array([[1, 2, 0],
              [0, 1, 1],
              [1, 3, 1]])  # three rows, each a list of three entries
say(f"A has {A.shape[0]} rows and {A.shape[1]} columns")
# Python counts rows and columns from 0: A[1, 2] is row 2, column 3 of A.
say(f"the entry in row 2, column 3 of A is A[1, 2] = {A[1, 2]}")
```

A matrix is an array of rows. `A.shape` is the pair (number of rows, number of columns). The second line prints the entry $A_{23} = 1$ under its Python name `A[1, 2]`.

```
# (A v)_i = sum over j of A_ij v_j, one number for each row i:
Av_by_hand = [sum(int(A[i, j]) * int(v[j]) for j in range(3)) for i in range(3)]
say(f"A v by the sum = {Av_by_hand}, with numpy = {(A @ v).tolist()}")
check(Av_by_hand == (A @ v).tolist() == [5, 2, 7],
      "A v computed with the sum rule is (5, 2, 7), the same as numpy's A @ v")
```

The list comprehension computes, for each row i, the sum over j of $A_{ij} v_j$: the rule of Section 1.18. `A @ v` is numpy's matrix times vector. Both give $(5, 2, 7)$. Output: A has 3 rows and 3 columns, the entry line, A v by the sum = [5, 2, 7], with numpy = [5, 2, 7], and the PASS line.

In [4], the matrix product, three ways.

```
B = np.array([[0, 1, 0],
              [1, 0, 2],
              [0, 1, 1]])
n = 3
AB_loops = np.zeros((n, n), dtype=int)  # a 3 x 3 matrix of zeros to fill in
for i in range(n):  # the row of the product (a free index)
    for k in range(n):  # the column of the product (a free index)
        for j in range(n):  # the summed (dummy) index
            AB_loops[i, k] += A[i, j] * B[j, k]
```

The second matrix B of Section 1.18. `np.zeros((n, n), dtype=int)` is a 3×3 array of whole-number zeros. The three nested loops compute $(AB)_{ik} = \sum_j A_{ij}B_{jk}$: the two outer loops run over the free indices i and k, the inner loop adds the terms of the dummy index j (`+=` adds to the entry).

```
AB_einsum = np.einsum("ij,jk->ik", A, B)  # j appears twice: it is summed
AB = A @ B
BA = B @ A
say(f"A B = {AB.tolist()}")
say(f"B A = {BA.tolist()}")
```

The same product with `einsum` (the summation convention of Section 1.18) and with `@`; then BA. Output: A B = [[2, 1, 4], [1, 1, 3], [3, 2, 7]] and B A = [[0, 1, 1], [3, 8, 2], [1, 4, 2]], the matrices of Section 1.18.

```
check((AB_loops == AB).all() and (AB_einsum == AB).all(),
      "loops, einsum and @ give the same product A B")
check(not (AB == BA).all(), "A B is not equal to B A: the order matters")
check(((A @ B).T == B.T @ A.T).all(), "the transpose rule (A B)^T = B^T A^T")
check(np.trace(AB) == np.trace(BA), "A B and B A have the same trace")
```

Four checks: the three ways agree entry by entry; $AB \neq BA$; the transpose rule (`.T` is numpy's transpose); equal traces (`np.trace`). Four PASS lines.

In [5], heat maps.

```
def heat_map(ax, matrix, title, labels=None, annotate=True, digits=0,
             skip_zeros=False):
    """Draw matrix on the axes ax: every entry a coloured square, red for
    positive, white for zero, blue for negative entries; when annotate is True
    the value is written in its square with the given number of digits (not
    for the zero entries when skip_zeros is True)."""
    m = np.asarray(matrix, dtype=float)
    limit = max(1.0, float(np.abs(m).max()))  # colours run from -limit to +limit
    image = ax.imshow(m, cmap="RdBu_r", vmin=-limit, vmax=limit)
```

A drawing helper used for every matrix picture of this notebook. `np.asarray(..., dtype=float)` makes a floating-point array of any table. `limit` is the largest size of an entry (at least 1). `ax.imsho w` draws the table as coloured squares; the **colour map** `"RdBu_r"` runs from blue through white to red, and `vmin=-limit, vmax=limit` puts white exactly at 0, so that red means positive and blue negative.

```
    ax.set_title(title)
    ax.grid(False)  # no grid lines across the coloured squares
    if labels is None:  # number rows and columns 1, 2, 3, ... as in mathematics
        labels = [str(k + 1) for k in range(m.shape[0])]
    ax.set_xticks(range(m.shape[1]), labels[:m.shape[1]])
    ax.set_yticks(range(m.shape[0]), labels[:m.shape[0]])
```

The title; no grid lines; if no labels are given, rows and columns are labelled 1, 2, 3, ... as in mathematics (Python's square number k gets the label $k + 1$); `set_xticks` and `set_yticks` put the labels under the columns and beside the rows.

```
    if annotate:
        for i in range(m.shape[0]):
            for j in range(m.shape[1]):
                if skip_zeros and m[i, j] == 0:
                    continue
                # white writing on dark squares, black writing on light ones
                ink = "white" if abs(m[i, j]) > 0.6 * limit else "black"
                ax.text(j, i, f"{m[i, j]:.{digits}f}", ha="center", va="center",
                        color=ink)
    return image
```

When `annotate` is true, the value of every entry is written in the middle of its square (`ha` and `va`: horizontal and vertical alignment), with `digits` digits after the point (the format inside the braces is itself built from `digits`); `continue` skips a zero entry when `skip_zeros` is true. Dark squares get white writing. The helper returns the picture, so that a colour bar can be added later.

```
fig, axes = plt.subplots(1, 4, figsize=(11.0, 3.2))
for ax, matrix, title in zip(axes, [A, B, AB, BA], ["A", "B", "A B", "B A"]):
    heat_map(ax, matrix, title)
fig.tight_layout()
save_figure(fig, "matrix_product",
            "Heat maps of the $3 \\times 3$ matrices $A$, $B$ and of their products "
            ...)
```

Four panels in a row, one heat map in each. What Figure 01a.1 shows: AB and BA have different entries, for example 4 and 1 in row 1, column 3.

In [6], permutations and their signs.

```
def inversions(order):
    """The number of pairs of places i < j whose entries stand in the wrong order."""
    return sum(1 for i in range(len(order)) for j in range(i + 1, len(order))
               if order[i] > order[j])


def sign(order):
    """+1 for an even number of inversions, -1 for an odd number."""
    return 1 if inversions(order) % 2 == 0 else -1
```

`inversions` counts the pairs of places $i < j$ (the inner range starts at $i + 1$) with `order[i] > order[j]`: it adds 1 for each such pair. `sign` returns $+1$ when that number is even (its remainder after division by 2 is 0) and -1 otherwise: the definition of Section 1.19.

```
for order in itertools.permutations(range(3)):  # all 6 orderings of 0, 1, 2
    say(f"permutation {order}: number of inversions {inversions(order)}, "
```

```
        f"sign {sign(order):+d}")
signs_of_three = [sign(order) for order in itertools.permutations(range(3))]
check(signs_of_three.count(1) == 3 and signs_of_three.count(-1) == 3,
      "of the 6 permutations of 3 objects, 3 are even and 3 are odd")
```

`itertools.permutations(range(3))` produces the six orderings of 0, 1, 2 as tuples (lists in round brackets). The loop prints each with its inversions and sign; the output is the table of Section 1.19. `.count(1)` counts the entries equal to 1. The check: three even, three odd.

In [7], the exchange theorem for up to 6 objects.

```
swap_rule_holds = True
for n in range(1, 7):
    orders = list(itertools.permutations(range(n)))
    even = sum(1 for order in orders if sign(order) == 1)
    say(f"n = {n}: n! = {len(orders):3d} permutations; even {even:3d}, "
        f"odd {len(orders) - even:3d}")
    swap_rule_holds &= len(orders) == math.factorial(n)
    swap_rule_holds &= (n == 1 or 2 * even == len(orders))
```

For $n = 1$ to 6: all permutations, the number of even ones, and a printed line; then two conditions are added to `swap_rule_holds` with `&=` (it stays true only if every condition holds): there are $n!$ permutations, and (for $n \geq 2$) exactly half are even. The output lines read n = 1: n! = 1 permutations; even 1, odd 0 up to n = 6: n! = 720 permutations; even 360, odd 360.

```
    for order in orders:
        for i in range(n):
            for j in range(i + 1, n):
                swapped = list(order)
                swapped[i], swapped[j] = swapped[j], swapped[i]  # exchange two
                swap_rule_holds &= sign(swapped) == -sign(order)
check(swap_rule_holds, "n! permutations, half of them even (n >= 2), and every "
      "exchange of two entries flips the sign (n = 1 to 6)")
```

For every permutation and every pair of places $i < j$: a copy of the permutation with the two entries exchanged (`a, b = b, a` exchanges two values in Python) must have the opposite sign. With n up to 6 this tests every exchange of every permutation, the theorem of Section 1.19. One PASS line.

In [8], the permutation matrices of 4 objects.

```
def permutation_matrix(order):
    """The matrix with a 1 in row i, column order[i], and 0 elsewhere."""
    P = np.zeros((len(order), len(order)), dtype=int)
    for i, column in enumerate(order):
        P[i, column] = 1
    return P
```

`enumerate(order)` gives the pairs (place, entry), so row `i` gets a 1 in column `order[i]`: the permutation matrix of Section 1.19.

```
fig, axes = plt.subplots(4, 6, figsize=(10.0, 7.4))
for ax, order in zip(axes.flat, itertools.permutations(range(4))):
    # Grey squares for the 1s; the title gives the permutation and its sign.
    ax.imshow(permutation_matrix(order), cmap="Greys", vmin=0, vmax=1.4)
    ax.set_title(f"{order}  {sign(order):+d}", fontsize=8)
    ax.set_xticks([])
    ax.set_yticks([])
fig.tight_layout()
save_figure(fig, "permutation_matrices",
            "The 24 permutation matrices of 4 objects; each small picture is a "
            ...)
```

A grid of 4 rows and 6 columns of small axes for the $4! = 24$ permutations; `axes.flat` runs through them one after the other. Each matrix is drawn with the grey colour map (`vmax=1.4` makes the 1s dark grey rather than black); the title gives the permutation and its sign; `set_xticks([])` removes the tick labels. What Figure 01a.2 shows: twelve signs $+1$ and twelve -1; the first picture is the identity; exchanging two rows of any picture gives a picture with the opposite sign.

In [9], the Leibniz formula in Python.

```
def leibniz_det(M):
    """The determinant of the square matrix M (a list of rows, a numpy array or a
    sympy Matrix) by the Leibniz formula: for every permutation, the sign times
    one entry from every row, row i giving the entry in column order[i]."""
    rows = [list(row) for row in (M.tolist() if hasattr(M, "tolist") else M)]
    total = 0
    for order in itertools.permutations(range(len(rows))):
        term = sign(order)
        for i in range(len(rows)):
            term = term * rows[i][order[i]]
        total = total + term
    return total
```

The Leibniz formula of Section 1.20, literally. First the matrix is turned into a list of rows, whatever it is (`hasattr(M, "tolist")` asks whether `M` can turn itself into a list, as numpy arrays and sympy matrices can). Then, for every permutation, the term starts as the sign and is multiplied by the entry of row `i` in column `order[i]` for every row; the terms are added. Because Python and sympy add and multiply whole numbers and symbols exactly, the result is exact.

```
a, b, c, d = sp.symbols("a b c d")  # four symbols: letters for any numbers
det_2 = leibniz_det([[a, b], [c, d]])
say(f"det of the 2 x 2 matrix with rows (a, b) and (c, d) = {det_2}")
check(sp.expand(det_2 - (a * d - b * c)) == 0, "Leibniz for 2 x 2: a d - b c")
```

With four symbols the formula gives `a*d - b*c`, the formula $ad - bc$.

```
say(f"det A = {leibniz_det(A)} (row 3 of A is row 1 plus row 2)")
check(leibniz_det(A) == 0, "det A = 0 because the rows of A are dependent")
check(all(leibniz_det(permutation_matrix(order)) == sign(order)
          for order in itertools.permutations(range(4))),
      "the determinant of every permutation matrix of 4 objects is its sign")
```

$\det A = 0$, as computed in Section 1.20, and $\det P_\sigma = \text{sign}(\sigma)$ for all 24 permutations of 4 objects, rule (b) of Section 1.20. Output: the two lines and three PASS lines.

In [10], three ways of computing a determinant.

```
generator = np.random.default_rng(12345)  # random numbers with a fixed seed
all_agree = True
for n in range(1, 8):
    M = generator.integers(-5, 6, size=(n, n))  # whole numbers -5 ... 5
    exact = leibniz_det(M)
    by_sympy = sp.Matrix(M.tolist()).det()
    by_numpy = np.linalg.det(M)  # a floating-point number
```

`np.random.default_rng(12345)` makes a **random-number generator** with the fixed starting number (**seed**) 12345, so that every run on every computer gets the same numbers. `generato r.integers(-5, 6, size=(n, n))` draws an $n \times n$ table of whole numbers from -5 to 5 (the end 6 is not included). For $n = 1$ to 7 the determinant is computed three ways: by `leibniz_det`, by sympy (exact, by elimination) and by numpy's `np.linalg.det` (floating point, by elimination).

```
    say(f"n = {n}: Leibniz {exact}, sympy {by_sympy}, numpy {by_numpy:.6f} "
        f"(n! = {math.factorial(n)} Leibniz terms)")
    all_agree &= exact == by_sympy and abs(by_numpy - exact) < 1e-6 * max(1, abs(exact))
check(all_agree, "Leibniz, sympy and numpy agree on 7 random integer matrices")
```

Each line prints the three results and the number of Leibniz terms; the exact two must agree exactly, numpy's to a millionth of the size of the result. Output: the determinants 2, 15, -35, -24, 730, 15445 and 60109 for $n = 1$ to 7, the same three times each, with 1 to 5040 Leibniz terms; then the PASS line.

In [11], the six rules.

```
M = sp.Matrix(generator.integers(-5, 6, size=(4, 4)).tolist())
N = sp.Matrix(generator.integers(-5, 6, size=(4, 4)).tolist())
say(f"det M = {M.det()}, det N = {N.det()}, det(M N) = {(M * N).det()}")
check((M * N).det() == M.det() * N.det(), "rule 1: det(M N) = det M det N")
check(M.T.det() == M.det(), "rule 2: det of the transpose = det")
```

Two random 4×4 integer matrices as exact sympy matrices (the generator continues where In [10] stopped). Output: det M = -192, det N = 128, det(M N) = -24576, and indeed $-192 \cdot 128 = -24576$. Rules 1 and 2 are checked exactly.

```
swapped = M.copy()
swapped.row_swap(0, 2)  # exchange rows 1 and 3
check(swapped.det() == -M.det(), "rule 3: exchanging two rows flips the sign")
twin = M.copy()
twin[3, :] = M[1, :]  # make row 4 equal to row 2
check(twin.det() == 0, "rule 4: two equal rows give determinant 0")
```

`.copy()` makes an independent copy, so that `M` itself stays unchanged. `row_swap(0, 2)` exchanges rows 1 and 3 (Python's 0 and 2): rule 3. `twin[3, :] = M[1, :]` replaces row 4 by row 2 (`[3, :]` means row 3 in Python's numbering, all columns): rule 4.

```
scaled = M.copy()
scaled[1, :] = 7 * M[1, :]  # multiply row 2 by 7
check(scaled.det() == 7 * M.det(), "rule 5: a row times 7 multiplies det by 7")
added = M.copy()
added[0, :] = M[0, :] + 5 * M[3, :]  # row 1 plus 5 times row 4
check(added.det() == M.det(), "rule 6: adding a multiple of a row keeps det")
D = sp.diag(2, -3, 5, 7)
check(D.det() == 2 * (-3) * 5 * 7, "a diagonal matrix: det = product of the diagonal")
```

Rule 5 (row 2 times 7), rule 6 (5 times row 4 added to row 1), and rule (a) for the diagonal matrix $\mathrm{diag}(2, -3, 5, 7)$. The cell prints seven PASS lines.

In [12], signed areas.

```
def signed_area(corners):
    """The shoelace formula: positive for corners listed counterclockwise."""
    total = 0.0
    for k in range(len(corners)):
        x1, y1 = corners[k]
        x2, y2 = corners[(k + 1) % len(corners)]  # % len: after the last, the first
        total += x1 * y2 - x2 * y1
    return total / 2
```

The shoelace formula of Section 1.21. For each corner k the next corner is number $(k + 1)$ modulo the number of corners, so that the last corner is followed by the first (the remainder `%` turns K into 0).

```
square = np.array([[0, 0], [1, 0], [1, 1], [0, 1]], dtype=float)  # counterclockwise
examples = [np.array([[2.0, 1.0], [1.0, 1.5]]),
            np.array([[1.0, 2.0], [1.0, 0.5]]),
            np.array([[1.0, 2.0], [0.5, 1.0]])]
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.9))
```

The unit square as four rows (corners) in counterclockwise order, three example matrices with the determinants $2 \cdot 1.5 - 1 \cdot 1 = 2$, $1 \cdot 0.5 - 2 \cdot 1 = -1.5$ and $1 \cdot 1 - 2 \cdot 0.5 = 0$, and a figure with three panels.

```
for ax, Mx in zip(axes, examples):
    image = square @ Mx.T  # each corner (a row) multiplied by the matrix
    ax.fill(square[:, 0], square[:, 1], color="0.85", label="unit square")
    ax.fill(image[:, 0], image[:, 1], color="tab:orange", alpha=0.5,
            label="its image")
```

For each matrix: `square @ Mx.T` multiplies every corner by the matrix (a corner is a row here, and a row r times M^T is the transpose of M times the column r, by the transpose rule). `square[:, 0]` is the column of the x values, `square[:, 1]` that of the y values. The square is filled light grey, its image orange and half transparent.

```
    ax.annotate("", xy=Mx[:, 0], xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": "tab:blue", "lw": 2})
    ax.annotate("", xy=Mx[:, 1], xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": "tab:red", "lw": 2})
    ax.text(*(Mx[:, 0] * 1.05), "$M e_1$", color="tab:blue")
    ax.text(*(Mx[:, 1] * 1.05), "$M e_2$", color="tab:red")
```

The two columns of the matrix, Me_1 and Me_2, as a blue and a red arrow from the origin, each labelled a little beyond its tip (the star hands the two coordinates of the point to `ax.text` as its first two arguments).

```
    ax.set_title(f"det M = {np.linalg.det(Mx):+.2f}, area {signed_area(image):+.2f}")
    ax.set_aspect("equal")
    ax.set_xlim(-0.3, 3.4)
    ax.set_ylim(-0.3, 2.9)
    ax.set_xlabel("$x$")
    ax.set_ylabel("$y$")
    ax.legend(loc="upper left", fontsize=8)
fig.tight_layout()
save_figure(fig, "area_and_determinant",
            "The unit square (grey) and its image (orange) under three $2 \\times 2$ "
            ...)
```

The title of each panel shows the determinant and the signed area of the image, which agree. What Figure 01a.3 shows: the area doubled (left); an area 1.5 with the two arrows in exchanged order, a mirrored picture (middle); the square flattened onto a line (right).

```
areas_match = all(abs(signed_area(square @ Mx.T) - np.linalg.det(Mx)) < 1e-12
                  for Mx in examples)
random_matrices = generator.normal(size=(1000, 2, 2))  # 1000 random 2 x 2 matrices
areas_match &= all(abs(signed_area(square @ Mx.T) - np.linalg.det(Mx)) < 1e-12
                   for Mx in random_matrices)
check(areas_match, "the signed area of the image of the unit square is det M "
      "(3 examples and 1000 random matrices)")
```

The signed area must equal the determinant to 10^{-12} for the three examples and for 1000 random 2×2 matrices (`generator.normal` draws **normal random numbers**, numbers around 0 with the bell-shaped distribution; `size=(1000, 2, 2)` gives 1000 matrices). One PASS line.

In [13], the cost of a determinant.

```
sizes = np.arange(1, 17)
leibniz_terms = np.array([math.factorial(int(k)) for k in sizes], dtype=float)
elimination_steps = sizes.astype(float) ** 3 / 3
report("number of terms of the Leibniz formula for n = 16", math.factorial(16))
check(math.factorial(16) == 20922789888000, "16! = 20 922 789 888 000")
```

For $n = 1$ to 16: the $n!$ terms of the Leibniz formula and the $n^3/3$ steps of elimination (`astype(float)` turns the whole numbers into floating-point numbers). The RESULT line prints 20922789888000, and the check compares it with 16! written out.

```
fig, ax = plt.subplots()
ax.semilogy(sizes, leibniz_terms, "o-", label="Leibniz formula: $n!$ terms")
ax.semilogy(sizes, elimination_steps, "s-", label="elimination: about $n^3/3$ steps")
ax.axvline(16, color="black", ls=":", label="$n = 16$: the gamma matrices")
ax.set_xlabel("size $n$ of the matrix")
ax.set_ylabel("number of terms or steps")
ax.set_title("The cost of a determinant")
ax.legend();
```

Both counts against n with a logarithmic vertical axis, and a dotted vertical line at $n = 16$. The semicolon after `ax.legend()` keeps Jupyter from printing the value of the line (Section 1.9).

```
# The two counts for n = 16, rounded, for the caption: 2.1e13 and 1365.
mantissa, exponent = f"{leibniz_terms[-1]:.1e}".split("e")
save_figure(fig, "determinant_cost",
            "The number of terms $n!$ of the Leibniz formula (circles) and the "
            ...
            f"$n = 16$ the Leibniz formula has about ${mantissa} \\times "
            f"10^{{{int(exponent)}}}$ terms against about "
            f"{elimination_steps[-1]:.0f} steps of elimination.")
```

The caption quotes the count for $n = 16$ as computed, not typed: `f"{x:.1e}"` writes 16! as the text 2.1e+13, and `.split("e")` cuts it at the letter e into the **mantissa** 2.1 and the **exponent** +13 (both are still texts). The line `...` stands for three ordinary text lines of the caption; the last three lines are f-strings, and Python joins all the pieces into one caption. In the first of them `{mantissa}` inserts the text 2.1, and `\\times` becomes the LaTeX command `\times`, the multiplication sign (in a Python string a backslash has a special meaning, so one backslash is written as two, Section 1.9). In the second, `10^{{{int(exponent)}}}` reads from the left: `10^`, then `{{`, which stands for one literal brace `{` (a single brace would start an insertion, Section 1.9), then the insertion `{int(exponent)}`, then `}}`, one literal `}`. `int(exponent)` turns the text +13 into the whole number 13 (it drops the plus sign; for a text such as -06 it would also drop the leading zero). So LaTeX receives `10^{13}` and prints 10^{13}: the braces hold the whole exponent, and without them only the first digit would be raised. In the third, `{elimination_steps[-1]:.0f}` inserts the last entry of the elimination counts, $16^3/3 = 1365.33\ldots$, with no digits after the point (`.0f`): 1365. The caption therefore ends "about 2.1×10^{13} terms against about 1365 steps of elimination.". What Figure 01a.4 shows: up to $n = 4$ the two counts are similar; then $n!$ grows far faster than $n^3/3$.

In [14], the gamma matrices of the record.

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
names = record["coordinates"]  # ["x0", "x1", ..., "x7"]
eta = record["eta"]  # the diagonal of the frame metric, in the order x0 ... x7
gamma = [np.array(matrix, dtype=int) for matrix in record["gamma"]]
say(f"coordinates: {names}")
say(f"diagonal of eta: {eta}")
```

The record file is read as text and turned into a Python dictionary (`json.loads`, Section 1.9). Its entry `coordinates` holds the names x0 to x7, `eta` the diagonal of η, and `gamma` the eight matrices,

each a list of 16 rows; they become numpy arrays of whole numbers. Output: coordinates: ['x0', 'x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7'] and diagonal of eta: [1, 1, 1, 1, -1, -1, -1, -1].

```
signed_permutation = True
for g_a in gamma:
    signed_permutation &= set(np.unique(g_a).tolist()) <= {-1, 0, 1}
    signed_permutation &= ((g_a != 0).sum(axis=1) == 1).all()  # one per row
    signed_permutation &= ((g_a != 0).sum(axis=0) == 1).all()  # one per column
```

For each matrix three conditions: `np.unique` lists the different entries, and `set(...) <= {-1, 0, 1}` asks whether they are among $-1, 0, 1$ (`<=` between sets means "is contained in"); `(g_a != 0)` is a table of True and False, and `.sum(axis=1)` counts the True values in each row (`axis=0`: in each column), which must be 1 everywhere. Together: a signed permutation matrix.

```
check(len(gamma) == 8 and all(g_a.shape == (16, 16) for g_a in gamma),
      "the record holds eight 16 x 16 gamma matrices")
check(signed_permutation, "every gamma matrix is a real signed permutation matrix",
      record="Revision/algebra/reports/python-algebra.json, "
             "check reality_signed_permutations")
```

Two checks: eight matrices of size 16×16, and the signed-permutation property; the second reproduces the record's check `reality_signed_permutations`, so `check` prints a second line naming it. Two PASS lines.

In [15], determinants and squares of the gamma matrices.

```
identity16 = np.eye(16, dtype=int)  # the 16 x 16 identity matrix
all_det_one = True
squares_right = True
for name, eta_aa, g_a in zip(names, eta, gamma):
    order = [int(np.flatnonzero(row)[0]) for row in g_a]  # column of each nonzero
    signs = [int(g_a[i, order[i]]) for i in range(16)]  # the nonzero entries
    det_by_structure = sign(order) * math.prod(signs)
```

`np.eye(16, dtype=int)` is the 16×16 identity. For each matrix: `np.flatnonzero(row)` lists the places of the nonzero entries of a row, and the first (only) one is its column, so `order` is the permutation σ; `signs` are the sixteen nonzero entries s_i; and the determinant is $\operatorname{sign}(\sigma)\, s_1 \cdots s_{16}$ by formula (c) of Section 1.20 (`math.prod` multiplies the entries of a list).

```
    det_exact = sp.Matrix(g_a.tolist()).det()
    det_numpy = np.linalg.det(g_a)
    say(f"gamma^({name}): eta {eta_aa:+d}, {signs.count(-1)} entries -1, "
        f"permutation sign {sign(order):+d}, det {det_by_structure:+d}, "
        f"sympy {det_exact}, numpy {det_numpy:.3f}")
    all_det_one &= det_by_structure == det_exact == 1 and abs(det_numpy - 1) < 1e-9
    squares_right &= (g_a @ g_a == eta_aa * identity16).all()
```

The same determinant exactly with sympy and in floating point with numpy; a line per matrix; then the two conditions: all three determinants equal 1, and $\gamma^a\gamma^a = \eta_{aa} I$. The output reads, for

$\gamma^{(x_0)}$ (the first line), 0 entries -1, permutation sign +1, det +1, sympy 1, numpy 1.000, and for $\gamma^{(x_1)}$ to $\gamma^{(x_7)}$ 8 entries -1 with the same determinants.

```
check(all_det_one, "every gamma matrix has determinant 1 (three ways)")
check(squares_right, "gamma^a gamma^a = eta_aa times the identity for every a",
      record="Revision/algebra/reports/python-algebra.json, check "
             "clifford_relation (its 8 relations with equal indices)")
```

Two PASS lines; the second reproduces the part of the record's check `clifford_relation` with equal indices.

In [16], heat maps of two gamma matrices.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
ticks = [str(k) for k in range(1, 17)]
pictures = [(gamma[1], "$\\gamma^{(x_1)}$"), (gamma[4], "$\\gamma^{(x_4)}$"),
            (gamma[4] @ gamma[4], "$\\gamma^{(x_4)} \\gamma^{(x_4)} = -I$")]
for ax, (matrix, title) in zip(axes, pictures):
    image = heat_map(ax, matrix, title, labels=ticks, annotate=False)
    ax.tick_params(labelsize=6)
```

Three panels: $\gamma^{(x_1)}$ (Python's `gamma[1]`: the list `gamma` holds the matrices in the author's order $x_0, \ldots, x_7$, so the index of $\gamma^{(x_k)}$ is k), $\gamma^{(x_4)}$ (`gamma[4]`) and its square, drawn with the helper of In [5] without written values; the row and column labels 1 to 16 are made small.

```
fig.colorbar(image, ax=axes, shrink=0.75, label="entry", ticks=[-1, 0, 1])
save_figure(fig, "gamma_matrices",
            "Heat maps of two of the eight real $16 \\times 16$ gamma matrices of "
            ...)
```

`fig.colorbar` adds a bar that explains the colours, shared by the three panels (`ax=axes`), 75 per cent as tall as them, with the marks -1, 0, 1. What Figure 01a.5 shows: in every row and every column of a gamma matrix exactly one coloured square; the square of the time-like $\gamma^{(x_4)}$ has blue squares (-1) on the diagonal and nothing else: $-I$.

In [17], the author's metric and its determinant, from the record.

```
a4 = sp.Symbol("a4", real=True)  # the value of the function a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of the function b4(x4) at one time
z = sp.Symbol("z", positive=True)  # z = 6 H x0, between 0 and pi/2
s = sp.sin(z) ** sp.Rational(1, 3)  # sin(z) to the power 1/3
g_typed = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * s] * 3 + [sp.Integer(-1)]
           + [-b4 * sp.exp(-2 * a4) * s] * 3)  # the author's order x0, x1, ..., x7
```

Symbols for the value of a_4 (any real number), for the value of b_4 (positive) and for z (positive), $s = \sin^{1/3} z$, and the author's metric typed as the list of its eight diagonal entries in the author's order $x_0, \ldots, x_7$, as in Section 1.1 (`sp.Integer(-1)` is the exact whole number -1).

```
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))


def from_record(text):
    """The record's Mathematica text as a sympy expression."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "sin(z)")
    text = text.replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "z": z, "E": sp.E})
```

The record file, and the translation of its Mathematica texts into sympy, as in Notebook 01f, In [11] (Section 1.9), except that here $\sin(6Hx_0)$ becomes `sin(z)` and $\cot(6Hx_0)$ becomes `cot(z)` with the symbol z.

```
g_record = [from_record(text) for text in curvature["metricDiagonal"]]
check(len(g_record) == 8 and
      all(sp.simplify(x - y) == 0 for x, y in zip(g_record, g_typed)),
      "the record's metric diagonal equals the author's metric typed here")
det_g = sp.Mul(*g_typed)  # the product of the 8 diagonal entries
say(f"product of the diagonal entries, as sympy writes it: {det_g}")
```

The record's eight entries are compared one by one with the typed ones (and there must be eight). `det_g` is the product of the diagonal, the determinant by rule (a). Output: sympy writes it `b4**3*si n(z)**2*cot(z)**2`, already without a_4.

```
check(sp.simplify(det_g - b4 ** 3 * sp.cos(z) ** 2) == 0,
      "det g = b4^3 cos(z)^2 for every a4: the e^(6 a4) of space and the e^(-6 a4) "
      "of the extra times cancel",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "sqrt_abs_det_g")
root_record = from_record(curvature["sqrtAbsDetG"])
say(f"the record's square root of |det g|: {root_record}")
check(sp.simplify(root_record - b4 ** sp.Rational(3, 2) * sp.cos(z)) == 0 and
      sp.simplify(root_record ** 2 - det_g) == 0,
      "the record's sqrtAbsDetG = b4^(3/2) sin z cot z = b4^(3/2) cos z, and its "
      "square is det g",
      record="Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG")
```

The check $\det g = b_4^3 \cos^2 z$ (Section 1.21), which reproduces the record's check `sqrt_abs_det_g`; then the record's `sqrtAbsDetG`, printed as `b4**(3/2)*sin(z)*cot(z)`, which must simplify to $b_4^{3/2} \cos z$ and square to $\det g$. Three PASS lines, two with a second line naming the record.

In [18], the metric in numbers.

```
metric_numbers = sp.lambdify((a4, b4, z), g_typed, "numpy")  # a4, b4, z -> 8 numbers
z_values = np.linspace(0.01, np.pi / 2 - 0.01, 300)  # inside (0, pi/2)
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6))
g_point = np.diag(np.array(metric_numbers(0.5, 1.0, 0.9), dtype=float))
coordinate_labels = [f"$x_{k}$" for k in range(8)]  # x with subscripts 0 ... 7
```

```
heat_map(left, g_point, "metric $g$ at $a_4 = 0.5$, $b_4 = 1$, $z = 0.9$",
         labels=coordinate_labels, digits=2, skip_zeros=True)
left.tick_params(labelsize=8)
```

`sp.lambdify` turns the exact list of formulas into a fast numpy function of a_4, b_4 and z that returns the eight numbers. `z_values` are 300 values inside $(0, \pi/2)$. On the left, the 8×8 metric at $a_4 = 0.5$, $b_4 = 1$, $z = 0.9$ (`np.diag` of a list builds the diagonal matrix) is drawn as a heat map with the coordinate names x_0 to x_7 as labels and two digits, the zero entries left blank.

```
largest_error = 0.0
for a4_value, b4_value, style in [(-1.0, 1.0, "-"), (0.0, 0.5, "--"), (1.0, 2.0, ":")]:
    dets = np.array([np.linalg.det(np.diag(np.array(metric_numbers(a4_value, b4_value,
                                                                   zv), dtype=float)))
                     for zv in z_values]) / b4_value ** 3
    largest_error = max(largest_error, float(np.max(np.abs(dets - np.cos(z_values)
                                                           ** 2))))
    right.plot(z_values, dets, style, lw=2,
               label=f"det g / $b_4^3$, $a_4 = {a4_value:+.0f}$, $b_4 = {b4_value:g}$")
```

For the three pairs $(a_4, b_4) = (-1, 1)$, $(0, 0.5)$ and $(1, 2)$ (solid, dashed and dotted lines): at each of the 300 values of z the 8×8 matrix is built and its determinant computed by numpy's elimination, then divided by b_4^3; `largest_error` keeps the largest difference from $\cos^2 z$; the 300 values are drawn against z.

```
right.plot(z_values, np.cos(z_values) ** 2, color="black", lw=0.8,
           label="$\\cos^2 z$")
right.set_xlabel("$z = 6 H x_0$")
right.set_ylabel("$\\det g / b_4^3$")
right.set_title("$\\det g / b_4^3$ does not depend on $a_4$")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "metric_and_determinant",
            "Left: heat map of the author's $8 \\times 8$ metric at $a_4 = 0.5$, "
            ...)
```

The curve $\cos^2 z$ as a thin black line, labels, a legend in a small font, and the figure. What Figure 01a.6 shows: on the left only the diagonal is coloured, red for x_0, x_1, x_2, x_3 and blue for x_4 to x_7; on the right the three curves of $\det g / b_4^3$ and $\cos^2 z$ lie on top of each other.

```
report("largest difference between det g / b4^3 and cos(z)^2 at 900 points",
       f"{largest_error:.1e}")
check(largest_error < 1e-12, "numpy: det g = b4^3 cos(z)^2 at 300 values of z for "
      "each of (a4, b4) = (-1, 1), (0, 0.5), (1, 2)")
```

The RESULT line prints 2.9×10^{-15}, rounding errors only, and the check requires less than 10^{-12} (COMPUTED).

In [19], the last check.

```
figure_names = ["01a_1_matrix_product.png", "01a_2_permutation_matrices.png",
                "01a_3_area_and_determinant.png", "01a_4_determinant_cost.png",
                "01a_5_gamma_matrices.png", "01a_6_metric_and_determinant.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

The six figure files must exist; the last line is ALL 30 CHECKS PASSED (notebook 01a). The 30 checks are: 1 in In [2], 1 in In [3], 4 in In [4], 1 in In [6], 1 in In [7], 3 in In [9], 1 in In [10], 7 in In [11], 1 in In [12], 1 in In [13], 2 in In [14], 2 in In [15], 3 in In [17], 1 in In [18] and 1 in In [19].

1.26 Index notation and the summation convention

Why indices. Every formula of the theory involves the eight coordinates at once. The field equation of the Revision record, $\gamma^\mu D_\mu \Psi = (m + U'(S))\Psi$ (Chapter 7), is a sum of eight terms, one for each coordinate; the Clifford relation of the gamma matrices is one line that stands for 64 equations between 16×16 matrices. **Index notation** writes such formulas compactly: a letter stands for each of the coordinates, and a rule says which letters are summed.

Indices and their positions. An **index** is a letter that stands for any one of the eight coordinates $x_0, \ldots, x_7$. The course uses the Latin letters a, b, c, d for the directions of the frame (the eight directions with the frame metric η, below) and the Greek letters μ, ν, λ for the coordinates themselves; both run over the eight values 0 to 7, the numbers of the author's coordinates. A vector v has the components $v^0, \ldots, v^7$, written with an **upper index**: the component of v along x_3 is v^3, which Python stores as `v[3]` (the author and Python both count from 0). An upper index is a label, not a power: v^3 is a component, and its square is written $(v^3)^2$. A second list of eight numbers that belongs to the same vector, $v_0, \ldots, v_7$, carries a **lower index**; Section 1.27 says how one list is made from the other. In index formulas the coordinates themselves are written x^μ with an upper index, and a small step along them dx^μ; the author's names $x_0, \ldots, x_7$ denote the same coordinates (the lower number is part of the name, as in Section 1.1), and the index of a coordinate is never lowered with a metric.

Free and dummy indices. Take a table $M^a{}_b$ (row a, column b) and a vector v^b. In

$$w^a = M^a{}_b v^b = \sum_b M^a{}_b v^b$$

the index a is **free**: it appears once in every term, and the formula holds for each of its values, one equation for each a. The index b is a **dummy**: it appears twice in one term, once up and once down, and it is summed over all its values. The **summation convention** (introduced by Einstein) is the rule that such a repeated pair is summed without writing $\sum$. A dummy index may be renamed, $M^a{}_b v^b = M^a{}_c v^c$, because the name of the letter in a sum does not change its value; a free index may not. In a correct equation every term has the same free indices in the same positions. A formula with k free indices over eight coordinates stands for 8^k equations: 1, 8, 64, 512 for $k = 0, 1, 2, 3$.

An example (PROVED by arithmetic). With the table M of rows $(1, 2, 0)$, $(0, 1, 1)$, $(1, 3, 1)$ and $v = (2, -1, 3)$ (three index values only, to keep it small):

$$w^1 = 1\cdot 2 + 2\cdot(-1) + 0\cdot 3 = 0, \quad w^2 = 0\cdot 2 + 1\cdot(-1) + 1\cdot 3 = 2, \quad w^3 = 1\cdot 2 + 3\cdot(-1) + 1\cdot 3 = 2$$

(for each free value a, the sum over the dummy b). Two numbers without any free index: the **trace** $M^a{}_a = M^1{}_1 + M^2{}_2 + M^3{}_3 = 1+1+1 = 3$, and, with $u_a = (1,0,2)$, the number $u_a M^a{}_b v^b = u_a w^a = 1\cdot 0 + 0\cdot 2 + 2\cdot 2 = 4$ (two dummy pairs, summed; first over b, which gives w^a, then over a). Setting an upper and a lower index equal and summing over it is called a **contraction**; it removes two free indices.

The Kronecker delta. With one upper and one lower index, $\delta^a{}_b$ (also written δ^a_b) is 1 if $a = b$ and 0 otherwise; as a table it is the identity matrix. Three rules (PROVED):

$$\delta^a{}_b v^b = v^a, \qquad \delta^a{}_b\, \delta^b{}_c = \delta^a{}_c, \qquad \delta^a{}_a = 8$$

(in $\sum_b \delta^a{}_b v^b$ only the term $b = a$ is not zero, and it is $1\cdot v^a$; the second rule is the first with the vector replaced by the column c of δ; the third adds $\delta^0{}_0 + \cdots + \delta^7{}_7 = 1 + \cdots + 1$, eight ones). Contracting with δ renames an index.

1.27 The frame metric: lowering, raising and squared lengths

The frame metric. The **frame metric** of the author's spacetime is the diagonal table

$$\eta_{ab} = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1) \quad \text{in the order } x_0,\ldots,x_7.$$

It records only which directions are space-like (+1: the hidden x_0 and x_1, x_2, x_3) and which are time-like (−1: the time x_4 and the three extra times x_5, x_6, x_7). The Revision record stores it in `Revision/algebra/gammas.json` and states it in the report `Revision/algebra/reports/python-algebra.json`, check `coordinate_map`. Its inverse, written with upper indices η^{ab}, is the diagonal table of the inverse entries, $1/(+1) = +1$ and $1/(-1) = -1$: the same entries. Hence (PROVED)

$$\eta^{ab}\eta_{bc} = \delta^a{}_c$$

(the product of two diagonal tables is the diagonal table of the products of their entries, and each product is $(\pm 1)(\pm 1) = 1$).

Lowering and raising (PROVED). The lower list of a vector is made with η:

$$v_a = \eta_{ab} v^b = \eta_{aa} v^a \quad \text{(no sum in the last expression)}$$

(only the term $b = a$ of the sum survives, because η is diagonal). So **lowering** an index keeps the four space-like components and changes the sign of the four time-like ones: for $v^a = (1,2,\ldots,8)$ (that is $v^0 = 1, v^1 = 2, \ldots, v^7 = 8$), $v_a = (1,2,3,4,-5,-6,-7,-8)$. **Raising** with η^{ab} undoes it: $\eta^{ab} v_b = \eta^{ab}\eta_{bc} v^c = \delta^a{}_c v^c = v^a$ (insert the lowered list; the inverse rule; the delta renames).

The squared length. The **squared length** of a vector is

$$Q(v) = \eta_{ab} v^a v^b = v^a v_a = (v^0)^2 + (v^1)^2 + (v^2)^2 + (v^3)^2 - (v^4)^2 - (v^5)^2 - (v^6)^2 - (v^7)^2$$

(the sum over b is the lowered list; in the double sum only the terms $a = b$ survive because η is diagonal). A vector $v \neq 0$ is **space-like** if $Q(v) > 0$, **time-like** if $Q(v) < 0$ and **light-like** (or **null**) if $Q(v) = 0$. Five examples (PROVED by arithmetic): the unit vector along x_1 has $Q = +1$ (space-like); the unit vector along x_5 has $Q = -1$ (time-like); their sum has $Q = 1 - 1 = 0$ (light-like although it is not zero); $(1, 1, \ldots, 1)$ has $Q = 4 - 4 = 0$ (light-like); and $v = (1, 2, \ldots, 8)$ has

$$Q = (1 + 4 + 9 + 16) - (25 + 36 + 49 + 64) = 30 - 174 = -144$$

(the squares of the space-like components v^0, v^1, v^2, v^3 minus those of the time-like components $v^4, \ldots, v^7$): time-like. The **signature** of η is $(4, 4)$: four entries $+1$ and four entries -1 (Section 1.36 gives the general definition). The boost of Section 1.13 keeps the squared length of the pair (t, x) with $\eta = \mathrm{diag}(-1, +1)$; written with tables, the squared length of Λv is $(\Lambda v)^T \eta (\Lambda v) = v^T (\Lambda^T \eta \Lambda) v$ (the transpose rule of Section 1.18), so $\Lambda^T \eta \Lambda = \eta$ says that it equals $v^T \eta v$ for every v.

Half of all random vectors are space-like (PROVED, with one ASSUMED fact of probability). Draw the eight components of a vector as independent **normal random numbers** (numbers around 0 whose histogram is the bell curve $e^{-x^2/2}/\sqrt{2\pi}$). Exchange its four space-like components (v^0, v^1, v^2, v^3) with its four time-like components (v^4, v^5, v^6, v^7). The new vector v' has

$$Q(v') = (v^4)^2 + (v^5)^2 + (v^6)^2 + (v^7)^2 - (v^0)^2 - (v^1)^2 - (v^2)^2 - (v^3)^2 = -Q(v)$$

(every square that was added is now subtracted and every square that was subtracted is now added). Because all eight components are drawn in the same way, the exchanged vector is exactly as likely as the original one; so $Q > 0$ and $Q < 0$ are equally likely, and $Q = 0$ exactly has probability 0 (ASSUMED from probability theory). Therefore, **in the signature (4,4) exactly half of all random vectors are space-like.** In ordinary spacetime with three space directions and one time, signature (3,1), $Q = (v^1)^2 + (v^2)^2 + (v^3)^2 - (v^4)^2$ adds three squares and subtracts one, and most random vectors are space-like; integral calculus (not shown here, ASSUMED) gives the fraction $1/2 + 1/\pi \approx 0.818$. Notebook 01e, In [6], draws 200,000 random vectors and measures the fractions 0.5011 and 0.8177 (COMPUTED). A measured fraction p from N tries is expected to scatter around the true value by about one **standard deviation** $\sqrt{p(1-p)/N}$ (ASSUMED from probability), here $\sqrt{0.25/200000} \approx 0.0011$ for the first; both measurements lie within 4 standard deviations of $1/2$ and of $1/2 + 1/\pi$ (Figure 01e.2).

1.28 Symmetric and antisymmetric arrays; commutators; the Clifford relation

The symmetric and the antisymmetric part. A square array is **symmetric** if $S_{ab} = S_{ba}$ and **antisymmetric** if $A_{ab} = -A_{ba}$; the diagonal of an antisymmetric array is 0, because $A_{aa} = -A_{aa}$ forces $A_{aa} = 0$. Every square array is the sum of a symmetric and an antisymmetric part:

$$M_{ab} = S_{ab} + A_{ab}, \qquad S_{ab} = \tfrac{1}{2}(M_{ab} + M_{ba}), \qquad A_{ab} = \tfrac{1}{2}(M_{ab} - M_{ba})$$

(adding the two parts gives back M_{ab}, because the terms M_{ba} cancel; exchanging a and b leaves S unchanged and changes the sign of A).

Theorem (PROVED). If $S_{ab} = S_{ba}$ and $A^{ab} = -A^{ba}$, then $S_{ab}A^{ab} = 0$. *Proof*, line by line:

$$S_{ab}A^{ab} = S_{ba}A^{ba}$$

(both sides are the same double sum over all pairs; only the names of the two dummy indices were exchanged, which does not change a sum),

$$S_{ba}A^{ba} = S_{ab}\,(-A^{ab})$$

(the symmetry of S and the antisymmetry of A), so $S_{ab}A^{ab} = -S_{ab}A^{ab}$, and a number that equals its own negative is 0. In words: the term (a, b) and the term (b, a) cancel, and the diagonal terms are 0. Raising both indices keeps an array antisymmetric: $A^{ab} = \eta^{ac}\eta^{bd}A_{cd}$, and $A^{ba} = \eta^{bc}\eta^{ad}A_{cd} = \eta^{ac}\eta^{bd}A_{dc} = -\eta^{ac}\eta^{bd}A_{cd} = -A^{ab}$ (the definition with a and b exchanged; the dummies c and d renamed into each other and the factors re-ordered; the antisymmetry of A_{cd}).

A consequence used again and again (PROVED). For a vector with ordinary (**commuting**) components, $v^a v^b = v^b v^a$ is symmetric in a and b, so by the theorem $v^a v^b A_{ab} = 0$ for every antisymmetric A, and $v^a v^b M_{ab} = v^a v^b S_{ab}$: only the symmetric part of an array counts in a squared expression. Later in the course (Chapter 7) the components of the field dirac16complex **anticommute**, $\theta_a\theta_b = -\theta_b\theta_a$; then the products are antisymmetric, and only the antisymmetric part of an array counts.

Commutators and anticommutators. For two square matrices P and R the **commutator** and the **anticommutator** are

$$[P, R] = PR - RP, \qquad \{P, R\} = PR + RP.$$

They split a product into two parts, $PR = \frac{1}{2}[P, R] + \frac{1}{2}\{P, R\}$ (the two halves add up to $\frac{1}{2}(PR - RP + PR + RP) = PR$), and directly from the definitions $[R, P] = -[P, R]$ and $\{R, P\} = \{P, R\}$. P and R **commute** if $[P, R] = 0$ and **anticommute** if $\{P, R\} = 0$, that is $PR = -RP$.

The Clifford relation. The eight gamma matrices of the Revision record obey

$$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}I \qquad (a, b = x_0, \ldots, x_7),$$

the **Clifford relation**. Chapter 4 derives it, and the Revision record checks it in the check `clifford_relation` of its report `Revision/algebra/reports/python-algebra.json`. With two free indices it stands for $8 \cdot 8 = 64$ matrix equations. For $a = b$ it says $(\gamma^a)^2 = \eta^{aa}I$, $+I$ for a space-like and $-I$ for a time-like direction (Section 1.21). For $a \neq b$ it says $\{\gamma^a, \gamma^b\} = 0$: different gamma matrices anticommute, and their commutator is $[\gamma^a, \gamma^b] = \gamma^a\gamma^b - \gamma^b\gamma^a = 2\gamma^a\gamma^b$. The product of two signed permutation matrices is again a signed permutation matrix (each row of the product picks one row of the second factor, with a sign), so this commutator has exactly 16 nonzero entries, ± 2.

Symmetric and antisymmetric gamma matrices (PROVED). A real matrix O with $O^T O = I$ is called **orthogonal**; then $O^T = O^{-1}$. Every signed permutation matrix is orthogonal: the entry (j, k) of $O^T O$ is $\sum_i O_{ij}O_{ik}$, the "dot product" of the columns j and k; for $j = k$ it is the square of the one nonzero entry ± 1 of the column, which is 1; for $j \neq k$ the nonzero entries of the two columns lie in different rows (a row holds only one nonzero entry), so every product $O_{ij}O_{ik}$ is 0.

For a gamma matrix, $(\gamma^a)^2 = \eta^{aa} I$ says $\gamma^a(\eta^{aa}\gamma^a) = I$ (multiply by the number $\eta^{aa} = \pm 1$, whose square is 1), so $(\gamma^a)^{-1} = \eta^{aa}\gamma^a$. Together:

$$(\gamma^a)^T = (\gamma^a)^{-1} = \eta_{aa}\gamma^a \quad \text{(no sum)},$$

the gamma matrices of the space-like directions x_0, x_1, x_2, x_3 are **symmetric** and those of the time-like directions $x_4, \ldots, x_7$ are **antisymmetric**. The record states this in its check `symmetry_pattern`.

Three contractions (PROVED). Lowering the index of the gamma matrices gives $\gamma_a = \eta_{ab}\gamma^b$, which is $+\gamma^a$ for a space-like and $-\gamma^a$ for a time-like direction. *First*, $\gamma^a\gamma_a = 8I$:

$$\gamma^a\gamma_a = \sum_a \gamma^a \eta_{ab}\gamma^b = \sum_a \eta_{aa}(\gamma^a)^2$$

(insert γ_a; only $b = a$ survives because η is diagonal),

$$\sum_a \eta_{aa}(\gamma^a)^2 = \sum_a \eta_{aa}\eta^{aa} I = \sum_a 1 \cdot I = 8I$$

(the Clifford relation with $a = b$; then $\eta_{aa}\eta^{aa} = (\pm 1)^2 = 1$ for each of the eight values of a). Each time-like direction contributes $(\gamma^a)^2 = -I$, but its $\eta_{aa} = -1$ turns the contribution into $+I$. *Second*, $\gamma^a\gamma^b\gamma_a = -6\gamma^b$. First lower one index of the Clifford relation: multiplying $\gamma^b\gamma^c + \gamma^c\gamma^b = 2\eta^{bc}I$ by η_{ca} and summing over c gives

$$\gamma^b\gamma_a + \gamma_a\gamma^b = 2\delta^b{}_a I, \qquad \text{that is} \qquad \gamma^b\gamma_a = -\gamma_a\gamma^b + 2\delta^b{}_a I$$

($\gamma^c\eta_{ca} = \gamma_a$ by the definition of lowering, and $\eta^{bc}\eta_{ca} = \delta^b{}_a$; then move one term to the other side). Then

$$\gamma^a\gamma^b\gamma_a = \gamma^a\big(-\gamma_a\gamma^b + 2\delta^b{}_a I\big) = -(\gamma^a\gamma_a)\gamma^b + 2\gamma^b = -8\gamma^b + 2\gamma^b = -6\gamma^b$$

(insert the lowered relation; multiply out, where $\gamma^a\delta^b{}_a = \gamma^b$ because only $a = b$ survives; the first contraction; collect). In n dimensions the same steps give $(2-n)\gamma^b$. *Third*, $\mathrm{tr}(\gamma^a\gamma^b) = 16\eta^{ab}$: since $\mathrm{tr}(\gamma^a\gamma^b) = \mathrm{tr}(\gamma^b\gamma^a)$ (Section 1.18), $\mathrm{tr}(\gamma^a\gamma^b) = \frac{1}{2}\mathrm{tr}(\gamma^a\gamma^b + \gamma^b\gamma^a) = \frac{1}{2}\cdot 2\eta^{ab}\,\mathrm{tr}\,I = 16\eta^{ab}$ (the average of two equal numbers; the Clifford relation; $\mathrm{tr}\,I = 16$ for the 16×16 identity). Notebook 01e, In [10] to In [13], checks all of this with the matrices of the record (Figures 01e.4 and 01e.5).

1.29 The author's metric in index notation

The metric and its inverse. The author's metric $g_{\mu\nu}$ is the diagonal table of Section 1.1, with the entries $g_{00} = \cot^2 z$, $g_{11} = g_{22} = g_{33} = e^{2a_4}s$, $g_{44} = -1$ and $g_{55} = g_{66} = g_{77} = -b_4 e^{-2a_4}s$, where $s = \sin^{1/3} z$ and $z = 6Hx_0$; the record stores it in `Revision/gkd_lovelock/results/curvature.json`, key `metricDiagonal`. Its inverse $g^{\mu\nu}$ is the diagonal table with the entries $g^{\mu\mu} = 1/g_{\mu\mu}$, and $g^{\mu\nu}g_{\nu\lambda} = \delta^\mu{}_\lambda$ (PROVED: the product of two diagonal tables, with $(1/g_{\mu\mu})\,g_{\mu\mu} = 1$); the record

checks it in `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `metric_inverse`. The metric lowers and raises coordinate indices as η does for frame indices: $dx_\mu = g_{\mu\nu}dx^\nu$.

The line element (PROVED). The squared length of a small step dx^μ in the coordinates is the **line element**

$$ds^2 = g_{\mu\nu}\,dx^\mu dx^\nu = \sum_\mu g_{\mu\mu}\,(dx^\mu)^2 = dx^\mu dx_\mu$$

(a double sum, in which only the terms $\nu = \mu$ survive because g is diagonal; the sum over ν is the lowered list). For the step with the same small size ϵ in all eight coordinates, $dx^\mu = \epsilon$:

$$ds^2 = \epsilon^2\big(\cot^2 z + 3e^{2a_4}s - 1 - 3b_4e^{-2a_4}s\big)$$

(the eight diagonal entries, each times ϵ^2: the hidden direction, three equal space entries, the time, three equal extra-time entries). From here on take $b_4 = 1$, the background of the Revision record:

$$= \epsilon^2\big(3s\,(e^{2a_4} - e^{-2a_4}) - 1 + \cot^2 z\big) = \epsilon^2\big(6s\sinh(2a_4) - 1 + \cot^2 z\big)$$

(insert $b_4 = 1$ and take out the common factor $3s$; then $e^x - e^{-x} = 2\sinh x$ with $x = 2a_4$, the definition of Section 1.13). As a_4 grows, the term of the three space directions grows like e^{2a_4} and the term of the three exponentially deflating extra times shrinks towards 0 like e^{-2a_4}.

Where the step changes its kind (PROVED). Take $b_4 = 1$. For fixed z, ds^2 grows when a_4 grows, because $6s > 0$ and sinh is an increasing function (its derivative cosh is positive; the derivative of $6s\sinh(2a_4)$ with respect to a_4 is $12s\cosh(2a_4) > 0$). Since sinh takes every real value exactly once, its inverse function arcsinh exists, and $ds^2 = 0$ exactly at

$$a_4^*(z) = \tfrac{1}{2}\operatorname{arcsinh}\Big(\frac{1 - \cot^2 z}{6s}\Big)$$

(set $ds^2 = 0$; solve for $\sinh(2a_4) = (1 - \cot^2 z)/(6s)$; apply arcsinh; divide by 2). For $a_4 < a_4^*(z)$ the step is time-like, for $a_4 > a_4^*(z)$ space-like: as space inflates and the extra times deflate, the same coordinate step turns from time-like to space-like. Notebook 01e, In [14] and In [15], checks the formula exactly and the sign change on a grid of 301×300 points (Figure 01e.6; COMPUTED).

1.30 Example: index notation in the author's spacetime

Notebook 01e puts Sections 1.26 to 1.29 to work. It computes sums with free and dummy indices with loops and with `einsum`; reads η from the Revision record, lowers and raises an index and checks $\eta^{ab}\eta_{bc} = \delta^a{}_c$ and $\delta^a{}_a = 8$; sorts vectors into space-like, time-like and light-like ones and measures with 200,000 random vectors that half of them are space-like in the signature (4,4) but about 82 per cent in the signature (3,1); checks the theorem on symmetric and antisymmetric arrays exactly with whole numbers; checks all 64 Clifford relations of the record's gamma matrices, their symmetry pattern and the three contractions; and writes the author's line element as a sum over indices and shows how the inflation of space and the deflation of the extra times change it. It ends with the line ALL 25 CHECKS PASSED (notebook 01e).

1.31 How to run Notebook 01e

Step 1. What this notebook does and what it needs.

Notebook 01e (Index notation, the summation convention and the metric) is the file `Revision /textbook/notebooks/01e_index_notation.ipynb` of the repository Dirac_claude. It teaches free and dummy indices and the summation convention with numpy einsum, reads the frame metric eta of the author's coordinates x0 to x7 from the Revision record, lowers and raises indices with it, sorts vectors into space-like, time-like and light-like ones in the signature (4,4) and compares random vectors with those of ordinary four-dimensional spacetime, proves that a symmetric array contracted with an antisymmetric one gives zero, checks the Clifford relation of the eight real gamma matrices of the Revision record as one index formula for all 64 pairs, derives and checks the contractions gamma^a gamma_a = 8 and gamma^a gamma^b gamma_a = -6 gamma^b, and writes the squared length of a coordinate step in the author's metric, whose three extra times deflate exponentially, as a sum over indices, with heat maps and plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
```

```
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01e_index_notation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a

number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01e_index_notation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01e_index_notation.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01e.captions.json`
- `Revision/textbook/figures/01e_1_eta_and_delta.png`
- `Revision/textbook/figures/01e_2_squared_lengths.png`
- `Revision/textbook/figures/01e_3_symmetric_antisymmetric.png`
- `Revision/textbook/figures/01e_4_clifford_tables.png`
- `Revision/textbook/figures/01e_5_gamma_contractions.png`
- `Revision/textbook/figures/01e_6_step_length.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 25 CHECKS PASSED (notebook 01e)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01e_index_notation.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01e_index_notation.ipynb
python -m nbconvert --execute --inplace 01e_index_notation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/algebra or Revision/gkd_lovelock: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

1.32 Notebook 01e: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01e_index_notation.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Every formula of the course is written in *index notation*: a letter such as $a$
or $\mu$ stands for each of the eight coordinates $x_0, \dots, x_7$ of the
author's spacetime, and a letter that appears twice is summed. This notebook
teaches that notation from zero and uses it on the objects of the theory. It

- shows free and dummy indices and the summation convention with numpy's
  function `einsum`, which reads the convention literally;
- reads the frame metric $\eta$ from the Revision record, lowers and raises
  indices with it, and checks $\eta^{ab}\eta_{bc} = \delta^a_c$ and
  $\delta^a_a = 8$;
- computes squared lengths $\eta_{ab} v^a v^b$ and sorts vectors into
  space-like, time-like and light-like ones; with four space-like and four
  time-like directions exactly half of all random vectors are space-like, unlike
  in ordinary four-dimensional spacetime;
- proves that a symmetric array contracted with an antisymmetric one gives 0;
- checks the Clifford relation $\gamma^a \gamma^b + \gamma^b \gamma^a =
  2\eta^{ab} I$ of the eight real gamma matrices of the Revision record: one
  index formula that stands for 64 matrix equations;
- derives and checks the contractions $\gamma^a \gamma_a = 8 I$ and
  $\gamma^a \gamma^b \gamma_a = -6 \gamma^b$;
- writes the squared length of a small coordinate step in the author's metric
  as a sum over indices, and shows how the growth of space and the exponential
  deflation of the three extra times change it.

Every check prints a line that starts with PASS; where a check repeats a result
of the Revision record, a second line names the record file and its check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01e (Index notation, the summation convention and the metric) is the file
`Revision/textbook/notebooks/01e_index_notation.ipynb` of the repository Dirac_claude. It
teaches free and dummy indices and the summation convention with numpy einsum, reads the
frame metric eta of the author's coordinates x0 to x7 from the Revision record, lowers
and raises indices with it, sorts vectors into space-like, time-like and light-like ones
in the signature (4,4) and compares random vectors with those of ordinary
```

four-dimensional spacetime, proves that a symmetric array contracted with an antisymmetric one gives zero, checks the Clifford relation of the eight real gamma matrices of the Revision record as one index formula for all 64 pairs, derives and checks the contractions gamma^a gamma_a = 8 and gamma^a gamma^b gamma_a = -6 gamma^b, and writes the squared length of a coordinate step in the author's metric, whose three extra times deflate exponentially, as a sum over indices, with heat maps and plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of

‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 01e_index_notation.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 15 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.

**Step 6. Or run the notebook without a browser (headless).**

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 01e_index_notation.ipynb

This runs every cell from the top to the bottom and saves the results into the notebook
```

file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01e_index_notation.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01e.captions.json‘
- ‘Revision/textbook/figures/01e_1_eta_and_delta.png‘
- ‘Revision/textbook/figures/01e_2_squared_lengths.png‘
- ‘Revision/textbook/figures/01e_3_symmetric_antisymmetric.png‘
- ‘Revision/textbook/figures/01e_4_clifford_tables.png‘
- ‘Revision/textbook/figures/01e_5_gamma_contractions.png‘
- ‘Revision/textbook/figures/01e_6_step_length.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 25 CHECKS PASSED (notebook 01e)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01e_index_notation.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use

  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 01e_index_notation.ipynb
      python -m nbconvert --execute --inplace 01e_index_notation.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/algebra or Revision/gkd_lovelock: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01e (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 01e (Index notation, the summation convention and the metric) is the file
# Revision/textbook/notebooks/01e_index_notation.ipynb of the repository Dirac_claude. It
# teaches free and dummy indices and the summation convention with numpy einsum, reads
# the frame metric eta of the author's coordinates x0 to x7 from the Revision record,
# lowers and raises indices with it, sorts vectors into space-like, time-like and
# light-like ones in the signature (4,4) and compares random vectors with those of
# ordinary four-dimensional spacetime, proves that a symmetric array contracted with an
# antisymmetric one gives zero, checks the Clifford relation of the eight real gamma
# matrices of the Revision record as one index formula for all 64 pairs, derives and
# checks the contractions gamma^a gamma_a = 8 and gamma^a gamma^b gamma_a = -6 gamma^b,
# and writes the squared length of a coordinate step in the author's metric, whose three
# extra times deflate exponentially, as a sum over indices, with heat maps and plots. It
# needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
```

```
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
```

```
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01e_index_notation.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01e_index_notation.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01e_index_notation.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01e.captions.json
# - Revision/textbook/figures/01e_1_eta_and_delta.png
# - Revision/textbook/figures/01e_2_squared_lengths.png
# - Revision/textbook/figures/01e_3_symmetric_antisymmetric.png
# - Revision/textbook/figures/01e_4_clifford_tables.png
# - Revision/textbook/figures/01e_5_gamma_contractions.png
# - Revision/textbook/figures/01e_6_step_length.png
#
```

```
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 25 CHECKS PASSED (notebook 01e)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01e_index_notation.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01e_index_notation.ipynb
#     python -m nbconvert --execute --inplace 01e_index_notation.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
```

```
#   a result.
# - "FileNotFoundError" for a file below Revision/algebra or Revision/gkd_lovelock: the
#   notebook reads Revision records of the repository; your copy of the repository is
#   incomplete. Download it again with git clone and open the notebook inside the new
#   copy.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "01e"  # this notebook: chapter 01, example e


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
```

```
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
```

```
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01e complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Index**: a letter ($a, b, c, d$ or $\mu, \nu, \lambda$) that stands for one
  of the eight coordinates $x_0, \dots, x_7$; the component of a vector $v$
  along $x_3$ is written $v^3$. Python numbers the same components 0 to 7 in
  the same order, so $v^3$ is `v[3]`.
- **Upper and lower index**: $v^a$ and $v_a$ are two lists of eight numbers
  that belong to the same vector (section 6 says how one is made from the
  other). An upper index is a label, not a power: $v^3$ is a component, and its
  square is written $(v^3)^2$.
- **Free index**: an index that appears once in every term of a formula; the
  formula holds for each of its values, so a formula with $k$ free indices over
  8 coordinates stands for $8^k$ equations.
- **Dummy index**: an index that appears twice in one term, once up and once
  down; it is summed over all its values. Its name does not matter.
- **Summation convention** (Einstein's): the rule that a repeated index is
  summed without writing $\sum$: $v^a w_a = \sum_{a} v^a w_a$.
- **einsum**: numpy's function that evaluates such a sum from a text that lists
  the indices of every factor, for example `np.einsum("ab,b->a", M, v)`.
- **Contraction**: setting an upper and a lower index equal and summing over
  it, as in $\delta^a_a$ or $v^a w_a$.
- **Kronecker delta** $\delta^a_b$: 1 if $a = b$ and 0 otherwise; as a table it
  is the identity matrix $I$.
- **Frame metric** $\eta_{ab}$: the diagonal table
  $\mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \dots,
  x_7$; $\eta^{ab}$ (upper indices) is its inverse.
- **Lowering and raising**: $v_a = \eta_{ab} v^b$ and $v^a = \eta^{ab} v_b$.
```

```
- **Squared length** $Q(v) = \eta_{ab} v^a v^b$. A vector $v \neq 0$ is
  **space-like** if $Q > 0$, **time-like** if $Q < 0$ and **light-like** (or
  **null**) if $Q = 0$.
- **Signature** (4,4): $\eta$ has four entries $+1$ and four entries $-1$.
- **Symmetric** array: $S_{ab} = S_{ba}$; **antisymmetric** array:
  $A_{ab} = -A_{ba}$ (its diagonal is 0).
- **Commutator** $[P, R] = PR - RP$; **anticommutator** $\{P, R\} = PR + RP$.
- **Gamma matrices** $\gamma^a$: the eight real $16 \times 16$ matrices of the
  author's theory, one for each coordinate, stored in the Revision record; they
  obey the **Clifford relation** $\{\gamma^a, \gamma^b\} = 2\eta^{ab} I$.
- **Orthogonal matrix**: a real matrix $O$ with $O^T O = I$, so $O^T = O^{-1}$.
- **Trace** $\mathrm{tr}\,M$: the sum of the diagonal entries of a matrix; it
  obeys $\mathrm{tr}(PR) = \mathrm{tr}(RP)$.
- **Metric** $g_{\mu\nu}$ and **line element**: the author's metric turns a
  small coordinate step $dx^\mu$ into the squared length
  $ds^2 = g_{\mu\nu} dx^\mu dx^\nu$.
- **Normal random numbers**: random numbers whose histogram is the bell curve
  $e^{-x^2/2}/\sqrt{2\pi}$; numpy's `normal` draws them.
- **Histogram**: bars that show how many numbers fall into each interval.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's spacetime has eight coordinates: $x_1, x_2, x_3$ are ordinary
space, $x_4$ is the time, $x_5, x_6, x_7$ are three extra times, and $x_0$ is a
hidden space direction. The *frame metric*

$$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$$

records only which directions are space-like ($+1$: $x_0, x_1, x_2, x_3$) and
which are time-like ($-1$: $x_4, \dots, x_7$). The full *metric* of the
primordial gravitational field is

$$g = \mathrm{diag}\big(\cot^2 z, e^{2a_4} s, e^{2a_4} s, e^{2a_4} s, -1,
-b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s\big),\quad
s = \sin^{1/3} z,\ z = 6 H x_0,$$

with a positive constant $H$, $0 < z < \pi/2$, a function $a_4(x_4)$ and a
positive function $b_4(x_4)$ of the time ($b_4 = 1$ in the background of the
Revision record): as $a_4$ grows, the lengths along $x_1, x_2, x_3$ grow like
$e^{a_4}$ and the lengths along the three extra times shrink like
$b_4^{1/2} e^{-a_4}$, that is, for constant $b_4$ they deflate exponentially.

Each equation of the theory is an index formula. For example the field
equation of the Revision record, $\gamma^\mu D_\mu \Psi = (m + U'(S))\Psi$, is a
sum over $\mu$ of eight terms, and the Clifford relation
$\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab} I$ is one line that stands
for $8 \cdot 8 = 64$ equations between $16 \times 16$ matrices. Reading and
checking such formulas is the skill this notebook trains.
```

Text cell:

```
## 5. Free and dummy indices

The next cell works with a $3 \times 3$ table $M^a{}_b$ (row $a$, column $b$) and
the vector $v^b = (2, -1, 3)$. In

$$w^a = M^a{}_b v^b = \sum_{b} M^a{}_b v^b$$

the index $a$ is free (one equation for each $a$) and $b$ is a dummy (summed).
The cell computes $w$ three ways: with loops; with
`np.einsum("ab,b->a", M, v)`, whose text says "the first factor has the indices
$a, b$, the second $b$; $b$ appears twice, so it is summed; the result keeps
$a$"; and with the dummy renamed to $c$, which changes nothing. Row 1 gives
$1 \cdot 2 + 2 \cdot (-1) + 0 \cdot 3 = 0$, row 2 gives $0 - 1 + 3 = 2$, row 3
gives $2 - 3 + 3 = 2$.

It then computes two numbers without any free index: the trace
$M^a{}_a = 1 + 1 + 1 = 3$ (a contraction of the two indices of one table) and
$u_a M^a{}_b v^b$ for $u_a = (1, 0, 2)$, which has two dummy indices; since
$M^a{}_b v^b = (0, 2, 2)$, it is $1 \cdot 0 + 0 \cdot 2 + 2 \cdot 2 = 4$.
Finally it prints how many equations a formula with 0 to 3 free indices over
the eight coordinates stands for.
```

In [2]:

```
import itertools  # all pairs (a, b) of indices
import math  # pi and square roots of plain numbers

import numpy as np  # arrays and einsum
import sympy as sp  # exact algebra with symbols

M = np.array([[1, 2, 0],
              [0, 1, 1],
              [1, 3, 1]])  # the entries M^a_b: row a, column b
v = np.array([2, -1, 3])  # the components v^b
u = np.array([1, 0, 2])  # the components u_a of a second vector
# w^a = M^a_b v^b: the free index a is kept, the dummy index b is summed.
w_loops = [sum(int(M[a, b]) * int(v[b]) for b in range(3)) for a in range(3)]
w_einsum = np.einsum("ab,b->a", M, v)  # "b" appears twice: it is summed
w_renamed = np.einsum("ac,c->a", M, v)  # the dummy renamed to c: the same sum
say(f"w = M v by loops {w_loops}, by einsum {w_einsum.tolist()}, with the dummy "
    f"renamed {w_renamed.tolist()}")
check(w_loops == w_einsum.tolist() == w_renamed.tolist() == (M @ v).tolist()
      == [0, 2, 2],
      "loops, einsum and @ give w = (0, 2, 2); renaming the dummy changes nothing")
trace = int(np.einsum("aa->", M))  # M^a_a: the repeated index a is summed
full = int(np.einsum("a,ab,b->", u, M, v))  # u_a M^a_b v^b: two dummies, no free
say(f"M^a_a = {trace}, u_a M^a_b v^b = {full}")
check(trace == int(np.trace(M)) == 3 and full == int(u @ M @ v) == 4,
      "the trace M^a_a = 3 and the number u_a M^a_b v^b = 4")
for free in range(4):
    word = "equation" if free == 0 else "equations"  # 8^0 = 1 equation
    say(f"{free} free indices over the 8 coordinates: 8^{free} = {8 ** free} {word}")
```

Out [2]:

```
w = M v by loops [0, 2, 2], by einsum [0, 2, 2], with the dummy renamed [0, 2, 2]
PASS loops, einsum and @ give w = (0, 2, 2); renaming the dummy changes nothing
M^a_a = 3, u_a M^a_b v^b = 4
PASS the trace M^a_a = 3 and the number u_a M^a_b v^b = 4
0 free indices over the 8 coordinates: 8^0 = 1 equation
1 free indices over the 8 coordinates: 8^1 = 8 equations
2 free indices over the 8 coordinates: 8^2 = 64 equations
3 free indices over the 8 coordinates: 8^3 = 512 equations
```

Text cell:

```
## 6. The frame metric: lowering and raising an index

The next cell reads the coordinate names and the diagonal of $\eta$ from the
Revision record file Revision/algebra/gammas.json, and the record's report
Revision/algebra/reports/python-algebra.json, whose check `coordinate_map`
states $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$. Then:

- $\eta_{ab}$ is the diagonal matrix with these entries. Its inverse
  $\eta^{ab}$ is the diagonal matrix of the inverses $1/(+1) = +1$ and
  $1/(-1) = -1$: the same entries.
- $\eta^{ab}\eta_{bc}$ (free indices $a$ up and $c$ down, dummy $b$) must be
  $\delta^a_c$, and contracting the two indices of $\delta$ counts the
  coordinates: $\delta^a_a = 1 + 1 + \dots + 1 = 8$.
- Lowering the index of $v^a = (1, 2, \dots, 8)$ gives
  $v_a = \eta_{ab} v^b$; because $\eta$ is diagonal, only the term $b = a$ of
  the sum survives, $v_a = \eta_{aa} v^a$ (no sum here): the four time-like
  components $x_4, \dots, x_7$ change sign. Raising again,
  $\eta^{ab} v_b$, gives back $v^a$, because $\eta^{ab}\eta_{bc} = \delta^a_c$.

The cell after it defines the helper `heat_map`, which draws a table as
coloured squares (red positive, white zero, blue negative), and draws
$\eta_{ab}$, $\delta^a_c$ and the two lists $v^a$ and $v_a$.
```

In [3]:

```
algebra = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
names = algebra["coordinates"]  # ["x0", "x1", ..., "x7"]
eta_diagonal = algebra["eta"]  # the diagonal of eta in the order x0 ... x7
algebra_report = json.loads(repository_file(
    "Revision/algebra/reports/python-algebra.json").read_text(encoding="utf-8"))
verdicts = {c["name"]: c["verdict"] for c in algebra_report["checks"]}
map_verdict = verdicts["coordinate_map"]  # "pass" in the record
say(f"coordinates {names}")
say(f"diagonal of eta {eta_diagonal}; record check coordinate_map: {map_verdict}")
check(eta_diagonal == [1, 1, 1, 1, -1, -1, -1, -1]
      and verdicts["coordinate_map"] == "pass",
      "eta = diag(+1, +1, +1, +1, -1, -1, -1, -1) in the order x0 ... x7",
```

```
      record="Revision/algebra/reports/python-algebra.json, check coordinate_map")
eta_lower = np.diag(eta_diagonal)  # eta_ab as an 8 x 8 matrix
eta_upper = np.diag([1 // e for e in eta_diagonal])  # eta^ab: 1/(+1) = 1, 1/(-1) = -1
delta = np.einsum("ab,bc->ac", eta_upper, eta_lower)  # eta^ab eta_bc
check((delta == np.eye(8, dtype=int)).all() and int(np.einsum("aa->", delta)) == 8,
      "eta^ab eta_bc = delta^a_c, and the contraction delta^a_a = 8")
v_up = np.arange(1, 9)  # v^a = 1, 2, ..., 8
v_down = np.einsum("ab,b->a", eta_lower, v_up)  # v_a = eta_ab v^b
v_back = np.einsum("ab,b->a", eta_upper, v_down)  # eta^ab v_b
flipped = [names[a] for a in range(8) if v_down[a] != v_up[a]]
say(f"v^a = {v_up.tolist()}")
say(f"v_a = {v_down.tolist()}; the components that changed sign: {flipped}")
check(flipped == ["x4", "x5", "x6", "x7"] and (v_back == v_up).all(),
      "lowering flips the signs of the time-like x4 ... x7; raising undoes it")
```

Out [3]:

```
coordinates ['x0', 'x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7']
diagonal of eta [1, 1, 1, 1, -1, -1, -1, -1]; record check coordinate_map: pass
PASS eta = diag(+1, +1, +1, +1, -1, -1, -1, -1) in the order x0 ... x7
     reproduces Revision/algebra/reports/python-algebra.json, check coordinate_map
PASS eta^ab eta_bc = delta^a_c, and the contraction delta^a_a = 8
v^a = [1, 2, 3, 4, 5, 6, 7, 8]
v_a = [1, 2, 3, 4, -5, -6, -7, -8]; the components that changed sign: ['x4', 'x5', 'x6',
    'x7']
PASS lowering flips the signs of the time-like x4 ... x7; raising undoes it
```

Text cell:

```
The next cell defines ‘heat_map‘ and draws the first figure: $\eta_{ab}$,
$\delta^a_c$ and the two lists $v^a$ and $v_a$ side by side.
```

In [4]:

```
labels = [f"$x_{k}$" for k in range(8)]  # x with the subscripts 0 ... 7


def heat_map(ax, matrix, title, ticks=labels, annotate=True, size=7.0):
    """Draw matrix on the axes ax: red positive, white zero, blue negative; with
    annotate the value is written in its square (%g: no needless digits)."""
    m = np.asarray(matrix, dtype=float)
    limit = max(1.0, float(np.abs(m).max()))  # colours from -limit to +limit
    ax.imshow(m, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.grid(False)  # no grid lines across the coloured squares
    ax.set_xticks(range(m.shape[1]), ticks[:m.shape[1]], fontsize=7)
    ax.set_yticks(range(m.shape[0]), ticks[:m.shape[0]], fontsize=7)
    ax.set_title(title, fontsize=10)
    if annotate:
        for i in range(m.shape[0]):
            for j in range(m.shape[1]):
                ink = "white" if abs(m[i, j]) > 0.6 * limit else "black"
```

```
                ax.text(j, i, f"{m[i, j]:g}", ha="center", va="center",
                        fontsize=size, color=ink)


fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
heat_map(axes[0], eta_lower, "$\\eta_{ab}$ (row $a$, column $b$)")
heat_map(axes[1], delta, "$\\eta^{ab}\\eta_{bc} = \\delta^a_c$ (row $a$, column $c$)")
positions = np.arange(8)
axes[2].bar(positions - 0.2, v_up, width=0.4, label="$v^a$ (upper index)")
axes[2].bar(positions + 0.2, v_down, width=0.4, label="$v_a = \\eta_{ab} v^b$")
axes[2].axhline(0, color="black", lw=0.8)
axes[2].set_xticks(positions, labels)
axes[2].set_ylabel("component")
axes[2].set_title("lowering the index of $v$", fontsize=10)
axes[2].legend(fontsize=8, loc="lower left")
fig.tight_layout()
save_figure(fig, "eta_and_delta",
            "Left: the frame metric $\\eta_{ab}$ of the author's coordinates as a "
            "heat map (rows $a$ and columns $b$ labelled $x_0$ to $x_7$; red $+1$, "
            "blue $-1$, white 0): $+1$ for the space-like $x_0, x_1, x_2, x_3$ and "
            "$-1$ for the time-like $x_4$ to $x_7$. Middle: the contraction "
            "$\\eta^{ab}\\eta_{bc}$, which is the identity matrix $\\delta^a_c$. "
            "Right: the components $v^a = 1, 2, \\dots, 8$ of a vector (blue) and "
            "the components $v_a = \\eta_{ab} v^b$ with the index lowered "
            "(orange); horizontal axis the coordinate, vertical axis the component "
            "(pure numbers). Lowering changes the sign of exactly the four "
            "time-like components.")
```

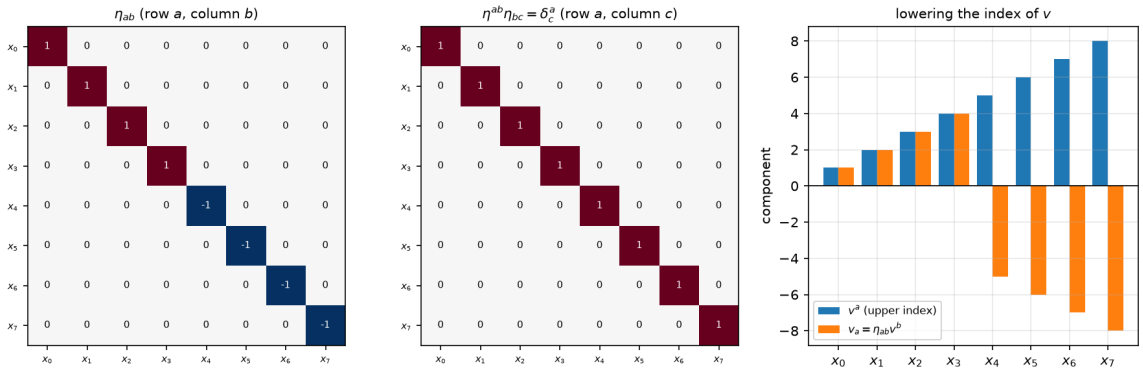

Figure 35: Left: the frame metric η_{ab} of the author's coordinates as a heat map (rows a and columns b labelled x_0 to x_7; red $+1$, blue -1, white 0): $+1$ for the space-like x_0, x_1, x_2, x_3 and -1 for the time-like x_4 to x_7. Middle: the contraction $\eta^{ab}\eta_{bc}$, which is the identity matrix δ^a_c. Right: the components $v^a = 1, 2, \dots, 8$ of a vector (blue) and the components $v_a = \eta_{ab}v^b$ with the index lowered (orange); horizontal axis the coordinate, vertical axis the component (pure numbers). Lowering changes the sign of exactly the four time-like components. (Notebook 01e, figure 1.)

Out [4]:

```
Figure 01e.1 saved as Revision/textbook/figures/01e_1_eta_and_delta.png
```

Text cell:

```
## 7. Squared lengths in the signature (4,4)

The squared length of a vector is

$$Q(v) = \eta_{ab} v^a v^b = v^a v_a = (v^0)^2 + (v^1)^2 + (v^2)^2 + (v^3)^2
- (v^4)^2 - (v^5)^2 - (v^6)^2 - (v^7)^2,$$

where the second form uses $v_a = \eta_{ab} v^b$, and the third writes out the
sum (only the terms $a = b$ survive because $\eta$ is diagonal). The next cell
defines `squared_length`, which evaluates $\eta_{ab} v^a v^b$ for many vectors
at once (`einsum("na,ab,nb->n", ...)`: $n$ numbers the vectors and is kept;
$a$ and $b$ are summed), and sorts five examples:

- the unit vector along $x_1$: $Q = +1$, space-like;
- the unit vector along $x_5$: $Q = -1$, time-like;
- their sum: $Q = 1 - 1 = 0$, light-like although it is not zero;
- $(1, 1, \dots, 1)$: $Q = 4 - 4 = 0$, light-like as well;
- $v^a = (1, 2, \dots, 8)$: $Q = 1 + 4 + 9 + 16 - 25 - 36 - 49 - 64 = -144$,
  time-like.

It checks the sorting and that $\eta_{ab} v^a v^b$ equals $v^a v_a$ for the last
example (the list $v_a$ of section 6).
```

In [5]:

```
def squared_length(vectors, metric_diagonal):
    """eta_ab v^a v^b for every row v of the array vectors (a and b are summed;
    the index n, which numbers the rows, is kept)."""
    metric = np.diag(metric_diagonal)
    return np.einsum("na,ab,nb->n", vectors, metric, vectors)


unit = np.eye(8, dtype=int)  # row k is the unit vector along x_k
examples = {"e(x1)": unit[1], "e(x5)": unit[5], "e(x1) + e(x5)": unit[1] + unit[5],
            "(1, 1, 1, 1, 1, 1, 1, 1)": np.ones(8, dtype=int),
            "(1, 2, 3, 4, 5, 6, 7, 8)": v_up}
kinds = []
for name, vector in examples.items():
    q = int(squared_length(vector[None, :], eta_diagonal)[0])  # one row
    kind = "space-like" if q > 0 else ("time-like" if q < 0 else "light-like")
    kinds.append(kind)
    q_text = f"{q:+d}" if q != 0 else "0"  # a sign in front of nonzero values only
    say(f"v = {name}: eta_ab v^a v^b = {q_text}, {kind}")
check(kinds == ["space-like", "time-like", "light-like", "light-like", "time-like"]
      and int(squared_length(v_up[None, :], eta_diagonal)[0]) == int(v_up @ v_down)
      == -144,
      "the five examples are sorted correctly, and eta_ab v^a v^b = v^a v_a = -144")
```

Out [5]:

```
v = e(x1): eta_ab v^a v^b = +1, space-like
v = e(x5): eta_ab v^a v^b = -1, time-like
v = e(x1) + e(x5): eta_ab v^a v^b = 0, light-like
v = (1, 1, 1, 1, 1, 1, 1, 1): eta_ab v^a v^b = 0, light-like
v = (1, 2, 3, 4, 5, 6, 7, 8): eta_ab v^a v^b = -144, time-like
PASS the five examples are sorted correctly, and eta_ab v^a v^b = v^a v_a = -144
```

Text cell:

```
How many vectors are space-like? Draw the eight components of a vector as
independent normal random numbers. Exchange its four space-like components
$(v^0, v^1, v^2, v^3)$ with its four time-like components
$(v^4, v^5, v^6, v^7)$: the new vector $v'$ has

$$Q(v') = (v^4)^2 + (v^5)^2 + (v^6)^2 + (v^7)^2 - (v^0)^2 - (v^1)^2 - (v^2)^2
- (v^3)^2 = -Q(v),$$

because every square that was added is now subtracted and every square that
was subtracted is now added. The exchanged vector is exactly as likely as the
original one (all eight components are drawn the same way), so $Q > 0$ and
$Q < 0$ are equally likely, and $Q = 0$ exactly has probability 0: **in the
signature (4,4) exactly half of all random vectors are space-like.** In
ordinary spacetime with one time and three space directions, signature
(3,1), $Q = (v^1)^2 + (v^2)^2 + (v^3)^2 - (v^4)^2$ has three squares added
and one subtracted, and most random vectors are space-like; integral calculus
(not shown here) gives the fraction $1/2 + 1/\pi \approx 0.818$.

The next cell draws 200,000 random vectors (seed 12345), checks the exchange
rule on every one of them, measures both fractions and compares them with
$1/2$ and $1/2 + 1/\pi$, allowing 4 standard deviations
$\sqrt{p(1-p)/N}$ of a measured fraction $p$ from $N$ tries.
```

In [6]:

```
generator = np.random.default_rng(12345)  # random numbers with a fixed seed
samples = generator.normal(size=(200000, 8))  # 200,000 vectors of 8 components
q44 = squared_length(samples, eta_diagonal)  # signature (4,4)
space_part, time_part = [0, 1, 2, 3], [4, 5, 6, 7]  # x0 x1 x2 x3, and x4 ... x7
exchanged = samples.copy()
exchanged[:, space_part] = samples[:, time_part]  # time components -> space slots
exchanged[:, time_part] = samples[:, space_part]  # space components -> time slots
check(np.allclose(squared_length(exchanged, eta_diagonal), -q44, rtol=0, atol=1e-9),
      "exchanging the space-like and the time-like components turns Q into -Q "
      "(all 200000 vectors)")
fraction44 = float(np.mean(q44 > 0))  # the fraction of space-like vectors
q31 = squared_length(samples[:, :4], [1, 1, 1, -1])  # 3 space, 1 time direction
fraction31 = float(np.mean(q31 > 0))
exact31 = 0.5 + 1 / math.pi
sd44 = math.sqrt(0.5 * 0.5 / 200000)  # one standard deviation of a fraction
sd31 = math.sqrt(exact31 * (1 - exact31) / 200000)
report("fraction of space-like random vectors, signature (4,4)", f"{fraction44:.4f}")
report("fraction of space-like random vectors, signature (3,1)", f"{fraction31:.4f}")
```

```
check(abs(fraction44 - 0.5) < 4 * sd44,
      "signature (4,4): half of the random vectors are space-like")
check(abs(fraction31 - exact31) < 4 * sd31,
      "signature (3,1): the fraction is 1/2 + 1/pi = 0.818 (within 4 standard "
      "deviations)")
```

Out [6]:

```
PASS exchanging the space-like and the time-like components turns Q into -Q (all 200000
    vectors)
RESULT fraction of space-like random vectors, signature (4,4) = 0.5011
RESULT fraction of space-like random vectors, signature (3,1) = 0.8177
PASS signature (4,4): half of the random vectors are space-like
PASS signature (3,1): the fraction is 1/2 + 1/pi = 0.818 (within 4 standard deviations)
```

Text cell:

```
The next cell draws the two histograms of $Q$: the signature (4,4) of the
author's spacetime and the signature (3,1) of ordinary spacetime.
```

In [7]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
bins = np.linspace(-15.0, 15.0, 121)  # 120 intervals of width 0.25
for ax, values, fraction, title, color in (
        (axes[0], q44, fraction44, "signature (4,4): 4 space-like, 4 time-like",
         "tab:purple"),
        (axes[1], q31, fraction31, "signature (3,1): 3 space-like, 1 time-like",
         "tab:green")):
    ax.hist(values, bins=bins, density=True, color=color, alpha=0.75)
    ax.axvline(0.0, color="black", lw=1)
    ax.text(0.97, 0.95, f"space-like: {fraction:.3f}", transform=ax.transAxes,
            ha="right", va="top")
    ax.text(0.03, 0.95, f"time-like: {1 - fraction:.3f}", transform=ax.transAxes,
            ha="left", va="top")
    ax.set_xlabel("squared length $Q = \\eta_{ab} v^a v^b$")
    ax.set_title(title, fontsize=10)
axes[0].set_ylabel("fraction of vectors per unit of $Q$")
fig.tight_layout()
save_figure(fig, "squared_lengths",
            "Histograms of the squared length $Q = \\eta_{ab} v^a v^b$ of 200,000 "
            "random vectors whose components are normal random numbers; "
            "horizontal axis $Q$ (a pure number), vertical axis the fraction of "
            "vectors per unit of $Q$. Left: the signature (4,4) of the author's "
            "spacetime; the histogram is the same on both sides of $Q = 0$, so "
            "half of the vectors are space-like ($Q > 0$) and half time-like. "
            "Right: ordinary spacetime with one time and three space directions; "
            "about 82 per cent of the vectors are space-like.")
```

Out [7]:

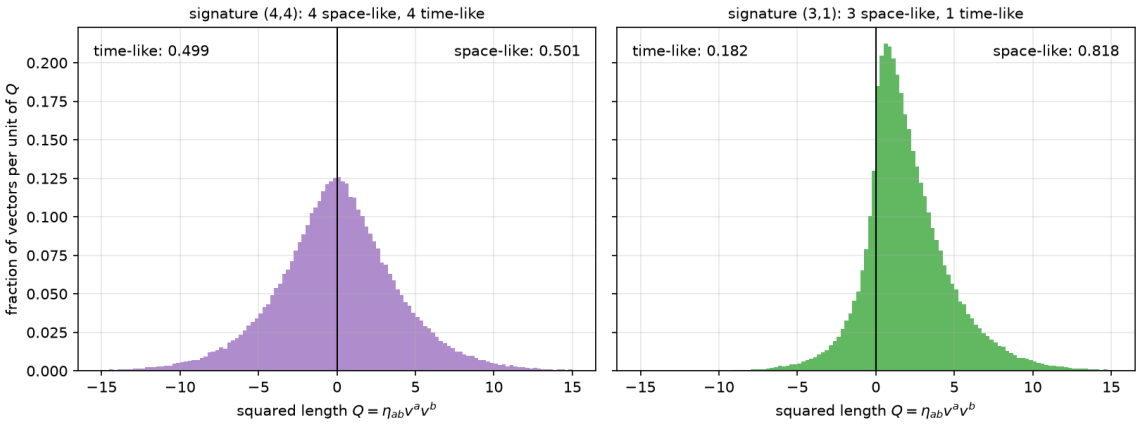

Figure 36: Histograms of the squared length $Q = \eta_{ab} v^a v^b$ of 200,000 random vectors whose components are normal random numbers; horizontal axis Q (a pure number), vertical axis the fraction of vectors per unit of Q. Left: the signature (4,4) of the author's spacetime; the histogram is the same on both sides of $Q = 0$, so half of the vectors are space-like ($Q > 0$) and half time-like. Right: ordinary spacetime with one time and three space directions; about 82 per cent of the vectors are space-like. (Notebook 01e, figure 2.)

```
Figure 01e.2 saved as Revision/textbook/figures/01e_2_squared_lengths.png
```

Text cell:

```
## 8. A symmetric array contracted with an antisymmetric one gives zero

Every square array $M_{ab}$ is a sum of a symmetric and an antisymmetric part:

$$M_{ab} = S_{ab} + A_{ab},\qquad S_{ab} = \tfrac12 (M_{ab} + M_{ba}),\qquad
A_{ab} = \tfrac12 (M_{ab} - M_{ba}).$$

Adding the two parts gives back $M_{ab}$ (the terms $M_{ba}$ cancel);
exchanging $a$ and $b$ leaves $S$ unchanged and changes the sign of $A$.

**Claim.** If $S_{ab} = S_{ba}$ and $A^{ab} = -A^{ba}$, then
$S_{ab} A^{ab} = 0$. **Proof**, line by line:

$$S_{ab} A^{ab} = S_{ba} A^{ba}$$

(both sides are the same sum over all pairs; the names of the two summed
indices were exchanged, which does not change a sum),

$$S_{ba} A^{ba} = S_{ab} \, (-A^{ab})$$

(the symmetry of $S$ and the antisymmetry of $A$), so
$S_{ab} A^{ab} = -S_{ab} A^{ab}$, and a number that equals its own negative is
0. In words: the term $(a, b)$ and the term $(b, a)$ cancel, and the diagonal
terms are 0.

Raising both indices keeps an array antisymmetric:
```

```
$A^{ab} = \eta^{ac}\eta^{bd} A_{cd}$, and
$A^{ba} = \eta^{bc}\eta^{ad} A_{cd} = \eta^{ad}\eta^{bc} A_{cd}$; renaming the
dummies $c \leftrightarrow d$ gives $\eta^{ac}\eta^{bd} A_{dc} =
-\eta^{ac}\eta^{bd} A_{cd} = -A^{ab}$.

A consequence used again and again: for a vector with ordinary (commuting)
components, $v^a v^b$ is symmetric in $a$ and $b$, so
$v^a v^b A_{ab} = 0$ and $v^a v^b M_{ab} = v^a v^b S_{ab}$: only the
symmetric part of an array counts in a squared expression. (Later in the course
the components of the field dirac16complex *anticommute*; then the opposite
holds, and only the antisymmetric part counts.)

The next cell checks all of this exactly, with whole numbers: it works with
$2S$ and $2A$, so that no fraction appears.
```

In [8]:

```
M8 = generator.integers(-5, 6, size=(8, 8))  # a random 8 x 8 array, entries -5 ... 5
S2 = M8 + M8.T  # twice the symmetric part
A2 = M8 - M8.T  # twice the antisymmetric part
check((S2 == S2.T).all() and (A2 == -A2.T).all() and (S2 + A2 == 2 * M8).all(),
      "M = S + A with S = (M + M^T)/2 symmetric and A = (M - M^T)/2 antisymmetric")
A2_up = np.einsum("ac,bd,cd->ab", eta_upper, eta_upper, A2)  # A^ab (times 2)
terms = S2 * A2_up  # the 64 terms S_ab A^ab (times 4), before the sum
total = int(np.einsum("ab,ab->", S2, A2_up))  # the full contraction (times 4)
say(f"4 S_ab A^ab = {total}; the largest single term is "
    f"{int(np.abs(terms).max())}")
check((A2_up == -A2_up.T).all() and (terms == -terms.T).all() and total == 0,
      "A^ab is antisymmetric, the terms (a, b) and (b, a) cancel, S_ab A^ab = 0")
all_zero = True
for _ in range(100):  # 100 more random pairs of arrays
    X = generator.integers(-9, 10, size=(8, 8))
    Y = generator.integers(-9, 10, size=(8, 8))
    all_zero &= int(np.einsum("ab,ac,bd,cd->", X + X.T, eta_upper, eta_upper,
                              Y - Y.T)) == 0
check(all_zero, "S_ab A^ab = 0 for 100 more random pairs of arrays")
squares_ok = True
for _ in range(100):  # 100 random vectors with whole-number components
    w8 = generator.integers(-9, 10, size=8)
    squares_ok &= int(np.einsum("a,b,ab->", w8, w8, A2)) == 0
    squares_ok &= (int(np.einsum("a,b,ab->", w8, w8, 2 * M8))
                   == int(np.einsum("a,b,ab->", w8, w8, S2)))
check(squares_ok, "v^a v^b A_ab = 0 and v^a v^b M_ab = v^a v^b S_ab (100 random "
      "vectors with commuting components)")
```

Out [8]:

```
PASS M = S + A with S = (M + M^T)/2 symmetric and A = (M - M^T)/2 antisymmetric
4 S_ab A^ab = 0; the largest single term is 24
PASS A^ab is antisymmetric, the terms (a, b) and (b, a) cancel, S_ab A^ab = 0
PASS S_ab A^ab = 0 for 100 more random pairs of arrays
PASS v^a v^b A_ab = 0 and v^a v^b M_ab = v^a v^b S_ab (100 random vectors with commuting
    components)
```

Text cell:

```
The next cell draws the array $M_{ab}$, its symmetric part, its antisymmetric
part and the 64 terms $S_{ab} A^{ab}$ of the contraction, whose colours show
that each term is cancelled by its mirror image across the diagonal.
```

In [9]:

```
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.7))
heat_map(axes[0], M8, "$M_{ab}$", size=6.0)
heat_map(axes[1], S2 / 2, "symmetric part $S_{ab}$", size=5.5)
heat_map(axes[2], A2 / 2, "antisymmetric part $A_{ab}$", size=5.5)
heat_map(axes[3], terms / 4, "terms $S_{ab} A^{ab}$ (sum 0)", annotate=False)
fig.tight_layout()
save_figure(fig, "symmetric_antisymmetric",
            "A random $8 \\times 8$ array $M_{ab}$ of whole numbers (left), its "
            "symmetric part $S_{ab} = (M_{ab} + M_{ba})/2$, its antisymmetric part "
            "$A_{ab} = (M_{ab} - M_{ba})/2$, and the 64 terms $S_{ab} A^{ab}$ of "
            "their contraction (right), as heat maps with rows $a$ and columns $b$ "
            "labelled $x_0$ to $x_7$ (red positive, white zero, blue negative). "
            "$S$ is mirror-symmetric across the diagonal, $A$ changes sign under "
            "the mirror and has a white diagonal, and every term of the "
            "contraction is cancelled by its mirror image, so the sum is 0.")
```

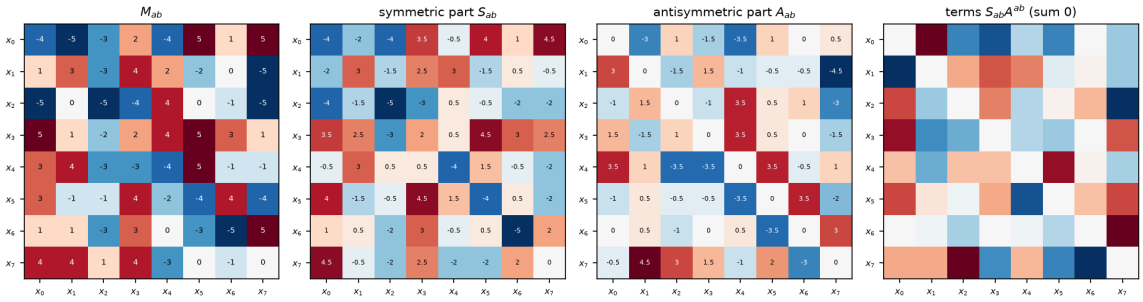

Figure 37: A random 8×8 array M_{ab} of whole numbers (left), its symmetric part $S_{ab} = (M_{ab} + M_{ba})/2$, its antisymmetric part $A_{ab} = (M_{ab} - M_{ba})/2$, and the 64 terms $S_{ab}A^{ab}$ of their contraction (right), as heat maps with rows a and columns b labelled x_0 to x_7 (red positive, white zero, blue negative). S is mirror-symmetric across the diagonal, A changes sign under the mirror and has a white diagonal, and every term of the contraction is cancelled by its mirror image, so the sum is 0. (Notebook 01e, figure 3.)

Out [9]:

```
Figure 01e.3 saved as Revision/textbook/figures/01e_3_symmetric_antisymmetric.png
```

Text cell:

```
## 9. Commutators, anticommutators and the Clifford relation
```

```
For two square matrices $P$ and $R$ the commutator $[P, R] = PR - RP$ and the
anticommutator $\{P, R\} = PR + RP$ split the product into two parts:

$$PR = \tfrac12 [P, R] + \tfrac12 \{P, R\},$$

because the two halves add up to $\tfrac12 (PR - RP + PR + RP) = PR$; and
$[R, P] = -[P, R]$, $\{R, P\} = \{P, R\}$ directly from the definitions.

The eight gamma matrices obey the Clifford relation

$$\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab} I \qquad (a, b = x_0, \dots,
x_7).$$

It has two free indices, so it stands for 64 matrix equations: for $a = b$ it
says $(\gamma^a)^2 = \eta^{aa} I$ ($+I$ for a space-like and $-I$ for a time-like
direction); for $a \neq b$ it says that $\gamma^a$ and $\gamma^b$
anticommute, $\gamma^a \gamma^b = -\gamma^b \gamma^a$, so their commutator is
$2\gamma^a\gamma^b$.

A second fact follows from it. Each gamma matrix of the record is a signed
permutation matrix, so it is orthogonal: $(\gamma^a)^T \gamma^a = I$, that is
$(\gamma^a)^T = (\gamma^a)^{-1}$. And $(\gamma^a)^2 = \eta^{aa} I$ says
$(\gamma^a)^{-1} = \eta^{aa}\gamma^a$ (because $\eta^{aa} = \pm 1$ is its own
inverse). Together: $(\gamma^a)^T = \eta_{aa}\gamma^a$ (no sum; $\eta_{aa} =
\eta^{aa}$): the gamma matrices of the space-like directions are symmetric and
those of the time-like directions antisymmetric. The record's check
'symmetry_pattern' states exactly this.

The next cell checks the commutator rules on two random matrices, then reads the
gamma matrices from the record, checks all 64 anticommutators exactly, keeps the
number $c_{ab}$ in $\{\gamma^a, \gamma^b\} = c_{ab} I$ and the number of
nonzero entries of each commutator, and checks orthogonality and the symmetry
pattern.
```

In [10]:

```
P = generator.integers(-3, 4, size=(4, 4))
R = generator.integers(-3, 4, size=(4, 4))
commutator = P @ R - R @ P
anticommutator = P @ R + R @ P
check((2 * (P @ R) == commutator + anticommutator).all()
      and (R @ P - P @ R == -commutator).all()
      and (R @ P + P @ R == anticommutator).all(),
      "P R = [P, R]/2 + {P, R}/2, [R, P] = -[P, R] and {R, P} = {P, R}")
gamma = [np.array(matrix, dtype=int) for matrix in algebra["gamma"]]  # gamma^a
identity16 = np.eye(16, dtype=int)
anti_table = np.zeros((8, 8), dtype=int)  # {gamma^a, gamma^b} = anti_table[a, b] I
commutator_size = np.zeros((8, 8), dtype=int)  # nonzero entries of the commutator
clifford_holds = True
for a, b in itertools.product(range(8), repeat=2):  # all 64 ordered pairs
    anti = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
    anti_table[a, b] = anti[0, 0]  # the number in front of I
    clifford_holds &= bool((anti == 2 * eta_upper[a, b] * identity16).all())
```

```
    commutator_size[a, b] = np.count_nonzero(gamma[a] @ gamma[b]
                                             - gamma[b] @ gamma[a])
check(clifford_holds and verdicts["clifford_relation"] == "pass",
      "{gamma^a, gamma^b} = 2 eta^ab I for all 64 ordered pairs (a, b)",
      record="Revision/algebra/reports/python-algebra.json, check "
             "clifford_relation")
orthogonal = all((g.T @ g == identity16).all() for g in gamma)
transpose_sign = [1 if (g.T == g).all() else (-1 if (g.T == -g).all() else 0)
                  for g in gamma]  # +1 symmetric, -1 antisymmetric
say("symmetric: " + ", ".join(names[a] for a in range(8) if transpose_sign[a] == 1)
    + "; antisymmetric: "
    + ", ".join(names[a] for a in range(8) if transpose_sign[a] == -1))
check(orthogonal and transpose_sign == eta_diagonal
      and verdicts["symmetry_pattern"] == "pass",
      "each gamma^a is orthogonal and (gamma^a)^T = eta_aa gamma^a: symmetric "
      "for x0, x1, x2, x3, antisymmetric for x4 ... x7",
      record="Revision/algebra/reports/python-algebra.json, check "
             "symmetry_pattern")
```

Out [10]:

```
PASS P R = [P, R]/2 + {P, R}/2, [R, P] = -[P, R] and {R, P} = {P, R}
PASS {gamma^a, gamma^b} = 2 eta^ab I for all 64 ordered pairs (a, b)
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
symmetric: x0, x1, x2, x3; antisymmetric: x4, x5, x6, x7
PASS each gamma^a is orthogonal and (gamma^a)^T = eta_aa gamma^a: symmetric for x0, x1,
    x2, x3, antisymmetric for x4 ... x7
     reproduces Revision/algebra/reports/python-algebra.json, check symmetry_pattern
```

Text cell:

```
The next cell draws the three results as pictures: the table $c_{ab}$, which is
$2\eta^{ab}$; the number of nonzero entries of every commutator; and, for each
direction, the sign $\pm 1$ in $(\gamma^a)^T = \pm\gamma^a$ next to $\eta_{aa}$.
```

In [11]:

```
fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.0),
                         gridspec_kw={"width_ratios": [1, 1, 1.2]})
heat_map(axes[0], anti_table, "$\\{\\gamma^a, \\gamma^b\\} = c_{ab} I$: $c_{ab}$")
heat_map(axes[1], commutator_size,
         "nonzero entries of $[\\gamma^a, \\gamma^b]$", size=6.0)
axes[2].bar(positions - 0.2, transpose_sign, width=0.4, color="tab:blue",
            label="$s_a$ in $(\\gamma^a)^T = s_a \\gamma^a$")
axes[2].bar(positions + 0.2, eta_diagonal, width=0.4, color="tab:orange",
            label="$\\eta_{aa}$")
axes[2].axhline(0, color="black", lw=0.8)
axes[2].set_xticks(positions, labels)
axes[2].set_ylim(-1.6, 1.6)
axes[2].set_title("symmetric ($+1$) or antisymmetric ($-1$)", fontsize=10)
axes[2].legend(fontsize=8, loc="lower left")
```

```
fig.tight_layout()
save_figure(fig, "clifford_tables",
            "The Clifford relation of the eight real $16 \\times 16$ gamma matrices "
            "of the Revision record. Left: the number $c_{ab}$ in "
            "$\\gamma^a\\gamma^b + \\gamma^b\\gamma^a = c_{ab} I$ for all 64 pairs "
            "(rows $a$, columns $b$, labelled $x_0$ to $x_7$): $c_{ab} = "
            "2\\eta^{ab}$, that is $+2$ or $-2$ on the diagonal and 0 elsewhere. "
            "Middle: the number of nonzero entries of the commutator "
            "$\\gamma^a\\gamma^b - \\gamma^b\\gamma^a$: 0 on the diagonal and 16 "
            "elsewhere, because different gamma matrices anticommute. Right: the "
            "sign $s_a$ with $(\\gamma^a)^T = s_a \\gamma^a$ (blue) next to "
            "$\\eta_{aa}$ (orange) for each coordinate; they agree.")
```

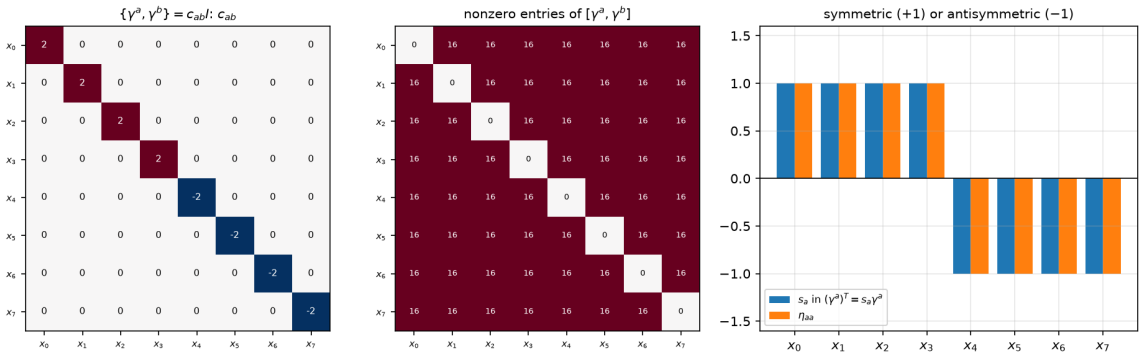

Figure 38: The Clifford relation of the eight real 16×16 gamma matrices of the Revision record. Left: the number c_{ab} in $\gamma^a\gamma^b + \gamma^b\gamma^a = c_{ab} I$ for all 64 pairs (rows a, columns b, labelled x_0 to x_7): $c_{ab} = 2\eta^{ab}$, that is $+2$ or -2 on the diagonal and 0 elsewhere. Middle: the number of nonzero entries of the commutator $\gamma^a\gamma^b - \gamma^b\gamma^a$: 0 on the diagonal and 16 elsewhere, because different gamma matrices anticommute. Right: the sign s_a with $(\gamma^a)^T = s_a\gamma^a$ (blue) next to η_{aa} (orange) for each coordinate; they agree. (Notebook 01e, figure 4.)

Out [11]:

```
Figure 01e.4 saved as Revision/textbook/figures/01e_4_clifford_tables.png
```

Text cell:

```
## 10. Contractions with the gamma matrices

Lowering the index of the gamma matrices gives $\gamma_a = \eta_{ab}\gamma^b$,
which is $+\gamma^a$ for a space-like and $-\gamma^a$ for a time-like
direction. Two contractions follow from the Clifford relation alone.

**First:** $\gamma^a \gamma_a = 8 I$. Line by line:

$$\gamma^a \gamma_a = \sum_a \gamma^a \eta_{ab} \gamma^b
= \sum_a \eta_{aa} (\gamma^a)^2$$

(only $b = a$ survives because $\eta$ is diagonal),
```

```
$$\sum_a \eta_{aa} (\gamma^a)^2 = \sum_a \eta_{aa} \eta^{aa} I$$

(the Clifford relation with $a = b$: $2(\gamma^a)^2 = 2\eta^{aa} I$),

$$\sum_a \eta_{aa}\eta^{aa} I = \sum_a 1 \cdot I = 8 I$$

($\eta_{aa}\eta^{aa} = (\pm 1)^2 = 1$ for each of the 8 values of $a$). Each
time-like direction contributes $(\gamma^a)^2 = -I$, but its $\eta_{aa} = -1$
turns the contribution into $+I$.

**Second:** $\gamma^a \gamma^b \gamma_a = -6\gamma^b$. First lower one index of
the Clifford relation: multiplying $\gamma^b\gamma^c + \gamma^c\gamma^b =
2\eta^{bc} I$ by $\eta_{ca}$ and summing over $c$ gives
$\gamma^b\gamma_a + \gamma_a\gamma^b = 2\delta^b_a I$, that is
$\gamma^b\gamma_a = -\gamma_a\gamma^b + 2\delta^b_a I$. Then

$$\gamma^a \gamma^b \gamma_a = \gamma^a\big(-\gamma_a\gamma^b + 2\delta^b_a
I\big) = -(\gamma^a\gamma_a)\gamma^b + 2\gamma^b = -8\gamma^b + 2\gamma^b =
-6\gamma^b$$

(the bracket was multiplied out; $\gamma^a\delta^b_a = \gamma^b$ because only
$a = b$ survives; the first contraction was used). In $n$ dimensions the same
steps give $(2 - n)\gamma^b$.

**Third:** $\mathrm{tr}(\gamma^a\gamma^b) = 16\eta^{ab}$, because
$\mathrm{tr}(\gamma^a\gamma^b) = \mathrm{tr}(\gamma^b\gamma^a)$, so
$\mathrm{tr}(\gamma^a\gamma^b) = \tfrac12\mathrm{tr}(\{\gamma^a, \gamma^b\}) =
\tfrac12 \cdot 2\eta^{ab} \cdot \mathrm{tr}\,I = 16\eta^{ab}$.

The next cell checks the three results with the matrices of the record.
```

In [12]:

```
gamma_lower = [sum(eta_lower[a, b] * gamma[b] for b in range(8)) for a in range(8)]
contraction = sum(gamma[a] @ gamma_lower[a] for a in range(8))  # gamma^a gamma_a
check((contraction == 8 * identity16).all(), "gamma^a gamma_a = 8 I")
lowered_ok = all((gamma[b] @ gamma_lower[a] + gamma_lower[a] @ gamma[b]
                  == 2 * int(a == b) * identity16).all()
                 for a, b in itertools.product(range(8), repeat=2))
sandwich = [sum(gamma[a] @ gamma[b] @ gamma_lower[a] for a in range(8))
            for b in range(8)]  # gamma^a gamma^b gamma_a for each b
factors = [int(sandwich[b][np.nonzero(gamma[b])][0]
               // gamma[b][np.nonzero(gamma[b])][0]) for b in range(8)]
say(f"gamma^a gamma^b gamma_a = c gamma^b with c = {factors} for b = x0 ... x7")
check(lowered_ok and all((sandwich[b] == -6 * gamma[b]).all() for b in range(8)),
      "gamma^b gamma_a + gamma_a gamma^b = 2 delta^b_a I, and gamma^a gamma^b "
      "gamma_a = -6 gamma^b for every b")
traces = np.array([[int(np.trace(gamma[a] @ gamma[b])) for b in range(8)]
                   for a in range(8)])
check((traces == 16 * eta_upper).all(), "tr(gamma^a gamma^b) = 16 eta^ab")
```

Out [12]:

```
PASS gamma^a gamma_a = 8 I
gamma^a gamma^b gamma_a = c gamma^b with c = [-6, -6, -6, -6, -6, -6, -6, -6] for b = x0
    ... x7
PASS gamma^b gamma_a + gamma_a gamma^b = 2 delta^b_a I, and gamma^a gamma^b gamma_a = -6
    gamma^b for every b
PASS tr(gamma^a gamma^b) = 16 eta^ab
```

Text cell:

```
The next cell draws the first contraction term by term (left) and the second
for every $b$ (right).
```

In [13]:

```
square_sign = [int(np.trace(g @ g)) // 16 for g in gamma]  # (gamma^a)^2 = +-I
term_sign = [eta_diagonal[a] * square_sign[a] for a in range(8)]  # eta_aa (gamma^a)^2
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.bar(positions - 0.27, square_sign, width=0.27, color="tab:blue",
         label="$(\\gamma^a)^2 = \\pm I$: the sign")
left.bar(positions, eta_diagonal, width=0.27, color="tab:orange",
         label="$\\eta_{aa}$")
left.bar(positions + 0.27, term_sign, width=0.27, color="tab:green",
         label="term $\\eta_{aa}(\\gamma^a)^2 = +I$")
left.plot(positions, np.cumsum(term_sign) / 8, "k.--", lw=1,
          label="running sum / 8")
left.axhline(0, color="black", lw=0.8)
left.set_xticks(positions, labels)
left.set_ylim(-1.5, 1.9)
left.set_title("$\\gamma^a\\gamma_a$: eight terms, each $+I$, sum $8I$",
               fontsize=10)
left.legend(fontsize=7, loc="lower left", ncol=2)
right.plot(positions, factors, "o", color="tab:red", ms=9,
           label="$c$ in $\\gamma^a\\gamma^b\\gamma_a = c\\,\\gamma^b$")
right.axhline(2 - 8, color="black", ls=":", label="$2 - n$ for $n = 8$")
right.set_xticks(positions, [f"$b = x_{k}$" for k in range(8)], fontsize=8)
right.set_ylim(-8.0, 0.5)
right.set_ylabel("factor $c$")
right.set_title("$\\gamma^a\\gamma^b\\gamma_a = -6\\gamma^b$ for every $b$",
                fontsize=10)
right.legend(fontsize=8, loc="upper right")
fig.tight_layout()
save_figure(fig, "gamma_contractions",
            "Left: the contraction $\\gamma^a\\gamma_a$ term by term for $a = x_0$ "
            "to $x_7$: the sign of $(\\gamma^a)^2 = \\pm I$ (blue), $\\eta_{aa}$ "
            "(orange) and their product, the term $\\eta_{aa}(\\gamma^a)^2$ "
            "(green), which is $+I$ for every $a$; the dashed line is the running "
            "sum divided by 8, which reaches 1, so the sum is $8I$. Right: the "
            "number $c$ in $\\gamma^a\\gamma^b\\gamma_a = c\\,\\gamma^b$ for each "
            "$b$ (red dots), computed with the gamma matrices of the Revision "
            "record, and the line $2 - n = -6$ for $n = 8$ dimensions (dotted); "
            "vertical axes pure numbers.")
```

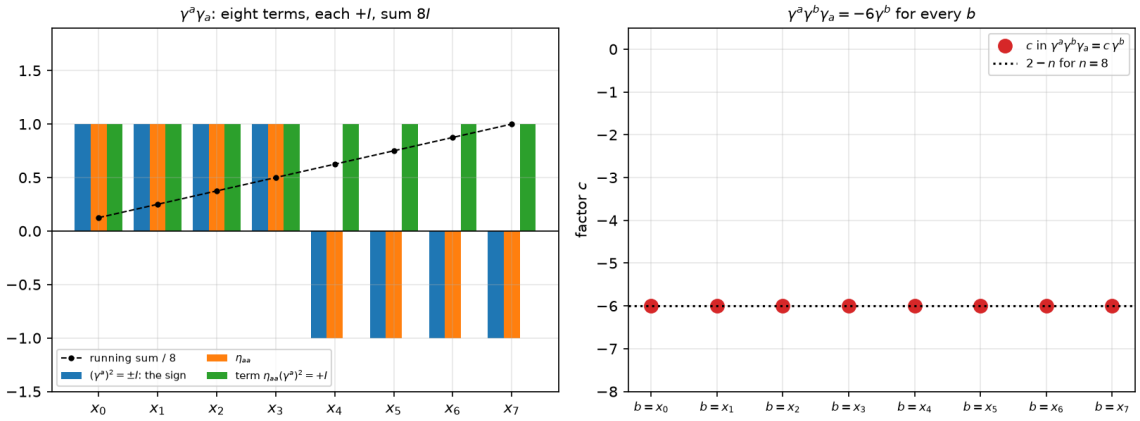

Figure 39: Left: the contraction $\gamma^a\gamma_a$ term by term for $a = x_0$ to x_7: the sign of $(\gamma^a)^2 = \pm I$ (blue), η_{aa} (orange) and their product, the term $\eta_{aa}(\gamma^a)^2$ (green), which is $+I$ for every a; the dashed line is the running sum divided by 8, which reaches 1, so the sum is $8I$. Right: the number c in $\gamma^a\gamma^b\gamma_a = c\,\gamma^b$ for each b (red dots), computed with the gamma matrices of the Revision record, and the line $2 - n = -6$ for $n = 8$ dimensions (dotted); vertical axes pure numbers. (Notebook 01e, figure 5.)

Out [13]:

```
Figure 01e.5 saved as Revision/textbook/figures/01e_5_gamma_contractions.png
```

Text cell:

```
## 11. The author's metric in index notation

The Revision record file Revision/gkd_lovelock/results/curvature.json stores
the diagonal of the author's metric as text in the notation of the program
Mathematica, for example 'E^(2*a4[x4])*Sin[6*H*x0]^(1/3)' and
'-E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*b4[x4]'. The next cell turns the text into
sympy expressions (with the symbols $a_4$, $b_4$ and $z = 6 H x_0$) and checks
them against the metric of section 4. Then:

- the inverse metric of a diagonal metric is diagonal with the inverse entries,
  $g^{\mu\mu} = 1/g_{\mu\mu}$, and $g^{\mu\nu} g_{\nu\lambda} =
  \delta^\mu_\lambda$; the record's report python-lovelock-report.json checks
  this in its check 'metric_inverse';
- the squared length of a small step $dx^\mu$ is the double sum
  $ds^2 = g_{\mu\nu} dx^\mu dx^\nu$, in which only the terms $\nu = \mu$
  survive: $ds^2 = \sum_\mu g_{\mu\mu} (dx^\mu)^2$;
- for the step with the same small size $\epsilon$ in all eight coordinates,
  $dx^\mu = \epsilon$, this is

$$ds^2 = \epsilon^2\big(3e^{2a_4}s - 1 - 3b_4e^{-2a_4}s + \cot^2 z\big),$$

and at $b_4 = 1$, the background of the Revision record,
```

```
$$ds^2 = \epsilon^2\big(6 s \sinh(2a_4) - 1 + \cot^2 z\big),$$

because $e^{x} - e^{-x} = 2\sinh x$ with $x = 2a_4$; and lowering the index,
$dx_\mu = g_{\mu\nu} dx^\nu$, gives the same number as $dx^\mu dx_\mu$.

The cell checks all three statements exactly (in units with $\epsilon = 1$).
```

In [14]:

```
a4 = sp.Symbol("a4", real=True)  # the value of a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of b4(x4) at one time
z = sp.Symbol("z", positive=True)  # z = 6 H x0, between 0 and pi/2
s = sp.sin(z) ** sp.Rational(1, 3)
g_typed = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * s] * 3 + [sp.Integer(-1)]
           + [-b4 * sp.exp(-2 * a4) * s] * 3)  # the author's order x0, x1, ..., x7
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))


def from_record(text):
    """The record's Mathematica text as a sympy expression."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "sin(z)")
    text = text.replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "z": z, "E": sp.E})


g_record = [from_record(text) for text in curvature["metricDiagonal"]]
check(len(g_record) == 8 and
      all(sp.simplify(x - y) == 0 for x, y in zip(g_record, g_typed)),
      "the record's metric diagonal is the author's metric of section 4",
      record="Revision/gkd_lovelock/results/curvature.json, metricDiagonal")
g_lower = sp.diag(*g_record)  # g_mu nu
g_upper = sp.diag(*[1 / entry for entry in g_record])  # g^mu nu
lovelock_report = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json")
    .read_text(encoding="utf-8"))
lovelock_verdicts = {c["name"]: c["verdict"] for c in lovelock_report["checks"]}
check((g_upper * g_lower).applyfunc(sp.simplify) == sp.eye(8)
      and lovelock_verdicts["metric_inverse"] == "PASS",
      "g^mu nu g_nu lambda = delta^mu_lambda",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "metric_inverse")
step = sp.Matrix([1] * 8)  # dx^mu = 1 for every mu (units with epsilon = 1)
ds2 = (step.T * g_lower * step)[0, 0]  # g_mu nu dx^mu dx^nu
step_lower = g_lower * step  # dx_mu = g_mu nu dx^nu
ds2_contracted = (step.T * step_lower)[0, 0]  # dx^mu dx_mu
general_form = 3 * sp.exp(2 * a4) * s - 1 - 3 * b4 * sp.exp(-2 * a4) * s + sp.cot(z) ** 2
closed_form = 6 * s * sp.sinh(2 * a4) - 1 + sp.cot(z) ** 2  # at b4 = 1
say(f"ds^2 for dx^mu = 1: {ds2}")
check(sp.expand(ds2 - general_form) == 0
      and sp.expand((ds2.subs(b4, 1) - closed_form).rewrite(sp.exp)) == 0
      and sp.expand(ds2_contracted - ds2) == 0,
```

```
      "ds^2 = 3 e^(2 a4) s - 1 - 3 b4 e^(-2 a4) s + cot(z)^2 for the step dx^mu = 1, "
      "= 6 s sinh(2 a4) - 1 + cot(z)^2 at b4 = 1, and dx^mu dx_mu gives the same")
```

Out [14]:

```
PASS the record's metric diagonal is the author's metric of section 4
     reproduces Revision/gkd_lovelock/results/curvature.json, metricDiagonal
PASS g^mu nu g_nu lambda = delta^mu_lambda
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    metric_inverse
ds^2 for dx^mu = 1: -3*b4*exp(-2*a4)*sin(z)**(1/3) + 3*exp(2*a4)*sin(z)**(1/3) +
    cot(z)**2 - 1
PASS ds^2 = 3 e^(2 a4) s - 1 - 3 b4 e^(-2 a4) s + cot(z)^2 for the step dx^mu = 1, = 6 s
    sinh(2 a4) - 1 + cot(z)^2 at b4 = 1, and dx^mu dx_mu gives the same
```

Text cell:

```
The rest of this section takes $b_4 = 1$. For each $z$, $ds^2$ grows when $a_4$
grows ($\sinh$ is an increasing function and $6s > 0$), from very negative values
(time-like steps) to very positive ones (space-like steps). It is zero where
$6 s \sinh(2a_4) = 1 - \cot^2 z$, that is at

$$a_4^{\ast}(z) = \tfrac12\,\mathrm{arcsinh}\!\left(\frac{1 - \cot^2 z}{6 s}\right),$$

where $\mathrm{arcsinh}$ undoes $\sinh$. The next cell evaluates $ds^2$ on a
grid of 301 values of $a_4$ from $-1.5$ to $1.5$ and 300 values of $z$, checks
that it grows with $a_4$ on every row of the grid and that its sign changes
exactly at $a_4^{\ast}(z)$, and draws two pictures: the sign map with the curve
$a_4^{\ast}(z)$, and at $z = 0.9$ the four kinds of terms of the sum against
$a_4$: the growing term of the three space directions, the constant term of
the time, the shrinking term of the three deflating extra times, and the
constant term of the hidden direction.
```

In [15]:

```
ds2_numbers = sp.lambdify((a4, z), closed_form, "numpy")  # a formula -> a function
a4_values = np.linspace(-1.5, 1.5, 301)
z_values = np.linspace(0.15, np.pi / 2 - 0.005, 300)
A4_grid, Z_grid = np.meshgrid(a4_values, z_values)  # rows: z, columns: a4
values = ds2_numbers(A4_grid, Z_grid)
threshold = 0.5 * np.arcsinh((1 - 1 / np.tan(z_values) ** 2)
                             / (6 * np.sin(z_values) ** (1 / 3)))
check(bool((np.diff(values, axis=1) > 0).all()),
      "on every row of the grid, ds^2 grows as a4 grows")
away = np.abs(A4_grid - threshold[:, None]) > 1e-9  # leave out points on the curve
check(bool(((values > 0) == (A4_grid > threshold[:, None]))[away].all()),
      "ds^2 > 0 exactly to the right of the curve a4*(z)")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.5, 4.5))
image = left.imshow(np.tanh(values), origin="lower", aspect="auto", cmap="RdBu_r",
                    vmin=-1, vmax=1, extent=(a4_values[0], a4_values[-1],
                                             z_values[0], z_values[-1]))
```

```
left.plot(threshold, z_values, color="black", lw=1.5, label="$ds^2 = 0$")
left.set_xlim(a4_values[0], a4_values[-1])
left.grid(False)  # no grid lines across the coloured map
left.text(0.75, 0.4, "space-like\n$ds^2 > 0$", ha="center", color="white")
left.text(-0.9, 1.2, "time-like\n$ds^2 < 0$", ha="center", color="white")
left.set_xlabel("$a_4$")
left.set_ylabel("$z = 6 H x_0$")
left.set_title("sign of $ds^2$ for the step $dx^\\mu = \\epsilon$", fontsize=10)
left.legend(loc="upper right", fontsize=8)
fig.colorbar(image, ax=left, label="$\\tanh(ds^2/\\epsilon^2)$")
z_fixed = 0.9
s_fixed = np.sin(z_fixed) ** (1 / 3)
space_term = 3 * np.exp(2 * a4_values) * s_fixed  # x1, x2, x3
extra_term = -3 * np.exp(-2 * a4_values) * s_fixed  # x5, x6, x7 (deflating)
hidden_term = np.full_like(a4_values, 1 / np.tan(z_fixed) ** 2)  # x0
right.plot(a4_values, space_term, color="tab:red", label="space $x_1, x_2, x_3$")
right.plot(a4_values, np.full_like(a4_values, -1.0), color="tab:gray",
           label="time $x_4$")
right.plot(a4_values, extra_term, color="tab:blue",
           label="extra times $x_5, x_6, x_7$")
right.plot(a4_values, hidden_term, color="tab:green", label="hidden $x_0$")
right.plot(a4_values, space_term - 1 + extra_term + hidden_term, "k", lw=2.5,
           label="sum $ds^2/\\epsilon^2$")
right.axhline(0, color="black", lw=0.8)
right.set_ylim(-12, 12)
right.set_xlabel("$a_4$")
right.set_ylabel("contribution to $ds^2/\\epsilon^2$")
right.set_title(f"the terms of $g_{{\\mu\\nu}} dx^\\mu dx^\\nu$ at $z = {z_fixed}$",
                fontsize=10)
right.legend(fontsize=8, loc="lower right")
fig.tight_layout()
save_figure(fig, "step_length",
            "The squared length $ds^2 = g_{\\mu\\nu} dx^\\mu dx^\\nu$ of the "
            "coordinate step with $dx^\\mu = \\epsilon$ in all eight coordinates, "
            "in the author's metric at $b_4 = 1$. Left: $\\tanh(ds^2/\\epsilon^2)$ "
            "as colours "
            "(red space-like, blue time-like) against $a_4$ (horizontal) and "
            "$z = 6 H x_0$ (vertical), pure numbers, with the curve "
            "$a_4^{\\ast}(z)$ where $ds^2 = 0$ (black). Right: at $z = 0.9$ the four "
            "kinds of terms against $a_4$: the three space directions (red, growing "
            "like $e^{2a_4}$), the time (grey, $-1$), the three extra times (blue, "
            "shrinking towards 0 like $e^{-2a_4}$ as they deflate) and the hidden "
            "direction (green), and their sum (black). As $a_4$ grows the same "
            "coordinate step turns from time-like to space-like.")
```

Out [15]:

```
PASS on every row of the grid, ds^2 grows as a4 grows
PASS ds^2 > 0 exactly to the right of the curve a4*(z)
```

Out [15]:

```
Figure 01e.6 saved as Revision/textbook/figures/01e_6_step_length.png
```

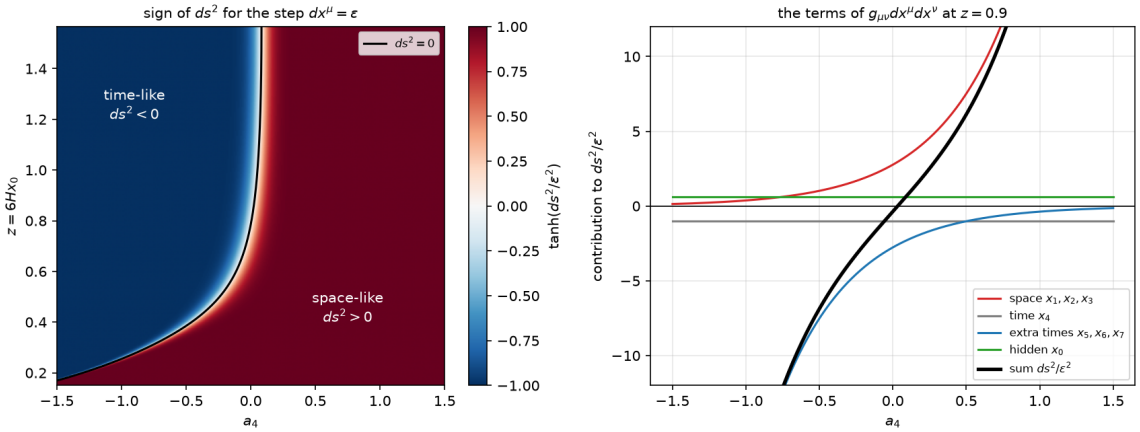

Figure 40: The squared length $ds^2 = g_{\mu\nu}dx^\mu dx^\nu$ of the coordinate step with $dx^\mu = \epsilon$ in all eight coordinates, in the author's metric at $b_4 = 1$. Left: $\tanh(ds^2/\epsilon^2)$ as colours (red space-like, blue time-like) against a_4 (horizontal) and $z = 6Hx_0$ (vertical), pure numbers, with the curve $a_4^*(z)$ where $ds^2 = 0$ (black). Right: at $z = 0.9$ the four kinds of terms against a_4: the three space directions (red, growing like e^{2a_4}), the time (grey, -1), the three extra times (blue, shrinking towards 0 like e^{-2a_4} as they deflate) and the hidden direction (green), and their sum (black). As a_4 grows the same coordinate step turns from time-like to space-like. (Notebook 01e, figure 6.)

Text cell:

```
## 12. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [16]:

```
figure_names = ["01e_1_eta_and_delta.png", "01e_2_squared_lengths.png",
                "01e_3_symmetric_antisymmetric.png", "01e_4_clifford_tables.png",
                "01e_5_gamma_contractions.png", "01e_6_step_length.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

Out [16]:

```
PASS all 6 figure files of this notebook exist
ALL 25 CHECKS PASSED (notebook 01e)
```

Text cell:

```
## 13. What this notebook showed
```

```
- An index that appears once in every term is free (the formula holds for each
  of its values: $8^k$ equations for $k$ free indices); an index that appears
  twice is a dummy and is summed (the summation convention). numpy's 'einsum'
  evaluates such sums from the list of indices; renaming a dummy changes
  nothing.
- The frame metric of the Revision record is $\eta =
  \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$; $\eta^{ab}$ has the same
  entries, $\eta^{ab}\eta_{bc} = \delta^a_c$ and $\delta^a_a = 8$. Lowering an
  index flips the signs of the four time-like components $x_4, \dots, x_7$.
- The squared length $\eta_{ab}v^a v^b = v^a v_a$ sorts vectors into
  space-like, time-like and light-like ones; in the signature (4,4) exchanging
  the space-like and the time-like components turns $Q$ into $-Q$, so exactly
  half of all random vectors are space-like (measured: 0.50), while in ordinary
  spacetime about 82 per cent are.
- A symmetric array contracted with an antisymmetric one gives 0; so only the
  symmetric part of an array counts in $v^a v^b M_{ab}$ for commuting
  components.
- The eight gamma matrices of the Revision record obey
  $\{\gamma^a, \gamma^b\} = 2\eta^{ab} I$ for all 64 pairs; they are orthogonal,
  symmetric for the space-like and antisymmetric for the time-like directions;
  and $\gamma^a\gamma_a = 8I$, $\gamma^a\gamma^b\gamma_a = -6\gamma^b$,
  $\mathrm{tr}(\gamma^a\gamma^b) = 16\eta^{ab}$, derived line by line and
  checked.
- In the author's metric the step $dx^\mu = \epsilon$ has
  $ds^2 = \epsilon^2(3e^{2a_4}s - 1 - 3b_4e^{-2a_4}s + \cot^2 z)$, which is
  $\epsilon^2(6 s\sinh(2a_4) - 1 + \cot^2 z)$ at $b_4 = 1$; as $a_4$ grows, the term
  of the three space directions grows like $e^{2a_4}$ and the term of the
  three exponentially deflating extra times shrinks like $e^{-2a_4}$, and the
  step turns from time-like to space-like at $a_4^{\ast}(z)$.
```

1.33 Line-by-line walk-through of Notebook 01e

The notebook has sixteen code cells, In [1] to In [16]. As in Section 1.9, a quoted line `...)` stands for the remaining lines of a figure caption, which Section 1.32 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 1.31; its code is the set-up code explained in Section 1.9, with `NOTEBOOK_ID = "01e"`. It prints Set-up of notebook 01e complete: repository folder found, helpers defined.

In [2], free and dummy indices.

```
import itertools  # all pairs (a, b) of indices
import math  # pi and square roots of plain numbers

import numpy as np  # arrays and einsum
import sympy as sp  # exact algebra with symbols
```

The modules of this notebook: `itertools.product` will produce all pairs of indices, `math` gives π and square roots of single numbers.

```
M = np.array([[1, 2, 0],
              [0, 1, 1],
```

```
              [1, 3, 1]])  # the entries M^a_b: row a, column b
v = np.array([2, -1, 3])  # the components v^b
u = np.array([1, 0, 2])  # the components u_a of a second vector
```

The table $M^a{}_b$, the vector v^b and the lower list u_a of the example of Section 1.26.

```
# w^a = M^a_b v^b: the free index a is kept, the dummy index b is summed.
w_loops = [sum(int(M[a, b]) * int(v[b]) for b in range(3)) for a in range(3)]
w_einsum = np.einsum("ab,b->a", M, v)  # "b" appears twice: it is summed
w_renamed = np.einsum("ac,c->a", M, v)  # the dummy renamed to c: the same sum
```

w^a three ways: with loops (for each free value a, the sum over the dummy b); with `einsum`, whose text `"ab,b->a"` lists the indices of each factor and of the result (the letter b appears in both factors and not in the result, so it is summed); and with the dummy renamed to c.

```
say(f"w = M v by loops {w_loops}, by einsum {w_einsum.tolist()}, with the dummy "
    f"renamed {w_renamed.tolist()}")
check(w_loops == w_einsum.tolist() == w_renamed.tolist() == (M @ v).tolist()
      == [0, 2, 2],
      "loops, einsum and @ give w = (0, 2, 2); renaming the dummy changes nothing")
```

The three results are printed, each `[0, 2, 2]`, and checked against each other, against `M @ v` and against the hand calculation.

```
trace = int(np.einsum("aa->", M))  # M^a_a: the repeated index a is summed
full = int(np.einsum("a,ab,b->", u, M, v))  # u_a M^a_b v^b: two dummies, no free
say(f"M^a_a = {trace}, u_a M^a_b v^b = {full}")
check(trace == int(np.trace(M)) == 3 and full == int(u @ M @ v) == 4,
      "the trace M^a_a = 3 and the number u_a M^a_b v^b = 4")
```

`"aa->"` contracts the two indices of one table: the trace. `"a,ab,b->"` contracts two pairs and keeps no index (nothing after the arrow): a single number. Output: `M^a_a = 3, u_a M^a_b v^b = 4`, and the PASS line.

```
for free in range(4):
    word = "equation" if free == 0 else "equations"  # 8^0 = 1 equation
    say(f"{free} free indices over the 8 coordinates: 8^{free} = {8 ** free} {word}")
```

Four lines: a formula with 0, 1, 2, 3 free indices stands for 1, 8, 64, 512 equations (the singular "equation" for 1). The cell prints two PASS lines in all.

In [3], the frame metric from the record.

```
algebra = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
names = algebra["coordinates"]  # ["x0", "x1", ..., "x7"]
eta_diagonal = algebra["eta"]  # the diagonal of eta in the order x0 ... x7
```

The record file of the gamma matrices, read as in Notebook 01a; its coordinate names and the diagonal of η.

```
algebra_report = json.loads(repository_file(
    "Revision/algebra/reports/python-algebra.json").read_text(encoding="utf-8"))
verdicts = {c["name"]: c["verdict"] for c in algebra_report["checks"]}
map_verdict = verdicts["coordinate_map"]  # "pass" in the record
say(f"coordinates {names}")
say(f"diagonal of eta {eta_diagonal}; record check coordinate_map: {map_verdict}")
```

The record's report: its list of checks is turned into a dictionary from the check names to their verdicts (this report writes the verdict in small letters, `pass`). Output: the eight names, the diagonal `[1, 1, 1, 1, -1, -1, -1, -1]` and the record's verdict pass.

```
check(eta_diagonal == [1, 1, 1, 1, -1, -1, -1, -1]
      and verdicts["coordinate_map"] == "pass",
      "eta = diag(+1, +1, +1, +1, -1, -1, -1, -1) in the order x0 ... x7",
      record="Revision/algebra/reports/python-algebra.json, check coordinate_map")
```

The diagonal of Section 1.27 and the record's verdict; this check reproduces the record's check `coordinate_map`.

```
eta_lower = np.diag(eta_diagonal)  # eta_ab as an 8 x 8 matrix
eta_upper = np.diag([1 // e for e in eta_diagonal])  # eta^ab: 1/(+1) = 1, 1/(-1) = -1
delta = np.einsum("ab,bc->ac", eta_upper, eta_lower)  # eta^ab eta_bc
check((delta == np.eye(8, dtype=int)).all() and int(np.einsum("aa->", delta)) == 8,
      "eta^ab eta_bc = delta^a_c, and the contraction delta^a_a = 8")
```

η_{ab} and η^{ab} as 8×8 tables; the inverse entries are computed with the whole-number division `//`, which gives $1//1 = 1$ and $1//(-1) = -1$ exactly. The contraction $\eta^{ab}\eta_{bc}$ must be the identity, and $\delta^a{}_a$ must be 8.

```
v_up = np.arange(1, 9)  # v^a = 1, 2, ..., 8
v_down = np.einsum("ab,b->a", eta_lower, v_up)  # v_a = eta_ab v^b
v_back = np.einsum("ab,b->a", eta_upper, v_down)  # eta^ab v_b
flipped = [names[a] for a in range(8) if v_down[a] != v_up[a]]
say(f"v^a = {v_up.tolist()}")
say(f"v_a = {v_down.tolist()}; the components that changed sign: {flipped}")
check(flipped == ["x4", "x5", "x6", "x7"] and (v_back == v_up).all(),
      "lowering flips the signs of the time-like x4 ... x7; raising undoes it")
```

$v^a = (1, \ldots, 8)$, lowered and raised again with `einsum`; `flipped` lists the names of the components that changed. Output: `v_a = [1, 2, 3, 4, -5, -6, -7, -8]`, the changed components x4 to x7, and the PASS line; the cell prints three PASS lines in all.

In [4], the first heat maps.

```
labels = [f"$x_{k}$" for k in range(8)]  # x with the subscripts 0 ... 7
```

```
def heat_map(ax, matrix, title, ticks=labels, annotate=True, size=7.0):
    """Draw matrix on the axes ax: red positive, white zero, blue negative; with
    annotate the value is written in its square (%g: no needless digits)."""
    m = np.asarray(matrix, dtype=float)
    limit = max(1.0, float(np.abs(m).max()))  # colours from -limit to +limit
    ax.imshow(m, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.grid(False)  # no grid lines across the coloured squares
    ax.set_xticks(range(m.shape[1]), ticks[:m.shape[1]], fontsize=7)
    ax.set_yticks(range(m.shape[0]), ticks[:m.shape[0]], fontsize=7)
    ax.set_title(title, fontsize=10)
```

The labels $x_0, \ldots, x_7$ for the axes, and a heat-map helper like the one of Notebook 01a (Section 1.25), with the coordinate names as default labels (`ticks=labels`) and an adjustable size of the written values.

```
    if annotate:
        for i in range(m.shape[0]):
            for j in range(m.shape[1]):
                ink = "white" if abs(m[i, j]) > 0.6 * limit else "black"
                ax.text(j, i, f"{m[i, j]:g}", ha="center", va="center",
                        fontsize=size, color=ink)
```

The value of every entry is written in its square with the format `:g`, which drops needless digits (1.0 is written 1).

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
heat_map(axes[0], eta_lower, "$\\eta_{ab}$ (row $a$, column $b$)")
heat_map(axes[1], delta, "$\\eta^{ab}\\eta_{bc} = \\delta^a_c$ (row $a$, column $c$)")
positions = np.arange(8)
axes[2].bar(positions - 0.2, v_up, width=0.4, label="$v^a$ (upper index)")
axes[2].bar(positions + 0.2, v_down, width=0.4, label="$v_a = \\eta_{ab} v^b$")
```

Three panels: the heat maps of η_{ab} and of $\eta^{ab}\eta_{bc}$, and a bar chart of the two lists v^a and v_a side by side (`ax.bar(positions, heights, width=...)` draws bars; shifting the positions by ∓ 0.2 puts the two bars of each coordinate next to each other).

```
axes[2].axhline(0, color="black", lw=0.8)
axes[2].set_xticks(positions, labels)
axes[2].set_ylabel("component")
axes[2].set_title("lowering the index of $v$", fontsize=10)
axes[2].legend(fontsize=8, loc="lower left")
fig.tight_layout()
save_figure(fig, "eta_and_delta",
            "Left: the frame metric $\\eta_{ab}$ of the author's coordinates as a "
            ...)
```

A zero line, labels and the figure. What Figure 01e.1 shows: η red $(+1)$ for x_0, x_1, x_2, x_3 and blue (-1) for x_4 to x_7; the contraction is the identity; the orange bars of v_a point down exactly at the four time-like coordinates.

In [5], squared lengths of five vectors.

```
def squared_length(vectors, metric_diagonal):
    """eta_ab v^a v^b for every row v of the array vectors (a and b are summed;
    the index n, which numbers the rows, is kept)."""
    metric = np.diag(metric_diagonal)
    return np.einsum("na,ab,nb->n", vectors, metric, vectors)
```

$Q(v) = \eta_{ab} v^a v^b$ for many vectors at once: the array `vectors` has one vector per row, numbered by `n`; in `"na,ab,nb->n"` the letters a and b appear twice and are summed, while n is kept, so the result is one number per vector.

```
unit = np.eye(8, dtype=int)  # row k is the unit vector along x_k
examples = {"e(x1)": unit[1], "e(x5)": unit[5], "e(x1) + e(x5)": unit[1] + unit[5],
            "(1, 1, 1, 1, 1, 1, 1, 1)": np.ones(8, dtype=int),
            "(1, 2, 3, 4, 5, 6, 7, 8)": v_up}
kinds = []
```

The rows of the identity are the unit vectors, row k along x_k (the author's index and Python's agree); the dictionary holds the five examples of Section 1.27 with their names (`np.ones(8)` is the vector of eight ones).

```
for name, vector in examples.items():
    q = int(squared_length(vector[None, :], eta_diagonal)[0])  # one row
    kind = "space-like" if q > 0 else ("time-like" if q < 0 else "light-like")
    kinds.append(kind)
    q_text = f"{q:+d}" if q != 0 else "0"  # a sign in front of nonzero values only
    say(f"v = {name}: eta_ab v^a v^b = {q_text}, {kind}")
```

For each example: `vector[None, :]` makes the vector a table with one row, as `squared_length` expects, and `[0]` takes the one result. The kind follows from the sign of Q (two nested `a if condition else b`). Output: +1 space-like, -1 time-like, 0 light-like, 0 light-like, -144 time-like.

```
check(kinds == ["space-like", "time-like", "light-like", "light-like", "time-like"]
      and int(squared_length(v_up[None, :], eta_diagonal)[0]) == int(v_up @ v_down)
      == -144,
      "the five examples are sorted correctly, and eta_ab v^a v^b = v^a v_a = -144")
```

The sorting, and $\eta_{ab} v^a v^b = v^a v_a = -144$ for the last example, with the lowered list of In [3]. One PASS line.

In [6], how many vectors are space-like.

```
generator = np.random.default_rng(12345)  # random numbers with a fixed seed
samples = generator.normal(size=(200000, 8))  # 200,000 vectors of 8 components
q44 = squared_length(samples, eta_diagonal)  # signature (4,4)
space_part, time_part = [0, 1, 2, 3], [4, 5, 6, 7]  # x0 x1 x2 x3, and x4 ... x7
exchanged = samples.copy()
exchanged[:, space_part] = samples[:, time_part]  # time components -> space slots
exchanged[:, time_part] = samples[:, space_part]  # space components -> time slots
```

200,000 vectors of eight normal random numbers (seed 12345) and their squared lengths. `space_part` and `time_part` are Python's places of the space-like components $v^0, \ldots, v^3$ and of the time-like components $v^4, \ldots, v^7$. `exchanged` is a copy in which the four time-like components are written into the space-like places and the four space-like ones into the time-like places (`[:, space_part]` selects these columns in every row).

```
check(np.allclose(squared_length(exchanged, eta_diagonal), -q44, rtol=0, atol=1e-9),
      "exchanging the space-like and the time-like components turns Q into -Q "
      "(all 200000 vectors)")
```

The exchange rule $Q(v') = -Q(v)$ of Section 1.27 for every one of the vectors (`np.allclose` compares two arrays entry by entry with the stated tolerances: `rtol=0`, no relative tolerance, `atol=1e-9`, an absolute one of 10^{-9}).

```
fraction44 = float(np.mean(q44 > 0))  # the fraction of space-like vectors
q31 = squared_length(samples[:, :4], [1, 1, 1, -1])  # 3 space, 1 time direction
fraction31 = float(np.mean(q31 > 0))
exact31 = 0.5 + 1 / math.pi
sd44 = math.sqrt(0.5 * 0.5 / 200000)  # one standard deviation of a fraction
sd31 = math.sqrt(exact31 * (1 - exact31) / 200000)
```

`np.mean(q44 > 0)` is the mean of a table of True (counted as 1) and False (0): the fraction of space-like vectors. For the signature (3,1) the first four components of each vector are used with the metric $\mathrm{diag}(1, 1, 1, -1)$. `exact31` is $1/2 + 1/\pi$, and `sd44`, `sd31` are the standard deviations $\sqrt{p(1-p)/N}$ of Section 1.27.

```
report("fraction of space-like random vectors, signature (4,4)", f"{fraction44:.4f}")
report("fraction of space-like random vectors, signature (3,1)", f"{fraction31:.4f}")
check(abs(fraction44 - 0.5) < 4 * sd44,
      "signature (4,4): half of the random vectors are space-like")
check(abs(fraction31 - exact31) < 4 * sd31,
      "signature (3,1): the fraction is 1/2 + 1/pi = 0.818 (within 4 standard "
      "deviations)")
```

The RESULT lines print 0.5011 and 0.8177; the checks require each to lie within 4 standard deviations of its expected value. The cell prints three PASS lines.

In [7], the two histograms.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
bins = np.linspace(-15.0, 15.0, 121)  # 120 intervals of width 0.25
for ax, values, fraction, title, color in (
        (axes[0], q44, fraction44, "signature (4,4): 4 space-like, 4 time-like",
         "tab:purple"),
        (axes[1], q31, fraction31, "signature (3,1): 3 space-like, 1 time-like",
         "tab:green")):
    ax.hist(values, bins=bins, density=True, color=color, alpha=0.75)
    ax.axvline(0.0, color="black", lw=1)
```

Two panels with a shared vertical axis (`sharey=True`). The 121 edges `bins` cut the range -15 to 15 into 120 intervals; `ax.hist` counts how many values fall into each interval and draws the counts as bars, scaled (`density=True`) so that the bars show the fraction of vectors per unit of Q. A black line marks $Q = 0$.

```
    ax.text(0.97, 0.95, f"space-like: {fraction:.3f}", transform=ax.transAxes,
            ha="right", va="top")
    ax.text(0.03, 0.95, f"time-like: {1 - fraction:.3f}", transform=ax.transAxes,
            ha="left", va="top")
    ax.set_xlabel("squared length $Q = \\eta_{ab} v^a v^b$")
    ax.set_title(title, fontsize=10)
axes[0].set_ylabel("fraction of vectors per unit of $Q$")
fig.tight_layout()
save_figure(fig, "squared_lengths",
            "Histograms of the squared length $Q = \\eta_{ab} v^a v^b$ of 200,000 "
            ...)
```

The measured fractions are written into the upper corners (`transform=ax.transAxes` measures the position in fractions of the panel, 0 to 1, instead of in data units). What Figure 01e.2 shows: on the left a histogram that is the same on both sides of 0 (half space-like, half time-like); on the right one that leans to the positive side (82 per cent space-like).

In [8], symmetric times antisymmetric, exactly.

```
M8 = generator.integers(-5, 6, size=(8, 8))  # a random 8 x 8 array, entries -5 ... 5
S2 = M8 + M8.T  # twice the symmetric part
A2 = M8 - M8.T  # twice the antisymmetric part
check((S2 == S2.T).all() and (A2 == -A2.T).all() and (S2 + A2 == 2 * M8).all(),
      "M = S + A with S = (M + M^T)/2 symmetric and A = (M - M^T)/2 antisymmetric")
```

A random 8×8 array of whole numbers. The cell works with $2S$ and $2A$, which are whole numbers, so that every computation is exact. The check: $2S$ is symmetric, $2A$ antisymmetric, and $2S + 2A = 2M$.

```
A2_up = np.einsum("ac,bd,cd->ab", eta_upper, eta_upper, A2)  # A^ab (times 2)
terms = S2 * A2_up  # the 64 terms S_ab A^ab (times 4), before the sum
total = int(np.einsum("ab,ab->", S2, A2_up))  # the full contraction (times 4)
say(f"4 S_ab A^ab = {total}; the largest single term is "
    f"{int(np.abs(terms).max())}")
```

$A^{ab} = \eta^{ac}\eta^{bd}A_{cd}$ (both indices raised; `einsum` with two summed pairs), the 64 single products $S_{ab}A^{ab}$ (`*` between arrays multiplies entry by entry) and their sum. Output: `4 S_ab A^ab = 0; the largest single term is 24`: the terms are not zero, but they cancel in pairs.

```
check((A2_up == -A2_up.T).all() and (terms == -terms.T).all() and total == 0,
      "A^ab is antisymmetric, the terms (a, b) and (b, a) cancel, S_ab A^ab = 0")
```

The three statements of Section 1.28: raising keeps antisymmetry; the term (a, b) is minus the term (b, a); the sum is 0.

```
all_zero = True
for _ in range(100):  # 100 more random pairs of arrays
    X = generator.integers(-9, 10, size=(8, 8))
    Y = generator.integers(-9, 10, size=(8, 8))
    all_zero &= int(np.einsum("ab,ac,bd,cd->", X + X.T, eta_upper, eta_upper,
                              Y - Y.T)) == 0
check(all_zero, "S_ab A^ab = 0 for 100 more random pairs of arrays")
```

The theorem for 100 more random pairs: $X + X^T$ is symmetric, $Y - Y^T$ antisymmetric, and the contraction with both indices of the second raised is computed in one `einsum` (the loop variable `_` is not used).

```
squares_ok = True
for _ in range(100):  # 100 random vectors with whole-number components
    w8 = generator.integers(-9, 10, size=8)
    squares_ok &= int(np.einsum("a,b,ab->", w8, w8, A2)) == 0
    squares_ok &= (int(np.einsum("a,b,ab->", w8, w8, 2 * M8))
                   == int(np.einsum("a,b,ab->", w8, w8, S2)))
check(squares_ok, "v^a v^b A_ab = 0 and v^a v^b M_ab = v^a v^b S_ab (100 random "
      "vectors with commuting components)")
```

The consequence for commuting components: for 100 random vectors $v^a v^b A_{ab} = 0$ and $v^a v^b (2M_{ab}) = v^a v^b (2S_{ab})$. The cell prints four PASS lines.

In [9], the picture of the cancellation.

```
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.7))
heat_map(axes[0], M8, "$M_{ab}$", size=6.0)
heat_map(axes[1], S2 / 2, "symmetric part $S_{ab}$", size=5.5)
heat_map(axes[2], A2 / 2, "antisymmetric part $A_{ab}$", size=5.5)
heat_map(axes[3], terms / 4, "terms $S_{ab} A^{ab}$ (sum 0)", annotate=False)
fig.tight_layout()
save_figure(fig, "symmetric_antisymmetric",
            "A random $8 \\times 8$ array $M_{ab}$ of whole numbers (left), its "
            ...)
```

Four heat maps: M, S, A (the doubled arrays divided by 2) and the 64 terms (divided by 4), the last without written values. What Figure 01e.3 shows: S looks the same mirrored in the diagonal; A changes colour under the mirror and has a white diagonal; in the right panel every coloured square has a square of the opposite colour at its mirror place.

In [10], commutators and the Clifford relation of the record.

```
P = generator.integers(-3, 4, size=(4, 4))
R = generator.integers(-3, 4, size=(4, 4))
commutator = P @ R - R @ P
anticommutator = P @ R + R @ P
check((2 * (P @ R) == commutator + anticommutator).all()
      and (R @ P - P @ R == -commutator).all()
      and (R @ P + P @ R == anticommutator).all(),
      "P R = [P, R]/2 + {P, R}/2, [R, P] = -[P, R] and {R, P} = {P, R}")
```

Two random 4×4 integer matrices, their commutator and anticommutator, and the three rules of Section 1.28 (the first written as $2PR = [P, R] + \{P, R\}$ to stay with whole numbers).

```
gamma = [np.array(matrix, dtype=int) for matrix in algebra["gamma"]]  # gamma^a
identity16 = np.eye(16, dtype=int)
anti_table = np.zeros((8, 8), dtype=int)  # {gamma^a, gamma^b} = anti_table[a, b] I
commutator_size = np.zeros((8, 8), dtype=int)  # nonzero entries of the commutator
clifford_holds = True
```

The eight gamma matrices of the record; two 8×8 tables to be filled: the number c_{ab} in $\{\gamma^a, \gamma^b\} = c_{ab} I$, and the number of nonzero entries of each commutator.

```
for a, b in itertools.product(range(8), repeat=2):  # all 64 ordered pairs
    anti = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
    anti_table[a, b] = anti[0, 0]  # the number in front of I
    clifford_holds &= bool((anti == 2 * eta_upper[a, b] * identity16).all())
    commutator_size[a, b] = np.count_nonzero(gamma[a] @ gamma[b]
                                             - gamma[b] @ gamma[a])
```

`itertools.product(range(8), repeat=2)` produces all 64 ordered pairs (a, b). For each: the anticommutator; its top-left entry (if it is $c_{ab} I$, every diagonal entry is c_{ab}); the exact comparison with $2\eta^{ab} I$; and `np.count_nonzero` of the commutator.

```
check(clifford_holds and verdicts["clifford_relation"] == "pass",
      "{gamma^a, gamma^b} = 2 eta^ab I for all 64 ordered pairs (a, b)",
      record="Revision/algebra/reports/python-algebra.json, check "
             "clifford_relation")
```

All 64 relations hold, and the record's verdict is pass: this reproduces the record's check `clifford_relation`.

```
orthogonal = all((g.T @ g == identity16).all() for g in gamma)
transpose_sign = [1 if (g.T == g).all() else (-1 if (g.T == -g).all() else 0)
                  for g in gamma]  # +1 symmetric, -1 antisymmetric
say("symmetric: " + ", ".join(names[a] for a in range(8) if transpose_sign[a] == 1)
    + "; antisymmetric: "
    + ", ".join(names[a] for a in range(8) if transpose_sign[a] == -1))
```

Each gamma matrix is tested for $O^T O = I$, and `transpose_sign` records $+1$ for a symmetric matrix, -1 for an antisymmetric one (and 0 for neither, which does not occur). Output: symmetric: x0, x1, x2, x3; antisymmetric: x4, x5, x6, x7.

```
check(orthogonal and transpose_sign == eta_diagonal
      and verdicts["symmetry_pattern"] == "pass",
      "each gamma^a is orthogonal and (gamma^a)^T = eta_aa gamma^a: symmetric "
      "for x0, x1, x2, x3, antisymmetric for x4 ... x7",
      record="Revision/algebra/reports/python-algebra.json, check "
             "symmetry_pattern")
```

The theorem $(\gamma^a)^T = \eta_{aa}\gamma^a$ of Section 1.28 (the list of signs equals the diagonal of η), reproducing the record's check `symmetry_pattern`. The cell prints three PASS lines.

In [11], the Clifford tables as pictures.

```
fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.0),
                         gridspec_kw={"width_ratios": [1, 1, 1.2]})
heat_map(axes[0], anti_table, "$\\{\\gamma^a, \\gamma^b\\} = c_{ab} I$: $c_{ab}$")
heat_map(axes[1], commutator_size,
         "nonzero entries of $[\\gamma^a, \\gamma^b]$", size=6.0)
```

Three panels, the third a little wider (`width_ratios`). The table c_{ab} and the table of commutator sizes as heat maps.

```
axes[2].bar(positions - 0.2, transpose_sign, width=0.4, color="tab:blue",
            label="$s_a$ in $(\\gamma^a)^T = s_a \\gamma^a$")
axes[2].bar(positions + 0.2, eta_diagonal, width=0.4, color="tab:orange",
            label="$\\eta_{aa}$")
axes[2].axhline(0, color="black", lw=0.8)
axes[2].set_xticks(positions, labels)
axes[2].set_ylim(-1.6, 1.6)
axes[2].set_title("symmetric ($+1$) or antisymmetric ($-1$)", fontsize=10)
axes[2].legend(fontsize=8, loc="lower left")
fig.tight_layout()
save_figure(fig, "clifford_tables",
            "The Clifford relation of the eight real $16 \\times 16$ gamma matrices "
            ...)
```

The third panel draws, for each coordinate, the transpose sign next to η_{aa}. What Figure 01e.4 shows: c_{ab} is ± 2 on the diagonal and 0 elsewhere; every commutator of two different gamma matrices has 16 nonzero entries and the diagonal (a matrix with itself) has none; the blue and orange bars are equal for every coordinate.

In [12], the three contractions.

```
gamma_lower = [sum(eta_lower[a, b] * gamma[b] for b in range(8)) for a in range(8)]
contraction = sum(gamma[a] @ gamma_lower[a] for a in range(8))  # gamma^a gamma_a
check((contraction == 8 * identity16).all(), "gamma^a gamma_a = 8 I")
```

$\gamma_a = \eta_{ab}\gamma^b$ as a list of eight matrices (the sum over b written out), then $\gamma^a\gamma_a$ (the sum over a) and the first contraction, $8I$.

```
lowered_ok = all((gamma[b] @ gamma_lower[a] + gamma_lower[a] @ gamma[b]
                  == 2 * int(a == b) * identity16).all()
                 for a, b in itertools.product(range(8), repeat=2))
sandwich = [sum(gamma[a] @ gamma[b] @ gamma_lower[a] for a in range(8))
            for b in range(8)]  # gamma^a gamma^b gamma_a for each b
```

`lowered_ok` tests the lowered Clifford relation $\gamma^b\gamma_a + \gamma_a\gamma^b = 2\delta^b{}_a I$ for all 64 pairs (`int(a == b)` is 1 for equal indices and 0 otherwise: the Kronecker delta). `sandwich` holds $\gamma^a\gamma^b\gamma_a$ for each b.

```
factors = [int(sandwich[b][np.nonzero(gamma[b])][0]
               // gamma[b][np.nonzero(gamma[b])][0]) for b in range(8)]
say(f"gamma^a gamma^b gamma_a = c gamma^b with c = {factors} for b = x0 ... x7")
check(lowered_ok and all((sandwich[b] == -6 * gamma[b]).all() for b in range(8)),
      "gamma^b gamma_a + gamma_a gamma^b = 2 delta^b_a I, and gamma^a gamma^b "
      "gamma_a = -6 gamma^b for every b")
```

To find the factor c in $\gamma^a\gamma^b\gamma_a = c\,\gamma^b$, the first nonzero entry of γ^b (`np.nonzero` gives the places of the nonzero entries) and the entry of the sandwich at the same place are divided. Output: c = [-6, -6, -6, -6, -6, -6, -6, -6]. The check requires the lowered relation and the exact equality with $-6\gamma^b$.

```
traces = np.array([[int(np.trace(gamma[a] @ gamma[b])) for b in range(8)]
                   for a in range(8)])
check((traces == 16 * eta_upper).all(), "tr(gamma^a gamma^b) = 16 eta^ab")
```

The table of the 64 traces must be $16\eta^{ab}$, the third contraction. The cell prints three PASS lines.

In [13], the contractions as pictures.

```
square_sign = [int(np.trace(g @ g)) // 16 for g in gamma]  # (gamma^a)^2 = +-I
term_sign = [eta_diagonal[a] * square_sign[a] for a in range(8)]  # eta_aa (gamma^a)^2
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
```

The sign of $(\gamma^a)^2 = \pm I$ is read from its trace (± 16, divided by 16), and `term_sign` is the sign of the term $\eta_{aa}(\gamma^a)^2$ of the first contraction.

```
left.bar(positions - 0.27, square_sign, width=0.27, color="tab:blue",
         label="$(\\gamma^a)^2 = \\pm I$: the sign")
left.bar(positions, eta_diagonal, width=0.27, color="tab:orange",
         label="$\\eta_{aa}$")
left.bar(positions + 0.27, term_sign, width=0.27, color="tab:green",
         label="term $\\eta_{aa}(\\gamma^a)^2 = +I$")
left.plot(positions, np.cumsum(term_sign) / 8, "k.--", lw=1,
          label="running sum / 8")
```

Three bars per coordinate: the sign of the square, η_{aa} and their product; and the **running sum** of the terms (`np.cumsum` adds the entries one after the other: 1, 2, ..., 8), divided by 8, as a dashed black line with dots (the style string made of a dot and two hyphens).

```
left.axhline(0, color="black", lw=0.8)
left.set_xticks(positions, labels)
left.set_ylim(-1.5, 1.9)
left.set_title("$\\gamma^a\\gamma_a$: eight terms, each $+I$, sum $8I$",
               fontsize=10)
left.legend(fontsize=7, loc="lower left", ncol=2)
```

Zero line, labels, range, title and a two-column legend.

```
right.plot(positions, factors, "o", color="tab:red", ms=9,
           label="$c$ in $\\gamma^a\\gamma^b\\gamma_a = c\\,\\gamma^b$")
right.axhline(2 - 8, color="black", ls=":", label="$2 - n$ for $n = 8$")
right.set_xticks(positions, [f"$b = x_{k}$" for k in range(8)], fontsize=8)
right.set_ylim(-8.0, 0.5)
right.set_ylabel("factor $c$")
right.set_title("$\\gamma^a\\gamma^b\\gamma_a = -6\\gamma^b$ for every $b$",
                fontsize=10)
right.legend(fontsize=8, loc="upper right")
fig.tight_layout()
save_figure(fig, "gamma_contractions",
            "Left: the contraction $\\gamma^a\\gamma_a$ term by term for $a = x_0$ "
            ...)
```

On the right the eight factors c as red dots and the dotted line $2 - n = -6$. What Figure 01e.5 shows: the blue and orange bars are both negative exactly for x_4 to x_7, so every green bar is $+1$ and the running sum climbs to 1 (that is, $8I$); all eight red dots sit on the line -6.

In [14], the author's metric in index notation.

```
a4 = sp.Symbol("a4", real=True)  # the value of a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of b4(x4) at one time
z = sp.Symbol("z", positive=True)  # z = 6 H x0, between 0 and pi/2
s = sp.sin(z) ** sp.Rational(1, 3)
g_typed = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * s] * 3 + [sp.Integer(-1)]
           + [-b4 * sp.exp(-2 * a4) * s] * 3)  # the author's order x0, x1, ..., x7
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))
```

The symbols, the typed metric and the record file, exactly as in Notebook 01a, In [17] (Section 1.25).

```
def from_record(text):
    """The record's Mathematica text as a sympy expression."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "sin(z)")
    text = text.replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "z": z, "E": sp.E})


g_record = [from_record(text) for text in curvature["metricDiagonal"]]
check(len(g_record) == 8 and
      all(sp.simplify(x - y) == 0 for x, y in zip(g_record, g_typed)),
      "the record's metric diagonal is the author's metric of section 4",
      record="Revision/gkd_lovelock/results/curvature.json, metricDiagonal")
```

The same translation function and the comparison with the typed metric (the "section 4" of the check's name is section 4 of the notebook, which states the metric); this check reproduces the record's key `metricDiagonal`.

```
g_lower = sp.diag(*g_record)  # g_mu nu
g_upper = sp.diag(*[1 / entry for entry in g_record])  # g^mu nu
lovelock_report = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json")
    .read_text(encoding="utf-8"))
lovelock_verdicts = {c["name"]: c["verdict"] for c in lovelock_report["checks"]}
```

$g_{\mu\nu}$ and $g^{\mu\nu}$ as diagonal sympy matrices (the star hands the eight entries to `sp.diag` one by one), and the verdicts of the record's report (this one writes them in capitals, PASS).

```
check((g_upper * g_lower).applyfunc(sp.simplify) == sp.eye(8)
      and lovelock_verdicts["metric_inverse"] == "PASS",
      "g^mu nu g_nu lambda = delta^mu_lambda",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "metric_inverse")
```

$g^{\mu\nu}g_{\nu\lambda} = \delta^{\mu}{}_{\lambda}$, reproducing the record's check `metric_inverse`.

```
step = sp.Matrix([1] * 8)  # dx^mu = 1 for every mu (units with epsilon = 1)
ds2 = (step.T * g_lower * step)[0, 0]  # g_mu nu dx^mu dx^nu
step_lower = g_lower * step  # dx_mu = g_mu nu dx^nu
ds2_contracted = (step.T * step_lower)[0, 0]  # dx^mu dx_mu
general_form = 3 * sp.exp(2 * a4) * s - 1 - 3 * b4 * sp.exp(-2 * a4) * s + sp.cot(z) ** 2
closed_form = 6 * s * sp.sinh(2 * a4) - 1 + sp.cot(z) ** 2  # at b4 = 1
say(f"ds^2 for dx^mu = 1: {ds2}")
```

The step $dx^{\mu} = 1$ (that is, $\epsilon = 1$) as a column of eight ones. `step.T * g_lower * step` is the row times the table times the column, a 1×1 matrix whose one entry `[0, 0]` is $g_{\mu\nu}dx^{\mu}dx^{\nu}$. The lowered step and the contraction $dx^{\mu}dx_{\mu}$; the general form and, at $b_4 = 1$, the closed form of Section 1.29. Output: `-3*b4*exp(-2*a4)*sin(z)**(1/3) + 3*exp(2*a4)*sin(z)**(1/3) + cot(z)**2 - 1`, the first line of the derivation (sympy chooses its own order of the terms).

```
check(sp.expand(ds2 - general_form) == 0
      and sp.expand((ds2.subs(b4, 1) - closed_form).rewrite(sp.exp)) == 0
      and sp.expand(ds2_contracted - ds2) == 0,
      "ds^2 = 3 e^(2 a4) s - 1 - 3 b4 e^(-2 a4) s + cot(z)^2 for the step dx^mu = 1, "
      "= 6 s sinh(2 a4) - 1 + cot(z)^2 at b4 = 1, and dx^mu dx_mu gives the same")
```

The difference from the general form multiplies out to 0; at $b_4 = 1$ (`.subs(b4, 1)`) the difference from the closed form, with sinh written as exponentials, multiplies out to 0 as well; and both ways of contracting agree. The cell prints three PASS lines.

In [15], where the step changes its kind.

```
ds2_numbers = sp.lambdify((a4, z), closed_form, "numpy")  # a formula -> a function
a4_values = np.linspace(-1.5, 1.5, 301)
z_values = np.linspace(0.15, np.pi / 2 - 0.005, 300)
A4_grid, Z_grid = np.meshgrid(a4_values, z_values)  # rows: z, columns: a4
values = ds2_numbers(A4_grid, Z_grid)
```

The closed form at $b_4 = 1$ as a numpy function; 301 values of a_4 and 300 values of z. `np.meshgrid` makes two tables of the same shape, one holding a_4 and one holding z at every grid point (a row for each z, a column for each a_4), so that `values` holds ds^2/ϵ^2 at all $301 \cdot 300$ points.

```
threshold = 0.5 * np.arcsinh((1 - 1 / np.tan(z_values) ** 2)
                             / (6 * np.sin(z_values) ** (1 / 3)))
check(bool((np.diff(values, axis=1) > 0).all()),
      "on every row of the grid, ds^2 grows as a4 grows")
away = np.abs(A4_grid - threshold[:, None]) > 1e-9  # leave out points on the curve
check(bool(((values > 0) == (A4_grid > threshold[:, None]))[away].all()),
      "ds^2 > 0 exactly to the right of the curve a4*(z)")
```

`threshold` is $a_4^*(z)$ for each z ($\cot z = 1/\tan z$). `np.diff(values, axis=1)` takes the differences of neighbours along each row; all positive means that ds^2 grows with a_4. `threshold[:, None]` turns the list into a column so that it is compared with every column of the grid; at every point not within 10^{-9} of the curve, $ds^2 > 0$ must hold exactly when $a_4 > a_4^*(z)$. Two PASS lines.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.5, 4.5))
image = left.imshow(np.tanh(values), origin="lower", aspect="auto", cmap="RdBu_r",
                    vmin=-1, vmax=1, extent=(a4_values[0], a4_values[-1],
                                             z_values[0], z_values[-1]))
left.plot(threshold, z_values, color="black", lw=1.5, label="$ds^2 = 0$")
left.set_xlim(a4_values[0], a4_values[-1])
left.grid(False)  # no grid lines across the coloured map
```

The left panel shows the whole grid as a coloured map. tanh squeezes every value into the range -1 to 1 while keeping its sign, so that red means space-like and blue time-like. `origin="lower"` puts the first row at the bottom, `extent` gives the map the coordinates of a_4 and z, and `aspect="auto"` lets it fill the panel. The black curve is $a_4^*(z)$.

```
left.text(0.75, 0.4, "space-like\n$ds^2 > 0$", ha="center", color="white")
left.text(-0.9, 1.2, "time-like\n$ds^2 < 0$", ha="center", color="white")
left.set_xlabel("$a_4$")
left.set_ylabel("$z = 6 H x_0$")
left.set_title("sign of $ds^2$ for the step $dx^\\mu = \\epsilon$", fontsize=10)
left.legend(loc="upper right", fontsize=8)
fig.colorbar(image, ax=left, label="$\\tanh(ds^2/\\epsilon^2)$")
```

Two white labels (`\n` in a string starts a new line), the axis labels, the title, the legend and a colour bar.

```
z_fixed = 0.9
s_fixed = np.sin(z_fixed) ** (1 / 3)
space_term = 3 * np.exp(2 * a4_values) * s_fixed  # x1, x2, x3
extra_term = -3 * np.exp(-2 * a4_values) * s_fixed  # x5, x6, x7 (deflating)
hidden_term = np.full_like(a4_values, 1 / np.tan(z_fixed) ** 2)  # x0
```

On the right, at $z = 0.9$, the four kinds of terms of the sum against a_4: the three space directions, the three deflating extra times, and the hidden direction (`np.full_like` makes an array of the same length filled with one value).

```
right.plot(a4_values, space_term, color="tab:red", label="space $x_1, x_2, x_3$")
right.plot(a4_values, np.full_like(a4_values, -1.0), color="tab:gray",
           label="time $x_4$")
right.plot(a4_values, extra_term, color="tab:blue",
           label="extra times $x_5, x_6, x_7$")
right.plot(a4_values, hidden_term, color="tab:green", label="hidden $x_0$")
right.plot(a4_values, space_term - 1 + extra_term + hidden_term, "k", lw=2.5,
           label="sum $ds^2/\\epsilon^2$")
```

The four terms (the time term is the constant -1) and their sum as a thick black line.

```
right.axhline(0, color="black", lw=0.8)
right.set_ylim(-12, 12)
right.set_xlabel("$a_4$")
right.set_ylabel("contribution to $ds^2/\\epsilon^2$")
right.set_title(f"the terms of $g_{{\\mu\\nu}} dx^\\mu dx^\\nu$ at $z = {z_fixed}$",
                fontsize=10)
right.legend(fontsize=8, loc="lower right")
fig.tight_layout()
save_figure(fig, "step_length",
            "The squared length $ds^2 = g_{\\mu\\nu} dx^\\mu dx^\\nu$ of the "
            ...)
```

The title is an f-string, so the braces of the LaTeX subscript are doubled. What Figure 01e.6 shows: on the left a blue (time-like) region to the left of the black curve and a red (space-like) region to its right; on the right the red space term climbing, the blue extra-time term rising from very negative values towards 0 as the extra times deflate, and the black sum crossing zero.

In [16], the last check.

```
figure_names = ["01e_1_eta_and_delta.png", "01e_2_squared_lengths.png",
                "01e_3_symmetric_antisymmetric.png", "01e_4_clifford_tables.png",
                "01e_5_gamma_contractions.png", "01e_6_step_length.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

The last line is ALL 25 CHECKS PASSED (notebook 01e). The 25 checks are: 2 in In [2], 3 in In [3], 1 in In [5], 3 in In [6], 4 in In [8], 3 in In [10], 3 in In [12], 3 in In [14], 2 in In [15] and 1 in In [16].

1.34 Eigenvalues and eigenvectors; the characteristic polynomial

Directions that are only stretched. A square matrix M usually turns a vector into a vector that points somewhere else. A few special directions are only stretched (or shrunk, or reversed):

$$Mv = \lambda v, \qquad v \neq 0.$$

The number λ (the Greek letter lambda) is an **eigenvalue** of M and v an **eigenvector** belonging to it. Any multiple cv with $c \neq 0$ is an eigenvector too ($M(cv) = cMv = c\lambda v = \lambda(cv)$), so only the direction of v matters. The list of all eigenvalues is the **spectrum** of M. Eigenvalues are everywhere in the course: the signature (4,4) of the author's spacetime is a count of positive and negative eigenvalues of its metric (Section 1.36); the gamma matrices are known by their eigenvalues (Section 1.37); and the energies of quantum states are eigenvalues (Chapters 10, 14 and 15).

Why a determinant decides. $Mv = \lambda v$ can be written

$$(M - \lambda I)v = 0$$

(move $\lambda v = \lambda I v$ to the left side and take out v). If $\det(M - \lambda I) \neq 0$, the matrix $M - \lambda I$ can be undone (Section 1.20), and undoing it on both sides gives $v = 0$: no eigenvector. Conversely, if $\det(M - \lambda I) = 0$, the equation has a solution $v \neq 0$ (a standard theorem of linear algebra, ASSUMED). So the eigenvalues are exactly the numbers with

$$p(\lambda) = \det(M - \lambda I) = 0.$$

p is the **characteristic polynomial** of M. By the Leibniz formula every term of $\det(M - \lambda I)$ is a product of n entries, of which at most n (the diagonal ones) contain λ; so p is a **polynomial** (a sum of powers of λ with number coefficients) of degree n. A **root** of a polynomial is a number at which it is 0. The **fundamental theorem of algebra** (ASSUMED) says that a polynomial of degree n has exactly n roots $\lambda_1, \ldots, \lambda_n$ when complex numbers are allowed and a repeated root is counted as often as it appears, its **multiplicity**. So an $n \times n$ matrix has n eigenvalues, some of which may be complex or repeated.

The eigenvalues add up to the trace and multiply to the determinant (PROVED). A polynomial of degree n whose highest term is $(-\lambda)^n$ and whose roots are $\lambda_1, \ldots, \lambda_n$ is

$$p(\lambda) = (\lambda_1 - \lambda)(\lambda_2 - \lambda)\cdots(\lambda_n - \lambda)$$

(the factored form; the highest term of $\det(M - \lambda I)$ is the product of the n diagonal terms $-\lambda$, which is $(-\lambda)^n$). Setting $\lambda = 0$:

$$p(0) = \det M = \lambda_1 \lambda_2 \cdots \lambda_n$$

(at $\lambda = 0$ the matrix $M - \lambda I$ is M): the determinant is the **product** of the eigenvalues. Multiplying out the factored form, the terms with λ^{n-1} take $-\lambda$ from all brackets but one and λ_i from that one, so they add up to $(-1)^{n-1}(\lambda_1 + \cdots + \lambda_n)\lambda^{n-1}$. In the Leibniz formula only the product of the diagonal entries, $(M_{11} - \lambda)\cdots(M_{nn} - \lambda)$, contains λ^{n-1} (every other term misses at least two diagonal entries, so it has at most $n - 2$ factors with λ), and in the same way its λ^{n-1} terms are $(-1)^{n-1}(M_{11} + \cdots + M_{nn})\lambda^{n-1}$. Comparing the two: the **sum** of the eigenvalues is the trace.

A 2×2 example by hand (PROVED). Take M with the rows $(2, 1)$ and $(1, 2)$. Line by line:

$$\det(M - \lambda I) = (2 - \lambda)(2 - \lambda) - 1 \cdot 1$$

(the determinant $ad - bc$ of the matrix with the rows $(2-\lambda, 1)$ and $(1, 2-\lambda)$),

$$= \lambda^2 - 4\lambda + 4 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3)$$

(multiply out; collect; two numbers whose product is 3 and whose sum is 4). So the eigenvalues are 1 and 3. For $\lambda = 3$ the equation $(M - 3I)v = 0$ has the rows $-v_1 + v_2 = 0$ and $v_1 - v_2 = 0$, which say the same, so $v = (1, 1)$; for $\lambda = 1$ it reads $v_1 + v_2 = 0$, so $v = (1, -1)$. Check: $3 + 1 = 4 = \operatorname{tr} M$ and $3 \cdot 1 = 3 = 2 \cdot 2 - 1 \cdot 1 = \det M$.

What the example means (PROVED). Take the vectors of length 1, $x = (\cos t, \sin t)$, which run around the unit circle as the angle t runs from 0 to 2π. Their images are $Mx = (2\cos t + \sin t, \cos t + 2\sin t)$, with the squared length

$$(2\cos t + \sin t)^2 + (\cos t + 2\sin t)^2 = 4\cos^2 t + 4\cos t \sin t + \sin^2 t + \cos^2 t + 4\cos t \sin t + 4\sin^2 t$$

(multiply out both squares with $(u + v)^2 = u^2 + 2uv + v^2$),

$$= 5\cos^2 t + 5\sin^2 t + 8\cos t \sin t = 5 + 8\cos t \sin t = 5 + 4\sin 2t$$

(collect; $\cos^2 t + \sin^2 t = 1$; the double-angle formula $2\sin t \cos t = \sin 2t$ of school trigonometry, ASSUMED). It is largest, $5 + 4 = 9$, where $\sin 2t = 1$, at $t = \pi/4$, the direction $(1, 1)/\sqrt{2}$; and smallest, $5 - 4 = 1$, at $t = -\pi/4$, the direction $(1, -1)/\sqrt{2}$. So the unit circle becomes an **ellipse** whose longest and shortest half-axes have the lengths $\sqrt{9} = 3$ and $\sqrt{1} = 1$, the two eigenvalues, and lie along the two eigenvectors (Figure 01g.1).

The chain matrix of size 3 (PROVED). The 3×3 **chain matrix** T_3 has 2 on the diagonal, -1 just above and just below it, and 0 elsewhere. With the six-term formula of Section 1.20 (the entries $a_{11} = a_{22} = a_{33} = 2 - \lambda$, $a_{12} = a_{21} = a_{23} = a_{32} = -1$, $a_{13} = a_{31} = 0$):

$$\det(T_3 - \lambda I) = (2-\lambda)^3 + 0 + 0 - (-1)(-1)(2-\lambda) - (2-\lambda)(-1)(-1) - 0$$

(the six terms in the order of Section 1.20; the second, third and sixth contain a_{13} or a_{31}, which are 0),

$$= (2-\lambda)^3 - 2(2-\lambda) = (2-\lambda)\left[(2-\lambda)^2 - 2\right]$$

(collect the two equal terms; take out the factor $2 - \lambda$). The roots are $\lambda = 2$ and $(2-\lambda)^2 = 2$, that is $\lambda = 2 \pm \sqrt{2}$. Multiplied out, $p(\lambda) = -\lambda^3 + 6\lambda^2 - 10\lambda + 4$ (from $(2-\lambda)^3 = 8 - 12\lambda + 6\lambda^2 - \lambda^3$ and $-2(2-\lambda) = -4 + 2\lambda$). The sum of the roots is $6 = \operatorname{tr} T_3$, and their product is $(2-\sqrt{2}) \cdot 2 \cdot (2+\sqrt{2}) = 2(4-2) = 4 = p(0) = \det T_3$ (Figure 01g.2).

1.35 The chain matrix of any size; rotations and boosts; conjugate pairs

The chain matrix T_n (PROVED). Row j of $T_n v$ is $-v_{j-1} + 2v_j - v_{j+1}$, where we put $v_0 = v_{n+1} = 0$ (the first and the last row have only one neighbour; a missing neighbour counts as 0). Try $v_j = \sin(j\theta)$ for an angle θ:

$$\sin((j-1)\theta) + \sin((j+1)\theta) = 2\sin(j\theta)\cos\theta$$

(the addition theorems $\sin(x \pm y) = \sin x \cos y \pm \cos x \sin y$ with $x = j\theta$ and $y = \theta$, added; the terms $\cos x \sin y$ cancel), so

$$-v_{j-1} + 2v_j - v_{j+1} = 2\sin(j\theta) - 2\sin(j\theta)\cos\theta = (2 - 2\cos\theta)\sin(j\theta)$$

(insert; take out the common factor). This is λv_j with $\lambda = 2 - 2\cos\theta$, provided the two end conditions hold. The start $v_0 = \sin 0 = 0$ holds for every θ. The end $v_{n+1} = \sin((n+1)\theta) = 0$ holds when $(n+1)\theta$ is a whole multiple of π, $\theta = k\pi/(n+1)$. For $k = 1, \ldots, n$ this gives n different eigenvalues

$$\lambda_k = 2 - 2\cos\frac{k\pi}{n+1}, \qquad k = 1, \ldots, n$$

(different because the cosine decreases between 0 and π), with the eigenvectors $v_j = \sin(jk\pi/(n+1))$: sine waves sampled at the points $j = 1, \ldots, n$, with k half waves along the chain. For $n = 3$ they are $2 - 2\cos(\pi/4) = 2 - \sqrt{2}$, $2 - 2\cos(\pi/2) = 2$ and $2 - 2\cos(3\pi/4) = 2 + \sqrt{2}$, as found above. This matrix comes back whenever a second derivative is computed on a grid of points (Chapter 2): then the sine waves are the shapes of a vibrating string or of a quantum particle in a box. Notebook 01g, In [5], checks the formula for $n = 10$: numpy's eigenvalues differ from it by at most 8.9×10^{-16} (COMPUTED; Figure 01g.3).

Rotations have complex eigenvalues (PROVED). The rotation matrix $R(\alpha)$ of Section 1.12 turns every vector, so for $0 < \alpha < \pi$ no real direction is kept. Its characteristic polynomial is

$$(\cos\alpha - \lambda)^2 + \sin^2\alpha = \lambda^2 - 2\cos\alpha\,\lambda + 1$$

(the determinant $ad - bc$ of the matrix with the rows $(\cos\alpha - \lambda, -\sin\alpha)$ and $(\sin\alpha, \cos\alpha - \lambda)$; multiply out and use $\cos^2\alpha + \sin^2\alpha = 1$). By the formula for the roots of a quadratic $\lambda^2 + p\lambda + q$, $\lambda = -p/2 \pm \sqrt{p^2/4 - q}$ (school algebra, ASSUMED),

$$\lambda = \cos\alpha \pm \sqrt{\cos^2\alpha - 1} = \cos\alpha \pm \sqrt{-\sin^2\alpha} = \cos\alpha \pm i\sin\alpha = e^{\pm i\alpha}$$

(insert $p = -2\cos\alpha$, $q = 1$; $\cos^2 - 1 = -\sin^2$; a square root of $-x^2$ is ix, because $(ix)^2 = -x^2$; Euler's formula): two complex numbers of modulus 1. The eigenvectors are $(1, \mp i)$: $R(\alpha)(1, -i) = (\cos\alpha + i\sin\alpha, \sin\alpha - i\cos\alpha) = e^{i\alpha}(1, -i)$, because $e^{i\alpha} \cdot (-i) = -i\cos\alpha + \sin\alpha$ (row times column; multiply out). For $\alpha = \pi/2$, R is the matrix J of Section 1.12, whose eigenvalues are $\pm i$: the real matrix that plays the role of i has the eigenvalues $\pm i$.

Boosts have real eigenvalues. The boost $\Lambda(\varphi)$ of Section 1.13 has the characteristic polynomial $(\cosh\varphi - \lambda)^2 - \sinh^2\varphi = \lambda^2 - 2\cosh\varphi\,\lambda + 1$ (now with $\cosh^2 - \sinh^2 = 1$), whose roots are $\cosh\varphi \pm \sqrt{\cosh^2\varphi - 1} = \cosh\varphi \pm \sinh\varphi = e^{\pm\varphi}$ (for $\varphi \geq 0$, where $\sinh\varphi \geq 0$), with the eigenvectors $(1, \pm 1)$, the two light-like directions found in Section 1.13 (PROVED). Their product is 1 (Figure 01g.4).

Complex eigenvalues of a real matrix come in pairs (PROVED). If M is real and $Mv = \lambda v$, conjugating every entry gives $M^*v^* = \lambda^* v^*$, and $M^* = M$; so $Mv^* = \lambda^* v^*$: λ^* is an eigenvalue too, with the eigenvector v^*. Notebook 01g, In [6], checks this on 200 random real 5×5 matrices.

1.36 Symmetric and Hermitian matrices; the signature and Sylvester's law

Words. The **conjugate transpose** of a matrix is $M^\dagger = (M^*)^T$: conjugate every entry, then exchange rows and columns; for a column v, $v^\dagger$ is the row of the conjugated components, and $v^\dagger v = \sum_i |v_i|^2$, which is positive for $v \neq 0$. Like the transpose it reverses products, $(AB)^\dagger = B^\dagger A^\dagger$ (the conjugate of a product of matrices is the product of the conjugates, entry by entry, by rule 1 of Section 1.10; then the transpose rule of Section 1.18). A matrix is **Hermitian** if $H^\dagger = H$; a real Hermitian matrix is a symmetric matrix. The **length** of a real vector is $|v| = \sqrt{v_1^2 + \cdots + v_n^2}$; two real vectors are **perpendicular** (or orthogonal) if $u^T w = \sum_i u_i w_i = 0$.

Theorem 1: a symmetric matrix has only real eigenvalues (PROVED). Let S be real and symmetric, and $Sv = \lambda v$ with a column $v \neq 0$ that may be complex.

- Conjugate and transpose both sides: $v^\dagger S^\dagger = \lambda^* v^\dagger$ (the product rule of †; a number only gets conjugated), and $S^\dagger = S$ because S is real ($S^* = S$) and symmetric ($S^T = S$); so $v^\dagger S = \lambda^* v^\dagger$.
- Multiply $Sv = \lambda v$ from the left by $v^\dagger$: $v^\dagger S v = \lambda\, v^\dagger v$.
- Multiply $v^\dagger S = \lambda^* v^\dagger$ from the right by v: $v^\dagger S v = \lambda^*\, v^\dagger v$.
- Subtract the two lines: $0 = (\lambda - \lambda^*)\, v^\dagger v$.
- $v^\dagger v = |v_1|^2 + \cdots + |v_n|^2 > 0$, so $\lambda - \lambda^* = 0$: λ equals its conjugate, so it is real.

The same lines, with $S^\dagger = S$ as the only property used, prove it for a complex Hermitian matrix.

Theorem 2: eigenvectors with different eigenvalues are perpendicular (PROVED). Let $Su = \lambda u$ and $Sw = \mu w$ with $\lambda \neq \mu$ (real vectors, real eigenvalues). Then $u^T S w = \mu\, u^T w$ (insert Sw), and also $u^T S w = (S^T u)^T w = (Su)^T w = \lambda\, u^T w$ (the transpose rule; $S^T = S$; insert Su). Subtracting: $(\lambda - \mu)\, u^T w = 0$, so $u^T w = 0$.

The spectral theorem (ASSUMED). A standard theorem of linear algebra, used here without proof, says that also when eigenvalues repeat, a real symmetric $n \times n$ matrix has n perpendicular eigenvectors of length 1, and that, with these eigenvectors as the columns of a matrix O,

$$S = O \Lambda O^T, \qquad O^T O = I,$$

with the eigenvalues on the diagonal of the diagonal matrix Λ (O is orthogonal, Section 1.28). One can see that both sides do the same to every eigenvector: $O^T u_i = e_i$, the i-th unit vector, because the columns of O are perpendicular and of length 1; so $O \Lambda O^T u_i = O \Lambda e_i = \lambda_i O e_i = \lambda_i u_i = S u_i$. Notebook 01g, In [8], checks this for 300 random symmetric 8×8 matrices $S = A + A^T$: every eigenvalue is real (the largest imaginary part found by the general eigenvalue program is 0.0) and $O \Lambda O^T = S$ to 10^{-12}; the 300 non-symmetric matrices A have 67.7 per cent complex eigenvalues (COMPUTED; Figure 01g.5).

The quadratic form and the signature. A symmetric matrix S defines the **quadratic form** $Q(v) = v^T S v = S_{ab} v^a v^b$; for the frame metric it is the squared length of Section 1.27. In the eigenvector coordinates $v = Ow$:

$$Q = (Ow)^T S(Ow) = w^T (O^T S O) w = w^T \Lambda w = \lambda_1 w_1^2 + \cdots + \lambda_n w_n^2$$

(insert; the transpose rule $(Ow)^T = w^T O^T$; $O^T S O = O^T O \Lambda O^T O = \Lambda$; a diagonal matrix in a quadratic form gives a sum of squares). So the **signature** (p, q) of a symmetric matrix, the numbers of its positive and its negative eigenvalues, says in how many perpendicular directions Q is positive and in how many it is negative. An eigenvalue 0 would be a direction in which Q vanishes; the metrics of the course have none (their determinants, the products of the eigenvalues, are not 0).

Sylvester's law of inertia (PROVED from one ASSUMED fact). A change of coordinates $v = Pw$ with an invertible matrix P gives $Q = w^T (P^T S P) w$: the matrix of the form becomes $P^T S P$. Its eigenvalues are different, but its signature is the same. We use one fact of linear algebra without proof (ASSUMED): more than n vectors with n components are **linearly dependent**, that is, some combination $c_1 u_1 + c_2 u_2 + \cdots$ with numbers c_i not all 0 gives the zero vector. *Proof.* Let S have p positive eigenvalues with perpendicular unit eigenvectors $u_1, \ldots, u_p$, and let $S' = P^T S P$ have p' positive eigenvalues; suppose $p' < p$.

- On every combination $x = \sum_i c_i u_i$ with not all $c_i = 0$: $Q(x) = \sum_i \sum_j c_i c_j \, u_i^T S u_j = \sum_i \lambda_i c_i^2 > 0$ (multiply out; $S u_j = \lambda_j u_j$ and $u_i^T u_j = \delta_{ij}$ by Theorem 2 and the length 1; every $\lambda_i > 0$).
- Let $u'_1, \ldots, u'_{n-p'}$ be perpendicular unit eigenvectors of S' with eigenvalues ≤ 0. On every combination $y = \sum_k d_k u'_k$: $y^T S' y = \sum_k \lambda'_k d_k^2 \leq 0$ (the same computation), and $y^T S' y = (Py)^T S (Py) = Q(Py)$ (the definition of S'; the transpose rule).
- The $p + (n - p') > n$ vectors $u_1, \ldots, u_p, Pu'_1, \ldots, Pu'_{n-p'}$ are linearly dependent: $\sum_i c_i u_i + \sum_k d_k P u'_k = 0$ with numbers not all 0. Put $x = \sum_i c_i u_i$ and $y = -\sum_k d_k u'_k$, so $x = Py$. If all c_i were 0, then $P \sum_k d_k u'_k = 0$, hence $\sum_k d_k u'_k = 0$ (P can be undone), hence all $d_k = 0$ (multiply by u'^T_l and use the perpendicularity); so some $c_i \neq 0$.
- Then $Q(x) > 0$ by the first line, but $Q(x) = Q(Py) \leq 0$ by the second: impossible.

So $p' \geq p$. The same argument with the roles exchanged ($S = (P^{-1})^T S' P^{-1}$) gives $p \geq p'$, and the argument for $-S$ counts the negative eigenvalues. So **the signature is a property of the form, not of the coordinates used to describe it.** Notebook 01g, In [10], checks the law on 2000 random matrices P: every $P^T \eta P$ has the signature (4,4), with eigenvalues of sizes from 1.4×10^{-8} to 37.0, all far enough from 0 that rounding cannot change their signs (COMPUTED; Figure 01g.6, left).

The signature of the author's spacetime (PROVED). The eigenvalues of a diagonal matrix are its diagonal entries (each unit vector e_i is an eigenvector: $D e_i = d_i e_i$). So η has the eigenvalues $+1$ (four times) and -1 (four times): the signature (4,4) (record `Revision/algebra/reports/python-algebra.json`, check `coordinate_map`, which states η). The author's metric g has the eigenvalues $\cot^2 z > 0$, $e^{2a_4} s > 0$ (three times), -1 and $-b_4 e^{-2a_4} s < 0$ (three times), for every a_4, every $b_4 > 0$ and every $0 < z < \pi/2$: again four positive and four negative. As a_4 grows (with b_4 constant) the three extra-time eigenvalues $-b_4 e^{-2a_4} s$ shrink towards 0, the exponential deflation of the extra

times, but they never change sign; and their product with the others is $\det g = b_4^3 \cos^2 z > 0$ (Section 1.21). Notebook 01g, In [11], checks the signs at $3 \times 61 \times 50 = 9150$ grid points (a_4, z, b_4) and the product to 10^{-12} (COMPUTED; Figure 01g.6, right, drawn at $b_4 = 1$).

1.37 The eigenvalues of the gamma matrices and of B; power iteration

The gamma matrices (PROVED). The gamma matrices obey $(\gamma^a)^2 = \eta^{aa} I$ (Section 1.28). If $\gamma^a v = \lambda v$ with $v \neq 0$, then

$$(\gamma^a)^2 v = \gamma^a(\lambda v) = \lambda\, \gamma^a v = \lambda^2 v$$

(apply γ^a twice; a number can be moved in front of a matrix), and also $(\gamma^a)^2 v = \eta^{aa} v$. So $\lambda^2 = \eta^{aa}$: $\lambda = \pm 1$ for the space-like directions x_0, x_1, x_2, x_3 and $\lambda = \pm i$ for the time-like directions $x_4, \ldots, x_7$. How many of each? The trace of a gamma matrix is 0: for any $b \neq a$,

$$\operatorname{tr} \gamma^a = \eta^{bb} \operatorname{tr}(\gamma^b \gamma^b \gamma^a) = \eta^{bb} \operatorname{tr}(\gamma^b \gamma^a \gamma^b) = -\eta^{bb} \operatorname{tr}(\gamma^b \gamma^b \gamma^a) = -\operatorname{tr} \gamma^a$$

(insert $\eta^{bb}(\gamma^b)^2 = \eta^{bb}\eta^{bb} I = I$; the trace does not change when the first factor is moved to the end, $\operatorname{tr}(XY) = \operatorname{tr}(YX)$ with $X = \gamma^b$; the anticommutation $\gamma^a \gamma^b = -\gamma^b \gamma^a$ for $a \neq b$; the first step backwards), and a number equal to its own negative is 0. The trace is the sum of the 16 eigenvalues (Section 1.34), so $+1$ and -1 (or $+i$ and $-i$) occur 8 times each, and the characteristic polynomial is $(\lambda - 1)^8(\lambda + 1)^8 = (\lambda^2 - 1)^8$ for a space-like and $(\lambda - i)^8(\lambda + i)^8 = (\lambda^2 + 1)^8$ for a time-like direction, in one formula $(\lambda^2 - \eta_{aa})^8$. (sympy's `charpoly` computes $\det(\lambda I - M)$, which for a 16×16 matrix equals $\det(M - \lambda I)$, because changing the sign of all 16 rows multiplies the determinant by $(-1)^{16} = 1$, rule 5 of Section 1.20.) Notebook 01g, In [13], checks this exactly for the eight matrices of the record (Figure 01g.7, left).

The gamma matrix of a vector (PROVED). For a vector v the matrix $\gamma(v) = v_a \gamma^a$ (with $v_a = \eta_{ab} v^b$) squares to

$$\gamma(v)^2 = v_a v_b\, \gamma^a \gamma^b = \tfrac{1}{2} v_a v_b (\gamma^a \gamma^b + \gamma^b \gamma^a) = v_a v_b\, \eta^{ab} I = Q(v)\, I$$

(multiply out the two sums; the numbers $v_a v_b$ are symmetric in a and b, so only the symmetric part of $\gamma^a \gamma^b$ counts, by the argument of Section 1.28 applied entry by entry; the Clifford relation; $v_a v_b \eta^{ab} = v_a v^a = Q(v)$). So, as above, its eigenvalues are $\pm\sqrt{Q(v)}$: real for a space-like, imaginary for a time-like v (eight of each sign, because $\operatorname{tr} \gamma(v) = v_a \operatorname{tr} \gamma^a = 0$). For a light-like $v \neq 0$, $\gamma(v)$ is not the zero matrix, because $\operatorname{tr}(\gamma(v)\gamma^b) = v_a \operatorname{tr}(\gamma^a \gamma^b) = 16\, v_a \eta^{ab} = 16\, v^b$ (the third contraction of Section 1.28; raising the index) is not 0 for a component $v^b \neq 0$; but $\gamma(v)^2 = 0$: a matrix with a power equal to 0 is called **nilpotent**, and all its eigenvalues are 0 ($\lambda^2 v = \gamma(v)^2 v = 0$ gives $\lambda = 0$); its characteristic polynomial is λ^{16}. Notebook 01g, In [14], checks $\gamma(v)^2 = Q(v) I$ exactly for four examples and 100 random vectors (Figure 01g.7, right).

The Hermitian matrix B and its signature (8,8). The Revision record stores, in `Revision /algebra/gammas.json`, the real matrix $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ and the matrix $B = -i\, C \gamma^{(x_4)}$. B is complex: its real part is 0 and its imaginary part is a real 16×16 matrix of entries -1, 0, 1. In the theory of the course B gives the charge density $J^{(x_4)} = \Psi^\dagger B \Psi$ of the fields (Chapters 7 and 10). Its properties:

- B is Hermitian. For a matrix iA with real A, $(iA)^\dagger = i^* A^T = -iA^T$ (conjugate the number i; a real matrix only gets transposed), so $(iA)^\dagger = iA$ means $A^T = -A$: B is Hermitian exactly when its imaginary part is antisymmetric. By Theorem 1 its eigenvalues are real.
- $B^2 = I$, so, as for the gamma matrices, every eigenvalue has $\lambda^2 = 1$: $\lambda = \pm 1$.
- $\operatorname{tr} B = 0$, so $+1$ and -1 occur 8 times each: the signature (8,8), and the characteristic polynomial is $(\lambda - 1)^8(\lambda + 1)^8$.

The record checks these properties twice: in its algebra report `Revision/algebra/reports/python-algebra.json`, in the check named `B_hermitian_involution_signature`, and in its theory report `Revision/theory/reports/python-field-theory.json`, in the check named `B_properties`. Notebook 01g, In [16], reproduces them exactly from the record's matrices (PROVED by exact computation; Figure 01g.8). Because half of the eigenvalues of B are negative, the charge $\Psi^\dagger B \Psi$ can be positive or negative: the record calls it an **indefinite** form, and its quantisation entry reads "B Hermitian, B^2 = 1, signature (8,8): indefinite (Krein) state space". A **Krein space** is a space of states whose product of two states, built with B as in $\Psi^\dagger B \Psi$, takes the place of the dot product, so that a state can have a negative "squared length" (the dot product of an ordinary vector with itself is never negative); Chapter 10 explains what this means for the quantised field.

Power iteration (PROVED). A computer does not find eigenvalues as the roots of $\det(M - \lambda I)$ (that polynomial is very sensitive to rounding for large matrices); it transforms the matrix step by step. The simplest such method is **power iteration**. Write a starting vector as a sum of eigenvectors, $x_0 = c_1 u_1 + c_2 u_2 + \cdots$, with $|\lambda_1| > |\lambda_2| \geq \cdots$ and $c_1 \neq 0$. Multiplying by M k times gives

$$M^k x_0 = c_1 \lambda_1^k u_1 + c_2 \lambda_2^k u_2 + \cdots = \lambda_1^k \Big(c_1 u_1 + c_2 \Big(\frac{\lambda_2}{\lambda_1}\Big)^k u_2 + \cdots \Big)$$

(each eigenvector is only multiplied by its eigenvalue, once per step; then λ_1^k is taken out). The ratios $(\lambda_j/\lambda_1)^k$ shrink to 0, so the direction of $M^k x_0$ approaches u_1, and the leftover error shrinks by the factor $|\lambda_2/\lambda_1|$ in each step. Rescaling the vector to length 1 after each step keeps its numbers from growing without changing its direction. For the matrix M of Section 1.34 the factor is $1/3$; for T_3 it is

$$\frac{2}{2+\sqrt{2}} = \frac{2(2-\sqrt{2})}{(2+\sqrt{2})(2-\sqrt{2})} = \frac{2(2-\sqrt{2})}{2} = 2 - \sqrt{2} \approx 0.586$$

(multiply numerator and denominator by $2 - \sqrt{2}$; $(2+\sqrt{2})(2-\sqrt{2}) = 4 - 2$). Notebook 01g, In [18], measures 0.333333 and 0.585786 to 0.585789 (COMPUTED; Figure 01g.9).

1.38 Example: eigenvalues, eigenvectors and the signature (4,4)

Notebook 01g puts Sections 1.34 to 1.37 to work. It finds the eigenvalues of a 2×2 matrix by hand and with sympy and numpy and draws how the matrix turns the unit circle into an ellipse; computes the characteristic polynomial of the chain matrix and checks its exact eigenvalues and sine-wave eigenvectors; checks the complex eigenvalues of rotations and the real eigenvalues of

boosts; checks on random matrices that symmetric matrices have real eigenvalues and perpendicular eigenvectors; reads η and the author's metric from the Revision record, counts their positive and negative eigenvalues and checks Sylvester's law on 2000 random coordinate changes; derives and checks the eigenvalues of the eight gamma matrices and of $\gamma(v)$; finds the signature (8,8) of the record's matrix B; and runs power iteration. It ends with the line ALL 30 CHECKS PASSED (notebook 01g).

1.39 How to run Notebook 01g

Step 1. What this notebook does and what it needs.

Notebook 01g (Eigenvalues, eigenvectors and the signature (4,4)) is the file `Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb` of the repository Dirac_claude. It finds eigenvalues and eigenvectors by hand and with sympy and numpy, derives the characteristic polynomial and the rules that the eigenvalues add up to the trace and multiply to the determinant, solves the chain matrix of a grid exactly (sine-shaped eigenvectors), shows that real rotations have complex eigenvalues and boosts real ones, proves that symmetric and Hermitian matrices have real eigenvalues, counts the positive and negative eigenvalues of the frame metric and of the author's metric (the signature (4,4)) and checks that a change of coordinates never changes these counts (Sylvester's law of inertia), derives the eigenvalues of the eight real gamma matrices of the Revision record and of the Hermitian matrix B with its signature (8,8), and shows how a computer finds an eigenvalue by power iteration. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01g_eigenvalues_signature.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01g_eigenvalues_signature.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01g_eigenvalues_signature.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01g.captions.json`
- `Revision/textbook/figures/01g_1_circle_to_ellipse.png`
- `Revision/textbook/figures/01g_2_characteristic_polynomial.png`
- `Revision/textbook/figures/01g_3_chain_matrix.png`
- `Revision/textbook/figures/01g_4_rotation_and_boost.png`
- `Revision/textbook/figures/01g_5_random_spectra.png`
- `Revision/textbook/figures/01g_6_signature.png`
- `Revision/textbook/figures/01g_7_gamma_eigenvalues.png`
- `Revision/textbook/figures/01g_8_hermitian_b.png`
- `Revision/textbook/figures/01g_9_power_iteration.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which

is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 9 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 01g)
```

and the notebook must show 9 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01g_eigenvalues_signature.ipynb
python -m nbconvert --execute --inplace 01g_eigenvalues_signature.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock, Revision/algebra or Revision/theory: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

1.40 Notebook 01g: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A square matrix $M$ usually turns a vector into a vector that points somewhere
else. A few special directions are only stretched: $M v = \lambda v$. The
number $\lambda$ is an **eigenvalue** and $v$ an **eigenvector**. Eigenvalues
are everywhere in the course: the signature (4,4) of the author's spacetime is a
count of positive and negative eigenvalues of its metric, the gamma matrices
are known by their eigenvalues, and energies of quantum states are
eigenvalues. This notebook

- finds the eigenvalues and eigenvectors of a $2 \times 2$ matrix by hand and
  draws how the matrix turns the unit circle into an ellipse;
- derives the *characteristic polynomial* $\det(M - \lambda I)$ and the rules
  "the eigenvalues add up to the trace and multiply to the determinant";
- solves exactly the *chain matrix* (2 on the diagonal, $-1$ next to it), whose
  eigenvectors are sampled sine waves;
- shows that real rotation matrices have complex eigenvalues $e^{\pm i\alpha}$,
  while boosts have real ones $e^{\pm\varphi}$;
- proves that symmetric (and Hermitian) matrices have real eigenvalues and
  perpendicular eigenvectors, and checks it on random matrices;
- counts the positive and negative eigenvalues of the frame metric $\eta$ and of
  the author's metric $g$ (the signature (4,4)) and checks that no change of
  coordinates changes these counts (Sylvester's law of inertia);
- derives the eigenvalues of the eight real gamma matrices of the Revision
  record ($\pm 1$ or $\pm i$, eight times each) and of the matrix
```

```
  $\gamma(v) = v_a \gamma^a$ of a vector, which are $\pm\sqrt{Q(v)}$;
- finds the eigenvalues of the Hermitian matrix $B$ of the record, $+1$ and
  $-1$ eight times each: the signature (8,8);
- shows how a computer finds an eigenvalue by *power iteration*.

Every check prints a line that starts with PASS; where a check repeats a result
of the Revision record, a second line names the record file and its check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01g (Eigenvalues, eigenvectors and the signature (4,4)) is the file
‘Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb‘ of the repository
Dirac_claude. It finds eigenvalues and eigenvectors by hand and with sympy and numpy,
derives the characteristic polynomial and the rules that the eigenvalues add up to the
trace and multiply to the determinant, solves the chain matrix of a grid exactly
(sine-shaped eigenvectors), shows that real rotations have complex eigenvalues and boosts
real ones, proves that symmetric and Hermitian matrices have real eigenvalues, counts the
positive and negative eigenvalues of the frame metric and of the author's metric (the
signature (4,4)) and checks that a change of coordinates never changes these counts
(Sylvester's law of inertia), derives the eigenvalues of the eight real gamma matrices of
the Revision record and of the Hermitian matrix B with its signature (8,8), and shows how
a computer finds an eigenvalue by power iteration. It needs a computer with Windows 11,
macOS or Linux, an internet connection for the installation, the program Git, and Python
3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at
exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
```

two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01g_eigenvalues_signature.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01g_eigenvalues_signature.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01g_eigenvalues_signature.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01g.captions.json‘
- ‘Revision/textbook/figures/01g_1_circle_to_ellipse.png‘
- ‘Revision/textbook/figures/01g_2_characteristic_polynomial.png‘
- ‘Revision/textbook/figures/01g_3_chain_matrix.png‘
- ‘Revision/textbook/figures/01g_4_rotation_and_boost.png‘
- ‘Revision/textbook/figures/01g_5_random_spectra.png‘
- ‘Revision/textbook/figures/01g_6_signature.png‘
- ‘Revision/textbook/figures/01g_7_gamma_eigenvalues.png‘
- ‘Revision/textbook/figures/01g_8_hermitian_b.png‘
- ‘Revision/textbook/figures/01g_9_power_iteration.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the

notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS all 9 figure files of this notebook exist
    ALL 30 CHECKS PASSED (notebook 01g)
```

and the notebook must show 9 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 01g_eigenvalues_signature.ipynb
      python -m nbconvert --execute --inplace 01g_eigenvalues_signature.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock, Revision/algebra or Revision/theory: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.

The first code cell below (the set-up cell) repeats these instructions as comments; then

it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01g (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 01g (Eigenvalues, eigenvectors and the signature (4,4)) is the file
# Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb of the repository
# Dirac_claude. It finds eigenvalues and eigenvectors by hand and with sympy and numpy,
# derives the characteristic polynomial and the rules that the eigenvalues add up to the
# trace and multiply to the determinant, solves the chain matrix of a grid exactly
# (sine-shaped eigenvectors), shows that real rotations have complex eigenvalues and
# boosts real ones, proves that symmetric and Hermitian matrices have real eigenvalues,
# counts the positive and negative eigenvalues of the frame metric and of the author's
# metric (the signature (4,4)) and checks that a change of coordinates never changes
# these counts (Sylvester's law of inertia), derives the eigenvalues of the eight real
# gamma matrices of the Revision record and of the Hermitian matrix B with its signature
# (8,8), and shows how a computer finds an eigenvalue by power iteration. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
```

```
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
```

```
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01g_eigenvalues_signature.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
```

```
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01g_eigenvalues_signature.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01g_eigenvalues_signature.ipynb (in the same folder, without running the cells again)
# to read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01g.captions.json
# - Revision/textbook/figures/01g_1_circle_to_ellipse.png
# - Revision/textbook/figures/01g_2_characteristic_polynomial.png
# - Revision/textbook/figures/01g_3_chain_matrix.png
# - Revision/textbook/figures/01g_4_rotation_and_boost.png
# - Revision/textbook/figures/01g_5_random_spectra.png
# - Revision/textbook/figures/01g_6_signature.png
# - Revision/textbook/figures/01g_7_gamma_eigenvalues.png
# - Revision/textbook/figures/01g_8_hermitian_b.png
# - Revision/textbook/figures/01g_9_power_iteration.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 9 figure files of this notebook exist
#     ALL 30 CHECKS PASSED (notebook 01g)
#
# and the notebook must show 9 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
```

```
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01g_eigenvalues_signature.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01g_eigenvalues_signature.ipynb
#     python -m nbconvert --execute --inplace 01g_eigenvalues_signature.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below Revision/gkd_lovelock, Revision/algebra or
#   Revision/theory: the notebook reads Revision records of the repository; your copy of
#   the repository is incomplete. Download it again with git clone and open the notebook
#   inside the new copy.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "01g"  # this notebook: chapter 01, example g


def find_repository_root():
    """Return the repository folder (Dirac_claude).
```

```
    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
```

```
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01g complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Eigenvalue, eigenvector**: a number $\lambda$ and a vector $v \neq 0$ with
  $M v = \lambda v$ for a square matrix $M$. Any multiple $c v$ ($c \neq 0$) is
  an eigenvector too, so only the *direction* of $v$ matters.
- **Spectrum**: the list of all eigenvalues of a matrix.
- **Characteristic polynomial**: $p(\lambda) = \det(M - \lambda I)$, a
  polynomial in $\lambda$ whose roots are the eigenvalues.
- **Root** (or zero) of a polynomial: a number where it is 0. A root that
  appears $k$ times in the factorised polynomial has **multiplicity** $k$.
- **Trace** $\mathrm{tr}\,M$: the sum of the diagonal entries.
- **Determinant** $\det M$: the number of notebook 01a; $\det M \neq 0$ exactly
  when $M$ can be undone (inverted).
- **Length** (norm) of a vector, $|v| = \sqrt{v_1^2 + \dots + v_n^2}$; a **unit
  vector** has length 1.
- **Perpendicular** (orthogonal) vectors: $u^T w = \sum_i u_i w_i = 0$.
- **Symmetric matrix**: $S^T = S$. **Orthogonal matrix**: $O^T O = I$; its
  columns are perpendicular unit vectors.
- **Complex conjugate** $z^*$ and **conjugate transpose**
  $M^\dagger = (M^*)^T$ (conjugate every entry, then exchange rows and
  columns); for a column $v$, $v^\dagger v = \sum_i |v_i|^2$.
- **Hermitian matrix**: $H^\dagger = H$; a real Hermitian matrix is a
  symmetric matrix.
- **Quadratic form** of a symmetric matrix $S$: the number
  $Q(v) = v^T S v = S_{ab} v^a v^b$.
- **Signature** $(p, q)$ of a symmetric matrix: $p$ eigenvalues are positive
  and $q$ negative (the rest are 0).
- **Sylvester's law of inertia**: the matrices $S$ and $P^T S P$ ($P$
  invertible) have the same signature.
- **Light-like** vector: $v \neq 0$ with $Q(v) = 0$ (notebook 01e).
- **Nilpotent matrix**: a matrix $N \neq 0$ with $N^k = 0$ for some power $k$;
  all its eigenvalues are 0.
- **Power iteration**: multiply a vector by $M$ again and again and rescale it
  to length 1; it turns towards the eigenvector of the eigenvalue of largest
  size.
```

Text cell:

```
## 4. The physical and mathematical situation

**Why a determinant decides.** $M v = \lambda v$ can be written
$(M - \lambda I) v = 0$ (move $\lambda v = \lambda I v$ to the left side). If
$\det(M - \lambda I) \neq 0$, the matrix $M - \lambda I$ can be undone, and
undoing it on both sides gives $v = 0$: no eigenvector. So eigenvalues are
exactly the numbers with

$$p(\lambda) = \det(M - \lambda I) = 0.$$

By the Leibniz formula every term of $\det(M - \lambda I)$ is a product of $n$
entries, and at most $n$ of them contain $\lambda$; so $p$ is a polynomial of
```

```
degree $n$. A theorem of algebra (the *fundamental theorem of algebra*) says
that such a polynomial has exactly $n$ roots $\lambda_1, \dots, \lambda_n$ when
complex numbers are allowed and a repeated root is counted as often as its
multiplicity says; so an $n \times n$ matrix has $n$ eigenvalues, some of which
may be complex or repeated.

**Trace and determinant.** A polynomial of degree $n$ whose highest term is
$(-\lambda)^n$ and whose roots are $\lambda_1, \dots, \lambda_n$ is

$$p(\lambda) = (\lambda_1 - \lambda)(\lambda_2 - \lambda)\cdots(\lambda_n -
\lambda).$$

Setting $\lambda = 0$ gives $p(0) = \det M = \lambda_1 \lambda_2 \cdots
\lambda_n$: the determinant is the **product** of the eigenvalues. Multiplying
out, the terms with $\lambda^{n-1}$ are
$(-1)^{n-1}(\lambda_1 + \dots + \lambda_n)\lambda^{n-1}$. In the Leibniz
formula only the product of the diagonal entries,
$(M_{11} - \lambda)\cdots(M_{nn} - \lambda)$, contains $\lambda^{n-1}$ (every
other term misses at least two diagonal entries), and its $\lambda^{n-1}$ terms
are $(-1)^{n-1}(M_{11} + \dots + M_{nn})\lambda^{n-1}$. Comparing: the
**sum** of the eigenvalues is the trace.

**The course.** The author's spacetime has the frame metric
$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order
$x_0, \dots, x_7$ ($x_1, x_2, x_3$ space, $x_4$ time, $x_5, x_6, x_7$ the
three extra times, $x_0$ the hidden direction) and the metric

$$g = \mathrm{diag}\big(\cot^2 z, e^{2a_4} s, e^{2a_4} s, e^{2a_4} s, -1,
-b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s, -b_4 e^{-2a_4} s\big),\quad
s = \sin^{1/3} z,\ z = 6 H x_0,$$

with a function $a_4(x_4)$ and a positive function $b_4(x_4)$ of the time
($b_4 = 1$ in the background of the Revision record), whose extra-time entries
shrink like $e^{-2a_4}$ as $a_4$ grows (for constant $b_4$): the extra
times deflate exponentially. "Signature (4,4)" means: four positive and four
negative eigenvalues. The eight real $16 \times 16$ gamma matrices obey
$(\gamma^a)^2 = \eta^{aa} I$, which fixes their eigenvalues; and the matrix
$B = -i C \gamma^{(x_4)}$ of the record, which gives the charge density of
the fields of the theory, is Hermitian with eigenvalues $+1$ and $-1$.
```

Text cell:

```
## 5. A 2 x 2 example by hand

Take $M$ with the rows $(2, 1)$ and $(1, 2)$. Line by line:

$$\det(M - \lambda I) = (2 - \lambda)(2 - \lambda) - 1 \cdot 1$$

(the $2 \times 2$ determinant $ad - bc$ of the matrix with rows
$(2 - \lambda, 1)$ and $(1, 2 - \lambda)$),

$$= \lambda^2 - 4\lambda + 4 - 1 = \lambda^2 - 4\lambda + 3$$
```

```
(multiply out), $= (\lambda - 1)(\lambda - 3)$ (two numbers with product 3 and
sum 4). So the eigenvalues are 1 and 3. For $\lambda = 3$ the equation
$(M - 3I)v = 0$ reads $-v_1 + v_2 = 0$ (both rows say the same), so
$v = (1, 1)$; for $\lambda = 1$ it reads $v_1 + v_2 = 0$, so $v = (1, -1)$.
Check: $3 + 1 = 4 = \mathrm{tr}\,M$ and $3 \cdot 1 = 3 = 4 - 1 = \det M$.

The next cell repeats this with sympy (‘factor‘ writes a polynomial as a
product, ‘solve‘ finds its roots), checks $M v = \lambda v$ for both
eigenvectors, and compares with numpy’s ‘eigh‘ (the eigenvalue program for
symmetric matrices; it returns the eigenvalues from the smallest up and the
eigenvectors, of length 1, as the columns of a matrix).
```

In [2]:

```
import math  # square roots and pi

import numpy as np  # arrays and numerical linear algebra
import sympy as sp  # exact algebra

lam = sp.Symbol("lambda")  # the unknown eigenvalue
M = sp.Matrix([[2, 1], [1, 2]])
char_poly = sp.expand((M - lam * sp.eye(2)).det())  # det(M - lambda I)
say(f"det(M - lambda I) = {char_poly} = {sp.factor(char_poly)}")
eigenvalues = sorted(sp.solve(char_poly, lam))  # the roots
check(eigenvalues == [1, 3], "the eigenvalues of M are 1 and 3")
v3, v1 = sp.Matrix([1, 1]), sp.Matrix([1, -1])
check(M * v3 == 3 * v3 and M * v1 == 1 * v1,
      "M (1, 1) = 3 (1, 1) and M (1, -1) = 1 (1, -1)")
check(sum(eigenvalues) == M.trace() == 4 and eigenvalues[0] * eigenvalues[1]
      == M.det() == 3, "sum of the eigenvalues = trace = 4, product = det = 3")
values, vectors = np.linalg.eigh(np.array(M.tolist(), dtype=float))
# The sign of an eigenvector is free (v and -v are both eigenvectors), and
# different computers may return either; make the first entry of each column
# positive so that every computer prints the same.
vectors = vectors * np.sign(vectors[0])
say(f"numpy eigh: eigenvalues {values.round(12).tolist()}")
say(f"            eigenvectors (columns) {vectors.round(6).tolist()}")
check(np.allclose(values, [1.0, 3.0], atol=1e-12) and
      abs(abs(vectors[:, 1] @ np.array([1.0, 1.0])) - math.sqrt(2)) < 1e-12,
      "numpy finds the eigenvalues 1, 3 and the direction (1, 1) for 3")
```

Out [2]:

```
det(M - lambda I) = lambda**2 - 4*lambda + 3 = (lambda - 3)*(lambda - 1)
PASS the eigenvalues of M are 1 and 3
PASS M (1, 1) = 3 (1, 1) and M (1, -1) = 1 (1, -1)
PASS sum of the eigenvalues = trace = 4, product = det = 3
numpy eigh: eigenvalues [1.0, 3.0]
            eigenvectors (columns) [[0.707107, 0.707107], [-0.707107, 0.707107]]
PASS numpy finds the eigenvalues 1, 3 and the direction (1, 1) for 3
```

Text cell:

```
The next cell shows the meaning of the result. It takes 721 points on the unit
circle (all vectors of length 1), multiplies each by $M$ and draws the image,
an ellipse. The two eigenvector directions $(1, 1)/\sqrt 2$ and
$(1, -1)/\sqrt 2$ are drawn with their images: they keep their direction and
are stretched by 3 and by 1. The vector $u = (1, 0)$ is drawn as well: its
image $M u = (2, 1)$ points in another direction. The cell checks that the
longest and the shortest image have the lengths 3 and 1, the two eigenvalues
(for a symmetric matrix the ellipse's half-axes are the eigenvalues' sizes).
```

In [3]:

```
angles = np.linspace(0.0, 2.0 * np.pi, 721)  # every half degree
circle = np.array([np.cos(angles), np.sin(angles)])  # 2 rows: x and y
M_numbers = np.array(M.tolist(), dtype=float)
ellipse = M_numbers @ circle  # every point of the circle multiplied by M
lengths = np.sqrt((ellipse ** 2).sum(axis=0))  # length of every image
say(f"longest image {lengths.max():.6f}, shortest image {lengths.min():.6f}")
check(abs(lengths.max() - 3) < 1e-9 and abs(lengths.min() - 1) < 1e-4,
      "the unit circle becomes an ellipse with half-axes 3 and 1")
fig, ax = plt.subplots(figsize=(6.6, 6.0))
ax.plot(circle[0], circle[1], color="0.6", label="unit circle")
ax.plot(ellipse[0], ellipse[1], color="black", label="its image under $M$")
arrows = [(np.array([1.0, 1.0]) / math.sqrt(2), "tab:red", "eigenvector, 3"),
          (np.array([1.0, -1.0]) / math.sqrt(2), "tab:blue", "eigenvector, 1"),
          (np.array([1.0, 0.0]), "tab:green", "u = (1, 0)")]
for vector, color, name in arrows:
    image = M_numbers @ vector
    # the image first: thick, dashed and see-through, so that the vector drawn
    # on top of it stays visible where the two coincide
    ax.annotate("", xy=image, xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": color, "lw": 5,
                            "linestyle": "--", "alpha": 0.4})
    ax.annotate("", xy=vector, xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": color, "lw": 2})
    ax.plot([], [], color=color, lw=2, label=f"{name} (dashed: its image)")
ax.set_aspect("equal")
ax.set_xlim(-3.3, 3.3)
ax.set_ylim(-3.3, 3.3)
ax.set_xlabel("first component")
ax.set_ylabel("second component")
ax.set_title("$M$ with rows (2, 1), (1, 2) acting on the plane")
ax.legend(loc="lower right", fontsize=8)
save_figure(fig, "circle_to_ellipse",
            "The unit circle (grey) and its image under the matrix $M$ with rows "
            "(2, 1) and (1, 2), an ellipse (black); axes the two components of a "
            "vector (pure numbers). Thin solid arrows are vectors of length 1, "
            "thick dashed see-through arrows their images. The eigenvector "
            "$(1, 1)/\\sqrt{2}$ (red) keeps its direction and is stretched by 3, "
            "the eigenvector $(1, -1)/\\sqrt{2}$ (blue) keeps its direction and "
            "its length (eigenvalue 1, so its image lies under it), while "
            "$u = (1, 0)$ (green) is turned to $(2, 1)$. "
            "The half-axes of the ellipse lie along the eigenvectors.")
```

Out [3]:

```
longest image 3.000000, shortest image 1.000000
PASS the unit circle becomes an ellipse with half-axes 3 and 1
```

Out [3]:

```
Figure 01g.1 saved as Revision/textbook/figures/01g_1_circle_to_ellipse.png
```

Text cell:

```
## 6. The characteristic polynomial and the chain matrix

The next cell takes the $3 \times 3$ *chain matrix* $T_3$ with 2 on the
diagonal and $-1$ just above and just below it. Its characteristic polynomial,
multiplied out by sympy, is $-\lambda^3 + 6\lambda^2 - 10\lambda + 4 =
-(\lambda - 2)(\lambda^2 - 4\lambda + 2)$, with the roots $2$ and
$2 \pm \sqrt 2$ (the quadratic formula: $\lambda = 2 \pm \sqrt{4 - 2}$). The
cell checks the roots, the sum rule ($6 = \mathrm{tr}\,T_3$) and the product
rule ($(2 - \sqrt 2) \cdot 2 \cdot (2 + \sqrt 2) = 2 \cdot (4 - 2) = 4 =
\det T_3$), and draws $p(\lambda)$ with its three roots.
```

In [4]:

```
T3 = sp.Matrix([[2, -1, 0], [-1, 2, -1], [0, -1, 2]])
p3 = sp.expand((T3 - lam * sp.eye(3)).det())
say(f"det(T3 - lambda I) = {p3} = {sp.factor(p3)}")
roots3 = sorted(sp.roots(p3, lam).keys(), key=lambda r: float(r))  # small first
say(f"roots: {roots3}")
check(roots3 == [2 - sp.sqrt(2), 2, 2 + sp.sqrt(2)],
      "the eigenvalues of T3 are 2 - sqrt 2, 2 and 2 + sqrt 2")
check(sp.simplify(sum(roots3) - T3.trace()) == 0 and
      sp.simplify(roots3[0] * roots3[1] * roots3[2] - T3.det()) == 0,
      "sum of the eigenvalues = trace = 6, product = det = 4")
p3_numbers = sp.lambdify(lam, p3, "numpy")  # the polynomial as a numpy function
grid = np.linspace(-0.2, 4.2, 400)
fig, ax = plt.subplots(figsize=(7.2, 4.2))
ax.plot(grid, p3_numbers(grid), color="black", label="$p(\\lambda)$")
ax.axhline(0.0, color="0.5", lw=0.8)
root_values = [float(r) for r in roots3]
ax.plot(root_values, [0.0, 0.0, 0.0], "o", color="tab:red", ms=8,
        label="roots = eigenvalues")
for r_text, r_value in zip(["$2 - \\sqrt{2}$", "$2$", "$2 + \\sqrt{2}$"],
                           root_values):
    ax.text(r_value, 0.6, r_text, ha="center", color="tab:red")
ax.set_xlabel("$\\lambda$")
ax.set_ylabel("$p(\\lambda) = \\det(T_3 - \\lambda I)$")
ax.set_title("The characteristic polynomial of the chain matrix $T_3$")
ax.set_ylim(-3.0, 5.0)
ax.legend(loc="upper right");
```

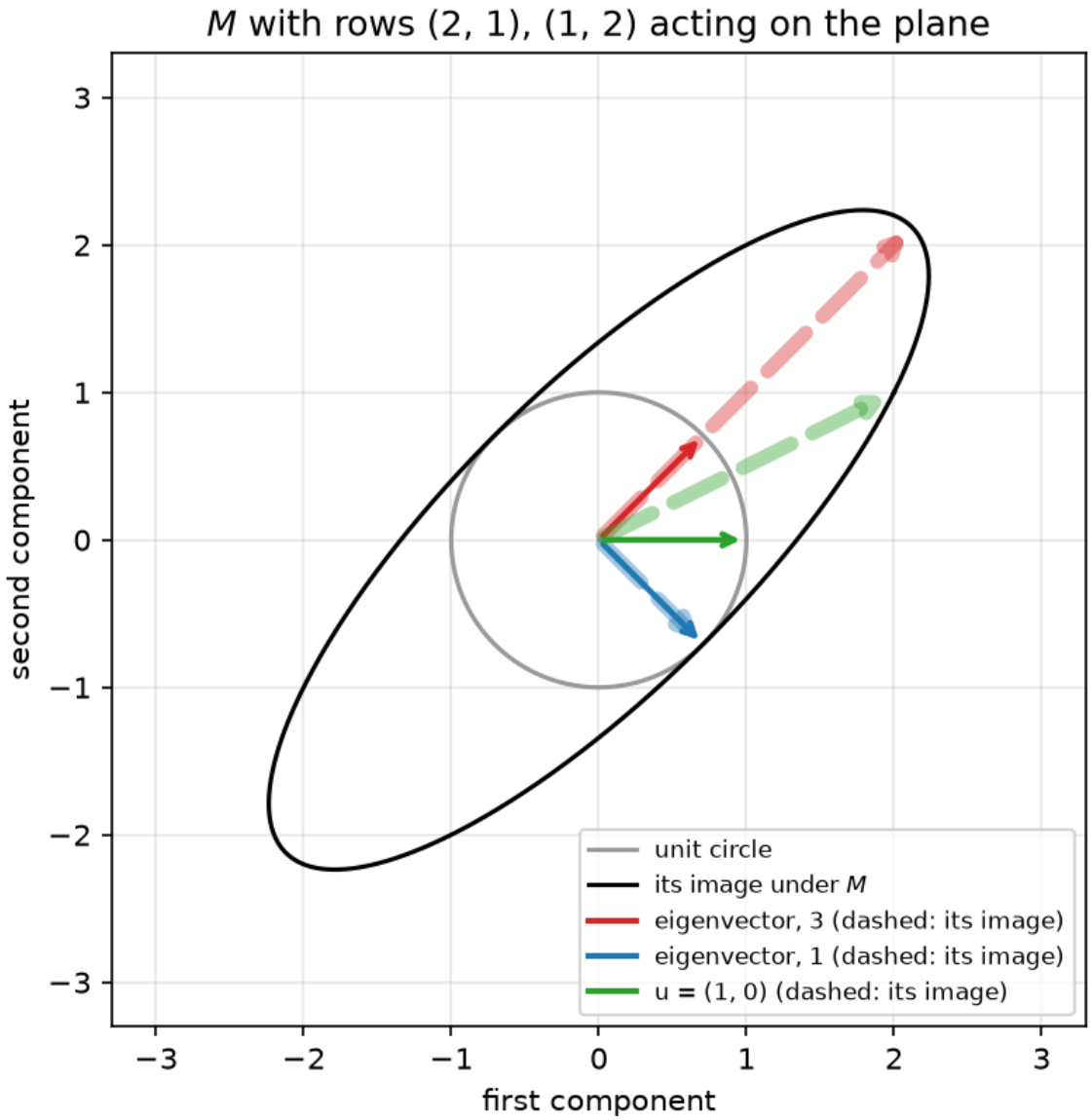

Figure 41: The unit circle (grey) and its image under the matrix M with rows (2, 1) and (1, 2), an ellipse (black); axes the two components of a vector (pure numbers). Thin solid arrows are vectors of length 1, thick dashed see-through arrows their images. The eigenvector $(1,1)/\sqrt{2}$ (red) keeps its direction and is stretched by 3, the eigenvector $(1,-1)/\sqrt{2}$ (blue) keeps its direction and its length (eigenvalue 1, so its image lies under it), while $u = (1,0)$ (green) is turned to $(2,1)$. The half-axes of the ellipse lie along the eigenvectors. (Notebook 01g, figure 1.)

```
save_figure(fig, "characteristic_polynomial",
            "The characteristic polynomial $p(\\lambda) = \\det(T_3 - \\lambda I) "
            "= -\\lambda^3 + 6\\lambda^2 - 10\\lambda + 4$ of the $3 \\times 3$ "
            "chain matrix $T_3$ (2 on the diagonal, $-1$ next to it) against "
            "$\\lambda$ (black), with its three roots $2 - \\sqrt{2}$, 2 and "
            "$2 + \\sqrt{2}$ (red dots); both axes pure numbers. The curve crosses "
            "zero exactly at the eigenvalues; their sum is the trace 6 and their "
            "product the determinant $p(0) = 4$.")
```

Out [4]:

```
det(T3 - lambda I) = -lambda**3 + 6*lambda**2 - 10*lambda + 4 = -(lambda - 2)*(lambda**2
    - 4*lambda + 2)
roots: [2 - sqrt(2), 2, sqrt(2) + 2]
PASS the eigenvalues of T3 are 2 - sqrt 2, 2 and 2 + sqrt 2
PASS sum of the eigenvalues = trace = 6, product = det = 4
```

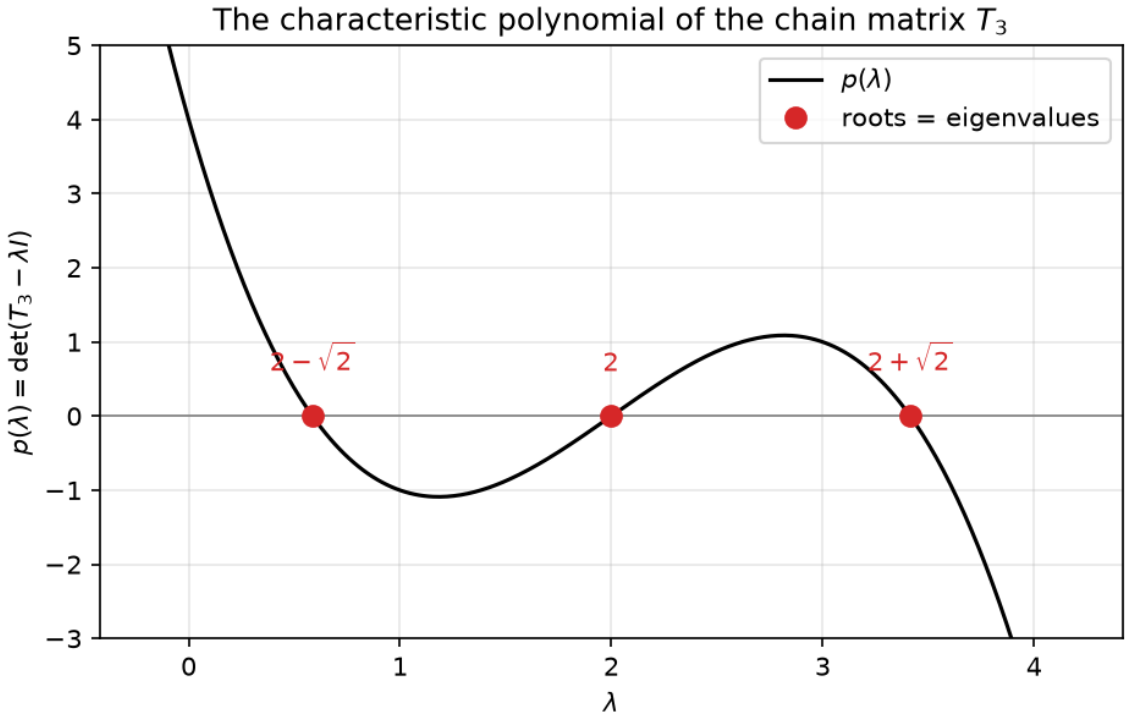

Figure 42: The characteristic polynomial $p(\lambda) = \det(T_3 - \lambda I) = -\lambda^3 + 6\lambda^2 - 10\lambda + 4$ of the 3×3 chain matrix T_3 (2 on the diagonal, -1 next to it) against λ (black), with its three roots $2 - \sqrt{2}$, 2 and $2 + \sqrt{2}$ (red dots); both axes pure numbers. The curve crosses zero exactly at the eigenvalues; their sum is the trace 6 and their product the determinant $p(0) = 4$. (Notebook 01g, figure 2.)

Out [4]:

```
Figure 01g.2 saved as Revision/textbook/figures/01g_2_characteristic_polynomial.png
```

Text cell:

```
The chain matrix $T_n$ of any size $n$ has an exact solution. Row $j$ of
$T_n v$ is $-v_{j-1} + 2v_j - v_{j+1}$, with $v_0 = v_{n+1} = 0$ (the rows at
the ends have only one neighbour). Try $v_j = \sin(j\theta)$:

$$\sin((j-1)\theta) + \sin((j+1)\theta) = 2\sin(j\theta)\cos\theta$$

(the addition theorems $\sin(x \pm y) = \sin x\cos y \pm \cos x\sin y$ with
$x = j\theta$, $y = \theta$, added: the $\cos x \sin y$ terms cancel), so

$$-v_{j-1} + 2v_j - v_{j+1} = (2 - 2\cos\theta)\sin(j\theta).$$

The start $v_0 = \sin 0 = 0$ holds for every $\theta$; the end
$v_{n+1} = \sin((n+1)\theta) = 0$ holds for $\theta = k\pi/(n+1)$,
$k = 1, \dots, n$. So the eigenvalues are $\lambda_k = 2 - 2\cos(k\pi/(n+1))$
and the eigenvectors are sine waves sampled at the points $j = 1, \dots, n$.
This matrix comes back whenever a second derivative is computed on a grid of
points; then the sine waves are the shapes of a vibrating string or of a
quantum particle in a box. The next cell checks the formula for $n = 10$ with
numpy and draws the first three eigenvectors and all ten eigenvalues.
```

In [5]:

```
n = 10
T = 2 * np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1)  # k=1: just above the diagonal
values_T, vectors_T = np.linalg.eigh(T)  # eigenvalues from the smallest up
k = np.arange(1, n + 1)
formula = 2 - 2 * np.cos(k * np.pi / (n + 1))
say(f"largest difference eigh - formula: {np.max(np.abs(values_T - formula)):.1e}")
check(np.allclose(values_T, formula, rtol=0, atol=1e-12),
      "T_10 has the eigenvalues 2 - 2 cos(k pi/11), k = 1 ... 10")
j = np.arange(1, n + 1)  # the points of the chain
sines = np.array([np.sin(j * kk * np.pi / (n + 1)) for kk in k]).T  # columns
sines = sines / np.sqrt((sines ** 2).sum(axis=0))  # length 1
overlaps = np.abs((vectors_T * sines).sum(axis=0))  # |cos of the angle|, column-wise
check(np.allclose(overlaps, 1.0, rtol=0, atol=1e-12),
      "every eigenvector of T_10 is a sampled sine wave sin(j k pi/11)")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
fine = np.linspace(0, n + 1, 400)  # the sine curves between the points
for kk, color in zip((1, 2, 3), ("tab:blue", "tab:orange", "tab:green")):
    column = vectors_T[:, kk - 1] * np.sign(vectors_T[0, kk - 1])  # first entry > 0
    scale = column[0] / math.sin(kk * math.pi / (n + 1))
    left.plot(fine, scale * np.sin(fine * kk * math.pi / (n + 1)), color=color, lw=1)
    left.plot(j, column, "o", color=color, label=f"$k = {kk}$")
left.axhline(0, color="0.5", lw=0.8)
left.set_xlabel("point $j$ of the chain")
left.set_ylabel("entry $v_j$ of the eigenvector")
left.set_title("the first three eigenvectors of $T_{10}$")
left.legend(fontsize=8)
fine_k = np.linspace(0, n + 1, 400)
right.plot(fine_k, 2 - 2 * np.cos(fine_k * np.pi / (n + 1)), color="black", lw=1,
           label="$2 - 2\\cos(k\\pi/11)$")
right.plot(k, values_T, "o", color="tab:red", label="eigenvalues from numpy")
right.set_xlabel("number $k$")
```

```
right.set_ylabel("eigenvalue $\\lambda_k$")
right.set_title("the ten eigenvalues of $T_{10}$")
right.legend(fontsize=8, loc="upper left")
fig.tight_layout()
save_figure(fig, "chain_matrix",
            "The $10 \\times 10$ chain matrix $T_{10}$ (2 on the diagonal, $-1$ "
            "next to it). Left: the entries $v_j$ of its eigenvectors for the "
            "three smallest eigenvalues $k = 1, 2, 3$ (dots) against the point "
            "$j = 1$ to 10, with the sine curves $\\sin(jk\\pi/11)$ through them "
            "(lines): half a wave, a full wave, one and a half waves. Right: the "
            "ten eigenvalues from numpy (red dots) on the curve "
            "$2 - 2\\cos(k\\pi/11)$ (black) against $k$; all axes pure numbers.")
```

Out [5]:

```
largest difference eigh - formula: 8.9e-16
PASS T_10 has the eigenvalues 2 - 2 cos(k pi/11), k = 1 ... 10
PASS every eigenvector of T_10 is a sampled sine wave sin(j k pi/11)
```

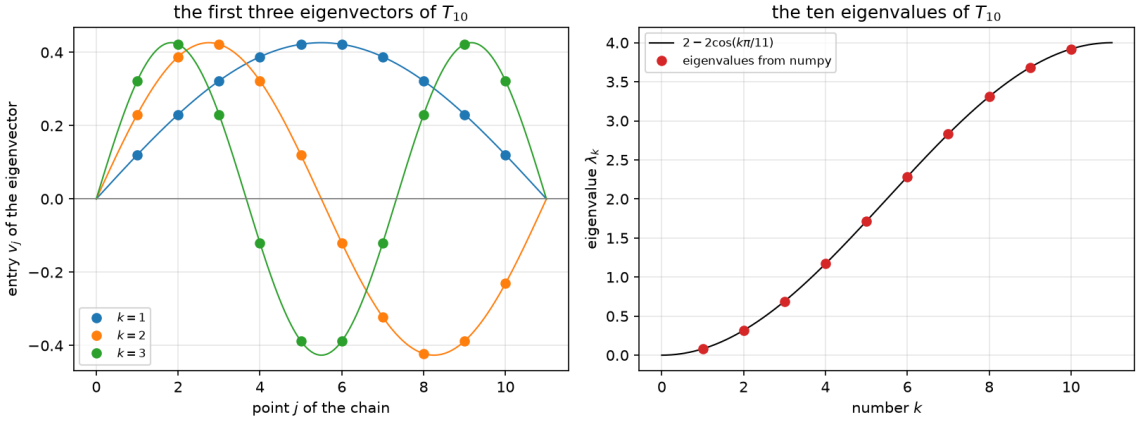

Figure 43: The 10×10 chain matrix T_{10} (2 on the diagonal, -1 next to it). Left: the entries v_j of its eigenvectors for the three smallest eigenvalues $k = 1, 2, 3$ (dots) against the point $j = 1$ to 10, with the sine curves $\sin(jk\pi/11)$ through them (lines): half a wave, a full wave, one and a half waves. Right: the ten eigenvalues from numpy (red dots) on the curve $2 - 2\cos(k\pi/11)$ (black) against k; all axes pure numbers. (Notebook 01g, figure 3.)

Out [5]:

```
Figure 01g.3 saved as Revision/textbook/figures/01g_3_chain_matrix.png
```

Text cell:

```
## 7. Real matrices with complex eigenvalues: rotations and boosts

The rotation matrix $R(\alpha)$ with rows $(\cos\alpha, -\sin\alpha)$ and
$(\sin\alpha, \cos\alpha)$ turns every vector (notebook 01b), so for
```

```
$0 < \alpha < \pi$ no real direction is kept. Its characteristic polynomial is

$$(\cos\alpha - \lambda)^2 + \sin^2\alpha = \lambda^2 - 2\cos\alpha\,\lambda +
1$$

(multiply out and use $\cos^2\alpha + \sin^2\alpha = 1$), with the roots
$\lambda = \cos\alpha \pm \sqrt{\cos^2\alpha - 1} = \cos\alpha \pm
i\sin\alpha = e^{\pm i\alpha}$: complex numbers of modulus 1, with the complex
eigenvectors $(1, \mp i)$. For $\alpha = \pi/2$, $R$ is the matrix $J$ of
notebook 01b with $J^2 = -I$, and its eigenvalues are $\pm i$.

The boost $\Lambda(\varphi)$ with rows $(\cosh\varphi, \sinh\varphi)$ and
$(\sinh\varphi, \cosh\varphi)$ has the characteristic polynomial
$\lambda^2 - 2\cosh\varphi\,\lambda + 1$ (now
$\cosh^2\varphi - \sinh^2\varphi = 1$), with the real roots
$\cosh\varphi \pm \sinh\varphi = e^{\pm\varphi}$ and the eigenvectors
$(1, \pm 1)$, the two light-like directions.

A real matrix can have complex eigenvalues, but they come in pairs: taking the
complex conjugate of $M v = \lambda v$ for a real $M$ gives
$M v^* = \lambda^* v^*$, so $\lambda^*$ is an eigenvalue too. The next cell
checks all of this exactly with sympy (‘expand_complex‘ writes a number as
real part plus $i$ times imaginary part) and draws the eigenvalues.
```

In [6]:

```
alpha, phi = sp.symbols("alpha phi", real=True)
R = sp.Matrix([[sp.cos(alpha), -sp.sin(alpha)], [sp.sin(alpha), sp.cos(alpha)]])
p_rot = sp.expand((R - lam * sp.eye(2)).det())
check(sp.simplify(p_rot - (lam ** 2 - 2 * sp.cos(alpha) * lam + 1)) == 0,
      "det(R - lambda I) = lambda^2 - 2 cos(alpha) lambda + 1")
rot_ok = True
for sign_value in (1, -1):  # the eigenvalue e^(+-i alpha), eigenvector (1, -+i)
    eigen = sp.exp(sign_value * sp.I * alpha)
    vector = sp.Matrix([1, -sign_value * sp.I])
    gap = (R * vector - eigen * vector).applyfunc(
        lambda e: sp.simplify(sp.expand_complex(e)))
    rot_ok &= gap == sp.zeros(2, 1)
check(rot_ok, "R(alpha) (1, -+i) = e^(+-i alpha) (1, -+i) for every angle alpha")
J = R.subs(alpha, sp.pi / 2)  # the matrix of i
say(f"J = {J.tolist()}, eigenvalues {sorted(J.eigenvals().keys(), key=str)}")
check(set(J.eigenvals().keys()) == {sp.I, -sp.I}, "the eigenvalues of J are +i, -i")
boost = sp.Matrix([[sp.cosh(phi), sp.sinh(phi)], [sp.sinh(phi), sp.cosh(phi)]])
boost_ok = True
for sign_value in (1, -1):  # the eigenvalue e^(+-phi), eigenvector (1, +-1)
    vector = sp.Matrix([1, sign_value])
    gap = (boost * vector - sp.exp(sign_value * phi) * vector).applyfunc(
        lambda e: sp.simplify(e.rewrite(sp.exp)))
    boost_ok &= gap == sp.zeros(2, 1)
check(boost_ok, "a boost has the real eigenvalues e^(+-phi) with the light-like "
      "eigenvectors (1, +-1)")
generator = np.random.default_rng(12345)  # random numbers with a fixed seed
pairs_ok = True
```

```
for _ in range(200):  # 200 random real 5 x 5 matrices
    found = np.linalg.eigvals(generator.normal(size=(5, 5)))
    pairs_ok &= np.allclose(np.sort_complex(found), np.sort_complex(found.conj()))
check(pairs_ok, "the complex eigenvalues of 200 random real 5 x 5 matrices come "
      "in conjugate pairs")
```

Out [6]:

```
PASS det(R - lambda I) = lambda^2 - 2 cos(alpha) lambda + 1
PASS R(alpha) (1, -+i) = e^(+-i alpha) (1, -+i) for every angle alpha
J = [[0, -1], [1, 0]], eigenvalues [-I, I]
PASS the eigenvalues of J are +i, -i
PASS a boost has the real eigenvalues e^(+-phi) with the light-like eigenvectors (1, +-1)
PASS the complex eigenvalues of 200 random real 5 x 5 matrices come in conjugate pairs
```

Text cell:

```
The next cell draws the eigenvalue pairs $e^{\pm i\alpha}$ of rotations by
$30, 60, \dots, 180$ degrees on the unit circle of the complex plane (left) and
the eigenvalues $e^{\varphi}$ and $e^{-\varphi}$ of boosts against the rapidity
$\varphi$ (right), whose product is always 1.
```

In [7]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6))
circle_line = np.exp(1j * np.linspace(0, 2 * np.pi, 400))
left.plot(circle_line.real, circle_line.imag, color="0.75", lw=1)
colors = plt.cm.viridis(np.linspace(0.0, 0.9, 6))  # six colours from a colour map
for step, color in zip(range(1, 7), colors):
    angle = step * np.pi / 6
    eig = np.linalg.eigvals(np.array([[np.cos(angle), -np.sin(angle)],
                                      [np.sin(angle), np.cos(angle)]]))
    left.plot(eig.real, eig.imag, "o", color=color, ms=8,
              label=f"$\\alpha = {30 * step}$ degrees")
left.annotate("$+i$ (the matrix $J$)", xy=(0, 1), xytext=(0.25, 1.25), fontsize=8,
              arrowprops={"arrowstyle": "->"})
left.set_aspect("equal")
left.set_xlim(-1.5, 1.5)
left.set_ylim(-1.5, 1.5)
left.set_xlabel("real part")
left.set_ylabel("imaginary part")
left.set_title("rotations: $e^{\\pm i\\alpha}$")
left.legend(fontsize=7, loc="lower left")
rapidity = np.linspace(0.0, 1.5, 200)
right.plot(rapidity, np.exp(rapidity), color="tab:red", label="$e^{\\varphi}$")
right.plot(rapidity, np.exp(-rapidity), color="tab:blue", label="$e^{-\\varphi}$")
right.plot(rapidity, np.exp(rapidity) * np.exp(-rapidity), "k--", lw=1,
           label="their product = 1")
right.set_xlabel("rapidity $\\varphi$")
right.set_ylabel("eigenvalue")
right.set_title("boosts: real eigenvalues $e^{\\pm\\varphi}$")
```

```
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "rotation_and_boost",
            "Left: the eigenvalues $e^{\\pm i\\alpha}$ of the real rotation "
            "matrices $R(\\alpha)$ for $\\alpha = 30$ to 180 degrees in steps of "
            "30 (one colour per angle) in the complex plane, on the unit circle "
            "(grey); horizontal axis the real part, vertical axis the imaginary "
            "part. Each pair is mirror-symmetric in the real axis; $\\alpha = 90$ "
            "degrees gives $\\pm i$, the eigenvalues of $J$. Right: the real "
            "eigenvalues $e^{\\varphi}$ (red) and $e^{-\\varphi}$ (blue) of a "
            "boost against its rapidity $\\varphi$ (pure numbers); their product "
            "is always 1 (dashed).")
```

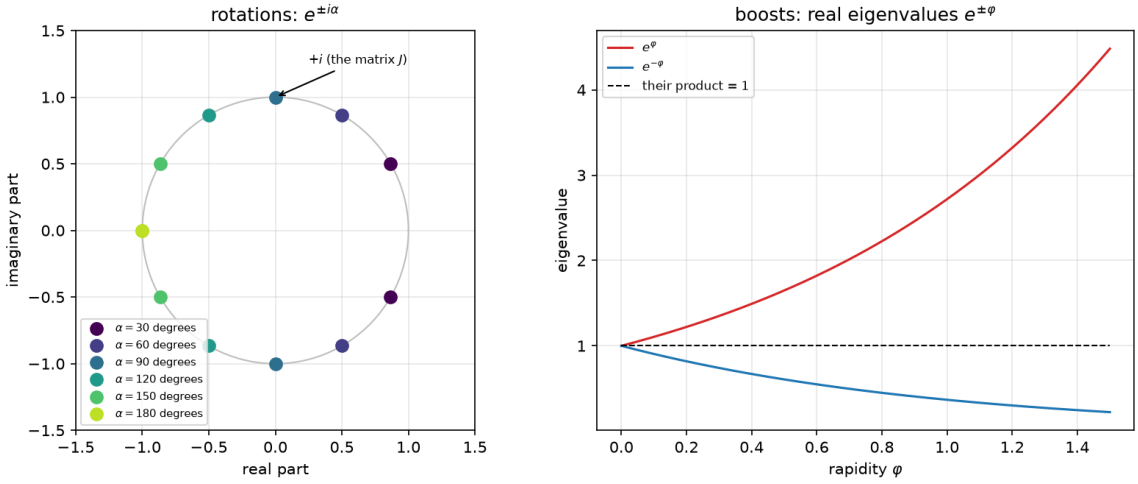

Figure 44: Left: the eigenvalues $e^{\pm i\alpha}$ of the real rotation matrices $R(\alpha)$ for $\alpha = 30$ to 180 degrees in steps of 30 (one colour per angle) in the complex plane, on the unit circle (grey); horizontal axis the real part, vertical axis the imaginary part. Each pair is mirror-symmetric in the real axis; $\alpha = 90$ degrees gives $\pm i$, the eigenvalues of J. Right: the real eigenvalues e^{φ} (red) and $e^{-\varphi}$ (blue) of a boost against its rapidity φ (pure numbers); their product is always 1 (dashed). (Notebook 01g, figure 4.)

Out [7]:

```
Figure 01g.4 saved as Revision/textbook/figures/01g_4_rotation_and_boost.png
```

Text cell:

```
## 8. Symmetric matrices: real eigenvalues, perpendicular eigenvectors

**Claim 1.** A real symmetric matrix $S$ has only real eigenvalues. **Proof**,
line by line. Let $S v = \lambda v$ with a column $v \neq 0$ that may be
complex.

- Conjugate and transpose both sides: $v^\dagger S^\dagger = \lambda^*
  v^\dagger$, and $S^\dagger = S$ because $S$ is real ($S^* = S$) and
  symmetric ($S^T = S$); so $v^\dagger S = \lambda^* v^\dagger$.
```

```
- Multiply $S v = \lambda v$ from the left by $v^\dagger$:
  $v^\dagger S v = \lambda\, v^\dagger v$.
- Multiply $v^\dagger S = \lambda^* v^\dagger$ from the right by $v$:
  $v^\dagger S v = \lambda^*\, v^\dagger v$.
- Subtract the two lines: $0 = (\lambda - \lambda^*)\, v^\dagger v$.
- $v^\dagger v = |v_1|^2 + \dots + |v_n|^2 > 0$, so $\lambda = \lambda^*$:
  $\lambda$ is real.

The same lines, with $S^\dagger = S$ as the only property used, prove it for a
complex **Hermitian** matrix.

**Claim 2.** Eigenvectors of a symmetric matrix with different eigenvalues are
perpendicular. **Proof.** Let $S u = \lambda u$, $S w = \mu w$, $\lambda \neq
\mu$. Then $u^T S w = \mu\, u^T w$, and also $u^T S w = (S^T u)^T w = (S u)^T w
= \lambda\, u^T w$. Subtracting: $(\lambda - \mu)\, u^T w = 0$, so
$u^T w = 0$.

Hence (and one can show this also when eigenvalues repeat) $S = O \Lambda O^T$,
with the eigenvalues on the diagonal of $\Lambda$ and the eigenvectors as the
columns of an orthogonal matrix $O$. The next cell checks this on 300 random
symmetric $8 \times 8$ matrices $S = A + A^T$ and contrasts them with the 300
non-symmetric matrices $A$, whose eigenvalues are often complex.
```

In [8]:

```
symmetric_spectra, general_spectra = [], []
largest_imaginary = 0.0
decomposition_error = 0.0
for _ in range(300):
    A = generator.normal(size=(8, 8))  # a random real 8 x 8 matrix
    S = A + A.T  # its symmetric part, doubled
    found = np.linalg.eigvals(S)  # the general program: complex results allowed
    largest_imaginary = max(largest_imaginary, float(np.max(np.abs(found.imag))))
    symmetric_spectra.append(found.real)
    general_spectra.append(np.linalg.eigvals(A))
    w, O = np.linalg.eigh(S)  # eigenvalues w and eigenvectors (columns of O)
    decomposition_error = max(
        decomposition_error,
        float(np.max(np.abs(O.T @ O - np.eye(8)))),  # O^T O = I
        float(np.max(np.abs(O @ np.diag(w) @ O.T - S))))  # O Lambda O^T = S
general_all = np.concatenate(general_spectra)
complex_share = float(np.mean(np.abs(general_all.imag) > 1e-9))
say(f"symmetric: largest imaginary part {largest_imaginary:.1e}; "
    f"non-symmetric: share of complex eigenvalues {complex_share:.3f}")
check(largest_imaginary < 1e-9, "300 random symmetric matrices: every eigenvalue "
      "is real")
check(decomposition_error < 1e-12,
      "S = O Lambda O^T with O^T O = I for all 300 symmetric matrices")
check(complex_share > 0.3, "the non-symmetric matrices have many complex "
      "eigenvalues")
report("share of complex eigenvalues of 300 random 8 x 8 matrices",
       f"{complex_share:.3f}")
```

Out [8]:

```
symmetric: largest imaginary part 0.0e+00; non-symmetric: share of complex eigenvalues
    0.677
PASS 300 random symmetric matrices: every eigenvalue is real
PASS S = O Lambda O^T with O^T O = I for all 300 symmetric matrices
PASS the non-symmetric matrices have many complex eigenvalues
RESULT share of complex eigenvalues of 300 random 8 x 8 matrices = 0.677
```

Text cell:

```
The next cell draws all $300 \cdot 8 = 2400$ eigenvalues of each kind in the
complex plane.
```

In [9]:

```
symmetric_all = np.concatenate(symmetric_spectra)
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4), sharey=True)
left.plot(symmetric_all, np.zeros_like(symmetric_all), "|", color="tab:blue",
          ms=14, alpha=0.3)
left.set_title("symmetric $S = A + A^T$: all on the real axis")
right.plot(general_all.real, general_all.imag, ".", color="tab:red", ms=3,
           alpha=0.5)
right.set_title("non-symmetric $A$: conjugate pairs")
for ax in (left, right):
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("real part")
left.set_ylabel("imaginary part")
fig.tight_layout()
save_figure(fig, "random_spectra",
            "The eigenvalues of 300 random real $8 \\times 8$ matrices in the "
            "complex plane (horizontal axis real part, vertical axis imaginary "
            "part, pure numbers). Left: the symmetric matrices $S = A + A^T$; "
            "every one of the 2400 eigenvalues lies on the real axis (blue "
            "strokes). Right: the non-symmetric matrices $A$ (red dots); many "
            "eigenvalues are complex, and the picture is mirror-symmetric in the "
            "real axis because complex eigenvalues of a real matrix come in "
            "conjugate pairs.")
```

Out [9]:

```
Figure 01g.5 saved as Revision/textbook/figures/01g_5_random_spectra.png
```

Text cell:

```
## 9. The signature and Sylvester's law of inertia

A symmetric matrix $S$ defines the quadratic form $Q(v) = v^T S v$. For the
frame metric, $Q(v) = \eta_{ab} v^a v^b$ is the squared length of notebook
```

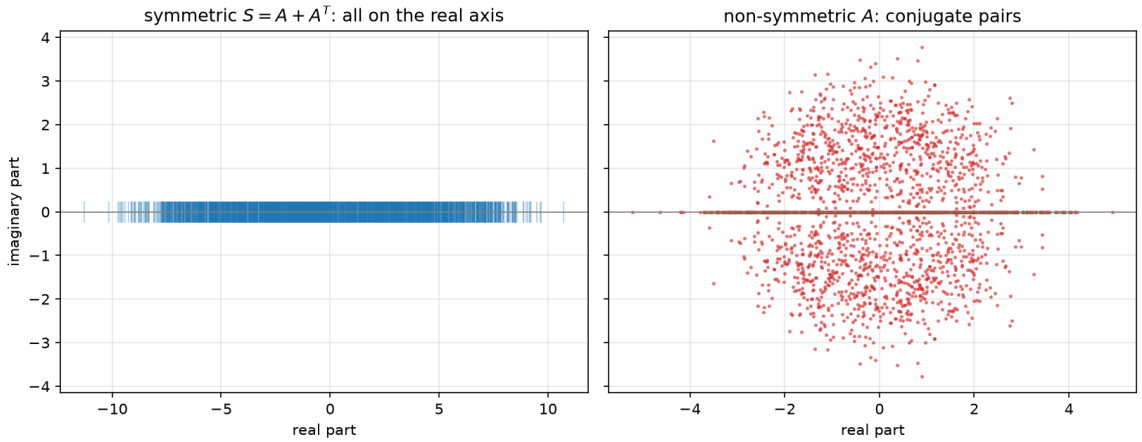

Figure 45: The eigenvalues of 300 random real 8×8 matrices in the complex plane (horizontal axis real part, vertical axis imaginary part, pure numbers). Left: the symmetric matrices $S = A + A^T$; every one of the 2400 eigenvalues lies on the real axis (blue strokes). Right: the non-symmetric matrices A (red dots); many eigenvalues are complex, and the picture is mirror-symmetric in the real axis because complex eigenvalues of a real matrix come in conjugate pairs. (Notebook 01g, figure 5.)

```
01e. In the eigenvector coordinates $v = O w$ (the columns of $O$ are the
eigenvectors of $S$) it becomes $Q = \lambda_1 w_1^2 + \dots + \lambda_n
w_n^2$, because $O^T S O = \Lambda$. So the **signature** $(p, q)$, the
numbers of positive and negative eigenvalues, says in how many perpendicular
directions $Q$ is positive and in how many negative.

A change of coordinates $v = P w$ with an invertible matrix $P$ gives
$Q = w^T (P^T S P) w$: the matrix of the form becomes $P^T S P$. Its
eigenvalues are different, but **Sylvester's law of inertia** says that the
signature is the same. The reason in words: let $S$ have $p$ and $P^T S P$
have $p'$ positive eigenvalues, and suppose $p' < p$. On every nonzero
combination of the $p$ eigenvectors of $S$ with positive eigenvalues,
$Q > 0$. On every combination of the $n - p'$ vectors $P u$, where $u$ runs
through the eigenvectors of $P^T S P$ with eigenvalues $\leq 0$, $Q \leq 0$.
Together these are $p + n - p' > n$ vectors in $n$-dimensional space, so a
nonzero combination of the first kind equals a combination of the second
kind, and on that vector $Q > 0$ and $Q \leq 0$ at once, which is
impossible. So $p' \geq p$; exchanging the roles of the two matrices gives
$p \geq p'$, and the same argument for $-S$ counts the negative eigenvalues.
So the signature (4,4) of the author's spacetime is a property of the
spacetime, not of the coordinates used to describe it.

The next cell reads $\eta$ from the Revision record, finds its eigenvalues
(for a diagonal matrix: its diagonal entries) and its signature, and checks the
law on 2000 random invertible matrices $P$ (it also makes sure that no
eigenvalue is so close to 0 that rounding could change its sign: each must be
larger than 1000 times the rounding error, $2.2 \times 10^{-16}$ times the
largest size). Then it reads the author's metric
from the record and counts the signs of its eigenvalues on a grid of 61 values
of $a_4$ from $-3$ to 3, 50 values of $z$ in $(0, \pi/2)$ and the three values
```

```
$b_4 = 0.5$, 1 and 2; their product must be $\det g = b_4^3 \cos^2 z > 0$.
```

In [10]:

```
algebra = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
names = algebra["coordinates"]  # ["x0", ..., "x7"]
eta = algebra["eta"]  # the diagonal of the frame metric
eta_matrix = np.diag(np.array(eta, dtype=float))
eta_values = np.linalg.eigvalsh(eta_matrix)  # eigenvalues from the smallest up
signature = (int((eta_values > 0).sum()), int((eta_values < 0).sum()))
say(f"eigenvalues of eta: {eta_values.tolist()}; signature {signature}")
algebra_report = json.loads(repository_file(
    "Revision/algebra/reports/python-algebra.json").read_text(encoding="utf-8"))
verdicts = {c["name"]: c["verdict"] for c in algebra_report["checks"]}
check(signature == (4, 4) and verdicts["coordinate_map"] == "pass",
      "eta has 4 positive and 4 negative eigenvalues: the signature (4,4)",
      record="Revision/algebra/reports/python-algebra.json, check coordinate_map")
sylvester_counts = set()
smallest_size, largest_size = np.inf, 0.0
transformed_spectra = []
for trial in range(2000):
    P = generator.normal(size=(8, 8))  # a random matrix (invertible: det != 0)
    changed = np.linalg.eigvalsh(P.T @ eta_matrix @ P)
    sylvester_counts.add((int((changed > 0).sum()), int((changed < 0).sum())))
    smallest_size = min(smallest_size, float(np.min(np.abs(changed))))
    largest_size = max(largest_size, float(np.max(np.abs(changed))))
    if trial < 40:
        transformed_spectra.append(changed)
say(f"signatures of P^T eta P found: {sorted(sylvester_counts)}; eigenvalue "
    f"sizes from {smallest_size:.1e} to {largest_size:.1f}")
# Rounding changes an eigenvalue by about 2.2e-16 times the largest size; the
# signs are reliable if the smallest size is far above that.
reliable = smallest_size > 1000 * 2.2e-16 * largest_size
check(sylvester_counts == {(4, 4)} and reliable, "Sylvester: P^T eta P has the "
      "signature (4,4) for all 2000 random P, although its eigenvalues change")
```

Out [10]:

```
eigenvalues of eta: [-1.0, -1.0, -1.0, -1.0, 1.0, 1.0, 1.0, 1.0]; signature (4, 4)
PASS eta has 4 positive and 4 negative eigenvalues: the signature (4,4)
     reproduces Revision/algebra/reports/python-algebra.json, check coordinate_map
signatures of P^T eta P found: [(4, 4)]; eigenvalue sizes from 1.4e-08 to 37.0
PASS Sylvester: P^T eta P has the signature (4,4) for all 2000 random P, although its
    eigenvalues change
```

Text cell:

```
The next cell reads the author's metric from the record (as in notebook 01a),
evaluates its eight eigenvalues on the grid and checks their signs and their
product.
```

In [11]:

```
a4 = sp.Symbol("a4", real=True)  # the value of a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of b4(x4) at one time
z = sp.Symbol("z", positive=True)  # z = 6 H x0, between 0 and pi/2
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))


def from_record(text):
    """The record's Mathematica text as a sympy expression."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "sin(z)")
    text = text.replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "z": z, "E": sp.E})


g_record = [from_record(text) for text in curvature["metricDiagonal"]]
# a4, b4, z -> the 8 numbers on the diagonal
metric_numbers = sp.lambdify((a4, b4, z), g_record, "numpy")
signs_ok, product_error, points = True, 0.0, 0
for b4_value in (0.5, 1.0, 2.0):
    for a4_value in np.linspace(-3.0, 3.0, 61):
        for z_value in np.linspace(0.05, np.pi / 2 - 0.05, 50):
            g_values = np.linalg.eigvalsh(np.diag(np.array(
                metric_numbers(a4_value, b4_value, z_value), dtype=float)))
            signs_ok &= ((int((g_values > 0).sum()), int((g_values < 0).sum()))
                         == (4, 4))
            product_error = max(product_error, abs(
                np.prod(g_values) / b4_value ** 3 - np.cos(z_value) ** 2))
            points += 1
check(signs_ok and points == 9150, "the author's metric has the signature (4,4) "
      "at all 9150 grid points (a4, z, b4)")
lovelock_report = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json")
    .read_text(encoding="utf-8"))
root_verdict = {c["name"]: c["verdict"]
                for c in lovelock_report["checks"]}["sqrt_abs_det_g"]
check(product_error < 1e-12 and root_verdict == "PASS",
      "the product of the eight eigenvalues is det g = b4^3 cos(z)^2 at every grid "
      "point",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "sqrt_abs_det_g")
```

Out [11]:

```
PASS the author's metric has the signature (4,4) at all 9150 grid points (a4, z, b4)
PASS the product of the eight eigenvalues is det g = b4^3 cos(z)^2 at every grid point
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    sqrt_abs_det_g
```

Text cell:

```
The next cell draws both results. Left: the eight eigenvalues of
$P^T \eta P$ for 40 random $P$ (one column per $P$). Right: the eight
eigenvalues of the author's metric at $z = 0.9$, $b_4 = 1$ against $a_4$. Both
vertical
axes are *symmetric logarithmic*: linear in a small band around 0 (between
$-0.001$ and 0.001 on the left, between $-0.1$ and 0.1 on the right) and
logarithmic outside, so that large and small values of both signs fit.
```

In [12]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.5, 4.6))
for column, spectrum in enumerate(transformed_spectra):
    colors_column = ["tab:red" if value > 0 else "tab:blue" for value in spectrum]
    left.scatter(np.full(8, column + 1), spectrum, c=colors_column, s=12)
left.axhline(0.0, color="black", lw=0.8)
left.set_yscale("symlog", linthresh=0.001)  # linear between -0.001 and 0.001
left.set_xlabel("random matrix $P$ (number)")
left.set_ylabel("eigenvalues of $P^T \\eta P$")
left.set_title("always 4 positive (red) and 4 negative (blue)")
a4_line = np.linspace(-3.0, 3.0, 301)
curves = np.array([metric_numbers(value, 1.0, 0.9) for value in a4_line],
                  dtype=float)  # at b4 = 1
styles = [("tab:green", "hidden $x_0$", 0), ("tab:red", "space $x_1, x_2, x_3$", 1),
          ("tab:gray", "time $x_4$", 4), ("tab:blue", "extra times $x_5, x_6, x_7$", 5)]
for color, label, index in styles:
    right.plot(a4_line, curves[:, index], color=color, lw=2, label=label)
right.axhline(0.0, color="black", lw=0.8)
right.set_yscale("symlog", linthresh=0.1)
right.set_xlabel("$a_4$")
right.set_ylabel("eigenvalues of $g$ at $z = 0.9$")
right.set_title("the signs never change: signature (4,4)")
right.legend(fontsize=8, loc="upper left")
fig.tight_layout()
# how many of the 320 drawn eigenvalues are smaller than 0.001 in size
tiny = sum(int((np.abs(spectrum) < 0.001).sum()) for spectrum in transformed_spectra)
tiny_text = "the one eigenvalue" if tiny == 1 else f"the {tiny} eigenvalues"
tiny_verb = "sits" if tiny == 1 else "sit"  # singular or plural
save_figure(fig, "signature",
            "Left: the eight eigenvalues of $P^T \\eta P$ for 40 random matrices "
            "$P$ (one column of dots per matrix; red positive, blue negative); "
            "the values change from matrix to matrix, but every column has four "
            "positive and four negative eigenvalues (Sylvester's law of inertia); "
            f"{tiny_text} smaller than 0.001 in size {tiny_verb} on the zero "
            "line, but none is zero. "
            "Right: the eigenvalues of the author's metric $g$ at $z = 6 H x_0 = "
            "0.9$ and $b_4 = 1$ against $a_4$: the three space entries "
            "$e^{2a_4} s$ (red, "
            "growing), the time entry $-1$ (grey), the three extra-time entries "
            "$-e^{-2a_4} s$ (blue, shrinking towards 0 as the extra times deflate, "
            "but never crossing it) and the hidden entry $\\cot^2 z$ (green). The "
            "vertical axes are linear near 0 (between $-0.001$ and 0.001 on the "
            "left, $-0.1$ and 0.1 on the right) and logarithmic outside; all "
            "quantities are pure numbers.")
```

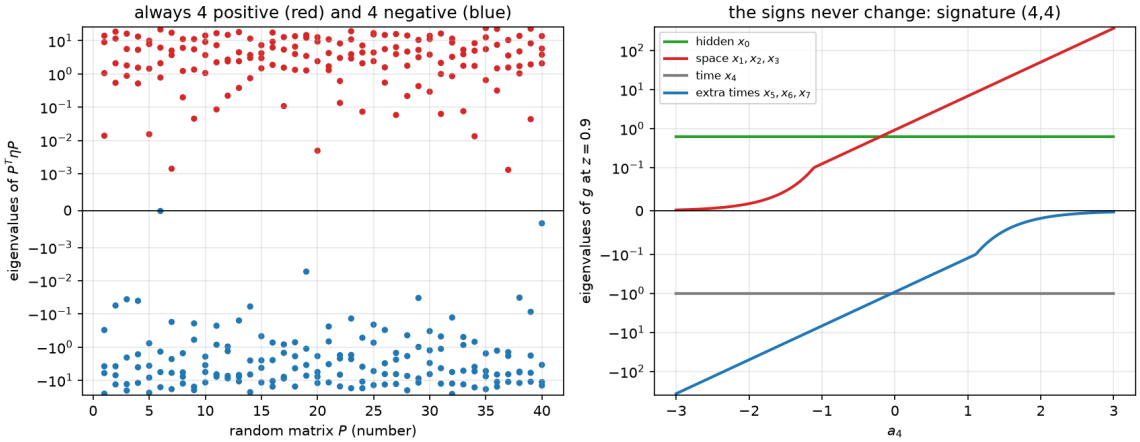

Figure 46: Left: the eight eigenvalues of $P^T\eta P$ for 40 random matrices P (one column of dots per matrix; red positive, blue negative); the values change from matrix to matrix, but every column has four positive and four negative eigenvalues (Sylvester's law of inertia); the 2 eigenvalues smaller than 0.001 in size sit on the zero line, but none is zero. Right: the eigenvalues of the author's metric g at $z = 6Hx_0 = 0.9$ and $b_4 = 1$ against a_4: the three space entries $e^{2a_4}s$ (red, growing), the time entry -1 (grey), the three extra-time entries $-e^{-2a_4}s$ (blue, shrinking towards 0 as the extra times deflate, but never crossing it) and the hidden entry $\cot^2 z$ (green). The vertical axes are linear near 0 (between -0.001 and 0.001 on the left, -0.1 and 0.1 on the right) and logarithmic outside; all quantities are pure numbers. (Notebook 01g, figure 6.)

Out [12]:

```
Figure 01g.6 saved as Revision/textbook/figures/01g_6_signature.png
```

Text cell:

```
## 10. The eigenvalues of the gamma matrices

The eight real gamma matrices of the Revision record obey
$(\gamma^a)^2 = \eta^{aa} I$ (the Clifford relation with equal indices). If
$\gamma^a v = \lambda v$, then, line by line,

$$(\gamma^a)^2 v = \gamma^a(\lambda v) = \lambda\,\gamma^a v = \lambda^2 v$$

(apply $\gamma^a$ twice; a number can be moved in front), and also
$(\gamma^a)^2 v = \eta^{aa} v$. So $\lambda^2 = \eta^{aa}$:
$\lambda = \pm 1$ for the space-like directions $x_0, x_1, x_2, x_3$ and
$\lambda = \pm i$ for the time-like directions $x_4, \dots, x_7$.

How many of each? The trace is 0: for any $b \neq a$,
$\mathrm{tr}\,\gamma^a = \eta^{bb}\,\mathrm{tr}(\gamma^b\gamma^b\gamma^a)$
(insert $\eta^{bb}(\gamma^b)^2 = \eta^{bb}\eta^{bb} I = I$),
$= \eta^{bb}\,\mathrm{tr}(\gamma^b\gamma^a\gamma^b)$ (the trace does not
change when the first factor is moved to the end),
$= -\eta^{bb}\,\mathrm{tr}(\gamma^b\gamma^b\gamma^a)$ (the anticommutation
```

```
$\gamma^a\gamma^b = -\gamma^b\gamma^a$), $= -\mathrm{tr}\,\gamma^a$. A number
equal to its own negative is 0. The trace is the sum of the 16 eigenvalues, so
$+1$ and $-1$ (or $+i$ and $-i$) occur 8 times each: the characteristic
polynomial is $(\lambda^2 - \eta^{aa})^8$.

More generally, for any vector $v$ the matrix $\gamma(v) = v_a\gamma^a$ (with
$v_a = \eta_{ab} v^b$) squares to $\gamma(v)^2 = v_a v_b \gamma^a \gamma^b =
\tfrac12 v_a v_b (\gamma^a\gamma^b + \gamma^b\gamma^a) = v_a v_b \eta^{ab} I =
Q(v)\, I$ (only the symmetric part of $\gamma^a\gamma^b$ counts, notebook 01e),
so its eigenvalues are $\pm\sqrt{Q(v)}$: real for a space-like, imaginary for a
time-like and 0 for a light-like $v$. A light-like $v$ gives a *nilpotent*
matrix: $\gamma(v) \neq 0$ but $\gamma(v)^2 = 0$. The next cell checks all of
this with the matrices of the record.
```

In [13]:

```
gamma = [np.array(matrix, dtype=int) for matrix in algebra["gamma"]]  # gamma^a
polynomials_ok, traces_ok, numeric_ok = True, True, True
for name, eta_aa, g_a in zip(names, eta, gamma):
    poly = sp.Matrix(g_a.tolist()).charpoly(lam).as_expr()  # det(lambda I - g_a)
    polynomials_ok &= sp.expand(poly - (lam ** 2 - eta_aa) ** 8) == 0
    traces_ok &= int(np.trace(g_a)) == 0
    found = np.linalg.eigvals(g_a.astype(float))
    if eta_aa == 1:  # space-like: +1 and -1, eight times each
        numeric_ok &= bool(np.allclose(np.sort(found.real), [-1] * 8 + [1] * 8)
                           and np.allclose(found.imag, 0))
    else:  # time-like: +i and -i, eight times each
        numeric_ok &= bool(np.allclose(found.real, 0)
                           and np.allclose(np.sort(found.imag), [-1] * 8 + [1] * 8))
    say(f"gamma^({name}): eta {eta_aa:+d}, trace {int(np.trace(g_a))}, "
        f"characteristic polynomial {sp.factor(poly)}")
check(traces_ok and polynomials_ok,
      "every gamma^a has trace 0 and the characteristic polynomial "
      "(lambda^2 - eta_aa)^8", record="Revision/algebra/reports/python-algebra.json, "
      "check clifford_relation (the relations with equal indices)")
check(numeric_ok and verdicts["symmetry_pattern"] == "pass",
      "numpy: +1 and -1 (x0, x1, x2, x3: symmetric matrices) or +i and -i "
      "(x4 ... x7: antisymmetric matrices), 8 times each",
      record="Revision/algebra/reports/python-algebra.json, check symmetry_pattern")
```

Out [13]:

```
gamma^(x0): eta +1, trace 0, characteristic polynomial (lambda - 1)**8*(lambda + 1)**8
gamma^(x1): eta +1, trace 0, characteristic polynomial (lambda - 1)**8*(lambda + 1)**8
gamma^(x2): eta +1, trace 0, characteristic polynomial (lambda - 1)**8*(lambda + 1)**8
gamma^(x3): eta +1, trace 0, characteristic polynomial (lambda - 1)**8*(lambda + 1)**8
gamma^(x4): eta -1, trace 0, characteristic polynomial (lambda**2 + 1)**8
gamma^(x5): eta -1, trace 0, characteristic polynomial (lambda**2 + 1)**8
gamma^(x6): eta -1, trace 0, characteristic polynomial (lambda**2 + 1)**8
gamma^(x7): eta -1, trace 0, characteristic polynomial (lambda**2 + 1)**8
PASS every gamma^a has trace 0 and the characteristic polynomial (lambda^2 - eta_aa)^8
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
```

```
    (the relations with equal indices)
PASS numpy: +1 and -1 (x0, x1, x2, x3: symmetric matrices) or +i and -i (x4 ... x7:
    antisymmetric matrices), 8 times each
     reproduces Revision/algebra/reports/python-algebra.json, check symmetry_pattern
```

Text cell:

```
The next cell builds $\gamma(v)$ for four vectors: the unit vectors along $x_1$
($Q = 1$) and $x_5$ ($Q = -1$), their sum ($Q = 0$, light-like) and
$v = (1, 2, \dots, 8)$ ($Q = -48$, notebook 01e). It checks
$\gamma(v)^2 = Q(v) I$ exactly with whole numbers, the eigenvalues
$\pm\sqrt{Q}$ with numpy, and for the light-like vector that $\gamma(v)$ is
not zero but squares to zero and has the characteristic polynomial
$\lambda^{16}$ (computed exactly with sympy: numpy is not reliable for a
nilpotent matrix, whose eigenvectors do not fill the space). Then it repeats
the square rule for 100 random vectors with whole-number components.
```

In [14]:

```
def gamma_of(vector):
    """gamma(v) = v_a gamma^a with v_a = eta_ab v^b (whole numbers)."""
    lowered = [eta[a] * int(vector[a]) for a in range(8)]
    return sum(lowered[a] * gamma[a] for a in range(8))


def squared_length(vector):
    """Q(v) = eta_ab v^a v^b."""
    return sum(eta[a] * int(vector[a]) ** 2 for a in range(8))


unit = np.eye(8, dtype=int)
examples = [("e(x1)", unit[1]), ("e(x5)", unit[5]), ("e(x1) + e(x5)", unit[1] + unit[5]),
            ("(1, 2, ..., 8)", np.arange(1, 9))]
example_spectra = {}
squares_ok, nilpotent_ok = True, True
for label, vector in examples:
    matrix = gamma_of(vector)
    q = squared_length(vector)
    squares_ok &= bool((matrix @ matrix == q * np.eye(16, dtype=int)).all())
    if q == 0:  # light-like: exact answer from sympy
        poly = sp.Matrix(matrix.tolist()).charpoly(lam).as_expr()
        nilpotent_ok &= poly == lam ** 16 and bool(np.count_nonzero(matrix) > 0)
        example_spectra[label] = np.zeros(16, dtype=complex)
    else:
        example_spectra[label] = np.linalg.eigvals(matrix.astype(float))
        target = np.sqrt(complex(q))  # sqrt(Q), imaginary for Q < 0
        # count the eigenvalues within 1e-9 of +sqrt(Q) and of -sqrt(Q)
        near_plus = int((np.abs(example_spectra[label] - target) < 1e-9).sum())
        near_minus = int((np.abs(example_spectra[label] + target) < 1e-9).sum())
        squares_ok &= near_plus == 8 and near_minus == 8
    say(f"v = {label}: Q(v) = {q}, gamma(v)^2 = Q(v) I")
for _ in range(100):  # random vectors with whole-number components -5 ... 5
```

```
    vector = generator.integers(-5, 6, size=8)
    matrix = gamma_of(vector)
    squares_ok &= bool((matrix @ matrix
                        == squared_length(vector) * np.eye(16, dtype=int)).all())
check(squares_ok, "gamma(v)^2 = Q(v) I exactly and the eigenvalues are +-sqrt(Q(v)) "
      "(4 examples and 100 random vectors)")
check(nilpotent_ok, "a light-like v gives a nilpotent gamma(v): not zero, square "
      "zero, characteristic polynomial lambda^16")
```

Out [14]:

```
v = e(x1): Q(v) = 1, gamma(v)^2 = Q(v) I
v = e(x5): Q(v) = -1, gamma(v)^2 = Q(v) I
v = e(x1) + e(x5): Q(v) = 0, gamma(v)^2 = Q(v) I
v = (1, 2, ..., 8): Q(v) = -144, gamma(v)^2 = Q(v) I
PASS gamma(v)^2 = Q(v) I exactly and the eigenvalues are +-sqrt(Q(v)) (4 examples and 100
    random vectors)
PASS a light-like v gives a nilpotent gamma(v): not zero, square zero, characteristic
    polynomial lambda^16
```

Text cell:

```
The next cell draws the eigenvalues of the eight gamma matrices (left) and of
the four matrices $\gamma(v)$ (right) in the complex plane.
```

In [15]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.8))
for name, eta_aa, g_a in zip(names, eta, gamma):
    found = np.linalg.eigvals(g_a.astype(float))
    color = "tab:red" if eta_aa == 1 else "tab:blue"
    left.plot(found.real, found.imag, "o", color=color, ms=10, alpha=0.3)
left.text(1.0, 0.18, "$+1$: 8 times\n($x_0, x_1, x_2, x_3$)", ha="center",
          fontsize=8)
left.text(-1.0, 0.18, "$-1$: 8 times", ha="center", fontsize=8)
left.text(0.08, 1.0, "$+i$: 8 times ($x_4 \\dots x_7$)", fontsize=8)
left.text(0.08, -1.05, "$-i$: 8 times", fontsize=8)
left.set_xlim(-1.6, 1.9)
left.set_ylim(-1.5, 1.5)
left.set_title("the eight gamma matrices")
markers = {"e(x1)": ("s", "tab:red"), "e(x5)": ("D", "tab:blue"),
           "e(x1) + e(x5)": ("*", "black"), "(1, 2, ..., 8)": ("o", "tab:purple")}
for label, spectrum in example_spectra.items():
    shape, color = markers[label]
    right.plot(spectrum.real, spectrum.imag, shape, color=color, ms=10,
               label=f"$v$ = {label}, $Q = {squared_length(dict(examples)[label])}$")
right.set_xlim(-3.0, 3.0)
right.set_ylim(-8.0, 8.0)
right.set_title("$\\gamma(v) = v_a \\gamma^a$: eigenvalues $\\pm\\sqrt{Q(v)}$")
right.legend(fontsize=8, loc="lower right")
for ax in (left, right):
```

```
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.axvline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("real part")
    ax.set_ylabel("imaginary part")
fig.tight_layout()
save_figure(fig, "gamma_eigenvalues",
            "Eigenvalues in the complex plane (horizontal axis real part, vertical "
            "axis imaginary part, pure numbers). Left: the 16 eigenvalues of each "
            "of the eight real gamma matrices of the Revision record: $\\pm 1$, "
            "eight times each, for the space-like $x_0, x_1, x_2, x_3$ (red) and "
            "$\\pm i$, eight times each, for the time-like $x_4$ to $x_7$ (blue). "
            "Right: the eigenvalues of $\\gamma(v) = v_a \\gamma^a$ for four "
            "vectors: $\\pm 1$ for the unit vector along $x_1$ ($Q = 1$), "
            "$\\pm i$ along $x_5$ ($Q = -1$), only 0 for the light-like sum of "
            "the two ($Q = 0$, black star) and $\\pm i\\sqrt{48}$ for "
            "$v = (1, 2, \\dots, 8)$ ($Q = -48$).")
```

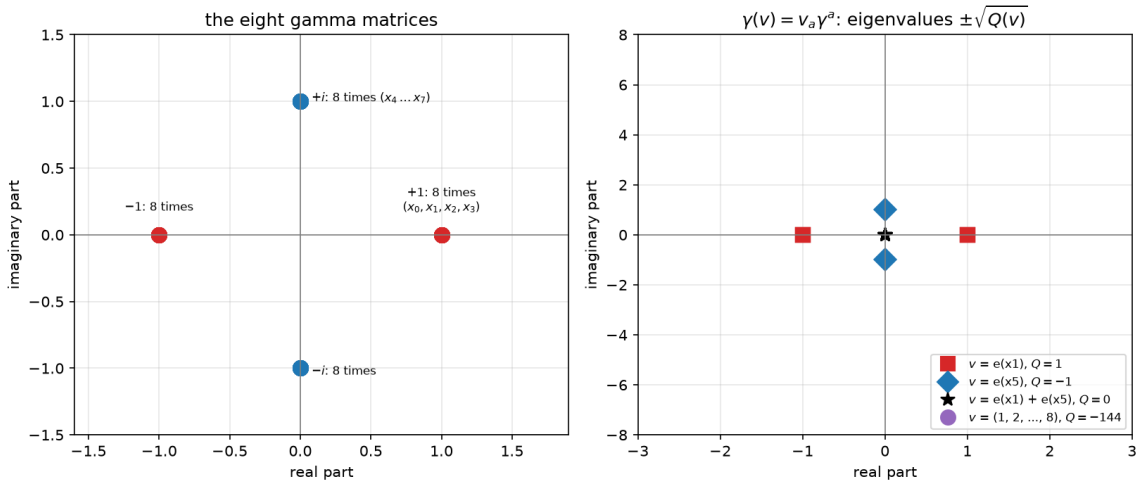

Figure 47: Eigenvalues in the complex plane (horizontal axis real part, vertical axis imaginary part, pure numbers). Left: the 16 eigenvalues of each of the eight real gamma matrices of the Revision record: ± 1, eight times each, for the space-like x_0, x_1, x_2, x_3 (red) and $\pm i$, eight times each, for the time-like x_4 to x_7 (blue). Right: the eigenvalues of $\gamma(v) = v_a \gamma^a$ for four vectors: ± 1 for the unit vector along x_1 ($Q = 1$), $\pm i$ along x_5 ($Q = -1$), only 0 for the light-like sum of the two ($Q = 0$, black star) and $\pm i\sqrt{48}$ for $v = (1, 2, \dots, 8)$ ($Q = -48$). (Notebook 01g, figure 7.)

Out [15]:

```
Figure 01g.7 saved as Revision/textbook/figures/01g_7_gamma_eigenvalues.png
```

Text cell:

```
## 11. A Hermitian matrix of the record: B and its signature (8,8)

The record stores the matrix $B = -i\, C\gamma^{(x_4)}$, where
$C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ is real. $B$ is
```

```
complex: its real part is 0, its imaginary part a real $16 \times 16$ matrix of
entries $-1$, 0, 1. In the theory of the course $B$ gives the charge density
$J^{(x_4)} = \Psi^\dagger B \Psi$ of the fields. Its properties, line by line:

- $B$ is Hermitian ($B^\dagger = B$); for a matrix $iA$ with real $A$ this
  means $(iA)^\dagger = -i A^T = iA$, that is $A^T = -A$: the imaginary part is
  antisymmetric. By section 8 its eigenvalues are real.
- $B^2 = I$, so as in section 10 every eigenvalue has $\lambda^2 = 1$:
  $\lambda = \pm 1$.
- $\mathrm{tr}\,B = 0$, so $+1$ and $-1$ occur 8 times each: the signature
  (8,8), and the characteristic polynomial is $(\lambda - 1)^8(\lambda + 1)^8$.

Because half of the eigenvalues of $B$ are negative, the charge
$\Psi^\dagger B \Psi$ can be positive or negative: the record calls it an
indefinite form, and for the quantised field it leads to an indefinite
(Krein) state space: a space of states whose product of two states, built with
$B$ as in $\Psi^\dagger B \Psi$, takes the place of the dot product, so that a
state can have a negative "squared length" (the dot product of an ordinary
vector with itself is never negative). The next cell checks the four properties
exactly with sympy and with numpy (‘eigh‘ also works for Hermitian matrices; its
eigenvectors are perpendicular in the sense $u^\dagger w = 0$).
```

In [16]:

```
B = sp.Matrix(algebra["B"]["re"]) + sp.I * sp.Matrix(algebra["B"]["im"])
C = sp.Matrix(algebra["C"])
check(B == -sp.I * C * sp.Matrix(algebra["gamma"][4]),
      "the record's B equals -i C gamma^(x4) computed from its C and gamma^(x4)")
poly_B = sp.factor(B.charpoly(lam).as_expr())
real_part_zero = sp.Matrix(algebra["B"]["re"]).is_zero_matrix  # True or False
say(f"B: real part zero {real_part_zero}, trace {B.trace()}, characteristic "
    f"polynomial {poly_B}")
check(B.H == B and B * B == sp.eye(16) and B.trace() == 0
      and sp.expand(poly_B - (lam - 1) ** 8 * (lam + 1) ** 8) == 0,
      "B is Hermitian, B^2 = I, tr B = 0, characteristic polynomial "
      "(lambda - 1)^8 (lambda + 1)^8: the signature (8,8)",
      record="Revision/algebra/reports/python-algebra.json, check "
             "B_hermitian_involution_signature")
B_numbers = np.array(B.tolist(), dtype=complex)
values_B, vectors_B = np.linalg.eigh(B_numbers)
unitary_error = float(np.max(np.abs(vectors_B.conj().T @ vectors_B - np.eye(16))))
say(f"numpy eigh: eigenvalues {values_B.round(12).tolist()}")
theory = json.loads(repository_file(
    "Revision/theory/reports/python-field-theory.json").read_text(encoding="utf-8"))
theory_verdict = {c["name"]: c["verdict"] for c in theory["checks"]}["B_properties"]
krein_entry = theory["formulas"]["quantisation"]["krein"]  # a sentence of the record
say(f"the record's quantisation entry: {krein_entry}")
check(np.allclose(values_B, [-1.0] * 8 + [1.0] * 8, rtol=0, atol=1e-12)
      and unitary_error < 1e-12 and theory_verdict == "pass",
      "numpy: eight eigenvalues -1 and eight +1, perpendicular eigenvectors",
      record="Revision/theory/reports/python-field-theory.json, check B_properties")
```

Out [16]:

```
PASS the record's B equals -i C gamma^(x4) computed from its C and gamma^(x4)
B: real part zero True, trace 0, characteristic polynomial (lambda - 1)**8*(lambda +
    1)**8
PASS B is Hermitian, B^2 = I, tr B = 0, characteristic polynomial (lambda - 1)^8 (lambda
    + 1)^8: the signature (8,8)
     reproduces Revision/algebra/reports/python-algebra.json, check
    B_hermitian_involution_signature
numpy eigh: eigenvalues [-1.0, -1.0, -1.0, -1.0, -1.0, -1.0, -1.0, -1.0, 1.0, 1.0, 1.0,
    1.0, 1.0, 1.0, 1.0, 1.0]
the record's quantisation entry: B Hermitian, B^2 = 1, signature (8,8): indefinite
    (Krein) state space
PASS numpy: eight eigenvalues -1 and eight +1, perpendicular eigenvectors
     reproduces Revision/theory/reports/python-field-theory.json, check B_properties
```

Text cell:

```
The next cell draws the imaginary part of $B$ as a heat map and the 16
eigenvalues of $B$ in order.
```

In [17]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6),
                                  gridspec_kw={"width_ratios": [1.1, 1]})
image = left.imshow(B_numbers.imag, cmap="RdBu_r", vmin=-1, vmax=1)
left.grid(False)
ticks = range(0, 16, 3)
left.set_xticks(ticks, [str(t + 1) for t in ticks])
left.set_yticks(ticks, [str(t + 1) for t in ticks])
left.set_xlabel("column")
left.set_ylabel("row")
left.set_title("imaginary part of $B$ (the real part is 0)")
fig.colorbar(image, ax=left, shrink=0.8, ticks=[-1, 0, 1])
right.bar(np.arange(1, 17), values_B,
          color=["tab:blue" if value < 0 else "tab:red" for value in values_B])
right.axhline(0.0, color="black", lw=0.8)
right.set_xticks(range(1, 17, 3))
right.set_ylim(-1.4, 1.4)
right.set_xlabel("eigenvalue number (from the smallest up)")
right.set_ylabel("eigenvalue of $B$")
right.set_title("eight $-1$ and eight $+1$: signature (8,8)")
fig.tight_layout()
save_figure(fig, "hermitian_b",
            "Left: heat map of the imaginary part of the Hermitian matrix "
            "$B = -i C\\gamma^{(x_4)}$ of the Revision record (rows and columns 1 "
            "to 16; red $+1$, blue $-1$, white 0; its real part is 0). The picture "
            "changes sign when mirrored in the diagonal: the imaginary part is "
            "antisymmetric, which makes $B$ Hermitian. Right: the 16 eigenvalues "
            "of $B$ from the smallest up (pure numbers): eight times $-1$ (blue) "
            "and eight times $+1$ (red), the signature (8,8).")
```

Out [17]:

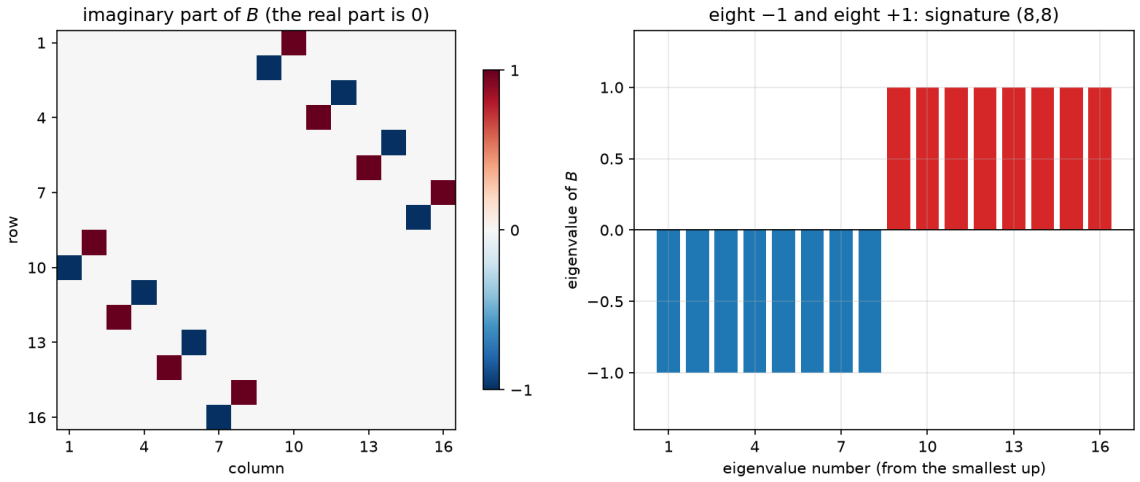

Figure 48: Left: heat map of the imaginary part of the Hermitian matrix $B = -iC\gamma^{(x_4)}$ of the Revision record (rows and columns 1 to 16; red +1, blue −1, white 0; its real part is 0). The picture changes sign when mirrored in the diagonal: the imaginary part is antisymmetric, which makes B Hermitian. Right: the 16 eigenvalues of B from the smallest up (pure numbers): eight times −1 (blue) and eight times +1 (red), the signature (8,8). (Notebook 01g, figure 8.)

```
Figure 01g.8 saved as Revision/textbook/figures/01g_8_hermitian_b.png
```

Text cell:

```
## 12. How a computer finds an eigenvalue: power iteration

A computer does not find eigenvalues as the roots of $\det(M - \lambda I)$
(that polynomial is very sensitive to rounding for large matrices); it
transforms the matrix step by step. The simplest such method is **power
iteration**. Write a starting vector as a sum of eigenvectors,
$x_0 = c_1 u_1 + c_2 u_2 + \dots$, with $|\lambda_1| > |\lambda_2| \geq \dots$.
Multiplying by $M$ $k$ times gives

$$M^k x_0 = c_1 \lambda_1^k u_1 + c_2 \lambda_2^k u_2 + \dots =
\lambda_1^k\Big(c_1 u_1 + c_2 (\lambda_2/\lambda_1)^k u_2 + \dots\Big)$$

(each eigenvector is only multiplied by its eigenvalue; then $\lambda_1^k$ is
taken out), so the direction approaches $u_1$, and the error shrinks by the
factor $|\lambda_2/\lambda_1|$ in each step. Rescaling to length 1 after each
step keeps the numbers from growing. The next cell runs 40 steps from
$x_0 = (1, 0)$ for the matrix $M$ of section 5 (factor $1/3$) and from
$(1, 0, 0)$ for $T_3$ of section 6 (factor $2/(2 + \sqrt 2) = 2 - \sqrt 2
\approx 0.586$), measures the distance to the true eigenvector and checks the
factor.
```

In [18]:

```
def power_iteration(matrix, start, steps):
    """Multiply by matrix and rescale to length 1, steps times; return the
```

```
    distances to the eigenvector of the largest eigenvalue after each step."""
    values_m, vectors_m = np.linalg.eigh(matrix)
    top = vectors_m[:, -1]  # the eigenvector of the largest eigenvalue
    x = np.array(start, dtype=float)
    distances = []
    for _ in range(steps):
        x = matrix @ x
        x = x / np.sqrt(x @ x)  # rescale to length 1
        aligned = top if x @ top > 0 else -top  # the sign of an eigenvector is free
        distances.append(float(np.sqrt(((x - aligned) ** 2).sum())))
    return np.array(distances), abs(values_m[-2] / values_m[-1])


runs = {"M (2 x 2)": power_iteration(M_numbers, [1, 0], 40),
        "T3 (3 x 3)": power_iteration(np.array(T3.tolist(), dtype=float),
                                      [1, 0, 0], 40)}
factors_ok = True
for label, (distances, ratio) in runs.items():
    measured = distances[11:21] / distances[10:20]  # steps 11 to 20
    say(f"{label}: |lambda2/lambda1| = {ratio:.6f}, measured factors "
        f"{measured.min():.6f} to {measured.max():.6f}")
    factors_ok &= bool(np.all(np.abs(measured - ratio) < 1e-3))
check(factors_ok, "power iteration: each step shrinks the error by |lambda2/lambda1|"
      " (1/3 for M, 2 - sqrt 2 for T3)")
fig, ax = plt.subplots(figsize=(7.5, 4.4))
steps_axis = np.arange(1, 41)
for (label, (distances, ratio)), color in zip(runs.items(), ("tab:red", "tab:blue")):
    # a distance 0 (exact agreement) is drawn at 1e-17, the bottom of the axis
    ax.semilogy(steps_axis, np.maximum(distances, 1e-17), "o", color=color, ms=4,
                label=label)
    ax.semilogy(steps_axis, distances[0] * ratio ** (steps_axis - 1), "-",
                color=color, lw=1, label=f"factor {ratio:.3f} per step")
ax.set_ylim(1e-17, 2.0)
ax.set_xlabel("step $k$")
ax.set_ylabel("distance to the eigenvector")
ax.set_title("Power iteration")
ax.legend(fontsize=8);
save_figure(fig, "power_iteration",
            "Power iteration: the distance between the rescaled vector "
            "$M^k x_0/|M^k x_0|$ and the eigenvector of the largest eigenvalue "
            "after $k$ steps (dots), for the $2 \\times 2$ matrix $M$ (red, "
            "eigenvalues 3 and 1) and the $3 \\times 3$ chain matrix $T_3$ (blue, "
            "largest eigenvalues $2 + \\sqrt{2}$ and 2), with the lines "
            "$|\\lambda_2/\\lambda_1|^{k-1}$ times the first distance; horizontal "
            "axis $k$, vertical axis the distance on a logarithmic scale (pure "
            "numbers). The dots follow straight lines until the distance reaches "
            "the rounding of floating-point numbers, about $10^{-16}$ (a distance "
            "that is exactly 0 is drawn at the bottom of the axis).")
```

Out [18]:

```
M (2 x 2): |lambda2/lambda1| = 0.333333, measured factors 0.333333 to 0.333333
T3 (3 x 3): |lambda2/lambda1| = 0.585786, measured factors 0.585786 to 0.585789
```

```
PASS power iteration: each step shrinks the error by |lambda2/lambda1| (1/3 for M, 2 -
    sqrt 2 for T3)
```

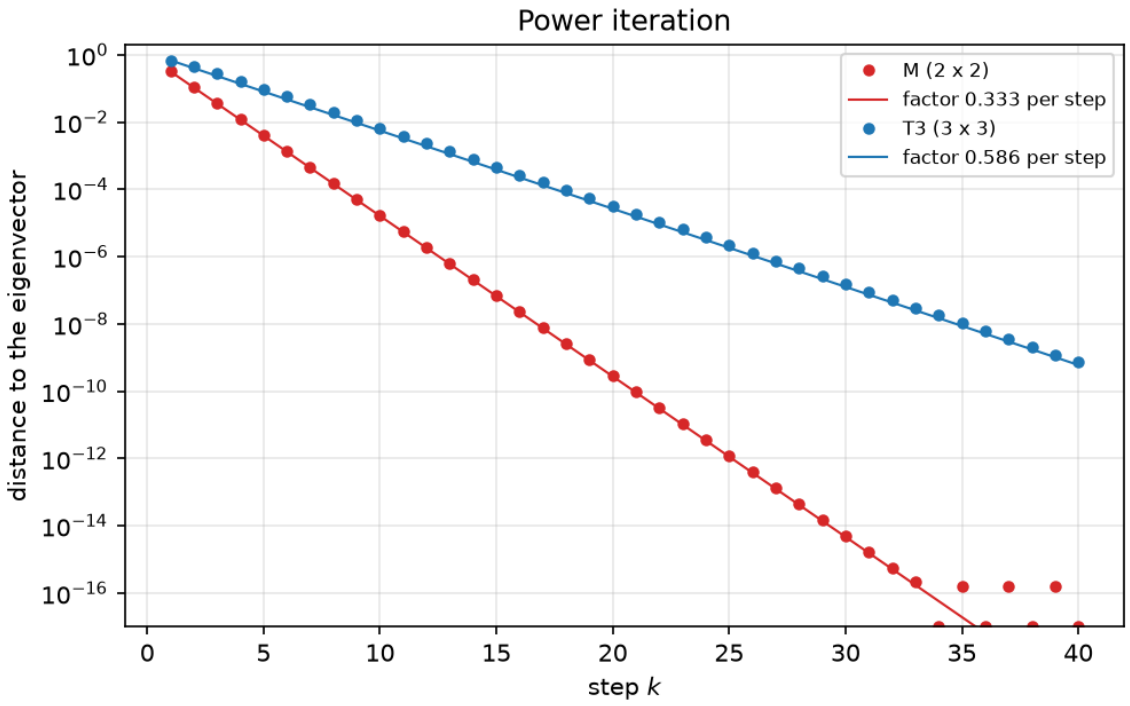

Figure 49: Power iteration: the distance between the rescaled vector $M^k x_0/|M^k x_0|$ and the eigenvector of the largest eigenvalue after k steps (dots), for the 2×2 matrix M (red, eigenvalues 3 and 1) and the 3×3 chain matrix T_3 (blue, largest eigenvalues $2 + \sqrt{2}$ and 2), with the lines $|\lambda_2/\lambda_1|^{k-1}$ times the first distance; horizontal axis k, vertical axis the distance on a logarithmic scale (pure numbers). The dots follow straight lines until the distance reaches the rounding of floating-point numbers, about 10^{-16} (a distance that is exactly 0 is drawn at the bottom of the axis). (Notebook 01g, figure 9.)

Out [18]:

```
Figure 01g.9 saved as Revision/textbook/figures/01g_9_power_iteration.png
```

Text cell:

```
## 13. The last check

The last cell checks that the nine figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [19]:

```
figure_names = [f"01g_{k}_{name}.png" for k, name in enumerate(
    ["circle_to_ellipse", "characteristic_polynomial", "chain_matrix",
```

```
     "rotation_and_boost", "random_spectra", "signature", "gamma_eigenvalues",
     "hermitian_b", "power_iteration"], start=1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 9 figure files of this notebook exist")
all_checks_passed()
```

Out [19]:

```
PASS all 9 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 01g)
```

Text cell:

```
## 14. What this notebook showed

- The eigenvalues of a square matrix $M$ are the roots of its characteristic
  polynomial $\det(M - \lambda I)$; they add up to the trace and multiply to
  the determinant (derived, and checked for $2 \times 2$ and $3 \times 3$
  examples). An eigenvector keeps its direction; for a symmetric matrix the
  unit circle becomes an ellipse whose half-axes lie along the eigenvectors.
- The chain matrix $T_n$ has the eigenvalues $2 - 2\cos(k\pi/(n+1))$ and sampled
  sine waves as eigenvectors (derived, and checked for $n = 10$).
- Real rotations have the complex eigenvalues $e^{\pm i\alpha}$ (for $J$:
  $\pm i$), boosts the real eigenvalues $e^{\pm\varphi}$ with light-like
  eigenvectors; complex eigenvalues of a real matrix come in conjugate pairs.
- Symmetric and Hermitian matrices have real eigenvalues and perpendicular
  eigenvectors (proved; checked on 300 random matrices).
- The frame metric $\eta$ and the author's metric $g$ have four positive and
  four negative eigenvalues, the signature (4,4), at every point; the deflation
  of the extra times shrinks their eigenvalues towards 0 without changing their
  sign, and the product of the eigenvalues is $\det g = b_4^3 \cos^2 z$. No change of
  coordinates changes the signature (Sylvester's law of inertia, checked on 2000
  random coordinate changes).
- The real gamma matrices of the record have the eigenvalues $\pm 1$
  (space-like directions) or $\pm i$ (time-like directions), eight times each,
  because $(\gamma^a)^2 = \eta^{aa} I$ and $\mathrm{tr}\,\gamma^a = 0$; the
  matrix $\gamma(v) = v_a\gamma^a$ has the eigenvalues $\pm\sqrt{Q(v)}$ and is
  nilpotent for a light-like $v$.
- The Hermitian matrix $B = -iC\gamma^{(x_4)}$ of the record has eight
  eigenvalues $+1$ and eight $-1$: the signature (8,8), which makes the charge
  $\Psi^\dagger B \Psi$ an indefinite form.
- Power iteration finds the eigenvector of the largest eigenvalue; each step
  shrinks the error by $|\lambda_2/\lambda_1|$.
```

1.41 Line-by-line walk-through of Notebook 01g

The notebook has nineteen code cells, In [1] to In [19]. As in Section 1.9, a quoted line `...)` stands for the remaining lines of a figure caption, which Section 1.40 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 1.39; its code is the set-up code explained in Section 1.9, with `NOTEBOOK_ID = "01g"`. It prints Set-up of notebook 01g complete: repository folder found, helpers defined.

In [2], a 2 × 2 example.

```
import math  # square roots and pi

import numpy as np  # arrays and numerical linear algebra
import sympy as sp  # exact algebra
```

The modules: `math` for square roots and π, numpy for numerical linear algebra (its sub-module `np.linalg` holds the eigenvalue programs), sympy for exact algebra.

```
lam = sp.Symbol("lambda")  # the unknown eigenvalue
M = sp.Matrix([[2, 1], [1, 2]])
char_poly = sp.expand((M - lam * sp.eye(2)).det())  # det(M - lambda I)
say(f"det(M - lambda I) = {char_poly} = {sp.factor(char_poly)}")
```

A symbol for λ (named `lam`, because `lambda` is a reserved word of Python), the matrix M, and its characteristic polynomial $\det(M - \lambda I)$, multiplied out. `sp.factor` writes it as a product. Output: `det(M - lambda I) = lambda**2 - 4*lambda + 3 = (lambda - 3)*(lambda - 1)`.

```
eigenvalues = sorted(sp.solve(char_poly, lam))  # the roots
check(eigenvalues == [1, 3], "the eigenvalues of M are 1 and 3")
v3, v1 = sp.Matrix([1, 1]), sp.Matrix([1, -1])
check(M * v3 == 3 * v3 and M * v1 == 1 * v1,
      "M (1, 1) = 3 (1, 1) and M (1, -1) = 1 (1, -1)")
check(sum(eigenvalues) == M.trace() == 4 and eigenvalues[0] * eigenvalues[1]
      == M.det() == 3, "sum of the eigenvalues = trace = 4, product = det = 3")
```

`sp.solve(p, lam)` returns the roots of p, and `sorted` puts them in increasing order. Three exact checks: the eigenvalues 1 and 3, the two eigenvector equations $Mv = \lambda v$, and the sum and product rules of Section 1.34.

```
values, vectors = np.linalg.eigh(np.array(M.tolist(), dtype=float))
# The sign of an eigenvector is free (v and -v are both eigenvectors), and
# different computers may return either; make the first entry of each column
# positive so that every computer prints the same.
vectors = vectors * np.sign(vectors[0])
```

`np.linalg.eigh` is numpy's eigenvalue program for symmetric (and Hermitian) matrices; it returns the eigenvalues from the smallest up and the eigenvectors, of length 1, as the columns of a matrix. Because v and $-v$ are both eigenvectors, the program may return either; `np.sign(vectors[0])` is the sign of the first entry of each column, and multiplying each column by it makes that entry positive, so that every computer prints the same.

```
say(f"numpy eigh: eigenvalues {values.round(12).tolist()}")
say(f"            eigenvectors (columns) {vectors.round(6).tolist()}")
check(np.allclose(values, [1.0, 3.0], atol=1e-12) and
      abs(abs(vectors[:, 1] @ np.array([1.0, 1.0])) - math.sqrt(2)) < 1e-12,
      "numpy finds the eigenvalues 1, 3 and the direction (1, 1) for 3")
```

The results, rounded (`.round(12)`: 12 digits after the point), printed as `[1.0, 3.0]` and the columns `[[0.707107, 0.707107], [-0.707107, 0.707107]]` (the rows of the matrix are printed; its first column is $(1,-1)/\sqrt{2}$, its second $(1,1)/\sqrt{2}$, and $1/\sqrt{2} = 0.707107$). The check: the eigenvalues, and the second column has the dot product $\pm\sqrt{2}$ with $(1,1)$, so it is $\pm(1,1)/\sqrt{2}$. The cell prints four PASS lines.

In [3], the unit circle becomes an ellipse.

```
angles = np.linspace(0.0, 2.0 * np.pi, 721)  # every half degree
circle = np.array([np.cos(angles), np.sin(angles)])  # 2 rows: x and y
M_numbers = np.array(M.tolist(), dtype=float)
ellipse = M_numbers @ circle  # every point of the circle multiplied by M
lengths = np.sqrt((ellipse ** 2).sum(axis=0))  # length of every image
say(f"longest image {lengths.max():.6f}, shortest image {lengths.min():.6f}")
check(abs(lengths.max() - 3) < 1e-9 and abs(lengths.min() - 1) < 1e-4,
      "the unit circle becomes an ellipse with half-axes 3 and 1")
```

721 points of the unit circle (every half degree) as a table with two rows; their images under M; the length of every image (square the entries, add the two rows with `.sum(axis=0)`, take the root). Output: longest image 3.000000, shortest image 1.000000, the half-axes of Section 1.34. The check uses a tight tolerance for the longest and a looser one, 10^{-4}, for the shortest length; the printed values show that both are met with room to spare.

```
fig, ax = plt.subplots(figsize=(6.6, 6.0))
ax.plot(circle[0], circle[1], color="0.6", label="unit circle")
ax.plot(ellipse[0], ellipse[1], color="black", label="its image under $M$")
arrows = [(np.array([1.0, 1.0]) / math.sqrt(2), "tab:red", "eigenvector, 3"),
          (np.array([1.0, -1.0]) / math.sqrt(2), "tab:blue", "eigenvector, 1"),
          (np.array([1.0, 0.0]), "tab:green", "u = (1, 0)")]
```

The circle in grey and the ellipse in black; a list of three vectors of length 1 with colours and names: the two eigenvectors and $u = (1,0)$.

```
for vector, color, name in arrows:
    image = M_numbers @ vector
    # the image first: thick, dashed and see-through, so that the vector drawn
    # on top of it stays visible where the two coincide
    ax.annotate("", xy=image, xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": color, "lw": 5,
                            "linestyle": "--", "alpha": 0.4})
    ax.annotate("", xy=vector, xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": color, "lw": 2})
    ax.plot([], [], color=color, lw=2, label=f"{name} (dashed: its image)")
```

For each vector its image is drawn first as a thick, dashed, see-through arrow, then the vector itself as a thin solid arrow on top. `ax.plot([], [], ...)` draws nothing (empty lists) but makes an entry for the legend.

```
ax.set_aspect("equal")
ax.set_xlim(-3.3, 3.3)
```

```
ax.set_ylim(-3.3, 3.3)
ax.set_xlabel("first component")
ax.set_ylabel("second component")
ax.set_title("$M$ with rows (2, 1), (1, 2) acting on the plane")
ax.legend(loc="lower right", fontsize=8)
save_figure(fig, "circle_to_ellipse",
            "The unit circle (grey) and its image under the matrix $M$ with rows "
            ...)
```

What Figure 01g.1 shows: the red eigenvector and its image point the same way, the image three times as long; the blue eigenvector's image lies on top of it (eigenvalue 1); the green u is turned to $(2, 1)$; the ellipse's long axis lies along the red, its short axis along the blue direction.

In [4], the characteristic polynomial of the chain matrix.

```
T3 = sp.Matrix([[2, -1, 0], [-1, 2, -1], [0, -1, 2]])
p3 = sp.expand((T3 - lam * sp.eye(3)).det())
say(f"det(T3 - lambda I) = {p3} = {sp.factor(p3)}")
roots3 = sorted(sp.roots(p3, lam).keys(), key=lambda r: float(r))  # small first
say(f"roots: {roots3}")
```

The chain matrix T_3 and its characteristic polynomial, multiplied out and factored: `-lambda**3 + 6*lambda**2 - 10*lambda + 4 = -(lambda - 2)*(lambda**2 - 4*lambda + 2)`, as derived in Section 1.34. `sp.roots` returns a dictionary from the roots to their multiplicities; its keys are sorted by their numerical value (`key=lambda r: float(r)` tells `sorted` to compare the roots as decimal numbers). Output: `roots: [2 - sqrt(2), 2, sqrt(2) + 2]`.

```
check(roots3 == [2 - sp.sqrt(2), 2, 2 + sp.sqrt(2)],
      "the eigenvalues of T3 are 2 - sqrt 2, 2 and 2 + sqrt 2")
check(sp.simplify(sum(roots3) - T3.trace()) == 0 and
      sp.simplify(roots3[0] * roots3[1] * roots3[2] - T3.det()) == 0,
      "sum of the eigenvalues = trace = 6, product = det = 4")
```

The exact roots and the sum and product rules. Two PASS lines.

```
p3_numbers = sp.lambdify(lam, p3, "numpy")  # the polynomial as a numpy function
grid = np.linspace(-0.2, 4.2, 400)
fig, ax = plt.subplots(figsize=(7.2, 4.2))
ax.plot(grid, p3_numbers(grid), color="black", label="$p(\\lambda)$")
ax.axhline(0.0, color="0.5", lw=0.8)
root_values = [float(r) for r in roots3]
ax.plot(root_values, [0.0, 0.0, 0.0], "o", color="tab:red", ms=8,
        label="roots = eigenvalues")
```

The polynomial as a numpy function, drawn for 400 values of λ from -0.2 to 4.2, a grey zero line, and the three roots as red dots on it.

```
for r_text, r_value in zip(["$2 - \\sqrt{2}$", "$2$", "$2 + \\sqrt{2}$"],
                           root_values):
```

```
    ax.text(r_value, 0.6, r_text, ha="center", color="tab:red")
ax.set_xlabel("$\\lambda$")
ax.set_ylabel("$p(\\lambda) = \\det(T_3 - \\lambda I)$")
ax.set_title("The characteristic polynomial of the chain matrix $T_3$")
ax.set_ylim(-3.0, 5.0)
ax.legend(loc="upper right");
save_figure(fig, "characteristic_polynomial",
            "The characteristic polynomial $p(\\lambda) = \\det(T_3 - \\lambda I) "
            ...)
```

Each root labelled above its dot, axis labels, title, range, legend (with the semicolon of Section 1.9). What Figure 01g.2 shows: the cubic curve crosses zero exactly at $2 - \sqrt{2}$, 2 and $2 + \sqrt{2}$, and meets the vertical axis at $p(0) = 4 = \det T_3$.

In [5], the chain matrix of size 10.

```
n = 10
T = 2 * np.eye(n) - np.eye(n, k=1) - np.eye(n, k=-1)  # k=1: just above the diagonal
values_T, vectors_T = np.linalg.eigh(T)  # eigenvalues from the smallest up
k = np.arange(1, n + 1)
formula = 2 - 2 * np.cos(k * np.pi / (n + 1))
say(f"largest difference eigh - formula: {np.max(np.abs(values_T - formula)):.1e}")
check(np.allclose(values_T, formula, rtol=0, atol=1e-12),
      "T_10 has the eigenvalues 2 - 2 cos(k pi/11), k = 1 ... 10")
```

`np.eye(n, k=1)` has ones just above the diagonal and `k=-1` just below, so `T` is T_{10}. Its eigenvalues from numpy are compared with the formula $2 - 2\cos(k\pi/11)$ of Section 1.35 (which is increasing in k, the same order). Output: largest difference eigh - formula: 8.9e-16.

```
j = np.arange(1, n + 1)  # the points of the chain
sines = np.array([np.sin(j * kk * np.pi / (n + 1)) for kk in k]).T  # columns
sines = sines / np.sqrt((sines ** 2).sum(axis=0))  # length 1
overlaps = np.abs((vectors_T * sines).sum(axis=0))  # |cos of the angle|, column-wise
check(np.allclose(overlaps, 1.0, rtol=0, atol=1e-12),
      "every eigenvector of T_10 is a sampled sine wave sin(j k pi/11)")
```

The sampled sine waves $\sin(jk\pi/11)$, one column for each k (`.T` turns the rows into columns), scaled to length 1. For two vectors of length 1 the dot product is the cosine of the angle between them; it is ± 1 exactly when they point along the same line. `(vectors_T * sines).sum(axis=0)` computes the dot product of each numpy eigenvector with its sine wave, column by column; all ten must be ± 1. Two PASS lines.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
fine = np.linspace(0, n + 1, 400)  # the sine curves between the points
for kk, color in zip((1, 2, 3), ("tab:blue", "tab:orange", "tab:green")):
    column = vectors_T[:, kk - 1] * np.sign(vectors_T[0, kk - 1])  # first entry > 0
    scale = column[0] / math.sin(kk * math.pi / (n + 1))
    left.plot(fine, scale * np.sin(fine * kk * math.pi / (n + 1)), color=color, lw=1)
    left.plot(j, column, "o", color=color, label=f"$k = {kk}$")
```

For the three smallest eigenvalues ($k = 1, 2, 3$): the eigenvector, with the sign chosen so that its first entry is positive, is drawn as dots at the points $j = 1, \ldots, 10$; through them the continuous sine curve, scaled to pass through the first dot (`scale`), drawn at 400 points from 0 to 11.

```
left.axhline(0, color="0.5", lw=0.8)
left.set_xlabel("point $j$ of the chain")
left.set_ylabel("entry $v_j$ of the eigenvector")
left.set_title("the first three eigenvectors of $T_{10}$")
left.legend(fontsize=8)
fine_k = np.linspace(0, n + 1, 400)
right.plot(fine_k, 2 - 2 * np.cos(fine_k * np.pi / (n + 1)), color="black", lw=1,
           label="$2 - 2\\cos(k\\pi/11)$")
right.plot(k, values_T, "o", color="tab:red", label="eigenvalues from numpy")
right.set_xlabel("number $k$")
right.set_ylabel("eigenvalue $\\lambda_k$")
right.set_title("the ten eigenvalues of $T_{10}$")
right.legend(fontsize=8, loc="upper left")
fig.tight_layout()
save_figure(fig, "chain_matrix",
            "The $10 \\times 10$ chain matrix $T_{10}$ (2 on the diagonal, $-1$ "
            ...)
```

Labels of the left panel; on the right the curve $2 - 2\cos(k\pi/11)$ for continuous k and the ten numpy eigenvalues as red dots. What Figure 01g.3 shows: the dots of the eigenvectors lie on half a sine wave ($k = 1$), a full wave ($k = 2$) and one and a half waves ($k = 3$), each vanishing at $j = 0$ and $j = 11$; the ten eigenvalues lie on the cosine curve.

In [6], rotations, boosts and conjugate pairs.

```
alpha, phi = sp.symbols("alpha phi", real=True)
R = sp.Matrix([[sp.cos(alpha), -sp.sin(alpha)], [sp.sin(alpha), sp.cos(alpha)]])
p_rot = sp.expand((R - lam * sp.eye(2)).det())
check(sp.simplify(p_rot - (lam ** 2 - 2 * sp.cos(alpha) * lam + 1)) == 0,
      "det(R - lambda I) = lambda^2 - 2 cos(alpha) lambda + 1")
```

The rotation matrix with a symbolic angle, and its characteristic polynomial, which must simplify to $\lambda^2 - 2\cos\alpha\,\lambda + 1$ (Section 1.35).

```
rot_ok = True
for sign_value in (1, -1):  # the eigenvalue e^(+-i alpha), eigenvector (1, -+i)
    eigen = sp.exp(sign_value * sp.I * alpha)
    vector = sp.Matrix([1, -sign_value * sp.I])
    gap = (R * vector - eigen * vector).applyfunc(
        lambda e: sp.simplify(sp.expand_complex(e)))
    rot_ok &= gap == sp.zeros(2, 1)
check(rot_ok, "R(alpha) (1, -+i) = e^(+-i alpha) (1, -+i) for every angle alpha")
```

For both signs: the eigenvalue $e^{\pm i\alpha}$ and the eigenvector $(1, \mp i)$; `gap` is $Rv - \lambda v$ with every entry written as real part plus i times imaginary part (`expand_complex` uses Euler's formula) and simplified; it must be the zero column, for every angle.

```
J = R.subs(alpha, sp.pi / 2)  # the matrix of i
say(f"J = {J.tolist()}, eigenvalues {sorted(J.eigenvals().keys(), key=str)}")
check(set(J.eigenvals().keys()) == {sp.I, -sp.I}, "the eigenvalues of J are +i, -i")
```

`.subs(alpha, sp.pi / 2)` puts $\alpha = \pi/2$ into the matrix, which gives J. `.eigenvals()` returns sympy's exact eigenvalues as a dictionary (eigenvalue: multiplicity); they are printed sorted by their text (`key=str`), so that the order is the same on every computer. Output: `J = [[0, -1], [1, 0]], eigenvalues [-I, I]`.

```
boost = sp.Matrix([[sp.cosh(phi), sp.sinh(phi)], [sp.sinh(phi), sp.cosh(phi)]])
boost_ok = True
for sign_value in (1, -1):  # the eigenvalue e^(+-phi), eigenvector (1, +-1)
    vector = sp.Matrix([1, sign_value])
    gap = (boost * vector - sp.exp(sign_value * phi) * vector).applyfunc(
        lambda e: sp.simplify(e.rewrite(sp.exp)))
    boost_ok &= gap == sp.zeros(2, 1)
check(boost_ok, "a boost has the real eigenvalues e^(+-phi) with the light-like "
      "eigenvectors (1, +-1)")
```

The boost and its two eigenvalue equations, checked as in Notebook 01b, In [12] (Section 1.17).

```
generator = np.random.default_rng(12345)  # random numbers with a fixed seed
pairs_ok = True
for _ in range(200):  # 200 random real 5 x 5 matrices
    found = np.linalg.eigvals(generator.normal(size=(5, 5)))
    pairs_ok &= np.allclose(np.sort_complex(found), np.sort_complex(found.conj()))
check(pairs_ok, "the complex eigenvalues of 200 random real 5 x 5 matrices come "
      "in conjugate pairs")
```

`np.linalg.eigvals` is numpy's eigenvalue program for any square matrix; it returns complex numbers. For each of 200 random real 5×5 matrices the list of eigenvalues, sorted (`np.sort_complex` sorts by real part, then imaginary part), must equal the sorted list of their conjugates: the conjugate-pair theorem of Section 1.35. The cell prints five PASS lines.

In [7], the eigenvalues of rotations and boosts as pictures.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6))
circle_line = np.exp(1j * np.linspace(0, 2 * np.pi, 400))
left.plot(circle_line.real, circle_line.imag, color="0.75", lw=1)
colors = plt.cm.viridis(np.linspace(0.0, 0.9, 6))  # six colours from a colour map
```

The unit circle of the complex plane in light grey, and six colours taken from the colour map viridis (`plt.cm.viridis(x)` gives the colour at the position x between 0 and 1).

```
for step, color in zip(range(1, 7), colors):
    angle = step * np.pi / 6
    eig = np.linalg.eigvals(np.array([[np.cos(angle), -np.sin(angle)],
                                      [np.sin(angle), np.cos(angle)]]))
    left.plot(eig.real, eig.imag, "o", color=color, ms=8,
              label=f"$\\alpha = {30 * step}$ degrees")
```

For the angles $\pi/6, 2\pi/6, \ldots, \pi$ (30 to 180 degrees) the two eigenvalues of the rotation matrix, computed by numpy, are drawn in the complex plane.

```
left.annotate("$+i$ (the matrix $J$)", xy=(0, 1), xytext=(0.25, 1.25), fontsize=8,
              arrowprops={"arrowstyle": "->"})
left.set_aspect("equal")
left.set_xlim(-1.5, 1.5)
left.set_ylim(-1.5, 1.5)
left.set_xlabel("real part")
left.set_ylabel("imaginary part")
left.set_title("rotations: $e^{\\pm i\\alpha}$")
left.legend(fontsize=7, loc="lower left")
```

An arrow with a label points at $+i$, the eigenvalue of J (90 degrees); scales, labels, title and legend.

```
rapidity = np.linspace(0.0, 1.5, 200)
right.plot(rapidity, np.exp(rapidity), color="tab:red", label="$e^{\\varphi}$")
right.plot(rapidity, np.exp(-rapidity), color="tab:blue", label="$e^{-\\varphi}$")
right.plot(rapidity, np.exp(rapidity) * np.exp(-rapidity), "k--", lw=1,
           label="their product = 1")
right.set_xlabel("rapidity $\\varphi$")
right.set_ylabel("eigenvalue")
right.set_title("boosts: real eigenvalues $e^{\\pm\\varphi}$")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "rotation_and_boost",
            "Left: the eigenvalues $e^{\\pm i\\alpha}$ of the real rotation "
            ...)
```

On the right the two eigenvalues of a boost and their product against the rapidity. What Figure 01g.4 shows: on the left each pair of dots lies on the unit circle, mirror-symmetric in the real axis, the pair moving from near 1 (30 degrees) to -1 (180 degrees, where the two coincide); on the right one growing and one decaying exponential whose product stays 1.

In [8], symmetric matrices, on random examples.

```
symmetric_spectra, general_spectra = [], []
largest_imaginary = 0.0
decomposition_error = 0.0
for _ in range(300):
    A = generator.normal(size=(8, 8))  # a random real 8 x 8 matrix
    S = A + A.T  # its symmetric part, doubled
    found = np.linalg.eigvals(S)  # the general program: complex results allowed
    largest_imaginary = max(largest_imaginary, float(np.max(np.abs(found.imag))))
    symmetric_spectra.append(found.real)
    general_spectra.append(np.linalg.eigvals(A))
```

For 300 random 8×8 matrices A: the symmetric matrix $S = A + A^T$, its eigenvalues found by the general program `eigvals`, which would report complex values if there were any, and the largest imaginary part found so far; the real parts are kept for the picture, and so are the eigenvalues of A itself.

```
    w, O = np.linalg.eigh(S)  # eigenvalues w and eigenvectors (columns of O)
    decomposition_error = max(
        decomposition_error,
        float(np.max(np.abs(O.T @ O - np.eye(8)))),  # O^T O = I
        float(np.max(np.abs(O @ np.diag(w) @ O.T - S))))  # O Lambda O^T = S
```

`eigh` gives the eigenvalues `w` and the matrix `O` of eigenvectors; the largest deviation from $O^T O = I$ and from $O\Lambda O^T = S$ (with $\Lambda =$ `np.diag(w)`) is kept: the spectral theorem of Section 1.36.

```
general_all = np.concatenate(general_spectra)
complex_share = float(np.mean(np.abs(general_all.imag) > 1e-9))
say(f"symmetric: largest imaginary part {largest_imaginary:.1e}; "
    f"non-symmetric: share of complex eigenvalues {complex_share:.3f}")
```

`np.concatenate` joins the 300 lists into one of 2400 eigenvalues; `complex_share` is the fraction with an imaginary part larger than 10^{-9}. Output: largest imaginary part 0.0e+00; share of complex eigenvalues 0.677.

```
check(largest_imaginary < 1e-9, "300 random symmetric matrices: every eigenvalue "
      "is real")
check(decomposition_error < 1e-12,
      "S = O Lambda O^T with O^T O = I for all 300 symmetric matrices")
check(complex_share > 0.3, "the non-symmetric matrices have many complex "
      "eigenvalues")
report("share of complex eigenvalues of 300 random 8 x 8 matrices",
       f"{complex_share:.3f}")
```

Three PASS lines and the RESULT line 0.677 (COMPUTED).

In [9], all 2400 eigenvalues of each kind.

```
symmetric_all = np.concatenate(symmetric_spectra)
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4), sharey=True)
left.plot(symmetric_all, np.zeros_like(symmetric_all), "|", color="tab:blue",
          ms=14, alpha=0.3)
left.set_title("symmetric $S = A + A^T$: all on the real axis")
right.plot(general_all.real, general_all.imag, ".", color="tab:red", ms=3,
           alpha=0.5)
right.set_title("non-symmetric $A$: conjugate pairs")
```

On the left the 2400 real eigenvalues of the symmetric matrices as short vertical strokes (`"|"`) on the real axis (imaginary part 0, `np.zeros_like`); on the right the 2400 eigenvalues of the non-symmetric matrices as small red dots in the complex plane.

```
for ax in (left, right):
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("real part")
left.set_ylabel("imaginary part")
fig.tight_layout()
```

```
save_figure(fig, "random_spectra",
            "The eigenvalues of 300 random real $8 \\times 8$ matrices in the "
            ...)
```

What Figure 01g.5 shows: on the left everything lies on the real axis; on the right a cloud of dots that is mirror-symmetric in the real axis, with a band of real eigenvalues on it.

In [10], the signature of eta and Sylvester's law.

```
algebra = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
names = algebra["coordinates"]  # ["x0", ..., "x7"]
eta = algebra["eta"]  # the diagonal of the frame metric
eta_matrix = np.diag(np.array(eta, dtype=float))
eta_values = np.linalg.eigvalsh(eta_matrix)  # eigenvalues from the smallest up
signature = (int((eta_values > 0).sum()), int((eta_values < 0).sum()))
say(f"eigenvalues of eta: {eta_values.tolist()}; signature {signature}")
```

η from the record, as an 8×8 matrix; `eigvalsh` returns only the eigenvalues of a symmetric matrix. The signature is the pair of the counts of positive and of negative eigenvalues. Output: eight eigenvalues, four -1.0 and four 1.0, signature (4, 4).

```
algebra_report = json.loads(repository_file(
    "Revision/algebra/reports/python-algebra.json").read_text(encoding="utf-8"))
verdicts = {c["name"]: c["verdict"] for c in algebra_report["checks"]}
check(signature == (4, 4) and verdicts["coordinate_map"] == "pass",
      "eta has 4 positive and 4 negative eigenvalues: the signature (4,4)",
      record="Revision/algebra/reports/python-algebra.json, check coordinate_map")
```

The verdicts of the record's report, and the check of the signature, which reproduces the record's check `coordinate_map`.

```
sylvester_counts = set()
smallest_size, largest_size = np.inf, 0.0
transformed_spectra = []
for trial in range(2000):
    P = generator.normal(size=(8, 8))  # a random matrix (invertible: det != 0)
    changed = np.linalg.eigvalsh(P.T @ eta_matrix @ P)
    sylvester_counts.add((int((changed > 0).sum()), int((changed < 0).sum())))
    smallest_size = min(smallest_size, float(np.min(np.abs(changed))))
    largest_size = max(largest_size, float(np.max(np.abs(changed))))
    if trial < 40:
        transformed_spectra.append(changed)
```

For 2000 random matrices P (a random matrix has a determinant that is not 0, except in cases of probability 0): the eigenvalues of $P^T \eta P$, the signature found (collected in a **set**, which keeps each different value once), and the smallest and largest size of an eigenvalue seen (`np.inf` is infinity, a start value larger than any number). The spectra of the first 40 are kept for the picture.

```
say(f"signatures of P^T eta P found: {sorted(sylvester_counts)}; eigenvalue "
    f"sizes from {smallest_size:.1e} to {largest_size:.1f}")
# Rounding changes an eigenvalue by about 2.2e-16 times the largest size; the
# signs are reliable if the smallest size is far above that.
reliable = smallest_size > 1000 * 2.2e-16 * largest_size
check(sylvester_counts == {(4, 4)} and reliable, "Sylvester: P^T eta P has the "
      "signature (4,4) for all 2000 random P, although its eigenvalues change")
```

Output: signatures found [(4, 4)]; eigenvalue sizes from 1.4e-08 to 37.0. A computed eigenvalue can be off by about machine epsilon (2.2×10^{-16}, Section 1.4) times the largest size; the signs can be trusted if the smallest size is more than 1000 times that, here 1.4×10^{-8} against 8.1×10^{-12}. The check: only the signature (4,4) occurred, and the signs are reliable. Two PASS lines.

In [11], the signature of the author's metric.

```
a4 = sp.Symbol("a4", real=True)  # the value of a4(x4) at one time
b4 = sp.Symbol("b4", positive=True)  # the value of b4(x4) at one time
z = sp.Symbol("z", positive=True)  # z = 6 H x0, between 0 and pi/2
curvature = json.loads(repository_file("Revision/gkd_lovelock/results/curvature.json")
                       .read_text(encoding="utf-8"))


def from_record(text):
    """The record's Mathematica text as a sympy expression."""
    text = text.replace("a4[x4]", "a4").replace("b4[x4]", "b4")
    text = text.replace("Sin[6*H*x0]", "sin(z)")
    text = text.replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
    return sp.sympify(text, locals={"a4": a4, "b4": b4, "z": z, "E": sp.E})
```

The symbols, the record file and the translation function, exactly as in Notebook 01a, In [17] (Section 1.25).

```
g_record = [from_record(text) for text in curvature["metricDiagonal"]]
# a4, b4, z -> the 8 numbers on the diagonal
metric_numbers = sp.lambdify((a4, b4, z), g_record, "numpy")
signs_ok, product_error, points = True, 0.0, 0
for b4_value in (0.5, 1.0, 2.0):
    for a4_value in np.linspace(-3.0, 3.0, 61):
        for z_value in np.linspace(0.05, np.pi / 2 - 0.05, 50):
            g_values = np.linalg.eigvalsh(np.diag(np.array(
                metric_numbers(a4_value, b4_value, z_value), dtype=float)))
            signs_ok &= ((int((g_values > 0).sum()), int((g_values < 0).sum()))
                         == (4, 4))
            product_error = max(product_error, abs(
                np.prod(g_values) / b4_value ** 3 - np.cos(z_value) ** 2))
            points += 1
```

The record's metric as a numpy function of a_4, b_4 and z; then, for $b_4 = 0.5$, 1 and 2, at 61 values of a_4 from -3 to 3 and 50 values of z inside $(0, \pi/2)$ (9150 points in all, counted in `points`), the eight eigenvalues of the 8×8 metric (computed by numpy, although for a diagonal matrix they are its entries), their signs and the distance of their product (`np.prod`) divided by b_4^3 from $\cos^2 z$.

```
check(signs_ok and points == 9150, "the author's metric has the signature (4,4) "
      "at all 9150 grid points (a4, z, b4)")
lovelock_report = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json")
    .read_text(encoding="utf-8"))
root_verdict = {c["name"]: c["verdict"]
                for c in lovelock_report["checks"]}["sqrt_abs_det_g"]
check(product_error < 1e-12 and root_verdict == "PASS",
      "the product of the eight eigenvalues is det g = b4^3 cos(z)^2 at every grid "
      "point",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "sqrt_abs_det_g")
```

Two checks: the signature at all 9150 points, and the product of the eigenvalues equal to $\det g = b_4^3 \cos^2 z$ to 10^{-12} (relative to b_4^3), together with the record's verdict PASS of the check `sqrt_abs_det_g`. Two PASS lines.

In [12], the picture of the signatures.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.5, 4.6))
for column, spectrum in enumerate(transformed_spectra):
    colors_column = ["tab:red" if value > 0 else "tab:blue" for value in spectrum]
    left.scatter(np.full(8, column + 1), spectrum, c=colors_column, s=12)
left.axhline(0.0, color="black", lw=0.8)
left.set_yscale("symlog", linthresh=0.001)  # linear between -0.001 and 0.001
left.set_xlabel("random matrix $P$ (number)")
left.set_ylabel("eigenvalues of $P^T \\eta P$")
left.set_title("always 4 positive (red) and 4 negative (blue)")
```

On the left, for each of the first 40 matrices P, its eight eigenvalues as a column of dots at the horizontal position $1, \ldots, 40$ (`np.full(8, column + 1)` is a list of eight equal positions; `scatter` draws dots with one colour each, red for positive and blue for negative). The vertical axis is **symmetric logarithmic** (`"symlog"`): linear between -0.001 and 0.001 and logarithmic outside, so that large and small values of both signs fit.

```
a4_line = np.linspace(-3.0, 3.0, 301)
curves = np.array([metric_numbers(value, 1.0, 0.9) for value in a4_line],
                  dtype=float)  # at b4 = 1
styles = [("tab:green", "hidden $x_0$", 0), ("tab:red", "space $x_1, x_2, x_3$", 1),
          ("tab:gray", "time $x_4$", 4), ("tab:blue", "extra times $x_5, x_6, x_7$", 5)]
for color, label, index in styles:
    right.plot(a4_line, curves[:, index], color=color, lw=2, label=label)
```

On the right the eight diagonal entries of the metric (its eigenvalues) at $z = 0.9$ and $b_4 = 1$ for 301 values of a_4, as a table with one column per entry; one curve is drawn for each kind of direction (column 0 for the hidden direction x_0, 1 for space, 4 for the time, 5 for the extra times: the column of x_k is k).

```
right.axhline(0.0, color="black", lw=0.8)
right.set_yscale("symlog", linthresh=0.1)
```

```
right.set_xlabel("$a_4$")
right.set_ylabel("eigenvalues of $g$ at $z = 0.9$")
right.set_title("the signs never change: signature (4,4)")
right.legend(fontsize=8, loc="upper left")
fig.tight_layout()
```

A symmetric logarithmic axis linear between -0.1 and 0.1, labels and legend.

```
# how many of the 320 drawn eigenvalues are smaller than 0.001 in size
tiny = sum(int((np.abs(spectrum) < 0.001).sum()) for spectrum in transformed_spectra)
tiny_text = "the one eigenvalue" if tiny == 1 else f"the {tiny} eigenvalues"
tiny_verb = "sits" if tiny == 1 else "sit"  # singular or plural
save_figure(fig, "signature",
            "Left: the eight eigenvalues of $P^T \\eta P$ for 40 random matrices "
            ...
            f"{tiny_text} smaller than 0.001 in size {tiny_verb} on the zero "
            "line, but none is zero. "
            ...)
```

The caption is written from the data: it counts how many of the $40 \cdot 8 = 320$ drawn eigenvalues are smaller than 0.001 in size (they appear on the zero line of the left panel) and chooses the singular or the plural wording. `tiny == 1` is True when exactly one is that small; `x if condition else y` is a **conditional expression**, which gives `x` when the condition is True and `y` otherwise. So `tiny_text` is "the one eigenvalue" or, for example, "the 3 eigenvalues", and `tiny_verb` is "sits" or "sit". The first `...` stands for three ordinary text lines of the caption; the f-string line inserts `tiny_text` and `tiny_verb` where their names stand in braces. In this run two eigenvalues are that small, and the caption reads "the 2 eigenvalues smaller than 0.001 in size sit on the zero line, but none is zero.". The last `...)` stands for the rest of the caption, about the right panel, which has no insertions. What Figure 01g.6 shows: on the left every column has four red and four blue dots although the values change from column to column; on the right the blue extra-time curve (the three equal extra-time eigenvalues) rises towards 0 as a_4 grows, the deflation, without ever crossing it, while the red space curve grows and the grey and green curves stay constant.

In [13], the eigenvalues of the gamma matrices.

```
gamma = [np.array(matrix, dtype=int) for matrix in algebra["gamma"]]  # gamma^a
polynomials_ok, traces_ok, numeric_ok = True, True, True
for name, eta_aa, g_a in zip(names, eta, gamma):
    poly = sp.Matrix(g_a.tolist()).charpoly(lam).as_expr()  # det(lambda I - g_a)
    polynomials_ok &= sp.expand(poly - (lam ** 2 - eta_aa) ** 8) == 0
    traces_ok &= int(np.trace(g_a)) == 0
    found = np.linalg.eigvals(g_a.astype(float))
```

For each gamma matrix: its exact characteristic polynomial (`charpoly(lam)` gives $\det(\lambda I - \gamma^a)$ as a polynomial object, `.as_expr()` as an ordinary sympy expression), compared with $(\lambda^2 - \eta_{aa})^8$; its trace, which must be 0; and its eigenvalues from numpy.

```
    if eta_aa == 1:  # space-like: +1 and -1, eight times each
        numeric_ok &= bool(np.allclose(np.sort(found.real), [-1] * 8 + [1] * 8)
```

```
                        and np.allclose(found.imag, 0))
    else:  # time-like: +i and -i, eight times each
        numeric_ok &= bool(np.allclose(found.real, 0)
                        and np.allclose(np.sort(found.imag), [-1] * 8 + [1] * 8))
    say(f"gamma^({name}): eta {eta_aa:+d}, trace {int(np.trace(g_a))}, "
        f"characteristic polynomial {sp.factor(poly)}")
```

For a space-like direction the sorted real parts must be eight -1 followed by eight $+1$ (`[-1] * 8 + [1] * 8`) and the imaginary parts 0; for a time-like direction the real parts 0 and the sorted imaginary parts eight -1 and eight $+1$. One line per matrix with its factored polynomial: `(lambda - 1)**8*(lambda + 1)**8` for x0, x1, x2, x3 and `(lambda**2 + 1)**8` for x4 to x7.

```
check(traces_ok and polynomials_ok,
      "every gamma^a has trace 0 and the characteristic polynomial "
      "(lambda^2 - eta_aa)^8", record="Revision/algebra/reports/python-algebra.json, "
      "check clifford_relation (the relations with equal indices)")
check(numeric_ok and verdicts["symmetry_pattern"] == "pass",
      "numpy: +1 and -1 (x0, x1, x2, x3: symmetric matrices) or +i and -i "
      "(x4 ... x7: antisymmetric matrices), 8 times each",
      record="Revision/algebra/reports/python-algebra.json, check symmetry_pattern")
```

Two PASS lines, each with the record check it rests on: the eigenvalues follow from the relations with equal indices of `clifford_relation`, and the symmetric (real eigenvalues) or antisymmetric (imaginary eigenvalues) pattern is the record's `symmetry_pattern`.

In [14], the gamma matrix of a vector.

```
def gamma_of(vector):
    """gamma(v) = v_a gamma^a with v_a = eta_ab v^b (whole numbers)."""
    lowered = [eta[a] * int(vector[a]) for a in range(8)]
    return sum(lowered[a] * gamma[a] for a in range(8))


def squared_length(vector):
    """Q(v) = eta_ab v^a v^b."""
    return sum(eta[a] * int(vector[a]) ** 2 for a in range(8))
```

`gamma_of` lowers the index of the vector and forms $v_a\gamma^a$, a 16×16 matrix of whole numbers; `squared_length` is $Q(v)$, written out for the diagonal η.

```
unit = np.eye(8, dtype=int)
examples = [("e(x1)", unit[1]), ("e(x5)", unit[5]), ("e(x1) + e(x5)", unit[1] + unit[5]),
            ("(1, 2, ..., 8)", np.arange(1, 9))]
example_spectra = {}
squares_ok, nilpotent_ok = True, True
```

The four example vectors of Section 1.27 with their names; a dictionary for their spectra.

```
for label, vector in examples:
    matrix = gamma_of(vector)
```

```
    q = squared_length(vector)
    squares_ok &= bool((matrix @ matrix == q * np.eye(16, dtype=int)).all())
    if q == 0:  # light-like: exact answer from sympy
        poly = sp.Matrix(matrix.tolist()).charpoly(lam).as_expr()
        nilpotent_ok &= poly == lam ** 16 and bool(np.count_nonzero(matrix) > 0)
        example_spectra[label] = np.zeros(16, dtype=complex)
```

For each example: $\gamma(v)$, $Q(v)$, and the exact check $\gamma(v)^2 = Q(v)I$. For the light-like vector the characteristic polynomial is computed exactly; it must be λ^{16}, and the matrix must have nonzero entries: nilpotent, not zero. Its spectrum, sixteen zeros, is stored for the picture. (numpy is not reliable for a nilpotent matrix, whose eigenvectors do not fill the space; the exact computation is.)

```
    else:
        example_spectra[label] = np.linalg.eigvals(matrix.astype(float))
        target = np.sqrt(complex(q))  # sqrt(Q), imaginary for Q < 0
        # count the eigenvalues within 1e-9 of +sqrt(Q) and of -sqrt(Q)
        near_plus = int((np.abs(example_spectra[label] - target) < 1e-9).sum())
        near_minus = int((np.abs(example_spectra[label] + target) < 1e-9).sum())
        squares_ok &= near_plus == 8 and near_minus == 8
    say(f"v = {label}: Q(v) = {q}, gamma(v)^2 = Q(v) I")
```

Otherwise numpy's eigenvalues are compared with $\pm\sqrt{Q}$ (`np.sqrt(complex(q))` is imaginary for a negative Q): exactly eight must lie within 10^{-9} of each. One line per example: Q(v) = 1, -1, 0 and -144.

```
for _ in range(100):  # random vectors with whole-number components -5 ... 5
    vector = generator.integers(-5, 6, size=8)
    matrix = gamma_of(vector)
    squares_ok &= bool((matrix @ matrix
                        == squared_length(vector) * np.eye(16, dtype=int)).all())
check(squares_ok, "gamma(v)^2 = Q(v) I exactly and the eigenvalues are +-sqrt(Q(v)) "
      "(4 examples and 100 random vectors)")
check(nilpotent_ok, "a light-like v gives a nilpotent gamma(v): not zero, square "
      "zero, characteristic polynomial lambda^16")
```

The square rule for 100 random whole-number vectors, exactly; then the two checks. Two PASS lines.

In [15], the eigenvalues as pictures.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.8))
for name, eta_aa, g_a in zip(names, eta, gamma):
    found = np.linalg.eigvals(g_a.astype(float))
    color = "tab:red" if eta_aa == 1 else "tab:blue"
    left.plot(found.real, found.imag, "o", color=color, ms=10, alpha=0.3)
```

On the left the 16 eigenvalues of each gamma matrix in the complex plane, red for space-like and blue for time-like directions, see-through (`alpha=0.3`), so that the many dots on top of each other show as a darker colour.

```
left.text(1.0, 0.18, "$+1$: 8 times\n($x_0, x_1, x_2, x_3$)", ha="center",
          fontsize=8)
left.text(-1.0, 0.18, "$-1$: 8 times", ha="center", fontsize=8)
left.text(0.08, 1.0, "$+i$: 8 times ($x_4 \\dots x_7$)", fontsize=8)
left.text(0.08, -1.05, "$-i$: 8 times", fontsize=8)
left.set_xlim(-1.6, 1.9)
left.set_ylim(-1.5, 1.5)
left.set_title("the eight gamma matrices")
```

Four labels at the four points ± 1, $\pm i$, the ranges and the title.

```
markers = {"e(x1)": ("s", "tab:red"), "e(x5)": ("D", "tab:blue"),
           "e(x1) + e(x5)": ("*", "black"), "(1, 2, ..., 8)": ("o", "tab:purple")}
for label, spectrum in example_spectra.items():
    shape, color = markers[label]
    right.plot(spectrum.real, spectrum.imag, shape, color=color, ms=10,
               label=f"$v$ = {label}, $Q = {squared_length(dict(examples)[label])}$")
right.set_xlim(-3.0, 3.0)
right.set_ylim(-8.0, 8.0)
right.set_title("$\\gamma(v) = v_a \\gamma^a$: eigenvalues $\\pm\\sqrt{Q(v)}$")
right.legend(fontsize=8, loc="lower right")
```

On the right the spectra of the four matrices $\gamma(v)$, each with its own marker shape (square, diamond, star, circle); `dict(examples)[label]` looks up the vector of a name, to print its Q in the legend.

```
for ax in (left, right):
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.axvline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("real part")
    ax.set_ylabel("imaginary part")
fig.tight_layout()
save_figure(fig, "gamma_eigenvalues",
            "Eigenvalues in the complex plane (horizontal axis real part, vertical "
            ...)
```

Axes and labels on both panels. What Figure 01g.7 shows: on the left the dots sit only at ± 1 and $\pm i$; on the right ± 1, $\pm i$, a single point at 0 for the light-like vector, and $\pm i\sqrt{48} \approx \pm 6.93i$.

In [16], the Hermitian matrix B of the record.

```
B = sp.Matrix(algebra["B"]["re"]) + sp.I * sp.Matrix(algebra["B"]["im"])
C = sp.Matrix(algebra["C"])
check(B == -sp.I * C * sp.Matrix(algebra["gamma"][4]),
      "the record's B equals -i C gamma^(x4) computed from its C and gamma^(x4)")
```

The record stores B as its real part `re` and its imaginary part `im`; the exact sympy matrix is built from them. C is read as well, and the check confirms $B = -iC\gamma^{(x_4)}$ exactly ($\gamma^{(x_4)}$ is entry 4 of the list, which holds the matrices in the author's order $x_0, \dots, x_7$).

```
poly_B = sp.factor(B.charpoly(lam).as_expr())
real_part_zero = sp.Matrix(algebra["B"]["re"]).is_zero_matrix  # True or False
say(f"B: real part zero {real_part_zero}, trace {B.trace()}, characteristic "
    f"polynomial {poly_B}")
```

The factored characteristic polynomial of B, and whether its real part is the zero matrix. Output:

```
B: real part zero True, trace 0, characteristic polynomial (lambda - 1)**8*(lambda +
    1)**8
```

```
check(B.H == B and B * B == sp.eye(16) and B.trace() == 0
      and sp.expand(poly_B - (lam - 1) ** 8 * (lam + 1) ** 8) == 0,
      "B is Hermitian, B^2 = I, tr B = 0, characteristic polynomial "
      "(lambda - 1)^8 (lambda + 1)^8: the signature (8,8)",
      record="Revision/algebra/reports/python-algebra.json, check "
             "B_hermitian_involution_signature")
```

`B.H` is sympy's conjugate transpose $B^\dagger$. The four properties of Section 1.37, exactly; this reproduces the record's check `B_hermitian_involution_signature`.

```
B_numbers = np.array(B.tolist(), dtype=complex)
values_B, vectors_B = np.linalg.eigh(B_numbers)
unitary_error = float(np.max(np.abs(vectors_B.conj().T @ vectors_B - np.eye(16))))
say(f"numpy eigh: eigenvalues {values_B.round(12).tolist()}")
```

B as a complex numpy array; `eigh` works for Hermitian matrices too. `vectors_B.conj().T` is the conjugate transpose of the eigenvector matrix; its product with the eigenvector matrix must be I (eigenvectors perpendicular in the sense $u^\dagger w = 0$, of length 1). Output: eight -1.0 and eight 1.0.

```
theory = json.loads(repository_file(
    "Revision/theory/reports/python-field-theory.json").read_text(encoding="utf-8"))
theory_verdict = {c["name"]: c["verdict"] for c in theory["checks"]}["B_properties"]
krein_entry = theory["formulas"]["quantisation"]["krein"]  # a sentence of the record
say(f"the record's quantisation entry: {krein_entry}")
```

A second record, the theory report: the verdict of its check `B_properties` and its entry `formulas`, `quantisation`, `krein`, a sentence that is printed: B Hermitian, B^2 = 1, signature (8,8): indefinite (Krein) state space.

```
check(np.allclose(values_B, [-1.0] * 8 + [1.0] * 8, rtol=0, atol=1e-12)
      and unitary_error < 1e-12 and theory_verdict == "pass",
      "numpy: eight eigenvalues -1 and eight +1, perpendicular eigenvectors",
      record="Revision/theory/reports/python-field-theory.json, check B_properties")
```

The numerical eigenvalues, the perpendicular eigenvectors and the record's verdict; this reproduces the check `B_properties`. The cell prints three PASS lines.

In [17], the picture of B.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.6),
                                  gridspec_kw={"width_ratios": [1.1, 1]})
image = left.imshow(B_numbers.imag, cmap="RdBu_r", vmin=-1, vmax=1)
left.grid(False)
ticks = range(0, 16, 3)
left.set_xticks(ticks, [str(t + 1) for t in ticks])
left.set_yticks(ticks, [str(t + 1) for t in ticks])
left.set_xlabel("column")
left.set_ylabel("row")
left.set_title("imaginary part of $B$ (the real part is 0)")
fig.colorbar(image, ax=left, shrink=0.8, ticks=[-1, 0, 1])
```

The imaginary part of B as a heat map, with labels at every third row and column (`range(0, 16, 3)` is 0, 3, 6, 9, 12, 15, labelled 1, 4, 7, 10, 13, 16), and a colour bar.

```
right.bar(np.arange(1, 17), values_B,
          color=["tab:blue" if value < 0 else "tab:red" for value in values_B])
right.axhline(0.0, color="black", lw=0.8)
right.set_xticks(range(1, 17, 3))
right.set_ylim(-1.4, 1.4)
right.set_xlabel("eigenvalue number (from the smallest up)")
right.set_ylabel("eigenvalue of $B$")
right.set_title("eight $-1$ and eight $+1$: signature (8,8)")
fig.tight_layout()
save_figure(fig, "hermitian_b",
            "Left: heat map of the imaginary part of the Hermitian matrix "
            ...)
```

On the right the 16 eigenvalues as bars, blue for negative and red for positive. What Figure 01g.8 shows: on the left a pattern of red and blue squares whose mirror image in the diagonal has the opposite colours (antisymmetric imaginary part); on the right eight bars down and eight up.

In [18], power iteration.

```
def power_iteration(matrix, start, steps):
    """Multiply by matrix and rescale to length 1, steps times; return the
    distances to the eigenvector of the largest eigenvalue after each step."""
    values_m, vectors_m = np.linalg.eigh(matrix)
    top = vectors_m[:, -1]  # the eigenvector of the largest eigenvalue
    x = np.array(start, dtype=float)
    distances = []
```

The function first finds the true answer with `eigh` (the last column belongs to the largest eigenvalue), to measure the error of the iteration; `x` is the starting vector.

```
    for _ in range(steps):
        x = matrix @ x
        x = x / np.sqrt(x @ x)  # rescale to length 1
        aligned = top if x @ top > 0 else -top  # the sign of an eigenvector is free
        distances.append(float(np.sqrt(((x - aligned) ** 2).sum())))
    return np.array(distances), abs(values_m[-2] / values_m[-1])
```

Each step multiplies by the matrix and divides by the length (`x @ x` is the squared length). The true eigenvector is taken with the sign that points the same way as `x`, and the distance between the two is recorded. The function returns the distances and the factor $|\lambda_2/\lambda_1|$ of Section 1.37 (the second-largest eigenvalue divided by the largest).

```
runs = {"M (2 x 2)": power_iteration(M_numbers, [1, 0], 40),
        "T3 (3 x 3)": power_iteration(np.array(T3.tolist(), dtype=float),
                                      [1, 0, 0], 40)}
factors_ok = True
for label, (distances, ratio) in runs.items():
    measured = distances[11:21] / distances[10:20]  # steps 11 to 20
    say(f"{label}: |lambda2/lambda1| = {ratio:.6f}, measured factors "
        f"{measured.min():.6f} to {measured.max():.6f}")
    factors_ok &= bool(np.all(np.abs(measured - ratio) < 1e-3))
check(factors_ok, "power iteration: each step shrinks the error by |lambda2/lambda1|"
      " (1/3 for M, 2 - sqrt 2 for T3)")
```

40 steps for M from $(1, 0)$ and for T_3 from $(1, 0, 0)$. The measured factors are the ratios of consecutive distances for the steps 11 to 20 (`distances[11:21] / distances[10:20]` divides each distance by the one before); they must agree with $|\lambda_2/\lambda_1|$ to 10^{-3}. Output: M (2 x 2): |lambda2/lambda1| = 0.333333, measured factors 0.333333 to 0.333333, and T3 (3 x 3): 0.585786, measured 0.585786 to 0.585789, and the PASS line.

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
steps_axis = np.arange(1, 41)
for (label, (distances, ratio)), color in zip(runs.items(), ("tab:red", "tab:blue")):
    # a distance 0 (exact agreement) is drawn at 1e-17, the bottom of the axis
    ax.semilogy(steps_axis, np.maximum(distances, 1e-17), "o", color=color, ms=4,
                label=label)
    ax.semilogy(steps_axis, distances[0] * ratio ** (steps_axis - 1), "-",
                color=color, lw=1, label=f"factor {ratio:.3f} per step")
```

For both runs: the 40 distances as dots on a logarithmic axis (a distance that is exactly 0 is drawn at 10^{-17}), and the straight line $|\lambda_2/\lambda_1|^{k-1}$ times the first distance.

```
ax.set_ylim(1e-17, 2.0)
ax.set_xlabel("step $k$")
ax.set_ylabel("distance to the eigenvector")
ax.set_title("Power iteration")
ax.legend(fontsize=8);
save_figure(fig, "power_iteration",
            "Power iteration: the distance between the rescaled vector "
            ...)
```

What Figure 01g.9 shows: the red dots (M) fall fast, by the factor 1/3 per step, along their straight line until, after about 33 steps, they reach the rounding level of floating-point numbers; from there on the distance jumps between about 10^{-16} and exactly 0 (drawn at the bottom). The blue dots (T_3) fall more slowly, by 0.586 per step, along their line, down to about 10^{-9} after 40 steps.

In [19], the last check.

```
figure_names = [f"01g_{k}_{name}.png" for k, name in enumerate(
    ["circle_to_ellipse", "characteristic_polynomial", "chain_matrix",
     "rotation_and_boost", "random_spectra", "signature", "gamma_eigenvalues",
     "hermitian_b", "power_iteration"], start=1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 9 figure files of this notebook exist")
all_checks_passed()
```

The nine file names are built from their numbers and names (`enumerate(..., start=1)` numbers the list from 1). The last line is ALL 30 CHECKS PASSED (notebook 01g). The 30 checks are: 4 in In [2], 1 in In [3], 2 in In [4], 2 in In [5], 5 in In [6], 3 in In [8], 2 in In [10], 2 in In [11], 2 in In [13], 2 in In [14], 3 in In [16], 1 in In [18] and 1 in In [19].

1.42 The generalized Kronecker delta: the author's definition and its four cases

Why it matters. In eight dimensions the field equations of gravity that the author's theory uses are the **Lovelock equations** (Chapters 11 and 12). Every one of their terms carries a weight, a number $+1$, -1 or 0, given by one combinatorial tool: the **generalized Kronecker delta**. The Revision record computes these weights with a fast program called GKD; this section defines the tool exactly as the author did and proves the rule that GKD uses.

Words. A **label** is a name of a coordinate, here one of $x_0, \dots, x_7$; the computer often numbers them $0, \dots, 7$. Only the question "are two labels equal?" matters below, so the choice of names changes no result. An **index list** is an ordered list of labels, such as (x_1, x_3, x_3); its **length** is the number of its entries (here 3). The ordinary **Kronecker delta** of two labels, $\delta(a, b)$, is 1 if they are equal and 0 otherwise (Section 1.26 wrote it $\delta^a{}_b$).

The author's definition. The author defined the generalized delta in their Mathematica notebook, in the input cell stored in the file as In[87] (their In[54]); the Revision record quotes the cell verbatim in `Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md` and lists it in `Revision/gkd_lovelock/results/notebook-input-cells.txt`. In Mathematica's plain-text spelling (`\[Delta]` is the Greek letter δ):

```
k\[Delta][lower_, upper_] /; Length[lower] == Length[upper] :=
    Det[Outer[delta, lower, upper]]
```

In words: for a lower list $(l_1, \dots, l_p)$ and an upper list $(u_1, \dots, u_p)$ of the same length p (the condition after `/;`; for lists of different lengths the definition does not apply and Mathematica leaves the expression unevaluated), build the $p \times p$ matrix whose entry in row i and column j is $\delta(l_i, u_j)$ (`Outer[delta, lower, upper]`), and take its determinant (`Det`). The book writes

$$\delta^{u_1 \dots u_p}_{l_1 \dots l_p} = \det\left[\delta(l_i, u_j)\right]_{i,j=1,\dots,p}.$$

For one index this is the ordinary Kronecker delta: the 1×1 matrix $(\delta(l_1, u_1))$ has this number as its determinant. For two indices the 2×2 determinant gives (PROVED)

$$\delta^{u_1 u_2}_{l_1 l_2} = \det \begin{pmatrix} \delta(l_1, u_1) & \delta(l_1, u_2) \\ \delta(l_2, u_1) & \delta(l_2, u_2) \end{pmatrix} = \delta(l_1, u_1)\,\delta(l_2, u_2) - \delta(l_1, u_2)\,\delta(l_2, u_1)$$

(the definition with $p = 2$; the formula $ad - bc$).

Theorem: the four cases (PROVED). Let $M_{ij} = \delta(l_i, u_j)$.

- (a) If two lower labels are equal, $l_i = l_k$ with $i \neq k$, then the rows i and k of M are equal, so $\det M = 0$ (rule 4 of Section 1.20).
- (b) If two upper labels are equal, two columns of M are equal; the transpose M^T then has two equal rows, so $\det M = \det M^T = 0$ (rules 2 and 4).
- (c) If some lower label l_i is not among the upper labels, row i of M consists of zeros; every term of the Leibniz formula contains one entry of that row, so $\det M = 0$.
- (d) Otherwise the lower labels are p different labels, all among the upper labels, which are p different labels as well; so both lists hold the same p labels, and the lower list is a re-ordering of the upper one. Each l_i equals exactly one upper label $u_{\sigma(i)}$, and σ is a permutation of $1, \ldots, p$. Row i of M has its only 1 in column $\sigma(i)$: M is the permutation matrix of σ, and $\det M = \text{sign}(\sigma)$ (Section 1.20, rule (b) of the three special matrices).

So the generalized delta is 0 when a label is repeated or missing, and otherwise the sign of the permutation that turns the upper list into the lower one. The Revision program GKD (the Rust file `Revision/gkd_lovelock/code/src/gkd.rs`) computes it in exactly this way, without building a matrix: it costs about p^2 comparisons (to count the inversions of σ) instead of the $p!$ terms of the Leibniz formula. The file states this proof in its opening comment, and its tests compare GKD with a literal determinant.

Examples (PROVED). For the lists of length 3 over the labels x_1, x_2, x_3:

lower list	upper list	case	value
(x_1, x_2, x_3)	(x_1, x_2, x_3)	(d), $\sigma = (1, 2, 3)$, no inversion	$+1$
(x_1, x_2, x_3)	(x_2, x_3, x_1)	(d), $\sigma = (3, 1, 2)$, two inversions	$+1$
(x_1, x_2, x_3)	(x_1, x_3, x_2)	(d), $\sigma = (1, 3, 2)$, one inversion	-1
(x_1, x_1, x_3)	(x_1, x_2, x_3)	(a), a repeated lower label	0
(x_1, x_2, x_4)	(x_1, x_2, x_3)	(c), x_4 missing above	0

(In the second row, $l_1 = x_1$ is the third upper label, so $\sigma(1) = 3$; $l_2 = x_2$ is the first, $\sigma(2) = 1$; $l_3 = x_3$ is the second, $\sigma(3) = 2$; the inversions of $(3, 1, 2)$ are the pairs $(3, 1)$ and $(3, 2)$.) The record lists five examples with the labels 1 to 4 in `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `gkd_examples`: $\delta^{12}_{12} = 1$, $\delta^{21}_{12} = -1$, $\delta^{11}_{11} = 0$, $\delta^{231}_{123} = 1$, $\delta^{124}_{123} = 0$ (written there as `kdelta[{1,2},{2,1}] = -1` and so on, lower list first); two more with the labels 0 to 2, and the refusal of lists of different lengths, in `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, check `definition_unequal_lengths_stay_unevaluated`; and the two values that the Rust program prints for itself, $+1$ for the lists $(0, 1, 2)$, $(1, 2, 0)$ and -1 for $(0, 1)$, $(1, 0)$, in `Revision/gkd_lovelock/results/lovelock-report.json`, field `gkdSelfCheck`. Notebook 01c, In [6], reproduces all of them with both the literal determinant and the fast rule (Figure 01c.1 draws the five matrices of the table).

1.43 Counting the values; nine indices in eight dimensions

Two indices (PROVED). With two indices over the eight labels there are $8^2 = 64$ lower lists and 64 upper lists, so $64 \cdot 64 = 4096$ values. A value is not 0 only in case (d): the upper list holds two different labels (there are $8 \cdot 7 = 56$ such lists: 8 choices for the first label, 7 for the second), and the lower list is the same list (value $+1$) or its reverse (value -1). So there are 56 values $+1$, 56 values -1 and $4096 - 112 = 3984$ zeros (Figure 01c.2).

The number of nonzero values (PROVED). In general, with n labels and lists of length $p \le n$: there are $n(n-1)\cdots(n-p+1) = n!/(n-p)!$ upper lists of p different labels (a **falling factorial**: n choices for the first label, $n-1$ for the second, and so on), and $p!$ re-orderings of each, so

$$N_{\neq 0}(n,p) = \frac{n!}{(n-p)!}\, p! \qquad (p \le n), \qquad N_{\neq 0}(n,p) = 0 \qquad (p > n).$$

For $p \ge 2$ half of the re-orderings are even (Section 1.19), so $+1$ and -1 occur equally often; for $p = 1$ every nonzero value is $+1$. For $n = 8$:

length p	**all pairs** 8^{2p}	**value** $+1$	**value** -1	**value 0**
1	64	8	0	56
2	4096	56	56	3984
3	262144	1008	1008	260128
4	16777216	20160	20160	16736896

(for $p = 3$: $8 \cdot 7 \cdot 6 = 336$ lists times $3! = 6$ re-orderings is 2016, half of it 1008; for $p = 4$: $8 \cdot 7 \cdot 6 \cdot 5 = 1680$ times $4! = 24$ is 40320, half of it 20160; the zeros are the rest). The Revision record compared GKD with the author's definition for every pair of lengths 1, 2 and 3, 266,304 pairs in all, and recorded these counts in `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, field `measurements`, entry `gkdComparison`, with the checks `gkd_equals_kdelta_exhaustive_length_1` to `_3`; its Rust self-test compared all 16,777,216 pairs of length 4 (`Revision/gkd_lovelock/results/gkd-selftest.json`). Notebook 01c, In [8] and In [10], repeats both comparisons, every pair, and gets the same counts and no disagreement (COMPUTED, and the counts PROVED by the formula). The nonzero values become rare as p grows: their fraction $N_{\neq 0}(8,p)/8^{2p}$ falls from $1/8$ at $p = 1$ to $8! \cdot 8!/8^{16} \approx 5.8 \times 10^{-6}$ at $p = 8$ (Figures 01c.3 and 01c.4).

A shortcut for the sign (PROVED, with rule 1 of Section 1.20 for general n, which is ASSUMED). When the lower list L is a re-ordering of the upper list U of p different labels, both are re-orderings of the same sorted list $s_1 < s_2 < \cdots < s_p$. Let P_L be the matrix with the entries $(P_L)_{ik} = \delta(l_i, s_k)$ and P_U the one with $(P_U)_{jk} = \delta(u_j, s_k)$; both are permutation matrices. Then

$$(P_L P_U^T)_{ij} = \sum_k \delta(l_i, s_k)\, \delta(u_j, s_k) = \delta(l_i, u_j) = M_{ij}$$

(the matrix product with the transpose; the sum has one nonzero term, the k with $s_k = l_i$, and that term is 1 exactly when $u_j = s_k = l_i$), so $\det M = \det P_L \det P_U^T = \det P_L \det P_U$ (rules 1 and 2). The permutation of P_L sends place i to the place of l_i in the sorted list, and two places $i < j$ are an inversion of it exactly when $l_i > l_j$; so $\det P_L$ is the sign counted by the inversions of the list L itself, $\mathrm{sign}(L)$, and

$$\delta^U_L = \operatorname{sign}(L)\operatorname{sign}(U).$$

This lets numpy compute millions of values at once (Notebook 01c, In [9]).

Nine indices in eight dimensions (PROVED). A list of 9 labels taken from only 8 must repeat a label, by the **pigeonhole principle**: if more than n objects are put into n boxes, some box gets at least two of them. So case (a) applies and the value is 0, for every one of the 8^{18} pairs of lists of length 9. In the **Lovelock tensors** of the theory (the terms of the Lovelock equations built from the **curvature** of spacetime, the measure of how it is bent, Chapters 3 and 11; a **tensor** is a table of numbers labelled by indices, like the metric of Section 1.29, whose entries change in a fixed way when the coordinates are changed, Chapter 3) the generalized delta appears with $2k+1$ indices, $k = 1, 2, 3, \ldots$; the term $k = 4$ would need 9 indices, so it vanishes identically, and the Lovelock sum of an eight-dimensional spacetime stops at $k = 3$. The record checks this in `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `gkd_nine_indices_in_eight_dimensions_vanish`, and `Revision/gkd_lovelock/results/lovelock-report.json`, check `k4_tensor_vanishes`. With 8 labels a nonzero value is still possible: the list $(x_0, \ldots, x_7)$ against its reverse has $8 \cdot 7/2 = 28$ inversions (every one of the $\binom{8}{2} = 28$ pairs is in the wrong order), an even number, so the value is $+1$.

1.44 Contracting one index; the Lovelock trace factors; the Levi-Civita symbol

The contraction identity (PROVED). Take two lists $A = (a_1, \ldots, a_{p-1})$ and $B = (b_1, \ldots, b_{p-1})$, append the same label c to both, and add over all n values of c. Then

$$\sum_c \delta^{a_1 \ldots a_{p-1} c}_{b_1 \ldots b_{p-1} c} = (n - p + 1)\, \delta^{a_1 \ldots a_{p-1}}_{b_1 \ldots b_{p-1}}.$$

Proof by cases. (i) If A or B repeats a label, every extended list repeats it too: every term is 0 by case (a) or (b), and so is the right side. (ii) If A and B consist of different labels but do not hold the same labels, then for each c: if c is a label of A or of B, an extended list repeats it (term 0); if not, the extended lists hold the labels of A plus c and those of B plus c, which still differ (term 0 by case (c)). The right side is 0 as well. (iii) If B is a re-ordering σ of A: for the $p-1$ values of c that are labels of A, the extended lists repeat c (term 0); for each of the other $n - (p-1)$ values, the extended lists are re-ordered by σ with c left at the last place, and that permutation has the same inversions as σ (the last place is in the right order with every other place), so each such term equals $\operatorname{sign}(\sigma)$. Together: $(n - p + 1)\operatorname{sign}(\sigma)$, which is the right side. For $p = 1$ the lists A and B are empty, and the delta of two empty lists is 1 (the determinant of the empty matrix: the Leibniz formula has one term, the empty product, which is 1); the identity then says $\sum_c \delta^c_c = n$, which is $\delta^a{}_a = 8$ of Section 1.26.

The factors of the Lovelock trace identities (PROVED). The Lovelock tensors of order k are built as

$$P_{(k)}{}^h{}_j = \delta^{h h_1 \ldots h_{2k}}_{j j_1 \ldots j_{2k}}\, R^{j_1 j_2}{}_{h_1 h_2} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}}, \qquad L_{(k)} = \delta^{h_1 \ldots h_{2k}}_{j_1 \ldots j_{2k}}\, R^{j_1 j_2}{}_{h_1 h_2} \cdots$$

(summation convention over all repeated indices; R is the curvature of spacetime, whose meaning Chapter 3 explains; here only the generalized delta matters). The delta in $P_{(k)}$ has $p = 2k + 1$

indices. The **trace** $\sum_h P_{(k)}{}^h{}_h$ sets $j = h$ and sums: it contracts the first upper index with the first lower one. Moving these two indices from the front to the end of both lists re-orders the rows and the columns of the matrix M by the same permutation, which multiplies the determinant by its sign twice, that is, not at all (rule 3 of Section 1.20 applied once per exchange, to rows and to columns alike). So the contraction identity applies with $n = 8$ and $p = 2k + 1$:

$$\sum_h P_{(k)}{}^h{}_h = (8 - (2k + 1) + 1)\, L_{(k)} = (8 - 2k)\, L_{(k)},$$

with the factors 6, 4 and 2 for $k = 1, 2, 3$. These are exactly the factors of the record's trace identities: `Revision/gkd_lovelock/results/lovelock-report.json`, checks `k1_trace_identity` to `k3_trace_identity` (their texts read `sum_h P_(1)^h_h = (8 - 2) L_(1)` and so on), and `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, checks `k1_trace_equals_6_L1`, `k2_trace_equals_4_L2`, `k3_trace_equals_2_L3`. Notebook 01c, In [14] and In [15], finds the factor $n - p + 1$ for $n = 8$ and every $p = 1$ to 8, and for $n = 4$ and every $p = 1$ to 4 (COMPUTED; Figure 01c.5).

The Levi-Civita symbol (PROVED). The **Levi-Civita symbol** of n labels is the generalized delta with the upper list $(0, 1, \ldots, n - 1)$, all labels in their natural order:

$$\varepsilon_{i_1 \ldots i_n} = \delta^{0\,1\,\ldots\,n-1}_{i_1 \ldots i_n},$$

by case (d) the sign of the list $(i_1, \ldots, i_n)$ if it is a re-ordering of all n labels, and 0 otherwise. For three labels: $\varepsilon_{012} = \varepsilon_{120} = \varepsilon_{201} = +1$, $\varepsilon_{021} = \varepsilon_{102} = \varepsilon_{210} = -1$, and 0 for every list with a repeated label (Figure 01c.6). The product of two of them, summed over their last $n - p$ indices, is

$$\sum_{c_{p+1}, \ldots, c_n} \varepsilon_{a_1 \ldots a_p c_{p+1} \ldots c_n}\, \varepsilon_{b_1 \ldots b_p c_{p+1} \ldots c_n} = (n - p)!\; \delta^{b_1 \ldots b_p}_{a_1 \ldots a_p}.$$

Proof. For full lists ($p = n$), both sides are the product of the signs of the two lists, or 0: by the sign shortcut of Section 1.43, $\delta^B_A = \operatorname{sign}(A) \operatorname{sign}(B) = \varepsilon_A \varepsilon_B$ when both are re-orderings of all labels, and both sides are 0 otherwise. Then contract the last index $n - p$ times with the contraction identity: the first contraction (of a delta with n indices) gives the factor $n - n + 1 = 1$, the next 2, and the last (of a delta with $p + 1$ indices) $n - p$; the product of the factors is $1 \cdot 2 \cdots (n - p) = (n - p)!$.

The author's second route. The author's notebook builds the generalized delta also from two Levi-Civita tensors, in the input cell In[32] (`delta11 = Simplify[(epsilong[-a, -f, ...] epsilong[b, f, ...])/(8 - 1)!]`): the formula above with $n = 8$, $p = 1$ and the divisor 7!. In the author's notation the second ε carries upper indices, raised with the metric, and the minus signs mark lower indices. For the frame metric η, raising all eight indices of ε multiplies each nonzero entry by the product of the eight diagonal entries of η^{-1} (each label occurs exactly once in a nonzero entry), that is by $\det \eta = (+1)^4(-1)^4 = +1$ (PROVED). The author's `epsilong` is the Levi-Civita tensor of a metric g that the author declares in the cell In[29] with `DefMetric[{4, 4, 0}, ...]`: four positive, four negative and no zero eigenvalues, the signature (4,4). For such a tensor the same step gives the sign of $\det g$ (a fact of tensor calculus, ASSUMED here), which is $(-1)^4 = +1$ as for η (and for the author's metric of the Revision record, $\det g = \cos^2 z > 0$, Section 1.21). In four-dimensional spacetime, with one -1, the same step would give the factor -1. So in the signature (4,4) the author's second route gives the generalized delta with no extra sign;

Notebook 01c, In [17], checks this: for all 64 pairs (a, b) the sums are $5040 = 7!$ for $a = b$ and 0 otherwise, the 8×8 identity $\delta^b{}_a$ after the division by 7! (COMPUTED).

1.45 Example: the generalized Kronecker delta

Notebook 01c puts Sections 1.42 to 1.44 to work. It reads the author's definition and the cells of their notebook from the Revision record; writes the definition in Python literally, as a Leibniz determinant of the matrix `Outer`, and again as the fast rule of GKD; reproduces every example of the record; computes all 4096 values with two indices; compares the literal determinant with the fast rule for every one of the 266,304 pairs of lengths 1 to 3 and the 16,777,216 pairs of length 4, and compares the counts with the record; checks the counting formula and the vanishing with nine indices; finds the contraction factors $n - p + 1$ and the factors 6, 4, 2 of the record's Lovelock trace identities; and checks the Levi-Civita product formula and the author's second route. It ends with the line ALL 28 CHECKS PASSED (notebook 01c).

1.46 How to run Notebook 01c

Step 1. What this notebook does and what it needs.

Notebook 01c (The generalized Kronecker delta as a determinant) is the file `Revision/textbook/notebooks/01c_generalized_delta.ipynb` of the repository Dirac_claude. It reads the author's definition of the generalized Kronecker delta from the Revision record (the determinant of the matrix of ordinary Kronecker deltas of two index lists), writes it in Python twice, literally with the Leibniz determinant and as the fast rule of the Revision program GKD (zero for a repeated or missing index, otherwise the sign of a permutation), reproduces the examples and the exhaustive counts of the Revision record for index lists of length 1 to 4 over the eight coordinates x0 to x7 (16,777,216 pairs for length 4), proves and checks the counting formula for the nonzero values, the vanishing for nine indices in eight dimensions, the two-index formula, the contraction identity that gives the factors 6, 4 and 2 of the Lovelock trace identities, and the product formula of two Levi-Civita symbols that the author's notebook also uses, with heat maps and plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then

open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01c_generalized_delta.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01c_generalized_delta.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01c_generalized_delta.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01c.captions.json`
- `Revision/textbook/figures/01c_1_outer_matrices.png`
- `Revision/textbook/figures/01c_2_two_index_table.png`
- `Revision/textbook/figures/01c_3_value_counts.png`
- `Revision/textbook/figures/01c_4_nonzero_fraction.png`
- `Revision/textbook/figures/01c_5_contraction_factor.png`
- `Revision/textbook/figures/01c_6_levi_civita.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 28 CHECKS PASSED (notebook 01c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01c_generalized_delta.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01c_generalized_delta.ipynb
python -m nbconvert --execute --inplace 01c_generalized_delta.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock or Revision/algebra: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.
- the computer becomes slow during the cell that checks all 16,777,216 pairs: that cell needs about 250 MB of memory and 10 to 30 seconds; close other programs and run the cell again. It works in blocks of 1,048,576 pairs, so it never needs more memory than that.

1.47 Notebook 01c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01c_generalized_delta.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field equations of gravity that the author's theory uses (the Lovelock
equations) are built with one combinatorial tool: the **generalized Kronecker
delta**. The author defined it in his Mathematica notebook as the determinant of
a small matrix of zeros and ones; the Revision record computes it with a fast
program called GKD. This notebook

- reads the author's definition from the Revision record and writes it in Python
  literally (a Leibniz determinant), and again as the fast rule of GKD;
- reproduces every example of the record and the record's counts of the values
  $+1$, $-1$ and $0$ for all pairs of index lists of length 1, 2 and 3 over the
  eight coordinates $x_0, \dots, x_7$, and checks all 16,777,216 pairs of length
  4, as the record's self-test did;
- proves and checks a formula for the number of nonzero values, and that nine
  indices in eight dimensions always give 0;
- proves and checks the contraction identity, which explains the factors 6, 4
```

  and 2 that appear in the record's Lovelock trace identities;
- shows that the Levi-Civita symbol is a special generalized delta and checks
  the product formula of two Levi-Civita symbols, the second route of the
  author's notebook.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01c (The generalized Kronecker delta as a determinant) is the file
‘Revision/textbook/notebooks/01c_generalized_delta.ipynb‘ of the repository Dirac_claude.
It reads the author's definition of the generalized Kronecker delta from the Revision
record (the determinant of the matrix of ordinary Kronecker deltas of two index lists),
writes it in Python twice, literally with the Leibniz determinant and as the fast rule of
the Revision program GKD (zero for a repeated or missing index, otherwise the sign of a
permutation), reproduces the examples and the exhaustive counts of the Revision record
for index lists of length 1 to 4 over the eight coordinates x0 to x7 (16,777,216 pairs
for length 4), proves and checks the counting formula for the nonzero values, the
vanishing for nine indices in eight dimensions, the two-index formula, the contraction
identity that gives the factors 6, 4 and 2 of the Lovelock trace identities, and the
product formula of two Levi-Civita symbols that the author's notebook also uses, with
heat maps and plots. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:
```

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in

this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01c_generalized_delta.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the

address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01c_generalized_delta.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01c_generalized_delta.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01c.captions.json‘
- ‘Revision/textbook/figures/01c_1_outer_matrices.png‘
- ‘Revision/textbook/figures/01c_2_two_index_table.png‘
- ‘Revision/textbook/figures/01c_3_value_counts.png‘
- ‘Revision/textbook/figures/01c_4_nonzero_fraction.png‘
- ‘Revision/textbook/figures/01c_5_contraction_factor.png‘
- ‘Revision/textbook/figures/01c_6_levi_civita.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 28 CHECKS PASSED (notebook 01c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01c_generalized_delta.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 01c_generalized_delta.ipynb
      python -m nbconvert --execute --inplace 01c_generalized_delta.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock or Revision/algebra: the notebook reads Revision records of the repository; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.
- the computer becomes slow during the cell that checks all 16,777,216 pairs: that cell needs about 250 MB of memory and 10 to 30 seconds; close other programs and run the cell again. It works in blocks of 1,048,576 pairs, so it never needs more memory than that.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a

check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 01c (The generalized Kronecker delta as a determinant) is the file
# Revision/textbook/notebooks/01c_generalized_delta.ipynb of the repository Dirac_claude.
# It reads the author’s definition of the generalized Kronecker delta from the Revision
# record (the determinant of the matrix of ordinary Kronecker deltas of two index lists),
# writes it in Python twice, literally with the Leibniz determinant and as the fast rule
# of the Revision program GKD (zero for a repeated or missing index, otherwise the sign
# of a permutation), reproduces the examples and the exhaustive counts of the Revision
# record for index lists of length 1 to 4 over the eight coordinates x0 to x7 (16,777,216
# pairs for length 4), proves and checks the counting formula for the nonzero values, the
# vanishing for nine indices in eight dimensions, the two-index formula, the contraction
# identity that gives the factors 6, 4 and 2 of the Lovelock trace identities, and the
# product formula of two Levi-Civita symbols that the author’s notebook also uses, with
# heat maps and plots. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy and matplotlib; the other packages run Jupyter, the program that shows
# and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
```

```
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
```

```
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01c_generalized_delta.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
```

```
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01c_generalized_delta.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01c_generalized_delta.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01c.captions.json
# - Revision/textbook/figures/01c_1_outer_matrices.png
# - Revision/textbook/figures/01c_2_two_index_table.png
# - Revision/textbook/figures/01c_3_value_counts.png
# - Revision/textbook/figures/01c_4_nonzero_fraction.png
# - Revision/textbook/figures/01c_5_contraction_factor.png
# - Revision/textbook/figures/01c_6_levi_civita.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 28 CHECKS PASSED (notebook 01c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01c_generalized_delta.ipynb inside the folder
#   Dirac_claude.
```

```
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01c_generalized_delta.ipynb
#     python -m nbconvert --execute --inplace 01c_generalized_delta.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below Revision/gkd_lovelock or Revision/algebra: the
#   notebook reads Revision records of the repository; your copy of the repository is
#   incomplete. Download it again with git clone and open the notebook inside the new
#   copy.
# - the computer becomes slow during the cell that checks all 16,777,216 pairs: that cell
#   needs about 250 MB of memory and 10 to 30 seconds; close other programs and run the
#   cell again. It works in blocks of 1,048,576 pairs, so it never needs more memory than
#   that.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "01c"  # this notebook: chapter 01, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).
```

```
    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
```

```
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Label**: a name of a coordinate, here $x_0, \dots, x_7$. The computer often
  numbers them $0, \dots, 7$; only the question "are two labels equal?" matters
  below, so the names do not change any result.
- **Index list**: an ordered list of labels, such as $(x_1, x_3, x_3)$; its
  **length** is the number of entries (here 3).
- **Kronecker delta** $\delta(a, b)$: 1 if the labels $a$ and $b$ are equal, 0
  otherwise (also written $\delta^a_b$).
- **Generalized Kronecker delta** of a lower list $(l_1, \dots, l_p)$ and an upper
  list $(u_1, \dots, u_p)$ of the same length $p$: the determinant of the
  $p \times p$ matrix whose entry in row $i$ and column $j$ is
  $\delta(l_i, u_j)$; written $\delta^{u_1 \dots u_p}_{l_1 \dots l_p}$.
- **Outer**: Mathematica's name for the operation that builds that matrix,
  `Outer[delta, lower, upper]`. The author's function is called
  `k\[Delta]` (Mathematica's plain-text spelling of "k" followed by the Greek
  letter delta).
- **GKD**: the Revision record's program (written in the language Rust) that
  computes the generalized delta without a determinant.
- **Permutation, inversion, sign**: a re-ordering of a list; a pair of places in
  the wrong order; $(-1)^{\text{number of inversions}}$.
- **Exhaustive check**: a check of *every* possible case, not of examples.
- **Vectorized computation**: numpy applies one operation to millions of numbers
  in an array at once, much faster than a Python loop.
- **Pigeonhole principle**: if more than $n$ objects are put into $n$ boxes, some
  box gets at least two of them.
- **Contraction**: setting an upper and a lower index equal and adding over all
  values of that index (the summation convention).
- **Levi-Civita symbol** $\varepsilon_{i_1 \dots i_n}$: the sign of the list
  $(i_1, \dots, i_n)$ if it is a re-ordering of all $n$ labels, and 0 otherwise.
- **Falling factorial** $n!/(n-p)! = n (n-1) \cdots (n-p+1)$: the number of ways
  to choose $p$ different labels from $n$ in order (`math.perm(n, p)`).
- **Lovelock tensors**: combinations of the curvature of spacetime that enter the
  field equations of gravity in more than four dimensions; the Revision record
  computes three of them for the author's metric, with GKD as the weight of every
  term.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's Mathematica notebook defines (input cell In[54], stored in the file
as In[87]):

`k\[Delta][lower_, upper_] /; Length[lower] == Length[upper] :=
Det[Outer[delta, lower, upper]]`

In words: for two index lists of equal length, build the matrix
$M_{ij} = \delta(l_i, u_j)$ and take its determinant. For one index this is the
ordinary Kronecker delta. For two indices the $2 \times 2$ determinant gives

$$\delta^{u_1 u_2}_{l_1 l_2} = \delta(l_1, u_1)\,\delta(l_2, u_2)
- \delta(l_1, u_2)\,\delta(l_2, u_1).$$
```

```
The Revision program GKD avoids the determinant with a proof in four cases
(written in the file Revision/gkd_lovelock/code/src/gkd.rs):

- (a) two equal lower labels make two equal rows of $M$, so $\det M = 0$;
- (b) two equal upper labels make two equal columns, so $\det M = 0$;
- (c) a lower label that is not among the upper labels makes a row of zeros, so
  $\det M = 0$;
- (d) otherwise both lists consist of the same $p$ different labels; each $l_i$
  equals exactly one $u_{\sigma(i)}$, $M$ is the permutation matrix of $\sigma$,
  and $\det M = \mathrm{sign}(\sigma)$ (only one term of the Leibniz formula is
  not zero).

In the Lovelock tensors of the author's theory the generalized delta appears with
$2k + 1$ indices, $k = 1, 2, 3$, in the eight dimensions $x_0, \dots, x_7$:
$P_{(k)}{}^h{}_j = \delta^{h h_1 \dots h_{2k}}_{j j_1 \dots j_{2k}}
R^{j_1 j_2}{}_{h_1 h_2} \cdots$ (summation convention). Two facts of this
notebook matter there: with 9 indices ($k = 4$) the delta is always 0, so the
sum stops at $k = 3$; and contracting $h$ with $j$ multiplies by $8 - 2k$, which
is why the record's trace identities read $\sum_h P_{(k)}{}^h{}_h = (8 - 2k)
L_{(k)}$ with the factors 6, 4 and 2.
```

Text cell:

```
## 5. The author's definition, read from the record

The next cell reads two files of the Revision record. The file
PROVENANCE_OF_THE_COMPUTATION.md records the author's definition verbatim, with
the Greek letter $\delta$; this notebook is written in plain ASCII, so the letter
is typed as its code `\u03b4`. The file notebook-input-cells.txt lists the input
cells of the author's notebook, each under the label stored in the file; the cell
finds the definition (In[87]), the first cell of the author's second route
(In[32], a product of two Levi-Civita tensors, section 15 of this notebook) and
the cell that declares the metric $g$ used by that route (In[29]), and prints
them, with the Greek letter written as `\[Delta]`. `re.search` looks for a
pattern in a text; `HoldForm[...]` is how the record wraps every cell.
```

In [2]:

```
import itertools  # all lists of labels, all permutations
import json  # reads the JSON reports of the Revision record
import math  # factorials
import re  # finds patterns in texts

import numpy as np  # arrays: many index lists at once

DELTA = "\u03b4"  # the Greek letter delta, written with its code
definition = ("k" + DELTA + "[lower_, upper_] /; Length[lower] == Length[upper] := "
              "Det[Outer[delta, lower, upper]]")
provenance_path = repository_file("Revision/gkd_lovelock/results/"
                                  "PROVENANCE_OF_THE_COMPUTATION.md")
provenance = provenance_path.read_text(encoding="utf-8")
```

```
check(definition in provenance, "the record holds the author's definition verbatim",
      record="Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md")
input_cells = repository_file("Revision/gkd_lovelock/results/"
                              "notebook-input-cells.txt").read_text(encoding="utf-8")
found = {}
for line in input_cells.splitlines():
    for label in ("In[87]:=", "In[32]:=", "In[29]:="):
        if line.startswith(label):
            found[label] = re.search(r"HoldForm\[(.*)\]", line).group(1)
for label in ("In[87]:=", "In[32]:=", "In[29]:="):
    plain_text = found[label].replace(DELTA, "\\[Delta]")  # Mathematica's spelling
    say(f"author's cell {label} {plain_text}")
metric_cell = found["In[29]:="]  # kept for section 15 (the declaration of g)
check("Det[Outer[delta, lower, upper]]" in found["In[87]:="] and
      "epsilong" in found["In[32]:="] and "(8 - 1)!" in found["In[32]:="],
      "the author's notebook defines the delta as a determinant (In[87]) and "
      "also uses a product of two Levi-Civita tensors (In[32])",
      record="Revision/gkd_lovelock/results/notebook-input-cells.txt")
```

Out [2]:

```
PASS the record holds the author's definition verbatim
     reproduces Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md
author's cell In[87]:= Clear[k\[Delta]]; k\[Delta][lower_, upper_] /; Length[lower] ==
    Length[upper] := Det[Outer[delta, lower, upper]]
author's cell In[32]:= delta11 = Simplify[(epsilong[-a, -f, -i, -j, -k, -l, -m,
    -n]*epsilong[b, f, i, j, k, l, m, n])/(8 - 1)!]
author's cell In[29]:= DefMetric[{4, 4, 0}, g[-a, -b], CD, {";", "D"}, WeightedWithBasis
    -> AIndex]
PASS the author's notebook defines the delta as a determinant (In[87]) and also uses a
    product of two Levi-Civita tensors (In[32])
     reproduces Revision/gkd_lovelock/results/notebook-input-cells.txt
```

Text cell:

```
## 6. The generalized Kronecker delta, literally

The next cell writes the author's definition in Python, word for word:

- `kronecker(a, b)` is the ordinary Kronecker delta of two labels;
- `outer_matrix(lower, upper)` is `Outer[delta, lower, upper]`;
- `leibniz_det(rows)` is the determinant by the Leibniz formula; the list of the
  permutations of each size and their signs is computed once and kept in the
  dictionary `SIGNED`, because the same lists are needed millions of times;
- `kdelta_literal(lower, upper)` refuses lists of different lengths (the
  author's definition then stays unevaluated) and otherwise returns the
  determinant.

The labels may be any names: here the strings `"x0"`, ..., `"x7"`.
```

In [3]:

```
def kronecker(a, b):
    """The ordinary Kronecker delta of two labels: 1 if they are equal, else 0."""
    return 1 if a == b else 0


def outer_matrix(lower, upper):
    """Outer[delta, lower, upper]: row i, column j is kronecker(lower[i], upper[j])"""
    return [[kronecker(l_label, u_label) for u_label in upper] for l_label in lower]


def inversions(order):
    """The number of pairs of places i < j whose entries stand in the wrong order."""
    return sum(1 for i in range(len(order)) for j in range(i + 1, len(order))
               if order[i] > order[j])


def sign(order):
    """+1 for an even number of inversions, -1 for an odd number."""
    return 1 if inversions(order) % 2 == 0 else -1


SIGNED = {}  # size n -> list of (permutation, its sign), computed once per size


def signed_permutations(n):
    if n not in SIGNED:
        SIGNED[n] = [(order, sign(order)) for order in itertools.permutations(range(n))]
    return SIGNED[n]


def leibniz_det(rows):
    """The Leibniz determinant of a square matrix given as a list of rows."""
    total = 0
    for order, order_sign in signed_permutations(len(rows)):
        term = order_sign
        for i, column in enumerate(order):
            term *= rows[i][column]
            if term == 0:  # one factor 0 makes the whole term 0: stop early
                break
        total += term
    return total


def kdelta_literal(lower, upper):
    """The author's k-delta: Det[Outer[delta, lower, upper]] for equal lengths."""
    if len(lower) != len(upper):
        raise ValueError("the two index lists must have the same length")
    return leibniz_det(outer_matrix(lower, upper))


def signed(value):
    """A value +1, -1 or 0 as the text "+1", "-1" or "0"."""
    return f"{value:+d}" if value else "0"
```

```
same = kdelta_literal(("x1",), ("x1",))  # lists of length 1: ("x1",) has one entry
different = kdelta_literal(("x1",), ("x2",))
say(f"one index: delta(x1, x1) = {same}, delta(x1, x2) = {different}")
check(same == 1 and different == 0,
      "for one index the generalized delta is the ordinary Kronecker delta")
```

Out [3]:

```
one index: delta(x1, x1) = 1, delta(x1, x2) = 0
PASS for one index the generalized delta is the ordinary Kronecker delta
```

Text cell:

```
The next cell draws the matrix `Outer[delta, lower, upper]` for five pairs of
index lists of length 3 and writes the determinant in each title. The rows belong
to the lower list, the columns to the upper list; a dark square is a 1. They are
the four cases of the proof in section 4: the same list twice (the identity
matrix, $+1$); the upper list turned around cyclically (a permutation with two
inversions, $+1$); two upper labels exchanged (one inversion, $-1$); a repeated
lower label (two equal rows, 0); a lower label $x_4$ missing from the upper list
(a row of zeros, 0).
```

In [4]:

```
examples = [(("x1", "x2", "x3"), ("x1", "x2", "x3"), "same lists"),
            (("x1", "x2", "x3"), ("x2", "x3", "x1"), "cyclic turn"),
            (("x1", "x2", "x3"), ("x1", "x3", "x2"), "one exchange"),
            (("x1", "x1", "x3"), ("x1", "x2", "x3"), "repeated lower"),
            (("x1", "x2", "x4"), ("x1", "x2", "x3"), "x4 missing above")]
fig, axes = plt.subplots(1, 5, figsize=(12.0, 3.4))
for ax, (lower, upper, what) in zip(axes, examples):
    matrix = np.array(outer_matrix(lower, upper))
    ax.imshow(matrix, cmap="Greys", vmin=0, vmax=1.4)
    ax.grid(False)
    ax.set_xticks(range(3), [f"${u[0]}_{u[1]}$" for u in upper])  # "x1" -> x_1
    ax.set_yticks(range(3), [f"${l_label[0]}_{l_label[1]}$" for l_label in lower])
    ax.set_xlabel("upper list")
    ax.set_title(f"{what}: det = {signed(kdelta_literal(lower, upper))}", fontsize=9)
axes[0].set_ylabel("lower list")
fig.tight_layout()
save_figure(fig, "outer_matrices",
            "The matrix Outer of ordinary Kronecker deltas for five pairs of index "
            "lists of length 3: rows are the lower list, columns the upper list, a "
            "dark square is an entry 1 and a white square an entry 0; the title "
            "gives the determinant, which is the generalized Kronecker delta. From "
            "left to right: equal lists (identity matrix, $+1$), a cyclic turn "
            "($+1$), one exchange ($-1$), a repeated lower label (two equal rows, "
            "0) and a lower label missing from the upper list (a row of zeros, 0).")
values = [kdelta_literal(lower, upper) for lower, upper, _ in examples]
check(values == [1, 1, -1, 0, 0], "the five pictured examples give +1, +1, -1, 0, 0")
```

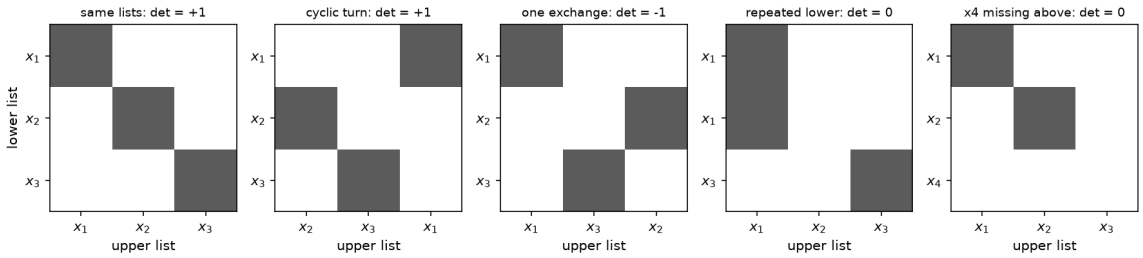

Figure 50: The matrix Outer of ordinary Kronecker deltas for five pairs of index lists of length 3: rows are the lower list, columns the upper list, a dark square is an entry 1 and a white square an entry 0; the title gives the determinant, which is the generalized Kronecker delta. From left to right: equal lists (identity matrix, +1), a cyclic turn (+1), one exchange (−1), a repeated lower label (two equal rows, 0) and a lower label missing from the upper list (a row of zeros, 0). (Notebook 01c, figure 1.)

Out [4]:

```
Figure 01c.1 saved as Revision/textbook/figures/01c_1_outer_matrices.png
PASS the five pictured examples give +1, +1, -1, 0, 0
```

Text cell:

```
## 7. The fast rule of the program GKD

The next cell writes the four cases of the proof as the Python function
‘gkd_rule‘, line by line as the Rust function ‘GKD‘ of the record does it: first
it notes the place of every upper label (and returns 0 at a repeated upper label,
case b); then for every lower label it looks up its place among the upper labels
(returning 0 if it is missing, case c, or if that place was taken already, which
means a repeated lower label, case a); the places form the permutation
$\sigma$, whose sign is the result (case d). It never builds a matrix.

The cell checks that the rule gives the literal values for the five examples, and
that renaming the labels ($x_k$ to $x_{k+1}$, with $x_7$ to $x_0$) changes no value:
only equal or different matters.
```

In [5]:

```
def gkd_rule(lower, upper):
    """The program GKD of Revision/gkd_lovelock/code/src/gkd.rs, in Python."""
    if len(lower) != len(upper):
        raise ValueError("the two index lists must have the same length")
    place = {}  # upper label -> its place 0, 1, ..., p-1
    for j, u_label in enumerate(upper):
        if u_label in place:  # case (b): a repeated upper label
            return 0
        place[u_label] = j
    sigma = []  # sigma[i] = the place of lower[i] among the upper labels
```

```
    for l_label in lower:
        if l_label not in place:  # case (c): a lower label missing above
            return 0
        if place[l_label] in sigma:  # case (a): a repeated lower label
            return 0
        sigma.append(place[l_label])
    return sign(sigma)  # case (d): the sign of the permutation sigma


check(all(gkd_rule(lower, upper) == kdelta_literal(lower, upper)
          for lower, upper, _ in examples),
      "the fast rule gives the literal values for the five examples")
renamed = {f"x{k}": f"x{(k + 1) % 8}" for k in range(8)}  # x0->x1, ..., x7->x0
two_lists = list(itertools.product([f"x{k}" for k in range(8)], repeat=2))
check(all(gkd_rule(lo, up) == gkd_rule(tuple(renamed[a] for a in lo),
                                       tuple(renamed[b] for b in up))
          for lo in two_lists for up in two_lists),
      "renaming the labels changes no value (all 4096 pairs of length 2)")
```

Out [5]:

```
PASS the fast rule gives the literal values for the five examples
PASS renaming the labels changes no value (all 4096 pairs of length 2)
```

Text cell:

```
## 8. The examples of the record

The next cell reads three reports of the Revision record and repeats their
examples with both functions:

- python-lovelock-report.json, check ‘gkd_examples‘: five values written as
  ‘kdelta[{1,2},{2,1}] = -1‘ and so on (labels 1, 2, 3, 4); a regular expression
  (‘re.findall‘) picks the two lists and the value out of the text;
- wolfram-gkd-report.json, check ‘definition_unequal_lengths_stay_unevaluated‘:
  the values $-1$ and $+1$ of two examples with labels 0, 1, 2, and the refusal
  of lists of different lengths (our functions raise a ‘ValueError‘);
- lovelock-report.json, field ‘gkdSelfCheck‘: the two values that the Rust
  program printed for the lists $(0, 1, 2), (1, 2, 0)$ and $(0, 1), (1, 0)$.
```

In [6]:

```
def read_report(name):
    """A JSON report of Revision/gkd_lovelock/results as a Python dictionary."""
    path = repository_file(f"Revision/gkd_lovelock/results/{name}")
    return json.loads(path.read_text(encoding="utf-8"))


def lists_from_text(text):
    """"1,2,3" -> (1, 2, 3)"""
    return tuple(int(part) for part in text.replace(" ", "").split(","))
```

```
python_report = read_report("python-lovelock-report.json")
python_checks = {c["name"]: c for c in python_report["checks"]}
pattern = r"kdelta\[\{([\d,]+)\},\{([\d,]+)\}\] = (-?\d+)"
record_examples = re.findall(pattern, python_checks["gkd_examples"]["detail"])
for lower_text, upper_text, value in record_examples:
    lower, upper = lists_from_text(lower_text), lists_from_text(upper_text)
    say(f"lower {lower}, upper {upper}: record {signed(int(value))}, literal "
        f"{signed(kdelta_literal(lower, upper))}, rule {signed(gkd_rule(lower, upper))}")
check(len(record_examples) == 5 and all(
    kdelta_literal(lists_from_text(lo), lists_from_text(up)) == int(v) ==
    gkd_rule(lists_from_text(lo), lists_from_text(up)) for lo, up, v in record_examples),
    "the five examples of the record", record="Revision/gkd_lovelock/results/"
    "python-lovelock-report.json, check gkd_examples")
wolfram_report = read_report("wolfram-gkd-report.json")
wolfram_checks = {c["name"]: c for c in wolfram_report["checks"]}
detail = wolfram_checks["definition_unequal_lengths_stay_unevaluated"]["detail"]
wolfram_pattern = "k" + DELTA + r"\[\{([\d, ]+)\}, \{([\d, ]+)\}\] = ([+-]?\d+)"
wolfram_examples = re.findall(wolfram_pattern, detail)
try:
    kdelta_literal((0,), (0, 1))
    refused = False
except ValueError:  # lists of different lengths are refused
    refused = True
check(refused and len(wolfram_examples) == 2 and all(
    kdelta_literal(lists_from_text(lo), lists_from_text(up)) == int(v)
    for lo, up, v in wolfram_examples),
    "lists of different lengths are refused; the two Wolfram examples",
    record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, check "
    "definition_unequal_lengths_stay_unevaluated")
self_check = read_report("lovelock-report.json")["gkdSelfCheck"]
length3_pair, transposition = self_check["length3Pair"], self_check["transposition"]
say(f"lovelock-report.json gkdSelfCheck: length3Pair = {length3_pair}, "
    f"transposition = {transposition}")
check(self_check["length3Pair"] == gkd_rule((0, 1, 2), (1, 2, 0)) and
      self_check["transposition"] == gkd_rule((0, 1), (1, 0)),
      "the two values printed by the Rust program",
      record="Revision/gkd_lovelock/results/lovelock-report.json, gkdSelfCheck")
```

Out [6]:

```
lower (1, 2), upper (1, 2): record +1, literal +1, rule +1
lower (1, 2), upper (2, 1): record -1, literal -1, rule -1
lower (1, 1), upper (1, 1): record 0, literal 0, rule 0
lower (1, 2, 3), upper (2, 3, 1): record +1, literal +1, rule +1
lower (1, 2, 3), upper (1, 2, 4): record 0, literal 0, rule 0
PASS the five examples of the record
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    gkd_examples
PASS lists of different lengths are refused; the two Wolfram examples
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, check
    definition_unequal_lengths_stay_unevaluated
```

```
lovelock-report.json gkdSelfCheck: length3Pair = 1, transposition = -1
PASS the two values printed by the Rust program
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, gkdSelfCheck
```

Text cell:

```
## 9. Two indices: the 64 by 64 table

With two indices over eight labels there are $8^2 = 64$ lower lists and 64 upper
lists, so $64 \cdot 64 = 4096$ values. The next cell computes all of them, checks
the two-term formula
$\delta^{u_1 u_2}_{l_1 l_2} = \delta(l_1, u_1)\delta(l_2, u_2)
- \delta(l_1, u_2)\delta(l_2, u_1)$ for every one of them, counts the values
$+1$ and $-1$ (each must be 56: a list of two *different* labels, $8 \cdot 7 = 56$
of them, gives $+1$ with itself and $-1$ with its reverse), and draws the table.
From here on the labels are the numbers 0 to 7, the computer's names for
$x_0$ to $x_7$.
```

In [7]:

```
pairs = list(itertools.product(range(8), repeat=2))  # (0, 0), (0, 1), ..., (7, 7)
table = np.array([[gkd_rule(lo, up) for up in pairs] for lo in pairs])  # 64 x 64
two_term_ok = all(
    gkd_rule(lo, up) == kronecker(lo[0], up[0]) * kronecker(lo[1], up[1])
    - kronecker(lo[0], up[1]) * kronecker(lo[1], up[0])
    for lo in pairs for up in pairs)
check(two_term_ok, "the two-term formula holds for all 4096 pairs of length 2")
say(f"values +1: {(table == 1).sum()}, values -1: {(table == -1).sum()}, "
    f"zeros: {(table == 0).sum()}")
check((table == 1).sum() == 56 and (table == -1).sum() == 56,
      "56 values +1 and 56 values -1 among the 4096")
fig, ax = plt.subplots(figsize=(6.6, 6.2))
image = ax.imshow(table, cmap="RdBu_r", vmin=-1, vmax=1, interpolation="nearest")
ax.grid(False)
ticks = [8 * k for k in range(8)]  # the first pair of each block of 8
ax.set_xticks(ticks, [f"$(x_{k}, x_0)$" for k in range(8)], rotation=90,
              fontsize=7)
ax.set_yticks(ticks, [f"$(x_{k}, x_0)$" for k in range(8)], fontsize=7)
ax.set_xlabel("upper list $(u_1, u_2)$")
ax.set_ylabel("lower list $(l_1, l_2)$")
ax.set_title("all 4096 values of the two-index delta")
fig.colorbar(image, ax=ax, shrink=0.8, ticks=[-1, 0, 1])
save_figure(fig, "two_index_table",
            "All 4096 values of the generalized delta with two indices over the "
            "eight labels $x_0$ to $x_7$: row $(l_1, l_2)$, column $(u_1, u_2)$, "
            "both running through $(x_0, x_0), (x_0, x_1), \\dots, (x_7, x_7)$ "
            "(the tick marks the first pair of each block of 8); red $+1$, blue "
            "$-1$, white 0. The 56 red dots lie on the diagonal (equal lists of two "
            "different labels), the 56 blue dots where the upper list is the "
            "lower list reversed; the 8 lists with two equal labels give white "
            "gaps on the diagonal.")
```

Out [7]:

```
PASS the two-term formula holds for all 4096 pairs of length 2
values +1: 56, values -1: 56, zeros: 3984
PASS 56 values +1 and 56 values -1 among the 4096
```

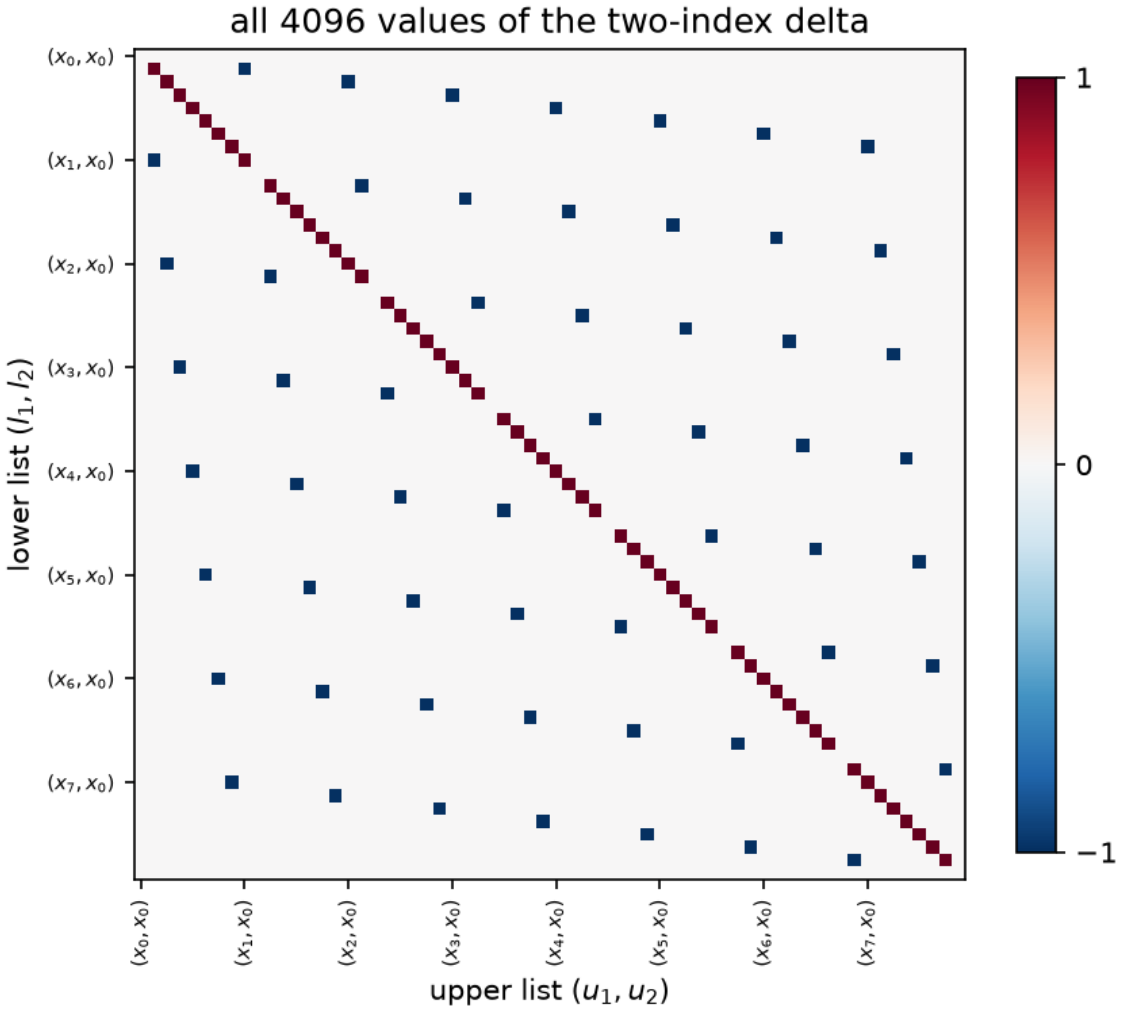

Figure 51: All 4096 values of the generalized delta with two indices over the eight labels x_0 to x_7: row (l_1, l_2), column (u_1, u_2), both running through $(x_0, x_0), (x_0, x_1), \ldots, (x_7, x_7)$ (the tick marks the first pair of each block of 8); red $+1$, blue -1, white 0. The 56 red dots lie on the diagonal (equal lists of two different labels), the 56 blue dots where the upper list is the lower list reversed; the 8 lists with two equal labels give white gaps on the diagonal. (Notebook 01c, figure 2.)

Out [7]:

```
Figure 01c.2 saved as Revision/textbook/figures/01c_2_two_index_table.png
```

Text cell:

10. All pairs of index lists of length 1, 2 and 3

The Revision record compared GKD with the author's definition for *every* pair of index lists of length 1, 2 and 3 over the eight labels: $8^2 = 64$, $8^4 = 4096$ and 8^6 = 262,144 pairs, 266,304 in all, and recorded how many values are $+1$, -1 and 0. The next cell does the same with plain Python loops: for every pair it computes the literal determinant and the fast rule, counts disagreements and tallies the values. Then it compares the tallies with the record's `measurements` (the field `gkdComparison` of wolfram-gkd-report.json), and its number of disagreements, 0, with the record's three exhaustive checks, which also report 0 mismatches. It keeps every value in an array for the next section. This cell takes several seconds.

In [8]:

```
comparison = wolfram_report["measurements"]["gkdComparison"]  # one entry per length
recorded = {entry["p"]: entry for entry in comparison}
plain_values = {}  # length p -> array of all values, lower lists in the outer loop
tallies = {}
total_pairs = 0
mismatches = 0
for p in (1, 2, 3):
    lists = list(itertools.product(range(8), repeat=p))
    values = []
    for lower in lists:
        for upper in lists:
            fast = gkd_rule(lower, upper)
            mismatches += fast != kdelta_literal(lower, upper)
            values.append(fast)
    plain_values[p] = np.array(values)
    total_pairs += len(values)
    tally = {v: int((plain_values[p] == v).sum()) for v in (1, -1, 0)}
    tallies[p] = tally
    in_record = (recorded[p]["plusOne"], recorded[p]["minusOne"], recorded[p]["zero"])
    here = (tally[1], tally[-1], tally[0])
    say(f"length {p}: {len(values)} pairs")
    say(f"    counts of (+1, -1, 0): here {here}, record {in_record}")
    check(here == in_record,
          f"length {p}: the counts of +1, -1 and 0 equal the record",
          record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, check "
                 f"gkd_equals_kdelta_exhaustive_length_{p}")
report("pairs of length 1, 2, 3 compared", total_pairs)
exhaustive = [wolfram_checks[f"gkd_equals_kdelta_exhaustive_length_{p}"]
              for p in (1, 2, 3)]  # the record's three exhaustive comparisons
check(total_pairs == 266304 and mismatches == 0 and all(
      c["verdict"] == "PASS" and "(0 mismatches;" in c["detail"] for c in exhaustive),
      "literal determinant = fast rule for all 266304 pairs of length 1, 2, 3",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks "
             "gkd_equals_kdelta_exhaustive_length_1 to 3 (0 mismatches)")
```

Out [8]:

```
length 1: 64 pairs
    counts of (+1, -1, 0): here (8, 0, 56), record (8, 0, 56)
PASS length 1: the counts of +1, -1 and 0 equal the record
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, check
    gkd_equals_kdelta_exhaustive_length_1
length 2: 4096 pairs
    counts of (+1, -1, 0): here (56, 56, 3984), record (56, 56, 3984)
PASS length 2: the counts of +1, -1 and 0 equal the record
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, check
    gkd_equals_kdelta_exhaustive_length_2
length 3: 262144 pairs
    counts of (+1, -1, 0): here (1008, 1008, 260128), record (1008, 1008, 260128)
PASS length 3: the counts of +1, -1 and 0 equal the record
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, check
    gkd_equals_kdelta_exhaustive_length_3
RESULT pairs of length 1, 2, 3 compared = 266304
PASS literal determinant = fast rule for all 266304 pairs of length 1, 2, 3
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks
    gkd_equals_kdelta_exhaustive_length_1 to 3 (0 mismatches)
```

Text cell:

```
## 11. All 16,777,216 pairs of length 4

For length 4 there are $8^8$ = 16,777,216 pairs, too many for a plain Python
loop. The next cell writes both functions again so that numpy treats a whole
block of pairs at once (one index list per row of an array):

- ‘literal_many‘ builds all the matrices ‘Outer‘ of the block at once (an array of
  True and False of shape pairs $\times p \times p$) and adds the Leibniz terms:
  for each permutation, the term is not zero exactly when all its entries are 1;
- ‘rule_many‘ uses a shortcut for the sign. If the lower list $L$ is a re-ordering
  of the upper list $U$, both are re-orderings of the same sorted list $S$. The
  matrix $M_{ij} = \delta(l_i, u_j)$ is then the product $P_L P_U^T$ of the two
  permutation matrices with $(P_L)_{ik} = \delta(l_i, s_k)$ and
  $(P_U)_{jk} = \delta(u_j, s_k)$ (the sum over $k$ has one nonzero term), so
  $\det M = \det P_L \det P_U = \mathrm{sign}(L)\,\mathrm{sign}(U)$, where
  $\mathrm{sign}(L)$ counts the inversions of the list $L$ itself. The other
  cases give 0: a repeated upper label, or different sorted lists.

Before it is trusted, the cell checks both new functions against the plain values
of section 10 for all 266,304 pairs of length 1, 2 and 3.
```

In [9]:

```
def all_lists(p):
    """Every index list of length p over the labels 0 ... 7, one per row."""
    return np.array(list(itertools.product(range(8), repeat=p)), dtype=np.int8)


def literal_many(lowers, uppers):
    """The author’s definition for many pairs at once (exact whole numbers)."""
```

```
    p = lowers.shape[1]
    outer = lowers[:, :, None] == uppers[:, None, :]  # shape (pairs, p, p)
    values = np.zeros(len(lowers), dtype=np.int64)
    rows = np.arange(p)
    for order, order_sign in signed_permutations(p):
        # True where the entries (i, order[i]) of the matrix are all 1
        values += order_sign * outer[:, rows, list(order)].all(axis=1)
    return values


def rule_many(lowers, uppers):
    """The fast rule for many pairs at once."""
    p = lowers.shape[1]
    upper_sorted = np.sort(uppers, axis=1)
    upper_distinct = (np.diff(upper_sorted, axis=1) != 0).all(axis=1)
    same_labels = (np.sort(lowers, axis=1) == upper_sorted).all(axis=1)
    first, second = np.triu_indices(p, 1)  # all pairs of places first < second
    inversions_lower = (lowers[:, first] > lowers[:, second]).sum(axis=1)
    inversions_upper = (uppers[:, first] > uppers[:, second]).sum(axis=1)
    signs = np.where((inversions_lower + inversions_upper) % 2 == 0, 1, -1)
    return np.where(upper_distinct & same_labels, signs, 0)


agree = True
for p in (1, 2, 3):
    lists = all_lists(p)
    lowers = np.repeat(lists, len(lists), axis=0)  # each lower list 8^p times
    uppers = np.tile(lists, (len(lists), 1))  # all upper lists, again and again
    agree &= bool((literal_many(lowers, uppers) == plain_values[p]).all())
    agree &= bool((rule_many(lowers, uppers) == plain_values[p]).all())
check(agree, "the block functions agree with the plain ones on all pairs of "
      "length 1, 2, 3")
```

Out [9]:

```
PASS the block functions agree with the plain ones on all pairs of length 1, 2, 3
```

Text cell:

```
The next cell runs through all 16,777,216 pairs of length 4 in 16 blocks of
256 lower lists times all 4096 upper lists (1,048,576 pairs per block),
computes the literal determinant and the fast rule for each pair, counts the
disagreements and tallies the values. The record's self-test did exactly this
with the Rust program; its file gkd-selftest.json gives the number of pairs and
of mismatches. The cell takes 10 to 30 seconds.
```

In [10]:

```
selftest = read_report("gkd-selftest.json")
recorded_selftest = {entry["p"]: entry for entry in selftest["results"]}
lists4 = all_lists(4)  # 4096 lists
```

```
tally4 = {1: 0, -1: 0, 0: 0}
mismatches4 = 0
for start in range(0, 4096, 256):
    lowers = np.repeat(lists4[start:start + 256], 4096, axis=0)
    uppers = np.tile(lists4, (256, 1))
    literal = literal_many(lowers, uppers)
    fast = rule_many(lowers, uppers)
    mismatches4 += int((literal != fast).sum())
    for v in (1, -1, 0):
        tally4[v] += int((fast == v).sum())
tallies[4] = tally4
pairs4 = sum(tally4.values())
say(f"length 4: {pairs4} pairs; +1: {tally4[1]}, -1: {tally4[-1]}, 0: {tally4[0]}; "
    f"mismatches {mismatches4}")
report("length 4, counts of the values +1, -1, 0",
       f"{tally4[1]}, {tally4[-1]}, {tally4[0]}")
for p in (1, 2, 3, 4):
    mode, pairs_p, wrong = (recorded_selftest[p]["mode"], recorded_selftest[p]["pairs"],
                            recorded_selftest[p]["mismatches"])
    say(f"record gkd-selftest.json, length {p}: {mode}, {pairs_p} pairs, {wrong} "
        "mismatches")
check(pairs4 == recorded_selftest[4]["pairs"] == 16777216 and
      mismatches4 == recorded_selftest[4]["mismatches"] == 0,
      "all 16777216 pairs of length 4: literal determinant = fast rule",
      record="Revision/gkd_lovelock/results/gkd-selftest.json, length 4 "
             "(exhaustive)")
check(all(recorded_selftest[p]["pairs"] == 8 ** (2 * p) and
          recorded_selftest[p]["mismatches"] == 0 for p in (1, 2, 3)),
      "the record's exhaustive self-tests of length 1, 2, 3 cover 8^(2p) pairs "
      "with no mismatch, as found in section 10",
      record="Revision/gkd_lovelock/results/gkd-selftest.json, lengths 1 to 3")
```

Out [10]:

```
length 4: 16777216 pairs; +1: 20160, -1: 20160, 0: 16736896; mismatches 0
RESULT length 4, counts of the values +1, -1, 0 = 20160, 20160, 16736896
record gkd-selftest.json, length 1: exhaustive, 64 pairs, 0 mismatches
record gkd-selftest.json, length 2: exhaustive, 4096 pairs, 0 mismatches
record gkd-selftest.json, length 3: exhaustive, 262144 pairs, 0 mismatches
record gkd-selftest.json, length 4: exhaustive, 16777216 pairs, 0 mismatches
PASS all 16777216 pairs of length 4: literal determinant = fast rule
     reproduces Revision/gkd_lovelock/results/gkd-selftest.json, length 4 (exhaustive)
PASS the record's exhaustive self-tests of length 1, 2, 3 cover 8^(2p) pairs with no
    mismatch, as found in section 10
     reproduces Revision/gkd_lovelock/results/gkd-selftest.json, lengths 1 to 3
```

Text cell:

```
The next cell draws the counts of the values $+1$, $-1$ and $0$ for the lengths
1 to 4 as bars on a logarithmic scale (each step of the scale multiplies by 10).
Almost all values are 0, and the number of $+1$ equals the number of $-1$ from
length 2 on.
```

In [11]:

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
lengths = np.arange(1, 5)
for shift, value, color, name in [(-0.27, 1, "tab:red", "value +1"),
                                  (0.0, -1, "tab:blue", "value -1"),
                                  (0.27, 0, "0.6", "value 0")]:
    counts = [max(tallies[p][value], 0.8) for p in lengths]  # 0.8 marks a count 0
    ax.bar(lengths + shift, counts, width=0.25, color=color, label=name)
    for x, count in zip(lengths + shift, counts):
        ax.text(x, count * 1.25, f"{count:.0f}" if count >= 1 else "0",
                ha="center", fontsize=7, rotation=90)
ax.set_yscale("log")
ax.set_ylim(0.5, 2e9)
ax.set_xticks(lengths)
ax.set_xlabel("length $p$ of the two index lists")
ax.set_ylabel("number of pairs (logarithmic scale)")
ax.set_title("Values of the generalized delta over 8 labels")
ax.legend(loc="upper left");
save_figure(fig, "value_counts",
            "The number of pairs of index lists of length $p = 1$ to 4 over the "
            "eight labels $x_0$ to $x_7$ for which the generalized delta is $+1$ "
            "(red), $-1$ (blue) and 0 (grey), counted over all $8^{2p}$ pairs; "
            "horizontal axis $p$, vertical axis the number of pairs on a "
            "logarithmic scale, with the exact counts written above the bars "
            "(a count 0 is drawn as a short stub). The counts for $p = 1$ to 3 "
            "equal those of the Revision record.")
```

Out [11]:

```
Figure 01c.3 saved as Revision/textbook/figures/01c_3_value_counts.png
```

Text cell:

```
## 12. How many values are not zero: a formula

A value is not zero exactly in case (d): the upper list consists of $p$
different labels, and the lower list is a re-ordering of it. There are
$n(n-1)\cdots(n-p+1) = n!/(n-p)!$ upper lists of $p$ different labels from $n$
(the first label has $n$ choices, the second $n - 1$, and so on), and $p!$
re-orderings of each. So

$$N_{\neq 0}(n, p) = \frac{n!}{(n-p)!} \, p! \quad (p \leq n), \qquad
N_{\neq 0}(n, p) = 0 \quad (p > n).$$

For $p \geq 2$ half of the re-orderings are even, so $+1$ and $-1$ occur equally
often; for $p = 1$ every nonzero value is $+1$. The next cell checks the formula
against the counts of sections 9 to 11, prints it for $p = 1$ to 9, and plots the
total number of pairs $n^{2p}$ and the nonzero ones for $n = 8$.
```

In [12]:

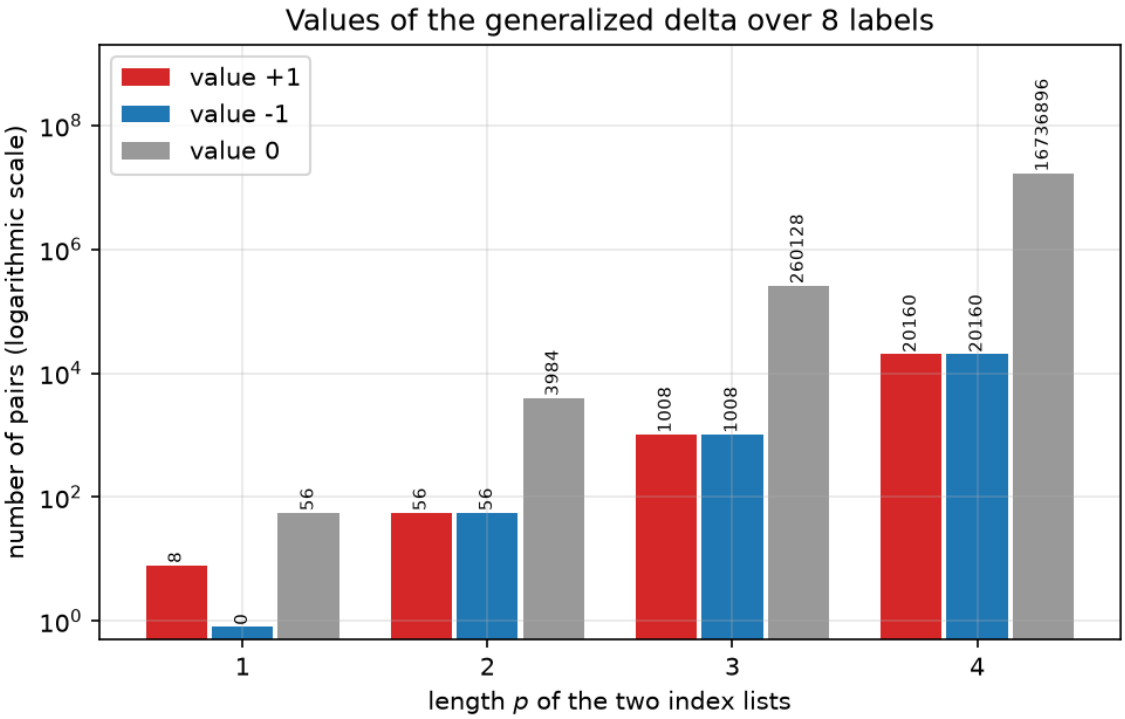

Figure 52: The number of pairs of index lists of length $p = 1$ to 4 over the eight labels x_0 to x_7 for which the generalized delta is $+1$ (red), -1 (blue) and 0 (grey), counted over all 8^{2p} pairs; horizontal axis p, vertical axis the number of pairs on a logarithmic scale, with the exact counts written above the bars (a count 0 is drawn as a short stub). The counts for $p = 1$ to 3 equal those of the Revision record. (Notebook 01c, figure 3.)

```
def nonzero_count(n, p):
    """The number of pairs of index lists of length p over n labels with value +-1."""
    return math.perm(n, p) * math.factorial(p) if p <= n else 0


formula_ok = tallies[1][1] == 8 and tallies[1][-1] == 0
for p in (2, 3, 4):
    formula_ok &= tallies[p][1] == tallies[p][-1] == nonzero_count(8, p) // 2
check(formula_ok, "the counting formula n!/(n-p)! p! gives the counts of lengths 1 to 4")
for p in range(1, 10):
    total = 8 ** (2 * p)
    say(f"p = {p}: {total:20d} pairs, {nonzero_count(8, p):11d} not zero, "
        f"fraction {nonzero_count(8, p) / total:.3e}")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
lengths = np.arange(1, 9)
left.semilogy(lengths, [8.0 ** (2 * p) for p in lengths], "s-",
              label="all pairs $8^{2p}$")
left.semilogy(lengths, [nonzero_count(8, p) for p in lengths], "o-",
              label="pairs with value $\\pm 1$")
left.set_xlabel("length $p$")
left.set_ylabel("number of pairs (logarithmic scale)")
left.set_title("for $p = 9$ no pair is left")
left.legend()
right.semilogy(lengths, [nonzero_count(8, p) / 8.0 ** (2 * p) for p in lengths], "o-",
               color="tab:purple")
right.set_xlabel("length $p$")
right.set_ylabel("fraction of pairs with value $\\pm 1$")
right.set_title("the nonzero values become rare")
fig.tight_layout()
# The fraction for p = 8 written as "5.8 \times 10^{-6}" for the caption:
mantissa, exponent = f"{nonzero_count(8, 8) / 8 ** 16:.1e}".split("e")
save_figure(fig, "nonzero_fraction",
            "Left: the number of all pairs of index lists of length $p$ over 8 "
            "labels, $8^{2p}$ (squares), and of the pairs whose generalized delta "
            "is not zero, $8!/(8-p)! \\cdot p!$ (circles), for $p = 1$ to 8, on a "
            "logarithmic scale; for $p = 9$ the second number is 0. Right: their "
            "ratio, the fraction of nonzero values, which falls from $1/8$ at "
            f"$p = 1$ to about ${mantissa} \\times 10^{{{int(exponent)}}}$ at "
            "$p = 8$; horizontal axes the length $p$.")
```

Out [12]:

```
PASS the counting formula n!/(n-p)! p! gives the counts of lengths 1 to 4
p = 1:                   64 pairs,           8 not zero, fraction 1.250e-01
p = 2:                 4096 pairs,         112 not zero, fraction 2.734e-02
p = 3:               262144 pairs,        2016 not zero, fraction 7.690e-03
p = 4:             16777216 pairs,       40320 not zero, fraction 2.403e-03
p = 5:           1073741824 pairs,      806400 not zero, fraction 7.510e-04
p = 6:          68719476736 pairs,    14515200 not zero, fraction 2.112e-04
p = 7:        4398046511104 pairs,   203212800 not zero, fraction 4.621e-05
p = 8:      281474976710656 pairs,  1625702400 not zero, fraction 5.776e-06
p = 9:    18014398509481984 pairs,           0 not zero, fraction 0.000e+00
```

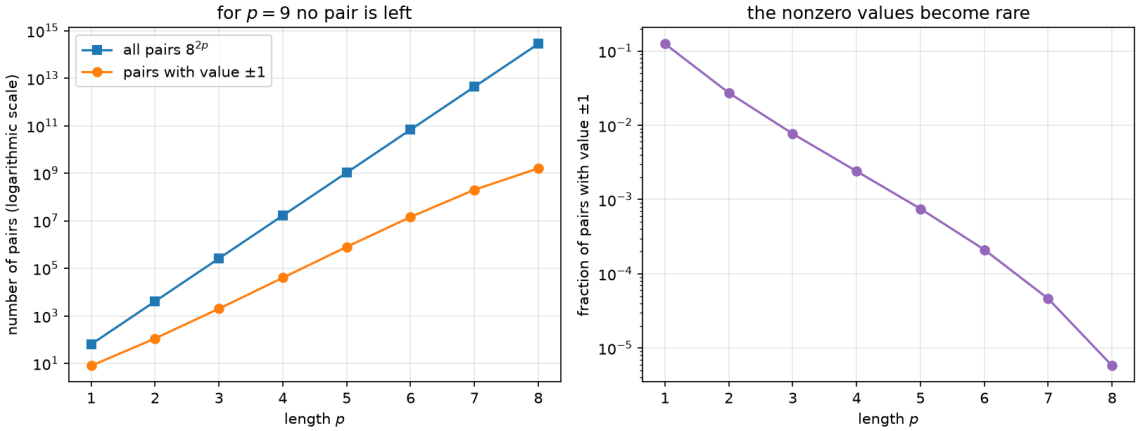

Figure 53: Left: the number of all pairs of index lists of length p over 8 labels, 8^{2p} (squares), and of the pairs whose generalized delta is not zero, $8!/(8-p)! \cdot p!$ (circles), for $p = 1$ to 8, on a logarithmic scale; for $p = 9$ the second number is 0. Right: their ratio, the fraction of nonzero values, which falls from $1/8$ at $p = 1$ to about 5.8×10^{-6} at $p = 8$; horizontal axes the length p. (Notebook 01c, figure 4.)

Out [12]:

```
Figure 01c.4 saved as Revision/textbook/figures/01c_4_nonzero_fraction.png
```

Text cell:

```
## 13. Nine indices in eight dimensions

A list of 9 labels taken from only 8 must repeat a label (pigeonhole principle),
so case (a) or (b) applies and the value is 0, for every one of the $8^{18}$
pairs. This is why the Lovelock sum of the author's theory stops at $k = 3$: the
term $k = 4$ would need $2k + 1 = 9$ indices. The next cell confirms it on 300
pairs of random lists of 9 labels (numpy's random generator with the seed 12345)
and on 300 pairs in which the lower list is a re-ordering of the upper one; here
the literal determinant of the $9 \times 9$ matrix (which would have $9!$ =
362,880 Leibniz terms) is computed with numpy's elimination. With 8 labels a
nonzero value is still possible: the cell prints the value for the list
$(x_0, \dots, x_7)$ against its reverse, which has 28 inversions.
```

In [13]:

```
generator = np.random.default_rng(12345)
nine_all_zero = True
for sample in range(600):
    upper = tuple(int(x) for x in generator.integers(0, 8, size=9))
    if sample % 2 == 0:  # an arbitrary lower list
        lower = tuple(int(x) for x in generator.integers(0, 8, size=9))
    else:  # a re-ordering of the upper list
        lower = tuple(int(x) for x in generator.permutation(np.array(upper)))
```

```
    determinant = np.linalg.det(np.array(outer_matrix(lower, upper), dtype=float))
    nine_all_zero &= gkd_rule(lower, upper) == 0 and abs(determinant) < 1e-9
check(nine_all_zero, "600 pairs of lists of 9 labels from 8: every value is 0",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "gkd_nine_indices_in_eight_dimensions_vanish (with our own random lists)")
eight = tuple(range(8))
reverse = eight[::-1]  # (7, 6, ..., 0)
say(f"8 labels against their reverse: {inversions(reverse)} inversions, value "
    f"{gkd_rule(eight, reverse):+d}")
check(gkd_rule(eight, reverse) == 1 and inversions(reverse) == 28,
      "with 8 labels a nonzero value is possible (the reverse order has 28 "
      "inversions, value +1)")
```

Out [13]:

```
PASS 600 pairs of lists of 9 labels from 8: every value is 0
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    gkd_nine_indices_in_eight_dimensions_vanish (with our own random lists)
8 labels against their reverse: 28 inversions, value +1
PASS with 8 labels a nonzero value is possible (the reverse order has 28 inversions,
    value +1)
```

Text cell:

```
## 14. Contracting one index: the factor $n - p + 1$

Take lists $A = (a_1, \dots, a_{p-1})$ and $B = (b_1, \dots, b_{p-1})$, append the
same label $c$ to both, and add over all $n$ values of $c$. The claim is

$$\sum_{c} \delta^{a_1 \dots a_{p-1} c}_{b_1 \dots b_{p-1} c}
= (n - p + 1)\, \delta^{a_1 \dots a_{p-1}}_{b_1 \dots b_{p-1}}.$$

Proof by cases. If $A$ or $B$ repeats a label, every term repeats it too: all
terms are 0, and so is the right side. If $A$ and $B$ consist of different labels
but are not re-orderings of each other, then for each $c$ the extended lists are
not re-orderings of each other either (if $c$ is among the labels of $A$ or $B$
it is repeated; if not, adding it to both changes nothing): all terms are 0. If
$B$ is a re-ordering $\sigma$ of $A$: for the $p - 1$ values of $c$ that are among
the labels, the extended lists repeat $c$ (term 0); for each of the other
$n - (p - 1)$ values, the extended lists are re-ordered by $\sigma$ with $c$ left
in place, which has the same inversions, so each such term equals
$\mathrm{sign}(\sigma)$. Together: $(n - p + 1)\,\mathrm{sign}(\sigma)$.

The next cell checks this for $n = 8$ and $p = 1$ to 8 (exhaustively up to
$p = 4$, with the block function for $p = 4$; for $p = 5$ to 8 on 4000 random
pairs, half of them re-orderings) and for $n = 4$ and $p = 1$ to 4
(exhaustively). For each $(n, p)$ it records the factor found; for $p = 1$ the
empty lists have the value 1 (the determinant of the empty matrix).
```

In [14]:

```
def contraction_factors(n, p, pairs_of_lists):
    """The set of the ratios sum_c delta(A + c, B + c) / delta(A, B) over the pairs
    with delta(A, B) != 0; returns None if some pair breaks the identity."""
    found_factors = set()
    for lower, upper in pairs_of_lists:
        total = sum(gkd_rule(lower + (c,), upper + (c,)) for c in range(n))
        base = gkd_rule(lower, upper)
        if base == 0:
            if total != 0:
                return None
        else:
            found_factors.add(total // base)
            if total != (n - p + 1) * base:
                return None
    return found_factors


factors = {}  # (n, p) -> the factor found
for n in (4, 8):
    for p in range(1, n + 1):
        q = p - 1  # the length of the lists A and B
        if (n == 8 and p <= 3) or n == 4:  # every pair of lists, plain loops
            lists = list(itertools.product(range(n), repeat=q))
            pairs_q = [(lo, up) for lo in lists for up in lists]
        elif n == 8 and p == 4:  # every pair, with the block function
            lists = all_lists(3)
            lowers = np.repeat(lists, len(lists), axis=0)
            uppers = np.tile(lists, (len(lists), 1))
            base = rule_many(lowers, uppers)  # delta(A, B) for all 262144 pairs
            total = sum(rule_many(np.column_stack([lowers, np.full(len(lowers), c)]),
                                  np.column_stack([uppers, np.full(len(uppers), c)]))
                        for c in range(8))  # the sum over c, for all pairs at once
            nonzero = base != 0
            ratios = np.unique(total[nonzero] // base[nonzero])  # base is +1 or -1
            identity_holds = (bool((total[~nonzero] == 0).all()) and len(ratios) == 1
                              and bool((total == (n - p + 1) * base).all()))
            factors[(n, p)] = int(ratios[0]) if identity_holds else None
            continue
        else:  # random pairs: half re-orderings of different labels, half arbitrary
            pairs_q = []
            for sample in range(4000):
                if sample % 2 == 0:
                    upper = tuple(int(x) for x in generator.permutation(n)[:q])
                    lower = tuple(int(x) for x in generator.permutation(np.array(upper)))
                else:
                    upper = tuple(int(x) for x in generator.integers(0, n, size=q))
                    lower = tuple(int(x) for x in generator.integers(0, n, size=q))
                pairs_q.append((lower, upper))
        found = contraction_factors(n, p, pairs_q)
        factors[(n, p)] = found.pop() if found and len(found) == 1 else None
say("factors found for n = 8, p = 1 ... 8: "
    + ", ".join(str(factors[(8, p)]) for p in range(1, 9)))
say("factors found for n = 4, p = 1 ... 4: "
    + ", ".join(str(factors[(4, p)]) for p in range(1, 5)))
```

```
report("contraction factors for 8 labels, p = 1 to 8",
       ", ".join(str(factors[(8, p)]) for p in range(1, 9)))
check(all(factors[(n, p)] == n - p + 1 for n in (4, 8) for p in range(1, n + 1)),
      "contracting one index multiplies by n - p + 1 (n = 4 and 8, every p)")
```

Out [14]:

```
factors found for n = 8, p = 1 ... 8: 8, 7, 6, 5, 4, 3, 2, 1
factors found for n = 4, p = 1 ... 4: 4, 3, 2, 1
RESULT contraction factors for 8 labels, p = 1 to 8 = 8, 7, 6, 5, 4, 3, 2, 1
PASS contracting one index multiplies by n - p + 1 (n = 4 and 8, every p)
```

Text cell:

```
For the Lovelock tensor of order $k$ the delta has $p = 2k + 1$ indices, and the
trace $\sum_h P_{(k)}{}^h{}_h$ contracts the first upper with the first lower
index (moving them to the end of both lists re-orders rows and columns in the
same way, which changes the sign twice, that is not at all). With $n = 8$ the
factor is $n - p + 1 = 8 - 2k$: 6, 4 and 2 for $k = 1, 2, 3$. The Wolfram report
of the record names its trace checks after these factors
(‘k1_trace_equals_6_L1‘ and so on). The next cell reads the names, takes the
factors out of them and compares them with the factors just found.
```

In [15]:

```
trace_checks = [c for c in wolfram_report["checks"]
                if re.fullmatch(r"k\d_trace_equals_\d_L\d", c["name"])]
named = {}
for c in trace_checks:
    name, verdict = c["name"], c["verdict"]
    k_text, factor_text = re.fullmatch(r"k(\d)_trace_equals_(\d)_L\d", name).groups()
    k = int(k_text)
    named[k] = int(factor_text)
    say(f"record check {name}: {verdict}; factor for k = {k}: {factor_text}; "
        f"found here for p = {2 * k + 1}: {factors[(8, 2 * k + 1)]}")
check(named == {1: 6, 2: 4, 3: 2} and
      all(factors[(8, 2 * k + 1)] == named[k] for k in (1, 2, 3)) and
      all(c["verdict"] == "PASS" for c in trace_checks),
      "the factors 8 - 2k = 6, 4, 2 of the Lovelock trace identities",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks "
             "k1_trace_equals_6_L1, k2_trace_equals_4_L2, k3_trace_equals_2_L3")
```

Out [15]:

```
record check k1_trace_equals_6_L1: PASS; factor for k = 1: 6; found here for p = 3: 6
record check k2_trace_equals_4_L2: PASS; factor for k = 2: 4; found here for p = 5: 4
record check k3_trace_equals_2_L3: PASS; factor for k = 3: 2; found here for p = 7: 2
PASS the factors 8 - 2k = 6, 4, 2 of the Lovelock trace identities
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks
    k1_trace_equals_6_L1, k2_trace_equals_4_L2, k3_trace_equals_2_L3
```

Text cell:

```
The next cell plots the factors found against $p$ for $n = 8$ and $n = 4$,
together with the straight lines $n - p + 1$, and marks the three values of the
Lovelock trace identities.
```

In [16]:

```
fig, ax = plt.subplots(figsize=(7.2, 4.4))
for n, marker, color in ((8, "o", "tab:blue"), (4, "s", "tab:orange")):
    p_values = np.arange(1, n + 1)
    ax.plot(p_values, n - p_values + 1, "-", color=color, lw=1,
            label=f"$n - p + 1$ for $n = {n}$")
    ax.plot(p_values, [factors[(n, p)] for p in p_values], marker, color=color,
            ms=8, fillstyle="none", label=f"factors found, $n = {n}$")
for k in (1, 2, 3):
    ax.annotate(f"$k = {k}$: $8 - 2k = {8 - 2 * k}$", xy=(2 * k + 1, 8 - 2 * k),
                xytext=(2 * k + 1.3, 8 - 2 * k + 1.3), fontsize=9,
                arrowprops={"arrowstyle": "->", "color": "black"})
ax.set_xlabel("number $p$ of indices before the contraction")
ax.set_ylabel("factor")
ax.set_title("Contracting one index of the generalized delta")
ax.legend(loc="upper right", fontsize=8);
save_figure(fig, "contraction_factor",
            "The factor by which contracting one upper with one lower index "
            "multiplies the generalized delta with $p$ indices, found by the checks "
            "of this notebook (open circles for $n = 8$ labels, open squares for "
            "$n = 4$), and the lines $n - p + 1$; horizontal axis $p$, vertical "
            "axis the factor. The arrows mark $p = 2k + 1 = 3, 5, 7$ in eight "
            "dimensions, where the factors $8 - 2k = 6, 4, 2$ are those of the "
            "Lovelock trace identities of the Revision record.")
```

Out [16]:

```
Figure 01c.5 saved as Revision/textbook/figures/01c_5_contraction_factor.png
```

Text cell:

```
## 15. The Levi-Civita symbol and the author's second route

The Levi-Civita symbol $\varepsilon_{i_1 \dots i_n}$ is the generalized delta
with the upper list $(0, 1, \dots, n - 1)$: by case (d) it is the sign of the
list if the list is a re-ordering of all $n$ labels, and 0 otherwise. The
author's notebook builds the generalized delta also from two of them (input cell
In[32]: `epsilong[-a, -f, ...] epsilong[b, f, ...] / (8 - 1)!`). The general
formula is

$$\sum_{c_{p+1}, \dots, c_n} \varepsilon_{a_1 \dots a_p c_{p+1} \dots c_n}\,
\varepsilon_{b_1 \dots b_p c_{p+1} \dots c_n}
= (n - p)!\; \delta^{b_1 \dots b_p}_{a_1 \dots a_p}.$$
```

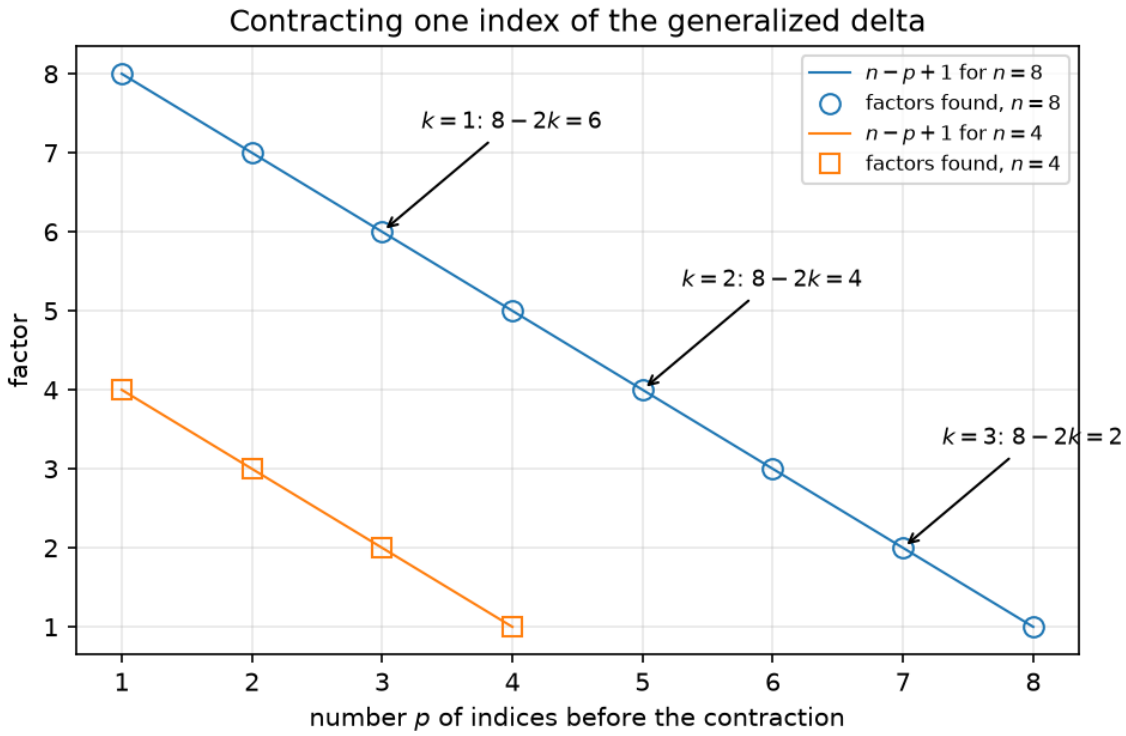

Figure 54: The factor by which contracting one upper with one lower index multiplies the generalized delta with p indices, found by the checks of this notebook (open circles for $n = 8$ labels, open squares for $n = 4$), and the lines $n - p + 1$; horizontal axis p, vertical axis the factor. The arrows mark $p = 2k + 1 = 3, 5, 7$ in eight dimensions, where the factors $8 - 2k = 6, 4, 2$ are those of the Lovelock trace identities of the Revision record. (Notebook 01c, figure 5.)

```
It is the contraction identity of section 14 used $n - p$ times, starting from
$\varepsilon_{a_1 \dots a_n} \varepsilon_{b_1 \dots b_n}
= \delta^{b_1 \dots b_n}_{a_1 \dots a_n}$ (both sides are the product of the
signs of the two lists, or 0), with the factors $1, 2, \dots, n - p$.

In the author's coordinates the second $\varepsilon$ has upper indices, raised
with the metric; raising all $n$ indices of $\varepsilon$ multiplies it by the
product of the diagonal entries of $\eta^{-1}$, which is $\det \eta = +1$ for the
four $+1$ and four $-1$ of $\eta$ (in four-dimensional spacetime, with one $-1$,
the same step gives a factor $-1$). The author's notebook uses the Levi-Civita
tensor 'epsilong' of a metric $g$ that it declares in the cell In[29] with
'DefMetric[{4, 4, 0}, ...]': four positive, four negative and no zero
eigenvalues, the signature (4,4). For that tensor the factor is the sign of
$\det g$, which is $(+1)^4 (-1)^4 = +1$, the same as for $\eta$ (and for the
author's metric of the Revision record, $\det g = \cos^2 z > 0$).
The next cell reads $\eta$ from the Revision record and checks
$\det\eta = +1$; checks the declaration (4, 4, 0) in the cell In[29]; checks
the formula for $n = 4$ and every $p = 0$ to 4 over all lists; and repeats the
author's In[32] for $n = 8$ and $p = 1$: for all 64 pairs $(a, b)$ it adds the
products over all lists $(c_2, \dots, c_8)$ of 7 different labels and divides by
$7! = 5040$; the result must be the $8 \times 8$ identity matrix $\delta^b_a$.
```

In [17]:

```
def levi_civita(indices, n):
    """epsilon_(indices): the generalized delta with the upper list 0, 1, ..., n-1."""
    return gkd_rule(tuple(indices), tuple(range(n)))


eta = json.loads(repository_file("Revision/algebra/gammas.json")
                 .read_text(encoding="utf-8"))["eta"]
say(f"diagonal of eta (x0 ... x7) from the record: {eta}; det eta = {math.prod(eta)}")
check(math.prod(eta) == 1, "det eta = +1: raising all 8 indices of epsilon gives no "
      "extra sign in the signature (4,4)")
# (4, 4, 0): 4 positive, 4 negative, 0 zero eigenvalues; sign of det g = (-1)^4
signature = re.search(r"DefMetric\[\{(\d), (\d), (\d)\}", metric_cell)
positive, negative, zero = (int(part) for part in signature.groups())
say(f"the author's metric g: {positive} positive, {negative} negative, {zero} zero "
    f"eigenvalues; sign of det g = (-1)^{negative} = {(-1) ** negative:+d}")
check((positive, negative, zero) == (4, 4, 0) and (-1) ** negative == 1,
      "the author's notebook declares its metric with the signature (4,4), so the "
      "product of two Levi-Civita tensors has the sign +1",
      record="Revision/gkd_lovelock/results/notebook-input-cells.txt, In[29]")
formula_ok = True
for p in range(0, 5):  # n = 4, every length p of the lists a and b
    lists = list(itertools.product(range(4), repeat=p))
    for a in lists:
        for b in lists:
            total = sum(levi_civita(a + c, 4) * levi_civita(b + c, 4)
                        for c in itertools.product(range(4), repeat=4 - p))
            formula_ok &= total == math.factorial(4 - p) * gkd_rule(a, b)
check(formula_ok, "n = 4: sum of epsilon epsilon = (n - p)! delta for p = 0 to 4, "
```

```
      "all lists")
route_sums = np.zeros((8, 8), dtype=np.int64)
for a in range(8):
    others = [label for label in range(8) if label != a]
    for b in range(8):
        # Only lists c of 7 different labels other than a give a nonzero first factor.
        route_sums[a, b] = sum(levi_civita((a,) + c, 8) * levi_civita((b,) + c, 8)
                               for c in itertools.permutations(others))
author_route = route_sums // math.factorial(7)  # divide by 7! = 5040
say(f"the sums for a = b are {sorted(set(np.diag(route_sums).tolist()))}, "
    f"for a different from b {sorted(set(route_sums[~np.eye(8, dtype=bool)].tolist()))}")
check((route_sums == math.factorial(7) * np.eye(8, dtype=np.int64)).all(),
      "the author's route In[32]: sum of epsilon epsilon / 7! = delta^b_a, the "
      "8 x 8 identity")
```

Out [17]:

```
diagonal of eta (x0 ... x7) from the record: [1, 1, 1, 1, -1, -1, -1, -1]; det eta = 1
PASS det eta = +1: raising all 8 indices of epsilon gives no extra sign in the signature
    (4,4)
the author's metric g: 4 positive, 4 negative, 0 zero eigenvalues; sign of det g = (-1)^4
    = +1
PASS the author's notebook declares its metric with the signature (4,4), so the product
    of two Levi-Civita tensors has the sign +1
     reproduces Revision/gkd_lovelock/results/notebook-input-cells.txt, In[29]
PASS n = 4: sum of epsilon epsilon = (n - p)! delta for p = 0 to 4, all lists
the sums for a = b are [5040], for a different from b [0]
PASS the author's route In[32]: sum of epsilon epsilon / 7! = delta^b_a, the 8 x 8
    identity
```

Text cell:

```
The next cell draws the Levi-Civita symbol of three labels as three $3 \times 3$
heat maps (one for each value of the first index; the six nonzero entries are the
signs of the six permutations) and the $8 \times 8$ result of the author's route,
the identity matrix $\delta^b_a$.
```

In [18]:

```
fig, axes = plt.subplots(1, 4, figsize=(12.0, 3.5),
                         gridspec_kw={"width_ratios": [1, 1, 1, 1.5]})
names3 = ["$x_1$", "$x_2$", "$x_3$"]
for first in range(3):
    ax = axes[first]
    slice_values = np.array([[levi_civita((first, j, k), 3) for k in range(3)]
                             for j in range(3)])
    ax.imshow(slice_values, cmap="RdBu_r", vmin=-1, vmax=1)
    ax.grid(False)
    for j in range(3):
        for k in range(3):
            ax.text(k, j, signed(int(slice_values[j, k])), ha="center", va="center",
```

```
                    color="white" if slice_values[j, k] else "black")
    ax.set_xticks(range(3), names3)
    ax.set_yticks(range(3), names3)
    ax.set_title(f"$\\varepsilon$ with first index $x_{first + 1}$", fontsize=9)
image = axes[3].imshow(author_route, cmap="RdBu_r", vmin=-1, vmax=1)
axes[3].grid(False)
eight_names = [f"$x_{k}$" for k in range(8)]
axes[3].set_xticks(range(8), eight_names, fontsize=7)
axes[3].set_yticks(range(8), eight_names, fontsize=7)
axes[3].set_xlabel("$b$")
axes[3].set_ylabel("$a$")
axes[3].set_title("$\\sum \\varepsilon\\varepsilon / 7!$ in 8 dimensions", fontsize=9)
fig.tight_layout()
save_figure(fig, "levi_civita",
            "Left three panels: the Levi-Civita symbol $\\varepsilon_{ijk}$ of the "
            "three labels $x_1, x_2, x_3$, one panel for each first index $i$ "
            "(rows $j$, columns $k$; red $+1$, blue $-1$, white 0); its six "
            "nonzero entries are the signs of the six orderings. Right: the "
            "author's second route in eight dimensions, the sum over $c_2$ to $c_8$ "
            "of $\\varepsilon_{a c_2 \\dots c_8} \\varepsilon_{b c_2 \\dots c_8}$ "
            "divided by $7!$, for all 64 pairs of labels $a$ (rows) and $b$ "
            "(columns): exactly the identity matrix, the ordinary Kronecker delta.")
```

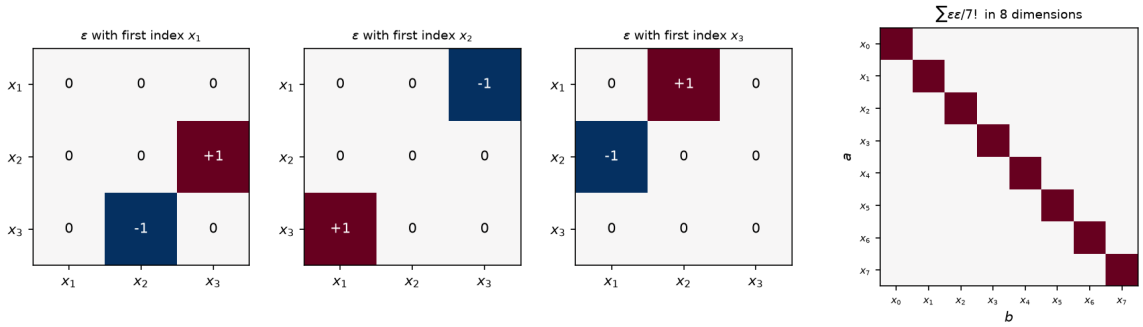

Figure 55: Left three panels: the Levi-Civita symbol ε_{ijk} of the three labels x_1, x_2, x_3, one panel for each first index i (rows j, columns k; red $+1$, blue -1, white 0); its six nonzero entries are the signs of the six orderings. Right: the author's second route in eight dimensions, the sum over c_2 to c_8 of $\varepsilon_{ac_2\dots c_8}\varepsilon_{bc_2\dots c_8}$ divided by 7!, for all 64 pairs of labels a (rows) and b (columns): exactly the identity matrix, the ordinary Kronecker delta. (Notebook 01c, figure 6.)

Out [18]:

```
Figure 01c.6 saved as Revision/textbook/figures/01c_6_levi_civita.png
```

Text cell:

```
## 16. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [19]:

```
figure_names = ["01c_1_outer_matrices.png", "01c_2_two_index_table.png",
                "01c_3_value_counts.png", "01c_4_nonzero_fraction.png",
                "01c_5_contraction_factor.png", "01c_6_levi_civita.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

Out [19]:

```
PASS all 6 figure files of this notebook exist
ALL 28 CHECKS PASSED (notebook 01c)
```

Text cell:

```
## 17. What this notebook showed

- The author's generalized Kronecker delta, `Det[Outer[delta, lower, upper]]`,
  written literally in Python, equals the fast rule of the Revision program GKD
  (0 for a repeated or missing label, otherwise the sign of a permutation) for
  every pair of index lists of length 1 to 4 over the eight coordinates
  $x_0, \dots, x_7$: 266,304 + 16,777,216 pairs, no disagreement, the same
  result as the record's self-test.
- The counts of $+1$, $-1$ and $0$ (8, 0, 56; 56, 56, 3984; 1008, 1008, 260128;
  20160, 20160, 16736896 for the lengths 1 to 4) agree with the record and with
  the formula $N_{\neq 0} = 8!/(8-p)! \cdot p!$; every example of the record is
  reproduced.
- With 9 indices in 8 dimensions the delta is always 0 (pigeonhole), so the
  Lovelock sum of the theory stops at $k = 3$.
- Contracting one index multiplies by $n - p + 1$; in eight dimensions and with
  $p = 2k + 1$ this is $8 - 2k = 6, 4, 2$, the factors of the record's Lovelock
  trace identities.
- The Levi-Civita symbol is a generalized delta, and the product of two
  Levi-Civita symbols summed over $n - p$ indices is $(n - p)!$ times the
  generalized delta; in the signature (4,4) of the author's metric raising the
  indices adds no sign ($\det\eta = +1$), and the author's second route gives the
  Kronecker delta exactly.
```

1.48 Line-by-line walk-through of Notebook 01c

The notebook has nineteen code cells, In [1] to In [19]. As in Section 1.9, a quoted line `...)` stands for the remaining lines of a figure caption, which Section 1.47 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 1.46; its code is the set-up code explained in Section 1.9, with `NOTEBOOK_ID = "01c"`. It prints Set-up of notebook 01c complete: repository folder found, helpers defined.

In [2], the author's definition, read from the record.

```
import itertools  # all lists of labels, all permutations
import json  # reads the JSON reports of the Revision record
import math  # factorials
import re  # finds patterns in texts

import numpy as np  # arrays: many index lists at once
```

The modules. `re` (part of Python) finds **patterns** in texts: a pattern, a **regular expression**, describes a piece of text with placeholders, for example `\d+` for "one or more digits" and `(...)` for a part to be taken out.

```
DELTA = "\u03b4"  # the Greek letter delta, written with its code
definition = ("k" + DELTA + "[lower_, upper_] /; Length[lower] == Length[upper] := "
              "Det[Outer[delta, lower, upper]]")
provenance_path = repository_file("Revision/gkd_lovelock/results/"
                                  "PROVENANCE_OF_THE_COMPUTATION.md")
provenance = provenance_path.read_text(encoding="utf-8")
check(definition in provenance, "the record holds the author's definition verbatim",
      record="Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md")
```

The notebook is written in plain ASCII, so the Greek letter δ is typed by its code, `\u03b4`. `definition` is the author's definition as one string (two strings next to each other are joined). The record's provenance file is read, and `definition in provenance` asks whether the string occurs in it, letter for letter.

```
input_cells = repository_file("Revision/gkd_lovelock/results/"
                              "notebook-input-cells.txt").read_text(encoding="utf-8")
found = {}
for line in input_cells.splitlines():
    for label in ("In[87]:=", "In[32]:=", "In[29]:="):
        if line.startswith(label):
            found[label] = re.search(r"HoldForm\[(.*)\]", line).group(1)
```

The record's list of the author's input cells has one line per cell, starting with the label stored in the file. For the three labels of interest, the pattern `HoldForm\[(.*)\]` takes out the text between `HoldForm[` and the last `]` of the line (`.*` means "any characters"; the backslash makes the bracket an ordinary character), and `.group(1)` is the part in round brackets.

```
for label in ("In[87]:=", "In[32]:=", "In[29]:="):
    plain_text = found[label].replace(DELTA, "\\[Delta]")  # Mathematica's spelling
    say(f"author's cell {label} {plain_text}")
metric_cell = found["In[29]:="]  # kept for section 15 (the declaration of g)
```

The three cells are printed, with δ written in Mathematica's spelling `\[Delta]`: the definition (In[87]), the first cell of the second route (In[32]) and the declaration of the metric (In[29]), which is kept for In [17].

```
check("Det[Outer[delta, lower, upper]]" in found["In[87]:="] and
      "epsilong" in found["In[32]:="] and "(8 - 1)!" in found["In[32]:="],
```

```
      "the author's notebook defines the delta as a determinant (In[87]) and "
      "also uses a product of two Levi-Civita tensors (In[32])",
      record="Revision/gkd_lovelock/results/notebook-input-cells.txt")
```

The check confirms what the three printed cells show. The cell prints two PASS lines, each with the record it reproduces.

In [3], the generalized delta, literally.

```
def kronecker(a, b):
    """The ordinary Kronecker delta of two labels: 1 if they are equal, else 0."""
    return 1 if a == b else 0


def outer_matrix(lower, upper):
    """Outer[delta, lower, upper]: row i, column j is kronecker(lower[i], upper[j])"""
    return [[kronecker(l_label, u_label) for u_label in upper] for l_label in lower]
```

The ordinary Kronecker delta, and the matrix `Outer` as a list of rows: one row for each lower label, one entry for each upper label.

```
def inversions(order):
    """The number of pairs of places i < j whose entries stand in the wrong order."""
    return sum(1 for i in range(len(order)) for j in range(i + 1, len(order))
               if order[i] > order[j])


def sign(order):
    """+1 for an even number of inversions, -1 for an odd number."""
    return 1 if inversions(order) % 2 == 0 else -1
```

Inversions and sign, as in Notebook 01a, In [6] (Section 1.25).

```
SIGNED = {}  # size n -> list of (permutation, its sign), computed once per size


def signed_permutations(n):
    if n not in SIGNED:
        SIGNED[n] = [(order, sign(order)) for order in itertools.permutations(range(n))]
    return SIGNED[n]
```

The permutations of n objects with their signs are needed millions of times; they are computed once for each size and kept in the dictionary `SIGNED` (`if n not in SIGNED` is true only the first time).

```
def leibniz_det(rows):
    """The Leibniz determinant of a square matrix given as a list of rows."""
    total = 0
    for order, order_sign in signed_permutations(len(rows)):
        term = order_sign
```

```
        for i, column in enumerate(order):
            term *= rows[i][column]
            if term == 0:  # one factor 0 makes the whole term 0: stop early
                break
        total += term
    return total
```

The Leibniz formula as in Notebook 01a, with one saving: as soon as a factor 0 makes a term 0, the inner loop stops (`break`), because the rest of the product cannot change it.

```
def kdelta_literal(lower, upper):
    """The author's k-delta: Det[Outer[delta, lower, upper]] for equal lengths."""
    if len(lower) != len(upper):
        raise ValueError("the two index lists must have the same length")
    return leibniz_det(outer_matrix(lower, upper))
```

The author's definition, word for word: lists of different lengths are refused (Python stops with a `ValueError`, the counterpart of Mathematica leaving the expression unevaluated); otherwise the determinant of `Outer`.

```
def signed(value):
    """A value +1, -1 or 0 as the text "+1", "-1" or "0"."""
    return f"{value:+d}" if value else "0"


same = kdelta_literal(("x1",), ("x1",))  # lists of length 1: ("x1",) has one entry
different = kdelta_literal(("x1",), ("x2",))
say(f"one index: delta(x1, x1) = {same}, delta(x1, x2) = {different}")
check(same == 1 and different == 0,
      "for one index the generalized delta is the ordinary Kronecker delta")
```

A helper that writes a value with its sign (0 without one). Then the case of one index with the labels written as the strings "x1" and "x2" (`("x1",)` is a tuple with one entry; the comma makes it a tuple). Output: one index: delta(x1, x1) = 1, delta(x1, x2) = 0, and the PASS line.

In [4], the five matrices of the table.

```
examples = [(("x1", "x2", "x3"), ("x1", "x2", "x3"), "same lists"),
            (("x1", "x2", "x3"), ("x2", "x3", "x1"), "cyclic turn"),
            (("x1", "x2", "x3"), ("x1", "x3", "x2"), "one exchange"),
            (("x1", "x1", "x3"), ("x1", "x2", "x3"), "repeated lower"),
            (("x1", "x2", "x4"), ("x1", "x2", "x3"), "x4 missing above")]
fig, axes = plt.subplots(1, 5, figsize=(12.0, 3.4))
```

The five pairs of lists of the table of Section 1.42, each with a short description, and five panels.

```
for ax, (lower, upper, what) in zip(axes, examples):
    matrix = np.array(outer_matrix(lower, upper))
    ax.imshow(matrix, cmap="Greys", vmin=0, vmax=1.4)
```

```
    ax.grid(False)
    ax.set_xticks(range(3), [f"${u[0]}_{u[1]}$" for u in upper])  # "x1" -> x_1
    ax.set_yticks(range(3), [f"${l_label[0]}_{l_label[1]}$" for l_label in lower])
    ax.set_xlabel("upper list")
    ax.set_title(f"{what}: det = {signed(kdelta_literal(lower, upper))}", fontsize=9)
```

For each pair the matrix `Outer` is drawn in grey (dark for 1, white for 0); the labels "x1" are written as x_1 (the first character, an underscore for the subscript, the second character); the title gives the determinant.

```
axes[0].set_ylabel("lower list")
fig.tight_layout()
save_figure(fig, "outer_matrices",
            "The matrix Outer of ordinary Kronecker deltas for five pairs of index "
            ...)
values = [kdelta_literal(lower, upper) for lower, upper, _ in examples]
check(values == [1, 1, -1, 0, 0], "the five pictured examples give +1, +1, -1, 0, 0")
```

What Figure 01c.1 shows: the identity matrix (+1); a permutation matrix of a cyclic turn (+1); one with two rows exchanged (−1); two equal rows (0); a white row of zeros (0). The check compares the five values with the table. One PASS line.

In [5], the fast rule of the program GKD.

```
def gkd_rule(lower, upper):
    """The program GKD of Revision/gkd_lovelock/code/src/gkd.rs, in Python."""
    if len(lower) != len(upper):
        raise ValueError("the two index lists must have the same length")
    place = {}  # upper label -> its place 0, 1, ..., p-1
    for j, u_label in enumerate(upper):
        if u_label in place:  # case (b): a repeated upper label
            return 0
        place[u_label] = j
```

The four cases of Section 1.42 as a program, in the order of the Rust function GKD. First every upper label is entered in the dictionary `place` with its place; meeting a label that is already there is case (b), and the function returns 0.

```
    sigma = []  # sigma[i] = the place of lower[i] among the upper labels
    for l_label in lower:
        if l_label not in place:  # case (c): a lower label missing above
            return 0
        if place[l_label] in sigma:  # case (a): a repeated lower label
            return 0
        sigma.append(place[l_label])
    return sign(sigma)  # case (d): the sign of the permutation sigma
```

Then, for every lower label, its place among the upper labels is looked up: a missing label is case (c); a place that was taken already means a repeated lower label, case (a). The places form the permutation σ, and its sign is the value, case (d). No matrix is built.

```
check(all(gkd_rule(lower, upper) == kdelta_literal(lower, upper)
          for lower, upper, _ in examples),
      "the fast rule gives the literal values for the five examples")
renamed = {f"x{k}": f"x{(k + 1) % 8}" for k in range(8)}  # x0->x1, ..., x7->x0
two_lists = list(itertools.product([f"x{k}" for k in range(8)], repeat=2))
check(all(gkd_rule(lo, up) == gkd_rule(tuple(renamed[a] for a in lo),
                                       tuple(renamed[b] for b in up))
          for lo in two_lists for up in two_lists),
      "renaming the labels changes no value (all 4096 pairs of length 2)")
```

The fast rule agrees with the literal determinant on the five examples. Then the labels are renamed, x_k to x_{k+1} and x_7 to x_0 (`(k + 1) % 8`: the remainder of $k+1$ after division by 8, which is 0 for $k = 7$), and all $64 \cdot 64 = 4096$ pairs of lists of length 2 (`itertools.product(..., repeat=2)` gives all lists of two labels) must keep their values: only "equal or different" matters. Two PASS lines.

In [6], the examples of the record.

```
def read_report(name):
    """A JSON report of Revision/gkd_lovelock/results as a Python dictionary."""
    path = repository_file(f"Revision/gkd_lovelock/results/{name}")
    return json.loads(path.read_text(encoding="utf-8"))


def lists_from_text(text):
    """"1,2,3" -> (1, 2, 3)"""
    return tuple(int(part) for part in text.replace(" ", "").split(","))
```

A helper that reads a report of the record, and one that turns a text such as 1,2,3 into the tuple of whole numbers (blanks removed, cut at the commas).

```
python_report = read_report("python-lovelock-report.json")
python_checks = {c["name"]: c for c in python_report["checks"]}
pattern = r"kdelta\[\{([\d,]+)\},\{([\d,]+)\}\] = (-?\d+)"
record_examples = re.findall(pattern, python_checks["gkd_examples"]["detail"])
```

The record's report, its checks as a dictionary by name, and a pattern that matches texts like `kdelta[{1,2},{2,1}] = -1`: the backslashes make the brackets and braces ordinary characters, `([\d,]+)` takes out a run of digits and commas (a list), and `(-?\d+)` a whole number with an optional minus sign. `re.findall` returns all matches in the detail text of the check `gkd_examples`, each as the three parts.

```
for lower_text, upper_text, value in record_examples:
    lower, upper = lists_from_text(lower_text), lists_from_text(upper_text)
    say(f"lower {lower}, upper {upper}: record {signed(int(value))}, literal "
        f"{signed(kdelta_literal(lower, upper))}, rule {signed(gkd_rule(lower, upper))}")
```

Each of the five examples is printed with the record's value, the literal determinant and the fast rule: lower (1, 2), upper (2, 1): record -1, literal -1, rule -1, and so on.

```
check(len(record_examples) == 5 and all(
    kdelta_literal(lists_from_text(lo), lists_from_text(up)) == int(v) ==
    gkd_rule(lists_from_text(lo), lists_from_text(up)) for lo, up, v in record_examples),
    "the five examples of the record", record="Revision/gkd_lovelock/results/"
    "python-lovelock-report.json, check gkd_examples")
```

All five agree, three ways; this reproduces the record's check `gkd_examples`.

```
wolfram_report = read_report("wolfram-gkd-report.json")
wolfram_checks = {c["name"]: c for c in wolfram_report["checks"]}
detail = wolfram_checks["definition_unequal_lengths_stay_unevaluated"]["detail"]
wolfram_pattern = "k" + DELTA + r"\[\{([\d, ]+)\}, \{([\d, ]+)\}\] = ([+-]?\d+)"
wolfram_examples = re.findall(wolfram_pattern, detail)
```

The Wolfram report writes its examples with the Greek letter and blanks, as in `[{0, 1}, {1, 0}] = -1` after $k\delta$; the pattern is adapted accordingly, and finds the two examples with a value (the third list pair of the detail, of unequal lengths, has none).

```
try:
    kdelta_literal((0,), (0, 1))
    refused = False
except ValueError:  # lists of different lengths are refused
    refused = True
check(refused and len(wolfram_examples) == 2 and all(
    kdelta_literal(lists_from_text(lo), lists_from_text(up)) == int(v)
    for lo, up, v in wolfram_examples),
    "lists of different lengths are refused; the two Wolfram examples",
    record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, check "
    "definition_unequal_lengths_stay_unevaluated")
```

`try: ... except ValueError:` runs the first block and, if it stops with a `ValueError`, runs the second block instead of stopping the notebook: lists of the lengths 1 and 2 must be refused. Then the two examples with values; this reproduces the record's check.

```
self_check = read_report("lovelock-report.json")["gkdSelfCheck"]
length3_pair, transposition = self_check["length3Pair"], self_check["transposition"]
say(f"lovelock-report.json gkdSelfCheck: length3Pair = {length3_pair}, "
    f"transposition = {transposition}")
check(self_check["length3Pair"] == gkd_rule((0, 1, 2), (1, 2, 0)) and
      self_check["transposition"] == gkd_rule((0, 1), (1, 0)),
      "the two values printed by the Rust program",
      record="Revision/gkd_lovelock/results/lovelock-report.json, gkdSelfCheck")
```

The two values that the Rust program wrote into its report: length3Pair = 1 and transposition = -1, equal to the fast rule for the lists $(0,1,2)$, $(1,2,0)$ and $(0,1)$, $(1,0)$. The cell prints three PASS lines.

In [7], the 4096 values with two indices.

```
pairs = list(itertools.product(range(8), repeat=2))  # (0, 0), (0, 1), ..., (7, 7)
table = np.array([[gkd_rule(lo, up) for up in pairs] for lo in pairs])  # 64 x 64
```

```
two_term_ok = all(
    gkd_rule(lo, up) == kronecker(lo[0], up[0]) * kronecker(lo[1], up[1])
    - kronecker(lo[0], up[1]) * kronecker(lo[1], up[0])
    for lo in pairs for up in pairs)
check(two_term_ok, "the two-term formula holds for all 4096 pairs of length 2")
```

From here on the labels are the numbers 0 to 7, the computer's names for x_0 to x_7. All 64 lists of length 2; the 64×64 table of values (row: the lower list, column: the upper list); and the two-term formula of Section 1.42 for every entry.

```
say(f"values +1: {(table == 1).sum()}, values -1: {(table == -1).sum()}, "
    f"zeros: {(table == 0).sum()}")
check((table == 1).sum() == 56 and (table == -1).sum() == 56,
      "56 values +1 and 56 values -1 among the 4096")
```

The counts: values +1: 56, values -1: 56, zeros: 3984, as derived in Section 1.43.

```
fig, ax = plt.subplots(figsize=(6.6, 6.2))
image = ax.imshow(table, cmap="RdBu_r", vmin=-1, vmax=1, interpolation="nearest")
ax.grid(False)
ticks = [8 * k for k in range(8)]  # the first pair of each block of 8
ax.set_xticks(ticks, [f"$(x_{k}, x_0)$" for k in range(8)], rotation=90,
              fontsize=7)
ax.set_yticks(ticks, [f"$(x_{k}, x_0)$" for k in range(8)], fontsize=7)
ax.set_xlabel("upper list $(u_1, u_2)$")
ax.set_ylabel("lower list $(l_1, l_2)$")
ax.set_title("all 4096 values of the two-index delta")
fig.colorbar(image, ax=ax, shrink=0.8, ticks=[-1, 0, 1])
save_figure(fig, "two_index_table",
            "All 4096 values of the generalized delta with two indices over the "
            ...)
```

The table as a heat map (`interpolation="nearest"` keeps every entry a sharp square). The lists run in the order $(x_0, x_0), (x_0, x_1), \ldots, (x_7, x_7)$; a tick marks the first list of each block of eight, labelled (x_k, x_0), rotated by 90 degrees below the picture. What Figure 01c.2 shows: 56 red dots on the diagonal (equal lists of two different labels), with 8 white gaps where a list repeats its label, and 56 blue dots where the upper list is the reverse of the lower one. The cell prints two PASS lines.

In [8], every pair of length 1, 2 and 3, with plain loops.

```
comparison = wolfram_report["measurements"]["gkdComparison"]  # one entry per length
recorded = {entry["p"]: entry for entry in comparison}
plain_values = {}  # length p -> array of all values, lower lists in the outer loop
tallies = {}
total_pairs = 0
mismatches = 0
```

The record's table `gkdComparison` (one entry per length, with the counts of $+1$, -1 and 0), made into a dictionary by length; containers for the values, the counts (**tallies**), the number of pairs and of disagreements.

```
for p in (1, 2, 3):
    lists = list(itertools.product(range(8), repeat=p))
    values = []
    for lower in lists:
        for upper in lists:
            fast = gkd_rule(lower, upper)
            mismatches += fast != kdelta_literal(lower, upper)
            values.append(fast)
    plain_values[p] = np.array(values)
    total_pairs += len(values)
```

For $p = 1, 2, 3$: all 8^p lists, and for every pair the fast rule and the literal determinant; `mismatches += fast != ...` adds 1 when the two differ (True counts as 1). The values are kept, lower lists in the outer loop, for In [9].

```
    tally = {v: int((plain_values[p] == v).sum()) for v in (1, -1, 0)}
    tallies[p] = tally
    in_record = (recorded[p]["plusOne"], recorded[p]["minusOne"], recorded[p]["zero"])
    here = (tally[1], tally[-1], tally[0])
    say(f"length {p}: {len(values)} pairs")
    say(f"    counts of (+1, -1, 0): here {here}, record {in_record}")
    check(here == in_record,
          f"length {p}: the counts of +1, -1 and 0 equal the record",
          record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, check "
                 f"gkd_equals_kdelta_exhaustive_length_{p}")
```

The counts of the three values, compared with the record's: for length 3 the output reads here (1008, 1008, 260128), record (1008, 1008, 260128). Each length gives a PASS line that reproduces the record's check of that length.

```
report("pairs of length 1, 2, 3 compared", total_pairs)
exhaustive = [wolfram_checks[f"gkd_equals_kdelta_exhaustive_length_{p}"]
              for p in (1, 2, 3)]  # the record's three exhaustive comparisons
check(total_pairs == 266304 and mismatches == 0 and all(
      c["verdict"] == "PASS" and "(0 mismatches;" in c["detail"] for c in exhaustive),
      "literal determinant = fast rule for all 266304 pairs of length 1, 2, 3",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks "
             "gkd_equals_kdelta_exhaustive_length_1 to 3 (0 mismatches)")
```

The RESULT line 266304 ($64 + 4096 + 262144$). `exhaustive` is the list of the record's three checks `gkd_equals_kdelta_exhaustive_length_1`, `_length_2` and `_length_3`, taken from the dictionary `wolfram_checks` of In [6]; the f-string puts the length p at the end of the name. The check requires three things at once: 266,304 pairs; no disagreement between the literal determinant and the fast rule here; and, for each of the three record checks, the verdict PASS and the text `(0 mismatches;` in its detail (`"..." in text` is True when the first text occurs inside the second; `all(...)` is True when the condition holds for every entry of the list). So the notebook's count of disagreements, 0, reproduces the record's count of mismatches between the author's definition and GKD. (The record's sympy verifier ran a different exhaustive comparison on the same 266,304 pairs: the literal determinant against a determinant computed by an independent method, the **cofactor expansion**,

which writes a determinant as a sum over the entries of its first row, each entry times the determinant of the smaller matrix left when its row and column are removed, with the signs $+, -, +, \ldots$ in turn; its check is `gkd_literal_equals_cofactor_expansion` in `python-lovelock-report.json`. It compares two determinant programs, not a determinant with the fast rule, so this notebook does not claim to reproduce it.) The cell takes several seconds and prints four PASS lines.

In [9], many pairs at once.

```
def all_lists(p):
    """Every index list of length p over the labels 0 ... 7, one per row."""
    return np.array(list(itertools.product(range(8), repeat=p)), dtype=np.int8)
```

All 8^p lists of length p as the rows of a numpy array of small whole numbers (`np.int8`: one byte each, enough for 0 to 7).

```
def literal_many(lowers, uppers):
    """The author's definition for many pairs at once (exact whole numbers)."""
    p = lowers.shape[1]
    outer = lowers[:, :, None] == uppers[:, None, :]  # shape (pairs, p, p)
    values = np.zeros(len(lowers), dtype=np.int64)
    rows = np.arange(p)
    for order, order_sign in signed_permutations(p):
        # True where the entries (i, order[i]) of the matrix are all 1
        values += order_sign * outer[:, rows, list(order)].all(axis=1)
    return values
```

The literal determinant for a whole block of pairs: row r of `lowers` and of `uppers` is one pair. `lowers[:, :, None] == uppers[:, None, :]` compares every lower label with every upper label of the same pair at once (numpy **broadcasting**: the inserted axes of length 1 are stretched to match), giving one matrix `Outer` of True and False per pair. For each permutation, `outer[:, rows, list(order)]` picks the entries $(i, \sigma(i))$ of every matrix, `.all(axis=1)` is true when they are all 1 (then the term is ± 1, otherwise 0), and the term's sign is added. This is the Leibniz formula, done for all pairs together.

```
def rule_many(lowers, uppers):
    """The fast rule for many pairs at once."""
    p = lowers.shape[1]
    upper_sorted = np.sort(uppers, axis=1)
    upper_distinct = (np.diff(upper_sorted, axis=1) != 0).all(axis=1)
    same_labels = (np.sort(lowers, axis=1) == upper_sorted).all(axis=1)
```

The fast rule for a block, with the sign shortcut of Section 1.43. Sorting each upper list, `upper_di stinct` is true when no two neighbours of the sorted list are equal (no repeated upper label), and `same_labels` is true when the sorted lower list equals the sorted upper list (the two lists hold the same labels).

```
    first, second = np.triu_indices(p, 1)  # all pairs of places first < second
    inversions_lower = (lowers[:, first] > lowers[:, second]).sum(axis=1)
```

```
    inversions_upper = (uppers[:, first] > uppers[:, second]).sum(axis=1)
    signs = np.where((inversions_lower + inversions_upper) % 2 == 0, 1, -1)
    return np.where(upper_distinct & same_labels, signs, 0)
```

`np.triu_indices(p, 1)` lists all pairs of places $i < j$; the inversions of each lower and each upper list are counted, and $\mathrm{sign}(L)\,\mathrm{sign}(U)$ is $+1$ when their total is even. `np.where(condition, a, b)` takes `a` where the condition holds and `b` elsewhere: the sign where the lists hold the same different labels, 0 otherwise (with distinct upper labels and the same sorted labels, the lower labels are distinct as well, so cases (a), (b) and (c) all give 0).

```
agree = True
for p in (1, 2, 3):
    lists = all_lists(p)
    lowers = np.repeat(lists, len(lists), axis=0)  # each lower list 8^p times
    uppers = np.tile(lists, (len(lists), 1))  # all upper lists, again and again
    agree &= bool((literal_many(lowers, uppers) == plain_values[p]).all())
    agree &= bool((rule_many(lowers, uppers) == plain_values[p]).all())
check(agree, "the block functions agree with the plain ones on all pairs of "
      "length 1, 2, 3")
```

Before the new functions are trusted, they must give the plain values of In [8] for all 266,304 pairs. `np.repeat` repeats each lower list 8^p times and `np.tile` repeats the whole list of upper lists 8^p times, which together produce every pair in the order of In [8] (lower lists in the outer loop). One PASS line.

In [10], all 16,777,216 pairs of length 4.

```
selftest = read_report("gkd-selftest.json")
recorded_selftest = {entry["p"]: entry for entry in selftest["results"]}
lists4 = all_lists(4)  # 4096 lists
tally4 = {1: 0, -1: 0, 0: 0}
mismatches4 = 0
```

The record's self-test report, by length; the 4096 lists of length 4; counters.

```
for start in range(0, 4096, 256):
    lowers = np.repeat(lists4[start:start + 256], 4096, axis=0)
    uppers = np.tile(lists4, (256, 1))
    literal = literal_many(lowers, uppers)
    fast = rule_many(lowers, uppers)
    mismatches4 += int((literal != fast).sum())
    for v in (1, -1, 0):
        tally4[v] += int((fast == v).sum())
tallies[4] = tally4
pairs4 = sum(tally4.values())
```

The $4096 \cdot 4096 = 16{,}777{,}216$ pairs in 16 blocks: each block takes 256 lower lists (`start` runs 0, 256, ..., 3840), each paired with all 4096 upper lists, 1,048,576 pairs per block, few enough to fit in memory. For each block the literal determinant and the fast rule are compared and the values tallied.

```
say(f"length 4: {pairs4} pairs; +1: {tally4[1]}, -1: {tally4[-1]}, 0: {tally4[0]}; "
    f"mismatches {mismatches4}")
report("length 4, counts of the values +1, -1, 0",
       f"{tally4[1]}, {tally4[-1]}, {tally4[0]}")
for p in (1, 2, 3, 4):
    mode, pairs_p, wrong = (recorded_selftest[p]["mode"], recorded_selftest[p]["pairs"],
                            recorded_selftest[p]["mismatches"])
    say(f"record gkd-selftest.json, length {p}: {mode}, {pairs_p} pairs, {wrong} "
        "mismatches")
```

Output: length 4: 16777216 pairs; +1: 20160, -1: 20160, 0: 16736896; mismatches 0, the RESULT line, and four lines of the record's self-test: exhaustive, 64, 4096, 262144 and 16777216 pairs, 0 mismatches each.

```
check(pairs4 == recorded_selftest[4]["pairs"] == 16777216 and
      mismatches4 == recorded_selftest[4]["mismatches"] == 0,
      "all 16777216 pairs of length 4: literal determinant = fast rule",
      record="Revision/gkd_lovelock/results/gkd-selftest.json, length 4 "
             "(exhaustive)")
check(all(recorded_selftest[p]["pairs"] == 8 ** (2 * p) and
          recorded_selftest[p]["mismatches"] == 0 for p in (1, 2, 3)),
      "the record's exhaustive self-tests of length 1, 2, 3 cover 8^(2p) pairs "
      "with no mismatch, as found in section 10",
      record="Revision/gkd_lovelock/results/gkd-selftest.json, lengths 1 to 3")
```

Two PASS lines: the length-4 comparison of the record is reproduced, and the record's lengths 1 to 3 cover every pair (8^{2p}) with no mismatch, as In [8] found (section 10 of the notebook). The cell takes 10 to 30 seconds (COMPUTED).

In [11], the counts as a picture.

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
lengths = np.arange(1, 5)
for shift, value, color, name in [(-0.27, 1, "tab:red", "value +1"),
                                  (0.0, -1, "tab:blue", "value -1"),
                                  (0.27, 0, "0.6", "value 0")]:
    counts = [max(tallies[p][value], 0.8) for p in lengths]  # 0.8 marks a count 0
    ax.bar(lengths + shift, counts, width=0.25, color=color, label=name)
    for x, count in zip(lengths + shift, counts):
        ax.text(x, count * 1.25, f"{count:.0f}" if count >= 1 else "0",
                ha="center", fontsize=7, rotation=90)
```

Three bars per length: the counts of $+1$, -1 and 0. A logarithmic axis cannot show 0, so a count 0 is drawn as 0.8, a short stub, and labelled "0"; every bar carries its count above it, written vertically (`rotation=90`).

```
ax.set_yscale("log")
ax.set_ylim(0.5, 2e9)
ax.set_xticks(lengths)
ax.set_xlabel("length $p$ of the two index lists")
```

```
ax.set_ylabel("number of pairs (logarithmic scale)")
ax.set_title("Values of the generalized delta over 8 labels")
ax.legend(loc="upper left");
save_figure(fig, "value_counts",
            "The number of pairs of index lists of length $p = 1$ to 4 over the "
            ...)
```

What Figure 01c.3 shows: the grey bars (zeros) tower over the others, more so with every length; the red and blue bars are equal from $p = 2$ on.

In [12], the counting formula.

```
def nonzero_count(n, p):
    """The number of pairs of index lists of length p over n labels with value +-1."""
    return math.perm(n, p) * math.factorial(p) if p <= n else 0
```

The formula of Section 1.43: `math.perm(n, p)` is the falling factorial $n!/(n-p)!$.

```
formula_ok = tallies[1][1] == 8 and tallies[1][-1] == 0
for p in (2, 3, 4):
    formula_ok &= tallies[p][1] == tallies[p][-1] == nonzero_count(8, p) // 2
check(formula_ok, "the counting formula n!/(n-p)! p! gives the counts of lengths 1 to 4")
```

The formula against the counts of In [8] and In [10]: 8 values $+1$ and none -1 for $p = 1$; half of $N_{\neq 0}$ each for $p = 2, 3, 4$.

```
for p in range(1, 10):
    total = 8 ** (2 * p)
    say(f"p = {p}: {total:20d} pairs, {nonzero_count(8, p):11d} not zero, "
        f"fraction {nonzero_count(8, p) / total:.3e}")
```

The table for $p = 1$ to 9: the number of pairs, of nonzero values and their fraction. It ends with p = 8: 281474976710656 pairs, 1625702400 not zero, fraction 5.776e-06, and p = 9: 0 not zero.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
lengths = np.arange(1, 9)
left.semilogy(lengths, [8.0 ** (2 * p) for p in lengths], "s-",
              label="all pairs $8^{2p}$")
left.semilogy(lengths, [nonzero_count(8, p) for p in lengths], "o-",
              label="pairs with value $\\pm 1$")
left.set_xlabel("length $p$")
left.set_ylabel("number of pairs (logarithmic scale)")
left.set_title("for $p = 9$ no pair is left")
left.legend()
```

On the left both numbers for $p = 1$ to 8 on a logarithmic axis.

```
right.semilogy(lengths, [nonzero_count(8, p) / 8.0 ** (2 * p) for p in lengths], "o-",
               color="tab:purple")
right.set_xlabel("length $p$")
right.set_ylabel("fraction of pairs with value $\\pm 1$")
right.set_title("the nonzero values become rare")
fig.tight_layout()
# The fraction for p = 8 written as "5.8 \times 10^{-6}" for the caption:
mantissa, exponent = f"{nonzero_count(8, 8) / 8 ** 16:.1e}".split("e")
save_figure(fig, "nonzero_fraction",
            "Left: the number of all pairs of index lists of length $p$ over 8 "
            ...
            f"$p = 1$ to about ${mantissa} \\times 10^{{{int(exponent)}}}$ at "
            "$p = 8$; horizontal axes the length $p$.")
```

On the right their ratio. As in Notebook 01a, In [13], the fraction for $p = 8$, $1625702400/8^{16} = 5.78\ldots \times 10^{-6}$, is written with one digit after the point as the text 5.8e-06 and split at the letter e into the mantissa 5.8 and the exponent -06. The line `...` stands for four ordinary text lines of the caption. In the f-string line, `{mantissa}` inserts 5.8; `\\times` is one backslash written twice, so LaTeX receives its multiplication sign `\times`; in `10^{{{int(exponent)}}}` the doubled braces `{{` and `}}` stand for one literal brace each, and the insertion between them, `{int(exponent)}`, turns the text -06 into the whole number -6 (the leading zero disappears). So LaTeX receives `10^{-6}`, and the caption reads "to about 5.8×10^{-6} at $p = 8$". The last line is an ordinary text without insertions. What Figure 01c.4 shows: the number of all pairs grows along a straight line on the logarithmic axis, the number of nonzero ones much more slowly, and the fraction falls steadily. The cell prints one PASS line and the table.

In [13], nine indices in eight dimensions.

```
generator = np.random.default_rng(12345)
nine_all_zero = True
for sample in range(600):
    upper = tuple(int(x) for x in generator.integers(0, 8, size=9))
    if sample % 2 == 0:  # an arbitrary lower list
        lower = tuple(int(x) for x in generator.integers(0, 8, size=9))
    else:  # a re-ordering of the upper list
        lower = tuple(int(x) for x in generator.permutation(np.array(upper)))
```

600 random pairs of lists of 9 labels (seed 12345): for even samples an arbitrary lower list, for odd ones a re-ordering of the upper list (`generator.permutation` returns the entries in a random order), the hardest case for the rule.

```
    determinant = np.linalg.det(np.array(outer_matrix(lower, upper), dtype=float))
    nine_all_zero &= gkd_rule(lower, upper) == 0 and abs(determinant) < 1e-9
check(nine_all_zero, "600 pairs of lists of 9 labels from 8: every value is 0",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "gkd_nine_indices_in_eight_dimensions_vanish (with our own random lists)")
```

The literal determinant of the 9×9 matrix is computed by numpy's elimination (the Leibniz formula would have $9! = 362{,}880$ terms); both it and the fast rule must be 0, as the pigeonhole principle says. The record made the same test with its own random lists.

```
eight = tuple(range(8))
reverse = eight[::-1]  # (7, 6, ..., 0)
say(f"8 labels against their reverse: {inversions(reverse)} inversions, value "
    f"{gkd_rule(eight, reverse):+d}")
check(gkd_rule(eight, reverse) == 1 and inversions(reverse) == 28,
      "with 8 labels a nonzero value is possible (the reverse order has 28 "
      "inversions, value +1)")
```

`[::-1]` reverses a list. Output: 8 labels against their reverse: 28 inversions, value +1. The cell prints two PASS lines.

In [14], contracting one index.

```
def contraction_factors(n, p, pairs_of_lists):
    """The set of the ratios sum_c delta(A + c, B + c) / delta(A, B) over the pairs
    with delta(A, B) != 0; returns None if some pair breaks the identity."""
    found_factors = set()
    for lower, upper in pairs_of_lists:
        total = sum(gkd_rule(lower + (c,), upper + (c,)) for c in range(n))
        base = gkd_rule(lower, upper)
```

For each pair of lists A (lower) and B (upper) of length $p-1$: `total` is the sum over the n labels c of the delta of the extended lists (`lower + (c,)` appends c to a tuple), and `base` is the delta of A and B.

```
        if base == 0:
            if total != 0:
                return None
        else:
            found_factors.add(total // base)
            if total != (n - p + 1) * base:
                return None
    return found_factors
```

If the base is 0 the sum must be 0 too; otherwise the ratio is recorded and must be $n-p+1$. A broken pair makes the function return `None` (Python's "nothing"); otherwise it returns the set of ratios found, which must contain one number.

```
factors = {}  # (n, p) -> the factor found
for n in (4, 8):
    for p in range(1, n + 1):
        q = p - 1  # the length of the lists A and B
        if (n == 8 and p <= 3) or n == 4:  # every pair of lists, plain loops
            lists = list(itertools.product(range(n), repeat=q))
            pairs_q = [(lo, up) for lo in lists for up in lists]
```

For $n = 4$ and $n = 8$ and every p: when there are few lists ($n = 8$ with $p \le 3$, or $n = 4$), every pair of lists of length $q = p - 1$ is tested with plain loops. (For $p = 1$ the only list is the empty tuple, whose delta with itself is 1.)

```
        elif n == 8 and p == 4:  # every pair, with the block function
            lists = all_lists(3)
            lowers = np.repeat(lists, len(lists), axis=0)
            uppers = np.tile(lists, (len(lists), 1))
            base = rule_many(lowers, uppers)  # delta(A, B) for all 262144 pairs
            total = sum(rule_many(np.column_stack([lowers, np.full(len(lowers), c)]),
                                  np.column_stack([uppers, np.full(len(uppers), c)]))
                        for c in range(8))  # the sum over c, for all pairs at once
```

For $n = 8$, $p = 4$ all $512 \cdot 512 = 262{,}144$ pairs of lists of length 3 are handled with the block function of In [9]: `np.column_stack` appends a column holding the label c to every list (`np.full(len(lowers), c)` is that column), and the sum over c is taken for all pairs at once.

```
            nonzero = base != 0
            ratios = np.unique(total[nonzero] // base[nonzero])  # base is +1 or -1
            identity_holds = (bool((total[~nonzero] == 0).all()) and len(ratios) == 1
                              and bool((total == (n - p + 1) * base).all()))
            factors[(n, p)] = int(ratios[0]) if identity_holds else None
            continue
```

The same tests as in `contraction_factors`, on arrays: where the base is 0 the sum must be 0 (`~` turns True into False and back), the ratios must be one number, and the identity must hold everywhere. `continue` skips the rest of the loop body for this p.

```
        else:  # random pairs: half re-orderings of different labels, half arbitrary
            pairs_q = []
            for sample in range(4000):
                if sample % 2 == 0:
                    upper = tuple(int(x) for x in generator.permutation(n)[:q])
                    lower = tuple(int(x) for x in generator.permutation(np.array(upper)))
                else:
                    upper = tuple(int(x) for x in generator.integers(0, n, size=q))
                    lower = tuple(int(x) for x in generator.integers(0, n, size=q))
                pairs_q.append((lower, upper))
        found = contraction_factors(n, p, pairs_q)
        factors[(n, p)] = found.pop() if found and len(found) == 1 else None
```

For $n = 8$ and $p = 5$ to 8 there are too many pairs, so 4000 random ones are tested: half of them with q different labels (the first q entries of a random order of all n labels) and a re-ordering of them, where the value is not 0; half arbitrary. Then the function of above is applied, and the one factor found is stored (`found.pop()` takes the element out of the set; `if found and len(found) == 1` guards against `None` and against several factors).

```
say("factors found for n = 8, p = 1 ... 8: "
    + ", ".join(str(factors[(8, p)]) for p in range(1, 9)))
say("factors found for n = 4, p = 1 ... 4: "
    + ", ".join(str(factors[(4, p)]) for p in range(1, 5)))
report("contraction factors for 8 labels, p = 1 to 8",
       ", ".join(str(factors[(8, p)]) for p in range(1, 9)))
check(all(factors[(n, p)] == n - p + 1 for n in (4, 8) for p in range(1, n + 1)),
      "contracting one index multiplies by n - p + 1 (n = 4 and 8, every p)")
```

Output: the factors 8, 7, 6, 5, 4, 3, 2, 1 for $n = 8$ and 4, 3, 2, 1 for $n = 4$, the RESULT line, and the PASS line of the contraction identity of Section 1.44.

In [15], the factors of the record's trace identities.

```
trace_checks = [c for c in wolfram_report["checks"]
                if re.fullmatch(r"k\d_trace_equals_\d_L\d", c["name"])]
named = {}
for c in trace_checks:
    name, verdict = c["name"], c["verdict"]
    k_text, factor_text = re.fullmatch(r"k(\d)_trace_equals_(\d)_L\d", name).groups()
    k = int(k_text)
    named[k] = int(factor_text)
```

The Wolfram report names its trace checks after their factors, for example `k1_trace_equals_6_L1`. `re.fullmatch` requires the whole name to fit the pattern; the second pattern takes out the order k and the factor, and `named` maps k to the factor in the name.

```
    say(f"record check {name}: {verdict}; factor for k = {k}: {factor_text}; "
        f"found here for p = {2 * k + 1}: {factors[(8, 2 * k + 1)]}")
check(named == {1: 6, 2: 4, 3: 2} and
      all(factors[(8, 2 * k + 1)] == named[k] for k in (1, 2, 3)) and
      all(c["verdict"] == "PASS" for c in trace_checks),
      "the factors 8 - 2k = 6, 4, 2 of the Lovelock trace identities",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks "
             "k1_trace_equals_6_L1, k2_trace_equals_4_L2, k3_trace_equals_2_L3")
```

For each check the line printed compares the factor in its name with the factor found in In [14] for $p = 2k + 1$: 6 for $p = 3$, 4 for $p = 5$, 2 for $p = 7$. The check reproduces the three record checks. One PASS line.

In [16], the contraction factors as a picture.

```
fig, ax = plt.subplots(figsize=(7.2, 4.4))
for n, marker, color in ((8, "o", "tab:blue"), (4, "s", "tab:orange")):
    p_values = np.arange(1, n + 1)
    ax.plot(p_values, n - p_values + 1, "-", color=color, lw=1,
            label=f"$n - p + 1$ for $n = {n}$")
    ax.plot(p_values, [factors[(n, p)] for p in p_values], marker, color=color,
            ms=8, fillstyle="none", label=f"factors found, $n = {n}$")
```

For $n = 8$ and $n = 4$: the line $n - p + 1$ and the factors found, as open circles or open squares.

```
for k in (1, 2, 3):
    ax.annotate(f"$k = {k}$: $8 - 2k = {8 - 2 * k}$", xy=(2 * k + 1, 8 - 2 * k),
                xytext=(2 * k + 1.3, 8 - 2 * k + 1.3), fontsize=9,
                arrowprops={"arrowstyle": "->", "color": "black"})
ax.set_xlabel("number $p$ of indices before the contraction")
ax.set_ylabel("factor")
ax.set_title("Contracting one index of the generalized delta")
```

```
ax.legend(loc="upper right", fontsize=8);
save_figure(fig, "contraction_factor",
            "The factor by which contracting one upper with one lower index "
            ...)
```

Three arrows mark the Lovelock values. What Figure 01c.5 shows: all symbols on their straight lines, and the arrows at $p = 3, 5, 7$ on the line of $n = 8$ with the factors 6, 4, 2.

In [17], the Levi-Civita symbol and the author's second route.

```
def levi_civita(indices, n):
    """epsilon_(indices): the generalized delta with the upper list 0, 1, ..., n-1."""
    return gkd_rule(tuple(indices), tuple(range(n)))
```

The Levi-Civita symbol as defined in Section 1.44.

```
eta = json.loads(repository_file("Revision/algebra/gammas.json")
                 .read_text(encoding="utf-8"))["eta"]
say(f"diagonal of eta (x0 ... x7) from the record: {eta}; det eta = {math.prod(eta)}")
check(math.prod(eta) == 1, "det eta = +1: raising all 8 indices of epsilon gives no "
      "extra sign in the signature (4,4)")
```

η from the record; its determinant is the product of its diagonal, `math.prod(eta)`, which is 1. Output: the diagonal and det eta = 1, then the PASS line.

```
# (4, 4, 0): 4 positive, 4 negative, 0 zero eigenvalues; sign of det g = (-1)^4
signature = re.search(r"DefMetric\[\{(\d), (\d), (\d)\}", metric_cell)
positive, negative, zero = (int(part) for part in signature.groups())
say(f"the author's metric g: {positive} positive, {negative} negative, {zero} zero "
    f"eigenvalues; sign of det g = (-1)^{negative} = {(-1) ** negative:+d}")
check((positive, negative, zero) == (4, 4, 0) and (-1) ** negative == 1,
      "the author's notebook declares its metric with the signature (4,4), so the "
      "product of two Levi-Civita tensors has the sign +1",
      record="Revision/gkd_lovelock/results/notebook-input-cells.txt, In[29]")
```

From the author's cell In[29], kept in In [2], the pattern takes out the three numbers of `DefMetric[{4, 4, 0}`; `.groups()` returns them as texts, and they are made whole numbers. The sign of $\det g$ is $(-1)^4 = +1$. One PASS line, reproducing the record's cell.

```
formula_ok = True
for p in range(0, 5):  # n = 4, every length p of the lists a and b
    lists = list(itertools.product(range(4), repeat=p))
    for a in lists:
        for b in lists:
            total = sum(levi_civita(a + c, 4) * levi_civita(b + c, 4)
                        for c in itertools.product(range(4), repeat=4 - p))
            formula_ok &= total == math.factorial(4 - p) * gkd_rule(a, b)
check(formula_ok, "n = 4: sum of epsilon epsilon = (n - p)! delta for p = 0 to 4, "
      "all lists")
```

The product formula of Section 1.44 for $n = 4$ and every $p = 0$ to 4: for every pair of lists a, b of length p, the sum over all lists c of length $4 - p$ of $\varepsilon_{ac}\varepsilon_{bc}$ must be $(4 - p)!\,\delta^b_a$.

```
route_sums = np.zeros((8, 8), dtype=np.int64)
for a in range(8):
    others = [label for label in range(8) if label != a]
    for b in range(8):
        # Only lists c of 7 different labels other than a give a nonzero first factor.
        route_sums[a, b] = sum(levi_civita((a,) + c, 8) * levi_civita((b,) + c, 8)
                               for c in itertools.permutations(others))
author_route = route_sums // math.factorial(7)  # divide by 7! = 5040
```

The author's route In[32] for $n = 8$ and $p = 1$. Summing over all 8^7 lists c would be slow, but the first factor $\varepsilon_{ac_2\ldots c_8}$ is 0 unless c consists of the 7 labels other than a in some order; so the sum runs only over the $7! = 5040$ re-orderings of these labels (`itertools.permutations(others)`). Then the sums are divided by 7!.

```
say(f"the sums for a = b are {sorted(set(np.diag(route_sums).tolist()))}, "
    f"for a different from b {sorted(set(route_sums[~np.eye(8, dtype=bool)].tolist()))}")
check((route_sums == math.factorial(7) * np.eye(8, dtype=np.int64)).all(),
      "the author's route In[32]: sum of epsilon epsilon / 7! = delta^b_a, the "
      "8 x 8 identity")
```

The different values found on the diagonal and off it are printed (`set` keeps each value once; `~np.eye(8, dtype=bool)` selects the entries off the diagonal): [5040] and [0]. The check: the sums are 7! times the identity. The cell prints four PASS lines.

In [18], the Levi-Civita symbol as a picture.

```
fig, axes = plt.subplots(1, 4, figsize=(12.0, 3.5),
                         gridspec_kw={"width_ratios": [1, 1, 1, 1.5]})
names3 = ["$x_1$", "$x_2$", "$x_3$"]
for first in range(3):
    ax = axes[first]
    slice_values = np.array([[levi_civita((first, j, k), 3) for k in range(3)]
                             for j in range(3)])
    ax.imshow(slice_values, cmap="RdBu_r", vmin=-1, vmax=1)
    ax.grid(False)
```

Four panels, the last wider. In the first three the Levi-Civita symbol of three labels is drawn as three 3×3 heat maps, one for each first index (row j, column k).

```
    for j in range(3):
        for k in range(3):
            ax.text(k, j, signed(int(slice_values[j, k])), ha="center", va="center",
                    color="white" if slice_values[j, k] else "black")
    ax.set_xticks(range(3), names3)
    ax.set_yticks(range(3), names3)
    ax.set_title(f"$\\varepsilon$ with first index $x_{first + 1}$", fontsize=9)
```

Every square carries its value, white on the coloured squares; labels and a title.

```
image = axes[3].imshow(author_route, cmap="RdBu_r", vmin=-1, vmax=1)
axes[3].grid(False)
eight_names = [f"$x_{k}$" for k in range(8)]
axes[3].set_xticks(range(8), eight_names, fontsize=7)
axes[3].set_yticks(range(8), eight_names, fontsize=7)
axes[3].set_xlabel("$b$")
axes[3].set_ylabel("$a$")
axes[3].set_title("$\\sum \\varepsilon\\varepsilon / 7!$ in 8 dimensions", fontsize=9)
fig.tight_layout()
save_figure(fig, "levi_civita",
            "Left three panels: the Levi-Civita symbol $\\varepsilon_{ijk}$ of the "
            ...)
```

The fourth panel shows the result of the author's route. What Figure 01c.6 shows: in each of the first three panels two nonzero squares, one red and one blue, six in all, the signs of the six orderings; on the right the 8×8 identity matrix.

In [19], the last check.

```
figure_names = ["01c_1_outer_matrices.png", "01c_2_two_index_table.png",
                "01c_3_value_counts.png", "01c_4_nonzero_fraction.png",
                "01c_5_contraction_factor.png", "01c_6_levi_civita.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

The last line is ALL 28 CHECKS PASSED (notebook 01c). The 28 checks are: 2 in In [2], 1 in In [3], 1 in In [4], 2 in In [5], 3 in In [6], 2 in In [7], 4 in In [8], 1 in In [9], 2 in In [10], 1 in In [12], 2 in In [13], 1 in In [14], 1 in In [15], 4 in In [17] and 1 in In [19].

1.49 Binary numbers, bit operations and a pseudo-random generator

Why this section. The Revision record tested its program GKD twice (Section 1.43): a Rust self-test compared GKD with the author's determinant on millions of pairs of index lists, and a Wolfram check received the GKD values of 346,304 pairs from a small Rust program, the **exporter**, and compared them with the author's own definition in Mathematica. Many of these lists were drawn by a **pseudo-random generator**: a fixed formula that turns a number, its **state**, into a new state and an output, again and again; the outputs look random, but they are completely fixed by the first state, the **seed**. Because the formula and the seeds are recorded, the lists can be made again, exactly, and every number of the record can be checked. To do that we need the binary numbers in which the generator computes.

Binary numbers (PROVED). Our numbers are written in base 10: each digit counts a power of 10. In **binary** (base 2) there are only the digits 0 and 1, and each digit, a **bit**, counts a power of 2: the last digit counts $2^0 = 1$, the one before $2^1 = 2$, then $4, 8, 16, \ldots$. The binary digits of a whole number come from repeated division by 2, the remainder method of long division (Section 1.2): for 2026,

$$2026 = 2 \cdot 1013 + 0, \quad 1013 = 2 \cdot 506 + 1, \quad 506 = 2 \cdot 253 + 0, \quad 253 = 2 \cdot 126 + 1, \quad 126 = 2 \cdot 63 + 0,$$

$$63 = 2 \cdot 31 + 1, \quad 31 = 2 \cdot 15 + 1, \quad 15 = 2 \cdot 7 + 1, \quad 7 = 2 \cdot 3 + 1, \quad 3 = 2 \cdot 1 + 1, \quad 1 = 2 \cdot 0 + 1,$$

and the remainders, read from the last to the first, are the binary digits: $2026 = 11111101010$ in binary, 11 binary digits. Check: $1024 + 512 + 256 + 128 + 64 + 32 + 8 + 2 = 2026$ (the powers of 2 at the places of the digits 1). A computer stores whole numbers as a fixed number of bits; 64 bits hold the numbers 0 to $2^{64} - 1 = 18446744073709551615$.

Bytes and negative numbers. A **byte** is 8 bits, a whole number from 0 to 255. A **signed byte** uses the same 8 bits for the numbers -128 to 127: a negative number $-m$ is stored as $256 - m$ (the **two's complement**). So -1 is stored as 255, the eight bits 11111111. The exporter writes the GKD values $+1$, 0, -1 as the bytes 1, 0, 255.

Hexadecimal. Long binary numbers are written more compactly in base 16, **hexadecimal**, with the sixteen digits 0 to 9 and A to F (worth 10 to 15). One hexadecimal digit is exactly 4 bits ($16 = 2^4$). Python writes such a number with the prefix `0x`, for example `0xFF` $= 15 \cdot 16 + 15 = 255$.

The four bit operations (PROVED). The generator uses four operations on the bits of a whole number $x \geq 0$:

- **Shift right** by k places (Python writes it with the operator made of two greater-than signs): the last k bits are dropped. This is whole-number division by 2^k: writing $x = 2^k q + r$ with $0 \leq r < 2^k$, the last k bits of x are the bits of r, and the bits before them are those of q.
- **Shift left** by k places (two less-than signs): k zero bits are appended, which multiplies by 2^k (every bit moves to a place worth 2^k times more).
- **Keeping 64 bits** (`x & MASK`, with `MASK` $= 2^{64} - 1$, sixty-four ones; `&` keeps a bit where both numbers have a 1): the remainder of x divided by 2^{64}. The Rust programs compute with 64-bit numbers, so every result is cut to 64 bits in this way.
- **Exclusive or** (XOR, `x ^ y`): the two numbers are compared bit by bit, and a result bit is 1 when exactly one of the two bits is 1. Applying the same exclusive or twice gives back the number: for single bits, $(a \oplus b) \oplus b = a$ in all four cases ($b = 0$ changes nothing twice; $b = 1$ flips the bit twice), so it holds bit by bit. That is why an XOR step loses nothing.

For the 8-bit number $x = 182 = 10110110$: shifted right by 2 it is $101101 = 45$, and indeed $182 = 4 \cdot 45 + 2$; shifted left by 3 it is $10110110000 = 1456 = 182 \cdot 8$; keeping its last 8 bits gives $10110000 = 176$, and indeed $1456 = 5 \cdot 256 + 176$; and 182 XOR 11110000 flips the first four of the eight bits: $01000110 = 70$.

The generator `xorshift64*`. The Revision programs (the function `compare_random` of `Revision /gkd_lovelock/code/src/gkd.rs` and the exporter of the Wolfram check) use the generator called `xorshift64*` (shift, exclusive or, 64 bits, and a final multiplication, the star). Its state x is a 64-bit number, and one step reads, in Rust:

```
x ^= x >> 12;        x XOR (x shifted right by 12)
x ^= x << 25;        x XOR (x shifted left by 25, cut to 64 bits)
x ^= x >> 27;        x XOR (x shifted right by 27): the new state
output = x * 0x2545F4914F6CDD1D   (cut to 64 bits)
```

(`^=` means "replace by the exclusive or with"). A label from 0 to $n-1$ is the remainder of the output after division by n; the notebook calls this `below(n)`. The exporter starts, for the lists of length p, from the seed `0x243F6A8885A308D3` XOR p; these sixteen hexadecimal digits are the first digits of π after the point, written in base 16 ($\pi = 3.243F6A88\ldots$ in hexadecimal), a number that nobody chose to make a test pass. The Rust self-test starts from (`0x2545F4914F6CDD1D` XOR p) OR 1, where OR 1 sets the last bit to 1. To put a list into a random order the programs use the **Fisher-Yates shuffle**: for i from the last place down to the second, exchange the entry at place i with the entry at a random place from 0 to i.

A test is not a proof. A pseudo-random sample, however large, is evidence, not a proof; the proof that GKD equals the author's determinant is the four-case argument of Section 1.42. What the regenerated lists show is something else, just as important for a record: its numbers are exactly reproducible.

1.50 Fingerprints, the exporter's file and the birthday problem

The sha256 fingerprint. A **sha256 fingerprint** is a 64-digit hexadecimal number computed from the bytes of a file by the method SHA-256. Two different files have different fingerprints for all practical purposes (a property of the method, ASSUMED here), so equal fingerprints mean equal files, byte for byte; and changing a single bit of a file changes its fingerprint completely. The Revision record keeps the fingerprints of the files it used.

The exporter's file (PROVED by arithmetic). The exporter writes, for every pair of index lists, one record of signed bytes: the length p, the p lower labels, the p upper labels and the GKD value, $2p+2$ bytes in all. It writes every pair of the lengths 1, 2 and 3, and 20,000 pseudo-random pairs of each length 4 to 7. So the file has

$$64\cdot 4 + 4096\cdot 6 + 262144\cdot 8 + 20000\,(10+12+14+16) = 256 + 24576 + 2097152 + 1040000 = 3161984$$

bytes (the number of pairs of each length times its record size; multiply out and add). The record states the same size and the file's fingerprint in `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, field `gkdValuesSource`. Notebook 01d, In [8], regenerates all 3,161,984 bytes from the seeds and the recorded source code and finds the recorded fingerprint, `3ddccfa744b3708b` followed by 48 more digits (COMPUTED). It then reproduces every number of the record's table `gkdComparison` (In [9]): for the lengths 1 to 7 the numbers of pairs (64, 4096, 262144 and 20,000 four times; 346,304 in all), the counts of $+1$, -1 and 0, no mismatch between GKD and the author's determinant, and the result of a **negative control**: the same comparison against $-$GKD, a test that must fail, fails exactly at the nonzero values, which shows that the comparison can fail.

The self-test for the lengths 5 to 9. The record's file `Revision/gkd_lovelock/results/gkd-selftest.json` gives, for 200,000 pseudo-random pairs of each length 5 to 9, the numbers of nonzero values, 20538, 7812, 1949, 244 and 0, and no mismatch. Notebook 01d, In [11], regenerates the 1,000,000 pairs and gets exactly these numbers (COMPUTED).

The birthday problem (PROVED). Why these numbers? In the self-test an even sample takes p upper labels with `below(8)` and a re-ordering of them as the lower list; its value is not zero exactly when its p upper labels are all different. Labels drawn one after the other, each with 8 equally likely values, are all different with the probability

$$P(p) = \frac{8}{8} \cdot \frac{7}{8} \cdots \frac{8-p+1}{8} = \frac{8!/(8-p)!}{8^p}$$

(the first label may be anything; the second must avoid it, 7 of 8 values; the third must avoid both, 6 of 8; and so on; independent chances multiply, a rule of probability ASSUMED here). This is the **birthday problem**: the chance that p people have p different birthdays, with 8 "days" instead of 365. For $p = 9$ it is 0, because $8 - 9 + 1 = 0$ (the pigeonhole principle again). An odd sample draws two arbitrary lists; it is nonzero with the probability $q(p) = N_{\neq 0}(8, p)/8^{2p}$ of Section 1.43. For N tries that each succeed with probability q, the number of successes is about Nq, its **expected value**, typically within one **standard deviation** $\sqrt{Nq(1-q)}$ of it (ASSUMED from probability theory). With 100,000 even and 100,000 odd samples the expected number of nonzero values is $100000\,P(p) + 100000\,q(p)$; for $p = 5$:

$$100000 \cdot \frac{8 \cdot 7 \cdot 6 \cdot 5 \cdot 4}{8^5} + 100000 \cdot \frac{806400}{8^{10}} = 100000 \cdot \frac{6720}{32768} + 75.1 = 20507.8 + 75.1 = 20582.9$$

($8^5 = 32768$; $N_{\neq 0}(8, 5) = 6720 \cdot 5! = 806400$ and $8^{10} = 1073741824$), with a standard deviation of about 128; the recorded 20538 lies 0.35 standard deviations below it. Notebook 01d, In [12], makes this comparison for every length 5 to 9 and for the odd samples of the Wolfram check, all within 4 standard deviations (COMPUTED). The classic birthday problem says that among 23 people two share a birthday more often than not: $P_{365}(22) = 0.5243$ and $P_{365}(23) = 0.4927$ (COMPUTED; Figure 01d.6).

1.51 Example: reproducing the record's tests of the generalized delta

Notebook 01d puts Sections 1.49 and 1.50 to work. It writes 2026 in binary and shows how a byte stores -1; applies the four bit operations; writes the generator `xorshift64*` in Python, follows its bits through one step and draws its output; checks that its labels come out about equally often; regenerates the 3,161,984 bytes that the exporter gave the Wolfram check and confirms their size and fingerprint; reproduces every number of the record's table `gkdComparison` and of the Rust self-test for the lengths 5 to 9; and explains the counts with the birthday problem. It ends with the line ALL 19 CHECKS PASSED (notebook 01d).

1.52 How to run Notebook 01d

Step 1. What this notebook does and what it needs.

Notebook 01d (Reproducing the GKD self-tests of the Revision record bit for bit) is the file `Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb` of the repository Dirac_claude. It explains whole numbers in binary, the storage of negative numbers in a byte and the bit operations of the pseudo-random generator xorshift64star used by the Revision programs, re-implements

that generator in Python, regenerates the 3,161,984 bytes of generalized Kronecker delta values that the Rust program GKD gave the Wolfram check of the Revision record and confirms their sha256 fingerprint and every count of that record, repeats the Rust self-test of GKD with 200,000 pseudo-random pairs of index lists for each length 5 to 9 and obtains exactly the recorded numbers of nonzero values, compares every value with the determinant of the author's definition, and explains these numbers with the probability that labels drawn at random are all different (the birthday problem). It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 01d_gkd_record_selftests.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 01d_gkd_record_selftests.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 01d_gkd_record_selftests.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/01d.captions.json`
- `Revision/textbook/figures/01d_1_binary_counting.png`
- `Revision/textbook/figures/01d_2_generator_step.png`
- `Revision/textbook/figures/01d_3_generator_bits.png`
- `Revision/textbook/figures/01d_4_label_histogram.png`
- `Revision/textbook/figures/01d_5_record_counts.png`
- `Revision/textbook/figures/01d_6_birthday_problem.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 19 CHECKS PASSED (notebook 01d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 01d_gkd_record_selftests.ipynb
python -m nbconvert --execute --inplace 01d_gkd_record_selftests.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock: the notebook reads reports of the Revision record; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.
- "AssertionError: check failed: the regenerated values have the size and the sha256 fingerprint of the record": the file wolfram-gkd-report.json of your copy differs from the one the book was built with, or a cell above was changed; download the repository again and run all cells from the top without changing them.

1.53 Notebook 01d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb`: every cell in order, with the output that the stored, executed

notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Revision record checked its program GKD (the generalized Kronecker delta of
notebook 01c) twice: a Rust self-test compared GKD with the author's determinant
on millions of pairs of index lists, and a Wolfram check received the values of
GKD for 346,304 pairs of index lists from a small Rust *exporter* program and
compared them with the author's own definition. Many of these lists were drawn
by a *pseudo-random* generator. Because such a generator is a fixed formula, the
lists can be made again, exactly. This notebook

- explains whole numbers in binary, how a byte stores $-1$, and the three bit
  operations (shift, exclusive or, keeping 64 bits) of the generator
  `xorshift64*` that the Revision programs use;
- writes that generator in Python and shows step by step what it does to the
  bits, and that its output looks random;
- regenerates, in Python, all 3,161,984 bytes that the Rust exporter wrote for
  the Wolfram check, and confirms the size and the sha256 fingerprint recorded in
  the Wolfram report, and every count of that report;
- repeats the Rust self-test for the lengths 5 to 9 (200,000 pairs each) and gets
  exactly the recorded numbers of nonzero values, comparing every value with the
  determinant;
- explains these numbers with the probability that labels drawn at random are
  all different, the "birthday problem".
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 01d (Reproducing the GKD self-tests of the Revision record bit for bit) is the
file `Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb` of the repository
Dirac_claude. It explains whole numbers in binary, the storage of negative numbers in a
byte and the bit operations of the pseudo-random generator xorshift64star used by the
Revision programs, re-implements that generator in Python, regenerates the 3,161,984
bytes of generalized Kronecker delta values that the Rust program GKD gave the Wolfram
check of the Revision record and confirms their sha256 fingerprint and every count of
that record, repeats the Rust self-test of GKD with 200,000 pseudo-random pairs of index
lists for each length 5 to 9 and obtains exactly the recorded numbers of nonzero values,
compares every value with the determinant of the author's definition, and explains these
numbers with the probability that labels drawn at random are all different (the birthday
problem). It needs a computer with Windows 11, macOS or Linux, an internet connection for
the installation, the program Git, and Python 3.12 or newer (the notebooks were built
with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
```

0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these

lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 01d_gkd_record_selftests.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 01d_gkd_record_selftests.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 01d_gkd_record_selftests.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/01d.captions.json‘
- ‘Revision/textbook/figures/01d_1_binary_counting.png‘
- ‘Revision/textbook/figures/01d_2_generator_step.png‘
- ‘Revision/textbook/figures/01d_3_generator_bits.png‘
- ‘Revision/textbook/figures/01d_4_label_histogram.png‘
- ‘Revision/textbook/figures/01d_5_record_counts.png‘
- ‘Revision/textbook/figures/01d_6_birthday_problem.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 19 CHECKS PASSED (notebook 01d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs).

Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 01d_gkd_record_selftests.ipynb
python -m nbconvert --execute --inplace 01d_gkd_record_selftests.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below Revision/gkd_lovelock: the notebook reads reports of the Revision record; your copy of the repository is incomplete. Download it again with git clone and open the notebook inside the new copy.
- "AssertionError: check failed: the regenerated values have the size and the sha256 fingerprint of the record": the file wolfram-gkd-report.json of your copy differs from the one the book was built with, or a cell above was changed; download the repository again and run all cells from the top without changing them.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 01d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 01d (Reproducing the GKD self-tests of the Revision record bit for bit) is the
# file Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb of the repository
# Dirac_claude. It explains whole numbers in binary, the storage of negative numbers in a
# byte and the bit operations of the pseudo-random generator xorshift64star used by the
# Revision programs, re-implements that generator in Python, regenerates the 3,161,984
# bytes of generalized Kronecker delta values that the Rust program GKD gave the Wolfram
# check of the Revision record and confirms their sha256 fingerprint and every count of
# that record, repeats the Rust self-test of GKD with 200,000 pseudo-random pairs of
# index lists for each length 5 to 9 and obtains exactly the recorded numbers of nonzero
# values, compares every value with the determinant of the author’s definition, and
# explains these numbers with the probability that labels drawn at random are all
# different (the birthday problem). It needs a computer with Windows 11, macOS or Linux,
# an internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
```

```
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
```

```
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
```

```
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 01d_gkd_record_selftests.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 01d_gkd_record_selftests.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 01d_gkd_record_selftests.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/01d.captions.json
# - Revision/textbook/figures/01d_1_binary_counting.png
# - Revision/textbook/figures/01d_2_generator_step.png
# - Revision/textbook/figures/01d_3_generator_bits.png
# - Revision/textbook/figures/01d_4_label_histogram.png
# - Revision/textbook/figures/01d_5_record_counts.png
# - Revision/textbook/figures/01d_6_birthday_problem.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
```

```
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 19 CHECKS PASSED (notebook 01d)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/01d_gkd_record_selftests.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 01d_gkd_record_selftests.ipynb
#     python -m nbconvert --execute --inplace 01d_gkd_record_selftests.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below Revision/gkd_lovelock: the notebook reads
#   reports of the Revision record; your copy of the repository is incomplete. Download
#   it again with git clone and open the notebook inside the new copy.
```

```
# - "AssertionError: check failed: the regenerated values have the size and the sha256
#   fingerprint of the record": the file wolfram-gkd-report.json of your copy differs
#   from the one the book was built with, or a cell above was changed; download the
#   repository again and run all cells from the top without changing them.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "01d"  # this notebook: chapter 01, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
```

```
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
```

```
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 01d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Binary**: writing a whole number with the two digits 0 and 1; each digit
  (a **bit**) counts a power of 2: $2026 = 1024 + 512 + 256 + 128 + 64 + 32 + 8 +
  2$ is ‘11111101010‘ in binary.
- **Byte**: 8 bits, a whole number from 0 to 255. A **signed byte** uses the same
  8 bits for the numbers $-128$ to $127$: a negative number $-m$ is stored as
  $256 - m$ (the **two’s complement**), so $-1$ is stored as 255, ‘11111111‘.
- **Hexadecimal**: writing a number with the 16 digits ‘0‘–‘9‘ and ‘A‘–‘F‘
  (worth 10 to 15); Python writes such a number with the prefix ‘0x‘. One
  hexadecimal digit is exactly 4 bits.
- **Shift**: ‘x >> k‘ drops the last $k$ bits (the same as dividing by $2^k$ and
  dropping the remainder); ‘x << k‘ appends $k$ zero bits (multiplying by $2^k$).
- **Exclusive or** (‘x ^ y‘, XOR): compares the two numbers bit by bit; a result
  bit is 1 when exactly one of the two bits is 1.
- **Keeping 64 bits** (‘x & MASK‘ with ‘MASK‘ $= 2^{64} - 1$): the remainder of $x$
  divided by $2^{64}$; the Rust programs compute with 64-bit numbers, so every
  result is cut to 64 bits.
- **Pseudo-random generator**: a formula that turns a number (its **state**) into
  a new state and an output, again and again; the outputs look random but are
  completely fixed by the first state, the **seed**.
- ‘xorshift64*‘: the generator of the Revision programs: three shift-and-XOR
  steps on a 64-bit state and a multiplication of the output by a fixed odd
  number.
- **Fisher-Yates shuffle**: a way to put a list into a random order by swapping
  entries one after the other.
- **sha256 fingerprint**: a 64-digit hexadecimal number computed from a file by
  the method SHA-256; two different files have different fingerprints for all
```

```
  practical purposes, so equal fingerprints mean equal files, byte for byte.
- **Exporter**: the small Rust program, held as text inside the Wolfram check
  Revision/gkd_lovelock/verification/verify_lovelock_gkd.wls, that wrote the GKD
  values into a file for the Wolfram check.
- **Expected value, standard deviation**: for $N$ tries that each succeed with
  probability $q$, the number of successes is about $Nq$ (the expected value),
  typically within $\sqrt{Nq(1-q)}$ (one standard deviation) of it.
```

Text cell:

```
## 4. The physical and mathematical situation

The generalized Kronecker delta of two index lists of the same length $p$ over
the eight coordinates $x_0, \dots, x_7$ (labels $0, \dots, 7$ for the computer)
is the determinant of the matrix $M_{ij} = \delta(l_i, u_j)$; the program GKD
computes it as 0 for a repeated or missing label and otherwise as the sign of the
permutation that turns the upper list into the lower one (notebook 01c proves
this). It is the weight of every term of the Lovelock tensors of the author's
theory, so the record tested it hard:

1. the **Rust self-test** (‘lovelock_gkd gkd-selftest‘, file gkd-selftest.json)
   compared GKD with the literal Leibniz determinant for all pairs of lengths 1
   to 4 and for 200,000 pseudo-random pairs of each length 5 to 9;
2. the **Wolfram check** (wolfram-gkd-report.json) had the exporter write, as
   bytes, every pair of lengths 1, 2, 3 and 20,000 pseudo-random pairs of each
   length 4 to 7 with their GKD values, and called the author's own definition
   in Mathematica on every pair.

Both used the generator ‘xorshift64*‘ with stated seeds. This notebook rebuilds
the lists from the seeds and the recorded source code, so it can compare its
results with the record exactly: the same bytes, the same counts. A pseudo-random
sample is not a proof; the proof that GKD equals the determinant is the
four-case argument of notebook 01c. What this notebook shows is that the record's
numbers are exactly reproducible.
```

Text cell:

```
## 5. Whole numbers in binary

The next cell writes 2026 in binary with ‘format(number, "b")‘, adds up the
binary digits times their powers of 2 to get the number back (the last digit
counts $2^0 = 1$, the one before $2^1 = 2$, and so on), prints the largest number
that 64 bits can hold, and shows how a signed byte stores $-1$: as 255, eight
ones. Python's ‘& 0xFF‘ keeps the last 8 bits of a number (0xFF is 255);
‘int.from_bytes(..., signed=True)‘ reads a byte back as a signed number.
```

In [2]:

```
import hashlib  # the sha256 fingerprint
import itertools  # all index lists of a given length
import json  # reads the JSON reports of the record
```

```
import math  # factorials and falling factorials

import mpmath  # numbers with many digits
import numpy as np  # arrays

number = 2026
bits = format(number, "b")  # the binary digits, a text of 0s and 1s
say(f"{number} in binary: {bits} ({len(bits)} binary digits)")
# The last digit counts 2^0, the one before 2^1, ...: reversed() starts at the end.
by_places = sum(int(digit) * 2 ** place for place, digit in enumerate(reversed(bits)))
check(by_places == number == int(bits, 2), "the binary digits of 2026 give back 2026")
say(f"the largest number that 64 bits can hold: 2^64 - 1 = {2 ** 64 - 1}")
stored = (-1) & 0xFF  # the 8 bits that a signed byte uses for -1
read_back = int.from_bytes(bytes([stored]), "big", signed=True)
stored_bits = format(stored, "08b")  # 8 binary digits, zeros in front
say(f"-1 in a signed byte: {stored} = {stored_bits}, read back as {read_back}")
check(stored == 255 and read_back == -1, "a signed byte stores -1 as 255 = 11111111")
```

Out [2]:

```
2026 in binary: 11111101010 (11 binary digits)
PASS the binary digits of 2026 give back 2026
the largest number that 64 bits can hold: 2^64 - 1 = 18446744073709551615
-1 in a signed byte: 255 = 11111111, read back as -1
PASS a signed byte stores -1 as 255 = 11111111
```

Text cell:

```
The next cell draws the numbers 0 to 31 in binary: one column per number, one
row per bit, from $2^4 = 16$ at the top to $2^0 = 1$ at the bottom (dark is 1).
The bottom row alternates with every number, the next row every two numbers, the
next every four: binary counting.
```

In [3]:

```
numbers = np.arange(32)
bit_rows = np.array([(numbers >> k) & 1 for k in range(4, -1, -1)])  # 2^4 ... 2^0
fig, ax = plt.subplots(figsize=(11.0, 2.8))
ax.imshow(bit_rows, cmap="Greys", vmin=0, vmax=1.3, aspect="auto")
ax.grid(False)
ax.set_xticks(range(32), [str(k) for k in range(32)], fontsize=7)
ax.set_yticks(range(5), [f"$2^{k}$" for k in range(4, -1, -1)])
ax.set_xlabel("the number")
ax.set_ylabel("bit")
ax.set_title("The numbers 0 to 31 in binary (dark = 1)")
save_figure(fig, "binary_counting",
            "The numbers 0 to 31 (columns) written in binary: each row is one bit, "
            "counting $2^4 = 16$ (top) down to $2^0 = 1$ (bottom), dark for the "
            "digit 1 and white for 0. Adding the powers of 2 of the dark squares "
            "of a column gives its number; the lowest bit changes with every "
            "number, the next one every 2 numbers, the next every 4.")
```

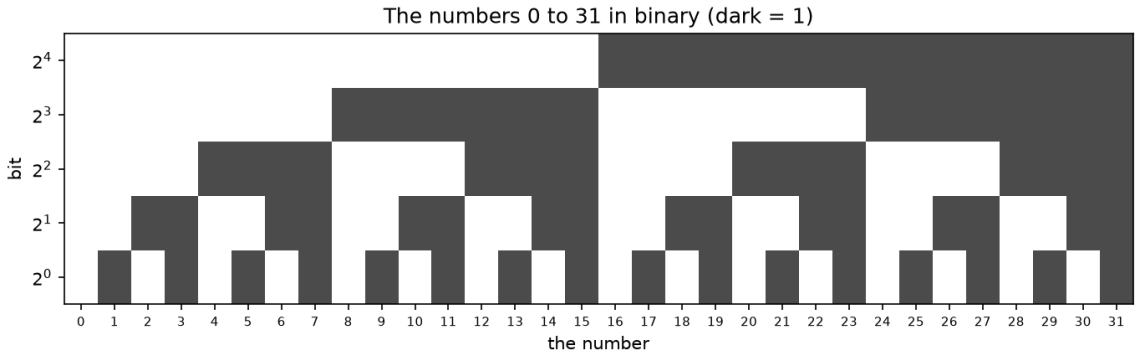

Figure 56: The numbers 0 to 31 (columns) written in binary: each row is one bit, counting $2^4 = 16$ (top) down to $2^0 = 1$ (bottom), dark for the digit 1 and white for 0. Adding the powers of 2 of the dark squares of a column gives its number; the lowest bit changes with every number, the next one every 2 numbers, the next every 4. (Notebook 01d, figure 1.)

Out [3]:

```
Figure 01d.1 saved as Revision/textbook/figures/01d_1_binary_counting.png
```

Text cell:

```
## 6. The three bit operations

The next cell applies the operations of the generator to the 8-bit number
$x = 182$ = ‘10110110‘ and prints the bits of each result (‘format(v, "08b")‘
writes a number with at least 8 binary digits):

- ‘x >> 2‘ drops the last two bits: ‘101101‘ = 45, the same as $182 // 4$ (whole
  number division);
- ‘x << 3‘ appends three zero bits: $182 \cdot 8 = 1456$;
- ‘(x << 3) & 0xFF‘ keeps the last 8 bits: the remainder of 1456 divided by 256;
- ‘x ^ 0b11110000‘ flips the first four of the eight bits.

It checks these rules on 1000 random 64-bit numbers too, and that applying the
same exclusive or twice gives back the number, which is why an XOR step loses
nothing.
```

In [4]:

```
def binary(value, width=8):
    """value in binary with at least width digits (zeros in front)."""
    return format(value, "b").zfill(width)


x = 182
say(f"x                 = {binary(x)} = {x}")
say(f"x >> 2            = {binary(x >> 2)} = {x >> 2} = {x} // 4")
```

```
say(f"x << 3            = {binary(x << 3, 11)} = {x << 3} = {x} * 8")
say(f"(x << 3) & 0xFF   = {binary((x << 3) & 0xFF)} = {(x << 3) & 0xFF} "
    f"= {x << 3} % 256")
say(f"x ^ 0b11110000    = {binary(x ^ 0b11110000)} = {x ^ 0b11110000}")
MASK = 2 ** 64 - 1  # 64 ones: "& MASK" keeps the last 64 bits
generator = np.random.default_rng(12345)
rules_hold = True
for _ in range(1000):
    a = int(generator.integers(0, 2 ** 63)) * 2 + int(generator.integers(0, 2))
    b = int(generator.integers(0, 2 ** 63)) * 2 + int(generator.integers(0, 2))
    k = int(generator.integers(1, 64))
    rules_hold &= (a >> k) == a // 2 ** k
    rules_hold &= ((a << k) & MASK) == (a * 2 ** k) % 2 ** 64
    rules_hold &= ((a ^ b) ^ b) == a
check(rules_hold, "shift = division or multiplication by 2^k, & MASK = remainder "
      "modulo 2^64, and XOR twice gives back the number (1000 random 64-bit numbers)")
```

Out [4]:

```
x                 = 10110110 = 182
x >> 2            = 00101101 = 45 = 182 // 4
x << 3            = 10110110000 = 1456 = 182 * 8
(x << 3) & 0xFF   = 10110000 = 176 = 1456 % 256
x ^ 0b11110000    = 01000110 = 70
PASS shift = division or multiplication by 2^k, & MASK = remainder modulo 2^64, and XOR
    twice gives back the number (1000 random 64-bit numbers)
```

Text cell:

```
## 7. The generator ‘xorshift64*‘, step by step

One step of the generator, as written in the record’s Rust code (the function
‘compare_random‘ of Revision/gkd_lovelock/code/src/gkd.rs and the exporter in
verify_lovelock_gkd.wls):

1. ‘x ^= x >> 12‘: XOR the state with itself shifted 12 bits to the right;
2. ‘x ^= x << 25‘: XOR with itself shifted 25 bits to the left (Rust keeps 64
   bits, so we write ‘& MASK‘);
3. ‘x ^= x >> 27‘: XOR with itself shifted 27 bits to the right; this is the new
   state;
4. the output is the new state times the odd constant ‘0x2545F4914F6CDD1D‘, cut
   to 64 bits.

The generator’s output ‘below(n)‘, a label from 0 to $n - 1$, is the output’s
remainder after division by $n$. The exporter starts from the seed
‘0x243F6A8885A308D3 XOR p‘ for the length $p$; these 16 hexadecimal digits are
the first digits of $\pi$ after the point, written in base 16 (a number that
nobody chose to make a test pass). The Rust self-test starts from
‘(0x2545F4914F6CDD1D XOR p) OR 1‘ (‘| 1‘ makes the last bit 1).

The next cell writes the generator as a small Python class, checks the digits of
$\pi$ with mpmath, and checks that the same seed always gives the same numbers
while a different seed gives different ones.
```

In [5]:

```
MULTIPLIER = 0x2545F4914F6CDD1D  # the constant of xorshift64*
EXPORTER_SEED = 0x243F6A8885A308D3  # the first 16 hexadecimal digits of pi
SELFTEST_SEED = 0x2545F4914F6CDD1D  # the Rust self-test uses the same constant


class XorShift64Star:
    """The pseudo-random generator of the Revision programs."""

    def __init__(self, seed):
        self.state = seed

    def below(self, n):
        """One step of the generator; returns its output modulo n (0 ... n-1)."""
        x = self.state
        x ^= x >> 12
        x ^= (x << 25) & MASK
        x ^= x >> 27
        self.state = x
        return ((x * MULTIPLIER) & MASK) % n


say(f"MULTIPLIER = {MULTIPLIER} (decimal), EXPORTER_SEED = {EXPORTER_SEED} (decimal)")
mpmath.mp.dps = 40  # 40 significant digits are plenty for 16 hexadecimal digits
# frac(pi) = 0.14159...; times 16^16 shifts 16 hexadecimal digits before the point
pi_hex = int(mpmath.floor(mpmath.frac(mpmath.pi) * 16 ** 16))
say(f"the first 16 hexadecimal digits of pi after the point: {pi_hex:016X}")
check(pi_hex == EXPORTER_SEED, "the exporter's seed is made of the hexadecimal "
      "digits of pi")
first = XorShift64Star(EXPORTER_SEED ^ 4)
again = XorShift64Star(EXPORTER_SEED ^ 4)
other = XorShift64Star(EXPORTER_SEED ^ 5)
numbers_first = [first.below(2 ** 64) for _ in range(1000)]
check(numbers_first == [again.below(2 ** 64) for _ in range(1000)] and
      numbers_first != [other.below(2 ** 64) for _ in range(1000)],
      "the same seed gives the same 1000 outputs, another seed other outputs")
```

Out [5]:

```
MULTIPLIER = 2685821657736338717 (decimal), EXPORTER_SEED = 2611923443488327891 (decimal)
the first 16 hexadecimal digits of pi after the point: 243F6A8885A308D3
PASS the exporter's seed is made of the hexadecimal digits of pi
PASS the same seed gives the same 1000 outputs, another seed other outputs
```

Text cell:

```
The next cell follows the bits through one step, starting from the exporter's
seed for length 4, and draws them as five rows of 64 squares (dark = 1, the
highest bit on the left): the state, the state after each of the three XOR
steps, and the output.
```

In [6]:

```
def bit_row(value):
    """The 64 bits of value, highest first, as a list of 0s and 1s."""
    return [(value >> k) & 1 for k in range(63, -1, -1)]


state = EXPORTER_SEED ^ 4
after_1 = state ^ (state >> 12)
after_2 = after_1 ^ ((after_1 << 25) & MASK)
after_3 = after_2 ^ (after_2 >> 27)  # the new state
output = (after_3 * MULTIPLIER) & MASK
check(XorShift64Star(state).below(2 ** 64) == output,
      "the class gives the output computed step by step")
fig, ax = plt.subplots(figsize=(11.0, 2.9))
ax.imshow([bit_row(v) for v in (state, after_1, after_2, after_3, output)],
          cmap="Greys", vmin=0, vmax=1.3, aspect="auto")
ax.grid(False)
ax.set_yticks(range(5), ["seed", "x ^= x >> 12", "x ^= x << 25", "x ^= x >> 27",
                         "output"])
ax.set_xticks(range(0, 64, 8), [str(63 - k) for k in range(0, 64, 8)])
ax.set_xlabel("bit number (63 = the bit worth $2^{63}$, 0 = the bit worth $2^0$)")
ax.set_title("One step of the generator xorshift64*")
save_figure(fig, "generator_step",
            "The 64 bits (dark = 1) of the exporter's seed for length 4 (top row) "
            "and of the number after each of the three shift-and-exclusive-or "
            "steps of the generator; the fourth row is the new state and the "
            "bottom row the output (new state times the constant "
            "0x2545F4914F6CDD1D, cut to 64 bits). Horizontal axis the bit "
            "number from 63 (worth $2^{63}$, left) to 0 (worth 1, right). Each "
            "step mixes the bits further.")
```

Out [6]:

```
PASS the class gives the output computed step by step
```

Out [6]:

```
Figure 01d.2 saved as Revision/textbook/figures/01d_2_generator_step.png
```

Text cell:

```
## 8. Do the numbers look random?

The next cell draws the 64 bits of the first 64 outputs of the same generator
(one row per output) and then counts how often each label 0 to 7 comes out among
the first 160,000 calls of `below(8)`. Each label should come out about
160,000/8 = 20,000 times, with a standard deviation of
$\sqrt{160000 \cdot \tfrac18 \cdot \tfrac78} \approx 132$. The check asks that
every count lies within 5 standard deviations of 20,000. A single count of a fair
```

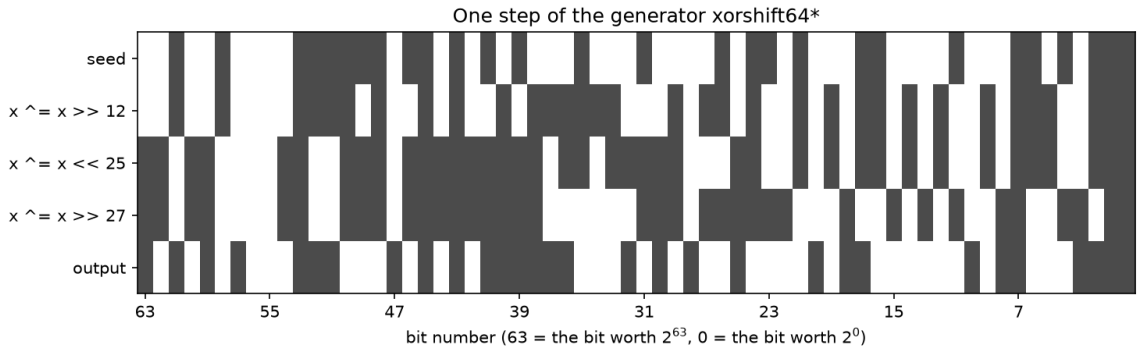

Figure 57: The 64 bits (dark = 1) of the exporter's seed for length 4 (top row) and of the number after each of the three shift-and-exclusive-or steps of the generator; the fourth row is the new state and the bottom row the output (new state times the constant 0x2545F4914F6CDD1D, cut to 64 bits). Horizontal axis the bit number from 63 (worth 2^{63}, left) to 0 (worth 1, right). Each step mixes the bits further. (Notebook 01d, figure 2.)

```
source lies further away with a probability of about 0.6 in a million, so with
8 counts a fair source fails this check with a probability of about 5 in a
million.
```

In [7]:

```
stream = XorShift64Star(EXPORTER_SEED ^ 4)
outputs = [stream.below(2 ** 64) for _ in range(64)]
fig, ax = plt.subplots(figsize=(7.2, 7.0))
ax.imshow([bit_row(v) for v in outputs], cmap="Greys", vmin=0, vmax=1.3)
ax.grid(False)
# column 0 holds the bit worth 2^63, column 63 the bit worth 2^0: label by bit number
ax.set_xticks(range(0, 64, 8), [str(63 - k) for k in range(0, 64, 8)])
ax.set_xlabel("bit number (63 = the bit worth $2^{63}$, 0 = the bit worth $2^0$)")
ax.set_ylabel("output number")
ax.set_title("The bits of the first 64 outputs")
save_figure(fig, "generator_bits",
            "The 64 bits (columns, dark = 1, the highest bit on the left) of the "
            "first 64 outputs (rows, the first at the top) of the generator "
            "of the record started from the exporter's seed for length 4. No row, "
            "column or diagonal pattern is visible: the bits look random, "
            "although every one of them is fixed by the seed.")
stream = XorShift64Star(EXPORTER_SEED ^ 4)
labels_drawn = np.array([stream.below(8) for _ in range(160000)])
label_counts = np.bincount(labels_drawn, minlength=8)  # how often 0, 1, ..., 7
spread = math.sqrt(160000 * (1 / 8) * (7 / 8))  # the standard deviation
say(f"counts of the labels 0 ... 7: {label_counts.tolist()}")
say(f"largest distance from 20000: {int(np.max(np.abs(label_counts - 20000)))}, "
    f"standard deviation {spread:.1f}")
check(bool(np.all(np.abs(label_counts - 20000) < 5 * spread)),
      "every label comes out within 5 standard deviations of 20000 times")
fig, ax = plt.subplots(figsize=(7.0, 4.0))
ax.axhspan(20000 - spread, 20000 + spread, color="0.75", alpha=0.6, zorder=3,
```

```
          label="one standard deviation")  # drawn over the bars, see-through
ax.bar(range(8), label_counts, color="tab:blue", zorder=2, label="counted")
ax.axhline(20000, color="black", lw=1, zorder=4, label="expected 20000")
ax.set_ylim(19000, 20800)
ax.set_xticks(range(8), [f"label {k}" for k in range(8)], fontsize=8)
ax.set_ylabel("number of times")
ax.set_title("160,000 labels drawn with below(8)")
ax.legend(loc="upper center", ncol=3, fontsize=8);  # above the bars, not on them
save_figure(fig, "label_histogram",
            "How often each label 0 to 7 came out in 160,000 calls of below(8) of "
            "the generator (blue bars), the expected number 20,000 (black line) "
            "and the band of one standard deviation, about 132, around it (grey); "
            "horizontal axis the label, vertical axis the number of times (the "
            "axis starts at 19,000 to make the small differences visible). The "
            "counts scatter around 20,000 as a fair random source would.")
```

Out [7]:

```
Figure 01d.3 saved as Revision/textbook/figures/01d_3_generator_bits.png
counts of the labels 0 ... 7: [20002, 20179, 20098, 19871, 20045, 19959, 20036, 19810]
largest distance from 20000: 190, standard deviation 132.3
PASS every label comes out within 5 standard deviations of 20000 times
```

Out [7]:

```
Figure 01d.4 saved as Revision/textbook/figures/01d_4_label_histogram.png
```

Text cell:

```
## 9. The GKD values that the Rust exporter gave the Wolfram check

The exporter, whose Rust source is printed inside the Wolfram check
verify_lovelock_gkd.wls, writes one **record** of signed bytes per pair of index
lists: the length $p$, the $p$ lower labels, the $p$ upper labels, and the GKD
value ($+1$, 0, or $-1$ stored as 255). It writes

- for $p = 1, 2, 3$ every pair, in the order of Mathematica's
  `Tuples[Range[0, 7], 2 p]` (all lists of $2p$ labels in dictionary order; the
  first $p$ are the lower list, the last $p$ the upper list), which is the order
  of Python's `itertools.product(range(8), repeat=2 * p)`;
- for $p = 4, 5, 6, 7$ 20,000 pairs from the generator with the seed
  `0x243F6A8885A308D3 XOR p`: an even sample (number 0, 2, 4, ...) takes $p$
  different labels (for $i = 0, \dots, p - 1$ it swaps place $i$ of the list
  $0, \dots, 7$ with place $i$ + `below(8 - i)`) as the upper list, and a
  re-ordering of them (for $i = p - 1$ down to 1 it swaps place $i$ with place
  `below(i + 1)`) as the lower list; an odd sample draws $p$ upper labels and
  then $p$ lower labels with `below(8)`.

The next cell defines the GKD rule (as in notebook 01c), regenerates all records
in this order, and keeps the lists and values for the next sections. Then it
compares the size of the bytes and their sha256 fingerprint with the record's
```

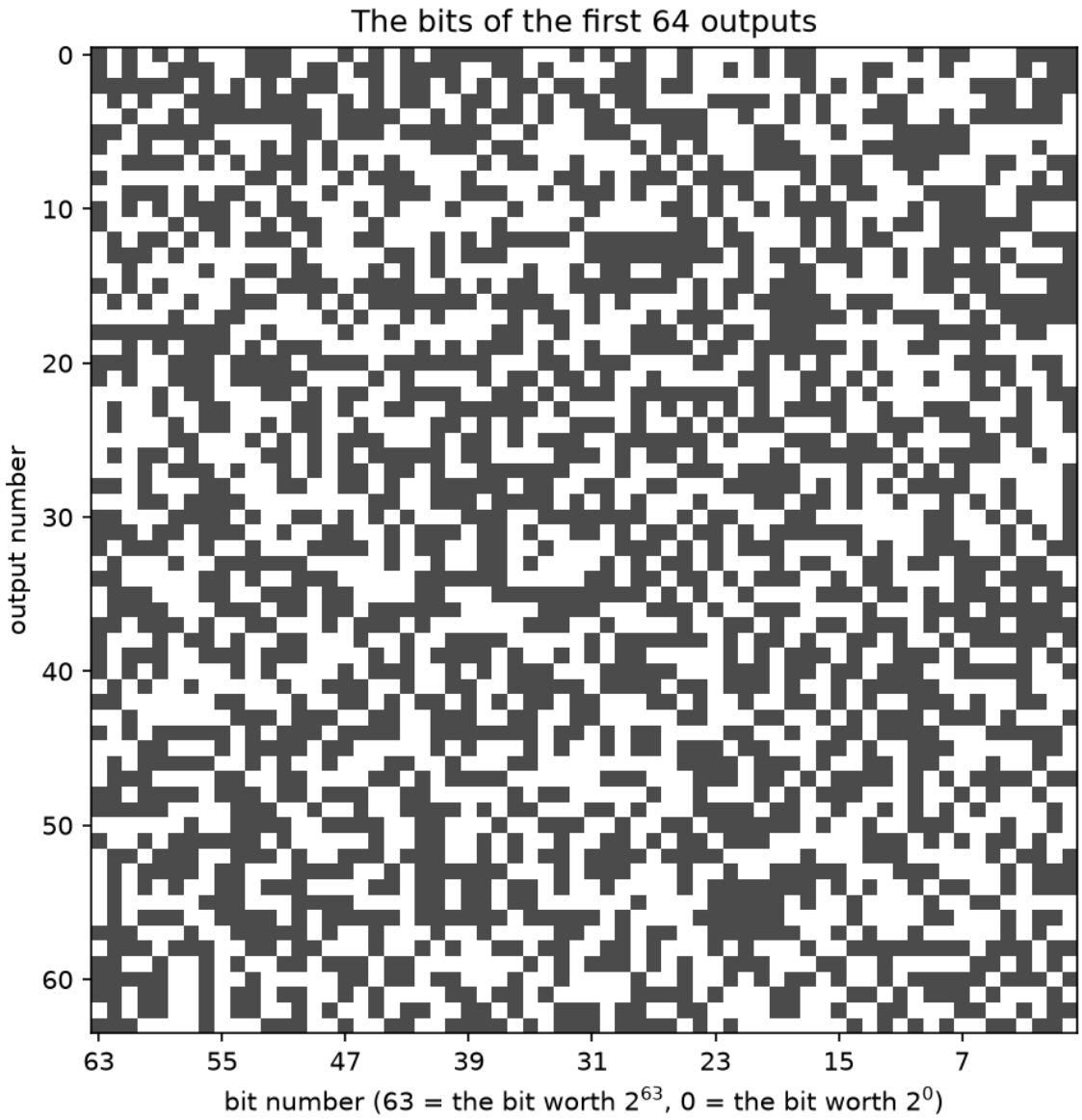

Figure 58: The 64 bits (columns, dark = 1, the highest bit on the left) of the first 64 outputs (rows, the first at the top) of the generator of the record started from the exporter's seed for length 4. No row, column or diagonal pattern is visible: the bits look random, although every one of them is fixed by the seed. (Notebook 01d, figure 3.)

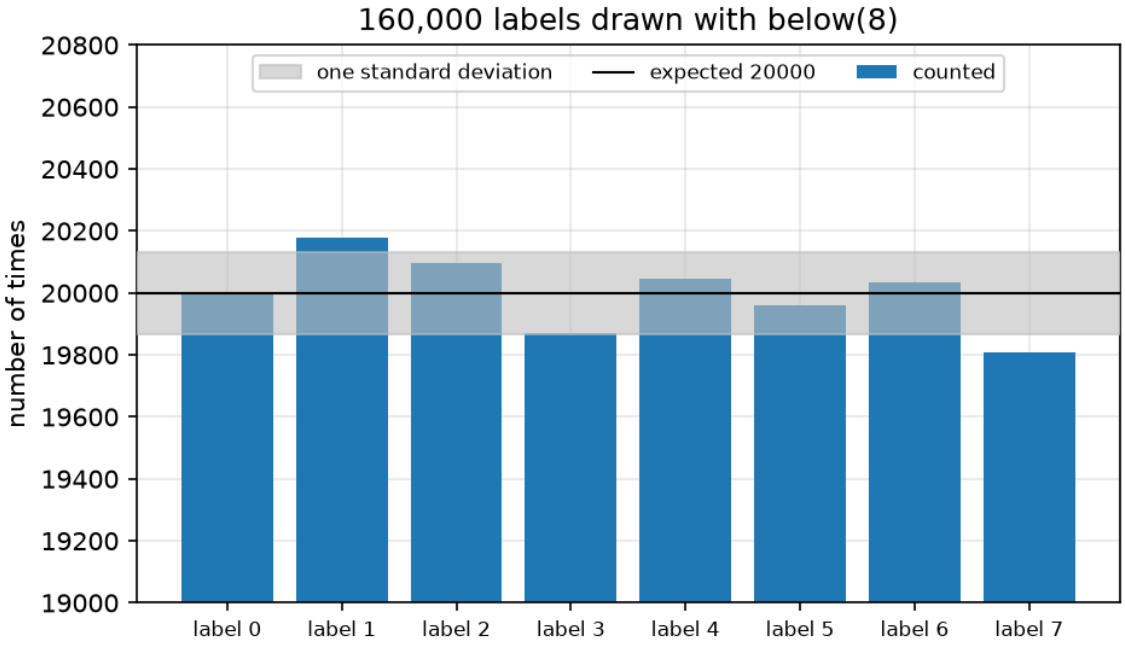

Figure 59: How often each label 0 to 7 came out in 160,000 calls of below(8) of the generator (blue bars), the expected number 20,000 (black line) and the band of one standard deviation, about 132, around it (grey); horizontal axis the label, vertical axis the number of times (the axis starts at 19,000 to make the small differences visible). The counts scatter around 20,000 as a fair random source would. (Notebook 01d, figure 4.)

‘gkdValuesSource‘. Equal fingerprints mean that every one of the 3,161,984 bytes
is the same: the same pairs, in the same order, with the same GKD values. To show
how sensitive the fingerprint is, it also prints the fingerprint after changing
a single byte.

In [8]:

```
def inversions(order):
    """The number of pairs of places i < j whose entries stand in the wrong order."""
    return sum(1 for i in range(len(order)) for j in range(i + 1, len(order))
               if order[i] > order[j])


def gkd_rule(lower, upper):
    """The program GKD (Revision/gkd_lovelock/code/src/gkd.rs) in Python."""
    place = {}
    for j, u_label in enumerate(upper):
        if u_label in place:  # a repeated upper label: two equal columns
            return 0
        place[u_label] = j
    sigma = []
    for l_label in lower:
        if l_label not in place or place[l_label] in sigma:  # missing or repeated
            return 0
        sigma.append(place[l_label])
    return 1 if inversions(sigma) % 2 == 0 else -1  # the sign of the permutation


values_bytes = bytearray()  # the regenerated file, byte by byte
pairs_of = {p: ([], []) for p in range(1, 8)}  # p -> (lower lists, upper lists)
for p in (1, 2, 3):  # every pair, in the order of Tuples[Range[0, 7], 2 p]
    for digits in itertools.product(range(8), repeat=2 * p):
        pairs_of[p][0].append(list(digits[:p]))
        pairs_of[p][1].append(list(digits[p:]))
for p in (4, 5, 6, 7):  # 20000 pseudo-random pairs each
    rng = XorShift64Star(EXPORTER_SEED ^ p)
    for sample in range(20000):
        if sample % 2 == 0:  # p different labels and a re-ordering of them
            labels = list(range(8))
            for i in range(p):
                j = i + rng.below(8 - i)
                labels[i], labels[j] = labels[j], labels[i]
            upper = labels[:p]
            lower = list(upper)
            for i in range(p - 1, 0, -1):
                j = rng.below(i + 1)
                lower[i], lower[j] = lower[j], lower[i]
        else:  # two arbitrary lists, the upper one drawn first
            upper = [rng.below(8) for _ in range(p)]
            lower = [rng.below(8) for _ in range(p)]
        pairs_of[p][0].append(lower)
        pairs_of[p][1].append(upper)
gkd_values = {}  # p -> array of the GKD values, in the order of the file
```

```
for p in range(1, 8):
    values_p = []
    for lower, upper in zip(*pairs_of[p]):
        value = gkd_rule(lower, upper)
        values_p.append(value)
        values_bytes.append(p)
        values_bytes.extend(lower)
        values_bytes.extend(upper)
        values_bytes.append(value & 0xFF)  # -1 is stored as 255
    gkd_values[p] = np.array(values_p)
wolfram_path = repository_file("Revision/gkd_lovelock/results/wolfram-gkd-report.json")
wolfram = json.loads(wolfram_path.read_text(encoding="utf-8"))
source = wolfram["gkdValuesSource"]
fingerprint = hashlib.sha256(bytes(values_bytes)).hexdigest()
record_bytes, record_sha256 = source["valuesFileBytes"], source["valuesFileSha256"]
say(f"bytes regenerated {len(values_bytes)}, in the record {record_bytes}")
say(f"sha256 regenerated:   {fingerprint}")
report("regenerated GKD values (bytes)", len(values_bytes))
report("their sha256, first 16 digits", fingerprint[:16])
say(f"sha256 in the record: {record_sha256}")
check(len(values_bytes) == source["valuesFileBytes"] and
      fingerprint == source["valuesFileSha256"],
      "the regenerated values have the size and the sha256 fingerprint of the record",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, gkdValuesSource")
changed = bytearray(values_bytes)
changed[1000] ^= 1  # change one bit of one byte
say("after changing one bit of byte 1000:")
say(f"sha256 of the changed: {hashlib.sha256(bytes(changed)).hexdigest()}")
layout = [len(pairs_of[p][0]) for p in range(1, 8)]
wolfram_checks = {c["name"]: c for c in wolfram["checks"]}
check(layout == [64, 4096, 262144, 20000, 20000, 20000, 20000] and
      wolfram_checks["gkd_values_file_layout"]["verdict"] == "PASS",
      "the layout: 64, 4096, 262144 pairs of length 1, 2, 3 and 20000 of each "
      "length 4 to 7", record="Revision/gkd_lovelock/results/"
      "wolfram-gkd-report.json, check gkd_values_file_layout")
```

Out [8]:

```
bytes regenerated 3161984, in the record 3161984
sha256 regenerated:   3ddccfa744b3708bbd3e251852236b0a2d3370e957c67bae8d94168de6d2d695
RESULT regenerated GKD values (bytes) = 3161984
RESULT their sha256, first 16 digits = 3ddccfa744b3708b
sha256 in the record: 3ddccfa744b3708bbd3e251852236b0a2d3370e957c67bae8d94168de6d2d695
PASS the regenerated values have the size and the sha256 fingerprint of the record
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, gkdValuesSource
after changing one bit of byte 1000:
sha256 of the changed: 4463b17c1baf0a3e56278f2a4834d64cc1e8901e92a0a37c6036d8e93aa0445e
PASS the layout: 64, 4096, 262144 pairs of length 1, 2, 3 and 20000 of each length 4 to 7
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, check
    gkd_values_file_layout
```

Text cell:

10. Their counts, compared with the record

For each length p the Wolfram report records in ‘measurements.gkdComparison‘: the number of pairs, the number of mismatches between GKD and the author’s determinant, the numbers of values $+1$, -1 and 0, the number of mismatches when the determinant is compared with $-$GKD instead (a *negative control*: a test that must fail, to show that the comparison can fail; it fails exactly where GKD is not zero), and for $p \geq 4$ the number of nonzero values among the 10,000 even samples (all of them, by construction).

The next cell computes the author’s determinant for all 346,304 pairs, with the Leibniz formula applied to many pairs at once as in notebook 01c (‘literal_many‘; for $p = 7$ the formula has $7! = 5040$ terms), builds the same table and compares it, entry by entry, with the record.

In [9]:

```
SIGNED = {}  # size n -> list of (permutation, sign)


def signed_permutations(n):
    if n not in SIGNED:
        SIGNED[n] = [(order, 1 if inversions(order) % 2 == 0 else -1)
                     for order in itertools.permutations(range(n))]
    return SIGNED[n]


def literal_many(lowers, uppers):
    """Det[Outer[delta, lower, upper]] for many pairs at once (Leibniz formula)."""
    p = lowers.shape[1]
    outer = lowers[:, :, None] == uppers[:, None, :]  # shape (pairs, p, p)
    values = np.zeros(len(lowers), dtype=np.int64)
    rows = np.arange(p)
    for order, order_sign in signed_permutations(p):
        values += order_sign * outer[:, rows, list(order)].all(axis=1)
    return values


measured = []
for p in range(1, 8):
    lowers = np.array(pairs_of[p][0], dtype=np.int8)
    uppers = np.array(pairs_of[p][1], dtype=np.int8)
    determinant = literal_many(lowers, uppers)
    values_p = gkd_values[p]
    row = {"p": p, "pairs": len(values_p),
           "mismatches": int((determinant != values_p).sum()),
           "plusOne": int((values_p == 1).sum()),
           "minusOne": int((values_p == -1).sum()),
           "zero": int((values_p == 0).sum()),
           "mismatchesAgainstMinusGKD": int((determinant != -values_p).sum())}
    if p >= 4:  # the even samples are the re-orderings
        row["permutationSamplesNonzero"] = int((values_p[0::2] != 0).sum())
    measured.append(row)
```

```
    # format(**row) fills the names in braces with the entries of the dictionary
    say("p = {p}: pairs {pairs:6d}, mismatches {mismatches}, +1 {plusOne:4d}, "
        "-1 {minusOne:4d}, 0 {zero:6d}, against -GKD "
        "{mismatchesAgainstMinusGKD:5d}".format(**row))
recorded = wolfram["measurements"]["gkdComparison"]
check(measured == recorded, "every number of the record's gkdComparison table is "
      "reproduced (lengths 1 to 7)",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, measurements "
             "gkdComparison; checks gkd_equals_kdelta_exhaustive_length_1 to 3 and "
             "gkd_equals_kdelta_random_length_4 to 7")
check(all(r["mismatches"] == 0 for r in measured),
      "the author's determinant equals GKD on all 346304 pairs")
check(all(r["mismatchesAgainstMinusGKD"] == r["plusOne"] + r["minusOne"] > 0
          for r in measured),
      "negative control: compared with -GKD, the test fails exactly at the "
      "nonzero values", record="Revision/gkd_lovelock/results/wolfram-gkd-report"
      ".json, check negative_control_gkd_comparison_detects_a_sign_flip")
```

Out [9]:

```
p = 1: pairs     64, mismatches 0, +1    8, -1    0, 0     56, against -GKD     8
p = 2: pairs   4096, mismatches 0, +1   56, -1   56, 0   3984, against -GKD   112
p = 3: pairs 262144, mismatches 0, +1 1008, -1 1008, 0 260128, against -GKD  2016
p = 4: pairs  20000, mismatches 0, +1 4998, -1 5024, 0   9978, against -GKD 10022
p = 5: pairs  20000, mismatches 0, +1 5024, -1 4981, 0   9995, against -GKD 10005
p = 6: pairs  20000, mismatches 0, +1 4882, -1 5120, 0   9998, against -GKD 10002
p = 7: pairs  20000, mismatches 0, +1 4935, -1 5065, 0  10000, against -GKD 10000
PASS every number of the record's gkdComparison table is reproduced (lengths 1 to 7)
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, measurements
    gkdComparison; checks gkd_equals_kdelta_exhaustive_length_1 to 3 and
    gkd_equals_kdelta_random_length_4 to 7
PASS the author's determinant equals GKD on all 346304 pairs
PASS negative control: compared with -GKD, the test fails exactly at the nonzero values
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, check
    negative_control_gkd_comparison_detects_a_sign_flip
```

Text cell:

```
The next cell draws the reproduced counts of $+1$, $-1$ and 0 for every length as
bars on a logarithmic scale and marks the record's numbers with black crosses on
top of them: every cross sits exactly on the top of its bar.
```

In [10]:

```
fig, ax = plt.subplots(figsize=(9.0, 4.6))
lengths = np.arange(1, 8)
for shift, key, color, name in [(-0.27, "plusOne", "tab:red", "value +1"),
                                (0.0, "minusOne", "tab:blue", "value -1"),
                                (0.27, "zero", "0.6", "value 0")]:
    heights = [max(row[key], 0.8) for row in measured]  # 0.8 marks a count 0
    ax.bar(lengths + shift, heights, width=0.25, color=color, label=name)
```

```
    ax.plot(lengths + shift, [max(row[key], 0.8) for row in recorded], "kx", ms=7)
ax.plot([], [], "kx", label="the record's numbers")  # an entry for the legend only
ax.set_yscale("log")
ax.set_ylim(0.5, 1e6)
ax.set_xticks(lengths)
ax.set_xlabel("length $p$ of the index lists")
ax.set_ylabel("number of pairs (logarithmic scale)")
ax.set_title("The pairs of the Wolfram check: reproduced and recorded")
ax.legend(loc="upper right", ncol=2, fontsize=8);
save_figure(fig, "record_counts",
            "The number of pairs of index lists with the GKD value $+1$ (red), "
            "$-1$ (blue) and 0 (grey) among the pairs that the exporter wrote for "
            "the Wolfram check: all pairs of the lengths 1 to 3 and 20,000 "
            "pseudo-random pairs of each length 4 to 7, as regenerated by this "
            "notebook (bars), and the numbers stored in the record (black "
            "crosses); horizontal axis the length $p$, vertical axis the number of "
            "pairs on a logarithmic scale (a count 0 is drawn as a short stub). "
            "Every cross sits on its bar.")
```

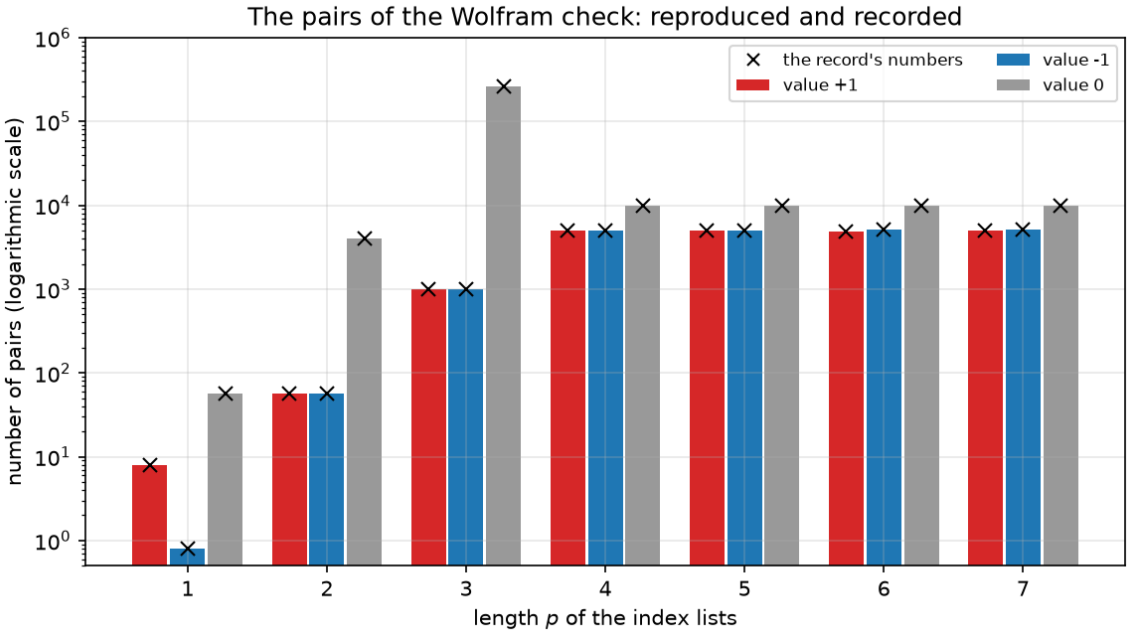

Figure 60: The number of pairs of index lists with the GKD value +1 (red), −1 (blue) and 0 (grey) among the pairs that the exporter wrote for the Wolfram check: all pairs of the lengths 1 to 3 and 20,000 pseudo-random pairs of each length 4 to 7, as regenerated by this notebook (bars), and the numbers stored in the record (black crosses); horizontal axis the length p, vertical axis the number of pairs on a logarithmic scale (a count 0 is drawn as a short stub). Every cross sits on its bar. (Notebook 01d, figure 5.)

Out [10]:

```
Figure 01d.5 saved as Revision/textbook/figures/01d_5_record_counts.png
```

Text cell:

11. The Rust self-test for the lengths 5 to 9

The function ‘compare_random(p, 8, 200000, seed)‘ of gkd.rs, called by the self-test with the seed ‘0x2545F4914F6CDD1D XOR p‘, starts the generator at ‘seed OR 1‘ and draws 200,000 samples: for each one p upper labels with ‘below(8)‘; for an even sample the lower list is the upper list re-ordered (for $i = p - 1$ down to 1 swap place i with place ‘below(i + 1)‘), for an odd sample p more labels with ‘below(8)‘. It counts the nonzero GKD values and the mismatches with the Leibniz determinant; gkd-selftest.json records both.

The next cell repeats this. GKD is computed for all samples at once with the sign shortcut of notebook 01c (‘rule_many‘: the sign of the re-ordering is the product of the signs of the two lists), and the determinant with numpy’s elimination (‘np.linalg.det‘, floating point, then rounded; the Leibniz formula would have $9!$ = 362,880 terms per matrix). The cell checks that every determinant is a whole number up to 10^{-6}. It takes 10 to 30 seconds.

In [11]:

```
def rule_many(lowers, uppers):
    """GKD for many pairs at once: 0, or sign(lower list) times sign(upper list)."""
    p = lowers.shape[1]
    upper_sorted = np.sort(uppers, axis=1)
    upper_distinct = (np.diff(upper_sorted, axis=1) != 0).all(axis=1)
    same_labels = (np.sort(lowers, axis=1) == upper_sorted).all(axis=1)
    first, second = np.triu_indices(p, 1)  # all pairs of places first < second
    parity = ((lowers[:, first] > lowers[:, second]).sum(axis=1)
              + (uppers[:, first] > uppers[:, second]).sum(axis=1)) % 2
    return np.where(upper_distinct & same_labels, np.where(parity == 0, 1, -1), 0)


check(all((rule_many(np.array(pairs_of[p][0], dtype=np.int8),
                     np.array(pairs_of[p][1], dtype=np.int8)) == gkd_values[p]).all()
           for p in range(1, 8)),
      "rule_many equals the plain GKD rule on all 346304 pairs of the exporter")
selftest = json.loads(repository_file("Revision/gkd_lovelock/results/"
                                      "gkd-selftest.json").read_text(encoding="utf-8"))
recorded_random = {e["p"]: e for e in selftest["results"] if e["mode"] == "random"}
selftest_rows = {}
whole_numbers = True
for p in (5, 6, 7, 8, 9):
    rng = XorShift64Star((SELFTEST_SEED ^ p) | 1)
    lowers, uppers = [], []
    for sample in range(200000):
        upper = [rng.below(8) for _ in range(p)]
        if sample % 2 == 0:  # a re-ordering of the upper list
            lower = list(upper)
            for i in range(p - 1, 0, -1):
                j = rng.below(i + 1)
                lower[i], lower[j] = lower[j], lower[i]
        else:  # an arbitrary lower list
            lower = [rng.below(8) for _ in range(p)]
        lowers.append(lower)
```

```
        uppers.append(upper)
    lowers = np.array(lowers, dtype=np.int8)
    uppers = np.array(uppers, dtype=np.int8)
    values_p = rule_many(lowers, uppers)
    determinants = np.concatenate([  # numpy's elimination, in 4 blocks of 50000
        np.linalg.det((lowers[s:s + 50000, :, None]
                       == uppers[s:s + 50000, None, :]).astype(float))
        for s in range(0, 200000, 50000)])
    rounded = np.rint(determinants)  # the nearest whole numbers
    whole_numbers &= bool(np.all(np.abs(determinants - rounded) < 1e-6))
    selftest_rows[p] = {"pairs": len(values_p), "nonzero": int((values_p != 0).sum()),
                        "mismatches": int((rounded != values_p).sum()),
                        "even": int((values_p[0::2] != 0).sum()),
                        "odd": int((values_p[1::2] != 0).sum())}
    say("p = {p}: {pairs} pairs, nonzero {nonzero:5d}, mismatches {mismatches}"
        .format(p=p, **selftest_rows[p]))
    say("       the record: nonzero {nonzero:5d}, mismatches {mismatches}"
        .format(**recorded_random[p]))
report("nonzero values of the self-test, p = 5 to 9",
       ", ".join(str(selftest_rows[p]["nonzero"]) for p in (5, 6, 7, 8, 9)))
check(whole_numbers, "every numpy determinant is a whole number up to 1e-6")
check(all(selftest_rows[p][key] == recorded_random[p][key]
          for p in (5, 6, 7, 8, 9) for key in ("pairs", "nonzero", "mismatches")),
      "the self-test for the lengths 5 to 9: the same numbers of pairs, nonzero "
      "values and mismatches", record="Revision/gkd_lovelock/results/"
      "gkd-selftest.json, lengths 5 to 9 (random)")
```

Out [11]:

```
PASS rule_many equals the plain GKD rule on all 346304 pairs of the exporter
p = 5: 200000 pairs, nonzero 20538, mismatches 0
       the record: nonzero 20538, mismatches 0
p = 6: 200000 pairs, nonzero  7812, mismatches 0
       the record: nonzero  7812, mismatches 0
p = 7: 200000 pairs, nonzero  1949, mismatches 0
       the record: nonzero  1949, mismatches 0
p = 8: 200000 pairs, nonzero   244, mismatches 0
       the record: nonzero   244, mismatches 0
p = 9: 200000 pairs, nonzero     0, mismatches 0
       the record: nonzero     0, mismatches 0
RESULT nonzero values of the self-test, p = 5 to 9 = 20538, 7812, 1949, 244, 0
PASS every numpy determinant is a whole number up to 1e-6
PASS the self-test for the lengths 5 to 9: the same numbers of pairs, nonzero values and
    mismatches
     reproduces Revision/gkd_lovelock/results/gkd-selftest.json, lengths 5 to 9 (random)
```

Text cell:

```
## 12. Why these numbers: the birthday problem

In the self-test an even sample is nonzero exactly when its $p$ upper labels are
all different (the lower list is then a re-ordering of them). Labels drawn one
```

```
after the other, each with 8 equally likely values, are all different with the
probability

$$P(p) = \frac{8}{8} \cdot \frac{7}{8} \cdots \frac{8 - p + 1}{8}
= \frac{8!/(8-p)!}{8^p},$$

because the second label must avoid the first (7 of 8 values), the third the
first two (6 of 8), and so on. This is the **birthday problem**: the chance that
$p$ people have $p$ different birthdays, with 8 "days" instead of 365. An odd
sample is nonzero with the probability $q(p) = (8!/(8-p)! \cdot p!)/8^{2p}$ (the
counting formula of notebook 01c divided by the number of all pairs).

The next cell computes the expected numbers $100000\,P(p) + 100000\,q(p)$ of
nonzero values in the self-test and their standard deviation
$\sqrt{100000\,P(1 - P) + 100000\,q(1 - q)}$, and checks that every recorded
number lies within 4 standard deviations (and that it is exactly 0 for $p = 9$,
where $P = 0$); "sd" in the printed lines means standard deviation. It does the
same for the odd samples of the Wolfram check
(10,000 per length). Finally it checks the classic birthday problem: among 23
people two share a birthday more often than not.
```

In [12]:

```
def all_different(p, days=8):
    """The probability that p labels drawn at random from days are all different."""
    return math.perm(days, p) / days ** p


def odd_probability(p):
    """The probability that an arbitrary pair of lists of length p is nonzero."""
    return math.perm(8, p) * math.factorial(p) / 8 ** (2 * p)


selftest_ok = True
for p in (5, 6, 7, 8, 9):
    big_p, small_q = all_different(p), odd_probability(p)
    expected = 100000 * big_p + 100000 * small_q
    spread = math.sqrt(100000 * big_p * (1 - big_p) + 100000 * small_q * (1 - small_q))
    found = recorded_random[p]["nonzero"]
    even, odd = selftest_rows[p]["even"], selftest_rows[p]["odd"]
    say(f"self-test p = {p}: recorded {found:5d}, expected {expected:8.1f}, "
        f"sd {spread:5.1f}; even samples {even}, odd {odd}")
    selftest_ok &= (abs(found - expected) < 4 * spread) if p < 9 else found == 0
check(selftest_ok, "the recorded nonzero counts of the self-test lie within 4 "
      "standard deviations of the birthday-problem expectation (0 exactly for p = 9)")
wolfram_ok = True
for row in recorded:
    p = row["p"]
    if p < 4:
        continue
    odd_nonzero = row["plusOne"] + row["minusOne"] - row["permutationSamplesNonzero"]
    expected = 10000 * odd_probability(p)
    spread = math.sqrt(10000 * odd_probability(p) * (1 - odd_probability(p)))
```

```
    say(f"Wolfram check p = {p}: nonzero odd samples {odd_nonzero:2d}, expected "
        f"{expected:5.2f}, sd {spread:4.2f}")
    wolfram_ok &= abs(odd_nonzero - expected) < 4 * spread
check(wolfram_ok, "the nonzero odd samples of the Wolfram check lie within 4 "
      "standard deviations of their expectation")
say(f"birthday problem: 22 people {all_different(22, 365):.4f}, 23 people "
    f"{all_different(23, 365):.4f} all different")
check(all_different(22, 365) > 0.5 > all_different(23, 365),
      "23 is the smallest group in which a shared birthday is more likely than not")
```

Out [12]:

```
self-test p = 5: recorded 20538, expected  20582.9, sd 128.0; even samples 20461, odd 77
self-test p = 6: recorded  7812, expected   7711.6, sd  84.4; even samples 7788, odd 24
self-test p = 7: recorded  1949, expected   1927.2, sd  43.5; even samples 1944, odd 5
self-test p = 8: recorded   244, expected    240.9, sd  15.5; even samples 243, odd 1
self-test p = 9: recorded     0, expected      0.0, sd   0.0; even samples 0, odd 0
PASS the recorded nonzero counts of the self-test lie within 4 standard deviations of the
    birthday-problem expectation (0 exactly for p = 9)
Wolfram check p = 4: nonzero odd samples 22, expected 24.03, sd 4.90
Wolfram check p = 5: nonzero odd samples  5, expected  7.51, sd 2.74
Wolfram check p = 6: nonzero odd samples  2, expected  2.11, sd 1.45
Wolfram check p = 7: nonzero odd samples  0, expected  0.46, sd 0.68
PASS the nonzero odd samples of the Wolfram check lie within 4 standard deviations of
    their expectation
birthday problem: 22 people 0.5243, 23 people 0.4927 all different
PASS 23 is the smallest group in which a shared birthday is more likely than not
```

Text cell:

```
The next cell draws the two birthday problems: on the left the probability
$P(p)$ that $p$ labels drawn from 8 are all different, with the fractions of
nonzero even samples found in the self-test for $p = 5$ to 9; on the right the
classic case of 365 days against the number of people.
```

In [13]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
p_values = np.arange(1, 10)
left.plot(p_values, [all_different(int(p)) for p in p_values], "o-", color="tab:blue",
          label="$P(p) = (8!/(8-p)!)/8^p$")
left.plot([5, 6, 7, 8, 9], [selftest_rows[p]["even"] / 100000 for p in (5, 6, 7, 8, 9)],
          "x", color="tab:red", ms=10, mew=2,
          label="self-test: nonzero fraction of the even samples")
left.set_xlabel("number $p$ of labels drawn from 8")
left.set_ylabel("probability that all are different")
left.set_title("8 labels")
left.legend(fontsize=8)
people = np.arange(1, 61)
right.plot(people, [all_different(int(k), 365) for k in people], color="tab:green")
right.axhline(0.5, color="black", lw=0.8)
```

```
right.axvline(23, color="black", ls=":", lw=1)
right.text(24, 0.8, "23 people", fontsize=9)
right.set_xlabel("number of people")
right.set_ylabel("probability of all different birthdays")
right.set_title("365 days")
fig.tight_layout()
save_figure(fig, "birthday_problem",
            "Left: the probability $P(p)$ that $p$ labels drawn at random from 8 "
            "are all different (blue circles, joined), and the fraction of the "
            "100,000 even samples of the record's self-test that gave a nonzero "
            "generalized delta, as regenerated here (red crosses, $p = 5$ to 9); "
            "horizontal axis $p$, vertical axis the probability. The crosses lie "
            "on the curve, and $P(9) = 0$. Right: the classic birthday problem, "
            "the probability that $k$ people have $k$ different birthdays among "
            "365 days, against $k$; it falls below one half (black line) at "
            "$k = 23$ (dotted line).")
```

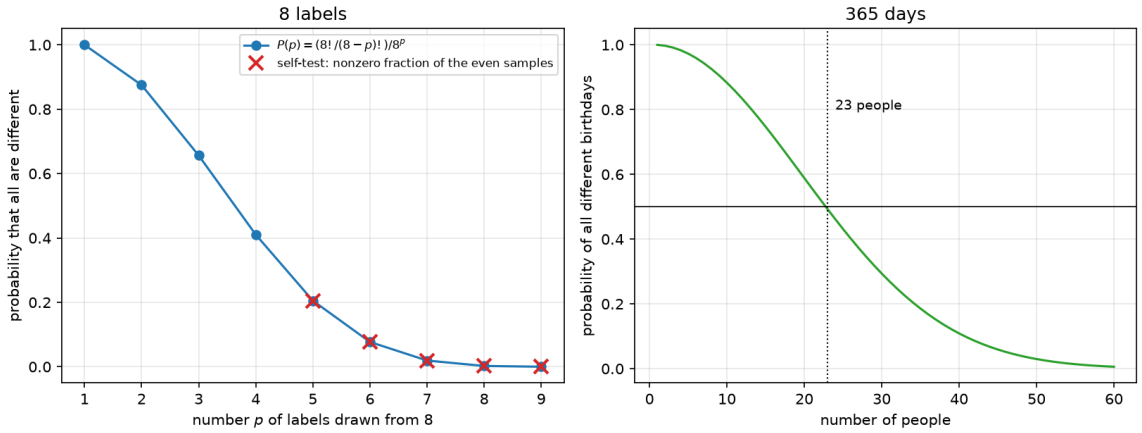

Figure 61: Left: the probability $P(p)$ that p labels drawn at random from 8 are all different (blue circles, joined), and the fraction of the 100,000 even samples of the record's self-test that gave a nonzero generalized delta, as regenerated here (red crosses, $p = 5$ to 9); horizontal axis p, vertical axis the probability. The crosses lie on the curve, and $P(9) = 0$. Right: the classic birthday problem, the probability that k people have k different birthdays among 365 days, against k; it falls below one half (black line) at $k = 23$ (dotted line). (Notebook 01d, figure 6.)

Out [13]:

```
Figure 01d.6 saved as Revision/textbook/figures/01d_6_birthday_problem.png
```

Text cell:

```
## 13. The last check

The last cell checks that the six figure files of this notebook exist in the
folder Revision/textbook/figures and prints the number of checks that passed.
```

In [14]:

```
figure_names = ["01d_1_binary_counting.png", "01d_2_generator_step.png",
                "01d_3_generator_bits.png", "01d_4_label_histogram.png",
                "01d_5_record_counts.png", "01d_6_birthday_problem.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

Out [14]:

```
PASS all 6 figure files of this notebook exist
ALL 19 CHECKS PASSED (notebook 01d)
```

Text cell:

```
## 14. What this notebook showed

- Whole numbers are strings of bits; a signed byte stores $-1$ as 255; shifts,
  exclusive or and the cut to 64 bits are simple arithmetic on the bits.
- The generator ‘xorshift64*‘ of the Revision programs, written in Python, makes
  outputs that look random but are fixed by the seed; the exporter’s seed is made
  of the hexadecimal digits of $\pi$.
- From the seeds and the recorded source code, the notebook regenerates all
  3,161,984 bytes of GKD values that the Rust exporter gave the Wolfram check;
  their sha256 fingerprint is the one in the record, so every byte is the same.
  Every number of the record’s ‘gkdComparison‘ table (346,304 pairs, the counts
  of $+1$, $-1$ and 0, no mismatch with the author’s determinant, the negative
  control) is reproduced.
- The Rust self-test for the lengths 5 to 9 is reproduced exactly: 200,000 pairs
  each, the nonzero counts 20538, 7812, 1949, 244 and 0, and no mismatch with the
  determinant.
- These counts are what the birthday problem predicts: $p$ labels drawn from 8
  are all different with the probability $8!/((8-p)!\,8^p)$, which is 0 for
  $p = 9$.
- A pseudo-random test is evidence, not a proof; the proof that GKD equals the
  author’s determinant is the four-case argument of notebook 01c. The record’s
  numbers are exactly reproducible, which is what a record must be.
```

1.54 Line-by-line walk-through of Notebook 01d

The notebook has fourteen code cells, In [1] to In [14]. As in Section 1.9, a quoted line `...)` stands for the remaining lines of a figure caption, which Section 1.53 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 1.52; its code is the set-up code explained in Section 1.9, with `NOTEBOOK_ID = "01d"`. It prints Set-up of notebook 01d complete: repository folder found, helpers defined.

In [2], binary numbers and bytes.

```
import hashlib  # the sha256 fingerprint
import itertools  # all index lists of a given length
```

```
import json  # reads the JSON reports of the record
import math  # factorials and falling factorials

import mpmath  # numbers with many digits
import numpy as np  # arrays
```

`hashlib` (part of Python) computes fingerprints such as sha256; the other modules as before.

```
number = 2026
bits = format(number, "b")  # the binary digits, a text of 0s and 1s
say(f"{number} in binary: {bits} ({len(bits)} binary digits)")
# The last digit counts 2^0, the one before 2^1, ...: reversed() starts at the end.
by_places = sum(int(digit) * 2 ** place for place, digit in enumerate(reversed(bits)))
check(by_places == number == int(bits, 2), "the binary digits of 2026 give back 2026")
```

`format(number, "b")` writes a whole number in binary. `reversed(bits)` runs through the digits from the last one, and `enumerate` numbers them 0, 1, 2, ..., so the sum adds each digit times its power of 2. `int(bits, 2)` reads a text of binary digits back as a number. Output: 2026 in binary: 11111101010 (11 binary digits), and the PASS line.

```
say(f"the largest number that 64 bits can hold: 2^64 - 1 = {2 ** 64 - 1}")
stored = (-1) & 0xFF  # the 8 bits that a signed byte uses for -1
read_back = int.from_bytes(bytes([stored]), "big", signed=True)
stored_bits = format(stored, "08b")  # 8 binary digits, zeros in front
say(f"-1 in a signed byte: {stored} = {stored_bits}, read back as {read_back}")
check(stored == 255 and read_back == -1, "a signed byte stores -1 as 255 = 11111111")
```

The largest 64-bit number is printed. Python's whole numbers have no fixed number of bits, and `(-1) & 0xFF` keeps the last 8 bits of -1 as a computer stores it, which gives 255. `bytes([stored])` makes a one-byte sequence, and `int.from_bytes(..., "big", signed=True)` reads it back as a signed byte ("big" names the order of the bytes, which does not matter for one byte). `format(stored, "08b")` writes 8 binary digits with zeros in front. Output: -1 in a signed byte: 255 = 11111111, read back as -1. The cell prints two PASS lines.

In [3], counting in binary, as a picture.

```
numbers = np.arange(32)
bit_rows = np.array([(numbers >> k) & 1 for k in range(4, -1, -1)])  # 2^4 ... 2^0
fig, ax = plt.subplots(figsize=(11.0, 2.8))
ax.imshow(bit_rows, cmap="Greys", vmin=0, vmax=1.3, aspect="auto")
ax.grid(False)
```

For the numbers 0 to 31, bit number k is the number shifted right by k places, keeping only its last bit (`& 1`); the rows are made for $k = 4, 3, 2, 1, 0$ (`range(4, -1, -1)` counts down), so the bit worth 2^4 is on top. The table is drawn in grey, dark for 1.

```
ax.set_xticks(range(32), [str(k) for k in range(32)], fontsize=7)
ax.set_yticks(range(5), [f"$2^{k}$" for k in range(4, -1, -1)])
```

```
ax.set_xlabel("the number")
ax.set_ylabel("bit")
ax.set_title("The numbers 0 to 31 in binary (dark = 1)")
save_figure(fig, "binary_counting",
            "The numbers 0 to 31 (columns) written in binary: each row is one bit, "
            ...)
```

What Figure 01d.1 shows: the bottom row alternates with every number, the row above every two numbers, the next every four: binary counting.

In [4], the bit operations.

```
def binary(value, width=8):
    """value in binary with at least width digits (zeros in front)."""
    return format(value, "b").zfill(width)
```

`zfill(width)` adds zeros in front up to the given width.

```
x = 182
say(f"x                 = {binary(x)} = {x}")
say(f"x >> 2            = {binary(x >> 2)} = {x >> 2} = {x} // 4")
say(f"x << 3            = {binary(x << 3, 11)} = {x << 3} = {x} * 8")
say(f"(x << 3) & 0xFF   = {binary((x << 3) & 0xFF)} = {(x << 3) & 0xFF} "
    f"= {x << 3} % 256")
say(f"x ^ 0b11110000    = {binary(x ^ 0b11110000)} = {x ^ 0b11110000}")
```

The example of Section 1.49, printed with the bits of every result (`0b11110000` is Python's spelling of a binary number). The output is

```
x                 = 10110110 = 182
x >> 2            = 00101101 = 45 = 182 // 4
x << 3            = 10110110000 = 1456 = 182 * 8
(x << 3) & 0xFF   = 10110000 = 176 = 1456 % 256
x ^ 0b11110000    = 01000110 = 70
```

```
MASK = 2 ** 64 - 1  # 64 ones: "& MASK" keeps the last 64 bits
generator = np.random.default_rng(12345)
rules_hold = True
for _ in range(1000):
    a = int(generator.integers(0, 2 ** 63)) * 2 + int(generator.integers(0, 2))
    b = int(generator.integers(0, 2 ** 63)) * 2 + int(generator.integers(0, 2))
    k = int(generator.integers(1, 64))
```

The mask of 64 ones, and 1000 random 64-bit numbers `a` and `b` (numpy draws whole numbers below 2^{63}, so a 64-bit number is made as twice such a number plus a random last bit) and a random shift k from 1 to 63 (seed 12345, numpy's generator, not the generator of the record).

```
    rules_hold &= (a >> k) == a // 2 ** k
    rules_hold &= ((a << k) & MASK) == (a * 2 ** k) % 2 ** 64
```

```
    rules_hold &= ((a ^ b) ^ b) == a
check(rules_hold, "shift = division or multiplication by 2^k, & MASK = remainder "
      "modulo 2^64, and XOR twice gives back the number (1000 random 64-bit numbers)")
```

The three rules of Section 1.49: a right shift is whole-number division, a left shift cut to 64 bits is multiplication taken modulo 2^{64}, and an exclusive or done twice is undone. One PASS line.

In [5], the generator `xorshift64*`.

```
MULTIPLIER = 0x2545F4914F6CDD1D  # the constant of xorshift64*
EXPORTER_SEED = 0x243F6A8885A308D3  # the first 16 hexadecimal digits of pi
SELFTEST_SEED = 0x2545F4914F6CDD1D  # the Rust self-test uses the same constant
```

The three constants of the record's programs, in hexadecimal.

```
class XorShift64Star:
    """The pseudo-random generator of the Revision programs."""

    def __init__(self, seed):
        self.state = seed
```

A **class** is a recipe for objects that carry their own data: `XorShift64Star(seed)` makes a generator object, and the function `__init__` (run when the object is made) stores the seed as its state `self.state`.

```
    def below(self, n):
        """One step of the generator; returns its output modulo n (0 ... n-1)."""
        x = self.state
        x ^= x >> 12
        x ^= (x << 25) & MASK
        x ^= x >> 27
        self.state = x
        return ((x * MULTIPLIER) & MASK) % n
```

One step of the generator, exactly as in the Rust code of Section 1.49: the three shift-and-XOR steps (the left shift cut to 64 bits), the new state stored, and the output, the state times the constant cut to 64 bits, reduced modulo n. Called with $n = 2^{64}$ it returns the whole output.

```
say(f"MULTIPLIER = {MULTIPLIER} (decimal), EXPORTER_SEED = {EXPORTER_SEED} (decimal)")
mpmath.mp.dps = 40  # 40 significant digits are plenty for 16 hexadecimal digits
# frac(pi) = 0.14159...; times 16^16 shifts 16 hexadecimal digits before the point
pi_hex = int(mpmath.floor(mpmath.frac(mpmath.pi) * 16 ** 16))
say(f"the first 16 hexadecimal digits of pi after the point: {pi_hex:016X}")
check(pi_hex == EXPORTER_SEED, "the exporter's seed is made of the hexadecimal "
      "digits of pi")
```

The constants in decimal. Then the digits of π: `mpmath.frac` is the part after the point, $0.14159\ldots$; multiplying by 16^{16} moves sixteen hexadecimal digits before the point (as multiplying by 10^{16} moves sixteen decimal digits), and `mpmath.floor` drops the rest. `{pi_hex:016X}` prints the number with 16 hexadecimal digits in capitals: 243F6A8885A308D3, the exporter's seed.

```
first = XorShift64Star(EXPORTER_SEED ^ 4)
again = XorShift64Star(EXPORTER_SEED ^ 4)
other = XorShift64Star(EXPORTER_SEED ^ 5)
numbers_first = [first.below(2 ** 64) for _ in range(1000)]
check(numbers_first == [again.below(2 ** 64) for _ in range(1000)] and
      numbers_first != [other.below(2 ** 64) for _ in range(1000)],
      "the same seed gives the same 1000 outputs, another seed other outputs")
```

Two generators with the same seed (the exporter's seed for length 4) and one with another seed: the first two give the same 1000 outputs, the third different ones. The cell prints two PASS lines.

In [6], one step, bit by bit.

```
def bit_row(value):
    """The 64 bits of value, highest first, as a list of 0s and 1s."""
    return [(value >> k) & 1 for k in range(63, -1, -1)]
```

The 64 bits of a number, from the bit worth 2^{63} down to the bit worth 1.

```
state = EXPORTER_SEED ^ 4
after_1 = state ^ (state >> 12)
after_2 = after_1 ^ ((after_1 << 25) & MASK)
after_3 = after_2 ^ (after_2 >> 27)  # the new state
output = (after_3 * MULTIPLIER) & MASK
check(XorShift64Star(state).below(2 ** 64) == output,
      "the class gives the output computed step by step")
```

The step written out with a name for every intermediate number, starting from the exporter's seed for length 4; the class must give the same output. One PASS line.

```
fig, ax = plt.subplots(figsize=(11.0, 2.9))
ax.imshow([bit_row(v) for v in (state, after_1, after_2, after_3, output)],
          cmap="Greys", vmin=0, vmax=1.3, aspect="auto")
ax.grid(False)
ax.set_yticks(range(5), ["seed", "x ^= x >> 12", "x ^= x << 25", "x ^= x >> 27",
                         "output"])
ax.set_xticks(range(0, 64, 8), [str(63 - k) for k in range(0, 64, 8)])
ax.set_xlabel("bit number (63 = the bit worth $2^{63}$, 0 = the bit worth $2^0$)")
ax.set_title("One step of the generator xorshift64*")
save_figure(fig, "generator_step",
            "The 64 bits (dark = 1) of the exporter's seed for length 4 (top row) "
            ...)
```

The five numbers as five rows of 64 squares, each row labelled with its step; the columns are labelled with the bit number (column 0 holds bit 63). What Figure 01d.2 shows: each shift-and-XOR step leaves some bits untouched and changes the others. The first step cannot change the 12 highest bits (the shifted copy has zeros there), the second the 25 lowest bits, the third the 27 highest bits; compare the rows column by column. The multiplication then mixes all the bits, and the output row shows no trace of the seed.

In [7], do the outputs look random?

```
stream = XorShift64Star(EXPORTER_SEED ^ 4)
outputs = [stream.below(2 ** 64) for _ in range(64)]
fig, ax = plt.subplots(figsize=(7.2, 7.0))
ax.imshow([bit_row(v) for v in outputs], cmap="Greys", vmin=0, vmax=1.3)
ax.grid(False)
# column 0 holds the bit worth 2^63, column 63 the bit worth 2^0: label by bit number
ax.set_xticks(range(0, 64, 8), [str(63 - k) for k in range(0, 64, 8)])
ax.set_xlabel("bit number (63 = the bit worth $2^{63}$, 0 = the bit worth $2^0$)")
ax.set_ylabel("output number")
ax.set_title("The bits of the first 64 outputs")
save_figure(fig, "generator_bits",
            "The 64 bits (columns, dark = 1, the highest bit on the left) of the "
            ...)
```

The first 64 outputs of the generator, one row of 64 bits each. What Figure 01d.3 shows: a square of dark and light squares without any row, column or diagonal pattern.

```
stream = XorShift64Star(EXPORTER_SEED ^ 4)
labels_drawn = np.array([stream.below(8) for _ in range(160000)])
label_counts = np.bincount(labels_drawn, minlength=8)  # how often 0, 1, ..., 7
spread = math.sqrt(160000 * (1 / 8) * (7 / 8))  # the standard deviation
say(f"counts of the labels 0 ... 7: {label_counts.tolist()}")
say(f"largest distance from 20000: {int(np.max(np.abs(label_counts - 20000)))}, "
    f"standard deviation {spread:.1f}")
check(bool(np.all(np.abs(label_counts - 20000) < 5 * spread)),
      "every label comes out within 5 standard deviations of 20000 times")
```

A new generator with the same seed draws 160,000 labels with `below(8)`; `np.bincount` counts how often each label 0 to 7 occurs. Each should occur about $160000/8 = 20000$ times, with the standard deviation $\sqrt{160000 \cdot \frac{1}{8} \cdot \frac{7}{8}} \approx 132.3$. Output:

```
counts of the labels 0 ... 7: [20002, 20179, 20098, 19871, 20045, 19959, 20036, 19810]
largest distance from 20000: 190, standard deviation 132.3
```

The check requires every count within 5 standard deviations of 20000. A single count of a fair source lies further away with a probability of about 0.6 in a million, so with eight counts a fair source fails with a probability of about 5 in a million.

```
fig, ax = plt.subplots(figsize=(7.0, 4.0))
ax.axhspan(20000 - spread, 20000 + spread, color="0.75", alpha=0.6, zorder=3,
           label="one standard deviation")  # drawn over the bars, see-through
ax.bar(range(8), label_counts, color="tab:blue", zorder=2, label="counted")
ax.axhline(20000, color="black", lw=1, zorder=4, label="expected 20000")
ax.set_ylim(19000, 20800)
ax.set_xticks(range(8), [f"label {k}" for k in range(8)], fontsize=8)
ax.set_ylabel("number of times")
ax.set_title("160,000 labels drawn with below(8)")
ax.legend(loc="upper center", ncol=3, fontsize=8);  # above the bars, not on them
save_figure(fig, "label_histogram",
            "How often each label 0 to 7 came out in 160,000 calls of below(8) of "
            ...)
```

The counts as blue bars, a see-through grey band of one standard deviation around 20,000 (`axhspan`; `zorder` decides what is drawn over what: the band over the bars, the black line over the band), and the expected number. The vertical axis starts at 19,000 to make the small differences visible. What Figure 01d.4 shows: the eight bars scatter around 20,000, most of them within or near the grey band. The cell prints one PASS line and two figure lines.

In [8], the exporter's file, regenerated.

```
def inversions(order):
    """The number of pairs of places i < j whose entries stand in the wrong order."""
    return sum(1 for i in range(len(order)) for j in range(i + 1, len(order))
               if order[i] > order[j])
```

Inversions, as in Notebook 01a.

```
def gkd_rule(lower, upper):
    """The program GKD (Revision/gkd_lovelock/code/src/gkd.rs) in Python."""
    place = {}
    for j, u_label in enumerate(upper):
        if u_label in place:  # a repeated upper label: two equal columns
            return 0
        place[u_label] = j
    sigma = []
    for l_label in lower:
        if l_label not in place or place[l_label] in sigma:  # missing or repeated
            return 0
        sigma.append(place[l_label])
    return 1 if inversions(sigma) % 2 == 0 else -1  # the sign of the permutation
```

The fast rule of Notebook 01c, In [5] (Section 1.48), in a shorter form: the cases (c) and (a) are tested in one condition (`or`), and the sign is computed directly.

```
values_bytes = bytearray()  # the regenerated file, byte by byte
pairs_of = {p: ([], []) for p in range(1, 8)}  # p -> (lower lists, upper lists)
for p in (1, 2, 3):  # every pair, in the order of Tuples[Range[0, 7], 2 p]
    for digits in itertools.product(range(8), repeat=2 * p):
        pairs_of[p][0].append(list(digits[:p]))
        pairs_of[p][1].append(list(digits[p:]))
```

A `bytearray` is a list of bytes that can grow. For each length a pair of lists collects the lower and the upper lists. For $p = 1, 2, 3$ the exporter writes every pair in the order of Mathematica's `Tuples[Range[0, 7], 2 p]`, all lists of $2p$ labels in dictionary order, which is the order of `itertools.product(range(8), repeat=2 * p)`; the first p labels are the lower list (`digits[:p]`), the last p the upper list.

```
for p in (4, 5, 6, 7):  # 20000 pseudo-random pairs each
    rng = XorShift64Star(EXPORTER_SEED ^ p)
    for sample in range(20000):
        if sample % 2 == 0:  # p different labels and a re-ordering of them
```

```
            labels = list(range(8))
            for i in range(p):
                j = i + rng.below(8 - i)
                labels[i], labels[j] = labels[j], labels[i]
            upper = labels[:p]
```

For $p = 4$ to 7, a generator with the exporter's seed for that length and 20,000 samples. An even sample takes p different labels: for $i = 0$ to $p - 1$ it exchanges place i of the list $0, \ldots, 7$ with a random place from i to 7 (`i + rng.below(8 - i)`), the first p steps of a shuffle; the first p entries are the upper list.

```
            lower = list(upper)
            for i in range(p - 1, 0, -1):
                j = rng.below(i + 1)
                lower[i], lower[j] = lower[j], lower[i]
        else:  # two arbitrary lists, the upper one drawn first
            upper = [rng.below(8) for _ in range(p)]
            lower = [rng.below(8) for _ in range(p)]
        pairs_of[p][0].append(lower)
        pairs_of[p][1].append(upper)
```

Its lower list is a copy re-ordered by a Fisher-Yates shuffle (for i from $p - 1$ down to 1, exchange place i with a random place from 0 to i). An odd sample draws p upper labels and then p lower labels with `below(8)`. The order of the calls of the generator matters: it is that of the exporter's source code, which the record keeps as text inside the Wolfram check script `Revision/gkd_lovelock/verification/verify_lovelock_gkd.wls`.

```
gkd_values = {}  # p -> array of the GKD values, in the order of the file
for p in range(1, 8):
    values_p = []
    for lower, upper in zip(*pairs_of[p]):
        value = gkd_rule(lower, upper)
        values_p.append(value)
        values_bytes.append(p)
        values_bytes.extend(lower)
        values_bytes.extend(upper)
        values_bytes.append(value & 0xFF)  # -1 is stored as 255
    gkd_values[p] = np.array(values_p)
```

For every pair, in the order of the file, the GKD value and the record of $2p + 2$ bytes: the length, the lower labels, the upper labels (`extend` appends several bytes) and the value as a signed byte (`value & 0xFF` turns -1 into 255). `zip(*pairs_of[p])` pairs the lower and the upper lists.

```
wolfram_path = repository_file("Revision/gkd_lovelock/results/wolfram-gkd-report.json")
wolfram = json.loads(wolfram_path.read_text(encoding="utf-8"))
source = wolfram["gkdValuesSource"]
fingerprint = hashlib.sha256(bytes(values_bytes)).hexdigest()
record_bytes, record_sha256 = source["valuesFileBytes"], source["valuesFileSha256"]
say(f"bytes regenerated {len(values_bytes)}, in the record {record_bytes}")
```

```
say(f"sha256 regenerated:   {fingerprint}")
report("regenerated GKD values (bytes)", len(values_bytes))
report("their sha256, first 16 digits", fingerprint[:16])
say(f"sha256 in the record: {record_sha256}")
```

The Wolfram report and its field `gkdValuesSource`; the fingerprint of the regenerated bytes (`hexdigest` writes it in hexadecimal); the size and fingerprint stated in the record. The output prints both sizes, 3161984, and both fingerprints, which are the same 64 digits:

```
sha256 regenerated:   3ddccfa744b3708bbd3e251852236b0a2d3370e957c67bae8d94168de6d2d695
sha256 in the record: 3ddccfa744b3708bbd3e251852236b0a2d3370e957c67bae8d94168de6d2d695
```

```
check(len(values_bytes) == source["valuesFileBytes"] and
      fingerprint == source["valuesFileSha256"],
      "the regenerated values have the size and the sha256 fingerprint of the record",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, gkdValuesSource")
changed = bytearray(values_bytes)
changed[1000] ^= 1  # change one bit of one byte
say("after changing one bit of byte 1000:")
say(f"sha256 of the changed: {hashlib.sha256(bytes(changed)).hexdigest()}")
```

The check reproduces the record's size and fingerprint: every one of the 3,161,984 bytes is the same. To show how sensitive a fingerprint is, a copy with one bit of byte 1000 flipped gets a completely different fingerprint, beginning 4463b17c1baf0a3e.

```
layout = [len(pairs_of[p][0]) for p in range(1, 8)]
wolfram_checks = {c["name"]: c for c in wolfram["checks"]}
check(layout == [64, 4096, 262144, 20000, 20000, 20000, 20000] and
      wolfram_checks["gkd_values_file_layout"]["verdict"] == "PASS",
      "the layout: 64, 4096, 262144 pairs of length 1, 2, 3 and 20000 of each "
      "length 4 to 7", record="Revision/gkd_lovelock/results/"
      "wolfram-gkd-report.json, check gkd_values_file_layout")
```

The numbers of pairs per length, compared with the layout that the record's check `gkd_valu es_file_layout` states. The cell prints two PASS lines.

In [9], the counts of the Wolfram check.

```
SIGNED = {}  # size n -> list of (permutation, sign)
```

```
def signed_permutations(n):
    if n not in SIGNED:
        SIGNED[n] = [(order, 1 if inversions(order) % 2 == 0 else -1)
                     for order in itertools.permutations(range(n))]
    return SIGNED[n]
```

The permutations with their signs, kept once per size, as in Notebook 01c, In [3].

```
def literal_many(lowers, uppers):
    """Det[Outer[delta, lower, upper]] for many pairs at once (Leibniz formula)."""
    p = lowers.shape[1]
    outer = lowers[:, :, None] == uppers[:, None, :]  # shape (pairs, p, p)
    values = np.zeros(len(lowers), dtype=np.int64)
    rows = np.arange(p)
    for order, order_sign in signed_permutations(p):
        values += order_sign * outer[:, rows, list(order)].all(axis=1)
    return values
```

The literal determinant for many pairs at once, as in Notebook 01c, In [9] (Section 1.48); for $p = 7$ the Leibniz formula has $7! = 5040$ terms.

```
measured = []
for p in range(1, 8):
    lowers = np.array(pairs_of[p][0], dtype=np.int8)
    uppers = np.array(pairs_of[p][1], dtype=np.int8)
    determinant = literal_many(lowers, uppers)
    values_p = gkd_values[p]
    row = {"p": p, "pairs": len(values_p),
           "mismatches": int((determinant != values_p).sum()),
           "plusOne": int((values_p == 1).sum()),
           "minusOne": int((values_p == -1).sum()),
           "zero": int((values_p == 0).sum()),
           "mismatchesAgainstMinusGKD": int((determinant != -values_p).sum())}
```

For each length the author's determinant of all pairs and a row of the table with the same keys as the record's `gkdComparison`: the number of pairs, the mismatches between the determinant and GKD, the counts of $+1$, -1 and 0, and the mismatches against $-$GKD (the negative control).

```
    if p >= 4:  # the even samples are the re-orderings
        row["permutationSamplesNonzero"] = int((values_p[0::2] != 0).sum())
    measured.append(row)
    # format(**row) fills the names in braces with the entries of the dictionary
    say("p = {p}: pairs {pairs:6d}, mismatches {mismatches}, +1 {plusOne:4d}, "
        "-1 {minusOne:4d}, 0 {zero:6d}, against -GKD "
        "{mismatchesAgainstMinusGKD:5d}".format(**row))
```

For $p \geq 4$ the record also counts the nonzero values among the even samples (`values_p[0::2]` takes every second entry, starting with the first), all 10,000 by construction. A text with names in braces is filled from the dictionary by `.format(**row)` (the two stars hand the entries of the dictionary over by name). The output:

```
p = 1: pairs     64, mismatches 0, +1    8, -1    0, 0     56, against -GKD     8
p = 2: pairs   4096, mismatches 0, +1   56, -1   56, 0   3984, against -GKD   112
p = 3: pairs 262144, mismatches 0, +1 1008, -1 1008, 0 260128, against -GKD  2016
p = 4: pairs  20000, mismatches 0, +1 4998, -1 5024, 0   9978, against -GKD 10022
p = 5: pairs  20000, mismatches 0, +1 5024, -1 4981, 0   9995, against -GKD 10005
p = 6: pairs  20000, mismatches 0, +1 4882, -1 5120, 0   9998, against -GKD 10002
p = 7: pairs  20000, mismatches 0, +1 4935, -1 5065, 0  10000, against -GKD 10000
```

```
recorded = wolfram["measurements"]["gkdComparison"]
check(measured == recorded, "every number of the record's gkdComparison table is "
      "reproduced (lengths 1 to 7)",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, measurements "
             "gkdComparison; checks gkd_equals_kdelta_exhaustive_length_1 to 3 and "
             "gkd_equals_kdelta_random_length_4 to 7")
```

The whole table, every key of every row, must equal the record's (two lists of dictionaries are equal when all their entries are). This reproduces the record's table and its seven comparison checks.

```
check(all(r["mismatches"] == 0 for r in measured),
      "the author's determinant equals GKD on all 346304 pairs")
check(all(r["mismatchesAgainstMinusGKD"] == r["plusOne"] + r["minusOne"] > 0
          for r in measured),
      "negative control: compared with -GKD, the test fails exactly at the "
      "nonzero values", record="Revision/gkd_lovelock/results/wolfram-gkd-report"
      ".json, check negative_control_gkd_comparison_detects_a_sign_flip")
```

No mismatch on all 346,304 pairs; and the negative control fails exactly as often as there are nonzero values, and at least once for each length, which reproduces the record's check of the negative control. The cell prints three PASS lines.

In [10], the counts as a picture.

```
fig, ax = plt.subplots(figsize=(9.0, 4.6))
lengths = np.arange(1, 8)
for shift, key, color, name in [(-0.27, "plusOne", "tab:red", "value +1"),
                                (0.0, "minusOne", "tab:blue", "value -1"),
                                (0.27, "zero", "0.6", "value 0")]:
    heights = [max(row[key], 0.8) for row in measured]  # 0.8 marks a count 0
    ax.bar(lengths + shift, heights, width=0.25, color=color, label=name)
    ax.plot(lengths + shift, [max(row[key], 0.8) for row in recorded], "kx", ms=7)
ax.plot([], [], "kx", label="the record's numbers")  # an entry for the legend only
```

For each length three bars, the reproduced counts of $+1$, -1 and 0 (a count 0 drawn as a stub, as in Notebook 01c), and on top of each bar a black cross at the record's number.

```
ax.set_yscale("log")
ax.set_ylim(0.5, 1e6)
ax.set_xticks(lengths)
ax.set_xlabel("length $p$ of the index lists")
ax.set_ylabel("number of pairs (logarithmic scale)")
ax.set_title("The pairs of the Wolfram check: reproduced and recorded")
ax.legend(loc="upper right", ncol=2, fontsize=8);
save_figure(fig, "record_counts",
            "The number of pairs of index lists with the GKD value $+1$ (red), "
            ...)
```

What Figure 01d.5 shows: every cross sits exactly on the top of its bar. For the lengths 1 to 3 the zeros dominate, as counted in Section 1.43; for the lengths 4 to 7 the bars of $+1$ and -1 are about

5,000 each and the bar of 0 about 10,000, because half of the samples are re-orderings (value ± 1) and almost all of the other half give 0. For $p = 1$ the blue bar is a stub: no value -1.

In [11], the Rust self-test for the lengths 5 to 9.

```
def rule_many(lowers, uppers):
    """GKD for many pairs at once: 0, or sign(lower list) times sign(upper list)."""
    p = lowers.shape[1]
    upper_sorted = np.sort(uppers, axis=1)
    upper_distinct = (np.diff(upper_sorted, axis=1) != 0).all(axis=1)
    same_labels = (np.sort(lowers, axis=1) == upper_sorted).all(axis=1)
    first, second = np.triu_indices(p, 1)  # all pairs of places first < second
    parity = ((lowers[:, first] > lowers[:, second]).sum(axis=1)
              + (uppers[:, first] > uppers[:, second]).sum(axis=1)) % 2
    return np.where(upper_distinct & same_labels, np.where(parity == 0, 1, -1), 0)
```

The fast rule for many pairs with the sign shortcut of Section 1.43, as in Notebook 01c, In [9], with the parity of the total number of inversions computed in one line.

```
check(all((rule_many(np.array(pairs_of[p][0], dtype=np.int8),
                     np.array(pairs_of[p][1], dtype=np.int8)) == gkd_values[p]).all()
           for p in range(1, 8)),
      "rule_many equals the plain GKD rule on all 346304 pairs of the exporter")
```

Before it is used, `rule_many` must give the plain values of In [8] on all 346,304 pairs.

```
selftest = json.loads(repository_file("Revision/gkd_lovelock/results/"
                                      "gkd-selftest.json").read_text(encoding="utf-8"))
recorded_random = {e["p"]: e for e in selftest["results"] if e["mode"] == "random"}
selftest_rows = {}
whole_numbers = True
```

The self-test report; its random entries (lengths 5 to 9) by length.

```
for p in (5, 6, 7, 8, 9):
    rng = XorShift64Star((SELFTEST_SEED ^ p) | 1)
    lowers, uppers = [], []
    for sample in range(200000):
        upper = [rng.below(8) for _ in range(p)]
        if sample % 2 == 0:  # a re-ordering of the upper list
            lower = list(upper)
            for i in range(p - 1, 0, -1):
                j = rng.below(i + 1)
                lower[i], lower[j] = lower[j], lower[i]
        else:  # an arbitrary lower list
            lower = [rng.below(8) for _ in range(p)]
        lowers.append(lower)
        uppers.append(upper)
```

The self-test's samples, as the function `compare_random` of `gkd.rs` draws them: the generator starts at (seed XOR p) OR 1 (`|` is Python's OR); every sample draws p upper labels with `below(8)`; an even sample re-orders them by a Fisher-Yates shuffle, an odd sample draws p more labels. (Unlike the exporter, the upper labels of an even sample may repeat; then the value is 0.)

```
    lowers = np.array(lowers, dtype=np.int8)
    uppers = np.array(uppers, dtype=np.int8)
    values_p = rule_many(lowers, uppers)
    determinants = np.concatenate([  # numpy's elimination, in 4 blocks of 50000
        np.linalg.det((lowers[s:s + 50000, :, None]
                       == uppers[s:s + 50000, None, :]).astype(float))
        for s in range(0, 200000, 50000)])
    rounded = np.rint(determinants)  # the nearest whole numbers
    whole_numbers &= bool(np.all(np.abs(determinants - rounded) < 1e-6))
```

GKD for all 200,000 pairs at once, and the determinant of every matrix `Outer` by numpy's elimination (`np.linalg.det` accepts a stack of matrices), in four blocks of 50,000 to save memory; the Leibniz formula would have $9! = 362{,}880$ terms per matrix. The floating-point determinants are rounded to whole numbers, and each must lie within 10^{-6} of its whole number.

```
    selftest_rows[p] = {"pairs": len(values_p), "nonzero": int((values_p != 0).sum()),
                        "mismatches": int((rounded != values_p).sum()),
                        "even": int((values_p[0::2] != 0).sum()),
                        "odd": int((values_p[1::2] != 0).sum())}
    say("p = {p}: {pairs} pairs, nonzero {nonzero:5d}, mismatches {mismatches}"
        .format(p=p, **selftest_rows[p]))
    say("       the record: nonzero {nonzero:5d}, mismatches {mismatches}"
        .format(**recorded_random[p]))
```

For each length: the number of pairs, of nonzero values, of mismatches with the determinant, and of nonzero values among the even and among the odd samples; then a line with these numbers and a line with the record's. The output shows for $p = 5$ to 9 the nonzero counts 20538, 7812, 1949, 244, 0, each equal to the record's, and no mismatch.

```
report("nonzero values of the self-test, p = 5 to 9",
       ", ".join(str(selftest_rows[p]["nonzero"]) for p in (5, 6, 7, 8, 9)))
check(whole_numbers, "every numpy determinant is a whole number up to 1e-6")
check(all(selftest_rows[p][key] == recorded_random[p][key]
          for p in (5, 6, 7, 8, 9) for key in ("pairs", "nonzero", "mismatches")),
      "the self-test for the lengths 5 to 9: the same numbers of pairs, nonzero "
      "values and mismatches", record="Revision/gkd_lovelock/results/"
      "gkd-selftest.json, lengths 5 to 9 (random)")
```

The RESULT line with the five counts, and the checks: whole-number determinants, and the record's numbers of pairs, nonzero values and mismatches reproduced exactly. The cell takes 10 to 30 seconds and prints three PASS lines.

In [12], the birthday problem.

```
def all_different(p, days=8):
    """The probability that p labels drawn at random from days are all different."""
    return math.perm(days, p) / days ** p


def odd_probability(p):
    """The probability that an arbitrary pair of lists of length p is nonzero."""
    return math.perm(8, p) * math.factorial(p) / 8 ** (2 * p)
```

$P(p) = (n!/(n-p)!)/n^p$ for n "days" (8 unless given), and $q(p) = N_{\neq 0}(8,p)/8^{2p}$, the formulas of Section 1.50.

```
selftest_ok = True
for p in (5, 6, 7, 8, 9):
    big_p, small_q = all_different(p), odd_probability(p)
    expected = 100000 * big_p + 100000 * small_q
    spread = math.sqrt(100000 * big_p * (1 - big_p) + 100000 * small_q * (1 - small_q))
    found = recorded_random[p]["nonzero"]
    even, odd = selftest_rows[p]["even"], selftest_rows[p]["odd"]
    say(f"self-test p = {p}: recorded {found:5d}, expected {expected:8.1f}, "
        f"sd {spread:5.1f}; even samples {even}, odd {odd}")
    selftest_ok &= (abs(found - expected) < 4 * spread) if p < 9 else found == 0
check(selftest_ok, "the recorded nonzero counts of the self-test lie within 4 "
      "standard deviations of the birthday-problem expectation (0 exactly for p = 9)")
```

For each length the expected number of nonzero values and its standard deviation (the two kinds of samples are independent, so their variances, the squares of the standard deviations, add), compared with the record's count; for $p = 9$ the count must be exactly 0. The output:

```
self-test p = 5: recorded 20538, expected  20582.9, sd 128.0; even samples 20461, odd 77
self-test p = 6: recorded  7812, expected   7711.6, sd  84.4; even samples 7788, odd 24
self-test p = 7: recorded  1949, expected   1927.2, sd  43.5; even samples 1944, odd 5
self-test p = 8: recorded   244, expected    240.9, sd  15.5; even samples 243, odd 1
self-test p = 9: recorded     0, expected      0.0, sd   0.0; even samples 0, odd 0
```

```
wolfram_ok = True
for row in recorded:
    p = row["p"]
    if p < 4:
        continue
    odd_nonzero = row["plusOne"] + row["minusOne"] - row["permutationSamplesNonzero"]
    expected = 10000 * odd_probability(p)
    spread = math.sqrt(10000 * odd_probability(p) * (1 - odd_probability(p)))
    say(f"Wolfram check p = {p}: nonzero odd samples {odd_nonzero:2d}, expected "
        f"{expected:5.2f}, sd {spread:4.2f}")
    wolfram_ok &= abs(odd_nonzero - expected) < 4 * spread
check(wolfram_ok, "the nonzero odd samples of the Wolfram check lie within 4 "
      "standard deviations of their expectation")
```

The same for the odd samples of the Wolfram check (10,000 per length 4 to 7): the nonzero values minus those of the even samples, compared with $10000\,q(p)$. Output: 22 against 24.03 (standard deviation 4.90), 5 against 7.51, 2 against 2.11 and 0 against 0.46.

```
say(f"birthday problem: 22 people {all_different(22, 365):.4f}, 23 people "
    f"{all_different(23, 365):.4f} all different")
check(all_different(22, 365) > 0.5 > all_different(23, 365),
      "23 is the smallest group in which a shared birthday is more likely than not")
```

The classic case: 22 people have different birthdays with the probability 0.5243, 23 people with 0.4927. Since $P(p)$ only falls as p grows (each new factor is below 1), 23 is the smallest group in which a shared birthday is more likely than not. The cell prints three PASS lines.

In [13], the two birthday problems as a picture.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.3))
p_values = np.arange(1, 10)
left.plot(p_values, [all_different(int(p)) for p in p_values], "o-", color="tab:blue",
          label="$P(p) = (8!/(8-p)!)/8^p$")
left.plot([5, 6, 7, 8, 9], [selftest_rows[p]["even"] / 100000 for p in (5, 6, 7, 8, 9)],
          "x", color="tab:red", ms=10, mew=2,
          label="self-test: nonzero fraction of the even samples")
left.set_xlabel("number $p$ of labels drawn from 8")
left.set_ylabel("probability that all are different")
left.set_title("8 labels")
left.legend(fontsize=8)
```

On the left $P(p)$ for $p = 1$ to 9, and as red crosses the fractions of nonzero values among the 100,000 even samples of the self-test (`mew=2`: crosses drawn with thick strokes).

```
people = np.arange(1, 61)
right.plot(people, [all_different(int(k), 365) for k in people], color="tab:green")
right.axhline(0.5, color="black", lw=0.8)
right.axvline(23, color="black", ls=":", lw=1)
right.text(24, 0.8, "23 people", fontsize=9)
right.set_xlabel("number of people")
right.set_ylabel("probability of all different birthdays")
right.set_title("365 days")
fig.tight_layout()
save_figure(fig, "birthday_problem",
            "Left: the probability $P(p)$ that $p$ labels drawn at random from 8 "
            ...)
```

On the right the classic case for 1 to 60 people, the line one half and a dotted line at 23 people. What Figure 01d.6 shows: the red crosses lie on the blue curve, which reaches 0 at $p = 9$; the green curve falls below one half at 23 people.

In [14], the last check.

```
figure_names = ["01d_1_binary_counting.png", "01d_2_generator_step.png",
                "01d_3_generator_bits.png", "01d_4_label_histogram.png",
                "01d_5_record_counts.png", "01d_6_birthday_problem.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

The last line is ALL 19 CHECKS PASSED (notebook 01d). The 19 checks are: 2 in In [2], 1 in In [4], 2 in In [5], 1 in In [6], 1 in In [7], 2 in In [8], 3 in In [9], 3 in In [11], 3 in In [12] and 1 in In [14].

1.55 What we proved, what we computed, what we assumed

PROVED in this chapter (by school algebra and the definitions, every derivation written out line by line; every one is also confirmed by a check of a notebook):

- numbers (Sections 1.2 to 1.5): the decimal digits of a fraction p/q end or repeat a block of at most $q-1$ digits; the formula that turns repeating digits back into a fraction; the digits of p/q in lowest terms end exactly when $q = 2^a 5^b$; $\sqrt{2}$ is not a fraction; the fractions made by $p' = p+2q$, $q' = p+q$ have $p^2 - 2q^2 = \pm 1$ and the error $|p/q - \sqrt{2}| \approx 0.35355/q^2$; the step from a floating-point number x to the next is 2^{e-53}, between $\varepsilon x/2$ and εx; $1 - \cos x = 2\sin^2(x/2)$; $\ln(xy) = \ln x + \ln y$; $e - (1+1/n)^n \approx e/(2n)$; the length factors $e^{a_4}\sin^{1/6} z$, 1, $e^{-a_4}\sin^{1/6} z$ and $\cot z$ of the author's metric and their product $\cos z$ for every a_4;

- complex numbers (Sections 1.10 to 1.13): the product rule, $(zw)^* = z^* w^*$, $zz^* = |z|^2$, $|zw| = |z|\,|w|$; Euler's formula and the error bound $e^\theta \theta^{N+1}/(N+1)!$ of its partial sums; the law of exponents $e^{i\alpha}e^{i\beta} = e^{i(\alpha+\beta)}$; multiplication by $re^{i\alpha}$ turns by α and stretches by r; $M(z)M(w) = M(zw)$, $J^2 = -I$, $\det M(z) = |z|^2$, $R(\alpha)R(\beta) = R(\alpha+\beta)$, $\det R = 1$, $R^T R = I$; the roots of unity add up to 0; conjugation leaves real numbers unchanged; $\cosh^2 - \sinh^2 = 1$ and the addition theorems of cosh and sinh; a boost keeps $-t^2 + x^2$, adds rapidities and stretches the light-like directions by $e^{\pm\varphi}$; $e^{-i\varepsilon t}$ has modulus 1 for a real and grows or decays like $e^{\gamma t}$ for an imaginary frequency $\varepsilon = i\gamma$;

- matrices and determinants (Sections 1.18 to 1.21): $(AB)v = A(Bv)$, associativity, $(AB)^T = B^T A^T$, $\mathrm{tr}(AB) = \mathrm{tr}(BA)$; an exchange of two entries flips the sign of a permutation, half of the permutations are even, and a permutation and its inverse have the same sign; the determinants of diagonal, permutation, signed permutation and triangular matrices; the rules 2 to 6 of determinants for every size and rule 1 for 2×2 matrices; the inverse of a 2×2 matrix; the signed area of the image of the unit square is $\det M$; $\det \gamma^a = \pm 1$ for the gamma matrices; $\det g = \cos^2 z$ and $\sqrt{|\det g|} = \sin z \cot z = \cos z$ for the author's metric, for every a_4: the growth of space and the exponential deflation of the extra times cancel;

- index notation (Sections 1.26 to 1.29): the rules of the Kronecker delta, $\delta^a{}_a = 8$; $\eta^{ab}\eta_{bc} = \delta^a{}_c$; lowering changes the signs of the four time-like components; the squared lengths of the examples; in the signature (4,4) exactly half of all random vectors are space-like (with one assumed fact of probability); $S_{ab}A^{ab} = 0$ and its consequence for squared expressions; signed permutation matrices are orthogonal; $(\gamma^a)^T = \eta_{aa}\gamma^a$; $\gamma^a\gamma_a = 8I$, $\gamma^a\gamma^b\gamma_a = -6\gamma^b$, $\mathrm{tr}(\gamma^a\gamma^b) = 16\eta^{ab}$, all from the Clifford relation; the line element $ds^2 = \epsilon^2(\cot^2 z + 3e^{2a_4}s - 1 - 3b_4 e^{-2a_4}s)$ of the step $dx^\mu = \epsilon$, which is $\epsilon^2(6s\sinh(2a_4) - 1 + \cot^2 z)$ at $b_4 = 1$, and the place $a_4^*(z)$ where it then changes from time-like to space-like;

- eigenvalues (Sections 1.34 to 1.37): the eigenvalues add up to the trace and multiply to the determinant; the eigenvalues and eigenvectors of the 2×2 example and the ellipse with half-axes 3 and 1; the eigenvalues $2 - 2\cos(k\pi/(n+1))$ and the sine-wave eigenvectors of the chain matrix; $e^{\pm i\alpha}$ for rotations and $e^{\pm\varphi}$ for boosts; complex eigenvalues of real matrices come in conjugate pairs; symmetric and Hermitian matrices have real eigenvalues, and eigenvectors

with different eigenvalues are perpendicular; the quadratic form in the eigenvector coordinates; Sylvester's law of inertia (with one assumed fact of linear algebra); the signature (4,4) of η and of the author's metric for every a_4 and z; the eigenvalues ± 1 or $\pm i$ of the gamma matrices, eight times each; $\gamma(v)^2 = Q(v)I$, and $\gamma(v)$ is nilpotent but not zero for a light-like v; B is Hermitian exactly when its imaginary part is antisymmetric; the factor $|\lambda_2/\lambda_1|$ of power iteration;

- the generalized Kronecker delta (Sections 1.42 to 1.44): the four cases, so the fast rule of the program GKD equals the author's determinant for every pair of index lists; the two-index formula and the counts 56, 56, 3984; the number $n!/(n-p)! \cdot p!$ of nonzero values; the sign shortcut $\delta^U_L = \mathrm{sign}(L)\,\mathrm{sign}(U)$ (with rule 1 assumed); the vanishing with nine indices in eight dimensions, so the Lovelock sum stops at $k = 3$; the contraction identity with the factor $n - p + 1$ and the factors $8 - 2k = 6, 4, 2$ of the Lovelock trace identities; the Levi-Civita product formula with $(n-p)!$; $\det \eta = +1$;

- binary numbers and the record's tests (Sections 1.49 and 1.50): a right shift is whole-number division by 2^k, a left shift multiplication by 2^k, an exclusive or done twice is undone; the exporter's file has 3,161,984 bytes; the birthday probability $P(p) = (8!/(8-p)!)/8^p$ (with the assumed rule that independent chances multiply).

COMPUTED by the notebooks (each number in the cell named; where a Revision record is reproduced, the record file and its key or check):

- Notebook 01f: the periods of $1/q$ for $q = 2$ to 300, 23 denominators with the full period $q - 1$ (In [4]); the error 1.84×10^{-9} of 19601/13860 and the limit 0.353553 of the error times q^2 (In [5]); $\sqrt{2}$ between two fractions 2^{-60} apart (In [6]); 0.1 stored as $3602879701896397/2^{55}$, $\varepsilon = 2^{-52}$ and the step formula at 2301 numbers (In [7]); the largest relative errors 4.4×10^{-16} and 1.0 of the two formulas for $(1 - \cos x)/x^2$ (In [9]); $e/(2n)$ within 1 per cent and the floating-point error $e - 1$ at $n = 10^{16}$ (In [12]); $\ln 1000 = 6.9078$ (In [12]); the length factors and their product (In [11]; reproduces `Revision/gkd_lovelock/results/curvature.json`, key `sqrtAbsDetG`, and `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `sqrt_abs_det_g`).

- Notebook 01b: the error 4.1×10^{-14} of 21 terms of the series of e^{2i} (In [5]); the factor $\theta/(N+2)$ within 10 per cent (In [6]); $e^{i\pi} = -1$ with a leftover imaginary part of size 1.0×10^{-51} at 50 digits (In [7]); the rotations, roots of unity, boosts and waves in numbers (In [8] to In [14]). It reads no record.

- Notebook 01a: the determinants 2, 15, -35, -24, 730, 15445, 60109 of seven random matrices, three ways (In [10]); $\det M = -192$, $\det N = 128$, $\det(MN) = -24576$ (In [11]); the signed areas of 1000 random matrices (In [12]); $16! = 20922789888000$ (In [13]); the eight gamma matrices are signed permutation matrices with determinant 1 whose squares are $\pm I$ (In [14], In [15]; reproduces `Revision/algebra/reports/python-algebra.json`, checks `reality_signed_permutations` and `clifford_relation`); $\det g = b_4^3 \cos^2 z$ exactly and at 900 points to 2.9×10^{-15} (In [17], In [18]; reproduces `Revision/gkd_lovelock/results/curvature.json`, key `sqrtAbsDetG`, and `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `sqrt_abs_det_g`).

- Notebook 01e: η from the record (In [3]; reproduces `python-algebra.json`, check `coordinate_map`); the fractions 0.5011 and 0.8177 of space-like random vectors in the signatures (4,4) and (3,1) (In [6]); all 64 Clifford relations and the symmetry pattern (In [10]; reproduces the checks `clifford_relation` and `symmetry_pattern` of the same report); the three contractions (In [12]); the metric and its inverse (In [14]; reproduces `curvature.json`, key `metricDiagonal`, and `python-lovelock-report.json`, check `metric_inverse`); the sign of ds^2 on a grid of 301×300 points (In [15]).

- Notebook 01g: the eigenvalues of T_{10} within 8.9×10^{-16} of the formula (In [5]); 200 random 5×5 matrices with conjugate pairs (In [6]); 300 random symmetric matrices with real eigenvalues, and 67.7 per cent complex eigenvalues of 300 non-symmetric ones (In [8]); the signature (4,4) of η (In [10]; reproduces `python-algebra.json`, check `coordinate_map`); Sylvester's law for 2000 random coordinate changes, eigenvalue sizes 1.4×10^{-8} to 37.0 (In [10]); the signature (4,4) of the author's metric at 9150 grid points and the product $b_4^3 \cos^2 z$ (In [11]; reproduces `python-lovelock-report.json`, check `sqrt_abs_det_g`); the eigenvalues of the gamma matrices (In [13]; the checks `clifford_relation` and `symmetry_pattern`); $\gamma(v)^2 = Q(v)I$ for 104 vectors (In [14]); the signature (8,8) of B (In [16]; reproduces `python-algebra.json`, check `B_hermitian_involution_signature`, and `Revision/theory/reports/python-field-theory.json`, check `B_properties`); the power-iteration factors 0.333333 and 0.585786 to 0.585789 (In [18]).

- Notebook 01c, first part: the author's definition, verbatim as in the record's file `PROVENANCE_OF_THE_COMPUTATION.md`, and their cells In[87], In[32] and In[29] as listed in the record's file `notebook-input-cells.txt` (In [2]; both files lie in the folder `Revision/gkd_lovelock/results`, as do the reports named below); the record's examples (In [6]; reproduces the check `gkd_examples` of `python-lovelock-report.json`, the check `definition_unequal_lengths_stay_unevaluated` of `wolfram-gkd-report.json` and the field `gkdSelfCheck` of `lovelock-report.json`).

- Notebook 01c, second part: the literal determinant equals the fast rule on all 266,304 pairs of the lengths 1 to 3, with the record's counts (In [8]; reproduces the checks `gkd_equals_kdelta_exhaustive_length_1`, `_length_2` and `_length_3` of `wolfram-gkd-report.json`, their counts and their 0 mismatches), and on all 16,777,216 pairs of length 4 (In [10]; reproduces `gkd-selftest.json`).

- Notebook 01c, third part: 600 pairs of nine-index lists, all 0 (In [13]; the record's check `gkd_nine_indices_in_eight_dimensions_vanish`); the contraction factors 8 to 1 and 4 to 1 (In [14]); the trace factors 6, 4, 2 (In [15]; reproduces the checks `k1_trace_equals_6_L1`, `k2_trace_equals_4_L2` and `k3_trace_equals_2_L3` of `wolfram-gkd-report.json`); the author's second route, $7!\,\delta_a^b$ (In [17]; with the declaration (4, 4, 0) of the author's cell In[29]).

- Notebook 01d: the 3,161,984 regenerated bytes and their sha256 fingerprint (In [8]; reproduces `wolfram-gkd-report.json`, field `gkdValuesSource`, and check `gkd_values_file_layout`); every number of the table `gkdComparison` and the negative control (In [9]; reproduces `wolfram-gkd-report.json`, field `measurements`, and its checks `gkd_equals_kdelta_exhaustive_length_1` to `_3`, `gkd_equals_kdelta_random_length_4` to `_7` and `negative_control_gkd_comparison_detects_a_sign_flip`); the self-test counts 20538, 7812, 1949, 244, 0 (In [11]; reproduces `gkd-selftest.json`, lengths 5 to 9); all of them within 4 standard deviations of the birthday-problem expectation, and the 23 people of the classic problem (In [12]).

ASSUMED (used, not derived here):

- school algebra and trigonometry: the laws of powers; $\cos^2 + \sin^2 = 1$, the addition theorems, the double-angle formula; the formula for the roots of a quadratic; every point at distance 1 from the origin is $(\cos\theta, \sin\theta)$; Pythagoras; the uniqueness of the factorisation into primes;
- calculus: the series of e^x, cos, sin and $\ln(1+u)$ (Chapter 2 derives the series of e^x from Taylor's theorem, which it also assumes); the triangle inequality for complex numbers;
- linear algebra: $\det(MN) = \det M \det N$ for matrices larger than 2×2; a square matrix can be undone exactly when its determinant is not 0, and $(M - \lambda I)v = 0$ has a solution $v \neq 0$ when $\det(M - \lambda I) = 0$; the fundamental theorem of algebra; the spectral theorem; more than n vectors with n components are linearly dependent; the shoelace formula for signed areas;
- probability: a continuous random quantity takes one exact value with probability 0; independent chances multiply; the expected value Nq and the standard deviation $\sqrt{Nq(1-q)}$ of a count; the fraction $1/2 + 1/\pi$ for the signature (3,1);
- the property of SHA-256 that different files have different fingerprints for all practical purposes;
- tensor calculus: raising all indices of the Levi-Civita tensor of a metric multiplies it by the sign of $\det g$ (Chapter 11);
- taken from the Revision record and derived in later chapters: the gamma matrices, the Clifford relation and the matrices C and B (Chapters 4 and 5; here read from `Revision/algebra/gammas.json` and checked); the meaning of B as the charge density (Chapters 7 and 10); the curvature and the Lovelock tensors (Chapters 3 and 11);
- the author's metric itself, the starting point of the theory. Nothing in this chapter depends on the form of the functions $a_4(x_4)$ and $b_4(x_4)$: every statement holds for every value of a_4 and every positive value of b_4, except where $b_4 = 1$ is said (the line element in the form with sinh, and the pictures drawn at $b_4 = 1$).

HYPOTHESIS and OPEN. No hypothesis enters this chapter, and it leaves no question open. It says nothing about pair creation or about matter and antimatter.

1.56 Exercises

Exercise 1. Find the decimal digits of 3/11 by long division, state the period, and turn the repeating digits back into the fraction with the formula of Section 1.2.

Answer. Start with the remainder 3. $30 = 2 \cdot 11 + 8$: digit 2, remainder 8. $80 = 7 \cdot 11 + 3$: digit 7, remainder 3, the remainder we started with. So $3/11 = 0.(27)$ with the period 2. Back: the block is $C = 27$ with $m = 2$ digits, so $x = C/(10^m - 1) = 27/99$, and dividing numerator and denominator by their greatest common divisor 9 gives 3/11.

Exercise 2. (a) Which of the numbers 0.75 and 0.2 does a computer store exactly, and why? (b) How large is the step from $x = 10^6$ to the next floating-point number, and what is the step divided by x? Compare with $\varepsilon/2$ and ε.

Answer. (a) $0.75 = 3/4$ has the denominator 2^2, a power of 2, so it is stored exactly; $0.2 = 1/5$ has the prime factor 5 in its denominator, so its binary digits do not end (Section 1.4) and it is rounded. (b) $2^{19} = 524288 < 10^6 < 2^{20} = 1048576$, so $10^6 = f \cdot 2^{20}$ with $f = 10^6/2^{20} \approx 0.954$, which lies between $\frac{1}{2}$ and 1; the step is $2^{20-53} = 2^{-33} \approx 1.164 \times 10^{-10}$. Divided by 10^6 it is 1.164×10^{-16}, which lies between $\varepsilon/2 \approx 1.110 \times 10^{-16}$ and $\varepsilon \approx 2.220 \times 10^{-16}$, as Section 1.4 proved.

Exercise 3. (a) Compute $(2 + i)(1 - 3i)$. (b) Write $1/(3 + 4i)$ as real part plus i times imaginary part, and give $|3 + 4i|$. (c) For the complex frequency $\varepsilon = 2 - 0.5i$, what is the size $|e^{-i\varepsilon t}|$ of the wave, and does it grow or decay?

Answer. (a) $(2 + i)(1 - 3i) = 2 - 6i + i - 3i^2 = 2 - 5i + 3 = 5 - 5i$ (multiply out; $i^2 = -1$). (b) $\dfrac{1}{3+4i} = \dfrac{3-4i}{(3+4i)(3-4i)} = \dfrac{3-4i}{9+16} = \dfrac{3}{25} - \dfrac{4}{25}i$, and $|3 + 4i| = \sqrt{9+16} = 5$. (c) $-i\varepsilon t = -i(2 - 0.5i)t = -2it + 0.5i^2t = -2it - 0.5t$, so $e^{-i\varepsilon t} = e^{-0.5t}e^{-2it}$ (Section 1.13); since $|e^{-2it}| = 1$, the size is $e^{-0.5t}$: the wave oscillates and decays.

Exercise 4. For the matrices A with rows $(1, 2), (0, 1)$ and B with rows $(0, 1), (1, 0)$ compute AB, BA, the commutator $[A, B]$, the anticommutator $\{A, B\}$, the traces of AB and BA, and $\det A$, $\det B$, $\det(AB)$.

Answer. Row times column: AB has the rows $(1\cdot0+2\cdot1, 1\cdot1+2\cdot0) = (2, 1)$ and $(0+1, 0+0) = (1, 0)$; BA has the rows $(0 \cdot 1 + 1 \cdot 0, 0 \cdot 2 + 1 \cdot 1) = (0, 1)$ and $(1, 2)$. So $[A, B] = AB - BA$ has the rows $(2, 0)$ and $(0, -2)$, and $\{A, B\} = AB + BA$ has the rows $(2, 2)$ and $(2, 2)$. Both traces are 2 (Section 1.18 proved that they must be equal). $\det A = 1 \cdot 1 - 2 \cdot 0 = 1$, $\det B = 0 - 1 = -1$, $\det(AB) = 2 \cdot 0 - 1 \cdot 1 = -1 = \det A \det B$ (rule 1).

Exercise 5. (a) Find the inversions and the sign of the permutation $(2, 0, 3, 1)$. (b) Compute the determinant of the matrix with the rows $(2, 1, 0)$, $(0, 1, 3)$, $(1, 0, 1)$ with the six-term formula of Section 1.20.

Answer. (a) The pairs of entries in the wrong order are $(2, 0)$, $(2, 1)$ and $(3, 1)$: three inversions, an odd number, so the sign is -1. (b) With $A_{11} = 2$, $A_{12} = 1$, $A_{13} = 0$, $A_{21} = 0$, $A_{22} = 1$, $A_{23} = 3$, $A_{31} = 1$, $A_{32} = 0$, $A_{33} = 1$: $A_{11}A_{22}A_{33} = 2$, $A_{12}A_{23}A_{31} = 3$, $A_{13}A_{21}A_{32} = 0$, $A_{12}A_{21}A_{33} = 0$, $A_{11}A_{23}A_{32} = 0$, $A_{13}A_{22}A_{31} = 0$, so $\det = 2 + 3 + 0 - 0 - 0 - 0 = 5$.

Exercise 6. With the frame metric η of the author's spacetime (Section 1.27): (a) for $v = (0, 1, 0, 0, 0, 1, 0, 0)$ (components in the order $v^0, \ldots, v^7$, so $v^1 = v^5 = 1$) compute $Q(v)$ and the lowered components v_a; (b) for $w = (0, 0, 0, 0, 1, 1, 0, 0)$ compute $Q(w)$; (c) for $u = (1, 1, 1, 1, 1, 1, 0, 0)$ compute $Q(u)$. Classify each vector. (d) Write $\delta^a{}_b\,\delta^b{}_c$ for $a = c = 4$ as an explicit sum and evaluate it.

Answer. (a) $Q(v) = (v^1)^2 - (v^5)^2 = 1 - 1 = 0$: light-like; $v_a = \eta_{aa}v^a = (0, 1, 0, 0, 0, -1, 0, 0)$, because x_5 is time-like. (b) $Q(w) = -(w^4)^2 - (w^5)^2 = -2$: time-like. (c) $Q(u) = 1+1+1+1-1-1 = 2$ (the components along x_0, x_1, x_2, x_3 count $+1$, those along x_4, x_5 count -1): space-like. (d) $\sum_{b=0}^{7} \delta^4{}_b\,\delta^b{}_4$; only $b = 4$ contributes, giving $1 \cdot 1 = 1 = \delta^4{}_4$.

Exercise 7. (a) Check that $\sum_a \sum_b S_{ab}A_{ab} = 0$ for S with the rows $(1, 4), (4, 0)$ and A with the rows $(0, -3), (3, 0)$. (b) Show, with the Clifford relation of the record's gamma matrices, that $\gamma^{(x_1)}\gamma^{(x_4)}\gamma^{(x_1)} = -\gamma^{(x_4)}$.

Answer. (a) The four terms are $1 \cdot 0 + 4 \cdot (-3) + 4 \cdot 3 + 0 \cdot 0 = 0$: the terms $(1, 2)$ and $(2, 1)$ cancel, the diagonal terms are 0 (Section 1.28). (b) $\gamma^{(x_1)}\gamma^{(x_4)} = -\gamma^{(x_4)}\gamma^{(x_1)}$ (different gamma

matrices anticommute), so $\gamma^{(x_1)}\gamma^{(x_4)}\gamma^{(x_1)} = -\gamma^{(x_4)}\gamma^{(x_1)}\gamma^{(x_1)} = -\gamma^{(x_4)}(\gamma^{(x_1)})^2 = -\gamma^{(x_4)}I = -\gamma^{(x_4)}$ (associativity; $(\gamma^{(x_1)})^2 = \eta_{11}I = +I$ for the space-like x_1).

Exercise 8. Let A have the rows $(2, 1-i)$ and $(1+i, 3)$. (a) Show that A is Hermitian. (b) Find its characteristic polynomial and its eigenvalues. (c) Find an eigenvector for each eigenvalue and check that the two are perpendicular ($w^\dagger v = 0$).

Answer. (a) The transpose has the rows $(2, 1+i)$ and $(1-i, 3)$; conjugating every entry gives back A, so $A^\dagger = A$. (b) $\det(A-\lambda I) = (2-\lambda)(3-\lambda)-(1-i)(1+i) = \lambda^2-5\lambda+6-2 = \lambda^2-5\lambda+4 = (\lambda-1)(\lambda-4)$, using $(1-i)(1+i) = 1-i^2 = 2$; the eigenvalues 1 and 4 are real, as Theorem 1 of Section 1.36 requires. (c) For 4: the first row of $(A-4I)v = 0$ reads $-2v_1+(1-i)v_2 = 0$; with $v_2 = 2$, $v = (1-i, 2)$ (second row: $(1+i)(1-i)-2 = 2-2 = 0$). For 1: $v_1+(1-i)v_2 = 0$; with $v_2 = 1$, $w = (-1+i, 1)$ (second row: $(1+i)(-1+i)+2 = (i^2-1)+2 = 0$). Then $w^\dagger v = (-1-i)(1-i)+1\cdot 2 = (-1+i-i+i^2)+2 = -2+2 = 0$.

Exercise 9. Take the author's metric at $a_4 = 0$, $b_4 = 1$ and $z = \pi/4$. (a) Write its eight diagonal entries with $s = \sin^{1/3}(\pi/4)$. (b) Compute the product of the eight entries and compare it with $\cos^2(\pi/4)$. (c) What is its signature, and what happens to its three extra-time entries when a_4 grows to $\ln 2$?

Answer. (a) $\sin(\pi/4) = \cos(\pi/4) = \sqrt{2}/2$ and $\cot(\pi/4) = 1$, so the entries, in the order $x_0, \ldots, x_7$, are $1, s, s, s, -1, -s, -s, -s$ with $s = (\sqrt{2}/2)^{1/3} = 2^{-1/6} \approx 0.8909$ (since $\sqrt{2}/2 = 2^{-1/2}$ and $(2^{-1/2})^{1/3} = 2^{-1/6}$). (b) The product is $1 \cdot s^3 \cdot (-1) \cdot (-s)^3 = s^6 = (2^{-1/6})^6 = 2^{-1} = \frac{1}{2}$, and $\cos^2(\pi/4) = (\sqrt{2}/2)^2 = \frac{1}{2}$: equal, as Section 1.21 proved. (c) Four positive entries (1 and s three times) and four negative ones: the signature (4,4). At $a_4 = \ln 2$ the extra-time entries $-b_4 e^{-2a_4}s$ are multiplied by $e^{-2\ln 2} = \frac{1}{4}$ (and the space entries by 4): they shrink towards 0 but keep their sign, so the signature stays (4,4) (Section 1.36).

Exercise 10. (a) Compute $\delta^{x_2x_3x_1x_4}_{x_1x_2x_3x_4}$ (lower list (x_1, x_2, x_3, x_4), upper list (x_2, x_3, x_1, x_4)) with the four cases of Section 1.42. (b) Compute $\delta^{x_1x_2x_3x_4}_{x_1x_2x_3x_5}$. (c) How many of the 8^{10} pairs of index lists of length 5 over the eight labels have a nonzero value, and how many of them are $+1$?

Answer. (a) No label repeats, and both lists hold $x_1, \ldots, x_4$: case (d). The lower labels stand at the places 3, 1, 2, 4 of the upper list (x_1 is the third upper label, x_2 the first, x_3 the second, x_4 the fourth), so $\sigma = (3, 1, 2, 4)$, with the inversions $(3, 1)$ and $(3, 2)$: even, value $+1$. (b) The lower label x_5 is not among the upper labels: case (c), value 0. (c) $N_{\neq 0}(8, 5) = 8\cdot 7\cdot 6\cdot 5\cdot 4\cdot 5! = 6720\cdot 120 = 806400$, of which half, 403200, are $+1$ (Section 1.43).

Exercise 11. Use the contraction identity of Section 1.44 twice to compute $\sum_c \sum_d \delta^{acd}_{bcd}$ in eight dimensions, and check the result by counting the nonzero terms.

Answer. First contract d in the delta with $p = 3$ indices: $\sum_d \delta^{acd}_{bcd} = (8-3+1)\,\delta^{ac}_{bc} = 6\,\delta^{ac}_{bc}$. Then contract c in the delta with $p = 2$ indices: $\sum_c \delta^{ac}_{bc} = (8-2+1)\,\delta^a_b = 7\,\delta^a_b$. Together $\sum_c \sum_d \delta^{acd}_{bcd} = 42\,\delta^a_b$. Check by counting: for $a \neq b$ every term is 0: if c or d equals a, the upper list repeats a (case (b)); if c or d equals b, the lower list repeats b (case (a)); otherwise c and d differ from a and b, so the lower label b is not among the upper labels a, c, d (case (c)). For $a = b$ the term is $+1$ exactly when c and d differ from a and from each other (equal lists of three different labels) and 0 otherwise; there are 7 choices for c and then 6 for d, so the sum is $7 \cdot 6 = 42$.

Exercise 12. (a) In the record's self-test of length 4, what is the probability that the 4 upper labels of a sample, drawn with `below(8)`, are all different? (b) About how many of 100,000 such samples have four different labels? (c) Why can no sample of length 9 have a nonzero value?

Answer. (a) $P(4) = \frac{8}{8} \cdot \frac{7}{8} \cdot \frac{6}{8} \cdot \frac{5}{8} = \frac{1680}{4096} \approx 0.4102$ (Section 1.50). (b) About $100000 \cdot 0.4102 \approx 41{,}000$ (the expected value NP). (c) Nine labels taken from eight must repeat one (the pigeonhole principle), so case (a) or (b) of Section 1.42 applies and the value is 0; this is why the record's count for $p = 9$ is exactly 0.

2 Mathematics from zero II: several variables, ODEs, Euler and RK4, convergence, the shooting method

This chapter teaches the second half of the mathematics that the book needs: differential equations and how a computer solves them, how to tell how accurate a computed answer is, how to find the special energies of an equation with conditions at both ends of an interval (the shooting method), and how to differentiate a function of several variables. Every idea is built from school algebra and one-variable calculus, and every idea is tried out at once in a notebook. The chapter has four worked examples, Notebooks 02a, 02b, 02c and 02d; the last one rewrites, line by line in Python, the method with which the Rust program of the Revision record solves the Kohn-Sham equations of this book, and reproduces that program's published numbers.

2.1 What this chapter is for

The book's physics lives in equations that say how quantities change. Two places in the book need the tools of this chapter.

First, the author's metric. The metric (introduced from zero in Chapter 3; here we only need its formulas) is a list of eight numbers $g_{00}, \ldots, g_{77}$ that turn small coordinate steps along the eight directions $x_0, \ldots, x_7$ into true lengths. Its entries depend on TWO variables: the time x_4 and the hidden coordinate x_0. Lengths along the three directions of ordinary space, x_1, x_2, x_3, carry the **scale factor** $e^{a_4} \sin^{1/6} z$, and lengths along the three **extra times** x_5, x_6, x_7 carry the scale factor $b_4^{1/2} e^{-a_4} \sin^{1/6} z$, where $a_4(x_4)$ and $b_4(x_4) > 0$ are functions of the time ($b_4 = 1$ in the background of the Revision record) and $z = 6Hx_0$ (with the author's positive constant H) is the author's variable for the hidden direction. When a_4 grows and b_4 stays constant, ordinary space inflates and the three extra times **deflate exponentially**. To say how fast each length changes with x_4 or with x_0 we need **partial derivatives**, the derivatives of a function of several variables (Sections 2.18 and 2.19).

Second, the Kohn-Sham equations of the book (Chapters 14 and 15). They are two coupled first-order differential equations for the two components of an **orbital** (the wave function of one particle of the model) along the hidden coordinate, with one condition at each end of an interval, and they have solutions only for special energies, the **energy levels**. The Rust program of the Revision record solves them by the **shooting method**: it guesses an energy, integrates the equations from one end to the other with the **classical fourth-order Runge-Kutta method (RK4)** in 900 steps, measures by how much the condition at the far end fails, and corrects the energy with Newton's method. Sections 2.2 to 2.7 teach RK4 and the measurement of its accuracy, Sections 2.12 and 2.13 the shooting method, and Section 2.24 the special form of shooting (with the **Prüfer angle**) that the program uses.

The four notebooks of the chapter are:

notebook	what it computes	Revision record it reproduces
02a	Euler, midpoint and RK4 on the deflating extra-time scale factor and on an oscillator; orders 1, 2, 4 on log-log plots; the rounding floor; the predicted error at the step of the Revision solver; Richardson's rule	`Revision/kohn_sham/results/parameters.json` (900 RK4 steps on an interval of length 3; the history constants $A = H = 1$)
02b	the shooting method for a vibrating string and for a quantum well; bisection against the secant rule; exact levels with 30 digits	none (exact formulas only)
02c	partial derivatives, finite differences, the chain rule for $z = 6Hx_0$, the growth and deflation rates of the author's metric, the volume factor, all 25 nonzero Christoffel entries	`Revision/gkd_lovelock/results/curvature.json` and `Revision/field_equations_a4/reports/wolfram-a4-report.json`
02d	the Prüfer-angle shooting and the Newton root finder of the Revision Kohn-Sham solver, rewritten line by line; all 54 levels of its free spectrum	`Revision/kohn_sham/results/spectrum/free-k0-analytic.csv` and two checks of `Revision/kohn_sham/reports/ks-rust-solver.json`

Every statement of the chapter carries one of the five labels of Chapter 0. Most are PROVED here by elementary algebra (and confirmed by a check of a notebook) or COMPUTED by a notebook (with the cell that computes the number and, where it reproduces a Revision record, the record file and its check). A few standard theorems of calculus are quoted without proof; we label them ASSUMED, as the book does for everything it does not derive. Two physical inputs are ASSUMED as well and are named where they enter: the linear history $a_4 = AHx_4$, which is a PRESCRIBED BACKGROUND, and the mirror (brane) conditions of the Kohn-Sham model. No HYPOTHESIS enters this chapter.

2.2 Differential equations

The derivative. For a function $y(t)$ of one variable the derivative is

$$y'(t) = \frac{dy}{dt}(t) = \lim_{h \to 0} \frac{y(t+h) - y(t)}{h},$$

the slope of the graph of y at t, or the rate of change of y per unit of t. We write y'' for the derivative of y' (the second derivative), and $y^{(n)}$ for the n-th derivative.

An ordinary differential equation (ODE) is an equation of the form

$$y'(t) = f(t, y(t))$$

for an unknown function $y(t)$, where f is a known rule that produces a number from t and y, the **right-hand side**. It is "ordinary" because the unknown depends on one variable only. Together with a starting value $y(0) = y_0$ it is an **initial-value problem**. When f is smooth (it has continuous derivatives), the initial-value problem has exactly one solution, at least for some time after the start: this is the theorem of Picard and Lindelöf, which we quote without proof (ASSUMED: a standard theorem of calculus). A computer tries to approximate that one solution.

Systems. Several unknown functions that change together, say a position $x(t)$ and a velocity $v(t)$, form a **system**. We collect them into one list $Y = (x, v)$, a **vector**, and write the system again as $Y' = f(t, Y)$, where now f produces a list of the same length. An equation with a second derivative becomes a system of first-order equations by naming the first derivative: $x'' = -x$ becomes $x' = v$ and $v' = -x$, because $v' = (x')' = x'' = -x$.

Problem A: the deflating extra time. We now derive the first equation of Notebook 02a from the author's metric. The entry of the metric for the extra time x_5 is

$$g_{55} = -e^{-2a_4} \sin^{1/3} z.$$

(This is the author's metric entry, which Notebook 02c reads from the Revision record `Revision /gkd_lovelock/results/curvature.json`, key `metricDiagonal`; its minus sign marks a time-like direction.)

$$s_5 = \sqrt{|g_{55}|} = \sqrt{e^{-2a_4} \sin^{1/3} z}$$

(The scale factor of a direction is defined as the square root of the absolute value of its diagonal metric entry; the absolute value removes the minus sign.)

$$s_5 = e^{-a_4} \sin^{1/6} z$$

(The square root of a product is the product of the square roots; $\sqrt{e^{-2a_4}} = e^{-a_4}$ because $(e^{-a_4})^2 = e^{-2a_4}$, and $\sqrt{\sin^{1/3} z} = \sin^{1/6} z$ because $(\sin^{1/6} z)^2 = \sin^{2/6} z = \sin^{1/3} z$.)

$$s_5(x_4) = e^{-AHx_4} \sin^{1/6} z$$

(We insert the linear history $a_4 = AHx_4$; the number A fixes how fast a_4 grows.)

$$\frac{ds_5}{dx_4} = -AH\, e^{-AHx_4} \sin^{1/6} z$$

(Chain rule: the derivative of e^{cx_4} is $c\, e^{cx_4}$, here with $c = -AH$; at a fixed hidden position $\sin^{1/6} z$ does not depend on x_4 and stays a constant factor.)

$$\frac{ds_5}{dx_4} = -AH\, s_5$$

(The right side of the previous line is $-AH$ times s_5 itself.)

With $A = 1$ and $H = 1$, and after dividing by the constant $\sin^{1/6} z$, the function $b(x_4) = e^{-x_4}$ obeys

$$\frac{db}{dx_4} = -b, \qquad b(0) = 1.$$

This is **Problem A** of Notebook 02a, written with $t = x_4$ and $y = b$: $y' = -y$, $y(0) = 1$, with the exact solution $y(t) = e^{-t}$ (its derivative is $-e^{-t}$ by the chain rule, and $e^0 = 1$). The same steps with $g_{11} = e^{2a_4} \sin^{1/3} z$ give the 3-space factor $a(x_4) = e^{x_4}$ with $da/dx_4 = +a$: ordinary space inflates while the extra time deflates. Their product never changes:

$$a\, b = e^{x_4} e^{-x_4} = e^{x_4 - x_4} = e^0 = 1$$

(the exponential turns sums into products, $e^u e^v = e^{u+v}$).

Status. The algebra above is PROVED. The history $a_4 = AHx_4$ with $A = 1$ is ASSUMED: it is a PRESCRIBED BACKGROUND along which the Revision record computes its Kohn-Sham states (record `Revision/kohn_sham/results/parameters.json`, keys `historyA` $= 1$ and `H` $= 1$, reproduced by Notebook 02a, In [11]); the Revision check `ks_history_is_a_prescribed_backg round` of `Revision/field_equations_a4/reports/ks-source-conditions.json` records that the Kohn-Sham states are not admissible sources of the metric along it, so the history is not derived from the field equations (the equations that determine a_4, Chapter 12): it is put in by hand. The deflation itself is not frozen or static anywhere in this book: b shrinks exponentially as x_4 grows.

Problem B: the oscillator. $x'' = -x$ with $x(0) = 1$ and $x'(0) = 0$. As a system, $x' = v$ and $v' = -x$ with $x(0) = 1$, $v(0) = 0$. The exact solution is $x = \cos t$, $v = -\sin t$: indeed $x' = -\sin t = v$ and $v' = -\cos t = -x$. Its **energy** $E = (x^2 + v^2)/2$ does not change:

$$\frac{dE}{dt} = x\, x' + v\, v'$$

(the derivative of $x^2/2$ is xx' by the chain rule, and likewise for $v^2/2$);

$$\frac{dE}{dt} = x\, v + v\, (-x) = 0$$

(we insert $x' = v$ and $v' = -x$). So $E = (1^2 + 0^2)/2 = 1/2$ at every time, and the point (x, v) runs round the circle $x^2 + v^2 = 1$. A numerical method that does not keep E fixed reveals its error at once; Notebook 02a uses this.

2.3 Taylor's theorem and the exponential series

Taylor's theorem (ASSUMED: a standard theorem of one-variable calculus, quoted without proof). If y has $n + 1$ continuous derivatives, then for every step h

$$y(t + h) = y(t) + h\, y'(t) + \frac{h^2}{2!}\, y''(t) + \cdots + \frac{h^n}{n!}\, y^{(n)}(t) + \frac{h^{n+1}}{(n+1)!}\, y^{(n+1)}(s)$$

for some number s between t and $t+h$. Here $n! = 1 \cdot 2 \cdot 3 \cdots n$ (with $0! = 1$) is the **factorial**. The last term is the **remainder**: it shows that the error of keeping only the terms up to h^n is proportional to h^{n+1} when h is small.

The exponential series. Take $y(t) = e^t$ at $t = 0$ and the step $h = z$. Every derivative of e^t is e^t itself, so every derivative at $t = 0$ is $e^0 = 1$; the term with h^k is therefore $z^k/k!$, and

$$e^z = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24} + \frac{z^5}{120} + \dots$$

where the dots stand for the terms with higher powers of z (the remainder goes to zero as more terms are kept, for every z). This series is the yardstick of Sections 2.5 and 2.6.

2.4 Three step rules: Euler, midpoint and RK4

A computer cannot follow a solution continuously. It chooses a **step size** h, the times $t_n = nh$, and computes numbers y_n that approximate $y(t_n)$, one step after the other. N steps of size $h = T/N$ reach the end time T. A **step rule** says how to compute y_{n+1} from y_n.

Euler's method. Taylor's theorem with $n = 1$ says $y(t+h) = y(t) + h\,y'(t) + \frac{h^2}{2}y''(s)$. Dropping the last term and using $y' = f(t, y)$ gives

$$y_{n+1} = y_n + h\,f(t_n, y_n).$$

It follows the slope at the start of the step for the whole step.

The midpoint method. It first uses the starting slope to estimate the value in the middle of the step, and then uses the slope there for the whole step:

$$k_1 = f(t_n, y_n), \qquad k_2 = f(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_1), \qquad y_{n+1} = y_n + h\,k_2.$$

The classical fourth-order Runge-Kutta method (RK4). It computes four slopes and averages them with the weights $1 : 2 : 2 : 1$:

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_1), \quad k_3 = f(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_2), \quad k_4 = f(t_n + h,\, y_n + h\,k_3),$$

$$y_{n+1} = y_n + \frac{h}{6}\,(k_1 + 2k_2 + 2k_3 + k_4).$$

k_1 is the slope at the start, k_2 the slope at the midpoint reached with k_1, k_3 the slope at the midpoint reached with k_2, k_4 the slope at the end reached with k_3. RK4 is the method of the Rust program of the Revision record that solves the Kohn-Sham equations (file `Revision/kohn_sham/solver/src/shoot.rs`). For a system the same formulas hold with y, f and the k's read as vectors.

One step by hand. Take Problem A, $f(t, y) = -y$, from $y_0 = 1$ with the step $h = 1/2$. Euler:

$$y_1 = 1 + \tfrac{1}{2} \cdot (-1) = \tfrac{1}{2}$$

(Euler's rule with $f(0,1) = -1$). The midpoint method:

$$k_1 = f(0,1) = -1$$

(the slope at the start is $-y_0$);

$$k_2 = f\big(\tfrac{1}{4},\, 1 + \tfrac{1}{4} \cdot (-1)\big) = -\big(1 - \tfrac{1}{4}\big) = -\tfrac{3}{4}$$

(half a step is $h/2 = 1/4$; the slope at the estimated midpoint value $3/4$ is minus that value);

$$y_1 = 1 + \tfrac{1}{2} \cdot \big(-\tfrac{3}{4}\big) = 1 - \tfrac{3}{8} = \tfrac{5}{8}$$

(the midpoint rule $y_1 = y_0 + hk_2$). RK4:

$$k_1 = -1, \qquad k_2 = -\big(1 + \tfrac{1}{4} \cdot (-1)\big) = -\tfrac{3}{4}$$

(the same two slopes as in the midpoint method);

$$k_3 = -\big(1 + \tfrac{1}{4} \cdot \big(-\tfrac{3}{4}\big)\big) = -\big(1 - \tfrac{3}{16}\big) = -\tfrac{13}{16}$$

(the midpoint value is now reached with k_2);

$$k_4 = -\big(1 + \tfrac{1}{2} \cdot \big(-\tfrac{13}{16}\big)\big) = -\big(1 - \tfrac{13}{32}\big) = -\tfrac{19}{32}$$

(the end value is reached with the whole step $h = 1/2$ and the slope k_3);

$$k_1 + 2k_2 + 2k_3 + k_4 = -\tfrac{32}{32} - \tfrac{48}{32} - \tfrac{52}{32} - \tfrac{19}{32} = -\tfrac{151}{32}$$

(every term is written over the common denominator 32 and the numerators are added);

$$y_1 = 1 + \tfrac{1/2}{6} \cdot \big(-\tfrac{151}{32}\big) = 1 - \tfrac{151}{384} = \tfrac{233}{384}$$

(the RK4 rule with $h/6 = 1/12$, and $12 \cdot 32 = 384$). The exact value is $e^{-1/2} = 0.6065306597\ldots$; the three methods give 0.5, 0.625 and $233/384 = 0.6067708333\ldots$ (PROVED here; Notebook 02a, In [3], repeats the arithmetic with exact fractions and checks the three results). RK4 is already correct to about four digits after a single large step.

2.5 The amplification factor

To understand the accuracy of a method we apply it to the simplest equation of all, the **test equation**

$$y' = \lambda y$$

with a constant number λ; its exact solution is $y(t) = e^{\lambda t}y(0)$, so one exact step multiplies y by $e^{\lambda h}$. Write $z = \lambda h$. We now show that one step of each method multiplies y_n by a number $R(z)$, its **amplification factor**, and find R.

Euler.

$$y_{n+1} = y_n + h\,\lambda y_n = (1 + z)\,y_n$$

(Euler's rule with $f(t, y) = \lambda y$, then $h\lambda = z$ and y_n taken out as a common factor). So $R(z) = 1 + z$.

Midpoint.

$$k_1 = \lambda y_n$$

(the slope at the start);

$$k_2 = \lambda(y_n + \tfrac{h}{2}\lambda y_n) = \lambda y_n(1 + \tfrac{z}{2})$$

(the slope at the estimated midpoint, with $h\lambda = z$);

$$y_{n+1} = y_n + h\lambda y_n(1 + \tfrac{z}{2}) = (1 + z + \tfrac{z^2}{2})\,y_n$$

(the midpoint rule, then $h\lambda = z$ and the product multiplied out). So $R(z) = 1 + z + z^2/2$.

RK4.

$$k_1 = \lambda y_n, \qquad k_2 = \lambda y_n(1 + \tfrac{z}{2})$$

(as for the midpoint method);

$$k_3 = \lambda(y_n + \tfrac{h}{2}k_2) = \lambda y_n(1 + \tfrac{z}{2} + \tfrac{z^2}{4})$$

(we insert k_2: $\frac{h}{2}k_2 = \frac{z}{2}y_n(1 + \frac{z}{2})$);

$$k_4 = \lambda(y_n + hk_3) = \lambda y_n(1 + z + \tfrac{z^2}{2} + \tfrac{z^3}{4})$$

(we insert k_3: $hk_3 = zy_n(1 + \frac{z}{2} + \frac{z^2}{4})$);

$$k_1 + 2k_2 + 2k_3 + k_4 = \lambda y_n(6 + 3z + z^2 + \tfrac{z^3}{4})$$

(we add the four brackets, the middle two counted twice: the constants $1+2+2+1=6$, the terms with z: $1+1+1=3$, with z^2: $\frac{1}{2}+\frac{1}{2}=1$, with z^3: $\frac{1}{4}$);

$$y_{n+1} = y_n + \frac{h}{6}\lambda y_n(6 + 3z + z^2 + \tfrac{z^3}{4}) = (1 + z + \tfrac{z^2}{2} + \tfrac{z^3}{6} + \tfrac{z^4}{24})\, y_n$$

(the RK4 rule; $h\lambda = z$, and each term of the bracket is multiplied by $z/6$). So

$$R_{\text{Euler}}(z) = 1 + z, \qquad R_{\text{midpoint}}(z) = 1 + z + \frac{z^2}{2}, \qquad R_{\text{RK4}}(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

Each R is a **polynomial** in z (a sum of powers of z with constant coefficients). Compare with the exponential series of Section 2.3: Euler keeps its first 2 terms, the midpoint method its first 3, RK4 its first 5 (PROVED here; Notebook 02a, In [4], derives the same polynomials with sympy and compares them with the series). The first term that a method misses decides its accuracy, as the next section shows.

2.6 Error, order, the log-log plot, rounding and Richardson's rule

Error and order. The **error** of a computation is the computed value minus the exact value. A method has **order** p when its error at a fixed end time is close to Ch^p for small h, with a number C that does not depend on h. Halving h then divides the error by 2^p:

$$\frac{Ch^p}{C(h/2)^p} = \frac{h^p}{h^p/2^p} = 2^p$$

(the constant C cancels, and $(h/2)^p = h^p/2^p$ because a power of a quotient is the quotient of the powers).

Why Euler has order 1. Taylor's theorem says that one Euler step makes an error of about $\frac{1}{2}h^2|y''|$. To reach the end time T one needs $N = T/h$ steps, and N errors of size about h^2 add up to about $Nh^2 = Th$, which is proportional to h^1.

The exact leading error for Problem A. For Problem A we can do better and compute the error of each method to leading order. Here $\lambda = -1$, so one step multiplies y by $R(-h)$, while the exact factor is e^{-h}. A method of order p keeps the terms of the series of e^{-h} up to the power h^p (Euler $p=1$, midpoint $p=2$, RK4 $p=4$):

$$R(-h) = \sum_{k=0}^{p} \frac{(-h)^k}{k!}, \qquad e^{-h} = \sum_{k=0}^{\infty} \frac{(-h)^k}{k!}$$

(Section 2.5 for R and Section 2.3 for the series of the exponential, both at $z = -h$).

$$R(-h) = e^{-h} - \frac{(-h)^{p+1}}{(p+1)!} - \dots$$

(R is the series minus all the terms it misses; the dots stand for the missed terms with the powers h^{p+2} and higher).

$$R(-h) = e^{-h}(1-q), \qquad q = e^{h}\Big(\frac{(-h)^{p+1}}{(p+1)!} + \dots\Big) = \frac{(-h)^{p+1}}{(p+1)!} + \dots$$

(we take out the factor e^{-h}, which multiplies everything in the bracket by e^{h}; since $e^{h} = 1 + h + \dots$, that changes only the terms with higher powers of h).

$$y_N = R(-h)^N = e^{-Nh}(1-q)^N = e^{-1}(1-q)^N$$

(N steps multiply the start value $y_0 = 1$ by $R(-h)$ N times; a power of a product is the product of the powers; and $Nh = 1$ when we compute up to $t = 1$).

$$(1-q)^N \approx 1 - Nq$$

(the first two terms of the binomial theorem $(1-q)^N = 1 - Nq + \frac{N(N-1)}{2}q^2 - \dots$; the next term is of the size $(Nq)^2$, which is much smaller than Nq when Nq is small).

$$Nq \approx \frac{1}{h} \cdot \frac{(-1)^{p+1}h^{p+1}}{(p+1)!} = \frac{(-1)^{p+1}h^{p}}{(p+1)!}$$

($N = 1/h$, $(-h)^{p+1} = (-1)^{p+1}h^{p+1}$, and one factor h cancels).

$$y_N - e^{-1} \approx -e^{-1}Nq = e^{-1}\,\frac{(-1)^{p}\,h^{p}}{(p+1)!}$$

(we subtract e^{-1} from $y_N \approx e^{-1}(1-Nq)$, and $-(-1)^{p+1} = (-1)^{p}$).

So at $t = 1$ Euler ends too LOW by $e^{-1}h/2$, the midpoint method too HIGH by $e^{-1}h^2/6$, and RK4 too high by $e^{-1}h^4/120$ (PROVED here, to leading order in h). The Rust solver of the Revision record makes $G = 900$ RK4 steps on an interval of length $L = 3$, so its step is $h = L/G = 1/300$ (COMPUTED from the record `Revision/kohn_sham/results/parameters.json`, keys `rk4Steps` and `L_tipCutoff`; Notebook 02a, In [11]). At this step the formula predicts -6.1313×10^{-4} (Euler), $+6.8126 \times 10^{-7}$ (midpoint) and $+3.7848 \times 10^{-13}$ (RK4); Notebook 02a measures -6.1399×10^{-4}, $+6.8296 \times 10^{-7}$ and $+3.7920 \times 10^{-13}$, the ratios 1.0014, 1.0025 and 1.0019 (COMPUTED, In [11]); the small remaining differences are the next terms of the expansion, which the formula leaves out.

The log-log plot. The **logarithm** $\log_{10} x$ is the power to which 10 must be raised to give x; for example $\log_{10} 1000 = 3$ and $\log_{10} 0.001 = -3$. Write $\Delta = Ch^{p}$ for the size of the error and take the logarithm:

$$\log_{10}\Delta = \log_{10} C + p\log_{10} h$$

(the logarithm of a product is the sum of the logarithms, and $\log_{10}(h^{p}) = p\log_{10} h$). As a function of $\log_{10} h$ this is a straight line with **slope** p. A **log-log plot** marks both axes in powers of ten, so

the errors of a method of order p lie on a straight line of slope p, and the order can be read off as a slope. A straight line fitted through measured points by **least squares** (the line that makes the sum of the squared vertical distances smallest) gives the measured order.

Rounding errors. A computer stores a number with about 16 significant decimal digits (53 binary digits). The spacing of the stored numbers near 1 is the **machine epsilon** $\epsilon = 2^{-52} \approx 2.2 \times 10^{-16}$, and every arithmetic operation rounds its result to a relative error of at most about $\epsilon/2$. With N steps about N such rounding errors add up. If all of them had their largest size and the same sign, their total would be about $N\epsilon = \epsilon/h$ times the size of the solution; this is a pessimistic **bound**, the scale of the worst case. In practice the rounding errors have both signs and partly cancel, so their total stays far below the bound; but it still grows with the number of steps, while the truncation error Ch^p falls. A smaller step is therefore not always better: below some step size the rounding errors take over, and the error grows again, slowly. For RK4 on Problem A this **rounding floor** is near 10^{-16}, and with more steps the error grows again but stays 524 to 3709 times below the bound $N\epsilon e^{-1}$ (COMPUTED, Notebook 02a, In [10]).

Richardson's rule. In a real problem the exact answer Y is unknown. Suppose a method of order p gives the value Y_h with the step h and the value $Y_{h/2}$ with half the step, and that both errors are already close to Ch^p:

$$Y_h = Y + Ch^p, \qquad Y_{h/2} = Y + \frac{Ch^p}{2^p}$$

(the order-p error law at the steps h and $h/2$).

$$Y_h - Y_{h/2} = Ch^p\Big(1 - \frac{1}{2^p}\Big) = \frac{Ch^p}{2^p}\,(2^p - 1)$$

(we subtract the second equation from the first; Y cancels; then $1 - 2^{-p} = (2^p - 1)/2^p$).

$$Y_{h/2} - Y = \frac{Ch^p}{2^p} = \frac{Y_h - Y_{h/2}}{2^p - 1}$$

(the error of the finer value, from the second equation, and then divided out of the previous line).

So the difference of two runs, divided by $2^p - 1$, estimates the error of the finer run without knowing Y; for RK4 it is $(Y_h - Y_{h/2})/15$. Subtracting the estimate gives the **extrapolated** value

$$Y_{h/2} - \frac{Y_h - Y_{h/2}}{15} = \frac{16\,Y_{h/2} - Y_h}{15}$$

(both terms written over the denominator 15), which is more accurate than either run (Notebook 02a, In [14]). Richardson's rule is only as good as its assumption: the errors must already follow Ch^p.

2.7 The oscillator and the product of the two scale factors

A complex number turns the oscillator into the test equation. A **complex number** is $w = x + iv$ with real numbers x, v and the number i with $i^2 = -1$ (complex numbers are the subject

of Chapter 1); its **complex conjugate** is $\bar{w} = x - iv$, and $|w|^2 = w\bar{w} = x^2 + v^2$. For the oscillator of Problem B put $w = x + iv$. Then

$$w' = x' + iv' = v - ix$$

(differentiate each part, and insert $x' = v$, $v' = -x$);

$$w' = -i\,(x + iv) = -i\,w$$

(multiply out: $-ix - i^2 v = -ix + v$, which is the previous line). So w obeys the test equation with $\lambda = -i$, and one step of a method multiplies w by $R(-ih)$. The energy is $E = (x^2 + v^2)/2 = |w|^2/2$, so one step multiplies E by

$$|R(-ih)|^2 = R(-ih)\,R(ih)$$

(the conjugate of $R(-ih)$ is $R(ih)$, because the coefficients of the polynomial R are real numbers and the conjugate of $-i$ is i).

Euler.

$$R(-ih)R(ih) = (1 - ih)(1 + ih) = 1 - i^2h^2 = 1 + h^2$$

(the product $(a - b)(a + b) = a^2 - b^2$, and $i^2 = -1$). Every Euler step multiplies the energy by $1 + h^2 > 1$: the computed point spirals outwards.

Midpoint. $R(ih) = 1 + ih + (ih)^2/2 = (1 - \frac{h^2}{2}) + ih$, a complex number with the real part $1 - h^2/2$ and the imaginary part h, so

$$|R(ih)|^2 = \Big(1 - \frac{h^2}{2}\Big)^2 + h^2 = 1 - h^2 + \frac{h^4}{4} + h^2 = 1 + \frac{h^4}{4}$$

($|w|^2 = x^2 + v^2$, then the square multiplied out).

RK4. $R(ih) = 1 + ih - \frac{h^2}{2} - \frac{ih^3}{6} + \frac{h^4}{24}$ (powers of i: $i^2 = -1$, $i^3 = -i$, $i^4 = 1$), with the real part $c = 1 - \frac{h^2}{2} + \frac{h^4}{24}$ and the imaginary part $s = h - \frac{h^3}{6}$.

$$c^2 = 1 - h^2 + \frac{h^4}{4} + \frac{h^4}{12} - \frac{h^6}{24} + \frac{h^8}{576} = 1 - h^2 + \frac{h^4}{3} - \frac{h^6}{24} + \frac{h^8}{576}$$

(the square of a sum of three terms, $(a + b + d)^2 = a^2 + b^2 + d^2 + 2ab + 2ad + 2bd$ with $a = 1$, $b = -h^2/2$, $d = h^4/24$; then $\frac{1}{4} + \frac{1}{12} = \frac{1}{3}$);

$$s^2 = h^2 - \frac{h^4}{3} + \frac{h^6}{36}$$

($(a + b)^2 = a^2 + 2ab + b^2$ with $a = h$, $b = -h^3/6$);

$$c^2 + s^2 = 1 + \Big(-\frac{1}{24} + \frac{1}{36}\Big)h^6 + \frac{h^8}{576} = 1 - \frac{h^6}{72} + \frac{h^8}{576}$$

(we add; the terms with h^2 and h^4 cancel, and $-\frac{3}{72} + \frac{2}{72} = -\frac{1}{72}$).

So one step multiplies the energy by $1 + h^2$ (Euler), $1 + h^4/4$ (midpoint) and $1 - h^6/72 + h^8/576$ (RK4) (PROVED here; Notebook 02a, In [4], derives the same three polynomials with sympy, and In [12] shows that the computed energies follow them to 13 digits). With $h = 0.2$ the factors are 1.04, 1.0004 and 0.99999911.

The product of the scale factors. The 3-space factor obeys $a' = +a$ ($\lambda = +1$) and the extra-time factor $b' = -b$ ($\lambda = -1$), so one step multiplies a by $R(h)$, b by $R(-h)$ and their product, exactly 1 for the true solution, by $R(h)R(-h)$. For Euler

$$R(h)R(-h) = (1 + h)(1 - h) = 1 - h^2$$

(again $(a + b)(a - b) = a^2 - b^2$): the product shrinks by the factor $1 - h^2$ at every step, and the compensation of inflation by deflation is lost. For RK4 split $R(h) = c + s$ into its even part $c = 1 + \frac{h^2}{2} + \frac{h^4}{24}$ and its odd part $s = h + \frac{h^3}{6}$ (the letters c and s now stand for these new polynomials); then $R(-h) = c - s$, because the odd powers change sign, and

$$R(h)R(-h) = (c + s)(c - s) = c^2 - s^2$$

(again $(a + b)(a - b) = a^2 - b^2$).

$$c^2 = 1 + h^2 + \frac{h^4}{4} + \frac{h^4}{12} + \frac{h^6}{24} + \frac{h^8}{576} = 1 + h^2 + \frac{h^4}{3} + \frac{h^6}{24} + \frac{h^8}{576}$$

(the square of a sum of three terms, $(a + b + d)^2 = a^2 + b^2 + d^2 + 2ab + 2ad + 2bd$, now with $a = 1$, $b = +h^2/2$, $d = h^4/24$: $a^2 = 1$, $2ab = h^2$, $b^2 = h^4/4$, $2ad = h^4/12$, $2bd = h^6/24$, $d^2 = h^8/576$; then $\frac{1}{4} + \frac{1}{12} = \frac{1}{3}$);

$$s^2 = h^2 + \frac{h^4}{3} + \frac{h^6}{36}$$

($(a + b)^2 = a^2 + 2ab + b^2$ with $a = h$, $b = +h^3/6$: $a^2 = h^2$, $2ab = h^4/3$, $b^2 = h^6/36$);

$$R(h)R(-h) = c^2 - s^2 = 1 + \Big(\frac{1}{24} - \frac{1}{36}\Big)h^6 + \frac{h^8}{576} = 1 + \frac{h^6}{72} + \frac{h^8}{576}$$

(we subtract; the terms with h^2 and h^4 cancel, and $\frac{1}{24} - \frac{1}{36} = \frac{3}{72} - \frac{2}{72} = \frac{1}{72}$). With $h = 0.25$ and 8 steps, Euler multiplies the product by $0.9375^8 = 0.597$, RK4 by a number within 3×10^{-5} of 1 (PROVED here; COMPUTED in Notebook 02a, In [6]).

2.8 Example: Euler, midpoint and RK4 on the deflating scale factor and the oscillator

Notebook 02a puts Sections 2.2 to 2.7 to work. It makes one step of each method by hand with exact fractions; derives the amplification factors and the energy factors with sympy; solves the inflating and the deflating scale factor with Euler and RK4 and shows what Euler does to their product; measures the errors of the three methods for step sizes from 1/2 down to 1/4096 and reads the orders 1, 2 and 4 from a log-log plot; shows the rounding floor of RK4 with up to 262144 steps; reads the step of the Revision Kohn-Sham solver from its record and checks the predicted errors of Section 2.6 at that step; draws the phase portrait of the oscillator, measures by what angle each method runs ahead of the exact solution and draws the energy drift; and tests Richardson's rule. It ends with the line ALL 29 CHECKS PASSED (notebook 02a).

2.9 How to run Notebook 02a

Step 1. What this notebook does and what it needs.

Notebook 02a (Euler, midpoint and RK4: the order of a method on a log-log plot) is the file `Revision/textbook/notebooks/02a_rk4_convergence.ipynb` of the repository Dirac_claude. It solves two differential equations whose exact solutions are known, the decay equation (the deflation of an extra-time scale factor along a linear history) and the oscillator equation, with the Euler method, the midpoint method and the classical fourth-order Runge-Kutta method (RK4); it derives the exact amplification factor of each method with sympy, measures how the error at a fixed end time shrinks when the step is halved, finds the orders 1, 2 and 4 as slopes on log-log plots, shows where rounding errors stop the improvement, reads the step of the Revision Kohn-Sham solver (900 RK4 steps on an interval of length 3) from its record and predicts the error of each method at that step from the first missed term of the exponential series, predicts the energy drift of the oscillator exactly, and tests the Richardson estimate of the error. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
```

```
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 02a_rk4_convergence.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 02a_rk4_convergence.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 02a_rk4_convergence.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/02a.captions.json`
- `Revision/textbook/figures/02a_1_scale_factors.png`
- `Revision/textbook/figures/02a_2_scale_factor_product.png`
- `Revision/textbook/figures/02a_3_convergence_loglog.png`
- `Revision/textbook/figures/02a_4_error_ratios.png`
- `Revision/textbook/figures/02a_5_rounding_floor.png`
- `Revision/textbook/figures/02a_6_phase_portrait.png`

- `Revision/textbook/figures/02a_7_energy_drift.png`
- `Revision/textbook/figures/02a_8_richardson.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of notebook 02a exist
ALL 29 CHECKS PASSED (notebook 02a)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/02a_rk4_convergence.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 02a_rk4_convergence.ipynb
python -m nbconvert --execute --inplace 02a_rk4_convergence.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

2.10 Notebook 02a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/02a_rk4_convergence.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A differential equation says how fast a quantity changes. A computer solves it by
taking many small *steps*. This notebook compares three ways of making one step,
the **Euler method**, the **midpoint method** and the **classical fourth-order
Runge-Kutta method (RK4)**, on two equations whose exact solutions we know, so
that every error can be measured exactly. It

- makes one step of each method by hand, with exact fractions;
- derives with sympy the exact *amplification factor* of each method, the number
  by which one step multiplies the solution of $y' = \lambda y$;
- solves the growth of the 3-space scale factor and the deflation of an
  extra-time scale factor of the author's metric along a linear history, and
  shows that Euler's method spoils their exact product $1$;
- measures the error at a fixed end time for step sizes from $1/2$ down to
  $1/262144$, finds the *orders* $1$, $2$ and $4$ of the three methods as the
  slopes of straight lines on log-log plots, and shows where rounding errors stop
  the improvement;
- reads from the Revision record Revision/kohn_sham/results/parameters.json the
  step of the Rust program that solves the book's Kohn-Sham equations with RK4,
  and predicts exactly how large the error of each method is at that step;
- solves the oscillator $d^2x/dt^2 = -x$, draws its phase portrait, and predicts the
  drift of its energy exactly from the amplification factors;
- tests Richardson's rule, which estimates the error of a computation without
  knowing the exact answer.

It draws 8 figures and prints a PASS line for every check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 02a (Euler, midpoint and RK4: the order of a method on a log-log plot) is the
file ‘Revision/textbook/notebooks/02a_rk4_convergence.ipynb‘ of the repository
Dirac_claude. It solves two differential equations whose exact solutions are known, the
decay equation (the deflation of an extra-time scale factor along a linear history) and
the oscillator equation, with the Euler method, the midpoint method and the classical
fourth-order Runge-Kutta method (RK4); it derives the exact amplification factor of each
method with sympy, measures how the error at a fixed end time shrinks when the step is
halved, finds the orders 1, 2 and 4 as slopes on log-log plots, shows where rounding
errors stop the improvement, reads the step of the Revision Kohn-Sham solver (900 RK4
steps on an interval of length 3) from its record and predicts the error of each method
at that step from the first missed term of the exponential series, predicts the energy
drift of the oscillator exactly, and tests the Richardson estimate of the error. It needs
a computer with Windows 11, macOS or Linux, an internet connection for the installation,
the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv

Then close the terminal and open a NEW one (a terminal opened before the installation
does not know the new programs), and check the version of Python; it must print 3.12 or a
```

higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
```

```
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 02a_rk4_convergence.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 10 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
```

Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 02a_rk4_convergence.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 02a_rk4_convergence.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/02a.captions.json‘
- ‘Revision/textbook/figures/02a_1_scale_factors.png‘
- ‘Revision/textbook/figures/02a_2_scale_factor_product.png‘
- ‘Revision/textbook/figures/02a_3_convergence_loglog.png‘
- ‘Revision/textbook/figures/02a_4_error_ratios.png‘
- ‘Revision/textbook/figures/02a_5_rounding_floor.png‘
- ‘Revision/textbook/figures/02a_6_phase_portrait.png‘
- ‘Revision/textbook/figures/02a_7_energy_drift.png‘
- ‘Revision/textbook/figures/02a_8_richardson.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of notebook 02a exist
ALL 29 CHECKS PASSED (notebook 02a)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started

without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

    python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/02a_rk4_convergence.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

    python -m jupyterlab 02a_rk4_convergence.ipynb
    python -m nbconvert --execute --inplace 02a_rk4_convergence.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 02a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 02a (Euler, midpoint and RK4: the order of a method on a log-log plot) is the
# file Revision/textbook/notebooks/02a_rk4_convergence.ipynb of the repository
# Dirac_claude. It solves two differential equations whose exact solutions are known, the
```

```
# decay equation (the deflation of an extra-time scale factor along a linear history) and
# the oscillator equation, with the Euler method, the midpoint method and the classical
# fourth-order Runge-Kutta method (RK4); it derives the exact amplification factor of
# each method with sympy, measures how the error at a fixed end time shrinks when the
# step is halved, finds the orders 1, 2 and 4 as slopes on log-log plots, shows where
# rounding errors stop the improvement, reads the step of the Revision Kohn-Sham solver
# (900 RK4 steps on an interval of length 3) from its record and predicts the error of
# each method at that step from the first missed term of the exponential series, predicts
# the energy drift of the oscillator exactly, and tests the Richardson estimate of the
# error. It needs a computer with Windows 11, macOS or Linux, an internet connection for
# the installation, the program Git, and Python 3.12 or newer (the notebooks were built
# with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
```

```
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 02a_rk4_convergence.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 02a_rk4_convergence.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 02a_rk4_convergence.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
```

```
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/02a.captions.json
# - Revision/textbook/figures/02a_1_scale_factors.png
# - Revision/textbook/figures/02a_2_scale_factor_product.png
# - Revision/textbook/figures/02a_3_convergence_loglog.png
# - Revision/textbook/figures/02a_4_error_ratios.png
# - Revision/textbook/figures/02a_5_rounding_floor.png
# - Revision/textbook/figures/02a_6_phase_portrait.png
# - Revision/textbook/figures/02a_7_energy_drift.png
# - Revision/textbook/figures/02a_8_richardson.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 8 figure files of notebook 02a exist
#     ALL 29 CHECKS PASSED (notebook 02a)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/02a_rk4_convergence.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
```

```
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 02a_rk4_convergence.ipynb
#     python -m nbconvert --execute --inplace 02a_rk4_convergence.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "02a"  # this notebook: chapter 02, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 02a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Derivative**: $y'(t) = dy/dt$ is the rate of change of $y$ at time $t$, the
  slope of the graph of $y$.
- **Ordinary differential equation (ODE)**: an equation $y' = f(t, y)$ that gives
  the rate of change of an unknown function $y(t)$ from $t$ and $y$ itself. Here
  $f$ is a known rule, the *right-hand side*.
- **Initial-value problem**: an ODE together with the starting value $y(0)$.
  Exactly one solution belongs to it (for the smooth $f$ of this notebook).
- **System of ODEs**: several unknown functions changing together, for example
  a position $x$ and a velocity $v$; $y$ is then a list (a *vector*) of numbers.
- **Step size** $h$: the time between two computed points; $N$ steps of size
  $h = T/N$ go from $t = 0$ to $t = T$.
- **Euler, midpoint, RK4**: three rules that compute the value after one step
  from the value before it (the formulas are in section 4).
- **Error**: the computed value minus the exact value (we print its size, the
  *absolute value*).
- **Order** $p$: a method has order $p$ when its error at a fixed end time is
```

```
  close to $C h^p$ for small $h$, with a number $C$ that does not depend on $h$.
  Halving $h$ then divides the error by $2^p$.
- **Logarithm, log-log plot**: $\log_{10} x$ is the power to which 10 must be
  raised to give $x$ ($\log_{10} 1000 = 3$). A log-log plot has both axes marked
  in powers of ten, so that $e = C h^p$ becomes the straight line
  $\log e = \log C + p \log h$ whose *slope* is $p$.
- **Rounding error, machine epsilon**: a computer stores about 16 significant
  digits of a number; every operation rounds its result. The spacing of the
  stored numbers near 1 is the machine epsilon $2^{-52} \approx 2.2 \times
  10^{-16}$.
- **Amplification factor** $R(z)$: for the test equation $y' = \lambda y$, one
  step of a method multiplies $y$ by a number $R(z)$ that depends only on
  $z = \lambda h$.
- **Phase portrait**: the curve traced by the point $(x, v)$ (position,
  velocity) as time goes on.
- **Richardson's rule**: an estimate of the error made from two computations with
  the step sizes $h$ and $h/2$.
```

Text cell:

```
## 4. The physical and mathematical situation

**The equations.** We solve two initial-value problems.

*Problem A (decay and growth).* $y' = -y$ with $y(0) = 1$. The exact solution is
$y(t) = e^{-t}$, because the derivative of $e^{-t}$ is $-e^{-t}$ and $e^{0} = 1$.
This equation describes the extra times of the author's metric. Lengths along the
three extra times $x_5, x_6, x_7$ carry the scale factor $e^{-a_4}\sin^{1/6}z$
and lengths along 3-space $x_1, x_2, x_3$ the factor $e^{a_4}\sin^{1/6}z$, where
$a_4(x_4)$ depends on the time $x_4$ and $z = 6 H x_0$ on the hidden coordinate.
Along the linear history $a_4 = A H x_4$, which the book uses as a prescribed
background (here $A = 1$ and $H = 1$), at a fixed hidden position the factor
$\sin^{1/6}z$ does not change with $x_4$, so we leave it out and follow
$b = e^{-x_4}$ and $a = e^{x_4}$. The extra-time factor obeys $db/dx_4 = -b$
(problem A with $t = x_4$: the extra times DEFLATE) and the 3-space factor obeys
$da/dx_4 = +a$ (3-space inflates). Their product $a b = e^{x_4} e^{-x_4} = 1$
never changes.

*Problem B (oscillation).* $d^2x/dt^2 = -x$ with $x(0) = 1$, $x'(0) = 0$. With the
velocity $v = x'$ it becomes a system of two first-order equations, $x' = v$ and
$v' = -x$. The exact solution is $x = \cos t$, $v = -\sin t$, and the energy
$E = (x^2 + v^2)/2$ stays exactly $1/2$, because $\cos^2 t + \sin^2 t = 1$.

**The three methods.** Write $y_n$ for the computed value at $t_n = n h$.

- Euler: $y_{n+1} = y_n + h f(t_n, y_n)$. It follows the slope at the start of
  the step for the whole step.
- Midpoint: $k_1 = f(t_n, y_n)$, $k_2 = f(t_n + h/2, y_n + (h/2) k_1)$,
  $y_{n+1} = y_n + h k_2$. It uses the slope at an estimated midpoint.
- RK4: $k_1 = f(t_n, y_n)$, $k_2 = f(t_n + h/2, y_n + (h/2) k_1)$,
  $k_3 = f(t_n + h/2, y_n + (h/2) k_2)$, $k_4 = f(t_n + h, y_n + h k_3)$,
  $y_{n+1} = y_n + (h/6)(k_1 + 2 k_2 + 2 k_3 + k_4)$. It averages four slopes
  with the weights $1 : 2 : 2 : 1$. RK4 is the method with which the Rust
```

```
  program of the Revision record solves the Kohn-Sham equations of this book
  (file Revision/kohn_sham/solver/src/shoot.rs); section 12 reads its number of
  steps from the record.

**Why Euler has order 1.** Taylor's theorem gives $y(t + h) = y(t) + h y'(t) +
\frac{1}{2} h^2 y''(s)$ for some $s$ between $t$ and $t + h$. Euler keeps the
first two terms, so one step makes an error of size about $\frac{1}{2} h^2
|y''|$. To reach the end time $T$ we need $N = T/h$ steps; $N$ errors of size
$h^2$ add up to about $N h^2 = T h$, proportional to $h^1$. The midpoint method
has order 2 and RK4 order 4; the amplification factors below show why.

**The amplification factor.** For $y' = \lambda y$ the exact solution after one
step is $e^{\lambda h} y_n = e^{z} y_n$, and $e^{z} = 1 + z + z^2/2 + z^3/6 +
z^4/24 + z^5/120 + \dots$ (the Taylor series of the exponential). We shall find
that Euler keeps the first 2 terms of this series, the midpoint method the first
3 and RK4 the first 5. The first term a method misses decides its order.
```

Text cell:

```
## 5. The colours, the three step rules and the solver

The next cell imports the packages, fixes the colours of the plots, and defines
the three step rules exactly as written in section 4. The rules use only
‘+‘, ‘-‘, ‘*‘ and ‘/‘, so the same function works with ordinary numbers, with
exact fractions, with sympy symbols and with numpy vectors. The function ‘solve‘
makes $n$ equal steps from $t = 0$ to $t = T$.
```

In [2]:

```
import math  # exp, log10 and other functions of one number
from fractions import Fraction  # exact fractions such as 233/384

import numpy as np  # arrays (vectors) of numbers
import sympy as sp  # exact algebra with symbols

# One colour per method (a set that colour-blind readers can tell apart); every
# method also has its own line style and marker, so the plots read in grey too.
COLORS = {"euler": "#eb6834", "midpoint": "#1baf7a", "rk4": "#2a78d6",
          "exact": "#000000", "guide": "#8a8986"}
MARKERS = {"euler": "s", "midpoint": "^", "rk4": "o"}
LABELS = {"euler": "Euler", "midpoint": "midpoint", "rk4": "RK4"}


def euler_step(f, t, y, h):
    """One Euler step: follow the slope at the start for the whole step."""
    return y + h * f(t, y)


def midpoint_step(f, t, y, h):
    """One midpoint step: use the slope at the estimated middle of the step."""
    k1 = f(t, y)  # the slope at the start
    k2 = f(t + h / 2, y + (h / 2) * k1)  # the slope at the estimated midpoint
```

```
    return y + h * k2


def rk4_step(f, t, y, h):
    """One step of the classical fourth-order Runge-Kutta method (RK4)."""
    k1 = f(t, y)  # the slope at the start
    k2 = f(t + h / 2, y + (h / 2) * k1)  # at the midpoint, reached with k1
    k3 = f(t + h / 2, y + (h / 2) * k2)  # at the midpoint again, reached with k2
    k4 = f(t + h, y + h * k3)  # at the end, reached with k3
    return y + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)  # weights 1 : 2 : 2 : 1


METHODS = {"euler": euler_step, "midpoint": midpoint_step, "rk4": rk4_step}


def solve(step, f, y0, t_end, n):
    """n equal steps of the rule step from t = 0 to t = t_end.  Returns the list
    of the n + 1 times and the list of the n + 1 computed values."""
    h = t_end / n  # the step size
    times, values = [0 * h], [y0]
    y = y0
    for i in range(n):
        y = step(f, i * h, y, h)  # t_i = i h (no rounding drift of the time)
        times.append((i + 1) * h)
        values.append(y)
    return times, values


def decay(t, y):
    """Problem A: y' = -y (an extra-time scale factor along the linear history)."""
    return -y


def growth(t, y):
    """y' = +y (the 3-space scale factor along the same history)."""
    return y


say("Three step rules and the solver are defined.")
```

Out [2]:

```
Three step rules and the solver are defined.
```

Text cell:

```
## 6. One step by hand, with exact fractions

We make ONE step of size $h = 1/2$ from $y(0) = 1$ for problem A ($y' = -y$).
With exact fractions the results are: Euler $1 - 1/2 = 1/2$; midpoint
$k_1 = -1$, $k_2 = -(1 - 1/4) = -3/4$, so $1 - 3/8 = 5/8$; RK4
$1 - 1/2 + 1/8 - 1/48 + 1/384 = 233/384$. The exact value is
```

```
$e^{-1/2} = 0.6065306597\dots$. The cell lets Python's `Fraction` do the same
arithmetic exactly and checks the three fractions.
```

In [3]:

```
h = Fraction(1, 2)  # the step size 1/2 as an exact fraction
one_step = {name: step(decay, 0, Fraction(1), h) for name, step in METHODS.items()}
for name, value in one_step.items():
    say(f"{LABELS[name]:9} after one step: {str(value):8} = {float(value):.10f}")
say(f"exact     e^(-1/2)                 = {math.exp(-0.5):.10f}")
check(one_step["euler"] == Fraction(1, 2), "one Euler step of y' = -y gives 1/2")
check(one_step["midpoint"] == Fraction(5, 8), "one midpoint step gives 5/8")
check(one_step["rk4"] == Fraction(233, 384), "one RK4 step gives 233/384")
```

Out [3]:

```
Euler     after one step: 1/2      = 0.5000000000
midpoint  after one step: 5/8      = 0.6250000000
RK4       after one step: 233/384  = 0.6067708333
exact     e^(-1/2)                 = 0.6065306597
PASS one Euler step of y' = -y gives 1/2
PASS one midpoint step gives 5/8
PASS one RK4 step gives 233/384
```

Text cell:

```
## 7. The exact amplification factors

The next cell applies one step of each method to the test equation $y' = z y$
with the step $h = 1$, starting from $y = 1$, where $z$ is a sympy symbol. The
result is the amplification factor $R(z)$ as a polynomial in $z$. The checks
compare it with the first terms of the Taylor series of $e^{z}$, which sympy
computes with `sp.series`.

The oscillator (problem B) is the same test equation in disguise. Put
$w = x + i v$ with $i^2 = -1$. Then $w' = x' + i v' = v - i x = -i(x + i v) =
-i w$: the test equation with $\lambda = -i$, so one step multiplies $w$ by
$R(-ih)$. The energy is $E = (x^2 + v^2)/2 = |w|^2/2$, so one step multiplies
it by $|R(-ih)|^2 = R(-ih) R(ih)$ (the second factor is the complex conjugate of
the first, because the coefficients of $R$ are real numbers). The cell computes
these three polynomials in $h$ as well; we use them in section 13.
```

In [4]:

```
z = sp.symbols("z")  # z = lambda h
hs = sp.symbols("h", positive=True)  # a positive step size, as a symbol


def linear(t, y):
```

```
    """The test equation y' = z y."""
    return z * y


one = sp.Integer(1)  # sympy's exact 1 (so that 1/2 stays the exact fraction 1/2)
R = {name: sp.expand(step(linear, 0, one, one)) for name, step in METHODS.items()}
taylor = sp.series(sp.exp(z), z, 0, 6).removeO()  # 1 + z + ... + z^5/120
terms = sp.Poly(taylor, z).all_coeffs()[::-1]  # the coefficients 1, 1, 1/2, ...
for name, keep in (("euler", 2), ("midpoint", 3), ("rk4", 5)):
    partial = sum(terms[k] * z ** k for k in range(keep))  # the first keep terms
    say(f"R_{name}(z) = {sp.expand(R[name])}")
    check(sp.expand(R[name] - partial) == 0,
          f"R_{name}(z) equals the first {keep} terms of the series of e^z")

# The energy factor of one oscillator step, R(i h) R(-i h), as a polynomial in h.
ENERGY_FACTOR = {name: sp.expand(R[name].subs(z, sp.I * hs) *
                                 R[name].subs(z, -sp.I * hs))
                 for name in METHODS}
for name, factor in ENERGY_FACTOR.items():
    say(f"energy factor per step, {LABELS[name]:8}: {factor}")
check(ENERGY_FACTOR["euler"] == 1 + hs ** 2
      and ENERGY_FACTOR["midpoint"] == 1 + hs ** 4 / 4
      and ENERGY_FACTOR["rk4"] == 1 - hs ** 6 / 72 + hs ** 8 / 576,
      "energy factors 1 + h^2, 1 + h^4/4, 1 - h^6/72 + h^8/576")
```

Out [4]:

```
R_euler(z) = z + 1
PASS R_euler(z) equals the first 2 terms of the series of e^z
R_midpoint(z) = z**2/2 + z + 1
PASS R_midpoint(z) equals the first 3 terms of the series of e^z
R_rk4(z) = z**4/24 + z**3/6 + z**2/2 + z + 1
PASS R_rk4(z) equals the first 5 terms of the series of e^z
energy factor per step, Euler   : h**2 + 1
energy factor per step, midpoint: h**4/4 + 1
energy factor per step, RK4     : h**8/576 - h**6/72 + 1
PASS energy factors 1 + h^2, 1 + h^4/4, 1 - h^6/72 + h^8/576
```

Text cell:

```
## 8. The inflating and the deflating scale factor

The next cell solves $da/dt = a$ (the 3-space factor $a = e^{t}$) and
$db/dt = -b$ (the extra-time factor $b = e^{-t}$) from $t = 0$ to $t = 2$ with
the coarse step $h = 0.25$ (8 steps), by Euler and by RK4, and draws them with
the exact curves. Euler multiplies by $1 + h = 1.25$ and by $1 - h = 0.75$ per
step, so after 8 steps it gives $1.25^8$ and $0.75^8$; the checks confirm this.
```

In [5]:

```
H_COARSE, STEPS = 0.25, 8  # step size and number of steps: t from 0 to 2
curves = {}
```

```
for name in ("euler", "rk4"):
    t_list, a_list = solve(METHODS[name], growth, 1.0, 2.0, STEPS)
    _, b_list = solve(METHODS[name], decay, 1.0, 2.0, STEPS)
    curves[name] = (np.array(t_list), np.array(a_list), np.array(b_list))
t_fine = np.linspace(0.0, 2.0, 201)  # many points for the exact curves

fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.plot(t_fine, np.exp(t_fine), color=COLORS["exact"], lw=1.0,
          label="exact $e^{t}$")
right.plot(t_fine, np.exp(-t_fine), color=COLORS["exact"], lw=1.0,
           label="exact $e^{-t}$")
for name in ("euler", "rk4"):
    t_n, a_n, b_n = curves[name]
    style = dict(color=COLORS[name], marker=MARKERS[name], ms=5, lw=1.2)
    left.plot(t_n, a_n, **style, label=f"{LABELS[name]}, $h = 0.25$")
    right.plot(t_n, b_n, **style, label=f"{LABELS[name]}, $h = 0.25$")
left.set_title("3-space factor $a$: $da/dx_4 = +a$ (inflates)")
right.set_title("extra-time factor $b$: $db/dx_4 = -b$ (deflates)")
for ax in (left, right):
    ax.set_xlabel("time $x_4$ (units $1/H$, $A = 1$)")
    ax.legend(fontsize=8)
left.set_ylabel("scale factor (pure number)")
save_figure(fig, "scale_factors",
            "Left: the 3-space scale factor $a = e^{x_4}$, which obeys "
            "$da/dx_4 = a$; right: the extra-time scale factor $b = e^{-x_4}$, "
            "which obeys $db/dx_4 = -b$, along the linear history $a_4 = A H x_4$ "
            "with $A = H = 1$ at a fixed hidden position. Horizontal axis the time "
            "$x_4$ in units of $1/H$, vertical axis the scale factor (a pure "
            "number). Black: the exact curves; squares: Euler with the step "
            "$h = 0.25$; circles: RK4 with the same step. Euler falls behind the "
            "growing curve and decays too fast; RK4 lies on both exact curves.")
a_euler, b_euler = curves["euler"][1][-1], curves["euler"][2][-1]
a_rk4, b_rk4 = curves["rk4"][1][-1], curves["rk4"][2][-1]
report("a(2): exact, Euler, RK4", f"{math.exp(2):.6f}, {a_euler:.6f}, {a_rk4:.6f}")
report("b(2): exact, Euler, RK4", f"{math.exp(-2):.6f}, {b_euler:.6f}, {b_rk4:.6f}")
check(abs(a_euler - 1.25 ** 8) < 1e-12 and abs(b_euler - 0.75 ** 8) < 1e-15,
      "Euler gives exactly 1.25^8 and 0.75^8 after 8 steps")
check(abs(a_rk4 / math.exp(2) - 1) < 1e-3 and abs(b_rk4 / math.exp(-2) - 1) < 1e-3,
      "RK4 with h = 0.25 is within 0.1 percent of e^2 and e^(-2)")
```

Out [5]:

```
Figure 02a.1 saved as Revision/textbook/figures/02a_1_scale_factors.png
RESULT a(2): exact, Euler, RK4 = 7.389056, 5.960464, 7.388665
RESULT b(2): exact, Euler, RK4 = 0.135335, 0.100113, 0.135346
PASS Euler gives exactly 1.25^8 and 0.75^8 after 8 steps
PASS RK4 with h = 0.25 is within 0.1 percent of e^2 and e^(-2)
```

Text cell:

```
The exact product $a b = e^{t} e^{-t}$ is $1$ at every time: the inflation of a
3-space direction is compensated exactly by the deflation of an extra-time
```

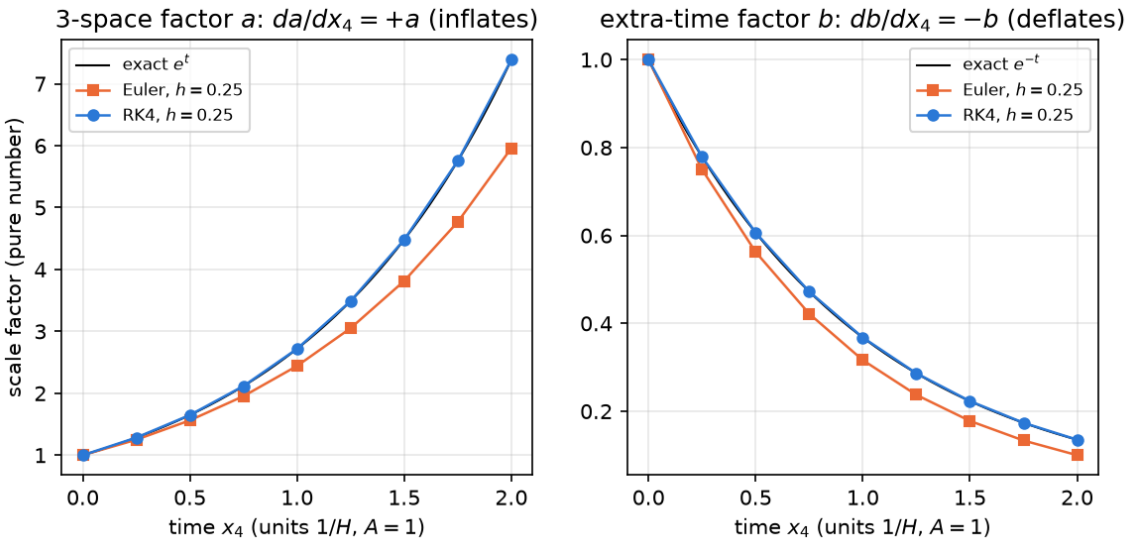

Figure 62: Left: the 3-space scale factor $a = e^{x_4}$, which obeys $da/dx_4 = a$; right: the extra-time scale factor $b = e^{-x_4}$, which obeys $db/dx_4 = -b$, along the linear history $a_4 = AHx_4$ with $A = H = 1$ at a fixed hidden position. Horizontal axis the time x_4 in units of $1/H$, vertical axis the scale factor (a pure number). Black: the exact curves; squares: Euler with the step $h = 0.25$; circles: RK4 with the same step. Euler falls behind the growing curve and decays too fast; RK4 lies on both exact curves. (Notebook 02a, figure 1.)

```
direction. A numerical method multiplies $a$ by $R(h)$ and $b$ by $R(-h)$ per
step, so it multiplies the product by $R(h) R(-h)$. For Euler that is
$(1 + h)(1 - h) = 1 - h^2$: the product shrinks by the factor $1 - h^2$ at every
step, however long we compute. For RK4 the cell lets sympy multiply out
$R(h) R(-h)$: the terms with $h$, $h^2$, $h^3$, $h^4$ and $h^5$ all cancel and
$1 + h^6/72 + h^8/576$ remains, which is much closer to $1$. The left panel of
the figure shows the product itself, the right panel its distance from $1$ on a
logarithmic scale, where both methods can be seen.
```

In [6]:

```
PRODUCT_FACTOR = {name: sp.expand(R[name].subs(z, hs) * R[name].subs(z, -hs))
                  for name in ("euler", "rk4")}
for name, factor in PRODUCT_FACTOR.items():
    say(f"factor of a b per step, {LABELS[name]:5}: {factor}")
check(PRODUCT_FACTOR["euler"] == 1 - hs ** 2
      and PRODUCT_FACTOR["rk4"] == 1 + hs ** 6 / 72 + hs ** 8 / 576,
      "R(h) R(-h) is 1 - h^2 (Euler) and 1 + h^6/72 + h^8/576 (RK4)")

fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.axhline(1.0, color=COLORS["exact"], lw=1.0, label="exact $a b = 1$")
products = {}
for name in ("euler", "rk4"):
    t_n, a_n, b_n = curves[name]
    products[name] = a_n * b_n  # the computed product at every step
    factor = float(PRODUCT_FACTOR[name].subs(hs, sp.Rational(1, 4)))  # h = 1/4
```

```
    style = dict(color=COLORS[name], marker=MARKERS[name], ms=5, lw=1.2)
    left.plot(t_n, products[name], **style, label=f"{LABELS[name]}, $h = 0.25$")
    right.semilogy(t_n[1:], np.abs(products[name][1:] - 1.0), **style,
                   label=f"{LABELS[name]}: measured")
    predicted = np.abs(factor ** np.arange(1, STEPS + 1) - 1.0)
    right.semilogy(t_n[1:], predicted, ":", color=COLORS["exact"], lw=1.0)
right.plot([], [], ":", color=COLORS["exact"], label="predicted $|R(h)^n R(-h)^n - 1|$")
left.set_ylabel("product $a b$ (pure number)")
right.set_ylabel("$|a b - 1|$ (pure number)")
left.set_title("the product $a b$")
right.set_title("its distance from 1 (logarithmic axis)")
for ax in (left, right):
    ax.set_xlabel("time $x_4$ (units $1/H$)")
    ax.legend(fontsize=8)
save_figure(fig, "scale_factor_product",
            "Left: the product $a b$ of the 3-space factor and the extra-time "
            "factor of the previous figure, computed with the step $h = 0.25$; "
            "right: its distance $|a b - 1|$ from the exact value on a logarithmic "
            "axis. Horizontal axes the time $x_4$ in units of $1/H$; the product is "
            "a pure number. Exactly $a b = e^{x_4} e^{-x_4} = 1$ (black line). Euler "
            "(squares) multiplies the product by $1 - h^2 = 0.9375$ at every step "
            "and ends at $0.9375^8 = 0.597$; RK4 (circles) multiplies it by "
            "$1 + h^6/72 + h^8/576$ and ends within $3 \\times 10^{-5}$ of 1. The "
            "dotted lines are these predictions; the measured points lie on them.")
check(abs(products["euler"][-1] - (1 - H_COARSE ** 2) ** 8) < 1e-14,
      "Euler multiplies the product a b by 1 - h^2 per step")
rk4_factor = 1 + H_COARSE ** 6 / 72 + H_COARSE ** 8 / 576
check(abs(products["rk4"][-1] - rk4_factor ** 8) < 1e-14
      and abs(products["rk4"][-1] - 1) < 3e-5,
      "RK4 multiplies a b by 1 + h^6/72 + h^8/576 per step (within 3e-5 of 1)")
```

Out [6]:

```
factor of a b per step, Euler: 1 - h**2
factor of a b per step, RK4  : h**8/576 + h**6/72 + 1
PASS R(h) R(-h) is 1 - h^2 (Euler) and 1 + h^6/72 + h^8/576 (RK4)
```

Out [6]:

```
Figure 02a.2 saved as Revision/textbook/figures/02a_2_scale_factor_product.png
PASS Euler multiplies the product a b by 1 - h^2 per step
PASS RK4 multiplies a b by 1 + h^6/72 + h^8/576 per step (within 3e-5 of 1)
```

Text cell:

```
## 9. The convergence study: errors on a log-log plot

Now the main experiment. For each method we solve problem A from $t = 0$ to
$t = 1$ with $N = 2, 4, 8, \dots, 4096$ steps ($h = 1/N$) and record the error
$|y_N - e^{-1}|$ at the end time. The cell prints the table and fits a straight
line $\log e = \log C + p \log h$ through the points where the error is still
```

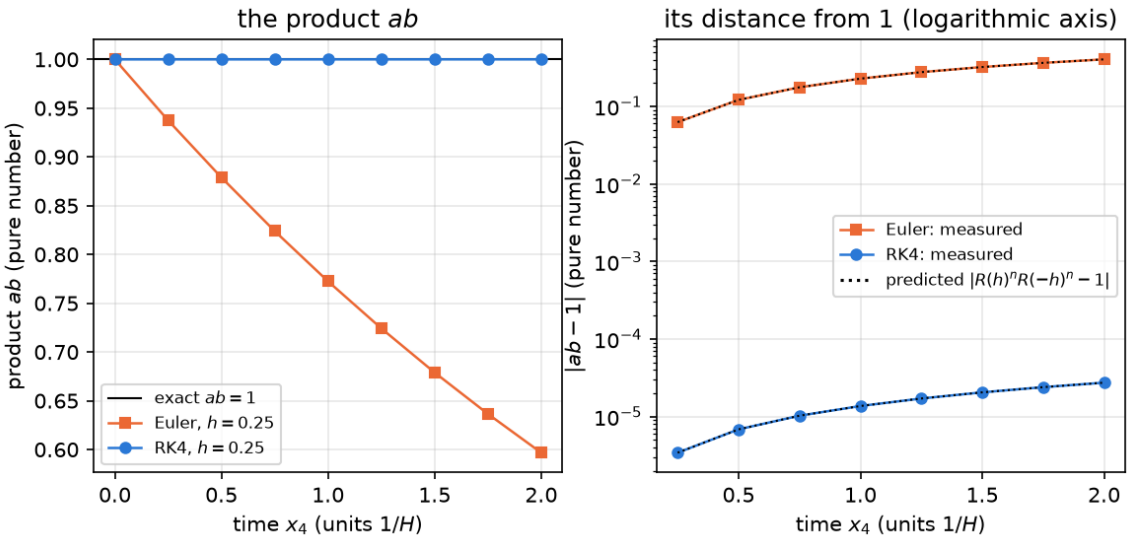

Figure 63: Left: the product ab of the 3-space factor and the extra-time factor of the previous figure, computed with the step $h = 0.25$; right: its distance $|ab - 1|$ from the exact value on a logarithmic axis. Horizontal axes the time x_4 in units of $1/H$; the product is a pure number. Exactly $ab = e^{x_4}e^{-x_4} = 1$ (black line). Euler (squares) multiplies the product by $1 - h^2 = 0.9375$ at every step and ends at $0.9375^8 = 0.597$; RK4 (circles) multiplies it by $1 + h^6/72 + h^8/576$ and ends within 3×10^{-5} of 1. The dotted lines are these predictions; the measured points lie on them. (Notebook 02a, figure 2.)

```
much larger than the rounding errors, with numpy's 'polyfit' (a least-squares
fit: the line with the smallest sum of squared vertical distances). The slope
$p$ is the measured order. The table also reproduces the classic numbers of
Euler's method, $(1 - h)^N$ = 0.250000, 0.316406, 0.343609, 0.356074 for
$h = 1/2, 1/4, 1/8, 1/16$.
```

In [7]:

```
EXACT_A = math.exp(-1.0)  # the exact value y(1) = e^(-1)
N_LIST = [2 ** k for k in range(1, 13)]  # 2, 4, ..., 4096 steps
ends = {name: [solve(step, decay, 1.0, 1.0, n)[1][-1] for n in N_LIST]
        for name, step in METHODS.items()}
errors = {name: np.abs(np.array(values) - EXACT_A) for name, values in ends.items()}
say("     N        h     Euler y_N   error Euler  error midpoint     error RK4")
for i, n in enumerate(N_LIST):
    y_euler = ends["euler"][i]  # the Euler value y_N
    e_euler, e_mid, e_rk4 = (errors[name][i] for name in METHODS)  # three errors
    say(f"{n:6d} {1 / n:8.6f}  {y_euler:12.6f}  {e_euler:11.3e}  {e_mid:14.3e}  "
        f"{e_rk4:12.3e}")

h_array = 1.0 / np.array(N_LIST, dtype=float)
FIT = {"euler": slice(5, 12), "midpoint": slice(5, 12), "rk4": slice(1, 8)}
slopes = {}
for name, part in FIT.items():
    # polyfit(x, y, 1) returns (slope, intercept) of the best straight line.
```

```
    slope, _ = np.polyfit(np.log10(h_array[part]), np.log10(errors[name][part]), 1)
    slopes[name] = slope
    report(f"measured order of {LABELS[name]} (slope)", f"{slope:.4f}")
check([round(v, 6) for v in ends["euler"][:4]]
      == [0.25, 0.316406, 0.343609, 0.356074],
      "Euler values (1 - h)^N for h = 1/2, 1/4, 1/8, 1/16")
check(abs(slopes["euler"] - 1) < 0.02 and abs(slopes["midpoint"] - 2) < 0.02
      and abs(slopes["rk4"] - 4) < 0.1,
      "the measured orders are 1, 2 and 4")
```

Out [7]:

```
     N        h     Euler y_N   error Euler  error midpoint     error RK4
     2 0.500000      0.250000    1.179e-01       2.275e-02     2.914e-04
     4 0.250000      0.316406    5.147e-02       4.650e-03     1.476e-05
     8 0.125000      0.343609    2.427e-02       1.054e-03     8.308e-07
    16 0.062500      0.356074    1.181e-02       2.511e-04     4.928e-08
    32 0.031250      0.362055    5.824e-03       6.130e-05     3.001e-09
    64 0.015625      0.364987    2.893e-03       1.515e-05     1.851e-10
   128 0.007812      0.366438    1.442e-03       3.764e-06     1.149e-11
   256 0.003906      0.367160    7.197e-04       9.383e-07     7.159e-13
   512 0.001953      0.367520    3.595e-04       2.342e-07     4.452e-14
  1024 0.000977      0.367700    1.797e-04       5.852e-08     2.054e-15
  2048 0.000488      0.367790    8.983e-05       1.462e-08     1.665e-16
  4096 0.000244      0.367835    4.491e-05       3.655e-09     1.110e-16
RESULT measured order of Euler (slope) = 1.0014
RESULT measured order of midpoint (slope) = 2.0025
RESULT measured order of RK4 (slope) = 4.0439
PASS Euler values (1 - h)^N for h = 1/2, 1/4, 1/8, 1/16
PASS the measured orders are 1, 2 and 4
```

Text cell:

```
The next cell draws the table as a log-log plot. The grey dashed guide lines have
the slopes 1, 2 and 4 exactly; each passes through the last fitted point of its
method. The measured points must lie on lines parallel to them.
```

In [8]:

```
fig, ax = plt.subplots(figsize=(7.0, 5.0))
for name in METHODS:
    ax.loglog(h_array, errors[name], color=COLORS[name], marker=MARKERS[name],
              ms=5, lw=1.2, label=f"{LABELS[name]} (measured slope "
              f"{slopes[name]:.2f})")
    order = {"euler": 1, "midpoint": 2, "rk4": 4}[name]
    anchor = FIT[name].stop - 1  # the last fitted point
    guide = errors[name][anchor] * (h_array / h_array[anchor]) ** order
    ax.loglog(h_array, guide, "--", color=COLORS["guide"], lw=0.9)
    ax.text(h_array[0] * 1.15, guide[0], f"slope {order}", fontsize=8,
            color=COLORS["guide"], va="center")
ax.set_xlim(h_array[-1] / 1.5, h_array[0] * 3.0)
```

```
ax.set_ylim(1e-17, 1.0)
ax.set_xlabel("step size $h$ (time units)")
ax.set_ylabel("error $|y_N - e^{-1}|$ at $t = 1$")
ax.set_title("Problem A: error at the end time against the step size")
ax.legend(loc="lower right")
save_figure(fig, "convergence_loglog",
            "The error at the end time $t = 1$ of problem A, $y' = -y$, for the "
            "Euler method (squares), the midpoint method (triangles) and RK4 "
            "(circles), against the step size $h$ from $1/2$ down to $1/4096$, on "
            "logarithmic axes (each tick a power of ten; time in arbitrary units, "
            "the error a pure number). The dashed grey lines have the slopes 1, 2 "
            "and 4. The points lie on straight lines parallel to them: the error is "
            "$C h^p$ with the orders $p = 1, 2, 4$. The RK4 points bend away at the "
            "bottom left, where rounding errors take over.")
check(errors["rk4"][7] < 1e-11 < errors["midpoint"][7] < errors["euler"][7],
      "at N = 256: RK4 error < 1e-11 < midpoint error < Euler error")
```

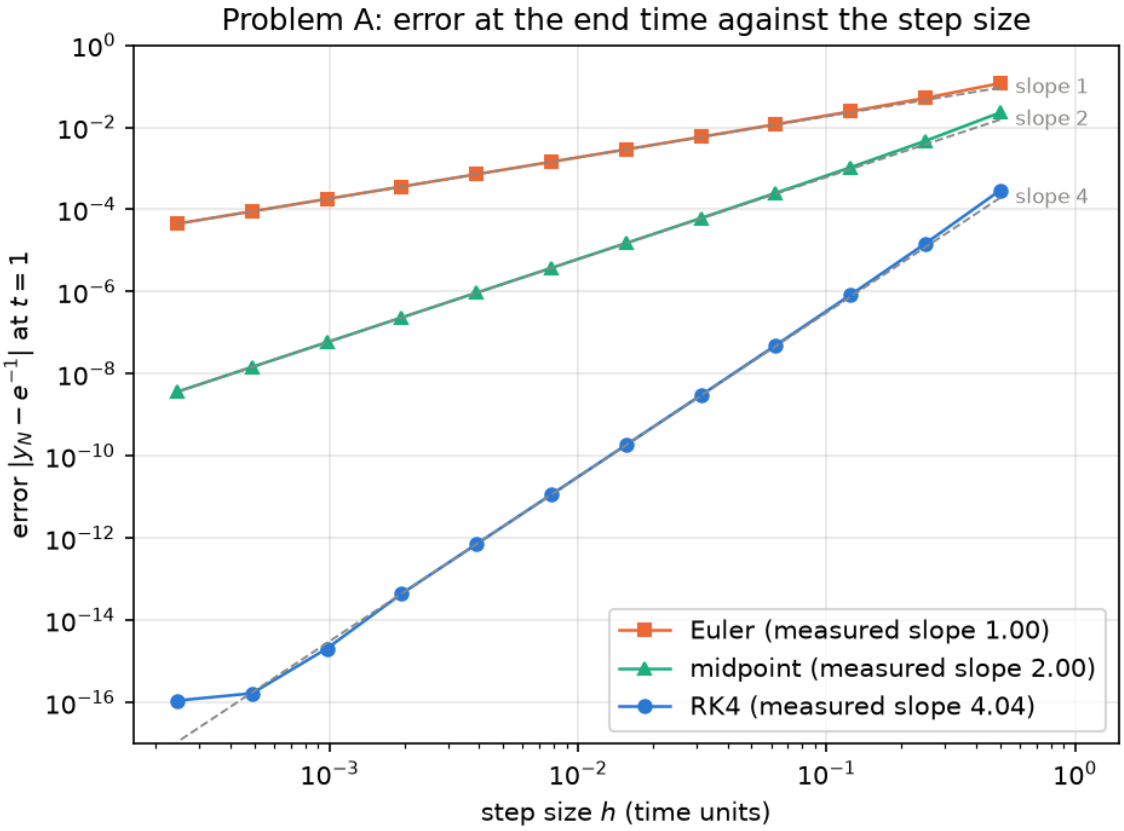

Figure 64: The error at the end time $t = 1$ of problem A, $y' = -y$, for the Euler method (squares), the midpoint method (triangles) and RK4 (circles), against the step size h from 1/2 down to 1/4096, on logarithmic axes (each tick a power of ten; time in arbitrary units, the error a pure number). The dashed grey lines have the slopes 1, 2 and 4. The points lie on straight lines parallel to them: the error is Ch^p with the orders $p = 1, 2, 4$. The RK4 points bend away at the bottom left, where rounding errors take over. (Notebook 02a, figure 3.)

Out [8]:

```
Figure 02a.3 saved as Revision/textbook/figures/02a_3_convergence_loglog.png
PASS at N = 256: RK4 error < 1e-11 < midpoint error < Euler error
```

Text cell:

```
## 10. What halving the step does

If the error is $C h^p$, then halving $h$ divides it by exactly $2^p$: by 2 for
Euler, 4 for the midpoint method and 16 for RK4. The next cell computes the
ratios $e(h)/e(h/2)$ for successive rows of the table, draws them, and checks the
ratios where the error is not yet limited by rounding.
```

In [9]:

```
ratios = {name: errors[name][:-1] / errors[name][1:] for name in METHODS}
fig, ax = plt.subplots()
for name in METHODS:
    shown = slice(0, 8) if name == "rk4" else slice(0, 11)  # RK4: before rounding
    ax.semilogx(N_LIST[1:][shown], ratios[name][shown], color=COLORS[name],
                marker=MARKERS[name], ms=5, lw=1.2, label=LABELS[name])
for target in (2, 4, 16):
    # a dashed line from N = 3 to just beyond the data, its label to the right
    ax.hlines(target, 3.0, N_LIST[-1] * 1.4, ls="--", color=COLORS["guide"], lw=0.9)
    ax.text(N_LIST[-1] * 1.6, target, f"$2^{{{int(math.log2(target))}}}$ = "
            f"{target}", va="center", ha="left", fontsize=8, color=COLORS["guide"])
ax.set_xlim(3.0, N_LIST[-1] * 8.0)  # room on the right for the labels
ax.set_xlabel("number of steps $N$ of the finer run ($h = 1/N$)")
ax.set_ylabel("error ratio $e(2h)/e(h)$")
ax.set_title("Halving the step divides the error by $2^p$")
ax.legend(loc="center left")
save_figure(fig, "error_ratios",
            "The factor by which the error of problem A at $t = 1$ shrinks when the "
            "step is halved, against the number of steps $N$ of the finer run "
            "(logarithmic horizontal axis; the ratio is a pure number). The ratios "
            "approach 2 for Euler (squares), 4 for the midpoint method "
            "(triangles) and 16 for RK4 (circles), the values $2^p$ for the "
            "orders $p = 1, 2, 4$ (dashed grey lines). RK4 is drawn only up to "
            "$N = 512$; beyond, its error is rounding noise.")
ratio_euler = ratios["euler"][-1]  # the last ratio: N = 2048 -> 4096
ratio_midpoint = ratios["midpoint"][-1]
ratio_rk4 = ratios["rk4"][5]  # N = 64 -> 128, before rounding matters
report("ratio Euler at N = 4096", f"{ratio_euler:.5f}")
report("ratio midpoint at N = 4096", f"{ratio_midpoint:.5f}")
report("ratio RK4 at N = 128", f"{ratio_rk4:.4f}")
check(abs(ratio_euler - 2) < 0.001 and abs(ratio_midpoint - 4) < 0.001
      and abs(ratio_rk4 - 16) < 0.2,
      "halving h divides the errors by 2, 4 and 16")
```

Out [9]:

```
Figure 02a.4 saved as Revision/textbook/figures/02a_4_error_ratios.png
RESULT ratio Euler at N = 4096 = 2.00020
```

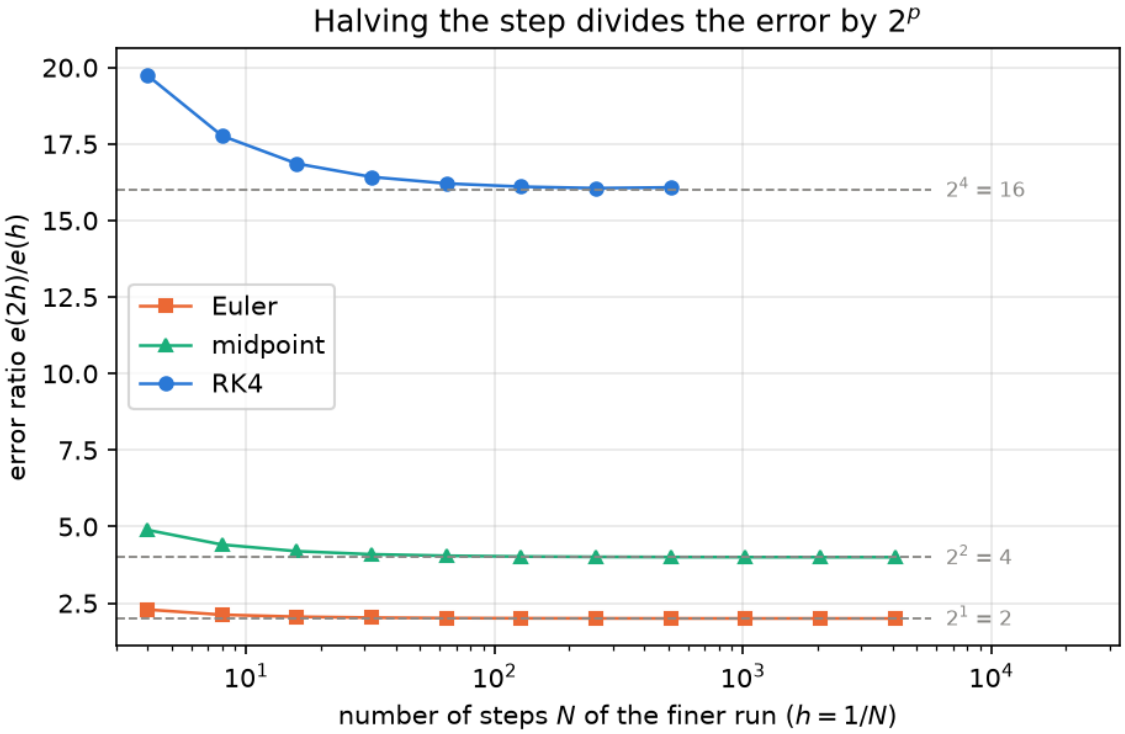

Figure 65: The factor by which the error of problem A at $t = 1$ shrinks when the step is halved, against the number of steps N of the finer run (logarithmic horizontal axis; the ratio is a pure number). The ratios approach 2 for Euler (squares), 4 for the midpoint method (triangles) and 16 for RK4 (circles), the values 2^p for the orders $p = 1, 2, 4$ (dashed grey lines). RK4 is drawn only up to $N = 512$; beyond, its error is rounding noise. (Notebook 02a, figure 4.)

```
RESULT ratio midpoint at N = 4096 = 4.00073
RESULT ratio RK4 at N = 128 = 16.1047
PASS halving h divides the errors by 2, 4 and 16
```

Text cell:

```
## 11. Where rounding takes over

A smaller step is not always better. Every arithmetic operation rounds its result
to about 16 significant digits, and with $N$ steps about $N$ such rounding errors
add up. If they all had their largest size and the same sign, their total would
be about $N \epsilon$ times the size of the solution: a pessimistic bound. In
practice they have both signs and partly cancel, so the total stays far below
this bound, but it still grows with the number of steps $N = 1/h$, while the
truncation error of RK4 falls like $h^4$. The next cell runs RK4 for problem A
with up to $2^{18} = 262144$ steps (this takes about a second) and draws both
effects: the error falls along the slope-4 line until it reaches about
$10^{-16}$, then rises again, slowly and far below the bound.
```

In [10]:

```
N_LONG = [2 ** k for k in range(1, 19)]  # 2 ... 262144 steps
rk4_long = np.array([abs(solve(rk4_step, decay, 1.0, 1.0, n)[1][-1] - EXACT_A)
                     for n in N_LONG])
h_long = 1.0 / np.array(N_LONG, dtype=float)
best = int(np.argmin(rk4_long))  # the position of the smallest error
EPS = 2.0 ** -52  # the machine epsilon
fig, ax = plt.subplots()
shown = rk4_long > 0  # an error of exactly 0 cannot be drawn on a log axis
ax.loglog(h_long[shown], rk4_long[shown], color=COLORS["rk4"], marker="o", ms=5,
          lw=1.2, label="RK4: measured error")
ax.loglog(h_long, errors["rk4"][3] * (h_long / h_array[3]) ** 4, "--",
          color=COLORS["guide"], lw=0.9, label="truncation $C h^4$")
ax.loglog(h_long, EPS * EXACT_A / h_long, ":", color=COLORS["exact"], lw=1.0,
          label="rounding bound $N \\epsilon\\, e^{-1}$")
ax.set_ylim(1e-18, 1e-2)
ax.set_xlabel("step size $h = 1/N$ (time units)")
ax.set_ylabel("error $|y_N - e^{-1}|$")
ax.set_title("RK4: truncation error against rounding error")
ax.legend(loc="upper center")
save_figure(fig, "rounding_floor",
            "The error of RK4 for problem A at $t = 1$ against the step size $h$ "
            "from $1/2$ to $1/262144$, on logarithmic axes (time in arbitrary "
            "units, the error a pure number). Circles: measured. Dashed grey: the "
            "truncation error $C h^4$. Dotted black: the pessimistic bound "
            "$N \\epsilon\\, e^{-1}$, reached only if all $N$ rounding errors of "
            "relative size $\\epsilon = 2^{-52}$ had their largest size and the "
            "same sign. Coming from the right, the error falls with slope 4 down "
            "to about $10^{-16}$; for smaller steps it grows again, but slowly "
            "and hundreds to thousands of times below the dotted bound, because "
            "the rounding errors partly cancel.")
report("smallest RK4 error", f"{rk4_long[best]:.3e} at N = {N_LONG[best]}")
```

```
report("RK4 error at N = 262144", f"{rk4_long[-1]:.3e}")
bound = EPS * EXACT_A * np.array(N_LONG, dtype=float)  # the dotted line
below = bound[best + 1:] / rk4_long[best + 1:]  # past the minimum: bound / error
report("bound N eps e^-1 / measured error past the minimum (smallest, largest)",
       f"{below.min():.0f}, {below.max():.0f}")
check(rk4_long[best] < 1e-15 and 256 <= N_LONG[best] <= 16384,
      "the smallest RK4 error is below 1e-15, reached at N between 256 and 16384")
check(rk4_long[-1] > 10 * max(rk4_long[best], EPS * EXACT_A)
      and rk4_long[-1] < 1e-11,
      "with 262144 steps rounding has made the error grow again")
check(np.all(below > 100),
      "past the minimum the error stays over 100 times below the bound N eps e^-1")
```

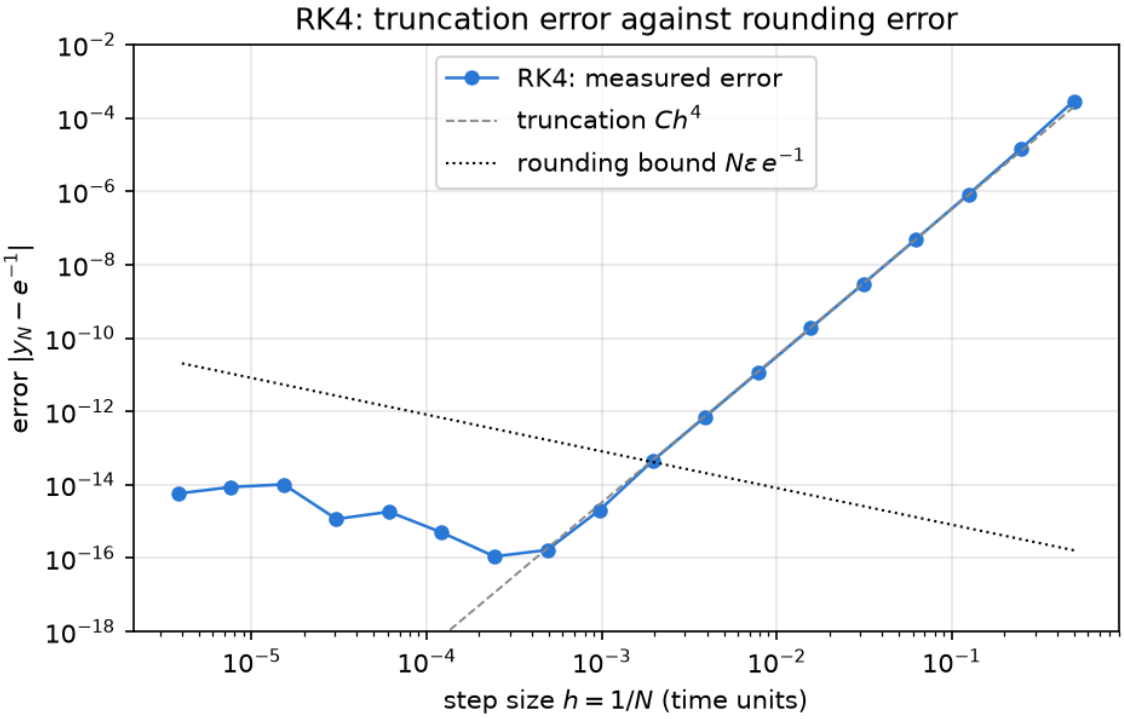

Figure 66: The error of RK4 for problem A at $t = 1$ against the step size h from 1/2 to 1/262144, on logarithmic axes (time in arbitrary units, the error a pure number). Circles: measured. Dashed grey: the truncation error Ch^4. Dotted black: the pessimistic bound $N\epsilon\, e^{-1}$, reached only if all N rounding errors of relative size $\epsilon = 2^{-52}$ had their largest size and the same sign. Coming from the right, the error falls with slope 4 down to about 10^{-16}; for smaller steps it grows again, but slowly and hundreds to thousands of times below the dotted bound, because the rounding errors partly cancel. (Notebook 02a, figure 5.)

Out [10]:

```
Figure 02a.5 saved as Revision/textbook/figures/02a_5_rounding_floor.png
RESULT smallest RK4 error = 1.110e-16 at N = 4096
RESULT RK4 error at N = 262144 = 5.773e-15
RESULT bound N eps e^-1 / measured error past the minimum (smallest, largest) = 524, 3709
```

```
PASS the smallest RK4 error is below 1e-15, reached at N between 256 and 16384
PASS with 262144 steps rounding has made the error grow again
PASS past the minimum the error stays over 100 times below the bound N eps e^-1
```

Text cell:

```
## 12. The step of the Revision Kohn-Sham solver, and the first missed term

The Rust program of the Revision record that solves the Kohn-Sham equations of
this book makes $G$ equal RK4 steps over an interval of length $L$ of the hidden
coordinate. The next cell reads $G$ and $L$ from the record
Revision/kohn_sham/results/parameters.json, together with the constants $A$ and
$H$ of the linear history $a_4 = A H x_4$ that section 8 used, and computes the
solver's step $h = L/G$. Here we use only this step, not the solver's equations.

How large is the error of a method of order $p$ at this step? For problem A we
can say it exactly to leading order. One step multiplies $y$ by $R(-h)$ instead
of the exact factor $e^{-h} = 1 - h + h^2/2 - h^3/6 + \dots$ Section 7 showed
that $R$ keeps the terms of this series up to the power $h^p$, so $R(-h)$
differs from $e^{-h}$ first by the missed term with the power $h^{p + 1}$:

$$R(-h) = e^{-h} - \frac{(-h)^{p+1}}{(p+1)!} + \dots$$

(the dots stand for terms with higher powers of $h$). Taking out the factor
$e^{-h}$ gives $R(-h) = e^{-h}(1 - q)$ with $q = e^{h}(-h)^{p+1}/(p+1)! + \dots
\approx (-h)^{p+1}/(p+1)!$, because $e^{h} = 1 + h + \dots$ changes only the
higher terms. After $N = 1/h$ steps (up to $t = 1$) the computed value is
$y_N = R(-h)^N = e^{-N h}(1 - q)^N = e^{-1}(1 - q)^N$. For a small $q$,
$(1 - q)^N \approx 1 - N q$ (the first two terms of the binomial theorem), and
$N q = (-1)^{p+1} h^{p}/(p+1)!$ (one factor $h$ cancels against $N = 1/h$). So

$$y_N - e^{-1} \approx e^{-1}\, \frac{(-1)^{p}\, h^{p}}{(p+1)!} .$$

Euler ($p = 1$) ends too LOW by $e^{-1} h/2$; the midpoint method ($p = 2$)
too HIGH by $e^{-1} h^2/6$; RK4 ($p = 4$) too high by $e^{-1} h^4/120$. The
cell solves problem A with the solver's step and compares the measured signed
errors with these predictions.
```

In [11]:

```
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
G_KS = parameters["numerics"]["rk4Steps"]  # the solver's number of RK4 steps
L_KS = parameters["physics"]["L_tipCutoff"]  # the length of its interval in y
A_KS = parameters["physics"]["historyA"]  # the constant A of a4 = A H x4
H_KS = parameters["physics"]["H"]  # the author's constant H
h_ks = L_KS / G_KS  # the solver's step
report("solver: RK4 steps G, interval length L, step h = L/G",
       f"{G_KS}, {L_KS}, {h_ks:.10f}")
report("linear history a4 = A H x4 of the record: A, H", f"{A_KS}, {H_KS}")
check(G_KS == 900 and L_KS == 3.0 and A_KS == 1.0 and H_KS == 1.0,
      "the solver makes 900 RK4 steps on L = 3 (h = 1/300); the history has A = H = 1",
```

```
      record=f"{PARAMETERS}, keys rk4Steps, L_tipCutoff, historyA and H")

N_KS = round(1.0 / h_ks)  # 300 steps of h = 1/300 reach t = 1
ORDER = {"euler": 1, "midpoint": 2, "rk4": 4}  # the orders measured in section 9
agreement = []
for name, step in METHODS.items():
    p = ORDER[name]
    measured = solve(step, decay, 1.0, 1.0, N_KS)[1][-1] - EXACT_A  # signed error
    predicted = EXACT_A * (-1) ** p * h_ks ** p / math.factorial(p + 1)
    agreement.append(measured / predicted)
    say(f"{LABELS[name]:9} with h = 1/{N_KS}: error {measured:+.4e}, predicted "
        f"{predicted:+.4e}, ratio {measured / predicted:.5f}")
check(all(abs(ratio - 1) < 0.01 for ratio in agreement),
      "at the solver's step the errors are e^(-1) (-1)^p h^p/(p+1)! within 1 %")
```

Out [11]:

```
RESULT solver: RK4 steps G, interval length L, step h = L/G = 900, 3.0, 0.0033333333
RESULT linear history a4 = A H x4 of the record: A, H = 1.0, 1.0
PASS the solver makes 900 RK4 steps on L = 3 (h = 1/300); the history has A = H = 1
     reproduces Revision/kohn_sham/results/parameters.json, keys rk4Steps, L_tipCutoff,
    historyA and H
Euler     with h = 1/300: error -6.1399e-04, predicted -6.1313e-04, ratio 1.00139
midpoint  with h = 1/300: error +6.8296e-07, predicted +6.8126e-07, ratio 1.00250
RK4       with h = 1/300: error +3.7920e-13, predicted +3.7848e-13, ratio 1.00190
PASS at the solver's step the errors are e^(-1) (-1)^p h^p/(p+1)! within 1 %
```

Text cell:

```
## 13. The oscillator: phase portrait and energy

Problem B as a system: the state is the vector $Y = (x, v)$ and the right-hand
side is $f(t, Y) = (v, -x)$. The exact solution goes round the circle
$x^2 + v^2 = 1$ in the $(x, v)$ plane, once every $2\pi$ time units. The next cell
solves it with the step $h = 0.2$ up to $t = 10$ (50 steps) with each method and
draws the three phase portraits. Section 7 predicts what happens to the energy
$E = (x^2 + v^2)/2$: each step multiplies it by $1 + h^2 = 1.04$ (Euler: the
point spirals outwards), by $1 + h^4/4 = 1.0004$ (midpoint) and by
$1 - h^6/72 + h^8/576 = 0.99999911$ (RK4).

The radius $\sqrt{x^2 + v^2} = \sqrt{2E}$ of the midpoint method grows only to
$\sqrt{1.0004^{50}} = 1.010$ in 50 steps, too little to be seen on the whole
circle. The right panel therefore zooms in on the last three points, at
$t = 9.6$, $9.8$ and $10$, where crosses mark the exact solution at the same
times. There a second error shows: each step
turns the point by a slightly wrong angle. With $w = x + i v$, one step
multiplies $w$ by $R(-ih)$, whose angle is $-\arg R(ih)$ (the angle of a complex
number is called its *argument*, $\arg$), while the exact factor $e^{-ih}$ turns
by $-h$. So after 50 steps the computed point runs ahead of the exact one,
clockwise, by the angle $50\,(\arg R(ih) - h)$; a negative value means that it
lags behind. The cell checks this prediction for the three methods.
```

In [12]:

```
def oscillator(t, Y):
    """Problem B: x' = v, v' = -x for the state Y = (x, v)."""
    return np.array([Y[1], -Y[0]])


Y0 = np.array([1.0, 0.0])  # x(0) = 1, v(0) = 0
H_OSC = 0.2  # the step size
portraits = {name: np.array(solve(step, oscillator, Y0, 10.0, 50)[1])
             for name, step in METHODS.items()}
angle = np.linspace(0.0, 2 * np.pi, 400)
t_n = H_OSC * np.arange(51)  # the times 0, 0.2, ..., 10 of the 51 points
fig, (whole, zoom) = plt.subplots(1, 2, figsize=(10.0, 5.2))
for name in METHODS:  # left panel: the whole portrait of each method
    whole.plot(portraits[name][:, 0], portraits[name][:, 1], color=COLORS[name],
               marker=MARKERS[name], ms=3, lw=0.9,
               label=f"{LABELS[name]}, $h = 0.2$, 50 steps")
for name in ("midpoint", "rk4"):  # right panel: the last three points only
    zoom.plot(portraits[name][48:, 0], portraits[name][48:, 1], ls="none",
              color=COLORS[name], marker=MARKERS[name], ms=8, label=LABELS[name])
for ax in (whole, zoom):  # the exact circle, dashed, drawn on top (zorder 5)
    ax.plot(np.cos(angle), -np.sin(angle), "--", color=COLORS["exact"], lw=0.9,
            zorder=5, label="exact circle $x^2 + v^2 = 1$")
    ax.set_aspect("equal")
    ax.set_xlabel("position $x$")
    ax.set_ylabel("velocity $v$")
whole.plot([1.0], [0.0], "o", color=COLORS["exact"], ms=6, zorder=6)  # the start
zoom.plot(np.cos(t_n[48:]), -np.sin(t_n[48:]), "x", color=COLORS["exact"], ms=9,
          zorder=6, label="exact solution, same times")
zoom.set_xlim(-1.12, -0.74)  # a window round the points at t = 9.6, 9.8 and 10
zoom.set_ylim(0.12, 0.66)
whole.set_title("Phase portrait of $d^2x/dt^2 = -x$ up to $t = 10$")
zoom.set_title("Zoom: the points at $t = 9.6$, $9.8$, $10$")
whole.legend(loc="lower left", fontsize=8)
zoom.legend(loc="upper left", fontsize=8)
save_figure(fig, "phase_portrait",
            "Phase portrait of the oscillator $d^2x/dt^2 = -x$ started at $x = 1$, "
            "$v = 0$ (black dot): the velocity $v$ against the position $x$ "
            "(arbitrary units), computed with the step $h = 0.2$ up to $t = 10$. "
            "The exact solution runs clockwise round the dashed unit circle. "
            "Left: Euler (squares) spirals outwards, because every step "
            "multiplies the energy by $1 + h^2$; the midpoint method (triangles) "
            "and RK4 (circles) stay close to the circle. Right: a zoom on the "
            "last three points, at $t = 9.6$, $9.8$ and $10$; the crosses are "
            "the exact solution at these times. RK4 sits on them; the midpoint "
            "method has run ahead along the circle (by 0.066 rad at $t = 10$) "
            "and lies slightly outside it (radius 1.010).")
for name in METHODS:
    energy = 0.5 * (portraits[name][:, 0] ** 2 + portraits[name][:, 1] ** 2)
    factor = float(ENERGY_FACTOR[name].subs(hs, sp.Rational(1, 5)))  # at h = 0.2
    predicted = 0.5 * factor ** np.arange(51)  # E_0 times the factor n times
    report(f"energy after 50 steps, {LABELS[name]}", f"{energy[-1]:.10f}")
    check(np.max(np.abs(energy / predicted - 1)) < 1e-13,
```

```
          f"{LABELS[name]}: E_n = E_0 times the exact factor to the power n")
angle_agrees = []
for name in METHODS:
    x_end, v_end = portraits[name][-1]  # the point at t = 10
    # the clockwise angle from the exact point e^(-10 i) to the computed one
    ahead = -np.angle(complex(x_end, v_end) * np.exp(10j))
    R_ih = complex(R[name].subs(z, sp.I * sp.Rational(1, 5)))  # R(ih), h = 0.2
    predicted_ahead = 50 * (np.angle(R_ih) - 0.2)  # 50 (arg R(ih) - h)
    report(f"t = 10, {LABELS[name]}: radius, angle ahead of the exact point",
           f"{math.hypot(x_end, v_end):.4f}, {ahead:+.3e} rad")
    angle_agrees.append(abs(ahead - predicted_ahead) < 1e-12)
check(all(angle_agrees),
      "at t = 10 each point is ahead of the exact one by 50 (arg R(ih) - h)")
```

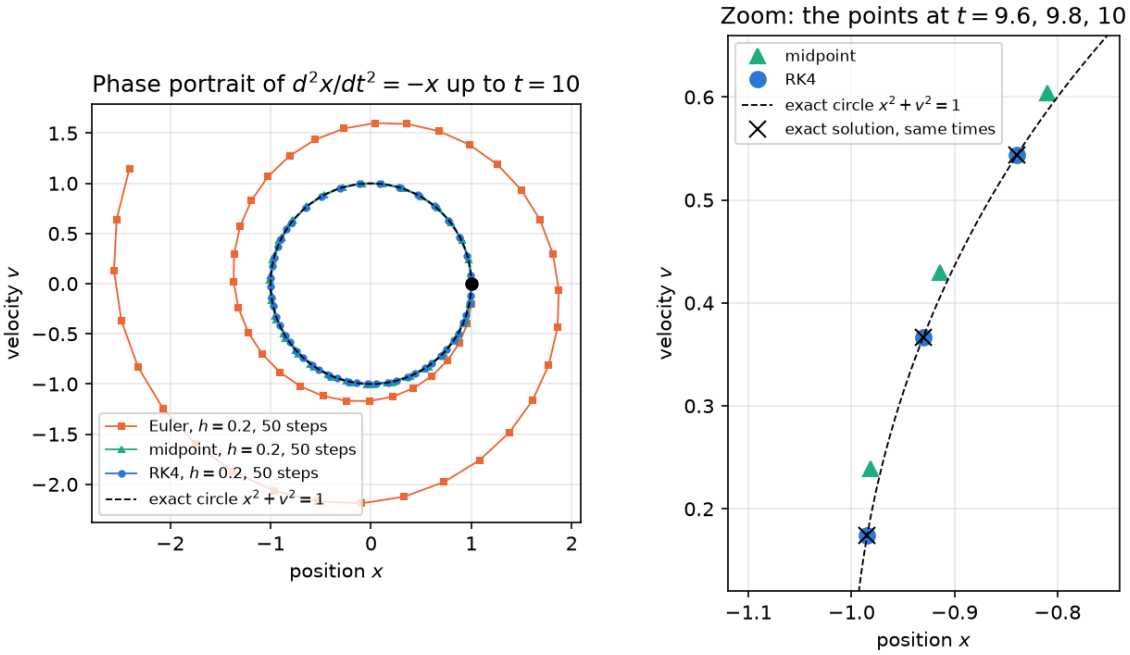

Figure 67: Phase portrait of the oscillator $d^2x/dt^2 = -x$ started at $x = 1$, $v = 0$ (black dot): the velocity v against the position x (arbitrary units), computed with the step $h = 0.2$ up to $t = 10$. The exact solution runs clockwise round the dashed unit circle. Left: Euler (squares) spirals outwards, because every step multiplies the energy by $1 + h^2$; the midpoint method (triangles) and RK4 (circles) stay close to the circle. Right: a zoom on the last three points, at $t = 9.6$, 9.8 and 10; the crosses are the exact solution at these times. RK4 sits on them; the midpoint method has run ahead along the circle (by 0.066 rad at $t = 10$) and lies slightly outside it (radius 1.010). (Notebook 02a, figure 6.)

Out [12]:

```
Figure 02a.6 saved as Revision/textbook/figures/02a_6_phase_portrait.png
RESULT energy after 50 steps, Euler = 3.5533416731
PASS Euler: E_n = E_0 times the exact factor to the power n
RESULT energy after 50 steps, midpoint = 0.5100986302
```

```
PASS midpoint: E_n = E_0 times the exact factor to the power n
RESULT energy after 50 steps, RK4 = 0.4999778894
PASS RK4: E_n = E_0 times the exact factor to the power n
RESULT t = 10, Euler: radius, angle ahead of the exact point = 2.6658, -1.302e-01 rad
RESULT t = 10, midpoint: radius, angle ahead of the exact point = 1.0100, +6.586e-02 rad
RESULT t = 10, RK4: radius, angle ahead of the exact point = 1.0000, -1.314e-04 rad
PASS at t = 10 each point is ahead of the exact one by 50 (arg R(ih) - h)
```

Text cell:

```
The next cell follows the energy much longer, up to $t = 100$ (500 steps of
$h = 0.2$), and draws the relative energy error $|E_n/E_0 - 1|$ on a logarithmic
scale. After $n$ steps the energy is $E_0 q^n$, where $q$ is the factor per step,
so the relative error is $|q^n - 1|$. On a logarithmic vertical axis a straight
rising line means that the plotted quantity is multiplied by the same factor at
every step: that is Euler's $q^n$ once it is much larger than 1. For the midpoint
method and RK4, $q$ is so close to 1 that $q^n - 1 \approx n (q - 1)$: the error
grows in proportion to $n$, and on the logarithmic axis that is a curve that
bends over. The three methods differ by many powers of ten.
```

In [13]:

```
long_runs = {name: np.array(solve(step, oscillator, Y0, 100.0, 500)[1])
             for name, step in METHODS.items()}
fig, ax = plt.subplots()
t_long = np.linspace(0.0, 100.0, 501)
drift = {}
for name in METHODS:
    energy = 0.5 * (long_runs[name][:, 0] ** 2 + long_runs[name][:, 1] ** 2)
    drift[name] = np.abs(energy / 0.5 - 1.0)
    ax.semilogy(t_long[1:], drift[name][1:], color=COLORS[name], lw=1.4,
                label=LABELS[name])
ax.set_xlabel("time $t$ (arbitrary units)")
ax.set_ylabel("relative energy error $|E_n/E_0 - 1|$")
ax.set_title("Energy drift of the oscillator, step $h = 0.2$")
ax.legend()
save_figure(fig, "energy_drift",
            "The relative error of the oscillator energy, $|E_n/E_0 - 1|$, on a "
            "logarithmic vertical axis, against the time $t$ from 0 to 100 "
            "(arbitrary units), for the step $h = 0.2$ (500 steps). Euler: the "
            "energy grows by the factor $1.04$ per step and is $3 \\times 10^{8}$ "
            "times too large at the end. Midpoint: it grows by the factor "
            "$1.0004$ per step (22 percent after 500 steps). RK4: it shrinks by "
            "the fraction $8.8 \\times 10^{-7}$ per step, a relative error below "
            "$5 \\times 10^{-4}$ at the end. The Euler line is straight (the same "
            "factor at every step); the other two curves bend over, because "
            "their small errors grow in proportion to the number of steps.")
report("energy errors at t = 100 (Euler, midpoint, RK4)",
       ", ".join(f"{drift[name][-1]:.3e}" for name in METHODS))
check(drift["euler"][-1] > 1e8 and 0.1 < drift["midpoint"][-1] < 0.3
      and drift["rk4"][-1] < 5e-4,
      "after 500 steps: Euler energy off by > 1e8, midpoint by 10-30 %, RK4 < 5e-4")
```

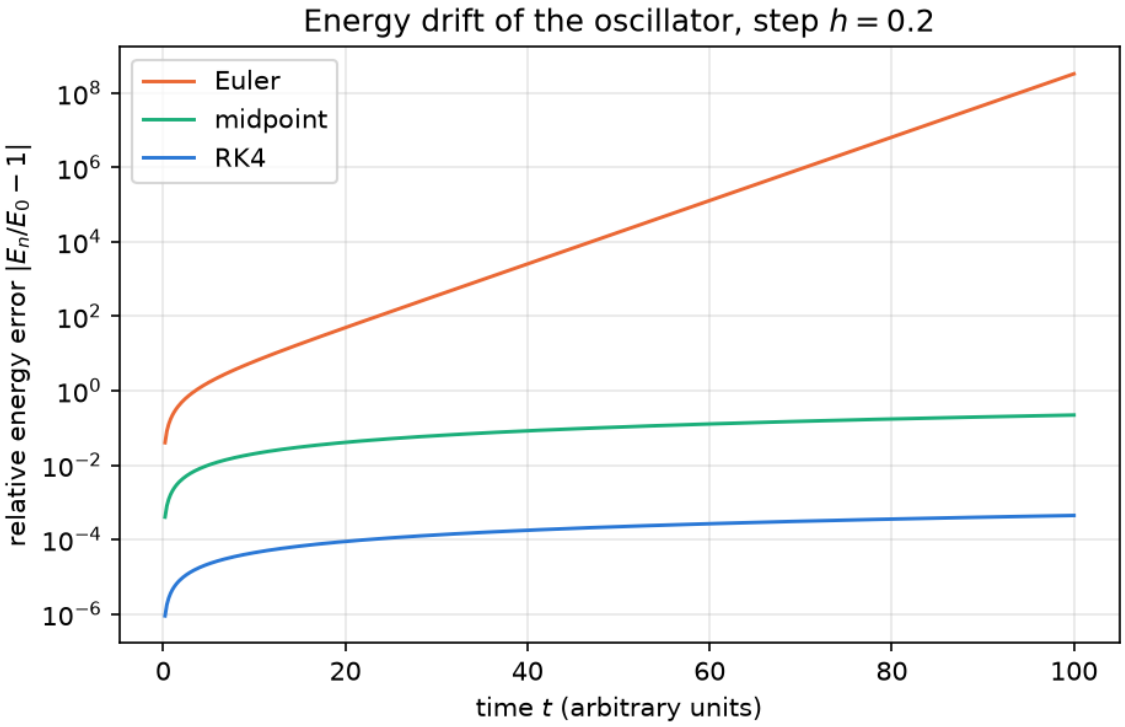

Figure 68: The relative error of the oscillator energy, $|E_n/E_0 - 1|$, on a logarithmic vertical axis, against the time t from 0 to 100 (arbitrary units), for the step $h = 0.2$ (500 steps). Euler: the energy grows by the factor 1.04 per step and is 3×10^8 times too large at the end. Midpoint: it grows by the factor 1.0004 per step (22 percent after 500 steps). RK4: it shrinks by the fraction 8.8×10^{-7} per step, a relative error below 5×10^{-4} at the end. The Euler line is straight (the same factor at every step); the other two curves bend over, because their small errors grow in proportion to the number of steps. (Notebook 02a, figure 7.)

Out [13]:

```
Figure 02a.7 saved as Revision/textbook/figures/02a_7_energy_drift.png
RESULT energy errors at t = 100 (Euler, midpoint, RK4) = 3.286e+08, 2.214e-01, 4.421e-04
PASS after 500 steps: Euler energy off by > 1e8, midpoint by 10-30 %, RK4 < 5e-4
```

Text cell:

```
## 14. Richardson's rule: an error estimate without the exact answer

In real problems the exact answer is unknown. Suppose a method of order $p$ gives
$Y_h = Y + C h^p$ and $Y_{h/2} = Y + C h^p / 2^p$, where $Y$ is the exact value.
Subtracting the second from the first, $Y_h - Y_{h/2} = C h^p (1 - 2^{-p}) =
(C h^p / 2^p)(2^p - 1)$, so the error of the finer result is

$$Y_{h/2} - Y = \frac{C h^p}{2^p} = \frac{Y_h - Y_{h/2}}{2^p - 1},$$

which for RK4 ($p = 4$) is $(Y_h - Y_{h/2})/15$. Removing this estimated error
gives the *extrapolated* value $Y_{h/2} - (Y_h - Y_{h/2})/15 = (16 Y_{h/2} -
Y_h)/15$. The next cell tests both statements on problem A for $N = 2$ to 128
(finer run $2N$ steps). The rule assumes that the error is already close to
$C h^p$; the test shows from which $N$ on this is true.
```

In [14]:

```
N_R = [2 ** k for k in range(1, 8)]  # the coarse runs: 2 ... 128 steps
coarse = np.array([solve(rk4_step, decay, 1.0, 1.0, n)[1][-1] for n in N_R])
fine = np.array([solve(rk4_step, decay, 1.0, 1.0, 2 * n)[1][-1] for n in N_R])
estimated = (coarse - fine) / 15.0  # Richardson's estimate of the error of fine
actual = fine - EXACT_A  # the true error of fine (we know the exact answer here)
extrapolated = (16.0 * fine - coarse) / 15.0
for n, e_est, e_act, e_ext in zip(N_R, estimated, actual, extrapolated - EXACT_A):
    say(f"N = {n:3d} -> {2 * n:3d}: estimated {e_est: .3e}, actual {e_act: .3e}, "
        f"extrapolated error {e_ext: .2e}")
fig, ax = plt.subplots(figsize=(6.0, 5.4))
ax.loglog(np.abs(actual), np.abs(estimated), "o", color=COLORS["rk4"], ms=6,
          label="RK4 runs with $2N = 4$ to $256$ steps")
for n, x_value, y_value in zip(N_R, np.abs(actual), np.abs(estimated)):
    ax.annotate(f"$N = {n}$", (x_value, y_value), textcoords="offset points",
                xytext=(6, -10), fontsize=7)
diagonal = np.array([1e-14, 1e-4])
ax.loglog(diagonal, diagonal, "--", color=COLORS["guide"], lw=0.9,
          label="estimate = actual")
ax.set_xlabel("actual error of the finer run")
ax.set_ylabel("Richardson estimate $(Y_h - Y_{h/2})/15$")
ax.set_title("Richardson's estimate against the true error")
ax.legend(loc="upper left")
save_figure(fig, "richardson",
            "Richardson's error estimate $(Y_h - Y_{h/2})/15$ of RK4 for problem A "
            "at $t = 1$ (vertical axis) against the true error of the finer run "
            "(horizontal axis), both pure numbers on logarithmic axes; each point "
```

```
            "is labelled with the number of steps $N$ of the coarse run, the finer "
            "run has $2N$. The points lie on the dashed diagonal: the estimate, "
            "made without knowing the exact answer, is the error itself to within "
            "10 percent once the coarse run has $N \\geq 8$ steps.")
ratio = estimated[2:] / actual[2:]
check(np.all((ratio > 0.9) & (ratio < 1.1)),
      "Richardson's estimate is within 10 % of the actual error for N >= 8")
check(np.all(np.abs(extrapolated[2:6] - EXACT_A) < np.abs(actual[2:6]) / 10),
      "the extrapolated value is more than 10 times more accurate (N = 8 to 64)")
```

Out [14]:

```
N =   2 ->   4: estimated  1.844e-05, actual  1.476e-05, extrapolated error -3.68e-06
N =   4 ->   8: estimated  9.285e-07, actual  8.308e-07, extrapolated error -9.77e-08
N =   8 ->  16: estimated  5.210e-08, actual  4.928e-08, extrapolated error -2.82e-09
N =  16 ->  32: estimated  3.085e-09, actual  3.001e-09, extrapolated error -8.45e-11
N =  32 ->  64: estimated  1.877e-10, actual  1.851e-10, extrapolated error -2.59e-12
N =  64 -> 128: estimated  1.158e-11, actual  1.149e-11, extrapolated error -8.02e-14
N = 128 -> 256: estimated  7.186e-13, actual  7.159e-13, extrapolated error -2.72e-15
```

Out [14]:

```
Figure 02a.8 saved as Revision/textbook/figures/02a_8_richardson.png
PASS Richardson's estimate is within 10 % of the actual error for N >= 8
PASS the extrapolated value is more than 10 times more accurate (N = 8 to 64)
```

Text cell:

```
## 15. The last check

The last cell checks that all 8 figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [15]:

```
names = ["scale_factors", "scale_factor_product", "convergence_loglog",
         "error_ratios", "rounding_floor", "phase_portrait", "energy_drift",
         "richardson"]
present = [output_file(f"{FIGURE_FOLDER}/02a_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02a exist")
all_checks_passed()
```

Out [15]:

```
PASS all 8 figure files of notebook 02a exist
ALL 29 CHECKS PASSED (notebook 02a)
```

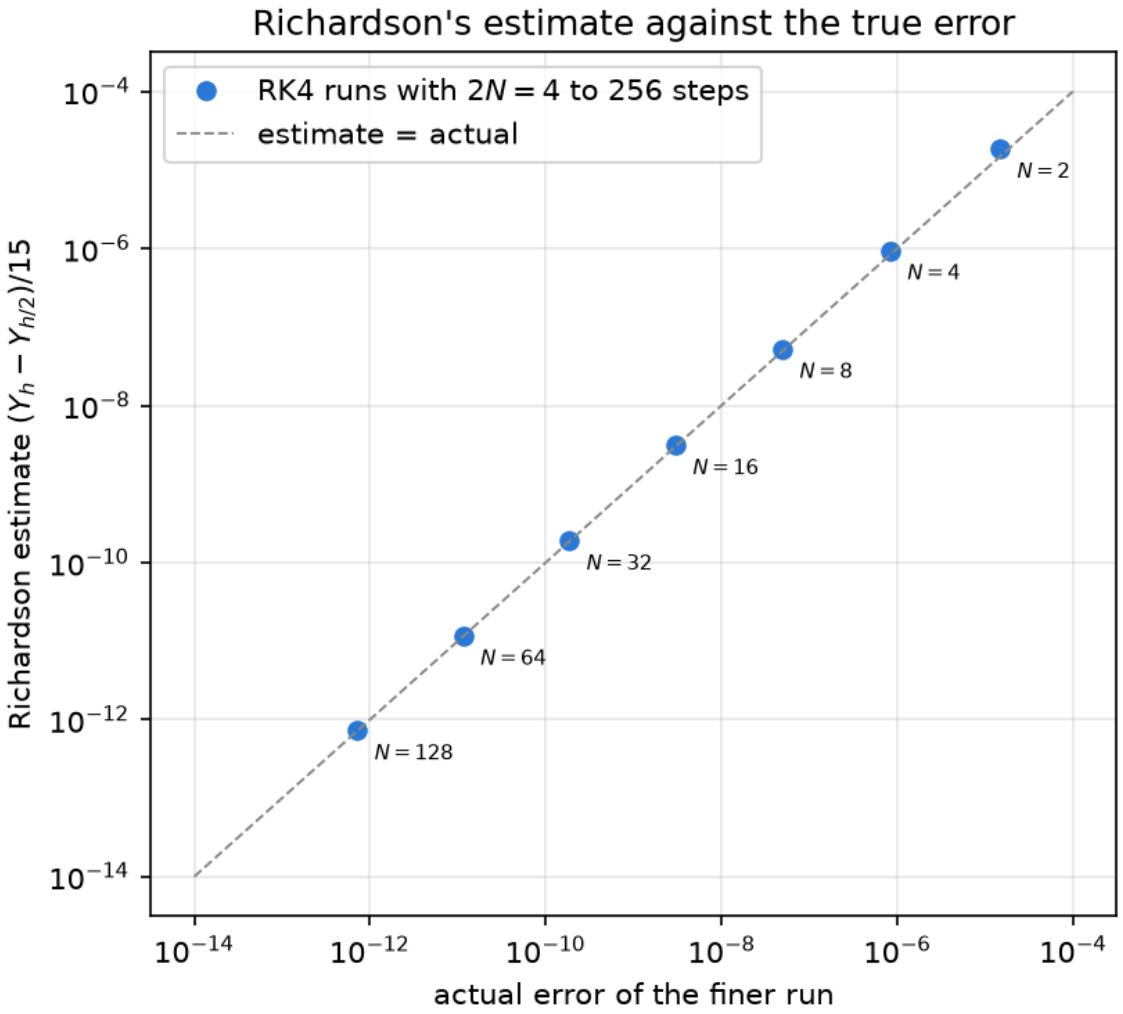

Figure 69: Richardson's error estimate $(Y_h - Y_{h/2})/15$ of RK4 for problem A at $t = 1$ (vertical axis) against the true error of the finer run (horizontal axis), both pure numbers on logarithmic axes; each point is labelled with the number of steps N of the coarse run, the finer run has $2N$. The points lie on the dashed diagonal: the estimate, made without knowing the exact answer, is the error itself to within 10 percent once the coarse run has $N \geq 8$ steps. (Notebook 02a, figure 8.)

Text cell:

```
## 16. What this notebook showed

- One step of size $h$ multiplies the solution of $y' = \lambda y$ by an
  amplification factor $R(\lambda h)$: $1 + z$ (Euler), $1 + z + z^2/2$
  (midpoint) and $1 + z + z^2/2 + z^3/6 + z^4/24$ (RK4), the first 2, 3 and 5
  terms of the series of $e^{z}$.
- Measured on a log-log plot, the error at a fixed end time is $C h^p$ with the
  orders $p = 1, 2, 4$; halving the step divides the error by 2, 4 and 16.
- RK4 reaches an error of about $10^{-16}$ with a few thousand steps; smaller
  steps make the result worse, because rounding errors add up, though they
  partly cancel and stay far below the pessimistic bound $N \epsilon$.
- The first term of the series of $e^{z}$ that a method misses predicts its
  error: for $y' = -y$ at $t = 1$ the error is $e^{-1}(-1)^p h^p/(p+1)!$. At the
  step $h = 1/300$ of the Revision Kohn-Sham solver (900 RK4 steps on an
  interval of length 3, read from its record) that is about $-6 \times 10^{-4}$
  for Euler, $7 \times 10^{-7}$ for the midpoint method and
  $4 \times 10^{-13}$ for RK4.
- The product of the inflating 3-space factor $e^{x_4}$ and the deflating
  extra-time factor $e^{-x_4}$ is exactly 1; Euler's method destroys this
  compensation (factor $1 - h^2$ per step), RK4 keeps it to $3 \times 10^{-5}$
  even with the coarse step $h = 0.25$ (factor $1 + h^6/72 + h^8/576$).
- For the oscillator the energy factors per step are exactly $1 + h^2$,
  $1 + h^4/4$ and $1 - h^6/72 + h^8/576$; the computed energies follow these
  predictions to 13 digits. The angle of $R(ih)$ predicts how far each method
  runs ahead of the exact solution (midpoint) or lags behind it (Euler, RK4).
- Richardson's rule $(Y_h - Y_{h/2})/(2^p - 1)$ estimates the error of the finer
  run $Y_{h/2}$ without the exact answer, and $(16 Y_{h/2} - Y_h)/15$ is a better
  value than either run.
```

2.11 Line-by-line walk-through of Notebook 02a

The notebook has fifteen code cells, In [1] to In [15]. This section explains every line of every one of them, in order. A line that starts with `#` is a **comment**: Python skips it; it is there for the reader. In the quoted code a line `...)` stands for the remaining lines of a long figure caption; every caption is printed in full under its figure in Section 2.10, and the paragraph "What Figure 02a.k shows" after the code says what to look for.

In [1], the set-up cell. Its first part is the complete run instructions of Section 2.9 again, as comment lines, so that the notebook file carries its own instructions. The code starts below the line of `=` signs that reads THE SET-UP. This part is the same in every notebook of the book except for one line (the notebook's name); we explain it here once, and the walk-throughs of Notebooks 02b, 02c and 02d refer back to this explanation.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it.

`json` reads and writes the text format JSON, in which the Revision record stores its results; `os` gives access to the computer's environment; `textwrap` breaks long text into lines; `from pathlib import Path` takes the single name `Path` out of the module `pathlib`. A `Path` is the address of a file or a folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell.

```
NOTEBOOK_ID = "02a"  # this notebook: chapter 02, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"02a"`. It is the only line of the set-up code that differs between notebooks.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. The text in triple quotes under the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd()` is the folder in which the notebook runs and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with all of them. The `for` loop takes these folders one after the other; the operator `/` joins a folder and a name into a longer path; `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository and `return` hands it back. If none does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

REPO is the repository folder. os.environ holds the **environment variables** of the running program (named texts that the computer hands to it); .get(name, default) returns the value of TEXTBOOK_OUTPUT_ROOT if it is set and otherwise the default, the repository folder written as a string. When you run the notebook the variable is not set, so the notebook writes into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

repository_file("Revision/...") gives the full path of a repository file, for reading. output_file("Revision/...") gives the full path at which to write a file; path.parent.mkdir(parents=True, exist_ok=True) first creates the folder that will hold it (and any missing folder above it) and does nothing if it exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

say prints a text in lines of at most 89 characters; textwrap.fill breaks it at blanks, and every line after the first starts with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

matplotlib.rcdefaults() returns to matplotlib's built-in settings, so that a personal settings file cannot change the figures. plt.rcParams.update({...}) then sets, for every figure of the notebook, the size (7.0 by 4.2 inches), the size of the letters (10 points) and a faint grid (30 per cent opaque). The braces make a **dictionary**: pairs key: value.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with f is an **f-string**: every name in braces is replaced by its value, so CAPTION_FILE is "Revision/textbook/figures/02a.captions.json". FIGURE_NUMBERS and CAPTIONS

start as empty dictionaries. The last line writes an empty dictionary and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far; so the figures are numbered 1, 2, 3, ..., and a cell that is run twice keeps its numbers. The file name joins the notebook id, the number and the name, for example `02a_1_scale_factors.png`.

```
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`fig.savefig` writes the PNG file with 150 dots per inch, without an empty margin and without the program's name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory. The caption is stored, and the whole dictionary of captions is written into the captions file (`json.dumps` turns it into JSON text with sorted keys). `display(Image(...))` shows the saved picture below the cell; its `metadata` tells the book's tools which file it is. The last line prints where the figure was saved.

```
PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
```

```
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection in square brackets). `check` is the function behind every check of the book. `condition` is either `True` or `False`; if it is false, `raise AssertionError(...)` stops the notebook with an error that names the check. Otherwise the name is appended to `PASSED` and the line PASS followed by the name is printed. `record=None` makes the third argument optional; a check that reproduces a Revision record passes the record's file and check name, and a second line names them.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a RESULT line; the expression in round brackets adds a blank and the unit when a unit is given and nothing otherwise. `all_checks_passed` prints the last line of the notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one output line of In [1].

In [2], the step rules and the solver.

```
import math  # exp, log10 and other functions of one number
from fractions import Fraction  # exact fractions such as 233/384

import numpy as np  # arrays (vectors) of numbers
import sympy as sp  # exact algebra with symbols
```

`math` holds the functions of single numbers (`math.exp`, `math.log2`, `math.factorial`). `Fraction` stores a fraction exactly as a numerator and a denominator, with no rounding. numpy (short name `np`) works with **arrays**, lists of numbers on which arithmetic acts entry by entry. sympy (short name `sp`) does exact algebra with symbols.

```
# One colour per method (a set that colour-blind readers can tell apart); every
# method also has its own line style and marker, so the plots read in grey too.
COLORS = {"euler": "#eb6834", "midpoint": "#1baf7a", "rk4": "#2a78d6",
          "exact": "#000000", "guide": "#8a8986"}
MARKERS = {"euler": "s", "midpoint": "^", "rk4": "o"}
LABELS = {"euler": "Euler", "midpoint": "midpoint", "rk4": "RK4"}
```

Three dictionaries fix, for each method, a colour (written as a hexadecimal code of red, green and blue: orange for Euler, green for the midpoint method, blue for RK4, black for exact curves, grey

for guide lines), a marker (`"s"` a square, `"^"` a triangle, `"o"` a circle) and the name printed in the legends.

```
def euler_step(f, t, y, h):
    """One Euler step: follow the slope at the start for the whole step."""
    return y + h * f(t, y)
```

This is Euler's rule $y_{n+1} = y_n + hf(t_n, y_n)$ of Section 2.4. The argument `f` is itself a function, the right-hand side; `f(t, y)` is its value, the slope.

```
def midpoint_step(f, t, y, h):
    """One midpoint step: use the slope at the estimated middle of the step."""
    k1 = f(t, y)  # the slope at the start
    k2 = f(t + h / 2, y + (h / 2) * k1)  # the slope at the estimated midpoint
    return y + h * k2
```

The midpoint rule, line for line as in Section 2.4.

```
def rk4_step(f, t, y, h):
    """One step of the classical fourth-order Runge-Kutta method (RK4)."""
    k1 = f(t, y)  # the slope at the start
    k2 = f(t + h / 2, y + (h / 2) * k1)  # at the midpoint, reached with k1
    k3 = f(t + h / 2, y + (h / 2) * k2)  # at the midpoint again, reached with k2
    k4 = f(t + h, y + h * k3)  # at the end, reached with k3
    return y + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)  # weights 1 : 2 : 2 : 1
```

The RK4 rule of Section 2.4. Because these three functions use only `+`, `-`, `*` and `/`, they work unchanged with ordinary numbers, exact fractions, sympy symbols and numpy arrays (vectors); the notebook uses all four.

```
METHODS = {"euler": euler_step, "midpoint": midpoint_step, "rk4": rk4_step}
```

A dictionary from the name of each method to its step function, so that a loop can run all three.

```
def solve(step, f, y0, t_end, n):
    """n equal steps of the rule step from t = 0 to t = t_end.  Returns the list
    of the n + 1 times and the list of the n + 1 computed values."""
    h = t_end / n  # the step size
    times, values = [0 * h], [y0]
    y = y0
    for i in range(n):
        y = step(f, i * h, y, h)  # t_i = i h (no rounding drift of the time)
        times.append((i + 1) * h)
        values.append(y)
    return times, values
```

`solve` makes `n` steps of size $h = T/n$. The two lists start with the time 0 (written `0 * h`, so that it is a fraction when `h` is one) and the start value. `range(n)` counts $i = 0, 1, \ldots, n-1$; each pass makes one step from the time $t_i = ih$ and appends the new time and value. The time is computed as `i * h` rather than by adding `h` again and again, so that no rounding error piles up in the time.

```
def decay(t, y):
    """Problem A: y' = -y (an extra-time scale factor along the linear history)."""
    return -y


def growth(t, y):
    """y' = +y (the 3-space scale factor along the same history)."""
    return y


say("Three step rules and the solver are defined.")
```

The two right-hand sides of Section 2.2: `decay` for the deflating extra-time factor b and `growth` for the inflating 3-space factor a. The cell prints one line.

In [3], one step by hand.

```
h = Fraction(1, 2)  # the step size 1/2 as an exact fraction
one_step = {name: step(decay, 0, Fraction(1), h) for name, step in METHODS.items()}
```

`Fraction(1, 2)` is the exact fraction 1/2. The second line is a **dictionary comprehension**: for every pair (name, step function) of `METHODS.items()` it makes one step of Problem A from $y = 1$ and stores the result under the method's name. Because all numbers are fractions, the results are exact.

```
for name, value in one_step.items():
    say(f"{LABELS[name]:9} after one step: {str(value):8} = {float(value):.10f}")
say(f"exact     e^(-1/2)                 = {math.exp(-0.5):.10f}")
```

The loop prints each result as a fraction and as a decimal number; `{...:9}` pads the name to 9 characters so that the columns line up, and `{...:.10f}` writes 10 digits after the decimal point. The last line prints $e^{-1/2} = 0.6065306597$.

```
check(one_step["euler"] == Fraction(1, 2), "one Euler step of y' = -y gives 1/2")
check(one_step["midpoint"] == Fraction(5, 8), "one midpoint step gives 5/8")
check(one_step["rk4"] == Fraction(233, 384), "one RK4 step gives 233/384")
```

Three checks compare the exact results with the hand calculation of Section 2.4: 1/2, 5/8 and 233/384. The output shows 0.5, 0.625 and 0.6067708333 against the exact 0.6065306597.

In [4], the amplification factors.

```
z = sp.symbols("z")  # z = lambda h
hs = sp.symbols("h", positive=True)  # a positive step size, as a symbol
```

`sp.symbols` makes sympy symbols: letters that stand for any number. `z` is λh; `hs` is a step size h, declared positive.

```
def linear(t, y):
    """The test equation y' = z y."""
    return z * y


one = sp.Integer(1)  # sympy's exact 1 (so that 1/2 stays the exact fraction 1/2)
R = {name: sp.expand(step(linear, 0, one, one)) for name, step in METHODS.items()}
```

`linear` is the right-hand side of the test equation with $\lambda h = z$. With the step $h = 1$ (sympy's exact integer 1, so that `h / 2` is the exact fraction $1/2$ and not the rounded number 0.5) and the start value 1, one step of each method gives exactly $R(z)$; `sp.expand` multiplies out all brackets. This is the derivation of Section 2.5, done by the computer.

```
taylor = sp.series(sp.exp(z), z, 0, 6).removeO()  # 1 + z + ... + z^5/120
terms = sp.Poly(taylor, z).all_coeffs()[::-1]  # the coefficients 1, 1, 1/2, ...
```

`sp.series(sp.exp(z), z, 0, 6)` is the series of e^z around $z = 0$ up to the power z^5, followed by a symbol for the left-out terms, which `.removeO()` drops. `sp.Poly(...).all_coeffs()` lists the coefficients from the highest power down, and `[::-1]` reverses the list, so that `terms[k]` is the coefficient $1/k!$ of z^k.

```
for name, keep in (("euler", 2), ("midpoint", 3), ("rk4", 5)):
    partial = sum(terms[k] * z ** k for k in range(keep))  # the first keep terms
    say(f"R_{name}(z) = {sp.expand(R[name])}")
    check(sp.expand(R[name] - partial) == 0,
          f"R_{name}(z) equals the first {keep} terms of the series of e^z")
```

For each method, `partial` is the sum of the first `keep` terms of the series (z^k is written `z ** k` in Python). The cell prints $R(z)$ and checks that $R(z)$ minus that partial sum is exactly zero: Euler keeps 2 terms, the midpoint method 3, RK4 5. The output reads `z + 1`, `z**2/2 + z + 1` and `z**4/24 + z**3/6 + z**2/2 + z + 1`.

```
# The energy factor of one oscillator step, R(i h) R(-i h), as a polynomial in h.
ENERGY_FACTOR = {name: sp.expand(R[name].subs(z, sp.I * hs) *
                                 R[name].subs(z, -sp.I * hs))
                 for name in METHODS}
for name, factor in ENERGY_FACTOR.items():
    say(f"energy factor per step, {LABELS[name]:8}: {factor}")
check(ENERGY_FACTOR["euler"] == 1 + hs ** 2
      and ENERGY_FACTOR["midpoint"] == 1 + hs ** 4 / 4
      and ENERGY_FACTOR["rk4"] == 1 - hs ** 6 / 72 + hs ** 8 / 576,
      "energy factors 1 + h^2, 1 + h^4/4, 1 - h^6/72 + h^8/576")
```

`.subs(z, sp.I * hs)` replaces z by ih (`sp.I` is sympy's i). The product $R(ih)R(-ih)$, multiplied out, is the factor by which one step multiplies the oscillator's energy (Section 2.7). The check compares it with $1 + h^2$, $1 + h^4/4$ and $1 - h^6/72 + h^8/576$, the results of Section 2.7; the output prints the three polynomials.

In [5], the inflating and the deflating scale factor.

```
H_COARSE, STEPS = 0.25, 8  # step size and number of steps: t from 0 to 2
curves = {}
for name in ("euler", "rk4"):
    t_list, a_list = solve(METHODS[name], growth, 1.0, 2.0, STEPS)
    _, b_list = solve(METHODS[name], decay, 1.0, 2.0, STEPS)
    curves[name] = (np.array(t_list), np.array(a_list), np.array(b_list))
t_fine = np.linspace(0.0, 2.0, 201)  # many points for the exact curves
```

Two names are given two values at once: the step 0.25 and the number of steps 8 (so t runs from 0 to 2). For Euler and RK4 the loop solves $a' = a$ (`growth`) and $b' = -b$ (`decay`) from the value 1; `_` is the conventional name for a result that is not needed (the times, already known). The three lists are turned into numpy arrays and stored in `curves`. `np.linspace(0.0, 2.0, 201)` makes 201 equally spaced times for drawing the exact curves smoothly.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.plot(t_fine, np.exp(t_fine), color=COLORS["exact"], lw=1.0,
          label="exact $e^{t}$")
right.plot(t_fine, np.exp(-t_fine), color=COLORS["exact"], lw=1.0,
           label="exact $e^{-t}$")
```

`plt.subplots(1, 2, ...)` makes a figure with one row of two panels, named `left` and `right`. `.plot(xs, ys, ...)` draws a line through the points; `lw` is the line width and `label` the name in the legend (text between dollar signs is typeset as mathematics). The exact curves e^t and e^{-t} are drawn in black.

```
for name in ("euler", "rk4"):
    t_n, a_n, b_n = curves[name]
    style = dict(color=COLORS[name], marker=MARKERS[name], ms=5, lw=1.2)
    left.plot(t_n, a_n, **style, label=f"{LABELS[name]}, $h = 0.25$")
    right.plot(t_n, b_n, **style, label=f"{LABELS[name]}, $h = 0.25$")
```

For each method the three arrays are unpacked; `style` is a dictionary of drawing options (colour, marker, marker size `ms`, line width), and `**style` passes its entries as named arguments. The computed points of a go into the left panel, those of b into the right panel.

```
left.set_title("3-space factor $a$: $da/dx_4 = +a$ (inflates)")
right.set_title("extra-time factor $b$: $db/dx_4 = -b$ (deflates)")
for ax in (left, right):
    ax.set_xlabel("time $x_4$ (units $1/H$, $A = 1$)")
    ax.legend(fontsize=8)
left.set_ylabel("scale factor (pure number)")
save_figure(fig, "scale_factors",
            "Left: the 3-space scale factor $a = e^{x_4}$, which obeys "
            ...)
```

Titles, axis labels and legends; then `save_figure` saves the figure as `02a_1_scale_factors.png` with its caption (printed in full under Figure 02a.1 in Section 2.10; Python joins strings written next to each other into one). **What Figure 02a.1 shows:** in the left panel the Euler squares fall more and more below the growing exact curve, in the right panel they decay faster than the exact curve; the RK4 circles lie on both exact curves even with the coarse step 0.25.

```
a_euler, b_euler = curves["euler"][1][-1], curves["euler"][2][-1]
a_rk4, b_rk4 = curves["rk4"][1][-1], curves["rk4"][2][-1]
report("a(2): exact, Euler, RK4", f"{math.exp(2):.6f}, {a_euler:.6f}, {a_rk4:.6f}")
report("b(2): exact, Euler, RK4", f"{math.exp(-2):.6f}, {b_euler:.6f}, {b_rk4:.6f}")
```

`curves["euler"][1]` is the array of the Euler values of a, and `[-1]` its last entry (the value at $t = 2$). The two RESULT lines print $a(2)$: exact 7.389056, Euler 5.960464, RK4 7.388665, and $b(2)$: exact 0.135335, Euler 0.100113, RK4 0.135346.

```
check(abs(a_euler - 1.25 ** 8) < 1e-12 and abs(b_euler - 0.75 ** 8) < 1e-15,
      "Euler gives exactly 1.25^8 and 0.75^8 after 8 steps")
check(abs(a_rk4 / math.exp(2) - 1) < 1e-3 and abs(b_rk4 / math.exp(-2) - 1) < 1e-3,
      "RK4 with h = 0.25 is within 0.1 percent of e^2 and e^(-2)")
```

Euler multiplies by $R(0.25) = 1.25$ and $R(-0.25) = 0.75$ per step, so after 8 steps it must give $1.25^8 = 5.960464\ldots$ and $0.75^8 = 0.100113\ldots$; `abs` is the absolute value and `1e-12` means 10^{-12} (the allowed rounding). The second check confirms that RK4 is within 10^{-3}, that is 0.1 per cent, of the exact values.

In [6], the product of the two scale factors.

```
PRODUCT_FACTOR = {name: sp.expand(R[name].subs(z, hs) * R[name].subs(z, -hs))
                  for name in ("euler", "rk4")}
for name, factor in PRODUCT_FACTOR.items():
    say(f"factor of a b per step, {LABELS[name]:5}: {factor}")
check(PRODUCT_FACTOR["euler"] == 1 - hs ** 2
      and PRODUCT_FACTOR["rk4"] == 1 + hs ** 6 / 72 + hs ** 8 / 576,
      "R(h) R(-h) is 1 - h^2 (Euler) and 1 + h^6/72 + h^8/576 (RK4)")
```

sympy multiplies out $R(h)R(-h)$, the factor by which one step multiplies the product ab (Section 2.7), prints it and checks the two results of Section 2.7.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.axhline(1.0, color=COLORS["exact"], lw=1.0, label="exact $a b = 1$")
products = {}
```

A new figure with two panels; `axhline(1.0, ...)` draws a horizontal line at height 1, the exact product. `products` will hold the computed products.

```
for name in ("euler", "rk4"):
    t_n, a_n, b_n = curves[name]
    products[name] = a_n * b_n  # the computed product at every step
    factor = float(PRODUCT_FACTOR[name].subs(hs, sp.Rational(1, 4)))  # h = 1/4
    style = dict(color=COLORS[name], marker=MARKERS[name], ms=5, lw=1.2)
    left.plot(t_n, products[name], **style, label=f"{LABELS[name]}, $h = 0.25$")
    right.semilogy(t_n[1:], np.abs(products[name][1:] - 1.0), **style,
                   label=f"{LABELS[name]}: measured")
    predicted = np.abs(factor ** np.arange(1, STEPS + 1) - 1.0)
    right.semilogy(t_n[1:], predicted, ":", color=COLORS["exact"], lw=1.0)
```

For each method `a_n * b_n` multiplies the two arrays entry by entry. `factor` is the per-step factor at $h = 1/4$ (sympy's exact `Rational(1, 4)`), turned into an ordinary number by `float`. The left panel shows the product; the right panel shows its distance from 1, $|ab - 1|$, with `semilogy`, which draws the vertical axis logarithmically. `[1:]` leaves out the starting point, where the distance is exactly 0 and cannot be drawn on a logarithmic axis. `np.arange(1, STEPS + 1)` is the array $1, 2, \ldots, 8$, so `factor ** np.arange(...)` is the predicted product after n steps; the dotted black line (`":"`) draws its distance from 1.

```
right.plot([], [], ":", color=COLORS["exact"], label="predicted $|R(h)^n R(-h)^n - 1|$")
left.set_ylabel("product $a b$ (pure number)")
right.set_ylabel("$|a b - 1|$ (pure number)")
left.set_title("the product $a b$")
right.set_title("its distance from 1 (logarithmic axis)")
for ax in (left, right):
    ax.set_xlabel("time $x_4$ (units $1/H$)")
    ax.legend(fontsize=8)
save_figure(fig, "scale_factor_product",
            "Left: the product $a b$ of the 3-space factor and the extra-time "
            ...)
```

`right.plot([], [], ...)` draws nothing (two empty lists) but adds an entry for the dotted prediction to the legend. Then labels, titles, legends and the saved figure `02a_2_scale_factor_product.png`. **What Figure 02a.2 shows:** the Euler product falls step by step to $0.9375^8 = 0.597$; the RK4 product stays at 1 to within 3×10^{-5}; on the logarithmic right panel every measured point lies on its dotted prediction.

```
check(abs(products["euler"][-1] - (1 - H_COARSE ** 2) ** 8) < 1e-14,
      "Euler multiplies the product a b by 1 - h^2 per step")
rk4_factor = 1 + H_COARSE ** 6 / 72 + H_COARSE ** 8 / 576
check(abs(products["rk4"][-1] - rk4_factor ** 8) < 1e-14
      and abs(products["rk4"][-1] - 1) < 3e-5,
      "RK4 multiplies a b by 1 + h^6/72 + h^8/576 per step (within 3e-5 of 1)")
```

The final products must equal $(1 - h^2)^8$ (Euler) and $(1 + h^6/72 + h^8/576)^8$ (RK4) up to rounding, and the RK4 product must be within 3×10^{-5} of 1.

In [7], the convergence study.

```
EXACT_A = math.exp(-1.0)  # the exact value y(1) = e^(-1)
N_LIST = [2 ** k for k in range(1, 13)]  # 2, 4, ..., 4096 steps
ends = {name: [solve(step, decay, 1.0, 1.0, n)[1][-1] for n in N_LIST]
        for name, step in METHODS.items()}
errors = {name: np.abs(np.array(values) - EXACT_A) for name, values in ends.items()}
```

`EXACT_A` is $e^{-1} = 0.367879\ldots$. `N_LIST` is the list $2^1, 2^2, \ldots, 2^{12}$ (`range(1, 13)` counts 1 to 12). For each method and each N, `solve(...)[1][-1]` is the computed value at $t = 1$; `ends` stores the twelve end values of each method and `errors` their distances from e^{-1}.

```
say("     N        h     Euler y_N   error Euler  error midpoint     error RK4")
for i, n in enumerate(N_LIST):
```

```
    y_euler = ends["euler"][i]  # the Euler value y_N
    e_euler, e_mid, e_rk4 = (errors[name][i] for name in METHODS)  # three errors
    say(f"{n:6d} {1 / n:8.6f}  {y_euler:12.6f}  {e_euler:11.3e}  {e_mid:14.3e}  "
        f"{e_rk4:12.3e}")
```

`enumerate(N_LIST)` gives the position `i` together with each `n`. Each row prints N, $h = 1/N$, the Euler value and the three errors; `:6d` writes a whole number in 6 places, `:11.3e` a number in scientific notation with 3 digits after the point (for example `1.179e-01` means 1.179×10^{-1}). The table shows the Euler error halving and the midpoint error quartering from row to row, while the RK4 error falls 16-fold until it reaches about 10^{-16} at $N = 2048$.

```
h_array = 1.0 / np.array(N_LIST, dtype=float)
FIT = {"euler": slice(5, 12), "midpoint": slice(5, 12), "rk4": slice(1, 8)}
slopes = {}
for name, part in FIT.items():
    # polyfit(x, y, 1) returns (slope, intercept) of the best straight line.
    slope, _ = np.polyfit(np.log10(h_array[part]), np.log10(errors[name][part]), 1)
    slopes[name] = slope
    report(f"measured order of {LABELS[name]} (slope)", f"{slope:.4f}")
```

`h_array` holds the twelve step sizes. `FIT` chooses, for each method, the rows used for the straight-line fit: `slice(5, 12)` is the positions 5 to 11 ($N = 64$ to 4096), where the Euler and midpoint errors are already close to Ch^p; `slice(1, 8)` is the positions 1 to 7 ($N = 4$ to 256), where the RK4 error is not yet touched by rounding. `np.polyfit(x, y, 1)` fits the least-squares straight line through the points $(\log_{10} h, \log_{10} e)$ and returns its slope and its intercept. The measured orders are 1.0014, 2.0025 and 4.0439 (COMPUTED).

```
check([round(v, 6) for v in ends["euler"][:4]]
      == [0.25, 0.316406, 0.343609, 0.356074],
      "Euler values (1 - h)^N for h = 1/2, 1/4, 1/8, 1/16")
check(abs(slopes["euler"] - 1) < 0.02 and abs(slopes["midpoint"] - 2) < 0.02
      and abs(slopes["rk4"] - 4) < 0.1,
      "the measured orders are 1, 2 and 4")
```

The first check compares the first four Euler values, rounded to 6 digits, with $(1-h)^N$: $(1/2)^2 = 0.25$, $(3/4)^4 = 0.316406$, $(7/8)^8 = 0.343609$, $(15/16)^{16} = 0.356074$. The second requires the slopes to be within 0.02 of 1 and 2 and within 0.1 of 4.

In [8], the log-log plot.

```
fig, ax = plt.subplots(figsize=(7.0, 5.0))
for name in METHODS:
    ax.loglog(h_array, errors[name], color=COLORS[name], marker=MARKERS[name],
              ms=5, lw=1.2, label=f"{LABELS[name]} (measured slope "
              f"{slopes[name]:.2f})")
```

`ax.loglog` draws with both axes logarithmic. Each method's twelve errors are drawn against the step sizes, with the measured slope in the legend.

```
    order = {"euler": 1, "midpoint": 2, "rk4": 4}[name]
    anchor = FIT[name].stop - 1  # the last fitted point
    guide = errors[name][anchor] * (h_array / h_array[anchor]) ** order
    ax.loglog(h_array, guide, "--", color=COLORS["guide"], lw=0.9)
    ax.text(h_array[0] * 1.15, guide[0], f"slope {order}", fontsize=8,
            color=COLORS["guide"], va="center")
```

Still inside the loop: `order` is the expected order. `FIT[name].stop - 1` is the last position of the fitted rows. The guide line $\Delta_{\text{anchor}}(h/h_{\text{anchor}})^p$ (with Δ_{anchor} the error at that point) passes through that point with the exact slope p; it is drawn dashed (the line-style string of two hyphens) in grey, and `ax.text` writes "slope p" next to its right end (`va="center"` centres the text vertically).

```
ax.set_xlim(h_array[-1] / 1.5, h_array[0] * 3.0)
ax.set_ylim(1e-17, 1.0)
ax.set_xlabel("step size $h$ (time units)")
ax.set_ylabel("error $|y_N - e^{-1}|$ at $t = 1$")
ax.set_title("Problem A: error at the end time against the step size")
ax.legend(loc="lower right")
save_figure(fig, "convergence_loglog",
            "The error at the end time $t = 1$ of problem A, $y' = -y$, for the "
            ...)
check(errors["rk4"][7] < 1e-11 < errors["midpoint"][7] < errors["euler"][7],
      "at N = 256: RK4 error < 1e-11 < midpoint error < Euler error")
```

The axis ranges leave room for the slope labels; then labels, title, legend and the saved figure `02a_3_convergence_loglog.png`. The check is a chain of comparisons at position 7 ($N = 256$): the RK4 error 7.2×10^{-13} is below 10^{-11}, which is below the midpoint error 9.4×10^{-7}, which is below the Euler error 7.2×10^{-4}. **What Figure 02a.3 shows:** three straight lines of points, parallel to the guides of slopes 1, 2 and 4: the orders can be read off by eye. The RK4 line bends away at the bottom left, where rounding takes over.

In [9], what halving the step does.

```
ratios = {name: errors[name][:-1] / errors[name][1:] for name in METHODS}
```

`errors[name][:-1]` is the array without its last entry and `errors[name][1:]` without its first, so the quotient is $e(h)/e(h/2)$ for every pair of neighbouring rows.

```
fig, ax = plt.subplots()
for name in METHODS:
    shown = slice(0, 8) if name == "rk4" else slice(0, 11)  # RK4: before rounding
    ax.semilogx(N_LIST[1:][shown], ratios[name][shown], color=COLORS[name],
                marker=MARKERS[name], ms=5, lw=1.2, label=LABELS[name])
```

Each ratio is drawn against the N of the finer run (`N_LIST[1:]`), with a logarithmic horizontal axis (`semilogx`); for RK4 only the first 8 ratios (up to $N = 512$) are drawn, because later its errors are rounding noise.

```
for target in (2, 4, 16):
    # a dashed line from N = 3 to just beyond the data, its label to the right
    ax.hlines(target, 3.0, N_LIST[-1] * 1.4, ls="--", color=COLORS["guide"], lw=0.9)
    ax.text(N_LIST[-1] * 1.6, target, f"$2^{{{int(math.log2(target))}}}$ = "
            f"{target}", va="center", ha="left", fontsize=8, color=COLORS["guide"])
ax.set_xlim(3.0, N_LIST[-1] * 8.0)  # room on the right for the labels
```

Dashed horizontal guide lines at $2 = 2^1$, $4 = 2^2$ and $16 = 2^4$, each labelled; `math.log2(target)` is the power of 2, and in an f-string a doubled brace prints one brace, so the label reads $2^1 = 2$, and so on.

```
ax.set_xlabel("number of steps $N$ of the finer run ($h = 1/N$)")
ax.set_ylabel("error ratio $e(2h)/e(h)$")
ax.set_title("Halving the step divides the error by $2^p$")
ax.legend(loc="center left")
save_figure(fig, "error_ratios",
            "The factor by which the error of problem A at $t = 1$ shrinks when the "
            ...)
```

Labels, title, legend and the saved figure `02a_4_error_ratios.png`. **What Figure 02a.4 shows:** the three curves settle on the guide lines 2, 4 and 16, which is what order 1, 2 and 4 means.

```
ratio_euler = ratios["euler"][-1]  # the last ratio: N = 2048 -> 4096
ratio_midpoint = ratios["midpoint"][-1]
ratio_rk4 = ratios["rk4"][5]  # N = 64 -> 128, before rounding matters
report("ratio Euler at N = 4096", f"{ratio_euler:.5f}")
report("ratio midpoint at N = 4096", f"{ratio_midpoint:.5f}")
report("ratio RK4 at N = 128", f"{ratio_rk4:.4f}")
check(abs(ratio_euler - 2) < 0.001 and abs(ratio_midpoint - 4) < 0.001
      and abs(ratio_rk4 - 16) < 0.2,
      "halving h divides the errors by 2, 4 and 16")
```

The cell picks the last ratios of Euler and of the midpoint method and the RK4 ratio from $N = 64$ to 128, prints them (2.00020, 4.00073 and 16.1047; COMPUTED) and checks them against 2, 4 and 16.

In [10], where rounding takes over.

```
N_LONG = [2 ** k for k in range(1, 19)]  # 2 ... 262144 steps
rk4_long = np.array([abs(solve(rk4_step, decay, 1.0, 1.0, n)[1][-1] - EXACT_A)
                     for n in N_LONG])
h_long = 1.0 / np.array(N_LONG, dtype=float)
best = int(np.argmin(rk4_long))  # the position of the smallest error
EPS = 2.0 ** -52  # the machine epsilon
```

RK4 is run with $N = 2, 4, \ldots, 2^{18} = 262144$ steps and the eighteen errors at $t = 1$ are stored. `np.argmin` is the position of the smallest error. `EPS` is the machine epsilon 2^{-52} of Section 2.6.

```
fig, ax = plt.subplots()
shown = rk4_long > 0  # an error of exactly 0 cannot be drawn on a log axis
```

```
ax.loglog(h_long[shown], rk4_long[shown], color=COLORS["rk4"], marker="o", ms=5,
          lw=1.2, label="RK4: measured error")
ax.loglog(h_long, errors["rk4"][3] * (h_long / h_array[3]) ** 4, "--",
          color=COLORS["guide"], lw=0.9, label="truncation $C h^4$")
ax.loglog(h_long, EPS * EXACT_A / h_long, ":", color=COLORS["exact"], lw=1.0,
          label="rounding bound $N \\epsilon\\, e^{-1}$")
```

`rk4_long > 0` is an array of `True` and `False`; used as an index (`h_long[shown]`) it keeps only the entries where it is `True`. The measured errors are drawn as circles; the dashed line is the truncation error Ch^4 through the point $N = 16$; the dotted line is $N\epsilon e^{-1} = \epsilon e^{-1}/h$, the pessimistic bound of Section 2.6: the size that N rounding errors of relative size ϵ would reach only if all of them had their largest size and the same sign (in a Python string `\\` stands for one backslash, which matplotlib's mathematics needs).

```
ax.set_ylim(1e-18, 1e-2)
ax.set_xlabel("step size $h = 1/N$ (time units)")
ax.set_ylabel("error $|y_N - e^{-1}|$")
ax.set_title("RK4: truncation error against rounding error")
ax.legend(loc="upper center")
save_figure(fig, "rounding_floor",
            "The error of RK4 for problem A at $t = 1$ against the step size $h$ "
            ...)
report("smallest RK4 error", f"{rk4_long[best]:.3e} at N = {N_LONG[best]}")
report("RK4 error at N = 262144", f"{rk4_long[-1]:.3e}")
```

Axis range, labels, legend, the saved figure `02a_5_rounding_floor.png`, and two RESULT lines: the smallest error is 1.110×10^{-16} at $N = 4096$, and the error with 262144 steps is 5.773×10^{-15}, fifty times larger (COMPUTED).

```
bound = EPS * EXACT_A * np.array(N_LONG, dtype=float)  # the dotted line
below = bound[best + 1:] / rk4_long[best + 1:]  # past the minimum: bound / error
report("bound N eps e^-1 / measured error past the minimum (smallest, largest)",
       f"{below.min():.0f}, {below.max():.0f}")
```

`bound` holds the value $N\epsilon e^{-1}$ of the dotted line for each of the eighteen runs. `bound[best + 1:]` keeps the entries after the position of the smallest error (the runs with more steps than $N = 4096$); dividing them by the measured errors of the same runs gives, for each run, how many times its error lies below the bound. The RESULT line prints the smallest and the largest of these ratios, 524 and 3709 (COMPUTED). **What Figure 02a.5 shows:** coming from the right (large steps), the error falls along the slope-4 line; near 10^{-16} it stops; for smaller steps it grows again, but slowly and 524 to 3709 times below the dotted bound. The rounding errors partly cancel, so the bound gives only the scale of the worst case, not the error itself.

```
check(rk4_long[best] < 1e-15 and 256 <= N_LONG[best] <= 16384,
      "the smallest RK4 error is below 1e-15, reached at N between 256 and 16384")
check(rk4_long[-1] > 10 * max(rk4_long[best], EPS * EXACT_A)
      and rk4_long[-1] < 1e-11,
      "with 262144 steps rounding has made the error grow again")
check(np.all(below > 100),
      "past the minimum the error stays over 100 times below the bound N eps e^-1")
```

Three checks: the floor is below 10^{-15} and is reached at a moderate N; with the most steps the error is more than ten times the floor (so rounding has made it grow), yet still below 10^{-11}; and past the minimum every error lies more than 100 times below the bound $N\epsilon e^{-1}$. The checks use ranges rather than exact values, because rounding errors can differ in the last bits from one computer to another.

In [11], the step of the Revision solver and the first missed term.

```
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
G_KS = parameters["numerics"]["rk4Steps"]  # the solver's number of RK4 steps
L_KS = parameters["physics"]["L_tipCutoff"]  # the length of its interval in y
A_KS = parameters["physics"]["historyA"]  # the constant A of a4 = A H x4
H_KS = parameters["physics"]["H"]  # the author's constant H
h_ks = L_KS / G_KS  # the solver's step
```

The cell reads the Revision record of the Kohn-Sham solver's parameters: `read_text` reads the file as text and `json.loads` turns the JSON text into nested dictionaries. `parameters["numerics"]["rk4Steps"]` is the entry `rk4Steps` inside the part `numerics`: the number $G = 900$ of RK4 steps. From the part `physics` come the length $L = 3$ of the interval in the hidden coordinate (the tip cut-off), the constant A of the history and the author's H. The solver's step is $h = L/G$.

```
report("solver: RK4 steps G, interval length L, step h = L/G",
       f"{G_KS}, {L_KS}, {h_ks:.10f}")
report("linear history a4 = A H x4 of the record: A, H", f"{A_KS}, {H_KS}")
check(G_KS == 900 and L_KS == 3.0 and A_KS == 1.0 and H_KS == 1.0,
      "the solver makes 900 RK4 steps on L = 3 (h = 1/300); the history has A = H = 1",
      record=f"{PARAMETERS}, keys rk4Steps, L_tipCutoff, historyA and H")
```

The two RESULT lines print 900, 3.0, 0.0033333333 and 1.0, 1.0; the check confirms the values and names the record and its keys in its second line (COMPUTED: reproduces `Revision/kohn_sham/results/parameters.json`, keys `rk4Steps`, `L_tipCutoff`, `historyA` and `H`).

```
N_KS = round(1.0 / h_ks)  # 300 steps of h = 1/300 reach t = 1
ORDER = {"euler": 1, "midpoint": 2, "rk4": 4}  # the orders measured in section 9
agreement = []
```

`round(1.0 / h_ks)` is the whole number 300: so many steps of the solver's size reach $t = 1$. `ORDER` holds the orders, and `agreement` will collect the ratios of measured to predicted errors.

```
for name, step in METHODS.items():
    p = ORDER[name]
    measured = solve(step, decay, 1.0, 1.0, N_KS)[1][-1] - EXACT_A  # signed error
    predicted = EXACT_A * (-1) ** p * h_ks ** p / math.factorial(p + 1)
    agreement.append(measured / predicted)
    say(f"{LABELS[name]:9} with h = 1/{N_KS}: error {measured:+.4e}, predicted "
        f"{predicted:+.4e}, ratio {measured / predicted:.5f}")
check(all(abs(ratio - 1) < 0.01 for ratio in agreement),
      "at the solver's step the errors are e^(-1) (-1)^p h^p/(p+1)! within 1 %")
```

For each method the signed error at $t = 1$ with 300 steps is measured (this time without `abs`, so that its sign is kept), and the prediction $e^{-1}(-1)^p h^p/(p+1)!$ of Section 2.6 is computed (`math.factorial(p + 1)` is $(p+1)!$, and `(-1) ** p` is $(-1)^p$). The quotient of the two is appended to `agreement`. The printed lines show Euler -6.1399×10^{-4} against the predicted -6.1313×10^{-4}, the midpoint method $+6.8296 \times 10^{-7}$ against $+6.8126 \times 10^{-7}$, and RK4 $+3.7920 \times 10^{-13}$ against $+3.7848 \times 10^{-13}$; the ratios are 1.00139, 1.00250 and 1.00190 (COMPUTED). `all(...)` is true when every entry of the sequence in its brackets is true, so the check requires every ratio to be within 1 per cent of 1. The predicted signs (Euler too low, the other two too high) and sizes are right, and at the Revision solver's step RK4 is about a billion times more accurate than Euler.

In [12], the oscillator: phase portrait and energy.

```
def oscillator(t, Y):
    """Problem B: x' = v, v' = -x for the state Y = (x, v)."""
    return np.array([Y[1], -Y[0]])


Y0 = np.array([1.0, 0.0])  # x(0) = 1, v(0) = 0
H_OSC = 0.2  # the step size
portraits = {name: np.array(solve(step, oscillator, Y0, 10.0, 50)[1])
             for name, step in METHODS.items()}
```

The state is a numpy array `Y` with `Y[0]` $= x$ and `Y[1]` $= v$ (positions are counted from 0), and the right-hand side returns the array $(v, -x)$. The start is $(1, 0)$. Each method makes 50 steps of $h = 0.2$ up to $t = 10$; the list of the 51 states becomes an array with 51 rows and 2 columns.

```
angle = np.linspace(0.0, 2 * np.pi, 400)
t_n = H_OSC * np.arange(51)  # the times 0, 0.2, ..., 10 of the 51 points
fig, (whole, zoom) = plt.subplots(1, 2, figsize=(10.0, 5.2))
```

400 angles from 0 to 2π will give the exact circle $x = \cos t$, $v = -\sin t$. `t_n` holds the 51 times $t_n = 0.2\,n$ of the computed points. The figure has two panels side by side: `whole` (left) for the whole portrait and `zoom` (right) for a close-up.

```
for name in METHODS:  # left panel: the whole portrait of each method
    whole.plot(portraits[name][:, 0], portraits[name][:, 1], color=COLORS[name],
               marker=MARKERS[name], ms=3, lw=0.9,
               label=f"{LABELS[name]}, $h = 0.2$, 50 steps")
for name in ("midpoint", "rk4"):  # right panel: the last three points only
    zoom.plot(portraits[name][48:, 0], portraits[name][48:, 1], ls="none",
              color=COLORS[name], marker=MARKERS[name], ms=8, label=LABELS[name])
```

`[:, 0]` is the first column of the array (all the positions) and `[:, 1]` the second (all the velocities). The left panel draws each method's whole path, points joined by lines. The right panel draws only the rows from position 48 on (`[48:, 0]`), the last three points at $t = 9.6$, 9.8 and 10, of the midpoint method and RK4, as large markers without lines (`ls="none"`); Euler's points are far outside the window of this panel.

```
for ax in (whole, zoom):  # the exact circle, dashed, drawn on top (zorder 5)
    ax.plot(np.cos(angle), -np.sin(angle), "--", color=COLORS["exact"], lw=0.9,
            zorder=5, label="exact circle $x^2 + v^2 = 1$")
    ax.set_aspect("equal")
    ax.set_xlabel("position $x$")
    ax.set_ylabel("velocity $v$")
whole.plot([1.0], [0.0], "o", color=COLORS["exact"], ms=6, zorder=6)  # the start
zoom.plot(np.cos(t_n[48:]), -np.sin(t_n[48:]), "x", color=COLORS["exact"], ms=9,
          zorder=6, label="exact solution, same times")
```

In both panels the exact circle is drawn as a thin dashed black line. `zorder=5` puts it on top of the coloured paths (a drawing with a larger `zorder` covers one with a smaller), so it stays visible where the RK4 path lies on it. `set_aspect("equal")` makes one unit equally long on both axes, so that the circle looks round. The black dot marks the start; in the right panel black crosses mark the exact solution $(\cos t_n, -\sin t_n)$ at the same three times.

```
zoom.set_xlim(-1.12, -0.74)  # a window round the points at t = 9.6, 9.8 and 10
zoom.set_ylim(0.12, 0.66)
whole.set_title("Phase portrait of $d^2x/dt^2 = -x$ up to $t = 10$")
zoom.set_title("Zoom: the points at $t = 9.6$, $9.8$, $10$")
whole.legend(loc="lower left", fontsize=8)
zoom.legend(loc="upper left", fontsize=8)
save_figure(fig, "phase_portrait",
            "Phase portrait of the oscillator $d^2x/dt^2 = -x$ started at $x = 1$, "
            ...)
```

The right panel shows only the window $-1.12 \le x \le -0.74$, $0.12 \le v \le 0.66$ around the last three points. Titles, legends, and the figure is saved as `02a_6_phase_portrait.png`. **What Figure 02a.6 shows:** on the left, Euler's path spirals outwards (each step multiplies the energy by 1.04), while the midpoint and RK4 paths stay close to the dashed circle; on this scale they are hard to tell apart, because the midpoint radius grows only to $\sqrt{1.0004^{50}} = 1.010$. The zoom on the right separates them: the RK4 circles sit on the exact crosses, while the midpoint triangles have run ahead along the circle and lie slightly outside it.

```
for name in METHODS:
    energy = 0.5 * (portraits[name][:, 0] ** 2 + portraits[name][:, 1] ** 2)
    factor = float(ENERGY_FACTOR[name].subs(hs, sp.Rational(1, 5)))  # at h = 0.2
    predicted = 0.5 * factor ** np.arange(51)  # E_0 times the factor n times
    report(f"energy after 50 steps, {LABELS[name]}", f"{energy[-1]:.10f}")
    check(np.max(np.abs(energy / predicted - 1)) < 1e-13,
          f"{LABELS[name]}: E_n = E_0 times the exact factor to the power n")
```

For each method the energy $E = (x^2 + v^2)/2$ is computed at all 51 points; the exact per-step factor of In [4] is evaluated at $h = 1/5$, and the prediction is $E_n = \frac{1}{2}q^n$ for $n = 0, \ldots, 50$. The energies after 50 steps are 3.5533416731 (Euler), 0.5100986302 (midpoint) and 0.4999778894 (RK4); the checks require every computed energy to equal its prediction to a relative 10^{-13}, and they pass: the energy drift of each method is exactly the one derived in Section 2.7.

```
angle_agrees = []
for name in METHODS:
```

```
    x_end, v_end = portraits[name][-1]  # the point at t = 10
    # the clockwise angle from the exact point e^(-10 i) to the computed one
    ahead = -np.angle(complex(x_end, v_end) * np.exp(10j))
```

The last part measures the angle error that the zoom shows. The **argument** $\arg w$ of a complex number w is its angle: the angle between the positive real axis and the arrow from 0 to w, counted anticlockwise; `np.angle` computes it, as a number between $-\pi$ and π. Write the computed point at $t = 10$ as $w = x + iv$ (`complex(x_end, v_end)`; `[-1]` is the last row). The exact solution is $x = \cos t$, $v = -\sin t$, that is $w = \cos t - i \sin t = e^{-it}$, so the exact point at $t = 10$ is e^{-10i}. In Python `10j` is the imaginary number $10i$, and `np.exp(10j)` is e^{10i}. Multiplying w by e^{10i} turns it back by the exact angle, so the argument of the product is the angle from the exact point to the computed one, counted anticlockwise; the minus sign counts it clockwise, the direction in which the point runs. So `ahead` is positive when the computed point has run ahead of the exact one and negative when it lags behind.

```
    R_ih = complex(R[name].subs(z, sp.I * sp.Rational(1, 5)))  # R(ih), h = 0.2
    predicted_ahead = 50 * (np.angle(R_ih) - 0.2)  # 50 (arg R(ih) - h)
```

The prediction. `R[name]` is the amplification factor of In [4], a polynomial in `z`; `.subs(z, sp.I * sp.Rational(1, 5))` puts in $z = ih$ with $h = 1/5$ (`sp.I` is sympy's i), and `complex(...)` turns the exact sympy number into an ordinary complex number. One step multiplies w by $R(-ih)$ (Section 2.7). Its argument is $-\arg R(ih)$, because $R(-ih)$ is the complex conjugate of $R(ih)$ (the coefficients of R are real) and conjugation reflects a point in the real axis, which reverses its angle. The exact factor e^{-ih} has the argument $-h$. The arguments of a product add up, so after 50 steps the computed point has turned by $-50 \arg R(ih)$ and the exact one by $-50h$; the computed point is ahead, clockwise, by $50\,(\arg R(ih) - h)$.

```
    report(f"t = 10, {LABELS[name]}: radius, angle ahead of the exact point",
           f"{math.hypot(x_end, v_end):.4f}, {ahead:+.3e} rad")
    angle_agrees.append(abs(ahead - predicted_ahead) < 1e-12)
check(all(angle_agrees),
      "at t = 10 each point is ahead of the exact one by 50 (arg R(ih) - h)")
```

`math.hypot(x_end, v_end)` is the radius $\sqrt{x^2 + v^2}$ of the end point. The three RESULT lines give the radius and the angle ahead: Euler 2.6658 and -1.302×10^{-1} rad (it lags behind by 0.13 rad); midpoint 1.0100 and $+6.586 \times 10^{-2}$ rad (ahead by 0.066 rad, the shift seen in the zoom); RK4 1.0000 and -1.314×10^{-4} rad (COMPUTED). The RK4 value agrees with the lag of $\omega^5/120$ per step derived in Section 2.24: $50 \cdot 0.2^5/120 = 1.33 \times 10^{-4}$ to leading order. The check requires each measured angle to equal its prediction within 10^{-12}, and it passes.

In [13], the energy over a long time.

```
long_runs = {name: np.array(solve(step, oscillator, Y0, 100.0, 500)[1])
             for name, step in METHODS.items()}
fig, ax = plt.subplots()
t_long = np.linspace(0.0, 100.0, 501)
drift = {}
for name in METHODS:
```

```
    energy = 0.5 * (long_runs[name][:, 0] ** 2 + long_runs[name][:, 1] ** 2)
    drift[name] = np.abs(energy / 0.5 - 1.0)
    ax.semilogy(t_long[1:], drift[name][1:], color=COLORS[name], lw=1.4,
                label=LABELS[name])
```

Now 500 steps of $h = 0.2$, up to $t = 100$. For each method the relative energy error $|E_n/E_0 - 1|$ (with $E_0 = 0.5$) is drawn on a logarithmic vertical axis against the 500 times after the start.

```
ax.set_xlabel("time $t$ (arbitrary units)")
ax.set_ylabel("relative energy error $|E_n/E_0 - 1|$")
ax.set_title("Energy drift of the oscillator, step $h = 0.2$")
ax.legend()
save_figure(fig, "energy_drift",
            "The relative error of the oscillator energy, $|E_n/E_0 - 1|$, on a "
            ...)
report("energy errors at t = 100 (Euler, midpoint, RK4)",
       ", ".join(f"{drift[name][-1]:.3e}" for name in METHODS))
check(drift["euler"][-1] > 1e8 and 0.1 < drift["midpoint"][-1] < 0.3
      and drift["rk4"][-1] < 5e-4,
      "after 500 steps: Euler energy off by > 1e8, midpoint by 10-30 %, RK4 < 5e-4")
```

Labels, title, legend and the saved figure `02a_7_energy_drift.png`. `", ".join(...)` writes the three final errors separated by commas: 3.286×10^8 (Euler: $1.04^{500} - 1$), 0.2214 (midpoint: $1.0004^{500} - 1$) and 4.421×10^{-4} (RK4: $1 - 0.99999911^{500}$). The check confirms these sizes. **What Figure 02a.7 shows:** the Euler curve is a straight rising line (the same factor at every step makes a straight line on a logarithmic axis); the midpoint and RK4 curves bend over, because their factors are so close to 1 that $q^n - 1 \approx n(q - 1)$ grows only in proportion to n; the three methods differ by many powers of ten.

In [14], Richardson's rule.

```
N_R = [2 ** k for k in range(1, 8)]  # the coarse runs: 2 ... 128 steps
coarse = np.array([solve(rk4_step, decay, 1.0, 1.0, n)[1][-1] for n in N_R])
fine = np.array([solve(rk4_step, decay, 1.0, 1.0, 2 * n)[1][-1] for n in N_R])
estimated = (coarse - fine) / 15.0  # Richardson's estimate of the error of fine
actual = fine - EXACT_A  # the true error of fine (we know the exact answer here)
extrapolated = (16.0 * fine - coarse) / 15.0
```

For $N = 2, 4, \ldots, 128$ the cell runs RK4 with N steps (`coarse`, the value Y_h) and with $2N$ steps (`fine`, the value $Y_{h/2}$). `estimated` is Richardson's estimate $(Y_h - Y_{h/2})/15$ of the error of the finer run, `actual` its true error, and `extrapolated` the value $(16Y_{h/2} - Y_h)/15$ of Section 2.6.

```
for n, e_est, e_act, e_ext in zip(N_R, estimated, actual, extrapolated - EXACT_A):
    say(f"N = {n:3d} -> {2 * n:3d}: estimated {e_est: .3e}, actual {e_act: .3e}, "
        f"extrapolated error {e_ext: .2e}")
```

`zip` walks through four sequences together. Each line prints the estimate, the true error and the error of the extrapolated value (the blank in `{e_est: .3e}` leaves room for a minus sign). From $N = 8$ on, the estimate agrees with the true error to within 6 per cent, and the extrapolated value is between 17 and 260 times more accurate than the finer run (COMPUTED).

```
fig, ax = plt.subplots(figsize=(6.0, 5.4))
ax.loglog(np.abs(actual), np.abs(estimated), "o", color=COLORS["rk4"], ms=6,
          label="RK4 runs with $2N = 4$ to $256$ steps")
for n, x_value, y_value in zip(N_R, np.abs(actual), np.abs(estimated)):
    ax.annotate(f"$N = {n}$", (x_value, y_value), textcoords="offset points",
                xytext=(6, -10), fontsize=7)
diagonal = np.array([1e-14, 1e-4])
ax.loglog(diagonal, diagonal, "--", color=COLORS["guide"], lw=0.9,
          label="estimate = actual")
```

Each run becomes one point (true error, estimate) on logarithmic axes; `ax.annotate` writes its N next to it, shifted by 6 points to the right and 10 points down. The dashed diagonal is the line on which estimate and true error are equal.

```
ax.set_xlabel("actual error of the finer run")
ax.set_ylabel("Richardson estimate $(Y_h - Y_{h/2})/15$")
ax.set_title("Richardson's estimate against the true error")
ax.legend(loc="upper left")
save_figure(fig, "richardson",
            "Richardson's error estimate $(Y_h - Y_{h/2})/15$ of RK4 for problem A "
            ...)
ratio = estimated[2:] / actual[2:]
check(np.all((ratio > 0.9) & (ratio < 1.1)),
      "Richardson's estimate is within 10 % of the actual error for N >= 8")
check(np.all(np.abs(extrapolated[2:6] - EXACT_A) < np.abs(actual[2:6]) / 10),
      "the extrapolated value is more than 10 times more accurate (N = 8 to 64)")
```

Labels, legend and the saved figure `02a_8_richardson.png`. `ratio` is the estimate divided by the true error for $N \geq 8$ (positions 2 and later); `&` combines two arrays of truth values entry by entry, and `np.all` requires every entry to be true: all ratios lie between 0.9 and 1.1. The second check requires the extrapolated values for $N = 8$ to 64 to be more than ten times more accurate than the finer runs. **What Figure 02a.8 shows:** the points lie on the diagonal: Richardson's estimate, made without the exact answer, is the error itself once the coarse run has at least 8 steps; for $N = 2$ and 4 the error is not yet close to Ch^4 and the estimate is off by up to 25 per cent.

In [15], the last check.

```
names = ["scale_factors", "scale_factor_product", "convergence_loglog",
         "error_ratios", "rounding_floor", "phase_portrait", "energy_drift",
         "richardson"]
present = [output_file(f"{FIGURE_FOLDER}/02a_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02a exist")
all_checks_passed()
```

`enumerate(names, 1)` numbers the eight figure names from 1. For each, the cell asks whether the file `02a_<k>_<name>.png` exists; the check requires all eight to exist, and `all_checks_passed()` prints the last line, ALL 29 CHECKS PASSED (notebook 02a). The 29 checks are: 3 in In [3], 4 in In [4], 2 in In [5], 3 in In [6], 2 in In [7], 1 in In [8], 1 in In [9], 3 in In [10], 2 in In [11], 4 in In [12], 1 in In [13], 2 in In [14] and 1 in In [15].

2.12 Boundary-value problems and the shooting method

Conditions at both ends. So far every condition was given at the start: $y(0)$, or $x(0)$ and $x'(0)$. Many problems give one condition at each END of an interval instead. Such a problem is a **boundary-value problem**. Often it contains a number that is not known in advance, and it has a solution that is not zero everywhere only for special values of that number: these values are the **eigenvalues** and the solutions the **eigenfunctions**. The energy levels of quantum mechanics, and the Kohn-Sham levels of this book, are eigenvalues of this kind.

The vibrating string. The simplest example is

$$u''(x) = -\lambda\, u(x), \qquad u(0) = 0, \qquad u(1) = 0,$$

the shape $u(x)$ of a string fixed at $x = 0$ and $x = 1$ that vibrates in one of its natural modes; λ is the unknown number. For $\lambda > 0$ every solution of the equation has the form

$$u(x) = A\sin(\sqrt{\lambda}\,x) + B\cos(\sqrt{\lambda}\,x)$$

with two constants A and B. (Differentiating twice brings each term back multiplied by $-\lambda$: the derivative of $\sin(cx)$ is $c\cos(cx)$ and that of $\cos(cx)$ is $-c\sin(cx)$, with $c = \sqrt{\lambda}$ and $c^2 = \lambda$. That these are ALL the solutions follows from the uniqueness theorem of Section 2.2, because A and B can match any starting values $u(0)$, $u'(0)$.)

$$u(0) = A\sin 0 + B\cos 0 = B$$

($\sin 0 = 0$ and $\cos 0 = 1$), so the left condition $u(0) = 0$ forces $B = 0$. A multiple of a solution is again a solution, so we may fix the size by choosing the starting slope $u'(0) = 1$:

$$u'(0) = A\sqrt{\lambda}\cos 0 = A\sqrt{\lambda} = 1, \qquad A = \frac{1}{\sqrt{\lambda}}$$

(the derivative of $A\sin(\sqrt{\lambda}\,x)$ at $x = 0$). So the solution that starts with $u(0) = 0$ and $u'(0) = 1$ is

$$u(x;\lambda) = \frac{\sin(\sqrt{\lambda}\,x)}{\sqrt{\lambda}}.$$

The right condition $u(1) = 0$ holds when $\sin\sqrt{\lambda} = 0$, that is when $\sqrt{\lambda}$ is a whole multiple of π:

$$\sqrt{\lambda} = n\pi, \qquad \lambda_n = n^2\pi^2, \qquad n = 1, 2, 3, \ldots$$

(the zeros of the sine are the whole multiples of π; $n = 0$ gives $\lambda = 0$, where the solution is $u = x$, which is not zero at $x = 1$). So the eigenvalues of the string are $\pi^2 = 9.8696$, $4\pi^2 = 39.478$, $9\pi^2 = 88.826$, ... (PROVED).

The shooting method. The computer does not know this formula. It turns the boundary-value problem into a sequence of initial-value problems, like aiming a cannon:

1. guess a value of λ;
2. solve the initial-value problem from the left end with the conditions there ($u(0) = 0$, $u'(0) = 1$), for example with RK4;
3. see where the "shot" lands at the right end: the **mismatch** $F(\lambda) = u(1; \lambda)$;
4. change the guess until the mismatch is zero.

The function $F(\lambda)$ is the **shooting function**, and the eigenvalues are its zeros. For the string we know it exactly, $F(\lambda) = \sin\sqrt{\lambda}/\sqrt{\lambda}$, which lets Notebook 02b check every shot.

Brackets. Two guesses λ_{lo} and λ_{hi} at which F has opposite signs form a **bracket**. If F is continuous, it has a zero between them: this is the **intermediate value theorem** of calculus (ASSUMED: quoted without proof). For the string $F(5) = 0.35184 > 0$ and $F(15) = -0.17245 < 0$, so $5 < \lambda < 15$ brackets an eigenvalue (it is π^2).

Bisection. Take the midpoint $m = (\lambda_{\text{lo}} + \lambda_{\text{hi}})/2$ of the bracket and evaluate $F(m)$. If $F(m)$ has the same sign as $F(\lambda_{\text{lo}})$, the zero lies in the right half and m becomes the new λ_{lo}; otherwise it lies in the left half and m becomes the new λ_{hi}. Each step halves the bracket, so after k steps its width is

$$w_k = \frac{\lambda_{\text{hi}} - \lambda_{\text{lo}}}{2^k}$$

(the starting width halved k times). Bisection always works, but slowly: it gains one binary digit (a factor 2) per step; starting from the width 10, after 45 steps the width is $10/2^{45} = 2.8 \times 10^{-13}$. *Worked example.* From the bracket $(5, 15)$: $m = 10$, and $F(10) = \sin(3.1623)/3.1623 = -0.00654 < 0$ (because $\sqrt{10} = 3.1623$ is a little larger than π, where the sine turns negative), the opposite sign of $F(5)$, so the new bracket is $(5, 10)$; then $m = 7.5$ with $F > 0$, bracket $(7.5, 10)$; then 8.75, 9.375, 9.6875, 9.84375, ... (Notebook 02b, In [5], prints these midpoints and the signs of F).

The secant rule. Draw the straight line through the last two points $(x_0, F(x_0))$ and $(x_1, F(x_1))$ and take its zero as the next guess x_2. The line is

$$\ell(x) = F(x_1) + \frac{F(x_1) - F(x_0)}{x_1 - x_0}\,(x - x_1)$$

(the line through $(x_1, F(x_1))$ with the slope of the chord between the two points). Setting $\ell(x_2) = 0$:

$$\frac{F(x_1) - F(x_0)}{x_1 - x_0}\,(x_2 - x_1) = -F(x_1)$$

(subtract $F(x_1)$ from both sides);

$$x_2 = x_1 - F(x_1)\,\frac{x_1 - x_0}{F(x_1) - F(x_0)}$$

(divide by the slope and add x_1). Then repeat with the two newest points. *Worked example.* With $x_0 = 5$, $x_1 = 15$, $F(5) = 0.35184$, $F(15) = -0.17245$:

$$x_2 = 15 - (-0.17245)\cdot\frac{15-5}{-0.17245-0.35184} = 15 - \frac{1.7245}{0.52429} = 15 - 3.2892 = 11.7108$$

(the secant formula with these numbers). Notebook 02b prints 11.7107898. Close to a zero, the number of correct digits of the secant rule is multiplied by about 1.618 at every step (the golden ratio; ASSUMED: a standard result of numerical analysis, quoted without proof and only illustrated by the notebook), much faster than the one binary digit per step of bisection. Unlike bisection, the secant rule does not keep the root bracketed and can fail when the line is nearly flat; Section 2.24 shows how the Revision solver combines a fast rule with the safety of bisection.

2.13 The finite square well of quantum mechanics

The equation. In quantum mechanics the allowed energies E of a particle in one dimension, moving in a **potential** $V(x)$ (its potential energy at the position x), are the eigenvalues of the **Schrödinger equation**; in units in which Planck's constant divided by 2π (written $\hbar$) and the mass m are both 1, it reads

$$-\frac{1}{2}\,u''(x) + V(x)\,u(x) = E\,u(x).$$

We take this equation as given (ASSUMED here: it is the basic equation of quantum mechanics, used in this chapter only as an example of an eigenvalue problem). The function u is the **wave function**: $u(x)^2$ is the probability density of finding the particle at x, so u is **normalised**, $\int u^2\,dx = 1$. Multiplying the equation by -2 gives

$$u'' - 2Vu = -2Eu$$

(each term multiplied by -2), and adding $2Vu$ to both sides

$$u'' = 2\,(V(x) - E)\,u.$$

The well. We take $V(x) = -V_0$ inside $|x| < a$ and $V(x) = 0$ outside, with the depth $V_0 = 15$ and the half-width $a = 1$. A **bound state** is a solution with $-V_0 < E < 0$ that decays to zero far away on both sides; its energy is an eigenvalue. Write

$$k = \sqrt{2(E+V_0)}, \qquad \kappa = \sqrt{-2E}$$

(both are real and positive for $-V_0 < E < 0$).

Outside the well ($V = 0$): $u'' = -2Eu = \kappa^2 u$, solved by $e^{\kappa x}$ and $e^{-\kappa x}$ (differentiating twice brings back the factor κ^2). For $x < -a$ only $e^{\kappa x}$ goes to zero as $x \to -\infty$, so the solution on the left is a multiple of $e^{\kappa(x+a)}$; we choose the multiple 1. At $x = -a$ this gives the starting values

$$u(-a) = 1, \qquad u'(-a) = \kappa$$

(the value and the derivative of $e^{\kappa(x+a)}$ at $x=-a$, where the exponent is 0). So, unlike the string, here we know exactly how to start the shot.

Inside the well ($V=-V_0$): $u''=2(-V_0-E)u=-k^2u$, solved by $\cos(kx)$ and $\sin(kx)$.

The solution inside that continues the left tail is

$$u(x)=\cos\big(k(x+a)\big)+\frac{\kappa}{k}\sin\big(k(x+a)\big),$$

because at $x=-a$ it gives $u=\cos 0+0=1$ and $u'=-k\sin 0+\kappa\cos 0=\kappa$ (derivative term by term), the starting values above, and it solves $u''=-k^2u$.

Symmetry. The well is symmetric, $V(-x)=V(x)$. If $u(x)$ solves the equation, so does $u(-x)$ (the second derivative does not notice the change of sign of x, by the chain rule applied twice), and a bound-state energy of a one-dimensional problem belongs to only one wave function up to a factor (ASSUMED: a standard fact of one-dimensional quantum mechanics, quoted without proof). So $u(-x)=c\,u(x)$, and applying this twice gives $c^2=1$: every bound state is **even**, $u(-x)=u(x)$, or **odd**, $u(-x)=-u(x)$. An even smooth function has $u'(0)=0$ (its graph is flat at the centre) and an odd one has $u(0)=0$. These are the conditions at the far end of the shot from $x=-a$ to $x=0$:

$$F_{\text{even}}(E)=u'(0),\qquad F_{\text{odd}}(E)=u(0).$$

The exact conditions. At $x=0$ the inside solution gives

$$u(0)=\cos(ka)+\frac{\kappa}{k}\sin(ka),\qquad u'(0)=-k\sin(ka)+\kappa\cos(ka)$$

(insert $x=0$ into u and into its derivative). The even condition $u'(0)=0$ becomes

$$\kappa\cos(ka)=k\sin(ka),\qquad k\tan(ka)=\kappa$$

(move the sine term to the other side; then divide by $\cos(ka)$, using $\tan=\sin/\cos$). The odd condition $u(0)=0$ becomes

$$\cos(ka)=-\frac{\kappa}{k}\sin(ka),\qquad -k\cot(ka)=\kappa$$

(move the sine term; then multiply by k and divide by $\sin(ka)$, using $\cot=\cos/\sin$). Now put $z=ka$ and $z_0=a\sqrt{2V_0}$:

$$z^2+(\kappa a)^2=2(E+V_0)a^2-2Ea^2=2V_0a^2=z_0^2$$

(insert $k^2=2(E+V_0)$ and $\kappa^2=-2E$; the terms with E cancel), so $\kappa a=\sqrt{z_0^2-z^2}$. Multiplying the even and the odd condition by a and dividing by $z=ka$:

$$\text{even: }\tan z=\frac{\sqrt{z_0^2-z^2}}{z},\qquad \text{odd: }-\cot z=\frac{\sqrt{z_0^2-z^2}}{z},\qquad E=\frac{z^2}{2a^2}-V_0$$

(the last relation is $k^2 = 2(E + V_0)$ solved for E, with $k = z/a$). These are **transcendental equations**: no formula gives z, but a computer solves them to any number of digits. Here $z_0 = \sqrt{30} = 5.477$. The right side $\sqrt{z_0^2 - z^2}/z$ falls from $+\infty$ at $z = 0$ to 0 at $z = z_0$; on each interval of length $\pi/2$ one of the branches of $\tan z$ (on $(0, \pi/2)$, $(\pi, 3\pi/2)$, ...) or of $-\cot z$ (on $(\pi/2, \pi)$, ...) rises from 0 to $+\infty$ and crosses it exactly once. So the well has one bound state for every interval of length $\pi/2$ that begins below z_0: since $z_0/(\pi/2) = 3.487$, there are exactly 4 bound states, even, odd, even, odd. Their energies, computed with 30 digits by Notebook 02b (In [6]), are

$$E_0 = -14.1206, \quad E_1 = -11.5181, \quad E_2 = -7.3330, \quad E_3 = -2.0437$$

(COMPUTED from the exact equations).

Why shots miss. For an energy that is NOT an eigenvalue, the shot that starts with the decaying left tail arrives on the right side of the well as a mixture of $e^{-\kappa x}$ and the growing $e^{+\kappa x}$; the growing part takes over and the shot flies off to $+\infty$ or $-\infty$. Only at an eigenvalue is the growing part exactly absent. Just below and just above an eigenvalue the shot flies off to OPPOSITE infinities, which is what makes the shooting function change sign there (Figure 02b.6 shows it).

Normalisation and orthogonality. The tail on the left contributes to $\int u^2\,dx$ the amount

$$\int_{-\infty}^{-a} e^{2\kappa(x+a)}\,dx = \Big[\frac{e^{2\kappa(x+a)}}{2\kappa}\Big]_{-\infty}^{-a} = \frac{1}{2\kappa} - 0 = \frac{1}{2\kappa}$$

(an antiderivative of e^{cx} is e^{cx}/c; at $x = -a$ the exponential is $e^0 = 1$, and at $-\infty$ it is 0). Two wave functions u_m, u_n are **orthogonal** when $\int u_m u_n\,dx = 0$. For an even and an odd state the product $p(x) = u_m(x)u_n(x)$ is odd, $p(-x) = -p(x)$, and the integral of an odd function over the whole line is zero:

$$\int_{-\infty}^{\infty} p(x)\,dx = \int_{-\infty}^{0} p(x)\,dx + \int_{0}^{\infty} p(x)\,dx = -\int_{0}^{\infty} p(s)\,ds + \int_{0}^{\infty} p(x)\,dx = 0$$

(split at 0; in the first integral substitute $x = -s$, which turns dx into $-ds$, reverses the limits and turns $p(x)$ into $p(-s) = -p(s)$). Two states of the same parity with different energies are also orthogonal (ASSUMED here: a theorem of quantum mechanics, which Notebook 02b confirms numerically).

Simpson's rule. To integrate a function given at equally spaced points $x_0, x_1, \ldots, x_n$ (with n even and spacing h), Simpson's rule passes a parabola through every three neighbouring points and integrates it exactly. For three points at $-h$, 0, h with the values f_-, f_0, f_+, write the parabola as $p(x) = \alpha + \beta x + \gamma x^2$:

$$f_- = \alpha - \beta h + \gamma h^2, \qquad f_0 = \alpha, \qquad f_+ = \alpha + \beta h + \gamma h^2$$

(the parabola takes the three values);

$$\gamma h^2 = \frac{f_- + f_+}{2} - f_0$$

(add the first and the third equation, $f_- + f_+ = 2\alpha + 2\gamma h^2$, then use $\alpha = f_0$);

$$\int_{-h}^{h} p(x)\,dx = 2\alpha h + \frac{2\gamma h^3}{3}$$

(integrate term by term; the odd term βx gives zero);

$$\int_{-h}^{h} p(x)\,dx = 2f_0 h + \frac{2h}{3}\Big(\frac{f_- + f_+}{2} - f_0\Big) = \frac{h}{3}(f_- + 4f_0 + f_+)$$

(insert α and γh^2; collect: $2h - \frac{2h}{3} = \frac{4h}{3}$). Adding these pieces over the pairs of intervals gives Simpson's rule

$$\int_{x_0}^{x_n} f\,dx \approx \frac{h}{3}(f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 2f_{n-2} + 4f_{n-1} + f_n)$$

(every inner point with an even index belongs to two pieces and gets $1 + 1 = 2$). Its error is proportional to h^4 (ASSUMED: a standard result, quoted without proof), the same order as RK4.

The connection to this book. The Kohn-Sham equations of the book are solved in the same way. Two first-order equations along the hidden coordinate are shot from the tip of an interval to its other end, the brane, where, because of a mirror symmetry of the model that the Revision record labels ASSUMED, an even or an odd condition must hold, exactly like $u'(0) = 0$ or $u(0) = 0$ here (Section 2.24).

2.14 Example: shooting a string and a quantum well

Notebook 02b carries out Sections 2.12 and 2.13. Part A shoots the string with RK4 (200 steps), draws three shots and the whole shooting function, and finds $\lambda_1 = \pi^2$ both by bisection and by the secant rule, comparing how fast they converge. Part B solves the exact well equations with mpmath at 30 digits, shoots the well with RK4 from $x = -a$ with the even and the odd condition at the centre, refines all four levels at once by bisection, shows how a shot misses, draws the four normalised wave functions, checks their nodes, normalisation and orthogonality (for states of different parity also numerically), and measures the order 4 of the computed energies. It ends with ALL 16 CHECKS PASSED (notebook 02b).

2.15 How to run Notebook 02b

Step 1. What this notebook does and what it needs.

Notebook 02b (The shooting method: a vibrating string and a quantum well) is the file `Revision /textbook/notebooks/02b_shooting_quantum_well.ipynb` of the repository Dirac_claude. It finds the eigenvalues of two boundary-value problems by shooting: the string fixed at both ends, whose eigenvalues are n squared times pi squared, with bisection and the secant rule, and the finite square well of quantum mechanics (depth 15, half-width 1, in units with hbar = m = 1), whose four bound states it computes with RK4 and the even and odd conditions at the centre; it compares every energy with the exact transcendental equations solved with mpmath to 30 digits, counts the

nodes of the eigenfunctions, checks their orthogonality, and measures the order 4 of the eigenvalue error. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 02b_shooting_quantum_well.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 02b_shooting_quantum_well.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 02b_shooting_quantum_well.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/02b.captions.json`
- `Revision/textbook/figures/02b_1_trial_solutions_string.png`
- `Revision/textbook/figures/02b_2_shooting_function_string.png`
- `Revision/textbook/figures/02b_3_bisection_vs_secant.png`
- `Revision/textbook/figures/02b_4_graphical_solution.png`
- `Revision/textbook/figures/02b_5_shooting_functions_well.png`
- `Revision/textbook/figures/02b_6_trial_solutions_well.png`
- `Revision/textbook/figures/02b_7_eigenfunctions.png`
- `Revision/textbook/figures/02b_8_eigenvalue_convergence.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of notebook 02b exist
ALL 16 CHECKS PASSED (notebook 02b)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/02b_shooting_quantum_well.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 02b_shooting_quantum_well.ipynb
python -m nbconvert --execute --inplace 02b_shooting_quantum_well.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

2.16 Notebook 02b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/02b_shooting_quantum_well.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Some differential equations have conditions at BOTH ends of an interval, and they
have solutions only for special values of a number in the equation, the
*eigenvalues*. The **shooting method** finds them: guess the number, solve the
equation from one end as an initial-value problem, see how badly the condition
at the other end fails, and correct the guess. This notebook

- Part A: shoots the equation of a string fixed at both ends,
  $u'' = -\lambda u$ with $u(0) = u(1) = 0$, whose eigenvalues are exactly
  $\lambda_n = n^2 \pi^2$; it finds $\lambda_1 = \pi^2$ by bisection and by the
  secant rule and compares how fast the two converge;
- Part B: shoots the Schroedinger equation of a particle in a finite square
  well (depth $V_0 = 15$, half-width $a = 1$), finds its four bound states with
  RK4 and the even and odd conditions at the centre, and compares every energy
  with the exact solution of the well's transcendental equations, computed with
  mpmath to 30 digits;
- draws the trial solutions that miss and the ones that hit, the shooting
  functions, the bound-state wave functions in the well, and the convergence of
  the computed energies (order 4, as for RK4 itself);
- checks the number of nodes and the orthogonality of the wave functions.

It draws 8 figures and prints a PASS line for every check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 02b (The shooting method: a vibrating string and a quantum well) is the file
`Revision/textbook/notebooks/02b_shooting_quantum_well.ipynb` of the repository
Dirac_claude. It finds the eigenvalues of two boundary-value problems by shooting: the
string fixed at both ends, whose eigenvalues are n squared times pi squared, with
bisection and the secant rule, and the finite square well of quantum mechanics (depth 15,
half-width 1, in units with hbar = m = 1), whose four bound states it computes with RK4
and the even and odd conditions at the centre; it compares every energy with the exact
transcendental equations solved with mpmath to 30 digits, counts the nodes of the
eigenfunctions, checks their orthogonality, and measures the order 4 of the eigenvalue
error. It needs a computer with Windows 11, macOS or Linux, an internet connection for
the installation, the program Git, and Python 3.12 or newer (the notebooks were built
with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.
```

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another

notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 02b_shooting_quantum_well.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 02b_shooting_quantum_well.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 02b_shooting_quantum_well.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/02b.captions.json‘
- ‘Revision/textbook/figures/02b_1_trial_solutions_string.png‘
- ‘Revision/textbook/figures/02b_2_shooting_function_string.png‘
- ‘Revision/textbook/figures/02b_3_bisection_vs_secant.png‘
- ‘Revision/textbook/figures/02b_4_graphical_solution.png‘
- ‘Revision/textbook/figures/02b_5_shooting_functions_well.png‘
- ‘Revision/textbook/figures/02b_6_trial_solutions_well.png‘
- ‘Revision/textbook/figures/02b_7_eigenfunctions.png‘
- ‘Revision/textbook/figures/02b_8_eigenvalue_convergence.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of notebook 02b exist
ALL 16 CHECKS PASSED (notebook 02b)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/02b_shooting_quantum_well.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 02b_shooting_quantum_well.ipynb
  python -m nbconvert --execute --inplace 02b_shooting_quantum_well.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues

normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 02b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 02b (The shooting method: a vibrating string and a quantum well) is the file
# Revision/textbook/notebooks/02b_shooting_quantum_well.ipynb of the repository
# Dirac_claude. It finds the eigenvalues of two boundary-value problems by shooting: the
# string fixed at both ends, whose eigenvalues are n squared times pi squared, with
# bisection and the secant rule, and the finite square well of quantum mechanics (depth
# 15, half-width 1, in units with hbar = m = 1), whose four bound states it computes with
# RK4 and the even and odd conditions at the centre; it compares every energy with the
# exact transcendental equations solved with mpmath to 30 digits, counts the nodes of the
# eigenfunctions, checks their orthogonality, and measures the order 4 of the eigenvalue
# error. It needs a computer with Windows 11, macOS or Linux, an internet connection for
# the installation, the program Git, and Python 3.12 or newer (the notebooks were built
# with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
```

```
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
```

```
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 02b_shooting_quantum_well.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
```

```
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 02b_shooting_quantum_well.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 02b_shooting_quantum_well.ipynb (in the same folder, without running the cells again)
# to read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/02b.captions.json
# - Revision/textbook/figures/02b_1_trial_solutions_string.png
# - Revision/textbook/figures/02b_2_shooting_function_string.png
# - Revision/textbook/figures/02b_3_bisection_vs_secant.png
# - Revision/textbook/figures/02b_4_graphical_solution.png
# - Revision/textbook/figures/02b_5_shooting_functions_well.png
# - Revision/textbook/figures/02b_6_trial_solutions_well.png
# - Revision/textbook/figures/02b_7_eigenfunctions.png
# - Revision/textbook/figures/02b_8_eigenvalue_convergence.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 8 figure files of notebook 02b exist
#     ALL 16 CHECKS PASSED (notebook 02b)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
```

```
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/02b_shooting_quantum_well.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 02b_shooting_quantum_well.ipynb
#     python -m nbconvert --execute --inplace 02b_shooting_quantum_well.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "02b"  # this notebook: chapter 02, example b


def find_repository_root():
```

```
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
```

```
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 02b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Boundary-value problem**: a differential equation with conditions at two
  different points (here at both ends of an interval), instead of all conditions
  at the start.
- **Eigenvalue, eigenfunction**: a value of the number $\lambda$ (or the energy
  $E$) for which the boundary-value problem has a solution that is not zero
  everywhere; that solution is an eigenfunction.
- **Shooting**: solve from one end with the conditions there and a guessed
  eigenvalue (like aiming a cannon), measure the *mismatch* at the other end
  (where the shot lands), and change the guess until the mismatch is zero.
- **Shooting function** $F$: the mismatch as a function of the guess; the
  eigenvalues are its zeros.
- **Bracket**: two guesses at which $F$ has opposite signs; a continuous $F$ has
  a zero between them.
- **Bisection**: replace the bracket by the half in which the sign changes; the
  bracket halves at every step.
- **Secant rule**: draw the straight line through the last two points
  $(\lambda, F(\lambda))$ and take its zero as the next guess.
- **Schroedinger equation**: the equation of quantum mechanics that gives the
  allowed energies $E$ of a particle in a potential $V(x)$; here, for one
  dimension and in units with $\hbar = m = 1$ ($\hbar$ is Planck's constant
  divided by $2\pi$, $m$ the mass), $-\frac{1}{2} u''(x) + V(x) u(x) = E u(x)$.
  We take it as given.
- **Potential well, bound state**: a region where $V$ is lower than outside; a
  bound state is a solution with $E$ below the outside value of $V$ that decays
  to zero far away. Its energy is an eigenvalue.
- **Wave function** $u(x)$: the eigenfunction of the Schroedinger equation;
  $u(x)^2$ is the probability density of finding the particle at $x$, so we
  *normalise* it: $\int u^2\,dx = 1$.
- **Even, odd (parity)**: $u$ is even if $u(-x) = u(x)$, odd if $u(-x) = -u(x)$.
  An even smooth function has $u'(0) = 0$, an odd one $u(0) = 0$.
- **Node**: a point where the eigenfunction changes sign.
- **Orthogonal**: two functions with $\int u_m u_n\,dx = 0$.
```

Text cell:

```
## 4. The physical and mathematical situation

**Part A: the string.** $u'' = -\lambda u$ on $0 \le x \le 1$ with $u(0) = 0$ and
$u(1) = 0$. For $\lambda > 0$ the solutions of the equation are
$A \sin(\sqrt{\lambda}\, x) + B \cos(\sqrt{\lambda}\, x)$ (differentiate twice:
each term comes back multiplied by $-\lambda$). The condition $u(0) = 0$ gives
$B = 0$. We may choose $A$ freely (a multiple of a solution is a solution), so
we start with $u(0) = 0$ and slope $u'(0) = 1$, which gives
$A = 1/\sqrt{\lambda}$:

$$u(x; \lambda) = \frac{\sin(\sqrt{\lambda}\, x)}{\sqrt{\lambda}}, \qquad
F(\lambda) = u(1; \lambda) = \frac{\sin\sqrt{\lambda}}{\sqrt{\lambda}}.$$
```

```
$F(\lambda) = 0$ when $\sqrt{\lambda} = n\pi$, that is $\lambda_n = n^2\pi^2$
for $n = 1, 2, 3, \dots$ The same equation describes a particle in a box with
infinitely high walls, with the energy $E = \lambda/2$ (units $\hbar = m = 1$).
The computer does not know the formula: it computes $F(\lambda)$ by solving the
initial-value problem with RK4, and we use the formula only to check it.

**Part B: the finite square well.** The potential is $V(x) = -V_0$ for
$|x| < a$ and $V(x) = 0$ for $|x| > a$, with $V_0 = 15$ and $a = 1$. The
Schroedinger equation $-\frac{1}{2} u'' + V u = E u$ multiplied by $-2$ reads

$$u'' = 2\,(V(x) - E)\, u .$$

A bound state has $-V_0 < E < 0$. Write $k = \sqrt{2(E + V_0)}$ and
$\kappa = \sqrt{-2E}$ (both positive).

- Outside the well ($V = 0$): $u'' = \kappa^2 u$, solved by $e^{\kappa x}$ and
  $e^{-\kappa x}$. For $x < -a$ only $e^{\kappa x}$ goes to zero as
  $x \to -\infty$. So we know exactly how to start: at $x = -a$ take $u = 1$
  and $u' = \kappa$ (the values of $e^{\kappa (x + a)}$ there).
- Inside ($V = -V_0$): $u'' = -k^2 u$, solved by $\cos$ and $\sin$ of $k x$.
- The well is symmetric, so every bound state is even or odd. An even state has
  $u'(0) = 0$, an odd state $u(0) = 0$. These are the conditions at the far end
  of the shot: we integrate from $x = -a$ to $x = 0$ and use the shooting
  functions $F_{\rm even}(E) = u'(0)$ and $F_{\rm odd}(E) = u(0)$.

**The exact answer.** Inside, the solution that starts with $u(-a) = 1$,
$u'(-a) = \kappa$ is $u = \cos(k(x + a)) + (\kappa/k) \sin(k(x + a))$ (check
the two start values by putting $x = -a$). At $x = 0$:
$F_{\rm even} = -k \sin(ka) + \kappa \cos(ka)$, which vanishes when
$k \tan(ka) = \kappa$, and $F_{\rm odd} = \cos(ka) + (\kappa/k) \sin(ka)$, which
vanishes when $-k \cot(ka) = \kappa$. With $z = ka$ and
$z_0 = a\sqrt{2V_0} = \sqrt{30}$ we have $\kappa a = \sqrt{z_0^2 - z^2}$, so

$$\text{even: } \tan z = \frac{\sqrt{z_0^2 - z^2}}{z}, \qquad
\text{odd: } -\cot z = \frac{\sqrt{z_0^2 - z^2}}{z}, \qquad
E = \frac{z^2}{2a^2} - V_0 .$$

These equations have no formula for $z$, but mpmath solves them to any number
of digits. The well has one bound state for each interval of length $\pi/2$
that begins below $z_0$. The intervals begin at $0$, $\pi/2 = 1.571$,
$\pi = 3.142$, $3\pi/2 = 4.712$, $2\pi = 6.283$, ...; with $z_0 = 5.477$
(so $z_0/(\pi/2) = 3.49$) the first four begin below $z_0$ and the fifth does
not, so the well has 4 bound states.

**The connection to this book.** The Kohn-Sham equations of the book are solved
in the same way: two first-order equations in the hidden coordinate are shot
from the tip of the interval to the brane, where (with the ASSUMED mirror
symmetry) an even or an odd condition must hold, exactly like $u'(0) = 0$ or
$u(0) = 0$ here.
```

Text cell:

```
## 5. The tools: RK4 for a system, bisection and the secant rule
```

The next cell defines one RK4 step for a system $Y' = f(x, Y)$ (the same rule as for one equation, applied to the vector $Y = (u, u')$), the bisection and the secant rule. Both root finders print nothing; they return the list of all the guesses they made, so that we can study how fast they approach the root.

In [2]:

```
import math  # sqrt, sin, pi for single numbers

import mpmath  # numbers with as many digits as we ask for
import numpy as np  # arrays (vectors) of numbers

# Colours that colour-blind readers can tell apart; curves also differ by style.
BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
GREY, BLACK = "#8a8986", "#000000"
LEVEL_COLORS = [BLUE, ORANGE, AQUA, VIOLET]  # one colour per bound state


def rk4_step(f, x, Y, h):
    """One classical RK4 step for the system Y' = f(x, Y)."""
    k1 = f(x, Y)
    k2 = f(x + h / 2, Y + (h / 2) * k1)
    k3 = f(x + h / 2, Y + (h / 2) * k2)
    k4 = f(x + h, Y + h * k3)
    return Y + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)


def bisection(F, lo, hi, steps):
    """steps bisection steps on the bracket (lo, hi); the list of midpoints."""
    f_lo = F(lo)
    guesses = []
    for _ in range(steps):
        mid = 0.5 * (lo + hi)
        f_mid = F(mid)
        guesses.append(mid)
        if (f_mid > 0) == (f_lo > 0):  # same sign as at lo: the root is right
            lo, f_lo = mid, f_mid
        else:  # opposite sign: the root lies between lo and mid
            hi = mid
    return guesses


def secant(F, x0, x1, steps):
    """steps secant steps from the guesses x0, x1; the list of new guesses."""
    f0, f1 = F(x0), F(x1)
    guesses = []
    for _ in range(steps):
        if f1 == f0:  # the line is flat: no new information (converged)
            break
        x2 = x1 - f1 * (x1 - x0) / (f1 - f0)  # the zero of the line
        x0, f0, x1, f1 = x1, f1, x2, F(x2)
        guesses.append(x2)
```

```
    return guesses


say("RK4 for systems, bisection and the secant rule are defined.")
```

Out [2]:

```
RK4 for systems, bisection and the secant rule are defined.
```

Text cell:

```
## 6. Part A: shooting the string

The function `shoot_string(lam, n)` solves $u'' = -\lambda u$ as the system
$u' = v$, $v' = -\lambda u$ from $x = 0$ with $u = 0$, $v = 1$, in $n$ RK4 steps,
and returns all the points. The first cell draws three shots: with
$\lambda = 5$ the string comes down too late, with $\lambda = 15$ too early, and
with $\lambda = \pi^2$ it lands exactly at $u(1) = 0$.
```

In [3]:

```
def shoot_string(lam, n):
    """RK4 solution of u'' = -lam u, u(0) = 0, u'(0) = 1 on 0 <= x <= 1 in n
    steps.  Returns the n + 1 positions and the n + 1 values of u."""
    def f(x, Y):
        return np.array([Y[1], -lam * Y[0]])  # (u', v') = (v, -lam u)
    h = 1.0 / n
    Y = np.array([0.0, 1.0])  # u(0) = 0, u'(0) = 1
    xs, us = [0.0], [0.0]
    for i in range(n):
        Y = rk4_step(f, i * h, Y, h)
        xs.append((i + 1) * h)
        us.append(Y[0])
    return np.array(xs), np.array(us)


def F_string(lam, n=200):
    """The shooting function: where the shot lands, u(1)."""
    return shoot_string(lam, n)[1][-1]


fig, ax = plt.subplots()
for lam, color, style in ((5.0, ORANGE, "--"), (math.pi ** 2, BLUE, "-"),
                          (15.0, AQUA, "-.")):
    xs, us = shoot_string(lam, 200)
    ax.plot(xs, us, style, color=color, lw=1.6,
            label=f"$\\lambda = {lam:.4f}$: $u(1) = {us[-1]:+.4f}$")
    ax.plot([1.0], [us[-1]], "o", color=color, ms=6)
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.plot([0.0, 1.0], [0.0, 0.0], "s", color=BLACK, ms=7, label="the two fixed ends")
ax.set_xlabel("position $x$ along the string")
```

```
ax.set_ylabel("displacement $u(x)$")
ax.set_title("Three shots for $u'' = -\\lambda u$, $u(0) = 0$, $u'(0) = 1$")
ax.legend(fontsize=8)
save_figure(fig, "trial_solutions_string",
            "Three shots for the string equation $d^2u/dx^2 = -\\lambda u$ started at "
            "$x = 0$ with $u = 0$ and slope 1, computed with RK4 (200 steps). "
            "Horizontal axis the position $x$ from 0 to 1, vertical axis the "
            "displacement $u$ (arbitrary units). The target is $u(1) = 0$ (black "
            "squares: the fixed ends). With $\\lambda = 5$ the shot lands above "
            "the target, with $\\lambda = 15$ below it, and with "
            "$\\lambda = \\pi^2 = 9.8696$ exactly on it: $\\pi^2$ is an "
            "eigenvalue.")
check(F_string(5.0) > 0 > F_string(15.0) and abs(F_string(math.pi ** 2)) < 1e-9,
      "u(1) > 0 for lambda = 5, < 0 for lambda = 15, = 0 for lambda = pi^2")
```

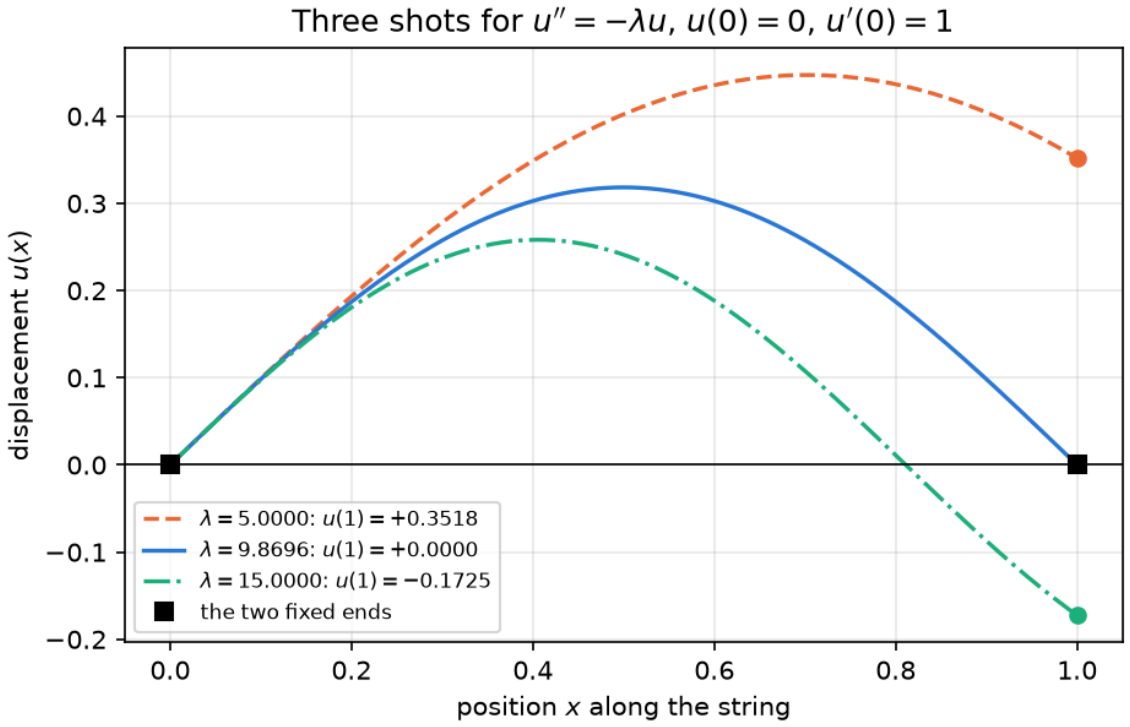

Figure 70: Three shots for the string equation $d^2u/dx^2 = -\lambda u$ started at $x = 0$ with $u = 0$ and slope 1, computed with RK4 (200 steps). Horizontal axis the position x from 0 to 1, vertical axis the displacement u (arbitrary units). The target is $u(1) = 0$ (black squares: the fixed ends). With $\lambda = 5$ the shot lands above the target, with $\lambda = 15$ below it, and with $\lambda = \pi^2 = 9.8696$ exactly on it: π^2 is an eigenvalue. (Notebook 02b, figure 1.)

Out [3]:

```
Figure 02b.1 saved as Revision/textbook/figures/02b_1_trial_solutions_string.png
PASS u(1) > 0 for lambda = 5, < 0 for lambda = 15, = 0 for lambda = pi^2
```

Text cell:

```
The next cell draws the whole shooting function $F(\lambda) = u(1; \lambda)$ for
$\lambda$ from 0.5 to 100: the RK4 values (points) and the exact
$\sin\sqrt{\lambda}/\sqrt{\lambda}$ (line). Its zeros are the eigenvalues
$\pi^2$, $4\pi^2$ and $9\pi^2$.
```

In [4]:

```
lam_points = np.linspace(0.5, 100.0, 60)  # 60 guesses for RK4
lam_fine = np.linspace(0.5, 100.0, 800)  # many points for the exact curve
F_points = np.array([F_string(lam) for lam in lam_points])
F_exact = np.sin(np.sqrt(lam_fine)) / np.sqrt(lam_fine)
fig, ax = plt.subplots()
ax.plot(lam_fine, F_exact, color=BLACK, lw=1.0,
        label="exact $\\sin\\sqrt{\\lambda}/\\sqrt{\\lambda}$")
ax.plot(lam_points, F_points, "o", color=BLUE, ms=4, label="RK4 shots, 200 steps")
for n in (1, 2, 3):
    ax.axvline(n ** 2 * math.pi ** 2, ls=":", color=GREY, lw=1.0)
    ax.text(n ** 2 * math.pi ** 2, 0.62, f"${n * n}\\pi^2$", ha="center",
            fontsize=9)
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_xlabel("guessed eigenvalue $\\lambda$")
ax.set_ylabel("$F(\\lambda) = u(1; \\lambda)$")
ax.set_ylim(-0.3, 0.7)
ax.set_title("The shooting function of the string")
ax.legend()
save_figure(fig, "shooting_function_string",
            "The shooting function $F(\\lambda) = u(1; \\lambda)$ of the string, "
            "for $\\lambda$ from 0.5 to 100 (both axes pure numbers): the "
            "values computed by RK4 with 200 steps (blue points) and the exact "
            "$\\sin\\sqrt{\\lambda}/\\sqrt{\\lambda}$ (black line). The zeros, "
            "marked by dotted lines, are the eigenvalues $\\pi^2 = 9.87$, "
            "$4\\pi^2 = 39.48$ and $9\\pi^2 = 88.83$; between two zeros $F$ "
            "keeps its sign, so every pair of guesses with opposite signs of $F$ "
            "brackets an eigenvalue.")
worst = np.max(np.abs(F_points - np.sin(np.sqrt(lam_points)) /
                      np.sqrt(lam_points)))
report("largest difference RK4 - exact shooting function", f"{worst:.2e}")
check(worst < 1e-7, "the RK4 shooting function equals sin(sqrt(lam))/sqrt(lam)")
```

Out [4]:

```
Figure 02b.2 saved as Revision/textbook/figures/02b_2_shooting_function_string.png
RESULT largest difference RK4 - exact shooting function = 4.54e-08
PASS the RK4 shooting function equals sin(sqrt(lam))/sqrt(lam)
```

Text cell:

```
## 7. Bisection against the secant rule

Both start from the bracket $5 < \lambda < 15$ (where $F(5) > 0 > F(15)$). The
next cell prints the first six bisection steps (each one gains one binary digit)
```

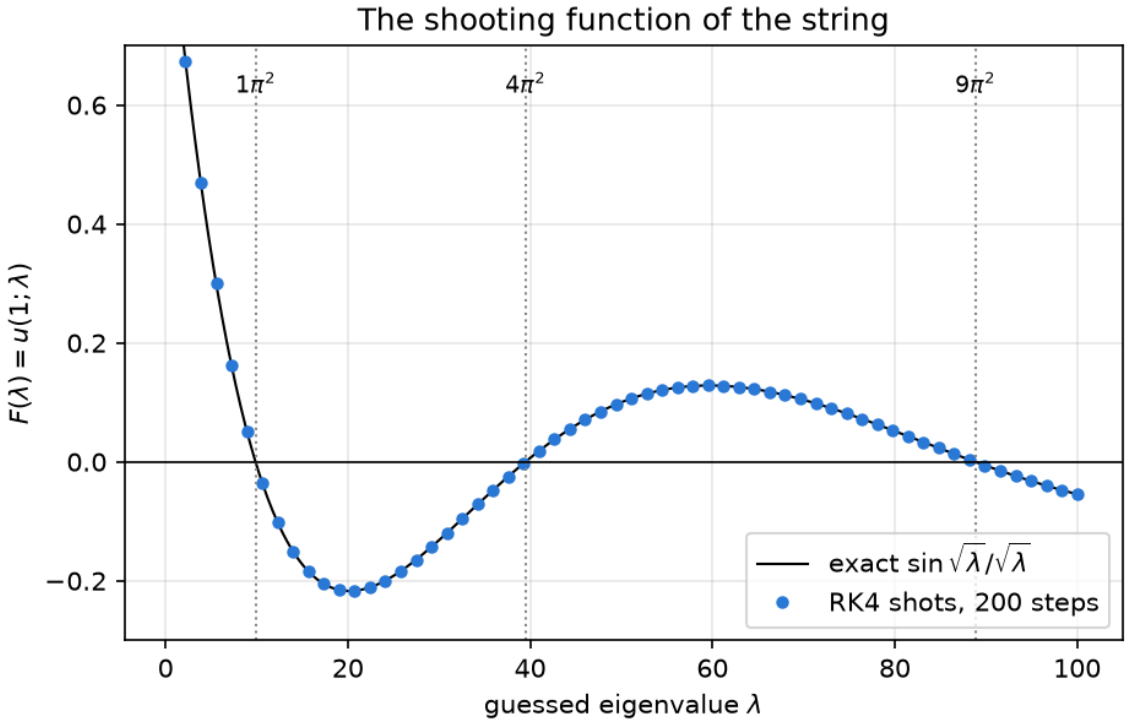

Figure 71: The shooting function $F(\lambda) = u(1;\lambda)$ of the string, for λ from 0.5 to 100 (both axes pure numbers): the values computed by RK4 with 200 steps (blue points) and the exact $\sin\sqrt{\lambda}/\sqrt{\lambda}$ (black line). The zeros, marked by dotted lines, are the eigenvalues $\pi^2 = 9.87$, $4\pi^2 = 39.48$ and $9\pi^2 = 88.83$; between two zeros F keeps its sign, so every pair of guesses with opposite signs of F brackets an eigenvalue. (Notebook 02b, figure 2.)

```
and the first six secant steps (each one roughly multiplies the number of
correct digits by 1.6), then runs both further and draws the error
$|\lambda_k - \pi^2|$ after each step on a logarithmic axis.
```

In [5]:

```
PI2 = math.pi ** 2  # the exact eigenvalue
bis = bisection(F_string, 5.0, 15.0, 45)
sec = secant(F_string, 5.0, 15.0, 12)
say("step   bisection midpoint   F(midpoint)    secant guess")
for k in range(6):
    say(f"{k + 1:4d}   {bis[k]:18.6f}   {F_string(bis[k]):+11.5f}   {sec[k]:13.7f}")
lam_rk4 = sec[-1]  # the converged secant value: the RK4 eigenvalue (200 steps)
report("eigenvalue by RK4 shooting (200 steps)", f"{lam_rk4:.12f}")
report("pi^2", f"{PI2:.12f}")
errors_bis = np.abs(np.array(bis) - lam_rk4)
errors_sec = np.abs(np.array(sec) - lam_rk4)
fig, ax = plt.subplots()
ax.semilogy(range(1, len(bis) + 1), np.maximum(errors_bis, 1e-17), "s-",
            color=ORANGE, ms=4, lw=1.2, label="bisection")
ax.semilogy(range(1, len(sec) + 1), np.maximum(errors_sec, 1e-17), "o-",
            color=BLUE, ms=5, lw=1.2, label="secant rule")
k_values = np.arange(1, len(bis) + 1)
ax.semilogy(k_values, 10.0 / 2.0 ** k_values, ":", color=GREY,
            label="bracket width $10/2^k$")
ax.set_xlabel("step $k$")
ax.set_ylabel("error $|\\lambda_k - \\lambda_1|$")
ax.set_ylim(1e-17, 10.0)
ax.set_title("How fast the two root finders approach the eigenvalue $\\lambda_1$")
ax.legend()
save_figure(fig, "bisection_vs_secant",
            "The error of the guess after $k$ steps, $|\\lambda_k - \\lambda_1|$, "
            "where $\\lambda_1$ is the zero of the RK4 shooting function (200 "
            "steps), which differs from $\\pi^2$ by $1.0 \\times 10^{-8}$, "
            "for bisection (squares) and the secant rule (circles), both started "
            "from 5 and 15, on a logarithmic vertical axis (pure numbers); errors "
            "of exactly zero are drawn at $10^{-17}$. Bisection follows the "
            "dotted line of the halving bracket, one binary digit per step, and "
            "after 45 steps it is still about $10^{-13}$ away (the bracket is "
            "$10/2^{45} = 2.8 \\times 10^{-13}$ wide); the secant rule reaches the "
            "rounding level in about 9 steps, its error falling faster and faster.")
check(bis[:6] == [10.0, 7.5, 8.75, 9.375, 9.6875, 9.84375],
      "the bisection midpoints 10, 7.5, 8.75, 9.375, 9.6875, 9.84375")
check(abs(sec[5] - PI2) < 1e-6 and abs(bis[5] - PI2) > 0.02,
      "after 6 steps: secant within 1e-6 of pi^2, bisection still 0.02 away")
check(abs(lam_rk4 - PI2) < 2e-8 and abs(bis[-1] - lam_rk4) < 1e-11,
      "RK4 eigenvalue = pi^2 within 2e-8; 45 bisections agree within 1e-11")
```

Out [5]:

```
step   bisection midpoint   F(midpoint)    secant guess
   1            10.000000      -0.00654      11.7107898
```

```
   2             7.500000      +0.14320       8.8043277
   3             8.750000      +0.06170      10.0237185
   4             9.375000      +0.02601       9.8815597
   5             9.687500      +0.00935       9.8694633
   6             9.843750      +0.00131       9.8696045
RESULT eigenvalue by RK4 shooting (200 steps) = 9.869604411103
RESULT pi^2 = 9.869604401089
```

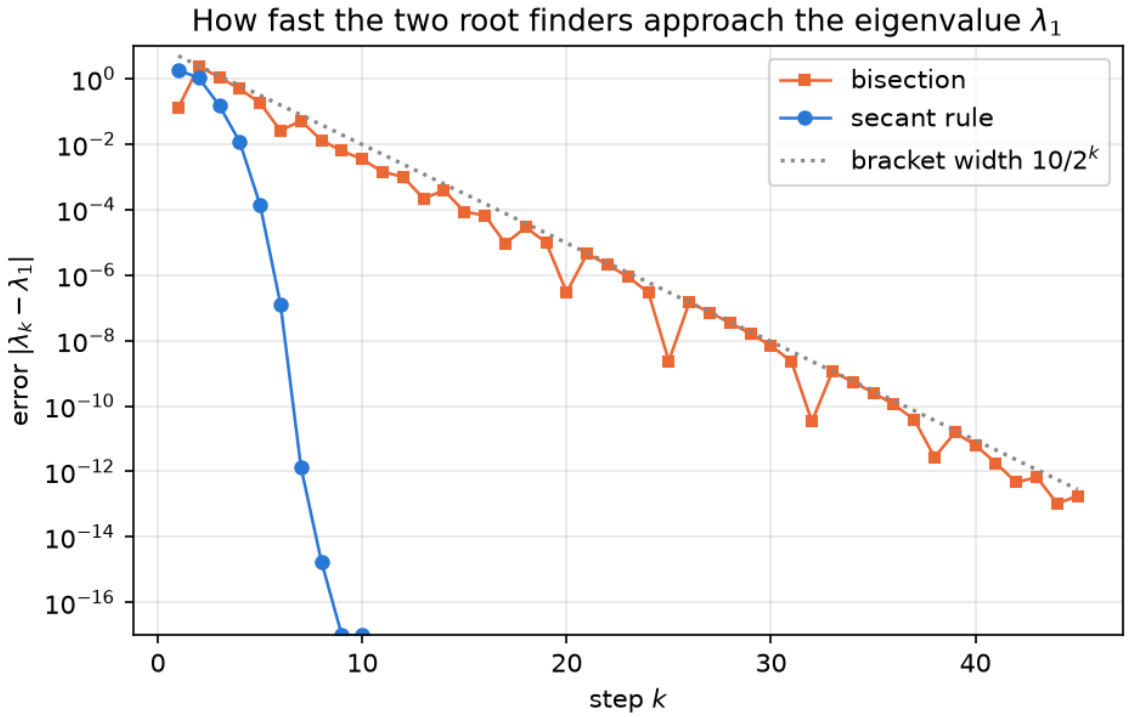

Figure 72: The error of the guess after k steps, $|\lambda_k - \lambda_1|$, where λ_1 is the zero of the RK4 shooting function (200 steps), which differs from π^2 by 1.0×10^{-8}, for bisection (squares) and the secant rule (circles), both started from 5 and 15, on a logarithmic vertical axis (pure numbers); errors of exactly zero are drawn at 10^{-17}. Bisection follows the dotted line of the halving bracket, one binary digit per step, and after 45 steps it is still about 10^{-13} away (the bracket is $10/2^{45} = 2.8 \times 10^{-13}$ wide); the secant rule reaches the rounding level in about 9 steps, its error falling faster and faster. (Notebook 02b, figure 3.)

Out [5]:

```
Figure 02b.3 saved as Revision/textbook/figures/02b_3_bisection_vs_secant.png
PASS the bisection midpoints 10, 7.5, 8.75, 9.375, 9.6875, 9.84375
PASS after 6 steps: secant within 1e-6 of pi^2, bisection still 0.02 away
PASS RK4 eigenvalue = pi^2 within 2e-8; 45 bisections agree within 1e-11
```

Text cell:

```
## 8. Part B: the finite square well, solved exactly with mpmath
```

```
The next cell solves the even and odd equations of section 4 with mpmath at 30
significant digits. To avoid the infinities of $\tan$ and $\cot$ it multiplies
them out: even $g(z) = z \sin z - \sqrt{z_0^2 - z^2}\cos z = 0$, odd
$g(z) = z \cos z + \sqrt{z_0^2 - z^2}\sin z = 0$. Even roots lie between $n\pi$
and $n\pi + \pi/2$, odd ones between $n\pi + \pi/2$ and $(n + 1)\pi$ (but never
beyond $z_0$); mpmath's 'findroot' with the method "anderson" refines each
bracket. The figure shows the classic graphical solution: the curves $\tan z$
and $-\cot z$ cross the curve $\sqrt{z_0^2 - z^2}/z$ at the solutions.
```

In [6]:

```
V0, A = 15.0, 1.0  # the depth and the half-width of the well
mpmath.mp.dps = 30  # work with 30 significant digits
Z0 = mpmath.sqrt(2 * mpmath.mpf(V0)) * A  # z0 = a sqrt(2 V0) = sqrt(30)


def g_even(z):
    return z * mpmath.sin(z) - mpmath.sqrt(Z0 ** 2 - z ** 2) * mpmath.cos(z)


def g_odd(z):
    return z * mpmath.cos(z) + mpmath.sqrt(Z0 ** 2 - z ** 2) * mpmath.sin(z)


EXACT = []  # (energy, parity) of the bound states, lowest first
for j in range(8):  # the intervals of length pi/2 between 0 and z0
    lo = j * mpmath.pi / 2
    hi = min((j + 1) * mpmath.pi / 2, Z0)
    if lo >= Z0:
        break
    g, parity = (g_even, "even") if j % 2 == 0 else (g_odd, "odd")
    # the root inside the interval (lo, hi), searched a hair away from its ends
    z = mpmath.findroot(g, (lo + mpmath.mpf("1e-20"), hi - mpmath.mpf("1e-20")),
                        solver="anderson")
    EXACT.append((z ** 2 / (2 * A ** 2) - V0, parity, z))
for n, (energy, parity, z) in enumerate(EXACT):
    say(f"state {n} ({parity:4}): z = {mpmath.nstr(z, 20)}, "
        f"E = {mpmath.nstr(energy, 20)}")
check(len(EXACT) == 4 and [p for _, p, _ in EXACT] == ["even", "odd", "even", "odd"],
      "the well has 4 bound states: even, odd, even, odd")

z_axis = np.linspace(0.01, float(Z0), 2000)
rhs = np.sqrt(float(Z0) ** 2 - z_axis ** 2) / z_axis
tan_z = np.tan(z_axis)
cot_z = -1.0 / np.tan(z_axis)
tan_z[np.abs(tan_z) > 12] = np.nan  # do not draw the jumps at the poles
cot_z[np.abs(cot_z) > 12] = np.nan
fig, ax = plt.subplots()
ax.plot(z_axis, rhs, color=BLACK, lw=1.5, label="$\\sqrt{z_0^2 - z^2}/z$")
ax.plot(z_axis, tan_z, color=BLUE, lw=1.2, label="$\\tan z$ (even states)")
ax.plot(z_axis, cot_z, "--", color=ORANGE, lw=1.2, label="$-\\cot z$ (odd states)")
for n, (energy, parity, z) in enumerate(EXACT):
    zf = float(z)
```

```
    ax.plot([zf], [math.sqrt(float(Z0) ** 2 - zf ** 2) / zf], "o",
            color=LEVEL_COLORS[n], ms=7, zorder=5)
    ax.annotate(f"$n = {n}$", (zf, math.sqrt(float(Z0) ** 2 - zf ** 2) / zf),
                textcoords="offset points", xytext=(5, 8), fontsize=9)
ax.axvline(float(Z0), color=GREY, ls=":", lw=1.0)
ax.text(float(Z0), 7.3, "$z_0 = \\sqrt{30}$", ha="right", fontsize=9)
ax.set_ylim(0.0, 8.0)
ax.set_xlabel("$z = k a$")
ax.set_ylabel("value of each side of the equation")
ax.set_title("Graphical solution of the finite-well equations, $z_0 = 5.477$")
ax.legend(loc="upper center", fontsize=8)
save_figure(fig, "graphical_solution",
            "The graphical solution of the finite-well equations for "
            "$z_0 = a\\sqrt{2V_0} = \\sqrt{30} = 5.477$: horizontal axis "
            "$z = ka$ (a pure number), vertical axis the value of each side. The "
            "black curve is $\\sqrt{z_0^2 - z^2}/z$, the blue branches $\\tan z$ "
            "(even states), the dashed orange branches $-\\cot z$ (odd states). "
            "Each crossing, marked by a dot, is one bound state; there are four, "
            "alternately even and odd, and none beyond $z_0$, where the black "
            "curve reaches zero.")
```

Out [6]:

```
state 0 (even): z = 1.3262301881908503282, E = -14.120556743965630861
state 1 (odd ): z = 2.6388922601494099741, E = -11.518123819661769376
state 2 (even): z = 3.9158583447044145113, E = -7.3330267121044013902
state 3 (odd ): z = 5.0904451651058404757, E = -2.0436840105252862495
PASS the well has 4 bound states: even, odd, even, odd
```

Out [6]:

```
Figure 02b.4 saved as Revision/textbook/figures/02b_4_graphical_solution.png
```

Text cell:

```
## 9. Part B by shooting with RK4

The function ‘shoot_well(E, n)‘ starts at $x = -a$ with $u = 1$, $u’ = \kappa$
and makes $n$ RK4 steps to $x = 0$ with the inside equation $u’’ = -2(V_0 + E)
u$. It works for one energy or for a whole array of energies at once (numpy
computes elementwise), which makes the scan of the shooting functions fast. The
next cell draws $F_{\rm even}(E) = u’(0)$ and $F_{\rm odd}(E) = u(0)$ for 400
energies between $-V_0$ and 0 and marks the exact energies.
```

In [7]:

```
def shoot_well(E, n):
    """RK4 from x = -a (u = 1, u’ = kappa) to x = 0 inside the well; E may be an
    array.  Returns (u(0), u’(0))."""
```

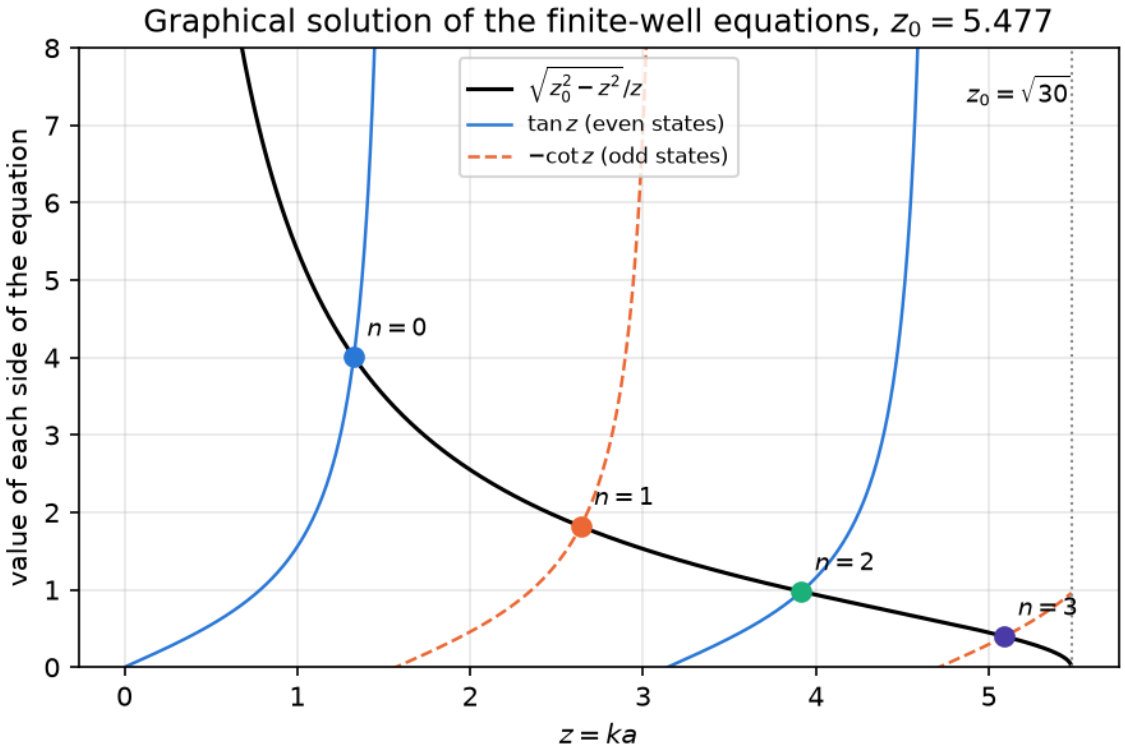

Figure 73: The graphical solution of the finite-well equations for $z_0 = a\sqrt{2V_0} = \sqrt{30} = 5.477$: horizontal axis $z = ka$ (a pure number), vertical axis the value of each side. The black curve is $\sqrt{z_0^2 - z^2}/z$, the blue branches $\tan z$ (even states), the dashed orange branches $-\cot z$ (odd states). Each crossing, marked by a dot, is one bound state; there are four, alternately even and odd, and none beyond z_0, where the black curve reaches zero. (Notebook 02b, figure 4.)

```
    E = np.asarray(E, dtype=float)
    kappa = np.sqrt(-2.0 * E)  # the decay rate outside the well

    def f(x, Y):
        return np.array([Y[1], 2.0 * (-V0 - E) * Y[0]])  # (u', 2 (V - E) u)
    h = A / n
    Y = np.array([np.ones_like(E), kappa])
    for i in range(n):
        Y = rk4_step(f, -A + i * h, Y, h)
    return Y[0], Y[1]


def F_even(E, n=200):
    return shoot_well(E, n)[1]  # u'(0): zero for an even state


def F_odd(E, n=200):
    return shoot_well(E, n)[0]  # u(0): zero for an odd state


E_scan = np.linspace(-V0 + 1e-3, -1e-3, 400)  # 400 guesses inside (-V0, 0)
fe, fo = F_even(E_scan), F_odd(E_scan)
fig, ax = plt.subplots()
ax.plot(E_scan, fe, color=BLUE, lw=1.4, label="$F_{\\rm even}(E) = u'(0)$")
ax.plot(E_scan, fo, "--", color=ORANGE, lw=1.4, label="$F_{\\rm odd}(E) = u(0)$")
for n, (energy, parity, _) in enumerate(EXACT):
    ax.axvline(float(energy), ls=":", color=GREY, lw=1.0)
    ax.plot([float(energy)], [0.0], "o", color=LEVEL_COLORS[n], ms=7, zorder=5)
    ax.text(float(energy), 4.6, f"$E_{n}$", ha="center", fontsize=9)
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_ylim(-5.5, 5.5)
ax.set_xlabel("guessed energy $E$ (units $\\hbar^2/(m a^2)$)")
ax.set_ylabel("mismatch at $x = 0$")
ax.set_title("Shooting functions of the finite well ($V_0 = 15$, $a = 1$)")
ax.legend(loc="lower left")
save_figure(fig, "shooting_functions_well",
            "The shooting functions of the finite well: $F_{\\rm even}(E) = "
            "u'(0)$ (solid blue) and $F_{\\rm odd}(E) = u(0)$ (dashed orange) for "
            "400 guessed energies between $-V_0 = -15$ and $0$ (horizontal axis, "
            "units $\\hbar^2/(m a^2)$; vertical axis the mismatch, arbitrary "
            "units), each computed by shooting with RK4 from $x = -a$, where "
            "$u = 1$ and $u' = \\kappa$. The dots on the axis are the exact "
            "energies from the transcendental equations: the zeros of "
            "$F_{\\rm even}$ give the even states $E_0, E_2$, those of "
            "$F_{\\rm odd}$ the odd states $E_1, E_3$.")
even_changes = int(np.sum(np.sign(fe[:-1]) != np.sign(fe[1:])))
odd_changes = int(np.sum(np.sign(fo[:-1]) != np.sign(fo[1:])))
report("sign changes of F_even and F_odd in the scan", f"{even_changes}, {odd_changes}")
check(even_changes == 2 and odd_changes == 2,
      "the scan finds 2 even and 2 odd sign changes, one per bound state")
```

Out [7]:

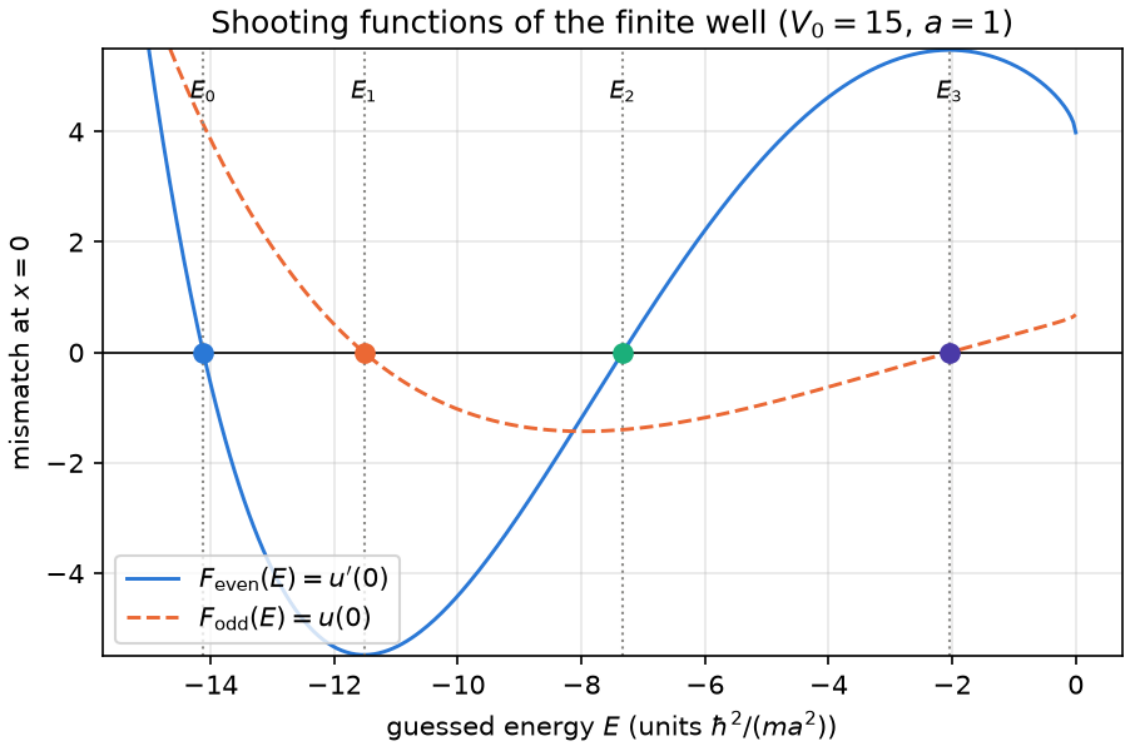

Figure 74: The shooting functions of the finite well: $F_{\text{even}}(E) = u'(0)$ (solid blue) and $F_{\text{odd}}(E) = u(0)$ (dashed orange) for 400 guessed energies between $-V_0 = -15$ and 0 (horizontal axis, units $\hbar^2/(ma^2)$; vertical axis the mismatch, arbitrary units), each computed by shooting with RK4 from $x = -a$, where $u = 1$ and $u' = \kappa$. The dots on the axis are the exact energies from the transcendental equations: the zeros of F_{even} give the even states E_0, E_2, those of F_{odd} the odd states E_1, E_3. (Notebook 02b, figure 5.)

```
Figure 02b.5 saved as Revision/textbook/figures/02b_5_shooting_functions_well.png
RESULT sign changes of F_even and F_odd in the scan = 2, 2
PASS the scan finds 2 even and 2 odd sign changes, one per bound state
```

Text cell:

```
The next cell turns every sign change of a scan (made with the same number of
RK4 steps as the refinement) into a bracket and refines all four brackets AT
ONCE by bisection (each step evaluates both shooting functions
for the four midpoints as one numpy array), until the brackets are narrower
than $10^{-13}$. Then it compares the four energies with the 30-digit exact
values.
```

In [8]:

```
def well_levels(n, tolerance=1e-13):
    """The four energies with n RK4 steps: scan, then bisection on the brackets."""
    lo, hi, even = [], [], []
    for values, is_even in ((F_even(E_scan, n), True), (F_odd(E_scan, n), False)):
        # the places i where the sign of the mismatch differs from that at i + 1
        for i in np.nonzero(np.sign(values[:-1]) != np.sign(values[1:]))[0]:
            lo.append(E_scan[i])
            hi.append(E_scan[i + 1])
            even.append(is_even)
    order = np.argsort(lo)  # lowest energy first
    lo, hi = np.array(lo)[order], np.array(hi)[order]
    even = np.array(even)[order]

    def mismatch(E):
        u0, du0 = shoot_well(E, n)
        return np.where(even, du0, u0)  # u'(0) for even, u(0) for odd states
    f_lo = mismatch(lo)
    while np.max(hi - lo) > tolerance:
        mid = 0.5 * (lo + hi)
        f_mid = mismatch(mid)
        same = np.sign(f_mid) == np.sign(f_lo)  # the root is above mid
        lo, f_lo = np.where(same, mid, lo), np.where(same, f_mid, f_lo)
        hi = np.where(same, hi, mid)
    return 0.5 * (lo + hi), even


LEVELS, EVEN = well_levels(200)
exact = np.array([float(energy) for energy, _, _ in EXACT])
for n, (E_n, E_x) in enumerate(zip(LEVELS, exact)):
    say(f"E_{n}: RK4 (200 steps) {E_n:.12f}, exact {E_x:.12f}, "
        f"difference {E_n - E_x:+.1e}")
check(np.max(np.abs(LEVELS - exact)) < 1e-7,
      "the four shooting energies agree with the exact ones within 1e-7")
check(list(EVEN) == [True, False, True, False],
      "the levels alternate: even, odd, even, odd")
```

Out [8]:

```
E_0: RK4 (200 steps) -14.120556743942, exact -14.120556743966, difference +2.4e-11
E_1: RK4 (200 steps) -11.518123818206, exact -11.518123819662, difference +1.5e-09
E_2: RK4 (200 steps) -7.333026697216, exact -7.333026712104, difference +1.5e-08
E_3: RK4 (200 steps) -2.043683949908, exact -2.043684010525, difference +6.1e-08
PASS the four shooting energies agree with the exact ones within 1e-7
PASS the levels alternate: even, odd, even, odd
```

Text cell:

```
## 10. What a shot looks like when it misses

For the ground state $E_0$, the next cell shoots across the WHOLE line from
$x = -3$ to $x = +3$ (600 RK4 steps of $h = 0.01$, so that the walls $x = \pm 1$
are grid points and the potential is constant inside every step). It starts
with the exactly decaying solution $u = e^{\kappa(x + a)}$, $u' = \kappa u$ at
$x = -3$. Only at the eigenvalue does the shot come down to zero on the right;
a little below or above $E_0$, the growing solution $e^{\kappa x}$ takes over
and the shot flies off to $+\infty$ or $-\infty$. That is the reason why only
special energies are allowed.
```

In [9]:

```
def shoot_line(E, x_start=-3.0, x_end=3.0, n=600):
    """RK4 across the whole line; the potential is taken at the middle of every
    step (constant inside a step, because the walls are grid points)."""
    kappa = math.sqrt(-2.0 * E)
    h = (x_end - x_start) / n
    Y = np.array([1.0, kappa]) * math.exp(kappa * (x_start + A))  # exact tail
    xs, us = [x_start], [Y[0]]
    for i in range(n):
        x = x_start + i * h
        V = -V0 if abs(x + h / 2) < A else 0.0  # the potential of this step

        def f(x_value, Y_value, V=V):  # V=V freezes this step's potential in f
            return np.array([Y_value[1], 2.0 * (V - E) * Y_value[0]])
        Y = rk4_step(f, x, Y, h)
        xs.append(x + h)
        us.append(Y[0])
    return np.array(xs), np.array(us)


E0 = float(EXACT[0][0])
fig, ax = plt.subplots()
for shift, color, style in ((-0.05, ORANGE, "--"), (0.0, BLUE, "-"),
                            (0.05, AQUA, "-.")):
    xs, us = shoot_line(E0 + shift)
    ax.plot(xs, us / np.max(np.abs(us[xs <= 0])), style, color=color, lw=1.6,
            label=f"$E = E_0 {shift:+.2f}$")
ax.axvspan(-A, A, color=GREY, alpha=0.15, label="inside the well")
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_ylim(-2.0, 2.0)
ax.set_xlabel("position $x$ (units $a$)")
```

```
ax.set_ylabel("$u(x)$ (scaled to 1 at the left peak)")
ax.set_title("Shots across the well near the ground state $E_0 = %.4f$" % E0)
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "trial_solutions_well",
            "Shots across the whole line from $x = -3$ to $x = 3$ (horizontal "
            "axis, units of the half-width $a$; the shaded band is the well) for "
            f"the ground-state energy $E_0 = {E0:.4f}$ and for $E_0 \\pm 0.05$; "
            "vertical axis the solution $u$, scaled to 1 at its largest value "
            "left of the centre (arbitrary units). Every shot starts as the "
            "decaying solution on the left. At $E_0$ (solid blue) it decays on the "
            "right as well: a bound state. Slightly below (dashed orange) or above "
            "(dash-dotted aqua), the growing exponential takes over and the shot "
            "flies off to plus or minus infinity.")
tail = {shift: shoot_line(E0 + shift)[1][-1] for shift in (-0.05, 0.05)}
check(tail[-0.05] * tail[0.05] < 0 and min(abs(tail[-0.05]), abs(tail[0.05])) > 10,
      "slightly below and above E0 the shots fly off to opposite infinities")
```

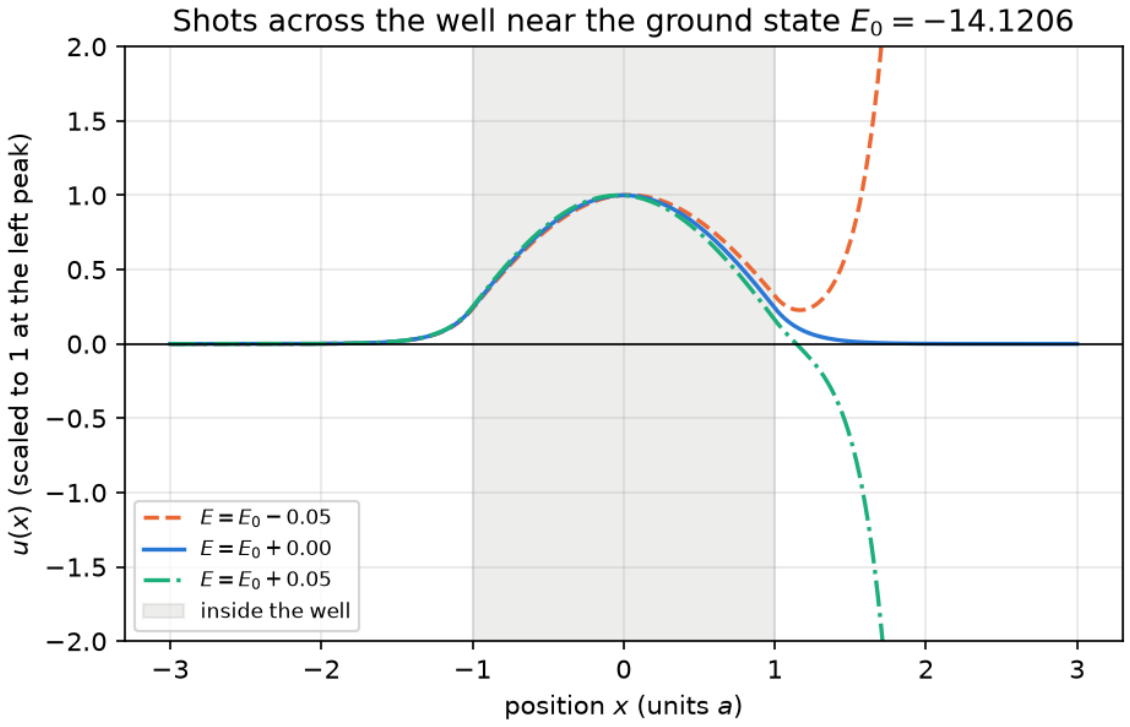

Figure 75: Shots across the whole line from $x = -3$ to $x = 3$ (horizontal axis, units of the half-width a; the shaded band is the well) for the ground-state energy $E_0 = -14.1206$ and for $E_0 \pm 0.05$; vertical axis the solution u, scaled to 1 at its largest value left of the centre (arbitrary units). Every shot starts as the decaying solution on the left. At E_0 (solid blue) it decays on the right as well: a bound state. Slightly below (dashed orange) or above (dash-dotted aqua), the growing exponential takes over and the shot flies off to plus or minus infinity. (Notebook 02b, figure 6.)

Out [9]:

```
Figure 02b.6 saved as Revision/textbook/figures/02b_6_trial_solutions_well.png
PASS slightly below and above E0 the shots fly off to opposite infinities
```

Text cell:

```
## 11. The four wave functions

The next cell builds each normalised wave function on $-3 \le x \le 3$: inside
the left half of the well from the RK4 shot (200 steps), on the right half by
the symmetry $u(-x) = u(x)$ (even) or $u(-x) = -u(x)$ (odd), and outside from
the exact tails $u(-a) e^{\kappa(x + a)}$. The normalisation integral uses
Simpson's rule inside (an integration rule with error $\propto h^4$) and the
exact tail integral $\int_{-\infty}^{-a} e^{2\kappa(x + a)} dx = 1/(2\kappa)$.
It draws each wave function lifted to the height of its energy, inside the
drawing of the potential, counts the nodes, and checks the normalisation and
the orthogonality. For an even and an odd state the product $u_m u_n$ is odd
(it changes sign when $x$ is replaced by $-x$), so the contributions of the
left and the right half cancel and its integral is exactly zero; for two states
of the same parity the cell computes the integral (twice the left half: Simpson
inside, the exact tail $u_m(-a) u_n(-a)/(\kappa_m + \kappa_n)$ outside).
Finally the cell tests the cancellation for the four pairs of different parity
numerically: it integrates $u_m u_n$ with Simpson's rule over the whole drawn
range $-3 \le x \le 3$, and over the left half $-3 \le x \le 0$ alone. The
whole integral must vanish, although the half does not.
```

In [10]:

```
def simpson(values, h):
    """Simpson's rule for equally spaced values (an even number of intervals)."""
    return h / 3 * (values[0] + values[-1] + 4 * values[1:-1:2].sum()
                    + 2 * values[2:-1:2].sum())


def wave_function(E, is_even, n=200):
    """Normalised u on a grid of -3 <= x <= 3 (step a/n), built as described."""
    kappa = math.sqrt(-2.0 * E)
    h = A / n

    def f(x, Y):
        return np.array([Y[1], 2.0 * (-V0 - E) * Y[0]])
    Y = np.array([1.0, kappa])
    inside = [1.0]
    for i in range(n):
        Y = rk4_step(f, -A + i * h, Y, h)
        inside.append(Y[0])
    inside = np.array(inside)  # u on -a <= x <= 0
    norm = 2.0 * (simpson(inside ** 2, h) + 1.0 / (2.0 * kappa))  # both halves
    x_left = np.arange(-3.0 * n, -n) * h  # -3 <= x < -a
    left = np.concatenate([np.exp(kappa * (x_left + A)), inside])
    x_half = np.concatenate([x_left, -A + np.arange(n + 1) * h])  # up to x = 0
    sign = 1.0 if is_even else -1.0
    x_all = np.concatenate([x_half, -x_half[-2::-1]])  # mirror x -> -x
    u_all = np.concatenate([left, sign * left[-2::-1]])
    return x_all, u_all / math.sqrt(norm), kappa, inside / math.sqrt(norm), h
```

```
STATES = [wave_function(E_n, bool(e)) for E_n, e in zip(LEVELS, EVEN)]
fig, ax = plt.subplots(figsize=(7.0, 5.6))
x_pot = np.array([-3.0, -A, -A, A, A, 3.0])
ax.plot(x_pot, [0.0, 0.0, -V0, -V0, 0.0, 0.0], color=BLACK, lw=1.5,
        label="potential $V(x)$")
nodes = []
for n, (x_all, u_all, kappa, inside, h) in enumerate(STATES):
    E_n = LEVELS[n]
    ax.axhline(E_n, ls=":", color=GREY, lw=0.8)
    parity = "even" if EVEN[n] else "odd"
    ax.plot(x_all, E_n + 1.8 * u_all, color=LEVEL_COLORS[n], lw=1.6,
            label=f"$E_{n} = {E_n:.4f}$ ({parity})")
    big = u_all[np.abs(u_all) > 1e-8 * np.max(np.abs(u_all))]  # drop tiny values
    nodes.append(int(np.sum(np.sign(big[:-1]) != np.sign(big[1:]))))
ax.set_xlabel("position $x$ (units $a$)")
ax.set_ylabel("energy (units $\\hbar^2/(m a^2)$) and $E_n + 1.8\\, u_n(x)$")
ax.set_ylim(-16.0, 5.0)  # room for the legend above the well
ax.set_title("The four bound states of the finite square well")
ax.legend(loc="upper center", ncol=2, fontsize=8)
save_figure(fig, "eigenfunctions",
            "The four bound states of the finite square well of depth 15 and "
            "half-width 1 (black: the potential $V(x)$). Horizontal axis the "
            "position $x$ in units of $a$; vertical axis the energy in units of "
            "$\\hbar^2/(m a^2)$. Each normalised wave function $u_n$ is drawn "
            "as $E_n + 1.8\\, u_n(x)$, so that it sits on its energy level "
            "(dotted line). The state $n$ has $n$ nodes; even and odd states "
            "alternate; the higher the energy, the further the wave function "
            "leaks out of the well, where it decays like $e^{-\\kappa |x|}$.")
report("nodes of the states 0 to 3", nodes)
check(nodes == [0, 1, 2, 3], "the state n has exactly n nodes")


def overlap(m, n):
    """The integral of u_m u_n over the whole line: Simpson inside, exact tails."""
    _, _, k_m, in_m, h = STATES[m]
    _, _, k_n, in_n, _ = STATES[n]
    half = simpson(in_m * in_n, h) + in_m[0] * in_n[0] / (k_m + k_n)
    same = (EVEN[m] == EVEN[n])  # different parity: the halves cancel exactly
    return 2.0 * half if same else 0.0


gram = np.array([[overlap(m, n) for n in range(4)] for m in range(4)])
report("largest |integral u_m u_n - (1 if m = n else 0)|",
       f"{np.max(np.abs(gram - np.eye(4))):.1e}")
check(np.max(np.abs(gram - np.eye(4))) < 1e-8,
      "the wave functions are normalised and orthogonal within 1e-8")

middle = len(STATES[0][0]) // 2  # the position of x = 0 in the grid -3 ... 3
whole_line, left_half = [], []
for m, n in [(0, 1), (0, 3), (1, 2), (2, 3)]:  # the pairs of different parity
    product = STATES[m][1] * STATES[n][1]  # u_m u_n on the grid
    h_grid = STATES[m][4]  # the grid spacing a/200
    whole_line.append(abs(simpson(product, h_grid)))  # from x = -3 to x = 3
```

```
    left_half.append(abs(simpson(product[:middle + 1], h_grid)))  # -3 to 0
report("different parity: largest |whole integral|, smallest |left half|",
       f"{max(whole_line):.1e}, {min(left_half):.2f}")
check(max(whole_line) < 1e-12 < 0.01 < min(left_half),
      "different parity: the two halves cancel, the whole integral is 0")
```

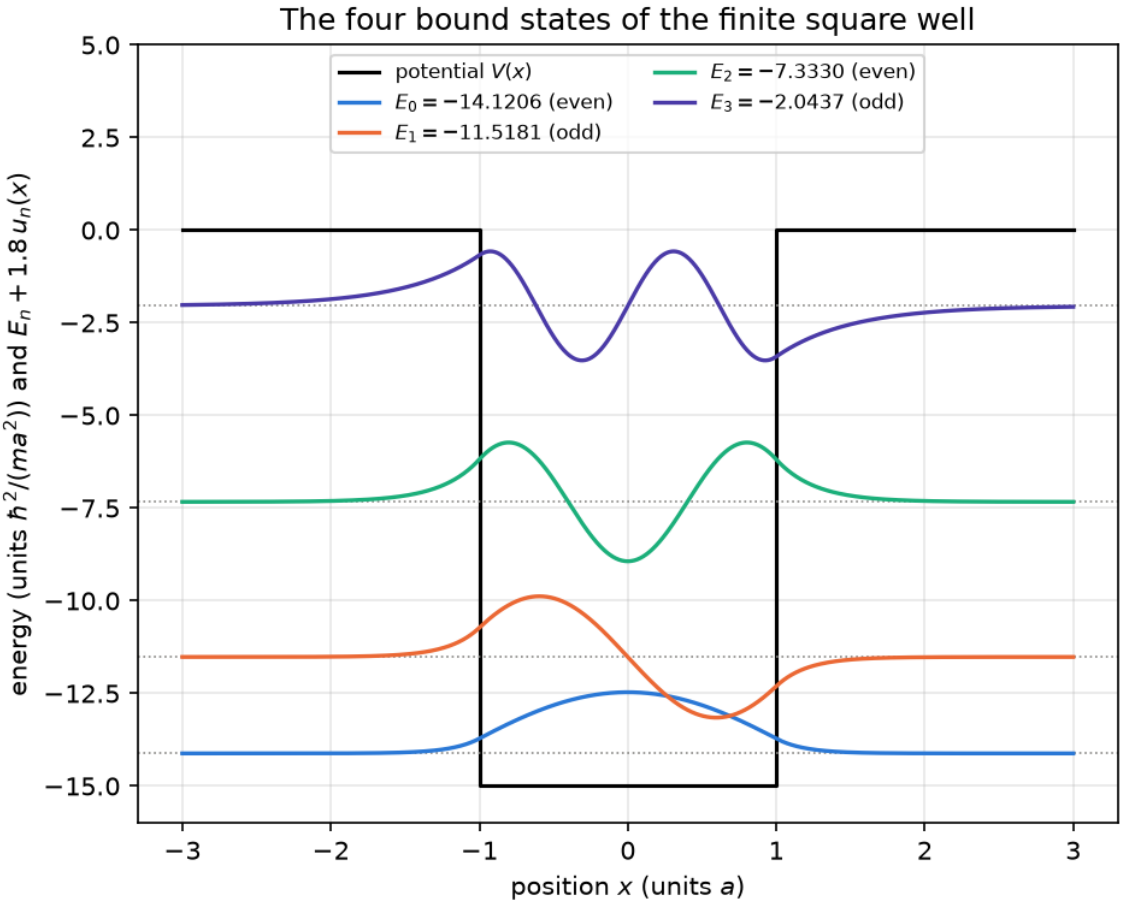

Figure 76: The four bound states of the finite square well of depth 15 and half-width 1 (black: the potential $V(x)$). Horizontal axis the position x in units of a; vertical axis the energy in units of $\hbar^2/(ma^2)$. Each normalised wave function u_n is drawn as $E_n + 1.8\,u_n(x)$, so that it sits on its energy level (dotted line). The state n has n nodes; even and odd states alternate; the higher the energy, the further the wave function leaks out of the well, where it decays like $e^{-\kappa|x|}$. (Notebook 02b, figure 7.)

Out [10]:

```
Figure 02b.7 saved as Revision/textbook/figures/02b_7_eigenfunctions.png
RESULT nodes of the states 0 to 3 = [0, 1, 2, 3]
PASS the state n has exactly n nodes
RESULT largest |integral u_m u_n - (1 if m = n else 0)| = 1.6e-10
PASS the wave functions are normalised and orthogonal within 1e-8
RESULT different parity: largest |whole integral|, smallest |left half| = 9.5e-17, 0.16
PASS different parity: the two halves cancel, the whole integral is 0
```

Text cell:

```
## 12. The order of the computed energies

The shooting energy inherits the error of the integrator. The next cell repeats
the whole level search with $n = 4, 8, \dots, 1024$ RK4 steps on $-a \le x \le 0$
and draws $|E_n(\text{RK4}) - E_n(\text{exact})|$ against the step size
$h = a/n$ on a log-log plot. The points lie on lines of slope 4: the energies
converge with the order of RK4. The higher states have larger errors, because
their wave functions oscillate faster (the error grows like $(k h)^4$).
```

In [11]:

```
N_STEPS = [4, 8, 16, 32, 64, 128, 256, 512, 1024]
level_errors = np.array([np.abs(well_levels(n)[0] - exact) for n in N_STEPS])
h_values = A / np.array(N_STEPS, dtype=float)
fig, ax = plt.subplots()
slopes = []
for n in range(4):
    ax.loglog(h_values, level_errors[:, n], "o-", color=LEVEL_COLORS[n], ms=5,
              lw=1.2, label=f"$E_{n}$")
    fit = level_errors[:, n] > 1e-12  # leave out the rounding-limited points
    slope, _ = np.polyfit(np.log10(h_values[fit]), np.log10(level_errors[fit, n]), 1)
    slopes.append(slope)
guide = level_errors[0, 3] * (h_values / h_values[0]) ** 4
ax.loglog(h_values, guide, "--", color=GREY, lw=1.0, label="slope 4")
ax.set_xlabel("RK4 step size $h = a/n$ (units $a$)")
ax.set_ylabel("error of the energy (units $\\hbar^2/(m a^2)$)")
ax.set_title("Shooting energies converge like $h^4$")
ax.legend()
save_figure(fig, "eigenvalue_convergence",
            "The error of the four shooting energies of the finite well, "
            "$|E_n(\\mathrm{RK4}) - E_n(\\mathrm{exact})|$ in units of "
            "$\\hbar^2/(m a^2)$, against the RK4 step size $h = a/n$ for "
            "$n = 4$ to $1024$ steps, on logarithmic axes. The points follow lines "
            "of slope 4 (dashed guide): halving the step divides the error of "
            "an eigenvalue by 16, the order of RK4. Higher states (faster "
            "oscillation) have larger errors at the same step.")
report("measured orders of E_0 ... E_3", ", ".join(f"{s:.2f}" for s in slopes))
check(all(3.8 < s < 4.3 for s in slopes),
      "the energy errors fall with the order 4 (slopes between 3.8 and 4.3)")
check(np.all(level_errors[-1] < 1e-9), "with 1024 steps every energy is within 1e-9")
```

Out [11]:

```
Figure 02b.8 saved as Revision/textbook/figures/02b_8_eigenvalue_convergence.png
RESULT measured orders of E_0 ... E_3 = 3.99, 3.98, 3.96, 3.92
PASS the energy errors fall with the order 4 (slopes between 3.8 and 4.3)
PASS with 1024 steps every energy is within 1e-9
```

Text cell:

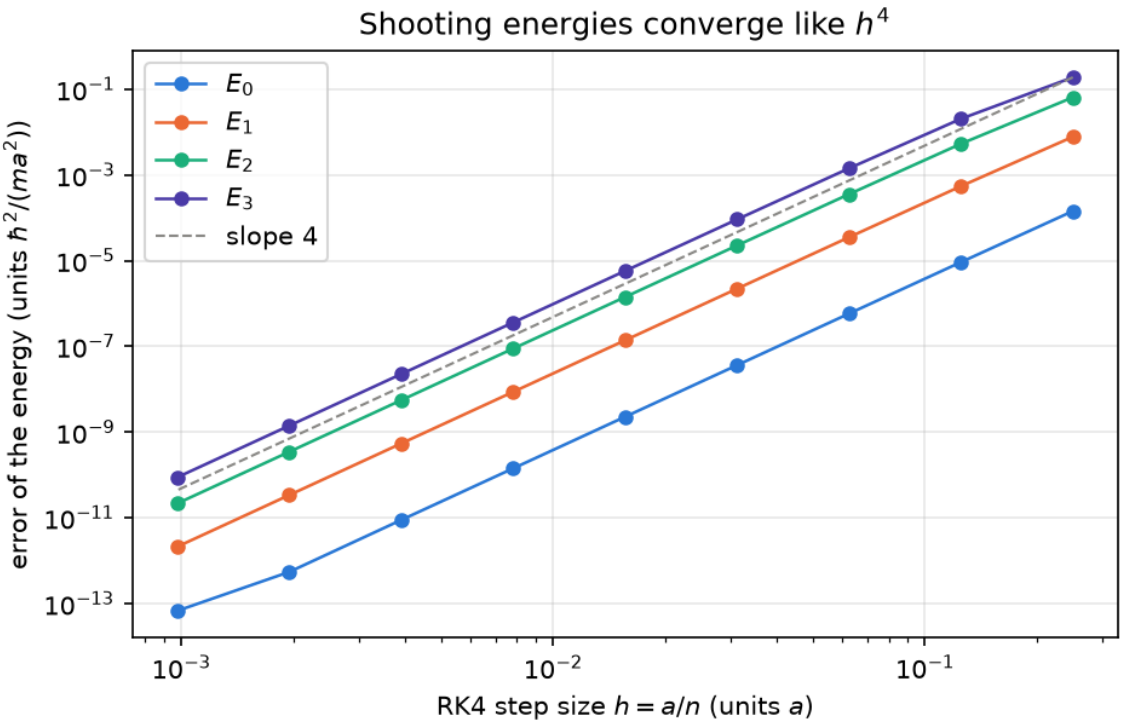

Figure 77: The error of the four shooting energies of the finite well, $|E_n(\text{RK4}) - E_n(\text{exact})|$ in units of $\hbar^2/(ma^2)$, against the RK4 step size $h = a/n$ for $n = 4$ to 1024 steps, on logarithmic axes. The points follow lines of slope 4 (dashed guide): halving the step divides the error of an eigenvalue by 16, the order of RK4. Higher states (faster oscillation) have larger errors at the same step. (Notebook 02b, figure 8.)

```
## 13. The last check

The last cell checks that all 8 figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [12]:

```
names = ["trial_solutions_string", "shooting_function_string",
         "bisection_vs_secant", "graphical_solution", "shooting_functions_well",
         "trial_solutions_well", "eigenfunctions", "eigenvalue_convergence"]
present = [output_file(f"{FIGURE_FOLDER}/02b_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02b exist")
all_checks_passed()
```

Out [12]:

```
PASS all 8 figure files of notebook 02b exist
ALL 16 CHECKS PASSED (notebook 02b)
```

Text cell:

```
## 14. What this notebook showed

- A boundary-value problem with an unknown number in it becomes a sequence of
  initial-value problems: shoot from one end, measure the mismatch at the other
  end, and find the zeros of the shooting function.
- For the string $u'' = -\lambda u$, $u(0) = u(1) = 0$, the RK4 shooting function
  equals $\sin\sqrt{\lambda}/\sqrt{\lambda}$ and its zeros are
  $\lambda_n = n^2\pi^2$. Bisection gains one binary digit per step (after 45
  steps the bracket is still $2.8 \times 10^{-13}$ wide); the secant rule
  reaches the rounding level in about 9 steps.
- The finite square well of depth 15 and half-width 1 has exactly four bound
  states, alternately even and odd, with $n$ nodes in the state $n$; shooting
  with RK4 and the parity conditions $u'(0) = 0$ or $u(0) = 0$ at the centre
  reproduces the exact energies of the transcendental equations (within
  $10^{-7}$ with 200 steps, $10^{-9}$ with 1024 steps), and the energy errors
  fall like $h^4$.
- Away from an eigenvalue the shot flies off to $\pm\infty$: only special
  energies give a solution that decays on both sides.
- The wave functions of different energies are orthogonal.
```

2.17 Line-by-line walk-through of Notebook 02b

The notebook has twelve code cells, In [1] to In [12]. As in Section 2.11, a quoted line `...)` stands for the rest of a figure caption, which Section 2.16 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 2.15. Its code is, line for line, the set-up code of Notebook 02a explained in Section 2.11, with the single difference

NOTEBOOK_ID = "02b", so that the figures are named 02b_<k>_<name>.png and the captions file is Revision/textbook/figures/02b.captions.json. It prints one line, Set-up of notebook 02b complete: repository folder found, helpers defined.

In [2], RK4 for systems, bisection and the secant rule.

```
import math  # sqrt, sin, pi for single numbers

import mpmath  # numbers with as many digits as we ask for
import numpy as np  # arrays (vectors) of numbers
```

math for single numbers, mpmath for numbers with as many digits as we ask for (used for the exact well levels), numpy for arrays.

```
# Colours that colour-blind readers can tell apart; curves also differ by style.
BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
GREY, BLACK = "#8a8986", "#000000"
LEVEL_COLORS = [BLUE, ORANGE, AQUA, VIOLET]  # one colour per bound state
```

Six colour names, given their hexadecimal codes at once (Python assigns the values on the right to the names on the left in order), and a list of four colours, one for each bound state of the well.

```
def rk4_step(f, x, Y, h):
    """One classical RK4 step for the system Y' = f(x, Y)."""
    k1 = f(x, Y)
    k2 = f(x + h / 2, Y + (h / 2) * k1)
    k3 = f(x + h / 2, Y + (h / 2) * k2)
    k4 = f(x + h, Y + h * k3)
    return Y + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)
```

The RK4 rule of Section 2.4, now with the variable called x (a position) and the state Y a numpy array (u, u'); since numpy adds and multiplies arrays entry by entry, the formulas act on both components at once.

```
def bisection(F, lo, hi, steps):
    """steps bisection steps on the bracket (lo, hi); the list of midpoints."""
    f_lo = F(lo)
    guesses = []
    for _ in range(steps):
        mid = 0.5 * (lo + hi)
        f_mid = F(mid)
        guesses.append(mid)
        if (f_mid > 0) == (f_lo > 0):  # same sign as at lo: the root is right
            lo, f_lo = mid, f_mid
        else:  # opposite sign: the root lies between lo and mid
            hi = mid
    return guesses
```

Bisection as in Section 2.12. f_lo is the value of F at the left end. Each of the steps passes (the loop variable _ is not used) computes the midpoint and F there and records the midpoint. (f_mid >

`0) == (f_lo > 0)` is true when both values have the same sign (both positive or both not positive): then the zero lies to the right of `mid`, which becomes the new left end (with its value); otherwise `mid` becomes the new right end. The function returns all the midpoints, so that their errors can be drawn.

```
def secant(F, x0, x1, steps):
    """steps secant steps from the guesses x0, x1; the list of new guesses."""
    f0, f1 = F(x0), F(x1)
    guesses = []
    for _ in range(steps):
        if f1 == f0:  # the line is flat: no new information (converged)
            break
        x2 = x1 - f1 * (x1 - x0) / (f1 - f0)  # the zero of the line
        x0, f0, x1, f1 = x1, f1, x2, F(x2)
        guesses.append(x2)
    return guesses


say("RK4 for systems, bisection and the secant rule are defined.")
```

The secant rule of Section 2.12. If the two last values of F are equal the line is flat and has no zero; this happens when both guesses have already reached the root to the last digit, and `break` leaves the loop. Otherwise `x2` is the zero of the line through the last two points; the line `x0, f0, x1, f1 = x1, f1, x2, F(x2)` shifts the pair forward (the old newest point becomes the older one, and the new guess with its value becomes the newest). The cell prints one line.

In [3], three shots for the string.

```
def shoot_string(lam, n):
    """RK4 solution of u'' = -lam u, u(0) = 0, u'(0) = 1 on 0 <= x <= 1 in n
    steps.  Returns the n + 1 positions and the n + 1 values of u."""
    def f(x, Y):
        return np.array([Y[1], -lam * Y[0]])  # (u', v') = (v, -lam u)
    h = 1.0 / n
    Y = np.array([0.0, 1.0])  # u(0) = 0, u'(0) = 1
    xs, us = [0.0], [0.0]
    for i in range(n):
        Y = rk4_step(f, i * h, Y, h)
        xs.append((i + 1) * h)
        us.append(Y[0])
    return np.array(xs), np.array(us)
```

The string equation $u'' = -\lambda u$ as the system $u' = v$, $v' = -\lambda u$: the inner function `f` returns $(v, -\lambda u)$ for the state `Y` $= (u, v)$ (`lam` stands for λ, because `lambda` is a reserved word of Python). The shot starts with $u = 0$, $u' = 1$ and makes `n` RK4 steps of size $1/n$, recording every position and value of u.

```
def F_string(lam, n=200):
    """The shooting function: where the shot lands, u(1)."""
    return shoot_string(lam, n)[1][-1]
```

The shooting function $F(\lambda) = u(1;\lambda)$: the last value of u; `n=200` makes 200 steps the default.

```
fig, ax = plt.subplots()
for lam, color, style in ((5.0, ORANGE, "--"), (math.pi ** 2, BLUE, "-"),
                          (15.0, AQUA, "-.")):
    xs, us = shoot_string(lam, 200)
    ax.plot(xs, us, style, color=color, lw=1.6,
            label=f"$\\lambda = {lam:.4f}$: $u(1) = {us[-1]:+.4f}$")
    ax.plot([1.0], [us[-1]], "o", color=color, ms=6)
```

Three shots, with $\lambda = 5$ (dashed orange), $\lambda = \pi^2$ (solid blue) and $\lambda = 15$ (dash-dotted aqua); each is drawn with its landing value $u(1)$ in the legend (`:+.4f` writes the sign and 4 decimals), and a dot marks where it lands.

```
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.plot([0.0, 1.0], [0.0, 0.0], "s", color=BLACK, ms=7, label="the two fixed ends")
ax.set_xlabel("position $x$ along the string")
ax.set_ylabel("displacement $u(x)$")
ax.set_title("Three shots for $u'' = -\\lambda u$, $u(0) = 0$, $u'(0) = 1$")
ax.legend(fontsize=8)
save_figure(fig, "trial_solutions_string",
            "Three shots for the string equation $d^2u/dx^2 = -\\lambda u$ started at "
            ...)
check(F_string(5.0) > 0 > F_string(15.0) and abs(F_string(math.pi ** 2)) < 1e-9,
      "u(1) > 0 for lambda = 5, < 0 for lambda = 15, = 0 for lambda = pi^2")
```

The horizontal axis line, two black squares at the fixed ends, labels, the legend and the saved figure `02b_1_trial_solutions_string.png`. The check confirms the picture in numbers: the shot lands above the target for $\lambda = 5$, below it for $\lambda = 15$, and within 10^{-9} of it for $\lambda = \pi^2$. **What Figure 02b.1 shows:** a small λ bends the string too little (it comes down too late), a large one too much (it crosses zero too early); the eigenvalue π^2 lands exactly on the target.

In [4], the whole shooting function.

```
lam_points = np.linspace(0.5, 100.0, 60)  # 60 guesses for RK4
lam_fine = np.linspace(0.5, 100.0, 800)  # many points for the exact curve
F_points = np.array([F_string(lam) for lam in lam_points])
F_exact = np.sin(np.sqrt(lam_fine)) / np.sqrt(lam_fine)
```

60 values of λ between 0.5 and 100 are shot with RK4; 800 values give a smooth exact curve $\sin\sqrt{\lambda}/\sqrt{\lambda}$.

```
fig, ax = plt.subplots()
ax.plot(lam_fine, F_exact, color=BLACK, lw=1.0,
        label="exact $\\sin\\sqrt{\\lambda}/\\sqrt{\\lambda}$")
ax.plot(lam_points, F_points, "o", color=BLUE, ms=4, label="RK4 shots, 200 steps")
for n in (1, 2, 3):
    ax.axvline(n ** 2 * math.pi ** 2, ls=":", color=GREY, lw=1.0)
    ax.text(n ** 2 * math.pi ** 2, 0.62, f"${n * n}\\pi^2$", ha="center",
            fontsize=9)
```

The exact curve in black, the RK4 values as blue points, and dotted vertical lines (`axvline`) at the first three eigenvalues $n^2\pi^2$, labelled $1\pi^2$, $4\pi^2$, $9\pi^2$ near the top.

```
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_xlabel("guessed eigenvalue $\\lambda$")
ax.set_ylabel("$F(\\lambda) = u(1; \\lambda)$")
ax.set_ylim(-0.3, 0.7)
ax.set_title("The shooting function of the string")
ax.legend()
save_figure(fig, "shooting_function_string",
            "The shooting function $F(\\lambda) = u(1; \\lambda)$ of the string, "
            ...)
```

The zero line, labels, the vertical range and the saved figure `02b_2_shooting_function_string.png`. **What Figure 02b.2 shows:** the RK4 points lie on the exact curve; it crosses zero exactly at the dotted lines, and between two crossings it keeps its sign, so any two guesses with opposite signs of F bracket an eigenvalue.

```
worst = np.max(np.abs(F_points - np.sin(np.sqrt(lam_points)) /
                      np.sqrt(lam_points)))
report("largest difference RK4 - exact shooting function", f"{worst:.2e}")
check(worst < 1e-7, "the RK4 shooting function equals sin(sqrt(lam))/sqrt(lam)")
```

The largest distance between the 60 RK4 values and the exact formula is 4.54×10^{-8} (COMPUTED; it is largest at the biggest λ, where the solution oscillates fastest), and the check requires it to be below 10^{-7}.

In [5], bisection against the secant rule.

```
PI2 = math.pi ** 2  # the exact eigenvalue
bis = bisection(F_string, 5.0, 15.0, 45)
sec = secant(F_string, 5.0, 15.0, 12)
say("step   bisection midpoint   F(midpoint)    secant guess")
for k in range(6):
    say(f"{k + 1:4d}   {bis[k]:18.6f}   {F_string(bis[k]):+11.5f}   {sec[k]:13.7f}")
```

45 bisection steps and at most 12 secant steps, both started from 5 and 15. The table prints the first six of each: the bisection midpoints 10, 7.5, 8.75, 9.375, 9.6875, 9.84375 with the signs of F that decided each halving (only the first is negative, so the bracket moves up each time after the first step), and the secant guesses 11.7107898, 8.8043277, 10.0237185, 9.8815597, 9.8694633, 9.8696045 (the first one is the worked example of Section 2.12).

```
lam_rk4 = sec[-1]  # the converged secant value: the RK4 eigenvalue (200 steps)
report("eigenvalue by RK4 shooting (200 steps)", f"{lam_rk4:.12f}")
report("pi^2", f"{PI2:.12f}")
errors_bis = np.abs(np.array(bis) - lam_rk4)
errors_sec = np.abs(np.array(sec) - lam_rk4)
```

The last secant guess is the zero of the RK4 shooting function, 9.869604411103; it differs from $\pi^2 = 9.869604401089$ by 1.0×10^{-8}, the error of RK4 with 200 steps (COMPUTED). The errors of all guesses are measured against this zero (not against π^2), because that is the number the root finders are looking for.

```
fig, ax = plt.subplots()
ax.semilogy(range(1, len(bis) + 1), np.maximum(errors_bis, 1e-17), "s-",
            color=ORANGE, ms=4, lw=1.2, label="bisection")
ax.semilogy(range(1, len(sec) + 1), np.maximum(errors_sec, 1e-17), "o-",
            color=BLUE, ms=5, lw=1.2, label="secant rule")
k_values = np.arange(1, len(bis) + 1)
ax.semilogy(k_values, 10.0 / 2.0 ** k_values, ":", color=GREY,
            label="bracket width $10/2^k$")
```

The errors after each step on a logarithmic axis; `np.maximum(errors, 1e-17)` replaces an error of exactly 0 (which a logarithmic axis cannot show) by 10^{-17}. The dotted grey line is the bracket width $10/2^k$ of Section 2.12.

```
ax.set_xlabel("step $k$")
ax.set_ylabel("error $|\\lambda_k - \\lambda_1|$")
ax.set_ylim(1e-17, 10.0)
ax.set_title("How fast the two root finders approach the eigenvalue $\\lambda_1$")
ax.legend()
save_figure(fig, "bisection_vs_secant",
            "The error of the guess after $k$ steps, $|\\lambda_k - \\lambda_1|$, "
            ...)
```

Labels and the saved figure `02b_3_bisection_vs_secant.png`. **What Figure 02b.3 shows:** the bisection errors follow the dotted halving line, one binary digit per step, and are still about 10^{-13} after 45 steps; the secant errors fall faster and faster and reach the rounding level in about 9 steps.

```
check(bis[:6] == [10.0, 7.5, 8.75, 9.375, 9.6875, 9.84375],
      "the bisection midpoints 10, 7.5, 8.75, 9.375, 9.6875, 9.84375")
check(abs(sec[5] - PI2) < 1e-6 and abs(bis[5] - PI2) > 0.02,
      "after 6 steps: secant within 1e-6 of pi^2, bisection still 0.02 away")
check(abs(lam_rk4 - PI2) < 2e-8 and abs(bis[-1] - lam_rk4) < 1e-11,
      "RK4 eigenvalue = pi^2 within 2e-8; 45 bisections agree within 1e-11")
```

Three checks: the first six midpoints are exactly those of the worked example (these numbers are exact in binary, so `==` is safe); after six steps the secant rule is within 10^{-6} of π^2 while bisection is still more than 0.02 away; the RK4 eigenvalue is within 2×10^{-8} of π^2 and the 45th bisection midpoint within 10^{-11} of it.

In [6], the exact well levels with mpmath.

```
V0, A = 15.0, 1.0  # the depth and the half-width of the well
mpmath.mp.dps = 30  # work with 30 significant digits
Z0 = mpmath.sqrt(2 * mpmath.mpf(V0)) * A  # z0 = a sqrt(2 V0) = sqrt(30)
```

The depth $V_0 = 15$ and the half-width $a = 1$ (called `A` in the code). `mpmath.mp.dps = 30` makes mpmath compute with 30 significant digits; `mpmath.mpf(V0)` turns 15 into an mpmath number, so that $z_0 = \sqrt{30}$ is computed to 30 digits.

```
def g_even(z):
    return z * mpmath.sin(z) - mpmath.sqrt(Z0 ** 2 - z ** 2) * mpmath.cos(z)


def g_odd(z):
    return z * mpmath.cos(z) + mpmath.sqrt(Z0 ** 2 - z ** 2) * mpmath.sin(z)
```

The even condition $\tan z = \sqrt{z_0^2 - z^2}/z$ multiplied by $z\cos z$ becomes $z\sin z - \sqrt{z_0^2 - z^2}\cos z = 0$, and the odd condition $-\cot z = \sqrt{z_0^2 - z^2}/z$ multiplied by $z\sin z$ becomes $-z\cos z = \sqrt{z_0^2 - z^2}\sin z$, that is $z\cos z + \sqrt{z_0^2 - z^2}\sin z = 0$. These forms have no infinities, which makes them safe for a root finder.

```
EXACT = []  # (energy, parity) of the bound states, lowest first
for j in range(8):  # the intervals of length pi/2 between 0 and z0
    lo = j * mpmath.pi / 2
    hi = min((j + 1) * mpmath.pi / 2, Z0)
    if lo >= Z0:
        break
    g, parity = (g_even, "even") if j % 2 == 0 else (g_odd, "odd")
    # the root inside the interval (lo, hi), searched a hair away from its ends
    z = mpmath.findroot(g, (lo + mpmath.mpf("1e-20"), hi - mpmath.mpf("1e-20")),
                        solver="anderson")
    EXACT.append((z ** 2 / (2 * A ** 2) - V0, parity, z))
```

The loop walks through the intervals $(j\pi/2, (j+1)\pi/2)$, the last one cut off at z_0, and stops (`break`) when an interval starts beyond z_0. `j % 2` is the remainder of j divided by 2: on the intervals with even j the even condition has its root, on the others the odd one (Section 2.13). `mpmath.findroot` with the method `"anderson"` (a bracketing root finder) refines the root inside the interval, started a hair (10^{-20}) away from its ends; then the energy $E = z^2/(2a^2) - V_0$ is stored with the parity and z. Each entry of `EXACT` is a **tuple**, a fixed group of values in round brackets.

```
for n, (energy, parity, z) in enumerate(EXACT):
    say(f"state {n} ({parity:4}): z = {mpmath.nstr(z, 20)}, "
        f"E = {mpmath.nstr(energy, 20)}")
check(len(EXACT) == 4 and [p for _, p, _ in EXACT] == ["even", "odd", "even", "odd"],
      "the well has 4 bound states: even, odd, even, odd")
```

The four states are printed with 20 significant digits (`mpmath.nstr`): $E_0 = -14.120556743965630861$, $E_1 = -11.518123819661769376$, $E_2 = -7.3330267121044013902$, $E_3 = -2.0436840105252862495$ (COMPUTED). The check confirms that there are 4 and that the parities alternate.

```
z_axis = np.linspace(0.01, float(Z0), 2000)
rhs = np.sqrt(float(Z0) ** 2 - z_axis ** 2) / z_axis
```

```
tan_z = np.tan(z_axis)
cot_z = -1.0 / np.tan(z_axis)
tan_z[np.abs(tan_z) > 12] = np.nan  # do not draw the jumps at the poles
cot_z[np.abs(cot_z) > 12] = np.nan
```

For the graphical solution: 2000 values of z up to z_0, the right side $\sqrt{z_0^2 - z^2}/z$, and the two left sides $\tan z$ and $-\cot z$. Where these jump between $+\infty$ and $-\infty$ the plotting line would draw a misleading vertical stroke; the entries larger than 12 in size are therefore set to `np.nan` ("not a number"), which matplotlib leaves out.

```
fig, ax = plt.subplots()
ax.plot(z_axis, rhs, color=BLACK, lw=1.5, label="$\\sqrt{z_0^2 - z^2}/z$")
ax.plot(z_axis, tan_z, color=BLUE, lw=1.2, label="$\\tan z$ (even states)")
ax.plot(z_axis, cot_z, "--", color=ORANGE, lw=1.2, label="$-\\cot z$ (odd states)")
for n, (energy, parity, z) in enumerate(EXACT):
    zf = float(z)
    ax.plot([zf], [math.sqrt(float(Z0) ** 2 - zf ** 2) / zf], "o",
            color=LEVEL_COLORS[n], ms=7, zorder=5)
    ax.annotate(f"$n = {n}$", (zf, math.sqrt(float(Z0) ** 2 - zf ** 2) / zf),
                textcoords="offset points", xytext=(5, 8), fontsize=9)
```

The three curves; then a dot at each solution (on the black curve), with `zorder=5` so that the dot is drawn on top of the lines, and the label n next to it.

```
ax.axvline(float(Z0), color=GREY, ls=":", lw=1.0)
ax.text(float(Z0), 7.3, "$z_0 = \\sqrt{30}$", ha="right", fontsize=9)
ax.set_ylim(0.0, 8.0)
ax.set_xlabel("$z = k a$")
ax.set_ylabel("value of each side of the equation")
ax.set_title("Graphical solution of the finite-well equations, $z_0 = 5.477$")
ax.legend(loc="upper center", fontsize=8)
save_figure(fig, "graphical_solution",
            "The graphical solution of the finite-well equations for "
            ...)
```

A dotted line at z_0, labels and the saved figure `02b_4_graphical_solution.png`. **What Figure 02b.4 shows:** the black curve falls from large values to zero at z_0; it crosses each rising branch of $\tan z$ and $-\cot z$ once, four times in all, alternately on a blue (even) and an orange (odd) branch.

In [7], the shooting functions of the well.

```
def shoot_well(E, n):
    """RK4 from x = -a (u = 1, u' = kappa) to x = 0 inside the well; E may be an
    array.  Returns (u(0), u'(0))."""
    E = np.asarray(E, dtype=float)
    kappa = np.sqrt(-2.0 * E)  # the decay rate outside the well
```

`np.asarray` turns `E` into a numpy array (a single number becomes an array with no axes), so that the same code works for one energy and for many; `kappa` is $\kappa = \sqrt{-2E}$ for every energy.

```
    def f(x, Y):
        return np.array([Y[1], 2.0 * (-V0 - E) * Y[0]])  # (u', 2 (V - E) u)
    h = A / n
    Y = np.array([np.ones_like(E), kappa])
    for i in range(n):
        Y = rk4_step(f, -A + i * h, Y, h)
    return Y[0], Y[1]
```

The right-hand side inside the well, $(u', 2(V-E)u)$ with $V = -V_0$. The step is a/n; the start is $u = 1$ (`np.ones_like(E)` is an array of ones of the same shape as `E`) and $u' = \kappa$. After `n` RK4 steps from $x = -a$ the function returns $u(0)$ and $u'(0)$, for all energies at once.

```
def F_even(E, n=200):
    return shoot_well(E, n)[1]  # u'(0): zero for an even state


def F_odd(E, n=200):
    return shoot_well(E, n)[0]  # u(0): zero for an odd state
```

The two shooting functions of Section 2.13.

```
E_scan = np.linspace(-V0 + 1e-3, -1e-3, 400)  # 400 guesses inside (-V0, 0)
fe, fo = F_even(E_scan), F_odd(E_scan)
fig, ax = plt.subplots()
ax.plot(E_scan, fe, color=BLUE, lw=1.4, label="$F_{\\rm even}(E) = u'(0)$")
ax.plot(E_scan, fo, "--", color=ORANGE, lw=1.4, label="$F_{\\rm odd}(E) = u(0)$")
```

400 energies, kept 10^{-3} away from the ends $-V_0$ and 0 (where k or κ would be zero), are shot all at once; the two shooting functions are drawn.

```
for n, (energy, parity, _) in enumerate(EXACT):
    ax.axvline(float(energy), ls=":", color=GREY, lw=1.0)
    ax.plot([float(energy)], [0.0], "o", color=LEVEL_COLORS[n], ms=7, zorder=5)
    ax.text(float(energy), 4.6, f"$E_{n}$", ha="center", fontsize=9)
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_ylim(-5.5, 5.5)
ax.set_xlabel("guessed energy $E$ (units $\\hbar^2/(m a^2)$)")
ax.set_ylabel("mismatch at $x = 0$")
ax.set_title("Shooting functions of the finite well ($V_0 = 15$, $a = 1$)")
ax.legend(loc="lower left")
save_figure(fig, "shooting_functions_well",
            "The shooting functions of the finite well: $F_{\\rm even}(E) = "
            ...)
```

A dotted line, a dot and a label at each exact energy; then axis labels (the energy unit $\hbar^2/(ma^2)$ is 1 in the units of the notebook) and the saved figure `02b_5_shooting_functions_well.png`. **What Figure 02b.5 shows:** the solid blue F_{even} crosses zero exactly at E_0 and E_2, the dashed orange F_{odd} at E_1 and E_3; each crossing of a shooting function is a level of its parity.

```
even_changes = int(np.sum(np.sign(fe[:-1]) != np.sign(fe[1:])))
odd_changes = int(np.sum(np.sign(fo[:-1]) != np.sign(fo[1:])))
report("sign changes of F_even and F_odd in the scan", f"{even_changes}, {odd_changes}")
check(even_changes == 2 and odd_changes == 2,
      "the scan finds 2 even and 2 odd sign changes, one per bound state")
```

`np.sign` gives $+1$, -1 or 0 for each entry; comparing the signs of neighbouring entries with `!=` gives `True` wherever the sign changes, and `np.sum` counts the `True` entries. The scan finds 2 sign changes of each function, one per bound state of that parity.

In [8], all four levels by bisection.

```
def well_levels(n, tolerance=1e-13):
    """The four energies with n RK4 steps: scan, then bisection on the brackets."""
    lo, hi, even = [], [], []
    for values, is_even in ((F_even(E_scan, n), True), (F_odd(E_scan, n), False)):
        # the places i where the sign of the mismatch differs from that at i + 1
        for i in np.nonzero(np.sign(values[:-1]) != np.sign(values[1:]))[0]:
            lo.append(E_scan[i])
            hi.append(E_scan[i + 1])
            even.append(is_even)
```

`well_levels` first scans both shooting functions on the 400 energies with `n` RK4 steps (the same `n` as the refinement, so that the brackets belong to the same shooting function). `np.nonzero(...)[0]` lists the positions `i` at which the sign changes between `i` and `i + 1`; each such pair of neighbouring energies is a bracket, stored with its parity.

```
    order = np.argsort(lo)  # lowest energy first
    lo, hi = np.array(lo)[order], np.array(hi)[order]
    even = np.array(even)[order]
```

`np.argsort(lo)` is the order of positions that sorts the left ends; using it as an index sorts all three lists the same way, lowest energy first.

```
    def mismatch(E):
        u0, du0 = shoot_well(E, n)
        return np.where(even, du0, u0)  # u'(0) for even, u(0) for odd states
    f_lo = mismatch(lo)
    while np.max(hi - lo) > tolerance:
        mid = 0.5 * (lo + hi)
        f_mid = mismatch(mid)
        same = np.sign(f_mid) == np.sign(f_lo)  # the root is above mid
        lo, f_lo = np.where(same, mid, lo), np.where(same, f_mid, f_lo)
        hi = np.where(same, hi, mid)
    return 0.5 * (lo + hi), even
```

`mismatch` shoots an array of energies and picks, entry by entry, $u'(0)$ for the even brackets and $u(0)$ for the odd ones: `np.where(condition, a, b)` takes the entry of `a` where the condition is true and that of `b` elsewhere. The `while` loop is bisection on all four brackets at once: as long as the

widest bracket is wider than 10^{-13}, it shoots the four midpoints, and where the sign at the midpoint equals the sign at the left end it moves the left end up, elsewhere the right end down. The function returns the four midpoints of the final brackets and their parities.

```
LEVELS, EVEN = well_levels(200)
exact = np.array([float(energy) for energy, _, _ in EXACT])
for n, (E_n, E_x) in enumerate(zip(LEVELS, exact)):
    say(f"E_{n}: RK4 (200 steps) {E_n:.12f}, exact {E_x:.12f}, "
        f"difference {E_n - E_x:+.1e}")
check(np.max(np.abs(LEVELS - exact)) < 1e-7,
      "the four shooting energies agree with the exact ones within 1e-7")
check(list(EVEN) == [True, False, True, False],
      "the levels alternate: even, odd, even, odd")
```

The four levels with 200 RK4 steps are compared with the exact ones. The differences are $+2.4\times10^{-11}$, $+1.5\times10^{-9}$, $+1.5\times10^{-8}$ and $+6.1\times10^{-8}$ (COMPUTED): all below 10^{-7}, and larger for the higher levels. The second check confirms the alternation of parities.

In [9], what a shot looks like when it misses.

```
def shoot_line(E, x_start=-3.0, x_end=3.0, n=600):
    """RK4 across the whole line; the potential is taken at the middle of every
    step (constant inside a step, because the walls are grid points)."""
    kappa = math.sqrt(-2.0 * E)
    h = (x_end - x_start) / n
    Y = np.array([1.0, kappa]) * math.exp(kappa * (x_start + A))  # exact tail
    xs, us = [x_start], [Y[0]]
```

This shot crosses the whole line from $x=-3$ to $x=3$ in 600 steps of $h=0.01$. It starts with the exact decaying tail: $u=e^{\kappa(x+a)}$ and $u'=\kappa u$ at $x=-3$.

```
    for i in range(n):
        x = x_start + i * h
        V = -V0 if abs(x + h / 2) < A else 0.0  # the potential of this step

        def f(x_value, Y_value, V=V):  # V=V freezes this step's potential in f
            return np.array([Y_value[1], 2.0 * (V - E) * Y_value[0]])
        Y = rk4_step(f, x, Y, h)
        xs.append(x + h)
        us.append(Y[0])
    return np.array(xs), np.array(us)
```

For every step the potential is read at the middle of the step: $-V_0$ if $|x+h/2|<a$, else 0. Because the walls $x=\pm1$ are grid points, the potential is constant inside every step, and RK4 keeps its accuracy. The right-hand side is defined anew for each step; the default argument `V=V` stores the current step's potential inside the function. After each step the position and u are recorded.

```
E0 = float(EXACT[0][0])
fig, ax = plt.subplots()
```

```
for shift, color, style in ((-0.05, ORANGE, "--"), (0.0, BLUE, "-"),
                            (0.05, AQUA, "-.")):
    xs, us = shoot_line(E0 + shift)
    ax.plot(xs, us / np.max(np.abs(us[xs <= 0])), style, color=color, lw=1.6,
            label=f"$E = E_0 {shift:+.2f}$")
```

Three shots: at the ground-state energy E_0 and 0.05 below and above it. Each is divided by its largest size on the left half (`us[xs <= 0]` keeps the values with $x \leq 0$), so that the three curves have comparable heights.

```
ax.axvspan(-A, A, color=GREY, alpha=0.15, label="inside the well")
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_ylim(-2.0, 2.0)
ax.set_xlabel("position $x$ (units $a$)")
ax.set_ylabel("$u(x)$ (scaled to 1 at the left peak)")
ax.set_title("Shots across the well near the ground state $E_0 = %.4f$" % E0)
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "trial_solutions_well",
            "Shots across the whole line from $x = -3$ to $x = 3$ (horizontal "
            ...)
```

`axvspan` shades the well; `"... %.4f" % E0` is an older way of writing a number into a string (the same as an f-string with `:.4f`). The figure is saved as `02b_6_trial_solutions_well.png`; one line of its caption is an f-string, which writes the value of E_0 into the caption. **What Figure 02b.6 shows:** all three shots start alike on the left; only the one at E_0 decays again on the right, while the one just below flies off to one infinity and the one just above to the other.

```
tail = {shift: shoot_line(E0 + shift)[1][-1] for shift in (-0.05, 0.05)}
check(tail[-0.05] * tail[0.05] < 0 and min(abs(tail[-0.05]), abs(tail[0.05])) > 10,
      "slightly below and above E0 the shots fly off to opposite infinities")
```

The end values of the two missing shots have opposite signs (their product is negative) and are both larger than 10 in size.

In [10], the four wave functions.

```
def simpson(values, h):
    """Simpson's rule for equally spaced values (an even number of intervals)."""
    return h / 3 * (values[0] + values[-1] + 4 * values[1:-1:2].sum()
                    + 2 * values[2:-1:2].sum())
```

Simpson's rule of Section 2.13. `values[1:-1:2]` takes every second entry starting at position 1 and stopping before the last (the odd positions, weight 4); `values[2:-1:2]` every second entry starting at position 2 (the inner even positions, weight 2).

```
def wave_function(E, is_even, n=200):
    """Normalised u on a grid of -3 <= x <= 3 (step a/n), built as described."""
    kappa = math.sqrt(-2.0 * E)
```

```
    h = A / n

    def f(x, Y):
        return np.array([Y[1], 2.0 * (-V0 - E) * Y[0]])
    Y = np.array([1.0, kappa])
    inside = [1.0]
    for i in range(n):
        Y = rk4_step(f, -A + i * h, Y, h)
        inside.append(Y[0])
    inside = np.array(inside)  # u on -a <= x <= 0
```

For one level, the shot from $x = -a$ to 0 inside the well (as in In [7], for a single energy), keeping all $n + 1$ values of u.

```
    norm = 2.0 * (simpson(inside ** 2, h) + 1.0 / (2.0 * kappa))  # both halves
```

$\int u^2\,dx$ over the whole line: on the left half, Simpson's rule inside the well plus the exact tail integral $1/(2\kappa)$ of Section 2.13 (the tail starts with $u(-a) = 1$); the right half gives the same by symmetry, hence the factor 2.

```
    x_left = np.arange(-3.0 * n, -n) * h  # -3 <= x < -a
    left = np.concatenate([np.exp(kappa * (x_left + A)), inside])
    x_half = np.concatenate([x_left, -A + np.arange(n + 1) * h])  # up to x = 0
```

`np.arange(-3.0 * n, -n)` counts from -600 up to -201, so times $h = 1/200$ it gives the positions from -3 to just before -1. On them u is the exact tail; `np.concatenate` joins it with the inside values. `x_half` holds the matching positions from -3 to 0.

```
    sign = 1.0 if is_even else -1.0
    x_all = np.concatenate([x_half, -x_half[-2::-1]])  # mirror x -> -x
    u_all = np.concatenate([left, sign * left[-2::-1]])
    return x_all, u_all / math.sqrt(norm), kappa, inside / math.sqrt(norm), h
```

The right half is the mirror image: `x_half[-2::-1]` runs backwards from the second-to-last entry to the first (leaving out $x = 0$, which is already there), and its negatives are the positions $x > 0$; the values are copied with the sign $+1$ for an even and -1 for an odd state. The function returns the positions, the normalised wave function (divided by $\sqrt{\int u^2 dx}$), κ, the normalised inside values and the step.

```
STATES = [wave_function(E_n, bool(e)) for E_n, e in zip(LEVELS, EVEN)]
fig, ax = plt.subplots(figsize=(7.0, 5.6))
x_pot = np.array([-3.0, -A, -A, A, A, 3.0])
ax.plot(x_pot, [0.0, 0.0, -V0, -V0, 0.0, 0.0], color=BLACK, lw=1.5,
        label="potential $V(x)$")
nodes = []
```

The four wave functions are built; the potential is drawn as a black line through six corner points (the walls are vertical because two points share each $x = \pm a$).

```
for n, (x_all, u_all, kappa, inside, h) in enumerate(STATES):
    E_n = LEVELS[n]
    ax.axhline(E_n, ls=":", color=GREY, lw=0.8)
    parity = "even" if EVEN[n] else "odd"
    ax.plot(x_all, E_n + 1.8 * u_all, color=LEVEL_COLORS[n], lw=1.6,
            label=f"$E_{n} = {E_n:.4f}$ ({parity})")
    big = u_all[np.abs(u_all) > 1e-8 * np.max(np.abs(u_all))]  # drop tiny values
    nodes.append(int(np.sum(np.sign(big[:-1]) != np.sign(big[1:]))))
```

Each state is drawn lifted to its energy, as $E_n + 1.8\,u_n(x)$, on a dotted line at its energy. To count the **nodes** (sign changes), the values smaller than 10^{-8} times the largest are dropped first, so that a value that is zero up to rounding (the centre of an odd state) cannot be counted as an extra sign change; then the sign changes are counted as in In [7].

```
ax.set_xlabel("position $x$ (units $a$)")
ax.set_ylabel("energy (units $\\hbar^2/(m a^2)$) and $E_n + 1.8\\, u_n(x)$")
ax.set_ylim(-16.0, 5.0)  # room for the legend above the well
ax.set_title("The four bound states of the finite square well")
ax.legend(loc="upper center", ncol=2, fontsize=8)
save_figure(fig, "eigenfunctions",
            "The four bound states of the finite square well of depth 15 and "
            ...)
report("nodes of the states 0 to 3", nodes)
check(nodes == [0, 1, 2, 3], "the state n has exactly n nodes")
```

Labels, a legend in two columns (`ncol=2`) and the saved figure `02b_7_eigenfunctions.png`; the node counts are $[0, 1, 2, 3]$ and the check requires exactly that. **What Figure 02b.7 shows:** the state n has n nodes, even and odd states alternate, and the higher the energy, the further the wave function leaks out of the well, where it decays like $e^{-\kappa|x|}$ (κ is smaller for a higher level).

```
def overlap(m, n):
    """The integral of u_m u_n over the whole line: Simpson inside, exact tails."""
    _, _, k_m, in_m, h = STATES[m]
    _, _, k_n, in_n, _ = STATES[n]
    half = simpson(in_m * in_n, h) + in_m[0] * in_n[0] / (k_m + k_n)
    same = (EVEN[m] == EVEN[n])  # different parity: the halves cancel exactly
    return 2.0 * half if same else 0.0
```

$\int u_m u_n\,dx$ over the left half is Simpson's rule inside the well plus the exact tail integral: on the tail $u_m u_n = u_m(-a)\,u_n(-a)\,e^{(\kappa_m+\kappa_n)(x+a)}$, and $\int_{-\infty}^{-a} e^{(\kappa_m+\kappa_n)(x+a)}\,dx = 1/(\kappa_m+\kappa_n)$ (the same antiderivative as in Section 2.13), which gives the term `in_m[0] * in_n[0] / (k_m + k_n)`. For the same parity the right half equals the left half; for different parity the integral is exactly zero (Section 2.13).

```
gram = np.array([[overlap(m, n) for n in range(4)] for m in range(4)])
report("largest |integral u_m u_n - (1 if m = n else 0)|",
       f"{np.max(np.abs(gram - np.eye(4))):.1e}")
check(np.max(np.abs(gram - np.eye(4))) < 1e-8,
      "the wave functions are normalised and orthogonal within 1e-8")
```

`gram` is the 4 by 4 table of all the integrals. For normalised, orthogonal functions it must be the unit matrix `np.eye(4)` (1 on the diagonal, 0 elsewhere); the largest difference is 1.6×10^{-10} (COMPUTED), within the required 10^{-8}. The entries for two states of different parity are set to 0 by `overlap` itself, using the proof; so this check tests the normalisation and the orthogonality of states of the same parity only. The next lines test the different-parity case numerically.

```
middle = len(STATES[0][0]) // 2  # the position of x = 0 in the grid -3 ... 3
whole_line, left_half = [], []
```

`STATES[0][0]` is the grid of x values from -3 to 3 (the first entry returned by `wave_function`). It has $6n + 1 = 1201$ points, so the integer division `// 2` gives 600, the position of the middle point $x = 0$. Two empty lists will collect the results.

```
for m, n in [(0, 1), (0, 3), (1, 2), (2, 3)]:  # the pairs of different parity
    product = STATES[m][1] * STATES[n][1]  # u_m u_n on the grid
    h_grid = STATES[m][4]  # the grid spacing a/200
    whole_line.append(abs(simpson(product, h_grid)))  # from x = -3 to x = 3
    left_half.append(abs(simpson(product[:middle + 1], h_grid)))  # -3 to 0
```

The states 0 and 2 are even, 1 and 3 odd, so these are the four pairs of different parity. `STATES[m][1]` is the normalised wave function u_m on the whole grid (the second entry returned by `wave_function`) and `STATES[m][4]` the grid spacing $a/200 = 0.005$. For each pair the product $u_m u_n$ is integrated with Simpson's rule over the whole grid ($-3 \le x \le 3$, 1200 intervals, an even number as Simpson's rule needs) and over the left half alone (`product[:middle + 1]`, the points from $x = -3$ to $x = 0$, 600 intervals); the sizes are stored.

```
report("different parity: largest |whole integral|, smallest |left half|",
       f"{max(whole_line):.1e}, {min(left_half):.2f}")
check(max(whole_line) < 1e-12 < 0.01 < min(left_half),
      "different parity: the two halves cancel, the whole integral is 0")
```

The RESULT line prints the largest whole integral, 9.5×10^{-17}, and the smallest half integral, 0.16 (COMPUTED). The chained comparison `a < b < c < d` means $a < b$ and $b < c$ and $c < d$: every whole integral is below 10^{-12} (zero up to rounding), while every half integral is above 0.01. So the two halves are far from zero each but cancel exactly, as the symmetry argument of Section 2.13 says: the product of an even and an odd function is odd.

In [11], the order of the computed energies.

```
N_STEPS = [4, 8, 16, 32, 64, 128, 256, 512, 1024]
level_errors = np.array([np.abs(well_levels(n)[0] - exact) for n in N_STEPS])
h_values = A / np.array(N_STEPS, dtype=float)
```

The whole level search is repeated with 4 to 1024 RK4 steps on $-a \le x \le 0$; `level_errors` is a table with one row per step count and one column per level.

```
fig, ax = plt.subplots()
slopes = []
```

```
for n in range(4):
    ax.loglog(h_values, level_errors[:, n], "o-", color=LEVEL_COLORS[n], ms=5,
              lw=1.2, label=f"$E_{n}$")
    fit = level_errors[:, n] > 1e-12  # leave out the rounding-limited points
    slope, _ = np.polyfit(np.log10(h_values[fit]), np.log10(level_errors[fit, n]), 1)
    slopes.append(slope)
```

For each level the errors are drawn against the step on a log-log plot, and a straight line is fitted through the points above 10^{-12} (below that the errors are limited by rounding and by the bisection tolerance).

```
guide = level_errors[0, 3] * (h_values / h_values[0]) ** 4
ax.loglog(h_values, guide, "--", color=GREY, lw=1.0, label="slope 4")
ax.set_xlabel("RK4 step size $h = a/n$ (units $a$)")
ax.set_ylabel("error of the energy (units $\\hbar^2/(m a^2)$)")
ax.set_title("Shooting energies converge like $h^4$")
ax.legend()
save_figure(fig, "eigenvalue_convergence",
            "The error of the four shooting energies of the finite well, "
            ...)
```

A dashed guide of slope 4 through the first point of the highest level, labels and the saved figure `02b_8_eigenvalue_convergence.png`. **What Figure 02b.8 shows:** four parallel lines of slope 4, the highest level on top: an eigenvalue found by shooting inherits the order of the integrator, and a level whose wave function oscillates faster (larger k) has a larger error, which grows like $(kh)^4$.

```
report("measured orders of E_0 ... E_3", ", ".join(f"{s:.2f}" for s in slopes))
check(all(3.8 < s < 4.3 for s in slopes),
      "the energy errors fall with the order 4 (slopes between 3.8 and 4.3)")
check(np.all(level_errors[-1] < 1e-9), "with 1024 steps every energy is within 1e-9")
```

The measured orders are 3.99, 3.98, 3.96 and 3.92 (COMPUTED); the checks require them between 3.8 and 4.3 and every error with 1024 steps below 10^{-9}.

In [12], the last check. It is built like In [15] of Notebook 02a: it checks that the eight figure files `02b_1_trial_solutions_string.png` to `02b_8_eigenvalue_convergence.png` exist and prints ALL 16 CHECKS PASSED (notebook 02b).

```
names = ["trial_solutions_string", "shooting_function_string",
         "bisection_vs_secant", "graphical_solution", "shooting_functions_well",
         "trial_solutions_well", "eigenfunctions", "eigenvalue_convergence"]
present = [output_file(f"{FIGURE_FOLDER}/02b_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02b exist")
all_checks_passed()
```

The 16 checks are: 1 in In [3], 1 in In [4], 3 in In [5], 1 in In [6], 1 in In [7], 2 in In [8], 1 in In [9], 3 in In [10], 2 in In [11] and 1 in In [12].

2.18 Functions of several variables and partial derivatives

Functions of several variables. A function of two variables is a rule $f(x, y)$ that gives a number for every pair of numbers (x, y); a function of eight variables gives a number for every list $(x_0, \dots, x_7)$. The entries of the author's metric are functions of this kind (of x_4 and x_0 only).

Partial derivatives. The **partial derivative** of f with respect to x is the ordinary derivative with respect to x while all the other variables are held fixed:

$$\frac{\partial f}{\partial x}(x, y) = \lim_{h \to 0} \frac{f(x + h, y) - f(x, y)}{h}.$$

The curly ∂ reminds us that other variables exist and are held fixed. We also write $\partial_x f$, and for the author's coordinates $\partial_4 = \partial/\partial x_4$ and $\partial_0 = \partial/\partial x_0$. Holding y fixed at a value y_0 turns f into the function $x \mapsto f(x, y_0)$ of one variable, a **slice**; the partial derivative is the slope of the slice. Every rule of one-variable calculus (sums, products, quotients, the chain rule) therefore holds for each partial derivative.

Example. $f(x, y) = x^2 \sin y$.

$$\frac{\partial f}{\partial x} = 2x \sin y$$

(with y held fixed, $\sin y$ is a constant factor, and the derivative of x^2 is $2x$);

$$\frac{\partial f}{\partial y} = x^2 \cos y$$

(with x held fixed, x^2 is a constant factor, and the derivative of $\sin y$ is $\cos y$). At the point $(x_0, y_0) = (1.2, 0.7)$ these slopes are $2 \cdot 1.2 \cdot \sin 0.7 = 1.5461$ and $1.44 \cos 0.7 = 1.1014$ (COMPUTED in Notebook 02c, In [3]).

Second and mixed derivatives. Differentiating again gives second partial derivatives; the **mixed** ones differentiate once with respect to each variable:

$$\frac{\partial}{\partial y}\Big(\frac{\partial f}{\partial x}\Big) = \frac{\partial}{\partial y}(2x \sin y) = 2x \cos y, \qquad \frac{\partial}{\partial x}\Big(\frac{\partial f}{\partial y}\Big) = \frac{\partial}{\partial x}(x^2 \cos y) = 2x \cos y$$

(each time the other variable is a constant factor). The two orders agree. This is **Schwarz's theorem**: for a function whose second partial derivatives are continuous, the order of differentiation does not matter (ASSUMED: a standard theorem of calculus, quoted without proof; Notebook 02c confirms it for this example, and Section 2.24 uses it).

The linear approximation. How much does f change when BOTH variables change a little, $x \to x + dx$ and $y \to y + dy$? Split the change into two moves, one variable at a time:

$$f(x + dx, y + dy) - f(x, y) = \big[f(x + dx, y + dy) - f(x, y + dy)\big] + \big[f(x, y + dy) - f(x, y)\big]$$

(we add and subtract $f(x, y + dy)$);

$$f(x+dx, y+dy) - f(x,y) \approx \frac{\partial f}{\partial x}(x, y+dy)\,dx + \frac{\partial f}{\partial y}(x,y)\,dy$$

(each bracket is the change of a slice, which is its slope times the step, up to terms of second order in the step);

$$f(x+dx, y+dy) - f(x,y) \approx \frac{\partial f}{\partial x}\,dx + \frac{\partial f}{\partial y}\,dy$$

(the slope in x changes only a little when y moves by dy, if $\partial f/\partial x$ is continuous; both slopes are now taken at (x, y)).

The gradient and the level curves. The **gradient** is the arrow $\nabla f = (\partial f/\partial x,\, \partial f/\partial y)$. A **level curve** (contour) is a curve on which f has one fixed value. Move a small distance s in the direction of a unit arrow (c, d) (one with $c^2 + d^2 = 1$), that is $dx = sc$ and $dy = sd$. By the linear approximation the change of f is

$$\Delta f \approx s\left(c\,\frac{\partial f}{\partial x} + d\,\frac{\partial f}{\partial y}\right),$$

s times the **dot product** of the direction with the gradient (the dot product of two arrows (c, d) and (p, q) is the number $cp + dq$). For short write $f_x = \partial f/\partial x$ and $f_y = \partial f/\partial y$, so that $\nabla f = (f_x, f_y)$, and let

$$|\nabla f| = \sqrt{f_x^2 + f_y^2}$$

be the **length** of the gradient arrow (Pythagoras' theorem; we take a point where $\nabla f \neq (0,0)$). Two facts follow. (1) The arrow $(c, d) = (-f_y,\, f_x)/|\nabla f|$ is a unit arrow, because $c^2 + d^2 = (f_y^2 + f_x^2)/|\nabla f|^2 = 1$. Along it the bracket is $cf_x + df_y = (-f_y f_x + f_x f_y)/|\nabla f| = 0$: f does not change to first order, so this direction runs along the level curve, and it is perpendicular to the gradient (their dot product is zero). (2) Every unit arrow can be written $(c, d) = (\cos\varphi, \sin\varphi)$, where φ is its angle to the x axis, and the gradient as $(f_x, f_y) = |\nabla f|\,(\cos\psi, \sin\psi)$ with its angle ψ. Then

$$cf_x + df_y = |\nabla f|\,(\cos\varphi\cos\psi + \sin\varphi\sin\psi) = |\nabla f|\cos(\varphi - \psi)$$

(insert both; then the addition formula of the cosine, $\cos(a-b) = \cos a\cos b + \sin a\sin b$). The cosine is largest, 1, when $\varphi = \psi$: the change of f per unit length is largest when the arrow points along the gradient, and there it is $|\nabla f|$. So the gradient points uphill, in the direction of the steepest increase, and its length is the steepest slope; in the opposite direction ($\varphi - \psi = \pi$, cosine -1) f falls fastest. Notebook 02c, In [4], checks both facts at the point $(1.2, 0.7)$: a step of 10^{-4} along the level curve changes f by only -9.58×10^{-9} (a second-order amount), and along the gradient f rises at the rate 1.898407 per unit length, against $|\nabla f| = 1.898293$ (COMPUTED).

Finite differences. A computer approximates a derivative from values of the function at nearby points with a small spacing h. Taylor's theorem (Section 2.3) gives

$$f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \dots$$

and, with h replaced by $-h$ (the odd powers change sign),

$$f(x-h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \dots$$

The **forward difference**:

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + \dots$$

(subtract $f(x)$ from the first series and divide by h): its error is proportional to h (order 1). The **central difference**:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \dots$$

(subtract the second series from the first: the even powers cancel, $f(x+h) - f(x-h) = 2hf'(x) + \frac{h^3}{3}f'''(x) + \dots$; then divide by $2h$): its error is proportional to h^2 (order 2). For a partial derivative the same formulas are applied to a slice. A **mixed** derivative $\partial^2 f/\partial x \partial y$ is approximated by applying the central difference twice, first in y and then in x:

$$\frac{\partial^2 f}{\partial x\, \partial y} \approx \frac{f(x+h, y+h) - f(x+h, y-h) - f(x-h, y+h) + f(x-h, y-h)}{4h^2}$$

(the central difference in y, $g(x) = [f(x, y+h) - f(x, y-h)]/(2h)$, inserted into the central difference in x, $[g(x+h) - g(x-h)]/(2h)$; each of the two steps has an error of order h^2, so the result has order 2).

The best spacing. The truncation error falls as h shrinks, but rounding grows: each value of f carries a rounding error of about $\epsilon|f|$ (Section 2.6), and a difference of two such values divided by h carries an error of about $\epsilon|f|/h$. For the forward difference the total error is about

$$E(h) = \alpha h + \frac{\beta}{h}, \qquad \alpha = \frac{|f''|}{2}, \quad \beta = \epsilon|f|$$

(the truncation term of the forward difference plus the rounding term). It is smallest where its derivative vanishes:

$$E'(h) = \alpha - \frac{\beta}{h^2} = 0, \qquad h_{\text{best}} = \sqrt{\beta/\alpha}$$

(differentiate term by term; then solve for h). For $f = x^2 \sin y$ at $(1.2, 0.7)$ in the variable y: $|f| = 0.928$ and $|\partial^2 f/\partial y^2| = |x^2 \sin y| = 0.928$, so $\beta = 2.2 \times 10^{-16} \cdot 0.928 = 2.0 \times 10^{-16}$, $\alpha = 0.46$ and $h_{\text{best}} \approx 2 \times 10^{-8}$. For the central difference $E(h) = \alpha h^2 + \beta/h$ with $\alpha = |f'''|/6$, and $E'(h) =$

$2\alpha h - \beta/h^2 = 0$ gives $h_{\text{best}} = (\beta/(2\alpha))^{1/3} \approx 10^{-5}$ (here $|f'''| = |\partial^3 f/\partial y^3| = |x^2 \cos y| = 1.101$, so $\alpha = 0.184$ and $h_{\text{best}} = 8 \times 10^{-6}$).

For the mixed formula both parts of the error are different. Its truncation term: the central difference in y is $g(x) = \partial_y f(x,y) + \frac{h^2}{6}\,\partial_y^3 f(x,y) + \dots$ (the series of the central difference above, applied to the slice in y; ∂_y^3 means ∂_y applied three times), and the central difference in x of g is $\partial_x g + \frac{h^2}{6}\,\partial_x^3 g + \dots$ (the same series in x). Inserting g into the second series gives

$$\frac{g(x+h) - g(x-h)}{2h} = \partial_x \partial_y f + \frac{h^2}{6}(\partial_x \partial_y^3 f + \partial_x^3 \partial_y f) + \dots$$

(the term $\frac{h^2}{6}\partial_x^3 g$ contributes $\frac{h^2}{6}\partial_x^3 \partial_y f$; the terms with h^4 are left out). Its rounding term: each of the four values of f carries about $\epsilon|f|$, together at most about $4\epsilon|f|$, and the formula divides by $4h^2$, which gives about $\epsilon|f|/h^2$. So

$$E(h) = \alpha h^2 + \frac{\beta}{h^2}, \qquad \alpha = \frac{|\partial_x \partial_y^3 f + \partial_x^3 \partial_y f|}{6}, \quad \beta = \epsilon|f|$$

(the truncation term plus the rounding term);

$$E'(h) = 2\alpha h - \frac{2\beta}{h^3} = 0, \qquad h^4 = \frac{\beta}{\alpha}, \qquad h_{\text{best}} = \Big(\frac{\beta}{\alpha}\Big)^{1/4}$$

(differentiate term by term, using $d(h^{-2})/dh = -2h^{-3}$; multiply by $h^3/(2\alpha)$; take the fourth root). For $f = x^2 \sin y$: $\partial_y^3 f = -x^2 \cos y$, so $\partial_x \partial_y^3 f = -2x \cos y$, while $\partial_x^3 \partial_y f = \partial_x^3 (x^2 \cos y) = 0$ (the third derivative of x^2 is 0). At $(1.2, 0.7)$ this gives $\alpha = 2 \cdot 1.2 \cdot \cos 0.7/6 = 0.306$ and $h_{\text{best}} = (2.0 \times 10^{-16}/0.306)^{1/4} = 1.6 \times 10^{-4}$, of the order 10^{-4}.

These are estimates of the scale only (rounding errors are not always of their largest size); Notebook 02c (In [6]) measures the best spacings 1.0×10^{-8}, 1.8×10^{-6} and 5.6×10^{-5} (COMPUTED). The lesson: the best spacing is far from the smallest one.

2.19 The chain rule and the derivatives of the author's metric

The chain rule for $z = 6Hx_0$. The author writes the hidden direction with the variable $z = 6Hx_0$, which runs over $0 < z < \pi/2$. A function of z, such as $g_{00} = \cot^2 z$, depends on x_0 through z, and the chain rule of one-variable calculus gives

$$\frac{\partial}{\partial x_0} F(z) = \frac{dF}{dz}\,\frac{\partial z}{\partial x_0} = 6H\,\frac{dF}{dz}$$

(the chain rule, and $\partial z/\partial x_0 = 6H$ because z is $6H$ times x_0). In short, $\partial_0 = 6H\, d/dz$ on functions of z. *Example.*

$$\frac{d}{dz}\cot^2 z = 2\cot z \cdot \Big(-\frac{1}{\sin^2 z}\Big) = -\frac{2\cot z}{\sin^2 z}$$

(the chain rule for the square, and the derivative of $\cot z$ is $-1/\sin^2 z$), so $\partial_0 \cot^2(6Hx_0) = -12H \cot z/\sin^2 z$ (multiply by $6H$); Notebook 02c, In [7], prints exactly this.

The metric and its scale factors. The metric is an 8×8 table of numbers g_{ab}, a **matrix** (Chapter 1), with rows and columns numbered by the author's coordinates $x_0, \ldots, x_7$, so that $a, b = 0, 1, \ldots, 7$. In the author's coordinates it is **diagonal**: only the entries $g_{00}, \ldots, g_{77}$ on its diagonal are nonzero, with

$$g_{00} = \cot^2 z, \quad g_{11} = g_{22} = g_{33} = e^{2a_4} \sin^{1/3} z, \quad g_{44} = -1, \quad g_{55} = g_{66} = g_{77} = -b_4\, e^{-2a_4} \sin^{1/3} z$$

(the author's metric, with the two functions $a_4(x_4)$ and $b_4(x_4) > 0$ of the time; $b_4 = 1$ is the background of the Revision record; COMPUTED: Notebook 02c, In [8], reads these eight entries from the Revision record `Revision/gkd_lovelock/results/curvature.json`, key `metricDiagonal`, and checks them against the formulas). The entries are positive for the space-like directions x_0, x_1, x_2, x_3 and negative for the time-like directions x_4, x_5, x_6, x_7; four positive and four negative entries are what the **signature** (4,4) means. The scale factor $s_a = \sqrt{|g_{aa}|}$ of each direction is

$$s_0 = \cot z, \qquad s_{1,2,3} = e^{a_4} \sin^{1/6} z, \qquad s_4 = 1, \qquad s_{5,6,7} = b_4^{1/2}\, e^{-a_4} \sin^{1/6} z$$

(the square root of each absolute value, as in Section 2.2; for s_0 we use $\cot z > 0$ on $0 < z < \pi/2$).

The logarithmic rates. The rate at which a scale factor grows, as a fraction of itself, is its **logarithmic derivative** $\partial_c \ln s_a = (\partial_c s_a)/s_a$ (the chain rule for the logarithm, whose derivative is $1/s$). Since $s_a^2 = |g_{aa}|$,

$$\partial_c \ln s_a = \frac{1}{2}\, \partial_c \ln |g_{aa}| = \frac{\partial_c g_{aa}}{2 g_{aa}}$$

($\ln s = \frac{1}{2} \ln s^2$; then the chain rule for the logarithm; the absolute value does not matter because $\partial_c |g|/|g| = \partial_c g/g$). For 3-space:

$$\ln s_1 = a_4 + \tfrac{1}{6} \ln \sin z$$

(the logarithm of a product is the sum of the logarithms, $\ln e^{a_4} = a_4$, and $\ln(\sin^{1/6} z) = \frac{1}{6} \ln \sin z$);

$$\partial_4 \ln s_1 = a_4', \qquad \partial_0 \ln s_1 = \tfrac{1}{6} \cdot 6H \cdot \frac{\cos z}{\sin z} = H \cot z$$

(a_4 depends on x_4 only, z on x_0 only; the derivative of $\ln \sin z$ is $\cos z/\sin z$, and the chain rule supplies $6H$). For the extra times $\ln s_5 = \frac{1}{2} \ln b_4 - a_4 + \frac{1}{6} \ln \sin z$, so

$$\partial_4 \ln s_5 = -a_4' + \frac{b_4'}{2 b_4}, \qquad \partial_0 \ln s_5 = H \cot z$$

(the same steps with $-a_4$; the new term $\frac{1}{2} \ln b_4$ has the derivative $\frac{1}{2} b_4'/b_4$ by the chain rule for the logarithm, and does not depend on x_0). For the hidden direction $\ln s_0 = \ln \cot z$:

$$\partial_0 \ln s_0 = 6H\,\frac{-1/\sin^2 z}{\cot z} = -\frac{6H}{\sin z\cos z}$$

(the chain rule twice; then $\cot z \sin^2 z = \sin z \cos z$), and $\partial_4 \ln s_0 = 0$; the time direction has $s_4 = 1$ and both rates 0. Along the history $a_4 = AHx_4$, $b_4 = 1$ we have $a_4' = AH$ and $b_4' = 0$; with $A = H = 1$ the three 3-space factors grow at the rate $+1$ and the three extra-time factors shrink at the rate -1: 3-space inflates and the extra times deflate exponentially (Notebook 02c, In [9]).

The rates add up to the rate of the volume. Along x_4:

$$\sum_{a=0}^{7} \partial_4 \ln s_a = 0 + 3a_4' + 0 + 3\Big(-a_4' + \frac{b_4'}{2b_4}\Big) = \frac{3}{2}\,\frac{b_4'}{b_4}$$

(the hidden direction, three 3-space directions, the time, three extra times); it is 0 when b_4 is constant. Along x_0:

$$\sum_{a=0}^{7} \partial_0 \ln s_a = -\frac{6H}{\sin z\cos z} + 6H\cot z = 6H\,\frac{\cos^2 z - 1}{\sin z\cos z} = -6H\,\frac{\sin^2 z}{\sin z\cos z} = -6H\tan z$$

(six directions with $H\cot z$; both terms written over the denominator $\sin z\cos z$; then $\cos^2 z - 1 = -\sin^2 z$). We now show that these sums are the rates of the **volume factor** $\sqrt{|\det g|}$.

The determinant. The **determinant** of a diagonal matrix is the product of its diagonal entries (Chapter 1). So

$$\det g = \cot^2 z \cdot \big(e^{2a_4}\sin^{1/3} z\big)^3 \cdot (-1) \cdot \big(-b_4\, e^{-2a_4}\sin^{1/3} z\big)^3$$

(the product of the eight entries);

$$\det g = (-1)(-1)^3\, b_4^3\, e^{6a_4} e^{-6a_4} \sin^{6/3} z\, \cot^2 z = b_4^3 \sin^2 z \cot^2 z$$

(the four minus signs multiply to $+1$; $(e^{2a_4})^3 = e^{6a_4}$ and $(e^{-2a_4})^3 = e^{-6a_4}$ multiply to $e^0 = 1$; the three factors b_4 give b_4^3; the six factors $\sin^{1/3} z$ give $\sin^2 z$);

$$\det g = b_4^3 \sin^2 z\,\frac{\cos^2 z}{\sin^2 z} = b_4^3\cos^2 z, \qquad \sqrt{|\det g|} = b_4^{3/2}\sin z\cot z = b_4^{3/2}\cos z$$

($\cot z = \cos z/\sin z$; and $\cos z > 0$ on $0 < z < \pi/2$). The factors e^{6a_4} of the inflating 3-space and e^{-6a_4} of the deflating extra times cancel: the volume factor depends on x_4 only through b_4, and for constant b_4 (the background) it does not depend on x_4, for EVERY function $a_4(x_4)$ (PROVED here; COMPUTED in Notebook 02c, In [11], which reproduces the record's entry `sqrtAbsDetG` $= b_4^{3/2}\sin z\cot z$ of `Revision/gkd_lovelock/results/curvature.json` and the Wolfram check `sqrt_abs_det_g_is_cos_z` of `Revision/field_equations_a4/reports/wolfram-a4-report.json`, which states $\sqrt{|\det g|} = b_4^{3/2}\cos z$).

Jacobi's formula for the derivative of the volume factor of a diagonal metric:

$$\sqrt{|\det g|} = \prod_{a=0}^{7} s_a$$

($|\det g|$ is the product of the $|g_{aa}| = s_a^2$, and the square root of a product is the product of the square roots);

$$\partial_c \ln \sqrt{|\det g|} = \sum_{a=0}^{7} \partial_c \ln s_a = \sum_{a=0}^{7} \frac{\partial_c g_{aa}}{2 g_{aa}} = \frac{1}{2}\,\mathrm{tr}(g^{-1}\,\partial_c g)$$

(the logarithm of a product is the sum of the logarithms; then the rates computed above; finally, for a diagonal g the inverse g^{-1} is diagonal with the entries $1/g_{aa}$, the matrix $g^{-1}\partial_c g$ is diagonal with the entries $\partial_c g_{aa}/g_{aa}$, and the **trace** tr is the sum of the diagonal entries). Multiplying by $\sqrt{|\det g|}$ gives Jacobi's formula $\partial_c \sqrt{|\det g|} = \frac{1}{2}\sqrt{|\det g|}\,\mathrm{tr}(g^{-1}\partial_c g)$. With the sums above: $\partial_4 \ln(b_4^{3/2}\cos z) = \frac{3}{2} b_4'/b_4$ and $\partial_0 \ln(b_4^{3/2}\cos z) = 6H \cdot (-\sin z/\cos z) = -6H\tan z$, in agreement (Notebook 02c, In [10] and In [11]).

Christoffel symbols of a diagonal metric. The geometry of Chapter 3 combines first partial derivatives of the metric into the **Christoffel symbols** $\Gamma^a{}_{bc}$. Here we take their formula for a diagonal metric as given (ASSUMED in this chapter; derived in Chapter 3) and use it only as an exercise in partial derivatives:

$$\Gamma^a{}_{bc} = \frac{1}{2g_{aa}}(\partial_b g_{ac} + \partial_c g_{ab} - \partial_a g_{bc}) \qquad \text{(no sum over } a\text{)}.$$

Because g is diagonal, g_{ac} is zero unless $a = c$. Going through the cases: if a, b and c are all different, every term contains an off-diagonal entry and vanishes. If $b = a$ (c arbitrary):

$$\Gamma^a{}_{ac} = \frac{1}{2g_{aa}}(\partial_a g_{ac} + \partial_c g_{aa} - \partial_a g_{ac}) = \frac{\partial_c g_{aa}}{2g_{aa}} = \partial_c \ln s_a$$

(the first and third terms cancel; what remains is the logarithmic rate; the case $c = a$ gives the same by the symmetry in b and c). If $b = c \neq a$:

$$\Gamma^a{}_{bb} = \frac{1}{2g_{aa}}(\partial_b g_{ab} + \partial_b g_{ab} - \partial_a g_{bb}) = -\frac{\partial_a g_{bb}}{2g_{aa}}$$

($g_{ab} = 0$ for $b \neq a$). Since the metric depends only on x_4 and x_0, the nonzero entries are (listing $\Gamma^a{}_{bc}$ with b not after c in the order $x_0, \ldots, x_7$, because the symbol is symmetric in b and c):

entries	**value**	**how many**
$\Gamma^0{}_{00}$	$-6H/(\sin z \cos z)$	1
$\Gamma^a{}_{0a}$, $a = 1,2,3,5,6,7$	$H \cot z$	6
$\Gamma^a{}_{a4}$, $a = 1,2,3$	a_4'	3
$\Gamma^a{}_{4a}$, $a = 5,6,7$	$-a_4' + b_4'/(2b_4)$	3
$\Gamma^4{}_{bb}$, $b = 1,2,3$	$a_4'\, e^{2a_4} \sin^{1/3} z$	3
$\Gamma^4{}_{bb}$, $b = 5,6,7$	$(a_4' b_4 - \frac{1}{2} b_4')\, e^{-2a_4} \sin^{1/3} z$	3

entries	value	how many
${\Gamma^0}_{bb}$, $b = 1,2,3$	$-He^{2a_4}\sin^{1/3} z\,\tan z$	3
${\Gamma^0}_{bb}$, $b = 5,6,7$	$+Hb_4e^{-2a_4}\sin^{1/3} z\,\tan z$	3

which makes $1+6+3+3+3+3+3+3 = 25$ entries; at $b_4 = 1$ the terms with b_4' drop out. Three of them worked out: ${\Gamma^4}_{11} = -\partial_4 g_{11}/(2g_{44}) = -2a_4' g_{11}/(2\cdot(-1)) = a_4' g_{11}$ (the derivative of e^{2a_4} is $2a_4'e^{2a_4}$); ${\Gamma^4}_{55} = -\partial_4 g_{55}/(2g_{44}) = \frac{1}{2}\partial_4 g_{55} = \frac{1}{2}\partial_4(-b_4e^{-2a_4}\sin^{1/3} z) = -\frac{1}{2}(b_4' - 2a_4'b_4)e^{-2a_4}\sin^{1/3} z$ (the product rule for $b_4e^{-2a_4}$, and the derivative of e^{-2a_4} is $-2a_4'e^{-2a_4}$), which is the table's $(a_4'b_4 - \frac{1}{2}b_4')\,e^{-2a_4}\sin^{1/3} z$; and

$$ {\Gamma^0}_{11} = -\frac{\partial_0 g_{11}}{2g_{00}} = -\frac{e^{2a_4}\cdot\frac{1}{3}\sin^{-2/3} z\cos z\cdot 6H}{2\cot^2 z} $$

(the case formula; the power rule and the chain rule for $\sin^{1/3} z$);

$$ {\Gamma^0}_{11} = -He^{2a_4}\sin^{-2/3} z\,\cos z\,\frac{\sin^2 z}{\cos^2 z} = -He^{2a_4}\sin^{1/3} z\,\tan z $$

($\frac{1}{3}\cdot 6H/2 = H$ and $1/\cot^2 z = \sin^2 z/\cos^2 z$; then $\sin^{-2/3} z\sin^2 z = \sin^{4/3} z = \sin^{1/3} z\cdot\sin z$ and $\sin z/\cos z = \tan z$). The extra-time entries ${\Gamma^0}_{55}$ have the opposite sign and the factor b_4 because g_{55} carries the minus sign of a time-like direction and the factor b_4. Notebook 02c, In [12], computes all $8\times 36 = 288$ entries with b not after c from the formula, finds exactly 25 nonzero ones and compares each with the Revision record (COMPUTED: reproduces `Revision/gkd_lovelock/results/curvature.json`, key `christoffelNonzero_b_le_c`).

2.20 Example: partial derivatives of a model function and of the author's metric

Notebook 02c carries out Sections 2.18 and 2.19. It differentiates $f = x^2\sin y$ with sympy, checks Schwarz's theorem, draws the level curves with the gradient arrows and checks that the gradient is perpendicular to them and points uphill, draws two slices with their tangent lines, measures the orders and the rounding limits of three finite-difference formulas, checks the chain rule for $z = 6Hx_0$, and then reads the author's metric from the Revision record and computes its rates along x_4 and x_0, the volume factor $b_4^{3/2}\cos z$, Jacobi's formula and all 25 Christoffel entries. It ends with ALL 17 CHECKS PASSED (notebook 02c).

2.21 How to run Notebook 02c

Step 1. What this notebook does and what it needs.

Notebook 02c (Partial derivatives, finite differences and the chain rule) is the file `Revision/textbook/notebooks/02c_partial_derivatives.ipynb` of the repository Dirac_claude. It computes partial derivatives of a function of two variables with sympy and by finite differences (forward and central differences, their orders 1 and 2 and the limit set by rounding), draws level curves with the gradient and slices with their tangent lines, checks that mixed partial derivatives do not depend on the order, checks the chain rule for the author's variable z = 6 H x0, and then reads the author's

metric from a Revision record and computes its partial derivatives: the growth and deflation rates of the eight scale factors, the volume factor sqrt of the absolute determinant (equal to b4^(3/2) cos z, independent of the time x4 for constant b4), Jacobi's formula, and all 25 nonzero Christoffel entries listed in the record. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 02c_partial_derivatives.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 02c_partial_derivatives.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 02c_partial_derivatives.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/02c.captions.json`
- `Revision/textbook/figures/02c_1_level_curves_gradient.png`
- `Revision/textbook/figures/02c_2_slices_and_slopes.png`
- `Revision/textbook/figures/02c_3_finite_differences.png`
- `Revision/textbook/figures/02c_4_chain_rule.png`
- `Revision/textbook/figures/02c_5_rates_along_x4.png`
- `Revision/textbook/figures/02c_6_rates_along_x0.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of notebook 02c exist
ALL 17 CHECKS PASSED (notebook 02c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/02c_partial_derivatives.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 02c_partial_derivatives.ipynb
python -m nbconvert --execute --inplace 02c_partial_derivatives.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

2.22 Notebook 02c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/02c_partial_derivatives.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The quantities of this book depend on several variables at once: the author's
```

metric depends on the time x_4 and on the hidden coordinate x_0. This
notebook teaches how to differentiate such functions and checks every step. It

- computes the partial derivatives of $f(x, y) = x^2 \sin y$ with sympy, checks
  that the two mixed second derivatives agree, and draws the level curves of f
  with its gradient arrows and two slices with their tangent lines;
- approximates partial derivatives by finite differences and measures their
  orders (1 for the forward difference, 2 for the central one) and the limit
  that rounding errors set;
- checks the chain rule for the author's variable $z = 6 H x_0$;
- reads the author's metric from the Revision record
  Revision/gkd_lovelock/results/curvature.json and computes its partial
  derivatives: the growth rates of the eight scale factors along x_4 and along
  x_0, the volume factor $\sqrt{|\det g|} = b_4^{3/2} \cos z$ (which does not
  depend on x_4 for constant b_4), Jacobi's formula for the derivative of a
  determinant, and all 25 nonzero *Christoffel symbols* listed in the record
  (combinations of first partial derivatives of the metric that the geometry of
  the book uses).

It draws 6 figures and prints a PASS line for every check.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 02c (Partial derivatives, finite differences and the chain rule) is the file
‘Revision/textbook/notebooks/02c_partial_derivatives.ipynb‘ of the repository
Dirac_claude. It computes partial derivatives of a function of two variables with sympy
and by finite differences (forward and central differences, their orders 1 and 2 and the
limit set by rounding), draws level curves with the gradient and slices with their
tangent lines, checks that mixed partial derivatives do not depend on the order, checks
the chain rule for the author's variable z = 6 H x0, and then reads the author's metric
from a Revision record and computes its partial derivatives: the growth and deflation
rates of the eight scale factors, the volume factor sqrt of the absolute determinant
(equal to b4^(3/2) cos z, independent of the time x4 for constant b4), Jacobi's formula,
and all 25 nonzero Christoffel entries listed in the record. It needs a computer with
Windows 11, macOS or Linux, an internet connection for the installation, the program Git,
and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the

default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
```

```
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 02c_partial_derivatives.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 02c_partial_derivatives.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 02c_partial_derivatives.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/02c.captions.json‘
- ‘Revision/textbook/figures/02c_1_level_curves_gradient.png‘
- ‘Revision/textbook/figures/02c_2_slices_and_slopes.png‘
- ‘Revision/textbook/figures/02c_3_finite_differences.png‘
- ‘Revision/textbook/figures/02c_4_chain_rule.png‘
- ‘Revision/textbook/figures/02c_5_rates_along_x4.png‘
- ‘Revision/textbook/figures/02c_6_rates_along_x0.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on

another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of notebook 02c exist
ALL 17 CHECKS PASSED (notebook 02c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/02c_partial_derivatives.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 02c_partial_derivatives.ipynb
  python -m nbconvert --execute --inplace 02c_partial_derivatives.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 02c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 02c (Partial derivatives, finite differences and the chain rule) is the file
# Revision/textbook/notebooks/02c_partial_derivatives.ipynb of the repository
# Dirac_claude. It computes partial derivatives of a function of two variables with sympy
# and by finite differences (forward and central differences, their orders 1 and 2 and
# the limit set by rounding), draws level curves with the gradient and slices with their
# tangent lines, checks that mixed partial derivatives do not depend on the order, checks
# the chain rule for the author's variable z = 6 H x0, and then reads the author's metric
# from a Revision record and computes its partial derivatives: the growth and deflation
# rates of the eight scale factors, the volume factor sqrt of the absolute determinant
# (equal to b4^(3/2) cos z, independent of the time x4 for constant b4), Jacobi's
# formula, and all 25 nonzero Christoffel entries listed in the record. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
```

```
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
```

```
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 02c_partial_derivatives.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
```

```
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 02c_partial_derivatives.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 02c_partial_derivatives.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/02c.captions.json
# - Revision/textbook/figures/02c_1_level_curves_gradient.png
# - Revision/textbook/figures/02c_2_slices_and_slopes.png
# - Revision/textbook/figures/02c_3_finite_differences.png
# - Revision/textbook/figures/02c_4_chain_rule.png
# - Revision/textbook/figures/02c_5_rates_along_x4.png
# - Revision/textbook/figures/02c_6_rates_along_x0.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of notebook 02c exist
#     ALL 17 CHECKS PASSED (notebook 02c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
```

```
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/02c_partial_derivatives.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 02c_partial_derivatives.ipynb
#     python -m nbconvert --execute --inplace 02c_partial_derivatives.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "02c"  # this notebook: chapter 02, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
```

```
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
```

```
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 02c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Function of several variables**: a rule $f(x, y)$ that gives a number for
  every pair of numbers $(x, y)$ (or for every list $x_0, \dots, x_7$).
- **Partial derivative** $\partial f/\partial x$: the ordinary derivative with
  respect to $x$ while all the other variables are held fixed. We also write
  $\partial_x f$, and for the author's coordinates $\partial_4 = \partial/\partial
  x_4$ and $\partial_0 = \partial/\partial x_0$.
- **Slice**: the function of ONE variable obtained by holding the others fixed;
  the partial derivative is the slope of its graph.
- **Level curve (contour)**: the set of points where $f$ has one fixed value.
- **Gradient**: the arrow $(\partial f/\partial x, \partial f/\partial y)$. It
  points in the direction in which $f$ grows fastest and is perpendicular to the
  level curve through its point.
- **Mixed partial derivative**: $\partial^2 f/\partial x \partial y$, first with
  respect to $y$, then $x$ (or the other way round). Schwarz's theorem: for a
  function with continuous second derivatives the order does not matter.
- **Chain rule**: if $f$ depends on $z$ and $z$ on $x_0$, then
  $\partial f/\partial x_0 = (df/dz)(\partial z/\partial x_0)$.
- **Finite difference**: an approximation of a derivative from values of the
  function at nearby points, with a small spacing $h$.
- **Metric, diagonal entries** $g_{aa}$: the eight numbers (functions of $x_4$ and
  $x_0$) that turn coordinate distances into true distances in the author's
  8-dimensional world; positive for the space directions $x_0, x_1, x_2, x_3$ and
  negative for the time directions $x_4, x_5, x_6, x_7$.
- **Scale factor** $s_a = \sqrt{|g_{aa}|}$: the factor by which a coordinate
  length along $x_a$ is multiplied to give a true length.
- **Logarithmic derivative** $\partial \ln s/\partial x = (\partial s/\partial
  x)/s$: the rate of growth of $s$ per unit of $x$, as a fraction of $s$
  itself.
- **Determinant of a diagonal matrix**: the product of its diagonal entries.
- **Christoffel symbols** $\Gamma^a{}_{bc}$: numbers built from first partial
  derivatives of the metric; the geometry of the book explains their meaning.
  Here they are only an exercise in partial derivatives.
```

Text cell:

```
## 4. The physical and mathematical situation

**Definition.** For a function $f(x, y)$,

$$\frac{\partial f}{\partial x}(x, y) = \lim_{h \to 0}
\frac{f(x + h, y) - f(x, y)}{h},$$

the derivative of the slice $x \mapsto f(x, y)$ with $y$ held fixed; likewise for
$y$. All the rules of one-variable calculus hold for each partial derivative.

**The first example.** $f(x, y) = x^2 \sin y$. Holding $y$ fixed, $\sin y$ is a
constant factor and $\partial f/\partial x = 2x \sin y$. Holding $x$ fixed,
$x^2$ is a constant factor and $\partial f/\partial y = x^2 \cos y$. Then
$\partial/\partial y$ of $2x \sin y$ is $2x \cos y$ and $\partial/\partial x$ of
$x^2 \cos y$ is also $2x \cos y$: the mixed derivatives agree, as Schwarz's
theorem says.
```

```
**Finite differences.** Taylor's theorem in one variable gives
$f(x + h) = f(x) + h f'(x) + \frac{h^2}{2} f''(x) + \frac{h^3}{6} f'''(x) +
\dots$ Therefore the *forward difference* $(f(x + h) - f(x))/h = f'(x) +
\frac{h}{2} f''(x) + \dots$ has an error proportional to $h$ (order 1), and the
*central difference* $(f(x + h) - f(x - h))/(2h) = f'(x) + \frac{h^2}{6}
f'''(x) + \dots$ has an error proportional to $h^2$ (order 2), because the
terms with $h^2 f''$ cancel in the difference. Rounding adds an error of about
$\epsilon |f| / h$ ($\epsilon = 2.2 \times 10^{-16}$), which GROWS as $h$
shrinks; the best $h$ balances the two.

**The author's metric** (coordinates $x_0, \dots, x_7$; $z = 6 H x_0$ with
$0 < z < \pi/2$; $a_4(x_4)$ and $b_4(x_4) > 0$ functions of the time, $b_4 = 1$
in the background of the Revision record):

$$g_{00} = \cot^2 z, \quad g_{11} = g_{22} = g_{33} = e^{2 a_4} \sin^{1/3} z, \quad
g_{44} = -1, \quad g_{55} = g_{66} = g_{77} = -b_4 e^{-2 a_4} \sin^{1/3} z .$$

The scale factors are $\cot z$ for the hidden direction $x_0$,
$e^{a_4} \sin^{1/6} z$ for 3-space ($x_1, x_2, x_3$), $1$ for the time $x_4$ and
$b_4^{1/2} e^{-a_4} \sin^{1/6} z$ for the three extra times $x_5, x_6, x_7$ (they
DEFLATE when $a_4$ grows and $b_4$ stays constant). By the chain rule,
$\partial_0 = (dz/dx_0)\, d/dz = 6H\, d/dz$ on a function of $z$. The
logarithmic rates follow from $\ln(e^{a_4} \sin^{1/6} z) = a_4 + \frac{1}{6} \ln
\sin z$ and $\ln(b_4^{1/2} e^{-a_4} \sin^{1/6} z) = \frac{1}{2} \ln b_4 - a_4 +
\frac{1}{6} \ln \sin z$:

$$\partial_4 \ln s_{1,2,3} = a_4', \quad
\partial_4 \ln s_{5,6,7} = -a_4' + \frac{b_4'}{2 b_4}, \quad
\partial_0 \ln s_{1,2,3} = \partial_0 \ln s_{5,6,7} = \tfrac{1}{6} \cdot 6H \cot z
= H \cot z,$$

and $\partial_0 \ln \cot z = 6H \cdot \frac{-1/\sin^2 z}{\cot z} =
-\frac{6H}{\sin z \cos z}$. The determinant of the diagonal metric is the
product of the entries; four of them are negative, so it is positive:
$\det g = \cot^2 z\, (e^{2a_4})^3 (b_4 e^{-2a_4})^3 (\sin^{1/3} z)^6 = b_4^3
\sin^2 z \cot^2 z = b_4^3 \cos^2 z$ and $\sqrt{|\det g|} = b_4^{3/2} \cos z$. The
factors $e^{6 a_4}$ and $e^{-6 a_4}$ cancel: the inflation of 3-space and the
deflation of the extra times compensate, and the volume factor depends on $x_4$
only through $b_4$; for constant $b_4$ (the background) it does not depend on
$x_4$.

**Christoffel symbols of a diagonal metric.** We take the formula
$\Gamma^a{}_{bc} = \frac{1}{2 g_{aa}} (\partial_b g_{ac} + \partial_c g_{ab} -
\partial_a g_{bc})$ (no sum over $a$) as given; the geometry of the book
derives it. For a diagonal metric only two kinds of entry can be nonzero:
$\Gamma^a{}_{ac} = \partial_c g_{aa}/(2 g_{aa}) = \partial_c \ln s_a$, the
logarithmic rates above, and $\Gamma^c{}_{aa} = -\partial_c g_{aa}/(2 g_{cc})$
for $c \neq a$.
```

Text cell:

```
## 5. The set-up of this notebook
```

```
The next cell imports the packages, chooses the colours of the plots and
defines the sympy symbols $x$, $y$ of the first example.
```

In [2]:

```
import math  # functions of single numbers

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra and calculus with symbols

BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
GREY, BLACK = "#8a8986", "#000000"
x, y = sp.symbols("x y", real=True)  # the two variables of the first example
say("Packages imported; symbols x and y defined.")
```

Out [2]:

```
Packages imported; symbols x and y defined.
```

Text cell:

```
## 6. Partial derivatives with sympy

‘sp.diff(f, x)‘ differentiates with respect to $x$ and treats every other
symbol as a constant: exactly the definition of a partial derivative. The cell
prints the two first derivatives and the two mixed second derivatives of
$f = x^2 \sin y$, checks them against the hand calculation of section 4, and
evaluates everything at the point $(x_0, y_0) = (1.2, 0.7)$.
```

In [3]:

```
f = x ** 2 * sp.sin(y)
f_x = sp.diff(f, x)  # hold y fixed
f_y = sp.diff(f, y)  # hold x fixed
f_xy = sp.diff(f, x, y)  # first x, then y
f_yx = sp.diff(f, y, x)  # first y, then x
say(f"df/dx = {f_x},  df/dy = {f_y}")
say(f"d2f/dxdy = {f_xy},  d2f/dydx = {f_yx}")
check(sp.simplify(f_x - 2 * x * sp.sin(y)) == 0
      and sp.simplify(f_y - x ** 2 * sp.cos(y)) == 0,
      "df/dx = 2 x sin y and df/dy = x^2 cos y")
check(sp.simplify(f_xy - f_yx) == 0 and sp.simplify(f_xy - 2 * x * sp.cos(y)) == 0,
      "the mixed derivatives agree (Schwarz): both are 2 x cos y")
X0, Y0 = 1.2, 0.7  # the point at which we evaluate
at_point = {x: X0, y: Y0}
for label, expression in (("f", f), ("df/dx", f_x), ("df/dy", f_y),
                          ("d2f/dxdy", f_xy)):
    report(f"{label} at (1.2, 0.7)", f"{float(expression.subs(at_point)):.12f}")
```

Out [3]:

```
df/dx = 2*x*sin(y),  df/dy = x**2*cos(y)
d2f/dxdy = 2*x*cos(y),  d2f/dydx = 2*x*cos(y)
PASS df/dx = 2 x sin y and df/dy = x^2 cos y
PASS the mixed derivatives agree (Schwarz): both are 2 x cos y
RESULT f at (1.2, 0.7) = 0.927673469622
RESULT df/dx at (1.2, 0.7) = 1.546122449370
RESULT df/dy at (1.2, 0.7) = 1.101372749690
RESULT d2f/dxdy at (1.2, 0.7) = 1.835621249483
```

Text cell:

```
## 7. Pictures of a function of two variables

The first figure is a map of $f$: coloured bands between level curves, and at
the points of a coarse grid the gradient arrow $(\partial f/\partial x,
\partial f/\partial y)$. ‘sp.lambdify‘ turns a sympy expression into a fast
numpy function. The check confirms at one point that the gradient is
perpendicular to the level curve: the direction along the level curve,
$(-\partial f/\partial y, \partial f/\partial x)$, has zero dot product with the
gradient, and moving a small step along it changes $f$ only in second order.
A second check confirms that the gradient points uphill: a small step of length
$10^{-4}$ along the unit arrow $\nabla f/|\nabla f|$, where
$|\nabla f| = \sqrt{(\partial f/\partial x)^2 + (\partial f/\partial y)^2}$ is
the length of the gradient, raises $f$ by about $10^{-4}\,|\nabla f|$ (the
steepest slope), and such a step raises $f$ at every one of the 120 arrows of
the figure.
```

In [4]:

```
F = sp.lambdify((x, y), f, "numpy")  # f as a numpy function
FX = sp.lambdify((x, y), f_x, "numpy")
FY = sp.lambdify((x, y), f_y, "numpy")
xs = np.linspace(-2.0, 2.0, 201)
ys = np.linspace(0.0, 2.0 * np.pi, 201)
XX, YY = np.meshgrid(xs, ys)  # all grid points as two 2-dimensional arrays
fig, ax = plt.subplots(figsize=(7.0, 5.2))
bands = ax.contourf(XX, YY, F(XX, YY), levels=17, cmap="RdBu_r")  # coloured bands
ax.contour(XX, YY, F(XX, YY), levels=17, colors=BLACK, linewidths=0.4)
fig.colorbar(bands, ax=ax, label="$f(x, y) = x^2 \\sin y$")
xq, yq = np.meshgrid(np.linspace(-1.8, 1.8, 10), np.linspace(0.3, 6.0, 12))  # coarse
# an arrow (df/dx, df/dy) at every coarse grid point; scale=40 shortens all arrows
ax.quiver(xq, yq, FX(xq, yq), FY(xq, yq), color=BLACK, scale=40, width=0.004)
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
ax.set_title("Level curves of $f = x^2 \\sin y$ and its gradient arrows")
save_figure(fig, "level_curves_gradient",
            "A map of the function $f(x, y) = x^2 \\sin y$ for $x$ from $-2$ to $2$ "
            "(horizontal axis) and $y$ from $0$ to $2\\pi$ (vertical axis); all "
            "quantities are pure numbers. The colour shows the value of $f$ (red "
```

```
            "positive, blue negative), the thin black lines are level curves, and "
            "the arrows are the gradient $(\\partial f/\\partial x, \\partial "
            "f/\\partial y)$ at a grid of points. Every arrow points uphill and "
            "crosses the level curves at a right angle; long arrows mean a steep "
            "slope.")
gx, gy = float(f_x.subs(at_point)), float(f_y.subs(at_point))
tangent = np.array([-gy, gx]) / math.hypot(gx, gy)  # along the level curve
step = 1e-4
change = F(X0 + step * tangent[0], Y0 + step * tangent[1]) - F(X0, Y0)
report("change of f along the level curve for a step of 1e-4", f"{change:.2e}")
check(abs(gx * tangent[0] + gy * tangent[1]) < 1e-15 and abs(change) < 1e-7,
      "the gradient is perpendicular to the level curve")

length = math.hypot(gx, gy)  # the length |grad f| of the gradient arrow
rise = F(X0 + step * gx / length, Y0 + step * gy / length) - F(X0, Y0)
report("rise of f per unit length along the gradient, |grad f|",
       f"{rise / step:.6f}, {length:.6f}")
gxq, gyq = FX(xq, yq), FY(xq, yq)  # the arrows of the figure
lengths = np.hypot(gxq, gyq)
rises = F(xq + step * gxq / lengths, yq + step * gyq / lengths) - F(xq, yq)
check(abs(rise / step / length - 1) < 1e-3 and np.all(rises > 0),
      "the gradient points uphill: f rises at the rate |grad f| along it")
```

Out [4]:

```
Figure 02c.1 saved as Revision/textbook/figures/02c_1_level_curves_gradient.png
RESULT change of f along the level curve for a step of 1e-4 = -9.58e-09
PASS the gradient is perpendicular to the level curve
RESULT rise of f per unit length along the gradient, |grad f| = 1.898407, 1.898293
PASS the gradient points uphill: f rises at the rate |grad f| along it
```

Text cell:

```
The second figure shows what a partial derivative IS: the slope of a slice. On
the left, $y = y_0 = 0.7$ is held fixed and $f$ is drawn as a function of $x$; on
the right, $x = x_0 = 1.2$ is held fixed and $f$ is drawn as a function of $y$.
The dashed lines through the point have the slopes $\partial f/\partial x$ and
$\partial f/\partial y$ at $(x_0, y_0)$: they are the tangent lines of the
slices.
```

In [5]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
fig.subplots_adjust(wspace=0.3)  # room between the two panels
f0 = F(X0, Y0)
x_line = np.linspace(-0.5, 2.5, 200)
left.plot(x_line, F(x_line, Y0), color=BLUE, lw=1.6,
          label="slice $f(x, 0.7)$")
left.plot(x_line, f0 + gx * (x_line - X0), "--", color=ORANGE, lw=1.2,
          label=f"slope $\\partial f/\\partial x = {gx:.4f}$")
left.set_xlabel("$x$ (with $y = 0.7$ fixed)")
```

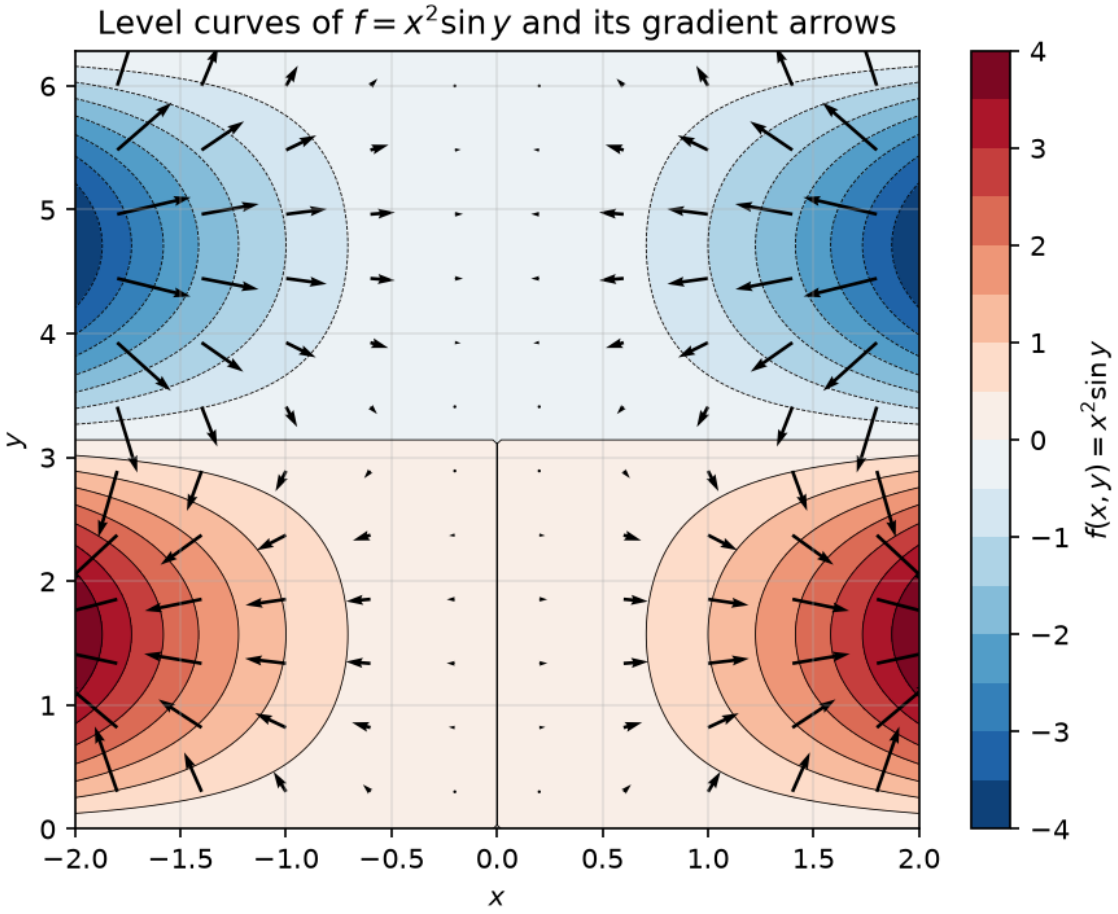

Figure 78: A map of the function $f(x, y) = x^2 \sin y$ for x from -2 to 2 (horizontal axis) and y from 0 to 2π (vertical axis); all quantities are pure numbers. The colour shows the value of f (red positive, blue negative), the thin black lines are level curves, and the arrows are the gradient $(\partial f/\partial x, \partial f/\partial y)$ at a grid of points. Every arrow points uphill and crosses the level curves at a right angle; long arrows mean a steep slope. (Notebook 02c, figure 1.)

```
y_line = np.linspace(-0.6, 2.4, 200)
right.plot(y_line, F(X0, y_line), color=BLUE, lw=1.6, label="slice $f(1.2, y)$")
right.plot(y_line, f0 + gy * (y_line - Y0), "--", color=ORANGE, lw=1.2,
           label=f"slope $\\partial f/\\partial y = {gy:.4f}$")
right.set_xlabel("$y$ (with $x = 1.2$ fixed)")
for ax in (left, right):
    ax.plot([X0 if ax is left else Y0], [f0], "o", color=BLACK, ms=6)
    ax.set_ylabel("$f$")
    ax.legend(fontsize=8)
save_figure(fig, "slices_and_slopes",
            "Two slices of $f(x, y) = x^2 \\sin y$ through the point "
            "$(x_0, y_0) = (1.2, 0.7)$ (black dot); all quantities are pure "
            "numbers. Left: $f$ as a function of $x$ with $y = 0.7$ held fixed, "
            "and its tangent line (dashed) of slope $\\partial f/\\partial x = "
            "2 x_0 \\sin y_0 = 1.5461$. Right: $f$ as a function of $y$ with "
            "$x = 1.2$ held fixed, and its tangent line of slope $\\partial "
            "f/\\partial y = x_0^2 \\cos y_0 = 1.1014$. A partial derivative is "
            "the slope of a slice.")
check(abs(gx - 2 * X0 * math.sin(Y0)) < 1e-15 and abs(gy - X0 ** 2 * math.cos(Y0))
      < 1e-15, "the slopes of the slices are 2 x0 sin y0 and x0^2 cos y0")
```

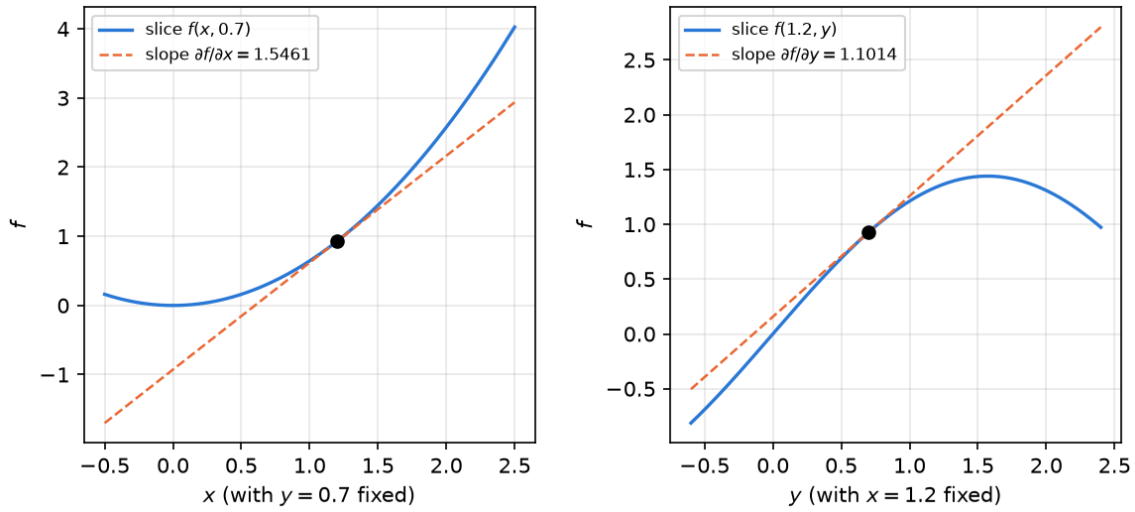

Figure 79: Two slices of $f(x, y) = x^2 \sin y$ through the point $(x_0, y_0) = (1.2, 0.7)$ (black dot); all quantities are pure numbers. Left: f as a function of x with $y = 0.7$ held fixed, and its tangent line (dashed) of slope $\partial f/\partial x = 2x_0 \sin y_0 = 1.5461$. Right: f as a function of y with $x = 1.2$ held fixed, and its tangent line of slope $\partial f/\partial y = x_0^2 \cos y_0 = 1.1014$. A partial derivative is the slope of a slice. (Notebook 02c, figure 2.)

Out [5]:

```
Figure 02c.2 saved as Revision/textbook/figures/02c_2_slices_and_slopes.png
PASS the slopes of the slices are 2 x0 sin y0 and x0^2 cos y0
```

Text cell:

```
## 8. Finite differences: order and rounding

The next cell approximates three derivatives at $(1.2, 0.7)$ for 49 spacings
$h$ from $10^{-12}$ to $1$ (equally spaced in $\log h$):

- forward difference for $\partial f/\partial y$: $(f(x_0, y_0 + h) -
  f(x_0, y_0))/h$;
- central difference for $\partial f/\partial y$: $(f(x_0, y_0 + h) -
  f(x_0, y_0 - h))/(2h)$;
- central difference for the mixed derivative $\partial^2 f/\partial x \partial
  y$: $[f(x_0{+}h, y_0{+}h) - f(x_0{+}h, y_0{-}h) - f(x_0{-}h, y_0{+}h) +
  f(x_0{-}h, y_0{-}h)]/(4h^2)$.

It fits the slopes on the log-log plot where truncation dominates, and finds the
best $h$ of each formula, where truncation and rounding balance.
```

In [6]:

```
exact_y = X0 ** 2 * math.cos(Y0)  # df/dy at the point
exact_xy = 2 * X0 * math.cos(Y0)  # d2f/dxdy at the point
h_values = np.logspace(-12, 0, 49)  # 1e-12 ... 1
forward = np.array([(F(X0, Y0 + h) - F(X0, Y0)) / h for h in h_values])
central = np.array([(F(X0, Y0 + h) - F(X0, Y0 - h)) / (2 * h) for h in h_values])
mixed = np.array([(F(X0 + h, Y0 + h) - F(X0 + h, Y0 - h) - F(X0 - h, Y0 + h)
                   + F(X0 - h, Y0 - h)) / (4 * h * h) for h in h_values])
err = {"forward": np.abs(forward - exact_y), "central": np.abs(central - exact_y),
       "mixed": np.abs(mixed - exact_xy)}
RANGES = {"forward": (1e-5, 1e-2), "central": (1e-3, 1e-1), "mixed": (1e-2, 1e-1)}
slope_of = {}
for name, (low, high) in RANGES.items():
    part = (h_values >= low) & (h_values <= high)
    slope_of[name], _ = np.polyfit(np.log10(h_values[part]),
                                   np.log10(err[name][part]), 1)
    best = int(np.argmin(err[name]))
    report(f"{name}: slope {slope_of[name]:.3f}, best h", f"{h_values[best]:.1e} "
           f"(error {err[name][best]:.1e})")
fig, ax = plt.subplots(figsize=(7.0, 5.0))
styles = {"forward": (ORANGE, "s", "forward difference, $\\partial f/\\partial y$"),
          "central": (BLUE, "o", "central difference, $\\partial f/\\partial y$"),
          "mixed": (AQUA, "^", "central mixed, $\\partial^2 f/\\partial x "
                               "\\partial y$")}
for name, (color, marker, label) in styles.items():
    shown = err[name] > 0  # an exact zero cannot be drawn on a log axis
    ax.loglog(h_values[shown], err[name][shown], marker=marker, color=color, ms=4,
              lw=1.0, label=label)
ax.loglog(h_values, 2.2e-16 / h_values, ":", color=BLACK, lw=1.0,
          label="rounding scale $\\epsilon / h$")
ax.loglog(h_values, 2.2e-16 / h_values ** 2, "-.", color=GREY, lw=1.0,
          label="rounding scale $\\epsilon / h^2$ (mixed)")
ax.set_ylim(1e-14, 10.0)
ax.set_xlabel("spacing $h$")
ax.set_ylabel("error of the approximation")
ax.set_title("Finite differences at $(1.2, 0.7)$: truncation against rounding")
```

```
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "finite_differences",
            "The error of three finite-difference approximations at the point "
            "$(1.2, 0.7)$ against the spacing $h$ from $10^{-12}$ to $1$, on "
            "logarithmic axes (pure numbers): forward difference for "
            "$\\partial f/\\partial y$ (squares), central difference for "
            "$\\partial f/\\partial y$ (circles) and the central formula for the "
            "mixed derivative (triangles), for $f = x^2 \\sin y$. Coming from the "
            "right, the errors fall with slope 1 (forward) and slope 2 (central, "
            "mixed). Coming from the left, rounding errors, of the size "
            "$\\epsilon / h$ (dotted) or $\\epsilon / h^2$ (dash-dotted) for the "
            "mixed formula, "
            "dominate. The best spacings are near $10^{-8}$, $10^{-6}$ and "
            "$10^{-4}$.")
check(abs(slope_of["forward"] - 1) < 0.05 and abs(slope_of["central"] - 2) < 0.05
      and abs(slope_of["mixed"] - 2) < 0.05,
      "measured orders: forward 1, central 2, mixed central 2")
check(err["central"].min() < 1e-10 < err["forward"].min() < 1e-7,
      "best central error < 1e-10 < best forward error < 1e-7")
```

Out [6]:

```
RESULT forward: slope 1.000, best h = 1.0e-08 (error 1.1e-09)
RESULT central: slope 2.000, best h = 1.8e-06 (error 5.3e-12)
RESULT mixed: slope 2.000, best h = 5.6e-05 (error 1.4e-09)
```

Out [6]:

```
Figure 02c.3 saved as Revision/textbook/figures/02c_3_finite_differences.png
PASS measured orders: forward 1, central 2, mixed central 2
PASS best central error < 1e-10 < best forward error < 1e-7
```

Text cell:

```
## 9. The chain rule with $z = 6 H x_0$

The author writes the hidden direction with $z = 6 H x_0$. A function of $z$,
such as $g_{00} = \cot^2 z$, depends on $x_0$ through $z$, and the chain rule
says $\partial_0 \cot^2(6 H x_0) = 6H \cdot \frac{d}{dz} \cot^2 z$. The cell
checks it with sympy, and the figure shows it with $H = 1$: the same curve
drawn against $z$ and against $x_0 = z/6$ is squeezed sideways by the factor 6,
so every slope is 6 times steeper.
```

In [7]:

```
x4, x0 = sp.symbols("x4 x0", real=True)  # the time and the hidden coordinate
H = sp.symbols("H", positive=True)  # the author's constant H > 0
zs = sp.symbols("z", positive=True)  # z = 6 H x0 as a symbol of its own
through_x0 = sp.diff(sp.cot(6 * H * x0) ** 2, x0)  # differentiate in x0 directly
```

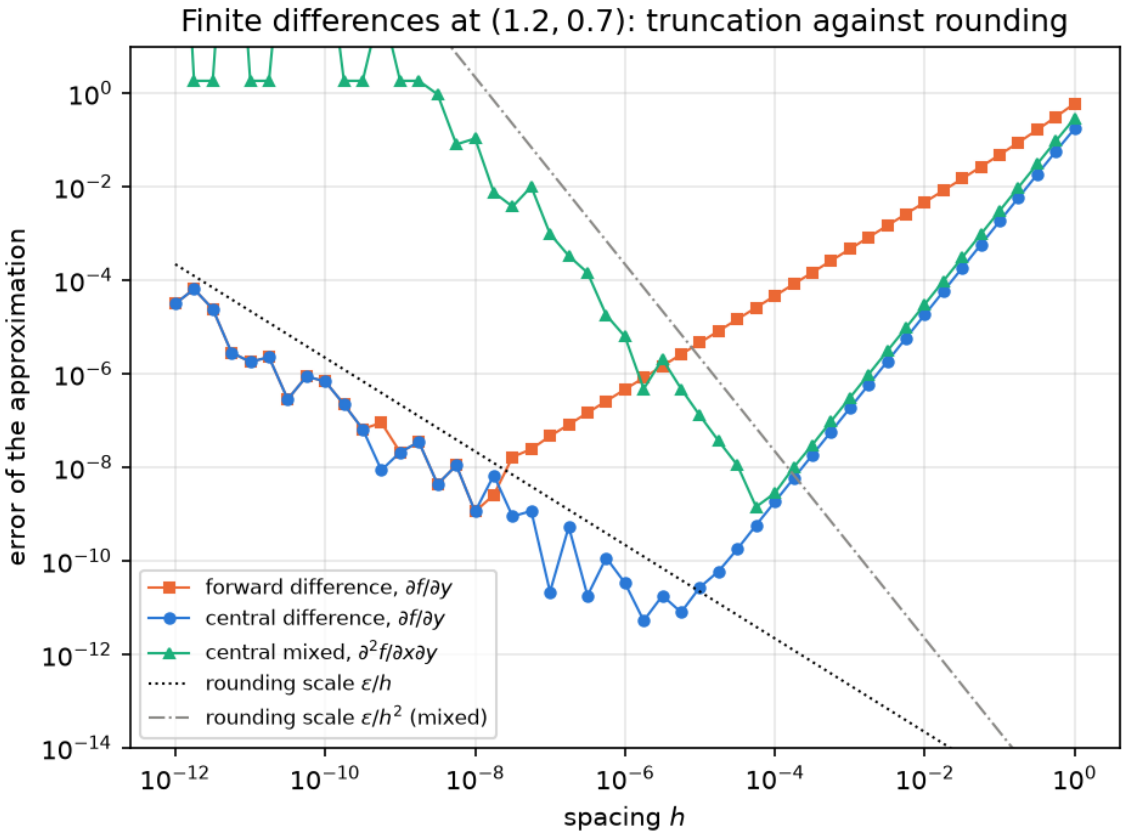

Figure 80: The error of three finite-difference approximations at the point $(1.2, 0.7)$ against the spacing h from 10^{-12} to 1, on logarithmic axes (pure numbers): forward difference for $\partial f/\partial y$ (squares), central difference for $\partial f/\partial y$ (circles) and the central formula for the mixed derivative (triangles), for $f = x^2 \sin y$. Coming from the right, the errors fall with slope 1 (forward) and slope 2 (central, mixed). Coming from the left, rounding errors, of the size ϵ/h (dotted) or ϵ/h^2 (dash-dotted) for the mixed formula, dominate. The best spacings are near 10^{-8}, 10^{-6} and 10^{-4}. (Notebook 02c, figure 3.)

```
through_z = 6 * H * sp.diff(sp.cot(zs) ** 2, zs).subs(zs, 6 * H * x0)  # chain rule
say(f"d/dx0 cot(6 H x0)^2 = {sp.simplify(through_x0)}")
check(sp.simplify(through_x0 - through_z) == 0,
      "chain rule: d/dx0 of cot(6 H x0)^2 = 6 H times d/dz of cot(z)^2")

z0 = 0.6  # the point z0 and the matching x0 = z0/6 (H = 1)
slope_z = -2.0 * math.cos(z0) / math.sin(z0) ** 3  # d/dz cot^2 z = -2 cot z / sin^2 z
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
z_line = np.linspace(0.35, 1.5, 300)
left.plot(z_line, 1 / np.tan(z_line) ** 2, color=BLUE, lw=1.6,
          label="$\\cot^2 z$")
left.plot(z_line, 1 / math.tan(z0) ** 2 + slope_z * (z_line - z0), "--",
          color=ORANGE, label=f"tangent, slope {slope_z:.3f}")
left.set_xlabel("$z$")
x0_line = z_line / 6.0
right.plot(x0_line, 1 / np.tan(6 * x0_line) ** 2, color=BLUE, lw=1.6,
           label="$\\cot^2(6 H x_0)$, $H = 1$")
right.plot(x0_line, 1 / math.tan(z0) ** 2 + 6 * slope_z * (x0_line - z0 / 6), "--",
           color=ORANGE, label=f"tangent, slope $6 \\times$ {slope_z:.3f}")
right.set_xlabel("$x_0$ (units $1/H$)")
for ax, where in ((left, z0), (right, z0 / 6)):
    ax.plot([where], [1 / math.tan(z0) ** 2], "o", color=BLACK, ms=6)
    ax.set_ylim(0.0, 6.0)
    ax.set_ylabel("$g_{00}$ (pure number)")
    ax.legend(fontsize=8)
save_figure(fig, "chain_rule",
            "The chain rule for the author's hidden coordinate. Left: "
            "$g_{00} = \\cot^2 z$ against $z$ from 0.35 to 1.5, with its tangent "
            "line at $z = 0.6$. Right: the same function against $x_0 = z/(6H)$ "
            "with $H = 1$ ($x_0$ in units of $1/H$), with its tangent line at "
            "$x_0 = 0.1$. Both vertical axes show $g_{00}$, a pure number. The "
            "right curve is the left one squeezed sideways by the factor 6, so "
            "its slope is 6 times steeper: $\\partial_0 = 6H\\, d/dz$.")
```

Out [7]:

```
d/dx0 cot(6 H x0)^2 = -12*H*cot(6*H*x0)/sin(6*H*x0)**2
PASS chain rule: d/dx0 of cot(6 H x0)^2 = 6 H times d/dz of cot(z)^2
```

Out [7]:

```
Figure 02c.4 saved as Revision/textbook/figures/02c_4_chain_rule.png
```

Text cell:

```
## 10. The author's metric, read from the Revision record

The Rust program of the Revision record that computed the curvature of the
author's metric stored the metric's diagonal in the file
Revision/gkd_lovelock/results/curvature.json, written in the notation of the
Wolfram Language (`E^(...)` is $e^{\dots}$, `Sin[...]` is $\sin(\dots)$,
```

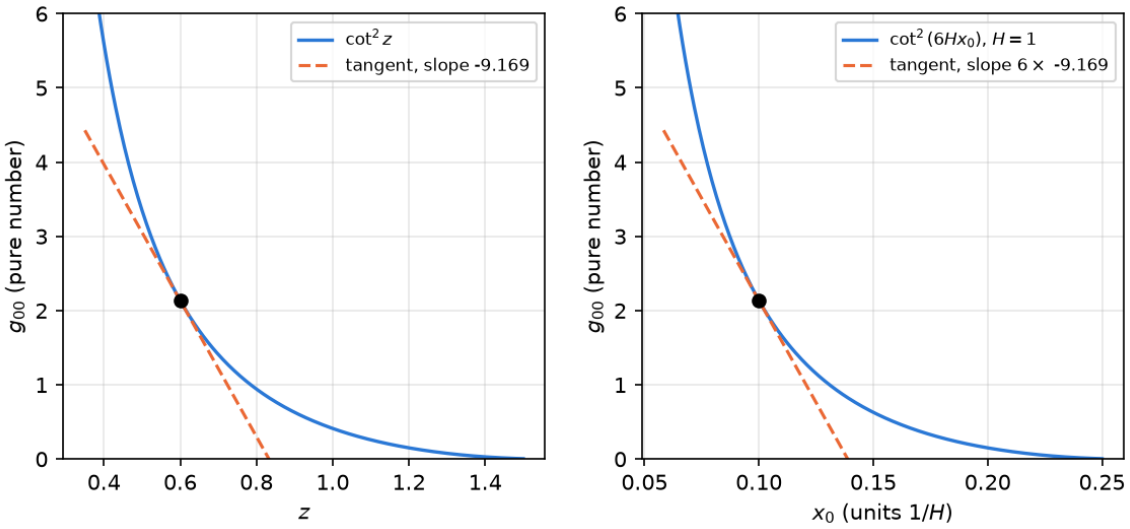

Figure 81: The chain rule for the author's hidden coordinate. Left: $g_{00} = \cot^2 z$ against z from 0.35 to 1.5, with its tangent line at $z = 0.6$. Right: the same function against $x_0 = z/(6H)$ with $H = 1$ (x_0 in units of $1/H$), with its tangent line at $x_0 = 0.1$. Both vertical axes show g_{00}, a pure number. The right curve is the left one squeezed sideways by the factor 6, so its slope is 6 times steeper: $\partial_0 = 6H\, d/dz$. (Notebook 02c, figure 4.)

```
‘Derivative[1][a4][x4]‘ is $a_4’$). The function ‘from_record‘ translates such a
text into a sympy expression: square brackets become round ones and ‘^‘ becomes
‘**‘. The cell compares the eight entries with the formulas of section 4,
typed in by hand.
```

In [8]:

```
CURVATURE = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE).read_text(encoding="utf-8"))
a4 = sp.Function("a4")  # the unknown function a4(x4) of the metric
b4 = sp.Function("b4", positive=True)  # the unknown function b4(x4) > 0
NAMES = {"a4": a4, "b4": b4, "x4": x4, "x0": x0, "H": H, "Sin": sp.sin,
         "Cos": sp.cos, "Cot": sp.cot, "Tan": sp.tan}


def from_record(text):
    """A Wolfram-Language text of the record as a sympy expression."""
    for f in ("a4", "b4"):  # Derivative[n][f][x4] is the n-th derivative of f
        for n in (1, 2):
            text = text.replace(f"Derivative[{n}][{f}][x4]",
                                f"Derivative({f}(x4), (x4, {n}))")
    text = text.replace("[", "(").replace("]", ")").replace("^", "**")
    return sp.parse_expr(text, local_dict=NAMES)


def tidy(expression):
    """Write cot and tan as quotients of cos and sin (helps sympy simplify)."""
```

```
    expression = expression.replace(sp.cot, lambda u: sp.cos(u) / sp.sin(u))
    return expression.replace(sp.tan, lambda u: sp.sin(u) / sp.cos(u))


def same(first, second):
    """True when sympy simplifies first - second to zero."""
    return sp.simplify(tidy(first - second)) == 0


COORDS = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
X = [sp.Symbol(name, real=True) for name in COORDS]
X[4], X[0] = x4, x0  # the metric depends on these two only
z = 6 * H * x0
third = sp.Rational(1, 3)
# the author's order x0, x1, ..., x7
BY_HAND = ([sp.cot(z) ** 2] + [sp.exp(2 * a4(x4)) * sp.sin(z) ** third] * 3
           + [sp.Integer(-1)]
           + [-b4(x4) * sp.exp(-2 * a4(x4)) * sp.sin(z) ** third] * 3)
G_RECORD = [from_record(text) for text in record["metricDiagonal"]]
for name, entry in zip(COORDS, G_RECORD):
    say(f"g_{name[1]}{name[1]} = {entry}")
check(record["coordinates"] == COORDS and len(G_RECORD) == 8
      and all(same(a, b) for a, b in zip(G_RECORD, BY_HAND)),
      "the record's metric equals the author's metric typed in by hand",
      record=f"{CURVATURE}, key metricDiagonal")
G = sp.diag(*G_RECORD)  # the metric as an 8 x 8 diagonal sympy matrix
```

Out [8]:

```
g_00 = cot(6*H*x0)**2
g_11 = exp(2*a4(x4))*sin(6*H*x0)**(1/3)
g_22 = exp(2*a4(x4))*sin(6*H*x0)**(1/3)
g_33 = exp(2*a4(x4))*sin(6*H*x0)**(1/3)
g_44 = -1
g_55 = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
g_66 = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
g_77 = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
PASS the record's metric equals the author's metric typed in by hand
     reproduces Revision/gkd_lovelock/results/curvature.json, key metricDiagonal
```

Text cell:

```
## 11. How fast the scale factors change: rates along $x_4$ and $x_0$

The next cell computes the logarithmic rates $\partial_c \ln s_a =
\partial_c g_{aa}/(2 g_{aa})$ of the eight scale factors with respect to $x_4$
and $x_0$ and checks the formulas of section 4. It then evaluates them along the
linear history $a_4 = A H x_4$ with $A = 1$ and $H = 1$ (so $a_4' = 1$) and
$b_4 = 1$ (so $b_4' = 0$), the prescribed background that the book uses for its
Kohn-Sham states: the bar chart shows the rates along $x_4$, $0$ for $x_0$ and
$x_4$, $+1$ for the three 3-space directions (inflation) and $-1$ for the three
extra times (deflation). Their sum is $\frac{3}{2} b_4'/b_4$, zero for constant
```

```
$b_4$, which is why the volume then does not change with time.
sympy prints $H \cot z$ as `H/tan(6*H*x0)` and $-6H/(\sin z \cos z)$ as
`-12*H/sin(12*H*x0)`; these are the same, because $\sin 2z = 2 \sin z \cos z$.
```

In [9]:

```
rate4 = [sp.simplify(sp.diff(G[a, a], x4) / (2 * G[a, a])) for a in range(8)]
rate0 = [sp.simplify(tidy(sp.diff(G[a, a], x0) / (2 * G[a, a]))) for a in range(8)]
da4 = sp.diff(a4(x4), x4)  # a4', the derivative of a4
db4 = sp.diff(b4(x4), x4)  # b4', the derivative of b4
# the author's order x0, x1, ..., x7
expected4 = [0] + [da4] * 3 + [0] + [-da4 + db4 / (2 * b4(x4))] * 3
expected0 = ([-6 * H / (sp.sin(z) * sp.cos(z))] + [H * sp.cot(z)] * 3 + [0]
             + [H * sp.cot(z)] * 3)
for a, name in enumerate(COORDS):
    say(f"{name}: d4 ln s = {rate4[a]},  d0 ln s = {rate0[a]}")
check(all(same(r, e) for r, e in zip(rate4, expected4))
      and all(same(r, e) for r, e in zip(rate0, expected0)),
      "the rates a4', -a4' + b4'/(2 b4), H cot z and -6H/(sin z cos z) of section 4")
check(same(sum(rate4), sp.Rational(3, 2) * db4 / b4(x4))
      and sp.simplify(sum(rate4).subs(db4, 0)) == 0,
      "the eight rates along x4 add up to (3/2) b4'/b4, zero for constant b4")

rates_now = [float(r.subs({da4: 1, db4: 0})) for r in rate4]  # a4' = 1, b4' = 0
fig, ax = plt.subplots()
colors = [GREY] + [BLUE] * 3 + [GREY] + [ORANGE] * 3
ax.bar(range(8), rates_now, color=colors, width=0.6)
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_xticks(range(8), [f"$x_{k}$" for k in range(8)])
for k, value in enumerate(rates_now):
    ax.text(k, value + (0.06 if value >= 0 else -0.14), f"{value:+.0f}",
            ha="center", fontsize=9)
ax.set_ylim(-1.4, 1.4)
ax.set_xlabel("direction")
ax.set_ylabel("$\\partial_4 \\ln s_a$ (units $H$)")
ax.set_title("Growth rate of each scale factor along the time $x_4$ "
             "($A = H = 1$, $b_4 = 1$)")
save_figure(fig, "rates_along_x4",
            "The rate $\\partial_4 \\ln s_a$ at which the scale factor $s_a$ of "
            "each direction changes with the time $x_4$ (vertical axis, units of "
            "$H$), along the linear history $a_4 = A H x_4$ with $A = H = 1$ and "
            "$b_4 = 1$. "
            "The three 3-space directions $x_1, x_2, x_3$ (blue) grow at the rate "
            "$+a_4' = +1$: 3-space inflates. The three extra times $x_5, x_6, x_7$ "
            "(orange) shrink at the rate $-a_4' + b_4'/(2 b_4) = -1$: they "
            "deflate exponentially. The time $x_4$ and the hidden direction $x_0$ "
            "(grey) do not change with $x_4$. The eight rates add up to zero.")
```

Out [9]:

```
x0: d4 ln s = 0,  d0 ln s = -12*H/sin(12*H*x0)
x1: d4 ln s = Derivative(a4(x4), x4),  d0 ln s = H/tan(6*H*x0)
```

```
x2: d4 ln s = Derivative(a4(x4), x4),  d0 ln s = H/tan(6*H*x0)
x3: d4 ln s = Derivative(a4(x4), x4),  d0 ln s = H/tan(6*H*x0)
x4: d4 ln s = 0,  d0 ln s = 0
x5: d4 ln s = -Derivative(a4(x4), x4) + Derivative(b4(x4), x4)/(2*b4(x4)),  d0 ln s =
    H/tan(6*H*x0)
x6: d4 ln s = -Derivative(a4(x4), x4) + Derivative(b4(x4), x4)/(2*b4(x4)),  d0 ln s =
    H/tan(6*H*x0)
x7: d4 ln s = -Derivative(a4(x4), x4) + Derivative(b4(x4), x4)/(2*b4(x4)),  d0 ln s =
    H/tan(6*H*x0)
PASS the rates a4', -a4' + b4'/(2 b4), H cot z and -6H/(sin z cos z) of section 4
PASS the eight rates along x4 add up to (3/2) b4'/b4, zero for constant b4
```

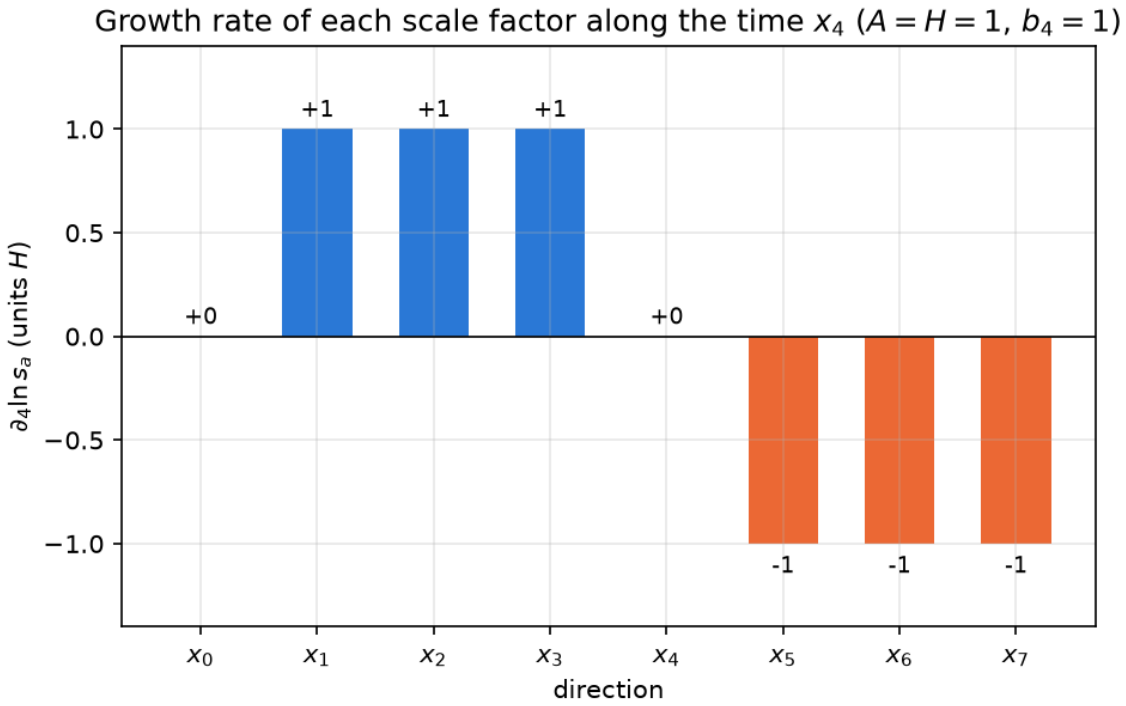

Figure 82: The rate $\partial_4 \ln s_a$ at which the scale factor s_a of each direction changes with the time x_4 (vertical axis, units of H), along the linear history $a_4 = AHx_4$ with $A = H = 1$ and $b_4 = 1$. The three 3-space directions x_1, x_2, x_3 (blue) grow at the rate $+a_4' = +1$: 3-space inflates. The three extra times x_5, x_6, x_7 (orange) shrink at the rate $-a_4' + b_4'/(2b_4) = -1$: they deflate exponentially. The time x_4 and the hidden direction x_0 (grey) do not change with x_4. The eight rates add up to zero. (Notebook 02c, figure 5.)

Out [9]:

```
Figure 02c.5 saved as Revision/textbook/figures/02c_5_rates_along_x4.png
```

Text cell:

```
The rates along the hidden coordinate depend on $z$. The next cell draws them
with $H = 1$ for $0.05 < z < \pi/2 - 0.05$: $H \cot z$ for the 3-space and
```

```
extra-time factors, $-6H/(\sin z \cos z)$ for the hidden factor $\cot z$, and
their sum over all eight directions, which must be the logarithmic rate of the
volume factor, $\partial_0 \ln (b_4^{3/2} \cos z) = -6H \tan z$. The check
confirms the sum
exactly with sympy.
```

In [10]:

```
rate_sum0 = sp.simplify(tidy(sum(rate0)))
check(same(rate_sum0, -6 * H * sp.tan(z)),
      "the eight rates along x0 add up to d0 ln cos z = -6 H tan z")
z_line = np.linspace(0.05, np.pi / 2 - 0.05, 400)
fig, ax = plt.subplots()
ax.plot(z_line, 1 / np.tan(z_line), color=BLUE, lw=1.6,
        label="$H \\cot z$: directions $x_1, x_2, x_3$ and $x_5, x_6, x_7$")
ax.plot(z_line, -6 / (np.sin(z_line) * np.cos(z_line)), "--", color=VIOLET, lw=1.6,
        label="$-6H/(\\sin z \\cos z)$: direction $x_0$")
ax.plot(z_line, -6 * np.tan(z_line), "-.", color=ORANGE, lw=1.6,
        label="sum of all eight $= -6 H \\tan z$")
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_ylim(-60.0, 20.0)
ax.set_xlabel("$z = 6 H x_0$")
ax.set_ylabel("$\\partial_0 \\ln s_a$ (units $H$)")
ax.set_title("Rates of the scale factors along the hidden coordinate ($H = 1$)")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "rates_along_x0",
            "The rates $\\partial_0 \\ln s_a$ at which the scale factors change "
            "along the hidden coordinate $x_0$ (vertical axis, units of $H$, "
            "$H = 1$), against $z = 6 H x_0$ from 0.05 to $\\pi/2 - 0.05$. Solid "
            "blue: $H \\cot z$, the same for the three 3-space and the three "
            "extra-time directions. Dashed violet: $-6H/(\\sin z \\cos z)$ for "
            "the hidden direction, whose scale factor is $\\cot z$. Dash-dotted "
            "orange: the sum of all eight rates, which equals the rate "
            "$-6 H \\tan z$ of the volume factor $b_4^{3/2} \\cos z$.")
```

Out [10]:

```
PASS the eight rates along x0 add up to d0 ln cos z = -6 H tan z
```

Out [10]:

```
Figure 02c.6 saved as Revision/textbook/figures/02c_6_rates_along_x0.png
```

Text cell:

```
## 12. The volume factor and Jacobi's formula

The determinant of the diagonal metric is the product of its entries. The cell
computes it from the record's metric, compares $\sqrt{|\det g|}$ with the
```

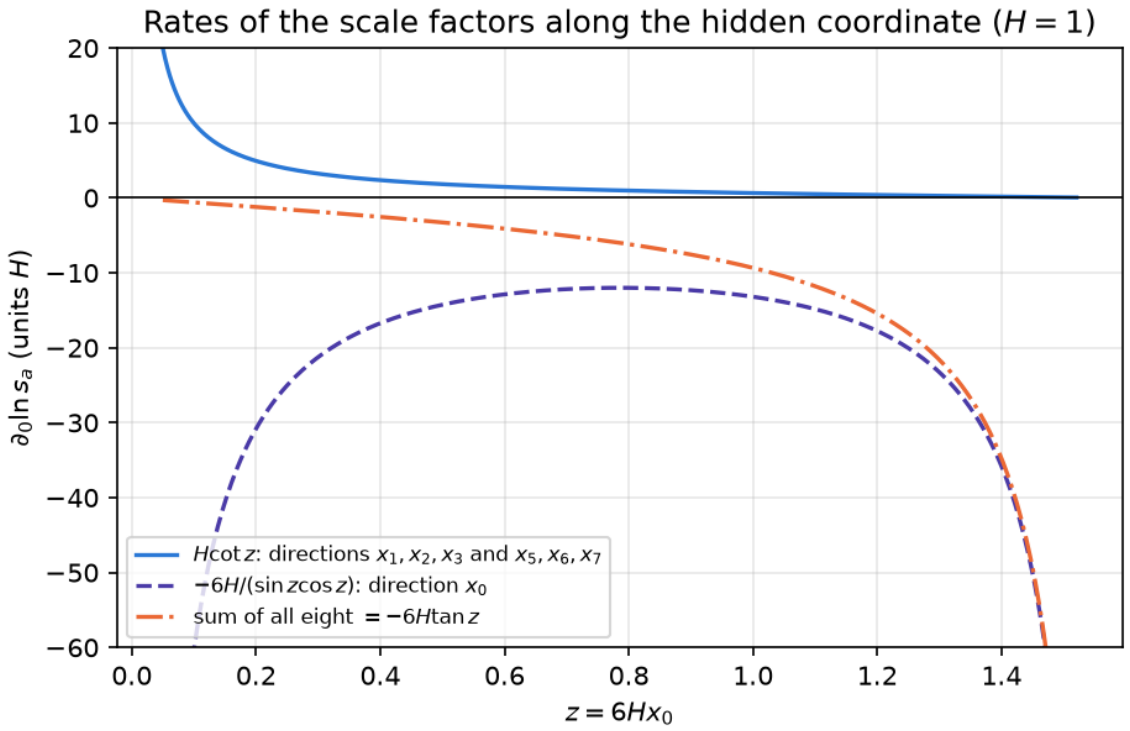

Figure 83: The rates $\partial_0 \ln s_a$ at which the scale factors change along the hidden coordinate x_0 (vertical axis, units of H, $H = 1$), against $z = 6Hx_0$ from 0.05 to $\pi/2 - 0.05$. Solid blue: $H \cot z$, the same for the three 3-space and the three extra-time directions. Dashed violet: $-6H/(\sin z \cos z)$ for the hidden direction, whose scale factor is $\cot z$. Dash-dotted orange: the sum of all eight rates, which equals the rate $-6H \tan z$ of the volume factor $b_4^{3/2} \cos z$. (Notebook 02c, figure 6.)

```
record's own entry 'sqrtAbsDetG' ($b_4^{3/2} \sin z \cot z$) and with
$b_4^{3/2} \cos z$ (the Wolfram check 'sqrt_abs_det_g_is_cos_z' of the Revision
record Revision/field_equations_a4/reports/wolfram-a4-report.json), and checks
that its partial derivative with respect to $x_4$ is
$\frac{3}{2} (b_4'/b_4) \sqrt{|\det g|}$, zero for constant $b_4$: then the
volume factor does not change with time, for EVERY function $a_4(x_4)$. Finally
it checks
Jacobi's formula $\partial_c \sqrt{|\det g|} = \frac{1}{2} \sqrt{|\det g|}\,
\mathrm{tr}(g^{-1} \partial_c g)$ for $c = x_4$ and $c = x_0$ (tr, the trace,
is the sum of the diagonal entries of a matrix).
```

In [11]:

```
det_g = sp.simplify(tidy(G.det()))  # the product of the eight entries
say(f"det g = {det_g}")
sqrt_record = from_record(record["sqrtAbsDetG"])
say(f"sqrtAbsDetG of the record = {sqrt_record}")
check(same(det_g, b4(x4) ** 3 * sp.cos(z) ** 2)
      and same(sqrt_record, b4(x4) ** sp.Rational(3, 2) * sp.cos(z)),
      "det g = b4^3 cos(z)^2 and the record's sqrt|det g| = b4^(3/2) sin z cot z "
      "= b4^(3/2) cos z",
      record=f"{CURVATURE}, key sqrtAbsDetG")
A4REPORT = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
a4_report = json.loads(repository_file(A4REPORT).read_text(encoding="utf-8"))
verdict = [c["verdict"] for c in a4_report["checks"]
           if c["name"] == "sqrt_abs_det_g_is_cos_z"]
db4 = sp.diff(b4(x4), x4)  # b4', the derivative of b4
check(verdict == ["PASS"]
      and same(sp.diff(sqrt_record, x4),
               sp.Rational(3, 2) * db4 / b4(x4) * sqrt_record)
      and sp.simplify(sp.diff(sqrt_record, x4).subs(db4, 0)) == 0,
      "d4 sqrt|det g| = (3/2)(b4'/b4) sqrt|det g|: for constant b4 the volume "
      "factor does not depend on x4 (inflation and deflation cancel)",
      record=f"{A4REPORT}, check sqrt_abs_det_g_is_cos_z")
jacobi = [same(sp.diff(sqrt_record, c),
               sp.Rational(1, 2) * sqrt_record * (G.inv() * G.diff(c)).trace())
          for c in (x4, x0)]
check(all(jacobi), "Jacobi's formula for the derivative of sqrt|det g| (x4 and x0)")
```

Out [11]:

```
det g = b4(x4)**3*cos(6*H*x0)**2
sqrtAbsDetG of the record = b4(x4)**(3/2)*sin(6*H*x0)*cot(6*H*x0)
PASS det g = b4^3 cos(z)^2 and the record's sqrt|det g| = b4^(3/2) sin z cot z = b4^(3/2)
    cos z
     reproduces Revision/gkd_lovelock/results/curvature.json, key sqrtAbsDetG
PASS d4 sqrt|det g| = (3/2)(b4'/b4) sqrt|det g|: for constant b4 the volume factor does
    not depend on x4 (inflation and deflation cancel)
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    sqrt_abs_det_g_is_cos_z
PASS Jacobi's formula for the derivative of sqrt|det g| (x4 and x0)
```

Text cell:

```
## 13. All 25 Christoffel entries of the record

The record lists every nonzero $\Gamma^a{}_{bc}$ with $b$ not after $c$ in the
order $x_0, \dots, x_7$ (the symbol is symmetric in $b$ and $c$). The next cell
computes all $8 \times 36 = 288$ such entries from the formula of section 4,
keeps the nonzero ones, and compares them one by one with the record. Only
first partial derivatives of the metric entries are needed.
```

In [12]:

```
computed = {}
for a in range(8):
    for b in range(8):
        for c in range(b, 8):
            value = (sp.diff(G[a, c], X[b]) + sp.diff(G[a, b], X[c])
                     - sp.diff(G[b, c], X[a])) / (2 * G[a, a])
            value = sp.simplify(tidy(value))
            if value != 0:
                computed[(COORDS[a], COORDS[b], COORDS[c])] = value
listed = {(e["a"], e["b"], e["c"]): from_record(e["value"])
          for e in record["christoffelNonzero_b_le_c"]}
for key in [("x1", "x1", "x4"), ("x5", "x4", "x5"), ("x0", "x1", "x1"),
            ("x0", "x0", "x0")]:
    say(f"Gamma^{key[0]}_({key[1]} {key[2]}) = {computed[key]}")
report("nonzero entries computed / listed in the record",
       f"{len(computed)} / {len(listed)}")
check(sorted(computed) == sorted(listed)
      and all(same(computed[k], listed[k]) for k in listed),
      "all 25 nonzero Christoffel entries equal those of the record",
      record=f"{CURVATURE}, key christoffelNonzero_b_le_c")
```

Out [12]:

```
Gamma^x1_(x1 x4) = Derivative(a4(x4), x4)
Gamma^x5_(x4 x5) = -Derivative(a4(x4), x4) + Derivative(b4(x4), x4)/(2*b4(x4))
Gamma^x0_(x1 x1) = -H*exp(2*a4(x4))*sin(6*H*x0)**(4/3)/cos(6*H*x0)
Gamma^x0_(x0 x0) = -12*H/sin(12*H*x0)
RESULT nonzero entries computed / listed in the record = 25 / 25
PASS all 25 nonzero Christoffel entries equal those of the record
     reproduces Revision/gkd_lovelock/results/curvature.json, key
    christoffelNonzero_b_le_c
```

Text cell:

```
## 14. The last check

The last cell checks that all 6 figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [13]:

```
names = ["level_curves_gradient", "slices_and_slopes", "finite_differences",
         "chain_rule", "rates_along_x4", "rates_along_x0"]
present = [output_file(f"{FIGURE_FOLDER}/02c_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02c exist")
all_checks_passed()
```

Out [13]:

```
PASS all 6 figure files of notebook 02c exist
ALL 17 CHECKS PASSED (notebook 02c)
```

Text cell:

```
## 15. What this notebook showed

- A partial derivative is an ordinary derivative with the other variables held
  fixed: the slope of a slice. The gradient points uphill, perpendicular to the
  level curves. The two mixed derivatives of $x^2 \sin y$ agree (Schwarz).
- Forward differences have order 1, central differences order 2; rounding
  errors grow like $\epsilon/h$, so the best spacing is about $10^{-8}$
  (forward) or $10^{-6}$ (central), not the smallest one.
- The chain rule for $z = 6 H x_0$: $\partial_0 = 6H\, d/dz$.
- The author's metric, read from the Revision record, has the scale-factor rates
  $+a_4'$ (3-space, inflating), $-a_4' + b_4'/(2 b_4)$ (the three extra times,
  deflating), $0$ (the time $x_4$ and the hidden $x_0$) along $x_4$, and
  $H \cot z$, $H \cot z$ and $-6H/(\sin z \cos z)$ along $x_0$.
- $\sqrt{|\det g|} = b_4^{3/2} \sin z \cot z = b_4^{3/2} \cos z$ for every
  $a_4(x_4)$: the inflation of 3-space and the deflation of the extra times
  cancel in the volume, which changes with $x_4$ only through $b_4$ (reproduces
  the record's sqrtAbsDetG and the Wolfram check sqrt_abs_det_g_is_cos_z).
  Jacobi's formula holds.
- All 25 nonzero Christoffel entries of the record follow from first partial
  derivatives of the metric.
```

2.23 Line-by-line walk-through of Notebook 02c

The notebook has thirteen code cells, In [1] to In [13]. A quoted line `...)` stands for the rest of a figure caption, which Section 2.22 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 2.21; its code is the set-up code of Notebook 02a explained in Section 2.11, with `NOTEBOOK_ID = "02c"`. It prints Set-up of notebook 02c complete: repository folder found, helpers defined.

In [2], packages and symbols.

```
import math  # functions of single numbers
```

```
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra and calculus with symbols

BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
GREY, BLACK = "#8a8986", "#000000"
x, y = sp.symbols("x y", real=True)  # the two variables of the first example
say("Packages imported; symbols x and y defined.")
```

The packages and the colours as in Notebook 02b, and two sympy symbols x and y, declared real (so that sympy may use rules that hold for real numbers only).

In [3], partial derivatives with sympy.

```
f = x ** 2 * sp.sin(y)
f_x = sp.diff(f, x)  # hold y fixed
f_y = sp.diff(f, y)  # hold x fixed
f_xy = sp.diff(f, x, y)  # first x, then y
f_yx = sp.diff(f, y, x)  # first y, then x
```

`f` is the expression $x^2 \sin y$. `sp.diff(f, x)` differentiates with respect to x and treats every other symbol as a constant, which is exactly the definition of the partial derivative of Section 2.18. `sp.diff(f, x, y)` differentiates first with respect to x and then with respect to y; `sp.diff(f, y, x)` in the other order.

```
say(f"df/dx = {f_x},  df/dy = {f_y}")
say(f"d2f/dxdy = {f_xy},  d2f/dydx = {f_yx}")
check(sp.simplify(f_x - 2 * x * sp.sin(y)) == 0
      and sp.simplify(f_y - x ** 2 * sp.cos(y)) == 0,
      "df/dx = 2 x sin y and df/dy = x^2 cos y")
check(sp.simplify(f_xy - f_yx) == 0 and sp.simplify(f_xy - 2 * x * sp.cos(y)) == 0,
      "the mixed derivatives agree (Schwarz): both are 2 x cos y")
```

The four derivatives are printed (`2*x*sin(y)`, `x**2*cos(y)`, and `2*x*cos(y)` twice), and two checks compare them with the hand calculation of Section 2.18: `sp.simplify` reduces the difference to its simplest form, which must be 0. The second check is Schwarz's theorem for this example.

```
X0, Y0 = 1.2, 0.7  # the point at which we evaluate
at_point = {x: X0, y: Y0}
for label, expression in (("f", f), ("df/dx", f_x), ("df/dy", f_y),
                          ("d2f/dxdy", f_xy)):
    report(f"{label} at (1.2, 0.7)", f"{float(expression.subs(at_point)):.12f}")
```

The dictionary `at_point` tells `.subs` to put $x = 1.2$ and $y = 0.7$ into an expression; `float` turns the result into an ordinary number. The RESULT lines give $f = 0.927673469622$, $\partial f/\partial x = 1.546122449370$, $\partial f/\partial y = 1.101372749690$ and $\partial^2 f/\partial x \partial y = 1.835621249483$ (COMPUTED; for example $2 \cdot 1.2 \cdot \cos 0.7 = 1.8356$).

In [4], level curves and gradient arrows.

```
F = sp.lambdify((x, y), f, "numpy")  # f as a numpy function
FX = sp.lambdify((x, y), f_x, "numpy")
FY = sp.lambdify((x, y), f_y, "numpy")
```

`sp.lambdify` turns a sympy expression into an ordinary Python function of (x, y) that computes with numpy, so that it can be applied to whole arrays of points at once: `F` computes f, `FX` and `FY` its two partial derivatives.

```
xs = np.linspace(-2.0, 2.0, 201)
ys = np.linspace(0.0, 2.0 * np.pi, 201)
XX, YY = np.meshgrid(xs, ys)  # all grid points as two 2-dimensional arrays
```

201 values of x from -2 to 2 and 201 values of y from 0 to 2π. `np.meshgrid` makes the grid of all 201×201 pairs: `XX` holds the x of every grid point and `YY` its y, both as tables of 201 rows and 201 columns.

```
fig, ax = plt.subplots(figsize=(7.0, 5.2))
bands = ax.contourf(XX, YY, F(XX, YY), levels=17, cmap="RdBu_r")  # coloured bands
ax.contour(XX, YY, F(XX, YY), levels=17, colors=BLACK, linewidths=0.4)
fig.colorbar(bands, ax=ax, label="$f(x, y) = x^2 \\sin y$")
```

`contourf` fills the regions between 17 levels of f with colours from the colour map `"RdBu_r"` (red for large, blue for small values; `_r` reverses the map); `contour` draws the level curves themselves as thin black lines; `colorbar` adds the scale that translates colours into values.

```
xq, yq = np.meshgrid(np.linspace(-1.8, 1.8, 10), np.linspace(0.3, 6.0, 12))  # coarse
# an arrow (df/dx, df/dy) at every coarse grid point; scale=40 shortens all arrows
ax.quiver(xq, yq, FX(xq, yq), FY(xq, yq), color=BLACK, scale=40, width=0.004)
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
ax.set_title("Level curves of $f = x^2 \\sin y$ and its gradient arrows")
save_figure(fig, "level_curves_gradient",
            "A map of the function $f(x, y) = x^2 \\sin y$ for $x$ from $-2$ to $2$ "
            ...)
```

A coarse grid of 10×12 points carries the gradient arrows: `quiver` draws at each point (x, y) an arrow with the components $(\partial f/\partial x, \partial f/\partial y)$, all shortened by the same factor (`scale=40`) so that they fit; `width` sets their thickness. Labels and the saved figure `02c_1_level_curves_gradient.png`. **What Figure 02c.1 shows:** every arrow points uphill, from blue towards red, and crosses the level curves at a right angle; where the level curves crowd together the arrows are long, because the slope is steep (Section 2.18).

```
gx, gy = float(f_x.subs(at_point)), float(f_y.subs(at_point))
tangent = np.array([-gy, gx]) / math.hypot(gx, gy)  # along the level curve
step = 1e-4
change = F(X0 + step * tangent[0], Y0 + step * tangent[1]) - F(X0, Y0)
report("change of f along the level curve for a step of 1e-4", f"{change:.2e}")
check(abs(gx * tangent[0] + gy * tangent[1]) < 1e-15 and abs(change) < 1e-7,
      "the gradient is perpendicular to the level curve")
```

`gx` and `gy` are the gradient's components at $(1.2, 0.7)$. `math.hypot(gx, gy)` is the length $\sqrt{g_x^2 + g_y^2}$, so `tangent` is the unit arrow $(-g_y, g_x)/|\nabla f|$ of Section 2.18. A step of 10^{-4} along it changes f by only -9.58×10^{-9} (COMPUTED), a second-order amount (10^{-8} is the square of 10^{-4}). The check requires the dot product of the gradient with this direction to vanish (up to rounding) and the change of f to be below 10^{-7}.

```
length = math.hypot(gx, gy)  # the length |grad f| of the gradient arrow
rise = F(X0 + step * gx / length, Y0 + step * gy / length) - F(X0, Y0)
report("rise of f per unit length along the gradient, |grad f|",
       f"{rise / step:.6f}, {length:.6f}")
```

Now the second fact of Section 2.18. `length` is $|\nabla f| = \sqrt{g_x^2 + g_y^2}$, and $(g_x, g_y)/|\nabla f|$ is the unit arrow along the gradient. `rise` is the change of f for a step of 10^{-4} along it, and `rise / step` the change per unit length. The RESULT line prints 1.898407 and $|\nabla f| = 1.898293$ (COMPUTED): along the gradient f rises at the rate $|\nabla f|$, the steepest slope; the two differ by 1.1×10^{-4}, the second-order term of the step.

```
gxq, gyq = FX(xq, yq), FY(xq, yq)  # the arrows of the figure
lengths = np.hypot(gxq, gyq)
rises = F(xq + step * gxq / lengths, yq + step * gyq / lengths) - F(xq, yq)
check(abs(rise / step / length - 1) < 1e-3 and np.all(rises > 0),
      "the gradient points uphill: f rises at the rate |grad f| along it")
```

The same test at all 120 points of the coarse grid of the figure: `gxq`, `gyq` are the components of the 120 arrows, `np.hypot` their lengths (one for each point), and `rises` the change of f for a step of 10^{-4} along each arrow. The check requires the measured rate at $(1.2, 0.7)$ to equal $|\nabla f|$ within one part in a thousand, and every one of the 120 rises to be positive: every arrow of Figure 02c.1 points uphill.

In [5], two slices and their slopes.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
fig.subplots_adjust(wspace=0.3)  # room between the two panels
f0 = F(X0, Y0)
x_line = np.linspace(-0.5, 2.5, 200)
left.plot(x_line, F(x_line, Y0), color=BLUE, lw=1.6,
          label="slice $f(x, 0.7)$")
left.plot(x_line, f0 + gx * (x_line - X0), "--", color=ORANGE, lw=1.2,
          label=f"slope $\\partial f/\\partial x = {gx:.4f}$")
left.set_xlabel("$x$ (with $y = 0.7$ fixed)")
```

Two panels with some room between them (`wspace`). `f0` is $f(1.2, 0.7)$. The left panel draws the slice $x \mapsto f(x, 0.7)$ and the straight line $f_0 + g_x(x - x_0)$ through the point with the slope $\partial f/\partial x$.

```
y_line = np.linspace(-0.6, 2.4, 200)
right.plot(y_line, F(X0, y_line), color=BLUE, lw=1.6, label="slice $f(1.2, y)$")
right.plot(y_line, f0 + gy * (y_line - Y0), "--", color=ORANGE, lw=1.2,
           label=f"slope $\\partial f/\\partial y = {gy:.4f}$")
right.set_xlabel("$y$ (with $x = 1.2$ fixed)")
```

The right panel does the same for the slice $y \mapsto f(1.2, y)$ and the slope $\partial f/\partial y$.

```
for ax in (left, right):
    ax.plot([X0 if ax is left else Y0], [f0], "o", color=BLACK, ms=6)
    ax.set_ylabel("$f$")
    ax.legend(fontsize=8)
save_figure(fig, "slices_and_slopes",
            "Two slices of $f(x, y) = x^2 \\sin y$ through the point "
            ...)
check(abs(gx - 2 * X0 * math.sin(Y0)) < 1e-15 and abs(gy - X0 ** 2 * math.cos(Y0))
      < 1e-15, "the slopes of the slices are 2 x0 sin y0 and x0^2 cos y0")
```

In each panel a black dot marks the point (x_0 in the left panel, y_0 in the right one: `X0 if ax is left else Y0` chooses); then the saved figure `02c_2_slices_and_slopes.png`, and a check that the two slopes are $2x_0 \sin y_0$ and $x_0^2 \cos y_0$. **What Figure 02c.2 shows:** each dashed line touches its slice at the dot: a partial derivative is the slope of a slice.

In [6], finite differences: order and rounding.

```
exact_y = X0 ** 2 * math.cos(Y0)  # df/dy at the point
exact_xy = 2 * X0 * math.cos(Y0)  # d2f/dxdy at the point
h_values = np.logspace(-12, 0, 49)  # 1e-12 ... 1
```

The exact values of $\partial f/\partial y$ and $\partial^2 f/\partial x \partial y$ at the point, and 49 spacings from 10^{-12} to 1, equally spaced in $\log_{10} h$ (`np.logspace(-12, 0, 49)` gives $10^{-12}, 10^{-11.75}, \ldots, 10^{0}$).

```
forward = np.array([(F(X0, Y0 + h) - F(X0, Y0)) / h for h in h_values])
central = np.array([(F(X0, Y0 + h) - F(X0, Y0 - h)) / (2 * h) for h in h_values])
mixed = np.array([(F(X0 + h, Y0 + h) - F(X0 + h, Y0 - h) - F(X0 - h, Y0 + h)
                   + F(X0 - h, Y0 - h)) / (4 * h * h) for h in h_values])
```

The three formulas of Section 2.18, evaluated for every spacing: the forward and the central difference in y, and the central formula for the mixed derivative.

```
err = {"forward": np.abs(forward - exact_y), "central": np.abs(central - exact_y),
       "mixed": np.abs(mixed - exact_xy)}
RANGES = {"forward": (1e-5, 1e-2), "central": (1e-3, 1e-1), "mixed": (1e-2, 1e-1)}
slope_of = {}
for name, (low, high) in RANGES.items():
    part = (h_values >= low) & (h_values <= high)
    slope_of[name], _ = np.polyfit(np.log10(h_values[part]),
                                   np.log10(err[name][part]), 1)
    best = int(np.argmin(err[name]))
    report(f"{name}: slope {slope_of[name]:.3f}, best h", f"{h_values[best]:.1e} "
           f"(error {err[name][best]:.1e})")
```

`err` holds the three error arrays. `RANGES` gives, for each formula, the range of spacings where the truncation error clearly dominates (larger than the rounding error, but small enough for the leading Taylor term to rule); `part` selects those spacings, and `np.polyfit` fits the slope there, as in

Notebook 02a. `np.argmin` finds the spacing with the smallest error. The RESULT lines: forward slope 1.000, best $h = 1.0 \times 10^{-8}$ (error 1.1×10^{-9}); central slope 2.000, best $h = 1.8 \times 10^{-6}$ (error 5.3×10^{-12}); mixed slope 2.000, best $h = 5.6 \times 10^{-5}$ (error 1.4×10^{-9}) (COMPUTED; compare the estimates of Section 2.18).

```
fig, ax = plt.subplots(figsize=(7.0, 5.0))
styles = {"forward": (ORANGE, "s", "forward difference, $\\partial f/\\partial y$"),
          "central": (BLUE, "o", "central difference, $\\partial f/\\partial y$"),
          "mixed": (AQUA, "^", "central mixed, $\\partial^2 f/\\partial x "
                               "\\partial y$")}
for name, (color, marker, label) in styles.items():
    shown = err[name] > 0  # an exact zero cannot be drawn on a log axis
    ax.loglog(h_values[shown], err[name][shown], marker=marker, color=color, ms=4,
              lw=1.0, label=label)
```

A colour, a marker and a legend text for each formula; each error curve is drawn on log-log axes, leaving out errors that happen to be exactly zero.

```
ax.loglog(h_values, 2.2e-16 / h_values, ":", color=BLACK, lw=1.0,
          label="rounding scale $\\epsilon / h$")
ax.loglog(h_values, 2.2e-16 / h_values ** 2, "-.", color=GREY, lw=1.0,
          label="rounding scale $\\epsilon / h^2$ (mixed)")
ax.set_ylim(1e-14, 10.0)
ax.set_xlabel("spacing $h$")
ax.set_ylabel("error of the approximation")
ax.set_title("Finite differences at $(1.2, 0.7)$: truncation against rounding")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "finite_differences",
            "The error of three finite-difference approximations at the point "
            ...)
```

The two rounding scales ϵ/h (dotted) and ϵ/h^2 (dash-dotted, for the mixed formula, which divides by h^2), labels and the saved figure `02c_3_finite_differences.png`. **What Figure 02c.3 shows:** read from the right, the errors fall along straight lines of slope 1 (forward) and 2 (central, mixed); read from the left, they follow the rising rounding lines; each curve has its lowest point where the two effects balance, near 10^{-8}, 10^{-6} and 10^{-4}.

```
check(abs(slope_of["forward"] - 1) < 0.05 and abs(slope_of["central"] - 2) < 0.05
      and abs(slope_of["mixed"] - 2) < 0.05,
      "measured orders: forward 1, central 2, mixed central 2")
check(err["central"].min() < 1e-10 < err["forward"].min() < 1e-7,
      "best central error < 1e-10 < best forward error < 1e-7")
```

Two checks: the measured orders are 1, 2 and 2 within 0.05; the best central error is below 10^{-10}, which is below the best forward error, which is below 10^{-7} (a higher order buys accuracy even at the best spacing).

In [7], the chain rule with $z = 6Hx_0$.

```
x4, x0 = sp.symbols("x4 x0", real=True)  # the time and the hidden coordinate
H = sp.symbols("H", positive=True)  # the author's constant H > 0
zs = sp.symbols("z", positive=True)  # z = 6 H x0 as a symbol of its own
```

Symbols for the time x_4, the hidden coordinate x_0, the author's constant H (positive) and z as a variable of its own.

```
through_x0 = sp.diff(sp.cot(6 * H * x0) ** 2, x0)  # differentiate in x0 directly
through_z = 6 * H * sp.diff(sp.cot(zs) ** 2, zs).subs(zs, 6 * H * x0)  # chain rule
say(f"d/dx0 cot(6 H x0)^2 = {sp.simplify(through_x0)}")
check(sp.simplify(through_x0 - through_z) == 0,
      "chain rule: d/dx0 of cot(6 H x0)^2 = 6 H times d/dz of cot(z)^2")
```

`through_x0` differentiates $\cot^2(6Hx_0)$ directly with respect to x_0; `through_z` differentiates $\cot^2 z$ with respect to z, puts $z = 6Hx_0$ back in with `.subs`, and multiplies by $6H$, as the chain rule of Section 2.19 says. The printed derivative is `-12*H*cot(6*H*x0)/sin(6*H*x0)**2`, and the check confirms that both ways agree.

```
z0 = 0.6  # the point z0 and the matching x0 = z0/6 (H = 1)
slope_z = -2.0 * math.cos(z0) / math.sin(z0) ** 3  # d/dz cot^2 z = -2 cot z / sin^2 z
```

At $z_0 = 0.6$ the slope of $\cot^2 z$ is $-2\cot z_0/\sin^2 z_0 = -2\cos z_0/\sin^3 z_0$ (Section 2.19).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
z_line = np.linspace(0.35, 1.5, 300)
left.plot(z_line, 1 / np.tan(z_line) ** 2, color=BLUE, lw=1.6,
          label="$\\cot^2 z$")
left.plot(z_line, 1 / math.tan(z0) ** 2 + slope_z * (z_line - z0), "--",
          color=ORANGE, label=f"tangent, slope {slope_z:.3f}")
left.set_xlabel("$z$")
```

The left panel draws $\cot^2 z = 1/\tan^2 z$ for z from 0.35 to 1.5 and its tangent line at z_0.

```
x0_line = z_line / 6.0
right.plot(x0_line, 1 / np.tan(6 * x0_line) ** 2, color=BLUE, lw=1.6,
           label="$\\cot^2(6 H x_0)$, $H = 1$")
right.plot(x0_line, 1 / math.tan(z0) ** 2 + 6 * slope_z * (x0_line - z0 / 6), "--",
           color=ORANGE, label=f"tangent, slope $6 \\times$ {slope_z:.3f}")
right.set_xlabel("$x_0$ (units $1/H$)")
```

The right panel draws the same function against $x_0 = z/6$ (with $H = 1$) and the tangent line with 6 times the slope, through the matching point $x_0 = 0.1$.

```
for ax, where in ((left, z0), (right, z0 / 6)):
    ax.plot([where], [1 / math.tan(z0) ** 2], "o", color=BLACK, ms=6)
    ax.set_ylim(0.0, 6.0)
    ax.set_ylabel("$g_{00}$ (pure number)")
    ax.legend(fontsize=8)
save_figure(fig, "chain_rule",
            "The chain rule for the author's hidden coordinate. Left: "
            ...)
```

A dot at the point in each panel, the vertical range, labels, and the saved figure `02c_4_chain_rule.png`. **What Figure 02c.4 shows:** the right curve is the left one squeezed sideways by the factor 6, and both dashed lines touch their curves; the right slope is 6 times the left one: $\partial_0 = 6H\, d/dz$.

In [8], the author's metric from the Revision record.

```
CURVATURE = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE).read_text(encoding="utf-8"))
a4 = sp.Function("a4")  # the unknown function a4(x4) of the metric
b4 = sp.Function("b4", positive=True)  # the unknown function b4(x4) > 0
NAMES = {"a4": a4, "b4": b4, "x4": x4, "x0": x0, "H": H, "Sin": sp.sin,
         "Cos": sp.cos, "Cot": sp.cot, "Tan": sp.tan}
```

The cell reads the Revision record of the curvature program. `sp.Function("a4")` makes an unknown function a_4, so that $a_4(x_4)$ can be differentiated without knowing its form, and `sp.Function("b4", positive=True)` the second metric function b_4, whose values are positive. `NAMES` tells the translator below which sympy object each name of the record stands for.

```
def from_record(text):
    """A Wolfram-Language text of the record as a sympy expression."""
    for f in ("a4", "b4"):  # Derivative[n][f][x4] is the n-th derivative of f
        for n in (1, 2):
            text = text.replace(f"Derivative[{n}][{f}][x4]",
                                f"Derivative({f}(x4), (x4, {n}))")
    text = text.replace("[", "(").replace("]", ")").replace("^", "**")
    return sp.parse_expr(text, local_dict=NAMES)
```

The record writes its formulas in the notation of the Wolfram Language: `E^(...)` for $e^{\cdots}$, `Sin[...]` for $\sin(\dots)$, `Derivative[1][a4][x4]` for a_4' and `Derivative[1][b4][x4]` for b_4'. `from_record` translates such a text: first the first and second derivatives of a_4 and b_4 (the two loops write the four possible texts with `f` and `n` filled in; `(x4, 2)` asks sympy for the second derivative), then square brackets into round ones and `^` into Python's `**`; `sp.parse_expr` reads the result as a sympy expression with the names of `NAMES` (sympy knows `E` as e by itself).

```
def tidy(expression):
    """Write cot and tan as quotients of cos and sin (helps sympy simplify)."""
    expression = expression.replace(sp.cot, lambda u: sp.cos(u) / sp.sin(u))
    return expression.replace(sp.tan, lambda u: sp.sin(u) / sp.cos(u))


def same(first, second):
    """True when sympy simplifies first - second to zero."""
    return sp.simplify(tidy(first - second)) == 0
```

`tidy` replaces every $\cot u$ by $\cos u/\sin u$ and every $\tan u$ by $\sin u/\cos u$ (a `lambda` is a one-line function without a name); with only sines and cosines left, `sp.simplify` finds cancellations more reliably. `same` decides whether two expressions are equal by simplifying their difference to 0.

```
COORDS = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
X = [sp.Symbol(name, real=True) for name in COORDS]
```

```
X[4], X[0] = x4, x0  # the metric depends on these two only
z = 6 * H * x0
third = sp.Rational(1, 3)
```

The eight coordinate names and a list of eight symbols; positions 4 and 0 (the position of x_k is k) are replaced by the symbols x_4 and x_0 already in use. `z` is now the expression $6Hx_0$, and `third` the exact fraction 1/3.

```
# the author's order x0, x1, ..., x7
BY_HAND = ([sp.cot(z) ** 2] + [sp.exp(2 * a4(x4)) * sp.sin(z) ** third] * 3
           + [sp.Integer(-1)]
           + [-b4(x4) * sp.exp(-2 * a4(x4)) * sp.sin(z) ** third] * 3)
G_RECORD = [from_record(text) for text in record["metricDiagonal"]]
for name, entry in zip(COORDS, G_RECORD):
    say(f"g_{name[1]}{name[1]} = {entry}")
```

`BY_HAND` is the diagonal of the author's metric typed in by hand: a list times 3 repeats it three times, and `+` joins lists, giving the eight entries of Section 2.19 in the author's order. `G_RECORD` translates the record's eight texts; the loop prints them (`name[1]` is the digit of the coordinate name), from `g_00 = cot(6*H*x0)**2` to `g_77 = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)`.

```
check(record["coordinates"] == COORDS and len(G_RECORD) == 8
      and all(same(a, b) for a, b in zip(G_RECORD, BY_HAND)),
      "the record's metric equals the author's metric typed in by hand",
      record=f"{CURVATURE}, key metricDiagonal")
G = sp.diag(*G_RECORD)  # the metric as an 8 x 8 diagonal sympy matrix
```

The check requires the record's coordinate list to be $x_0, \ldots, x_7$, eight entries, and every entry to equal the hand-typed one (COMPUTED: reproduces `Revision/gkd_lovelock/results/curvature.json`, key `metricDiagonal`). `sp.diag(*G_RECORD)` builds the 8×8 diagonal matrix with these entries (`*` unpacks the list into eight arguments).

In [9], the rates along x_4 and x_0.

```
rate4 = [sp.simplify(sp.diff(G[a, a], x4) / (2 * G[a, a])) for a in range(8)]
rate0 = [sp.simplify(tidy(sp.diff(G[a, a], x0) / (2 * G[a, a]))) for a in range(8)]
da4 = sp.diff(a4(x4), x4)  # a4', the derivative of a4
db4 = sp.diff(b4(x4), x4)  # b4', the derivative of b4
```

For each of the eight diagonal entries `G[a, a]` the logarithmic rates $\partial_4 g_{aa}/(2g_{aa})$ and $\partial_0 g_{aa}/(2g_{aa})$ of Section 2.19. `da4` and `db4` are the symbols for a_4' and b_4'.

```
# the author's order x0, x1, ..., x7
expected4 = [0] + [da4] * 3 + [0] + [-da4 + db4 / (2 * b4(x4))] * 3
expected0 = ([-6 * H / (sp.sin(z) * sp.cos(z))] + [H * sp.cot(z)] * 3 + [0]
             + [H * sp.cot(z)] * 3)
for a, name in enumerate(COORDS):
    say(f"{name}: d4 ln s = {rate4[a]},  d0 ln s = {rate0[a]}")
```

The expected rates of Section 2.19 in the author's order, and a printout of the computed ones. sympy writes $H \cot z$ as `H/tan(6*H*x0)` and $-6H/(\sin z \cos z)$ as `-12*H/sin(12*H*x0)`; they are equal because $\sin 2z = 2 \sin z \cos z$.

```
check(all(same(r, e) for r, e in zip(rate4, expected4))
      and all(same(r, e) for r, e in zip(rate0, expected0)),
      "the rates a4', -a4' + b4'/(2 b4), H cot z and -6H/(sin z cos z) of section 4")
check(same(sum(rate4), sp.Rational(3, 2) * db4 / b4(x4))
      and sp.simplify(sum(rate4).subs(db4, 0)) == 0,
      "the eight rates along x4 add up to (3/2) b4'/b4, zero for constant b4")
```

The first check compares all sixteen rates with the expected ones; the second confirms that the eight rates along x_4 add up to $\frac{3}{2} b_4'/b_4$ and, with $b_4' = 0$ put in by `.subs(db4, 0)`, to zero (Section 2.19). (The check's text says "section 4" because it refers to section 4 of the notebook.)

```
rates_now = [float(r.subs({da4: 1, db4: 0})) for r in rate4]  # a4' = 1, b4' = 0
fig, ax = plt.subplots()
colors = [GREY] + [BLUE] * 3 + [GREY] + [ORANGE] * 3
ax.bar(range(8), rates_now, color=colors, width=0.6)
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_xticks(range(8), [f"$x_{k}$" for k in range(8)])
```

On the history $a_4 = AHx_4$, $b_4 = 1$ with $A = H = 1$ we have $a_4' = 1$ and $b_4' = 0$; `.subs({da4: 1, db4: 0})` puts this in (a dictionary of replacements). `ax.bar` draws a bar for each direction (grey for x_0 and x_4, blue for 3-space, orange for the extra times), and `set_xticks` labels the positions 0 to 7 with x_0 to x_7.

```
for k, value in enumerate(rates_now):
    ax.text(k, value + (0.06 if value >= 0 else -0.14), f"{value:+.0f}",
            ha="center", fontsize=9)
ax.set_ylim(-1.4, 1.4)
ax.set_xlabel("direction")
ax.set_ylabel("$\\partial_4 \\ln s_a$ (units $H$)")
ax.set_title("Growth rate of each scale factor along the time $x_4$ "
             "($A = H = 1$, $b_4 = 1$)")
save_figure(fig, "rates_along_x4",
            "The rate $\\partial_4 \\ln s_a$ at which the scale factor $s_a$ of "
            ...)
```

Each bar gets its value written just above (positive) or below (negative) it, then the axis range, labels and the saved figure `02c_5_rates_along_x4.png`. **What Figure 02c.5 shows:** two bars at 0 (x_0 and x_4), three at $+1$ (3-space inflates), three at -1 (the extra times deflate exponentially); the bars cancel, so at constant b_4 the 7-volume of space and extra times does not change with time.

In [10], the rates along the hidden coordinate.

```
rate_sum0 = sp.simplify(tidy(sum(rate0)))
check(same(rate_sum0, -6 * H * sp.tan(z)),
      "the eight rates along x0 add up to d0 ln cos z = -6 H tan z")
```

The sum of the eight rates along x_0 must be $-6H\tan z$, the rate of $\cos z$ and of $b_4^{3/2}\cos z$ (Section 2.19).

```
z_line = np.linspace(0.05, np.pi / 2 - 0.05, 400)
fig, ax = plt.subplots()
ax.plot(z_line, 1 / np.tan(z_line), color=BLUE, lw=1.6,
        label="$H \\cot z$: directions $x_1, x_2, x_3$ and $x_5, x_6, x_7$")
ax.plot(z_line, -6 / (np.sin(z_line) * np.cos(z_line)), "--", color=VIOLET, lw=1.6,
        label="$-6H/(\\sin z \\cos z)$: direction $x_0$")
ax.plot(z_line, -6 * np.tan(z_line), "-.", color=ORANGE, lw=1.6,
        label="sum of all eight $= -6 H \\tan z$")
```

With $H = 1$, on $0.05 < z < \pi/2 - 0.05$ (the ends are left out because the rates become infinite there), the three rates are drawn.

```
ax.axhline(0.0, color=BLACK, lw=0.8)
ax.set_ylim(-60.0, 20.0)
ax.set_xlabel("$z = 6 H x_0$")
ax.set_ylabel("$\\partial_0 \\ln s_a$ (units $H$)")
ax.set_title("Rates of the scale factors along the hidden coordinate ($H = 1$)")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "rates_along_x0",
            "The rates $\\partial_0 \\ln s_a$ at which the scale factors change "
            ...)
```

Axis range, labels and the saved figure `02c_6_rates_along_x0.png`. **What Figure 02c.6 shows:** the 3-space and extra-time factors grow along x_0 near $z = 0$ and level off near $z = \pi/2$ (where $\cot z = 0$); the hidden factor $\cot z$ shrinks everywhere, fastest at both ends; the sum $-6H\tan z$ is the rate of the volume factor $b_4^{3/2}\cos z$ along x_0, which falls to zero at $z = \pi/2$.

In [11], the volume factor and Jacobi's formula.

```
det_g = sp.simplify(tidy(G.det()))  # the product of the eight entries
say(f"det g = {det_g}")
sqrt_record = from_record(record["sqrtAbsDetG"])
say(f"sqrtAbsDetG of the record = {sqrt_record}")
check(same(det_g, b4(x4) ** 3 * sp.cos(z) ** 2)
      and same(sqrt_record, b4(x4) ** sp.Rational(3, 2) * sp.cos(z)),
      "det g = b4^3 cos(z)^2 and the record's sqrt|det g| = b4^(3/2) sin z cot z "
      "= b4^(3/2) cos z",
      record=f"{CURVATURE}, key sqrtAbsDetG")
```

`G.det()` is the determinant; sympy simplifies it to $b_4^3\cos^2(6Hx_0)$, and the record's entry `sqrtAbsDetG` reads $b_4^{3/2}\sin(6Hx_0)\cot(6Hx_0)$ (the printed lines show both as sympy writes them). The check confirms $\det g = b_4^3\cos^2 z$ and $\sqrt{|\det g|} = b_4^{3/2}\sin z\cot z = b_4^{3/2}\cos z$ (COMPUTED: reproduces `Revision/gkd_lovelock/results/curvature.json`, key `sqrtAbsDetG`).

```
A4REPORT = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
a4_report = json.loads(repository_file(A4REPORT).read_text(encoding="utf-8"))
```

```
verdict = [c["verdict"] for c in a4_report["checks"]
           if c["name"] == "sqrt_abs_det_g_is_cos_z"]
db4 = sp.diff(b4(x4), x4)  # b4', the derivative of b4
check(verdict == ["PASS"]
      and same(sp.diff(sqrt_record, x4),
               sp.Rational(3, 2) * db4 / b4(x4) * sqrt_record)
      and sp.simplify(sp.diff(sqrt_record, x4).subs(db4, 0)) == 0,
      "d4 sqrt|det g| = (3/2)(b4'/b4) sqrt|det g|: for constant b4 the volume "
      "factor does not depend on x4 (inflation and deflation cancel)",
      record=f"{A4REPORT}, check sqrt_abs_det_g_is_cos_z")
```

The cell reads the Wolfram report of the a_4 field equations and collects the verdict of its check `sqrt_abs_det_g_is_cos_z` (a list comprehension with an `if` keeps only the matching check). The check requires that verdict to be PASS, the derivative of $\sqrt{|\det g|}$ with respect to x_4 to be $\frac{3}{2}(b_4'/b_4)\sqrt{|\det g|}$, and, with $b_4' = 0$, to be exactly 0 (COMPUTED: reproduces `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `sqrt_abs_det_g_is_cos_z`).

```
jacobi = [same(sp.diff(sqrt_record, c),
               sp.Rational(1, 2) * sqrt_record * (G.inv() * G.diff(c)).trace())
          for c in (x4, x0)]
check(all(jacobi), "Jacobi's formula for the derivative of sqrt|det g| (x4 and x0)")
```

Jacobi's formula of Section 2.19 for $c = x_4$ and $c = x_0$: `G.inv()` is the inverse matrix, `G.diff(c)` the matrix of the derivatives of the entries, `*` the matrix product and `.trace()` the sum of the diagonal entries.

In [12], all 25 Christoffel entries.

```
computed = {}
for a in range(8):
    for b in range(8):
        for c in range(b, 8):
            value = (sp.diff(G[a, c], X[b]) + sp.diff(G[a, b], X[c])
                     - sp.diff(G[b, c], X[a])) / (2 * G[a, a])
            value = sp.simplify(tidy(value))
            if value != 0:
                computed[(COORDS[a], COORDS[b], COORDS[c])] = value
```

Three nested loops run over a (8 values), b (8 values) and c from b to 7, that is over the $8 \times 36 = 288$ entries $\Gamma^a{}_{bc}$ with b not after c. Each is computed from the formula of Section 2.19 (the derivative with respect to the coordinate `X[b]`, and so on), simplified, and stored under the key (a, b, c) of coordinate names if it is not zero.

```
listed = {(e["a"], e["b"], e["c"]): from_record(e["value"])
          for e in record["christoffelNonzero_b_le_c"]}
for key in [("x1", "x1", "x4"), ("x5", "x4", "x5"), ("x0", "x1", "x1"),
            ("x0", "x0", "x0")]:
    say(f"Gamma^{key[0]}_({key[1]} {key[2]}) = {computed[key]}")
```

`listed` translates the record's list of nonzero entries into the same form. Four entries are printed: ${\Gamma^1}_{14} = a_4'$, ${\Gamma^5}_{45} = -a_4' + b_4'/(2b_4)$, ${\Gamma^0}_{11} = -He^{2a_4}\sin^{4/3} z/\cos z$ (which is $-He^{2a_4}\sin^{1/3} z\tan z$ of Section 2.19) and ${\Gamma^0}_{00} = -12H/\sin(12Hx_0)$.

```
report("nonzero entries computed / listed in the record",
       f"{len(computed)} / {len(listed)}")
check(sorted(computed) == sorted(listed)
      and all(same(computed[k], listed[k]) for k in listed),
      "all 25 nonzero Christoffel entries equal those of the record",
      record=f"{CURVATURE}, key christoffelNonzero_b_le_c")
```

25 entries are computed and 25 are listed. The check requires the same keys (`sorted` puts the keys of a dictionary in order) and equal values for every key (COMPUTED: reproduces `Revision/gkd_lovelock/results/curvature.json`, key `christoffelNonzero_b_le_c`).

In [13], the last check.

```
names = ["level_curves_gradient", "slices_and_slopes", "finite_differences",
         "chain_rule", "rates_along_x4", "rates_along_x0"]
present = [output_file(f"{FIGURE_FOLDER}/02c_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02c exist")
all_checks_passed()
```

As in Notebook 02a: the six figure files must exist; the last line is ALL 17 CHECKS PASSED (notebook 02c). The 17 checks are: 2 in In [3], 2 in In [4], 1 in In [5], 2 in In [6], 1 in In [7], 1 in In [8], 2 in In [9], 1 in In [10], 3 in In [11], 1 in In [12] and 1 in In [13].

2.24 Two first-order equations, the Prüfer angle and Newton's method

This section prepares the last notebook, which rewrites the shooting method of the Revision Kohn-Sham solver. We use the solver's equations only as mathematics here; where they come from, and what their levels mean physically, is the subject of Chapters 14 and 15.

The equations. The Revision record `Revision/kohn_sham/ks-theory.json` gives the Kohn-Sham equation of one two-component block in a real form (key `blockEquation.realForm`): an orbital is described by two real functions $a(y)$ and $b(y)$ of the **hidden coordinate** y, which is the record's coordinate along x_0, $y = \ln(\sin z)/(6H)$ (key `geometry.hiddenCoordinate`; $y = 0$ is the end of the patch $z = \pi/2$, called the **brane**, and $y \to -\infty$ is $z \to 0$). The record's general block contains a mass M, a momentum k in 3-space, potentials and a sign $j = \pm 1$ of the block type. We take the simplest case: constant mass M, no 3-space momentum ($k = 0$), no potential, and $j = +1$. Then the equations are

$$a' = Ma - \varepsilon b, \qquad b' = \varepsilon a - Mb, \qquad -L \le y \le 0,$$

where the prime is now d/dy and ε is the unknown energy. The interval is cut off at $y = -L$, the **tip**. The conditions are

$$b(-L) = 0 \ \text{(tip)}, \qquad b(0) = 0 \ \text{(even orbitals)} \quad \text{or} \quad a(0) = 0 \ \text{(odd orbitals)}.$$

Status. The tip condition is a choice of the model (the record's status: "chosen (regular tip)"); the brane conditions follow from a mirror symmetry of the model across the brane that the record labels ASSUMED (key `boundaryConditions.brane.status`). Everything below is exact mathematics for these equations under these conditions. The record's values are $M = m = 1$, $H = 1$, $L = 3$ (units with $m = H = 1$: energies in units of the mass m, lengths along y in units of $1/H$); the program also checks $L = 2$ and $M = 2$.

The exact levels. We eliminate b.

$$b = \frac{Ma - a'}{\varepsilon} \qquad (\varepsilon \neq 0)$$

(solve the first equation for b);

$$b' = \frac{Ma' - a''}{\varepsilon}$$

(differentiate; M and ε are constants);

$$\frac{Ma' - a''}{\varepsilon} = \varepsilon a - M\,\frac{Ma - a'}{\varepsilon}$$

(insert both into the second equation);

$$Ma' - a'' = \varepsilon^2 a - M^2 a + Ma'$$

(multiply by ε);

$$a'' = (M^2 - \varepsilon^2)\, a$$

(cancel Ma' on both sides and change all signs). The condition $b = 0$ at a point means $Ma - a' = 0$ there (because $b = (Ma - a')/\varepsilon$).

Case $|\varepsilon| > M$. Put $p = \sqrt{\varepsilon^2 - M^2} > 0$; then $a'' = -p^2 a$, and $a = \cos(py + \varphi)$ up to a constant factor, with a phase φ.

- Even orbitals: $Ma - a' = 0$ at $y = 0$ and at $y = -L$. With $a = \cos(py + \varphi)$ this reads $M\cos(py + \varphi) + p\sin(py + \varphi) = 0$, that is $\tan(py + \varphi) = -M/p$ at both points ($\cos(py + \varphi)$ cannot vanish there, since then the left side would be $\pm p \neq 0$). The tangent repeats itself after π and takes each value once per period, so the two arguments φ and $-pL + \varphi$ differ by a whole multiple of π: $pL = n\pi$ with $n = 1, 2, \ldots$ Hence $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$.
- Odd orbitals: $a(0) = 0$ makes a a multiple of $\sin(py)$. The tip condition $Ma(-L) - a'(-L) = 0$ becomes $M\sin(-pL) - p\cos(-pL) = 0$, that is $-M\sin(pL) - p\cos(pL) = 0$ (the sine is odd, the cosine even), or

$$\tan(pL) = -\frac{p}{M}.$$

This transcendental equation has exactly one root p_n on each interval $(n+\frac{1}{2})\pi/L < p < (n+1)\pi/L$, $n = 0, 1, 2, \ldots$, and no other positive root. The reasons, in three steps:

- On the intervals $n\pi/L < p < (n+\frac{1}{2})\pi/L$ the angle pL lies between $n\pi$ and $n\pi + \pi/2$, where the tangent is positive, while $-p/M$ is negative: no root.
- At the points $pL = n\pi$ the tangent is 0, not the negative number $-p/M$. At the points $pL = (n+\frac{1}{2})\pi$ the cosine vanishes, so the tangent is not defined, and the equation in the form $M\sin(pL) + p\cos(pL) = 0$ (multiplied by $\cos(pL)$ and by M) would demand $M\sin(pL) = \pm M = 0$: no root at these points either.
- On $(n+\frac{1}{2})\pi/L < p < (n+1)\pi/L$ consider $D(p) = \tan(pL) + p/M$, which vanishes exactly at the roots. It is continuous there and strictly increasing, because $\tan(pL)$ rises (its derivative $L/\cos^2(pL)$ is positive, by the chain rule) and p/M rises ($M > 0$); in words, the tangent rises while the line $-p/M$ falls. Just to the right of the left end $\tan(pL)$ is a large negative number, so $D < 0$; at the right end $\tan(pL) = 0$ and $D = (n+1)\pi/(LM) > 0$. By the intermediate value theorem D vanishes somewhere in between, and since it is strictly increasing it vanishes only once.

Each root gives the two odd levels $\pm\sqrt{M^2 + p_n^2}$.

Case $\varepsilon = 0$. The equations separate: $a' = Ma$ and $b' = -Mb$. With $b(-L) = 0$ the solution of $b' = -Mb$ is $b = 0$ everywhere (the uniqueness theorem of Section 2.2), and $a = e^{M(y+L)}$, a multiple of e^{My}. This is an even orbital ($b(0) = 0$) with the energy exactly 0, the **zero mode**; an odd one would need $a(0) = 0$, which $e^{M(y+L)}$ never is.

Case $0 < |\varepsilon| \le M$*: no level.* For $0 < |\varepsilon| < M$ put $q = \sqrt{M^2 - \varepsilon^2}$, with $0 < q < M$; then $a = \alpha e^{qy} + \beta e^{-qy}$ and

$$Ma - a' = (M-q)\,\alpha\, e^{qy} + (M+q)\,\beta\, e^{-qy}$$

(insert a and $a' = q\alpha e^{qy} - q\beta e^{-qy}$). For an even orbital this must vanish at two different points; setting it to zero gives $e^{2qy} = -(M+q)\beta/((M-q)\alpha)$, and the left side takes each value only once (it is strictly increasing), so this can hold at two points only if $\alpha = \beta = 0$. For an odd orbital $a(0) = 0$ gives $\beta = -\alpha$, so $a = 2\alpha\sinh(qy)$ with $\sinh u = (e^u - e^{-u})/2$ and $\cosh u = (e^u + e^{-u})/2$, and the tip condition becomes $2\alpha(-M\sinh(qL) - q\cosh(qL)) = 0$; the bracket is negative, so $\alpha = 0$. For $|\varepsilon| = M$ the same steps with $a = \alpha + \beta y$ give $\alpha = \beta = 0$ (Exercise 7). So, between $-M$ and M the only level is the zero mode (PROVED here; Notebook 02d, In [4], confirms it numerically for the three cases of the record).

These are the exact levels that the record states (key `boundaryConditions.exactK0Spectra` of `Revision/kohn_sham/ks-theory.json`) and lists in its column `eps_analytic` (Notebook 02d, In [7], recomputes all 54 of them; COMPUTED).

The Prüfer angle. The shooting method could use $b(0)$ or $a(0)$ directly as the mismatch. The Revision program does something better: it follows the ANGLE of the point (a, b). Write

$$a = r\cos\theta, \qquad b = r\sin\theta, \qquad r = \sqrt{a^2 + b^2} > 0.$$

$\theta(y)$ is the **Prüfer angle** (after the mathematician Heinz Prüfer; the notebooks, which use plain ASCII, write Pruefer). The point (a, b) never reaches the origin: if $a = b = 0$ at some y, the uniqueness theorem would make the solution zero everywhere. So θ is defined everywhere, and we let it grow continuously, counting every full turn (**winding**), so that it can exceed 2π. Where $a \neq 0$, $\theta = \arctan(b/a)$ plus a constant, and

$$\theta' = \frac{1}{1 + (b/a)^2} \cdot \frac{b'a - ba'}{a^2}$$

(the chain rule with the derivative $1/(1 + w^2)$ of $\arctan w$, and the quotient rule for b/a);

$$\theta' = \frac{ab' - ba'}{a^2 + b^2} = \frac{ab' - ba'}{r^2}$$

(multiply out: $\frac{1}{1+b^2/a^2} \cdot \frac{1}{a^2} = \frac{1}{a^2+b^2}$); the same formula holds where $a = 0$ (use $\theta = -\arctan(a/b)$ plus a constant there). Now insert the equations:

$$ab' - ba' = a(\varepsilon a - Mb) - b(Ma - \varepsilon b) = \varepsilon(a^2 + b^2) - 2Mab$$

(multiply out and collect);

$$\theta' = \varepsilon - M\,\frac{2ab}{r^2} = \varepsilon - M\sin 2\theta$$

(divide by r^2; then $2ab/r^2 = 2\sin\theta\cos\theta = \sin 2\theta$, the double-angle formula). The start $b(-L) = 0$ with $a(-L) = 1$ means $\theta(-L) = 0$. The brane conditions become conditions on the end angle $\Phi(\varepsilon) = \theta(0)$:

$$\text{even: } b(0) = 0 \iff \Phi = l\pi, \qquad \text{odd: } a(0) = 0 \iff \Phi = \frac{\pi}{2} + l\pi,$$

for a whole number l, the **label** of the level ($\sin\theta = 0$ exactly at the multiples of π, $\cos\theta = 0$ at $\pi/2$ plus multiples of π).

The end angle grows with the energy. First the radius:

$$rr' = aa' + bb' = a(Ma - \varepsilon b) + b(\varepsilon a - Mb) = M(a^2 - b^2)$$

(differentiate $r^2 = a^2 + b^2$, which gives $2rr' = 2aa' + 2bb'$; insert the equations; the terms with ε cancel);

$$\frac{r'}{r} = M\,\frac{a^2 - b^2}{r^2} = M\cos 2\theta$$

(divide by r^2; $\cos^2\theta - \sin^2\theta = \cos 2\theta$). Now let $u(y) = \partial\theta(y)/\partial\varepsilon$, the change of the angle at y per unit change of the energy (a partial derivative: θ depends on y and on ε).

$$u' = \frac{\partial}{\partial\varepsilon}\,\theta' = 1 - 2M\cos(2\theta)\,u$$

(Schwarz's theorem of Section 2.18 lets us swap the derivatives with respect to y and ε; then differentiate $\varepsilon - M\sin 2\theta$ with respect to ε by the chain rule);

$$u' = 1 - 2\,\frac{r'}{r}\,u$$

(insert $M\cos 2\theta = r'/r$);

$$r^2u' + 2rr'u = r^2$$

(multiply by r^2 and move the u term to the left);

$$(r^2u)' = r^2$$

(the product rule: $(r^2u)' = 2rr'u + r^2u'$);

$$r(0)^2u(0) - r(-L)^2u(-L) = \int_{-L}^{0} r^2\,dy$$

(integrate from $-L$ to 0: the integral of a derivative is the difference of the end values);

$$\frac{d\Phi}{d\varepsilon} = u(0) = \frac{1}{r(0)^2}\int_{-L}^{0} r^2\,dy > 0$$

($u(-L) = 0$, because $\theta(-L) = 0$ for every energy; divide by $r(0)^2$; the integral of the positive function r^2 is positive). So $\Phi(\varepsilon)$ is **strictly increasing**: each target value $l\pi$ or $\pi/2 + l\pi$ is reached for exactly one energy. Every level has its own label l, and no level in a range of energies can be missed (PROVED here; this is the argument of the program's header comment in `Revision/kohn_sham/solver/src/shoot.rs`). The formula also gives the slope that Newton's method below needs, without any extra shot.

Two values of the slope. Far from the mass, $\theta' = \varepsilon - M\sin 2\theta$ is ε on average, so $\Phi \approx \varepsilon L$ plus a bounded amount and the slope is about L. At $\varepsilon = 0$ the shot is $a = e^{M(y+L)}$, $b = 0$, so

$$\frac{d\Phi}{d\varepsilon}\Big|_{\varepsilon=0} = \frac{\int_{-L}^{0} e^{2M(y+L)}\,dy}{e^{2ML}} = \frac{(e^{2ML}-1)/(2M)}{e^{2ML}} = \frac{1 - e^{-2ML}}{2M}$$

($r^2 = e^{2M(y+L)}$; the antiderivative $e^{2M(y+L)}/(2M)$ between $-L$ and 0; divide by $r(0)^2 = e^{2ML}$), which is 0.498761 for $M = 1$, $L = 3$ (Notebook 02d, In [5]).

Counting the turns. A computer's function atan2(b, a) returns the angle of the point (a, b) in the range from $-\pi$ to π; it does not count turns. The program therefore adds up the changes of angle

from step to step: after each RK4 step it computes the new atan2, subtracts the previous one, and brings the difference into the range $(-\pi, \pi]$ by adding or subtracting 2π. This is correct as long as one step turns the point by less than π; the program refuses a shot in which a single step turns by 2 or more (radians), a safe margin. Along the way it adds up $\int r^2\,dy$ with the **trapezoid rule** (the integral over one step is the step length times the average of the two end values). When r^2 exceeds 10^{250} it divides a, b and the running integral by suitable powers of r, which changes neither the angle nor the quotient $\int r^2 dy / r(0)^2$ (in the free case of this chapter that never happens).

Newton's method. To find a level we solve $F(\varepsilon) = \Phi(\varepsilon) - t = 0$ for its target t. Replace F near the current guess ε_c by its tangent line

$$F(\varepsilon) \approx F(\varepsilon_c) + F'(\varepsilon_c)\,(\varepsilon - \varepsilon_c)$$

(the linear approximation of one-variable calculus), and take the zero of the line as the next guess:

$$\varepsilon_{\text{new}} = \varepsilon_c - \frac{F(\varepsilon_c)}{F'(\varepsilon_c)}, \qquad F'(\varepsilon) = \frac{d\Phi}{d\varepsilon} = \frac{1}{r(0)^2}\int_{-L}^{0} r^2\,dy$$

(set the line to zero and solve for ε; the slope is the formula proved above). Close to the root the number of correct digits roughly doubles at every step (ASSUMED: a standard result of numerical analysis, quoted without proof; the notebook shows it). Far from the root the tangent can point anywhere, so the program (function `find_level` in `Revision/kohn_sham/solver/src/shoot.rs`) adds a safety net:

1. Shoot at the starting guess $\varepsilon = 0$; if $F(0) = 0$ exactly (the zero mode), stop.
2. Make a bracket: walk upwards if $F < 0$ (downwards if $F > 0$), with a first step of 1.2 times Newton's estimate $|F/F'|$, but at least 10^{-4} and at most 2, doubling the step until F changes sign.
3. Inside the bracket take Newton steps; whenever Newton's point falls outside the bracket, or $|F|$ has not at least halved since the previous step, take the midpoint of the bracket instead (bisection). After each shot keep the half of the bracket that holds the sign change.
4. Stop when the bracket is narrower than the tolerance 10^{-13} (the record's `rootTolerance`), or after a Newton step shorter than it.

The orbital between the grid points. RK4 gives a and b at the ends of the steps (the **nodes**). The program also needs them in the middle of every step and takes them from the **cubic Hermite interpolation**: the polynomial of degree 3 that has the right values f_0, f_1 AND the right slopes f_0', f_1' at the two ends of a step of length h (the slopes come from the equations themselves). Put the step at $0 \le s \le h$ and write the cubic as $P(s) = f_0 + f_0' s + c s^2 + e s^3$ (its value and slope at $s = 0$ are then right). The two conditions at $s = h$ are

$$f_0 + f_0' h + c h^2 + e h^3 = f_1, \qquad f_0' + 2ch + 3eh^2 = f_1'$$

(the value and the derivative of P at $s = h$);

$$eh^3 = (f_0' + f_1')\,h - 2(f_1 - f_0), \qquad ch^2 = 3(f_1 - f_0) - (2f_0' + f_1')\,h$$

(multiply the second equation by h, subtract twice the first equation written as $ch^2 + eh^3 = f_1 - f_0 - f_0'h$, which leaves eh^3; then ch^2 from the first equation);

$$P(\tfrac{h}{2}) = f_0 + \frac{f_0'h}{2} + \frac{ch^2}{4} + \frac{eh^3}{8} = \frac{f_0 + f_1}{2} + \frac{h}{8}\,(f_0' - f_1')$$

(insert $s = h/2$; then insert ch^2 and eh^3 and collect: the values give $f_0 + (f_1 - f_0)(\frac{3}{4} - \frac{1}{4}) = \frac{f_0+f_1}{2}$, the slopes give $hf_0'(\frac{1}{2} - \frac{1}{2} + \frac{1}{8}) + hf_1'(-\frac{1}{4} + \frac{1}{8}) = \frac{h}{8}(f_0' - f_1')$). Its error is of order h^4, the order of RK4 (ASSUMED: quoted). With the values at the nodes and the midpoints the program integrates with Simpson's rule (Section 2.13) on the fine grid of $2G + 1$ points, with spacing $h/2$.

Why the error grows with the level. Far from the mass (far above $+M$ or far below $-M$) the point (a, b) turns at the rate $\theta' \approx \varepsilon$, like the oscillator of Section 2.7 with the frequency ε: forwards for $\varepsilon > 0$, backwards for $\varepsilon < 0$. One RK4 step multiplies the turning point by $R(i\omega)$ with $\omega = \varepsilon h$. Its angle ϕ is a little smaller than ω in size: a little smaller than ω for $\omega > 0$, a little larger (closer to 0) for $\omega < 0$. With $c = 1 - \frac{\omega^2}{2} + \frac{\omega^4}{24}$ and $s = \omega - \frac{\omega^3}{6}$ (the real and imaginary parts of $R(i\omega)$, Section 2.7),

$$\tan\phi = \frac{s}{c}, \qquad \tan\omega = \frac{\omega - \frac{\omega^3}{6} + \frac{\omega^5}{120} - \dots}{1 - \frac{\omega^2}{2} + \frac{\omega^4}{24} - \frac{\omega^6}{720} + \dots}$$

(the angle of a complex number has the tangent imaginary part over real part; tan = sin / cos with the Taylor series of the sine and the cosine, which follow from Section 2.3 as the series of $e^{i\omega}$);

$$\tan\phi - \tan\omega = -\frac{\omega^5/120}{c} + (\text{terms with } \omega^7 \text{ and higher}) = -\frac{\omega^5}{120} + \dots$$

(the numerators differ by $\omega^5/120$, the denominators only from ω^6 on, which changes the quotient from ω^7 on; and $1/c = 1 + \frac{\omega^2}{2} + \dots$ changes $\omega^5/120$ only from ω^7 on);

$$\phi - \omega = -\frac{\omega^5}{120} + \dots$$

(by the **mean value theorem** of calculus, $g(\phi) - g(\omega) = (\phi - \omega)\,g'(\xi)$ for some ξ between ω and ϕ; applied to $g = \tan$, whose derivative is $1 + \tan^2$, it gives $\tan\phi - \tan\omega = (\phi - \omega)(1 + \tan^2\xi)$, and $1 + \tan^2\xi = 1 + \dots$ for small angles). Every series used here holds for negative ω as well, so this result holds for both signs of the energy. So every step turns the point by $\omega^5/120 = \varepsilon^5h^5/120$ too little in the direction in which it turns: for $\varepsilon > 0$ the point turns forwards and its angle comes out too small; for $\varepsilon < 0$ it turns backwards ($\omega < 0$, $\omega^5 < 0$) and its angle comes out too large, that is, again too close to 0. After $G = L/h$ steps the computed end angle differs from the exact one by

$$\Phi_{\text{computed}}(\varepsilon) - \Phi_{\text{exact}}(\varepsilon) = -\frac{L}{h} \cdot \frac{\varepsilon^5h^5}{120} = -\frac{L\,\varepsilon^5h^4}{120}$$

(the number of steps times the change per step). The computed level is the energy $\varepsilon + \delta\varepsilon$ at which the computed end angle reaches the target, and the target is $\Phi_{\text{exact}}(\varepsilon)$, the end angle of the exact level ε. Since Φ grows by about L per unit of energy (for either sign of ε), $\Phi_{\text{computed}}(\varepsilon + \delta\varepsilon) \approx \Phi_{\text{exact}}(\varepsilon) + L\,\delta\varepsilon - L\varepsilon^5 h^4/120$, and setting this equal to the target gives

$$L\,\delta\varepsilon - \frac{L\,\varepsilon^5 h^4}{120} = 0, \qquad \delta\varepsilon \approx \frac{\varepsilon^5 h^4}{120}, \qquad \frac{\delta\varepsilon}{\varepsilon} \approx \frac{(h\varepsilon)^4}{120}$$

(subtract $\Phi_{\text{exact}}(\varepsilon)$ from both sides; divide by L; then by ε). Because ε^5 has the sign of ε, so has $\delta\varepsilon$: the energy must move away from zero to make up for the error of the angle. The computed levels come out slightly too large in size, too high for $\varepsilon > 0$ and too low for $\varepsilon < 0$, with an error of order h^4 that grows like the fifth power of the level (PROVED to leading order for levels far from the mass; near the mass the point does not turn uniformly and the error is smaller). Notebook 02d, In [12], compares this prediction with all 51 nonzero levels of the record and confirms the sign: all 33 positive levels of the record come out too high and all 18 negative ones too low (COMPUTED).

2.25 Example: the shooting method of the Revision Kohn-Sham solver, rewritten

Notebook 02d reads the theory, the parameters and the table of the free spectrum from the Revision record; rewrites the program's RK4 shot with the counted Prüfer angle and checks it against the angle equation; draws the staircase $\Phi(\varepsilon)$, checks that no level except the zero mode lies in the mass gap $|\varepsilon| \le M$, draws the slope of the staircase and checks the slope formula; draws the angle along y; recomputes the 54 exact levels; rewrites the program's Newton root finder with its safety net and compares it with plain bisection; reproduces all 54 numerical levels of the record and the numbers quoted by two checks of the program; builds the normalised orbitals as the program does; and measures the order 4 of the levels, the phase-lag law and the sign of the level errors. It ends with ALL 19 CHECKS PASSED (notebook 02d).

2.26 How to run Notebook 02d

Step 1. What this notebook does and what it needs.

Notebook 02d (Shooting two first-order equations with the Pruefer angle) is the file `Revision /textbook/notebooks/02d_pruefer_shooting.ipynb` of the repository Dirac_claude. It solves the simplest Kohn-Sham equations of the book, two first-order equations for the components a and b of an orbital in the hidden coordinate (constant mass, no momentum, no interaction), as an eigenvalue problem: it derives the exact levels, introduces the Pruefer angle, proves with the product rule that its end value increases with the energy, shoots with RK4 exactly as the Revision Rust solver does (900 steps on the interval of length 3, the winding count of the angle, the Newton root finder with a bisection safety net, tolerance 1e-13), reproduces all 54 levels of the Revision record of the free spectrum, numeric and exact, and the numbers of the solver checks of that spectrum and of the zero mode, and measures the fourth-order convergence of the levels. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself

imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 02d_pruefer_shooting.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 02d_pruefer_shooting.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 02d_pruefer_shooting.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/02d.captions.json`
- `Revision/textbook/figures/02d_1_pruefer_staircase.png`
- `Revision/textbook/figures/02d_2_staircase_slope.png`
- `Revision/textbook/figures/02d_3_angle_along_y.png`
- `Revision/textbook/figures/02d_4_odd_level_equation.png`
- `Revision/textbook/figures/02d_5_newton_vs_bisection.png`
- `Revision/textbook/figures/02d_6_orbitals.png`
- `Revision/textbook/figures/02d_7_level_convergence.png`
- `Revision/textbook/figures/02d_8_record_differences.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of notebook 02d exist
ALL 19 CHECKS PASSED (notebook 02d)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/02d_pruefer_shooting.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 02d_pruefer_shooting.ipynb
python -m nbconvert --execute --inplace 02d_pruefer_shooting.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

2.27 Notebook 02d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/02d_pruefer_shooting.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Kohn-Sham equations of this book (its density-functional model of the field
```

dirac16complex in the author's primordial universe) are solved by a Rust program of the Revision record, Revision/kohn_sham/solver, with the shooting method. This notebook teaches that method on the simplest case of those equations and REPRODUCES the program's results for it. It

- takes from the Revision record Revision/kohn_sham/ks-theory.json the two first-order equations for the two real components $a(y)$ and $b(y)$ of an orbital in the hidden coordinate y, in the free case (constant mass M, no 3-space momentum, no interaction), with their boundary conditions;
- derives their exact levels: $\varepsilon = 0$ and $\pm\sqrt{M^2 + (n\pi/L)^2}$ for the even orbitals, $\pm\sqrt{M^2 + p^2}$ with $\tan(pL) = -p/M$ for the odd ones;
- introduces the **Pruefer angle** $\theta = \mathrm{atan2}(b, a)$ and proves, with nothing more than the product rule, that its end value Φ grows with the energy, at the rate $d\Phi/d\varepsilon = \int r^2\,dy / r(0)^2$; so every level is found exactly once and carries a whole-number label;
- shoots with RK4 exactly as the Rust program does (900 steps on an interval of length 3) and finds every level with the program's own root finder (Newton's method with a bisection safety net, tolerance 10^{-13}), re-written here line by line;
- reproduces all 54 levels of the record Revision/kohn_sham/results/spectrum/free-k0-analytic.csv (on the computer that made the record, in all 16 printed digits), the exact column of the record, and the numbers that the program's checks of this spectrum and of the zero mode quote;
- draws the staircase of the Pruefer angle and its slope, the winding of the angle along y, Newton against bisection, the orbitals, and the fourth-order convergence of the levels.

It draws 8 figures and prints a PASS line for every check.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 02d (Shooting two first-order equations with the Pruefer angle) is the file
‘Revision/textbook/notebooks/02d_pruefer_shooting.ipynb‘ of the repository Dirac_claude.
It solves the simplest Kohn-Sham equations of the book, two first-order equations for the
components a and b of an orbital in the hidden coordinate (constant mass, no momentum, no
interaction), as an eigenvalue problem: it derives the exact levels, introduces the
Pruefer angle, proves with the product rule that its end value increases with the energy,
shoots with RK4 exactly as the Revision Rust solver does (900 steps on the interval of
length 3, the winding count of the angle, the Newton root finder with a bisection safety
net, tolerance 1e-13), reproduces all 54 levels of the Revision record of the free
spectrum, numeric and exact, and the numbers of the solver checks of that spectrum and of
the zero mode, and measures the fourth-order convergence of the levels. It needs a
computer with Windows 11, macOS or Linux, an internet connection for the installation,
the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib;
```

the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already

active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 02d_pruefer_shooting.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 02d_pruefer_shooting.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 02d_pruefer_shooting.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/02d.captions.json‘
- ‘Revision/textbook/figures/02d_1_pruefer_staircase.png‘
- ‘Revision/textbook/figures/02d_2_staircase_slope.png‘
- ‘Revision/textbook/figures/02d_3_angle_along_y.png‘
- ‘Revision/textbook/figures/02d_4_odd_level_equation.png‘
- ‘Revision/textbook/figures/02d_5_newton_vs_bisection.png‘
- ‘Revision/textbook/figures/02d_6_orbitals.png‘
- ‘Revision/textbook/figures/02d_7_level_convergence.png‘
- ‘Revision/textbook/figures/02d_8_record_differences.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of notebook 02d exist
ALL 19 CHECKS PASSED (notebook 02d)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/02d_pruefer_shooting.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for

  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 02d_pruefer_shooting.ipynb
      python -m nbconvert --execute --inplace 02d_pruefer_shooting.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 02d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 02d (Shooting two first-order equations with the Pruefer angle) is the file
# Revision/textbook/notebooks/02d_pruefer_shooting.ipynb of the repository Dirac_claude.
# It solves the simplest Kohn-Sham equations of the book, two first-order equations for
# the components a and b of an orbital in the hidden coordinate (constant mass, no
# momentum, no interaction), as an eigenvalue problem: it derives the exact levels,
# introduces the Pruefer angle, proves with the product rule that its end value increases
# with the energy, shoots with RK4 exactly as the Revision Rust solver does (900 steps on
# the interval of length 3, the winding count of the angle, the Newton root finder with a
# bisection safety net, tolerance 1e-13), reproduces all 54 levels of the Revision record
# of the free spectrum, numeric and exact, and the numbers of the solver checks of that
# spectrum and of the zero mode, and measures the fourth-order convergence of the levels.
# It needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy,
# mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
```

```
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
```

```
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
```

```
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 02d_pruefer_shooting.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 02d_pruefer_shooting.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 02d_pruefer_shooting.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/02d.captions.json
# - Revision/textbook/figures/02d_1_pruefer_staircase.png
# - Revision/textbook/figures/02d_2_staircase_slope.png
# - Revision/textbook/figures/02d_3_angle_along_y.png
# - Revision/textbook/figures/02d_4_odd_level_equation.png
# - Revision/textbook/figures/02d_5_newton_vs_bisection.png
# - Revision/textbook/figures/02d_6_orbitals.png
# - Revision/textbook/figures/02d_7_level_convergence.png
# - Revision/textbook/figures/02d_8_record_differences.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
```

```
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 8 figure files of notebook 02d exist
#     ALL 19 CHECKS PASSED (notebook 02d)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/02d_pruefer_shooting.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 02d_pruefer_shooting.ipynb
#     python -m nbconvert --execute --inplace 02d_pruefer_shooting.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
```

```
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "02d"  # this notebook: chapter 02, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
```

```
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 02d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Orbital**: a single-particle state of the Kohn-Sham model. Here it is
  described by two real functions $a(y)$ and $b(y)$, its *components*.
- **Hidden coordinate** $y$: the book's coordinate along the hidden direction
  $x_0$, $y = \ln(\sin z)/(6H)$ with $z = 6 H x_0$; $y = 0$ is the end of the
  patch $z = \pi/2$ (the *brane*), and the interval is cut off at $y = -L$ (the
  *tip*).
- **Energy level** $\varepsilon$: a value of the energy for which the equations
  have a solution with all boundary conditions; an eigenvalue. Unit: the mass
  $m$ (in units with $m = H = 1$).
- **Mass** $M$: the constant mass of the free case; here $M = m$.
- **Boundary conditions**: at the tip $b(-L) = 0$ (the *regular tip*, a choice
  of the model); at the brane $b(0) = 0$ (*even* orbitals) or $a(0) = 0$
  (*odd* orbitals), which follow from the ASSUMED mirror symmetry of the model
  (the record labels it ASSUMED).
- **Pruefer angle** $\theta(y)$: the angle of the point $(a, b)$ seen from the
  origin, $a = r \cos\theta$, $b = r \sin\theta$ with $r > 0$; we count every
  full turn, so $\theta$ can grow beyond $2\pi$ (*winding*). $r$ is the
  distance of the point from the origin, $r^2 = a^2 + b^2$.
- **atan2(b, a)**: the angle of the point $(a, b)$ between $-\pi$ and $\pi$.
- **End angle** $\Phi(\varepsilon) = \theta(0)$: the angle reached at the brane
  when we shoot with the energy $\varepsilon$.
- **Label** $l$: the whole number that names a level through its target angle,
  $\theta(0) = l\pi$ (even) or $\pi/2 + l\pi$ (odd).
- **Monotone (increasing)**: a function that grows whenever its argument grows.
- **RK4 step count** $G$: the number of RK4 steps on $-L \le y \le 0$; the step
  is $h = L/G$.
- **Trapezoid rule**: the integral over one step approximated by the step length
  times the average of the two end values.
- **Bracket**: two energies at which $\Phi$ minus the target has opposite signs;
  the level lies between them.
- **Newton's method**: replace a function by its tangent line at the current
  guess and take the zero of that line as the next guess.
- **Transcendental equation**: an equation such as $\tan(pL) = -p/M$ that has no
  solution formula and is solved numerically.
```

Text cell:

4. The physical and mathematical situation

The equations. The record gives the Kohn-Sham equation of one 2-component block in the real form (key ‘blockEquation.realForm‘)
$a’ = M a - (\kappa k + j(\varepsilon - v)) b$,
$b’ = (j(\varepsilon - v) - \kappa k) a - M b$, where the prime is d/dy. In the free case with no 3-space momentum ($k = 0$), no potential ($v = 0$), constant mass M and the block type $j = +1$ they are

$$a’ = M a - \varepsilon b, \qquad b’ = \varepsilon a - M b, \qquad -L \le y \le 0,$$

with $b(-L) = 0$ at the tip and $b(0) = 0$ (even) or $a(0) = 0$ (odd) at the brane. The record’s values are $M = m = 1$, $H = 1$, $L = 3$; the program also checks $L = 2$ and $M = 2$. In this notebook we take the equations as given; the physics chapters derive them.

The exact levels. Step 1: from the first equation, $b = (M a - a’)/\varepsilon$ (for $\varepsilon \ne 0$). Step 2: differentiate: $b’ = (M a’ - a’’)/\varepsilon$. Step 3: put both into the second equation: $(M a’ - a’’)/\varepsilon = \varepsilon a - M (M a - a’)/\varepsilon$. Step 4: multiply by ε: $M a’ - a’’ = \varepsilon^2 a - M^2 a + M a’$. Step 5: cancel $M a’$:

$$a’’ = (M^2 - \varepsilon^2)\, a .$$

For $|\varepsilon| > M$ put $p = \sqrt{\varepsilon^2 - M^2}$; then $a’’ = -p^2 a$ and $a = \cos(p y + \varphi)$ with a constant φ. The conditions on b become conditions on a: $b = 0$ means $M a - a’ = 0$.

- Even: $M a - a’ = 0$ at BOTH ends. $M\cos(py + \varphi) + p \sin(py + \varphi) = R \cos(py + \varphi - \delta)$ with $\tan\delta = p/M$ vanishes at $y = 0$ and at $y = -L$ only if the two phases differ by a multiple of π: $pL = n\pi$. So $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$, $n = 1, 2, \dots$
- Odd: $a(0) = 0$ gives $a = \sin(py)$; the tip condition $M \sin(-pL) - p\cos(-pL) = 0$ gives $\tan(pL) = -p/M$.
- $\varepsilon = 0$: the equations become $a’ = M a$, $b’ = -M b$; with $b(-L) = 0$ we get $b = 0$ everywhere and $a = e^{M y}$: the *zero mode*, an even orbital ($b(0) = 0$) with energy exactly 0.

The Pruefer angle. Write $a = r\cos\theta$, $b = r\sin\theta$. Then $\theta’ = (a b’ - b a’)/r^2$ (the derivative of the angle of a moving point). Insert the equations: $a b’ - b a’ = a(\varepsilon a - M b) - b(M a - \varepsilon b) = \varepsilon (a^2 + b^2) - 2 M a b$. Divide by $r^2 = a^2 + b^2$ and use $2ab/r^2 = 2\sin\theta\cos\theta = \sin 2\theta$:

$$\theta’ = \varepsilon - M \sin 2\theta, \qquad \theta(-L) = 0 .$$

The brane conditions say $b(0) = 0$, that is $\theta(0) = l\pi$ (even), or $a(0) = 0$, that is $\theta(0) = \pi/2 + l\pi$ (odd), for a whole number l.

How fast the end angle grows. First the distance r: from $r^2 = a^2 + b^2$, $r r’ = a a’ + b b’ = a(M a - \varepsilon b) + b(\varepsilon a - M b) = M(a^2 - b^2)$, and $(a^2 - b^2)/r^2 = \cos^2\theta - \sin^2\theta = \cos 2\theta$, so $r’/r = M\cos 2\theta$. Now let $u(y) = \partial\theta(y)/\partial\varepsilon$,

```
the change of the angle at $y$ per unit change of the energy. Differentiate
$\theta' = \varepsilon - M\sin 2\theta$ with respect to $\varepsilon$ (the order
of the two derivatives does not matter): $u' = 1 - 2M\cos(2\theta)\, u =
1 - 2 (r'/r)\, u$. Multiply by $r^2$: $r^2 u' + 2 r r' u = r^2$. By the product
rule the left side is $(r^2 u)'$. Integrate from $-L$ to $0$; at the tip
$u(-L) = 0$, because $\theta(-L) = 0$ for every energy:

$$\frac{d\Phi}{d\varepsilon} = u(0) = \frac{1}{r(0)^2}\int_{-L}^{0} r^2\,dy > 0 .$$

So $\Phi(\varepsilon)$ increases strictly, and each target $l\pi$ or
$\pi/2 + l\pi$ is reached for exactly one $\varepsilon$: every level has its own
label $l$ and none can be missed. This is how the Rust program finds and names
its levels, and the formula gives its root finder the slope it needs.
```

Text cell:

```
## 5. The records and the settings

The next cell reads the three Revision records this notebook uses: the theory
(the equations, the boundary conditions with their status, the exact spectra),
the solver's parameters (900 RK4 steps, root tolerance $10^{-13}$) and the
table of the free spectrum. It prints the statements it relies on.
```

In [2]:

```
import csv  # reads the table of levels (comma-separated values)
import math  # sqrt, atan2, pi for single numbers
import re  # finds numbers inside a text

import mpmath  # numbers with as many digits as we ask for
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
GREY, BLACK = "#8a8986", "#000000"

THEORY = "Revision/kohn_sham/ks-theory.json"
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
TABLE = "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
SOLVER_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
theory = json.loads(repository_file(THEORY).read_text(encoding="utf-8"))
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
with open(repository_file(TABLE), encoding="utf-8", newline="") as table:
    ROWS = list(csv.DictReader(table))  # one dictionary per level
say("realForm: " + theory["blockEquation"]["realForm"])
say("brane status: " + theory["boundaryConditions"]["brane"]["status"])
say("tip status: " + theory["boundaryConditions"]["tip"]["status"])
say("exact k = 0 spectra: " + theory["boundaryConditions"]["exactK0Spectra"])
G_CANONICAL = parameters["numerics"]["rk4Steps"]  # RK4 steps for L = 3
ROOT_TOLERANCE = parameters["numerics"]["rootTolerance"]
M_RECORD, L_RECORD = parameters["physics"]["m"], parameters["physics"]["L_tipCutoff"]
report("RK4 steps, root tolerance, m, L of the record",
```

```
       f"{G_CANONICAL}, {ROOT_TOLERANCE}, {M_RECORD}, {L_RECORD}")
report("levels in the table", len(ROWS))
check(theory["boundaryConditions"]["brane"]["status"] == "ASSUMED"
      and G_CANONICAL == 900 and ROOT_TOLERANCE == 1e-13 and len(ROWS) == 54,
      "records read: brane ASSUMED, 900 RK4 steps, tolerance 1e-13, 54 levels")
```

Out [2]:

```
realForm: chi = (a, i b), a, b real: a' = M_eff a - (kappa k + j(eps - v_v)) b, b' =
    (j(eps - v_v) - kappa k) a - M_eff b
brane status: ASSUMED
tip status: chosen (regular tip)
exact k = 0 spectra: constant M > 0, v = 0, theta = 0: even: eps = 0 with chi = (e^{My},
    0), and eps = +-sqrt(M^2 + (n pi/L)^2); odd: eps = +-sqrt(M^2 + p^2) with tan(pL) =
    -p/M
RESULT RK4 steps, root tolerance, m, L of the record = 900, 1e-13, 1.0, 3.0
RESULT levels in the table = 54
PASS records read: brane ASSUMED, 900 RK4 steps, tolerance 1e-13, 54 levels
```

Text cell:

```
## 6. Shooting with RK4 and counting the turns of the angle

The function ‘shoot‘ follows the Rust program (file
Revision/kohn_sham/solver/src/shoot.rs) line by line: it starts at the tip
with $(a, b) = (1, 0)$, so $\theta = 0$; makes $G$ RK4 steps of the two
equations; after every step it takes the new angle ‘atan2(b, a)‘, adds the
change of angle since the last step (brought into the range from $-\pi$ to
$\pi$, so that a full turn is never lost: one step turns the point by much less
than $\pi$) and so counts every turn. Along the way it adds up
$\int r^2\,dy$ with the trapezoid rule. It returns $\Phi = \theta(0)$, the slope
$d\Phi/d\varepsilon = \int r^2\,dy / r(0)^2$ of section 4, the largest turn of a
single step (the program refuses a shot in which one step turns by 2 or more),
and, when asked, the whole path. (The program also shrinks $(a, b)$ when they
become astronomically large, above $10^{125}$; that never happens here.)

The second function integrates the single angle equation
$\theta' = \varepsilon - M \sin 2\theta$ with RK4 instead; the two different
computations must agree up to the small RK4 errors.
```

In [3]:

```
def shoot(eps, M, L, G, keep=False):
    """RK4 from y = -L, (a, b) = (1, 0), to y = 0 in G steps.  Returns Phi =
    theta(0), the slope dPhi/deps = (integral of r^2) / r(0)^2 and the largest
    turn of one step; with keep=True also the arrays y, a, b, theta of the path."""
    h = L / G  # the step
    a, b = 1.0, 0.0  # the tip: b(-L) = 0, so theta(-L) = 0
    theta, raw = 0.0, 0.0  # the counted angle and the last atan2 value
    integral, r2_before = 0.0, 1.0  # running integral of r^2; r^2 at the last node
    largest = 0.0  # the largest turn of one step so far
```

```
    path = [(-L, a, b, theta)]
    for i in range(G):
        p1a, p1b = M * a - eps * b, eps * a - M * b  # the slopes (a', b') at start
        a2, b2 = a + h / 2 * p1a, b + h / 2 * p1b  # half a step with them
        p2a, p2b = M * a2 - eps * b2, eps * a2 - M * b2  # the slopes there
        a3, b3 = a + h / 2 * p2a, b + h / 2 * p2b  # half a step with these
        p3a, p3b = M * a3 - eps * b3, eps * a3 - M * b3  # the slopes there
        a4, b4 = a + h * p3a, b + h * p3b  # a whole step with the third slopes
        p4a, p4b = M * a4 - eps * b4, eps * a4 - M * b4  # the slopes at the end
        a += h / 6 * (p1a + 2 * p2a + 2 * p3a + p4a)  # weights 1 : 2 : 2 : 1
        b += h / 6 * (p1b + 2 * p2b + 2 * p3b + p4b)
        new = math.atan2(b, a)  # the angle of (a, b), between -pi and pi
        change = new - raw
        if change > math.pi:  # crossed from just below pi to just above -pi
            change -= 2 * math.pi
        elif change <= -math.pi:  # crossed the other way
            change += 2 * math.pi
        largest = max(largest, abs(change))
        theta += change
        raw = new
        r2 = a * a + b * b  # r^2 at the new node
        integral += h / 2 * (r2_before + r2)  # the trapezoid rule for this step
        r2_before = r2
        if keep:
            path.append((-L + (i + 1) * h, a, b, theta))
    if keep:
        return theta, integral / r2_before, largest, np.array(path).T
    return theta, integral / r2_before, largest


def angle_equation(eps, M, L, G):
    """RK4 for the single equation theta' = eps - M sin(2 theta), theta(-L) = 0."""
    h = L / G
    theta = 0.0

    def slope(t):
        return eps - M * math.sin(2 * t)
    for _ in range(G):
        k1 = slope(theta)
        k2 = slope(theta + h / 2 * k1)
        k3 = slope(theta + h / 2 * k2)
        k4 = slope(theta + h * k3)
        theta += h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return theta


M1, L3 = 1.0, 3.0  # the canonical case of the record
for eps in (-2.0, 0.0, 1.0, 2.5):
    Phi = shoot(eps, M1, L3, 900)[0]  # element 0 of the result: Phi
    say(f"eps = {eps:+.1f}: Phi by (a, b) = {Phi:+.12f},  "
        f"by the angle equation = {angle_equation(eps, M1, L3, 900):+.12f}")
differences = [abs(shoot(e, M1, L3, 900)[0] - angle_equation(e, M1, L3, 900))
               for e in np.linspace(-5.0, 5.0, 21)]
check(max(differences) < 1e-8 and shoot(0.0, M1, L3, 900)[0] == 0.0,
```

```
      "both ways of computing Phi agree within 1e-8; Phi(0) is exactly 0")
```

Out [3]:

```
eps = -2.0: Phi by (a, b) = -4.667432266938,  by the angle equation = -4.667432266940
eps = +0.0: Phi by (a, b) = +0.000000000000,  by the angle equation = +0.000000000000
eps = +1.0: Phi by (a, b) = +0.643501108793,  by the angle equation = +0.643501108793
eps = +2.5: Phi by (a, b) = +6.798217350015,  by the angle equation = +6.798217350074
PASS both ways of computing Phi agree within 1e-8; Phi(0) is exactly 0
```

Text cell:

```
## 7. The staircase of the Pruefer angle

The next cell computes $\Phi(\varepsilon)$ for 651 energies from $-6$ to $7$
($M = 1$, $L = 3$, 900 steps) and checks that it increases. The figure shows
it with the target lines $l\pi$ (even) and $\pi/2 + l\pi$ (odd): the levels are
the crossings, every target is crossed once, and the levels alternate between
even and odd. Away from the levels the curve rises with slope about $L = 3$,
because $\theta' = \varepsilon - M\sin 2\theta$ is $\varepsilon$ on average.

Inside the *mass gap* $|\varepsilon| \le M$ the equation
$a'' = (M^2 - \varepsilon^2)\, a$ has no oscillating solutions, and the only
level there is the zero mode. The cell checks this for the three cases
$(M, L)$ of the table: $\Phi(-M)$ and $\Phi(+M)$ both lie between $-\pi/2$ and
$\pi/2$, so the increasing staircase crosses no target except $0$ while
$|\varepsilon| \le M$.
```

In [4]:

```
eps_grid = np.linspace(-6.0, 7.0, 651)
shots = [shoot(e, M1, L3, 900) for e in eps_grid]  # (Phi, slope, largest turn)
Phi_grid = np.array([s[0] for s in shots])
slope_grid = np.array([s[1] for s in shots])  # dPhi/deps by the formula
check(np.all(np.diff(Phi_grid) > 0), "Phi(eps) increases on the whole grid")
gap_ends = []  # Phi at eps = -M and eps = +M for the three cases of the table
for M, L in sorted({(float(r["m"]), float(r["L"])) for r in ROWS}):
    G = round(G_CANONICAL * L / 3.0)  # the record's step h = 1/300
    gap_ends += [shoot(-M, M, L, G)[0], shoot(M, M, L, G)[0]]
report("largest |Phi(-M)|, |Phi(+M)| in units of pi (three cases)",
       f"{max(abs(p) for p in gap_ends) / math.pi:.4f}")
check(max(abs(p) for p in gap_ends) < math.pi / 2,
      "for |eps| <= M the staircase crosses no target but 0: only the zero mode")


def target(parity, label):
    """The target value of Phi for a level of the given parity and label."""
    return label * math.pi if parity == "even" else math.pi / 2 + label * math.pi


canonical = [r for r in ROWS if float(r["m"]) == 1.0 and float(r["L"]) == 3.0]
```

```
fig, ax = plt.subplots(figsize=(7.0, 5.4))
ax.plot(eps_grid, Phi_grid / math.pi, color=BLACK, lw=1.4,
        label="$\\Phi(\\varepsilon)/\\pi$ (RK4, 900 steps)")
for label in range(-4, 7):
    ax.axhline(label, color=BLUE, lw=0.6, alpha=0.6)
    ax.axhline(label + 0.5, color=ORANGE, lw=0.6, ls="--", alpha=0.6)
for r in canonical:
    is_even = r["parity"] == "even"
    ax.plot(float(r["eps_numeric"]), target(r["parity"], int(r["label"])) / math.pi,
            "o" if is_even else "s", color=BLUE if is_even else ORANGE, ms=6)
ax.plot([], [], "o", color=BLUE, label="even levels: $\\Phi = l\\pi$")
ax.plot([], [], "s", color=ORANGE, label="odd levels: $\\Phi = \\pi/2 + l\\pi$")
ax.set_ylim(-4.2, 6.2)
ax.set_xlabel("energy $\\varepsilon$ (units $m$)")
ax.set_ylabel("end angle $\\Phi = \\theta(0)$ in units of $\\pi$")
ax.set_title("The Pruefer staircase: $M = 1$, $L = 3$")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "pruefer_staircase",
            "The end value of the Pruefer angle, $\\Phi(\\varepsilon) = "
            "\\theta(0)$ in units of $\\pi$ (vertical axis), against the energy "
            "$\\varepsilon$ in units of the mass $m$ from $-6$ to $7$ (horizontal "
            "axis), for $M = 1$, $L = 3$ and 900 RK4 steps. Blue lines: the even "
            "targets $l\\pi$; dashed orange lines: the odd targets "
            "$\\pi/2 + l\\pi$. Dots and squares: the 18 levels of the Revision "
            "record for this case. $\\Phi$ increases steadily, so it crosses every "
            "target exactly once: each level has its own label $l$, and even and "
            "odd levels alternate. The zero mode sits at $\\varepsilon = 0$, "
            "$\\Phi = 0$.")
```

Out [4]:

```
PASS Phi(eps) increases on the whole grid
RESULT largest |Phi(-M)|, |Phi(+M)| in units of pi (three cases) = 0.2256
PASS for |eps| <= M the staircase crosses no target but 0: only the zero mode
```

Out [4]:

```
Figure 02d.1 saved as Revision/textbook/figures/02d_1_pruefer_staircase.png
```

Text cell:

```
## 8. The slope of the staircase

Section 4 proved $d\Phi/d\varepsilon = \int_{-L}^{0} r^2\,dy / r(0)^2$, which
is never negative. The next cell tests this formula in two ways. First, it
compares the formula (computed by ‘shoot‘ with the trapezoid rule) with the
*central difference* $(\Phi(\varepsilon + \delta) - \Phi(\varepsilon -
\delta))/(2\delta)$, $\delta = 10^{-4}$, at seven energies, and with the
value $(1 - e^{-2ML})/(2M)$ at $\varepsilon = 0$, where the shot is exactly
$a = e^{M(y + L)}$, $b = 0$ and the integral can be done by hand. Second, it
draws the formula over the whole grid together with the central differences of
```

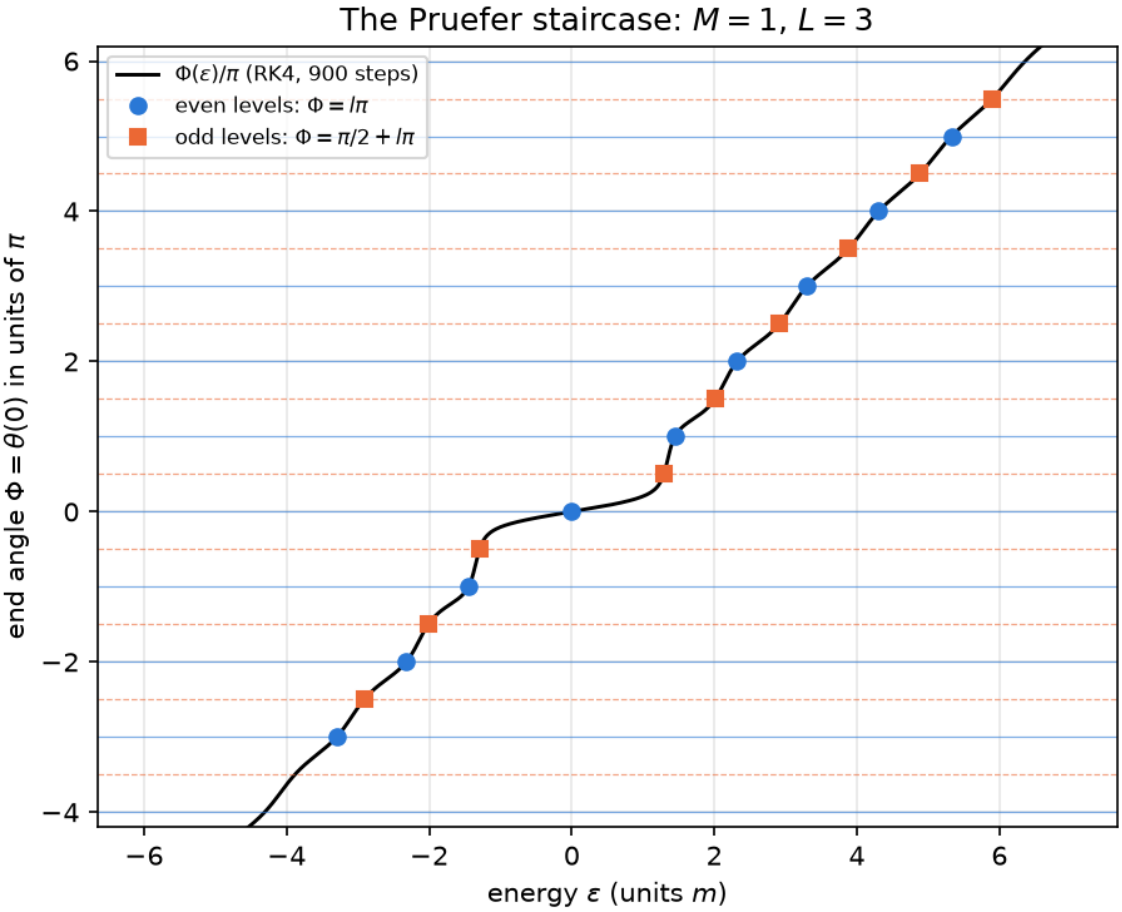

Figure 84: The end value of the Pruefer angle, $\Phi(\varepsilon) = \theta(0)$ in units of π (vertical axis), against the energy ε in units of the mass m from -6 to 7 (horizontal axis), for $M = 1$, $L = 3$ and 900 RK4 steps. Blue lines: the even targets $l\pi$; dashed orange lines: the odd targets $\pi/2 + l\pi$. Dots and squares: the 18 levels of the Revision record for this case. Φ increases steadily, so it crosses every target exactly once: each level has its own label l, and even and odd levels alternate. The zero mode sits at $\varepsilon = 0$, $\Phi = 0$. (Notebook 02d, figure 1.)

```
the grid of section 7 (spacing 0.02). The slope is about $L = 3$ far from the
mass. Near $\varepsilon = 0$ it is about $1/(2M) = 0.5$: there the shot grows
like $e^{M(y + L)}$ and is largest at the brane, so $r(0)^2$ is large. Near
$|\varepsilon| = 1.34$ it is much larger, about 12.5: there the shot ends with a
small $r(0)$ compared with its size inside the interval, so a small change of
energy turns the end angle a lot, and the staircase climbs steeply.
```

In [5]:

```
DELTA = 1e-4  # the half-width of the central difference
tested = [-4.0, -1.34, -0.5, 0.0, 0.7, 1.3, 3.0]  # seven energies
relative = []
for e in tested:
    formula = shoot(e, M1, L3, 900)[1]  # element 1 of the result: the slope
    central = (shoot(e + DELTA, M1, L3, 900)[0]
               - shoot(e - DELTA, M1, L3, 900)[0]) / (2 * DELTA)
    relative.append(abs(central / formula - 1))
    say(f"eps = {e:+.2f}: formula {formula:9.5f}, central difference {central:9.5f}")
report("largest relative difference formula - central difference",
       f"{max(relative):.1e}")
check(max(relative) < 1e-4 and np.all(slope_grid > 0),
      "dPhi/deps = (integral of r^2)/r(0)^2 > 0 (agrees with central differences)")
# At eps = 0 the shot is exactly a = e^(M(y + L)), b = 0, so the integral can be
# done by hand: (e^(2ML) - 1)/(2M) divided by r(0)^2 = e^(2ML).
at_zero = (1 - math.exp(-2 * M1 * L3)) / (2 * M1)
report("slope at eps = 0: formula, exact (1 - e^(-2ML))/(2M)",
       f"{shoot(0.0, M1, L3, 900)[1]:.6f}, {at_zero:.6f}")
check(abs(shoot(0.0, M1, L3, 900)[1] - at_zero) < 1e-4,
      "at eps = 0 the slope is (1 - e^(-2ML))/(2M), about 1/(2M)")

grid_difference = (Phi_grid[2:] - Phi_grid[:-2]) / (eps_grid[2:] - eps_grid[:-2])
fig, ax = plt.subplots()
ax.plot(eps_grid, slope_grid, color=BLACK, lw=1.4,
        label="formula $\\int r^2 dy / r(0)^2$")
ax.plot(eps_grid[1:-1][::6], grid_difference[::6], "o", color=AQUA, ms=4,
        label="central differences of the staircase")
ax.axhline(L3, color=GREY, ls="--", lw=1.0, label="$L = 3$")
ax.set_ylim(0.0, 17.5)  # room for the legend above the two peaks
ax.set_xlabel("energy $\\varepsilon$ (units $m$)")
ax.set_ylabel("$d\\Phi/d\\varepsilon$ (units $1/m$)")
ax.set_title("The slope of the staircase is never negative")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "staircase_slope",
            "The slope $d\\Phi/d\\varepsilon$ of the end angle (vertical axis, "
            "units $1/m$) against the energy $\\varepsilon$ in units of $m$ "
            "(horizontal axis), for $M = 1$, $L = 3$, 900 RK4 steps. Black line: "
            "the formula $\\int r^2 dy / r(0)^2$ proved with the product rule; "
            "aqua dots: central differences of the staircase of the previous "
            "figure. The slope is positive everywhere, which is why each level "
            "is crossed once. It is about $L = 3$ (dashed) far from the mass, "
            "about $1/(2M) = 0.5$ near $\\varepsilon = 0$, where the shot grows "
            "towards the brane, and large near $|\\varepsilon| = 1.34$, where the "
            "shot ends with a small $r(0)$ and the staircase climbs steeply.")
```

Out [5]:

```
eps = -4.00: formula   3.85717, central difference   3.85717
eps = -1.34: formula  12.49831, central difference  12.49830
eps = -0.50: formula   0.57078, central difference   0.57078
eps = +0.00: formula   0.49876, central difference   0.49876
eps = +0.70: formula   0.67374, central difference   0.67374
eps = +1.30: formula   9.67899, central difference   9.67899
eps = +3.00: formula   4.20995, central difference   4.20995
RESULT largest relative difference formula - central difference = 3.7e-06
PASS dPhi/deps = (integral of r^2)/r(0)^2 > 0 (agrees with central differences)
RESULT slope at eps = 0: formula, exact (1 - e^(-2ML))/(2M) = 0.498762, 0.498761
PASS at eps = 0 the slope is (1 - e^(-2ML))/(2M), about 1/(2M)
```

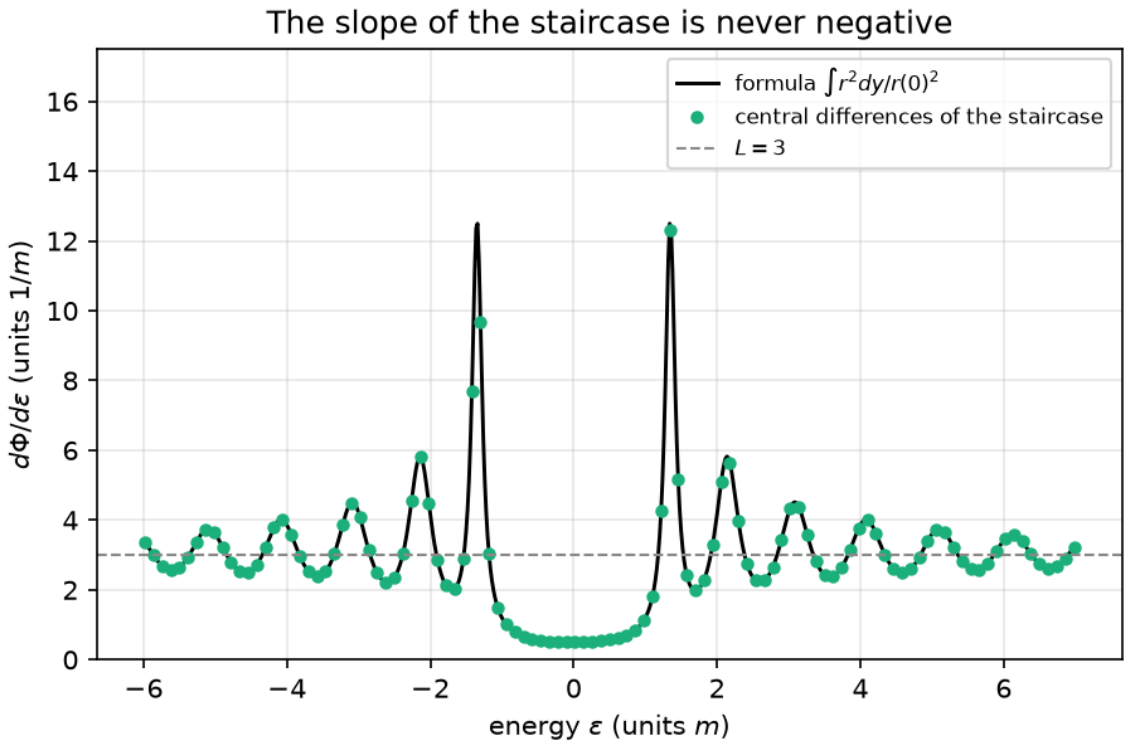

Figure 85: The slope $d\Phi/d\varepsilon$ of the end angle (vertical axis, units $1/m$) against the energy ε in units of m (horizontal axis), for $M = 1$, $L = 3$, 900 RK4 steps. Black line: the formula $\int r^2 dy/r(0)^2$ proved with the product rule; aqua dots: central differences of the staircase of the previous figure. The slope is positive everywhere, which is why each level is crossed once. It is about $L = 3$ (dashed) far from the mass, about $1/(2M) = 0.5$ near $\varepsilon = 0$, where the shot grows towards the brane, and large near $|\varepsilon| = 1.34$, where the shot ends with a small $r(0)$ and the staircase climbs steeply. (Notebook 02d, figure 2.)

Out [5]:

```
Figure 02d.2 saved as Revision/textbook/figures/02d_2_staircase_slope.png
```

Text cell:

```
## 9. The angle along the interval

For five of the levels the next cell keeps the whole path and draws
$\theta(y)/\pi$ from the tip $y = -3$ to the brane $y = 0$. Each curve starts
at $0$ and ends exactly on its target; the number of half-turns it makes is the
label. The zero mode stays at $\theta = 0$ all the way ($b = 0$ everywhere).
```

In [6]:

```
chosen = [("even", 0), ("odd", 0), ("even", 1), ("odd", 1), ("even", 3)]
level_of = {(r["parity"], int(r["label"])): float(r["eps_numeric"]) for r in canonical}
fig, ax = plt.subplots()
for (parity, label), color in zip(chosen, [BLACK, ORANGE, BLUE, VIOLET, AQUA]):
    eps = level_of[(parity, label)]
    y_path, a_path, b_path, theta_path = shoot(eps, M1, L3, 900, keep=True)[3]
    ax.plot(y_path, theta_path / math.pi, color=color, lw=1.5,
            ls="-" if parity == "even" else "--",
            label=f"{parity} $l = {label}$, $\\varepsilon = {eps:.4f}$")
    ax.plot([0.0], [target(parity, label) / math.pi], "o", color=color, ms=6)
ax.set_xlabel("hidden coordinate $y$ (units $1/H$): tip at $-3$, brane at $0$")
ax.set_ylabel("Pruefer angle $\\theta(y)/\\pi$")
ax.set_title("The angle winds up to its target at the brane")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "angle_along_y",
            "The Pruefer angle $\\theta(y)$ in units of $\\pi$ (vertical axis) "
            "along the hidden coordinate $y$ from the tip $y = -3$ to the brane "
            "$y = 0$ (horizontal axis, units $1/H$) for five levels of the case "
            "$M = 1$, $L = 3$: the zero mode (black, stays at 0), the odd label 0 "
            "(1.2923, ends at $\\pi/2$), the even label 1 (1.4480, ends at $\\pi$), "
            "the odd label 1 (2.0106, ends at $3\\pi/2$) and the even label 3 "
            "(3.2969, ends at $3\\pi$). Solid lines are even, dashed lines odd "
            "orbitals; every curve starts at 0 and ends on its target (dots).")
```

Out [6]:

```
Figure 02d.3 saved as Revision/textbook/figures/02d_3_angle_along_y.png
```

Text cell:

```
## 10. The exact levels, computed again

The odd levels need the roots $p$ of $\tan(pL) = -p/M$, which we write as
$M\sin(pL) + p\cos(pL) = 0$ (no infinities). The $n$-th root ($n = 0, 1, 2,
\dots$) lies between $(n + \frac{1}{2})\pi/L$ and $(n + 1)\pi/L$, where the
left side changes sign. The cell solves them with mpmath at 30 digits, forms
all 54 exact levels of the table (labels $-3$ to $5$ of both parities, for
$(M, L) = (1, 3), (1, 2), (2, 3)$; the odd label $l \ge 0$ uses the root
$p_l$ and $l \le -1$ the root $p_{-l-1}$ with the minus sign) and compares them
with the record's column `eps_analytic`. The figure shows the graphical
solution for $M = 1$, $L = 3$.
```

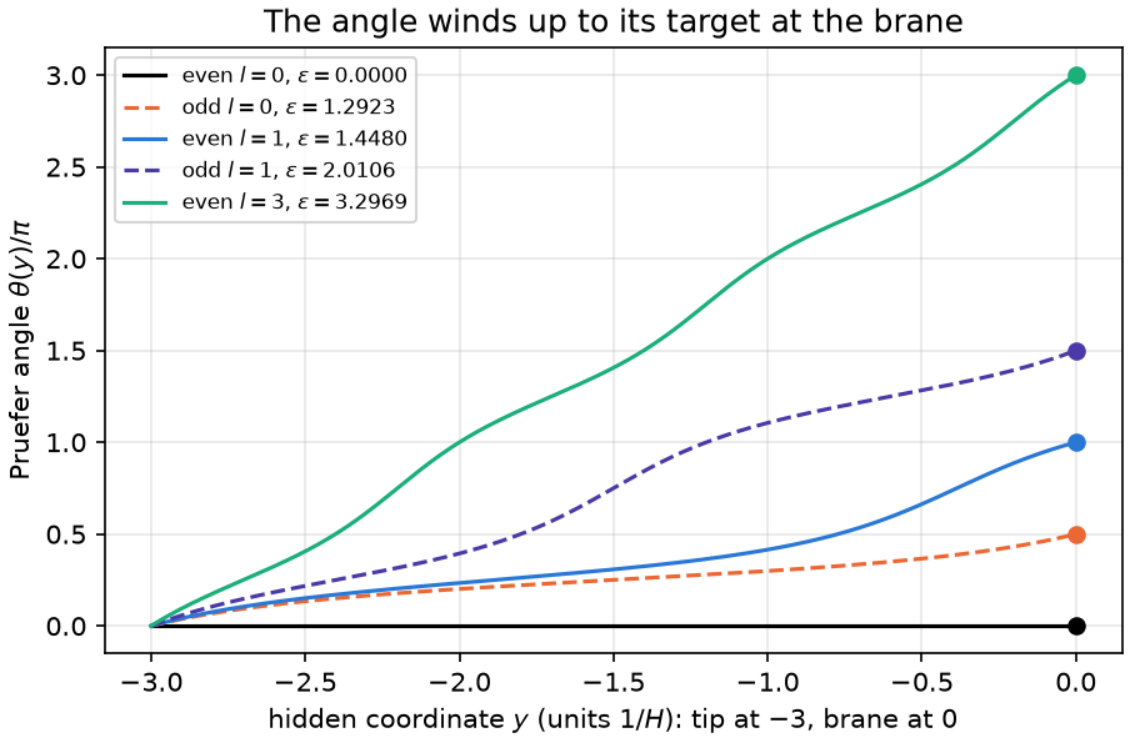

Figure 86: The Pruefer angle $\theta(y)$ in units of π (vertical axis) along the hidden coordinate y from the tip $y = -3$ to the brane $y = 0$ (horizontal axis, units $1/H$) for five levels of the case $M = 1$, $L = 3$: the zero mode (black, stays at 0), the odd label 0 (1.2923, ends at $\pi/2$), the even label 1 (1.4480, ends at π), the odd label 1 (2.0106, ends at $3\pi/2$) and the even label 3 (3.2969, ends at 3π). Solid lines are even, dashed lines odd orbitals; every curve starts at 0 and ends on its target (dots). (Notebook 02d, figure 3.)

In [7]:

```
mpmath.mp.dps = 30


def odd_root(M, L, n):
    """The n-th positive root p of M sin(pL) + p cos(pL) = 0 (30 digits)."""
    g = lambda p: M * mpmath.sin(p * L) + p * mpmath.cos(p * L)  # noqa: E731
    return mpmath.findroot(g, ((n + 0.5) * mpmath.pi / L, (n + 1) * mpmath.pi / L),
                           solver="anderson")


def exact_level(M, L, parity, label):
    """The exact level of the given parity and label."""
    if parity == "even":
        if label == 0:
            return 0.0  # the zero mode
        return math.copysign(float(mpmath.sqrt(M ** 2 + (label * mpmath.pi / L) ** 2)),
                             label)
    n = label if label >= 0 else -label - 1
    value = float(mpmath.sqrt(M ** 2 + odd_root(M, L, n) ** 2))
    return value if label >= 0 else -value


analytic_ours = [exact_level(float(r["m"]), float(r["L"]), r["parity"],
                             int(r["label"])) for r in ROWS]
analytic_record = [float(r["eps_analytic"]) for r in ROWS]
worst_analytic = max(abs(a - b) for a, b in zip(analytic_ours, analytic_record))
report("largest |exact level (here) - eps_analytic (record)|", f"{worst_analytic:.1e}")
check(worst_analytic < 1e-14, "all 54 exact levels equal the column eps_analytic",
      record=f"{TABLE}, column eps_analytic")

p_axis = np.linspace(0.01, 3.3, 3000)
tangent = np.tan(p_axis * L3)
tangent[np.abs(tangent) > 8] = np.nan  # do not draw the jumps at the poles
fig, ax = plt.subplots()
ax.plot(p_axis, tangent, color=BLUE, lw=1.4, label="$\\tan(pL)$, $L = 3$")
ax.plot(p_axis, -p_axis / M1, color=ORANGE, lw=1.4, ls="--", label="$-p/M$, $M = 1$")
for n in range(3):
    p_n = float(odd_root(M1, L3, n))
    ax.plot([p_n], [-p_n], "o", color=BLACK, ms=6)
    ax.annotate(f"$p_{n} = {p_n:.4f}$", (p_n, -p_n), textcoords="offset points",
                xytext=(6, -14), fontsize=8)
ax.axhline(0.0, color=BLACK, lw=0.6)
ax.set_ylim(-6.0, 6.0)
ax.set_xlabel("$p$ (units $m$)")
ax.set_ylabel("value of each side")
ax.set_title("Odd levels: $\\tan(pL) = -p/M$, then $\\varepsilon = \\sqrt{M^2 + p^2}$")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "odd_level_equation",
            "The equation of the odd levels for $M = 1$ and $L = 3$: the branches "
            "of $\\tan(pL)$ (solid blue) and the line $-p/M$ (dashed orange) "
            "against $p$ in units of $m$ (horizontal axis; the vertical axis is "
            "the value of each side, a pure number). Each crossing, one on every "
```

```
"branch between $(n + 1/2)\\pi/L$ and $(n + 1)\\pi/L$, is a root "
"$p_n$ and gives the two odd levels $\\pm\\sqrt{M^2 + p_n^2}$; the "
"first three give 1.2923, 2.0106 and 2.9119.")
```

Out [7]:

```
RESULT largest |exact level (here) - eps_analytic (record)| = 1.8e-15
PASS all 54 exact levels equal the column eps_analytic
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, column
    eps_analytic
```

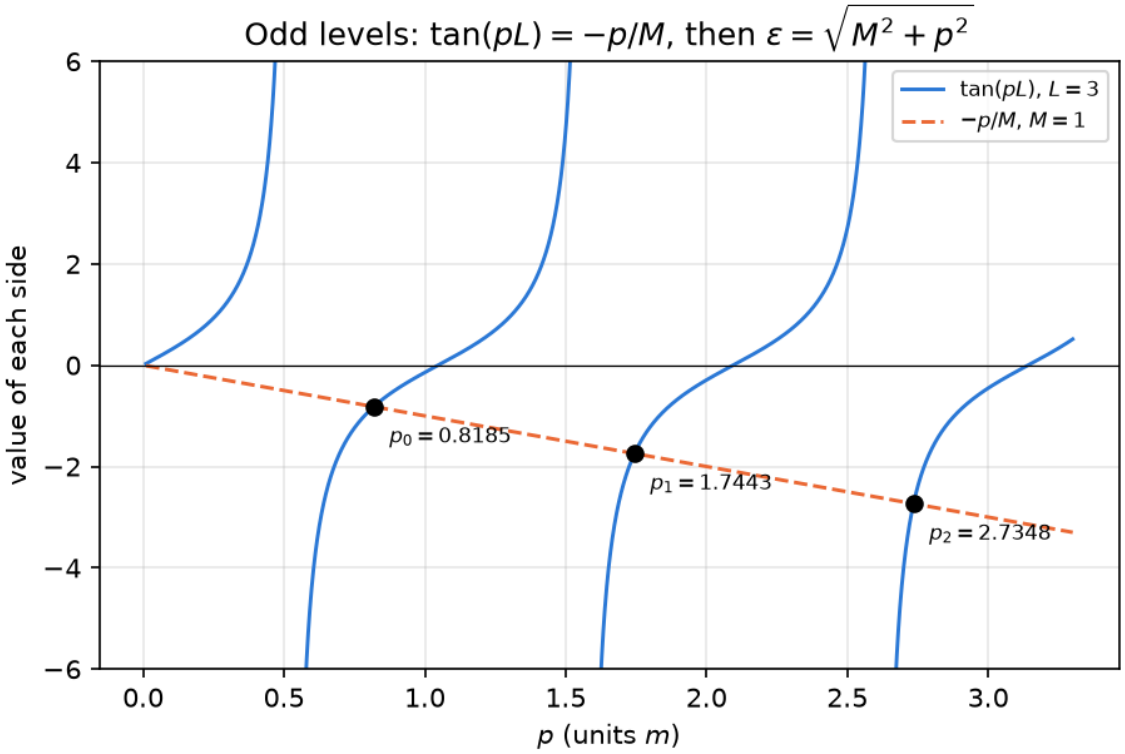

Figure 87: The equation of the odd levels for $M = 1$ and $L = 3$: the branches of $\tan(pL)$ (solid blue) and the line $-p/M$ (dashed orange) against p in units of m (horizontal axis; the vertical axis is the value of each side, a pure number). Each crossing, one on every branch between $(n + 1/2)\pi/L$ and $(n + 1)\pi/L$, is a root p_n and gives the two odd levels $\pm\sqrt{M^2 + p_n^2}$; the first three give 1.2923, 2.0106 and 2.9119. (Notebook 02d, figure 4.)

Out [7]:

```
Figure 02d.4 saved as Revision/textbook/figures/02d_4_odd_level_equation.png
```

Text cell:

```
## 11. Newton's method with a safety net: the program's root finder
```

```
To find a level we must solve $F(\varepsilon) = \Phi(\varepsilon) - t = 0$ for
its target $t$. **Newton's method** replaces $F$ by its tangent line at the
current guess $\varepsilon_c$, $F(\varepsilon) \approx F(\varepsilon_c) +
F'(\varepsilon_c)(\varepsilon - \varepsilon_c)$, and takes the zero of that line
as the next guess:

$$\varepsilon_{\rm new} = \varepsilon_c - \frac{F(\varepsilon_c)}
{F'(\varepsilon_c)}, \qquad F'(\varepsilon) = \frac{d\Phi}{d\varepsilon} =
\frac{1}{r(0)^2}\int_{-L}^{0} r^2\,dy .$$

Close to the root the number of correct digits roughly doubles at every step.
Far from it the tangent can point anywhere, so the Rust program (function
`find_level` in Revision/kohn_sham/solver/src/shoot.rs) adds a safety net:

1. Shoot at the starting guess $\varepsilon = 0$; if $F(0) = 0$ exactly (the zero
   mode), stop.
2. Make a bracket: walk upwards (if $F < 0$) or downwards (if $F > 0$) with a
   first step of 1.2 times Newton's estimate (at least $10^{-4}$, at most 2),
   doubling the step until $F$ changes sign.
3. Newton steps inside the bracket; whenever Newton's point falls outside the
   bracket, or the mismatch $|F|$ has not at least halved since the last step,
   bisect instead. After each shot the bracket shrinks to the side that keeps
   the sign change.
4. Stop when the bracket is narrower than the tolerance $10^{-13}$, or when a
   Newton step is smaller than it.

The next cell writes this down in Python, step for step as in the Rust code,
and compares it with plain bisection (bracket $-12$ to $12$, halved until it is
narrower than $10^{-13}$) on the even level with label 3 of the case $M = 1$,
$L = 3$: the figure shows the error of every energy that is shot.
```

In [8]:

```
def find_level(M, L, G, goal, tolerance=ROOT_TOLERANCE, log=None):
    """The energy with Phi(eps) = goal, found as the Rust program finds it.  If log
    is a list, every energy that is shot is appended to it."""
    def mismatch(eps):
        Phi, slope, largest = shoot(eps, M, L, G)
        if largest >= 2.0:  # the program refuses such a shot
            raise ValueError(f"one step turns the angle by {largest:.2f}")
        if log is not None:
            log.append(eps)
        return Phi - goal, slope  # F(eps) and F'(eps)

    e_c = 0.0  # step 1: the starting guess
    F_c, dF_c = mismatch(e_c)
    if F_c == 0.0:
        return e_c
    if F_c < 0.0:  # step 2, upwards: the level lies above
        low = e_c
        step = min(max(1.2 * (-F_c / dF_c), 1e-4), 2.0)
        while True:
            e = low + step
```

```
            F, dF = mismatch(e)
            if F >= 0.0:  # the sign changed: the bracket is (low, e)
                high = e
                if abs(F) < abs(F_c):  # keep the better of the two guesses
                    e_c, F_c, dF_c = e, F, dF
                break
            low, e_c, F_c, dF_c = e, e, F, dF  # still below: move on
            step *= 2.0
    else:  # step 2, downwards: the level lies below
        high = e_c
        step = min(max(1.2 * (F_c / dF_c), 1e-4), 2.0)
        while True:
            e = high - step
            F, dF = mismatch(e)
            if F <= 0.0:  # the sign changed: the bracket is (e, high)
                low = e
                if abs(F) < abs(F_c):
                    e_c, F_c, dF_c = e, F, dF
                break
            high, e_c, F_c, dF_c = e, e, F, dF
            step *= 2.0
    if F_c == 0.0:
        return e_c
    previous = math.inf  # |F| at the previous step
    for _ in range(300):  # step 3
        if high - low <= tolerance:  # step 4: the bracket is narrow enough
            break
        e_new = e_c - F_c / dF_c  # Newton's point
        bisect = not (low < e_new < high) or abs(F_c) > 0.5 * previous
        if bisect:
            e_new = 0.5 * (low + high)  # the safety net
        previous = abs(F_c)
        F, dF = mismatch(e_new)
        moved = abs(e_new - e_c)
        e_c, F_c, dF_c = e_new, F, dF
        if F == 0.0:
            break
        if F < 0.0:  # keep the half of the bracket with the sign change
            low = e_new
        else:
            high = e_new
        if moved <= tolerance and not bisect:  # step 4: a tiny Newton step
            break
    return e_c


def find_level_by_bisection(M, L, G, goal, tolerance=ROOT_TOLERANCE, log=None):
    """The same level by plain bisection on the bracket (-12, 12)."""
    low, high = -12.0, 12.0
    while high - low > tolerance:
        middle = 0.5 * (low + high)
        if log is not None:
            log.append(middle)
        if shoot(middle, M, L, G)[0] < goal:
```

```
            low = middle
        else:
            high = middle
    return 0.5 * (low + high)


newton_log, bisection_log = [], []
by_newton = find_level(M1, L3, 900, target("even", 3), log=newton_log)
by_bisection = find_level_by_bisection(M1, L3, 900, target("even", 3),
                                       log=bisection_log)
report("even label 3 by Newton", f"{by_newton:.15f} ({len(newton_log)} shots)")
report("even label 3 by bisection",
       f"{by_bisection:.15f} ({len(bisection_log)} shots)")
for k, e in enumerate(newton_log, 1):
    say(f"Newton shot {k}: eps = {e:.15f}, error {abs(e - by_newton):.1e}")
fig, ax = plt.subplots()
ax.semilogy(range(1, len(bisection_log) + 1),
            np.maximum(np.abs(np.array(bisection_log) - by_newton), 1e-17), "s-",
            color=ORANGE, ms=4, lw=1.2, label="bisection on $(-12, 12)$")
ax.semilogy(range(1, len(newton_log) + 1),
            np.maximum(np.abs(np.array(newton_log) - by_newton), 1e-17), "o-",
            color=BLUE, ms=5, lw=1.2, label="the program's Newton with safety net")
ax.set_ylim(1e-17, 30.0)
ax.set_xlabel("number of the shot")
ax.set_ylabel("$|\\varepsilon - \\varepsilon_3|$ (units $m$)")
ax.set_title("Finding the even level $l = 3$ ($M = 1$, $L = 3$)")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "newton_vs_bisection",
            "The distance $|\\varepsilon - \\varepsilon_3|$ (units of $m$, "
            "logarithmic vertical axis) of every energy that is shot from the "
            "even level with label 3, $\\varepsilon_3 = 3.2969$, of the case "
            "$M = 1$, $L = 3$, against the number of the shot (horizontal axis). "
            "Squares: bisection, which halves the bracket $(-12, 12)$ at every "
            "shot and needs 48 shots to reach the tolerance $10^{-13}$. Circles: "
            "the Rust program's root finder, which first walks up from 0 to make "
            "a bracket and then takes Newton steps; once close, each step about "
            "doubles the number of correct digits, and it stops after 9 shots. "
            "Distances of exactly zero are drawn at $10^{-17}$.")
check(abs(by_newton - level_of[("even", 3)]) < 1e-12
      and abs(by_bisection - by_newton) < 1e-13
      and len(newton_log) < 15 < 45 < len(bisection_log),
      "Newton (fewer than 15 shots) gives the record's level; bisection agrees",
      record=f"{TABLE}, column eps_numeric")
```

Out [8]:

```
RESULT even label 3 by Newton = 3.296908309775607 (9 shots)
RESULT even label 3 by bisection = 3.296908309775617 (48 shots)
Newton shot 1: eps = 0.000000000000000, error 3.3e+00
Newton shot 2: eps = 2.000000000000000, error 1.3e+00
Newton shot 3: eps = 6.000000000000000, error 2.7e+00
Newton shot 4: eps = 3.147568065578289, error 1.5e-01
Newton shot 5: eps = 3.279643841475118, error 1.7e-02
```

```
Newton shot 6: eps = 3.296597009921515, error 3.1e-04
Newton shot 7: eps = 3.296908208194133, error 1.0e-07
Newton shot 8: eps = 3.296908309775594, error 1.3e-14
Newton shot 9: eps = 3.296908309775607, error 0.0e+00
```

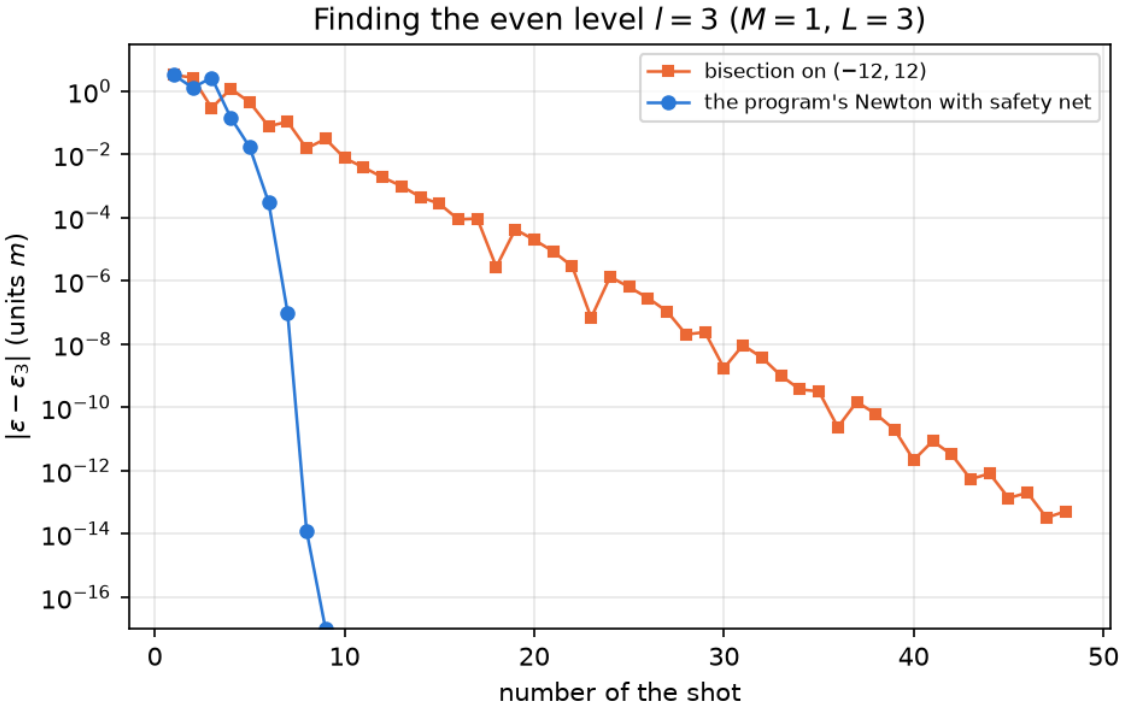

Figure 88: The distance $|\varepsilon - \varepsilon_3|$ (units of m, logarithmic vertical axis) of every energy that is shot from the even level with label 3, $\varepsilon_3 = 3.2969$, of the case $M = 1$, $L = 3$, against the number of the shot (horizontal axis). Squares: bisection, which halves the bracket $(-12, 12)$ at every shot and needs 48 shots to reach the tolerance 10^{-13}. Circles: the Rust program's root finder, which first walks up from 0 to make a bracket and then takes Newton steps; once close, each step about doubles the number of correct digits, and it stops after 9 shots. Distances of exactly zero are drawn at 10^{-17}. (Notebook 02d, figure 5.)

Out [8]:

```
Figure 02d.5 saved as Revision/textbook/figures/02d_5_newton_vs_bisection.png
PASS Newton (fewer than 15 shots) gives the record's level; bisection agrees
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, column
    eps_numeric
```

Text cell:

```
## 12. All 54 levels, against the Rust program

Now the whole table. For each row the cell takes the target of its parity and
label and finds the energy with `find_level`, with the program's number of RK4
```

steps $G = 900 L/3$ (900 for $L = 3$, 600 for $L = 2$: the same step
$h = 1/300$). It writes every level as the program writes numbers (16
significant digits, exponents like ‘e0‘) and compares the text with the
record’s column ‘eps_numeric‘. On the computer that made the record all 54 are
identical; on another computer the function atan2 of the system’s mathematics
library may round differently in the last binary place, and a last digit may
differ, so the check asks for agreement within 10^{-12} (ten times the root
tolerance). The record keeps 16 digits, so even an identical level differs from
the number read back from the record by up to about 10^{-15}.

Then it recomputes the two numbers that the program’s check
‘free_k0_analytic_spectra‘ quotes: the largest $|\varepsilon_{\rm numeric} -
\varepsilon_{\rm exact}|$ for $|\varepsilon| < 4m$ and for $|\varepsilon| \ge
4m$. (The check’s text calls the second band "$4m \le |\varepsilon| < 7m$", but
the Rust code, file Revision/kohn_sham/solver/src/spectrum.rs, puts every level
with $|\varepsilon| \ge 4m$ into it, and the table goes up to $8.75m$; the cell
prints both numbers.)

In [9]:

```
def rust_text(x):
    """x written as the Rust program writes numbers: 16 digits, exponent like e0."""
    mantissa, exponent = f"{x:.15e}".split("e")  # Python writes e+00, Rust e0
    return f"{mantissa}e{int(exponent)}"


CASES = [(1.0, 3.0), (1.0, 2.0), (2.0, 3.0)]  # the three cases (M, L) of the table
numeric_ours, shots_per_level = [], []
for r in ROWS:
    M, L = float(r["m"]), float(r["L"])
    G = round(G_CANONICAL * L / 3.0)  # 900 steps for L = 3, 600 for L = 2
    log = []
    numeric_ours.append(find_level(M, L, G, target(r["parity"], int(r["label"])),
                                   log=log))
    shots_per_level.append(len(log))
numeric_record = [float(r["eps_numeric"]) for r in ROWS]
identical = sum(rust_text(x) == r["eps_numeric"] for x, r in zip(numeric_ours, ROWS))
worst_numeric = max(abs(a - b) for a, b in zip(numeric_ours, numeric_record))
report("levels whose 16 digits equal the column eps_numeric", f"{identical} of 54")
report("largest |level (here) - eps_numeric (Rust program)|", f"{worst_numeric:.1e}")
report("shots per level (fewest, most, all 54 together)",
       f"{min(shots_per_level)}, {max(shots_per_level)}, {sum(shots_per_level)}")
check(worst_numeric < 1e-12 and max(shots_per_level) <= 20,
      "all 54 shooting levels equal the column eps_numeric (at most 20 shots each)",
      record=f"{TABLE}, column eps_numeric")
zero_rows = [i for i, r in enumerate(ROWS) if r["parity"] == "even"
             and int(r["label"]) == 0]
check(all(numeric_ours[i] == 0.0 == numeric_record[i] for i in zero_rows),
      "the zero mode has the energy exactly 0 in all three cases",
      record=f"{SOLVER_REPORT}, check free_zero_mode_exact")

differences_ours = [n - a for n, a in zip(numeric_ours, analytic_ours)]
pairs = list(zip(differences_ours, analytic_ours))
```

```
low_band = max(abs(d) for d, a in pairs if abs(a) < 4)  # all levels below 4 m
high_band = max(abs(d) for d, a in pairs if abs(a) >= 4)  # all levels from 4 m on
below_7 = max(abs(d) for d, a in pairs if 4 <= abs(a) < 7)
top = max(abs(a) for _, a in pairs)  # the highest level of the table
solver_report = json.loads(repository_file(SOLVER_REPORT).read_text(encoding="utf-8"))
detail = [c["detail"] for c in solver_report["checks"]
          if c["name"] == "free_k0_analytic_spectra"][0]
quoted = float(re.findall(r"max \|difference\| ([0-9.]+e-[0-9]+) for \|eps\| < 4 m",
                          detail)[0])
quoted_high = float(re.findall(r"([0-9.]+e-[0-9]+) for 4 m <= \|eps\| < 7 m",
                               detail)[0])
report("largest |numeric - exact| for |eps| < 4 m (here, record)",
       f"{low_band:.2e}, {quoted:.2e}")
report("largest |numeric - exact| for |eps| >= 4 m (here, record)",
       f"{high_band:.2e}, {quoted_high:.2e}")
say(f"Note: the record's check text names its second band 4 m <= |eps| < 7 m, but "
    f"the solver puts every level with |eps| >= 4 m into it, and the table's "
    f"levels reach {top:.4f} m; the quoted number belongs to that level. For "
    f"4 m <= |eps| < 7 m alone the largest difference is {below_7:.2e}.")
check(abs(low_band / quoted - 1) < 0.005 and abs(high_band / quoted_high - 1) < 0.005,
      "the two largest differences quoted by the solver check are reproduced",
      record=f"{SOLVER_REPORT}, check free_k0_analytic_spectra")
```

Out [9]:

```
RESULT levels whose 16 digits equal the column eps_numeric = 54 of 54
RESULT largest |level (here) - eps_numeric (Rust program)| = 8.9e-16
RESULT shots per level (fewest, most, all 54 together) = 1, 15, 510
PASS all 54 shooting levels equal the column eps_numeric (at most 20 shots each)
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, column
    eps_numeric
PASS the zero mode has the energy exactly 0 in all three cases
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_zero_mode_exact
RESULT largest |numeric - exact| for |eps| < 4 m (here, record) = 7.23e-10, 7.23e-10
RESULT largest |numeric - exact| for |eps| >= 4 m (here, record) = 5.05e-08, 5.05e-08
Note: the record's check text names its second band 4 m <= |eps| < 7 m, but the solver
    puts every level with |eps| >= 4 m into it, and the table's levels reach 8.7539 m;
    the quoted number belongs to that level. For 4 m <= |eps| < 7 m alone the largest
    difference is 9.95e-09.
PASS the two largest differences quoted by the solver check are reproduced
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_k0_analytic_spectra
```

Text cell:

```
## 13. The orbitals

The next cell builds the normalised orbitals of four levels of the canonical
case as the Rust program builds them (function ‘profile‘ in shoot.rs): the
values at the $G + 1$ step ends (*nodes*) come from the RK4 shot; the values in
the middle of every step from *cubic interpolation*: the cubic polynomial that
```

```
has the values $f_0, f_1$ and the slopes $f_0', f_1'$ at the two ends of a step
of length $h$ takes in the middle the value $(f_0 + f_1)/2 + h(f_0' - f_1')/8$
(check it on $f = y^2$ and $f = y^3$ on the step from 0 to $h$); the slopes
come from the equations themselves. On this fine grid of $2G + 1$ points the
norm $\int_{-L}^{0} (a^2 + b^2)\, dy$ is computed with Simpson's rule (weights
$1, 4, 2, 4, \dots, 4, 1$ times one third of the spacing $h/2$), and the orbital
is divided by its square root. The cell draws $a(y)$ and $b(y)$, checks the
brane condition of each ($b(0) = 0$ even, $a(0) = 0$ odd), and measures how far
the computed zero mode is from its exact form
$a = \sqrt{2M/(1 - e^{-2ML})}\, e^{M y}$, $b = 0$ (the normalised $e^{My}$): the
program's check `free_zero_mode_exact` quotes this distance.
```

In [10]:

```
def orbital(eps, M, L, G):
    """The normalised orbital on the fine grid of 2G + 1 points, made as the Rust
    program makes it.  Returns the arrays y, a, b."""
    h = L / G
    nf = 2 * G + 1  # nodes and step midpoints
    y = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))  # the fine grid, as in the program
    _, a_nodes, b_nodes, _ = shoot(eps, M, L, G, keep=True)[3]
    da = M * a_nodes - eps * b_nodes  # the slopes a' at the nodes (the equations)
    db = eps * a_nodes - M * b_nodes  # the slopes b'
    a, b = np.zeros(nf), np.zeros(nf)
    a[0::2], b[0::2] = a_nodes, b_nodes  # the nodes take the even places
    a[1::2] = 0.5 * (a_nodes[:-1] + a_nodes[1:]) + h / 8.0 * (da[:-1] - da[1:])
    b[1::2] = 0.5 * (b_nodes[:-1] + b_nodes[1:]) + h / 8.0 * (db[:-1] - db[1:])
    half = 0.5 * h  # the spacing of the fine grid
    weights = np.full(nf, 2.0 * half / 3.0)  # Simpson: 2 at the even inner places
    weights[1::2] = 4.0 * half / 3.0  # 4 at the odd places (the midpoints)
    weights[0] = weights[-1] = half / 3.0  # 1 at the two ends
    scale = 1.0 / math.sqrt(np.sum(weights * (a * a + b * b)))
    return y, a * scale, b * scale


fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.4), sharex=True)
residuals = []
for ax, (parity, label) in zip(axes.flat, [("even", 0), ("odd", 0), ("even", 1),
                                           ("odd", 2)]):
    eps = level_of[(parity, label)]
    y_fine, a_fine, b_fine = orbital(eps, M1, L3, 900)
    end = b_fine[-1] if parity == "even" else a_fine[-1]  # must vanish at y = 0
    residuals.append(abs(end))
    ax.plot(y_fine, a_fine, color=BLUE, lw=1.5, label="$a(y)$")
    ax.plot(y_fine, b_fine, color=ORANGE, lw=1.5, ls="--", label="$b(y)$")
    ax.axhline(0.0, color=BLACK, lw=0.6)
    ax.set_title(f"{parity}, $l = {label}$, $\\varepsilon = {eps:.4f}$", fontsize=10)
    ax.legend(fontsize=8, loc="upper left")
    if parity == "even" and label == 0:  # the zero mode against its exact form
        exact_a = math.sqrt(2 * M1 / (1 - math.exp(-2 * M1 * L3))) * np.exp(M1 * y_fine)
        zero_mode_distance = np.max(np.abs(a_fine - exact_a))
        zero_mode_b = np.max(np.abs(b_fine))
for ax in axes[1]:
```

```
    ax.set_xlabel("hidden coordinate $y$ (units $1/H$)")
for ax in axes[:, 0]:
    ax.set_ylabel("component (units $H^{1/2}$)")
save_figure(fig, "orbitals",
            "The normalised orbitals of four levels of the case $M = 1$, $L = 3$: "
            "the components $a(y)$ (solid blue) and $b(y)$ (dashed orange) against "
            "the hidden coordinate $y$ from the tip $-3$ to the brane $0$ "
            "(horizontal axes, units $1/H$; vertical axes in units of $H^{1/2}$, so "
            "that $\\int (a^2 + b^2) dy = 1$). Top left: the zero mode "
            "$a \\propto e^{y}$, $b = 0$, concentrated at the brane. Top right: odd "
            "label 0. Bottom: even label 1 and odd label 2. Every orbital has "
            "$b = 0$ at the tip; at the brane the even ones have $b = 0$ and the "
            "odd ones $a = 0$.")
report("largest brane residual |b(0)| or |a(0)|", f"{max(residuals):.1e}")
zero_detail = [c["detail"] for c in solver_report["checks"]
               if c["name"] == "free_zero_mode_exact"][0]
zero_quoted = float(re.findall(r"e\^\(My\) to ([0-9.]+e-[0-9]+)", zero_detail)[0])
report("zero mode: largest distance from the exact normalised form (here, record)",
       f"{zero_mode_distance:.2e}, {zero_quoted:.2e}")
check(max(residuals) < 1e-10, "every orbital meets its brane condition within 1e-10")
check(zero_mode_b == 0.0 and abs(zero_mode_distance / zero_quoted - 1) < 0.01,
      "the zero mode: b = 0 exactly, distance 1.35e-12 from sqrt(2M/(1 - e^(-2ML))) "
      "e^(My)", record=f"{SOLVER_REPORT}, check free_zero_mode_exact")
```

Out [10]:

```
Figure 02d.6 saved as Revision/textbook/figures/02d_6_orbitals.png
RESULT largest brane residual |b(0)| or |a(0)| = 8.6e-16
RESULT zero mode: largest distance from the exact normalised form (here, record) =
    1.35e-12, 1.35e-12
PASS every orbital meets its brane condition within 1e-10
PASS the zero mode: b = 0 exactly, distance 1.35e-12 from sqrt(2M/(1 - e^(-2ML))) e^(My)
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_zero_mode_exact
```

Text cell:

```
## 14. How the error depends on the step

The levels inherit the RK4 error. The next cell repeats the search for three
levels of the canonical case with $G = 25, 50, 100, 200, 400, 800$ steps and
draws the error against the step $h = L/G$ (log-log; the slope is the order).
```

In [11]:

```
G_LIST = [25, 50, 100, 200, 400, 800]
tracked = [("even", 1), ("odd", 2), ("even", 5)]
fig, ax = plt.subplots()
orders = []
for (parity, label), color, marker in zip(tracked, [BLUE, ORANGE, AQUA],
                                          ["o", "s", "^"]):
```

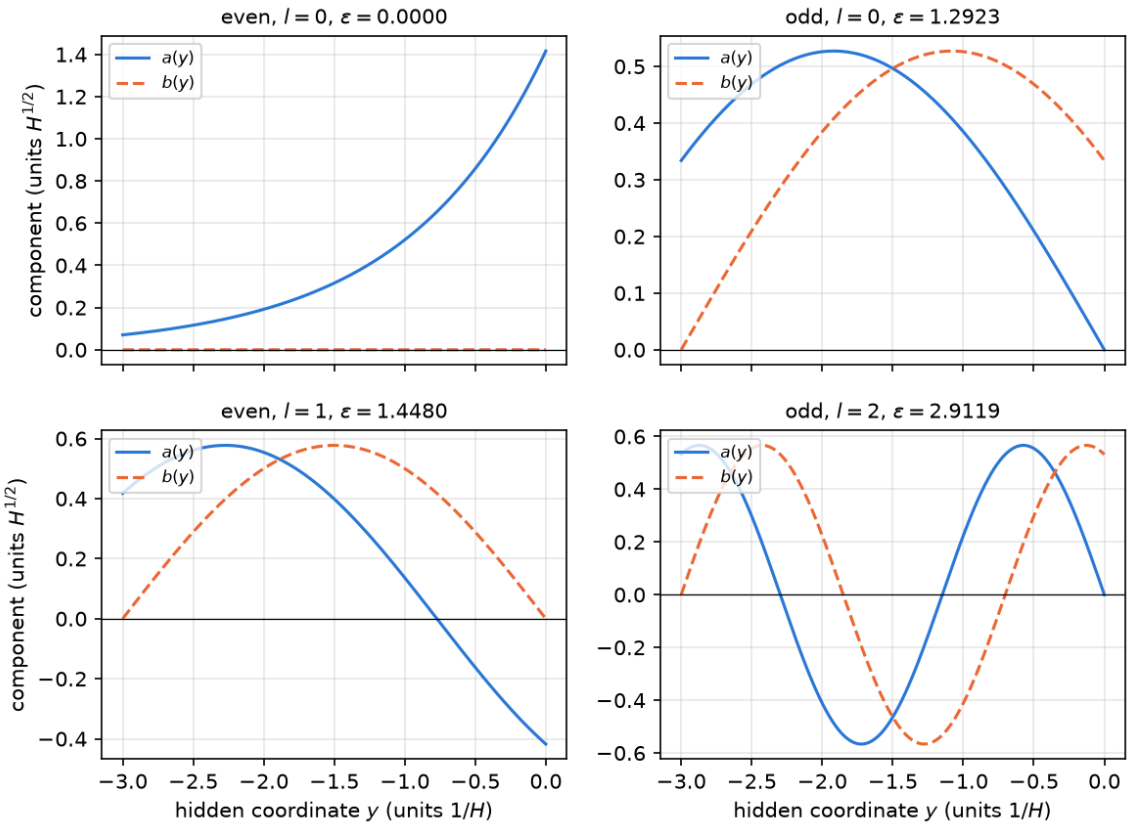

Figure 89: The normalised orbitals of four levels of the case $M = 1$, $L = 3$: the components $a(y)$ (solid blue) and $b(y)$ (dashed orange) against the hidden coordinate y from the tip -3 to the brane 0 (horizontal axes, units $1/H$; vertical axes in units of $H^{1/2}$, so that $\int (a^2 + b^2) dy = 1$). Top left: the zero mode $a \propto e^y$, $b = 0$, concentrated at the brane. Top right: odd label 0. Bottom: even label 1 and odd label 2. Every orbital has $b = 0$ at the tip; at the brane the even ones have $b = 0$ and the odd ones $a = 0$. (Notebook 02d, figure 6.)

```
    exact = exact_level(M1, L3, parity, label)
    errors = np.array([abs(find_level(M1, L3, G, target(parity, label)) - exact)
                       for G in G_LIST])
    h_list = L3 / np.array(G_LIST, dtype=float)
    fit = errors > 1e-11  # leave out points limited by the root tolerance
    orders.append(np.polyfit(np.log10(h_list[fit]), np.log10(errors[fit]), 1)[0])
    ax.loglog(h_list, errors, marker=marker, color=color, ms=5, lw=1.2,
              label=f"{parity} $l = {label}$ ($\\varepsilon = {exact:.4f}$)")
h_guide = L3 / np.array(G_LIST, dtype=float)
ax.loglog(h_guide, 2e-3 * (h_guide / h_guide[0]) ** 4, "--", color=GREY,
          label="slope 4")
ax.axvline(L3 / 900, color=BLACK, ls=":", lw=1.0)
ax.text(L3 / 900 * 1.08, 1e-4, "the record's\nstep 1/300", fontsize=8)
ax.set_xlabel("RK4 step $h = L/G$ (units $1/H$)")
ax.set_ylabel("error of the level (units $m$)")
ax.set_title("The levels converge like $h^4$")
ax.legend(loc="lower right", fontsize=8)
save_figure(fig, "level_convergence",
            "The error of three levels of the case $M = 1$, $L = 3$ found by "
            "shooting, against the RK4 step $h = L/G$ for $G = 25$ to 800 steps "
            "(logarithmic axes; the error in units of $m$, the step in units of "
            "$1/H$): even label 1 (circles), odd label 2 (squares), even label 5 "
            "(triangles). The points follow lines of slope 4 (dashed guide): the "
            "levels have the order of RK4, and higher levels have larger errors. "
            "The dotted line marks the record's step $1/300$.")
report("measured orders", ", ".join(f"{q:.2f}" for q in orders))
check(all(3.8 < q < 4.2 for q in orders), "the level errors fall like h^4")
```

Out [11]:

```
Figure 02d.7 saved as Revision/textbook/figures/02d_7_level_convergence.png
RESULT measured orders = 4.00, 3.99, 3.97
PASS the level errors fall like h^4
```

Text cell:

```
**Why the error grows with the level.** Far from the mass, $\theta' \approx
\varepsilon$: the point $(a, b)$ turns at the rate $\varepsilon$ (forwards for
$\varepsilon > 0$, backwards for $\varepsilon < 0$). One RK4 step multiplies a
turning point by $R(i\omega)$ with $\omega = \varepsilon h$ (the amplification
factor of the test equation $y' = i\varepsilon y$), whose angle is a little
SMALLER IN SIZE than the exact $\omega$. The next cell lets sympy expand the
angle of $R(i\omega)$: it is $\omega - \omega^5/120 + \dots$ (for $\omega < 0$
too, because the angle of $R(-i|\omega|)$ is minus that of $R(i|\omega|)$).
After $G = L/h$ steps the computed end angle differs by about
$-L\, \varepsilon^5 h^4/120$ from the exact one: it is too small for
$\varepsilon > 0$ and too large for $\varepsilon < 0$. Since $\Phi$ grows by
about $L$ per unit of energy, the computed level must move AWAY FROM ZERO by

$$\delta\varepsilon \approx \frac{\varepsilon^5 h^4}{120}, \qquad
\frac{\delta\varepsilon}{\varepsilon} \approx \frac{(h\varepsilon)^4}{120}$$
```

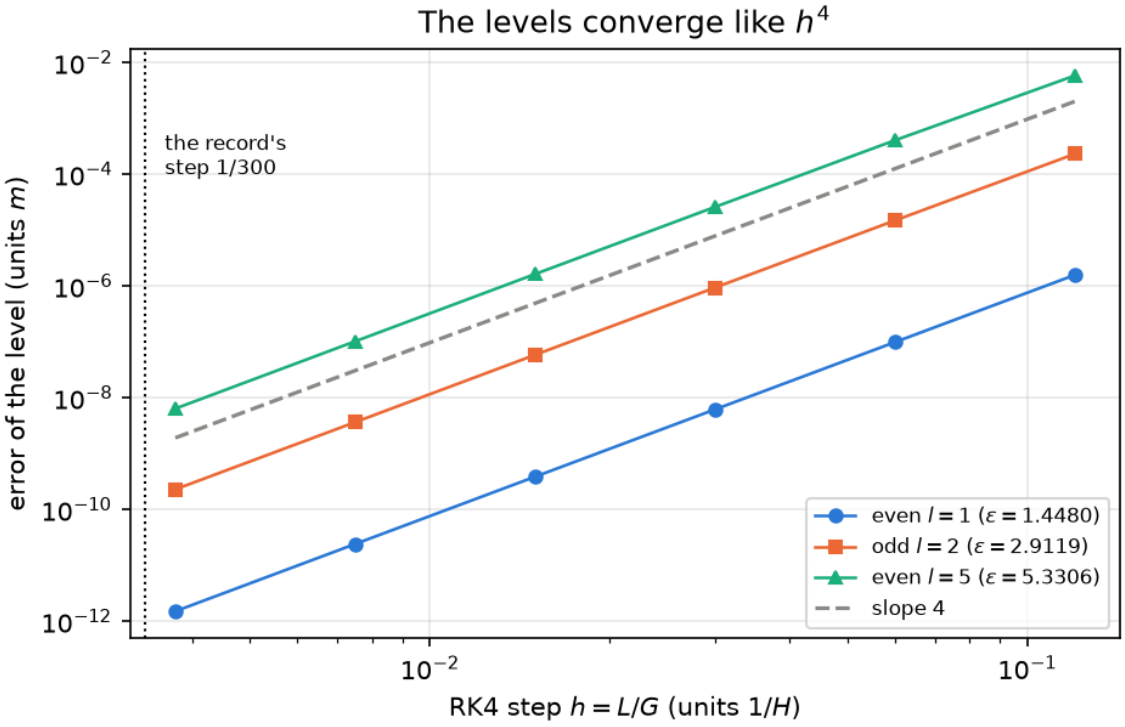

Figure 90: The error of three levels of the case $M = 1$, $L = 3$ found by shooting, against the RK4 step $h = L/G$ for $G = 25$ to 800 steps (logarithmic axes; the error in units of m, the step in units of $1/H$): even label 1 (circles), odd label 2 (squares), even label 5 (triangles). The points follow lines of slope 4 (dashed guide): the levels have the order of RK4, and higher levels have larger errors. The dotted line marks the record's step 1/300. (Notebook 02d, figure 7.)

```
to make up for it ($\varepsilon^5$ has the sign of $\varepsilon$): positive
levels come out too high, negative levels too low, both too large in size.
The cell checks this sign in the record's column 'difference'. The figure draws
the relative error $|\delta\varepsilon/\varepsilon|$ of all 51 nonzero levels
of the record against $h|\varepsilon|$ with $h = 1/300$, with our own
differences on top, and this prediction as a line. Close to the mass the point
does not turn uniformly ($\theta' = \varepsilon - M \sin 2\theta$), and the
error is smaller than the prediction; far from it the points approach the line.
```

In [12]:

```
w = sp.symbols("omega", positive=True)  # the angle turned in one step, eps h
R_rk4 = 1 + sp.I * w - w ** 2 / 2 - sp.I * w ** 3 / 6 + w ** 4 / 24  # R(i omega)
turned = sp.atan(sp.im(R_rk4) / sp.re(R_rk4))  # the angle of R(i omega)
lag = sp.series(turned - w, w, 0, 7).removeO()
say(f"angle of R(i omega) - omega = {lag} + ...")
check(sp.simplify(lag + w ** 5 / 120) == 0,
      "one RK4 step turns by omega - omega^5/120: the phase lags")

nonzero = [i for i, a in enumerate(analytic_record) if a != 0.0]
eps_abs = np.array([abs(analytic_record[i]) for i in nonzero])
x_values = eps_abs / 300.0  # h |eps| with h = 1/300
record_diff = np.array([abs(float(ROWS[i]["difference"])) for i in nonzero])
our_diff = np.array([abs(differences_ours[i]) for i in nonzero])
masses = np.array([float(ROWS[i]["m"]) for i in nonzero])
fig, ax = plt.subplots()
for mass, color in ((1.0, BLUE), (2.0, ORANGE)):
    pick = masses == mass
    ax.loglog(x_values[pick], record_diff[pick] / eps_abs[pick], "o", color=color,
              ms=6, label=f"record, $M = {mass:.0f}$")
ax.loglog(x_values, our_diff / eps_abs, "x", color=BLACK, ms=5,
          label="this notebook")
x_guide = np.linspace(x_values.min(), x_values.max(), 50)
ax.loglog(x_guide, x_guide ** 4 / 120, "--", color=GREY,
          label="prediction $(h\\varepsilon)^4/120$")
ax.set_xlabel("$h |\\varepsilon|$ with $h = 1/300$ (pure number)")
ax.set_ylabel("relative error $\\delta\\varepsilon / |\\varepsilon|$")
ax.set_title("The RK4 error of all 51 nonzero levels of the record")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "record_differences",
            "The relative error $\\delta\\varepsilon/|\\varepsilon|$ of the "
            "shooting levels for all 51 nonzero levels of the Revision record of "
            "the free spectrum (vertical axis, a pure number), against "
            "$h|\\varepsilon|$ with the record's step $h = 1/300$ (horizontal "
            "axis, a pure number), on logarithmic axes. Dots: the record's column "
            "difference divided by the level, for $M = 1$ (blue; $L = 3$ and "
            "$L = 2$) and $M = 2$ (orange); crosses: this notebook's shooting, "
            "which falls on the record's points. Dashed: the prediction "
            "$(h\\varepsilon)^4/120$ from the phase lag of one RK4 step; the "
            "levels far from the mass approach it, those near the mass lie "
            "below it.")
ratio = (record_diff / eps_abs) / (x_values ** 4 / 120)
report("error / prediction for the levels above 7 m",
```

```
       ", ".join(f"{q:.3f}" for q in ratio[eps_abs > 7]))
check(np.all(ratio < 1) and np.all(ratio[eps_abs > 7] > 0.9),
      "every level error lies below the prediction, within 10 % above 7 m")
report("largest |difference (here) - difference (record)|",
       f"{np.max(np.abs(record_diff - our_diff)):.1e}")
check(np.max(np.abs(record_diff - our_diff)) < 1e-12,
      "our differences equal the record's column difference within 1e-12",
      record=f"{TABLE}, column difference")
signed = np.array([float(ROWS[i]["difference"]) for i in nonzero])  # with sign
levels = np.array([analytic_record[i] for i in nonzero])
report("computed too high (levels > 0), too low (levels < 0)",
       f"{np.sum((levels > 0) & (signed > 0))} of {np.sum(levels > 0)}, "
       f"{np.sum((levels < 0) & (signed < 0))} of {np.sum(levels < 0)}")
check(np.all(np.sign(signed) == np.sign(levels)),
      "every computed level lies farther from 0 than the exact one (sign of eps^5)",
      record=f"{TABLE}, columns difference and eps_analytic")
```

Out [12]:

```
angle of R(i omega) - omega = -omega**5/120 + ...
PASS one RK4 step turns by omega - omega^5/120: the phase lags
```

Out [12]:

```
Figure 02d.8 saved as Revision/textbook/figures/02d_8_record_differences.png
RESULT error / prediction for the levels above 7 m = 0.934, 0.953, 0.955
PASS every level error lies below the prediction, within 10 % above 7 m
RESULT largest |difference (here) - difference (record)| = 1.8e-15
PASS our differences equal the record's column difference within 1e-12
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, column
    difference
RESULT computed too high (levels > 0), too low (levels < 0) = 33 of 33, 18 of 18
PASS every computed level lies farther from 0 than the exact one (sign of eps^5)
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, columns
    difference and eps_analytic
```

Text cell:

```
## 15. The last check

The last cell checks that all 8 figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [13]:

```
names = ["pruefer_staircase", "staircase_slope", "angle_along_y",
         "odd_level_equation", "newton_vs_bisection", "orbitals",
         "level_convergence", "record_differences"]
present = [output_file(f"{FIGURE_FOLDER}/02d_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02d exist")
all_checks_passed()
```

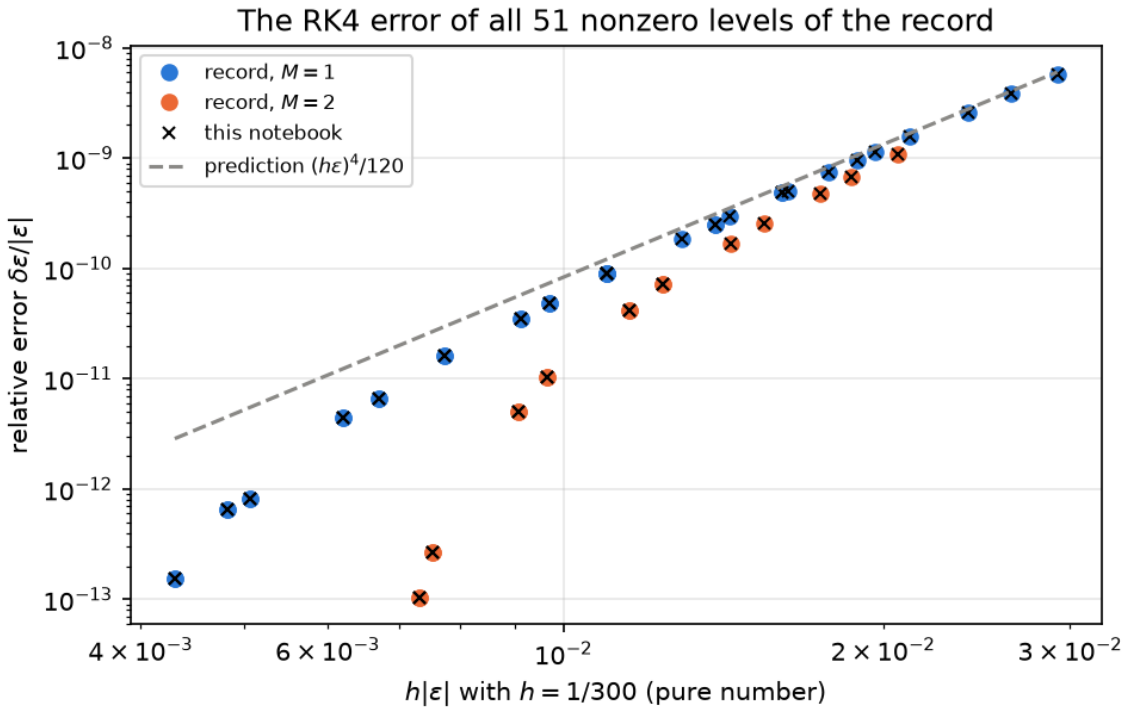

Figure 91: The relative error $\delta\varepsilon/|\varepsilon|$ of the shooting levels for all 51 nonzero levels of the Revision record of the free spectrum (vertical axis, a pure number), against $h|\varepsilon|$ with the record's step $h = 1/300$ (horizontal axis, a pure number), on logarithmic axes. Dots: the record's column difference divided by the level, for $M = 1$ (blue; $L = 3$ and $L = 2$) and $M = 2$ (orange); crosses: this notebook's shooting, which falls on the record's points. Dashed: the prediction $(h\varepsilon)^4/120$ from the phase lag of one RK4 step; the levels far from the mass approach it, those near the mass lie below it. (Notebook 02d, figure 8.)

Out [13]:

```
PASS all 8 figure files of notebook 02d exist
ALL 19 CHECKS PASSED (notebook 02d)
```

Text cell:

```
## 16. What this notebook showed

- The free Kohn-Sham block of the book, $a' = M a - \varepsilon b$,
  $b' = \varepsilon a - M b$ with $b(-L) = 0$ and $b(0) = 0$ or $a(0) = 0$, has
  the exact levels $0$ (the zero mode $a \propto e^{My}$, $b = 0$),
  $\pm\sqrt{M^2 + (n\pi/L)^2}$ (even) and $\pm\sqrt{M^2 + p^2}$ with
  $\tan(pL) = -p/M$ (odd).
- The Pruefer angle obeys $\theta' = \varepsilon - M\sin 2\theta$, and the
  product rule gives $d\Phi/d\varepsilon = \int r^2\,dy / r(0)^2 > 0$: the end
  value increases with $\varepsilon$, so each level is the unique crossing of a
  target $l\pi$ or $\pi/2 + l\pi$; no level can be missed and each has a label.
- The Rust program finds a level with Newton's method (slope from the same
  formula) and a bisection safety net: 9 shots for the even level $l = 3$,
  where plain bisection needs 48.
- Re-written line by line in Python, the program's shooting (900 RK4 steps for
  $L = 3$) and root finder reproduce all 54 levels of the record
  Revision/kohn_sham/results/spectrum/free-k0-analytic.csv (on the computer
  that made the record, identical in all 16 printed digits), the exact column,
  the zero mode exactly, the distance $1.35 \times 10^{-12}$ of the computed zero
  mode from its exact form, and the numbers $7.23 \times 10^{-10}$ (levels below
  $4m$) and $5.05 \times 10^{-8}$ (all levels from $4m$ up to $8.75m$; the
  check's text says "below $7m$") quoted by the program's check
  free_k0_analytic_spectra.
- The level errors fall like $h^4$; for levels far from the mass the relative
  error is close to $(h\varepsilon)^4/120$, the phase lag of RK4, and every
  computed level lies farther from zero than the exact one (too high above
  zero, too low below it). The canonical step $h = 1/300$ gives errors below
  $10^{-9}$ for $|\varepsilon| < 4m$.
- Inside the mass gap $|\varepsilon| \le M$ the only level is the zero mode.
- What this notebook does not show: the physics of these equations (where they
  come from, the meaning of the levels, the interacting case); the brane
  conditions rest on the ASSUMED mirror symmetry and the tip condition is a
  choice of the model, as the record states.
```

2.28 Line-by-line walk-through of Notebook 02d

The notebook has thirteen code cells, In [1] to In [13]. A quoted line `...)` stands for the rest of a figure caption, which Section 2.27 prints in full under its figure.

In [1], the set-up cell. Its comment lines are the run instructions of Section 2.26; its code is the set-up code of Notebook 02a explained in Section 2.11, with `NOTEBOOK_ID = "02d"`. It prints Set-up of notebook 02d complete: repository folder found, helpers defined.

In [2], the records and the settings.

```
import csv  # reads the table of levels (comma-separated values)
import math  # sqrt, atan2, pi for single numbers
import re  # finds numbers inside a text

import mpmath  # numbers with as many digits as we ask for
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

BLUE, ORANGE, AQUA, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7"
GREY, BLACK = "#8a8986", "#000000"
```

`csv` reads tables stored as comma-separated values (one row per line, the cells separated by commas), `re` finds patterns (here numbers) inside a text (**regular expressions**); the other packages and the colours are as before.

```
THEORY = "Revision/kohn_sham/ks-theory.json"
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
TABLE = "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
SOLVER_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
theory = json.loads(repository_file(THEORY).read_text(encoding="utf-8"))
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
with open(repository_file(TABLE), encoding="utf-8", newline="") as table:
    ROWS = list(csv.DictReader(table))  # one dictionary per level
```

The four Revision records of the notebook: the Kohn-Sham theory, the solver's parameters, the table of the free spectrum written by the Rust program, and the program's report of checks. The two JSON files are read as in Notebook 02a. `with open(...) as table:` opens the table file and closes it again at the end of the block; `csv.DictReader` reads it row by row, each row as a dictionary from the column names of the first line (`m`, `L`, `parity`, `label`, `eps_numeric`, `eps_analytic`, `difference`) to the cell texts.

```
say("realForm: " + theory["blockEquation"]["realForm"])
say("brane status: " + theory["boundaryConditions"]["brane"]["status"])
say("tip status: " + theory["boundaryConditions"]["tip"]["status"])
say("exact k = 0 spectra: " + theory["boundaryConditions"]["exactK0Spectra"])
```

Four statements of the theory record are printed: the real form of the block equation (with the general mass `M_eff`, momentum weight `kappa k` and potential `v_v` that Section 2.24 sets to M, 0 and 0), the status of the brane conditions (ASSUMED), the status of the tip condition (chosen, regular tip), and the exact $k = 0$ spectra derived in Section 2.24.

```
G_CANONICAL = parameters["numerics"]["rk4Steps"]  # RK4 steps for L = 3
ROOT_TOLERANCE = parameters["numerics"]["rootTolerance"]
M_RECORD, L_RECORD = parameters["physics"]["m"], parameters["physics"]["L_tipCutoff"]
report("RK4 steps, root tolerance, m, L of the record",
       f"{G_CANONICAL}, {ROOT_TOLERANCE}, {M_RECORD}, {L_RECORD}")
report("levels in the table", len(ROWS))
check(theory["boundaryConditions"]["brane"]["status"] == "ASSUMED"
      and G_CANONICAL == 900 and ROOT_TOLERANCE == 1e-13 and len(ROWS) == 54,
      "records read: brane ASSUMED, 900 RK4 steps, tolerance 1e-13, 54 levels")
```

The solver's settings: 900 RK4 steps, the root tolerance 10^{-13}, $m = 1$ and $L = 3$; the table has 54 rows. The check confirms these values and that the brane conditions are labelled ASSUMED in the record.

In [3], shooting with RK4 and counting the turns.

```
def shoot(eps, M, L, G, keep=False):
    """RK4 from y = -L, (a, b) = (1, 0), to y = 0 in G steps.  Returns Phi =
    theta(0), the slope dPhi/deps = (integral of r^2) / r(0)^2 and the largest
    turn of one step; with keep=True also the arrays y, a, b, theta of the path."""
    h = L / G  # the step
    a, b = 1.0, 0.0  # the tip: b(-L) = 0, so theta(-L) = 0
    theta, raw = 0.0, 0.0  # the counted angle and the last atan2 value
    integral, r2_before = 0.0, 1.0  # running integral of r^2; r^2 at the last node
    largest = 0.0  # the largest turn of one step so far
    path = [(-L, a, b, theta)]
```

The function follows the Rust function `shoot` of `Revision/kohn_sham/solver/src/shoot.rs` for the free case. The start at the tip is $(a, b) = (1, 0)$, so $\theta = 0$. `theta` is the counted angle and `raw` the last value returned by `atan2`; `integral` collects $\int r^2 dy$ and `r2_before` holds r^2 at the last node (1 at the start); `largest` records the largest turn of one step; `path` holds (y, a, b, θ) at the nodes when it is asked for.

```
    for i in range(G):
        p1a, p1b = M * a - eps * b, eps * a - M * b  # the slopes (a', b') at start
        a2, b2 = a + h / 2 * p1a, b + h / 2 * p1b  # half a step with them
        p2a, p2b = M * a2 - eps * b2, eps * a2 - M * b2  # the slopes there
        a3, b3 = a + h / 2 * p2a, b + h / 2 * p2b  # half a step with these
        p3a, p3b = M * a3 - eps * b3, eps * a3 - M * b3  # the slopes there
        a4, b4 = a + h * p3a, b + h * p3b  # a whole step with the third slopes
        p4a, p4b = M * a4 - eps * b4, eps * a4 - M * b4  # the slopes at the end
        a += h / 6 * (p1a + 2 * p2a + 2 * p3a + p4a)  # weights 1 : 2 : 2 : 1
        b += h / 6 * (p1b + 2 * p2b + 2 * p3b + p4b)
```

One RK4 step of the two equations $a' = Ma - \varepsilon b$, $b' = \varepsilon a - Mb$, written out component by component as the Rust code writes it: the four pairs of slopes `p1` to `p4` are the k_1 to k_4 of Section 2.4 for a and for b, and `a += ...` means "add the right side to `a`". (The names `a4`, `b4` here are the RK4 end-of-step values; they have nothing to do with the metric function a_4.)

```
        new = math.atan2(b, a)  # the angle of (a, b), between -pi and pi
        change = new - raw
        if change > math.pi:  # crossed from just below pi to just above -pi
            change -= 2 * math.pi
        elif change <= -math.pi:  # crossed the other way
            change += 2 * math.pi
        largest = max(largest, abs(change))
        theta += change
        raw = new
```

The winding count of Section 2.24: the new `atan2` angle minus the previous one, brought into $(-\pi, \pi]$ (a jump of almost $+2\pi$ means that the point crossed the negative horizontal axis downwards,

where `atan2` jumps from π to $-\pi$, and so on); the change is added to the counted angle, and the largest change is remembered.

```
        r2 = a * a + b * b  # r^2 at the new node
        integral += h / 2 * (r2_before + r2)  # the trapezoid rule for this step
        r2_before = r2
        if keep:
            path.append((-L + (i + 1) * h, a, b, theta))
```

The trapezoid rule adds the step length times the average of r^2 at the two ends of the step; the node is stored if the path was asked for.

```
    if keep:
        return theta, integral / r2_before, largest, np.array(path).T
    return theta, integral / r2_before, largest
```

After the last step `r2_before` is $r(0)^2$, so `integral / r2_before` is the slope $d\Phi/d\varepsilon$ of Section 2.24. The function returns $\Phi = \theta(0)$, that slope and the largest turn; with `keep=True` also the path, turned by `.T` (transpose) into four rows: all y, all a, all b, all θ. (The Rust function also rescales (a, b) when r^2 exceeds 10^{250}; in this free case r^2 stays far below that, so the rewrite leaves it out.)

```
def angle_equation(eps, M, L, G):
    """RK4 for the single equation theta' = eps - M sin(2 theta), theta(-L) = 0."""
    h = L / G
    theta = 0.0

    def slope(t):
        return eps - M * math.sin(2 * t)
    for _ in range(G):
        k1 = slope(theta)
        k2 = slope(theta + h / 2 * k1)
        k3 = slope(theta + h / 2 * k2)
        k4 = slope(theta + h * k3)
        theta += h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return theta
```

An independent way to the same end angle: RK4 applied to the single angle equation $\theta' = \varepsilon - M \sin 2\theta$ of Section 2.24 (the right side does not depend on y, so `slope` takes only the angle).

```
M1, L3 = 1.0, 3.0  # the canonical case of the record
for eps in (-2.0, 0.0, 1.0, 2.5):
    Phi = shoot(eps, M1, L3, 900)[0]  # element 0 of the result: Phi
    say(f"eps = {eps:+.1f}: Phi by (a, b) = {Phi:+.12f},  "
        f"by the angle equation = {angle_equation(eps, M1, L3, 900):+.12f}")
```

For four energies the two computations are printed side by side, for example $\Phi(2.5) = 6.798217350015$ by (a, b) and 6.798217350074 by the angle equation.

```
differences = [abs(shoot(e, M1, L3, 900)[0] - angle_equation(e, M1, L3, 900))
               for e in np.linspace(-5.0, 5.0, 21)]
check(max(differences) < 1e-8 and shoot(0.0, M1, L3, 900)[0] == 0.0,
      "both ways of computing Phi agree within 1e-8; Phi(0) is exactly 0")
```

On 21 energies from -5 to 5 the two ways agree within 10^{-8} (they are different computations, each with its own small RK4 error), and at $\varepsilon = 0$ the shot gives exactly $\Phi = 0$, because b stays exactly 0.

In [4], the staircase.

```
eps_grid = np.linspace(-6.0, 7.0, 651)
shots = [shoot(e, M1, L3, 900) for e in eps_grid]  # (Phi, slope, largest turn)
Phi_grid = np.array([s[0] for s in shots])
slope_grid = np.array([s[1] for s in shots])  # dPhi/deps by the formula
check(np.all(np.diff(Phi_grid) > 0), "Phi(eps) increases on the whole grid")
```

651 energies from -6 to 7 (spacing 0.02) are shot; `Phi_grid` holds the end angles and `slope_grid` the slopes by the formula. `np.diff` gives the differences of neighbouring entries; the check requires all of them to be positive: Φ increases, as proved in Section 2.24.

```
gap_ends = []  # Phi at eps = -M and eps = +M for the three cases of the table
for M, L in sorted({(float(r["m"]), float(r["L"])) for r in ROWS}):
    G = round(G_CANONICAL * L / 3.0)  # the record's step h = 1/300
    gap_ends += [shoot(-M, M, L, G)[0], shoot(M, M, L, G)[0]]
report("largest |Phi(-M)|, |Phi(+M)| in units of pi (three cases)",
       f"{max(abs(p) for p in gap_ends) / math.pi:.4f}")
check(max(abs(p) for p in gap_ends) < math.pi / 2,
      "for |eps| <= M the staircase crosses no target but 0: only the zero mode")
```

These lines test the statement of Section 2.24 that between $-M$ and M the only level is the zero mode. The curly brackets `{...}` make a **set**, a collection in which each element appears once: the pairs (M, L) of all 54 rows give the three cases $(1, 2)$, $(1, 3)$ and $(2, 3)$, and `sorted` puts them in a fixed order. For each case `G` is the record's number of steps for that L (900 for $L = 3$, 600 for $L = 2$), and the end angle is computed at the edges of the mass gap, $\varepsilon = -M$ and $\varepsilon = +M$ (`+=` appends both to the list). Since Φ increases with ε, every $\Phi(\varepsilon)$ with $|\varepsilon| \le M$ lies between $\Phi(-M)$ and $\Phi(+M)$. The RESULT line prints the largest of the six sizes, 0.2256π (COMPUTED, for $M = 2$); the check requires all six to be below $\pi/2$. Then for $|\varepsilon| \le M$ the end angle stays strictly between $-\pi/2$ and $\pi/2$, and the only target in that range is 0 (the even target with $l = 0$): no level except the zero mode lies in the gap, in all three cases.

```
def target(parity, label):
    """The target value of Phi for a level of the given parity and label."""
    return label * math.pi if parity == "even" else math.pi / 2 + label * math.pi


canonical = [r for r in ROWS if float(r["m"]) == 1.0 and float(r["L"]) == 3.0]
```

`target` returns $l\pi$ for an even and $\pi/2 + l\pi$ for an odd level (the program's function of the same name). `canonical` keeps the 18 rows of the table with $M = 1$ and $L = 3$ (the cells are texts, so `float` turns them into numbers).

```
fig, ax = plt.subplots(figsize=(7.0, 5.4))
ax.plot(eps_grid, Phi_grid / math.pi, color=BLACK, lw=1.4,
        label="$\\Phi(\\varepsilon)/\\pi$ (RK4, 900 steps)")
for label in range(-4, 7):
    ax.axhline(label, color=BLUE, lw=0.6, alpha=0.6)
    ax.axhline(label + 0.5, color=ORANGE, lw=0.6, ls="--", alpha=0.6)
```

The staircase Φ/π in black, and horizontal target lines: blue at the whole numbers (even targets $l\pi$), dashed orange halfway between them (odd targets).

```
for r in canonical:
    is_even = r["parity"] == "even"
    ax.plot(float(r["eps_numeric"]), target(r["parity"], int(r["label"])) / math.pi,
            "o" if is_even else "s", color=BLUE if is_even else ORANGE, ms=6)
ax.plot([], [], "o", color=BLUE, label="even levels: $\\Phi = l\\pi$")
ax.plot([], [], "s", color=ORANGE, label="odd levels: $\\Phi = \\pi/2 + l\\pi$")
```

Each of the record's 18 levels is marked at its energy and its target (a blue dot for even, an orange square for odd); two empty plots add the legend entries.

```
ax.set_ylim(-4.2, 6.2)
ax.set_xlabel("energy $\\varepsilon$ (units $m$)")
ax.set_ylabel("end angle $\\Phi = \\theta(0)$ in units of $\\pi$")
ax.set_title("The Pruefer staircase: $M = 1$, $L = 3$")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "pruefer_staircase",
            "The end value of the Pruefer angle, $\\Phi(\\varepsilon) = "
            ...)
```

Labels and the saved figure `02d_1_pruefer_staircase.png`. **What Figure 02d.1 shows:** the curve rises steadily and crosses every target line exactly once, at a level of the record; even and odd levels alternate; the zero mode sits at $\varepsilon = 0$, $\Phi = 0$; between -1 and 1 (inside the mass gap, $|\varepsilon| < M$) the curve rises only slowly (In [5] finds slopes between 0.5 and 0.7 there) and crosses only the line 0, in agreement with Section 2.24.

In [5], the slope of the staircase.

```
DELTA = 1e-4  # the half-width of the central difference
tested = [-4.0, -1.34, -0.5, 0.0, 0.7, 1.3, 3.0]  # seven energies
relative = []
for e in tested:
    formula = shoot(e, M1, L3, 900)[1]  # element 1 of the result: the slope
    central = (shoot(e + DELTA, M1, L3, 900)[0]
               - shoot(e - DELTA, M1, L3, 900)[0]) / (2 * DELTA)
    relative.append(abs(central / formula - 1))
    say(f"eps = {e:+.2f}: formula {formula:9.5f}, central difference {central:9.5f}")
```

At seven energies the slope formula $\int r^2 dy/r(0)^2$ is compared with the central difference $(\Phi(\varepsilon+\delta) - \Phi(\varepsilon-\delta))/(2\delta)$ of Section 2.18 with $\delta = 10^{-4}$; the relative differences are collected. The printout shows, for example, 3.85717 at $\varepsilon = -4$, 12.49831 at -1.34 and 0.49876 at 0.

```
report("largest relative difference formula - central difference",
       f"{max(relative):.1e}")
check(max(relative) < 1e-4 and np.all(slope_grid > 0),
      "dPhi/deps = (integral of r^2)/r(0)^2 > 0 (agrees with central differences)")
```

The largest relative difference is 3.7×10^{-6} (COMPUTED); the check requires less than 10^{-4}, and a positive slope at all 651 energies of the grid.

```
# At eps = 0 the shot is exactly a = e^(M(y + L)), b = 0, so the integral can be
# done by hand: (e^(2ML) - 1)/(2M) divided by r(0)^2 = e^(2ML).
at_zero = (1 - math.exp(-2 * M1 * L3)) / (2 * M1)
report("slope at eps = 0: formula, exact (1 - e^(-2ML))/(2M)",
       f"{shoot(0.0, M1, L3, 900)[1]:.6f}, {at_zero:.6f}")
check(abs(shoot(0.0, M1, L3, 900)[1] - at_zero) < 1e-4,
      "at eps = 0 the slope is (1 - e^(-2ML))/(2M), about 1/(2M)")
```

The slope at $\varepsilon = 0$ computed by hand in Section 2.24, $(1 - e^{-6})/2 = 0.498761$, against the program's 0.498762 (the difference is the error of the trapezoid rule).

```
grid_difference = (Phi_grid[2:] - Phi_grid[:-2]) / (eps_grid[2:] - eps_grid[:-2])
fig, ax = plt.subplots()
ax.plot(eps_grid, slope_grid, color=BLACK, lw=1.4,
        label="formula $\\int r^2 dy / r(0)^2$")
ax.plot(eps_grid[1:-1][::6], grid_difference[::6], "o", color=AQUA, ms=4,
        label="central differences of the staircase")
ax.axhline(L3, color=GREY, ls="--", lw=1.0, label="$L = 3$")
```

`grid_difference` is the central difference of the staircase on the grid itself (the value one place ahead minus the value one place back, divided by the energy distance 0.04 between them: entry i of `grid_difference` is $(\Phi_{i+2} - \Phi_i)/(\varepsilon_{i+2} - \varepsilon_i)$, centred on the grid point $i + 1$), belonging to the inner energies `eps_grid[1:-1]`; every sixth one is drawn (`[::6]`) as an aqua dot over the black formula curve, with a dashed line at $L = 3$.

```
ax.set_ylim(0.0, 17.5)  # room for the legend above the two peaks
ax.set_xlabel("energy $\\varepsilon$ (units $m$)")
ax.set_ylabel("$d\\Phi/d\\varepsilon$ (units $1/m$)")
ax.set_title("The slope of the staircase is never negative")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "staircase_slope",
            "The slope $d\\Phi/d\\varepsilon$ of the end angle (vertical axis, "
            ...)
```

Labels and the saved figure `02d_2_staircase_slope.png`. **What Figure 02d.2 shows:** the dots lie on the formula curve; the slope is positive everywhere; it is about $L = 3$ far from the mass, about $1/(2M) = 0.5$ near $\varepsilon = 0$ (the zero-mode shot grows towards the brane, so $r(0)^2$ is large), and it has two peaks near $|\varepsilon| = 1.34$ (there the shot ends with a small $r(0)$ compared with its size inside, so a small change of energy turns the end angle a lot).

In [6], the angle along the interval.

```
chosen = [("even", 0), ("odd", 0), ("even", 1), ("odd", 1), ("even", 3)]
level_of = {(r["parity"], int(r["label"])): float(r["eps_numeric"]) for r in canonical}
fig, ax = plt.subplots()
```

Five levels of the canonical case, named by parity and label; `level_of` maps each (parity, label) of the canonical rows to the record's numerical level.

```
for (parity, label), color in zip(chosen, [BLACK, ORANGE, BLUE, VIOLET, AQUA]):
    eps = level_of[(parity, label)]
    y_path, a_path, b_path, theta_path = shoot(eps, M1, L3, 900, keep=True)[3]
    ax.plot(y_path, theta_path / math.pi, color=color, lw=1.5,
            ls="-" if parity == "even" else "--",
            label=f"{parity} $l = {label}$, $\\varepsilon = {eps:.4f}$")
    ax.plot([0.0], [target(parity, label) / math.pi], "o", color=color, ms=6)
```

For each chosen level the shot is repeated with `keep=True`; element 3 of the result is the path, unpacked into its four rows. θ/π is drawn against y (solid for even, dashed for odd), and a dot marks the target at the brane.

```
ax.set_xlabel("hidden coordinate $y$ (units $1/H$): tip at $-3$, brane at $0$")
ax.set_ylabel("Pruefer angle $\\theta(y)/\\pi$")
ax.set_title("The angle winds up to its target at the brane")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "angle_along_y",
            "The Pruefer angle $\\theta(y)$ in units of $\\pi$ (vertical axis) "
            ...)
```

Labels and the saved figure `02d_3_angle_along_y.png`. **What Figure 02d.3 shows:** every curve starts at 0 at the tip and ends exactly on its dot; the zero mode stays at 0; the number of half-turns made is the label; higher levels wind faster, at the average rate ε.

In [7], the exact levels, computed again.

```
mpmath.mp.dps = 30


def odd_root(M, L, n):
    """The n-th positive root p of M sin(pL) + p cos(pL) = 0 (30 digits)."""
    g = lambda p: M * mpmath.sin(p * L) + p * mpmath.cos(p * L)  # noqa: E731
    return mpmath.findroot(g, ((n + 0.5) * mpmath.pi / L, (n + 1) * mpmath.pi / L),
                           solver="anderson")
```

mpmath at 30 digits. The odd condition $\tan(pL) = -p/M$ multiplied by $M\cos(pL)$ is $M\sin(pL) + p\cos(pL) = 0$ (no infinities). The n-th root lies between $(n + \frac{1}{2})\pi/L$ and $(n+1)\pi/L$ (Section 2.24), and `findroot` refines it in that bracket. (The comment `noqa: E731` tells a style checker that the one-line `lambda` function is intended.)

```
def exact_level(M, L, parity, label):
    """The exact level of the given parity and label."""
```

```
    if parity == "even":
        if label == 0:
            return 0.0  # the zero mode
        return math.copysign(float(mpmath.sqrt(M ** 2 + (label * mpmath.pi / L) ** 2)),
                             label)
    n = label if label >= 0 else -label - 1
    value = float(mpmath.sqrt(M ** 2 + odd_root(M, L, n) ** 2))
    return value if label >= 0 else -value
```

The exact level for a parity and a label, as the record defines them: even label 0 is the zero mode; even label $l \neq 0$ is $\pm\sqrt{M^2 + (l\pi/L)^2}$ with the sign of l (`math.copysign(x, s)` is $|x|$ with the sign of s); the odd label $l \geq 0$ uses the root p_l and gives $+\sqrt{M^2 + p_l^2}$, the odd label $l \leq -1$ uses the root p_{-l-1} and gives the negative level.

```
analytic_ours = [exact_level(float(r["m"]), float(r["L"]), r["parity"],
                             int(r["label"])) for r in ROWS]
analytic_record = [float(r["eps_analytic"]) for r in ROWS]
worst_analytic = max(abs(a - b) for a, b in zip(analytic_ours, analytic_record))
report("largest |exact level (here) - eps_analytic (record)|", f"{worst_analytic:.1e}")
check(worst_analytic < 1e-14, "all 54 exact levels equal the column eps_analytic",
      record=f"{TABLE}, column eps_analytic")
```

All 54 exact levels are computed here and compared with the record's column `eps_analytic`: the largest difference is 1.8×10^{-15}, the size of the last printed digit (COMPUTED: reproduces `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv`, column `eps_analytic`).

```
p_axis = np.linspace(0.01, 3.3, 3000)
tangent = np.tan(p_axis * L3)
tangent[np.abs(tangent) > 8] = np.nan  # do not draw the jumps at the poles
fig, ax = plt.subplots()
ax.plot(p_axis, tangent, color=BLUE, lw=1.4, label="$\\tan(pL)$, $L = 3$")
ax.plot(p_axis, -p_axis / M1, color=ORANGE, lw=1.4, ls="--", label="$-p/M$, $M = 1$")
```

The graphical solution of the odd condition for $M = 1$, $L = 3$: the branches of $\tan(pL)$ (with the jumps removed as in Notebook 02b) and the falling line $-p/M$.

```
for n in range(3):
    p_n = float(odd_root(M1, L3, n))
    ax.plot([p_n], [-p_n], "o", color=BLACK, ms=6)
    ax.annotate(f"$p_{n} = {p_n:.4f}$", (p_n, -p_n), textcoords="offset points",
                xytext=(6, -14), fontsize=8)
ax.axhline(0.0, color=BLACK, lw=0.6)
ax.set_ylim(-6.0, 6.0)
ax.set_xlabel("$p$ (units $m$)")
ax.set_ylabel("value of each side")
ax.set_title("Odd levels: $\\tan(pL) = -p/M$, then $\\varepsilon = \\sqrt{M^2 + p^2}$")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "odd_level_equation",
            "The equation of the odd levels for $M = 1$ and $L = 3$: the branches "
            ...)
```

The first three roots are marked and labelled on the line; labels and the saved figure `02d_4_odd_level_equation.png`. **What Figure 02d.4 shows:** one crossing on every branch of the tangent, each a root p_n; the first three give the odd levels 1.2923, 2.0106 and 2.9119.

In [8], Newton's method with a safety net.

```
def find_level(M, L, G, goal, tolerance=ROOT_TOLERANCE, log=None):
    """The energy with Phi(eps) = goal, found as the Rust program finds it.  If log
    is a list, every energy that is shot is appended to it."""
    def mismatch(eps):
        Phi, slope, largest = shoot(eps, M, L, G)
        if largest >= 2.0:  # the program refuses such a shot
            raise ValueError(f"one step turns the angle by {largest:.2f}")
        if log is not None:
            log.append(eps)
        return Phi - goal, slope  # F(eps) and F'(eps)
```

`find_level` is the program's function of the same name, line for line (Section 2.24). The inner function `mismatch` shoots one energy, refuses a shot in which one step turns by 2 or more, records the energy if a list was given, and returns $F = \Phi - t$ and $F' = d\Phi/d\varepsilon$.

```
    e_c = 0.0  # step 1: the starting guess
    F_c, dF_c = mismatch(e_c)
    if F_c == 0.0:
        return e_c
```

Step 1: shoot at $\varepsilon = 0$; for the zero mode ($t = 0$) the mismatch is exactly 0 and the search ends at once with the energy exactly 0.

```
    if F_c < 0.0:  # step 2, upwards: the level lies above
        low = e_c
        step = min(max(1.2 * (-F_c / dF_c), 1e-4), 2.0)
        while True:
            e = low + step
            F, dF = mismatch(e)
            if F >= 0.0:  # the sign changed: the bracket is (low, e)
                high = e
                if abs(F) < abs(F_c):  # keep the better of the two guesses
                    e_c, F_c, dF_c = e, F, dF
                break
            low, e_c, F_c, dF_c = e, e, F, dF  # still below: move on
            step *= 2.0
```

Step 2 when $F(0) < 0$ (the level lies above, because Φ increases): the first step is 1.2 times Newton's estimate $-F/F'$, limited to between 10^{-4} and 2 (`max` then `min`). The `while True` loop repeats until `break`: it shoots one step further up; if the sign has changed the bracket is (`low`, `e`), and the better of the two end guesses becomes the current guess; otherwise the new point becomes the lower end and the current guess, and the step is doubled.

```
    else:  # step 2, downwards: the level lies below
        high = e_c
        step = min(max(1.2 * (F_c / dF_c), 1e-4), 2.0)
        while True:
            e = high - step
            F, dF = mismatch(e)
            if F <= 0.0:  # the sign changed: the bracket is (e, high)
                low = e
                if abs(F) < abs(F_c):
                    e_c, F_c, dF_c = e, F, dF
                break
            high, e_c, F_c, dF_c = e, e, F, dF
            step *= 2.0
    if F_c == 0.0:
        return e_c
```

The same walk downwards when $F(0) > 0$. If the walk happens to land exactly on the level, the search ends. (The Rust function can also be given a known lower bound for the downward bracket, and it gives up after 400 shots without a bracket; neither is needed in this notebook.)

```
    previous = math.inf  # |F| at the previous step
    for _ in range(300):  # step 3
        if high - low <= tolerance:  # step 4: the bracket is narrow enough
            break
        e_new = e_c - F_c / dF_c  # Newton's point
        bisect = not (low < e_new < high) or abs(F_c) > 0.5 * previous
        if bisect:
            e_new = 0.5 * (low + high)  # the safety net
        previous = abs(F_c)
```

Step 3, at most 300 times: stop if the bracket is narrower than the tolerance (step 4). Otherwise compute Newton's point; if it is not strictly inside the bracket, or if $|F|$ has not at least halved since the previous pass (`math.inf`, infinity, makes the first pass always pass this test), take the midpoint of the bracket instead. `previous` remembers $|F|$ for the next pass.

```
        F, dF = mismatch(e_new)
        moved = abs(e_new - e_c)
        e_c, F_c, dF_c = e_new, F, dF
        if F == 0.0:
            break
        if F < 0.0:  # keep the half of the bracket with the sign change
            low = e_new
        else:
            high = e_new
        if moved <= tolerance and not bisect:  # step 4: a tiny Newton step
            break
    return e_c
```

Shoot the new point; it becomes the current guess. An exact zero ends the search; otherwise the new point replaces the end of the bracket with the same sign of F. A Newton step shorter than the tolerance also ends the search (step 4). The function returns the current guess.

```
def find_level_by_bisection(M, L, G, goal, tolerance=ROOT_TOLERANCE, log=None):
    """The same level by plain bisection on the bracket (-12, 12)."""
    low, high = -12.0, 12.0
    while high - low > tolerance:
        middle = 0.5 * (low + high)
        if log is not None:
            log.append(middle)
        if shoot(middle, M, L, G)[0] < goal:
            low = middle
        else:
            high = middle
    return 0.5 * (low + high)
```

For comparison, plain bisection on the bracket $(-12, 12)$: because Φ increases, $\Phi(m) <$ target means that the level lies above the midpoint m.

```
newton_log, bisection_log = [], []
by_newton = find_level(M1, L3, 900, target("even", 3), log=newton_log)
by_bisection = find_level_by_bisection(M1, L3, 900, target("even", 3),
                                       log=bisection_log)
report("even label 3 by Newton", f"{by_newton:.15f} ({len(newton_log)} shots)")
report("even label 3 by bisection",
       f"{by_bisection:.15f} ({len(bisection_log)} shots)")
for k, e in enumerate(newton_log, 1):
    say(f"Newton shot {k}: eps = {e:.15f}, error {abs(e - by_newton):.1e}")
```

Both search the even level with label 3 (target 3π) of the case $M = 1$, $L = 3$, recording every energy they shoot. Newton finds 3.296908309775607 with 9 shots, bisection 3.296908309775617 with 48 shots (COMPUTED). The nine Newton shots are printed: 0 (step 1), 2 and 6 (the bracket walk: the first step is limited to 2, the doubled step 4 reaches 6, where F has changed sign), then 3.1476, 3.2796, 3.29660, 3.2969082, ... with the errors 1.5×10^{-1}, 1.7×10^{-2}, 3.1×10^{-4}, 1.0×10^{-7}, 1.3×10^{-14}, 0: from the fourth shot on, each error is roughly the square of the previous one times a constant, so the number of correct digits about doubles.

```
fig, ax = plt.subplots()
ax.semilogy(range(1, len(bisection_log) + 1),
            np.maximum(np.abs(np.array(bisection_log) - by_newton), 1e-17), "s-",
            color=ORANGE, ms=4, lw=1.2, label="bisection on $(-12, 12)$")
ax.semilogy(range(1, len(newton_log) + 1),
            np.maximum(np.abs(np.array(newton_log) - by_newton), 1e-17), "o-",
            color=BLUE, ms=5, lw=1.2, label="the program's Newton with safety net")
ax.set_ylim(1e-17, 30.0)
ax.set_xlabel("number of the shot")
ax.set_ylabel("$|\\varepsilon - \\varepsilon_3|$ (units $m$)")
ax.set_title("Finding the even level $l = 3$ ($M = 1$, $L = 3$)")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "newton_vs_bisection",
            "The distance $|\\varepsilon - \\varepsilon_3|$ (units of $m$, "
            ...)
```

The distance of every shot energy from the level, on a logarithmic axis (exact zeros drawn at 10^{-17}), for both methods; the saved figure is `02d_5_newton_vs_bisection.png`. **What Figure 02d.5**

shows: the bisection squares fall along a straight line (a factor 2 per shot) and need 48 shots; the Newton circles first jump around while the bracket is made, then plunge, and stop after 9 shots.

```
check(abs(by_newton - level_of[("even", 3)]) < 1e-12
      and abs(by_bisection - by_newton) < 1e-13
      and len(newton_log) < 15 < 45 < len(bisection_log),
      "Newton (fewer than 15 shots) gives the record's level; bisection agrees",
      record=f"{TABLE}, column eps_numeric")
```

The Newton result equals the record's level within 10^{-12}, bisection agrees within 10^{-13}, and the numbers of shots are fewer than 15 and more than 45 (COMPUTED: reproduces `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv`, column `eps_numeric`, for this level).

In [9], all 54 levels against the Rust program.

```
def rust_text(x):
    """x written as the Rust program writes numbers: 16 digits, exponent like e0."""
    mantissa, exponent = f"{x:.15e}".split("e")  # Python writes e+00, Rust e0
    return f"{mantissa}e{int(exponent)}"
```

`f"{x:.15e}"` writes x with 16 significant digits in scientific notation, for example `3.296908309775607e+00`; the program writes the exponent as `e0`. The function splits the text at the `e` and writes the exponent back as a whole number, which turns `+00` into `0` and `-10` into `-10`.

```
CASES = [(1.0, 3.0), (1.0, 2.0), (2.0, 3.0)]  # the three cases (M, L) of the table
numeric_ours, shots_per_level = [], []
for r in ROWS:
    M, L = float(r["m"]), float(r["L"])
    G = round(G_CANONICAL * L / 3.0)  # 900 steps for L = 3, 600 for L = 2
    log = []
    numeric_ours.append(find_level(M, L, G, target(r["parity"], int(r["label"])),
                                   log=log))
    shots_per_level.append(len(log))
```

`CASES` names the three cases of the table (a reminder; the loop takes M and L from each row). For every row the level is found with the program's number of steps, $G = 900L/3$, which keeps the step $h = 1/300$, and the number of shots is recorded.

```
numeric_record = [float(r["eps_numeric"]) for r in ROWS]
identical = sum(rust_text(x) == r["eps_numeric"] for x, r in zip(numeric_ours, ROWS))
worst_numeric = max(abs(a - b) for a, b in zip(numeric_ours, numeric_record))
report("levels whose 16 digits equal the column eps_numeric", f"{identical} of 54")
report("largest |level (here) - eps_numeric (Rust program)|", f"{worst_numeric:.1e}")
report("shots per level (fewest, most, all 54 together)",
       f"{min(shots_per_level)}, {max(shots_per_level)}, {sum(shots_per_level)}")
```

`identical` counts the levels whose text, written as the program writes it, equals the record's text (`sum` of `True`/`False` values counts the `True` ones): 54 of 54 on the computer that made the record. The largest numerical difference is 8.9×10^{-16}, and the searches needed between 1 and 15 shots, 510 in all (COMPUTED).

```
check(worst_numeric < 1e-12 and max(shots_per_level) <= 20,
      "all 54 shooting levels equal the column eps_numeric (at most 20 shots each)",
      record=f"{TABLE}, column eps_numeric")
zero_rows = [i for i, r in enumerate(ROWS) if r["parity"] == "even"
             and int(r["label"]) == 0]
check(all(numeric_ours[i] == 0.0 == numeric_record[i] for i in zero_rows),
      "the zero mode has the energy exactly 0 in all three cases",
      record=f"{SOLVER_REPORT}, check free_zero_mode_exact")
```

The first check allows 10^{-12} rather than requiring identical digits, because on another computer the system's `atan2` may round differently in the last binary place (COMPUTED: reproduces `Revision /kohn_sham/results/spectrum/free-k0-analytic.csv`, column `eps_numeric`). The second finds the three zero-mode rows and requires the energy exactly 0 both here and in the record (the chained comparison `a == 0.0 == b` means `a == 0.0` and `0.0 == b`; COMPUTED: reproduces `Revision /kohn_sham/reports/ks-rust-solver.json`, check `free_zero_mode_exact`).

```
differences_ours = [n - a for n, a in zip(numeric_ours, analytic_ours)]
pairs = list(zip(differences_ours, analytic_ours))
low_band = max(abs(d) for d, a in pairs if abs(a) < 4)  # all levels below 4 m
high_band = max(abs(d) for d, a in pairs if abs(a) >= 4)  # all levels from 4 m on
below_7 = max(abs(d) for d, a in pairs if 4 <= abs(a) < 7)
top = max(abs(a) for _, a in pairs)  # the highest level of the table
```

The numerical minus the exact level for every row, and the largest difference in three bands of $|\varepsilon|$: below $4m$, from $4m$ on, and from $4m$ to below $7m$; `top` is the highest level of the table.

```
solver_report = json.loads(repository_file(SOLVER_REPORT).read_text(encoding="utf-8"))
detail = [c["detail"] for c in solver_report["checks"]
          if c["name"] == "free_k0_analytic_spectra"][0]
quoted = float(re.findall(r"max \|difference\| ([0-9.]+e-[0-9]+) for \|eps\| < 4 m",
                          detail)[0])
quoted_high = float(re.findall(r"([0-9.]+e-[0-9]+) for 4 m <= \|eps\| < 7 m",
                               detail)[0])
```

The cell reads the text of the program's check `free_k0_analytic_spectra` and finds in it the two numbers it quotes. `re.findall(pattern, text)` returns every piece of the text that matches the pattern; in the pattern, `[0-9.]+` is one or more digits or points, `e-[0-9]+` an exponent, `\|` a literal vertical bar, and the round brackets mark the part to return. The two numbers are 7.23×10^{-10} (levels below $4m$) and 5.05×10^{-8} (the second band).

```
report("largest |numeric - exact| for |eps| < 4 m (here, record)",
       f"{low_band:.2e}, {quoted:.2e}")
report("largest |numeric - exact| for |eps| >= 4 m (here, record)",
       f"{high_band:.2e}, {quoted_high:.2e}")
say(f"Note: the record's check text names its second band 4 m <= |eps| < 7 m, but "
    f"the solver puts every level with |eps| >= 4 m into it, and the table's "
    f"levels reach {top:.4f} m; the quoted number belongs to that level. For "
    f"4 m <= |eps| < 7 m alone the largest difference is {below_7:.2e}.")
check(abs(low_band / quoted - 1) < 0.005 and abs(high_band / quoted_high - 1) < 0.005,
      "the two largest differences quoted by the solver check are reproduced",
      record=f"{SOLVER_REPORT}, check free_k0_analytic_spectra")
```

Both quoted numbers are reproduced: 7.23×10^{-10} and 5.05×10^{-8} (COMPUTED: reproduces `Revision/kohn_sham/reports/ks-rust-solver.json`, check `free_k0_analytic_spectra`). The printed note records a wording error in that record: its text calls the second band "$4m \le |\varepsilon| < 7m$", but the Rust code (file `Revision/kohn_sham/solver/src/spectrum.rs`) puts every level with $|\varepsilon| \ge 4m$ into it, and the table reaches $8.7539m$; the quoted 5.05×10^{-8} belongs to that highest level, while for $4m \le |\varepsilon| < 7m$ alone the largest difference is 9.95×10^{-9}. The check's verdict is unaffected, because its tolerance 3×10^{-7} covers both numbers.

In [10], the orbitals.

```
def orbital(eps, M, L, G):
    """The normalised orbital on the fine grid of 2G + 1 points, made as the Rust
    program makes it.  Returns the arrays y, a, b."""
    h = L / G
    nf = 2 * G + 1  # nodes and step midpoints
    y = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))  # the fine grid, as in the program
    _, a_nodes, b_nodes, _ = shoot(eps, M, L, G, keep=True)[3]
```

The program's function `profile` rewritten. The fine grid has $2G + 1$ points (the nodes and the step midpoints); its positions are computed exactly as the program computes them, $y_f = -L\,(n_f - 1 - f)/(n_f - 1)$, which runs from $-L$ to 0 (the same formula, so that the rounding is the same). The shot with `keep=True` gives a and b at the $G + 1$ nodes.

```
    da = M * a_nodes - eps * b_nodes  # the slopes a' at the nodes (the equations)
    db = eps * a_nodes - M * b_nodes  # the slopes b'
    a, b = np.zeros(nf), np.zeros(nf)
    a[0::2], b[0::2] = a_nodes, b_nodes  # the nodes take the even places
    a[1::2] = 0.5 * (a_nodes[:-1] + a_nodes[1:]) + h / 8.0 * (da[:-1] - da[1:])
    b[1::2] = 0.5 * (b_nodes[:-1] + b_nodes[1:]) + h / 8.0 * (db[:-1] - db[1:])
```

The slopes at the nodes come from the equations themselves. The node values fill the even places of the fine grid (`0::2`: every second place from 0), and the odd places (`1::2`) get the cubic Hermite midpoint values $(f_0 + f_1)/2 + h(f'_0 - f'_1)/8$ of Section 2.24, for all steps at once.

```
    half = 0.5 * h  # the spacing of the fine grid
    weights = np.full(nf, 2.0 * half / 3.0)  # Simpson: 2 at the even inner places
    weights[1::2] = 4.0 * half / 3.0  # 4 at the odd places (the midpoints)
    weights[0] = weights[-1] = half / 3.0  # 1 at the two ends
    scale = 1.0 / math.sqrt(np.sum(weights * (a * a + b * b)))
    return y, a * scale, b * scale
```

Simpson's weights for the spacing $h/2$: $\frac{h/2}{3}$ times 1 at the ends, 4 at the odd places and 2 at the inner even places (`np.full` makes an array filled with one value). The norm $\int (a^2 + b^2) dy$ is the weighted sum, and the orbital is divided by its square root, so that $\int (a^2 + b^2) dy = 1$.

```
fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.4), sharex=True)
residuals = []
for ax, (parity, label) in zip(axes.flat, [("even", 0), ("odd", 0), ("even", 1),
```

```
                                        ("odd", 2)]):
    eps = level_of[(parity, label)]
    y_fine, a_fine, b_fine = orbital(eps, M1, L3, 900)
    end = b_fine[-1] if parity == "even" else a_fine[-1]  # must vanish at y = 0
    residuals.append(abs(end))
```

A figure with 2×2 panels sharing the horizontal axis; `axes.flat` walks through the four panels. For four levels the orbital is built and the component that the brane condition must make zero ($b(0)$ for even, $a(0)$ for odd) is recorded.

```
    ax.plot(y_fine, a_fine, color=BLUE, lw=1.5, label="$a(y)$")
    ax.plot(y_fine, b_fine, color=ORANGE, lw=1.5, ls="--", label="$b(y)$")
    ax.axhline(0.0, color=BLACK, lw=0.6)
    ax.set_title(f"{parity}, $l = {label}$, $\\varepsilon = {eps:.4f}$", fontsize=10)
    ax.legend(fontsize=8, loc="upper left")
```

Each panel draws $a(y)$ (solid blue) and $b(y)$ (dashed orange) with the zero line and a title naming the level.

```
    if parity == "even" and label == 0:  # the zero mode against its exact form
        exact_a = math.sqrt(2 * M1 / (1 - math.exp(-2 * M1 * L3))) * np.exp(M1 * y_fine)
        zero_mode_distance = np.max(np.abs(a_fine - exact_a))
        zero_mode_b = np.max(np.abs(b_fine))
```

For the zero mode the exact normalised form is $a = \sqrt{2M/(1-e^{-2ML})}\, e^{My}$, $b = 0$ (the factor makes $\int_{-L}^{0} a^2 dy = 1$, because $\int_{-L}^{0} e^{2My} dy = (1-e^{-2ML})/(2M)$). The cell records the largest distance of the computed a from it and the largest $|b|$.

```
for ax in axes[1]:
    ax.set_xlabel("hidden coordinate $y$ (units $1/H$)")
for ax in axes[:, 0]:
    ax.set_ylabel("component (units $H^{1/2}$)")
save_figure(fig, "orbitals",
            "The normalised orbitals of four levels of the case $M = 1$, $L = 3$: "
            ...)
```

Axis labels on the bottom row (`axes[1]`) and the left column (`axes[:, 0]`); the components carry the unit $H^{1/2}$ because $\int (a^2 + b^2) dy = 1$ with y in units of $1/H$. The saved figure is `02d_6_orbitals.png`. **What Figure 02d.6 shows:** the zero mode ($a \propto e^{y}$, $b = 0$) is concentrated at the brane; every orbital has $b = 0$ at the tip $y = -3$; at the brane the even orbitals have $b = 0$ and the odd ones $a = 0$; higher levels oscillate more.

```
report("largest brane residual |b(0)| or |a(0)|", f"{max(residuals):.1e}")
zero_detail = [c["detail"] for c in solver_report["checks"]
               if c["name"] == "free_zero_mode_exact"][0]
zero_quoted = float(re.findall(r"e\^\(My\) to ([0-9.]+e-[0-9]+)", zero_detail)[0])
report("zero mode: largest distance from the exact normalised form (here, record)",
       f"{zero_mode_distance:.2e}, {zero_quoted:.2e}")
```

The largest brane residual is 8.6×10^{-16}. The text of the program's check `free_zero_mode_exact` quotes the distance of the computed zero mode from its exact form after the words `e^(My) to`; in the pattern, `\^`, `\(` and `\)` stand for the literal characters. Both distances are 1.35×10^{-12} (COMPUTED).

```
check(max(residuals) < 1e-10, "every orbital meets its brane condition within 1e-10")
check(zero_mode_b == 0.0 and abs(zero_mode_distance / zero_quoted - 1) < 0.01,
      "the zero mode: b = 0 exactly, distance 1.35e-12 from sqrt(2M/(1 - e^(-2ML))) "
      "e^(My)", record=f"{SOLVER_REPORT}, check free_zero_mode_exact")
```

Two checks: the brane conditions hold within 10^{-10}; the zero mode has b exactly 0 and reproduces the quoted distance within 1 per cent (COMPUTED: reproduces `Revision/kohn_sham/reports/ks-rust-solver.json`, check `free_zero_mode_exact`).

In [11], how the error depends on the step.

```
G_LIST = [25, 50, 100, 200, 400, 800]
tracked = [("even", 1), ("odd", 2), ("even", 5)]
fig, ax = plt.subplots()
orders = []
```

Six step counts and three levels to follow.

```
for (parity, label), color, marker in zip(tracked, [BLUE, ORANGE, AQUA],
                                          ["o", "s", "^"]):
    exact = exact_level(M1, L3, parity, label)
    errors = np.array([abs(find_level(M1, L3, G, target(parity, label)) - exact)
                       for G in G_LIST])
    h_list = L3 / np.array(G_LIST, dtype=float)
    fit = errors > 1e-11  # leave out points limited by the root tolerance
    orders.append(np.polyfit(np.log10(h_list[fit]), np.log10(errors[fit]), 1)[0])
    ax.loglog(h_list, errors, marker=marker, color=color, ms=5, lw=1.2,
              label=f"{parity} $l = {label}$ ($\\varepsilon = {exact:.4f}$)")
```

For each level the search is repeated with every step count and the error against the exact level is stored; a straight line is fitted through the errors above 10^{-11} (below that the root tolerance and rounding interfere), and its slope (element 0 of `polyfit`'s result) is the measured order. The errors are drawn on a log-log plot.

```
h_guide = L3 / np.array(G_LIST, dtype=float)
ax.loglog(h_guide, 2e-3 * (h_guide / h_guide[0]) ** 4, "--", color=GREY,
          label="slope 4")
ax.axvline(L3 / 900, color=BLACK, ls=":", lw=1.0)
ax.text(L3 / 900 * 1.08, 1e-4, "the record's\nstep 1/300", fontsize=8)
```

A guide of slope 4, and a dotted vertical line at the record's step $h = 3/900 = 1/300$ with a two-line label (`\n` starts a new line).

```
ax.set_xlabel("RK4 step $h = L/G$ (units $1/H$)")
ax.set_ylabel("error of the level (units $m$)")
ax.set_title("The levels converge like $h^4$")
ax.legend(loc="lower right", fontsize=8)
save_figure(fig, "level_convergence",
            "The error of three levels of the case $M = 1$, $L = 3$ found by "
            ...)
report("measured orders", ", ".join(f"{q:.2f}" for q in orders))
check(all(3.8 < q < 4.2 for q in orders), "the level errors fall like h^4")
```

Labels and the saved figure `02d_7_level_convergence.png`; the measured orders are 4.00, 3.99 and 3.97 (COMPUTED), and the check requires them between 3.8 and 4.2. **What Figure 02d.7 shows:** three parallel lines of slope 4; the higher the level, the larger its error at a given step; at the record's step 1/300 the errors are far below 10^{-6}.

In [12], why the error grows with the level.

```
w = sp.symbols("omega", positive=True)  # the angle turned in one step, eps h
R_rk4 = 1 + sp.I * w - w ** 2 / 2 - sp.I * w ** 3 / 6 + w ** 4 / 24  # R(i omega)
turned = sp.atan(sp.im(R_rk4) / sp.re(R_rk4))  # the angle of R(i omega)
lag = sp.series(turned - w, w, 0, 7).removeO()
say(f"angle of R(i omega) - omega = {lag} + ...")
check(sp.simplify(lag + w ** 5 / 120) == 0,
      "one RK4 step turns by omega - omega^5/120: the phase lags")
```

`R_rk4` is $R(i\omega)$ written out (Section 2.7). `sp.im` and `sp.re` take its imaginary and real parts, and the arctangent of their quotient is its angle (for small ω the real part is positive, so no turn is lost). `sp.series(..., w, 0, 7)` expands the angle minus ω up to ω^6; the printout is `-omega**5/120`, and the check confirms the phase lag $\omega^5/120$ derived in Section 2.24. The symbol is declared positive (`positive=True`), which keeps sympy's expansion simple; for a negative angle the same result follows, because $R(-i|\omega|)$ is the complex conjugate of $R(i|\omega|)$, so its angle is minus that of $R(i|\omega|)$, and $-(|\omega| - |\omega|^5/120) = \omega - \omega^5/120$ for $\omega = -|\omega|$.

```
nonzero = [i for i, a in enumerate(analytic_record) if a != 0.0]
eps_abs = np.array([abs(analytic_record[i]) for i in nonzero])
x_values = eps_abs / 300.0  # h |eps| with h = 1/300
record_diff = np.array([abs(float(ROWS[i]["difference"])) for i in nonzero])
our_diff = np.array([abs(differences_ours[i]) for i in nonzero])
masses = np.array([float(ROWS[i]["m"]) for i in nonzero])
```

The 51 rows with a nonzero level (all except the three zero modes): their sizes $|\varepsilon|$, the quantity $h|\varepsilon|$ with $h = 1/300$ (the same step for all three cases), the size (`abs`) of the record's column `difference` (numerical minus exact level), the sizes of our own differences from In [9], and the masses. The sizes are needed for the logarithmic axes; the signs are tested at the end of the cell.

```
fig, ax = plt.subplots()
for mass, color in ((1.0, BLUE), (2.0, ORANGE)):
    pick = masses == mass
    ax.loglog(x_values[pick], record_diff[pick] / eps_abs[pick], "o", color=color,
```

```
              ms=6, label=f"record, $M = {mass:.0f}$")
ax.loglog(x_values, our_diff / eps_abs, "x", color=BLACK, ms=5,
          label="this notebook")
x_guide = np.linspace(x_values.min(), x_values.max(), 50)
ax.loglog(x_guide, x_guide ** 4 / 120, "--", color=GREY,
          label="prediction $(h\\varepsilon)^4/120$")
```

The relative errors $\delta\varepsilon/|\varepsilon|$ of the record (dots, blue for $M = 1$ and orange for $M = 2$; `pick` selects the rows of one mass) and of this notebook (black crosses) against $h|\varepsilon|$ on log-log axes, with the prediction $(h\varepsilon)^4/120$ of Section 2.24 as a dashed line.

```
ax.set_xlabel("$h |\\varepsilon|$ with $h = 1/300$ (pure number)")
ax.set_ylabel("relative error $\\delta\\varepsilon / |\\varepsilon|$")
ax.set_title("The RK4 error of all 51 nonzero levels of the record")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "record_differences",
            "The relative error $\\delta\\varepsilon/|\\varepsilon|$ of the "
            ...)
```

Labels and the saved figure `02d_8_record_differences.png`. **What Figure 02d.8 shows:** the crosses sit on the record's dots (our shooting reproduces the program's errors); all points lie below the dashed prediction, and the levels far from the mass approach it, while those near the mass (where the point does not turn uniformly) lie further below.

```
ratio = (record_diff / eps_abs) / (x_values ** 4 / 120)
report("error / prediction for the levels above 7 m",
       ", ".join(f"{q:.3f}" for q in ratio[eps_abs > 7]))
check(np.all(ratio < 1) and np.all(ratio[eps_abs > 7] > 0.9),
      "every level error lies below the prediction, within 10 % above 7 m")
```

The ratio of each measured relative error to the prediction; for the three levels above $7m$ it is 0.934, 0.953 and 0.955 (COMPUTED). The check requires every ratio below 1 and those above $7m$ above 0.9.

```
report("largest |difference (here) - difference (record)|",
       f"{np.max(np.abs(record_diff - our_diff)):.1e}")
check(np.max(np.abs(record_diff - our_diff)) < 1e-12,
      "our differences equal the record's column difference within 1e-12",
      record=f"{TABLE}, column difference")
```

Our differences equal the record's column `difference` to 1.8×10^{-15} (COMPUTED: reproduces `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv`, column `difference`).

```
signed = np.array([float(ROWS[i]["difference"]) for i in nonzero])  # with sign
levels = np.array([analytic_record[i] for i in nonzero])
report("computed too high (levels > 0), too low (levels < 0)",
       f"{np.sum((levels > 0) & (signed > 0))} of {np.sum(levels > 0)}, "
       f"{np.sum((levels < 0) & (signed < 0))} of {np.sum(levels < 0)}")
check(np.all(np.sign(signed) == np.sign(levels)),
      "every computed level lies farther from 0 than the exact one (sign of eps^5)",
      record=f"{TABLE}, columns difference and eps_analytic")
```

The last lines test the sign of the error, which Section 2.24 predicts: $\delta\varepsilon \approx \varepsilon^5 h^4/120$ has the sign of ε. `signed` holds the record's column `difference` with its sign, and `levels` the exact levels. `(levels > 0) & (signed > 0)` is `True` for a positive level computed too high (`&` means "and" for arrays), and `np.sum` counts the `True` entries. The RESULT line prints 33 of 33 and 18 of 18 (COMPUTED): every positive level comes out too high and every negative level too low. `np.sign` is $+1$, -1 or 0 for a positive, negative or zero number, and the check requires the sign of each difference to equal the sign of its level; that is, every computed level lies farther from 0 than the exact one (reproduces `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv`, columns `difference` and `eps_analytic`).

In [13], the last check.

```
names = ["pruefer_staircase", "staircase_slope", "angle_along_y",
         "odd_level_equation", "newton_vs_bisection", "orbitals",
         "level_convergence", "record_differences"]
present = [output_file(f"{FIGURE_FOLDER}/02d_{k}_{name}.png").is_file()
           for k, name in enumerate(names, 1)]
check(all(present), f"all {len(names)} figure files of notebook 02d exist")
all_checks_passed()
```

As in Notebook 02a: the eight figure files must exist, and the last line is ALL 19 CHECKS PASSED (notebook 02d). The 19 checks are: 1 in In [2], 1 in In [3], 2 in In [4], 2 in In [5], 1 in In [7], 1 in In [8], 3 in In [9], 2 in In [10], 1 in In [11], 4 in In [12] and 1 in In [13].

2.29 What we proved, what we computed, what we assumed

PROVED in this chapter (by elementary algebra and calculus, each derivation written out line by line). Each statement is also confirmed by a check of a notebook, directly or through the check of a computation that uses it, with two exceptions that the notebooks only illustrate: the conservation of the oscillator energy by the exact solution (the dashed circle of Figure 02a.6) and the estimates of the best spacing (Notebook 02c, In [6], measures the best spacings but does not check the estimates).

- along the history $a_4 = AHx_4$ the extra-time scale factor obeys $ds_5/dx_4 = -AH\, s_5$ (it deflates exponentially) and the 3-space factor $ds_1/dx_4 = +AH\, s_1$; their product is constant (Section 2.2); the oscillator conserves $E = (x^2 + v^2)/2$;
- one step of size 1/2 for $y' = -y$ gives 1/2, 5/8 and 233/384; the amplification factors of Euler, midpoint and RK4 are the first 2, 3 and 5 terms of the series of e^z (Sections 2.4 and 2.5);
- to leading order, the error of a method of order p for Problem A at $t = 1$ is $e^{-1}(-1)^p h^p/(p+1)!$; halving the step divides an error Ch^p by 2^p; Richardson's estimate $(Y_h - Y_{h/2})/(2^p - 1)$ (Section 2.6);
- the energy factors per step $1 + h^2$, $1 + h^4/4$, $1 - h^6/72 + h^8/576$ and the scale-factor product factors $1 - h^2$ (Euler) and $1 + h^6/72 + h^8/576$ (RK4) (Section 2.7);
- the string's eigenvalues $n^2\pi^2$; the secant formula; the even and odd conditions of the finite well, $\tan z = \sqrt{z_0^2 - z^2}/z$ and $-\cot z = \sqrt{z_0^2 - z^2}/z$, and its four bound states for $z_0 = \sqrt{30}$; the

tail integral $1/(2\kappa)$; the orthogonality of states of different parity; Simpson's weights (Sections 2.12 and 2.13);

- the partial derivatives of $x^2 \sin y$; the linear approximation; the gradient is perpendicular to the level curves and points uphill; the orders 1 and 2 of the forward and central differences and the scale of the best spacing (Section 2.18);

- $\partial_0 = 6H\, d/dz$; the rates a_4', $-a_4' + b_4'/(2b_4)$, $H \cot z$ and $-6H/(\sin z \cos z)$ of the scale factors; $\det g = b_4^3 \cos^2 z$ and $\sqrt{|\det g|} = b_4^{3/2} \sin z \cot z = b_4^{3/2} \cos z$, independent of x_4 for every $a_4(x_4)$ when b_4 is constant, because the inflation of 3-space and the deflation of the extra times cancel; Jacobi's formula for a diagonal metric; the 25 nonzero Christoffel entries, given their formula (Section 2.19);

- the exact levels of the free Kohn-Sham block (the zero mode, $\pm\sqrt{M^2 + (n\pi/L)^2}$, $\pm\sqrt{M^2 + p^2}$ with $\tan(pL) = -p/M$) and the absence of any other level with $|\varepsilon| \le M$; the Prüfer equation $\theta' = \varepsilon - M \sin 2\theta$; $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2 > 0$, so every level is found exactly once with its own label; the slope $(1 - e^{-2ML})/(2M)$ at $\varepsilon = 0$; the cubic Hermite midpoint formula; the RK4 phase lag $\omega^5/120$ per step and the level error $\delta\varepsilon \approx \varepsilon^5 h^4/120$ far from the mass, which has the sign of ε, so that every computed level lies farther from zero than the exact one (Section 2.24).

COMPUTED by the notebooks (each number in the cell named; where a Revision record is reproduced, the record file and its key or check):

- Notebook 02a: measured orders 1.0014, 2.0025, 4.0439 (In [7]); halving ratios 2.00020, 4.00073, 16.1047 (In [9]); the smallest RK4 error 1.110×10^{-16} at $N = 4096$ and 5.773×10^{-15} at $N = 262144$, past the minimum 524 to 3709 times below the bound $N\epsilon e^{-1}$ (In [10]); the Revision solver's step $h = L/G = 3/900 = 1/300$ and the history constants $A = H = 1$ (In [11]; reproduces `Revision/kohn_sham/results/parameters.json`, keys `rk4Steps`, `L_tipCutoff`, `historyA` and `H`); the errors at that step, -6.1399×10^{-4}, $+6.8296 \times 10^{-7}$, $+3.7920 \times 10^{-13}$, within 0.3 per cent of the predictions (In [11]); the oscillator energies (In [12], In [13]); the radii 2.6658, 1.0100, 1.0000 and the angles -0.1302, $+0.06586$ and -1.314×10^{-4} rad by which Euler, midpoint and RK4 run ahead of the exact solution at $t = 10$, each equal to $50\,(\arg R(ih) - h)$ (In [12]); Richardson's estimate within 6 per cent from $N = 8$ on (In [14]).

- Notebook 02b: the RK4 eigenvalue of the string 9.869604411103, within 1.0×10^{-8} of π^2; 45 bisection steps against about 9 secant steps (In [5]); the four well levels -14.1206, -11.5181, -7.3330, -2.0437 (In [6]) and their shooting values within 10^{-7} (In [8]); nodes 0, 1, 2, 3 and orthonormality within 1.6×10^{-10}; for the four pairs of different parity the integral over $-3 \le x \le 3$ at most 9.5×10^{-17}, although each half integral is at least 0.16 in size (In [10]); orders 3.99, 3.98, 3.96, 3.92 (In [11]).

- Notebook 02c: the rise of f along the gradient, 1.898407 per unit length against $|\nabla f| = 1.898293$ (In [4]); the best finite-difference spacings 1.0×10^{-8}, 1.8×10^{-6}, 5.6×10^{-5} (In [6]); the metric (In [8]; reproduces `Revision/gkd_lovelock/results/curvature.json`, key `metricDiagonal`); the volume factor (In [11]; reproduces the same record's key `sqrtAbsDetG` and `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `sqrt_abs_det_g_is_cos_z`); all 25 Christoffel entries (In [12]; reproduces `Revision/gkd_lovelock/results/curvature.json`, key `christoffelNonzero_b_le_c`).

- Notebook 02d: all 54 levels of the free spectrum, identical in all 16 printed digits on the computer that made the record, largest difference 8.9×10^{-16} (In [9]; reproduces `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv`, column `eps_numeric`); the exact column to 1.8×10^{-15} (In [7]; column `eps_analytic`) and the column `difference` to 1.8×10^{-15} (In [12]); the zero mode exactly 0 and its distance 1.35×10^{-12} from the exact form (In [9], In [10]; reproduces `Revision/kohn_sham/reports/ks-rust-solver.json`, check `free_zero_mode_exact`); the quoted largest differences 7.23×10^{-10} and 5.05×10^{-8} (In [9]; reproduces the same report's check `free_k0_analytic_spectra`, whose text misnames its second band as "$4m \leq |\varepsilon| < 7m$" although it contains every level from $4m$ up to $8.7539m$; for the band below $7m$ alone the largest difference is 9.95×10^{-9}; the verdict is unaffected); Newton with 9 shots against bisection with 48 (In [8]); orders 4.00, 3.99, 3.97 (In [11]); errors 0.934 to 0.955 times the phase-lag prediction above $7m$, and the signs of all 51 nonzero differences: 33 of 33 positive levels too high, 18 of 18 negative levels too low (In [12]; reproduces the same table's columns `difference` and `eps_analytic`); the end angles at the edges of the mass gap, $|\Phi(\pm M)| \leq 0.2256\pi < \pi/2$ in all three cases, so no level but the zero mode lies in the gap (In [4]).

ASSUMED (used, not derived here):

- standard theorems of calculus and numerical analysis, quoted without proof: the existence and uniqueness of solutions of initial-value problems (Picard and Lindelöf); Taylor's theorem; the intermediate value theorem; Schwarz's theorem; the mean value theorem; the speed of the secant rule (digits multiplied by about 1.618) and of Newton's method (digits doubled); the h^4 error orders of Simpson's rule and of cubic Hermite interpolation;
- from quantum mechanics, used only as an example: the Schrödinger equation, the uniqueness (up to a factor) of a one-dimensional bound state, and the orthogonality of states of different energies;
- the formula of the Christoffel symbols (derived in Chapter 3) and the Kohn-Sham block equations (taken from `Revision/kohn_sham/ks-theory.json`; derived in Chapter 14);
- the linear history $a_4 = AHx_4$ with $A = 1$, a PRESCRIBED BACKGROUND, not a solution of the field equations with the Kohn-Sham source (`Revision/field_equations_a4/reports/ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`);
- the brane conditions $b(0) = 0$ or $a(0) = 0$, which rest on the mirror symmetry that the record labels ASSUMED; the tip condition $b(-L) = 0$ is a choice of the model ("chosen (regular tip)").

HYPOTHESIS and OPEN. No hypothesis enters this chapter, and it leaves no question open. It does not show where the Kohn-Sham equations come from or what their levels mean physically; that is the work of Chapters 14 and 15.

2.30 Exercises

Exercise 1. Make one Euler step and one midpoint step for $y' = -2y$ from $y(0) = 1$ with the step $h = 1/4$, with exact fractions. Compare with Section 2.4 and explain the agreement.

Answer. Euler: $y_1 = 1 + \frac{1}{4}\cdot(-2) = \frac{1}{2}$. Midpoint: $k_1 = -2$; the midpoint value is $1 + \frac{1}{8}\cdot(-2) = \frac{3}{4}$; $k_2 = -2\cdot\frac{3}{4} = -\frac{3}{2}$; $y_1 = 1 + \frac{1}{4}\cdot(-\frac{3}{2}) = 1 - \frac{3}{8} = \frac{5}{8}$. These are the same numbers as for $y' = -y$ with $h = 1/2$ in Section 2.4, because one step of each method multiplies y by $R(z)$, which depends only on $z = \lambda h$, and here $z = -2\cdot\frac{1}{4} = -\frac{1}{2}$ as there: $R_{\text{Euler}}(-\frac{1}{2}) = \frac{1}{2}$ and $R_{\text{midpoint}}(-\frac{1}{2}) = 1 - \frac{1}{2} + \frac{1}{8} = \frac{5}{8}$.

Exercise 2. A method gives the errors 2.0×10^{-4} with $h = 0.1$ and 1.25×10^{-5} with $h = 0.05$. What is its order, and what error do you expect with $h = 0.025$?

Answer. Halving the step divided the error by $2.0 \times 10^{-4}/1.25 \times 10^{-5} = 16 = 2^4$, so (Section 2.6) the order is $p = 4$. Halving again divides by 16 once more: about $1.25 \times 10^{-5}/16 = 7.8 \times 10^{-7}$. Equivalently, the slope on a log-log plot is $(\log_{10} 1.25{\times}10^{-5} - \log_{10} 2.0{\times}10^{-4})/(\log_{10} 0.05 - \log_{10} 0.1) = (-1.2041)/(-0.30103) = 4.0$.

Exercise 3. Two RK4 runs of Problem A up to $t = 1$ give $Y_h = 0.3678802720$ (8 steps) and $Y_{h/2} = 0.3678794905$ (16 steps). Without using e^{-1}, estimate the error of $Y_{h/2}$ and give the extrapolated value. Then compare with $e^{-1} = 0.3678794412$.

Answer. By Richardson's rule (Section 2.6) with $p = 4$, the error of $Y_{h/2}$ is about $(Y_h - Y_{h/2})/15 = 7.815 \times 10^{-7}/15 = 5.21 \times 10^{-8}$. The extrapolated value is $Y_{h/2} - 5.21 \times 10^{-8} = 0.3678794384$. The true error of $Y_{h/2}$ is $0.3678794905 - 0.3678794412 = 4.93 \times 10^{-8}$ (the estimate is 6 per cent too large), and the extrapolated value is off by only -2.8×10^{-9}, about 17 times better; these are the numbers of the row $N = 8 \to 16$ printed by Notebook 02a, In [14].

Exercise 4. Using the energy factor of the midpoint method (Section 2.7), compute the oscillator's energy after 50 steps of $h = 0.2$, starting from $E_0 = 1/2$.

Answer. Each step multiplies E by $1 + h^4/4 = 1 + 0.0016/4 = 1.0004$. After 50 steps $E = \frac{1}{2}\cdot 1.0004^{50}$. With $1.0004^{50} = e^{50\ln 1.0004} = e^{50\cdot 0.00039992} = e^{0.019996} = 1.020197$, we get $E = 0.5100986$, the value 0.5100986302 printed by Notebook 02a, In [12].

Exercise 5. How many bound states does a finite square well of half-width $a = 1$ have when its depth is $V_0 = 2$? And when $V_0 = 1$?

Answer. The number is the number of intervals of length $\pi/2$ that begin below $z_0 = a\sqrt{2V_0}$ (Section 2.13). For $V_0 = 2$: $z_0 = 2$ and $z_0/(\pi/2) = 1.27$, so two intervals begin below z_0 (at 0 and at $\pi/2 = 1.571$): two bound states, one even and one odd. For $V_0 = 1$: $z_0 = \sqrt{2} = 1.414 < \pi/2$: one bound state, the even ground state. (A symmetric well in one dimension always has at least one bound state, however shallow, because the first interval starts at 0.)

Exercise 6. Plain bisection starts from the bracket $(-12, 12)$ and stops when the bracket is narrower than 10^{-13}. How many shots does it need?

Answer. After k shots the width is $24/2^k$ (Section 2.12). We need $24/2^k \le 10^{-13}$, that is $2^k \ge 2.4 \times 10^{14}$, or $k \ge \log_2(2.4 \times 10^{14}) = \log_{10}(2.4 \times 10^{14})/\log_{10} 2 = 14.380/0.30103 = 47.8$. So $k = 48$ shots, the number printed by Notebook 02d, In [8].

Exercise 7. Show that the free block of Section 2.24 has no level with $\varepsilon = +M$ or $\varepsilon = -M$.

Answer. For $\varepsilon^2 = M^2$ the equation $a'' = (M^2 - \varepsilon^2)a$ becomes $a'' = 0$, so $a = \alpha + \beta y$ with constants α, β, and $b = (Ma - a')/\varepsilon$ vanishes where $Ma - a' = M(\alpha + \beta y) - \beta = 0$. Even orbitals need this at $y = 0$ and at $y = -L$: $M\alpha - \beta = 0$ and $M\alpha - M\beta L - \beta = 0$; subtracting the first from the second gives $-M\beta L = 0$, so $\beta = 0$ (because $M > 0$ and $L > 0$), and then $\alpha = \beta/M = 0$. Odd orbitals need $a(0) = \alpha = 0$ and the tip condition $M(-\beta L) - \beta = -\beta(ML + 1) = 0$, so $\beta = 0$. In both cases $a = 0$,

and then $b = 0$: no level.

Exercise 8. Check the cubic Hermite midpoint formula $(f_0 + f_1)/2 + h(f_0' - f_1')/8$ of Section 2.24 on $f(s) = s^2$ and $f(s) = s^3$ on the step $0 \le s \le h$.

Answer. For s^2: $f_0 = 0$, $f_1 = h^2$, $f_0' = 0$, $f_1' = 2h$; the formula gives $\frac{h^2}{2} + \frac{h}{8}(0 - 2h) = \frac{h^2}{2} - \frac{h^2}{4} = \frac{h^2}{4} = (h/2)^2$, the exact value at $s = h/2$. For s^3: $f_0 = 0$, $f_1 = h^3$, $f_0' = 0$, $f_1' = 3h^2$; the formula gives $\frac{h^3}{2} - \frac{3h^3}{8} = \frac{h^3}{8} = (h/2)^3$, exact again. (It is exact for every polynomial of degree at most 3, because then the cubic through the four data is the polynomial itself.)

Exercise 9. For $M = 0$ the Prüfer equation is $\theta' = \varepsilon$. Find the even and odd levels directly from it and compare with the formulas of Section 2.24 at $M = 0$.

Answer. With $\theta(-L) = 0$ and $\theta' = \varepsilon$ we get $\theta(y) = \varepsilon(y + L)$, so $\Phi = \varepsilon L$. Even: $\varepsilon L = l\pi$, $\varepsilon = l\pi/L$. Odd: $\varepsilon L = \pi/2 + l\pi$, $\varepsilon = (l + \frac{1}{2})\pi/L$. The general formulas give the same: $\pm\sqrt{0 + (n\pi/L)^2} = \pm n\pi/L$ for the even levels; and as $M \to 0$ the odd condition $\tan(pL) = -p/M$ demands an infinitely large tangent, $pL = (n + \frac{1}{2})\pi$, so $\pm p = \pm(n + \frac{1}{2})\pi/L$.

Exercise 10. Compute $\partial_0 g_{11}$ for $g_{11} = e^{2a_4}\sin^{1/3}(6Hx_0)$, and use it to confirm the Christoffel entry $\Gamma^1{}_{01} = H\cot z$ of Section 2.19.

Answer. With $z = 6Hx_0$: $\partial_0 \sin^{1/3} z = \frac{1}{3}\sin^{-2/3} z \cdot \cos z \cdot 6H = 2H\sin^{-2/3} z\cos z$ (the power rule, the derivative of the sine, and the chain rule with $\partial z/\partial x_0 = 6H$). So $\partial_0 g_{11} = 2He^{2a_4}\sin^{-2/3} z\cos z$. Then $\Gamma^1{}_{01} = \partial_0 g_{11}/(2g_{11}) = 2He^{2a_4}\sin^{-2/3} z\cos z/(2e^{2a_4}\sin^{1/3} z) = H\cos z/\sin z = H\cot z$ (the factors e^{2a_4} cancel and $\sin^{-2/3} z/\sin^{1/3} z = 1/\sin z$), the value listed in the record and in the table of Section 2.19.

3 Spacetime geometry and the author's metric; curvature

Gravity, in Einstein's theory, is not a force that acts inside space and time: it is the shape of space and time itself. This chapter builds, from nothing but school algebra and one-variable calculus, the language in which that shape is written down and measured: coordinates, the metric, its signature, scale factors and volumes, the Christoffel symbols, geodesics (the paths of free fall), and curvature in the form of the Riemann tensor, the Ricci tensor, the Ricci scalar, the Einstein tensor and the Kretschmann scalar. Every tool is tried first where we can picture it, on the flat plane and on a sphere, and then applied, completely and line by line, to the metric that the author wrote down for the primordial gravitational field. The chapter has four worked examples, Notebooks 03a, 03c, 03b and 03d (in the order in which the chapter uses them); each reproduces, number for number, the parts of the Revision record that it overlaps and says so in a PASS line.

3.1 What this chapter is for

Every later chapter of the book lives in the author's eight-dimensional spacetime. The spinor fields of Chapters 4 to 10 need its vielbein (the scale factors of Section 3.7); the energy-momentum tensor of Chapter 9 needs its Christoffel symbols (Section 3.23); the field equations of the metric functions a_4 and b_4 in Chapter 12 need its Einstein tensor (Section 3.25); the Kohn-Sham model of Chapters 14 to 17 (**Kohn-Sham**: an approximation of density functional theory that replaces many interacting particles by independent particles, each moving in one common effective potential;

Chapter 13 teaches it from zero) is written in the hidden coordinate y of Section 3.9. This chapter derives all of these from the metric itself, and it ends with what the metric does to a body that moves freely in it (Section 3.31).

The author's metric is the following diagonal 8×8 matrix, in the order of the author's coordinates $x_0, \dots, x_7$ (Section 3.5 reads it exactly as the author typed it):

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4}\sin^{1/3} z,\ e^{2a_4}\sin^{1/3} z,\ e^{2a_4}\sin^{1/3} z,\ -1,\\ -b_4 e^{-2a_4}\sin^{1/3} z,\ -b_4 e^{-2a_4}\sin^{1/3} z,\ -b_4 e^{-2a_4}\sin^{1/3} z\big), \qquad z = 6Hx_0.$$

The coordinates have fixed roles. x_0 is the **hidden** space direction, with $z = 6Hx_0$ between 0 and $\pi/2$ and $g_{00} = \cot^2 z$. x_1, x_2, x_3 are ordinary **3-space**; it inflates, with the scale factor $e^{a_4}\sin^{1/6} z$. x_4 is **the time**. x_5, x_6, x_7 are three **extra times**: they are time-like like x_4, and they **deflate exponentially**, with the scale factor $b_4^{1/2} e^{-a_4}\sin^{1/6} z$, while a_4 grows. $H > 0$ is a constant of the author, and $a_4(x_4)$ and $b_4(x_4)$ are functions of the time only, the **metric functions**, with $b_4 > 0$; a prime means a derivative with respect to x_4, $a_4' = da_4/dx_4$ and $b_4' = db_4/dx_4$. The function b_4 is the author's change of 2026-10-14: with $b_4 = 1$ the metric is the one used before that date (the record of the change: `Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md`). Every formula of this chapter is derived for general $a_4(x_4)$ and $b_4(x_4)$; the pictures and numbers along a history take $b_4 = 1$, as the Revision record does, and say so (Section 3.8 gives the reason).

The four notebooks of the chapter are:

notebook	**what it computes**	**Revision records it reproduces**
03a	reads the metric exactly as the author typed it; determinant, signature (4,4), vielbein, expansion rates, the deflating history, proper volumes, the hidden coordinate y and the warped form; 7 figures, 29 checks	`Revision/gkd_lovelock/results/curvature.json`, `python-lovelock-report.json`, `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`, `Revision/kohn_sham/results/parameters.json`, `Revision/kohn_sham/ks-theory.json`
03c	curvature where it can be pictured: the flat plane in polar coordinates and the sphere; parallel transport, geodesics, geodesic deviation, all with RK4; 5 figures, 16 checks	none (exact formulas only)

notebook	what it computes	Revision records it reproduces
03b	all Christoffel symbols, Riemann, Ricci and Einstein components and the Kretschmann scalar of the author's metric, exactly; finite differences; the curvature along the deflating history; the source Einstein's equations would require; a negative control; 7 figures, 40 checks	`curvature.json`, `python-lovelock-report.json`, `lovelock-report.json` (all three in `Revision/gkd_lovelock/results/`), `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, `Revision/field_equations_a4/a4-equations.json`, `Revision/field_equations_a4/reports/python-a4-report.json`
03d	free fall in the author's metric: conserved momenta, redshift in 3-space, blueshift along the extra times, the push along the hidden direction, the turning point in x_4; RK4 paths with the record's Christoffel symbols; 5 figures, 23 checks	`curvature.json`, `parameters.json`, `ks-theory.json`, `Revision/theory/reports/python-scope.json`

Every statement of the chapter carries one of the five labels of Chapter 0. **PROVED** means derived exactly here, line by line, and confirmed by an exact check of a notebook; where the Revision record proves the same, the record file and its check are named. **COMPUTED** means a number obtained numerically by a notebook, with its measured accuracy. **ASSUMED** marks two kinds of statements: standard theorems that we quote without proof (each is named where it is used), and the physical inputs: that the history of the metric functions is $a_4 = AHx_4$, $b_4 = 1$, which the Revision record itself calls a PRESCRIBED BACKGROUND (Section 3.8), and that the bodies of Section 3.31 are test particles, too small to change the metric. One question is **OPEN** (Section 3.31): whether wave packets of the fields follow the free-fall paths computed here. No HYPOTHESIS enters this chapter. Nothing in this chapter concerns pairs of universes, their creation, or matter and antimatter; those questions belong to Chapters 18 to 21, which also state precisely what is and what is not proved about them.

3.2 Coordinates, indices and sums

Coordinates. A point of the author's spacetime is named by eight real numbers, its **coordinates** $x_0, x_1, \ldots, x_7$. Their roles were listed in Section 3.1. In Python a list of eight numbers is counted from 0, and the author's names are counted from 0 too: position k holds x_k, so position 0 is the hidden coordinate x_0 and position 4 the time x_4; every notebook says so where it matters.

Indices. A letter written as a small label, such as the a in x_a, is an **index**: it stands for any one of the numbers $0, \ldots, 7$. An equation with an index is eight equations, one for each value. Many objects of this chapter carry several indices: the metric g_{ab} has two (it is a table with 64 entries), the

Christoffel symbols $\Gamma^a{}_{bc}$ have three (512 entries), the Riemann tensor $R^a{}_{bcd}$ has four (4096 entries). An index may be written **up** or **down**; Section 3.14 explains why the position matters for vectors and tensors. For the coordinates themselves the position carries no meaning: x_4 and x^4 name the same coordinate, and we write x_a in prose and dx^a for a small step along x_a.

Sums. In this chapter every sum is written with the sign $\sum$; for example $\sum_{a=0}^{7} V^a W_a = V^0W_0 + V^1W_1 + \cdots + V^7W_7$. When the range is the eight coordinates we write $\sum_a$. Where a formula repeats an index but does not sum over it we write "(no sum)".

The Kronecker delta. $\delta^a{}_b$ (also written δ_{ab}) is 1 when $a = b$ and 0 otherwise. It is the table of the 8×8 unit matrix.

Partial derivatives. For a function $f(x_0, \ldots, x_7)$ the **partial derivative** $\partial_a f = \partial f / \partial x_a$ is the ordinary derivative with respect to x_a while the seven other coordinates are held fixed (Chapter 2 teaches it from zero). The author's metric depends on two coordinates only: on x_4 through $a_4(x_4)$ and $b_4(x_4)$, and on x_0 through $z = 6Hx_0$. By the **chain rule**, a function of z has

$$\partial_0 f = \frac{df}{dz}\,\frac{dz}{dx_0} = 6H\,\frac{df}{dz},$$

(the derivative of a function of z with respect to x_0 is its derivative with respect to z times $dz/dx_0 = 6H$), and a function of a_4 and b_4 has $\partial_4 f = a_4' \,\partial f/\partial a_4 + b_4' \,\partial f/\partial b_4$ (the same rule for each of the two functions, with $da_4/dx_4 = a_4'$ and $db_4/dx_4 = b_4'$). Every other partial derivative of the metric is zero.

3.3 The metric: lengths from coordinate steps

Pythagoras. On a flat sheet of paper with perpendicular axes x and y, a small step dx to the right and dy upwards has the squared length

$$ds^2 = dx^2 + dy^2.$$

The coefficients of dx^2 and dy^2 are both 1, and there is no term with $dx\,dy$.

The same plane in polar coordinates. Name a point instead by its distance r from the origin and its angle φ from the x axis, so that $x = r\cos\varphi$ and $y = r\sin\varphi$. We compute ds^2 in these coordinates, line by line.

$$dx = \cos\varphi\, dr - r\sin\varphi\, d\varphi, \qquad dy = \sin\varphi\, dr + r\cos\varphi\, d\varphi$$

(the change of a function of two variables is the sum of its two partial derivatives times the two small changes: $dx = (\partial x/\partial r)\,dr + (\partial x/\partial\varphi)\,d\varphi$, and the same for y)

$$dx^2 = \cos^2\varphi\, dr^2 - 2r\sin\varphi\cos\varphi\, dr\, d\varphi + r^2\sin^2\varphi\, d\varphi^2$$

(the square of a difference, $(p-q)^2 = p^2 - 2pq + q^2$)

$$dy^2 = \sin^2\varphi\, dr^2 + 2r\sin\varphi\cos\varphi\, dr\, d\varphi + r^2\cos^2\varphi\, d\varphi^2$$

(the square of a sum, $(p+q)^2 = p^2 + 2pq + q^2$)

$$ds^2 = dx^2 + dy^2 = dr^2 + r^2\, d\varphi^2$$

(add the two lines: the mixed terms cancel, and $\cos^2\varphi + \sin^2\varphi = 1$ in both remaining pairs). The plane has not changed, but the coefficient of $d\varphi^2$ is now r^2, a number that depends on the point. A step $d\varphi$ in angle is a true length $r\, d\varphi$: the farther from the origin, the longer the arc.

The metric. In general coordinates $x_1, \ldots, x_n$ of an n-dimensional space the squared length of a small step $dx^1, \ldots, dx^n$ is a sum of products of two steps,

$$ds^2 = \sum_{a=1}^{n}\sum_{b=1}^{n} g_{ab}\, dx^a dx^b,$$

with coefficients g_{ab} that may depend on the point. The table g_{ab} is the **metric**; we always take it **symmetric**, $g_{ab} = g_{ba}$ (the two products $dx^a dx^b$ and $dx^b dx^a$ are the same number, so only the sum $g_{ab} + g_{ba}$ matters, and we may share it equally). The formula for ds^2 is called the **line element**. A metric is **diagonal** when $g_{ab} = 0$ for $a \neq b$; then $ds^2 = \sum_a g_{aa}\,(dx^a)^2$. The plane in polar coordinates has the diagonal metric $g = \mathrm{diag}(1, r^2)$ in the order (r, φ).

The sphere. On a sphere of radius a name a point by the angle θ from the north pole and the angle φ around the axis; in three dimensions it is $(a\sin\theta\cos\varphi,\ a\sin\theta\sin\varphi,\ a\cos\theta)$. A step $d\theta$ moves the point along a meridian, a circle of radius a, by the length $a\, d\theta$. A step $d\varphi$ moves it along the circle of latitude, whose radius is $a\sin\theta$, by the length $a\sin\theta\, d\varphi$. The two directions are perpendicular, so by Pythagoras

$$ds^2 = a^2\, d\theta^2 + a^2\sin^2\theta\, d\varphi^2, \qquad g = \mathrm{diag}(a^2,\ a^2\sin^2\theta).$$

The inverse metric. The inverse matrix of g_{ab} is written with upper indices, g^{ab}: $\sum_c g^{ac} g_{cb} = \delta^a{}_b$. For a diagonal metric it is diagonal too, with $g^{aa} = 1/g_{aa}$ (no sum), because the product of two diagonal matrices is the diagonal matrix of the products of their entries.

3.4 Signature: space-like and time-like directions

In the spacetime of special relativity with one space direction x and the time t (in units in which the speed of light is 1) the line element is

$$ds^2 = dx^2 - dt^2.$$

The coefficient of dt^2 is negative. A step dt alone has $ds^2 = -dt^2 < 0$: it is not a length but a **duration**, and the time that a clock carried along the step shows, its **proper time**, is $d\tau = \sqrt{-ds^2} = dt$. A step with $dx = dt$ (one unit of distance per unit of time: a flash of light) has $ds^2 = 0$.

Definitions. A vector V with components $V^1, \ldots, V^n$ (Section 3.14 makes this precise) has the **squared length** $g(V,V) = \sum_{a,b} g_{ab} V^a V^b$. It is **space-like** if $g(V,V) > 0$, **time-like** if $g(V,V) < 0$ and **null** if $g(V,V) = 0$ but V is not zero. For a diagonal metric a step along one coordinate x_a is space-like when $g_{aa} > 0$ and time-like when $g_{aa} < 0$.

Signature. The **signature** (p, q) of a metric at a point is the number p of positive and the number q of negative eigenvalues of the matrix g_{ab}. For a diagonal matrix the eigenvalues are the diagonal entries, so the signature counts the positive and the negative diagonal entries. A theorem of linear algebra, **Sylvester's law of inertia**, says that the signature does not depend on the coordinates (ASSUMED: quoted without proof). The flat metric with the signs of the author's metric is

$$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$$

in the order $x_0, \ldots, x_7$: four positive and four negative entries, the signature **(4,4)**. Four directions are space-like (x_0, x_1, x_2, x_3) and four are time-like (x_4, x_5, x_6, x_7). With four time-like directions one of them must be chosen as the time in which everything evolves; the author chooses x_4, and the other three are the extra times.

3.5 The author's metric

The Rust program `lovelock_gkd`, which computed the curvature of the metric for the Revision record, stored the author's text of the metric under the name `metricAsGiven` in its output file `Revision/gkd_lovelock/results/curvature.json`, followed by a note in brackets. Printed with one row of the matrix per line (Notebook 03a, In [3]):

```
{{Cot[6 H x0]^2,0,0,0,0,0,0,0},
{0,E^(2 a4[x4]) Sin[6 H x0]^(1/3),0,0,0,0,0,0},
{0,0,E^(2 a4[x4]) Sin[6 H x0]^(1/3),0,0,0,0,0},
{0,0,0,E^(2 a4[x4]) Sin[6 H x0]^(1/3),0,0,0,0},
{0,0,0,0,-1,0,0,0},
{0,0,0,0,0,-E^(-2 a4[x4]) b4[x4] Sin[6 H x0]^(1/3),0,0},
{0,0,0,0,0,0,-E^(-2 a4[x4]) b4[x4] Sin[6 H x0]^(1/3),0},
{0,0,0,0,0,0,0,-E^(-2 a4[x4]) b4[x4] Sin[6 H x0]^(1/3)}}
```

It is written in the notation of Mathematica: braces enclose a list (the matrix is a list of eight rows of eight entries, the rows and columns in the order $x_0, x_1, \ldots, x_7$), `E^(...)` is e to the power in brackets, `Sin` and `Cot` are the sine and the cotangent, `a4[x4]` and `b4[x4]` are the values of the functions a_4 and b_4 at x_4, and a blank between two factors means times. The note of the record says that this is the user's metric of 2026-10-14 in the author's coordinates $x_0, \ldots, x_7$, as in the author's notebook `Dirac_Einstein-Lovelock_4+4_Nash.nb`, and that $b_4 = 1$ is the metric before 2026-10-14. Only the eight diagonal entries are non-zero; the line element is therefore

$$\begin{aligned} ds^2 = {} & \cot^2 z\, dx_0^2 + e^{2a_4} \sin^{1/3} z\, (dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2 \\ & - b_4 e^{-2a_4} \sin^{1/3} z\, (dx_5^2 + dx_6^2 + dx_7^2), \qquad z = 6Hx_0. \end{aligned}$$

Reading it term by term. A step dx_1 in 3-space has the true (proper) length $e^{a_4} \sin^{1/6} z\, dx_1$, the square root of its coefficient: when a_4 grows, 3-space **inflates**. A step dx_5 along an extra time

has the proper duration $b_4^{1/2} e^{-a_4} \sin^{1/6} z \, dx_5$: when a_4 grows and b_4 stays constant, the three extra times **deflate exponentially** (in general their rate of change is $-a_4' + b_4'/(2b_4)$, Section 3.7). A step dx_4 has $ds^2 = -dx_4^2$, so a clock at rest shows exactly x_4: the coordinate x_4 is the proper time of an observer at rest (Section 3.23 shows that such an observer is in free fall). The hidden direction has the coefficient $\cot^2 z$, which depends on x_0 only.

Units. $z = 6Hx_0$ is an angle (a pure number), so H has the unit of an inverse length. In all numerical work of the chapter lengths and times are measured in units of $1/H$, which is the same as setting $H = 1$.

The patch. The metric is used for $0 < z < \pi/2$, that is $0 < x_0 < \pi/(12H)$; we call this range the **patch**. Its end $z \to 0$ is the **tip** and its end $z = \pi/2$ the **patch end**. Inside the patch $\sin z > 0$, so the cube root $\sin^{1/3} z$ is a positive number, and $\cot z = \cos z / \sin z > 0$. At the patch end $\cot z = 0$, so $g_{00} = 0$ there and the matrix g has no inverse: the coordinate x_0 stops being a good label there (Section 3.9 shows that another coordinate, y, has no such problem, just as polar coordinates fail at $r = 0$ while the plane is perfectly regular there).

The signature is (4,4) at every point of the patch, for every a_4 and every $b_4 > 0$ (PROVED). The proof is one line per sign: $\cot^2 z > 0$ because $\cot z > 0$ inside the patch; $e^{2a_4} > 0$ and $e^{-2a_4} > 0$ for every real a_4 (an exponential is positive), $b_4 > 0$, and $\sin^{1/3} z > 0$ inside the patch (shown above); hence the three 3-space entries are positive and the three extra-time entries, which carry a minus sign, are negative; and $g_{44} = -1 < 0$. So the signs are $(+,+,+,+,-,-,-,-)$, those of η. Notebook 03a confirms this on a grid of 400 values of z, 13 values of a_4 and three values of b_4 (In [8]); at $z = 0.7$, $a_4 = 0.5$ and $b_4 = 1$ it prints $g_{11} = 2.347679$ and $g_{55} = -0.317724$.

3.6 The determinant and proper volumes

The volume of a small box. In flat Cartesian coordinates a small box with edges dx, dy, dz has the volume $dx\,dy\,dz$. In the plane in polar coordinates a small box with edges dr and $d\varphi$ has the true edge lengths dr and $r\,d\varphi$, so its area is $r\,dr\,d\varphi$. For any diagonal metric the same reasoning gives: a box with coordinate edges $dx^1, \ldots, dx^n$ has the proper edge lengths $\sqrt{|g_{aa}|}\,dx^a$, and its proper volume is the product

$$\prod_{a=1}^{n} \sqrt{|g_{aa}|}\; dx^1 \cdots dx^n = \sqrt{|\det g|}\; dx^1 \cdots dx^n,$$

because the **determinant** of a diagonal matrix is the product of its diagonal entries, and the square root of a product of positive numbers is the product of their square roots. (For a metric that is not diagonal the volume factor is also $\sqrt{|\det g|}$; we quote this without proof, ASSUMED, and do not need it.) The factor $\sqrt{|\det g|}$ is the **volume factor**.

The determinant of the author's metric. We compute it line by line; after each line we name the rule that produced it.

$$\det g = g_{00}\, g_{11}\, g_{22}\, g_{33}\, g_{44}\, g_{55}\, g_{66}\, g_{77}$$

(the determinant of a diagonal matrix is the product of its diagonal entries)

$$= \cot^2 z \cdot \left(e^{2a_4} \sin^{1/3} z\right)^3 \cdot (-1) \cdot \left(-b_4 e^{-2a_4} \sin^{1/3} z\right)^3$$

(the entries inserted; three equal factors written as a cube)

$$= \cot^2 z \cdot e^{6a_4} \sin z \cdot (-1) \cdot \left(-b_4^3 e^{-6a_4} \sin z\right)$$

(a power of a product is the product of the powers: $(e^{2a_4})^3 = e^{6a_4}$, $(\sin^{1/3} z)^3 = \sin z$, $(b_4)^3 = b_4^3$, and an odd power keeps the minus sign, $(-1)^3 = -1$)

$$= b_4^3 \, e^{6a_4} e^{-6a_4} \, \sin^2 z \, \cot^2 z = b_4^3 \sin^2 z \, \cot^2 z$$

(the two minus signs multiply to $+1$, and $e^{6a_4} e^{-6a_4} = e^0 = 1$)

$$= b_4^3 \sin^2 z \, \frac{\cos^2 z}{\sin^2 z} = b_4^3 \cos^2 z$$

($\cot z = \cos z / \sin z$). Inside the patch $\cos z > 0$, and $b_4 > 0$, so

$$\sqrt{|\det g|} = b_4^{3/2} \cos z.$$

This is PROVED here and in the Revision record (`Revision/gkd_lovelock/results/python-lovelock-report.json`, check `sqrt_abs_det_g`; `curvature.json` stores $\sqrt{|\det g|}$ as `Sin[6*H*x0]*Cot[6*H*x0]*b4[x4]^(3/2)` under the name `sqrtAbsDetG`); Notebook 03a reproduces both (In [7]). The function a_4 has dropped out in the fourth line: the factor e^{6a_4} of the three inflating 3-space directions and the factor e^{-6a_4} of the three deflating extra times cancel. The factor b_4^3 of the three extra-time entries stays; at $b_4 = 1$ the volume factor is $\cos z$.

The volume of a slice of constant time. The time x_4 is the time in which everything in this book evolves (it is the time shown by the clocks of observers at rest, Section 3.23). The natural volume is therefore that of a slice $x_4 = \text{const}$, the whole space at one instant of x_4: a seven-dimensional space with the coordinates $x_0, x_1, x_2, x_3, x_5, x_6, x_7$. This 7-volume still mixes kinds of steps: four of its edges are lengths (along x_0, x_1, x_2, x_3) and three are durations (along the extra times x_5, x_6, x_7, which are time-like, Section 3.4), and its volume factor multiplies all seven proper sizes. Its volume factor is the product of the seven scale factors other than that of x_4, which is $\sqrt{|g_{44}|} = 1$; so it equals $\sqrt{|\det g|} = b_4^{3/2} \cos z$. It does not depend on a_4: **the proper 7-volume of a region with fixed coordinate ranges changes in time only through the factor $b_4^{3/2}$; for a constant b_4, in particular on the backgrounds of this book with $b_4 = 1$, it does not change at all** (PROVED; Notebook 03a, In [17]).

Three partial volumes. Take a box with the coordinate edges $\Delta x_1 = \Delta x_2 = \Delta x_3 = 1$ (small enough that z hardly changes along it, or at one value of z). Its proper 3-volume in 3-space is

$$V_3 = \left(e^{a_4} \sin^{1/6} z\right)^3 = e^{3a_4} \sin^{1/2} z$$

(the product of three equal scale factors; a power of a product is the product of the powers). The same box in the three extra times has $V_t = (b_4^{1/2} e^{-a_4} \sin^{1/6} z)^3 = b_4^{3/2} e^{-3a_4} \sin^{1/2} z$, and $V_3 V_t = b_4^{3/2} \sin z$

(the exponentials cancel and $\sin^{1/2} z \cdot \sin^{1/2} z = \sin z$). What 3-space gains in volume through a_4, the extra times lose.

The 7-volume of the whole patch. Per unit coordinate volume of the six directions $x_1, x_2, x_3, x_5, x_6, x_7$, the proper 7-volume of the patch is the integral of the volume factor $b_4^{3/2} \cos(6Hx_0)$ over x_0. The factor $b_4^{3/2}$ does not depend on x_0 and comes out of the integral, which leaves

$$\int_0^{\pi/(12H)} \cos(6Hx_0)\, dx_0 = \Big[\frac{\sin(6Hx_0)}{6H}\Big]_0^{\pi/(12H)}$$

(an antiderivative of $\cos(6Hx_0)$ is $\sin(6Hx_0)/(6H)$: differentiate it with the chain rule to check)

$$= \frac{\sin(\pi/2) - \sin 0}{6H} = \frac{1}{6H}$$

(insert the two limits; $6H \cdot \pi/(12H) = \pi/2$). So the 7-volume is $b_4^{3/2}/(6H)$, which is $1/(6H)$ at $b_4 = 1$. PROVED; Notebook 03a computes the integral of $\cos(6Hx_0)$ with sympy (In [17]).

3.7 Scale factors, the vielbein and the rates of expansion

Scale factors. Write each diagonal entry as a sign times a square, $g_{aa} = \eta_{aa}\, h_a^2$ with a positive number h_a, the **scale factor** of the direction x_a: a coordinate interval Δx_a over which h_a does not change has the proper length (or, for a time-like direction, the proper duration) $h_a\, \Delta x_a$. For the author's metric, inside the patch,

$$\begin{aligned} h = (&\cot z,\ e^{a_4} \sin^{1/6} z,\ e^{a_4} \sin^{1/6} z,\ e^{a_4} \sin^{1/6} z,\ 1,\\ &b_4^{1/2} e^{-a_4} \sin^{1/6} z,\ b_4^{1/2} e^{-a_4} \sin^{1/6} z,\ b_4^{1/2} e^{-a_4} \sin^{1/6} z) \end{aligned}$$

in the order $x_0, \ldots, x_7$ (the positive square roots of the absolute values of the diagonal entries: $\sqrt{\cot^2 z} = \cot z$ because $\cot z > 0$ in the patch, $\sqrt{e^{2a_4} \sin^{1/3} z} = e^{a_4} \sin^{1/6} z$, and $\sqrt{b_4 e^{-2a_4} \sin^{1/3} z} = b_4^{1/2} e^{-a_4} \sin^{1/6} z$ because the root of a product is the product of the roots).

The vielbein. At every point imagine eight measuring rods and clocks of unit proper length, one along each coordinate direction, all perpendicular to each other. Such a set is a **frame**, and the matrix $e^a{}_\mu$ that converts coordinate steps into frame components is the **vielbein** (German for "many legs"): the metric is

$$g_{\mu\nu} = \sum_{a,b} e^a{}_\mu\, \eta_{ab}\, e^b{}_\nu .$$

For a diagonal metric the diagonal vielbein $e^a{}_\mu = h_\mu\, \delta^a{}_\mu$ (no sum) does this: the double sum keeps only $a = b = \mu = \nu$ and gives $\eta_{\mu\mu} h_\mu^2 = g_{\mu\mu}$. The spinor fields of the book need exactly this frame (Chapter 6). PROVED here. The Revision record also contains the **lead checks**: short, independent Python programs in the folder `Revision/lead_checks`, written from scratch by the coordinator of the Revision work (the "lead") without using any other Revision code, each of which writes a

report listing its named checks and their verdicts. The lead's check `vielbein_reproduces_metric` of `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json` checks the same, and Notebook 03a reproduces it (In [11]).

The rate of expansion. A coordinate interval Δx_a has the proper length $L = h_a\,\Delta x_a$. The fraction by which it grows per unit of time is

$$\frac{1}{L}\frac{\partial L}{\partial x_4} = \frac{\partial_4 h_a}{h_a} = \partial_4 \ln h_a$$

(the coordinate interval does not change; the derivative of $\ln u$ is u'/u). For 3-space,

$$\ln h_1 = a_4 + \tfrac{1}{6}\ln\sin z \quad \Rightarrow \quad \partial_4 \ln h_1 = a_4'$$

(the logarithm of a product is the sum of the logarithms, $\ln e^{a_4} = a_4$, $\ln \sin^{1/6} z = \frac{1}{6}\ln\sin z$; only a_4 depends on x_4). For an extra time,

$$\ln h_5 = \tfrac{1}{2}\ln b_4 - a_4 + \tfrac{1}{6}\ln\sin z \quad \Rightarrow \quad \partial_4 \ln h_5 = -a_4' + \frac{b_4'}{2b_4} \equiv \alpha_t$$

(the same rules, with $\ln b_4^{1/2} = \frac{1}{2}\ln b_4$ and the derivative b_4'/b_4 of $\ln b_4$). We call α_t the **expansion rate of the extra times**. $h_0 = \cot z$ and $h_4 = 1$ do not depend on x_4, so their rates are 0. When $a_4' > 0$ and b_4 is constant, $\alpha_t = -a_4'$: **3-space expands at the rate a_4' and the three extra times shrink at the same rate**. The six **transverse** scale factors (those of $x_1, x_2, x_3, x_5, x_6, x_7$) multiply to

$$h_1 h_2 h_3 h_5 h_6 h_7 = e^{3a_4}\sin^{1/2} z \cdot b_4^{3/2} e^{-3a_4}\sin^{1/2} z = b_4^{3/2}\sin z,$$

without a_4. All of this is PROVED and is checked exactly by Notebook 03a (In [12]). The factor $\sin^{1/6} z$ that the six transverse scale factors share is the **warp factor** $W = \sin^{1/6} z$; the six transverse metric entries contain its square $W^2 = \sin^{1/3} z$.

3.8 The deflating history: a prescribed background

The metric does not fix the functions $a_4(x_4)$ and $b_4(x_4)$. The field equations of gravity relate them to the energy and the pressures of the matter in the spacetime; Chapter 12 derives them. With the author's own weights of the field equations every vacuum solution has a constant b_4 (record `Revision/metric_b4/results/vacuum-b4.json`), and a constant b_4 is removed by the rescaling $x_5, x_6, x_7 \to b_4^{1/2}x_5, b_4^{1/2}x_6, b_4^{1/2}x_7$ of the extra times: it gives back the metric with $b_4 = 1$. Every Revision calculation that needs a background history therefore keeps $b_4 = 1$ and says so (`Revision/SPEC.md`, section 1). For pictures and numbers this chapter does the same and uses the simplest member of the family, the **linear history**

$$a_4 = AHx_4, \qquad a_4' = AH, \qquad a_4'' = 0, \qquad b_4 = 1,$$

with a constant **slope** A. With $A > 0$ the function a_4 increases, 3-space inflates like e^{AHx_4} and the extra times deflate like e^{-AHx_4}: exponential deflation, as the author requires. The Revision Kohn-Sham work uses $A = 1$ and $H = 1$, and computes its states on five **slices** $a_4 = 0, 0.5, 1, 1.5, 2$ (record `Revision/kohn_sham/results/parameters.json`, keys `physics.H`, `physics.historyA`, `physics.slicesA4`; Notebook 03a, In [13]). Between the first and the last slice, 3-space lengths grow by the factor $e^2 = 7.389056$ and extra-time durations shrink by the factor $e^{-2} = 0.135335$ (Notebook 03a, In [14]); the product of the two factors stays 1.

Status: ASSUMED. The history is not a solution of the coupled field equations. The Revision record says so in its own words (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.historyStatus`): "PRESCRIBED BACKGROUND: the history a4 = A H x4 is prescribed, not solved for." The reason, also stated there: the field equations of a_4 allow this linear member only for a source with equal pressures in all directions and a constant energy density, and the Kohn-Sham states computed along it do not have these properties (record `Revision/field_equations_a4/reports/ks-source-conditions.json`). Every formula of this chapter written with general $a_4(x_4)$ and $b_4(x_4)$ is exact; only the pictures and the numbers along the history use the assumption. The pictures and numbers of this chapter use only slopes $A > 0$, for which the extra times deflate, as the author requires; a slope $A = 0$ would freeze the extra times and a slope $A < 0$ would make them inflate and 3-space deflate.

3.9 The hidden coordinate y and the warped form of the metric

The entry $\cot^2 z$ makes the hidden direction awkward: it grows without bound at the tip and vanishes at the patch end. The Revision Kohn-Sham work uses instead the coordinate

$$y = \frac{\ln(\sin z)}{6H}, \qquad z = 6Hx_0.$$

It measures proper distance. Line by line:

$$\frac{dy}{dx_0} = \frac{1}{6H} \cdot \frac{\cos z}{\sin z} \cdot 6H$$

(the chain rule: the derivative of $\ln u$ is $1/u$, the derivative of $\sin z$ is $\cos z$, and $dz/dx_0 = 6H$)

$$= \cot z$$

(the factors $6H$ cancel, and $\cos z / \sin z = \cot z$). So $dy = \cot z \, dx_0$ and $dy^2 = \cot^2 z \, dx_0^2 = g_{00} \, dx_0^2$: a step dy is a step of proper length dy along the hidden direction. (Record: `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`, check `ks_coordinate_jacobian`; Notebook 03a, In [19].)

The warp factor in y. From the definition, $6Hy = \ln \sin z$, so

$$\sin z = e^{6Hy}$$

(e to the power of both sides; $e^{\ln u} = u$), and therefore

$$W = \sin^{1/6} z = \left(e^{6Hy}\right)^{1/6} = e^{Hy}, \qquad W^2 = \sin^{1/3} z = e^{2Hy}$$

(a power of a power multiplies the exponents). Inserting dy^2 and W^2 into the line element of Section 3.5 gives the **warped form**

$$ds^2 = dy^2 - dx_4^2 + W^2\Big[e^{2a_4}\left(dx_1^2 + dx_2^2 + dx_3^2\right) - b_4 e^{-2a_4}\left(dx_5^2 + dx_6^2 + dx_7^2\right)\Big], \qquad W = e^{Hy}.$$

At $b_4 = 1$ this is exactly the line element of the record (`Revision/kohn_sham/ks-theory.json`, keys `geometry.lineElement` and `geometry.warp`), which is written for the Kohn-Sham background with $b_4 = 1$; Notebook 03a checks the form with b_4 entry by entry with sympy (In [19]). PROVED.

The range of y. As z runs from 0 to $\pi/2$, $\sin z$ runs from 0 to 1 and $\ln \sin z$ from $-\infty$ to 0. So y runs from $-\infty$ (the tip) to 0 (the patch end). The tip lies at an infinite proper distance from every point of the patch (PROVED; Notebook 03a, In [19]). At the patch end $y = 0$ nothing goes wrong in the warped form: the coefficient of dy^2 is 1. The failure of g_{00} there was a failure of the coordinate x_0, not of the geometry.

The volume factor in y. In the coordinates $(y, x_1, \ldots, x_7)$ the scale factor of the hidden direction is 1, so the volume factor is the product of the six transverse scale factors, $b_4^{3/2} W^6 = b_4^{3/2} e^{6Hy} = b_4^{3/2} \sin z$; at $b_4 = 1$ it is $e^{6Hy} = \sin z$ (record key `geometry.sqrtDetG`: "e^{6Hy} (= cos z in the author's chart x0..x7)"). The two volume factors $b_4^{3/2} \sin z$ (in y) and $b_4^{3/2} \cos z$ (in x_0) describe the same volume, because $\cos z\, dx_0 = \cos z\, \tan z\, dy = \sin z\, dy$ (from $dx_0 = dy/\cot z = \tan z\, dy$).

The tip cut-off. The Revision Kohn-Sham solver keeps only $-L \le y \le 0$ with $L = 3/H$ (record key `physics.L_tipCutoff`). The fraction of the 7-volume that it leaves out is (the factor $b_4^{3/2}$ does not depend on y and cancels in the ratio)

$$\frac{\int_{-\infty}^{-L} e^{6Hy}\, dy}{\int_{-\infty}^{0} e^{6Hy}\, dy} = \frac{e^{-6HL}/(6H)}{1/(6H)} = e^{-6HL}$$

(an antiderivative of e^{6Hy} is $e^{6Hy}/(6H)$, which tends to 0 as $y \to -\infty$), which for $L = 3/H$ is $e^{-18} = 1.5230 \times 10^{-8}$. The cut-off lies at the tiny angle $z = \arcsin e^{-18} = 1.5230 \times 10^{-8}$ (both COMPUTED in Notebook 03a, In [20]; the fraction is exact).

3.10 Example: the author's metric, read and checked

Notebook 03a puts Sections 3.2 to 3.9 to work. It reads the metric exactly as the author typed it from the record `curvature.json` and turns the text into an exact sympy matrix; checks it against a second, hand-typed copy and against the record's diagonal entries; computes the determinant and the volume factor; checks the signature on a grid; builds the vielbein and the expansion rates; reads the history of the Revision Kohn-Sham work ($b_4 = 1$) and draws the scale factors along it; computes the proper volumes; and checks the hidden coordinate y, the warped form and the volume factor in y against the record `ks-theory.json`. It needs no Rust, runs in about 15 seconds and ends with the line ALL 29 CHECKS PASSED (notebook 03a).

3.11 How to run Notebook 03a

Step 1. What this notebook does and what it needs.

Notebook 03a (The author's metric: signature, scale factors, warp and proper volumes) is the file `Revision/textbook/notebooks/03a_authors_metric.ipynb` of the repository Dirac_claude. It reads the metric of the primordial gravitational field exactly as the author typed it, with the two metric functions a4 and b4 (from the Revision record of the curvature computation), turns the text into an exact sympy matrix, and checks its diagonal entries, its determinant, its signature (4,4) and its diagonal vielbein against the Revision records. It then computes the expansion rate of every direction, the proper volumes and the hidden coordinate y, and draws the metric as a heat map, its entries versus z, the scale factors of 3-space (inflating) and of the three extra times (deflating exponentially) along the deflating history a4 = A H x4, b4 = 1 of the Revision Kohn-Sham work, the warp factor and the proper volumes. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
```

```
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 03a_authors_metric.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a

number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 03a_authors_metric.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 03a_authors_metric.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/03a.captions.json`
- `Revision/textbook/figures/03a_1_metric_heat_map.png`
- `Revision/textbook/figures/03a_2_diagonal_entries.png`
- `Revision/textbook/figures/03a_3_history_scale_factors.png`
- `Revision/textbook/figures/03a_4_hidden_direction.png`
- `Revision/textbook/figures/03a_5_scale_factor_maps.png`
- `Revision/textbook/figures/03a_6_proper_volumes.png`
- `Revision/textbook/figures/03a_7_hidden_coordinate_y.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all seven figure files exist
ALL 29 CHECKS PASSED (notebook 03a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/03a_authors_metric.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 03a_authors_metric.ipynb
python -m nbconvert --execute --inplace 03a_authors_metric.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

- "FileNotFoundError" for curvature.json, python-lovelock-report.json, emt-divergence-and-spin-connection.json, parameters.json or ks-theory.json: the notebook reads these five Revision records of the repository, so the folder Revision of your copy is incomplete. Restore it with the following command in the repository folder (or clone the repository again) and run the notebook again:

```
git checkout -- Revision
```

- "AssertionError: record check failed": a Revision report no longer lists a check that the notebook relies on as passed, so your copy of the folder Revision differs from the one the book was built with. Restore it with the same command and run the notebook again:

```
git checkout -- Revision
```

3.12 Notebook 03a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/03a_authors_metric.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook studies the **metric of the primordial gravitational field**, the
$8 \times 8$ matrix that the author wrote down and on which every later computation
of the book is built. It

- reads the metric **exactly as the author typed it** (the text is stored in the
  Revision record `Revision/gkd_lovelock/results/curvature.json`) and turns the text
  into an exact sympy matrix, step by step;
- checks the eight diagonal entries, the determinant $\det g = b_4^3 \cos^2 z$ and
  $\sqrt{|\det g|} = b_4^{3/2} \cos z$ against the Revision records;
- checks the **signature** (4,4): four space-like directions $x_0, x_1, x_2, x_3$ and
  four time-like directions $x_4, x_5, x_6, x_7$;
- builds the diagonal **vielbein** (the scale factors) and checks that it reproduces
  the metric;
- computes the **expansion rate** of every direction: 3-space expands with the rate
  $a_4'$, the three extra times shrink (deflate) with the rate $-a_4' + b_4'/(2b_4)$;
- draws the scale factors along the **deflating history** $a_4 = A H x_4$, $b_4 = 1$
  that the Revision Kohn-Sham work uses as a prescribed background ($A = 1$, $H = 1$);
- computes the **proper volumes** (3-space grows like $e^{3 a_4}$, the extra times
  shrink like $b_4^{3/2} e^{-3 a_4}$, the 7-volume stays constant when $b_4$ does) and
  the **hidden coordinate** $y = \ln(\sin z)/(6H)$ in which the metric takes its
  warped form;
- draws 7 figures and prints a PASS line for every check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 03a (The author's metric: signature, scale factors, warp and proper volumes) is
the file 'Revision/textbook/notebooks/03a_authors_metric.ipynb' of the repository
Dirac_claude. It reads the metric of the primordial gravitational field exactly as the
author typed it, with the two metric functions a4 and b4 (from the Revision record of the
curvature computation), turns the text into an exact sympy matrix, and checks its
diagonal entries, its determinant, its signature (4,4) and its diagonal vielbein against
the Revision records. It then computes the expansion rate of every direction, the proper
volumes and the hidden coordinate y, and draws the metric as a heat map, its entries
versus z, the scale factors of 3-space (inflating) and of the three extra times
(deflating exponentially) along the deflating history a4 = A H x4, b4 = 1 of the Revision
Kohn-Sham work, the warp factor and the proper volumes. It needs a computer with Windows
11, macOS or Linux, an internet connection for the installation, the program Git, and
Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at
exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv

Then close the terminal and open a NEW one (a terminal opened before the installation
does not know the new programs), and check the version of Python; it must print 3.12 or a
higher number:
```

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 03a_authors_metric.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 15 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.
```

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 03a_authors_metric.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 03a_authors_metric.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/03a.captions.json‘
- ‘Revision/textbook/figures/03a_1_metric_heat_map.png‘
- ‘Revision/textbook/figures/03a_2_diagonal_entries.png‘
- ‘Revision/textbook/figures/03a_3_history_scale_factors.png‘
- ‘Revision/textbook/figures/03a_4_hidden_direction.png‘
- ‘Revision/textbook/figures/03a_5_scale_factor_maps.png‘
- ‘Revision/textbook/figures/03a_6_proper_volumes.png‘
- ‘Revision/textbook/figures/03a_7_hidden_coordinate_y.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all seven figure files exist
ALL 29 CHECKS PASSED (notebook 03a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel

  called python3; with the environment active, run the following command and start
  JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from
  a copy outside the repository. Open the file
  ‘Revision/textbook/notebooks/03a_authors_metric.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 03a_authors_metric.ipynb
      python -m nbconvert --execute --inplace 03a_authors_metric.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for curvature.json, python-lovelock-report.json,
  emt-divergence-and-spin-connection.json, parameters.json or ks-theory.json: the
  notebook reads these five Revision records of the repository, so the folder Revision of
  your copy is incomplete. Restore it with the following command in the repository folder
  (or clone the repository again) and run the notebook again:

      git checkout -- Revision

- "AssertionError: record check failed": a Revision report no longer lists a check that
  the notebook relies on as passed, so your copy of the folder Revision differs from the
  one the book was built with. Restore it with the same command and run the notebook
  again:

      git checkout -- Revision

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a

RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 03a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 03a (The author’s metric: signature, scale factors, warp and proper volumes)
# is the file Revision/textbook/notebooks/03a_authors_metric.ipynb of the repository
# Dirac_claude. It reads the metric of the primordial gravitational field exactly as the
# author typed it, with the two metric functions a4 and b4 (from the Revision record of
# the curvature computation), turns the text into an exact sympy matrix, and checks its
# diagonal entries, its determinant, its signature (4,4) and its diagonal vielbein
# against the Revision records. It then computes the expansion rate of every direction,
# the proper volumes and the hidden coordinate y, and draws the metric as a heat map, its
# entries versus z, the scale factors of 3-space (inflating) and of the three extra times
# (deflating exponentially) along the deflating history a4 = A H x4, b4 = 1 of the
# Revision Kohn-Sham work, the warp factor and the proper volumes. It needs a computer
# with Windows 11, macOS or Linux, an internet connection for the installation, the
# program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
```

```
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 03a_authors_metric.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
```

```
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 03a_authors_metric.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 03a_authors_metric.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/03a.captions.json
# - Revision/textbook/figures/03a_1_metric_heat_map.png
# - Revision/textbook/figures/03a_2_diagonal_entries.png
# - Revision/textbook/figures/03a_3_history_scale_factors.png
# - Revision/textbook/figures/03a_4_hidden_direction.png
# - Revision/textbook/figures/03a_5_scale_factor_maps.png
# - Revision/textbook/figures/03a_6_proper_volumes.png
# - Revision/textbook/figures/03a_7_hidden_coordinate_y.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all seven figure files exist
#     ALL 29 CHECKS PASSED (notebook 03a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/03a_authors_metric.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
```

```
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 03a_authors_metric.ipynb
#     python -m nbconvert --execute --inplace 03a_authors_metric.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for curvature.json, python-lovelock-report.json,
#   emt-divergence-and-spin-connection.json, parameters.json or ks-theory.json: the
#   notebook reads these five Revision records of the repository, so the folder Revision
#   of your copy is incomplete. Restore it with the following command in the repository
#   folder (or clone the repository again) and run the notebook again:
#     git checkout -- Revision
# - "AssertionError: record check failed": a Revision report no longer lists a check that
#   the notebook relies on as passed, so your copy of the folder Revision differs from
#   the one the book was built with. Restore it with the same command and run the
#   notebook again:
#     git checkout -- Revision
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "03a"  # this notebook: chapter 03, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).
```

```
    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
```

```
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 03a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$: eight numbers that name a point of the
  8-dimensional spacetime. The author's names and roles: $x_0$ the **hidden** space
  direction; $x_1, x_2, x_3$ ordinary 3-space; $x_4$ the time; $x_5, x_6, x_7$ the
  three **extra times**. (In Python lists they are counted 0 to 7 as well: $x_k$ is
  entry $k$.)
- **Metric** $g_{\mu\nu}$: the table of numbers (a symmetric matrix) that turns a
  small coordinate step $dx^\mu$ into a squared length,
  $ds^2 = \sum_{\mu,\nu} g_{\mu\nu}\, dx^\mu dx^\nu$. The entries may depend on the
  point.
- **Diagonal metric**: $g_{\mu\nu} = 0$ whenever $\mu \ne \nu$; then
  $ds^2 = \sum_\mu g_{\mu\mu} (dx^\mu)^2$.
- **Line element**: the formula for $ds^2$.
- **Space-like, time-like**: a step along a direction with $g_{\mu\mu} > 0$ has a
  positive squared length (space-like); with $g_{\mu\mu} < 0$ a negative one
  (time-like: it measures a time, not a distance).
- **Signature** $(p, q)$: the number $p$ of positive and $q$ of negative eigenvalues
  of the metric matrix. For a diagonal matrix the eigenvalues are the diagonal
  entries. The author's metric has signature (4,4).
- $\eta$: the flat metric with the same signs, $\eta = \mathrm{diag}(+1, +1, +1,
  +1, -1, -1, -1, -1)$ in the order $x_0, \dots, x_7$.
- **Determinant** $\det g$: for a diagonal matrix the product of its diagonal
  entries. $\sqrt{|\det g|}\, d^8x$ is the proper (true) volume of a small
  coordinate box.
- **Scale factor** of a direction: the number $h_\mu = \sqrt{|g_{\mu\mu}|}$ that
  turns a coordinate interval $\Delta x^\mu$ into a proper length
  $h_\mu \Delta x^\mu$.
- **Vielbein** $e^a{}_\mu$: a matrix with $g_{\mu\nu} = \sum_{a,b} e^a{}_\mu
  \eta_{ab} e^b{}_\nu$; for a diagonal metric the diagonal matrix of the scale
  factors.
- **Expansion rate** (Hubble rate) of a direction: $\partial_{x_4} \ln h_\mu$, the
  fraction by which its lengths grow per unit of time $x_4$.
- **Inflate, deflate**: grow, shrink exponentially. 3-space inflates; the extra
  times deflate.
- $z = 6 H x_0$: the hidden coordinate in angle form, between $0$ and $\pi/2$.
- **Patch**: the range $0 < z < \pi/2$ on which the metric is used. Its end
  $z = \pi/2$ is the **patch end**; the end $z \to 0$ is the **tip**.
- **Warp factor** $W = \sin^{1/6} z$: the factor that the scale factors of the six
  **transverse** directions $x_1, x_2, x_3, x_5, x_6, x_7$ share; the six transverse
  entries $g_{11}, g_{22}, g_{33}, g_{55}, g_{66}, g_{77}$ of the metric contain its
  square $W^2 = \sin^{1/3} z$.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: a second coordinate for the hidden
  direction, in which the warp factor is $W = e^{Hy}$ (section 13).
- **Prescribed background**: a metric that is given and not solved for from field
  equations.
- **Lead checks**: short independent Python programs of the Revision record (folder
  `Revision/lead_checks`), written from scratch by the coordinator of the Revision
  work (the "lead") without importing any other Revision code; each writes a report
  with a list of named checks and their verdicts.
- **sympy**: the Python package for exact algebra with symbols; **numpy**: arrays of
  numbers; **matplotlib**: plots.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's metric of the primordial gravitational field is diagonal. In the order
$x_0, \dots, x_7$ its diagonal entries are

$$g = \mathrm{diag}\bigl(\cot^2 z,\ e^{2a_4}\sin^{1/3}z,\ e^{2a_4}\sin^{1/3}z,\
e^{2a_4}\sin^{1/3}z,\ -1,\ -b_4 e^{-2a_4}\sin^{1/3}z,\ -b_4 e^{-2a_4}\sin^{1/3}z,\
-b_4 e^{-2a_4}\sin^{1/3}z\bigr),\qquad z = 6 H x_0 .$$

Here $H > 0$ is a constant of the author (a number with the unit 1/length), and
$a_4 = a_4(x_4)$ and $b_4 = b_4(x_4) > 0$ are functions of the time $x_4$ only, the
**metric functions**; $b_4$ is the author's change of 2026-10-14, and $b_4 = 1$ gives
the metric used before. A prime means the derivative with respect to $x_4$:
$a_4' = d a_4 / d x_4$, $b_4' = d b_4 / d x_4$.

The line element is therefore

$$ds^2 = \cot^2 z\, dx_0^2 + e^{2a_4}\sin^{1/3}z\,(dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2
- b_4 e^{-2a_4}\sin^{1/3}z\,(dx_5^2 + dx_6^2 + dx_7^2) .$$

Read it term by term. A step $dx_1$ in 3-space has the proper length
$e^{a_4}\sin^{1/6} z\, dx_1$: when $a_4$ grows, 3-space **inflates**. A step $dx_5$
along an extra time has the proper duration $b_4^{1/2} e^{-a_4}\sin^{1/6} z\, dx_5$:
when $a_4$ grows and $b_4$ stays constant, the extra times **deflate
exponentially**. The time $x_4$ has the coefficient
$-1$, so a step $dx_4$ is a proper time interval of exactly $dx_4$: the clock of an
observer at rest shows $x_4$. The hidden direction has the coefficient $\cot^2 z$,
which depends on $x_0$ only.

The metric alone does not fix the functions $a_4(x_4)$ and $b_4(x_4)$; the field
equations of $a_4$ and $b_4$, derived later in the book, relate them to the energy
and the pressures of the matter that fills the spacetime (in the vacuum they force
$b_4$ to be constant). For the plots this notebook uses the simplest member of the
family, $a_4 = A H x_4$ and $b_4 = 1$, with the values of the Revision Kohn-Sham
work: $A = 1$ and the unit of length chosen so that $H = 1$. This history is a
**prescribed background** (it is assumed, not solved for). With $A > 0$ the function
$a_4$ increases, so the extra times deflate, as the author requires.
```

Text cell:

```
## 5. Reading the metric exactly as the author wrote it

First a technical step. While a cell runs, Jupyter sends what the cell prints to the
notebook in pieces, about one piece every 0.2 seconds, so where one piece ends and
the next begins depends on the speed of the computer. The tools that build this
book read the PASS line of a check together with the line under it that names the
Revision record, and they read the two correctly only when both arrive in the same
piece. The next cell therefore defines the function `in_one_piece(helper)`: it
returns a new function that does exactly what `helper` does, but lets `helper`
print into a text buffer in memory (an `io.StringIO`, with
```

‘contextlib.redirect_stdout‘) and then writes the whole buffer with one call of ‘sys.stdout.write‘, which arrives as one piece. (The stars in ‘*arguments‘ and ‘**options‘ pass on all the values the new function is given, whatever they are.) The cell then replaces the helpers ‘check‘ and ‘report‘ of the set-up cell by such functions. Nothing else changes: the same checks, the same PASS lines, the same numbers.

In [2]:

```
import contextlib  # redirect_stdout: send printed text into a buffer
import io  # StringIO: a text buffer in memory
import sys  # sys.stdout: the place where printed text goes


def in_one_piece(helper):
    """A function that does what helper does, with all the lines that helper prints
    written in one piece."""
    def helper_in_one_piece(*arguments, **options):
        collected = io.StringIO()  # an empty text buffer
        with contextlib.redirect_stdout(collected):  # print() writes into it
            helper(*arguments, **options)  # a failing check stops here, as before
        sys.stdout.write(collected.getvalue())  # all the lines with one write
    return helper_in_one_piece


check = in_one_piece(check)  # a PASS line and its record line stay together
report = in_one_piece(report)  # a long RESULT line and its second line, too
say("From now on check and report print each of their results in one piece.")
```

Out [2]:

```
From now on check and report print each of their results in one piece.
```

Text cell:

The Rust program ‘lovelock_gkd‘, which computed the curvature of this metric for the Revision record, stored the author’s text of the metric in its output file ‘Revision/gkd_lovelock/results/curvature.json‘ under the name ‘metricAsGiven‘. The next cell reads that file and prints the text, one row of the matrix per line (it only starts a new line after each ‘},‘ and changes nothing else). It is written in the language of Mathematica: ‘{...}‘ is a list (the whole matrix is a list of eight rows, each row a list of eight entries), ‘E^(...)‘ means e to the power $(...)$, ‘Sin[...]‘ and ‘Cot[...]‘ are the sine and the cotangent, ‘a4[x4]‘ and ‘b4[x4]‘ are the values of the functions a_4 and b_4 at x_4, and a blank between two factors means times. After the matrix the record adds, in brackets, where the text comes from.

The cell also defines the function ‘record_check(report_file, check_name, detail_part)‘. Some PASS lines below say that a result reproduces a named check of a Revision report (a JSON file with a list or a dictionary ‘checks‘, each check with its name, its verdict and a detail text). ‘record_check‘ opens the report, finds the

check with that name and stops the notebook with an error unless the check is there
and passed (and, if ‘detail_part‘ is given, its detail contains that text). Every
PASS line that names a check of a Revision report calls it first, so that a renamed,
missing or failing check in the record is caught.

In [3]:

```
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
from sympy.parsing.sympy_parser import (implicit_multiplication, parse_expr,
                                        standard_transformations)


def record_check(report_file, check_name, detail_part=""):
    """Return True when the Revision report report_file lists the check check_name
    as passed (and its detail contains detail_part); otherwise stop the notebook."""
    checks = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
    checks = checks["checks"]  # a dictionary or a list, depending on the report
    if isinstance(checks, dict):  # {name: {"passed": true, "detail": ...}}
        entry = checks.get(check_name, {})
        passed = entry.get("passed") is True
    else:  # [{"name": ..., "verdict": "PASS", "detail": ...}, ...]
        entry = next((e for e in checks if e.get("name") == check_name), {})
        passed = entry.get("verdict") == "PASS"
    if not passed or detail_part not in entry.get("detail", ""):
        raise AssertionError(f"record check failed: {report_file} does not list "
                             f"{check_name} as passed")
    return True


CURVATURE_RECORD = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE_RECORD).read_text(encoding="utf-8"))
# the metric exactly as the author typed it, then a note in brackets
author_text, note = record["metricAsGiven"].split("}} ", 1)
author_text += "}}"  # the two closing brackets that split removed
say(f"The metric as the author wrote it ({len(author_text)} characters):")
for line in author_text.replace("},", "},\n").split("\n"):  # one row per line
    print(line)
say("The note of the record: " + note)
```

Out [3]:

```
The metric as the author wrote it (359 characters):
{{Cot[6 H x0]^2,0,0,0,0,0,0,0},
{0,E^(2 a4[x4]) Sin[6 H x0]^(1/3),0,0,0,0,0,0},
{0,0,E^(2 a4[x4]) Sin[6 H x0]^(1/3),0,0,0,0,0},
{0,0,0,E^(2 a4[x4]) Sin[6 H x0]^(1/3),0,0,0,0},
{0,0,0,0,-1,0,0,0},
{0,0,0,0,0,-E^(-2 a4[x4]) b4[x4] Sin[6 H x0]^(1/3),0,0},
{0,0,0,0,0,0,-E^(-2 a4[x4]) b4[x4] Sin[6 H x0]^(1/3),0},
{0,0,0,0,0,0,0,-E^(-2 a4[x4]) b4[x4] Sin[6 H x0]^(1/3)}}
The note of the record: (the user’s metric of 2026-10-14 in the author’s coordinates
```

```
    x0..x7, as in the author's notebook Dirac_Einstein-Lovelock_4+4_Nash.nb; b4 = 1 is
    the metric before 2026-10-14)
```

Text cell:

```
The next cell makes the **symbols** of the computation: the eight coordinates, the
constant $H$ (declared positive), the metric functions $a_4(x_4)$ and $b_4(x_4)$
(unknown functions of $x_4$, so that sympy can differentiate them; $b_4$ declared
positive), and five plain symbols that we use for printing: ‘a4p‘ and ‘b4p‘ stand for
$a_4'$ and $b_4'$, ‘a4v‘ and ‘b4v‘ for the values of $a_4$ and $b_4$, and ‘z‘ for
$6 H x_0$.
```

In [4]:

```
x0, x1, x2, x3, x4, x5, x6, x7 = sp.symbols("x0:8", real=True)  # 8 coordinates
COORDS = [x0, x1, x2, x3, x4, x5, x6, x7]
H = sp.symbols("H", positive=True)  # the constant of the author, H > 0
a4 = sp.Function("a4")(x4)  # the metric function a4: an unknown function of x4
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0 of x4
a4p, a4v = sp.symbols("a4p a4v", real=True)  # a4p = a4 prime, a4v = value of a4
b4p = sp.symbols("b4p", real=True)  # b4p = b4 prime
b4v = sp.symbols("b4v", positive=True)  # b4v = value of b4
z = sp.symbols("z", positive=True)  # z = 6 H x0, for printing


def plain(expression):
    """Write the derivatives of a4 and b4 as a4p and b4p, their values as a4v and
    b4v, and 6 H x0 as z."""
    expression = expression.subs(sp.Derivative(a4, x4), a4p).subs(a4, a4v)
    expression = expression.subs(sp.Derivative(b4, x4), b4p).subs(b4, b4v)
    return expression.subs(x0, z / (6 * H))  # then 6*H*x0 becomes z


say("Symbols: x0 ... x7, H > 0, the functions a4(x4) and b4(x4), and a4p, a4v, b4p, "
    "b4v, z for printing.")
```

Out [4]:

```
Symbols: x0 ... x7, H > 0, the functions a4(x4) and b4(x4), and a4p, a4v, b4p, b4v, z for
    printing.
```

Text cell:

```
The next cell turns the author's text into Python, one rule at a time, and lets
sympy read the result. Each ‘replace‘ changes one piece of Mathematica notation into
the Python notation that sympy understands: ‘exp^‘ (which an older text of the
author used) becomes ‘E^‘ (sympy's ‘E‘ is the number $e = 2.718\ldots$), the square
brackets of ‘Sin[6 H x0]‘ and ‘Cot[6 H x0]‘ become round brackets, ‘a4[x4]‘ and
‘b4[x4]‘ become ‘a4‘ and ‘b4‘ (which the dictionary ‘names‘ connects to the functions
```

```
$a_4(x_4)$ and $b_4(x_4)$), the curly brackets of the lists become square brackets,
and the power sign ‘^‘ becomes ‘**‘. The option ‘implicit_multiplication‘ reads a
blank between two factors as a multiplication, as Mathematica does: ‘6 H x0‘ means
$6 \cdot H \cdot x_0$.
```

In [5]:

```
text = author_text
text = text.replace("exp^", "E^")  # e to the power ...
text = text.replace("Sin[6 H x0]", "sin(6 H x0)")  # Mathematica [ ] -> Python ( )
text = text.replace("Cot[6 H x0]", "cot(6 H x0)")
text = text.replace("a4[x4]", "a4")  # the value of the function a4 at x4
text = text.replace("b4[x4]", "b4")  # the value of the function b4 at x4
text = text.replace("{", "[").replace("}", "]")  # Mathematica lists -> Python lists
text = text.replace("^", "**")  # Mathematica power ^ -> Python power **
names = {"E": sp.E, "a4": a4, "b4": b4, "H": H, "x0": x0, "sin": sp.sin,
         "cot": sp.cot}
rows = parse_expr(text, local_dict=names,
                  transformations=standard_transformations + (implicit_multiplication,))
g = sp.Matrix(rows)  # the metric of the author as an exact 8 x 8 sympy matrix
say(f"The matrix has {g.rows} rows and {g.cols} columns. Its diagonal entries:")
for k in range(8):
    say(f"  g[x{k}, x{k}] = {g[k, k]}")
off_diagonal = [g[i, j] for i in range(8) for j in range(8) if i != j]
check(all(entry == 0 for entry in off_diagonal),
      "the 56 entries off the diagonal are zero: the metric is diagonal")
```

Out [5]:

```
The matrix has 8 rows and 8 columns. Its diagonal entries:
  g[x0, x0] = cot(6*H*x0)**2
  g[x1, x1] = exp(2*a4(x4))*sin(6*H*x0)**(1/3)
  g[x2, x2] = exp(2*a4(x4))*sin(6*H*x0)**(1/3)
  g[x3, x3] = exp(2*a4(x4))*sin(6*H*x0)**(1/3)
  g[x4, x4] = -1
  g[x5, x5] = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
  g[x6, x6] = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
  g[x7, x7] = -b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
PASS the 56 entries off the diagonal are zero: the metric is diagonal
```

Text cell:

```
Two independent checks of this reading follow. First, the next cell types the
metric a second time by hand, from the formula of section 4 of this notebook, and
checks that the two matrices agree entry by entry (the difference of every entry
simplifies to $0$). Second, it compares each diagonal entry with the entry that the
Rust program ‘lovelock_gkd‘ wrote into the record (‘metricDiagonal‘, in Mathematica
notation with explicit ‘*‘ signs). The small function ‘from_mathematica‘ translates
such a record entry into sympy with the same rules as above; the check that no
square bracket is left proves that every piece of notation was translated.
```

In [6]:

```
warp = sp.sin(6 * H * x0) ** sp.Rational(1, 3)  # sin(z)^(1/3), Rational: exact 1/3
typed = sp.diag(sp.cot(6 * H * x0) ** 2,  # the author's order x0, x1, ..., x7
                sp.exp(2 * a4) * warp, sp.exp(2 * a4) * warp, sp.exp(2 * a4) * warp,
                -1,
                -b4 * sp.exp(-2 * a4) * warp, -b4 * sp.exp(-2 * a4) * warp,
                -b4 * sp.exp(-2 * a4) * warp)
difference = (g - typed).applyfunc(sp.simplify)  # simplify every entry
check(difference == sp.zeros(8, 8),
      "the text of the author equals the metric typed by hand from the formula")


def from_mathematica(entry_text):
    """A record entry in Mathematica notation, as a sympy expression."""
    t = entry_text.replace("Sin[6*H*x0]", "sin(6*H*x0)")
    t = t.replace("Cot[6*H*x0]", "cot(6*H*x0)")
    t = t.replace("a4[x4]", "a4").replace("b4[x4]", "b4").replace("^", "**")
    if "[" in t or "]" in t:  # a piece of notation that was not translated
        raise ValueError(f"cannot translate {entry_text}")
    return parse_expr(t, local_dict=names)


agree = [sp.simplify(g[k, k] - from_mathematica(entry)) == 0
         for k, entry in enumerate(record["metricDiagonal"])]
check(len(agree) == 8 and all(agree),
      "the 8 diagonal entries equal those of the Rust program lovelock_gkd",
      record=f"{CURVATURE_RECORD}, metricDiagonal")
```

Out [6]:

```
PASS the text of the author equals the metric typed by hand from the formula
PASS the 8 diagonal entries equal those of the Rust program lovelock_gkd
     reproduces Revision/gkd_lovelock/results/curvature.json, metricDiagonal
```

Text cell:

```
## 6. The determinant and the volume factor

The determinant of a diagonal matrix is the product of its diagonal entries:

$$\det g = \cot^2 z \cdot \bigl(e^{2a_4}\sin^{1/3}z\bigr)^3 \cdot (-1) \cdot
\bigl(-b_4 e^{-2a_4}\sin^{1/3}z\bigr)^3 .$$

Line by line: $\bigl(e^{2a_4}\sin^{1/3}z\bigr)^3 = e^{6a_4}\sin z$ (a power of a
product is the product of the powers); $\bigl(-b_4 e^{-2a_4}\sin^{1/3}z\bigr)^3 =
-b_4^3 e^{-6a_4}\sin z$ (an odd power keeps the minus sign); the two minus signs
multiply to $+1$ and $e^{6a_4} e^{-6a_4} = 1$, so $\det g = b_4^3 \sin^2 z \cot^2 z$;
and $\cot z = \cos z / \sin z$ gives $\det g = b_4^3 \cos^2 z$. On the patch
$0 < z < \pi/2$ the cosine is positive, so $\sqrt{|\det g|} = b_4^{3/2} \cos z$.
The function $a_4$ has dropped out: the volume factor changes with time only through
```

```
$b_4$, and not at all when $b_4$ is constant. The next cell lets sympy do the same and
compares with the record (‘sqrtAbsDetG‘, written there as
‘Sin[6*H*x0]*Cot[6*H*x0]*b4[x4]^(3/2)‘); then it compares numbers at seven points with
$b_4 = 1$.
```

In [7]:

```
det_g = g.det()  # the product of the diagonal entries; sympy cancels exp(6 a4)
say(f"det g = {plain(det_g)}")
record_check("Revision/gkd_lovelock/results/python-lovelock-report.json",
             "sqrt_abs_det_g", "det g = b4^3 cos^2(6 H x0) exactly")  # stops if not
check(sp.simplify(det_g - b4 ** 3 * sp.cos(6 * H * x0) ** 2) == 0,
      "det g = b4^3 cos(z)^2: the function a4 drops out",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check sqrt_abs_det_g")
# sqrt|det g| on the patch, where cos z > 0 and b4 > 0
volume_factor = b4 ** sp.Rational(3, 2) * sp.cos(6 * H * x0)
check(sp.simplify(from_mathematica(record["sqrtAbsDetG"]) - volume_factor) == 0,
      "sqrt|det g| = b4^(3/2) sin(z) cot(z) = b4^(3/2) cos(z)",
      record=f"{CURVATURE_RECORD}, sqrtAbsDetG")
# numbers: sqrt(|det g|) at 7 points of the patch with b4 = 1, compared with cos z
z_points = np.linspace(0.1, 1.5, 7)
det_numbers = np.array([float(det_g.subs(b4, 1).subs({H: 1, x0: zp / 6}))
                        for zp in z_points])
check(np.max(np.abs(np.sqrt(np.abs(det_numbers)) - np.cos(z_points))) < 1e-14,
      "at b4 = 1, sqrt|det g| equals cos z at 7 points of the patch")
```

Out [7]:

```
det g = b4v**3*sin(z)**2*cot(z)**2
PASS det g = b4^3 cos(z)^2: the function a4 drops out
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    sqrt_abs_det_g
PASS sqrt|det g| = b4^(3/2) sin(z) cot(z) = b4^(3/2) cos(z)
     reproduces Revision/gkd_lovelock/results/curvature.json, sqrtAbsDetG
PASS at b4 = 1, sqrt|det g| equals cos z at 7 points of the patch
```

Text cell:

```
## 7. The signature (4,4)

For a diagonal matrix the eigenvalues are the diagonal entries, so the signature is
read off the signs of the eight entries. The next cell turns the eight entries into
fast numerical functions of $z$ and of the values of $a_4$ and $b_4$
(‘sp.lambdify‘), evaluates them on a grid of 400 values of $z$ strictly inside the
patch, 13 values of $a_4$ from $-3$ to $3$ and the three values $b_4 = 0.5$, 1 and 2,
and checks that the sign pattern is always that of $\eta = (+,+,+,+,-,-,-,-)$: four
positive and four negative entries. It also checks two vectors: a step along $x_1$ has
a positive squared length (space-like), a step along $x_5$ a negative one
(time-like).
```

In [8]:

```
ETA = np.array([1, 1, 1, 1, -1, -1, -1, -1])  # the flat metric diag(+,+,+,+,-,-,-,-)
# the entries as functions of z, a4v and b4v
diagonal_z = [plain(g[k, k]) for k in range(8)]
functions = [sp.lambdify((z, a4v, b4v), entry, "numpy") for entry in diagonal_z]


def diagonal_values(z_values, a4_value, b4_value=1.0):
    """The 8 diagonal entries at the points z_values (one row per entry)."""
    return np.array([np.broadcast_to(f(z_values, a4_value, b4_value), z_values.shape)
                     for f in functions], dtype=float)


z_grid = np.linspace(0.0, np.pi / 2, 402)[1:-1]  # 400 points strictly inside
patterns = set()
for b4_value in (0.5, 1.0, 2.0):
    for a4_value in np.linspace(-3.0, 3.0, 13):
        # +1 or -1 for each entry
        signs = np.sign(diagonal_values(z_grid, a4_value, b4_value))
        patterns.update(tuple(int(s) for s in column) for column in signs.T)
say(f"sign patterns found on the grid: {sorted(patterns)}")
check(patterns == {tuple(int(s) for s in ETA)},
      "on the whole grid the signs are (+,+,+,+,-,-,-,-): signature (4,4)")
step_x1 = diagonal_values(np.array([0.7]), 0.5)[1, 0]  # g11: step along x1
step_x5 = diagonal_values(np.array([0.7]), 0.5)[5, 0]  # g55: step along x5
say(f"at z = 0.7, a4 = 0.5, b4 = 1: g11 = {step_x1:.6f} (space-like), "
    f"g55 = {step_x5:.6f} (time-like)")
check(step_x1 > 0 > step_x5, "a step along x1 is space-like, along x5 time-like")
```

Out [8]:

```
sign patterns found on the grid: [(1, 1, 1, 1, -1, -1, -1, -1)]
PASS on the whole grid the signs are (+,+,+,+,-,-,-,-): signature (4,4)
at z = 0.7, a4 = 0.5, b4 = 1: g11 = 2.347679 (space-like), g55 = -0.317724 (time-like)
PASS a step along x1 is space-like, along x5 time-like
```

Text cell:

```
The next cell sets the colours of all figures of this notebook (a palette chosen so
that the curves can be told apart also by readers with a colour-vision deficiency;
every figure also uses line styles and labels, so that it can be read in black and
white) and then draws the metric at one point, $z = \pi/4$, $a_4 = 0.5$ and
$b_4 = 1$, as a **heat map**: each of the $8 \times 8$ entries is a coloured square,
red for positive, blue for negative, grey for zero, and its value is written in it.
```

In [9]:

```
from matplotlib.colors import LinearSegmentedColormap
```

```
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
MAGENTA, GREEN, VIOLET, RED = "#e87ba4", "#008300", "#4a3aa7", "#e34948"
GREY = "#f0efec"  # the neutral middle of the two-sided colour scale
DIVERGING = LinearSegmentedColormap.from_list("blue_grey_red", [BLUE, GREY, RED])
LABELS = [f"$x_{k}$" for k in range(8)]

z_point, a4_point = np.pi / 4, 0.5
values = np.zeros((8, 8))
values[np.diag_indices(8)] = diagonal_values(np.array([z_point]), a4_point)[:, 0]
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(values, cmap=DIVERGING, vmin=-3.0, vmax=3.0)
for i in range(8):
    for j in range(8):
        ax.text(j, i, f"{values[i, j]:.3f}" if i == j else "0", ha="center",
                va="center", fontsize=8)
ax.set_xticks(range(8), LABELS)
ax.set_yticks(range(8), LABELS)
ax.grid(False)  # no grid lines across the squares
ax.set_title("The metric $g_{\\mu\\nu}$ at $z = \\pi/4$, $a_4 = 0.5$, $b_4 = 1$")
fig.colorbar(image, ax=ax, label="value of the entry (no unit)")
save_figure(fig, "metric_heat_map",
            "The author's metric $g_{\\mu\\nu}$ at the point $z = 6Hx_0 = \\pi/4$ "
            "with $a_4 = 0.5$ and $b_4 = 1$, drawn as a heat map: row $\\mu$ and "
            "column $\\nu$ run "
            "over the coordinates $x_0$ to $x_7$, red squares are positive entries, "
            "blue squares negative ones, grey squares zero; the number in each "
            "square is the entry. Only the diagonal is non-zero; the hidden entry is "
            "$\\cot^2(\\pi/4) = 1$, the three 3-space "
            "entries are equal, $e^{1}\\,(\\sin \\pi/4)^{1/3} = 2.422$, the three "
            "extra-time entries are equal, $-e^{-1}(\\sin \\pi/4)^{1/3} = -0.328$, "
            "and the time entry is $-1$: "
            "four positive and four negative entries, the signature (4,4).")
check(abs(values[1, 1] - np.e * np.sin(np.pi / 4) ** (1 / 3)) < 1e-12,
      "g11 at z = pi/4, a4 = 0.5 equals e times (sin pi/4)^(1/3)")
```

Out [9]:

```
Figure 03a.1 saved as Revision/textbook/figures/03a_1_metric_heat_map.png
PASS g11 at z = pi/4, a4 = 0.5 equals e times (sin pi/4)^(1/3)
```

Text cell:

```
The next cell draws the eight diagonal entries as functions of $z$ across the whole
patch, for $a_4 = 0.5$ and $b_4 = 1$, together with the determinant $\cos^2 z$.
Entries that are equal are drawn as one curve. The hidden entry $\cot^2 z$ grows
without bound towards the tip $z \to 0$ and vanishes at the patch end $z = \pi/2$,
where the determinant vanishes too: there the coordinate $x_0$ stops being a good
coordinate.
```

In [10]:

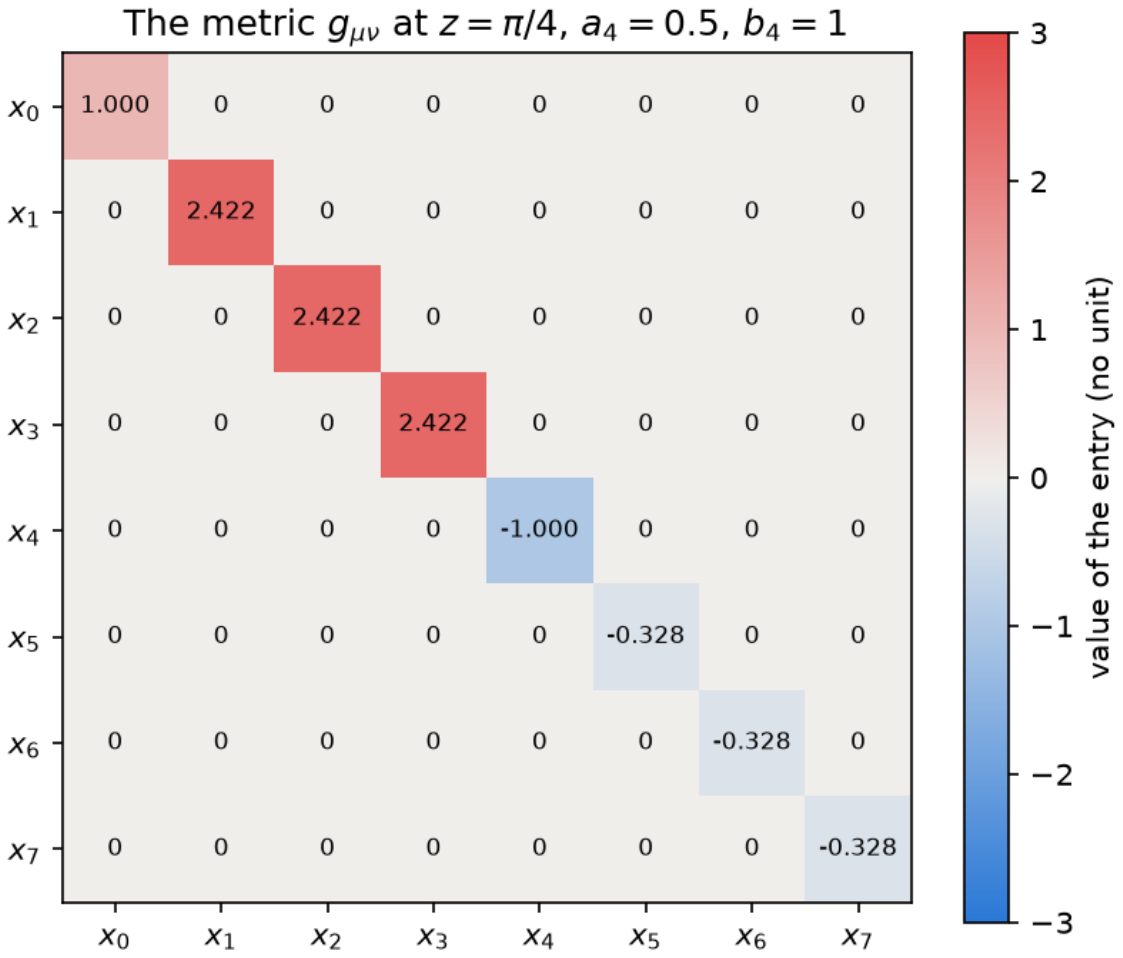

Figure 92: The author's metric $g_{\mu\nu}$ at the point $z = 6Hx_0 = \pi/4$ with $a_4 = 0.5$ and $b_4 = 1$, drawn as a heat map: row μ and column ν run over the coordinates x_0 to x_7, red squares are positive entries, blue squares negative ones, grey squares zero; the number in each square is the entry. Only the diagonal is non-zero; the hidden entry is $\cot^2(\pi/4) = 1$, the three 3-space entries are equal, $e^1\,(\sin\pi/4)^{1/3} = 2.422$, the three extra-time entries are equal, $-e^{-1}(\sin\pi/4)^{1/3} = -0.328$, and the time entry is -1: four positive and four negative entries, the signature (4,4). (Notebook 03a, figure 1.)

```
z_line = np.linspace(0.02, np.pi / 2, 600)
entries = diagonal_values(z_line, 0.5)  # at b4 = 1
fig, ax = plt.subplots()
ax.plot(z_line, entries[0], color=AQUA, lw=1.8, label="$g_{00} = \\cot^2 z$")
ax.plot(z_line, entries[1], color=RED, lw=1.8,
        label="$g_{11} = g_{22} = g_{33} = e^{2a_4}\\sin^{1/3}z$")
ax.plot(z_line, entries[4], color=VIOLET, lw=1.8, ls="-.", label="$g_{44} = -1$")
ax.plot(z_line, entries[5], color=BLUE, lw=1.8, ls="--",
        label="$g_{55} = g_{66} = g_{77} = -b_4 e^{-2a_4}\\sin^{1/3}z$")
ax.plot(z_line, np.cos(z_line) ** 2, color="black", lw=1.2, ls=":",
        label="$\\det g = \\cos^2 z$")
ax.axhline(0.0, color="black", lw=0.8)
ax.axvline(np.pi / 2, color="grey", lw=0.8, ls=":")
ax.text(np.pi / 2 - 0.02, 3.3, "patch end", ha="right", fontsize=9)
ax.text(0.05, 3.3, "towards the tip", fontsize=9)
ax.set_ylim(-3.4, 3.6)  # room below the curves for the legend
ax.set_xlabel("$z = 6 H x_0$ (radians)")
ax.set_ylabel("entry of the metric (no unit)")
ax.set_title("The diagonal entries across the patch ($a_4 = 0.5$, $b_4 = 1$)")
ax.legend(fontsize=8, loc="lower center", ncol=2)
save_figure(fig, "diagonal_entries",
            "The diagonal entries of the author's metric as functions of "
            "$z = 6Hx_0$ from the tip side to the patch end $z = \\pi/2$, for "
            "$a_4 = 0.5$ and $b_4 = 1$; horizontal axis $z$ in radians, vertical "
            "axis the value of "
            "the entry (pure numbers). The three 3-space entries (solid red) stay "
            "positive and the three extra-time entries (dashed blue) and the time "
            "entry (dash-dotted) stay negative everywhere, so the signature is (4,4) "
            "at every point; the hidden entry $\\cot^2 z$ (aqua) grows without bound "
            "towards the tip and reaches $0$ at the patch end, where the determinant "
            "$\\cos^2 z$ (dotted) also vanishes.")
check(np.all(entries[0] > 0) and np.cos(np.pi / 2) ** 2 < 1e-30,
      "g00 > 0 inside the patch and det g = 0 at the patch end")
```

Out [10]:

```
Figure 03a.2 saved as Revision/textbook/figures/03a_2_diagonal_entries.png
PASS g00 > 0 inside the patch and det g = 0 at the patch end
```

Text cell:

```
## 8. The vielbein: the scale factor of each direction

Write each diagonal entry as a sign times a square: $g_{\mu\mu} = \eta_{\mu\mu}
h_\mu^2$ with a positive **scale factor** $h_\mu$. On the patch all the factors below
are positive (and $\cot z > 0$), so

$$h = \bigl(\cot z,\ e^{a_4}\sin^{1/6}z,\ e^{a_4}\sin^{1/6}z,\ e^{a_4}\sin^{1/6}z,\
1,\ b_4^{1/2}e^{-a_4}\sin^{1/6}z,\ b_4^{1/2}e^{-a_4}\sin^{1/6}z,\
b_4^{1/2}e^{-a_4}\sin^{1/6}z\bigr).$$

The diagonal matrix of these numbers is the **vielbein** $e^a{}_\mu$ used for the
```

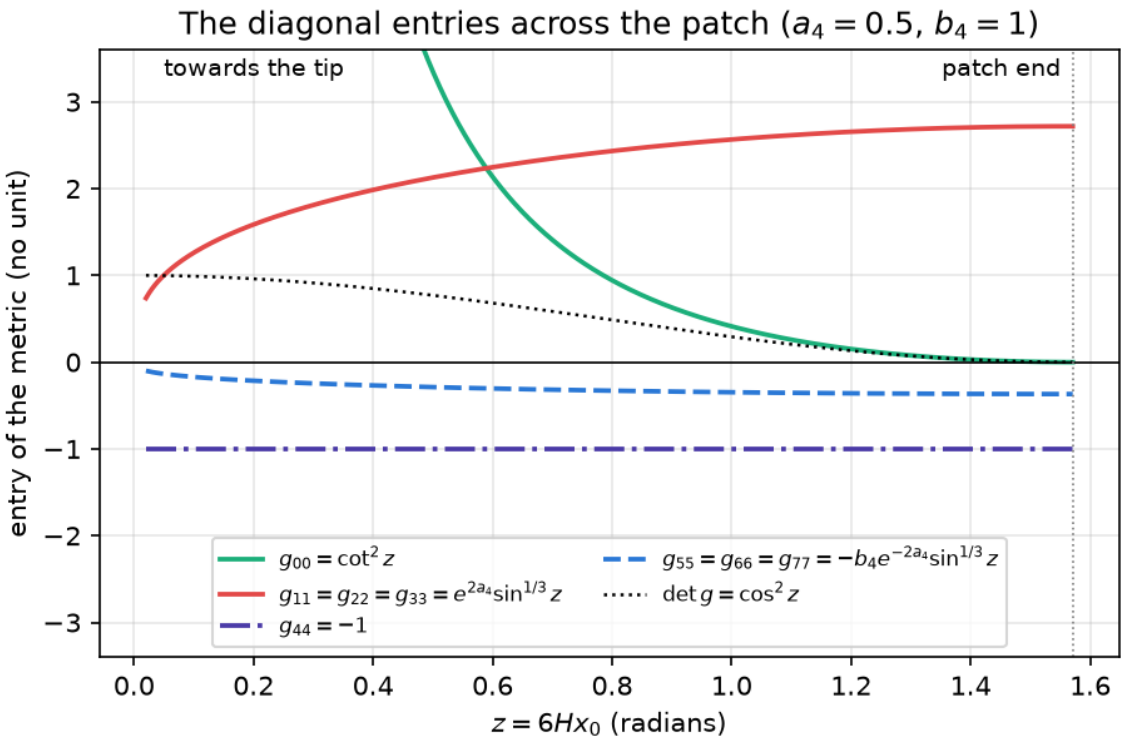

Figure 93: The diagonal entries of the author's metric as functions of $z = 6Hx_0$ from the tip side to the patch end $z = \pi/2$, for $a_4 = 0.5$ and $b_4 = 1$; horizontal axis z in radians, vertical axis the value of the entry (pure numbers). The three 3-space entries (solid red) stay positive and the three extra-time entries (dashed blue) and the time entry (dash-dotted) stay negative everywhere, so the signature is (4,4) at every point; the hidden entry $\cot^2 z$ (aqua) grows without bound towards the tip and reaches 0 at the patch end, where the determinant $\cos^2 z$ (dotted) also vanishes. (Notebook 03a, figure 2.)

spinors later in the book. The next cell checks $\eta_{\mu\mu} h_\mu^2 = g_{\mu\mu}$ for all eight directions; the lead's independent check of the Revision record did the same.

In [11]:

```
sixth = sp.sin(6 * H * x0) ** sp.Rational(1, 6)  # sin(z)^(1/6)
# the author's order x0, x1, ..., x7
h = [sp.cot(6 * H * x0)] + [sp.exp(a4) * sixth] * 3 + [sp.Integer(1)] \
    + [sp.sqrt(b4) * sp.exp(-a4) * sixth] * 3
for k in range(8):
    say(f"  h[x{k}] = {plain(h[k])}")
LEAD_EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
record_check(LEAD_EMT, "vielbein_reproduces_metric")  # stops if not passed
check(all(sp.simplify(int(ETA[k]) * h[k] ** 2 - g[k, k]) == 0 for k in range(8)),
      "eta times h squared reproduces every diagonal entry of the metric",
      record=f"{LEAD_EMT}, check vielbein_reproduces_metric")
```

Out [11]:

```
  h[x0] = cot(z)
  h[x1] = exp(a4v)*sin(z)**(1/6)
  h[x2] = exp(a4v)*sin(z)**(1/6)
  h[x3] = exp(a4v)*sin(z)**(1/6)
  h[x4] = 1
  h[x5] = sqrt(b4v)*exp(-a4v)*sin(z)**(1/6)
  h[x6] = sqrt(b4v)*exp(-a4v)*sin(z)**(1/6)
  h[x7] = sqrt(b4v)*exp(-a4v)*sin(z)**(1/6)
PASS eta times h squared reproduces every diagonal entry of the metric
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check vielbein_reproduces_metric
```

Text cell:

```
## 9. The expansion rate of each direction

The proper length of a coordinate interval $\Delta x_\mu$ is $h_\mu \Delta x_\mu$.
Its relative growth per unit time is $\partial_{x_4} \ln h_\mu$. For 3-space,
$\ln h_1 = a_4 + \tfrac16 \ln \sin z$, and only $a_4$ depends on $x_4$, so the rate
is $a_4'$; for an extra time $\ln h_5 = \tfrac12 \ln b_4 - a_4 + \tfrac16 \ln \sin z$
and the rate is $-a_4' + b_4'/(2b_4)$; $h_0$ and $h_4$ do not depend on $x_4$, so their
rates are $0$. The next cell lets sympy differentiate. It also checks that the six
transverse scale factors multiply to $b_4^{3/2} \sin z$, without $a_4$: what 3-space
gains, the extra times lose.
```

In [12]:

```
rates = [sp.simplify(sp.diff(sp.log(h[k]), x4)) for k in range(8)]
for k in range(8):
```

```
    say(f"  expansion rate of x{k}: {plain(rates[k])}")
a4_prime, b4_prime = sp.Derivative(a4, x4), sp.Derivative(b4, x4)
expected = ([0] + [a4_prime] * 3 + [0]
            + [-a4_prime + b4_prime / (2 * b4)] * 3)  # the order x0, x1, ..., x7
check([sp.simplify(r - e) == 0 for r, e in zip(rates, expected)] == [True] * 8,
      "3-space expands with rate a4p, the extra times shrink with rate "
      "-a4p + b4p/(2 b4)")
transverse = sp.simplify(h[1] * h[2] * h[3] * h[5] * h[6] * h[7])
say(f"product of the six transverse scale factors = {plain(transverse)}")
check(sp.simplify(transverse - b4 ** sp.Rational(3, 2) * sp.sin(6 * H * x0)) == 0,
      "the six transverse scale factors multiply to b4^(3/2) sin z, without a4")
```

Out [12]:

```
  expansion rate of x0: 0
  expansion rate of x1: a4p
  expansion rate of x2: a4p
  expansion rate of x3: a4p
  expansion rate of x4: 0
  expansion rate of x5: -a4p + b4p/(2*b4v)
  expansion rate of x6: -a4p + b4p/(2*b4v)
  expansion rate of x7: -a4p + b4p/(2*b4v)
PASS 3-space expands with rate a4p, the extra times shrink with rate -a4p + b4p/(2 b4)
product of the six transverse scale factors = b4v**(3/2)*sin(z)
PASS the six transverse scale factors multiply to b4^(3/2) sin z, without a4
```

Text cell:

```
## 10. The deflating history of the Revision Kohn-Sham work

The next cell reads the parameters of the Revision Kohn-Sham solver
(‘Revision/kohn_sham/results/parameters.json‘): the constant $H$, the slope $A$ of
the history $a_4 = A H x_4$, the five values of $a_4$ (the **slices**) at which that
solver computes its states, and the tip cut-off $L$ (used in section 13 of this
notebook). It also reads the theory record of that work
(‘Revision/kohn_sham/ks-theory.json‘) and prints the first and the last sentence of
its statement about the **status of the history** (key ‘historyStatus‘). The
sentence in between, not printed, says why: the field equations of $a_4$ allow the
history $a_4 = A H x_4$ only for a source with equal pressures in all directions
and a constant energy density, and the Kohn-Sham states computed along it do not
have these properties. So the history is assumed, not derived; this notebook uses
it only to draw the time dependence of the metric.
```

In [13]:

```
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
physics = parameters["physics"]
H_value, A_value = physics["H"], physics["historyA"]
slices = physics["slicesA4"]  # the values of a4 at which the solver works
L_tip = physics["L_tipCutoff"]  # the tip cut-off, in units of 1/H
```

```
say(f"H = {H_value}, A = {A_value}, slices a4 = {slices}, tip cut-off L = {L_tip}")
check(H_value == 1.0 and A_value == 1.0 and slices == [0.0, 0.5, 1.0, 1.5, 2.0],
      "the history of the record is a4 = x4 (A = 1, H = 1) with slices 0 to 2")
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks_theory = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
status = ks_theory["adiabaticity"]["historyStatus"]  # the record's own statement
sentences = status.split(". ")  # the statement, cut into its sentences
say("status of the history (record): " + sentences[0] + ".")
say(sentences[-1])  # the last sentence already ends with a full stop
check(status.startswith("PRESCRIBED BACKGROUND") and "not solved for" in status,
      "the record states that the history is a prescribed background",
      record=f"{KS_THEORY}, adiabaticity.historyStatus")
```

Out [13]:

```
H = 1.0, A = 1.0, slices a4 = [0.0, 0.5, 1.0, 1.5, 2.0], tip cut-off L = 3.0
PASS the history of the record is a4 = x4 (A = 1, H = 1) with slices 0 to 2
status of the history (record): PRESCRIBED BACKGROUND: the history a4 = A H x4 is
    prescribed, not solved for.
The Kohn-Sham gas is therefore a test field on a prescribed background without back-
    reaction, and quantities derived along this history (energies, pressures, equations
    of state) are not consequences of the coupled field equations.
PASS the record states that the history is a prescribed background
     reproduces Revision/kohn_sham/ks-theory.json, adiabaticity.historyStatus
```

Text cell:

```
Along this history $a_4 = A H x_4$ and $b_4 = 1$, so $a_4 = 0, 0.5, \dots, 2$ at
$x_4 = 0, 0.5, \dots, 2$ (in units of $1/H$). The scale factor of 3-space is
proportional to $e^{a_4}$ and that of the extra times to $e^{-a_4}$. The next cell
draws both, and their product, against $x_4$ twice: on an ordinary vertical axis
(left) and on a logarithmic one (right), on which an exponential is a straight line
whose slope is its rate ($+AH$ and $-AH$). The dots mark the five slices.
```

In [14]:

```
x4_line = np.linspace(0.0, 2.5, 251)
a4_line = A_value * H_value * x4_line  # the history a4 = A H x4
slice_x4 = np.array(slices) / (A_value * H_value)  # the times of the slices
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
for ax, scale in zip(axes, ["linear", "log"]):
    ax.plot(x4_line, np.exp(a4_line), color=RED, lw=1.8,
            label="3-space: $e^{a_4}$ (inflates)")
    ax.plot(x4_line, np.exp(-a4_line), color=BLUE, lw=1.8, ls="--",
            label="extra times: $e^{-a_4}$ (deflate)")
    ax.plot(x4_line, np.exp(a4_line) * np.exp(-a4_line), color="black", lw=1.2,
            ls=":", label="product $= 1$")
    ax.plot(slice_x4, np.exp(slices), "o", color=RED, ms=6)
    ax.plot(slice_x4, np.exp(-np.array(slices)), "s", color=BLUE, ms=6)
    ax.set_yscale(scale)
    ax.set_xlabel("time $x_4$ (unit $1/H$)")
```

```
    ax.set_ylabel("scale factor relative to $x_4 = 0$")
axes[0].set_title("Ordinary vertical axis")
axes[1].set_title("Logarithmic vertical axis: straight lines")
axes[0].legend(fontsize=8)
save_figure(fig, "history_scale_factors",
            "The time dependence of the scale factors along the deflating history "
            "$a_4 = A H x_4$, $b_4 = 1$ with $A = 1$, $H = 1$ of the Revision "
            "Kohn-Sham work: "
            "3-space $e^{a_4}$ (solid red), the three extra times $e^{-a_4}$ (dashed "
            "blue) and their product (dotted black), on an ordinary (left) and a "
            "logarithmic (right) vertical axis; horizontal axis the time $x_4$ in "
            "units of $1/H$, vertical axis the factor by which lengths have grown "
            "since $x_4 = 0$ (a pure number). Dots and squares mark the five slices "
            "$a_4 = 0, 0.5, 1, 1.5, 2$. 3-space grows by $e^2 = 7.39$ and the extra "
            "times shrink to $e^{-2} = 0.135$; on the logarithmic axis both are "
            "straight lines with slopes $+1$ and $-1$, the product stays $1$.")
report("3-space growth factor at the last slice a4 = 2", f"{np.exp(2.0):.6f}")
report("extra-time shrink factor at the last slice a4 = 2", f"{np.exp(-2.0):.6f}")
check(abs(np.exp(slices[-1]) * np.exp(-slices[-1]) - 1.0) < 1e-15,
      "at every slice the two factors multiply to 1")
```

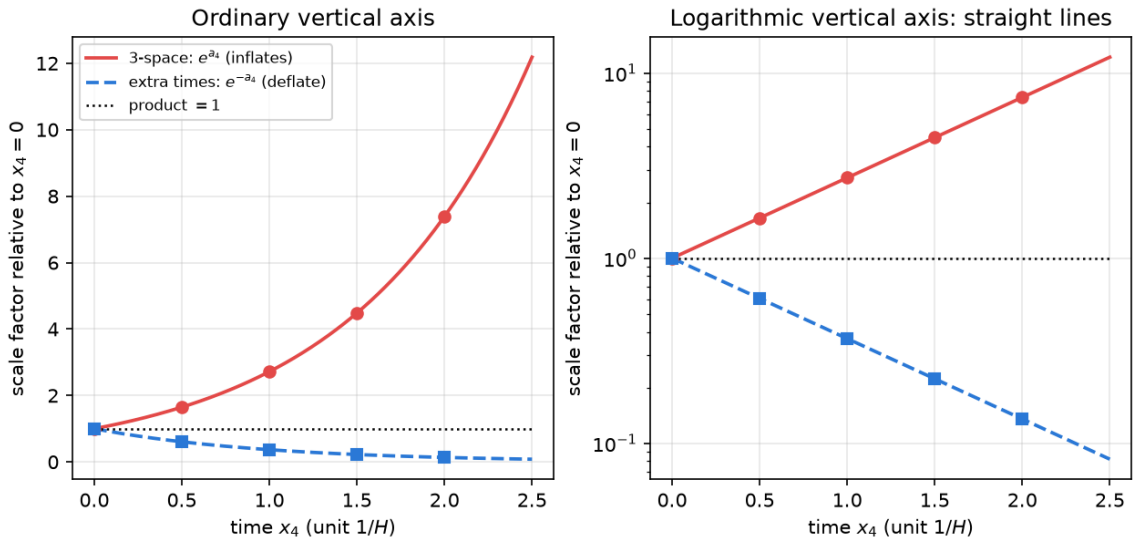

Figure 94: The time dependence of the scale factors along the deflating history $a_4 = AHx_4$, $b_4 = 1$ with $A = 1$, $H = 1$ of the Revision Kohn-Sham work: 3-space e^{a_4} (solid red), the three extra times e^{-a_4} (dashed blue) and their product (dotted black), on an ordinary (left) and a logarithmic (right) vertical axis; horizontal axis the time x_4 in units of $1/H$, vertical axis the factor by which lengths have grown since $x_4 = 0$ (a pure number). Dots and squares mark the five slices $a_4 = 0, 0.5, 1, 1.5, 2$. 3-space grows by $e^2 = 7.39$ and the extra times shrink to $e^{-2} = 0.135$; on the logarithmic axis both are straight lines with slopes $+1$ and -1, the product stays 1. (Notebook 03a, figure 3.)

Out [14]:

```
Figure 03a.3 saved as Revision/textbook/figures/03a_3_history_scale_factors.png
RESULT 3-space growth factor at the last slice a4 = 2 = 7.389056
```

```
RESULT extra-time shrink factor at the last slice a4 = 2 = 0.135335
PASS at every slice the two factors multiply to 1
```

Text cell:

```
## 11. The hidden direction: the functions of $z$

The scale factors also depend on the hidden direction through $z$. The six
transverse directions share the warp factor $W = \sin^{1/6} z$ (their metric
entries share its square $W^2 = \sin^{1/3} z$), and the hidden direction itself
has the scale factor $\cot z$ (its metric entry is $\cot^2 z$). The next cell draws
these four functions across the patch: the first two on an ordinary axis (they lie
between 0 and 1 and reach 1 at the patch end), the last two on a logarithmic axis
(they grow without bound at the tip and vanish at the patch end).
```

In [15]:

```
z_line = np.linspace(0.005, np.pi / 2 - 0.005, 600)
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
axes[0].plot(z_line, np.sin(z_line) ** (1 / 6), color=RED, lw=1.8,
             label="warp factor $W = \\sin^{1/6} z$")
axes[0].plot(z_line, np.sin(z_line) ** (1 / 3), color=ORANGE, lw=1.8, ls="--",
             label="$W^2 = \\sin^{1/3} z$ (in the entries of $g$)")
axes[0].set_ylim(0.0, 1.05)
axes[0].set_title("The warp of the six transverse directions")
axes[1].plot(z_line, 1 / np.tan(z_line), color=AQUA, lw=1.8,
             label="$\\cot z$ (scale factor of $x_0$)")
axes[1].plot(z_line, 1 / np.tan(z_line) ** 2, color=VIOLET, lw=1.8, ls="--",
             label="$\\cot^2 z$ (entry $g_{00}$)")
axes[1].set_yscale("log")
axes[1].set_title("The hidden direction (logarithmic axis)")
for ax in axes:
    ax.set_xlabel("$z = 6 H x_0$ (radians)")
    ax.set_ylabel("value (no unit)")
    ax.legend(fontsize=8)
save_figure(fig, "hidden_direction",
            "The functions of the hidden coordinate $z = 6Hx_0$ in the author's "
            "metric, across the patch $0 < z < \\pi/2$; horizontal axes $z$ in "
            "radians, vertical axes pure numbers. Left: the warp factor "
            "$W = \\sin^{1/6} z$ shared by the six transverse scale factors (solid) "
            "and its square $W^2 = \\sin^{1/3} z$, which stands in the six "
            "transverse metric entries (dashed), rise from $0$ at the tip to $1$ at "
            "the patch end. Right, on a logarithmic axis: the scale factor "
            "$\\cot z$ of the hidden direction (solid) and its square "
            "$g_{00} = \\cot^2 z$ (dashed) "
            "grow without bound towards the tip and fall to $0$ at the patch end.")
check(abs(np.sin(np.pi / 2) ** (1 / 6) - 1.0) < 1e-15,
      "the warp factor equals 1 at the patch end z = pi/2")
```

Out [15]:

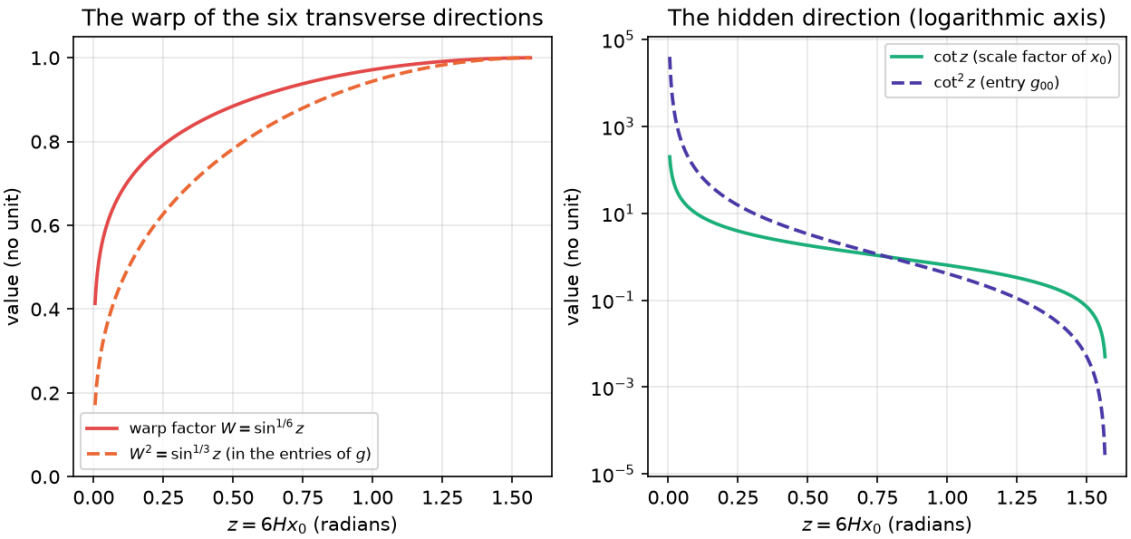

Figure 95: The functions of the hidden coordinate $z = 6Hx_0$ in the author's metric, across the patch $0 < z < \pi/2$; horizontal axes z in radians, vertical axes pure numbers. Left: the warp factor $W = \sin^{1/6} z$ shared by the six transverse scale factors (solid) and its square $W^2 = \sin^{1/3} z$, which stands in the six transverse metric entries (dashed), rise from 0 at the tip to 1 at the patch end. Right, on a logarithmic axis: the scale factor $\cot z$ of the hidden direction (solid) and its square $g_{00} = \cot^2 z$ (dashed) grow without bound towards the tip and fall to 0 at the patch end. (Notebook 03a, figure 4.)

```
Figure 03a.4 saved as Revision/textbook/figures/03a_4_hidden_direction.png
PASS the warp factor equals 1 at the patch end z = pi/2
```

Text cell:

```
The next cell shows how the two effects combine. It draws, over the plane of the
time $x_4$ (along the history $a_4 = x_4$, $b_4 = 1$) and the hidden coordinate $z$,
the
logarithm to base 10 of the scale factor of 3-space, $\log_{10}(e^{a_4}\sin^{1/6}z)$
(left), and of an extra time, $\log_{10}(e^{-a_4}\sin^{1/6}z)$ (right). The value $0$
(grey) means the factor $1$; red means larger, blue smaller.
```

In [16]:

```
x4_grid, z_grid2 = np.meshgrid(np.linspace(0.0, 2.5, 251),
                               np.linspace(0.01, np.pi / 2, 200))
a4_grid = A_value * H_value * x4_grid
space_map = np.log10(np.exp(a4_grid) * np.sin(z_grid2) ** (1 / 6))
extra_map = np.log10(np.exp(-a4_grid) * np.sin(z_grid2) ** (1 / 6))
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for ax, data, title in zip(axes, [space_map, extra_map],
                           ["3-space: $e^{a_4}\\sin^{1/6}z$",
                            "extra time: $e^{-a_4}\\sin^{1/6}z$"]):
    mesh = ax.pcolormesh(x4_grid, z_grid2, data, cmap=DIVERGING, vmin=-1.3,
```

```
                        vmax=1.3, shading="auto")
    ax.contour(x4_grid, z_grid2, data, levels=[0.0], colors="black",
               linewidths=1.0)
    ax.set_title(title)
    ax.set_xlabel("time $x_4$ (unit $1/H$)")
    ax.grid(False)
axes[0].set_ylabel("$z = 6 H x_0$ (radians)")
fig.colorbar(mesh, ax=axes, label="$\\log_{10}$ of the scale factor")
save_figure(fig, "scale_factor_maps",
            "Maps of the logarithm to base 10 of the scale factor of 3-space "
            "$e^{a_4}\\sin^{1/6}z$ (left) and of an extra time "
            "$e^{-a_4}\\sin^{1/6}z$ (right) along the deflating history $a_4 = x_4$, "
            "$b_4 = 1$; "
            "horizontal axes the time $x_4$ in units of $1/H$, vertical axes "
            "$z = 6Hx_0$ in radians, colour the value (red above $0$: lengths larger "
            "than the coordinate interval, blue below $0$: smaller, grey and the "
            "black line: factor $1$). As time goes on, 3-space grows everywhere "
            "(left, red spreads), the extra times shrink everywhere (right, blue "
            "deepens), and near the tip $z \\to 0$ both factors are small.")
check(np.all(np.diff(space_map, axis=1) > 0) and np.all(np.diff(extra_map, axis=1) < 0),
      "along the history 3-space grows and the extra times shrink at every z")
```

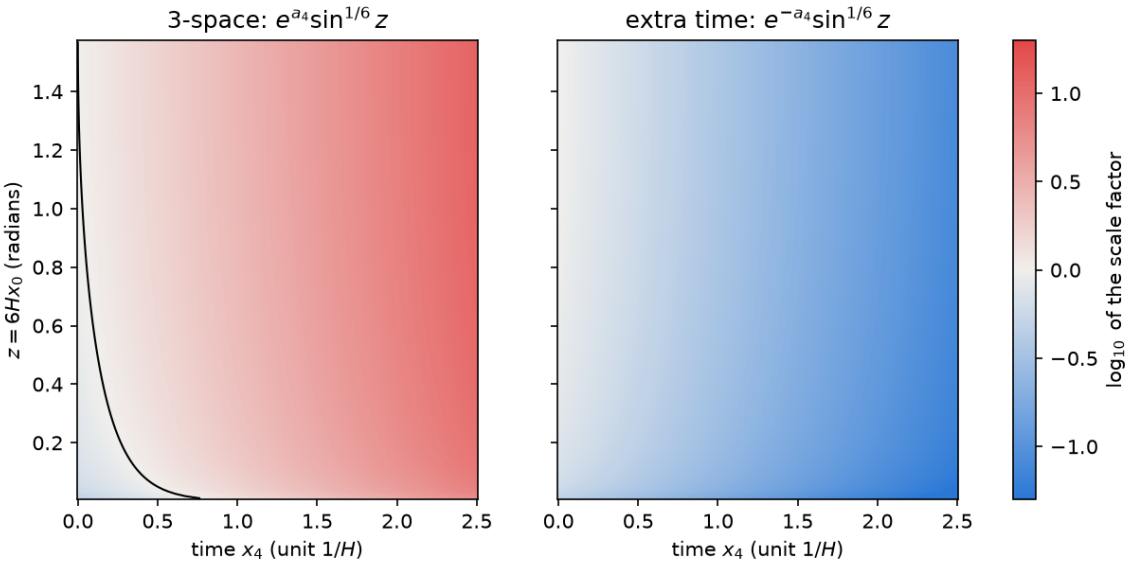

Figure 96: Maps of the logarithm to base 10 of the scale factor of 3-space $e^{a_4}\sin^{1/6} z$ (left) and of an extra time $e^{-a_4}\sin^{1/6} z$ (right) along the deflating history $a_4 = x_4$, $b_4 = 1$; horizontal axes the time x_4 in units of $1/H$, vertical axes $z = 6Hx_0$ in radians, colour the value (red above 0: lengths larger than the coordinate interval, blue below 0: smaller, grey and the black line: factor 1). As time goes on, 3-space grows everywhere (left, red spreads), the extra times shrink everywhere (right, blue deepens), and near the tip $z \to 0$ both factors are small. (Notebook 03a, figure 5.)

Out [16]:

```
Figure 03a.5 saved as Revision/textbook/figures/03a_5_scale_factor_maps.png
PASS along the history 3-space grows and the extra times shrink at every z
```

Text cell:

```
## 12. Proper volumes: 3-space inflates, the extra times deflate, the 7-volume stays

Take a box with coordinate edges $\Delta x_1 = \Delta x_2 = \Delta x_3 = 1$. Its
proper 3-volume is the product of the three scale factors,
$V_3 = (e^{a_4}\sin^{1/6}z)^3 = e^{3a_4}\sin^{1/2}z$. The same box in the three extra
times has the proper 3-volume $V_t = b_4^{3/2} e^{-3a_4}\sin^{1/2}z$, and
$V_3 V_t = b_4^{3/2} \sin z$. The slice $x_4 = \mathrm{const}$ is 7-dimensional
($x_0, x_1, x_2, x_3, x_5, x_6, x_7$); its volume factor is the product of the seven
scale factors other than $h_4 = 1$, which is $b_4^{3/2} \sin z \cot z =
b_4^{3/2} \cos z$: it changes with $x_4$ only through $b_4$, and not at all when $b_4$
is constant. The next cell computes these products and, at $b_4 = 1$, by an integral
over the whole patch $0 < x_0 < \pi/(12H)$, the proper 7-volume of the patch per unit
coordinate volume of the six transverse directions:
$\int_0^{\pi/(12H)} \cos(6Hx_0)\, dx_0 = \bigl[\sin(6Hx_0)/(6H)\bigr]_0^{\pi/(12H)}
= 1/(6H)$ (for a constant $b_4$ it is $b_4^{3/2}/(6H)$).
```

In [17]:

```
V3 = sp.simplify(h[1] * h[2] * h[3])  # proper 3-volume of a unit coordinate box
Vt = sp.simplify(h[5] * h[6] * h[7])  # the same in the three extra times
# the 7-volume: every scale factor except h[4] = 1 of the time x4
V7 = sp.simplify(h[0] * h[1] * h[2] * h[3] * h[5] * h[6] * h[7])
say(f"V3 = {plain(V3)},  Vt = {plain(Vt)},  V7 density = {plain(V7)}")
check(sp.simplify(V7 - b4 ** sp.Rational(3, 2) * sp.cos(6 * H * x0)) == 0
      and sp.simplify(sp.diff(V7 / b4 ** sp.Rational(3, 2), x4)) == 0,
      "the 7-volume density is b4^(3/2) cos z and changes with x4 only through b4")
patch_volume = sp.integrate(sp.cos(6 * H * x0), (x0, 0, sp.pi / (12 * H)))
say(f"proper 7-volume of the patch per unit transverse coordinate volume = "
    f"{patch_volume}")
check(sp.simplify(patch_volume - 1 / (6 * H)) == 0,
      "the whole patch has the proper 7-volume 1/(6H) per unit coordinate volume")
```

Out [17]:

```
V3 = exp(3*a4v)*sqrt(sin(z)),  Vt = b4v**(3/2)*exp(-3*a4v)*sqrt(sin(z)),  V7 density =
    b4v**(3/2)*cos(z)
PASS the 7-volume density is b4^(3/2) cos z and changes with x4 only through b4
proper 7-volume of the patch per unit transverse coordinate volume = 1/(6*H)
PASS the whole patch has the proper 7-volume 1/(6H) per unit coordinate volume
```

Text cell:

```
The next cell draws these volumes along the history at the fixed point
$z = \pi/3$ (where $\sin z = 0.866$ and $\cos z = 0.5$) with $b_4 = 1$, on a
logarithmic axis: $V_3$
grows like $e^{3x_4}$, $V_t$ shrinks like $e^{-3x_4}$, their product $\sin z$ and the
7-volume density $\cos z$ are constant.
```

In [18]:

```
z_fixed = np.pi / 3
v3 = np.exp(3 * a4_line) * np.sin(z_fixed) ** 0.5
vt = np.exp(-3 * a4_line) * np.sin(z_fixed) ** 0.5
fig, ax = plt.subplots()
ax.plot(x4_line, v3, color=RED, lw=1.8, label="$V_3 = e^{3a_4}\\sin^{1/2}z$")
ax.plot(x4_line, vt, color=BLUE, lw=1.8, ls="--",
        label="$V_t = e^{-3a_4}\\sin^{1/2}z$")
ax.plot(x4_line, v3 * vt, color=ORANGE, lw=1.8, ls="-.",
        label="$V_3 V_t = \\sin z$")
ax.plot(x4_line, np.full_like(x4_line, np.cos(z_fixed)), color="black", lw=1.2,
        ls=":", label="7-volume density $\\cos z$")
ax.set_yscale("log")
ax.set_xlabel("time $x_4$ (unit $1/H$), history $a_4 = x_4$, $b_4 = 1$")
ax.set_ylabel("proper volume per unit coordinate volume")
ax.set_title("Proper volumes at $z = \\pi/3$")
ax.legend(fontsize=8)
save_figure(fig, "proper_volumes",
            "Proper volumes per unit coordinate volume at the fixed hidden position "
            "$z = \\pi/3$ along the history $a_4 = x_4$, $b_4 = 1$, on a "
            "logarithmic vertical "
            "axis; horizontal axis the time $x_4$ in units of $1/H$. The proper "
            "3-volume of 3-space $V_3$ (solid red) grows like $e^{3x_4}$, the proper "
            "3-volume of the three extra times $V_t$ (dashed blue) shrinks like "
            "$e^{-3x_4}$, and both their product $\\sin z = 0.866$ (dash-dotted) and "
            "the density $\\cos z = 0.5$ of the proper 7-volume (dotted) stay "
            "constant: the inflation of 3-space is exactly compensated by the "
            "deflation of the extra times.")
check(np.max(np.abs(v3 * vt - np.sin(z_fixed))) < 1e-12,
      "V3 times Vt stays equal to sin z along the history")
```

Out [18]:

```
Figure 03a.6 saved as Revision/textbook/figures/03a_6_proper_volumes.png
PASS V3 times Vt stays equal to sin z along the history
```

Text cell:

```
## 13. The hidden coordinate $y$ and the warped form of the metric

The entry $\cot^2 z$ makes the hidden direction awkward. The Revision Kohn-Sham work
uses the coordinate $y = \ln(\sin z)/(6H)$, which measures proper distance along
$x_0$. Line by line: the chain rule gives
$dy/dx_0 = \frac{1}{6H}\,\frac{\cos z}{\sin z}\, 6H = \cot z$, so
$dy^2 = \cot^2 z\, dx_0^2 = g_{00}\, dx_0^2$. Next, $6Hy = \ln \sin z$ gives
$\sin z = e^{6Hy}$ ($e$ to the power of both sides), so the warp factor is
$W = \sin^{1/6} z = e^{Hy}$ and its square is $W^2 = \sin^{1/3} z = e^{2Hy}$. Hence
the **warped form**

$$ds^2 = dy^2 - dx_4^2 + W^2\bigl[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2)
```

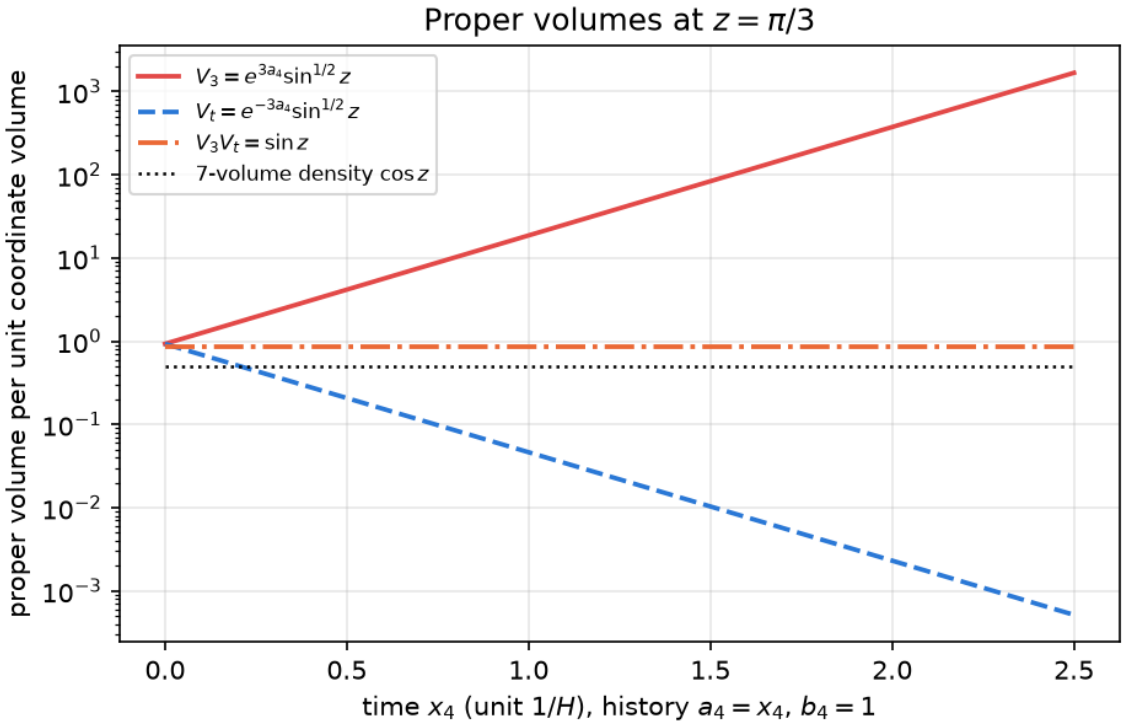

Figure 97: Proper volumes per unit coordinate volume at the fixed hidden position $z = \pi/3$ along the history $a_4 = x_4$, $b_4 = 1$, on a logarithmic vertical axis; horizontal axis the time x_4 in units of $1/H$. The proper 3-volume of 3-space V_3 (solid red) grows like e^{3x_4}, the proper 3-volume of the three extra times V_t (dashed blue) shrinks like e^{-3x_4}, and both their product $\sin z = 0.866$ (dash-dotted) and the density $\cos z = 0.5$ of the proper 7-volume (dotted) stay constant: the inflation of 3-space is exactly compensated by the deflation of the extra times. (Notebook 03a, figure 6.)

```
- b_4 e^{-2a_4}(dx_5^2 + dx_6^2 + dx_7^2)\bigr], \qquad W = e^{Hy}.$$

As $z$ runs from $0$ to $\pi/2$, $y$ runs from $-\infty$ (the tip) to $0$ (the patch
end). In the coordinates $(y, x_1, \dots, x_7)$ the scale factor of the hidden
direction is $1$, so the volume factor is the product of the six transverse scale
factors, $b_4^{3/2} W^6 = b_4^{3/2} e^{6Hy} = b_4^{3/2} \sin z$. The Kohn-Sham record
works at $b_4 = 1$, where these are $W^6 = e^{6Hy} = \sin z$.

The next cell prints four statements of the record
‘Revision/kohn_sham/ks-theory.json‘ (read in section 10): the definition of $y$, the
line element, the warp factor and the volume factor. Then it checks them with sympy;
the checks of the warp factor and of the volume factor also require that the record
still writes them as ‘W(y) = e^{Hy}‘ and as a text that starts with
‘e^{6Hy} (= cos z‘.
To write the metric in the coordinate $y$ it uses the inverse relation
$x_0 = \arcsin(e^{6Hy})/(6H)$: the hidden entry becomes $g_{00}\,(dx_0/dy)^2$
(because $dx_0 = (dx_0/dy)\,dy$), and the seven entries $g_{11}, \dots, g_{77}$ only
need $\sin z = e^{6Hy}$ put in. The record's line element is the case $b_4 = 1$; the
cell checks the warped form for every $b_4$ and compares with the record at $b_4 = 1$.
```

In [19]:

```
geometry = ks_theory["geometry"]  # the statements of the record about the geometry
for key in ("hiddenCoordinate", "lineElement", "warp", "sqrtDetG"):
    say(f"record, {key}: {geometry[key]}")
y_of_x0 = sp.log(sp.sin(6 * H * x0)) / (6 * H)  # y = ln(sin z)/(6H)
dy_dx0 = sp.diff(y_of_x0, x0)  # the chain rule, done by sympy
say(f"dy/dx0 = {plain(dy_dx0)}")
record_check(LEAD_EMT, "ks_coordinate_jacobian")  # stops if not passed
check(sp.simplify(dy_dx0 - sp.cot(6 * H * x0)) == 0,
      "dy/dx0 = cot z",
      record=f"{LEAD_EMT}, check ks_coordinate_jacobian")
check(sp.simplify(dy_dx0 ** 2 - g[0, 0]) == 0,
      "dy^2 = g00 dx0^2: y measures proper distance along x0")
check(sp.simplify(sp.exp(H * y_of_x0) - sixth) == 0,
      "the warp factor W = sin(z)^(1/6) equals e^(H y)")
y = sp.symbols("y", real=True)  # the hidden coordinate as a symbol of its own
x0_of_y = sp.asin(sp.exp(6 * H * y)) / (6 * H)  # the inverse: sin z = e^(6 H y)
dx0_dy = sp.diff(x0_of_y, y)  # dx0/dy
# the metric in y, x1, ..., x7: the hidden entry first, as in the author's order
g_y = sp.diag(g[0, 0].subs(x0, x0_of_y) * dx0_dy ** 2,
              *[g[k, k].subs(x0, x0_of_y) for k in range(1, 8)])
W = sp.exp(H * y)  # the warp factor in the coordinate y
warped = sp.diag(*([1] + [W ** 2 * sp.exp(2 * a4)] * 3 + [-1]
                   + [-W ** 2 * b4 * sp.exp(-2 * a4)] * 3))  # the warped form
for k in (0, 1, 4, 5):
    say(f"  in the coordinate y: g[{k}, {k}] = {plain(g_y[k, k])}")
check((g_y - warped).applyfunc(sp.simplify) == sp.zeros(8, 8)
      and geometry["warp"] == "W(y) = e^{Hy}",
      "in the coordinate y the metric is the warped form with W = e^(H y)",
      record="Revision/kohn_sham/ks-theory.json, geometry.lineElement and "
             "geometry.warp")
check(sp.simplify(g_y.det() - b4 ** 3 * sp.exp(12 * H * y)) == 0
```

```
      and sp.simplify(g_y.det().subs(b4, 1) - sp.exp(12 * H * y)) == 0
      and geometry["sqrtDetG"].startswith("e^{6Hy} (= cos z"),
      "in the coordinate y, sqrt|det g| is b4^(3/2) e^(6 H y); at b4 = 1 it is "
      "e^(6 H y) = sin z",
      record="Revision/kohn_sham/ks-theory.json, geometry.sqrtDetG")
distance_to_end = -y_of_x0  # proper distance from the point to the patch end
check(sp.limit(distance_to_end, x0, 0, "+") == sp.oo,
      "the tip z -> 0 lies at an infinite proper distance (y -> minus infinity)")
```

Out [19]:

```
record, hiddenCoordinate: y = ln(sin z)/(6H) in (-inf, 0], sin z = e^{6Hy}, dy = cot z
    dx0; the patch end z = pi/2 is y = 0, the tip z -> 0 is y -> -inf
record, lineElement: ds^2 = e^{2Hy}[e^{2a4(x4)}(dx1^2+dx2^2+dx3^2) -
    e^{-2a4(x4)}(dx5^2+dx6^2+dx7^2)] - dx4^2 + dy^2
record, warp: W(y) = e^{Hy}
record, sqrtDetG: e^{6Hy} (= cos z in the author's chart x0..x7); independent of a4: the
    7-volume is constant
dy/dx0 = cos(z)/sin(z)
PASS dy/dx0 = cot z
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check ks_coordinate_jacobian
PASS dy^2 = g00 dx0^2: y measures proper distance along x0
PASS the warp factor W = sin(z)^(1/6) equals e^(H y)
  in the coordinate y: g[0, 0] = 1
  in the coordinate y: g[1, 1] = exp(2*a4v)*exp(2*H*y)
  in the coordinate y: g[4, 4] = -1
  in the coordinate y: g[5, 5] = -b4v*exp(-2*a4v)*exp(2*H*y)
PASS in the coordinate y the metric is the warped form with W = e^(H y)
     reproduces Revision/kohn_sham/ks-theory.json, geometry.lineElement and geometry.warp
PASS in the coordinate y, sqrt|det g| is b4^(3/2) e^(6 H y); at b4 = 1 it is e^(6 H y) =
    sin z
     reproduces Revision/kohn_sham/ks-theory.json, geometry.sqrtDetG
PASS the tip z -> 0 lies at an infinite proper distance (y -> minus infinity)
```

Text cell:

```
The Revision Kohn-Sham solver keeps only the part $-L \le y \le 0$ of the hidden
direction, with the tip cut-off $L = 3/H$ read in section 10. How much of the
7-volume does the cut-off remove? In $y$ the 7-volume density is $b_4^{3/2} e^{6Hy}$,
and the constant factor $b_4^{3/2}$ cancels in a fraction, so the
fraction of the 7-volume beyond the cut-off is
$\int_{-\infty}^{-L} e^{6Hy} dy \big/ \int_{-\infty}^{0} e^{6Hy} dy = e^{-6HL}$,
which is $e^{-18} \approx 1.5 \times 10^{-8}$ for $L = 3/H$. The cut-off lies at the
tiny angle $z = \arcsin e^{-18} \approx 1.5 \times 10^{-8}$. The next cell computes
the fraction with sympy and draws $y$ as a function of $z$ (left; the $z$ axis is
logarithmic, so that the region near the tip is visible: there
$y \approx \ln(z)/(6H)$ is a straight line) and the warp factor $W = e^{Hy}$, its
square $W^2 = e^{2Hy}$ and the volume density $W^6 = e^{6Hy}$ as functions of $y$
(right).
```

In [20]:

```
L_exact = sp.nsimplify(L_tip)  # the number 3.0 of the record as the exact integer 3
fraction = sp.integrate(sp.exp(6 * H * y), (y, -sp.oo, -L_exact / H)) \
    / sp.integrate(sp.exp(6 * H * y), (y, -sp.oo, 0))  # y: the symbol made above
fraction = sp.simplify(fraction.subs(H, 1))
report("fraction of the 7-volume beyond the tip cut-off y = -3", sp.sstr(fraction),
       f"= {float(fraction):.4e}")
z_cut = float(sp.asin(sp.exp(-18)))  # the angle z of the cut-off, sin z = e^(-18)
report("angle z of the tip cut-off", f"{z_cut:.4e}", "radians")
z_line = np.logspace(-10, np.log10(np.pi / 2), 800)  # 1e-10 ... pi/2, evenly in log
y_line = np.log(np.sin(z_line)) / 6.0  # with H = 1
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
axes[0].plot(z_line, y_line, color=VIOLET, lw=1.8, label="$y(z)$")
axes[0].axhline(-L_tip, color="black", lw=1.0, ls="--",
                label=f"tip cut-off $y = -L = -{L_tip:g}$")
axes[0].plot([z_cut], [-L_tip], "o", color="black", ms=6)
axes[0].set_xscale("log")
axes[0].set_xlabel("$z = 6 H x_0$ (radians, logarithmic axis)")
axes[0].set_ylabel("$y = \\ln(\\sin z)/(6H)$ (unit $1/H$)")
axes[0].set_title("The hidden coordinate $y$")
axes[0].legend(fontsize=8)
y_axis = np.linspace(-4.0, 0.0, 400)
axes[1].plot(y_axis, np.exp(y_axis), color=RED, lw=1.8,
             label="warp factor $W = e^{Hy} = \\sin^{1/6} z$")
axes[1].plot(y_axis, np.exp(2 * y_axis), color=ORANGE, lw=1.8, ls="-.",
             label="$W^2 = e^{2Hy} = \\sin^{1/3} z$")
axes[1].plot(y_axis, np.exp(6 * y_axis), color=AQUA, lw=1.8, ls="--",
             label="7-volume density $W^6 = e^{6Hy} = \\sin z$")
axes[1].axvspan(-4.0, -L_tip, color="grey", alpha=0.25,
                label="removed by the tip cut-off")
axes[1].set_xlabel("$y$ (unit $1/H$)")
axes[1].set_ylabel("value (no unit)")
axes[1].set_title("Warp and volume density in $y$")
axes[1].legend(fontsize=8, loc="upper left")
save_figure(fig, "hidden_coordinate_y",
            "The hidden coordinate $y = \\ln(\\sin z)/(6H)$ with $H = 1$. Left: $y$ "
            "versus $z = 6Hx_0$ in radians on a logarithmic axis from $10^{-10}$ to "
            "$\\pi/2$; $y$ is $0$ at the patch end $z = \\pi/2$ and falls without "
            "bound towards the tip $z \\to 0$, along the straight line "
            "$y \\approx \\ln(z)/6$; the dashed line and the dot mark the tip cut-off "
            "$y = -3$ of the Revision Kohn-Sham solver, at $z = 1.5 \\times "
            "10^{-8}$. Right: the warp factor $W = e^{Hy}$ (solid), its square "
            "$W^2 = e^{2Hy}$ (dash-dotted) and the density $W^6 = e^{6Hy}$ of the "
            "proper 7-volume (dashed) versus $y$ in units of $1/H$; the grey band "
            "beyond "
            "the cut-off holds only the fraction $e^{-18} = 1.5 \\times 10^{-8}$ of "
            "the 7-volume of the patch.")
check(fraction == sp.exp(-18), "the cut-off removes exactly the fraction e^(-18)")
```

Out [20]:

```
RESULT fraction of the 7-volume beyond the tip cut-off y = -3 = exp(-18) = 1.5230e-08
RESULT angle z of the tip cut-off = 1.5230e-08 radians
```

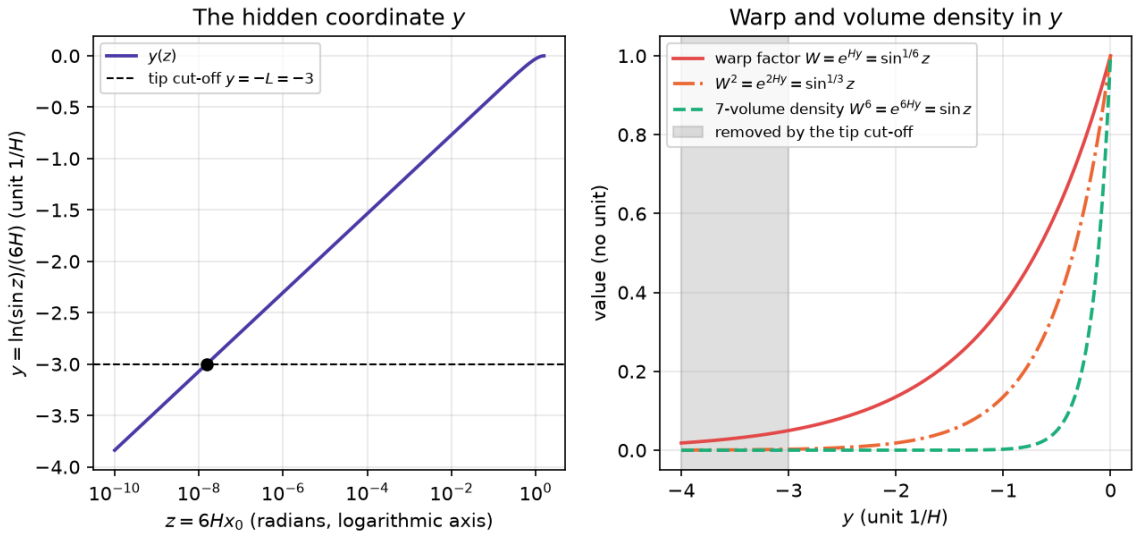

Figure 98: The hidden coordinate $y = \ln(\sin z)/(6H)$ with $H = 1$. Left: y versus $z = 6Hx_0$ in radians on a logarithmic axis from 10^{-10} to $\pi/2$; y is 0 at the patch end $z = \pi/2$ and falls without bound towards the tip $z \to 0$, along the straight line $y \approx \ln(z)/6$; the dashed line and the dot mark the tip cut-off $y = -3$ of the Revision Kohn-Sham solver, at $z = 1.5 \times 10^{-8}$. Right: the warp factor $W = e^{Hy}$ (solid), its square $W^2 = e^{2Hy}$ (dash-dotted) and the density $W^6 = e^{6Hy}$ of the proper 7-volume (dashed) versus y in units of $1/H$; the grey band beyond the cut-off holds only the fraction $e^{-18} = 1.5 \times 10^{-8}$ of the 7-volume of the patch. (Notebook 03a, figure 7.)

Out [20]:

```
Figure 03a.7 saved as Revision/textbook/figures/03a_7_hidden_coordinate_y.png
PASS the cut-off removes exactly the fraction e^(-18)
```

Text cell:

```
## 14. The last check

The last cell checks that the seven figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [21]:

```
figure_names = ["metric_heat_map", "diagonal_entries", "history_scale_factors",
                "hidden_direction", "scale_factor_maps", "proper_volumes",
                "hidden_coordinate_y"]
files = [output_file(f"{FIGURE_FOLDER}/03a_{k}_{n}.png")
         for k, n in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all seven figure files exist")
all_checks_passed()
```

Out [21]:

```
PASS all seven figure files exist
ALL 29 CHECKS PASSED (notebook 03a)
```

Text cell:

```
## 15. What this notebook showed

- The metric was read **exactly as the author typed it**, and the text agrees with
  the formula and with the Revision record of the Rust program ‘lovelock_gkd‘
  (all 8 diagonal entries; the 56 other entries are zero).
- $\det g = b_4^3 \cos^2 z$ and $\sqrt{|\det g|} = b_4^{3/2} \cos z$, without $a_4$:
  PROVED here by exact algebra (and in the Revision record).
- The signature is (4,4) at every point of the patch: $x_0, x_1, x_2, x_3$ are
  space-like, $x_4, x_5, x_6, x_7$ time-like.
- The diagonal vielbein $h = (\cot z, e^{a_4}\sin^{1/6}z\ (\times 3), 1,
  b_4^{1/2} e^{-a_4}\sin^{1/6}z\ (\times 3))$ reproduces the metric.
- 3-space expands with the rate $a_4'$ and the three extra times **deflate** with the
  rate $-a_4' + b_4'/(2b_4)$; along the prescribed history $a_4 = x_4$, $b_4 = 1$
  ($A = 1$, $H = 1$) 3-space
  lengths grow by $e^2 = 7.39$ and extra-time durations shrink to $e^{-2} = 0.135$
  between the first and the last slice.
- The proper 3-volume of 3-space grows like $e^{3a_4}$, that of the extra times
  shrinks like $b_4^{3/2} e^{-3a_4}$, and the 7-volume density $b_4^{3/2} \cos z$ is
  constant in time when $b_4$ is; at $b_4 = 1$ the whole patch has the 7-volume
  $1/(6H)$ per unit transverse coordinate volume.
- In the hidden coordinate $y = \ln(\sin z)/(6H)$ the metric has the warped form
  of the Revision record, with the warp factor $W = e^{Hy}$ and the volume factor
  $W^6 = e^{6Hy}$; the tip is at infinite proper distance, and the tip
  cut-off $y = -3/H$ of the Kohn-Sham solver removes only the fraction $e^{-18}$ of
  the 7-volume.
- ASSUMED: the history $a_4 = A H x_4$, $b_4 = 1$ is a prescribed background (the
  Revision
  record says so in its own words), used here only to draw the time dependence; the
  field equations of $a_4$ come later.
```

3.13 Line-by-line walk-through of Notebook 03a

The notebook has 21 code cells, In [1] to In [21]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted and then explained. A line that starts with **#** is a **comment**, which Python skips; it is there for the reader. In the quoted code a line **...** stands for the remaining lines of a long figure caption; every caption is printed in full under its figure in Section 3.12, and a paragraph "What Figure 03a.k shows" after the code says what to look for.

In [1], the set-up cell. Its first part is the complete run instructions of Section 3.11 again, as comment lines, so that the notebook file carries its own instructions. The code starts below the line that announces THE SET-UP. The set-up code is the same in every notebook of the book except for one line (the notebook's name); we explain it here once, and the walk-throughs of Notebooks 03c, 03b and 03d refer back to this explanation.

```
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ================================================================================
```

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json` reads and writes the text format JSON, in which the Revision record stores its results; `os` gives access to the computer's environment; `textwrap` breaks long text into lines; `from pathlib import Path` takes the single name `Path` out of the module `pathlib`. A `Path` is the address of a file or a folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "03a"  # this notebook: chapter 03, example a
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell. A **variable** is a name for a value: the last line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"03a"`. It is the only line of the set-up code that differs between notebooks.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. The text in triple quotes under the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd()` is the folder in which the notebook runs and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with all of them (the star unpacks one list into another). The `for` loop takes these folders one after the other; the operator `/` joins a folder and a name into a longer path; `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If none does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
```

```
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

`REPO` is the repository folder. `os.environ` holds the **environment variables** of the running program (named texts that the computer hands to it); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and otherwise the default, the repository folder written as a string. When you run the notebook the variable is not set, so the notebook writes into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` gives the full path of a repository file, for reading a Revision record. `output_file("Revision/...")` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder that will hold it (and any missing folder above it) and does nothing if it exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`say` prints a text in lines of at most 89 characters; `textwrap.fill` breaks it at blanks, and every line after the first starts with four blanks. `matplotlib.rcdefaults()` returns to matplotlib's built-in settings, so that a personal settings file cannot change the figures. `plt.rcParams.update({...})` then sets, for every figure of the notebook, the size (7.0 by 4.2 inches), the size of the letters (10 points) and a faint grid (30 per cent opaque). The braces make a **dictionary**: pairs `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/03a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far; so the figures are numbered 1, 2, 3, and a cell that is run twice keeps its numbers. The file name joins the notebook id, the number and the name, for example `03a_1_metric_heat_map.png`. `fig.savefig` writes the PNG file with 150 dots per inch, without an empty margin and without the program's name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory. The caption is stored, and the whole dictionary of captions is written into the captions file (`json.dumps` turns it into JSON text with sorted keys). `display(Image(...))` shows the saved picture below the cell; its `metadata` tells the book's tools which file it is. The last line prints where the figure was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection in square brackets). `check` is the function behind every check of the book. `condition` is either `True` or `False`; if it is false, `raise AssertionError(...)` stops the notebook with an error that names the check. Otherwise the name is appended to `PASSED` and the line PASS followed by the name is printed. `record=None` makes the third argument optional; a check that reproduces a Revision record passes the record's file and check name, and a second line names them.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a RESULT line; the expression in round brackets adds a blank and the unit when a unit is given and nothing otherwise. `all_checks_passed` prints the last line of the notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one output line of In [1].

In [2], every result in one piece. While a cell runs, Jupyter receives what it prints in pieces, about one piece every 0.2 seconds; where one piece ends depends on the speed of the computer. The book's tools read a PASS line together with the line under it that names the Revision record, and they read the two correctly only when both arrive in the same piece. This cell makes sure that they do.

```
import contextlib  # redirect_stdout: send printed text into a buffer
import io  # StringIO: a text buffer in memory
import sys  # sys.stdout: the place where printed text goes
```

Three modules of Python: `contextlib` holds `redirect_stdout`, which sends everything that `print` writes to another place; `io` holds `StringIO`, a text buffer in memory; `sys.stdout` is the normal destination of printed text.

```
def in_one_piece(helper):
    """A function that does what helper does, with all the lines that helper prints
    written in one piece."""
    def helper_in_one_piece(*arguments, **options):
        collected = io.StringIO()  # an empty text buffer
        with contextlib.redirect_stdout(collected):  # print() writes into it
            helper(*arguments, **options)  # a failing check stops here, as before
        sys.stdout.write(collected.getvalue())  # all the lines with one write
    return helper_in_one_piece
```

`in_one_piece` takes a function `helper` and returns a new function, `helper_in_one_piece`, defined inside it. The stars in `*arguments` and `**options` collect whatever values the new function is called with and pass all of them on to `helper` unchanged. Inside, `collected` is an empty buffer; the `with` block redirects printing into it while `helper` runs, so the PASS line and the record line land in the buffer; if the check fails, the `AssertionError` stops everything here exactly as before. After the block, `sys.stdout.write` writes the whole buffer at once, which reaches Jupyter as one piece.

```
check = in_one_piece(check)  # a PASS line and its record line stay together
report = in_one_piece(report)  # a long RESULT line and its second line, too
say("From now on check and report print each of their results in one piece.")
```

The names `check` and `report` now stand for the wrapped functions. They print the same lines as before; only the timing of the printing changes. The last line prints the output of In [2].

In [3], the author's text.

```
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
from sympy.parsing.sympy_parser import (implicit_multiplication, parse_expr,
                                        standard_transformations)
```

numpy (short name `np`) works with **arrays**, lists of numbers on which arithmetic acts entry by entry; sympy (short name `sp`) does exact algebra with symbols. `parse_expr` turns a text such as `"2*x + 1"` into a sympy expression; `standard_transformations` and `implicit_multiplication` are reading rules for it (explained at In [5]).

```
def record_check(report_file, check_name, detail_part=""):
    checks = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
    checks = checks["checks"]  # a dictionary or a list, depending on the report
```

Several PASS lines of this notebook say that a result reproduces a named check of a Revision report. A **report** is a JSON file in which a program of the Revision record lists the checks it ran, each with its name, its verdict and a short detail text. The function `record_check` makes sure that such a check really exists and passed; without it, a PASS line could name a check that has since been renamed, removed or failed. Its three **parameters** (the names in brackets, which receive the values given in a call) are the report file, the name of the check and an optional piece of text `detail_part`; `=""` makes the empty text its default, used when a call gives only two values. (Its docstring, a description in triple quotes under the `def` line, is not printed here; In [1] explained docstrings.) The first line reads the report into a dictionary, as for the record above; the second keeps only its entry `"checks"`. The reports of the Revision record store their checks in one of two ways, and the function accepts both.

```
    if isinstance(checks, dict):  # {name: {"passed": true, "detail": ...}}
        entry = checks.get(check_name, {})
        passed = entry.get("passed") is True
```

`isinstance(checks, dict)` is true when the checks form a dictionary whose keys are the check names (the Rust program writes them so). Then `checks.get(check_name, {})` is the entry of

that name, or an empty dictionary `{}` if there is none, and the check counts as passed only if the entry's `"passed"` is exactly `True` (`is True` compares with the value `True` itself). For a missing entry, `entry.get("passed")` gives `None`, which is not `True`.

```
    else:  # [{"name": ..., "verdict": "PASS", "detail": ...}, ...]
        entry = next((e for e in checks if e.get("name") == check_name), {})
        passed = entry.get("verdict") == "PASS"
```

Otherwise the checks form a list of dictionaries, each with its own `"name"` (the Python programs and the lead checks write them so). `(e for e in checks if ...)` runs through the list and keeps the entries with the wanted name; `next(..., {})` takes the first of them, or `{}` if there is none. The check counts as passed only if its `"verdict"` is the text `"PASS"`.

```
    if not passed or detail_part not in entry.get("detail", ""):
        raise AssertionError(f"record check failed: {report_file} does not list "
                             f"{check_name} as passed")
    return True
```

If the check did not pass, or its detail text does not contain `detail_part`, `raise AssertionError(...)` stops the notebook with an error message that names the report and the check (every text contains the empty text, so without a `detail_part` only the verdict matters). Otherwise the function returns `True`. It prints nothing, so it adds no PASS line and does not change the number of checks; it only guarantees that each record line printed under a PASS line names a check that exists and passed. Notebook 03b uses the same function.

```
CURVATURE_RECORD = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE_RECORD).read_text(encoding="utf-8"))
# the metric exactly as the author typed it, then a note in brackets
author_text, note = record["metricAsGiven"].split("}} ", 1)
author_text += "}}"  # the two closing brackets that split removed
say(f"The metric as the author wrote it ({len(author_text)} characters):")
for line in author_text.replace("},", "},\n").split("\n"):  # one row per line
    print(line)
say("The note of the record: " + note)
```

The first line names the record file. The second reads the file as text (`read_text`) and turns the JSON text into a Python dictionary (`json.loads`). `record["metricAsGiven"]` is the author's text of the metric followed by a note in brackets. `split("}} ", 1)` cuts this text once, at the first `}}` followed by a blank: `author_text` gets the matrix (without its two closing braces, which the split removed and `+= "}}"` puts back) and `note` the note. The matrix is 359 characters long, as the output shows. The loop prints it with one row of the matrix per line: `replace("},", "},\n")` puts a line break (`\n`) after every row and `split("\n")` cuts the text at the breaks. The output is the matrix of Section 3.5, in the author's order $x_0, x_1, \ldots, x_7$, with the two functions `a4[x4]` and `b4[x4]`; the last line prints the note, which says that $b_4 = 1$ gives the metric used before 2026-10-14.

In [4], the symbols.

```
x0, x1, x2, x3, x4, x5, x6, x7 = sp.symbols("x0:8", real=True)  # 8 coordinates
COORDS = [x0, x1, x2, x3, x4, x5, x6, x7]
```

```
H = sp.symbols("H", positive=True)  # the constant of the author, H > 0
a4 = sp.Function("a4")(x4)  # the metric function a4: an unknown function of x4
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0 of x4
a4p, a4v = sp.symbols("a4p a4v", real=True)  # a4p = a4 prime, a4v = value of a4
b4p = sp.symbols("b4p", real=True)  # b4p = b4 prime
b4v = sp.symbols("b4v", positive=True)  # b4v = value of b4
z = sp.symbols("z", positive=True)  # z = 6 H x0, for printing
```

`sp.symbols("x0:8", real=True)` makes the eight symbols `x0`, ..., `x7` (the range `0:8` means 0 to 7), declared real; `COORDS` is their list, in the author's order, so that `COORDS[k]` is x_k. `H` is declared positive, which lets sympy simplify, for example, $\sqrt{H^2} = H$. `sp.Function("a4")(x4)` is an unknown function of `x4`: sympy can differentiate it (giving a_4') without knowing it. `b4` is the second metric function, declared positive (`positive=True`), so that sympy may write $\sqrt{b_4^3} = b_4^{3/2}$. `a4p`, `a4v`, `b4p`, `b4v` and `z` are plain symbols used only for printing.

```
def plain(expression):
    """Write the derivatives of a4 and b4 as a4p and b4p, their values as a4v and
    b4v, and 6 H x0 as z."""
    expression = expression.subs(sp.Derivative(a4, x4), a4p).subs(a4, a4v)
    expression = expression.subs(sp.Derivative(b4, x4), b4p).subs(b4, b4v)
    return expression.subs(x0, z / (6 * H))  # then 6*H*x0 becomes z


say("Symbols: x0 ... x7, H > 0, the functions a4(x4) and b4(x4), and a4p, a4v, b4p, "
    "b4v, z for printing.")
```

`plain` rewrites an expression for printing: `.subs(old, new)` replaces `old` by `new`, first the derivative a_4' by `a4p`, then the function value by `a4v`, the same for b_4 (`b4p`, `b4v`); then x_0 by $z/(6H)$, so that $6Hx_0$ becomes z. The derivative is replaced before the value: otherwise the value inside the derivative would be replaced first and the derivative would no longer be recognised. The last line prints the output of In [4].

In [5], the text becomes a matrix.

```
text = author_text
text = text.replace("exp^", "E^")  # e to the power ...
text = text.replace("Sin[6 H x0]", "sin(6 H x0)")  # Mathematica [ ] -> Python ( )
text = text.replace("Cot[6 H x0]", "cot(6 H x0)")
text = text.replace("a4[x4]", "a4")  # the value of the function a4 at x4
text = text.replace("b4[x4]", "b4")  # the value of the function b4 at x4
text = text.replace("{", "[").replace("}", "]")  # Mathematica lists -> Python lists
text = text.replace("^", "**")  # Mathematica power ^ -> Python power **
```

Eight replacements translate Mathematica into Python, one rule each: `exp^` becomes `E^` (sympy's `E` is the number $e = 2.718\ldots$; the record's text already writes `E^`, and the rule is kept so that a text written with `exp^` is read the same way); the square brackets of the sine and the cotangent become round brackets; `a4[x4]` becomes `a4` and `b4[x4]` becomes `b4`; the braces of lists become square brackets (a Python list); the power sign `^` becomes `**`. The order matters: the square brackets of the functions are translated before any other bracket.

```
names = {"E": sp.E, "a4": a4, "b4": b4, "H": H, "x0": x0, "sin": sp.sin,
         "cot": sp.cot}
rows = parse_expr(text, local_dict=names,
                  transformations=standard_transformations + (implicit_multiplication,))
g = sp.Matrix(rows)  # the metric of the author as an exact 8 x 8 sympy matrix
```

The dictionary `names` tells `parse_expr` what each name in the text means: `a4` and `b4` are the functions $a_4(x_4)$ and $b_4(x_4)$, `H` and `x0` the symbols, `sin` and `cot` sympy's functions. `implicit_multiplication` reads a blank between two factors as a product, as Mathematica does (`6 H x0` is $6 \cdot H \cdot x_0$). The result `rows` is a list of eight lists, and `sp.Matrix(rows)` makes the exact 8×8 matrix `g`.

```
say(f"The matrix has {g.rows} rows and {g.cols} columns. Its diagonal entries:")
for k in range(8):
    say(f"  g[x{k}, x{k}] = {g[k, k]}")
off_diagonal = [g[i, j] for i in range(8) for j in range(8) if i != j]
check(all(entry == 0 for entry in off_diagonal),
      "the 56 entries off the diagonal are zero: the metric is diagonal")
```

The loop prints the eight diagonal entries; `range(8)` runs over $k = 0, \ldots, 7$ and `g[k, k]` is the entry in row k and column k, printed with the author's name `x{k}` (row k belongs to x_k). `off_diagonal` is a **list comprehension**, a list built by a loop inside square brackets: the $64-8 = 56$ entries with $i \neq j$. `all(...)` is true when every one of them is zero: the first PASS line. The output shows the diagonal of Section 3.5, written by sympy (`exp(2*a4(x4))*sin(6*H*x0)**(1/3)` is $e^{2a_4} \sin^{1/3}(6Hx_0)$, and `-b4(x4)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)` is $-b_4 e^{-2a_4} \sin^{1/3}(6Hx_0)$).

In [6], two independent checks of the reading.

```
warp = sp.sin(6 * H * x0) ** sp.Rational(1, 3)  # sin(z)^(1/3), Rational: exact 1/3
typed = sp.diag(sp.cot(6 * H * x0) ** 2,  # the author's order x0, x1, ..., x7
                sp.exp(2 * a4) * warp, sp.exp(2 * a4) * warp, sp.exp(2 * a4) * warp,
                -1,
                -b4 * sp.exp(-2 * a4) * warp, -b4 * sp.exp(-2 * a4) * warp,
                -b4 * sp.exp(-2 * a4) * warp)
difference = (g - typed).applyfunc(sp.simplify)  # simplify every entry
check(difference == sp.zeros(8, 8),
      "the text of the author equals the metric typed by hand from the formula")
```

`sp.Rational(1, 3)` is the exact fraction 1/3 (the Python number `1/3` would be rounded). `warp` is $\sin^{1/3} z$, the square W^2 of the warp factor. `sp.diag(...)` builds the diagonal matrix of the formula of Section 3.5, typed by hand in the author's order: first the hidden entry $\cot^2 z$, then the three 3-space entries, the time entry -1 and the three extra-time entries with the factor b_4. `applyfunc(sp.simplify)` simplifies every entry of the difference, and the check requires the 8×8 zero matrix `sp.zeros(8, 8)`: the reading of the text agrees with the formula.

```
def from_mathematica(entry_text):
    """A record entry in Mathematica notation, as a sympy expression."""
    t = entry_text.replace("Sin[6*H*x0]", "sin(6*H*x0)")
    t = t.replace("Cot[6*H*x0]", "cot(6*H*x0)")
```

```
    t = t.replace("a4[x4]", "a4").replace("b4[x4]", "b4").replace("^", "**")
    if "[" in t or "]" in t:  # a piece of notation that was not translated
        raise ValueError(f"cannot translate {entry_text}")
    return parse_expr(t, local_dict=names)
```

The record also stores the diagonal entries one by one (`metricDiagonal`), in Mathematica notation with explicit `*` signs, for example `E^(2*a4[x4])*Sin[6*H*x0]^(1/3)` and `-E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*b4[x4]`. `from_mathematica` translates such an entry with the same rules. If a square bracket is left, some notation was not translated, and the function stops with a `ValueError` instead of misreading it silently.

```
agree = [sp.simplify(g[k, k] - from_mathematica(entry)) == 0
         for k, entry in enumerate(record["metricDiagonal"])]
check(len(agree) == 8 and all(agree),
      "the 8 diagonal entries equal those of the Rust program lovelock_gkd",
      record=f"{CURVATURE_RECORD}, metricDiagonal")
```

`enumerate` hands out each record entry with its position k. For each, the difference between our entry and the translated record entry is simplified and compared with 0. The check requires eight agreements and prints the PASS line with the record line under it.

In [7], the determinant and the volume factor (Section 3.6).

```
det_g = g.det()  # the product of the diagonal entries; sympy cancels exp(6 a4)
say(f"det g = {plain(det_g)}")
record_check("Revision/gkd_lovelock/results/python-lovelock-report.json",
             "sqrt_abs_det_g", "det g = b4^3 cos^2(6 H x0) exactly")  # stops if not
check(sp.simplify(det_g - b4 ** 3 * sp.cos(6 * H * x0) ** 2) == 0,
      "det g = b4^3 cos(z)^2: the function a4 drops out",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check sqrt_abs_det_g")
```

`g.det()` is the determinant. It prints as `b4v**3*sin(z)**2*cot(z)**2`, the fourth line of the derivation of Section 3.6. `record_check` (In [3]) then confirms that the independent verification of the record lists its check `sqrt_abs_det_g` as passed and that the detail of that check contains the text `det g = b4^3 cos^2(6 H x0) exactly`; otherwise the notebook stops here. The check confirms that our determinant equals $b_4^3 \cos^2 z$. The function a_4 is gone; b_4 stays, with the power 3, once for each extra time. The two strings of the `record=` argument are written next to each other, which Python joins into one string.

```
# sqrt|det g| on the patch, where cos z > 0 and b4 > 0
volume_factor = b4 ** sp.Rational(3, 2) * sp.cos(6 * H * x0)
check(sp.simplify(from_mathematica(record["sqrtAbsDetG"]) - volume_factor) == 0,
      "sqrt|det g| = b4^(3/2) sin(z) cot(z) = b4^(3/2) cos(z)",
      record=f"{CURVATURE_RECORD}, sqrtAbsDetG")
```

The record writes the volume factor as `Sin[6*H*x0]*Cot[6*H*x0]*b4[x4]^(3/2)`; translated and simplified, it equals $b_4^{3/2} \cos z$ (`sp.Rational(3, 2)` is the exact power 3/2; sympy may take the square root of b_4^3 because `b4` is declared positive).

```
# numbers: sqrt(|det g|) at 7 points of the patch with b4 = 1, compared with cos z
z_points = np.linspace(0.1, 1.5, 7)
det_numbers = np.array([float(det_g.subs(b4, 1).subs({H: 1, x0: zp / 6}))
                        for zp in z_points])
check(np.max(np.abs(np.sqrt(np.abs(det_numbers)) - np.cos(z_points))) < 1e-14,
      "at b4 = 1, sqrt|det g| equals cos z at 7 points of the patch")
```

A numerical check of the same fact for $b_4 = 1$. `np.linspace(0.1, 1.5, 7)` gives seven equally spaced values of z from 0.1 to 1.5. For each, the exact determinant is evaluated with $b_4 = 1$ (`subs(b4, 1)`), $H = 1$ and $x_0 = z/6$ (a dictionary in `subs` replaces several symbols at once) and turned into a floating-point number with `float`. The check requires $\sqrt{|\det g|}$ to differ from $\cos z$ by less than 10^{-14} at all seven points.

In [8], the signature (Section 3.5).

```
ETA = np.array([1, 1, 1, 1, -1, -1, -1, -1])  # the flat metric diag(+,+,+,+,-,-,-,-)
# the entries as functions of z, a4v and b4v
diagonal_z = [plain(g[k, k]) for k in range(8)]
functions = [sp.lambdify((z, a4v, b4v), entry, "numpy") for entry in diagonal_z]
```

`ETA` holds the eight signs of η, in the author's order: $+$ for x_0, x_1, x_2, x_3 and $-$ for $x_4, \ldots, x_7$. `diagonal_z` holds the eight entries written with `z`, `a4v` and `b4v`. `sp.lambdify((z, a4v, b4v), entry, "numpy")` turns an exact expression into a fast numerical Python function of the three arguments that works on whole numpy arrays.

```
def diagonal_values(z_values, a4_value, b4_value=1.0):
    """The 8 diagonal entries at the points z_values (one row per entry)."""
    return np.array([np.broadcast_to(f(z_values, a4_value, b4_value), z_values.shape)
                     for f in functions], dtype=float)
```

`diagonal_values` evaluates the eight functions at an array of values of z for one value of a_4 and one of b_4 (`b4_value=1.0` is a **default value**: a call that leaves out the third argument gets $b_4 = 1$) and returns a table with eight rows, row k for g_{kk}. The entry -1 does not depend on z, so its function returns one number; `np.broadcast_to` repeats it so that every row has the same length as `z_values`.

```
z_grid = np.linspace(0.0, np.pi / 2, 402)[1:-1]  # 400 points strictly inside
patterns = set()
for b4_value in (0.5, 1.0, 2.0):
    for a4_value in np.linspace(-3.0, 3.0, 13):
        # +1 or -1 for each entry
        signs = np.sign(diagonal_values(z_grid, a4_value, b4_value))
        patterns.update(tuple(int(s) for s in column) for column in signs.T)
say(f"sign patterns found on the grid: {sorted(patterns)}")
check(patterns == {tuple(int(s) for s in ETA)},
      "on the whole grid the signs are (+,+,+,+,-,-,-,-): signature (4,4)")
```

`np.linspace(0.0, np.pi / 2, 402)` gives 402 points from 0 to $\pi/2$; `[1:-1]` drops the first and the last, leaving 400 points strictly inside the patch. A `set` is a collection without repetitions. For

three values of b_4 (0.5, 1, 2) and 13 values of a_4 from -3 to 3, `np.sign` replaces each entry by its sign; `signs.T` is the table turned on its side, so that each `column` holds the eight signs at one point; `tuple(...)` turns them into a fixed sequence that a set can store. At the end the set holds every sign pattern met anywhere on the grid. It is printed (sorted) and must be the single pattern of η. (The grid includes negative values of a_4 only to test the sign argument of Section 3.5, which holds for every real a_4 and every $b_4 > 0$.)

```
step_x1 = diagonal_values(np.array([0.7]), 0.5)[1, 0]  # g11: step along x1
step_x5 = diagonal_values(np.array([0.7]), 0.5)[5, 0]  # g55: step along x5
say(f"at z = 0.7, a4 = 0.5, b4 = 1: g11 = {step_x1:.6f} (space-like), "
    f"g55 = {step_x5:.6f} (time-like)")
check(step_x1 > 0 > step_x5, "a step along x1 is space-like, along x5 time-like")
```

At the single point $z = 0.7$, $a_4 = 0.5$, $b_4 = 1$ (the default): row 1 is g_{11} and row 5 is g_{55}; the second index `0` picks the only column. The format `:.6f` prints six digits after the decimal point: $g_{11} = 2.347679$ and $g_{55} = -0.317724$. A unit step along x_1 has the squared length $g_{11} > 0$ (space-like), along x_5 the squared length $g_{55} < 0$ (time-like). Python reads `step_x1 > 0 > step_x5` as both comparisons at once.

In [9], the colours and the heat map of the metric.

```
from matplotlib.colors import LinearSegmentedColormap

BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
MAGENTA, GREEN, VIOLET, RED = "#e87ba4", "#008300", "#4a3aa7", "#e34948"
GREY = "#f0efec"  # the neutral middle of the two-sided colour scale
DIVERGING = LinearSegmentedColormap.from_list("blue_grey_red", [BLUE, GREY, RED])
LABELS = [f"$x_{k}$" for k in range(8)]
```

Each colour is written as a **hex code**: `#` and three pairs of hexadecimal digits for the amounts of red, green and blue. The palette is chosen so that readers with a colour-vision deficiency can tell the colours apart; every figure also uses line styles and labels, so that it can be read in black and white. `DIVERGING` is a colour scale that runs from blue through grey to red; it will show negative numbers blue, zero grey and positive numbers red. `LABELS` holds the eight axis labels `$x_0$` to `$x_7$`, written in the math notation that matplotlib understands.

```
z_point, a4_point = np.pi / 4, 0.5
values = np.zeros((8, 8))
values[np.diag_indices(8)] = diagonal_values(np.array([z_point]), a4_point)[:, 0]
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(values, cmap=DIVERGING, vmin=-3.0, vmax=3.0)
```

At the point $z = \pi/4$, $a_4 = 0.5$ the table `values` starts as an 8×8 table of zeros, and its diagonal (`np.diag_indices(8)`) is filled with the eight entries of the metric. `plt.subplots` makes a figure `fig` with one drawing area `ax`; `ax.imshow` draws the table as coloured squares, with the colour scale running from -3 (blue) to 3 (red).

```
for i in range(8):
    for j in range(8):
```

```
        ax.text(j, i, f"{values[i, j]:.3f}" if i == j else "0", ha="center",
                va="center", fontsize=8)
ax.set_xticks(range(8), LABELS)
ax.set_yticks(range(8), LABELS)
ax.grid(False)  # no grid lines across the squares
ax.set_title("The metric $g_{\\mu\\nu}$ at $z = \\pi/4$, $a_4 = 0.5$, $b_4 = 1$")
fig.colorbar(image, ax=ax, label="value of the entry (no unit)")
```

The double loop writes the value into each square: three decimals on the diagonal, a plain `0` elsewhere (`ha` and `va` centre the text). The two tick lines label the rows and the columns x_0 to x_7. In a Python string `\\` stands for one backslash, so the title reads $g_{\mu\nu}$ in math notation. `fig.colorbar` adds the colour scale with its label.

```
save_figure(fig, "metric_heat_map",
            "The author's metric $g_{\\mu\\nu}$ at the point $z = 6Hx_0 = \\pi/4$ "
            "with $a_4 = 0.5$ and $b_4 = 1$, drawn as a heat map: row $\\mu$ and "
            "column $\\nu$ run "
            "over the coordinates $x_0$ to $x_7$, red squares are positive entries, "
...
check(abs(values[1, 1] - np.e * np.sin(np.pi / 4) ** (1 / 3)) < 1e-12,
      "g11 at z = pi/4, a4 = 0.5 equals e times (sin pi/4)^(1/3)")
```

`save_figure` saves and shows the figure with its caption (printed in full under Figure 03a.1). The check compares the entry `values[1, 1]`, which is g_{11}, with $e^{2 \cdot 0.5}(\sin \pi/4)^{1/3} = e\,(\sin \pi/4)^{1/3}$.

What Figure 03a.1 shows. An 8×8 table of coloured squares at $b_4 = 1$: only the diagonal is coloured. In the author's order, the top-left square is the hidden one, $\cot^2(\pi/4) = 1$; then come the three red squares of 3-space with 2.422, the time square -1, and the three pale blue squares of the extra times with -0.328. Four positive and four negative entries: the signature (4,4) at a glance. The 3-space entries are larger than the extra-time entries by the factor $e^{4a_4} = e^2$, because $a_4 = 0.5 > 0$ has already inflated 3-space and deflated the extra times.

In [10], the diagonal entries across the patch.

```
z_line = np.linspace(0.02, np.pi / 2, 600)
entries = diagonal_values(z_line, 0.5)  # at b4 = 1
fig, ax = plt.subplots()
ax.plot(z_line, entries[0], color=AQUA, lw=1.8, label="$g_{00} = \\cot^2 z$")
ax.plot(z_line, entries[1], color=RED, lw=1.8,
        label="$g_{11} = g_{22} = g_{33} = e^{2a_4}\\sin^{1/3}z$")
ax.plot(z_line, entries[4], color=VIOLET, lw=1.8, ls="-.", label="$g_{44} = -1$")
ax.plot(z_line, entries[5], color=BLUE, lw=1.8, ls="--",
        label="$g_{55} = g_{66} = g_{77} = -b_4 e^{-2a_4}\\sin^{1/3}z$")
ax.plot(z_line, np.cos(z_line) ** 2, color="black", lw=1.2, ls=":",
        label="$\\det g = \\cos^2 z$")
```

600 values of z from 0.02 to $\pi/2$ and the eight entries at $a_4 = 0.5$ and $b_4 = 1$ (the default of `diagonal_values`). `ax.plot(x, y, ...)` draws a curve; `lw` is the line width, `ls` the line style (a dash and a dot mean dash-dotted, two dashes mean dashed, a colon means dotted) and `label` its text in the legend. Equal entries are drawn once: row 0 for the hidden direction, row 1 for

3-space, row 4 for the time, row 5 for the extra times; the black dotted curve is $\det g = \cos^2 z$, the determinant at $b_4 = 1$.

```
ax.axhline(0.0, color="black", lw=0.8)
ax.axvline(np.pi / 2, color="grey", lw=0.8, ls=":")
ax.text(np.pi / 2 - 0.02, 3.3, "patch end", ha="right", fontsize=9)
ax.text(0.05, 3.3, "towards the tip", fontsize=9)
ax.set_ylim(-3.4, 3.6)  # room below the curves for the legend
ax.set_xlabel("$z = 6 H x_0$ (radians)")
ax.set_ylabel("entry of the metric (no unit)")
ax.set_title("The diagonal entries across the patch ($a_4 = 0.5$, $b_4 = 1$)")
ax.legend(fontsize=8, loc="lower center", ncol=2)
```

A horizontal line at 0 (`axhline`), a vertical dotted line at the patch end (`axvline`), two labels written into the drawing (`ax.text(x, y, text)`), the range of the vertical axis, the two axis labels, the title and the legend in two columns at the bottom.

```
save_figure(fig, "diagonal_entries",
            "The diagonal entries of the author's metric as functions of "
...
check(np.all(entries[0] > 0) and np.cos(np.pi / 2) ** 2 < 1e-30,
      "g00 > 0 inside the patch and det g = 0 at the patch end")
```

The figure is saved with its caption. The check: $g_{00} > 0$ at all 600 points, and $\cos^2(\pi/2)$ is zero up to rounding (the computer's $\cos(\pi/2)$ is about 6×10^{-17}, whose square is below 10^{-30}).

What Figure 03a.2 shows. The red 3-space curve stays above zero and the blue extra-time curve and the violet time line stay below it across the whole patch: the signature never changes. The aqua curve $\cot^2 z$ rises steeply towards the tip and reaches 0 at the patch end, where the black dotted determinant $\cos^2 z$ also reaches 0: there the coordinate x_0 fails (Section 3.9). The 3-space entry falls towards the tip because the warp $\sin^{1/3} z$ goes to 0 there.

In [11], the vielbein (Section 3.7).

```
sixth = sp.sin(6 * H * x0) ** sp.Rational(1, 6)  # sin(z)^(1/6)
# the author's order x0, x1, ..., x7
h = [sp.cot(6 * H * x0)] + [sp.exp(a4) * sixth] * 3 + [sp.Integer(1)] \
    + [sp.sqrt(b4) * sp.exp(-a4) * sixth] * 3
```

`sixth` is the warp factor $W = \sin^{1/6} z$. The list `h` holds the eight scale factors in the author's order: first $h_0 = \cot z$, then $h_1 = h_2 = h_3 = e^{a_4} W$, $h_4 = 1$ and $h_5 = h_6 = h_7 = b_4^{1/2} e^{-a_4} W$. `[...] * 3` repeats a one-element list three times, `+` joins lists, and a backslash at the end of a line continues the statement on the next line. `sp.Integer(1)` is the exact number 1.

```
for k in range(8):
    say(f"  h[x{k}] = {plain(h[k])}")
LEAD_EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
record_check(LEAD_EMT, "vielbein_reproduces_metric")  # stops if not passed
check(all(sp.simplify(int(ETA[k]) * h[k] ** 2 - g[k, k]) == 0 for k in range(8)),
      "eta times h squared reproduces every diagonal entry of the metric",
      record=f"{LEAD_EMT}, check vielbein_reproduces_metric")
```

The loop prints the eight scale factors (the output lists them; `exp(a4v)*sin(z)**(1/6)` is $e^{a_4}\sin^{1/6} z$ and `sqrt(b4v)*exp(-a4v)*sin(z)**(1/6)` is $b_4^{1/2}e^{-a_4}\sin^{1/6} z$). `LEAD_EMT` names the report of one of the lead checks (Section 3.7 says what the lead checks are), and `record_check` (In [3]) stops the notebook unless that report lists its check `vielbein_reproduces_metric` as passed. The check is $\eta_{kk}h_k^2 = g_{kk}$ for all eight k, the diagonal vielbein reproducing the metric, as in the lead's check.

In [12], the expansion rates.

```
rates = [sp.simplify(sp.diff(sp.log(h[k]), x4)) for k in range(8)]
for k in range(8):
    say(f"  expansion rate of x{k}: {plain(rates[k])}")
a4_prime, b4_prime = sp.Derivative(a4, x4), sp.Derivative(b4, x4)
expected = ([0] + [a4_prime] * 3 + [0]
            + [-a4_prime + b4_prime / (2 * b4)] * 3)  # the order x0, x1, ..., x7
check([sp.simplify(r - e) == 0 for r, e in zip(rates, expected)] == [True] * 8,
      "3-space expands with rate a4p, the extra times shrink with rate "
      "-a4p + b4p/(2 b4)")
```

`sp.diff(sp.log(h[k]), x4)` is $\partial_4 \ln h_k$, the rate of Section 3.7. The output prints `0` for x_0 and x_4, `a4p` for 3-space and `-a4p + b4p/(2*b4v)` for the extra times: the rate $\alpha_t = -a_4' + b_4'/(2b_4)$. `zip` pairs each rate with the expected one in the list `expected` (in the order $x_0, \ldots, x_7$: 0, three times a_4', 0, three times α_t), and the check requires all eight differences to vanish. For a constant b_4 the extra-time rate is $-a_4'$.

```
transverse = sp.simplify(h[1] * h[2] * h[3] * h[5] * h[6] * h[7])
say(f"product of the six transverse scale factors = {plain(transverse)}")
check(sp.simplify(transverse - b4 ** sp.Rational(3, 2) * sp.sin(6 * H * x0)) == 0,
      "the six transverse scale factors multiply to b4^(3/2) sin z, without a4")
```

The product of the six transverse scale factors (positions 1, 2, 3 and 5, 6, 7) prints as `b4v**(3/2)*sin(z)`: what 3-space gains through e^{a_4}, the extra times lose through e^{-a_4}; only the factor $b_4^{3/2}$ of the extra times remains.

In [13], the history of the Revision Kohn-Sham work (Section 3.8).

```
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
physics = parameters["physics"]
H_value, A_value = physics["H"], physics["historyA"]
slices = physics["slicesA4"]  # the values of a4 at which the solver works
L_tip = physics["L_tipCutoff"]  # the tip cut-off, in units of 1/H
say(f"H = {H_value}, A = {A_value}, slices a4 = {slices}, tip cut-off L = {L_tip}")
check(H_value == 1.0 and A_value == 1.0 and slices == [0.0, 0.5, 1.0, 1.5, 2.0],
      "the history of the record is a4 = x4 (A = 1, H = 1) with slices 0 to 2")
```

The record of the Revision Kohn-Sham solver is read, and from its part `physics` the constant H, the slope A, the five slices and the tip cut-off L. A line `a, b = c, d` gives two names at once. The output prints $H = 1$, $A = 1$, the slices $0, 0.5, 1, 1.5, 2$ and $L = 3$, and the check confirms them.

```
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks_theory = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
status = ks_theory["adiabaticity"]["historyStatus"]  # the record's own statement
sentences = status.split(". ")  # the statement, cut into its sentences
say("status of the history (record): " + sentences[0] + ".")
say(sentences[-1])  # the last sentence already ends with a full stop
check(status.startswith("PRESCRIBED BACKGROUND") and "not solved for" in status,
      "the record states that the history is a prescribed background",
      record=f"{KS_THEORY}, adiabaticity.historyStatus")
```

The theory record of the Kohn-Sham work is read and its statement about the status of the history is cut into sentences at every full stop followed by a blank. The first and the last sentence are printed (`sentences[-1]` is the last element of a list). The check requires the statement to start with PRESCRIBED BACKGROUND and to contain "not solved for": the record itself labels the history an assumption.

In [14], the scale factors along the history.

```
x4_line = np.linspace(0.0, 2.5, 251)
a4_line = A_value * H_value * x4_line  # the history a4 = A H x4
slice_x4 = np.array(slices) / (A_value * H_value)  # the times of the slices
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
```

251 times from 0 to 2.5 (in units of $1/H$), the history $a_4 = AHx_4$ at those times (with $b_4 = 1$, so that the extra-time scale factor relative to $x_4 = 0$ is e^{-a_4}), and the times $x_4 = a_4/(AH)$ of the five slices. `plt.subplots(1, 2, ...)` makes a figure with two drawing areas side by side, `axes[0]` and `axes[1]`.

```
for ax, scale in zip(axes, ["linear", "log"]):
    ax.plot(x4_line, np.exp(a4_line), color=RED, lw=1.8,
            label="3-space: $e^{a_4}$ (inflates)")
    ax.plot(x4_line, np.exp(-a4_line), color=BLUE, lw=1.8, ls="--",
            label="extra times: $e^{-a_4}$ (deflate)")
    ax.plot(x4_line, np.exp(a4_line) * np.exp(-a4_line), color="black", lw=1.2,
            ls=":", label="product $= 1$")
    ax.plot(slice_x4, np.exp(slices), "o", color=RED, ms=6)
    ax.plot(slice_x4, np.exp(-np.array(slices)), "s", color=BLUE, ms=6)
    ax.set_yscale(scale)
    ax.set_xlabel("time $x_4$ (unit $1/H$)")
    ax.set_ylabel("scale factor relative to $x_4 = 0$")
```

The loop draws the same picture twice, once with an ordinary and once with a logarithmic vertical axis (`set_yscale`). In each: e^{a_4} (red), e^{-a_4} (blue dashed), their product (black dotted), and the five slices as dots (`"o"`) and squares (`"s"`) of size `ms=6`.

```
axes[0].set_title("Ordinary vertical axis")
axes[1].set_title("Logarithmic vertical axis: straight lines")
axes[0].legend(fontsize=8)
save_figure(fig, "history_scale_factors",
            "The time dependence of the scale factors along the deflating history "
```

```
...
report("3-space growth factor at the last slice a4 = 2", f"{np.exp(2.0):.6f}")
report("extra-time shrink factor at the last slice a4 = 2", f"{np.exp(-2.0):.6f}")
check(abs(np.exp(slices[-1]) * np.exp(-slices[-1]) - 1.0) < 1e-15,
      "at every slice the two factors multiply to 1")
```

Titles, legend, the saved figure, two RESULT lines ($e^2 = 7.389056$ and $e^{-2} = 0.135335$) and a check that the two factors multiply to 1 at the last slice (and so, since $e^a e^{-a} = 1$ for every a, at every slice).

What Figure 03a.3 shows. Along the history with $b_4 = 1$, on the ordinary axis (left) the red curve of 3-space bends upwards to 7.39 at $x_4 = 2$ and the blue extra-time curve falls towards 0; on the logarithmic axis (right) both are straight lines, rising and falling with slopes $+1$ and -1: that is what exponential growth and exponential deflation look like. The product stays exactly 1. The dots and squares mark the five slices on which the Revision Kohn-Sham states are computed.

In [15], the functions of z.

```
z_line = np.linspace(0.005, np.pi / 2 - 0.005, 600)
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
axes[0].plot(z_line, np.sin(z_line) ** (1 / 6), color=RED, lw=1.8,
             label="warp factor $W = \\sin^{1/6} z$")
axes[0].plot(z_line, np.sin(z_line) ** (1 / 3), color=ORANGE, lw=1.8, ls="--",
             label="$W^2 = \\sin^{1/3} z$ (in the entries of $g$)")
axes[0].set_ylim(0.0, 1.05)
axes[0].set_title("The warp of the six transverse directions")
```

600 values strictly inside the patch. The left panel shows the warp factor $W = \sin^{1/6} z$ and its square $W^2 = \sin^{1/3} z$, which stands in the six transverse metric entries; the vertical axis runs from 0 to 1.05.

```
axes[1].plot(z_line, 1 / np.tan(z_line), color=AQUA, lw=1.8,
             label="$\\cot z$ (scale factor of $x_0$)")
axes[1].plot(z_line, 1 / np.tan(z_line) ** 2, color=VIOLET, lw=1.8, ls="--",
             label="$\\cot^2 z$ (entry $g_{00}$)")
axes[1].set_yscale("log")
axes[1].set_title("The hidden direction (logarithmic axis)")
for ax in axes:
    ax.set_xlabel("$z = 6 H x_0$ (radians)")
    ax.set_ylabel("value (no unit)")
    ax.legend(fontsize=8)
```

The right panel shows $\cot z = 1/\tan z$ and $\cot^2 z$ on a logarithmic axis. The loop gives both panels their axis labels and legends.

```
save_figure(fig, "hidden_direction",
            "The functions of the hidden coordinate $z = 6Hx_0$ in the author's "
...
check(abs(np.sin(np.pi / 2) ** (1 / 6) - 1.0) < 1e-15,
      "the warp factor equals 1 at the patch end z = pi/2")
```

The figure is saved; the check confirms $W = 1$ at the patch end.

What Figure 03a.4 shows. Left: both warp curves rise from 0 at the tip to 1 at the patch end; $W = \sin^{1/6} z$ rises very steeply near the tip (a sixth root of a small number is not small: $(10^{-6})^{1/6} = 0.1$). Right: $\cot z$ and $\cot^2 z$ fall over many powers of ten from the tip to the patch end, where they reach 0 (off the bottom of the logarithmic axis).

In [16], maps of the scale factors over time and z.

```
x4_grid, z_grid2 = np.meshgrid(np.linspace(0.0, 2.5, 251),
                               np.linspace(0.01, np.pi / 2, 200))
a4_grid = A_value * H_value * x4_grid
space_map = np.log10(np.exp(a4_grid) * np.sin(z_grid2) ** (1 / 6))
extra_map = np.log10(np.exp(-a4_grid) * np.sin(z_grid2) ** (1 / 6))
```

`np.meshgrid` makes two tables of the same shape (200 rows for z, 251 columns for x_4) that hold, at every grid point, its x_4 and its z. Along the history $a_4 = AHx_4$, $b_4 = 1$, the two maps hold the logarithm to base 10 of the scale factor of 3-space and of an extra time at every grid point.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for ax, data, title in zip(axes, [space_map, extra_map],
                           ["3-space: $e^{a_4}\\sin^{1/6}z$",
                            "extra time: $e^{-a_4}\\sin^{1/6}z$"]):
    mesh = ax.pcolormesh(x4_grid, z_grid2, data, cmap=DIVERGING, vmin=-1.3,
                         vmax=1.3, shading="auto")
    ax.contour(x4_grid, z_grid2, data, levels=[0.0], colors="black",
               linewidths=1.0)
    ax.set_title(title)
    ax.set_xlabel("time $x_4$ (unit $1/H$)")
    ax.grid(False)
axes[0].set_ylabel("$z = 6 H x_0$ (radians)")
fig.colorbar(mesh, ax=axes, label="$\\log_{10}$ of the scale factor")
```

Two panels that share their vertical axis (`sharey=True`). `pcolormesh` colours every grid cell by its value on the blue-grey-red scale from -1.3 to 1.3; `contour(..., levels=[0.0])` draws the black line on which the value is 0, the factor 1. One colour scale serves both panels.

```
save_figure(fig, "scale_factor_maps",
            "Maps of the logarithm to base 10 of the scale factor of 3-space "
...
check(np.all(np.diff(space_map, axis=1) > 0) and np.all(np.diff(extra_map, axis=1) < 0),
      "along the history 3-space grows and the extra times shrink at every z")
```

`np.diff(..., axis=1)` is the difference between neighbouring columns, that is between successive times. The check: at every z, the 3-space map increases and the extra-time map decreases from each time to the next.

What Figure 03a.5 shows. Left: from left to right the colour turns from grey to deeper and deeper red: 3-space grows at every height z. The black line is the factor 1; left of it, near the tip, the small warp makes the factor smaller than 1 although $a_4 > 0$. Right: the colour deepens to

blue from left to right: the extra times shrink at every z at the same rate, and their scale factor is smallest near the tip, where the small warp makes it smaller still.

In [17], proper volumes (Section 3.6).

```
V3 = sp.simplify(h[1] * h[2] * h[3])  # proper 3-volume of a unit coordinate box
Vt = sp.simplify(h[5] * h[6] * h[7])  # the same in the three extra times
# the 7-volume: every scale factor except h[4] = 1 of the time x4
V7 = sp.simplify(h[0] * h[1] * h[2] * h[3] * h[5] * h[6] * h[7])
say(f"V3 = {plain(V3)},  Vt = {plain(Vt)},  V7 density = {plain(V7)}")
check(sp.simplify(V7 - b4 ** sp.Rational(3, 2) * sp.cos(6 * H * x0)) == 0
      and sp.simplify(sp.diff(V7 / b4 ** sp.Rational(3, 2), x4)) == 0,
      "the 7-volume density is b4^(3/2) cos z and changes with x4 only through b4")
```

The three products of scale factors of Section 3.6. The output: `exp(3*a4v)*sqrt(sin(z))`, `b4v**(3/2)*exp(-3*a4v)*sqrt(sin(z))` and `b4v**(3/2)*cos(z)`. The check: the 7-volume density is $b_4^{3/2}\cos z$, and after division by $b_4^{3/2}$ its derivative with respect to x_4 is zero: the 7-volume changes with time only through b_4, never through a_4.

```
patch_volume = sp.integrate(sp.cos(6 * H * x0), (x0, 0, sp.pi / (12 * H)))
say(f"proper 7-volume of the patch per unit transverse coordinate volume = "
    f"{patch_volume}")
check(sp.simplify(patch_volume - 1 / (6 * H)) == 0,
      "the whole patch has the proper 7-volume 1/(6H) per unit coordinate volume")
```

`sp.integrate(f, (x0, 0, upper))` is the exact definite integral of Section 3.6 of the density $\cos z$, the density at $b_4 = 1$; it prints `1/(6*H)`. For another value of b_4 the result is multiplied by $b_4^{3/2}$, which does not depend on x_0.

In [18], the volumes along the history.

```
z_fixed = np.pi / 3
v3 = np.exp(3 * a4_line) * np.sin(z_fixed) ** 0.5
vt = np.exp(-3 * a4_line) * np.sin(z_fixed) ** 0.5
fig, ax = plt.subplots()
ax.plot(x4_line, v3, color=RED, lw=1.8, label="$V_3 = e^{3a_4}\\sin^{1/2}z$")
ax.plot(x4_line, vt, color=BLUE, lw=1.8, ls="--",
        label="$V_t = e^{-3a_4}\\sin^{1/2}z$")
ax.plot(x4_line, v3 * vt, color=ORANGE, lw=1.8, ls="-.",
        label="$V_3 V_t = \\sin z$")
ax.plot(x4_line, np.full_like(x4_line, np.cos(z_fixed)), color="black", lw=1.2,
        ls=":", label="7-volume density $\\cos z$")
```

At the fixed height $z = \pi/3$ ($\sin z = 0.866$, $\cos z = 0.5$) the numbers V_3 and V_t along the history with $b_4 = 1$, their product, and the constant 7-volume density (`np.full_like` makes an array of the same length filled with one number).

```
ax.set_yscale("log")
ax.set_xlabel("time $x_4$ (unit $1/H$), history $a_4 = x_4$, $b_4 = 1$")
```

```
ax.set_ylabel("proper volume per unit coordinate volume")
ax.set_title("Proper volumes at $z = \\pi/3$")
ax.legend(fontsize=8)
save_figure(fig, "proper_volumes",
            "Proper volumes per unit coordinate volume at the fixed hidden position "
...
check(np.max(np.abs(v3 * vt - np.sin(z_fixed))) < 1e-12,
      "V3 times Vt stays equal to sin z along the history")
```

A logarithmic vertical axis, labels, title, legend, the saved figure and the check that $V_3 V_t = \sin z$ at all 251 times.

What Figure 03a.6 shows. At $b_4 = 1$, on the logarithmic axis V_3 is a straight line rising with slope 3 (it grows like e^{3x_4}) and V_t a straight line falling with slope -3; their product and the 7-volume density are flat lines. The inflation of 3-space is exactly compensated by the deflation of the extra times.

In [19], the hidden coordinate y and the warped form (Section 3.9).

```
geometry = ks_theory["geometry"]  # the statements of the record about the geometry
for key in ("hiddenCoordinate", "lineElement", "warp", "sqrtDetG"):
    say(f"record, {key}: {geometry[key]}")
```

Four statements of the record `ks-theory.json` are printed: the definition of y, the line element in y, the warp factor and the volume factor (the output shows them).

```
y_of_x0 = sp.log(sp.sin(6 * H * x0)) / (6 * H)  # y = ln(sin z)/(6H)
dy_dx0 = sp.diff(y_of_x0, x0)  # the chain rule, done by sympy
say(f"dy/dx0 = {plain(dy_dx0)}")
record_check(LEAD_EMT, "ks_coordinate_jacobian")  # stops if not passed
check(sp.simplify(dy_dx0 - sp.cot(6 * H * x0)) == 0,
      "dy/dx0 = cot z",
      record=f"{LEAD_EMT}, check ks_coordinate_jacobian")
check(sp.simplify(dy_dx0 ** 2 - g[0, 0]) == 0,
      "dy^2 = g00 dx0^2: y measures proper distance along x0")
check(sp.simplify(sp.exp(H * y_of_x0) - sixth) == 0,
      "the warp factor W = sin(z)^(1/6) equals e^(H y)")
```

y as a function of x_0 and its derivative, which prints as `cos(z)/sin(z)`. `record_check` (In [3]) stops the notebook unless the lead's report `LEAD_EMT` (named in In [11]) lists its check `ks_coordinate_jacobian` as passed. Three checks: $dy/dx_0 = \cot z$ (the lead's check), $(dy/dx_0)^2 = g_{00}$, and $e^{Hy} = \sin^{1/6} z$.

```
y = sp.symbols("y", real=True)  # the hidden coordinate as a symbol of its own
x0_of_y = sp.asin(sp.exp(6 * H * y)) / (6 * H)  # the inverse: sin z = e^(6 H y)
dx0_dy = sp.diff(x0_of_y, y)  # dx0/dy
# the metric in y, x1, ..., x7: the hidden entry first, as in the author's order
g_y = sp.diag(g[0, 0].subs(x0, x0_of_y) * dx0_dy ** 2,
              *[g[k, k].subs(x0, x0_of_y) for k in range(1, 8)])
```

To write the metric in the coordinate y, the notebook needs x_0 as a function of y: from $\sin z = e^{6Hy}$, $x_0 = \arcsin(e^{6Hy})/(6H)$ (`sp.asin` is the inverse sine). The seven entries of $x_1, \ldots, x_7$ only need this put in. The hidden entry, the first in the author's order, becomes $g_{00}\,(dx_0/dy)^2$, because $dx_0 = (dx_0/dy)\,dy$ turns $g_{00}\,dx_0^2$ into $g_{00}(dx_0/dy)^2\,dy^2$. The star in `sp.diag(..., *[...])` unpacks the list of the seven entries with $k = 1, \ldots, 7$ into seven separate arguments.

```
W = sp.exp(H * y)  # the warp factor in the coordinate y
warped = sp.diag(*([1] + [W ** 2 * sp.exp(2 * a4)] * 3 + [-1]
                   + [-W ** 2 * b4 * sp.exp(-2 * a4)] * 3))  # the warped form
for k in (0, 1, 4, 5):
    say(f"  in the coordinate y: g[{k}, {k}] = {plain(g_y[k, k])}")
check((g_y - warped).applyfunc(sp.simplify) == sp.zeros(8, 8)
      and geometry["warp"] == "W(y) = e^{Hy}",
      "in the coordinate y the metric is the warped form with W = e^(H y)",
      record="Revision/kohn_sham/ks-theory.json, geometry.lineElement and "
             "geometry.warp")
```

`warped` is the warped form of Section 3.9, with $W = e^{Hy}$: the record's line element, which is written for $b_4 = 1$, with the factor b_4 in the three extra-time entries. Four entries of the transformed metric are printed, g_{00}, g_{11}, g_{44} and g_{55}: 1, $e^{2a_4}e^{2Hy}$, -1 and $-b_4e^{-2a_4}e^{2Hy}$. The check requires the transformed metric to equal the warped form entry by entry, and (after `and`) the record's statement of the warp factor to be still exactly the text `W(y) = e^{Hy}`, so that a change of the record would be caught.

```
check(sp.simplify(g_y.det() - b4 ** 3 * sp.exp(12 * H * y)) == 0
      and sp.simplify(g_y.det().subs(b4, 1) - sp.exp(12 * H * y)) == 0
      and geometry["sqrtDetG"].startswith("e^{6Hy} (= cos z"),
      "in the coordinate y, sqrt|det g| is b4^(3/2) e^(6 H y); at b4 = 1 it is "
      "e^(6 H y) = sin z",
      record="Revision/kohn_sham/ks-theory.json, geometry.sqrtDetG")
distance_to_end = -y_of_x0  # proper distance from the point to the patch end
check(sp.limit(distance_to_end, x0, 0, "+") == sp.oo,
      "the tip z -> 0 lies at an infinite proper distance (y -> minus infinity)")
```

The determinant in y is $b_4^3e^{12Hy}$, so the volume factor is $b_4^{3/2}e^{6Hy}$; at $b_4 = 1$ (`subs(b4, 1)`) the determinant is e^{12Hy}, the case of the record, and the check also requires the record's statement of the volume factor to start with the text `e^{6Hy} (= cos z` (`startswith`). Since y measures proper distance and the patch end is $y = 0$, the proper distance from a point to the patch end is $-y$; `sp.limit(f, x0, 0, "+")` is the limit as x_0 tends to 0 from above, and `sp.oo` is infinity: the tip is infinitely far away.

In [20], the tip cut-off and the coordinate y in pictures.

```
L_exact = sp.nsimplify(L_tip)  # the number 3.0 of the record as the exact integer 3
fraction = sp.integrate(sp.exp(6 * H * y), (y, -sp.oo, -L_exact / H)) \
    / sp.integrate(sp.exp(6 * H * y), (y, -sp.oo, 0))  # y: the symbol made above
fraction = sp.simplify(fraction.subs(H, 1))
report("fraction of the 7-volume beyond the tip cut-off y = -3", sp.sstr(fraction),
       f"= {float(fraction):.4e}")
z_cut = float(sp.asin(sp.exp(-18)))  # the angle z of the cut-off, sin z = e^(-18)
report("angle z of the tip cut-off", f"{z_cut:.4e}", "radians")
```

`sp.nsimplify(3.0)` gives the exact integer 3. The fraction of Section 3.9 is computed as a ratio of two exact integrals and simplified with $H = 1$ (the factor $b_4^{3/2}$ of the density does not depend on y and cancels in the ratio, so the fraction is the same for every b_4); it is `exp(-18)`, and the RESULT line prints it also as 1.5230×10^{-8} (the format `.4e` writes four digits after the point and a power of ten). The angle of the cut-off, $\arcsin e^{-18}$, is 1.5230×10^{-8} radians too, because $\arcsin u \approx u$ for small u.

```
z_line = np.logspace(-10, np.log10(np.pi / 2), 800)  # 1e-10 ... pi/2, evenly in log
y_line = np.log(np.sin(z_line)) / 6.0  # with H = 1
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
axes[0].plot(z_line, y_line, color=VIOLET, lw=1.8, label="$y(z)$")
axes[0].axhline(-L_tip, color="black", lw=1.0, ls="--",
                label=f"tip cut-off $y = -L = -{L_tip:g}$")
axes[0].plot([z_cut], [-L_tip], "o", color="black", ms=6)
axes[0].set_xscale("log")
axes[0].set_xlabel("$z = 6 H x_0$ (radians, logarithmic axis)")
axes[0].set_ylabel("$y = \\ln(\\sin z)/(6H)$ (unit $1/H$)")
axes[0].set_title("The hidden coordinate $y$")
axes[0].legend(fontsize=8)
```

`np.logspace(-10, np.log10(np.pi / 2), 800)` gives 800 values of z spread evenly on a logarithmic scale from 10^{-10} to $\pi/2$, so that the region near the tip is visible. The left panel draws $y(z)$ with $H = 1$, the cut-off line $y = -3$ (the format `:g` prints `3.0` as `3`) and a black dot at the cut-off, with a logarithmic horizontal axis.

```
y_axis = np.linspace(-4.0, 0.0, 400)
axes[1].plot(y_axis, np.exp(y_axis), color=RED, lw=1.8,
             label="warp factor $W = e^{Hy} = \\sin^{1/6} z$")
axes[1].plot(y_axis, np.exp(2 * y_axis), color=ORANGE, lw=1.8, ls="-.",
             label="$W^2 = e^{2Hy} = \\sin^{1/3} z$")
axes[1].plot(y_axis, np.exp(6 * y_axis), color=AQUA, lw=1.8, ls="--",
             label="7-volume density $W^6 = e^{6Hy} = \\sin z$")
axes[1].axvspan(-4.0, -L_tip, color="grey", alpha=0.25,
                label="removed by the tip cut-off")
axes[1].set_xlabel("$y$ (unit $1/H$)")
axes[1].set_ylabel("value (no unit)")
axes[1].set_title("Warp and volume density in $y$")
axes[1].legend(fontsize=8, loc="upper left")
```

The right panel draws W, W^2 and W^6 as functions of y from -4 to 0, and shades (`axvspan`) the band beyond the cut-off.

```
save_figure(fig, "hidden_coordinate_y",
            "The hidden coordinate $y = \\ln(\\sin z)/(6H)$ with $H = 1$. Left: $y$ "
...
check(fraction == sp.exp(-18), "the cut-off removes exactly the fraction e^(-18)")
```

The figure is saved, and the check confirms that the fraction is exactly e^{-18}.

What Figure 03a.7 shows. Left: on the logarithmic z axis, y is a straight line near the tip, because there $\sin z \approx z$ and $y \approx \ln(z)/6$; it bends over to 0 at the patch end. The cut-off $y = -3$ sits at $z = 1.5 \times 10^{-8}$. Right: the warp factor and its powers fall towards 0 as y decreases; the volume density e^{6y} is already tiny at $y = -1$, and the grey band beyond the cut-off holds only the fraction e^{-18} of the 7-volume.

In [21], the last check.

```
figure_names = ["metric_heat_map", "diagonal_entries", "history_scale_factors",
                "hidden_direction", "scale_factor_maps", "proper_volumes",
                "hidden_coordinate_y"]
files = [output_file(f"{FIGURE_FOLDER}/03a_{k}_{n}.png")
         for k, n in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all seven figure files exist")
all_checks_passed()
```

The names of the seven figures; `enumerate(figure_names, 1)` numbers them from 1, giving the file names `03a_1_metric_heat_map.png` to `03a_7_hidden_coordinate_y.png`. The check requires all seven files to exist, and `all_checks_passed()` prints the last line, ALL 29 CHECKS PASSED (notebook 03a). The 29 checks are: 1 in In [5], 2 in In [6], 3 in In [7], 2 in In [8], 1 each in In [9] and In [10], 1 in In [11], 2 in In [12], 2 in In [13], 1 each in In [14], In [15] and In [16], 2 in In [17], 1 in In [18], 6 in In [19], 1 in In [20] and 1 in In [21].

3.14 Vectors, covectors and tensors

The metric turns coordinate steps into lengths. To speak about curvature we must first say precisely what a vector is and why some indices are written up and others down. The idea is simple: the same arrow has different components in different coordinates, and the rule by which the components change decides where the index goes.

A change of coordinates. Suppose a second set of coordinates $x'^1, \dots, x'^n$ names the same points, each new coordinate being a function of the old ones, $x'^a = x'^a(x^1, \dots, x^n)$, and each old one a function of the new ones. The $n \times n$ table of partial derivatives $\partial x'^a / \partial x^b$ is the **Jacobian matrix** of the change. The two tables $\partial x'^a / \partial x^b$ and $\partial x^a / \partial x'^b$ are inverse matrices:

$$\sum_{b=1}^{n} \frac{\partial x'^a}{\partial x^b} \frac{\partial x^b}{\partial x'^c} = \frac{\partial x'^a}{\partial x'^c} = \delta^a{}_c$$

(the chain rule applied to the function $x'^a(x(x'))$, which is just x'^a; the derivative of a coordinate with respect to a coordinate of the same set is 1 for the same one and 0 for another).

Vectors. A **curve** is a point that moves with a parameter λ (a time, or a length along the curve): $x^a(\lambda)$. Its **velocity** has the components $u^a = dx^a/d\lambda$. In the new coordinates the same curve is $x'^a(x(\lambda))$, and

$$u'^a = \frac{dx'^a}{d\lambda} = \sum_b \frac{\partial x'^a}{\partial x^b} \frac{dx^b}{d\lambda} = \sum_b \frac{\partial x'^a}{\partial x^b} u^b$$

(the chain rule for a function of several variables, each of which depends on λ). A **vector** at a point is a set of n numbers V^a, one set for every system of coordinates, such that the sets of two systems are related by exactly this rule, $V'^a = \sum_b (\partial x'^a/\partial x^b)\, V^b$. Its index is written **up**. A small coordinate step dx^a is a vector, which is why we write its index up.

Covectors. The **gradient** of a function f has the components $W_a = \partial_a f$. In the new coordinates

$$W'_a = \frac{\partial f}{\partial x'^a} = \sum_b \frac{\partial x^b}{\partial x'^a}\, \frac{\partial f}{\partial x^b} = \sum_b \frac{\partial x^b}{\partial x'^a}\, W_b$$

(the chain rule again, now for $f(x(x'))$). A **covector** is a set of n numbers W_a for every system of coordinates that changes by this rule. Its index is written **down**. The table that converts it is the inverse Jacobian matrix, $\partial x/\partial x'$ instead of $\partial x'/\partial x$.

An upper index summed with a lower index gives the same number in every system of coordinates. For a vector V and a covector W:

$$\sum_a V'^a W'_a = \sum_a \Big(\sum_b \frac{\partial x'^a}{\partial x^b} V^b\Big)\Big(\sum_c \frac{\partial x^c}{\partial x'^a} W_c\Big)$$

(the two rules inserted)

$$= \sum_{b,c} \Big(\sum_a \frac{\partial x^c}{\partial x'^a}\, \frac{\partial x'^a}{\partial x^b}\Big) V^b W_c$$

(the order of finite sums may be changed, and numbers may be multiplied in any order)

$$= \sum_{b,c} \delta^c{}_b\, V^b W_c = \sum_b V^b W_b$$

(the inverse-matrix identity above, with the roles of the two systems exchanged; then the Kronecker delta keeps only $c = b$). Such a sum is called a **contraction**. For example the rate of change of a function along a curve, $df/d\lambda = \sum_a u^a\, \partial_a f$ (the chain rule), is a contraction of a vector with a covector, so it does not depend on the coordinates, as it must not. A warning: summing an upper index with another upper index does NOT have this property. In one dimension, under the change $x' = 2x$ every vector is doubled, $V' = 2V$ and $W' = 2W$ for two vectors, so the product $V'W' = 4VW$ depends on the coordinates.

Tensors. A **tensor** with p upper and q lower indices is a table of n^{p+q} numbers for every system of coordinates that changes with one factor $\partial x'/\partial x$ for every upper index and one factor $\partial x/\partial x'$ for every lower index. For example $T'^a{}_b = \sum_{c,d} (\partial x'^a/\partial x^c)(\partial x^d/\partial x'^b)\, T^c{}_d$. Three facts follow from the definition, by the computation just made: a contraction of one upper with one lower index of a tensor is again a tensor (with two fewer indices); a product of tensors is a tensor; and a quantity in which every index is contracted, an **invariant** (or scalar), has the same value in every system of coordinates. Moreover, if all the numbers of a tensor are zero in one system of coordinates, they are zero in every system, because each new number is a sum of old numbers times factors.

The metric is a tensor with two lower indices. The squared length of a small step is a property of the step, not of the coordinates in which we describe it. So

$$\sum_{a,b} g_{ab}\,dx^a dx^b = \sum_{a,b} g_{ab}\Big(\sum_c \frac{\partial x^a}{\partial x'^c} dx'^c\Big)\Big(\sum_d \frac{\partial x^b}{\partial x'^d} dx'^d\Big)$$

(each old step written with the new steps by the chain rule), and comparing with $\sum_{c,d} g'_{cd}\,dx'^c dx'^d$, the coefficients of the new metric are

$$g'_{cd} = \sum_{a,b} \frac{\partial x^a}{\partial x'^c}\,\frac{\partial x^b}{\partial x'^d}\,g_{ab}$$

(the coefficient of $dx'^c dx'^d$ collected from the double sum; we may compare coefficient by coefficient because both metrics are symmetric). Two lower indices, two factors $\partial x/\partial x'$. The inverse metric g^{ab} is a tensor with two upper indices.

Raising and lowering indices. The metric turns a vector into a covector, $V_a = \sum_b g_{ab} V^b$, and the inverse metric turns a covector into a vector, $W^a = \sum_b g^{ab} W_b$; doing both returns the original, because $\sum_b g^{ab} g_{bc} = \delta^a{}_c$. The same rule raises or lowers any index of any tensor. The squared length of a vector is the invariant $g(V,V) = \sum_{a,b} g_{ab} V^a V^b = \sum_a V^a V_a$. For a diagonal metric raising or lowering only multiplies: $V_a = g_{aa} V^a$ (no sum).

Example: a constant field in polar coordinates. In the plane take the field e_x that points one unit to the right everywhere: Cartesian components $(V^x, V^y) = (1, 0)$. With $r = \sqrt{x^2+y^2}$ and the angle φ (in the right half of the plane $\varphi = \arctan(y/x)$),

$$\frac{\partial r}{\partial x} = \frac{x}{r} = \cos\varphi, \qquad \frac{\partial \varphi}{\partial x} = \frac{1}{1+y^2/x^2}\cdot\Big(-\frac{y}{x^2}\Big) = -\frac{y}{x^2+y^2} = -\frac{\sin\varphi}{r}$$

(the chain rule: the derivative of $\sqrt{u}$ is $1/(2\sqrt{u})$ times $du/dx = 2x$; the derivative of $\arctan u$ is $1/(1+u^2)$ times $du/dx = -y/x^2$; then $x = r\cos\varphi$ and $y = r\sin\varphi$). The vector rule gives the polar components

$$V^r = \frac{\partial r}{\partial x}\cdot 1 + \frac{\partial r}{\partial y}\cdot 0 = \cos\varphi, \qquad V^\varphi = \frac{\partial \varphi}{\partial x}\cdot 1 + \frac{\partial \varphi}{\partial y}\cdot 0 = -\frac{\sin\varphi}{r}.$$

The field is constant, yet its polar components change from point to point. So the plain derivatives $\partial_b V^a$ of the components are not a good measure of how a vector field changes. The next section repairs this.

3.15 The covariant derivative and the Christoffel symbols

The problem. In the example of Section 3.14, $\partial_\varphi V^r = -\sin\varphi$ is not zero, although the field does not change at all: the derivative also records the turning of the coordinate directions. In general, differentiating the vector rule $V'^a = \sum_b (\partial x'^a/\partial x^b) V^b$ produces, besides the tensor rule, a term with the second derivatives $\partial^2 x'^a/\partial x^b \partial x^c$ of the change of coordinates, so $\partial_b V^a$ is not a tensor.

Definition (covariant derivative). We add a correction that is linear in V:

$$\nabla_b V^a = \partial_b V^a + \sum_c \Gamma^a{}_{bc}\, V^c.$$

The n^3 numbers $\Gamma^a{}_{bc}$ at each point are the **connection coefficients**; they are chosen so that $\nabla_b V^a$ is a tensor, which fixes how they change between systems of coordinates (we do not need that rule). For a function f (a tensor without indices) we set $\nabla_b f = \partial_b f$.

Covectors. We require the product rule for the function $\sum_a V^a W_a$:

$$\partial_b\Big(\sum_a V^a W_a\Big) = \sum_a (\nabla_b V^a) W_a + \sum_a V^a\, \nabla_b W_a.$$

The left side is $\sum_a (\partial_b V^a) W_a + \sum_a V^a\, \partial_b W_a$ (the ordinary product rule). The first term on the right is $\sum_a (\partial_b V^a) W_a + \sum_{a,c} \Gamma^a{}_{bc} V^c W_a$ (the definition). Subtracting,

$$\sum_a V^a\, \nabla_b W_a = \sum_a V^a\, \partial_b W_a - \sum_{a,c} \Gamma^a{}_{bc} V^c W_a = \sum_a V^a \Big(\partial_b W_a - \sum_c \Gamma^c{}_{ba} W_c\Big)$$

(in the double sum the two summed letters a and c were renamed c and a; a summed letter is only a name). This holds for every vector V, so

$$\nabla_b W_a = \partial_b W_a - \sum_c \Gamma^c{}_{ba}\, W_c.$$

Tensors with two indices. Again we require the product rule. Take first a product $T^a{}_b = V^a W_b$ of a vector and a covector (a tensor with one upper and one lower index, Section 3.14):

$$\nabla_c (V^a W_b) = (\nabla_c V^a) W_b + V^a\, \nabla_c W_b$$

(the product rule, required)

$$= \Big(\partial_c V^a + \sum_e \Gamma^a{}_{ce} V^e\Big) W_b + V^a \Big(\partial_c W_b - \sum_e \Gamma^e{}_{cb} W_e\Big)$$

(the two formulas just proved, with the letters b, c of the vector formula renamed c, e and the letters b, a, c of the covector formula renamed c, b, e)

$$= \partial_c (V^a W_b) + \sum_e \Gamma^a{}_{ce}\, (V^e W_b) - \sum_e \Gamma^e{}_{cb}\, (V^a W_e)$$

(the ordinary product rule $\partial_c (V^a W_b) = (\partial_c V^a) W_b + V^a\, \partial_c W_b$ collects the two terms without Γ; in the other two terms the factors are regrouped). In the notation $T^a{}_b = V^a W_b$, so that $V^e W_b = T^e{}_b$ and $V^a W_e = T^a{}_e$, this reads

$$\nabla_c T^a{}_b = \partial_c T^a{}_b + \sum_e \Gamma^a{}_{ce}\, T^e{}_b - \sum_e \Gamma^e{}_{cb}\, T^a{}_e.$$

The same three steps for a product $T_{ab} = W_a U_b$ of two covectors give

$$\nabla_c(W_a U_b) = (\nabla_c W_a) U_b + W_a\, \nabla_c U_b = \partial_c(W_a U_b) - \sum_e \Gamma^e{}_{ca}\, (W_e U_b) - \sum_e \Gamma^e{}_{cb}\, (W_a U_e)$$

(the product rule; the covector formula twice; the ordinary product rule collects the terms without Γ). Every tensor with two indices is a sum of such products. For example, let $E_{(d)}$ be the vector field with the components $\delta^a{}_d$ (the unit step along x_d) and $\theta^{(f)}$ the covector field with the components $\delta^f{}_b$; then

$$T^a{}_b = \sum_{d,f} T^d{}_f\, \delta^a{}_d\, \delta^f{}_b = \sum_{d,f} \left(T^d{}_f\, E_{(d)}\right)^a \left(\theta^{(f)}\right)_b$$

(in the double sum only $d = a$ and $f = b$ give a non-zero product of deltas), a sum of products (64 of them in eight dimensions) of a vector (the field $E_{(d)}$ multiplied by the function $T^d{}_f$) and a covector. Both sides of the formula for $\nabla_c T^a{}_b$ are sums of terms each containing T once, so the formula holds for a sum of products when it holds for each product; it therefore holds for every such tensor, and in the same way the formula for two lower indices holds for every tensor with two lower indices. With more indices the same steps, with more factors, give the general rule: **one term $+\sum_e \Gamma$ for every upper index and one term $-\sum_e \Gamma$ for every lower index**. For the metric (two lower indices):

$$\nabla_c\, g_{ab} = \partial_c\, g_{ab} - \sum_e \Gamma^e{}_{ca}\, g_{eb} - \sum_e \Gamma^e{}_{cb}\, g_{ae}.$$

The two conditions of Levi-Civita. A metric singles out one connection by two natural conditions:

1. no torsion: $\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$ (symmetric in the two lower indices);
2. metric compatibility: $\nabla_c\, g_{ab} = 0$ (lengths and angles do not change when vectors are carried along without turning).

Theorem (the Christoffel formula). Exactly one connection satisfies both conditions:

$$\Gamma^a{}_{bc} = \tfrac{1}{2} \sum_d g^{ad} (\partial_b\, g_{dc} + \partial_c\, g_{db} - \partial_d\, g_{bc}).$$

These are the **Christoffel symbols**. Every Revision record and every notebook of the book uses this connection.

Proof, line by line. Lower the upper index: $\Gamma_{dbc} = \sum_a g_{da} \Gamma^a{}_{bc}$. Condition 1 says $\Gamma_{dbc} = \Gamma_{dcb}$. Condition 2, with the formula for $\nabla_c\, g_{ab}$, reads

$$\partial_c\, g_{ab} = \Gamma_{bca} + \Gamma_{acb}$$

(the two sums $\sum_e \Gamma^e{}_{ca} g_{eb}$ and $\sum_e \Gamma^e{}_{cb} g_{ae}$ are, by the definition of the lowered symbol and the symmetry of g, Γ_{bca} and Γ_{acb}). Write this equation three times, with the three letters in the three cyclic orders:

$$\partial_b g_{cd} = \Gamma_{dbc} + \Gamma_{cbd}, \qquad \partial_c g_{bd} = \Gamma_{dcb} + \Gamma_{bcd}, \qquad \partial_d g_{bc} = \Gamma_{cdb} + \Gamma_{bdc}$$

(the previous equation with the letters renamed). Add the first two and subtract the third:

$$\partial_b g_{cd} + \partial_c g_{bd} - \partial_d g_{bc} = \Gamma_{dbc} + \Gamma_{dcb} + (\Gamma_{cbd} - \Gamma_{cdb}) + (\Gamma_{bcd} - \Gamma_{bdc}) = 2\,\Gamma_{dbc}$$

(by condition 1, $\Gamma_{cbd} = \Gamma_{cdb}$ and $\Gamma_{bcd} = \Gamma_{bdc}$, so the two brackets are zero, and $\Gamma_{dcb} = \Gamma_{dbc}$). Multiply by $\frac{1}{2}g^{ad}$ and sum over d:

$$\Gamma^a{}_{bc} = \sum_d g^{ad}\,\Gamma_{dbc} = \tfrac{1}{2}\sum_d g^{ad}(\partial_b g_{cd} + \partial_c g_{bd} - \partial_d g_{bc})$$

(the inverse metric undoes the lowering, because $\sum_d g^{ad}g_{de} = \delta^a{}_e$; and $g_{cd} = g_{dc}$, $g_{bd} = g_{db}$). So a connection with the two properties can only be this one. Conversely, the formula is symmetric in b and c (exchanging them exchanges the first two terms), and putting it into $\Gamma_{bca} + \Gamma_{acb}$ gives back $\partial_c g_{ab}$ (the four terms with derivatives of g along other directions cancel in pairs), so it has both properties. (That the symbols computed in two systems of coordinates describe the same ∇ is a standard fact that we quote, ASSUMED.) $\square$

Diagonal metrics: four cases. All metrics of this chapter are diagonal. Write $g_{aa} = \eta_{aa}h_a^2$ with the sign $\eta_{aa} = \pm 1$ and the positive scale factor h_a (Section 3.7), so that $g^{aa} = \eta_{aa}/h_a^2$ (no sum here and in the four cases). In the formula only $d = a$ survives, because $g^{ad} = 0$ for $d \neq a$:

$$\Gamma^a{}_{bc} = \tfrac{1}{2}\,g^{aa}(\partial_b g_{ac} + \partial_c g_{ab} - \partial_a g_{bc}) \quad \text{(no sum)}.$$

(a) All three indices equal: $\Gamma^a{}_{aa} = \frac{1}{2}g^{aa}\,\partial_a g_{aa} = \frac{1}{2}\,\frac{\eta_{aa}}{h_a^2}\,\eta_{aa}\,2h_a\,\partial_a h_a = \partial_a \ln h_a$ (two of the three terms cancel; the derivative of h_a^2 is $2h_a\,\partial_a h_a$; $\eta_{aa}^2 = 1$; and $\partial h/h = \partial \ln h$).

(b) One lower index equal to the upper one, $b \neq a$: $\Gamma^a{}_{ab} = \Gamma^a{}_{ba} = \frac{1}{2}g^{aa}(\partial_a g_{ab} + \partial_b g_{aa} - \partial_a g_{ab}) = \frac{1}{2}g^{aa}\,\partial_b g_{aa} = \partial_b \ln h_a$ (the first and the third terms cancel; the rest as in (a)).

(c) Two equal lower indices different from the upper one, $b \neq a$: $\Gamma^a{}_{bb} = \frac{1}{2}g^{aa}(2\,\partial_b g_{ab} - \partial_a g_{bb}) = -\frac{1}{2}\,g^{aa}\,\partial_a g_{bb} = -\eta_{aa}\eta_{bb}\,\frac{h_b^2}{h_a^2}\,\partial_a \ln h_b$ (here $g_{ab} = 0$ because $a \neq b$; then $\partial_a g_{bb} = \eta_{bb}\,2h_b\,\partial_a h_b = 2\eta_{bb}h_b^2\,\partial_a \ln h_b$).

(d) Three different indices: $\Gamma^a{}_{bc} = 0$ (every metric entry in the bracket has two different indices and is zero).

Example: the plane in polar coordinates. $h_r = 1$, $h_\varphi = r$, both signs $+1$. Only h_φ depends on a coordinate, on r. Case (b): $\Gamma^\varphi{}_{r\varphi} = \Gamma^\varphi{}_{\varphi r} = \partial_r \ln r = 1/r$. Case (c): $\Gamma^r{}_{\varphi\varphi} = -\frac{r^2}{1}\,\partial_r \ln r = -r$. All others are zero. Now the constant field $V^r = \cos\varphi$, $V^\varphi = -\sin\varphi/r$ of Section 3.14:

$$\nabla_r V^r = \partial_r \cos\varphi + \Gamma^r{}_{rr}V^r + \Gamma^r{}_{r\varphi}V^\varphi = 0 + 0 + 0 = 0,$$

$$\nabla_\varphi V^r = \partial_\varphi \cos\varphi + \Gamma^r{}_{\varphi\varphi}V^\varphi = -\sin\varphi + (-r)\Big(-\frac{\sin\varphi}{r}\Big) = 0,$$

$$\nabla_r V^\varphi = \partial_r\Big(-\frac{\sin\varphi}{r}\Big) + \Gamma^\varphi{}_{r\varphi} V^\varphi = \frac{\sin\varphi}{r^2} + \frac{1}{r}\Big(-\frac{\sin\varphi}{r}\Big) = 0,$$

$$\nabla_\varphi V^\varphi = \partial_\varphi\Big(-\frac{\sin\varphi}{r}\Big) + \Gamma^\varphi{}_{\varphi r} V^r = -\frac{\cos\varphi}{r} + \frac{1}{r}\cos\varphi = 0$$

(in each line the definition of ∇, then the symbols just found, the others being zero). The covariant derivative of the constant field is zero, as it must be: the Christoffel terms remove exactly the change of the components that comes from the turning of the polar directions. Notebook 03c checks these four numbers (In [5]).

Example: the sphere of radius a. $h_\theta = a$, $h_\varphi = a\sin\theta$, both signs $+1$; only h_φ depends on a coordinate, on θ, with $\partial_\theta \ln(a\sin\theta) = \cos\theta/\sin\theta = \cot\theta$ (the logarithm of a product is the sum of the logarithms; the derivative of $\ln\sin\theta$ is $\cos\theta/\sin\theta$). Case (b): $\Gamma^\varphi{}_{\theta\varphi} = \Gamma^\varphi{}_{\varphi\theta} = \cot\theta$. Case (c): $\Gamma^\theta{}_{\varphi\varphi} = -\frac{a^2\sin^2\theta}{a^2}\cot\theta = -\sin\theta\cos\theta$. All others are zero.

The contracted symbol and the volume factor. For a diagonal metric, summing case (a) and case (b) over the upper index,

$$\sum_a \Gamma^a{}_{ab} = \sum_a \partial_b \ln h_a = \partial_b \ln\Big(\prod_a h_a\Big) = \partial_b \ln\sqrt{|\det g|}$$

(for $a = b$ case (a), for $a \neq b$ case (b), both give $\partial_b \ln h_a$; a sum of logarithms is the logarithm of the product; the product of the scale factors is the volume factor of Section 3.6). This identity is used for the author's metric in Section 3.23.

3.16 Parallel transport and geodesics

The derivative along a curve. Let $x^a(\lambda)$ be a curve with velocity $u^a = dx^a/d\lambda$, and $V^a(\lambda)$ a vector given at each point of the curve. If V is the value of a vector field on the curve, then $dV^a/d\lambda = \sum_b u^b\,\partial_b V^a$ (the chain rule), and therefore

$$\sum_b u^b\,\nabla_b V^a = \frac{dV^a}{d\lambda} + \sum_{b,c}\Gamma^a{}_{bc}\,u^b V^c =: \frac{DV^a}{d\lambda}$$

(the definition of ∇, multiplied by u^b and summed). The right side needs V only on the curve, so it defines the **covariant derivative along the curve** for any $V(\lambda)$. A vector is **parallel transported** along the curve when $DV^a/d\lambda = 0$: it is carried along without turning.

Geodesics. A **geodesic** is a curve whose velocity is parallel transported along itself:

$$\frac{d^2x^a}{d\lambda^2} + \sum_{b,c}\Gamma^a{}_{bc}\,\frac{dx^b}{d\lambda}\,\frac{dx^c}{d\lambda} = 0 \qquad (a = 1,\ldots,n).$$

In flat space with Cartesian coordinates every Γ is zero, and the equation says $d^2x^a/d\lambda^2 = 0$: straight lines, run through at constant speed. A geodesic is the straightest curve that a curved space allows. In Einstein's theory a body on which no force other than gravity acts (a body in **free**

fall) moves on a time-like geodesic; this is the physical content of the equation, which we take from the theory of gravity (ASSUMED).

The squared speed does not change along a geodesic. Lower the first index of the Christoffel symbol: by the proof of Section 3.15, $2\Gamma_{abc} = \partial_b g_{ca} + \partial_c g_{ba} - \partial_a g_{bc}$. Then, along a geodesic,

$$\frac{d}{d\lambda}\sum_{a,b} g_{ab}u^a u^b = \sum_{a,b,c}(\partial_c g_{ab})\, u^c u^a u^b + 2\sum_{a,b} g_{ab}\, u^a \frac{du^b}{d\lambda}$$

(the product rule; the metric changes along the curve by the chain rule; the two terms with $du/d\lambda$ are equal because g is symmetric)

$$= \sum_{a,b,c}(\partial_c g_{ab})\, u^c u^a u^b - 2\sum_{a,b,c}\Gamma_{abc}\, u^a u^b u^c$$

(the geodesic equation $du^b/d\lambda = -\sum \Gamma^b{}_{cd} u^c u^d$, and $\sum_b g_{ab}\Gamma^b{}_{cd} = \Gamma_{acd}$, with the summed letters renamed)

$$= \sum_{a,b,c}(\partial_c g_{ab})\, u^c u^a u^b - \sum_{a,b,c}(\partial_b g_{ca} + \partial_c g_{ba} - \partial_a g_{bc}) u^a u^b u^c = 0$$

(the formula for $2\Gamma_{abc}$; each of the three terms in the bracket, after renaming the summed letters, equals the first sum, with the signs $+$, $+$, $-$, so together they are exactly the first sum). So a geodesic that starts time-like stays time-like with the same $g(u,u)$. For a time-like geodesic we choose the parameter so that $g(u,u) = -1$; then λ is the **proper time** τ of the body, the time shown by a clock carried along (since $d\tau^2 = -ds^2$, Section 3.4).

A conserved momentum. Suppose the diagonal metric does not depend on one coordinate x_k at all. Then the **momentum** $p_k = g_{kk}u^k$ (no sum) does not change along a geodesic. Line by line:

$$\frac{dp_k}{d\lambda} = \sum_c (\partial_c g_{kk})\, u^c\, u^k + g_{kk}\frac{du^k}{d\lambda}$$

(the product rule and the chain rule).

$$\frac{du^k}{d\lambda} = -\sum_{b,c}\Gamma^k{}_{bc} u^b u^c = -2\sum_{c\neq k}\Gamma^k{}_{kc}\, u^k u^c$$

(of the four cases of Section 3.15, case (a) $\Gamma^k{}_{kk} = \partial_k \ln h_k$ and case (c) $\Gamma^k{}_{bb} = -\frac{1}{2}g^{kk}\partial_k g_{bb}$ are zero because nothing depends on x_k, case (d) is zero, and case (b) appears twice, as $\Gamma^k{}_{kc}$ and $\Gamma^k{}_{ck}$)

$$= -2\sum_{c\neq k}\frac{\partial_c g_{kk}}{2\, g_{kk}}\, u^k u^c = -\frac{1}{g_{kk}}\sum_c (\partial_c g_{kk})\, u^c u^k$$

(case (b) written as $\frac{1}{2}g^{kk}\partial_c g_{kk}$; the term $c = k$ may be added because $\partial_k g_{kk} = 0$). Inserting the last line into the first, the two terms cancel: $dp_k/d\lambda = 0$. **A metric that does not depend on a**

coordinate conserves the momentum of that coordinate. In the plane in polar coordinates the metric does not depend on φ, and $p_\varphi = r^2\, d\varphi/d\lambda$ is the angular momentum of mechanics; on the sphere $p_\varphi = a^2 \sin^2\theta\, d\varphi/d\lambda$ is conserved.

The geodesics of the sphere. With the two symbols of Section 3.15 (${\Gamma^\theta}_{\varphi\varphi} = -\sin\theta\cos\theta$ and ${\Gamma^\varphi}_{\theta\varphi} = {\Gamma^\varphi}_{\varphi\theta} = \cot\theta$) and the length s along the curve as parameter,

$$\frac{d^2\theta}{ds^2} = \sin\theta\cos\theta\Big(\frac{d\varphi}{ds}\Big)^2, \qquad \frac{d^2\varphi}{ds^2} = -2\cot\theta\,\frac{d\theta}{ds}\,\frac{d\varphi}{ds}$$

(the geodesic equation for $a = \theta$ has the single term ${\Gamma^\theta}_{\varphi\varphi}(d\varphi/ds)^2$ moved to the right; for $a = \varphi$ the two equal terms ${\Gamma^\varphi}_{\theta\varphi}$ and ${\Gamma^\varphi}_{\varphi\theta}$ give the factor 2). The **equator** $\theta = \pi/2$, $\varphi = s/a$ is a geodesic: $d^2\theta/ds^2 = 0$ and $\sin(\pi/2)\cos(\pi/2) = 0$, and $d^2\varphi/ds^2 = 0$ with $d\theta/ds = 0$. A **circle of latitude** $\theta = \theta_0$ with $0 < \theta_0 < \pi/2$, run through at unit speed, is NOT a geodesic: along it $d^2\theta/ds^2 = 0$, while the right side is $\sin\theta_0\cos\theta_0\,(d\varphi/ds)^2 = \sin\theta_0\cos\theta_0/(a^2\sin^2\theta_0) = \cot\theta_0/a^2 \neq 0$ (unit speed means $a\sin\theta_0\, d\varphi/ds = 1$). For $a = 1$ and $\theta_0 = \pi/3$ the missing acceleration is $\cot(\pi/3) = 1/\sqrt{3} = 0.577350$, the number Notebook 03c prints (In [11]). The geodesics of the sphere are its **great circles**, the circles whose centre is the centre of the sphere (the equator, the meridians and all their rotations); Notebook 03c shows this numerically for three of them (In [11]).

Parallel transport around a circle of latitude. Carry a vector once around the circle $\theta = \theta_0$, using φ as the parameter (so $d\theta/d\varphi = 0$ and $d\varphi/d\varphi = 1$). The transport equation $dV^a/d\varphi + \sum_c {\Gamma^a}_{\varphi c} V^c = 0$ has one term for each component:

$$\frac{dV^\theta}{d\varphi} = \sin\theta_0\cos\theta_0\, V^\varphi, \qquad \frac{dV^\varphi}{d\varphi} = -\cot\theta_0\, V^\theta$$

(for $a = \theta$ the only symbol is ${\Gamma^\theta}_{\varphi\varphi}$, for $a = \varphi$ the only one is ${\Gamma^\varphi}_{\varphi\theta}$). Measure the vector with the unit vectors pointing south and east: its lengths along them are $u = aV^\theta$ and $w = a\sin\theta_0\, V^\varphi$ (the scale factors $h_\theta = a$, $h_\varphi = a\sin\theta_0$). Multiplying the two equations by a and by $a\sin\theta_0$,

$$\frac{du}{d\varphi} = \cos\theta_0\, w, \qquad \frac{dw}{d\varphi} = -\cos\theta_0\, u$$

(in the first, $a\sin\theta_0\cos\theta_0 V^\varphi = \cos\theta_0\, w$; in the second, $a\sin\theta_0\cot\theta_0 V^\theta = \cos\theta_0\, u$). Start with the unit vector pointing south, $u = 1$, $w = 0$. The solution is

$$u = \cos(\varphi\cos\theta_0), \qquad w = -\sin(\varphi\cos\theta_0)$$

(check: $du/d\varphi = -\cos\theta_0\sin(\varphi\cos\theta_0) = \cos\theta_0\, w$ and $dw/d\varphi = -\cos\theta_0\cos(\varphi\cos\theta_0) = -\cos\theta_0\, u$, and at $\varphi = 0$ the values are 1 and 0; the solution with given starting values is unique, a standard theorem that we quote, ASSUMED). The length $\sqrt{u^2 + w^2} = 1$ never changes, and relative to the local south and east directions the vector turns at the constant rate $\cos\theta_0$. After one round, $\varphi = 2\pi$, it has turned by $-2\pi\cos\theta_0$, which is the same direction as a turn by $2\pi - 2\pi\cos\theta_0 = 2\pi(1 - \cos\theta_0)$ (two angles that differ by a whole turn 2π give the same direction). At $\theta_0 = \pi/3$, $\cos\theta_0 = 1/2$: $u = \cos\pi = -1$, $w = -\sin\pi = 0$. The vector comes back pointing north, exactly reversed. On the plane the same walk would return the vector unchanged.

The turning angle is the enclosed area times the curvature. The cap north of the circle has the area

$$\int_0^{\theta_0}\int_0^{2\pi} a^2 \sin\theta \, d\varphi \, d\theta = 2\pi a^2 \int_0^{\theta_0} \sin\theta \, d\theta = 2\pi a^2 \big[-\cos\theta\big]_0^{\theta_0} = 2\pi a^2(1-\cos\theta_0)$$

(the area factor $\sqrt{\det g} = a \cdot a\sin\theta$ of Section 3.6; the inner integral gives 2π; an antiderivative of $\sin\theta$ is $-\cos\theta$; $\cos 0 = 1$). So the turning angle $2\pi(1-\cos\theta_0)$ equals the enclosed area divided by a^2. Section 3.18 shows that $1/a^2$ is the curvature of the sphere. Notebook 03c measures the turning angle for 15 circles with RK4 and finds it equal to the area divided by a^2 within 5.0×10^{-12} radians (In [10]).

3.17 Curvature: the Riemann tensor and its contractions

The idea. On a flat sheet a vector carried around a closed loop without turning comes back unchanged; on the sphere it comes back turned (Section 3.16). For a very small loop the turning is measured by the difference between taking two covariant derivatives in one order and in the other order. In flat Cartesian coordinates $\nabla_c\nabla_d = \partial_c\partial_d$, and partial derivatives can be taken in either order; in a curved space covariant derivatives cannot.

Theorem (the Riemann tensor). For every vector field V,

$$\nabla_c\nabla_d V^a - \nabla_d\nabla_c V^a = \sum_b R^a{}_{bcd}\, V^b, \qquad R^a{}_{bcd} = \partial_c\Gamma^a{}_{bd} - \partial_d\Gamma^a{}_{bc} + \sum_e (\Gamma^a{}_{ce}\Gamma^e{}_{bd} - \Gamma^a{}_{de}\Gamma^e{}_{bc}).$$

This is the convention of the textbook of Misner, Thorne and Wheeler (MTW), used by every Revision record (`Revision/gkd_lovelock/results/python-lovelock-report.json`, key `conventions.riemann`).

Proof, line by line. $\nabla_d V^a$ has one upper index a and one lower index d, so by the rule of Section 3.15

$$\nabla_c(\nabla_d V^a) = \partial_c(\nabla_d V^a) + \sum_e \Gamma^a{}_{ce}\,\nabla_d V^e - \sum_e \Gamma^e{}_{cd}\,\nabla_e V^a$$

(one $+\Gamma$ for the upper index, one $-\Gamma$ for the lower index). Insert $\nabla_d V^a = \partial_d V^a + \sum_b \Gamma^a{}_{db} V^b$:

$$= \partial_c\partial_d V^a + \sum_b (\partial_c\Gamma^a{}_{db}) V^b + \sum_b \Gamma^a{}_{db}\,\partial_c V^b + \sum_e \Gamma^a{}_{ce}\,\partial_d V^e + \sum_{e,b} \Gamma^a{}_{ce}\Gamma^e{}_{db} V^b - \sum_e \Gamma^e{}_{cd}\,\nabla_e V^a$$

(the product rule for $\partial_c(\Gamma V)$). Now subtract the same expression with c and d exchanged. Three kinds of terms drop out: $\partial_c\partial_d V^a$ (partial derivatives can be taken in either order); the pair $\sum_b \Gamma^a{}_{db}\partial_c V^b + \sum_e \Gamma^a{}_{ce}\partial_d V^e$, which after renaming e to b is unchanged by the exchange of c and d; and $\sum_e \Gamma^e{}_{cd}\nabla_e V^a$ (no torsion: $\Gamma^e{}_{cd} = \Gamma^e{}_{dc}$). What remains is

$$\sum_b \Big(\partial_c \Gamma^a{}_{db} - \partial_d \Gamma^a{}_{cb} + \sum_e \Gamma^a{}_{ce}\Gamma^e{}_{db} - \sum_e \Gamma^a{}_{de}\Gamma^e{}_{cb} \Big) V^b,$$

which is $\sum_b R^a{}_{bcd} V^b$ because $\Gamma^a{}_{db} = \Gamma^a{}_{bd}$, $\Gamma^a{}_{cb} = \Gamma^a{}_{bc}$, $\Gamma^e{}_{db} = \Gamma^e{}_{bd}$ and $\Gamma^e{}_{cb} = \Gamma^e{}_{bc}$. □

The left side is a tensor for every V and contains no derivative of V, so $R^a{}_{bcd}$ is a tensor (one upper, three lower indices). Hence: **if it is zero in one system of coordinates, it is zero in every system** (Section 3.14). A space whose Riemann tensor is zero everywhere is **flat**; conversely, near each point of a space with zero Riemann tensor there are coordinates in which the metric is constant (a standard theorem, quoted, ASSUMED).

Symmetries. (S1) $R^a{}_{bcd} = -R^a{}_{bdc}$: exchanging c and d in the formula exchanges the two derivative terms with a sign change, and so for the two products. (S2) The **first Bianchi identity** $R^a{}_{bcd} + R^a{}_{cdb} + R^a{}_{dbc} = 0$. Proof: write the three terms with the formula. The six derivative terms are $\partial_c \Gamma^a{}_{bd} - \partial_d \Gamma^a{}_{bc} + \partial_d \Gamma^a{}_{cb} - \partial_b \Gamma^a{}_{cd} + \partial_b \Gamma^a{}_{dc} - \partial_c \Gamma^a{}_{db}$; they cancel in pairs because the symbols are symmetric in their lower indices. The six products are $\Gamma^a{}_{ce}\Gamma^e{}_{bd} - \Gamma^a{}_{de}\Gamma^e{}_{bc} + \Gamma^a{}_{de}\Gamma^e{}_{cb} - \Gamma^a{}_{be}\Gamma^e{}_{cd} + \Gamma^a{}_{be}\Gamma^e{}_{dc} - \Gamma^a{}_{ce}\Gamma^e{}_{db}$ (summed over e); they cancel in pairs for the same reason. □ Two more symmetries hold for every metric; we quote them (ASSUMED as general theorems; Notebook 03b checks both exactly for the author's metric, In [14]): with the first index lowered, $R_{abcd} = \sum_e g_{ae} R^e{}_{bcd}$, (S3) $R_{abcd} = -R_{bacd}$ and (S4) $R_{abcd} = R_{cdab}$ (pair symmetry).

The form with two upper indices. The Revision records store the tensor as

$$R^{ab}{}_{cd} = \sum_e g^{be} R^a{}_{ecd},$$

which for a diagonal metric is $g^{bb} R^a{}_{bcd}$ (no sum). By (S1) and (S3) it is antisymmetric in a, b and in c, d; with all indices down, for a diagonal metric, $R_{abcd} = g_{aa} g_{bb} R^{ab}{}_{cd}$ (no sum). For two different coordinates x_a and x_b the number

$$\sigma(a,b) = R^{ab}{}_{ab} \quad \text{(no sum)}$$

is the **curvature of the coordinate plane** of x_a and x_b (its **sectional curvature**; "plane" means here the plane of the two directions x_a and x_b at one point). We write it with the Greek letter σ (sigma), because the letter K is kept for the Kretschmann scalar below. What its sign means depends on the kind of directions:

- **Two space-like directions.** Then every step in the plane has a positive squared length, as on an ordinary surface, and $\sigma(a,b)$ is read like the curvature of a surface: positive for a plane curved like a sphere, negative for one curved like a saddle, zero for a flat one. Section 3.18 makes this precise: geodesics that start parallel in such a plane approach each other when $\sigma > 0$ and separate when $\sigma < 0$.
- **One space-like and one time-like direction.** There is no shape of this kind to picture. The sign is read from free fall instead: for two neighbouring free-fall paths that run along the time-like direction, the positive sign means that they accelerate APART, the opposite of the sphere. Section 3.18 states the rule and Section 3.24 verifies it by hand in the author's metric.

- **Two time-like directions.** Every step in the plane has a negative squared length. Replace g by $-g$: the Christoffel symbols do not change (both factors of $\frac{1}{2}g^{ad}(\partial g + \partial g - \partial g)$ change sign), so $R^a{}_{bcd}$ does not change, but every g^{bb} changes sign, so $\sigma(a,b) = g^{bb}R^a{}_{bab}$ (diagonal metric) changes sign. With $-g$ the plane has positive squared lengths, so its shape is that of the curvature $-\sigma(a,b)$: here a negative σ means curved like a sphere, the reverse of the first case. The free-fall reading of the second case still holds, because free-fall paths along a time-like direction are involved.

By the two antisymmetries, $R^{ba}{}_{ba} = R^{ab}{}_{ab}$ and $R^{ab}{}_{ba} = R^{ba}{}_{ab} = -R^{ab}{}_{ab}$.

Contractions. The **Ricci tensor**, the **Ricci scalar**, the **Einstein tensor** and the **Kretschmann scalar** are

$$R^a{}_b = \sum_c R^{ac}{}_{bc}, \qquad R = \sum_a R^a{}_a, \qquad G^a{}_b = R^a{}_b - \tfrac{1}{2}\,\delta^a{}_b\,R, \qquad K = \sum_{a,b,c,d} R^{ab}{}_{cd}\,R^{cd}{}_{ab}.$$

R and K have every index contracted: they are invariants, the same in every system of coordinates (Section 3.14). The Einstein tensor is the combination that enters Einstein's field equations (Section 3.26). Two consequences for the diagonal entries are used again and again:

$$R^a{}_a = \sum_c R^{ac}{}_{ac} = \sum_{c\neq a} \sigma(a,c)$$

(the definition with $b = a$; the term $c = a$ is $R^{aa}{}_{aa} = 0$ by the antisymmetry in the upper pair): **each diagonal entry of the Ricci tensor is the sum of the curvatures of the $n-1$ coordinate planes that contain that direction**; and

$$R = \sum_a \sum_{c\neq a} \sigma(a,c) = 2\sum_{a<c} \sigma(a,c)$$

(every plane is counted twice, once as (a,c) and once as (c,a), with $\sigma(c,a) = \sigma(a,c)$).

A surface. A space of two dimensions has a single coordinate plane, and its curvature $\sigma(1,2)$ is the **Gaussian curvature**. Then $R^1{}_1 = R^2{}_2 = \sigma(1,2)$ and $R = 2\sigma(1,2)$. The non-zero components $R^{ab}{}_{cd}$ are the four $R^{12}{}_{12} = R^{21}{}_{21} = \sigma(1,2)$ and $R^{12}{}_{21} = R^{21}{}_{12} = -\sigma(1,2)$, so the Kretschmann scalar is $K = 4\sigma(1,2)^2$ (each of the four products $R^{ab}{}_{cd}R^{cd}{}_{ab}$ is $\sigma(1,2)^2$).

3.18 The curvature of the plane and of the sphere, by hand; geodesic deviation

The plane in polar coordinates is flat. With the symbols of Section 3.15:

$$R^r{}_{\varphi r\varphi} = \partial_r\Gamma^r{}_{\varphi\varphi} - \partial_\varphi\Gamma^r{}_{\varphi r} + \sum_e (\Gamma^r{}_{re}\Gamma^e{}_{\varphi\varphi} - \Gamma^r{}_{\varphi e}\Gamma^e{}_{\varphi r})$$

(the formula with $a = r$, $b = \varphi$, $c = r$, $d = \varphi$)

$$= \partial_r(-r) - 0 + 0 - \Gamma^r{}_{\varphi\varphi}\Gamma^\varphi{}_{\varphi r} = -1 - (-r)\cdot\frac{1}{r} = 0$$

($\Gamma^r{}_{\varphi r} = 0$; every $\Gamma^r{}_{re}$ is zero; in the last sum only $e = \varphi$ survives). In two dimensions this is the only independent component, so the Riemann tensor of the plane is zero, although its polar Christoffel symbols are not. **Christoffel symbols describe the coordinates; the Riemann tensor describes the space.**

The sphere of radius a.

$$R^\theta{}_{\varphi\theta\varphi} = \partial_\theta \Gamma^\theta{}_{\varphi\varphi} - \partial_\varphi \Gamma^\theta{}_{\varphi\theta} + \sum_e \left(\Gamma^\theta{}_{\theta e}\Gamma^e{}_{\varphi\varphi} - \Gamma^\theta{}_{\varphi e}\Gamma^e{}_{\varphi\theta}\right)$$

(the formula with $a = \theta$, $b = \varphi$, $c = \theta$, $d = \varphi$)

$$= \partial_\theta(-\sin\theta\cos\theta) - 0 + 0 - \Gamma^\theta{}_{\varphi\varphi}\Gamma^\varphi{}_{\varphi\theta} = -(\cos^2\theta - \sin^2\theta) - (-\sin\theta\cos\theta)\cot\theta$$

(the product rule: the derivative of $\sin\theta\cos\theta$ is $\cos^2\theta - \sin^2\theta$; $\Gamma^\theta{}_{\varphi\theta} = 0$ and every $\Gamma^\theta{}_{\theta e} = 0$; in the last sum only $e = \varphi$ survives)

$$= -\cos^2\theta + \sin^2\theta + \cos^2\theta = \sin^2\theta$$

($\sin\theta\cos\theta\cot\theta = \cos^2\theta$). Raising the second index,

$$\sigma(\theta,\varphi) = R^{\theta\varphi}{}_{\theta\varphi} = g^{\varphi\varphi} R^\theta{}_{\varphi\theta\varphi} = \frac{\sin^2\theta}{a^2\sin^2\theta} = \frac{1}{a^2}$$

(Section 3.17; $g^{\varphi\varphi} = 1/(a^2\sin^2\theta)$). The Gaussian curvature of the sphere is $1/a^2$ at every point: a small sphere is strongly curved, a large one weakly. By Section 3.17, $R^\theta{}_\theta = R^\varphi{}_\varphi = 1/a^2$, the Ricci scalar is $R = 2/a^2$, and the Kretschmann scalar is $K = 4/a^4$. All PROVED here and computed exactly by Notebook 03c (In [7]). With Section 3.16: the turning angle $2\pi(1-\cos\theta_0)$ of a vector carried around a circle of latitude is the enclosed area $2\pi a^2(1-\cos\theta_0)$ times the Gaussian curvature $1/a^2$.

Geodesic deviation. Two meridians cross the equator at right angles, so they start out parallel. On a plane, lines that start parallel keep their distance for ever. On the sphere, after the length s along the meridians from the equator ($\theta = \pi/2 - s/a$), two meridians a small angle $\Delta\varphi$ apart are separated along the circle of latitude by

$$\xi(s) = a\sin\theta\,\Delta\varphi = a\sin\Big(\frac{\pi}{2} - \frac{s}{a}\Big)\Delta\varphi = a\cos\Big(\frac{s}{a}\Big)\Delta\varphi$$

(the length of an arc of the circle of latitude, whose radius is $a\sin\theta$; $\sin(\pi/2 - x) = \cos x$). Differentiating twice with respect to s,

$$\frac{d^2\xi}{ds^2} = -\frac{1}{a^2}\,a\cos\Big(\frac{s}{a}\Big)\Delta\varphi = -\frac{1}{a^2}\,\xi$$

(the chain rule: each derivative of $\cos(s/a)$ brings a factor $1/a$ and turns cos into $-\sin$ and sin into cos). The distance shrinks, and it is zero at $s = \pi a/2$: the meridians meet at the pole. In general the distance $\xi(s)$ between two neighbouring geodesics of a surface obeys the **geodesic deviation**

equation $d^2\xi/ds^2 = -\sigma\,\xi$ with the Gaussian curvature $\sigma = \sigma(1,2)$ of Section 3.17 (a standard theorem, quoted, ASSUMED; on the sphere we have just verified it, with $\sigma = 1/a^2$). On a surface, where all lengths are positive, positive curvature focuses neighbouring geodesics, negative curvature drives them apart, zero curvature does neither. In a spacetime the same theorem also covers free-fall paths that run along a time-like direction (the paths of observers, with their proper time τ in place of the length s), and for them the sign is reversed: if two such paths are a small proper distance ξ apart along the other direction of a coordinate plane with the curvature σ, then $d^2\xi/d\tau^2 = +\sigma\,\xi$ (quoted, ASSUMED in general; the reversal comes from the negative squared length of a time-like step). There a positive curvature drives the paths apart. Section 3.24 verifies this by hand for the observers at rest in the author's metric. Notebook 03c solves the equation with RK4 and finds it equal to the exact distance of two meridians within 8.0×10^{-14} (In [13]).

3.19 Example: curvature where it can be pictured

Notebook 03c applies Sections 3.14 to 3.18 to the plane and the sphere. It defines general functions that compute the Christoffel symbols, the Riemann tensor, the tensor with two upper indices, the Ricci tensor and the Kretschmann scalar of any metric given as a sympy matrix; these are the same formulas that Notebook 03b later applies to the author's metric. It computes the polar Christoffel symbols of the plane and finds its Riemann tensor zero; checks that the constant field e_x has zero covariant derivative; computes the curvature $1/a^2$, the Ricci scalar $2/a^2$ and the Kretschmann scalar $4/a^4$ of the sphere; carries a vector around circles of latitude with RK4 and compares the turning angle with the enclosed area; integrates three geodesics and finds great circles; and solves the geodesic deviation equation. It reads no Revision record (everything is exact mathematics), needs no Rust, runs in about 15 seconds and ends with the line ALL 16 CHECKS PASSED (notebook 03c).

3.20 How to run Notebook 03c

Step 1. What this notebook does and what it needs.

Notebook 03c (Curvature from zero: the plane, the sphere, parallel transport and geodesics) is the file `Revision/textbook/notebooks/03c_sphere_curvature.ipynb` of the repository Dirac_claude. It applies the formulas of the Christoffel symbols, the Riemann tensor, the Ricci tensor and the Ricci and Kretschmann scalars to two surfaces that can be pictured, the flat plane in polar coordinates and a sphere of radius a; it shows that the plane has zero curvature although its Christoffel symbols are not zero, computes the curvature 1/a^2 of the sphere, carries a vector around a circle of latitude by parallel transport (a numerical integration with the Runge-Kutta method) and finds the turning angle equal to the enclosed area divided by a^2, integrates geodesics and shows that they are great circles, and shows that initially parallel geodesics approach each other as the curvature predicts. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private

Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 03c_sphere_curvature.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 03c_sphere_curvature.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 03c_sphere_curvature.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- Revision/textbook/figures/03c.captions.json
- Revision/textbook/figures/03c_1_plane_polar_components.png
- Revision/textbook/figures/03c_2_parallel_transport_sphere.png
- Revision/textbook/figures/03c_3_turning_angle.png
- Revision/textbook/figures/03c_4_geodesics_sphere.png
- Revision/textbook/figures/03c_5_geodesic_deviation.png

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files exist
ALL 16 CHECKS PASSED (notebook 03c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/03c_sphere_curvature.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 03c_sphere_curvature.ipynb
python -m nbconvert --execute --inplace 03c_sphere_curvature.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

3.21 Notebook 03c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/03c_sphere_curvature.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Curvature is the central idea of this chapter, and it is easiest to understand on
surfaces that we can picture. This notebook applies exactly the formulas that
Notebook 03b applies to the author's 8-dimensional metric to two 2-dimensional
surfaces:

- the **flat plane in polar coordinates** $(r, \varphi)$: its Christoffel symbols are
  not zero, yet its Riemann tensor is zero: Christoffel symbols describe the
  coordinates, the Riemann tensor describes the space;
- a **sphere of radius** $a$ in the coordinates $(\theta, \varphi)$ (angle from the
  north pole, angle around the axis): its curvature is $1/a^2$, its Ricci scalar
  $2/a^2$, its Kretschmann scalar $4/a^4$.

Then it shows what curvature does, with numerical integrations (the Runge-Kutta
```

```
method RK4):

- a vector carried around a circle of latitude **without turning** (parallel
  transport) comes back turned, by an angle equal to the enclosed area divided by
  $a^2$;
- the straightest curves of the sphere (**geodesics**) are great circles;
- two geodesics that start **parallel** approach each other and meet, at the rate
  that the curvature predicts.

It draws 5 figures and prints a PASS line for every check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 03c (Curvature from zero: the plane, the sphere, parallel transport and
geodesics) is the file ‘Revision/textbook/notebooks/03c_sphere_curvature.ipynb‘ of the
repository Dirac_claude. It applies the formulas of the Christoffel symbols, the Riemann
tensor, the Ricci tensor and the Ricci and Kretschmann scalars to two surfaces that can
be pictured, the flat plane in polar coordinates and a sphere of radius a; it shows that
the plane has zero curvature although its Christoffel symbols are not zero, computes the
curvature 1/a^2 of the sphere, carries a vector around a circle of latitude by parallel
transport (a numerical integration with the Runge-Kutta method) and finds the turning
angle equal to the enclosed area divided by a^2, integrates geodesics and shows that they
are great circles, and shows that initially parallel geodesics approach each other as the
curvature predicts. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy,
sympy and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
```

```
    jupyter lab 03c_sphere_curvature.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 03c_sphere_curvature.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 03c_sphere_curvature.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/03c.captions.json‘
- ‘Revision/textbook/figures/03c_1_plane_polar_components.png‘
- ‘Revision/textbook/figures/03c_2_parallel_transport_sphere.png‘
- ‘Revision/textbook/figures/03c_3_turning_angle.png‘
- ‘Revision/textbook/figures/03c_4_geodesics_sphere.png‘
- ‘Revision/textbook/figures/03c_5_geodesic_deviation.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS all five figure files exist
    ALL 16 CHECKS PASSED (notebook 03c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/03c_sphere_curvature.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 03c_sphere_curvature.ipynb
  python -m nbconvert --execute --inplace 03c_sphere_curvature.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 03c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 03c (Curvature from zero: the plane, the sphere, parallel transport and
# geodesics) is the file Revision/textbook/notebooks/03c_sphere_curvature.ipynb of the
# repository Dirac_claude. It applies the formulas of the Christoffel symbols, the
# Riemann tensor, the Ricci tensor and the Ricci and Kretschmann scalars to two surfaces
# that can be pictured, the flat plane in polar coordinates and a sphere of radius a; it
# shows that the plane has zero curvature although its Christoffel symbols are not zero,
# computes the curvature 1/a^2 of the sphere, carries a vector around a circle of
# latitude by parallel transport (a numerical integration with the Runge-Kutta method)
# and finds the turning angle equal to the enclosed area divided by a^2, integrates
# geodesics and shows that they are great circles, and shows that initially parallel
# geodesics approach each other as the curvature predicts. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
```

```
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
```

```
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 03c_sphere_curvature.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 03c_sphere_curvature.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
```

```
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 03c_sphere_curvature.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/03c.captions.json
# - Revision/textbook/figures/03c_1_plane_polar_components.png
# - Revision/textbook/figures/03c_2_parallel_transport_sphere.png
# - Revision/textbook/figures/03c_3_turning_angle.png
# - Revision/textbook/figures/03c_4_geodesics_sphere.png
# - Revision/textbook/figures/03c_5_geodesic_deviation.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all five figure files exist
#     ALL 16 CHECKS PASSED (notebook 03c)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/03c_sphere_curvature.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
```

```
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 03c_sphere_curvature.ipynb
#     python -m nbconvert --execute --inplace 03c_sphere_curvature.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "03c"  # this notebook: chapter 03, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
```

```
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 03c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates**: numbers that name the points of a space. **Polar coordinates**
  of the plane: the distance $r$ from the origin and the angle $\varphi$ from the
  horizontal axis; $x = r\cos\varphi$, $y = r\sin\varphi$.
- **Coordinates on the sphere** of radius $a$: $\theta$ (from $0$ at the north pole
  to $\pi$ at the south pole) and $\varphi$ (around the axis); the point is
  $(a\sin\theta\cos\varphi,\ a\sin\theta\sin\varphi,\ a\cos\theta)$.
- **Metric** $g_{\mu\nu}$: the rule $ds^2 = \sum g_{\mu\nu}\,dx^\mu dx^\nu$ for the
  squared length of a small step. Plane: $ds^2 = dr^2 + r^2 d\varphi^2$. Sphere:
  $ds^2 = a^2(d\theta^2 + \sin^2\theta\, d\varphi^2)$.
- **Vector components**: a vector $V = V^r e_r + V^\varphi e_\varphi$ is written with
  the coordinate basis vectors $e_r$ (length 1) and $e_\varphi$ (length $r$).
- **Christoffel symbols** $\Gamma^a{}_{bc}$, **covariant derivative**
  $\nabla_b V^a = \partial_b V^a + \sum_c \Gamma^a{}_{bc} V^c$: the derivative of a
```

```
  vector field corrected for the turning of the coordinate basis.
- **Parallel transport** along a curve $x^a(t)$: carrying a vector so that its
  covariant derivative along the curve is zero,
  $dV^a/dt + \sum_{b,c} \Gamma^a{}_{bc}\, (dx^b/dt)\, V^c = 0$ ("not turning").
- **Geodesic**: a curve whose velocity is parallel transported along itself,
  $d^2x^a/ds^2 + \sum_{b,c}\Gamma^a{}_{bc}\,(dx^b/ds)(dx^c/ds) = 0$; the straightest
  possible curve. **Great circle**: a circle on the sphere whose centre is the
  centre of the sphere (the equator, the meridians, and all their rotations).
- **Riemann tensor** $R^a{}_{bcd}$ (the MTW convention of the Revision records),
  **Gaussian curvature** $\sigma = R^{\theta\varphi}{}_{\theta\varphi}$ of a surface,
  **Ricci scalar** $R$, **Kretschmann scalar** $K$: the measures of curvature.
- **Holonomy** (turning angle): the angle by which a parallel-transported vector is
  turned after going once around a closed curve.
- **RK4**: the classical Runge-Kutta method of order 4 for solving
  $dy/dt = f(t, y)$ step by step.
```

Text cell:

```
## 4. The physical and mathematical situation

Two surfaces, each with two coordinates:

$$\text{plane: } g = \begin{pmatrix} 1 & 0 \\ 0 & r^2 \end{pmatrix},\qquad
\text{sphere: } g = \begin{pmatrix} a^2 & 0 \\ 0 & a^2 \sin^2\theta
\end{pmatrix}.$$

The plane is flat: on a sheet of paper a vector carried around any closed loop
without turning comes back unchanged, parallel lines stay parallel, and the angles
of a triangle add up to $\pi$. The sphere is curved: a vector carried around a loop
comes back turned, lines that start parallel meet (two meridians that cross the
equator at right angles meet at the poles), and the angles of a triangle add up to
more than $\pi$. The Riemann tensor measures exactly this. The formulas are those
used for the author's metric:

$$\Gamma^a{}_{bc} = \tfrac12 \sum_d g^{ad}(\partial_b g_{dc} + \partial_c g_{db}
- \partial_d g_{bc}),\qquad
R^a{}_{bcd} = \partial_c \Gamma^a{}_{bd} - \partial_d \Gamma^a{}_{bc}
+ \sum_e (\Gamma^a{}_{ce}\Gamma^e{}_{bd} - \Gamma^a{}_{de}\Gamma^e{}_{bc}),$$

$R^{ab}{}_{cd} = \sum_e g^{be} R^a{}_{ecd}$, $R^a{}_b = \sum_c R^{ac}{}_{bc}$,
$R = \sum_a R^a{}_a$, $K = \sum R^{ab}{}_{cd} R^{cd}{}_{ab}$. For a surface the only
independent component is the **Gaussian curvature** $R^{12}{}_{12}$; a sphere of
radius $a$ has the Gaussian curvature $1/a^2$, the same at every point.
```

Text cell:

```
## 5. The curvature machinery for any metric

First a technical step. While a cell runs, Jupyter sends what the cell prints to the
notebook in pieces, about one piece every 0.2 seconds, so where one piece ends and
the next begins depends on the speed of the computer. The tools that build this
book read the PASS line of a check together with the line under it that names the
```

Revision record, and they read the two correctly only when both arrive in the same piece. The next cell therefore defines the function ‘in_one_piece(helper)‘: it returns a new function that does exactly what ‘helper‘ does, but lets ‘helper‘ print into a text buffer in memory (an ‘io.StringIO‘, with ‘contextlib.redirect_stdout‘) and then writes the whole buffer with one call of ‘sys.stdout.write‘, which arrives as one piece. (The stars in ‘*arguments‘ and ‘**options‘ pass on all the values the new function is given, whatever they are.) The cell then replaces the helpers ‘check‘ and ‘report‘ of the set-up cell by such functions. Nothing else changes: the same checks, the same PASS lines, the same numbers.

In [2]:

```
import contextlib  # redirect_stdout: send printed text into a buffer
import io  # StringIO: a text buffer in memory
import sys  # sys.stdout: the place where printed text goes


def in_one_piece(helper):
    """A function that does what helper does, with all the lines that helper prints
    written in one piece."""
    def helper_in_one_piece(*arguments, **options):
        collected = io.StringIO()  # an empty text buffer
        with contextlib.redirect_stdout(collected):  # print() writes into it
            helper(*arguments, **options)  # a failing check stops here, as before
        sys.stdout.write(collected.getvalue())  # all the lines with one write
    return helper_in_one_piece


check = in_one_piece(check)  # a PASS line and its record line stay together
report = in_one_piece(report)  # a long RESULT line and its second line, too
say("From now on check and report print each of their results in one piece.")
```

Out [2]:

```
From now on check and report print each of their results in one piece.
```

Text cell:

The next cell imports the packages and defines, for a metric of any size given as a sympy matrix, the functions ‘christoffel‘, ‘riemann‘ (the non-zero $R^a{}_{bcd}$), ‘raise_second‘ ($R^{ab}{}_{cd}$), ‘ricci‘ ($R^a{}_b$), and ‘kretschmann‘. They apply the formulas of section 4 literally, with every result simplified by sympy; they are the same formulas that Notebook 03b applies to the author’s metric.

In [3]:

```
import itertools  # loops over all index combinations
```

```
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols


def christoffel(metric, coordinates):
    """All Christoffel symbols table[a][b][c] of a metric."""
    n = len(coordinates)
    inverse = metric.inv()
    d_metric = [metric.diff(c) for c in coordinates]  # d_metric[c][i, j] = d_c g_ij
    table = [[[0] * n for _ in range(n)] for _ in range(n)]
    for a, b, c in itertools.product(range(n), repeat=3):
        table[a][b][c] = sp.simplify(sum(
            inverse[a, d] * (d_metric[b][d, c] + d_metric[c][d, b]
                             - d_metric[d][b, c]) for d in range(n)) / 2)
    return table


def riemann(table, coordinates):
    """The non-zero R^a_bcd (MTW convention) as a dictionary."""
    n = len(coordinates)
    result = {}
    for a, b, c, d in itertools.product(range(n), repeat=4):
        value = sp.simplify(
            sp.diff(table[a][b][d], coordinates[c])
            - sp.diff(table[a][b][c], coordinates[d])
            + sum(table[a][c][e] * table[e][b][d] - table[a][d][e] * table[e][b][c]
                  for e in range(n)))
        if value != 0:
            result[(a, b, c, d)] = value
    return result


def raise_second(riemann_table, metric):
    """The non-zero R^ab_cd = sum over e of g^(be) R^a_ecd."""
    inverse = metric.inv()
    result = {}
    for (a, e, c, d), value in riemann_table.items():
        for b in range(metric.rows):
            result[(a, b, c, d)] = result.get((a, b, c, d), 0) + inverse[b, e] * value
    return {k: sp.simplify(v) for k, v in result.items() if sp.simplify(v) != 0}


def ricci(mixed_table, n):
    """R^a_b = sum over c of R^ac_bc."""
    result = sp.zeros(n, n)
    for (a, c, b, d), value in mixed_table.items():
        if c == d:
            result[a, b] += value
    return result.applyfunc(sp.simplify)


def kretschmann(mixed_table):
    """K = sum of R^ab_cd R^cd_ab."""
    return sp.simplify(sum(v * mixed_table.get((c, d, a, b), 0)
```

```
                        for (a, b, c, d), v in mixed_table.items()))


say("Defined: christoffel, riemann, raise_second, ricci, kretschmann.")
```

Out [3]:

```
Defined: christoffel, riemann, raise_second, ricci, kretschmann.
```

Text cell:

```
## 6. The flat plane in polar coordinates

With $x = r\cos\varphi$ and $y = r\sin\varphi$ a small step has
$dx = \cos\varphi\, dr - r\sin\varphi\, d\varphi$ and
$dy = \sin\varphi\, dr + r\cos\varphi\, d\varphi$; squaring and adding (the mixed
terms cancel, and $\cos^2 + \sin^2 = 1$) gives $ds^2 = dr^2 + r^2 d\varphi^2$. The
next cell computes the Christoffel symbols of this metric and its Riemann tensor.
By hand: only $g_{\varphi\varphi} = r^2$ depends on a coordinate, so
$\Gamma^r{}_{\varphi\varphi} = -\tfrac12 \partial_r r^2 = -r$ and
$\Gamma^\varphi{}_{r\varphi} = \Gamma^\varphi{}_{\varphi r} = \tfrac12 r^{-2}
\partial_r r^2 = 1/r$. The Riemann tensor must be zero, because the plane is flat.
```

In [4]:

```
r, phi = sp.symbols("r varphi", positive=True)  # polar coordinates, r > 0
plane = sp.diag(1, r ** 2)  # ds^2 = dr^2 + r^2 dphi^2
POLAR = [r, phi]
POLAR_NAMES = ["r", "phi"]
plane_gamma = christoffel(plane, POLAR)
for a, b, c in itertools.product(range(2), repeat=3):
    if plane_gamma[a][b][c] != 0:
        say(f"  Gamma^{POLAR_NAMES[a]}_({POLAR_NAMES[b]} {POLAR_NAMES[c]}) = "
            f"{plane_gamma[a][b][c]}")
check(plane_gamma[0][1][1] == -r and plane_gamma[1][0][1] == 1 / r
      and plane_gamma[1][1][0] == 1 / r,
      "plane in polar coordinates: Gamma^r_(phi phi) = -r, Gamma^phi_(r phi) = 1/r")
plane_riemann = riemann(plane_gamma, POLAR)
say(f"non-zero Riemann components of the plane: {len(plane_riemann)}")
check(plane_riemann == {},
      "the Riemann tensor of the plane is zero: the plane is flat")
```

Out [4]:

```
  Gamma^r_(phi phi) = -r
  Gamma^phi_(r phi) = 1/r
  Gamma^phi_(phi r) = 1/r
PASS plane in polar coordinates: Gamma^r_(phi phi) = -r, Gamma^phi_(r phi) = 1/r
non-zero Riemann components of the plane: 0
PASS the Riemann tensor of the plane is zero: the plane is flat
```

Text cell:

```
The Christoffel symbols of the plane are not zero because the polar basis vectors
turn from point to point. Take the constant vector field $V = e_x$ (one unit to the
right everywhere). In polar components it is $V^r = \cos\varphi$,
$V^\varphi = -\sin\varphi / r$ (because $e_x = \cos\varphi\, e_r - (\sin\varphi /
r)\, e_\varphi$). These components change from point to point although the arrow
does not; the Christoffel terms of the covariant derivative must exactly cancel
their change. The next cell checks $\nabla_b V^a = \partial_b V^a + \sum_c
\Gamma^a{}_{bc} V^c = 0$ for all four combinations of $a$ and $b$.
```

In [5]:

```
V = [sp.cos(phi), -sp.sin(phi) / r]  # the constant field e_x in polar components
nabla = [[sp.simplify(sp.diff(V[a], POLAR[b])
                      + sum(plane_gamma[a][b][c] * V[c] for c in range(2)))
          for b in range(2)] for a in range(2)]
say(f"covariant derivative of e_x: {nabla}")
check(nabla == [[0, 0], [0, 0]],
      "the covariant derivative of the constant field e_x is zero")
```

Out [5]:

```
covariant derivative of e_x: [[0, 0], [0, 0]]
PASS the covariant derivative of the constant field e_x is zero
```

Text cell:

```
The next cell draws this. Left: the polar grid (circles of constant $r$, rays of
constant $\varphi$), the constant field $e_x$ at twelve points (black arrows, all
the same), and at the same points the polar unit vectors $e_r$ (red) and
$e_\varphi / r$ (blue), which turn with $\varphi$. Right: the components of $e_x$
along these unit vectors, $\cos\varphi$ and $-\sin\varphi$, as functions of
$\varphi$: they change all the time although the field is constant.
```

In [6]:

```
BLUE, ORANGE, AQUA, RED, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#e34948", "#4a3aa7"
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.6))
fig.subplots_adjust(wspace=0.3)  # room for the label of the right vertical axis
ax = axes[0]
angles = np.linspace(0.0, 2 * np.pi, 400)
for radius in (0.5, 1.0, 1.5, 2.0):  # circles of constant r
    ax.plot(radius * np.cos(angles), radius * np.sin(angles), color="grey", lw=0.6)
for ray in np.arange(12) * np.pi / 6:  # rays of constant phi
    ax.plot([0, 2.2 * np.cos(ray)], [0, 2.2 * np.sin(ray)], color="grey", lw=0.6)
points = np.arange(12) * np.pi / 6 + np.pi / 12  # the angles of the 12 points
px, py = 1.5 * np.cos(points), 1.5 * np.sin(points)
ax.quiver(px, py, np.ones(12), np.zeros(12), color="black", scale=8, width=0.006,
```

```
          label="constant field $e_x$")
ax.quiver(px, py, np.cos(points), np.sin(points), color=RED, scale=12,
          width=0.005, label="unit vector $e_r$")
ax.quiver(px, py, -np.sin(points), np.cos(points), color=BLUE, scale=12,
          width=0.005, label="unit vector $e_\\varphi / r$")
ax.set_aspect("equal")
ax.set_xlim(-2.4, 2.4)
ax.set_ylim(-2.4, 2.4)
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
ax.set_title("A constant field on the polar grid")
ax.legend(fontsize=7, loc="lower left")
ax = axes[1]
ax.plot(angles, np.cos(angles), color=RED, lw=1.8,
        label="component along $e_r$: $\\cos\\varphi$")
ax.plot(angles, -np.sin(angles), color=BLUE, lw=1.8, ls="--",
        label="component along $e_\\varphi/r$: $-\\sin\\varphi$")
ax.set_xlabel("$\\varphi$ (radians)")
ax.set_ylabel("component (no unit)")
ax.set_title("Its polar components change")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "plane_polar_components",
            "The flat plane in polar coordinates. Left: the polar grid (grey circles "
            "of constant $r$ and rays of constant $\\varphi$), the constant vector "
            "field $e_x$ at twelve points on the circle $r = 1.5$ (black arrows, all "
            "equal), and the polar unit vectors $e_r$ (red) and $e_\\varphi/r$ "
            "(blue) at the same points, which turn with the angle; axes $x$ and $y$ "
            "(pure numbers). Right: the components of $e_x$ along $e_r$ "
            "($\\cos\\varphi$, solid) and along $e_\\varphi/r$ ($-\\sin\\varphi$, "
            "dashed) as functions of $\\varphi$ in radians. The components change "
            "although the field does not: the Christoffel symbols of the polar "
            "coordinates account for this change, while the Riemann tensor of the "
            "plane is zero.")
check(np.allclose(np.cos(points) ** 2 + np.sin(points) ** 2, 1.0),
      "the components of the unit field e_x have length 1 at every point")
```

Out [6]:

```
Figure 03c.1 saved as Revision/textbook/figures/03c_1_plane_polar_components.png
PASS the components of the unit field e_x have length 1 at every point
```

Text cell:

```
## 7. The sphere of radius $a$

The point $(a\sin\theta\cos\varphi,\ a\sin\theta\sin\varphi,\ a\cos\theta)$ moves by
$a\, d\theta$ along a meridian and by $a\sin\theta\, d\varphi$ along a circle of
latitude, at right angles, so $ds^2 = a^2(d\theta^2 + \sin^2\theta\, d\varphi^2)$.
By hand, with $\partial_\theta(a^2\sin^2\theta) = 2a^2\sin\theta\cos\theta$:
$\Gamma^\theta{}_{\varphi\varphi} = -\sin\theta\cos\theta$ and
$\Gamma^\varphi{}_{\theta\varphi} = \cos\theta/\sin\theta = \cot\theta$. The next
cell computes these, the Riemann tensor, its Gaussian curvature
```

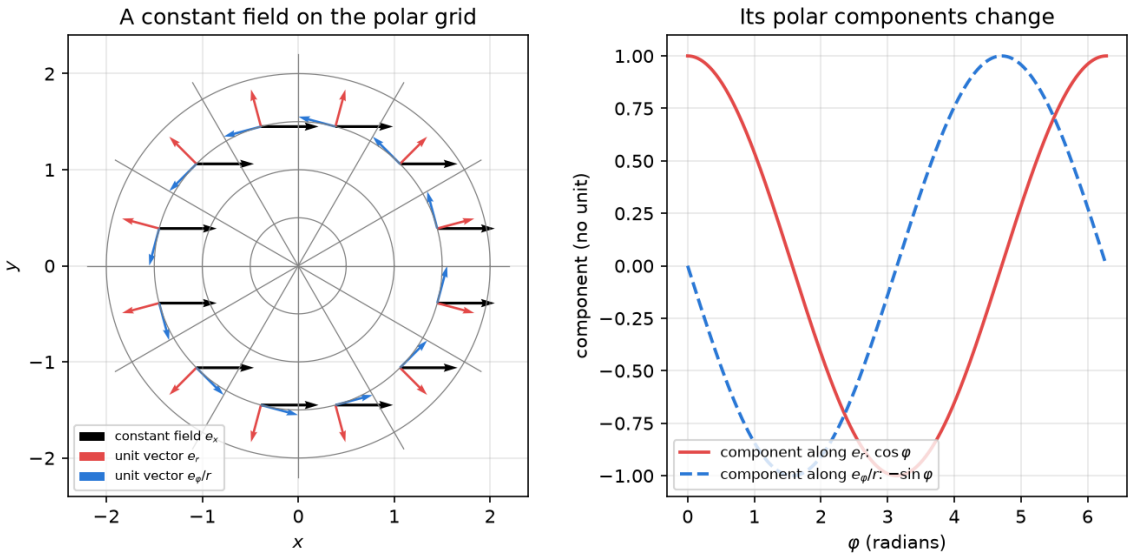

Figure 99: The flat plane in polar coordinates. Left: the polar grid (grey circles of constant r and rays of constant φ), the constant vector field e_x at twelve points on the circle $r = 1.5$ (black arrows, all equal), and the polar unit vectors e_r (red) and e_φ/r (blue) at the same points, which turn with the angle; axes x and y (pure numbers). Right: the components of e_x along e_r ($\cos\varphi$, solid) and along e_φ/r ($-\sin\varphi$, dashed) as functions of φ in radians. The components change although the field does not: the Christoffel symbols of the polar coordinates account for this change, while the Riemann tensor of the plane is zero. (Notebook 03c, figure 1.)

```
$R^{\theta\varphi}{}_{\theta\varphi}$, the Ricci tensor, the Ricci scalar and the
Kretschmann scalar, and checks $1/a^2$, $2/a^2$ and $4/a^4$. (sympy prints
$\Gamma^\theta{}_{\varphi\varphi}$ as $-\sin(2\theta)/2$, which is the same
number because $\sin 2\theta = 2\sin\theta\cos\theta$, and $\cot\theta$ as
$1/\tan\theta$.)
```

In [7]:

```
theta = sp.symbols("theta", positive=True)
a = sp.symbols("a", positive=True)  # the radius of the sphere
sphere = sp.diag(a ** 2, a ** 2 * sp.sin(theta) ** 2)
ANGLES = [theta, phi]
ANGLE_NAMES = ["theta", "phi"]
sphere_gamma = christoffel(sphere, ANGLES)
for i, j, k in itertools.product(range(2), repeat=3):
    if sphere_gamma[i][j][k] != 0:
        say(f"  Gamma^{ANGLE_NAMES[i]}_({ANGLE_NAMES[j]} {ANGLE_NAMES[k]}) = "
            f"{sphere_gamma[i][j][k]}")
sphere_riemann = riemann(sphere_gamma, ANGLES)
sphere_mixed = raise_second(sphere_riemann, sphere)
for key, value in sorted(sphere_mixed.items()):
    say(f"  R^({ANGLE_NAMES[key[0]]} {ANGLE_NAMES[key[1]]})_"
        f"({ANGLE_NAMES[key[2]]} {ANGLE_NAMES[key[3]]}) = {value}")
sphere_ricci = ricci(sphere_mixed, 2)
sphere_R = sp.simplify(sphere_ricci.trace())
```

```
sphere_K = kretschmann(sphere_mixed)
say(f"Ricci tensor R^a_b = {sphere_ricci.tolist()}")
say(f"Ricci scalar R = {sphere_R},  Kretschmann scalar K = {sphere_K}")
check(sp.simplify(sphere_gamma[0][1][1] + sp.sin(theta) * sp.cos(theta)) == 0
      and sp.simplify(sphere_gamma[1][0][1] - sp.cot(theta)) == 0,
      "sphere: Gamma^theta_(phi phi) = -sin cos, Gamma^phi_(theta phi) = cot theta")
check(sphere_mixed[(0, 1, 0, 1)] == 1 / a ** 2,
      "the Gaussian curvature of the sphere is 1/a^2 at every point")
check(sphere_R == 2 / a ** 2 and sphere_K == 4 / a ** 4,
      "Ricci scalar 2/a^2 and Kretschmann scalar 4/a^4")
```

Out [7]:

```
  Gamma^theta_(phi phi) = -sin(2*theta)/2
  Gamma^phi_(theta phi) = 1/tan(theta)
  Gamma^phi_(phi theta) = 1/tan(theta)
  R^(theta phi)_(theta phi) = a**(-2)
  R^(theta phi)_(phi theta) = -1/a**2
  R^(phi theta)_(theta phi) = -1/a**2
  R^(phi theta)_(phi theta) = a**(-2)
Ricci tensor R^a_b = [[a**(-2), 0], [0, a**(-2)]]
Ricci scalar R = 2/a**2,  Kretschmann scalar K = 4/a**4
PASS sphere: Gamma^theta_(phi phi) = -sin cos, Gamma^phi_(theta phi) = cot theta
PASS the Gaussian curvature of the sphere is 1/a^2 at every point
PASS Ricci scalar 2/a^2 and Kretschmann scalar 4/a^4
```

Text cell:

```
## 8. Parallel transport around a circle of latitude

Walk once around the circle of latitude $\theta = \theta_0$ (so $\varphi$ runs from
$0$ to $2\pi$ and $d\theta/d\varphi = 0$), carrying a vector without turning it. The
transport equation $dV^a/d\varphi + \sum_c \Gamma^a{}_{\varphi c} V^c = 0$ has only
two terms, one for each component:

$$\frac{dV^\theta}{d\varphi} = \sin\theta_0\cos\theta_0\, V^\varphi,\qquad
\frac{dV^\varphi}{d\varphi} = -\cot\theta_0\, V^\theta .$$

With the lengths $u = a V^\theta$ and $w = a\sin\theta_0\, V^\varphi$ along the unit
vectors $e_\theta$ (south) and $e_\varphi/(a \sin\theta_0)$ (east) this reads
$du/d\varphi = \cos\theta_0\, w$, $dw/d\varphi = -\cos\theta_0\, u$: the vector
turns relative to the local south-east frame at the constant rate $\cos\theta_0$.
After one round it has turned by $-2\pi\cos\theta_0$, which is the same direction as
a turn by $2\pi(1 - \cos\theta_0)$ (they differ by one whole turn). And
$2\pi a^2(1 - \cos\theta_0)$ is the area of the cap north of the circle: **the
turning angle is the enclosed area times the curvature $1/a^2$**. On the plane the
angle would be zero.

The next cell defines a general RK4 integrator and integrates the two equations
numerically with $a = 1$, starting with the unit vector pointing south
($u = 1$, $w = 0$), for $\theta_0 = \pi/3$; there $1 - \cos\theta_0 = 1/2$, so the
vector must come back pointing north, exactly reversed.
```

In [8]:

```
def rk4(f, y0, t0, t1, steps):
    """Solve dy/dt = f(t, y) from t0 to t1 with the classical Runge-Kutta method in
    steps equal steps; return the times and the states (one row per time)."""
    h = (t1 - t0) / steps
    times = t0 + h * np.arange(steps + 1)
    states = np.zeros((steps + 1, len(y0)))
    states[0] = y0
    for n in range(steps):
        t, y = times[n], states[n]
        k1 = f(t, y)
        k2 = f(t + h / 2, y + h / 2 * k1)
        k3 = f(t + h / 2, y + h / 2 * k2)
        k4 = f(t + h, y + h * k3)
        states[n + 1] = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return times, states


def transport(theta0, steps=2000):
    """Parallel transport of the unit south vector once around latitude theta0
    (a = 1): the components V^theta, V^phi as functions of phi."""
    def f(t, y):
        return np.array([np.sin(theta0) * np.cos(theta0) * y[1],
                         -np.cos(theta0) / np.sin(theta0) * y[0]])
    return rk4(f, np.array([1.0, 0.0]), 0.0, 2 * np.pi, steps)


def turning_angle(theta0):
    """The angle (0 to 2 pi) by which the vector comes back turned."""
    _, states = transport(theta0)
    u, w = states[-1, 0], np.sin(theta0) * states[-1, 1]  # lengths south, east
    return np.mod(np.arctan2(w, u), 2 * np.pi)


theta0 = np.pi / 3
phis, states = transport(theta0)
lengths = np.sqrt(states[:, 0] ** 2 + (np.sin(theta0) * states[:, 1]) ** 2)
# round to 12 digits; adding 0.0 turns a rounded -0.0 into 0.0
say(f"start: V = {(states[0].round(12) + 0.0).tolist()},  end: V = "
    f"{(states[-1].round(12) + 0.0).tolist()}")
check(np.max(np.abs(lengths - 1.0)) < 1e-12,
      "parallel transport keeps the length of the vector")
check(np.max(np.abs(states[-1] - np.array([-1.0, 0.0]))) < 1e-10,
      "at theta0 = pi/3 the vector comes back exactly reversed")
```

Out [8]:

```
start: V = [1.0, 0.0],  end: V = [-1.0, 0.0]
PASS parallel transport keeps the length of the vector
PASS at theta0 = pi/3 the vector comes back exactly reversed
```

Text cell:

The next cell draws the transport on the sphere: a wire model of the sphere, the circle of latitude $\theta_0 = \pi/3$, and the transported vector at 17 points of the circle, one every $\pi/8$ (the first and the last at the same place, in red). The arrow at φ is $u\, e_\theta + w\, e_\varphi^{\rm unit}$ with the 3-dimensional unit vectors $e_\theta = (\cos\theta\cos\varphi,\ \cos\theta\sin\varphi,\ -\sin\theta)$ and $e_\varphi^{\rm unit} = (-\sin\varphi,\ \cos\varphi,\ 0)$.

In [9]:

```
def point(th, ph):
    """The point of the unit sphere at the angles th, ph."""
    return np.array([np.sin(th) * np.cos(ph), np.sin(th) * np.sin(ph), np.cos(th)])


def wire_sphere(ax):
    """Draw a light wire model of the unit sphere into the 3-D axes ax."""
    th, ph = np.meshgrid(np.linspace(0, np.pi, 13), np.linspace(0, 2 * np.pi, 25))
    ax.plot_wireframe(np.sin(th) * np.cos(ph), np.sin(th) * np.sin(ph), np.cos(th),
                      color="grey", lw=0.3)
    ax.set_box_aspect((1, 1, 1), zoom=0.9)  # a little smaller: the labels fit
    ax.set_xlim(-1, 1)
    ax.set_ylim(-1, 1)
    ax.set_zlim(-1, 1)
    ax.view_init(elev=25, azim=-60)
    for set_ticks in (ax.set_xticks, ax.set_yticks, ax.set_zticks):
        set_ticks([-1.0, 0.0, 1.0])  # three ticks per axis: labels do not overlap
    ax.tick_params(labelsize=8)
    ax.set_xlabel("$X$")
    ax.set_ylabel("$Y$")
    ax.set_zlabel("$Z$")


fig = plt.figure(figsize=(6.4, 6.0))
ax = fig.add_subplot(projection="3d")
wire_sphere(ax)
circle = np.array([point(theta0, p) for p in np.linspace(0, 2 * np.pi, 200)])
ax.plot(circle[:, 0], circle[:, 1], circle[:, 2], color=VIOLET, lw=1.8)
for n in range(0, 2001, 125):  # 17 points, the last one equal to the first
    ph = phis[n]
    u, w = states[n, 0], np.sin(theta0) * states[n, 1]
    e_theta = np.array([np.cos(theta0) * np.cos(ph), np.cos(theta0) * np.sin(ph),
                        -np.sin(theta0)])
    e_phi = np.array([-np.sin(ph), np.cos(ph), 0.0])
    arrow = 0.35 * (u * e_theta + w * e_phi)
    start = point(theta0, ph)
    colour = RED if n in (0, 2000) else BLUE
    ax.quiver(*start, *arrow, color=colour, lw=1.5, arrow_length_ratio=0.25)
ax.set_title("Parallel transport around the latitude $\\theta_0 = \\pi/3$")
save_figure(fig, "parallel_transport_sphere",
            "Parallel transport on the unit sphere (wire model; axes $X$, $Y$, $Z$ "
            "in units of the radius $a = 1$): a vector that starts at "
            "$\\varphi = 0$ on the circle of latitude $\\theta_0 = \\pi/3$ (violet) "
```

```
            "pointing south is carried once around the circle without turning; the "
            "blue arrows show it at fifteen intermediate points, computed with RK4. "
            "Seen from the local south and east directions it turns steadily, and "
            "when it is back at the start (red arrow, drawn at both ends) it points "
            "north: it has turned by $\\pi$, the area $2\\pi(1 - \\cos\\theta_0) = "
            "\\pi$ of the cap above the circle divided by $a^2$.")
check(abs(turning_angle(theta0) - np.pi) < 1e-10,
      "the turning angle at theta0 = pi/3 is pi")
```

Out [9]:

```
Figure 03c.2 saved as Revision/textbook/figures/03c_2_parallel_transport_sphere.png
PASS the turning angle at theta0 = pi/3 is pi
```

Text cell:

The next cell repeats the transport for 15 latitudes θ_0 between 0.1 and 1.5 and compares the measured turning angle with the area of the enclosed cap divided by a^2, $2\pi(1 - \cos\theta_0)$. On the flat plane the turning angle of every loop would be 0 (the dotted line).

In [10]:

```
latitudes = np.linspace(0.1, 1.5, 15)
measured = np.array([turning_angle(t0) for t0 in latitudes])
curve = np.linspace(0.0, np.pi / 2, 300)
fig, ax = plt.subplots()
ax.plot(curve, 2 * np.pi * (1 - np.cos(curve)), color=VIOLET, lw=1.8,
        label="cap area / $a^2 = 2\\pi(1 - \\cos\\theta_0)$")
ax.plot(latitudes, measured, "o", color=ORANGE, ms=7,
        label="turning angle measured by parallel transport")
ax.plot(curve, np.zeros_like(curve), ":", color="black", lw=1.2,
        label="flat plane: no turning")
ax.set_xlabel("latitude angle $\\theta_0$ of the loop (radians from the pole)")
ax.set_ylabel("angle (radians)")
ax.set_title("Curvature turns vectors: angle = enclosed area $\\times$ $1/a^2$")
ax.legend(fontsize=8)
deviation = np.max(np.abs(measured - 2 * np.pi * (1 - np.cos(latitudes))))
report("largest difference between the turning angle and area/a^2",
       f"{deviation:.1e}", "radians")
save_figure(fig, "turning_angle",
            "The turning angle of a vector carried once around a circle of latitude "
            "on a sphere of radius $a = 1$ (orange dots, computed by parallel "
            "transport with RK4 for 15 circles) and the area of the cap enclosed by "
            "the circle divided by $a^2$, $2\\pi(1 - \\cos\\theta_0)$ (violet line), "
            "against the angle $\\theta_0$ of the circle from the north pole in "
            "radians; vertical axis the angle in radians. The dots lie on the line: "
            "the turning angle equals the enclosed area times the Gaussian "
            "curvature $1/a^2$. On the flat plane (dotted line) a vector never "
            "turns.")
check(deviation < 1e-9, "the turning angle equals the enclosed area over a^2")
```

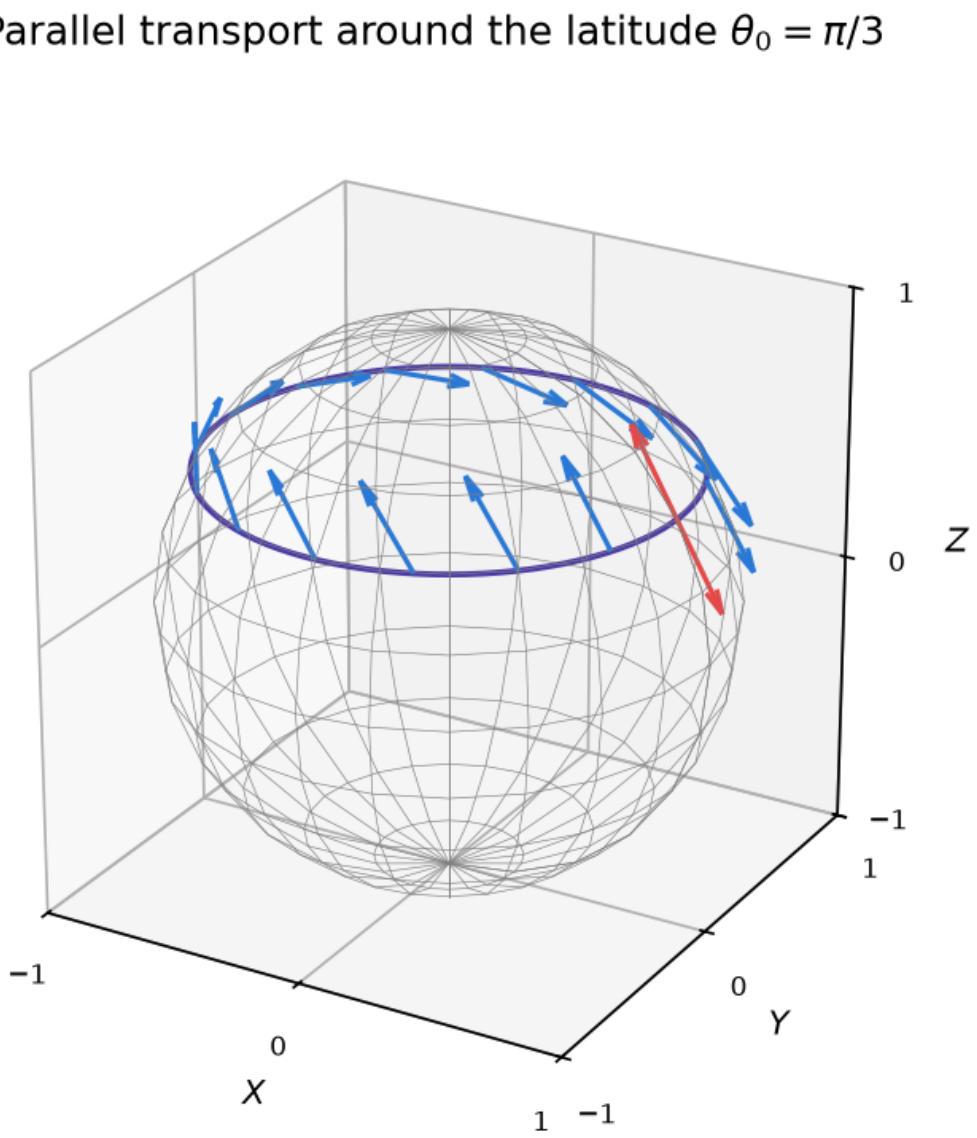

Figure 100: Parallel transport on the unit sphere (wire model; axes X, Y, Z in units of the radius $a = 1$): a vector that starts at $\varphi = 0$ on the circle of latitude $\theta_0 = \pi/3$ (violet) pointing south is carried once around the circle without turning; the blue arrows show it at fifteen intermediate points, computed with RK4. Seen from the local south and east directions it turns steadily, and when it is back at the start (red arrow, drawn at both ends) it points north: it has turned by π, the area $2\pi(1 - \cos\theta_0) = \pi$ of the cap above the circle divided by a^2. (Notebook 03c, figure 2.)

Out [10]:

```
RESULT largest difference between the turning angle and area/a^2 = 5.0e-12 radians
```

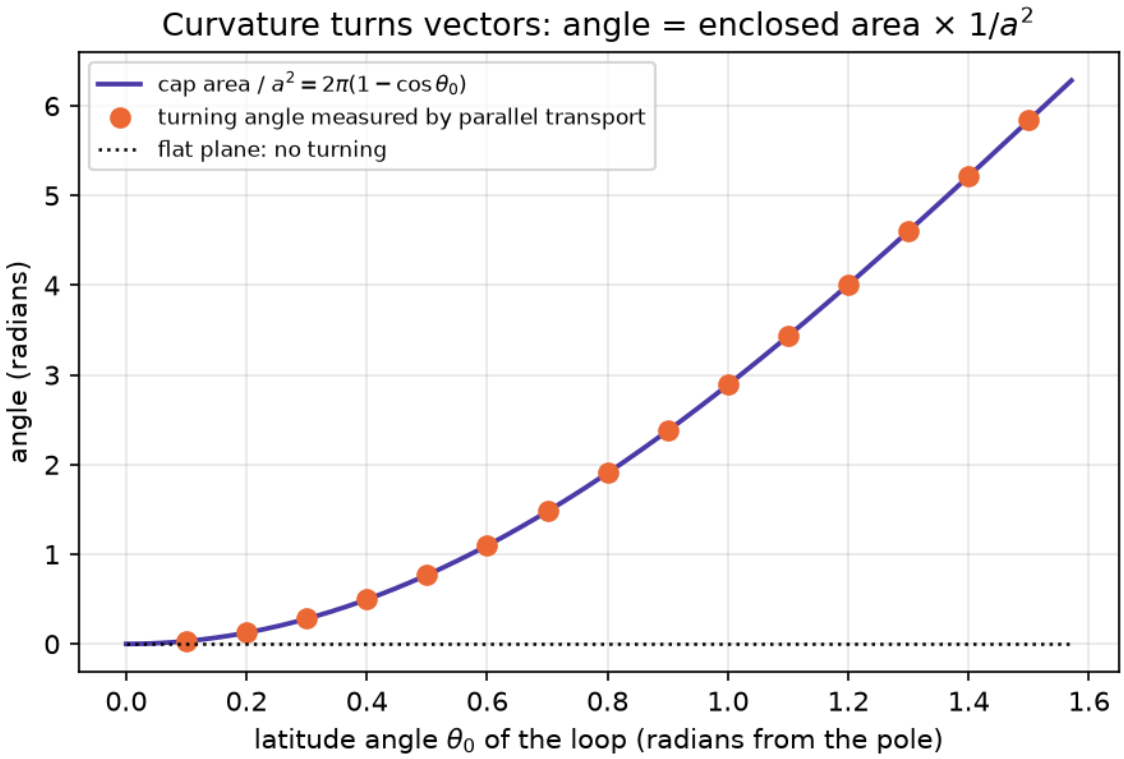

Figure 101: The turning angle of a vector carried once around a circle of latitude on a sphere of radius $a = 1$ (orange dots, computed by parallel transport with RK4 for 15 circles) and the area of the cap enclosed by the circle divided by a^2, $2\pi(1 - \cos\theta_0)$ (violet line), against the angle θ_0 of the circle from the north pole in radians; vertical axis the angle in radians. The dots lie on the line: the turning angle equals the enclosed area times the Gaussian curvature $1/a^2$. On the flat plane (dotted line) a vector never turns. (Notebook 03c, figure 3.)

Out [10]:

```
Figure 03c.3 saved as Revision/textbook/figures/03c_3_turning_angle.png
PASS the turning angle equals the enclosed area over a^2
```

Text cell:

```
## 9. Geodesics are great circles

The geodesic equations of the sphere follow from its two Christoffel symbols
($s$ is the length along the curve, $a = 1$):

$$\frac{d^2\theta}{ds^2} = \sin\theta\cos\theta \Bigl(\frac{d\varphi}{ds}\Bigr)^2,
```

```
\qquad \frac{d^2\varphi}{ds^2} = -2\cot\theta\, \frac{d\theta}{ds}
\frac{d\varphi}{ds}.$$

(The factor 2 comes from the two equal terms $\Gamma^\varphi{}_{\theta\varphi}$ and
$\Gamma^\varphi{}_{\varphi\theta}$.) The next cell writes them as four first-order
equations for $(\theta, \varphi, d\theta/ds, d\varphi/ds)$, integrates three
geodesics with RK4 that leave the point $\theta = \pi/2$, $\varphi = 0$ on the
equator in three directions (at $20$, $50$ and $80$ degrees north of east), each for
the length $2\pi$ (once around), and checks that each stays in the plane through the
centre of the sphere spanned by its starting point and starting direction, which
means it is a great circle. It also checks that the circle of latitude
$\theta_0 = \pi/3$ is NOT a geodesic: along it $d^2\theta/ds^2 = 0$, but the
right-hand side $\sin\theta_0\cos\theta_0 (d\varphi/ds)^2$ is not zero.
```

In [11]:

```
def geodesic_rhs(s, y):
    """The geodesic equations of the unit sphere for y = (theta, phi, dtheta,
    dphi)."""
    th, ph, dth, dph = y
    return np.array([dth, dph, np.sin(th) * np.cos(th) * dph ** 2,
                     -2 * np.cos(th) / np.sin(th) * dth * dph])


geodesics = {}
for degrees in (20, 50, 80):
    alpha = np.radians(degrees)  # the direction, measured from east towards north
    # unit speed at the equator: dtheta/ds = -sin(alpha) (north), dphi/ds = cos(alpha)
    start = np.array([np.pi / 2, 0.0, -np.sin(alpha), np.cos(alpha)])
    _, path = rk4(geodesic_rhs, start, 0.0, 2 * np.pi, 4000)
    xyz = np.array([point(th, ph) for th, ph in path[:, :2]])
    # the starting direction in X, Y, Z: east is (0, 1, 0), north is (0, 0, 1)
    velocity0 = np.array([0.0, np.cos(alpha), np.sin(alpha)])
    normal = np.cross(xyz[0], velocity0)  # perpendicular to the expected plane
    geodesics[degrees] = (xyz, np.max(np.abs(xyz @ normal)))
    say(f"  direction {degrees:2d} degrees: largest distance from the plane "
        f"= {geodesics[degrees][1]:.1e}, end point back at start: "
        f"{np.allclose(xyz[-1], xyz[0], atol=1e-8)}")
check(all(distance < 1e-8 for _, distance in geodesics.values()),
      "the three geodesics stay in planes through the centre: great circles")
theta_circle = np.pi / 3
residual = np.sin(theta_circle) * np.cos(theta_circle) * (1 / np.sin(theta_circle)) ** 2
report("geodesic equation of the latitude theta0 = pi/3: missing acceleration",
       f"{residual:.6f}")
check(residual > 0.5, "the circle of latitude theta0 = pi/3 is not a geodesic")
```

Out [11]:

```
  direction 20 degrees: largest distance from the plane = 1.2e-13, end point back at
    start: True
  direction 50 degrees: largest distance from the plane = 6.2e-13, end point back at
    start: True
```

```
  direction 80 degrees: largest distance from the plane = 7.1e-11, end point back at
    start: True
PASS the three geodesics stay in planes through the centre: great circles
RESULT geodesic equation of the latitude theta0 = pi/3: missing acceleration = 0.577350
PASS the circle of latitude theta0 = pi/3 is not a geodesic
```

Text cell:

```
The next cell draws the three geodesics and, for contrast, the circle of latitude
$\theta_0 = \pi/3$, on the wire model of the sphere.
```

In [12]:

```
fig = plt.figure(figsize=(6.4, 6.0))
ax = fig.add_subplot(projection="3d")
wire_sphere(ax)
for (degrees, (xyz, _)), colour, style in zip(geodesics.items(),
                                              (RED, ORANGE, AQUA), ("-", "--", "-.")):
    ax.plot(xyz[:, 0], xyz[:, 1], xyz[:, 2], color=colour, ls=style, lw=1.8,
            label=f"geodesic leaving at {degrees} degrees")
ax.plot(circle[:, 0], circle[:, 1], circle[:, 2], color=VIOLET, ls=":", lw=1.8,
        label="circle of latitude (not a geodesic)")
ax.scatter([1.0], [0.0], [0.0], color="black", s=30)
ax.set_title("Geodesics of the sphere from one point")
ax.legend(fontsize=7, loc="upper left")
save_figure(fig, "geodesics_sphere",
            "Three geodesics of the unit sphere (axes $X$, $Y$, $Z$ in units of the "
            "radius), computed with RK4 from the geodesic equations, leaving the "
            "point $(1, 0, 0)$ on the equator (black dot) at $20$, $50$ and $80$ "
            "degrees north of east (solid, dashed and dash-dotted); each is a great "
            "circle, a circle whose centre is the centre of the sphere, and comes "
            "back to its start after the length $2\\pi a$. The dotted circle of "
            "latitude $\\theta_0 = \\pi/3$ is not a geodesic: a traveller who follows "
            "it must keep turning towards the pole.")
check(all(np.allclose(xyz[-1], xyz[0], atol=1e-8) for xyz, _ in geodesics.values()),
      "every geodesic returns to its start after the length 2 pi")
```

Out [12]:

```
Figure 03c.4 saved as Revision/textbook/figures/03c_4_geodesics_sphere.png
PASS every geodesic returns to its start after the length 2 pi
```

Text cell:

```
## 10. Initially parallel geodesics meet

Two meridians cross the equator at right angles, so they start parallel; on a
plane they would stay at the same distance for ever. On the sphere their distance,
measured along the circle of latitude, is $a\sin\theta\,\Delta\varphi =
```

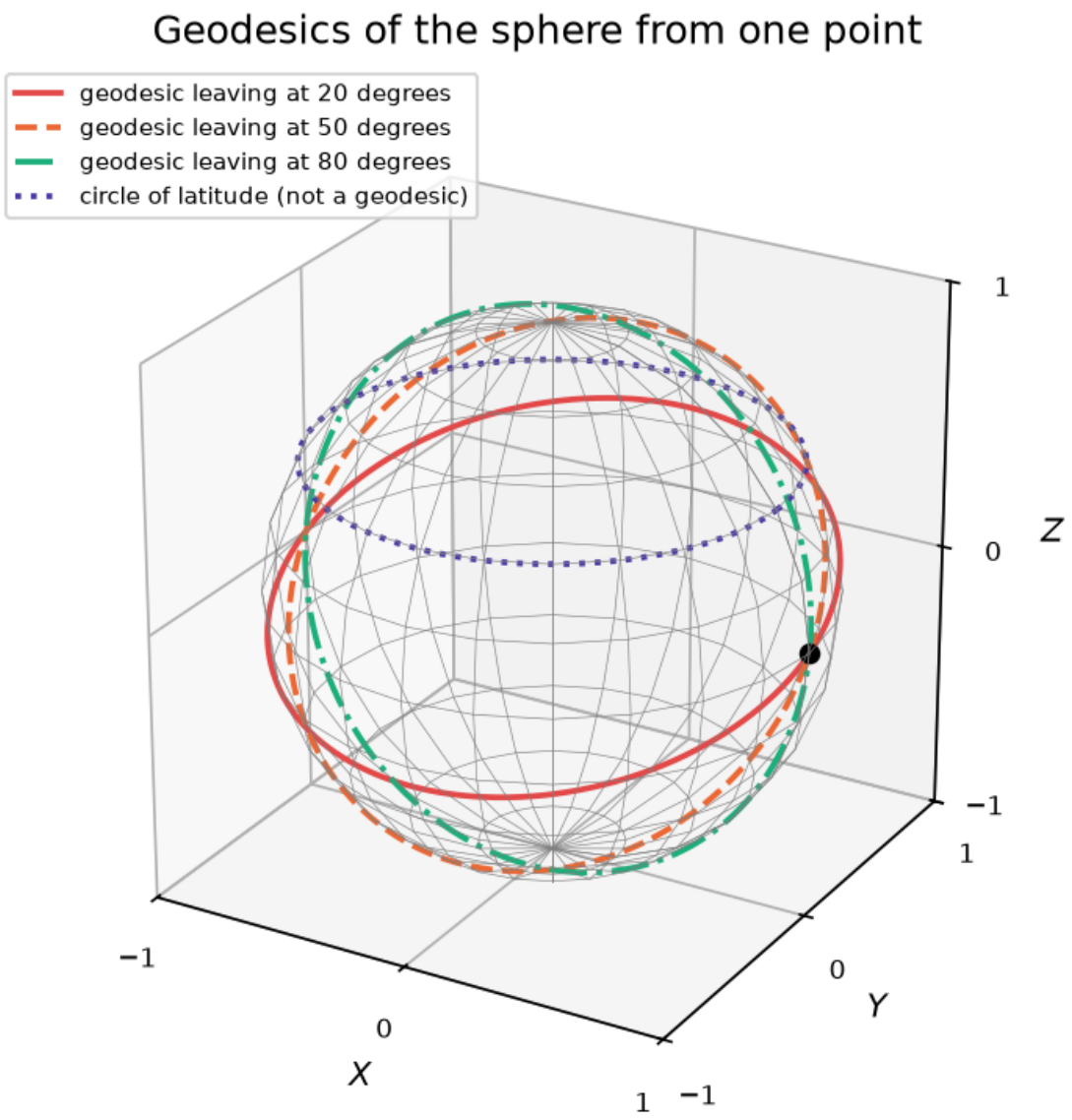

Figure 102: Three geodesics of the unit sphere (axes X, Y, Z in units of the radius), computed with RK4 from the geodesic equations, leaving the point $(1, 0, 0)$ on the equator (black dot) at 20, 50 and 80 degrees north of east (solid, dashed and dash-dotted); each is a great circle, a circle whose centre is the centre of the sphere, and comes back to its start after the length $2\pi a$. The dotted circle of latitude $\theta_0 = \pi/3$ is not a geodesic: a traveller who follows it must keep turning towards the pole. (Notebook 03c, figure 4.)

```
a\cos(s/a)\,\Delta\varphi$ after the length $s$ ($\theta = \pi/2 - s/a$): they
approach each other and meet at the pole, $s = \pi a/2$. In general the distance
$\xi(s)$ between two neighbouring geodesics obeys the **geodesic deviation
equation** $d^2\xi/ds^2 = -\sigma\,\xi$ with the Gaussian curvature $\sigma =
R^{\theta\varphi}{}_{\theta\varphi} = 1/a^2$ computed in section 7 (this sign rule
holds for geodesics and distances measured with ordinary positive lengths, as on a
surface: a positive curvature brings neighbouring geodesics together). The next cell
solves this equation with RK4 from $\xi = 1$, $d\xi/ds = 0$ (in units of the starting
distance, $a = 1$), compares it with the exact distance of two meridians
$\Delta\varphi = 10^{-3}$ apart (the length of the straight chord between them,
divided by its starting value), and draws both together with the flat plane.
```

In [13]:

```
sigma = float(sphere_mixed[(0, 1, 0, 1)].subs(a, 1))  # the Gaussian curvature, a = 1
s_values, deviation_states = rk4(lambda s, y: np.array([y[1], -sigma * y[0]]),
                                 np.array([1.0, 0.0]), 0.0, np.pi / 2, 1000)
d_phi = 1e-3  # the angle between the two meridians
chord = np.array([np.linalg.norm(point(np.pi / 2 - s, 0.0) - point(np.pi / 2 - s, d_phi))
                  for s in s_values]) / np.linalg.norm(point(np.pi / 2, 0.0)
                                                       - point(np.pi / 2, d_phi))
fig, ax = plt.subplots()
ax.plot(s_values, chord, color=BLUE, lw=3.0, alpha=0.6,
        label="two meridians $10^{-3}$ apart (exact)")
ax.plot(s_values, deviation_states[:, 0], "--", color=RED, lw=1.8,
        label="deviation equation $\\xi^{\\prime\\prime} = -\\xi/a^2$ (RK4)")
ax.plot(s_values, np.ones_like(s_values), ":", color="black", lw=1.2,
        label="flat plane: parallel lines keep their distance")
ax.set_xlabel("length $s$ along the geodesics (unit $a$)")
ax.set_ylabel("distance / starting distance")
ax.set_title("Initially parallel geodesics meet at the pole")
ax.legend(fontsize=8)
agreement = np.max(np.abs(chord - deviation_states[:, 0]))
report("largest difference between the meridians and the deviation equation",
       f"{agreement:.1e}")
save_figure(fig, "geodesic_deviation",
            "The distance between two neighbouring geodesics that start parallel, "
            "divided by its starting value, against the length $s$ travelled along "
            "them in units of the radius $a$: two meridians of the unit sphere "
            "$\\Delta\\varphi = 10^{-3}$ apart that leave the equator northwards "
            "(thick blue line, exact), the solution of the geodesic deviation "
            "equation $\\xi^{\\prime\\prime} = -\\sigma\\,\\xi$ with the Gaussian "
            "curvature $\\sigma = 1/a^2$ computed from the Riemann tensor (dashed "
            "red, RK4), and the flat plane (dotted). On the sphere the distance "
            "shrinks like $\\cos(s/a)$ and becomes zero at the pole, "
            "$s = \\pi a/2$; on the plane it stays constant.")
check(agreement < 1e-6,
      "the curvature 1/a^2 predicts how fast neighbouring geodesics approach")
```

Out [13]:

```
RESULT largest difference between the meridians and the deviation equation = 8.0e-14
```

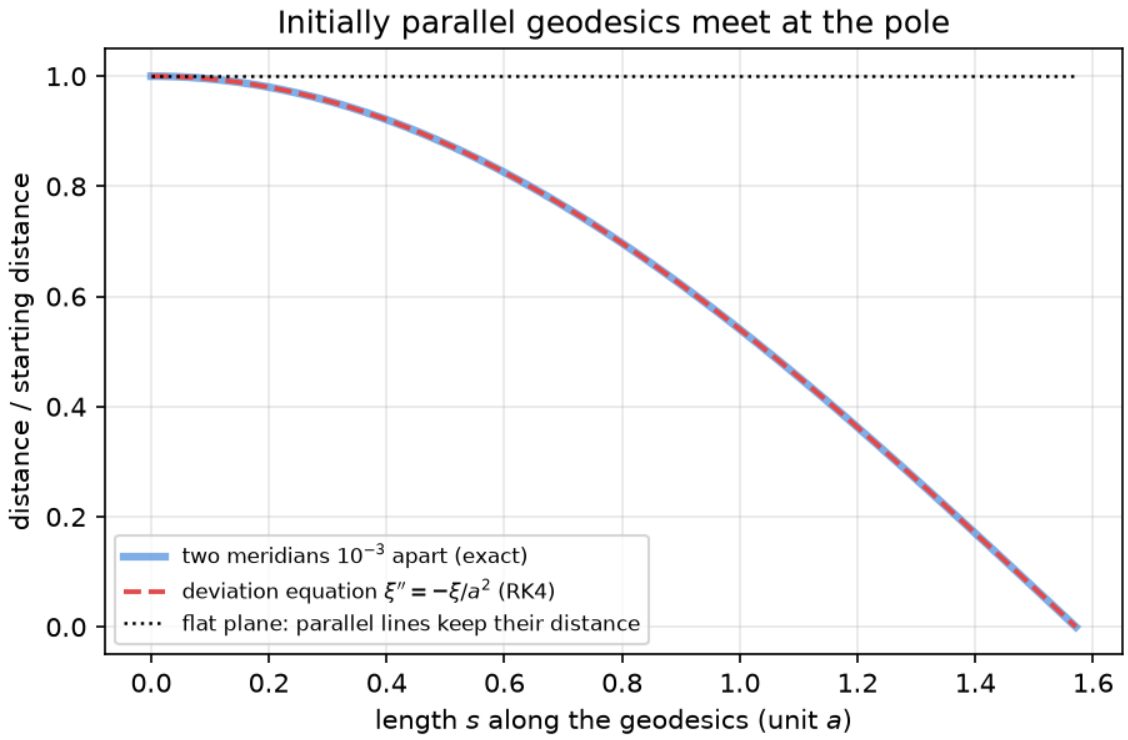

Figure 103: The distance between two neighbouring geodesics that start parallel, divided by its starting value, against the length s travelled along them in units of the radius a: two meridians of the unit sphere $\Delta\varphi = 10^{-3}$ apart that leave the equator northwards (thick blue line, exact), the solution of the geodesic deviation equation $\xi'' = -\sigma\,\xi$ with the Gaussian curvature $\sigma = 1/a^2$ computed from the Riemann tensor (dashed red, RK4), and the flat plane (dotted). On the sphere the distance shrinks like $\cos(s/a)$ and becomes zero at the pole, $s = \pi a/2$; on the plane it stays constant. (Notebook 03c, figure 5.)

Out [13]:

```
Figure 03c.5 saved as Revision/textbook/figures/03c_5_geodesic_deviation.png
PASS the curvature 1/a^2 predicts how fast neighbouring geodesics approach
```

Text cell:

```
## 11. The last check

The last cell checks that the five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [14]:

```
figure_names = ["plane_polar_components", "parallel_transport_sphere",
                "turning_angle", "geodesics_sphere", "geodesic_deviation"]
files = [output_file(f"{FIGURE_FOLDER}/03c_{k}_{n}.png")
         for k, n in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all five figure files exist")
all_checks_passed()
```

Out [14]:

```
PASS all five figure files exist
ALL 16 CHECKS PASSED (notebook 03c)
```

Text cell:

```
## 12. What this notebook showed

- The flat plane in polar coordinates has the non-zero Christoffel symbols
  $\Gamma^r{}_{\varphi\varphi} = -r$ and $\Gamma^\varphi{}_{r\varphi} = 1/r$, but its
  Riemann tensor is zero: Christoffel symbols belong to the coordinates, curvature to
  the space. The constant field $e_x$ has changing polar components and zero
  covariant derivative.
- The sphere of radius $a$ has the Gaussian curvature
  $R^{\theta\varphi}{}_{\theta\varphi} = 1/a^2$ at every point, the Ricci scalar
  $2/a^2$ and the Kretschmann scalar $4/a^4$ (exact, with sympy).
- Parallel transport around a circle of latitude turns a vector by the enclosed area
  divided by $a^2$ (numerically, RK4, to better than $10^{-9}$); at
  $\theta_0 = \pi/3$ the vector comes back reversed. Its length never changes.
- The geodesics of the sphere are great circles; a circle of latitude other than
  the equator is not a geodesic.
- Geodesics that start parallel approach each other as the geodesic deviation
  equation with the curvature $1/a^2$ predicts, and meet at the pole.
- These are the same formulas, applied in the same order (Christoffel symbols,
  Riemann tensor, Ricci tensor, scalars), that Notebook 03b applies to the author's
  8-dimensional metric, where they give the curvature of the primordial
  gravitational field.
```

3.22 Line-by-line walk-through of Notebook 03c

The notebook has 14 code cells, In [1] to In [14]. As in Section 3.13, a line or a small group of lines is quoted and then explained, and a line `...` in a quoted figure command stands for the remaining lines of the caption, which is printed in full under its figure in Section 3.21.

In [1], the set-up cell. Its comment lines are the run instructions of Section 3.20. Its code is, line for line, the set-up code of Notebook 03a explained in Section 3.13 under In [1], with the single difference `NOTEBOOK_ID = "03c" # this notebook: chapter 03, example c`, so that the figures are named `03c_<k>_<name>.png` and the captions file is `Revision/textbook/figures/03c.captions.json`. It prints one line, Set-up of notebook 03c complete: repository folder found, helpers defined.

In [2], every result in one piece. The cell is, word for word, In [2] of Notebook 03a, explained in Section 3.13: it defines `in_one_piece` and replaces `check` and `report` by functions that write each of their results with one call of `sys.stdout.write`. It prints From now on check and report print each of their results in one piece.

In [3], the curvature machinery for any metric.

```
import itertools  # loops over all index combinations

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
```

`itertools` is a module of Python with tools for loops; its function `product` runs through all combinations of indices. numpy (arrays of numbers) and sympy (exact algebra) are loaded under their usual short names.

```
def christoffel(metric, coordinates):
    """All Christoffel symbols table[a][b][c] of a metric."""
    n = len(coordinates)
    inverse = metric.inv()
    d_metric = [metric.diff(c) for c in coordinates]  # d_metric[c][i, j] = d_c g_ij
    table = [[[0] * n for _ in range(n)] for _ in range(n)]
```

The function receives the metric as a sympy matrix and the list of its coordinates. `n` is the dimension (`len` counts the list). `metric.inv()` is the inverse matrix g^{ab}. `metric.diff(c)` differentiates every entry with respect to the coordinate `c`; the list `d_metric` holds one such matrix per coordinate, so `d_metric[c][i, j]` is $\partial_c g_{ij}$. `table` is a list of n lists of n lists of n zeros, the empty table $\Gamma^a{}_{bc}$ (the name `_` is used for a loop counter that is not needed).

```
    for a, b, c in itertools.product(range(n), repeat=3):
        table[a][b][c] = sp.simplify(sum(
            inverse[a, d] * (d_metric[b][d, c] + d_metric[c][d, b]
                             - d_metric[d][b, c]) for d in range(n)) / 2)
    return table
```

`itertools.product(range(n), repeat=3)` gives every list of three indices (a, b, c), n^3 of them. For each, the formula of Section 3.15 is applied literally: the sum over d of $g^{ad}(\partial_b g_{dc} + \partial_c g_{db} - \partial_d g_{bc})$, divided by 2 and simplified by `sp.simplify`. The finished table is returned.

```
def riemann(table, coordinates):
    """The non-zero R^a_bcd (MTW convention) as a dictionary."""
    n = len(coordinates)
    result = {}
    for a, b, c, d in itertools.product(range(n), repeat=4):
        value = sp.simplify(
            sp.diff(table[a][b][d], coordinates[c])
            - sp.diff(table[a][b][c], coordinates[d])
            + sum(table[a][c][e] * table[e][b][d] - table[a][d][e] * table[e][b][c]
                  for e in range(n)))
        if value != 0:
            result[(a, b, c, d)] = value
    return result
```

For every list of four indices the formula of Section 3.17 is applied literally: $\partial_c \Gamma^a{}_{bd} - \partial_d \Gamma^a{}_{bc} + \sum_e (\Gamma^a{}_{ce}\Gamma^e{}_{bd} - \Gamma^a{}_{de}\Gamma^e{}_{bc})$, simplified. Only the components that are not zero are kept, in a **dictionary** whose keys are the index lists `(a, b, c, d)`; the zero ones are simply absent.

```
def raise_second(riemann_table, metric):
    """The non-zero R^ab_cd = sum over e of g^(be) R^a_ecd."""
    inverse = metric.inv()
    result = {}
    for (a, e, c, d), value in riemann_table.items():
        for b in range(metric.rows):
            result[(a, b, c, d)] = result.get((a, b, c, d), 0) + inverse[b, e] * value
    return {k: sp.simplify(v) for k, v in result.items() if sp.simplify(v) != 0}
```

This raises the second index, $R^{ab}{}_{cd} = \sum_e g^{be} R^a{}_{ecd}$. `items()` hands out every key with its value; writing the key as `(a, e, c, d)` gives the four indices names. For each b the product $g^{be}R^a{}_{ecd}$ is added to the entry `(a, b, c, d)`; `result.get(key, 0)` is the value collected so far, or 0 if there is none yet. The last line is a **dictionary comprehension**: it builds a new dictionary from the simplified values and keeps only those that are not zero.

```
def ricci(mixed_table, n):
    """R^a_b = sum over c of R^ac_bc."""
    result = sp.zeros(n, n)
    for (a, c, b, d), value in mixed_table.items():
        if c == d:
            result[a, b] += value
    return result.applyfunc(sp.simplify)
```

The Ricci tensor $R^a{}_b = \sum_c R^{ac}{}_{bc}$ of Section 3.17. The key of the table is read as (a, c, b, d): upper indices a, c and lower indices b, d; a component contributes to $R^a{}_b$ exactly when its second upper index equals its second lower index, $c = d$. `+=` adds to the entry. Every entry of the result is simplified.

```
def kretschmann(mixed_table):
    """K = sum of R^ab_cd R^cd_ab."""
    return sp.simplify(sum(v * mixed_table.get((c, d, a, b), 0)
```

```
                        for (a, b, c, d), v in mixed_table.items()))

say("Defined: christoffel, riemann, raise_second, ricci, kretschmann.")
```

The Kretschmann scalar $K = \sum R^{ab}{}_{cd} R^{cd}{}_{ab}$: each stored component is multiplied by its partner with the two pairs exchanged (0 if the partner is absent), and everything is summed. The last line prints the output of In [3].

In [4], the flat plane in polar coordinates.

```
r, phi = sp.symbols("r varphi", positive=True)  # polar coordinates, r > 0
plane = sp.diag(1, r ** 2)  # ds^2 = dr^2 + r^2 dphi^2
POLAR = [r, phi]
POLAR_NAMES = ["r", "phi"]
plane_gamma = christoffel(plane, POLAR)
```

Two positive symbols, the distance r and the angle φ (sympy's name for it is `varphi`). `sp.diag(1, r ** 2)` is the metric $\mathrm{diag}(1, r^2)$ of Section 3.3. `POLAR` is the list of the coordinates and `POLAR_NAMES` their names for printing. The last line computes all $2^3 = 8$ Christoffel symbols.

```
for a, b, c in itertools.product(range(2), repeat=3):
    if plane_gamma[a][b][c] != 0:
        say(f"  Gamma^{POLAR_NAMES[a]}_({POLAR_NAMES[b]} {POLAR_NAMES[c]}) = "
            f"{plane_gamma[a][b][c]}")
check(plane_gamma[0][1][1] == -r and plane_gamma[1][0][1] == 1 / r
      and plane_gamma[1][1][0] == 1 / r,
      "plane in polar coordinates: Gamma^r_(phi phi) = -r, Gamma^phi_(r phi) = 1/r")
```

The loop prints the symbols that are not zero: the output shows $\Gamma^r{}_{\varphi\varphi} = -r$ and $\Gamma^\varphi{}_{r\varphi} = \Gamma^\varphi{}_{\varphi r} = 1/r$, the values found by hand in Section 3.15. Position 0 is r and position 1 is φ, so `plane_gamma[0][1][1]` is $\Gamma^r{}_{\varphi\varphi}$. The check confirms the three values.

```
plane_riemann = riemann(plane_gamma, POLAR)
say(f"non-zero Riemann components of the plane: {len(plane_riemann)}")
check(plane_riemann == {},
      "the Riemann tensor of the plane is zero: the plane is flat")
```

The Riemann tensor of the plane has no non-zero component: the output prints 0 and the check requires the empty dictionary `{}` (Section 3.18).

In [5], the constant field has zero covariant derivative.

```
V = [sp.cos(phi), -sp.sin(phi) / r]  # the constant field e_x in polar components
nabla = [[sp.simplify(sp.diff(V[a], POLAR[b])
                      + sum(plane_gamma[a][b][c] * V[c] for c in range(2)))
          for b in range(2)] for a in range(2)]
say(f"covariant derivative of e_x: {nabla}")
check(nabla == [[0, 0], [0, 0]],
      "the covariant derivative of the constant field e_x is zero")
```

`V` holds the polar components of e_x found in Section 3.14. The nested list comprehension computes $\nabla_b V^a = \partial_b V^a + \sum_c \Gamma^a{}_{bc} V^c$ for $a = 0, 1$ (outer list) and $b = 0, 1$ (inner list), exactly the four lines of Section 3.15. The output prints `[[0, 0], [0, 0]]` and the check requires it.

In [6], Figure 03c.1.

```
BLUE, ORANGE, AQUA, RED, VIOLET = "#2a78d6", "#eb6834", "#1baf7a", "#e34948", "#4a3aa7"
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.6))
fig.subplots_adjust(wspace=0.3)  # room for the label of the right vertical axis
ax = axes[0]
angles = np.linspace(0.0, 2 * np.pi, 400)
```

Five colours of the palette of Section 3.13 (In [9] of Notebook 03a). A figure with two panels side by side; `subplots_adjust(wspace=0.3)` widens the gap between them. `ax` names the left panel. `angles` holds 400 angles from 0 to 2π.

```
for radius in (0.5, 1.0, 1.5, 2.0):  # circles of constant r
    ax.plot(radius * np.cos(angles), radius * np.sin(angles), color="grey", lw=0.6)
for ray in np.arange(12) * np.pi / 6:  # rays of constant phi
    ax.plot([0, 2.2 * np.cos(ray)], [0, 2.2 * np.sin(ray)], color="grey", lw=0.6)
```

The polar grid: four grey circles of constant r (a circle is the curve $(r\cos\varphi, r\sin\varphi)$), and twelve grey rays of constant φ, one every $\pi/6$ (`np.arange(12)` is $0, 1, \ldots, 11$), each a straight segment from the origin to the distance 2.2.

```
points = np.arange(12) * np.pi / 6 + np.pi / 12  # the angles of the 12 points
px, py = 1.5 * np.cos(points), 1.5 * np.sin(points)
ax.quiver(px, py, np.ones(12), np.zeros(12), color="black", scale=8, width=0.006,
          label="constant field $e_x$")
ax.quiver(px, py, np.cos(points), np.sin(points), color=RED, scale=12,
          width=0.005, label="unit vector $e_r$")
ax.quiver(px, py, -np.sin(points), np.cos(points), color=BLUE, scale=12,
          width=0.005, label="unit vector $e_\\varphi / r$")
```

Twelve points on the circle $r = 1.5$, halfway between the rays. `ax.quiver(x, y, dx, dy, ...)` draws an arrow at each point (x, y) in the direction (dx, dy); `scale` sets how long the arrows are drawn (a larger scale gives shorter arrows) and `width` their thickness. Black: the constant field, $(1, 0)$ everywhere. Red: the unit vector $e_r = (\cos\varphi, \sin\varphi)$, pointing away from the origin. Blue: the unit vector $e_\varphi / r = (-\sin\varphi, \cos\varphi)$, pointing around the circle.

```
ax.set_aspect("equal")
ax.set_xlim(-2.4, 2.4)
ax.set_ylim(-2.4, 2.4)
ax.set_xlabel("$x$")
ax.set_ylabel("$y$")
ax.set_title("A constant field on the polar grid")
ax.legend(fontsize=7, loc="lower left")
```

Equal units on both axes (so circles look round), the ranges of both axes, their labels, the title and the legend in the lower left corner.

```
ax = axes[1]
ax.plot(angles, np.cos(angles), color=RED, lw=1.8,
        label="component along $e_r$: $\\cos\\varphi$")
ax.plot(angles, -np.sin(angles), color=BLUE, lw=1.8, ls="--",
        label="component along $e_\\varphi/r$: $-\\sin\\varphi$")
ax.set_xlabel("$\\varphi$ (radians)")
ax.set_ylabel("component (no unit)")
ax.set_title("Its polar components change")
ax.legend(fontsize=8, loc="lower left")
```

The right panel: the components of e_x along the two unit vectors, $\cos\varphi$ (solid) and $-\sin\varphi$ (dashed), against φ. (Along the unit vector e_φ/r the component is $rV^\varphi = -\sin\varphi$.)

```
save_figure(fig, "plane_polar_components",
            "The flat plane in polar coordinates. Left: the polar grid (grey circles "
...
check(np.allclose(np.cos(points) ** 2 + np.sin(points) ** 2, 1.0),
      "the components of the unit field e_x have length 1 at every point")
```

The figure is saved with its caption. The check is a simple consistency test of the drawing: at each of the twelve points the two components $\cos\varphi$ and $-\sin\varphi$ of the unit field e_x along the two perpendicular unit vectors give the length $\sqrt{\cos^2\varphi + \sin^2\varphi} = 1$ (`np.allclose` compares arrays up to rounding).

What Figure 03c.1 shows. Left: all twelve black arrows are equal, while the red and blue unit vectors at the same points turn as one goes around the circle. Right: the two components of the constant field oscillate between -1 and 1 as φ runs once around. A derivative that only looked at components would call this field changing; the covariant derivative, with the Christoffel symbols, correctly says it does not change.

In [7], the sphere of radius a.

```
theta = sp.symbols("theta", positive=True)
a = sp.symbols("a", positive=True)  # the radius of the sphere
sphere = sp.diag(a ** 2, a ** 2 * sp.sin(theta) ** 2)
ANGLES = [theta, phi]
ANGLE_NAMES = ["theta", "phi"]
sphere_gamma = christoffel(sphere, ANGLES)
```

The angle θ and the radius a as positive symbols; the metric $\mathrm{diag}(a^2, a^2\sin^2\theta)$ of Section 3.3; the coordinates (θ, φ) (the symbol φ of In [4] is reused) and their names; all eight Christoffel symbols.

```
for i, j, k in itertools.product(range(2), repeat=3):
    if sphere_gamma[i][j][k] != 0:
        say(f"  Gamma^{ANGLE_NAMES[i]}_({ANGLE_NAMES[j]} {ANGLE_NAMES[k]}) = "
            f"{sphere_gamma[i][j][k]}")
sphere_riemann = riemann(sphere_gamma, ANGLES)
sphere_mixed = raise_second(sphere_riemann, sphere)
```

The loop prints the non-zero symbols. sympy writes $\Gamma^{\theta}{}_{\varphi\varphi} = -\sin\theta\cos\theta$ as `-sin(2*theta)/2`, the same number because $\sin 2\theta = 2\sin\theta\cos\theta$, and $\cot\theta$ as `1/tan(theta)`. Then the Riemann tensor and its form with two upper indices.

```
for key, value in sorted(sphere_mixed.items()):
    say(f"  R^({ANGLE_NAMES[key[0]]} {ANGLE_NAMES[key[1]]})_"
        f"({ANGLE_NAMES[key[2]]} {ANGLE_NAMES[key[3]]}) = {value}")
sphere_ricci = ricci(sphere_mixed, 2)
sphere_R = sp.simplify(sphere_ricci.trace())
sphere_K = kretschmann(sphere_mixed)
say(f"Ricci tensor R^a_b = {sphere_ricci.tolist()}")
say(f"Ricci scalar R = {sphere_R},  Kretschmann scalar K = {sphere_K}")
```

`sorted` puts the components in a fixed order (by their index lists) before printing; `key[0]` to `key[3]` are the four indices. The output shows the four components of a surface (Section 3.17): $R^{\theta\varphi}{}_{\theta\varphi} = R^{\varphi\theta}{}_{\varphi\theta} = 1/a^2$ (written `a**(-2)`) and $R^{\theta\varphi}{}_{\varphi\theta} = R^{\varphi\theta}{}_{\theta\varphi} = -1/a^2$. `trace()` is the sum of the diagonal entries of the Ricci matrix, the Ricci scalar; `tolist()` writes a matrix as a list of rows for printing. The output: $R^a{}_b = \mathrm{diag}(1/a^2, 1/a^2)$, $R = 2/a^2$, $K = 4/a^4$.

```
check(sp.simplify(sphere_gamma[0][1][1] + sp.sin(theta) * sp.cos(theta)) == 0
      and sp.simplify(sphere_gamma[1][0][1] - sp.cot(theta)) == 0,
      "sphere: Gamma^theta_(phi phi) = -sin cos, Gamma^phi_(theta phi) = cot theta")
check(sphere_mixed[(0, 1, 0, 1)] == 1 / a ** 2,
      "the Gaussian curvature of the sphere is 1/a^2 at every point")
check(sphere_R == 2 / a ** 2 and sphere_K == 4 / a ** 4,
      "Ricci scalar 2/a^2 and Kretschmann scalar 4/a^4")
```

Three checks of Section 3.18: the two Christoffel symbols found by hand (the difference with the hand result simplifies to zero), the Gaussian curvature $R^{\theta\varphi}{}_{\theta\varphi} = 1/a^2$ (key `(0, 1, 0, 1)`), and $R = 2/a^2$, $K = 4/a^4$.

In [8], parallel transport around a circle of latitude.

```
def rk4(f, y0, t0, t1, steps):
    """Solve dy/dt = f(t, y) from t0 to t1 with the classical Runge-Kutta method in
    steps equal steps; return the times and the states (one row per time)."""
    h = (t1 - t0) / steps
    times = t0 + h * np.arange(steps + 1)
    states = np.zeros((steps + 1, len(y0)))
    states[0] = y0
```

The RK4 method of Chapter 2 for a system $dy/dt = f(t, y)$ of several equations. `h` is the step; `times` holds the $steps + 1$ times $t_0, t_0 + h, \ldots, t_1$; `states` is a table with one row per time and one column per unknown, filled with zeros; its first row is the starting state `y0`.

```
    for n in range(steps):
        t, y = times[n], states[n]
        k1 = f(t, y)
        k2 = f(t + h / 2, y + h / 2 * k1)
```

```
        k3 = f(t + h / 2, y + h / 2 * k2)
        k4 = f(t + h, y + h * k3)
        states[n + 1] = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return times, states
```

One RK4 step per pass: four evaluations of the right side (at the start, twice at the middle, at the end of the step) and the weighted average $(k_1 + 2k_2 + 2k_3 + k_4)/6$ times h added to the state. The function returns the times and the table of states.

```
def transport(theta0, steps=2000):
    """Parallel transport of the unit south vector once around latitude theta0
    (a = 1): the components V^theta, V^phi as functions of phi."""
    def f(t, y):
        return np.array([np.sin(theta0) * np.cos(theta0) * y[1],
                         -np.cos(theta0) / np.sin(theta0) * y[0]])
    return rk4(f, np.array([1.0, 0.0]), 0.0, 2 * np.pi, steps)
```

The transport equations of Section 3.16 with $a = 1$: the state is $y = (V^\theta, V^\varphi)$, and `f` returns $(\sin\theta_0 \cos\theta_0 \, V^\varphi, \ -\cot\theta_0 \, V^\theta)$. The parameter is φ, from 0 to 2π in 2000 steps (the default value of `steps`). The start $(1, 0)$ is the unit vector pointing south ($u = aV^\theta = 1$, $w = 0$).

```
def turning_angle(theta0):
    """The angle (0 to 2 pi) by which the vector comes back turned."""
    _, states = transport(theta0)
    u, w = states[-1, 0], np.sin(theta0) * states[-1, 1]  # lengths south, east
    return np.mod(np.arctan2(w, u), 2 * np.pi)
```

After one round (the last row, `states[-1]`) the lengths along south and east are $u = V^\theta$ and $w = \sin\theta_0 \, V^\varphi$ (with $a = 1$). `np.arctan2(w, u)` is the angle of the arrow (u, w) measured from the south direction towards the east, between $-\pi$ and π; `np.mod(..., 2 * np.pi)` adds 2π to a negative angle, so the result lies between 0 and 2π.

```
theta0 = np.pi / 3
phis, states = transport(theta0)
lengths = np.sqrt(states[:, 0] ** 2 + (np.sin(theta0) * states[:, 1]) ** 2)
# round to 12 digits; adding 0.0 turns a rounded -0.0 into 0.0
say(f"start: V = {(states[0].round(12) + 0.0).tolist()},  end: V = "
    f"{(states[-1].round(12) + 0.0).tolist()}")
```

The transport around $\theta_0 = \pi/3$. `lengths` holds $\sqrt{u^2 + w^2}$ at all 2001 points (`states[:, 0]` is the first column, all rows). The start and end vectors are printed rounded to 12 digits; a tiny negative number rounds to -0.0, which would print as `-0.0`, and adding 0.0 turns it into 0.0. The output: start $[1.0, 0.0]$, end $[-1.0, 0.0]$.

```
check(np.max(np.abs(lengths - 1.0)) < 1e-12,
      "parallel transport keeps the length of the vector")
check(np.max(np.abs(states[-1] - np.array([-1.0, 0.0]))) < 1e-10,
      "at theta0 = pi/3 the vector comes back exactly reversed")
```

Two checks: the length stays 1 within 10^{-12} at every point, and the vector comes back reversed within 10^{-10}, as derived in Section 3.16.

In [9], Figure 03c.2: the transport drawn on the sphere.

```
def point(th, ph):
    """The point of the unit sphere at the angles th, ph."""
    return np.array([np.sin(th) * np.cos(ph), np.sin(th) * np.sin(ph), np.cos(th)])
```

The point $(\sin\theta\cos\varphi, \sin\theta\sin\varphi, \cos\theta)$ of the unit sphere, as an array of its three coordinates X, Y, Z.

```
def wire_sphere(ax):
    """Draw a light wire model of the unit sphere into the 3-D axes ax."""
    th, ph = np.meshgrid(np.linspace(0, np.pi, 13), np.linspace(0, 2 * np.pi, 25))
    ax.plot_wireframe(np.sin(th) * np.cos(ph), np.sin(th) * np.sin(ph), np.cos(th),
                      color="grey", lw=0.3)
    ax.set_box_aspect((1, 1, 1), zoom=0.9)  # a little smaller: the labels fit
```

A helper that draws the sphere as a grey net of 13 circles of latitude and 25 meridians: `np.meshgrid` makes the grid of angles, and `plot_wireframe` connects the corresponding points. `set_box_aspect((1, 1, 1), zoom=0.9)` makes the three axes equally long and the box a little smaller, so that the labels fit in the picture.

```
    ax.set_xlim(-1, 1)
    ax.set_ylim(-1, 1)
    ax.set_zlim(-1, 1)
    ax.view_init(elev=25, azim=-60)
    for set_ticks in (ax.set_xticks, ax.set_yticks, ax.set_zticks):
        set_ticks([-1.0, 0.0, 1.0])  # three ticks per axis: labels do not overlap
    ax.tick_params(labelsize=8)
    ax.set_xlabel("$X$")
    ax.set_ylabel("$Y$")
    ax.set_zlabel("$Z$")
```

The ranges of the three axes; the point of view (25 degrees above the equator, turned by -60 degrees around the vertical axis); three tick marks per axis (the loop calls each of the three tick functions in turn); small tick labels; the axis names.

```
fig = plt.figure(figsize=(6.4, 6.0))
ax = fig.add_subplot(projection="3d")
wire_sphere(ax)
circle = np.array([point(theta0, p) for p in np.linspace(0, 2 * np.pi, 200)])
ax.plot(circle[:, 0], circle[:, 1], circle[:, 2], color=VIOLET, lw=1.8)
```

A figure with one three-dimensional drawing area (`projection="3d"`), the wire sphere, and the circle of latitude $\theta_0 = \pi/3$ as 200 points joined by a violet line.

```
for n in range(0, 2001, 125):  # 17 points, the last one equal to the first
    ph = phis[n]
    u, w = states[n, 0], np.sin(theta0) * states[n, 1]
    e_theta = np.array([np.cos(theta0) * np.cos(ph), np.cos(theta0) * np.sin(ph),
                        -np.sin(theta0)])
    e_phi = np.array([-np.sin(ph), np.cos(ph), 0.0])
```

`range(0, 2001, 125)` gives $n = 0, 125, \ldots, 2000$: 17 steps of the transport, one every $2\pi \cdot 125/2000 = \pi/8$. At each, `ph` is the angle and u, w the lengths south and east. `e_theta` is the unit vector pointing south at that point (the derivative of the point with respect to θ), and `e_phi` the unit vector pointing east (the derivative with respect to φ, divided by $\sin\theta_0$).

```
    arrow = 0.35 * (u * e_theta + w * e_phi)
    start = point(theta0, ph)
    colour = RED if n in (0, 2000) else BLUE
    ax.quiver(*start, *arrow, color=colour, lw=1.5, arrow_length_ratio=0.25)
ax.set_title("Parallel transport around the latitude $\\theta_0 = \\pi/3$")
```

The transported vector in three dimensions, $u\,e_\theta + w\,e_\varphi$, shortened to 0.35 for the drawing. It is drawn at its point on the circle, red at the start and the end, blue in between. In `ax.quive r(*start, *arrow, ...)` the stars unpack the three coordinates of the point and of the arrow into six arguments; `arrow_length_ratio` is the size of the arrow head.

```
save_figure(fig, "parallel_transport_sphere",
            "Parallel transport on the unit sphere (wire model; axes $X$, $Y$, $Z$ "
...
check(abs(turning_angle(theta0) - np.pi) < 1e-10,
      "the turning angle at theta0 = pi/3 is pi")
```

The figure is saved; the check confirms that the turning angle at $\theta_0 = \pi/3$ is π within 10^{-10}.

What Figure 03c.2 shows. The violet circle of latitude on the grey wire sphere and the transported vector along it. At the start (red, on the right) it points south, down the sphere. The blue arrows turn steadily relative to the circle, and at the end (red again, at the same place) the vector points north, up the sphere: it has turned by π although it was never turned along the way. On a flat sheet it would have come back unchanged.

In [10], Figure 03c.3: the turning angle for 15 circles.

```
latitudes = np.linspace(0.1, 1.5, 15)
measured = np.array([turning_angle(t0) for t0 in latitudes])
curve = np.linspace(0.0, np.pi / 2, 300)
fig, ax = plt.subplots()
```

Fifteen circles of latitude, $\theta_0 = 0.1, 0.2, \ldots, 1.5$; for each the turning angle measured by the transport of In [8]; 300 values of θ_0 for the smooth curve.

```
ax.plot(curve, 2 * np.pi * (1 - np.cos(curve)), color=VIOLET, lw=1.8,
        label="cap area / $a^2 = 2\\pi(1 - \\cos\\theta_0)$")
```

```
ax.plot(latitudes, measured, "o", color=ORANGE, ms=7,
        label="turning angle measured by parallel transport")
ax.plot(curve, np.zeros_like(curve), ":", color="black", lw=1.2,
        label="flat plane: no turning")
ax.set_xlabel("latitude angle $\\theta_0$ of the loop (radians from the pole)")
ax.set_ylabel("angle (radians)")
ax.set_title("Curvature turns vectors: angle = enclosed area $\\times$ $1/a^2$")
ax.legend(fontsize=8)
```

The violet line is the area of the cap divided by a^2 (Section 3.16), the orange dots are the measured angles, and the dotted line at zero is the flat plane (`np.zeros_like` makes an array of zeros of the same length). Labels, title and legend.

```
deviation = np.max(np.abs(measured - 2 * np.pi * (1 - np.cos(latitudes))))
report("largest difference between the turning angle and area/a^2",
       f"{deviation:.1e}", "radians")
save_figure(fig, "turning_angle",
            "The turning angle of a vector carried once around a circle of latitude "
...
check(deviation < 1e-9, "the turning angle equals the enclosed area over a^2")
```

The largest difference between the measured angles and the formula is printed in a RESULT line: 5.0×10^{-12} radians (the format `.1e` writes one digit after the point and a power of ten). The figure is saved, and the check requires the difference to be below 10^{-9}.

What Figure 03c.3 shows. The fifteen orange dots lie exactly on the violet curve $2\pi(1 - \cos\theta_0)$: the larger the cap enclosed by the loop, the more the vector turns. The dotted line shows that on the flat plane every loop gives zero.

In [11], geodesics are great circles.

```
def geodesic_rhs(s, y):
    """The geodesic equations of the unit sphere for y = (theta, phi, dtheta,
    dphi)."""
    th, ph, dth, dph = y
    return np.array([dth, dph, np.sin(th) * np.cos(th) * dph ** 2,
                     -2 * np.cos(th) / np.sin(th) * dth * dph])
```

The two second-order geodesic equations of Section 3.16 (with $a = 1$) written as four first-order equations for the state $(\theta, \varphi, d\theta/ds, d\varphi/ds)$: the derivatives of the first two are the last two, and the derivatives of the last two are the right sides $\sin\theta\cos\theta(d\varphi/ds)^2$ and $-2\cot\theta\,(d\theta/ds)(d\varphi/ds)$. The first line of the body unpacks the four numbers of `y`.

```
geodesics = {}
for degrees in (20, 50, 80):
    alpha = np.radians(degrees)  # the direction, measured from east towards north
    # unit speed at the equator: dtheta/ds = -sin(alpha) (north), dphi/ds = cos(alpha)
    start = np.array([np.pi / 2, 0.0, -np.sin(alpha), np.cos(alpha)])
    _, path = rk4(geodesic_rhs, start, 0.0, 2 * np.pi, 4000)
    xyz = np.array([point(th, ph) for th, ph in path[:, :2]])
```

Three geodesics start at the point $\theta = \pi/2$, $\varphi = 0$ on the equator, in the directions 20, 50 and 80 degrees north of east (`np.radians` converts degrees to radians). Moving north decreases θ (the angle from the north pole), hence $d\theta/ds = -\sin\alpha$; moving east increases φ, $d\varphi/ds = \cos\alpha$. The speed is 1, because $(d\theta/ds)^2 + \sin^2\theta\,(d\varphi/ds)^2 = \sin^2\alpha + \cos^2\alpha = 1$ at the equator. RK4 integrates each path over the length 2π (once around a great circle) in 4000 steps, and `xyz` holds the path as points in three dimensions (`path[:, :2]` is the table of the first two columns, θ and φ).

```
    # the starting direction in X, Y, Z: east is (0, 1, 0), north is (0, 0, 1)
    velocity0 = np.array([0.0, np.cos(alpha), np.sin(alpha)])
    normal = np.cross(xyz[0], velocity0)  # perpendicular to the expected plane
    geodesics[degrees] = (xyz, np.max(np.abs(xyz @ normal)))
```

A great circle lies in the plane through the centre of the sphere spanned by its starting point and its starting direction. `np.cross` is the cross product of two vectors in space, a vector perpendicular to both; it has length 1 here because the two are perpendicular unit vectors. The distance of a point from that plane is the size of its dot product with the perpendicular vector; `xyz @ normal` computes this dot product for every point of the path at once (`@` is the product of a matrix with a vector). The path and its largest distance from the plane are stored in the dictionary under the angle.

```
    say(f"  direction {degrees:2d} degrees: largest distance from the plane "
        f"= {geodesics[degrees][1]:.1e}, end point back at start: "
        f"{np.allclose(xyz[-1], xyz[0], atol=1e-8)}")
check(all(distance < 1e-8 for _, distance in geodesics.values()),
      "the three geodesics stay in planes through the centre: great circles")
```

For each direction one line is printed: the largest distance (1.2×10^{-13}, 6.2×10^{-13} and 7.1×10^{-11}) and whether the end point equals the start within 10^{-8} (`True` for all three). The check requires all three distances to be below 10^{-8}.

```
theta_circle = np.pi / 3
residual = np.sin(theta_circle) * np.cos(theta_circle) * (1 / np.sin(theta_circle)) ** 2
report("geodesic equation of the latitude theta0 = pi/3: missing acceleration",
       f"{residual:.6f}")
check(residual > 0.5, "the circle of latitude theta0 = pi/3 is not a geodesic")
```

The circle of latitude $\theta_0 = \pi/3$ at unit speed has $d\varphi/ds = 1/\sin\theta_0$ and $d^2\theta/ds^2 = 0$; the right side of the first geodesic equation is then $\sin\theta_0\cos\theta_0/\sin^2\theta_0 = \cot\theta_0$, printed as 0.577350 (Section 3.16). It is not zero, so the circle of latitude is not a geodesic.

In [12], Figure 03c.4.

```
fig = plt.figure(figsize=(6.4, 6.0))
ax = fig.add_subplot(projection="3d")
wire_sphere(ax)
for (degrees, (xyz, _)), colour, style in zip(geodesics.items(),
                                              (RED, ORANGE, AQUA), ("-", "--", "-.")):
    ax.plot(xyz[:, 0], xyz[:, 1], xyz[:, 2], color=colour, ls=style, lw=1.8,
            label=f"geodesic leaving at {degrees} degrees")
```

The wire sphere again, and the three geodesics, each in its own colour and line style; `zip` walks through the stored geodesics, the colours and the styles together, and the pattern `(degrees, (xyz, _))` takes apart each dictionary entry (the stored distance is not needed).

```
ax.plot(circle[:, 0], circle[:, 1], circle[:, 2], color=VIOLET, ls=":", lw=1.8,
        label="circle of latitude (not a geodesic)")
ax.scatter([1.0], [0.0], [0.0], color="black", s=30)
ax.set_title("Geodesics of the sphere from one point")
ax.legend(fontsize=7, loc="upper left")
```

The circle of latitude of In [9], dotted; the common starting point $(1, 0, 0)$ as a black dot (`scatter` draws points, `s` is their size); title and legend.

```
save_figure(fig, "geodesics_sphere",
            "Three geodesics of the unit sphere (axes $X$, $Y$, $Z$ in units of the "
...
check(all(np.allclose(xyz[-1], xyz[0], atol=1e-8) for xyz, _ in geodesics.values()),
      "every geodesic returns to its start after the length 2 pi")
```

The figure is saved; the check requires each geodesic to come back to its start after the length 2π, within 10^{-8}.

What Figure 03c.4 shows. The three geodesics leave the black dot on the equator in three directions; each is a full circle around the sphere, centred at its centre (a great circle), tilted the more steeply the more northward it starts, and each passes through the opposite point $(-1, 0, 0)$. The dotted circle of latitude is smaller and is not centred at the centre of the sphere: a traveller following it would have to keep turning towards the pole.

In [13], Figure 03c.5: geodesic deviation.

```
sigma = float(sphere_mixed[(0, 1, 0, 1)].subs(a, 1))  # the Gaussian curvature, a = 1
s_values, deviation_states = rk4(lambda s, y: np.array([y[1], -sigma * y[0]]),
                                 np.array([1.0, 0.0]), 0.0, np.pi / 2, 1000)
```

`sigma` is the Gaussian curvature σ computed in In [7], $1/a^2$, with $a = 1$, as a floating-point number. The deviation equation $\xi'' = -\sigma\xi$ of Section 3.18 as two first-order equations for (ξ, ξ'); `lambda s , y: ...` is a short function without a name that returns $(\xi', -\sigma\xi)$. RK4 solves it from $\xi = 1$, $\xi' = 0$ (start parallel, distance 1) over the length $\pi/2$ in 1000 steps.

```
d_phi = 1e-3  # the angle between the two meridians
chord = np.array([np.linalg.norm(point(np.pi / 2 - s, 0.0) - point(np.pi / 2 - s, d_phi))
                  for s in s_values]) / np.linalg.norm(point(np.pi / 2, 0.0)
                                                       - point(np.pi / 2, d_phi))
```

Two meridians 10^{-3} apart, followed from the equator northwards ($\theta = \pi/2 - s$). Their distance is measured as the length of the straight chord between the two points (`np.linalg.norm` is the length of a vector), divided by its value at the equator. The chord is $2\sin\theta\sin(\Delta\varphi/2)$, so the ratio is exactly $\sin\theta/\sin(\pi/2) = \cos s$, the result of Section 3.18 with $a = 1$.

```
fig, ax = plt.subplots()
ax.plot(s_values, chord, color=BLUE, lw=3.0, alpha=0.6,
        label="two meridians $10^{-3}$ apart (exact)")
ax.plot(s_values, deviation_states[:, 0], "--", color=RED, lw=1.8,
        label="deviation equation $\\xi^{\\prime\\prime} = -\\xi/a^2$ (RK4)")
ax.plot(s_values, np.ones_like(s_values), ":", color="black", lw=1.2,
        label="flat plane: parallel lines keep their distance")
ax.set_xlabel("length $s$ along the geodesics (unit $a$)")
ax.set_ylabel("distance / starting distance")
ax.set_title("Initially parallel geodesics meet at the pole")
ax.legend(fontsize=8)
```

The exact distance as a thick, partly transparent blue line (`alpha=0.6`), the RK4 solution as a dashed red line on top of it, and the flat plane as a dotted line at 1. Labels, title, legend.

```
agreement = np.max(np.abs(chord - deviation_states[:, 0]))
report("largest difference between the meridians and the deviation equation",
       f"{agreement:.1e}")
save_figure(fig, "geodesic_deviation",
            "The distance between two neighbouring geodesics that start parallel, "
...
check(agreement < 1e-6,
      "the curvature 1/a^2 predicts how fast neighbouring geodesics approach")
```

The largest difference between the two curves is printed: 8.0×10^{-14}. The figure is saved; the check requires the difference to be below 10^{-6}.

What Figure 03c.5 shows. The red dashed curve lies exactly on the blue one: both fall from 1 like $\cos s$ and reach 0 at $s = \pi/2$, the pole, where the meridians meet. The curvature $1/a^2$, computed from the Riemann tensor, predicts how fast two geodesics that start parallel approach each other. On the plane (dotted) the distance would stay 1.

In [14], the last check.

```
figure_names = ["plane_polar_components", "parallel_transport_sphere",
                "turning_angle", "geodesics_sphere", "geodesic_deviation"]
files = [output_file(f"{FIGURE_FOLDER}/03c_{k}_{n}.png")
         for k, n in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all five figure files exist")
all_checks_passed()
```

The names of the five figures, numbered from 1 by `enumerate(figure_names, 1)`; the check that all five files exist; and the last line, ALL 16 CHECKS PASSED (notebook 03c). The 16 checks are: 2 in In [4], 1 each in In [5] and In [6], 3 in In [7], 2 in In [8], 1 each in In [9] and In [10], 2 in In [11], and 1 each in In [12], In [13] and In [14].

3.23 The Christoffel symbols of the author's metric

Now the tools of Sections 3.15 to 3.17 are applied to the author's metric, by hand and completely. Throughout, i stands for any of the three 3-space directions x_1, x_2, x_3, j for any of the three

extra times x_5, x_6, x_7, and k for any of the six **transverse** directions (i or j). The scale factors (Section 3.7) are $h_0 = \cot z$, $h_i = e^{a_4}\sin^{1/6} z$, $h_4 = 1$, $h_j = b_4^{1/2} e^{-a_4}\sin^{1/6} z$, with the signs $\eta = (+,+,+,+,-,-,-,-)$ in the order $x_0, \ldots, x_7$.

The logarithmic derivatives of the scale factors. Only x_4 and x_0 appear in the metric, so only ∂_4 and ∂_0 can give something non-zero.

$$\partial_4 \ln h_i = \partial_4\big(a_4 + \tfrac{1}{6}\ln\sin z\big) = a_4' \equiv \alpha, \qquad \partial_4 \ln h_j = \partial_4\big(\tfrac{1}{2}\ln b_4 - a_4 + \tfrac{1}{6}\ln\sin z\big) = -a_4' + \frac{b_4'}{2b_4} \equiv \alpha_t$$

(the logarithm of a product is the sum of the logarithms; $\ln e^{\pm a_4} = \pm a_4$; $\ln b_4^{1/2} = \frac{1}{2}\ln b_4$, whose derivative is $b_4'/(2b_4)$; z does not depend on x_4). These are the two expansion rates of Section 3.7: α of 3-space and α_t of the extra times. We write α_k for the rate of the transverse direction k ($\alpha_i = \alpha$, $\alpha_j = \alpha_t$). For a constant b_4, $\alpha_t = -\alpha$.

$$\partial_0 \ln h_k = \tfrac{1}{6}\,\frac{\cos z}{\sin z}\cdot 6H = H\cot z$$

(for every transverse k, the parts a_4 and $\ln b_4$ do not depend on x_0; the chain rule with $dz/dx_0 = 6H$, Section 3.2).

$$\partial_0 \ln h_0 = \frac{1}{\cot z}\cdot\Big(-\frac{1}{\sin^2 z}\Big)\cdot 6H = -\frac{\sin z}{\cos z}\cdot\frac{6H}{\sin^2 z} = -\frac{6H}{\sin z\cos z}$$

(the derivative of $\ln u$ is u'/u; the derivative of $\cot z$ is $-1/\sin^2 z$; the chain rule). Since $1 = \sin^2 z + \cos^2 z$, $\frac{1}{\sin z\cos z} = \frac{\sin^2 z+\cos^2 z}{\sin z\cos z} = \tan z + \cot z$, so $\partial_0 \ln h_0 = -6H(\tan z + \cot z)$. Finally $h_4 = 1$ gives $\partial_4 \ln h_4 = \partial_0 \ln h_4 = 0$, and h_0 does not depend on x_4.

The four cases of Section 3.15. Case (a), $\Gamma^a{}_{aa} = \partial_a \ln h_a$: only $a = x_0$ gives a non-zero value,

$$\Gamma^{x_0}{}_{x_0x_0} = -6H(\tan z + \cot z) = -\frac{6H}{\sin z\cos z}.$$

Case (b), $\Gamma^a{}_{ab} = \Gamma^a{}_{ba} = \partial_b \ln h_a$ with $b \neq a$: with $b = x_4$,

$$\Gamma^{x_i}{}_{x_ix_4} = \alpha = a_4', \qquad \Gamma^{x_j}{}_{x_jx_4} = \alpha_t = -a_4' + \frac{b_4'}{2b_4},$$

and with $b = x_0$, for all six transverse directions,

$$\Gamma^{x_k}{}_{x_kx_0} = H\cot z$$

(the pairs $a = x_4$, $b = x_0$ and $a = x_0$, $b = x_4$ give zero, because h_4 is constant and h_0 does not depend on x_4). Case (c), $\Gamma^a{}_{bb} = -\eta_{aa}\eta_{bb}\,(h_b^2/h_a^2)\,\partial_a \ln h_b$ with $b \neq a$: it needs $\partial_a \ln h_b \neq 0$, so a is x_4 or x_0. For $a = x_4$ ($\eta_{44} = -1$, $h_4 = 1$) it is $+\eta_{bb}\,h_b^2\,\partial_4 \ln h_b = \eta_{kk}\,\alpha_k\,h_k^2$:

$$\Gamma^{x_4}{}_{x_ix_i} = (+1)\,h_i^2\,\alpha = a_4'\,e^{2a_4}\sin^{1/3} z, \qquad \Gamma^{x_4}{}_{x_jx_j} = (-1)\,h_j^2\,\alpha_t = \Big(a_4'b_4 - \frac{b_4'}{2}\Big)e^{-2a_4}\sin^{1/3} z$$

(with $h_j^2 = b_4 e^{-2a_4}\sin^{1/3} z$; for a constant b_4 the two minus signs of the extra times, from η_{jj} and from the deflation $\alpha_t = -a_4'$, multiply to $+1$). For $a = x_0$ ($\eta_{00} = +1$, $h_0^2 = \cot^2 z$) it is $-\eta_{bb}\,(h_b^2/\cot^2 z)\,H\cot z = -\eta_{bb}\,H\tan z\,h_b^2$:

$$\Gamma^{x_0}{}_{x_i x_i} = -H\tan z\; e^{2a_4}\sin^{1/3} z, \qquad \Gamma^{x_0}{}_{x_j x_j} = +H\tan z\; b_4 e^{-2a_4}\sin^{1/3} z$$

($\cot z/\cot^2 z = 1/\cot z = \tan z$). Case (d) gives only zeros.

The complete list. Counting both orders of the lower indices: case (a) 1 symbol; case (b) 2×6 with x_4 and 2×6 with x_0; case (c) 6 with upper x_4 and 6 with upper x_0: $1+12+12+6+6 = 37$ non-zero symbols of the $8^3 = 512$. With the lower indices ordered ($b \le c$, in the order $x_0,\dots,x_7$) there are $1+6+6+6+6 = 25$:

symbol ($b \le c$)	**value**	**how many**
$\Gamma^{x_0}{}_{x_0 x_0}$	$-6H(\tan z + \cot z)$	1
$\Gamma^{x_0}{}_{x_i x_i}$	$-H\tan z\, e^{2a_4}\sin^{1/3} z$	3
$\Gamma^{x_0}{}_{x_j x_j}$	$H\tan z\, b_4 e^{-2a_4}\sin^{1/3} z$	3
$\Gamma^{x_k}{}_{x_0 x_k}$, $k = 1,2,3,5,6,7$	$H\cot z$	6
$\Gamma^{x_i}{}_{x_i x_4}$, $i = 1,2,3$	a_4'	3
$\Gamma^{x_4}{}_{x_i x_i}$	$a_4'\, e^{2a_4}\sin^{1/3} z$	3
$\Gamma^{x_4}{}_{x_j x_j}$	$\big(a_4' b_4 - \frac{1}{2}b_4'\big)e^{-2a_4}\sin^{1/3} z$	3
$\Gamma^{x_j}{}_{x_4 x_j}$, $j = 5,6,7$	$-a_4' + b_4'/(2b_4)$	3

This is exactly the list `christoffelNonzero_b_le_c` of the record `Revision/gkd_lovelock/results/curvature.json` (PROVED here by hand; Notebook 03b reproduces it symbol by symbol, In [7]; the independent verification of the record agrees, `python-lovelock-report.json`, check `rust_christoffels_agree`). The record writes $\tan z$ as `Cot[6*H*x0]^(-1)` and $\Gamma^{x_4}{}_{x_j x_j}$ as the sum `-(1/2)*E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*Derivative[1][b4][x4] + Derivative[1][a4][x4]*E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*b4[x4]`. Every symbol has at least one index x_4 or x_0, the two coordinates on which the metric depends. The deflation is visible in the last row: for a constant b_4 the extra-time symbols $\Gamma^{x_j}{}_{x_4 x_j} = -a_4'$ have the opposite sign of the 3-space ones.

Observers at rest fall freely. No symbol has the lower pair (x_4, x_4): case (a) with $a = x_4$ is $\partial_4 \ln h_4 = 0$, and case (c) with $b = x_4$ contains $\partial_a \ln h_4 = 0$. So $\Gamma^a{}_{x_4 x_4} = 0$ for every a, for all functions a_4 and b_4. An observer who keeps $x_0, x_1, x_2, x_3, x_5, x_6, x_7$ fixed while $x_4 = \tau$ runs has $u^a = dx^a/d\tau = 1$ for $a = x_4$ and 0 otherwise, and $du^a/d\tau = 0$; the geodesic equation of Section 3.16 reads $0 + \Gamma^a{}_{x_4 x_4}\cdot 1\cdot 1 = 0$, which holds. The squared speed is $g(u,u) = g_{44} = -1$, so $\tau = x_4$ is the observer's proper time. **Observers at rest are in free fall, and their clocks show** x_4 (PROVED; Notebook 03b, In [12]).

The contracted symbols. By Section 3.15, $\sum_a \Gamma^a{}_{ab} = \partial_b \ln\sqrt{|\det g|} = \partial_b \ln(b_4^{3/2}\cos z)$. Check with the list: for $b = x_4$, $\sum_a \Gamma^a{}_{a4} = 3\alpha + 3\alpha_t = 3a_4' - 3a_4' + \frac{3}{2}b_4'/b_4 = \frac{3}{2}b_4'/b_4$, and $\partial_4 \ln b_4^{3/2} = \frac{3}{2}b_4'/b_4$; for $b = x_0$, $\sum_a \Gamma^a{}_{a0} = 6H\cot z - 6H(\tan z + \cot z) = -6H\tan z$, and $\partial_0 \ln\cos z = -\frac{\sin z}{\cos z}\cdot 6H = -6H\tan z$. Both agree (Notebook 03b, In [12]). In the x_4 sum the three inflating and the three deflating directions cancel through a_4; only b_4 is left.

3.24 The curvature of the author's metric, by hand

The curvature of every coordinate plane. For a diagonal metric, Section 3.17 gives $\sigma(a,b) = R^{ab}{}_{ab} = g^{bb}R^{a}{}_{bab}$ (no sum), with

$$R^{a}{}_{bab} = \partial_a\Gamma^{a}{}_{bb} - \partial_b\Gamma^{a}{}_{ba} + \sum_e (\Gamma^{a}{}_{ae}\Gamma^{e}{}_{bb} - \Gamma^{a}{}_{be}\Gamma^{e}{}_{ba})$$

(the formula of Section 3.17 with $c = a$ and $d = b$). The 28 coordinate planes fall into seven kinds. We compute each kind once, with general labels, so that the computation holds for every plane of that kind. Two facts from the list of Section 3.23 are used again and again: a derivative ∂_i, ∂_j of anything is zero (nothing depends on the transverse coordinates), and the only non-zero symbols with the upper index x_k are $\Gamma^{x_k}{}_{x_kx_4}$, $\Gamma^{x_k}{}_{x_4x_k}$, $\Gamma^{x_k}{}_{x_kx_0}$ and $\Gamma^{x_k}{}_{x_0x_k}$. In the list, $\Gamma^{4}{}_{kk} = \eta_{kk}\alpha_k h_k^2$ and $\Gamma^{0}{}_{kk} = -\eta_{kk}H\tan z\, h_k^2$.

(1) Two different 3-space directions i and i' (3 planes):

$$R^{i}{}_{i'ii'} = 0 - 0 + \Gamma^{i}{}_{i4}\Gamma^{4}{}_{i'i'} + \Gamma^{i}{}_{i0}\Gamma^{0}{}_{i'i'} - 0 = \alpha\cdot\alpha h_{i'}^2 + H\cot z\cdot\left(-H\tan z\, h_{i'}^2\right) = (\alpha^2 - H^2)\, h_{i'}^2$$

($\Gamma^{i}{}_{i'i'} = \Gamma^{i}{}_{i'i} = 0$ because $i' \neq i$; in $\sum_e \Gamma^{i}{}_{ie}\Gamma^{e}{}_{i'i'}$ only $e = x_4$ and $e = x_0$ survive; every $\Gamma^{i}{}_{i'e}$ is zero; $\cot z\tan z = 1$), so $\sigma(i,i') = g^{i'i'}(\alpha^2 - H^2)h_{i'}^2 = \alpha^2 - H^2$ (with $g^{i'i'} = 1/h_{i'}^2$).

(2) Two different extra times j and j' (3 planes):

$$R^{j}{}_{j'jj'} = \Gamma^{j}{}_{j4}\Gamma^{4}{}_{j'j'} + \Gamma^{j}{}_{j0}\Gamma^{0}{}_{j'j'} = \alpha_t\cdot(-\alpha_t h_{j'}^2) + H\cot z\cdot H\tan z\, h_{j'}^2 = (H^2 - \alpha_t^2)\, h_{j'}^2$$

($\Gamma^{j}{}_{j'j'} = \Gamma^{j}{}_{j'j} = 0$ because $j' \neq j$, so both derivative terms vanish; in $\sum_e \Gamma^{j}{}_{je}\Gamma^{e}{}_{j'j'}$ only $e = x_4$ and $e = x_0$ survive; every $\Gamma^{j}{}_{j'e}$ is zero; $\cot z\tan z = 1$), so $\sigma(j,j') = g^{j'j'}(H^2 - \alpha_t^2)h_{j'}^2 = \alpha_t^2 - H^2$ (with $g^{j'j'} = -1/h_{j'}^2$).

(3) A 3-space direction i and an extra time j (9 planes):

$$R^{i}{}_{jij} = \Gamma^{i}{}_{i4}\Gamma^{4}{}_{jj} + \Gamma^{i}{}_{i0}\Gamma^{0}{}_{jj} = \alpha\cdot(-\alpha_t h_j^2) + H\cot z\cdot H\tan z\, h_j^2 = (H^2 - \alpha\alpha_t)\, h_j^2,$$

($\Gamma^{i}{}_{jj} = \Gamma^{i}{}_{ji} = 0$, so both derivative terms vanish; in $\sum_e \Gamma^{i}{}_{ie}\Gamma^{e}{}_{jj}$ only $e = x_4$ and $e = x_0$ survive; every $\Gamma^{i}{}_{je}$ is zero; $\cot z\tan z = 1$), so $\sigma(i,j) = g^{jj}(H^2 - \alpha\alpha_t)h_j^2 = \alpha\alpha_t - H^2$ (with $g^{jj} = -1/h_j^2$).

(4) A 3-space direction i and the time x_4 (3 planes):

$$R^{i}{}_{4i4} = \partial_i\Gamma^{i}{}_{44} - \partial_4\Gamma^{i}{}_{4i} + \sum_e \Gamma^{i}{}_{ie}\Gamma^{e}{}_{44} - \sum_e \Gamma^{i}{}_{4e}\Gamma^{e}{}_{4i} = 0 - \alpha' + 0 - \alpha^2$$

(∂_i gives zero; $\partial_4\alpha = \alpha' = a_4''$; every $\Gamma^{e}{}_{44}$ is zero, Section 3.23; in the last sum only $e = i$ survives), so $\sigma(i,4) = g^{44}(-\alpha' - \alpha^2) = \alpha^2 + \alpha'$ (with $g^{44} = -1$).

(5) An extra time j and the time x_4 (3 planes): the same three steps with $a = j$,

$$R^{j}{}_{4j4} = -\partial_4 \Gamma^{j}{}_{4j} - (\Gamma^{j}{}_{4j})^2 = -\alpha_t' - \alpha_t^2,$$

so $\sigma(j,4) = g^{44}(-\alpha_t' - \alpha_t^2) = \alpha_t^2 + \alpha_t'$, with the derivative of the extra-time rate

$$\alpha_t' = -a_4'' + \frac{b_4''}{2b_4} - \frac{b_4'^2}{2b_4^2}$$

(the quotient rule for $b_4'/(2b_4)$).

(6) A transverse direction k and the hidden direction x_0 (6 planes):

$$R^{k}{}_{0k0} = \partial_k \Gamma^{k}{}_{00} - \partial_0 \Gamma^{k}{}_{0k} + \Gamma^{k}{}_{k0}\Gamma^{0}{}_{00} - \Gamma^{k}{}_{0k}\Gamma^{k}{}_{0k}$$

(in $\sum_e \Gamma^{k}{}_{ke}\Gamma^{e}{}_{00}$ only $e = x_0$ survives, because $\Gamma^{e}{}_{00}$ is non-zero only for $e = x_0$; in $\sum_e \Gamma^{k}{}_{0e}\Gamma^{e}{}_{0k}$ only $e = k$)

$$= 0 + \frac{6H^2}{\sin^2 z} + H\cot z \cdot \big(-6H(\tan z + \cot z)\big) - H^2\cot^2 z = \frac{6H^2}{\sin^2 z} - 6H^2(1 + \cot^2 z) - H^2\cot^2 z$$

($\partial_0(H\cot z) = H\cdot(-1/\sin^2 z)\cdot 6H$, so $-\partial_0\Gamma^{k}{}_{0k} = +6H^2/\sin^2 z$; $\cot z \tan z = 1$)

$$= -H^2\cot^2 z$$

($1 + \cot^2 z = (\sin^2 z + \cos^2 z)/\sin^2 z = 1/\sin^2 z$, so the first two terms cancel). Hence $\sigma(k,0) = g^{00}(-H^2\cot^2 z) = \tan^2 z \cdot (-H^2\cot^2 z) = -H^2$. Neither a_4 nor b_4 enters: the x_0-dependence of every h_k is the same factor $\sin^{1/6} z$.

(7) The time x_4 and the hidden direction x_0 (1 plane):

$$R^{4}{}_{040} = \partial_4 \Gamma^{4}{}_{00} - \partial_0 \Gamma^{4}{}_{04} + \sum_e \Gamma^{4}{}_{4e}\Gamma^{e}{}_{00} - \sum_e \Gamma^{4}{}_{0e}\Gamma^{e}{}_{04} = 0$$

(the only non-zero symbols with upper index x_4 are $\Gamma^{4}{}_{kk}$, and none of $\Gamma^{4}{}_{00}$, $\Gamma^{4}{}_{04}$, $\Gamma^{4}{}_{4e}$, $\Gamma^{4}{}_{0e}$ is of that form), so $\sigma(4,0) = 0$: this plane is flat.

The table of the plane curvatures (PROVED; Notebook 03b computes all 28 and finds these seven formulas, In [16]), with $\alpha = a_4'$ and $\alpha_t = -a_4' + b_4'/(2b_4)$:

plane	**how many**	**curvature** $\sigma(a,b) = R^{ab}{}_{ab}$	**for a constant** b_4
two 3-space directions	3	$\alpha^2 - H^2$	$a_4'^2 - H^2$
two extra times	3	$\alpha_t^2 - H^2$	$a_4'^2 - H^2$
a 3-space direction and an extra time	9	$\alpha\alpha_t - H^2$	$-(a_4'^2 + H^2)$
a 3-space direction and the time x_4	3	$\alpha^2 + \alpha'$	$a_4'^2 + a_4''$

plane	**how many**	**curvature** $\sigma(a,b) = R^{ab}{}_{ab}$	**for a constant** b_4
an extra time and the time x_4	3	$\alpha_t^2 + \alpha_t'$	$a_4'^2 - a_4''$
a transverse direction and the hidden x_0	6	$-H^2$	$-H^2$
the time x_4 and the hidden x_0	1	0	0

None depends on z or on the value of a_4; only H, the rates α, α_t and their derivatives appear (and so a_4', a_4'', b_4'/b_4 and b_4''/b_4). Along the history $a_4 = AHx_4$, $b_4 = 1$ ($a_4' = AH$, $a_4'' = 0$) the planes inside 3-space and inside the extra times have $H^2(A^2-1)$: negative for $A < 1$, zero for the canonical $A = 1$, positive for $A > 1$. The planes that mix 3-space and the extra times have $-H^2(A^2+1)$, negative for every A.

What the signs mean. By Section 3.17 the sign of $\sigma(a,b)$ describes a shape only in the 6 planes of two space-like directions: the 3 planes inside 3-space ($\alpha^2 - H^2$; along the history saddle-like for $A < 1$, flat for $A = 1$, sphere-like for $A > 1$) and the 3 planes of a 3-space direction with x_0 ($-H^2$, saddle-like). In the 6 planes of two time-like directions (inside the extra times, and an extra time with x_4) the shape reading is reversed: there a positive σ would mean saddle-like. The other 16 planes contain one space-like and one time-like direction; for them, and for every plane that contains x_4, the sign is read from free fall, $d^2\xi/d\tau^2 = +\sigma\,\xi$ (Section 3.18). We now verify this rule by hand where it can be seen most simply: for the observers at rest of Section 3.23, which fall freely along x_4 with the proper time $\tau = x_4$. Take two of them that differ only by a small coordinate distance Δx_k along a transverse direction x_k. Their proper distance is

$$\xi = h_k\,\Delta x_k$$

(Section 3.7; Δx_k and z do not change along the two paths). Differentiate twice with respect to $\tau = x_4$, with $\partial_4 h_k = \alpha_k h_k$:

$$\frac{d\xi}{d\tau} = \alpha_k\,h_k\,\Delta x_k = \alpha_k\,\xi, \qquad \frac{d^2\xi}{d\tau^2} = \alpha_k'\,\xi + \alpha_k\,\frac{d\xi}{d\tau} = (\alpha_k^2 + \alpha_k')\,\xi$$

(the product rule). For $k = x_1$ the factor is exactly $\sigma(x_1,x_4) = \alpha^2 + \alpha'$ of case (4), and for $k = x_5$ exactly $\sigma(x_5,x_4) = \alpha_t^2 + \alpha_t'$ of case (5): $d^2\xi/d\tau^2 = +\sigma\,\xi$. For two observers a distance Δx_0 apart along x_0, $\xi = \cot z\,\Delta x_0$ does not change in time, $d^2\xi/d\tau^2 = 0$, in agreement with the flat plane of case (7). All three PROVED; Notebook 03b checks the first two exactly (In [16]). Along the history both $\sigma(x_1,x_4)$ and $\sigma(x_5,x_4)$ equal $A^2H^2 > 0$: a positive curvature, and yet the 3-space distance grows like $e^{AH\tau}$ and the extra-time distance $e^{-AH\tau}$ falls ever more slowly, both with a positive second derivative. On the sphere of Section 3.18 a positive curvature pulls neighbouring geodesics together; here it drives them apart.

The components that are not plane curvatures. Each plane with $\sigma \neq 0$ gives four non-zero components, $R^{ab}{}_{ab} = R^{ba}{}_{ba} = \sigma(a,b)$ and $R^{ab}{}_{ba} = R^{ba}{}_{ab} = -\sigma(a,b)$ (Section 3.17): $27 \times 4 = 108$ components (for general functions every one of the 27 curved planes has $\sigma \neq 0$). The other non-zero components connect the planes (x_4,x_k) and (x_0,x_k). First

$$R^4{}_{k0k} = \partial_0\Gamma^4{}_{kk} - \partial_k\Gamma^4{}_{k0} + \sum_e \Gamma^4{}_{0e}\Gamma^e{}_{kk} - \sum_e \Gamma^4{}_{ke}\Gamma^e{}_{k0} = \partial_0\Gamma^4{}_{kk} - \Gamma^4{}_{kk}\Gamma^k{}_{k0}$$

(the formula with $a = x_4$, $b = k$, $c = x_0$, $d = k$; $\Gamma^4{}_{k0} = 0$ and every $\Gamma^4{}_{0e} = 0$; in the last sum only $e = k$). With $\Gamma^4{}_{kk} = \eta_{kk}\alpha_k h_k^2$ (Section 3.23) and $\partial_0 h_k^2 = 2H\cot z\, h_k^2$ (twice the logarithmic derivative),

$$R^4{}_{k0k} = 2\eta_{kk}\alpha_k H\cot z\, h_k^2 - \eta_{kk}\alpha_k h_k^2\, H\cot z = \eta_{kk}\,\alpha_k H\cot z\, h_k^2,$$

and raising the second index, $R^{4k}{}_{0k} = g^{kk}R^4{}_{k0k}$ with $g^{kk} = \eta_{kk}/h_k^2$ and $\eta_{kk}^2 = 1$:

$$R^{4k}{}_{0k} = H\alpha_k\cot z, \qquad \text{that is} \qquad R^{4i}{}_{0i} = Ha_4'\cot z, \quad R^{4j}{}_{0j} = H\alpha_t\cot z.$$

Next, the components with the upper index x_0:

$$R^0{}_{k4k} = \partial_4\Gamma^0{}_{kk} - \partial_k\Gamma^0{}_{k4} + \sum_e \Gamma^0{}_{4e}\Gamma^e{}_{kk} - \sum_e \Gamma^0{}_{ke}\Gamma^e{}_{k4} = \partial_4\Gamma^0{}_{kk} - \Gamma^0{}_{kk}\Gamma^k{}_{k4}$$

(the formula with $a = x_0$, $b = k$, $c = x_4$, $d = k$; $\Gamma^0{}_{k4} = 0$, every $\Gamma^0{}_{4e} = 0$; in the last sum only $e = k$). With $\Gamma^0{}_{kk} = -\eta_{kk}H\tan z\, h_k^2$, $\partial_4 h_k^2 = 2\alpha_k h_k^2$ and $\Gamma^k{}_{k4} = \alpha_k$,

$$R^0{}_{k4k} = -2\eta_{kk}H\alpha_k\tan z\, h_k^2 + \eta_{kk}H\tan z\, h_k^2\,\alpha_k = -\eta_{kk}\,H\alpha_k\tan z\, h_k^2, \qquad R^{0k}{}_{4k} = -H\alpha_k\tan z.$$

For each of the six transverse k, the antisymmetries give four components of each of these two kinds: $6 \times 8 = 48$ components, of size $H|\alpha_k|\cot z$ or $H|\alpha_k|\tan z$. Together $108 + 48 = 156$, exactly the number of non-zero components $R^{ab}{}_{cd}$ that the Rust program of the record and Notebook 03b find (`Revision/gkd_lovelock/results/lovelock-report.json`, check `riemann_antisymmetry`: "156 nonzero entries"); so the list is complete. They take 20 different values: $\pm$ the five plane curvatures that are not $\pm H^2$, $\pm H^2$, and $\pm H\alpha\cot z$, $\pm H\alpha\tan z$, $\pm H\alpha_t\cot z$, $\pm H\alpha_t\tan z$ (Notebook 03b, In [15]). These 48 components depend on z, and they grow without bound at one end of the patch or the other. This is an effect of the coordinate x_0, not of the geometry. Measured with the rulers and clocks of the frame of Section 3.7, an upper index a is multiplied by its scale factor h_a (a step dx^a has the proper length $h_a\,dx^a$), and a lower index is divided by it (so that a contraction keeps its value, Section 3.14). The frame component of $R^{4k}{}_{0k}$ is therefore $\frac{h_4 h_k}{h_0 h_k}R^{4k}{}_{0k} = \tan z\cdot H\alpha_k\cot z = H\alpha_k$, and that of $R^{0k}{}_{4k}$ is $\cot z\cdot(-H\alpha_k\tan z) = -H\alpha_k$: they no longer depend on z. The plane curvatures keep their values, because there the factors cancel.

Where the deflation shows. Every formula above holds for each transverse direction with its own rate α_k; 3-space and the extra times differ only through α and α_t. For a constant b_4, $\alpha_t = -\alpha$: then the 48 components of 3-space and of the extra times have opposite signs, and the next section shows that these opposite signs make the Ricci tensor diagonal. When b_4 changes, $\alpha + \alpha_t = b_4'/(2b_4) \neq 0$ and the cancellation is incomplete.

3.25 Ricci, Einstein and Kretschmann of the author's metric; the contracted Bianchi identity

The diagonal of the Ricci tensor. By Section 3.17, $R^a{}_a$ is the sum of the curvatures of the seven planes that contain x_a. From the table of Section 3.24:

$$R^i{}_i = 2(\alpha^2 - H^2) + 3(\alpha\alpha_t - H^2) + (\alpha^2 + \alpha') - H^2 = 3\alpha^2 + 3\alpha\alpha_t + \alpha' - 6H^2$$

(two planes with the other 3-space directions, three with the extra times, one with x_4, one with x_0; then collect: $-2-3-1=-6$ times H^2);

$$R^4{}_4 = 3(\alpha^2 + \alpha') + 3(\alpha_t^2 + \alpha_t') + 0$$

(three planes with 3-space, three with the extra times, one with x_0);

$$R^j{}_j = 3(\alpha\alpha_t - H^2) + 2(\alpha_t^2 - H^2) + (\alpha_t^2 + \alpha_t') - H^2 = 3\alpha\alpha_t + 3\alpha_t^2 + \alpha_t' - 6H^2$$

(three planes with 3-space, two with the other extra times, one with x_4, one with x_0);

$$R^0{}_0 = 6\cdot(-H^2) + 0 = -6H^2$$

(six planes with the transverse directions, one with x_4). The Ricci scalar is the sum of the diagonal:

$$R = 3R^i{}_i + R^4{}_4 + 3R^j{}_j + R^0{}_0 = 12\alpha^2 + 18\alpha\alpha_t + 12\alpha_t^2 + 6\alpha' + 6\alpha_t' - 42H^2$$

(collect: $9+3=12$ times α^2, $9+9=18$ times $\alpha\alpha_t$, $3+9=12$ times α_t^2, $3+3=6$ times α' and α_t', $-18-18-6=-42$ times H^2). Inserting $\alpha = a_4'$, $\alpha_t = -a_4' + \beta$ with $\beta = b_4'/(2b_4)$, and $\alpha_t' = -a_4'' + b_4''/(2b_4) - 2\beta^2$: the terms $12\alpha^2 + 18\alpha\alpha_t + 12\alpha_t^2$ become $6a_4'^2 - 6a_4'\beta + 12\beta^2$, the terms $6\alpha' + 6\alpha_t'$ become $3b_4''/b_4 - 12\beta^2$, and

$$R = 6a_4'^2 - 3a_4'\frac{b_4'}{b_4} + 3\frac{b_4''}{b_4} - 42H^2.$$

For a constant b_4 ($\alpha_t = -\alpha$, $\alpha_t' = -\alpha'$) the four diagonal entries are $R^i{}_i = a_4'' - 6H^2$, $R^4{}_4 = 6a_4'^2$, $R^j{}_j = -a_4'' - 6H^2$, $R^0{}_0 = -6H^2$, and $R = 6a_4'^2 - 42H^2$.

The entries off the diagonal. An entry $R^a{}_b = \sum_c R^{ac}{}_{bc}$ with $a \neq b$ needs a non-zero component whose upper pair (a,c) and lower pair (b,c) differ in one index. The 108 plane components have the same two indices up and down, so they give nothing. The 48 others have the pairs $\{x_4, x_k\}$ and $\{x_0, x_k\}$, so the only candidates are $R^4{}_0$ and $R^0{}_4$:

$$R^4{}_0 = \sum_i R^{4i}{}_{0i} + \sum_j R^{4j}{}_{0j} = 3H(\alpha + \alpha_t)\cot z = \frac{3}{2}H\cot z\,\frac{b_4'}{b_4},$$

$$R^0{}_4 = \sum_i R^{0i}{}_{4i} + \sum_j R^{0j}{}_{4j} = -3H(\alpha + \alpha_t)\tan z = -\frac{3}{2}H\tan z\,\frac{b_4'}{b_4}$$

(Section 3.24; the terms $c = x_4$ and $c = x_0$ vanish by the antisymmetries; $\alpha + \alpha_t = b_4'/(2b_4)$). **The Ricci tensor is diagonal exactly when b_4 is constant**: then the three inflating and the three deflating directions cancel exactly. A changing b_4 leaves the two entries above. If the extra times inflated like 3-space, $g_{jj} = -e^{2a_4}\sin^{1/3} z$ (the member $b_4 = e^{4a_4}$ of the family, with $\alpha_t = +a_4'$), the extra-time terms would have the same sign as the 3-space terms, and $R^4{}_0$ would be $6Ha_4'\cot z$: Notebook 03b computes exactly this negative control (In [27]) and finds $G^{x_4}{}_{x_0} = 6Ha_4'\cot z$ and $G^{x_0}{}_{x_4} = -6Ha_4'\tan z$, the two general entries with $b_4'/b_4 = 4a_4'$.

The Einstein tensor. $G^a{}_b = R^a{}_b - \frac{1}{2}\delta^a{}_b R$. Off the diagonal $\delta^a{}_b = 0$, so $G^4{}_0 = R^4{}_0$ and $G^0{}_4 = R^0{}_4$. On the diagonal, subtracting $\frac{1}{2}R = 3a_4'^2 - \frac{3}{2}a_4'b_4'/b_4 + \frac{3}{2}b_4''/b_4 - 21H^2$ from each entry of the Ricci tensor (written out with a_4 and b_4 as above):

$$G^0{}_0 = 15H^2 - 3a_4'^2 + \frac{3a_4'b_4'}{2b_4} - \frac{3b_4''}{2b_4}, \quad G^i{}_i = 15H^2 - 3a_4'^2 + a_4'' + \frac{3a_4'b_4'}{b_4} - \frac{3b_4''}{2b_4},$$
$$G^4{}_4 = 21H^2 + 3a_4'^2 - \frac{3a_4'b_4'}{2b_4} - \frac{3b_4'^2}{4b_4^2}, \quad G^j{}_j = 15H^2 - 3a_4'^2 - a_4'' + \frac{b_4'^2}{4b_4^2} - \frac{b_4''}{b_4}.$$

$G^4{}_4$ contains no second derivative: in the rates, $G^4{}_4 = 21H^2 - 3(\alpha^2 + 3\alpha\alpha_t + \alpha_t^2)$. For a constant b_4,

$$G^0{}_0 = 15H^2 - 3a_4'^2, \qquad G^i{}_i = a_4'' - 3a_4'^2 + 15H^2, \qquad G^4{}_4 = 3a_4'^2 + 21H^2, \qquad G^j{}_j = -a_4'' - 3a_4'^2 + 15H^2,$$

and every entry off the diagonal is zero. These are exactly the entries `ricciMixed`, `ricciScalar` and `einsteinMixed` of `Revision/gkd_lovelock/results/curvature.json` (PROVED here; Notebook 03b reproduces all 64 + 64 + 1 of them, In [17]; record check `rust_ricci_einstein_scalar_agree` of `python-lovelock-report.json`), and the components `b4.lovelockTensors.E1` of the record `Revision/field_equations_a4/a4-equations.json`. The verifier of the metric with b_4 states the off-diagonal pair (`Revision/metric_b4/results/wolfram-metric-b4-report.json`, checks `off_diagonal_entries_x0_x4_only` and `off_diagonal_proportional_to_b4prime`). The lead's independent check, made for $b_4 = 1$, states the same properties there (`Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, checks `einstein_off_diagonal_zero`, `einstein_x0_independent`, `einstein_isotropy`): at $b_4 = 1$, G is diagonal and no entry depends on x_0; for every b_4 the three 3-space entries are equal, as are the three extra-time entries. Moreover, at $b_4 = 1$,

$$G^4{}_4 - G^0{}_0 = 3a_4'^2 + 21H^2 - 15H^2 + 3a_4'^2 = 6(a_4'^2 + H^2) > 0 \quad \text{for } H > 0$$

(record check `no_vacuum_for_H_positive`), a fact used in Section 3.26.

The Kretschmann scalar. In $K = \sum_{a,b,c,d} R^{ab}{}_{cd}R^{cd}{}_{ab}$ each component is multiplied by its partner with the two pairs exchanged. The four components of a plane are their own partners or each other's, and each of the four products is $\sigma(a,b)^2$ (Section 3.17), so the planes contribute $4\sum_{\text{planes}}\sigma(a,b)^2$. Each of the 48 other components is paired with one of the other kind, $R^{4k}{}_{0k}R^{0k}{}_{4k} = (H\alpha_k\cot z)(-H\alpha_k\tan z) = -H^2\alpha_k^2$; the sign changes of the antisymmetries act on both factors, so the 24 products of 3-space equal $-H^2\alpha^2$ and the 24 of the extra times $-H^2\alpha_t^2$. Therefore

$$K = 4\sum_{\text{planes}}\sigma(a,b)^2 - 24H^2(\alpha^2 + \alpha_t^2),$$

$$\sum_{\text{planes}} \sigma(a,b)^2 = 3(\alpha^2 - H^2)^2 + 3(\alpha_t^2 - H^2)^2 + 9(\alpha\alpha_t - H^2)^2 + 3(\alpha^2 + \alpha')^2 + 3(\alpha_t^2 + \alpha_t')^2 + 6H^4$$

(the table of Section 3.24). In every product $\cot z$ meets $\tan z$, so although 48 components depend on z and grow without bound at the ends of the patch, the invariant K does not depend on z, and it does not depend on the value of a_4. For a constant b_4 ($\alpha_t = -\alpha = -a_4'$, $\alpha_t' = -a_4''$) the sum is

$$\sum_{\text{planes}} \sigma(a,b)^2 = 6(a_4'^2 - H^2)^2 + 9(a_4'^2 + H^2)^2 + 3(a_4'^2 + a_4'')^2 + 3(a_4'^2 - a_4'')^2 + 6H^4$$

$$= (6a_4'^4 - 12a_4'^2H^2 + 6H^4) + (9a_4'^4 + 18a_4'^2H^2 + 9H^4) + (6a_4'^4 + 6a_4''^2) + 6H^4 = 21a_4'^4 + 6a_4'^2H^2 + 21H^4 + 6a_4''^2$$

(the squares expanded; in $3(p+q)^2 + 3(p-q)^2 = 6p^2 + 6q^2$ the mixed terms cancel), and therefore

$$K = 4(21a_4'^4 + 6a_4'^2H^2 + 21H^4 + 6a_4''^2) - 48H^2a_4'^2 = 12(7H^4 - 2H^2a_4'^2 + 7a_4'^4 + 2a_4''^2)$$

($24 - 48 = -24$ times $a_4'^2H^2$; then 12 taken out). Completing the square,

$$\frac{K}{12} = 7\Big(a_4'^2 - \frac{H^2}{7}\Big)^2 + \frac{48}{7}H^4 + 2a_4''^2$$

(expand: $7a_4'^4 - 2H^2a_4'^2 + H^4/7$, and $H^4/7 + 48H^4/7 = 7H^4$), so $K \geq \frac{576}{7}H^4 > 0$ for every history with a constant b_4: **with a constant b_4 the author's metric is curved everywhere, for every function $a_4(x_4)$ and every $H > 0$** (PROVED; Notebook 03b checks the formula and computes K a second time from the 156 components of the record, In [22]). For a changing b_4 there is no such bound: for the member $b_4 = e^{4a_4}$ of the negative control ($\alpha_t = \alpha$, $\alpha_t' = \alpha'$) the same two formulas give $K = 12\big(7(a_4'^2 - H^2)^2 + 4a_4'^2a_4'' + 2a_4''^2\big)$, which vanishes for $a_4' = H$, $a_4'' = 0$ (Notebook 03b, In [27]). Along the history $a_4 = AHx_4$, $b_4 = 1$: $R = 6H^2(A^2 - 7)$ and $K = 12H^4(7A^4 - 2A^2 + 7)$; for the canonical $A = 1$, $R = -36H^2$ and $K = 144H^4$.

The contracted Bianchi identity. For every metric the Einstein tensor has zero **divergence**, $\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$ for each ν (the divergence is the covariant derivative with its index μ set equal to the upper index of G and summed; a standard theorem, the **contracted Bianchi identity**; quoted, ASSUMED in general). For the author's metric we prove it directly. By the rule of Section 3.15 (one $+\Gamma$ for the upper index, one $-\Gamma$ for the lower one, then the upper index contracted with the derivative),

$$\sum_\mu \nabla_\mu G^\mu{}_\nu = \sum_\mu \partial_\mu G^\mu{}_\nu + \sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda} G^\lambda{}_\nu - \sum_{\mu,\lambda} \Gamma^\lambda{}_{\mu\nu} G^\mu{}_\lambda .$$

For the diagonal entries of G the first sum keeps $\mu = \nu$, the second $\lambda = \nu$ and the third $\lambda = \mu$; the two entries $G^4{}_0$ and $G^0{}_4$ add their own terms. Write $\Gamma_b = \sum_\mu \Gamma^\mu{}_{\mu b}$ for the contracted symbols of Section 3.23: $\Gamma_4 = 3(\alpha + \alpha_t)$ and $\Gamma_0 = -6H\tan z$.

For $\nu = x_4$: the diagonal part is $\partial_4 G^4{}_4 + 3\alpha(G^4{}_4 - G^i{}_i) + 3\alpha_t(G^4{}_4 - G^j{}_j)$ (the sum over μ of $\Gamma^\mu{}_{\mu 4}(G^4{}_4 - G^\mu{}_\mu)$, with $\Gamma^i{}_{i4} = \alpha$ and $\Gamma^j{}_{j4} = \alpha_t$), and $G^0{}_4$ adds $\partial_0 G^0{}_4 + \Gamma_0\, G^0{}_4$:

$$\partial_0 G^0{}_4 + \Gamma_0\, G^0{}_4 = -\frac{3}{2}\,6H^2 \frac{1}{\cos^2 z}\frac{b_4'}{b_4} + 6H\tan z \cdot \frac{3}{2} H \tan z\, \frac{b_4'}{b_4} = -9H^2 \frac{b_4'}{b_4}\Big(\frac{1}{\cos^2 z} - \tan^2 z\Big) = -9H^2 \frac{b_4'}{b_4}$$

(the derivative of $\tan z$ is $1/\cos^2 z$, times $6H$; and $1/\cos^2 z - \tan^2 z = (1-\sin^2 z)/\cos^2 z = 1$). For a constant b_4 this term is zero, $\partial_4 G^4{}_4 = \partial_4(3a_4'^2 + 21H^2) = 6a_4' a_4''$ (the chain rule), and with $\alpha_t = -a_4'$:

$$\begin{aligned}\sum_\mu \nabla_\mu G^\mu{}_4 &= 6a_4' a_4'' + 3a_4'(G^4{}_4 - G^i{}_i) - 3a_4'(G^4{}_4 - G^j{}_j) \\ &= 6a_4' a_4'' + 3a_4'(G^j{}_j - G^i{}_i) = 6a_4' a_4'' + 3a_4'(-2a_4'') = 0.\end{aligned}$$

For a changing b_4 the H^2 terms show how the off-diagonal entry is needed: in $G^4{}_4 - G^i{}_i$ and $G^4{}_4 - G^j{}_j$ the constants are $21H^2 - 15H^2 = 6H^2$, which contribute $3(\alpha+\alpha_t)\cdot 6H^2 = 9H^2\, b_4'/b_4$, and this is cancelled exactly by $-9H^2\, b_4'/b_4$ from $G^0{}_4$; the terms without H cancel among themselves, as sympy confirms.

For $\nu = x_0$: the diagonal part has $\partial_0 G^0{}_0 = 0$ and $\Gamma^k{}_{k0} = H\cot z$ for the six transverse k (the term $\mu = x_0$ has the factor $G^0{}_0 - G^0{}_0 = 0$), and $G^4{}_0$ adds $\partial_4 G^4{}_0 + \Gamma_4\, G^4{}_0$:

$$\sum_\mu \nabla_\mu G^\mu{}_0 = H\cot z\Big[3(G^0{}_0 - G^i{}_i) + 3(G^0{}_0 - G^j{}_j) + \frac{3}{2}\Big(\frac{b_4'}{b_4}\Big)' + \frac{9}{4}\Big(\frac{b_4'}{b_4}\Big)^2\Big]$$

(with $\Gamma_4 = 3(\alpha+\alpha_t) = \frac{3}{2}b_4'/b_4$). From the entries above, $3(G^0{}_0 - G^i{}_i) + 3(G^0{}_0 - G^j{}_j) = -\frac{3}{2}b_4''/b_4 - \frac{3}{4}(b_4'/b_4)^2$, and $\frac{3}{2}(b_4'/b_4)' = \frac{3}{2}b_4''/b_4 - \frac{3}{2}(b_4'/b_4)^2$ (the quotient rule): the bracket is $-\frac{3}{2} + \frac{3}{2} = 0$ times b_4''/b_4 and $-\frac{3}{4} - \frac{3}{2} + \frac{9}{4} = 0$ times $(b_4'/b_4)^2$, so it vanishes. (For a constant b_4 the bracket is $3(-a_4'') + 3a_4''$.)

For ν a transverse direction: $\partial_\nu G^\mu{}_\nu = 0$, every $\Gamma^\mu{}_{\mu\nu}$ is zero (case (b) of Section 3.15 gives $\partial_\nu \ln h_\mu = 0$), and every term $\Gamma^\lambda{}_{\mu\nu} G^\mu{}_\lambda$ needs an entry of G with one transverse index and one other index, which is zero. So $\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$ for all eight ν and all functions $a_4(x_4)$, $b_4(x_4)$ (PROVED; Notebook 03b, In [21]; record `Revision/gkd_lovelock/results/lovelock-report.json`, checks `k1_equals_minus_4_einstein` and `k1_divergence_free`). The same computation for a diagonal energy-momentum tensor $T = \mathrm{diag}(p_0, p_3, p_3, p_3, -\rho, p_t, p_t, p_t)$ gives $\sum_\mu \nabla_\mu T^\mu{}_4 = -\partial_4\rho - 3\alpha(\rho+p_3) - 3\alpha_t(\rho+p_t)$, which for a constant b_4 is $-\partial_4\rho - 3a_4'(p_3 - p_t)$: energy flows between 3-space and the extra times when their pressures differ (record `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`, check `divergence_x4_component`, made for $b_4 = 1$; Chapter 9 derives and uses it).

3.26 What Einstein's equations would ask of the source

Einstein's field equations. In Einstein's theory of gravity the curvature is tied to the matter by

$$G^\mu{}_\nu + \Lambda\, \delta^\mu{}_\nu = \kappa\, T^\mu{}_\nu,$$

with a positive constant κ (the strength of gravity), a constant Λ (the **cosmological constant**) and the **energy-momentum tensor** $T^{\mu}{}_{\nu}$ of the matter, whose diagonal for the sources of this book is $T = \mathrm{diag}(p_0, p_3, p_3, p_3, -\rho, p_t, p_t, p_t)$ in the order $x_0, \ldots, x_7$: the **energy density** ρ and the **pressures** p_0 of the hidden direction, p_3 of 3-space and p_t of the extra times (record `Revision/field_equations_a4/a4-equations.json`, key `conventions.source`). We take this form of the equations as given here (ASSUMED in this chapter); Chapter 9 derives T for the fields of the book, and Chapter 12 derives the equations, together with their higher-order Lovelock companions, and solves them. (The **Lovelock tensors** are curvature tensors built from products of one, two or three Riemann tensors with the generalized Kronecker delta of Chapter 1; the first of them is a multiple of the Einstein tensor, $P_{(1)} = -4G$, and the other two vanish identically in four dimensions; Chapters 11 and 12 treat them in full.) Why does the left side have this form? The conservation of energy and momentum demands that the right side has zero divergence, $\sum_{\mu} \nabla_{\mu} T^{\mu}{}_{\nu} = 0$, so the left side must have zero divergence for every metric. The contracted Bianchi identity of Section 3.25 shows that the Einstein tensor has this property, and $\Lambda\, \delta^{\mu}{}_{\nu}$ has it too (its covariant derivative is zero, by the rule of Section 3.15 with constant entries, because the $+\Gamma$ and the $-\Gamma$ term cancel). The Einstein tensor is the simplest curvature tensor with zero divergence, but not the only one: in eight dimensions the second and the third Lovelock tensors also have zero divergence for every metric (record `Revision/gkd_lovelock/results/lovelock-report.json`, checks `k2_divergence_free` and `k3_divergence_free`), and the field equations of the Revision record use all three, $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^{\mu}{}_{\nu} + \Lambda\, \delta^{\mu}{}_{\nu} = \kappa\, T^{\mu}{}_{\nu}$ with $E_{(1)} = G$ (record `Revision/field_equations_a4/a4-equations.json`, key `conventions.fieldEquations`). This chapter uses only the simplest case, Einstein's equations ($\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$).

A diagonal source fixes b_4. The x_4-x_0 component of the equations reads $G^4{}_0 = \kappa T^4{}_0$. For a source of the diagonal form above, $T^4{}_0 = 0$, and so $\frac{3}{2} H \cot z\, b_4'/b_4 = 0$: since $H > 0$ and $\cot z > 0$ inside the patch, $b_4' = 0$ (PROVED here, from Section 3.25). Within Einstein's equations a diagonal source therefore requires a constant b_4, and a constant b_4 is removed by the rescaling of the extra times of Section 3.8, which gives back $b_4 = 1$. (With the higher Lovelock terms the x_4-x_0 component carries a further factor, and the statement changes; `Revision/SPEC.md`, section 5, and Chapter 12.) The rest of this section therefore takes $b_4 = 1$.

The equations for the author's metric. With the Einstein tensor of Section 3.25 at $b_4 = 1$ the four independent diagonal components are

$$3a_4'^2 + 21H^2 + \Lambda = -\kappa\rho, \qquad -3a_4'^2 + a_4'' + 15H^2 + \Lambda = \kappa p_3,$$

$$-3a_4'^2 - a_4'' + 15H^2 + \Lambda = \kappa p_t, \qquad -3a_4'^2 + 15H^2 + \Lambda = \kappa p_0$$

(the x_4 component uses $T^4{}_4 = -\rho$; the others $T^{\mu}{}_{\mu} = p_{\mu}$). These are the four equations `einstein.constraint_x4`, `einstein.space_x1`, `einstein.extraTime_x5` and `einstein.hidden_x0` of the record `a4-equations.json`. Subtracting the third from the second gives $2a_4'' = \kappa(p_3 - p_t)$, the **evolution equation** of the record (`einstein.evolution`): the acceleration a_4'' of the metric function is driven by the difference between the pressure of 3-space and that of the extra times. The record also states that every twice-differentiable function $a_4(x_4)$ is allowed, with the source then fixed by these equations (`einstein.allowedA4`).

No empty spacetime has this metric. Without matter, $T = 0$, the x_4-x_0 component gives a constant b_4, that is $b_4 = 1$ after the rescaling, and the x_4 and x_0 components would give $G^4{}_4 + \Lambda = 0$

and $G^0{}_0 + \Lambda = 0$, hence $G^4{}_4 = G^0{}_0$; but $G^4{}_4 - G^0{}_0 = 6(a_4'^2 + H^2) > 0$ (Section 3.25). So for $H > 0$ the author's metric is not a solution of Einstein's equations without matter, for any Λ and any b_4 (PROVED; record `a4-equations.json`, key `einstein.noVacuum`; lead check `no_vacuum_for_H_positive`). Subtracting the x_0 equation from the x_4 equation also gives $\kappa(\rho+p_0) = -6(a_4'^2+H^2) < 0$ for every history (record key `einstein.nullEnergy`; lead check `einstein_null_energy`). (With the author's own Lovelock weights the vacuum equations do have solutions, b_4 constant and $a_4 = \pm Hx_4 + \text{const}$: `Revision/metric_b4/results/vacuum-b4.json`; Chapter 12.)

The source along the deflating history. For $a_4 = AHx_4$, $b_4 = 1$ ($a_4' = AH$, $a_4'' = 0$) the evolution equation forces $p_3 = p_t$, and the hidden, 3-space and extra-time equations have the same left side, so all three pressures are one number p:

$$\kappa\rho = -3H^2(7 + A^2) - \Lambda, \qquad \kappa p = 3H^2(5 - A^2) + \Lambda, \qquad \kappa(\rho + p) = -6H^2(A^2 + 1)$$

(the x_4 equation solved for $\kappa\rho$; the x_1 equation with $a_4'' = 0$; the sum of the two, in which Λ cancels). These are the entries `rhoEinstein`, `pEinstein` and `rhoPlusPEinstein` of the key `linearMember` of the record `Revision/field_equations_a4/a4-equations.json` (PROVED here; Notebook 03b reads the three texts of the record and checks them against its Einstein tensor, In [26]). For the canonical $A = 1$, $H = 1$: $G = \text{diag}(12, 12, 12, 12, 24, 12, 12, 12)$ in the order $x_0, \ldots, x_7$, $\kappa\rho = -24 - \Lambda$ and $\kappa p = 12 + \Lambda$.

What this means, with its status. Ordinary matter (dust, radiation, a gas) has $\rho + p \geq 0$; this is the **null energy condition** (a standard condition of physics, used here as the definition of "ordinary", ASSUMED). Within Einstein's equations, the deflating history needs $\kappa(\rho + p) = -6H^2(A^2 + 1) < 0$ for every slope A and every Λ: whatever fills such a spacetime, it is not ordinary matter (PROVED). Three things are NOT shown here: whether a field of this book can supply such a source (Chapters 9, 12 and 17 answer this; the Kohn-Sham states computed along the history cannot, `Revision/field_equations_a4/reports/ks-source-conditions.json`, which is why the history is a PRESCRIBED BACKGROUND, Section 3.8); what the higher-order Lovelock terms change (Chapter 12); and anything about pairs of universes or about matter and antimatter, which this chapter does not touch.

3.27 Example: the curvature of the author's metric

Notebook 03b does Sections 3.23 to 3.26 by computer algebra, for general functions $a_4(x_4)$ and $b_4(x_4)$. It reads the metric exactly as the author typed it; computes all 512 Christoffel symbols with the formula of Section 3.15 and compares the 25 non-zero ones with the record; draws them; checks them a second way by finite differences at the test point of the record's brute-force check; checks that observers at rest fall freely; computes the 156 non-zero Riemann components, checks their symmetries and the first Bianchi identity and compares every one with the record; computes the 28 plane curvatures, the Ricci tensor, the Ricci scalar and the Einstein tensor and compares them with the record, with the verifier of the metric with b_4 and, at $b_4 = 1$, with the lead's checks; compares 310 components with the record numerically with 30 digits at the record's five test points; checks the contracted Bianchi identity; computes the Kretschmann scalar; evaluates everything along the deflating history ($b_4 = 1$); reproduces the source of Section 3.26 from the record; and repeats the curvature for the negative control with inflating extra times ($b_4 = e^{4a_4}$). It needs no Rust, runs in about 1 minute (the cells with the Riemann tensor and the negative control take up to half a minute each) and ends with the line ALL 40 CHECKS PASSED (notebook 03b).

3.28 How to run Notebook 03b

Step 1. What this notebook does and what it needs.

Notebook 03b (The curvature of the author's metric: Christoffel, Riemann, Ricci, Einstein, Kretschmann) is the file `Revision/textbook/notebooks/03b_curvature.ipynb` of the repository Dirac_claude. It computes with sympy, from the metric exactly as the author typed it and with general functions a4(x4) and b4(x4), all 512 Christoffel symbols, the Riemann tensor, the Ricci tensor, the Ricci scalar, the Einstein tensor and the Kretschmann scalar; it compares every component with the Revision record of the Rust program lovelock_gkd (exactly, and numerically at the five test points of the independent Revision verification), checks the symmetries of the Riemann tensor, the first and the contracted Bianchi identities, the Christoffel symbols by finite differences, and a negative control with inflating extra times; and it draws the Christoffel symbols, the curvature of every coordinate plane, the components and the curvature scalars versus z and versus the expansion rate, and the Einstein tensor; along the deflating history a4 = A H x4, b4 = 1 it reproduces the source that the Einstein equations require in the Revision record of the field equations of a4. It shows that the Einstein tensor has the off-diagonal components G^x4_x0 and G^x0_x4, proportional to b4', and that for the planes that contain the time x4 the plane curvature is the relative acceleration of neighbouring observers at rest. Every check of a Revision report that a PASS line names is opened and asserted to exist and to have passed. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
```

```
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 03b_curvature.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 03b_curvature.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 03b_curvature.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/03b.captions.json`
- `Revision/textbook/figures/03b_1_christoffel_heat_maps.png`
- `Revision/textbook/figures/03b_2_christoffel_versus_z.png`
- `Revision/textbook/figures/03b_3_finite_differences.png`
- `Revision/textbook/figures/03b_4_plane_curvatures.png`
- `Revision/textbook/figures/03b_5_riemann_and_kretschmann_z.png`
- `Revision/textbook/figures/03b_6_scalars_versus_a.png`
- `Revision/textbook/figures/03b_7_einstein_tensor.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all seven figure files exist
ALL 40 CHECKS PASSED (notebook 03b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/03b_curvature.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 03b_curvature.ipynb
python -m nbconvert --execute --inplace 03b_curvature.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cells with the Riemann tensor and the negative control run for up to half a minute each, and their label shows a star meanwhile: this is normal: sympy simplifies several thousand expressions exactly; wait until the label shows a number.

3.29 Notebook 03b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/03b_curvature.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook computes the **curvature** of the author's metric of the primordial
gravitational field, exactly, with sympy, for general metric functions $a_4(x_4)$
and $b_4(x_4)$. It

- reads the metric exactly as the author typed it (from the Revision record
  `Revision/gkd_lovelock/results/curvature.json`);
- computes all $8^3 = 512$ **Christoffel symbols** $\Gamma^a{}_{bc}$, finds the 37
  that are not zero, checks them against the record of the Rust program
  `lovelock_gkd`, draws them as heat maps and as functions of $z$, and checks them a
  second, independent way by **finite differences** (a numerical derivative);
- shows that the lines along the time $x_4$ are **geodesics** (free fall) with
  proper time $x_4$;
- computes the **Riemann tensor** $R^a{}_{bcd}$ and its form $R^{ab}{}_{cd}$ with two
  upper indices, finds its 156 non-zero components, checks its symmetries and the
  first Bianchi identity, and compares all 156 with the record;
- computes the curvature $\sigma(a, b)$ of each of the 28 coordinate planes and
  shows that for the planes with the time $x_4$ it is the relative acceleration of
  two neighbouring observers at rest;
- computes the **Ricci tensor**, the **Ricci scalar** and the **Einstein tensor**,
  compares them with the record and the lead checks, finds its two off-diagonal
  components $G^{x_4}{}_{x_0}$ and $G^{x_0}{}_{x_4}$, proportional to $b_4'$, and
  checks the contracted Bianchi identity $\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$;
- computes the **Kretschmann scalar** $K = R^{ab}{}_{cd} R^{cd}{}_{ab}$, shows that it
  does not depend on $z$ although single components do, and that for a constant
  $b_4$ it is never zero for $H > 0$;
- evaluates everything along the **deflating history** $a_4 = A H x_4$ ($A > 0$),
  $b_4 = 1$ and
  checks that minus the time component and the space component of the Einstein
  tensor are exactly the energy density and the pressure that the Revision record
  of the field equations of $a_4$ lists for this history;
```

- repeats the curvature for a **negative control** in which the extra times inflate
  instead of deflating, and shows what then changes;
- opens every Revision report whose check a PASS line names, and stops unless that
  check is listed there as passed;
- draws 7 figures and prints a PASS line for every check.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 03b (The curvature of the author's metric: Christoffel, Riemann, Ricci,
Einstein, Kretschmann) is the file 'Revision/textbook/notebooks/03b_curvature.ipynb' of
the repository Dirac_claude. It computes with sympy, from the metric exactly as the
author typed it and with general functions a4(x4) and b4(x4), all 512 Christoffel
symbols, the Riemann tensor, the Ricci tensor, the Ricci scalar, the Einstein tensor and
the Kretschmann scalar; it compares every component with the Revision record of the Rust
program lovelock_gkd (exactly, and numerically at the five test points of the independent
Revision verification), checks the symmetries of the Riemann tensor, the first and the
contracted Bianchi identities, the Christoffel symbols by finite differences, and a
negative control with inflating extra times; and it draws the Christoffel symbols, the
curvature of every coordinate plane, the components and the curvature scalars versus z
and versus the expansion rate, and the Einstein tensor; along the deflating history a4 =
A H x4, b4 = 1 it reproduces the source that the Einstein equations require in the
Revision record of the field equations of a4. It shows that the Einstein tensor has the
off-diagonal components G^x4_x0 and G^x0_x4, proportional to b4', and that for the planes
that contain the time x4 the plane curvature is the relative acceleration of neighbouring
observers at rest. Every check of a Revision report that a PASS line names is opened and
asserted to exist and to have passed. It needs a computer with Windows 11, macOS or
Linux, an internet connection for the installation, the program Git, and Python 3.12 or
newer (the notebooks were built with Python 3.14.5) with these packages at exactly these
versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10,
nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook
itself imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the
program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):
```

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 03b_curvature.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 03b_curvature.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 03b_curvature.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/03b.captions.json‘
- ‘Revision/textbook/figures/03b_1_christoffel_heat_maps.png‘
- ‘Revision/textbook/figures/03b_2_christoffel_versus_z.png‘
- ‘Revision/textbook/figures/03b_3_finite_differences.png‘
- ‘Revision/textbook/figures/03b_4_plane_curvatures.png‘
- ‘Revision/textbook/figures/03b_5_riemann_and_kretschmann_z.png‘
- ‘Revision/textbook/figures/03b_6_scalars_versus_a.png‘
- ‘Revision/textbook/figures/03b_7_einstein_tensor.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all seven figure files exist
ALL 40 CHECKS PASSED (notebook 03b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/03b_curvature.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 03b_curvature.ipynb
  python -m nbconvert --execute --inplace 03b_curvature.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cells with the Riemann tensor and the negative control run for up to half a minute each, and their label shows a star meanwhile: this is normal: sympy simplifies several thousand expressions exactly; wait until the label shows a number.

The first code cell below (the set-up cell) repeats these instructions as comments; then

it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 03b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 03b (The curvature of the author's metric: Christoffel, Riemann, Ricci,
# Einstein, Kretschmann) is the file Revision/textbook/notebooks/03b_curvature.ipynb of
# the repository Dirac_claude. It computes with sympy, from the metric exactly as the
# author typed it and with general functions a4(x4) and b4(x4), all 512 Christoffel
# symbols, the Riemann tensor, the Ricci tensor, the Ricci scalar, the Einstein tensor
# and the Kretschmann scalar; it compares every component with the Revision record of the
# Rust program lovelock_gkd (exactly, and numerically at the five test points of the
# independent Revision verification), checks the symmetries of the Riemann tensor, the
# first and the contracted Bianchi identities, the Christoffel symbols by finite
# differences, and a negative control with inflating extra times; and it draws the
# Christoffel symbols, the curvature of every coordinate plane, the components and the
# curvature scalars versus z and versus the expansion rate, and the Einstein tensor;
# along the deflating history a4 = A H x4, b4 = 1 it reproduces the source that the
# Einstein equations require in the Revision record of the field equations of a4. It
# shows that the Einstein tensor has the off-diagonal components G^x4_x0 and G^x0_x4,
# proportional to b4', and that for the planes that contain the time x4 the plane
# curvature is the relative acceleration of neighbouring observers at rest. Every check
# of a Revision report that a PASS line names is opened and asserted to exist and to have
# passed. It needs a computer with Windows 11, macOS or Linux, an internet connection for
# the installation, the program Git, and Python 3.12 or newer (the notebooks were built
# with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy,
# mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
```

```
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
```

```
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 03b_curvature.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 1 minute
```

```
# on a typical laptop. Scroll to the end and compare the last printed lines with the
# lines in the step "What the notebook writes and what you must see" below. To keep the
# results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the
# menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 03b_curvature.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 03b_curvature.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/03b.captions.json
# - Revision/textbook/figures/03b_1_christoffel_heat_maps.png
# - Revision/textbook/figures/03b_2_christoffel_versus_z.png
# - Revision/textbook/figures/03b_3_finite_differences.png
# - Revision/textbook/figures/03b_4_plane_curvatures.png
# - Revision/textbook/figures/03b_5_riemann_and_kretschmann_z.png
# - Revision/textbook/figures/03b_6_scalars_versus_a.png
# - Revision/textbook/figures/03b_7_einstein_tensor.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all seven figure files exist
#     ALL 40 CHECKS PASSED (notebook 03b)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
```

```
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/03b_curvature.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 03b_curvature.ipynb
#     python -m nbconvert --execute --inplace 03b_curvature.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - The cells with the Riemann tensor and the negative control run for up to half a
#   minute each, and their label shows a star meanwhile: this is normal: sympy simplifies
#   several thousand expressions exactly; wait until the label shows a number.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "03b"  # this notebook: chapter 03, example b
```

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 03b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Metric** $g_{\mu\nu}$, **inverse metric** $g^{\mu\nu}$ (the inverse matrix:
  $\sum_\lambda g^{\mu\lambda} g_{\lambda\nu} = \delta^\mu{}_\nu$, which is $1$ when
  $\mu = \nu$ and $0$ otherwise). For a diagonal metric $g^{\mu\mu} = 1/g_{\mu\mu}$.
- **Index notation**: $a, b, c, \dots$ each run over the eight coordinates
  $x_0, \dots, x_7$; an upper and a lower index are positions in a table of numbers;
  $\partial_c$ means the partial derivative $\partial/\partial x_c$.
- **Christoffel symbols** $\Gamma^a{}_{bc} = \tfrac12 \sum_d g^{ad}(\partial_b g_{dc}
  + \partial_c g_{db} - \partial_d g_{bc})$: the correction terms that make
  derivatives of vectors independent of the coordinates. They are symmetric in $b$
  and $c$.
- **Geodesic**: a curve $x^a(\tau)$ with $d^2x^a/d\tau^2 + \sum_{b,c}
  \Gamma^a{}_{bc}\,(dx^b/d\tau)(dx^c/d\tau) = 0$, the path of free fall, the
  straightest possible curve.
- **Riemann tensor** $R^a{}_{bcd} = \partial_c \Gamma^a{}_{bd} - \partial_d
  \Gamma^a{}_{bc} + \sum_e (\Gamma^a{}_{ce}\Gamma^e{}_{bd} - \Gamma^a{}_{de}
  \Gamma^e{}_{bc})$ (the convention of the textbook of Misner, Thorne and Wheeler,
  MTW, used by every Revision record): it measures how much a vector turns when it
  is carried around a small closed loop. It is zero everywhere exactly when the space
  is flat.
- $R^{ab}{}_{cd} = \sum_e g^{be} R^a{}_{ecd}$: the same tensor with the second index
  raised. For the coordinate plane of $x_a$ and $x_b$, $\sigma(a, b) =
  R^{ab}{}_{ab}$ (no sum) is the **curvature of that plane** (the sectional
  curvature). Only for a plane of two space-like directions does its sign describe
  a shape: positive curved like a sphere, negative like a saddle. For a plane with
  a time-like direction the sign is read from free fall: two neighbouring
  free-fall paths that move along the time-like direction $x_b$, a small proper
  distance $\xi$ apart along $x_a$, accelerate apart as $d^2\xi/d\tau^2 =
  +\sigma(a, b)\,\xi$, while on a sphere $d^2\xi/ds^2 = -\sigma\,\xi$: the same sign
  of $\sigma$ has the opposite effect.
- **Lead checks**: short independent Python programs of the Revision record (folder
  ‘Revision/lead_checks‘), written from scratch by the coordinator of the Revision
  work (the "lead") without importing any other Revision code.
- **Ricci tensor** $R^a{}_b = \sum_c R^{ac}{}_{bc}$, **Ricci scalar**
  $R = \sum_a R^a{}_a$, **Einstein tensor** $G^a{}_b = R^a{}_b - \tfrac12
  \delta^a{}_b R$: the averages of the curvature that enter Einstein’s field
  equations.
- **Kretschmann scalar** $K = \sum_{a,b,c,d} R^{ab}{}_{cd} R^{cd}{}_{ab}$: one number
  per point, the same in every coordinate system (an **invariant**).
- **Einstein’s field equations** $G^\mu{}_\nu + \Lambda\,\delta^\mu{}_\nu =
  \kappa\, T^\mu{}_\nu$: the Einstein tensor plus a constant $\Lambda$ (the
  **cosmological constant**) times $\delta^\mu{}_\nu$ equals a positive constant
  $\kappa$ times the **energy-momentum tensor** $T^\mu{}_\nu$ of the matter. For the
  sources of this book its diagonal is $T = \mathrm{diag}(p_0, p_3, p_3, p_3, -\rho,
  p_t, p_t, p_t)$ in the order $x_0, \dots, x_7$: the **energy density** $\rho$
  (energy per unit volume) and the
  **pressures** $p_3$ of 3-space, $p_t$ of the extra times and $p_0$ of the hidden
  direction. A later chapter derives these equations; this notebook only reads
```

```
  them in a Revision record.
- **Bianchi identities**: $R^a{}_{bcd} + R^a{}_{cdb} + R^a{}_{dbc} = 0$ (first) and
  $\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$ (contracted): identities that every metric
  satisfies; checking them checks the computation.
- **Lovelock scalars and tensors**: curvature quantities built from products of one,
  two or three Riemann tensors with the generalized Kronecker delta; the first
  Lovelock scalar is $L_{(1)} = 2R$ and the first Lovelock tensor is
  $P_{(1)} = -4G$. A later chapter treats them in full.
- **Finite difference**: the numerical derivative $f'(x) \approx (f(x+h) -
  f(x-h))/(2h)$; its error shrinks like $h^2$.
- `a4p`, `a4pp`, `a4v`: the names the notebook prints for $a_4'$, $a_4''$ and the
  value of $a_4$; `b4p`, `b4pp`, `b4v` the same for $b_4$; `z` stands for $6 H x_0$.
```

Text cell:

```
## 4. The physical and mathematical situation

The metric of the author is diagonal,

$$g = \mathrm{diag}\bigl(\cot^2 z,\ e^{2a_4}\sin^{1/3}z\ (\times 3),\ -1,\
-b_4 e^{-2a_4}\sin^{1/3}z\ (\times 3)\bigr), \qquad z = 6 H x_0,$$

in the order $x_0, \dots, x_7$, with the constant $H > 0$ and the metric functions
$a_4(x_4)$ and $b_4(x_4) > 0$ ($b_4 = 1$ is the metric used before 2026-10-14). It
depends on two coordinates only, $x_4$ (through $a_4$ and $b_4$) and $x_0$ (through
$z$). Every derivative along $x_1, x_2, x_3, x_5, x_6, x_7$ is therefore
zero, which is why most of the $512$ Christoffel symbols and most of the $4096$
Riemann components vanish.

Curvature is computed in three steps, each from the previous one: the Christoffel
symbols need the first derivatives of the metric; the Riemann tensor needs the
first derivatives of the Christoffel symbols and their products (so the second
derivatives of the metric, and $a_4''$ and $b_4''$ appear); the Ricci tensor, the
Ricci scalar
and the Einstein tensor are sums of Riemann components. The Revision record
computed all of this with an exact Rust program (`lovelock_gkd`); an independent
sympy program checked every component, and the lead checks recomputed the Einstein
tensor with their own sympy code. This notebook does it once more, in small steps,
and compares every single component.

Nothing here involves the matter fields yet: these are exact properties of the given
metric. The field equations, which connect the Einstein tensor to the energy and the
pressures of the fields, are the subject of a later chapter. For the plots the
notebook uses $H = 1$ and the deflating history $a_4 = A H x_4$, $b_4 = 1$ of the
Revision Kohn-Sham work (so $a_4' = A H$, $a_4'' = 0$ and $b_4' = b_4'' = 0$, with
$A = 1$ the canonical value): a PRESCRIBED BACKGROUND, assumed here and not obtained
by solving the field equations. Every formula with general $a_4(x_4)$ and
$b_4(x_4)$ is exact.
```

Text cell:

```
## 5. The metric, read from the record
```

First a technical step. While a cell runs, Jupyter sends what the cell prints to the notebook in pieces, about one piece every 0.2 seconds, so where one piece ends and the next begins depends on the speed of the computer. The tools that build this book read the PASS line of a check together with the line under it that names the Revision record, and they read the two correctly only when both arrive in the same piece. The next cell therefore defines the function ‘in_one_piece(helper)‘: it returns a new function that does exactly what ‘helper‘ does, but lets ‘helper‘ print into a text buffer in memory (an ‘io.StringIO‘, with ‘contextlib.redirect_stdout‘) and then writes the whole buffer with one call of ‘sys.stdout.write‘, which arrives as one piece. (The stars in ‘*arguments‘ and ‘**options‘ pass on all the values the new function is given, whatever they are.) The cell then replaces the helpers ‘check‘ and ‘report‘ of the set-up cell by such functions. Nothing else changes: the same checks, the same PASS lines, the same numbers.

In [2]:

```
import contextlib  # redirect_stdout: send printed text into a buffer
import io  # StringIO: a text buffer in memory
import sys  # sys.stdout: the place where printed text goes


def in_one_piece(helper):
    """A function that does what helper does, with all the lines that helper prints
    written in one piece."""
    def helper_in_one_piece(*arguments, **options):
        collected = io.StringIO()  # an empty text buffer
        with contextlib.redirect_stdout(collected):  # print() writes into it
            helper(*arguments, **options)  # a failing check stops here, as before
        sys.stdout.write(collected.getvalue())  # all the lines with one write
    return helper_in_one_piece


check = in_one_piece(check)  # a PASS line and its record line stay together
report = in_one_piece(report)  # a long RESULT line and its second line, too
say("From now on check and report print each of their results in one piece.")
```

Out [2]:

```
From now on check and report print each of their results in one piece.
```

Text cell:

The next cell imports the packages, reads the record, and makes the symbols: the eight coordinates, $H > 0$, the unknown functions $a_4(x_4)$ and $b_4(x_4)$ (b_4 positive), and the printing names ‘a4p‘, ‘a4pp‘, ‘a4v‘, ‘b4p‘, ‘b4pp‘, ‘b4v‘ and ‘z‘. The function ‘plain‘ rewrites an expression with these printing names; the function ‘symbolic‘ does the same but keeps x_0 (the record writes its components with x_0). It also names the two reports of the Revision curvature computation and defines the function ‘record_check(report_file, check_name, detail_part)‘, the same as in Notebook 03a: it opens a Revision report,

finds the check with that name, and stops the notebook with an error unless the
check is there and passed (and, if ‘detail_part‘ is given, its detail contains that
text). Every PASS line below that names a check of a Revision report calls it
first, so that a renamed, missing or failing check in the record is caught.

In [3]:

```
import itertools  # loops over all index combinations

import mpmath  # numbers with many digits
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
from sympy.parsing.sympy_parser import (implicit_multiplication, parse_expr,
                                        standard_transformations)

CURVATURE_RECORD = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE_RECORD).read_text(encoding="utf-8"))
PYTHON_REPORT = "Revision/gkd_lovelock/results/python-lovelock-report.json"  # sympy
RUST_REPORT = "Revision/gkd_lovelock/results/lovelock-report.json"  # the Rust checks


def record_check(report_file, check_name, detail_part=""):
    """Return True when the Revision report report_file lists the check check_name
    as passed (and its detail contains detail_part); otherwise stop the notebook."""
    checks = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
    checks = checks["checks"]  # a dictionary or a list, depending on the report
    if isinstance(checks, dict):  # {name: {"passed": true, "detail": ...}}
        entry = checks.get(check_name, {})
        passed = entry.get("passed") is True
    else:  # [{"name": ..., "verdict": "PASS", "detail": ...}, ...]
        entry = next((e for e in checks if e.get("name") == check_name), {})
        passed = entry.get("verdict") == "PASS"
    if not passed or detail_part not in entry.get("detail", ""):
        raise AssertionError(f"record check failed: {report_file} does not list "
                             f"{check_name} as passed")
    return True


x0, x1, x2, x3, x4, x5, x6, x7 = sp.symbols("x0:8", real=True)
X = [x0, x1, x2, x3, x4, x5, x6, x7]  # the coordinates, counted 0 to 7 in Python
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
H = sp.symbols("H", positive=True)  # the constant of the author
a4 = sp.Function("a4")(x4)  # the metric function a4, unknown
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0, unknown
a4p, a4pp, a4v = sp.symbols("a4p a4pp a4v", real=True)  # a4 prime, a4 two primes, a4
b4p, b4pp = sp.symbols("b4p b4pp", real=True)  # b4 prime, b4 two primes
b4v = sp.symbols("b4v", positive=True)  # the value of b4
z = sp.symbols("z", positive=True)  # z = 6 H x0, for printing


def symbolic(expression):
    """Write the derivatives of a4 and b4 as a4p, a4pp, b4p, b4pp and the values
    a4(x4), b4(x4) as a4v, b4v."""
```

```
    expression = expression.subs(sp.Derivative(a4, (x4, 2)), a4pp)  # second first
    expression = expression.subs(sp.Derivative(b4, (x4, 2)), b4pp)
    expression = expression.subs(sp.Derivative(a4, x4), a4p).subs(a4, a4v)
    return expression.subs(sp.Derivative(b4, x4), b4p).subs(b4, b4v)


def plain(expression):
    """As symbolic, and 6 H x0 written as z."""
    return symbolic(expression).subs(x0, z / (6 * H))


say("Record read; symbols x0 ... x7, H, a4(x4), b4(x4), a4p, a4pp, a4v, b4p, b4pp, "
    "b4v and z defined.")
```

Out [3]:

```
Record read; symbols x0 ... x7, H, a4(x4), b4(x4), a4p, a4pp, a4v, b4p, b4pp, b4v and z
    defined.
```

Text cell:

The next cell turns the author's text of the metric into a sympy matrix, with the same translation rules as Notebook 03a (the note in brackets after the matrix is cut off; square brackets of functions become round brackets, ‘a4[x4]‘ and ‘b4[x4]‘ become ‘a4‘ and ‘b4‘, curly brackets of lists become square brackets, ‘^‘ becomes ‘**‘, and a blank between factors means times), and prints the diagonal.

In [4]:

```
# the metric exactly as the author typed it, without the note in brackets after it
text = record["metricAsGiven"].split("}} ", 1)[0] + "}}"
text = text.replace("exp^", "E^").replace("a4[x4]", "a4").replace("b4[x4]", "b4")
text = text.replace("Sin[6 H x0]", "sin(6 H x0)").replace("Cot[6 H x0]", "cot(6 H x0)")
text = text.replace("{", "[").replace("}", "]").replace("^", "**")
metric_names = {"E": sp.E, "a4": a4, "b4": b4, "H": H, "x0": x0, "sin": sp.sin,
                "cot": sp.cot}
g = sp.Matrix(parse_expr(text, local_dict=metric_names, transformations=(
    standard_transformations + (implicit_multiplication,))))
for k in range(8):
    say(f"  g[{NAMES[k]}, {NAMES[k]}] = {plain(g[k, k])}")
check(all(g[i, j] == 0 for i in range(8) for j in range(8) if i != j),
      "the metric read from the text of the author is diagonal")
```

Out [4]:

```
  g[x0, x0] = cot(z)**2
  g[x1, x1] = exp(2*a4v)*sin(z)**(1/3)
  g[x2, x2] = exp(2*a4v)*sin(z)**(1/3)
  g[x3, x3] = exp(2*a4v)*sin(z)**(1/3)
```

```
  g[x4, x4] = -1
  g[x5, x5] = -b4v*exp(-2*a4v)*sin(z)**(1/3)
  g[x6, x6] = -b4v*exp(-2*a4v)*sin(z)**(1/3)
  g[x7, x7] = -b4v*exp(-2*a4v)*sin(z)**(1/3)
PASS the metric read from the text of the author is diagonal
```

Text cell:

```
The record writes its components in Mathematica notation, for example
‘Derivative[1][a4][x4]*E^(2*a4[x4])*Sin[6*H*x0]^(1/3)‘. The next cell defines the
function ‘from_mathematica‘ that translates such a text into sympy:
‘Derivative[1][a4][x4]‘ ($a_4’$) becomes ‘a4p‘, ‘Derivative[2][a4][x4]‘ becomes
‘a4pp‘, ‘a4[x4]‘ becomes ‘a4v‘, the same for $b_4$ (‘b4p‘, ‘b4pp‘, ‘b4v‘), the sine
and the cotangent get round brackets, and ‘^‘ becomes ‘**‘. If any square bracket is
left, the translation is incomplete and the function stops with an error, so a
silent mistranslation is impossible.

The cell also defines three helpers. ‘tidy(expression)‘ simplifies an expression:
it writes $6Hx_0$ as $z$, simplifies it once as it is and once after writing
$\sin 2z$ as $2\sin z\cos z$ (‘expand_trig‘), and keeps the shorter of the two
results (sympy sometimes reaches the simplest form only one way: it writes, for
example, $-6H/(\sin z\cos z)$ as $-12H/\sin(2z)$, the same number because
$\sin 2z = 2\sin z\cos z$, and a sum such as $6H\cot z - 12H/\sin 2z + 6H\tan z$ is
recognised as $0$ only after the rewriting). ‘vanishes(expression)‘ is true when
‘tidy‘ gives $0$, and ‘same(mine, theirs)‘ is true when the difference of our
expression and the record’s vanishes.
```

In [5]:

```
record_names = {"a4p": a4p, "a4pp": a4pp, "a4v": a4v, "b4p": b4p, "b4pp": b4pp,
                "b4v": b4v, "H": H, "x0": x0, "E": sp.E, "sin": sp.sin, "cot": sp.cot}


def from_mathematica(entry_text):
    """A component of the record (Mathematica notation) as a sympy expression."""
    t = entry_text.replace("Derivative[1][a4][x4]", "a4p")
    t = t.replace("Derivative[2][a4][x4]", "a4pp").replace("a4[x4]", "a4v")
    t = t.replace("Derivative[1][b4][x4]", "b4p")
    t = t.replace("Derivative[2][b4][x4]", "b4pp").replace("b4[x4]", "b4v")
    t = t.replace("Sin[6*H*x0]", "sin(6*H*x0)").replace("Cot[6*H*x0]", "cot(6*H*x0)")
    t = t.replace("^", "**")
    if "[" in t or "]" in t:
        raise ValueError(f"cannot translate {entry_text}")
    return parse_expr(t, local_dict=record_names)


def tidy(expression):
    """Simplify an expression.  It writes 6 H x0 as z (so that 12 H x0 becomes 2z),
    simplifies it twice, once as it is and once after writing sin(2z) as
    2 sin(z) cos(z) (expand_trig), keeps the shorter result (count_ops counts the
    operations), and writes z as 6 H x0 again."""
    expression = sp.sympify(expression).subs(x0, z / (6 * H))
```

```
    first = sp.simplify(expression)
    second = sp.simplify(sp.expand_trig(expression))
    best = second if sp.count_ops(second) < sp.count_ops(first) else first
    return best.subs(z, 6 * H * x0)


def vanishes(expression):
    """True when the expression simplifies to zero."""
    return tidy(expression) == 0


def same(mine, theirs):
    """True when mine (with the function a4) equals theirs (record symbols)."""
    return vanishes(symbolic(mine) - theirs)


example = next(e["value"] for e in record["christoffelNonzero_b_le_c"]
               if (e["a"], e["b"], e["c"]) == ("x4", "x5", "x5"))  # Gamma^x4_(x5 x5)
say(f"example of a record entry: {example}")
say(f"translated: {from_mathematica(example)}")
```

Out [5]:

```
example of a record entry: -(1/2)*E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*Derivative[1][b4][x4] +
    Derivative[1][a4][x4]*E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*b4[x4]
translated: a4p*b4v*exp(-2*a4v)*sin(6*H*x0)**(1/3) - b4p*exp(-2*a4v)*sin(6*H*x0)**(1/3)/2
```

Text cell:

```
## 6. The Christoffel symbols

The Christoffel symbols of the metric are

$$\Gamma^a{}_{bc} = \tfrac12 \sum_{d} g^{ad}\bigl(\partial_b g_{dc} +
\partial_c g_{db} - \partial_d g_{bc}\bigr).$$

The next cell defines the function ‘christoffel(metric, coordinates)‘, which applies
this formula literally to every one of the $8 \times 8 \times 8 = 512$ index
combinations (it does not use the symmetry in $b$ and $c$, so that the symmetry can
be checked afterwards), simplifies each result, and returns the table
‘Gamma[a][b][c]‘. Then it computes the symbols of the author’s metric and counts
the non-zero ones.
```

In [6]:

```
def christoffel(metric, coordinates):
    """All Christoffel symbols Gamma[a][b][c] of a metric, by the formula above."""
    n = len(coordinates)
    inverse = metric.inv()  # the inverse metric g^(ad)
    d_metric = [metric.diff(c) for c in coordinates]  # d_metric[c][i, j] = d_c g_ij
    table = [[[0] * n for _ in range(n)] for _ in range(n)]
```

```
    for a, b, c in itertools.product(range(n), repeat=3):
        value = sum(inverse[a, d] * (d_metric[b][d, c] + d_metric[c][d, b]
                                     - d_metric[d][b, c]) for d in range(n)) / 2
        table[a][b][c] = tidy(value)
    return table


Gamma = christoffel(g, X)
nonzero = [(a, b, c) for a, b, c in itertools.product(range(8), repeat=3)
           if Gamma[a][b][c] != 0]
upper_half = [(a, b, c) for a, b, c in nonzero if b <= c]
report("non-zero Christoffel symbols (all orders of b, c)", len(nonzero))
report("non-zero Christoffel symbols with b <= c", len(upper_half))
check(all(Gamma[a][b][c] == Gamma[a][c][b]
          for a, b, c in itertools.product(range(8), repeat=3)),
      "the Christoffel symbols are symmetric in their two lower indices")
```

Out [6]:

```
RESULT non-zero Christoffel symbols (all orders of b, c) = 37
RESULT non-zero Christoffel symbols with b <= c = 25
PASS the Christoffel symbols are symmetric in their two lower indices
```

Text cell:

```
The next cell prints the 25 non-zero symbols with $b \le c$ (the others follow by the
symmetry) and compares them with the list ‘christoffelNonzero_b_le_c‘ of the record:
first that the two lists name the same 25 index combinations, then that each value
agrees exactly. In the record the keys ‘a‘, ‘b‘, ‘c‘ are the indices of
$\Gamma^a{}_{bc}$, and the list runs in the author’s order $x_0, \dots, x_7$.
sympy chooses its own way of writing some symbols: $H\cot z$ appears as
‘H/tan(z)‘, $-He^{2a_4}\sin^{1/3}z\,\tan z$ as ‘-H*exp(2*a4v)*sin(z)**(4/3)/cos(z)‘
(because $\sin^{1/3}z\,\tan z = \sin^{4/3}z/\cos z$), and
$\Gamma^{x_0}{}_{x_0x_0} = -6H/(\sin z\cos z)$ as ‘-12*H/sin(2*z)‘. The symbols of
the extra times contain $b_4$: $\Gamma^{x_5}{}_{x_4x_5} = -a_4’ + b_4’/(2b_4)$.
```

In [7]:

```
for a, b, c in upper_half:
    say(f"  Gamma^{NAMES[a]}_({NAMES[b]} {NAMES[c]}) = {plain(Gamma[a][b][c])}")
record_gamma = {(NAMES.index(e["a"]), NAMES.index(e["b"]), NAMES.index(e["c"])):
                from_mathematica(e["value"])
                for e in record["christoffelNonzero_b_le_c"]}
check(sorted(record_gamma) == upper_half and len(upper_half) == 25,
      "the same 25 non-zero Christoffel symbols as the record")
record_check(PYTHON_REPORT, "rust_christoffels_agree")  # stops if not passed
check(all(same(Gamma[a][b][c], value) for (a, b, c), value in record_gamma.items()),
      "every Christoffel symbol equals the record exactly",
      record=f"{CURVATURE_RECORD}, christoffelNonzero_b_le_c (and "
             "python-lovelock-report.json, check rust_christoffels_agree)")
```

Out [7]:

```
  Gamma^x0_(x0 x0) = -12*H/sin(2*z)
  Gamma^x0_(x1 x1) = -H*exp(2*a4v)*sin(z)**(4/3)/cos(z)
  Gamma^x0_(x2 x2) = -H*exp(2*a4v)*sin(z)**(4/3)/cos(z)
  Gamma^x0_(x3 x3) = -H*exp(2*a4v)*sin(z)**(4/3)/cos(z)
  Gamma^x0_(x5 x5) = H*b4v*exp(-2*a4v)*sin(z)**(4/3)/cos(z)
  Gamma^x0_(x6 x6) = H*b4v*exp(-2*a4v)*sin(z)**(4/3)/cos(z)
  Gamma^x0_(x7 x7) = H*b4v*exp(-2*a4v)*sin(z)**(4/3)/cos(z)
  Gamma^x1_(x0 x1) = H/tan(z)
  Gamma^x1_(x1 x4) = a4p
  Gamma^x2_(x0 x2) = H/tan(z)
  Gamma^x2_(x2 x4) = a4p
  Gamma^x3_(x0 x3) = H/tan(z)
  Gamma^x3_(x3 x4) = a4p
  Gamma^x4_(x1 x1) = a4p*exp(2*a4v)*sin(z)**(1/3)
  Gamma^x4_(x2 x2) = a4p*exp(2*a4v)*sin(z)**(1/3)
  Gamma^x4_(x3 x3) = a4p*exp(2*a4v)*sin(z)**(1/3)
  Gamma^x4_(x5 x5) = (a4p*b4v - b4p/2)*exp(-2*a4v)*sin(z)**(1/3)
  Gamma^x4_(x6 x6) = (a4p*b4v - b4p/2)*exp(-2*a4v)*sin(z)**(1/3)
  Gamma^x4_(x7 x7) = (a4p*b4v - b4p/2)*exp(-2*a4v)*sin(z)**(1/3)
  Gamma^x5_(x0 x5) = H/tan(z)
  Gamma^x5_(x4 x5) = -a4p + b4p/(2*b4v)
  Gamma^x6_(x0 x6) = H/tan(z)
  Gamma^x6_(x4 x6) = -a4p + b4p/(2*b4v)
  Gamma^x7_(x0 x7) = H/tan(z)
  Gamma^x7_(x4 x7) = -a4p + b4p/(2*b4v)
PASS the same 25 non-zero Christoffel symbols as the record
PASS every Christoffel symbol equals the record exactly
     reproduces Revision/gkd_lovelock/results/curvature.json, christoffelNonzero_b_le_c
    (and python-lovelock-report.json, check rust_christoffels_agree)
```

Text cell:

```
The next cell sets the colours of the figures (a palette whose colours can also be
told apart with a colour-vision deficiency; the figures also use line styles and
labels) and turns every Christoffel symbol into a fast numerical function of $z$,
$a_4$, $a_4'$, $H$, $b_4$ and $b_4'$ (`sp.lambdify`). It then draws the eight
$8 \times 8$ tables $\Gamma^{x_k}{}_{bc}$ at the point $z = \pi/4$ with $H = 1$,
$a_4 = 0.5$, $a_4' = 1$, $b_4 = 1$, $b_4' = 0$ as heat maps. The colour scale is
**symmetric-logarithmic**: above $0.1$ in size, equal steps of colour stand for
equal factors of size (below $0.1$ the scale is
ordinary), so that small and large symbols are both visible; grey is zero, red
positive, blue negative.
```

In [8]:

```
from matplotlib.colors import LinearSegmentedColormap, SymLogNorm

BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
MAGENTA, GREEN, VIOLET, RED = "#e87ba4", "#008300", "#4a3aa7", "#e34948"
```

```
DIVERGING = LinearSegmentedColormap.from_list("blue_grey_red", [BLUE, "#f0efec", RED])
LABELS = [f"$x_{k}$" for k in range(8)]
ARGS = (z, a4v, a4p, H, b4v, b4p)  # the arguments of the numerical functions


def numeric(expression):
    """A numerical function f(z, a4v, a4p, H, b4v, b4p) of an expression."""
    return sp.lambdify(ARGS, plain(expression), "numpy")


gamma_functions = {(a, b, c): numeric(Gamma[a][b][c]) for a, b, c in nonzero}
# z, a4, a4p, H, b4, b4p: the canonical slice a4 = 0.5 of the history, b4 = 1
POINT = (np.pi / 4, 0.5, 1.0, 1.0, 1.0, 0.0)
tables = np.zeros((8, 8, 8))
for (a, b, c), f in gamma_functions.items():
    tables[a, b, c] = f(*POINT)
fig, axes = plt.subplots(2, 4, figsize=(11.0, 5.4))
norm = SymLogNorm(linthresh=0.1, vmin=-15.0, vmax=15.0, base=10)
for a, ax in enumerate(axes.flat):
    image = ax.imshow(tables[a], cmap=DIVERGING, norm=norm)
    for b, c in itertools.product(range(8), repeat=2):
        if tables[a, b, c] != 0:
            ax.text(c, b, f"{tables[a, b, c]:.2f}", ha="center", va="center",
                    fontsize=6.0)
    ax.set_title(f"$\\Gamma^{{x_{a}}}{{}}_{{bc}}$", fontsize=10)
    ax.set_xticks(range(8), [str(k) for k in range(8)], fontsize=7)
    ax.set_yticks(range(8), [str(k) for k in range(8)], fontsize=7)
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.8, label="value (unit $H$)")
fig.suptitle("Christoffel symbols at $z = \\pi/4$, $a_4 = 0.5$, "
             "$a_4^{\\prime} = 1$, $b_4 = 1$, $H = 1$ (row $b$, column $c$)")
save_figure(fig, "christoffel_heat_maps",
            "The 512 Christoffel symbols $\\Gamma^{a}{}_{bc}$ of the author's metric "
            "at the point $z = \\pi/4$ on the slice $a_4 = 0.5$ of the history "
            "$a_4 = x_4$, $b_4 = 1$ ($a_4^{\\prime} = 1$, $H = 1$), as eight heat "
            "maps, one "
            "for each upper index $a = x_0, \\dots, x_7$; in each map the row is $b$ "
            "and the column is $c$ (numbers $0$ to $7$ for $x_0$ to $x_7$), the "
            "colour is the value on a symmetric logarithmic scale (grey zero, red "
            "positive, blue negative) and every non-zero value is printed. Only 37 "
            "of the 512 squares are coloured, each map is symmetric about its "
            "diagonal, and every non-zero symbol has at least one index equal to "
            "$x_4$ or $x_0$, the two coordinates on which the metric depends.")
check(np.allclose(tables, tables.transpose(0, 2, 1))
      and all(4 in key or 0 in key for key in nonzero),
      "the tables are symmetric in b, c; every non-zero symbol has an index x4 or x0")
```

Out [8]:

```
Figure 03b.1 saved as Revision/textbook/figures/03b_1_christoffel_heat_maps.png
PASS the tables are symmetric in b, c; every non-zero symbol has an index x4 or x0
```

Text cell:

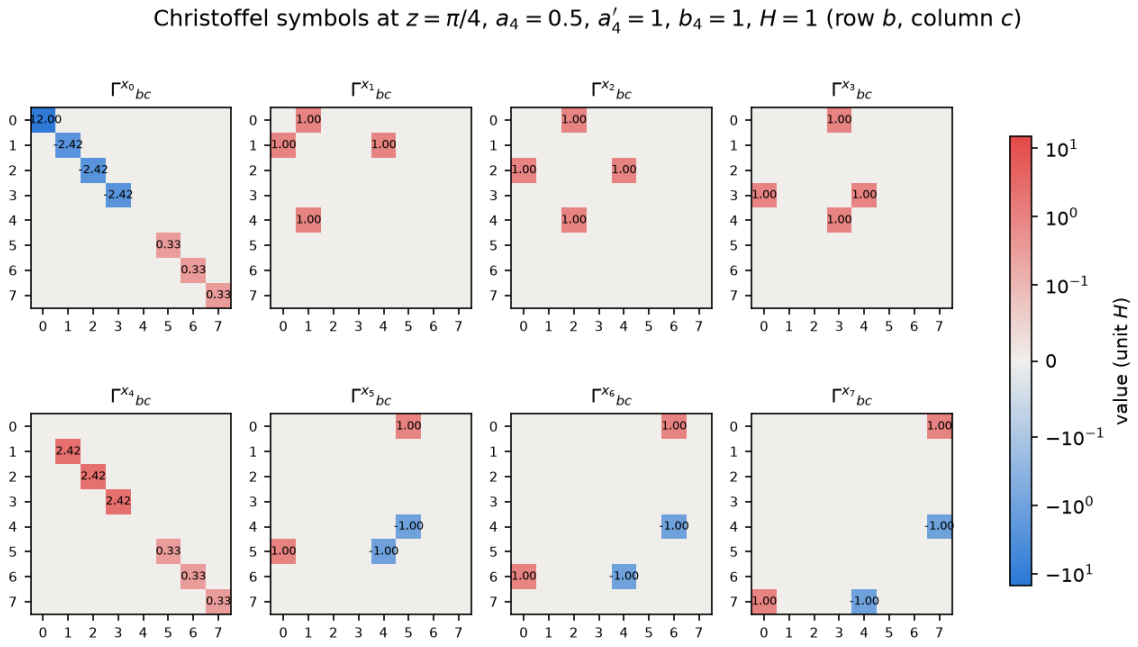

Figure 104: The 512 Christoffel symbols $\Gamma^{a}{}_{bc}$ of the author's metric at the point $z = \pi/4$ on the slice $a_4 = 0.5$ of the history $a_4 = x_4$, $b_4 = 1$ ($a_4' = 1$, $H = 1$), as eight heat maps, one for each upper index $a = x_0, \ldots, x_7$; in each map the row is b and the column is c (numbers 0 to 7 for x_0 to x_7), the colour is the value on a symmetric logarithmic scale (grey zero, red positive, blue negative) and every non-zero value is printed. Only 37 of the 512 squares are coloured, each map is symmetric about its diagonal, and every non-zero symbol has at least one index equal to x_4 or x_0, the two coordinates on which the metric depends. (Notebook 03b, figure 1.)

```
The symbols with an index $x_0$ depend on $z$. The next cell draws the size of five
typical symbols across the patch on a logarithmic axis (for $a_4 = 0.5$,
$a_4' = 1$, $b_4 = 1$, $H = 1$): $\Gamma^{x_1}{}_{x_1 x_0} = H\cot z$ and
$\Gamma^{x_0}{}_{x_0 x_0}$ grow without bound at the tip $z \to 0$;
$\Gamma^{x_0}{}_{x_1 x_1}$, $\Gamma^{x_0}{}_{x_0 x_0}$ and
$\Gamma^{x_0}{}_{x_5 x_5}$ grow without bound at the patch end $z \to \pi/2$;
$\Gamma^{x_1}{}_{x_1 x_4} = a_4'$ does not depend on $z$. Large Christoffel symbols
are a property of the coordinates; whether the geometry itself is extreme is decided
by invariants such as the Kretschmann scalar of section 12.
```

In [9]:

```
z_line = np.linspace(0.01, np.pi / 2 - 0.01, 500)
chosen = [((1, 1, 0), RED, "-", "$\\Gamma^{x_1}{}_{x_1 x_0}$"),
          ((0, 0, 0), VIOLET, "--", "$\\Gamma^{x_0}{}_{x_0 x_0}$"),
          ((0, 1, 1), ORANGE, "-.", "$\\Gamma^{x_0}{}_{x_1 x_1}$"),
          ((0, 5, 5), BLUE, ":", "$\\Gamma^{x_0}{}_{x_5 x_5}$"),
          ((1, 1, 4), AQUA, "-", "$\\Gamma^{x_1}{}_{x_1 x_4}$")]
fig, ax = plt.subplots()
for index, colour, style, label in chosen:
    values = np.broadcast_to(gamma_functions[index](z_line, 0.5, 1.0, 1.0, 1.0,
                                                    0.0), z_line.shape)
    sign = "+" if values[len(values) // 2] > 0 else "minus"
    ax.plot(z_line, np.abs(values), color=colour, ls=style, lw=1.8,
            label=f"{label} (sign {sign})")
ax.set_yscale("log")
ax.set_xlabel("$z = 6 H x_0$ (radians)")
ax.set_ylabel("absolute value of the symbol (unit $H$)")
ax.set_title("Christoffel symbols across the patch "
             "($a_4 = 0.5$, $a_4^{\\prime} = 1$, $b_4 = 1$, $H = 1$)")
ax.set_ylim(1e-5, 3e3)  # room below the curves for the legend
ax.legend(fontsize=8, loc="lower center", ncol=2)
save_figure(fig, "christoffel_versus_z",
            "The absolute values of five Christoffel symbols of the author's metric "
            "across the patch $0 < z < \\pi/2$, for $a_4 = 0.5$, $a_4^{\\prime} = 1$, "
            "$b_4 = 1$, $H = 1$, on a logarithmic vertical axis; horizontal axis "
            "$z = 6Hx_0$ in radians, vertical axis the absolute value in units of "
            "$H$; the legend gives each sign. $\\Gamma^{x_1}{}_{x_1x_0} = H\\cot z$ "
            "falls from the tip to the patch end, $\\Gamma^{x_0}{}_{x_0x_0}$ is "
            "large at both ends, $\\Gamma^{x_0}{}_{x_1x_1}$ and "
            "$\\Gamma^{x_0}{}_{x_5x_5}$ grow towards the patch end, and "
            "$\\Gamma^{x_1}{}_{x_1x_4} = a_4^{\\prime} = 1$ is the same everywhere.")
check(abs(gamma_functions[(1, 1, 0)](np.pi / 4, 0.5, 1.0, 1.0, 1.0, 0.0) - 1.0)
      < 1e-14,
      "Gamma^x1_(x1 x0) = H cot z equals 1 at z = pi/4, H = 1")
```

Out [9]:

```
Figure 03b.2 saved as Revision/textbook/figures/03b_2_christoffel_versus_z.png
PASS Gamma^x1_(x1 x0) = H cot z equals 1 at z = pi/4, H = 1
```

Text cell:

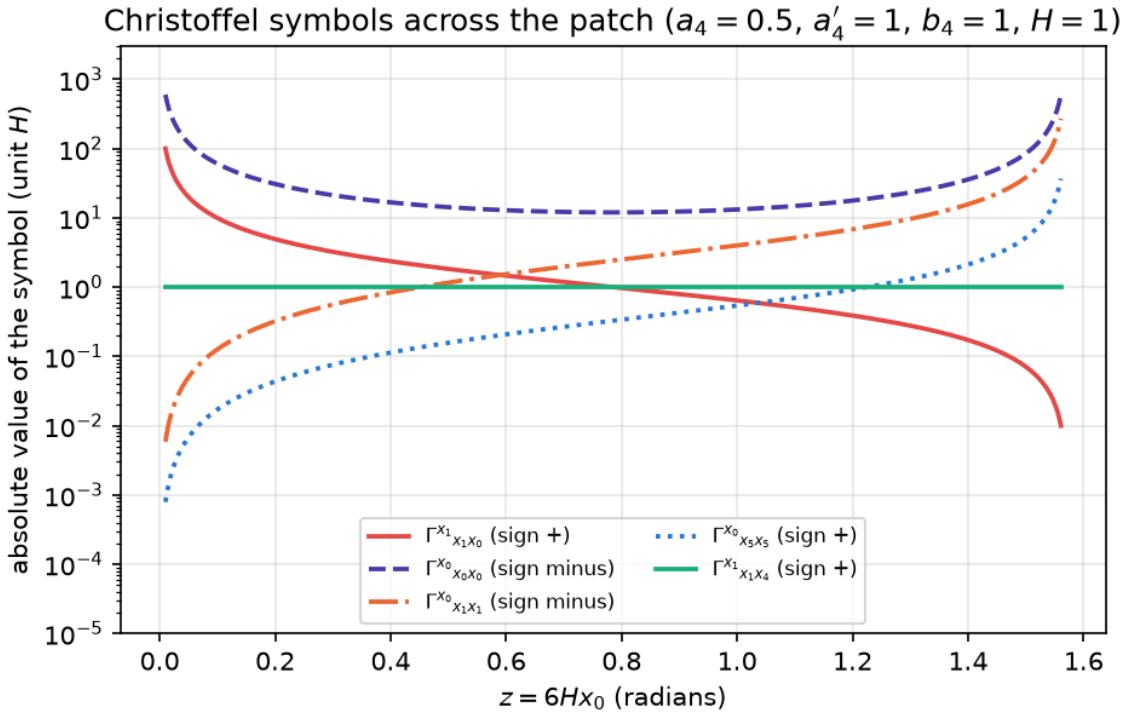

Figure 105: The absolute values of five Christoffel symbols of the author's metric across the patch $0 < z < \pi/2$, for $a_4 = 0.5$, $a_4' = 1$, $b_4 = 1$, $H = 1$, on a logarithmic vertical axis; horizontal axis $z = 6Hx_0$ in radians, vertical axis the absolute value in units of H; the legend gives each sign. $\Gamma^{x_1}{}_{x_1x_0} = H\cot z$ falls from the tip to the patch end, $\Gamma^{x_0}{}_{x_0x_0}$ is large at both ends, $\Gamma^{x_0}{}_{x_1x_1}$ and $\Gamma^{x_0}{}_{x_5x_5}$ grow towards the patch end, and $\Gamma^{x_1}{}_{x_1x_4} = a_4' = 1$ is the same everywhere. (Notebook 03b, figure 2.)

```
## 7. A second, independent check: finite differences

The exact symbols can be checked without any algebra. Take concrete functions
$a_4(x_4) = 0.17 + 0.61\,x_4 - 0.185\,x_4^2$ and
$b_4(x_4) = 1.37 + 0.31\,x_4 - 0.115\,x_4^2$ (so that at $x_4 = 0$:
$a_4 = 0.17$, $a_4' = 0.61$, $a_4'' = -0.37$, $b_4 = 1.37$, $b_4' = 0.31$,
$b_4'' = -0.23$), $H = 0.23$ and the point $x_0 = 0.41$: the test point at which the
Rust program checked its Lovelock tensors by brute force (record
‘lovelock-report.json‘, check ‘k1_brute_force_numeric‘).
The next cell first confirms these numbers in the record. Then it computes the
metric as a plain table of numbers at points $x \pm h\,e_c$ (one coordinate moved by
$\pm h$), the derivative of the metric by the central difference
$\partial_c g \approx (g(x + h e_c) - g(x - h e_c))/(2h)$, and from it all 512
Christoffel symbols with the same formula; it compares them with the exact symbols
for eight step sizes $h$ from $10^{-1}$ to $10^{-8}$.
```

In [10]:

```
prime = chr(39)  # the apostrophe (character number 39): the record writes a4 prime so
point_text = (f"H = 0.23, a4 = 0.17, a4{prime} = 0.61, a4{prime}{prime} = -0.37, "
              f"b4 = 1.37, b4{prime} = 0.31, b4{prime}{prime} = -0.23, "
              "x0 = 0.41")  # the test point as the record writes it
check(record_check(RUST_REPORT, "k1_brute_force_numeric", point_text),
      "the test point is the one of the brute-force check of the Rust program")
H_n, a0, a1, a2, x0_n = 0.23, 0.17, 0.61, -0.37, 0.41  # the test point
b0, b1, b2 = 1.37, 0.31, -0.23  # b4, b4' and b4'' at x4 = 0


def metric_numbers(x):
    """The metric as an 8 x 8 numpy table at the point x (8 coordinates)."""
    a = a0 + a1 * x[4] + 0.5 * a2 * x[4] ** 2  # a4(x4) with a4(0) = 0.17, ...
    b = b0 + b1 * x[4] + 0.5 * b2 * x[4] ** 2  # b4(x4) with b4(0) = 1.37, ...
    warp_n = np.sin(6 * H_n * x[0]) ** (1 / 3)
    # the author's order x0, x1, ..., x7
    return np.diag([1 / np.tan(6 * H_n * x[0]) ** 2] + [np.exp(2 * a) * warp_n] * 3
                   + [-1.0] + [-b * np.exp(-2 * a) * warp_n] * 3)


def christoffel_numbers(x, h):
    """All 512 Christoffel symbols at x from central differences with step h."""
    d_metric = np.zeros((8, 8, 8))  # d_metric[c] = derivative of g along x_c
    for c in range(8):
        step = np.zeros(8)
        step[c] = h
        d_metric[c] = (metric_numbers(x + step) - metric_numbers(x - step)) / (2 * h)
    inverse = np.linalg.inv(metric_numbers(x))
    table = np.zeros((8, 8, 8))
    for a, b, c in itertools.product(range(8), repeat=3):
        table[a, b, c] = 0.5 * sum(inverse[a, d] * (d_metric[b][d, c]
                                   + d_metric[c][d, b] - d_metric[d][b, c])
                                   for d in range(8))
    return table
```

```
x_point = np.array([x0_n, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])  # x0 = 0.41, x4 = 0
exact = np.zeros((8, 8, 8))
for (a, b, c), f in gamma_functions.items():
    exact[a, b, c] = f(6 * H_n * x0_n, a0, a1, H_n, b0, b1)
steps = 10.0 ** -np.arange(1, 9)  # h = 0.1, 0.01, ..., 1e-8
errors = np.array([np.max(np.abs(christoffel_numbers(x_point, h) - exact))
                   for h in steps])
for h, e in zip(steps, errors):
    say(f"  h = {h:.0e}: largest error of the 512 symbols = {e:.2e}")
order = np.log10(errors[0] / errors[1])  # the slope between h = 0.1 and h = 0.01
report("measured order of the error between h = 0.1 and h = 0.01", f"{order:.3f}")
check(1.9 < order < 2.1 and errors.min() < 1e-8,
      "finite differences reproduce all 512 symbols; the error falls like h^2")
```

Out [10]:

```
PASS the test point is the one of the brute-force check of the Rust program
  h = 1e-01: largest error of the 512 symbols = 4.01e-01
  h = 1e-02: largest error of the 512 symbols = 3.66e-03
  h = 1e-03: largest error of the 512 symbols = 3.65e-05
  h = 1e-04: largest error of the 512 symbols = 3.65e-07
  h = 1e-05: largest error of the 512 symbols = 3.66e-09
  h = 1e-06: largest error of the 512 symbols = 5.67e-11
  h = 1e-07: largest error of the 512 symbols = 8.46e-10
  h = 1e-08: largest error of the 512 symbols = 9.00e-09
RESULT measured order of the error between h = 0.1 and h = 0.01 = 2.040
PASS finite differences reproduce all 512 symbols; the error falls like h^2
```

Text cell:

```
The next cell draws the largest error against the step $h$ on logarithmic axes.
For large $h$ the error of the central difference is about $C h^2$, a straight line
of slope 2 (the dashed reference line); for very small $h$ the rounding errors of
the computer (about $10^{-16}$ relative) are divided by $h$ and the error grows
again. The best step is where the two meet.
```

In [11]:

```
fig, ax = plt.subplots()
ax.loglog(steps, errors, "o-", color=BLUE, lw=1.8, ms=7,
          label="largest error of the 512 symbols")
ax.loglog(steps, errors[0] * (steps / steps[0]) ** 2, "--", color="black", lw=1.2,
          label="slope 2: error proportional to $h^2$")
ax.set_xlabel("step $h$ of the central difference")
ax.set_ylabel("largest absolute error (unit $H$)")
ax.set_title("Christoffel symbols by finite differences")
ax.legend(fontsize=8)
best = int(np.argmin(errors))  # the position of the smallest error in the list
report("best step and its error", f"h = {steps[best]:.0e}, error {errors[best]:.1e}")
mantissa, exponent = f"{errors[best]:.1e}".split("e")  # e.g. "3.2" and "-10"
```

```
best_error = f"{mantissa} \\times 10^{{{int(exponent)}}}"  # 3.2 x 10^-10 in LaTeX
best_step = f"10^{{{round(np.log10(steps[best]))}}}"  # the step as a power of 10
save_figure(fig, "finite_differences",
            "The largest difference between the 512 Christoffel symbols computed "
            "numerically by central differences of the metric and the exact "
            "symbols, at the test point $H = 0.23$, $a_4 = 0.17$, "
            "$a_4^{\\prime} = 0.61$, $a_4^{\\prime\\prime} = -0.37$, $b_4 = 1.37$, "
            "$b_4^{\\prime} = 0.31$, $b_4^{\\prime\\prime} = -0.23$, $x_0 = 0.41$ of "
            "the Revision record, against the step $h$, on logarithmic axes; "
            "horizontal axis $h$, vertical axis the error in units of $H$. For "
            "large steps the error follows the dashed line of slope 2 (error "
            "proportional to $h^2$); the smallest error, "
            f"${best_error}$, is reached at $h = {best_step}$; for "
            "smaller steps the rounding errors of the computer, which grow like "
            "$1/h$, take over.")
check(errors[0] > errors[1] > errors[2] > errors[3],
      "the error shrinks as the step shrinks from 0.1 to 0.0001")
```

Out [11]:

```
RESULT best step and its error = h = 1e-06, error 5.7e-11
```

Out [11]:

```
Figure 03b.3 saved as Revision/textbook/figures/03b_3_finite_differences.png
PASS the error shrinks as the step shrinks from 0.1 to 0.0001
```

Text cell:

```
## 8. Free fall along the time $x_4$

Consider an observer who stays at fixed $x_1, x_2, x_3, x_5, x_6, x_7, x_0$ while
$x_4 = \tau$ runs. Then $dx^a/d\tau$ is $1$ for $a = x_4$ and $0$ otherwise, and
$d^2x^a/d\tau^2 = 0$, so the geodesic equation reduces to
$\Gamma^a{}_{x_4 x_4} = 0$ for every $a$. Since $g_{44} = -1$, the proper time of
the observer is $\sqrt{-g_{44}}\, d\tau = d\tau$: $x_4$ is the time shown by the
observer's clock. The next cell checks that the eight symbols $\Gamma^a{}_{x_4x_4}$
are zero. It also checks the useful identity $\sum_a \Gamma^a{}_{ab} =
\partial_b \ln\sqrt{|\det g|}$, with $\sqrt{|\det g|} = b_4^{3/2} \cos z$.
```

In [12]:

```
check(all(Gamma[a][4][4] == 0 for a in range(8)),
      "Gamma^a_(x4 x4) = 0: observers at rest fall freely, x4 is their proper time")
volume = b4 ** sp.Rational(3, 2) * sp.cos(6 * H * x0)  # sqrt|det g|
contracted = [vanishes(sum(Gamma[a][a][b] for a in range(8))
                       - sp.diff(sp.log(volume), X[b]))
              for b in range(8)]
check(all(contracted),
      "sum over a of Gamma^a_(a b) equals the derivative of ln(b4^(3/2) cos z)")
```

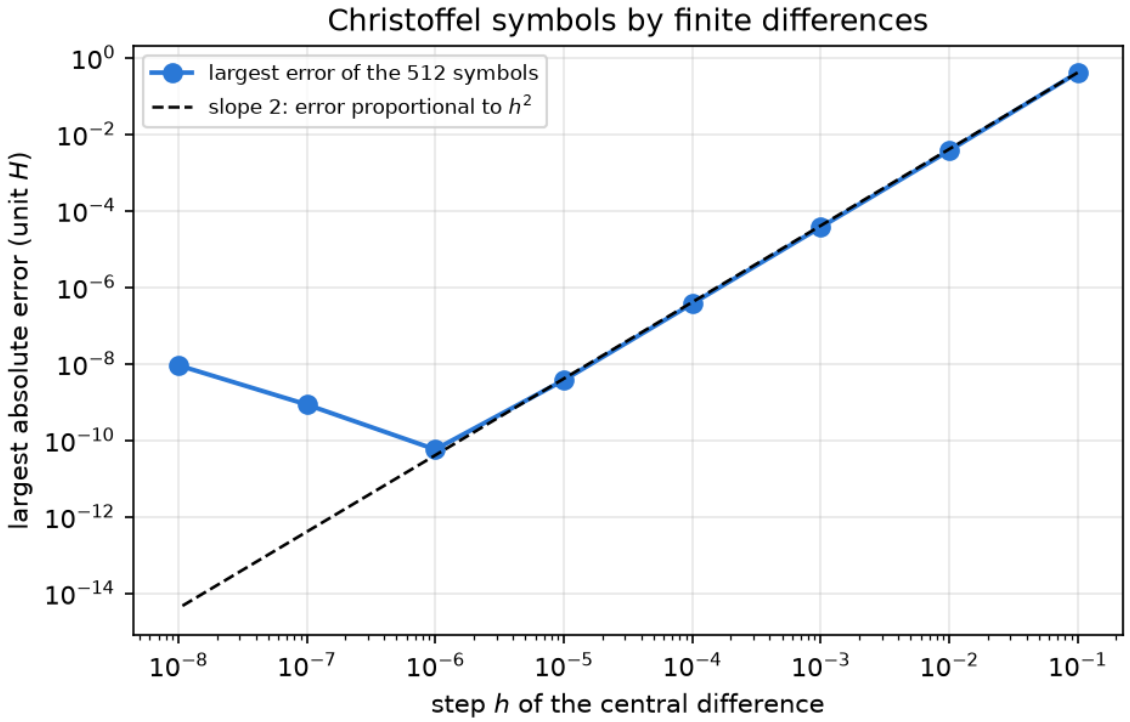

Figure 106: The largest difference between the 512 Christoffel symbols computed numerically by central differences of the metric and the exact symbols, at the test point $H = 0.23$, $a_4 = 0.17$, $a_4' = 0.61$, $a_4'' = -0.37$, $b_4 = 1.37$, $b_4' = 0.31$, $b_4'' = -0.23$, $x_0 = 0.41$ of the Revision record, against the step h, on logarithmic axes; horizontal axis h, vertical axis the error in units of H. For large steps the error follows the dashed line of slope 2 (error proportional to h^2); the smallest error, 5.7×10^{-11}, is reached at $h = 10^{-6}$; for smaller steps the rounding errors of the computer, which grow like $1/h$, take over. (Notebook 03b, figure 3.)

Out [12]:

```
PASS Gamma^a_(x4 x4) = 0: observers at rest fall freely, x4 is their proper time
PASS sum over a of Gamma^a_(a b) equals the derivative of ln(b4^(3/2) cos z)
```

Text cell:

```
## 9. The Riemann tensor

The Riemann tensor in the convention of all Revision records (MTW) is

$$R^a{}_{bcd} = \partial_c \Gamma^a{}_{bd} - \partial_d \Gamma^a{}_{bc}
+ \sum_e \bigl(\Gamma^a{}_{ce}\Gamma^e{}_{bd} - \Gamma^a{}_{de}\Gamma^e{}_{bc}\bigr).$$

It is antisymmetric in its last two indices $c, d$ (exchanging $c$ and $d$ turns the
formula into its negative: the first two terms exchange places and signs, and so do
the two products), so the next cell computes it for $c < d$ and fills $c > d$ with
the opposite sign; the components with $c = d$ are zero. It keeps only the
non-zero components in a dictionary. Then it raises the second index,
$R^{ab}{}_{cd} = \sum_e g^{be} R^a{}_{ecd}$, the form in which the record stores
the tensor. This cell takes a few seconds.
```

In [13]:

```
def riemann(table, coordinates):
    """The non-zero R^a_bcd (MTW) as a dictionary {(a, b, c, d): value}."""
    n = len(coordinates)
    result = {}
    for a, b in itertools.product(range(n), repeat=2):
        for c, d in itertools.combinations(range(n), 2):  # all pairs c < d
            value = (sp.diff(table[a][b][d], coordinates[c])
                     - sp.diff(table[a][b][c], coordinates[d])
                     + sum(table[a][c][e] * table[e][b][d]
                           - table[a][d][e] * table[e][b][c] for e in range(n)))
            value = tidy(value)
            if value != 0:
                result[(a, b, c, d)] = value
                result[(a, b, d, c)] = -value  # antisymmetric in c and d
    return result


def raise_second(riemann_table, metric):
    """R^ab_cd = sum over e of g^(be) R^a_ecd, the non-zero ones."""
    inverse = metric.inv()
    result = {}
    for (a, e, c, d), value in riemann_table.items():
        for b in range(metric.rows):
            if inverse[b, e] != 0:
                result[(a, b, c, d)] = result.get((a, b, c, d), 0) \
                    + inverse[b, e] * value
    result = {k: tidy(v) for k, v in result.items()}
```

```
    return {k: v for k, v in result.items() if v != 0}


R_down = riemann(Gamma, X)  # R^a_bcd
R_mixed = raise_second(R_down, g)  # R^ab_cd
report("non-zero components R^a_bcd", len(R_down))
report("non-zero components R^ab_cd", len(R_mixed))
record_check(RUST_REPORT, "riemann_antisymmetry", "156 nonzero entries")
check(len(R_mixed) == 156, "R^ab_cd has 156 non-zero components, as in the record",
      record="Revision/gkd_lovelock/results/lovelock-report.json, check "
             "riemann_antisymmetry (156 nonzero entries)")
```

Out [13]:

```
RESULT non-zero components R^a_bcd = 156
RESULT non-zero components R^ab_cd = 156
PASS R^ab_cd has 156 non-zero components, as in the record
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    riemann_antisymmetry (156 nonzero entries)
```

Text cell:

```
The next cell checks the symmetries that every Riemann tensor has, as a test of the
computation: (1) $R^{ab}{}_{cd} = -R^{ba}{}_{cd} = -R^{ab}{}_{dc}$; (2) with all
indices lowered, $R_{abcd} = g_{aa} g_{bb} R^{ab}{}_{cd}$ (diagonal metric), the
pair symmetry $R_{abcd} = R_{cdab}$; (3) the first Bianchi identity
$R^a{}_{bcd} + R^a{}_{cdb} + R^a{}_{dbc} = 0$ for all $8^4 = 4096$ index lists.
It also checks that no component $R^{ab}{}_{cd}$ contains the factor
$\sin^{1/3} z$ of the transverse metric entries (the square $W^2$ of the warp factor
$W = \sin^{1/6} z$ of Notebook 03a) or the exponential $e^{a_4}$: they cancel when
one index is raised.
```

In [14]:

```
def get(table, key):
    return table.get(key, 0)


antisymmetric = all(vanishes(v + get(R_mixed, (b, a, c, d)))
                    and vanishes(v + get(R_mixed, (a, b, d, c)))
                    for (a, b, c, d), v in R_mixed.items())
lowered = {(a, b, c, d): g[a, a] * g[b, b] * v
           for (a, b, c, d), v in R_mixed.items()}
pair_symmetric = all(vanishes(v - get(lowered, (c, d, a, b)))
                     for (a, b, c, d), v in lowered.items())
record_check(PYTHON_REPORT, "riemann_antisymmetry_and_pair_symmetry")
check(antisymmetric and pair_symmetric,
      "R^ab_cd is antisymmetric in a, b and in c, d; R_abcd = R_cdab",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "riemann_antisymmetry_and_pair_symmetry")
bianchi = [vanishes(get(R_down, (a, b, c, d)) + get(R_down, (a, c, d, b))
```

```
                    + get(R_down, (a, d, b, c)))
           for a, b, c, d in itertools.product(range(8), repeat=4)]
record_check(RUST_REPORT, "riemann_first_bianchi")
check(all(bianchi) and len(bianchi) == 4096,
      "the first Bianchi identity holds for all 4096 index lists",
      record="Revision/gkd_lovelock/results/lovelock-report.json, check "
             "riemann_first_bianchi")
warp_free = all(not symbolic(v).has(a4v) and not any(
    isinstance(p, sp.Pow) and p.base == sp.sin(6 * H * x0) and not p.exp.is_integer
    for p in sp.preorder_traversal(symbolic(v))) for v in R_mixed.values())
record_check(RUST_REPORT, "mixed_riemann_free_of_sin_third")
record_check(PYTHON_REPORT, "mixed_riemann_free_of_warp_and_exponential")
check(warp_free, "no R^ab_cd contains sin(z)^(1/3) or e^(a4): the warp cancels",
      record="Revision/gkd_lovelock/results/lovelock-report.json, check "
             "mixed_riemann_free_of_sin_third, and python-lovelock-report.json, "
             "check mixed_riemann_free_of_warp_and_exponential")
```

Out [14]:

```
PASS R^ab_cd is antisymmetric in a, b and in c, d; R_abcd = R_cdab
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    riemann_antisymmetry_and_pair_symmetry
PASS the first Bianchi identity holds for all 4096 index lists
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    riemann_first_bianchi
PASS no R^ab_cd contains sin(z)^(1/3) or e^(a4): the warp cancels
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    mixed_riemann_free_of_sin_third, and python-lovelock-report.json, check
    mixed_riemann_free_of_warp_and_exponential
```

Text cell:

```
The next cell compares all 156 components with the list ‘riemannMixedNonzero‘ of the
record, in which ‘up‘ holds $a, b$ and ‘down‘ holds $c, d$ of $R^{ab}{}_{cd}$: first
that the same 156 index lists occur, then that each value agrees exactly. It then
groups the 156 components by their value and prints the different values (20 for
a general $b_4$), how often each occurs and one example of where. Each value is
first multiplied out (‘sp.expand‘), so that one value that sympy happens to write
in two ways is counted once.
```

In [15]:

```
record_riemann = {}
for entry in record["riemannMixedNonzero"]:
    key = tuple(NAMES.index(n) for n in entry["up"] + entry["down"])
    record_riemann[key] = from_mathematica(entry["value"])
check(sorted(record_riemann) == sorted(R_mixed),
      "the same 156 non-zero components R^ab_cd as the record")
record_check(PYTHON_REPORT, "rust_riemann_agrees")
check(all(same(R_mixed[k], v) for k, v in record_riemann.items()),
      "every component R^ab_cd equals the record exactly",
```

```
      record=f"{CURVATURE_RECORD}, riemannMixedNonzero (and "
             "python-lovelock-report.json, check rust_riemann_agrees)")
groups = {}
for key in sorted(R_mixed):
    # the value multiplied out: one value written in two ways counts once
    groups.setdefault(sp.sstr(sp.expand(plain(R_mixed[key]))), []).append(key)
for value_text in sorted(groups):
    a, b, c, d = groups[value_text][0]
    say(f"  {len(groups[value_text]):2d} x  {value_text:30s}  e.g. "
        f"R^({NAMES[a]} {NAMES[b]})_({NAMES[c]} {NAMES[d]})")
report("different values among the 156 components", len(groups))
```

Out [15]:

```
PASS the same 156 non-zero components R^ab_cd as the record
PASS every component R^ab_cd equals the record exactly
     reproduces Revision/gkd_lovelock/results/curvature.json, riemannMixedNonzero (and
    python-lovelock-report.json, check rust_riemann_agrees)
  12 x  -H**2                           e.g. R^(x0 x1)_(x0 x1)
   6 x  -H**2 + a4p**2                  e.g. R^(x1 x2)_(x1 x2)
   6 x  -H**2 + a4p**2 - a4p*b4p/b4v + b4p**2/(4*b4v**2)  e.g. R^(x5 x6)_(x5 x6)
  18 x  -H**2 - a4p**2 + a4p*b4p/(2*b4v)  e.g. R^(x1 x5)_(x1 x5)
   6 x  -H*a4p*tan(z)                   e.g. R^(x0 x1)_(x4 x1)
   6 x  -H*a4p*tan(z) + H*b4p*tan(z)/(2*b4v)  e.g. R^(x0 x5)_(x5 x4)
   6 x  -H*a4p/tan(z)                   e.g. R^(x1 x4)_(x0 x1)
   6 x  -H*a4p/tan(z) + H*b4p/(2*b4v*tan(z))  e.g. R^(x4 x5)_(x0 x5)
   6 x  -a4p**2 + a4p*b4p/b4v + a4pp + b4p**2/(4*b4v**2) - b4pp/(2*b4v)  e.g. R^(x4
    x5)_(x5 x4)
   6 x  -a4p**2 - a4pp                  e.g. R^(x1 x4)_(x4 x1)
  12 x  H**2                            e.g. R^(x0 x1)_(x1 x0)
  18 x  H**2 + a4p**2 - a4p*b4p/(2*b4v)  e.g. R^(x1 x5)_(x5 x1)
   6 x  H**2 - a4p**2                   e.g. R^(x1 x2)_(x2 x1)
   6 x  H**2 - a4p**2 + a4p*b4p/b4v - b4p**2/(4*b4v**2)  e.g. R^(x5 x6)_(x6 x5)
   6 x  H*a4p*tan(z)                    e.g. R^(x0 x1)_(x1 x4)
   6 x  H*a4p*tan(z) - H*b4p*tan(z)/(2*b4v)  e.g. R^(x0 x5)_(x4 x5)
   6 x  H*a4p/tan(z)                    e.g. R^(x1 x4)_(x1 x0)
   6 x  H*a4p/tan(z) - H*b4p/(2*b4v*tan(z))  e.g. R^(x4 x5)_(x5 x0)
   6 x  a4p**2 + a4pp                   e.g. R^(x1 x4)_(x1 x4)
   6 x  a4p**2 - a4p*b4p/b4v - a4pp - b4p**2/(4*b4v**2) + b4pp/(2*b4v)  e.g. R^(x4
    x5)_(x4 x5)
RESULT different values among the 156 components = 20
```

Text cell:

```
## 10. The curvature of every coordinate plane

For two different coordinates $x_a$ and $x_b$ the number $\sigma(a, b) =
R^{ab}{}_{ab}$ (no sum) is the curvature of the coordinate plane spanned by them.
The next cell computes all 28 of them as formulas, prints one plane of each kind,
and checks seven formulas. Write $\alpha = a_4'$ for the expansion rate of 3-space
and $\alpha_t = -a_4' + b_4'/(2b_4)$ for that of the extra times (Notebook 03a). Then:
3-space with 3-space $\alpha^2 - H^2$; extra time with extra time
```

$\alpha_t^2 - H^2$; 3-space with an extra time $\alpha\alpha_t - H^2$; 3-space with the time x_4 $\alpha^2 + \alpha'$; the time with an extra time $\alpha_t^2 + \alpha_t'$; a 3-space or extra-time direction with the hidden x_0 $-H^2$; the time with the hidden x_0 zero. For a constant b_4, $\alpha_t = -\alpha$ and these are the formulas $a_4'^2 - H^2$, $-(a_4'^2 + H^2)$, $a_4'^2 \pm a_4''$ of the metric with $b_4 = 1$. None depends on z or on the value of a_4.

What does the sign mean? Only 6 of the 28 planes are spanned by two space-like directions (two of x_1, x_2, x_3, or one of them with x_0); only for these does a positive value mean curved like a sphere and a negative one like a saddle. The other 22 planes contain a time-like direction, and there the sign is read from free fall. The clearest case: two observers at rest (they fall freely, section 8), a small coordinate distance Δx_a apart along a transverse direction x_a, are a proper distance $\xi = h_a\,\Delta x_a$ apart, where h_a is the scale factor; the proper time of both is $\tau = x_4$. The cell computes $(d^2\xi/d\tau^2)/\xi = (\partial_4^2 h_a)/h_a$ for $x_a = x_1$ (with $h_1 = e^{a_4}\sin^{1/6}z$) and $x_a = x_5$ (with $h_5 = b_4^{1/2} e^{-a_4}\sin^{1/6}z$) and checks that it is exactly $\sigma(a, x_4)$: $d^2\xi/d\tau^2 = +\sigma(a, x_4)\,\xi$. Along the history $a_4 = AHx_4$, $b_4 = 1$ both are $A^2H^2 > 0$: a positive curvature, and yet the 3-space distance grows like $e^{AH\tau}$ (the extra-time distance shrinks like $e^{-AH\tau}$, also with a positive second derivative); on a sphere a positive curvature pulls neighbouring paths together, $d^2\xi/ds^2 = -\sigma\,\xi$. For the 6 planes of two time-like directions (two extra times, or an extra time and x_4) even the shape reading is reversed: if g is replaced by $-g$, the Christoffel symbols and $R^a{}_{bcd}$ do not change but every g^{bb} changes sign, so $\sigma(a, b)$ changes sign; with $-g$ such a plane has positive squared lengths, and its shape is that of the curvature $-\sigma(a, b)$.

Then the cell draws the 28 plane curvatures as 8×8 heat maps along the deflating history $a_4 = AHx_4$, $b_4 = 1$ ($a_4' = AH$, $a_4'' = 0$, $H = 1$) for the three
slopes $A = 0.5$, 1 (the canonical value of the Revision record) and 2, all positive, so that in all three the extra times deflate; the diagonal is left empty. The planes inside 3-space and inside the extra times change sign at $A = 1$: $a_4'^2 - H^2 = H^2(A^2 - 1)$ is negative for $A < 1$, zero at $A = 1$ and positive for $A > 1$.

In [16]:

```
plane = {(a, b): plain(sp.S(get(R_mixed, (a, b, a, b))))  # sp.S: 0 as a sympy 0
         for a, b in itertools.product(range(8), repeat=2) if a != b}
alpha_t = -a4p + b4p / (2 * b4v)  # the expansion rate of the extra times
alpha_t_prime = -a4pp + b4pp / (2 * b4v) - b4p ** 2 / (2 * b4v ** 2)  # its derivative
# the planes (row, column) in the author's order x0, x1, ..., x7
expected = {(1, 2): a4p ** 2 - H ** 2, (5, 6): alpha_t ** 2 - H ** 2,
            (1, 5): a4p * alpha_t - H ** 2, (1, 4): a4p ** 2 + a4pp,
            (4, 5): alpha_t ** 2 + alpha_t_prime, (1, 0): -H ** 2, (5, 0): -H ** 2,
            (4, 0): 0}
for (a, b), formula in expected.items():
    say(f"  plane ({NAMES[a]}, {NAMES[b]}): R^ab_ab = {plane[(a, b)]}")
check(all(sp.expand(plane[key] - formula) == 0 for key, formula in expected.items())
      and all(not v.has(z) and not v.has(a4v) for v in plane.values()),
      "the plane curvatures have the seven formulas and do not depend on z or a4")
```

```
sixth = sp.sin(6 * H * x0) ** sp.Rational(1, 6)  # sin(z)^(1/6)
apart = {1: sp.exp(a4) * sixth, 5: sp.sqrt(b4) * sp.exp(-a4) * sixth}  # h1, h5
growth = {}  # (d^2 xi/d tau^2)/xi for observers at rest apart along x1 or x5
for a, h_a in apart.items():  # xi = h_a times a fixed coordinate distance, tau = x4
    growth[a] = sp.expand(plain(sp.simplify(sp.diff(h_a, x4, 2) / h_a)))
    say(f"  at rest, apart along {NAMES[a]}: (d^2 xi/d tau^2)/xi = {growth[a]}")
check(sp.simplify(apart[1] ** 2 - g[1, 1]) == 0
      and sp.simplify(apart[5] ** 2 + g[5, 5]) == 0
      and all(sp.simplify(growth[a] - plane[(a, 4)]) == 0 for a in apart),
      "observers at rest: d^2 xi/d tau^2 = +R^ab_ab xi in the planes (x1, x4) and "
      "(x5, x4)")
SLOPES = (0.5, 1.0, 2.0)  # three deflating histories, A > 0
fig, axes = plt.subplots(1, 3, figsize=(14.0, 4.8))  # wide: room for -1.25
for ax, slope in zip(axes, SLOPES):
    curv = np.full((8, 8), np.nan)  # NaN: an empty square on the diagonal
    for (a, b), value in plane.items():
        curv[a, b] = float(value.subs({a4p: slope, a4pp: 0, H: 1, b4v: 1, b4p: 0,
                                       b4pp: 0}))
    image = ax.imshow(curv, cmap=DIVERGING, vmin=-5.0, vmax=5.0)
    for a, b in plane:
        ax.text(b, a, f"{curv[a, b]:g}", ha="center", va="center", fontsize=6.5)
    ax.set_xticks(range(8), LABELS, fontsize=8)
    ax.set_yticks(range(8), LABELS, fontsize=8)
    ax.grid(False)
    ax.set_title(f"$A = {slope:g}$")
fig.colorbar(image, ax=axes, shrink=0.85, label="curvature (unit $H^2$)")
save_figure(fig, "plane_curvatures",
            "The curvature $\\sigma(a, b) = R^{ab}{}_{ab}$ (no sum) of the plane of "
            "$x_a$ (row) and $x_b$ (column), unit $H^2$, along the history "
            "$a_4 = AHx_4$, $b_4 = 1$ ($a_4^{\\prime\\prime} = 0$, $H = 1$) for "
            "$A = 0.5$, "
            "$1$ (canonical) and $2$ (left to right); red positive, blue negative, "
            "grey zero. Only for two space-like directions (two of $x_1, x_2, x_3$, "
            "or one of them with $x_0$) does red mean curved like a sphere and blue "
            "like a saddle; with the time $x_4$ a positive value means that "
            "observers at rest accelerate apart, $d^2\\xi/d\\tau^2 = +\\sigma\\xi$. "
            "Inside 3-space and inside the extra times $a_4^{\\prime 2} - H^2$ "
            "($-0.75$, $0$, $3$); 3-space with an extra time "
            "$-(a_4^{\\prime 2} + H^2)$ ($-1.25$, $-2$, $-5$); with $x_4$ "
            "$a_4^{\\prime 2} \\pm a_4^{\\prime\\prime}$ ($0.25$, $1$, $4$), but the "
            "plane of $x_4$ and $x_0$ is flat; the other planes with $x_0$ have "
            "$-H^2 = -1$.")
check([float(plane[(1, 2)].subs({a4p: s, H: 1})) for s in SLOPES] == [-0.75, 0.0, 3.0],
      "the planes inside 3-space: -0.75, 0 and 3 for A = 0.5, 1 and 2")
```

Out [16]:

```
  plane (x1, x2): R^ab_ab = -H**2 + a4p**2
  plane (x5, x6): R^ab_ab = -H**2 + a4p**2 - a4p*b4p/b4v + b4p**2/(4*b4v**2)
  plane (x1, x5): R^ab_ab = -H**2 - a4p**2 + a4p*b4p/(2*b4v)
  plane (x1, x4): R^ab_ab = a4p**2 + a4pp
  plane (x4, x5): R^ab_ab = a4p**2 - a4p*b4p/b4v - a4pp - b4p**2/(4*b4v**2) +
    b4pp/(2*b4v)
```

```
  plane (x1, x0): R^ab_ab = -H**2
  plane (x5, x0): R^ab_ab = -H**2
  plane (x4, x0): R^ab_ab = 0
PASS the plane curvatures have the seven formulas and do not depend on z or a4
  at rest, apart along x1: (d^2 xi/d tau^2)/xi = a4p**2 + a4pp
  at rest, apart along x5: (d^2 xi/d tau^2)/xi = a4p**2 - a4p*b4p/b4v - a4pp -
    b4p**2/(4*b4v**2) + b4pp/(2*b4v)
PASS observers at rest: d^2 xi/d tau^2 = +R^ab_ab xi in the planes (x1, x4) and (x5, x4)
```

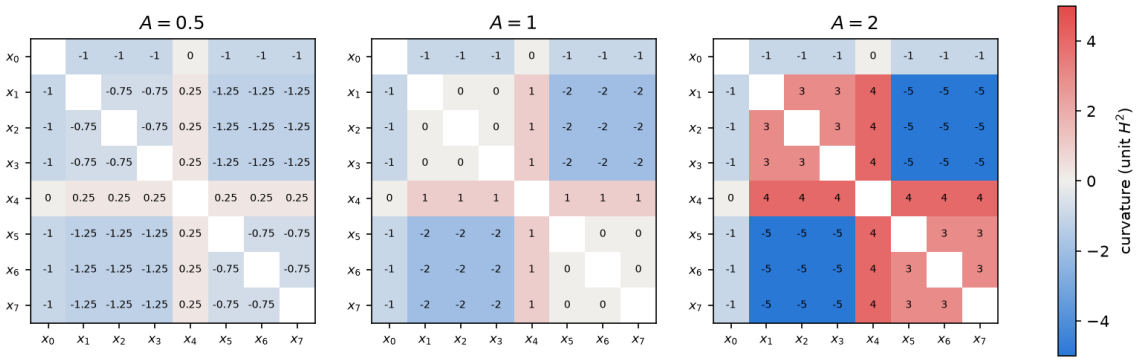

Figure 107: The curvature $\sigma(a,b) = R^{ab}{}_{ab}$ (no sum) of the plane of x_a (row) and x_b (column), unit H^2, along the history $a_4 = AHx_4$, $b_4 = 1$ ($a_4'' = 0$, $H = 1$) for $A = 0.5$, 1 (canonical) and 2 (left to right); red positive, blue negative, grey zero. Only for two space-like directions (two of x_1, x_2, x_3, or one of them with x_0) does red mean curved like a sphere and blue like a saddle; with the time x_4 a positive value means that observers at rest accelerate apart, $d^2\xi/d\tau^2 = +\sigma\xi$. Inside 3-space and inside the extra times $a_4'^2 - H^2$ (-0.75, 0, 3); 3-space with an extra time $-(a_4'^2 + H^2)$ (-1.25, -2, -5); with x_4 $a_4'^2 \pm a_4''$ (0.25, 1, 4), but the plane of x_4 and x_0 is flat; the other planes with x_0 have $-H^2 = -1$. (Notebook 03b, figure 4.)

Out [16]:

```
Figure 03b.4 saved as Revision/textbook/figures/03b_4_plane_curvatures.png
PASS the planes inside 3-space: -0.75, 0 and 3 for A = 0.5, 1 and 2
```

Text cell:

```
## 11. Ricci tensor, Ricci scalar, Einstein tensor

The next cell contracts the Riemann tensor: $R^a{}_b = \sum_c R^{ac}{}_{bc}$, then
$R = \sum_a R^a{}_a$ and $G^a{}_b = R^a{}_b - \tfrac12 \delta^a{}_b R$. It prints the
diagonal and compares all 64 components of each tensor and the scalar with the
record (‘ricciMixed‘, ‘ricciScalar‘, ‘einsteinMixed‘), after confirming with
‘record_check‘ that the independent verification of the record passed its check
‘rust_ricci_einstein_scalar_agree‘.
```

In [17]:

```
def ricci(mixed_table, n):
    """R^a_b = sum over c of R^ac_bc, as an n x n sympy matrix."""
    result = sp.zeros(n, n)
    for (a, c, b, d), value in mixed_table.items():
        if c == d:
            result[a, b] += value
    return result.applyfunc(tidy)


Ric = ricci(R_mixed, 8)
R_scalar = tidy(Ric.trace())
G = (Ric - R_scalar / 2 * sp.eye(8)).applyfunc(tidy)
for k in range(8):
    say(f"  {NAMES[k]}: R^a_a = {sp.sstr(plain(Ric[k, k])):18s} "
        f"G^a_a = {plain(G[k, k])}")
say(f"Ricci scalar R = {plain(R_scalar)}")
ricci_ok = all(same(Ric[i, j], from_mathematica(
    record["ricciMixed"][f"{NAMES[i]},{NAMES[j]}"]["mathematica"]))
    for i in range(8) for j in range(8))
einstein_ok = all(same(G[i, j], from_mathematica(
    record["einsteinMixed"][f"{NAMES[i]},{NAMES[j]}"]["mathematica"]))
    for i in range(8) for j in range(8))
scalar_ok = same(R_scalar, from_mathematica(record["ricciScalar"]))
record_check(PYTHON_REPORT, "rust_ricci_einstein_scalar_agree")  # stops if not
check(ricci_ok and einstein_ok and scalar_ok,
      "all 64 Ricci, all 64 Einstein components and R equal the record",
      record=f"{CURVATURE_RECORD}, ricciMixed, einsteinMixed, ricciScalar (and "
             "python-lovelock-report.json, check rust_ricci_einstein_scalar_agree)")
```

Out [17]:

```
  x0: R^a_a = -6*H**2           G^a_a = 3*(a4p*b4p - b4pp + 2*b4v*(5*H**2 -
    a4p**2))/(2*b4v)
  x1: R^a_a = -6*H**2 + 3*a4p*b4p/(2*b4v) + a4pp G^a_a = (3*a4p*b4p - 3*b4pp/2 +
    b4v*(15*H**2 - 3*a4p**2 + a4pp))/b4v
  x2: R^a_a = -6*H**2 + 3*a4p*b4p/(2*b4v) + a4pp G^a_a = (3*a4p*b4p - 3*b4pp/2 +
    b4v*(15*H**2 - 3*a4p**2 + a4pp))/b4v
  x3: R^a_a = -6*H**2 + 3*a4p*b4p/(2*b4v) + a4pp G^a_a = (3*a4p*b4p - 3*b4pp/2 +
    b4v*(15*H**2 - 3*a4p**2 + a4pp))/b4v
  x4: R^a_a = 3*(8*a4p**2*b4v**2 - b4p**2 + 2*b4v*(-2*a4p*b4p + b4pp))/(4*b4v**2) G^a_a =
    21*H**2 + 3*a4p**2 - 3*a4p*b4p/(2*b4v) - 3*b4p**2/(4*b4v**2)
  x5: R^a_a = -6*H**2 - 3*a4p*b4p/(2*b4v) - a4pp + b4p**2/(4*b4v**2) + b4pp/(2*b4v) G^a_a
    = 15*H**2 - 3*a4p**2 - a4pp + b4p**2/(4*b4v**2) - b4pp/b4v
  x6: R^a_a = -6*H**2 - 3*a4p*b4p/(2*b4v) - a4pp + b4p**2/(4*b4v**2) + b4pp/(2*b4v) G^a_a
    = 15*H**2 - 3*a4p**2 - a4pp + b4p**2/(4*b4v**2) - b4pp/b4v
  x7: R^a_a = -6*H**2 - 3*a4p*b4p/(2*b4v) - a4pp + b4p**2/(4*b4v**2) + b4pp/(2*b4v) G^a_a
    = 15*H**2 - 3*a4p**2 - a4pp + b4p**2/(4*b4v**2) - b4pp/b4v
Ricci scalar R = -42*H**2 + 6*a4p**2 - 3*a4p*b4p/b4v + 3*b4pp/b4v
PASS all 64 Ricci, all 64 Einstein components and R equal the record
     reproduces Revision/gkd_lovelock/results/curvature.json, ricciMixed, einsteinMixed,
    ricciScalar (and python-lovelock-report.json, check rust_ricci_einstein_scalar_agree)
```

Text cell:

```
The Ricci scalar appears in one more Revision record, in a different form. The
record of the field equations of $a_4$ builds them from three **Lovelock
scalars** $L_{(1)}, L_{(2)}, L_{(3)}$ (the subject of a later chapter), and the
first of them is exactly twice the Ricci scalar, $L_{(1)} = 2R$ (this identity is
the check ‘L1_equals_2R‘ of the record
‘Revision/gkd_lovelock/results/python-lovelock-report.json‘). The sympy
verification of the record of the field equations,
‘Revision/field_equations_a4/reports/python-a4-report.json‘, prints $L_{(1)}$ in its
check ‘L1_equals_gkd_branch‘, with the name ‘ad1‘ for $a_4’$, for the metric at
$b_4 = 1$. The next cell reads this text, checks that the record marks the check as
passed, turns the text after the equals sign into a sympy expression, confirms with
‘record_check‘ that the check ‘L1_equals_2R‘ passed, and checks $L_{(1)} = 2R$ with
our $R$ at $b_4 = 1$.
```

In [18]:

```
A4_REPORT = "Revision/field_equations_a4/reports/python-a4-report.json"
a4_report = json.loads(repository_file(A4_REPORT).read_text(encoding="utf-8"))
# the report's checks form a list of dictionaries; take the one with this name
L1_entry = next(entry for entry in a4_report["checks"]
                if entry["name"] == "L1_equals_gkd_branch")
L1_detail, L1_verdict = L1_entry["detail"], L1_entry["verdict"]
say(f"record: {L1_detail} ({L1_verdict})")
L1_text = L1_detail.split("=")[1]  # the text after the equals sign
L1_record = parse_expr(L1_text, local_dict={"H": H, "ad1": a4p})
record_check(PYTHON_REPORT, "L1_equals_2R", "L_(1) = 2 R exactly")  # stops if not
R_at_b4_1 = plain(R_scalar).subs({b4pp: 0, b4p: 0, b4v: 1})  # R at b4 = 1
check(L1_verdict == "PASS" and sp.expand(L1_record - 2 * R_at_b4_1) == 0,
      "at b4 = 1 the first Lovelock scalar of the record is twice our Ricci scalar",
      record=f"{A4_REPORT}, check L1_equals_gkd_branch (with "
             "python-lovelock-report.json, check L1_equals_2R)")
```

Out [18]:

```
record: L_(1) = -84*H**2 + 12*ad1**2 (PASS)
PASS at b4 = 1 the first Lovelock scalar of the record is twice our Ricci scalar
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    L1_equals_gkd_branch (with python-lovelock-report.json, check L1_equals_2R)
```

Text cell:

```
Exact agreement was decided by sympy's simplification. A second, independent test
uses numbers only: the independent Revision verification
(‘Revision/gkd_lovelock/results/python-lovelock-report.json‘, list ‘randomPoints‘)
chose five test points, each a set of exact fractions for $H$, $x_0$, $a_4$, $a_4'$,
$a_4''$, $b_4$, $b_4'$ and $b_4''$. The next cell evaluates every component of the
record (25 Christoffel symbols, 156 Riemann components, 64 Ricci and 64 Einstein
components and the Ricci scalar) and our component with the same indices at these
five points, with
**30 significant digits** (the package mpmath; ‘sp.lambdify(..., "mpmath")‘ turns an
expression into an mpmath function), and reports the largest relative difference
```

$|{\rm ours} - {\rm record}| / \max(1, |{\rm record}|)$. Before it reports, it
confirms with ‘record_check‘ that the three exact comparisons of that verification
(‘rust_christoffels_agree‘, ‘rust_riemann_agrees‘,
‘rust_ricci_einstein_scalar_agree‘) passed.

In [19]:

```
PYTHON_REPORT = "Revision/gkd_lovelock/results/python-lovelock-report.json"
python_report = json.loads(repository_file(PYTHON_REPORT).read_text(encoding="utf-8"))
mpmath.mp.dps = 30  # mpmath works with 30 significant digits from now on


def exact(text):
    """A fraction such as 46/75 of the record as an mpmath number."""
    fraction = sp.Rational(text)
    return mpmath.mpf(fraction.p) / fraction.q  # numerator / denominator


test_points = [[exact(p["H"]), exact(p["x0"]), exact(p["a4"]),
                exact(p["a4" + prime]), exact(p["a4" + prime + prime]),
                exact(p["b4"]), exact(p["b4" + prime]), exact(p["b4" + prime + prime])]
               for p in python_report["randomPoints"]]
pairs = [(Gamma[a][b][c], v) for (a, b, c), v in record_gamma.items()]
pairs += [(R_mixed[key], v) for key, v in record_riemann.items()]
pairs += [(Ric[i, j], from_mathematica(record["ricciMixed"][f"{NAMES[i]},{NAMES[j]}"]
                                       ["mathematica"]))
          for i in range(8) for j in range(8)]
pairs += [(G[i, j], from_mathematica(record["einsteinMixed"][f"{NAMES[i]},{NAMES[j]}"]
                                     ["mathematica"]))
          for i in range(8) for j in range(8)]
pairs += [(R_scalar, from_mathematica(record["ricciScalar"]))]
# the order of the eight numbers of a test point
ARGS30 = (H, x0, a4v, a4p, a4pp, b4v, b4p, b4pp)
largest, evaluations = mpmath.mpf(0), 0
for mine, theirs in pairs:
    f_mine = sp.lambdify(ARGS30, symbolic(sp.S(mine)), "mpmath")
    f_record = sp.lambdify(ARGS30, sp.S(theirs), "mpmath")
    for point in test_points:
        ours, recorded = f_mine(*point), f_record(*point)
        largest = max(largest, abs(ours - recorded) / max(1, abs(recorded)))
        evaluations += 1
for name in ("rust_christoffels_agree", "rust_riemann_agrees",
             "rust_ricci_einstein_scalar_agree"):
    record_check(PYTHON_REPORT, name)  # stops unless the check passed
report("components compared numerically", len(pairs))
report("evaluations at the five test points", evaluations)
report("largest relative difference (30 digits)", mpmath.nstr(largest, 3))
check(len(test_points) == 5 and largest < mpmath.mpf(10) ** -25,
      "at the five test points every component agrees to more than 25 digits",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "randomPoints, checks rust_christoffels_agree, rust_riemann_agrees, "
             "rust_ricci_einstein_scalar_agree")
```

Out [19]:

```
RESULT components compared numerically = 310
RESULT evaluations at the five test points = 1550
RESULT largest relative difference (30 digits) = 2.96e-31
PASS at the five test points every component agrees to more than 25 digits
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, randomPoints,
    checks rust_christoffels_agree, rust_riemann_agrees, rust_ricci_einstein_scalar_agree
```

Text cell:

```
The lead's independent check of the field equations
(`Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`) states four
properties of the Einstein tensor of the metric at $b_4 = 1$. The next cell first
confirms with `record_check` that the lead's report lists the four checks as passed,
then checks the four properties on our result with $b_4$ set to 1: $G$ is diagonal
(in particular $G^{x_4}{}_{x_0} = 0$), no component depends on $x_0$, the three
3-space components are equal and the three extra-time components are equal, and
$G^{x_4}{}_{x_4} - G^{x_0}{}_{x_0} = 6(a_4'^2 + H^2)$, which is positive for $H > 0$
(a later chapter uses this to show that for $H > 0$ no empty spacetime, not even one
with a cosmological constant, has this metric: some matter is needed). For a
general $b_4$ the first check also confirms what the verifier of the metric with
$b_4$ recorded (`Revision/metric_b4/results/wolfram-metric-b4-report.json`, checks
`off_diagonal_entries_x0_x4_only` and `off_diagonal_proportional_to_b4prime`): the
only off-diagonal components are $G^{x_4}{}_{x_0} = \tfrac32 H\cot z\, b_4'/b_4$
and $G^{x_0}{}_{x_4} = -\tfrac32 H\tan z\, b_4'/b_4$, both zero when $b_4$ is
constant; and the isotropy holds for every $b_4$.
```

In [20]:

```
LEAD = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"
for name in ("einstein_off_diagonal_zero", "einstein_x0_independent",
             "einstein_isotropy", "no_vacuum_for_H_positive"):
    record_check(LEAD, name)  # stops unless the lead check passed
METRIC_B4 = "Revision/metric_b4/results/wolfram-metric-b4-report.json"
for name in ("off_diagonal_entries_x0_x4_only", "off_diagonal_proportional_to_b4prime"):
    record_check(METRIC_B4, name)  # stops unless the check passed
G_b4_1 = G.applyfunc(lambda e: symbolic(e).subs({b4pp: 0, b4p: 0, b4v: 1}))
off_diagonal = {(i, j): plain(G[i, j]) for i in range(8) for j in range(8)
                if i != j and G[i, j] != 0}
for (i, j), value in sorted(off_diagonal.items()):
    say(f"  G^{NAMES[i]}_{NAMES[j]} = {value}")
check(sorted(off_diagonal) == [(0, 4), (4, 0)]
      and vanishes(symbolic(G[4, 0]) - sp.Rational(3, 2) * H * sp.cot(6 * H * x0)
                   * b4p / b4v)
      and vanishes(symbolic(G[0, 4]) + sp.Rational(3, 2) * H * sp.tan(6 * H * x0)
                   * b4p / b4v)
      and all(G_b4_1[i, j] == 0 for i in range(8) for j in range(8) if i != j),
      "the only off-diagonal components G^x4_x0, G^x0_x4 are proportional to b4p; "
      "at b4 = 1 the Einstein tensor is diagonal",
      record=f"{LEAD}, check einstein_off_diagonal_zero, and "
```

```
            "wolfram-metric-b4-report.json, checks off_diagonal_entries_x0_x4_only, "
            "off_diagonal_proportional_to_b4prime")
check(all(sp.diff(G_b4_1[i, j], x0) == 0 for i in range(8) for j in range(8)),
      "at b4 = 1 no component of the Einstein tensor depends on x0",
      record=f"{LEAD}, check einstein_x0_independent")
check(G[1, 1] == G[2, 2] == G[3, 3] and G[5, 5] == G[6, 6] == G[7, 7],
      "3-space components equal, extra-time components equal",
      record=f"{LEAD}, check einstein_isotropy")
gap = sp.simplify(G_b4_1[4, 4] - G_b4_1[0, 0])
say(f"at b4 = 1: G^x4_x4 - G^x0_x0 = {sp.factor(gap)}")
check(sp.simplify(gap - 6 * (a4p ** 2 + H ** 2)) == 0,
      "at b4 = 1: G^x4_x4 - G^x0_x0 = 6 (a4p^2 + H^2) > 0",
      record=f"{LEAD}, check no_vacuum_for_H_positive")
```

Out [20]:

```
  G^x0_x4 = -3*H*b4p*tan(z)/(2*b4v)
  G^x4_x0 = 3*H*b4p/(2*b4v*tan(z))
PASS the only off-diagonal components G^x4_x0, G^x0_x4 are proportional to b4p; at b4 = 1
    the Einstein tensor is diagonal
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_off_diagonal_zero, and wolfram-metric-b4-report.json, checks
    off_diagonal_entries_x0_x4_only, off_diagonal_proportional_to_b4prime
PASS at b4 = 1 no component of the Einstein tensor depends on x0
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_x0_independent
PASS 3-space components equal, extra-time components equal
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_isotropy
at b4 = 1: G^x4_x4 - G^x0_x0 = 6*(H**2 + a4p**2)
PASS at b4 = 1: G^x4_x4 - G^x0_x0 = 6 (a4p^2 + H^2) > 0
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    no_vacuum_for_H_positive
```

Text cell:

```
Every Einstein tensor satisfies the **contracted Bianchi identity**
$\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$, where for a tensor with one upper and one
lower index $\sum_\mu \nabla_\mu G^\mu{}_\nu = \sum_\mu \partial_\mu G^\mu{}_\nu +
\sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda} G^\lambda{}_\nu - \sum_{\mu,\lambda}
\Gamma^\lambda{}_{\mu\nu} G^\mu{}_\lambda$. Within Einstein's equations it makes the
energy and momentum of the matter that sources the metric conserved. The next cell
computes the eight components with the function $a_4(x_4)$ (so that sympy can
differentiate $a_4''$ and produce $a_4'''$) and checks that each simplifies to zero.
The Rust program checked the same for its tensor $P_{(1)} = -4G$ (its checks
`k1_equals_minus_4_einstein` and `k1_divergence_free`, which the cell confirms with
`record_check`).
```

In [21]:

```
divergence = []
for nu in range(8):
```

```
    value = sum(sp.diff(G[mu, nu], X[mu]) for mu in range(8)) \
        + sum(Gamma[mu][mu][lam] * G[lam, nu] for mu in range(8) for lam in range(8)) \
        - sum(Gamma[lam][mu][nu] * G[mu, lam] for mu in range(8) for lam in range(8))
    divergence.append(vanishes(value))
record_check(RUST_REPORT, "k1_equals_minus_4_einstein")  # stops if not passed
record_check(RUST_REPORT, "k1_divergence_free")
check(all(divergence),
      "the contracted Bianchi identity: the divergence of G vanishes",
      record="Revision/gkd_lovelock/results/lovelock-report.json, checks "
             "k1_equals_minus_4_einstein and k1_divergence_free")
```

Out [21]:

```
PASS the contracted Bianchi identity: the divergence of G vanishes
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, checks
    k1_equals_minus_4_einstein and k1_divergence_free
```

Text cell:

```
## 12. The Kretschmann scalar: an invariant that does not see $z$

The next cell computes $K = \sum R^{ab}{}_{cd} R^{cd}{}_{ab}$ and
$\sum_{a,b} R^a{}_b R^b{}_a$ from our components, and computes $K$ a second time
from the 156 components of the record alone. Although some components depend on
$z$ (for example $R^{x_1x_4}{}_{x_1x_0} = H a_4' \cot z$), their products combine so
that $K$ depends only on $H$, $a_4'$, $a_4''$, $b_4$, $b_4'$ and $b_4''$. For a
constant $b_4$ (then $b_4$ drops out entirely), completing the square,

$$\frac{K}{12} = 7H^4 - 2H^2 a_4'^2 + 7 a_4'^4 + 2 a_4''^2
= 7\Bigl(a_4'^2 - \frac{H^2}{7}\Bigr)^2 + \frac{48}{7} H^4 + 2 a_4''^2 ,$$

(expand the square: $7 a_4'^4 - 2 H^2 a_4'^2 + H^4/7$, and $H^4/7 + 48 H^4/7 =
7 H^4$), so $K \ge \frac{576}{7} H^4 > 0$ for every history with a constant $b_4$:
the metric is curved for every $H > 0$. (For a changing $b_4$ there is no such
bound: section 14 shows a member of the family whose $K$ vanishes.) The cell checks
all of this.
```

In [22]:

```
K = sp.factor(sp.expand(symbolic(sum(v * get(R_mixed, (c, d, a, b))
                                     for (a, b, c, d), v in R_mixed.items()))))
K_record = sp.factor(sp.expand(sum(v * record_riemann.get((c, d, a, b), 0)
                                   for (a, b, c, d), v in record_riemann.items())))
ricci_square = sp.factor(sp.expand(symbolic(sum(Ric[i, j] * Ric[j, i]
                                                for i in range(8)
                                                for j in range(8)))))
say(f"Kretschmann scalar K = {K}")
say(f"sum of R^a_b R^b_a  = {ricci_square}")
check(sp.simplify(K - K_record) == 0,
      "K from our components equals K from the 156 components of the record")
check(not K.has(x0) and not K.has(a4v),
```

```
      "K depends only on H, a4p, a4pp, b4v, b4p and b4pp, not on z and not on a4")
square_form = 7 * (a4p ** 2 - H ** 2 / 7) ** 2 + sp.Rational(48, 7) * H ** 4 \
    + 2 * a4pp ** 2
K_constant_b4 = sp.factor(K.subs({b4pp: 0, b4p: 0}))  # b4 constant: b4v drops out
say(f"for a constant b4: K = {K_constant_b4}")
check(sp.expand(K_constant_b4 / 12 - square_form) == 0,
      "for constant b4: K/12 = 7 (a4p^2 - H^2/7)^2 + 48 H^4/7 + 2 a4pp^2, so K > 0 "
      "for H > 0")
report("smallest possible K for constant b4 (at a4p^2 = H^2/7, a4pp = 0)",
       sp.sstr(sp.Rational(576, 7) * H ** 4))
```

Out [22]:

```
Kretschmann scalar K = 3*(56*H**4*b4v**4 - 16*H**2*a4p**2*b4v**4 + 8*H**2*a4p*b4p*b4v**3
    - 8*H**2*b4p**2*b4v**2 + 56*a4p**4*b4v**4 - 56*a4p**3*b4p*b4v**3 +
    22*a4p**2*b4p**2*b4v**2 + 8*a4p**2*b4pp*b4v**3 + 16*a4p*a4pp*b4p*b4v**3 -
    8*a4p*b4p*b4pp*b4v**2 + 16*a4pp**2*b4v**4 + 4*a4pp*b4p**2*b4v**2 - 8*a4pp*b4pp*b4v**3
    + b4p**4 - 2*b4p**2*b4pp*b4v + 2*b4pp**2*b4v**2)/(2*b4v**4)
sum of R^a_b R^b_a  = 3*(336*H**4*b4v**4 - 18*H**2*b4p**2*b4v**2 - 24*H**2*b4pp*b4v**3 +
    48*a4p**4*b4v**4 - 48*a4p**3*b4p*b4v**3 + 18*a4p**2*b4p**2*b4v**2 +
    24*a4p**2*b4pp*b4v**3 + 24*a4p*a4pp*b4p*b4v**3 + 3*a4p*b4p**3*b4v -
    18*a4p*b4p*b4pp*b4v**2 + 8*a4pp**2*b4v**4 - 2*a4pp*b4p**2*b4v**2 - 4*a4pp*b4pp*b4v**3
    + b4p**4 - 2*b4p**2*b4pp*b4v + 4*b4pp**2*b4v**2)/(4*b4v**4)
PASS K from our components equals K from the 156 components of the record
PASS K depends only on H, a4p, a4pp, b4v, b4p and b4pp, not on z and not on a4
for a constant b4: K = 12*(7*H**4 - 2*H**2*a4p**2 + 7*a4p**4 + 2*a4pp**2)
PASS for constant b4: K/12 = 7 (a4p^2 - H^2/7)^2 + 48 H^4/7 + 2 a4pp^2, so K > 0 for H >
    0
RESULT smallest possible K for constant b4 (at a4p^2 = H^2/7, a4pp = 0) = 576*H**4/7
```

Text cell:

```
The next cell shows the cancellation in pictures, for $a_4'' = 0$, $b_4 = 1$ and
$H = 1$. Left:
the sizes of two $z$-dependent components, $R^{x_1x_4}{}_{x_1x_0} = H a_4' \cot z$
and $R^{x_1x_0}{}_{x_1x_4} = -H a_4' \tan z$, of their product $-H^2 a_4'^2$ and of
the constant component $R^{x_1x_0}{}_{x_1x_0} = -H^2$, for $a_4' = 1.5$ (so that the
product, $2.25$, and $H^2 = 1$ are different numbers). Right: $K$
computed at 300 values of $z$ by summing all 156 products numerically, for four
expansion rates $a_4' = A H$ of deflating histories, $A = 0.5, 1, 1.5, 2$: four
flat lines.
```

In [23]:

```
# numerical functions of the 156 components with a4pp = 0 and b4pp = 0 (true on
# the history a4 = A H x4, b4 = 1); the second derivatives are set to 0 first
riemann_functions = {k: numeric(v.subs({sp.Derivative(a4, (x4, 2)): 0,
                                        sp.Derivative(b4, (x4, 2)): 0}))
                     for k, v in R_mixed.items()}
```

```
def kretschmann_numbers(z_values, rate):
    """K at the points z_values, summed from the 156 numerical components."""
    total = np.zeros_like(z_values)
    for (a, b, c, d), f in riemann_functions.items():
        partner = riemann_functions.get((c, d, a, b))
        if partner is not None:
            total += (f(z_values, 0.5, rate, 1.0, 1.0, 0.0)
                      * partner(z_values, 0.5, rate, 1.0, 1.0, 0.0))
    return total


z_line = np.linspace(0.02, np.pi / 2 - 0.02, 300)
cot_part = riemann_functions[(1, 4, 1, 0)](z_line, 0.5, 1.5, 1.0, 1.0, 0.0)
tan_part = riemann_functions[(1, 0, 1, 4)](z_line, 0.5, 1.5, 1.0, 1.0, 0.0)
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
fig.subplots_adjust(wspace=0.32)  # room for the label of the right vertical axis
axes[0].plot(z_line, np.abs(cot_part), color=RED, lw=1.8,
             label="$R^{x_1x_4}{}_{x_1x_0} = Ha_4^{\\prime}\\cot z$")
axes[0].plot(z_line, np.abs(tan_part), color=BLUE, lw=1.8, ls="--",
             label="minus $R^{x_1x_0}{}_{x_1x_4} = Ha_4^{\\prime}\\tan z$")
axes[0].plot(z_line, np.abs(cot_part * tan_part), color="black", lw=1.2, ls=":",
             label="minus their product $= H^2 a_4^{\\prime 2}$")
axes[0].plot(z_line, np.abs(np.broadcast_to(
    riemann_functions[(1, 0, 1, 0)](z_line, 0.5, 1.5, 1.0, 1.0, 0.0), z_line.shape)),
    color=AQUA, lw=1.8, ls="-.", label="minus $R^{x_1x_0}{}_{x_1x_0} = H^2$")
axes[0].set_yscale("log")
axes[0].set_xlabel("$z = 6 H x_0$ (radians)")
axes[0].set_ylabel("absolute value (unit $H^2$)")
axes[0].set_title("Riemann components, $a_4^{\\prime} = 1.5$")
axes[0].legend(fontsize=8)
rates = (0.5, 1.0, 1.5, 2.0)  # four expansion rates a4p = A H (A > 0), unit H
for rate, colour, style in zip(rates, (VIOLET, AQUA, RED, ORANGE),
                               ("-", "--", "-.", ":")):
    axes[1].plot(z_line, kretschmann_numbers(z_line, rate), color=colour, ls=style,
                 lw=1.8, label=f"$a_4^{{\\prime}} = {rate:g}\\,H$")
axes[1].set_yscale("log")
axes[1].set_xlabel("$z = 6 H x_0$ (radians)")
axes[1].set_ylabel("$K$ (unit $H^4$)")
axes[1].set_title("Kretschmann scalar summed from 156 components")
axes[1].legend(fontsize=8)
# the values of the formula for K at the four rates, for the caption and the check
K_values = [float(K.subs({a4p: r, a4pp: 0, H: 1, b4v: 1, b4p: 0, b4pp: 0}))
            for r in rates]
K_text = ", ".join(f"${k:g}$" for k in K_values)
save_figure(fig, "riemann_and_kretschmann_z",
            "Left: the absolute values of the Riemann components "
            "$R^{x_1x_4}{}_{x_1x_0} = Ha_4^{\\prime}\\cot z$ (solid) and "
            "$R^{x_1x_0}{}_{x_1x_4} = -Ha_4^{\\prime}\\tan z$ (dashed), of their "
            "product (dotted) and of $R^{x_1x_0}{}_{x_1x_0} = -H^2$ (dash-dotted) "
            "across the patch, for $a_4^{\\prime} = 1.5$, "
            "$a_4^{\\prime\\prime} = 0$, $b_4 = 1$, $H = 1$; horizontal axis "
            "$z = 6Hx_0$ in "
            "radians, vertical axis on a logarithmic scale in units of $H^2$. "
            "Right: the Kretschmann scalar $K$ summed numerically from all 156 "
```

```
            "components at 300 values of $z$ for $a_4^{\\prime} = 0.5, 1, 1.5, 2$ "
            "(in units of $H$), vertical axis $K$ in units of $H^4$, logarithmic. "
            "Single components grow without bound at the ends of the patch, but $K$ "
            f"is the same at every $z$: {K_text}.")
numbers = [kretschmann_numbers(z_line, r) for r in rates]
check(all(np.max(np.abs(n - k)) < 1e-9 * k for n, k in zip(numbers, K_values)),
      "summed numerically, K is the same at all 300 values of z and equals the "
      "formula")
```

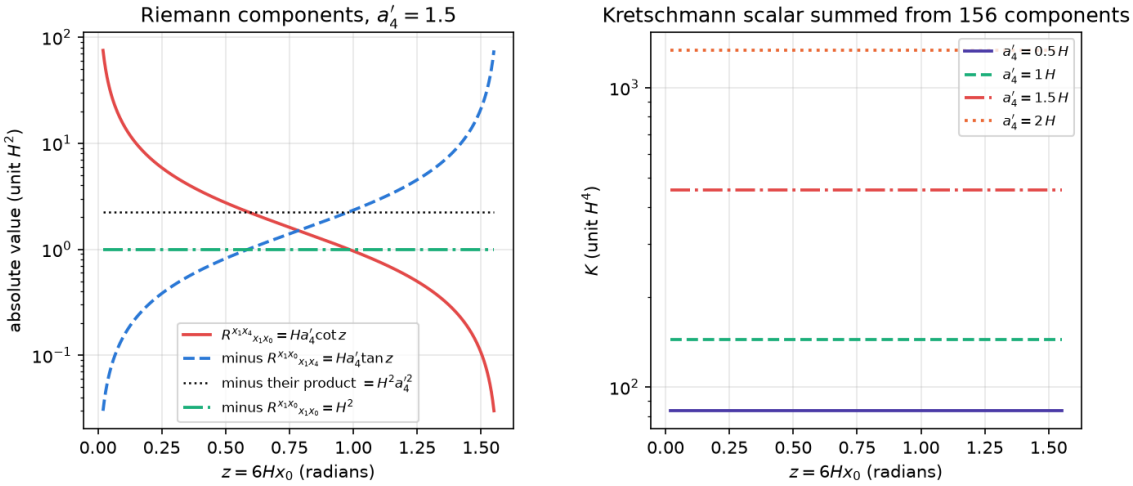

Figure 108: Left: the absolute values of the Riemann components $R^{x_1x_4}{}_{x_1x_0} = Ha_4' \cot z$ (solid) and $R^{x_1x_0}{}_{x_1x_4} = -Ha_4' \tan z$ (dashed), of their product (dotted) and of $R^{x_1x_0}{}_{x_1x_0} = -H^2$ (dash-dotted) across the patch, for $a_4' = 1.5$, $a_4'' = 0$, $b_4 = 1$, $H = 1$; horizontal axis $z = 6Hx_0$ in radians, vertical axis on a logarithmic scale in units of H^2. Right: the Kretschmann scalar K summed numerically from all 156 components at 300 values of z for $a_4' = 0.5, 1, 1.5, 2$ (in units of H), vertical axis K in units of H^4, logarithmic. Single components grow without bound at the ends of the patch, but K is the same at every z: 83.25, 144, 455.25, 1332. (Notebook 03b, figure 5.)

Out [23]:

```
Figure 03b.5 saved as Revision/textbook/figures/03b_5_riemann_and_kretschmann_z.png
PASS summed numerically, K is the same at all 300 values of z and equals the formula
```

Text cell:

```
## 13. The curvature along the deflating history

Along the history $a_4 = A H x_4$, $b_4 = 1$ we have $a_4' = A H$, $a_4'' = 0$ and
$b_4' = b_4'' = 0$, so every curvature scalar is a polynomial in the slope $A$. The
author's extra times deflate when $a_4$ increases, that is for $A > 0$; the Revision
record uses $A = 1$. The next cell evaluates the Ricci scalar $R = 6H^2(A^2 - 7)$,
the sum $\sum R^a{}_b R^b{}_a$ and $K$ as formulas in $A$, checks that they are
**even** in $A$ (they do not change when $A$ is replaced by $-A$; so these three
scalars cannot tell which of the two families of directions deflates, the metric and
the components of the curvature tensors can), and draws them for $0.1 \le A \le 3$
```

```
(every one of these histories deflates the extra times): $R$ on an ordinary axis
(it vanishes at $A = \sqrt 7$), the two squares on a logarithmic axis.
```

In [24]:

```
A = sp.symbols("A", real=True)  # the slope of the history a4 = A H x4
on_history = {a4p: A * H, a4pp: 0, b4pp: 0, b4p: 0, b4v: 1}
R_A = sp.factor(symbolic(R_scalar).subs(on_history))
K_A = sp.factor(K.subs(on_history))
RR_A = sp.factor(ricci_square.subs(on_history))
say(f"on the history: R = {R_A},  K = {K_A},  R^a_b R^b_a = {RR_A}")
check(all(sp.expand(f.subs(A, -A) - f) == 0 for f in (R_A, K_A, RR_A)),
      "the curvature scalars are even in A")
A_line = np.linspace(0.1, 3.0, 291)  # slopes 0.1, 0.11, ..., 3; A_line[90] = 1
R_numbers = sp.lambdify(A, R_A.subs(H, 1))(A_line)
K_numbers = sp.lambdify(A, K_A.subs(H, 1))(A_line)
RR_numbers = sp.lambdify(A, RR_A.subs(H, 1))(A_line)
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
axes[0].plot(A_line, R_numbers, color=VIOLET, lw=1.8, label="$R = 6H^2(A^2 - 7)$")
axes[0].axhline(0.0, color="black", lw=0.8)
axes[0].plot([np.sqrt(7)], [0.0], "o", color=VIOLET, ms=7)  # the zero of R
axes[0].set_ylabel("Ricci scalar $R$ (unit $H^2$)")
axes[0].set_title("Ricci scalar")
axes[1].plot(A_line, K_numbers, color=RED, lw=1.8, label="Kretschmann $K$")
axes[1].plot(A_line, RR_numbers, color=BLUE, lw=1.8, ls="--",
             label="$\\sum R^a{}_b R^b{}_a$")
axes[1].set_yscale("log")
axes[1].set_ylabel("value (unit $H^4$)")
axes[1].set_title("Squares of the curvature")
for ax in axes:
    ax.axvline(1.0, color="grey", lw=1.0, ls=":")  # the canonical slope A = 1
    ax.set_xlabel("slope $A$ of the deflating history $a_4 = A H x_4$")
    ax.legend(fontsize=8, loc="upper left")
axes[0].text(1.05, -40, "$A = 1$", fontsize=9)
save_figure(fig, "scalars_versus_a",
            "The curvature scalars of the author's metric along the deflating "
            "history $a_4 = AHx_4$, $b_4 = 1$ ($a_4^{\\prime} = AH$, "
            "$a_4^{\\prime\\prime} = 0$) as functions of the slope $A$ from $0.1$ "
            "to $3$, with $H = 1$; horizontal axes $A$ (a pure number), the dotted "
            "vertical line marks the canonical $A = 1$ of the Revision record. "
            "Left: the Ricci scalar $R = 6H^2(A^2 - 7)$ in units of $H^2$, "
            "negative for $A < \\sqrt 7$ and zero at the dot. Right, on a "
            "logarithmic axis in units of $H^4$: the Kretschmann scalar "
            "$K = 12H^4(7A^4 - 2A^2 + 7)$ (solid), smallest at "
            "$A^2 = 1/7$, and $\\sum R^a{}_b R^b{}_a = 36H^4(A^4 + 7)$ (dashed), "
            "both positive for every $A$.")
check(abs(R_numbers[90] - (-36.0)) < 1e-9 and abs(K_numbers[90] - 144.0) < 1e-9,
      "at A = 1: R = -36 H^2 and K = 144 H^4")
```

Out [24]:

```
on the history: R = 6*H**2*(A**2 - 7),  K = 12*H**4*(7*A**4 - 2*A**2 + 7),  R^a_b R^b_a =
```

```
    36*H**4*(A**4 + 7)
PASS the curvature scalars are even in A
```

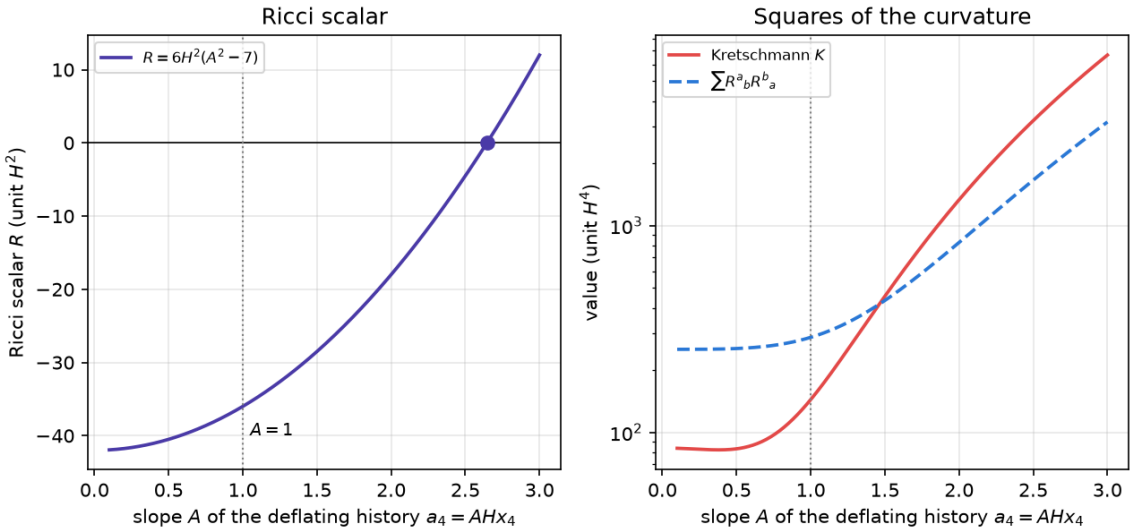

Figure 109: The curvature scalars of the author's metric along the deflating history $a_4 = AHx_4$, $b_4 = 1$ ($a_4' = AH$, $a_4'' = 0$) as functions of the slope A from 0.1 to 3, with $H = 1$; horizontal axes A (a pure number), the dotted vertical line marks the canonical $A = 1$ of the Revision record. Left: the Ricci scalar $R = 6H^2(A^2 - 7)$ in units of H^2, negative for $A < \sqrt{7}$ and zero at the dot. Right, on a logarithmic axis in units of H^4: the Kretschmann scalar $K = 12H^4(7A^4 - 2A^2 + 7)$ (solid), smallest at $A^2 = 1/7$, and $\sum R^a{}_b R^b{}_a = 36H^4(A^4 + 7)$ (dashed), both positive for every A. (Notebook 03b, figure 6.)

Out [24]:

```
Figure 03b.6 saved as Revision/textbook/figures/03b_6_scalars_versus_a.png
PASS at A = 1: R = -36 H^2 and K = 144 H^4
```

Text cell:

```
The next cell draws the Einstein tensor along the deflating history (with $b_4 = 1$
it is diagonal). Left: its eight diagonal components as bars, for the canonical slope
$A = 1$ of the Revision record and for $A = 2$. Right: as functions of $A$; with
$a_4'' = 0$ the hidden, 3-space and extra-time components coincide,
$G^{x_1}{}_{x_1} = G^{x_5}{}_{x_5} = G^{x_0}{}_{x_0} = 3H^2(5 - A^2)$, and
$G^{x_4}{}_{x_4} = 3H^2(7 + A^2)$; the gap between them, $6H^2(A^2 + 1)$, is never
zero.
```

In [25]:

```
G_A = [sp.factor(symbolic(G[k, k]).subs(on_history)) for k in range(8)]
for k in (0, 1, 4, 5):
    say(f"  on the history: G^{NAMES[k]}_{NAMES[k]} = {G_A[k]}")
```

```
bars = {value: [float(G_A[k].subs({A: value, H: 1})) for k in range(8)]
        for value in (1, 2)}
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
positions = np.arange(8)
axes[0].bar(positions - 0.2, bars[1], width=0.38, color=RED, hatch="//",
            label="$A = 1$")
axes[0].bar(positions + 0.2, bars[2], width=0.38, color=BLUE, label="$A = 2$")
axes[0].set_xticks(positions, [f"$G^{{x_{k}}}{{}}_{{x_{k}}}$" for k in range(8)],
                   fontsize=8)
axes[0].set_ylabel("component (unit $H^2$)")
axes[0].set_title("Diagonal of the Einstein tensor")
axes[0].legend(fontsize=8)
space_numbers = sp.lambdify(A, G_A[1].subs(H, 1))(A_line)
time_numbers = sp.lambdify(A, G_A[4].subs(H, 1))(A_line)
axes[1].plot(A_line, space_numbers, color=RED, lw=1.8,
             label="$G^{x_1}{}_{x_1} = G^{x_5}{}_{x_5} = G^{x_0}{}_{x_0}$")
axes[1].plot(A_line, time_numbers, color=VIOLET, lw=1.8, ls="--",
             label="$G^{x_4}{}_{x_4}$")
axes[1].fill_between(A_line, space_numbers, time_numbers, color="grey", alpha=0.2,
                     label="gap $6H^2(A^2 + 1) > 0$")
axes[1].axhline(0.0, color="black", lw=0.8)
axes[1].axvline(1.0, color="grey", lw=1.0, ls=":")  # the canonical slope A = 1
axes[1].set_xlabel("slope $A$ of the deflating history $a_4 = A H x_4$")
axes[1].set_ylabel("component (unit $H^2$)")
axes[1].set_title("Einstein tensor versus $A$")
axes[1].legend(fontsize=8, loc="lower left")
save_figure(fig, "einstein_tensor",
            "The Einstein tensor $G^{\\mu}{}_{\\nu}$ of the author's metric along "
            "the deflating history $a_4 = AHx_4$, $b_4 = 1$ with $H = 1$, in units of "
            "$H^2$. "
            "Left: the eight diagonal components (the only non-zero ones) for the "
            "canonical slope $A = 1$ of the Revision record (hatched bars: "
            "$12, 12, 12, 12, 24, 12, 12, 12$) and for $A = 2$ (plain bars: "
            "$3, 3, 3, 3, 33, 3, 3, 3$). Right: the hidden, 3-space and extra-time "
            "components $3H^2(5 - A^2)$ (solid) and the time component "
            "$3H^2(7 + A^2)$ (dashed) as functions of the slope $A$ from $0.1$ to "
            "$3$; the shaded gap $6H^2(A^2 + 1)$ between them never closes.")
check(bars[1] == [12.0, 12.0, 12.0, 12.0, 24.0, 12.0, 12.0, 12.0]
      and bars[2] == [3.0, 3.0, 3.0, 3.0, 33.0, 3.0, 3.0, 3.0],
      "diagonal of G: 12 (24 for x4) at A = 1 and 3 (33 for x4) at A = 2, unit H^2")
```

Out [25]:

```
  on the history: G^x0_x0 = -3*H**2*(A**2 - 5)
  on the history: G^x1_x1 = -3*H**2*(A**2 - 5)
  on the history: G^x4_x4 = 3*H**2*(A**2 + 7)
  on the history: G^x5_x5 = -3*H**2*(A**2 - 5)
```

Out [25]:

```
Figure 03b.7 saved as Revision/textbook/figures/03b_7_einstein_tensor.png
PASS diagonal of G: 12 (24 for x4) at A = 1 and 3 (33 for x4) at A = 2, unit H^2
```

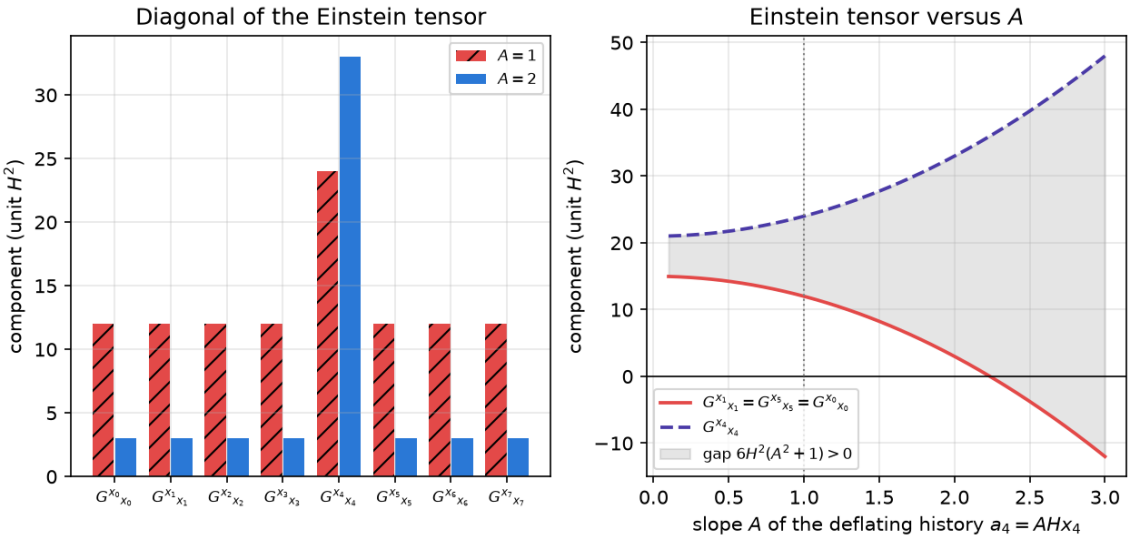

Figure 110: The Einstein tensor $G^{\mu}{}_{\nu}$ of the author's metric along the deflating history $a_4 = AHx_4$, $b_4 = 1$ with $H = 1$, in units of H^2. Left: the eight diagonal components (the only non-zero ones) for the canonical slope $A = 1$ of the Revision record (hatched bars: $12, 12, 12, 12, 24, 12, 12, 12$) and for $A = 2$ (plain bars: $3, 3, 3, 3, 33, 3, 3, 3$). Right: the hidden, 3-space and extra-time components $3H^2(5 - A^2)$ (solid) and the time component $3H^2(7 + A^2)$ (dashed) as functions of the slope A from 0.1 to 3; the shaded gap $6H^2(A^2 + 1)$ between them never closes. (Notebook 03b, figure 7.)

Text cell:

```
What do these numbers mean for matter? Einstein's field equations (section 3 of
this notebook) set $G^\mu{}_\nu + \Lambda\,\delta^\mu{}_\nu = \kappa\, T^\mu{}_\nu$
with $T = \mathrm{diag}(p_0, p_3, p_3, p_3, -\rho, p_t, p_t, p_t)$. Their $x_4$
component reads $G^{x_4}{}_{x_4} + \Lambda = -\kappa\rho$, so
$\kappa\rho = -G^{x_4}{}_{x_4} - \Lambda$; their $x_1$, $x_5$ and $x_0$ components
read $G^{x_1}{}_{x_1} + \Lambda = \kappa p_3$, $G^{x_5}{}_{x_5} + \Lambda = \kappa
p_t$ and $G^{x_0}{}_{x_0} + \Lambda = \kappa p_0$. Along the history (with
$b_4 = 1$) the three components on the left are equal (previous cell), so all three
pressures must be the same number $p$. The Revision record of the field equations of
$a_4$ (‘Revision/field_equations_a4/a4-equations.json‘, entry ‘linearMember‘) lists this
required source in its texts ‘rhoEinstein‘ ($\rho$), ‘pEinstein‘ ($p$) and
‘rhoPlusPEinstein‘ ($\rho + p$), written with ‘AA‘ for $A$ and ‘Lam‘ for $\Lambda$.
The next cell reads the three texts, turns them into sympy expressions, confirms
with ‘record_check‘ that the sympy verification of that record
(‘python-a4-report.json‘, check ‘json_linear_member‘) passed, and checks the three
texts against our Einstein tensor. The sum $\kappa(\rho + p) = -6(A^2 + 1)H^2$ is
negative for every $A$. Ordinary matter (dust, radiation, a gas) has
$\rho + p \ge 0$ (the **null energy condition**), so within Einstein's equations
the source that this history requires is not ordinary matter; a later chapter
discusses this, also for the field equations with the additional Lovelock terms.
Here we only check that the record and our curvature agree.
```

In [26]:

```
A4_EQUATIONS = "Revision/field_equations_a4/a4-equations.json"
a4_equations = json.loads(repository_file(A4_EQUATIONS).read_text(encoding="utf-8"))
linear = a4_equations["linearMember"]  # the entry of the linear history
kappa = sp.symbols("kappa", positive=True)  # the constant of Einstein's equations
Lam = sp.symbols("Lam", real=True)  # the cosmological constant Lambda
source_names = {"AA": A, "H": H, "kappa": kappa, "Lam": Lam}


def source(key):
    """The record's text linear[key] (Mathematica notation) as sympy."""
    return parse_expr(linear[key]["input"].replace("^", "**"),
                      local_dict=source_names)


rho_record, p_record = source("rhoEinstein"), source("pEinstein")
sum_record = source("rhoPlusPEinstein")  # rho + p
say(f"record: kappa rho       = {sp.expand(kappa * rho_record)}")
say(f"record: kappa p         = {sp.expand(kappa * p_record)}")
say(f"record: kappa (rho + p) = {sp.factor(kappa * sum_record)}")
record_check(A4_REPORT, "json_linear_member")  # stops if not passed
check(sp.expand(kappa * rho_record - (-G_A[4] - Lam)) == 0,
      "kappa rho of the record equals -G^x4_x4 - Lambda",
      record=f"{A4_EQUATIONS}, linearMember.rhoEinstein (and "
             "python-a4-report.json, check json_linear_member)")
check(all(sp.expand(kappa * p_record - (G_A[k] + Lam)) == 0 for k in (1, 5, 0)),
      "kappa p of the record equals G^x1_x1, G^x5_x5 and G^x0_x0 plus Lambda",
      record=f"{A4_EQUATIONS}, linearMember.pEinstein")
check(sp.expand(kappa * sum_record - (G_A[1] - G_A[4])) == 0,
      "kappa (rho + p) of the record equals G^x1_x1 - G^x4_x4 = -6 (A^2 + 1) H^2",
      record=f"{A4_EQUATIONS}, linearMember.rhoPlusPEinstein")
```

Out [26]:

```
record: kappa rho       = -3*A**2*H**2 - 21*H**2 - Lam
record: kappa p         = -3*A**2*H**2 + 15*H**2 + Lam
record: kappa (rho + p) = -6*H**2*(A**2 + 1)
PASS kappa rho of the record equals -G^x4_x4 - Lambda
     reproduces Revision/field_equations_a4/a4-equations.json, linearMember.rhoEinstein
    (and python-a4-report.json, check json_linear_member)
PASS kappa p of the record equals G^x1_x1, G^x5_x5 and G^x0_x0 plus Lambda
     reproduces Revision/field_equations_a4/a4-equations.json, linearMember.pEinstein
PASS kappa (rho + p) of the record equals G^x1_x1 - G^x4_x4 = -6 (A^2 + 1) H^2
     reproduces Revision/field_equations_a4/a4-equations.json,
    linearMember.rhoPlusPEinstein
```

Text cell:

```
## 14. A negative control: what if the extra times inflated?

A check that cannot fail teaches nothing. So the next cell changes the metric on
purpose: the extra times now **inflate** like 3-space, $g_{55} = g_{66} = g_{77} =
-e^{+2a_4}\sin^{1/3} z$, and computes the curvature again with the same functions.
```

```
This is not a deflating metric; within the author's family it is the member
$b_4 = e^{4a_4}$, whose extra-time rate $\alpha_t = -a_4' + b_4'/(2b_4) = +a_4'$
equals that of 3-space. With a constant $b_4$ the $x_4$-$x_0$ components of the
Einstein tensor cancel between the three inflating and the three deflating
directions; here they add up. The cell prints the off-diagonal Einstein components
that appear and checks that they are not zero, that they depend on $z$, and that
they are the general components $\pm\tfrac32 H (\cot z, \tan z)\, b_4'/b_4$ of
section 11 with $b_4 = e^{4a_4}$. It also computes the Kretschmann scalar of the
control and checks that it vanishes for $a_4' = H$, $a_4'' = 0$: the bound
$K \ge 576H^4/7$ of section 12 needs a constant $b_4$.
```

In [27]:

```
g_control = g.copy()
for k in (5, 6, 7):  # the three extra times, now inflating
    g_control[k, k] = -sp.exp(2 * a4) * sp.sin(6 * H * x0) ** sp.Rational(1, 3)
Gamma_control = christoffel(g_control, X)
R_control = raise_second(riemann(Gamma_control, X), g_control)
Ric_control = ricci(R_control, 8)
G_control = (Ric_control - Ric_control.trace() / 2 * sp.eye(8)).applyfunc(tidy)
off = {(i, j): plain(G_control[i, j]) for i in range(8) for j in range(8)
       if i != j and G_control[i, j] != 0}
for (i, j), value in sorted(off.items()):
    say(f"  control: G^{NAMES[i]}_{NAMES[j]} = {value}")
b4_control = {sp.Derivative(b4, x4): 4 * sp.Derivative(a4, x4) * sp.exp(4 * a4),
              b4: sp.exp(4 * a4)}  # b4 = e^(4 a4) and its derivative
general_off = [G[i, j].subs(b4_control) for i, j in ((0, 4), (4, 0))]
K_control = sp.expand(symbolic(sum(
    v * get(R_control, (c, d, a, b)) for (a, b, c, d), v in R_control.items())))
check(sorted(off) == [(0, 4), (4, 0)] and all(v.has(z) for v in off.values())
      and vanishes(G_control[0, 4] - general_off[0])
      and vanishes(G_control[4, 0] - general_off[1])
      and sp.simplify(K_control.subs({a4pp: 0, a4p: H})) == 0,
      "negative control: with inflating extra times (b4 = e^(4 a4)) G^x4_x0 and "
      "G^x0_x4 are not zero, depend on z, equal the general ones; K = 0 at a4p = H")
```

Out [27]:

```
  control: G^x0_x4 = -6*H*a4p*tan(z)
  control: G^x4_x0 = 6*H*a4p/tan(z)
PASS negative control: with inflating extra times (b4 = e^(4 a4)) G^x4_x0 and G^x0_x4 are
    not zero, depend on z, equal the general ones; K = 0 at a4p = H
```

Text cell:

```
## 15. The last check

The last cell checks that the seven figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [28]:

```
figure_names = ["christoffel_heat_maps", "christoffel_versus_z",
                "finite_differences", "plane_curvatures",
                "riemann_and_kretschmann_z", "scalars_versus_a", "einstein_tensor"]
files = [output_file(f"{FIGURE_FOLDER}/03b_{k}_{n}.png")
         for k, n in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all seven figure files exist")
all_checks_passed()
```

Out [28]:

```
PASS all seven figure files exist
ALL 40 CHECKS PASSED (notebook 03b)
```

Text cell:

```
## 16. What this notebook showed

- From the metric exactly as the author typed it, with general $a_4(x_4)$ and
  $b_4(x_4)$, sympy computed 37 non-zero Christoffel symbols (25 with $b \le c$),
  156 non-zero Riemann components $R^{ab}{}_{cd}$ with 20 different values, the Ricci
  tensor, the Ricci scalar ($R = 6a_4'^2 - 42H^2$ at $b_4 = 1$) and the Einstein
  tensor; every component agrees exactly
  with the Revision record of the Rust program `lovelock_gkd` (PROVED by exact
  algebra, here and in the record); at the five test points of the independent
  Revision verification all 310 recorded components agree with ours to more than 25
  significant digits. At $b_4 = 1$ twice our Ricci scalar equals the first Lovelock
  scalar $L_{(1)}$ of the record of the field equations of $a_4$.
- The Christoffel symbols were confirmed a second way, by finite differences, with
  an error that falls like $h^2$.
- The lines along $x_4$ are geodesics: observers at rest fall freely and $x_4$ is
  their proper time.
- The Riemann tensor has all the symmetries of a Riemann tensor and satisfies the
  first Bianchi identity; the Einstein tensor satisfies the contracted Bianchi
  identity and is isotropic in 3-space and in the extra times; its only off-diagonal
  components are $G^{x_4}{}_{x_0} = \tfrac32 H\cot z\, b_4'/b_4$ and
  $G^{x_0}{}_{x_4} = -\tfrac32 H\tan z\, b_4'/b_4$. At $b_4 = 1$ it is diagonal,
  does not depend on $x_0$, and $G^{x_4}{}_{x_4} - G^{x_0}{}_{x_0} = 6(a_4'^2 + H^2)
  > 0$.
- The Kretschmann scalar does not depend on $z$ although single components do; for
  a constant $b_4$ it is $K = 12(7H^4 - 2H^2a_4'^2 + 7a_4'^4 + 2a_4''^2)
  \ge 576H^4/7 > 0$: the metric is curved for every $H > 0$.
- Along the deflating history $a_4 = AHx_4$, $b_4 = 1$ the curvature scalars are
  $R = 6H^2(A^2 - 7)$, $K = 12H^4(7A^4 - 2A^2 + 7)$ and
  $\sum R^a{}_b R^b{}_a = 36H^4(A^4 + 7)$, all even in $A$. For the canonical
  history of the Revision record ($A = 1$, $H = 1$): $R = -36$, $K = 144$ and
  $G = \mathrm{diag}(12, 12, 12, 12, 24, 12, 12, 12)$ (units $H^2$ and $H^4$); for
  $A = 2$: $G = \mathrm{diag}(3, 3, 3, 3, 33, 3, 3, 3)$.
- Through Einstein's equations this Einstein tensor is exactly the source that the
  Revision record of the field equations of $a_4$ lists for the linear history:
  $\kappa\rho = -3(7 + A^2)H^2 - \Lambda$, $\kappa p = 3(5 - A^2)H^2 + \Lambda$ for
  all three pressures, and $\kappa(\rho + p) = -6(A^2 + 1)H^2 < 0$.
- Negative control: if the extra times inflated instead of deflating (the member
```

```
  $b_4 = e^{4a_4}$ of the family), the Einstein tensor acquires the off-diagonal
  components $G^{x_4}{}_{x_0}$ and $G^{x_0}{}_{x_4}$ of the general formula, and its
  Kretschmann scalar can vanish; with a constant $b_4$ the deflation of the extra
  times makes the Einstein tensor of the author's metric diagonal.
- ASSUMED for the plots and the evaluation along the history only: the history
  $a_4 = AHx_4$ with $A > 0$ and $b_4 = 1$ (a prescribed background, not solved
  for); every formula with general $a_4(x_4)$ and $b_4(x_4)$ is exact.
```

3.30 Line-by-line walk-through of Notebook 03b

The notebook has 28 code cells, In [1] to In [28]. As before, a line or a small group of lines is quoted and then explained, and a line `...` in a quoted figure command stands for the remaining lines of the caption, printed in full under its figure in Section 3.29.

In [1], the set-up cell. Its comment lines are the run instructions of Section 3.28. Its code is, line for line, the set-up code of Notebook 03a explained in Section 3.13 under In [1], with the single difference `NOTEBOOK_ID = "03b" # this notebook:  chapter 03, example b`. It prints Set-up of notebook 03b complete: repository folder found, helpers defined.

In [2], every result in one piece. Word for word In [2] of Notebook 03a (Section 3.13). It prints From now on check and report print each of their results in one piece.

In [3], packages, record and symbols.

```
import itertools  # loops over all index combinations

import mpmath  # numbers with many digits
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
from sympy.parsing.sympy_parser import (implicit_multiplication, parse_expr,
                                        standard_transformations)
```

The packages: `itertools` for loops over indices (Section 3.22), `mpmath` for numbers with many digits (used in In [19]), numpy, sympy, and the text reader `parse_expr` of sympy with its two reading rules (Section 3.13, In [3] and In [5]).

```
CURVATURE_RECORD = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE_RECORD).read_text(encoding="utf-8"))
PYTHON_REPORT = "Revision/gkd_lovelock/results/python-lovelock-report.json"  # sympy
RUST_REPORT = "Revision/gkd_lovelock/results/lovelock-report.json"  # the Rust checks


def record_check(report_file, check_name, detail_part=""):
    checks = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
    checks = checks["checks"]  # a dictionary or a list, depending on the report
    if isinstance(checks, dict):  # {name: {"passed": true, "detail": ...}}
        entry = checks.get(check_name, {})
        passed = entry.get("passed") is True
    else:  # [{"name": ..., "verdict": "PASS", "detail": ...}, ...]
        entry = next((e for e in checks if e.get("name") == check_name), {})
        passed = entry.get("verdict") == "PASS"
```

```
    if not passed or detail_part not in entry.get("detail", ""):
        raise AssertionError(f"record check failed: {report_file} does not list "
                             f"{check_name} as passed")
    return True


x0, x1, x2, x3, x4, x5, x6, x7 = sp.symbols("x0:8", real=True)
X = [x0, x1, x2, x3, x4, x5, x6, x7]  # the coordinates, counted 0 to 7 in Python
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
H = sp.symbols("H", positive=True)  # the constant of the author
a4 = sp.Function("a4")(x4)  # the metric function a4, unknown
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0, unknown
a4p, a4pp, a4v = sp.symbols("a4p a4pp a4v", real=True)  # a4 prime, a4 two primes, a4
b4p, b4pp = sp.symbols("b4p b4pp", real=True)  # b4 prime, b4 two primes
b4v = sp.symbols("b4v", positive=True)  # the value of b4
z = sp.symbols("z", positive=True)  # z = 6 H x0, for printing
```

The record of the Rust program `lovelock_gkd` is read into the dictionary `record`. `PYTHON_REPORT` and `RUST_REPORT` are the names of two further Revision reports: the report of the independent sympy checker and the report of the Rust program's own checks. The function `record_check` reads such a report and requires that it lists the check `check_name` as passed. The reports come in two layouts: in one, `checks` is a dictionary from each check name to an entry with the key `passed`; in the other, it is a list of entries, each with a `name` and a `verdict`. `isinstance(checks, dict)` asks which layout this report has; `checks.get(check_name, {})` gives the entry or an empty dictionary when the name is missing, and `next(..., {})` finds the entry with that name in the list (or the empty dictionary). The entry must say passed (the value `True`, or the verdict `"PASS"`), and, when `detail_part` is given, its `detail` text must contain that piece; otherwise `raise AssertionError(...)` stops the notebook with a message that names the report and the check. So whenever this notebook says that a number agrees with a Revision record, it also confirms, from the record itself, that the record's own check of that number passed. Then the eight coordinates, their list `X` and their printed names `NAMES`, in the author's order, so that `X[k]` is x_k and `NAMES[k]` is `"xk"`; the constant $H > 0$; the unknown functions $a_4(x_4)$ and $b_4(x_4)$, the second declared positive; and seven plain symbols for printing: `a4p` for a_4', `a4pp` for a_4'', `a4v` for the value of a_4, the same three for b_4 (`b4p`, `b4pp`, `b4v`, the last one positive), and `z` for $6Hx_0$.

```
def symbolic(expression):
    """Write the derivatives of a4 and b4 as a4p, a4pp, b4p, b4pp and the values
    a4(x4), b4(x4) as a4v, b4v."""
    expression = expression.subs(sp.Derivative(a4, (x4, 2)), a4pp)  # second first
    expression = expression.subs(sp.Derivative(b4, (x4, 2)), b4pp)
    expression = expression.subs(sp.Derivative(a4, x4), a4p).subs(a4, a4v)
    return expression.subs(sp.Derivative(b4, x4), b4p).subs(b4, b4v)


def plain(expression):
    """As symbolic, and 6 H x0 written as z."""
    return symbolic(expression).subs(x0, z / (6 * H))


say("Record read; symbols x0 ... x7, H, a4(x4), b4(x4), a4p, a4pp, a4v, b4p, b4pp, "
    "b4v and z defined.")
```

`symbolic` replaces the second derivatives of a_4 and b_4 (written `sp.Derivative(a4, (x4, 2))` and `sp.Derivative(b4, (x4, 2))`) by `a4pp` and `b4pp`, then the first derivatives by `a4p` and `b4p`, then the functions themselves by `a4v` and `b4v`. The second derivative is replaced first, because replacing the first derivative first could also change the second one (which is the derivative of the first). `plain` does the same and also writes $6Hx_0$ as z. The last line prints the output of In [3].

In [4], the metric from the author's text.

```
# the metric exactly as the author typed it, without the note in brackets after it
text = record["metricAsGiven"].split("}} ", 1)[0] + "}}"
text = text.replace("exp^", "E^").replace("a4[x4]", "a4").replace("b4[x4]", "b4")
text = text.replace("Sin[6 H x0]", "sin(6 H x0)").replace("Cot[6 H x0]", "cot(6 H x0)")
text = text.replace("{", "[").replace("}", "]").replace("^", "**")
metric_names = {"E": sp.E, "a4": a4, "b4": b4, "H": H, "x0": x0, "sin": sp.sin,
                "cot": sp.cot}
g = sp.Matrix(parse_expr(text, local_dict=metric_names, transformations=(
    standard_transformations + (implicit_multiplication,))))
```

The first line cuts the record's text at the first `}}` followed by a blank, as Notebook 03a does (Section 3.13, In [3]): `[0]` keeps the matrix, without the note in brackets, and `+ "}}"` puts back its two closing braces. Then the same translation of the author's Mathematica text as in Notebook 03a (Section 3.13, In [5]), with two or three replacements per line, eight in all: `exp^` becomes `E^`, `a4[x4]` becomes `a4` and `b4[x4]` becomes `b4`, the square brackets of the sine and the cotangent become round ones, the braces of lists become square brackets and `^` becomes `**`. The dictionary `metric_names` says what each name means, and `parse_expr` with implicit multiplication reads the text; `sp.Matrix` makes the exact 8×8 matrix `g`.

```
for k in range(8):
    say(f"  g[{NAMES[k]}, {NAMES[k]}] = {plain(g[k, k])}")
check(all(g[i, j] == 0 for i in range(8) for j in range(8) if i != j),
      "the metric read from the text of the author is diagonal")
```

The eight diagonal entries are printed with `plain`, in the order $x_0, \ldots, x_7$, for example `exp(2*a4v)*sin(z)**(1/3)` for $e^{2a_4} \sin^{1/3} z$ and `-b4v*exp(-2*a4v)*sin(z)**(1/3)` for $-b_4 e^{-2a_4} \sin^{1/3} z$, and the check requires the 56 entries off the diagonal to be zero.

In [5], reading the record's components.

```
record_names = {"a4p": a4p, "a4pp": a4pp, "a4v": a4v, "b4p": b4p, "b4pp": b4pp,
                "b4v": b4v, "H": H, "x0": x0, "E": sp.E, "sin": sp.sin, "cot": sp.cot}


def from_mathematica(entry_text):
    """A component of the record (Mathematica notation) as a sympy expression."""
    t = entry_text.replace("Derivative[1][a4][x4]", "a4p")
    t = t.replace("Derivative[2][a4][x4]", "a4pp").replace("a4[x4]", "a4v")
    t = t.replace("Derivative[1][b4][x4]", "b4p")
    t = t.replace("Derivative[2][b4][x4]", "b4pp").replace("b4[x4]", "b4v")
    t = t.replace("Sin[6*H*x0]", "sin(6*H*x0)").replace("Cot[6*H*x0]", "cot(6*H*x0)")
    t = t.replace("^", "**")
```

```
    if "[" in t or "]" in t:
        raise ValueError(f"cannot translate {entry_text}")
    return parse_expr(t, local_dict=record_names)
```

The record writes a_4' as `Derivative[1][a4][x4]` and a_4'' as `Derivative[2][a4][x4]`, and the same for b_4. `from_mathematica` translates a component of the record into sympy: the two derivatives of a_4 become `a4p` and `a4pp` and `a4[x4]` becomes `a4v`, the same for b_4 (`b4p`, `b4pp`, `b4v`), the functions get round brackets, `^` becomes `**`. If a square bracket is left, some notation was not translated and the function stops with an error instead of misreading silently. The dictionary `record_names` says what the names in the translated text mean.

```
def tidy(expression):
    """Simplify an expression.  It writes 6 H x0 as z (so that 12 H x0 becomes 2z),
    simplifies it twice, once as it is and once after writing sin(2z) as
    2 sin(z) cos(z) (expand_trig), keeps the shorter result (count_ops counts the
    operations), and writes z as 6 H x0 again."""
    expression = sp.sympify(expression).subs(x0, z / (6 * H))
    first = sp.simplify(expression)
    second = sp.simplify(sp.expand_trig(expression))
    best = second if sp.count_ops(second) < sp.count_ops(first) else first
    return best.subs(z, 6 * H * x0)
```

`tidy` is the simplifier of the notebook. `sp.sympify` turns a plain Python number into a sympy number (so that `.subs` works on it). It writes $6Hx_0$ as z, simplifies once directly and once after `sp.expand_trig`, which writes $\sin 2z$ as $2\sin z\cos z$, and keeps the result with fewer operations (`sp.count_ops` counts them); sympy sometimes reaches the simplest form only one of the two ways. At the end z is written as $6Hx_0$ again. The line `best = second if ...  else first` chooses one of two values by a condition.

```
def vanishes(expression):
    """True when the expression simplifies to zero."""
    return tidy(expression) == 0


def same(mine, theirs):
    """True when mine (with the function a4) equals theirs (record symbols)."""
    return vanishes(symbolic(mine) - theirs)
```

`vanishes` is true when an expression simplifies to 0; `same` compares one of our expressions (written with the functions a_4 and b_4) with one of the record (written with `a4p`, `a4pp`, `a4v`, `b4p`, `b4pp`, `b4v`): it rewrites ours with `symbolic` and asks whether the difference vanishes.

```
example = next(e["value"] for e in record["christoffelNonzero_b_le_c"]
               if (e["a"], e["b"], e["c"]) == ("x4", "x5", "x5"))  # Gamma^x4_(x5 x5)
say(f"example of a record entry: {example}")
say(f"translated: {from_mathematica(example)}")
```

The entry of the record's Christoffel list with the indices `a = "x4"`, `b = "x5"`, `c = "x5"` serves as an example: `next(...  for e in ...  if ...)` runs through the list and takes the first entry

whose three indices, compared as one tuple, are these. The output shows it in the record's notation and then its translation:

```
example of a record entry: -(1/2)*E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*Derivative[1][b4][x4] +
    Derivative[1][a4][x4]*E^(-2*a4[x4])*Sin[6*H*x0]^(1/3)*b4[x4]
translated: a4p*b4v*exp(-2*a4v)*sin(6*H*x0)**(1/3) - b4p*exp(-2*a4v)*sin(6*H*x0)**(1/3)/2
```

This is the symbol $\Gamma^{x_4}{}_{x_5x_5} = (a_4'b_4 - \frac{1}{2}b_4')e^{-2a_4}\sin^{1/3} z$ of Section 3.23, the one in which both functions appear.

In [6], the Christoffel symbols.

```
def christoffel(metric, coordinates):
    """All Christoffel symbols Gamma[a][b][c] of a metric, by the formula above."""
    n = len(coordinates)
    inverse = metric.inv()  # the inverse metric g^(ad)
    d_metric = [metric.diff(c) for c in coordinates]  # d_metric[c][i, j] = d_c g_ij
    table = [[[0] * n for _ in range(n)] for _ in range(n)]
    for a, b, c in itertools.product(range(n), repeat=3):
        value = sum(inverse[a, d] * (d_metric[b][d, c] + d_metric[c][d, b]
                                     - d_metric[d][b, c]) for d in range(n)) / 2
        table[a][b][c] = tidy(value)
    return table
```

The same function as in Notebook 03c (Section 3.22, In [3]), with `tidy` instead of `sp.simplify`: the Christoffel formula of Section 3.15 applied to all $8^3 = 512$ index lists, without using the symmetry in b and c (so that the symmetry can be checked).

```
Gamma = christoffel(g, X)
nonzero = [(a, b, c) for a, b, c in itertools.product(range(8), repeat=3)
           if Gamma[a][b][c] != 0]
upper_half = [(a, b, c) for a, b, c in nonzero if b <= c]
report("non-zero Christoffel symbols (all orders of b, c)", len(nonzero))
report("non-zero Christoffel symbols with b <= c", len(upper_half))
check(all(Gamma[a][b][c] == Gamma[a][c][b]
          for a, b, c in itertools.product(range(8), repeat=3)),
      "the Christoffel symbols are symmetric in their two lower indices")
```

`Gamma` is the table of the author's metric. `nonzero` lists the index lists of the non-zero symbols and `upper_half` those with $b \le c$. The two RESULT lines print 37 and 25, the counts of Section 3.23, and the check confirms the symmetry $\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$ for all 512.

In [7], comparison with the record.

```
for a, b, c in upper_half:
    say(f"  Gamma^{NAMES[a]}_({NAMES[b]} {NAMES[c]}) = {plain(Gamma[a][b][c])}")
```

The 25 symbols are printed, in the order of their index lists (first Γ^{x_0}, last Γ^{x_7}): they are the table of Section 3.23, in sympy's way of writing. It writes $H\cot z$ as `H/tan(z)`, $-H\tan z\, e^{2a_4}\sin^{1/3} z$

as `-H*exp(2*a4v)*sin(z)**(4/3)/cos(z)` (because $\sin^{1/3} z \tan z = \sin^{4/3} z / \cos z$), $-6H(\tan z + \cot z) = -6H/(\sin z \cos z)$ as `-12*H/sin(2*z)` (because $\sin 2z = 2 \sin z \cos z$), and the extra-time rate as `-a4p + b4p/(2*b4v)`.

```
record_gamma = {(NAMES.index(e["a"]), NAMES.index(e["b"]), NAMES.index(e["c"])):
                from_mathematica(e["value"])
                for e in record["christoffelNonzero_b_le_c"]}
check(sorted(record_gamma) == upper_half and len(upper_half) == 25,
      "the same 25 non-zero Christoffel symbols as the record")
record_check(PYTHON_REPORT, "rust_christoffels_agree")  # stops if not passed
check(all(same(Gamma[a][b][c], value) for (a, b, c), value in record_gamma.items()),
      "every Christoffel symbol equals the record exactly",
      record=f"{CURVATURE_RECORD}, christoffelNonzero_b_le_c (and "
             "python-lovelock-report.json, check rust_christoffels_agree)")
```

The line `record_check(PYTHON_REPORT, "rust_christoffels_agree")` confirms that the sympy checker of the Revision record found every Christoffel symbol of the Rust program correct; the notebook stops there if that check had not passed.

`record_gamma` is a dictionary built from the record's list: the key is the index list (`NAMES.index("x4")` is 4, the position of a name in the list) and the value is the translated symbol. The first check requires the record and the notebook to name the same 25 index lists (`sorted` turns the keys into an ordered list); the second requires every value to agree exactly. The PASS line names the record and its independent verification.

In [8], Figure 03b.1: the Christoffel symbols as heat maps.

```
from matplotlib.colors import LinearSegmentedColormap, SymLogNorm

BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
MAGENTA, GREEN, VIOLET, RED = "#e87ba4", "#008300", "#4a3aa7", "#e34948"
DIVERGING = LinearSegmentedColormap.from_list("blue_grey_red", [BLUE, "#f0efec", RED])
LABELS = [f"$x_{k}$" for k in range(8)]
ARGS = (z, a4v, a4p, H, b4v, b4p)  # the arguments of the numerical functions
```

The colour palette and the blue-grey-red colour scale of Notebook 03a (Section 3.13, In [9]); `SymLogNorm` is a colour scale that is logarithmic on both sides of zero. `LABELS` are the axis labels x_0 to x_7. `ARGS` lists the six arguments of the numerical functions made next: z, a_4, a_4', H, b_4, b_4' (no Christoffel symbol contains a second derivative).

```
def numeric(expression):
    """A numerical function f(z, a4v, a4p, H, b4v, b4p) of an expression."""
    return sp.lambdify(ARGS, plain(expression), "numpy")


gamma_functions = {(a, b, c): numeric(Gamma[a][b][c]) for a, b, c in nonzero}
# z, a4, a4p, H, b4, b4p: the canonical slice a4 = 0.5 of the history, b4 = 1
POINT = (np.pi / 4, 0.5, 1.0, 1.0, 1.0, 0.0)
tables = np.zeros((8, 8, 8))
for (a, b, c), f in gamma_functions.items():
    tables[a, b, c] = f(*POINT)
```

`numeric` turns an exact expression into a fast numerical function (`sp.lambdify`, Section 3.13, In [8]). `gamma_functions` holds one such function for each of the 37 non-zero symbols. `POINT` is the point $z = \pi/4$ on the slice $a_4 = 0.5$ of the history $a_4 = x_4$, $b_4 = 1$ (so $a'_4 = 1$ and $b'_4 = 0$), with $H = 1$; `f(*POINT)` calls a function with the six numbers. The $8 \times 8 \times 8$ array `tables` holds the 512 numbers (zero where the symbol is zero).

```
fig, axes = plt.subplots(2, 4, figsize=(11.0, 5.4))
norm = SymLogNorm(linthresh=0.1, vmin=-15.0, vmax=15.0, base=10)
for a, ax in enumerate(axes.flat):
    image = ax.imshow(tables[a], cmap=DIVERGING, norm=norm)
    for b, c in itertools.product(range(8), repeat=2):
        if tables[a, b, c] != 0:
            ax.text(c, b, f"{tables[a, b, c]:.2f}", ha="center", va="center",
                    fontsize=6.0)
```

Eight panels in two rows of four, one for each upper index a (`axes.flat` runs through them in order). The colour scale is ordinary between -0.1 and 0.1 and logarithmic beyond, up to ± 15, so that small and large symbols are both visible. In each panel the 8×8 table ${\Gamma^a}_{bc}$ is drawn with row b and column c, and every non-zero value is written into its square with two decimals.

```
    ax.set_title(f"$\\Gamma^{{x_{a}}}{{}}_{{bc}}$", fontsize=10)
    ax.set_xticks(range(8), [str(k) for k in range(8)], fontsize=7)
    ax.set_yticks(range(8), [str(k) for k in range(8)], fontsize=7)
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.8, label="value (unit $H$)")
fig.suptitle("Christoffel symbols at $z = \\pi/4$, $a_4 = 0.5$, "
             "$a_4^{\\prime} = 1$, $b_4 = 1$, $H = 1$ (row $b$, column $c$)")
```

Each panel gets the title ${\Gamma^{x_a}}_{bc}$ (in an f-string a doubled brace `{{` prints one brace, so that the braces of the math notation survive) and the tick labels 0 to 7, the indices of x_0 to x_7. One colour bar serves all panels; `suptitle` is the title of the whole figure.

```
save_figure(fig, "christoffel_heat_maps",
            "The 512 Christoffel symbols $\\Gamma^{a}{}_{bc}$ of the author's metric "
...
check(np.allclose(tables, tables.transpose(0, 2, 1))
      and all(4 in key or 0 in key for key in nonzero),
      "the tables are symmetric in b, c; every non-zero symbol has an index x4 or x0")
```

The figure is saved. The check: `tables.transpose(0, 2, 1)` exchanges the second and the third index, so the first condition is ${\Gamma^a}_{bc} = {\Gamma^a}_{cb}$ as numbers; the second requires every non-zero index list to contain position 4 (x_4) or position 0 (x_0).

What Figure 03b.1 shows. At $b_4 = 1$, only 37 of the 512 squares are coloured. In the panels of 3-space (Γ^{x_1}, Γ^{x_2}, Γ^{x_3}) the value 1.00 appears four times, at the positions (x_k, x_4), (x_4, x_k), (x_k, x_0), (x_0, x_k): here both $a'_4 = 1$ and $H \cot(\pi/4) = 1$. In the panels of the extra times the squares with x_4 are blue, -1.00: the deflation. The panel Γ^{x_4} has 2.42 on the 3-space diagonal and 0.33 on the extra-time diagonal; the panel Γ^{x_0} has -2.42, 0.33 and the largest value, ${\Gamma^{x_0}}_{x_0 x_0} = -12$. Each panel is symmetric about its diagonal.

In [9], Figure 03b.2: the symbols across the patch.

```
z_line = np.linspace(0.01, np.pi / 2 - 0.01, 500)
chosen = [((1, 1, 0), RED, "-", "$\\Gamma^{x_1}{}_{x_1 x_0}$"),
          ((0, 0, 0), VIOLET, "--", "$\\Gamma^{x_0}{}_{x_0 x_0}$"),
          ((0, 1, 1), ORANGE, "-.", "$\\Gamma^{x_0}{}_{x_1 x_1}$"),
          ((0, 5, 5), BLUE, ":", "$\\Gamma^{x_0}{}_{x_5 x_5}$"),
          ((1, 1, 4), AQUA, "-", "$\\Gamma^{x_1}{}_{x_1 x_4}$")]
fig, ax = plt.subplots()
```

500 values of z inside the patch, and five chosen symbols, each with its index list, colour, line style and label.

```
for index, colour, style, label in chosen:
    values = np.broadcast_to(gamma_functions[index](z_line, 0.5, 1.0, 1.0, 1.0,
                                                    0.0), z_line.shape)
    sign = "+" if values[len(values) // 2] > 0 else "minus"
    ax.plot(z_line, np.abs(values), color=colour, ls=style, lw=1.8,
            label=f"{label} (sign {sign})")
```

For each symbol the values at the 500 points ($a_4 = 0.5$, $a_4' = 1$, $H = 1$, $b_4 = 1$, $b_4' = 0$); `np.broadcast_to` repeats a constant (the symbol a_4' does not depend on z, and its function returns one number). The sign is read at the middle point (`//` divides and rounds down), and the absolute value is drawn, because a logarithmic axis can only show positive numbers.

```
ax.set_yscale("log")
ax.set_xlabel("$z = 6 H x_0$ (radians)")
ax.set_ylabel("absolute value of the symbol (unit $H$)")
ax.set_title("Christoffel symbols across the patch "
             "($a_4 = 0.5$, $a_4^{\\prime} = 1$, $b_4 = 1$, $H = 1$)")
ax.set_ylim(1e-5, 3e3)  # room below the curves for the legend
ax.legend(fontsize=8, loc="lower center", ncol=2)
save_figure(fig, "christoffel_versus_z",
            "The absolute values of five Christoffel symbols of the author's metric "
...
check(abs(gamma_functions[(1, 1, 0)](np.pi / 4, 0.5, 1.0, 1.0, 1.0, 0.0) - 1.0)
      < 1e-14,
      "Gamma^x1_(x1 x0) = H cot z equals 1 at z = pi/4, H = 1")
```

A logarithmic vertical axis, labels, title, the range of the axis and the legend; the figure is saved; the check confirms $H \cot(\pi/4) = 1$.

What Figure 03b.2 shows. ${\Gamma^{x_1}}_{x_1 x_0} = H \cot z$ (red) falls from about 100 near the tip to almost 0 at the patch end; ${\Gamma^{x_0}}_{x_0 x_0}$ (violet dashed) is large at both ends and smallest, 12, at $z = \pi/4$; ${\Gamma^{x_0}}_{x_1 x_1}$ and ${\Gamma^{x_0}}_{x_5 x_5}$ grow towards the patch end, where $\tan z$ grows without bound; ${\Gamma^{x_1}}_{x_1 x_4} = a_4' = 1$ (aqua) is flat. Large Christoffel symbols belong to the coordinates; whether the geometry itself is extreme is decided by an invariant, the Kretschmann scalar of In [22].

In [10], a second check by finite differences.

```
prime = chr(39)  # the apostrophe (character number 39): the record writes a4 prime so
point_text = (f"H = 0.23, a4 = 0.17, a4{prime} = 0.61, a4{prime}{prime} = -0.37, "
```

```
            f"b4 = 1.37, b4{prime} = 0.31, b4{prime}{prime} = -0.23, "
            "x0 = 0.41")  # the test point as the record writes it
check(record_check(RUST_REPORT, "k1_brute_force_numeric", point_text),
      "the test point is the one of the brute-force check of the Rust program")
H_n, a0, a1, a2, x0_n = 0.23, 0.17, 0.61, -0.37, 0.41  # the test point
b0, b1, b2 = 1.37, 0.31, -0.23  # b4, b4' and b4'' at x4 = 0
```

The Rust program's check `k1_brute_force_numeric` names its test point in a text that writes $H = 0.23$, $a_4 = 0.17$, $a_4' = 0.61$, $a_4'' = -0.37$, $b_4 = 1.37$, $b_4' = 0.31$, $b_4'' = -0.23$ and $x_0 = 0.41$, with one apostrophe for each prime (the line `point_text` builds exactly this text). `chr(39)` is the character number 39, the apostrophe, and the f-string puts it after `a4` and `b4` once for the first and twice for the second derivative (the comment at the end of the line says so; the book prints a straight apostrophe in code as a curly one, and the character number makes plain which character is meant). `record_check(RUST_REPORT, "k1_brute_force_numeric", point_text)` (In [3]) confirms that this check of the record passed and that its detail contains exactly this text; `check` prints the PASS line. The eight numbers are then named: `a0`, `a1`, `a2` for a_4 and its two derivatives and `b0`, `b1`, `b2` for b_4 and its two derivatives at $x_4 = 0$.

```
def metric_numbers(x):
    """The metric as an 8 x 8 numpy table at the point x (8 coordinates)."""
    a = a0 + a1 * x[4] + 0.5 * a2 * x[4] ** 2  # a4(x4) with a4(0) = 0.17, ...
    b = b0 + b1 * x[4] + 0.5 * b2 * x[4] ** 2  # b4(x4) with b4(0) = 1.37, ...
    warp_n = np.sin(6 * H_n * x[0]) ** (1 / 3)
    # the author's order x0, x1, ..., x7
    return np.diag([1 / np.tan(6 * H_n * x[0]) ** 2] + [np.exp(2 * a) * warp_n] * 3
                   + [-1.0] + [-b * np.exp(-2 * a) * warp_n] * 3)
```

The metric as a plain table of numbers at a point `x` (a list of eight coordinates in the author's order, `x[4]` is x_4 and `x[0]` is x_0), for the concrete functions $a_4(x_4) = 0.17 + 0.61x_4 - 0.185x_4^2$ and $b_4(x_4) = 1.37 + 0.31x_4 - 0.115x_4^2$, which have at $x_4 = 0$ the values, slopes and second derivatives of the test point. `np.diag` makes the diagonal matrix, with the hidden entry first.

```
def christoffel_numbers(x, h):
    """All 512 Christoffel symbols at x from central differences with step h."""
    d_metric = np.zeros((8, 8, 8))  # d_metric[c] = derivative of g along x_c
    for c in range(8):
        step = np.zeros(8)
        step[c] = h
        d_metric[c] = (metric_numbers(x + step) - metric_numbers(x - step)) / (2 * h)
```

The derivatives of the metric without any algebra: for each coordinate c the point is moved by $\pm h$ along x_c (`step` is zero except at position c), and the **central difference** $(g(x+he_c) - g(x-he_c))/(2h)$ approximates $\partial_c g$ (Chapter 2).

```
    inverse = np.linalg.inv(metric_numbers(x))
    table = np.zeros((8, 8, 8))
    for a, b, c in itertools.product(range(8), repeat=3):
        table[a, b, c] = 0.5 * sum(inverse[a, d] * (d_metric[b][d, c]
                                   + d_metric[c][d, b] - d_metric[d][b, c])
```

```
                                    for d in range(8))
    return table
```

The inverse metric as numbers (`np.linalg.inv`) and the Christoffel formula of Section 3.15, now with numbers.

```
x_point = np.array([x0_n, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])  # x0 = 0.41, x4 = 0
exact = np.zeros((8, 8, 8))
for (a, b, c), f in gamma_functions.items():
    exact[a, b, c] = f(6 * H_n * x0_n, a0, a1, H_n, b0, b1)
steps = 10.0 ** -np.arange(1, 9)  # h = 0.1, 0.01, ..., 1e-8
errors = np.array([np.max(np.abs(christoffel_numbers(x_point, h) - exact))
                   for h in steps])
for h, e in zip(steps, errors):
    say(f"  h = {h:.0e}: largest error of the 512 symbols = {e:.2e}")
```

The test point in the author's order (only $x_0 = 0.41$, the first entry, matters, with $x_4 = 0$); the exact symbols there from the functions of In [8], with $z = 6 \cdot 0.23 \cdot 0.41$, $a_4 = 0.17$, $a_4' = 0.61$, $H = 0.23$, $b_4 = 1.37$, $b_4' = 0.31$; eight steps $h = 10^{-1}, \ldots, 10^{-8}$ (`10.0 ** -np.arange(1, 9)` raises 10 to the powers -1 to -8); for each the largest difference between the 512 numerical and exact symbols. The eight output lines show the errors 4.01×10^{-1}, 3.66×10^{-3}, 3.65×10^{-5}, 3.65×10^{-7}, 3.66×10^{-9}, 5.67×10^{-11}, 8.46×10^{-10}, 9.00×10^{-9}.

```
order = np.log10(errors[0] / errors[1])  # the slope between h = 0.1 and h = 0.01
report("measured order of the error between h = 0.1 and h = 0.01", f"{order:.3f}")
check(1.9 < order < 2.1 and errors.min() < 1e-8,
      "finite differences reproduce all 512 symbols; the error falls like h^2")
```

When h shrinks tenfold, an error Ch^p shrinks by 10^p, so $p = \log_{10}$ of the ratio of the errors. The RESULT line prints 2.040: the central difference has order 2 (Chapter 2). The check requires the order between 1.9 and 2.1 and the best error below 10^{-8}.

In [11], Figure 03b.3.

```
fig, ax = plt.subplots()
ax.loglog(steps, errors, "o-", color=BLUE, lw=1.8, ms=7,
          label="largest error of the 512 symbols")
ax.loglog(steps, errors[0] * (steps / steps[0]) ** 2, "--", color="black", lw=1.2,
          label="slope 2: error proportional to $h^2$")
ax.set_xlabel("step $h$ of the central difference")
ax.set_ylabel("largest absolute error (unit $H$)")
ax.set_title("Christoffel symbols by finite differences")
ax.legend(fontsize=8)
```

`ax.loglog` draws with logarithmic scales on both axes: the errors as dots joined by lines, and a reference line proportional to h^2 that starts at the first error. Labels, title, legend.

```
best = int(np.argmin(errors))  # the position of the smallest error in the list
report("best step and its error", f"h = {steps[best]:.0e}, error {errors[best]:.1e}")
```

```
mantissa, exponent = f"{errors[best]:.1e}".split("e")  # e.g. "3.2" and "-10"
best_error = f"{mantissa} \\times 10^{{{int(exponent)}}}"  # 3.2 x 10^-10 in LaTeX
best_step = f"10^{{{round(np.log10(steps[best]))}}}"  # the step as a power of 10
```

`np.argmin` gives the position of the smallest error; the RESULT line prints h = 1e-06, error 5.7e-11. The next three lines write the best error and step in math notation for the caption: the text `5.7e-11` is split at the letter e into `5.7` and `-11`, and the caption receives 5.7×10^{-11} and 10^{-6} (a tripled brace in an f-string gives one literal brace around the inserted value).

```
save_figure(fig, "finite_differences",
            "The largest difference between the 512 Christoffel symbols computed "
...
check(errors[0] > errors[1] > errors[2] > errors[3],
      "the error shrinks as the step shrinks from 0.1 to 0.0001")
```

The figure is saved; its caption is assembled from pieces, two of which are the f-strings just made. The check requires the error to decrease over the first four steps.

What Figure 03b.3 shows. From $h = 10^{-1}$ to $h = 10^{-6}$ the blue dots lie on the dashed line of slope 2: each tenfold smaller step gives a hundredfold smaller error. Below $h = 10^{-6}$ the error grows again: the rounding errors of the computer, about 10^{-16} of each number, are divided by h in the difference quotient. The best step, 10^{-6}, gives 5.7×10^{-11}. Numbers computed without any algebra agree with the exact symbols, also for the symbols that contain b_4 and b_4'.

In [12], free fall along x_4 and the contracted symbols.

```
check(all(Gamma[a][4][4] == 0 for a in range(8)),
      "Gamma^a_(x4 x4) = 0: observers at rest fall freely, x4 is their proper time")
volume = b4 ** sp.Rational(3, 2) * sp.cos(6 * H * x0)  # sqrt|det g|
contracted = [vanishes(sum(Gamma[a][a][b] for a in range(8))
                       - sp.diff(sp.log(volume), X[b]))
              for b in range(8)]
check(all(contracted),
      "sum over a of Gamma^a_(a b) equals the derivative of ln(b4^(3/2) cos z)")
```

The first check: the eight symbols ${\Gamma^a}_{x_4 x_4}$ (lower positions 4 and 4) are zero, so observers at rest fall freely (Section 3.23). The second: for each b, $\sum_a {\Gamma^a}_{ab} - \partial_b \ln(b_4^{3/2} \cos z)$ vanishes (Sections 3.15 and 3.23); `X[b]` is the coordinate x_b.

In [13], the Riemann tensor.

```
def riemann(table, coordinates):
    """The non-zero R^a_bcd (MTW) as a dictionary {(a, b, c, d): value}."""
    n = len(coordinates)
    result = {}
    for a, b in itertools.product(range(n), repeat=2):
        for c, d in itertools.combinations(range(n), 2):  # all pairs c < d
            value = (sp.diff(table[a][b][d], coordinates[c])
                     - sp.diff(table[a][b][c], coordinates[d])
                     + sum(table[a][c][e] * table[e][b][d]
```

```
                        - table[a][d][e] * table[e][b][c] for e in range(n)))
            value = tidy(value)
            if value != 0:
                result[(a, b, c, d)] = value
                result[(a, b, d, c)] = -value  # antisymmetric in c and d
    return result
```

The formula of Section 3.17, as in Notebook 03c, but with less work: `itertools.combinations(range(n), 2)` gives only the pairs $c < d$ (28 of them); the component with c and d exchanged is stored with the opposite sign (symmetry (S1)), and those with $c = d$ are zero. Non-zero values are kept in a dictionary.

```
def raise_second(riemann_table, metric):
    """R^ab_cd = sum over e of g^(be) R^a_ecd, the non-zero ones."""
    inverse = metric.inv()
    result = {}
    for (a, e, c, d), value in riemann_table.items():
        for b in range(metric.rows):
            if inverse[b, e] != 0:
                result[(a, b, c, d)] = result.get((a, b, c, d), 0) \
                    + inverse[b, e] * value
    result = {k: tidy(v) for k, v in result.items()}
    return {k: v for k, v in result.items() if v != 0}
```

The same as in Notebook 03c, except that products with a zero entry of the inverse metric are skipped (for a diagonal metric only $b = e$ remains), and the results are simplified with `tidy`.

```
R_down = riemann(Gamma, X)  # R^a_bcd
R_mixed = raise_second(R_down, g)  # R^ab_cd
report("non-zero components R^a_bcd", len(R_down))
report("non-zero components R^ab_cd", len(R_mixed))
record_check(RUST_REPORT, "riemann_antisymmetry", "156 nonzero entries")
check(len(R_mixed) == 156, "R^ab_cd has 156 non-zero components, as in the record",
      record="Revision/gkd_lovelock/results/lovelock-report.json, check "
             "riemann_antisymmetry (156 nonzero entries)")
```

The call of `record_check` with `RUST_REPORT`, the check name `riemann_antisymmetry` and the text `156 nonzero entries` confirms that the Rust program's own check passed and that its detail text reports the same 156 non-zero components.

The two forms of the tensor for the author's metric; both have 156 non-zero components (two RESULT lines), the number found by hand in Section 3.24 and by the Rust program. This cell takes a few seconds.

In [14], the symmetries.

```
def get(table, key):
    return table.get(key, 0)
```

```
antisymmetric = all(vanishes(v + get(R_mixed, (b, a, c, d)))
                    and vanishes(v + get(R_mixed, (a, b, d, c)))
                    for (a, b, c, d), v in R_mixed.items())
```

`get` returns a component, or 0 if it is absent. `antisymmetric` is true when every component plus the component with the upper pair exchanged vanishes, and the same for the lower pair: $R^{ab}{}_{cd} = -R^{ba}{}_{cd} = -R^{ab}{}_{dc}$.

```
lowered = {(a, b, c, d): g[a, a] * g[b, b] * v
          for (a, b, c, d), v in R_mixed.items()}
pair_symmetric = all(vanishes(v - get(lowered, (c, d, a, b)))
                     for (a, b, c, d), v in lowered.items())
record_check(PYTHON_REPORT, "riemann_antisymmetry_and_pair_symmetry")
check(antisymmetric and pair_symmetric,
      "R^ab_cd is antisymmetric in a, b and in c, d; R_abcd = R_cdab",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "riemann_antisymmetry_and_pair_symmetry")
```

`record_check(PYTHON_REPORT, "riemann_antisymmetry_and_pair_symmetry")` confirms that the sympy checker's check of the same symmetries passed.

All indices lowered, $R_{abcd} = g_{aa} g_{bb} R^{ab}{}_{cd}$ (Section 3.17), and the pair symmetry $R_{abcd} = R_{cdab}$ (S4) checked for every component. One check for both, naming the record's check.

```
bianchi = [vanishes(get(R_down, (a, b, c, d)) + get(R_down, (a, c, d, b))
                    + get(R_down, (a, d, b, c)))
           for a, b, c, d in itertools.product(range(8), repeat=4)]
record_check(RUST_REPORT, "riemann_first_bianchi")
check(all(bianchi) and len(bianchi) == 4096,
      "the first Bianchi identity holds for all 4096 index lists",
      record="Revision/gkd_lovelock/results/lovelock-report.json, check "
             "riemann_first_bianchi")
```

`record_check(RUST_REPORT, "riemann_first_bianchi")` confirms that the Rust program's check of the same identity passed.

The first Bianchi identity (S2) for all $8^4 = 4096$ index lists.

```
warp_free = all(not symbolic(v).has(a4v) and not any(
    isinstance(p, sp.Pow) and p.base == sp.sin(6 * H * x0) and not p.exp.is_integer
    for p in sp.preorder_traversal(symbolic(v))) for v in R_mixed.values())
record_check(RUST_REPORT, "mixed_riemann_free_of_sin_third")
record_check(PYTHON_REPORT, "mixed_riemann_free_of_warp_and_exponential")
check(warp_free, "no R^ab_cd contains sin(z)^(1/3) or e^(a4): the warp cancels",
      record="Revision/gkd_lovelock/results/lovelock-report.json, check "
             "mixed_riemann_free_of_sin_third, and python-lovelock-report.json, "
             "check mixed_riemann_free_of_warp_and_exponential")
```

The two `record_check` lines confirm that both records' checks of the same two properties passed: the Rust program's `mixed_riemann_free_of_sin_third` and the sympy checker's `mixed_riemann_free_of_warp_and_exponential`.

`sp.preorder_traversal` walks through every piece of an expression (its sums, products, powers and their parts). The condition says: no component contains the value a_4 (only its derivatives), and no piece is a power of $\sin z$ with an exponent that is not a whole number (`p.base` and `p.exp` are the base and the exponent of a power `p`; `isinstance(p, sp.Pow)` asks whether `p` is a power). So the warp $\sin^{1/3} z$ and the exponentials $e^{\pm 2a_4}$ of the metric entries cancel in every $R^{ab}{}_{cd}$, as found by hand in Section 3.24; b_4 is left only in the ratios b_4'/b_4 and b_4''/b_4 of the rate α_t and its derivative.

In [15], comparison with the record's 156 components.

```
record_riemann = {}
for entry in record["riemannMixedNonzero"]:
    key = tuple(NAMES.index(n) for n in entry["up"] + entry["down"])
    record_riemann[key] = from_mathematica(entry["value"])
check(sorted(record_riemann) == sorted(R_mixed),
      "the same 156 non-zero components R^ab_cd as the record")
record_check(PYTHON_REPORT, "rust_riemann_agrees")
check(all(same(R_mixed[k], v) for k, v in record_riemann.items()),
      "every component R^ab_cd equals the record exactly",
      record=f"{CURVATURE_RECORD}, riemannMixedNonzero (and "
             "python-lovelock-report.json, check rust_riemann_agrees)")
```

`record_check(PYTHON_REPORT, "rust_riemann_agrees")` confirms that the sympy checker of the Revision record found all 156 components of the Rust program correct.

Each entry of the record's list `riemannMixedNonzero` has the upper indices `up`, the lower indices `down` and the value; `entry["up"] + entry["down"]` joins the two lists of names, and `tuple(...)` turns their positions into a key. Two checks: the same 156 index lists, and every value equal exactly.

```
groups = {}
for key in sorted(R_mixed):
    # the value multiplied out: one value written in two ways counts once
    groups.setdefault(sp.sstr(sp.expand(plain(R_mixed[key]))), []).append(key)
for value_text in sorted(groups):
    a, b, c, d = groups[value_text][0]
    say(f"  {len(groups[value_text]):2d} x  {value_text:30s}  e.g. "
        f"R^({NAMES[a]} {NAMES[b]})_({NAMES[c]} {NAMES[d]})")
report("different values among the 156 components", len(groups))
```

The components are grouped by their value, multiplied out (`sp.expand`) and written as text (`sp.sstr`): multiplying out first gives every value one standard form, so that a value that sympy happens to write in two ways is counted once (without it, sympy writes $\pm(\alpha_t^2 + \alpha_t')$ in two different ways and the count would be 22). `setdefault` starts an empty list for a new value, and `append` adds the index list. For each of the 20 different values the number of components, the value (padded to 30 characters by `:30s`) and one example are printed. The output is the result of Section 3.24 in symbols (with `/tan(z)` for $\cot z$): the plane values $\pm(\alpha^2 - H^2)$, $\pm(\alpha_t^2 - H^2)$, $\pm(\alpha^2 + \alpha')$ and $\pm(\alpha_t^2 + \alpha_t')$ 6 times each, $\pm(\alpha\alpha_t - H^2)$ 18 times each, $\pm H^2$ 12 times each, and the 48 components $\pm H\alpha \cot z$, $\pm H\alpha \tan z$, $\pm H\alpha_t \cot z$ and $\pm H\alpha_t \tan z$, 6 times each.

In [16], Figure 03b.4: the curvature of every coordinate plane.

```
plane = {(a, b): plain(sp.S(get(R_mixed, (a, b, a, b))))  # sp.S: 0 as a sympy 0
         for a, b in itertools.product(range(8), repeat=2) if a != b}
alpha_t = -a4p + b4p / (2 * b4v)  # the expansion rate of the extra times
alpha_t_prime = -a4pp + b4pp / (2 * b4v) - b4p ** 2 / (2 * b4v ** 2)  # its derivative
# the planes (row, column) in the author's order x0, x1, ..., x7
expected = {(1, 2): a4p ** 2 - H ** 2, (5, 6): alpha_t ** 2 - H ** 2,
            (1, 5): a4p * alpha_t - H ** 2, (1, 4): a4p ** 2 + a4pp,
            (4, 5): alpha_t ** 2 + alpha_t_prime, (1, 0): -H ** 2, (5, 0): -H ** 2,
            (4, 0): 0}
for (a, b), formula in expected.items():
    say(f"  plane ({NAMES[a]}, {NAMES[b]}): R^ab_ab = {plane[(a, b)]}")
```

`plane` holds the curvature $\sigma(a,b) = R^{ab}{}_{ab}$ of all 56 ordered pairs of different coordinates (`sp.S` turns the Python number 0 into a sympy zero, which understands `.has` below). `alpha_t` and `alpha_t_prime` are the extra-time rate $\alpha_t = -a_4' + b_4'/(2b_4)$ and its derivative α_t' (Section 3.24, case (5)), written with the printing symbols. `expected` lists one plane of each of the seven kinds of Section 3.24 (two for the transverse-hidden kind) with its formula, the pairs given by positions in the author's order (`(1, 2)` is the plane of x_1 and x_2, `(4, 0)` that of x_4 and x_0); the loop prints them. The output shows the formulas written out with a_4 and b_4, for example `-H**2 + a4p**2 - a4p*b4p/b4v + b4p**2/(4*b4v**2)` for $\alpha_t^2 - H^2$.

```
check(all(sp.expand(plane[key] - formula) == 0 for key, formula in expected.items())
      and all(not v.has(z) and not v.has(a4v) for v in plane.values()),
      "the plane curvatures have the seven formulas and do not depend on z or a4")
```

The check: the printed planes have the formulas of Section 3.24 (`sp.expand` multiplies out, so equal polynomials give a zero difference), and no plane curvature depends on z or on the value of a_4.

```
sixth = sp.sin(6 * H * x0) ** sp.Rational(1, 6)  # sin(z)^(1/6)
apart = {1: sp.exp(a4) * sixth, 5: sp.sqrt(b4) * sp.exp(-a4) * sixth}  # h1, h5
growth = {}  # (d^2 xi/d tau^2)/xi for observers at rest apart along x1 or x5
for a, h_a in apart.items():  # xi = h_a times a fixed coordinate distance, tau = x4
    growth[a] = sp.expand(plain(sp.simplify(sp.diff(h_a, x4, 2) / h_a)))
    say(f"  at rest, apart along {NAMES[a]}: (d^2 xi/d tau^2)/xi = {growth[a]}")
```

The meaning of the sign, computed (Section 3.24, "What the signs mean"). `apart` holds the scale factors $h_1 = e^{a_4}\sin^{1/6} z$ and $h_5 = b_4^{1/2} e^{-a_4}\sin^{1/6} z$ of x_1 and x_5, with the functions $a_4(x_4)$ and $b_4(x_4)$ themselves, so that sympy can differentiate them. Two observers at rest a fixed coordinate distance apart along x_a are the proper distance $\xi = h_a\,\Delta x_a$ apart, and their proper time is $\tau = x_4$; so $(d^2\xi/d\tau^2)/\xi = (\partial_4^2 h_a)/h_a$, which `sp.diff(h_a, x4, 2) / h_a` computes (the second derivative with respect to x_4, divided by h_a). `plain` writes the result with `a4p`, `a4pp`, `b4p`, `b4pp` and `b4v`. The output shows `a4p**2 + a4pp` for x_1, that is $\alpha^2 + \alpha'$, and for x_5 the long expression $\alpha_t^2 + \alpha_t'$ written out.

```
check(sp.simplify(apart[1] ** 2 - g[1, 1]) == 0
      and sp.simplify(apart[5] ** 2 + g[5, 5]) == 0
      and all(sp.simplify(growth[a] - plane[(a, 4)]) == 0 for a in apart),
      "observers at rest: d^2 xi/d tau^2 = +R^ab_ab xi in the planes (x1, x4) and "
      "(x5, x4)")
```

The check has three parts: the two scale factors are the right ones ($h_1^2 = g_{11}$ and $h_5^2 = -g_{55}$), and for both directions $(d^2\xi/d\tau^2)/\xi$ equals the plane curvature `plane[(a, 4)]` $= \sigma(x_a, x_4)$ (position 4 is x_4). So $d^2\xi/d\tau^2 = +\sigma\,\xi$: a positive curvature of a plane with the time drives observers at rest apart, the opposite of the sphere.

```
SLOPES = (0.5, 1.0, 2.0)  # three deflating histories, A > 0
fig, axes = plt.subplots(1, 3, figsize=(14.0, 4.8))  # wide: room for -1.25
for ax, slope in zip(axes, SLOPES):
    curv = np.full((8, 8), np.nan)  # NaN: an empty square on the diagonal
    for (a, b), value in plane.items():
        curv[a, b] = float(value.subs({a4p: slope, a4pp: 0, H: 1, b4v: 1, b4p: 0,
                                       b4pp: 0}))
    image = ax.imshow(curv, cmap=DIVERGING, vmin=-5.0, vmax=5.0)
```

Three panels for the slopes $A = 0.5$, 1 and 2 of the history $a_4 = AHx_4$, $b_4 = 1$, with $H = 1$, $a_4' = A$, $a_4'' = 0$ and $b_4' = b_4'' = 0$ (the dictionary in `subs` sets all six symbols). `np.full((8, 8), np.nan)` is an 8×8 table of NaN ("not a number"), which matplotlib leaves blank: the diagonal stays empty. The other squares receive the plane curvatures, and the table is drawn with the colour scale from -5 to 5.

```
    for a, b in plane:
        ax.text(b, a, f"{curv[a, b]:g}", ha="center", va="center", fontsize=6.5)
    ax.set_xticks(range(8), LABELS, fontsize=8)
    ax.set_yticks(range(8), LABELS, fontsize=8)
    ax.grid(False)
    ax.set_title(f"$A = {slope:g}$")
fig.colorbar(image, ax=axes, shrink=0.85, label="curvature (unit $H^2$)")
```

The value written into each square (`:g` writes a number without needless digits), the labels x_0 to x_7, the title with the slope, and one colour bar.

```
save_figure(fig, "plane_curvatures",
            "The curvature $\\sigma(a, b) = R^{ab}{}_{ab}$ (no sum) of the plane of "
...
check([float(plane[(1, 2)].subs({a4p: s, H: 1})) for s in SLOPES] == [-0.75, 0.0, 3.0],
      "the planes inside 3-space: -0.75, 0 and 3 for A = 0.5, 1 and 2")
```

The figure is saved; the check confirms $a_4'^2 - H^2 = -0.75$, 0, 3 for the three slopes.

What Figure 03b.4 shows. Three symmetric tables, with x_0 in the first row and column. The blocks inside 3-space and inside the extra times change from pale blue (-0.75 at $A = 0.5$) through grey (0 at $A = 1$) to red (3 at $A = 2$). The blocks that mix 3-space with the extra times are always blue and deepen: -1.25, -2, -5. The row and column of x_4 are red, A^2: 0.25, 1, 4. The row and column of x_0 are -1 everywhere, except the flat plane of x_4 and x_0, which is 0. The colours give signs, not shapes: only the squares of two of x_1, x_2, x_3 and those of one of them with x_0 belong to planes of two space-like directions, where red means curved like a sphere and blue like a saddle (so the 3-space planes are saddle-like at $A = 0.5$ and sphere-like at $A = 2$). In the planes of two extra times the shape reading is reversed, and the red row and column of x_4 mean, by In [16]'s check, that

observers at rest drift apart ever faster along 3-space and that their extra-time separation shrinks ever more slowly (Section 3.24).

In [17], Ricci, Ricci scalar and Einstein tensor.

```
def ricci(mixed_table, n):
    """R^a_b = sum over c of R^ac_bc, as an n x n sympy matrix."""
    result = sp.zeros(n, n)
    for (a, c, b, d), value in mixed_table.items():
        if c == d:
            result[a, b] += value
    return result.applyfunc(tidy)
```

The function of Notebook 03c (Section 3.22, In [3]), with `tidy` as the simplifier.

```
Ric = ricci(R_mixed, 8)
R_scalar = tidy(Ric.trace())
G = (Ric - R_scalar / 2 * sp.eye(8)).applyfunc(tidy)
for k in range(8):
    say(f"  {NAMES[k]}: R^a_a = {sp.sstr(plain(Ric[k, k])):18s} "
        f"G^a_a = {plain(G[k, k])}")
say(f"Ricci scalar R = {plain(R_scalar)}")
```

The Ricci tensor, its trace (the Ricci scalar) and the Einstein tensor $G = \mathrm{Ric} - \frac{R}{2}\mathbb{1}$ (`sp.eye(8)` is the 8×8 unit matrix). The loop prints the diagonal of both tensors (`:18s` pads the text to at least 18 characters, so that short entries line up), and the last line the scalar. The output is the result of Section 3.25, written out with a_4 and b_4: $R^a{}_a = -6H^2$ for x_0, $-6H^2 + \frac{3}{2}a_4'b_4'/b_4 + a_4''$ for 3-space, the long entry for x_4, and $-6H^2 - \frac{3}{2}a_4'b_4'/b_4 - a_4'' + \frac{1}{4}b_4'^2/b_4^2 + \frac{1}{2}b_4''/b_4$ for the extra times; $G^4{}_4 = 21H^2 + 3a_4'^2 - \frac{3}{2}a_4'b_4'/b_4 - \frac{3}{4}b_4'^2/b_4^2$; $R = -42H^2 + 6a_4'^2 - 3a_4'b_4'/b_4 + 3b_4''/b_4$. For a constant b_4 these are $a_4'' - 6H^2$, $6a_4'^2$, $-a_4'' - 6H^2$, $3a_4'^2 + 21H^2$ and $6a_4'^2 - 42H^2$.

```
ricci_ok = all(same(Ric[i, j], from_mathematica(
    record["ricciMixed"][f"{NAMES[i]},{NAMES[j]}"]["mathematica"]))
    for i in range(8) for j in range(8))
einstein_ok = all(same(G[i, j], from_mathematica(
    record["einsteinMixed"][f"{NAMES[i]},{NAMES[j]}"]["mathematica"]))
    for i in range(8) for j in range(8))
scalar_ok = same(R_scalar, from_mathematica(record["ricciScalar"]))
record_check(PYTHON_REPORT, "rust_ricci_einstein_scalar_agree")  # stops if not
check(ricci_ok and einstein_ok and scalar_ok,
      "all 64 Ricci, all 64 Einstein components and R equal the record",
      record=f"{CURVATURE_RECORD}, ricciMixed, einsteinMixed, ricciScalar (and "
             "python-lovelock-report.json, check rust_ricci_einstein_scalar_agree)")
```

The record stores each of the 64 components under a key such as `"x1,x4"`, with its Mathematica text under `"mathematica"`. `record_check` (In [3]) stops the notebook unless the independent verification of the record (`PYTHON_REPORT`, named in In [3]) lists its check `rust_ricci_einstein_scalar_agree` as passed. All 64 Ricci and all 64 Einstein components and the scalar are compared exactly, in one check.

In [18], the first Lovelock scalar of the record of the field equations.

```
A4_REPORT = "Revision/field_equations_a4/reports/python-a4-report.json"
a4_report = json.loads(repository_file(A4_REPORT).read_text(encoding="utf-8"))
# the report's checks form a list of dictionaries; take the one with this name
L1_entry = next(entry for entry in a4_report["checks"]
                if entry["name"] == "L1_equals_gkd_branch")
L1_detail, L1_verdict = L1_entry["detail"], L1_entry["verdict"]
say(f"record: {L1_detail} ({L1_verdict})")
```

The sympy verification of the record of the field equations of a_4 is read. Its checks form a list; `next(...)` takes the first entry whose name is `L1_equals_gkd_branch`. The output prints its text and verdict: $L_{(1)} = -84H^2 + 12\,ad1^2$ (PASS), where `ad1` is the record's name for a_4'; this check is made for the metric at $b_4 = 1$. $L_{(1)}$ is the first **Lovelock scalar**: the Revision record builds the field equations from three curvature scalars $L_{(1)}, L_{(2)}, L_{(3)}$, made from products of one, two or three Riemann tensors with the generalized Kronecker delta of Chapter 1, and the first of them is exactly twice the Ricci scalar, $L_{(1)} = 2R$ (Chapter 11 treats all three).

```
L1_text = L1_detail.split("=")[1]  # the text after the equals sign
L1_record = parse_expr(L1_text, local_dict={"H": H, "ad1": a4p})
record_check(PYTHON_REPORT, "L1_equals_2R", "L_(1) = 2 R exactly")  # stops if not
R_at_b4_1 = plain(R_scalar).subs({b4pp: 0, b4p: 0, b4v: 1})  # R at b4 = 1
check(L1_verdict == "PASS" and sp.expand(L1_record - 2 * R_at_b4_1) == 0,
      "at b4 = 1 the first Lovelock scalar of the record is twice our Ricci scalar",
      record=f"{A4_REPORT}, check L1_equals_gkd_branch (with "
             "python-lovelock-report.json, check L1_equals_2R)")
```

The text after the equals sign is read as a sympy expression, with `ad1` meaning a_4'. `record_check` stops the notebook unless the independent verification of the curvature record lists its check `L1_equals_2R` as passed with the detail `L_(1) = 2 R exactly`. `R_at_b4_1` is our Ricci scalar at $b_4 = 1$: the dictionary in `subs` sets $b_4'' = 0$, $b_4' = 0$ and $b_4 = 1$, and R becomes $6a_4'^2 - 42H^2$. The check: the record marks the check `L1_equals_gkd_branch` as passed, and $L_{(1)} = 2R$ with our R at $b_4 = 1$ ($2(6a_4'^2 - 42H^2) = 12a_4'^2 - 84H^2$).

In [19], all 310 components at five test points with 30 digits.

```
PYTHON_REPORT = "Revision/gkd_lovelock/results/python-lovelock-report.json"
python_report = json.loads(repository_file(PYTHON_REPORT).read_text(encoding="utf-8"))
mpmath.mp.dps = 30  # mpmath works with 30 significant digits from now on


def exact(text):
    """A fraction such as 46/75 of the record as an mpmath number."""
    fraction = sp.Rational(text)
    return mpmath.mpf(fraction.p) / fraction.q  # numerator / denominator
```

The independent verification of the record lists five test points (`randomPoints`), each a set of exact fractions written as texts such as `"46/75"`. `mpmath.mp.dps = 30` makes mpmath compute with 30 significant digits. `exact` reads such a text as an exact fraction (`sp.Rational`) and divides its numerator `p` by its denominator `q` with 30 digits.

```
test_points = [[exact(p["H"]), exact(p["x0"]), exact(p["a4"]),
                exact(p["a4" + prime]), exact(p["a4" + prime + prime]),
                exact(p["b4"]), exact(p["b4" + prime]), exact(p["b4" + prime + prime])]
               for p in python_report["randomPoints"]]
pairs = [(Gamma[a][b][c], v) for (a, b, c), v in record_gamma.items()]
pairs += [(R_mixed[key], v) for key, v in record_riemann.items()]
```

Each test point becomes the eight numbers H, x_0, a_4, a_4', a_4'', b_4, b_4', b_4'' (the record's keys contain the apostrophe, built from `prime` of In [10]). `pairs` collects pairs (our expression, the record's expression): first the 25 Christoffel symbols, then (`+=` appends a list) the 156 Riemann components.

```
pairs += [(Ric[i, j], from_mathematica(record["ricciMixed"][f"{NAMES[i]},{NAMES[j]}"]
                                       ["mathematica"]))
          for i in range(8) for j in range(8)]
pairs += [(G[i, j], from_mathematica(record["einsteinMixed"][f"{NAMES[i]},{NAMES[j]}"]
                                     ["mathematica"]))
          for i in range(8) for j in range(8)]
pairs += [(R_scalar, from_mathematica(record["ricciScalar"]))]
# the order of the eight numbers of a test point
ARGS30 = (H, x0, a4v, a4p, a4pp, b4v, b4p, b4pp)
largest, evaluations = mpmath.mpf(0), 0
```

Then the 64 Ricci and 64 Einstein components and the scalar: $25 + 156 + 64 + 64 + 1 = 310$ pairs. `ARGS30` names the eight arguments in the order of a test point. The largest relative difference starts at zero and the count of evaluations at 0.

```
for mine, theirs in pairs:
    f_mine = sp.lambdify(ARGS30, symbolic(sp.S(mine)), "mpmath")
    f_record = sp.lambdify(ARGS30, sp.S(theirs), "mpmath")
    for point in test_points:
        ours, recorded = f_mine(*point), f_record(*point)
        largest = max(largest, abs(ours - recorded) / max(1, abs(recorded)))
        evaluations += 1
for name in ("rust_christoffels_agree", "rust_riemann_agrees",
             "rust_ricci_einstein_scalar_agree"):
    record_check(PYTHON_REPORT, name)  # stops unless the check passed
```

For each pair, both expressions become functions that compute with mpmath's 30 digits (`"mpmath"` instead of `"numpy"`); at each of the five points both are evaluated, and the relative difference $|\text{ours} - \text{record}|/\max(1, |\text{record}|)$ updates the largest one so far (`max(1, ...)` avoids dividing by a tiny number when a value is close to zero). The last loop runs `record_check` for the three checks that the PASS line below names: the notebook stops unless the independent verification lists all three as passed.

```
report("components compared numerically", len(pairs))
report("evaluations at the five test points", evaluations)
report("largest relative difference (30 digits)", mpmath.nstr(largest, 3))
check(len(test_points) == 5 and largest < mpmath.mpf(10) ** -25,
      "at the five test points every component agrees to more than 25 digits",
```

```
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "randomPoints, checks rust_christoffels_agree, rust_riemann_agrees, "
             "rust_ricci_einstein_scalar_agree")
```

Three RESULT lines: 310 components, 1550 evaluations, and the largest relative difference 2.96×10^{-31} (`mpmath.nstr` writes an mpmath number with 3 digits), which is the rounding of the last of the 30 digits. The check requires more than 25 agreeing digits. This is a second, purely numerical confirmation of the exact agreement of In [7], In [15] and In [17].

In [20], the lead's four properties of the Einstein tensor.

```
LEAD = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"
for name in ("einstein_off_diagonal_zero", "einstein_x0_independent",
             "einstein_isotropy", "no_vacuum_for_H_positive"):
    record_check(LEAD, name)  # stops unless the lead check passed
METRIC_B4 = "Revision/metric_b4/results/wolfram-metric-b4-report.json"
for name in ("off_diagonal_entries_x0_x4_only", "off_diagonal_proportional_to_b4prime"):
    record_check(METRIC_B4, name)  # stops unless the check passed
G_b4_1 = G.applyfunc(lambda e: symbolic(e).subs({b4pp: 0, b4p: 0, b4v: 1}))
off_diagonal = {(i, j): plain(G[i, j]) for i in range(8) for j in range(8)
                if i != j and G[i, j] != 0}
for (i, j), value in sorted(off_diagonal.items()):
    say(f"  G^{NAMES[i]}_{NAMES[j]} = {value}")
check(sorted(off_diagonal) == [(0, 4), (4, 0)]
      and vanishes(symbolic(G[4, 0]) - sp.Rational(3, 2) * H * sp.cot(6 * H * x0)
                   * b4p / b4v)
      and vanishes(symbolic(G[0, 4]) + sp.Rational(3, 2) * H * sp.tan(6 * H * x0)
                   * b4p / b4v)
      and all(G_b4_1[i, j] == 0 for i in range(8) for j in range(8) if i != j),
      "the only off-diagonal components G^x4_x0, G^x0_x4 are proportional to b4p; "
      "at b4 = 1 the Einstein tensor is diagonal",
      record=f"{LEAD}, check einstein_off_diagonal_zero, and "
             "wolfram-metric-b4-report.json, checks off_diagonal_entries_x0_x4_only, "
             "off_diagonal_proportional_to_b4prime")
check(all(sp.diff(G_b4_1[i, j], x0) == 0 for i in range(8) for j in range(8)),
      "at b4 = 1 no component of the Einstein tensor depends on x0",
      record=f"{LEAD}, check einstein_x0_independent")
check(G[1, 1] == G[2, 2] == G[3, 3] and G[5, 5] == G[6, 6] == G[7, 7],
      "3-space components equal, extra-time components equal",
      record=f"{LEAD}, check einstein_isotropy")
```

`LEAD` names the report of the lead check of the field equations (the lead checks are explained in Section 3.7), made for the metric at $b_4 = 1$, and `METRIC_B4` the report of the verifier of the metric with b_4. The two loops run `record_check` (In [3]) for the four checks of the first report and the two of the second that this cell's PASS lines name; the notebook stops unless each is listed as passed. `G_b4_1` is the Einstein tensor at $b_4 = 1$: `applyfunc` applies a function to every entry, and `lambda e: ...` is a function without a name that writes an entry with the printing symbols and sets $b_4'' = 0$, $b_4' = 0$, $b_4 = 1$. `off_diagonal` collects the non-zero entries off the diagonal; the output prints the two of them, `G^x0_x4 = -3*H*b4p*tan(z)/(2*b4v)` and `G^x4_x0 = 3*H*b4p/(2*b4v *tan(z))`. Then three checks of Section 3.25, each naming the check of the reports: the only entries off the diagonal are ${G^{x_4}}_{x_0} = \frac{3}{2} H \cot z \, b_4'/b_4$ and ${G^{x_0}}_{x_4} = -\frac{3}{2} H \tan z \, b_4'/b_4$, so that at $b_4 = 1$ there is

none; at $b_4 = 1$ no entry depends on x_0; and, for every b_4, equal 3-space entries and equal extra-time entries.

```
gap = sp.simplify(G_b4_1[4, 4] - G_b4_1[0, 0])
say(f"at b4 = 1: G^x4_x4 - G^x0_x0 = {sp.factor(gap)}")
check(sp.simplify(gap - 6 * (a4p ** 2 + H ** 2)) == 0,
      "at b4 = 1: G^x4_x4 - G^x0_x0 = 6 (a4p^2 + H^2) > 0",
      record=f"{LEAD}, check no_vacuum_for_H_positive")
```

The difference $G^{x_4}{}_{x_4} - G^{x_0}{}_{x_0}$ at $b_4 = 1$, printed in factored form (`sp.factor`) as `6*(H**2 + a4p**2)`, and the check of Section 3.25 behind the statement that no empty spacetime has this metric (Section 3.26).

In [21], the contracted Bianchi identity.

```
divergence = []
for nu in range(8):
    value = sum(sp.diff(G[mu, nu], X[mu]) for mu in range(8)) \
        + sum(Gamma[mu][mu][lam] * G[lam, nu] for mu in range(8) for lam in range(8)) \
        - sum(Gamma[lam][mu][nu] * G[mu, lam] for mu in range(8) for lam in range(8))
    divergence.append(vanishes(value))
record_check(RUST_REPORT, "k1_equals_minus_4_einstein")  # stops if not passed
record_check(RUST_REPORT, "k1_divergence_free")
check(all(divergence),
      "the contracted Bianchi identity: the divergence of G vanishes",
      record="Revision/gkd_lovelock/results/lovelock-report.json, checks "
             "k1_equals_minus_4_einstein and k1_divergence_free")
```

For each ν the three sums of the divergence formula of Section 3.25 are computed with the functions $a_4(x_4)$ and $b_4(x_4)$ themselves (so that sympy can differentiate a_4' and b_4' into a_4'' and b_4''; the backslashes continue the statement), with all entries of G, including the two off the diagonal, and the list `divergence` records whether each simplifies to zero. The check requires all eight. The record's Rust program checked the same for its tensor $P_{(1)} = -4G$, the first Lovelock tensor (Section 3.26; Chapter 11): the two calls of `record_check` stop the notebook unless the Rust report (`RUST_REPORT`, named in In [3]) lists its checks `k1_equals_minus_4_einstein` ($P_{(1)} = -4G$) and `k1_divergence_free` (the divergence of $P_{(1)}$ is zero) as passed.

In [22], the Kretschmann scalar.

```
K = sp.factor(sp.expand(symbolic(sum(v * get(R_mixed, (c, d, a, b))
                                     for (a, b, c, d), v in R_mixed.items()))))
K_record = sp.factor(sp.expand(sum(v * record_riemann.get((c, d, a, b), 0)
                                   for (a, b, c, d), v in record_riemann.items())))
ricci_square = sp.factor(sp.expand(symbolic(sum(Ric[i, j] * Ric[j, i]
                                                for i in range(8)
                                                for j in range(8)))))
say(f"Kretschmann scalar K = {K}")
say(f"sum of R^a_b R^b_a  = {ricci_square}")
```

$K = \sum R^{ab}{}_{cd} R^{cd}{}_{ab}$ from our 156 components, then a second time from the record's 156 components alone, and $\sum_{a,b} R^a{}_b R^b{}_a$; each is multiplied out (`sp.expand`) and factored (`sp.factor`). The output

shows two long fractions in H, a_4', a_4'', b_4, b_4' and b_4''; b_4 appears in them only together with its derivatives, in the ratios b_4'/b_4 and b_4''/b_4. For a constant b_4 the first is $K = 12(7H^4 - 2H^2a_4'^2 + 7a_4'^4 + 2a_4''^2)$, the result of Section 3.25 (printed by the next block), and the second is $\sum R^a{}_b R^b{}_a = 6(42H^4 + 6a_4'^4 + a_4''^2)$ (the sum of the squares of the four kinds of diagonal entries, three, one, three and one of them, and the entries off the diagonal are then zero).

```
check(sp.simplify(K - K_record) == 0,
      "K from our components equals K from the 156 components of the record")
check(not K.has(x0) and not K.has(a4v),
      "K depends only on H, a4p, a4pp, b4v, b4p and b4pp, not on z and not on a4")
square_form = 7 * (a4p ** 2 - H ** 2 / 7) ** 2 + sp.Rational(48, 7) * H ** 4 \
    + 2 * a4pp ** 2
K_constant_b4 = sp.factor(K.subs({b4pp: 0, b4p: 0}))  # b4 constant: b4v drops out
say(f"for a constant b4: K = {K_constant_b4}")
check(sp.expand(K_constant_b4 / 12 - square_form) == 0,
      "for constant b4: K/12 = 7 (a4p^2 - H^2/7)^2 + 48 H^4/7 + 2 a4pp^2, so K > 0 "
      "for H > 0")
report("smallest possible K for constant b4 (at a4p^2 = H^2/7, a4pp = 0)",
       sp.sstr(sp.Rational(576, 7) * H ** 4))
```

Three checks: our K equals the record's; K contains neither x_0 nor the value of a_4; and, for a constant b_4, the completed square of Section 3.25 is exactly $K/12$. `K_constant_b4` is K with $b_4'' = 0$ and $b_4' = 0$, after which b_4 drops out; the printed line shows it, `12*(7*H**4 - 2*H**2*a4p**2 + 7*a4p**4 + 2*a4pp**2)`. The RESULT line prints the smallest possible value $576H^4/7$ for a constant b_4.

In [23], Figure 03b.5: components that see z and an invariant that does not.

```
# numerical functions of the 156 components with a4pp = 0 and b4pp = 0 (true on
# the history a4 = A H x4, b4 = 1); the second derivatives are set to 0 first
riemann_functions = {k: numeric(v.subs({sp.Derivative(a4, (x4, 2)): 0,
                                        sp.Derivative(b4, (x4, 2)): 0}))
                     for k, v in R_mixed.items()}
```

The 156 components as numerical functions of $(z, a_4, a_4', H, b_4, b_4')$, with $a_4'' = 0$ and $b_4'' = 0$, which hold on every linear history with $b_4 = 1$; the second derivatives are replaced by 0 before `numeric` (In [8]) is applied.

```
def kretschmann_numbers(z_values, rate):
    """K at the points z_values, summed from the 156 numerical components."""
    total = np.zeros_like(z_values)
    for (a, b, c, d), f in riemann_functions.items():
        partner = riemann_functions.get((c, d, a, b))
        if partner is not None:
            total += (f(z_values, 0.5, rate, 1.0, 1.0, 0.0)
                      * partner(z_values, 0.5, rate, 1.0, 1.0, 0.0))
    return total
```

K summed numerically at many values of z at once, for a given rate a_4' (with $a_4 = 0.5$, $H = 1$, $b_4 = 1$, $b_4' = 0$): each component times its partner with the pairs exchanged (`None` means that no partner was found; every component has one).

```
z_line = np.linspace(0.02, np.pi / 2 - 0.02, 300)
cot_part = riemann_functions[(1, 4, 1, 0)](z_line, 0.5, 1.5, 1.0, 1.0, 0.0)
tan_part = riemann_functions[(1, 0, 1, 4)](z_line, 0.5, 1.5, 1.0, 1.0, 0.0)
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
fig.subplots_adjust(wspace=0.32)  # room for the label of the right vertical axis
```

300 values of z; the two components ${R^{x_1x_4}}_{x_1x_0} = Ha_4' \cot z$ (key `(1, 4, 1, 0)`) and ${R^{x_1x_0}}_{x_1x_4} = -Ha_4' \tan z$ (key `(1, 0, 1, 4)`) at $a_4' = 1.5$, $b_4 = 1$; a figure with two panels.

```
axes[0].plot(z_line, np.abs(cot_part), color=RED, lw=1.8,
             label="$R^{x_1x_4}{}_{x_1x_0} = Ha_4^{\\prime}\\cot z$")
axes[0].plot(z_line, np.abs(tan_part), color=BLUE, lw=1.8, ls="--",
             label="minus $R^{x_1x_0}{}_{x_1x_4} = Ha_4^{\\prime}\\tan z$")
axes[0].plot(z_line, np.abs(cot_part * tan_part), color="black", lw=1.2, ls=":",
             label="minus their product $= H^2 a_4^{\\prime 2}$")
axes[0].plot(z_line, np.abs(np.broadcast_to(
    riemann_functions[(1, 0, 1, 0)](z_line, 0.5, 1.5, 1.0, 1.0, 0.0), z_line.shape)),
    color=AQUA, lw=1.8, ls="-.", label="minus $R^{x_1x_0}{}_{x_1x_0} = H^2$")
```

The left panel: the sizes of the two components, of their product, and of the constant plane component ${R^{x_1x_0}}_{x_1x_0} = -H^2$ (key `(1, 0, 1, 0)`; repeated to the length of `z_line` by `np.broadcast_to`).

```
axes[0].set_yscale("log")
axes[0].set_xlabel("$z = 6 H x_0$ (radians)")
axes[0].set_ylabel("absolute value (unit $H^2$)")
axes[0].set_title("Riemann components, $a_4^{\\prime} = 1.5$")
axes[0].legend(fontsize=8)
rates = (0.5, 1.0, 1.5, 2.0)  # four expansion rates a4p = A H (A > 0), unit H
for rate, colour, style in zip(rates, (VIOLET, AQUA, RED, ORANGE),
                               ("-", "--", "-.", ":")):
    axes[1].plot(z_line, kretschmann_numbers(z_line, rate), color=colour, ls=style,
                 lw=1.8, label=f"$a_4^{{\\prime}} = {rate:g}\\,H$")
```

A logarithmic axis, labels, title and legend for the left panel. The right panel: K summed numerically at the 300 points for four rates $a_4' = 0.5, 1, 1.5, 2$ (in units of H), each with its colour and line style.

```
axes[1].set_yscale("log")
axes[1].set_xlabel("$z = 6 H x_0$ (radians)")
axes[1].set_ylabel("$K$ (unit $H^4$)")
axes[1].set_title("Kretschmann scalar summed from 156 components")
axes[1].legend(fontsize=8)
# the values of the formula for K at the four rates, for the caption and the check
K_values = [float(K.subs({a4p: r, a4pp: 0, H: 1, b4v: 1, b4p: 0, b4pp: 0}))
            for r in rates]
K_text = ", ".join(f"${k:g}$" for k in K_values)
```

The right panel's axis, labels, title and legend. `K_values` are the values of the formula for K at the four rates, and `K_text` joins them with commas for the caption: 83.25, 144, 455.25, 1332 (from $12(7 - 2r^2 + 7r^4)$ with $H = 1$).

```
save_figure(fig, "riemann_and_kretschmann_z",
            "Left: the absolute values of the Riemann components "
...
numbers = [kretschmann_numbers(z_line, r) for r in rates]
check(all(np.max(np.abs(n - k)) < 1e-9 * k for n, k in zip(numbers, K_values)),
      "summed numerically, K is the same at all 300 values of z and equals the "
      "formula")
```

The figure is saved; the check requires the numerical sum to equal the formula at all 300 points within a relative 10^{-9}, for all four rates.

What Figure 03b.5 shows. At $b_4 = 1$. Left: the red curve $Ha_4' \cot z$ falls by more than three powers of ten across the patch, the blue dashed curve $Ha_4' \tan z$ rises by as much; their product (dotted) is the constant $H^2 a_4'^2 = 2.25$, and the plane component $H^2 = 1$ (dash-dotted) is constant too. Right: the four values of K are four flat lines, 83.25, 144, 455.25 and 1332: although single components blow up at the ends of the patch, the invariant is the same at every z. The Kretschmann scalar therefore gives no sign of a singularity anywhere in the patch: the large components are an effect of the coordinates.

In [24], Figure 03b.6: the curvature scalars along the history.

```
A = sp.symbols("A", real=True)  # the slope of the history a4 = A H x4
on_history = {a4p: A * H, a4pp: 0, b4pp: 0, b4p: 0, b4v: 1}
R_A = sp.factor(symbolic(R_scalar).subs(on_history))
K_A = sp.factor(K.subs(on_history))
RR_A = sp.factor(ricci_square.subs(on_history))
say(f"on the history: R = {R_A},  K = {K_A},  R^a_b R^b_a = {RR_A}")
check(all(sp.expand(f.subs(A, -A) - f) == 0 for f in (R_A, K_A, RR_A)),
      "the curvature scalars are even in A")
```

The slope A as a symbol and the history $a_4' = AH$, $a_4'' = 0$, $b_4 = 1$, $b_4' = b_4'' = 0$ (the dictionary `on_history`) put into the three scalars. The output: $R = 6H^2(A^2 - 7)$, $K = 12H^4(7A^4 - 2A^2 + 7)$, $\sum R^a{}_b R^b{}_a = 36H^4(A^4 + 7)$. The check: all three are **even** in A (replacing A by $-A$ changes nothing), because they contain only A^2. These three numbers therefore cannot tell which of the two families of directions deflates; the metric and the components of the curvature can (Section 3.24).

```
A_line = np.linspace(0.1, 3.0, 291)  # slopes 0.1, 0.11, ..., 3; A_line[90] = 1
R_numbers = sp.lambdify(A, R_A.subs(H, 1))(A_line)
K_numbers = sp.lambdify(A, K_A.subs(H, 1))(A_line)
RR_numbers = sp.lambdify(A, RR_A.subs(H, 1))(A_line)
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
```

291 slopes from 0.1 to 3 in steps of 0.01 (position 90 is $A = 1$), all positive, so every history drawn deflates the extra times; the three scalars at these slopes with $H = 1$; a figure with two panels.

```
axes[0].plot(A_line, R_numbers, color=VIOLET, lw=1.8, label="$R = 6H^2(A^2 - 7)$")
axes[0].axhline(0.0, color="black", lw=0.8)
axes[0].plot([np.sqrt(7)], [0.0], "o", color=VIOLET, ms=7)  # the zero of R
axes[0].set_ylabel("Ricci scalar $R$ (unit $H^2$)")
```

```
axes[0].set_title("Ricci scalar")
axes[1].plot(A_line, K_numbers, color=RED, lw=1.8, label="Kretschmann $K$")
axes[1].plot(A_line, RR_numbers, color=BLUE, lw=1.8, ls="--",
             label="$\\sum R^a{}_b R^b{}_a$")
axes[1].set_yscale("log")
axes[1].set_ylabel("value (unit $H^4$)")
axes[1].set_title("Squares of the curvature")
```

Left: R against A, the zero line, and a dot at the zero $A = \sqrt{7}$. Right, on a logarithmic axis: K and $\sum R^a{}_b R^b{}_a$.

```
for ax in axes:
    ax.axvline(1.0, color="grey", lw=1.0, ls=":")  # the canonical slope A = 1
    ax.set_xlabel("slope $A$ of the deflating history $a_4 = A H x_4$")
    ax.legend(fontsize=8, loc="upper left")
axes[0].text(1.05, -40, "$A = 1$", fontsize=9)
save_figure(fig, "scalars_versus_a",
            "The curvature scalars of the author's metric along the deflating "
...
check(abs(R_numbers[90] - (-36.0)) < 1e-9 and abs(K_numbers[90] - 144.0) < 1e-9,
      "at A = 1: R = -36 H^2 and K = 144 H^4")
```

A dotted vertical line at the canonical $A = 1$ in both panels, the axis label and legend, a label at the line; the figure is saved; the check confirms $R = -36$ and $K = 144$ at $A = 1$ (Section 3.25).

What Figure 03b.6 shows. Left: the Ricci scalar rises from -42 near $A = 0$ through -36 at $A = 1$ and crosses zero at $A = \sqrt{7} = 2.65$ (the dot). Right: K (red) is smallest, $576/7 = 82.3$, near $A = 1/\sqrt{7} = 0.38$ and grows like $84A^4$ for large slopes; $\sum R^a{}_b R^b{}_a$ (blue dashed) starts at 252 and grows like $36A^4$. Both stay far above zero: every deflating history is curved.

In [25], Figure 03b.7: the Einstein tensor along the history.

```
G_A = [sp.factor(symbolic(G[k, k]).subs(on_history)) for k in range(8)]
for k in (0, 1, 4, 5):
    say(f"  on the history: G^{NAMES[k]}_{NAMES[k]} = {G_A[k]}")
bars = {value: [float(G_A[k].subs({A: value, H: 1})) for k in range(8)]
        for value in (1, 2)}
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.2))
positions = np.arange(8)
```

The eight diagonal entries of G on the history (with $b_4 = 1$), factored; four of them, those of x_0, x_1, x_4 and x_5, are printed: $G^{x_1}{}_{x_1} = G^{x_5}{}_{x_5} = G^{x_0}{}_{x_0} = -3H^2(A^2 - 5)$ and $G^{x_4}{}_{x_4} = 3H^2(A^2 + 7)$. `bars` holds their values for $A = 1$ and $A = 2$ with $H = 1$.

```
axes[0].bar(positions - 0.2, bars[1], width=0.38, color=RED, hatch="//",
            label="$A = 1$")
axes[0].bar(positions + 0.2, bars[2], width=0.38, color=BLUE, label="$A = 2$")
axes[0].set_xticks(positions, [f"$G^{{x_{k}}}{{}}_{{x_{k}}}$" for k in range(8)],
                   fontsize=8)
axes[0].set_ylabel("component (unit $H^2$)")
axes[0].set_title("Diagonal of the Einstein tensor")
axes[0].legend(fontsize=8)
```

The left panel is a bar chart: `ax.bar(x, heights, width=...)` draws bars; the bars of $A = 1$ are shifted left by 0.2 and hatched (`hatch="//"`), those of $A = 2$ shifted right, so that each pair stands side by side. The tick labels name the eight components, from ${G^{x_0}}_{x_0}$ to ${G^{x_7}}_{x_7}$.

```
space_numbers = sp.lambdify(A, G_A[1].subs(H, 1))(A_line)
time_numbers = sp.lambdify(A, G_A[4].subs(H, 1))(A_line)
axes[1].plot(A_line, space_numbers, color=RED, lw=1.8,
             label="$G^{x_1}{}_{x_1} = G^{x_5}{}_{x_5} = G^{x_0}{}_{x_0}$")
axes[1].plot(A_line, time_numbers, color=VIOLET, lw=1.8, ls="--",
             label="$G^{x_4}{}_{x_4}$")
axes[1].fill_between(A_line, space_numbers, time_numbers, color="grey", alpha=0.2,
                     label="gap $6H^2(A^2 + 1) > 0$")
```

The right panel: the common 3-space, extra-time and hidden entry and the time entry against A; `fill_between` shades the gap between the two curves.

```
axes[1].axhline(0.0, color="black", lw=0.8)
axes[1].axvline(1.0, color="grey", lw=1.0, ls=":")  # the canonical slope A = 1
axes[1].set_xlabel("slope $A$ of the deflating history $a_4 = A H x_4$")
axes[1].set_ylabel("component (unit $H^2$)")
axes[1].set_title("Einstein tensor versus $A$")
axes[1].legend(fontsize=8, loc="lower left")
save_figure(fig, "einstein_tensor",
            "The Einstein tensor $G^{\\mu}{}_{\\nu}$ of the author's metric along "
...
check(bars[1] == [12.0, 12.0, 12.0, 12.0, 24.0, 12.0, 12.0, 12.0]
      and bars[2] == [3.0, 3.0, 3.0, 3.0, 33.0, 3.0, 3.0, 3.0],
      "diagonal of G: 12 (24 for x4) at A = 1 and 3 (33 for x4) at A = 2, unit H^2")
```

The zero line, the line at $A = 1$, labels, title, legend; the figure is saved; the check confirms the two rows of bar heights.

What Figure 03b.7 shows. At $b_4 = 1$. Left: for $A = 1$ seven bars of height 12 and the time bar 24 (the fifth bar, x_4); for $A = 2$ seven bars of height 3 and the time bar 33. Right: the common entry $3H^2(5 - A^2)$ falls and becomes negative beyond $A = \sqrt{5} = 2.24$, the time entry $3H^2(7 + A^2)$ rises, and the shaded gap $6H^2(A^2 + 1)$ never closes: no cosmological constant, which adds the same number to every entry, can make all of them zero (Section 3.26).

In [26], the source required by Einstein's equations, from the record.

```
A4_EQUATIONS = "Revision/field_equations_a4/a4-equations.json"
a4_equations = json.loads(repository_file(A4_EQUATIONS).read_text(encoding="utf-8"))
linear = a4_equations["linearMember"]  # the entry of the linear history
kappa = sp.symbols("kappa", positive=True)  # the constant of Einstein's equations
Lam = sp.symbols("Lam", real=True)  # the cosmological constant Lambda
source_names = {"AA": A, "H": H, "kappa": kappa, "Lam": Lam}
```

The record of the field equations of a_4 is read and its entry `linearMember` taken. κ and Λ become symbols; the dictionary says that the record's names `AA` and `Lam` mean A and Λ.

```
def source(key):
    """The record's text linear[key] (Mathematica notation) as sympy."""
    return parse_expr(linear[key]["input"].replace("^", "**"),
                      local_dict=source_names)


rho_record, p_record = source("rhoEinstein"), source("pEinstein")
sum_record = source("rhoPlusPEinstein")  # rho + p
say(f"record: kappa rho       = {sp.expand(kappa * rho_record)}")
say(f"record: kappa p         = {sp.expand(kappa * p_record)}")
say(f"record: kappa (rho + p) = {sp.factor(kappa * sum_record)}")
```

`source` reads the Mathematica text (key `"input"`) of an entry as a sympy expression. The three texts are read and printed multiplied by κ: $\kappa\rho = -3A^2H^2 - 21H^2 - \Lambda$, $\kappa p = -3A^2H^2 + 15H^2 + \Lambda$, $\kappa(\rho + p) = -6H^2(A^2 + 1)$.

```
record_check(A4_REPORT, "json_linear_member")  # stops if not passed
check(sp.expand(kappa * rho_record - (-G_A[4] - Lam)) == 0,
      "kappa rho of the record equals -G^x4_x4 - Lambda",
      record=f"{A4_EQUATIONS}, linearMember.rhoEinstein (and "
             "python-a4-report.json, check json_linear_member)")
check(all(sp.expand(kappa * p_record - (G_A[k] + Lam)) == 0 for k in (1, 5, 0)),
      "kappa p of the record equals G^x1_x1, G^x5_x5 and G^x0_x0 plus Lambda",
      record=f"{A4_EQUATIONS}, linearMember.pEinstein")
check(sp.expand(kappa * sum_record - (G_A[1] - G_A[4])) == 0,
      "kappa (rho + p) of the record equals G^x1_x1 - G^x4_x4 = -6 (A^2 + 1) H^2",
      record=f"{A4_EQUATIONS}, linearMember.rhoPlusPEinstein")
```

`record_check` (In [3]) stops the notebook unless the sympy verification of the record of the field equations (`A4_REPORT`, named in In [18]) lists its check `json_linear_member` as passed: that check confirmed that the entries of `linearMember` agree with its own computation. Then three checks, the three lines of Section 3.26: $\kappa\rho = -G^{x_4}{}_{x_4} - \Lambda$; $\kappa p = G^{x_1}{}_{x_1} + \Lambda = G^{x_5}{}_{x_5} + \Lambda = G^{x_0}{}_{x_0} + \Lambda$; and $\kappa(\rho + p) = G^{x_1}{}_{x_1} - G^{x_4}{}_{x_4}$.

In [27], the negative control.

```
g_control = g.copy()
for k in (5, 6, 7):  # the three extra times, now inflating
    g_control[k, k] = -sp.exp(2 * a4) * sp.sin(6 * H * x0) ** sp.Rational(1, 3)
Gamma_control = christoffel(g_control, X)
R_control = raise_second(riemann(Gamma_control, X), g_control)
Ric_control = ricci(R_control, 8)
G_control = (Ric_control - Ric_control.trace() / 2 * sp.eye(8)).applyfunc(tidy)
```

A check that cannot fail teaches nothing, so the metric is changed on purpose: `g.copy()` is a copy of the author's metric, and its three extra-time entries (positions 5, 6, 7) are replaced by $-e^{+2a_4}\sin^{1/3} z$, extra times that INFLATE like 3-space. This is NOT a deflating metric; within the author's family it is the member $b_4 = e^{4a_4}$, with the extra-time rate $\alpha_t = +a_4'$. The curvature is computed again with the same functions, up to the Einstein tensor. This cell runs for up to half a minute.

```
off = {(i, j): plain(G_control[i, j]) for i in range(8) for j in range(8)
       if i != j and G_control[i, j] != 0}
for (i, j), value in sorted(off.items()):
    say(f"  control: G^{NAMES[i]}_{NAMES[j]} = {value}")
b4_control = {sp.Derivative(b4, x4): 4 * sp.Derivative(a4, x4) * sp.exp(4 * a4),
              b4: sp.exp(4 * a4)}  # b4 = e^(4 a4) and its derivative
general_off = [G[i, j].subs(b4_control) for i, j in ((0, 4), (4, 0))]
K_control = sp.expand(symbolic(sum(
    v * get(R_control, (c, d, a, b)) for (a, b, c, d), v in R_control.items())))
check(sorted(off) == [(0, 4), (4, 0)] and all(v.has(z) for v in off.values())
      and vanishes(G_control[0, 4] - general_off[0])
      and vanishes(G_control[4, 0] - general_off[1])
      and sp.simplify(K_control.subs({a4pp: 0, a4p: H})) == 0,
      "negative control: with inflating extra times (b4 = e^(4 a4)) G^x4_x0 and "
      "G^x0_x4 are not zero, depend on z, equal the general ones; K = 0 at a4p = H")
```

The entries off the diagonal that are not zero are collected and printed: ${G^{x_0}}_{x_4} = -6Ha_4'\tan z$ and ${G^{x_4}}_{x_0} = 6Ha_4'/\tan z$, exactly as derived by hand in Section 3.25. `b4_control` replaces b_4 by e^{4a_4} and b_4' by $4a_4'e^{4a_4}$, and `general_off` holds the two general off-diagonal entries of In [20] with this b_4 ($b_4'/b_4 = 4a_4'$). `K_control` is the Kretschmann scalar of the control. The check requires these two entries and only these two, that both depend on z (`.has(z)`), that they equal the general entries with $b_4 = e^{4a_4}$, and that K of the control vanishes at $a_4' = H$, $a_4'' = 0$ (Section 3.25). With the author's deflating extra times and a constant b_4 the two entries are zero (In [20]): the deflation of the extra times, at the same rate as the inflation of 3-space, is what makes the Einstein tensor of the author's metric diagonal; and the bound $K \geq 576H^4/7$ of In [22] needs a constant b_4.

In [28], the last check.

```
figure_names = ["christoffel_heat_maps", "christoffel_versus_z",
                "finite_differences", "plane_curvatures",
                "riemann_and_kretschmann_z", "scalars_versus_a", "einstein_tensor"]
files = [output_file(f"{FIGURE_FOLDER}/03b_{k}_{n}.png")
         for k, n in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all seven figure files exist")
all_checks_passed()
```

The seven figure files must exist; the last line prints ALL 40 CHECKS PASSED (notebook 03b). The 40 checks are: 1 in In [4], 1 in In [6], 2 in In [7], 1 each in In [8] and In [9], 2 in In [10], 1 in In [11], 2 in In [12], 1 in In [13], 3 in In [14], 2 in In [15], 3 in In [16], 1 each in In [17], In [18] and In [19], 4 in In [20], 1 in In [21], 3 in In [22], 1 in In [23], 2 in In [24], 1 in In [25], 3 in In [26], and 1 each in In [27] and In [28].

3.31 Free fall in the author's metric

What does the author's metric do to a body that moves in it with no force acting on it, only the geometry? This section answers with the geodesic equation of Section 3.16 and the Christoffel symbols of Section 3.23.

Test particles in free fall. A **test particle** is a body so small that it does not change the metric (ASSUMED: the particles of this section are test particles). In free fall it moves on a time-like

geodesic (Section 3.16; ASSUMED from the theory of gravity), parametrised by its proper time τ, with the velocity $u^a = dx^a/d\tau$ and $g(u,u) = -1$. We keep the labels of Section 3.23: i for 3-space, j for the extra times, k for any of these six transverse directions.

The geodesic equations. $du^a/d\tau = -\sum_{b,c} \Gamma^a{}_{bc} u^b u^c$ with the 37 symbols of Section 3.23 (a symbol with two different lower indices appears twice, once in each order, hence the factors 2), with the two rates $\alpha = a_4'$ and $\alpha_t = -a_4' + b_4'/(2b_4)$:

$$\frac{du^0}{d\tau} = H\tan z \sum_i h_i^2 (u^i)^2 - H\tan z \sum_j h_j^2 (u^j)^2 + 6H(\tan z + \cot z)(u^0)^2,$$

$$\frac{du^i}{d\tau} = -2\alpha\, u^i u^4 - 2H\cot z\, u^i u^0, \qquad \frac{du^j}{d\tau} = -2\alpha_t\, u^j u^4 - 2H\cot z\, u^j u^0,$$

$$\frac{du^4}{d\tau} = -\alpha \sum_i h_i^2 (u^i)^2 + \alpha_t \sum_j h_j^2 (u^j)^2$$

(for u^0 the symbols $\Gamma^0{}_{ii} = -H\tan z\, h_i^2$, $\Gamma^0{}_{jj} = +H\tan z\, h_j^2$ and $\Gamma^0{}_{00} = -6H(\tan z + \cot z)$; for u^i the symbols $\Gamma^i{}_{i4} = \alpha$ and $\Gamma^i{}_{i0} = H\cot z$; for u^j the symbols $\Gamma^j{}_{j4} = \alpha_t$ and $\Gamma^j{}_{j0} = H\cot z$; for u^4 the symbols $\Gamma^4{}_{ii} = \alpha h_i^2$ and $\Gamma^4{}_{jj} = -\alpha_t h_j^2$; each moved to the right side with its minus sign). For a constant b_4, $\alpha_t = -a_4'$: then $du^j/d\tau = +2a_4' u^j u^4 - 2H\cot z\, u^j u^0$ and $du^4/d\tau = -a_4' \sum_k h_k^2 (u^k)^2$. Together with $dx^a/d\tau = u^a$ these are 16 first-order equations for the 8 coordinates and the 8 velocities.

Six conserved momenta. The metric does not depend on the six transverse coordinates, so by Section 3.16 the six **momenta** $p_k = g_{kk} u^k$ (no sum) do not change during free fall:

$$p_i = h_i^2\, u^i, \qquad p_j = -h_j^2\, u^j \qquad \text{(both constant)}$$

(PROVED in Section 3.16 for every diagonal metric that does not depend on x_k; Notebook 03d proves it again with sympy for all six, for general $a_4(x_4)$ and $b_4(x_4)$, In [4]). The squared speed $g(u,u)$ is conserved too (Section 3.16).

Frame velocities: redshift in 3-space, blueshift along the extra times. An observer at rest at the particle's place measures lengths and durations with the frame of Section 3.7. Per unit of the particle's proper time, the particle covers along x_a the proper length (along an extra time, the proper duration) $\hat{u}^a = h_a u^a$, its **frame velocity**. From the momenta:

$$\hat{u}^i = h_i u^i = \frac{p_i}{h_i} = p_i\, e^{-a_4} \sin^{-1/6} z, \qquad \hat{u}^j = h_j u^j = -\frac{p_j}{h_j} = -p_j\, b_4^{-1/2} e^{a_4} \sin^{-1/6} z$$

(divide the momentum by the scale factor; $1/h_i = e^{-a_4}\sin^{-1/6} z$ and $1/h_j = b_4^{-1/2} e^{a_4} \sin^{-1/6} z$). With $\sin^{1/6} z = e^{Hy}$ (Section 3.9):

$$\hat{u}^i = p_i\, e^{-Hy - a_4}, \qquad \hat{u}^j = -p_j\, b_4^{-1/2} e^{a_4 - Hy}.$$

At a fixed height y, along a deflating history (a_4 increasing, b_4 constant), the frame velocity in 3-space falls like e^{-a_4}: motion in 3-space is **redshifted**, because 3-space inflates and stretches it out. The frame velocity along an extra time grows like e^{a_4}: motion along the extra times is **blueshifted**, because they deflate (in general it grows like $1/h_j$, that is like $e^{-\int \alpha_t\, dx_4}$). The factor e^{-Hy-a_4} is exactly the **momentum weight** $\kappa(y, x_4) = e^{-Hy-a_4(x_4)}$ (the Revision record's name for it; this $\kappa(y, x_4)$, always written in this chapter with its two arguments, is a function and has nothing to do with the constant κ of Einstein's equations in Section 3.26) that the Revision Kohn-Sham record uses to turn a conserved 3-space momentum into the momentum seen at height y and time x_4 (`Revision/kohn_sham/ks-theory.json`, key `geometry.kappa`; PROVED here; Notebook 03d, In [5]). It does not contain b_4: the 3-space scale factors do not.

The push along the hidden direction. Write $S = \sum_i (\hat{u}^i)^2$ and $E = \sum_j (\hat{u}^j)^2$ for the squared frame velocities in 3-space and along the extra times. The frame velocity along the hidden direction is $\hat{u}^0 = h_0 u^0 = \cot z\, u^0$, and it equals $dy/d\tau$, because $dy/d\tau = (dy/dx_0)(dx_0/d\tau) = \cot z\, u^0$ (Section 3.9). Its rate of change, line by line:

$$\frac{d\hat{u}^0}{d\tau} = \frac{d\cot z}{d\tau}\, u^0 + \cot z\, \frac{du^0}{d\tau}$$

(the product rule)

$$= -\frac{6H}{\sin^2 z}\,(u^0)^2 + \cot z\Big[H\tan z\,(S-E) + 6H(\tan z + \cot z)(u^0)^2\Big]$$

($d\cot z/d\tau = -\frac{1}{\sin^2 z}\cdot 6H\cdot u^0$ by the chain rule; the geodesic equation for u^0 with $h_i^2(u^i)^2 = (\hat{u}^i)^2$ and $h_j^2(u^j)^2 = (\hat{u}^j)^2$)

$$= -\frac{6H}{\sin^2 z}(u^0)^2 + H(S-E) + 6H\big(1+\cot^2 z\big)(u^0)^2 = H(S-E)$$

($\cot z \tan z = 1$; $1 + \cot^2 z = 1/\sin^2 z$, so the two terms with $(u^0)^2$ cancel). Hence

$$\frac{d^2y}{d\tau^2} = H\Big(\sum_i (\hat{u}^i)^2 - \sum_j (\hat{u}^j)^2\Big).$$

Motion in 3-space pushes the particle towards the patch end (y grows); motion along an extra time pushes it towards the tip (y falls). A particle at rest feels no push. Neither a_4 nor b_4 appears in this law (PROVED; Notebook 03d, In [6]).

The slowing of x_4. The geodesic equation for u^4, written with frame velocities, is

$$\frac{du^4}{d\tau} = -a_4' \sum_i (\hat{u}^i)^2 + \alpha_t \sum_j (\hat{u}^j)^2 = -a_4' S + \alpha_t E$$

(again $h_k^2(u^k)^2 = (\hat{u}^k)^2$). Along a deflating history, $a_4' > 0$ and $\alpha_t < 0$ (for a constant b_4, $\alpha_t = -a_4'$ and $du^4/d\tau = -a_4'(S+E)$), the x_4 velocity can never increase, and it decreases whenever the particle moves in a transverse direction (PROVED; Notebook 03d, In [6]).

The x_4 velocity from the squared speed. The squared speed, written with frame velocities, is

$$g(u,u) = h_0^2(u^0)^2 + \sum_i h_i^2(u^i)^2 - (u^4)^2 - \sum_j h_j^2(u^j)^2 = (\hat{u}^0)^2 + S - (u^4)^2 - E$$

(the diagonal entries $+h_0^2$, $+h_i^2$, -1, $-h_j^2$ of the metric). Setting it equal to -1 and solving for $(u^4)^2$:

$$(u^4)^2 = 1 + (\hat{u}^0)^2 + \sum_i (\hat{u}^i)^2 - \sum_j (\hat{u}^j)^2 = 1 + (\hat{u}^0)^2 + S - E$$

(PROVED, for every a_4 and b_4; Notebook 03d, In [6]).

No turning without extra-time motion; turning with it. A particle with no extra-time motion at the start has $p_j = 0$, so $\hat{u}^j = 0$ for ever (the momenta are conserved), and $(u^4)^2 = 1+(\hat{u}^0)^2+S \geq 1$: u^4 never reaches zero, keeps its sign, and x_4 keeps advancing. A particle that moves along an extra time is different: its E grows by the blueshift, and $(u^4)^2$ can reach zero. There $u^4 = 0$, the **turning point**; there $E = 1 + (\hat{u}^0)^2 + S \geq 1$, so along a deflating history $du^4/d\tau = -a_4' S + \alpha_t E < 0$, u^4 becomes negative and x_4 decreases afterwards. With four time-like directions, a time-like path need not keep advancing in the chosen time x_4 (PROVED; computed in Notebook 03d, In [11]). At the turning point of a particle that moves only along x_5 and y, the formula gives $(\hat{u}^5)^2 - (\hat{u}^0)^2 = 1$.

The same quadratic form as the plane waves of the Revision record. Multiply the last formula by m^2, the squared mass of the particle, and write $k_a = m\hat{u}^a$ for its frame momenta and $\mathcal{E} = mu^4$ for its energy:

$$\mathcal{E}^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2.$$

The Revision scope report finds exactly this expression for the squared frequency of plane waves of the fields of the book in flat 4+4 space (or with the coefficients of the field equation frozen at one instant): a wave with the momenta k_a has $\mathcal{E}^2$ equal to it, and when the extra-time momentum is so large that $\mathcal{E}^2 < 0$ the frequency is imaginary and the wave grows exponentially in x_4, with a rate that has no upper bound (`Revision/theory/reports/python-scope.json`, check `extra_time_growth_rates_unbounded`; Chapter 8 studies this). The identity of the two quadratic forms is PROVED (Notebook 03d, In [7]). A particle can never be where $\mathcal{E}^2 < 0$, because $(u^4)^2 \geq 0$; where a plane wave would start to grow, the particle reaches its turning point instead. Whether wave packets of the fields follow these geodesics in the curved metric is NOT computed in this book: **OPEN**.

The good sector. The Revision Kohn-Sham work keeps only states that do not depend on x_5, x_6, x_7: their extra-time momenta are zero (`Revision/kohn_sham/ks-theory.json`, key `sectorAndAnsatz.sector`: "good sector: no dependence on x5, x6, x7 (extra-time momenta zero)"). The particle with the same property, $p_j = 0$, is the one that never turns. The reason for this choice of the record comes from the quantum theory (Chapter 10).

Numbers along the history (COMPUTED by Notebook 03d). With the history of the record ($A = 1$, $H = 1$, so $a_4 = x_4$, and $b_4 = 1$; ASSUMED, a prescribed background, Section 3.8), RK4 with the step $\Delta\tau = 0.001$, all particles starting at $x_4 = 0$ and $y = -1$ ($z = \arcsin e^{-6} = 0.002479$): a particle at rest stays at rest with $x_4 = \tau$ (In [9]). Three particles started along x_1 with the frame velocities 0.25, 0.5 and 1 keep p_1 and $g(u,u)$ constant within a relative 10^{-9}, their frame

velocity equals $p_1\kappa(y, x_4)$ within 10^{-9}, and u^4 falls at every step; the fastest has at $\tau = 3$ the values $x_4 = 3.3070$, $\hat{u}^1 = 0.014370$, $u^4 = 1.060733$ and $y = -0.0643$: it has slowed down by redshift and risen towards the patch end (In [10]). A particle started along x_5 with $\hat{u}^5 = 0.2$ reaches its turning point at $\tau = 1.987$, where x_4 is largest, 1.4864, with $\hat{u}^5 = 1.4013$ and $\hat{u}^0 = -0.9817$ (it has sunk towards the tip), and x_4 decreases afterwards (In [11]). The numerical acceleration of y agrees with the push law within 3.4×10^{-6} (In [14]), and halving the RK4 step divides the error by 16.2 to 17.2, the order 4 of RK4 (In [16]).

3.32 Example: free fall in the author's metric

Notebook 03d reads the 25 Christoffel symbols of the record, builds the geodesic equations, and proves with sympy, for general $a_4(x_4)$ and $b_4(x_4)$, the conservation of the six momenta and of $g(u, u)$, the momentum weight $\kappa(y, x_4)$ of the Kohn-Sham record, the push law, the law of the x_4 velocity, the formula for $(u^4)^2$, and its identity with the quadratic form of the record's plane-wave check. Then it integrates five paths with RK4 along the history of the record ($b_4 = 1$), checks every exact law on the computed paths, finds the turning point of the particle that moves along an extra time, measures the order of RK4, and draws five figures. It needs no Rust, runs in about 10 seconds and ends with the line ALL 23 CHECKS PASSED (notebook 03d).

3.33 How to run Notebook 03d

Step 1. What this notebook does and what it needs.

Notebook 03d (Free fall in the author's metric: conserved momenta, redshift, blueshift and the push along the hidden direction) is the file `Revision/textbook/notebooks/03d_free_fall.ipynb` of the repository Dirac_claude. It integrates the geodesic equations of free fall in the author's metric with the Christoffel symbols of the Revision record, along the deflating history a4 = A H x4, b4 = 1 with A = 1 and H = 1, by the Runge-Kutta method RK4. With sympy it proves exactly, for general a4(x4) and b4(x4), that the six momenta along x1, x2, x3, x5, x6, x7 and the squared length of the velocity are conserved, that the frame velocity in 3-space falls like e^(-a4) (redshift) while the frame velocity along an extra time grows like e^(a4) b4^(-1/2) (blueshift), that the hidden coordinate y is pushed with the acceleration H times the difference of the squared 3-space and extra-time frame velocities, and the law of the x4 velocity, which can only decrease along a deflating history; it reproduces the momentum weight of the Revision Kohn-Sham record and the quadratic form of the plane-wave check of the Revision scope report. It checks all of this on the computed paths, finds the turning point in x4 of a particle that moves along an extra time, measures the order 4 of RK4, and draws 5 figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at

a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are

those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 03d_free_fall.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 50 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 03d_free_fall.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 03d_free_fall.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/03d.captions.json`

- `Revision/textbook/figures/03d_1_frame_velocities.png`
- `Revision/textbook/figures/03d_2_x4_velocity.png`
- `Revision/textbook/figures/03d_3_hidden_push.png`
- `Revision/textbook/figures/03d_4_quadratic_form.png`
- `Revision/textbook/figures/03d_5_accuracy.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files exist
ALL 23 CHECKS PASSED (notebook 03d)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/03d_free_fall.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 03d_free_fall.ipynb
python -m nbconvert --execute --inplace 03d_free_fall.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for curvature.json, parameters.json, ks-theory.json or python-scope.json: the notebook reads these four Revision records of the repository, so the folder Revision of your copy is incomplete. Restore it with the following command in the repository folder (or clone the repository again) and run the notebook again:

```
git checkout -- Revision
```

3.34 Notebook 03d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/03d_free_fall.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Notebooks 03a and 03b described the author's metric and its curvature. This notebook
asks what the metric DOES: how does a small body move when no force acts on it, only
the geometry? Such a body is in **free fall**, and its path is a **geodesic**. The
notebook

- reads the 25 non-zero Christoffel symbols of the Revision record
  `Revision/gkd_lovelock/results/curvature.json` and builds from them the geodesic
  equations, 16 first-order equations for the 8 coordinates and the 8 velocities;
```

- proves exactly, with sympy, what stays constant during free fall: the six
  momenta along x_1, x_2, x_3 and x_5, x_6, x_7 and the squared length of the
  velocity;
- proves that the velocity of a body moving in 3-space, measured with rulers and
  clocks at its place, falls like e^{-a_4} at a fixed height in the hidden
  direction (**redshift**: 3-space inflates), while its velocity along an extra time
  grows like $e^{a_4} b_4^{-1/2}$ (**blueshift**: the extra times deflate), and
  reproduces the
  momentum weight $\kappa = e^{-Hy - a_4}$ of the Revision Kohn-Sham record;
- proves that motion in 3-space pushes the body towards the patch end of the hidden
  direction, motion along an extra time towards the tip, and finds the law of the
  velocity along the time x_4, which can only decrease along a deflating history;
- shows that the squared x_4 velocity of the body is the same quadratic form as the
  squared frequency of a plane wave in the Revision record's check
  `extra_time_growth_rates_unbounded`;
- integrates the geodesic equations numerically (RK4) for a body at rest, three
  bodies moving in 3-space and one body moving along an extra time, along the
  deflating history $a_4 = AHx_4$, $b_4 = 1$ ($A = 1$, $H = 1$), checks every exact
  law on the
  computed paths, finds the instant at which the last body stops advancing in x_4,
  and measures the accuracy of RK4;
- draws 5 figures and prints a PASS line for every check.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 03d (Free fall in the author's metric: conserved momenta, redshift, blueshift
and the push along the hidden direction) is the file
`Revision/textbook/notebooks/03d_free_fall.ipynb` of the repository Dirac_claude. It
integrates the geodesic equations of free fall in the author's metric with the
Christoffel symbols of the Revision record, along the deflating history a4 = A H x4, b4 =
1 with A = 1 and H = 1, by the Runge-Kutta method RK4. With sympy it proves exactly, for
general a4(x4) and b4(x4), that the six momenta along x1, x2, x3, x5, x6, x7 and the
squared length of the velocity are conserved, that the frame velocity in 3-space falls
like e^(-a4) (redshift) while the frame velocity along an extra time grows like e^(a4)
b4^(-1/2) (blueshift), that the hidden coordinate y is pushed with the acceleration H
times the difference of the squared 3-space and extra-time frame velocities, and the law
of the x4 velocity, which can only decrease along a deflating history; it reproduces the
momentum weight of the Revision Kohn-Sham record and the quadratic form of the plane-wave
check of the Revision scope report. It checks all of this on the computed paths, finds
the turning point in x4 of a particle that moves along an extra time, measures the order
4 of RK4, and draws 5 figures. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
notebooks were built with Python 3.14.5) with these packages at exactly these versions:
numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**
```

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the

packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 03d_free_fall.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 50 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 03d_free_fall.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 03d_free_fall.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/03d.captions.json‘
- ‘Revision/textbook/figures/03d_1_frame_velocities.png‘
- ‘Revision/textbook/figures/03d_2_x4_velocity.png‘
- ‘Revision/textbook/figures/03d_3_hidden_push.png‘

- ‘Revision/textbook/figures/03d_4_quadratic_form.png‘
- ‘Revision/textbook/figures/03d_5_accuracy.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files exist
ALL 23 CHECKS PASSED (notebook 03d)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/03d_free_fall.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 03d_free_fall.ipynb
  python -m nbconvert --execute --inplace 03d_free_fall.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event

  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for curvature.json, parameters.json, ks-theory.json or python-scope.json: the notebook reads these four Revision records of the repository, so the folder Revision of your copy is incomplete. Restore it with the following command in the repository folder (or clone the repository again) and run the notebook again:

      git checkout -- Revision

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 03d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 03d (Free fall in the author's metric: conserved momenta, redshift, blueshift
# and the push along the hidden direction) is the file
# Revision/textbook/notebooks/03d_free_fall.ipynb of the repository Dirac_claude. It
# integrates the geodesic equations of free fall in the author's metric with the
# Christoffel symbols of the Revision record, along the deflating history a4 = A H x4, b4
# = 1 with A = 1 and H = 1, by the Runge-Kutta method RK4. With sympy it proves exactly,
# for general a4(x4) and b4(x4), that the six momenta along x1, x2, x3, x5, x6, x7 and
# the squared length of the velocity are conserved, that the frame velocity in 3-space
# falls like e^(-a4) (redshift) while the frame velocity along an extra time grows like
# e^(a4) b4^(-1/2) (blueshift), that the hidden coordinate y is pushed with the
# acceleration H times the difference of the squared 3-space and extra-time frame
# velocities, and the law of the x4 velocity, which can only decrease along a deflating
# history; it reproduces the momentum weight of the Revision Kohn-Sham record and the
# quadratic form of the plane-wave check of the Revision scope report. It checks all of
# this on the computed paths, finds the turning point in x4 of a particle that moves
# along an extra time, measures the order 4 of RK4, and draws 5 figures. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
```

```
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
```

```
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
```

```
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 03d_free_fall.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 50
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 03d_free_fall.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 03d_free_fall.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/03d.captions.json
# - Revision/textbook/figures/03d_1_frame_velocities.png
# - Revision/textbook/figures/03d_2_x4_velocity.png
# - Revision/textbook/figures/03d_3_hidden_push.png
# - Revision/textbook/figures/03d_4_quadratic_form.png
# - Revision/textbook/figures/03d_5_accuracy.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
```

```
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all five figure files exist
#     ALL 23 CHECKS PASSED (notebook 03d)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/03d_free_fall.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 03d_free_fall.ipynb
#     python -m nbconvert --execute --inplace 03d_free_fall.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for curvature.json, parameters.json, ks-theory.json or
#   python-scope.json: the notebook reads these four Revision records of the repository,
#   so the folder Revision of your copy is incomplete. Restore it with the following
#   command in the repository folder (or clone the repository again) and run the notebook
#   again:
#     git checkout -- Revision
#
# ------------------------------------------------------------------------------------
```

```
# ================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "03d"  # this notebook: chapter 03, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
```

```
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 03d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$ (the author's names): $x_0$ the **hidden**
  direction, $x_1, x_2, x_3$ ordinary 3-space, $x_4$ the time, $x_5, x_6, x_7$ the
  three **extra times**; $z = 6Hx_0$ with $0 < z < \pi/2$. In Python lists they are
  counted 0 to 7 as well: $x_k$ is entry $k$, its velocity $u^k$ too.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: the proper distance along the
  hidden direction, $y = 0$ at the **patch end** $z = \pi/2$ and $y \to -\infty$ at
  the **tip** $z \to 0$.
- **Free fall**: motion under no force other than the geometry. **Test particle**: a
  body so small that it does not change the metric.
- **Proper time** $\tau$: the time shown by a clock carried by the body.
- **Velocity** $u^a = dx^a/d\tau$: how fast each coordinate changes per unit of
  proper time.
- **Geodesic equation**: $du^a/d\tau = -\sum_{b,c} \Gamma^a{}_{bc}\, u^b u^c$, with
  the Christoffel symbols $\Gamma^a{}_{bc}$ of the metric; its solutions are the
  paths of free fall.
- **Squared length of the velocity**: $g(u, u) = \sum_a g_{aa} (u^a)^2$ (the metric
  is diagonal). For a body with mass it equals $-1$ when $\tau$ is its proper time.
- **Scale factor** $h_a = \sqrt{|g_{aa}|}$ and **frame velocity**
  $\hat u^a = h_a u^a$: the proper length (along an extra time: the proper
  duration) that the body covers along $x_a$ per unit of its own proper time,
  measured with the rulers and clocks of an observer at rest at its place; divided
  by $u^4$ it is the velocity that this observer measures.
- **Momentum** $p_a = g_{aa} u^a$ (no sum) of a direction $x_a$. **Conserved**: the
  same number at every point of the path.
- **Redshift, blueshift**: the decrease, increase of a frame velocity (or momentum)
  caused by the expansion, contraction of the space it points along.
- **History** $a_4 = AHx_4$, $b_4 = 1$: the time dependence of the metric used for
  the numbers, the one of the Revision Kohn-Sham work ($A = 1$, $H = 1$); with $A > 0$
  the extra times deflate. It is a **prescribed background**: assumed, not solved
  for.
- **RK4**: the classical Runge-Kutta method of order 4; halving its step makes its
  error about $2^4 = 16$ times smaller.
```

```
- **Turning point**: the instant at which the $x_4$ velocity $u^4$ passes through
  zero, so that $x_4$ stops growing.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's metric is diagonal,
$g = \mathrm{diag}(h_0^2, h_1^2, h_2^2, h_3^2, -1, -h_5^2, -h_6^2, -h_7^2)$ with
the scale factors $h_0 = \cot z$, $h_1 = h_2 = h_3 = e^{a_4}\sin^{1/6}z$ (3-space,
inflating) and $h_5 = h_6 = h_7 = b_4^{1/2} e^{-a_4}\sin^{1/6}z$ (the extra times,
deflating when $b_4$ is constant). A test particle with mass falls freely along a
solution of the **geodesic equations**

$$\frac{dx^a}{d\tau} = u^a, \qquad \frac{du^a}{d\tau} = -\sum_{b,c}
\Gamma^a{}_{bc}\, u^b u^c \qquad (a = 0, \dots, 7),$$

sixteen first-order equations. The Christoffel symbols are those of the Revision
record (Notebook 03b checked them component by component). The metric depends only
on $x_4$ and $x_0$; for each of the other six directions this makes a momentum
$p_a = g_{aa} u^a$ conserved (section 6). The squared length $g(u, u)$ of the
velocity is conserved too, and we choose it equal to $-1$: then $\tau$ is the
proper time of the particle and the path is time-like.

For the numbers the notebook uses the deflating history $a_4 = AHx_4$, $b_4 = 1$ of
the Revision Kohn-Sham work with $A = 1$ and $H = 1$ (so $a_4' = 1$, $b_4' = 0$). It
is a prescribed background, and the particles are test particles: they move in the
given metric and do not change it. Everything in sections 6 to 9 holds for general
functions $a_4(x_4)$ and $b_4(x_4)$; only the numerical paths of sections 10 to 12
use the history.
```

Text cell:

```
## 5. The Christoffel symbols, read from the record

First a technical step. While a cell runs, Jupyter sends what the cell prints to the
notebook in pieces, about one piece every 0.2 seconds, so where one piece ends and
the next begins depends on the speed of the computer. The tools that build this
book read the PASS line of a check together with the line under it that names the
Revision record, and they read the two correctly only when both arrive in the same
piece. The next cell therefore defines the function `in_one_piece(helper)`: it
returns a new function that does exactly what `helper` does, but lets `helper`
print into a text buffer in memory (an `io.StringIO`, with
`contextlib.redirect_stdout`) and then writes the whole buffer with one call of
`sys.stdout.write`, which arrives as one piece. (The stars in `*arguments` and
`**options` pass on all the values the new function is given, whatever they are.)
The cell then replaces the helpers `check` and `report` of the set-up cell by such
functions. Nothing else changes: the same checks, the same PASS lines, the same
numbers.
```

In [2]:

```
import contextlib  # redirect_stdout: send printed text into a buffer
import io  # StringIO: a text buffer in memory
import sys  # sys.stdout: the place where printed text goes


def in_one_piece(helper):
    """A function that does what helper does, with all the lines that helper prints
    written in one piece."""
    def helper_in_one_piece(*arguments, **options):
        collected = io.StringIO()  # an empty text buffer
        with contextlib.redirect_stdout(collected):  # print() writes into it
            helper(*arguments, **options)  # a failing check stops here, as before
        sys.stdout.write(collected.getvalue())  # all the lines with one write
    return helper_in_one_piece


check = in_one_piece(check)  # a PASS line and its record line stay together
report = in_one_piece(report)  # a long RESULT line and its second line, too
say("From now on check and report print each of their results in one piece.")
```

Out [2]:

```
From now on check and report print each of their results in one piece.
```

Text cell:

```
The next cell imports the packages, makes the symbols (the eight coordinates, the
constant $H > 0$, the functions $a_4(x_4)$ and $b_4(x_4)$, and the printing names
‘a4p‘ for $a_4’$, ‘a4v‘ for the value of $a_4$, ‘b4p‘ and ‘b4v‘ the same for $b_4$),
and reads the list ‘christoffelNonzero_b_le_c‘ of
the record. Each entry gives the three indices ‘a‘, ‘b‘, ‘c‘ of
$\Gamma^a{}_{bc}$ (with $b \le c$) and its value in Mathematica notation, which the
function ‘from_mathematica‘ translates into sympy (the same translation as in
Notebook 03b: ‘Derivative[1][a4][x4]‘ becomes ‘a4p‘, ‘a4[x4]‘ becomes ‘a4v‘, the
same for $b_4$, square brackets of functions become round ones, ‘^‘ becomes ‘**‘).
Because $\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$, every entry with $b < c$ is stored
twice, once for each order of the lower indices: the 25 entries give 37 symbols. The
cell prints four of them.
```

In [3]:

```
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
from sympy.parsing.sympy_parser import parse_expr  # text -> sympy expression

CURVATURE_RECORD = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE_RECORD).read_text(encoding="utf-8"))
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
X = sp.symbols("x0:8", real=True)  # the coordinates x0 ... x7 (X[0] ... X[7])
x4, x0 = X[4], X[0]
H = sp.symbols("H", positive=True)  # the constant of the author
```

```
a4 = sp.Function("a4")(x4)  # the metric function a4, any function of x4
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0
a4p, a4v, b4p = sp.symbols("a4p a4v b4p", real=True)  # printing names
b4v = sp.symbols("b4v", positive=True)  # printing name of the value of b4
record_names = {"a4p": a4p, "a4v": a4v, "b4p": b4p, "b4v": b4v, "H": H, "x0": x0,
                "E": sp.E, "sin": sp.sin, "cot": sp.cot}


def from_mathematica(entry_text):
    """A value of the record (Mathematica notation) as a sympy expression."""
    t = entry_text.replace("Derivative[1][a4][x4]", "a4p").replace("a4[x4]", "a4v")
    t = t.replace("Derivative[1][b4][x4]", "b4p").replace("b4[x4]", "b4v")
    t = t.replace("Sin[6*H*x0]", "sin(6*H*x0)").replace("Cot[6*H*x0]", "cot(6*H*x0)")
    t = t.replace("^", "**")
    if "[" in t or "]" in t:  # a piece of notation that was not translated
        raise ValueError(f"cannot translate {entry_text}")
    return parse_expr(t, local_dict=record_names)


Gamma = {}  # (a, b, c) -> the value of Gamma^a_bc, with a4p, a4v, b4p, b4v
for entry in record["christoffelNonzero_b_le_c"]:
    a, b, c = (NAMES.index(entry[key]) for key in ("a", "b", "c"))
    Gamma[(a, b, c)] = from_mathematica(entry["value"])
    Gamma[(a, c, b)] = Gamma[(a, b, c)]  # the same symbol with b and c exchanged
for key in [(1, 1, 4), (4, 5, 5), (0, 1, 1), (0, 0, 0)]:
    a, b, c = key
    say(f"  Gamma^{NAMES[a]}_({NAMES[b]} {NAMES[c]}) = {Gamma[key]}")
report("entries of the record (b <= c)", len(record["christoffelNonzero_b_le_c"]))
check(len(record["christoffelNonzero_b_le_c"]) == 25 and len(Gamma) == 37,
      "the 25 entries of the record give 37 non-zero Christoffel symbols",
      record=f"{CURVATURE_RECORD}, christoffelNonzero_b_le_c")
```

Out [3]:

```
  Gamma^x1_(x1 x4) = a4p
  Gamma^x4_(x5 x5) = a4p*b4v*exp(-2*a4v)*sin(6*H*x0)**(1/3) -
    b4p*exp(-2*a4v)*sin(6*H*x0)**(1/3)/2
  Gamma^x0_(x1 x1) = -H*exp(2*a4v)*sin(6*H*x0)**(1/3)/cot(6*H*x0)
  Gamma^x0_(x0 x0) = -6*H*cot(6*H*x0) - 6*H/cot(6*H*x0)
RESULT entries of the record (b <= c) = 25
PASS the 25 entries of the record give 37 non-zero Christoffel symbols
     reproduces Revision/gkd_lovelock/results/curvature.json, christoffelNonzero_b_le_c
```

Text cell:

```
## 6. What stays constant during free fall (exact)

A quantity $f$ that depends on the position $x$ and the velocity $u$ changes along a
path at the rate given by the chain rule,

$$\frac{df}{d\tau} = \sum_a \frac{\partial f}{\partial x^a}\, u^a
+ \sum_a \frac{\partial f}{\partial u^a}\, \frac{du^a}{d\tau},$$
```

```
and on a path of free fall $du^a/d\tau$ is given by the geodesic equation. The next
cell defines this rate as the function ‘along_path(f)‘, with the record’s
Christoffel symbols for general functions $a_4(x_4)$ and $b_4(x_4)$ (‘a4p‘ becomes
the derivative of $a_4$, ‘a4v‘ the value $a_4(x_4)$, and the same for $b_4$).

By hand, for the momentum $p_1 = g_{11} u^1$ of the direction $x_1$. Line 1, the
chain rule (only $x_4$ and $x_0$ appear in $g_{11}$):
$dp_1/d\tau = (\partial_4 g_{11}\, u^4 + \partial_0 g_{11}\, u^0)\, u^1
+ g_{11}\, du^1/d\tau$. Line 2, the geodesic equation for $u^1$ has the four terms
with $\Gamma^{x_1}{}_{x_1x_4} = \Gamma^{x_1}{}_{x_4x_1} = a_4'$ and
$\Gamma^{x_1}{}_{x_1x_0} = \Gamma^{x_1}{}_{x_0x_1} = H\cot z$:
$du^1/d\tau = -2a_4' u^1 u^4 - 2H\cot z\, u^1 u^0$. Line 3, from
$g_{11} = e^{2a_4}\sin^{1/3}z$: $\partial_4 g_{11} = 2a_4' g_{11}$ and
$\partial_0 g_{11} = \tfrac13 \cdot 6H \cot z\, g_{11} = 2H\cot z\, g_{11}$
(the derivative of $\sin^{1/3} z$ is $\tfrac13 \sin^{-2/3}z \cos z$ times
$dz/dx_0 = 6H$). Line 4, inserting lines 2 and 3 into line 1:
$dp_1/d\tau = 2a_4' g_{11} u^4 u^1 + 2H\cot z\, g_{11} u^0 u^1 - 2a_4' g_{11} u^1 u^4
- 2H\cot z\, g_{11} u^1 u^0 = 0$. The momentum is conserved because the metric does
not depend on $x_1$. The cell lets sympy do the same for all six momenta of
$x_1, x_2, x_3, x_5, x_6, x_7$, and for the squared length
$g(u, u) = \sum_a g_{aa} (u^a)^2$.
```

In [4]:

```
u = sp.symbols("u0:8", real=True)  # the velocity components u^0 ... u^7
# back to the functions a4(x4) and b4(x4)
to_function = {a4p: sp.Derivative(a4, x4), a4v: a4, b4p: sp.Derivative(b4, x4),
               b4v: b4}
acceleration = [sp.S(0)] * 8  # du^a/dtau from the geodesic equation
for (a, b, c), value in Gamma.items():
    acceleration[a] -= value.subs(to_function) * u[b] * u[c]


def along_path(f):
    """The rate of change df/dtau of f(x, u) along a path of free fall."""
    return sum(sp.diff(f, X[k]) * u[k] + sp.diff(f, u[k]) * acceleration[k]
               for k in range(8))


warp = sp.sin(6 * H * x0) ** sp.Rational(1, 6)  # sin(z)^(1/6), z = 6 H x0
# the eight scale factors h_a in the author's order x0, x1, ..., x7
h = [sp.cot(6 * H * x0)] + [sp.exp(a4) * warp] * 3 + [sp.Integer(1)] \
    + [sp.sqrt(b4) * sp.exp(-a4) * warp] * 3
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the signs of the metric entries
g = [ETA[k] * h[k] ** 2 for k in range(8)]  # the diagonal of the metric
say(f"du1/dtau = {sp.factor(acceleration[1]).subs(sp.Derivative(a4, x4), a4p)}")
momenta = {k: g[k] * u[k] for k in (1, 2, 3, 5, 6, 7)}  # p_a = g_aa u^a
rates = [sp.simplify(along_path(p)) for p in momenta.values()]
check(rates == [0] * 6,
      "the six momenta of x1, x2, x3, x5, x6, x7 are conserved in free fall")
squared_length = sum(g[k] * u[k] ** 2 for k in range(8))  # g(u, u)
check(sp.simplify(along_path(squared_length)) == 0,
```

```
      "the squared length g(u, u) of the velocity is conserved in free fall")
```

Out [4]:

```
du1/dtau = -2*u1*(H*u0*cot(6*H*x0) + a4p*u4)
PASS the six momenta of x1, x2, x3, x5, x6, x7 are conserved in free fall
PASS the squared length g(u, u) of the velocity is conserved in free fall
```

Text cell:

```
## 7. Redshift in 3-space, blueshift along the extra times (exact)

The frame velocity is $\hat u^a = h_a u^a$. Since $p_a = g_{aa} u^a = \pm h_a^2 u^a$
(plus for 3-space, minus for the extra times),

$$\hat u^1 = \frac{p_1}{h_1} = p_1\, e^{-a_4} \sin^{-1/6} z, \qquad
\hat u^5 = -\frac{p_5}{h_5} = -p_5\, b_4^{-1/2} e^{a_4} \sin^{-1/6} z .$$

With $p_1$ and $p_5$ constant, the frame velocity in 3-space falls like $e^{-a_4}$
when $a_4$ grows (redshift), and the frame velocity along an extra time grows like
$e^{a_4} b_4^{-1/2}$, like $e^{a_4}$ for a constant $b_4$ (blueshift). In the hidden
coordinate, $\sin^{1/6} z = e^{Hy}$, so
$1/h_1 = e^{-Hy - a_4}$: exactly the **momentum weight**
$\kappa(y, x_4) = e^{-Hy - a_4(x_4)}$ that the Revision Kohn-Sham record uses to turn
a conserved 3-space momentum into the momentum seen at height $y$ and time $x_4$.
The next cell reads that statement of the record and checks it, and checks
$1/h_5 = b_4^{-1/2} e^{a_4 - Hy}$ for the extra times.
```

In [5]:

```
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks_theory = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
say("record, geometry.kappa: " + ks_theory["geometry"]["kappa"])
y_of_x0 = sp.log(sp.sin(6 * H * x0)) / (6 * H)  # the hidden coordinate y
kappa = sp.exp(-H * y_of_x0 - a4)  # the record's e^{-Hy - a4(x4)}
check(sp.simplify(1 / h[1] - kappa) == 0,
      "1/h1 = e^(-H y - a4): the frame velocity in 3-space is p1 times kappa",
      record=f"{KS_THEORY}, geometry.kappa")
check(sp.simplify(1 / h[5] - sp.exp(a4 - H * y_of_x0) / sp.sqrt(b4)) == 0,
      "1/h5 = b4^(-1/2) e^(a4 - H y): the frame velocity along an extra time grows "
      "with a4")
```

Out [5]:

```
record, geometry.kappa: kappa(y, x4) = e^{-Hy - a4(x4)} (the momentum weight)
PASS 1/h1 = e^(-H y - a4): the frame velocity in 3-space is p1 times kappa
     reproduces Revision/kohn_sham/ks-theory.json, geometry.kappa
PASS 1/h5 = b4^(-1/2) e^(a4 - H y): the frame velocity along an extra time grows with a4
```

Text cell:

```
## 8. The push along the hidden direction and the slowing of $x_4$ (exact)

Two more exact laws follow from the record's symbols. First the hidden direction.
The geodesic equation for $u^0$ has the terms with
$\Gamma^{x_0}{}_{x_ix_i} = -H h_i^2 \tan z$ (3-space, $i = 1, 2, 3$),
$\Gamma^{x_0}{}_{x_jx_j} = +H h_j^2 \tan z$ (extra times, $j = 5, 6, 7$; with the
factor $b_4$ inside $h_j^2$) and $\Gamma^{x_0}{}_{x_0x_0} = -6H(\tan z + \cot z)$, so
$du^0/d\tau = H\tan z\,(\sum_i (\hat u^i)^2 - \sum_j (\hat u^j)^2)
+ 6H(\tan z + \cot z)(u^0)^2$. The frame velocity along $x_0$ is
$\hat u^0 = \cot z\, u^0 = dy/d\tau$ (because $dy/dx_0 = \cot z$). Its rate is
$d\hat u^0/d\tau = -6H (u^0)^2/\sin^2 z + \cot z\, du^0/d\tau$ (the product rule,
with $d\cot z/d\tau = -6H u^0/\sin^2 z$); inserting $du^0/d\tau$ and using
$\cot z \tan z = 1$ and $1 + \cot^2 z = 1/\sin^2 z$, the $(u^0)^2$ terms cancel:

$$\frac{d^2 y}{d\tau^2} = \frac{d\hat u^0}{d\tau}
= H\Bigl(\sum_{i=1}^{3} (\hat u^i)^2 - \sum_{j=5}^{7} (\hat u^j)^2\Bigr).$$

Motion in 3-space pushes the particle towards the patch end ($y$ grows), motion
along an extra time towards the tip ($y$ falls). Second, the time $x_4$: the only
symbols with upper index $x_4$ are $\Gamma^{x_4}{}_{x_ix_i} = a_4' h_i^2$ and
$\Gamma^{x_4}{}_{x_jx_j} = -\alpha_t h_j^2$, with the expansion rate
$\alpha_t = -a_4' + b_4'/(2b_4)$ of the extra times (Notebook 03a), so

$$\frac{du^4}{d\tau} = -a_4' \sum_{i=1}^{3} (\hat u^i)^2
+ \alpha_t \sum_{j=5}^{7} (\hat u^j)^2 :$$

for a deflating history ($a_4' > 0$ and $\alpha_t < 0$; with a constant $b_4$,
$\alpha_t = -a_4'$ and $du^4/d\tau = -a_4'(\sum_i (\hat u^i)^2 + \sum_j
(\hat u^j)^2)$) the $x_4$ velocity can only decrease. Third, in frame velocities the
squared length reads
$g(u, u) = (\hat u^0)^2 + \sum_i (\hat u^i)^2 - (u^4)^2 - \sum_j (\hat u^j)^2$, so
with $g(u, u) = -1$:

$$(u^4)^2 = 1 + (\hat u^0)^2 + \sum_{i=1}^{3} (\hat u^i)^2
- \sum_{j=5}^{7} (\hat u^j)^2 .$$

The next cell checks all three with sympy for general $a_4(x_4)$ and $b_4(x_4)$.
```

In [6]:

```
frame = [h[k] * u[k] for k in range(8)]  # the frame velocities u-hat^a = h_a u^a
space_squared = sum(frame[k] ** 2 for k in (1, 2, 3))  # 3-space part
extra_squared = sum(frame[k] ** 2 for k in (5, 6, 7))  # extra-time part
push = along_path(frame[0]) - H * (space_squared - extra_squared)
check(sp.simplify(push) == 0,
      "d^2y/dtau^2 = H (3-space frame velocity squared - extra-time one squared)")
alpha_t = -sp.diff(a4, x4) + sp.diff(b4, x4) / (2 * b4)  # the extra-time rate
slowing = (along_path(u[4]) + sp.diff(a4, x4) * space_squared
           - alpha_t * extra_squared)
check(sp.simplify(slowing) == 0,
      "du4/dtau = -a4' (3-space frame velocities squared) + alpha_t (extra-time ones "
      "squared)")
in_frame = frame[0] ** 2 + space_squared - u[4] ** 2 - extra_squared
```

```
check(sp.simplify(squared_length - in_frame) == 0,
      "g(u, u) written with the frame velocities")
```

Out [6]:

```
PASS d^2y/dtau^2 = H (3-space frame velocity squared - extra-time one squared)
PASS du4/dtau = -a4' (3-space frame velocities squared) + alpha_t (extra-time ones
    squared)
PASS g(u, u) written with the frame velocities
```

Text cell:

```
## 9. The same quadratic form as the plane waves of the Revision record

Multiply the last formula by $m^2$, the squared mass of the particle, and call
$k_a = m\,\hat u^a$ its frame momenta and $E = m\,u^4$ its energy: then
$E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$. The Revision
scope report `Revision/theory/reports/python-scope.json` has, in its check
`extra_time_growth_rates_unbounded`, exactly this expression for plane waves of the
fields in flat 4+4 space (or with the coefficients of the equation frozen at one
instant): a wave $e^{i(k \cdot x - E x_4)}$ has $E^2$ equal to it, and when the
extra-time momentum is so large that $E^2 < 0$, $E$ is imaginary and the wave grows
exponentially in $x_4$ instead of oscillating. The next cell cuts the expression out
of the record's text, turns it into sympy and compares it with $m^2 (u^4)^2$ of the
particle. For the particle $(u^4)^2$ can never be negative; where a plane wave would
start to grow, the particle instead reaches $u^4 = 0$, its **turning point** in
$x_4$ (section 11). (That a wave packet of the fields follows such a path in the
curved metric is not computed here.)
```

In [7]:

```
SCOPE = "Revision/theory/reports/python-scope.json"
scope = json.loads(repository_file(SCOPE).read_text(encoding="utf-8"))
growth = next(entry for entry in scope["checks"]
              if entry["name"] == "extra_time_growth_rates_unbounded")
form_text = growth["detail"].split("h_k^2 = (")[1].split(") I16")[0]  # the formula
say(f"record ({growth['verdict']}): E^2 = {form_text}")
k = sp.symbols("k0:8", real=True)  # frame momenta k0 ... k7 (k4 is not used)
m = sp.symbols("m", positive=True)  # the mass
form = parse_expr(form_text, local_dict={**{f"k{n}": k[n] for n in range(8)},
                                         "m": m})
U = sp.symbols("U0:8", real=True)  # the frame velocities as plain symbols
particle = m ** 2 * (1 + U[0] ** 2 + U[1] ** 2 + U[2] ** 2 + U[3] ** 2 - U[5] ** 2
                     - U[6] ** 2 - U[7] ** 2)  # m^2 (u^4)^2 from section 8
same_form = sp.expand(form.subs({k[n]: m * U[n] for n in range(8)}) - particle)
check(growth["verdict"] == "PASS" and same_form == 0,
      "m^2 (u4)^2 of the particle is the plane-wave E^2 of the record",
      record=f"{SCOPE}, check extra_time_growth_rates_unbounded")
```

Out [7]:

```
record (PASS): E^2 = k0**2 + k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + m**2
PASS m^2 (u4)^2 of the particle is the plane-wave E^2 of the record
     reproduces Revision/theory/reports/python-scope.json, check
    extra_time_growth_rates_unbounded
```

Text cell:

```
## 10. Free fall computed step by step (RK4)

Now the numbers. The next cell reads the history of the Revision Kohn-Sham record
($H = 1$, slope $A = 1$, so $a_4 = x_4$ and $a_4' = 1$; and $b_4 = 1$) and its
statement that this
history is a prescribed background, and builds the tools of the computation:

- ‘gamma_numbers‘: the 37 symbols as fast numerical functions of $a_4$, $a_4'$,
  $x_0$, $H$, $b_4$ and $b_4'$ (‘sp.lambdify‘ with the module ‘math‘);
- ‘free_fall(state)‘: the right-hand side of the 16 equations for the state
  $(x^0, \dots, x^7, u^0, \dots, u^7)$ on the history, with $b_4 = 1$, $b_4' = 0$
  (entry $k$ of the state is $x^k$ and entry $8 + k$ is $u^k$);
- ‘rk4_path(state0, tau_end, steps)‘: the RK4 method, the same as in chapter 2,
  from $\tau = 0$ to ‘tau_end‘ in ‘steps‘ equal steps;
- ‘start(y0, frame_velocity)‘: the state at $\tau = 0$ at the point
  $x_4 = 0$, $y = y_0$ (all other coordinates 0) with the given frame velocities;
  $u^4$ is chosen so that $g(u, u) = -1$, by the last formula of section 8;
- ‘describe(states)‘: for every computed state the frame velocities, $y$ and
  $g(u, u)$.

All particles start at $y_0 = -1$ (that is $z_0 = \arcsin e^{-6} = 0.0025$), well
inside the patch, on the slice $x_4 = 0$ where $a_4 = 0$.
```

In [8]:

```
import math  # sin, tan, exp, ... for single numbers (faster than numpy here)

PARAMETERS = "Revision/kohn_sham/results/parameters.json"
physics = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
H_value, A_value = physics["physics"]["H"], physics["physics"]["historyA"]
status = ks_theory["adiabaticity"]["historyStatus"]
say(f"H = {H_value}, A = {A_value}; record: " + status.split(". ")[0] + ".")
check(H_value == 1.0 and A_value == 1.0 and status.startswith("PRESCRIBED BACKGROUND"),
      "the history of the record: a4 = A H x4 with A = 1, H = 1, prescribed",
      record=f"{PARAMETERS}, physics.H and physics.historyA; {KS_THEORY}, "
             "adiabaticity.historyStatus")
ARGUMENTS = (a4v, a4p, x0, H, b4v, b4p)
gamma_numbers = [(a, b, c, sp.lambdify(ARGUMENTS, value, "math"))
                 for (a, b, c), value in sorted(Gamma.items())]
ETA_N = np.array(ETA, dtype=float)  # the signs as numbers


def free_fall(state):
    """d state/d tau for state = (x0..x7, u0..u7) on the history a4 = A H x4,
    b4 = 1."""
```

```
    x, v = state[:8], state[8:]
    a4_now, rate = A_value * H_value * x[4], A_value * H_value  # a4 and a4'
    change = np.zeros(8)
    for a, b, c, f in gamma_numbers:  # du^a/dtau = - sum Gamma^a_bc u^b u^c
        change[a] -= f(a4_now, rate, x[0], H_value, 1.0, 0.0) * v[b] * v[c]
    return np.concatenate([v, change])


def rk4_path(state0, tau_end, steps):
    """RK4 from tau = 0 to tau_end; returns the times and the states (rows)."""
    dt = tau_end / steps
    states = np.zeros((steps + 1, 16))
    states[0] = state0
    for n in range(steps):
        s = states[n]
        k1 = free_fall(s)
        k2 = free_fall(s + dt / 2 * k1)
        k3 = free_fall(s + dt / 2 * k2)
        k4 = free_fall(s + dt * k3)
        states[n + 1] = s + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return np.linspace(0.0, tau_end, steps + 1), states


def scale_numbers(x):
    """The eight scale factors h_a at the point x on the history (numbers)."""
    a4_now, z_now = A_value * H_value * x[4], 6 * H_value * x[0]
    w = math.sin(z_now) ** (1 / 6)  # the warp factor sin(z)^(1/6)
    return np.array([1 / math.tan(z_now)] + [math.exp(a4_now) * w] * 3 + [1.0]
                    + [math.exp(-a4_now) * w] * 3)  # with b4 = 1


def start(y0, frame_velocity):
    """The state at tau = 0: x4 = 0, y = y0, frame velocities as given (the entry
    for x4 is replaced by the value that makes g(u, u) = -1)."""
    x = np.zeros(8)
    x[0] = math.asin(math.exp(6 * H_value * y0)) / (6 * H_value)  # sin z = e^(6Hy)
    w = np.array(frame_velocity, dtype=float)
    w[4] = 0.0
    w[4] = math.sqrt(1.0 + np.sum(ETA_N * w ** 2))  # (u^4)^2 = 1 + sum eta w^2
    return np.concatenate([x, w / scale_numbers(x)])  # u^a = u-hat^a / h_a


def describe(states):
    """Frame velocities (rows), the coordinate y and g(u, u) along a path."""
    h_rows = np.array([scale_numbers(s[:8]) for s in states])
    frame_rows = h_rows * states[:, 8:]
    y_values = np.log(np.sin(6 * H_value * states[:, 0])) / (6 * H_value)
    lengths = np.sum(ETA_N * frame_rows ** 2, axis=1)  # g(u, u)
    return frame_rows, y_values, lengths, h_rows


Y0 = -1.0  # the starting height of every particle
say(f"start: y0 = {Y0}, z0 = {math.asin(math.exp(6 * Y0)):.6f}")
```

Out [8]:

```
H = 1.0, A = 1.0; record: PRESCRIBED BACKGROUND: the history a4 = A H x4 is prescribed,
    not solved for.
PASS the history of the record: a4 = A H x4 with A = 1, H = 1, prescribed
     reproduces Revision/kohn_sham/results/parameters.json, physics.H and
    physics.historyA; Revision/kohn_sham/ks-theory.json, adiabaticity.historyStatus
start: y0 = -1.0, z0 = 0.002479
```

Text cell:

```
**The particle at rest.** The first particle starts at rest: all frame velocities
are $0$ except $u^4 = 1$. Notebook 03b showed that $\Gamma^a{}_{x_4x_4} = 0$ for
every $a$, so it must stay at rest, with $x_4 = \tau$: its clock shows the time
$x_4$. The next cell integrates its path from $\tau = 0$ to $\tau = 3$ in 3000
steps and checks this.
```

In [9]:

```
tau, rest = rk4_path(start(Y0, [0.0] * 8), 3.0, 3000)
others = [0, 1, 2, 3, 5, 6, 7]  # every coordinate except x4
moved = np.max(np.abs(rest[:, others] - rest[0, others]))
check(moved == 0.0 and np.max(np.abs(rest[:, 4] - tau)) < 1e-12
      and np.max(np.abs(rest[:, 12] - 1.0)) < 1e-12,
      "the particle at rest stays at rest and x4 equals its proper time")
```

Out [9]:

```
PASS the particle at rest stays at rest and x4 equals its proper time
```

Text cell:

```
**Three particles moving in 3-space.** The next three particles start with the
frame velocities $\hat u^1 = 0.25$, $0.5$ and $1$ along $x_1$, so
$u^4 = \sqrt{1 + (\hat u^1)^2}$. (The velocity that an
observer at rest at the particle's place measures is $\hat u^1/u^4$; for
$\hat u^1 = 1$ it is $1/\sqrt 2 = 0.71$ times the velocity of light, which is $1$
in these units.) The next cell integrates each from $\tau = 0$ to $3$ in 3000
steps and checks on every computed state: the momentum $p_1 = g_{11} u^1$ stays
constant (relative change below $10^{-9}$); $g(u, u)$ stays $-1$; the frame velocity
equals $p_1 \kappa(y, x_4)$ (section 7); $u^4$ decreases at every step (section 8);
the particle stays inside the patch ($y < 0$). It reports the final values for the
fastest particle.
```

In [10]:

```
SPEEDS = (0.25, 0.5, 1.0)  # starting frame velocities along x1
space_paths = {}
```

```
for speed in SPEEDS:
    tau, states = rk4_path(start(Y0, [0, speed, 0, 0, 0, 0, 0, 0]), 3.0, 3000)
    frame_rows, y_values, lengths, h_rows = describe(states)
    p1 = h_rows[:, 1] ** 2 * states[:, 9]  # p_1 = g_11 u^1 = h_1^2 u^1
    weight = np.exp(-H_value * y_values - A_value * H_value * states[:, 4])  # kappa
    space_paths[speed] = (states, frame_rows, y_values, lengths, p1)
    check(np.max(np.abs(p1 / p1[0] - 1)) < 1e-9
          and np.max(np.abs(lengths + 1)) < 1e-9
          and np.max(np.abs(frame_rows[:, 1] - p1[0] * weight)) < 1e-9
          and np.all(np.diff(states[:, 12]) < 0) and np.all(y_values < 0),
          f"3-space speed {speed}: p1 and g(u, u) conserved, frame velocity "
          "p1 kappa, u4 falls")
states, frame_rows, y_values, lengths, p1 = space_paths[1.0]
report("fastest particle at tau = 3: x4", f"{states[-1, 4]:.4f}")
report("fastest particle at tau = 3: frame velocity along x1",
       f"{frame_rows[-1, 1]:.6f}")
report("fastest particle at tau = 3: u4", f"{states[-1, 12]:.6f}")
report("fastest particle at tau = 3: height y", f"{y_values[-1]:.4f}")
```

Out [10]:

```
PASS 3-space speed 0.25: p1 and g(u, u) conserved, frame velocity p1 kappa, u4 falls
PASS 3-space speed 0.5: p1 and g(u, u) conserved, frame velocity p1 kappa, u4 falls
PASS 3-space speed 1.0: p1 and g(u, u) conserved, frame velocity p1 kappa, u4 falls
RESULT fastest particle at tau = 3: x4 = 3.3070
RESULT fastest particle at tau = 3: frame velocity along x1 = 0.014370
RESULT fastest particle at tau = 3: u4 = 1.060733
RESULT fastest particle at tau = 3: height y = -0.0643
```

Text cell:

```
## 11. A particle moving along an extra time: the turning point

The last particle starts with the frame velocity $\hat u^5 = 0.2$ along the extra
time $x_5$, so $u^4 = \sqrt{1 - 0.04} = 0.980$ (rounded). Along the deflating
history ($b_4 = 1$) its frame velocity $-p_5\, e^{a_4 - Hy}$ grows, it is pushed
towards the tip, and by section 8 its $x_4$ velocity obeys
$(u^4)^2 = 1 + (\hat u^0)^2 - (\hat u^5)^2$. The
next cell integrates its path from $\tau = 0$ to $2.5$ in 2500 steps, checks the
conservation laws, finds the step at which $u^4$ changes sign (the turning point:
$x_4$ is largest there and decreases afterwards), and checks that there
$(\hat u^5)^2 - (\hat u^0)^2 = 1$, as the formula demands for $u^4 = 0$.

The cell also prints the definition of the **good sector** of the Revision
Kohn-Sham work: its states have no dependence on $x_5, x_6, x_7$, that is no
extra-time momenta (the reason, from the quantum theory, comes in a later chapter).
For a classical particle without extra-time momentum, $(u^4)^2 = 1 + (\hat u^0)^2 +
\sum_i (\hat u^i)^2 \ge 1$: such a particle never has a turning point.
```

In [11]:

```
tau_t, extra = rk4_path(start(Y0, [0, 0, 0, 0, 0, 0.2, 0, 0]), 2.5, 2500)
frame_t, y_t, lengths_t, h_t = describe(extra)
p5 = -h_t[:, 5] ** 2 * extra[:, 13]  # p_5 = g_55 u^5 = -h_5^2 u^5
check(np.max(np.abs(p5 / p5[0] - 1)) < 1e-8 and np.max(np.abs(lengths_t + 1)) < 1e-7
      and np.all(np.diff(extra[:, 12]) < 0),
      "extra-time particle: p5 and g(u, u) conserved, u4 decreases at every step")
n_turn = int(np.argmax(extra[:, 12] < 0))  # the first step with u4 < 0
x4_max = float(np.max(extra[:, 4]))  # the largest x4 of the path
report("turning point: proper time tau", f"{tau_t[n_turn]:.3f}")
report("turning point: largest x4 reached", f"{x4_max:.4f}")
report("turning point: frame velocity along x5", f"{frame_t[n_turn, 5]:.4f}")
report("turning point: frame velocity along y", f"{frame_t[n_turn, 0]:.4f}")
balance = frame_t[n_turn, 5] ** 2 - frame_t[n_turn, 0] ** 2 - 1  # 0 when u4 = 0
check(0 < n_turn and abs(balance) < 1e-4 and extra[-1, 4] < x4_max,
      "at the turning point (u5 hat)^2 - (u0 hat)^2 = 1, and x4 decreases afterwards")
say("record, sectorAndAnsatz.sector: " + ks_theory["sectorAndAnsatz"]["sector"])
check("extra-time momenta zero" in ks_theory["sectorAndAnsatz"]["sector"],
      "the good sector of the Kohn-Sham record has no extra-time momenta",
      record=f"{KS_THEORY}, sectorAndAnsatz.sector")
```

Out [11]:

```
PASS extra-time particle: p5 and g(u, u) conserved, u4 decreases at every step
RESULT turning point: proper time tau = 1.987
RESULT turning point: largest x4 reached = 1.4864
RESULT turning point: frame velocity along x5 = 1.4013
RESULT turning point: frame velocity along y = -0.9817
PASS at the turning point (u5 hat)^2 - (u0 hat)^2 = 1, and x4 decreases afterwards
record, sectorAndAnsatz.sector: good sector: no dependence on x5, x6, x7 (extra-time
    momenta zero); coordinate 3-torus of size ell, k in (2 pi/ell) Z^3 = Delta k Z^3;
    extra-time coordinate volume v_t
PASS the good sector of the Kohn-Sham record has no extra-time momenta
     reproduces Revision/kohn_sham/ks-theory.json, sectorAndAnsatz.sector
```

Text cell:

```
## 12. Pictures of free fall

The next cell draws the frame velocities. Left: the three particles that move in
3-space, their frame velocity $\hat u^1$ against the time $x_4$ on a logarithmic
axis, computed by RK4 (solid lines), together with the pure redshift
$\hat u^1(0)\, e^{-a_4}$ (dotted lines; on a logarithmic axis straight lines of
slope $-1$). The computed curves fall a little faster than $e^{-a_4}$ because the
particles also rise in $y$, where the warp factor $e^{Hy}$ is larger. Right: the
particle moving along the extra time $x_5$, its frame velocity $\hat u^5$ against
$x_4$ up to its turning point, together with the pure blueshift
$0.2\, e^{a_4}$ (dotted); it grows faster than $e^{a_4}$ because the particle sinks
towards the tip, where $e^{-Hy}$ is larger.
```

In [12]:

```
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
MAGENTA, GREEN, VIOLET, RED = "#e87ba4", "#008300", "#4a3aa7", "#e34948"
COLOURS = {0.25: AQUA, 0.5: ORANGE, 1.0: RED}  # one colour per 3-space particle
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)  # room for the label of the right vertical axis
for speed in SPEEDS:
    states, frame_rows = space_paths[speed][0], space_paths[speed][1]
    axes[0].plot(states[:, 4], frame_rows[:, 1], color=COLOURS[speed], lw=1.8,
                 label=f"RK4, start {speed:g}")
    axes[0].plot(states[:, 4], speed * np.exp(-A_value * H_value * states[:, 4]),
                 ":", color=COLOURS[speed], lw=1.4)
axes[0].plot([], [], ":", color="grey", label="pure redshift $e^{-a_4}$")
axes[0].set_yscale("log")
axes[0].set_xlabel("time $x_4$ (unit $1/H$), history $a_4 = x_4$, $b_4 = 1$")
axes[0].set_ylabel("frame velocity $\\hat u^1$ along $x_1$")
axes[0].set_title("3-space: redshift")
axes[0].legend(fontsize=8)
before = slice(0, n_turn)  # the states before the turning point
axes[1].plot(extra[before, 4], frame_t[before, 5], color=VIOLET, lw=1.8,
             label="RK4, start 0.2")
axes[1].plot(extra[before, 4], 0.2 * np.exp(A_value * H_value * extra[before, 4]),
             ":", color=VIOLET, lw=1.4, label="pure blueshift $0.2\\,e^{a_4}$")
axes[1].axvline(x4_max, color="grey", lw=1.0, ls="--", label="turning point")
axes[1].set_yscale("log")
axes[1].set_xlabel("time $x_4$ (unit $1/H$), history $a_4 = x_4$, $b_4 = 1$")
axes[1].set_ylabel("frame velocity $\\hat u^5$ along $x_5$")
axes[1].set_title("Extra time: blueshift")
axes[1].legend(fontsize=8, loc="upper left")
save_figure(fig, "frame_velocities",
            "Frame velocities of freely falling test particles in the author's metric "
            "along the deflating history $a_4 = x_4$, $b_4 = 1$ ($A = 1$, $H = 1$), "
            "all started "
            "at $x_4 = 0$, $y = -1$, computed with RK4 from the Christoffel symbols "
            "of the Revision record; horizontal axes the time $x_4$ in units of "
            "$1/H$, vertical axes the frame velocity (proper length per proper "
            "time, a pure number) on logarithmic scales. Left: three particles "
            "started along $x_1$ with $0.25$, $0.5$ and $1$; their frame velocity "
            "falls, a little faster than the pure redshift $e^{-a_4}$ (dotted), "
            "because they also rise towards the patch end. Right: a particle "
            "started along the extra time $x_5$ with $0.2$; its frame velocity "
            "grows, faster than the pure blueshift $0.2\\,e^{a_4}$ (dotted), until "
            f"its turning point at $x_4 = {x4_max:.3f}$ (dashed).")
check(frame_t[n_turn - 1, 5] > 0.2 * math.exp(extra[n_turn - 1, 4]),
      "the extra-time frame velocity grows faster than the pure blueshift")
```

Out [12]:

```
Figure 03d.1 saved as Revision/textbook/figures/03d_1_frame_velocities.png
PASS the extra-time frame velocity grows faster than the pure blueshift
```

Text cell:

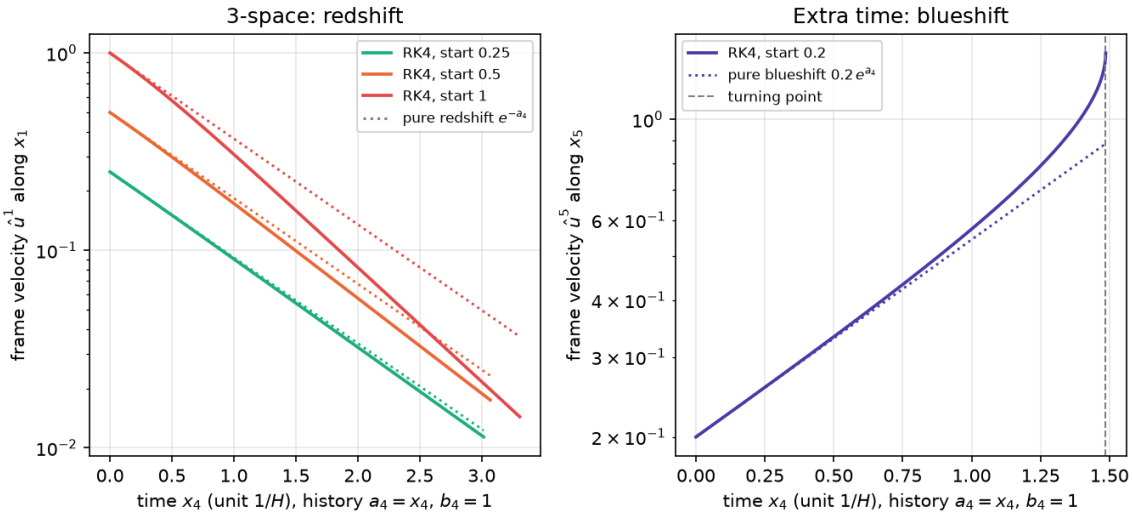

Figure 111: Frame velocities of freely falling test particles in the author's metric along the deflating history $a_4 = x_4$, $b_4 = 1$ ($A = 1$, $H = 1$), all started at $x_4 = 0$, $y = -1$, computed with RK4 from the Christoffel symbols of the Revision record; horizontal axes the time x_4 in units of $1/H$, vertical axes the frame velocity (proper length per proper time, a pure number) on logarithmic scales. Left: three particles started along x_1 with 0.25, 0.5 and 1; their frame velocity falls, a little faster than the pure redshift e^{-a_4} (dotted), because they also rise towards the patch end. Right: a particle started along the extra time x_5 with 0.2; its frame velocity grows, faster than the pure blueshift $0.2\,e^{a_4}$ (dotted), until its turning point at $x_4 = 1.486$ (dashed). (Notebook 03d, figure 1.)

```
The next cell draws the $x_4$ velocity $u^4$ (left) and the time $x_4$ itself
(right) against the proper time $\tau$ for all five particles. The particle at rest
keeps $u^4 = 1$. The particles in 3-space start with $u^4 = \sqrt{1 + (\hat u^1)^2}$
and slow down towards $1$ as their frame velocity is redshifted away. The particle
on the extra time starts with $u^4 = 0.98$, slows down to $0$ at its turning point
and continues with $u^4 < 0$: its $x_4$ reaches a largest value and then decreases.
With four time-like directions a time-like path need not keep advancing in $x_4$.
```

In [13]:

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)
axes[0].plot(tau, rest[:, 12], color="black", lw=1.4, ls="--", label="at rest")
axes[1].plot(tau, rest[:, 4], color="black", lw=1.4, ls="--", label="at rest")
for speed in SPEEDS:
    states = space_paths[speed][0]
    axes[0].plot(tau, states[:, 12], color=COLOURS[speed], lw=1.8,
                 label=f"3-space, start {speed:g}")
    axes[1].plot(tau, states[:, 4], color=COLOURS[speed], lw=1.8,
                 label=f"3-space, start {speed:g}")
axes[0].plot(tau_t, extra[:, 12], color=VIOLET, lw=1.8, ls="-.",
             label="extra time, start 0.2")
axes[1].plot(tau_t, extra[:, 4], color=VIOLET, lw=1.8, ls="-.",
             label="extra time, start 0.2")
axes[0].axhline(0.0, color="grey", lw=0.8)
```

```
axes[0].plot([tau_t[n_turn]], [0.0], "o", color=VIOLET, ms=7)  # the turning point
axes[1].plot([tau_t[n_turn]], [x4_max], "o", color=VIOLET, ms=7)
axes[0].set_ylabel("$x_4$ velocity $u^4 = dx_4/d\\tau$")
axes[1].set_ylabel("time $x_4$ (unit $1/H$)")
axes[0].set_title("The $x_4$ velocity can only decrease")
axes[1].set_title("The time $x_4$ along each path")
for ax, place in zip(axes, ("lower left", "upper left")):  # legends off the lines
    ax.set_xlabel("proper time $\\tau$ of the particle (unit $1/H$)")
    ax.legend(fontsize=8, loc=place)
save_figure(fig, "x4_velocity",
            "The $x_4$ velocity $u^4 = dx_4/d\\tau$ (left) and the time $x_4$ "
            "(right) of five freely falling test particles against their own proper "
            "time $\\tau$ in units of $1/H$, along the deflating history $a_4 = x_4$, "
            "$b_4 = 1$, "
            "computed with RK4: a particle at rest (dashed, $u^4 = 1$ and "
            "$x_4 = \\tau$), three particles moving in 3-space (solid, started with "
            "frame velocities $0.25$, $0.5$, $1$), whose $u^4$ falls towards $1$ as "
            "their motion is redshifted, and a particle moving along an extra time "
            "(dash-dotted, started with $0.2$), whose $u^4$ falls through $0$ at its "
            f"turning point (dot, $\\tau = {tau_t[n_turn]:.2f}$), after which its "
            f"$x_4$, largest there ($x_4 = {x4_max:.3f}$), decreases.")
check(np.all(np.diff(space_paths[1.0][0][:, 4]) > 0),
      "the particles in 3-space keep advancing in x4")
```

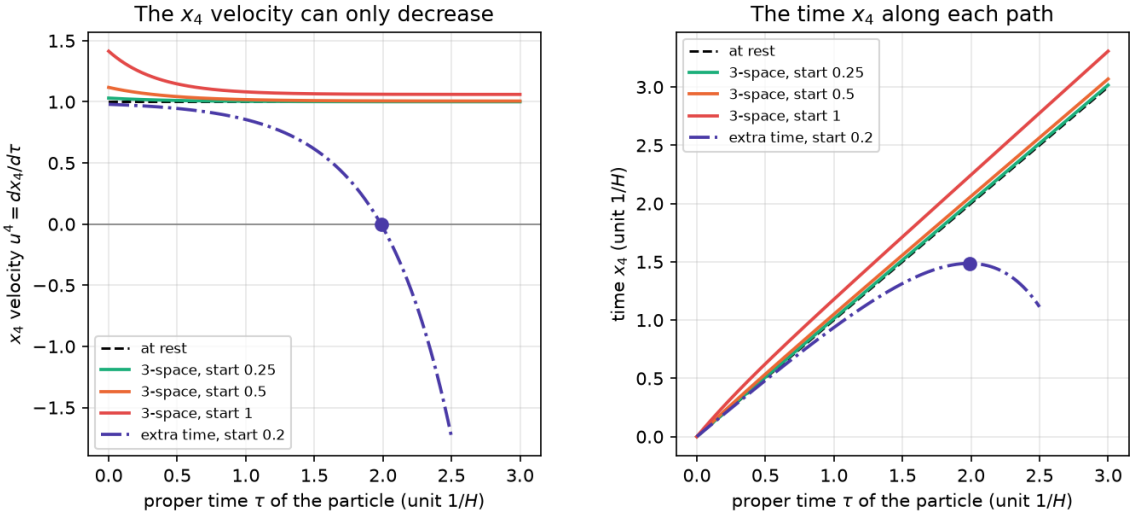

Figure 112: The x_4 velocity $u^4 = dx_4/d\tau$ (left) and the time x_4 (right) of five freely falling test particles against their own proper time τ in units of $1/H$, along the deflating history $a_4 = x_4$, $b_4 = 1$, computed with RK4: a particle at rest (dashed, $u^4 = 1$ and $x_4 = \tau$), three particles moving in 3-space (solid, started with frame velocities 0.25, 0.5, 1), whose u^4 falls towards 1 as their motion is redshifted, and a particle moving along an extra time (dash-dotted, started with 0.2), whose u^4 falls through 0 at its turning point (dot, $\tau = 1.99$), after which its x_4, largest there ($x_4 = 1.486$), decreases. (Notebook 03d, figure 2.)

Out [13]:

```
Figure 03d.2 saved as Revision/textbook/figures/03d_2_x4_velocity.png
```

```
PASS the particles in 3-space keep advancing in x4
```

Text cell:

```
The next cell shows the push along the hidden direction. Left: the height $y$ of
every particle against $\tau$; the particles in 3-space rise towards the patch end
$y = 0$, the particle on the extra time sinks towards the tip; the dashed line
marks the tip cut-off $y = -3$ of the Revision Kohn-Sham solver. Right: the
acceleration $d^2y/d\tau^2$ computed from the RK4 states by a numerical derivative
of $\hat u^0$ (‘np.gradient‘, central differences) as dots, and the exact law
$H(\sum_i (\hat u^i)^2 - \sum_j (\hat u^j)^2)$ of section 8 as lines, for the
fastest 3-space particle and the extra-time particle. The cell checks the agreement.
```

In [14]:

```
L_tip = physics["physics"]["L_tipCutoff"]  # the tip cut-off of the record, 3/H
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)
axes[0].plot(tau, np.full_like(tau, Y0), color="black", lw=1.4, ls="--",
             label="at rest")
for speed in SPEEDS:
    axes[0].plot(tau, space_paths[speed][2], color=COLOURS[speed], lw=1.8,
                 label=f"3-space, start {speed:g}")
axes[0].plot(tau_t, y_t, color=VIOLET, lw=1.8, ls="-.", label="extra time, start 0.2")
axes[0].axhline(0.0, color="grey", lw=1.0, label="patch end $y = 0$")
axes[0].axhline(-L_tip, color="grey", lw=1.0, ls=":", label="tip cut-off $y = -3$")
axes[0].set_xlabel("proper time $\\tau$ (unit $1/H$)")
axes[0].set_ylabel("height $y$ in the hidden direction (unit $1/H$)")
axes[0].set_title("Rising and sinking along $y$")
axes[0].legend(fontsize=7, loc="lower left")
largest_gap = 0.0
for (t_values, frame_rows), colour, label in (
        ((tau, space_paths[1.0][1]), RED, "3-space, start 1"),
        ((tau_t, frame_t), VIOLET, "extra time, start 0.2")):
    law = H_value * (np.sum(frame_rows[:, 1:4] ** 2, axis=1)
                     - np.sum(frame_rows[:, 5:8] ** 2, axis=1))
    numeric = np.gradient(frame_rows[:, 0], t_values)  # d(u0 hat)/dtau
    largest_gap = max(largest_gap, float(np.max(np.abs(numeric - law)[1:-1])))
    axes[1].plot(t_values, law, color=colour, lw=1.8, label=f"law, {label}")
    axes[1].plot(t_values[::100], numeric[::100], "o", color=colour, ms=5,
                 label=f"from RK4, {label}")
axes[1].axhline(0.0, color="grey", lw=0.8)
axes[1].set_xlabel("proper time $\\tau$ (unit $1/H$)")
axes[1].set_ylabel("$d^2y/d\\tau^2$ (unit $H$)")
axes[1].set_title("The push: $H$ times the difference of squares")
axes[1].legend(fontsize=7, loc="lower left")
report("largest difference between the numerical acceleration and the law",
       f"{largest_gap:.1e}")
save_figure(fig, "hidden_push",
            "The push along the hidden direction on freely falling test particles in "
            "the author’s metric along the deflating history $a_4 = x_4$, $b_4 = 1$, "
            "computed "
```

```
            "with RK4; horizontal axes the proper time $\\tau$ in units of $1/H$. "
            "Left: the height $y$ (unit $1/H$) of the particle at rest (dashed, it "
            "stays at $y = -1$), of three particles moving in 3-space (solid), which "
            "rise towards the patch end $y = 0$, and of a particle moving along an "
            "extra time (dash-dotted), which sinks towards the tip; the dotted line "
            "is the tip cut-off $y = -3$ of the Revision Kohn-Sham solver. Right: "
            "the acceleration $d^2y/d\\tau^2$ (unit $H$) obtained from the computed "
            "states by a numerical derivative (dots) agrees with the exact law "
            "$H$ times the squared 3-space frame velocity minus the squared "
            "extra-time frame velocity (lines): positive for motion in 3-space, "
            "negative for motion along an extra time.")
check(largest_gap < 1e-4,
      "the computed acceleration of y agrees with the exact push law")
```

Out [14]:

```
RESULT largest difference between the numerical acceleration and the law = 3.4e-06
```

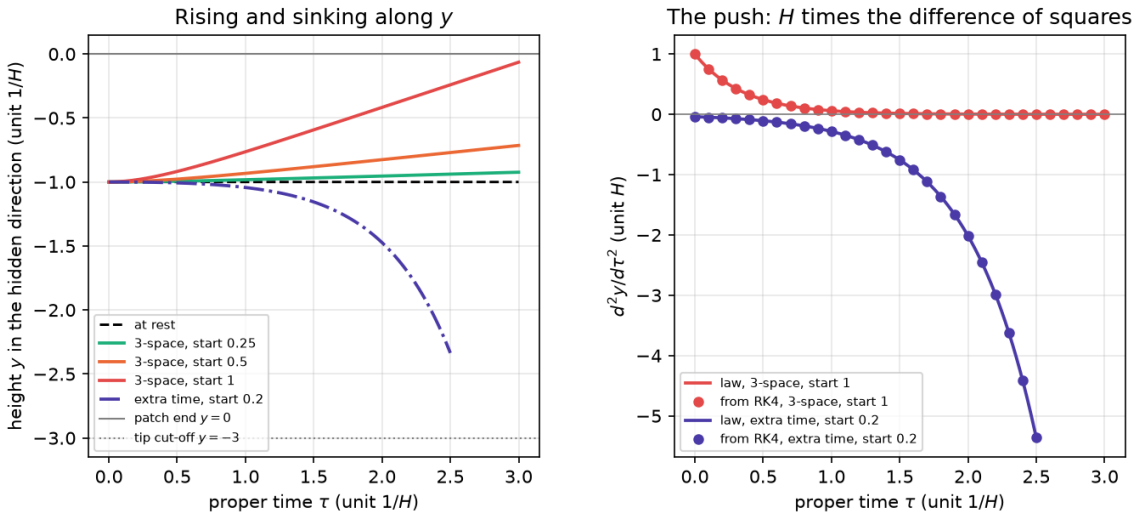

Figure 113: The push along the hidden direction on freely falling test particles in the author's metric along the deflating history $a_4 = x_4$, $b_4 = 1$, computed with RK4; horizontal axes the proper time τ in units of $1/H$. Left: the height y (unit $1/H$) of the particle at rest (dashed, it stays at $y = -1$), of three particles moving in 3-space (solid), which rise towards the patch end $y = 0$, and of a particle moving along an extra time (dash-dotted), which sinks towards the tip; the dotted line is the tip cut-off $y = -3$ of the Revision Kohn-Sham solver. Right: the acceleration $d^2y/d\tau^2$ (unit H) obtained from the computed states by a numerical derivative (dots) agrees with the exact law H times the squared 3-space frame velocity minus the squared extra-time frame velocity (lines): positive for motion in 3-space, negative for motion along an extra time. (Notebook 03d, figure 3.)

Out [14]:

```
Figure 03d.3 saved as Revision/textbook/figures/03d_3_hidden_push.png
PASS the computed acceleration of y agrees with the exact push law
```

Text cell:

```
The next cell draws the quadratic form of section 9 as a map. For a particle that
moves only along $x_5$ and $y$, $(u^4)^2 = 1 + (\hat u^0)^2 - (\hat u^5)^2$. The map
shows this number over the plane of $\hat u^5$ (horizontal) and $\hat u^0$
(vertical): white where it is positive, grey where it is negative. On the grey side a
plane wave with these frame momenta (times $m$) would grow instead of oscillating,
according to the record's check 'extra_time_growth_rates_unbounded'; a particle can
never be there, because $(u^4)^2 \ge 0$. The curve $(\hat u^5)^2 = 1 + (\hat u^0)^2$
separates the two regions. The path of the extra-time particle (violet) starts at
$(0.2, 0)$, moves right (blueshift) and down (it sinks), touches the curve at its
turning point and returns into the white region.
```

In [15]:

```
grid5, grid0 = np.meshgrid(np.linspace(0.0, 3.0, 301), np.linspace(-3.0, 0.5, 351))
squared_u4 = 1 + grid0 ** 2 - grid5 ** 2  # (u^4)^2 over the plane
fig, ax = plt.subplots(figsize=(6.4, 5.2))
ax.contourf(grid5, grid0, squared_u4, levels=[-100.0, 0.0], colors=["#d9d9d9"])
ax.contour(grid5, grid0, squared_u4, levels=[0.0], colors="black", linewidths=1.2)
ax.plot(frame_t[:, 5], frame_t[:, 0], color=VIOLET, lw=2.0,
        label="path of the extra-time particle")
ax.plot([frame_t[0, 5]], [frame_t[0, 0]], "s", color=VIOLET, ms=7, label="start")
ax.plot([frame_t[n_turn, 5]], [frame_t[n_turn, 0]], "o", color=VIOLET, ms=8,
        label="turning point, $u^4 = 0$")
ax.text(2.05, 0.15, "$(u^4)^2 < 0$:\nno particle;\nplane waves grow", fontsize=8)
ax.text(0.3, -1.2, "$(u^4)^2 > 0$", fontsize=9)
ax.set_xlabel("frame velocity $\\hat u^5$ along the extra time $x_5$")
ax.set_ylabel("frame velocity $\\hat u^0 = dy/d\\tau$")
ax.set_title("The quadratic form $(u^4)^2 = 1 + (\\hat u^0)^2 - (\\hat u^5)^2$")
ax.grid(False)
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "quadratic_form",
            "The squared $x_4$ velocity $(u^4)^2 = 1 + (\\hat u^0)^2 - (\\hat u^5)^2$ "
            "of a free particle that moves along the extra time $x_5$ and the "
            "hidden coordinate $y$, as a map over its frame velocities $\\hat u^5$ "
            "(horizontal) and $\\hat u^0$ (vertical), both pure numbers: white where "
            "it is positive, grey where it is negative, the black curve "
            "$(\\hat u^5)^2 = 1 + (\\hat u^0)^2$ between them. Multiplied by $m^2$ it "
            "is the squared frequency of a plane wave in the Revision record "
            "(check extra_time_growth_rates_unbounded), which grows instead of "
            "oscillating in the grey region. The violet path of the extra-time "
            "particle of this notebook (RK4, history $a_4 = x_4$, $b_4 = 1$) starts at "
            "$(0.2, 0)$ (square), touches the curve at its turning point (dot) and "
            "returns into the white region.")
check(np.all(1 + frame_t[:, 0] ** 2 - frame_t[:, 5] ** 2 > -1e-6),
      "the path never enters the region where (u4)^2 < 0")
```

Out [15]:

```
Figure 03d.4 saved as Revision/textbook/figures/03d_4_quadratic_form.png
PASS the path never enters the region where (u4)^2 < 0
```

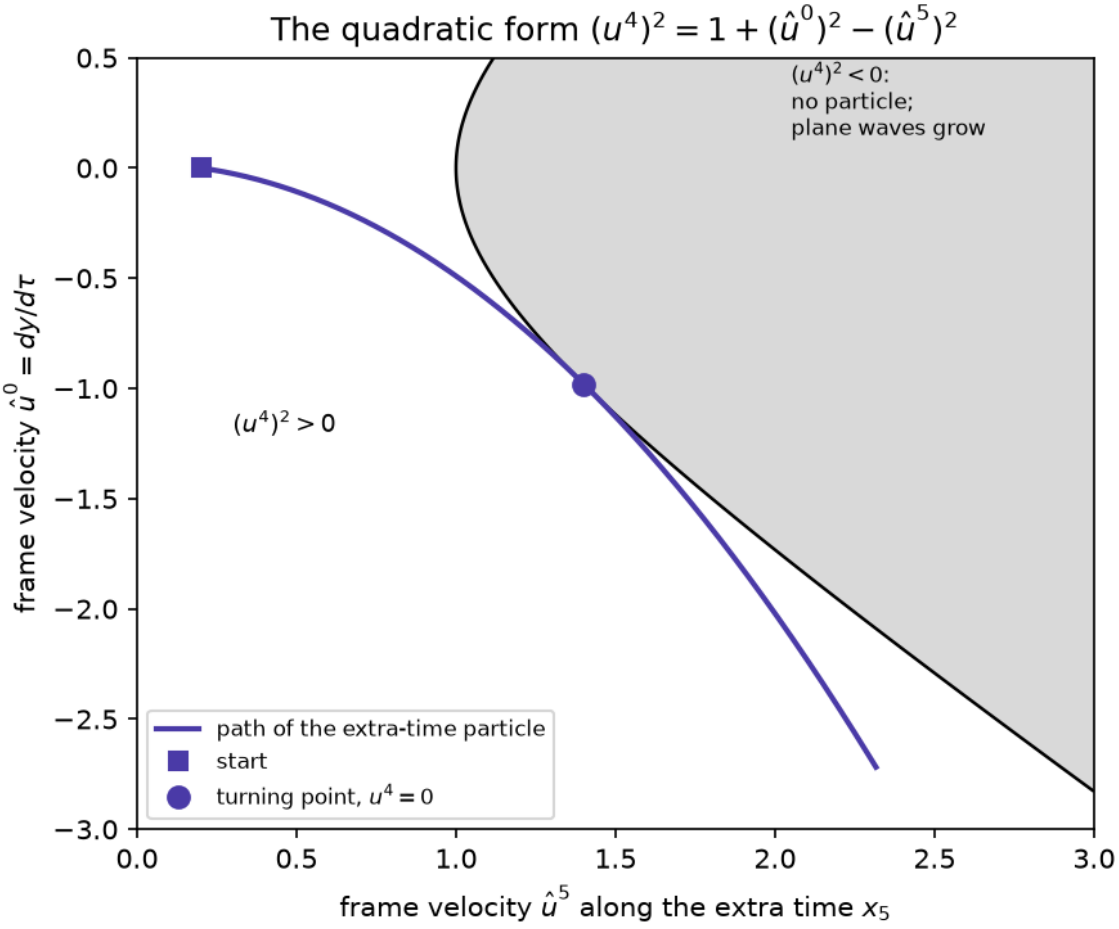

Figure 114: The squared x_4 velocity $(u^4)^2 = 1 + (\hat{u}^0)^2 - (\hat{u}^5)^2$ of a free particle that moves along the extra time x_5 and the hidden coordinate y, as a map over its frame velocities $\hat{u}^5$ (horizontal) and $\hat{u}^0$ (vertical), both pure numbers: white where it is positive, grey where it is negative, the black curve $(\hat{u}^5)^2 = 1 + (\hat{u}^0)^2$ between them. Multiplied by m^2 it is the squared frequency of a plane wave in the Revision record (check extra_time_growth_rates_unbounded), which grows instead of oscillating in the grey region. The violet path of the extra-time particle of this notebook (RK4, history $a_4 = x_4$, $b_4 = 1$) starts at $(0.2, 0)$ (square), touches the curve at its turning point (dot) and returns into the white region. (Notebook 03d, figure 4.)

Text cell:

```
## 13. How accurate is the computation?

Two tests. First, the conserved quantities: the next cell draws, against $\tau$, how
far the computed $p_1$ (fastest 3-space particle), $p_5$ (extra-time particle) and
$g(u, u)$ (both) drift from their starting values, on a logarithmic axis. Second,
the order of RK4: it integrates both particles from $\tau = 0$ to $2$ with
$50, 100, 200, 400, 800, 1600$ steps and measures the largest difference of the end
states of two successive step sizes; for a method of order 4 each halving of the
step divides this difference by about $2^4 = 16$, so on logarithmic axes the points
lie on a line of slope 4.
```

In [16]:

```
from matplotlib.ticker import NullFormatter  # an empty label for minor ticks

p1_fast = space_paths[1.0][4]  # p_1 along the fastest 3-space path
drift = {"$p_1$, 3-space particle": (tau, np.abs(p1_fast / p1_fast[0] - 1)),
         "$g(u,u) + 1$, 3-space particle": (tau, np.abs(space_paths[1.0][3] + 1)),
         "$p_5$, extra-time particle": (tau_t, np.abs(p5 / p5[0] - 1)),
         "$g(u,u) + 1$, extra-time particle": (tau_t, np.abs(lengths_t + 1))}
STEPS = (50, 100, 200, 400, 800, 1600)
differences = {}
for label, state0 in (("3-space, start 1", start(Y0, [0, 1, 0, 0, 0, 0, 0, 0])),
                      ("extra time, start 0.2", start(Y0, [0, 0, 0, 0, 0, 0.2, 0, 0]))):
    ends = [rk4_path(state0, 2.0, n)[1][-1] for n in STEPS]
    differences[label] = np.array([np.max(np.abs(ends[i] - ends[i + 1]))
                                   for i in range(len(STEPS) - 1)])
    ratios = differences[label][:-1] / differences[label][1:]
    say(f"  {label}: ratios of successive differences "
        + ", ".join(f"{r:.1f}" for r in ratios))
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)
for (label, (t_values, values)), colour, style in zip(
        drift.items(), (RED, ORANGE, VIOLET, AQUA), ("-", "--", "-.", ":")):
    axes[0].semilogy(t_values[1:], np.maximum(values[1:], 1e-17), color=colour,
                     ls=style, lw=1.6, label=label)
axes[0].set_xlabel("proper time $\\tau$ (unit $1/H$)")
axes[0].set_ylabel("relative drift from the start")
axes[0].set_title("Conserved quantities, RK4 with step $0.001$")
axes[0].legend(fontsize=7, loc="lower right")
dts = 2.0 / np.array(STEPS[:-1])  # the larger step of each pair
for (label, values), colour in zip(differences.items(), (RED, VIOLET)):
    axes[1].loglog(dts, values, "o-", color=colour, lw=1.6, label=label)
axes[1].loglog(dts, differences["extra time, start 0.2"][0] * (dts / dts[0]) ** 4,
               "--", color="black", lw=1.0, label="slope 4")
axes[1].set_xticks(dts, [f"{dt:g}" for dt in dts])  # the five steps as labels
axes[1].xaxis.set_minor_formatter(NullFormatter())  # no labels between them
axes[1].set_xlabel("step $\\Delta\\tau$ of RK4")
axes[1].set_ylabel("difference of end states (step and half step)")
axes[1].set_title("RK4 is of order 4")
```

```
axes[1].legend(fontsize=8)
save_figure(fig, "accuracy",
            "The accuracy of the RK4 computation of free fall in the author's metric "
            "along the deflating history $a_4 = x_4$, $b_4 = 1$. Left: the relative "
            "drift of "
            "the conserved momentum $p_1$ and of $g(u, u) + 1$ for the 3-space "
            "particle started with frame velocity $1$, and of $p_5$ and "
            "$g(u, u) + 1$ for the extra-time particle started with $0.2$, against "
            "the proper time $\\tau$ in units of $1/H$, logarithmic vertical axis; "
            "all stay far below $10^{-6}$ (the extra-time particle drifts more "
            "because its velocities grow). Right: the largest difference between "
            "the end states at $\\tau = 2$ computed with the step $\\Delta\\tau$ and "
            "with half of it, against $\\Delta\\tau$ on logarithmic axes, for the "
            "same two particles; the points follow the dashed line of slope 4: "
            "halving the step makes the error 16 times smaller.")
all_ratios = np.concatenate([d[1:-1] / d[2:] for d in differences.values()])
check(np.all((all_ratios > 14) & (all_ratios < 18)),
      "halving the RK4 step divides the error by 14 to 18: order 4")
```

Out [16]:

```
3-space, start 1: ratios of successive differences 17.1, 16.6, 16.3, 16.2
extra time, start 0.2: ratios of successive differences 17.2, 16.7, 16.3, 16.2
```

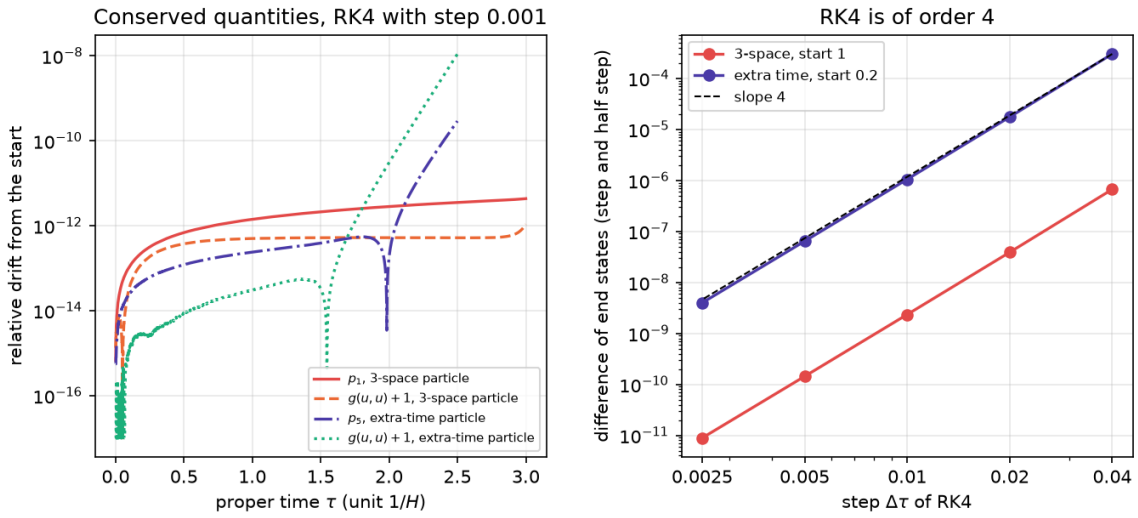

Figure 115: The accuracy of the RK4 computation of free fall in the author's metric along the deflating history $a_4 = x_4$, $b_4 = 1$. Left: the relative drift of the conserved momentum p_1 and of $g(u, u) + 1$ for the 3-space particle started with frame velocity 1, and of p_5 and $g(u, u) + 1$ for the extra-time particle started with 0.2, against the proper time τ in units of $1/H$, logarithmic vertical axis; all stay far below 10^{-6} (the extra-time particle drifts more because its velocities grow). Right: the largest difference between the end states at $\tau = 2$ computed with the step $\Delta\tau$ and with half of it, against $\Delta\tau$ on logarithmic axes, for the same two particles; the points follow the dashed line of slope 4: halving the step makes the error 16 times smaller. (Notebook 03d, figure 5.)

Out [16]:

```
Figure 03d.5 saved as Revision/textbook/figures/03d_5_accuracy.png
PASS halving the RK4 step divides the error by 14 to 18: order 4
```

Text cell:

```
## 14. The last check

The last cell checks that the five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [17]:

```
figure_names = ["frame_velocities", "x4_velocity", "hidden_push", "quadratic_form",
                "accuracy"]
files = [output_file(f"{FIGURE_FOLDER}/03d_{n}_{name}.png")
         for n, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all five figure files exist")
all_checks_passed()
```

Out [17]:

```
PASS all five figure files exist
ALL 23 CHECKS PASSED (notebook 03d)
```

Text cell:

```
## 15. What this notebook showed

- PROVED (exact, sympy, for all functions $a_4(x_4)$ and $b_4(x_4)$, with the
  Christoffel symbols of the Revision record): in free fall the six momenta
  $p_a = g_{aa} u^a$ of $x_1, x_2, x_3, x_5, x_6, x_7$ and the squared length
  $g(u, u)$ are conserved; the frame velocity in 3-space is $p_1 e^{-Hy - a_4}$ (the
  momentum weight $\kappa$ of the Revision Kohn-Sham record) and along an extra time
  $-p_5 b_4^{-1/2} e^{a_4 - Hy}$;
  $d^2y/d\tau^2 = H(\sum_i (\hat u^i)^2 - \sum_j (\hat u^j)^2)$;
  $du^4/d\tau = -a_4'\sum_i (\hat u^i)^2 + \alpha_t \sum_j (\hat u^j)^2$ with
  $\alpha_t = -a_4' + b_4'/(2b_4)$; and
  $(u^4)^2 = 1 + (\hat u^0)^2 + \sum_i (\hat u^i)^2 - \sum_j (\hat u^j)^2$, which,
  times $m^2$, is the quadratic form of the plane-wave check of the Revision scope
  report.
- So along a deflating history ($a_4$ increasing, $b_4$ constant) motion in 3-space
  is redshifted like $e^{-a_4}$ and motion along an extra time is blueshifted like
  $e^{a_4}$;
  3-space motion pushes a particle towards the patch end, extra-time motion towards
  the tip; the $x_4$ velocity can only decrease.
- COMPUTED (RK4 with the step $0.001$, along the prescribed history $a_4 = x_4$,
  $b_4 = 1$, $H = 1$, from $y = -1$): a particle at rest stays at rest with
  $x_4 = \tau$; three particles moving in 3-space slow down and rise towards the
  patch end; a particle
```

```
  started along the extra time $x_5$ with frame velocity $0.2$ reaches a largest
  $x_4$ (the RESULT lines above) and then moves back in $x_4$; every exact law holds
  on the computed paths, and RK4 shows its order 4.
- A particle without extra-time momentum has $(u^4)^2 \ge 1$ and never turns
  (PROVED); the Revision Kohn-Sham work keeps only states without extra-time
  momenta (its good sector).
- ASSUMED: the history $a_4 = AHx_4$, $b_4 = 1$ (a prescribed background); test
  particles
  that do not change the metric. NOT computed here: whether the waves of the fields
  follow these paths in the curved metric.
```

3.35 Line-by-line walk-through of Notebook 03d

The notebook has 17 code cells, In [1] to In [17]. The conventions of the earlier walk-throughs apply (a line `...` in a quoted figure command stands for the rest of the caption, printed in full in Section 3.34).

In [1], the set-up cell. Its comment lines are the run instructions of Section 3.33. Its code is, line for line, the set-up code of Notebook 03a explained in Section 3.13 under In [1], with the single difference `NOTEBOOK_ID = "03d" # this notebook:  chapter 03, example d`. It prints Set-up of notebook 03d complete: repository folder found, helpers defined.

In [2], every result in one piece. Word for word In [2] of Notebook 03a (Section 3.13). It prints From now on check and report print each of their results in one piece.

In [3], the Christoffel symbols of the record.

```
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
from sympy.parsing.sympy_parser import parse_expr  # text -> sympy expression

CURVATURE_RECORD = "Revision/gkd_lovelock/results/curvature.json"
record = json.loads(repository_file(CURVATURE_RECORD).read_text(encoding="utf-8"))
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
X = sp.symbols("x0:8", real=True)  # the coordinates x0 ... x7 (X[0] ... X[7])
x4, x0 = X[4], X[0]
```

The packages, the record of the curvature computation, the names of the coordinates in the author's order, and the eight coordinates as the tuple `X` (a tuple is a list that cannot be changed); `x4` and `x0` name its positions 4 and 0.

```
H = sp.symbols("H", positive=True)  # the constant of the author
a4 = sp.Function("a4")(x4)  # the metric function a4, any function of x4
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4 > 0
a4p, a4v, b4p = sp.symbols("a4p a4v b4p", real=True)  # printing names
b4v = sp.symbols("b4v", positive=True)  # printing name of the value of b4
record_names = {"a4p": a4p, "a4v": a4v, "b4p": b4p, "b4v": b4v, "H": H, "x0": x0,
                "E": sp.E, "sin": sp.sin, "cot": sp.cot}
```

$H > 0$, the functions $a_4(x_4)$ and $b_4(x_4)$ (the second positive), the plain symbols `a4p`, `a4v`, `b4p` and

b4v (the last positive), and the dictionary of names for reading the record (as in Notebook 03b, Section 3.30, In [5]).

```
def from_mathematica(entry_text):
    """A value of the record (Mathematica notation) as a sympy expression."""
    t = entry_text.replace("Derivative[1][a4][x4]", "a4p").replace("a4[x4]", "a4v")
    t = t.replace("Derivative[1][b4][x4]", "b4p").replace("b4[x4]", "b4v")
    t = t.replace("Sin[6*H*x0]", "sin(6*H*x0)").replace("Cot[6*H*x0]", "cot(6*H*x0)")
    t = t.replace("^", "**")
    if "[" in t or "]" in t:  # a piece of notation that was not translated
        raise ValueError(f"cannot translate {entry_text}")
    return parse_expr(t, local_dict=record_names)
```

The translation of a record entry into sympy, the same as in Notebook 03b but without a_4'' and b_4'', which no Christoffel symbol contains.

```
Gamma = {}  # (a, b, c) -> the value of Gamma^a_bc, with a4p, a4v, b4p, b4v
for entry in record["christoffelNonzero_b_le_c"]:
    a, b, c = (NAMES.index(entry[key]) for key in ("a", "b", "c"))
    Gamma[(a, b, c)] = from_mathematica(entry["value"])
    Gamma[(a, c, b)] = Gamma[(a, b, c)]  # the same symbol with b and c exchanged
```

The dictionary `Gamma` holds the symbols. For each of the 25 entries of the record, the three names of `a`, `b`, `c` become positions (the expression in round brackets produces three numbers, which are unpacked into `a, b, c`), the value is translated, and it is stored under both orders of the lower indices. For the 13 entries with $b = c$ both keys are the same, so the 25 entries give $13 + 2 \times 12 = 37$ keys.

```
for key in [(1, 1, 4), (4, 5, 5), (0, 1, 1), (0, 0, 0)]:
    a, b, c = key
    say(f"  Gamma^{NAMES[a]}_({NAMES[b]} {NAMES[c]}) = {Gamma[key]}")
report("entries of the record (b <= c)", len(record["christoffelNonzero_b_le_c"]))
check(len(record["christoffelNonzero_b_le_c"]) == 25 and len(Gamma) == 37,
      "the 25 entries of the record give 37 non-zero Christoffel symbols",
      record=f"{CURVATURE_RECORD}, christoffelNonzero_b_le_c")
```

Four symbols are printed in the record's own form, because the translation does not simplify:

```
  Gamma^x1_(x1 x4) = a4p
  Gamma^x4_(x5 x5) = a4p*b4v*exp(-2*a4v)*sin(6*H*x0)**(1/3) -
    b4p*exp(-2*a4v)*sin(6*H*x0)**(1/3)/2
  Gamma^x0_(x1 x1) = -H*exp(2*a4v)*sin(6*H*x0)**(1/3)/cot(6*H*x0)
  Gamma^x0_(x0 x0) = -6*H*cot(6*H*x0) - 6*H/cot(6*H*x0)
```

They are $\Gamma^{x_1}{}_{x_1x_4} = a_4'$, $\Gamma^{x_4}{}_{x_5x_5} = (a_4'b_4 - \frac{1}{2}b_4')e^{-2a_4}\sin^{1/3} z$, $\Gamma^{x_0}{}_{x_1x_1} = -H\tan z\, e^{2a_4}\sin^{1/3} z$ and $\Gamma^{x_0}{}_{x_0x_0} = -6H(\cot z + \tan z)$ (Section 3.23); the keys are positions in the author's order, `(1, 1, 4)` for $\Gamma^{x_1}{}_{x_1x_4}$ and `(0, 0, 0)` for $\Gamma^{x_0}{}_{x_0x_0}$. The RESULT line prints 25, and the check confirms the counts 25 and 37.

In [4], what stays constant (exact).

```
u = sp.symbols("u0:8", real=True)  # the velocity components u^0 ... u^7
# back to the functions a4(x4) and b4(x4)
to_function = {a4p: sp.Derivative(a4, x4), a4v: a4, b4p: sp.Derivative(b4, x4),
               b4v: b4}
acceleration = [sp.S(0)] * 8  # du^a/dtau from the geodesic equation
for (a, b, c), value in Gamma.items():
    acceleration[a] -= value.subs(to_function) * u[b] * u[c]
```

The eight velocity components $u^0, \ldots, u^7$ as symbols (`u[k]` is u^k). `to_function` turns the plain symbols back into the derivatives and the values of the functions $a_4(x_4)$ and $b_4(x_4)$, so that sympy can differentiate along the path. `acceleration` starts as eight zeros (`sp.S(0)` is sympy's zero); the loop subtracts $\Gamma^a{}_{bc} u^b u^c$ for every one of the 37 keys, which is the geodesic equation $du^a/d\tau = -\sum_{b,c} \Gamma^a{}_{bc} u^b u^c$ (the two orders of the lower indices are both in the dictionary, so the factors 2 of Section 3.31 appear by themselves).

```
def along_path(f):
    """The rate of change df/dtau of f(x, u) along a path of free fall."""
    return sum(sp.diff(f, X[k]) * u[k] + sp.diff(f, u[k]) * acceleration[k]
               for k in range(8))
```

The rate of change of a quantity $f(x, u)$ along a path of free fall, by the chain rule: $df/d\tau = \sum_a (\partial f/\partial x^a)\, u^a + \sum_a (\partial f/\partial u^a)\, du^a/d\tau$, with $du^a/d\tau$ from the geodesic equation.

```
warp = sp.sin(6 * H * x0) ** sp.Rational(1, 6)  # sin(z)^(1/6), z = 6 H x0
# the eight scale factors h_a in the author's order x0, x1, ..., x7
h = [sp.cot(6 * H * x0)] + [sp.exp(a4) * warp] * 3 + [sp.Integer(1)] \
    + [sp.sqrt(b4) * sp.exp(-a4) * warp] * 3
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the signs of the metric entries
g = [ETA[k] * h[k] ** 2 for k in range(8)]  # the diagonal of the metric
say(f"du1/dtau = {sp.factor(acceleration[1]).subs(sp.Derivative(a4, x4), a4p)}")
```

The warp factor, the eight scale factors in the author's order (Section 3.7: $h_0 = \cot z$ first, then three times $e^{a_4} W$, 1 and three times $b_4^{1/2} e^{-a_4} W$), the signs, and the diagonal $g_{aa} = \eta_{aa} h_a^2$. The printed line is the geodesic equation for u^1 (`acceleration[1]`), factored: `-2*u1*(H*u0*cot(6*H*x0) + a4p*u4)`, the equation for u^i of Section 3.31.

```
momenta = {k: g[k] * u[k] for k in (1, 2, 3, 5, 6, 7)}  # p_a = g_aa u^a
rates = [sp.simplify(along_path(p)) for p in momenta.values()]
check(rates == [0] * 6,
      "the six momenta of x1, x2, x3, x5, x6, x7 are conserved in free fall")
squared_length = sum(g[k] * u[k] ** 2 for k in range(8))  # g(u, u)
check(sp.simplify(along_path(squared_length)) == 0,
      "the squared length g(u, u) of the velocity is conserved in free fall")
```

The six momenta $p_k = g_{kk} u^k$, at the positions 1, 2, 3, 5, 6, 7; their rates along the path simplify to zero (first check). The squared speed $g(u, u) = \sum_a g_{aa} (u^a)^2$; its rate simplifies to zero (second check). Both hold for all functions $a_4(x_4)$ and $b_4(x_4)$.

In [5], the momentum weight of the Kohn-Sham record.

```
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks_theory = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
say("record, geometry.kappa: " + ks_theory["geometry"]["kappa"])
y_of_x0 = sp.log(sp.sin(6 * H * x0)) / (6 * H)  # the hidden coordinate y
kappa = sp.exp(-H * y_of_x0 - a4)  # the record's e^{-Hy - a4(x4)}
```

The Kohn-Sham theory record is read and its statement about the momentum weight printed: kappa(y, x4) = e^{-Hy - a4(x4)} (the momentum weight). Then y as a function of x_0 (Section 3.9) and the weight $\kappa = e^{-Hy-a_4}$.

```
check(sp.simplify(1 / h[1] - kappa) == 0,
      "1/h1 = e^(-H y - a4): the frame velocity in 3-space is p1 times kappa",
      record=f"{KS_THEORY}, geometry.kappa")
check(sp.simplify(1 / h[5] - sp.exp(a4 - H * y_of_x0) / sp.sqrt(b4)) == 0,
      "1/h5 = b4^(-1/2) e^(a4 - H y): the frame velocity along an extra time grows "
      "with a4")
```

Two checks of Section 3.31: $1/h_1 = \kappa(y, x_4)$, so $\hat{u}^1 = p_1/h_1 = p_1\kappa(y, x_4)$; and $1/h_5 = b_4^{-1/2}e^{a_4-Hy}$, so $\hat{u}^5 = -p_5 b_4^{-1/2}e^{a_4-Hy}$.

In [6], the push, the slowing, and $(u^4)^2$ (exact).

```
frame = [h[k] * u[k] for k in range(8)]  # the frame velocities u-hat^a = h_a u^a
space_squared = sum(frame[k] ** 2 for k in (1, 2, 3))  # 3-space part
extra_squared = sum(frame[k] ** 2 for k in (5, 6, 7))  # extra-time part
push = along_path(frame[0]) - H * (space_squared - extra_squared)
check(sp.simplify(push) == 0,
      "d^2y/dtau^2 = H (3-space frame velocity squared - extra-time one squared)")
```

The frame velocities $\hat{u}^a = h_a u^a$, the sums S (positions 1, 2, 3) and E (positions 5, 6, 7) of Section 3.31, and the difference between the rate of $\hat{u}^0 = dy/d\tau$ (`frame[0]`) and the push law $H(S - E)$, which simplifies to zero.

```
alpha_t = -sp.diff(a4, x4) + sp.diff(b4, x4) / (2 * b4)  # the extra-time rate
slowing = (along_path(u[4]) + sp.diff(a4, x4) * space_squared
           - alpha_t * extra_squared)
check(sp.simplify(slowing) == 0,
      "du4/dtau = -a4' (3-space frame velocities squared) + alpha_t (extra-time ones "
      "squared)")
in_frame = frame[0] ** 2 + space_squared - u[4] ** 2 - extra_squared
check(sp.simplify(squared_length - in_frame) == 0,
      "g(u, u) written with the frame velocities")
```

`alpha_t` is the extra-time rate $\alpha_t = -a_4' + b_4'/(2b_4)$, written with the functions. The law $du^4/d\tau = -a_4'S + \alpha_t E$ for the velocity `u[4]`, and $g(u, u) = (\hat{u}^0)^2 + S - (u^4)^2 - E$, both checked exactly, for all functions a_4 and b_4.

In [7], the quadratic form of the plane-wave check.

```
SCOPE = "Revision/theory/reports/python-scope.json"
scope = json.loads(repository_file(SCOPE).read_text(encoding="utf-8"))
growth = next(entry for entry in scope["checks"]
              if entry["name"] == "extra_time_growth_rates_unbounded")
form_text = growth["detail"].split("h_k^2 = (")[1].split(") I16")[0]  # the formula
say(f"record ({growth['verdict']}): E^2 = {form_text}")
```

The scope report is read and its check `extra_time_growth_rates_unbounded` taken from its list of checks. Its text contains the piece `h_k^2 = (` followed by the quadratic form and `) I16` (times the 16×16 unit matrix); the two `split` calls cut out the form between them. The output prints `PASS` and the form $k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2 + m^2$.

```
k = sp.symbols("k0:8", real=True)  # frame momenta k0 ... k7 (k4 is not used)
m = sp.symbols("m", positive=True)  # the mass
form = parse_expr(form_text, local_dict={**{f"k{n}": k[n] for n in range(8)},
                                         "m": m})
U = sp.symbols("U0:8", real=True)  # the frame velocities as plain symbols
particle = m ** 2 * (1 + U[0] ** 2 + U[1] ** 2 + U[2] ** 2 + U[3] ** 2 - U[5] ** 2
                     - U[6] ** 2 - U[7] ** 2)  # m^2 (u^4)^2 from section 8
```

The form is read with the momenta $k_0, \ldots, k_7$ and the mass m as symbols (the name `k{n}` of the text means `k[n]`); `{**{...}, "m": m}` builds one dictionary from a dictionary comprehension (unpacked by `**`) and one more entry. `particle` is $m^2(u^4)^2$ from the formula of Section 3.31, with the frame velocities as plain symbols $U_0, \ldots, U_7$.

```
same_form = sp.expand(form.subs({k[n]: m * U[n] for n in range(8)}) - particle)
check(growth["verdict"] == "PASS" and same_form == 0,
      "m^2 (u4)^2 of the particle is the plane-wave E^2 of the record",
      record=f"{SCOPE}, check extra_time_growth_rates_unbounded")
```

With $k_a = m\hat{u}^a$ put into the record's form, the difference from $m^2(u^4)^2$ multiplies out to zero; the check also requires the record's verdict PASS.

In [8], the numerical tools.

```
import math  # sin, tan, exp, ... for single numbers (faster than numpy here)

PARAMETERS = "Revision/kohn_sham/results/parameters.json"
physics = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
H_value, A_value = physics["physics"]["H"], physics["physics"]["historyA"]
status = ks_theory["adiabaticity"]["historyStatus"]
say(f"H = {H_value}, A = {A_value}; record: " + status.split(". ")[0] + ".")
```

The module `math` computes functions of single numbers, faster than numpy for one number at a time. The parameters of the Revision Kohn-Sham solver give $H = 1$ and the slope $A = 1$; the first sentence of the record's statement on the status of the history is printed: PRESCRIBED BACKGROUND: the history a4 = A H x4 is prescribed, not solved for.

```
check(H_value == 1.0 and A_value == 1.0 and status.startswith("PRESCRIBED BACKGROUND"),
      "the history of the record: a4 = A H x4 with A = 1, H = 1, prescribed",
      record=f"{PARAMETERS}, physics.H and physics.historyA; {KS_THEORY}, "
             "adiabaticity.historyStatus")
ARGUMENTS = (a4v, a4p, x0, H, b4v, b4p)
gamma_numbers = [(a, b, c, sp.lambdify(ARGUMENTS, value, "math"))
                 for (a, b, c), value in sorted(Gamma.items())]
ETA_N = np.array(ETA, dtype=float)  # the signs as numbers
```

The check confirms the history and its status. `gamma_numbers` is a list of the 37 symbols as fast numerical functions of $(a_4, a_4', x_0, H, b_4, b_4')$, each with its three indices (`"math"` makes functions of single numbers). `ETA_N` holds the signs as floating-point numbers.

```
def free_fall(state):
    """d state/d tau for state = (x0..x7, u0..u7) on the history a4 = A H x4,
    b4 = 1."""
    x, v = state[:8], state[8:]
    a4_now, rate = A_value * H_value * x[4], A_value * H_value  # a4 and a4'
    change = np.zeros(8)
    for a, b, c, f in gamma_numbers:  # du^a/dtau = - sum Gamma^a_bc u^b u^c
        change[a] -= f(a4_now, rate, x[0], H_value, 1.0, 0.0) * v[b] * v[c]
    return np.concatenate([v, change])
```

The right side of the 16 equations: the state holds the 8 coordinates (`state[:8]`, entry k is x^k) and the 8 velocities (`state[8:]`, entry $8+k$ of the state is u^k). On the history, $a_4 = AHx_4$ with x_4 = `x[4]`, $a_4' = AH$, $b_4 = 1$ and $b_4' = 0$ (the numbers `1.0, 0.0` in the call), and x_0 = `x[0]`. The accelerations are summed over the 37 symbols, and the function returns the derivatives of the state: the velocities, then the accelerations (`np.concatenate` joins two arrays).

```
def rk4_path(state0, tau_end, steps):
    """RK4 from tau = 0 to tau_end; returns the times and the states (rows)."""
    dt = tau_end / steps
    states = np.zeros((steps + 1, 16))
    states[0] = state0
    for n in range(steps):
        s = states[n]
        k1 = free_fall(s)
        k2 = free_fall(s + dt / 2 * k1)
        k3 = free_fall(s + dt / 2 * k2)
        k4 = free_fall(s + dt * k3)
        states[n + 1] = s + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return np.linspace(0.0, tau_end, steps + 1), states
```

The RK4 method of Chapter 2 (the same as in Notebook 03c, Section 3.22, In [8]) for the 16 unknowns, from $\tau = 0$ to `tau_end` in `steps` equal steps; it returns the times and the table of states.

```
def scale_numbers(x):
    """The eight scale factors h_a at the point x on the history (numbers)."""
    a4_now, z_now = A_value * H_value * x[4], 6 * H_value * x[0]
    w = math.sin(z_now) ** (1 / 6)  # the warp factor sin(z)^(1/6)
```

```
    return np.array([1 / math.tan(z_now)] + [math.exp(a4_now) * w] * 3 + [1.0]
                    + [math.exp(-a4_now) * w] * 3)  # with b4 = 1
```

The eight scale factors as numbers at a point on the history, in the author's order and with $b_4 = 1$.

```
def start(y0, frame_velocity):
    """The state at tau = 0: x4 = 0, y = y0, frame velocities as given (the entry
    for x4 is replaced by the value that makes g(u, u) = -1)."""
    x = np.zeros(8)
    x[0] = math.asin(math.exp(6 * H_value * y0)) / (6 * H_value)  # sin z = e^(6Hy)
    w = np.array(frame_velocity, dtype=float)
    w[4] = 0.0
    w[4] = math.sqrt(1.0 + np.sum(ETA_N * w ** 2))  # (u^4)^2 = 1 + sum eta w^2
    return np.concatenate([x, w / scale_numbers(x)])  # u^a = u-hat^a / h_a
```

The starting state: all coordinates zero except x_0, which is set from the height y_0 by $\sin z = e^{6Hy_0}$. The frame velocities are given in the author's order; the entry for x_4 (`w[4]`) is first set to zero, and then to $\sqrt{1 + \sum_a \eta_{aa} (\hat{u}^a)^2}$, which is the formula $(u^4)^2 = 1 + (\hat{u}^0)^2 + S - E$ of Section 3.31, so that $g(u, u) = -1$. The velocities are $u^a = \hat{u}^a / h_a$.

```
def describe(states):
    """Frame velocities (rows), the coordinate y and g(u, u) along a path."""
    h_rows = np.array([scale_numbers(s[:8]) for s in states])
    frame_rows = h_rows * states[:, 8:]
    y_values = np.log(np.sin(6 * H_value * states[:, 0])) / (6 * H_value)
    lengths = np.sum(ETA_N * frame_rows ** 2, axis=1)  # g(u, u)
    return frame_rows, y_values, lengths, h_rows
```

For every computed state: the scale factors, the frame velocities $\hat{u}^a = h_a u^a$ (a product of two tables entry by entry), the height y from x_0 (column 0), and $g(u, u) = \sum_a \eta_{aa} (\hat{u}^a)^2$ (`axis=1` sums along each row).

```
Y0 = -1.0  # the starting height of every particle
say(f"start: y0 = {Y0}, z0 = {math.asin(math.exp(6 * Y0)):.6f}")
```

Every particle starts at $y = -1$, that is $z = \arcsin e^{-6} = 0.002479$, as printed.

In [9], the particle at rest.

```
tau, rest = rk4_path(start(Y0, [0.0] * 8), 3.0, 3000)
others = [0, 1, 2, 3, 5, 6, 7]  # every coordinate except x4
moved = np.max(np.abs(rest[:, others] - rest[0, others]))
check(moved == 0.0 and np.max(np.abs(rest[:, 4] - tau)) < 1e-12
      and np.max(np.abs(rest[:, 12] - 1.0)) < 1e-12,
      "the particle at rest stays at rest and x4 equals its proper time")
```

A particle with all frame velocities zero (so $u^4 = 1$) is followed from $\tau = 0$ to 3 in 3000 steps. `rest[:, others]` is the table of the seven other coordinates; their largest change is exactly zero,

because every acceleration contains a factor $u^b u^c$ with a zero velocity (no symbol $\Gamma^a{}_{x_4 x_4}$ exists, Section 3.23). The check also requires $x_4 = \tau$ (column 4) and $u^4 = 1$ (column 12 of a state is u^4: 8 coordinates come first) within 10^{-12}.

In [10], three particles moving in 3-space.

```
SPEEDS = (0.25, 0.5, 1.0)  # starting frame velocities along x1
space_paths = {}
for speed in SPEEDS:
    tau, states = rk4_path(start(Y0, [0, speed, 0, 0, 0, 0, 0, 0]), 3.0, 3000)
    frame_rows, y_values, lengths, h_rows = describe(states)
    p1 = h_rows[:, 1] ** 2 * states[:, 9]  # p_1 = g_11 u^1 = h_1^2 u^1
    weight = np.exp(-H_value * y_values - A_value * H_value * states[:, 4])  # kappa
    space_paths[speed] = (states, frame_rows, y_values, lengths, p1)
```

For each starting frame velocity along x_1 (entry 1 of the list) the path from $\tau = 0$ to 3 in 3000 steps, its description, the momentum $p_1 = h_1^2 u^1$ along the path (column 9 is u^1), and the momentum weight $\kappa = e^{-Hy - a_4}$; everything is stored under the speed.

```
    check(np.max(np.abs(p1 / p1[0] - 1)) < 1e-9
          and np.max(np.abs(lengths + 1)) < 1e-9
          and np.max(np.abs(frame_rows[:, 1] - p1[0] * weight)) < 1e-9
          and np.all(np.diff(states[:, 12]) < 0) and np.all(y_values < 0),
          f"3-space speed {speed}: p1 and g(u, u) conserved, frame velocity "
          "p1 kappa, u4 falls")
```

One check per particle with five conditions: p_1 constant within a relative 10^{-9}; $g(u,u) = -1$ within 10^{-9}; the frame velocity equals $p_1 \kappa(y, x_4)$ within 10^{-9}; u^4 decreases at every step (`np.diff` gives the differences of successive values); the particle stays inside the patch ($y < 0$).

```
states, frame_rows, y_values, lengths, p1 = space_paths[1.0]
report("fastest particle at tau = 3: x4", f"{states[-1, 4]:.4f}")
report("fastest particle at tau = 3: frame velocity along x1",
       f"{frame_rows[-1, 1]:.6f}")
report("fastest particle at tau = 3: u4", f"{states[-1, 12]:.6f}")
report("fastest particle at tau = 3: height y", f"{y_values[-1]:.4f}")
```

Four RESULT lines for the fastest particle at $\tau = 3$: $x_4 = 3.3070$ (its clock has run 3 while x_4 advanced by more), $\hat{u}^1 = 0.014370$ (down from 1), $u^4 = 1.060733$ (down from $\sqrt{2} = 1.414$), $y = -0.0643$ (risen from -1 almost to the patch end).

In [11], the particle moving along an extra time.

```
tau_t, extra = rk4_path(start(Y0, [0, 0, 0, 0, 0, 0.2, 0, 0]), 2.5, 2500)
frame_t, y_t, lengths_t, h_t = describe(extra)
p5 = -h_t[:, 5] ** 2 * extra[:, 13]  # p_5 = g_55 u^5 = -h_5^2 u^5
check(np.max(np.abs(p5 / p5[0] - 1)) < 1e-8 and np.max(np.abs(lengths_t + 1)) < 1e-7
      and np.all(np.diff(extra[:, 12]) < 0),
      "extra-time particle: p5 and g(u, u) conserved, u4 decreases at every step")
```

A particle started with $\hat{u}^5 = 0.2$ (entry 5 of the list), so $u^4 = \sqrt{1 - 0.04} = 0.980$, followed from $\tau = 0$ to 2.5 in 2500 steps. Its momentum $p_5 = -h_5^2 u^5$ (column 13 is u^5) stays constant within a relative 10^{-8}, $g(u, u)$ stays -1 within 10^{-7}, and u^4 decreases at every step.

```
n_turn = int(np.argmax(extra[:, 12] < 0))  # the first step with u4 < 0
x4_max = float(np.max(extra[:, 4]))  # the largest x4 of the path
report("turning point: proper time tau", f"{tau_t[n_turn]:.3f}")
report("turning point: largest x4 reached", f"{x4_max:.4f}")
report("turning point: frame velocity along x5", f"{frame_t[n_turn, 5]:.4f}")
report("turning point: frame velocity along y", f"{frame_t[n_turn, 0]:.4f}")
```

`extra[:, 12] < 0` is a list of `True` and `False`, one per step; `np.argmax` of it is the position of the first `True`, the first step with $u^4 < 0$: the turning point, to within one step. Four RESULT lines: $\tau = 1.987$, the largest $x_4 = 1.4864$, $\hat{u}^5 = 1.4013$ (column 5 of `frame_t`), $\hat{u}^0 = -0.9817$ (column 0).

```
balance = frame_t[n_turn, 5] ** 2 - frame_t[n_turn, 0] ** 2 - 1  # 0 when u4 = 0
check(0 < n_turn and abs(balance) < 1e-4 and extra[-1, 4] < x4_max,
      "at the turning point (u5 hat)^2 - (u0 hat)^2 = 1, and x4 decreases afterwards")
say("record, sectorAndAnsatz.sector: " + ks_theory["sectorAndAnsatz"]["sector"])
check("extra-time momenta zero" in ks_theory["sectorAndAnsatz"]["sector"],
      "the good sector of the Kohn-Sham record has no extra-time momenta",
      record=f"{KS_THEORY}, sectorAndAnsatz.sector")
```

At the turning point $(\hat{u}^5)^2 - (\hat{u}^0)^2 = 1$ (Section 3.31; within 10^{-4}, because the step found is the first after the sign change), and the last x_4 of the path is smaller than the largest. Then the record's definition of the good sector is printed and checked to contain "extra-time momenta zero" (Section 3.31).

In [12], Figure 03d.1: redshift and blueshift.

```
BLUE, ORANGE, AQUA, YELLOW = "#2a78d6", "#eb6834", "#1baf7a", "#eda100"
MAGENTA, GREEN, VIOLET, RED = "#e87ba4", "#008300", "#4a3aa7", "#e34948"
COLOURS = {0.25: AQUA, 0.5: ORANGE, 1.0: RED}  # one colour per 3-space particle
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)  # room for the label of the right vertical axis
```

The palette, one colour per 3-space particle, and a figure with two panels.

```
for speed in SPEEDS:
    states, frame_rows = space_paths[speed][0], space_paths[speed][1]
    axes[0].plot(states[:, 4], frame_rows[:, 1], color=COLOURS[speed], lw=1.8,
                 label=f"RK4, start {speed:g}")
    axes[0].plot(states[:, 4], speed * np.exp(-A_value * H_value * states[:, 4]),
                 ":", color=COLOURS[speed], lw=1.4)
axes[0].plot([], [], ":", color="grey", label="pure redshift $e^{-a_4}$")
```

For each 3-space particle its frame velocity $\hat{u}^1$ against x_4 (solid) and the pure redshift $\hat{u}^1(0)e^{-a_4}$ that it would have at a fixed height (dotted). `axes[0].plot([], [], ...)` draws nothing; it only adds one grey dotted entry to the legend for all three dotted lines.

```
axes[0].set_yscale("log")
axes[0].set_xlabel("time $x_4$ (unit $1/H$), history $a_4 = x_4$, $b_4 = 1$")
axes[0].set_ylabel("frame velocity $\\hat u^1$ along $x_1$")
axes[0].set_title("3-space: redshift")
axes[0].legend(fontsize=8)
before = slice(0, n_turn)  # the states before the turning point
axes[1].plot(extra[before, 4], frame_t[before, 5], color=VIOLET, lw=1.8,
             label="RK4, start 0.2")
axes[1].plot(extra[before, 4], 0.2 * np.exp(A_value * H_value * extra[before, 4]),
             ":", color=VIOLET, lw=1.4, label="pure blueshift $0.2\\,e^{a_4}$")
```

A logarithmic axis and labels for the left panel. `slice(0, n_turn)` selects the rows before the turning point (after it x_4 runs backwards, and the curve would fold back). The right panel: $\hat{u}^5$ against x_4 and the pure blueshift $0.2e^{a_4}$.

```
axes[1].axvline(x4_max, color="grey", lw=1.0, ls="--", label="turning point")
axes[1].set_yscale("log")
axes[1].set_xlabel("time $x_4$ (unit $1/H$), history $a_4 = x_4$, $b_4 = 1$")
axes[1].set_ylabel("frame velocity $\\hat u^5$ along $x_5$")
axes[1].set_title("Extra time: blueshift")
axes[1].legend(fontsize=8, loc="upper left")
save_figure(fig, "frame_velocities",
            "Frame velocities of freely falling test particles in the author's metric "
...
check(frame_t[n_turn - 1, 5] > 0.2 * math.exp(extra[n_turn - 1, 4]),
      "the extra-time frame velocity grows faster than the pure blueshift")
```

A dashed line at the largest x_4, the logarithmic axis, labels and legend; the figure is saved (its caption contains the turning point $x_4 = 1.486$ through an f-string). The check: just before the turning point, $\hat{u}^5$ exceeds the pure blueshift.

What Figure 03d.1 shows. Along the history with $b_4 = 1$. Left: on the logarithmic axis the three frame velocities fall almost like straight lines of slope -1, the redshift e^{-a_4}; they fall a little faster than the dotted lines, most visibly for the fastest particle, because the particles rise towards the patch end where the warp factor e^{Hy} is larger and $\kappa(y, x_4) = e^{-Hy-a_4}$ smaller. Right: the frame velocity along x_5 grows from 0.2, first like the pure blueshift $0.2e^{a_4}$, then faster, because the particle sinks towards the tip where e^{-Hy} is larger; at the dashed line it reaches its turning point.

In [13], Figure 03d.2: the x_4 velocity and the time x_4.

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)
axes[0].plot(tau, rest[:, 12], color="black", lw=1.4, ls="--", label="at rest")
axes[1].plot(tau, rest[:, 4], color="black", lw=1.4, ls="--", label="at rest")
for speed in SPEEDS:
    states = space_paths[speed][0]
    axes[0].plot(tau, states[:, 12], color=COLOURS[speed], lw=1.8,
                 label=f"3-space, start {speed:g}")
    axes[1].plot(tau, states[:, 4], color=COLOURS[speed], lw=1.8,
                 label=f"3-space, start {speed:g}")
```

Two panels against the proper time τ: u^4 on the left, x_4 on the right; first the particle at rest (dashed black), then the three 3-space particles.

```
axes[0].plot(tau_t, extra[:, 12], color=VIOLET, lw=1.8, ls="-.",
             label="extra time, start 0.2")
axes[1].plot(tau_t, extra[:, 4], color=VIOLET, lw=1.8, ls="-.",
             label="extra time, start 0.2")
axes[0].axhline(0.0, color="grey", lw=0.8)
axes[0].plot([tau_t[n_turn]], [0.0], "o", color=VIOLET, ms=7)  # the turning point
axes[1].plot([tau_t[n_turn]], [x4_max], "o", color=VIOLET, ms=7)
```

The extra-time particle (dash-dotted), the zero line, and a dot at the turning point in each panel.

```
axes[0].set_ylabel("$x_4$ velocity $u^4 = dx_4/d\\tau$")
axes[1].set_ylabel("time $x_4$ (unit $1/H$)")
axes[0].set_title("The $x_4$ velocity can only decrease")
axes[1].set_title("The time $x_4$ along each path")
for ax, place in zip(axes, ("lower left", "upper left")):  # legends off the lines
    ax.set_xlabel("proper time $\\tau$ of the particle (unit $1/H$)")
    ax.legend(fontsize=8, loc=place)
save_figure(fig, "x4_velocity",
            "The $x_4$ velocity $u^4 = dx_4/d\\tau$ (left) and the time $x_4$ "
...
check(np.all(np.diff(space_paths[1.0][0][:, 4]) > 0),
      "the particles in 3-space keep advancing in x4")
```

Labels and titles; the loop gives both panels their horizontal label and a legend placed where no curve runs. The figure is saved; the check confirms that the fastest 3-space particle advances in x_4 at every step.

What Figure 03d.2 shows. Left: the particle at rest keeps $u^4 = 1$; the 3-space particles start above 1 and settle down towards 1 as their motion is redshifted away; the extra-time particle starts at 0.98 and its u^4 falls ever faster, through zero at the dot ($\tau = 1.99$) and on to negative values. Right: x_4 grows steadily for the first four particles, while for the extra-time particle it reaches a largest value, 1.486, and then decreases.

In [14], Figure 03d.3: the push along the hidden direction.

```
L_tip = physics["physics"]["L_tipCutoff"]  # the tip cut-off of the record, 3/H
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)
axes[0].plot(tau, np.full_like(tau, Y0), color="black", lw=1.4, ls="--",
             label="at rest")
for speed in SPEEDS:
    axes[0].plot(tau, space_paths[speed][2], color=COLOURS[speed], lw=1.8,
                 label=f"3-space, start {speed:g}")
axes[0].plot(tau_t, y_t, color=VIOLET, lw=1.8, ls="-.", label="extra time, start 0.2")
```

The tip cut-off $L = 3$ of the record (Section 3.9), a figure with two panels, and in the left panel the height y of every particle against τ (the particle at rest stays at $y = -1$, drawn with `np.full_like`).

```
axes[0].axhline(0.0, color="grey", lw=1.0, label="patch end $y = 0$")
axes[0].axhline(-L_tip, color="grey", lw=1.0, ls=":", label="tip cut-off $y = -3$")
axes[0].set_xlabel("proper time $\\tau$ (unit $1/H$)")
axes[0].set_ylabel("height $y$ in the hidden direction (unit $1/H$)")
axes[0].set_title("Rising and sinking along $y$")
axes[0].legend(fontsize=7, loc="lower left")
largest_gap = 0.0
```

Horizontal lines at the patch end and at the tip cut-off, labels, title and legend. `largest_gap` will collect the largest disagreement found below.

```
for (t_values, frame_rows), colour, label in (
        ((tau, space_paths[1.0][1]), RED, "3-space, start 1"),
        ((tau_t, frame_t), VIOLET, "extra time, start 0.2")):
    law = H_value * (np.sum(frame_rows[:, 1:4] ** 2, axis=1)
                     - np.sum(frame_rows[:, 5:8] ** 2, axis=1))
    numeric = np.gradient(frame_rows[:, 0], t_values)  # d(u0 hat)/dtau
    largest_gap = max(largest_gap, float(np.max(np.abs(numeric - law)[1:-1])))
```

For the fastest 3-space particle and the extra-time particle: the push law $H(S - E)$ at every computed state (`frame_rows[:, 1:4]` are the three 3-space columns 1, 2, 3, `frame_rows[:, 5:8]` the three extra-time columns 5, 6, 7), and the acceleration $d\hat{u}^0/d\tau$ (column 0) obtained from the computed states by `np.gradient`, which takes central differences inside the path and one-sided differences at its two ends. The largest difference inside the path (`[1:-1]` leaves out the two ends, where the one-sided differences are less accurate) is kept.

```
    axes[1].plot(t_values, law, color=colour, lw=1.8, label=f"law, {label}")
    axes[1].plot(t_values[::100], numeric[::100], "o", color=colour, ms=5,
                 label=f"from RK4, {label}")
axes[1].axhline(0.0, color="grey", lw=0.8)
axes[1].set_xlabel("proper time $\\tau$ (unit $1/H$)")
axes[1].set_ylabel("$d^2y/d\\tau^2$ (unit $H$)")
axes[1].set_title("The push: $H$ times the difference of squares")
axes[1].legend(fontsize=7, loc="lower left")
```

The law as a line and every hundredth numerical value as a dot (`[::100]` takes every hundredth element); the zero line, labels, title and legend.

```
report("largest difference between the numerical acceleration and the law",
       f"{largest_gap:.1e}")
save_figure(fig, "hidden_push",
            "The push along the hidden direction on freely falling test particles in "
...
check(largest_gap < 1e-4,
      "the computed acceleration of y agrees with the exact push law")
```

The RESULT line prints 3.4×10^{-6}, the error of the numerical derivative; the figure is saved, and the check requires agreement within 10^{-4}.

What Figure 03d.3 shows. Left: the 3-space particles rise towards the patch end $y = 0$, the faster the higher (the fastest reaches -0.06); the particle at rest stays at $y = -1$; the extra-time particle sinks towards the tip, ever faster, to about -2.3 at $\tau = 2.5$, still above the cut-off $y = -3$. Right: the dots computed from the paths lie on the lines of the exact law; the push is positive and dies away for the 3-space particle (its frame velocity is redshifted), negative and growing for the extra-time particle (its frame velocity is blueshifted).

In [15], Figure 03d.4: the quadratic form as a map.

```
grid5, grid0 = np.meshgrid(np.linspace(0.0, 3.0, 301), np.linspace(-3.0, 0.5, 351))
squared_u4 = 1 + grid0 ** 2 - grid5 ** 2  # (u^4)^2 over the plane
fig, ax = plt.subplots(figsize=(6.4, 5.2))
ax.contourf(grid5, grid0, squared_u4, levels=[-100.0, 0.0], colors=["#d9d9d9"])
ax.contour(grid5, grid0, squared_u4, levels=[0.0], colors="black", linewidths=1.2)
```

A grid of frame velocities, $\hat{u}^5$ from 0 to 3 (`grid5`) and $\hat{u}^0$ from -3 to 0.5 (`grid0`), and on it $(u^4)^2 = 1 + (\hat{u}^0)^2 - (\hat{u}^5)^2$, the formula of Section 3.31 for a particle that moves only along x_5 and y. `contourf` with the levels -100 and 0 fills in grey the region where the value is negative; `contour` at the level 0 draws the black boundary curve $(\hat{u}^5)^2 = 1 + (\hat{u}^0)^2$.

```
ax.plot(frame_t[:, 5], frame_t[:, 0], color=VIOLET, lw=2.0,
        label="path of the extra-time particle")
ax.plot([frame_t[0, 5]], [frame_t[0, 0]], "s", color=VIOLET, ms=7, label="start")
ax.plot([frame_t[n_turn, 5]], [frame_t[n_turn, 0]], "o", color=VIOLET, ms=8,
        label="turning point, $u^4 = 0$")
ax.text(2.05, 0.15, "$(u^4)^2 < 0$:\nno particle;\nplane waves grow", fontsize=8)
ax.text(0.3, -1.2, "$(u^4)^2 > 0$", fontsize=9)
```

The path of the extra-time particle in this plane (columns 5 and 0 of `frame_t`), a square at its start, a dot at its turning point, and two labels written into the regions (`\n` starts a new line of the label).

```
ax.set_xlabel("frame velocity $\\hat u^5$ along the extra time $x_5$")
ax.set_ylabel("frame velocity $\\hat u^0 = dy/d\\tau$")
ax.set_title("The quadratic form $(u^4)^2 = 1 + (\\hat u^0)^2 - (\\hat u^5)^2$")
ax.grid(False)
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "quadratic_form",
            "The squared $x_4$ velocity $(u^4)^2 = 1 + (\\hat u^0)^2 - (\\hat u^5)^2$ "
...
check(np.all(1 + frame_t[:, 0] ** 2 - frame_t[:, 5] ** 2 > -1e-6),
      "the path never enters the region where (u4)^2 < 0")
```

Labels, title, no grid lines, legend; the figure is saved; the check: along the whole path $(u^4)^2 > -10^{-6}$ (zero up to rounding at the turning point).

What Figure 03d.4 shows. The violet path starts at $(0.2, 0)$, moves to the right (blueshift) and downwards (it sinks, $dy/d\tau < 0$), touches the black curve at the turning point and then runs back into the white region, without ever entering the grey one. In the grey region the plane waves of the record's check would grow instead of oscillating; the particle avoids it by turning in x_4.

In [16], Figure 03d.5: how accurate the computation is.

```
from matplotlib.ticker import NullFormatter  # an empty label for minor ticks

p1_fast = space_paths[1.0][4]  # p_1 along the fastest 3-space path
drift = {"$p_1$, 3-space particle": (tau, np.abs(p1_fast / p1_fast[0] - 1)),
         "$g(u,u) + 1$, 3-space particle": (tau, np.abs(space_paths[1.0][3] + 1)),
         "$p_5$, extra-time particle": (tau_t, np.abs(p5 / p5[0] - 1)),
         "$g(u,u) + 1$, extra-time particle": (tau_t, np.abs(lengths_t + 1))}
```

`NullFormatter` is used below to suppress the labels of small ticks. `drift` is a dictionary of four curves, each a label with its times and the drift of a conserved quantity from its starting value: p_1 and $g(u,u)+1$ for the fastest 3-space particle, p_5 and $g(u,u)+1$ for the extra-time particle.

```
STEPS = (50, 100, 200, 400, 800, 1600)
differences = {}
for label, state0 in (("3-space, start 1", start(Y0, [0, 1, 0, 0, 0, 0, 0, 0])),
                      ("extra time, start 0.2", start(Y0, [0, 0, 0, 0, 0, 0.2, 0, 0]))):
    ends = [rk4_path(state0, 2.0, n)[1][-1] for n in STEPS]
    differences[label] = np.array([np.max(np.abs(ends[i] - ends[i + 1]))
                                   for i in range(len(STEPS) - 1)])
    ratios = differences[label][:-1] / differences[label][1:]
    say(f"  {label}: ratios of successive differences "
        + ", ".join(f"{r:.1f}" for r in ratios))
```

The order of RK4: each of the two particles is integrated from $\tau = 0$ to 2 with 50, 100, 200, 400, 800 and 1600 steps; `ends` holds the six end states (`[1][-1]` is the last row of the table of states). `differences` holds the five largest differences between the end states of successive step counts, and `ratios` the four ratios of successive differences, printed: 17.1, 16.6, 16.3, 16.2 and 17.2, 16.7, 16.3, 16.2. For a method of order 4 halving the step divides the error by about $2^4 = 16$ (Chapter 2).

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.3))
fig.subplots_adjust(wspace=0.3)
for (label, (t_values, values)), colour, style in zip(
        drift.items(), (RED, ORANGE, VIOLET, AQUA), ("-", "--", "-.", ":")):
    axes[0].semilogy(t_values[1:], np.maximum(values[1:], 1e-17), color=colour,
                     ls=style, lw=1.6, label=label)
axes[0].set_xlabel("proper time $\\tau$ (unit $1/H$)")
axes[0].set_ylabel("relative drift from the start")
axes[0].set_title("Conserved quantities, RK4 with step $0.001$")
axes[0].legend(fontsize=7, loc="lower right")
```

The left panel draws the four drifts on a logarithmic axis (`semilogy`); the first point ($\tau = 0$, where the drift is exactly zero) is left out, and `np.maximum(values, 1e-17)` replaces any other exact zero by 10^{-17}, because a logarithmic axis cannot show zero.

```
dts = 2.0 / np.array(STEPS[:-1])  # the larger step of each pair
for (label, values), colour in zip(differences.items(), (RED, VIOLET)):
    axes[1].loglog(dts, values, "o-", color=colour, lw=1.6, label=label)
axes[1].loglog(dts, differences["extra time, start 0.2"][0] * (dts / dts[0]) ** 4,
               "--", color="black", lw=1.0, label="slope 4")
axes[1].set_xticks(dts, [f"{dt:g}" for dt in dts])  # the five steps as labels
axes[1].xaxis.set_minor_formatter(NullFormatter())  # no labels between them
```

The right panel: the differences against the larger step $\Delta\tau$ of each pair (0.04, 0.02, 0.01, 0.005 and 0.0025), on logarithmic axes, with a dashed reference line of slope 4; the five steps are written at the ticks, and the small ticks between them get no labels.

```
axes[1].set_xlabel("step $\\Delta\\tau$ of RK4")
axes[1].set_ylabel("difference of end states (step and half step)")
axes[1].set_title("RK4 is of order 4")
axes[1].legend(fontsize=8)
save_figure(fig, "accuracy",
            "The accuracy of the RK4 computation of free fall in the author's metric "
...
all_ratios = np.concatenate([d[1:-1] / d[2:] for d in differences.values()])
check(np.all((all_ratios > 14) & (all_ratios < 18)),
      "halving the RK4 step divides the error by 14 to 18: order 4")
```

Labels, title, legend; the figure is saved. For the check, the coarsest pair (where the error is not yet in its final form) is left out: `d[1:-1] / d[2:]` gives the last three ratios of each particle, $16.6, 16.3, 16.2$ and $16.7, 16.3, 16.2$, and all six must lie between 14 and 18 (`&` combines the two conditions element by element).

What Figure 03d.5 shows. Left: the conserved quantities drift by at most about 10^{-8} (the squared speed of the extra-time particle at the end, when its velocities have grown large), the 3-space quantities by less than 10^{-11}. Right: both sets of points lie on lines of slope 4: halving the step makes the error 16 times smaller. The computed paths are accurate far beyond the precision of any number quoted from them.

In [17], the last check.

```
figure_names = ["frame_velocities", "x4_velocity", "hidden_push", "quadratic_form",
                "accuracy"]
files = [output_file(f"{FIGURE_FOLDER}/03d_{n}_{name}.png")
         for n, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in files), "all five figure files exist")
all_checks_passed()
```

The five figure files must exist; the last line prints ALL 23 CHECKS PASSED (notebook 03d). The 23 checks are: 1 in In [3], 2 each in In [4] and In [5], 3 in In [6], 1 each in In [7], In [8] and In [9], 3 each in In [10] and In [11], and 1 each in In [12] to In [17].

3.36 What we proved, what we computed, what we assumed

PROVED in this chapter (each derivation written out line by line; every one is also confirmed by an exact check of a notebook, and where a Revision record states the same, the record and its check are named in the section):

- the line elements of the plane in polar coordinates and of the sphere; the inverse of a diagonal metric (Section 3.3);

- for the author's metric, written in the author's coordinates $x_0, \ldots, x_7$ with the two metric functions $a_4(x_4)$ and $b_4(x_4) > 0$: the signature (4,4) at every point of the patch, for every a_4 and every $b_4 > 0$; $\det g = b_4^3 \cos^2 z$ and $\sqrt{|\det g|} = b_4^{3/2} \cos z$, without a_4 (record `python-lovelock-report.json`, check `sqrt_abs_det_g`; `curvature.json`, key `sqrtAbsDetG`); the 7-volume density, which changes in time only through $b_4^{3/2}$, and the 7-volume $b_4^{3/2}/(6H)$ of the patch per unit transverse coordinate volume; $V_3 V_t = b_4^{3/2} \sin z$ (Sections 3.5 and 3.6);

- the scale factors and the diagonal vielbein (lead check `vielbein_reproduces_metric`); the expansion rates $\alpha = a_4'$ of 3-space and $\alpha_t = -a_4' + b_4'/(2b_4)$ of the extra times ($-a_4'$ for a constant b_4); the transverse product $b_4^{3/2} \sin z$ (Section 3.7);

- the hidden coordinate y: $dy = \cot z \, dx_0$ (lead check `ks_coordinate_jacobian`), $W = \sin^{1/6} z = e^{Hy}$, the warped form and the volume factor $b_4^{3/2} e^{6Hy}$, at $b_4 = 1$ those of the record `ks-theory.json`; the tip at infinite proper distance; the fraction e^{-6HL} beyond the tip cut-off (Section 3.9);

- the transformation rules of vectors, covectors and the metric; the invariance of contractions (Section 3.14); the covariant derivative of covectors and of tensors with two indices (one $+\Gamma$ term for every upper index, one $-\Gamma$ term for every lower index), in particular of the metric; the Christoffel formula as the only connection without torsion that is compatible with the metric; the four cases for a diagonal metric; the contracted symbol $\sum_a \Gamma^a{}_{ab} = \partial_b \ln \sqrt{|\det g|}$ (Section 3.15);

- along a geodesic the squared speed is constant, and a coordinate absent from a diagonal metric has a conserved momentum; the geodesic equations of the sphere; the equator is a geodesic and other circles of latitude are not; parallel transport around a circle of latitude turns a vector by $2\pi(1 - \cos\theta_0)$, the enclosed area divided by a^2 (Section 3.16);

- the commutator of covariant derivatives is the Riemann tensor (MTW convention); its antisymmetry in the last two indices and the first Bianchi identity; each diagonal entry of the Ricci tensor is the sum of the curvatures of the coordinate planes through that direction (Section 3.17);

- the plane is flat; the sphere has the Gaussian curvature $1/a^2$, $R = 2/a^2$, $K = 4/a^4$; two meridians approach as $a \cos(s/a) \Delta\varphi$ (Section 3.18);

- the 37 non-zero Christoffel symbols of the author's metric (25 with $b \le c$, record `curvature.json`, key `christoffelNonzero_b_le_c`); observers at rest fall freely with proper time x_4, for all a_4 and b_4 (Section 3.23);

- the curvatures $\sigma(a, b)$ of the 28 coordinate planes in the rates α and α_t, and what their signs mean (a shape only for two space-like directions, reversed for two time-like ones); for two observers at rest $d^2\xi/d\tau^2 = \sigma(x_1, x_4)\, \xi$ along 3-space and $\sigma(x_5, x_4)\, \xi$ along an extra time; the 156 non-zero components $R^{ab}{}_{cd}$ with 20 different values (record `lovelock-report.json`, check `riemann_antisymmetry`); their frame values do not depend on z (Section 3.24);

- $R^a{}_a$, $R = 6a_4'^2 - 3a_4' b_4'/b_4 + 3b_4''/b_4 - 42H^2$ and the Einstein tensor (record `curvature.json`, keys `ricciMixed`, `ricciScalar`, `einsteinMixed`); its only entries off the diagonal are $G^4{}_0 = \frac{3}{2} H \cot z \, b_4'/b_4$ and $G^0{}_4 = -\frac{3}{2} H \tan z \, b_4'/b_4$ (verifier of the metric with b_4, checks `off_diagonal_entries_x0_x4_only`, `off_diagonal_proportional_to_b4prime`), so the Ricci and

Einstein tensors are diagonal exactly for a constant b_4, when the three inflating and the three deflating directions cancel (with inflating extra times, $b_4 = e^{4a_4}$, they would not); at $b_4 = 1$, $G^4{}_4 - G^0{}_0 = 6(a_4'^2 + H^2) > 0$; for a constant b_4, $K = 12(7H^4 - 2H^2a_4'^2 + 7a_4'^4 + 2a_4''^2) \geq 576H^4/7 > 0$, so the metric is curved for every such history; the contracted Bianchi identity for all $a_4(x_4)$ and $b_4(x_4)$ (record `lovelock-report.json`, checks `k1_equals_minus_4_einstein`, `k1_divergence_free`) (Section 3.25);

- within Einstein's equations: a diagonal source requires a constant b_4, that is $b_4 = 1$; no empty spacetime has this metric; $\kappa(\rho + p_0) = -6(a_4'^2 + H^2) < 0$; along $a_4 = AHx_4$, $b_4 = 1$ the required source $\kappa\rho = -3H^2(7 + A^2) - \Lambda$, $\kappa p = 3H^2(5 - A^2) + \Lambda$, $\kappa(\rho + p) = -6H^2(A^2 + 1) < 0$ (record `a4-equations.json`, keys `einstein` and `linearMember`) (Section 3.26);

- in free fall: the six momenta and $g(u, u)$ are conserved; $\hat{u}^i = p_i e^{-Hy - a_4}$ (the momentum weight $\kappa(y, x_4)$ of `ks-theory.json`) and $\hat{u}^j = -p_j b_4^{-1/2} e^{a_4 - Hy}$, so, for a constant b_4, 3-space motion is redshifted and extra-time motion blueshifted; $d^2y/d\tau^2 = H(S - E)$; $du^4/d\tau = -a_4' S + \alpha_t E$; $(u^4)^2 = 1 + (\hat{u}^0)^2 + S - E$, which times m^2 is the quadratic form of the record's check `extra_time_growth_rates_unbounded`; without extra-time motion a particle never turns in x_4 (Section 3.31).

COMPUTED by the notebooks (each number in the cell named; where a Revision record is reproduced, its file and key or check):

- Notebook 03a: $g_{11} = 2.347679$ and $g_{55} = -0.317724$ at $z = 0.7$, $a_4 = 0.5$, $b_4 = 1$ (In [8]); the signs on a grid of 400 values of z, 13 of a_4 and 3 of b_4 (In [8]); the slices, $H = 1$, $A = 1$ and $L = 3$ (In [13]; `Revision/kohn_sham/results/parameters.json`); the growth and shrink factors $e^2 = 7.389056$ and $e^{-2} = 0.135335$ (In [14]); the cut-off fraction $e^{-18} = 1.5230 \times 10^{-8}$ and the cut-off angle 1.5230×10^{-8} radians (In [20]).

- Notebook 03c: the turning angles of 15 circles within 5.0×10^{-12} radians of the area formula (In [10]); three geodesics within 1.2×10^{-13}, 6.2×10^{-13} and 7.1×10^{-11} of their planes (In [11]); the deviation equation within 8.0×10^{-14} of two meridians (In [13]).

- Notebook 03b: the 512 Christoffel symbols by central differences, order 2.040, best error 5.7×10^{-11} at $h = 10^{-6}$ at the test point of the record's check `k1_brute_force_numeric` (with $b_4 = 1.37$, $b_4' = 0.31$; In [10], In [11]); all 310 recorded components equal to ours at the five test points of `python-lovelock-report.json` to the relative 2.96×10^{-31} with 30 digits (In [19]); K summed from the 156 components equal to the formula at 300 values of z, at $b_4 = 1$ (In [23]).

- Notebook 03d: along the prescribed history ($b_4 = 1$) with RK4 and the step 0.001, from $y = -1$: the particle at rest stays at rest (In [9]); the fastest 3-space particle at $\tau = 3$ has $x_4 = 3.3070$, $\hat{u}^1 = 0.014370$, $u^4 = 1.060733$, $y = -0.0643$ (In [10]); the extra-time particle turns at $\tau = 1.987$ with the largest $x_4 = 1.4864$, $\hat{u}^5 = 1.4013$, $\hat{u}^0 = -0.9817$ (In [11]); the push law within 3.4×10^{-6} (In [14]); RK4 ratios 16.2 to 17.2 (In [16]).

ASSUMED (used, not derived here):

- standard theorems quoted without proof: Sylvester's law of inertia (Section 3.4); the volume factor of a metric that is not diagonal (not needed); that the Christoffel symbols of two systems

of coordinates describe the same covariant derivative (Section 3.15); that a space with zero Riemann tensor has constant metrics locally (Section 3.17); the symmetries (S3) and (S4) and the contracted Bianchi identity for a general metric (Sections 3.17 and 3.25; for the author's metric all three are checked or proved); the uniqueness of solutions of ordinary differential equations (Section 3.16); the geodesic deviation equation of a general surface, and its form $d^2\xi/d\tau^2 = +\sigma\,\xi$ for free-fall paths along a time-like direction (Section 3.18; for the observers at rest in the author's metric it is proved in Section 3.24);

- from physics: that a body in free fall moves on a time-like geodesic, and that the bodies of Section 3.31 are test particles; the form of Einstein's field equations and the null energy condition as the mark of ordinary matter (Section 3.26; derived and discussed in Chapters 9 and 12);
- the history $a_4 = AHx_4$, $b_4 = 1$ with $A = 1$ and $H = 1$, used for every picture and every number along the history: a PRESCRIBED BACKGROUND, not a solution of the field equations with a source of this book (key `adiabaticity.historyStatus` of `Revision/kohn_sham/ks-theory.json`, and `Revision/field_equations_a4/reports/ks-source-conditions.json`); $b_4 = 1$ is the choice of the Revision record for every background (`Revision/SPEC.md`, section 1; Section 3.8). Every formula written with general $a_4(x_4)$ and $b_4(x_4)$ is exact without it.

HYPOTHESIS. None enters this chapter.

OPEN. Whether wave packets of the fields of the book follow the free-fall paths of Section 3.31 in the curved metric is not computed (Section 3.31).

This chapter says nothing about pairs of universes, their creation, or matter and antimatter; Chapters 18 to 21 treat these questions and state exactly what is and what is not proved about them.

3.37 Exercises

Exercise 1. In the plane, use the coordinates $u = x + y$ and $v = x - y$. Find the line element, the metric and its Christoffel symbols.

Answer. Solving for x and y: $x = (u+v)/2$ and $y = (u-v)/2$ (add and subtract the two definitions). So $dx = (du + dv)/2$ and $dy = (du - dv)/2$ (each coefficient is constant), and

$$ds^2 = dx^2 + dy^2 = \frac{(du+dv)^2 + (du-dv)^2}{4} = \frac{2\,du^2 + 2\,dv^2}{4} = \frac{du^2 + dv^2}{2}$$

(the squares of a sum and of a difference; the mixed terms $\pm 2\,du\,dv$ cancel). The metric is $\mathrm{diag}(1/2, 1/2)$. It is constant, so all its derivatives vanish and every Christoffel symbol is zero (Section 3.15): in these coordinates straight lines are again $d^2u/d\lambda^2 = d^2v/d\lambda^2 = 0$.

Exercise 2. Figure 03a.1 shows the author's metric at $z = \pi/4$, $a_4 = 0.5$, $b_4 = 1$ with the entries 1, 2.422 (three times), -1 and -0.328 (three times), in the order $x_0, \ldots, x_7$. Multiply them and compare with $\det g = b_4^3 \cos^2 z$. Then do the same exactly, and say how the product changes for $b_4 = 2$.

Answer. With the rounded numbers: $2.422 \times 0.328 = 0.794416$, and $(0.794416)^3 = 0.5013$; the product of the eight entries is $1 \cdot 2.422^3 \cdot (-1) \cdot (-0.328)^3 = (2.422 \times 0.328)^3 = 0.5013$ (an odd power

keeps the minus sign, and the two minus signs cancel). $b_4^3\cos^2(\pi/4) = 1/2$. Exactly: the entries are $e\,s$ and $-e^{-1}s$ with $s = (\sin\pi/4)^{1/3}$, so the product is $\cot^2(\pi/4)\cdot(e\,s)^3\cdot(-1)\cdot(-e^{-1}s)^3 = s^6 = \sin^2(\pi/4) = 1/2$ ($e^3e^{-3} = 1$; $\cot(\pi/4) = 1$). The difference 0.0013 comes only from rounding the entries to three decimals. For $b_4 = 2$ each of the three extra-time entries doubles, so the product is multiplied by $2^3 = 8$: $\det g = 4$, as $b_4^3\cos^2 z$ says.

Exercise 3. Compute ${\Gamma^{x_4}}_{x_5x_5}$ and ${\Gamma^{x_0}}_{x_5x_5}$ of the author's metric directly from the Christoffel formula, without the four cases.

Answer. The formula of Section 3.15 with $a = x_4$, $b = c = x_5$ keeps only $d = x_4$ (the metric is diagonal): ${\Gamma^4}_{55} = \frac{1}{2}g^{44}(\partial_5 g_{45} + \partial_5 g_{45} - \partial_4 g_{55}) = \frac{1}{2}(-1)(0 + 0 - \partial_4 g_{55}) = \frac{1}{2}\partial_4(-b_4e^{-2a_4}\sin^{1/3} z) = \frac{1}{2}(-b_4' + 2a_4'b_4)e^{-2a_4}\sin^{1/3} z = (a_4'b_4 - \frac{1}{2}b_4')e^{-2a_4}\sin^{1/3} z$ ($g_{45} = 0$; $g^{44} = -1$; the product rule for $b_4\cdot e^{-2a_4}$ and the chain rule, $\partial_4 e^{-2a_4} = -2a_4'e^{-2a_4}$). For ${\Gamma^0}_{55}$ the formula with $a = x_0$, $b = c = x_5$ keeps only $d = x_0$, and the two terms with $g_{05} = 0$ vanish: ${\Gamma^0}_{55} = -\frac{1}{2}g^{00}\partial_0 g_{55} = -\frac{1}{2}\tan^2 z\cdot(-b_4e^{-2a_4})\cdot\frac{1}{3}\sin^{-2/3} z\cos z\cdot 6H = H\tan^2 z\, b_4e^{-2a_4}\sin^{-2/3} z\cos z$ ($g^{00} = 1/\cot^2 z = \tan^2 z$; the power rule and the chain rule for $\sin^{1/3}(6Hx_0)$). Since $\tan^2 z\cos z\sin^{-2/3} z = \tan z\cdot\frac{\sin z}{\cos z}\cos z\sin^{-2/3} z = \tan z\sin^{1/3} z$, this is $H\tan z\, b_4e^{-2a_4}\sin^{1/3} z$. Both agree with the table of Section 3.23.

Exercise 4. (a) By what angle does a vector turn when it is carried once around the equator of a sphere? (b) Show that for a small circle around the pole, $\theta_0 \ll 1$, the turning angle is about $\pi\theta_0^2$, and explain this number.

Answer. (a) With $\theta_0 = \pi/2$, $\cos\theta_0 = 0$: the transport equations of Section 3.16 give $du/d\varphi = dw/d\varphi = 0$, so the vector does not turn at all relative to the south and east directions; the formula $2\pi(1 - \cos\theta_0) = 2\pi$ is one whole turn, the same direction. This fits: the cap is a hemisphere of area $2\pi a^2$, and the equator is a geodesic, along which a vector keeps its angle to the direction of travel. (b) For small θ_0, $\cos\theta_0 \approx 1 - \theta_0^2/2$ (Taylor's formula, Chapter 2), so $2\pi(1 - \cos\theta_0) \approx 2\pi\cdot\theta_0^2/2 = \pi\theta_0^2$. A small cap is almost a flat disc of radius $a\theta_0$ (the distance from the pole along the sphere), with the area $\pi a^2\theta_0^2$; times the curvature $1/a^2$ this is $\pi\theta_0^2$. For $\theta_0 = 0.1$ the exact angle is $2\pi(1-\cos 0.1) = 0.031390$ and the estimate 0.031416; Figure 03c.3 shows the first dot at this height.

Exercise 5. Show that the Einstein tensor of every surface (two dimensions) is zero.

Answer. By Section 3.17 a surface has ${R^1}_1 = {R^2}_2 = \sigma(1,2)$ and $R = 2\sigma(1,2)$. The off-diagonal entries vanish: ${R^1}_2 = \sum_c {R^{1c}}_{2c} = {R^{11}}_{21} + {R^{12}}_{22} = 0$ (antisymmetry in the upper pair and in the lower pair). So ${G^1}_1 = \sigma(1,2) - \frac{1}{2}\cdot 2\sigma(1,2) = 0$, likewise ${G^2}_2 = 0$, and ${G^1}_2 = {G^2}_1 = 0$. For the sphere, $1/a^2 - \frac{1}{2}\cdot 2/a^2 = 0$. In two dimensions the Einstein tensor says nothing about the curvature; this is why gravity needs more dimensions.

Exercise 6. For the canonical history $A = 1$ and $\Lambda = 0$, find $\kappa\rho$, κp and the ratio $w = p/\rho$ that Einstein's equations require. Which Λ would make $\rho = 0$, and what is then κp?

Answer. From Section 3.26: $\kappa\rho = -3H^2(7+1) - 0 = -24H^2$ and $\kappa p = 3H^2(5-1) + 0 = 12H^2$, so $w = p/\rho = 12/(-24) = -1/2$; the energy density is negative. $\rho = 0$ needs $\Lambda = -3H^2(7 + A^2) = -24H^2$, and then $\kappa p = 12H^2 - 24H^2 = -12H^2$. In both cases $\kappa(\rho + p) = -12H^2 < 0$, as Section 3.26 says for every Λ: the cosmological constant cancels from $\rho + p$.

Exercise 7. Find the smallest value of the Kretschmann scalar of the author's metric for histories with $a_4'' = 0$, and the slope A at which it is reached.

Answer. With $a_4'' = 0$ and $s = a_4'^2 \geq 0$, $K = 12(7s^2 - 2H^2s + 7H^4)$ (Section 3.25). Its derivative

with respect to s is $12(14s - 2H^2)$, zero at $s = H^2/7$; there $K = 12H^4(7/49 - 2/7 + 7) = 12H^4(1/7 - 2/7 + 49/7) = 12H^4 \cdot 48/7 = 576H^4/7 = 82.29H^4$, a minimum because the parabola $7s^2 - \dots$ opens upwards. With $a_4' = AH$: $A^2 = 1/7$, $A = 1/\sqrt{7} = 0.378$. This is the low point of the red curve of Figure 03b.6, and the value $576H^4/7$ printed by Notebook 03b, In [22].

Exercise 8. The extra-time particle of Notebook 03d starts at $y = -1$ with $\hat{u}^5 = 0.2$. (a) If it stayed at $y = -1$ and never moved along y, at which x_4 would it turn? (b) Use the printed turning-point values $x_4 = 1.4864$ and $\hat{u}^5 = 1.4013$ and the conserved momentum p_5 to find the height y of the turning point.

Answer. (a) At fixed y, $\hat{u}^5 = 0.2\,e^{a_4}$ (pure blueshift, Section 3.31, with $b_4 = 1$) and $\hat{u}^0 = 0$; the turning condition $(\hat{u}^5)^2 = 1 + (\hat{u}^0)^2$ becomes $0.2\,e^{a_4} = 1$, so $a_4 = \ln 5 = 1.609$, that is $x_4 = 1.609$ on the history $a_4 = x_4$. (b) From $\hat{u}^5 = -p_5 b_4^{-1/2} e^{a_4 - Hy}$ with $b_4 = 1$ and the start ($a_4 = 0$, $y = -1$, $H = 1$): $-p_5 = 0.2\,e^{-1}$, so $\hat{u}^5 = 0.2\,e^{a_4 - (y+1)}$. At the turning point $1.4013 = 0.2\,e^{1.4864 - (y+1)}$, so $1.4864 - (y + 1) = \ln 7.0065 = 1.9468$ and $y = -1.460$. The particle has sunk by 0.46 towards the tip, which raised its frame velocity by the factor $e^{0.46} = 1.585$ beyond the pure blueshift, and it turned earlier ($x_4 = 1.486$) than the estimate (a), although its motion along y, $\hat{u}^0 = -0.98$, made the turning condition harder to reach ($(\hat{u}^5)^2 = 1.96$ instead of 1). Figure 03d.3 shows the particle near $y = -1.46$ at $\tau \approx 2$.

Exercise 9. What fraction of the 7-volume of the patch, and at which angle z, would a tip cut-off $L = 2/H$ remove?

Answer. By Section 3.9 the fraction is $e^{-6HL} = e^{-12} = 6.144 \times 10^{-6}$, about 400 times more than the $e^{-18} = 1.523 \times 10^{-8}$ of the record's $L = 3/H$ ($e^6 = 403$). The cut-off lies at $\sin z = e^{6Hy} = e^{-12}$, so $z = \arcsin e^{-12} = 6.144 \times 10^{-6}$ (for small u, $\arcsin u \approx u$).

4 Clifford algebra Cl(4,4) and the author's T16

The field equations of this book contain eight real 16×16 matrices, the **gamma matrices** $\gamma^{(x_0)}, \gamma^{(x_1)}, \dots, \gamma^{(x_7)}$, one for each direction of the author's eight-dimensional space-time. This chapter explains from zero why such matrices are needed, why they must have at least 16 rows and 16 columns, how the author built his matrices (he calls them T16) in his Mathematica notebook, which rules they obey, and what all their products together make up: the **Clifford algebra** Cl(4,4). The chapter has four worked examples, each a complete notebook. Notebook 04a rebuilds T16 from the author's formulas with exact whole-number arithmetic and checks every rule. Notebook 04b follows the square-root idea from the matrices of Pauli and Dirac to the author's eight gammas and to the plane waves of the field. Notebook 04c forms all 256 products of the gammas and proves that they make up all real 16×16 matrices. Notebook 04d builds a second set of gammas from three 2×2 matrices and finds the one and only change of basis between the two sets. Everything in this chapter is the algebra of constant matrices; it makes no statement about the creation of universes or about matter and antimatter.

4.1 Why matrices, and why sixteen components

The length of an arrow in the plane with the components p and q is $\sqrt{p^2 + q^2}$ (Pythagoras). A square root is an awkward object in an equation: it is not linear, and it cannot be differentiated term

by term. A **linear expression** in some variables $p, q, \ldots$ is a sum of the variables, each multiplied by a fixed number (or, below, a fixed matrix), its **coefficient**: $\alpha p + \beta q$ is linear, while p^2, pq and $\sqrt{p^2+q^2}$ are not. A **quadratic form** is, in this book, a sum of the squares of the variables, each with a fixed sign + or −: $p^2 + q^2$ and $p^2 - q^2$ are quadratic forms. In 1928 Paul Dirac looked for an equation for the electron that contains only first derivatives and whose square is the relation $E^2 = m^2 + k_1^2 + k_2^2 + k_3^2$ between the energy E, the mass m and the momentum (k_1, k_2, k_3) of a particle in special relativity; its right-hand side is a quadratic form in m, k_1, k_2, k_3. He found that a quadratic form can be written as the square of a linear expression, provided that its coefficients are matrices that **anticommute**: $\alpha\beta = -\beta\alpha$. Such matrices are called gamma matrices, and the rule they obey is the **Clifford relation**. Dirac needed four of them, for the time and the three directions of space, and the smallest ones are 4×4; that is why Dirac's electron field has four components.

The author's space-time has eight directions $x_0, x_1, \ldots, x_7$: a hidden space direction x_0, the three directions x_1, x_2, x_3 of ordinary space, the time x_4, and three **extra times** x_5, x_6, x_7, which deflate exponentially in the author's metric. Eight directions need eight gamma matrices, and Section 4.13 proves that eight such matrices must have at least 16×16 entries. This is why the fields dirac16complex and dirac16complex00 of this book have exactly sixteen components. The author's gamma matrices are **real**: every entry is −1, 0 or +1. Later chapters use this fact (Chapter 5 and Chapter 7); this chapter only proves it.

The chapter proceeds as follows.

- Section 4.2 collects the facts about matrices that the chapter uses, each with its proof.
- Section 4.3 solves the square-root problem and derives the Clifford relation.
- Section 4.4 describes the eight directions of the author's space-time and the signs η that tell space-like from time-like directions.
- Section 4.5 follows the author's construction of T16 step by step and proves its Clifford relation by hand; Notebook 04a (Sections 4.6 to 4.8) repeats the construction with the computer.
- Section 4.9 studies the square root of the 4+4 quadratic form and the plane waves of the field in flat 4+4 space, including what a momentum along an extra time does; Notebook 04b (Sections 4.10 to 4.12).
- Section 4.13 forms the 256 products of the gammas and proves that they are a basis of all real 16×16 matrices; Notebook 04c (Sections 4.14 to 4.16).
- Section 4.17 builds the gammas a second time, from 2×2 matrices, and proves that the two sets differ only by a renumbering of the sixteen components; Notebook 04d (Sections 4.18 to 4.20).
- Section 4.21 lists what was proved, computed and assumed, and Section 4.22 gives exercises with complete worked answers.

Every number of this chapter comes either from the Revision record or from the chapter's own notebooks, which reproduce the Revision record where they overlap. The Revision record files used here are:

- `Revision/algebra/gammas.json`: the eight gamma matrices of the author, written exactly, and the matrix B that Section 4.9 uses;
- `Revision/algebra/reports/wolfram-algebra.json`: the exact checks of the algebra computed with WolframScript (45 checks, all passed, as Notebook 04a reads and prints them);
- `Revision/algebra/reports/python-algebra.json`: the same algebra checked independently with Python (35 checks, all passed, as Notebook 04a prints them);
- `Revision/algebra/reports/python-gammas.json`: the gammas as rebuilt by the Python program;
- `Revision/theory/reports/python-field-theory.json` and `Revision/theory/reports/python-scope.json`, the theory reports from which Notebook 04b reproduces four checks about plane waves (each named where it is used).

Each statement carries a label: PROVED (exact, with the proof in the text and the record file and check that confirm it), COMPUTED (a floating-point computation with its measured accuracy), ASSUMED (taken as given), HYPOTHESIS or OPEN.

4.2 The matrix tools of this chapter

This section defines every word about matrices that the chapter uses and proves every rule.

Matrices and their entries. An $n \times n$ **matrix** M is a square table of numbers with n rows and n columns. M_{ij} is the **entry** in row i and column j. The notebooks count rows and columns from 0 (Python's convention), so a 16×16 matrix has the rows $0, 1, \ldots, 15$. The author's formulas for his 4×4 building blocks count from 1, as Mathematica does; where this chapter uses them it says so. A **column** u is a list of n numbers $u_0, \ldots, u_{n-1}$ written one below the other; the matrix acts on it by $(Mu)_i = \sum_j M_{ij} u_j$.

The product. The product of two $n \times n$ matrices is

$$(AB)_{ij} = \sum_k A_{ik} B_{kj},$$

row i of A times column j of B, entry by entry, added up. In Python it is written `A @ B`. Products are **associative**, $(AB)D = A(BD)$ (both sides are $\sum_{k,l} A_{ik} B_{kl} D_{lj}$), so brackets may be left out. They are not **commutative**: in general $AB \neq BA$. For example, with

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}: \qquad AB = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \qquad BA = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = -AB.$$

The **identity matrix** I_n has 1 on the diagonal and 0 elsewhere, so $I_n M = M I_n = M$. A number c times a matrix multiplies every entry by c; cI_n is called a **multiple of the identity**. The **Kronecker delta** δ_{ij} is 1 if $i = j$ and 0 otherwise, so $(I_n)_{ij} = \delta_{ij}$.

Linear maps are matrices. A rule f that turns every column u of n numbers into a column $f(u)$ of n numbers is called a **map**; it is **linear** if $f(u + v) = f(u) + f(v)$ and $f(cu) = c\, f(u)$ for

all columns u, v and all numbers c. A matrix M gives a linear map, the one that sends u to Mu, because every entry $(Mu)_i = \sum_j M_{ij}u_j$ is a linear expression in the entries of u. Conversely, every linear map is a matrix. Proof, line by line, with the **unit columns** $e_1, \dots, e_n$ (e_j has 1 in place j and 0 elsewhere):

1. $u = \sum_j u_j e_j$ — in place i the right side is $\sum_j u_j \delta_{ij} = u_i$.
2. $f(u) = \sum_j u_j\, f(e_j)$ — the two rules of a linear map, used for each term of line 1.
3. $f(u)_i = \sum_j f(e_j)_i\, u_j = (Mu)_i$ with $M_{ij} = f(e_j)_i$ — entry i of line 2, and the definition of the product of a matrix and a column.

So f is the matrix M whose column j is the column $f(e_j)$: **the images of the unit columns are the columns of the matrix**. A **linear combination** of matrices (or of columns) is a sum of them, each multiplied by a number.

Commutator and anticommutator. For two matrices A and B,

$$[A, B] = AB - BA, \qquad \{A, B\} = AB + BA.$$

A and B **commute** if $[A, B] = 0$, that is $AB = BA$, and **anticommute** if $\{A, B\} = 0$, that is $AB = -BA$. The two matrices of the example above anticommute.

The transpose. The transpose M^T exchanges rows and columns: $(M^T)_{ij} = M_{ji}$. The transpose of a product is the product of the transposes in the opposite order, $(AB)^T = B^T A^T$. Proof, line by line:

1. $((AB)^T)_{ij} = (AB)_{ji}$ — definition of the transpose.
2. $(AB)_{ji} = \sum_k A_{jk} B_{ki}$ — definition of the product.
3. $\sum_k A_{jk} B_{ki} = \sum_k (B^T)_{ik} (A^T)_{kj}$ — each factor rewritten with the definition of the transpose, and the two numbers in each term exchanged (numbers commute).
4. $\sum_k (B^T)_{ik} (A^T)_{kj} = (B^T A^T)_{ij}$ — definition of the product.

M is **symmetric** if $M^T = M$ and **antisymmetric** if $M^T = -M$. An antisymmetric matrix has zeros on its diagonal, because $M_{ii} = -M_{ii}$.

Inverse and orthogonal matrices. The **inverse** M^{-1} of M, when it exists, satisfies $MM^{-1} = M^{-1}M = I_n$. A real matrix is **orthogonal** if $M^T M = I_n$; then $M^{-1} = M^T$.

Signed permutation matrices. A **signed permutation matrix** has exactly one nonzero entry in every row and in every column, and that entry is $+1$ or -1. Acting on a column it puts the components into a new order and changes some of their signs. We describe such a matrix by the **code** of each row: the code $+5$ in row 2 means that the only nonzero entry of row 2 is $+1$ in column 5, so $(Mu)_2 = +u_5$; the code -0 means $-u_0$. Every signed permutation matrix is orthogonal. Proof: $(M^T M)_{ij} = \sum_k (M^T)_{ik} M_{kj} = \sum_k M_{ki} M_{kj}$ (definitions of product and transpose). For $i \neq j$ no row k has nonzero entries in both columns i and j (a row has only one nonzero entry), so every term is

0. For $i = j$ exactly one row k has a nonzero entry in column i, and it contributes $(\pm 1)^2 = 1$. So $M^T M = I_n$.

Products and transposes of signed permutation matrices are again signed permutation matrices. The transpose exchanges rows and columns, so it still has exactly one entry ± 1 in every row and every column. For a product AB of two signed permutation matrices, line by line:

1. $(AB)_{il} = \sum_k A_{ik} B_{kl} = A_{ij} B_{jl}$ for every l, where j is the one column in which row i of A is nonzero — definition of the product; all other terms contain a zero A_{ik}.
2. So row i of AB is $A_{ij} = \pm 1$ times row j of B, which has exactly one entry ± 1 — line 1.
3. In the same way $(AB)_{il} = A_{ik} B_{kl}$ for every i, where k is the one row in which column l of B is nonzero, so column l of AB is ± 1 times column k of A, which has exactly one entry ± 1.

So AB has exactly one entry ± 1 in every row and every column.

Block matrices. A 16×16 matrix can be cut into four 8×8 **blocks** (upper left, upper right, lower left, lower right), and an 8×8 matrix into four 4×4 blocks. Block matrices multiply like 2×2 matrices whose entries are matrices, keeping the order of the factors:

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} E & F \\ K & L \end{pmatrix} = \begin{pmatrix} AE + BK & AF + BL \\ CE + DK & CF + DL \end{pmatrix}.$$

Proof: the entry in row i and column j of the product is a sum over all columns k of the first factor; split it into the sum over the columns of the left half and the sum over the columns of the right half. For a row i of the upper half and a column j of the left half, the first part is the entry of AE and the second the entry of BK; the other three positions work in the same way. In the same way the transpose of a block matrix transposes the pattern and every block:

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix}^T = \begin{pmatrix} A^T & C^T \\ B^T & D^T \end{pmatrix}.$$

A matrix whose two off-diagonal blocks are zero is **block diagonal**; one whose two diagonal blocks are zero is **block off-diagonal**. By the block rule, the product of two block off-diagonal matrices is block diagonal, and the product of a block diagonal and a block off-diagonal matrix is block off-diagonal. A block diagonal or block off-diagonal matrix whose two nonzero blocks are signed permutation matrices is itself a signed permutation matrix: each of its rows runs through exactly one nonzero block, where it has exactly one entry ± 1, and so does each of its columns.

The trace. The trace $\operatorname{tr} M$ is the sum of the diagonal entries. It has the **cyclic property** $\operatorname{tr}(AB) = \operatorname{tr}(BA)$. Proof: $\operatorname{tr}(AB) = \sum_i \sum_k A_{ik} B_{ki}$ (definitions), and exchanging the order of the two finite sums and of the two numbers in each term gives $\sum_k \sum_i B_{ki} A_{ik} = \operatorname{tr}(BA)$.

The permutation sign. For a list of different numbers, an **inversion** is a pair of positions in which the larger number stands first. The **permutation sign** of the list (Mathematica calls it *Signature*) is $+1$ if the number of inversions is even and -1 if it is odd; for a list in which a number occurs twice it is 0. Examples: $(1, 2, 3, 4)$ has no inversion, sign $+1$; $(2, 1, 3, 4)$ has one, the pair $(2, 1)$, sign -1; $(2, 3, 1, 4)$ has two, the pairs $(2, 1)$ and $(3, 1)$, sign $+1$; $(1, 1, 3, 4)$ has sign 0. **Exchanging**

two entries of a list flips its sign. Proof: exchanging two *neighbouring* entries changes only whether this one pair is an inversion (every other pair keeps its order), so the number of inversions changes by exactly one. Exchanging the entries at the positions $i < j$ can be done with neighbour exchanges: move the entry at position i to the right, past $j - i$ neighbours, to position j; then the entry that was at position j stands at position $j - 1$; move it to the left, past $j - i - 1$ neighbours, to position i. That is $2(j - i) - 1$ neighbour exchanges, an odd number, so the sign flips.

4.3 The square-root problem and the Clifford relation

Numbers are not enough. Can $\sqrt{p^2 + q^2}$ be written as $\alpha p + \beta q$ with fixed coefficients α and β, for all numbers p and q? Square the candidate, keeping $\alpha\beta$ and $\beta\alpha$ apart, because for matrices the order matters:

1. $(\alpha p + \beta q)^2 = (\alpha p + \beta q)(\alpha p + \beta q)$ — definition of the square.
2. $= \alpha p\, \alpha p + \alpha p\, \beta q + \beta q\, \alpha p + \beta q\, \beta q$ — multiplied out (distributive law).
3. $= \alpha^2 p^2 + (\alpha\beta + \beta\alpha)\, pq + \beta^2 q^2$ — the numbers p and q commute with everything and are moved to the right.

This must equal $p^2 + q^2$ for all p and q. With $p = 1$, $q = 0$ it gives $\alpha^2 = 1$; with $p = 0$, $q = 1$ it gives $\beta^2 = 1$; with $p = q = 1$ it gives $\alpha^2 + \beta^2 + \alpha\beta + \beta\alpha = 2$, hence $\alpha\beta + \beta\alpha = 0$. Conversely, these three conditions make line 3 equal to $p^2 + q^2$. For ordinary numbers $\alpha\beta + \beta\alpha = 2\alpha\beta$, and $2\alpha\beta = 0$ forces $\alpha = 0$ or $\beta = 0$, which contradicts $\alpha^2 = \beta^2 = 1$. So no two numbers solve the problem (status PROVED; Notebook 04b confirms with sympy that the three equations have no solution).

Matrices are enough. The two matrices of Section 4.2,

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix},$$

satisfy $\sigma_x^2 = \sigma_z^2 = I_2$ (multiply out) and anticommute ($\sigma_z\sigma_x = -\sigma_x\sigma_z$, computed in Section 4.2). By line 3, with the number 1 replaced by I_2: $(p\sigma_x + q\sigma_z)^2 = (p^2 + q^2)I_2$. Example: $p = 3$, $q = 4$ gives

$$3\sigma_x + 4\sigma_z = \begin{pmatrix} 4 & 3 \\ 3 & -4 \end{pmatrix}, \qquad \begin{pmatrix} 4 & 3 \\ 3 & -4 \end{pmatrix}^2 = \begin{pmatrix} 16 + 9 & 12 - 12 \\ 12 - 12 & 9 + 16 \end{pmatrix} = 25\, I_2,$$

a matrix square root of $25 = 3^2 + 4^2$. Space-time needs **minus signs** too, and a real matrix can square to $-I_2$:

$$N = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \qquad N^2 = \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} = -I_2, \qquad \sigma_x N = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} = -N\sigma_x.$$

So $(p\sigma_x + qN)^2 = p^2 I_2 + q^2 N^2 = (p^2 - q^2)I_2$: a real square root of a *difference* of squares.

The general rule. We want matrices $\gamma^1, \ldots, \gamma^n$ such that, for given signs $\eta^{aa} = \pm 1$ and for all numbers $p_1, \ldots, p_n$,

$$\Big(\sum_a p_a\gamma^a\Big)^2 = \Big(\sum_a \eta^{aa}p_a^2\Big)I.$$

The signs are collected in the diagonal matrix $\eta = \mathrm{diag}(\eta^{11},\dots,\eta^{nn})$; its off-diagonal entries are $\eta^{ab}=0$ for $a\neq b$. The left side, line by line:

1. $\big(\sum_a p_a\gamma^a\big)^2 = \big(\sum_a p_a\gamma^a\big)\big(\sum_b p_b\gamma^b\big)$ — definition of the square; the summation letter of the second factor is called b, which changes nothing.
2. $= \sum_{a,b} p_a p_b\,\gamma^a\gamma^b$ — multiplied out; the numbers p_ap_b are moved to the front.
3. $= \sum_{a,b} p_a p_b\,\gamma^b\gamma^a$ — the same sum with the two summation letters renamed (a called b and b called a), and $p_bp_a = p_ap_b$.
4. $= \frac{1}{2}\sum_{a,b} p_a p_b\,(\gamma^a\gamma^b+\gamma^b\gamma^a)$ — half the sum of lines 2 and 3, which are equal.

If the **Clifford relation**

$$\gamma^a\gamma^b+\gamma^b\gamma^a = 2\eta^{ab}I \qquad \text{for all } a,b$$

holds, line 4 becomes $\frac{1}{2}\sum_{a,b} p_a p_b\,2\eta^{ab}I = \sum_a \eta^{aa}p_a^2\,I$ (only $a=b$ survives), the wanted square. Conversely, if the square rule holds for all p: choose $p_a=1$ and all other p zero, then the left side is $(\gamma^a)^2$ and the right side $\eta^{aa}I$; choose $p_a=p_b=1$ for two different a,b and all other p zero, then the left side is $(\gamma^a+\gamma^b)^2 = (\gamma^a)^2+(\gamma^b)^2+\gamma^a\gamma^b+\gamma^b\gamma^a$ and the right side $(\eta^{aa}+\eta^{bb})I$, so $\gamma^a\gamma^b+\gamma^b\gamma^a=0$. So **the square-root rule and the Clifford relation say the same thing** (status PROVED). In words: each gamma matrix squares to $+I$ for a direction with sign $+1$ (a **space-like** direction) and to $-I$ for a direction with sign -1 (a **time-like** direction), and two different gamma matrices anticommute.

Pauli's three matrices. For three space directions (signs $+1,+1,+1$) a third 2×2 matrix is needed that squares to $+I_2$ and anticommutes with σ_x and σ_z. No real one exists (Exercise 1 of Section 4.22 shows what the real N gives instead), but a complex one does. With the imaginary unit i, $i^2=-1$, the three **Pauli matrices** (Wolfgang Pauli, 1927) are

$$\sigma_x = \begin{pmatrix}0&1\\1&0\end{pmatrix}, \qquad \sigma_y = \begin{pmatrix}0&-i\\i&0\end{pmatrix}, \qquad \sigma_z = \begin{pmatrix}1&0\\0&-1\end{pmatrix}.$$

For example $\sigma_x\sigma_y = \begin{pmatrix}i&0\\0&-i\end{pmatrix} = i\sigma_z$ and $\sigma_y\sigma_x = -i\sigma_z$, so they anticommute; and $\sigma_x\sigma_y\sigma_z = i\sigma_z\sigma_z = iI_2$.

Dirac's four matrices. Four anticommuting matrices do not fit into 2×2 matrices (Section 4.13 proves that four need at least 4×4). Dirac's choice, in the convention of particle physicists with the time first and the signs $(+1,-1,-1,-1)$, is made of 2×2 blocks (the letter D keeps these matrices apart from the author's):

$$\gamma^0_D = \begin{pmatrix}I_2&0\\0&-I_2\end{pmatrix}, \qquad \gamma^j_D = \begin{pmatrix}0&\sigma_j\\-\sigma_j&0\end{pmatrix} \quad (j=x,y,z).$$

By the block rule of Section 4.2: $(\gamma_D^0)^2 = \mathrm{diag}(I_2, I_2) = I_4$; $(\gamma_D^j)^2 = \mathrm{diag}(-\sigma_j^2, -\sigma_j^2) = -I_4$; $\gamma_D^0\gamma_D^j = \begin{pmatrix} 0 & \sigma_j \\ \sigma_j & 0 \end{pmatrix}$ and $\gamma_D^j\gamma_D^0 = \begin{pmatrix} 0 & -\sigma_j \\ -\sigma_j & 0 \end{pmatrix}$, which anticommute; and for $j \neq k$, $\gamma_D^j\gamma_D^k + \gamma_D^k\gamma_D^j = \mathrm{diag}(-\{\sigma_j, \sigma_k\}, -\{\sigma_j, \sigma_k\}) = 0$.

Dirac's equation contains the energy relation. For a plane wave $\psi = u\, e^{-i(Et - k_1x_1 - k_2x_2 - k_3x_3)}$ with a constant column u, Dirac's equation $i(\gamma_D^0\partial_t + \gamma_D^x\partial_1 + \gamma_D^y\partial_2 + \gamma_D^z\partial_3)\psi = m\psi$ becomes $(P - mI_4)u = 0$ with $P = E\gamma_D^0 - k_1\gamma_D^x - k_2\gamma_D^y - k_3\gamma_D^z$ (the derivative of e^{cx} is $c\,e^{cx}$; multiply by i; divide by the exponential, which is never zero). By the general rule with the signs $(+1, -1, -1, -1)$, $P^2 = (E^2 - k_1^2 - k_2^2 - k_3^2)I_4$. A nonzero u with $Pu = mu$ gives $P^2u = P(mu) = m^2u$, hence $(E^2 - k_1^2 - k_2^2 - k_3^2 - m^2)u = 0$, and because $u \neq 0$, $E^2 = m^2 + k_1^2 + k_2^2 + k_3^2$: the energy relation of special relativity, the **mass shell**. Notebook 04b checks all of this exactly and shows it in pictures.

The Clifford algebra. Given n matrices with a Clifford relation, the set of all their real linear combinations of products is called the **Clifford algebra** of the signs η; with p signs $+1$ and q signs -1 it is written $\mathrm{Cl}(p, q)$. The author's space-time has four space-like and four time-like directions, so its Clifford algebra is Cl(4,4). Section 4.13 shows that it is the set of all real 16×16 matrices.

4.4 The eight directions of the author's space-time and their signs

The author's metric is the diagonal 8×8 matrix g that gives the squared length of a small step: a step dx_μ along the coordinate x_μ has the squared length $g_{\mu\mu}\,dx_\mu^2$. Its diagonal entries, in the order of the author's coordinates $x_0, \ldots, x_7$, are $\cot^2 z$, $e^{2a_4}\sin^{1/3} z$ three times, -1, and $-b_4e^{-2a_4}\sin^{1/3} z$ three times, where a_4 and $b_4 > 0$ are functions of the time x_4 (the metric functions; $b_4 = 1$ gives the metric used before 2026-10-14), $z = 6Hx_0$ lies between 0 and $\pi/2$, and $H > 0$ is the author's constant (Chapter 3 studies this metric in full). A direction with $g_{\mu\mu} > 0$ is **space-like**, one with $g_{\mu\mu} < 0$ is **time-like**. The **frame factor** of a direction is $\sqrt{|g_{\mu\mu}|}$: a coordinate step dx_μ is a step of the length $\sqrt{|g_{\mu\mu}|}\,|dx_\mu|$.

coordinate	**role**	**metric entry**	**frame factor**	**sign η**
x_0	the hidden space direction	$\cot^2 z$	$\cot z$	$+1$
x_1, x_2, x_3	ordinary 3-space, inflating	$e^{2a_4}\sin^{1/3} z$	$e^{a_4}\sin^{1/6} z$	$+1$
x_4	the time	-1	1	-1
x_5, x_6, x_7	the extra times, deflating exponentially	$-b_4e^{-2a_4}\sin^{1/3} z$	$b_4^{1/2}e^{-a_4}\sin^{1/6} z$	-1

When a_4 grows with the time x_4 (and b_4 stays constant), the frame factor e^{a_4} of 3-space grows (3-space **inflates**) and the frame factor $b_4^{1/2}e^{-a_4}$ of the extra times shrinks exponentially (the extra times **deflate**). Chapter 12 derives the equations that govern a_4 and b_4.

At every point one can choose eight perpendicular directions of unit length, one along each coordinate: a **frame**. Measured in the frame, the metric becomes the diagonal matrix of signs

$$\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1) \qquad (\text{order } x_0,\ldots,x_7):$$

four space-like directions (x_0,x_1,x_2,x_3) and four time-like ones (x_4,x_5,x_6,x_7), the **signature** (4,4). Because η is diagonal with entries ±1, the entries of η and of its inverse are the same numbers, $\eta^{ab}=\eta_{ab}$. The gamma matrices belong to the frame directions: $\gamma^{(x_a)}$ is the gamma matrix of the frame direction along x_a, and they satisfy the Clifford relation with this η. They are **constant**: the same matrices at every point and at every time. The inflation of 3-space and the deflation of the extra times reach the field equation only through the frame factors: the field equation contains $\gamma^{\mu}=\gamma^{(x_\mu)}/\sqrt{|g_{\mu\mu}|}$, for example $\gamma^{(x_5)}$ multiplied by $b_4^{-1/2}e^{a_4}\sin^{-1/6}z$, a factor that grows as the extra time x_5 deflates. The gamma matrices themselves never change.

The author's Mathematica notebook numbers the frame directions $A=0,\ldots,7$ in the order of his coordinates: $A=0$ is the hidden direction, $A=1,2,3$ are 3-space, $A=4$ is the time and $A=5,6,7$ are the extra times, and its matrix of signs is eta4488 $=\mathrm{diag}(1,1,1,1,-1,-1,-1,-1)$ (his input cell In[45]). The coordinate map of the Revision record:

coordinate	**index A of T16 in the author's notebook**	**sign η**
x_0 (hidden direction)	0	$+1$
x_1 (3-space)	1	$+1$
x_2 (3-space)	2	$+1$
x_3 (3-space)	3	$+1$
x_4 (the time)	4	-1
x_5 (extra time, deflating)	5	-1
x_6 (extra time, deflating)	6	-1
x_7 (extra time, deflating)	7	-1

So $\gamma^{(x_k)}=\mathrm{T16}[k]$ for $k=0,\ldots,7$: the index A of the author's notebook is the index of his coordinate, and the sign of each coordinate is the entry of eta4488 at that index. Status: ASSUMED (the metric and the coordinate roles are the author's input; the map is the one the Revision record uses and checks: `Revision/algebra/reports/wolfram-algebra.json`, checks `coordinate_map` and `eta_in_author_order`).

4.5 The author's construction of T16, step by step

The author builds his matrices in his Mathematica notebook, whose file name is

`Pair_Creation_of_Universes_WaveFunctionOfUniverse-4+4-Einstein-Lovelock-Nash.nb`

in four steps. The Revision record re-typed his formulas from the input cells In[45], In[46], In[294], In[300], In[301], In[338], In[351], In[370], In[371] and In[372] (the notebook itself is only read, never changed) and rebuilt the matrices twice, with WolframScript and with Python. This section follows the four steps and proves, by hand, every rule that the computer checks in Notebook 04a.

Step 1: six 4×4 blocks. For $h=1,2,3$ and $p,q=1,2,3,4$ (counted from 1, as in the author's formulas) the author defines two patterns (In[294]):

$$Q_a[h]_{pq} = \text{Signature}[\{h,p,q,4\}], \qquad Q_b[h]_{pq} = \delta_{p4}\delta_{qh} - \delta_{ph}\delta_{q4},$$

and from them the blocks $\text{s4}[h] = Q_a[h] - Q_b[h]$ (In[300]) and $\text{t4}[h] = Q_a[h] + Q_b[h]$ (In[301]). $Q_a[h]_{pq}$ is the permutation sign of the list $(h,p,q,4)$; it is nonzero only when $h,p,q,4$ are four different numbers. $Q_b[h]$ has exactly two nonzero entries: $+1$ in row 4, column h, and -1 in row h, column 4. A worked example for $h = 1$:

- row 1, column 4 of s4[1]: $Q_a = \text{Signature}[\{1,1,4,4\}] = 0$ (repeated numbers) and $Q_b = \delta_{14}\delta_{41} - \delta_{11}\delta_{44} = -1$, so the entry is $0 - (-1) = +1$;
- row 2, column 3: $Q_a = \text{Signature}[\{1,2,3,4\}] = +1$ and $Q_b = 0$, so the entry is $+1$;
- row 3, column 2: $Q_a = \text{Signature}[\{1,3,2,4\}] = -1$ (one inversion) and $Q_b = 0$, so the entry is -1;
- row 4, column 1: $Q_a = 0$ and $Q_b = \delta_{44}\delta_{11} = +1$, so the entry is $0 - 1 = -1$;
- every other entry is 0, because Q_a and Q_b vanish there.

All six blocks, computed in this way (Notebook 04a prints them and compares them with the six matrices that the author's notebook displays in its output Out[304]):

$$\text{s4}[1] = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}, \quad \text{s4}[2] = \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}, \quad \text{s4}[3] = \begin{pmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{pmatrix},$$

$$\text{t4}[1] = \begin{pmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \quad \text{t4}[2] = \begin{pmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \quad \text{t4}[3] = \begin{pmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

The blocks are antisymmetric. Exchanging p and q exchanges two entries of the list $(h,p,q,4)$, which flips its permutation sign (Section 4.2): $Q_a[h]_{qp} = -Q_a[h]_{pq}$. Exchanging p and q in Q_b turns $\delta_{p4}\delta_{qh} - \delta_{ph}\delta_{q4}$ into $\delta_{q4}\delta_{ph} - \delta_{qh}\delta_{p4}$, which is minus the original: $Q_b[h]_{qp} = -Q_b[h]_{pq}$. A sum or difference of antisymmetric matrices is antisymmetric, so s4[h] and t4[h] are antisymmetric.

The blocks are quaternion multiplications. A **quaternion** is $q = q_4 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}$ with four real numbers; we store it as the list (q_1, q_2, q_3, q_4), the real part last, to match the blocks. Quaternions are multiplied with Hamilton's rules $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1$, $\mathbf{ij} = \mathbf{k} = -\mathbf{ji}$, $\mathbf{jk} = \mathbf{i} = -\mathbf{kj}$, $\mathbf{ki} = \mathbf{j} = -\mathbf{ik}$, and their multiplication is associative. For a fixed quaternion u the **right multiplication** R_u maps q to $q\,u$ and the **left multiplication** L_u maps q to $u\,q$. Both are linear maps of the stored lists of four numbers: a product of quaternions is multiplied out term by term, so $(q + q')\,u = q\,u + q'\,u$ and $(cq)\,u = c\,(q\,u)$ for every real number c, and in the same way for L_u. By the rule "linear maps are matrices" of Section 4.2, each is therefore a 4×4 matrix whose column p holds the components of the image of the unit column e_p, where $e_1 = \mathbf{i}$, $e_2 = \mathbf{j}$, $e_3 = \mathbf{k}$, $e_4 = 1$ are stored as the four unit columns $(1,0,0,0)$, $(0,1,0,0)$, $(0,0,1,0)$, $(0,0,0,1)$: column p of R_u holds the components of $e_p u$, column p of L_u those of $u e_p$. Example: the columns of $R_{\mathbf{i}}$ are the components of $\mathbf{ii} = -1$, $\mathbf{ji} = -\mathbf{k}$,

$\mathbf{ki} = \mathbf{j}$ and $1\,\mathbf{i} = \mathbf{i}$, that is $(0,0,0,-1)$, $(0,0,-1,0)$, $(0,1,0,0)$ and $(1,0,0,0)$; these are exactly the four columns of s4[1] above. In the same way (Notebook 04a checks all six):

$$\text{s4}[h] = R_{u_h}, \qquad \text{t4}[h] = -L_{u_h}, \qquad u_1 = \mathbf{i},\ u_2 = \mathbf{j},\ u_3 = \mathbf{k}.$$

This explains every rule of the blocks. Line by line:

1. $R_u R_w = R_{wu}$ — because $R_u(R_w(q)) = (q\,w)\,u = q\,(w\,u)$ (associativity).
2. $L_u L_w = L_{uw}$ — because $L_u(L_w(q)) = u\,(w\,q) = (u\,w)\,q$.
3. $L_u R_w = R_w L_u$ — because $u\,(q\,w) = (u\,q)\,w$ (associativity).
4. $R_{-1} = L_{-1} = -I_4$ — multiplying by the number -1 changes every sign.
5. $\text{s4}[h]^2 = R_{u_h u_h} = R_{-1} = -I_4$ and $\text{t4}[h]^2 = (-1)^2 L_{u_h u_h} = -I_4$ — by lines 1, 2 and 4 and $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1$.
6. For $h \neq k$: $\text{s4}[h]\ \text{s4}[k] = R_{u_k u_h} = -R_{u_h u_k} = -\text{s4}[k]\ \text{s4}[h]$ — by line 1, because two different units anticommute ($u_k u_h = -u_h u_k$) and R is linear; t4 works in the same way with line 2.
7. $\text{s4}[h]\ \text{t4}[k] = -R_{u_h} L_{u_k} = -L_{u_k} R_{u_h} = \text{t4}[k]\ \text{s4}[h]$ — by line 3: every s-block commutes with every t-block.

So the s-blocks anticommute among themselves and square to $-I_4$, the t-blocks do the same, and every s-block commutes with every t-block: $\{\text{s4}[h], \text{s4}[k]\} = \{\text{t4}[h], \text{t4}[k]\} = -2\delta_{hk} I_4$ and $[\text{s4}[h], \text{t4}[k]] = 0$. Two further products are needed below. By lines 1 and 2, $\text{s4}[1]\ \text{s4}[2]\ \text{s4}[3] = R_{\mathbf{i}} R_{\mathbf{j}} R_{\mathbf{k}} = R_{\mathbf{kji}}$, and $\mathbf{kji} = (\mathbf{kj})\mathbf{i} = -\mathbf{ii} = 1$, so $\text{s4}[1]\ \text{s4}[2]\ \text{s4}[3] = I_4$; and $\text{t4}[3]\ \text{t4}[2]\ \text{t4}[1] = (-1)^3 L_{\mathbf{k}} L_{\mathbf{j}} L_{\mathbf{i}} = -L_{\mathbf{kji}} = -I_4$.

The author calls s4 *self-dual* and t4 *anti-self-dual*. The **dual** of an antisymmetric 4×4 matrix M is $(\star M)_{pq} = \frac{1}{2}\sum_{r,s} \epsilon_{pqrs} M_{rs}$, where the **Levi-Civita symbol** ϵ_{pqrs} is the permutation sign of (p,q,r,s). For an antisymmetric M the two surviving terms of each sum are equal; for example $(\star M)_{12} = \frac{1}{2}(\epsilon_{1234} M_{34} + \epsilon_{1243} M_{43}) = \frac{1}{2}(M_{34} + (-1)(-M_{34})) = M_{34}$. In the same way $(\star M)_{13} = -M_{24}$ and $(\star M)_{14} = M_{23}$, so M is self-dual ($\star M = M$) when $M_{12} = M_{34}$, $M_{13} = -M_{24}$ and $M_{14} = M_{23}$, and anti-self-dual ($\star M = -M$) when $M_{12} = -M_{34}$, $M_{13} = M_{24}$ and $M_{14} = -M_{23}$. Read off from the matrices above: s4[1] has $M_{12} = 0 = M_{34}$, $M_{13} = 0 = -M_{24}$, $M_{14} = 1 = M_{23}$ (self-dual), and t4[1] has $M_{14} = -1$, $M_{23} = 1$ (anti-self-dual).

Step 2: the 8×8 matrices. With 4×4 blocks (In[46], In[338], In[351]):

$$\sigma = \begin{pmatrix} 0 & I_4 \\ I_4 & 0 \end{pmatrix}, \quad \tau_0 = I_8, \quad \tau_h = \begin{pmatrix} 0 & \text{s4}[h] \\ \text{s4}[h] & 0 \end{pmatrix}, \quad \tau_{7-h} = \begin{pmatrix} 0 & \text{t4}[h] \\ -\text{t4}[h] & 0 \end{pmatrix} \quad (h = 1,2,3),$$

so τ_6 comes from t4[1], τ_5 from t4[2] and τ_4 from t4[3]; then $\tau_7 = \tau_1\tau_2\tau_3\tau_4\tau_5\tau_6$, $\bar\tau_0 = I_8$ and $\bar\tau_A = \sigma\,\tau_A^T\,\sigma$ for $A = 1, \ldots, 7$ (the author writes tau[A] and taubar[A]). The rules of these matrices follow from the block rule and Step 1, line by line:

1. $\sigma \begin{pmatrix} A & B \\ C & D \end{pmatrix} \sigma = \begin{pmatrix} D & C \\ B & A \end{pmatrix}$ — the block rule, applied twice: σ on the left exchanges the two block rows, on the right the two block columns.

2. $\tau_h^2 = \mathrm{diag}(\mathrm{s4}[h]^2, \mathrm{s4}[h]^2) = -I_8$ and $\tau_{7-h}^2 = \mathrm{diag}(-\mathrm{t4}[h]^2, -\mathrm{t4}[h]^2) = +I_8$ — block rule and Step 1, line 5.

3. $\tau_h\tau_k + \tau_k\tau_h = \mathrm{diag}(\{\mathrm{s4}[h], \mathrm{s4}[k]\}, \{\mathrm{s4}[h], \mathrm{s4}[k]\}) = 0$ for $h \neq k$, and in the same way $\tau_{7-h}\tau_{7-k} + \tau_{7-k}\tau_{7-h} = \mathrm{diag}(-\{\mathrm{t4}[h], \mathrm{t4}[k]\}, -\{\mathrm{t4}[h], \mathrm{t4}[k]\}) = 0$ — block rule and Step 1.

4. $\tau_h\tau_{7-k} + \tau_{7-k}\tau_h = \mathrm{diag}(-\mathrm{s4}[h]\mathrm{t4}[k] + \mathrm{t4}[k]\mathrm{s4}[h],\ \mathrm{s4}[h]\mathrm{t4}[k] - \mathrm{t4}[k]\mathrm{s4}[h]) = 0$ — block rule; the s-blocks commute with the t-blocks.

5. $\tau_1\tau_2\tau_3 = \begin{pmatrix} 0 & \mathrm{s4}[1]\mathrm{s4}[2]\mathrm{s4}[3] \\ \mathrm{s4}[1]\mathrm{s4}[2]\mathrm{s4}[3] & 0 \end{pmatrix} = \sigma$ — block rule twice, and s4[1] s4[2] s4[3] $= I_4$.

6. $\tau_4\tau_5\tau_6 = \begin{pmatrix} 0 & -\mathrm{t4}[3]\mathrm{t4}[2]\mathrm{t4}[1] \\ \mathrm{t4}[3]\mathrm{t4}[2]\mathrm{t4}[1] & 0 \end{pmatrix} = \begin{pmatrix} 0 & I_4 \\ -I_4 & 0 \end{pmatrix}$ — block rule twice, and t4[3] t4[2] t4[1] $= -I_4$.

7. $\tau_7 = (\tau_1\tau_2\tau_3)(\tau_4\tau_5\tau_6) = \begin{pmatrix} 0 & I_4 \\ I_4 & 0 \end{pmatrix} \begin{pmatrix} 0 & I_4 \\ -I_4 & 0 \end{pmatrix} = \begin{pmatrix} -I_4 & 0 \\ 0 & I_4 \end{pmatrix}$ — lines 5 and 6 and the block rule.

8. $\tau_7^2 = I_8$, and τ_7 anticommutes with $\tau_1, \ldots, \tau_6$ — τ_7 is diagonal with the blocks $-I_4$ and I_4; multiplying a block off-diagonal matrix by it from the left flips the sign of the upper-right block, from the right the sign of the lower-left block, so the two products differ by a sign.

9. $\tau_4\tau_5\tau_6\tau_7 = \begin{pmatrix} 0 & I_4 \\ -I_4 & 0 \end{pmatrix} \begin{pmatrix} -I_4 & 0 \\ 0 & I_4 \end{pmatrix} = \sigma$ and $\tau_1\tau_2\cdots\tau_7 = \tau_7\tau_7 = I_8$ — line 6, the block rule, and the definition $\tau_1\cdots\tau_6 = \tau_7$.

10. $\bar\tau_A = -\tau_A$ for $A = 1, \ldots, 7$ — for $A = h$: $\tau_h^T = \begin{pmatrix} 0 & \mathrm{s4}[h]^T \\ \mathrm{s4}[h]^T & 0 \end{pmatrix} = -\tau_h$ (block transpose, s4 antisymmetric), and by line 1 $\sigma\tau_h\sigma = \tau_h$, so $\bar\tau_h = -\tau_h$; for $A = 7 - h$: $\tau_{7-h}^T = \begin{pmatrix} 0 & -\mathrm{t4}[h]^T \\ \mathrm{t4}[h]^T & 0 \end{pmatrix} = \tau_{7-h}$, and by line 1 $\sigma\tau_{7-h}\sigma = \begin{pmatrix} 0 & -\mathrm{t4}[h] \\ \mathrm{t4}[h] & 0 \end{pmatrix} = -\tau_{7-h}$; for $A = 7$: τ_7 is symmetric and by line 1 $\sigma\tau_7\sigma = \mathrm{diag}(I_4, -I_4) = -\tau_7$.

Lines 2, 3, 4 and 8 together say that the seven matrices $\tau_1, \ldots, \tau_7$ satisfy a Clifford relation of their own: $\tau_A\tau_B + \tau_B\tau_A = -2\,\mathrm{eta4488}_{AB}\, I_8$ for $A, B = 1, \ldots, 7$ (τ_1, τ_2, τ_3 square to $-I_8$, $\tau_4, \ldots, \tau_7$ to $+I_8$). With line 10 this gives the **key rule** for all 64 pairs $A, B = 0, \ldots, 7$:

$$\tau_A\bar\tau_B + \tau_B\bar\tau_A = \bar\tau_A\tau_B + \bar\tau_B\tau_A = 2\,\mathrm{eta4488}_{AB}\, I_8.$$

Proof: for $A, B \geq 1$ both sides equal $-(\tau_A\tau_B + \tau_B\tau_A)$ by line 10, which is $2\,\mathrm{eta4488}_{AB} I_8$; for $A = 0$ and $B \geq 1$ the left side is $\bar\tau_B + \tau_B = 0 = 2\,\mathrm{eta4488}_{0B} I_8$; for $A = B = 0$ it is $2I_8$.

Step 3: the 16×16 matrices. The author puts $\bar\tau_A$ into the upper-right and τ_A into the lower-left 8×8 corner (In[371]) and multiplies the first eight (In[372]):

$$\mathrm{T16}[A] = \begin{pmatrix} 0 & \bar\tau_A \\ \tau_A & 0 \end{pmatrix} \quad (A = 0, \dots, 7), \qquad \mathrm{T16}[8] = \mathrm{T16}[0]\,\mathrm{T16}[1] \cdots \mathrm{T16}[7].$$

By the block rule, $\mathrm{T16}[A]\ \mathrm{T16}[B] = \mathrm{diag}(\bar\tau_A\tau_B, \tau_A\bar\tau_B)$. Adding the same with A and B exchanged and using the key rule:

$$\mathrm{T16}[A]\,\mathrm{T16}[B] + \mathrm{T16}[B]\,\mathrm{T16}[A] = \mathrm{diag}(\bar\tau_A\tau_B + \bar\tau_B\tau_A,\ \tau_A\bar\tau_B + \tau_B\bar\tau_A) = 2\,\mathrm{eta4488}_{AB}\, I_{16}.$$

This is the Clifford relation in the author's frame order. The product T16[8] is block diagonal (eight block off-diagonal factors). Its upper-left block is $\bar\tau_0\tau_1\,\bar\tau_2\tau_3\,\bar\tau_4\tau_5\,\bar\tau_6\tau_7 = \tau_1(-\tau_2)\tau_3(-\tau_4)\tau_5(-\tau_6)\tau_7 = -\tau_1\cdots\tau_7 = -I_8$ (pairs of factors multiplied with the block rule, line 10, line 9), and its lower-right block is $\tau_0\bar\tau_1\,\tau_2\bar\tau_3\,\tau_4\bar\tau_5\,\tau_6\bar\tau_7 = (+1)\tau_1\cdots\tau_7 = I_8$ (four minus signs). So $\mathrm{T16}[8] = \mathrm{diag}(-I_8, I_8)$, and $\mathrm{T16}[0]\ \mathrm{T16}[1]\ \cdots\ \mathrm{T16}[8] = \mathrm{T16}[8]^2 = I_{16}$. Chapter 5 calls this product the **chirality**; here it is only a product.

Step 4: the author's coordinates. With the map of Section 4.4, $\gamma^{(x_k)} = \mathrm{T16}[k]$ for $k = 0, \dots, 7$, and the signs are $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$, the same list as eta4488. Naming does not change a matrix, so

$$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I_{16} \qquad (a, b = x_0, \dots, x_7).$$

Three more properties follow. (i) Every γ^a is a **signed permutation matrix**, hence orthogonal: $(\gamma^a)^T = (\gamma^a)^{-1}$. Proof, line by line, with the rules of Section 4.2:

1. s4[h], t4[h], $-$t4[h] and I_4 are signed permutation matrices — read off from the six matrices of Step 1: each has exactly one entry ±1 in every row and every column.
2. σ, $\tau_0 = I_8$ and $\tau_1, \dots, \tau_6$ are signed permutation matrices — each is block diagonal or block off-diagonal with two blocks from line 1.
3. $\tau_7 = \tau_1\tau_2\cdots\tau_6$ is one — a product of signed permutation matrices.
4. $\bar\tau_0 = I_8$ and $\bar\tau_A = \sigma\,\tau_A^T\,\sigma$ are ones — a transpose and products of signed permutation matrices.
5. T16[A] is one — it is block off-diagonal with the two blocks $\bar\tau_A$ and τ_A of lines 2 to 4.

Renaming T16[A] as $\gamma^{(x_a)}$ changes no entry, so every γ^a is a signed permutation matrix, and Section 4.2 showed that such a matrix is orthogonal. (ii) **Reality**: every entry of every γ^a is -1, 0 or $+1$, by (i). (iii) The **symmetry pattern** $(\gamma^a)^T = \eta_{aa}\gamma^a$: symmetric for the space-like x_0, x_1, x_2, x_3, antisymmetric for the time-like $x_4, \dots, x_7$. Proof of (iii), line by line: $(\gamma^a)^2 = \eta_{aa} I_{16}$ (Clifford relation with $b = a$); multiplying by η_{aa} and using $\eta_{aa}^2 = 1$ gives $\gamma^a(\eta_{aa}\gamma^a) = I_{16}$, so $(\gamma^a)^{-1} = \eta_{aa}\gamma^a$; with (i), $(\gamma^a)^T = (\gamma^a)^{-1} = \eta_{aa}\gamma^a$.

Status. PROVED, by hand above, and exactly by the computer in two independent Revision programs. The table names, for each statement of this section, the checks that confirm it in the WolframScript report `Revision/algebra/reports/wolfram-algebra.json` and in the Python report `Revision/algebra/reports/python-algebra.json`.

statement of Section 4.5	check in wolfram-algebra.json	check in python-algebra.json
the six blocks are those the author displays	s4_t4_entries_from_Signature_and_deltas	notebook_blocks_s4_t4
the rules of the blocks	s4_t4_quaternion_algebras, s4_self_dual_t4_anti_self_dual	blocks_antisymmetric_square_minus_one_commute
the rules of σ and τ	sigma8_involution, tau7_and_sigma_identities	tau7_and_product, notebook_sigma_eq_tau1tau2tau3, taubar_relations, notebook_tau_clifford
the key rule	tau_taubar_Clifford_relation	notebook_tau_taubar_clifford
the block form of T16 and T16[8] = diag($-I_8, I_8$)	T16_block_form, Gamma_diag, notebook_product_identity	chirality_diag
the coordinate map and the signs η	coordinate_map, eta_in_author_order	coordinate_map
the Clifford relation for all 64 pairs	Clifford_relation	clifford_relation, clifford_relation_sympy
reality and signed permutations	reality, signed_permutation_matrices	reality_signed_permutations
the symmetry pattern	symmetry_pattern	symmetry_pattern
the stored gammas equal the rebuilt ones	fixture_round_trip	fixture_comparison_gammas_json

Notebook 04a reproduces every one of these 30 Revision checks (16 of the WolframScript report and 14 of the Python report) and compares all $8 \times 256 = 2048$ entries of the gammas with both Revision files: none differs.

The next three sections hold Notebook 04a: how to run it, its complete text, and the line-by-line walk-through.

4.6 How to run Notebook 04a

Step 1. What this notebook does and what it needs.

Notebook 04a (The author's gamma matrices T16 built from his formulas) is the file `Revision/textbook/notebooks/04a_t16_from_formulas.ipynb` of the repository Dirac_claude. It rebuilds, with exact integer arithmetic, the author's eight real 16 by 16 gamma matrices T16 from his own formulas (the 4 by 4 blocks s4 and t4, the 8 by 8 matrices sigma, tau and taubar), names them after the author's coordinates x0 to x7, checks the Clifford relation for all 64 pairs and every other property that the Revision record checks, compares every entry with the Revision record files, and draws six heat maps. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04a_t16_from_formulas.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04a_t16_from_formulas.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 04a_t16_from_formulas.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/04a.captions.json`
- `Revision/textbook/figures/04a_1_blocks_s4_t4.png`
- `Revision/textbook/figures/04a_2_tau_matrices.png`
- `Revision/textbook/figures/04a_3_block_form_of_t16.png`
- `Revision/textbook/figures/04a_4_gammas_x0_to_x7.png`
- `Revision/textbook/figures/04a_5_anticommutator_table.png`
- `Revision/textbook/figures/04a_6_symmetry_pattern.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 28 CHECKS PASSED (notebook 04a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/04a_t16_from_formulas.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.

- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 04a_t16_from_formulas.ipynb
python -m nbconvert --execute --inplace 04a_t16_from_formulas.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `Revision/algebra/gammas.json` or for a file in `Revision/algebra/reports`: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run `git status` in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

```
git checkout HEAD Revision/algebra
```

4.7 Notebook 04a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/04a_t16_from_formulas.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field equations of the theory contain eight real $16 \times 16$ matrices, the
```

gamma matrices $\gamma^{(x_0)}, \gamma^{(x_1)}, \dots, \gamma^{(x_7)}$, one for each direction $x_0, \dots, x_7$ of the author's 4+4 dimensional space-time. The author built them in his Mathematica notebook
'Pair_Creation_of_Universes_WaveFunctionOfUniverse-4+4-Einstein-Lovelock-Nash.nb'
from a few small building blocks, and he calls the finished 16×16 matrices T16. This notebook repeats his construction step by step, with exact integer arithmetic, and checks every property that the Revision record checks. It

- builds the six 4×4 blocks s4[h] and t4[h] (h = 1, 2, 3) from the author's formulas and compares them with the six matrices that his notebook displays;
- shows that these blocks are the multiplications of quaternions (a number system with four components), which explains their rules;
- builds the 8×8 matrices sigma, tau[A] and taubar[A] (A = 0, ..., 7) and checks their rules;
- assembles the 16×16 matrices T16[A] (A = 0, ..., 8);
- names them after the author's coordinates: $\gamma^{(x_k)}$ = T16[k] for k = 0, ..., 7;
- checks the Clifford relation $\gamma^a \gamma^b + \gamma^b \gamma^a = 2 \eta^{ab} I_{16}$ for all 64 pairs (a, b), and that every gamma matrix is real, a signed permutation matrix, and symmetric (space-like directions) or antisymmetric (time-like directions);
- compares every entry with the two Revision record files that hold the gammas;
- draws six figures (heat maps).

Every check prints a line that starts with PASS. When the Revision record made the same check, the lines below the PASS line name the record file and its check.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 04a (The author's gamma matrices T16 built from his formulas) is the file
'Revision/textbook/notebooks/04a_t16_from_formulas.ipynb' of the repository Dirac_claude.
It rebuilds, with exact integer arithmetic, the author's eight real 16 by 16 gamma
matrices T16 from his own formulas (the 4 by 4 blocks s4 and t4, the 8 by 8 matrices
sigma, tau and taubar), names them after the author's coordinates x0 to x7, checks the
Clifford relation for all 64 pairs and every other property that the Revision record
checks, compares every entry with the Revision record files, and draws six heat maps. It
needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
```

exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about

400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04a_t16_from_formulas.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04a_t16_from_formulas.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 04a_t16_from_formulas.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/04a.captions.json‘
- ‘Revision/textbook/figures/04a_1_blocks_s4_t4.png‘
- ‘Revision/textbook/figures/04a_2_tau_matrices.png‘
- ‘Revision/textbook/figures/04a_3_block_form_of_t16.png‘
- ‘Revision/textbook/figures/04a_4_gammas_x0_to_x7.png‘
- ‘Revision/textbook/figures/04a_5_anticommutator_table.png‘
- ‘Revision/textbook/figures/04a_6_symmetry_pattern.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 28 CHECKS PASSED (notebook 04a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/04a_t16_from_formulas.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 04a_t16_from_formulas.ipynb
  python -m nbconvert --execute --inplace 04a_t16_from_formulas.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.

- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘Revision/algebra/gammas.json‘ or for a file in ‘Revision/algebra/reports‘: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run ‘git status‘ in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

```
git checkout HEAD Revision/algebra
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 04a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 04a (The author's gamma matrices T16 built from his formulas) is the file
# Revision/textbook/notebooks/04a_t16_from_formulas.ipynb of the repository Dirac_claude.
# It rebuilds, with exact integer arithmetic, the author's eight real 16 by 16 gamma
# matrices T16 from his own formulas (the 4 by 4 blocks s4 and t4, the 8 by 8 matrices
# sigma, tau and taubar), names them after the author's coordinates x0 to x7, checks the
# Clifford relation for all 64 pairs and every other property that the Revision record
# checks, compares every entry with the Revision record files, and draws six heat maps.
# It needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
```

```
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
```

```
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 04a_t16_from_formulas.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
```

```
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 04a_t16_from_formulas.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 04a_t16_from_formulas.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/04a.captions.json
# - Revision/textbook/figures/04a_1_blocks_s4_t4.png
# - Revision/textbook/figures/04a_2_tau_matrices.png
# - Revision/textbook/figures/04a_3_block_form_of_t16.png
# - Revision/textbook/figures/04a_4_gammas_x0_to_x7.png
# - Revision/textbook/figures/04a_5_anticommutator_table.png
# - Revision/textbook/figures/04a_6_symmetry_pattern.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 28 CHECKS PASSED (notebook 04a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
```

```
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/04a_t16_from_formulas.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 04a_t16_from_formulas.ipynb
#     python -m nbconvert --execute --inplace 04a_t16_from_formulas.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json or for a file in
#   Revision/algebra/reports: the notebook reads these Revision record files of the
#   repository; they are part of every complete clone. Run git status in the repository
#   folder: if it reports them as deleted, restore them with the command below and run
#   the notebook again.
#     git checkout HEAD Revision/algebra
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "04a"  # this notebook: chapter 04, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
```

```
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 04a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Matrix**: a rectangular table of numbers. An $n \times n$ matrix has $n$ rows
  and $n$ columns; $M_{ij}$ is the *entry* in row $i$ and column $j$. numpy counts
  rows and columns from 0; the author's Mathematica notebook counts them from 1. This
  notebook counts from 0, except in the $4 \times 4$ blocks, whose formulas use the
  author's indices $p, q = 1, \dots, 4$ and $h = 1, 2, 3$.
- **Product of matrices**: $(AB)_{ij} = \sum_k A_{ik} B_{kj}$ (row $i$ of $A$ times
  column $j$ of $B$, entry by entry, added up). In Python it is written `A @ B`. The
  order matters: in general $AB \neq BA$.
- **Identity matrix** $I_n$: 1 on the diagonal, 0 elsewhere; $I_n M = M I_n = M$.
- **Transpose** $M^T$: rows and columns exchanged, $(M^T)_{ij} = M_{ji}$. $M$ is
  **symmetric** if $M^T = M$ and **antisymmetric** if $M^T = -M$.
- **Block matrix**: a big matrix cut into smaller matrices (blocks). For example
  $\begin{pmatrix} 0 & B \\ D & 0 \end{pmatrix}$ has the zero matrix as its upper-left
  and lower-right blocks.
- **Signed permutation matrix**: a matrix with exactly one nonzero entry, $+1$ or
  $-1$, in every row and every column. Acting on a column $u$ it reorders the
  components and flips some signs. We describe row $i$ by a short *code*: the code
  +5 in row 2 means $(Mu)_2 = +u_5$, and -0 means $-u_0$.
- **Permutation sign** (Mathematica's *Signature*): for a list of different numbers,
  $+1$ if the number of *inversions* (pairs of positions in which the larger number
  stands first) is even, $-1$ if it is odd; for a list with a repeated number, 0.
- **Kronecker delta** $\delta_{ij}$: 1 if $i = j$, otherwise 0.
- **Anticommutator** $\{A, B\} = AB + BA$; **commutator** $[A, B] = AB - BA$. Two
  matrices *anticommute* if $\{A, B\} = 0$ and *commute* if $[A, B] = 0$.
- **Frame metric** $\eta$: the diagonal matrix of signs that says which directions
  are space-like ($+1$) and which are time-like ($-1$); here, in the order
  $x_0, \dots, x_7$, $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$.
  Because $\eta$ is diagonal with entries $\pm 1$, $\eta^{ab} = \eta_{ab}$.
- **Clifford relation**: $\gamma^a \gamma^b + \gamma^b \gamma^a = 2 \eta^{ab} I_{16}$
  for all $a, b$: each gamma matrix squares to $+I_{16}$ (space-like direction) or
  $-I_{16}$ (time-like direction), and two different gamma matrices anticommute.
- **Heat map**: a picture of a matrix in which every entry is a small coloured
  square; in this notebook blue means $-1$, light grey $0$ and red $+1$.
- **Revision record**: the files under the folder `Revision` of the repository,
  computed by the Revision programs. A **report** is such a file that lists checks,
  each with a name and a verdict (PASS or FAIL).
```

Text cell:

4. The physical and mathematical situation

The author's space-time has eight coordinates $x_0, x_1, \dots, x_7$. In his metric (a diagonal 8×8 matrix g that measures lengths and durations, with the two metric functions $a_4(x_4)$ and $b_4(x_4) > 0$)

- x_0 is a hidden space direction, $z = 6 H x_0$ lies between 0 and $\pi/2$, and $g_{00} = \cot^2 z$;
- x_1, x_2, x_3 are ordinary 3-space; lengths along them carry the scale factor $e^{a_4} \sin^{1/6} z$, which grows (inflates) as a_4 grows;
- x_4 is the time, with $g_{44} = -1$;
- x_5, x_6, x_7 are three *extra times* (time-like, like x_4); they carry the scale factor $b_4^{1/2} e^{-a_4} \sin^{1/6} z$, so they **deflate exponentially** while 3-space inflates.

At every point one can choose eight perpendicular unit directions, a *frame*, one along each coordinate. In the frame the metric becomes the matrix of signs $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$: $+1$ for the space-like directions x_0, x_1, x_2, x_3 and -1 for the time-like directions x_4, x_5, x_6, x_7. Four of each kind: the signature is (4,4). The gamma matrices belong to these frame directions. They are constant matrices, the same at every point and at every time. The inflation of 3-space and the deflation of the extra times enter the field equations through the *frame factors* $\sqrt{|g_{\mu\mu}|}$ that convert coordinate directions into frame directions ($\cot z$ for x_0, $e^{a_4}\sin^{1/6} z$ for x_1, x_2, x_3, 1 for x_4 and $b_4^{1/2}e^{-a_4}\sin^{1/6} z$ for x_5, x_6, x_7), never through the gamma matrices.

The gamma matrices must satisfy the Clifford relation $\gamma^a \gamma^b + \gamma^b \gamma^a = 2 \eta^{ab} I_{16}$. The author builds them in four steps (his Mathematica input cells are named in brackets).

1. Two 4×4 patterns for h = 1, 2, 3 and p, q = 1, ..., 4 (In[294]):
   Qa[h]_pq = Signature[{h, p, q, 4}] and
   Qb[h]_pq = $\delta_{p4}\delta_{qh} - \delta_{ph}\delta_{q4}$; from them the blocks s4[h] = Qa[h] - Qb[h] (In[300]) and t4[h] = Qa[h] + Qb[h] (In[301]).
2. The 8×8 matrices (In[46], In[338], In[351]):
   sigma = $\begin{pmatrix} 0 & I_4 \\ I_4 & 0 \end{pmatrix}$;
   tau[0] = I_8;
   tau[h] = $\begin{pmatrix} 0 & s4[h] \\ s4[h] & 0 \end{pmatrix}$ and
   tau[7-h] = $\begin{pmatrix} 0 & t4[h] \\ -t4[h] & 0 \end{pmatrix}$ for h = 1, 2, 3;
   tau[7] = tau[1] tau[2] tau[3] tau[4] tau[5] tau[6];
   taubar[0] = I_8 and taubar[A] = sigma tau[A]^T sigma for A = 1, ..., 7.
3. The 16×16 matrices (In[371], In[372]):
   T16[A] = $\begin{pmatrix} 0 & \mathrm{taubar}[A] \\ \mathrm{tau}[A] & 0 \end{pmatrix}$ for A = 0, ..., 7, and T16[8] = T16[0] T16[1] ... T16[7].
4. The author's Mathematica notebook numbers its frame directions A = 0, ..., 7 in the order of his coordinates: A = 0 is the hidden direction x_0, A = 1, 2, 3 are 3-space, A = 4 is the time and A = 5, 6, 7 are the extra times; its frame metric is eta4488 = diag(1, 1, 1, 1, -1, -1, -1, -1) (In[45]). So $\gamma^{(x_k)}$ = T16[k] for k = 0, ..., 7: the frame index A is the index of the coordinate.

The Revision record re-typed these formulas from the author's notebook (the file

‘Revision/algebra/wolfram/RevisionAlgebra.wl‘), built the matrices twice, with WolframScript and with Python, and checked them exactly. This notebook builds them a third time, in a form a student can follow line by line.

Text cell:

```
## 5. Small tools

The next cell imports the packages and defines five small functions: the
permutation sign, the Kronecker delta, the anticommutator, the identity matrix and
the zero matrix. All matrices of this notebook hold whole numbers (numpy's type
‘int64‘); sums and products of whole numbers are computed exactly, without any
rounding, so every check below is exact. The check at the end tests the permutation
sign on four lists whose inversions are counted here by hand: (1, 2, 3, 4) has no
inversion, so its sign is $+1$; (2, 1, 3, 4) has one, the pair (2, 1), sign $-1$;
(2, 3, 1, 4) has two, the pairs (2, 1) and (3, 1), sign $+1$; and (1, 1, 3, 4)
repeats a number, sign 0.
```

In [2]:

```
import itertools  # loops over all pairs (or triples, ...) of indices
from fractions import Fraction  # exact fractions such as 1/2

import numpy as np  # integer matrices; their products are exact (no rounding)
import sympy as sp  # a second, independent engine for exact matrix algebra
from matplotlib.colors import BoundaryNorm, ListedColormap
from matplotlib.patches import Patch, Rectangle


def signature(numbers):
    """Mathematica's Signature: 0 if a number occurs twice; otherwise +1 for an even
    and -1 for an odd number of inversions (pairs of positions i < j with
    numbers[i] > numbers[j])."""
    if len(set(numbers)) != len(numbers):  # a set keeps each number only once
        return 0
    inversions = sum(1 for i, j in itertools.combinations(range(len(numbers)), 2)
                     if numbers[i] > numbers[j])
    return -1 if inversions % 2 else 1  # % 2 is the remainder after division by 2


def delta(i, j):
    """The Kronecker delta: 1 if i == j, otherwise 0."""
    return 1 if i == j else 0


def anticommutator(a, b):
    """{a, b} = a b + b a; the operator @ multiplies matrices."""
    return a @ b + b @ a


def identity(n):
    """The n x n identity matrix with whole-number entries."""
```

```
    return np.eye(n, dtype=np.int64)


def zeros(n):
    """The n x n zero matrix with whole-number entries."""
    return np.zeros((n, n), dtype=np.int64)


for numbers in ([1, 2, 3, 4], [2, 1, 3, 4], [2, 3, 1, 4], [1, 1, 3, 4]):
    say(f"the sign of {numbers} is {signature(numbers)}")
check(signature([1, 2, 3, 4]) == 1 and signature([2, 1, 3, 4]) == -1
      and signature([2, 3, 1, 4]) == 1 and signature([1, 1, 3, 4]) == 0,
      "the permutation sign agrees with the four examples worked out by hand")
```

Out [2]:

```
the sign of [1, 2, 3, 4] is 1
the sign of [2, 1, 3, 4] is -1
the sign of [2, 3, 1, 4] is 1
the sign of [1, 1, 3, 4] is 0
PASS the permutation sign agrees with the four examples worked out by hand
```

Text cell:

```
The next cell reads the two Revision reports of the algebra: the WolframScript
report (45 checks) and the Python report (35 checks). It keeps, for every check, its
verdict and its detail text. It then defines the function ‘check_record‘. It works
like ‘check‘, but it first makes sure that each named Revision check is recorded as
passed, and after the PASS line it prints, for each Revision check, the report file
(line "reproduces ...") and the check name (line "check ..."). So whenever this
notebook verifies a statement that the Revision record also verified, the output
says which record it reproduces.
```

In [3]:

```
WOLFRAM = "Revision/algebra/reports/wolfram-algebra.json"  # WolframScript report
PYTHON = "Revision/algebra/reports/python-algebra.json"  # Python report
RECORDED = {}  # report file -> {check name: the check (name, verdict, detail)}
for report_file in (WOLFRAM, PYTHON):
    text = repository_file(report_file).read_text(encoding="utf-8")
    RECORDED[report_file] = {entry["name"]: entry
                             for entry in json.loads(text)["checks"]}
    say(f"{report_file}: {len(RECORDED[report_file])} checks recorded")
REPRODUCED = set()  # the (report, check) pairs that this notebook reproduces


def check_record(condition, name, *records):
    """check(condition, name) for a statement that the Revision record verified too.
    records: pairs (report file, check name); each must be recorded as passed."""
    for report_file, check_name in records:
        entry = RECORDED[report_file].get(check_name)
```

```
        if entry is None or entry["verdict"].upper() != "PASS":
            raise AssertionError(f"{report_file} has no passed check {check_name}")
    check(condition, name)
    for report_file, check_name in records:
        print(f"     reproduces {report_file}")
        print(f"         check {check_name}")
        REPRODUCED.add((report_file, check_name))
```

Out [3]:

```
Revision/algebra/reports/wolfram-algebra.json: 45 checks recorded
Revision/algebra/reports/python-algebra.json: 35 checks recorded
```

Text cell:

```
The next cell defines the drawing tools of the figures. A matrix whose entries are
$-1$, $0$ and $+1$ is drawn as a heat map with three colours: blue for $-1$, light
grey for $0$ and red for $+1$. ‘draw_signs‘ draws one matrix into one panel (one pair
of axes) of a figure; with ‘numbers=True‘ it also writes every nonzero entry into its
square, and ‘first‘ is the number of the first row and column on the axes (1 for the
author’s 1-based $4 \times 4$ blocks, 0 otherwise). ‘sign_legend‘ adds the key of
the three colours below a figure.
```

In [4]:

```
# Three colours: entry -1 blue, entry 0 light grey, entry +1 red.
SIGN_COLOURS = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])
# The boundaries -1.5, -0.5, 0.5, 1.5 put -1, 0 and +1 into the three colours.
SIGN_NORM = BoundaryNorm([-1.5, -0.5, 0.5, 1.5], SIGN_COLOURS.N)


def draw_signs(ax, matrix, title, numbers=False, first=0):
    """Draw a matrix with entries -1, 0, +1 as a heat map in the panel ax."""
    size = matrix.shape[0]
    ax.imshow(matrix, cmap=SIGN_COLOURS, norm=SIGN_NORM)
    ax.set_title(title, fontsize=10)
    step = 1 if size <= 8 else 4  # label every row (small) or every 4th (large)
    ticks = list(range(0, size, step))
    ax.set_xticks(ticks, [str(t + first) for t in ticks], fontsize=7)
    ax.set_yticks(ticks, [str(t + first) for t in ticks], fontsize=7)
    ax.grid(False)  # no grid lines on top of the coloured squares
    if numbers:
        for (row, column), value in np.ndenumerate(matrix):
            if value:
                ax.text(column, row, f"{value:+d}", ha="center", va="center",
                        color="white", fontsize=8)


def sign_legend(fig):
    """The key of the three colours, below the panels of the figure."""
    handles = [Patch(facecolor=SIGN_COLOURS(k), edgecolor="#898781", label=label)
```

```
            for k, label in enumerate(["entry -1", "entry 0", "entry +1"])]
    fig.legend(handles=handles, loc="outside lower center", ncol=3, fontsize=9,
            frameon=False)
```

Text cell:

```
## 6. Step 1: the six $4 \times 4$ blocks s4[h] and t4[h]

The author's two patterns, with $h = 1, 2, 3$ and $p, q = 1, 2, 3, 4$:

$$Q_a[h]_{pq} = \mathrm{Signature}[\{h, p, q, 4\}], \qquad
Q_b[h]_{pq} = \delta_{p4}\,\delta_{qh} - \delta_{ph}\,\delta_{q4}.$$

$Q_a[h]_{pq}$ is $\pm 1$ only when $h, p, q, 4$ are four different numbers, so it is
nonzero only in the rows and columns different from $h$ and 4. $Q_b[h]$ has exactly
two nonzero entries: $+1$ in row 4, column $h$ (there $\delta_{p4}\delta_{qh} = 1$)
and $-1$ in row $h$, column 4. The blocks are $s4[h] = Q_a[h] - Q_b[h]$ and
$t4[h] = Q_a[h] + Q_b[h]$.

A worked example by hand, $h = 1$:

- row 1, column 4 of s4[1]: $Q_a = \mathrm{Signature}[\{1, 1, 4, 4\}] = 0$ (repeated
  numbers) and $Q_b = \delta_{14}\delta_{41} - \delta_{11}\delta_{44} = 0 - 1 = -1$,
  so the entry is $0 - (-1) = +1$;
- row 2, column 3: $Q_a = \mathrm{Signature}[\{1, 2, 3, 4\}] = +1$ (no inversion)
  and $Q_b = 0$, so the entry is $+1$;
- row 3, column 2: $Q_a = \mathrm{Signature}[\{1, 3, 2, 4\}] = -1$ (one inversion,
  the pair 3, 2), so the entry is $-1$;
- row 4, column 1: $Q_a = 0$ and $Q_b = \delta_{44}\delta_{11} - 0 = +1$, so the
  entry is $0 - 1 = -1$.

So the first row of s4[1] is (0, 0, 0, 1) and its last row is (-1, 0, 0, 0). The next
cell computes all six blocks with these formulas and prints them next to each other.
```

In [5]:

```
def Qa(h, p, q):
    """The author's Qa[h, p, q] (In[294]); h, p, q count from 1, as in Mathematica."""
    return signature([h, p, q, 4])


def Qb(h, p, q):
    """The author's Qb[h, p, q] (In[294])."""
    return delta(p, 4) * delta(q, h) - delta(p, h) * delta(q, 4)


# s4[h] = Qa - Qb (In[300]) and t4[h] = Qa + Qb (In[301]). Row p, column q of the
# formula (1 to 4) is row p - 1, column q - 1 of the numpy array (0 to 3).
s4 = {h: np.array([[Qa(h, p, q) - Qb(h, p, q) for q in range(1, 5)]
                   for p in range(1, 5)]) for h in (1, 2, 3)}
t4 = {h: np.array([[Qa(h, p, q) + Qb(h, p, q) for q in range(1, 5)]
                   for p in range(1, 5)]) for h in (1, 2, 3)}
```

```
def side_by_side(matrices, labels):
    """Print square matrices next to each other, one printed line per row."""
    # ^11: centred in 11 characters; rstrip() removes the blanks at the end
    print("   ".join(f"{label:^11}" for label in labels).rstrip())
    for row in range(matrices[0].shape[0]):
        print("   ".join(" ".join(f"{x:2d}" for x in m[row]) for m in matrices))


side_by_side([s4[1], s4[2], s4[3]], ["s4[1]", "s4[2]", "s4[3]"])
print()
side_by_side([t4[1], t4[2], t4[3]], ["t4[1]", "t4[2]", "t4[3]"])
```

Out [5]:

```
   s4[1]          s4[2]          s4[3]
 0  0  0  1    0  0 -1  0    0  1  0  0
 0  0  1  0    0  0  0  1   -1  0  0  0
 0 -1  0  0    1  0  0  0    0  0  0  1
-1  0  0  0    0 -1  0  0    0  0 -1  0

   t4[1]          t4[2]          t4[3]
 0  0  0 -1    0  0 -1  0    0  1  0  0
 0  0  1  0    0  0  0 -1   -1  0  0  0
 0 -1  0  0    1  0  0  0    0  0  0 -1
 1  0  0  0    0  1  0  0    0  0  1  0
```

Text cell:

The author's Mathematica notebook displays the six blocks in its output Out[304]. The next cell types them in, row by row (the Revision Python checker 'Revision/algebra/python/check_algebra.py' types in the same six matrices), and checks that the formulas give exactly these matrices. The Revision record made this comparison (Python report) and re-checked every entry against the formulas (WolframScript report).

In [6]:

```
DISPLAYED_S4 = {  # Out[304] of the author's notebook: s4by4[1], s4by4[2], s4by4[3]
    1: [[0, 0, 0, 1], [0, 0, 1, 0], [0, -1, 0, 0], [-1, 0, 0, 0]],
    2: [[0, 0, -1, 0], [0, 0, 0, 1], [1, 0, 0, 0], [0, -1, 0, 0]],
    3: [[0, 1, 0, 0], [-1, 0, 0, 0], [0, 0, 0, 1], [0, 0, -1, 0]],
}
DISPLAYED_T4 = {  # Out[304]: t4by4[1], t4by4[2], t4by4[3]
    1: [[0, 0, 0, -1], [0, 0, 1, 0], [0, -1, 0, 0], [1, 0, 0, 0]],
    2: [[0, 0, -1, 0], [0, 0, 0, -1], [1, 0, 0, 0], [0, 1, 0, 0]],
    3: [[0, 1, 0, 0], [-1, 0, 0, 0], [0, 0, 0, -1], [0, 0, 1, 0]],
}
# .all() is True when the comparison holds for every entry.
same = all((s4[h] == np.array(DISPLAYED_S4[h])).all()
```

```
           and (t4[h] == np.array(DISPLAYED_T4[h])).all() for h in (1, 2, 3))
check_record(same, "the formulas give the six blocks that the author displays",
             (PYTHON, "notebook_blocks_s4_t4"),
             (WOLFRAM, "s4_t4_entries_from_Signature_and_deltas"))
```

Out [6]:

```
PASS the formulas give the six blocks that the author displays
     reproduces Revision/algebra/reports/python-algebra.json
         check notebook_blocks_s4_t4
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check s4_t4_entries_from_Signature_and_deltas
```

Text cell:

```
The next cell draws the six blocks as heat maps, with the value of every nonzero
entry written into its square. Rows and columns are numbered 1 to 4, as in the
author's formulas.
```

In [7]:

```
fig, axes = plt.subplots(2, 3, figsize=(7.4, 5.6), layout="constrained")
for h in (1, 2, 3):
    draw_signs(axes[0, h - 1], s4[h], f"s4[{h}]", numbers=True, first=1)
    draw_signs(axes[1, h - 1], t4[h], f"t4[{h}]", numbers=True, first=1)
sign_legend(fig)
save_figure(fig, "blocks_s4_t4",
            "The six $4 \\times 4$ building blocks of the author: s4 for h = 1, 2, 3 "
            "(upper row) and t4 for h = 1, 2, 3 (lower row), computed from "
            "Signature and Kronecker deltas; rows and columns numbered 1 to 4 as in "
            "his formulas; blue $-1$, grey $0$, red $+1$. Every block has exactly "
            "one nonzero entry in each row and column and is antisymmetric: the "
            "square in row $p$, column $q$ has the opposite colour of the square in "
            "row $q$, column $p$.")
```

Out [7]:

```
Figure 04a.1 saved as Revision/textbook/figures/04a_1_blocks_s4_t4.png
```

Text cell:

```
## 7. The rules of the blocks

The next cell checks four rules, for all h, k = 1, 2, 3:

1. every block is antisymmetric: $s4[h]^T = -s4[h]$ and $t4[h]^T = -t4[h]$;
2. every block squares to minus the identity: $s4[h]^2 = t4[h]^2 = -I_4$;
3. two different s-blocks anticommute, and so do two different t-blocks; together
```

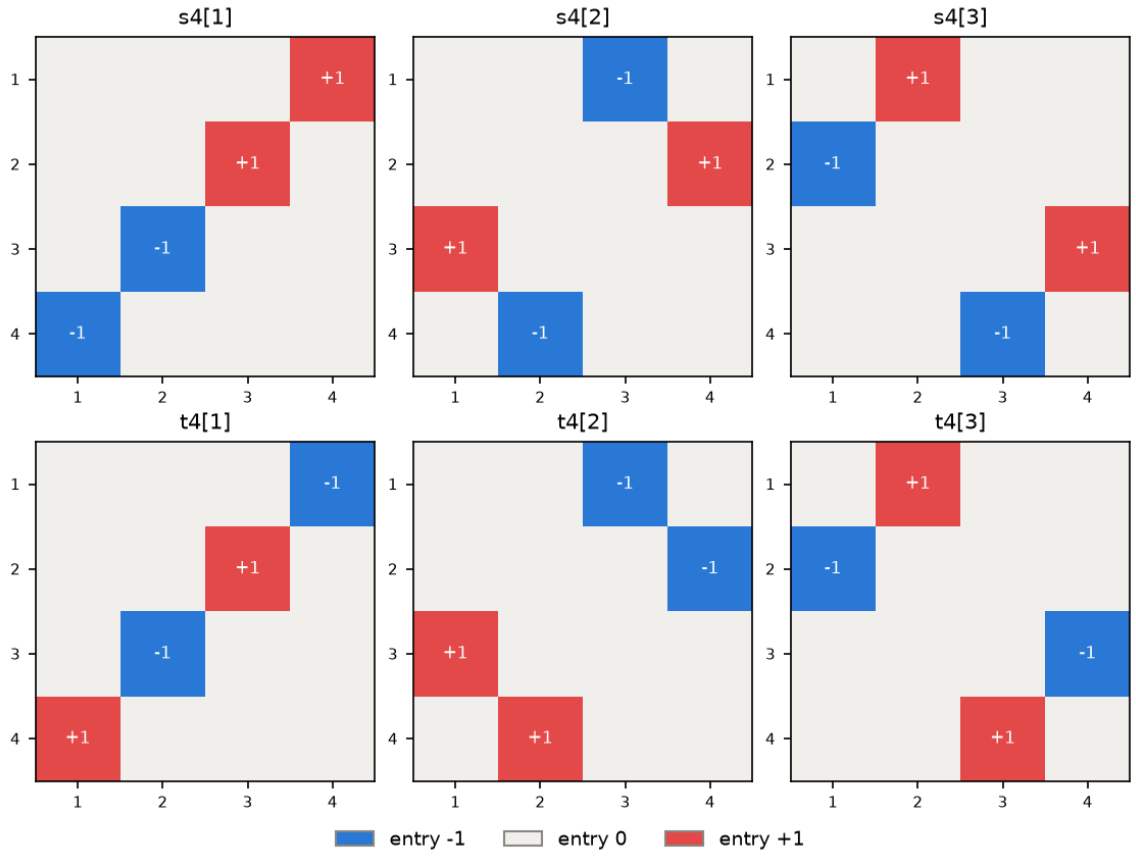

Figure 116: The six 4×4 building blocks of the author: s4 for h = 1, 2, 3 (upper row) and t4 for h = 1, 2, 3 (lower row), computed from Signature and Kronecker deltas; rows and columns numbered 1 to 4 as in his formulas; blue -1, grey 0, red $+1$. Every block has exactly one nonzero entry in each row and column and is antisymmetric: the square in row p, column q has the opposite colour of the square in row q, column p. (Notebook 04a, figure 1.)

```
   with rule 2 this is $\{s4[h], s4[k]\} = \{t4[h], t4[k]\} = -2\delta_{hk} I_4$;
4. every s-block commutes with every t-block: $s4[h]\,t4[k] = t4[k]\,s4[h]$.

Rules 2 and 3 are the rules of the imaginary units of the quaternions,
$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1$ and $\mathbf{ij} = -\mathbf{ji}$
(a later cell explains this). The Revision record checks them in both reports.
```

In [8]:

```
I4 = identity(4)
blocks = [s4[1], s4[2], s4[3], t4[1], t4[2], t4[3]]
antisymmetric = all((m.T == -m).all() for m in blocks)
square_minus_one = all((m @ m == -I4).all() for m in blocks)
anticommute = all((anticommutator(s4[h], s4[k]) == -2 * delta(h, k) * I4).all()
                  and (anticommutator(t4[h], t4[k]) == -2 * delta(h, k) * I4).all()
                  for h in (1, 2, 3) for k in (1, 2, 3))
s_t_commute = all((s4[h] @ t4[k] == t4[k] @ s4[h]).all()
                  for h in (1, 2, 3) for k in (1, 2, 3))
check_record(antisymmetric and square_minus_one and s_t_commute,
             "the blocks are antisymmetric, square to -I4, and s4 commutes with t4",
             (PYTHON, "blocks_antisymmetric_square_minus_one_commute"))
check_record(anticommute and s_t_commute and antisymmetric,
             "{s4[h], s4[k]} = {t4[h], t4[k]} = -2 delta_hk I4, [s4[h], t4[k]] = 0",
             (WOLFRAM, "s4_t4_quaternion_algebras"))
```

Out [8]:

```
PASS the blocks are antisymmetric, square to -I4, and s4 commutes with t4
     reproduces Revision/algebra/reports/python-algebra.json
         check blocks_antisymmetric_square_minus_one_commute
PASS {s4[h], s4[k]} = {t4[h], t4[k]} = -2 delta_hk I4, [s4[h], t4[k]] = 0
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check s4_t4_quaternion_algebras
```

Text cell:

```
**Self-dual and anti-self-dual.** The author calls s4 *self-dual* and t4
*anti-self-dual*. The *dual* of a $4 \times 4$ matrix $M$ is the matrix
$(\star M)_{pq} = \tfrac12 \sum_{r,s} \epsilon_{pqrs} M_{rs}$, where the *Levi-Civita
symbol* $\epsilon_{pqrs}$ is the permutation sign of $(p, q, r, s)$. For an
antisymmetric $M$ only two terms survive in each sum, and they are equal; for example
$(\star M)_{12} = \tfrac12(\epsilon_{1234} M_{34} + \epsilon_{1243} M_{43})
= \tfrac12((+1) M_{34} + (-1)(-M_{34})) = M_{34}$, because exchanging the last two
numbers of (1, 2, 3, 4) flips the sign and $M_{43} = -M_{34}$. In the same way
$(\star M)_{13} = \epsilon_{1324} M_{24} = -M_{24}$ and
$(\star M)_{14} = \epsilon_{1423} M_{23} = M_{23}$. So $M$ is self-dual
($\star M = M$) when $M_{12} = M_{34}$, $M_{13} = -M_{24}$ and $M_{14} = M_{23}$.
For s4[1]: $M_{12} = 0 = M_{34}$, $M_{13} = 0 = -M_{24}$, $M_{14} = 1 = M_{23}$.

The next cell builds $\epsilon_{pqrs}$ for all 256 index lists (counting from 0 now;
the sign of a list does not depend on where the counting starts), computes the dual
```

```
of each block with numpy's `einsum` (which performs exactly the sum over $r$ and
$s$), and checks that every s4 is self-dual and every t4 anti-self-dual
($\star t4[h] = -t4[h]$).
```

In [9]:

```
epsilon = np.zeros((4, 4, 4, 4), dtype=np.int64)
for p, q, r, s in itertools.product(range(4), repeat=4):  # all 4^4 = 256 lists
    epsilon[p, q, r, s] = signature([p, q, r, s])


def dual(m):
    """(1/2) sum_rs epsilon_pqrs m_rs for every p, q (a 4 x 4 matrix)."""
    twice = np.einsum("pqrs,rs->pq", epsilon, m)  # the sum over r and s
    if (twice % 2).any():  # for an antisymmetric m every entry of twice is even
        raise ValueError("the dual is not a whole-number matrix")
    return twice // 2  # exact division by 2


for h in (1, 2, 3):
    say(f"h = {h}: dual(s4) == s4: {(dual(s4[h]) == s4[h]).all()},  "
        f"dual(t4) == -t4: {(dual(t4[h]) == -t4[h]).all()}")
check_record(all((dual(s4[h]) == s4[h]).all() and (dual(t4[h]) == -t4[h]).all()
                 for h in (1, 2, 3)),
             "every s4[h] is self-dual and every t4[h] anti-self-dual",
             (WOLFRAM, "s4_self_dual_t4_anti_self_dual"))
```

Out [9]:

```
h = 1: dual(s4) == s4: True,  dual(t4) == -t4: True
h = 2: dual(s4) == s4: True,  dual(t4) == -t4: True
h = 3: dual(s4) == s4: True,  dual(t4) == -t4: True
PASS every s4[h] is self-dual and every t4[h] anti-self-dual
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check s4_self_dual_t4_anti_self_dual
```

Text cell:

```
**The blocks are quaternion multiplications.** A *quaternion* is
$q = q_4 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}$ with four real numbers; we
store it as the list $(q_1, q_2, q_3, q_4)$, real part last, to match the author's
blocks. Quaternions are multiplied with Hamilton's rules
$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1$,
$\mathbf{ij} = \mathbf{k} = -\mathbf{ji}$, $\mathbf{jk} = \mathbf{i} = -\mathbf{kj}$,
$\mathbf{ki} = \mathbf{j} = -\mathbf{ik}$; multiplying out $x\,y$ term by term with
these rules gives the four lines of `quaternion_product` below. For a fixed unit $u$
the maps $q \to q\,u$ (multiply on the right) and $q \to u\,q$ (multiply on the left)
are linear, so each is a $4 \times 4$ matrix: column $p$ of the matrix holds the
components of $e_p u$ (or $u e_p$), where $e_p$ is the quaternion with a 1 in
component $p$ and 0 elsewhere.
```

```
The next cell checks: s4[h] is the right multiplication by the unit
$\mathbf{i}, \mathbf{j}, \mathbf{k}$ (h = 1, 2, 3), and t4[h] is minus the left
multiplication by the same unit. This explains the rules above: the square of the
right multiplication by $\mathbf{i}$ is the right multiplication by
$\mathbf{i}^2 = -1$, which is $-I_4$; and multiplying on the left and on the right
can be done in either order, because $(u\,q)\,w = u\,(q\,w)$ for quaternions (their
multiplication is *associative*), which is why every s4 commutes with every t4.
```

In [10]:

```
def quaternion_product(x, y):
    """x y for quaternions stored as (q1, q2, q3, q4) = q4 + q1 i + q2 j + q3 k."""
    x1, x2, x3, x4 = x
    y1, y2, y3, y4 = y
    return (x4 * y1 + x1 * y4 + x2 * y3 - x3 * y2,  # the i component
            x4 * y2 + x2 * y4 + x3 * y1 - x1 * y3,  # the j component
            x4 * y3 + x3 * y4 + x1 * y2 - x2 * y1,  # the k component
            x4 * y4 - x1 * y1 - x2 * y2 - x3 * y3)  # the real component


UNITS = {1: (1, 0, 0, 0), 2: (0, 1, 0, 0), 3: (0, 0, 1, 0)}  # i, j, k
BASIS = [(1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1)]  # e_1, ..., e_4


def right_multiplication(u):
    """The 4 x 4 matrix of q -> q u: column p holds the components of e_p u."""
    return np.array([quaternion_product(e, u) for e in BASIS]).T


def left_multiplication(u):
    """The 4 x 4 matrix of q -> u q: column p holds the components of u e_p."""
    return np.array([quaternion_product(u, e) for e in BASIS]).T


say(f"i j = {quaternion_product(UNITS[1], UNITS[2])} (that is k), "
    f"j i = {quaternion_product(UNITS[2], UNITS[1])} (that is -k)")
check(quaternion_product(UNITS[1], UNITS[2]) == (0, 0, 1, 0)
      and quaternion_product(UNITS[1], UNITS[1]) == (0, 0, 0, -1),
      "the quaternion product follows Hamilton's rules i j = k and i i = -1")
check(all((s4[h] == right_multiplication(UNITS[h])).all()
          and (t4[h] == -left_multiplication(UNITS[h])).all() for h in (1, 2, 3)),
      "s4[h] is right and t4[h] minus left multiplication by i, j, k")
```

Out [10]:

```
i j = (0, 0, 1, 0) (that is k), j i = (0, 0, -1, 0) (that is -k)
PASS the quaternion product follows Hamilton's rules i j = k and i i = -1
PASS s4[h] is right and t4[h] minus left multiplication by i, j, k
```

Text cell:

8. Step 2: sigma and the eight matrices tau[A]

The next cell builds the 8×8 matrices of the author's step 2 exactly as the situation section states them: the frame metric eta4488 of his notebook, the matrix sigma (it exchanges the upper four and the lower four components of a column of eight), tau[0] = I_8, the three tau[h] made from s4[h], the three tau[7-h] made from t4[h] (so tau[6] comes from t4[1], tau[5] from t4[2] and tau[4] from t4[3]), tau[7] as the product of the first six, and taubar[A] = sigma tau[A]^T sigma. 'np.block' glues blocks into a big matrix, row of blocks by row of blocks. Then it prints the code of every row of tau[1], ..., tau[7] (the code +5 in row 2 means $(\tau u)_2 = +u_5$; all tau are signed permutation matrices, which the function 'codes' confirms row by row).

In [11]:

```
I8, Z4, Z8 = identity(8), zeros(4), zeros(8)
eta4488 = np.diag([1, 1, 1, 1, -1, -1, -1, -1])  # In[45]: frame order A = 0..7
sigma = np.block([[Z4, I4], [I4, Z4]])  # In[46]
tau = {0: I8}  # In[338]
for h in (1, 2, 3):
    tau[h] = np.block([[Z4, s4[h]], [s4[h], Z4]])  # tau[1], tau[2], tau[3]
    tau[7 - h] = np.block([[Z4, t4[h]], [-t4[h], Z4]])  # tau[6], tau[5], tau[4]
tau[7] = tau[1] @ tau[2] @ tau[3] @ tau[4] @ tau[5] @ tau[6]  # In[338]
taubar = {0: I8}  # In[351]
for A in range(1, 8):
    taubar[A] = sigma @ tau[A].T @ sigma


def codes(m):
    """The code of every row of a signed permutation matrix m: "+5" in row i means
    (m u)_i = +u_5. Stops with an error if a row has not exactly one nonzero."""
    result = []
    for row in m:
        columns = np.flatnonzero(row)  # the columns of the nonzero entries
        if len(columns) != 1 or abs(row[columns[0]]) != 1:
            raise ValueError("not a signed permutation matrix")
        column = int(columns[0])
        result.append(("+" if row[column] > 0 else "-") + str(column))
    return result


labels = [f"tau[{A}]" for A in range(1, 8)]  # the column titles
print("row " + "".join(f"{label:>8}" for label in labels))
tau_codes = {A: codes(tau[A]) for A in range(1, 8)}
for i in range(8):
    print(f"{i:3d} " + "".join(f"{tau_codes[A][i]:>8}" for A in range(1, 8)))
```

Out [11]:

```
row   tau[1]  tau[2]  tau[3]  tau[4]  tau[5]  tau[6]  tau[7]
  0       +7      -6      +5      +5      -6      -7      -0
  1       +6      +7      -4      -4      -7      +6      -1
```

```
  2       -5      +4      +7      -7      +4      -5      -2
  3       -4      -5      -6      +6      +5      +4      -3
  4       +3      -2      +1      -1      +2      +3      +4
  5       +2      +3      -0      +0      +3      -2      +5
  6       -1      +0      +3      +3      -0      +1      +6
  7       -0      -1      -2      -2      -1      -0      +7
```

Text cell:

```
The next cell draws the eight tau matrices as heat maps (rows and columns numbered
0 to 7).
```

In [12]:

```
fig, axes = plt.subplots(2, 4, figsize=(8.6, 5.0), layout="constrained")
for A in range(8):
    draw_signs(axes[A // 4, A % 4], tau[A], f"tau[{A}]")  # // and %: row, column
sign_legend(fig)
save_figure(fig, "tau_matrices",
            "The author's eight $8 \\times 8$ matrices tau for A = 0 to 7 (rows and "
            "columns numbered 0 to 7; blue $-1$, grey $0$, red $+1$). tau 0 is the "
            "identity; tau 1 to tau 6 are made of two $4 \\times 4$ blocks in the "
            "upper-right and lower-left corners (s4 for A = 1, 2, 3 and t4 for "
            "A = 6, 5, 4); tau 7, the product of the first six, is diagonal with "
            "four entries $-1$ and four entries $+1$.")
```

Out [12]:

```
Figure 04a.2 saved as Revision/textbook/figures/04a_2_tau_matrices.png
```

Text cell:

```
## 9. The rules of sigma and tau

The next cell checks, exactly, everything that the Revision record checks about
these matrices:

- sigma is symmetric, has trace 0 (the sum of its diagonal entries) and
  $\sigma^2 = I_8$;
- tau[7] = diag(-1, -1, -1, -1, 1, 1, 1, 1) and tau[1] tau[2] ... tau[7] = $I_8$;
- sigma = tau[1] tau[2] tau[3] = tau[4] tau[5] tau[6] tau[7], and
  sigma taubar[A] = (sigma tau[A])$^T$ for A = 0, ..., 7;
- taubar[A] = -tau[A] for A = 1, ..., 7;
- tau[A] tau[B] + tau[B] tau[A] = $-2\,$eta4488$_{AB} I_8$ for A, B = 1, ..., 7: the
  seven matrices tau[1], ..., tau[7] satisfy a Clifford relation of their own (three
  square to $-I_8$, four to $+I_8$);
- tau[A] taubar[B] + tau[B] taubar[A] = taubar[A] tau[B] + taubar[B] tau[A] =
  $2\,$eta4488$_{AB} I_8$ for A, B = 0, ..., 7 (64 pairs). This is the key rule:
  the next step turns it into the Clifford relation of the $16 \times 16$ matrices.
```

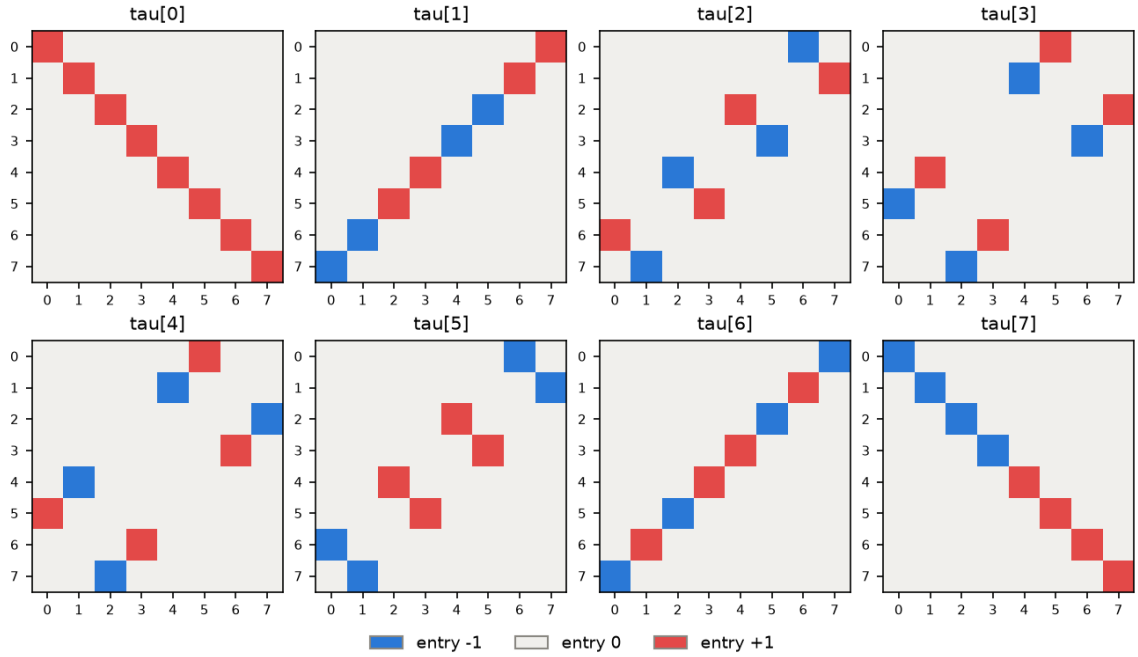

Figure 117: The author's eight 8×8 matrices tau for A = 0 to 7 (rows and columns numbered 0 to 7; blue -1, grey 0, red $+1$). tau 0 is the identity; tau 1 to tau 6 are made of two 4×4 blocks in the upper-right and lower-left corners (s4 for A = 1, 2, 3 and t4 for A = 6, 5, 4); tau 7, the product of the first six, is diagonal with four entries -1 and four entries $+1$. (Notebook 04a, figure 2.)

In [13]:

```
sigma_ok = (sigma.T == sigma).all() and np.trace(sigma) == 0 \
    and (sigma @ sigma == I8).all()
check_record(sigma_ok, "sigma is symmetric, traceless and squares to I8",
             (WOLFRAM, "sigma8_involution"))

product_1_to_7 = I8
for A in range(1, 8):
    product_1_to_7 = product_1_to_7 @ tau[A]
check_record((tau[7] == np.diag([-1, -1, -1, -1, 1, 1, 1, 1])).all()
             and (product_1_to_7 == I8).all(),
             "tau[7] = diag(-I4, I4) and tau[1] tau[2] ... tau[7] = I8",
             (PYTHON, "tau7_and_product"))

sigma_products = (sigma == tau[1] @ tau[2] @ tau[3]).all() and \
    (sigma == tau[4] @ tau[5] @ tau[6] @ tau[7]).all()
sigma_transpose = all((sigma @ taubar[A] == (sigma @ tau[A]).T).all()
                      for A in range(8))
check_record(sigma_products and sigma_transpose,
             "sigma = tau1 tau2 tau3 = tau4 ... tau7 and sigma taubar = (sigma tau)^T",
             (WOLFRAM, "tau7_and_sigma_identities"),
             (PYTHON, "notebook_sigma_eq_tau1tau2tau3"))
check_record(sigma_transpose and all((taubar[A] == -tau[A]).all()
                                     for A in range(1, 8)),
```

```
            "taubar[A] = -tau[A] for A = 1, ..., 7",
            (PYTHON, "taubar_relations"))
check_record(all((anticommutator(tau[A], tau[B]) == -2 * eta4488[A, B] * I8).all()
                 for A in range(1, 8) for B in range(1, 8)),
            "tau[A] tau[B] + tau[B] tau[A] = -2 eta4488_AB I8 for A, B = 1..7",
            (PYTHON, "notebook_tau_clifford"))
```

Out [13]:

```
PASS sigma is symmetric, traceless and squares to I8
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check sigma8_involution
PASS tau[7] = diag(-I4, I4) and tau[1] tau[2] ... tau[7] = I8
     reproduces Revision/algebra/reports/python-algebra.json
         check tau7_and_product
PASS sigma = tau1 tau2 tau3 = tau4 ... tau7 and sigma taubar = (sigma tau)^T
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check tau7_and_sigma_identities
     reproduces Revision/algebra/reports/python-algebra.json
         check notebook_sigma_eq_tau1tau2tau3
PASS taubar[A] = -tau[A] for A = 1, ..., 7
     reproduces Revision/algebra/reports/python-algebra.json
         check taubar_relations
PASS tau[A] tau[B] + tau[B] tau[A] = -2 eta4488_AB I8 for A, B = 1..7
     reproduces Revision/algebra/reports/python-algebra.json
         check notebook_tau_clifford
```

Text cell:

```
The last rule, for all 64 pairs (A, B). The next cell computes, for every pair, the
matrix tau[A] taubar[B] + tau[B] taubar[A], checks that it is a whole number times
$I_8$, prints these numbers as a table (row A, column B), and checks that the table is
twice eta4488. It does the same for the other order, taubar[A] tau[B] +
taubar[B] tau[A].
```

In [14]:

```
table = np.zeros((8, 8), dtype=np.int64)
multiples_of_identity = True
other_order = True
for A, B in itertools.product(range(8), repeat=2):  # all 64 pairs
    m = tau[A] @ taubar[B] + tau[B] @ taubar[A]
    table[A, B] = m[0, 0]  # the number that multiplies I8 (if it is a multiple)
    multiples_of_identity &= bool((m == table[A, B] * I8).all())
    other = taubar[A] @ tau[B] + taubar[B] @ tau[A]
    other_order &= bool((other == table[A, B] * I8).all())
print("A\\B" + "".join(f"{B:4d}" for B in range(8)))
for A in range(8):
    print(f"{A:3d}" + "".join(f"{table[A, B]:4d}" for B in range(8)))
check_record(multiples_of_identity and other_order and (table == 2 * eta4488).all(),
            "tau[A] taubar[B] + tau[B] taubar[A] = 2 eta4488_AB I8, both orders",
```

```
            (WOLFRAM, "tau_taubar_Clifford_relation"),
            (PYTHON, "notebook_tau_taubar_clifford"))
```

Out [14]:

```
A\B   0   1   2   3   4   5   6   7
  0   2   0   0   0   0   0   0   0
  1   0   2   0   0   0   0   0   0
  2   0   0   2   0   0   0   0   0
  3   0   0   0   2   0   0   0   0
  4   0   0   0   0  -2   0   0   0
  5   0   0   0   0   0  -2   0   0
  6   0   0   0   0   0   0  -2   0
  7   0   0   0   0   0   0   0  -2
PASS tau[A] taubar[B] + tau[B] taubar[A] = 2 eta4488_AB I8, both orders
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check tau_taubar_Clifford_relation
     reproduces Revision/algebra/reports/python-algebra.json
         check notebook_tau_taubar_clifford
```

Text cell:

```
## 10. Step 3: the $16 \times 16$ matrices T16[A]

The author puts taubar[A] into the upper-right and tau[A] into the lower-left
$8 \times 8$ corner of a $16 \times 16$ matrix (In[371]), and multiplies the first
eight in order to obtain T16[8] (In[372]). Why this works: by the rule for
multiplying block matrices (blocks multiply like the entries of $2 \times 2$
matrices, keeping their order),

$$\mathrm{T16}[A]\,\mathrm{T16}[B] =
\begin{pmatrix} 0 & \bar\tau_A \\ \tau_A & 0 \end{pmatrix}
\begin{pmatrix} 0 & \bar\tau_B \\ \tau_B & 0 \end{pmatrix} =
\begin{pmatrix} \bar\tau_A \tau_B & 0 \\ 0 & \tau_A \bar\tau_B \end{pmatrix},$$

where $\bar\tau$ stands for taubar. Adding the same with A and B exchanged gives the
two diagonal blocks taubar[A] tau[B] + taubar[B] tau[A] and tau[A] taubar[B] +
tau[B] taubar[A], which the previous cell found to be $2\,$eta4488$_{AB} I_8$ each.
So T16[A] T16[B] + T16[B] T16[A] = $2\,$eta4488$_{AB} I_{16}$: the Clifford relation
in the author's frame order. The next cell builds the nine matrices and checks the
block form, and checks two facts about T16[8] that the Revision record states: it is
the diagonal matrix diag(-1, ..., -1, 1, ..., 1) with eight entries of each sign, and
T16[0] T16[1] ... T16[8] = $I_{16}$ (this is T16[8] T16[8] = $I_{16}$). The meaning of
T16[8] (it is called the *chirality*) belongs to the study of the matrices built from
the gammas; here it is only a product.
```

In [15]:

```
T16 = {A: np.block([[Z8, taubar[A]], [tau[A], Z8]]) for A in range(8)}  # In[371]
T16[8] = T16[0]
for A in range(1, 8):
```

```
    T16[8] = T16[8] @ T16[A]  # In[372]: T16[8] = T16[0] T16[1] ... T16[7]
I16 = identity(16)

# [:8, 8:] means rows 0 to 7 and columns 8 to 15 (the upper-right corner), etc.
block_form = all((T16[A][:8, :8] == 0).all() and (T16[A][8:, 8:] == 0).all()
                 and (T16[A][:8, 8:] == taubar[A]).all()
                 and (T16[A][8:, :8] == tau[A]).all() for A in range(8))
check_record(block_form, "T16[A] = ((0, taubar[A]), (tau[A], 0)) for A = 0, ..., 7",
             (WOLFRAM, "T16_block_form"))
say(f"diagonal of T16[8]: {list(int(x) for x in np.diag(T16[8]))}")
check_record((T16[8] == np.diag([-1] * 8 + [1] * 8)).all(),
             "T16[8] = T16[0] ... T16[7] = diag(-I8, I8)",
             (WOLFRAM, "Gamma_diag"), (PYTHON, "chirality_diag"))
product_0_to_8 = I16
for A in range(9):
    product_0_to_8 = product_0_to_8 @ T16[A]
check_record((product_0_to_8 == I16).all(), "T16[0] T16[1] ... T16[8] = I16",
             (WOLFRAM, "notebook_product_identity"))
```

Out [15]:

```
PASS T16[A] = ((0, taubar[A]), (tau[A], 0)) for A = 0, ..., 7
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check T16_block_form
diagonal of T16[8]: [-1, -1, -1, -1, -1, -1, -1, -1, 1, 1, 1, 1, 1, 1, 1, 1]
PASS T16[8] = T16[0] ... T16[7] = diag(-I8, I8)
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check Gamma_diag
     reproduces Revision/algebra/reports/python-algebra.json
         check chirality_diag
PASS T16[0] T16[1] ... T16[8] = I16
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check notebook_product_identity
```

Text cell:

```
The next cell draws T16[4] (the matrix of the time direction $x_4$) and marks its
four $8 \times 8$ corners: the two zero blocks, taubar[4] (upper right) and tau[4]
(lower left). Rows and columns are numbered 0 to 15.
```

In [16]:

```
fig, ax = plt.subplots(figsize=(5.6, 5.6), layout="constrained")
draw_signs(ax, T16[4], "T16[4], the gamma matrix of the time $x_4$")
for left, top in ((-0.5, -0.5), (7.5, -0.5), (-0.5, 7.5), (7.5, 7.5)):
    ax.add_patch(Rectangle((left, top), 8, 8, fill=False, edgecolor="#0b0b0b",
                           linewidth=1.5))  # the frame of one 8 x 8 corner
labels = {(3.5, 3.5): "0", (11.5, 3.5): "taubar[4]", (3.5, 11.5): "tau[4]",
          (11.5, 11.5): "0"}
for (x, y), label in labels.items():
    ax.text(x, y, label, ha="center", va="center", fontsize=12, color="#0b0b0b",
```

```
            bbox={"facecolor": "white", "alpha": 0.85, "edgecolor": "none"})
sign_legend(fig)
save_figure(fig, "block_form_of_t16",
            "The $16 \\times 16$ matrix T16 4, the gamma matrix of the time "
            "direction $x_4$, with its four $8 \\times 8$ corners framed (rows and "
            "columns numbered 0 to 15; blue $-1$, grey $0$, red $+1$). The "
            "upper-left and lower-right corners are zero; the upper-right corner is "
            "taubar 4 and the lower-left corner is tau 4. Each of the sixteen rows "
            "holds exactly one nonzero entry.")
```

Out [16]:

```
Figure 04a.3 saved as Revision/textbook/figures/04a_3_block_form_of_t16.png
```

Text cell:

```
## 11. Step 4: the author's coordinates x0, ..., x7

The next cell names the matrices after the author's coordinates. The list
'FRAME_INDEX' holds, for $x_0, \dots, x_7$ in this order, the index A of T16[A] in the
author's Mathematica notebook: 0, 1, 2, 3, 4, 5, 6, 7, because his frame directions
come in the order of his coordinates. The frame metric in the order
$x_0, \dots, x_7$ is read off from eta4488 through the same list. The cell prints one
line per coordinate and checks that $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1,
-1)$: space-like $x_0, x_1, x_2, x_3$ and time-like $x_4, x_5, x_6, x_7$, as the
author's metric requires. From here on, 'gamma[0]' is $\gamma^{(x_0)}$, ...,
'gamma[7]' is $\gamma^{(x_7)}$ (a Python list counts from 0, like the author's
coordinates).
```

In [17]:

```
COORDINATES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
FRAME_INDEX = [0, 1, 2, 3, 4, 5, 6, 7]  # A of T16[A] for x0, ..., x7
ROLE = ["hidden direction", "3-space", "3-space", "3-space", "the time",
        "extra time (deflating)", "extra time (deflating)", "extra time (deflating)"]
gamma = [T16[A] for A in FRAME_INDEX]  # gamma[0] = gamma^(x0), ...
eta = np.array([eta4488[A, A] for A in FRAME_INDEX])  # signs in the order x0..x7
for a in range(8):
    kind = "space-like" if eta[a] > 0 else "time-like"
    print(f"gamma^({COORDINATES[a]}) = T16[{FRAME_INDEX[a]}]   eta = {eta[a]:+d}   "
          f"{kind:10}   {ROLE[a]}")
renamed = all(gamma[k] is T16[k] for k in range(8))
check_record(list(eta) == [1, 1, 1, 1, -1, -1, -1, -1] and renamed,
             "gamma^(xk) = T16[k] for k = 0..7, eta = diag(+,+,+,+,-,-,-,-)",
             (WOLFRAM, "coordinate_map"), (WOLFRAM, "eta_in_author_order"),
             (PYTHON, "coordinate_map"))
```

Out [17]:

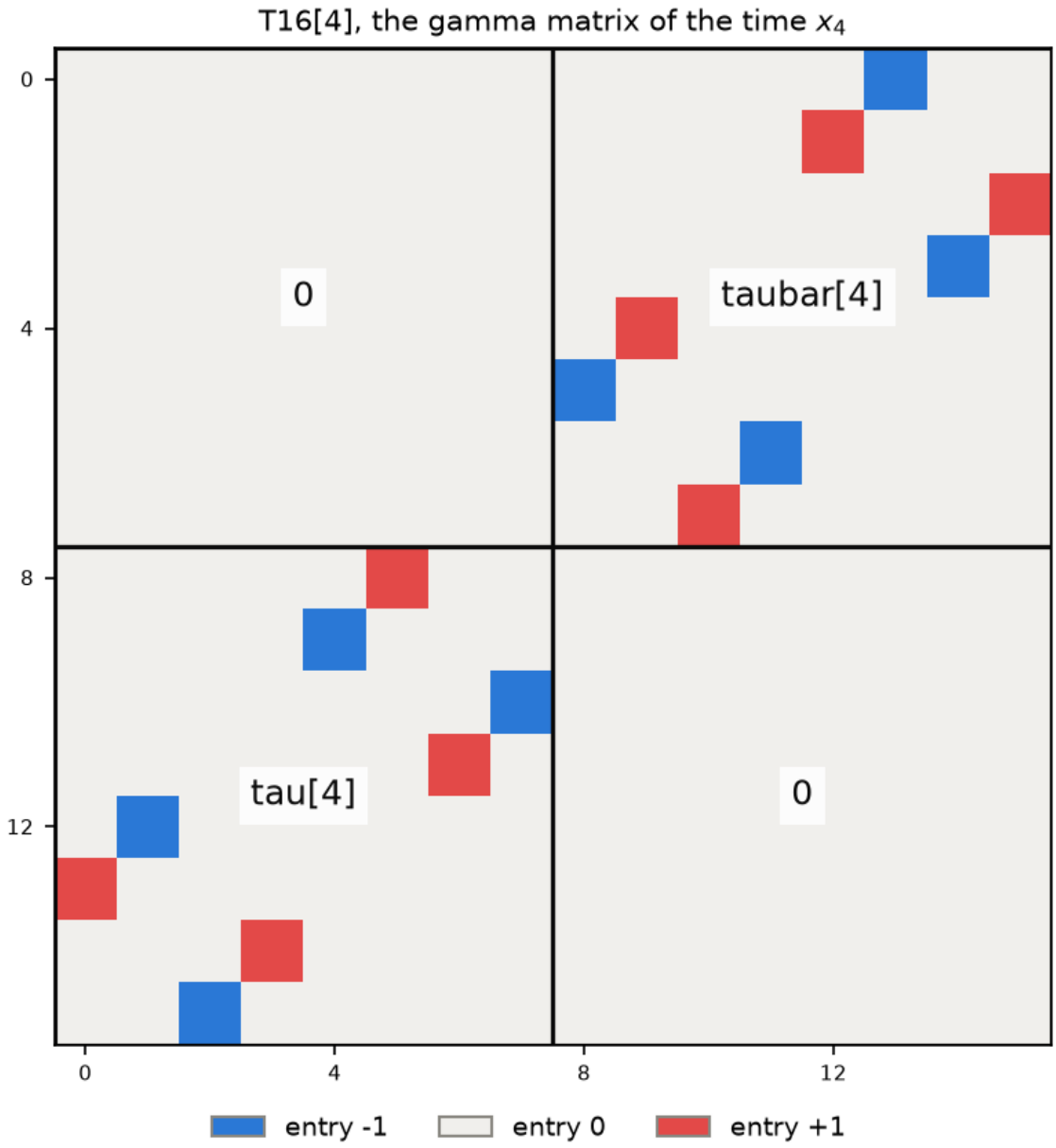

Figure 118: The 16×16 matrix T16 4, the gamma matrix of the time direction x_4, with its four 8×8 corners framed (rows and columns numbered 0 to 15; blue -1, grey 0, red $+1$). The upper-left and lower-right corners are zero; the upper-right corner is taubar 4 and the lower-left corner is tau 4. Each of the sixteen rows holds exactly one nonzero entry. (Notebook 04a, figure 3.)

```
gamma^(x0) = T16[0]   eta = +1   space-like   hidden direction
gamma^(x1) = T16[1]   eta = +1   space-like   3-space
gamma^(x2) = T16[2]   eta = +1   space-like   3-space
gamma^(x3) = T16[3]   eta = +1   space-like   3-space
gamma^(x4) = T16[4]   eta = -1   time-like    the time
gamma^(x5) = T16[5]   eta = -1   time-like    extra time (deflating)
gamma^(x6) = T16[6]   eta = -1   time-like    extra time (deflating)
gamma^(x7) = T16[7]   eta = -1   time-like    extra time (deflating)
PASS gamma^(xk) = T16[k] for k = 0..7, eta = diag(+,+,+,+,-,-,-,-)
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check coordinate_map
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check eta_in_author_order
     reproduces Revision/algebra/reports/python-algebra.json
         check coordinate_map
```

Text cell:

```
The next cell prints all eight gamma matrices in the code notation: row $i$ of the
table gives, for each coordinate, the column and the sign of the single nonzero
entry in row $i$ of that gamma matrix. For example the first entry of the column
x1 is -15: $(\gamma^{(x_1)} u)_0 = -u_{15}$.
```

In [18]:

```
gamma_codes = [codes(g) for g in gamma]
print("row" + "".join(f"{name:>6}" for name in COORDINATES))
for i in range(16):
    print(f"{i:3d}" + "".join(f"{gamma_codes[a][i]:>6}" for a in range(8)))
```

Out [18]:

```
row    x0    x1    x2    x3    x4    x5    x6    x7
  0    +8   -15   +14   -13   -13   +14   +15    +8
  1    +9   -14   -15   +12   +12   +15   -14    +9
  2   +10   +13   -12   -15   +15   -12   +13   +10
  3   +11   +12   +13   +14   -14   -13   -12   +11
  4   +12   -11   +10    -9    +9   -10   -11   -12
  5   +13   -10   -11    +8    -8   -11   +10   -13
  6   +14    +9    -8   -11   -11    +8    -9   -14
  7   +15    +8    +9   +10   +10    +9    +8   -15
  8    +0    +7    -6    +5    +5    -6    -7    -0
  9    +1    +6    +7    -4    -4    -7    +6    -1
 10    +2    -5    +4    +7    -7    +4    -5    -2
 11    +3    -4    -5    -6    +6    +5    +4    -3
 12    +4    +3    -2    +1    -1    +2    +3    +4
 13    +5    +2    +3    -0    +0    +3    -2    +5
 14    +6    -1    +0    +3    +3    -0    +1    +6
 15    +7    -0    -1    -2    -2    -1    -0    +7
```

Text cell:

The next cell draws the eight gamma matrices as heat maps.

In [19]:

```
fig, axes = plt.subplots(2, 4, figsize=(8.8, 5.0), layout="constrained")
for a in range(8):
    draw_signs(axes[a // 4, a % 4], gamma[a], f"$\\gamma^{{(x_{a})}}$")
sign_legend(fig)
save_figure(fig, "gammas_x0_to_x7",
            "The author's eight real $16 \\times 16$ gamma matrices in the order of "
            "his coordinates: the hidden direction $x_0$ and 3-space $x_1, x_2, x_3$ "
            "(upper row), the time $x_4$ and the deflating extra times "
            "$x_5, x_6, x_7$ (lower row) (rows and columns numbered 0 to 15; blue "
            "$-1$, grey $0$, red $+1$). Every matrix has exactly one nonzero entry in "
            "each row and column, all of them in the upper-right and lower-left "
            "$8 \\times 8$ corners; $\\gamma^{(x_0)}$ simply exchanges the upper and "
            "the lower eight components.")
```

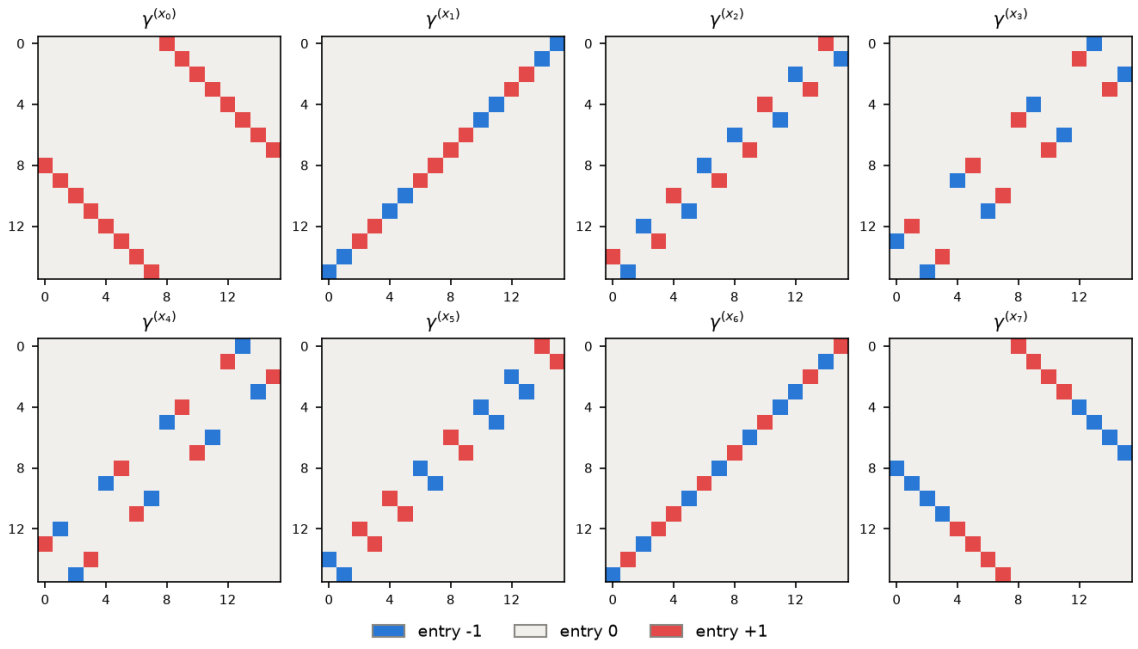

Figure 119: The author's eight real 16×16 gamma matrices in the order of his coordinates: the hidden direction x_0 and 3-space x_1, x_2, x_3 (upper row), the time x_4 and the deflating extra times x_5, x_6, x_7 (lower row) (rows and columns numbered 0 to 15; blue -1, grey 0, red $+1$). Every matrix has exactly one nonzero entry in each row and column, all of them in the upper-right and lower-left 8×8 corners; $\gamma^{(x_0)}$ simply exchanges the upper and the lower eight components. (Notebook 04a, figure 4.)

Out [19]:

```
Figure 04a.4 saved as Revision/textbook/figures/04a_4_gammas_x0_to_x7.png
```

Text cell:

```
## 12. The Clifford relation for all 64 pairs

The next cell computes the anticommutator $\gamma^a\gamma^b + \gamma^b\gamma^a$ for
all 64 ordered pairs $(a, b)$ of coordinates, checks that each one is a whole number
times $I_{16}$, prints these numbers as a table (row $a$, column $b$), and checks that
the table is $2\eta$: 2 on the diagonal for $x_0, x_1, x_2, x_3$, $-2$ on the diagonal
for $x_4, \dots, x_7$, and 0 off the diagonal. It also checks that the squares
$(\gamma^a)^2$, written as the record writes them ("x1:1, x2:1, ..."), appear word
for word in the detail text of the WolframScript check.
```

In [20]:

```
c = np.zeros((8, 8), dtype=np.int64)  # {gamma^a, gamma^b} = c[a, b] I16
all_multiples = True
for a, b in itertools.product(range(8), repeat=2):
    m = anticommutator(gamma[a], gamma[b])
    c[a, b] = m[0, 0]
    all_multiples &= bool((m == c[a, b] * I16).all())
print("a\\b" + "".join(f"{name:>5}" for name in COORDINATES))
for a in range(8):
    print(f"{COORDINATES[a]:>3}" + "".join(f"{c[a, b]:5d}" for b in range(8)))
squares = ", ".join(f"{COORDINATES[a]}:{c[a, a] // 2}" for a in range(8))
say(f"squares (gamma^a)^2 / I16: {squares}")
recorded = RECORDED[WOLFRAM]["Clifford_relation"]["detail"]
check_record(all_multiples and (c == 2 * np.diag(eta)).all() and squares in recorded,
             "{gamma^a, gamma^b} = 2 eta^ab I16 for all 64 pairs a, b",
             (WOLFRAM, "Clifford_relation"), (PYTHON, "clifford_relation"))
```

Out [20]:

```
a\b   x0   x1   x2   x3   x4   x5   x6   x7
 x0    2    0    0    0    0    0    0    0
 x1    0    2    0    0    0    0    0    0
 x2    0    0    2    0    0    0    0    0
 x3    0    0    0    2    0    0    0    0
 x4    0    0    0    0   -2    0    0    0
 x5    0    0    0    0    0   -2    0    0
 x6    0    0    0    0    0    0   -2    0
 x7    0    0    0    0    0    0    0   -2
squares (gamma^a)^2 / I16: x0:1, x1:1, x2:1, x3:1, x4:-1, x5:-1, x6:-1, x7:-1
PASS {gamma^a, gamma^b} = 2 eta^ab I16 for all 64 pairs a, b
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check Clifford_relation
     reproduces Revision/algebra/reports/python-algebra.json
         check clifford_relation
```

Text cell:

```
The next cell repeats the 36 different relations (pairs with $a \le b$; the others
follow because $\{\gamma^a,\gamma^b\} = \{\gamma^b,\gamma^a\}$) with sympy, a second
program that knows nothing of numpy, exactly as the Revision Python checker did with
its second engine.
```

In [21]:

```
sympy_gamma = [sp.Matrix(g.tolist()) for g in gamma]  # the same entries, in sympy
sympy_ok = all(
    sympy_gamma[a] * sympy_gamma[b] + sympy_gamma[b] * sympy_gamma[a]
    == 2 * int(eta[a] * delta(a, b)) * sp.eye(16)
    for a in range(8) for b in range(a, 8))
check_record(sympy_ok, "sympy confirms the 36 relations {gamma^a, gamma^b} = 2 eta^ab",
             (PYTHON, "clifford_relation_sympy"))
```

Out [21]:

```
PASS sympy confirms the 36 relations {gamma^a, gamma^b} = 2 eta^ab
     reproduces Revision/algebra/reports/python-algebra.json
         check clifford_relation_sympy
```

Text cell:

```
**One relation followed by hand.** With the codes one can check a row of a
relation without a computer. The next cell does it the way a student would with
pencil and paper, for row 0, and prints every step:

- $(\gamma^{(x_4)})^2 = -I_{16}$: row 0 of $\gamma^{(x_4)}$ says
  $(\gamma^{(x_4)} v)_0 = s_1 v_{c_1}$; with $v = \gamma^{(x_4)} u$, row $c_1$ says
  $v_{c_1} = s_2 u_{c_2}$; so $(\gamma^{(x_4)}\gamma^{(x_4)} u)_0 = s_1 s_2 u_{c_2}$,
  which must be $-u_0$;
- $\gamma^{(x_0)}\gamma^{(x_4)} = -\gamma^{(x_4)}\gamma^{(x_0)}$: the two orders
  must give row 0 with the same column and opposite signs.
```

In [22]:

```
def follow(m, row):
    """(m u)_row = sign * u_column for a signed permutation m: return (sign, column)."""
    column = int(np.flatnonzero(m[row])[0])
    return int(m[row, column]), column


def signed(sign, letter, index):
    """The text "+u_5" or "-u_5" for sign = +1 or -1, letter = "u", index = 5."""
    return ("+" if sign > 0 else "-") + letter + "_" + str(index)


g4, g0 = gamma[4], gamma[0]  # gamma^(x4) and gamma^(x0)
s1, c1 = follow(g4, 0)  # row 0 of gamma^(x4)
```

```
s2, c2 = follow(g4, c1)  # row c1 of gamma^(x4)
say(f"row 0 of gamma^(x4):  (gamma^(x4) v)_0 = {signed(s1, "v", c1)}")
say(f"row {c1} of gamma^(x4): (gamma^(x4) u)_{c1} = {signed(s2, "u", c2)}")
say(f"with v = gamma^(x4) u: (gamma^(x4) gamma^(x4) u)_0 = {signed(s1 * s2, "u", c2)}")
t1, d1 = follow(g0, 0)  # row 0 of gamma^(x0) gamma^(x4): first gamma^(x0) ...
t2, d2 = follow(g4, d1)  # ... then row d1 of gamma^(x4)
r1, e1 = follow(g4, 0)  # row 0 of gamma^(x4) gamma^(x0): first gamma^(x4) ...
r2, e2 = follow(g0, e1)  # ... then row e1 of gamma^(x0)
say(f"(gamma^(x0) gamma^(x4) u)_0 = {signed(t1 * t2, "u", d2)};  "
    f"(gamma^(x4) gamma^(x0) u)_0 = {signed(r1 * r2, "u", e2)}")
check(s1 * s2 == -1 and c2 == 0, "by hand: row 0 of (gamma^(x4))^2 is -u_0")
check(d2 == e2 and t1 * t2 == -(r1 * r2),
      "by hand: row 0 of gamma^(x0) gamma^(x4) is minus that of gamma^(x4) gamma^(x0)")
```

Out [22]:

```
row 0 of gamma^(x4):  (gamma^(x4) v)_0 = -v_13
row 13 of gamma^(x4): (gamma^(x4) u)_13 = +u_0
with v = gamma^(x4) u: (gamma^(x4) gamma^(x4) u)_0 = -u_0
(gamma^(x0) gamma^(x4) u)_0 = +u_5;  (gamma^(x4) gamma^(x0) u)_0 = -u_5
PASS by hand: row 0 of (gamma^(x4))^2 is -u_0
PASS by hand: row 0 of gamma^(x0) gamma^(x4) is minus that of gamma^(x4) gamma^(x0)
```

Text cell:

```
The next cell draws the table of the numbers $c_{ab}$ in
$\{\gamma^a,\gamma^b\} = c_{ab} I_{16}$ as a heat map, with each number written into
its square.
```

In [23]:

```
fig, ax = plt.subplots(figsize=(5.4, 4.9), layout="constrained")
colours = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -2, 0, +2
ax.imshow(c, cmap=colours, norm=BoundaryNorm([-3, -1, 1, 3], 3))
for (a, b), value in np.ndenumerate(c):
    ax.text(b, a, f"{value:+d}" if value else "0", ha="center", va="center",
            color="white" if value else "#52514e", fontsize=10)
labels = [f"$x_{k}$" for k in range(8)]
ax.set_xticks(range(8), labels)
ax.set_yticks(range(8), labels)
ax.set_xlabel("direction $b$")
ax.set_ylabel("direction $a$")
ax.set_title("$\\gamma^a\\gamma^b + \\gamma^b\\gamma^a = c_{ab}\\, I_{16}$")
ax.grid(False)
save_figure(fig, "anticommutator_table",
            "The Clifford relation of the author's gammas: the number $c_{ab}$ in "
            "$\\gamma^a\\gamma^b + \\gamma^b\\gamma^a = c_{ab} I_{16}$ for all 64 "
            "pairs of directions, row $a$ and column $b$ running over $x_0$ to "
            "$x_7$ (red $+2$, blue $-2$, grey $0$). The table is $2\\eta$: $+2$ on "
            "the diagonal for the space-like directions $x_0, x_1, x_2, x_3$, $-2$ "
            "for the time-like directions $x_4$ to $x_7$, and $0$ off the diagonal, "
            "because two different gamma matrices anticommute.")
```

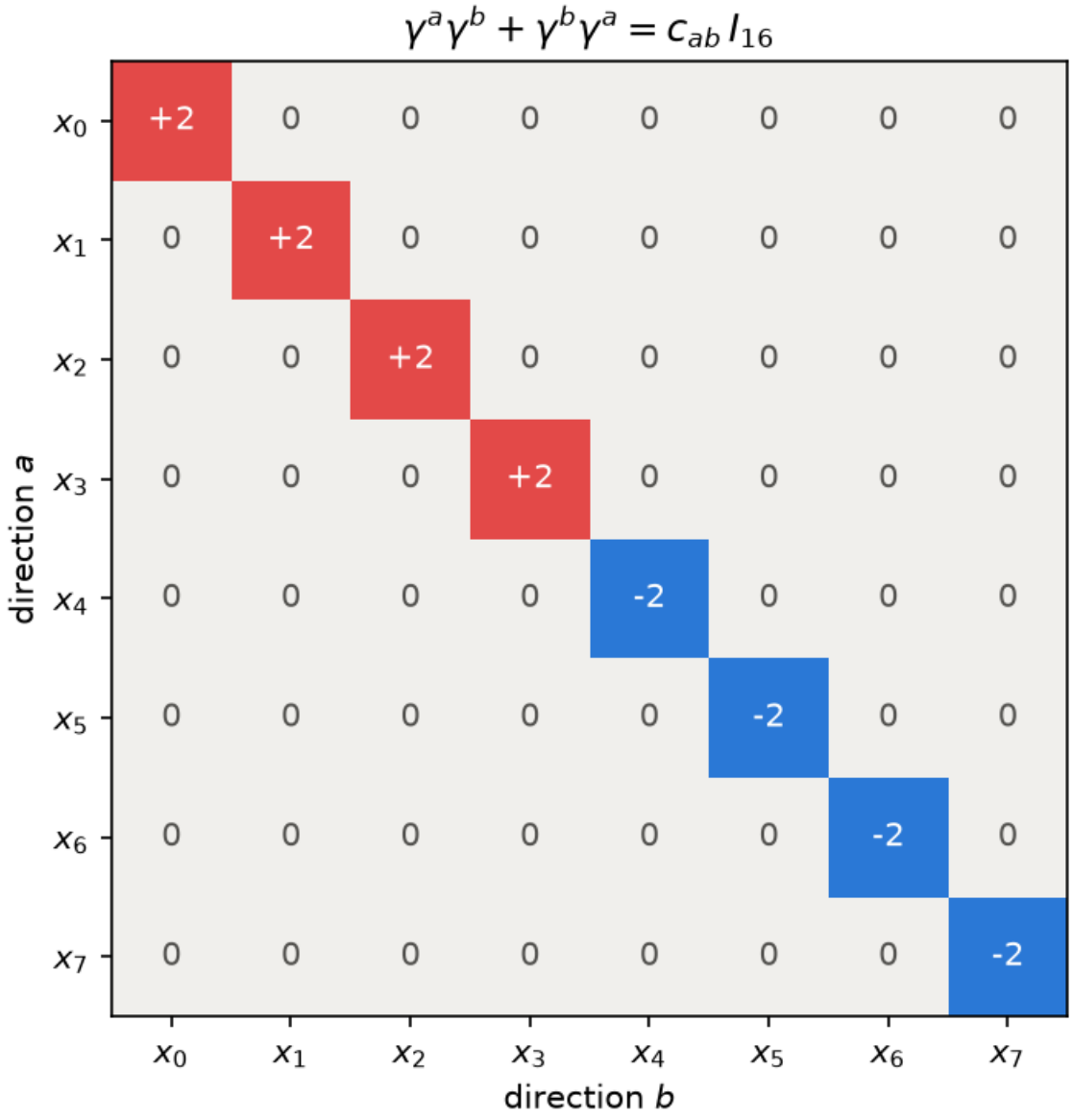

Figure 120: The Clifford relation of the author's gammas: the number c_{ab} in $\gamma^a\gamma^b + \gamma^b\gamma^a = c_{ab}I_{16}$ for all 64 pairs of directions, row a and column b running over x_0 to x_7 (red $+2$, blue -2, grey 0). The table is 2η: $+2$ on the diagonal for the space-like directions x_0, x_1, x_2, x_3, -2 for the time-like directions x_4 to x_7, and 0 off the diagonal, because two different gamma matrices anticommute. (Notebook 04a, figure 5.)

Out [23]:

```
Figure 04a.5 saved as Revision/textbook/figures/04a_5_anticommutator_table.png
```

Text cell:

```
## 13. Reality, signed permutations, symmetry

The next cell checks three more properties that the Revision record checks:

1. **reality**: every entry is $-1$, $0$ or $+1$ (in particular real);
2. every $\gamma^a$ is a **signed permutation matrix** (one nonzero entry in each row
   and in each column); such a matrix is *orthogonal*: $M^T M = I$;
3. the **symmetry pattern**: $\gamma^a$ is symmetric for the space-like directions
   $x_0, x_1, x_2, x_3$ and antisymmetric for the time-like directions
   $x_4, \dots, x_7$; in one formula $(\gamma^a)^T = \eta_{aa}\gamma^a$.

Point 3 follows from point 2 and the Clifford relation, line by line:
$(\gamma^a)^T = (\gamma^a)^{-1}$ (orthogonal); $(\gamma^a)^2 = \eta_{aa} I_{16}$
(Clifford relation with $b = a$), so $\gamma^a \cdot \eta_{aa}\gamma^a = I_{16}$
(multiply by $\eta_{aa}$ and use $\eta_{aa}^2 = 1$), that is
$(\gamma^a)^{-1} = \eta_{aa}\gamma^a$; together, $(\gamma^a)^T = \eta_{aa}\gamma^a$.
```

In [24]:

```
entries = sorted(set(int(x) for g in gamma for x in g.flatten()))
say(f"the different entries of the eight gamma matrices: {entries}")
rows_ok = all((np.count_nonzero(g, axis=1) == 1).all() for g in gamma)  # each row
columns_ok = all((np.count_nonzero(g, axis=0) == 1).all() for g in gamma)  # column
orthogonal = all((g.T @ g == I16).all() for g in gamma)
check_record(entries == [-1, 0, 1], "every gamma^a is real with entries -1, 0, +1",
             (WOLFRAM, "reality"))
check_record(rows_ok and columns_ok and orthogonal,
             "every gamma^a is a signed permutation matrix, hence orthogonal",
             (WOLFRAM, "signed_permutation_matrices"),
             (PYTHON, "reality_signed_permutations"))
symmetric = [COORDINATES[a] for a in range(8) if (gamma[a].T == gamma[a]).all()]
antisymmetric = [COORDINATES[a] for a in range(8) if (gamma[a].T == -gamma[a]).all()]
say("symmetric: " + ", ".join(symmetric) + ";  antisymmetric: "
    + ", ".join(antisymmetric))
check_record(symmetric == ["x0", "x1", "x2", "x3"]
             and antisymmetric == ["x4", "x5", "x6", "x7"]
             and all((gamma[a].T == eta[a] * gamma[a]).all() for a in range(8)),
             "(gamma^a)^T = eta_aa gamma^a: symmetric for x0, x1, x2, x3 only",
             (WOLFRAM, "symmetry_pattern"), (PYTHON, "symmetry_pattern"))
```

Out [24]:

```
the different entries of the eight gamma matrices: [-1, 0, 1]
PASS every gamma^a is real with entries -1, 0, +1
```

```
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check reality
PASS every gamma^a is a signed permutation matrix, hence orthogonal
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check signed_permutation_matrices
     reproduces Revision/algebra/reports/python-algebra.json
         check reality_signed_permutations
symmetric: x0, x1, x2, x3;  antisymmetric: x4, x5, x6, x7
PASS (gamma^a)^T = eta_aa gamma^a: symmetric for x0, x1, x2, x3 only
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check symmetry_pattern
     reproduces Revision/algebra/reports/python-algebra.json
         check symmetry_pattern
```

Text cell:

```
The next cell shows the symmetry pattern in a picture: $\gamma^{(x_1)}$ next to its
transpose (the same picture: symmetric) and $\gamma^{(x_4)}$ next to its transpose
(every colour flipped: antisymmetric).
```

In [25]:

```
fig, axes = plt.subplots(2, 2, figsize=(6.4, 6.6), layout="constrained")
draw_signs(axes[0, 0], gamma[1], "$\\gamma^{(x_1)}$ (space-like)")
draw_signs(axes[0, 1], gamma[1].T, "its transpose: the same")
draw_signs(axes[1, 0], gamma[4], "$\\gamma^{(x_4)}$ (time-like)")
draw_signs(axes[1, 1], gamma[4].T, "its transpose: all signs flipped")
sign_legend(fig)
save_figure(fig, "symmetry_pattern",
            "The symmetry pattern of the gammas (rows and columns numbered 0 to 15; "
            "blue $-1$, grey $0$, red $+1$). Upper row: the space-like "
            "$\\gamma^{(x_1)}$ and its transpose (rows and columns exchanged) are "
            "the same picture, so $\\gamma^{(x_1)}$ is symmetric. Lower row: the "
            "transpose of the time-like $\\gamma^{(x_4)}$ has every colour flipped, "
            "so $\\gamma^{(x_4)}$ is antisymmetric. In general "
            "$(\\gamma^a)^T = \\eta_{aa}\\gamma^a$.")
```

Out [25]:

```
Figure 04a.6 saved as Revision/textbook/figures/04a_6_symmetry_pattern.png
```

Text cell:

```
## 14. Comparison with the Revision record files

The Revision record stores the eight gammas in two files:
‘Revision/algebra/gammas.json‘ (written by the WolframScript verifier) and
‘Revision/algebra/reports/python-gammas.json‘ (written by the Python checker). In
‘gammas.json‘ every number is written exactly: a whole number as a JSON integer and a
```

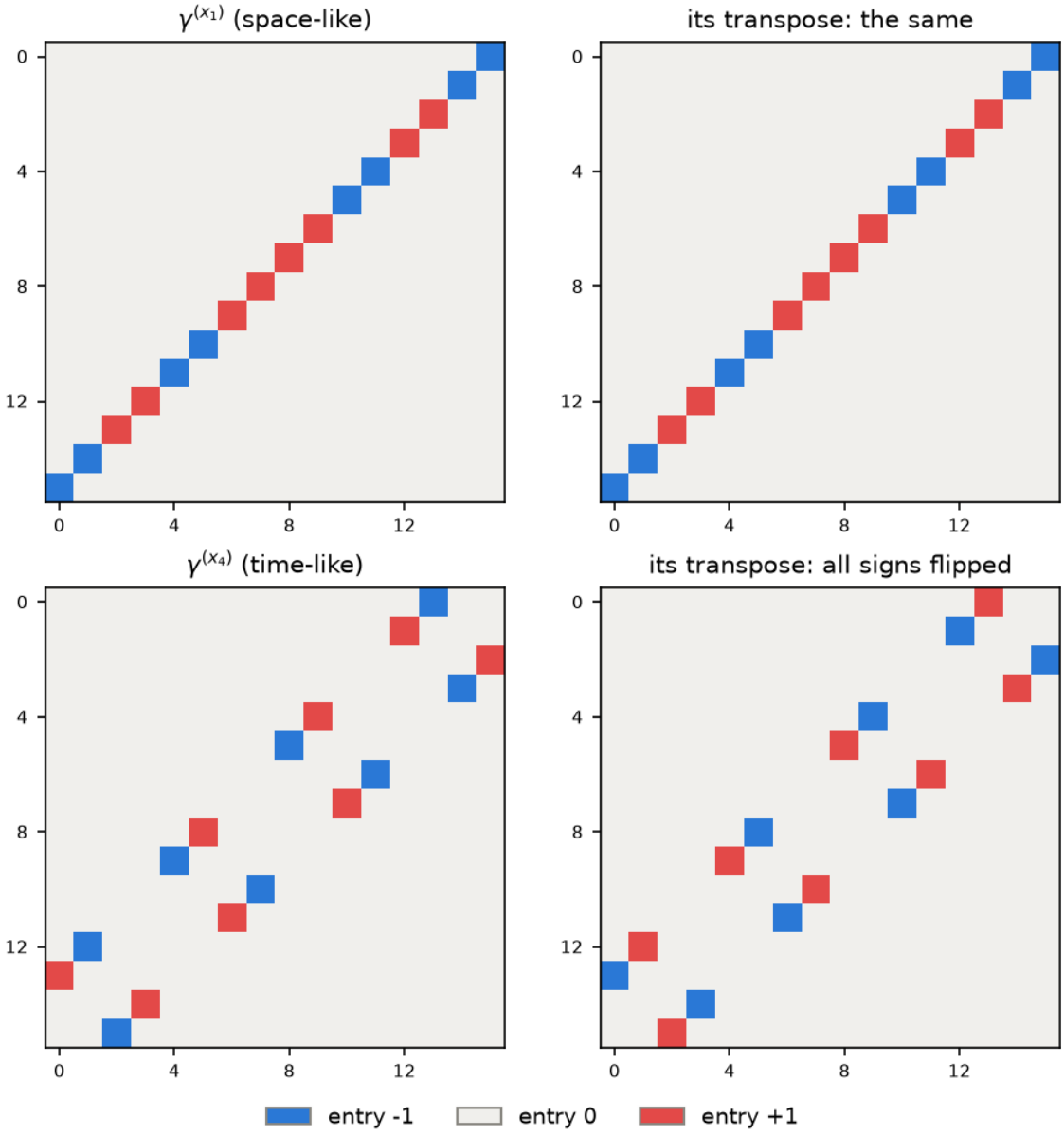

Figure 121: The symmetry pattern of the gammas (rows and columns numbered 0 to 15; blue -1, grey 0, red $+1$). Upper row: the space-like $\gamma^{(x_1)}$ and its transpose (rows and columns exchanged) are the same picture, so $\gamma^{(x_1)}$ is symmetric. Lower row: the transpose of the time-like $\gamma^{(x_4)}$ has every colour flipped, so $\gamma^{(x_4)}$ is antisymmetric. In general $(\gamma^a)^T = \eta_{aa}\gamma^a$. (Notebook 04a, figure 6.)

fraction as a text "p/q" (the function ‘exact‘ below reads both). The next cell reads both files, compares the coordinate names, the frame metric, the map to the author’s index A, and every one of the $8 \times 256 = 2048$ entries of the gammas with the matrices built above, and counts the entries that differ.

In [26]:

```
def exact(entry):
    """An exact number of gammas.json: a JSON integer, or a text "p/q"."""
    if isinstance(entry, int):
        return Fraction(entry)
    if isinstance(entry, str) and "/" in entry:
        numerator, denominator = entry.split("/")
        return Fraction(int(numerator), int(denominator))
    raise ValueError(f"an entry of gammas.json that is not exact: {entry}")


def differing(stored, ours):
    """How many entries of the stored matrix differ from our integer matrix."""
    return sum(1 for (i, j), x in np.ndenumerate(ours)
               if exact(stored[i][j]) != int(x))


fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
python_gammas = json.loads(repository_file(
    "Revision/algebra/reports/python-gammas.json").read_text(encoding="utf-8"))
fixture_differ = [differing(fixture["gamma"][a], gamma[a]) for a in range(8)]
python_differ = [differing(python_gammas["gamma"][a], gamma[a]) for a in range(8)]
report("entries compared per file", 8 * 16 * 16)
report("differing entries, gammas.json, per x0..x7", fixture_differ)
report("differing entries, python-gammas.json, per x0..x7", python_differ)
check_record(fixture["coordinates"] == COORDINATES and fixture["eta"] == list(eta)
             and fixture["notebookFrameIndex"] == FRAME_INDEX
             and sum(fixture_differ) == 0,
             "Revision/algebra/gammas.json holds exactly these eight gammas",
             (WOLFRAM, "fixture_round_trip"),
             (PYTHON, "fixture_comparison_gammas_json"))
python_map = python_gammas["map_coordinate_to_T16_index"]  # {"x0": 0, ...}
check(python_map == dict(zip(COORDINATES, FRAME_INDEX)) and sum(python_differ) == 0,
      "Revision/algebra/reports/python-gammas.json holds exactly these eight gammas")
```

Out [26]:

```
RESULT entries compared per file = 2048
RESULT differing entries, gammas.json, per x0..x7 = [0, 0, 0, 0, 0, 0, 0, 0]
RESULT differing entries, python-gammas.json, per x0..x7 = [0, 0, 0, 0, 0, 0, 0, 0]
PASS Revision/algebra/gammas.json holds exactly these eight gammas
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check fixture_round_trip
     reproduces Revision/algebra/reports/python-algebra.json
         check fixture_comparison_gammas_json
PASS Revision/algebra/reports/python-gammas.json holds exactly these eight gammas
```

Text cell:

```
The next cell prints the totals of the two Revision reports and how many different
Revision checks this notebook has reproduced.
```

In [27]:

```
for report_file in (WOLFRAM, PYTHON):
    verdicts = [entry["verdict"].upper() for entry in RECORDED[report_file].values()]
    passed = verdicts.count("PASS")  # how many checks of the report passed
    mine = [name for f, name in REPRODUCED if f == report_file]
    report(f"{report_file} passed", f"{passed} of {len(verdicts)}")
    report("    of these checks, reproduced in this notebook", len(mine))
check(len(REPRODUCED) == 30, "this notebook reproduces 30 checks of the two reports")
```

Out [27]:

```
RESULT Revision/algebra/reports/wolfram-algebra.json passed = 45 of 45
RESULT     of these checks, reproduced in this notebook = 16
RESULT Revision/algebra/reports/python-algebra.json passed = 35 of 35
RESULT     of these checks, reproduced in this notebook = 14
PASS this notebook reproduces 30 checks of the two reports
```

Text cell:

```
## 15. The last check

The last cell checks that the six figure files exist in the folder
‘Revision/textbook/figures‘ and prints the number of checks that passed.
```

In [28]:

```
names = ["04a_1_blocks_s4_t4.png", "04a_2_tau_matrices.png",
         "04a_3_block_form_of_t16.png", "04a_4_gammas_x0_to_x7.png",
         "04a_5_anticommutator_table.png", "04a_6_symmetry_pattern.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all six figure files exist")
all_checks_passed()
```

Out [28]:

```
PASS all six figure files exist
ALL 28 CHECKS PASSED (notebook 04a)
```

Text cell:

```
## 16. What this notebook showed

- PROVED (exact integer arithmetic, this notebook; also in the Revision reports
  ‘Revision/algebra/reports/wolfram-algebra.json‘ and ‘python-algebra.json‘): the
  author’s formulas give six $4 \times 4$ blocks, exactly the ones his notebook
  displays; they are antisymmetric, square to $-I_4$, s4 is self-dual and t4
  anti-self-dual, and every s4 commutes with every t4. The blocks are the right
  (s4) and minus the left (t4) multiplications by the quaternion units
  $\mathbf{i}, \mathbf{j}, \mathbf{k}$, which explains these rules.
- PROVED: the $8 \times 8$ matrices satisfy sigma = tau[1] tau[2] tau[3] =
  tau[4] tau[5] tau[6] tau[7], tau[7] = diag(-I4, I4), taubar[A] = -tau[A]
  (A = 1, ..., 7), and the key rule tau[A] taubar[B] + tau[B] taubar[A] =
  $2\,$eta4488$_{AB} I_8$ for all 64 pairs.
- PROVED: the $16 \times 16$ matrices T16[A] = ((0, taubar[A]), (tau[A], 0)) satisfy
  the Clifford relation; named after the author’s coordinates
  ($\gamma^{(x_k)}$ = T16[k] for k = 0, ..., 7) they satisfy
  $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I_{16}$ with
  $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ for all 64 pairs (numpy and
  sympy); they are real signed permutation matrices, symmetric for $x_0, x_1, x_2,
  x_3$ and antisymmetric for the time $x_4$ and the deflating extra times
  $x_5, x_6, x_7$.
- The matrices agree entry by entry with the Revision record files
  ‘Revision/algebra/gammas.json‘ and ‘Revision/algebra/reports/python-gammas.json‘;
  the notebook reproduces 30 checks of the two Revision reports.
- ASSUMED: nothing beyond the author’s formulas. The gamma matrices are pure
  algebra: they make no physical claim. They are constant; the inflation of 3-space
  and the deflation of the extra times enter only later, through the frame factors
  of the metric.
```

4.8 Line-by-line walk-through of Notebook 04a

The notebook has 28 code cells, In [1] to In [28]. This section quotes every line of every one of them, in order, and explains what each line or small group of lines does. Only the long caption texts handed to `save_figure` are shortened here, to a line `...)` whose three dots stand for the caption text and whose bracket closes the call of `save_figure`, because each caption is printed in full under its figure in Section 4.7; every walk-through of this chapter does the same. Two kinds of lines are text for the reader and are not executed: a **comment** is everything after a `#` sign on a line, and a **docstring** is a text in triple quotes `"""..."""` directly below a `def` line, which says in words what the function does (Python stores it with the function and skips it when the function runs). They are quoted with the code they describe; the explanations below add what they do not already say.

In [1], the set-up cell. Its first part is the complete run instructions of Section 4.6 again, as **comment lines**: every line that starts with `#` is a comment, which Python skips; they are there so that the notebook file carries its own instructions. The code starts after the second line of `=` signs. This set-up cell is the same in every notebook of the book, except for the line that names the notebook.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json`, `os`, `textwrap` and `pathlib` come with Python; `from pathlib import Path` takes only the name `Path` out of `pathlib`. A `Path` is the address of a file or folder, written the same way on every operating system. The plotting package matplotlib is loaded, its drawing functions under the short name `plt`, and from IPython (the part of Jupyter that runs Python code) the two functions `Image` and `display`, which show a picture file below a cell.

```
NOTEBOOK_ID = "04a"  # this notebook: chapter 04, example a
```

A **variable** is a name for a value; this line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"04a"`. The figure files and the last printed line use it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. `here.parents` lists its parent folder, the parent of that, and so on; `[here, *here.parents]` is the list that starts with `here` and continues with all of them (the star unpacks the parents into the list). The `for` loop takes the folders one after the other; the operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If none qualifies, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The line `REPO = find_repository_root()` calls the function and names the result `REPO`. As its comment says, it is never printed, because it differs from computer to computer while the printed output must not. The line that defines `OUTPUT_ROOT` chooses the folder below which files are written: `os.environ` holds the **environment variables** of the running program (named texts it receives from the computer), and `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and otherwise the default, the repository folder as a string. When you run the notebook the variable is not set, so the files go into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` gives the full path of a repository file, for reading a Revision record. `output_file("Revision/...")` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder that will hold it (and any missing folder above it) and does nothing if it exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters (the width of a page of the book); `textwrap.fill` breaks the text at blanks, and every continuation line starts with four blanks. `str(text)` turns any value into a string first.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings of matplotlib, so that personal settings on your computer cannot change the figures. `plt.rcParams.update({...})` then sets four settings for every figure: 7.0 by 4.2 inches, letters of 10 points, and a faint grid (30 per cent opaque) behind the curves. The braces `{...}` make a **dictionary**, a collection of pairs `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/04a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes the text `{}` (an empty JSON dictionary) and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` writes the same line end on every operating system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` saves and shows a figure, as its docstring says; `<id>` and `<k>` in the docstring stand for the notebook's name and the figure's number. `setdefault(name, value)` returns the number already stored for this figure name or stores and returns one more than the number of figures so far; so the figures are numbered 1, 2, 3, and a cell run twice keeps its number. `fig.savefig` writes the PNG file with 150 dots per inch, cuts away the empty margin (`bbox_inches="tight"`) and stores no program name (`metadata={"Software": None}`), so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time. The caption is stored, the whole dictionary of captions is written to the captions file (`json.dumps` turns it into JSON text, one entry per line, keys sorted), `display(Image(...))` shows the saved picture, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection in square brackets). `check` is behind every check of the book. `condition` is either `True` or `False`; if it is false, `raise AssertionError(...)` stops the notebook with an error naming the check. (Python also has a statement `assert condition`, but the docstring explains why it is not used: Python started with the option `-O` skips every `assert`, while an `if` statement always runs.) Otherwise the name is appended to `PASSED` and `PASS name` is printed. `record=None` makes the third argument optional; when given, a second line names the Revision record that the check reproduces.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; the expression `(f" {unit}" if u nit else "")` adds a blank and the unit only when a unit is given. `all_checks_passed` prints the last line of the notebook with `len(PASSED)`, the number of checks that passed. The last statement prints the only output of In [1].

In [2], the small tools.

```
import itertools  # loops over all pairs (or triples, ...) of indices
from fractions import Fraction  # exact fractions such as 1/2

import numpy as np  # integer matrices; their products are exact (no rounding)
import sympy as sp  # a second, independent engine for exact matrix algebra
from matplotlib.colors import BoundaryNorm, ListedColormap
from matplotlib.patches import Patch, Rectangle
```

`itertools` makes all pairs, triples or lists of indices; `Fraction` (used in In [26]) holds exact fractions. numpy, under the short name `np`, stores matrices as arrays; all matrices of this notebook hold whole numbers (numpy's type `int64`), whose sums and products are exact, so every check is exact. sympy, as `sp`, is a second program for exact algebra, used in In [21] as an independent judge. `Boundary Norm` and `ListedColormap` make the three-colour heat maps; `Patch` draws the coloured squares of a legend and `Rectangle` a frame.

```
def signature(numbers):
    """Mathematica's Signature: 0 if a number occurs twice; otherwise +1 for an even
    and -1 for an odd number of inversions (pairs of positions i < j with
    numbers[i] > numbers[j])."""
    if len(set(numbers)) != len(numbers):  # a set keeps each number only once
        return 0
    inversions = sum(1 for i, j in itertools.combinations(range(len(numbers)), 2)
```

```
                    if numbers[i] > numbers[j])
    return -1 if inversions % 2 else 1  # % 2 is the remainder after division by 2
```

The permutation sign of Section 4.2. `set(numbers)` keeps each number once, so if it is shorter than the list a number is repeated and the sign is 0. `itertools.combinations(range(len(numbers)), 2)` lists every pair of positions $i < j$; the expression `sum(1 for ... if numbers[i] > numbers[j])` adds 1 for every pair in which the larger number stands first, that is it counts the inversions. `inversions % 2` is 1 for an odd and 0 for an even count, and Python reads 1 as true and 0 as false, so the function returns -1 for an odd and $+1$ for an even number of inversions.

```
def delta(i, j):
    """The Kronecker delta: 1 if i == j, otherwise 0."""
    return 1 if i == j else 0


def anticommutator(a, b):
    """{a, b} = a b + b a; the operator @ multiplies matrices."""
    return a @ b + b @ a


def identity(n):
    """The n x n identity matrix with whole-number entries."""
    return np.eye(n, dtype=np.int64)


def zeros(n):
    """The n x n zero matrix with whole-number entries."""
    return np.zeros((n, n), dtype=np.int64)
```

Four short functions, each with one line of code below its docstring: the Kronecker delta (1 if $i = j$, else 0; `==` tests equality); the anticommutator $\{a, b\} = ab + ba$, with `@` the matrix product; the $n \times n$ identity matrix with whole-number entries (`np.eye`); and the $n \times n$ zero matrix.

```
for numbers in ([1, 2, 3, 4], [2, 1, 3, 4], [2, 3, 1, 4], [1, 1, 3, 4]):
    say(f"the sign of {numbers} is {signature(numbers)}")
check(signature([1, 2, 3, 4]) == 1 and signature([2, 1, 3, 4]) == -1
      and signature([2, 3, 1, 4]) == 1 and signature([1, 1, 3, 4]) == 0,
      "the permutation sign agrees with the four examples worked out by hand")
```

The loop prints the sign of the four example lists of Section 4.2, and the check requires the values $+1$, -1, $+1$ and 0 found there by hand. `and` is true only when both sides are true. Output: four lines and the first PASS line.

In [3], the Revision reports.

```
WOLFRAM = "Revision/algebra/reports/wolfram-algebra.json"  # WolframScript report
PYTHON = "Revision/algebra/reports/python-algebra.json"  # Python report
RECORDED = {}  # report file -> {check name: the check (name, verdict, detail)}
for report_file in (WOLFRAM, PYTHON):
    text = repository_file(report_file).read_text(encoding="utf-8")
```

```
    RECORDED[report_file] = {entry["name"]: entry
                             for entry in json.loads(text)["checks"]}
    say(f"{report_file}: {len(RECORDED[report_file])} checks recorded")
REPRODUCED = set()  # the (report, check) pairs that this notebook reproduces
```

The two report files of the Revision record are named. For each, `read_text` reads the file and `json.loads` turns the JSON text into Python lists and dictionaries; the report holds a list `"checks"` of entries, each with a `"name"`, a `"verdict"` and a `"detail"`. The **dictionary comprehension** `{entry["name"]: entry for entry in ...}` makes a dictionary from check names to entries. The cell prints how many checks each report holds: 45 and 35. `REPRODUCED` starts as an empty **set** (a collection without repetitions).

```
def check_record(condition, name, *records):
    """check(condition, name) for a statement that the Revision record verified too.
    records: pairs (report file, check name); each must be recorded as passed."""
    for report_file, check_name in records:
        entry = RECORDED[report_file].get(check_name)
        if entry is None or entry["verdict"].upper() != "PASS":
            raise AssertionError(f"{report_file} has no passed check {check_name}")
    check(condition, name)
    for report_file, check_name in records:
        print(f"     reproduces {report_file}")
        print(f"          check {check_name}")
        REPRODUCED.add((report_file, check_name))
```

`check_record` is `check` for a statement that the Revision record also verified. `*records` collects any number of further arguments, each a pair (report file, check name). The first loop looks each check up (`.get` returns `None` for a missing name) and stops with an error if it is missing or not recorded as passed; `.upper()` makes the comparison work for both spellings, PASS in the WolframScript report and pass in the Python report. Then `check` tests the notebook's own condition and prints the PASS line, and the second loop prints, for each Revision check, the file and the check name and adds the pair to `REPRODUCED`.

In [4], the drawing tools.

```
# Three colours: entry -1 blue, entry 0 light grey, entry +1 red.
SIGN_COLOURS = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])
# The boundaries -1.5, -0.5, 0.5, 1.5 put -1, 0 and +1 into the three colours.
SIGN_NORM = BoundaryNorm([-1.5, -0.5, 0.5, 1.5], SIGN_COLOURS.N)
```

A colour such as `"#2a78d6"` is written as three two-digit hexadecimal numbers (base 16) for the amounts of red, green and blue. `ListedColormap` makes a list of three colours; `BoundaryNorm` with the boundaries $-1.5, -0.5, 0.5, 1.5$ assigns every value between two neighbouring boundaries to one colour, so -1 is blue, 0 light grey and $+1$ red. `SIGN_COLOURS.N` is the number of colours, 3.

```
def draw_signs(ax, matrix, title, numbers=False, first=0):
    """Draw a matrix with entries -1, 0, +1 as a heat map in the panel ax."""
    size = matrix.shape[0]
    ax.imshow(matrix, cmap=SIGN_COLOURS, norm=SIGN_NORM)
```

```
    ax.set_title(title, fontsize=10)
    step = 1 if size <= 8 else 4  # label every row (small) or every 4th (large)
    ticks = list(range(0, size, step))
    ax.set_xticks(ticks, [str(t + first) for t in ticks], fontsize=7)
    ax.set_yticks(ticks, [str(t + first) for t in ticks], fontsize=7)
    ax.grid(False)  # no grid lines on top of the coloured squares
    if numbers:
        for (row, column), value in np.ndenumerate(matrix):
            if value:
                ax.text(column, row, f"{value:+d}", ha="center", va="center",
                        color="white", fontsize=8)
```

`draw_signs` draws one matrix into one **panel** `ax` (a pair of axes) of a figure. `matrix.shape[0]` is its number of rows. `ax.imshow` paints every entry as a small coloured square, row 0 at the top. The axis labels mark every row of a small matrix and every fourth row of a large one; `range(0, size, step)` counts from 0 in steps of `step`, and `t + first` shifts the labels to start at 1 for the author's 4×4 blocks. With `numbers=True`, `np.ndenumerate` runs through all entries with their positions, and every nonzero entry is written into its square in white, with its sign (`{value:+d}` writes $+1$ or -1); `ha` and `va` centre the text.

```
def sign_legend(fig):
    """The key of the three colours, below the panels of the figure."""
    handles = [Patch(facecolor=SIGN_COLOURS(k), edgecolor="#898781", label=label)
               for k, label in enumerate(["entry -1", "entry 0", "entry +1"])]
    fig.legend(handles=handles, loc="outside lower center", ncol=3, fontsize=9,
               frameon=False)
```

`sign_legend` adds the key of the three colours below the panels. `enumerate` numbers the three labels 0, 1, 2; `SIGN_COLOURS(k)` is colour number `k`; each `Patch` is a small coloured square with its label, and `fig.legend` places the three in one row (`ncol=3`) below the figure, without a frame.

In [5], the six blocks.

```
def Qa(h, p, q):
    """The author's Qa[h, p, q] (In[294]); h, p, q count from 1, as in Mathematica."""
    return signature([h, p, q, 4])


def Qb(h, p, q):
    """The author's Qb[h, p, q] (In[294])."""
    return delta(p, 4) * delta(q, h) - delta(p, h) * delta(q, 4)
```

The author's two patterns of Section 4.5, letter for letter: $Q_a[h]_{pq} = \text{Signature}[\{h, p, q, 4\}]$ and $Q_b[h]_{pq} = \delta_{p4}\delta_{qh} - \delta_{ph}\delta_{q4}$, with h, p, q counted from 1; `*` multiplies numbers.

```
# s4[h] = Qa - Qb (In[300]) and t4[h] = Qa + Qb (In[301]). Row p, column q of the
# formula (1 to 4) is row p - 1, column q - 1 of the numpy array (0 to 3).
s4 = {h: np.array([[Qa(h, p, q) - Qb(h, p, q) for q in range(1, 5)]
                   for p in range(1, 5)]) for h in (1, 2, 3)}
t4 = {h: np.array([[Qa(h, p, q) + Qb(h, p, q) for q in range(1, 5)]
                   for p in range(1, 5)]) for h in (1, 2, 3)}
```

`range(1, 5)` is 1, 2, 3, 4. The inner **list comprehension** `[... for q in range(1, 5)]` makes one row (the four columns q), the outer one the four rows p, and `np.array` turns the list of rows into a matrix. Row p of the formula becomes row $p-1$ of the array, because numpy counts from 0. The dictionaries `s4` and `t4` hold the three blocks under the keys 1, 2, 3.

```
def side_by_side(matrices, labels):
    """Print square matrices next to each other, one printed line per row."""
    # ^11: centred in 11 characters; rstrip() removes the blanks at the end
    print("   ".join(f"{label:^11}" for label in labels).rstrip())
    for row in range(matrices[0].shape[0]):
        print("   ".join(" ".join(f"{x:2d}" for x in m[row]) for m in matrices))


side_by_side([s4[1], s4[2], s4[3]], ["s4[1]", "s4[2]", "s4[3]"])
print()
side_by_side([t4[1], t4[2], t4[3]], ["t4[1]", "t4[2]", "t4[3]"])
```

`side_by_side` prints several matrices next to each other. `" ".join(...)` glues texts together with three blanks between them; `{label:^11}` centres a title in 11 characters. For each row number, the inner `" ".join(f"{x:2d}" ...)` writes the entries of that row of one matrix, each two characters wide, and the outer join puts the rows of the three matrices next to each other. The output shows the six blocks, exactly the matrices of Section 4.5; `print()` prints the empty line between the two groups.

In [6], comparison with the author's display.

```
DISPLAYED_S4 = {  # Out[304] of the author's notebook: s4by4[1], s4by4[2], s4by4[3]
    1: [[0, 0, 0, 1], [0, 0, 1, 0], [0, -1, 0, 0], [-1, 0, 0, 0]],
    2: [[0, 0, -1, 0], [0, 0, 0, 1], [1, 0, 0, 0], [0, -1, 0, 0]],
    3: [[0, 1, 0, 0], [-1, 0, 0, 0], [0, 0, 0, 1], [0, 0, -1, 0]],
}
DISPLAYED_T4 = {  # Out[304]: t4by4[1], t4by4[2], t4by4[3]
    1: [[0, 0, 0, -1], [0, 0, 1, 0], [0, -1, 0, 0], [1, 0, 0, 0]],
    2: [[0, 0, -1, 0], [0, 0, 0, -1], [1, 0, 0, 0], [0, 1, 0, 0]],
    3: [[0, 1, 0, 0], [-1, 0, 0, 0], [0, 0, 0, -1], [0, 0, 1, 0]],
}
```

The six matrices that the author's notebook displays in its output Out[304], typed in row by row (the Revision Python checker types in the same six).

```
# .all() is True when the comparison holds for every entry.
same = all((s4[h] == np.array(DISPLAYED_S4[h])).all()
           and (t4[h] == np.array(DISPLAYED_T4[h])).all() for h in (1, 2, 3))
check_record(same, "the formulas give the six blocks that the author displays",
             (PYTHON, "notebook_blocks_s4_t4"),
             (WOLFRAM, "s4_t4_entries_from_Signature_and_deltas"))
```

Comparing two numpy arrays with `==` gives an array of `True`/`False`, one per entry; `.all()` is true when every entry agrees. `all(... for h in (1, 2, 3))` requires it for the three values of h. The check prints PASS and names the two Revision checks it reproduces.

In [7], figure 1.

```
fig, axes = plt.subplots(2, 3, figsize=(7.4, 5.6), layout="constrained")
for h in (1, 2, 3):
    draw_signs(axes[0, h - 1], s4[h], f"s4[{h}]", numbers=True, first=1)
    draw_signs(axes[1, h - 1], t4[h], f"t4[{h}]", numbers=True, first=1)
sign_legend(fig)
save_figure(fig, "blocks_s4_t4",
            ...)
```

(The caption is shortened here to the line `...)`: the three dots stand for the caption text, and the bracket closes the call of `save_figure`. The caption is printed in full in Section 4.7, written as several strings next to each other, which Python joins into one.) `plt.subplots(2, 3, ...)` makes a figure with two rows of three panels, 7.4 by 5.6 inches; `layout="constrained"` spaces the panels so that nothing overlaps. `axes[0, h - 1]` is the panel in row 0 and column $h-1$. The upper row shows s4[1], s4[2], s4[3], the lower row the t-blocks, each entry written into its square, rows and columns numbered 1 to 4. **What figure 1 shows and why:** every block has exactly one coloured square in each row and each column (a signed permutation), and the square in row p, column q has the opposite colour of the square in row q, column p, because every block is antisymmetric (Section 4.5).

In [8], the rules of the blocks.

```
I4 = identity(4)
blocks = [s4[1], s4[2], s4[3], t4[1], t4[2], t4[3]]
antisymmetric = all((m.T == -m).all() for m in blocks)
square_minus_one = all((m @ m == -I4).all() for m in blocks)
```

`m.T` is the transpose of `m`. `antisymmetric` is true when $m^T = -m$ for all six blocks, and `square_minus_one` when $m\,m = -I_4$ for all six.

```
anticommute = all((anticommutator(s4[h], s4[k]) == -2 * delta(h, k) * I4).all()
                  and (anticommutator(t4[h], t4[k]) == -2 * delta(h, k) * I4).all()
                  for h in (1, 2, 3) for k in (1, 2, 3))
s_t_commute = all((s4[h] @ t4[k] == t4[k] @ s4[h]).all()
                  for h in (1, 2, 3) for k in (1, 2, 3))
```

The two `for` parts run through all nine pairs (h, k). `anticommute` tests $\{\mathrm{s4}[h], \mathrm{s4}[k]\} = \{\mathrm{t4}[h], \mathrm{t4}[k]\} = -2\delta_{hk} I_4$, and `s_t_commute` tests that every s-block commutes with every t-block.

```
check_record(antisymmetric and square_minus_one and s_t_commute,
             "the blocks are antisymmetric, square to -I4, and s4 commutes with t4",
             (PYTHON, "blocks_antisymmetric_square_minus_one_commute"))
check_record(anticommute and s_t_commute and antisymmetric,
             "{s4[h], s4[k]} = {t4[h], t4[k]} = -2 delta_hk I4, [s4[h], t4[k]] = 0",
             (WOLFRAM, "s4_t4_quaternion_algebras"))
```

Two checks, each worded as the Revision check it reproduces: the Python check combines antisymmetry, squares and commuting, the WolframScript check the anticommutators, the commuting and the antisymmetry.

In [9], self-dual and anti-self-dual.

```
epsilon = np.zeros((4, 4, 4, 4), dtype=np.int64)
for p, q, r, s in itertools.product(range(4), repeat=4):  # all 4^4 = 256 lists
    epsilon[p, q, r, s] = signature([p, q, r, s])
```

`epsilon` is a table with four indices, each 0 to 3 (counting from 0 does not change a permutation sign). `itertools.product(range(4), repeat=4)` lists all $4^4 = 256$ lists (p, q, r, s), and each entry is set to the permutation sign: the Levi-Civita symbol of Section 4.5.

```
def dual(m):
    """(1/2) sum_rs epsilon_pqrs m_rs for every p, q (a 4 x 4 matrix)."""
    twice = np.einsum("pqrs,rs->pq", epsilon, m)  # the sum over r and s
    if (twice % 2).any():  # for an antisymmetric m every entry of twice is even
        raise ValueError("the dual is not a whole-number matrix")
    return twice // 2  # exact division by 2
```

`np.einsum("pqrs,rs->pq", epsilon, m)` computes, for every p and q, the sum $\sum_{r,s} \epsilon_{pqrs} m_{rs}$: the letters before the arrow name the indices of the two inputs, the letters after it the indices of the result, and every letter that does not appear after the arrow is summed over. That is twice the dual. For an antisymmetric matrix every entry of it is even (the two equal terms of Section 4.5); the `if` line stops with an error otherwise, and `//` divides exactly by 2.

```
for h in (1, 2, 3):
    say(f"h = {h}: dual(s4) == s4: {(dual(s4[h]) == s4[h]).all()},  "
        f"dual(t4) == -t4: {(dual(t4[h]) == -t4[h]).all()}")
check_record(all((dual(s4[h]) == s4[h]).all() and (dual(t4[h]) == -t4[h]).all()
                 for h in (1, 2, 3)),
             "every s4[h] is self-dual and every t4[h] anti-self-dual",
             (WOLFRAM, "s4_self_dual_t4_anti_self_dual"))
```

For each h one line reports $\star\mathrm{s4}[h] = \mathrm{s4}[h]$ and $\star\mathrm{t4}[h] = -\mathrm{t4}[h]$ (three lines, all True), and the check requires both for all three.

In [10], the quaternions.

```
def quaternion_product(x, y):
    """x y for quaternions stored as (q1, q2, q3, q4) = q4 + q1 i + q2 j + q3 k."""
    x1, x2, x3, x4 = x
    y1, y2, y3, y4 = y
    return (x4 * y1 + x1 * y4 + x2 * y3 - x3 * y2,  # the i component
            x4 * y2 + x2 * y4 + x3 * y1 - x1 * y3,  # the j component
            x4 * y3 + x3 * y4 + x1 * y2 - x2 * y1,  # the k component
            x4 * y4 - x1 * y1 - x2 * y2 - x3 * y3)  # the real component
```

The product of two quaternions stored as (q_1, q_2, q_3, q_4). The first two lines **unpack** the four components of each. The four returned lines come from multiplying $(x_4 + x_1\mathbf{i} + x_2\mathbf{j} + x_3\mathbf{k})(y_4 + y_1\mathbf{i} + y_2\mathbf{j} + y_3\mathbf{k})$ term by term with Hamilton's rules; for example the $\mathbf{i}$ component collects x_4y_1 (from $x_4 \cdot y_1\mathbf{i}$), x_1y_4, x_2y_3 (from $\mathbf{jk} = \mathbf{i}$) and $-x_3y_2$ (from $\mathbf{kj} = -\mathbf{i}$), and the real component $x_4y_4 - x_1y_1 - x_2y_2 - x_3y_3$ (from $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1$).

```
UNITS = {1: (1, 0, 0, 0), 2: (0, 1, 0, 0), 3: (0, 0, 1, 0)}  # i, j, k
BASIS = [(1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1)]  # e_1, ..., e_4


def right_multiplication(u):
    """The 4 x 4 matrix of q -> q u: column p holds the components of e_p u."""
    return np.array([quaternion_product(e, u) for e in BASIS]).T


def left_multiplication(u):
    """The 4 x 4 matrix of q -> u q: column p holds the components of u e_p."""
    return np.array([quaternion_product(u, e) for e in BASIS]).T
```

`UNITS` holds $\mathbf{i}, \mathbf{j}, \mathbf{k}$ under the keys 1, 2, 3, and `BASIS` the four quaternions $e_1, \ldots, e_4$. `right_multiplication(u)` lists the products $e_p u$ as the rows of an array and transposes it with `.T`, so that they become the columns: the matrix R_u of Section 4.5. `left_multiplication(u)` does the same with ue_p: the matrix L_u.

```
say(f"i j = {quaternion_product(UNITS[1], UNITS[2])} (that is k), "
    f"j i = {quaternion_product(UNITS[2], UNITS[1])} (that is -k)")
check(quaternion_product(UNITS[1], UNITS[2]) == (0, 0, 1, 0)
      and quaternion_product(UNITS[1], UNITS[1]) == (0, 0, 0, -1),
      "the quaternion product follows Hamilton's rules i j = k and i i = -1")
check(all((s4[h] == right_multiplication(UNITS[h])).all()
          and (t4[h] == -left_multiplication(UNITS[h])).all() for h in (1, 2, 3)),
      "s4[h] is right and t4[h] minus left multiplication by i, j, k")
```

The printed line shows $\mathbf{ij} = (0, 0, 1, 0) = \mathbf{k}$ and $\mathbf{ji} = (0, 0, -1, 0) = -\mathbf{k}$. The first check confirms two of Hamilton's rules; the second confirms $\mathrm{s4}[h] = R_{u_h}$ and $\mathrm{t4}[h] = -L_{u_h}$ for $h = 1, 2, 3$, the fact from which Section 4.5 derived every rule of the blocks.

In [11], σ, τ and $\bar{\tau}$.

```
I8, Z4, Z8 = identity(8), zeros(4), zeros(8)
eta4488 = np.diag([1, 1, 1, 1, -1, -1, -1, -1])  # In[45]: frame order A = 0..7
sigma = np.block([[Z4, I4], [I4, Z4]])  # In[46]
```

Three matrices are named in one line (I_8 and the zero matrices of sizes 4 and 8). `np.diag(list)` makes the diagonal matrix with the listed diagonal: the author's eta4488. `np.block` glues blocks into a big matrix, row of blocks by row of blocks: $\sigma = \begin{pmatrix} 0 & I_4 \\ I_4 & 0 \end{pmatrix}$.

```
tau = {0: I8}  # In[338]
for h in (1, 2, 3):
    tau[h] = np.block([[Z4, s4[h]], [s4[h], Z4]])  # tau[1], tau[2], tau[3]
    tau[7 - h] = np.block([[Z4, t4[h]], [-t4[h], Z4]])  # tau[6], tau[5], tau[4]
tau[7] = tau[1] @ tau[2] @ tau[3] @ tau[4] @ tau[5] @ tau[6]  # In[338]
taubar = {0: I8}  # In[351]
for A in range(1, 8):
    taubar[A] = sigma @ tau[A].T @ sigma
```

The dictionary `tau` starts with $\tau_0 = I_8$; the loop adds τ_h from s4[h] and τ_{7-h} from t4[h] for $h = 1, 2, 3$; τ_7 is the product of the first six; and `taubar` holds $\bar{\tau}_0 = I_8$ and $\bar{\tau}_A = \sigma\tau_A^T\sigma$ for $A = 1, \ldots, 7$ (`range(1, 8)` is 1 to 7). These are the formulas of Step 2 in Section 4.5.

```
def codes(m):
    """The code of every row of a signed permutation matrix m: "+5" in row i means
    (m u)_i = +u_5. Stops with an error if a row has not exactly one nonzero."""
    result = []
    for row in m:
        columns = np.flatnonzero(row)  # the columns of the nonzero entries
        if len(columns) != 1 or abs(row[columns[0]]) != 1:
            raise ValueError("not a signed permutation matrix")
        column = int(columns[0])
        result.append(("+" if row[column] > 0 else "-") + str(column))
    return result
```

`codes` writes a signed permutation matrix in the code notation of Section 4.2. For every row, `np.flatnonzero(row)` lists the columns of its nonzero entries; if there is not exactly one, or it is not ± 1, the function stops with an error (so `codes` also checks that the matrix is a signed permutation matrix). Otherwise it appends the sign and the column number, for example `"+5"`.

```
labels = [f"tau[{A}]" for A in range(1, 8)]  # the column titles
print("row " + "".join(f"{label:>8}" for label in labels))
tau_codes = {A: codes(tau[A]) for A in range(1, 8)}
for i in range(8):
    print(f"{i:3d} " + "".join(f"{tau_codes[A][i]:>8}" for A in range(1, 8)))
```

The table of codes: one title line (`{label:>8}` writes each title right-aligned in 8 characters) and one line per row $i = 0, \ldots, 7$ with the code of row i of each of $\tau_1, \ldots, \tau_7$. In the output one reads, for example, that τ_7 has the codes $-0, -1, -2, -3, +4, +5, +6, +7$: it is $\mathrm{diag}(-I_4, I_4)$, as Step 2 proved.

In [12], figure 2.

```
fig, axes = plt.subplots(2, 4, figsize=(8.6, 5.0), layout="constrained")
for A in range(8):
    draw_signs(axes[A // 4, A % 4], tau[A], f"tau[{A}]")  # // and %: row, column
sign_legend(fig)
save_figure(fig, "tau_matrices",
            ...)
```

Two rows of four panels; matrix A goes into row $A//4$ (the whole part of $A/4$) and column $A\%4$ (the remainder). **What figure 2 shows and why:** τ_0 is the identity (red diagonal); τ_1 to τ_6 have their entries only in the upper-right and lower-left 4×4 corners, because they are made of the blocks s4 and t4 placed there; τ_7 is diagonal with four blue and four red entries, $\mathrm{diag}(-I_4, I_4)$.

In [13], the rules of σ and τ.

```
sigma_ok = (sigma.T == sigma).all() and np.trace(sigma) == 0 \
    and (sigma @ sigma == I8).all()
check_record(sigma_ok, "sigma is symmetric, traceless and squares to I8",
             (WOLFRAM, "sigma8_involution"))
```

A backslash at the end of a line continues the statement on the next line. σ must be symmetric, have trace 0 (`np.trace`) and square to I_8.

```
product_1_to_7 = I8
for A in range(1, 8):
    product_1_to_7 = product_1_to_7 @ tau[A]
check_record((tau[7] == np.diag([-1, -1, -1, -1, 1, 1, 1, 1])).all()
             and (product_1_to_7 == I8).all(),
             "tau[7] = diag(-I4, I4) and tau[1] tau[2] ... tau[7] = I8",
             (PYTHON, "tau7_and_product"))
```

The loop multiplies $\tau_1\tau_2\cdots\tau_7$ step by step, starting from I_8. The check requires $\tau_7 = \mathrm{diag}(-I_4, I_4)$ and $\tau_1\cdots\tau_7 = I_8$ (Step 2, lines 7 and 9).

```
sigma_products = (sigma == tau[1] @ tau[2] @ tau[3]).all() and \
    (sigma == tau[4] @ tau[5] @ tau[6] @ tau[7]).all()
sigma_transpose = all((sigma @ taubar[A] == (sigma @ tau[A]).T).all()
                      for A in range(8))
check_record(sigma_products and sigma_transpose,
             "sigma = tau1 tau2 tau3 = tau4 ... tau7 and sigma taubar = (sigma tau)^T",
             (WOLFRAM, "tau7_and_sigma_identities"),
             (PYTHON, "notebook_sigma_eq_tau1tau2tau3"))
```

Two identities that the author's notebook states: $\sigma = \tau_1\tau_2\tau_3 = \tau_4\tau_5\tau_6\tau_7$ (Step 2, lines 5 and 9), and $\sigma\bar{\tau}_A = (\sigma\tau_A)^T$ for all eight A (true because $(\sigma\tau_A)^T = \tau_A^T\sigma^T = \tau_A^T\sigma$, and $\sigma\bar{\tau}_A = \sigma\sigma\tau_A^T\sigma = \tau_A^T\sigma$ since $\sigma^2 = I_8$).

```
check_record(sigma_transpose and all((taubar[A] == -tau[A]).all()
                                     for A in range(1, 8)),
             "taubar[A] = -tau[A] for A = 1, ..., 7",
             (PYTHON, "taubar_relations"))
check_record(all((anticommutator(tau[A], tau[B]) == -2 * eta4488[A, B] * I8).all()
                 for A in range(1, 8) for B in range(1, 8)),
             "tau[A] tau[B] + tau[B] tau[A] = -2 eta4488_AB I8 for A, B = 1..7",
             (PYTHON, "notebook_tau_clifford"))
```

$\bar{\tau}_A = -\tau_A$ for $A = 1, \ldots, 7$ (Step 2, line 10), and the Clifford relation of the seven τ with the signs $-$eta4488, for all 49 pairs; `eta4488[A, B]` is the entry in row A, column B. Five PASS lines in all.

In [14], the key rule.

```
table = np.zeros((8, 8), dtype=np.int64)
multiples_of_identity = True
other_order = True
for A, B in itertools.product(range(8), repeat=2):  # all 64 pairs
    m = tau[A] @ taubar[B] + tau[B] @ taubar[A]
    table[A, B] = m[0, 0]  # the number that multiplies I8 (if it is a multiple)
    multiples_of_identity &= bool((m == table[A, B] * I8).all())
    other = taubar[A] @ tau[B] + taubar[B] @ tau[A]
    other_order &= bool((other == table[A, B] * I8).all())
```

For each of the 64 pairs the matrix $\tau_A\bar{\tau}_B + \tau_B\bar{\tau}_A$ is computed and its upper-left entry stored in `table`; if the matrix is a multiple of I_8, that entry is the multiple. `x &= y` means `x = x and y`: `multiples_of_identity` stays true only if every one of the 64 matrices equals its entry times I_8. The same is tested for the other order $\bar{\tau}_A\tau_B + \bar{\tau}_B\tau_A$. `bool(...)` turns numpy's truth value into Python's.

```
print("A\\B" + "".join(f"{B:4d}" for B in range(8)))
for A in range(8):
    print(f"{A:3d}" + "".join(f"{table[A, B]:4d}" for B in range(8)))
check_record(multiples_of_identity and other_order and (table == 2 * eta4488).all(),
             "tau[A] taubar[B] + tau[B] taubar[A] = 2 eta4488_AB I8, both orders",
             (WOLFRAM, "tau_taubar_Clifford_relation"),
             (PYTHON, "notebook_tau_taubar_clifford"))
```

The table is printed (`"A\\B"` prints `A\B`, the backslash written twice in the code because a single one has a special meaning in strings), row A, column B: 2 on the diagonal for $A = 0, 1, 2, 3$, -2 for $A = 4, \ldots, 7$, and 0 elsewhere. The check requires that it is twice eta4488: the key rule of Section 4.5.

In [15], the 16×16 matrices.

```
T16 = {A: np.block([[Z8, taubar[A]], [tau[A], Z8]]) for A in range(8)}  # In[371]
T16[8] = T16[0]
for A in range(1, 8):
    T16[8] = T16[8] @ T16[A]  # In[372]: T16[8] = T16[0] T16[1] ... T16[7]
I16 = identity(16)
```

T16$[A] = \begin{pmatrix} 0 & \bar{\tau}_A \\ \tau_A & 0 \end{pmatrix}$ for $A = 0, \ldots, 7$, then T16[8] as the product T16[0] T16[1] $\cdots$ T16[7], built up step by step, and I_{16}.

```
# [:8, 8:] means rows 0 to 7 and columns 8 to 15 (the upper-right corner), etc.
block_form = all((T16[A][:8, :8] == 0).all() and (T16[A][8:, 8:] == 0).all()
                 and (T16[A][:8, 8:] == taubar[A]).all()
                 and (T16[A][8:, :8] == tau[A]).all() for A in range(8))
check_record(block_form, "T16[A] = ((0, taubar[A]), (tau[A], 0)) for A = 0, ..., 7",
             (WOLFRAM, "T16_block_form"))
```

A **slice** `[:8, 8:]` picks rows 0 to 7 and columns 8 to 15 (`:8` means up to, not including, 8; `8:` from 8 to the end). The check confirms the four corners of every T16$[A]$.

```
say(f"diagonal of T16[8]: {list(int(x) for x in np.diag(T16[8]))}")
check_record((T16[8] == np.diag([-1] * 8 + [1] * 8)).all(),
             "T16[8] = T16[0] ... T16[7] = diag(-I8, I8)",
             (WOLFRAM, "Gamma_diag"), (PYTHON, "chirality_diag"))
```

`np.diag(matrix)` extracts the diagonal of a matrix (while `np.diag(list)` builds one); the line prints the 16 diagonal entries: eight -1, then eight $+1$. `[-1] * 8 + [1] * 8` is the list of eight -1 followed by eight 1. The check confirms T16[8] = diag$(-I_8, I_8)$, proved by hand in Step 3.

```
product_0_to_8 = I16
for A in range(9):
    product_0_to_8 = product_0_to_8 @ T16[A]
check_record((product_0_to_8 == I16).all(), "T16[0] T16[1] ... T16[8] = I16",
             (WOLFRAM, "notebook_product_identity"))
```

The product of all nine matrices T16[0] to T16[8] is I_{16}, an identity that the author's notebook states; it is $\mathrm{T16}[8]^2 = I_{16}$.

In [16], figure 3.

```
fig, ax = plt.subplots(figsize=(5.6, 5.6), layout="constrained")
draw_signs(ax, T16[4], "T16[4], the gamma matrix of the time $x_4$")
for left, top in ((-0.5, -0.5), (7.5, -0.5), (-0.5, 7.5), (7.5, 7.5)):
    ax.add_patch(Rectangle((left, top), 8, 8, fill=False, edgecolor="#0b0b0b",
                           linewidth=1.5))  # the frame of one 8 x 8 corner
```

One panel with T16[4]. In `imshow` the centre of the square of entry (i, j) lies at the point (j, i), so the square of entry $(0, 0)$ starts at -0.5; each `Rectangle((left, top), 8, 8, fill=False, ...)` draws an unfilled black frame of 8×8 squares around one corner.

```
labels = {(3.5, 3.5): "0", (11.5, 3.5): "taubar[4]", (3.5, 11.5): "tau[4]",
          (11.5, 11.5): "0"}
for (x, y), label in labels.items():
    ax.text(x, y, label, ha="center", va="center", fontsize=12, color="#0b0b0b",
            bbox={"facecolor": "white", "alpha": 0.85, "edgecolor": "none"})
sign_legend(fig)
save_figure(fig, "block_form_of_t16",
            ...)
```

The centres of the four corners get their names, each on a white, slightly transparent background (`bbox`). **What figure 3 shows and why:** the upper-left and lower-right corners are empty, because T16[4] is block off-diagonal; the upper-right corner is $\bar{\tau}_4$ and the lower-left τ_4, with one coloured square in each row.

In [17], the author's coordinates.

```
COORDINATES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
FRAME_INDEX = [0, 1, 2, 3, 4, 5, 6, 7]  # A of T16[A] for x0, ..., x7
ROLE = ["hidden direction", "3-space", "3-space", "3-space", "the time",
        "extra time (deflating)", "extra time (deflating)", "extra time (deflating)"]
gamma = [T16[A] for A in FRAME_INDEX]  # gamma[0] = gamma^(x0), ...
eta = np.array([eta4488[A, A] for A in FRAME_INDEX])  # signs in the order x0..x7
```

The coordinate map of Section 4.4: `FRAME_INDEX` lists, for $x_0, \ldots, x_7$, the index A of the author's T16[A]: the numbers 0 to 7 themselves. `gamma` is the list of the eight gamma matrices in the order of the coordinates (Python counts it from 0, as the author counts his coordinates: `gamma[0]` is $\gamma^{(x_0)}$, `gamma[7]` is $\gamma^{(x_7)}$), and `eta` the list of their signs, read off from the diagonal of eta4488 at the same indices.

```
for a in range(8):
    kind = "space-like" if eta[a] > 0 else "time-like"
    print(f"gamma^({COORDINATES[a]}) = T16[{FRAME_INDEX[a]}]   eta = {eta[a]:+d}   "
          f"{kind:10}   {ROLE[a]}")
```

One printed line per coordinate: its matrix, its sign, its kind and its role.

```
renamed = all(gamma[k] is T16[k] for k in range(8))
check_record(list(eta) == [1, 1, 1, 1, -1, -1, -1, -1] and renamed,
             "gamma^(xk) = T16[k] for k = 0..7, eta = diag(+,+,+,+,-,-,-,-)",
             (WOLFRAM, "coordinate_map"), (WOLFRAM, "eta_in_author_order"),
             (PYTHON, "coordinate_map"))
```

`is` tests that two names refer to the very same object (not only equal entries): `gamma[k]` is T16[k] itself for $k = 0, \ldots, 7$. The check requires these and $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$, reproducing three Revision checks.

In [18], the eight gammas in code notation.

```
gamma_codes = [codes(g) for g in gamma]
print("row" + "".join(f"{name:>6}" for name in COORDINATES))
for i in range(16):
    print(f"{i:3d}" + "".join(f"{gamma_codes[a][i]:>6}" for a in range(8)))
```

`codes` (which also confirms that each is a signed permutation matrix) is applied to all eight gammas, and the table prints, for each row $i = 0, \ldots, 15$, the code of row i of each gamma. This table is the complete content of the eight matrices: for example the column x4 starts with -13, so $(\gamma^{(x_4)}u)_0 = -u_{13}$, and the column x0 reads $+8, +9, \ldots, +15, +0, \ldots, +7$: $\gamma^{(x_0)}$ exchanges the upper and the lower eight components. Every code of rows 0 to 7 is a column 8 to 15 and vice versa: the block form.

In [19], figure 4.

```
fig, axes = plt.subplots(2, 4, figsize=(8.8, 5.0), layout="constrained")
for a in range(8):
    draw_signs(axes[a // 4, a % 4], gamma[a], f"$\\gamma^{{(x_{a})}}$")
sign_legend(fig)
save_figure(fig, "gammas_x0_to_x7",
            ...)
```

The eight gammas in two rows of four panels. In the f-string title, the doubled braces `{{` and `}}` print single braces and the doubled backslash prints one, so the title becomes the mathematical text $\gamma^{(x_0)}$, ..., $\gamma^{(x_7)}$. **What figure 4 shows and why:** each matrix has exactly one coloured square in every row and every column, all in the upper-right and lower-left 8×8 corners (signed permutations in block form); $\gamma^{(x_0)}$ consists of two red diagonal lines, because it is T16[0] $= \begin{pmatrix} 0 & I_8 \\ I_8 & 0 \end{pmatrix}$. These are the author's eight real 16×16 gamma matrices themselves.

In [20], the Clifford relation for all 64 pairs.

```
c = np.zeros((8, 8), dtype=np.int64)  # {gamma^a, gamma^b} = c[a, b] I16
all_multiples = True
for a, b in itertools.product(range(8), repeat=2):
    m = anticommutator(gamma[a], gamma[b])
    c[a, b] = m[0, 0]
    all_multiples &= bool((m == c[a, b] * I16).all())
```

As in In [14], now for the gammas: for all 64 ordered pairs (a, b) the anticommutator is computed, its upper-left entry stored as c_{ab}, and `all_multiples` stays true only if each anticommutator is exactly $c_{ab} I_{16}$.

```
print("a\\b" + "".join(f"{name:>5}" for name in COORDINATES))
for a in range(8):
    print(f"{COORDINATES[a]:>3}" + "".join(f"{c[a, b]:5d}" for b in range(8)))
squares = ", ".join(f"{COORDINATES[a]}:{c[a, a] // 2}" for a in range(8))
say(f"squares (gamma^a)^2 / I16: {squares}")
```

The table of the c_{ab} is printed; the string `squares` lists, for each coordinate, $c_{aa}/2$, the number in $(\gamma^a)^2 = (c_{aa}/2) I_{16}$, in the form `x0:1, x1:1, ...`.

```
recorded = RECORDED[WOLFRAM]["Clifford_relation"]["detail"]
check_record(all_multiples and (c == 2 * np.diag(eta)).all() and squares in recorded,
             "{gamma^a, gamma^b} = 2 eta^ab I16 for all 64 pairs a, b",
             (WOLFRAM, "Clifford_relation"), (PYTHON, "clifford_relation"))
```

`recorded` is the detail text of the WolframScript check `Clifford_relation`. The check requires every anticommutator to be a multiple of I_{16}, the table to be 2η, and the text `squares` to appear word for word in the Revision detail (`in` tests whether one string is part of another). The output shows the table: 2 on the diagonal for x_0, x_1, x_2, x_3, -2 for $x_4, \ldots, x_7$, 0 elsewhere.

In [21], the same with sympy.

```
sympy_gamma = [sp.Matrix(g.tolist()) for g in gamma]  # the same entries, in sympy
sympy_ok = all(
    sympy_gamma[a] * sympy_gamma[b] + sympy_gamma[b] * sympy_gamma[a]
    == 2 * int(eta[a] * delta(a, b)) * sp.eye(16)
    for a in range(8) for b in range(a, 8))
check_record(sympy_ok, "sympy confirms the 36 relations {gamma^a, gamma^b} = 2 eta^ab",
             (PYTHON, "clifford_relation_sympy"))
```

`g.tolist()` turns a numpy array into a list of rows, and `sp.Matrix` makes a sympy matrix of it; in sympy `*` is the matrix product and `sp.eye(16)` the identity. `range(a, 8)` makes b run from a to 7, so the 36 pairs with $a \le b$ are tested (the others follow, because $\{\gamma^a, \gamma^b\} = \{\gamma^b, \gamma^a\}$). $2\eta^{ab}$ is written as $2\,\eta_{aa}\delta_{ab}$, since η is diagonal.

In [22], one relation followed by hand.

```
def follow(m, row):
    """(m u)_row = sign * u_column for a signed permutation m: return (sign, column)."""
```

```
    column = int(np.flatnonzero(m[row])[0])
    return int(m[row, column]), column


def signed(sign, letter, index):
    """The text "+u_5" or "-u_5" for sign = +1 or -1, letter = "u", index = 5."""
    return ("+" if sign > 0 else "-") + letter + "_" + str(index)
```

`follow(m, row)` reads one row of a signed permutation matrix the way a student reads the code table: it returns the sign and the column of the single nonzero entry, so that $(mu)_{\text{row}} = \text{sign} \cdot u_{\text{column}}$. `signed` writes such a term as a text, for example `-u_13`.

```
g4, g0 = gamma[4], gamma[0]  # gamma^(x4) and gamma^(x0)
s1, c1 = follow(g4, 0)  # row 0 of gamma^(x4)
s2, c2 = follow(g4, c1)  # row c1 of gamma^(x4)
say(f"row 0 of gamma^(x4):  (gamma^(x4) v)_0 = {signed(s1, "v", c1)}")
say(f"row {c1} of gamma^(x4): (gamma^(x4) u)_{c1} = {signed(s2, "u", c2)}")
say(f"with v = gamma^(x4) u: (gamma^(x4) gamma^(x4) u)_0 = {signed(s1 * s2, "u", c2)}")
```

Row 0 of $\gamma^{(x_4)}$ gives $(\gamma^{(x_4)}v)_0 = -v_{13}$; row 13 gives $(\gamma^{(x_4)}u)_{13} = +u_0$. With $v = \gamma^{(x_4)}u$: $(\gamma^{(x_4)}\gamma^{(x_4)}u)_0 = -v_{13} = -u_0$, row 0 of $(\gamma^{(x_4)})^2 = -I_{16}$. (The f-strings contain double quotes inside their braces, such as `signed(s1, "v", c1)`; Python allows this from version 3.12 on, one reason why the book needs Python 3.12 or newer.)

```
t1, d1 = follow(g0, 0)  # row 0 of gamma^(x0) gamma^(x4): first gamma^(x0) ...
t2, d2 = follow(g4, d1)  # ... then row d1 of gamma^(x4)
r1, e1 = follow(g4, 0)  # row 0 of gamma^(x4) gamma^(x0): first gamma^(x4) ...
r2, e2 = follow(g0, e1)  # ... then row e1 of gamma^(x0)
say(f"(gamma^(x0) gamma^(x4) u)_0 = {signed(t1 * t2, "u", d2)};  "
    f"(gamma^(x4) gamma^(x0) u)_0 = {signed(r1 * r2, "u", e2)}")
check(s1 * s2 == -1 and c2 == 0, "by hand: row 0 of (gamma^(x4))^2 is -u_0")
check(d2 == e2 and t1 * t2 == -(r1 * r2),
      "by hand: row 0 of gamma^(x0) gamma^(x4) is minus that of gamma^(x4) gamma^(x0)")
```

The same for the two orders of $\gamma^{(x_0)}$ and $\gamma^{(x_4)}$: row 0 of $\gamma^{(x_0)}$ points to component 8, and row 8 of $\gamma^{(x_4)}$ to $+u_5$, so $(\gamma^{(x_0)}\gamma^{(x_4)}u)_0 = +u_5$; row 0 of $\gamma^{(x_4)}$ points to $-$component 13, and row 13 of $\gamma^{(x_0)}$ to $+u_5$, so $(\gamma^{(x_4)}\gamma^{(x_0)}u)_0 = -u_5$. The two checks require the hand results: $-u_0$, and the same component with opposite signs.

In [23], figure 5.

```
fig, ax = plt.subplots(figsize=(5.4, 4.9), layout="constrained")
colours = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -2, 0, +2
ax.imshow(c, cmap=colours, norm=BoundaryNorm([-3, -1, 1, 3], 3))
for (a, b), value in np.ndenumerate(c):
    ax.text(b, a, f"{value:+d}" if value else "0", ha="center", va="center",
            color="white" if value else "#52514e", fontsize=10)
```

The 8×8 table c as a heat map with the boundaries $-3, -1, 1, 3$, so -2 is blue, 0 grey and $+2$ red; each number is written into its square (white on colour, dark grey on grey).

```
labels = [f"$x_{k}$" for k in range(8)]
ax.set_xticks(range(8), labels)
ax.set_yticks(range(8), labels)
ax.set_xlabel("direction $b$")
ax.set_ylabel("direction $a$")
ax.set_title("$\\gamma^a\\gamma^b + \\gamma^b\\gamma^a = c_{ab}\\, I_{16}$")
ax.grid(False)
save_figure(fig, "anticommutator_table",
            ...)
```

The rows and columns are labelled $x_0, \ldots, x_7$, the axes named, the title set, and the figure saved. **What figure 5 shows and why:** a diagonal of four red and four blue squares: $c_{ab} = 2\eta^{ab}$, red for the space-like x_0, x_1, x_2, x_3, blue for the time-like x_4 to x_7, and grey off the diagonal because different gammas anticommute. It is the Clifford relation in one picture.

In [24], reality, signed permutations and the symmetry pattern.

```
entries = sorted(set(int(x) for g in gamma for x in g.flatten()))
say(f"the different entries of the eight gamma matrices: {entries}")
rows_ok = all((np.count_nonzero(g, axis=1) == 1).all() for g in gamma)  # each row
columns_ok = all((np.count_nonzero(g, axis=0) == 1).all() for g in gamma)  # column
orthogonal = all((g.T @ g == I16).all() for g in gamma)
```

`g.flatten()` lists all 256 entries of a matrix; collecting them from all eight gammas in a set and sorting gives the different values: $[-1, 0, 1]$. `np.count_nonzero(g, axis=1)` counts the nonzero entries of each row (`axis=0`: of each column); each count must be 1. `orthogonal` tests $g^T g = I_{16}$.

```
check_record(entries == [-1, 0, 1], "every gamma^a is real with entries -1, 0, +1",
             (WOLFRAM, "reality"))
check_record(rows_ok and columns_ok and orthogonal,
             "every gamma^a is a signed permutation matrix, hence orthogonal",
             (WOLFRAM, "signed_permutation_matrices"),
             (PYTHON, "reality_signed_permutations"))
```

Two checks: reality (the entries are whole numbers $-1, 0, +1$, in particular real), and signed permutation matrices that are orthogonal.

```
symmetric = [COORDINATES[a] for a in range(8) if (gamma[a].T == gamma[a]).all()]
antisymmetric = [COORDINATES[a] for a in range(8) if (gamma[a].T == -gamma[a]).all()]
say("symmetric: " + ", ".join(symmetric) + ";  antisymmetric: "
    + ", ".join(antisymmetric))
check_record(symmetric == ["x0", "x1", "x2", "x3"]
             and antisymmetric == ["x4", "x5", "x6", "x7"]
             and all((gamma[a].T == eta[a] * gamma[a]).all() for a in range(8)),
             "(gamma^a)^T = eta_aa gamma^a: symmetric for x0, x1, x2, x3 only",
             (WOLFRAM, "symmetry_pattern"), (PYTHON, "symmetry_pattern"))
```

The lists of the symmetric and of the antisymmetric gammas are made and printed: x_0, x_1, x_2, x_3 and x_4, x_5, x_6, x_7. The check requires exactly these lists and $(\gamma^a)^T = \eta_{aa}\gamma^a$ for all eight: the symmetry pattern proved in Step 4 of Section 4.5.

In [25], figure 6.

```
fig, axes = plt.subplots(2, 2, figsize=(6.4, 6.6), layout="constrained")
draw_signs(axes[0, 0], gamma[1], "$\\gamma^{(x_1)}$ (space-like)")
draw_signs(axes[0, 1], gamma[1].T, "its transpose: the same")
draw_signs(axes[1, 0], gamma[4], "$\\gamma^{(x_4)}$ (time-like)")
draw_signs(axes[1, 1], gamma[4].T, "its transpose: all signs flipped")
sign_legend(fig)
save_figure(fig, "symmetry_pattern",
            ...)
```

Four panels: $\gamma^{(x_1)}$ and its transpose, $\gamma^{(x_4)}$ and its transpose. **What figure 6 shows and why:** the upper two pictures are identical (a symmetric matrix equals its transpose); in the lower two every red square has become blue and every blue square red at the same place, because the transpose of an antisymmetric matrix is minus the matrix.

In [26], comparison with the Revision record files.

```
def exact(entry):
    """An exact number of gammas.json: a JSON integer, or a text "p/q"."""
    if isinstance(entry, int):
        return Fraction(entry)
    if isinstance(entry, str) and "/" in entry:
        numerator, denominator = entry.split("/")
        return Fraction(int(numerator), int(denominator))
    raise ValueError(f"an entry of gammas.json that is not exact: {entry}")
```

The file `gammas.json` writes every number exactly: a whole number as a JSON integer, a fraction as a text such as `"1/2"`. `exact` reads both into an exact `Fraction`: `isinstance(entry, int)` tests whether the entry is a whole number; for a text containing `/`, `split("/")` cuts it into numerator and denominator. Anything else stops the notebook with an error, so no rounded number can slip in.

```
def differing(stored, ours):
    """How many entries of the stored matrix differ from our integer matrix."""
    return sum(1 for (i, j), x in np.ndenumerate(ours)
               if exact(stored[i][j]) != int(x))
```

`differing` counts the entries in which a stored matrix (a list of rows from the file) differs from the notebook's matrix: it runs through all positions (i, j) and adds 1 whenever the two numbers are not equal (`!=`).

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
python_gammas = json.loads(repository_file(
    "Revision/algebra/reports/python-gammas.json").read_text(encoding="utf-8"))
fixture_differ = [differing(fixture["gamma"][a], gamma[a]) for a in range(8)]
python_differ = [differing(python_gammas["gamma"][a], gamma[a]) for a in range(8)]
```

The two Revision files are read: `gammas.json`, written by the WolframScript program, and `python-gammas.json`, written by the independent Python program. For each of the eight gammas the number of differing entries is counted, for both files.

```
report("entries compared per file", 8 * 16 * 16)
report("differing entries, gammas.json, per x0..x7", fixture_differ)
report("differing entries, python-gammas.json, per x0..x7", python_differ)
```

Three RESULT lines: 2048 entries are compared per file, and the number of differing entries is 0 for every one of the eight gammas, in both files.

```
check_record(fixture["coordinates"] == COORDINATES and fixture["eta"] == list(eta)
             and fixture["notebookFrameIndex"] == FRAME_INDEX
             and sum(fixture_differ) == 0,
             "Revision/algebra/gammas.json holds exactly these eight gammas",
             (WOLFRAM, "fixture_round_trip"),
             (PYTHON, "fixture_comparison_gammas_json"))
python_map = python_gammas["map_coordinate_to_T16_index"]  # {"x0": 0, ...}
check(python_map == dict(zip(COORDINATES, FRAME_INDEX)) and sum(python_differ) == 0,
      "Revision/algebra/reports/python-gammas.json holds exactly these eight gammas")
```

The first check also compares the coordinate names, the signs and the coordinate map stored in `gammas.json` with the notebook's. The second compares the map stored in the Python file, a dictionary from coordinate names to indices, with the dictionary made from the notebook's two lists (`zip` pairs the two lists entry by entry, `dict` turns the pairs into a dictionary), and requires no differing entry.

In [27], the totals.

```
for report_file in (WOLFRAM, PYTHON):
    verdicts = [entry["verdict"].upper() for entry in RECORDED[report_file].values()]
    passed = verdicts.count("PASS")  # how many checks of the report passed
    mine = [name for f, name in REPRODUCED if f == report_file]
    report(f"{report_file} passed", f"{passed} of {len(verdicts)}")
    report("    of these checks, reproduced in this notebook", len(mine))
check(len(REPRODUCED) == 30, "this notebook reproduces 30 checks of the two reports")
```

For each report, the verdicts of all checks are collected (`.values()` gives the entries of a dictionary), and `count("PASS")` counts the passed ones; `mine` lists the checks of this report that the notebook reproduced. The output: the WolframScript report passed 45 of 45, of which this notebook reproduced 16; the Python report passed 35 of 35, of which it reproduced 14. The check requires 30 reproduced Revision checks in all.

In [28], the last check.

```
names = ["04a_1_blocks_s4_t4.png", "04a_2_tau_matrices.png",
         "04a_3_block_form_of_t16.png", "04a_4_gammas_x0_to_x7.png",
         "04a_5_anticommutator_table.png", "04a_6_symmetry_pattern.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all six figure files exist")
all_checks_passed()
```

The check confirms that the six figure files were written, and `all_checks_passed()` prints the last line, ALL 28 CHECKS PASSED (notebook 04a): one check in each of In [2], In [6], In [9], In [14], In [17], In [20], In [21], In [27] and In [28]; two in each of In [8], In [10], In [22] and In [26]; three in each of In [15] and In [24]; and five in In [13].

4.9 Square roots in 4+4 dimensions and the plane waves of the field

Section 4.3 showed that the Clifford relation is the same statement as the square-root rule. For the author's gammas, whose Clifford relation Section 4.5 proved, the rule reads

$$\Big(\sum_{a=0}^{7} p_a \gamma^{(x_a)}\Big)^2 = (p_0^2 + p_1^2 + p_2^2 + p_3^2 - p_4^2 - p_5^2 - p_6^2 - p_7^2)\, I_{16}$$

for all numbers $p_0, \ldots, p_7$: the author's eight matrices take the square root of the quadratic form of his space-time, with a plus sign for the hidden direction and 3-space and a minus sign for the time and the three extra times. (Status: PROVED; Notebook 04b confirms it with sympy for all values of the p_a, and with floating-point numbers for 300 random vectors.) This section uses the rule to find what the field equation says about waves, and what a momentum along an extra time does.

Complex numbers and the conjugate transpose. A **complex number** is $x + iy$ with real x, y and $i^2 = -1$; its **complex conjugate** is $x - iy$. The **conjugate transpose** $M^\dagger$ of a matrix transposes it and conjugates every entry. A matrix is **Hermitian** if $M^\dagger = M$; a real symmetric matrix is Hermitian. An **eigenvalue** of M is a number λ for which a nonzero column u (an **eigenvector**) has $Mu = \lambda u$. A Hermitian matrix has only real eigenvalues. Proof: from $Mu = \lambda u$ follows $u^\dagger M u = \lambda\, u^\dagger u$ (multiply from the left by the row $u^\dagger$); the number $u^\dagger M u$ equals its own complex conjugate, because its conjugate is $u^\dagger M^\dagger u = u^\dagger M u$, so it is real; and $u^\dagger u = \sum_i |u_i|^2$ is real and positive; so λ is a real number divided by a positive one. We also use a fact that Section 1.34 proved for **every** square matrix, Hermitian or not: an $n \times n$ matrix M has n eigenvalues, the n roots of its **characteristic polynomial** $\det(M - \lambda I_n)$, where a root that occurs several times is counted as often as it occurs (its **multiplicity**); and the sum of these n eigenvalues is the trace of M. (The proof there compares two ways of writing the characteristic polynomial; it rests on the fundamental theorem of algebra, which Chapter 1 assumes.) Finally, the **smallest singular value** of a square matrix M is the smallest length of Mu over all columns u of length 1; it is 0 exactly when some nonzero u has $Mu = 0$. numpy computes it, and Notebook 04b uses it to locate the energies at which waves exist.

The model: flat 4+4 space. The full field equation of the theory, $\gamma^\mu D_\mu \Psi = (m + U'(S))\Psi$, is derived in Chapter 7; it contains the frame factors of Section 4.4, the gravitational correction in D_μ, and a self-interaction $U'(S)$. This section studies the simplest equation with the same algebra,

$$\sum_{a=0}^{7} \gamma^{(x_a)}\, \partial_a \Psi = m\Psi,$$

in **flat 4+4 space**, where the metric is the constant η: no gravity, no inflation, no deflation and no self-interaction (∂_a is the derivative with respect to x_a). This is a model for the algebra only (status ASSUMED for this section): in the author's space-time the extra times deflate at every moment. What carries over is the algebra of the frame: in a region so small that the metric functions change

little across it, waves with **frame momenta** k_a obey the same matrix algebra; the Revision record uses this local plane-wave reading with the coefficients of the equation held fixed.

Plane waves, line by line. A **plane wave** is $\Psi = u\, e^{i(k_0x_0+k_1x_1+k_2x_2+k_3x_3+k_5x_5+k_6x_6+k_7x_7-Ex_4)}$ with a constant column u; E is its **energy** (its frequency in the time x_4) and the k_a are its **momenta**.

1. $\partial_a\Psi = ik_a\Psi$ for $a \neq 4$ and $\partial_4\Psi = -iE\Psi$ — the derivative of e^{cx} is $c\,e^{cx}$.
2. $-iE\gamma^{(x_4)}u + i\sum_{a\neq4} k_a\gamma^{(x_a)}u = mu$ — line 1 inserted into the equation, and both sides divided by the exponential, which is never zero.
3. $iEu + i\sum_{a\neq4} k_a\gamma^{(x_4)}\gamma^{(x_a)}u = m\gamma^{(x_4)}u$ — both sides multiplied from the left by $\gamma^{(x_4)}$, and $(\gamma^{(x_4)})^2 = -I_{16}$ (x_4 is time-like).
4. $Eu + \sum_{a\neq4} k_a\gamma^{(x_4)}\gamma^{(x_a)}u = -im\gamma^{(x_4)}u$ — both sides multiplied by $-i$, with $-i\cdot i = 1$.
5. $Eu = hu$ with $h = -im\gamma^{(x_4)} - \sum_{a\neq4} k_a\gamma^{(x_4)}\gamma^{(x_a)}$ — the sum moved to the right-hand side.

So a plane wave exists exactly when E is an eigenvalue of the 16×16 matrix h. Write $h = m\beta + \sum_{a\neq4} k_a\alpha_a$ with $\beta = -i\gamma^{(x_4)}$ and $\alpha_a = -\gamma^{(x_4)}\gamma^{(x_a)}$, and abbreviate $g_4 = \gamma^{(x_4)}$, $g_a = \gamma^{(x_a)}$. From the Clifford relation ($g_4^2 = -I$, $g_a^2 = \eta^{aa}I$, different gammas anticommute):

- $\beta^2 = (-i)^2 g_4^2 = (-1)(-1)I = I$ — numbers move to the front;
- $\alpha_a^2 = g_4g_ag_4g_a = -g_4g_4g_ag_a = -(-1)\eta^{aa}I = \eta^{aa}I$ — one exchange of the neighbours g_ag_4 gives the factor -1;
- $\beta\alpha_a = ig_4g_4g_a = -ig_a$ and $\alpha_a\beta = ig_4g_ag_4 = -ig_4g_4g_a = ig_a$ — so $\beta\alpha_a + \alpha_a\beta = 0$;
- for $a \neq b$: $\alpha_a\alpha_b = g_4g_ag_4g_b = -g_4g_4g_ag_b = g_ag_b$ and $\alpha_b\alpha_a = g_bg_a = -g_ag_b$ — so they anticommute.

So β and the seven α_a obey a Clifford relation with the signs $+1$ for β and η^{aa} for α_a, and the general rule of Section 4.3, applied with the numbers m and k_a, gives

$$h^2 = (m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2)\, I_{16}.$$

If $hu = Eu$ with $u \neq 0$, then $h^2u = E^2u$, so

$$E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2,$$

the **mass shell** of the flat 4+4 equation. Status: PROVED (here, and exactly with sympy in Notebook 04b); it reproduces `Revision/theory/reports/python-field-theory.json`, check `mode_hamiltonian_B_selfadjoint_dispersion` (that check also verifies a statement about a matrix B, which Chapter 10 needs and this chapter does not).

Without momentum along the extra times. When $k_5 = k_6 = k_7 = 0$ (the Revision record calls this the **good sector**), h is Hermitian, so its eigenvalues are real. Proof: the gammas are real with $(g_a)^T = \eta_{aa}g_a$ (Section 4.5). Then $\beta^\dagger = ig_4^T = -ig_4 = \beta$ (conjugating $-i$ gives i, and g_4

is antisymmetric). α_a is real with $\alpha_a^T = -g_a^T g_4^T = -(\eta_{aa} g_a)(-g_4) = \eta_{aa} g_a g_4 = -\eta_{aa} g_4 g_a = \eta_{aa}\alpha_a$: symmetric, hence Hermitian, for the space-like x_0, x_1, x_2, x_3 (and antisymmetric for the extra times, which is why they are left out here). With real momenta, h is then a sum of Hermitian matrices. The **example of the Revision record**: $m = 2$, $k_0 = 4$ and $(k_1, k_2, k_3) = (1, 2, 0)$ give $E^2 = 4 + 16 + 1 + 4 + 0 = 25$, so every eigenvalue is $+5$ or -5. How often each occurs follows from the trace: $\mathrm{tr}\, g_4 = 0$ (a block off-diagonal matrix has zeros on its diagonal) and $\mathrm{tr}(g_4 g_a) = -\mathrm{tr}(g_a g_4) = -\mathrm{tr}(g_4 g_a)$ (anticommuting, then the cyclic property), so $\mathrm{tr}(g_4 g_a) = 0$ and $\mathrm{tr}\, h = 0$. With n_+ eigenvalues $+5$ and n_- eigenvalues -5: $n_+ + n_- = 16$ and $5n_+ - 5n_- = \mathrm{tr}\, h = 0$ (the sum of the eigenvalues is the trace), so $n_+ = n_- = 8$: the energies $+5$ and -5, eight times each. Status: PROVED (this derivation) and COMPUTED (Notebook 04b finds the 16 eigenvalues numerically with deviations below 10^{-12}, and eight independent solutions u at each of the two energies). The Revision record `Revision/theory/reports/python-field-theory.json`, check `good_sector_spectrum_and_B_sectors`, states this result for all sixteen components in the words `h is Hermitian on the full 16-dim space with energies +-sqrt(m^2 + k0^2 + k1^2 + k2^2 + k3^2) (8 each)`, and Notebook 04b reproduces these words. The same check states its exact example in one half of the sixteen components, the subject of the next paragraph.

The record's exact example: the sector $B = +1$. The record uses the matrix $B = -i\,C g_4$, where $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ is the product of the four space-like gammas; the record file `Revision/algebra/gammas.json` stores both. Section 1.37 met B: it is Hermitian, $B^2 = I_{16}$, and its eigenvalues $+1$ and -1 occur eight times each (Chapter 5 studies C and B in full). Two rules move a gamma g_a through C, a product of four different gammas: if g_a is not one of the four factors, it anticommutes with each of them, so $g_a C = (-1)^4 C g_a = C g_a$; if g_a is one of them, it anticommutes with the other three and commutes with itself, so $g_a C = (-1)^3 C g_a = -C g_a$. In particular $g_4 C = C g_4$. Now, line by line:

1. $B\beta = (-i\,C g_4)(-i\,g_4) = -C g_4 g_4 = C$ — the numbers moved to the front, $(-i)^2 = -1$, and $g_4^2 = -I$.
2. $\beta B = (-i\,g_4)(-i\,C g_4) = -g_4 C g_4 = -C g_4 g_4 = C$ — the same steps and $g_4 C = C g_4$; so $B\beta = \beta B$.
3. $B\alpha_a = (-i\,C g_4)(-g_4 g_a) = i\,C g_4 g_4 g_a = -i\,C g_a$ — the numbers moved to the front, and $g_4^2 = -I$.
4. $\alpha_a B = (-g_4 g_a)(-i\,C g_4) = i\,g_4 g_a C g_4$ — the numbers moved to the front.
5. For the space-like $a = x_0, x_1, x_2, x_3$, which are factors of C: $\alpha_a B = -i\,g_4 C g_a g_4 = -i\,C g_4 g_a g_4 = -i\,C g_a = B\alpha_a$ — line 4 with $g_a C = -C g_a$, then $g_4 C = C g_4$, then $g_4 g_a g_4 = -g_a g_4 g_4 = g_a$, then line 3.
6. For the extra times $a = x_5, x_6, x_7$, which are not factors of C: $\alpha_a B = +i\,C g_4 g_a g_4 = i\,C g_a = -B\alpha_a$ — the same steps with $g_a C = +C g_a$.

So without extra-time momentum $h = m\beta + \sum_a k_a \alpha_a$ (with $a = x_0, x_1, x_2, x_3$) **commutes** with B: $Bh = hB$. A momentum along an extra time adds an α_a that **anticommutes** with B. The record states both, in the words `[B, h] = 0` and `B anticommutes with gamma^(4) gamma^(x5,x6,x7)` (the record writes `gamma^(4)` for $\gamma^{(x_4)}$, and $\gamma^{(x_4)}\gamma^{(x_a)} = -\alpha_a$). Call the columns u with $Bu = u$ the **sector** $B = +1$. If $Bu = u$, then $B(hu) = h(Bu) = hu$: without extra-time momentum, h maps the

sector into itself. Notebook 04b takes eight **orthonormal** columns of the sector (columns of length 1 with $u^\dagger v = 0$ for two different ones; numpy finds them as eigenvectors of B) as the columns of a 16×8 matrix V, and checks $VV^\dagger = P$ with $P = \frac{1}{2}(I_{16} + B)$, the matrix that keeps the $B = +1$ part of a column. Inside the sector h acts as the 8×8 matrix $h_+ = V^\dagger h V$ (that is, $hV = Vh_+$), and $h_+^2 = V^\dagger h V V^\dagger h V = V^\dagger h P h V = V^\dagger h h P V = 25\, V^\dagger V = 25\, I_8$, because $hP = Ph$, $PV = V$ and $V^\dagger V = I_8$. Its trace is $\operatorname{tr} h_+ = \operatorname{tr}(V^\dagger h V) = \operatorname{tr}(hVV^\dagger) = \operatorname{tr}(hP)$ (the cyclic property; its proof in Section 4.2 works unchanged for an 8×16 and a 16×8 factor), and $\operatorname{tr}(hP) = \frac{1}{2}(\operatorname{tr} h + \operatorname{tr}(hB)) = 0$: $\operatorname{tr} h = 0$ was shown above, and $hB = m\,\beta B + \sum_a k_a \alpha_a B = m\,C - i \sum_a k_a C g_a$ (lines 2, 3 and 5) is a combination of C, a product of four different gammas, and of Cg_a, which is $\pm$ a product of three different gammas, all of trace 0 by the trace lemma of Section 4.13. With n_+ eigenvalues $+5$ and n_- eigenvalues -5 of h_+: $n_+ + n_- = 8$ and $5n_+ - 5n_- = 0$, so the energies $+5$ and -5 occur **four times each** in the sector $B = +1$. Status: PROVED (by hand above; Notebook 04b confirms $Bh = hB$, the anticommutation for the extra times and $\operatorname{tr}(hP) = 0$ exactly with sympy, for all values of m, k_0, k_1, k_2, k_3) and COMPUTED (the eight eigenvalues of h_+ lie within 10^{-12} of ± 5, four of each); it reproduces `Revision/theory/reports/python-field-theory.json`, check `good_sector_spectrum_and_B_sectors`, whose words for its exact example are `m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0) on B = +1: +5 (4 times), -5 (4 times)`.

A momentum along an extra time. The extra times enter E^2 with a minus sign. Adding a momentum k_5 along x_5 to the example gives $E^2 = 25 - k_5^2$. For $k_5 < 5$ the energies $\pm\sqrt{25 - k_5^2}$ are real. For $k_5 > 5$ they are imaginary, $E = \pm i\kappa$ with $\kappa = \sqrt{k_5^2 - 25}$, and the factor e^{-iEx_4} of the wave becomes, for $E = i\kappa$, $e^{-i \cdot i\kappa\, x_4} = e^{\kappa x_4}$ (because $-i \cdot i = 1$): the wave **grows exponentially** in the time instead of oscillating, and the one with $E = -i\kappa$ shrinks. In general the growth rate is $\kappa = \sqrt{k_5^2 - m^2 - k_0^2 - k_1^2 - k_2^2 - k_3^2}$ whenever this is real; it has no upper bound as k_5 grows. Status: PROVED (the formula) and COMPUTED (Notebook 04b checks all 16 eigenvalues for 160 values of k_5 to 10^{-10}); it reproduces `Revision/theory/reports/python-scope.json`, check `extra_time_growth_rates_unbounded`. A problem of the kind "given the field at one time, find it at later times" is called **well posed** (in the sense of the mathematician Jacques Hadamard) when a solution exists, is unique, and changes only a little when the given data change a little. The Revision record draws from the missing upper bound the conclusion that the first-order equation is not well posed in this sense for data that depend on the extra times (`Revision/theory/reports/python-scope.json`, check `extra_time_growth_rates_unbounded`): a wave with a tiny amplitude but a large momentum k_5 grows like $e^{\kappa x_4}$ with an arbitrarily large κ, so arbitrarily small changes of the data cause large changes after any fixed time. Chapter 8 explains this. The record's smallest exact example is $m = 1$ with only $k_5 = 2$: $h^2 = (1 - 4)I_{16} = -3I_{16}$, so every eigenvalue has $E^2 = -3$, $E = +i\sqrt{3}$ or $-i\sqrt{3}$, eight times each (the trace argument again: $\operatorname{tr} h = 0$ also with the momentum k_5, and the sum of the eigenvalues is the trace for every square matrix, also for this h, which is not Hermitian), and the wave with $E = i\sqrt{3}$ grows like $e^{\sqrt{3}\, x_4}$. Status: PROVED (exactly, with sympy, in Notebook 04b); it reproduces `Revision/theory/reports/python-field-theory.json`, check `extra_time_modes_grow`.

Why the deflation matters here. In the author's space-time a wave with a fixed **coordinate** momentum k_{x_5} along x_5, a factor $e^{ik_{x_5}x_5}$, has a **frame** momentum that grows: the derivative along the frame direction is the coordinate derivative divided by the frame factor $b_4^{1/2} e^{-a_4} \sin^{1/6} z$ of x_5 (Section 4.4), so the frame momentum is $b_4^{-1/2} e^{a_4} \sin^{-1/6}(z)\, k_{x_5}$. As the extra times deflate (a_4 grows, b_4 constant; the record states it for $b_4 = 1$), this frame momentum grows like e^{a_4}. In the local plane-wave reading that the same Revision check `extra_time_modes_grow` states, every wave

along an extra time therefore reaches, sooner or later, the region where its energy is imaginary. Notebook 04b computes only the flat algebra behind this statement; the waves in the curved, deflating space-time are the subject of Chapter 8.

The next three sections hold Notebook 04b.

4.10 How to run Notebook 04b

Step 1. What this notebook does and what it needs.

Notebook 04b (Square roots of quadratic forms: from Pauli and Dirac to the author's gammas) is the file `Revision/textbook/notebooks/04b_square_roots.ipynb` of the repository Dirac_claude. It shows why the gamma matrices exist: a sum of squares with plus and minus signs has a linear square root only when the coefficients are anticommuting matrices. It checks this exactly for two 2 by 2 examples, the Pauli matrices, Dirac's 4 by 4 matrices and the author's eight 16 by 16 gammas, derives the plane-wave relation of the first-order equation in flat 4+4 dimensions, shows that a momentum along an extra time makes the energies imaginary (growing waves), reproduces the Revision record of these relations and of their exact examples, and draws six figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04b_square_roots.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04b_square_roots.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 04b_square_roots.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/04b.captions.json`
- `Revision/textbook/figures/04b_1_square_roots_2x2.png`
- `Revision/textbook/figures/04b_2_pauli_dirac_tables.png`
- `Revision/textbook/figures/04b_3_dirac_mass_shell.png`
- `Revision/textbook/figures/04b_4_square_root_8d.png`
- `Revision/textbook/figures/04b_5_mass_shell_4p4.png`
- `Revision/textbook/figures/04b_6_extra_time_momentum.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 19 CHECKS PASSED (notebook 04b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/04b_square_roots.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 04b_square_roots.ipynb
python -m nbconvert --execute --inplace 04b_square_roots.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `Revision/algebra/gammas.json` or `Revision/theory/reports/python-field-theory.json`: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run `git status` in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

```
git checkout HEAD Revision/algebra Revision/theory
```

4.11 Notebook 04b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/04b_square_roots.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The length of a vector is the square root of a sum of squares. Paul Dirac found in
1928 that such a square root can be written as a *linear* expression, provided the
coefficients are matrices that anticommute. This is the reason why gamma matrices
exist, and why the field of this book has 16 components. This notebook

- shows that ordinary numbers cannot do it;
- checks exactly (with sympy, for all values of the variables) the square roots of
  $p^2 + q^2$ and of $p^2 - q^2$ made from real $2 \times 2$ matrices, the three
  Pauli matrices (three directions), and Dirac's $4 \times 4$ matrices (time and
  three space directions);
- checks exactly that the author's eight $16 \times 16$ gammas give the square root
  of the quadratic form of his 4+4 dimensional space-time:
  $(\sum_a p_a\gamma^{(x_a)})^2 = (p_0^2 + p_1^2 + p_2^2 + p_3^2 - p_4^2 - p_5^2
  - p_6^2 - p_7^2)\, I_{16}$;
- derives, line by line, what the first-order field equation
  $\sum_a \gamma^{(x_a)} \partial_a \Psi = m\Psi$ says about plane waves in flat
  4+4 dimensions: $E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$,
  and reproduces the Revision record of this relation and of its exact example;
- shows that a momentum along an extra time enters $E^2$ with a minus sign and
  makes the energies imaginary (waves that grow in time) when it is large enough,
  and reproduces the Revision record of this growth;
- draws six figures.
```

Text cell:

```
## 2. How to run this notebook
```

Step 1. What this notebook does and what it needs.

Notebook 04b (Square roots of quadratic forms: from Pauli and Dirac to the author's gammas) is the file ‘Revision/textbook/notebooks/04b_square_roots.ipynb‘ of the repository Dirac_claude. It shows why the gamma matrices exist: a sum of squares with plus and minus signs has a linear square root only when the coefficients are anticommuting matrices. It checks this exactly for two 2 by 2 examples, the Pauli matrices, Dirac's 4 by 4 matrices and the author's eight 16 by 16 gammas, derives the plane-wave relation of the first-order equation in flat 4+4 dimensions, shows that a momentum along an extra time makes the energies imaginary (growing waves), reproduces the Revision record of these relations and of their exact examples, and draws six figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04b_square_roots.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04b_square_roots.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 04b_square_roots.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/04b.captions.json‘
- ‘Revision/textbook/figures/04b_1_square_roots_2x2.png‘
- ‘Revision/textbook/figures/04b_2_pauli_dirac_tables.png‘
- ‘Revision/textbook/figures/04b_3_dirac_mass_shell.png‘
- ‘Revision/textbook/figures/04b_4_square_root_8d.png‘
- ‘Revision/textbook/figures/04b_5_mass_shell_4p4.png‘
- ‘Revision/textbook/figures/04b_6_extra_time_momentum.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 19 CHECKS PASSED (notebook 04b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from

  a copy outside the repository. Open the file
  ‘Revision/textbook/notebooks/04b_square_roots.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 04b_square_roots.ipynb
      python -m nbconvert --execute --inplace 04b_square_roots.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘Revision/algebra/gammas.json‘ or
  ‘Revision/theory/reports/python-field-theory.json‘: the notebook reads these Revision
  record files of the repository; they are part of every complete clone. Run ‘git status‘
  in the repository folder: if it reports them as deleted, restore them with the command
  below and run the notebook again.

      git checkout HEAD Revision/algebra Revision/theory

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 04b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 04b (Square roots of quadratic forms: from Pauli and Dirac to the author’s
# gammas) is the file Revision/textbook/notebooks/04b_square_roots.ipynb of the
# repository Dirac_claude. It shows why the gamma matrices exist: a sum of squares with
# plus and minus signs has a linear square root only when the coefficients are
```

```
# anticommuting matrices. It checks this exactly for two 2 by 2 examples, the Pauli
# matrices, Dirac's 4 by 4 matrices and the author's eight 16 by 16 gammas, derives the
# plane-wave relation of the first-order equation in flat 4+4 dimensions, shows that a
# momentum along an extra time makes the energies imaginary (growing waves), reproduces
# the Revision record of these relations and of their exact examples, and draws six
# figures. It needs a computer with Windows 11, macOS or Linux, an internet connection
# for the installation, the program Git, and Python 3.12 or newer (the notebooks were
# built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6,
# sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
# nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports
# numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and
# runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
```

```
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
```

```
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 04b_square_roots.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 04b_square_roots.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 04b_square_roots.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/04b.captions.json
```

```
# - Revision/textbook/figures/04b_1_square_roots_2x2.png
# - Revision/textbook/figures/04b_2_pauli_dirac_tables.png
# - Revision/textbook/figures/04b_3_dirac_mass_shell.png
# - Revision/textbook/figures/04b_4_square_root_8d.png
# - Revision/textbook/figures/04b_5_mass_shell_4p4.png
# - Revision/textbook/figures/04b_6_extra_time_momentum.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 19 CHECKS PASSED (notebook 04b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/04b_square_roots.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 04b_square_roots.ipynb
#     python -m nbconvert --execute --inplace 04b_square_roots.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
```

```
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json or
#   Revision/theory/reports/python-field-theory.json: the notebook reads these Revision
#   record files of the repository; they are part of every complete clone. Run git status
#   in the repository folder: if it reports them as deleted, restore them with the
#   command below and run the notebook again.
#     git checkout HEAD Revision/algebra Revision/theory
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "04b"  # this notebook: chapter 04, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 04b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Quadratic form**: a sum of squares of variables, each with a plus or a minus
  sign, such as $p^2 + q^2$ (Pythagoras) or $p^2 - q^2$.
- **Square root of a quadratic form**: an expression $L$, linear in the variables,
  whose square is the quadratic form: $L^2 = p^2 + q^2$ (times the identity matrix
  when $L$ is a matrix).
- **Complex number**: $x + iy$ with real $x, y$ and $i^2 = -1$. A matrix may have
  complex entries.
- **Conjugate transpose** $M^\dagger$: transpose $M$ and replace every entry
  $x + iy$ by $x - iy$. A matrix is **Hermitian** if $M^\dagger = M$.
- **Anticommute**: $AB = -BA$, that is $\{A, B\} = AB + BA = 0$.
- **Clifford relation**: $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I$, with
  $\eta$ a diagonal matrix of signs $\pm 1$ (the *metric* of the directions).
- **Eigenvalue and eigenvector**: a number $\lambda$ and a nonzero column $u$ with
  $M u = \lambda u$.
- **Multiplicity, trace**: an $n \times n$ matrix $M$ has $n$ eigenvalues, the $n$
  roots of the polynomial $\det(M - \lambda I)$ (its *characteristic polynomial*);
```

```
  a root that occurs several times is counted with its *multiplicity*. The *trace*
  $\mathrm{tr}\, M$, the sum of the diagonal entries, equals the sum of these $n$
  eigenvalues, for every square matrix.
- **Orthonormal columns**: columns of length 1 with $u^\dagger v = 0$ for any two
  different ones.
- **Sector** of a Hermitian matrix $B$ with $B^2 = I$: the columns $u$ with
  $Bu = u$ (the sector $B = +1$), or those with $Bu = -u$ (the sector $B = -1$).
- **Smallest singular value** of a square matrix $M$: the smallest length of $Mu$
  over all columns $u$ of length 1. It is 0 exactly when some nonzero $u$ has
  $Mu = 0$. numpy computes it with `np.linalg.svd`.
- **Plane wave**: a solution of the form $u\, e^{i(k \cdot x - E x_4)}$, with a
  constant column $u$; $E$ is its *energy* (frequency in the time $x_4$) and the
  $k_a$ are its *momenta* (wave numbers in the other directions).
- **Mass shell** (or *dispersion relation*): the relation between $E$, the $k_a$
  and the mass $m$ that a plane wave must obey.
- **Flat 4+4 space**: space-time with the constant metric
  $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order
  $x_0, \dots, x_7$: no gravity, no inflation, no deflation.
- **sympy, symbol**: sympy computes with letters (symbols) such as $p$ and $q$ that
  stand for any number; an identity that sympy proves holds for every value.
- **Floating-point number**: a number stored with about 16 significant digits;
  numpy computes with them, so its results may differ from the exact ones in the
  last digits (rounding). **Random numbers with a seed**: numbers that look random
  but are the same in every run, because the generator starts from a fixed number.
```

Text cell:

```
## 4. The physical and mathematical situation

**The square-root problem.** Can $\sqrt{p^2 + q^2}$ be written as
$\alpha p + \beta q$? Squaring,
$(\alpha p + \beta q)^2 = \alpha^2 p^2 + (\alpha\beta + \beta\alpha) pq + \beta^2 q^2$
(multiply out; $\alpha\beta$ and $\beta\alpha$ are kept apart because for matrices
the order matters). This equals $p^2 + q^2$ for all $p, q$ exactly when
$\alpha^2 = 1$, $\beta^2 = 1$ and $\alpha\beta + \beta\alpha = 0$. For numbers,
$\alpha\beta + \beta\alpha = 2\alpha\beta = 0$ forces $\alpha = 0$ or $\beta = 0$,
which contradicts $\alpha^2 = \beta^2 = 1$. For matrices it is possible.

**The general rule.** For $n$ directions with signs $\eta^{aa} = \pm 1$ we want
$(\sum_a p_a \gamma^a)^2 = (\sum_a \eta^{aa} p_a^2) I$ for all numbers $p_a$.
Multiplying out, $(\sum_a p_a\gamma^a)^2 = \sum_{a,b} p_a p_b \gamma^a\gamma^b$; the
terms $(a, b)$ and $(b, a)$ have the same factor $p_a p_b$, so the sum equals
$\tfrac12 \sum_{a,b} p_a p_b (\gamma^a\gamma^b + \gamma^b\gamma^a)$. This is
$\sum_a \eta^{aa} p_a^2 I$ for all $p$ exactly when the Clifford relation
$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I$ holds.

**Why physics needs it.** In special relativity a particle of mass $m$, energy $E$
and momentum $k$ obeys $E^2 = m^2 + k_1^2 + k_2^2 + k_3^2$. Dirac wanted a
first-order equation (only first derivatives) whose square is this relation; he
needed four anticommuting matrices, one with square $+1$ and three with square $-1$
in his sign convention, and found $4 \times 4$ ones.

**The author's 4+4 space-time.** It has eight directions: 3-space $x_1, x_2, x_3$
```

```
(space-like), the time $x_4$ and the three extra times $x_5, x_6, x_7$ (time-like;
in the author's metric they deflate exponentially, with the scale factor
$e^{-a_4}\sin^{1/6} z$), and the hidden space direction $x_0$. In a local frame the
signs are $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$, and the author's
eight real $16 \times 16$ matrices satisfy the Clifford relation with this $\eta$.
The field equation of the theory is $\gamma^\mu D_\mu\Psi = (m + U'(S))\Psi$, in which
$\gamma^\mu$ is $\gamma^{(x_\mu)}$ divided by the frame factor
$\sqrt{|g_{\mu\mu}|}$ of the direction $x_\mu$, $D_\mu$ is the derivative along
$x_\mu$ corrected by the gravitational field, and $U'(S)$ comes from the
self-interaction.

**What this notebook does with it.** It studies the equation
$\sum_a \gamma^{(x_a)} \partial_a\Psi = m\Psi$ in **flat** 4+4 space, where the
metric is the constant $\eta$: no gravity, no inflation and no deflation. It is a
model for the algebra only: in the author's space-time the extra times deflate at
every moment, and this notebook does not claim otherwise. What carries over is the
algebra of the frame: in a region so small that the metric functions change little
across it, plane waves with *frame momenta* $k_a$ obey the same matrix algebra (the
Revision record treats this local plane-wave analysis with the coefficients of the
equation held fixed). Nothing in this notebook depends on the curvature.
```

Text cell:

```
## 5. Numbers are not enough

The next cell asks sympy for all numbers $\alpha, \beta$ (real or complex) with
$\alpha^2 = 1$, $\beta^2 = 1$ and $\alpha\beta + \beta\alpha = 2\alpha\beta = 0$.
‘sp.solve‘ returns the list of all solutions; it must be empty.
```

In [2]:

```
import numpy as np  # floating-point arrays and linear algebra
import sympy as sp  # exact algebra with symbols

alpha, beta = sp.symbols("alpha beta")  # two unknown numbers
equations = [alpha**2 - 1, beta**2 - 1, 2 * alpha * beta]  # each must equal 0
solutions = sp.solve(equations, [alpha, beta], dict=True)  # ** is a power
say(f"number solutions of alpha^2 = 1, beta^2 = 1, 2 alpha beta = 0: {solutions}")
check(solutions == [], "no two numbers solve the square-root problem")
```

Out [2]:

```
number solutions of alpha^2 = 1, beta^2 = 1, 2 alpha beta = 0: []
PASS no two numbers solve the square-root problem
```

Text cell:

```
## 6. Two real $2 \times 2$ matrices
```

```
The matrices
$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and
$\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ square to $I_2$ and
anticommute. So $(p\sigma_x + q\sigma_z)^2 = (p^2 + q^2) I_2$. A real matrix can also
square to $-I_2$: $N = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ has
$N^2 = -I_2$, and it anticommutes with $\sigma_x$; so
$(p\sigma_x + qN)^2 = (p^2 - q^2) I_2$, a square root of a *difference* of squares,
the kind that space and time need. The next cell checks both identities exactly
with the symbols $p$ and $q$, and prints the worked example $p = 3$, $q = 4$:
$3\sigma_x + 4\sigma_z = \begin{pmatrix} 4 & 3 \\ 3 & -4 \end{pmatrix}$, whose square
is $\begin{pmatrix} 16 + 9 & 12 - 12 \\ 12 - 12 & 9 + 16 \end{pmatrix} = 25 I_2$.
```

In [3]:

```
sigma_x = sp.Matrix([[0, 1], [1, 0]])
sigma_z = sp.Matrix([[1, 0], [0, -1]])
N = sp.Matrix([[0, 1], [-1, 0]])
I2 = sp.eye(2)  # the 2 x 2 identity matrix
p, q = sp.symbols("p q", real=True)  # two symbols that stand for any real numbers

euclidean = ((p * sigma_x + q * sigma_z) ** 2).expand()  # multiplied out
indefinite = ((p * sigma_x + q * N) ** 2).expand()
say(f"(p sigma_x + q sigma_z)^2 = {euclidean.tolist()}")
say(f"(p sigma_x + q N)^2 = {indefinite.tolist()}")
example = 3 * sigma_x + 4 * sigma_z
say(f"3 sigma_x + 4 sigma_z = {example.tolist()}, its square = "
    f"{(example * example).tolist()}")
check(euclidean == (p**2 + q**2) * I2 and (example * example) == 25 * I2,
      "(p sigma_x + q sigma_z)^2 = (p^2 + q^2) I2 for all p, q")
check(indefinite == (p**2 - q**2) * I2,
      "(p sigma_x + q N)^2 = (p^2 - q^2) I2 for all p, q")
```

Out [3]:

```
(p sigma_x + q sigma_z)^2 = [[p**2 + q**2, 0], [0, p**2 + q**2]]
(p sigma_x + q N)^2 = [[p**2 - q**2, 0], [0, p**2 - q**2]]
3 sigma_x + 4 sigma_z = [[4, 3], [3, -4]], its square = [[25, 0], [0, 25]]
PASS (p sigma_x + q sigma_z)^2 = (p^2 + q^2) I2 for all p, q
PASS (p sigma_x + q N)^2 = (p^2 - q^2) I2 for all p, q
```

Text cell:

```
A square root of a matrix multiple $c I_2$ has the eigenvalues $+\sqrt{c}$ and
$-\sqrt{c}$ (if $Mu = \lambda u$ then $M^2 u = \lambda^2 u = c\, u$, so
$\lambda^2 = c$). For $p^2 - q^2 < 0$ these are imaginary. The next cell computes
the eigenvalues numerically for $q = 1$ and 201 values of $p$ from $-3$ to $3$, checks
them against $\pm\sqrt{p^2 + 1}$ and $\pm\sqrt{p^2 - 1}$, and draws them: on the
left the Euclidean root (always real), on the right the real and imaginary parts of
the eigenvalues of the indefinite root (real for $|p| > 1$, imaginary for
$|p| < 1$).
```

In [4]:

```
p_values = np.linspace(-3.0, 3.0, 201)  # 201 equally spaced values of p
sx = np.array(sigma_x, dtype=float)  # the same matrices as numpy arrays
sz = np.array(sigma_z, dtype=float)
n_matrix = np.array(N, dtype=float)
euclid_eigen = np.array([np.sort(np.linalg.eigvals(pv * sx + sz).real)
                         for pv in p_values])  # q = 1
indef_eigen = np.array([np.linalg.eigvals(pv * sx + n_matrix) for pv in p_values])
root_euclid = np.sqrt(p_values**2 + 1.0)
root_indef = np.sqrt((p_values**2 - 1.0).astype(complex))  # complex square root
errors = [np.max(np.abs(euclid_eigen[:, 1] - root_euclid)),
          np.max(np.abs(np.sort(np.abs(indef_eigen), axis=1)[:, 1]
                        - np.abs(root_indef)))]
check(max(errors) < 1e-12, "numpy eigenvalues equal +-sqrt(p^2 + 1), +-sqrt(p^2 - 1)")

fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 3.8), layout="constrained")
left.plot(p_values, root_euclid, color="#2a78d6", linewidth=2,
          label="$+\\sqrt{p^2+1}$")
left.plot(p_values, -root_euclid, color="#eb6834", linewidth=2,
          label="$-\\sqrt{p^2+1}$")
left.set_title("eigenvalues of $p\\,\\sigma_x + \\sigma_z$")
left.set_xlabel("$p$")
left.set_ylabel("eigenvalue")
left.legend(fontsize=8)
right.plot(p_values, np.abs(root_indef.real), color="#2a78d6", linewidth=2,
           label="real part of $+\\sqrt{p^2-1}$")
right.plot(p_values, np.abs(root_indef.imag), color="#1baf7a", linewidth=2,
           linestyle="--", label="imaginary part of $+\\sqrt{p^2-1}$")
right.set_title("eigenvalues of $p\\,\\sigma_x + N$")
right.set_xlabel("$p$")
right.set_ylabel("part of the eigenvalue")
right.legend(fontsize=8)
save_figure(fig, "square_roots_2x2",
            "Eigenvalues of two $2 \\times 2$ square roots, for $q = 1$ and $p$ "
            "from $-3$ to $3$ (pure numbers). Left: $p\\sigma_x + \\sigma_z$ squares "
            "to $(p^2 + 1) I_2$, and its eigenvalues are always the two real "
            "numbers $\\pm\\sqrt{p^2 + 1}$. Right: $p\\sigma_x + N$ squares to "
            "$(p^2 - 1) I_2$; its eigenvalues $\\pm\\sqrt{p^2 - 1}$ are real for "
            "$|p| > 1$ (solid line, the real part of the positive one) and "
            "imaginary for $|p| < 1$ (dashed line, the imaginary part): a minus "
            "sign in the quadratic form makes the root imaginary when the minus "
            "term wins.")
```

Out [4]:

```
PASS numpy eigenvalues equal +-sqrt(p^2 + 1), +-sqrt(p^2 - 1)
```

Out [4]:

```
Figure 04b.1 saved as Revision/textbook/figures/04b_1_square_roots_2x2.png
```

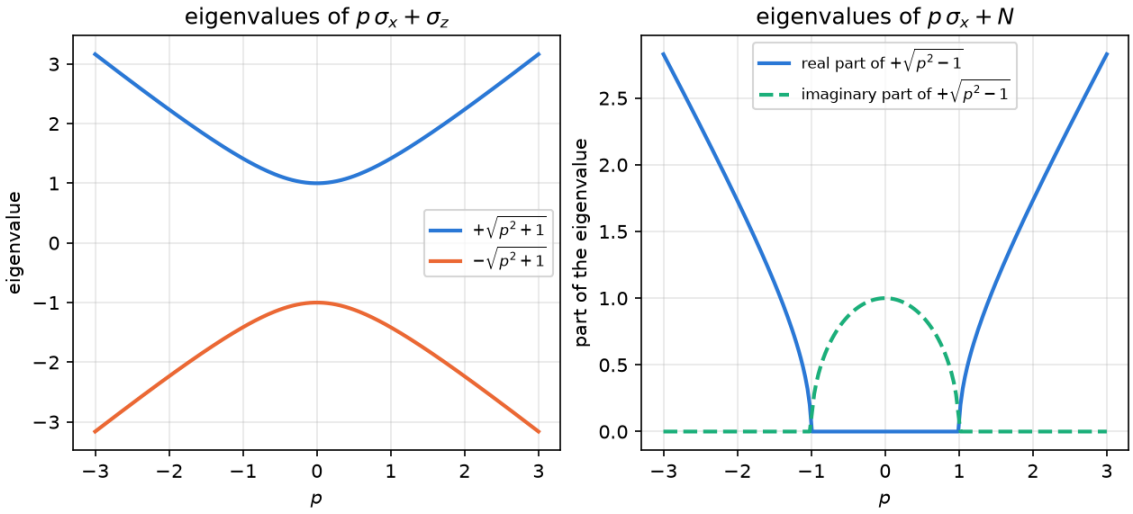

Figure 122: Eigenvalues of two 2×2 square roots, for $q = 1$ and p from -3 to 3 (pure numbers). Left: $p\sigma_x + \sigma_z$ squares to $(p^2+1)I_2$, and its eigenvalues are always the two real numbers $\pm\sqrt{p^2+1}$. Right: $p\sigma_x + N$ squares to $(p^2-1)I_2$; its eigenvalues $\pm\sqrt{p^2-1}$ are real for $|p| > 1$ (solid line, the real part of the positive one) and imaginary for $|p| < 1$ (dashed line, the imaginary part): a minus sign in the quadratic form makes the root imaginary when the minus term wins. (Notebook 04b, figure 1.)

Text cell:

```
## 7. The Pauli matrices: three directions

A third real $2 \times 2$ matrix that anticommutes with $\sigma_x$ and $\sigma_z$ and
squares to $+I_2$ does not exist, but a complex one does:
$\sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$. The three **Pauli
matrices** (Wolfgang Pauli, 1927) satisfy the Clifford relation with
$\eta = \mathrm{diag}(+1, +1, +1)$. The next cell computes the table of numbers
$c_{ab}$ with $\{\sigma_a, \sigma_b\} = c_{ab} I_2$, checks that it is $2\delta_{ab}$,
and checks the products $\sigma_x\sigma_y = i\sigma_z$ and
$\sigma_x\sigma_y\sigma_z = i I_2$ (by hand:
$\sigma_x\sigma_y = \begin{pmatrix} i & 0 \\ 0 & -i \end{pmatrix} = i\sigma_z$, then
$i\sigma_z\sigma_z = i I_2$). In sympy the imaginary unit is ‘sp.I‘.
```

In [5]:

```
sigma_y = sp.Matrix([[0, -sp.I], [sp.I, 0]])
pauli = [sigma_x, sigma_y, sigma_z]


def table_of(matrices, size):
    """The numbers c_ab with {M_a, M_b} = c_ab I; None where it is not a multiple."""
    rows = []
    for ma in matrices:
        row = []
```

```
        for mb in matrices:
            m = (ma * mb + mb * ma).expand()
            row.append(m[0, 0] if m == m[0, 0] * sp.eye(size) else None)
        rows.append(row)
    return rows


pauli_table = table_of(pauli, 2)
say(f"Pauli table c_ab: {pauli_table}")
check(pauli_table == [[2, 0, 0], [0, 2, 0], [0, 0, 2]],
      "{sigma_a, sigma_b} = 2 delta_ab I2 for the three Pauli matrices")
check(sigma_x * sigma_y == sp.I * sigma_z and sigma_x * sigma_y * sigma_z == sp.I * I2,
      "sigma_x sigma_y = i sigma_z and sigma_x sigma_y sigma_z = i I2")
```

Out [5]:

```
Pauli table c_ab: [[2, 0, 0], [0, 2, 0], [0, 0, 2]]
PASS {sigma_a, sigma_b} = 2 delta_ab I2 for the three Pauli matrices
PASS sigma_x sigma_y = i sigma_z and sigma_x sigma_y sigma_z = i I2
```

Text cell:

```
## 8. Dirac's matrices: time and three space directions

$2 \times 2$ matrices are too small for four anticommuting matrices. Dirac's choice,
in the particle physicists' convention $\eta = \mathrm{diag}(+1, -1, -1, -1)$ (time
first, sign $+1$), is built from $2 \times 2$ blocks:
$\gamma_D^0 = \begin{pmatrix} I_2 & 0 \\ 0 & -I_2 \end{pmatrix}$ and
$\gamma_D^j = \begin{pmatrix} 0 & \sigma_j \\ -\sigma_j & 0 \end{pmatrix}$ for
$j = x, y, z$. (The letter D keeps them apart from the author's gammas.) By the
block rule, $(\gamma_D^j)^2 = \mathrm{diag}(-\sigma_j^2, -\sigma_j^2) = -I_4$ and
$\gamma_D^0\gamma_D^j = -\gamma_D^j\gamma_D^0$. The next cell builds them with sympy's
`sp.Matrix(sp.BlockMatrix(...))` and checks the whole $4 \times 4$ table.
```

In [6]:

```
Z2 = sp.zeros(2, 2)
dirac = [sp.Matrix(sp.BlockMatrix([[I2, Z2], [Z2, -I2]]))]  # gamma_D^0
for s in pauli:  # gamma_D^x, gamma_D^y, gamma_D^z
    dirac.append(sp.Matrix(sp.BlockMatrix([[Z2, s], [-s, Z2]])))
dirac_table = table_of(dirac, 4)
say(f"Dirac table c_ab: {dirac_table}")
check(dirac_table == [[2, 0, 0, 0], [0, -2, 0, 0], [0, 0, -2, 0], [0, 0, 0, -2]],
      "{gamma_D^a, gamma_D^b} = 2 diag(+1, -1, -1, -1)_ab I4")
```

Out [6]:

```
Dirac table c_ab: [[2, 0, 0, 0], [0, -2, 0, 0], [0, 0, -2, 0], [0, 0, 0, -2]]
PASS {gamma_D^a, gamma_D^b} = 2 diag(+1, -1, -1, -1)_ab I4
```

Text cell:

```
The next cell draws the two tables as heat maps, each number written into its
square.
```

In [7]:

```
from matplotlib.colors import BoundaryNorm, ListedColormap

TABLE_COLOURS = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -2, 0, +2
TABLE_NORM = BoundaryNorm([-3, -1, 1, 3], 3)


def draw_table(ax, table, labels, title):
    """Draw a table of the numbers c_ab as a heat map with the numbers written in."""
    values = np.array(table, dtype=float)
    ax.imshow(values, cmap=TABLE_COLOURS, norm=TABLE_NORM)
    for (a, b), value in np.ndenumerate(values):
        ax.text(b, a, f"{int(value):+d}" if value else "0", ha="center",
                va="center", color="white" if value else "#52514e", fontsize=11)
    ax.set_xticks(range(len(labels)), labels)
    ax.set_yticks(range(len(labels)), labels)
    ax.set_title(title, fontsize=10)
    ax.grid(False)


fig, (left, right) = plt.subplots(1, 2, figsize=(8.0, 4.0), layout="constrained")
draw_table(left, pauli_table, ["$\\sigma_x$", "$\\sigma_y$", "$\\sigma_z$"],
           "Pauli: $\\{\\sigma_a, \\sigma_b\\} = c_{ab} I_2$")
draw_table(right, dirac_table, ["$\\gamma_D^0$", "$\\gamma_D^x$", "$\\gamma_D^y$",
                                "$\\gamma_D^z$"],
           "Dirac: $\\{\\gamma_D^a, \\gamma_D^b\\} = c_{ab} I_4$")
save_figure(fig, "pauli_dirac_tables",
            "The Clifford relation of the Pauli matrices (left) and of Dirac's "
            "matrices (right): the number $c_{ab}$ in "
            "$\\{M_a, M_b\\} = c_{ab} I$ for every pair, row $a$ and column $b$ "
            "(red $+2$, blue $-2$, grey $0$). Off the diagonal every entry is $0$: "
            "different matrices anticommute. On the diagonal stand twice the signs "
            "of the directions: $+2$ for the three space directions of Pauli, and "
            "$+2$ for Dirac's time and $-2$ for his three space directions.")
```

Out [7]:

```
Figure 04b.2 saved as Revision/textbook/figures/04b_2_pauli_dirac_tables.png
```

Text cell:

```
**What Dirac's matrices do.** Dirac's equation is
$i(\gamma_D^0\partial_t + \gamma_D^x\partial_1 + \gamma_D^y\partial_2 +
\gamma_D^z\partial_3)\psi = m\psi$. For a plane wave
```

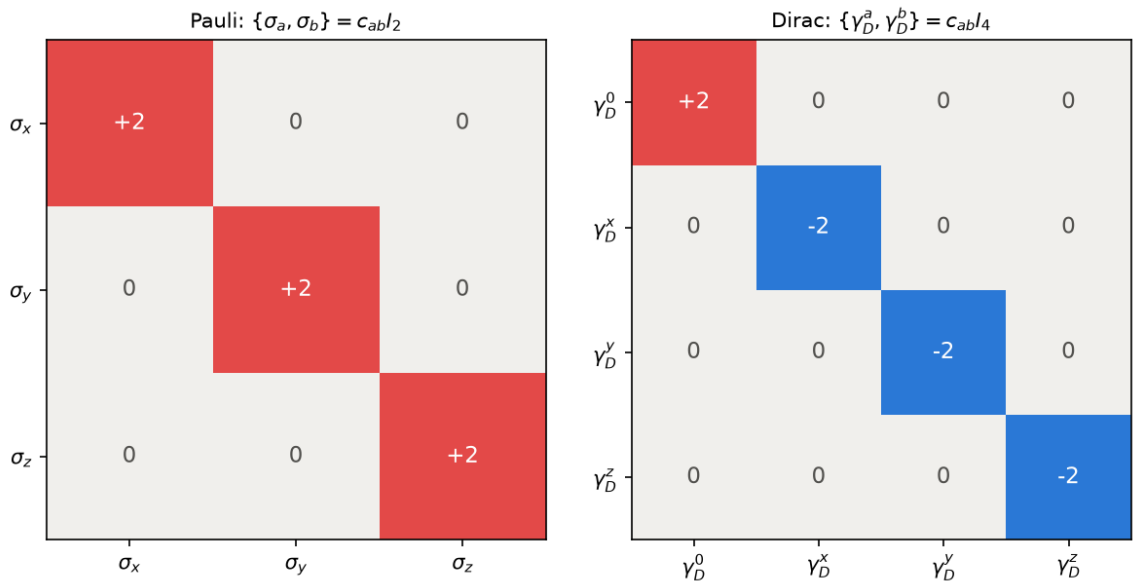

Figure 123: The Clifford relation of the Pauli matrices (left) and of Dirac's matrices (right): the number c_{ab} in $\{M_a, M_b\} = c_{ab} I$ for every pair, row a and column b (red $+2$, blue -2, grey 0). Off the diagonal every entry is 0: different matrices anticommute. On the diagonal stand twice the signs of the directions: $+2$ for the three space directions of Pauli, and $+2$ for Dirac's time and -2 for his three space directions. (Notebook 04b, figure 2.)

```
$\psi = u\,e^{-i(E t - k_1x_1 - k_2x_2 - k_3x_3)}$ the derivative $\partial_t$ gives the
factor $-iE$ and $\partial_j$ the factor $ik_j$; with the $i$ in front these become
$E$ and $-k_j$, so the plane wave solves the equation when
$(E\gamma_D^0 - k_1\gamma_D^x - k_2\gamma_D^y - k_3\gamma_D^z - m I_4)u = 0$. Call
$P = E\gamma_D^0 - k_1\gamma_D^x - k_2\gamma_D^y
- k_3\gamma_D^z$. By the general rule of the situation section, with the signs
$(+1, -1, -1, -1)$, $P^2 = (E^2 - k_1^2 - k_2^2 - k_3^2) I_4$. A nonzero $u$ with
$(P - m)u = 0$ satisfies $Pu = mu$, hence $P^2 u = m^2 u$, hence
$E^2 - k_1^2 - k_2^2 - k_3^2 = m^2$: the relativistic energy relation, the *mass
shell*. The next cell checks $P^2$ exactly with symbols, then takes $m = 3$ and
$k = (4, 0, 0)$, for which the mass shell is $E^2 = 9 + 16 = 25$, $E = \pm 5$, and
computes the smallest singular value of $P - m I_4$ for 801 energies from $-8$ to
$8$. It is zero exactly where a solution $u$ exists, and the check confirms that
this happens at $E = \pm 5$ and that there are two independent solutions there
(two singular values are zero).
```

In [8]:

```
E, k1, k2, k3 = sp.symbols("E k1 k2 k3", real=True)
P = E * dirac[0] - k1 * dirac[1] - k2 * dirac[2] - k3 * dirac[3]
check((P * P).expand() == (E**2 - k1**2 - k2**2 - k3**2) * sp.eye(4),
      "(E gamma_D^0 - k . gamma_D)^2 = (E^2 - k^2) I4 for all E and k")

dirac_numeric = [np.array(g, dtype=complex) for g in dirac]  # numpy copies
```

```
def smallest_singular_value(matrix):
    """The smallest length of matrix @ u over all columns u of length 1."""
    return np.linalg.svd(matrix, compute_uv=False).min()


mass, momentum = 3.0, (4.0, 0.0, 0.0)
energies = np.linspace(-8.0, 8.0, 801)  # steps of 0.02


def dirac_operator(energy):
    """P - m I4 for the energy energy and the fixed momentum and mass."""
    P_num = energy * dirac_numeric[0]
    for j in range(3):
        P_num = P_num - momentum[j] * dirac_numeric[j + 1]
    return P_num - mass * np.eye(4)


dirac_svals = np.array([smallest_singular_value(dirac_operator(e))
                        for e in energies])
# The energies where it vanishes, rounded to 6 decimals, as plain Python numbers:
zeros_at = [float(x) for x in np.round(energies[dirac_svals < 1e-9], 6)]
report("energies where a Dirac plane wave exists", zeros_at)
kernel = np.sum(np.linalg.svd(dirac_operator(5.0), compute_uv=False) < 1e-9)
report("independent solutions u at E = 5", int(kernel))
check(zeros_at == [-5.0, 5.0] and kernel == 2,
      "Dirac plane waves exist exactly at E = +-sqrt(m^2 + k^2) = +-5")
```

Out [8]:

```
PASS (E gamma_D^0 - k . gamma_D)^2 = (E^2 - k^2) I4 for all E and k
RESULT energies where a Dirac plane wave exists = [-5.0, 5.0]
RESULT independent solutions u at E = 5 = 2
PASS Dirac plane waves exist exactly at E = +-sqrt(m^2 + k^2) = +-5
```

Text cell:

```
The next cell draws the smallest singular value of $P - mI_4$ against the energy.
```

In [9]:

```
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.plot(energies, dirac_svals, color="#2a78d6", linewidth=2)
for e in (-5.0, 5.0):
    ax.axvline(e, color="#898781", linewidth=1, linestyle=":")
ax.set_xlabel("energy $E$")
ax.set_ylabel("smallest singular value of $P - m I_4$")
ax.set_title("Dirac plane waves for $m = 3$, $k = (4, 0, 0)$")
save_figure(fig, "dirac_mass_shell",
            "Where Dirac's equation has plane-wave solutions: the smallest "
            "singular value of $E\\gamma_D^0 - k\\cdot\\gamma_D - m I_4$ for mass "
            "$m = 3$, momentum $k = (4, 0, 0)$ and energies $E$ from $-8$ to $8$ "
```

```
"(units with the speed of light 1; all quantities pure numbers). It is "
"zero only at $E = \\pm 5$ (dotted lines), the two solutions of "
"$E^2 = m^2 + k^2 = 25$: the first-order matrix equation contains the "
"relativistic energy relation.")
```

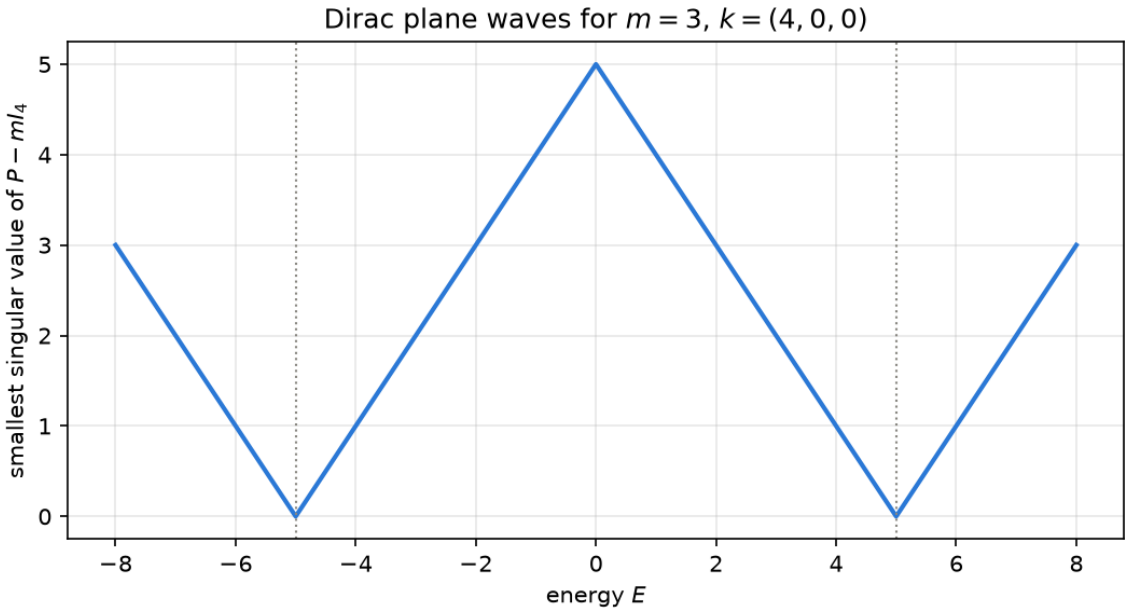

Figure 124: Where Dirac's equation has plane-wave solutions: the smallest singular value of $E\gamma_D^0 - k \cdot \gamma_D - mI_4$ for mass $m = 3$, momentum $k = (4, 0, 0)$ and energies E from -8 to 8 (units with the speed of light 1; all quantities pure numbers). It is zero only at $E = \pm 5$ (dotted lines), the two solutions of $E^2 = m^2 + k^2 = 25$: the first-order matrix equation contains the relativistic energy relation. (Notebook 04b, figure 3.)

Out [9]:

```
Figure 04b.3 saved as Revision/textbook/figures/04b_3_dirac_mass_shell.png
```

Text cell:

```
## 9. The author's eight gammas: the square root in 4+4 dimensions

The next cell reads the author's gammas from the Revision record file
'Revision/algebra/gammas.json' (a list of eight matrices in the order
$x_0, \dots, x_7$, entries whole numbers) together with the signs $\eta$, and makes
a sympy copy. With eight symbols $p_0, \dots, p_7$ it forms
$\not p = \sum_a p_a\gamma^{(x_a)}$ (read "p slash") and checks exactly that
$\not p^{\,2} = (p_0^2 + p_1^2 + p_2^2 + p_3^2 - p_4^2 - p_5^2 - p_6^2 - p_7^2)
I_{16}$ for all values of the $p_a$. By the general rule of the situation section
this identity is the same statement as the Clifford relation of the eight
matrices.
```

In [10]:

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
eta = record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1] in the order x0..x7
for matrix in record["gamma"]:  # the entries must be whole numbers
    if not all(isinstance(x, int) for row in matrix for x in row):
        raise ValueError("a gamma matrix of gammas.json has an entry that is not "
                         "a whole number")
gamma = [np.array(matrix, dtype=np.int64) for matrix in record["gamma"]]
sympy_gamma = [sp.Matrix(matrix) for matrix in record["gamma"]]
say("coordinates " + ", ".join(record["coordinates"]) + f"; eta = {eta}")

p_symbols = sp.symbols("p0:8", real=True)  # p0, ..., p7: p_symbols[a] is p_a
p_slash = sp.zeros(16, 16)
for a in range(8):
    p_slash += p_symbols[a] * sympy_gamma[a]
quadratic_form = sum(eta[a] * p_symbols[a] ** 2 for a in range(8))
say(f"quadratic form: {quadratic_form}")
check((p_slash * p_slash).expand() == quadratic_form * sp.eye(16),
      "(sum_a p_a gamma^(xa))^2 = eta(p, p) I16 for all p (exact)")
```

Out [10]:

```
coordinates x0, x1, x2, x3, x4, x5, x6, x7; eta = [1, 1, 1, 1, -1, -1, -1, -1]
quadratic form: p0**2 + p1**2 + p2**2 + p3**2 - p4**2 - p5**2 - p6**2 - p7**2
PASS (sum_a p_a gamma^(xa))^2 = eta(p, p) I16 for all p (exact)
```

Text cell:

```
The same identity with numbers: the next cell draws 300 random vectors $p$ (each
component from a normal distribution, seed 12345, so every run uses the same
numbers), computes $\not p^{\,2}$ with floating-point numbers, and compares it with
$\eta(p, p) I_{16}$, where $\eta(p,p) = \sum_a \eta^{aa} p_a^2$. The check allows a
rounding error of $10^{-12}$. The figure shows the diagonal entry
$(\not p^{\,2})_{00}$ against $\eta(p, p)$ (the points lie on the line of slope 1)
and the largest rounding error of each vector on a logarithmic scale.
```

In [11]:

```
generator = np.random.default_rng(12345)  # the seed makes the numbers repeatable
vectors = generator.normal(size=(300, 8))  # 300 vectors p with 8 components
forms, diagonal, worst = [], [], []
for vector in vectors:
    slash = sum(vector[a] * gamma[a] for a in range(8))  # p slash, a 16 x 16 array
    square = slash @ slash
    form = sum(eta[a] * vector[a] ** 2 for a in range(8))  # eta(p, p)
    forms.append(form)
    diagonal.append(square[0, 0])
    worst.append(np.max(np.abs(square - form * np.eye(16))))
# The exact size of the rounding errors depends on the computer (its numerical
# library); the right panel of the figure shows them. The check only requires them
# to stay below 1e-12, so the printed lines are the same on every computer.
```

```
check(max(worst) < 1e-12, "(p slash)^2 = eta(p, p) I16 for 300 random vectors p")
say("every rounding error of the 300 tests is below 1e-12")

fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 3.8), layout="constrained")
left.plot(forms, diagonal, "o", markersize=4, color="#2a78d6", alpha=0.7)
line = np.array([min(forms), max(forms)])
left.plot(line, line, color="#0b0b0b", linewidth=1, label="slope 1")
left.set_xlabel("$\\eta(p, p)$")
left.set_ylabel("diagonal entry of $(\\sum_a p_a\\gamma^{(x_a)})^2$")
left.set_title("$(\\sum_a p_a\\gamma^{(x_a)})^2 = \\eta(p,p)\\, I_{16}$")
left.legend(fontsize=8)
right.semilogy(range(1, 301), np.maximum(worst, 1e-18), "o", markersize=3,
               color="#eb6834")
right.set_xlabel("number of the random vector")
right.set_ylabel("largest deviation from $\\eta(p,p)\\, I_{16}$")
right.set_title("rounding errors")
save_figure(fig, "square_root_8d",
            "The author's gammas take the square root of the 4+4 quadratic form, "
            "tested with 300 random vectors $p$ (pure numbers). Left: the diagonal "
            "entry of the matrix $(\\sum_a p_a\\gamma^{(x_a)})^2$ against "
            "$\\eta(p,p) = p_0^2 + p_1^2 + p_2^2 + p_3^2 - p_4^2 - p_5^2 - p_6^2 - "
            "p_7^2$; all points lie on the line of slope 1, for positive and "
            "negative $\\eta(p,p)$. Right: for each vector the largest deviation of "
            "an entry of this matrix from $\\eta(p,p) I_{16}$, on a logarithmic "
            "scale; it is of the size of floating-point rounding, far below the "
            "tolerance $10^{-12}$ of the check (the exact identity is proved with "
            "symbols in the notebook).")
```

Out [11]:

```
PASS (p slash)^2 = eta(p, p) I16 for 300 random vectors p
every rounding error of the 300 tests is below 1e-12
```

Out [11]:

```
Figure 04b.4 saved as Revision/textbook/figures/04b_4_square_root_8d.png
```

Text cell:

```
## 10. Plane waves in flat 4+4 space

In flat 4+4 space the field equation without self-interaction is
$\sum_a \gamma^{(x_a)}\partial_a\Psi = m\Psi$ ($\partial_a$ is the derivative with
respect to $x_a$). Take a plane wave
$\Psi = u\,e^{i(k_1x_1 + k_2x_2 + k_3x_3 + k_5x_5 + k_6x_6 + k_7x_7 + k_0x_0 - Ex_4)}$
with a constant column $u$. Line by line:

1. $\partial_a\Psi = ik_a\Psi$ for $a \neq 4$ and $\partial_4\Psi = -iE\Psi$ (the
   derivative of $e^{cx}$ is $ce^{cx}$).
2. Inserting and dividing by the exponential (never zero):
   $-iE\gamma^{(x_4)}u + i\sum_{a\neq4}k_a\gamma^{(x_a)}u = mu$.
```

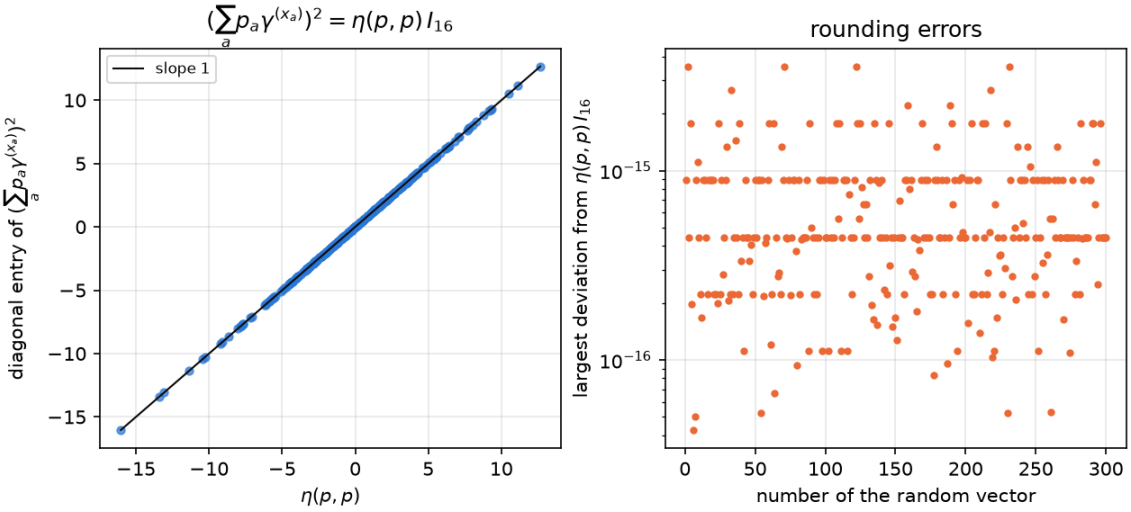

Figure 125: The author's gammas take the square root of the 4+4 quadratic form, tested with 300 random vectors p (pure numbers). Left: the diagonal entry of the matrix $(\sum_a p_a \gamma^{(x_a)})^2$ against $\eta(p,p) = p_0^2 + p_1^2 + p_2^2 + p_3^2 - p_4^2 - p_5^2 - p_6^2 - p_7^2$; all points lie on the line of slope 1, for positive and negative $\eta(p,p)$. Right: for each vector the largest deviation of an entry of this matrix from $\eta(p,p) I_{16}$, on a logarithmic scale; it is of the size of floating-point rounding, far below the tolerance 10^{-12} of the check (the exact identity is proved with symbols in the notebook). (Notebook 04b, figure 4.)

```
3. Multiplying from the left by $\gamma^{(x_4)}$ and using
   $(\gamma^{(x_4)})^2 = -I_{16}$ ($x_4$ is time-like):
   $iEu + i\sum_{a\neq4}k_a\gamma^{(x_4)}\gamma^{(x_a)}u = m\gamma^{(x_4)}u$.
4. Multiplying by $-i$ (and $-i \cdot i = 1$):
   $Eu + \sum_{a\neq4}k_a\gamma^{(x_4)}\gamma^{(x_a)}u = -im\gamma^{(x_4)}u$.
5. Moving the sum to the right-hand side:
   $Eu = hu$ with $h = -im\gamma^{(x_4)} - \sum_{a\neq4}k_a\gamma^{(x_4)}\gamma^{(x_a)}$.

So $E$ is an eigenvalue of the $16 \times 16$ matrix $h$. Write $h = m\beta +
\sum_{a\neq4}k_a\alpha_a$ with $\beta = -i\gamma^{(x_4)}$ and
$\alpha_a = -\gamma^{(x_4)}\gamma^{(x_a)}$. Then, from the Clifford relation
(write $g_4$ for $\gamma^{(x_4)}$ and $g_a$ for $\gamma^{(x_a)}$; $g_4^2 = -1$,
$g_a^2 = \eta^{aa}$, and different gammas anticommute):

- $\beta^2 = (-i)^2 g_4^2 = (-1)(-1) = 1$;
- $\alpha_a^2 = g_4 g_a g_4 g_a = -g_4 g_4 g_a g_a = -(-1)\eta^{aa} = \eta^{aa}$ (one
  exchange of $g_a$ and $g_4$ gives the minus sign);
- $\beta\alpha_a = i g_4 g_4 g_a = -i g_a$ and
  $\alpha_a\beta = i g_4 g_a g_4 = -i g_4 g_4 g_a = i g_a$: they anticommute;
- for $a \neq b$: $\alpha_a\alpha_b = g_4 g_a g_4 g_b = -g_4 g_4 g_a g_b = g_a g_b$,
  and $\alpha_b\alpha_a = g_b g_a = -g_a g_b$: they anticommute.

So $\beta$ and the $\alpha_a$ obey a Clifford relation, and by the general rule of
the situation section $h^2 = (m^2 + \sum_{a\neq4}\eta^{aa}k_a^2) I_{16}$, that is

$$h^2 = (m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2)\, I_{16},
\qquad E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2 .$$
```

The next cell checks the formula for h^2 exactly with symbols. The Revision record verified the same formula (‘Revision/theory/reports/python-field-theory.json‘, check ‘mode_hamiltonian_B_selfadjoint_dispersion‘; that check also verifies a statement about a matrix B that this notebook does not need). The cell reads the record and requires that its text contains the same h and the same formula for h^2, word for word: if the record ever says something else, the check fails.

In [12]:

```
THEORY = "Revision/theory/reports/python-field-theory.json"
theory = json.loads(repository_file(THEORY).read_text(encoding="utf-8"))
theory_checks = {entry["name"]: entry for entry in theory["checks"]}


def recorded_detail(name):
    """The text (detail) of the named check of the theory report when its verdict
    is pass; otherwise an empty text, which contains none of the quoted words."""
    entry = theory_checks.get(name, {})
    passed = entry.get("verdict", "").upper() == "PASS"
    return entry.get("detail", "") if passed else ""


m, k = sp.symbols("m", real=True), sp.symbols("k0:8", real=True)  # k[a] is k_a
g4 = sympy_gamma[4]  # gamma^(x4)
h = -sp.I * m * g4
for a in range(8):
    if a != 4:  # every direction except the time x4
        h -= k[a] * g4 * sympy_gamma[a]
dispersion = m**2 + k[0]**2 + k[1]**2 + k[2]**2 + k[3]**2 \
    - k[5]**2 - k[6]**2 - k[7]**2
# h and h^2 in the words of the record, quoted exactly as the record writes them
H_WORDS = "h = -i m gamma^(4) - sum_{a != x4} k_a gamma^(4) gamma^(a)"
H2_WORDS = "h^2 = (m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 - k7^2) I16"
detail = recorded_detail("mode_hamiltonian_B_selfadjoint_dispersion")
check((h * h).expand() == dispersion * sp.eye(16)
      and H_WORDS in detail and H2_WORDS in detail, H2_WORDS)
print(f"     reproduces {THEORY}")
print("         check mode_hamiltonian_B_selfadjoint_dispersion (formula for h^2)")
```

Out [12]:

```
PASS h^2 = (m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 - k7^2) I16
     reproduces Revision/theory/reports/python-field-theory.json
         check mode_hamiltonian_B_selfadjoint_dispersion (formula for h^2)
```

Text cell:

```
**The example of the Revision record.** Without momentum along the extra
times ($k_5 = k_6 = k_7 = 0$; the Revision record calls this the *good sector*)
$h$ is Hermitian and its eigenvalues are real. The reason: the author’s gammas are
```

```
real with $(g_a)^T = \eta^{aa} g_a$ (symmetric for space-like, antisymmetric for
time-like directions). So $\beta^\dagger = i g_4^T = -i g_4 = \beta$, and
$\alpha_a$ is real with $\alpha_a^T = -g_a^T g_4^T = \eta^{aa} g_a g_4
= -\eta^{aa} g_4 g_a = \eta^{aa}\alpha_a$: Hermitian for the space-like
$a = x_0, x_1, x_2, x_3$ (and anti-Hermitian for the extra times, which is why
they are left out here). The record uses the example $m = 2$, $k_0 = 4$ and
$(k_1, k_2, k_3) = (1, 2, 0)$: $E^2 = 4 + 16 + 1 + 4 + 0 = 25$,
$E = \pm 5$. The 16 eigenvalues add up to the trace of $h$ (true for every square
matrix), which is 0 (every gamma product in $h$ has trace 0); with $n_+$ eigenvalues
$+5$ and $n_-$ eigenvalues $-5$, $n_+ + n_- = 16$ and $5n_+ - 5n_- = 0$, so $+5$
and $-5$ occur 8 times each. The record states exactly this for the full
16-dimensional space: $h$ is Hermitian there, with the energies
$\pm\sqrt{m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2}$, "8 each". The next cell computes
the eigenvalues numerically, counts the independent solutions $u$ at $E = +5$ and
at $E = -5$ (the number of singular values of $EI_{16} - h$ below $10^{-9}$), checks
them, and requires the record's words. It also computes, for 801 energies from
$-8$ to $8$, the smallest singular value of $EI_{16} - h$; for a Hermitian $h$ it is
the distance from $E$ to the nearest eigenvalue.
```

In [13]:

```
gamma_complex = [g.astype(complex) for g in gamma]


def h_matrix(mass_value, momenta):
    """The numeric h for the mass and the eight momenta (momenta[4] is not used)."""
    result = -1j * mass_value * gamma_complex[4]
    for a in range(8):
        if a != 4:
            result = result - momenta[a] * gamma_complex[4] @ gamma_complex[a]
    return result


example = h_matrix(2.0, [4.0, 1.0, 2.0, 0.0, 0.0, 0.0, 0.0, 0.0])  # k0, k1, k2 = 4, 1, 2
hermitian = np.array_equal(example, example.conj().T)  # h^dagger = h, exactly
eigenvalues = np.linalg.eigvalsh(example)  # real eigenvalues, increasing order
rounded = [float(x) for x in np.round(eigenvalues, 9)]  # 9 decimals
multiplicities = {value: rounded.count(value) for value in sorted(set(rounded))}
report("eigenvalue: multiplicity of h for m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0)",
       multiplicities)
solutions = [int(np.sum(np.linalg.svd(e * np.eye(16) - example, compute_uv=False)
                        < 1e-9)) for e in (5.0, -5.0)]  # singular values near 0
report("independent solutions u at E = +5 and at E = -5", solutions)
eight_each = np.allclose(eigenvalues[:8], -5.0, atol=1e-12) and \
    np.allclose(eigenvalues[8:], 5.0, atol=1e-12)
good = recorded_detail("good_sector_spectrum_and_B_sectors")
check(hermitian and eight_each and solutions == [8, 8]
      and "h is Hermitian on the full 16-dim space" in good
      and f"({multiplicities.get(5.0)} each)" in good,
      "good sector: h is Hermitian with energies +5 and -5, eight each")
print(f"     reproduces {THEORY}")
print("         check good_sector_spectrum_and_B_sectors (full space: 8 each)")
```

```
energies_4p4 = np.linspace(-8.0, 8.0, 801)
svals_4p4 = np.array([smallest_singular_value(e * np.eye(16) - example)
                      for e in energies_4p4])
distance = np.minimum(np.abs(energies_4p4 - 5.0), np.abs(energies_4p4 + 5.0))
check(np.max(np.abs(svals_4p4 - distance)) < 1e-9,
      "smallest singular value of E - h = distance from E to +-5")
```

Out [13]:

```
RESULT eigenvalue: multiplicity of h for m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0) = {-5.0:
    8, 5.0: 8}
RESULT independent solutions u at E = +5 and at E = -5 = [8, 8]
PASS good sector: h is Hermitian with energies +5 and -5, eight each
     reproduces Revision/theory/reports/python-field-theory.json
         check good_sector_spectrum_and_B_sectors (full space: 8 each)
PASS smallest singular value of E - h = distance from E to +-5
```

Text cell:

```
The next cell draws the smallest singular value of $EI_{16} - h$ against $E$.
```

In [14]:

```
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.plot(energies_4p4, svals_4p4, color="#2a78d6", linewidth=2)
for e in (-5.0, 5.0):
    ax.axvline(e, color="#898781", linewidth=1, linestyle=":")
ax.set_xlabel("energy $E$")
ax.set_ylabel("smallest singular value of $E I_{16} - h$")
ax.set_title("plane waves in flat 4+4 space, $m = 2$, $k_0 = 4$, "
             "$(k_1, k_2, k_3) = (1, 2, 0)$")
save_figure(fig, "mass_shell_4p4",
            "Plane waves of the author's first-order equation in flat 4+4 space "
            "without extra-time momentum: the smallest singular value of "
            "$E I_{16} - h$ for $m = 2$, $k_0 = 4$, $(k_1, k_2, k_3) = (1, 2, 0)$ and "
            "energies $E$ from $-8$ to $8$ (pure numbers). It is the distance from "
            "$E$ to the nearest of the eigenvalues $\\pm 5$ (dotted lines) and "
            "vanishes only there, where $E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 "
            "= 25$; each of the two energies belongs to eight independent "
            "solutions. The curve has the same shape as for Dirac's matrices: the "
            "same algebra in more directions.")
```

Out [14]:

```
Figure 04b.5 saved as Revision/textbook/figures/04b_5_mass_shell_4p4.png
```

Text cell:

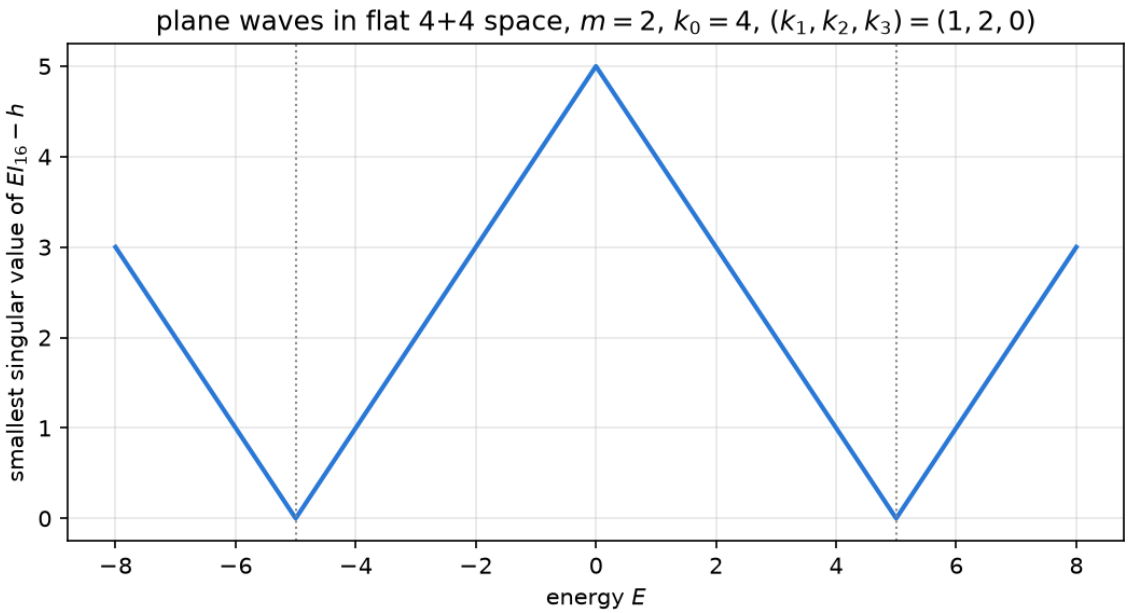

Figure 126: Plane waves of the author's first-order equation in flat 4+4 space without extra-time momentum: the smallest singular value of $EI_{16} - h$ for $m = 2$, $k_0 = 4$, $(k_1, k_2, k_3) = (1, 2, 0)$ and energies E from -8 to 8 (pure numbers). It is the distance from E to the nearest of the eigenvalues ± 5 (dotted lines) and vanishes only there, where $E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 = 25$; each of the two energies belongs to eight independent solutions. The curve has the same shape as for Dirac's matrices: the same algebra in more directions. (Notebook 04b, figure 5.)

```
**The record's exact example lives in the sector $B = +1$.** The Revision record
states the spectrum of its example not on all 16 components but in a sector of the
matrix $B = -iC\gamma^{(x_4)}$, where
$C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ is the product of
the four space-like gammas; the record file `Revision/algebra/gammas.json` stores
$B$. $B$ is Hermitian with $B^2 = I_{16}$ and trace 0, so its eigenvalues are $+1$
and $-1$, 8 times each. Without extra-time momentum $B$ commutes with $h$:
$Bh = hB$. Then $h$ maps every column $u$ of the sector $B = +1$ into the same
sector, because $B(hu) = h(Bu) = hu$; inside the sector $h$ acts as an $8 \times 8$
matrix. A momentum along an extra time spoils this: $B$ anticommutes with
$\gamma^{(x_4)}\gamma^{(x_a)}$ for $a = x_5, x_6, x_7$, so such a momentum mixes the
two sectors.

The next cell first checks exactly (sympy, for all values of $m, k_0, k_1, k_2,
k_3$): $B$ built from the gammas equals the stored $B$; $B^\dagger = B$,
$B^2 = I_{16}$, $\mathrm{tr}\, B = 0$; $Bh = hB$ without extra-time momentum; $B$
anticommutes with $\gamma^{(x_4)}\gamma^{(x_a)}$ for the three extra times; and
$\mathrm{tr}(hP) = 0$ for $P = (I_{16} + B)/2$, the matrix that keeps the $B = +1$
part of a column. Then, with numbers, it takes the 8 orthonormal eigenvectors of
$B$ with eigenvalue $+1$ (from `np.linalg.eigh`) as the columns of a
$16 \times 8$ matrix $V$; then $VV^\dagger = P$, $hV = Vh_+$ with the $8 \times 8$
matrix $h_+ = V^\dagger h V$, and $h_+^2 = 25 I_8$. The 8 eigenvalues of $h_+$ add
up to $\mathrm{tr}\, h_+ = \mathrm{tr}(hVV^\dagger) = \mathrm{tr}(hP) = 0$ (the
trace does not change when the last factor $V^\dagger$ is moved to the front), so
$+5$ and $-5$ occur four times each, as the record states. The cell computes them
and requires the record's words for this example.
```

In [15]:

```
C_exact = sympy_gamma[0] * sympy_gamma[1] * sympy_gamma[2] * sympy_gamma[3]
B_exact = -sp.I * C_exact * g4  # B = -i C gamma^(x4)
B_stored = sp.Matrix(record["B"]["re"]) + sp.I * sp.Matrix(record["B"]["im"])
h_good = h.subs({k[5]: 0, k[6]: 0, k[7]: 0})  # no momentum along an extra time
P_exact = (sp.eye(16) + B_exact) / 2  # keeps the B = +1 part of a column
mixes = all(B_exact * g4 * sympy_gamma[a] == -g4 * sympy_gamma[a] * B_exact
            for a in (5, 6, 7))  # anticommutes with gamma^(x4) gamma^(x5, x6, x7)
check(B_exact == B_stored and B_exact.H == B_exact
      and B_exact * B_exact == sp.eye(16) and B_exact.trace() == 0
      and (B_exact * h_good - h_good * B_exact).expand() == sp.zeros(16, 16)
      and mixes and (h_good * P_exact).trace().expand() == 0
      and "[B, h] = 0" in good
      and "B anticommutes with gamma^(4) gamma^(x5,x6,x7)" in good,
      "B is Hermitian, B^2 = I16, B h = h B without extra-time momentum")
print(f"     reproduces {THEORY}")
print("         check good_sector_spectrum_and_B_sectors (B and h commute)")

B_numeric = np.array(B_exact.tolist(), dtype=complex)
b_values, b_vectors = np.linalg.eigh(B_numeric)  # B is Hermitian: eigh applies
V = b_vectors[:, b_values > 0]  # the 8 orthonormal columns u with B u = u
h_plus = V.conj().T @ example @ V  # h inside the sector B = +1, 8 x 8
plus_values = np.linalg.eigvalsh(h_plus)  # its eigenvalues, increasing order
plus_rounded = [float(x) for x in np.round(plus_values, 9)]  # 9 decimals
plus_counts = {value: plus_rounded.count(value)
```

```
               for value in sorted(set(plus_rounded))}
report("eigenvalue: multiplicity of h in the sector B = +1", plus_counts)
sector_words = (f"m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0) on B = +1: +5 "
                f"({plus_counts.get(5.0)} times), -5 ({plus_counts.get(-5.0)} times)")
check(V.shape == (16, 8)
      and np.allclose(V @ V.conj().T, (np.eye(16) + B_numeric) / 2, atol=1e-12)
      and np.allclose(example @ V, V @ h_plus, atol=1e-12)
      and np.allclose(h_plus @ h_plus, 25.0 * np.eye(8), atol=1e-12)
      and np.allclose(plus_values, [-5.0] * 4 + [5.0] * 4, atol=1e-12)
      and sector_words in good,
      "sector B = +1: h has the energies +5 and -5, four times each")
print(f"     reproduces {THEORY}")
print("         check good_sector_spectrum_and_B_sectors (its exact example)")
```

Out [15]:

```
PASS B is Hermitian, B^2 = I16, B h = h B without extra-time momentum
     reproduces Revision/theory/reports/python-field-theory.json
         check good_sector_spectrum_and_B_sectors (B and h commute)
RESULT eigenvalue: multiplicity of h in the sector B = +1 = {-5.0: 4, 5.0: 4}
PASS sector B = +1: h has the energies +5 and -5, four times each
     reproduces Revision/theory/reports/python-field-theory.json
         check good_sector_spectrum_and_B_sectors (its exact example)
```

Text cell:

```
## 11. Momentum along an extra time

The extra times enter $E^2$ with a minus sign: $E^2 = 25 - k_5^2$ for the example
above with a momentum $k_5$ along the extra time $x_5$. For $k_5 < 5$ the energies
$\pm\sqrt{25 - k_5^2}$ are real; for $k_5 > 5$ they are imaginary, $E = \pm i
\sqrt{k_5^2 - 25}$, and the factor $e^{-iEx_4}$ of the plane wave then grows or
shrinks exponentially in the time $x_4$ instead of oscillating. This is the same
effect as in the right panel of the first figure, now in the author's 4+4
space-time; it is a fact about the flat equation, and the Revision record studies
its consequences separately. The next cell computes the 16 eigenvalues of $h$
numerically for 160 values of $k_5$ from 0.025 to 7.975 (the point $k_5 = 5$, where
both energies are 0, is left out), checks that 8 of them are
$+\sqrt{25 - k_5^2}$ and 8 are $-\sqrt{25 - k_5^2}$ (complex square root) to
$10^{-10}$, and draws their real and imaginary parts. For $k_5 > 5$ the imaginary
part $\sqrt{k_5^2 - 25} = \sqrt{k_5^2 - m^2 - k_0^2 - k_1^2 - k_2^2 - k_3^2}$ is the
*growth rate* of the wave; the Revision record states this formula
(`Revision/theory/reports/python-scope.json`, check
`extra_time_growth_rates_unbounded`). The cell first lets sympy write the formula
for $h^2$ of section 10 and, from it, the growth rate for a momentum $K$ along
$x_5$, and requires that the record's text contains both expressions exactly as
sympy writes them, and the words "no upper bound".
```

In [16]:

```
SCOPE = "Revision/theory/reports/python-scope.json"
scope = json.loads(repository_file(SCOPE).read_text(encoding="utf-8"))
```

```
scope_checks = {entry["name"]: entry for entry in scope["checks"]}
scope_entry = scope_checks.get("extra_time_growth_rates_unbounded", {})
scope_passed = scope_entry.get("verdict", "").upper() == "PASS"
scope_detail = scope_entry.get("detail", "") if scope_passed else ""
K = sp.symbols("K", positive=True)  # a momentum K along the extra time x5
rate = sp.sqrt(-dispersion.subs({k[5]: K, k[6]: 0, k[7]: 0}))  # Im E if E^2 < 0
say(f"h^2 = ({dispersion}) I16")
say(f"growth rate for k5 = K: Im E = {rate}")
scope_ok = (f"h_k^2 = ({dispersion}) I16" in scope_detail
            and f"Im E = {rate}" in scope_detail
            and "no upper bound" in scope_detail)  # the record's words

k5_values = np.linspace(0.025, 7.975, 160)  # steps of 0.05, avoiding k5 = 5
real_parts, imaginary_parts, deviations, counts_ok = [], [], [], True
for k5 in k5_values:
    values = np.linalg.eigvals(h_matrix(2.0, [4.0, 1.0, 2.0, 0.0, 0.0, k5, 0.0, 0.0]))
    root = np.sqrt(complex(25.0 - k5**2))  # +sqrt(25 - k5^2), complex if needed
    to_plus, to_minus = np.abs(values - root), np.abs(values + root)
    deviations.append(np.max(np.minimum(to_plus, to_minus)))
    counts_ok &= bool(np.sum(to_plus < to_minus) == 8)  # eight near +root
    real_parts.append(np.sort(values.real))
    imaginary_parts.append(np.sort(values.imag))
# As above, the size of the deviations depends on the computer; the check requires
# every one of the 160 x 16 eigenvalues to lie within 1e-10 of its exact value.
check(max(deviations) < 1e-10 and counts_ok and scope_ok,
      "with momentum k5: eight energies +sqrt(25 - k5^2), eight -sqrt(25 - k5^2)")
print(f"     reproduces {SCOPE}")
print("         check extra_time_growth_rates_unbounded (growth rate formula)")

real_parts, imaginary_parts = np.array(real_parts), np.array(imaginary_parts)
fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 3.8), layout="constrained")
for column in range(16):  # the 16 eigenvalues; they lie on two curves
    left.plot(k5_values, real_parts[:, column], ".", markersize=3, color="#2a78d6")
    right.plot(k5_values, imaginary_parts[:, column], ".", markersize=3,
               color="#1baf7a")
for ax, part in ((left, "real"), (right, "imaginary")):
    ax.axvline(5.0, color="#898781", linewidth=1, linestyle=":")
    ax.set_xlabel("momentum $k_5$ along the extra time $x_5$")
    ax.set_ylabel(f"{part} part of the energy $E$")
left.set_title("real parts: $\\pm\\sqrt{25 - k_5^2}$ for $k_5 < 5$")
right.set_title("imaginary parts: $\\pm\\sqrt{k_5^2 - 25}$ for $k_5 > 5$")
save_figure(fig, "extra_time_momentum",
            "The 16 energies $E$ of the plane waves of the example ($m = 2$, "
            "$k_0 = 4$, $(k_1, k_2, k_3) = (1, 2, 0)$) when a momentum $k_5$ along the "
            "extra time $x_5$ is added, for $k_5$ from 0 to 8 (pure numbers, flat "
            "4+4 space). Left: real parts; right: imaginary parts. The energies "
            "obey $E^2 = 25 - k_5^2$: eight equal $+\\sqrt{25 - k_5^2}$ and eight "
            "$-\\sqrt{25 - k_5^2}$; they are real for $k_5 < 5$ and imaginary for "
            "$k_5 > 5$ (dotted line at $k_5 = 5$), because the extra times enter "
            "the quadratic form with a minus sign.")
```

Out [16]:

```
h^2 = (k0**2 + k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + m**2) I16
growth rate for k5 = K: Im E = sqrt(K**2 - k0**2 - k1**2 - k2**2 - k3**2 - m**2)
PASS with momentum k5: eight energies +sqrt(25 - k5^2), eight -sqrt(25 - k5^2)
     reproduces Revision/theory/reports/python-scope.json
         check extra_time_growth_rates_unbounded (growth rate formula)
```

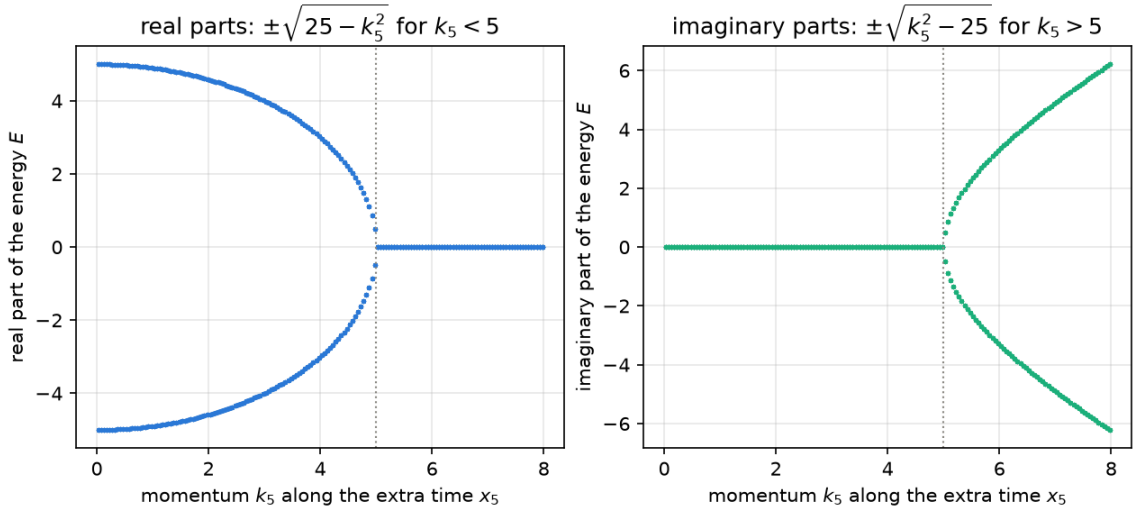

Figure 127: The 16 energies E of the plane waves of the example ($m = 2$, $k_0 = 4$, $(k_1, k_2, k_3) = (1, 2, 0)$) when a momentum k_5 along the extra time x_5 is added, for k_5 from 0 to 8 (pure numbers, flat 4+4 space). Left: real parts; right: imaginary parts. The energies obey $E^2 = 25 - k_5^2$: eight equal $+\sqrt{25 - k_5^2}$ and eight $-\sqrt{25 - k_5^2}$; they are real for $k_5 < 5$ and imaginary for $k_5 > 5$ (dotted line at $k_5 = 5$), because the extra times enter the quadratic form with a minus sign. (Notebook 04b, figure 6.)

Out [16]:

```
Figure 04b.6 saved as Revision/textbook/figures/04b_6_extra_time_momentum.png
```

Text cell:

```
**The exact example of the Revision record.** The Revision record also works out
the smallest example exactly (‘Revision/theory/reports/python-field-theory.json‘,
check ‘extra_time_modes_grow‘): mass $m = 1$, no momentum except $k_5 = 2$ along
the extra time $x_5$. Then $h^2 = (m^2 - k_5^2) I_{16} = (1 - 4) I_{16} = -3 I_{16}$,
so every eigenvalue $E$ has $E^2 = -3$: $E = +i\sqrt3$ or $E = -i\sqrt3$. The plane
wave carries the factor $e^{-iEx_4}$; for $E = i\sqrt3$ this is
$e^{-i \cdot i\sqrt3\, x_4} = e^{\sqrt3\, x_4}$ (because $-i \cdot i = 1$), a wave
that grows exponentially in the time $x_4$. The next cell substitutes these numbers
into the exact $h$ of section 10, checks $h^2 = -3 I_{16}$, asks sympy for the
exact eigenvalues and their multiplicities (‘eigenvals‘ returns a dictionary
eigenvalue: multiplicity), and compares them with the eigenvalues that the record
prints.

**Why the deflation matters here.** In the author’s metric the extra times
```

```
deflate. A wave with a fixed *coordinate* momentum $k_{x_5}$ along $x_5$ has the
*frame* momentum $b_4^{-1/2}e^{a_4}\sin^{-1/6}(z)\, k_{x_5}$: the derivative along
the coordinate $x_5$ is divided by the frame factor $b_4^{1/2}e^{-a_4}\sin^{1/6} z$
of the direction $x_5$. As $a_4$ grows (with $b_4$ constant; the record states it at
$b_4 = 1$) this frame momentum grows like $e^{a_4}$, so,
in the local plane-wave reading that the same Revision check states, every wave
along an extra time eventually reaches the region where $E$ is imaginary. This
notebook computes only the flat algebra behind that statement.
```

In [17]:

```
example_values = {m: 1, k[0]: 0, k[1]: 0, k[2]: 0, k[3]: 0, k[5]: 2, k[6]: 0,
                  k[7]: 0}  # mass 1, momentum 2 along x5, nothing else
h_example = h.subs(example_values)  # the exact 16 x 16 matrix h for these numbers
exact_eigenvalues = h_example.eigenvals()  # {eigenvalue: multiplicity}
names = sorted(str(value) for value in exact_eigenvalues)  # as sympy writes them
# printed in sorted order, so that every run prints the same line
listed = ", ".join(f"{value} ({count} times)" for value, count in
                   sorted(exact_eigenvalues.items(), key=lambda item: str(item[0])))
say(f"h^2 = -3 I16: {h_example * h_example == -3 * sp.eye(16)}; exact eigenvalues: "
    f"{listed}")
detail = recorded_detail("extra_time_modes_grow")
check(h_example * h_example == -3 * sp.eye(16)
      and exact_eigenvalues == {sp.sqrt(3) * sp.I: 8, -sp.sqrt(3) * sp.I: 8}
      and "m = 1, k5 = 2" in detail and str(names) in detail,
      "m = 1, k5 = 2: E = +i sqrt(3) and -i sqrt(3), eight each (growing modes)")
print(f"     reproduces {THEORY}")
print("         check extra_time_modes_grow (its exact example)")
```

Out [17]:

```
h^2 = -3 I16: True; exact eigenvalues: -sqrt(3)*I (8 times), sqrt(3)*I (8 times)
PASS m = 1, k5 = 2: E = +i sqrt(3) and -i sqrt(3), eight each (growing modes)
     reproduces Revision/theory/reports/python-field-theory.json
         check extra_time_modes_grow (its exact example)
```

Text cell:

```
## 12. The last check

The last cell checks that the six figure files exist in the folder
‘Revision/textbook/figures‘ and prints the number of checks that passed.
```

In [18]:

```
names = ["04b_1_square_roots_2x2.png", "04b_2_pauli_dirac_tables.png",
         "04b_3_dirac_mass_shell.png", "04b_4_square_root_8d.png",
         "04b_5_mass_shell_4p4.png", "04b_6_extra_time_momentum.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all six figure files exist")
all_checks_passed()
```

Out [18]:

```
PASS all six figure files exist
ALL 19 CHECKS PASSED (notebook 04b)
```

Text cell:

```
## 13. What this notebook showed

- PROVED (exact, sympy): two numbers cannot take the square root of $p^2 + q^2$;
  anticommuting matrices can. $(p\sigma_x + q\sigma_z)^2 = (p^2 + q^2)I_2$ and
  $(p\sigma_x + qN)^2 = (p^2 - q^2)I_2$ with real $2 \times 2$ matrices; the Pauli
  matrices satisfy the Clifford relation for three space directions and Dirac's
  matrices for $\mathrm{diag}(+1, -1, -1, -1)$; Dirac's first-order equation
  contains $E^2 = m^2 + k^2$.
- PROVED (exact, sympy, with the gammas of the Revision record file
  `Revision/algebra/gammas.json`): $(\sum_a p_a\gamma^{(x_a)})^2 =
  (p_0^2 + p_1^2 + p_2^2 + p_3^2 - p_4^2 - p_5^2 - p_6^2 - p_7^2)I_{16}$ for all
  $p$: the author's gammas take the square root of the quadratic form of his
  space-time.
- PROVED (exact): plane waves of $\sum_a\gamma^{(x_a)}\partial_a\Psi = m\Psi$ in flat
  4+4 space obey $Eu = hu$ with $h^2 = (m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2
  - k_6^2 - k_7^2) I_{16}$; this reproduces the Revision record
  (`Revision/theory/reports/python-field-theory.json`, check
  `mode_hamiltonian_B_selfadjoint_dispersion`).
- PROVED (exact, sympy): the record's matrix $B = -iC\gamma^{(x_4)}$ is Hermitian
  with $B^2 = I_{16}$ and commutes with $h$ when there is no extra-time momentum;
  a momentum along an extra time mixes the sectors $B = +1$ and $B = -1$ (check
  `good_sector_spectrum_and_B_sectors`).
- COMPUTED (floating point; every deviation from the exact values is below
  $10^{-9}$): the record's example ($m = 2$, $k_0 = 4$,
  $(k_1, k_2, k_3) = (1, 2, 0)$) has the energies $+5$ and $-5$, eight each, with
  eight independent solutions each, on all 16 components, and four each in the
  sector $B = +1$ (both statements of
  check `good_sector_spectrum_and_B_sectors`); a momentum along an extra time
  makes the energies imaginary when
  $k_5^2 > m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2$, with the growth rate
  $\sqrt{k_5^2 - m^2 - k_0^2 - k_1^2 - k_2^2 - k_3^2}$
  (`Revision/theory/reports/python-scope.json`, check
  `extra_time_growth_rates_unbounded`). PROVED (exact, sympy): the record's example
  $m = 1$, $k_5 = 2$ has the energies $\pm i\sqrt3$, eight each (check
  `extra_time_modes_grow`); the wave with $E = i\sqrt3$ grows like
  $e^{\sqrt3\,x_4}$.
- ASSUMED: flat 4+4 space (constant metric $\eta$). The author's space-time is
  curved, 3-space inflates and the extra times deflate; these effects are not part
  of this notebook. The deflation multiplies the frame momentum of a wave along an
  extra time by $e^{a_4}$, so it pushes such waves towards the growing region; the
  Revision record states this as a local plane-wave statement.
```

4.12 Line-by-line walk-through of Notebook 04b

The notebook has 18 code cells, In [1] to In [18]. As in Section 4.8, every line is quoted except two things: the set-up cell In [1], which is the set-up cell of Notebook 04a explained line by line in Section 4.8 and differs from it only as said below, and the long caption texts handed to `save_figure`, which are shortened to a line `...)` (the dots stand for the caption, the bracket closes the call); each caption is printed in full under its figure in Section 4.11.

In [1], the set-up cell. It is the set-up cell of Notebook 04a, explained line by line in Section 4.8, with one difference: the line `NOTEBOOK_ID = "04b"` names this notebook, so its figures are numbered 04b.1, 04b.2, ..., and its comment lines hold the run instructions of Section 4.10. It prints one line, Set-up of notebook 04b complete: repository folder found, helpers defined.

In [2], numbers are not enough.

```
import numpy as np  # floating-point arrays and linear algebra
import sympy as sp  # exact algebra with symbols
```

numpy computes with floating-point numbers in this notebook (numbers with about 16 significant digits, so the last digits may be rounded); sympy computes exactly with **symbols**, letters that stand for any number, so an identity that sympy confirms holds for every value.

```
alpha, beta = sp.symbols("alpha beta")  # two unknown numbers
equations = [alpha**2 - 1, beta**2 - 1, 2 * alpha * beta]  # each must equal 0
solutions = sp.solve(equations, [alpha, beta], dict=True)  # ** is a power
say(f"number solutions of alpha^2 = 1, beta^2 = 1, 2 alpha beta = 0: {solutions}")
check(solutions == [], "no two numbers solve the square-root problem")
```

`sp.symbols("alpha beta")` makes two symbols. The list `equations` holds three expressions that must all be zero: $\alpha^2 - 1$, $\beta^2 - 1$ and $2\alpha\beta$ (in Python `**` is a power). `sp.solve` returns the list of all solutions, real or complex, each as a dictionary; it is the empty list `[]`, which the cell prints and the check requires. This confirms the first paragraph of Section 4.3.

In [3], two real 2×2 matrices.

```
sigma_x = sp.Matrix([[0, 1], [1, 0]])
sigma_z = sp.Matrix([[1, 0], [0, -1]])
N = sp.Matrix([[0, 1], [-1, 0]])
I2 = sp.eye(2)  # the 2 x 2 identity matrix
p, q = sp.symbols("p q", real=True)  # two symbols that stand for any real numbers
```

The three matrices σ_x, σ_z and N of Section 4.3 as sympy matrices (each given by the list of its rows), the identity I_2, and two real symbols p and q.

```
euclidean = ((p * sigma_x + q * sigma_z) ** 2).expand()  # multiplied out
indefinite = ((p * sigma_x + q * N) ** 2).expand()
say(f"(p sigma_x + q sigma_z)^2 = {euclidean.tolist()}")
say(f"(p sigma_x + q N)^2 = {indefinite.tolist()}")
```

`** 2` squares the sympy matrix (a matrix product), and `.expand()` multiplies out every entry. The two printed matrices are $(p^2+q^2)I_2$ and $(p^2-q^2)I_2$; `.tolist()` writes a matrix as a list of rows.

```
example = 3 * sigma_x + 4 * sigma_z
say(f"3 sigma_x + 4 sigma_z = {example.tolist()}, its square = "
    f"{(example * example).tolist()}")
check(euclidean == (p**2 + q**2) * I2 and (example * example) == 25 * I2,
      "(p sigma_x + q sigma_z)^2 = (p^2 + q^2) I2 for all p, q")
check(indefinite == (p**2 - q**2) * I2,
      "(p sigma_x + q N)^2 = (p^2 - q^2) I2 for all p, q")
```

The worked example of Section 4.3, $3\sigma_x + 4\sigma_z$ and its square $25I_2$, is printed; the two checks confirm both identities for all p and q (sympy compares the matrices of expressions exactly).

In [4], the eigenvalues of the two roots, and figure 1.

```
p_values = np.linspace(-3.0, 3.0, 201)  # 201 equally spaced values of p
sx = np.array(sigma_x, dtype=float)  # the same matrices as numpy arrays
sz = np.array(sigma_z, dtype=float)
n_matrix = np.array(N, dtype=float)
```

`np.linspace(-3.0, 3.0, 201)` makes 201 numbers from -3 to 3 in equal steps of 0.03. The sympy matrices are copied into numpy arrays of floating-point numbers.

```
euclid_eigen = np.array([np.sort(np.linalg.eigvals(pv * sx + sz).real)
                         for pv in p_values])  # q = 1
indef_eigen = np.array([np.linalg.eigvals(pv * sx + n_matrix) for pv in p_values])
root_euclid = np.sqrt(p_values**2 + 1.0)
root_indef = np.sqrt((p_values**2 - 1.0).astype(complex))  # complex square root
```

For every value `pv` of p (and $q = 1$), `np.linalg.eigvals` computes the two eigenvalues of $p\sigma_x + \sigma_z$ (`.real` keeps their real parts, `np.sort` puts them in increasing order) and of $p\sigma_x + N$ (complex numbers in general). A square root of $c\,I_2$ has the eigenvalues $\pm\sqrt{c}$ (if $Mu = \lambda u$, then $M^2u = \lambda^2 u = cu$), so they are compared with $\sqrt{p^2+1}$ and $\sqrt{p^2-1}$; `.astype(complex)` turns the numbers $p^2 - 1$ into complex numbers, so that the square root of a negative one is imaginary instead of an error.

```
errors = [np.max(np.abs(euclid_eigen[:, 1] - root_euclid)),
          np.max(np.abs(np.sort(np.abs(indef_eigen), axis=1)[:, 1]
                        - np.abs(root_indef)))]
check(max(errors) < 1e-12, "numpy eigenvalues equal +-sqrt(p^2 + 1), +-sqrt(p^2 - 1)")
```

`euclid_eigen[:, 1]` is the column of the larger eigenvalue; its largest distance from $\sqrt{p^2+1}$ is the first error. For the second root, `np.abs` takes the size of each complex eigenvalue, and the larger size is compared with the size of $\sqrt{p^2-1}$. The check allows a rounding error of 10^{-12}.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 3.8), layout="constrained")
left.plot(p_values, root_euclid, color="#2a78d6", linewidth=2,
```

```
          label="$+\\sqrt{p^2+1}$")
left.plot(p_values, -root_euclid, color="#eb6834", linewidth=2,
          label="$-\\sqrt{p^2+1}$")
left.set_title("eigenvalues of $p\\,\\sigma_x + \\sigma_z$")
left.set_xlabel("$p$")
left.set_ylabel("eigenvalue")
left.legend(fontsize=8)
```

A figure with two panels side by side, named `left` and `right`. The left panel draws the two eigenvalues $\pm\sqrt{p^2+1}$ against p (blue and orange lines, two points wide), with a title, axis labels and a legend; `label=` is the name in the legend, and text between dollar signs is typeset as mathematics.

```
right.plot(p_values, np.abs(root_indef.real), color="#2a78d6", linewidth=2,
           label="real part of $+\\sqrt{p^2-1}$")
right.plot(p_values, np.abs(root_indef.imag), color="#1baf7a", linewidth=2,
           linestyle="--", label="imaginary part of $+\\sqrt{p^2-1}$")
right.set_title("eigenvalues of $p\\,\\sigma_x + N$")
right.set_xlabel("$p$")
right.set_ylabel("part of the eigenvalue")
right.legend(fontsize=8)
save_figure(fig, "square_roots_2x2",
            ...)
```

The right panel draws the real part (solid) and the imaginary part (dashed) of $\sqrt{p^2-1}$, and the figure is saved (its caption is printed in Section 4.11). **What figure 1 shows and why:** on the left two real curves that never meet, because $p^2+1>0$; on the right the solid curve is zero for $|p|<1$ and the dashed curve is zero for $|p|>1$: where the minus term of p^2-q^2 wins, the square root becomes imaginary. This is the two-dimensional picture of what a momentum along an extra time does in In [15].

In [5], the Pauli matrices.

```
sigma_y = sp.Matrix([[0, -sp.I], [sp.I, 0]])
pauli = [sigma_x, sigma_y, sigma_z]
```

σ_y with the imaginary unit, which sympy writes `sp.I`, and the list of the three Pauli matrices.

```
def table_of(matrices, size):
    """The numbers c_ab with {M_a, M_b} = c_ab I; None where it is not a multiple."""
    rows = []
    for ma in matrices:
        row = []
        for mb in matrices:
            m = (ma * mb + mb * ma).expand()
            row.append(m[0, 0] if m == m[0, 0] * sp.eye(size) else None)
        rows.append(row)
    return rows
```

`table_of` computes, for every pair of matrices of the list, the anticommutator; if it is a multiple of the identity, it stores the multiple (its upper-left entry), otherwise the special value `None`. The result is the table of the numbers c_{ab} in $\{M_a, M_b\} = c_{ab} I$, as a list of rows.

```
pauli_table = table_of(pauli, 2)
say(f"Pauli table c_ab: {pauli_table}")
check(pauli_table == [[2, 0, 0], [0, 2, 0], [0, 0, 2]],
      "{sigma_a, sigma_b} = 2 delta_ab I2 for the three Pauli matrices")
check(sigma_x * sigma_y == sp.I * sigma_z and sigma_x * sigma_y * sigma_z == sp.I * I2,
      "sigma_x sigma_y = i sigma_z and sigma_x sigma_y sigma_z = i I2")
```

The Pauli table is twice the identity table, $2\delta_{ab}$: the Clifford relation for three space directions. The second check confirms $\sigma_x\sigma_y = i\sigma_z$ and $\sigma_x\sigma_y\sigma_z = iI_2$, computed by hand in Section 4.3.

In [6], Dirac's matrices.

```
Z2 = sp.zeros(2, 2)
dirac = [sp.Matrix(sp.BlockMatrix([[I2, Z2], [Z2, -I2]]))]  # gamma_D^0
for s in pauli:  # gamma_D^x, gamma_D^y, gamma_D^z
    dirac.append(sp.Matrix(sp.BlockMatrix([[Z2, s], [-s, Z2]])))
dirac_table = table_of(dirac, 4)
say(f"Dirac table c_ab: {dirac_table}")
check(dirac_table == [[2, 0, 0, 0], [0, -2, 0, 0], [0, 0, -2, 0], [0, 0, 0, -2]],
      "{gamma_D^a, gamma_D^b} = 2 diag(+1, -1, -1, -1)_ab I4")
```

`sp.BlockMatrix` glues 2×2 blocks, and `sp.Matrix(...)` turns the result into an ordinary 4×4 matrix. The list `dirac` holds $\gamma_D^0 = \mathrm{diag}(I_2, -I_2)$ and, for each Pauli matrix s, $\gamma_D^j = \begin{pmatrix} 0 & s \\ -s & 0 \end{pmatrix}$ (`.append` adds an element to a list). Their table is $2\,\mathrm{diag}(+1, -1, -1, -1)$, which the check requires.

In [7], figure 2.

```
from matplotlib.colors import BoundaryNorm, ListedColormap

TABLE_COLOURS = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -2, 0, +2
TABLE_NORM = BoundaryNorm([-3, -1, 1, 3], 3)
```

Three colours and the boundaries $-3, -1, 1, 3$: -2 blue, 0 grey, $+2$ red.

```
def draw_table(ax, table, labels, title):
    """Draw a table of the numbers c_ab as a heat map with the numbers written in."""
    values = np.array(table, dtype=float)
    ax.imshow(values, cmap=TABLE_COLOURS, norm=TABLE_NORM)
    for (a, b), value in np.ndenumerate(values):
        ax.text(b, a, f"{int(value):+d}" if value else "0", ha="center",
                va="center", color="white" if value else "#52514e", fontsize=11)
    ax.set_xticks(range(len(labels)), labels)
    ax.set_yticks(range(len(labels)), labels)
    ax.set_title(title, fontsize=10)
    ax.grid(False)
```

`draw_table` paints a table of the numbers c_{ab} as a heat map, writes each number into its square (with its sign, or 0), names the rows and columns with `labels`, and sets the title.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.0, 4.0), layout="constrained")
draw_table(left, pauli_table, ["$\\sigma_x$", "$\\sigma_y$", "$\\sigma_z$"],
           "Pauli: $\\{\\sigma_a, \\sigma_b\\} = c_{ab} I_2$")
draw_table(right, dirac_table, ["$\\gamma_D^0$", "$\\gamma_D^x$", "$\\gamma_D^y$",
                                "$\\gamma_D^z$"],
           "Dirac: $\\{\\gamma_D^a, \\gamma_D^b\\} = c_{ab} I_4$")
save_figure(fig, "pauli_dirac_tables",
            ...)
```

The Pauli table on the left, Dirac's on the right. **What figure 2 shows and why:** grey off the diagonal (different matrices anticommute) and twice the signs of the directions on the diagonal: three red squares for Pauli's three space directions; one red square for Dirac's time and three blue ones for his space directions, in his convention $(+1, -1, -1, -1)$.

In [8], Dirac's mass shell.

```
E, k1, k2, k3 = sp.symbols("E k1 k2 k3", real=True)
P = E * dirac[0] - k1 * dirac[1] - k2 * dirac[2] - k3 * dirac[3]
check((P * P).expand() == (E**2 - k1**2 - k2**2 - k3**2) * sp.eye(4),
      "(E gamma_D^0 - k . gamma_D)^2 = (E^2 - k^2) I4 for all E and k")
```

The matrix $P = E\gamma_D^0 - k_1\gamma_D^x - k_2\gamma_D^y - k_3\gamma_D^z$ of Section 4.3 with real symbols, and the exact check $P^2 = (E^2 - k_1^2 - k_2^2 - k_3^2)I_4$ for all values.

```
dirac_numeric = [np.array(g, dtype=complex) for g in dirac]  # numpy copies


def smallest_singular_value(matrix):
    """The smallest length of matrix @ u over all columns u of length 1."""
    return np.linalg.svd(matrix, compute_uv=False).min()
```

numpy copies of Dirac's matrices with complex entries. `np.linalg.svd(matrix, compute_uv=False)` computes the singular values of a matrix (the possible lengths of Mu along special directions, for columns u of length 1), and `.min()` takes the smallest.

```
mass, momentum = 3.0, (4.0, 0.0, 0.0)
energies = np.linspace(-8.0, 8.0, 801)  # steps of 0.02


def dirac_operator(energy):
    """P - m I4 for the energy energy and the fixed momentum and mass."""
    P_num = energy * dirac_numeric[0]
    for j in range(3):
        P_num = P_num - momentum[j] * dirac_numeric[j + 1]
    return P_num - mass * np.eye(4)
```

The example $m = 3$, $k = (4, 0, 0)$, for which the mass shell is $E^2 = 9 + 16 = 25$. `energies` holds 801 energies from -8 to 8 in steps of 0.02. `dirac_operator(energy)` builds the numeric matrix $P - mI_4$ for one energy.

```
dirac_svals = np.array([smallest_singular_value(dirac_operator(e))
                        for e in energies])
# The energies where it vanishes, rounded to 6 decimals, as plain Python numbers:
zeros_at = [float(x) for x in np.round(energies[dirac_svals < 1e-9], 6)]
report("energies where a Dirac plane wave exists", zeros_at)
kernel = np.sum(np.linalg.svd(dirac_operator(5.0), compute_uv=False) < 1e-9)
report("independent solutions u at E = 5", int(kernel))
check(zeros_at == [-5.0, 5.0] and kernel == 2,
      "Dirac plane waves exist exactly at E = +-sqrt(m^2 + k^2) = +-5")
```

The smallest singular value is computed at all 801 energies. `energies[dirac_svals < 1e-9]` keeps the energies at which it is below 10^{-9} (zero up to rounding): a **boolean mask**, an array of True/False that selects entries. `np.round(..., 6)` rounds them to six decimals. At $E = 5$ the number of singular values below 10^{-9} is the number of independent solutions u. The output: plane waves exist exactly at $E = -5$ and $E = 5$, with two independent solutions at $E = 5$; the check requires both.

In [9], figure 3.

```
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.plot(energies, dirac_svals, color="#2a78d6", linewidth=2)
for e in (-5.0, 5.0):
    ax.axvline(e, color="#898781", linewidth=1, linestyle=":")
ax.set_xlabel("energy $E$")
ax.set_ylabel("smallest singular value of $P - m I_4$")
ax.set_title("Dirac plane waves for $m = 3$, $k = (4, 0, 0)$")
save_figure(fig, "dirac_mass_shell",
            ...)
```

The smallest singular value against the energy, with dotted vertical lines (`axvline`) at $E = \pm 5$. **What figure 3 shows and why:** the curve touches zero only at the two dotted lines, the two solutions of $E^2 = m^2 + k^2 = 25$: the first-order matrix equation contains the energy relation of special relativity.

In [10], the square root in 4+4 dimensions.

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
eta = record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1] in the order x0..x7
for matrix in record["gamma"]:  # the entries must be whole numbers
    if not all(isinstance(x, int) for row in matrix for x in row):
        raise ValueError("a gamma matrix of gammas.json has an entry that is not "
                         "a whole number")
gamma = [np.array(matrix, dtype=np.int64) for matrix in record["gamma"]]
sympy_gamma = [sp.Matrix(matrix) for matrix in record["gamma"]]
say("coordinates " + ", ".join(record["coordinates"]) + f"; eta = {eta}")
```

The author's gammas are read from the Revision record file `gammas.json` (the same matrices that Notebook 04a built from his formulas). The loop makes sure that every entry is a whole number (`isinstance(x, int)`). Then the eight matrices are kept twice: as numpy arrays of whole numbers (`gamma`) and as sympy matrices (`sympy_gamma`). The printed line lists the coordinates and the signs.

```
p_symbols = sp.symbols("p0:8", real=True)  # p0, ..., p7: p_symbols[a] is p_a
p_slash = sp.zeros(16, 16)
for a in range(8):
    p_slash += p_symbols[a] * sympy_gamma[a]
quadratic_form = sum(eta[a] * p_symbols[a] ** 2 for a in range(8))
say(f"quadratic form: {quadratic_form}")
check((p_slash * p_slash).expand() == quadratic_form * sp.eye(16),
      "(sum_a p_a gamma^(xa))^2 = eta(p, p) I16 for all p (exact)")
```

`sp.symbols("p0:8")` makes the eight symbols $p_0, \ldots, p_7$, so that `p_symbols[a]` is p_a. The loop builds $\sum_a p_a \gamma^{(x_a)}$, read "p slash" (`+=` adds to a variable). `quadratic_form` is $\eta(p,p) = \sum_a \eta^{aa} p_a^2$, printed as `p0**2 + p1**2 + p2**2 + p3**2 - p4**2 - p5**2 - p6**2 - p7**2`. The check confirms exactly the square-root rule at the start of Section 4.9.

In [11], the same with 300 random vectors, and figure 4.

```
generator = np.random.default_rng(12345)  # the seed makes the numbers repeatable
vectors = generator.normal(size=(300, 8))  # 300 vectors p with 8 components
forms, diagonal, worst = [], [], []
```

A **random-number generator** started from the fixed number 12345 (its **seed**) produces the same numbers in every run. `normal(size=(300, 8))` draws 300×8 numbers from the normal (bell-shaped) distribution: 300 vectors p. Three empty lists are prepared.

```
for vector in vectors:
    slash = sum(vector[a] * gamma[a] for a in range(8))  # p slash, a 16 x 16 array
    square = slash @ slash
    form = sum(eta[a] * vector[a] ** 2 for a in range(8))  # eta(p, p)
    forms.append(form)
    diagonal.append(square[0, 0])
    worst.append(np.max(np.abs(square - form * np.eye(16))))
```

For each vector: the floating-point matrix $\sum_a p_a \gamma^{(x_a)}$, its square, and $\eta(p,p)$; the lists collect $\eta(p,p)$, the upper-left entry of the square, and the largest deviation of any entry of the square from $\eta(p,p) I_{16}$.

```
# The exact size of the rounding errors depends on the computer (its numerical
# library); the right panel of the figure shows them. The check only requires them
# to stay below 1e-12, so the printed lines are the same on every computer.
check(max(worst) < 1e-12, "(p slash)^2 = eta(p, p) I16 for 300 random vectors p")
say("every rounding error of the 300 tests is below 1e-12")
```

The check allows rounding errors up to 10^{-12}; as the comment says, the exact sizes of rounding errors differ between computers, so the notebook prints only that they are below the bound.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 3.8), layout="constrained")
left.plot(forms, diagonal, "o", markersize=4, color="#2a78d6", alpha=0.7)
line = np.array([min(forms), max(forms)])
left.plot(line, line, color="#0b0b0b", linewidth=1, label="slope 1")
left.set_xlabel("$\\eta(p, p)$")
```

```
left.set_ylabel("diagonal entry of $(\\sum_a p_a\\gamma^{(x_a)})^2$")
left.set_title("$(\\sum_a p_a\\gamma^{(x_a)})^2 = \\eta(p,p)\\, I_{16}$")
left.legend(fontsize=8)
```

The left panel draws one dot (`"o"`, size 4, 70 per cent opaque) per vector, at $\eta(p,p)$ horizontally and the diagonal entry vertically, and the black line of slope 1 from the smallest to the largest $\eta(p,p)$.

```
right.semilogy(range(1, 301), np.maximum(worst, 1e-18), "o", markersize=3,
               color="#eb6834")
right.set_xlabel("number of the random vector")
right.set_ylabel("largest deviation from $\\eta(p,p)\\, I_{16}$")
right.set_title("rounding errors")
save_figure(fig, "square_root_8d",
            ...)
```

The right panel draws, on a **logarithmic** vertical axis (`semilogy`: equal distances for equal factors of ten), the largest deviation for each vector; `np.maximum(worst, 1e-18)` replaces a deviation of exactly 0 by 10^{-18}, because 0 has no logarithm. **What figure 4 shows and why:** on the left all 300 dots lie on the line of slope 1, for positive and for negative $\eta(p,p)$ (the minus signs of the time and the extra times make many values negative); on the right every deviation is of the size of floating-point rounding, that is, an error in about the sixteenth significant digit of entries whose size is between about 1 and 15, many powers of ten under the tolerance 10^{-12}. (This panel may look slightly different on another computer; the exact identity is the sympy check of In [10].)

In [12], the formula for h^2.

```
THEORY = "Revision/theory/reports/python-field-theory.json"
theory = json.loads(repository_file(THEORY).read_text(encoding="utf-8"))
theory_checks = {entry["name"]: entry for entry in theory["checks"]}
```

```
def recorded_detail(name):
    """The text (detail) of the named check of the theory report when its verdict
    is pass; otherwise an empty text, which contains none of the quoted words."""
    entry = theory_checks.get(name, {})
    passed = entry.get("verdict", "").upper() == "PASS"
    return entry.get("detail", "") if passed else ""
```

The theory report of the Revision record is read, and its checks are kept in a dictionary by name. Every check of the report has a name, a verdict and a **detail**, a text that says in words what was checked and found. `recorded_detail(name)` returns this text when the report holds the named check with the verdict pass, and otherwise the empty text `""`. The chained `.get(..., {})` and `.get("verdict", "")` return an empty dictionary or an empty text instead of an error when something is missing, `.upper()` writes the verdict in capital letters (the report writes `pass`), and `x if passed else ""` chooses between two values. The empty text contains no words at all, so every test "these words occur in the detail" fails when the check is missing or did not pass. The cells below use this function to compare the notebook's results with what the record **says**, not only with its verdict: if the record ever states a different formula or a different example, the check fails.

```
m, k = sp.symbols("m", real=True), sp.symbols("k0:8", real=True)  # k[a] is k_a
g4 = sympy_gamma[4]  # gamma^(x4)
h = -sp.I * m * g4
for a in range(8):
    if a != 4:  # every direction except the time x4
        h -= k[a] * g4 * sympy_gamma[a]
dispersion = m**2 + k[0]**2 + k[1]**2 + k[2]**2 + k[3]**2 \
    - k[5]**2 - k[6]**2 - k[7]**2
```

A real symbol m and the eight momentum symbols $k_0, \ldots, k_7$ (`k[a]` is k_a; k_4 is not used, because the time carries the energy). `h` is built exactly as in line 5 of Section 4.9: $-im\gamma^{(x_4)}$ minus $k_a\gamma^{(x_4)}\gamma^{(x_a)}$ for every a other than the time (`-=` subtracts from a variable). `dispersion` is $m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$.

```
# h and h^2 in the words of the record, quoted exactly as the record writes them
H_WORDS = "h = -i m gamma^(4) - sum_{a != x4} k_a gamma^(4) gamma^(a)"
H2_WORDS = "h^2 = (m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 - k7^2) I16"
detail = recorded_detail("mode_hamiltonian_B_selfadjoint_dispersion")
check((h * h).expand() == dispersion * sp.eye(16)
      and H_WORDS in detail and H2_WORDS in detail, H2_WORDS)
print(f"     reproduces {THEORY}")
print("         check mode_hamiltonian_B_selfadjoint_dispersion (formula for h^2)")
```

`H_WORDS` and `H2_WORDS` are the record's own words for h (the record writes `gamma^(4)` for $\gamma^{(x_4)}$) and for h^2. `text in detail` is true when the text occurs, character for character, inside `detail`. The check confirms h^2 exactly, for all values of the symbols, and requires that the record states the same h and the same formula; the name of the check is the formula itself. The two printed lines name the record.

In [13], the good-sector example on all sixteen components.

```
gamma_complex = [g.astype(complex) for g in gamma]


def h_matrix(mass_value, momenta):
    """The numeric h for the mass and the eight momenta (momenta[4] is not used)."""
    result = -1j * mass_value * gamma_complex[4]
    for a in range(8):
        if a != 4:
            result = result - momenta[a] * gamma_complex[4] @ gamma_complex[a]
    return result
```

Complex copies of the gammas (Python writes the imaginary unit `1j`), and the function `h_matrix` that builds the numeric h for a mass and a list of eight momenta in the order $x_0, \ldots, x_7$ (`momenta[4]` is not used).

```
example = h_matrix(2.0, [4.0, 1.0, 2.0, 0.0, 0.0, 0.0, 0.0, 0.0])  # k0, k1, k2 = 4, 1, 2
hermitian = np.array_equal(example, example.conj().T)  # h^dagger = h, exactly
eigenvalues = np.linalg.eigvalsh(example)  # real eigenvalues, increasing order
rounded = [float(x) for x in np.round(eigenvalues, 9)]  # 9 decimals
```

```
multiplicities = {value: rounded.count(value) for value in sorted(set(rounded))}
report("eigenvalue: multiplicity of h for m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0)",
       multiplicities)
```

The record's example $m = 2$, $k_0 = 4$, $(k_1, k_2, k_3) = (1, 2, 0)$, the list of momenta $[4, 1, 2, 0, 0, 0, 0, 0]$. `example.conj().T` is the conjugate transpose, and `np.array_equal` compares exactly: h is Hermitian. `np.linalg.eigvalsh` computes the eigenvalues of a Hermitian matrix (real numbers, in increasing order, each listed as often as its multiplicity). They are rounded to nine decimals and counted: the RESULT line reads `{-5.0:  8, 5.0:  8}`.

```
solutions = [int(np.sum(np.linalg.svd(e * np.eye(16) - example, compute_uv=False)
                        < 1e-9)) for e in (5.0, -5.0)]  # singular values near 0
report("independent solutions u at E = +5 and at E = -5", solutions)
```

As for Dirac's matrices in In [8], the number of singular values of $EI_{16} - h$ below 10^{-9} is the number of independent solutions u of $hu = Eu$. The **list comprehension** `[... for e in (5.0, -5.0)]` computes this number once for $E = 5$ and once for $E = -5$ and collects the two results in a list; `int(...)` makes each a plain whole number. The RESULT line reads `[8, 8]`: eight independent solutions at each energy.

```
eight_each = np.allclose(eigenvalues[:8], -5.0, atol=1e-12) and \
    np.allclose(eigenvalues[8:], 5.0, atol=1e-12)
good = recorded_detail("good_sector_spectrum_and_B_sectors")
check(hermitian and eight_each and solutions == [8, 8]
      and "h is Hermitian on the full 16-dim space" in good
      and f"({multiplicities.get(5.0)} each)" in good,
      "good sector: h is Hermitian with energies +5 and -5, eight each")
print(f"     reproduces {THEORY}")
print("         check good_sector_spectrum_and_B_sectors (full space: 8 each)")
```

`np.allclose(x, value, atol=1e-12)` is true when every entry of `x` lies within 10^{-12} of `value`: the first eight eigenvalues are -5 and the last eight $+5$. `good` is the text of the record's check `good_sector_spectrum_and_B_sectors`. `multiplicities.get(5.0)` is the multiplicity 8 that the notebook found for the eigenvalue $+5$, so the f-string makes the text `(8 each)` from the notebook's own count. The check requires h to be Hermitian, eight eigenvalues -5 and eight $+5$, eight independent solutions at each energy, and the record's words for this statement about all sixteen components; the printed lines name the record and say which of its statements is reproduced.

```
energies_4p4 = np.linspace(-8.0, 8.0, 801)
svals_4p4 = np.array([smallest_singular_value(e * np.eye(16) - example)
                      for e in energies_4p4])
distance = np.minimum(np.abs(energies_4p4 - 5.0), np.abs(energies_4p4 + 5.0))
check(np.max(np.abs(svals_4p4 - distance)) < 1e-9,
      "smallest singular value of E - h = distance from E to +-5")
```

The smallest singular value of $EI_{16} - h$ at 801 energies from -8 to 8. For a Hermitian h it equals the distance from E to the nearest eigenvalue, here $\min(|E - 5|, |E + 5|)$ (`np.minimum` takes the smaller of two numbers, entry by entry); the check confirms this to 10^{-9}.

In [14], figure 5.

```
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.plot(energies_4p4, svals_4p4, color="#2a78d6", linewidth=2)
for e in (-5.0, 5.0):
    ax.axvline(e, color="#898781", linewidth=1, linestyle=":")
ax.set_xlabel("energy $E$")
ax.set_ylabel("smallest singular value of $E I_{16} - h$")
ax.set_title("plane waves in flat 4+4 space, $m = 2$, $k_0 = 4$, "
             "$(k_1, k_2, k_3) = (1, 2, 0)$")
save_figure(fig, "mass_shell_4p4",
            ...)
```

The same kind of figure as figure 3, now for the author's gammas. **What figure 5 shows and why:** a zigzag of straight lines, the distance from E to the nearer of ± 5, touching zero only at the dotted lines $E = \pm 5$, where $E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 = 25$; each of the two energies belongs to eight independent solutions (In [13] counted them). It has the shape of the Dirac curve: the same algebra in more directions.

In [15], the sector $B = +1$ and the record's exact example.

```
C_exact = sympy_gamma[0] * sympy_gamma[1] * sympy_gamma[2] * sympy_gamma[3]
B_exact = -sp.I * C_exact * g4  # B = -i C gamma^(x4)
B_stored = sp.Matrix(record["B"]["re"]) + sp.I * sp.Matrix(record["B"]["im"])
```

$C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ from the exact gammas (`sympy_gamma[a]` is $\gamma^{(x_a)}$), and $B = -iC\gamma^{(x_4)}$, exactly. `record` is the content of the record file `Revision/algebra/gammas.json`, read in In [10]; it stores B as two tables of whole numbers, its real part `"re"` and its imaginary part `"im"`, and `B_stored` puts them together as the real part plus i times the imaginary part.

```
h_good = h.subs({k[5]: 0, k[6]: 0, k[7]: 0})  # no momentum along an extra time
P_exact = (sp.eye(16) + B_exact) / 2  # keeps the B = +1 part of a column
mixes = all(B_exact * g4 * sympy_gamma[a] == -g4 * sympy_gamma[a] * B_exact
            for a in (5, 6, 7))  # anticommutes with gamma^(x4) gamma^(x5, x6, x7)
```

`h_good` is the exact h of In [12] with $k_5 = k_6 = k_7 = 0$ (the entries 5, 6 and 7 of the list `k`, counted from 0 like the coordinates); the symbols m, k_0, k_1, k_2, k_3 stay free, so everything checked with `h_good` holds for all their values. `P_exact` is $P = \frac{1}{2}(I_{16} + B)$: for a column with $Bu = u$ it gives $Pu = u$, for one with $Bu = -u$ it gives $Pu = 0$. `mixes` is true when $B\gamma^{(x_4)}\gamma^{(x_a)} = -\gamma^{(x_4)}\gamma^{(x_a)}B$ for $a = x_5, x_6, x_7$ (`sympy_gamma[5]` to `sympy_gamma[7]`); `all(...)` is true when the test holds for every value of `a` in the list.

```
check(B_exact == B_stored and B_exact.H == B_exact
      and B_exact * B_exact == sp.eye(16) and B_exact.trace() == 0
      and (B_exact * h_good - h_good * B_exact).expand() == sp.zeros(16, 16)
      and mixes and (h_good * P_exact).trace().expand() == 0
      and "[B, h] = 0" in good
      and "B anticommutes with gamma^(4) gamma^(x5,x6,x7)" in good,
      "B is Hermitian, B^2 = I16, B h = h B without extra-time momentum")
print(f"     reproduces {THEORY}")
print("         check good_sector_spectrum_and_B_sectors (B and h commute)")
```

The exact part of Section 4.9: the rebuilt B equals the stored one; `.H` is sympy's conjugate transpose, so `B_exact.H == B_exact` says $B^\dagger = B$; $B^2 = I_{16}$ and $\operatorname{tr} B = 0$ (`.trace()` is the trace); $Bh - hB$, multiplied out, is the zero matrix `sp.zeros(16, 16)` for all values of the symbols; B anticommutes with $\gamma^{(x_4)}\gamma^{(x_a)}$ for the three extra times; $\operatorname{tr}(hP) = 0$, again for all values; and the record states the commutation and the anticommutation in these words.

```
B_numeric = np.array(B_exact.tolist(), dtype=complex)
b_values, b_vectors = np.linalg.eigh(B_numeric)  # B is Hermitian: eigh applies
V = b_vectors[:, b_values > 0]  # the 8 orthonormal columns u with B u = u
h_plus = V.conj().T @ example @ V  # h inside the sector B = +1, 8 x 8
```

A numpy copy of B (`tolist()` turns the sympy matrix into a list of rows). `np.linalg.eigh` returns, for a Hermitian matrix, its eigenvalues in increasing order and a matrix whose columns are orthonormal eigenvectors, one for each eigenvalue in the same order. `b_vectors[:, b_values > 0]` keeps all rows (`:`) of the columns whose eigenvalue is positive, that is $+1$: the 16×8 matrix V of Section 4.9. `h_plus` is $h_+ = V^\dagger h V$ for the example h of In [13].

```
plus_values = np.linalg.eigvalsh(h_plus)  # its eigenvalues, increasing order
plus_rounded = [float(x) for x in np.round(plus_values, 9)]  # 9 decimals
plus_counts = {value: plus_rounded.count(value)
               for value in sorted(set(plus_rounded))}
report("eigenvalue: multiplicity of h in the sector B = +1", plus_counts)
```

The eight eigenvalues of h_+, rounded and counted as in In [13]: the RESULT line reads `{-5.0:  4, 5.0:  4}`.

```
sector_words = (f"m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0) on B = +1: +5 "
                f"({plus_counts.get(5.0)} times), -5 ({plus_counts.get(-5.0)} times)")
check(V.shape == (16, 8)
      and np.allclose(V @ V.conj().T, (np.eye(16) + B_numeric) / 2, atol=1e-12)
      and np.allclose(example @ V, V @ h_plus, atol=1e-12)
      and np.allclose(h_plus @ h_plus, 25.0 * np.eye(8), atol=1e-12)
      and np.allclose(plus_values, [-5.0] * 4 + [5.0] * 4, atol=1e-12)
      and sector_words in good,
      "sector B = +1: h has the energies +5 and -5, four times each")
print(f"     reproduces {THEORY}")
print("         check good_sector_spectrum_and_B_sectors (its exact example)")
```

`sector_words` writes the record's sentence for its exact example with the counts that the notebook found (two texts written next to each other inside round brackets are joined into one). The check requires: V has 16 rows and 8 columns (`V.shape`); $VV^\dagger = P$; $hV = Vh_+$, so h maps the sector into itself; $h_+^2 = 25 I_8$; the eight eigenvalues are four times -5 and four times $+5$ (`[-5.0] * 4 + [5.0] * 4` is the list of four -5 followed by four $+5$), all within 10^{-12}; and the record's text contains `m = 2, k0 = 4, (k1, k2, k3) = (1, 2, 0) on B = +1:  +5 (4 times), -5 (4 times)`. This reproduces the record's exact example as the record states it.

In [16], a momentum along an extra time, and figure 6.

```
SCOPE = "Revision/theory/reports/python-scope.json"
scope = json.loads(repository_file(SCOPE).read_text(encoding="utf-8"))
scope_checks = {entry["name"]: entry for entry in scope["checks"]}
scope_entry = scope_checks.get("extra_time_growth_rates_unbounded", {})
scope_passed = scope_entry.get("verdict", "").upper() == "PASS"
scope_detail = scope_entry.get("detail", "") if scope_passed else ""
```

The scope report of the Revision record is read. `scope_detail` is the text of its check `extra_time_growth_rates_unbounded` when that check passed, and otherwise the empty text: the same rule as `recorded_detail` in In [12], for the other report.

```
K = sp.symbols("K", positive=True)  # a momentum K along the extra time x5
rate = sp.sqrt(-dispersion.subs({k[5]: K, k[6]: 0, k[7]: 0}))  # Im E if E^2 < 0
say(f"h^2 = ({dispersion}) I16")
say(f"growth rate for k5 = K: Im E = {rate}")
scope_ok = (f"h_k^2 = ({dispersion}) I16" in scope_detail
            and f"Im E = {rate}" in scope_detail
            and "no upper bound" in scope_detail)  # the record's words
```

A positive symbol K for the momentum along x_5. With $k_5 = K$ and $k_6 = k_7 = 0$, `dispersion` is E^2; when $E^2 < 0$, $E = \pm i\sqrt{-E^2}$, so the imaginary part of the growing energy, the growth rate, is $\sqrt{-E^2}$: `rate`. sympy writes the two expressions in its own order, and the two printed lines show them: `h^2 = (k0**2 + k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + m**2) I16` and `growth rate for k5 = K: Im E = sqrt(K**2 - k0**2 - k1**2 - k2**2 - k3**2 - m**2)`, where `**` is a power. `scope_ok` is true when the record's text contains both expressions exactly as sympy writes them, and the words `no upper bound`.

```
k5_values = np.linspace(0.025, 7.975, 160)  # steps of 0.05, avoiding k5 = 5
real_parts, imaginary_parts, deviations, counts_ok = [], [], [], True
for k5 in k5_values:
    values = np.linalg.eigvals(h_matrix(2.0, [4.0, 1.0, 2.0, 0.0, 0.0, k5, 0.0, 0.0]))
    root = np.sqrt(complex(25.0 - k5**2))  # +sqrt(25 - k5^2), complex if needed
    to_plus, to_minus = np.abs(values - root), np.abs(values + root)
    deviations.append(np.max(np.minimum(to_plus, to_minus)))
    counts_ok &= bool(np.sum(to_plus < to_minus) == 8)  # eight near +root
    real_parts.append(np.sort(values.real))
    imaginary_parts.append(np.sort(values.imag))
```

160 values of k_5 from 0.025 to 7.975 in steps of 0.05 (the value $k_5 = 5$, where both energies are 0, is skipped). For each, the 16 eigenvalues of h with the momentum k_5 added (in position 5 of the list, the direction x_5; h is no longer Hermitian, so the general `eigvals` is used); `root` is $\sqrt{25 - k_5^2}$, imaginary when $k_5 > 5$. For each eigenvalue its distance to +root and to −root is computed; the largest of the smaller distances is stored, and `counts_ok` stays true only if exactly eight eigenvalues lie nearer to +root. The sorted real and imaginary parts are kept for the figure.

```
# As above, the size of the deviations depends on the computer; the check requires
# every one of the 160 x 16 eigenvalues to lie within 1e-10 of its exact value.
check(max(deviations) < 1e-10 and counts_ok and scope_ok,
      "with momentum k5: eight energies +sqrt(25 - k5^2), eight -sqrt(25 - k5^2)")
```

```
print(f"     reproduces {SCOPE}")
print("         check extra_time_growth_rates_unbounded (growth rate formula)")
```

The check requires all 160×16 eigenvalues within 10^{-10} of $\pm\sqrt{25 - k_5^2}$, eight of each, and the record's words (`scope_ok`).

```
real_parts, imaginary_parts = np.array(real_parts), np.array(imaginary_parts)
fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 3.8), layout="constrained")
for column in range(16):  # the 16 eigenvalues; they lie on two curves
    left.plot(k5_values, real_parts[:, column], ".", markersize=3, color="#2a78d6")
    right.plot(k5_values, imaginary_parts[:, column], ".", markersize=3,
               color="#1baf7a")
```

The lists become arrays with one row per k_5 and one column per eigenvalue; each column is drawn as small dots, real parts on the left, imaginary parts on the right.

```
for ax, part in ((left, "real"), (right, "imaginary")):
    ax.axvline(5.0, color="#898781", linewidth=1, linestyle=":")
    ax.set_xlabel("momentum $k_5$ along the extra time $x_5$")
    ax.set_ylabel(f"{part} part of the energy $E$")
left.set_title("real parts: $\\pm\\sqrt{25 - k_5^2}$ for $k_5 < 5$")
right.set_title("imaginary parts: $\\pm\\sqrt{k_5^2 - 25}$ for $k_5 > 5$")
save_figure(fig, "extra_time_momentum",
            ...)
```

Both panels get a dotted line at $k_5 = 5$ and axis labels, then titles, and the figure is saved. **What figure 6 shows and why:** on the left the real parts follow the upper and the lower half of a circle of radius 5, $\pm\sqrt{25 - k_5^2}$, reach zero at $k_5 = 5$ and stay zero beyond; on the right the imaginary parts are zero up to $k_5 = 5$ and then open up as $\pm\sqrt{k_5^2 - 25}$, growing without bound. Because the extra times enter the quadratic form with a minus sign, a large enough momentum along an extra time turns oscillating waves into growing and shrinking ones.

In [17], the exact example with growing modes.

```
example_values = {m: 1, k[0]: 0, k[1]: 0, k[2]: 0, k[3]: 0, k[5]: 2, k[6]: 0,
                  k[7]: 0}  # mass 1, momentum 2 along x5, nothing else
h_example = h.subs(example_values)  # the exact 16 x 16 matrix h for these numbers
exact_eigenvalues = h_example.eigenvals()  # {eigenvalue: multiplicity}
```

`h.subs(...)` substitutes the numbers of the record's example ($m = 1$, $k_5 = 2$, all other momenta 0) into the exact matrix h of In [12]. `eigenvals()` returns the exact eigenvalues as a dictionary eigenvalue: multiplicity, the multiplicity being the number of times the eigenvalue occurs as a root of the characteristic polynomial.

```
names = sorted(str(value) for value in exact_eigenvalues)  # as sympy writes them
# printed in sorted order, so that every run prints the same line
listed = ", ".join(f"{value} ({count} times)" for value, count in
                   sorted(exact_eigenvalues.items(), key=lambda item: str(item[0])))
say(f"h^2 = -3 I16: {h_example * h_example == -3 * sp.eye(16)}; exact eigenvalues: "
    f"{listed}")
```

`names` lists the eigenvalues as sympy writes them, sorted. `listed` writes each eigenvalue with its multiplicity; `sorted(..., key=lambda item:  str(item[0]))` sorts the pairs by the text of the eigenvalue (a `lambda` is a one-line function without a name), so that every run prints the same order. The printed line: $h^2 = -3I_{16}$ is True, and the eigenvalues are `-sqrt(3)*I` and `sqrt(3)*I`, eight times each.

```
detail = recorded_detail("extra_time_modes_grow")
check(h_example * h_example == -3 * sp.eye(16)
      and exact_eigenvalues == {sp.sqrt(3) * sp.I: 8, -sp.sqrt(3) * sp.I: 8}
      and "m = 1, k5 = 2" in detail and str(names) in detail,
      "m = 1, k5 = 2: E = +i sqrt(3) and -i sqrt(3), eight each (growing modes)")
print(f"     reproduces {THEORY}")
print("         check extra_time_modes_grow (its exact example)")
```

`detail` is the text of the Revision check `extra_time_modes_grow` (the empty text if the check did not pass, by the rule of In [12]). The check requires $h^2 = -3I_{16}$, the eigenvalues $\pm i\sqrt{3}$ with multiplicity 8 each, and the record's text to contain the example and the same list of eigenvalues word for word.

In [18], the last check.

```
names = ["04b_1_square_roots_2x2.png", "04b_2_pauli_dirac_tables.png",
         "04b_3_dirac_mass_shell.png", "04b_4_square_root_8d.png",
         "04b_5_mass_shell_4p4.png", "04b_6_extra_time_momentum.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all six figure files exist")
all_checks_passed()
```

The six figure files must exist; the last line reads ALL 19 CHECKS PASSED (notebook 04b): one check in each of In [2], In [4], In [6], In [10], In [11], In [12], In [16], In [17] and In [18], and two in each of In [3], In [5], In [8], In [13] and In [15].

4.13 The 256 products: Cl(4,4) is all real 16 by 16 matrices

The Clifford algebra Cl(4,4) is the set of all real combinations of products of the eight gammas (Section 4.3). This section proves that it is the set of **all** real 16×16 matrices, and that sixteen is the smallest size eight such matrices can have. Everything follows from the Clifford relation alone.

Sets and how to count them. A **set** is a collection of different objects, written in braces, for example $A = \{x_2, x_5\}$; the **empty set** $\{\}$ has no element. A **subset** of a set S is a set whose elements all belong to S. A set of eight directions has $2^8 = 256$ subsets, because each of the eight directions is either in a subset or not (two choices, eight times). The **binomial coefficient** $\binom{n}{k}$ ("n choose k") is the number of subsets with k elements of a set with n elements. With the **factorial** $n! = 1 \cdot 2 \cdots n$ and $0! = 1$ it is $\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$: k different elements can be picked one after the other in $n(n-1)\cdots(n-k+1) = n!/(n-k)!$ ways, and every subset of k elements arises in this way $k!$ times, once for each order of its elements. For $n = 8$ the numbers for $k = 0, 1, \ldots, 8$ are $1, 8, 28, 56, 70, 56, 28, 8, 1$ (for example $\binom{8}{2} = 8 \cdot 7/2 = 28$). Exactly half of the 256 subsets have an even number of elements. Proof: the rule "add x_1 if it is missing, remove it if it is present" turns

every subset into another one, changes the number of elements by one (so it turns even into odd and odd into even), and applied twice gives the subset back; so it pairs every even subset with exactly one odd subset, and there are 128 of each.

The products. For a set A of directions write γ_A for the product of their gammas in increasing order, for example $\gamma_{\{x_2,x_5\}} = \gamma^{(x_2)}\gamma^{(x_5)}$, and $\gamma_{\{\}} = I_{16}$. The number k of factors is the **degree** of γ_A; the product is **even** if k is even and **odd** if k is odd. There are 256 products, $\binom{8}{k}$ of degree k, 128 even and 128 odd. The **symmetric difference** $A \triangle B$ is the set of the directions that lie in exactly one of A and B. Three rules follow from the Clifford relation.

Rule R1: products multiply into products. $\gamma_A\gamma_B = s(A,B)\,\gamma_{A\triangle B}$ with the sign $s(A,B) = (-1)^{N(A,B)} \prod_{c\in A\cap B} \eta^{cc}$, where $N(A,B)$ is the number of pairs (a,b) with a in A, b in B and a later than b in the order $x_0,\dots,x_7$, and $A\cap B$ is the set of the directions in both. Proof, step by step: in $\gamma_A\gamma_B$, take the factors of γ_B one after the other, the earliest direction first, and move each to the left until it stands in increasing order. A factor γ^b passes exactly the factors γ^a of A with a later than b (the factors of B moved before it are earlier than b and stay on its left); each of them is a different gamma, so each exchange gives a factor -1 (anticommutation). If b also lies in A, γ^b then stands next to its partner γ^b, and the two give $(\gamma^b)^2 = \eta^{bb} I_{16}$ and disappear. What remains is the product, in increasing order, of the directions in exactly one of A and B, and the collected signs are $(-1)^{N(A,B)}$ and one η^{cc} for each common direction.

Rule R2: squares. With $B = A$ the common directions are all of A, and $N(A,A)$ is the number of pairs in A with the later one first, $k(k-1)/2$ for a product of degree k (every pair of different elements of A counts once). So

$$\gamma_A\gamma_A = (-1)^{k(k-1)/2} \prod_{a\in A} \eta^{aa}\; I_{16}\,:$$

every product squares to $+I_{16}$ or to $-I_{16}$, and its inverse is $\gamma_A^{-1} = \pm\gamma_A$ with the same sign. For example a product of two gammas has $(\gamma^a\gamma^b)^2 = (-1)^1\eta^{aa}\eta^{bb} I_{16} = -\eta^{aa}\eta^{bb} I_{16}$.

Rule R3: passing one gamma. Moving γ^b from the left of γ_A to its right passes each of the k factors once: $\gamma^b\gamma_A = (-1)^k\gamma_A\gamma^b$ if b is not in A (all k factors are different from γ^b), and $\gamma^b\gamma_A = (-1)^{k-1}\gamma_A\gamma^b$ if b is in A (passing γ^b itself gives no sign).

The trace lemma. Every product other than I_{16} has trace 0. Proof, line by line, for a product γ_A of degree $k \geq 1$:

1. If k is odd, then $k \leq 7$, so some direction b is not in A, and R3 gives $\gamma^b\gamma_A = (-1)^k\gamma_A\gamma^b = -\gamma_A\gamma^b$.
2. If k is even, take b in A; R3 gives $\gamma^b\gamma_A = (-1)^{k-1}\gamma_A\gamma^b = -\gamma_A\gamma^b$.
3. In both cases multiply from the left by $(\gamma^b)^{-1} = \eta^{bb}\gamma^b$: $\gamma_A = -(\gamma^b)^{-1}\gamma_A\gamma^b$.
4. Take the trace and use the cyclic property with the two factors $(\gamma^b)^{-1}$ and $\gamma_A\gamma^b$: $\operatorname{tr}\gamma_A = -\operatorname{tr}(\gamma_A\gamma^b(\gamma^b)^{-1}) = -\operatorname{tr}\gamma_A$.
5. A number equal to its own negative is 0.

Line 1 needs an even number of directions: with an odd number, the product of all of them has odd degree and no direction outside it.

The products are perpendicular and independent. Each γ_A is a product of orthogonal matrices, hence orthogonal ($(MN)^T(MN) = N^T M^T MN = N^T N = I$), so $\gamma_A^T = \gamma_A^{-1} = \pm\gamma_A$ (R2). Then, line by line:

1. $\gamma_A^T \gamma_B = \pm\gamma_A \gamma_B = \pm\gamma_{A\triangle B}$ — the last remark and R1.
2. $\mathrm{tr}(\gamma_A^T \gamma_B) = 0$ for $A \neq B$ — then $A\triangle B$ is not empty, and the trace lemma applies.
3. $\mathrm{tr}(\gamma_A^T \gamma_A) = \mathrm{tr}\, I_{16} = 16$ — γ_A is orthogonal.

The 256 numbers $\mathrm{tr}(\gamma_A^T \gamma_B)$ therefore form the table $16\,\delta_{AB}$; this table is called the **Gram matrix**. Now suppose a combination vanishes, $\sum_B c_B \gamma_B = 0$. Multiplying from the left by γ_A^T and taking the trace gives $\sum_B c_B \,\mathrm{tr}(\gamma_A^T \gamma_B) = 16\, c_A = 0$, so every coefficient is 0: the 256 products are **linearly independent** (no combination of them with coefficients not all zero gives the zero matrix).

They are a basis of all real 16×16 **matrices.** The real 16×16 matrices form a space of **dimension** 256: every matrix is in exactly one way a combination $\sum_{i,j} M_{ij} E_{ij}$ of the 256 matrices E_{ij} that have a single 1 in row i and column j. We quote a fact of linear algebra: in a space of dimension m, any m independent elements form a **basis**, that is, every element of the space is exactly one combination of them. So every real 16×16 matrix M is a combination $M = \sum_A c_A \gamma_A$, and multiplying by γ_A^T and taking the trace gives the coefficients:

$$c_A = \tfrac{1}{16}\,\mathrm{tr}(\gamma_A^T M).$$

Cl(4,4) is the set of all real 16×16 **matrices** (status PROVED; reproduced exactly by Notebook 04c, which finds the **rank** 256 of the 256 products, the rank of a list of matrices being the largest number of independent matrices among them; `Revision/algebra/reports/wolfram-algebra.json`, check `Clifford_basis_spans_full_matrix_algebra`, and `Revision/algebra/reports/python-algebra.json`, check `clifford_products_span_M16`).

Which products are symmetric. A product is symmetric exactly when it squares to $+I_{16}$, because $\gamma_A^T = \gamma_A^{-1}$ and $\gamma_A^{-1} = \pm\gamma_A$ with the sign of the square; it is antisymmetric when it squares to $-I_{16}$. The symmetric 16×16 matrices form a space of dimension $16 \cdot 17/2 = 136$ (the free entries on and above the diagonal) and the antisymmetric ones a space of dimension $16 \cdot 15/2 = 120$ (the free entries above the diagonal). The symmetric products are independent, so there are at most 136 of them; the antisymmetric ones at most 120; together there are $256 = 136 + 120$. So there are exactly 136 symmetric and 120 antisymmetric products (status PROVED; Notebook 04c counts them by degree).

Even products are block diagonal, odd products block off-diagonal. Every gamma is block off-diagonal (Section 4.5). By the block rule (Section 4.2), a product of an even number of block off-diagonal matrices is block diagonal and a product of an odd number is block off-diagonal. The block diagonal real matrices form a space of dimension $64 + 64 = 128$, and the 128 even products are independent block diagonal matrices, so by the quoted fact they are a basis of all block diagonal matrices; in the same way the 128 odd products are a basis of all block off-diagonal matrices. Status PROVED; `Revision/algebra/reports/wolfram-algebra`

`.json`, check `even_subalgebra_dimension`, and `Revision/algebra/reports/python-algebra.json`, check `even_products_span_M8_plus_M8`.

How big must the matrices be? The argument above works for any even number n of $d \times d$ matrices with a Clifford relation, real or complex, if γ_A^{-1} is used in place of γ_A^T: the 2^n products are independent, and independent matrices cannot be more numerous than the dimension d^2 of the space of all $d \times d$ matrices. So $d^2 \geq 2^n$. For Dirac's $n = 4$: $d \geq 4$, and his 4×4 matrices are the smallest possible. For the author's $n = 8$: $d^2 \geq 256$, so $d \geq 16$. **A field on which eight gamma matrices act has at least sixteen components; the author's fields have exactly sixteen** (status PROVED). For an odd number of matrices the trace lemma fails, and fewer components suffice: the author's seven 8×8 matrices $\tau_1, \ldots, \tau_7$ satisfy a Clifford relation (Section 4.5), yet $8^2 = 64 < 2^7 = 128$. The reason is that their product is $\tau_1 \cdots \tau_7 = I_8$ (Section 4.5, Step 2, line 9): multiplying the product τ_A of a set A by the product $\tau_{A'}$ of the remaining directions gives $\pm\tau_1 \cdots \tau_7 = \pm I_8$ (R1), so $\tau_{A'} = \pm\tau_A^{-1} = \pm\tau_A$ (R2), and the 128 products come in pairs that are equal up to sign. Notebook 04c finds that exactly $64 = 8^2$ of them are independent.

The 28 products of two gammas and the matrices S^{ab}. The products of degree 2, $\gamma^a\gamma^b$ with a before b, are $\binom{8}{2} = 28$ matrices. The Revision record names the matrices

$$S^{ab} = \tfrac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a) = \tfrac{1}{4}[\gamma^a, \gamma^b].$$

Line by line, for $a \neq b$: $\gamma^b\gamma^a = -\gamma^a\gamma^b$ (different gammas anticommute); so $\gamma^a\gamma^b - \gamma^b\gamma^a = 2\gamma^a\gamma^b$; so $S^{ab} = \frac{1}{4} \cdot 2\gamma^a\gamma^b = \frac{1}{2}\gamma^a\gamma^b$. For $a = b$ the two terms cancel: $S^{aa} = 0$. Exchanging a and b changes the sign: $S^{ba} = -S^{ab}$. So the 64 matrices S^{ab} are the 28 products of degree 2, halved, their 28 negatives, and 8 zero matrices; their entries are 0 and $\pm\frac{1}{2}$. By R2, $(\gamma^a\gamma^b)^2 = -\eta^{aa}\eta^{bb}I_{16}$. Two directions of the same kind (both space-like or both time-like) have $\eta^{aa}\eta^{bb} = +1$, so their product squares to $-I_{16}$, like the imaginary unit; there are $\binom{4}{2} + \binom{4}{2} = 12$ such pairs, the planes of **rotations**. A space-like and a time-like direction have $\eta^{aa}\eta^{bb} = -1$, so their product squares to $+I_{16}$; there are $4 \cdot 4 = 16$ such pairs, the planes of **boosts** (the transformations that, in special relativity, change the velocity of an observer). All 28 are even products, hence block diagonal. The S^{ab} are the **generators** of the rotations and boosts of the 16-component field: a generator is a matrix from which a whole family of transformations is built, the transformation by a small angle ϵ being $I_{16} + \epsilon S^{ab}$ up to terms with ϵ^2 (Chapter 5 makes this precise). The Revision record checks that they obey the rules of the rotations and boosts of the 4+4 space-time (`Revision/algebra/reports/wolfram-algebra.json`, checks `S_gamma_commutator` and `S_Lorentz_algebra`), and Chapter 5 shows how they generate the groups Spin(4,4) and, with the gammas themselves, Pin(4,4). Status of this paragraph: PROVED; `Revision/algebra/reports/python-algebra.json`, check `S_definition`; `Revision/algebra/reports/wolfram-algebra.json`, checks `S_half_product` and `S_real_entries_in_half_integers`; Notebook 04c compares all 64 matrices S^{ab} entry by entry with the matrices stored in `Revision/algebra/gammas.json` and finds them equal.

The next three sections hold Notebook 04c.

4.14 How to run Notebook 04c

Step 1. What this notebook does and what it needs.

Notebook 04c (The 256 products of the author's gammas: Cl(4,4) is all real 16 by 16 matrices) is the

file `Revision/textbook/notebooks/04c_256_products.ipynb` of the repository Dirac_claude. It forms the 256 ordered products of the author's eight gammas, checks exactly the three rules by which they multiply, their squares and symmetry (136 symmetric, 120 antisymmetric), the trace lemma and their orthogonality, proves that they are linearly independent (exact rank 256), so that the Clifford algebra Cl(4,4) is the set of all real 16 by 16 matrices, writes any matrix as a combination of them, shows that the 128 even products fill the two diagonal 8 by 8 blocks, explains why 16 is the smallest possible size, reproduces the Revision record of these facts, checks that half of each product of two different gammas is the matrix S of the Revision record, and draws eight figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04c_256_products.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04c_256_products.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 04c_256_products.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/04c.captions.json`
- `Revision/textbook/figures/04c_1_products_by_degree.png`
- `Revision/textbook/figures/04c_2_multiplication_table.png`
- `Revision/textbook/figures/04c_3_squares_by_degree.png`
- `Revision/textbook/figures/04c_4_trace_products.png`
- `Revision/textbook/figures/04c_5_coefficients.png`
- `Revision/textbook/figures/04c_6_even_odd_blocks.png`
- `Revision/textbook/figures/04c_7_how_big.png`
- `Revision/textbook/figures/04c_8_two_gamma_products.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all eight figure files exist
ALL 19 CHECKS PASSED (notebook 04c)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/04c_256_products.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 04c_256_products.ipynb
python -m nbconvert --execute --inplace 04c_256_products.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `Revision/algebra/gammas.json` or for a file in `Revision/algebra/reports`: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run `git status` in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

```
git checkout HEAD Revision/algebra
```

4.15 Notebook 04c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/04c_256_products.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The *Clifford algebra* Cl(4,4) of the author's space-time is the set of all real
linear combinations of products of his eight gamma matrices
$\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$. This notebook shows, exactly, that it is the
set of ALL real $16 \times 16$ matrices, and why 16 is the smallest size that eight
such matrices can have. It

- forms the 256 ordered products $\gamma_A$, one for each set $A$ of directions, and
  counts them by their number of factors;
- checks the three rules by which the products multiply, square and pass each other,
  for all pairs; draws the multiplication table of the 16 products of the first
  four gammas;
- finds which products square to $+I_{16}$ (136, the symmetric ones) and which to
  $-I_{16}$ (120, the antisymmetric ones);
- checks the *trace lemma* (every product except $I_{16}$ has trace 0) and that the
  256 products are perpendicular to each other in the trace sense;
- proves that they are linearly independent (exact rank 256 = $16^2$) and writes
  any matrix as a combination of them;
- shows that the 128 even products fill the two diagonal $8 \times 8$ blocks and the
  128 odd ones the two other blocks;
- compares four examples to show why the matrices must be at least $16 \times 16$;
- shows the 28 products of two different gammas, half of which are the matrices
  $S^{ab}$ of the Revision record, and compares them entry by entry with the record;
- reproduces the Revision record of these facts and draws eight figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 04c (The 256 products of the author's gammas: Cl(4,4) is all real 16 by 16
matrices) is the file 'Revision/textbook/notebooks/04c_256_products.ipynb' of the
repository Dirac_claude. It forms the 256 ordered products of the author's eight gammas,
checks exactly the three rules by which they multiply, their squares and symmetry (136
symmetric, 120 antisymmetric), the trace lemma and their orthogonality, proves that they
are linearly independent (exact rank 256), so that the Clifford algebra Cl(4,4) is the
set of all real 16 by 16 matrices, writes any matrix as a combination of them, shows that
the 128 even products fill the two diagonal 8 by 8 blocks, explains why 16 is the
smallest possible size, reproduces the Revision record of these facts, checks that half
```

of each product of two different gammas is the matrix S of the Revision record, and draws eight figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04c_256_products.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04c_256_products.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 04c_256_products.ipynb‘ (in the same folder, without running the cells again) to read the

printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/04c.captions.json‘
- ‘Revision/textbook/figures/04c_1_products_by_degree.png‘
- ‘Revision/textbook/figures/04c_2_multiplication_table.png‘
- ‘Revision/textbook/figures/04c_3_squares_by_degree.png‘
- ‘Revision/textbook/figures/04c_4_trace_products.png‘
- ‘Revision/textbook/figures/04c_5_coefficients.png‘
- ‘Revision/textbook/figures/04c_6_even_odd_blocks.png‘
- ‘Revision/textbook/figures/04c_7_how_big.png‘
- ‘Revision/textbook/figures/04c_8_two_gamma_products.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all eight figure files exist
ALL 19 CHECKS PASSED (notebook 04c)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/04c_256_products.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 04c_256_products.ipynb
      python -m nbconvert --execute --inplace 04c_256_products.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘Revision/algebra/gammas.json‘ or for a file in ‘Revision/algebra/reports‘: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run ‘git status‘ in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

      git checkout HEAD Revision/algebra

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 04c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 04c (The 256 products of the author’s gammas: Cl(4,4) is all real 16 by 16
# matrices) is the file Revision/textbook/notebooks/04c_256_products.ipynb of the
# repository Dirac_claude. It forms the 256 ordered products of the author’s eight
# gammas, checks exactly the three rules by which they multiply, their squares and
# symmetry (136 symmetric, 120 antisymmetric), the trace lemma and their orthogonality,
# proves that they are linearly independent (exact rank 256), so that the Clifford
# algebra Cl(4,4) is the set of all real 16 by 16 matrices, writes any matrix as a
# combination of them, shows that the 128 even products fill the two diagonal 8 by 8
# blocks, explains why 16 is the smallest possible size, reproduces the Revision record
# of these facts, checks that half of each product of two different gammas is the matrix
# S of the Revision record, and draws eight figures. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
```

```
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
```

```
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 04c_256_products.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 04c_256_products.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 04c_256_products.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/04c.captions.json
# - Revision/textbook/figures/04c_1_products_by_degree.png
# - Revision/textbook/figures/04c_2_multiplication_table.png
# - Revision/textbook/figures/04c_3_squares_by_degree.png
# - Revision/textbook/figures/04c_4_trace_products.png
# - Revision/textbook/figures/04c_5_coefficients.png
# - Revision/textbook/figures/04c_6_even_odd_blocks.png
# - Revision/textbook/figures/04c_7_how_big.png
```

```
# - Revision/textbook/figures/04c_8_two_gamma_products.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all eight figure files exist
#     ALL 19 CHECKS PASSED (notebook 04c)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/04c_256_products.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 04c_256_products.ipynb
#     python -m nbconvert --execute --inplace 04c_256_products.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
```

```
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json or for a file in
#   Revision/algebra/reports: the notebook reads these Revision record files of the
#   repository; they are part of every complete clone. Run git status in the repository
#   folder: if it reports them as deleted, restore them with the command below and run
#   the notebook again.
#     git checkout HEAD Revision/algebra
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "04c"  # this notebook: chapter 04, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative
```

```
def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
```

```
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 04c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Set, subset**: a set is a collection of different objects, written in braces,
  for example $A = \{x_2, x_5\}$; a *subset* of a set $S$ is a set whose elements all
  belong to $S$. The empty set $\{\}$ has no element.
- **Binomial coefficient** $\binom{n}{k}$ ("n choose k"): the number of subsets with
  $k$ elements of a set with $n$ elements; $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ with
  $n! = 1 \cdot 2 \cdots n$ and $0! = 1$. For example $\binom{8}{2} = 28$.
- **Product** $\gamma_A$: for a set $A$ of directions, the product of their gamma
  matrices in increasing order, for example
  $\gamma_{\{x_2, x_5\}} = \gamma^{(x_2)}\gamma^{(x_5)}$; for the empty set,
  $\gamma_{\{\}} = I_{16}$. Its **degree** is the number $k$ of factors; the product
  is **even** if $k$ is even and **odd** if $k$ is odd.
- **Symmetric difference** $A \triangle B$: the directions that lie in exactly one of
  the sets $A$ and $B$.
- **Linear combination**: $c_1 M_1 + c_2 M_2 + \dots$ with numbers $c_j$. Matrices
  are **linearly independent** if the only combination that gives the zero matrix
  has all $c_j = 0$. The **span** of a list of matrices is the set of all their
  combinations; a **basis** of a space is an independent list that spans it, and the
  number of its elements is the **dimension** of the space. All real $16 \times 16$
  matrices form a space of dimension $16 \times 16 = 256$ (one number per entry).
- **Rank** of a list of matrices: the largest number of independent matrices in it,
  which is the dimension of its span. sympy computes it exactly with fractions.
```

```
- **Trace** $\mathrm{tr}\,M$: the sum of the diagonal entries. It has the *cyclic
  property* $\mathrm{tr}(MN) = \mathrm{tr}(NM)$, because both are
  $\sum_{i,j} M_{ij}N_{ji}$.
- **Orthogonal matrix**: $M^T M = I$, so $M^{-1} = M^T$.
- **Block diagonal**: a $16 \times 16$ matrix whose upper-right and lower-left
  $8 \times 8$ blocks are zero; **block off-diagonal**: one whose upper-left and
  lower-right blocks are zero.
- **Clifford algebra** Cl(4,4): the set of all real combinations of products of
  eight matrices that satisfy the Clifford relation with four signs $+1$ and four
  signs $-1$.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's gammas satisfy the Clifford relation
$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}I_{16}$ with
$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order of his coordinates
$x_0, \dots, x_7$: space-like hidden direction $x_0$ and 3-space $x_1, x_2, x_3$,
time-like time $x_4$ and extra times $x_5, x_6, x_7$ (the extra times deflate
exponentially in his metric; the gammas, which belong to the local frame, are
constant). Everything below follows from this relation alone; the computer checks
each rule for the author's matrices.

**Rule R1 (products multiply into products).** Two different gammas anticommute and
$\gamma^a\gamma^a = \eta^{aa}I_{16}$. To multiply $\gamma_A\gamma_B$, move each factor
of $\gamma_B$ to the left until it reaches its place in increasing order. Passing a
different gamma gives a factor $-1$; meeting the same gamma gives $\eta^{aa}$. So
$\gamma_A\gamma_B = s(A,B)\,\gamma_{A\triangle B}$ with the sign
$s(A,B) = (-1)^{N(A,B)}\prod_{c \in A\cap B}\eta^{cc}$, where $N(A,B)$ is the number of
pairs $(a, b)$ with $a$ in $A$, $b$ in $B$ and $a$ later than $b$.

**Rule R2 (squares).** By R1 with $B = A$:
$\gamma_A\gamma_A = (-1)^{k(k-1)/2}\prod_{a\in A}\eta^{aa}\,I_{16}$ for a product of
degree $k$ (here $N(A,A) = k(k-1)/2$, the number of pairs in $A$).

**Rule R3 (passing one gamma).** Moving $\gamma^b$ from the left of $\gamma_A$ to its
right passes each of the $k$ factors once: $\gamma^b\gamma_A = (-1)^k\gamma_A\gamma^b$
if $b$ is not in $A$ and $(-1)^{k-1}\gamma_A\gamma^b$ if it is.

**The trace lemma.** For eight gammas (an even number) every $\gamma_A$ with $A$ not
empty has trace 0. Proof, line by line: if $k$ is odd, then $k \le 7$, so some $b$ is
not in $A$, and R3 gives $\gamma^b\gamma_A = -\gamma_A\gamma^b$; if $k$ is even, take
$b$ in $A$, and R3 gives again $\gamma^b\gamma_A = -\gamma_A\gamma^b$. Multiply from
the left by $(\gamma^b)^{-1} = \eta^{bb}\gamma^b$:
$\gamma_A = -(\gamma^b)^{-1}\gamma_A\gamma^b$. Take the trace and use the cyclic
property: $\mathrm{tr}\,\gamma_A = -\mathrm{tr}(\gamma_A\gamma^b(\gamma^b)^{-1})
= -\mathrm{tr}\,\gamma_A$. A number equal to its negative is 0.

**Independence.** Each $\gamma_A$ is a product of orthogonal matrices, hence
orthogonal: $\gamma_A^T = \gamma_A^{-1} = \pm\gamma_A$ (the sign of R2). So
$\gamma_A^T\gamma_B = \pm\gamma_A\gamma_B = \pm\gamma_{A\triangle B}$ (R1), whose trace
is 0 unless $A = B$ (trace lemma), and $\mathrm{tr}(\gamma_A^T\gamma_A) =
```

```
\mathrm{tr}\,I_{16} = 16$. If $\sum_B c_B\gamma_B = 0$, multiply by $\gamma_A^T$ and
take the trace: $16\,c_A = 0$. So all coefficients vanish: the 256 products are
independent. Since the real $16 \times 16$ matrices form a space of dimension
$256$, the products are a basis of it: **Cl(4,4) is all real $16 \times 16$
matrices**, and every matrix $M$ is $\sum_A c_A\gamma_A$ with
$c_A = \mathrm{tr}(\gamma_A^T M)/16$.

**How big.** The same argument works for any even number $n$ of $d \times d$ matrices
with a Clifford relation, real or complex, if $\gamma_A^{-1}$ is used in place of
$\gamma_A^T$: by R2 every product squares to $\pm 1$, so $\gamma_A^{-1} = \pm\gamma_A$,
and $\mathrm{tr}(\gamma_A^{-1}\gamma_B)$ is 0 for $A \neq B$ and $d$ for $A = B$. So
the $2^n$ products are independent and must fit into the space of all $d \times d$
matrices, of dimension $d^2$: $d^2 \ge 2^n$. For $n = 8$: $d \ge 16$. A field on
which eight gammas act therefore has at least 16 components; the author's field
has exactly 16.
```

Text cell:

```
## 5. The 256 products

The next cell reads the gammas from the Revision record file
‘Revision/algebra/gammas.json‘ (eight matrices in the order $x_0, \dots, x_7$, whole
numbers), then forms the 256 products. ‘itertools.combinations(range(8), k)‘ lists the
sets of $k$ directions in increasing order (direction index 0 is $x_0$, ..., 7 is
$x_7$), for $k = 0, 1, \dots, 8$, which is also the order of the Revision verifier.
The products are kept in one numpy array ‘products‘ of shape (256, 16, 16): product
number $i$ is ‘products[i]‘. The cell then counts the products of each degree and
compares with the binomial coefficients $\binom{8}{k}$.
```

In [2]:

```
import itertools  # all subsets of a given size
from math import comb  # comb(n, k) is the binomial coefficient n choose k

import numpy as np
import sympy as sp
from matplotlib.colors import BoundaryNorm, ListedColormap
from sympy.polys.matrices import DomainMatrix  # exact matrices for the rank

record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
eta = record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1] in the order x0..x7
gamma = [np.array(matrix, dtype=np.int64) for matrix in record["gamma"]]
I16 = np.eye(16, dtype=np.int64)

SETS = [s for k in range(9) for s in itertools.combinations(range(8), k)]
INDEX = {s: i for i, s in enumerate(SETS)}  # set -> its number in the list
products = np.zeros((256, 16, 16), dtype=np.int64)
for i, s in enumerate(SETS):
    matrix = I16
    for a in s:  # multiply the gammas of the set in increasing order
        matrix = matrix @ gamma[a]
```

```
    products[i] = matrix
DEGREE = np.array([len(s) for s in SETS])  # the number of factors of each product


def label(s):
    """A short name of a set: its coordinate numbers, e.g. (1, 4) -> "14"."""
    return "".join(str(a) for a in s) if s else "I"


counts = [int(np.sum(DEGREE == k)) for k in range(9)]
report("number of products of degree 0, 1, ..., 8", counts)
report("even and odd products", (sum(counts[0::2]), sum(counts[1::2])))
check(counts == [comb(8, k) for k in range(9)] and sum(counts) == 256
      and sum(counts[0::2]) == 128,
      "the products by degree are 1, 8, 28, 56, 70, 56, 28, 8, 1 (128 even)")
```

Out [2]:

```
RESULT number of products of degree 0, 1, ..., 8 = [1, 8, 28, 56, 70, 56, 28, 8, 1]
RESULT even and odd products = (128, 128)
PASS the products by degree are 1, 8, 28, 56, 70, 56, 28, 8, 1 (128 even)
```

Text cell:

```
The next cell draws these numbers as a bar chart, even degrees in one colour and odd
degrees in another, with each number written above its bar.
```

In [3]:

```
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
colours = ["#2a78d6" if k % 2 == 0 else "#eb6834" for k in range(9)]
bars = ax.bar(range(9), counts, color=colours, width=0.7)
ax.bar_label(bars, labels=[str(n) for n in counts], padding=2, fontsize=9)
ax.set_xticks(range(9))
ax.set_xlabel("degree $k$ (number of gamma factors)")
ax.set_ylabel("number of products")
ax.set_ylim(0, 80)
ax.legend(handles=[bars[0], bars[1]], labels=["even degree (128 in all)",
                                              "odd degree (128 in all)"], fontsize=9)
save_figure(fig, "products_by_degree",
            "The 256 ordered products of the author's eight gammas, counted by their "
            "degree $k$, the number of gamma factors (horizontal axis $k$ from 0 to "
            "8, vertical axis the number of products). The counts are the binomial "
            "coefficients $\\binom{8}{k}$ = 1, 8, 28, 56, 70, 56, 28, 8, 1; the even "
            "degrees (blue) and the odd degrees (orange) contribute 128 products "
            "each.")
```

Out [3]:

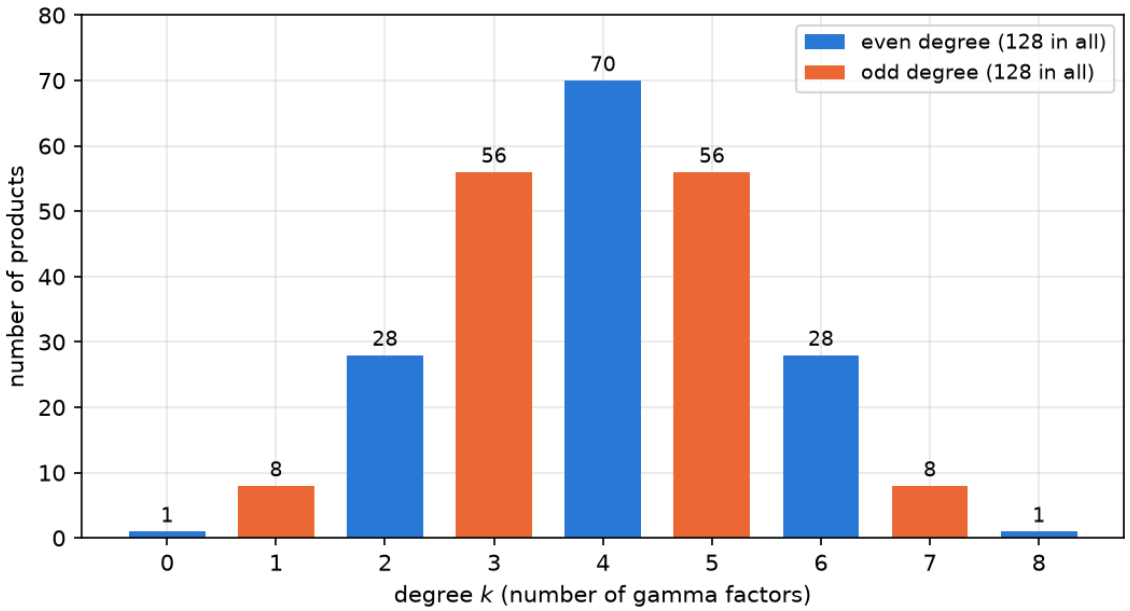

Figure 128: The 256 ordered products of the author's eight gammas, counted by their degree k, the number of gamma factors (horizontal axis k from 0 to 8, vertical axis the number of products). The counts are the binomial coefficients $\binom{8}{k} = 1, 8, 28, 56, 70, 56, 28, 8, 1$; the even degrees (blue) and the odd degrees (orange) contribute 128 products each. (Notebook 04c, figure 1.)

```
Figure 04c.1 saved as Revision/textbook/figures/04c_1_products_by_degree.png
```

Text cell:

```
## 6. Rule R1: the products multiply into products

The next cell multiplies every product with every product (256 times 256 = 65536
multiplications; `products[i] @ products` multiplies product $i$ with all 256 at
once) and checks that each result is $+\gamma_{A\triangle B}$ or
$-\gamma_{A\triangle B}$, with exactly the sign $s(A,B)$ of rule R1. It also checks
that every product is a signed permutation matrix (one entry $\pm 1$ in each row and
column), hence orthogonal.
```

In [4]:

```
def sign_formula(A, B):
    """s(A, B) of rule R1: (-1)^N(A, B) times the signs eta of the common directions."""
    later_pairs = sum(1 for a in A for b in B if a > b)  # N(A, B)
    sign = (-1) ** later_pairs
    for c in set(A) & set(B):  # & : the directions in both sets
        sign *= eta[c]
    return sign


rule_r1 = True
```

```
SIGN = np.zeros((256, 256), dtype=np.int64)  # SIGN[i, j] = s(A_i, A_j)
for i, A in enumerate(SETS):
    row = products[i] @ products  # gamma_A times each of the 256 products
    for j, B in enumerate(SETS):
        target = products[INDEX[tuple(sorted(set(A) ^ set(B)))]]  # ^ : symmetric diff.
        SIGN[i, j] = sign_formula(A, B)
        rule_r1 &= bool((row[j] == SIGN[i, j] * target).all())
report("pairs with sign +1 and with sign -1",
       (int(np.sum(SIGN == 1)), int(np.sum(SIGN == -1))))
check(rule_r1, "gamma_A gamma_B = s(A,B) gamma_(A sym. diff. B) for all 65536 pairs")
signed_permutations = all(
    (np.count_nonzero(m, axis=0) == 1).all() and (np.count_nonzero(m, axis=1) == 1).all()
    and set(np.unique(m)) <= {-1, 0, 1} for m in products[1:])  # <= : subset
check(signed_permutations and all((m.T @ m == I16).all() for m in products),
      "every product is a signed permutation matrix, hence orthogonal")
```

Out [4]:

```
RESULT pairs with sign +1 and with sign -1 = (33024, 32512)
PASS gamma_A gamma_B = s(A,B) gamma_(A sym. diff. B) for all 65536 pairs
PASS every product is a signed permutation matrix, hence orthogonal
```

Text cell:

```
The next cell draws the multiplication table of the 16 products of the first four
gammas $\gamma^{(x_1)}, \gamma^{(x_2)}, \gamma^{(x_3)}, \gamma^{(x_4)}$ (the three
directions of 3-space and the time): row $\gamma_A$, column $\gamma_B$, and in the
square the name of $A \triangle B$ (the coordinate numbers, I for the identity), red
for the sign $+1$ and blue for $-1$. The 16 products of four gammas multiply among
themselves; this is the Clifford algebra of four directions, the algebra of Dirac's
matrices.
```

In [5]:

```
four = [s for s in SETS if set(s) <= {1, 2, 3, 4}]  # the 16 sets inside x1..x4
table = np.array([[SIGN[INDEX[A], INDEX[B]] for B in four] for A in four])
fig, ax = plt.subplots(figsize=(7.4, 7.0), layout="constrained")
ax.imshow(table, cmap=ListedColormap(["#2a78d6", "#e34948"]),
          norm=BoundaryNorm([-2, 0, 2], 2))
for r, A in enumerate(four):
    for c_, B in enumerate(four):
        name = label(tuple(sorted(set(A) ^ set(B))))
        ax.text(c_, r, ("+" if table[r, c_] > 0 else "-") + name, ha="center",
                va="center", color="white", fontsize=7)
names = [label(s) for s in four]
ax.set_xticks(range(16), names, fontsize=8)
ax.set_yticks(range(16), names, fontsize=8)
ax.set_xlabel("second factor $\\gamma_B$ (coordinate numbers of $B$)")
ax.set_ylabel("first factor $\\gamma_A$")
ax.grid(False)
save_figure(fig, "multiplication_table",
```

```
"The multiplication table of the 16 products of the gammas of 3-space "
"and time, $\\gamma^{(x_1)}, \\gamma^{(x_2)}, \\gamma^{(x_3)}, "
"\\gamma^{(x_4)}$: row $\\gamma_A$, column $\\gamma_B$, both named by the "
"numbers of their coordinates (I is the identity). Each square holds "
"$\\gamma_A\\gamma_B = \\pm\\gamma_{A \\triangle B}$: the sign (red $+$, "
"blue $-$) and the name of $A \\triangle B$, the directions in exactly "
"one of $A$ and $B$. The diagonal holds the squares: $+$I or $-$I, "
"following rule R2.")
```

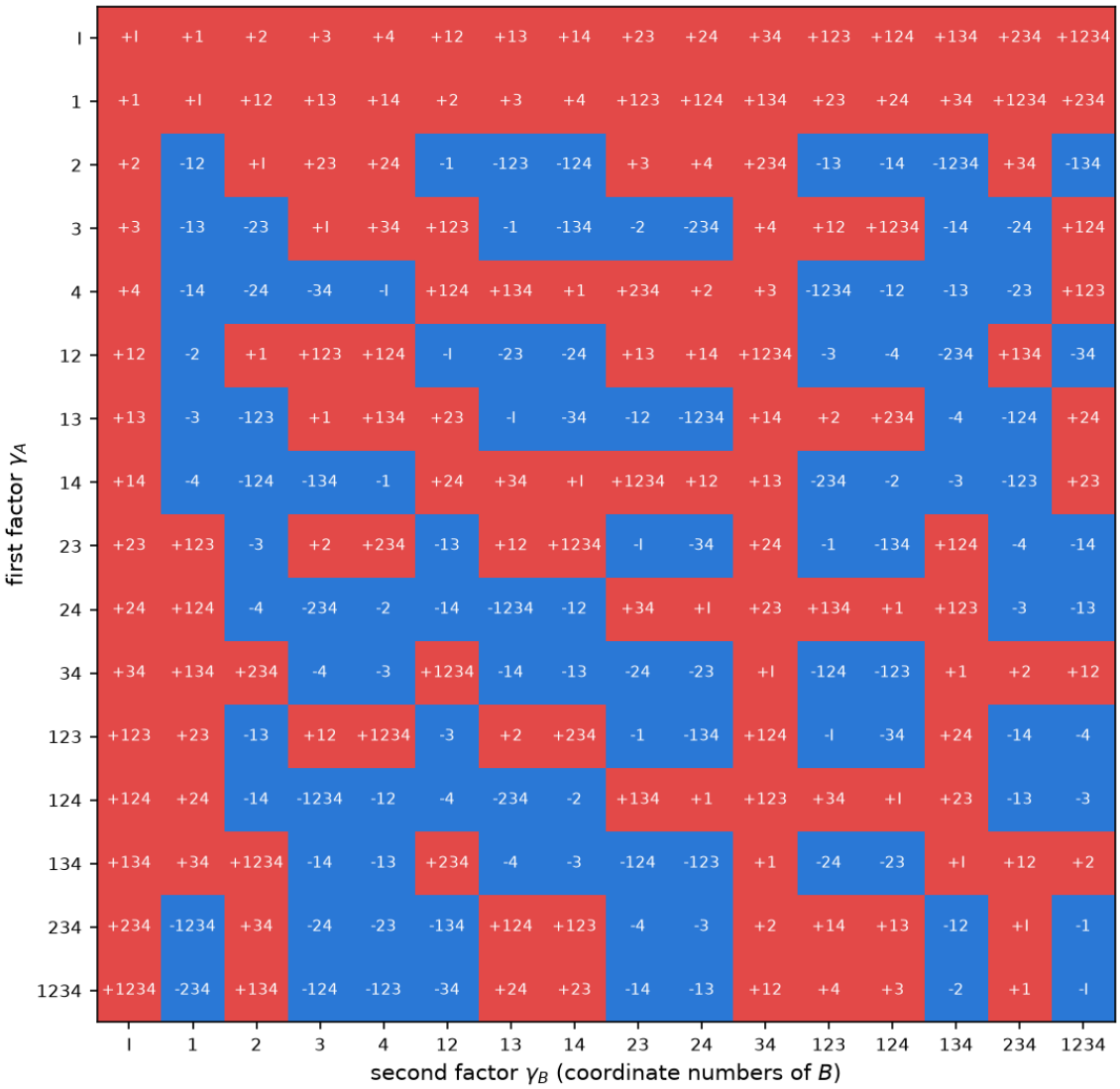

Figure 129: The multiplication table of the 16 products of the gammas of 3-space and time, $\gamma^{(x_1)}, \gamma^{(x_2)}, \gamma^{(x_3)}, \gamma^{(x_4)}$: row γ_A, column γ_B, both named by the numbers of their coordinates (I is the identity). Each square holds $\gamma_A\gamma_B = \pm\gamma_{A\triangle B}$: the sign (red $+$, blue $-$) and the name of $A\triangle B$, the directions in exactly one of A and B. The diagonal holds the squares: $+$I or $-$I, following rule R2. (Notebook 04c, figure 2.)

Out [5]:

```
Figure 04c.2 saved as Revision/textbook/figures/04c_2_multiplication_table.png
```

Text cell:

```
## 7. Rule R2: the squares, and which products are symmetric

The next cell checks rule R2 for all 256 products, counts how many square to
$+I_{16}$ and how many to $-I_{16}$, and checks that a product is symmetric exactly
when its square is $+I_{16}$ (and antisymmetric when it is $-I_{16}$). The reason:
$\gamma_A^T = \gamma_A^{-1}$ (orthogonal), and $\gamma_A^{-1} = \pm\gamma_A$ with the
sign of the square. The symmetric $16 \times 16$ matrices form a space of dimension
$16 \cdot 17/2 = 136$ (the free entries on and above the diagonal) and the
antisymmetric ones a space of dimension $16 \cdot 15/2 = 120$; the counts below fill
both spaces exactly.
```

In [6]:

```
rule_r2 = True
square_sign = np.zeros(256, dtype=np.int64)
for i, s in enumerate(SETS):
    k = len(s)
    predicted = (-1) ** (k * (k - 1) // 2)  # // : division without remainder
    for a in s:
        predicted *= eta[a]
    square_sign[i] = predicted
    rule_r2 &= bool((products[i] @ products[i] == predicted * I16).all())
symmetric = np.array([(m.T == m).all() for m in products])
antisymmetric = np.array([(m.T == -m).all() for m in products])
plus, minus = int(np.sum(square_sign == 1)), int(np.sum(square_sign == -1))
report("products with square +I16 and with square -I16", (plus, minus))
check(rule_r2, "gamma_A^2 = (-1)^(k(k-1)/2) prod(eta) I16 for all 256 products")
check(plus == 136 == 16 * 17 // 2 and minus == 120 == 16 * 15 // 2
      and (symmetric == (square_sign == 1)).all()
      and (antisymmetric == (square_sign == -1)).all(),
      "136 products are symmetric (square +I16), 120 antisymmetric (square -I16)")
```

Out [6]:

```
RESULT products with square +I16 and with square -I16 = (136, 120)
PASS gamma_A^2 = (-1)^(k(k-1)/2) prod(eta) I16 for all 256 products
PASS 136 products are symmetric (square +I16), 120 antisymmetric (square -I16)
```

Text cell:

```
The next cell draws, for each degree, how many products square to $+I_{16}$ and how
many to $-I_{16}$, as stacked bars.
```

In [7]:

```
plus_by_degree = [int(np.sum((DEGREE == k) & (square_sign == 1))) for k in range(9)]
minus_by_degree = [int(np.sum((DEGREE == k) & (square_sign == -1))) for k in range(9)]
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
low = ax.bar(range(9), plus_by_degree, color="#e34948", width=0.7,
             label="square $+I_{16}$ (symmetric)")
high = ax.bar(range(9), minus_by_degree, bottom=plus_by_degree, color="#2a78d6",
              width=0.7, label="square $-I_{16}$ (antisymmetric)")
# numbers inside the bars that are tall enough, above the two bars of height 1
ax.bar_label(low, labels=[str(n) if n > 2 else "" for n in plus_by_degree],
             label_type="center", color="white", fontsize=8)
ax.bar_label(high, labels=[str(n) if n else "" for n in minus_by_degree],
             label_type="center", color="white", fontsize=8)
for k in (0, 8):
    ax.text(k, plus_by_degree[k] + 1.5, str(plus_by_degree[k]), ha="center",
            fontsize=8, color="#0b0b0b")
ax.set_xticks(range(9))
ax.set_xlabel("degree $k$")
ax.set_ylabel("number of products")
ax.legend(fontsize=9)
save_figure(fig, "squares_by_degree",
            "The squares of the 256 products by degree $k$ (horizontal axis): red, "
            "the number of products $\\gamma_A$ with $\\gamma_A^2 = +I_{16}$, which "
            "are exactly the symmetric ones; blue, the number with "
            "$\\gamma_A^2 = -I_{16}$, the antisymmetric ones. The sign follows rule "
            "R2: $(-1)^{k(k-1)/2}$ times the signs $\\eta$ of the directions in $A$. "
            "In total 136 = $16 \\cdot 17/2$ symmetric and 120 = $16 \\cdot 15/2$ "
            "antisymmetric products, exactly the dimensions of the symmetric and "
            "antisymmetric $16 \\times 16$ matrices.")
```

Out [7]:

```
Figure 04c.3 saved as Revision/textbook/figures/04c_3_squares_by_degree.png
```

Text cell:

```
## 8. The trace lemma and the trace products

The next cell checks the trace lemma for the 255 products other than $I_{16}$, and
computes the $256 \times 256$ table of *trace products*
$G_{AB} = \mathrm{tr}(\gamma_A^T\gamma_B)$ (it is called the Gram matrix). The
situation section proved $G = 16\,I_{256}$: every product is perpendicular to every
other one. numpy computes all 65536 entries at once: ‘flat‘ holds each product as one
row of 256 numbers, and $\mathrm{tr}(\gamma_A^T\gamma_B) = \sum_{ij}(\gamma_A)_{ij}
(\gamma_B)_{ij}$ is the product of row $A$ of ‘flat‘ with row $B$, so
‘flat @ flat.T‘ is the whole table. The cell also computes the table without the
transpose, $\mathrm{tr}(\gamma_A\gamma_B)$, which is $16$ times the sign of the
square on the diagonal and 0 elsewhere.
```

In [8]:

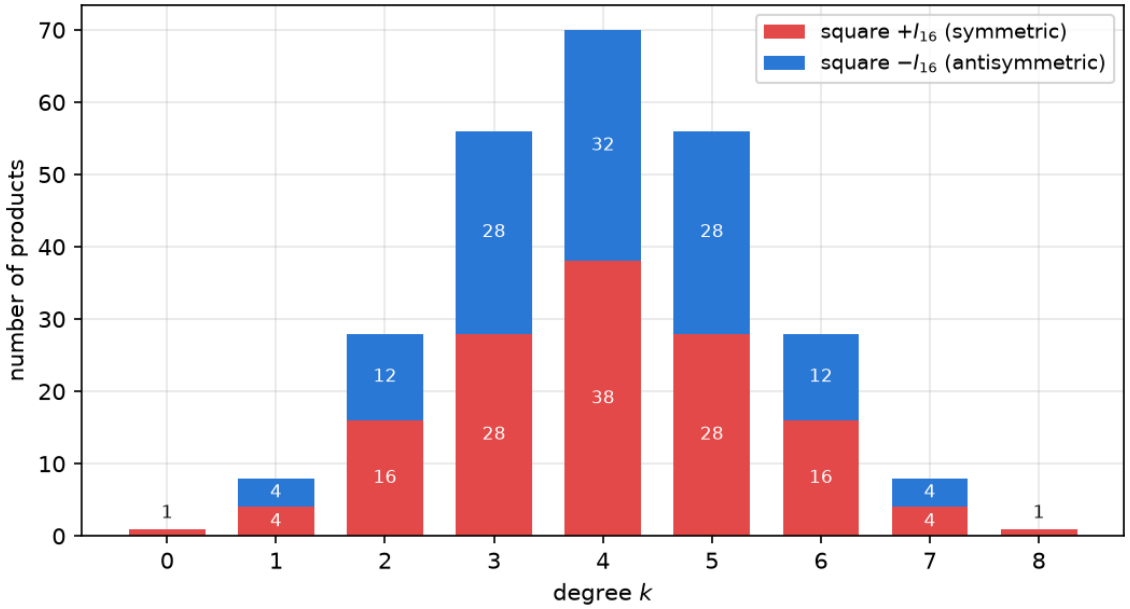

Figure 130: The squares of the 256 products by degree k (horizontal axis): red, the number of products γ_A with $\gamma_A^2 = +I_{16}$, which are exactly the symmetric ones; blue, the number with $\gamma_A^2 = -I_{16}$, the antisymmetric ones. The sign follows rule R2: $(-1)^{k(k-1)/2}$ times the signs η of the directions in A. In total $136 = 16 \cdot 17/2$ symmetric and $120 = 16 \cdot 15/2$ antisymmetric products, exactly the dimensions of the symmetric and antisymmetric 16×16 matrices. (Notebook 04c, figure 3.)

```
traces = np.array([np.trace(m) for m in products])
check(traces[0] == 16 and (traces[1:] == 0).all(),
      "trace lemma: tr gamma_A = 0 for all 255 products other than I16")
flat = products.reshape(256, 256)  # each product as one row of 256 entries
gram = flat @ flat.T  # gram[A, B] = tr(gamma_A^T gamma_B), exact whole numbers
# tr(gamma_A gamma_B) = sum_ij (gamma_A)_ij (gamma_B^T)_ij: the same with the
# transposed products in the second factor.
flat_transposed = products.transpose(0, 2, 1).reshape(256, 256)
without_transpose = flat @ flat_transposed.T
check((gram == 16 * np.eye(256, dtype=np.int64)).all(),
      "tr(gamma_A^T gamma_B) = 16 for A = B and 0 otherwise (all 65536 pairs)")
check((without_transpose == 16 * np.diag(square_sign)).all(),
      "tr(gamma_A gamma_B) = 16 times the sign of the square for A = B, else 0")
```

Out [8]:

```
PASS trace lemma: tr gamma_A = 0 for all 255 products other than I16
PASS tr(gamma_A^T gamma_B) = 16 for A = B and 0 otherwise (all 65536 pairs)
PASS tr(gamma_A gamma_B) = 16 times the sign of the square for A = B, else 0
```

Text cell:

```
The next cell draws both tables (divided by 16) as heat maps: on the left the whole
table $\mathrm{tr}(\gamma_A^T\gamma_B)/16$ for the 256 products (numbered 0 to 255 in
the order of degree), on the right a magnified corner of the table
$\mathrm{tr}(\gamma_A\gamma_B)/16$: the 37 products of degree 0, 1 and 2, each named
by the numbers of its coordinates. In the corner one can read off the squares: $+1$
for $I_{16}$ and for the space-like $\gamma^{(x_0)}, \gamma^{(x_1)}, \gamma^{(x_2)},
\gamma^{(x_3)}$, $-1$ for the time-like ones, and for a product of two gammas
$(\gamma^a\gamma^b)^2 = -\eta^{aa}\eta^{bb}$ (rule R2 with $k = 2$).
```

In [9]:

```
starts = [int(np.argmax(DEGREE == k)) for k in range(9)]  # first product of degree k
three = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -1, 0, +1
three_norm = BoundaryNorm([-1.5, -0.5, 0.5, 1.5], 3)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 4.8), layout="constrained")
left.imshow(gram // 16, cmap=three, norm=three_norm, interpolation="nearest")
left.set_xticks(range(0, 256, 64))
left.set_yticks(range(0, 256, 64))
left.set_xlabel("number of the product $\\gamma_B$")
left.set_ylabel("number of the product $\\gamma_A$")
left.set_title("$\\mathrm{tr}(\\gamma_A^T\\gamma_B)/16$, all 256 products")
left.grid(False)
corner = starts[3]  # 37: the products of degree 0, 1 and 2
right.imshow(without_transpose[:corner, :corner] // 16, cmap=three, norm=three_norm)
names = [label(s) for s in SETS[:corner]]
right.set_xticks(range(corner), names, fontsize=5, rotation=90)
right.set_yticks(range(corner), names, fontsize=5)
right.set_title("$\\mathrm{tr}(\\gamma_A\\gamma_B)/16$, degrees 0, 1, 2")
right.grid(False)
```

```
save_figure(fig, "trace_products",
            "The trace products of the products of the author's gammas (red $+1$, "
            "blue $-1$, grey $0$). Left: "
            "$\\mathrm{tr}(\\gamma_A^T\\gamma_B)/16$ for all 256 products, numbered "
            "0 to 255 in the order of their degree; it is 1 on the diagonal and 0 "
            "everywhere else, so the 256 products are perpendicular to each other "
            "and therefore independent. Right: a magnified corner of "
            "$\\mathrm{tr}(\\gamma_A\\gamma_B)/16$, the 37 products of degree 0, 1 "
            "and 2 named by their coordinate numbers (I is the identity); it is "
            "zero off the diagonal, and its diagonal shows the square of each "
            "product: $+1$ for I, 0, 1, 2, 3, $-1$ for 4, 5, 6, 7, and "
            "$-\\eta^{aa}\\eta^{bb}$ for a product $\\gamma^a\\gamma^b$.")
```

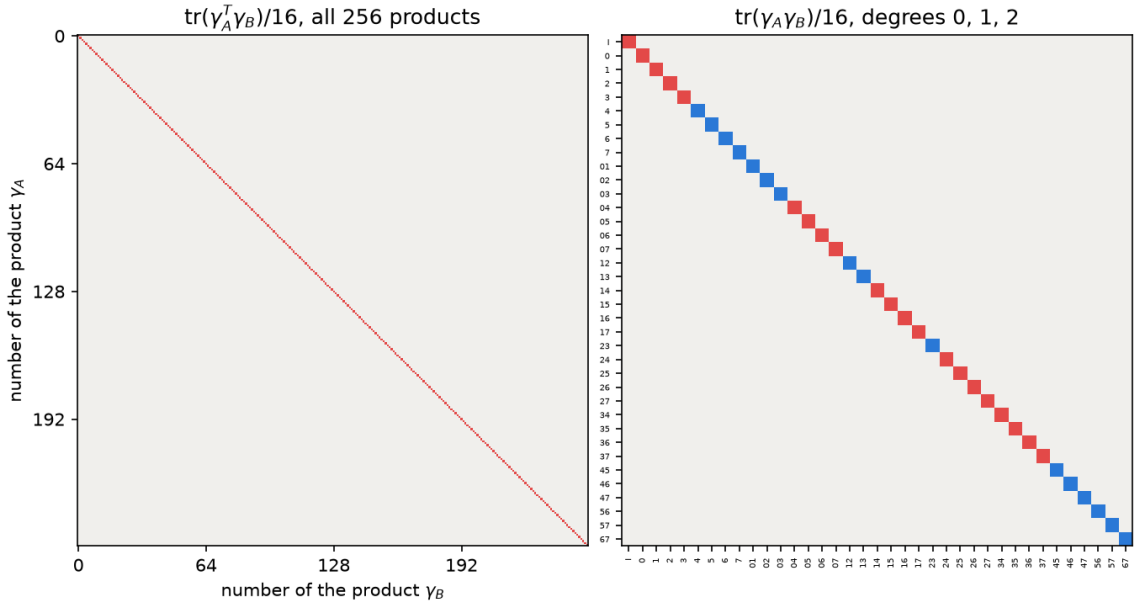

Figure 131: The trace products of the products of the author's gammas (red $+1$, blue -1, grey 0). Left: $\mathrm{tr}(\gamma_A^T\gamma_B)/16$ for all 256 products, numbered 0 to 255 in the order of their degree; it is 1 on the diagonal and 0 everywhere else, so the 256 products are perpendicular to each other and therefore independent. Right: a magnified corner of $\mathrm{tr}(\gamma_A\gamma_B)/16$, the 37 products of degree 0, 1 and 2 named by their coordinate numbers (I is the identity); it is zero off the diagonal, and its diagonal shows the square of each product: $+1$ for I, 0, 1, 2, 3, -1 for 4, 5, 6, 7, and $-\eta^{aa}\eta^{bb}$ for a product $\gamma^a\gamma^b$. (Notebook 04c, figure 4.)

Out [9]:

```
Figure 04c.4 saved as Revision/textbook/figures/04c_4_trace_products.png
```

Text cell:

```
## 9. Linear independence: exact rank 256

The trace argument already proves independence. The Revision record computed the
```

rank directly, by exact elimination with fractions. The next cell does the same with sympy's ‘DomainMatrix‘ over the rational numbers QQ (exact fractions, no rounding): the 256 products, each written as a row of 256 numbers, have rank 256. It also checks that the rank appears in the detail texts of the two Revision checks, exactly as they state it.

In [10]:

```
WOLFRAM = "Revision/algebra/reports/wolfram-algebra.json"
PYTHON = "Revision/algebra/reports/python-algebra.json"
RECORDED = {}
for report_file in (WOLFRAM, PYTHON):
    text = repository_file(report_file).read_text(encoding="utf-8")
    RECORDED[report_file] = {e["name"]: e for e in json.loads(text)["checks"]}


def recorded(report_file, name):
    """The detail text of a Revision check; it must be recorded as passed."""
    entry = RECORDED[report_file][name]
    if entry["verdict"].upper() != "PASS":
        raise AssertionError(f"{report_file}: {name} is not recorded as passed")
    return entry["detail"]


def exact_rank(rows):
    """The exact rank of a list of rows of whole numbers (fractions, over QQ)."""
    matrix = DomainMatrix([[sp.ZZ(int(x)) for x in row] for row in rows],
                          (len(rows), len(rows[0])), sp.ZZ)
    return matrix.convert_to(sp.QQ).rank()


rank_all = exact_rank(flat.tolist())
report("exact rank of the 256 products", rank_all)
check(rank_all == 256
      and f"span dimension {rank_all} = 16^2" in recorded(
          WOLFRAM, "Clifford_basis_spans_full_matrix_algebra")
      and f"{rank_all} (sympy DomainMatrix over QQ)" in recorded(
          PYTHON, "clifford_products_span_M16"),
      "the 256 products are independent: Cl(4,4) is all real 16 x 16 matrices")
print(f"     reproduces {WOLFRAM}")
print("         check Clifford_basis_spans_full_matrix_algebra")
print(f"     reproduces {PYTHON}")
print("         check clifford_products_span_M16")
```

Out [10]:

```
RESULT exact rank of the 256 products = 256
PASS the 256 products are independent: Cl(4,4) is all real 16 x 16 matrices
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check Clifford_basis_spans_full_matrix_algebra
     reproduces Revision/algebra/reports/python-algebra.json
         check clifford_products_span_M16
```

Text cell:

```
## 10. Every matrix is a combination of the products

Because the products are a basis, every real $16 \times 16$ matrix $M$ equals
$\sum_A c_A\gamma_A$, and the trace products give the coefficients:
$c_A = \mathrm{tr}(\gamma_A^T M)/16$ (multiply $M = \sum_B c_B\gamma_B$ by
$\gamma_A^T$ and take the trace). The next cell does this for two examples: the
matrix $E_{00}$ with a single 1 in row 0 and column 0, and a matrix of random whole
numbers between $-3$ and $3$ (seed 12345). It computes $16\,c_A$ (whole numbers, so
the arithmetic stays exact), rebuilds $16M = \sum_A (16c_A)\gamma_A$ and checks that
it is exactly $16M$. For $E_{00}$ only 16 coefficients are nonzero, each $\pm 1/16$:
they belong to the 16 products that are diagonal matrices.
```

In [11]:

```
e00 = np.zeros((16, 16), dtype=np.int64)
e00[0, 0] = 1
generator = np.random.default_rng(12345)
random_matrix = generator.integers(-3, 4, size=(16, 16))  # whole numbers -3..3
coefficients = {}
for name, matrix in (("E00", e00), ("random", random_matrix)):
    sixteen_c = flat @ matrix.reshape(256)  # 16 c_A = tr(gamma_A^T M) for all A
    rebuilt = np.einsum("a,aij->ij", sixteen_c, products)  # sum_A (16 c_A) gamma_A
    coefficients[name] = sixteen_c / 16.0
    check((rebuilt == 16 * matrix).all(),
          f"the matrix {name} equals sum_A c_A gamma_A with c_A = tr(gamma_A^T M)/16")
nonzero = [label(SETS[i]) for i in np.flatnonzero(coefficients["E00"])]
diagonal = [label(s) for i, s in enumerate(SETS)
            if (products[i] == np.diag(np.diag(products[i]))).all()]
say("the nonzero coefficients of E00 belong to: " + ", ".join(nonzero))
check(nonzero == diagonal and len(nonzero) == 16
      and set(np.abs(coefficients["E00"][np.flatnonzero(coefficients["E00"])]))
      == {1 / 16},
      "E00 has 16 coefficients +-1/16, on the 16 diagonal products")
```

Out [11]:

```
PASS the matrix E00 equals sum_A c_A gamma_A with c_A = tr(gamma_A^T M)/16
PASS the matrix random equals sum_A c_A gamma_A with c_A = tr(gamma_A^T M)/16
the nonzero coefficients of E00 belong to: I, 07, 16, 25, 34, 0167, 0257, 0347, 1256,
    1346, 2345, 012567, 013467, 023457, 123456, 01234567
PASS E00 has 16 coefficients +-1/16, on the 16 diagonal products
```

Text cell:

```
The next cell draws the 256 coefficients of both examples, product number on the
horizontal axis (ordered by degree as before).
```

In [12]:

```
fig, (top, bottom) = plt.subplots(2, 1, figsize=(8.0, 5.6), layout="constrained",
                                  sharex=True)
for ax, name, colour in ((top, "E00", "#2a78d6"), (bottom, "random", "#eb6834")):
    ax.bar(range(256), coefficients[name], color=colour, width=0.8)
    ax.axhline(0.0, color="#898781", linewidth=0.8)
    ax.set_ylabel("coefficient $c_A$")
    for start in starts[1:]:
        ax.axvline(start - 0.5, color="#c3c2b7", linewidth=0.6, linestyle=":")
top.set_title("$E_{00}$ (a single 1 in row 0, column 0)")
bottom.set_title("a matrix of random whole numbers from $-3$ to $3$")
bottom.set_xlabel("number of the product $\\gamma_A$ (dotted lines: degree changes)")
save_figure(fig, "coefficients",
            "Every real $16 \\times 16$ matrix is a combination "
            "$\\sum_A c_A\\gamma_A$ of the 256 products, with "
            "$c_A = \\mathrm{tr}(\\gamma_A^T M)/16$; horizontal axis the number of "
            "the product (ordered by degree, dotted lines where the degree changes), "
            "vertical axis the coefficient $c_A$ (pure numbers). Top: the matrix "
            "$E_{00}$ with a single entry 1 needs only the 16 products that are "
            "diagonal matrices, each with coefficient $\\pm 1/16$. Bottom: a matrix "
            "of random whole numbers needs almost all 256 products.")
```

Out [12]:

```
Figure 04c.5 saved as Revision/textbook/figures/04c_5_coefficients.png
```

Text cell:

```
## 11. Even products fill the diagonal blocks, odd products the others

Every gamma has zero upper-left and lower-right $8 \times 8$ blocks (it is block
off-diagonal). By the block rule a product of two block off-diagonal matrices is
block diagonal, and a product of a block diagonal and a block off-diagonal matrix is
block off-diagonal. So every even product is block diagonal and every odd one block
off-diagonal. The block diagonal real matrices form a space of dimension
$64 + 64 = 128$ and so do the block off-diagonal ones; the 128 even products are
independent (they are part of the 256), so they span all block diagonal matrices
(Mat(8) + Mat(8)), and the odd ones all block off-diagonal matrices. The next cell
checks the block forms and the two exact ranks, reproduces the Revision record, and
counts at every position $(i, j)$ how many even (and odd) products have a nonzero
entry there.
```

In [13]:

```
even, odd = flat[DEGREE % 2 == 0], flat[DEGREE % 2 == 1]  # rows of 256 numbers
even_blocks = all((m[:8, 8:] == 0).all() and (m[8:, :8] == 0).all()
                  for m in products[DEGREE % 2 == 0])
odd_blocks = all((m[:8, :8] == 0).all() and (m[8:, 8:] == 0).all()
                 for m in products[DEGREE % 2 == 1])
rank_even, rank_odd = exact_rank(even.tolist()), exact_rank(odd.tolist())
report("exact ranks of the even and of the odd products", (rank_even, rank_odd))
check(even_blocks and odd_blocks and rank_even == 128 and rank_odd == 128
```

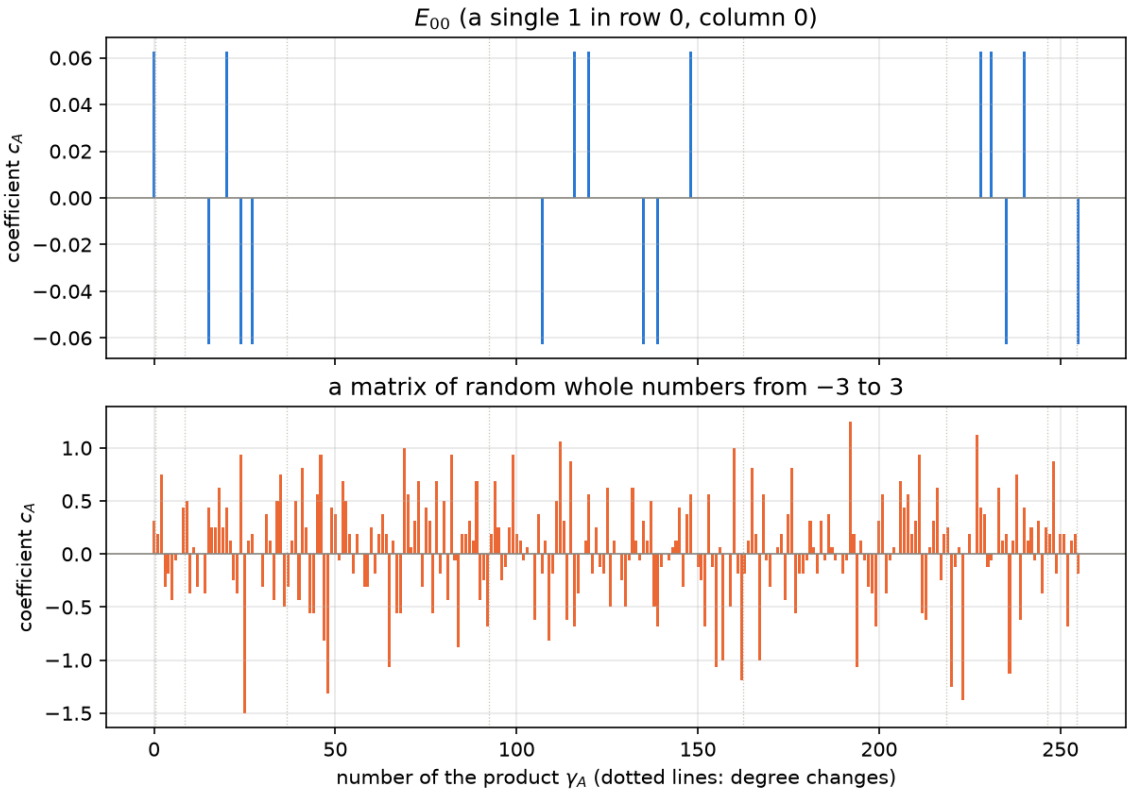

Figure 132: Every real 16×16 matrix is a combination $\sum_A c_A \gamma_A$ of the 256 products, with $c_A = \mathrm{tr}(\gamma_A^T M)/16$; horizontal axis the number of the product (ordered by degree, dotted lines where the degree changes), vertical axis the coefficient c_A (pure numbers). Top: the matrix E_{00} with a single entry 1 needs only the 16 products that are diagonal matrices, each with coefficient $\pm 1/16$. Bottom: a matrix of random whole numbers needs almost all 256 products. (Notebook 04c, figure 5.)

```
      and f"dimension {rank_even} = 2 x 8^2" in recorded(
          WOLFRAM, "even_subalgebra_dimension")
      and f"rank {rank_even} (Fraction), {rank_even} (sympy)" in recorded(
          PYTHON, "even_products_span_M8_plus_M8"),
      "even products block diagonal, odd block off-diagonal, ranks 128 and 128")
print(f"     reproduces {WOLFRAM}")
print("         check even_subalgebra_dimension")
print(f"     reproduces {PYTHON}")
print("         check even_products_span_M8_plus_M8")
support_even = np.sum(products[DEGREE % 2 == 0] != 0, axis=0)  # count per entry
support_odd = np.sum(products[DEGREE % 2 == 1] != 0, axis=0)
report("even products nonzero at each position (values found)",
       sorted(set(int(x) for x in support_even.flatten())))
upper = np.arange(16) < 8  # True for the rows (and columns) 0 to 7
# in_diagonal_blocks[i, j] is True when row i and column j lie in the same half;
# ~ turns True into False and back.
in_diagonal_blocks = np.equal.outer(upper, upper)
check((support_even[in_diagonal_blocks] == 16).all()
      and (support_even[~in_diagonal_blocks] == 0).all()
      and (support_odd[~in_diagonal_blocks] == 16).all()
      and (support_odd[in_diagonal_blocks] == 0).all(),
      "each position of its blocks is covered by exactly 16 even (or odd) products")
```

Out [13]:

```
RESULT exact ranks of the even and of the odd products = (128, 128)
PASS even products block diagonal, odd block off-diagonal, ranks 128 and 128
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check even_subalgebra_dimension
     reproduces Revision/algebra/reports/python-algebra.json
         check even_products_span_M8_plus_M8
RESULT even products nonzero at each position (values found) = [0, 16]
PASS each position of its blocks is covered by exactly 16 even (or odd) products
```

Text cell:

```
The next cell draws these two counts as heat maps.
```

In [14]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 4.2), layout="constrained")
cover = ListedColormap(["#f0efec", "#1c5cab"])  # 0 products, 16 products
for ax, values, title in ((left, support_even, "128 even products"),
                          (right, support_odd, "128 odd products")):
    ax.imshow(values, cmap=cover, norm=BoundaryNorm([-1, 8, 17], 2))
    ax.set_xticks(range(0, 16, 4))
    ax.set_yticks(range(0, 16, 4))
    ax.set_title(title)
    ax.set_xlabel("column")
    ax.set_ylabel("row")
    ax.axhline(7.5, color="#0b0b0b", linewidth=1)
```

```
    ax.axvline(7.5, color="#0b0b0b", linewidth=1)
    ax.grid(False)
save_figure(fig, "even_odd_blocks",
            "Where the products live: for every position (row and column 0 to 15) "
            "the number of products with a nonzero entry there (dark blue 16, light "
            "grey 0). Left: the 128 even products cover exactly the two diagonal "
            "$8 \\times 8$ blocks, each position 16 times; they span all block "
            "diagonal matrices, Mat(8) + Mat(8). Right: the 128 odd products cover "
            "exactly the two other blocks. Black lines separate the blocks.")
```

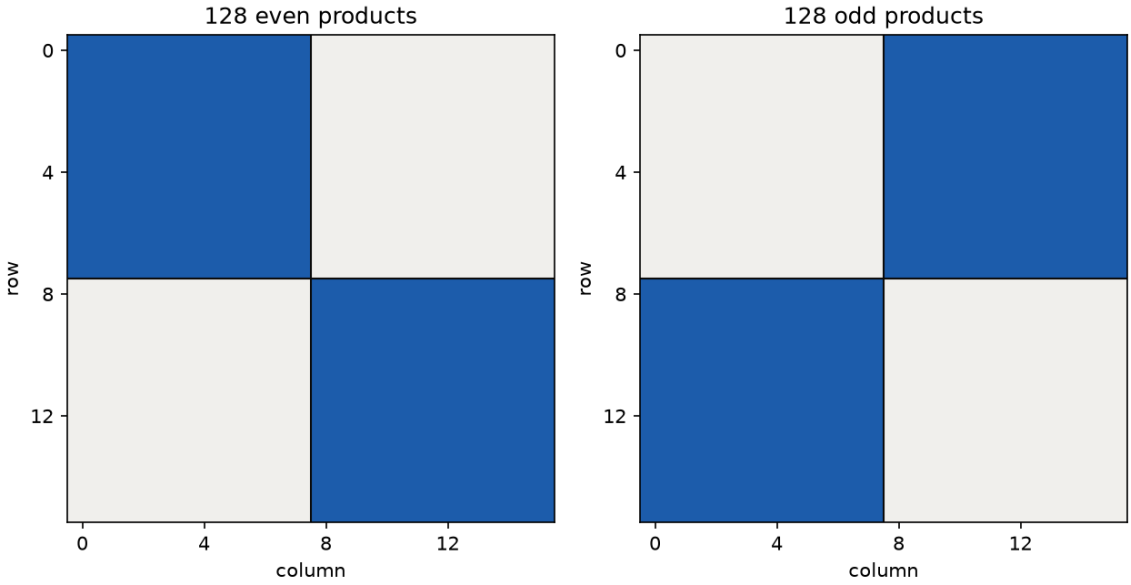

Figure 133: Where the products live: for every position (row and column 0 to 15) the number of products with a nonzero entry there (dark blue 16, light grey 0). Left: the 128 even products cover exactly the two diagonal 8×8 blocks, each position 16 times; they span all block diagonal matrices, Mat(8) + Mat(8). Right: the 128 odd products cover exactly the two other blocks. Black lines separate the blocks. (Notebook 04c, figure 6.)

Out [14]:

```
Figure 04c.6 saved as Revision/textbook/figures/04c_6_even_odd_blocks.png
```

Text cell:

```
## 12. How big must the matrices be?

The next cell compares four examples of matrices with a Clifford relation:

- $n = 2$: the real $2 \times 2$ matrices $\sigma_x$ and $N$ (signs $+1, -1$);
- $n = 4$: Dirac's complex $4 \times 4$ matrices (rebuilt here in three lines);
- $n = 7$: the author's seven $8 \times 8$ matrices tau[1], ..., tau[7], which are the
  lower-left $8 \times 8$ blocks of $\gamma^{(x_1)}, \dots, \gamma^{(x_7)}$ (his
  construction puts tau[k] there);
```

```
- $n = 8$: the author's gammas.

For each it computes the number $2^n$ of products, their exact rank and the
dimension $d^2$ of the space of all $d \times d$ matrices. For the even $n$ the rank
is $2^n = d^2$: the products fill everything, and $d$ is as small as possible. For
the odd $n = 7$ the trace lemma fails: the product of all seven taus is $I_8$, so the
products come in pairs that are equal up to sign, and only $64 = 8^2$ of the 128 are
independent. This is why seven anticommuting matrices fit into $8 \times 8$ matrices
but eight need $16 \times 16$.
```

In [15]:

```
def all_products(generators):
    """The 2^n ordered products of a list of square sympy matrices."""
    size = generators[0].shape[0]
    result = []
    for k in range(len(generators) + 1):
        for s in itertools.combinations(range(len(generators)), k):
            matrix = sp.eye(size)
            for a in s:
                matrix = matrix * generators[a]
            result.append(matrix)
    return result


sigma_x, sigma_y = sp.Matrix([[0, 1], [1, 0]]), sp.Matrix([[0, -sp.I], [sp.I, 0]])
sigma_z, N_matrix = sp.Matrix([[1, 0], [0, -1]]), sp.Matrix([[0, 1], [-1, 0]])
Z2, I2 = sp.zeros(2, 2), sp.eye(2)
dirac = [sp.Matrix(sp.BlockMatrix([[I2, Z2], [Z2, -I2]]))] + [
    sp.Matrix(sp.BlockMatrix([[Z2, s], [-s, Z2]])) for s in (sigma_x, sigma_y, sigma_z)]
taus = [sp.Matrix(gamma[a][8:, :8].tolist()) for a in range(1, 8)]  # tau[1..7]
examples = {"n = 2, real 2 x 2": [sigma_x, N_matrix], "n = 4, Dirac 4 x 4": dirac,
            "n = 7, tau 8 x 8": taus, "n = 8, gammas 16 x 16":
                [sp.Matrix(g.tolist()) for g in gamma]}
rows = []
for name, generators in examples.items():
    prods = all_products(generators)
    d = generators[0].shape[0]
    flat_rows = sp.Matrix([list(m) for m in prods])  # one row per product
    # exact rank; from_Matrix chooses exact numbers (Gaussian integers for Dirac)
    rank = DomainMatrix.from_Matrix(flat_rows).rank()
    rows.append((name, len(prods), rank, d * d))
    print(f"{name:24} products {len(prods):4d}   rank {rank:4d}   d^2 {d * d:4d}")
tau_product = sp.eye(8)
for t in taus:
    tau_product = tau_product * t
check([r[2] for r in rows] == [4, 16, 64, 256] and [r[1] for r in rows]
      == [4, 16, 128, 256], "ranks 4, 16, 64, 256: products fill d^2 for even n")
check(tau_product == sp.eye(8) and taus[6] == sp.diag(-1, -1, -1, -1, 1, 1, 1, 1)
      and "tau[7] = diag(-I4, I4) and tau[1]...tau[7] = ID8"
      in recorded(PYTHON, "tau7_and_product"),
      "tau[1] tau[2] ... tau[7] = I8 and tau[7] = diag(-I4, I4)")
print(f"     reproduces {PYTHON}")
```

```
print("         check tau7_and_product")
```

Out [15]:

```
n = 2, real 2 x 2        products    4   rank    4   d^2    4
n = 4, Dirac 4 x 4       products   16   rank   16   d^2   16
n = 7, tau 8 x 8         products  128   rank   64   d^2   64
n = 8, gammas 16 x 16    products  256   rank  256   d^2  256
PASS ranks 4, 16, 64, 256: products fill d^2 for even n
PASS tau[1] tau[2] ... tau[7] = I8 and tau[7] = diag(-I4, I4)
     reproduces Revision/algebra/reports/python-algebra.json
         check tau7_and_product
```

Text cell:

```
The next cell draws the three numbers of each example as grouped bars on a
logarithmic scale (base 2).
```

In [16]:

```
fig, ax = plt.subplots(figsize=(7.6, 4.0), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
positions = np.arange(len(rows))
for offset, column, colour, name in ((-0.25, 1, "#2a78d6", "number of products $2^n$"),
                                     (0.0, 2, "#eb6834", "exact rank"),
                                     (0.25, 3, "#1baf7a", "$d^2$ (all matrices)")):
    values = [r[column] for r in rows]
    bars = ax.bar(positions + offset, values, width=0.24, color=colour, label=name)
    ax.bar_label(bars, labels=[str(v) for v in values], fontsize=8, padding=1)
ax.set_yscale("log", base=2)
ax.set_ylim(1, 1024)
ax.set_xticks(positions, [r[0] for r in rows], fontsize=9)
ax.set_ylabel("count (logarithmic scale)")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "how_big",
            "Why eight gammas need $16 \\times 16$ matrices: for four examples of "
            "$n$ matrices of size $d \\times d$ with a Clifford relation, the number "
            "$2^n$ of their ordered products (blue), the exact rank of these "
            "products (orange) and the dimension $d^2$ of the space of all "
            "$d \\times d$ matrices (green), on a logarithmic scale. For even $n$ "
            "(2, 4, 8) all three are equal: the products are independent and fill "
            "all matrices, so $d$ cannot be smaller. For the odd $n = 7$ (the "
            "author's $8 \\times 8$ tau matrices) the 128 products have rank only "
            "64 = $8^2$, because the product of all seven is the identity.")
```

Out [16]:

```
Figure 04c.7 saved as Revision/textbook/figures/04c_7_how_big.png
```

Text cell:

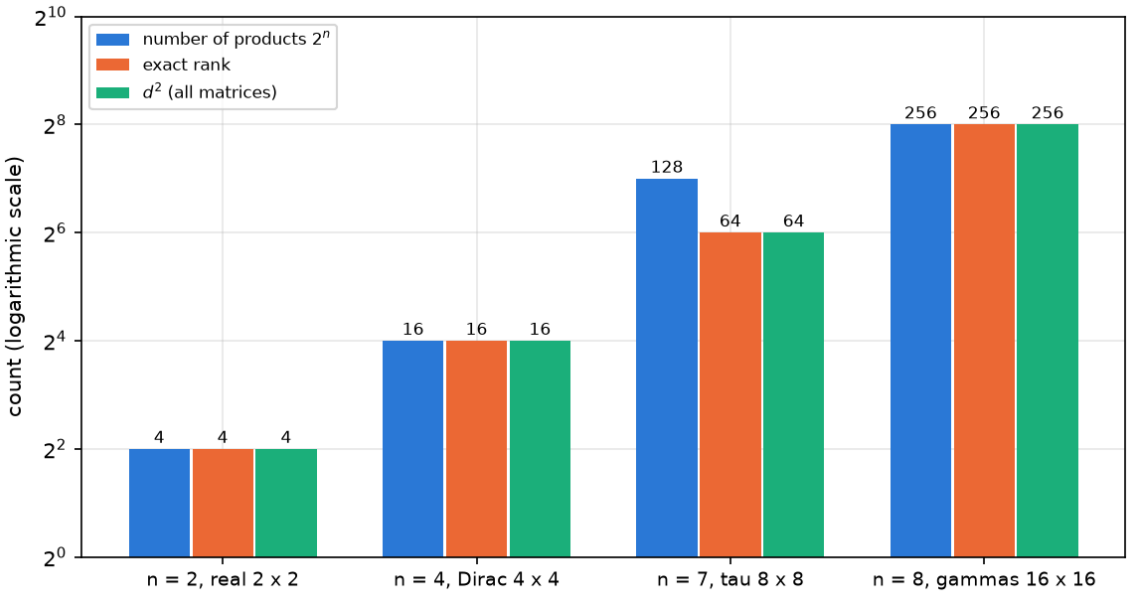

Figure 134: Why eight gammas need 16×16 matrices: for four examples of n matrices of size $d \times d$ with a Clifford relation, the number 2^n of their ordered products (blue), the exact rank of these products (orange) and the dimension d^2 of the space of all $d \times d$ matrices (green), on a logarithmic scale. For even n (2, 4, 8) all three are equal: the products are independent and fill all matrices, so d cannot be smaller. For the odd $n = 7$ (the author's 8×8 tau matrices) the 128 products have rank only $64 = 8^2$, because the product of all seven is the identity. (Notebook 04c, figure 7.)

```
## 13. The 28 products of two different gammas

The products of degree 2, $\gamma^a\gamma^b$ with $a$ before $b$ in the order
$x_0, \dots, x_7$, are $\binom{8}{2} = 28$ matrices. Half of each of them is a matrix
that the Revision record names $S^{ab}$:
$S^{ab} = \tfrac14(\gamma^a\gamma^b - \gamma^b\gamma^a)$. Line by line, for
$a \neq b$: $\gamma^b\gamma^a = -\gamma^a\gamma^b$ (two different gammas
anticommute), so $\gamma^a\gamma^b - \gamma^b\gamma^a = 2\gamma^a\gamma^b$, and
$S^{ab} = \tfrac14 \cdot 2\gamma^a\gamma^b = \tfrac12\gamma^a\gamma^b$. For $a = b$
the two terms cancel and $S^{aa} = 0$. The matrices $S^{ab}$ are the generators of
the rotations and boosts of the 16-component field (the record checks that they
obey the rules of the rotations and boosts of the 4+4 space-time).

By rule R2 with $k = 2$, $(\gamma^a\gamma^b)^2 = -\eta^{aa}\eta^{bb} I_{16}$. Two
directions of the same kind (both space-like or both time-like) have
$\eta^{aa}\eta^{bb} = +1$, so their product squares to $-I_{16}$, like the
imaginary unit $i$ of the complex numbers: there are
$\binom{4}{2} + \binom{4}{2} = 12$ such pairs, the planes of *rotations*. A
space-like and a time-like direction have $\eta^{aa}\eta^{bb} = -1$, so their
product squares to $+I_{16}$: there are $4 \cdot 4 = 16$ such pairs, the planes of
*boosts*.

The next cell checks, exactly: $\tfrac14(\gamma^a\gamma^b - \gamma^b\gamma^a)$ is
$\tfrac12\gamma^a\gamma^b$ for $a \neq b$ and 0 for $a = b$; it compares all
$8 \times 8 = 64$ matrices $S^{ab}$ (16384 entries) with the matrices that the
```

Revision record file ‘Revision/algebra/gammas.json‘ stores under the key S (their
entries are 0 and the fractions $\pm 1/2$, written as texts "1/2" and "-1/2");
and it checks the squares and the 12 + 16 count. All arithmetic is exact: the
fractions are Python ‘Fraction‘ numbers.

In [17]:

```
from fractions import Fraction  # exact fractions such as 1/2


def exact(entry):
    """A number of gammas.json: a JSON integer or a text "p/q", as a Fraction."""
    if isinstance(entry, int):
        return Fraction(entry)
    numerator, denominator = entry.split("/")
    return Fraction(int(numerator), int(denominator))


PAIRS_AB = [(a, b) for a in range(8) for b in range(a + 1, 8)]  # the 28 pairs
two = {(a, b): products[INDEX[(a, b)]] for a, b in PAIRS_AB}  # gamma^a gamma^b
half_ok, record_ok, values = True, True, set()
for a, b in itertools.product(range(8), repeat=2):  # all 64 ordered pairs
    ab, ba = gamma[a] @ gamma[b], gamma[b] @ gamma[a]
    commutator = ab - ba  # 4 S^ab, a whole-number matrix
    if a != b:
        half_ok &= bool((commutator == 2 * ab).all())  # 4 S^ab = 2 gamma^a gamma^b
    else:
        half_ok &= not commutator.any()  # S^aa = 0
    stored = record["S"][a][b]  # the record's S^ab: 16 rows of exact numbers
    for i, j in itertools.product(range(16), repeat=2):
        ours = Fraction(int(commutator[i, j]), 4)  # S^ab entry, exact
        values.add(ours)
        record_ok &= exact(stored[i][j]) == ours
report("the different entries of the 64 matrices S^ab",
       [str(v) for v in sorted(values)])
check(half_ok and record_ok and sorted(values) == [Fraction(-1, 2), 0,
                                                   Fraction(1, 2)]
      and "(1/2) gamma^a gamma^b for a != b" in recorded(PYTHON, "S_definition")
      and "S^ab = (1/2) gamma^a gamma^b" in recorded(WOLFRAM, "S_half_product")
      and "{-1/2, 0, 1/2}" in recorded(WOLFRAM, "S_real_entries_in_half_integers"),
      "S^ab = (1/4)(g^a g^b - g^b g^a) = (1/2) g^a g^b = the record's S, entry by entry")
print(f"     reproduces {PYTHON}")
print("         check S_definition")
print(f"     reproduces {WOLFRAM}")
print("         checks S_half_product, S_real_entries_in_half_integers")
square_of = {pair: int((m @ m)[0, 0]) for pair, m in two.items()}  # +1 or -1
rotations = [pair for pair in PAIRS_AB if square_of[pair] == -1]
boosts = [pair for pair in PAIRS_AB if square_of[pair] == 1]
report("pairs whose product squares to -I16 (rotations), to +I16 (boosts)",
       (len(rotations), len(boosts)))
check(all((m @ m == square_of[(a, b)] * I16).all()
          and square_of[(a, b)] == -eta[a] * eta[b]
          and not m[:8, 8:].any() and not m[8:, :8].any()
```

```
          for (a, b), m in two.items())
      and len(rotations) == 12 and len(boosts) == 16,
      "(g^a g^b)^2 = -eta^aa eta^bb I16; 12 rotation, 16 boost planes; block diagonal")
```

Out [17]:

```
RESULT the different entries of the 64 matrices S^ab = ['-1/2', '0', '1/2']
PASS S^ab = (1/4)(g^a g^b - g^b g^a) = (1/2) g^a g^b = the record's S, entry by entry
     reproduces Revision/algebra/reports/python-algebra.json
         check S_definition
     reproduces Revision/algebra/reports/wolfram-algebra.json
         checks S_half_product, S_real_entries_in_half_integers
RESULT pairs whose product squares to -I16 (rotations), to +I16 (boosts) = (12, 16)
PASS (g^a g^b)^2 = -eta^aa eta^bb I16; 12 rotation, 16 boost planes; block diagonal
```

Text cell:

```
The next cell draws the 28 products as a triangular table of heat maps: the panel
in row $x_a$ and column $x_b$ shows $\gamma^a\gamma^b$ (rows $x_0$ to $x_6$, columns
$x_1$ to $x_7$; the panels on the diagonal of the grid show $x_0x_1, x_1x_2, \dots,
x_6x_7$, and the panels below it stay empty). A black frame
marks a rotation plane (square $-I_{16}$), a green frame a boost plane (square
$+I_{16}$).
```

In [18]:

```
from matplotlib.patches import Patch  # coloured squares for the key

FRAME = {-1: "#0b0b0b", 1: "#1baf7a"}  # square -I16: black, square +I16: green
fig, axes = plt.subplots(7, 7, figsize=(8.6, 8.9), layout="constrained")
for r, c_ in itertools.product(range(7), repeat=2):
    ax = axes[r, c_]
    a, b = r, c_ + 1  # row: first factor x_a; column: second factor x_b
    if b <= a:
        ax.axis("off")  # no panel below the diagonal of the grid (c_ < r)
        continue
    ax.imshow(two[(a, b)], cmap=three, norm=three_norm)
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():  # the frame of the panel
        spine.set_edgecolor(FRAME[square_of[(a, b)]])
        spine.set_linewidth(2.0)
    ax.set_title(f"$x_{a}\\,x_{b}$", fontsize=8)
key = [Patch(facecolor="white", edgecolor=FRAME[-1], linewidth=2,
             label="square $-I_{16}$: rotation plane (12)"),
       Patch(facecolor="white", edgecolor=FRAME[1], linewidth=2,
             label="square $+I_{16}$: boost plane (16)"),
       Patch(facecolor="#2a78d6", label="entry $-1$"),
       Patch(facecolor="#e34948", label="entry $+1$")]
fig.legend(handles=key, loc="outside lower center", ncol=2, fontsize=9,
           frameon=False)
```

```
save_figure(fig, "two_gamma_products",
            "The 28 products $\\gamma^a\\gamma^b$ of two different gammas of the "
            "author, each drawn as a $16 \\times 16$ heat map (blue $-1$, grey $0$, "
            "red $+1$); the panel titled $x_a x_b$ shows $\\gamma^a\\gamma^b$, with "
            "$x_a$ before $x_b$ in the order $x_0$ to $x_7$. Every product is block "
            "diagonal, with its 16 nonzero entries in the two diagonal "
            "$8 \\times 8$ blocks. Black frames mark the 12 products that square to "
            "$-I_{16}$ (two directions of the same kind: rotation planes), green "
            "frames the 16 that square to $+I_{16}$ (one space-like and one "
            "time-like direction: boost planes). Half of each product is the "
            "matrix $S^{ab}$ of the Revision record.")
```

Out [18]:

```
Figure 04c.8 saved as Revision/textbook/figures/04c_8_two_gamma_products.png
```

Text cell:

```
## 14. The last check

The last cell checks that the eight figure files exist in the folder
‘Revision/textbook/figures‘ and prints the number of checks that passed.
```

In [19]:

```
names = ["04c_1_products_by_degree.png", "04c_2_multiplication_table.png",
         "04c_3_squares_by_degree.png", "04c_4_trace_products.png",
         "04c_5_coefficients.png", "04c_6_even_odd_blocks.png", "04c_7_how_big.png",
         "04c_8_two_gamma_products.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all eight figure files exist")
all_checks_passed()
```

Out [19]:

```
PASS all eight figure files exist
ALL 19 CHECKS PASSED (notebook 04c)
```

Text cell:

```
## 15. What this notebook showed

- PROVED (exact integer arithmetic, all pairs checked): the 256 ordered products of
  the author’s gammas multiply by rule R1,
  $\gamma_A\gamma_B = s(A,B)\gamma_{A\triangle B}$; they square to $\pm I_{16}$ by
  rule R2; 136 are symmetric (square $+I_{16}$) and 120 antisymmetric (square
  $-I_{16}$); every product other than $I_{16}$ has trace 0, and
  $\mathrm{tr}(\gamma_A^T\gamma_B) = 16\delta_{AB}$.
```

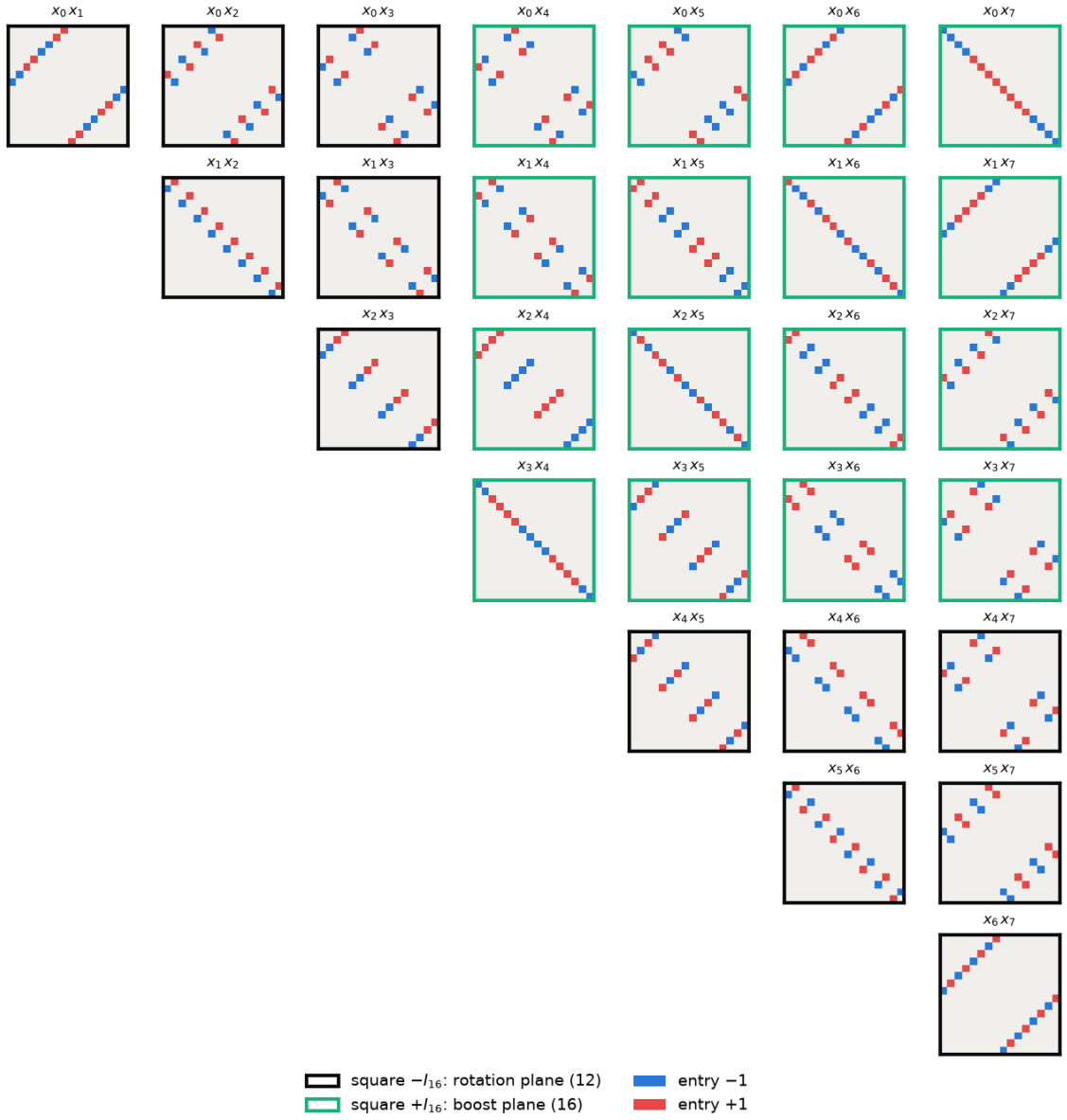

Figure 135: The 28 products $\gamma^a \gamma^b$ of two different gammas of the author, each drawn as a 16×16 heat map (blue -1, grey 0, red $+1$); the panel titled $x_a x_b$ shows $\gamma^a \gamma^b$, with x_a before x_b in the order x_0 to x_7. Every product is block diagonal, with its 16 nonzero entries in the two diagonal 8×8 blocks. Black frames mark the 12 products that square to $-I_{16}$ (two directions of the same kind: rotation planes), green frames the 16 that square to $+I_{16}$ (one space-like and one time-like direction: boost planes). Half of each product is the matrix S^{ab} of the Revision record. (Notebook 04c, figure 8.)

```
- PROVED (exact rank over the rational numbers; reproduces
  ‘Revision/algebra/reports/wolfram-algebra.json‘, check
  ‘Clifford_basis_spans_full_matrix_algebra‘, and ‘python-algebra.json‘, check
  ‘clifford_products_span_M16‘): the 256 products are independent, so the Clifford
  algebra Cl(4,4) of the author’s gammas is the set of all real $16 \times 16$
  matrices, and every matrix is $\sum_A c_A\gamma_A$ with
  $c_A = \mathrm{tr}(\gamma_A^T M)/16$.
- PROVED (reproduces the checks ‘even_subalgebra_dimension‘ and
  ‘even_products_span_M8_plus_M8‘): the 128 even products are block diagonal and
  span all block diagonal matrices (dimension 128); the 128 odd products span the
  block off-diagonal ones.
- PROVED: eight $d \times d$ matrices with a Clifford relation have $d \ge 16$
  ($d^2 \ge 2^8$); the author’s field has exactly 16 components. Seven such
  matrices fit into $8 \times 8$ (the author’s tau matrices, whose product is
  $I_8$; reproduces check ‘tau7_and_product‘).
- PROVED (exact fractions; reproduces ‘python-algebra.json‘, check ‘S_definition‘,
  and ‘wolfram-algebra.json‘, checks ‘S_half_product‘ and
  ‘S_real_entries_in_half_integers‘, and agrees entry by entry with the matrices S
  of ‘Revision/algebra/gammas.json‘): $S^{ab} = \tfrac14(\gamma^a\gamma^b -
  \gamma^b\gamma^a) = \tfrac12\gamma^a\gamma^b$ for $a \neq b$, with entries
  $0, \pm\tfrac12$. The 28 products of two gammas are block diagonal; 12 square to
  $-I_{16}$ (rotation planes) and 16 to $+I_{16}$ (boost planes).
- ASSUMED: nothing beyond the Clifford relation of the author’s gammas, which the
  Revision record verified exactly. These are statements of algebra; they make no
  physical claim.
```

4.16 Line-by-line walk-through of Notebook 04c

The notebook has 19 code cells, In [1] to In [19]. As in Section 4.8, every line is quoted except two things: the set-up cell In [1], which is the set-up cell of Notebook 04a explained line by line in Section 4.8 and differs from it only as said below, and the long caption texts handed to `save_figure`, which are shortened to a line `...)` (the dots stand for the caption, the bracket closes the call); each caption is printed in full under its figure in Section 4.15.

In [1], the set-up cell. It is the set-up cell of Notebook 04a, explained line by line in Section 4.8, except for the line `NOTEBOOK_ID = "04c"` and the run instructions of Section 4.14 in its comment lines. It prints one line.

In [2], the 256 products.

```
import itertools  # all subsets of a given size
from math import comb  # comb(n, k) is the binomial coefficient n choose k

import numpy as np
import sympy as sp
from matplotlib.colors import BoundaryNorm, ListedColormap
from sympy.polys.matrices import DomainMatrix  # exact matrices for the rank
```

`comb(n, k)` computes $\binom{n}{k}$. `DomainMatrix` is sympy’s fast matrix type for exact linear algebra with whole numbers and fractions, used for the ranks in In [10], In [13] and In [15]. The other imports are as in Notebook 04a.

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
eta = record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1] in the order x0..x7
gamma = [np.array(matrix, dtype=np.int64) for matrix in record["gamma"]]
I16 = np.eye(16, dtype=np.int64)
```

The author's eight gammas and their signs are read from the Revision record file `gammas.json` (Notebook 04a showed that they are exactly the matrices built from his formulas), as whole-number arrays.

```
SETS = [s for k in range(9) for s in itertools.combinations(range(8), k)]
INDEX = {s: i for i, s in enumerate(SETS)}  # set -> its number in the list
```

`itertools.combinations(range(8), k)` lists the sets of k directions as tuples of increasing numbers (0 stands for x_0, ..., 7 for x_7); the list comprehension collects them for $k = 0, 1, \ldots, 8$, so `SETS` holds the 256 sets ordered by degree, starting with the empty tuple `()`. `INDEX` gives the number of each set in this list.

```
products = np.zeros((256, 16, 16), dtype=np.int64)
for i, s in enumerate(SETS):
    matrix = I16
    for a in s:  # multiply the gammas of the set in increasing order
        matrix = matrix @ gamma[a]
    products[i] = matrix
DEGREE = np.array([len(s) for s in SETS])  # the number of factors of each product
```

`products` is one array of shape (256, 16, 16): product number i is `products[i]`, a 16×16 matrix. For each set the gammas are multiplied in increasing order, starting from I_{16} (so the empty set gives I_{16}). `DEGREE` holds the degree of every product.

```
def label(s):
    """A short name of a set: its coordinate numbers, e.g. (1, 4) -> "14"."""
    return "".join(str(a) for a in s) if s else "I"
```

`label` writes a short name of a set: the coordinate numbers glued together, for example the set $\{x_1, x_4\}$, stored as `(1, 4)`, becomes `"14"`; the empty set (which Python reads as false) becomes `"I"`.

```
counts = [int(np.sum(DEGREE == k)) for k in range(9)]
report("number of products of degree 0, 1, ..., 8", counts)
report("even and odd products", (sum(counts[0::2]), sum(counts[1::2])))
check(counts == [comb(8, k) for k in range(9)] and sum(counts) == 256
      and sum(counts[0::2]) == 128,
      "the products by degree are 1, 8, 28, 56, 70, 56, 28, 8, 1 (128 even)")
```

`np.sum(DEGREE == k)` counts the products of degree k (True counts as 1). `counts[0::2]` takes every second entry starting with the first (the even degrees), `counts[1::2]` the others. The RESULT

lines show $1, 8, 28, 56, 70, 56, 28, 8, 1$ and $(128, 128)$, and the check compares them with the binomial coefficients.

In [3], figure 1.

```
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
colours = ["#2a78d6" if k % 2 == 0 else "#eb6834" for k in range(9)]
bars = ax.bar(range(9), counts, color=colours, width=0.7)
ax.bar_label(bars, labels=[str(n) for n in counts], padding=2, fontsize=9)
ax.set_xticks(range(9))
ax.set_xlabel("degree $k$ (number of gamma factors)")
ax.set_ylabel("number of products")
ax.set_ylim(0, 80)
ax.legend(handles=[bars[0], bars[1]], labels=["even degree (128 in all)",
                                              "odd degree (128 in all)"], fontsize=9)
save_figure(fig, "products_by_degree",
            ...)
```

A bar chart: `ax.bar` draws one bar per degree, blue for even and orange for odd degrees; `ax.bar_label` writes each count above its bar; the vertical axis runs from 0 to 80; the legend uses the first two bars (degree 0, blue, and degree 1, orange) as samples of the two colours. **What figure 1 shows and why:** the counts rise from 1 to 70 at degree 4 and fall back to 1, symmetric about the middle because choosing k directions is the same as leaving out $8 - k$; the blue bars add up to 128, the orange ones to 128.

In [4], rule R1 for all pairs.

```
def sign_formula(A, B):
    """s(A, B) of rule R1: (-1)^N(A, B) times the signs eta of the common directions."""
    later_pairs = sum(1 for a in A for b in B if a > b)  # N(A, B)
    sign = (-1) ** later_pairs
    for c in set(A) & set(B):  # & : the directions in both sets
        sign *= eta[c]
    return sign
```

`sign_formula` computes the sign $s(A, B)$ of rule R1: `later_pairs` counts the pairs with a in A, b in B and a later than b; the sign starts as (-1) to that power and is multiplied (`*=`) by η^{cc} for every common direction (`set(A) & set(B)` is the set of the directions in both).

```
rule_r1 = True
SIGN = np.zeros((256, 256), dtype=np.int64)  # SIGN[i, j] = s(A_i, A_j)
for i, A in enumerate(SETS):
    row = products[i] @ products  # gamma_A times each of the 256 products
    for j, B in enumerate(SETS):
        target = products[INDEX[tuple(sorted(set(A) ^ set(B)))]]  # ^ : symmetric diff.
        SIGN[i, j] = sign_formula(A, B)
        rule_r1 &= bool((row[j] == SIGN[i, j] * target).all())
```

For every product γ_A, `products[i] @ products` multiplies it with all 256 products at once (numpy applies `@` to each of the 256 matrices in turn). For every B, `set(A) ^ set(B)` is the symmetric

difference; `sorted` puts it in increasing order and `tuple` makes it a key of `INDEX`, which finds the product $\gamma_{A \triangle B}$. `rule_r1` stays true only if every one of the 65536 products equals $s(A,B)\,\gamma_{A \triangle B}$.

```
report("pairs with sign +1 and with sign -1",
       (int(np.sum(SIGN == 1)), int(np.sum(SIGN == -1))))
check(rule_r1, "gamma_A gamma_B = s(A,B) gamma_(A sym. diff. B) for all 65536 pairs")
```

The RESULT line counts the pairs with sign $+1$ and -1: $(33024, 32512)$. These two numbers depend on the order in which the directions are numbered, because γ_A is the product in increasing order and $N(A,B)$ counts pairs in that order: with the hidden direction numbered last instead of first, the same rule gives $(33280, 32256)$. The rule itself holds in every numbering, and the check confirms it for all pairs.

```
signed_permutations = all(
    (np.count_nonzero(m, axis=0) == 1).all() and (np.count_nonzero(m, axis=1) == 1).all()
    and set(np.unique(m)) <= {-1, 0, 1} for m in products[1:])  # <= : subset
check(signed_permutations and all((m.T @ m == I16).all() for m in products),
      "every product is a signed permutation matrix, hence orthogonal")
```

For the products after I_{16} (`products[1:]` skips the first; I_{16} obviously qualifies), every column and every row must have exactly one nonzero entry, and the different values (`np.unique`) must form a subset of $\{-1, 0, 1\}$ (`<=` tests "is a subset of" for sets). The check also confirms $\gamma_A^T \gamma_A = I_{16}$ for all 256.

In [5], figure 2.

```
four = [s for s in SETS if set(s) <= {1, 2, 3, 4}]  # the 16 sets inside x1..x4
table = np.array([[SIGN[INDEX[A], INDEX[B]] for B in four] for A in four])
fig, ax = plt.subplots(figsize=(7.4, 7.0), layout="constrained")
ax.imshow(table, cmap=ListedColormap(["#2a78d6", "#e34948"]),
          norm=BoundaryNorm([-2, 0, 2], 2))
```

`four` keeps the 16 sets made only of x_1, x_2, x_3, x_4 (direction numbers 1 to 4; `set(s) <= {1, 2, 3, 4}` tests "is a subset of"). `table` is the 16×16 table of their signs $s(A,B)$, painted with two colours: blue for -1, red for $+1$.

```
for r, A in enumerate(four):
    for c_, B in enumerate(four):
        name = label(tuple(sorted(set(A) ^ set(B))))
        ax.text(c_, r, ("+" if table[r, c_] > 0 else "-") + name, ha="center",
                va="center", color="white", fontsize=7)
names = [label(s) for s in four]
ax.set_xticks(range(16), names, fontsize=8)
ax.set_yticks(range(16), names, fontsize=8)
ax.set_xlabel("second factor $\\gamma_B$ (coordinate numbers of $B$)")
ax.set_ylabel("first factor $\\gamma_A$")
ax.grid(False)
save_figure(fig, "multiplication_table",
            ...)
```

Every square gets the sign and the name of $A \triangle B$, so that it reads, for example, -14 for $\gamma_A \gamma_B = -\gamma^{(x_1)}\gamma^{(x_4)}$; rows and columns are labelled with the names of the 16 sets. **What figure 2 shows and why:** every product of two of these 16 matrices is again one of them, up to sign (rule R1): they form the Clifford algebra of four directions, the algebra of Dirac's matrices. The diagonal holds the squares, +I or −I as rule R2 predicts (for example the square of $\gamma^{(x_4)}$ is −I, because x_4 is time-like).

In [6], rule R2, and the symmetric products.

```
rule_r2 = True
square_sign = np.zeros(256, dtype=np.int64)
for i, s in enumerate(SETS):
    k = len(s)
    predicted = (-1) ** (k * (k - 1) // 2)  # // : division without remainder
    for a in s:
        predicted *= eta[a]
    square_sign[i] = predicted
    rule_r2 &= bool((products[i] @ products[i] == predicted * I16).all())
```

For every product the sign of rule R2, $(-1)^{k(k-1)/2} \prod_{a \in A} \eta^{aa}$, is computed and stored in `square_sign`, and `rule_r2` stays true only if every square equals that sign times I_{16}.

```
symmetric = np.array([(m.T == m).all() for m in products])
antisymmetric = np.array([(m.T == -m).all() for m in products])
plus, minus = int(np.sum(square_sign == 1)), int(np.sum(square_sign == -1))
report("products with square +I16 and with square -I16", (plus, minus))
check(rule_r2, "gamma_A^2 = (-1)^(k(k-1)/2) prod(eta) I16 for all 256 products")
check(plus == 136 == 16 * 17 // 2 and minus == 120 == 16 * 15 // 2
      and (symmetric == (square_sign == 1)).all()
      and (antisymmetric == (square_sign == -1)).all(),
      "136 products are symmetric (square +I16), 120 antisymmetric (square -I16)")
```

Two True/False arrays record which products are symmetric and which antisymmetric; `plus` and `minus` count the squares $+I_{16}$ and $-I_{16}$: $(136, 120)$. The first check confirms R2; the second confirms the counts (Python allows the chain `plus == 136 == 16 * 17 // 2`, meaning both equalities) and that a product is symmetric exactly when its square is $+I_{16}$.

In [7], figure 3.

```
plus_by_degree = [int(np.sum((DEGREE == k) & (square_sign == 1))) for k in range(9)]
minus_by_degree = [int(np.sum((DEGREE == k) & (square_sign == -1))) for k in range(9)]
fig, ax = plt.subplots(figsize=(7.0, 3.8), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
low = ax.bar(range(9), plus_by_degree, color="#e34948", width=0.7,
             label="square $+I_{16}$ (symmetric)")
high = ax.bar(range(9), minus_by_degree, bottom=plus_by_degree, color="#2a78d6",
              width=0.7, label="square $-I_{16}$ (antisymmetric)")
```

For each degree the products with square $+I_{16}$ and with square $-I_{16}$ are counted (`&` combines two True/False arrays entry by entry). The red bars show the first counts; the blue bars are stacked on top of them (`bottom=`).

```
# numbers inside the bars that are tall enough, above the two bars of height 1
ax.bar_label(low, labels=[str(n) if n > 2 else "" for n in plus_by_degree],
             label_type="center", color="white", fontsize=8)
ax.bar_label(high, labels=[str(n) if n else "" for n in minus_by_degree],
             label_type="center", color="white", fontsize=8)
for k in (0, 8):
    ax.text(k, plus_by_degree[k] + 1.5, str(plus_by_degree[k]), ha="center",
            fontsize=8, color="#0b0b0b")
ax.set_xticks(range(9))
ax.set_xlabel("degree $k$")
ax.set_ylabel("number of products")
ax.legend(fontsize=9)
save_figure(fig, "squares_by_degree",
            ...)
```

The counts are written into the middle of the bars that are tall enough; for degrees 0 and 8, whose bars have height 1, the number is written above the bar instead. **What figure 3 shows and why:** the red counts per degree are $1, 4, 16, 28, 38, 28, 16, 4, 1$ and the blue ones $0, 4, 12, 28, 32, 28, 12, 4, 0$; for example among the 28 products of degree 2, the 16 boost planes square to $+I_{16}$ and the 12 rotation planes to $-I_{16}$ (Section 4.13). The totals are 136 and 120, the dimensions of the symmetric and of the antisymmetric matrices.

In [8], the trace lemma and the Gram matrix.

```
traces = np.array([np.trace(m) for m in products])
check(traces[0] == 16 and (traces[1:] == 0).all(),
      "trace lemma: tr gamma_A = 0 for all 255 products other than I16")
```

The trace of every product: 16 for I_{16} and 0 for the 255 others, as the trace lemma says.

```
flat = products.reshape(256, 256)  # each product as one row of 256 entries
gram = flat @ flat.T  # gram[A, B] = tr(gamma_A^T gamma_B), exact whole numbers
# tr(gamma_A gamma_B) = sum_ij (gamma_A)_ij (gamma_B^T)_ij: the same with the
# transposed products in the second factor.
flat_transposed = products.transpose(0, 2, 1).reshape(256, 256)
without_transpose = flat @ flat_transposed.T
```

`reshape(256, 256)` writes each 16×16 product as one row of 256 numbers (row by row). Because $\mathrm{tr}(X^T Y) = \sum_{i,j} X_{ij} Y_{ij}$ (the definitions of trace, product and transpose), the trace product of two products is the product of their two rows, and `flat @ flat.T` computes all 256×256 of them at once: the Gram matrix. `products.transpose(0, 2, 1)` transposes every product (it exchanges the last two indices), and the same trick gives the table of $\mathrm{tr}(\gamma_A \gamma_B)$.

```
check((gram == 16 * np.eye(256, dtype=np.int64)).all(),
      "tr(gamma_A^T gamma_B) = 16 for A = B and 0 otherwise (all 65536 pairs)")
check((without_transpose == 16 * np.diag(square_sign)).all(),
      "tr(gamma_A gamma_B) = 16 times the sign of the square for A = B, else 0")
```

The Gram matrix is $16 I_{256}$, as Section 4.13 proved; and $\mathrm{tr}(\gamma_A \gamma_B)$ is 16 times the sign of the square of γ_A for $A = B$ and 0 otherwise.

In [9], figure 4.

```
starts = [int(np.argmax(DEGREE == k)) for k in range(9)]  # first product of degree k
three = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -1, 0, +1
three_norm = BoundaryNorm([-1.5, -0.5, 0.5, 1.5], 3)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 4.8), layout="constrained")
```

`np.argmax` of a True/False array gives the position of the first True: `starts[k]` is the number of the first product of degree k (0, 1, 9, 37, ...). The three colours and boundaries of the sign heat maps, and a figure with two panels.

```
left.imshow(gram // 16, cmap=three, norm=three_norm, interpolation="nearest")
left.set_xticks(range(0, 256, 64))
left.set_yticks(range(0, 256, 64))
left.set_xlabel("number of the product $\\gamma_B$")
left.set_ylabel("number of the product $\\gamma_A$")
left.set_title("$\\mathrm{tr}(\\gamma_A^T\\gamma_B)/16$, all 256 products")
left.grid(False)
```

The left panel paints the Gram matrix divided by 16 (`interpolation="nearest"` draws every entry as a sharp square).

```
corner = starts[3]  # 37: the products of degree 0, 1 and 2
right.imshow(without_transpose[:corner, :corner] // 16, cmap=three, norm=three_norm)
names = [label(s) for s in SETS[:corner]]
right.set_xticks(range(corner), names, fontsize=5, rotation=90)
right.set_yticks(range(corner), names, fontsize=5)
right.set_title("$\\mathrm{tr}(\\gamma_A\\gamma_B)/16$, degrees 0, 1, 2")
right.grid(False)
save_figure(fig, "trace_products",
            ...)
```

The right panel magnifies the corner of the table $\mathrm{tr}(\gamma_A\gamma_B)/16$ for the first 37 products ($1+8+28$, degrees 0, 1 and 2), labelled with their names (the column labels turned by 90 degrees). **What figure 4 shows and why:** on the left a single red diagonal line on grey: every product is perpendicular to every other, which is why they are independent. On the right the diagonal is red for I and for $\gamma^{(x_0)}, \gamma^{(x_1)}, \gamma^{(x_2)}, \gamma^{(x_3)}$ (squares $+I_{16}$), blue for $\gamma^{(x_4)}, \ldots, \gamma^{(x_7)}$ (squares $-I_{16}$), and for the products of two gammas red for the 16 boost planes and blue for the 12 rotation planes, $-\eta^{aa}\eta^{bb}$ by rule R2.

In [10], the exact rank.

```
WOLFRAM = "Revision/algebra/reports/wolfram-algebra.json"
PYTHON = "Revision/algebra/reports/python-algebra.json"
RECORDED = {}
for report_file in (WOLFRAM, PYTHON):
    text = repository_file(report_file).read_text(encoding="utf-8")
    RECORDED[report_file] = {e["name"]: e for e in json.loads(text)["checks"]}
```

The two algebra reports of the Revision record are read, as in Notebook 04a.

```
def recorded(report_file, name):
    """The detail text of a Revision check; it must be recorded as passed."""
    entry = RECORDED[report_file][name]
    if entry["verdict"].upper() != "PASS":
        raise AssertionError(f"{report_file}: {name} is not recorded as passed")
    return entry["detail"]
```

`recorded` returns the detail text of a Revision check after making sure that it is recorded as passed.

```
def exact_rank(rows):
    """The exact rank of a list of rows of whole numbers (fractions, over QQ)."""
    matrix = DomainMatrix([[sp.ZZ(int(x)) for x in row] for row in rows],
                          (len(rows), len(rows[0])), sp.ZZ)
    return matrix.convert_to(sp.QQ).rank()
```

The **rank** of a list of rows is the largest number of independent rows among them. `exact_rank` builds a `DomainMatrix` of whole numbers (`sp.ZZ`, with its numbers of rows and columns), converts it to the rational numbers `sp.QQ` (exact fractions) and computes its rank by exact elimination, without any rounding.

```
rank_all = exact_rank(flat.tolist())
report("exact rank of the 256 products", rank_all)
check(rank_all == 256
      and f"span dimension {rank_all} = 16^2" in recorded(
          WOLFRAM, "Clifford_basis_spans_full_matrix_algebra")
      and f"{rank_all} (sympy DomainMatrix over QQ)" in recorded(
          PYTHON, "clifford_products_span_M16"),
      "the 256 products are independent: Cl(4,4) is all real 16 x 16 matrices")
print(f"     reproduces {WOLFRAM}")
print("         check Clifford_basis_spans_full_matrix_algebra")
print(f"     reproduces {PYTHON}")
print("         check clifford_products_span_M16")
```

The 256 products, as 256 rows of 256 numbers, have the exact rank 256. The check also requires that both Revision checks state this rank in their own words (the texts are built with the computed rank, so they must contain 256). The printed lines name the two records.

In [11], every matrix is a combination of the products.

```
e00 = np.zeros((16, 16), dtype=np.int64)
e00[0, 0] = 1
generator = np.random.default_rng(12345)
random_matrix = generator.integers(-3, 4, size=(16, 16))  # whole numbers -3..3
coefficients = {}
```

Two example matrices: E_{00}, with a single 1 in row 0 and column 0, and a 16×16 matrix of random whole numbers from -3 to 3 (the upper limit 4 of `integers` is not included), from a generator with the fixed seed 12345.

```
for name, matrix in (("E00", e00), ("random", random_matrix)):
    sixteen_c = flat @ matrix.reshape(256)  # 16 c_A = tr(gamma_A^T M) for all A
    rebuilt = np.einsum("a,aij->ij", sixteen_c, products)  # sum_A (16 c_A) gamma_A
    coefficients[name] = sixteen_c / 16.0
    check((rebuilt == 16 * matrix).all(),
          f"the matrix {name} equals sum_A c_A gamma_A with c_A = tr(gamma_A^T M)/16")
```

For each example, `flat @ matrix.reshape(256)` computes the 256 numbers $\mathrm{tr}(\gamma_A^T M) = 16c_A$ (whole numbers, so the arithmetic stays exact). `np.einsum("a,aij->ij", ...)` forms $\sum_A (16c_A)\gamma_A$ (the letter `a`, missing after the arrow, is summed). The check requires that this is exactly $16M$; the coefficients c_A are stored for the figure. Two PASS lines.

```
nonzero = [label(SETS[i]) for i in np.flatnonzero(coefficients["E00"])]
diagonal = [label(s) for i, s in enumerate(SETS)
            if (products[i] == np.diag(np.diag(products[i]))).all()]
say("the nonzero coefficients of E00 belong to: " + ", ".join(nonzero))
check(nonzero == diagonal and len(nonzero) == 16
      and set(np.abs(coefficients["E00"][np.flatnonzero(coefficients["E00"])]))
      == {1 / 16},
      "E00 has 16 coefficients +-1/16, on the 16 diagonal products")
```

`nonzero` names the products with a nonzero coefficient in E_{00}; `diagonal` names the products that are diagonal matrices (a matrix equals the diagonal matrix of its own diagonal exactly when it is diagonal). The printed list is I, 07, 16, 25, 34, 0167, 0257, 0347, 1256, 1346, 2345, 012567, 013467, 023457, 123456, 01234567: exactly the 16 sets made of whole pairs (x_1, x_6), (x_2, x_5), (x_3, x_4), (x_0, x_7), each pair either in or out, a pattern that Section 4.17 explains. The check requires that the two lists agree, that there are 16, and that every nonzero coefficient is $\pm 1/16$. (The reason: $c_A = \mathrm{tr}(\gamma_A^T E_{00})/16 = (\gamma_A)_{00}/16$, which is nonzero only when row 0 of the signed permutation γ_A points to column 0.)

In [12], figure 5.

```
fig, (top, bottom) = plt.subplots(2, 1, figsize=(8.0, 5.6), layout="constrained",
                                  sharex=True)
for ax, name, colour in ((top, "E00", "#2a78d6"), (bottom, "random", "#eb6834")):
    ax.bar(range(256), coefficients[name], color=colour, width=0.8)
    ax.axhline(0.0, color="#898781", linewidth=0.8)
    ax.set_ylabel("coefficient $c_A$")
    for start in starts[1:]:
        ax.axvline(start - 0.5, color="#c3c2b7", linewidth=0.6, linestyle=":")
top.set_title("$E_{00}$ (a single 1 in row 0, column 0)")
bottom.set_title("a matrix of random whole numbers from $-3$ to $3$")
bottom.set_xlabel("number of the product $\\gamma_A$ (dotted lines: degree changes)")
save_figure(fig, "coefficients",
            ...)
```

Two panels one above the other, sharing the horizontal axis (`sharex=True`): one bar per product with its coefficient, a grey zero line, and dotted lines where the degree changes. **What figure 5 shows and why:** for E_{00} only 16 bars, all of height $\pm 1/16$, at the diagonal products; for the

random matrix nearly every one of the 256 bars is nonzero. Every matrix, however special or random, is a combination of the 256 products.

In [13], even and odd products.

```
even, odd = flat[DEGREE % 2 == 0], flat[DEGREE % 2 == 1]  # rows of 256 numbers
even_blocks = all((m[:8, 8:] == 0).all() and (m[8:, :8] == 0).all()
                  for m in products[DEGREE % 2 == 0])
odd_blocks = all((m[:8, :8] == 0).all() and (m[8:, 8:] == 0).all()
                 for m in products[DEGREE % 2 == 1])
rank_even, rank_odd = exact_rank(even.tolist()), exact_rank(odd.tolist())
report("exact ranks of the even and of the odd products", (rank_even, rank_odd))
```

The masks `DEGREE % 2 == 0` and `== 1` select the even and the odd products. `even_blocks` requires every even product to be block diagonal (zero upper-right and lower-left blocks), `odd_blocks` every odd one to be block off-diagonal. Both exact ranks are 128.

```
check(even_blocks and odd_blocks and rank_even == 128 and rank_odd == 128
      and f"dimension {rank_even} = 2 x 8^2" in recorded(
          WOLFRAM, "even_subalgebra_dimension")
      and f"rank {rank_even} (Fraction), {rank_even} (sympy)" in recorded(
          PYTHON, "even_products_span_M8_plus_M8"),
      "even products block diagonal, odd block off-diagonal, ranks 128 and 128")
print(f"     reproduces {WOLFRAM}")
print("         check even_subalgebra_dimension")
print(f"     reproduces {PYTHON}")
print("         check even_products_span_M8_plus_M8")
```

The check combines the block forms, the two ranks and the statements of the two Revision checks.

```
support_even = np.sum(products[DEGREE % 2 == 0] != 0, axis=0)  # count per entry
support_odd = np.sum(products[DEGREE % 2 == 1] != 0, axis=0)
report("even products nonzero at each position (values found)",
       sorted(set(int(x) for x in support_even.flatten())))
```

For each position (i, j), `support_even` counts how many even products have a nonzero entry there (`axis=0` adds over the 128 products), and `support_odd` the same for the odd ones. The RESULT line shows that the even counts take only the values 0 and 16.

```
upper = np.arange(16) < 8  # True for the rows (and columns) 0 to 7
# in_diagonal_blocks[i, j] is True when row i and column j lie in the same half;
# ~ turns True into False and back.
in_diagonal_blocks = np.equal.outer(upper, upper)
check((support_even[in_diagonal_blocks] == 16).all()
      and (support_even[~in_diagonal_blocks] == 0).all()
      and (support_odd[~in_diagonal_blocks] == 16).all()
      and (support_odd[in_diagonal_blocks] == 0).all(),
      "each position of its blocks is covered by exactly 16 even (or odd) products")
```

`upper` marks the upper half of the rows; `np.equal.outer(upper, upper)` is the 16×16 table that is True where row and column lie in the same half (the two diagonal blocks). The check requires 16 even products at every position of the diagonal blocks and none elsewhere, and the opposite for the odd products.

In [14], figure 6.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.4, 4.2), layout="constrained")
cover = ListedColormap(["#f0efec", "#1c5cab"])  # 0 products, 16 products
for ax, values, title in ((left, support_even, "128 even products"),
                          (right, support_odd, "128 odd products")):
    ax.imshow(values, cmap=cover, norm=BoundaryNorm([-1, 8, 17], 2))
    ax.set_xticks(range(0, 16, 4))
    ax.set_yticks(range(0, 16, 4))
    ax.set_title(title)
    ax.set_xlabel("column")
    ax.set_ylabel("row")
    ax.axhline(7.5, color="#0b0b0b", linewidth=1)
    ax.axvline(7.5, color="#0b0b0b", linewidth=1)
    ax.grid(False)
save_figure(fig, "even_odd_blocks",
            ...)
```

Two heat maps of the counts, light grey for 0 and dark blue for 16, with black lines between the blocks. **What figure 6 shows and why:** the even products fill exactly the two diagonal 8×8 blocks, and the odd ones exactly the two others, because every gamma is block off-diagonal (Section 4.13).

In [15], how big must the matrices be?

```
def all_products(generators):
    """The 2^n ordered products of a list of square sympy matrices."""
    size = generators[0].shape[0]
    result = []
    for k in range(len(generators) + 1):
        for s in itertools.combinations(range(len(generators)), k):
            matrix = sp.eye(size)
            for a in s:
                matrix = matrix * generators[a]
            result.append(matrix)
    return result
```

`all_products` forms the 2^n ordered products of a list of n sympy matrices, exactly as In [2] did for the gammas.

```
sigma_x, sigma_y = sp.Matrix([[0, 1], [1, 0]]), sp.Matrix([[0, -sp.I], [sp.I, 0]])
sigma_z, N_matrix = sp.Matrix([[1, 0], [0, -1]]), sp.Matrix([[0, 1], [-1, 0]])
Z2, I2 = sp.zeros(2, 2), sp.eye(2)
dirac = [sp.Matrix(sp.BlockMatrix([[I2, Z2], [Z2, -I2]]))] + [
    sp.Matrix(sp.BlockMatrix([[Z2, s], [-s, Z2]])) for s in (sigma_x, sigma_y, sigma_z)]
taus = [sp.Matrix(gamma[a][8:, :8].tolist()) for a in range(1, 8)]  # tau[1..7]
```

The Pauli matrices, N, and Dirac's four matrices (Section 4.3), rebuilt in three lines. `taus` takes the lower-left 8×8 blocks of $\gamma^{(x_1)}, \ldots, \gamma^{(x_7)}$; by the author's construction these are $\tau_1, \ldots, \tau_7$ (Section 4.5).

```
examples = {"n = 2, real 2 x 2": [sigma_x, N_matrix], "n = 4, Dirac 4 x 4": dirac,
            "n = 7, tau 8 x 8": taus, "n = 8, gammas 16 x 16":
                [sp.Matrix(g.tolist()) for g in gamma]}
rows = []
for name, generators in examples.items():
    prods = all_products(generators)
    d = generators[0].shape[0]
    flat_rows = sp.Matrix([list(m) for m in prods])  # one row per product
    # exact rank; from_Matrix chooses exact numbers (Gaussian integers for Dirac)
    rank = DomainMatrix.from_Matrix(flat_rows).rank()
    rows.append((name, len(prods), rank, d * d))
    print(f"{name:24} products {len(prods):4d}   rank {rank:4d}   d^2 {d * d:4d}")
```

Four examples, each a list of matrices with a Clifford relation: σ_x and N (signs $+1, -1$), Dirac's matrices, the seven τ, and the eight gammas. For each, all 2^n products are formed; `list(m)` lists the entries of a sympy matrix row by row, so `flat_rows` has one row per product; `DomainMatrix.fro m_Matrix` chooses an exact number system (for Dirac's matrices the **Gaussian integers**, whole numbers plus whole multiples of i), and `.rank()` computes the exact rank. One line per example is printed: products 4, 16, 128, 256; ranks 4, 16, 64, 256; d^2 4, 16, 64, 256.

```
tau_product = sp.eye(8)
for t in taus:
    tau_product = tau_product * t
check([r[2] for r in rows] == [4, 16, 64, 256] and [r[1] for r in rows]
      == [4, 16, 128, 256], "ranks 4, 16, 64, 256: products fill d^2 for even n")
check(tau_product == sp.eye(8) and taus[6] == sp.diag(-1, -1, -1, -1, 1, 1, 1, 1)
      and "tau[7] = diag(-I4, I4) and tau[1]...tau[7] = ID8"
      in recorded(PYTHON, "tau7_and_product"),
      "tau[1] tau[2] ... tau[7] = I8 and tau[7] = diag(-I4, I4)")
print(f"     reproduces {PYTHON}")
print("         check tau7_and_product")
```

The product of the seven τ is computed. The first check requires the ranks and the numbers of products of the table; the second that $\tau_1 \cdots \tau_7 = I_8$ and $\tau_7 = \mathrm{diag}(-I_4, I_4)$, the reason why only 64 of the 128 products of the seven τ are independent (Section 4.13).

In [16], figure 7.

```
fig, ax = plt.subplots(figsize=(7.6, 4.0), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
positions = np.arange(len(rows))
for offset, column, colour, name in ((-0.25, 1, "#2a78d6", "number of products $2^n$"),
                                     (0.0, 2, "#eb6834", "exact rank"),
                                     (0.25, 3, "#1baf7a", "$d^2$ (all matrices)")):
    values = [r[column] for r in rows]
    bars = ax.bar(positions + offset, values, width=0.24, color=colour, label=name)
```

```
    ax.bar_label(bars, labels=[str(v) for v in values], fontsize=8, padding=1)
ax.set_yscale("log", base=2)
ax.set_ylim(1, 1024)
ax.set_xticks(positions, [r[0] for r in rows], fontsize=9)
ax.set_ylabel("count (logarithmic scale)")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "how_big",
            ...)
```

For each of the four examples three bars side by side (shifted by -0.25, 0 and $+0.25$): the number of products, the exact rank and d^2, each labelled with its value; the vertical axis is logarithmic with base 2, from 1 to 1024. **What figure 7 shows and why:** for $n = 2$, 4 and 8 the three bars of each group have the same height, $2^n = d^2$: the products fill all $d \times d$ matrices, so no smaller d is possible. For $n = 7$ the first bar (128) is taller than the other two (64): seven matrices fit into 8×8, but eight need 16×16.

In [17], the 28 products of two gammas and the matrices S^{ab}.

```
from fractions import Fraction  # exact fractions such as 1/2


def exact(entry):
    """A number of gammas.json: a JSON integer or a text "p/q", as a Fraction."""
    if isinstance(entry, int):
        return Fraction(entry)
    numerator, denominator = entry.split("/")
    return Fraction(int(numerator), int(denominator))
```

`exact` reads a number of `gammas.json` (a whole number, or a text "p/q") as an exact fraction, as in Notebook 04a.

```
PAIRS_AB = [(a, b) for a in range(8) for b in range(a + 1, 8)]  # the 28 pairs
two = {(a, b): products[INDEX[(a, b)]] for a, b in PAIRS_AB}  # gamma^a gamma^b
half_ok, record_ok, values = True, True, set()
```

`PAIRS_AB` lists the 28 pairs $a < b$, and `two` looks up their products $\gamma^a\gamma^b$ among the 256 products. Three variables start: two truth values and an empty set of the entries found.

```
for a, b in itertools.product(range(8), repeat=2):  # all 64 ordered pairs
    ab, ba = gamma[a] @ gamma[b], gamma[b] @ gamma[a]
    commutator = ab - ba  # 4 S^ab, a whole-number matrix
    if a != b:
        half_ok &= bool((commutator == 2 * ab).all())  # 4 S^ab = 2 gamma^a gamma^b
    else:
        half_ok &= not commutator.any()  # S^aa = 0
```

For all 64 ordered pairs the commutator $\gamma^a\gamma^b - \gamma^b\gamma^a = 4S^{ab}$ is computed with whole numbers. For $a \neq b$ it must equal $2\gamma^a\gamma^b$ (so $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$); for $a = b$ it must vanish (`.any()` is true when some entry is nonzero, `not` reverses it).

```
    stored = record["S"][a][b]  # the record's S^ab: 16 rows of exact numbers
    for i, j in itertools.product(range(16), repeat=2):
        ours = Fraction(int(commutator[i, j]), 4)  # S^ab entry, exact
        values.add(ours)
        record_ok &= exact(stored[i][j]) == ours
```

Still inside the loop: the record file stores S^{ab} under the key `"S"`; every one of its 256 entries is compared with the exact fraction (commutator entry)/4, and the entry is added to the set `values`.

```
report("the different entries of the 64 matrices S^ab",
       [str(v) for v in sorted(values)])
check(half_ok and record_ok and sorted(values) == [Fraction(-1, 2), 0,
                                                   Fraction(1, 2)]
      and "(1/2) gamma^a gamma^b for a != b" in recorded(PYTHON, "S_definition")
      and "S^ab = (1/2) gamma^a gamma^b" in recorded(WOLFRAM, "S_half_product")
      and "{-1/2, 0, 1/2}" in recorded(WOLFRAM, "S_real_entries_in_half_integers"),
      "S^ab = (1/4)(g^a g^b - g^b g^a) = (1/2) g^a g^b = the record's S, entry by entry")
print(f"     reproduces {PYTHON}")
print("         check S_definition")
print(f"     reproduces {WOLFRAM}")
print("         checks S_half_product, S_real_entries_in_half_integers")
```

The RESULT line lists the entries found, $-1/2$, 0 and $1/2$. The check combines: $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$ and $S^{aa} = 0$; all $64 \times 256 = 16384$ entries equal to the record's; the entries are exactly $-\frac{1}{2}, 0, \frac{1}{2}$; and the statements of the three Revision checks.

```
square_of = {pair: int((m @ m)[0, 0]) for pair, m in two.items()}  # +1 or -1
rotations = [pair for pair in PAIRS_AB if square_of[pair] == -1]
boosts = [pair for pair in PAIRS_AB if square_of[pair] == 1]
report("pairs whose product squares to -I16 (rotations), to +I16 (boosts)",
       (len(rotations), len(boosts)))
check(all((m @ m == square_of[(a, b)] * I16).all()
          and square_of[(a, b)] == -eta[a] * eta[b]
          and not m[:8, 8:].any() and not m[8:, :8].any()
          for (a, b), m in two.items())
      and len(rotations) == 12 and len(boosts) == 16,
      "(g^a g^b)^2 = -eta^aa eta^bb I16; 12 rotation, 16 boost planes; block diagonal")
```

For each of the 28 products the sign of its square is read off its upper-left entry; the pairs are sorted into rotation planes (square $-I_{16}$) and boost planes (square $+I_{16}$): $(12, 16)$. The check requires, for every pair, that the square is exactly that sign times I_{16}, that the sign is $-\eta^{aa}\eta^{bb}$, and that the product is block diagonal; and the counts 12 and 16.

In [18], figure 8.

```
from matplotlib.patches import Patch  # coloured squares for the key

FRAME = {-1: "#0b0b0b", 1: "#1baf7a"}  # square -I16: black, square +I16: green
fig, axes = plt.subplots(7, 7, figsize=(8.6, 8.9), layout="constrained")
```

The frame colours: black for a rotation plane, green for a boost plane. The figure has a grid of 7×7 panels.

```
for r, c_ in itertools.product(range(7), repeat=2):
    ax = axes[r, c_]
    a, b = r, c_ + 1  # row: first factor x_a; column: second factor x_b
    if b <= a:
        ax.axis("off")  # no panel below the diagonal of the grid (c_ < r)
        continue
    ax.imshow(two[(a, b)], cmap=three, norm=three_norm)
    ax.set_xticks([])
    ax.set_yticks([])
    for spine in ax.spines.values():  # the frame of the panel
        spine.set_edgecolor(FRAME[square_of[(a, b)]])
        spine.set_linewidth(2.0)
    ax.set_title(f"$x_{a}\\,x_{b}$", fontsize=8)
```

Panel (r, c) shows the product of the directions $a = r$ and $b = c + 1$ (rows x_0 to x_6, columns x_1 to x_7). The panels on the diagonal of the grid ($c = r$, that is $b = a + 1$) show the neighbouring pairs $x_0x_1, x_1x_2, \ldots, x_6x_7$. The panels below the diagonal ($c < r$, that is $b \leq a$) would repeat a pair or pair a direction with itself; they are switched off, and `continue` jumps to the next panel. Each remaining panel shows $\gamma^a\gamma^b$ as a heat map without tick marks; its four border lines (`spines`) get the colour of its plane, two points wide, and its title names the two directions.

```
key = [Patch(facecolor="white", edgecolor=FRAME[-1], linewidth=2,
             label="square $-I_{16}$: rotation plane (12)"),
       Patch(facecolor="white", edgecolor=FRAME[1], linewidth=2,
             label="square $+I_{16}$: boost plane (16)"),
       Patch(facecolor="#2a78d6", label="entry $-1$"),
       Patch(facecolor="#e34948", label="entry $+1$")]
fig.legend(handles=key, loc="outside lower center", ncol=2, fontsize=9,
           frameon=False)
save_figure(fig, "two_gamma_products",
            ...)
```

The key below the figure explains the two frame colours and the two entry colours. **What figure 8 shows and why:** a triangle of 28 small heat maps, the products of two of the author's real gammas, each block diagonal with 16 nonzero entries (two of every four quadrants are empty), because each is an even product. The black frames sit where both directions are of the same kind (among x_0, x_1, x_2, x_3, or among $x_4, \ldots, x_7$), the green ones where one is space-like and one time-like. Four panels are diagonal matrices: x_0x_7, x_1x_6, x_2x_5 and x_3x_4 (Section 4.17 explains why exactly these). Half of each product is the matrix S^{ab} of the Revision record.

In [19], the last check.

```
names = ["04c_1_products_by_degree.png", "04c_2_multiplication_table.png",
         "04c_3_squares_by_degree.png", "04c_4_trace_products.png",
         "04c_5_coefficients.png", "04c_6_even_odd_blocks.png", "04c_7_how_big.png",
         "04c_8_two_gamma_products.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
```

```
     "all eight figure files exist")
all_checks_passed()
```

The eight figure files must exist; the last line reads ALL 19 CHECKS PASSED (notebook 04c): one check in each of In [2], In [10] and In [19]; two in each of In [4], In [6], In [13], In [15] and In [17]; and three in each of In [8] and In [11].

4.17 A second set of gammas and the one change of basis

The author's gammas are one solution of the Clifford relation. Are they special? This section builds a second set of eight real 16×16 gammas from three 2×2 matrices, with a recipe that needs nothing from the author's notebook, and proves that the two sets differ only by a renumbering of the sixteen components and some signs. It also proves that this renumbering is the only change of basis between them, and that only the multiples of I_{16} commute with all eight gammas.

Three 2×2 matrices.

$$P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad N = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \qquad G = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

(P and G are the σ_x and σ_z of Section 4.3). They are real; P and G are symmetric and N is antisymmetric; $P^2 = G^2 = I_2$ and $N^2 = -I_2$; and any two of them anticommute: $PN = -G = -NP$, $PG = -N = -GP$, $NG = -P = -GN$. Each of these is one 2×2 multiplication; for example $PG = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -N$ and $GP = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix} = N$.

The Kronecker product. For a 2×2 matrix A and an $m \times m$ matrix B, the **Kronecker product** (or tensor product) $A \otimes B$ is the $2m \times 2m$ matrix of four blocks whose block in position (i, j) is $A_{ij}B$; in indices, $(A \otimes B)_{im+k,\ jm+l} = A_{ij}B_{kl}$. For example

$$P \otimes G = \begin{pmatrix} 0 \cdot G & 1 \cdot G \\ 1 \cdot G & 0 \cdot G \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{pmatrix}.$$

The **mixed-product rule** $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$ holds. Proof, line by line, for the entry in row $im + k$ and column $jm + l$:

1. $\big((A \otimes B)(C \otimes D)\big)_{im+k,\,jm+l} = \sum_{r,s} (A \otimes B)_{im+k,\,rm+s}\,(C \otimes D)_{rm+s,\,jm+l}$ — the product rule, summing over all columns $rm + s$ of the first factor.
2. $= \sum_{r,s} A_{ir}B_{ks}C_{rj}D_{sl}$ — the index formula, twice.
3. $= \big(\sum_r A_{ir}C_{rj}\big)\big(\sum_s B_{ks}D_{sl}\big)$ — the double sum of products splits into a product of two sums.
4. $= (AC)_{ij}(BD)_{kl} = \big((AC) \otimes (BD)\big)_{im+k,\,jm+l}$ — the product rule and the index formula.

In the same way $(A \otimes B)^T = A^T \otimes B^T$ (exchange i with j and k with l in the index formula). A product of four 2×2 matrices, $M_1 \otimes M_2 \otimes M_3 \otimes M_4$, is a 16×16 matrix; M_1 stands in **slot** 1, ..., M_4 in slot 4, and products are computed slot by slot. The 16 components are numbered $j = 8b_1 + 4b_2 + 2b_3 + b_4$ by four binary digits (**bits**) $b_1, \ldots, b_4$, each 0 or 1, one per slot; for example $j = 5$ has the bits 0101.

The recipe. For a slot k and $M = P$ or N, put G into the slots before k, M into slot k and I_2 into the slots after k. Every space-like direction gets a P, every time-like direction an N, and each slot is shared by one space-like and one time-like direction. With the pairs (x_1, x_6) in slot 1, (x_2, x_5) in slot 2, (x_3, x_4) in slot 3 and (x_0, x_7) in slot 4 (why exactly these pairs is explained below):

direction	**slot 1**	**slot 2**	**slot 3**	**slot 4**
$\hat{\gamma}^{(x_1)}$	P	I_2	I_2	I_2
$\hat{\gamma}^{(x_6)}$	N	I_2	I_2	I_2
$\hat{\gamma}^{(x_2)}$	G	P	I_2	I_2
$\hat{\gamma}^{(x_5)}$	G	N	I_2	I_2
$\hat{\gamma}^{(x_3)}$	G	G	P	I_2
$\hat{\gamma}^{(x_4)}$	G	G	N	I_2
$\hat{\gamma}^{(x_0)}$	G	G	G	P
$\hat{\gamma}^{(x_7)}$	G	G	G	N

for example $\hat{\gamma}^{(x_5)} = G \otimes N \otimes I_2 \otimes I_2$ (read "gamma hat").

The recipe satisfies the Clifford relation. By the mixed-product rule every product is computed slot by slot.

1. Squares: every slot is squared; $G^2 = I_2^2 = P^2 = I_2$ and $N^2 = -I_2$, so a matrix with P squares to $+I_{16}$ (space-like) and one with N to $-I_{16}$ (time-like).

2. The two matrices of the same slot k: their two products agree in every slot except slot k, which holds PN in one order and $NP = -PN$ in the other; so they anticommute.

3. Two matrices of different slots $k < l$: compare the two orders slot by slot. Before k both have G ($GG = GG$); in slot k the first has P or N and the second G, which anticommute (one factor -1); between k and l the first has I_2 and the second G, which commute; in slot l the first has I_2, which commutes with everything; after l both have I_2. Exactly one factor -1: they anticommute.

Each $\hat{\gamma}$ is a signed permutation matrix (a Kronecker product of signed permutation matrices is one: each row combines one row of each factor and so holds one nonzero product), and it has the author's symmetry pattern $(\hat{\gamma}^{(x_a)})^T = \eta_{aa}\hat{\gamma}^{(x_a)}$, because P, G and I_2 are symmetric, N is antisymmetric and the transpose works slot by slot. By Section 4.13 its 256 products are a basis of all real 16×16 matrices.

The product of all eight, in both sets. The product of the two matrices of slot k is, slot by slot, $GG = I_2$ before k, $PN = -G$ in slot k and I_2 after k: minus the matrix with G in slot k and I_2 elsewhere. The product of all eight hat gammas in the author's order, $\hat{\Gamma} = \hat{\gamma}^{(x_0)}\hat{\gamma}^{(x_1)}\hat{\gamma}^{(x_2)} \ldots \hat{\gamma}^{(x_7)}$, can be reordered into the pairs $(x_1, x_6), (x_2, x_5), (x_3, x_4), (x_0, x_7)$; each exchange of two neighbouring (anticommuting) factors gives a factor -1 and changes the number of inversions by one, so the

reordering multiplies the product by the permutation sign of the reordering, which is $+1$ (Notebook 04d counts the 12 inversions). The four pair products then give $(-1)^4\, G\otimes G\otimes G\otimes G = G\otimes G\otimes G\otimes G$. Since $G = \mathrm{diag}(+1,-1)$, its diagonal entry in row j is $(-1)^{b_1+b_2+b_3+b_4}$: -1 in the eight rows $1,2,4,7,8,11,13,14$, whose bits contain an odd number of ones. The author's product is $\mathrm{diag}(-I_8, I_8)$ (Section 4.5). Both have eight entries of each sign, in different rows.

The change of basis, line by line. Let γ_A and $\hat\gamma_A$ be the products of the two sets for the 256 sets A (Section 4.13). For any 16×16 matrix M define

$$S = \sum_A \gamma_A\, M\, \hat\gamma_A^T.$$

1. Fix a direction b. By R1, $\gamma^{(b)}\gamma_A = \epsilon\,\gamma_{A'}$ with $A' = A\triangle\{b\}$ (b added to A, or removed if it was in A) and a sign $\epsilon = \pm 1$ that depends only on b, A and η. The hat gammas obey the same Clifford relation, so $\hat\gamma^{(b)}\hat\gamma_A = \epsilon\,\hat\gamma_{A'}$ with the SAME sign.
2. Both sets are signed permutation matrices, hence orthogonal: $(\hat\gamma^{(b)})^{-1} = (\hat\gamma^{(b)})^T$. From line 1, $\hat\gamma_A = \epsilon\,(\hat\gamma^{(b)})^T\hat\gamma_{A'}$, and transposing (Section 4.2), $\hat\gamma_A^T = \epsilon\,\hat\gamma_{A'}^T\hat\gamma^{(b)}$.
3. Therefore $\gamma^{(b)}S = \sum_A \epsilon\,\gamma_{A'}\, M\,\epsilon\,\hat\gamma_{A'}^T\hat\gamma^{(b)} = \big(\sum_A \gamma_{A'} M\hat\gamma_{A'}^T\big)\hat\gamma^{(b)}$ — lines 1 and 2 inserted, and $\epsilon^2 = 1$.
4. As A runs through all 256 sets, $A' = A\triangle\{b\}$ runs through all 256 sets too, each exactly once (doing it twice gives A back). So the sum is S again: $\gamma^{(b)}S = S\hat\gamma^{(b)}$ for every b.

A matrix X with $\gamma^{(b)}X = X\hat\gamma^{(b)}$ for every b is called an **intertwiner**; S is one, for every M.

S is orthogonal up to a factor. Transposing $\gamma^{(b)}S = S\hat\gamma^{(b)}$ gives $S^T(\gamma^{(b)})^T = (\hat\gamma^{(b)})^T S^T$; both sets have the same symmetry pattern, so after dividing by $\eta_{bb} = \pm 1$: $S^T\gamma^{(b)} = \hat\gamma^{(b)}S^T$. Then $(S^TS)\hat\gamma^{(b)} = S^T\gamma^{(b)}S = \hat\gamma^{(b)}(S^TS)$: the matrix S^TS commutes with all eight hat gammas.

Only multiples of the identity commute with all gammas. Suppose X commutes with every $\hat\gamma^{(b)}$. Then it commutes with every product $\hat\gamma_A$ and with every combination of them, that is (Section 4.13) with every real 16×16 matrix, in particular with E_{ij}, the matrix with a single 1 in row i and column j. The entry in row k, column j of XE_{ij} is X_{ki}; that of $E_{ij}X$ is X_{jj} if $k = i$ and 0 otherwise. So $X_{ki} = 0$ for $k \neq i$, and $X_{ii} = X_{jj}$: $X = c\,I_{16}$. The same holds for the author's gammas. Applied to S^TS: $S^TS = c\,I_{16}$, with $c > 0$ when $S \neq 0$ (the diagonal entries of S^TS are sums of squares of the columns of S). Then $Q = S/\sqrt{c}$ satisfies $Q^TQ = I_{16}$ and

$$\gamma^{(x_a)} = Q\,\hat\gamma^{(x_a)}\,Q^T \qquad \text{for all eight directions.}$$

Unique up to a factor. If X is any intertwiner, then $Q\hat\gamma^{(b)}Q^TX = X\hat\gamma^{(b)}$; multiplying from the left by Q^T gives $\hat\gamma^{(b)}(Q^TX) = (Q^TX)\hat\gamma^{(b)}$, so Q^TX commutes with every hat gamma, $Q^TX = cI_{16}$, and $X = cQ$.

What Notebook 04d finds. With $M = E_{08}$ (the first choice that gives a nonzero S), $S = 16\,Q$ with a **signed permutation matrix** Q ($c = 256$). Its codes, rows 0 to 15, are $+8,+4,+2,-14,-7,+11,-13,-1,-9,-5,-3,+15,-6,+10,-12,-0$; the code $+8$ in row 0 means that $(Q\chi)_0 = +\chi_8$: component 0 of the author's basis is component 8 of the tensor basis. **The**

author's T16 are the tensor-product gammas with the sixteen components renumbered and some of their signs flipped. The first eight rows pick the eight tensor components with an odd number of ones in their bits, where $\hat{\Gamma} = -1$, which is why the author's product of all eight gammas is $\mathrm{diag}(-I_8, I_8)$. Status: PROVED (exact integer arithmetic; Notebook 04d checks $\gamma_A = Q\hat{\gamma}_A Q^T$ for all 256 products).

What it means for the field equation. In flat 4+4 space write $\Psi = Q\chi$. Then $\gamma^{(x_a)}\partial_a\Psi = Q\hat{\gamma}^{(x_a)}Q^TQ\,\partial_a\chi = Q\,\hat{\gamma}^{(x_a)}\partial_a\chi$, so $\sum_a \gamma^{(x_a)}\partial_a\Psi = m\Psi$ becomes $Q(\sum_a \hat{\gamma}^{(x_a)}\partial_a\chi - m\chi) = 0$, and, since Q is invertible, $\sum_a \hat{\gamma}^{(x_a)}\partial_a\chi = m\chi$: the same equation in the other basis. The same argument applies to every equation of the book that is written with products of the gammas and numbers, because all 256 products transform with Q (Notebook 04d checks them all). The choice between the two sets is a choice of how to number the sixteen components, not a choice of physics.

Why these pairs of directions. A signed permutation Q turns a diagonal matrix D into a diagonal matrix QDQ^T (it only renumbers the diagonal entries and multiplies each by $(\pm1)^2 = 1$). So if $\gamma_A = Q\hat{\gamma}_A Q^T$ with a signed permutation Q, then γ_A is diagonal exactly when $\hat{\gamma}_A$ is: both sets must have their diagonal products for the same sets A. The author's 16 diagonal products belong to the sets made of whole pairs (x_1, x_6), (x_2, x_5), (x_3, x_4), (x_0, x_7) (Notebook 04c, In [11]); the diagonal products of the hat set belong to the sets made of whole slot pairs (the pair product of a slot is $-G$ in that slot, a diagonal matrix). So the slot pairs must be the author's pairs. With other pairs, for example $(x_1, x_4), (x_2, x_5), (x_3, x_6), (x_0, x_7)$, the Clifford relation still holds and a change of basis still exists (the argument above did not use the pairs), but it cannot be a signed permutation: Notebook 04d finds Q_2 with two entries $\pm1/\sqrt{2}$ in every row, which mixes the components two by two.

How strongly the equations reject every other matrix. The equations $\gamma^{(x_a)}X - X\hat{\gamma}^{(x_a)} = 0$ for the eight directions $x_a = x_0, \ldots, x_7$ are $8 \times 256 = 2048$ linear equations for the 256 entries of X. Write A for this system as a 2048×256 matrix, so that the sum of the squares of all 2048 left-hand sides is v^TKv with $K = A^TA$, where v lists the entries of X. K has the exact eigenvectors $X_B = Q\hat{\gamma}_B$ for the 256 sets B. Line by line:

1. Write $L_a(X) = \gamma^{(a)}X - X\hat{\gamma}^{(a)}$ for the left-hand side of equation block a; A is the stack of the eight blocks L_a, and $K = \sum_a L_a^T L_a$ (the product of a stack with its transpose adds up the products of the blocks).

2. The transpose of the block L_a is the map $Y \to (\gamma^{(a)})^TY - Y(\hat{\gamma}^{(a)})^T$ (the rows of the system become its columns). Both sets have $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$ and $(\hat{\gamma}^{(a)})^T = \eta_{aa}\hat{\gamma}^{(a)}$, so $L_a^T = \eta_{aa}L_a$ and $K = \sum_a \eta_{aa}L_aL_a$.

3. $L_a(X_B) = Q\hat{\gamma}^{(a)}Q^TQ\hat{\gamma}_B - Q\hat{\gamma}_B\hat{\gamma}^{(a)} = Q(\hat{\gamma}^{(a)}\hat{\gamma}_B - \hat{\gamma}_B\hat{\gamma}^{(a)})$ — insert $\gamma^{(a)} = Q\hat{\gamma}^{(a)}Q^T$ and $Q^TQ = I_{16}$. By R3 this is 0 when $\hat{\gamma}^{(a)}$ commutes with $\hat{\gamma}_B$, and $2Q\hat{\gamma}^{(a)}\hat{\gamma}_B$ when it anticommutes.

4. In the anticommuting case apply L_a once more: $L_a(2Q\hat{\gamma}^{(a)}\hat{\gamma}_B) = 2Q(\hat{\gamma}^{(a)}\hat{\gamma}^{(a)}\hat{\gamma}_B - \hat{\gamma}^{(a)}\hat{\gamma}_B\hat{\gamma}^{(a)}) = 2Q(\eta_{aa}\hat{\gamma}_B + \eta_{aa}\hat{\gamma}_B) = 4\eta_{aa}X_B$ — the square is η_{aa}, and $\hat{\gamma}_B\hat{\gamma}^{(a)} = -\hat{\gamma}^{(a)}\hat{\gamma}_B$. Multiplied by the η_{aa} of line 2 it gives $4X_B$, because $\eta_{aa}^2 = 1$.

5. Adding over a: $KX_B = 4n_BX_B$, where n_B is the number of directions whose gamma anticommutes with $\hat{\gamma}_B$; by R3 these are the k directions in B for an even degree k, and the $8 - k$ directions outside B for an odd k.

So the eigenvalues of K are $0, 4, 8, \ldots, 32$, with the multiplicities $\binom{8}{0}, \binom{8}{1}, \ldots, \binom{8}{8}$, and the eigenvalue 0, which means "solves all 2048 equations", belongs only to $B = \{\}$, that is to $X = Q$. Because the

256 eigenvectors X_B are perpendicular to each other (Section 4.13), every matrix X is a combination of them, and the part of X perpendicular to Q raises the sum of the squared equations by at least 4 times its squared size (the sum of the squares of its entries).

Status. PROVED (exact integer arithmetic and exact ranks in Notebook 04d): the hat gammas satisfy the Clifford relation with the author's η and symmetry pattern; their 256 products have rank 256; $\gamma_A = Q\hat{\gamma}_A Q^T$ for all 256 products with the signed permutation Q; the intertwiners form a space of dimension 1 (the system has the exact rank 255); and only the multiples of I_{16} commute with all eight of the author's gammas, which reproduces `Revision/algebra/reports/python-algebra.json`, check `pin_commutant_dimension_1`, and `Revision/algebra/reports/wolfram-algebra.json`, check `Pin44_irreducible_commutant_dim_1`. Chapter 5 explains what this last statement means for the group Pin(4,4): the sixteen components cannot be split into smaller pieces that the reflections and rotations of the 4+4 space-time keep apart.

The next three sections hold Notebook 04d.

4.18 How to run Notebook 04d

Step 1. What this notebook does and what it needs.

Notebook 04d (A second set of gammas from tensor products, and the change of basis to the author's T16) is the file `Revision/textbook/notebooks/04d_tensor_gammas.ipynb` of the repository Dirac_claude. It builds a second set of eight real 16 by 16 gamma matrices for the author's 4+4 directions from three 2 by 2 matrices with Kronecker (tensor) products, checks exactly that they satisfy the same Clifford relation, and constructs the change of basis that turns them into the author's gammas: a signed permutation matrix Q with gamma = Q times the tensor gamma times the transpose of Q for all eight directions. It proves exactly that this change of basis is unique up to a factor (the solution space has dimension 1; the commutant of the author's gammas has dimension 1, as the Revision record found), shows what changes when the 2 by 2 blocks are distributed differently, and draws six figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
```

```
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04d_tensor_gammas.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04d_tensor_gammas.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 04d_tensor_gammas.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/04d.captions.json`
- `Revision/textbook/figures/04d_1_slot_pattern.png`
- `Revision/textbook/figures/04d_2_tensor_gammas.png`
- `Revision/textbook/figures/04d_3_chirality_diagonals.png`
- `Revision/textbook/figures/04d_4_change_of_basis.png`
- `Revision/textbook/figures/04d_5_penalty_spectrum.png`
- `Revision/textbook/figures/04d_6_two_assignments.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 18 CHECKS PASSED (notebook 04d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/04d_tensor_gammas.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 04d_tensor_gammas.ipynb
python -m nbconvert --execute --inplace 04d_tensor_gammas.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `Revision/algebra/gammas.json` or for a file in `Revision/algebra/reports`: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run `git status` in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

```
git checkout HEAD Revision/algebra
```

4.19 Notebook 04d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/04d_tensor_gammas.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The author's eight gamma matrices are ONE solution of the Clifford relation
$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I_{16}$ with
$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order of his
coordinates $x_0, \dots, x_7$. Are they special? This notebook answers the question
exactly. It

- builds a second set of eight real $16 \times 16$ matrices $\hat\gamma^{(x_a)}$
  ("gamma hat") from three $2 \times 2$ matrices P, N and G with the *Kronecker
  product*, a recipe that needs no notebook of the author at all;
- checks exactly that the new set satisfies the same Clifford relation, has the same
  symmetry pattern, and that its 256 products are independent;
- compares the products of all eight gammas in both sets (the *chirality product*);
- constructs, with one formula, the change of basis that turns the new set into the
  author's: a $16 \times 16$ signed permutation matrix $Q$ with
  $\gamma^{(x_a)} = Q\,\hat\gamma^{(x_a)}\,Q^T$ for all eight directions, so the
  author's matrices are the new ones with the 16 components renumbered and some of
```

```
  their signs flipped;
- proves exactly that this change of basis is unique up to a factor, and reproduces
  the Revision record's result that only multiples of the identity commute with all
  eight of the author's gammas;
- distributes the $2 \times 2$ blocks in another way and shows that a change of
  basis still exists but is no longer a signed permutation, and why;
- draws six figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 04d (A second set of gammas from tensor products, and the change of basis to the
author's T16) is the file `Revision/textbook/notebooks/04d_tensor_gammas.ipynb` of the
repository Dirac_claude. It builds a second set of eight real 16 by 16 gamma matrices for
the author's 4+4 directions from three 2 by 2 matrices with Kronecker (tensor) products,
checks exactly that they satisfy the same Clifford relation, and constructs the change of
basis that turns them into the author's gammas: a signed permutation matrix Q with gamma
= Q times the tensor gamma times the transpose of Q for all eight directions. It proves
exactly that this change of basis is unique up to a factor (the solution space has
dimension 1; the commutant of the author's gammas has dimension 1, as the Revision record
found), shows what changes when the 2 by 2 blocks are distributed differently, and draws
six figures. It needs a computer with Windows 11, macOS or Linux, an internet connection
for the installation, the program Git, and Python 3.12 or newer (the notebooks were built
with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:
```

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 04d_tensor_gammas.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,

copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 04d_tensor_gammas.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 04d_tensor_gammas.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/04d.captions.json‘
- ‘Revision/textbook/figures/04d_1_slot_pattern.png‘
- ‘Revision/textbook/figures/04d_2_tensor_gammas.png‘
- ‘Revision/textbook/figures/04d_3_chirality_diagonals.png‘
- ‘Revision/textbook/figures/04d_4_change_of_basis.png‘
- ‘Revision/textbook/figures/04d_5_penalty_spectrum.png‘
- ‘Revision/textbook/figures/04d_6_two_assignments.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 18 CHECKS PASSED (notebook 04d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/04d_tensor_gammas.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 04d_tensor_gammas.ipynb
      python -m nbconvert --execute --inplace 04d_tensor_gammas.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘Revision/algebra/gammas.json‘ or for a file in ‘Revision/algebra/reports‘: the notebook reads these Revision record files of the repository; they are part of every complete clone. Run ‘git status‘ in the repository folder: if it reports them as deleted, restore them with the command below and run the notebook again.

      git checkout HEAD Revision/algebra

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a

check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 04d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 04d (A second set of gammas from tensor products, and the change of basis to
# the author's T16) is the file Revision/textbook/notebooks/04d_tensor_gammas.ipynb of
# the repository Dirac_claude. It builds a second set of eight real 16 by 16 gamma
# matrices for the author's 4+4 directions from three 2 by 2 matrices with Kronecker
# (tensor) products, checks exactly that they satisfy the same Clifford relation, and
# constructs the change of basis that turns them into the author's gammas: a signed
# permutation matrix Q with gamma = Q times the tensor gamma times the transpose of Q for
# all eight directions. It proves exactly that this change of basis is unique up to a
# factor (the solution space has dimension 1; the commutant of the author's gammas has
# dimension 1, as the Revision record found), shows what changes when the 2 by 2 blocks
# are distributed differently, and draws six figures. It needs a computer with Windows
# 11, macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
```

```
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 04d_tensor_gammas.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
```

```
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 04d_tensor_gammas.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 04d_tensor_gammas.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/04d.captions.json
# - Revision/textbook/figures/04d_1_slot_pattern.png
# - Revision/textbook/figures/04d_2_tensor_gammas.png
# - Revision/textbook/figures/04d_3_chirality_diagonals.png
# - Revision/textbook/figures/04d_4_change_of_basis.png
# - Revision/textbook/figures/04d_5_penalty_spectrum.png
# - Revision/textbook/figures/04d_6_two_assignments.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 18 CHECKS PASSED (notebook 04d)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/04d_tensor_gammas.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
```

```
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 04d_tensor_gammas.ipynb
#     python -m nbconvert --execute --inplace 04d_tensor_gammas.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json or for a file in
#   Revision/algebra/reports: the notebook reads these Revision record files of the
#   repository; they are part of every complete clone. Run git status in the repository
#   folder: if it reports them as deleted, restore them with the command below and run
#   the notebook again.
#     git checkout HEAD Revision/algebra
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "04d"  # this notebook: chapter 04, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
```

```
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
```

```
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 04d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Matrix, product, identity, transpose**: a matrix is a rectangular table of
```

```
  numbers, $M_{ij}$ its entry in row $i$ and column $j$ (counted from 0 here);
  $(AB)_{ij} = \sum_k A_{ik}B_{kj}$, written ‘A @ B‘ in Python; $I_n$ is the
  $n \times n$ identity; the transpose has $(M^T)_{ij} = M_{ji}$, and
  $(AB)^T = B^T A^T$. $M$ is symmetric if $M^T = M$ and antisymmetric if
  $M^T = -M$.
- **Anticommute**: $AB = -BA$. **Clifford relation**: $\gamma^a\gamma^b +
  \gamma^b\gamma^a = 2\eta^{ab} I_{16}$: each gamma squares to $+I_{16}$
  (space-like direction) or $-I_{16}$ (time-like direction), and two different
  gammas anticommute.
- **Product of a set of directions** $\gamma_A$: for a set $A$ of directions, the
  product of their gammas in increasing order, for example
  $\gamma_{\{x_2, x_5\}} = \gamma^{(x_2)}\gamma^{(x_5)}$; $\gamma_{\{\}} = I_{16}$.
  There are $2^8 = 256$ sets, so 256 products; they are a *basis* of all real
  $16 \times 16$ matrices when every such matrix is exactly one combination
  $\sum_A c_A\gamma_A$.
- **Kronecker product** (or tensor product) $A \otimes B$ of a $2 \times 2$ matrix $A$
  and an $m \times m$ matrix $B$: the $2m \times 2m$ matrix made of four blocks, the
  block in position $(i, j)$ being $A_{ij} B$. In indices,
  $(A \otimes B)_{im + k,\; jm + l} = A_{ij} B_{kl}$. numpy writes it ‘np.kron(A, B)‘.
- **Slot**: in $M_1 \otimes M_2 \otimes M_3 \otimes M_4$ (four $2 \times 2$
  matrices, a $16 \times 16$ matrix) the position of a factor: $M_1$ stands in slot 1,
  ..., $M_4$ in slot 4.
- **Bits**: the 16 components are numbered $j = 8b_1 + 4b_2 + 2b_3 + b_4$ with four
  binary digits (bits) $b_1, b_2, b_3, b_4$, each 0 or 1, one per slot; for example
  $j = 5$ has the bits 0101.
- **Signed permutation matrix**: exactly one nonzero entry, $+1$ or $-1$, in every row
  and every column. It renumbers the components of a column and flips some signs.
  It is **orthogonal**: $M^T M = I$, so $M^{-1} = M^T$.
- **Change of basis**: two sets of matrices are related by a change of basis when an
  invertible matrix $Q$ gives $\gamma^a = Q\,\hat\gamma^a Q^{-1}$ for every $a$.
  Writing the column $\Psi = Q\chi$ then turns every equation written with one set
  into the same equation written with the other.
- **Intertwiner**: a matrix $X$ with $\gamma^a X = X \hat\gamma^a$ for every $a$.
- **Commute with all gammas**: $X\gamma^a = \gamma^a X$ for all eight $a$.
- **Solution space**: the set of all solutions $X$ of a list of linear equations
  without constant terms; its **dimension** is the number of free parameters.
  For 256 unknowns it is $256$ minus the **rank** of the equations (the largest number
  of independent ones). sympy computes ranks exactly with fractions.
- **Eigenvalue, eigenvector, multiplicity**: $Mv = \lambda v$ with $v \neq 0$; the
  multiplicity of $\lambda$ is the number of independent eigenvectors that belong to it.
- **Chirality product**: the product
  $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)} \cdots \gamma^{(x_7)}$ of all
  eight gammas, in this order (the author’s order); the Revision record calls it the
  chirality. Here it is only a product of eight matrices.
- **Diagonal matrix**: all entries off the diagonal are 0.
```

Text cell:

```
## 4. The physical and mathematical situation

The author’s space-time has the coordinates $x_0$ (the hidden direction),
$x_1, x_2, x_3$ (3-space, inflating), $x_4$ (the time) and $x_5, x_6, x_7$ (three
extra times, which deflate exponentially: scale factor
```

$b_4^{1/2}e^{-a_4}\sin^{1/6} z$). In a local
frame the signs are $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$: four
space-like and four time-like directions. The gamma matrices are constant matrices
of the frame; the inflation and the deflation enter the field equation through the
frame factors, never through the gammas. So the question of this notebook is pure
algebra: how many essentially different real 16×16 solutions of the
Clifford relation are there?

Three 2×2 matrices.
$P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$,
$N = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$,
$G = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$. They are real; P and G are
symmetric, N is antisymmetric; $P^2 = G^2 = I_2$ and $N^2 = -I_2$; and any two
of them anticommute: $PN = -G = -NP$, $PG = -N = -GP$, $NG = -P = -GN$ (each is
one 2×2 multiplication; for example
$PG = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -N$).

The mixed-product rule. $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$. Line
by line: the entry in row $im + k$, column $jm + l$ of the left side is
$\sum_{r,s} (A \otimes B)_{im+k,\, rm+s} (C \otimes D)_{rm+s,\, jm+l}$ (the product
rule, summing over all columns $rm + s$ of the first factor); by the index formula
this is $\sum_{r,s} A_{ir}B_{ks}C_{rj}D_{sl}$; the sum splits into
$(\sum_r A_{ir}C_{rj})(\sum_s B_{ks}D_{sl}) = (AC)_{ij}(BD)_{kl}$, which is the entry
of $(AC) \otimes (BD)$. With four slots the rule works slot by slot. Also
$(A \otimes B)^T = A^T \otimes B^T$.

The recipe. For a slot $k = 1, 2, 3, 4$ and $M = P$ or N put G into the
slots before k, M into slot k and I_2 into the slots after k. Every
space-like direction gets a P and every time-like direction an N, and each slot
is shared by one space-like and one time-like direction. This notebook uses the
pairs (x1, x6) in slot 1, (x2, x5) in slot 2, (x3, x4) in slot 3 and (x0, x7) in
slot 4; for example
$\hat\gamma^{(x_1)} = P \otimes I_2 \otimes I_2 \otimes I_2$,
$\hat\gamma^{(x_5)} = G \otimes N \otimes I_2 \otimes I_2$ and
$\hat\gamma^{(x_0)} = G \otimes G \otimes G \otimes P$. (Section 12 of this notebook,
below, explains why these pairs are the ones that match the author's matrices.)

Why the recipe satisfies the Clifford relation (by the mixed-product rule, every
product is computed slot by slot):

1. Squares: every slot is squared; $G^2 = I_2^2 = P^2 = I_2$ and $N^2 = -I_2$, so a
   matrix with P squares to $+I_{16}$ (space-like) and one with N to $-I_{16}$
   (time-like).
2. The two matrices of the same slot k: their two products agree in every slot
   except slot k, which holds PN in one order and $NP = -PN$ in the other; so
   they anticommute.
3. Two matrices of different slots $k < l$: compare the two orders slot by slot.
   Before k both have G ($GG = GG$); in slot k the first has P or N and the
   second G, which anticommute (one factor -1); between k and l the first has
   I_2 and the second G (they commute); in slot l the first has I_2 (it
   commutes with everything); after l both have I_2. Exactly one factor -1:
   they anticommute.

The change of basis, line by line. Let γ_A and $\hat\gamma_A$ be the

products of the gammas of a set A of directions in increasing order, for all 256 sets A (with $\gamma_{\{\}} = I_{16}$). For any 16×16 matrix M define $S = \sum_A \gamma_A M \hat\gamma_A^T$.

1. Fix a direction b. Moving $\gamma^{(b)}$ into its place in $\gamma^{(b)}\gamma_A$ uses only the Clifford relation, so $\gamma^{(b)}\gamma_A = \epsilon\,\gamma_{A'}$ with $A' = A \triangle \{b\}$ (b added to A, or removed if it was in A) and a sign $\epsilon = \pm 1$ that depends only on b, A and η. The hat matrices satisfy the same relation, so $\hat\gamma^{(b)}\hat\gamma_A = \epsilon\,\hat\gamma_{A'}$ with the SAME sign.
2. Both sets are signed permutation matrices, hence orthogonal: $(\hat\gamma^{(b)})^{-1} = (\hat\gamma^{(b)})^T$. From step 1, $\hat\gamma_A = \epsilon\,(\hat\gamma^{(b)})^T\hat\gamma_{A'}$, and transposing, $\hat\gamma_A^T = \epsilon\,\hat\gamma_{A'}^T\hat\gamma^{(b)}$.
3. Therefore $\gamma^{(b)} S = \sum_A \epsilon\,\gamma_{A'} M\, \epsilon\,\hat\gamma_{A'}^T\hat\gamma^{(b)} = (\sum_A \gamma_{A'} M \hat\gamma_{A'}^T)\,\hat\gamma^{(b)}$, because $\epsilon^2 = 1$.
4. As A runs through all 256 sets, $A' = A \triangle \{b\}$ also runs through all 256 sets, each exactly once (doing it twice gives A back). So the sum is S again: $\gamma^{(b)} S = S\,\hat\gamma^{(b)}$ for every b. S is an intertwiner.

S is orthogonal up to a factor. Transpose $\gamma^{(b)} S = S\hat\gamma^{(b)}$: $S^T(\gamma^{(b)})^T = (\hat\gamma^{(b)})^T S^T$. Both sets have the same symmetry pattern, $(\gamma^{(b)})^T = \eta_{bb}\gamma^{(b)}$ and $(\hat\gamma^{(b)})^T = \eta_{bb}\hat\gamma^{(b)}$; dividing by $\eta_{bb} = \pm 1$ gives $S^T\gamma^{(b)} = \hat\gamma^{(b)} S^T$. Then $(S^T S)\hat\gamma^{(b)} = S^T\gamma^{(b)} S = \hat\gamma^{(b)}(S^T S)$: the matrix $S^T S$ commutes with all eight hat gammas.

Only multiples of the identity commute with all gammas. If X commutes with every $\hat\gamma^{(b)}$, it commutes with every product $\hat\gamma_A$, hence with every combination of them; the 256 products are a basis of all real 16×16 matrices (this notebook checks their exact rank, 256), so X commutes with every matrix, in particular with E_{ij}, the matrix with a single 1 in row i and column j. The entry in row k, column j of XE_{ij} is X_{ki}, and that of $E_{ij}X$ is X_{jj} if $k = i$ and 0 otherwise. So $X_{ki} = 0$ for $k \neq i$, and $X_{ii} = X_{jj}$: $X = cI_{16}$. Applied to $S^T S$: $S^T S = c\,I_{16}$, and $c > 0$ when $S \neq 0$ (c is the sum of the squares of a column of S). Then $Q = S/\sqrt{c}$ satisfies $Q^T Q = I_{16}$ and $\gamma^{(b)} = Q\,\hat\gamma^{(b)} Q^T$.

Unique up to a factor. If X is any intertwiner, then $Q\hat\gamma^{(b)}Q^T X = X\hat\gamma^{(b)}$, so $Q^T X$ commutes with every $\hat\gamma^{(b)}$ (multiply by Q^T on the left), so $Q^T X = cI_{16}$ and $X = cQ$.

What it means for the field equation. In flat 4+4 space write $\Psi = Q\chi$. Then $\gamma^{(a)}\partial_a\Psi = Q\hat\gamma^{(a)}Q^T Q\,\partial_a\chi = Q\,\hat\gamma^{(a)}\partial_a\chi$, so $\sum_a\gamma^{(a)}\partial_a\Psi = m\Psi$ becomes $Q(\sum_a\hat\gamma^{(a)}\partial_a\chi - m\chi) = 0$, and, because Q is invertible, $\sum_a\hat\gamma^{(a)}\partial_a\chi = m\chi$: the same equation in the other basis. The choice between the two sets is a choice of how to number the 16 components, not a choice of physics.

Text cell:

```
## 5. The three $2 \times 2$ matrices

The next cell imports the packages, types in P, N and G as integer matrices, prints
each with its square, and checks every rule of the situation section: the squares,
the six products ($PN = -G$ and $NP = +G$, and so on) and the symmetry.
```

In [2]:

```
import itertools  # all subsets of a given size, all pairs of indices
from math import comb  # comb(n, k): the binomial coefficient "n choose k"

import numpy as np  # integer matrices; their sums and products are exact
import sympy as sp  # exact ranks, computed with fractions
from matplotlib.colors import BoundaryNorm, ListedColormap
from matplotlib.patches import Patch
from sympy.polys.matrices import DomainMatrix  # exact matrices for the rank

P = np.array([[0, 1], [1, 0]], dtype=np.int64)
N = np.array([[0, 1], [-1, 0]], dtype=np.int64)
G = np.array([[1, 0], [0, -1]], dtype=np.int64)
I2 = np.eye(2, dtype=np.int64)
for name, m in (("P", P), ("N", N), ("G", G)):
    say(f"{name} = {m.tolist()},  {name} {name} = {(m @ m).tolist()}")
squares = (P @ P == I2).all() and (G @ G == I2).all() and (N @ N == -I2).all()
triples = [(P, N, -G), (P, G, -N), (N, G, -P)]  # (A, B, the product A B)
# A B = C and B A = -C for each triple: the two matrices anticommute
products_ok = all((a @ b == c).all() and (b @ a == -c).all() for a, b, c in triples)
symmetry_ok = (P.T == P).all() and (G.T == G).all() and (N.T == -N).all()
check(squares and products_ok and symmetry_ok,
      "P^2 = G^2 = I2, N^2 = -I2, PN = -G, PG = -N, NG = -P, any two anticommute")
```

Out [2]:

```
P = [[0, 1], [1, 0]],  P P = [[1, 0], [0, 1]]
N = [[0, 1], [-1, 0]],  N N = [[-1, 0], [0, -1]]
G = [[1, 0], [0, -1]],  G G = [[1, 0], [0, 1]]
PASS P^2 = G^2 = I2, N^2 = -I2, PN = -G, PG = -N, NG = -P, any two anticommute
```

Text cell:

```
## 6. The Kronecker product

The next cell computes $P \otimes G$ with ‘np.kron‘ and prints it; by the index
formula it is $\begin{pmatrix} 0 \cdot G & 1 \cdot G \\ 1 \cdot G & 0 \cdot G
\end{pmatrix}$. It checks the index formula entry by entry, defines ‘kron4‘ for four
slots, and checks the mixed-product rule exactly on random whole-number matrices
(seed 12345, so every run uses the same numbers): with two slots,
$(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$, and with four slots, slot by slot.
```

In [3]:

```
example = np.kron(P, G)  # the 4 x 4 matrix P (x) G
print("P (x) G =")
for row in example:
    print("    " + " ".join(f"{x:2d}" for x in row))
# index formula with m = 2: (A (x) B)[2i + k, 2j + l] = A[i, j] B[k, l]
formula_ok = all(example[2 * i + k, 2 * j + l] == P[i, j] * G[k, l]
                 for i, j, k, l in itertools.product(range(2), repeat=4))


def kron4(m1, m2, m3, m4):
    """m1 (x) m2 (x) m3 (x) m4: a 16 x 16 matrix made of four 2 x 2 matrices."""
    return np.kron(np.kron(np.kron(m1, m2), m3), m4)


generator = np.random.default_rng(12345)  # fixed seed: the same numbers every run
a1, b1, a2, b2 = (generator.integers(-3, 4, size=(2, 2)) for _ in range(4))
two_slots = (np.kron(a1, b1) @ np.kron(a2, b2) == np.kron(a1 @ a2, b1 @ b2)).all()
first = [generator.integers(-3, 4, size=(2, 2)) for _ in range(4)]  # four slots
second = [generator.integers(-3, 4, size=(2, 2)) for _ in range(4)]
four_slots = (kron4(*first) @ kron4(*second)
              == kron4(*[f @ s for f, s in zip(first, second)])).all()
check(formula_ok and two_slots and four_slots,
      "Kronecker product: index formula and mixed-product rule (2 and 4 slots)")
```

Out [3]:

```
P (x) G =
     0  0  1  0
     0  0  0 -1
     1  0  0  0
     0 -1  0  0
PASS Kronecker product: index formula and mixed-product rule (2 and 4 slots)
```

Text cell:

```
## 7. The eight tensor-product gammas

The next cell reads the coordinate names and the signs $\eta$ from the Revision
record file ‘Revision/algebra/gammas.json‘ (and the author’s eight gammas, used from
section 9 on), and builds the eight hat gammas with the recipe. The list ‘SLOT‘
holds the slot of $x_0, \dots, x_7$: slot 1 for $x_1$ and $x_6$, slot 2 for $x_2$ and
$x_5$, slot 3 for $x_3$ and $x_4$, slot 4 for $x_0$ and $x_7$. A space-like direction
($\eta = +1$) gets P, a time-like one N. The cell prints the four factors of each
matrix.
```

In [4]:

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
```

```
COORDINATES = record["coordinates"]  # ["x0", ..., "x7"]
eta = record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1]
gamma = [np.array(m, dtype=np.int64) for m in record["gamma"]]  # the author's
I16 = np.eye(16, dtype=np.int64)
SLOT = [4, 1, 2, 3, 3, 2, 1, 4]  # slots of x0..x7: (x1,x6) (x2,x5) (x3,x4) (x0,x7)


def factor_names(slot, middle):
    """The names of the four factors: G before the slot, middle in it, I after."""
    return ["G"] * (slot - 1) + [middle] + ["I"] * (4 - slot)


MATRIX_OF = {"G": G, "P": P, "N": N, "I": I2}  # name -> 2 x 2 matrix
FACTORS = [factor_names(SLOT[a], "P" if eta[a] > 0 else "N") for a in range(8)]
hat = [kron4(*[MATRIX_OF[name] for name in FACTORS[a]]) for a in range(8)]
for a in range(8):
    kind = "space-like" if eta[a] > 0 else "time-like "
    print(f"hat gamma^({COORDINATES[a]})  {kind}  = " + " (x) ".join(FACTORS[a]))
check(COORDINATES == [f"x{k}" for k in range(8)]
      and eta == [1, 1, 1, 1, -1, -1, -1, -1],
      "the record lists x0..x7 with eta = diag(+1, +1, +1, +1, -1, -1, -1, -1)")
```

Out [4]:

```
hat gamma^(x0)  space-like  = G (x) G (x) G (x) P
hat gamma^(x1)  space-like  = P (x) I (x) I (x) I
hat gamma^(x2)  space-like  = G (x) P (x) I (x) I
hat gamma^(x3)  space-like  = G (x) G (x) P (x) I
hat gamma^(x4)  time-like   = G (x) G (x) N (x) I
hat gamma^(x5)  time-like   = G (x) N (x) I (x) I
hat gamma^(x6)  time-like   = N (x) I (x) I (x) I
hat gamma^(x7)  time-like   = G (x) G (x) G (x) N
PASS the record lists x0..x7 with eta = diag(+1, +1, +1, +1, -1, -1, -1, -1)
```

Text cell:

```
The next cell draws the recipe as a table: one row per direction, one column per
slot, each square showing the $2 \times 2$ matrix in that slot. The rows are ordered
by the pairs that share a slot, so the pattern of G (grey), P (red), N (blue) and
$I_2$ (white) forms a staircase.
```

In [5]:

```
PAIR_ORDER = [1, 6, 2, 5, 3, 4, 0, 7]  # x1, x6, x2, x5, x3, x4, x0, x7
ROLE = ["hidden", "3-space", "3-space", "3-space", "the time", "extra time",
        "extra time", "extra time"]
CODE = {"I": 0, "G": 1, "P": 2, "N": 3}  # a number per letter, for the colours
grid = np.array([[CODE[name] for name in FACTORS[a]] for a in PAIR_ORDER])
slot_colours = ListedColormap(["#ffffff", "#c3c2b7", "#e34948", "#2a78d6"])
fig, ax = plt.subplots(figsize=(7.0, 4.8), layout="constrained")
# aspect="auto": the squares become rectangles that fill the panel
```

```
ax.imshow(grid, cmap=slot_colours, norm=BoundaryNorm([-0.5, 0.5, 1.5, 2.5, 3.5], 4),
          aspect="auto")
for row, a in enumerate(PAIR_ORDER):
    for column, name in enumerate(FACTORS[a]):
        text = "$I_2$" if name == "I" else name
        colour = "white" if name in ("P", "N") else "#0b0b0b"
        ax.text(column, row, text, ha="center", va="center", fontsize=12,
                color=colour)
ax.set_xticks(range(4), [f"slot {k}" for k in range(1, 5)])
ax.set_yticks(range(8), [f"$x_{a}$ ({ROLE[a]})" for a in PAIR_ORDER])
for boundary in (1.5, 3.5, 5.5):  # lines between the four pairs
    ax.axhline(boundary, color="#0b0b0b", linewidth=1.2)
ax.grid(False)
ax.set_title("$\\hat\\gamma$ = slot 1 $\\otimes$ slot 2 $\\otimes$ slot 3 "
             "$\\otimes$ slot 4")
save_figure(fig, "slot_pattern",
            "The recipe of the tensor-product gammas: row $x_a$ shows the four "
            "$2 \\times 2$ factors of $\\hat\\gamma^{(x_a)}$, slot 1 on the left "
            "(G grey, P red, N blue, $I_2$ white). The rows are grouped in the pairs "
            "that share a slot, one space-like direction (P) and one time-like "
            "direction (N) each: ($x_1$, $x_6$), ($x_2$, $x_5$), ($x_3$, $x_4$), "
            "($x_0$, $x_7$). Every matrix has G before its own slot and $I_2$ after "
            "it, the staircase that makes all eight anticommute.")
```

Out [5]:

```
Figure 04d.1 saved as Revision/textbook/figures/04d_1_slot_pattern.png
```

Text cell:

```
The next cell draws the eight hat gammas as heat maps (blue $-1$, light grey 0,
red $+1$; rows and columns numbered 0 to 15), in the order $x_0, \dots, x_7$.
```

In [6]:

```
SIGN_COLOURS = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -1, 0, +1
SIGN_NORM = BoundaryNorm([-1.5, -0.5, 0.5, 1.5], SIGN_COLOURS.N)


def draw_signs(ax, matrix, title):
    """Draw a matrix with entries -1, 0, +1 as a heat map in the panel ax."""
    ax.imshow(matrix, cmap=SIGN_COLOURS, norm=SIGN_NORM)
    ax.set_title(title, fontsize=10)
    ax.set_xticks(range(0, 16, 4))
    ax.set_yticks(range(0, 16, 4))
    ax.grid(False)  # no grid lines on top of the coloured squares


def sign_legend(fig):
    """The key of the three colours, below the panels of the figure."""
    handles = [Patch(facecolor=SIGN_COLOURS(k), edgecolor="#898781", label=label)
```

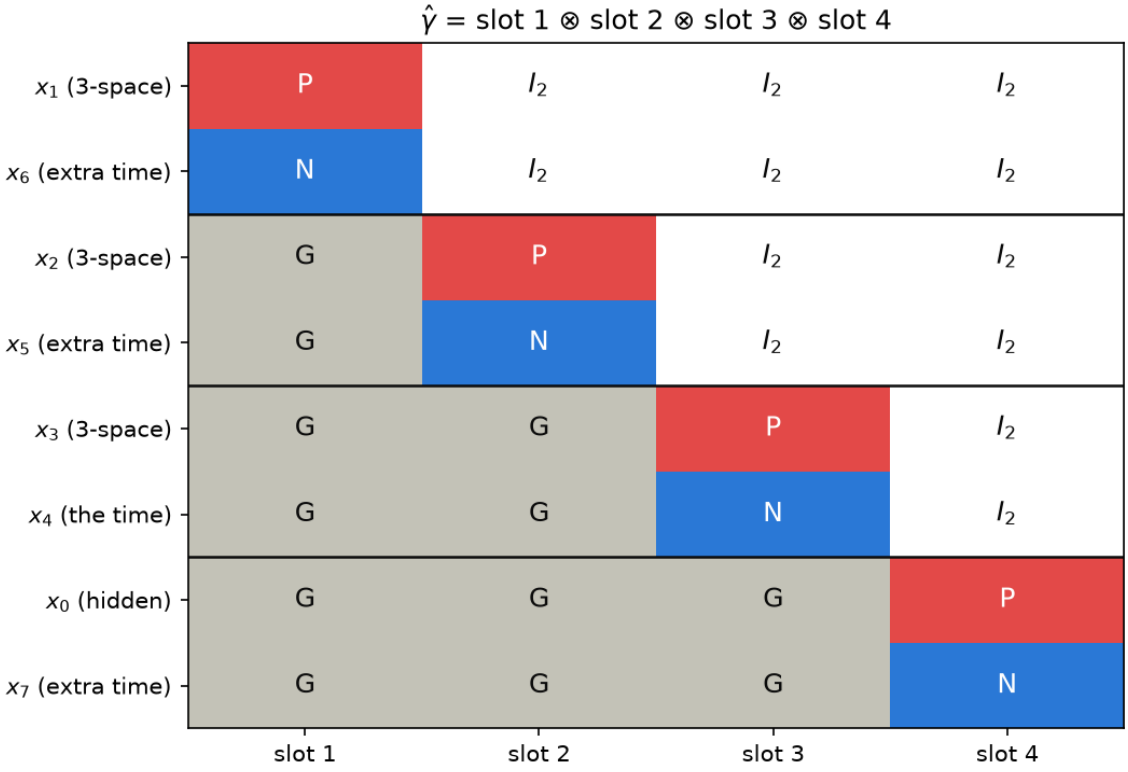

Figure 136: The recipe of the tensor-product gammas: row x_a shows the four 2×2 factors of $\hat{\gamma}^{(x_a)}$, slot 1 on the left (G grey, P red, N blue, I_2 white). The rows are grouped in the pairs that share a slot, one space-like direction (P) and one time-like direction (N) each: (x_1, x_6), (x_2, x_5), (x_3, x_4), (x_0, x_7). Every matrix has G before its own slot and I_2 after it, the staircase that makes all eight anticommute. (Notebook 04d, figure 1.)

```
            for k, label in enumerate(["entry -1", "entry 0", "entry +1"])]
    fig.legend(handles=handles, loc="outside lower center", ncol=3, fontsize=9,
               frameon=False)


fig, axes = plt.subplots(2, 4, figsize=(8.8, 5.0), layout="constrained")
for a in range(8):  # a // 4 is the row of the panel, a % 4 its column
    draw_signs(axes[a // 4, a % 4], hat[a], f"$\\hat\\gamma^{{(x_{a})}}$")
sign_legend(fig)
save_figure(fig, "tensor_gammas",
            "The eight tensor-product gammas $\\hat\\gamma^{(x_0)}$ to "
            "$\\hat\\gamma^{(x_7)}$ (rows and columns numbered 0 to 15; blue $-1$, "
            "grey 0, red $+1$). Each is a signed permutation matrix, like the "
            "author's, but the pattern is different: a factor P or N in slot $k$ "
            "exchanges components whose numbers differ in bit $k$, so the nonzero "
            "entries lie on diagonals at distance 8, 4, 2 or 1 from the main "
            "diagonal, and the factors G in front flip signs.")
```

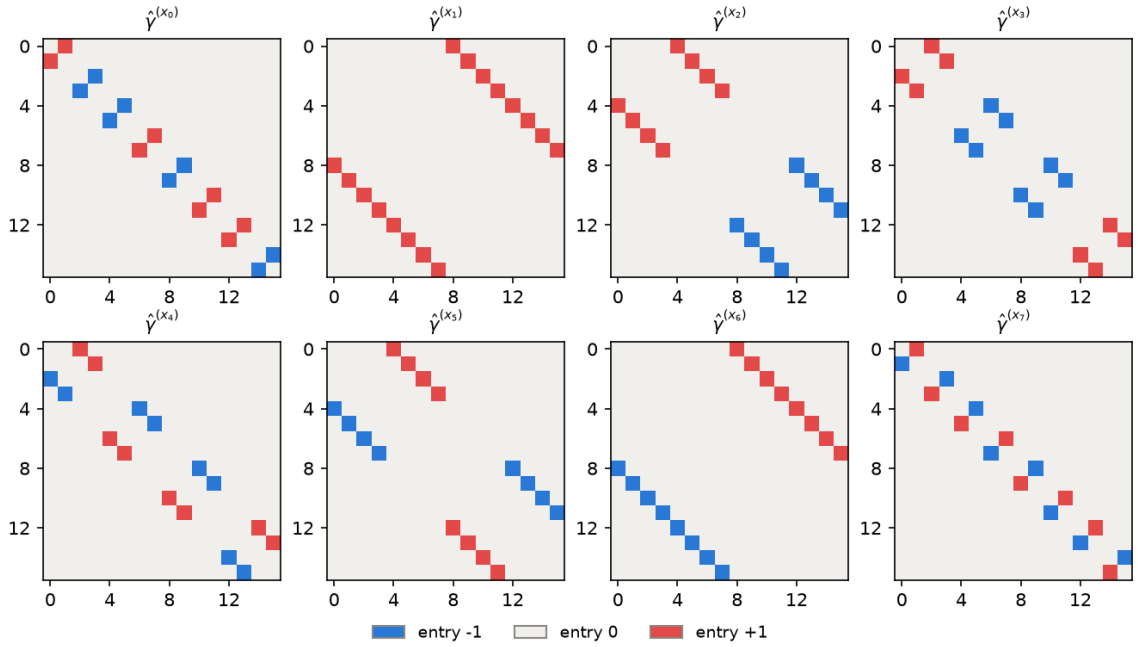

Figure 137: The eight tensor-product gammas $\hat\gamma^{(x_0)}$ to $\hat\gamma^{(x_7)}$ (rows and columns numbered 0 to 15; blue -1, grey 0, red $+1$). Each is a signed permutation matrix, like the author's, but the pattern is different: a factor P or N in slot k exchanges components whose numbers differ in bit k, so the nonzero entries lie on diagonals at distance 8, 4, 2 or 1 from the main diagonal, and the factors G in front flip signs. (Notebook 04d, figure 2.)

Out [6]:

```
Figure 04d.2 saved as Revision/textbook/figures/04d_2_tensor_gammas.png
```

Text cell:

```
## 8. The Clifford relation and the 256 products

The next cell checks exactly, for the hat gammas: the Clifford relation for all 64
ordered pairs (each anticommutator is $2\eta^{ab} I_{16}$); that every hat gamma is a
signed permutation matrix; and that it has the author's symmetry pattern
$(\hat\gamma^{(x_a)})^T = \eta_{aa}\hat\gamma^{(x_a)}$ (symmetric for
$x_0, x_1, x_2, x_3$, antisymmetric for $x_4, \dots, x_7$; P, G and $I_2$ are
symmetric and N antisymmetric, so this follows from $(A \otimes B)^T = A^T \otimes
B^T$). Then it forms the 256 ordered products of BOTH sets (sets of directions in
increasing order, as `itertools.combinations` lists them) and computes the exact
rank of the 256 hat products, each written as a row of 256 numbers.
```

In [7]:

```
clifford_ok = all(
    (hat[a] @ hat[b] + hat[b] @ hat[a] == 2 * eta[a] * (a == b) * I16).all()
    for a in range(8) for b in range(8))  # (a == b) is 1 or 0
permutation_ok = all((np.count_nonzero(m, axis=0) == 1).all()
                     and (np.count_nonzero(m, axis=1) == 1).all()
                     and (np.abs(m).sum() == 16) for m in hat)
symmetry_ok = all((hat[a].T == eta[a] * hat[a]).all() for a in range(8))
check(clifford_ok and permutation_ok and symmetry_ok,
      "hat gammas: Clifford relation (64 pairs), signed permutations, symmetry")

SETS = [s for k in range(9) for s in itertools.combinations(range(8), k)]


def product(matrices, indices):
    """The product of matrices[i] for i in indices, from left to right."""
    result = I16
    for i in indices:
        result = result @ matrices[i]
    return result


author_products = np.array([product(gamma, s) for s in SETS])  # gamma_A, 256 of them
hat_products = np.array([product(hat, s) for s in SETS])  # hat gamma_A


def exact_rank(rows):
    """The exact rank of a list of rows of whole numbers (fractions, over QQ)."""
    matrix = DomainMatrix([[sp.ZZ(int(x)) for x in row] for row in rows],
                          (len(rows), len(rows[0])), sp.ZZ)
    return matrix.convert_to(sp.QQ).rank()


hat_rank = exact_rank(hat_products.reshape(256, 256).tolist())
report("exact rank of the 256 products of the hat gammas", hat_rank)
check(hat_rank == 256, "the 256 hat products are a basis of all real 16 x 16 matrices")
```

Out [7]:

```
PASS hat gammas: Clifford relation (64 pairs), signed permutations, symmetry
RESULT exact rank of the 256 products of the hat gammas = 256
PASS the 256 hat products are a basis of all real 16 x 16 matrices
```

Text cell:

```
## 9. The chirality product in both pictures

For the hat gammas the product of all eight can be found by hand. First the product
of the two matrices of one slot $k$: slot by slot, $G \cdot G = I_2$ before $k$,
$PN = -G$ in slot $k$, and $I_2$ after $k$; so it is $-(G$ in slot $k$, $I_2$
elsewhere$)$. Second, the chirality product $\hat\Gamma$ takes the factors in the
author's order $x_0, x_1, x_2, \dots, x_7$; reordering them into the pairs
$(x_1, x_6), (x_2, x_5), (x_3, x_4), (x_0, x_7)$ exchanges neighbours that
anticommute, so it multiplies the product by the *permutation sign* of the
reordering: $+1$ if the number of *inversions* (pairs of factors whose order is
reversed) is even and $-1$ if it is odd, because each exchange of two neighbours
gives a factor $-1$ and changes the number of inversions by one.
Third, the four pair products give
$(-1)^4\, G \otimes G \otimes G \otimes G$ (each slot receives one G). Its diagonal
entry in row $j = 8b_1 + 4b_2 + 2b_3 + b_4$ is $(-1)^{b_1 + b_2 + b_3 + b_4}$,
because $G = \mathrm{diag}(+1, -1)$ contributes $-1$ for every bit equal to 1.

The next cell checks the pair products, computes the permutation sign of the
reordering, checks $\hat\Gamma = $ sign $\times\, G \otimes G \otimes G \otimes G$,
and compares with the author's chirality product
$\Gamma = \mathrm{diag}(-I_8, I_8)$, which the Revision record checked. `format(j,
"04b")` writes the number $j$ with four binary digits.
```

In [8]:

```
WOLFRAM = "Revision/algebra/reports/wolfram-algebra.json"
PYTHON = "Revision/algebra/reports/python-algebra.json"
RECORDED = {}  # report file -> {check name: the check (name, verdict, detail)}
for report_file in (WOLFRAM, PYTHON):
    text = repository_file(report_file).read_text(encoding="utf-8")
    RECORDED[report_file] = {e["name"]: e for e in json.loads(text)["checks"]}


def check_record(condition, name, *records):
    """check(condition, name) for a statement that the Revision record verified;
    records are pairs (report file, check name), each recorded as passed."""
    for report_file, check_name in records:
        entry = RECORDED[report_file].get(check_name)
        if entry is None or entry["verdict"].upper() != "PASS":
            raise AssertionError(f"{report_file} has no passed check {check_name}")
    check(condition, name)
    for report_file, check_name in records:
        print(f"     reproduces {report_file}")
        print(f"         check {check_name}")
```

```
def permutation_sign(numbers):
    """+1 for an even, -1 for an odd number of inversions (pairs i < j with
    numbers[i] > numbers[j])."""
    inversions = sum(1 for i, j in itertools.combinations(range(len(numbers)), 2)
                     if numbers[i] > numbers[j])
    return -1 if inversions % 2 else 1


PAIRS = [(1, 6), (2, 5), (3, 4), (0, 7)]  # (x1,x6) (x2,x5) (x3,x4) (x0,x7)
pair_ok = True
for slot, (s, t) in enumerate(PAIRS, start=1):
    only_g = kron4(*[G if k == slot else I2 for k in range(1, 5)])
    pair_ok &= bool((hat[s] @ hat[t] == -only_g).all())
AUTHOR_ORDER = [0, 1, 2, 3, 4, 5, 6, 7]  # x0, x1, x2, ..., x7
paired_order = [a for pair in PAIRS for a in pair]  # x1, x6, x2, x5, x3, x4, x0, x7
# where each factor of the paired order stands in the author's order
positions = [AUTHOR_ORDER.index(a) for a in paired_order]
reorder_sign = permutation_sign(positions)
say(f"positions of x1, x6, x2, x5, x3, x4, x0, x7 in the author's order: {positions}")
say(f"permutation sign of the reordering: {reorder_sign:+d}")
chirality_hat = product(hat, AUTHOR_ORDER)
gggg = kron4(G, G, G, G)
bits = [format(j, "04b") for j in range(16)]  # "0000", "0001", ..., "1111"
parity = [(-1) ** b.count("1") for b in bits]  # -1 for an odd number of 1s
check(pair_ok and (chirality_hat == reorder_sign * gggg).all()
      and (np.diag(chirality_hat) == parity).all()
      and (chirality_hat == np.diag(parity)).all(),
      "hat Gamma = (sign +1) G (x) G (x) G (x) G = diag((-1)^(b1+b2+b3+b4))")
chirality_author = product(gamma, AUTHOR_ORDER)
say("rows with -1 on the diagonal, author: "
    f"{[j for j in range(16) if chirality_author[j, j] < 0]}")
say("rows with -1 on the diagonal, hat:    "
    f"{[j for j in range(16) if chirality_hat[j, j] < 0]}")
check_record((chirality_author == np.diag([-1] * 8 + [1] * 8)).all(),
             "author: Gamma = gamma^(x0) gamma^(x1) ... gamma^(x7) = diag(-I8, I8)",
             (WOLFRAM, "Gamma_diag"), (PYTHON, "chirality_diag"))
```

Out [8]:

```
positions of x1, x6, x2, x5, x3, x4, x0, x7 in the author's order: [1, 6, 2, 5, 3, 4, 0,
    7]
permutation sign of the reordering: +1
PASS hat Gamma = (sign +1) G (x) G (x) G (x) G = diag((-1)^(b1+b2+b3+b4))
rows with -1 on the diagonal, author: [0, 1, 2, 3, 4, 5, 6, 7]
rows with -1 on the diagonal, hat:    [1, 2, 4, 7, 8, 11, 13, 14]
PASS author: Gamma = gamma^(x0) gamma^(x1) ... gamma^(x7) = diag(-I8, I8)
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check Gamma_diag
     reproduces Revision/algebra/reports/python-algebra.json
         check chirality_diag
```

Text cell:

```
The next cell draws the two diagonals: on top the author's $\Gamma$ (eight $-1$, then
eight $+1$), below the hat $\hat\Gamma$, whose sign follows the parity of the bits of
the row number (the bits are written under the axis).
```

In [9]:

```
fig, (top, bottom) = plt.subplots(2, 1, figsize=(8.0, 5.4), layout="constrained")
for ax, values, title in (
        (top, np.diag(chirality_author), "author: $\\Gamma = $ "
         "$\\gamma^{(x_0)}\\gamma^{(x_1)}\\cdots\\gamma^{(x_7)}$"),
        (bottom, np.diag(chirality_hat), "tensor picture: $\\hat\\Gamma = "
         "G \\otimes G \\otimes G \\otimes G$")):
    colours = ["#2a78d6" if v < 0 else "#e34948" for v in values]
    ax.bar(range(16), values, color=colours, width=0.7)
    ax.axhline(0.0, color="#898781", linewidth=0.8)
    ax.set_ylim(-1.4, 1.4)
    ax.set_yticks([-1, 0, 1])
    ax.set_ylabel("diagonal entry")
    ax.set_title(title, fontsize=10)
top.set_xticks(range(16))
bottom.set_xticks(range(16), [f"{j}\n{bits[j]}" for j in range(16)], fontsize=8)
bottom.set_xlabel("row number $j$ and its bits $b_1 b_2 b_3 b_4$")
save_figure(fig, "chirality_diagonals",
            "The chirality product in the two pictures; both are diagonal matrices, "
            "and the bars show their 16 diagonal entries (horizontal axis the row "
            "number $j$ from 0 to 15, blue $-1$, red $+1$). Top: the author's "
            "$\\Gamma$ has $-1$ in rows 0 to 7 and $+1$ in rows 8 to 15. Bottom: the "
            "tensor-product $\\hat\\Gamma = G \\otimes G \\otimes G \\otimes G$ has "
            "$(-1)$ to the power $b_1 + b_2 + b_3 + b_4$, $-1$ exactly in the rows "
            "1, 2, 4, 7, 8, 11, 13, 14 whose bits contain an odd number of ones. "
            "Both have eight entries of each sign; the change of basis of the next "
            "figure moves the eight rows with $-1$ to the top.")
```

Out [9]:

```
Figure 04d.3 saved as Revision/textbook/figures/04d_3_chirality_diagonals.png
```

Text cell:

```
## 10. The change of basis

The next cell computes $S = \sum_A \gamma_A M \hat\gamma_A^T$ for the simplest
choices $M = E_{ij}$. The term $\gamma_A E_{ij} \hat\gamma_A^T$ has the entry
$(\gamma_A)_{ki} (\hat\gamma_A)_{lj}$ in row $k$, column $l$ (only column $i$ of
$\gamma_A$ and column $j$ of $\hat\gamma_A$ survive), so the whole sum over the 256
sets is ONE matrix product: the $16 \times 256$ table of the columns $i$ of all
$\gamma_A$ times the $256 \times 16$ table of the columns $j$ of all $\hat\gamma_A$.
The cell confirms this shortcut against the plain sum for the first nonzero $S$,
which it finds by trying $(i, j) = (0, 0), (0, 1), \dots$ in order. Then it checks,
exactly: $\gamma^{(x_a)} S = S \hat\gamma^{(x_a)}$ for all eight directions,
$S^T S = c I_{16}$, and that $Q = S/\sqrt{c}$ is a signed permutation matrix. It
```

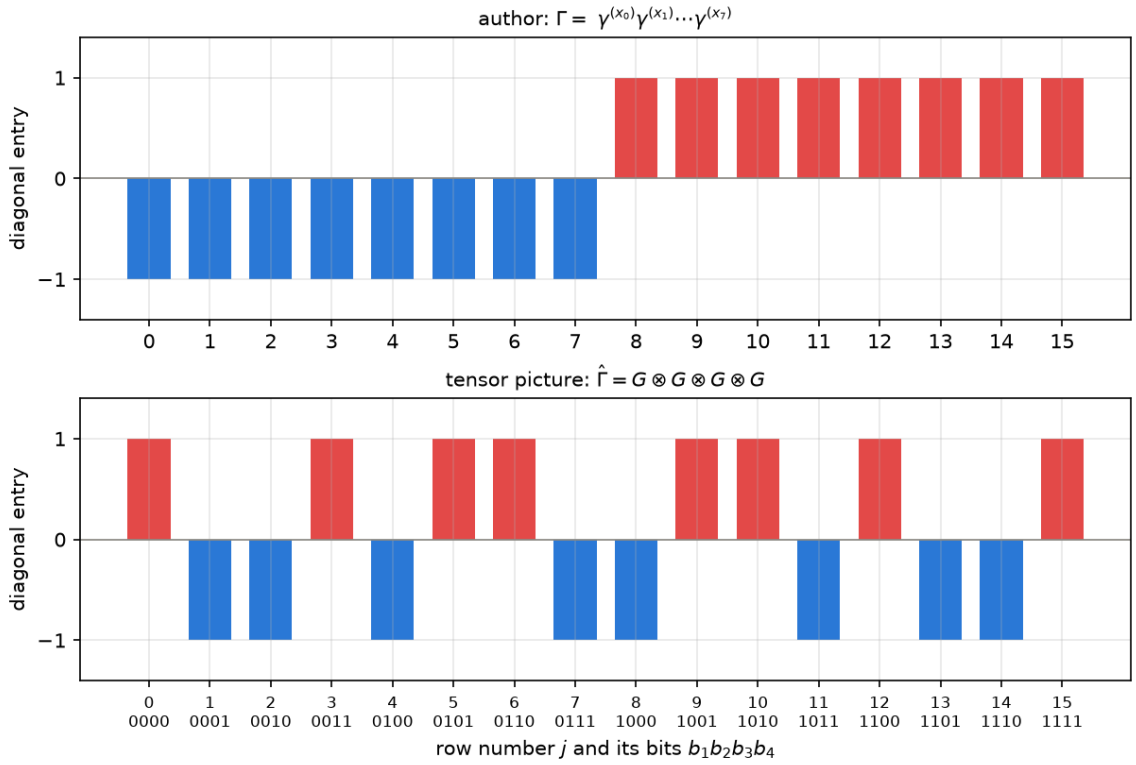

Figure 138: The chirality product in the two pictures; both are diagonal matrices, and the bars show their 16 diagonal entries (horizontal axis the row number j from 0 to 15, blue -1, red $+1$). Top: the author's Γ has -1 in rows 0 to 7 and $+1$ in rows 8 to 15. Bottom: the tensor-product $\hat{\Gamma} = G \otimes G \otimes G \otimes G$ has (-1) to the power $b_1 + b_2 + b_3 + b_4$, -1 exactly in the rows 1, 2, 4, 7, 8, 11, 13, 14 whose bits contain an odd number of ones. Both have eight entries of each sign; the change of basis of the next figure moves the eight rows with -1 to the top. (Notebook 04d, figure 3.)

```
prints $Q$ in a short code notation: for each row the column and the sign of its
single nonzero entry, so "+8" in row 0 means $(Qu)_0 = +u_8$ for every column $u$.
```

In [10]:

```
def change_of_basis(products_1, products_2, i, j):
    """S = sum_A products_1[A] E_ij products_2[A]^T as one matrix product:
    products_1[:, :, i] is the table of the columns i (256 rows of 16 numbers)."""
    return products_1[:, :, i].T @ products_2[:, :, j]


def first_nonzero(products_1, products_2):
    """The first (i, j) in reading order with a nonzero S, and that S."""
    for i, j in itertools.product(range(16), repeat=2):
        S = change_of_basis(products_1, products_2, i, j)
        if S.any():
            return i, j, S
    raise ValueError("every S is zero")


i0, j0, S = first_nonzero(author_products, hat_products)
E = np.zeros((16, 16), dtype=np.int64)
E[i0, j0] = 1  # the matrix E_ij with a single 1
plain_sum = sum(author_products[t] @ E @ hat_products[t].T for t in range(256))
report("first (i, j) with a nonzero S", (i0, j0))
report("the different entries of S", sorted(set(int(x) for x in S.flatten())))
c = int((S.T @ S)[0, 0])  # the factor c in S^T S = c I16
root = int(round(c ** 0.5))  # its square root, a whole number here
report("c in S^T S = c I16, and its square root", (c, root))
check((plain_sum == S).all()
      and all((gamma[a] @ S == S @ hat[a]).all() for a in range(8))
      and (S.T @ S == c * I16).all() and root * root == c,
      "S intertwines: gamma^a S = S hat gamma^a for all 8 a, and S^T S = c I16")
Q = S // root  # exact: every entry of S is a multiple of root
codes = []
for row in Q:
    column = int(np.flatnonzero(row)[0])  # the column of the nonzero entry
    codes.append(("+" if row[column] > 0 else "-") + str(column))
say("rows 0 to 15 of Q: " + " ".join(codes))
check((Q * root == S).all() and (np.count_nonzero(Q, axis=0) == 1).all()
      and (np.count_nonzero(Q, axis=1) == 1).all() and (Q.T @ Q == I16).all(),
      "Q = S / sqrt(c) is a signed permutation matrix (orthogonal)")
```

Out [10]:

```
RESULT first (i, j) with a nonzero S = (0, 8)
RESULT the different entries of S = [-16, 0, 16]
RESULT c in S^T S = c I16, and its square root = (256, 16)
PASS S intertwines: gamma^a S = S hat gamma^a for all 8 a, and S^T S = c I16
rows 0 to 15 of Q: +8 +4 +2 -14 -7 +11 -13 -1 -9 -5 -3 +15 -6 +10 -12 -0
PASS Q = S / sqrt(c) is a signed permutation matrix (orthogonal)
```

Text cell:

```
The next cell checks the full statement: not only the eight gammas but all 256
products and the chirality product transform with $Q$,
$\gamma_A = Q\hat\gamma_A Q^T$ and $\Gamma = Q\hat\Gamma Q^T$. It also checks a
formula that explains which $E_{ij}$ give a nonzero $S$: for every $i, j$,
$S(E_{ij}) = 16\,Q_{ij}\,Q$. (Reason: $\sum_A \hat\gamma_A Y \hat\gamma_A^T$ commutes
with every hat gamma by the argument of the situation section, so it is a multiple
of $I_{16}$; its trace is $\sum_A \mathrm{tr}\,Y = 256\,\mathrm{tr}\,Y$, so it is
$16\,\mathrm{tr}(Y) I_{16}$; with $\gamma_A = Q\hat\gamma_A Q^T$ and $Y = Q^T E_{ij}$
this gives $S(E_{ij}) = Q \cdot 16\,\mathrm{tr}(Q^T E_{ij}) = 16\,Q_{ij}\,Q$.) So $S$
is nonzero exactly for the 16 positions $(i, j)$ where $Q$ has its nonzero entries,
and the first of them is the one found above.
```

In [11]:

```
all_products_ok = all((author_products[t] == Q @ hat_products[t] @ Q.T).all()
                      for t in range(256))
check(all_products_ok and (chirality_author == Q @ chirality_hat @ Q.T).all(),
      "gamma_A = Q hat gamma_A Q^T for all 256 products, and Gamma = Q hat Gamma Q^T")
formula_ok = all((change_of_basis(author_products, hat_products, i, j)
                  == 16 * Q[i, j] * Q).all()
                 for i, j in itertools.product(range(16), repeat=2))
nonzero_positions = int(np.count_nonzero(Q))
report("positions (i, j) with a nonzero S(E_ij)", nonzero_positions)
check(formula_ok and nonzero_positions == 16,
      "S(E_ij) = 16 Q_ij Q for all 256 positions (i, j)")
```

Out [11]:

```
PASS gamma_A = Q hat gamma_A Q^T for all 256 products, and Gamma = Q hat Gamma Q^T
RESULT positions (i, j) with a nonzero S(E_ij) = 16
PASS S(E_ij) = 16 Q_ij Q for all 256 positions (i, j)
```

Text cell:

```
The next cell draws $Q$ as a heat map: row $i$ is a component of the author's basis,
column $j$ a component of the tensor basis, and the single nonzero entry of row $i$
says which tensor component, with which sign, becomes author component $i$.
```

In [12]:

```
fig, ax = plt.subplots(figsize=(6.0, 6.0), layout="constrained")
draw_signs(ax, Q, "$Q$: $\\gamma^{(x_a)} = Q\\,\\hat\\gamma^{(x_a)} Q^T$")
ax.set_xticks(range(16), [f"{j} ({bits[j]})" for j in range(16)], fontsize=6,
              rotation=90)
ax.set_yticks(range(16), [str(i) for i in range(16)], fontsize=8)
ax.set_xlabel("component $j$ of the tensor basis (its bits)")
ax.set_ylabel("component $i$ of the author's basis")
```

```
for i in range(16):
    j = int(np.flatnonzero(Q[i])[0])
    ax.text(j, i, f"{Q[i, j]:+d}", ha="center", va="center", color="white",
            fontsize=7)
sign_legend(fig)
save_figure(fig, "change_of_basis",
            "The change of basis $Q$ from the tensor-product gammas to the author's "
            "gammas, $\\gamma^{(x_a)} = Q\\hat\\gamma^{(x_a)}Q^T$ for all eight "
            "directions (rows: the author's components 0 to 15; columns: the tensor "
            "components 0 to 15 with their bits; blue $-1$, red $+1$). $Q$ is a "
            "signed permutation: every row and column holds one entry $\\pm 1$, so "
            "the author's matrices are the tensor-product matrices with the 16 "
            "components renumbered and some signs flipped. The first eight rows "
            "pick the eight tensor components with an odd number of ones in their "
            "bits, the rows where $\\hat\\Gamma = -1$.")
```

Out [12]:

```
Figure 04d.4 saved as Revision/textbook/figures/04d_4_change_of_basis.png
```

Text cell:

```
## 11. Only one change of basis

The situation section proved that every intertwiner is a multiple of $Q$. The next
cell checks it directly. The equations $\gamma^{(x_a)} X - X\hat\gamma^{(x_a)} = 0$
for the 256 entries of $X$ (read row by row: entry $(k, l)$ is unknown number
$16k + l$) form a system of $8 \times 256 = 2048$ equations; with this numbering
$LX$ becomes $(L \otimes I_{16})$ times the list of unknowns and $XR$ becomes
$(I_{16} \otimes R^T)$ times it (write out both products entry by entry). The cell
computes the exact rank of the system: 255, so the solutions form a space of
dimension $256 - 255 = 1$, the multiples of $Q$. It does the same for the
equations $\gamma^{(x_a)} X - X\gamma^{(x_a)} = 0$, the matrices that commute with
all eight of the author's gammas: dimension 1 again (the multiples of $I_{16}$), the
number that the Revision record found with two independent programs.
```

In [13]:

```
def intertwiner_system(left, right):
    """The 2048 x 256 matrix of the equations left[a] X - X right[a] = 0, a = 1..8,
    for the 256 entries of X read row by row."""
    return np.vstack([np.kron(left[a], I16) - np.kron(I16, right[a].T)
                      for a in range(8)])


system = intertwiner_system(gamma, hat)  # the author's gammas and the hat gammas
commutant = intertwiner_system(gamma, gamma)  # the author's gammas on both sides
rank_system, rank_commutant = exact_rank(system.tolist()), exact_rank(
    commutant.tolist())
report("exact ranks of the two systems (2048 equations, 256 unknowns)",
       (rank_system, rank_commutant))
```

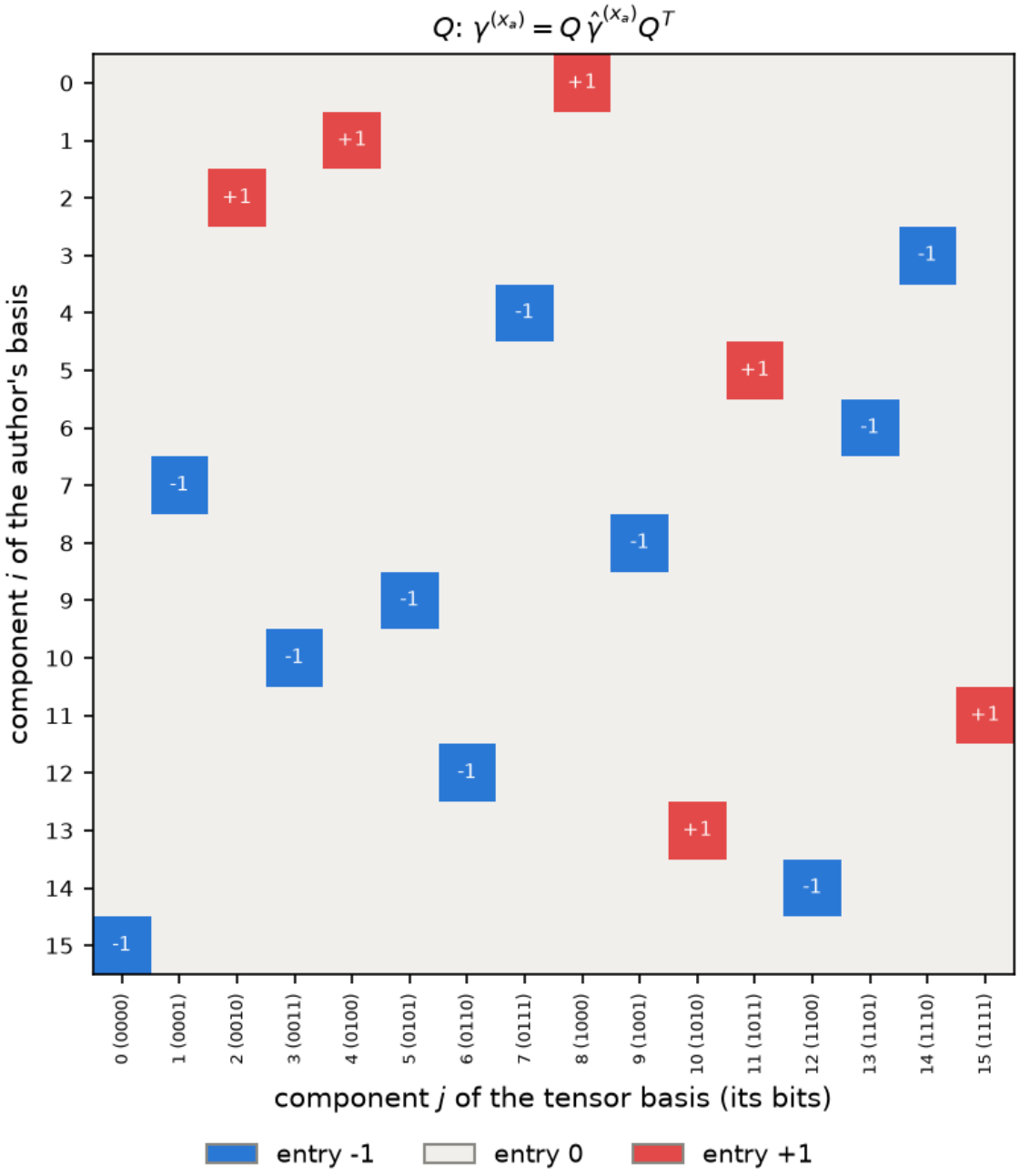

Figure 139: The change of basis Q from the tensor-product gammas to the author's gammas, $\gamma^{(x_a)} = Q\hat{\gamma}^{(x_a)}Q^T$ for all eight directions (rows: the author's components 0 to 15; columns: the tensor components 0 to 15 with their bits; blue -1, red $+1$). Q is a signed permutation: every row and column holds one entry ± 1, so the author's matrices are the tensor-product matrices with the 16 components renumbered and some signs flipped. The first eight rows pick the eight tensor components with an odd number of ones in their bits, the rows where $\hat{\Gamma} = -1$. (Notebook 04d, figure 4.)

```
q_solves = not (system @ Q.reshape(256)).any()  # Q, read row by row, solves it
check(rank_system == 255 and q_solves,
      "the intertwiners form a space of dimension 1: the multiples of Q")
python_detail = RECORDED[PYTHON]["pin_commutant_dimension_1"]["detail"]
wolfram_detail = RECORDED[WOLFRAM]["Pin44_irreducible_commutant_dim_1"]["detail"]
check_record(256 - rank_commutant == 1 and not (commutant @ I16.reshape(256)).any()
             and "= 1 (Fraction), 1 (sympy)" in python_detail
             and "has dimension 1" in wolfram_detail,
             "only the multiples of I16 commute with all eight author's gammas",
             (PYTHON, "pin_commutant_dimension_1"),
             (WOLFRAM, "Pin44_irreducible_commutant_dim_1"))
```

Out [13]:

```
RESULT exact ranks of the two systems (2048 equations, 256 unknowns) = (255, 255)
PASS the intertwiners form a space of dimension 1: the multiples of Q
PASS only the multiples of I16 commute with all eight author's gammas
     reproduces Revision/algebra/reports/python-algebra.json
         check pin_commutant_dimension_1
     reproduces Revision/algebra/reports/wolfram-algebra.json
         check Pin44_irreducible_commutant_dim_1
```

Text cell:

```
**How strongly the equations reject every other matrix.** Square the left-hand
sides and add them up: for a matrix $X$ (as a list $v$ of 256 numbers) the sum of
the squares of all 2048 equations is $v^T K v$ with the $256 \times 256$ matrix
$K = A^T A$, where $A$ is the system. $K$ has a complete set of exact eigenvectors:
the matrices $X_B = Q\hat\gamma_B$ for the 256 sets $B$. Line by line:

1. Write $L_a(X) = \gamma^{(a)} X - X\hat\gamma^{(a)}$ for the left-hand side of
   equation block $a$; then $A$ is the stack of the eight blocks $L_a$, and
   $K = \sum_a L_a^T L_a$ (a matrix product of a stack with its transpose adds up
   the products of the blocks).
2. The transpose of the block $L_a$ is the map
   $Y \to (\gamma^{(a)})^T Y - Y(\hat\gamma^{(a)})^T$ (the rows of the system become
   its columns). Both sets have $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$ and
   $(\hat\gamma^{(a)})^T = \eta_{aa}\hat\gamma^{(a)}$, so $L_a^T = \eta_{aa} L_a$
   and $K = \sum_a \eta_{aa} L_a L_a$.
3. $L_a(X_B) = Q\hat\gamma^{(a)}Q^T Q\hat\gamma_B - Q\hat\gamma_B\hat\gamma^{(a)}
   = Q(\hat\gamma^{(a)}\hat\gamma_B - \hat\gamma_B\hat\gamma^{(a)})$ (insert
   $\gamma^{(a)} = Q\hat\gamma^{(a)}Q^T$ and $Q^T Q = I_{16}$). By the rule for passing
   one gamma through a product of $k$ gammas this is 0 when $\hat\gamma^{(a)}$
   commutes with $\hat\gamma_B$, and $2Q\hat\gamma^{(a)}\hat\gamma_B$ when it
   anticommutes.
4. In the anticommuting case apply $L_a$ once more: $L_a(2Q\hat\gamma^{(a)}
   \hat\gamma_B) = 2Q(\hat\gamma^{(a)}\hat\gamma^{(a)}\hat\gamma_B -
   \hat\gamma^{(a)}\hat\gamma_B\hat\gamma^{(a)}) = 2Q(\eta_{aa}\hat\gamma_B +
   \eta_{aa}\hat\gamma_B) = 4\eta_{aa}X_B$ (the square is $\eta_{aa}$, and
   $\hat\gamma_B\hat\gamma^{(a)} = -\hat\gamma^{(a)}\hat\gamma_B$). Multiplied by
   the $\eta_{aa}$ of step 2 it gives $4X_B$, because $\eta_{aa}^2 = 1$.
5. Adding over $a$: $K v_B = 4\,n_B\,v_B$, where $n_B$ is the number of directions
```

```
   whose gamma anticommutes with $\hat\gamma_B$. These are the $k$ directions in
   $B$ for an even $k$, and the $8 - k$ directions outside $B$ for an odd $k$.

So the eigenvalues are $0, 4, 8, \dots, 32$ with the multiplicities
$\binom{8}{0}, \binom{8}{1}, \dots, \binom{8}{8}$, and the eigenvalue 0 belongs only
to $B = \{\}$, that is to $Q$. The next cell checks $K v_B = 4 n_B v_B$ exactly for
all 256 sets, counts the multiplicities, and confirms them with numpy's
floating-point eigenvalues (rounded to whole numbers).
```

In [14]:

```
K = system.T @ system  # 256 x 256, whole numbers
penalty = []  # 4 n_B for every set B
eigen_ok = True
for t, s in enumerate(SETS):
    k = len(s)
    n_B = k if k % 2 == 0 else 8 - k  # the number of anticommuting directions
    v = (Q @ hat_products[t]).reshape(256)  # X_B = Q hat gamma_B, row by row
    eigen_ok &= bool((K @ v == 4 * n_B * v).all())
    penalty.append(4 * n_B)
values = list(range(0, 33, 4))  # 0, 4, ..., 32
multiplicities = [penalty.count(v) for v in values]
numeric = np.linalg.eigvalsh(K.astype(float))  # floating-point eigenvalues
rounded = np.round(numeric).astype(int)  # nearest whole numbers
numeric_ok = np.max(np.abs(numeric - rounded)) < 1e-9 and all(
    int(np.sum(rounded == v)) == m for v, m in zip(values, multiplicities))
report("multiplicities of 0, 4, ..., 32 in K", multiplicities)
check(eigen_ok and multiplicities == [comb(8, j) for j in range(9)] and numeric_ok,
      "K = A^T A has the exact eigenvalues 4 n_B; only Q has the eigenvalue 0")
```

Out [14]:

```
RESULT multiplicities of 0, 4, ..., 32 in K = [1, 8, 28, 56, 70, 56, 28, 8, 1]
PASS K = A^T A has the exact eigenvalues 4 n_B; only Q has the eigenvalue 0
```

Text cell:

```
The next cell draws the multiplicities as bars.
```

In [15]:

```
fig, ax = plt.subplots(figsize=(7.2, 3.9), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
colours = ["#eb6834" if v == 0 else "#2a78d6" for v in values]
bars = ax.bar(values, multiplicities, width=2.8, color=colours)
ax.bar_label(bars, labels=[str(m) for m in multiplicities], padding=2, fontsize=9)
ax.set_xticks(values)
ax.set_ylim(0, 80)
ax.set_xlabel("eigenvalue of $K = A^T A$ (the penalty of a candidate $X$)")
```

```
ax.set_ylabel("multiplicity")
ax.set_title("$\\gamma^{(x_a)}X - X\\hat\\gamma^{(x_a)} = 0$: one solution, $X = Q$")
save_figure(fig, "penalty_spectrum",
            "How the 2048 equations $\\gamma^{(x_a)}X - X\\hat\\gamma^{(x_a)} = 0$ "
            "judge every $16 \\times 16$ matrix $X$: the eigenvalues of the matrix "
            "$K = A^T A$ (horizontal axis; the sum of the squared equations is the "
            "eigenvalue times the squared size of $X$ for an eigenvector $X$) and "
            "how many independent eigenvectors belong to each (vertical axis). The "
            "eigenvectors are the matrices $Q\\hat\\gamma_B$ for the 256 sets $B$ of "
            "directions, with eigenvalue 4 times the number of directions whose "
            "gamma anticommutes with $\\hat\\gamma_B$; the multiplicities are the "
            "binomial coefficients 1, 8, 28, 56, 70, 56, 28, 8, 1. Only one matrix, "
            "$Q$ itself (orange), has the eigenvalue 0: the change of basis is "
            "unique up to a factor.")
```

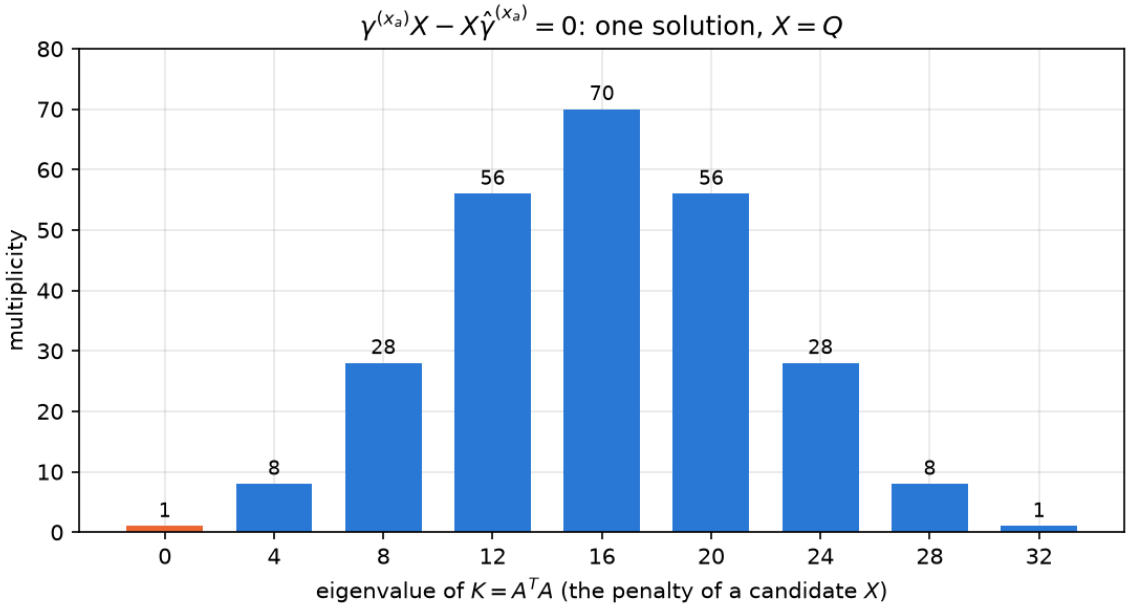

Figure 140: How the 2048 equations $\gamma^{(x_a)}X - X\hat{\gamma}^{(x_a)} = 0$ judge every 16×16 matrix X: the eigenvalues of the matrix $K = A^TA$ (horizontal axis; the sum of the squared equations is the eigenvalue times the squared size of X for an eigenvector X) and how many independent eigenvectors belong to each (vertical axis). The eigenvectors are the matrices $Q\hat{\gamma}_B$ for the 256 sets B of directions, with eigenvalue 4 times the number of directions whose gamma anticommutes with $\hat{\gamma}_B$; the multiplicities are the binomial coefficients 1, 8, 28, 56, 70, 56, 28, 8, 1. Only one matrix, Q itself (orange), has the eigenvalue 0: the change of basis is unique up to a factor. (Notebook 04d, figure 5.)

Out [15]:

```
Figure 04d.5 saved as Revision/textbook/figures/04d_5_penalty_spectrum.png
```

Text cell:

```
## 12. Another way to fill the slots

Any assignment of the eight directions to the slots works for the Clifford
relation, as long as every slot gets one P and one N. Take the simpler-looking one:
$x_1, x_2, x_3$ in slots 1, 2, 3 with P, $x_4, x_5, x_6$ in slots 1, 2, 3 with N,
and $x_0$ (P) and $x_7$ (N) in slot 4. The pairs are now (x1, x4), (x2, x5),
(x3, x6) and (x0, x7). By the theorem a change of basis must exist again. Is it a
signed permutation?

A signed permutation $Q$ turns a diagonal matrix $D$ into a diagonal matrix
$QDQ^T$ (it only renumbers the diagonal entries and multiplies each by
$(\pm 1)^2 = 1$). So if $\gamma_A = Q\hat\gamma_A Q^T$ with a signed permutation $Q$,
then $\gamma_A$ is diagonal exactly when $\hat\gamma_A$ is: the two sets must have
their diagonal products for the SAME sets $A$. The next cell lists these sets. For
the author's gammas they are the 16 sets made of whole pairs (x1, x6), (x2, x5),
(x3, x4), (x0, x7) (the products of the two matrices of one pair are diagonal, as
section 9 showed for the hat gammas); the first assignment has the same pairs, the
second has different ones. The cell then builds the second set, checks its Clifford
relation, and computes its change of basis $S_2$ with the same formula: $S_2^T S_2 =
128\,I_{16}$, and every row of $S_2$ has two nonzero entries $\pm 8$, so
$Q_2 = S_2/\sqrt{128}$ has the entries $\pm 8/\sqrt{128} = \pm 1/\sqrt2$: an
orthogonal matrix that mixes the components in pairs, not a signed permutation.
```

In [16]:

```
SLOT_2 = [4, 1, 2, 3, 1, 2, 3, 4]  # pairs (x1,x4) (x2,x5) (x3,x6) (x0,x7)
FACTORS_2 = [factor_names(SLOT_2[a], "P" if eta[a] > 0 else "N") for a in range(8)]
hat_2 = [kron4(*[MATRIX_OF[name] for name in FACTORS_2[a]]) for a in range(8)]
clifford_2 = all(
    (hat_2[a] @ hat_2[b] + hat_2[b] @ hat_2[a] == 2 * eta[a] * (a == b) * I16).all()
    for a in range(8) for b in range(8))
check(clifford_2 and all((hat_2[a].T == eta[a] * hat_2[a]).all() for a in range(8)),
      "second assignment: the Clifford relation and the symmetry pattern hold too")
hat_2_products = np.array([product(hat_2, s) for s in SETS])


def diagonal_sets(products):
    """The sets A (as coordinate names) whose product is a diagonal matrix."""
    return [tuple(COORDINATES[a] for a in s) for s, m in zip(SETS, products)
            if (m == np.diag(np.diag(m))).all()]


diagonal_author = diagonal_sets(author_products)
say("diagonal products, author:            " + ", ".join(
    "".join(n[1] for n in s) or "I" for s in diagonal_author[:5]) + ", ...")
say("diagonal products, second assignment: " + ", ".join(
    "".join(n[1] for n in s) or "I" for s in diagonal_sets(hat_2_products)[:5])
    + ", ...")
check(len(diagonal_author) == 16 and diagonal_sets(hat_products) == diagonal_author
      and diagonal_sets(hat_2_products) != diagonal_author,
      "the first assignment has the author's 16 diagonal products, the second not")
i2, j2, S_2 = first_nonzero(author_products, hat_2_products)
c_2 = int((S_2.T @ S_2)[0, 0])
```

```
report("second assignment: entries of S_2 and c_2",
       (sorted(set(int(x) for x in S_2.flatten())), c_2))
check(all((gamma[a] @ S_2 == S_2 @ hat_2[a]).all() for a in range(8))
      and (S_2.T @ S_2 == c_2 * I16).all() and c_2 == 128
      and (np.count_nonzero(S_2, axis=1) == 2).all(),
      "second assignment: S_2 intertwines, S_2^T S_2 = 128 I16, two entries per row")
Q_2 = S_2 / np.sqrt(c_2)  # floating point: the entries are +-1/sqrt(2)
```

Out [16]:

```
PASS second assignment: the Clifford relation and the symmetry pattern hold too
diagonal products, author:            I, 07, 16, 25, 34, ...
diagonal products, second assignment: I, 07, 14, 25, 36, ...
PASS the first assignment has the author's 16 diagonal products, the second not
RESULT second assignment: entries of S_2 and c_2 = ([-8, 0, 8], 128)
PASS second assignment: S_2 intertwines, S_2^T S_2 = 128 I16, two entries per row
```

Text cell:

```
The next cell draws the two changes of basis side by side with one colour scale
(blue negative, red positive, white zero); the entries of $Q_2$ are
$\pm 1/\sqrt2 \approx \pm 0.71$, a lighter colour than the $\pm 1$ of $Q$.
```

In [17]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.8, 4.6), layout="constrained")
for ax, matrix, title in (
        (left, Q, "pairs (x1,x6) (x2,x5) (x3,x4) (x0,x7): $Q$"),
        (right, Q_2, "pairs (x1,x4) (x2,x5) (x3,x6) (x0,x7): $Q_2$")):
    image = ax.imshow(matrix, cmap="RdBu_r", vmin=-1.0, vmax=1.0)
    ax.set_title(title, fontsize=9)
    ax.set_xticks(range(0, 16, 4))
    ax.set_yticks(range(0, 16, 4))
    ax.set_xlabel("tensor component $j$")
    ax.set_ylabel("author's component $i$")
    ax.grid(False)
fig.colorbar(image, ax=[left, right], shrink=0.8, label="entry")
save_figure(fig, "two_assignments",
            "The change of basis to the author's gammas for two ways of filling "
            "the slots of the tensor-product recipe (rows: the author's components, "
            "columns: the tensor components, both 0 to 15; colour scale from $-1$ "
            "blue to $+1$ red). Left: with the pairs of the author's construction "
            "the change of basis $Q$ is a signed permutation, entries $\\pm 1$. "
            "Right: with other pairs it is $Q_2$, with two entries $\\pm 1/\\sqrt{2}$ "
            "in every row and column, which mixes the components two by two. Both "
            "are orthogonal, both turn one set of gammas into the other exactly, "
            "and each is unique up to a factor.")
```

Out [17]:

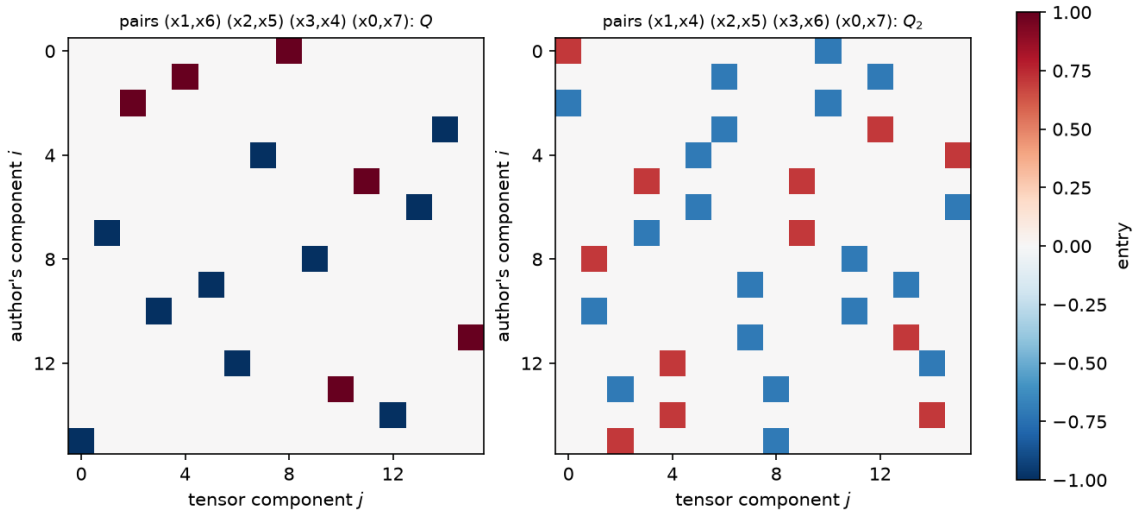

Figure 141: The change of basis to the author's gammas for two ways of filling the slots of the tensor-product recipe (rows: the author's components, columns: the tensor components, both 0 to 15; colour scale from -1 blue to $+1$ red). Left: with the pairs of the author's construction the change of basis Q is a signed permutation, entries ± 1. Right: with other pairs it is Q_2, with two entries $\pm 1/\sqrt{2}$ in every row and column, which mixes the components two by two. Both are orthogonal, both turn one set of gammas into the other exactly, and each is unique up to a factor. (Notebook 04d, figure 6.)

```
Figure 04d.6 saved as Revision/textbook/figures/04d_6_two_assignments.png
```

Text cell:

```
## 13. The last check

The last cell checks that the six figure files exist in the folder
‘Revision/textbook/figures‘ and prints the number of checks that passed.
```

In [18]:

```
names = ["04d_1_slot_pattern.png", "04d_2_tensor_gammas.png",
         "04d_3_chirality_diagonals.png", "04d_4_change_of_basis.png",
         "04d_5_penalty_spectrum.png", "04d_6_two_assignments.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all six figure files exist")
all_checks_passed()
```

Out [18]:

```
PASS all six figure files exist
ALL 18 CHECKS PASSED (notebook 04d)
```

Text cell:

```
## 14. What this notebook showed

- PROVED (exact integer arithmetic): Kronecker products of the three $2 \times 2$
  matrices P, N and G give eight real $16 \times 16$ signed permutation matrices
  $\hat\gamma^{(x_a)}$ that satisfy the Clifford relation of the author's 4+4
  space-time, $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$, with the
  author's symmetry pattern; their 256 products are a basis of all real
  $16 \times 16$ matrices (exact rank 256).
- PROVED: the chirality product of the tensor set is
  $G \otimes G \otimes G \otimes G$, diagonal with the sign $(-1)^{b_1 + b_2 + b_3 +
  b_4}$; the author's is $\mathrm{diag}(-I_8, I_8)$ (reproduces
  `Revision/algebra/reports/wolfram-algebra.json`, check `Gamma_diag`, and
  `python-algebra.json`, check `chirality_diag`).
- PROVED: the formula $S = \sum_A \gamma_A M \hat\gamma_A^T$ produces a change of
  basis; with the pairs (x1, x6), (x2, x5), (x3, x4), (x0, x7) it is $16\,Q$ with a
  signed permutation matrix $Q$, and $\gamma_A = Q\hat\gamma_A Q^T$ for all 256
  products. The author's T16 are the tensor-product gammas with the 16 components
  renumbered and some signs flipped.
- PROVED: the change of basis is unique up to a factor (the 2048 equations have
  rank 255; the penalty matrix $K$ has the eigenvalue 0 only for $Q$), and only the
  multiples of $I_{16}$ commute with all eight of the author's gammas (reproduces
  the checks `pin_commutant_dimension_1` and `Pin44_irreducible_commutant_dim_1`).
- PROVED: with other pairs in the slots the Clifford relation still holds and a
  change of basis still exists, but it mixes the components in pairs (entries
  $\pm 1/\sqrt2$), because the diagonal products then belong to different sets of
  directions.
- ASSUMED: nothing beyond the Clifford relation. These are statements of algebra:
  every equation written with the author's gammas is the same equation written with
  the tensor-product gammas after the renumbering $\Psi = Q\chi$; the choice of
  the set is a choice of basis, not of physics. The inflation of 3-space and the
  deflation of the extra times enter the field equation through the frame factors,
  the same for both sets.
```

4.20 Line-by-line walk-through of Notebook 04d

The notebook has 18 code cells, In [1] to In [18]. As in Section 4.8, every line is quoted except two things: the set-up cell In [1], which is the set-up cell of Notebook 04a explained line by line in Section 4.8 and differs from it only as said below, and the long caption texts handed to `save_figure`, which are shortened to a line `...)` (the dots stand for the caption, the bracket closes the call); each caption is printed in full under its figure in Section 4.19.

In [1], the set-up cell. It is the set-up cell of Notebook 04a, explained line by line in Section 4.8, except for the line `NOTEBOOK_ID = "04d"` and the run instructions of Section 4.18 in its comment lines. It prints one line.

In [2], the three 2×2 matrices.

```
import itertools  # all subsets of a given size, all pairs of indices
from math import comb  # comb(n, k): the binomial coefficient "n choose k"

import numpy as np  # integer matrices; their sums and products are exact
import sympy as sp  # exact ranks, computed with fractions
```

```
from matplotlib.colors import BoundaryNorm, ListedColormap
from matplotlib.patches import Patch
from sympy.polys.matrices import DomainMatrix  # exact matrices for the rank
```

The imports, as in Notebooks 04a and 04c.

```
P = np.array([[0, 1], [1, 0]], dtype=np.int64)
N = np.array([[0, 1], [-1, 0]], dtype=np.int64)
G = np.array([[1, 0], [0, -1]], dtype=np.int64)
I2 = np.eye(2, dtype=np.int64)
for name, m in (("P", P), ("N", N), ("G", G)):
    say(f"{name} = {m.tolist()},  {name} {name} = {(m @ m).tolist()}")
```

P, N, G and I_2 as whole-number arrays; the loop prints each matrix and its square: $PP = GG = I_2$ and $NN = -I_2$.

```
squares = (P @ P == I2).all() and (G @ G == I2).all() and (N @ N == -I2).all()
triples = [(P, N, -G), (P, G, -N), (N, G, -P)]  # (A, B, the product A B)
# A B = C and B A = -C for each triple: the two matrices anticommute
products_ok = all((a @ b == c).all() and (b @ a == -c).all() for a, b, c in triples)
symmetry_ok = (P.T == P).all() and (G.T == G).all() and (N.T == -N).all()
check(squares and products_ok and symmetry_ok,
      "P^2 = G^2 = I2, N^2 = -I2, PN = -G, PG = -N, NG = -P, any two anticommute")
```

`triples` lists three triples (A, B, C) with $AB = C$; the check requires $AB = C$ and $BA = -C$ for each (so the pairs anticommute), the squares, and the symmetry of P and G and antisymmetry of N: every rule of Section 4.17.

In [3], the Kronecker product.

```
example = np.kron(P, G)  # the 4 x 4 matrix P (x) G
print("P (x) G =")
for row in example:
    print("    " + " ".join(f"{x:2d}" for x in row))
```

`np.kron(P, G)` computes $P \otimes G$ (written `P (x) G` in printed text), and the loop prints its four rows: the matrix of Section 4.17.

```
# index formula with m = 2: (A (x) B)[2i + k, 2j + l] = A[i, j] B[k, l]
formula_ok = all(example[2 * i + k, 2 * j + l] == P[i, j] * G[k, l]
                 for i, j, k, l in itertools.product(range(2), repeat=4))
```

The index formula is checked for all 16 combinations of i, j, k, l in $\{0, 1\}$.

```
def kron4(m1, m2, m3, m4):
    """m1 (x) m2 (x) m3 (x) m4: a 16 x 16 matrix made of four 2 x 2 matrices."""
    return np.kron(np.kron(np.kron(m1, m2), m3), m4)
```

`kron4` forms $m_1 \otimes m_2 \otimes m_3 \otimes m_4$, a 16×16 matrix, by three Kronecker products in a row.

```
generator = np.random.default_rng(12345)  # fixed seed: the same numbers every run
a1, b1, a2, b2 = (generator.integers(-3, 4, size=(2, 2)) for _ in range(4))
two_slots = (np.kron(a1, b1) @ np.kron(a2, b2) == np.kron(a1 @ a2, b1 @ b2)).all()
first = [generator.integers(-3, 4, size=(2, 2)) for _ in range(4)]  # four slots
second = [generator.integers(-3, 4, size=(2, 2)) for _ in range(4)]
four_slots = (kron4(*first) @ kron4(*second)
              == kron4(*[f @ s for f, s in zip(first, second)])).all()
check(formula_ok and two_slots and four_slots,
      "Kronecker product: index formula and mixed-product rule (2 and 4 slots)")
```

Random 2×2 matrices of whole numbers from -3 to 3 (fixed seed) test the mixed-product rule: with two slots, $(a_1 \otimes b_1)(a_2 \otimes b_2) = (a_1 a_2) \otimes (b_1 b_2)$; with four slots, slot by slot (`*first` hands the four matrices of the list to `kron4` as four arguments; `zip` pairs the slots of the two lists; the name `_` marks a loop variable that is not used).

In [4], the eight tensor-product gammas.

```
record = json.loads(repository_file("Revision/algebra/gammas.json")
                    .read_text(encoding="utf-8"))
COORDINATES = record["coordinates"]  # ["x0", ..., "x7"]
eta = record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1]
gamma = [np.array(m, dtype=np.int64) for m in record["gamma"]]  # the author's
I16 = np.eye(16, dtype=np.int64)
SLOT = [4, 1, 2, 3, 3, 2, 1, 4]  # slots of x0..x7: (x1,x6) (x2,x5) (x3,x4) (x0,x7)
```

The coordinate names, the signs and the author's gammas are read from `gammas.json`. `SLOT` gives the slot of each direction $x_0, \ldots, x_7$: x_1 and x_6 in slot 1, x_2 and x_5 in slot 2, x_3 and x_4 in slot 3, x_0 and x_7 in slot 4; so the list is $4, 1, 2, 3, 3, 2, 1, 4$.

```
def factor_names(slot, middle):
    """The names of the four factors: G before the slot, middle in it, I after."""
    return ["G"] * (slot - 1) + [middle] + ["I"] * (4 - slot)


MATRIX_OF = {"G": G, "P": P, "N": N, "I": I2}  # name -> 2 x 2 matrix
FACTORS = [factor_names(SLOT[a], "P" if eta[a] > 0 else "N") for a in range(8)]
hat = [kron4(*[MATRIX_OF[name] for name in FACTORS[a]]) for a in range(8)]
```

`factor_names` writes the recipe as a list of four letters: `"G"` repeated for the slots before the own slot, the letter `middle` in it, and `"I"` after it. `FACTORS` gives every direction its four letters, with `P` for a space-like and `N` for a time-like direction; `hat` turns the letters into matrices with `MATRIX_OF` and forms their Kronecker product: the eight hat gammas, in the order $x_0, \ldots, x_7$.

```
for a in range(8):
    kind = "space-like" if eta[a] > 0 else "time-like "
    print(f"hat gamma^({COORDINATES[a]})  {kind}  = " + " (x) ".join(FACTORS[a]))
check(COORDINATES == [f"x{k}" for k in range(8)]
      and eta == [1, 1, 1, 1, -1, -1, -1, -1],
      "the record lists x0..x7 with eta = diag(+1, +1, +1, +1, -1, -1, -1, -1)")
```

One printed line per direction with its four factors (the table of Section 4.17). The check confirms the coordinates and signs read from the record.

In [5], figure 1.

```
PAIR_ORDER = [1, 6, 2, 5, 3, 4, 0, 7]  # x1, x6, x2, x5, x3, x4, x0, x7
ROLE = ["hidden", "3-space", "3-space", "3-space", "the time", "extra time",
        "extra time", "extra time"]
CODE = {"I": 0, "G": 1, "P": 2, "N": 3}  # a number per letter, for the colours
grid = np.array([[CODE[name] for name in FACTORS[a]] for a in PAIR_ORDER])
slot_colours = ListedColormap(["#ffffff", "#c3c2b7", "#e34948", "#2a78d6"])
```

The rows of the figure are ordered by the pairs that share a slot. Every letter gets a number (`CODE`), and `grid` is the 8×4 table of these numbers; the four colours are white for I_2, grey for G, red for P and blue for N.

```
fig, ax = plt.subplots(figsize=(7.0, 4.8), layout="constrained")
# aspect="auto": the squares become rectangles that fill the panel
ax.imshow(grid, cmap=slot_colours, norm=BoundaryNorm([-0.5, 0.5, 1.5, 2.5, 3.5], 4),
          aspect="auto")
for row, a in enumerate(PAIR_ORDER):
    for column, name in enumerate(FACTORS[a]):
        text = "$I_2$" if name == "I" else name
        colour = "white" if name in ("P", "N") else "#0b0b0b"
        ax.text(column, row, text, ha="center", va="center", fontsize=12,
                color=colour)
```

The table is painted, and every cell gets its letter (I_2 typeset as mathematics), white on the dark colours and black on the light ones.

```
ax.set_xticks(range(4), [f"slot {k}" for k in range(1, 5)])
ax.set_yticks(range(8), [f"$x_{a}$ ({ROLE[a]})" for a in PAIR_ORDER])
for boundary in (1.5, 3.5, 5.5):  # lines between the four pairs
    ax.axhline(boundary, color="#0b0b0b", linewidth=1.2)
ax.grid(False)
ax.set_title("$\\hat\\gamma$ = slot 1 $\\otimes$ slot 2 $\\otimes$ slot 3 "
             "$\\otimes$ slot 4")
save_figure(fig, "slot_pattern",
            ...)
```

Columns and rows are labelled, black lines separate the four pairs, and the title states the recipe. **What figure 1 shows and why:** a staircase: each pair has its own slot with a red P and a blue N, grey G before it and white I_2 after it; this staircase is what makes all eight anticommute (Section 4.17).

In [6], figure 2.

```
SIGN_COLOURS = ListedColormap(["#2a78d6", "#f0efec", "#e34948"])  # -1, 0, +1
SIGN_NORM = BoundaryNorm([-1.5, -0.5, 0.5, 1.5], SIGN_COLOURS.N)
```

```
def draw_signs(ax, matrix, title):
    """Draw a matrix with entries -1, 0, +1 as a heat map in the panel ax."""
    ax.imshow(matrix, cmap=SIGN_COLOURS, norm=SIGN_NORM)
    ax.set_title(title, fontsize=10)
    ax.set_xticks(range(0, 16, 4))
    ax.set_yticks(range(0, 16, 4))
    ax.grid(False)  # no grid lines on top of the coloured squares


def sign_legend(fig):
    """The key of the three colours, below the panels of the figure."""
    handles = [Patch(facecolor=SIGN_COLOURS(k), edgecolor="#898781", label=label)
               for k, label in enumerate(["entry -1", "entry 0", "entry +1"])]
    fig.legend(handles=handles, loc="outside lower center", ncol=3, fontsize=9,
               frameon=False)
```

The three-colour heat map and its key, as in Notebook 04a (without the numbers in the squares).

```
fig, axes = plt.subplots(2, 4, figsize=(8.8, 5.0), layout="constrained")
for a in range(8):  # a // 4 is the row of the panel, a % 4 its column
    draw_signs(axes[a // 4, a % 4], hat[a], f"$\\hat\\gamma^{{(x_{a})}}$")
sign_legend(fig)
save_figure(fig, "tensor_gammas",
            ...)
```

The eight hat gammas in two rows of four panels. **What figure 2 shows and why:** each is a signed permutation matrix, but the pattern differs from the author's: a P or N in slot k exchanges components whose numbers differ in bit k, so the coloured squares lie on diagonals at the distance 8 (slot 1), 4 (slot 2), 2 (slot 3) or 1 (slot 4) from the main diagonal, and the factors G in front flip signs ($\hat\gamma^{(x_1)}$ and $\hat\gamma^{(x_6)}$, with nothing before slot 1, show two clean diagonals at distance 8).

In [7], the Clifford relation and the 256 products.

```
clifford_ok = all(
    (hat[a] @ hat[b] + hat[b] @ hat[a] == 2 * eta[a] * (a == b) * I16).all()
    for a in range(8) for b in range(8))  # (a == b) is 1 or 0
permutation_ok = all((np.count_nonzero(m, axis=0) == 1).all()
                     and (np.count_nonzero(m, axis=1) == 1).all()
                     and (np.abs(m).sum() == 16) for m in hat)
symmetry_ok = all((hat[a].T == eta[a] * hat[a]).all() for a in range(8))
check(clifford_ok and permutation_ok and symmetry_ok,
      "hat gammas: Clifford relation (64 pairs), signed permutations, symmetry")
```

The Clifford relation for all 64 ordered pairs (in arithmetic, `(a == b)` counts as 1 or 0, so `2 * eta[a] * (a == b)` is $2\eta^{ab}$); signed permutations (one nonzero per column and per row, and the absolute values add up to 16, so every nonzero is ±1); and the symmetry pattern $(\hat\gamma^{(x_a)})^T = \eta_{aa}\hat\gamma^{(x_a)}$.

```
SETS = [s for k in range(9) for s in itertools.combinations(range(8), k)]
```

```
def product(matrices, indices):
    """The product of matrices[i] for i in indices, from left to right."""
    result = I16
    for i in indices:
        result = result @ matrices[i]
    return result


author_products = np.array([product(gamma, s) for s in SETS])  # gamma_A, 256 of them
hat_products = np.array([product(hat, s) for s in SETS])  # hat gamma_A
```

The 256 sets as in Notebook 04c; `product` multiplies the listed matrices from left to right; the 256 products of both sets are stored, each as an array of shape (256, 16, 16).

```
def exact_rank(rows):
    """The exact rank of a list of rows of whole numbers (fractions, over QQ)."""
    matrix = DomainMatrix([[sp.ZZ(int(x)) for x in row] for row in rows],
                          (len(rows), len(rows[0])), sp.ZZ)
    return matrix.convert_to(sp.QQ).rank()


hat_rank = exact_rank(hat_products.reshape(256, 256).tolist())
report("exact rank of the 256 products of the hat gammas", hat_rank)
check(hat_rank == 256, "the 256 hat products are a basis of all real 16 x 16 matrices")
```

The exact rank of Notebook 04c, applied to the 256 hat products written as rows: 256, so they too are a basis of all real 16×16 matrices.

In [8], the products of all eight gammas.

```
WOLFRAM = "Revision/algebra/reports/wolfram-algebra.json"
PYTHON = "Revision/algebra/reports/python-algebra.json"
RECORDED = {}  # report file -> {check name: the check (name, verdict, detail)}
for report_file in (WOLFRAM, PYTHON):
    text = repository_file(report_file).read_text(encoding="utf-8")
    RECORDED[report_file] = {e["name"]: e for e in json.loads(text)["checks"]}


def check_record(condition, name, *records):
    """check(condition, name) for a statement that the Revision record verified;
    records are pairs (report file, check name), each recorded as passed."""
    for report_file, check_name in records:
        entry = RECORDED[report_file].get(check_name)
        if entry is None or entry["verdict"].upper() != "PASS":
            raise AssertionError(f"{report_file} has no passed check {check_name}")
    check(condition, name)
    for report_file, check_name in records:
        print(f"     reproduces {report_file}")
        print(f"         check {check_name}")
```

The two algebra reports and the function `check_record`, as in Notebook 04a (without the set of reproduced checks).

```
def permutation_sign(numbers):
    """+1 for an even, -1 for an odd number of inversions (pairs i < j with
    numbers[i] > numbers[j])."""
    inversions = sum(1 for i, j in itertools.combinations(range(len(numbers)), 2)
                     if numbers[i] > numbers[j])
    return -1 if inversions % 2 else 1
```

The permutation sign of Section 4.2, for lists without repetitions.

```
PAIRS = [(1, 6), (2, 5), (3, 4), (0, 7)]  # (x1,x6) (x2,x5) (x3,x4) (x0,x7)
pair_ok = True
for slot, (s, t) in enumerate(PAIRS, start=1):
    only_g = kron4(*[G if k == slot else I2 for k in range(1, 5)])
    pair_ok &= bool((hat[s] @ hat[t] == -only_g).all())
```

For each slot (`enumerate(..., start=1)` numbers the pairs from 1), the product of its two hat gammas must be minus the matrix with G in that slot and I_2 elsewhere.

```
AUTHOR_ORDER = [0, 1, 2, 3, 4, 5, 6, 7]  # x0, x1, x2, ..., x7
paired_order = [a for pair in PAIRS for a in pair]  # x1, x6, x2, x5, x3, x4, x0, x7
# where each factor of the paired order stands in the author's order
positions = [AUTHOR_ORDER.index(a) for a in paired_order]
reorder_sign = permutation_sign(positions)
say(f"positions of x1, x6, x2, x5, x3, x4, x0, x7 in the author's order: {positions}")
say(f"permutation sign of the reordering: {reorder_sign:+d}")
```

`AUTHOR_ORDER` is the order of the factors in the author's product $\gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$; `paired_order` lists the directions pair by pair; `.index(a)` finds where each stands in the author's order. The printed list of positions is $[1, 6, 2, 5, 3, 4, 0, 7]$; it has 12 inversions (1 before 0; 6 before $2, 5, 3, 4, 0$; 2 before 0; 5 before $3, 4, 0$; 3 before 0; 4 before 0), an even number, so the sign is $+1$.

```
chirality_hat = product(hat, AUTHOR_ORDER)
gggg = kron4(G, G, G, G)
bits = [format(j, "04b") for j in range(16)]  # "0000", "0001", ..., "1111"
parity = [(-1) ** b.count("1") for b in bits]  # -1 for an odd number of 1s
check(pair_ok and (chirality_hat == reorder_sign * gggg).all()
      and (np.diag(chirality_hat) == parity).all()
      and (chirality_hat == np.diag(parity)).all(),
      "hat Gamma = (sign +1) G (x) G (x) G (x) G = diag((-1)^(b1+b2+b3+b4))")
```

$\hat{\Gamma}$ is the product of the hat gammas in the author's order. `format(j, "04b")` writes j with four binary digits, and `parity` is (-1) to the number of ones. The check requires the pair products, $\hat{\Gamma} = (+1)\, G \otimes G \otimes G \otimes G$, and that $\hat{\Gamma}$ is the diagonal matrix of the parities.

```
chirality_author = product(gamma, AUTHOR_ORDER)
say("rows with -1 on the diagonal, author: "
    f"{[j for j in range(16) if chirality_author[j, j] < 0]}")
say("rows with -1 on the diagonal, hat:    "
```

```
    f"{[j for j in range(16) if chirality_hat[j, j] < 0]}")
check_record((chirality_author == np.diag([-1] * 8 + [1] * 8)).all(),
             "author: Gamma = gamma^(x0) gamma^(x1) ... gamma^(x7) = diag(-I8, I8)",
             (WOLFRAM, "Gamma_diag"), (PYTHON, "chirality_diag"))
```

The same product of the author's gammas. The two printed lines list the rows with -1: 0 to 7 for the author, $1, 2, 4, 7, 8, 11, 13, 14$ for the hat set. The check confirms the author's $\mathrm{diag}(-I_8, I_8)$ and names the two Revision checks.

In [9], figure 3.

```
fig, (top, bottom) = plt.subplots(2, 1, figsize=(8.0, 5.4), layout="constrained")
for ax, values, title in (
        (top, np.diag(chirality_author), "author: $\\Gamma = $ "
         "$\\gamma^{(x_0)}\\gamma^{(x_1)}\\cdots\\gamma^{(x_7)}$"),
        (bottom, np.diag(chirality_hat), "tensor picture: $\\hat\\Gamma = "
         "G \\otimes G \\otimes G \\otimes G$")):
    colours = ["#2a78d6" if v < 0 else "#e34948" for v in values]
    ax.bar(range(16), values, color=colours, width=0.7)
    ax.axhline(0.0, color="#898781", linewidth=0.8)
    ax.set_ylim(-1.4, 1.4)
    ax.set_yticks([-1, 0, 1])
    ax.set_ylabel("diagonal entry")
    ax.set_title(title, fontsize=10)
top.set_xticks(range(16))
bottom.set_xticks(range(16), [f"{j}\n{bits[j]}" for j in range(16)], fontsize=8)
bottom.set_xlabel("row number $j$ and its bits $b_1 b_2 b_3 b_4$")
save_figure(fig, "chirality_diagonals",
            ...)
```

Two bar charts, one above the other: the 16 diagonal entries of the author's product and of the hat product, blue for -1 and red for $+1$; the lower one labels every row with its number and, on a second line (`"\n"` starts a new line), its bits. **What figure 3 shows and why:** the author's diagonal is eight blue bars followed by eight red ones; the hat diagonal alternates in a pattern that follows the parity of the bits. Both have eight entries of each sign, so a renumbering of the components can turn one into the other; In [10] finds it.

In [10], the change of basis.

```
def change_of_basis(products_1, products_2, i, j):
    """S = sum_A products_1[A] E_ij products_2[A]^T as one matrix product:
    products_1[:, :, i] is the table of the columns i (256 rows of 16 numbers)."""
    return products_1[:, :, i].T @ products_2[:, :, j]
```

The term $\gamma_A E_{ij} \hat{\gamma}_A^T$ has the entry $(\gamma_A)_{ki}(\hat{\gamma}_A)_{lj}$ in row k, column l (only column i of γ_A and column j of $\hat{\gamma}_A$ survive). `products_1[:, :, i]` is the 256×16 table whose row A is column i of γ_A; transposed and multiplied with the same table of the hat products for column j, it gives $\sum_A (\gamma_A)_{ki}(\hat{\gamma}_A)_{lj}$ for all k, l: the whole sum S for $M = E_{ij}$ in one matrix product.

```
def first_nonzero(products_1, products_2):
    """The first (i, j) in reading order with a nonzero S, and that S."""
```

```
    for i, j in itertools.product(range(16), repeat=2):
        S = change_of_basis(products_1, products_2, i, j)
        if S.any():
            return i, j, S
    raise ValueError("every S is zero")
```

`first_nonzero` tries $(i, j) = (0, 0), (0, 1), \ldots$ in this order and returns the first pair with a nonzero S, and that S.

```
i0, j0, S = first_nonzero(author_products, hat_products)
E = np.zeros((16, 16), dtype=np.int64)
E[i0, j0] = 1  # the matrix E_ij with a single 1
plain_sum = sum(author_products[t] @ E @ hat_products[t].T for t in range(256))
report("first (i, j) with a nonzero S", (i0, j0))
report("the different entries of S", sorted(set(int(x) for x in S.flatten())))
```

The first nonzero S belongs to $(i, j) = (0, 8)$. As a control, `plain_sum` computes $\sum_A \gamma_A E_{08} \hat{\gamma}_A^T$ term by term. The entries of S are -16, 0 and 16.

```
c = int((S.T @ S)[0, 0])  # the factor c in S^T S = c I16
root = int(round(c ** 0.5))  # its square root, a whole number here
report("c in S^T S = c I16, and its square root", (c, root))
check((plain_sum == S).all()
      and all((gamma[a] @ S == S @ hat[a]).all() for a in range(8))
      and (S.T @ S == c * I16).all() and root * root == c,
      "S intertwines: gamma^a S = S hat gamma^a for all 8 a, and S^T S = c I16")
```

c is read off the upper-left entry of $S^T S$: 256, with the square root 16 (`c ** 0.5` is the square root, `round` and `int` make it a whole number, and `root * root == c` confirms that it is exact). The check requires that the shortcut equals the plain sum, that S intertwines all eight pairs of gammas, and that $S^T S = c\, I_{16}$.

```
Q = S // root  # exact: every entry of S is a multiple of root
codes = []
for row in Q:
    column = int(np.flatnonzero(row)[0])  # the column of the nonzero entry
    codes.append(("+" if row[column] > 0 else "-") + str(column))
say("rows 0 to 15 of Q: " + " ".join(codes))
check((Q * root == S).all() and (np.count_nonzero(Q, axis=0) == 1).all()
      and (np.count_nonzero(Q, axis=1) == 1).all() and (Q.T @ Q == I16).all(),
      "Q = S / sqrt(c) is a signed permutation matrix (orthogonal)")
```

$Q = S/16$, computed exactly by whole-number division; its codes are printed, $+8\ +4\ +2\ -14\ -7\ +11\ -13\ -1\ -9\ -5\ -3\ +15\ -6\ +10\ -12\ -0$. The check confirms that the division was exact and that Q is an orthogonal signed permutation matrix.

In [11], all products, and the formula for S.

```
all_products_ok = all((author_products[t] == Q @ hat_products[t] @ Q.T).all()
                      for t in range(256))
```

```
check(all_products_ok and (chirality_author == Q @ chirality_hat @ Q.T).all(),
      "gamma_A = Q hat gamma_A Q^T for all 256 products, and Gamma = Q hat Gamma Q^T")
```

Not only the eight gammas but all 256 products, and the product of all eight, transform with Q.

```
formula_ok = all((change_of_basis(author_products, hat_products, i, j)
                  == 16 * Q[i, j] * Q).all()
                 for i, j in itertools.product(range(16), repeat=2))
nonzero_positions = int(np.count_nonzero(Q))
report("positions (i, j) with a nonzero S(E_ij)", nonzero_positions)
check(formula_ok and nonzero_positions == 16,
      "S(E_ij) = 16 Q_ij Q for all 256 positions (i, j)")
```

For every one of the 256 choices $M = E_{ij}$, $S = 16\,Q_{ij}\,Q$. (Reason: $\sum_A \hat\gamma_A Y \hat\gamma_A^T$ commutes with every hat gamma by the argument of Section 4.17, so it is a multiple of I_{16}; its trace is $\sum_A \operatorname{tr}(Y\hat\gamma_A^T\hat\gamma_A) = 256\operatorname{tr}Y$, so it is $16\operatorname{tr}(Y)\,I_{16}$; with $\gamma_A = Q\hat\gamma_A Q^T$ and $Y = Q^T E_{ij}$ this gives $S(E_{ij}) = 16\operatorname{tr}(Q^T E_{ij})\,Q = 16\,Q_{ij}\,Q$.) So S is nonzero exactly at the 16 positions where Q has its nonzero entries, and the first of them in reading order is $(0, 8)$, as found.

In [12], figure 4.

```
fig, ax = plt.subplots(figsize=(6.0, 6.0), layout="constrained")
draw_signs(ax, Q, "$Q$: $\\gamma^{(x_a)} = Q\\,\\hat\\gamma^{(x_a)} Q^T$")
ax.set_xticks(range(16), [f"{j} ({bits[j]})" for j in range(16)], fontsize=6,
              rotation=90)
ax.set_yticks(range(16), [str(i) for i in range(16)], fontsize=8)
ax.set_xlabel("component $j$ of the tensor basis (its bits)")
ax.set_ylabel("component $i$ of the author's basis")
for i in range(16):
    j = int(np.flatnonzero(Q[i])[0])
    ax.text(j, i, f"{Q[i, j]:+d}", ha="center", va="center", color="white",
            fontsize=7)
sign_legend(fig)
save_figure(fig, "change_of_basis",
            ...)
```

Q as a heat map; the columns are labelled with their number and bits (turned by 90 degrees), and the single nonzero entry of every row is written into its square. **What figure 4 shows and why:** sixteen coloured squares, one in each row and each column: Q renumbers the components and flips some signs. The first eight rows use exactly the columns whose bits contain an odd number of ones, where $\hat\Gamma = -1$; this moves the eight entries -1 of $\hat\Gamma$ to the top, as in the author's $\operatorname{diag}(-I_8, I_8)$.

In [13], only one change of basis.

```
def intertwiner_system(left, right):
    """The 2048 x 256 matrix of the equations left[a] X - X right[a] = 0, a = 1..8,
    for the 256 entries of X read row by row."""
    return np.vstack([np.kron(left[a], I16) - np.kron(I16, right[a].T)
                      for a in range(8)])
```

The equations $LX - XR = 0$ for the 256 entries of X, read row by row (entry (k, l) is unknown number $16k + l$). With this numbering, LX is $(L \otimes I_{16})$ times the list of unknowns: by the index formula, $(L \otimes I_{16})_{16i+j,\,16k+l} = L_{ik}\delta_{jl}$, and summing over k, l against X_{kl} gives $\sum_k L_{ik}X_{kj} = (LX)_{ij}$. In the same way XR is $(I_{16} \otimes R^T)$ times the list: $(I_{16} \otimes R^T)_{16i+j,\,16k+l} = \delta_{ik}R_{lj}$, which gives $\sum_l X_{il}R_{lj} = (XR)_{ij}$. `np.vstack` stacks the eight blocks of 256 equations into the 2048×256 system.

```
system = intertwiner_system(gamma, hat)  # the author's gammas and the hat gammas
commutant = intertwiner_system(gamma, gamma)  # the author's gammas on both sides
rank_system, rank_commutant = exact_rank(system.tolist()), exact_rank(
    commutant.tolist())
report("exact ranks of the two systems (2048 equations, 256 unknowns)",
       (rank_system, rank_commutant))
```

Two systems: the intertwiners from the hat gammas to the author's, and the matrices that commute with all eight of the author's gammas. Both have the exact rank 255, so each solution space has the dimension $256 - 255 = 1$ (the number of unknowns minus the number of independent equations).

```
q_solves = not (system @ Q.reshape(256)).any()  # Q, read row by row, solves it
check(rank_system == 255 and q_solves,
      "the intertwiners form a space of dimension 1: the multiples of Q")
```

Q, written as a list of 256 numbers, solves the first system; with the dimension 1, the solutions are exactly the multiples of Q.

```
python_detail = RECORDED[PYTHON]["pin_commutant_dimension_1"]["detail"]
wolfram_detail = RECORDED[WOLFRAM]["Pin44_irreducible_commutant_dim_1"]["detail"]
check_record(256 - rank_commutant == 1 and not (commutant @ I16.reshape(256)).any()
             and "= 1 (Fraction), 1 (sympy)" in python_detail
             and "has dimension 1" in wolfram_detail,
             "only the multiples of I16 commute with all eight author's gammas",
             (PYTHON, "pin_commutant_dimension_1"),
             (WOLFRAM, "Pin44_irreducible_commutant_dim_1"))
```

The second system has a one-dimensional solution space, and I_{16} solves it: only the multiples of I_{16} commute with all eight of the author's gammas. The check also requires the two Revision checks to state the dimension 1 in their own words, and names them.

In [14], the penalty matrix.

```
K = system.T @ system  # 256 x 256, whole numbers
penalty = []  # 4 n_B for every set B
eigen_ok = True
for t, s in enumerate(SETS):
    k = len(s)
    n_B = k if k % 2 == 0 else 8 - k  # the number of anticommuting directions
    v = (Q @ hat_products[t]).reshape(256)  # X_B = Q hat gamma_B, row by row
    eigen_ok &= bool((K @ v == 4 * n_B * v).all())
    penalty.append(4 * n_B)
```

$K = A^T A$ is computed with whole numbers. For each set B of degree k, n_B is k for even k and $8-k$ for odd k; the column v lists the entries of $X_B = Q\hat{\gamma}_B$; and `eigen_ok` stays true only if $Kv = 4n_B v$ exactly, line 5 of the derivation in Section 4.17.

```
values = list(range(0, 33, 4))  # 0, 4, ..., 32
multiplicities = [penalty.count(v) for v in values]
numeric = np.linalg.eigvalsh(K.astype(float))  # floating-point eigenvalues
rounded = np.round(numeric).astype(int)  # nearest whole numbers
numeric_ok = np.max(np.abs(numeric - rounded)) < 1e-9 and all(
    int(np.sum(rounded == v)) == m for v, m in zip(values, multiplicities))
report("multiplicities of 0, 4, ..., 32 in K", multiplicities)
check(eigen_ok and multiplicities == [comb(8, j) for j in range(9)] and numeric_ok,
      "K = A^T A has the exact eigenvalues 4 n_B; only Q has the eigenvalue 0")
```

The multiplicities of $0, 4, \ldots, 32$ are counted: $1, 8, 28, 56, 70, 56, 28, 8, 1$. As an independent control, numpy computes all 256 eigenvalues of K in floating point; they must lie within 10^{-9} of whole numbers and come with the same multiplicities. The check requires the exact eigenvectors, the binomial multiplicities and the numerical agreement.

In [15], figure 5.

```
fig, ax = plt.subplots(figsize=(7.2, 3.9), layout="constrained")
ax.set_axisbelow(True)  # grid lines behind the bars
colours = ["#eb6834" if v == 0 else "#2a78d6" for v in values]
bars = ax.bar(values, multiplicities, width=2.8, color=colours)
ax.bar_label(bars, labels=[str(m) for m in multiplicities], padding=2, fontsize=9)
ax.set_xticks(values)
ax.set_ylim(0, 80)
ax.set_xlabel("eigenvalue of $K = A^T A$ (the penalty of a candidate $X$)")
ax.set_ylabel("multiplicity")
ax.set_title("$\\gamma^{(x_a)}X - X\\hat\\gamma^{(x_a)} = 0$: one solution, $X = Q$")
save_figure(fig, "penalty_spectrum",
            ...)
```

A bar chart of the multiplicities against the eigenvalues, the bar of the eigenvalue 0 in orange. **What figure 5 shows and why:** one orange bar of height 1 at 0 (only Q solves all 2048 equations) and blue bars of the binomial heights 8, 28, 56, 70, 56, 28, 8, 1 at 4 to 32: the part of any matrix that is perpendicular to Q makes the sum of the squared equations at least 4 times its squared size.

In [16], another way to fill the slots.

```
SLOT_2 = [4, 1, 2, 3, 1, 2, 3, 4]  # pairs (x1,x4) (x2,x5) (x3,x6) (x0,x7)
FACTORS_2 = [factor_names(SLOT_2[a], "P" if eta[a] > 0 else "N") for a in range(8)]
hat_2 = [kron4(*[MATRIX_OF[name] for name in FACTORS_2[a]]) for a in range(8)]
clifford_2 = all(
    (hat_2[a] @ hat_2[b] + hat_2[b] @ hat_2[a] == 2 * eta[a] * (a == b) * I16).all()
    for a in range(8) for b in range(8))
check(clifford_2 and all((hat_2[a].T == eta[a] * hat_2[a]).all() for a in range(8)),
      "second assignment: the Clifford relation and the symmetry pattern hold too")
hat_2_products = np.array([product(hat_2, s) for s in SETS])
```

The second assignment of directions to slots, with the pairs (x_1, x_4), (x_2, x_5), (x_3, x_6), (x_0, x_7); the same recipe builds `hat_2`; the check confirms the Clifford relation and the symmetry pattern; and the 256 products of the new set are formed.

```
def diagonal_sets(products):
    """The sets A (as coordinate names) whose product is a diagonal matrix."""
    return [tuple(COORDINATES[a] for a in s) for s, m in zip(SETS, products)
            if (m == np.diag(np.diag(m))).all()]
```

`diagonal_sets` lists, as tuples of coordinate names, the sets whose product is a diagonal matrix.

```
diagonal_author = diagonal_sets(author_products)
say("diagonal products, author:            " + ", ".join(
    "".join(n[1] for n in s) or "I" for s in diagonal_author[:5]) + ", ...")
say("diagonal products, second assignment: " + ", ".join(
    "".join(n[1] for n in s) or "I" for s in diagonal_sets(hat_2_products)[:5])
    + ", ...")
check(len(diagonal_author) == 16 and diagonal_sets(hat_products) == diagonal_author
      and diagonal_sets(hat_2_products) != diagonal_author,
      "the first assignment has the author's 16 diagonal products, the second not")
```

The first five diagonal products of each set are printed by their coordinate numbers (`n[1]` is the digit of a name such as `"x6"`; `or "I"` names the empty set): I, 07, 16, 25, 34 for the author, but I, 07, 14, 25, 36 for the second assignment. The check requires 16 diagonal products for the author, the same sets for the first hat set, and different sets for the second: by the argument of Section 4.17, no signed permutation can relate the second set to the author's.

```
i2, j2, S_2 = first_nonzero(author_products, hat_2_products)
c_2 = int((S_2.T @ S_2)[0, 0])
report("second assignment: entries of S_2 and c_2",
       (sorted(set(int(x) for x in S_2.flatten())), c_2))
check(all((gamma[a] @ S_2 == S_2 @ hat_2[a]).all() for a in range(8))
      and (S_2.T @ S_2 == c_2 * I16).all() and c_2 == 128
      and (np.count_nonzero(S_2, axis=1) == 2).all(),
      "second assignment: S_2 intertwines, S_2^T S_2 = 128 I16, two entries per row")
Q_2 = S_2 / np.sqrt(c_2)  # floating point: the entries are +-1/sqrt(2)
```

The same formula gives a change of basis S_2 for the second set: entries -8, 0, 8, $S_2^T S_2 = 128\, I_{16}$, two nonzero entries in every row. So $Q_2 = S_2/\sqrt{128}$ has the entries $\pm 8/\sqrt{128} = \pm 1/\sqrt{2}$, computed in floating point: an orthogonal matrix that mixes the components in pairs, not a signed permutation.

In [17], figure 6.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(8.8, 4.6), layout="constrained")
for ax, matrix, title in (
        (left, Q, "pairs (x1,x6) (x2,x5) (x3,x4) (x0,x7): $Q$"),
        (right, Q_2, "pairs (x1,x4) (x2,x5) (x3,x6) (x0,x7): $Q_2$")):
    image = ax.imshow(matrix, cmap="RdBu_r", vmin=-1.0, vmax=1.0)
    ax.set_title(title, fontsize=9)
```

```
    ax.set_xticks(range(0, 16, 4))
    ax.set_yticks(range(0, 16, 4))
    ax.set_xlabel("tensor component $j$")
    ax.set_ylabel("author's component $i$")
    ax.grid(False)
fig.colorbar(image, ax=[left, right], shrink=0.8, label="entry")
save_figure(fig, "two_assignments",
            ...)
```

Q and Q_2 side by side with one continuous colour scale (`"RdBu_r"`: blue for -1, white for 0, red for $+1$; `vmin` and `vmax` fix its ends), and a colour bar explaining it. **What figure 6 shows and why:** on the left sixteen full-coloured squares, one per row and column; on the right thirty-two paler squares, two per row and column, of the size $1/\sqrt{2} \approx 0.71$. Both matrices are orthogonal and both turn one set of gammas exactly into the other; only the assignment that matches the author's pairs gives a pure renumbering.

In [18], the last check.

```
names = ["04d_1_slot_pattern.png", "04d_2_tensor_gammas.png",
         "04d_3_chirality_diagonals.png", "04d_4_change_of_basis.png",
         "04d_5_penalty_spectrum.png", "04d_6_two_assignments.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in names),
      "all six figure files exist")
all_checks_passed()
```

The six figure files must exist; the last line reads ALL 18 CHECKS PASSED (notebook 04d): one check in each of In [2], In [3], In [4], In [14] and In [18]; two in each of In [7], In [8], In [10], In [11] and In [13]; and three in In [16].

4.21 What we proved, what we computed, what we assumed

PROVED (exact; the proof is in this chapter, and the computer confirms it with exact arithmetic in the notebook named):

- The square of a linear expression $\sum_a p_a \gamma^a$ is the quadratic form $\sum_a \eta^{aa} p_a^2$ exactly when the coefficients satisfy the Clifford relation; no ordinary numbers do (Section 4.3; Notebook 04b).
- The author's formulas give six 4×4 blocks equal to the six matrices his notebook displays; they are the right (s4) and minus the left (t4) multiplications by the quaternion units, which gives all their rules; the 8×8 matrices satisfy $\tau_1\tau_2\tau_3 = \tau_4\tau_5\tau_6\tau_7 = \sigma$, $\tau_7 = \mathrm{diag}(-I_4, I_4)$, $\bar{\tau}_A = -\tau_A$ and the key rule; the sixteen-by-sixteen matrices T16 satisfy the Clifford relation, and $\mathrm{T16}[8] = \mathrm{diag}(-I_8, I_8)$; named with the coordinate map ($\gamma^{(x_k)} = \mathrm{T16}[k]$ for $k = 0, \ldots, 7$) they satisfy $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I_{16}$ with $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$; they are real signed permutation matrices, symmetric for x_0, x_1, x_2, x_3 and antisymmetric for the time x_4 and the deflating extra times x_5, x_6, x_7 (Section 4.5; Notebook 04a).
- The author's gammas take the square root of the 4+4 quadratic form; plane waves of the flat equation $\sum_a \gamma^{(x_a)} \partial_a \Psi = m\Psi$ obey $E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$; without

extra-time momentum h is Hermitian and commutes with the record's matrix $B = -iC\gamma^{(x_4)}$, while a momentum along an extra time mixes the sectors $B = +1$ and $B = -1$; the record's example has the energies ± 5, eight times each on all sixteen components and four times each in the sector $B = +1$; with a momentum along an extra time the energies become imaginary once $k_5^2 > m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2$, with the growth rate $\sqrt{k_5^2 - m^2 - k_0^2 - k_1^2 - k_2^2 - k_3^2}$, which has no upper bound; the record's example $m = 1$, $k_5 = 2$ has $E = \pm i\sqrt{3}$, eight times each (Section 4.9; Notebook 04b).

- The 256 products of the gammas multiply by rule R1, square to $\pm I_{16}$ by rule R2, have trace 0 except I_{16}, are perpendicular in the trace sense and independent: Cl(4,4) is the set of all real 16×16 matrices; 136 products are symmetric and 120 antisymmetric; the 128 even ones span the block diagonal matrices and the 128 odd ones the block off-diagonal matrices; eight gamma matrices need at least sixteen components; $S^{ab} = \frac{1}{4}[\gamma^a, \gamma^b] = \frac{1}{2}\gamma^a\gamma^b$, with 12 rotation planes and 16 boost planes (Section 4.13; Notebook 04c).
- A second set of real gammas, built from P, N and G with Kronecker products, satisfies the same Clifford relation; the author's gammas are this set with the sixteen components renumbered and some signs flipped, $\gamma^{(x_a)} = Q\hat{\gamma}^{(x_a)}Q^T$ with a signed permutation Q; this change of basis is unique up to a factor; and only the multiples of I_{16} commute with all eight of the author's gammas (Section 4.17; Notebook 04d).

The Revision record confirms these statements independently. The table names the record file and its checks for each of them; the notebook named in the list above reproduces every one of these checks and prints it. For the four theory checks of Section 4.9, Notebook 04b also requires that the record's text states the same formulas and examples word for word, so a later change of the record would make the notebook fail instead of going unnoticed.

statement	**Revision record file**	**its checks**
the construction of T16 and its rules (Section 4.5)	`Revision/algebra/reports/wolfram-algebra.json` and `Revision/algebra/reports/python-algebra.json`	the 30 checks of the table at the end of Section 4.5
the stored gammas are the rebuilt ones (Section 4.5)	`Revision/algebra/gammas.json` and `Revision/algebra/reports/python-gammas.json`	all 2×2048 entries compared, none differs
h^2 and the mass shell (Section 4.9)	`Revision/theory/reports/python-field-theory.json`	`mode_hamiltonian_B_selfadjoint_dispersion`
the good-sector example: ± 5 eight times each on all sixteen components, $Bh = hB$, and ± 5 four times each in the sector $B = +1$ (Section 4.9)	`Revision/theory/reports/python-field-theory.json`	`good_sector_spectrum_and_B_sectors`
the growing modes of the example $m = 1$, $k_5 = 2$	`Revision/theory/reports/python-field-theory.json`	`extra_time_modes_grow`

statement	Revision record file	its checks
the growth rate and its missing upper bound	Revision/theory/reports/python-scope.json	extra_time_growth_rates_unbounded
Cl(4,4) is all real 16×16 matrices (Section 4.13)	Revision/algebra/reports/wolfram-algebra.json	Clifford_basis_spans_full_matrix_algebra
the same	Revision/algebra/reports/python-algebra.json	clifford_products_span_M16
the even products span the block diagonal matrices	Revision/algebra/reports/wolfram-algebra.json	even_subalgebra_dimension
the same	Revision/algebra/reports/python-algebra.json	even_products_span_M8_plus_M8
$\tau_1 \cdots \tau_7 = I_8$, $\tau_7 = \mathrm{diag}(-I_4, I_4)$	Revision/algebra/reports/python-algebra.json	tau7_and_product
$S^{ab} = \frac{1}{2}\gamma^a\gamma^b$, entries 0 and $\pm\frac{1}{2}$	Revision/algebra/reports/wolfram-algebra.json	S_half_product, S_real_entries_in_half_integers
the same	Revision/algebra/reports/python-algebra.json	S_definition
only multiples of I_{16} commute with all gammas (Section 4.17)	Revision/algebra/reports/wolfram-algebra.json	Pin44_irreducible_commutant_dim_1
the same	Revision/algebra/reports/python-algebra.json	pin_commutant_dimension_1

COMPUTED (floating-point numbers, with the measured accuracy): the eigenvalues of the 2×2 roots agree with $\pm\sqrt{p^2 \pm 1}$ to 10^{-12}; $(\sum_a p_a \gamma^{(x_a)})^2 = \eta(p,p) I_{16}$ for 300 random vectors, every deviation below 10^{-12} (the bound that the notebook asserts; the sizes themselves, plotted in figure 4 of Notebook 04b, depend on the computer's numerical library); the sixteen energies of the good-sector example lie within 10^{-12} of ± 5, with eight independent solutions at each energy, the eight energies in the sector $B = +1$ lie within 10^{-12} of ± 5, four of each, and the smallest singular value of $EI_{16} - h$ equals the distance to ± 5 within 10^{-9}; the 160×16 energies with a momentum k_5 lie within 10^{-10} of $\pm\sqrt{25 - k_5^2}$; the floating-point eigenvalues of the penalty matrix K lie within 10^{-9} of $0, 4, \ldots, 32$ (Notebooks 04b and 04d). Each of these confirms an exact statement proved above.

ASSUMED: the author's metric, the roles of his coordinates and his formulas for T16 (the input of the theory; Section 4.4 and Section 4.5); in Section 4.9, flat 4+4 space without self-interaction, a model of the algebra only, since the author's extra times deflate at every moment; and two facts of linear algebra quoted without proof: in a space of dimension m any m independent elements form a basis (Section 4.13), and the number of singular values of a square matrix M that are 0 is the number of independent solutions u of $Mu = 0$ (used for the counts of Notebook 04b). The fact that the trace of every square matrix is the sum of its eigenvalues, counted with their multiplicities, is not assumed here: Section 1.34 proves it from the fundamental theorem of algebra, which Chapter 1

assumes.

HYPOTHESIS and OPEN: this chapter states none. What the growing waves along the extra times mean for the curved, deflating space-time is the subject of Chapter 8. Nothing in this chapter concerns the creation of universes or matter and antimatter; the matrices it builds are used in Part V, where the honest scope of those questions is stated.

4.22 Exercises

Exercise 1. Show that the three real matrices σ_x, σ_z and N of Section 4.3 satisfy $(p\sigma_x+q\sigma_z+rN)^2 = (p^2+q^2-r^2)I_2$ for all p, q, r. Compute $\sigma_x\sigma_z N$. Then prove that no real 2×2 matrix M with $M^2 = +I_2$ anticommutes with both σ_x and σ_z, and explain why three anticommuting matrices fit into 2×2 although $2^3 = 8 > 2^2$.

Answer. The squares are $\sigma_x^2 = \sigma_z^2 = I_2$ and $N^2 = -I_2$. The pairs anticommute: $\sigma_z\sigma_x = -\sigma_x\sigma_z$ (Section 4.2) and $N\sigma_x = -\sigma_x N$ (Section 4.3), and $\sigma_z N = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $N\sigma_z = \begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix} = -\sigma_z N$. So the general rule of Section 4.3 with the signs $(+1,+1,-1)$ gives $(p^2+q^2-r^2)I_2$. Next, $\sigma_x\sigma_z = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -N$, so $\sigma_x\sigma_z N = -N^2 = I_2$. For the second part, every real 2×2 matrix is $\begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} = \frac{\alpha+\delta}{2}I_2 + \frac{\beta+\gamma}{2}\sigma_x + \frac{\alpha-\delta}{2}\sigma_z + \frac{\beta-\gamma}{2}N$ (check the four entries), so write $M = aI_2 + b\sigma_x + c\sigma_z + dN$. Then $\sigma_x M + M\sigma_x = 2a\sigma_x + 2bI_2 + c(\sigma_x\sigma_z + \sigma_z\sigma_x) + d(\sigma_x N + N\sigma_x) = 2a\sigma_x + 2bI_2$, which vanishes only for $a = b = 0$; in the same way $\sigma_z M + M\sigma_z = 2a\sigma_z + 2cI_2$ forces $c = 0$. So $M = dN$ and $M^2 = -d^2 I_2$, never $+I_2$. Three matrices fit because $n = 3$ is odd: the trace lemma fails for the product of all three, which is I_2 with trace 2, so the eight products come in pairs that are equal up to sign (for example $\sigma_x\sigma_z = -N$) and only four of them are independent, $4 = 2^2$, like the seven τ of Section 4.13.

Exercise 2. The code table of Notebook 04a (output of In [18]) begins, for row 0, with -15 in the column x1 and $+14$ in the column x2; row 14 reads -1 (x1), row 15 reads -0 (x1) and -1 (x2). Use only these entries to show that row 0 of $(\gamma^{(x_1)})^2$ is $+u_0$ and that row 0 of $\gamma^{(x_1)}\gamma^{(x_2)}$ is minus row 0 of $\gamma^{(x_2)}\gamma^{(x_1)}$.

Answer. Row 0 of $\gamma^{(x_1)}$: $(\gamma^{(x_1)}v)_0 = -v_{15}$. With $v = \gamma^{(x_1)}u$ and row 15 of $\gamma^{(x_1)}$, $v_{15} = -u_0$, so $(\gamma^{(x_1)}\gamma^{(x_1)}u)_0 = -(-u_0) = +u_0$, as it must be for the space-like x_1. For the products: $(\gamma^{(x_1)}\gamma^{(x_2)}u)_0 = -(\gamma^{(x_2)}u)_{15} = -(-u_1) = +u_1$ (row 15 of $\gamma^{(x_2)}$ is -1), while $(\gamma^{(x_2)}\gamma^{(x_1)}u)_0 = +(\gamma^{(x_1)}u)_{14} = +(-u_1) = -u_1$ (row 14 of $\gamma^{(x_1)}$ is -1). The two differ by a sign: in row 0, the two gammas anticommute.

Exercise 3. How many of the 56 products of degree 3 square to $+I_{16}$, and how many to $-I_{16}$? Compare with figure 3 of Notebook 04c.

Answer. By rule R2 with $k = 3$, $\gamma_A^2 = (-1)^3 \prod_{a\in A}\eta^{aa} I_{16} = -\prod_{a\in A}\eta^{aa} I_{16}$. This is $+I_{16}$ exactly when the product of the three signs is -1, that is when A contains an odd number of time-like directions. One time-like and two space-like directions: $\binom{4}{1}\binom{4}{2} = 4\cdot 6 = 24$ sets; three time-like: $\binom{4}{3} = 4$ sets. So $24 + 4 = 28$ products square to $+I_{16}$ and $56 - 28 = 28$ to $-I_{16}$: the two equal parts (28 red and 28 blue) of the stacked bar at degree 3 in the figure.

Exercise 4. Compute the 4×4 matrix $P \otimes N$ with the index formula, and its square in two ways: with the mixed-product rule, and by following row 0 in the code notation.

Answer. $P \otimes N = \begin{pmatrix} 0 \cdot N & 1 \cdot N \\ 1 \cdot N & 0 \cdot N \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{pmatrix}$. By the mixed-product rule, $(P \otimes N)^2 = P^2 \otimes N^2 = I_2 \otimes (-I_2) = -I_4$. In code notation, row 0 is $+3$ and row 3 is -0, so $((P \otimes N)^2 u)_0 = +((P \otimes N)u)_3 = -u_0$, row 0 of $-I_4$.

Exercise 5. Take $m = 1$, no momentum in x_0, x_1, x_2, x_3, and a wave $e^{ik_{x_5} x_5}$ along the extra time x_5 with the coordinate momentum $k_{x_5} = 1$, at a point where $\sin z = 1/64$ and $b_4 = 1$. Using the frame factor of Section 4.4, find the frame momentum and E^2 (a) when $a_4 = 0$ and (b) when $a_4 = \ln 2$. What happens to the growth rate as the extra times deflate?

Answer. The frame factor of x_5 is $b_4^{1/2} e^{-a_4} \sin^{1/6} z = e^{-a_4}(1/64)^{1/6} = e^{-a_4}/2$, since $b_4 = 1$ and $2^6 = 64$. The frame momentum is the coordinate momentum divided by it: $k_5 = 2e^{a_4}$. (a) $a_4 = 0$: $k_5 = 2$, and $E^2 = m^2 - k_5^2 = 1 - 4 = -3$, so $E = \pm i\sqrt{3}$: the exact example of the Revision record, with the growth rate $\sqrt{3} \approx 1.73$. (b) $a_4 = \ln 2$: $e^{a_4} = 2$, $k_5 = 4$, $E^2 = 1 - 16 = -15$, growth rate $\sqrt{15} \approx 3.87$. As a_4 grows the extra times deflate, the frame momentum grows like e^{a_4}, and the growth rate grows with it. (This is the local plane-wave reading of Section 4.9, with the coefficients held fixed near the point.)

Exercise 6. For which of the pairs (x_1, x_2), (x_1, x_4), (x_5, x_6), (x_4, x_0) does $(2S^{ab})^2$ equal $-I_{16}$, and for which $+I_{16}$? Which are rotation planes and which boost planes? Show also that $S^{ba} = -S^{ab}$.

Answer. $2S^{ab} = \gamma^a \gamma^b$ for $a \neq b$ (Section 4.13), and $(\gamma^a \gamma^b)^2 = -\eta^{aa}\eta^{bb} I_{16}$. (x_1, x_2): both space-like, $\eta^{aa}\eta^{bb} = +1$, square $-I_{16}$, a rotation plane. (x_1, x_4): space-like and time-like, -1, square $+I_{16}$, a boost plane. (x_5, x_6): both time-like, $(-1)(-1) = +1$, square $-I_{16}$, a rotation plane. (x_4, x_0): time-like and space-like, square $+I_{16}$, a boost plane. Finally $S^{ba} = \frac{1}{4}(\gamma^b \gamma^a - \gamma^a \gamma^b) = -\frac{1}{4}(\gamma^a \gamma^b - \gamma^b \gamma^a) = -S^{ab}$.

Exercise 7. Without a computer, find the energies of the plane waves of the flat equation for $m = 3$, $k_0 = 0$, $(k_1, k_2, k_3) = (0, 0, 4)$ and no momentum along the extra times, and how often each occurs.

Answer. $h^2 = (m^2 + k_3^2) I_{16} = (9 + 16) I_{16} = 25 I_{16}$ (Section 4.9), so every eigenvalue is $+5$ or -5. Without extra-time momentum h is Hermitian, and its trace is 0 (every term of h is a multiple of $\gamma^{(x_4)}$ or of $\gamma^{(x_4)}\gamma^{(x_a)}$ with $a \neq x_4$, which have trace 0). With n_+ eigenvalues $+5$ and n_- eigenvalues -5: $n_+ + n_- = 16$ and $5n_+ - 5n_- = 0$, so $E = +5$ and $E = -5$ occur eight times each.

5 The matrices C, Γ and B, the groups Pin(4,4) and Spin(4,4), and the charge-conjugation matrices

Chapter 4 built the author's eight real gamma matrices, one 16×16 matrix for each of the eight directions of the author's universe, and proved that they obey the Clifford relation. This chapter builds from them everything else that a field with 16 components needs before its equations can be written down: the charge matrix C (which turns two spinors into a number), the chirality Γ (which cuts the 16 components into two halves of 8), the matrix B (whose bilinear is the charge

density), the groups Pin(4,4) and Spin(4,4) of spinor transformations with their generators S^{ab}, and the two charge-conjugation matrices $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$. It proves the author's statement that the 16 components carry an irreducible representation of Pin(4,4) that splits under Spin(4,4) into two inequivalent halves, and it answers exactly the question which group the scaled commutators S^{ab} generate.

5.1 What this chapter does

A spinor field Ψ attaches 16 numbers $\Psi_1, \ldots, \Psi_{16}$ to every point of spacetime. Its field equation (Chapter 7) multiplies Ψ by the gamma matrices. Four further questions must be answered before the equation can be used, and each needs one of the matrices of this chapter.

- How do we make a single number out of a spinor, a number that does not change when the eight directions are turned into each other? The answer is the **bilinear** $\Psi^\dagger C \Psi$ with the charge matrix C (Sections 5.4 and 5.18).
- Which parts of a spinor stay separate under every turning of the directions? The two **chiral halves**, components 1 to 8 and 9 to 16, picked out by the chirality Γ (Sections 5.5 and 5.13).
- What is the **charge density** of a spinor field, the quantity whose total stays constant in time when no charge flows in or out through the edges of space? It is $\Psi^\dagger B \Psi$ with the matrix B (Section 5.6).
- What exchanges particles and antiparticles? A **charge-conjugation matrix**. Because the author's gammas are real, plain complex conjugation does nothing to a real field and cannot exchange anything; the exchange must be made by a matrix. Section 5.28 defines a charge-conjugation matrix by a condition that it must obey together with every gamma, and proves that there are exactly two such matrices, up to a factor, $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$ (Sections 5.28 and 5.29). For the quantised field only one of the two, up to a phase factor, keeps the basic rule of the quantum theory, and it reverses the mass (Section 5.34). Section 5.39 states what these exact maps do and do not say about matter and antimatter in the universe.

Between these questions stands the group theory: the groups Pin(4,4) and Spin(4,4) that act on spinors, the words irreducible and inequivalent, and what the scaled commutators $S^{ab} = \frac{1}{4}[\gamma^a, \gamma^b]$ generate. Everything is introduced from zero: a reader who knows school algebra and the derivative of one-variable functions can follow every line.

The chapter has six worked examples, each a complete Jupyter notebook:

- Notebook 05a: the matrices C, Γ and B and every property that the Revision record states about them (26 checks, six figures);
- Notebook 05b: the commutants, which prove that Pin(4,4) acts irreducibly on the 16 components and that Spin(4,4) splits them into two inequivalent halves (17 checks, five figures);
- Notebook 05d: the spin transformations, rotations and boosts, the double cover and the forms kept by them (20 checks, six figures);
- Notebook 05f: what the scaled commutators generate (24 checks, six figures);

- Notebook 05c: the charge-conjugation matrices (20 checks, nine figures);
- Notebook 05e: the conjugation of the quantised field, computed on a space of $2^{16} = 65536$ quantum states (18 checks, eight figures).

They appear in this order, a, b, d, f, c, e: first the matrices, then the group theory, then charge conjugation. Every notebook is complete in itself, so you may run them in any order.

Notation. The author's coordinates are written $x0, \ldots, x7$ (Chapter 0 writes them $x_0, \ldots, x_7$): $x0$ is the hidden space direction; $x1$, $x2$, $x3$ are ordinary 3-space, which inflates; $x4$ is the time; $x5$, $x6$, $x7$ are the three extra times, which deflate exponentially. The gamma matrix of the direction $x4$ is written $\gamma^{(x4)}$, and so on. Letters a, b, c, d, e stand for any of the eight directions. The flat metric of the frame is

$$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$$

in the order $x0, \ldots, x7$; its entry for the direction a is written η_{aa} (or η^{aa}, which is the same number). The directions with $\eta_{aa} = +1$ ($x0$, $x1$, $x2$, $x3$) are called **space-like**, those with $\eta_{aa} = -1$ ($x4$, $x5$, $x6$, $x7$) **time-like**. The symbol 1 also stands for the 16×16 identity matrix, and 1_8 for the 8×8 one.

5.2 The eight real gamma matrices

The gamma matrices are eight real 16×16 matrices $\gamma^{(x0)}, \ldots, \gamma^{(x7)}$. Chapter 4 built them from the author's formulas (his matrices T16, with $\gamma^{(xk)} = \mathrm{T16}[k]$ for $k = 0, \ldots, 7$) and proved the **Clifford relation**

$$\gamma^a \gamma^b + \gamma^b \gamma^a = 2\eta^{ab}\, 1 \qquad \text{for all } a, b,$$

where η^{ab} is η_{aa} when $a = b$ and 0 when $a \neq b$. Written out, the relation says two things. For $a = b$: $\gamma^a \gamma^a = \eta_{aa} 1$, so a space-like gamma squares to $+1$ and a time-like gamma to -1. For $a \neq b$: $\gamma^a \gamma^b = -\gamma^b \gamma^a$, so two different gammas **anticommute** (exchanging them costs a sign). The Revision record stores the eight matrices in the file `Revision/algebra/gammas.json`, and two independent programs have checked them: `Revision/algebra/reports/python-algebra.json` (35 of 35 checks passed) and `Revision/algebra/reports/wolfram-algebra.json` (45 of 45 checks passed).

Every gamma is a **signed permutation matrix**: in each row exactly one entry is nonzero, and it is $+1$ or -1; the same holds in each column. Such a matrix is described completely by saying, for each row, in which column its nonzero entry stands and with which sign. The two following tables do this for all eight gammas, first the four space-like ones and then the four time-like ones. The entry $+9$ in row 1 of the column $\gamma^{(x0)}$ means: the matrix $\gamma^{(x0)}$ has the entry $+1$ in row 1 and column 9, and zeros in the other fifteen places of row 1. The tables are read off the record `Revision/algebra/gammas.json`.

row	$\gamma^{(x0)}$	$\gamma^{(x1)}$	$\gamma^{(x2)}$	$\gamma^{(x3)}$
1	+9	−16	+15	−14
2	+10	−15	−16	+13
3	+11	+14	−13	−16
4	+12	+13	+14	+15
5	+13	−12	+11	−10
6	+14	−11	−12	+9
7	+15	+10	−9	−12
8	+16	+9	+10	+11
9	+1	+8	−7	+6
10	+2	+7	+8	−5
11	+3	−6	+5	+8
12	+4	−5	−6	−7
13	+5	+4	−3	+2
14	+6	+3	+4	−1
15	+7	−2	+1	+4
16	+8	−1	−2	−3

The four time-like gammas, read in the same way:

row	$\gamma^{(x4)}$	$\gamma^{(x5)}$	$\gamma^{(x6)}$	$\gamma^{(x7)}$
1	−14	+15	+16	+9
2	+13	+16	−15	+10
3	+16	−13	+14	+11
4	−15	−14	−13	+12
5	+10	−11	−12	−13
6	−9	−12	+11	−14
7	−12	+9	−10	−15
8	+11	+10	+9	−16
9	+6	−7	−8	−1
10	−5	−8	+7	−2
11	−8	+5	−6	−3
12	+7	+6	+5	−4
13	−2	+3	+4	+5
14	+1	+4	−3	+6
15	+4	−1	+2	+7
16	−3	−2	−1	+8

How to multiply two signed permutation matrices. Let M have in row r the entry s (a sign) in column k, and let N have in row k the entry t in column c. The entry of MN in row r and column q is $\sum_j M_{rj}N_{jq}$; only $j = k$ contributes, and N_{kq} is nonzero only for $q = c$. So MN has in row r the single entry st in column c: follow the row number through the two columns of the table and multiply the signs. For example, for $\gamma^{(x4)}\gamma^{(x4)}$: row 1 of $\gamma^{(x4)}$ points to column 14 with the sign −, and row 14 of $\gamma^{(x4)}$ points to column 1 with the sign +; so row 1 of $\gamma^{(x4)}\gamma^{(x4)}$ has the entry $(-1)(+1) = -1$ in column 1, as $\gamma^{(x4)}\gamma^{(x4)} = \eta_{x4\,x4}1 = -1$ demands. For $\gamma^{(x0)}\gamma^{(x0)}$: row 1 points to column 9 with +, row 9 points back to column 1 with +; the product has +1 on the diagonal.

Symmetric and antisymmetric. The transpose M^T of a matrix exchanges rows and columns, $(M^T)_{rc} = M_{cr}$. The table shows that $\gamma^{(x1)}$ has −1 in row 1, column 16, and also −1 in row 16, column 1: the two entries mirror each other with the same sign. In fact the four space-like gammas

are **symmetric** ($M^T = M$) and the four time-like ones are **antisymmetric** ($M^T = -M$; for example $\gamma^{(x4)}$ has -1 in row 1, column 14 and $+1$ in row 14, column 1). In one formula:

$$(\gamma^a)^T = \eta_{aa}\,\gamma^a.$$

The statuses of these facts are listed in the next table. Here and in every later status table, the last column names the report file of the Revision record and the names of its checks; a report is a file in which a program has recorded every check with its name, its verdict and a detail text. The two reports `python-algebra.json` and `wolfram-algebra.json` lie in the folder `Revision/algebra/reports`.

statement	status	where it is verified
the Clifford relation for all 64 pairs of directions	PROVED (exact integer arithmetic)	`python-algebra.json`, checks `clifford_relation` and `clifford_relation_sympy`; `wolfram-algebra.json`, check `Clifford_relation`
every gamma is real and a signed permutation matrix	PROVED	`python-algebra.json`, check `reality_signed_permutations`; `wolfram-algebra.json`, checks `reality` and `signed_permutation_matrices`
$(\gamma^a)^T = \eta_{aa}\gamma^a$	PROVED	`python-algebra.json` and `wolfram-algebra.json`, check `symmetry_pattern`

Notebook 05a checks all three facts again. Everything in Sections 5.3 to 5.6 follows from the Clifford relation and the symmetry pattern alone.

5.3 Six rules for products of gammas

A **product of different gammas** is a product $P = \gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$ of k gammas of k different directions $a_1, \ldots, a_k$; k is its **degree**. There are $2^8 = 256$ of them if we take the directions in the order $x0, \ldots, x7$ and count the empty product 1 (each direction is either in the product or not). The six rules below are used again and again in this chapter.

Rule 1 (moving a gamma through a product). Take a gamma γ^e and the product P. Move γ^e from the left of P to its right, one factor at a time. Passing a factor γ^{a_j} with $a_j \neq e$ costs a sign, because

$$\gamma^e\gamma^{a_j} = -\gamma^{a_j}\gamma^e \qquad (a_j \neq e),$$

which is the Clifford relation for two different directions. Passing the factor γ^e itself costs nothing, because a matrix commutes with itself. So the sign is -1 to the power of the number of factors different from e:

$$\gamma^e P = (-1)^{k-1}\, P\gamma^e \quad \text{if } e \text{ is one of the } k \text{ factors}, \qquad \gamma^e P = (-1)^k\, P\gamma^e \quad \text{if it is not.}$$

Rule 2 (the square of a product). Write $P = \gamma^{a_1} P'$ with $P' = \gamma^{a_2} \cdots \gamma^{a_k}$. Then

$$PP = \gamma^{a_1} P' \gamma^{a_1} P'.$$

Move the second γ^{a_1} to the left through P'; P' has $k-1$ factors, all different from a_1, so Rule 1 (read from right to left) gives

$$PP = (-1)^{k-1} \gamma^{a_1}\gamma^{a_1} P'P' = (-1)^{k-1} \eta_{a_1 a_1}\, P'P',$$

where the second step is the Clifford relation $\gamma^{a_1}\gamma^{a_1} = \eta_{a_1 a_1} 1$. The product $P'P'$ is the square of a product of $k-1$ different gammas, so the same step applies to it, with $k-2$ factors to pass, and so on down to the empty product. Collecting the signs:

$$PP = (-1)^{(k-1)+(k-2)+\cdots+1+0}\ \eta_{a_1 a_1}\eta_{a_2 a_2}\cdots\eta_{a_k a_k}\, 1 = (-1)^{k(k-1)/2}\ \eta_{a_1 a_1}\cdots\eta_{a_k a_k}\, 1,$$

where the last step is the sum of the numbers $0, 1, \ldots, k-1$, which is $k(k-1)/2$ (write the sum forwards and backwards and add: k pairs, each adding up to $k-1$).

Rule 3 (reversing a product). $\gamma^{a_k}\cdots\gamma^{a_2}\gamma^{a_1} = (-1)^{k(k-1)/2}\,\gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$. Proof: in the reversed product, move γ^{a_1} from the last place to the first; it passes $k-1$ different factors, each costing a sign. Then move γ^{a_2} to the second place; it passes $k-2$ factors. Continuing, the total number of sign changes is $(k-1)+(k-2)+\cdots+0 = k(k-1)/2$.

Rule 4 (the transpose of a product). The transpose of a product is the product of the transposes in the reverse order, $(MN)^T = N^T M^T$. With the symmetry pattern $(\gamma^a)^T = \eta_{aa}\gamma^a$ and Rule 3:

$$P^T = (\gamma^{a_k})^T \cdots (\gamma^{a_1})^T = \eta_{a_1 a_1}\cdots\eta_{a_k a_k}\ \gamma^{a_k}\cdots\gamma^{a_1} = (-1)^{k(k-1)/2}\ \eta_{a_1 a_1}\cdots\eta_{a_k a_k}\ P.$$

Rule 5 (the trace of a product is zero). The **trace** $\operatorname{tr} M$ of a square matrix is the sum of its diagonal entries; for two square matrices $\operatorname{tr}(MN) = \operatorname{tr}(NM)$, because both are $\sum_{r,c} M_{rc} N_{cr}$. Claim: every product P of $k \geq 1$ different gammas has $\operatorname{tr} P = 0$. Proof: first find a gamma γ^e that anticommutes with P. If k is even, take e to be one of the factors: by Rule 1 the sign is $(-1)^{k-1} = -1$. If k is odd, then $k \leq 7$, so at least one of the eight directions is not a factor; take e to be such a direction: the sign is $(-1)^k = -1$. Now, using $\gamma^e\gamma^e = \eta_{ee} 1$ and $\eta_{ee}\eta_{ee} = 1$,

$$\operatorname{tr} P = \eta_{ee} \operatorname{tr}(\gamma^e\gamma^e P) = \eta_{ee} \operatorname{tr}(\gamma^e P \gamma^e) = -\eta_{ee} \operatorname{tr}(\gamma^e\gamma^e P) = -\operatorname{tr} P.$$

The first step inserts $\eta_{ee}\gamma^e\gamma^e = 1$; the second is $\operatorname{tr}(MN) = \operatorname{tr}(NM)$ with $M = \gamma^e$ and $N = \gamma^e P$; the third uses $P\gamma^e = -\gamma^e P$; the fourth undoes the first. A number equal to minus itself is 0.

Rule 6 (counting eigenvalues). An **eigenvalue** of a square matrix M is a number λ for which some column $v \neq 0$ has $Mv = \lambda v$. A 16×16 matrix has 16 eigenvalues when each is counted as often as it occurs as a root of the polynomial $\det(\lambda 1 - M)$ (its **multiplicity**), and the trace is their sum. Claim: if $MM = 1$ and $\operatorname{tr} M = 0$, then M has the eigenvalue $+1$ eight times and -1 eight times.

Proof: if $Mv = \lambda v$, then $v = MMv = M(\lambda v) = \lambda^2 v$, so $\lambda^2 = 1$ and $\lambda = \pm 1$. With n_+ eigenvalues $+1$ and n_- eigenvalues -1: $n_+ + n_- = 16$ and $n_+ - n_- = \operatorname{tr} M = 0$, so $n_+ = n_- = 8$. In the same way, if $MM = -1$ and $\operatorname{tr} M = 0$, then $\lambda^2 = -1$, $\lambda = \pm i$, and $i(n_+ - n_-) = 0$ gives eight $+i$ and eight $-i$. In both cases the **determinant**, the product of the 16 eigenvalues, is $(+1)^8(-1)^8 = 1$ or $i^8(-i)^8 = 1$.

Applied to the gammas themselves: a space-like gamma has $\gamma\gamma = +1$ and trace 0 (Rule 5 with $k = 1$), so eight eigenvalues $+1$ and eight -1; a time-like gamma has eight $+i$ and eight $-i$; every gamma has determinant $+1$. Status: PROVED (by the rules above). Notebook 05a checks the squaring rule on all 255 products of degree 1 to 8 and counts the eigenvalues; Notebook 05d computes the eight determinants exactly.

5.4 The charge matrix C and the Dirac adjoint

Definition. The **charge matrix** is the product of the four space-like gammas,

$$C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}.$$

The author calls it sigma16. It is used to make numbers out of spinors: the **Dirac adjoint** of a column Ψ is the row $\bar{\Psi} = \Psi^\dagger C$, where $\Psi^\dagger$ is the row of the complex conjugates of the components of Ψ, and the **scalar** of the field is

$$S = \bar{\Psi}\Psi = \Psi^\dagger C \Psi = \sum_{r,c} \Psi_r^* C_{rc} \Psi_c.$$

Section 5.18 proves that S does not change when the directions are turned by any element of Spin(4,4); this is why C, and not the identity matrix, stands between $\Psi^\dagger$ and Ψ. Here are its properties, each derived from Section 5.3.

(C1) How C moves past a gamma: $C\gamma^a = -\eta_{aa}\gamma^a C$**.** C is a product of $k = 4$ different gammas. If a is space-like, γ^a is one of its factors, and Rule 1 gives $\gamma^a C = (-1)^3 C\gamma^a = -C\gamma^a$. If a is time-like, it is not a factor, and Rule 1 gives $\gamma^a C = (-1)^4 C\gamma^a = C\gamma^a$. Both cases read $\gamma^a C = -\eta_{aa} C\gamma^a$. Multiplying this by $-\eta_{aa}$ and using $\eta_{aa}\eta_{aa} = 1$ gives the claim. So C anticommutes with the four space-like gammas and commutes with the four time-like ones.

(C2) C is symmetric. Rule 4 with $k = 4$ and four space-like factors: $C^T = (-1)^6(+1)^4\, C = C$.

(C3) $CC = 1$**.** Rule 2 with $k = 4$: $CC = (-1)^6(+1)^4\, 1 = 1$. So C is its own inverse, $C^{-1} = C$.

(C4) Every $C\gamma^a$ **is antisymmetric.** Line by line:

$$(C\gamma^a)^T = (\gamma^a)^T C^T = \eta_{aa}\gamma^a C = \eta_{aa}(-\eta_{aa})\, C\gamma^a = -C\gamma^a.$$

The first step is $(MN)^T = N^T M^T$; the second uses the symmetry pattern and (C2); the third is (C1) in the form $\gamma^a C = -\eta_{aa} C\gamma^a$; the fourth is $\eta_{aa}\eta_{aa} = 1$.

(C5) $C\gamma^a C^{-1} = -(\gamma^a)^T$**.** With $C^{-1} = C$ from (C3):

$$C\gamma^a C = (-\eta_{aa}\gamma^a C)\, C = -\eta_{aa}\gamma^a\, CC = -\eta_{aa}\gamma^a = -(\gamma^a)^T.$$

The first step is (C1), the second regroups the product, the third is (C3), the fourth is the symmetry pattern.

(C6) C has signature (8, 8). By Rule 5, $\operatorname{tr} C = 0$; with $CC = 1$, Rule 6 gives eight eigenvalues $+1$ and eight -1. For a real symmetric matrix the pair (number of positive eigenvalues, number of negative eigenvalues) is called its **signature**; C has signature $(8, 8)$. This means that the **quadratic form** $u^T C u = \sum_{r,c} u_r C_{rc} u_c$ of a real column u takes both signs. Example (Notebook 05a, section 7): the table of Section 5.6 shows $C_{1,5} = C_{5,1} = -1$. For $u(t) = \cos t\, e_1 + \sin t\, e_5$, where e_r is the column with 1 in row r and 0 elsewhere, only these two entries meet two nonzero components, and

$$u^T C u = 2 \cos t \, \sin t \, C_{1,5} = -\sin 2t,$$

by the double-angle formula $2 \sin t \cos t = \sin 2t$. At $t = \pi/4$ the form is -1; with the components 9 and 13, where $C_{9,13} = +1$, it is $+1$. The ordinary squared length $u^T u = \cos^2 t + \sin^2 t = 1$ never changes sign; the form of C does.

(C7) The shape of C. The table of Section 5.6 shows $C = \operatorname{diag}(-\sigma, \sigma)$, the block matrix with $-\sigma$ in the top left, $+\sigma$ in the bottom right and zeros elsewhere, where σ is the author's 8×8 matrix with the 4×4 identity in its two off-diagonal blocks:

$$\sigma = \begin{pmatrix} 0 & 1_4 \\ 1_4 & 0 \end{pmatrix}.$$

Consequences for the bilinears. The **currents** of the field are $J^a = -i\bar{\Psi}\gamma^a\Psi = \Psi^\dagger(-iC\gamma^a)\Psi$. By (C4), $C\gamma^a$ is real and antisymmetric, so the matrix $-iC\gamma^a$ is **Hermitian** (equal to its conjugate transpose): $(-iC\gamma^a)^\dagger = (+i)(C\gamma^a)^T = -iC\gamma^a$. A Hermitian matrix K gives a real number $\Psi^\dagger K \Psi$ for every complex column Ψ of ordinary numbers (its complex conjugate is $\Psi^\dagger K^\dagger \Psi$, the same number). C is real and symmetric, hence Hermitian too, so S and the eight currents are real.

statement	**status**	**where it is verified**
$C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$, the author's sigma16, equal to $\operatorname{diag}(-\sigma, \sigma)$	PROVED	`wolfram-algebra.json`, check `C_definition`; `python-algebra.json`, check `C_equals_notebook_sigma16`
(C2), (C3): $C^T = C$ and $CC = 1$	PROVED	`python-algebra.json`, check `C_real_symmetric_involution`; `wolfram-algebra.json`, checks `C_real_symmetric` and `C_squared_identity`
(C4): every $C\gamma^a$ is antisymmetric	PROVED	`python-algebra.json`, check `C_gamma_antisymmetric`; `wolfram-algebra.json`, check `C_gamma_real_antisymmetric`

statement	status	where it is verified
(C5): $C\gamma^a C^{-1} = -(\gamma^a)^T$	PROVED	`python-algebra.json`, check `C_conjugation`; `wolfram-algebra.json`, check `C_gamma_C_inverse`
(C1) and (C6): the commutation signs, the signature $(8,8)$, the form along two paths	PROVED above; COMPUTED in Notebook 05a (exactly, and to 10^{-14} for the form)	Notebook 05a, its sections 6 and 7 (the notebook's own computation)

Notebook 05a re-checks every row of this table.

5.5 The chirality and the two halves

Definition. The **chirality** is the product of all eight gammas in the author's order,

$$\Gamma = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}\gamma^{(x4)}\gamma^{(x5)}\gamma^{(x6)}\gamma^{(x7)}.$$

(The author calls it T16[8]; the label 8 is a name, not a ninth direction.) Its properties:

(X1) $\Gamma\Gamma = 1$**.** Rule 2 with $k = 8$, four space-like and four time-like factors: $\Gamma\Gamma = (-1)^{28}(+1)^4(-1)^4\,1 = 1$.

(X2) Γ **anticommutes with every gamma.** Every direction is one of the eight factors, so Rule 1 gives $\gamma^a\Gamma = (-1)^7\Gamma\gamma^a = -\Gamma\gamma^a$.

(X3) Γ **commutes with every product of an even number of gammas and anticommutes with every product of an odd number.** Moving Γ through a product of j gammas (different or not) passes j factors, each costing a sign by (X2): $\Gamma E = (-1)^j E\Gamma$. A product of an even number of gammas is called **even**, one of an odd number **odd**.

(X4) $\Gamma = C\gamma^{(x4)}\gamma^{(x5)}\gamma^{(x6)}\gamma^{(x7)}$**.** The first four factors of Γ are C.

(X5) $C\Gamma = \Gamma C$**.** C is even (four factors); use (X3).

(X6) Γ **is symmetric.** Rule 4 with $k = 8$: $\Gamma^T = (-1)^{28}(+1)^4(-1)^4\,\Gamma = \Gamma$.

(X7) $\Gamma^T C\Gamma = C$**.** Line by line: $\Gamma^T C\Gamma = \Gamma C\Gamma$ by (X6); $= C\Gamma\Gamma$ by (X5); $= C$ by (X1).

(X8) $\Gamma^T C\gamma^a\Gamma = -C\gamma^a$ **for every** a**.** Line by line: $\Gamma^T C\gamma^a\Gamma = \Gamma C\gamma^a\Gamma$ by (X6); $= C\Gamma\gamma^a\Gamma$ by (X5); $= -C\gamma^a\Gamma\Gamma$ by (X2); $= -C\gamma^a$ by (X1).

(X9) $\Gamma = \mathrm{diag}(-1_8, 1_8)$**.** For the author's gammas the product is diagonal: -1 on the first eight diagonal places and $+1$ on the last eight (the table of Section 5.6, column Γ: row r points to column r, with the sign $-$ for $r \le 8$). This agrees with Rule 6: eight eigenvalues -1 and eight $+1$.

The chiral projectors. A **projector** is a matrix with $PP = P$. Put

$$P_- = \tfrac{1}{2}(1 - \Gamma), \qquad P_+ = \tfrac{1}{2}(1 + \Gamma).$$

Then, multiplying out and using (X1),

$$P_-P_- = \tfrac{1}{4}(1 - 2\Gamma + \Gamma\Gamma) = \tfrac{1}{4}(2 - 2\Gamma) = P_-,$$

and in the same way $P_+P_+ = P_+$, while $P_-P_+ = \frac{1}{4}(1 - \Gamma\Gamma) = 0$ and $P_- + P_+ = 1$. By (X9), $P_- = \mathrm{diag}(1_8, 0)$ keeps the components 1 to 8 of a column and sets the others to zero, and $P_+ = \mathrm{diag}(0, 1_8)$ keeps the components 9 to 16. These are the two **chiral halves**: on the first half $\Gamma = -1$, on the second $\Gamma = +1$.

Every gamma exchanges the halves. Using (X2),

$$\gamma^a P_- = \tfrac{1}{2}(\gamma^a - \gamma^a\Gamma) = \tfrac{1}{2}(\gamma^a + \Gamma\gamma^a) = P_+\gamma^a.$$

So if a column Ψ lives in the first half ($P_-\Psi = \Psi$), then $\gamma^a\Psi = \gamma^a P_-\Psi = P_+\gamma^a\Psi$ lives in the second half, and the same with the halves exchanged. In the table of Section 5.2 this is visible: in every gamma, rows 1 to 8 point to columns 9 to 16 and rows 9 to 16 to columns 1 to 8.

The block rule. Cut a 16×16 matrix into four 8×8 blocks, $M = \begin{pmatrix} W & X \\ Y & Z \end{pmatrix}$. With $\Gamma = \mathrm{diag}(-1_8, 1_8)$, multiplying from the left changes the sign of the top rows and from the right the sign of the left columns:

$$\Gamma M = \begin{pmatrix} -W & -X \\ Y & Z \end{pmatrix}, \qquad M\Gamma = \begin{pmatrix} -W & X \\ -Y & Z \end{pmatrix}.$$

So M commutes with Γ exactly when $X = Y = 0$ (**block diagonal**), and anticommutes with it exactly when $W = Z = 0$ (**block off-diagonal**). With (X3): every even product of gammas is block diagonal and every odd one block off-diagonal.

Why (X7) and (X8) matter. Map a field Ψ to $\Gamma\Psi$. Because Γ is real, $(\Gamma\Psi)^\dagger = \Psi^\dagger\Gamma^T$, and so

$$(\Gamma\Psi)^\dagger C(\Gamma\Psi) = \Psi^\dagger\Gamma^T C\Gamma\Psi = \Psi^\dagger C\Psi,$$

by (X7): the scalar S is unchanged. By (X8) every bilinear $\Psi^\dagger C\gamma^a\Psi$ changes its sign, and so does every $\Psi^\dagger C\gamma^a E\Psi$ with an even product E (move Γ through E first, which costs nothing). The kinetic term of the Lagrangian of Chapter 7 is of this form, so the map $\Psi \to \Gamma\Psi$ keeps the mass term and reverses the kinetic term. These matrix identities are the input of the pairing theorem T1, which Chapter 18 proves: the map Γ carries the solutions with the parameters (m, λ) to the solutions with $(-m, -\lambda)$. T1 is an exact map between two sets of solutions; it does not say that anything is created (Chapter 20).

statement	**status**	**where it is verified**
(X9): $\Gamma = \mathrm{diag}(-1_8, 1_8)$, the author's T16[8]	PROVED	`python-algebra.json`, check `chirality_diag`; `wolfram-algebra.json`, checks `Gamma_definition` and `Gamma_diag`

statement	status	where it is verified
(X1), (X2): $\Gamma\Gamma = 1$, and Γ anticommutes with every gamma	PROVED	`python-algebra.json`, check `chirality_anticommutes`; `wolfram-algebra.json`, check `Gamma_anticommutes_with_gammas`
(X4): $\Gamma = C\gamma^{(x4)}\gamma^{(x5)}\gamma^{(x6)}\gamma^{(x7)}$	PROVED	`python-algebra.json`, check `chirality_eq_C_times_time_gammas`
(X5) to (X8)	PROVED	`python-algebra.json`, check `chirality_C_relation`; `wolfram-algebra.json`, check `Gamma_commutes_with_C`
every gamma exchanges the two halves	PROVED	`python-algebra.json`, check `reflections_exchange_halves`; `wolfram-algebra.json`, check `Spin_Pin_reflection_swaps_halves`

Notebook 05a re-checks every row of this table.

5.6 The matrix B and the charge density

Definition. $B = -i\,C\gamma^{(x4)} = -i\,\gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}\gamma^{(x4)}$, where i is the imaginary unit ($i^2 = -1$).

(B1) B is purely imaginary. $C\gamma^{(x4)}$ is a product of real matrices, so it is real, and B is $-i$ times it.

(B2) B is Hermitian. The conjugate transpose of B conjugates the number $-i$ to $+i$ and transposes the real matrix:

$$B^\dagger = (+i)\,(C\gamma^{(x4)})^T = (+i)(-C\gamma^{(x4)}) = -i\,C\gamma^{(x4)} = B,$$

where the second step is (C4) for $a = x4$.

(B3) $BB = 1$. First $BB = (-i)^2\,(C\gamma^{(x4)})(C\gamma^{(x4)}) = -(C\gamma^{(x4)})^2$. The matrix $C\gamma^{(x4)}$ is the product of the five different gammas of $x0, x1, x2, x3, x4$, four space-like and one time-like, so Rule 2 gives $(C\gamma^{(x4)})^2 = (-1)^{10}(+1)^4(-1)\,1 = -1$. Hence $BB = -(-1) = 1$.

(B4) B has signature (8, 8). By Rule 5 (with $k = 5$) $\operatorname{tr} B = 0$, and with $BB = 1$ Rule 6 gives eight eigenvalues $+1$ and eight -1; the characteristic polynomial is $\det(\lambda 1 - B) = (\lambda - 1)^8(\lambda + 1)^8$. As B is Hermitian, the number $\Psi^\dagger B\Psi$ is real for every complex column, and it takes both signs.

(B5) B commutes with five gammas and anticommutes with three. Rule 1 with the five factors $x0, x1, x2, x3, x4$: a gamma among them passes four others, sign $(-1)^4 = +1$; a gamma not among them ($x5$, $x6$, $x7$) passes five, sign $(-1)^5 = -1$. The number $-i$ commutes with every matrix. So B commutes with $\gamma^{(x0)}, \gamma^{(x1)}, \gamma^{(x2)}, \gamma^{(x3)}, \gamma^{(x4)}$ and anticommutes with the gammas of the three extra times.

What B is for. The time component of the current of Section 5.4 is

$$J^{(x4)} = \Psi^\dagger(-iC\gamma^{(x4)})\Psi = \Psi^\dagger B\Psi,$$

the **charge density** of the field. The Revision record proves a **local conservation law**: for every solution of the field equation,

$$\sum_\mu \partial_\mu(\sqrt{|g|}\, J^\mu) = 0 \qquad \text{at every point,}$$

where ∂_μ is the partial derivative along the coordinate μ (the sum runs over $x0, \ldots, x7$), $z = 6Hx0$, $\sqrt{|g|} = b_4^{3/2}\cos z$ is the volume factor of the author's metric (Chapter 3), and $J^\mu = -i\bar{\Psi}\gamma^\mu\Psi$ is the current built with the gammas γ^μ of the curved space (Section 5.28 and Chapter 6; for the time $\gamma^{x4} = \gamma^{(x4)}$, because $g_{44} = -1$, so $J^{x4} = \Psi^\dagger B\Psi$). The record checks this law for a general $b_4(x4)$ (check `u1_noether_matrix_identity_b4` of `Revision/theory/reports/python-field-theory-b4.json`) and, independently, at $b_4 = 1$, where $\sqrt{|g|} = \cos z$ (the lead check `u1_noether_matrix_identity`). The backgrounds of this book have $b_4 = 1$ (Chapter 3, Section 3.8), and the rest of this section writes $\cos z$; for a general b_4, which depends on $x4$ only, the factor $b_4^{3/2}$ multiplies the total charge below at each time. The **total charge** is the integral of the charge density over the seven directions other than the time,

$$Q = \int \cos z\ \Psi^\dagger B\Psi\ d^7x.$$

Line by line: the derivative of Q with respect to the time $x4$ is the integral of $\partial_{x4}(\cos z\, J^{x4})$; by the local law this equals minus the integral of the seven other terms $\partial_\mu(\cos z\, J^\mu)$, $\mu \neq x4$; and the integral of a derivative along one direction is the difference of the values at the two ends of that direction (the fundamental theorem of calculus). So $dQ/dx4$ equals minus the **flux** of the current through the boundary of the seven other directions. Hence Q does not change in time **for every solution whose current flux through that boundary vanishes**. The condition is needed: the local law alone does not make the flux zero. The seven directions are the unbounded directions of 3-space and of the extra times and the hidden interval $0 < z < \pi/2$, which has two ends. At the end $z \to 0$, the **tip**, the metric is singular ($g_{00} = \cot^2 z$ grows without bound). The other end, $z = \pi/2$, is the **brane** (the edge of the hidden direction, at a finite distance, where $g_{00} = \cot^2 z = 0$). The part of the flux that leaves through these two ends is $\cos z\, J^{x0}$ at $z = \pi/2$ minus its value at $z \to 0$, added up over the six other directions. The Revision record states that this boundary term vanishes at the tip $z \to 0$ but not, in general, at the brane $z = \pi/2$ (`Revision/theory/reports/python-scope.json`, check `good_sector_hermiticity_up_to_the_brane_flux`), and the record's check of the local law says the same: it proves the local law only, and the constancy of Q is not established by it. Chapter 18 (Section 18.21) derives a family of exact solutions, the fields that depend only on the time. Some of them, for example the one of Notebook 18c, send charge through the brane: for these fields the flow $\cos z\, J^{x0}$ at the brane equals a number q_0 that changes in time, and Notebook 18c finds it between -0.3454 and 0.3454, so that the charge of a region that spans the whole hidden interval changes in time. Chapter 10 (Sections 10.27 and 10.28) shows another exact solution whose charge grows. Not every member of the family of Section 18.21 sends charge through the brane. That section writes the field equation of these fields as $d\Psi/dx4 = M\Psi$ with a constant matrix M whose square, in the oscillating case of that section, is $-w^2$ times the unit

matrix ($w > 0$ a number), and its solution in that case as $\Psi(x4) = \cos(wx4)\,\Psi(0) + \frac{\sin(wx4)}{w}\,M\Psi(0)$. Take any column v for which $\chi = \frac{1}{2}(v - \frac{i}{w}Mv)$ is not zero, and start from $\Psi(0) = \chi$. Line by line: $M\chi = \frac{1}{2}(Mv - \frac{i}{w}M^2v) = \frac{1}{2}(Mv + iwv)$, because $M^2 = -w^2$ and $-\frac{i}{w}(-w^2) = iw$; and $iw\chi = \frac{1}{2}(iwv - \frac{i\cdot i\,w}{w}Mv) = \frac{1}{2}(iwv + Mv)$, because $i \cdot i = -1$; the two agree, so $M\chi = iw\chi$; inserting $M\chi = iw\chi$ into the solution gives $\Psi(x4) = \big(\cos(wx4) + i\sin(wx4)\big)\chi = e^{iwx4}\chi$, a **stationary** solution (a fixed column times a phase factor); its charge density is $\Psi^\dagger B\Psi = e^{-iwx4}e^{iwx4}\,\chi^\dagger B\chi = \chi^\dagger B\chi$, the same at every time; the field depends only on $x4$, so in the local law every derivative except those along $x4$ and $x0$ is zero, and the law reads $\partial_{x4}(\cos z\, J^{x4}) = -\partial_{x0}(\cos z\, J^{x0})$; the left side is zero, because $\cos z$ does not depend on $x4$ and J^{x4} does not change in time; so $\cos z\, J^{x0}$ has the same value at every point of the hidden interval; it is zero at the tip (the record's boundary term vanishes there), hence it is zero at the brane. So a constant total charge needs a **no-flux condition** at the brane, $\cos z\, J^{x0} = 0$ at $z = \pi/2$, which holds for some solutions (the stationary ones above) and fails for others (the one of Notebook 18c). No equation of the Revision record or of this book derives it for every solution, and the field equations and the quantisation of the record impose no condition at the brane (OPEN). The Kohn-Sham model of Chapter 14 ASSUMES one: a **Z2 mirror** at the brane (the hidden interval is glued at the brane to a mirror copy of itself), whose two conditions make the current along the hidden direction zero there (`Revision/kohn_sham/ks-theory.json`, entry `boundaryConditions.brane`, status ASSUMED). Wherever a constant total charge is used, this condition is ASSUMED. The status table at the end of this section names the checks; Chapter 21 derives the identity and the boundary terms. Because B has eight positive and eight negative eigenvalues, the charge density can be positive or negative: the field can carry charge of both signs. In the quantum theory B is the matrix of the canonical anticommutator, and its indefinite signature forces an indefinite (Krein) inner product (Section 5.34 and Chapter 10).

The matrices C, Γ, B and ΓC in one table. Each is a signed permutation matrix (for B, after division by i); the table is read like the one of Section 5.2. C, Γ and B are stored in the record `Revision/algebra/gammas.json`; ΓC (the charge-conjugation matrix $\mathcal{C}_-$ of Section 5.28) is C with the signs of rows 1 to 8 reversed, as Notebook 05c computes.

row	C	Γ	B/i	ΓC
1	−5	−1	+10	+5
2	−6	−2	−9	+6
3	−7	−3	−12	+7
4	−8	−4	+11	+8
5	−1	−5	−14	+1
6	−2	−6	+13	+2
7	−3	−7	+16	+3
8	−4	−8	−15	+4
9	+13	+9	+2	+13
10	+14	+10	−1	+14
11	+15	+11	−4	+15
12	+16	+12	+3	+16
13	+9	+13	−6	+9
14	+10	+14	+5	+10
15	+11	+15	+8	+11
16	+12	+16	−7	+12

Reading the table: C maps row 1 to column 5 with the sign −, so $C_{1,5} = -1$, the entry used in (C6); B/i has +10 in row 1 and −1 in row 10, so $B_{1,10} = +i$ and $B_{10,1} = -i = (B_{1,10})^*$, as a Hermitian

matrix requires; ΓC has $+\sigma$ in both diagonal blocks.

statement	status	where it is verified
(B1): $B = -iC\gamma^{(x4)}$ is purely imaginary	PROVED	`wolfram-algebra.json`, checks `B_definition` and `B_purely_imaginary`
(B2) to (B4): B is Hermitian, $BB = 1$, $\mathrm{tr}\, B = 0$, characteristic polynomial $(\lambda-1)^8(\lambda+1)^8$	PROVED	`python-algebra.json`, check `B_hermitian_involution_signature`; `wolfram-algebra.json`, checks `B_Hermitian`, `B_squared_identity` and `B_signature_8_8`
(B5): the commutation signs of B	PROVED	`python-algebra.json`, check `B_gamma_relations`
the local law $\sum_\mu \partial_\mu(\sqrt{\lvert g\rvert}\, J^\mu) = 0$ for every solution, with $\sqrt{\lvert g\rvert} = b_4^{3/2}\cos z$ ($\cos z$ at $b_4 = 1$)	PROVED in the record (derived in Chapter 21)	`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity` ($b_4 = 1$); `Revision/theory/reports/python-field-theory-b4.json`, check `u1_noether_matrix_identity_b4` (general b_4)
the total charge Q is constant in time for every solution whose current flux through the boundary of the seven other directions vanishes	PROVED (a conditional statement, from the local law, above; the boundary terms in Chapter 21)	the same check, together with the fundamental theorem of calculus
the flux through the brane $z = \pi/2$ vanishes (the no-flux condition), so that Q is constant	OPEN: not derived for every solution; it fails for exact solutions of Section 18.21, for example the one of Notebook 18c, whose flow through the brane ranges from -0.3454 to 0.3454, and it holds for the stationary members of that family (PROVED above); ASSUMED in the Kohn-Sham model of Chapter 14 (the Z2 mirror) and wherever a constant total charge is used	`Revision/theory/reports/python-scope.json`, check `good_sector_hermiticity_up_to_the_brane_flux`; the detail text of the check `u1_noether_matrix_identity` (the constancy of Q is not established); Notebook 18c; `Revision/kohn_sham/ks-theory.json`, entry `boundaryConditions.brane` (status ASSUMED)

Notebook 05a re-checks the first three rows.

5.7 Example: Notebook 05a checks C, Γ and B

Notebook 05a reads the eight gammas from the record `Revision/algebra/gammas.json` as exact whole numbers and checks, one by one, every statement of Sections 5.2 to 5.6. It reads the two reports of the record and, for every check that repeats a recorded check, prints a second line naming the report file and the check. It tests the squaring rule of Section 5.3 on all 255 products of different gammas, evaluates the quadratic form of C along two paths, computes the characteristic polynomial of B exactly with sympy, counts the eigenvalues of all eleven matrices and draws six teaching figures: heat maps of C and σ, the two values of the quadratic form, heat maps of Γ, its projectors and $\gamma^{(x1)}$, heat maps of the matrices that make B, the table of eigenvalue counts and the table of commutation signs. Its last line is ALL 26 CHECKS PASSED (notebook 05a).

5.8 How to run Notebook 05a

Step 1. What this notebook does and what it needs.

Notebook 05a (The matrices C, Gamma and B built from the author's gammas) is the file `Revision/textbook/notebooks/05a_c_gamma_b.ipynb` of the repository Dirac_claude. It reads the eight real 16 by 16 gamma matrices of the Revision record, checks the Clifford relation, builds the charge matrix C (the product of the four space-like gammas), the chirality Gamma (the product of all eight gammas) and the matrix B = -i C gamma^(x4), checks again with its own code every property that the Revision record states about them (symmetry, squares, commutation signs, eigenvalues and signature; COMPUTED here, partly in exact whole-number arithmetic and partly in floating-point numbers, while the record proves them exactly), and draws six teaching plots: heat maps of the matrices, an indefinite quadratic form, the spectra and a table of commutation signs. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05a_c_gamma_b.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05a_c_gamma_b.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 05a_c_gamma_b.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/05a.captions.json`
- `Revision/textbook/figures/05a_1_c_matrix.png`
- `Revision/textbook/figures/05a_2_c_form_signs.png`
- `Revision/textbook/figures/05a_3_chirality.png`
- `Revision/textbook/figures/05a_4_b_matrix.png`
- `Revision/textbook/figures/05a_5_spectra.png`
- `Revision/textbook/figures/05a_6_commutation_signs.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 05a exist
ALL 26 CHECKS PASSED (notebook 05a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/05a_c_gamma_b.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 05a_c_gamma_b.ipynb
python -m nbconvert --execute --inplace 05a_c_gamma_b.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming `Revision/algebra/gammas.json`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

5.9 Notebook 05a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/05a_c_gamma_b.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The author's theory uses eight real $16 \times 16$ matrices, the *gamma matrices*
$\gamma^{(x0)}, \dots, \gamma^{(x7)}$, one for each coordinate direction. From them
the theory builds three further matrices that appear in every formula of the book:

- the **charge matrix** $C = \gamma^{(x0)} \gamma^{(x1)} \gamma^{(x2)} \gamma^{(x3)}$,
  the product of the four space-like gammas (the author calls it sigma16);
- the **chirality** $\Gamma = \gamma^{(x0)} \gamma^{(x1)} \gamma^{(x2)} \cdots
  \gamma^{(x7)}$, the product of all eight gammas;
- the matrix $B = -i\, C \gamma^{(x4)}$, where $i$ is the imaginary unit
  ($i^2 = -1$).

This notebook reads the gammas from the Revision record
`Revision/algebra/gammas.json`, checks the Clifford relation, builds $C$, $\Gamma$
and $B$, and checks every property that the Revision reports
`Revision/algebra/reports/python-algebra.json` and
`Revision/algebra/reports/wolfram-algebra.json` record about them: which of them are
symmetric, what their squares are, with which gammas they commute or anticommute,
their eigenvalues, and the *signature* $(8, 8)$ of $C$ and $B$. Every check that
repeats a recorded check prints the record file and the check name. It draws six
teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 05a (The matrices C, Gamma and B built from the author's gammas) is the file
'Revision/textbook/notebooks/05a_c_gamma_b.ipynb' of the repository Dirac_claude. It
reads the eight real 16 by 16 gamma matrices of the Revision record, checks the Clifford
relation, builds the charge matrix C (the product of the four space-like gammas), the
chirality Gamma (the product of all eight gammas) and the matrix B = -i C gamma^(x4),
checks again with its own code every property that the Revision record states about them
(symmetry, squares, commutation signs, eigenvalues and signature; COMPUTED here, partly
in exact whole-number arithmetic and partly in floating-point numbers, while the record
proves them exactly), and draws six teaching plots: heat maps of the matrices, an
indefinite quadratic form, the spectra and a table of commutation signs. It needs a
computer with Windows 11, macOS or Linux, an internet connection for the installation,
the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv

Then close the terminal and open a NEW one (a terminal opened before the installation
does not know the new programs), and check the version of Python; it must print 3.12 or a
higher number:
```

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
```

```
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 05a_c_gamma_b.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05a_c_gamma_b.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 05a_c_gamma_b.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/05a.captions.json‘
- ‘Revision/textbook/figures/05a_1_c_matrix.png‘
- ‘Revision/textbook/figures/05a_2_c_form_signs.png‘
- ‘Revision/textbook/figures/05a_3_chirality.png‘
- ‘Revision/textbook/figures/05a_4_b_matrix.png‘
- ‘Revision/textbook/figures/05a_5_spectra.png‘
- ‘Revision/textbook/figures/05a_6_commutation_signs.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 05a exist
ALL 26 CHECKS PASSED (notebook 05a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
    python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from
  a copy outside the repository. Open the file
  ‘Revision/textbook/notebooks/05a_c_gamma_b.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 05a_c_gamma_b.ipynb
      python -m nbconvert --execute --inplace 05a_c_gamma_b.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming ‘Revision/algebra/gammas.json‘: the notebook was opened
  outside the repository, or the repository is incomplete; clone the repository again and
  open the notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 05a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 05a (The matrices C, Gamma and B built from the author’s gammas) is the file
# Revision/textbook/notebooks/05a_c_gamma_b.ipynb of the repository Dirac_claude. It
# reads the eight real 16 by 16 gamma matrices of the Revision record, checks the
# Clifford relation, builds the charge matrix C (the product of the four space-like
```

```
# gammas), the chirality Gamma (the product of all eight gammas) and the matrix B = -i C
# gamma^(x4), checks again with its own code every property that the Revision record
# states about them (symmetry, squares, commutation signs, eigenvalues and signature;
# COMPUTED here, partly in exact whole-number arithmetic and partly in floating-point
# numbers, while the record proves them exactly), and draws six teaching plots: heat maps
# of the matrices, an indefinite quadratic form, the spectra and a table of commutation
# signs. It needs a computer with Windows 11, macOS or Linux, an internet connection for
# the installation, the program Git, and Python 3.12 or newer (the notebooks were built
# with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
```

```
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
```

```
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 05a_c_gamma_b.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 05a_c_gamma_b.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 05a_c_gamma_b.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
```

```
# - Revision/textbook/figures/05a.captions.json
# - Revision/textbook/figures/05a_1_c_matrix.png
# - Revision/textbook/figures/05a_2_c_form_signs.png
# - Revision/textbook/figures/05a_3_chirality.png
# - Revision/textbook/figures/05a_4_b_matrix.png
# - Revision/textbook/figures/05a_5_spectra.png
# - Revision/textbook/figures/05a_6_commutation_signs.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the six figure files of notebook 05a exist
#     ALL 26 CHECKS PASSED (notebook 05a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/05a_c_gamma_b.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 05a_c_gamma_b.ipynb
#     python -m nbconvert --execute --inplace 05a_c_gamma_b.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
```

```
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming Revision/algebra/gammas.json: the notebook was opened
#   outside the repository, or the repository is incomplete; clone the repository again
#   and open the notebook from its folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "05a"  # this notebook: chapter 05, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
```

```
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 05a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Matrix**: a rectangular table of numbers. A $16 \times 16$ matrix has 16 rows and
  16 columns. The entry in row $r$ and column $c$ of the matrix $M$ is written
  $M_{rc}$; rows and columns are numbered 1 to 16 in the text and in the plots.
- **Matrix product**: $(MN)_{rc} = \sum_k M_{rk} N_{kc}$. In Python the sign ‘@‘
  multiplies two matrices. The order matters: $MN$ and $NM$ are in general
  different.
- **Identity matrix** $1$: ones on the diagonal, zeros elsewhere; $1M = M1 = M$.
- **Commute, anticommute**: $M$ and $N$ *commute* when $MN = NM$ and *anticommute*
  when $MN = -NM$.
- **Anticommutator**: $\{M, N\} = MN + NM$.
- **Transpose** $M^T$: the matrix with rows and columns exchanged,
  $(M^T)_{rc} = M_{cr}$. $M$ is **symmetric** when $M^T = M$ and **antisymmetric**
  when $M^T = -M$.
- **Inverse** $M^{-1}$: the matrix with $M M^{-1} = M^{-1} M = 1$. When $MM = 1$,
  the matrix is its own inverse.
- **Complex conjugate**: for a complex number $z = a + ib$ ($a$, $b$ real) it is
  $z^* = a - ib$. For a matrix, $M^*$ conjugates every entry, and the
  **conjugate transpose** is $M^\dagger = (M^*)^T$. A matrix with $M^\dagger = M$
```

```
  is **Hermitian**.
- **Eigenvalue, eigenvector**: a number $\lambda$ and a column $v \neq 0$ with
  $Mv = \lambda v$. The list of all eigenvalues (counted with their multiplicity)
  is the **spectrum** of $M$.
- **Trace** $\mathrm{tr}\, M$: the sum of the diagonal entries; it equals the sum
  of the eigenvalues.
- **Signature** $(p, q)$ of a symmetric or Hermitian matrix: it has $p$ positive
  and $q$ negative eigenvalues. A signature with both $p > 0$ and $q > 0$ is
  *indefinite*: the quadratic form $u^T M u$ (or $u^\dagger M u$) takes both
  signs.
- **Signed permutation matrix**: exactly one nonzero entry, $+1$ or $-1$, in each
  row and in each column.
- **Block**: a $16 \times 16$ matrix cut into four $8 \times 8$ pieces; rows and
  columns 1 to 8 form the first half, 9 to 16 the second half.
  $\mathrm{diag}(X, Y)$ is the matrix with the blocks $X$ (top left) and $Y$
  (bottom right) and zeros elsewhere.
- **Projector**: a matrix with $PP = P$.
- **Heat map**: a picture of a matrix in which every entry is a coloured square; in
  this notebook $-1$ is blue, $0$ light grey and $+1$ red.
- **Record, check**: a Revision record is a file of the repository written by the
  Revision programs; a *check* in it has a name, a verdict (pass or fail) and a
  detail text.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's spacetime has eight coordinates $x0, \dots, x7$: $x1, x2, x3$ are
ordinary 3-space, $x4$ is the time, $x5, x6, x7$ are three extra times (they deflate
exponentially in the author's metric) and $x0$ is a hidden space direction. The
flat metric of the frame is

$$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$$

in the order $x0, \dots, x7$: the four directions with $\eta_{aa} = +1$ ($x1, x2,
x3, x0$) are *space-like*, the four with $\eta_{aa} = -1$ ($x4, x5, x6, x7$) are
*time-like*. The gamma matrices obey the **Clifford relation**

$$\{\gamma^a, \gamma^b\} = 2 \eta^{ab}\, 1 ,$$

that is: $\gamma^a \gamma^a = \eta_{aa} 1$ (a space-like gamma squares to $+1$, a
time-like one to $-1$), and two different gammas anticommute.

**The sign rule for moving a gamma.** Let $P$ be a product of $k$ *different*
gammas. Move $\gamma^a$ from the left of $P$ to its right, one factor at a time.
Passing a factor different from $\gamma^a$ costs a sign $-1$ (they anticommute);
passing $\gamma^a$ itself costs nothing. Hence

- if $\gamma^a$ is one of the $k$ factors: $\gamma^a P = (-1)^{k-1} P \gamma^a$;
- if it is not: $\gamma^a P = (-1)^{k} P \gamma^a$.

For $C$ ($k = 4$, factors $x0, x1, x2, x3$): a space-like gamma is a factor and
gives $(-1)^3 = -1$ (anticommutes); a time-like gamma is not a factor and gives
```

```
$(-1)^4 = +1$ (commutes). In one formula: $C\gamma^a = -\eta_{aa}\, \gamma^a C$.

**The sign rule for squaring a product.** Write
$P = \gamma^{a_1} \cdots \gamma^{a_k}$ with different $a_j$. In $PP$, move the
second copy of $\gamma^{a_1}$ to the left until it meets the first copy: it passes
$k - 1$ different factors, giving $(-1)^{k-1}$, and then
$\gamma^{a_1}\gamma^{a_1} = \eta_{a_1 a_1}$. Repeating with $\gamma^{a_2}$
($k - 2$ passes), and so on, gives

$$P P = (-1)^{(k-1) + (k-2) + \dots + 0}\ \eta_{a_1 a_1} \cdots \eta_{a_k a_k}\, 1
    = (-1)^{k(k-1)/2}\ \eta_{a_1 a_1} \cdots \eta_{a_k a_k}\, 1 .$$

For $C$: $(-1)^{6} (+1)^4 = +1$, so $CC = 1$. For $\Gamma$ ($k = 8$): $(-1)^{28}
(+1)^4 (-1)^4 = +1$, so $\Gamma\Gamma = 1$. For $C\gamma^{(x4)}$ ($k = 5$):
$(-1)^{10} (+1)^4 (-1) = -1$, so $BB = (-i)^2 (C\gamma^{(x4)})^2 = (-1)(-1) = 1$.

**Why a matrix with $MM = 1$ and trace 0 has eight eigenvalues $+1$ and eight
$-1$.** If $Mv = \lambda v$, then $v = MMv = \lambda^2 v$, so $\lambda = \pm 1$.
The trace is the sum of the 16 eigenvalues: $n_+ - n_- = 0$ and $n_+ + n_- = 16$
give $n_+ = n_- = 8$. For a time-like gamma, $\gamma\gamma = -1$ gives
$\lambda^2 = -1$, so $\lambda = \pm i$, again eight of each.

The notebook checks each of these statements on the actual matrices.
```

Text cell:

```
## 5. The eight gamma matrices from the Revision record

The next cell reads the record ‘Revision/algebra/gammas.json‘. It holds the gammas in
the order $x0, \dots, x7$ as lists of rows of whole numbers. ‘np.array‘ turns each
list into a numpy matrix of whole numbers (‘int64‘), so every product below is
computed **exactly** (no rounding). The dictionary ‘gamma‘ gives each matrix by the
name of its coordinate: ‘gamma["x4"]‘ is $\gamma^{(x4)}$.
```

In [2]:

```
import numpy as np  # arrays of numbers, matrices and linear algebra

FIXTURE = "Revision/algebra/gammas.json"  # the Revision record of the gammas
fixture = json.loads(repository_file(FIXTURE).read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # the coordinate names "x0", "x1", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # eta_aa: +1 space-like, -1 time-like
# gamma["x0"], ..., gamma["x7"]: the eight 16 x 16 matrices of whole numbers
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the 16 x 16 identity matrix, written 1 in the text
for x in COORDS:
    kind = "space-like" if ETA[x] == 1 else "time-like"
    rows, columns = gamma[x].shape  # the numbers of rows and of columns
    say(f"gamma^({x}): {rows} x {columns}, eta = {ETA[x]:+d} ({kind})")
```

Out [2]:

```
gamma^(x0): 16 x 16, eta = +1 (space-like)
gamma^(x1): 16 x 16, eta = +1 (space-like)
gamma^(x2): 16 x 16, eta = +1 (space-like)
gamma^(x3): 16 x 16, eta = +1 (space-like)
gamma^(x4): 16 x 16, eta = -1 (time-like)
gamma^(x5): 16 x 16, eta = -1 (time-like)
gamma^(x6): 16 x 16, eta = -1 (time-like)
gamma^(x7): 16 x 16, eta = -1 (time-like)
```

Text cell:

```
The next cell reads the two Revision reports whose checks this notebook repeats, and
defines three small helpers: ‘recorded(key, name)‘ is true when the report ‘key‘
("python" or "wolfram") holds the check ‘name‘ with the verdict pass;
‘record_of(key, name)‘ is the text that a check prints after the word
"reproduces"; and ‘check_reproduces(condition, name, record)‘ is the helper ‘check‘
for a check that reproduces a Revision record. It lets ‘check‘ print the PASS line
and the line "reproduces ..." into a text buffer (‘contextlib.redirect_stdout‘) and
then sends both lines with one ‘sys.stdout.write‘: Jupyter delivers printed text in
pieces, and one piece keeps the two lines together, so that the tools that read the
notebook always see them as one check. The first check confirms the order of the
coordinates and the signs of $\eta$.
```

In [3]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    """True when the report key records the check name with the verdict pass."""
    return VERDICTS[(key, name)][0] == "pass"


def record_of(key, name):
    """The text printed after "reproduces": the record file and the check name."""
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
```

```
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


say(f"{len(VERDICTS)} recorded checks were read from the two reports.")
check_reproduces(COORDS == ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
                 and fixture["eta"] == [1, 1, 1, 1, -1, -1, -1, -1]
                 and recorded("wolfram", "eta_in_author_order"),
                 "the coordinates are x0..x7 and eta = diag(+1,+1,+1,+1,-1,-1,-1,-1)",
                 record=record_of("wolfram", "eta_in_author_order"))
```

Out [3]:

```
80 recorded checks were read from the two reports.
PASS the coordinates are x0..x7 and eta = diag(+1,+1,+1,+1,-1,-1,-1,-1)
     reproduces Revision/algebra/reports/wolfram-algebra.json, check eta_in_author_order
```

Text cell:

```
The next cell checks the Clifford relation $\{\gamma^a, \gamma^b\} = 2\eta^{ab} 1$
for all $8 \times 8 = 64$ ordered pairs: for $a = b$ the right side is
$2\eta_{aa} 1$, for $a \neq b$ it is the zero matrix. ‘np.array_equal‘ compares two
matrices entry by entry and is true only when all 256 entries agree.
```

In [4]:

```
def anticommutator(m, n):
    """{m, n} = m n + n m (the sign @ multiplies two matrices)."""
    return m @ n + n @ m


failures = []  # the pairs (x, y) for which the relation fails
for x in COORDS:
    for y in COORDS:
        # the right side: 2 eta_xx 1 when x = y, the zero matrix otherwise
        expected = 2 * ETA[x] * I16 if x == y else np.zeros_like(I16)
        if not np.array_equal(anticommutator(gamma[x], gamma[y]), expected):
            failures.append((x, y))
say(f"pairs tested: {len(COORDS) ** 2}; pairs that fail: {len(failures)}")
check_reproduces(failures == [] and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

Out [4]:

```
pairs tested: 64; pairs that fail: 0
PASS {gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
```

Text cell:

The next cell checks two facts about the shape of the gammas. First, every entry is -1, 0 or $+1$, and each gamma is a signed permutation matrix (‘gamma[x] != 0‘ is a table of true/false values; ‘.sum(axis=1)‘ counts the true values in each row and ‘.sum(axis=0)‘ in each column). Second, the space-like gammas are symmetric and the time-like ones antisymmetric: $(\gamma^a)^T = \eta_{aa} \gamma^a$.

In [5]:

```
values = sorted({int(v) for x in COORDS for v in gamma[x].flat})  # every entry once
one_per_row = all((gamma[x] != 0).sum(axis=1).tolist() == [1] * 16 for x in COORDS)
one_per_column = all((gamma[x] != 0).sum(axis=0).tolist() == [1] * 16 for x in COORDS)
say(f"the entries of the gammas are {values}")
check_reproduces(values == [-1, 0, 1] and one_per_row and one_per_column
                 and recorded("python", "reality_signed_permutations"),
                 "every gamma is real and a signed permutation matrix",
                 record=record_of("python", "reality_signed_permutations"))
symmetric = [x for x in COORDS if np.array_equal(gamma[x].T, gamma[x])]
antisymmetric = [x for x in COORDS if np.array_equal(gamma[x].T, -gamma[x])]
say("symmetric gammas: " + ", ".join(symmetric))
say("antisymmetric gammas: " + ", ".join(antisymmetric))
check_reproduces(symmetric == ["x0", "x1", "x2", "x3"]
                 and antisymmetric == ["x4", "x5", "x6", "x7"]
                 and recorded("python", "symmetry_pattern"),
                 "(gamma^a)^T = eta_aa gamma^a: space-like symmetric, time-like "
                 "antisymmetric",
                 record=record_of("python", "symmetry_pattern"))
```

Out [5]:

```
the entries of the gammas are [-1, 0, 1]
PASS every gamma is real and a signed permutation matrix
     reproduces Revision/algebra/reports/python-algebra.json, check
    reality_signed_permutations
symmetric gammas: x0, x1, x2, x3
antisymmetric gammas: x4, x5, x6, x7
PASS (gamma^a)^T = eta_aa gamma^a: space-like symmetric, time-like antisymmetric
     reproduces Revision/algebra/reports/python-algebra.json, check symmetry_pattern
```

Text cell:

```
## 6. The charge matrix C

The next cell defines the drawing helper ‘heat_map‘, used for every picture of a
matrix in this notebook, and the helper ‘commutation_sign(m, n)‘, which returns
$+1$ when $mn = nm$, $-1$ when $mn = -nm$, and $0$ otherwise. It also defines
‘product(directions)‘, the product of the gammas of a list of directions, in the
order of the list.
```

In [6]:

```
from matplotlib.colors import LinearSegmentedColormap

# Colours of the heat maps: -1 blue, 0 light grey, +1 red (a diverging colour scale).
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])


def heat_map(ax, matrix, title, halves=True, row_label=True):
    """Draw matrix (entries from -1 to +1) as coloured squares on the axes ax.
    Rows and columns are numbered from 1.  With halves=True and a 16 x 16 matrix,
    two thin black lines separate rows and columns 1-8 from 9-16.  row_label=False
    leaves out the word "row" (for the second and later pictures of a row)."""
    size = matrix.shape[0]
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ticks = [0, 3, 7, 11, 15] if size == 16 else [0, 3, 7]  # counted from 0
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])  # labels counted from 1
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)  # no grid lines across the coloured squares
    if halves and size == 16:
        ax.axhline(7.5, color="black", linewidth=0.8)  # between rows 8 and 9
        ax.axvline(7.5, color="black", linewidth=0.8)  # between columns 8 and 9
    return image


def commutation_sign(m, n):
    """+1 if m n = n m, -1 if m n = -n m, 0 if neither."""
    if np.array_equal(m @ n, n @ m):
        return 1
    if np.array_equal(m @ n, -(n @ m)):
        return -1
    return 0


def product(directions):
    """gamma^(d1) gamma^(d2) ... for the directions d1, d2, ... in this order."""
    result = I16
    for d in directions:
        result = result @ gamma[d]
    return result


say("helpers heat_map, commutation_sign and product are defined")
```

Out [6]:

```
helpers heat_map, commutation_sign and product are defined
```

Text cell:

```
The next cell builds $C = \gamma^{(x0)} \gamma^{(x1)} \gamma^{(x2)} \gamma^{(x3)}$
and checks three recorded facts: it equals the matrix $C$ stored in the record; it
equals $\mathrm{diag}(-\sigma, \sigma)$ with the author's $8 \times 8$ matrix
$\sigma = \begin{pmatrix} 0 & 1_4 \\ 1_4 & 0 \end{pmatrix}$ ($1_4$ the
$4 \times 4$ identity); and it is symmetric with $CC = 1$. `np.block` assembles a
matrix from blocks.
```

In [7]:

```
C = product(["x0", "x1", "x2", "x3"])  # the four space-like gammas
C_record = np.array(fixture["C"], dtype=np.int64)  # the C stored in the record
check_reproduces(np.array_equal(C, C_record) and recorded("wolfram", "C_definition"),
                 "C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3) equals the recorded C",
                 record=record_of("wolfram", "C_definition"))
I4, Z4 = np.eye(4, dtype=np.int64), np.zeros((4, 4), dtype=np.int64)
I8, Z8 = np.eye(8, dtype=np.int64), np.zeros((8, 8), dtype=np.int64)
sigma = np.block([[Z4, I4], [I4, Z4]])  # the author's 8 x 8 matrix sigma
check_reproduces(np.array_equal(C, np.block([[-sigma, Z8], [Z8, sigma]]))
                 and recorded("python", "C_equals_notebook_sigma16"),
                 "C = diag(-sigma, sigma), called sigma16 by the author",
                 record=record_of("python", "C_equals_notebook_sigma16"))
check_reproduces(np.array_equal(C.T, C) and np.array_equal(C @ C, I16)
                 and recorded("python", "C_real_symmetric_involution"),
                 "C is real and symmetric, and C C = 1 (so C is its own inverse)",
                 record=record_of("python", "C_real_symmetric_involution"))
```

Out [7]:

```
PASS C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3) equals the recorded C
     reproduces Revision/algebra/reports/wolfram-algebra.json, check C_definition
PASS C = diag(-sigma, sigma), called sigma16 by the author
     reproduces Revision/algebra/reports/python-algebra.json, check
    C_equals_notebook_sigma16
PASS C is real and symmetric, and C C = 1 (so C is its own inverse)
     reproduces Revision/algebra/reports/python-algebra.json, check
    C_real_symmetric_involution
```

Text cell:

```
The next cell draws $C$ and $\sigma$ as heat maps. In $C$ the top-left block is
$-\sigma$ (blue squares) and the bottom-right block is $+\sigma$ (red squares); the
two off-diagonal blocks are zero (grey).
```

In [8]:

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.4), width_ratios=[2, 1])
heat_map(axes[0], C, r"$C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$")
image = heat_map(axes[1], sigma, r"$\sigma$ (8 x 8)", halves=False,
                 row_label=False)
```

```
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.75, label="matrix entry")
save_figure(fig, "c_matrix",
            "Heat maps of the charge matrix $C$ (left, 16 by 16) and of the "
            "author's 8 by 8 matrix $\\sigma$ (right); horizontal axis the column, "
            "vertical axis the row, numbered from 1; blue $-1$, grey $0$, red $+1$. "
            "The black lines cut $C$ into four 8 by 8 blocks: the top-left block "
            "is $-\\sigma$, the bottom-right block is $+\\sigma$ and the other two "
            "are zero, so $C = \\mathrm{diag}(-\\sigma, \\sigma)$. Every row and "
            "every column holds exactly one nonzero entry, and the picture is "
            "mirror-symmetric about the diagonal because $C^T = C$.")
```

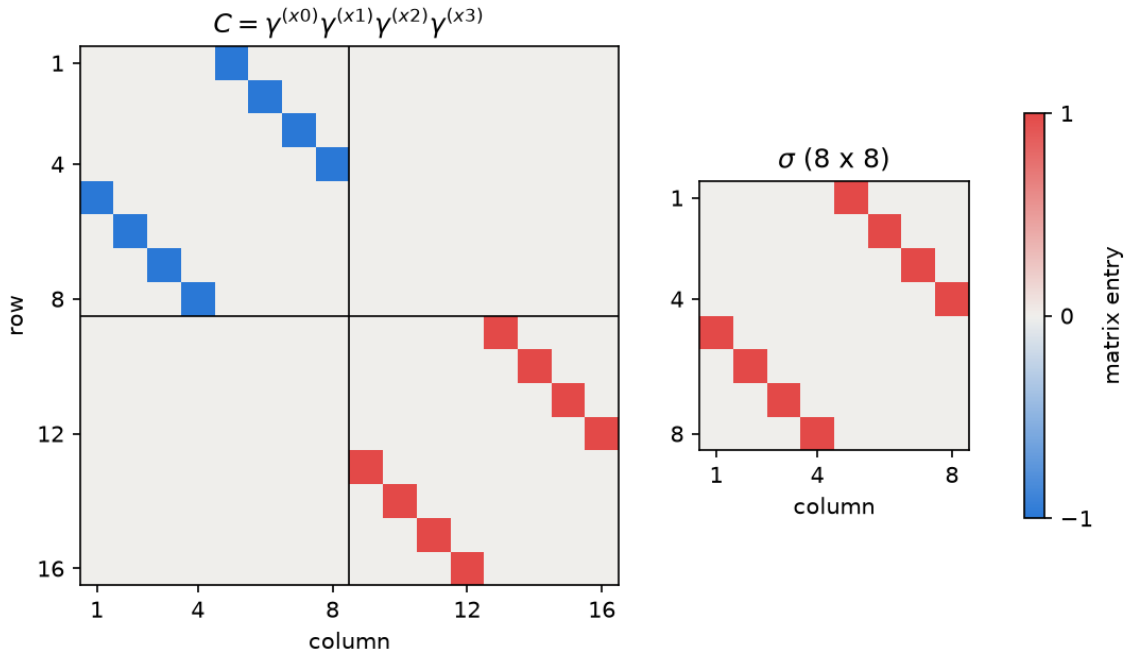

Figure 142: Heat maps of the charge matrix C (left, 16 by 16) and of the author's 8 by 8 matrix σ (right); horizontal axis the column, vertical axis the row, numbered from 1; blue -1, grey 0, red $+1$. The black lines cut C into four 8 by 8 blocks: the top-left block is $-\sigma$, the bottom-right block is $+\sigma$ and the other two are zero, so $C = \mathrm{diag}(-\sigma, \sigma)$. Every row and every column holds exactly one nonzero entry, and the picture is mirror-symmetric about the diagonal because $C^T = C$. (Notebook 05a, figure 1.)

Out [8]:

```
Figure 05a.1 saved as Revision/textbook/figures/05a_1_c_matrix.png
```

Text cell:

```
The next cell checks the sign rule of section 4 for $C$: $C\gamma^a =
-\eta_{aa}\gamma^a C$, so $C$ anticommutes with the four space-like gammas and
commutes with the four time-like ones. It then checks two recorded consequences.
First, $C\gamma^a$ is antisymmetric: $(C\gamma^a)^T = (\gamma^a)^T C^T = \eta_{aa}
```

```
\gamma^a C = \eta_{aa}(-\eta_{aa}) C\gamma^a = -C\gamma^a$, because
$\eta_{aa}^2 = 1$. Second, $C\gamma^a C^{-1} = -(\gamma^a)^T$: multiply
$\gamma^a C = -\eta_{aa} C\gamma^a$ from the left by $C$ and use $CC = 1$ to get
$C\gamma^a C = -\eta_{aa}\gamma^a = -(\gamma^a)^T$.
```

In [9]:

```
signs_C = {x: commutation_sign(C, gamma[x]) for x in COORDS}  # +1 or -1 for each x
say("C gamma^(x) = s gamma^(x) C with s = "
    + ", ".join(f"{x}: {signs_C[x]:+d}" for x in COORDS))
check(all(signs_C[x] == -ETA[x] for x in COORDS),
      "C gamma^a = -eta_aa gamma^a C (anticommutes with space-like, commutes with "
      "time-like gammas)")
check_reproduces(all(np.array_equal((C @ gamma[x]).T, -(C @ gamma[x])) for x in COORDS)
                 and recorded("python", "C_gamma_antisymmetric"),
                 "C gamma^a is antisymmetric for every a",
                 record=record_of("python", "C_gamma_antisymmetric"))
check_reproduces(all(np.array_equal(C @ gamma[x] @ C, -gamma[x].T) for x in COORDS)
                 and recorded("python", "C_conjugation"),
                 "C gamma^a C^-1 = -(gamma^a)^T for every a",
                 record=record_of("python", "C_conjugation"))
```

Out [9]:

```
C gamma^(x) = s gamma^(x) C with s = x0: -1, x1: -1, x2: -1, x3: -1, x4: +1, x5: +1, x6:
    +1, x7: +1
PASS C gamma^a = -eta_aa gamma^a C (anticommutes with space-like, commutes with time-like
    gammas)
PASS C gamma^a is antisymmetric for every a
     reproduces Revision/algebra/reports/python-algebra.json, check C_gamma_antisymmetric
PASS C gamma^a C^-1 = -(gamma^a)^T for every a
     reproduces Revision/algebra/reports/python-algebra.json, check C_conjugation
```

Text cell:

```
The next cell tests the squaring rule of section 4,
$PP = (-1)^{k(k-1)/2}\, \eta_{a_1 a_1} \cdots \eta_{a_k a_k} 1$, on **every** product
of different gammas taken in the order $x0, \dots, x7$: there are $2^8 - 1 = 255$ of
them (every non-empty subset of the eight directions). `itertools.combinations`
lists the subsets of each size $k$.
```

In [10]:

```
import itertools  # lists all subsets of a given size


def square_sign(directions):
    """The sign s of (gamma^(d1) ... gamma^(dk))^2 = s 1 predicted by the rule
    s = (-1)^(k (k - 1) / 2) times the product of the eta_dd."""
```

```
    k = len(directions)
    s = (-1) ** (k * (k - 1) // 2)  # // is division of whole numbers
    for d in directions:
        s *= ETA[d]
    return s


agree = 0  # how many subsets obey the rule
for k in range(1, 9):
    for subset in itertools.combinations(COORDS, k):
        p = product(subset)
        if np.array_equal(p @ p, square_sign(subset) * I16):
            agree += 1
say(f"subsets tested: 255; subsets that obey the squaring rule: {agree}")
for label, dirs in [("C", ["x0", "x1", "x2", "x3"]),
                    ("Gamma", ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]),
                    ("C gamma^(x4)", ["x0", "x1", "x2", "x3", "x4"])]:
    say(f"({label})^2 = {square_sign(dirs):+d} times 1 (k = {len(dirs)})")
check(agree == 255, "the squaring rule holds for all 255 products of different gammas")
```

Out [10]:

```
subsets tested: 255; subsets that obey the squaring rule: 255
(C)^2 = +1 times 1 (k = 4)
(Gamma)^2 = +1 times 1 (k = 8)
(C gamma^(x4))^2 = -1 times 1 (k = 5)
PASS the squaring rule holds for all 255 products of different gammas
```

Text cell:

```
## 7. C defines an indefinite quadratic form

For a real column $u$ of 16 numbers the number $u^T C u = \sum_{r,c} u_r C_{rc}
u_c$ is a *quadratic form*. Because $C$ has eigenvalues of both signs, this form
takes both signs. Take the path $u(t) = \cos t\, e_1 + \sin t\, e_5$, where $e_r$ is
the column with a 1 in row $r$ and zeros elsewhere. Only the entries $C_{1,5} =
C_{5,1}$ meet two nonzero components, so $u^T C u = 2 \cos t \sin t\, C_{1,5} =
C_{1,5} \sin 2t$ (double-angle formula $2\sin t\cos t = \sin 2t$). Along the path
$v(t) = \cos t\, e_9 + \sin t\, e_{13}$ the same steps give $C_{9,13} \sin 2t$. The
next cell reads $C_{1,5} = -1$ and $C_{9,13} = +1$ from the matrix (Python counts
rows from 0, so row 1 is index 0), evaluates both forms on 401 values of $t$ and
checks them against the formulas. It also computes the eigenvalues of $C$
(‘np.linalg.eigvalsh‘ is made for symmetric matrices) and counts the positive and
negative ones.
```

In [11]:

```
t = np.linspace(0.0, np.pi, 401)  # 401 equally spaced values of t from 0 to pi
e = np.eye(16)  # e[r - 1] is the column e_r (a 1 in row r)
u = np.outer(np.cos(t), e[0]) + np.outer(np.sin(t), e[4])  # u(t), one per row
v = np.outer(np.cos(t), e[8]) + np.outer(np.sin(t), e[12])  # v(t), one per row
```

```
form_u = np.einsum("tr,rc,tc->t", u, C, u)  # u^T C u for every t at once
form_v = np.einsum("tr,rc,tc->t", v, C, v)  # v^T C v for every t at once
say(f"C_(1,5) = {C[0, 4]:+d} and C_(9,13) = {C[8, 12]:+d}")
check(C[0, 4] == -1 and C[8, 12] == 1
      and np.max(np.abs(form_u + np.sin(2 * t))) < 1e-14
      and np.max(np.abs(form_v - np.sin(2 * t))) < 1e-14,
      "u^T C u = -sin 2t and v^T C v = +sin 2t along the two paths")
eigenvalues_C = np.linalg.eigvalsh(C.astype(float))  # 16 real eigenvalues, sorted
n_plus = int(np.sum(np.abs(eigenvalues_C - 1.0) < 1e-9))  # how many are +1
n_minus = int(np.sum(np.abs(eigenvalues_C + 1.0) < 1e-9))  # how many are -1
say(f"trace of C = {int(np.trace(C))}; eigenvalues +1: {n_plus}, -1: {n_minus}")
check(np.trace(C) == 0 and n_plus == 8 and n_minus == 8,
      "C has eight eigenvalues +1 and eight -1: signature (8, 8)")
```

Out [11]:

```
C_(1,5) = -1 and C_(9,13) = +1
PASS u^T C u = -sin 2t and v^T C v = +sin 2t along the two paths
trace of C = 0; eigenvalues +1: 8, -1: 8
PASS C has eight eigenvalues +1 and eight -1: signature (8, 8)
```

Text cell:

```
The next cell draws the two forms and, for comparison, the ordinary squared length
$u^T u = \cos^2 t + \sin^2 t = 1$, which never changes sign.
```

In [12]:

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(t, form_u, color="#2a78d6", linewidth=2,
        label=r"$u^T C u$ with $u = \cos t\, e_1 + \sin t\, e_5$")
ax.plot(t, form_v, color="#eb6834", linewidth=2, linestyle="--",
        label=r"$v^T C v$ with $v = \cos t\, e_9 + \sin t\, e_{13}$")
ax.plot(t, np.einsum("tr,tr->t", u, u), color="#1baf7a", linewidth=2,
        linestyle=":", label=r"$u^T u$ (ordinary squared length)")
ax.axhline(0.0, color="black", linewidth=0.6)
ax.set_xticks([0, np.pi / 4, np.pi / 2, 3 * np.pi / 4, np.pi],
              ["0", r"$\pi/4$", r"$\pi/2$", r"$3\pi/4$", r"$\pi$"])
ax.set_xlabel("$t$ (radians)")
ax.set_ylabel("value of the quadratic form")
ax.set_title("The form of $C$ takes both signs; the ordinary length does not")
# the legend goes below the picture, where it hides no curve
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=1)
save_figure(fig, "c_form_signs",
            "The quadratic form of the charge matrix along two paths of unit "
            "columns, $u^T C u = -\\sin 2t$ (solid, components 1 and 5) and "
            "$v^T C v = +\\sin 2t$ (dashed, components 9 and 13), and the ordinary "
            "squared length $u^T u = 1$ (dotted), for $t$ from $0$ to $\\pi$; "
            "horizontal axis $t$ in radians, vertical axis the value (a pure "
            "number). The form of $C$ is negative for some unit columns and "
            "positive for others: $C$ has signature $(8, 8)$, an indefinite form.")
```

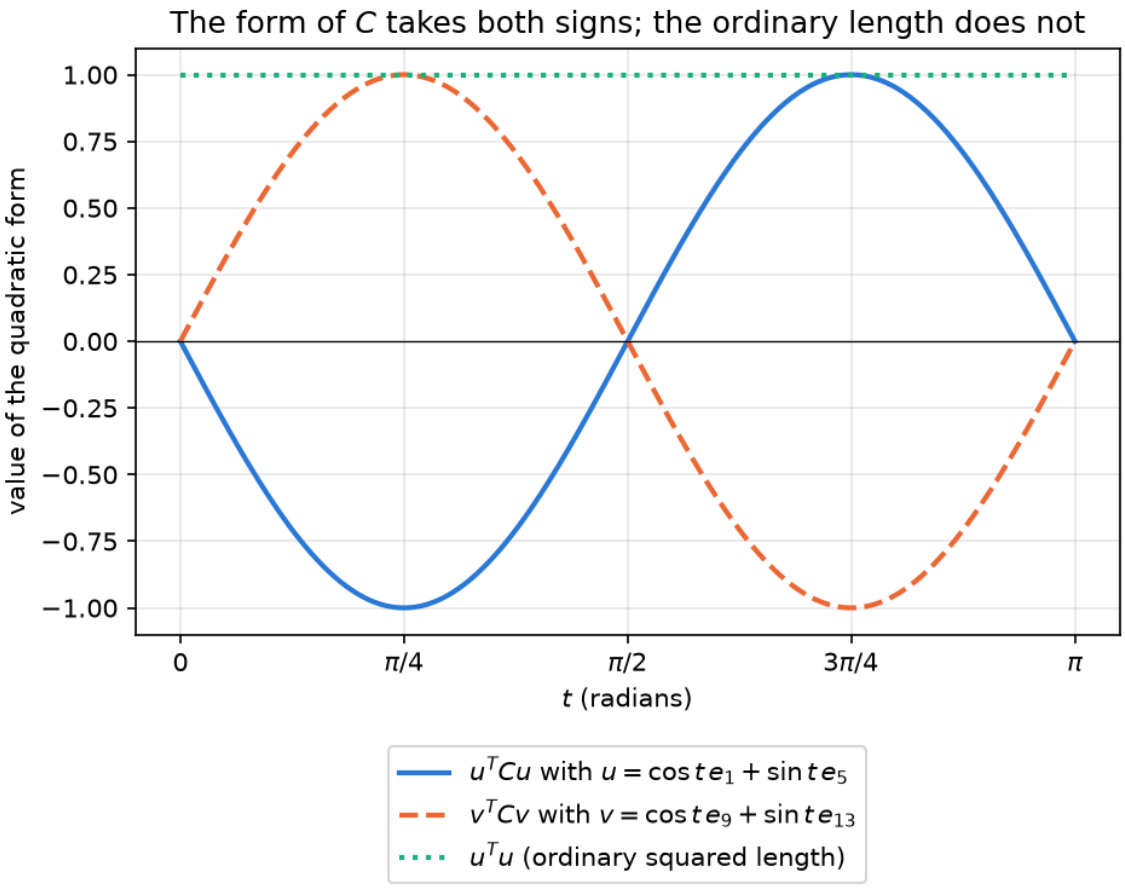

Figure 143: The quadratic form of the charge matrix along two paths of unit columns, $u^T C u = -\sin 2t$ (solid, components 1 and 5) and $v^T C v = +\sin 2t$ (dashed, components 9 and 13), and the ordinary squared length $u^T u = 1$ (dotted), for t from 0 to π; horizontal axis t in radians, vertical axis the value (a pure number). The form of C is negative for some unit columns and positive for others: C has signature $(8, 8)$, an indefinite form. (Notebook 05a, figure 2.)

Out [12]:

```
Figure 05a.2 saved as Revision/textbook/figures/05a_2_c_form_signs.png
```

Text cell:

```
## 8. The chirality Gamma and the two halves

The next cell builds $\Gamma = \gamma^{(x0)} \gamma^{(x1)} \cdots \gamma^{(x7)}$ (the
author's product order) and checks four recorded facts:

1. $\Gamma$ equals the recorded matrix and is $\mathrm{diag}(-1_8, 1_8)$;
2. $\Gamma\Gamma = 1$ and $\Gamma$ anticommutes with every $\gamma^a$ (sign rule:
   every gamma is one of the 8 factors, $(-1)^{8-1} = -1$);
3. $\Gamma = C \gamma^{(x4)}\gamma^{(x5)}\gamma^{(x6)}\gamma^{(x7)}$ (the first four
   factors of $\Gamma$ are $C$);
4. $C\Gamma = \Gamma C$ (both are products of an even number of gammas),
   $\Gamma^T = \Gamma$, $\Gamma^T C \Gamma = C$ and
   $\Gamma^T C\gamma^a \Gamma = -C\gamma^a$ for every $a$.

Fact 4 is the matrix input of the pairing theorem T1 of the Revision record: the
map $\Psi \to \Gamma\Psi$ keeps the bilinear $\Psi^\dagger C \Psi$ (because
$(\Gamma\Psi)^\dagger C(\Gamma\Psi) = \Psi^\dagger\Gamma^T C\Gamma\Psi$, $\Gamma$
being real) and reverses the sign of every $\Psi^\dagger C\gamma^a E\Psi$ in which
$E$ is a product of an even number of gammas ($\Gamma$ commutes with such an $E$).
```

In [13]:

```
Gamma = product(["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"])  # all eight
check_reproduces(np.array_equal(Gamma, np.array(fixture["Gamma"], dtype=np.int64))
                 and np.array_equal(Gamma, np.block([[-I8, Z8], [Z8, I8]]))
                 and recorded("python", "chirality_diag"),
                 "Gamma = gamma^(x0) gamma^(x1) ... gamma^(x7) = diag(-1_8, 1_8), as "
                 "recorded",
                 record=record_of("python", "chirality_diag"))
check_reproduces(np.array_equal(Gamma @ Gamma, I16)
                 and all(commutation_sign(Gamma, gamma[x]) == -1 for x in COORDS)
                 and recorded("python", "chirality_anticommutes"),
                 "Gamma Gamma = 1 and Gamma anticommutes with every gamma^a",
                 record=record_of("python", "chirality_anticommutes"))
check_reproduces(np.array_equal(Gamma, C @ product(["x4", "x5", "x6", "x7"]))
                 and recorded("python", "chirality_eq_C_times_time_gammas"),
                 "Gamma = C gamma^(x4) gamma^(x5) gamma^(x6) gamma^(x7)",
                 record=record_of("python", "chirality_eq_C_times_time_gammas"))
check_reproduces(commutation_sign(C, Gamma) == 1 and np.array_equal(Gamma.T, Gamma)
                 and np.array_equal(Gamma.T @ C @ Gamma, C)
                 and all(np.array_equal(Gamma.T @ C @ gamma[x] @ Gamma, -(C @ gamma[x]))
                         for x in COORDS)
                 and recorded("python", "chirality_C_relation"),
                 "C Gamma = Gamma C, Gamma^T C Gamma = C, Gamma^T C gamma^a Gamma = -C "
                 "gamma^a",
                 record=record_of("python", "chirality_C_relation"))
```

Out [13]:

```
PASS Gamma = gamma^(x0) gamma^(x1) ... gamma^(x7) = diag(-1_8, 1_8), as recorded
     reproduces Revision/algebra/reports/python-algebra.json, check chirality_diag
PASS Gamma Gamma = 1 and Gamma anticommutes with every gamma^a
     reproduces Revision/algebra/reports/python-algebra.json, check
    chirality_anticommutes
PASS Gamma = C gamma^(x4) gamma^(x5) gamma^(x6) gamma^(x7)
     reproduces Revision/algebra/reports/python-algebra.json, check
    chirality_eq_C_times_time_gammas
PASS C Gamma = Gamma C, Gamma^T C Gamma = C, Gamma^T C gamma^a Gamma = -C gamma^a
     reproduces Revision/algebra/reports/python-algebra.json, check chirality_C_relation
```

Text cell:

```
Because $\Gamma\Gamma = 1$, the matrices $P_- = \tfrac12(1 - \Gamma)$ and
$P_+ = \tfrac12(1 + \Gamma)$ are projectors: $P_-P_- = \tfrac14(1 - 2\Gamma +
\Gamma\Gamma) = \tfrac14(2 - 2\Gamma) = P_-$, and in the same way $P_+P_+ = P_+$ and
$P_-P_+ = \tfrac14(1 - \Gamma\Gamma) = 0$. With $\Gamma = \mathrm{diag}(-1_8, 1_8)$,
$P_- = \mathrm{diag}(1_8, 0)$ keeps the first half of a column (components 1 to 8,
chirality $-1$) and $P_+ = \mathrm{diag}(0, 1_8)$ the second half (9 to 16,
chirality $+1$). Since $\gamma^a\Gamma = -\Gamma\gamma^a$, we get $\gamma^a P_- =
\tfrac12(\gamma^a + \Gamma\gamma^a) = P_+\gamma^a$: every gamma carries the first
half into the second and back, so every gamma has zero diagonal blocks. The next
cell checks all of this (‘//‘ divides whole numbers exactly; the entries of
$1 \pm \Gamma$ are 0 or 2).
```

In [14]:

```
P_minus = (I16 - Gamma) // 2  # the projector onto components 1 to 8
P_plus = (I16 + Gamma) // 2  # the projector onto components 9 to 16
check(np.array_equal(P_minus, np.block([[I8, Z8], [Z8, Z8]]))
      and np.array_equal(P_plus, np.block([[Z8, Z8], [Z8, I8]]))
      and np.array_equal(P_minus @ P_minus, P_minus)
      and np.array_equal(P_plus @ P_plus, P_plus)
      and not np.any(P_minus @ P_plus) and np.array_equal(P_minus + P_plus, I16)
      and np.trace(P_minus) == 8 and np.trace(P_plus) == 8,
      "P_- = diag(1_8, 0) and P_+ = diag(0, 1_8) are complementary projectors of rank 8")
check_reproduces(all(np.array_equal(gamma[x] @ P_minus, P_plus @ gamma[x])
                     for x in COORDS)
                 and all(not np.any(gamma[x][:8, :8]) and not np.any(gamma[x][8:, 8:])
                         for x in COORDS)
                 and recorded("python", "reflections_exchange_halves")
                 and recorded("wolfram", "Spin_Pin_reflection_swaps_halves"),
                 "gamma^a P_- = P_+ gamma^a: every gamma exchanges the two halves",
                 record=record_of("python", "reflections_exchange_halves"))
```

Out [14]:

```
PASS P_- = diag(1_8, 0) and P_+ = diag(0, 1_8) are complementary projectors of rank 8
PASS gamma^a P_- = P_+ gamma^a: every gamma exchanges the two halves
```

```
 reproduces Revision/algebra/reports/python-algebra.json, check
reflections_exchange_halves
```

Text cell:

```
The next cell draws $\Gamma$, the two projectors and, for comparison,
$\gamma^{(x1)}$: the chirality and the projectors live in the diagonal blocks, a
gamma lives in the off-diagonal blocks.
```

In [15]:

```
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.9))
heat_map(axes[0], Gamma, r"$\Gamma$")
heat_map(axes[1], P_minus, r"$P_- = (1 - \Gamma)/2$", row_label=False)
heat_map(axes[2], P_plus, r"$P_+ = (1 + \Gamma)/2$", row_label=False)
image = heat_map(axes[3], gamma["x1"], r"$\gamma^{(x1)}$ for comparison",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "chirality",
            "Heat maps of the chirality $\\Gamma$, of the projectors $P_-$ and "
            "$P_+$ onto the two halves, and of the gamma matrix of the direction "
            "$x1$; horizontal axis the column, vertical axis the row (1 to 16); "
            "blue $-1$, grey $0$, red $+1$. $\\Gamma = \\mathrm{diag}(-1, 1)$ is "
            "blue on the first eight diagonal places and red on the last eight; "
            "$P_-$ keeps components 1 to 8 and $P_+$ components 9 to 16. The gamma "
            "has nonzero entries only in the two off-diagonal blocks: it carries "
            "one half into the other.")
```

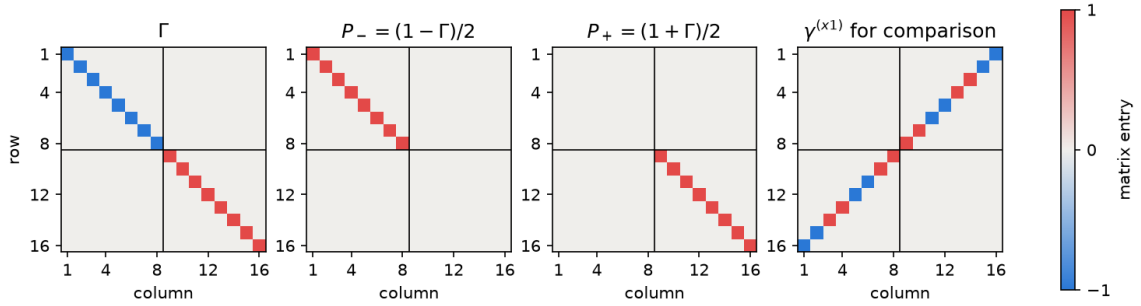

Figure 144: Heat maps of the chirality Γ, of the projectors P_- and P_+ onto the two halves, and of the gamma matrix of the direction $x1$; horizontal axis the column, vertical axis the row (1 to 16); blue -1, grey 0, red $+1$. $\Gamma = \mathrm{diag}(-1, 1)$ is blue on the first eight diagonal places and red on the last eight; P_- keeps components 1 to 8 and P_+ components 9 to 16. The gamma has nonzero entries only in the two off-diagonal blocks: it carries one half into the other. (Notebook 05a, figure 3.)

Out [15]:

```
Figure 05a.3 saved as Revision/textbook/figures/05a_3_chirality.png
```

Text cell:

```
## 9. The matrix B

The next cell builds $B = -i\, C\gamma^{(x4)}$ (in Python the imaginary unit is
written ‘1j‘) and checks it against the record, which stores the real and the
imaginary part separately. Then it checks the recorded properties: $B$ is purely
imaginary, Hermitian ($B^\dagger = B$), $BB = 1$ and $\mathrm{tr}\, B = 0$. Why
Hermitian: $C\gamma^{(x4)}$ is real and antisymmetric (section 6), so
$B^\dagger = (+i)(C\gamma^{(x4)})^T = (+i)(-C\gamma^{(x4)}) = B$. The entries of $B$
are $0$, $i$ or $-i$, so every product is again exact.
```

In [16]:

```
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4); 1j is the imaginary unit
B_record = (np.array(fixture["B"]["re"], dtype=np.int64)
            + 1j * np.array(fixture["B"]["im"], dtype=np.int64))
check_reproduces(np.array_equal(B, B_record) and recorded("wolfram", "B_definition"),
                 "B = -i C gamma^(x4) equals the recorded B",
                 record=record_of("wolfram", "B_definition"))
check_reproduces(not np.any(B.real) and np.array_equal(B.conj().T, B)
                 and np.array_equal(B @ B, I16) and np.trace(B) == 0
                 and recorded("python", "B_hermitian_involution_signature")
                 and recorded("wolfram", "B_Hermitian")
                 and recorded("wolfram", "B_squared_identity"),
                 "B is purely imaginary and Hermitian, B B = 1 and tr B = 0",
                 record=record_of("python", "B_hermitian_involution_signature"))
```

Out [16]:

```
PASS B = -i C gamma^(x4) equals the recorded B
     reproduces Revision/algebra/reports/wolfram-algebra.json, check B_definition
PASS B is purely imaginary and Hermitian, B B = 1 and tr B = 0
     reproduces Revision/algebra/reports/python-algebra.json, check
    B_hermitian_involution_signature
```

Text cell:

```
The next cell computes the *characteristic polynomial*
$\det(\lambda 1 - B)$ of $B$ exactly with sympy (whose roots are the eigenvalues)
and factors it. The result must be $(\lambda - 1)^8 (\lambda + 1)^8$: eight
eigenvalues $+1$ and eight $-1$, the signature $(8, 8)$ of the Hermitian form
$\Psi^\dagger B \Psi$. The recorded check states the same polynomial in its detail
text, which the cell compares character by character (sympy writes ‘**‘ for a
power and ‘*‘ for a product).
```

In [17]:

```
import sympy as sp  # exact algebra with symbols

lam = sp.Symbol("lam")  # the variable lambda of the polynomial
```

```
B_exact = sp.I * sp.Matrix((-(C @ gamma["x4"])).tolist())  # B with exact sympy numbers
polynomial = sp.factor(B_exact.charpoly(lam).as_expr())  # det(lam 1 - B), factored
say(f"characteristic polynomial of B: {polynomial}")
detail = VERDICTS[("python", "B_hermitian_involution_signature")][1]
check_reproduces(sp.expand(polynomial - (lam - 1) ** 8 * (lam + 1) ** 8) == 0
                 and f"= {polynomial}" in detail
                 and recorded("wolfram", "B_signature_8_8"),
                 "det(lam 1 - B) = (lam - 1)^8 (lam + 1)^8: signature (8, 8)",
                 record=record_of("python", "B_hermitian_involution_signature"))
```

Out [17]:

```
characteristic polynomial of B: (lam - 1)**8*(lam + 1)**8
PASS det(lam 1 - B) = (lam - 1)^8 (lam + 1)^8: signature (8, 8)
     reproduces Revision/algebra/reports/python-algebra.json, check
    B_hermitian_involution_signature
```

Text cell:

```
The next cell finds with which gammas $B$ commutes. By the sign rule of section 4,
$C\gamma^{(x4)}$ is a product of the five different gammas $x0, x1, x2, x3, x4$: a
gamma among them passes four others and commutes, $(-1)^4 = +1$; a gamma not among
them ($x5, x6, x7$) passes five and anticommutes. The factor $-i$ does not change
this. The recorded check states exactly this pattern.
```

In [18]:

```
signs_B = {x: commutation_sign(B, gamma[x]) for x in COORDS}
commute = [x for x in COORDS if signs_B[x] == 1]
anticommute = [x for x in COORDS if signs_B[x] == -1]
say("B commutes with gamma^(x) for x = " + ", ".join(commute))
say("B anticommutes with gamma^(x) for x = " + ", ".join(anticommute))
check_reproduces(commute == ["x0", "x1", "x2", "x3", "x4"]
                 and anticommute == ["x5", "x6", "x7"]
                 and recorded("python", "B_gamma_relations"),
                 "B commutes with gamma^a for a = x0, x1, x2, x3, x4 and anticommutes "
                 "for x5, x6, x7",
                 record=record_of("python", "B_gamma_relations"))
```

Out [18]:

```
B commutes with gamma^(x) for x = x0, x1, x2, x3, x4
B anticommutes with gamma^(x) for x = x5, x6, x7
PASS B commutes with gamma^a for a = x0, x1, x2, x3, x4 and anticommutes for x5, x6, x7
     reproduces Revision/algebra/reports/python-algebra.json, check B_gamma_relations
```

Text cell:

```
The next cell draws the three matrices from which $B$ is made: $\gamma^{(x4)}$,
the real antisymmetric matrix $C\gamma^{(x4)}$, and the imaginary part of $B$,
which is $-C\gamma^{(x4)}$ (the real part of $B$ is zero).
```

In [19]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], gamma["x4"], r"$\gamma^{(x4)}$ (antisymmetric)")
heat_map(axes[1], C @ gamma["x4"], r"$C\gamma^{(x4)}$ (antisymmetric)",
         row_label=False)
image = heat_map(axes[2], B.imag, r"imaginary part of $B = -iC\gamma^{(x4)}$",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "b_matrix",
            "Heat maps of the gamma matrix of the time $x4$ (left), of the product "
            "$C\\gamma^{(x4)}$ (middle) and of the imaginary part of "
            "$B = -iC\\gamma^{(x4)}$ (right); horizontal axis the column, vertical "
            "axis the row (1 to 16); blue $-1$, grey $0$, red $+1$. Each picture "
            "changes colour when it is mirrored about the diagonal: the three "
            "real matrices are antisymmetric. $B$ is $i$ times an antisymmetric "
            "real matrix, which makes it Hermitian.")
```

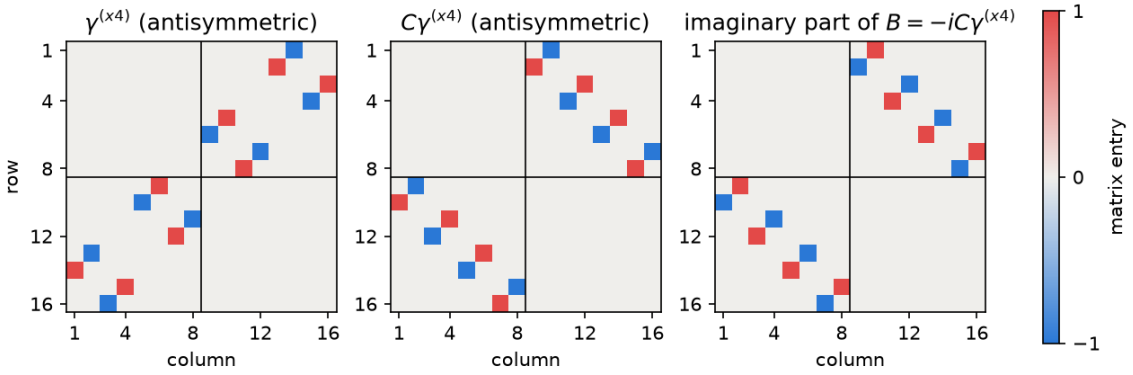

Figure 145: Heat maps of the gamma matrix of the time $x4$ (left), of the product $C\gamma^{(x4)}$ (middle) and of the imaginary part of $B = -iC\gamma^{(x4)}$ (right); horizontal axis the column, vertical axis the row (1 to 16); blue -1, grey 0, red $+1$. Each picture changes colour when it is mirrored about the diagonal: the three real matrices are antisymmetric. B is i times an antisymmetric real matrix, which makes it Hermitian. (Notebook 05a, figure 4.)

Out [19]:

```
Figure 05a.4 saved as Revision/textbook/figures/05a_4_b_matrix.png
```

Text cell:

```
## 10. The spectra
```

```
The next cell computes the eigenvalues of the eight gammas, of $C$, $\Gamma$ and
$B$ with ‘np.linalg.eigvals‘ (floating-point numbers, correct to about 15 digits)
and counts how many lie within $10^{-9}$ of each of $+1$, $-1$, $+i$ and $-i$.
Section 4 predicts: a matrix with $MM = 1$ and trace 0 has eight eigenvalues $+1$
and eight $-1$; one with $MM = -1$ and trace 0 has eight $+i$ and eight $-i$. The
exact facts behind the prediction ($MM = \pm 1$, trace 0) were checked above for
$C$, $\Gamma$ and $B$; for the gammas the cell checks the traces.
```

In [20]:

```
TARGETS = [1, -1, 1j, -1j]  # the four possible eigenvalues
TARGET_NAMES = ["+1", "-1", "+i", "-i"]
matrices = {f"gamma^({x})": gamma[x] for x in COORDS}
matrices.update({"C": C, "Gamma": Gamma, "B": B})
counts = {}  # name -> [how many eigenvalues near +1, -1, +i, -i]
for name, m in matrices.items():
    eigenvalues = np.linalg.eigvals(m.astype(complex))
    counts[name] = [int(np.sum(np.abs(eigenvalues - z) < 1e-9)) for z in TARGETS]
    say(f"{name:12} " + "  ".join(f"{n}: {c}" for n, c in
                                   zip(TARGET_NAMES, counts[name])))
predicted = {name: ([8, 8, 0, 0] if name in ("C", "Gamma", "B")
                    or ETA[name[7:9]] == 1 else [0, 0, 8, 8]) for name in matrices}
check(counts == predicted and all(np.trace(gamma[x]) == 0 for x in COORDS),
      "eigenvalues: +1 and -1 eight times each for the space-like gammas, C, Gamma "
      "and B; +i and -i eight times each for the time-like gammas")
```

Out [20]:

```
gamma^(x0)   +1: 8  -1: 8  +i: 0  -i: 0
gamma^(x1)   +1: 8  -1: 8  +i: 0  -i: 0
gamma^(x2)   +1: 8  -1: 8  +i: 0  -i: 0
gamma^(x3)   +1: 8  -1: 8  +i: 0  -i: 0
gamma^(x4)   +1: 0  -1: 0  +i: 8  -i: 8
gamma^(x5)   +1: 0  -1: 0  +i: 8  -i: 8
gamma^(x6)   +1: 0  -1: 0  +i: 8  -i: 8
gamma^(x7)   +1: 0  -1: 0  +i: 8  -i: 8
C            +1: 8  -1: 8  +i: 0  -i: 0
Gamma        +1: 8  -1: 8  +i: 0  -i: 0
B            +1: 8  -1: 8  +i: 0  -i: 0
PASS eigenvalues: +1 and -1 eight times each for the space-like gammas, C, Gamma and B;
    +i and -i eight times each for the time-like gammas
```

Text cell:

```
The next cell draws the table of these counts as a picture: one row per matrix,
one column per possible eigenvalue, the count written in each square.
```

In [21]:

```
from matplotlib.colors import LinearSegmentedColormap as Ramp

names = list(matrices)  # the 11 matrices in the order of the table
table = np.array([counts[name] for name in names])  # 11 rows, 4 columns
COUNT_COLORS = Ramp.from_list("counts", ["#f0efec", "#2a78d6"])  # 0 grey, 8 blue
fig, ax = plt.subplots(figsize=(6.0, 6.4))
ax.imshow(table, cmap=COUNT_COLORS, vmin=0, vmax=8, aspect="auto")
for k in range(1, table.shape[0]):  # white gaps between the rows
    ax.axhline(k - 0.5, color="white", linewidth=2)
for k in range(1, table.shape[1]):  # and between the columns
    ax.axvline(k - 0.5, color="white", linewidth=2)
for r in range(table.shape[0]):
    for c in range(table.shape[1]):
        ax.text(c, r, str(table[r, c]), ha="center", va="center",
                color="white" if table[r, c] == 8 else "black")
ax.set_xticks(range(4), ["$+1$", "$-1$", "$+i$", "$-i$"])
labels = [rf"$\gamma^{{({x})}}$" for x in COORDS] + ["$C$", r"$\Gamma$", "$B$"]
ax.set_yticks(range(len(names)), labels)
ax.set_xlabel("eigenvalue")
ax.set_title("How many times each eigenvalue occurs (16 in each row)")
ax.grid(False)
save_figure(fig, "spectra",
            "The spectra of the eight gamma matrices and of $C$, $\\Gamma$ and $B$: "
            "each row is one 16 by 16 matrix, each column one possible eigenvalue "
            "($+1$, $-1$, $+i$, $-i$), and each square holds how many of its 16 "
            "eigenvalues equal that number. The space-like gammas ($x0, x1, x2, "
            "x3$), $C$, $\\Gamma$ and $B$ square to $1$ and have eigenvalues "
            "$+1$ and $-1$ eight times each; the time-like gammas ($x4$ to $x7$) "
            "square to $-1$ and have $+i$ and $-i$ eight times each.")
```

Out [21]:

```
Figure 05a.5 saved as Revision/textbook/figures/05a_5_spectra.png
```

Text cell:

```
## 11. A table of commutation signs

The next cell collects, for each of $C$, $\Gamma$ and $B$ and each gamma, the sign
$s$ in $M\gamma^a = s\,\gamma^a M$ and draws the table. Section 4 predicts every
entry: $C$ has $s = -\eta_{aa}$; $\Gamma$ has $s = -1$ for all $a$; $B$ has $s = +1$
for $x0, x1, x2, x3, x4$ and $s = -1$ for $x5, x6, x7$.
```

In [22]:

```
rows = {"C": signs_C, "Gamma": {x: commutation_sign(Gamma, gamma[x]) for x in COORDS},
        "B": signs_B}
sign_table = np.array([[rows[name][x] for x in COORDS] for name in rows])
expected_table = np.array([[-ETA[x] for x in COORDS], [-1] * 8,
                           [1, 1, 1, 1, 1, -1, -1, -1]])
check(np.array_equal(sign_table, expected_table),
```

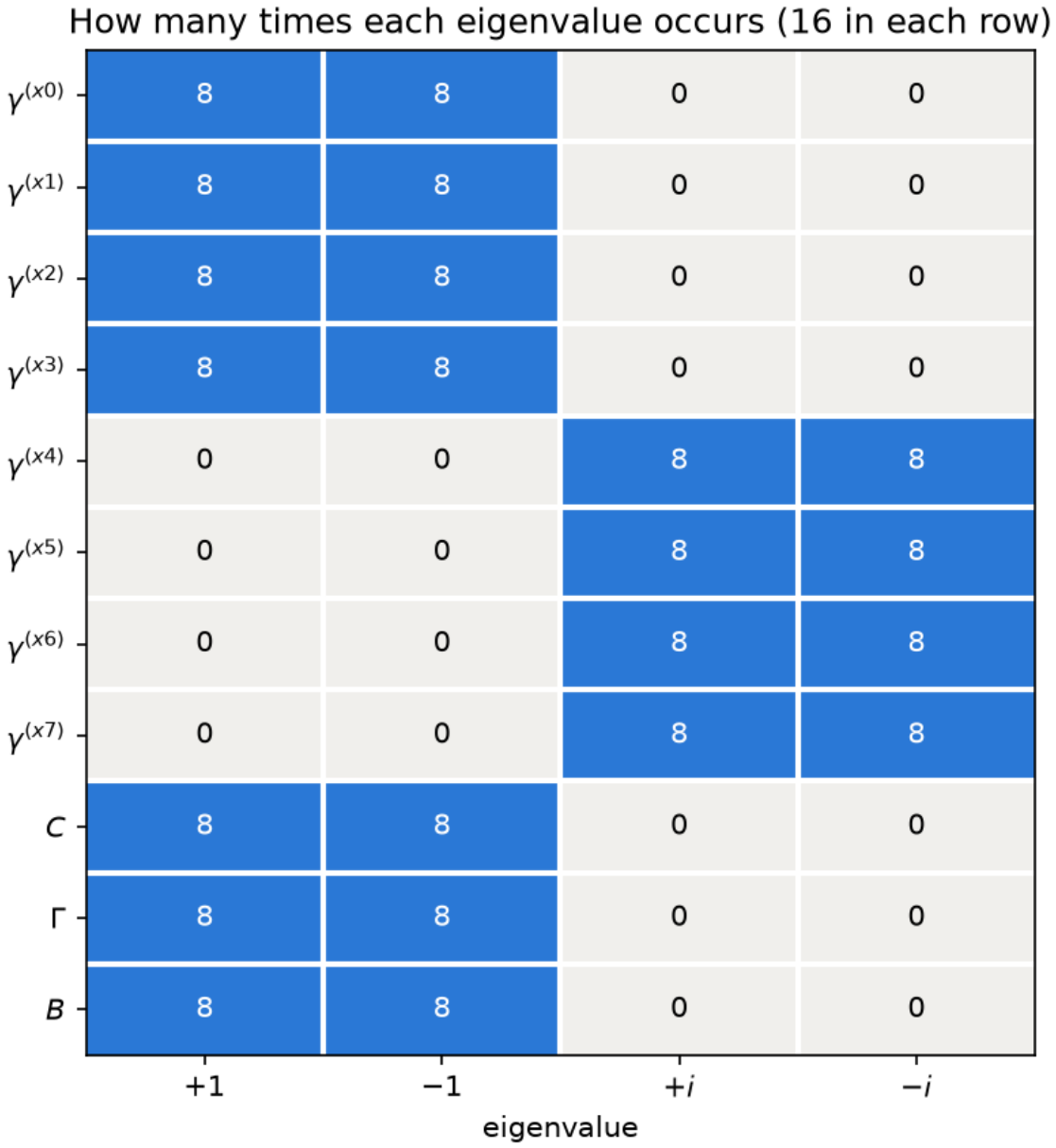

Figure 146: The spectra of the eight gamma matrices and of C, Γ and B: each row is one 16 by 16 matrix, each column one possible eigenvalue ($+1$, -1, $+i$, $-i$), and each square holds how many of its 16 eigenvalues equal that number. The space-like gammas ($x0, x1, x2, x3$), C, Γ and B square to 1 and have eigenvalues $+1$ and -1 eight times each; the time-like gammas ($x4$ to $x7$) square to -1 and have $+i$ and $-i$ eight times each. (Notebook 05a, figure 5.)

```
      "the commutation signs of C, Gamma and B with the eight gammas are as predicted")
fig, ax = plt.subplots(figsize=(8.0, 3.2))
ax.imshow(sign_table, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
for k in range(1, 3):  # white gaps between the rows
    ax.axhline(k - 0.5, color="white", linewidth=2)
for k in range(1, 8):  # and between the columns
    ax.axvline(k - 0.5, color="white", linewidth=2)
for r in range(3):
    for c in range(8):
        ax.text(c, r, f"{sign_table[r, c]:+d}", ha="center", va="center",
                color="white", fontweight="bold")
ax.set_xticks(range(8), [rf"$\gamma^{{({x})}}$" for x in COORDS])
ax.set_yticks(range(3), ["$C$", r"$\Gamma$", "$B$"])
ax.set_title(r"Sign $s$ in $M\gamma^a = s\,\gamma^a M$ (red $+1$ commute, "
             r"blue $-1$ anticommute)")
ax.grid(False)
save_figure(fig, "commutation_signs",
            "The commutation signs: row $M$ ($C$, $\\Gamma$ or $B$) and column "
            "$\\gamma^a$ show the sign $s$ in $M\\gamma^a = s\\,\\gamma^a M$; red "
            "$+1$ means the two commute, blue $-1$ that they anticommute. $C$ "
            "anticommutes with the space-like gammas $x0, x1, x2, x3$ and "
            "commutes with the time-like ones; $\\Gamma$ anticommutes with every "
            "gamma; $B$ anticommutes only with the extra times $x5, x6, x7$.")
```

Out [22]:

```
PASS the commutation signs of C, Gamma and B with the eight gammas are as predicted
```

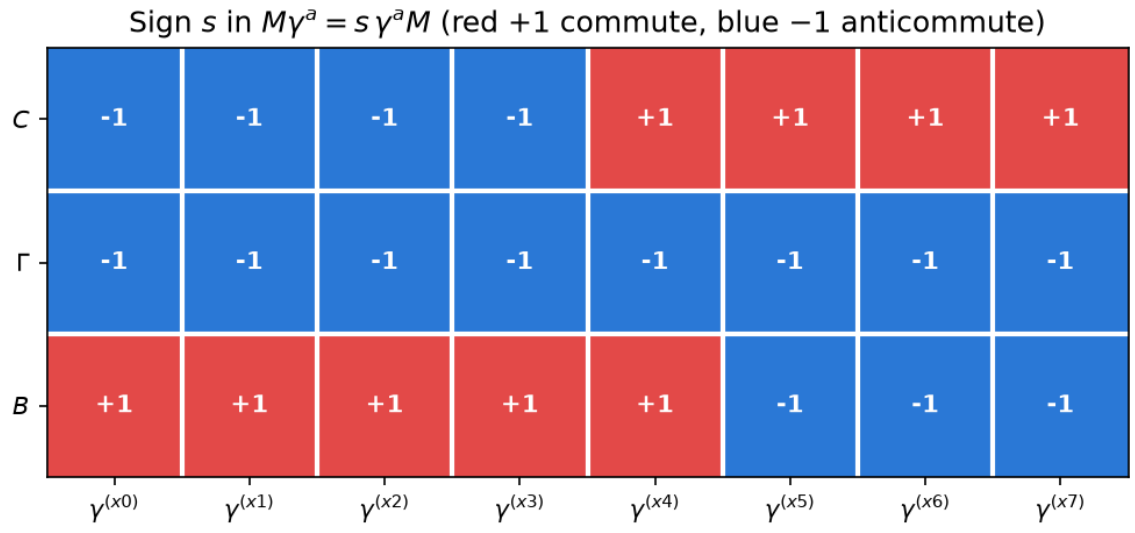

Figure 147: The commutation signs: row M (C, Γ or B) and column γ^a show the sign s in $M\gamma^a = s\,\gamma^a M$; red $+1$ means the two commute, blue -1 that they anticommute. C anticommutes with the space-like gammas $x0, x1, x2, x3$ and commutes with the time-like ones; Γ anticommutes with every gamma; B anticommutes only with the extra times $x5, x6, x7$. (Notebook 05a, figure 6.)

Out [22]:

```
Figure 05a.6 saved as Revision/textbook/figures/05a_6_commutation_signs.png
```

Text cell:

```
## 12. The last check

The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [23]:

```
FIGURES = ["05a_1_c_matrix.png", "05a_2_c_form_signs.png", "05a_3_chirality.png",
           "05a_4_b_matrix.png", "05a_5_spectra.png", "05a_6_commutation_signs.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the six figure files of notebook 05a exist")
all_checks_passed()
```

Out [23]:

```
PASS the six figure files of notebook 05a exist
ALL 26 CHECKS PASSED (notebook 05a)
```

Text cell:

```
## 13. What this notebook showed

- The eight gammas of the Revision record are real signed permutation matrices
  that obey the Clifford relation $\{\gamma^a, \gamma^b\} = 2\eta^{ab}1$; the
  space-like ones ($x0, x1, x2, x3$) are symmetric, the time-like ones ($x4, x5,
  x6, x7$) antisymmetric.
- $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)} =
  \mathrm{diag}(-\sigma, \sigma)$ is real and symmetric, $CC = 1$, it anticommutes
  with the space-like and commutes with the time-like gammas, every $C\gamma^a$ is
  antisymmetric and $C\gamma^a C^{-1} = -(\gamma^a)^T$. Its quadratic form has
  signature $(8, 8)$: it takes both signs.
- $\Gamma = \gamma^{(x0)}\gamma^{(x1)}\cdots\gamma^{(x7)} = \mathrm{diag}(-1_8,
  1_8)$ squares to 1, anticommutes with every gamma, commutes with $C$ and
  satisfies $\Gamma^T C\Gamma = C$ and $\Gamma^T C\gamma^a\Gamma = -C\gamma^a$. Its
  projectors split the 16 components into two halves of 8, and every gamma
  exchanges the halves.
- $B = -iC\gamma^{(x4)}$ is purely imaginary and Hermitian with $BB = 1$, its
  characteristic polynomial is $(\lambda - 1)^8(\lambda + 1)^8$ (signature
  $(8, 8)$), and it anticommutes exactly with the gammas of the three extra times.
- The squaring rule $(-1)^{k(k-1)/2}\prod\eta$ holds for all 255 products of
  different gammas.
- Every check printed with a line "reproduces ..." repeats a check of the Revision
  reports ‘Revision/algebra/reports/python-algebra.json‘ and
  ‘wolfram-algebra.json‘, whose verdicts the notebook read and confirmed. The
  squaring rule for all 255 products, the quadratic form along the two paths, the
  commutation signs of $C$ with each gamma and the eigenvalue counts are this
  notebook’s own computations.
```

5.10 Line-by-line walk-through of Notebook 05a

The notebook has 23 code cells, In [1] to In [23]. This section explains every line of every one of them. The words used are those of the notebook's section 3 (matrix, product, transpose, eigenvalue, signature, heat map and so on), which Sections 5.2 to 5.6 defined. When a cell gives a long caption to `save_figure`, the quotation replaces the caption, or the rest of it, by a line `...`); the full caption is printed under the figure in the complete text of the notebook (Section 5.9). The docstrings of the functions (explained under In [1] below) are left out of all quotations.

In [1], the set-up cell. It is the same in every notebook of the book; only the line that sets `NOTEBOOK_ID` differs. Its first part repeats the complete run instructions of Section 5.8 as comment lines: every line that starts with `#` is a **comment**, which Python skips. The code starts after the line THE SET-UP between two lines of `=` signs. (The function definitions in the cell also carry **docstrings**, texts in triple quotes below the `def` line that say what the function does; they are left out of the quotations here.)

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json`, `os`, `textwrap` and `pathlib` come with Python; `from pathlib import Path` takes the single name `Path` out of `pathlib`. A `Path` is the address of a file or folder, written the same way on every operating system. The plotting package matplotlib is loaded together with its drawing functions under the short name `plt`, and `Image` and `display` come from IPython, the part of Jupyter that runs Python code; together they show a saved picture below a cell.

```
NOTEBOOK_ID = "05a"  # this notebook: chapter 05, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"05a"`. The figure files and the last line of the notebook use it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. `here.parents` lists

the parent folder, its parent and so on; `[here, *here.parents]` is the list that starts with `here` and continues with all of them. The `for` loop takes the folders one after the other; `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If there is none, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The comment lines say in short what the two code lines do. The first code line calls the function and names its result `REPO`; it is never printed, because it differs from computer to computer while the printed output of a notebook must not. The second line chooses the folder below which files are written. `os.environ` holds the **environment variables** (named texts that a program receives from the computer); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and the repository folder otherwise. When you run the notebook the variable is not set; the book's checking tool sets it to a scratch folder, so that a check never changes the repository.

```
def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` is the full path of a repository file, for reading a Revision record. `output_file("Revision/...")` is the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates its folder (and any missing folder above it) and does nothing if it exists.

```
def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of the book; `textwrap.fill` breaks the text at blanks, and every line after the first starts with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings of matplotlib, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets the size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid behind the curves. The braces make a **dictionary**: pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: each name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/05a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary `{}` and a line end into the captions file, which `save_figure` then fills; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far; so the figures are numbered 1, 2, 3, and a cell run twice keeps its number. The file name joins the notebook id, the number and the name, for example `05a_1_c_matrix.png`. `fig.savefig` writes the picture as a PNG file with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time. The caption is stored and the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys). `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
```

```
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list**, an ordered collection written with square brackets. `check` is the function behind every check of the book. `condition` is either `True` or `False`; if it is false, `raise AssertionError(...)` stops the notebook with an error that names the check (Python's own statement `assert` is not used, because Python started with the option `-O` skips it). If it is true, the name is appended to `PASSED` and the line PASS name is printed; when the optional third argument names a Revision record, a second line says which record and check the result reproduces.

```
def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT, followed by the unit when one is given. `all_checks_passed` prints the last line of the notebook with the number of checks that passed (`len` is the length of a list). The last statement prints the single output line of In [1].

In [2], the gammas from the record.

```
import numpy as np  # arrays of numbers, matrices and linear algebra

FIXTURE = "Revision/algebra/gammas.json"  # the Revision record of the gammas
fixture = json.loads(repository_file(FIXTURE).read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # the coordinate names "x0", "x1", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # eta_aa: +1 space-like, -1 time-like
```

numpy is loaded under the short name `np`. `read_text` reads the record file as text and `json.loads` turns the text into Python objects: `fixture` is a dictionary whose keys are the names stored in the record. `fixture["coordinates"]` is the list `["x0", ..., "x7"]`. `zip` pairs each coordinate name with its entry of the list `fixture["eta"]`, and `dict` makes a dictionary of the pairs, so `ETA["x4"]` is -1.

```
# gamma["x0"], ..., gamma["x7"]: the eight 16 x 16 matrices of whole numbers
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the 16 x 16 identity matrix, written 1 in the text
```

The comment line names what the next line builds. A **dictionary comprehension** `{x: ... for x, m in ...}` builds a dictionary in one line: for each pair of a name `x` and a stored matrix `m` (a list of 16 rows of 16 whole numbers) it stores `np.array(m, dtype=np.int64)`, the same matrix as a numpy array of 64-bit whole numbers. With whole numbers every product below is exact: nothing is rounded. `np.eye(16)` is the identity matrix.

```
for x in COORDS:
    kind = "space-like" if ETA[x] == 1 else "time-like"
    rows, columns = gamma[x].shape  # the numbers of rows and of columns
    say(f"gamma^({x}): {rows} x {columns}, eta = {ETA[x]:+d} ({kind})")
```

For each direction the loop chooses the word space-like or time-like (`a if condition else b` is `a` when the condition holds and `b` otherwise), reads the shape of the matrix (16 rows and 16 columns) and prints one line; `{ETA[x]:+d}` writes the whole number with its sign. The eight printed lines show eight 16×16 matrices, four with $\eta = +1$ and four with $\eta = -1$.

In [3], the recorded checks.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])
```

Three more modules of Python are loaded. `REPORT_FILES` names the two reports of the record. The loop reads each report; `report_data["checks"]` is its list of checks, each a dictionary with the keys `name`, `verdict` and `detail`. Every check is stored in `VERDICTS` under the pair (report key, check name); `.lower()` writes the verdict in small letters, because one report writes PASS and the other pass.

```
def recorded(key, name):
    return VERDICTS[(key, name)][0] == "pass"


def record_of(key, name):
    return f"{REPORT_FILES[key]}, check {name}"
```

`recorded("python", "clifford_relation")` is true when the Python report holds that check with the verdict pass (`[0]` takes the first element of the stored pair). `record_of` builds the text that a reproducing check prints after the word reproduces.

```
def check_reproduces(condition, name, record):
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together
```

This is `check` for a check that reproduces a record. Inside the `with` block everything printed goes into the text buffer `collected` instead of the screen; `check` prints its PASS line and its reproduces

line there (or stops the notebook if the condition is false). Then `sys.stdout.write` sends both lines at once. Jupyter delivers printed text in pieces, and one piece keeps the two lines together for the tools that read the notebook.

```
say(f"{len(VERDICTS)} recorded checks were read from the two reports.")
check_reproduces(COORDS == ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
                 and fixture["eta"] == [1, 1, 1, 1, -1, -1, -1, -1]
                 and recorded("wolfram", "eta_in_author_order"),
                 "the coordinates are x0..x7 and eta = diag(+1,+1,+1,+1,-1,-1,-1,-1)",
                 record=record_of("wolfram", "eta_in_author_order"))
```

The first line prints the number of recorded checks read: 80, the 35 of the Python report and the 45 of the Wolfram report. The check compares the list of coordinate names and the list of the signs of η with the expected lists (`==` compares two lists element by element) and requires that the Wolfram report recorded the same fact; `and` is true only when all its parts are true. It prints PASS and the record it reproduces.

In [4], the Clifford relation.

```
def anticommutator(m, n):
    return m @ n + n @ m


failures = []  # the pairs (x, y) for which the relation fails
for x in COORDS:
    for y in COORDS:
        # the right side: 2 eta_xx 1 when x = y, the zero matrix otherwise
        expected = 2 * ETA[x] * I16 if x == y else np.zeros_like(I16)
        if not np.array_equal(anticommutator(gamma[x], gamma[y]), expected):
            failures.append((x, y))
```

In Python the sign `@` multiplies two matrices, so `anticommutator(m, n)` is $mn + nm$. Two loops run through all $8 \times 8 = 64$ ordered pairs of directions. For each pair the right side of the Clifford relation is $2\eta_{xx}1$ when the two directions are equal and the zero matrix (`np.zeros_like(I16)`, zeros of the same shape) otherwise. `np.array_equal` is true only when all 256 entries agree; a pair for which they do not is appended to `failures`.

```
say(f"pairs tested: {len(COORDS) ** 2}; pairs that fail: {len(failures)}")
check_reproduces(failures == [] and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

`**` is the power in Python, so `len(COORDS) ** 2` is 64. The printed line reports 64 pairs tested and 0 failures, and the check requires the list of failures to be empty.

In [5], signed permutations and the symmetry pattern.

```
values = sorted({int(v) for x in COORDS for v in gamma[x].flat})  # every entry once
one_per_row = all((gamma[x] != 0).sum(axis=1).tolist() == [1] * 16 for x in COORDS)
one_per_column = all((gamma[x] != 0).sum(axis=0).tolist() == [1] * 16 for x in COORDS)
say(f"the entries of the gammas are {values}")
```

`gamma[x].flat` runs through the 256 entries of a matrix. The braces make a **set**, which keeps every value only once; `sorted` turns it into an ordered list, here `[-1, 0, 1]`. `gamma[x] != 0` is a table of true and false values (true where the entry is not zero); `.sum(axis=1)` counts the true values in each row and `.sum(axis=0)` in each column; `.tolist()` turns the result into a list, and `[1] * 16` is the list of sixteen ones. `all(...)` is true when the condition holds for every direction.

```
check_reproduces(values == [-1, 0, 1] and one_per_row and one_per_column
                 and recorded("python", "reality_signed_permutations"),
                 "every gamma is real and a signed permutation matrix",
                 record=record_of("python", "reality_signed_permutations"))
symmetric = [x for x in COORDS if np.array_equal(gamma[x].T, gamma[x])]
antisymmetric = [x for x in COORDS if np.array_equal(gamma[x].T, -gamma[x])]
say("symmetric gammas: " + ", ".join(symmetric))
say("antisymmetric gammas: " + ", ".join(antisymmetric))
check_reproduces(symmetric == ["x0", "x1", "x2", "x3"]
                 and antisymmetric == ["x4", "x5", "x6", "x7"]
                 and recorded("python", "symmetry_pattern"),
                 "(gamma^a)^T = eta_aa gamma^a: space-like symmetric, time-like "
                 "antisymmetric",
                 record=record_of("python", "symmetry_pattern"))
```

The first check combines the three facts. Then two **list comprehensions** collect the directions whose matrix equals its transpose (`.T` is the transpose) and those whose matrix equals minus its transpose; `", ".join(...)` writes a list of names separated by commas. The printed lines are `x0, x1, x2, x3` and `x4, x5, x6, x7`, and the second check requires exactly these lists: the space-like gammas are symmetric and the time-like ones antisymmetric.

In [6], three helpers.

```
from matplotlib.colors import LinearSegmentedColormap

# Colours of the heat maps: -1 blue, 0 light grey, +1 red (a diverging colour scale).
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
```

A **colour map** assigns a colour to every number of a range. `from_list` makes one that runs smoothly through the three listed colours (each written as a hexadecimal code of its red, green and blue parts): blue for the lowest value, light grey in the middle, red for the highest.

```
def heat_map(ax, matrix, title, halves=True, row_label=True):
    size = matrix.shape[0]
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ticks = [0, 3, 7, 11, 15] if size == 16 else [0, 3, 7]  # counted from 0
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])  # labels counted from 1
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)  # no grid lines across the coloured squares
    if halves and size == 16:
```

```
        ax.axhline(7.5, color="black", linewidth=0.8)  # between rows 8 and 9
        ax.axvline(7.5, color="black", linewidth=0.8)  # between columns 8 and 9
    return image
```

`heat_map` draws a matrix on the **axes** `ax` (one picture area of a figure). `ax.imshow` draws one coloured square per entry with the colour map `SIGNS`, so that -1 is blue, 0 grey and $+1$ red (`vmin` and `vmax` fix the range). Python counts rows and columns from 0, the book from 1: the tick marks stand at the positions 0, 3, 7, 11, 15 and are labelled 1, 4, 8, 12, 16 (for an 8×8 matrix only 1, 4, 8). The optional argument `row_label=False` leaves out the word row on the second and later pictures of a row of pictures. For a 16×16 matrix two thin black lines at the position 7.5 (between the eighth and the ninth row and column) show the two halves. The function returns the drawn image, which a colour bar needs.

```
def commutation_sign(m, n):
    if np.array_equal(m @ n, n @ m):
        return 1
    if np.array_equal(m @ n, -(n @ m)):
        return -1
    return 0


def product(directions):
    result = I16
    for d in directions:
        result = result @ gamma[d]
    return result


say("helpers heat_map, commutation_sign and product are defined")
```

`commutation_sign(m, n)` returns $+1$ when $mn = nm$, -1 when $mn = -nm$ and 0 when neither holds. `product(["x0", "x1"])` multiplies the gammas of the listed directions in the order of the list, starting from the identity. The cell prints one line.

In [7], the charge matrix.

```
C = product(["x0", "x1", "x2", "x3"])  # the four space-like gammas
C_record = np.array(fixture["C"], dtype=np.int64)  # the C stored in the record
check_reproduces(np.array_equal(C, C_record) and recorded("wolfram", "C_definition"),
                 "C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3) equals the recorded C",
                 record=record_of("wolfram", "C_definition"))
```

$C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$ is computed and compared with the matrix C that the record stores.

```
I4, Z4 = np.eye(4, dtype=np.int64), np.zeros((4, 4), dtype=np.int64)
I8, Z8 = np.eye(8, dtype=np.int64), np.zeros((8, 8), dtype=np.int64)
sigma = np.block([[Z4, I4], [I4, Z4]])  # the author's 8 x 8 matrix sigma
check_reproduces(np.array_equal(C, np.block([[-sigma, Z8], [Z8, sigma]]))
                 and recorded("python", "C_equals_notebook_sigma16"),
                 "C = diag(-sigma, sigma), called sigma16 by the author",
```

```
                record=record_of("python", "C_equals_notebook_sigma16"))
check_reproduces(np.array_equal(C.T, C) and np.array_equal(C @ C, I16)
                and recorded("python", "C_real_symmetric_involution"),
                "C is real and symmetric, and C C = 1 (so C is its own inverse)",
                record=record_of("python", "C_real_symmetric_involution"))
```

The first two lines make identity and zero matrices of sizes 4 and 8 (a line `A, B = p, q` names two values at once). `np.block` assembles a matrix from a list of rows of blocks: `[[Z4, I4], [I4, Z4]]` is the author's σ, and `[[-sigma, Z8], [Z8, sigma]]` is $\mathrm{diag}(-\sigma, \sigma)$. The two checks confirm (C7), then (C2) and (C3) of Section 5.4.

In [8], the picture of C.

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.4), width_ratios=[2, 1])
heat_map(axes[0], C, r"$C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$")
image = heat_map(axes[1], sigma, r"$\sigma$ (8 x 8)", halves=False,
                row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.75, label="matrix entry")
save_figure(fig, "c_matrix",
            "Heat maps of the charge matrix $C$ (left, 16 by 16) and of the "
            ...)
```

`plt.subplots(1, 2, ...)` makes a figure with one row of two picture areas; `width_ratios=[2, 1]` makes the first twice as wide as the second. A string that starts with `r` is a **raw string**: its backslashes are kept as they are, which the mathematical titles (typeset by matplotlib between dollar signs) need. The first heat map shows C, the second σ without the half lines. `fig.colorbar` adds a bar that shows which colour means which value, with ticks at -1, 0, $+1$, shrunk to three quarters of the height. `save_figure` saves the figure as `05a_1_c_matrix.png` with its caption (the full caption is printed in the notebook text above; Python joins strings written next to each other into one). In the figure, look at the top-left block: blue squares in the pattern of σ; the bottom-right block: red squares in the same pattern; the picture is mirror-symmetric about its diagonal, because $C^T = C$.

In [9], how C moves past the gammas.

```
signs_C = {x: commutation_sign(C, gamma[x]) for x in COORDS}  # +1 or -1 for each x
say("C gamma^(x) = s gamma^(x) C with s = "
    + ", ".join(f"{x}: {signs_C[x]:+d}" for x in COORDS))
check(all(signs_C[x] == -ETA[x] for x in COORDS),
      "C gamma^a = -eta_aa gamma^a C (anticommutes with space-like, commutes with "
      "time-like gammas)")
```

`signs_C` stores, for each direction, the sign s in $C\gamma^a = s\,\gamma^a C$. The printed line lists them: -1 for $x0$, $x1$, $x2$, $x3$ and $+1$ for $x4$ to $x7$. The check is (C1): $s = -\eta_{aa}$ for every direction. It is this notebook's own check (no record holds it in this form), so it prints no reproduces line.

```
check_reproduces(all(np.array_equal((C @ gamma[x]).T, -(C @ gamma[x])) for x in COORDS)
                and recorded("python", "C_gamma_antisymmetric"),
                "C gamma^a is antisymmetric for every a",
```

```
                record=record_of("python", "C_gamma_antisymmetric"))
check_reproduces(all(np.array_equal(C @ gamma[x] @ C, -gamma[x].T) for x in COORDS)
                and recorded("python", "C_conjugation"),
                "C gamma^a C^-1 = -(gamma^a)^T for every a",
                record=record_of("python", "C_conjugation"))
```

The two checks are (C4) and (C5). The second writes $C\gamma^a C$ for $C\gamma^a C^{-1}$, because $C^{-1} = C$.

In [10], the squaring rule on all 255 products.

```
import itertools  # lists all subsets of a given size


def square_sign(directions):
    k = len(directions)
    s = (-1) ** (k * (k - 1) // 2)  # // is division of whole numbers
    for d in directions:
        s *= ETA[d]
    return s
```

`square_sign` computes the sign that Rule 2 of Section 5.3 predicts for the square of the product of the listed directions: $(-1)^{k(k-1)/2}$ times the product of the η_{dd}. `//` divides whole numbers without a remainder ($k(k-1)$ is always even), and `s *= ETA[d]` multiplies `s` by η_{dd}.

```
agree = 0  # how many subsets obey the rule
for k in range(1, 9):
    for subset in itertools.combinations(COORDS, k):
        p = product(subset)
        if np.array_equal(p @ p, square_sign(subset) * I16):
            agree += 1
say(f"subsets tested: 255; subsets that obey the squaring rule: {agree}")
```

`range(1, 9)` runs through $k = 1, \ldots, 8$. `itertools.combinations(COORDS, k)` lists every subset of k directions, each in the order $x0, \ldots, x7$; there are $\binom{8}{k}$ of them, $2^8 - 1 = 255$ in all. For each subset the cell multiplies the gammas, squares the product and compares it with the predicted sign times the identity; `agree += 1` adds one to the counter. The printed line reports 255 of 255.

```
for label, dirs in [("C", ["x0", "x1", "x2", "x3"]),
                    ("Gamma", ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]),
                    ("C gamma^(x4)", ["x0", "x1", "x2", "x3", "x4"])]:
    say(f"({label})^2 = {square_sign(dirs):+d} times 1 (k = {len(dirs)})")
check(agree == 255, "the squaring rule holds for all 255 products of different gammas")
```

The loop prints the predicted squares of the three products used in this chapter: $C^2 = +1$, $\Gamma^2 = +1$ and $(C\gamma^{(x4)})^2 = -1$, the facts behind (C3), (X1) and (B3). The check requires all 255 subsets to agree.

In [11], the quadratic form of C.

```
t = np.linspace(0.0, np.pi, 401)  # 401 equally spaced values of t from 0 to pi
e = np.eye(16)  # e[r - 1] is the column e_r (a 1 in row r)
u = np.outer(np.cos(t), e[0]) + np.outer(np.sin(t), e[4])  # u(t), one per row
v = np.outer(np.cos(t), e[8]) + np.outer(np.sin(t), e[12])  # v(t), one per row
```

`np.linspace(0.0, np.pi, 401)` is an array of 401 numbers from 0 to π with equal steps. `e[0]` is the first row of the identity, the column e_1 written as a row; Python counts from 0, so e_5 is `e[4]`. `np.outer(a, b)` is the table whose row j is the number `a[j]` times the row `b`; so `u` has 401 rows, row j being $\cos t_j\, e_1 + \sin t_j\, e_5$, and `v` holds $\cos t\, e_9 + \sin t\, e_{13}$ in the same way.

```
form_u = np.einsum("tr,rc,tc->t", u, C, u)  # u^T C u for every t at once
form_v = np.einsum("tr,rc,tc->t", v, C, v)  # v^T C v for every t at once
say(f"C_(1,5) = {C[0, 4]:+d} and C_(9,13) = {C[8, 12]:+d}")
check(C[0, 4] == -1 and C[8, 12] == 1
      and np.max(np.abs(form_u + np.sin(2 * t))) < 1e-14
      and np.max(np.abs(form_v - np.sin(2 * t))) < 1e-14,
      "u^T C u = -sin 2t and v^T C v = +sin 2t along the two paths")
```

`np.einsum` evaluates a sum over repeated letters: `"tr,rc,tc->t"` means $\sum_{r,c} u_{tr} C_{rc} u_{tc}$ for every t, that is $u^T C u$ at all 401 times at once. The printed line shows $C_{1,5} = -1$ and $C_{9,13} = +1$ (entries `[0, 4]` and `[8, 12]`). The check compares the computed forms with $-\sin 2t$ and $+\sin 2t$: the largest difference (`np.max(np.abs(...))`) must be below 10^{-14}, because the numbers are now floating-point numbers, which keep about 16 significant digits.

```
eigenvalues_C = np.linalg.eigvalsh(C.astype(float))  # 16 real eigenvalues, sorted
n_plus = int(np.sum(np.abs(eigenvalues_C - 1.0) < 1e-9))  # how many are +1
n_minus = int(np.sum(np.abs(eigenvalues_C + 1.0) < 1e-9))  # how many are -1
say(f"trace of C = {int(np.trace(C))}; eigenvalues +1: {n_plus}, -1: {n_minus}")
check(np.trace(C) == 0 and n_plus == 8 and n_minus == 8,
      "C has eight eigenvalues +1 and eight -1: signature (8, 8)")
```

`np.linalg.eigvalsh` computes the eigenvalues of a symmetric matrix (`.astype(float)` turns the whole numbers into floating-point numbers first). `np.abs(eigenvalues_C - 1.0) < 1e-9` is true for every eigenvalue within 10^{-9} of $+1$, and `np.sum` counts the true values. The printed line shows trace 0, eight $+1$ and eight -1: (C6) of Section 5.4.

In [12], the picture of the two forms.

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(t, form_u, color="#2a78d6", linewidth=2,
        label=r"$u^T C u$ with $u = \cos t\, e_1 + \sin t\, e_5$")
ax.plot(t, form_v, color="#eb6834", linewidth=2, linestyle="--",
        label=r"$v^T C v$ with $v = \cos t\, e_9 + \sin t\, e_{13}$")
ax.plot(t, np.einsum("tr,tr->t", u, u), color="#1baf7a", linewidth=2,
        linestyle=":", label=r"$u^T u$ (ordinary squared length)")
ax.axhline(0.0, color="black", linewidth=0.6)
```

`ax.plot(xs, ys, ...)` draws a curve through the points; `color` gives its colour, `linewidth` its thickness, `linestyle` makes it dashed or dotted, and `label` is its name in the legend. The third

curve is $u^T u$ (`"tr,tr->t"` sums $u_{tr} u_{tr}$ over r), which is 1 everywhere. `ax.axhline(0.0)` draws the horizontal line at height 0.

```
ax.set_xticks([0, np.pi / 4, np.pi / 2, 3 * np.pi / 4, np.pi],
              ["0", r"$\pi/4$", r"$\pi/2$", r"$3\pi/4$", r"$\pi$"])
ax.set_xlabel("$t$ (radians)")
ax.set_ylabel("value of the quadratic form")
ax.set_title("The form of $C$ takes both signs; the ordinary length does not")
# the legend goes below the picture, where it hides no curve
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=1)
save_figure(fig, "c_form_signs",
            ...)
```

The tick marks of the horizontal axis are placed at multiples of $\pi/4$ and labelled in radians; the axes and the picture get their labels and title. `ax.legend` draws the box that names the curves; `bbox_to_anchor=(0.5, -0.17)` puts its top centre below the picture, where it hides no curve. The figure, `05a_2_c_form_signs.png`, shows the solid curve $-\sin 2t$ going down to -1, the dashed curve $+\sin 2t$ going up to $+1$, and the dotted line at 1: the form of C takes both signs on unit columns, the ordinary length does not.

In [13], the chirality.

```
Gamma = product(["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"])  # all eight
check_reproduces(np.array_equal(Gamma, np.array(fixture["Gamma"], dtype=np.int64))
                 and np.array_equal(Gamma, np.block([[-I8, Z8], [Z8, I8]]))
                 and recorded("python", "chirality_diag"),
                 "Gamma = gamma^(x0) gamma^(x1) ... gamma^(x7) = diag(-1_8, 1_8), as "
                 "recorded",
                 record=record_of("python", "chirality_diag"))
```

Γ is the product of all eight gammas in the author's order. The check compares it with the recorded matrix and with $\mathrm{diag}(-1_8, 1_8)$: (X9) of Section 5.5.

```
check_reproduces(np.array_equal(Gamma @ Gamma, I16)
                 and all(commutation_sign(Gamma, gamma[x]) == -1 for x in COORDS)
                 and recorded("python", "chirality_anticommutes"),
                 "Gamma Gamma = 1 and Gamma anticommutes with every gamma^a",
                 record=record_of("python", "chirality_anticommutes"))
check_reproduces(np.array_equal(Gamma, C @ product(["x4", "x5", "x6", "x7"]))
                 and recorded("python", "chirality_eq_C_times_time_gammas"),
                 "Gamma = C gamma^(x4) gamma^(x5) gamma^(x6) gamma^(x7)",
                 record=record_of("python", "chirality_eq_C_times_time_gammas"))
```

These are (X1) with (X2), and (X4).

```
check_reproduces(commutation_sign(C, Gamma) == 1 and np.array_equal(Gamma.T, Gamma)
                 and np.array_equal(Gamma.T @ C @ Gamma, C)
                 and all(np.array_equal(Gamma.T @ C @ gamma[x] @ Gamma, -(C @ gamma[x]))
                         for x in COORDS)
```

```
                and recorded("python", "chirality_C_relation"),
                "C Gamma = Gamma C, Gamma^T C Gamma = C, Gamma^T C gamma^a Gamma = -C "
                "gamma^a",
                record=record_of("python", "chirality_C_relation"))
```

This check combines (X5), (X6), (X7) and (X8), the matrix input of the pairing theorem T1.

In [14], the projectors.

```
P_minus = (I16 - Gamma) // 2  # the projector onto components 1 to 8
P_plus = (I16 + Gamma) // 2  # the projector onto components 9 to 16
check(np.array_equal(P_minus, np.block([[I8, Z8], [Z8, Z8]]))
      and np.array_equal(P_plus, np.block([[Z8, Z8], [Z8, I8]]))
      and np.array_equal(P_minus @ P_minus, P_minus)
      and np.array_equal(P_plus @ P_plus, P_plus)
      and not np.any(P_minus @ P_plus) and np.array_equal(P_minus + P_plus, I16)
      and np.trace(P_minus) == 8 and np.trace(P_plus) == 8,
      "P_- = diag(1_8, 0) and P_+ = diag(0, 1_8) are complementary projectors of rank 8")
```

The entries of $1 \pm \Gamma$ are 0 and 2, so the division by 2 with `//` is exact and keeps whole numbers. The check confirms the shapes $\mathrm{diag}(1_8, 0)$ and $\mathrm{diag}(0, 1_8)$, the projector rules $P_- P_- = P_-$, $P_+ P_+ = P_+$, $P_- P_+ = 0$ (`np.any` is true when some entry is not zero, so `not np.any(...)` means the matrix is zero) and $P_- + P_+ = 1$, and that each keeps 8 components (trace 8).

```
check_reproduces(all(np.array_equal(gamma[x] @ P_minus, P_plus @ gamma[x])
                     for x in COORDS)
                and all(not np.any(gamma[x][:8, :8]) and not np.any(gamma[x][8:, 8:])
                        for x in COORDS)
                and recorded("python", "reflections_exchange_halves")
                and recorded("wolfram", "Spin_Pin_reflection_swaps_halves"),
                "gamma^a P_- = P_+ gamma^a: every gamma exchanges the two halves",
                record=record_of("python", "reflections_exchange_halves"))
```

The check is $\gamma^a P_- = P_+ \gamma^a$ for every direction, and that the two diagonal blocks of every gamma are zero. `gamma[x][:8, :8]` is the top-left block: the **slice** `:8` takes the rows (or columns) 0 to 7 in Python's counting, the book's 1 to 8; `8:` takes the rest.

In [15], the picture of the chirality.

```
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.9))
heat_map(axes[0], Gamma, r"$\Gamma$")
heat_map(axes[1], P_minus, r"$P_- = (1 - \Gamma)/2$", row_label=False)
heat_map(axes[2], P_plus, r"$P_+ = (1 + \Gamma)/2$", row_label=False)
image = heat_map(axes[3], gamma["x1"], r"$\gamma^{(x1)}$ for comparison",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "chirality",
            ...)
```

Four heat maps in one row and a colour bar, saved as `05a_3_chirality.png`. In the figure, Γ is blue on the first eight diagonal places and red on the last eight; P_- and P_+ are red on the diagonal of one half; $\gamma^{(x1)}$ has its coloured squares only in the two off-diagonal blocks.

In [16], the matrix B.

```
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4); 1j is the imaginary unit
B_record = (np.array(fixture["B"]["re"], dtype=np.int64)
            + 1j * np.array(fixture["B"]["im"], dtype=np.int64))
check_reproduces(np.array_equal(B, B_record) and recorded("wolfram", "B_definition"),
                 "B = -i C gamma^(x4) equals the recorded B",
                 record=record_of("wolfram", "B_definition"))
```

Python writes the imaginary unit as `1j`. The record stores the real and the imaginary part of B separately (`fixture["B"]["re"]` and `fixture["B"]["im"]`), and the cell assembles them into one complex matrix before comparing. The entries of B are 0, i and $-i$, which the computer stores exactly.

```
check_reproduces(not np.any(B.real) and np.array_equal(B.conj().T, B)
                 and np.array_equal(B @ B, I16) and np.trace(B) == 0
                 and recorded("python", "B_hermitian_involution_signature")
                 and recorded("wolfram", "B_Hermitian")
                 and recorded("wolfram", "B_squared_identity"),
                 "B is purely imaginary and Hermitian, B B = 1 and tr B = 0",
                 record=record_of("python", "B_hermitian_involution_signature"))
```

`B.real` is the real part (it must be zero everywhere), and `B.conj().T` the conjugate transpose $B^\dagger$. The check combines (B1) to (B4).

In [17], the characteristic polynomial of B.

```
import sympy as sp  # exact algebra with symbols

lam = sp.Symbol("lam")  # the variable lambda of the polynomial
B_exact = sp.I * sp.Matrix((-(C @ gamma["x4"])).tolist())  # B with exact sympy numbers
polynomial = sp.factor(B_exact.charpoly(lam).as_expr())  # det(lam 1 - B), factored
say(f"characteristic polynomial of B: {polynomial}")
```

sympy computes with symbols and exact numbers. `sp.Symbol("lam")` is the letter λ. Since $B = -iC\gamma^{(x4)} = i(-C\gamma^{(x4)})$, the matrix is built as sympy's imaginary unit `sp.I` times the whole-number matrix $-C\gamma^{(x4)}$ (`.tolist()` turns the numpy array into a list of rows, which `sp.Matrix` accepts). `charpoly(lam)` computes $\det(\lambda 1 - B)$ exactly, `.as_expr()` writes it as an ordinary expression, and `sp.factor` writes it as a product of factors. The printed result is `(lam - 1)**8*(lam + 1)**8`, sympy's way of writing $(\lambda - 1)^8(\lambda + 1)^8$.

```
detail = VERDICTS[("python", "B_hermitian_involution_signature")][1]
check_reproduces(sp.expand(polynomial - (lam - 1) ** 8 * (lam + 1) ** 8) == 0
                 and f"= {polynomial}" in detail
                 and recorded("wolfram", "B_signature_8_8"),
                 "det(lam 1 - B) = (lam - 1)^8 (lam + 1)^8: signature (8, 8)",
                 record=record_of("python", "B_hermitian_involution_signature"))
```

`detail` is the detail text of the recorded check, which states the polynomial. The check requires that the difference between the computed polynomial and $(\lambda-1)^8(\lambda+1)^8$ multiplies out (`sp.expand`) to exactly 0, and that the text `= (lam - 1)**8*(lam + 1)**8` occurs in the recorded detail (`in` tests whether one string occurs in another): the notebook and the record wrote the same polynomial, character by character.

In [18], how B moves past the gammas.

```
signs_B = {x: commutation_sign(B, gamma[x]) for x in COORDS}
commute = [x for x in COORDS if signs_B[x] == 1]
anticommute = [x for x in COORDS if signs_B[x] == -1]
say("B commutes with gamma^(x) for x = " + ", ".join(commute))
say("B anticommutes with gamma^(x) for x = " + ", ".join(anticommute))
check_reproduces(commute == ["x0", "x1", "x2", "x3", "x4"]
                 and anticommute == ["x5", "x6", "x7"]
                 and recorded("python", "B_gamma_relations"),
                 "B commutes with gamma^a for a = x0, x1, x2, x3, x4 and anticommutes "
                 "for x5, x6, x7",
                 record=record_of("python", "B_gamma_relations"))
```

The sign of B with each gamma is computed and the directions are sorted into two lists. The printed lines and the check confirm (B5): B anticommutes exactly with the gammas of the three extra times.

In [19], the picture of B.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], gamma["x4"], r"$\gamma^{(x4)}$ (antisymmetric)")
heat_map(axes[1], C @ gamma["x4"], r"$C\gamma^{(x4)}$ (antisymmetric)",
         row_label=False)
image = heat_map(axes[2], B.imag, r"imaginary part of $B = -iC\gamma^{(x4)}$",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "b_matrix",
            ...)
```

Three heat maps: $\gamma^{(x4)}$, $C\gamma^{(x4)}$ and the imaginary part of B (`B.imag`), which is $-C\gamma^{(x4)}$. In `05a_4_b_matrix.png` every picture changes colour when it is mirrored about its diagonal: all three are antisymmetric, and B, being i times an antisymmetric real matrix, is Hermitian.

In [20], the spectra.

```
TARGETS = [1, -1, 1j, -1j]  # the four possible eigenvalues
TARGET_NAMES = ["+1", "-1", "+i", "-i"]
matrices = {f"gamma^({x})": gamma[x] for x in COORDS}
matrices.update({"C": C, "Gamma": Gamma, "B": B})
counts = {}  # name -> [how many eigenvalues near +1, -1, +i, -i]
for name, m in matrices.items():
    eigenvalues = np.linalg.eigvals(m.astype(complex))
    counts[name] = [int(np.sum(np.abs(eigenvalues - z) < 1e-9)) for z in TARGETS]
    say(f"{name:12} " + "  ".join(f"{n}: {c}" for n, c in
                                   zip(TARGET_NAMES, counts[name])))
```

`matrices` collects the eleven matrices under their names (`update` adds three more entries). For each, `np.linalg.eigvals` computes its 16 eigenvalues as complex floating-point numbers, and the list comprehension counts how many lie within 10^{-9} of each of $+1$, -1, $+i$, $-i$ (for a complex number `np.abs` is its distance from 0). `{name:12}` pads the name to 12 characters, so that the eleven printed lines form a table.

```
predicted = {name: ([8, 8, 0, 0] if name in ("C", "Gamma", "B")
                    or ETA[name[7:9]] == 1 else [0, 0, 8, 8]) for name in matrices}
check(counts == predicted and all(np.trace(gamma[x]) == 0 for x in COORDS),
      "eigenvalues: +1 and -1 eight times each for the space-like gammas, C, Gamma "
      "and B; +i and -i eight times each for the time-like gammas")
```

The prediction of Rule 6: eight $+1$ and eight -1 for C, Γ, B and the space-like gammas, eight $+i$ and eight $-i$ for the time-like ones. For a gamma, `name[7:9]` cuts the two characters at positions 7 and 8 out of a name such as `gamma^(x1)`, which are `x1`; the `or` looks at it only when the name is not one of the three (Python evaluates `a or b` from the left and stops when `a` is true). The check compares all counts with the prediction and checks that every gamma has trace 0, the condition of Rule 6.

In [21], the picture of the spectra.

```
from matplotlib.colors import LinearSegmentedColormap as Ramp

names = list(matrices)  # the 11 matrices in the order of the table
table = np.array([counts[name] for name in names])  # 11 rows, 4 columns
COUNT_COLORS = Ramp.from_list("counts", ["#f0efec", "#2a78d6"])  # 0 grey, 8 blue
fig, ax = plt.subplots(figsize=(6.0, 6.4))
ax.imshow(table, cmap=COUNT_COLORS, vmin=0, vmax=8, aspect="auto")
for k in range(1, table.shape[0]):  # white gaps between the rows
    ax.axhline(k - 0.5, color="white", linewidth=2)
for k in range(1, table.shape[1]):  # and between the columns
    ax.axvline(k - 0.5, color="white", linewidth=2)
```

The table of counts becomes an array of 11 rows and 4 columns and is drawn as coloured squares, grey for 0 and blue for 8 (`as Ramp` gives the imported name a shorter name; `aspect="auto"` lets the squares be stretched to fill the picture). White lines between the rows and columns separate the squares.

```
for r in range(table.shape[0]):
    for c in range(table.shape[1]):
        ax.text(c, r, str(table[r, c]), ha="center", va="center",
                color="white" if table[r, c] == 8 else "black")
ax.set_xticks(range(4), ["$+1$", "$-1$", "$+i$", "$-i$"])
labels = [rf"$\gamma^{{({x})}}$" for x in COORDS] + ["$C$", r"$\Gamma$", "$B$"]
ax.set_yticks(range(len(names)), labels)
ax.set_xlabel("eigenvalue")
ax.set_title("How many times each eigenvalue occurs (16 in each row)")
ax.grid(False)
save_figure(fig, "spectra",
            ...)
```

`ax.text(c, r, ...)` writes each count in the middle of its square (`ha` and `va` centre it horizontally and vertically), in white on blue and in black on grey. The column labels are the four eigenvalues; the row labels are made by an `rf`-string, raw and formatted at once, in which a doubled brace `{{` stands for a single brace in the result, so that `$\gamma^{(x1)}$` and so on come out. The figure `05a_5_spectra.png` shows blue squares with 8 in the columns $+1$ and -1 for the space-like gammas, C, Γ and B, and in the columns $+i$ and $-i$ for the time-like gammas.

In [22], the table of commutation signs.

```
rows = {"C": signs_C, "Gamma": {x: commutation_sign(Gamma, gamma[x]) for x in COORDS},
        "B": signs_B}
sign_table = np.array([[rows[name][x] for x in COORDS] for name in rows])
expected_table = np.array([[-ETA[x] for x in COORDS], [-1] * 8,
                           [1, 1, 1, 1, 1, -1, -1, -1]])
check(np.array_equal(sign_table, expected_table),
      "the commutation signs of C, Gamma and B with the eight gammas are as predicted")
```

The signs of C (In [9]), of Γ (computed here) and of B (In [18]) with the eight gammas form a table of 3 rows and 8 columns. The predicted table has the rows $-\eta_{aa}$ (C1), eight times -1 (X2) and the pattern of (B5). The check compares them.

```
fig, ax = plt.subplots(figsize=(8.0, 3.2))
ax.imshow(sign_table, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
for k in range(1, 3):  # white gaps between the rows
    ax.axhline(k - 0.5, color="white", linewidth=2)
for k in range(1, 8):  # and between the columns
    ax.axvline(k - 0.5, color="white", linewidth=2)
for r in range(3):
    for c in range(8):
        ax.text(c, r, f"{sign_table[r, c]:+d}", ha="center", va="center",
                color="white", fontweight="bold")
ax.set_xticks(range(8), [rf"$\gamma^{{({x})}}$" for x in COORDS])
ax.set_yticks(range(3), ["$C$", r"$\Gamma$", "$B$"])
ax.set_title(r"Sign $s$ in $M\gamma^a = s\,\gamma^a M$ (red $+1$ commute, "
             r"blue $-1$ anticommute)")
ax.grid(False)
save_figure(fig, "commutation_signs",
            ...)
```

The drawing works as in In [21], with the colour map of the heat maps: red squares $+1$ (commute), blue squares -1 (anticommute), each with its sign written in bold white. In `05a_6_commutatio n_signs.png` the row C is blue for $x0$, $x1$, $x2$, $x3$ and red for $x4$ to $x7$; the row Γ is blue everywhere; the row B is blue only for the extra times $x5$, $x6$, $x7$.

In [23], the last check.

```
FIGURES = ["05a_1_c_matrix.png", "05a_2_c_form_signs.png", "05a_3_chirality.png",
           "05a_4_b_matrix.png", "05a_5_spectra.png", "05a_6_commutation_signs.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the six figure files of notebook 05a exist")
all_checks_passed()
```

The cell checks that the six figure files exist and prints the last line, ALL 26 CHECKS PASSED (notebook 05a). The 26 checks are: one in In [3], one in In [4], two in In [5], three in In [7], three in In [9], one in In [10], two in In [11], four in In [13], two in In [14], two in In [16], and one each in In [17], In [18], In [20], In [22] and In [23]. The cells In [1], In [2], In [6], In [8], In [12], In [15], In [19] and In [21] check nothing.

5.11 Groups and representations from zero

This section introduces the words of group theory that the rest of the chapter needs, each with a proof of the one fact used later. All groups here are groups of matrices.

Group. A **group** of $n \times n$ matrices is a set G of invertible matrices that contains the identity, the product of any two of its members and the inverse of each member. Examples: the two matrices $\{1, -1\}$; all invertible $n \times n$ matrices; the rotations of the plane $R(\varphi) = \begin{pmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{pmatrix}$, because $R(\varphi)R(\psi) = R(\varphi + \psi)$ (the addition theorems of sine and cosine) and $R(\varphi)^{-1} = R(-\varphi)$. A **subgroup** is a subset that is itself a group. The group **generated** by some matrices is the set of all finite products of these matrices and of their inverses; it is the smallest group that contains them.

Representation. A group of 16×16 matrices acts on columns of 16 numbers by multiplication, $\Psi \to g\Psi$. One says that the 16 components **carry a representation** of the group. A **subspace** is a set of columns that contains every sum and every multiple of its members (for example all columns whose components 9 to 16 are zero). A subspace W is **invariant** when gw lies in W for every g of the group and every w of W. The zero subspace $\{0\}$ and the set of all columns are always invariant. The representation is **irreducible** when there is no other invariant subspace: the 16 components cannot be cut into smaller pieces that the group keeps apart.

Span. A **combination** of matrices $M_1, \ldots, M_N$ is $c_1 M_1 + \cdots + c_N M_N$ with numbers c_j; the **span** is the set of all combinations. The matrices are **linearly independent** when no one of them is a combination of the others; then their span has **dimension** N. Writing each 16×16 matrix as one row of 256 numbers, N matrices are independent exactly when the N rows have **rank** N (the rank of a list of rows is the number of independent ones among them).

Commutant, intertwiner, equivalence. The **commutant** of a list of matrices is the set of all matrices X that commute with every matrix of the list. Two groups (or lists) of matrices $g \to \rho_1(g)$ and $g \to \rho_2(g)$, labelled by the same elements g, have an **intertwiner** T when $T\rho_1(g) = \rho_2(g)T$ for every g; they are **equivalent** when an invertible intertwiner exists, because then $\rho_2(g) = T\rho_1(g)T^{-1}$ and the two are the same matrices written in another basis.

Lemma 1. For an intertwiner T, the set of columns v with $Tv = 0$ (its **kernel**) is invariant under ρ_1, and the set of columns Tv (its **image**) is invariant under ρ_2. Proof: if $Tv = 0$, then $T\rho_1(g)v = \rho_2(g)Tv = 0$, so $\rho_1(g)v$ lies in the kernel; and $\rho_2(g)(Tv) = T(\rho_1(g)v)$ lies in the image.

Lemma 2 (Schur's lemma). An intertwiner T between two irreducible representations is either zero or invertible. Proof: by Lemma 1 and irreducibility, the kernel of T is $\{0\}$ or everything, and so is its image. If $T \neq 0$, the kernel is not everything, so it is $\{0\}$: T maps different columns to different columns. And the image is not $\{0\}$, so it is everything: every column is reached. A matrix with both properties is invertible.

Lemma 3 (span criterion). If the matrices of a group G of $n \times n$ matrices span all $n \times n$

matrices, then (a) the representation is irreducible, and (b) its commutant consists of the multiples of the identity. Proof of (a): let $W \neq \{0\}$ be invariant and $w \neq 0$ a column of W. An invariant subspace is mapped into itself by every combination of group matrices (each term maps w into W, and W contains sums and multiples), hence by every matrix. For any column v, the matrix $M = v\,w^T/(w^T w)$ (the column v times the row w^T, divided by the number $w^T w > 0$; for complex columns use $w^\dagger$) satisfies $Mw = v$. So v lies in W, and W is everything. Proof of (b): a matrix X that commutes with every group matrix commutes with every combination of them, hence with every matrix, in particular with the matrix E_{rc} that has a single 1 in row r and column c. The entry (i, j) of XE_{rc} is X_{ir} when $j = c$ and 0 otherwise; that of $E_{rc}X$ is X_{cj} when $i = r$ and 0 otherwise. Taking $j = c$ and $i \neq r$: $X_{ir} = 0$, so every entry off the diagonal is zero. Taking $i = r$ and $j = c$: $X_{rr} = X_{cc}$, so all diagonal entries are equal. So X is a number times 1.

Lemma 4 (two blocks). Suppose the matrices of a group of 16×16 matrices span exactly the block-diagonal matrices $\mathrm{diag}(X, Y)$, with X and Y any two 8×8 matrices. Then (a) the first half (components 1 to 8) and the second half (9 to 16) are invariant subspaces; (b) each half is irreducible; (c) the two halves are inequivalent; (d) the commutant has dimension 2 and is spanned by $P_- = \mathrm{diag}(1_8, 0)$ and $P_+ = \mathrm{diag}(0, 1_8)$. Proof: (a) a block-diagonal matrix maps a column with zeros in one half to a column with zeros in the same half. (b) On the first half the group acts by the blocks X, which span all 8×8 matrices; Lemma 3 applies, and the same for Y. (c) Let T be an 8×8 intertwiner, $TX = YT$ for every group element. The equation then holds for every combination, hence for every pair (X, Y) of 8×8 matrices (the span contains $\mathrm{diag}(X, Y)$ for all X and Y independently). Take $X = 1_8$ and $Y = 0$: $T = 0$. The only intertwiner is zero, so the halves are inequivalent. (d) Write a 16×16 matrix in blocks, $Z = \begin{pmatrix} W & U \\ V & R \end{pmatrix}$. Then

$$\mathrm{diag}(X, Y)\, Z = \begin{pmatrix} XW & XU \\ YV & YR \end{pmatrix}, \qquad Z\,\mathrm{diag}(X, Y) = \begin{pmatrix} WX & UY \\ VX & RY \end{pmatrix}.$$

Z commutes with all of them when W commutes with every X (so $W = w1_8$ by Lemma 3(b)), R with every Y (so $R = r1_8$), and $XU = UY$ and $YV = VX$ for all X, Y (so $U = V = 0$, by the argument of (c)). Hence $Z = wP_- + rP_+$: a space of dimension 2.

The negative control. If instead the second half carried a copy of the first ($Y = X$ always), the commutant would contain every $\begin{pmatrix} p1_8 & q1_8 \\ s1_8 & t1_8 \end{pmatrix}$, which commutes with every $\mathrm{diag}(X, X)$: dimension 4. So the commutant dimension 2 distinguishes two inequivalent halves (dimension 2) from two equivalent ones (dimension 4); Notebook 05b computes both.

Real or complex numbers. All the equations of this chapter for an unknown matrix have whole-number coefficients. Gaussian elimination, which finds their solutions, uses only addition, subtraction, multiplication and division of the coefficients, so it takes exactly the same steps whether the unknowns are allowed to be rational, real or complex numbers. The dimension of the space of solutions is therefore the same in all three cases, and the irreducibility proved below holds over the real and over the complex numbers.

5.12 Pin(4,4), Spin(4,4) and the generators S^{ab}

Vectors in the algebra. A **vector** is a list of eight real numbers $v = (v_{x0}, \dots, v_{x7})$. To it belongs the matrix $\gamma(v) = \sum_a v_a \gamma^a$, and two vectors have the **metric product** $\eta(u, v) = \sum_a \eta_{aa} u_a v_a$. From the Clifford relation, line by line:

$$\gamma(u)\gamma(v) + \gamma(v)\gamma(u) = \sum_{a,b} u_a v_b \, (\gamma^a \gamma^b + \gamma^b \gamma^a) = \sum_{a,b} u_a v_b \, 2\eta^{ab} 1 = 2\eta(u, v)\, 1.$$

The first step multiplies out both products and collects the terms with the same pair (a, b); the second is the Clifford relation; the third keeps only the terms $a = b$ (where $\eta^{aa} = \eta_{aa}$). With $u = v$: $\gamma(v)\gamma(v) = \eta(v, v)\, 1$. A vector with $\eta(u, u) = +1$ or -1 is a **unit vector**. For a unit vector, $\gamma(u)$ is invertible with $\gamma(u)^{-1} = \eta(u, u)\, \gamma(u)$, because $\gamma(u)\, \eta(u, u)\gamma(u) = \eta(u, u)^2\, 1 = 1$. (A **null** vector, such as $e_{x1} + e_{x4}$ with $\eta = 1 - 1 = 0$, has $\gamma(v)\gamma(v) = 0$ and no inverse; it is excluded.)

O(4,4) and SO(4,4). The real 8×8 matrices Λ that keep the metric product, $\eta(\Lambda u, \Lambda v) = \eta(u, v)$ for all u, v, are those with $\Lambda^T \eta \Lambda = \eta$; they form the group **O(4,4)**. Taking the determinant of both sides, $(\det \Lambda)^2 \det \eta = \det \eta$, and $\det \eta = (+1)^4(-1)^4 = 1$, so $\det \Lambda = \pm 1$. Those with $\det \Lambda = +1$ form the subgroup **SO(4,4)**.

Reflections. For a unit vector u, the **reflection** along u is the 8×8 matrix R_u with

$$R_u v = v - 2\, \frac{\eta(u, v)}{\eta(u, u)}\, u.$$

It sends u to $u - 2u = -u$ and keeps every w with $\eta(u, w) = 0$. These w form a 7-dimensional subspace that does not contain u, so in a basis made of u and seven such w the matrix is $\mathrm{diag}(-1, 1, \dots, 1)$, and $\det R_u = -1$. It keeps the metric product: expanding $\eta(R_u v, R_u w)$ gives $\eta(v, w)$ plus three terms with the factor $\eta(u, v)\eta(u, w)/\eta(u, u)$ and the coefficients -2, -2 and $+4$, which add up to zero. So R_u lies in O(4,4) but not in SO(4,4).

The key identity. For a unit vector u and any vector v:

$$-\gamma(u)\gamma(v)\gamma(u)^{-1} = \gamma(R_u v).$$

Proof, line by line. By the relation above, $\gamma(u)\gamma(v) = 2\eta(u, v)1 - \gamma(v)\gamma(u)$. Multiply from the right by $\gamma(u)^{-1}$:

$$\gamma(u)\gamma(v)\gamma(u)^{-1} = 2\eta(u, v)\, \gamma(u)^{-1} - \gamma(v) = 2\eta(u, v)\, \eta(u, u)\, \gamma(u) - \gamma(v).$$

Since $\eta(u, u) = \pm 1$, $\eta(u, u) = 1/\eta(u, u)$, and γ is linear ($\gamma(v) - c\,\gamma(u) = \gamma(v - cu)$), so the right side is $-\gamma(v - 2\eta(u, v)u/\eta(u, u)) = -\gamma(R_u v)$. Multiplying by -1 gives the identity. **Conjugation by a unit vector is a reflection, up to the sign.**

Pin(4,4) and Spin(4,4). Pin(4,4) is the set of all products

$$g = \gamma(u_1)\gamma(u_2) \cdots \gamma(u_k)$$

of finitely many unit vectors ($k = 0, 1, 2, \ldots$; the empty product is 1). It is a group: the product of two such products is again one, and the inverse $g^{-1} = \gamma(u_k)^{-1} \cdots \gamma(u_1)^{-1}$ is one too, because $\gamma(u)^{-1} = \gamma(\eta(u,u)u)$ and $\eta(u,u)u$ is a unit vector. **Spin(4,4)** is the subgroup of the products with an **even** number k of factors. Examples: every $\gamma^a = \gamma(e_a)$ lies in Pin(4,4), where e_a is the basic unit vector of the direction a; so does every product of different gammas, and it lies in Spin(4,4) when it is even. In particular C (four factors), Γ (eight), $1 = \gamma^{(x1)}\gamma^{(x1)}$ and $-1 = \gamma^{(x4)}\gamma^{(x4)}$ lie in Spin(4,4).

Even and odd are well defined. Multiply out a product of k vectors: each $\gamma(u_j)$ is a combination of single gammas, so the product is a combination of products of k gammas. Each such product can be brought into the order $x0, \ldots, x7$ by exchanging neighbours (a sign each time) and then shortened by $\gamma^a\gamma^a = \eta_{aa}1$, which removes two factors at a time. So the product is a combination of products of different gammas whose degrees have the same parity (evenness or oddness) as k. Section 5.13 shows that the 256 products of different gammas are linearly independent; so a nonzero matrix cannot be both a combination of even products and a combination of odd products. An element of Pin(4,4) is invertible, hence not zero, so it is either **even** or **odd**, never both.

The vector matrix. For g in Pin(4,4) put $\alpha(g) = g$ if g is even and $\alpha(g) = -g$ if it is odd (the **twisted** action). For a single unit vector the key identity reads $\alpha(\gamma(u))\,\gamma^c\,\gamma(u)^{-1} = \gamma(R_u e_c) = \sum_d (R_u)_{dc}\gamma^d$. For a product, $\alpha(gh) = \alpha(g)\alpha(h)$, and

$$\alpha(gh)\gamma^c(gh)^{-1} = \alpha(g)\big[\alpha(h)\gamma^c h^{-1}\big]g^{-1} = \sum_d \Lambda(h)_{dc}\,\alpha(g)\gamma^d g^{-1} = \sum_e \big(\Lambda(g)\Lambda(h)\big)_{ec}\gamma^e,$$

where $\Lambda(g)$ is the 8×8 matrix with $\alpha(g)\gamma^c g^{-1} = \sum_d \Lambda(g)_{dc}\gamma^d$. So $\Lambda(gh) = \Lambda(g)\Lambda(h)$, and for $g = \gamma(u_1)\cdots\gamma(u_k)$:

$$\Lambda(g) = R_{u_1}R_{u_2}\cdots R_{u_k},$$

a product of k reflections. It lies in O(4,4), with $\det\Lambda(g) = (-1)^k$: the even elements, those of Spin(4,4), have vector matrices in SO(4,4). The matrix $\Lambda(g)$ says how g moves the eight directions; it is called the **vector matrix** of g. Section 5.18 shows that g and $-g$ have the same vector matrix and that these are the only two (the **double cover**).

The generators. For two directions a and b the **scaled commutator**

$$S^{ab} = \tfrac{1}{4}[\gamma^a, \gamma^b] = \tfrac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$$

is called a **generator**. $S^{aa} = 0$ and $S^{ba} = -S^{ab}$. For $a \neq b$ the two gammas anticommute, so $\gamma^a\gamma^b - \gamma^b\gamma^a = 2\gamma^a\gamma^b$ and $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$: its entries are 0 and $\pm\frac{1}{2}$. With a before b in the order $x0, \ldots, x7$ there are $8 \cdot 7/2 = 28$ generators, one for each **coordinate plane** (a, b). When $\eta_{aa}\eta_{bb} = +1$ (both directions space-like, or both time-like) the plane is called a **rotation** plane: $\binom{4}{2} + \binom{4}{2} = 6 + 6 = 12$ of them. When $\eta_{aa}\eta_{bb} = -1$ (one space-like and one time-like direction) it is a **boost** plane: $4 \cdot 4 = 16$ of them. Note that a plane of two times, such as $(x4, x5)$, is a rotation plane.

The vector rule. For all a, b, c:

$$[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a - \eta^{ac}\gamma^b.$$

Proof for $a \neq b$ (for $a = b$ both sides are zero). Use the Clifford relation first as $\gamma^b\gamma^c = 2\eta^{bc}1 - \gamma^c\gamma^b$ and then as $\gamma^a\gamma^c = 2\eta^{ac}1 - \gamma^c\gamma^a$:

$$\gamma^a\gamma^b\gamma^c = 2\eta^{bc}\gamma^a - \gamma^a\gamma^c\gamma^b = 2\eta^{bc}\gamma^a - 2\eta^{ac}\gamma^b + \gamma^c\gamma^a\gamma^b.$$

Hence $[\gamma^a\gamma^b, \gamma^c] = \gamma^a\gamma^b\gamma^c - \gamma^c\gamma^a\gamma^b = 2\eta^{bc}\gamma^a - 2\eta^{ac}\gamma^b$, and dividing by 2 gives the rule. It says that a generator turns the gammas into combinations of each other, as an infinitesimal turning of the directions turns the components of a vector: in the plane $(x1, x2)$, $[S^{(x1)(x2)}, \gamma^{(x1)}] = -\gamma^{(x2)}$ and $[S^{(x1)(x2)}, \gamma^{(x2)}] = \gamma^{(x1)}$, and every other gamma commutes with $S^{(x1)(x2)}$.

statement	**status**	**where it is verified**
$S^{ab} = -S^{ba}$, $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$ for $a \neq b$, entries 0, $\pm\frac{1}{2}$	PROVED	`python-algebra.json`, check `S_definition`; `wolfram-algebra.json`, checks `S_antisymmetric_in_ab`, `S_half_product` and `S_real_entries_in_half_integers`
the vector rule for all 512 triples (a, b, c)	PROVED	`python-algebra.json`, check `S_vector_action`; `wolfram-algebra.json`, check `S_gamma_commutator`
the key identity, the vector matrix, $\det\Lambda(g) = (-1)^k$	PROVED (above)	Notebook 05d checks the key identity for three unit vectors

5.13 Irreducible under Pin(4,4), two inequivalent halves under Spin(4,4)

The author's statement about the field dirac16complex is that its 16 components transform under a 16-dimensional irreducible representation of Pin(4,4), and, for the transformations of determinant 1, under a direct sum of two inequivalent 8×8 irreducible representations of Spin(4,4). This section proves both halves of the statement.

From a matrix equation to linear equations. Every statement below is about an unknown matrix X that obeys equations such as $\gamma^a X = X\gamma^a$. Let X be an unknown $n \times m$ matrix, L a known $n \times n$ matrix and R a known $m \times m$ matrix. The entry (r, c) of $LX - XR$ is

$$(LX)_{rc} - (XR)_{rc} = \sum_k L_{rk}X_{kc} - \sum_k X_{rk}R_{kc}.$$

So $LX - XR = 0$ is a list of nm **linear equations** for the nm entries of X: in the equation (r, c) the unknown X_{kc} has the coefficient L_{rk} and the unknown X_{rk} the coefficient $-R_{kc}$. Listing the unknowns row by row $(X_{11}, X_{12}, \dots)$, these coefficients form the matrix

$$A = \mathrm{kron}(L, 1_m) - \mathrm{kron}(1_n, R^T),$$

with one row per equation and one column per unknown; $\mathrm{kron}(P, Q)$, the **Kronecker product**, is the big matrix whose block in block row r and block column k is P_{rk} times the whole matrix Q.

For a list of equations (for example one for each of the eight gammas) the coefficient matrices are stacked on top of each other. The solutions form a space of dimension (number of unknowns) minus (rank of A). The rank can be computed **exactly** by Gaussian elimination with fractions, without any rounding.

A second, independent engine. For the coefficient matrix A, the matrix $N = A^T A$ is symmetric, and $Ny = 0$ exactly when $Ay = 0$: if $Ny = 0$, then $0 = y^T Ny = (Ay)^T(Ay)$, which is the sum of the squares of the entries of Ay, so $Ay = 0$; the converse is clear. So the dimension of the solution space equals the number of zero eigenvalues of N, which a floating-point eigenvalue program computes independently of the exact elimination.

Theorem P (Pin(4,4) is irreducible). The 16 components carry an irreducible representation of Pin(4,4), and only the multiples of 1 commute with every element of Pin(4,4).

Proof, step by step.

- Step 1: the span of Pin(4,4) is the span of the 256 products of different gammas. Every product of different gammas is a product of unit vectors $\gamma(e_a)$, hence an element of Pin(4,4). Conversely, every element of Pin(4,4) is, by the multiplication argument of Section 5.12, a combination of products of different gammas.
- Step 2: the 256 products of different gammas are linearly independent. This is an exact computation: the 256×256 matrix of their entries has rank 256 (Revision record and Notebook 05b). 256 independent 16×16 matrices span the space of all 16×16 matrices, which has dimension 256.
- Step 3: by Lemma 3 of Section 5.11, the representation is irreducible and its commutant consists of the multiples of 1.

The second engine confirms the commutant directly: the $8 \times 256 = 2048$ equations $\gamma^a X - X\gamma^a = 0$ ($a = x0, \ldots, x7$) have rank 255, so their solutions form a space of dimension $256 - 255 = 1$, the multiples of 1. (A matrix that commutes with the eight gammas commutes with all their products and combinations, hence with all of Pin(4,4); the converse holds because every gamma is in Pin(4,4).)

Theorem S (two inequivalent halves under Spin(4,4)). Under Spin(4,4) the 16 components split into the two chiral halves of 8 components (Section 5.5); each half carries an irreducible representation of Spin(4,4); the two are inequivalent; and the commutant of Spin(4,4) is spanned by P_- and P_+.

Proof, step by step.

- Step 1: the span of Spin(4,4) is the span of the 128 even products of different gammas, by the argument of Theorem P restricted to even products (Section 5.12: an even element is a combination of even products; every even product of different gammas is an even element).
- Step 2: every even product is block diagonal (the block rule of Section 5.5), and the 128 even products are linearly independent (exact rank 128). The block-diagonal matrices $\mathrm{diag}(X, Y)$ form a space of dimension $64 + 64 = 128$. So the even products span exactly all block-diagonal matrices.
- Step 3: Lemma 4 of Section 5.11 gives all four claims.

The same with the 28 generators. The Revision record and Notebook 05b also compute the commutant of the 28 generators S^{ab}: dimension 2, spanned by P_- and P_+; on each half the commutant of the 28 blocks has dimension 1; and the equations for an intertwiner between the halves have only the solution 0. A matrix commutes with every exponential $\exp(\theta S^{ab})$ (Section 5.18) exactly when it commutes with every S^{ab} (take the derivative at $\theta = 0$ in one direction; in the other, a matrix that commutes with S commutes with every power of S and hence with the power series of the exponential). So these numbers show that the conclusions of Theorem S hold already for the part of Spin(4,4) made of products of exponentials, which Section 5.23 calls $\mathrm{Spin}_0(4,4)$.

Why Pin(4,4) sees one block of 16. Every gamma is an odd element of Pin(4,4), and every gamma exchanges the two halves (Section 5.5). So neither half is invariant under Pin(4,4): the two inequivalent halves of Spin(4,4) are joined by the reflections into one irreducible representation of Pin(4,4).

Imposing the gammas one at a time. How large is the commutant of only the first j gammas? With no condition all 256 entries are free. The first gamma in the order $x0, \ldots, x7$ is $\gamma^{(x0)}$. The matrices that commute with $\gamma^{(x0)}$ are those that map each of its two eigenspaces into itself: if $\gamma v = \lambda v$ and $X\gamma = \gamma X$, then $\gamma(Xv) = X\gamma v = \lambda Xv$, so Xv has the same eigenvalue; conversely a matrix that acts separately on the two eigenspaces commutes with γ. A real symmetric matrix has a basis of eigenvectors, and $\gamma^{(x0)}$ (space-like, hence symmetric) has eight eigenvalues $+1$ and eight -1 (Section 5.3), so there are $8^2 + 8^2 = 128$ free entries. Notebook 05b computes, exactly, that every further gamma halves the dimension again: $256, 128, 64, 32, 16, 8, 4, 2, 1$. (Only the first step is proved here; the whole sequence is COMPUTED.)

statement	**status**	**where it is verified**
the 256 products of different gammas are independent (rank 256)	PROVED (exact computation)	`python-algebra.json`, check `clifford_products_span_M16`; `wolfram-algebra.json`, check `Clifford_basis_spans_full_matrix_algebra`
Theorem P: commutant of the eight gammas has dimension 1	PROVED	`python-algebra.json`, check `pin_commutant_dimension_1`; `wolfram-algebra.json`, check `Pin44_irreducible_commutant_dim_1`
the 128 even products are block diagonal and independent	PROVED (exact computation)	`python-algebra.json`, check `even_products_span_M8_plus_M8`; `wolfram-algebra.json`, check `even_subalgebra_dimension`
commutant of the 28 S^{ab} has dimension 2, spanned by P_-, P_+	PROVED	`python-algebra.json`, check `spin_commutant_dimension_2`; `wolfram-algebra.json`, check `Spin44_commutant_dim_2_chiral_projectors`

statement	status	where it is verified
each half irreducible; the halves inequivalent (intertwiners 0)	PROVED	`python-algebra.json`, checks `spin_halves_irreducible` and `spin_halves_inequivalent`; `wolfram-algebra.json`, checks `chiral_halves_irreducible` and `chiral_halves_inequivalent_intertwiners_0`
the halving sequence 256, 128, ..., 1; the negative control gives 4	COMPUTED (exact rank)	Notebook 05b (its own computation)

5.14 Example: Notebook 05b computes the commutants

Notebook 05b turns every statement of Section 5.13 into a system of linear equations and computes its exact rank with sympy. It first tests the construction of the coefficient matrix on a random matrix and on a 2×2 warm-up that can be done by hand; then it computes the ranks of the 256 and of the 128 even products of different gammas, the commutant of the eight gammas, the commutants obtained by imposing the gammas one at a time, the commutant of the 28 generators, the commutants on each half, the intertwiners between the halves and the negative control with two equal halves. It confirms every dimension a second time with the zero eigenvalues of $A^T A$, and predicts all 256 eigenvalues of $A^T A$ for the eight gammas exactly. It draws five figures and ends with the line ALL 17 CHECKS PASSED (notebook 05b).

5.15 How to run Notebook 05b

Step 1. What this notebook does and what it needs.

Notebook 05b (Commutants: Pin(4,4) acts irreducibly, Spin(4,4) splits into two inequivalent halves) is the file `Revision/textbook/notebooks/05b_commutants.ipynb` of the repository Dirac_claude. It turns the matrix equations X M = M X into systems of linear equations for the 256 entries of an unknown 16 by 16 matrix X, solves them exactly, and so computes: the rank of the 256 Clifford products, the commutant of the eight gammas (dimension 1: the 16 components carry an irreducible representation of Pin(4,4)), the commutant of the 28 generators S^ab (dimension 2, spanned by the chiral projectors), the commutants of the two halves (1 and 1) and the intertwiners between them (0 and 0: inequivalent halves), with a negative control that shows what equivalent halves would give. Five teaching plots show the counts, the eigenvalues of the equation systems and the dimensions. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private

Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05b_commutants.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 25 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05b_commutants.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 05b_commutants.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/05b.captions.json`
- `Revision/textbook/figures/05b_1_clifford_products.png`
- `Revision/textbook/figures/05b_2_system_eigenvalues.png`
- `Revision/textbook/figures/05b_3_commutant_halving.png`
- `Revision/textbook/figures/05b_4_spin_commutant.png`
- `Revision/textbook/figures/05b_5_dimensions.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the five figure files of notebook 05b exist
ALL 17 CHECKS PASSED (notebook 05b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/05b_commutants.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 05b_commutants.ipynb
python -m nbconvert --execute --inplace 05b_commutants.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming `Revision/algebra/gammas.json`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.
- the cells of sections 10 and 11 take much longer than a few seconds: they solve 7168 linear equations exactly; on a slow computer they may take a minute. Wait until the star in the brackets left of the cell turns into a number.

5.16 Notebook 05b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/05b_commutants.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The 16 components of the author's spinor field are acted on by two groups of
$16 \times 16$ matrices: **Pin(4,4)**, made of products of gamma matrices, and its
subgroup **Spin(4,4)**, made of products of an *even* number of them. The Revision
record states, and this notebook verifies with its own exact computation:
```

1. the $2^8 = 256$ products of different gammas are linearly independent, so they span all 16×16 matrices;
2. only the multiples of the identity commute with all eight gammas (the *commutant* has dimension 1): the 16 components form an **irreducible** representation of Pin(4,4);
3. the matrices that commute with all 28 generators S^{ab} of Spin(4,4) form a space of dimension 2, spanned by the two chiral projectors P_- and P_+: under Spin(4,4) the 16 components split into two halves of 8;
4. each half is irreducible (commutant dimension 1), and the two halves are **inequivalent**: the only matrix that carries one half into the other while respecting Spin(4,4) is zero.

The method is the same for every statement: a matrix equation such as $X\gamma^a = \gamma^a X$ for an unknown matrix X is a system of *linear equations* for the 256 entries of X, and the number of independent solutions is 256 minus the *rank* of the system. Every check that repeats a recorded check prints the record file and the check name. Five teaching plots show the counts and the dimensions; a negative control shows what the computation would give if the two halves were equivalent.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 05b (Commutants: Pin(4,4) acts irreducibly, Spin(4,4) splits into two inequivalent halves) is the file ‘Revision/textbook/notebooks/05b_commutants.ipynb‘ of the repository Dirac_claude. It turns the matrix equations X M = M X into systems of linear equations for the 256 entries of an unknown 16 by 16 matrix X, solves them exactly, and so computes: the rank of the 256 Clifford products, the commutant of the eight gammas (dimension 1: the 16 components carry an irreducible representation of Pin(4,4)), the commutant of the 28 generators S^ab (dimension 2, spanned by the chiral projectors), the commutants of the two halves (1 and 1) and the intertwiners between them (0 and 0: inequivalent halves), with a negative control that shows what equivalent halves would give. Five teaching plots show the counts, the eigenvalues of the equation systems and the dimensions. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the

default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
```

```
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 05b_commutants.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 25 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 05b_commutants.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 05b_commutants.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/05b.captions.json‘
- ‘Revision/textbook/figures/05b_1_clifford_products.png‘
- ‘Revision/textbook/figures/05b_2_system_eigenvalues.png‘
- ‘Revision/textbook/figures/05b_3_commutant_halving.png‘
- ‘Revision/textbook/figures/05b_4_spin_commutant.png‘
- ‘Revision/textbook/figures/05b_5_dimensions.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the

stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the five figure files of notebook 05b exist
ALL 17 CHECKS PASSED (notebook 05b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/05b_commutants.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 05b_commutants.ipynb
  python -m nbconvert --execute --inplace 05b_commutants.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands

  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming ‘Revision/algebra/gammas.json‘: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder ‘Revision/textbook/notebooks‘.
- the cells of sections 10 and 11 take much longer than a few seconds: they solve 7168 linear equations exactly; on a slow computer they may take a minute. Wait until the star in the brackets left of the cell turns into a number.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 05b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 05b (Commutants: Pin(4,4) acts irreducibly, Spin(4,4) splits into two
# inequivalent halves) is the file Revision/textbook/notebooks/05b_commutants.ipynb of
# the repository Dirac_claude. It turns the matrix equations X M = M X into systems of
# linear equations for the 256 entries of an unknown 16 by 16 matrix X, solves them
# exactly, and so computes: the rank of the 256 Clifford products, the commutant of the
# eight gammas (dimension 1: the 16 components carry an irreducible representation of
# Pin(4,4)), the commutant of the 28 generators S^ab (dimension 2, spanned by the chiral
# projectors), the commutants of the two halves (1 and 1) and the intertwiners between
# them (0 and 0: inequivalent halves), with a negative control that shows what equivalent
# halves would give. Five teaching plots show the counts, the eigenvalues of the equation
# systems and the dimensions. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
```

```
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

```
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 05b_commutants.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
```

```
# notebook has finished when the last code cell shows a number; this takes about 25
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 05b_commutants.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 05b_commutants.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/05b.captions.json
# - Revision/textbook/figures/05b_1_clifford_products.png
# - Revision/textbook/figures/05b_2_system_eigenvalues.png
# - Revision/textbook/figures/05b_3_commutant_halving.png
# - Revision/textbook/figures/05b_4_spin_commutant.png
# - Revision/textbook/figures/05b_5_dimensions.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the five figure files of notebook 05b exist
#     ALL 17 CHECKS PASSED (notebook 05b)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
```

```
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/05b_commutants.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 05b_commutants.ipynb
#     python -m nbconvert --execute --inplace 05b_commutants.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming Revision/algebra/gammas.json: the notebook was opened
#   outside the repository, or the repository is incomplete; clone the repository again
#   and open the notebook from its folder Revision/textbook/notebooks.
# - the cells of sections 10 and 11 take much longer than a few seconds: they solve 7168
#   linear equations exactly; on a slow computer they may take a minute. Wait until the
#   star in the brackets left of the cell turns into a number.
#
# ----------------------------------------------------------------------------------------
# ========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "05b"  # this notebook: chapter 05, example b
```

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 05b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Linear equation, system**: an equation of the form $c_1 y_1 + c_2 y_2 + \dots
  + c_n y_n = 0$ for unknown numbers $y_1, \dots, y_n$ with given coefficients
  $c_j$. A *system* is a list of such equations; its coefficients form the
  **coefficient matrix** $A$ (one row per equation), and the system reads
  $Ay = 0$.
- **Solution space, dimension**: the set of all solutions $y$ of $Ay = 0$. Sums
  and multiples of solutions are solutions. Its **dimension** is the number of
  solutions needed so that every solution is a combination of them (a
  **basis**).
- **Rank**: the number of independent rows of $A$ (rows that are not combinations
  of other rows). For $n$ unknowns: dimension of the solution space $= n -
  \text{rank}$.
- **Exact arithmetic**: computing with fractions, never rounding. The package
  sympy does this (`DomainMatrix` over `QQ`, the rational numbers).
- **Linearly independent matrices**: no one of them is a combination of the
  others. Writing each $16 \times 16$ matrix as one row of 256 numbers, $N$ matrices
  are independent when these $N$ rows have rank $N$.
- **Span**: all combinations of a list of matrices.
- **Clifford product**: a product $\gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$ of
  $k$ *different* gammas, taken in the order $x0, \dots, x7$; $k$ is its
  **degree**; it is **even** or **odd** with $k$. The degree-0 product is $1$.
- **Group**: a set of invertible matrices that contains the products and the
  inverses of its members.
- **Representation, invariant subspace, irreducible**: a group of $16 \times 16$
  matrices *represents* the group on columns of 16 numbers. A set of columns that
  every group matrix maps into itself is *invariant*. The representation is
  **irreducible** when the only invariant subspaces are $\{0\}$ and everything.
- **Commutant**: all matrices $X$ that commute with every matrix of a list.
- **Intertwiner**: a matrix $X$ with $X M_k = N_k X$ for two lists of matrices
  $M_k$, $N_k$ (it carries the first representation into the second).
  **Equivalent** representations have an invertible intertwiner.
- **Schur's lemma** (used here, proved in the text): when the matrices of a
  representation span all matrices, the representation is irreducible and its
  commutant is the multiples of 1; an intertwiner between two irreducible
  representations is zero or invertible.
- **Generators** $S^{ab} = \tfrac14[\gamma^a, \gamma^b] = \tfrac14(\gamma^a\gamma^b
  - \gamma^b\gamma^a)$: the 28 matrices ($a < b$) from which every element of the
  part of Spin(4,4) connected to 1 is built as a product of exponentials
  $\exp(\theta S^{ab})$.
- **Chiral halves**: the components 1 to 8 (chirality $\Gamma = -1$) and 9 to 16
  ($\Gamma = +1$); $P_- = \mathrm{diag}(1_8, 0)$ and $P_+ = \mathrm{diag}(0, 1_8)$.
- **Kronecker product** `np.kron(L, R)`: the big matrix whose block in block row
  $r$ and block column $k$ is $L_{rk}$ times the whole matrix $R$.
```

Text cell:

```
## 4. The physical and mathematical situation

**From a matrix equation to linear equations.** Let $X$ be an unknown $n \times m$
matrix, $L$ a known $n \times n$ and $R$ a known $m \times m$ matrix, and consider
$LX - XR = 0$. List the $nm$ entries of $X$ row by row: $X_{11}, X_{12}, \dots,
X_{1m}, X_{21}, \dots$. The entry $(r, c)$ of the equation is

$$(LX)_{rc} - (XR)_{rc} = \sum_k L_{rk} X_{kc} - \sum_k X_{rk} R_{kc} = 0 .$$

In the first sum the unknown $X_{kc}$ has the coefficient $L_{rk}$; in the second
the unknown $X_{rk}$ has the coefficient $R_{kc}$. Collected over all $(r, c)$,
these coefficients form the matrix $\mathrm{kron}(L, 1_m) - \mathrm{kron}(1_n,
R^T)$, with one row per equation and one column per unknown. For a list of pairs
$(L_j, R_j)$ the coefficient matrices are stacked on top of each other. For the
commutant of the eight gammas: $n = m = 16$, $L_j = R_j = \gamma^{a}$, so
$8 \times 256 = 2048$ equations for 256 unknowns.

**How the two computations prove irreducibility.** Every element of Pin(4,4) is a
product of matrices $\gamma(u) = \sum_a u_a \gamma^a$, so it is a combination of
Clifford products; conversely every Clifford product is, up to a sign, an element
of Pin(4,4) (each $\gamma^a$ is one). So the span of Pin(4,4) is the span of the 256
Clifford products. *First computation*: if they span all $16 \times 16$ matrices,
then every invariant subspace is $\{0\}$ or everything (a subspace kept by every
matrix is $\{0\}$ or everything), and only the multiples of 1 commute with them
(Schur's lemma in its span form). *Second computation*: the commutant of the eight
gammas, solved directly; it confirms the second statement independently. For
Spin(4,4) the even products play the same role: if they span exactly the
block-diagonal matrices $\mathrm{diag}(X, Y)$, each half is irreducible and the two
halves are inequivalent (an intertwiner $T$ with $TX = YT$ for all $X$, $Y$ must
vanish: take $X = 1_8$, $Y = 0$). The commutant of the 28 generators $S^{ab}$, which
build the part of Spin(4,4) connected to 1, and the intertwiners between the two
halves are then computed directly, in the form in which the Revision record states
them.

**Two halves, equivalent or not.** If the 16 components split into two invariant
halves of 8 and each half is irreducible, the commutant has dimension 2 when the
halves are inequivalent ($X = p P_- + q P_+$) and dimension 4 when they are
equivalent (then a matrix that carries one half onto the other also commutes).
The notebook computes both the real case and an artificial *control* with two
equal halves.

**A second engine.** For a coefficient matrix $A$ the matrix $N = A^T A$ is
symmetric, and $Ny = 0$ exactly when $Ay = 0$: if $Ny = 0$, then
$0 = y^T N y = (Ay)^T(Ay) = |Ay|^2$, the sum of the squares of the entries of
$Ay$, so $Ay = 0$; the converse is clear. Hence the dimension of the solution space
equals the number of zero eigenvalues of $N$, which numpy computes in
floating-point numbers, independently of sympy.
```

Text cell:

```
## 5. The gammas and the recorded checks

The next cell reads the gammas from the record ‘Revision/algebra/gammas.json‘ (exact
```

whole numbers), reads the two Revision reports, and defines the helpers
‘recorded(key, name)‘ (true when the report ‘key‘, "python" or "wolfram", holds the
check ‘name‘ with the verdict pass), ‘detail(key, name)‘ (the detail text of that
check), ‘record_of(key, name)‘ (the text printed after "reproduces") and
‘check_reproduces(condition, name, record)‘, the helper ‘check‘ for a check that
reproduces a Revision record: it lets ‘check‘ print the PASS line and the line
"reproduces ..." into a text buffer (‘contextlib.redirect_stdout‘) and sends both
lines with one ‘sys.stdout.write‘, because Jupyter delivers printed text in pieces
and one piece keeps the two lines together for the tools that read the notebook.
Then it checks the Clifford relation, on which everything below rests.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1

REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    """True when the report key records the check name with the verdict pass."""
    return VERDICTS[(key, name)][0] == "pass"


def detail(key, name):
    """The detail text of the recorded check."""
    return VERDICTS[(key, name)][1]


def record_of(key, name):
    """The text printed after "reproduces": the record file and the check name."""
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    collected = io.StringIO()
```

```
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


clifford_ok = all(
    np.array_equal(gamma[x] @ gamma[y] + gamma[y] @ gamma[x],
                   2 * ETA[x] * I16 if x == y else 0 * I16)
    for x in COORDS for y in COORDS)
check_reproduces(clifford_ok and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

Out [2]:

```
PASS {gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
```

Text cell:

```
## 6. The tools: equation systems and their exact rank

The next cell defines three tools.

- ‘commutation_system(lefts, rights)‘ builds the coefficient matrix of the
  equations $L X - X R = 0$ for all pairs $(L, R)$ of the two lists, following
  section 4.
- ‘exact_rank(A)‘ computes the rank of a matrix of whole numbers exactly, with
  sympy’s ‘DomainMatrix‘ over the rational numbers ‘QQ‘ (Gaussian elimination with
  fractions).
- ‘solution_basis(A, n, m)‘ returns a basis of the solutions as $n \times m$
  matrices (sympy’s ‘nullspace‘, also exact).

Then the cell tests the construction: for a random matrix $X$ of whole numbers
(from a random-number generator with the fixed seed 12345, so every run uses the
same numbers), the coefficient matrix applied to the entries of $X$ must give the
entries of $LX - XR$.
```

In [3]:

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over QQ


def commutation_system(lefts, rights):
    """Coefficient matrix of L X - X R = 0 for every pair (L, R); X is n x m with
    n = size of L and m = size of R; the unknowns are the entries of X row by row."""
    blocks = []
    for L, R in zip(lefts, rights):
        n, m = L.shape[0], R.shape[0]
        blocks.append(np.kron(L, np.eye(m, dtype=np.int64))
```

```
                    - np.kron(np.eye(n, dtype=np.int64), R.T))
    return np.vstack(blocks)  # the blocks stacked on top of each other


def exact_rank(A):
    """The rank of the whole-number matrix A, computed exactly."""
    return DomainMatrix.from_list(A.tolist(), sp.QQ).rank()


def solution_basis(A, n, m):
    """A basis of all solutions of A y = 0, each reshaped to an n x m matrix."""
    null = DomainMatrix.from_list(A.tolist(), sp.QQ).nullspace().to_Matrix()
    return [np.array(null.row(k).tolist()[0], dtype=float).reshape(n, m)
            for k in range(null.rows)]


rng = np.random.default_rng(12345)  # random numbers with a fixed seed
X = rng.integers(-5, 6, size=(16, 16))  # a random 16 x 16 matrix, entries -5..5
L, R = gamma["x1"], gamma["x4"]
A = commutation_system([L], [R])
say(f"one pair (L, R) gives {A.shape[0]} equations for {A.shape[1]} unknowns")
check(np.array_equal(A @ X.reshape(256), (L @ X - X @ R).reshape(256)),
      "the coefficient matrix applied to X row by row gives L X - X R")
```

Out [3]:

```
one pair (L, R) gives 256 equations for 256 unknowns
PASS the coefficient matrix applied to X row by row gives L X - X R
```

Text cell:

```
**A warm-up with $2 \times 2$ matrices.** Which $2 \times 2$ matrices
$X = \begin{pmatrix} p & q \\ r & s \end{pmatrix}$ commute with
$M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$? By hand: $MX =
\begin{pmatrix} r & s \\ p & q \end{pmatrix}$ and $XM = \begin{pmatrix} q & p \\ s & r
\end{pmatrix}$, so $MX = XM$ means $r = q$ and $s = p$: the solutions are
$\begin{pmatrix} p & q \\ q & p \end{pmatrix}$, a space of dimension 2 (basis: the
identity and $M$ itself). The next cell gets the same answer from the tools: 4
equations, rank 2, dimension $4 - 2 = 2$.
```

In [4]:

```
M = np.array([[0, 1], [1, 0]], dtype=np.int64)
A_small = commutation_system([M], [M])  # 4 equations for the 4 entries p, q, r, s
rank_small = exact_rank(A_small)
basis_small = solution_basis(A_small, 2, 2)
say(f"coefficient matrix (rows = equations, columns = p, q, r, s):")
for row in A_small.tolist():
    say(f"    {row}")
say(f"rank {rank_small}; dimension of the solution space {4 - rank_small}")
for k, b in enumerate(basis_small, 1):
```

```
    say(f"basis solution {k}: rows {b.astype(int).tolist()}")
check(rank_small == 2 and len(basis_small) == 2,
      "warm-up: the 2 x 2 matrices commuting with M form a space of dimension 2")
```

Out [4]:

```
coefficient matrix (rows = equations, columns = p, q, r, s):
    [0, -1, 1, 0]
    [-1, 0, 0, 1]
    [1, 0, 0, -1]
    [0, 1, -1, 0]
rank 2; dimension of the solution space 2
basis solution 1: rows [[0, 1], [1, 0]]
basis solution 2: rows [[1, 0], [0, 1]]
PASS warm-up: the 2 x 2 matrices commuting with M form a space of dimension 2
```

Text cell:

```
## 7. The 256 Clifford products span all 16 x 16 matrices

The next cell builds the Clifford products of every degree $k = 0, \dots, 8$ (for
each $k$, ‘itertools.combinations‘ lists the $\binom{8}{k}$ subsets of $k$
directions in the order $x0, \dots, x7$). It writes each product as one row of 256
numbers and computes exactly the rank of all 256 rows (must be 256: independent),
of the 128 even ones (must be 128), and the rank after adding the products degree
by degree. It also checks that every even product is block diagonal (it maps each
half into itself) and every odd product block off-diagonal (it exchanges the
halves).
```

In [5]:

```
import itertools  # subsets of a given size
from math import comb  # comb(8, k): the number of subsets of size k

products = {}  # degree k -> list of the Clifford products of degree k
for k in range(9):
    products[k] = []
    for subset in itertools.combinations(COORDS, k):
        p = I16
        for x in subset:
            p = p @ gamma[x]
        products[k].append(p)
rows_all = np.array([p.reshape(256) for k in range(9) for p in products[k]])
rows_even = np.array([p.reshape(256) for k in (0, 2, 4, 6, 8) for p in products[k]])
rank_all, rank_even = exact_rank(rows_all), exact_rank(rows_even)
cumulative = []  # the rank after adding the degrees 0, 1, ..., k
for k in range(9):
    cumulative.append(exact_rank(np.array(
        [p.reshape(256) for j in range(k + 1) for p in products[j]])))
for k in range(9):
    say(f"degree {k}: {len(products[k]):2d} products (comb(8, {k}) = {comb(8, k):2d});"
```

```
        f" rank of all products up to degree {k}: {cumulative[k]}")
say(f"rank of all 256 products: {rank_all}; rank of the 128 even products: "
    f"{rank_even}")
check_reproduces(rank_all == 256 and f"rank {rank_all} (Fraction elimination)"
                 in detail("python", "clifford_products_span_M16")
                 and recorded("python", "clifford_products_span_M16")
                 and recorded("wolfram", "Clifford_basis_spans_full_matrix_algebra"),
                 "the 256 Clifford products are independent: they span all 16 x 16 "
                 "matrices",
                 record=record_of("python", "clifford_products_span_M16"))
even_diagonal = all(not p[:8, 8:].any() and not p[8:, :8].any()
                    for k in (0, 2, 4, 6, 8) for p in products[k])
odd_off = all(not p[:8, :8].any() and not p[8:, 8:].any()
              for k in (1, 3, 5, 7) for p in products[k])
check_reproduces(rank_even == 128 and even_diagonal and odd_off
                 and f"rank {rank_even} (Fraction)"
                 in detail("python", "even_products_span_M8_plus_M8")
                 and recorded("python", "even_products_span_M8_plus_M8")
                 and recorded("wolfram", "even_subalgebra_dimension"),
                 "the 128 even products are block diagonal and independent (64 + 64); "
                 "the odd ones are block off-diagonal",
                 record=record_of("python", "even_products_span_M8_plus_M8"))
```

Out [5]:

```
degree 0:  1 products (comb(8, 0) =  1); rank of all products up to degree 0: 1
degree 1:  8 products (comb(8, 1) =  8); rank of all products up to degree 1: 9
degree 2: 28 products (comb(8, 2) = 28); rank of all products up to degree 2: 37
degree 3: 56 products (comb(8, 3) = 56); rank of all products up to degree 3: 93
degree 4: 70 products (comb(8, 4) = 70); rank of all products up to degree 4: 163
degree 5: 56 products (comb(8, 5) = 56); rank of all products up to degree 5: 219
degree 6: 28 products (comb(8, 6) = 28); rank of all products up to degree 6: 247
degree 7:  8 products (comb(8, 7) =  8); rank of all products up to degree 7: 255
degree 8:  1 products (comb(8, 8) =  1); rank of all products up to degree 8: 256
rank of all 256 products: 256; rank of the 128 even products: 128
PASS the 256 Clifford products are independent: they span all 16 x 16 matrices
     reproduces Revision/algebra/reports/python-algebra.json, check
    clifford_products_span_M16
PASS the 128 even products are block diagonal and independent (64 + 64); the odd ones are
    block off-diagonal
     reproduces Revision/algebra/reports/python-algebra.json, check
    even_products_span_M8_plus_M8
```

Text cell:

```
The next cell draws two pictures: the number of Clifford products of each degree
(even degrees in blue, odd in orange), and the rank after adding the degrees one
after the other, which climbs exactly along the number of products added: no
product is a combination of the others.
```

In [6]:

```
from matplotlib.patches import Patch  # a coloured square for a legend

degrees = np.arange(9)
counts = [len(products[k]) for k in range(9)]
colors = ["#2a78d6" if k % 2 == 0 else "#eb6834" for k in range(9)]
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
axes[0].bar(degrees, counts, color=colors, edgecolor="white", linewidth=2)
for k in range(9):
    axes[0].text(k, counts[k] + 1.5, str(counts[k]), ha="center")
axes[0].set_xlabel("degree $k$ (number of different gammas multiplied)")
axes[0].set_ylabel("number of products")
axes[0].set_title(r"$\binom{8}{k}$ products of degree $k$")
axes[0].set_ylim(0, 80)
axes[0].legend(handles=[Patch(color="#2a78d6", label="even degree"),
                        Patch(color="#eb6834", label="odd degree")],
               loc="upper right")
added = np.cumsum(counts)  # how many products have been added up to degree k
axes[1].plot(added, cumulative, "o-", color="#2a78d6", linewidth=2, markersize=8,
             label="exact rank")
axes[1].plot([0, 256], [0, 256], ":", color="black", linewidth=1,
             label="rank = number of products")
axes[1].set_xlabel("number of products added (degrees 0 to $k$)")
axes[1].set_ylabel("rank")
axes[1].set_title("The rank grows by one for every product")
axes[1].legend(loc="upper left")
save_figure(fig, "clifford_products",
            "Left: the number of Clifford products of each degree $k$ (the number of "
            "different gammas multiplied), which is the binomial number "
            "$\\binom{8}{k}$: 1, 8, 28, 56, 70, 56, 28, 8, 1, in total 256; blue "
            "even degrees, orange odd degrees. Right: the exact rank of all "
            "products of degrees 0 to $k$ (dots) against the number of products "
            "added (horizontal axis); the dots lie on the dotted line rank = "
            "number of products, ending at 256: the 256 products are independent "
            "and span all 16 by 16 matrices.")
```

Out [6]:

```
Figure 05b.1 saved as Revision/textbook/figures/05b_1_clifford_products.png
```

Text cell:

```
## 8. The commutant of the eight gammas: Pin(4,4) is irreducible

The next cell builds the $2048 \times 256$ system $X\gamma^a - \gamma^a X = 0$
($a = x0, \dots, x7$; with `commutation_system`, $L = R = \gamma^a$ gives
$\gamma^a X - X\gamma^a$, the same equations with the opposite sign), computes its
exact rank and a basis of its solutions, and checks that the only solutions are
the multiples of the identity.
```

In [7]:

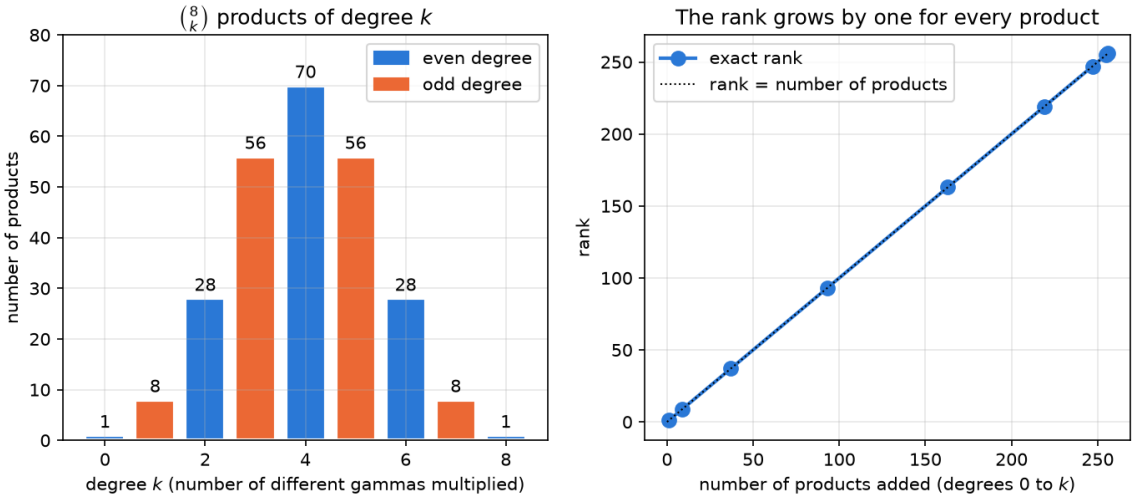

Figure 148: Left: the number of Clifford products of each degree k (the number of different gammas multiplied), which is the binomial number $\binom{8}{k}$: 1, 8, 28, 56, 70, 56, 28, 8, 1, in total 256; blue even degrees, orange odd degrees. Right: the exact rank of all products of degrees 0 to k (dots) against the number of products added (horizontal axis); the dots lie on the dotted line rank = number of products, ending at 256: the 256 products are independent and span all 16 by 16 matrices. (Notebook 05b, figure 1.)

```
gammas = [gamma[x] for x in COORDS]
A_pin = commutation_system(gammas, gammas)  # 8 x 256 = 2048 equations
rank_pin = exact_rank(A_pin)
basis_pin = solution_basis(A_pin, 16, 16)
dimension_pin = 256 - rank_pin
say(f"equations {A_pin.shape[0]}, unknowns {A_pin.shape[1]}, rank {rank_pin}, "
    f"dimension of the commutant {dimension_pin}")
only = basis_pin[0]
proportional = np.array_equal(only / only[0, 0], np.eye(16))  # a multiple of 1?
say(f"the basis solution divided by its entry (1,1) equals the identity: "
    f"{proportional}")
check_reproduces(dimension_pin == 1 and len(basis_pin) == 1 and proportional
                 and f"= {dimension_pin} (Fraction)"
                 in detail("python", "pin_commutant_dimension_1")
                 and recorded("python", "pin_commutant_dimension_1")
                 and recorded("wolfram", "Pin44_irreducible_commutant_dim_1"),
                 "only the multiples of 1 commute with all eight gammas: Pin(4,4) acts "
                 "irreducibly on the 16 components",
                 record=record_of("python", "pin_commutant_dimension_1"))
```

Out [7]:

```
equations 2048, unknowns 256, rank 255, dimension of the commutant 1
the basis solution divided by its entry (1,1) equals the identity: True
PASS only the multiples of 1 commute with all eight gammas: Pin(4,4) acts irreducibly on
    the 16 components
     reproduces Revision/algebra/reports/python-algebra.json, check
    pin_commutant_dimension_1
```

Text cell:

```
**The second engine and what its eigenvalues mean.** The next cell computes the
eigenvalues of $N = A^T A$ ($256 \times 256$) with numpy and counts those below
$10^{-9}$: by section 4 their number is the dimension of the commutant. The
eigenvalues can even be predicted exactly. The equations for $\gamma^a$ act on $X$
as $X \to \gamma^a X - X\gamma^a$; the transposed equations act as $X \to
\eta_{aa}(\gamma^a X - X\gamma^a)$ (because $(\gamma^a)^T = \eta_{aa}\gamma^a$).
Take for $X$ a Clifford product $P$. If $\gamma^a$ commutes with $P$, the first
step gives 0. If it anticommutes, the first step gives $2\gamma^a P$, the second
$\eta_{aa}(2\gamma^a\gamma^a P - 2\gamma^a P\gamma^a) = \eta_{aa}(2\eta_{aa}P +
2\eta_{aa}P) = 4P$. So $NP = 4\, n(P)\, P$, where $n(P)$ is the number of gammas
that anticommute with $P$. By the sign rule for moving a gamma through $k$
different factors, $n(P) = k$ for even $k$ and $n(P) = 8 - k$ for odd $k$. The cell
checks $NP = 4n(P)P$ exactly for all 256 products and compares the predicted list
of eigenvalues with numpy's.
```

In [8]:

```
N_pin = A_pin.T @ A_pin  # 256 x 256, whole numbers, symmetric
eigen_pin = np.linalg.eigvalsh(N_pin.astype(float))  # sorted from small to large
zeros_pin = int(np.sum(np.abs(eigen_pin) < 1e-9))
say(f"numpy: eigenvalues of A^T A below 1e-9: {zeros_pin}")
predicted = []  # the eigenvalue 4 n(P) of every Clifford product P
exact_eigenvectors = True
for k in range(9):
    n_anti = k if k % 2 == 0 else 8 - k  # gammas anticommuting with a product
    for p in products[k]:
        predicted.append(4 * n_anti)
        exact_eigenvectors &= np.array_equal(N_pin @ p.reshape(256),
                                             4 * n_anti * p.reshape(256))
predicted = np.sort(np.array(predicted))
values, multiplicities = np.unique(predicted, return_counts=True)
say("predicted eigenvalues of A^T A, written value x multiplicity:")
say("    " + ", ".join(f"{v} x {c}" for v, c in zip(values, multiplicities)))
check(exact_eigenvectors and zeros_pin == 1
      and np.max(np.abs(eigen_pin - predicted)) < 1e-9,
      "A^T A has exactly one zero eigenvalue; every Clifford product P is an "
      "eigenvector with eigenvalue 4 n(P)")
```

Out [8]:

```
numpy: eigenvalues of A^T A below 1e-9: 1
predicted eigenvalues of A^T A, written value x multiplicity:
    0 x 1, 4 x 8, 8 x 28, 12 x 56, 16 x 70, 20 x 56, 24 x 28, 28 x 8, 32 x 1
PASS A^T A has exactly one zero eigenvalue; every Clifford product P is an eigenvector
    with eigenvalue 4 n(P)
```

Text cell:

```
## 9. Imposing the gammas one at a time

How does the commutant shrink as more gammas are required to commute with $X$?
With no condition, all 256 entries are free. The matrices that commute with one
gamma, say $\gamma^{(x1)}$, are those that map each of its two eigenspaces
(eigenvalue $+1$ and $-1$, eight dimensions each) into itself: $8^2 + 8^2 = 128$
of them. The next cell computes the dimension after imposing $\gamma^{(x1)}$, then
also $\gamma^{(x2)}$, and so on up to all eight, exactly.
```

In [9]:

```
halving = [256]  # dimension of the commutant of the first j gammas, j = 0, 1, ..., 8
for j in range(1, 9):
    A_j = commutation_system(gammas[:j], gammas[:j])
    halving.append(256 - exact_rank(A_j))
for j in range(9):
    names = ", ".join(COORDS[:j]) if j else "none"
    say(f"gammas imposed: {j} ({names}); commutant dimension {halving[j]}")
check(halving == [256 // 2 ** j for j in range(9)],
      "every further gamma halves the commutant: 256, 128, 64, ..., 2, 1")
```

Out [9]:

```
gammas imposed: 0 (none); commutant dimension 256
gammas imposed: 1 (x0); commutant dimension 128
gammas imposed: 2 (x0, x1); commutant dimension 64
gammas imposed: 3 (x0, x1, x2); commutant dimension 32
gammas imposed: 4 (x0, x1, x2, x3); commutant dimension 16
gammas imposed: 5 (x0, x1, x2, x3, x4); commutant dimension 8
gammas imposed: 6 (x0, x1, x2, x3, x4, x5); commutant dimension 4
gammas imposed: 7 (x0, x1, x2, x3, x4, x5, x6); commutant dimension 2
gammas imposed: 8 (x0, x1, x2, x3, x4, x5, x6, x7); commutant dimension 1
PASS every further gamma halves the commutant: 256, 128, 64, ..., 2, 1
```

Text cell:

```
## 10. The generators S^ab and the commutant of Spin(4,4)

The next cell builds the 28 generators $S^{ab} = \tfrac14[\gamma^a, \gamma^b]$
($a$ before $b$ in the order $x0, \dots, x7$). For $a \neq b$ the two gammas
anticommute, so $[\gamma^a, \gamma^b] = 2\gamma^a\gamma^b$ and $S^{ab} =
\tfrac12\gamma^a\gamma^b$: its entries are $0$ and $\pm\tfrac12$. It checks these
recorded facts and that every $S^{ab}$ is block diagonal. For the exact equation
system it uses the whole-number matrices $2S^{ab} = \gamma^a\gamma^b$, which have
the same commutant (a matrix commutes with $S$ exactly when it commutes with $2S$).
```

In [10]:

```
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 pairs
S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}
```

```
entries = sorted({float(v) for s in S.values() for v in s.flat})
say(f"{len(S)} generators S^ab; their entries are {entries}")
check_reproduces(len(S) == 28 and entries == [-0.5, 0.0, 0.5]
                 and all(np.array_equal(S[(a, b)], gamma[a] @ gamma[b] / 2)
                         for a, b in pairs)
                 and recorded("python", "S_definition"),
                 "S^ab = (1/4)[gamma^a, gamma^b] = (1/2) gamma^a gamma^b: 28 matrices "
                 "with entries 0, +1/2, -1/2",
                 record=record_of("python", "S_definition"))
check_reproduces(all(not s[:8, 8:].any() and not s[8:, :8].any() for s in S.values())
                 and recorded("python", "S_block_diagonal"),
                 "every S^ab is block diagonal: it maps each chiral half into itself",
                 record=record_of("python", "S_block_diagonal"))
```

Out [10]:

```
28 generators S^ab; their entries are [-0.5, 0.0, 0.5]
PASS S^ab = (1/4)[gamma^a, gamma^b] = (1/2) gamma^a gamma^b: 28 matrices with entries 0,
    +1/2, -1/2
     reproduces Revision/algebra/reports/python-algebra.json, check S_definition
PASS every S^ab is block diagonal: it maps each chiral half into itself
     reproduces Revision/algebra/reports/python-algebra.json, check S_block_diagonal
```

Text cell:

```
The next cell solves $[X, 2S^{ab}] = 0$ for all 28 generators: $28 \times 256 =
7168$ equations for 256 unknowns. The solution space must have dimension 2 and be
spanned by $P_-$ and $P_+$: the cell checks that the two basis matrices that sympy
returns, together with $P_-$ and $P_+$, have rank 2 (they span the same space). The
second engine counts the zero eigenvalues of $A^T A$.
```

In [11]:

```
generators = [(gamma[a] @ gamma[b]).astype(np.int64) for a, b in pairs]  # 2 S^ab
A_spin = commutation_system(generators, generators)  # 7168 equations
rank_spin = exact_rank(A_spin)
basis_spin = solution_basis(A_spin, 16, 16)
dimension_spin = 256 - rank_spin
P_minus = np.diag([1] * 8 + [0] * 8)  # diag(1_8, 0): components 1 to 8
P_plus = np.diag([0] * 8 + [1] * 8)  # diag(0, 1_8): components 9 to 16
together = np.array([b.reshape(256) for b in basis_spin]
                    + [P_minus.reshape(256), P_plus.reshape(256)])
same_span = np.linalg.matrix_rank(together) == 2  # four matrices, two directions
eigen_spin = np.linalg.eigvalsh((A_spin.T @ A_spin).astype(float))
zeros_spin = int(np.sum(np.abs(eigen_spin) < 1e-9))
say(f"equations {A_spin.shape[0]}, rank {rank_spin}, dimension {dimension_spin}; "
    f"numpy: zero eigenvalues of A^T A: {zeros_spin}")
say(f"the basis and P_-, P_+ span the same space: {same_span}")
check_reproduces(dimension_spin == 2 and zeros_spin == 2 and same_span
                 and f"= {dimension_spin} (Fraction)"
                 in detail("python", "spin_commutant_dimension_2")
```

```
                and recorded("python", "spin_commutant_dimension_2")
                and recorded("wolfram", "Spin44_commutant_dim_2_chiral_projectors"),
                "the commutant of the 28 S^ab has dimension 2, spanned by P_- and P_+",
                record=record_of("python", "spin_commutant_dimension_2"))
```

Out [11]:

```
equations 7168, rank 254, dimension 2; numpy: zero eigenvalues of A^T A: 2
the basis and P_-, P_+ span the same space: True
PASS the commutant of the 28 S^ab has dimension 2, spanned by P_- and P_+
     reproduces Revision/algebra/reports/python-algebra.json, check
    spin_commutant_dimension_2
```

Text cell:

```
The next cell draws the eigenvalues of $A^T A$ for both systems, sorted from the
smallest to the largest: for the eight gammas exactly one is zero (the levels 0,
4, 8, ..., 32 predicted above), for the 28 generators exactly two.
```

In [12]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for ax, eig, zeros, title in [
        (axes[0], eigen_pin, zeros_pin, "eight gammas (Pin(4,4))"),
        (axes[1], eigen_spin, zeros_spin, "28 generators $S^{ab}$ (Spin(4,4))")]:
    index = np.arange(1, 257)  # the eigenvalues numbered 1 to 256
    ax.plot(index, eig, color="#2a78d6", linewidth=2, label="eigenvalue")
    ax.plot(index[:zeros], eig[:zeros], "o", color="#eb6834", markersize=9,
            label=f"zero eigenvalues: {zeros}")
    ax.set_xlabel("number of the eigenvalue (sorted)")
    ax.set_title(f"$A^T A$ for the {title}")
    ax.legend(loc="lower right")
axes[0].set_ylabel("eigenvalue of $A^T A$")
save_figure(fig, "system_eigenvalues",
            "The 256 eigenvalues of $A^T A$, sorted (horizontal axis: their number "
            "from 1 to 256; vertical axis: their value, a pure number), for the "
            "equation system of the commutant of the eight gammas (left) and of "
            "the 28 generators $S^{ab}$ (right). The number of zero eigenvalues "
            "(orange dots) is the dimension of the commutant: one on the left "
            "(only multiples of 1: Pin(4,4) is irreducible), two on the right "
            "(the projectors onto the two halves). On the left the eigenvalues "
            "come in the exact steps 0, 4, 8, ..., 32, four times the number of "
            "gammas that anticommute with a Clifford product.")
```

Out [12]:

```
Figure 05b.2 saved as Revision/textbook/figures/05b_2_system_eigenvalues.png
```

Text cell:

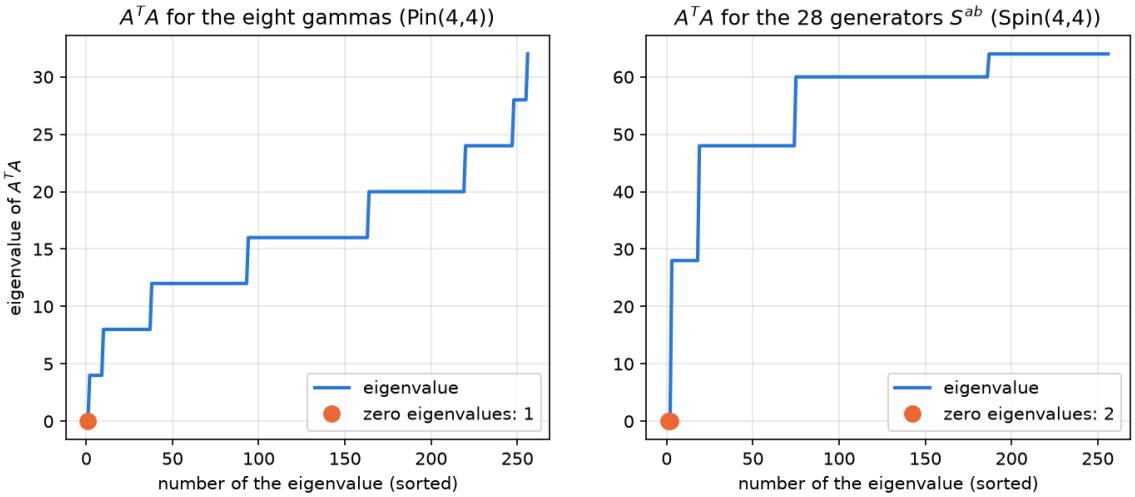

Figure 149: The 256 eigenvalues of A^TA, sorted (horizontal axis: their number from 1 to 256; vertical axis: their value, a pure number), for the equation system of the commutant of the eight gammas (left) and of the 28 generators S^{ab} (right). The number of zero eigenvalues (orange dots) is the dimension of the commutant: one on the left (only multiples of 1: Pin(4,4) is irreducible), two on the right (the projectors onto the two halves). On the left the eigenvalues come in the exact steps 0, 4, 8, ..., 32, four times the number of gammas that anticommute with a Clifford product. (Notebook 05b, figure 2.)

```
The next cell draws how the commutant shrinks when the gammas are imposed one at
a time (section 9). The vertical axis is logarithmic with base 2, so halving is a
step of equal size.
```

In [13]:

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(range(9), halving, "o-", color="#2a78d6", linewidth=2, markersize=8)
for j in range(9):
    ax.annotate(str(halving[j]), (j, halving[j]), textcoords="offset points",
                xytext=(8, 4))
ax.set_yscale("log", base=2)
ax.set_yticks([1, 4, 16, 64, 256], ["1", "4", "16", "64", "256"])
ax.set_xticks(range(9), ["none"] + COORDS)
ax.set_xlabel("gammas imposed, in the order x0, x1, ..., x7 (the last one named)")
ax.set_ylabel("dimension of the commutant")
ax.set_title("Each further gamma halves the commutant")
save_figure(fig, "commutant_halving",
            "The dimension of the space of 16 by 16 matrices that commute with the "
            "gammas imposed so far (vertical axis, logarithmic with base 2), when the "
            "gammas are imposed one at a time in the order $x0$ to $x7$ (horizontal "
            "axis: the last gamma added). Without a condition all 256 entries are free; "
            "every further gamma halves the dimension, and with all eight only the "
            "multiples of the identity remain (dimension 1), computed exactly.")
```

Out [13]:

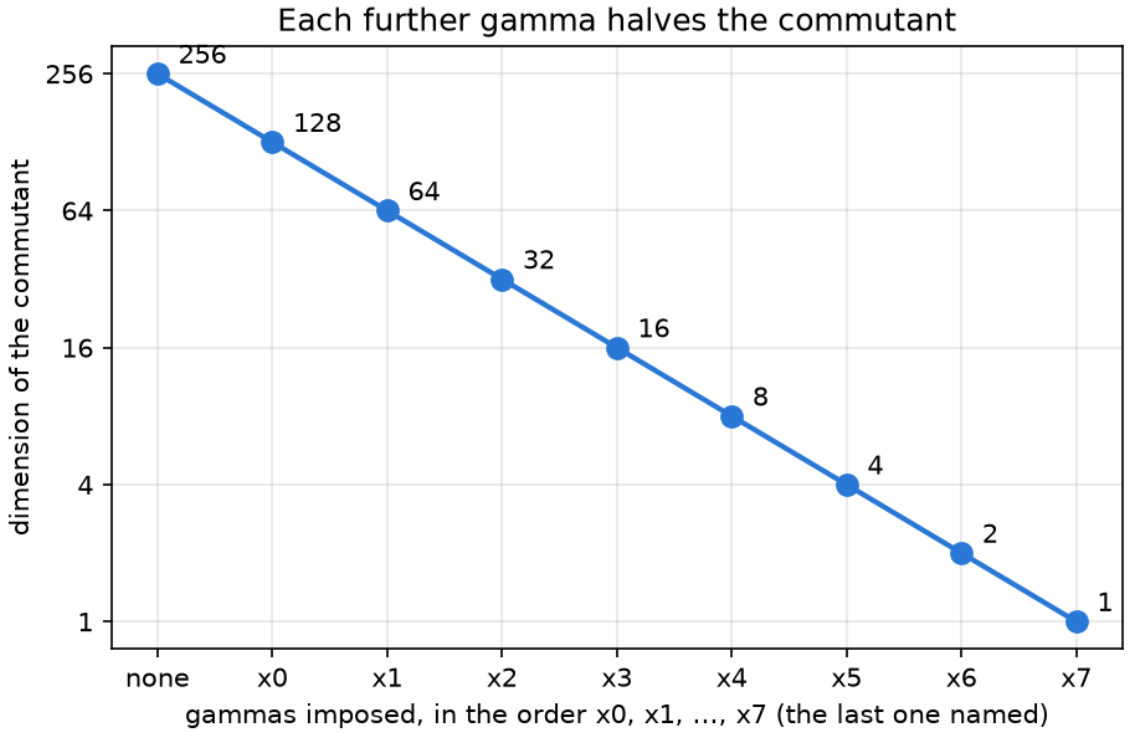

Figure 150: The dimension of the space of 16 by 16 matrices that commute with the gammas imposed so far (vertical axis, logarithmic with base 2), when the gammas are imposed one at a time in the order $x0$ to $x7$ (horizontal axis: the last gamma added). Without a condition all 256 entries are free; every further gamma halves the dimension, and with all eight only the multiples of the identity remain (dimension 1), computed exactly. (Notebook 05b, figure 3.)

```
Figure 05b.3 saved as Revision/textbook/figures/05b_3_commutant_halving.png
```

Text cell:

```
The next cell draws the two basis matrices of the Spin(4,4) commutant as sympy
returned them, and $P_-$ and $P_+$. The basis matrices may look different from the
projectors (sympy picks its own basis); the check above showed that both pairs span
the same space: every matrix that commutes with all $S^{ab}$ is $pP_- + qP_+$, a
number $p$ on the first half and a number $q$ on the second. Before drawing, the
cell checks what the caption of the picture states about the two basis matrices:
each is diagonal, has no negative entry and is constant on each half.
```

In [14]:

```
from matplotlib.colors import LinearSegmentedColormap

# What the caption below says about the two basis matrices, checked: each is
# diagonal, has no negative entry, and is constant on each half (rows 1-8, 9-16).
shapes_ok = all(np.array_equal(b, np.diag(np.diag(b))) and b.min() >= 0
                and len(set(np.diag(b)[:8].tolist())) == 1
                and len(set(np.diag(b)[8:].tolist())) == 1 for b in basis_spin)
check(shapes_ok, "the two basis matrices are diagonal, nonnegative and constant on "
      "each half")
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
pictures = [(basis_spin[0], "sympy basis matrix 1"),
            (basis_spin[1], "sympy basis matrix 2"),
            (P_minus, "$P_- = \\mathrm{diag}(1_8, 0)$"),
            (P_plus, "$P_+ = \\mathrm{diag}(0, 1_8)$")]
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.9))
for k, (ax, (matrix, title)) in enumerate(zip(axes, pictures)):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if k == 0:
        ax.set_ylabel("row")
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "spin_commutant",
            "Heat maps of the two basis matrices of the commutant of the 28 generators "
            "$S^{ab}$ found by the exact solver (left two) and of the chiral projectors "
            "$P_-$ and $P_+$ (right two); horizontal axis the column, vertical axis the "
            "row (1 to 16); grey $0$, red $+1$ (no entry is negative). All four are "
            "diagonal and constant on each half (rows 1 to 8 and 9 to 16, separated by "
            "the black lines): every matrix that commutes with Spin(4,4) is a number "
            "times $P_-$ plus a number times $P_+$.")
```

Out [14]:

```
PASS the two basis matrices are diagonal, nonnegative and constant on each half
```

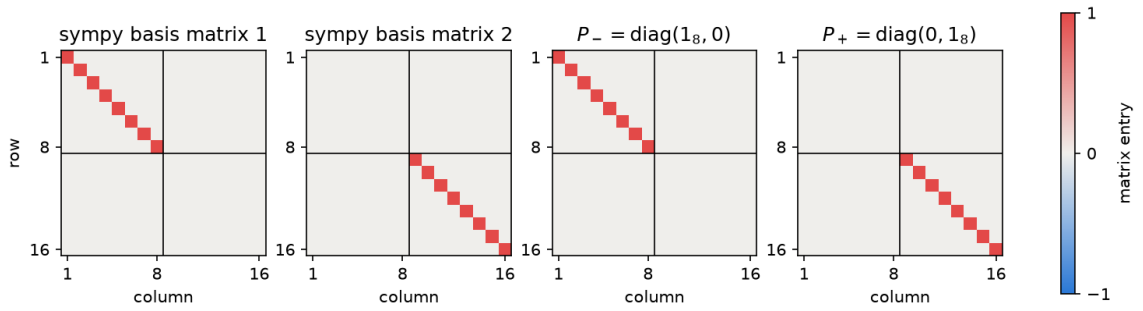

Figure 151: Heat maps of the two basis matrices of the commutant of the 28 generators S^{ab} found by the exact solver (left two) and of the chiral projectors P_- and P_+ (right two); horizontal axis the column, vertical axis the row (1 to 16); grey 0, red +1 (no entry is negative). All four are diagonal and constant on each half (rows 1 to 8 and 9 to 16, separated by the black lines): every matrix that commutes with Spin(4,4) is a number times P_- plus a number times P_+. (Notebook 05b, figure 4.)

Out [14]:

```
Figure 05b.4 saved as Revision/textbook/figures/05b_4_spin_commutant.png
```

Text cell:

```
## 11. The two halves: irreducible and inequivalent

Every $S^{ab}$ is $\mathrm{diag}(S_-^{ab}, S_+^{ab})$ with $8 \times 8$ blocks. The
next cell computes, exactly:

- the commutant of the 28 blocks $S_-^{ab}$ (64 unknowns): dimension 1 means the
  first half is irreducible; the same for the 28 blocks $S_+^{ab}$;
- the intertwiners $X$ with $X S_+^{ab} = S_-^{ab} X$ and with $X S_-^{ab} =
  S_+^{ab} X$ (with ‘commutation_system‘ the pairs are $(S_-, S_+)$ and $(S_+,
  S_-)$): dimension 0 means the only intertwiner is zero, so the halves are
  inequivalent;
- the **negative control**: the artificial $16 \times 16$ matrices
  $\mathrm{diag}(S_-^{ab}, S_-^{ab})$, two copies of the *same* half. Their
  commutant must have dimension 4 ($X = \begin{pmatrix} p 1_8 & q 1_8 \\ r 1_8 & s 1_8
  \end{pmatrix}$): this shows that the computation would detect equivalent halves.
```

In [15]:

```
minus_blocks = [g[:8, :8] for g in generators]  # the blocks of 2 S^ab on half -
plus_blocks = [g[8:, 8:] for g in generators]  # the blocks of 2 S^ab on half +
dim_minus = 64 - exact_rank(commutation_system(minus_blocks, minus_blocks))
dim_plus = 64 - exact_rank(commutation_system(plus_blocks, plus_blocks))
dim_minus_plus = 64 - exact_rank(commutation_system(minus_blocks, plus_blocks))
dim_plus_minus = 64 - exact_rank(commutation_system(plus_blocks, minus_blocks))
```

```
Z8 = np.zeros((8, 8), dtype=np.int64)
doubled = [np.block([[m, Z8], [Z8, m]]) for m in minus_blocks]  # the control
dim_control = 256 - exact_rank(commutation_system(doubled, doubled))
say(f"commutant on half -: {dim_minus}; on half +: {dim_plus}")
say(f"intertwiners: {dim_minus_plus} and {dim_plus_minus}")
say(f"control (two copies of half -): commutant dimension {dim_control}")
check_reproduces(dim_minus == 1 and dim_plus == 1
                 and "dimension 1 (sympy 1)"
                 in detail("python", "spin_halves_irreducible")
                 and recorded("python", "spin_halves_irreducible")
                 and recorded("wolfram", "chiral_halves_irreducible"),
                 "each chiral half is an irreducible representation of Spin(4,4)",
                 record=record_of("python", "spin_halves_irreducible"))
check_reproduces(dim_minus_plus == 0 and dim_plus_minus == 0
                 and "dimension 0 (sympy 0)"
                 in detail("python", "spin_halves_inequivalent")
                 and recorded("python", "spin_halves_inequivalent")
                 and recorded("wolfram", "chiral_halves_inequivalent_intertwiners_0"),
                 "the two halves are inequivalent: the only intertwiner is zero",
                 record=record_of("python", "spin_halves_inequivalent"))
check(dim_control == 4,
      "negative control: two copies of the same half give commutant dimension 4")
```

Out [15]:

```
commutant on half -: 1; on half +: 1
intertwiners: 0 and 0
control (two copies of half -): commutant dimension 4
PASS each chiral half is an irreducible representation of Spin(4,4)
     reproduces Revision/algebra/reports/python-algebra.json, check
    spin_halves_irreducible
PASS the two halves are inequivalent: the only intertwiner is zero
     reproduces Revision/algebra/reports/python-algebra.json, check
    spin_halves_inequivalent
PASS negative control: two copies of the same half give commutant dimension 4
```

Text cell:

```
**Why Pin(4,4) sees one block of 16 and Spin(4,4) two blocks of 8.** The next
cell checks the recorded fact $\gamma^a P_- = P_+ \gamma^a$: every gamma (an odd
element of Pin(4,4), a *reflection*) carries the first half into the second and
back. So no half is invariant under Pin(4,4), while each half is invariant under
the even elements. Then it draws all the dimensions computed in this notebook.
```

In [16]:

```
check_reproduces(all(np.array_equal(gamma[x] @ P_minus, P_plus @ gamma[x])
                     for x in COORDS)
                 and recorded("python", "reflections_exchange_halves"),
                 "gamma^a P_- = P_+ gamma^a: every reflection exchanges the two halves",
                 record=record_of("python", "reflections_exchange_halves"))
```

```
labels = ["Pin(4,4) on all 16", "Spin(4,4) on all 16", "Spin(4,4) on half -",
          "Spin(4,4) on half +", "intertwiners - to +", "intertwiners + to -",
          "control: two equal halves"]
values = [dimension_pin, dimension_spin, dim_minus, dim_plus, dim_minus_plus,
          dim_plus_minus, dim_control]
colors = ["#2a78d6"] * 6 + ["#eb6834"]  # blue: the theory, orange: the control
fig, ax = plt.subplots(figsize=(8.0, 4.4))
positions = np.arange(len(labels))[::-1]  # the first label at the top
ax.barh(positions, values, color=colors, edgecolor="white", linewidth=2)
for y, v in zip(positions, values):
    ax.text(v + 0.06, y, str(v), va="center")
ax.set_yticks(positions, labels)
ax.set_xlim(0, 4.6)
ax.set_xlabel("dimension of the space of solutions")
ax.set_title("Commutants and intertwiners")
legend_squares = [Patch(color="#2a78d6", label="the author's spinor (computed)"),
                  Patch(color="#eb6834", label="negative control (artificial)")]
ax.legend(handles=legend_squares, loc="upper right")
save_figure(fig, "dimensions",
            "The dimensions computed exactly in this notebook (horizontal axis): the "
            "commutant of Pin(4,4) on the 16 components is 1 (irreducible); the "
            "commutant of Spin(4,4) is 2 (two invariant halves); the commutant on each "
            "half is 1 (each half irreducible); the intertwiners between the halves are "
            "0 in both directions (inequivalent halves). The orange bar is the negative "
            "control with two copies of the same half: equivalent halves give 4, so the "
            "value 2 above proves inequivalence.")
```

Out [16]:

```
PASS gamma^a P_- = P_+ gamma^a: every reflection exchanges the two halves
     reproduces Revision/algebra/reports/python-algebra.json, check
    reflections_exchange_halves
```

Out [16]:

```
Figure 05b.5 saved as Revision/textbook/figures/05b_5_dimensions.png
```

Text cell:

```
## 12. The last check

The last cell checks that the five figure files exist and prints the number of
checks that passed.
```

In [17]:

```
FIGURES = ["05b_1_clifford_products.png", "05b_2_system_eigenvalues.png",
           "05b_3_commutant_halving.png", "05b_4_spin_commutant.png",
           "05b_5_dimensions.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the five figure files of notebook 05b exist")
all_checks_passed()
```

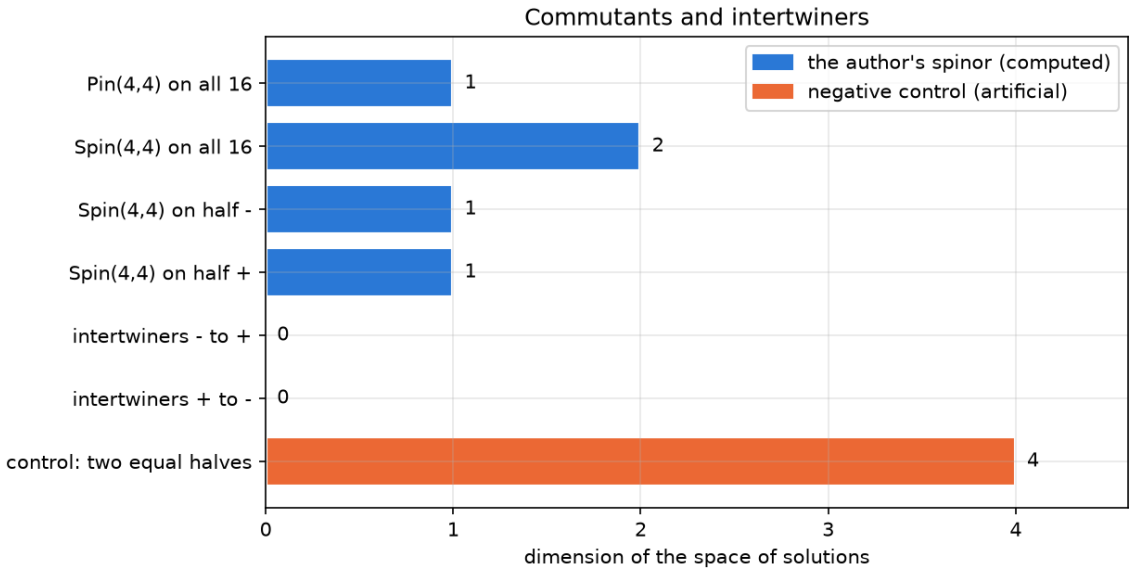

Figure 152: The dimensions computed exactly in this notebook (horizontal axis): the commutant of Pin(4,4) on the 16 components is 1 (irreducible); the commutant of Spin(4,4) is 2 (two invariant halves); the commutant on each half is 1 (each half irreducible); the intertwiners between the halves are 0 in both directions (inequivalent halves). The orange bar is the negative control with two copies of the same half: equivalent halves give 4, so the value 2 above proves inequivalence. (Notebook 05b, figure 5.)

Out [17]:

```
PASS the five figure files of notebook 05b exist
ALL 17 CHECKS PASSED (notebook 05b)
```

Text cell:

```
## 13. What this notebook showed

- A matrix equation $LX = XR$ for an unknown matrix is a system of linear
  equations; its solutions form a space of dimension (number of unknowns) minus
  (rank), and the rank can be computed exactly.
- The 256 Clifford products of the author's gammas are independent: they span all
  $16 \times 16$ matrices; the 128 even ones are block diagonal and span all
  block-diagonal matrices ($64 + 64$).
- Only the multiples of 1 commute with all eight gammas: the 16 components carry
  an **irreducible** representation of Pin(4,4). The equation system has exactly
  one zero eigenvalue of $A^T A$, and all its eigenvalues are predicted exactly by
  the sign rule. Each further gamma halves the commutant: 256, 128, ..., 1.
- The matrices that commute with the 28 generators $S^{ab}$ are exactly
  $pP_- + qP_+$ (dimension 2): under Spin(4,4) the components split into the two
  chiral halves of 8.
- Each half is irreducible, and the two halves are **inequivalent** (no nonzero
  intertwiner); the negative control with two equal halves gives 4 instead of 2.
  The gammas exchange the halves, which is how the two inequivalent halves of
```

```
  Spin(4,4) form one irreducible representation of Pin(4,4).
- The ranks 256 and 128 and the dimensions 1, 2, 1, 1, 0 and 0 equal the numbers
  recorded in ‘Revision/algebra/reports/python-algebra.json‘ and
  ‘wolfram-algebra.json‘. The halving sequence 256, 128, ..., 1, the eigenvalue
  steps of $A^TA$ and the value 4 of the negative control are this notebook’s own
  computations.
```

5.17 Line-by-line walk-through of Notebook 05b

The notebook has 17 code cells. In [1] is the set-up cell, word for word the one of Notebook 05a explained in Section 5.10, except the line `NOTEBOOK_ID = "05b" # this notebook: chapter 05 , example b`; its comment lines repeat the instructions of Section 5.15. As in Section 5.10, the docstrings of the functions (the texts in triple quotes below a `def` line) are left out of the quotations, and a long caption given to `save_figure` is replaced by a line `...`); the full caption is printed under the figure in the complete text of the notebook.

In [2], the gammas and the recorded checks.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1
```

These lines load the modules and read the gammas exactly as In [2] and In [3] of Notebook 05a do (Section 5.10): `fixture` is the record, `COORDS` the coordinate names, `ETA` the signs η_{aa}, `gamma` the eight matrices of whole numbers and `I16` the identity.

```
REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    return VERDICTS[(key, name)][0] == "pass"


def detail(key, name):
    return VERDICTS[(key, name)][1]
```

```
def record_of(key, name):
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together
```

The two reports are read and the helpers `recorded`, `record_of` and `check_reproduces` are defined as in Notebook 05a. The new helper `detail(key, name)` returns the detail text of a recorded check (`[1]`, the second element of the stored pair); several checks below compare a number with the number written in that text.

```
clifford_ok = all(
    np.array_equal(gamma[x] @ gamma[y] + gamma[y] @ gamma[x],
                   2 * ETA[x] * I16 if x == y else 0 * I16)
    for x in COORDS for y in COORDS)
check_reproduces(clifford_ok and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

The Clifford relation is checked for the 64 pairs in one expression: `all(... for x in COORDS for y in COORDS)` runs through all pairs and is true when every comparison is; `0 * I16` is the zero matrix.

In [3], the tools.

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over QQ


def commutation_system(lefts, rights):
    blocks = []
    for L, R in zip(lefts, rights):
        n, m = L.shape[0], R.shape[0]
        blocks.append(np.kron(L, np.eye(m, dtype=np.int64))
                      - np.kron(np.eye(n, dtype=np.int64), R.T))
    return np.vstack(blocks)  # the blocks stacked on top of each other
```

`commutation_system` builds the coefficient matrix of Section 5.13 for a list of equations $LX - XR = 0$, one for each pair (L, R) of the two lists (`zip` pairs them). For each pair it reads the sizes n and m, forms $\mathrm{kron}(L, 1_m) - \mathrm{kron}(1_n, R^T)$ with numpy's Kronecker product `np.kron`, and `np.vstack` stacks all blocks on top of each other.

```
def exact_rank(A):
    return DomainMatrix.from_list(A.tolist(), sp.QQ).rank()
```

```
def solution_basis(A, n, m):
    null = DomainMatrix.from_list(A.tolist(), sp.QQ).nullspace().to_Matrix()
    return [np.array(null.row(k).tolist()[0], dtype=float).reshape(n, m)
            for k in range(null.rows)]
```

`DomainMatrix.from_list(..., sp.QQ)` makes an exact sympy matrix whose entries are rational numbers (`QQ` is sympy's name for the rational numbers). Its `.rank()` is computed by Gaussian elimination with fractions, without rounding. `.nullspace()` returns a basis of the solutions of $Ay = 0$, one solution per row; `.to_Matrix()` turns it into an ordinary sympy matrix. `solution_basis` takes each row (`null.row(k)`), turns it into a numpy array of floating-point numbers and reshapes the nm numbers into an $n \times m$ matrix (the unknowns were listed row by row, so `reshape` restores the matrix).

```
rng = np.random.default_rng(12345)  # random numbers with a fixed seed
X = rng.integers(-5, 6, size=(16, 16))  # a random 16 x 16 matrix, entries -5..5
L, R = gamma["x1"], gamma["x4"]
A = commutation_system([L], [R])
say(f"one pair (L, R) gives {A.shape[0]} equations for {A.shape[1]} unknowns")
check(np.array_equal(A @ X.reshape(256), (L @ X - X @ R).reshape(256)),
      "the coefficient matrix applied to X row by row gives L X - X R")
```

The construction is tested before it is used. `np.random.default_rng(12345)` is a random-number generator started with the fixed **seed** 12345, so that every run draws the same numbers; `rng.integers(-5, 6, size=(16, 16))` draws a 16×16 matrix of whole numbers from -5 to 5 (the upper end 6 is excluded). For $L = \gamma^{(x1)}$ and $R = \gamma^{(x4)}$ the coefficient matrix has 256 rows and 256 columns (printed). The check multiplies it with the 256 entries of X listed row by row (`X.reshape(256)`) and compares the result with the entries of $LX - XR$ listed in the same order.

In [4], the warm-up.

```
M = np.array([[0, 1], [1, 0]], dtype=np.int64)
A_small = commutation_system([M], [M])  # 4 equations for the 4 entries p, q, r, s
rank_small = exact_rank(A_small)
basis_small = solution_basis(A_small, 2, 2)
say(f"coefficient matrix (rows = equations, columns = p, q, r, s):")
for row in A_small.tolist():
    say(f"    {row}")
say(f"rank {rank_small}; dimension of the solution space {4 - rank_small}")
for k, b in enumerate(basis_small, 1):
    say(f"basis solution {k}: rows {b.astype(int).tolist()}")
check(rank_small == 2 and len(basis_small) == 2,
      "warm-up: the 2 x 2 matrices commuting with M form a space of dimension 2")
```

For $M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $X = \begin{pmatrix} p & q \\ r & s \end{pmatrix}$ the equation $MX - XM = 0$ reads $r - q = 0$, $s - p = 0$, $p - s = 0$, $q - r = 0$, the four printed rows of coefficients (columns p, q, r, s). Two of them repeat the other two, so the rank is 2 and the solution space has dimension $4 - 2 = 2$. `enumerate(basis_`

`small, 1)` numbers the basis solutions from 1; `.astype(int)` turns them into whole numbers for printing. The two printed solutions are M itself and the identity, as the hand computation of the notebook's text predicts.

In [5], the 256 products of different gammas.

```
import itertools  # subsets of a given size
from math import comb  # comb(8, k): the number of subsets of size k

products = {}  # degree k -> list of the Clifford products of degree k
for k in range(9):
    products[k] = []
    for subset in itertools.combinations(COORDS, k):
        p = I16
        for x in subset:
            p = p @ gamma[x]
        products[k].append(p)
```

For each degree $k = 0, \ldots, 8$ the cell lists every subset of k directions in the order $x0, \ldots, x7$ and multiplies their gammas; for $k = 0$ the only subset is empty and the product is the identity. `products[k]` is the list of the $\binom{8}{k}$ products of degree k; `comb(8, k)` from the module `math` computes this binomial number.

```
rows_all = np.array([p.reshape(256) for k in range(9) for p in products[k]])
rows_even = np.array([p.reshape(256) for k in (0, 2, 4, 6, 8) for p in products[k]])
rank_all, rank_even = exact_rank(rows_all), exact_rank(rows_even)
cumulative = []  # the rank after adding the degrees 0, 1, ..., k
for k in range(9):
    cumulative.append(exact_rank(np.array(
        [p.reshape(256) for j in range(k + 1) for p in products[j]])))
```

Each product is written as one row of 256 numbers (`reshape(256)`), giving a 256×256 array for all products and a 128×256 array for the even ones; their exact ranks are computed. `cumulati ve[k]` is the rank of all products of degrees 0 to k.

```
for k in range(9):
    say(f"degree {k}: {len(products[k]):2d} products (comb(8, {k}) = {comb(8, k):2d});"
        f" rank of all products up to degree {k}: {cumulative[k]}")
say(f"rank of all 256 products: {rank_all}; rank of the 128 even products: "
    f"{rank_even}")
check_reproduces(rank_all == 256 and f"rank {rank_all} (Fraction elimination)"
                 in detail("python", "clifford_products_span_M16")
                 and recorded("python", "clifford_products_span_M16")
                 and recorded("wolfram", "Clifford_basis_spans_full_matrix_algebra"),
                 "the 256 Clifford products are independent: they span all 16 x 16 "
                 "matrices",
                 record=record_of("python", "clifford_products_span_M16"))
```

The nine printed lines show the numbers of products 1, 8, 28, 56, 70, 56, 28, 8, 1 and the cumulative ranks 1, 9, 37, 93, 163, 219, 247, 255, 256: each degree raises the rank by exactly the number of its

products, so no product is a combination of the others. (`{len(products[k]):2d}` writes a whole number in two places.) The check requires the rank 256 and that the recorded detail text contains the words `rank 256 (Fraction elimination)`: the record found the same number.

```
even_diagonal = all(not p[:8, 8:].any() and not p[8:, :8].any()
                    for k in (0, 2, 4, 6, 8) for p in products[k])
odd_off = all(not p[:8, :8].any() and not p[8:, 8:].any()
              for k in (1, 3, 5, 7) for p in products[k])
check_reproduces(rank_even == 128 and even_diagonal and odd_off
                 and f"rank {rank_even} (Fraction)"
                 in detail("python", "even_products_span_M8_plus_M8")
                 and recorded("python", "even_products_span_M8_plus_M8")
                 and recorded("wolfram", "even_subalgebra_dimension"),
                 "the 128 even products are block diagonal and independent (64 + 64); "
                 "the odd ones are block off-diagonal",
                 record=record_of("python", "even_products_span_M8_plus_M8"))
```

`p[:8, 8:]` is the top-right block (rows 1 to 8, columns 9 to 16) and `p[8:, :8]` the bottom-left block; `.any()` is true when some entry is not zero. So `even_diagonal` says that every even product has zero off-diagonal blocks, and `odd_off` that every odd product has zero diagonal blocks, the block rule of Section 5.5. The check adds the rank 128 and the recorded number.

In [6], the picture of the products and ranks.

```
from matplotlib.patches import Patch  # a coloured square for a legend

degrees = np.arange(9)
counts = [len(products[k]) for k in range(9)]
colors = ["#2a78d6" if k % 2 == 0 else "#eb6834" for k in range(9)]
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
axes[0].bar(degrees, counts, color=colors, edgecolor="white", linewidth=2)
for k in range(9):
    axes[0].text(k, counts[k] + 1.5, str(counts[k]), ha="center")
axes[0].set_xlabel("degree $k$ (number of different gammas multiplied)")
axes[0].set_ylabel("number of products")
axes[0].set_title(r"$\binom{8}{k}$ products of degree $k$")
axes[0].set_ylim(0, 80)
axes[0].legend(handles=[Patch(color="#2a78d6", label="even degree"),
                        Patch(color="#eb6834", label="odd degree")],
               loc="upper right")
```

`np.arange(9)` is the array $0, 1, \ldots, 8$. `k % 2` is the remainder of k divided by 2, so even degrees get blue and odd degrees orange. `axes[0].bar` draws one bar per degree with the height of its count, and `text` writes the count just above each bar. A `Patch` is a coloured square for a legend that is made by hand (the bars themselves carry no labels).

```
added = np.cumsum(counts)  # how many products have been added up to degree k
axes[1].plot(added, cumulative, "o-", color="#2a78d6", linewidth=2, markersize=8,
             label="exact rank")
axes[1].plot([0, 256], [0, 256], ":", color="black", linewidth=1,
```

```
             label="rank = number of products")
axes[1].set_xlabel("number of products added (degrees 0 to $k$)")
axes[1].set_ylabel("rank")
axes[1].set_title("The rank grows by one for every product")
axes[1].legend(loc="upper left")
save_figure(fig, "clifford_products",
            ...)
```

`np.cumsum` adds up the counts: 1, 9, 37, 93, 163, 219, 247, 255, 256 products after the degrees 0 to k. The right picture plots the cumulative ranks against these numbers (`"o-"` draws dots joined by lines) and the dotted line rank = number of products. In `05b_1_clifford_products.png` the dots lie exactly on the dotted line.

In [7], the commutant of the eight gammas.

```
gammas = [gamma[x] for x in COORDS]
A_pin = commutation_system(gammas, gammas)  # 8 x 256 = 2048 equations
rank_pin = exact_rank(A_pin)
basis_pin = solution_basis(A_pin, 16, 16)
dimension_pin = 256 - rank_pin
say(f"equations {A_pin.shape[0]}, unknowns {A_pin.shape[1]}, rank {rank_pin}, "
    f"dimension of the commutant {dimension_pin}")
```

With $L = R = \gamma^a$ the equations $\gamma^a X - X\gamma^a = 0$ say that X commutes with γ^a; for the eight gammas they are 2048 equations for 256 unknowns. The printed line shows the rank 255 and the dimension $256 - 255 = 1$.

```
only = basis_pin[0]
proportional = np.array_equal(only / only[0, 0], np.eye(16))  # a multiple of 1?
say(f"the basis solution divided by its entry (1,1) equals the identity: "
    f"{proportional}")
check_reproduces(dimension_pin == 1 and len(basis_pin) == 1 and proportional
                 and f"= {dimension_pin} (Fraction)"
                 in detail("python", "pin_commutant_dimension_1")
                 and recorded("python", "pin_commutant_dimension_1")
                 and recorded("wolfram", "Pin44_irreducible_commutant_dim_1"),
                 "only the multiples of 1 commute with all eight gammas: Pin(4,4) acts "
                 "irreducibly on the 16 components",
                 record=record_of("python", "pin_commutant_dimension_1"))
```

The only basis solution is divided by its entry in row 1, column 1 (`only[0, 0]`), and the result is compared with the identity: True. The check is Theorem P, with the recorded dimension.

In [8], the second engine.

```
N_pin = A_pin.T @ A_pin  # 256 x 256, whole numbers, symmetric
eigen_pin = np.linalg.eigvalsh(N_pin.astype(float))  # sorted from small to large
zeros_pin = int(np.sum(np.abs(eigen_pin) < 1e-9))
say(f"numpy: eigenvalues of A^T A below 1e-9: {zeros_pin}")
```

$N = A^T A$ is computed in whole numbers, its 256 eigenvalues in floating-point numbers, and the eigenvalues below 10^{-9} in size are counted: exactly one, the dimension of the commutant again.

```
predicted = []  # the eigenvalue 4 n(P) of every Clifford product P
exact_eigenvectors = True
for k in range(9):
    n_anti = k if k % 2 == 0 else 8 - k  # gammas anticommuting with a product
    for p in products[k]:
        predicted.append(4 * n_anti)
        exact_eigenvectors &= np.array_equal(N_pin @ p.reshape(256),
                                             4 * n_anti * p.reshape(256))
```

The notebook's text explains why every product P of different gammas, written as a row of 256 numbers, is an eigenvector of N with the eigenvalue $4n(P)$, where $n(P)$ is the number of gammas that anticommute with P. By Rule 1 of Section 5.3, a product of even degree k anticommutes with its k factors and commutes with the others, so $n = k$; a product of odd degree anticommutes with the $8 - k$ gammas that are not factors, so $n = 8 - k$. The loop collects the predicted eigenvalue of each product and checks the eigenvector equation exactly (`&=` keeps `exact_eigenvectors` true only while every test passes).

```
predicted = np.sort(np.array(predicted))
values, multiplicities = np.unique(predicted, return_counts=True)
say("predicted eigenvalues of A^T A, written value x multiplicity:")
say("    " + ", ".join(f"{v} x {c}" for v, c in zip(values, multiplicities)))
check(exact_eigenvectors and zeros_pin == 1
      and np.max(np.abs(eigen_pin - predicted)) < 1e-9,
      "A^T A has exactly one zero eigenvalue; every Clifford product P is an "
      "eigenvector with eigenvalue 4 n(P)")
```

The 256 predicted eigenvalues are sorted; `np.unique(..., return_counts=True)` lists each different value once together with how often it occurs. The printed line reads 0 x 1, 4 x 8, 8 x 28, ..., 32 x 1: the value $4n$ occurs as often as there are products with that n. The check requires the exact eigenvector equations, one zero eigenvalue, and that numpy's sorted eigenvalues agree with the sorted predictions to within 10^{-9}. The 256 products are independent, so they are all the eigenvectors, and the prediction is the complete list.

In [9], imposing the gammas one at a time.

```
halving = [256]  # dimension of the commutant of the first j gammas, j = 0, 1, ..., 8
for j in range(1, 9):
    A_j = commutation_system(gammas[:j], gammas[:j])
    halving.append(256 - exact_rank(A_j))
for j in range(9):
    names = ", ".join(COORDS[:j]) if j else "none"
    say(f"gammas imposed: {j} ({names}); commutant dimension {halving[j]}")
check(halving == [256 // 2 ** j for j in range(9)],
      "every further gamma halves the commutant: 256, 128, 64, ..., 2, 1")
```

`gammas[:j]` is the list of the first j gammas. For $j = 1, \ldots, 8$ the exact dimension of their commutant is appended to `halving`, which starts with 256 (no condition). The printed lines show 256, 128,

64, 32, 16, 8, 4, 2, 1, and the check compares the list with $256/2^j$ (`2 ** j` is 2^j; `//` divides whole numbers).

In [10], the 28 generators.

```
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 pairs
S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}
entries = sorted({float(v) for s in S.values() for v in s.flat})
say(f"{len(S)} generators S^ab; their entries are {entries}")
```

`enumerate(COORDS)` gives each name with its position `i`; `COORDS[i + 1:]` is the list of the names after it, so `pairs` lists the 28 planes (a, b) with a before b. `S` stores $\frac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$ for each plane; the division by 4 makes floating-point numbers, but halves are stored exactly. The printed line shows 28 generators with the entries -0.5, 0, 0.5.

```
check_reproduces(len(S) == 28 and entries == [-0.5, 0.0, 0.5]
                 and all(np.array_equal(S[(a, b)], gamma[a] @ gamma[b] / 2)
                         for a, b in pairs)
                 and recorded("python", "S_definition"),
                 "S^ab = (1/4)[gamma^a, gamma^b] = (1/2) gamma^a gamma^b: 28 matrices "
                 "with entries 0, +1/2, -1/2",
                 record=record_of("python", "S_definition"))
check_reproduces(all(not s[:8, 8:].any() and not s[8:, :8].any() for s in S.values())
                 and recorded("python", "S_block_diagonal"),
                 "every S^ab is block diagonal: it maps each chiral half into itself",
                 record=record_of("python", "S_block_diagonal"))
```

The first check confirms $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$ and the entries; the second that every generator is block diagonal (it is even).

In [11], the commutant of the generators.

```
generators = [(gamma[a] @ gamma[b]).astype(np.int64) for a, b in pairs]  # 2 S^ab
A_spin = commutation_system(generators, generators)  # 7168 equations
rank_spin = exact_rank(A_spin)
basis_spin = solution_basis(A_spin, 16, 16)
dimension_spin = 256 - rank_spin
```

For the exact computation the cell uses the whole-number matrices $2S^{ab} = \gamma^a\gamma^b$, which have the same commutant as the S^{ab} (a matrix commutes with S exactly when it commutes with $2S$). The $28 \times 256 = 7168$ equations have the exact rank 254 and a solution space of dimension 2.

```
P_minus = np.diag([1] * 8 + [0] * 8)  # diag(1_8, 0): components 1 to 8
P_plus = np.diag([0] * 8 + [1] * 8)  # diag(0, 1_8): components 9 to 16
together = np.array([b.reshape(256) for b in basis_spin]
                    + [P_minus.reshape(256), P_plus.reshape(256)])
same_span = np.linalg.matrix_rank(together) == 2  # four matrices, two directions
eigen_spin = np.linalg.eigvalsh((A_spin.T @ A_spin).astype(float))
zeros_spin = int(np.sum(np.abs(eigen_spin) < 1e-9))
```

`np.diag(list)` makes the diagonal matrix with the listed diagonal: the two projectors. The two basis solutions and the two projectors, written as rows, form four rows; if their rank is 2, the two pairs span the same space (`np.linalg.matrix_rank` computes a floating-point rank, which is reliable for such small whole numbers). The second engine counts the zero eigenvalues of $A^T A$.

```
say(f"equations {A_spin.shape[0]}, rank {rank_spin}, dimension {dimension_spin}; "
    f"numpy: zero eigenvalues of A^T A: {zeros_spin}")
say(f"the basis and P_-, P_+ span the same space: {same_span}")
check_reproduces(dimension_spin == 2 and zeros_spin == 2 and same_span
                 and f"= {dimension_spin} (Fraction)"
                 in detail("python", "spin_commutant_dimension_2")
                 and recorded("python", "spin_commutant_dimension_2")
                 and recorded("wolfram", "Spin44_commutant_dim_2_chiral_projectors"),
                 "the commutant of the 28 S^ab has dimension 2, spanned by P_- and P_+",
                 record=record_of("python", "spin_commutant_dimension_2"))
```

The printed lines report 7168 equations, rank 254, dimension 2, two zero eigenvalues and True. The check confirms the recorded dimension 2.

In [12], the picture of the eigenvalues of $A^T A$.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for ax, eig, zeros, title in [
        (axes[0], eigen_pin, zeros_pin, "eight gammas (Pin(4,4))"),
        (axes[1], eigen_spin, zeros_spin, "28 generators $S^{ab}$ (Spin(4,4))")]:
    index = np.arange(1, 257)  # the eigenvalues numbered 1 to 256
    ax.plot(index, eig, color="#2a78d6", linewidth=2, label="eigenvalue")
    ax.plot(index[:zeros], eig[:zeros], "o", color="#eb6834", markersize=9,
            label=f"zero eigenvalues: {zeros}")
    ax.set_xlabel("number of the eigenvalue (sorted)")
    ax.set_title(f"$A^T A$ for the {title}")
    ax.legend(loc="lower right")
axes[0].set_ylabel("eigenvalue of $A^T A$")
save_figure(fig, "system_eigenvalues",
            ...)
```

The loop draws the same kind of picture for the two systems: the sorted eigenvalues against their numbers 1 to 256 (`np.arange(1, 257)`), and orange dots on the first `zeros` of them, the zero eigenvalues. In `05b_2_system_eigenvalues.png` the left curve climbs in the steps 0, 4, 8, ..., 32 with one dot at 0; the right curve has two dots at 0.

In [13], the picture of the halving.

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(range(9), halving, "o-", color="#2a78d6", linewidth=2, markersize=8)
for j in range(9):
    ax.annotate(str(halving[j]), (j, halving[j]), textcoords="offset points",
                xytext=(8, 4))
ax.set_yscale("log", base=2)
ax.set_yticks([1, 4, 16, 64, 256], ["1", "4", "16", "64", "256"])
ax.set_xticks(range(9), ["none"] + COORDS)
```

```
ax.set_xlabel("gammas imposed, in the order x0, x1, ..., x7 (the last one named)")
ax.set_ylabel("dimension of the commutant")
ax.set_title("Each further gamma halves the commutant")
save_figure(fig, "commutant_halving",
            ...)
```

The nine dimensions are drawn against the number of gammas imposed; `annotate` writes each value next to its dot, shifted by 8 and 4 points. `set_yscale("log", base=2)` makes the vertical axis logarithmic with base 2, so that each halving is a step of the same size and the dots lie on a straight line. In `05b_3_commutant_halving.png` the line descends evenly from 256 to 1.

In [14], the picture of the Spin(4,4) commutant.

```
from matplotlib.colors import LinearSegmentedColormap

# What the caption below says about the two basis matrices, checked: each is
# diagonal, has no negative entry, and is constant on each half (rows 1-8, 9-16).
shapes_ok = all(np.array_equal(b, np.diag(np.diag(b))) and b.min() >= 0
                and len(set(np.diag(b)[:8].tolist())) == 1
                and len(set(np.diag(b)[8:].tolist())) == 1 for b in basis_spin)
check(shapes_ok, "the two basis matrices are diagonal, nonnegative and constant on "
      "each half")
```

Before drawing, the cell checks what the caption says. `np.diag(b)` of a matrix is its diagonal, and `np.diag` of that list is the diagonal matrix with it; equality means that `b` is diagonal. `b.min() >= 0` says no entry is negative. `set(...)` of the first eight diagonal entries has one element exactly when they are all equal; the same for the last eight.

```
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
pictures = [(basis_spin[0], "sympy basis matrix 1"),
            (basis_spin[1], "sympy basis matrix 2"),
            (P_minus, "$P_- = \\mathrm{diag}(1_8, 0)$"),
            (P_plus, "$P_+ = \\mathrm{diag}(0, 1_8)$")]
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.9))
for k, (ax, (matrix, title)) in enumerate(zip(axes, pictures)):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if k == 0:
        ax.set_ylabel("row")
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "spin_commutant",
            ...)
```

The colour map of Notebook 05a is made again. `pictures` pairs the four matrices with their titles; in an ordinary string a backslash must be doubled, so `"\\mathrm"` gives `\mathrm`. The loop draws

each as a heat map with the half lines (`enumerate` gives the number `k` of each picture, so that only the first gets the word row). `05b_4_spin_commutant.png` shows four diagonal matrices, each grey or red, constant on each half.

In [15], the two halves.

```
minus_blocks = [g[:8, :8] for g in generators]  # the blocks of 2 S^ab on half -
plus_blocks = [g[8:, 8:] for g in generators]  # the blocks of 2 S^ab on half +
dim_minus = 64 - exact_rank(commutation_system(minus_blocks, minus_blocks))
dim_plus = 64 - exact_rank(commutation_system(plus_blocks, plus_blocks))
dim_minus_plus = 64 - exact_rank(commutation_system(minus_blocks, plus_blocks))
dim_plus_minus = 64 - exact_rank(commutation_system(plus_blocks, minus_blocks))
```

`minus_blocks` are the top-left 8×8 blocks of the 28 matrices $2S^{ab}$, `plus_blocks` the bottom-right ones. Each system has 28×64 equations for the 64 entries of an 8×8 unknown X. With the pairs $(L, R) = (S_-, S_-)$ the solutions are the commutant on the first half; with (S_+, S_+) on the second; with (S_-, S_+) they are the X with $S_- X = X S_+$, the intertwiners from the second half to the first; with (S_+, S_-) those in the other direction.

```
Z8 = np.zeros((8, 8), dtype=np.int64)
doubled = [np.block([[m, Z8], [Z8, m]]) for m in minus_blocks]  # the control
dim_control = 256 - exact_rank(commutation_system(doubled, doubled))
say(f"commutant on half -: {dim_minus}; on half +: {dim_plus}")
say(f"intertwiners: {dim_minus_plus} and {dim_plus_minus}")
say(f"control (two copies of half -): commutant dimension {dim_control}")
```

The negative control builds artificial 16×16 matrices $\mathrm{diag}(S_-, S_-)$ with the same block twice and computes their commutant. The printed lines show the dimensions 1 and 1, the intertwiners 0 and 0, and 4 for the control.

```
check_reproduces(dim_minus == 1 and dim_plus == 1
                 and "dimension 1 (sympy 1)"
                 in detail("python", "spin_halves_irreducible")
                 and recorded("python", "spin_halves_irreducible")
                 and recorded("wolfram", "chiral_halves_irreducible"),
                 "each chiral half is an irreducible representation of Spin(4,4)",
                 record=record_of("python", "spin_halves_irreducible"))
check_reproduces(dim_minus_plus == 0 and dim_plus_minus == 0
                 and "dimension 0 (sympy 0)"
                 in detail("python", "spin_halves_inequivalent")
                 and recorded("python", "spin_halves_inequivalent")
                 and recorded("wolfram", "chiral_halves_inequivalent_intertwiners_0"),
                 "the two halves are inequivalent: the only intertwiner is zero",
                 record=record_of("python", "spin_halves_inequivalent"))
check(dim_control == 4,
      "negative control: two copies of the same half give commutant dimension 4")
```

The three checks state Theorem S in the form of the record (irreducible halves, no intertwiner) and the control value 4 of Lemma 4.

In [16], why Pin(4,4) sees one block, and the dimensions.

```
check_reproduces(all(np.array_equal(gamma[x] @ P_minus, P_plus @ gamma[x])
                     for x in COORDS)
                 and recorded("python", "reflections_exchange_halves"),
                 "gamma^a P_- = P_+ gamma^a: every reflection exchanges the two halves",
                 record=record_of("python", "reflections_exchange_halves"))
```

The check repeats the fact of Section 5.5 that every gamma exchanges the halves.

```
labels = ["Pin(4,4) on all 16", "Spin(4,4) on all 16", "Spin(4,4) on half -",
          "Spin(4,4) on half +", "intertwiners - to +", "intertwiners + to -",
          "control: two equal halves"]
values = [dimension_pin, dimension_spin, dim_minus, dim_plus, dim_minus_plus,
          dim_plus_minus, dim_control]
colors = ["#2a78d6"] * 6 + ["#eb6834"]  # blue: the theory, orange: the control
fig, ax = plt.subplots(figsize=(8.0, 4.4))
positions = np.arange(len(labels))[::-1]  # the first label at the top
ax.barh(positions, values, color=colors, edgecolor="white", linewidth=2)
for y, v in zip(positions, values):
    ax.text(v + 0.06, y, str(v), va="center")
ax.set_yticks(positions, labels)
ax.set_xlim(0, 4.6)
ax.set_xlabel("dimension of the space of solutions")
ax.set_title("Commutants and intertwiners")
legend_squares = [Patch(color="#2a78d6", label="the author's spinor (computed)"),
                  Patch(color="#eb6834", label="negative control (artificial)")]
ax.legend(handles=legend_squares, loc="upper right")
save_figure(fig, "dimensions",
            ...)
```

The seven computed dimensions are drawn as horizontal bars (`barh`), six blue and the control orange. `[::-1]` reverses the array of positions, so that the first label stands at the top; each value is written just right of its bar. `05b_5_dimensions.png` shows the bars 1, 2, 1, 1, 0, 0 and the orange bar 4.

In [17], the last check.

```
FIGURES = ["05b_1_clifford_products.png", "05b_2_system_eigenvalues.png",
           "05b_3_commutant_halving.png", "05b_4_spin_commutant.png",
           "05b_5_dimensions.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the five figure files of notebook 05b exist")
all_checks_passed()
```

The five figure files must exist, and the last line reads ALL 17 CHECKS PASSED (notebook 05b): one check each in In [2], In [3], In [4], In [7], In [8], In [9], In [11], In [14], In [16] and In [17], two each in In [5] and In [10], and three in In [15]. Ten of them reproduce recorded checks; the other seven (in In [3], In [4], In [8], In [9], In [14], the control in In [15], and In [17]) are the notebook's own computations.

5.18 Spin transformations: rotations, boosts and the double cover

A **spin transformation** is a 16×16 matrix R that acts on the components, $\Psi \to R\Psi$, and at the same time turns the eight directions into each other. This section builds them from the generators and proves how they act.

The commutation rules of so(4,4). The **commutator** of two matrices is $[M, N] = MN - NM$. For commutators the **product rule** $[A, BC] = [A, B]C + B[A, C]$ holds, because both sides are $ABC - BCA$ (on the right, $ABC - BAC + BAC - BCA$). For $c \neq d$, $S^{cd} = \frac{1}{2}\gamma^c\gamma^d$, and the product rule with the vector rule of Section 5.12 gives

$$[S^{ab}, \gamma^c\gamma^d] = (\eta^{bc}\gamma^a - \eta^{ac}\gamma^b)\gamma^d + \gamma^c(\eta^{bd}\gamma^a - \eta^{ad}\gamma^b).$$

Each product of two gammas is $\gamma^x\gamma^y = \frac{1}{2}[\gamma^x, \gamma^y] + \frac{1}{2}\{\gamma^x, \gamma^y\} = 2S^{xy} + \eta^{xy}1$ (the Clifford relation for the second part). Inserting this, the four number terms $\eta^{bc}\eta^{ad} - \eta^{ac}\eta^{bd} + \eta^{bd}\eta^{ca} - \eta^{ad}\eta^{cb}$ cancel in pairs, and with $S^{ca} = -S^{ac}$, $S^{cb} = -S^{bc}$ and a division by 2:

$$[S^{ab}, S^{cd}] = \eta^{bc}S^{ad} - \eta^{ac}S^{bd} - \eta^{bd}S^{ac} + \eta^{ad}S^{bc}.$$

These are the commutation rules of the **Lie algebra so(4,4)**, the algebra of infinitesimal turnings of the 4+4 directions. The Revision record checks them for all $28 \times 28 = 784$ pairs of generators.

The exponential of a matrix. For a square matrix M the **exponential** is the power series

$$\exp(M) = 1 + M + \tfrac{1}{2}M^2 + \tfrac{1}{6}M^3 + \cdots = \sum_{k \geq 0} \frac{M^k}{k!},$$

the same series as for e^x; it converges for every square matrix (a fact of analysis, quoted here without proof). Fix a plane (a, b) with $a \neq b$ and put $J = \gamma^a\gamma^b = 2S^{ab}$, so that $\theta S^{ab} = \frac{\theta}{2}J$. Then

$$JJ = \gamma^a\gamma^b\gamma^a\gamma^b = -\gamma^a\gamma^a\gamma^b\gamma^b = -\eta_{aa}\eta_{bb}\,1,$$

where the second step exchanges the two middle factors (they anticommute) and the third is the Clifford relation twice.

- **Rotation** ($\eta_{aa}\eta_{bb} = +1$): $JJ = -1$, so $J^{2n} = (-1)^n$ and $J^{2n+1} = (-1)^n J$. Splitting the series into even and odd powers and using the power series of cosine and sine, $\exp(\frac{\theta}{2}J) = \cos\frac{\theta}{2}\,1 + \sin\frac{\theta}{2}\,J$.
- **Boost** ($\eta_{aa}\eta_{bb} = -1$): $JJ = +1$, every power has the sign $+$, and the series of $\cosh x = \frac{1}{2}(e^x + e^{-x})$ and $\sinh x = \frac{1}{2}(e^x - e^{-x})$ give $\exp(\frac{\theta}{2}J) = \cosh\frac{\theta}{2}\,1 + \sinh\frac{\theta}{2}\,J$.

In both cases the inverse is $\exp(-\theta S^{ab})$: $(c1 + sJ)(c1 - sJ) = c^2 - s^2JJ$, which is $\cos^2 + \sin^2 = 1$ for a rotation and $\cosh^2 - \sinh^2 = 1$ for a boost (from the definitions, $\cosh^2 x - \sinh^2 x = \frac{1}{4}[(e^{2x} + 2 + e^{-2x}) - (e^{2x} - 2 + e^{-2x})] = 1$). The number θ is called the **angle** of a rotation and the **rapidity** of a boost.

How a rotation moves the directions: the half angle. Take the rotation in the plane $(x1, x2)$: $R = c1 + sJ$ with $J = \gamma^{(x1)}\gamma^{(x2)}$, $c = \cos\frac{\theta}{2}$, $s = \sin\frac{\theta}{2}$. By Rule 1, J anticommutes with $\gamma^{(x1)}$ and $\gamma^{(x2)}$ (a factor of a product of two: sign $(-1)^1$) and commutes with the other six gammas. Line by line:

$$R\gamma^{(x1)} = \gamma^{(x1)}(c1 - sJ) = \gamma^{(x1)}R^{-1}, \qquad \text{so} \qquad R\gamma^{(x1)}R^{-1} = \gamma^{(x1)}(c1 - sJ)^2.$$

The first step moves $\gamma^{(x1)}$ to the left through J (a sign); the second multiplies from the right by $R^{-1} = c1 - sJ$. Now

$$(c1 - sJ)^2 = c^2 - 2csJ + s^2JJ = (c^2 - s^2)1 - 2cs\,J = \cos\theta\,1 - \sin\theta\,J,$$

by $JJ = -1$ and the double-angle formulas $\cos^2\frac{\theta}{2} - \sin^2\frac{\theta}{2} = \cos\theta$ and $2\sin\frac{\theta}{2}\cos\frac{\theta}{2} = \sin\theta$. With $\gamma^{(x1)}J = \gamma^{(x1)}\gamma^{(x1)}\gamma^{(x2)} = \gamma^{(x2)}$:

$$R\gamma^{(x1)}R^{-1} = \cos\theta\,\gamma^{(x1)} - \sin\theta\,\gamma^{(x2)}.$$

In the same way $R\gamma^{(x2)}R^{-1} = \gamma^{(x2)}(\cos\theta\,1 - \sin\theta\,J) = \cos\theta\,\gamma^{(x2)} + \sin\theta\,\gamma^{(x1)}$ (because $\gamma^{(x2)}\gamma^{(x1)}\gamma^{(x2)} = -\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x2)} = -\gamma^{(x1)}$), and $R\gamma^cR^{-1} = \gamma^c$ for the other six directions. So conjugation by R turns the directions $x1$ and $x2$ by the **full** angle θ, while R itself contains only the **half** angle $\theta/2$. At $\theta = 2\pi$ the directions are back where they started, but

$$R(2\pi) = \cos\pi\,1 + \sin\pi\,J = -1.$$

A turn by 2π multiplies every spinor by -1; only a turn by 4π gives $R = +1$. This is the defining property of spinors.

How a boost moves the directions. For the boost in the plane $(x1, x4)$, $J = \gamma^{(x1)}\gamma^{(x4)}$ with $JJ = +1$, $c = \cosh\frac{\theta}{2}$ and $s = \sinh\frac{\theta}{2}$. The same steps give $R\gamma^{(x1)}R^{-1} = \gamma^{(x1)}(c1 - sJ)^2$ with $(c1 - sJ)^2 = (c^2 + s^2)1 - 2cs\,J = \cosh\theta\,1 - \sinh\theta\,J$; here $\cosh^2 x + \sinh^2 x = \cosh 2x$ and $2\sinh x\cosh x = \sinh 2x$ follow from the definitions by multiplying out, as above. Hence

$$R\gamma^{(x1)}R^{-1} = \cosh\theta\,\gamma^{(x1)} - \sinh\theta\,\gamma^{(x4)}, \qquad R\gamma^{(x4)}R^{-1} = \cosh\theta\,\gamma^{(x4)} - \sinh\theta\,\gamma^{(x1)}.$$

The pair of directions $(x1, x4)$ is mixed by a hyperbolic turn, which keeps $v_1^2 - v_4^2$ because $\cosh^2\theta - \sinh^2\theta = 1$, and never comes back.

Reading off the vector matrix with traces. For an even R write $R\gamma^cR^{-1} = \sum_d \Lambda_{dc}\gamma^d$ (Section 5.12). Since $\mathrm{tr}(\gamma^d\gamma^e) = 16\,\eta^{de}$ (for $d \neq e$ by Rule 5 with $k = 2$; for $d = e$ it is the trace of $\eta_{dd}1$), multiplying by γ^d and taking the trace leaves one term:

$$\Lambda_{dc} = \eta_{dd}\,\mathrm{tr}\big(\gamma^dR\gamma^cR^{-1}\big)/16.$$

Notebook 05d computes vector matrices this way.

The double cover. Since $\Lambda(-g) = \Lambda(g)$ (the two signs cancel in $g\gamma^c g^{-1}$), every vector matrix belongs to at least two spinor matrices, g and $-g$. There are no others. Proof: if $\Lambda(g) = \Lambda(h)$, then $k = gh^{-1}$ has $\Lambda(k) = \Lambda(g)\Lambda(h)^{-1} = 1$, that is $\alpha(k)\gamma^c k^{-1} = \gamma^c$ for every c. If k is even, $k\gamma^c = \gamma^c k$: k commutes with every gamma, so $k = \lambda 1$ by Theorem P of Section 5.13; and by the spinor norm below, $k^T C k = \pm C$, so $\lambda^2 = \pm 1$, and since k is a real matrix, $\lambda = \pm 1$. If k were odd, $k\gamma^c = -\gamma^c k$ for every c; then Γk commutes with every gamma (moving γ^c through Γk costs two signs), so $\Gamma k = \lambda 1$ and $k = \lambda\Gamma$, an even matrix, which is impossible for an odd element (Section 5.12). So $h = \pm g$. Status: PROVED. That every matrix of O(4,4) is the vector matrix of some element of Pin(4,4) is the theorem of Cartan and Dieudonné (every such matrix is a product of reflections), which this book quotes without proof (ASSUMED). With it, Pin(4,4) is a **double cover** of O(4,4) and Spin(4,4) of SO(4,4).

The forms that are kept. Two bilinears matter: $\Psi^\dagger C\Psi$ (the scalar) and $\Psi^\dagger B\Psi$ (the charge density).

First, for every generator, $(S^{ab})^T C + CS^{ab} = 0$. Proof for $a \neq b$: by Rule 4, $(S^{ab})^T = \frac{1}{2}\eta_{aa}\eta_{bb}\gamma^b\gamma^a = -\frac{1}{2}\eta_{aa}\eta_{bb}\gamma^a\gamma^b$, and by (C1) used twice, $\gamma^a\gamma^b C = \eta_{aa}\eta_{bb}C\gamma^a\gamma^b$. So $(S^{ab})^T C = -\frac{1}{2}(\eta_{aa}\eta_{bb})^2 C\gamma^a\gamma^b = -CS^{ab}$. For $R = \exp(\theta S^{ab})$ the transpose is $\exp(\theta(S^{ab})^T)$ (transpose every power), and $(S^T)^k C = C(-S)^k$ (move C to the left one factor at a time), so

$$R^T C = C\exp(-\theta S^{ab}) = CR^{-1}, \qquad R^T C R = C.$$

Products of exponentials keep it too: if $R_1^T C R_1 = C$ and $R_2^T C R_2 = C$, then $(R_1R_2)^T C(R_1R_2) = R_2^T(R_1^T C R_1)R_2 = C$. Since R is real, $(R\Psi)^\dagger C(R\Psi) = \Psi^\dagger R^T C R\Psi = \Psi^\dagger C\Psi$: **the scalar S is invariant** under every product of exponentials.

Second, $B = -iC\gamma^{(x4)}$. For a generator that does not involve $x4$ ($a, b \neq x4$), $(S^{ab})^\dagger B + BS^{ab} = 0$; for the seven generators $S^{(x4)b}$, $(S^{(x4)b})^\dagger B + BS^{(x4)b} = iC\gamma^b \neq 0$.

Proof of the first statement, line by line. Let $a \neq b$, both different from $x4$, and $S = S^{ab} = \frac{1}{2}\gamma^a\gamma^b$. S is real, so $S^\dagger = S^T$. First, by Rule 4 with $k = 2$,

$$S^T = \tfrac{1}{2}(-1)^1\eta_{aa}\eta_{bb}\,\gamma^a\gamma^b = -\tfrac{1}{2}\eta_{aa}\eta_{bb}\,\gamma^a\gamma^b.$$

Second, multiply by $B = -iC\gamma^{(x4)}$ from the right; the two minus signs give a plus:

$$S^T B = \tfrac{i}{2}\,\eta_{aa}\eta_{bb}\,\gamma^a\gamma^b C\gamma^{(x4)}.$$

Third, move C to the front with (C1) in the form $\gamma^c C = -\eta_{cc}C\gamma^c$, first past γ^b, then past γ^a:

$$\gamma^a\gamma^b C = \gamma^a(-\eta_{bb}C\gamma^b) = -\eta_{bb}(\gamma^a C)\gamma^b = -\eta_{bb}(-\eta_{aa}C\gamma^a)\gamma^b = \eta_{aa}\eta_{bb}\,C\gamma^a\gamma^b,$$

so that, with $(\eta_{aa}\eta_{bb})^2 = 1$,

$$S^T B = \tfrac{i}{2}\,(\eta_{aa}\eta_{bb})^2\,C\gamma^a\gamma^b\gamma^{(x4)} = \tfrac{i}{2}\,C\gamma^a\gamma^b\gamma^{(x4)}.$$

Fourth, $\gamma^{(x4)}$ commutes with the pair $\gamma^a\gamma^b$: by Rule 1 it passes two factors that are both different from $x4$, which costs $(-1)^2 = +1$. So $\gamma^a\gamma^b\gamma^{(x4)} = \gamma^{(x4)}\gamma^a\gamma^b$ and

$$S^T B = \tfrac{i}{2}\, C\gamma^{(x4)}\gamma^a\gamma^b.$$

Fifth, directly from the definitions,

$$BS = -iC\gamma^{(x4)}\,\tfrac{1}{2}\gamma^a\gamma^b = -\tfrac{i}{2}\, C\gamma^{(x4)}\gamma^a\gamma^b.$$

The sum of the last two lines is zero: $S^\dagger B + BS = 0$.

Proof of the second statement: with $S = \frac{1}{2}\gamma^{(x4)}\gamma^b$ (S is real, so $S^\dagger = S^T$),

$$BS = -\tfrac{i}{2}C\gamma^{(x4)}\gamma^{(x4)}\gamma^b = \tfrac{i}{2}C\gamma^b, \qquad S^T B = -\tfrac{i}{2}\eta_{bb}\gamma^{(x4)}\gamma^b C\gamma^{(x4)} = \tfrac{i}{2}C\gamma^{(x4)}\gamma^b\gamma^{(x4)} = \tfrac{i}{2}C\gamma^b,$$

where the first uses $\gamma^{(x4)}\gamma^{(x4)} = -1$; the second uses Rule 4 for S^T, then (C1) to move C to the front (sign $-\eta_{bb}$ for γ^b, $+1$ for $\gamma^{(x4)}$), then $\gamma^b\gamma^{(x4)} = -\gamma^{(x4)}\gamma^b$ and $\gamma^{(x4)}\gamma^{(x4)} = -1$. The sum is $iC\gamma^b$. So the charge density $\Psi^\dagger B\Psi$ is kept only by the transformations that leave the time $x4$ alone. This is what one expects: the charge density is the time component of the current J^a, and a boost that mixes the time $x4$ with another direction mixes it with the other components, as the charge density of special relativity does.

The spinor norm. For $g = \gamma(u_1)\cdots\gamma(u_k)$ in Pin(4,4) put $N(g) = \eta(u_1,u_1)\cdots\eta(u_k,u_k) = \pm 1$. Then

$$g^T C g = (-1)^k\, N(g)\, C.$$

Proof: from (C5), $(\gamma^a)^T = -C\gamma^a C$, so $(\gamma^a)^T C = -C\gamma^a$ and, by linearity, $\gamma(u)^T C = -C\gamma(u)$. Hence $\gamma(u)^T C\gamma(u) = -C\gamma(u)\gamma(u) = -\eta(u,u)\, C$. In $g^T C g = \gamma(u_k)^T\cdots\gamma(u_1)^T\, C\,\gamma(u_1)\cdots\gamma(u_k)$ apply this to the innermost pair, then to the next, and so on: each factor contributes $-\eta(u_j,u_j)$. Consequence: a product of exponentials has $g^T C g = +C$, but $g = \gamma^{(x1)}\gamma^{(x4)}$, an element of Spin(4,4) with $k = 2$ and $N = (+1)(-1) = -1$, has $g^T C g = -C$. **So Spin(4,4) contains elements that are not products of exponentials.** Section 5.23 finds all of them.

Determinant 1. Every gamma has determinant $+1$ as a 16×16 matrix (Section 5.3, Rule 6), and so has every product of gammas, every $\gamma(u)$ of a unit vector (its square is ± 1 and its trace 0) and every element of Pin(4,4). So the words determinant 1 in the author's statement about Spin(4,4) refer to the vector matrix Λ in O(4,4), never to the spinor matrix itself.

statement	**status**	**where it is verified**
the so(4,4) rules for all 784 pairs of generators	PROVED	`python-algebra.json`, check `S_lorentz_algebra`; `wolfram-algebra.json`, check `S_Lorentz_algebra`

statement	status	where it is verified
$(S^{ab})^T C + C S^{ab} = 0$ and $[\Gamma, S^{ab}] = 0$	PROVED	`python-algebra.json`, check `S_preserves_C_and_commutes_with_Gamma`; `wolfram-algebra.json`, checks `S_preserves_C` and `S_commutes_with_Gamma`
B kept exactly by the 21 generators without $x4$; $iC\gamma^b$ for $S^{(x4)b}$	PROVED	`wolfram-algebra.json`, check `S_preserves_B_only_off_x4`
closed formulas, half angles, $R(2\pi) = -1$, the spinor norm, the double cover (kernel ± 1), determinants $+1$	PROVED above; COMPUTED in Notebook 05d	Notebook 05d (its own computation, to 10^{-10} or better; the determinants exactly)
every matrix of O(4,4) is a vector matrix (Cartan and Dieudonné)	ASSUMED (quoted theorem)	not computed in this book

5.19 Example: Notebook 05d computes spin transformations

Notebook 05d builds the 28 generators, repeats the recorded so(4,4) rules (784 pairs), the vector rule (512 triples) and the invariance of C, sorts the 28 planes into 12 rotations and 16 boosts, compares the power series of the exponential with the closed formulas, computes the vector matrices by the trace formula, shows the half angle and the sign -1 after a turn by 2π, follows a unit vector around a circle and along hyperbolas, measures which transformations keep the forms of C and B, and checks the reflections, the spinor norm, the double cover and the determinants. It draws six figures and ends with ALL 20 CHECKS PASSED (notebook 05d).

5.20 How to run Notebook 05d

Step 1. What this notebook does and what it needs.

Notebook 05d (Spin(4,4) transformations: rotations, boosts, reflections and the double cover) is the file `Revision/textbook/notebooks/05d_spin_transformations.ipynb` of the repository Dirac_claude. It builds the 28 generators S^ab = (1/4)(gamma^a gamma^b - gamma^b gamma^a) from the Revision gammas, checks the recorded commutation relations (the Lie algebra so(4,4), the action on the gammas, the invariance of the charge matrix C and of the chirality), sorts the 28 planes into 12 rotations and 16 boosts, computes the finite transformations exp(theta S^ab) by their power series and by the closed formulas with half angles, shows that a rotation by 2 pi multiplies every spinor by -1 while it returns every vector to itself (the double cover), computes the 8 by 8 matrices by which spinor transformations move vectors, checks which transformations keep the forms of C and of B, and checks reflections, the spinor norm and the determinants. Six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05d_spin_transformations.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05d_spin_transformations.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 05d_spin_transformations.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/05d.captions.json`
- `Revision/textbook/figures/05d_1_plane_types.png`
- `Revision/textbook/figures/05d_2_half_angles.png`
- `Revision/textbook/figures/05d_3_rotation_matrices.png`
- `Revision/textbook/figures/05d_4_orbits.png`
- `Revision/textbook/figures/05d_5_invariant_forms.png`
- `Revision/textbook/figures/05d_6_vector_matrices.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 05d exist
ALL 20 CHECKS PASSED (notebook 05d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/05d_spin_transformations.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.

- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 05d_spin_transformations.ipynb
python -m nbconvert --execute --inplace 05d_spin_transformations.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming `Revision/algebra/gammas.json`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

5.21 Notebook 05d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/05d_spin_transformations.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A **spinor transformation** is a $16 \times 16$ matrix $R$ that acts on the 16
components of the spinor field, $\Psi \to R\Psi$, and at the same time turns the
eight coordinate directions into each other. The transformations that can be reached
continuously from the identity are built from the 28 **generators**
```

```
$S^{ab} = \tfrac14[\gamma^a, \gamma^b]$ as exponentials $R = \exp(\theta S^{ab})$
and products of such exponentials. This notebook

1. builds the 28 generators from the Revision gammas and repeats the recorded
   checks: they obey the commutation rules of the Lie algebra so(4,4), they turn
   the gammas into each other like the components of a vector, they keep the
   bilinear $\bar\Psi\Psi = \Psi^\dagger C\Psi$ and they commute with the chirality
   $\Gamma$;
2. sorts the 28 coordinate planes $(a, b)$ into 12 **rotations** and 16 **boosts**;
3. computes $\exp(\theta S^{ab})$ twice, by its power series and by the closed
   formulas $\cos\tfrac\theta2 + \gamma^a\gamma^b\sin\tfrac\theta2$ (rotation) and
   $\cosh\tfrac\theta2 + \gamma^a\gamma^b\sinh\tfrac\theta2$ (boost);
4. computes how $R$ moves the eight directions (an $8 \times 8$ matrix $\Lambda$),
   and shows that a rotation by $2\pi$ brings every vector back but multiplies every
   spinor by $-1$: the spinor group is a **double cover**;
5. checks which transformations keep the form of $C$ (all of them) and of $B =
   -iC\gamma^{(x4)}$ (only those that do not involve the time $x4$);
6. checks reflections, the *spinor norm* and the determinants.

Every check that repeats a recorded check prints the record file and the check
name. Six teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 05d (Spin(4,4) transformations: rotations, boosts, reflections and the double
cover) is the file ‘Revision/textbook/notebooks/05d_spin_transformations.ipynb‘ of the
repository Dirac_claude. It builds the 28 generators S^ab = (1/4)(gamma^a gamma^b -
gamma^b gamma^a) from the Revision gammas, checks the recorded commutation relations (the
Lie algebra so(4,4), the action on the gammas, the invariance of the charge matrix C and
of the chirality), sorts the 28 planes into 12 rotations and 16 boosts, computes the
finite transformations exp(theta S^ab) by their power series and by the closed formulas
with half angles, shows that a rotation by 2 pi multiplies every spinor by -1 while it
returns every vector to itself (the double cover), computes the 8 by 8 matrices by which
spinor transformations move vectors, checks which transformations keep the forms of C and
of B, and checks reflections, the spinor norm and the determinants. Six teaching plots.
It needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.
```

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another

notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 05d_spin_transformations.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 05d_spin_transformations.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 05d_spin_transformations.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/05d.captions.json‘
- ‘Revision/textbook/figures/05d_1_plane_types.png‘
- ‘Revision/textbook/figures/05d_2_half_angles.png‘
- ‘Revision/textbook/figures/05d_3_rotation_matrices.png‘
- ‘Revision/textbook/figures/05d_4_orbits.png‘
- ‘Revision/textbook/figures/05d_5_invariant_forms.png‘
- ‘Revision/textbook/figures/05d_6_vector_matrices.png‘

It changes no other file of the repository; running it headless or saving it in

JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 05d exist
ALL 20 CHECKS PASSED (notebook 05d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/05d_spin_transformations.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 05d_spin_transformations.ipynb
  python -m nbconvert --execute --inplace 05d_spin_transformations.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the

  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming ‘Revision/algebra/gammas.json‘: the notebook was opened
  outside the repository, or the repository is incomplete; clone the repository again and
  open the notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 05d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 05d (Spin(4,4) transformations: rotations, boosts, reflections and the double
# cover) is the file Revision/textbook/notebooks/05d_spin_transformations.ipynb of the
# repository Dirac_claude. It builds the 28 generators S^ab = (1/4)(gamma^a gamma^b -
# gamma^b gamma^a) from the Revision gammas, checks the recorded commutation relations
# (the Lie algebra so(4,4), the action on the gammas, the invariance of the charge matrix
# C and of the chirality), sorts the 28 planes into 12 rotations and 16 boosts, computes
# the finite transformations exp(theta S^ab) by their power series and by the closed
# formulas with half angles, shows that a rotation by 2 pi multiplies every spinor by -1
# while it returns every vector to itself (the double cover), computes the 8 by 8
# matrices by which spinor transformations move vectors, checks which transformations
# keep the forms of C and of B, and checks reflections, the spinor norm and the
# determinants. Six teaching plots. It needs a computer with Windows 11, macOS or Linux,
# an internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
```

```
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 05d_spin_transformations.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
```

```
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 05d_spin_transformations.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 05d_spin_transformations.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/05d.captions.json
# - Revision/textbook/figures/05d_1_plane_types.png
# - Revision/textbook/figures/05d_2_half_angles.png
# - Revision/textbook/figures/05d_3_rotation_matrices.png
# - Revision/textbook/figures/05d_4_orbits.png
# - Revision/textbook/figures/05d_5_invariant_forms.png
# - Revision/textbook/figures/05d_6_vector_matrices.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the six figure files of notebook 05d exist
#     ALL 20 CHECKS PASSED (notebook 05d)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
```

```
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/05d_spin_transformations.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 05d_spin_transformations.ipynb
#     python -m nbconvert --execute --inplace 05d_spin_transformations.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming Revision/algebra/gammas.json: the notebook was opened
#   outside the repository, or the repository is incomplete; clone the repository again
#   and open the notebook from its folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

```
NOTEBOOK_ID = "05d"  # this notebook: chapter 05, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
```

```
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 05d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Commutator** $[M, N] = MN - NM$.
- **Generators** $S^{ab} = \tfrac14[\gamma^a, \gamma^b]$; for $a \neq b$ the two
  gammas anticommute and $S^{ab} = \tfrac12\gamma^a\gamma^b$. There are 28 of them
  ($a$ before $b$ in the order $x0, \dots, x7$).
- **Lie algebra so(4,4)**: the commutation rules $[S^{ab}, S^{cd}] = \eta^{bc}S^{ad}
  - \eta^{ac}S^{bd} - \eta^{bd}S^{ac} + \eta^{ad}S^{bc}$ of the 28 generators.
- **Exponential of a matrix**: $\exp(M) = 1 + M + \tfrac12 M^2 + \tfrac16 M^3 +
  \dots = \sum_{k \ge 0} M^k/k!$, the same power series as for $e^x$.
- **Hyperbolic functions**: $\cosh x = \tfrac12(e^x + e^{-x})$ and $\sinh x =
  \tfrac12(e^x - e^{-x})$; $\cosh^2 x - \sinh^2 x = 1$.
- **Rotation**: a transformation in a plane of two directions of the same kind
  (both space-like or both time-like); it turns them by an angle $\theta$ and comes
  back after $2\pi$.
- **Boost**: a transformation in a plane of one space-like and one time-like
  direction; it mixes them with $\cosh\theta$ and $\sinh\theta$ and never comes
  back ($\theta$ is the *rapidity*).
- **Vector**, $\gamma(v)$: eight numbers $v_a$; the matrix $\gamma(v) =
  \sum_a v_a\gamma^a$. The **metric product** is $\eta(u, v) = \sum_a \eta_{aa}u_a
  v_a$; $u$ is a **unit vector** when $\eta(u, u) = +1$ or $-1$.
- **O(4,4), SO(4,4)**: the real $8 \times 8$ matrices $\Lambda$ with $\Lambda^T\eta
  \Lambda = \eta$ (they keep the metric product); SO(4,4): those with determinant
  $+1$.
- **Reflection** $R_u$ along a unit vector $u$: $R_u v = v - 2\,\eta(u, v)\,u/
  \eta(u, u)$; it reverses $u$ and keeps every direction orthogonal to $u$.
- **Pin(4,4)**: all products $\gamma(u_1)\cdots\gamma(u_k)$ of unit vectors;
  **Spin(4,4)**: those with an even number $k$ of factors.
- **Spinor norm** $N(g) = \eta(u_1, u_1)\cdots\eta(u_k, u_k)$ of such a product.
- **Double cover**: two spinor matrices, $R$ and $-R$, move the vectors in the
  same way.
```

Text cell:

```
## 4. The physical and mathematical situation

**The exponential in closed form.** Fix a plane $(a, b)$ with $a \neq b$ and put
$J = \gamma^a\gamma^b = 2S^{ab}$. Then $\theta S^{ab} = \tfrac\theta2 J$ and

$$JJ = \gamma^a\gamma^b\gamma^a\gamma^b = -\gamma^a\gamma^a\gamma^b\gamma^b =
  -\eta_{aa}\eta_{bb}\,1 ,$$

using $\gamma^b\gamma^a = -\gamma^a\gamma^b$ and $\gamma^a\gamma^a = \eta_{aa}1$.

- If $\eta_{aa}\eta_{bb} = +1$ (both space-like or both time-like): $JJ = -1$, so
```

```
  $J^{2n} = (-1)^n$ and $J^{2n+1} = (-1)^n J$. Splitting the series of $\exp$ into
  even and odd powers,
  $\exp(\tfrac\theta2 J) = \sum_n \frac{(-1)^n(\theta/2)^{2n}}{(2n)!} + J\sum_n
  \frac{(-1)^n(\theta/2)^{2n+1}}{(2n+1)!} = \cos\tfrac\theta2 + J\sin\tfrac\theta2$
  (the power series of cosine and sine): a **rotation**.
- If $\eta_{aa}\eta_{bb} = -1$: $JJ = +1$, all signs are $+$, and $\exp(\tfrac\theta2
  J) = \cosh\tfrac\theta2 + J\sinh\tfrac\theta2$: a **boost**.

**How the directions move.** From the Clifford relation, $[S^{ab}, \gamma^c] =
\eta^{bc}\gamma^a - \eta^{ac}\gamma^b$. For the plane $(x1, x2)$ this gives
$[S, \gamma^{(x1)}] = -\gamma^{(x2)}$ and $[S, \gamma^{(x2)}] = \gamma^{(x1)}$.
The functions $f(\theta) = R\gamma^{(x1)}R^{-1}$ and $g(\theta) =
R\gamma^{(x2)}R^{-1}$ with $R = \exp(\theta S)$ then obey $f' = -g$ and $g' = f$
(the derivative of $R$ is $RS$, of $R^{-1}$ it is $-SR^{-1}$), with the solution
$f = \cos\theta\,\gamma^{(x1)} - \sin\theta\,\gamma^{(x2)}$ and
$g = \sin\theta\,\gamma^{(x1)} + \cos\theta\,\gamma^{(x2)}$: the directions turn by
the **full** angle $\theta$, while $R$ contains only the **half** angle. At
$\theta = 2\pi$ the directions are back, but $R = \cos\pi + J\sin\pi = -1$.

**Reading off the vector matrix.** Write $R\gamma^cR^{-1} = \sum_d \Lambda_{dc}
\gamma^d$. Because $\mathrm{tr}(\gamma^d\gamma^e) = 16\,\eta^{de}$ (for $d \neq e$
the trace of an anticommuting product is zero, since $\mathrm{tr}(MN) =
\mathrm{tr}(NM)$ gives $\mathrm{tr}(\gamma^d\gamma^e) =
-\mathrm{tr}(\gamma^d\gamma^e)$; for $d = e$ it is the trace of $\eta_{dd}1$),
taking the trace with $\gamma^d$ gives $\Lambda_{dc} = \eta_{dd}\,
\mathrm{tr}(\gamma^d R\gamma^c R^{-1})/16$.

**Reflections.** For a unit vector $u$, from $\gamma(u)\gamma(v) + \gamma(v)
\gamma(u) = 2\eta(u, v)1$ and $\gamma(u)^{-1} = \gamma(u)/\eta(u, u)$:
$\gamma(u)\gamma(v)\gamma(u)^{-1} = 2\eta(u, v)\gamma(u)/\eta(u, u) - \gamma(v) =
-\gamma(R_u v)$. So $-\gamma(u)\gamma(v)\gamma(u)^{-1} = \gamma(R_uv)$:
conjugation by a unit vector is a reflection, up to the sign.

**The spinor norm.** For $g = \gamma(u_1)\cdots\gamma(u_k)$: $g^TCg = (-1)^k
N(g)\,C$. (Each factor gives $\gamma(u)^TC\gamma(u) = -\eta(u,u)C$, from
$\gamma(u)^TC = -C\gamma(u)$, which follows from $C\gamma^aC^{-1} =
-(\gamma^a)^T$.) A product of exponentials keeps $C$: if $R_1^TCR_1 = C$ and
$R_2^TCR_2 = C$, then $(R_1R_2)^TC(R_1R_2) = R_2^TR_1^TCR_1R_2 = C$.
```

Text cell:

```
## 5. The gammas and the recorded checks

The next cell reads the gammas (exact whole numbers) from the Revision record and
the two algebra reports, and defines: the helpers `recorded(key, name)` (true when
the report `key`, "python" or "wolfram", holds the check `name` with the verdict
pass) and `record_of(key, name)` (the text printed after "reproduces"); the helper
`check_reproduces(condition, name, record)`, which is `check` for a check that
reproduces a Revision record, printed in one piece (it lets `check` print the PASS
line and the line "reproduces ..." into a text buffer with
`contextlib.redirect_stdout` and sends both with one `sys.stdout.write`, because
Jupyter delivers printed text in pieces); the $8 \times 8$ metric `ETA_MATRIX`
$= \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$, $C$, $\Gamma$ and $B$. Then it checks
```

the Clifford relation.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
ETA_MATRIX = np.diag([float(ETA[x]) for x in COORDS])  # the 8 x 8 metric eta
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)
C = gamma["x0"] @ gamma["x1"] @ gamma["x2"] @ gamma["x3"]  # the charge matrix
Gamma = I16
for x in ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]:
    Gamma = Gamma @ gamma[x]  # the chirality, the product of all eight gammas
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4)

REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    """True when the report key records the check name with the verdict pass."""
    return VERDICTS[(key, name)][0] == "pass"


def record_of(key, name):
    """The text printed after "reproduces": the record file and the check name."""
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


def eta(a, b):
    """eta^ab of the frame metric: eta_aa when a = b, 0 otherwise."""
    return ETA[a] if a == b else 0
```

```
check_reproduces(all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                                    2 * eta(a, b) * I16) for a in COORDS for b in COORDS)
                 and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

Out [2]:

```
PASS {gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
```

Text cell:

```
## 6. The 28 generators and their commutation rules

The next cell builds $S^{ab}$ for all 64 ordered pairs (with $S^{aa} = 0$), as
arrays of floating-point numbers. Their entries are $0$ and $\pm\tfrac12$, and every
product and sum below involves only halves and quarters, which the computer stores
**exactly** (they are sums of powers of 2), so `np.array_equal` compares exactly. It
checks three recorded facts:

1. $S^{ab} = -S^{ba}$ and $S^{ab} = \tfrac12\gamma^a\gamma^b$ for $a \neq b$;
2. the so(4,4) rules $[S^{ab}, S^{cd}] = \eta^{bc}S^{ad} - \eta^{ac}S^{bd} -
   \eta^{bd}S^{ac} + \eta^{ad}S^{bc}$ for all $28 \times 28 = 784$ pairs of
   generators;
3. $[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a - \eta^{ac}\gamma^b$ for all
   $8 \times 8 \times 8 = 512$ triples: the gammas turn like the components of a
   vector.
```

In [3]:

```
S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4.0
     for a in COORDS for b in COORDS}  # all 64 ordered pairs
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 pairs


def commutator(m, n):
    """[m, n] = m n - n m."""
    return m @ n - n @ m


definition_ok = all(np.array_equal(S[(a, b)], -S[(b, a)]) for a in COORDS
                    for b in COORDS) and all(
    np.array_equal(S[(a, b)], gamma[a] @ gamma[b] / 2.0) for a, b in pairs)
check_reproduces(len(pairs) == 28 and definition_ok
                 and recorded("python", "S_definition"),
                 "S^ab = -S^ba and S^ab = (1/2) gamma^a gamma^b: 28 independent "
                 "generators",
                 record=record_of("python", "S_definition"))
```

```
algebra_failures = 0
for a, b in pairs:
    for c, d in pairs:
        right = (eta(b, c) * S[(a, d)] - eta(a, c) * S[(b, d)]
                 - eta(b, d) * S[(a, c)] + eta(a, d) * S[(b, c)])
        if not np.array_equal(commutator(S[(a, b)], S[(c, d)]), right):
            algebra_failures += 1
say(f"so(4,4) rules tested for {len(pairs) ** 2} pairs; failures {algebra_failures}")
check_reproduces(algebra_failures == 0 and recorded("python", "S_lorentz_algebra")
                 and recorded("wolfram", "S_Lorentz_algebra"),
                 "[S^ab, S^cd] = eta^bc S^ad - eta^ac S^bd - eta^bd S^ac + eta^ad S^bc",
                 record=record_of("python", "S_lorentz_algebra"))
vector_failures = sum(
    not np.array_equal(commutator(S[(a, b)], gamma[c]),
                       eta(b, c) * gamma[a] - eta(a, c) * gamma[b])
    for a in COORDS for b in COORDS for c in COORDS)
say(f"vector rule tested for 512 triples; failures {vector_failures}")
check_reproduces(vector_failures == 0 and recorded("python", "S_vector_action")
                 and recorded("wolfram", "S_gamma_commutator"),
                 "[S^ab, gamma^c] = eta^bc gamma^a - eta^ac gamma^b for all 512 triples",
                 record=record_of("python", "S_vector_action"))
```

Out [3]:

```
PASS S^ab = -S^ba and S^ab = (1/2) gamma^a gamma^b: 28 independent generators
     reproduces Revision/algebra/reports/python-algebra.json, check S_definition
so(4,4) rules tested for 784 pairs; failures 0
PASS [S^ab, S^cd] = eta^bc S^ad - eta^ac S^bd - eta^bd S^ac + eta^ad S^bc
     reproduces Revision/algebra/reports/python-algebra.json, check S_lorentz_algebra
vector rule tested for 512 triples; failures 0
PASS [S^ab, gamma^c] = eta^bc gamma^a - eta^ac gamma^b for all 512 triples
     reproduces Revision/algebra/reports/python-algebra.json, check S_vector_action
```

Text cell:

```
The next cell checks the recorded invariances of the generators: $(S^{ab})^TC +
CS^{ab} = 0$ (so the bilinear $\Psi^\dagger C\Psi$ does not change to first order
under $\Psi \to (1 + \epsilon S^{ab})\Psi$: its change is $\epsilon\Psi^\dagger
((S^{ab})^TC + CS^{ab})\Psi$) and $[\Gamma, S^{ab}] = 0$ (the two chiral halves are
kept).
```

In [4]:

```
check_reproduces(all(not ((S[k].T @ C + C @ S[k]).any()) for k in S)
                 and all(not commutator(Gamma, S[k]).any() for k in S)
                 and recorded("python", "S_preserves_C_and_commutes_with_Gamma"),
                 "(S^ab)^T C + C S^ab = 0 and [Gamma, S^ab] = 0 for all a, b",
                 record=record_of("python", "S_preserves_C_and_commutes_with_Gamma"))
```

Out [4]:

```
PASS (S^ab)^T C + C S^ab = 0 and [Gamma, S^ab] = 0 for all a, b
     reproduces Revision/algebra/reports/python-algebra.json, check
    S_preserves_C_and_commutes_with_Gamma
```

Text cell:

```
## 7. Rotations and boosts

The next cell computes $JJ$ for $J = \gamma^a\gamma^b$ in each of the 28 planes and
checks $JJ = -\eta_{aa}\eta_{bb}1$; it calls the plane a rotation when
$\eta_{aa}\eta_{bb} = +1$ and a boost when $\eta_{aa}\eta_{bb} = -1$, counts them
and draws the table. Expected: the 6 planes inside $\{x0, x1, x2, x3\}$ and the 6
inside $\{x4, x5, x6, x7\}$ are rotations (12); the $4 \times 4 = 16$ mixed planes
are boosts. Note that a plane of two times, such as $(x4, x5)$, is a rotation.
```

In [5]:

```
kind = {}  # (a, b) -> +1 for a rotation, -1 for a boost
squares_ok = True
for a, b in pairs:
    J = gamma[a] @ gamma[b]
    squares_ok &= np.array_equal(J @ J, -ETA[a] * ETA[b] * I16)
    kind[(a, b)] = ETA[a] * ETA[b]
rotations = [p for p in pairs if kind[p] == 1]
boosts = [p for p in pairs if kind[p] == -1]
say(f"rotations: {len(rotations)}; boosts: {len(boosts)}")
say("rotation planes: " + " ".join(f"({a},{b})" for a, b in rotations))
check(squares_ok and len(rotations) == 12 and len(boosts) == 16,
      "(gamma^a gamma^b)^2 = -eta_aa eta_bb: 12 rotation planes and 16 boost planes")
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
table = np.zeros((8, 8))
for (a, b), value in kind.items():
    i, j = COORDS.index(a), COORDS.index(b)
    table[i, j] = table[j, i] = value  # the same plane in both orders
fig, ax = plt.subplots(figsize=(6.4, 5.6))
ax.imshow(table, cmap=SIGNS, vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        word = "" if i == j else ("rot" if table[i, j] == 1 else "boost")
        ax.text(j, i, word, ha="center", va="center", color="white", fontsize=8,
                fontweight="bold")
for k in range(1, 8):
    ax.axhline(k - 0.5, color="white", linewidth=2)
    ax.axvline(k - 0.5, color="white", linewidth=2)
ax.set_xticks(range(8), COORDS)
ax.set_yticks(range(8), COORDS)
ax.set_title("The 28 planes: rotations (red) and boosts (blue)")
ax.grid(False)
save_figure(fig, "plane_types",
            "The kind of the transformation $\\exp(\\theta S^{ab})$ in each coordinate "
```

```
            "plane $(a, b)$: row $a$ and column $b$ name the two directions; red rot "
            "means a rotation ($\\eta_{aa}\\eta_{bb} = +1$, $(\\gamma^a\\gamma^b)^2 = "
            "-1$), blue boost a boost ($\\eta_{aa}\\eta_{bb} = -1$, "
            "$(\\gamma^a\\gamma^b)^2 = +1$); the diagonal is empty. The 6 planes inside "
            "3-space plus the hidden direction and the 6 planes inside the four times "
            "are rotations; the 16 planes that mix a space-like with a time-like "
            "direction are boosts.")
```

Out [5]:

```
rotations: 12; boosts: 16
rotation planes: (x0,x1) (x0,x2) (x0,x3) (x1,x2) (x1,x3) (x2,x3) (x4,x5) (x4,x6) (x4,x7)
    (x5,x6) (x5,x7) (x6,x7)
PASS (gamma^a gamma^b)^2 = -eta_aa eta_bb: 12 rotation planes and 16 boost planes
```

Out [5]:

```
Figure 05d.1 saved as Revision/textbook/figures/05d_1_plane_types.png
```

Text cell:

```
## 8. The exponential: power series and closed formula

The next cell defines two ways of computing $R = \exp(\theta S^{ab})$: the power
series summed up to 80 terms (each term is the previous one times $M/k$), and the
closed formula of section 4. It compares them for two rotations, $(x1, x2)$ and
$(x4, x5)$, at the angles $0.3$, $1$, $2.5$, $2\pi$ and $4\pi$, and for two boosts,
$(x1, x4)$ and $(x5, x0)$, at the rapidities $-1.5$, $0.3$, $1$ and $2.5$. The
largest difference of any entry must be below $10^{-10}$ (the series is summed in
floating-point numbers).
```

In [6]:

```
def exp_series(M, terms=80):
    """exp(M) = 1 + M + M^2/2 + ... summed up to M^(terms-1)/(terms-1)!."""
    result = np.eye(M.shape[0])
    term = np.eye(M.shape[0])
    for k in range(1, terms):
        term = term @ M / k  # M^k / k! from M^(k-1) / (k-1)!
        result = result + term
    return result


def spin_transformation(a, b, theta):
    """exp(theta S^ab) from the closed formula with half angles."""
    J = (gamma[a] @ gamma[b]).astype(float)  # J = 2 S^ab
    if ETA[a] * ETA[b] == 1:  # J J = -1: a rotation
        return np.cos(theta / 2) * np.eye(16) + np.sin(theta / 2) * J
    return np.cosh(theta / 2) * np.eye(16) + np.sinh(theta / 2) * J  # a boost
```

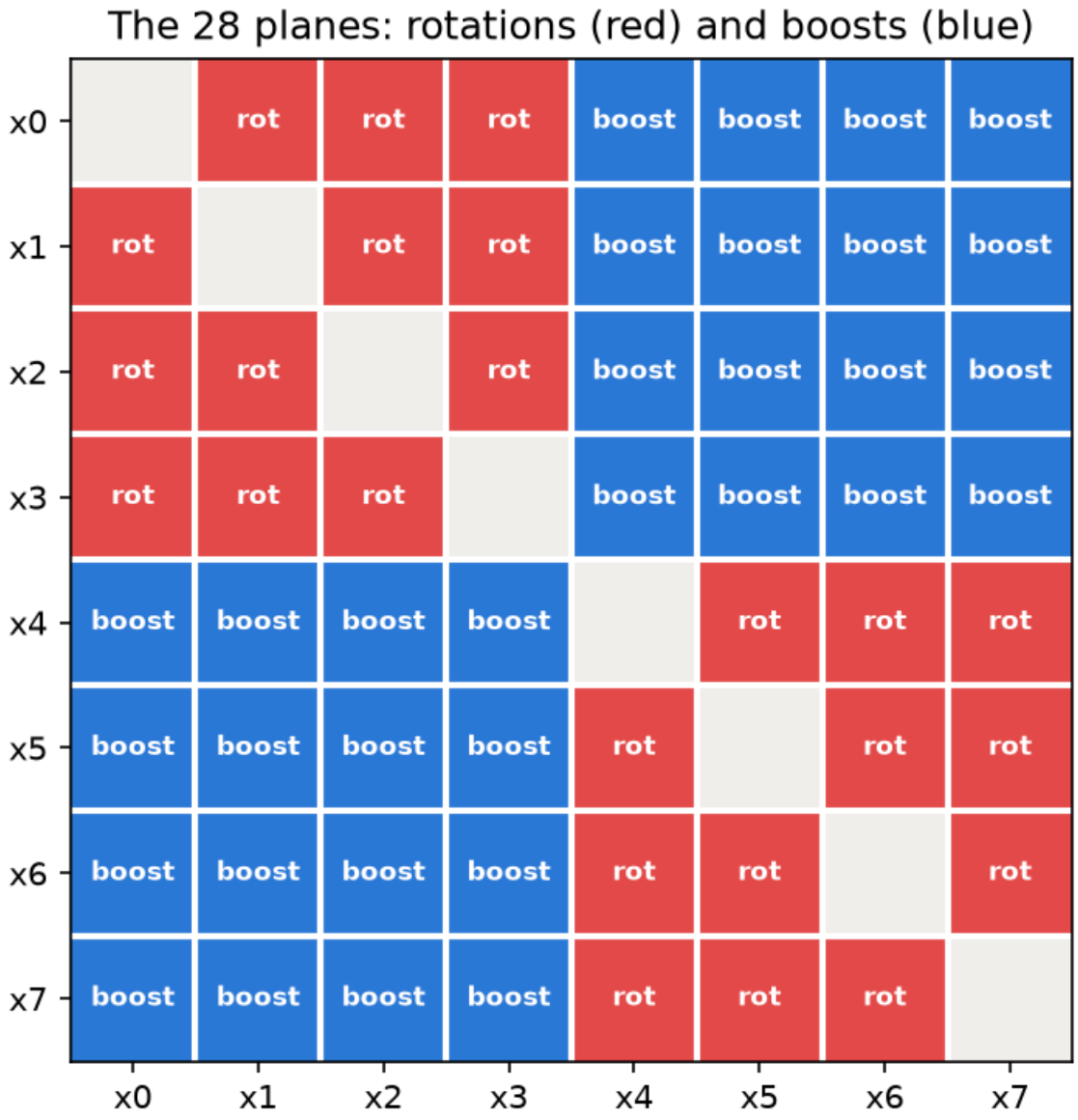

Figure 153: The kind of the transformation $\exp(\theta S^{ab})$ in each coordinate plane (a, b): row a and column b name the two directions; red rot means a rotation ($\eta_{aa}\eta_{bb} = +1$, $(\gamma^a\gamma^b)^2 = -1$), blue boost a boost ($\eta_{aa}\eta_{bb} = -1$, $(\gamma^a\gamma^b)^2 = +1$); the diagonal is empty. The 6 planes inside 3-space plus the hidden direction and the 6 planes inside the four times are rotations; the 16 planes that mix a space-like with a time-like direction are boosts. (Notebook 05d, figure 1.)

```
tests = [(("x1", "x2"), [0.3, 1.0, 2.5, 2 * np.pi, 4 * np.pi]),
         (("x4", "x5"), [0.3, 1.0, 2.5, 2 * np.pi, 4 * np.pi]),
         (("x1", "x4"), [-1.5, 0.3, 1.0, 2.5]),
         (("x0", "x5"), [-1.5, 0.3, 1.0, 2.5])]
largest_difference = 0.0
for (a, b), angles in tests:
    for theta in angles:
        difference = np.max(np.abs(exp_series(theta * S[(a, b)])
                                   - spin_transformation(a, b, theta)))
        largest_difference = max(largest_difference, difference)
    name = "rotation" if kind[(a, b)] == 1 else "boost"
    say(f"plane ({a},{b}), a {name}: series and closed formula compared at "
        f"{len(angles)} values")
check(largest_difference < 1e-10,
      "the power series of exp(theta S^ab) equals the closed formula with half "
      "angles (difference below 1e-10)")
```

Out [6]:

```
plane (x1,x2), a rotation: series and closed formula compared at 5 values
plane (x4,x5), a rotation: series and closed formula compared at 5 values
plane (x1,x4), a boost: series and closed formula compared at 4 values
plane (x0,x5), a boost: series and closed formula compared at 4 values
PASS the power series of exp(theta S^ab) equals the closed formula with half angles
    (difference below 1e-10)
```

Text cell:

```
## 9. How the directions move, and the sign after 2 pi

The next cell defines ‘vector_matrix(R)‘, the $8 \times 8$ matrix $\Lambda$ with
$R\gamma^cR^{-1} = \sum_d\Lambda_{dc}\gamma^d$, read off with the trace formula of
section 4, and checks for the rotation in the plane $(x1, x2)$ at nine angles that:
the sum $\sum_d \Lambda_{dc}\gamma^d$ rebuilds $R\gamma^cR^{-1}$; $\Lambda$ is the
rotation by the full angle $\theta$ (entries $\cos\theta$ and $\pm\sin\theta$ in the
$(x1, x2)$ block, the identity elsewhere); $\Lambda^T\eta\Lambda = \eta$ and
$\det\Lambda = 1$ (it lies in SO(4,4)). Then it checks the double cover:
$R(2\pi) = -1$ and $R(4\pi) = +1$, while $\Lambda(2\pi) = \Lambda(4\pi) = 1$.
```

In [7]:

```
def vector_matrix(R):
    """Lambda (8 x 8) with R gamma^c R^-1 = sum over d of Lambda_dc gamma^d."""
    R_inverse = np.linalg.inv(R)
    Lam = np.zeros((8, 8))
    for j, c in enumerate(COORDS):
        moved = R @ gamma[c] @ R_inverse  # where the direction c is moved to
        for i, d in enumerate(COORDS):
            Lam[i, j] = ETA[d] * np.trace(gamma[d] @ moved) / 16.0
```

```
    return Lam


def rebuilds(R, Lam):
    """True when sum_d Lambda_dc gamma^d equals R gamma^c R^-1 for every c."""
    R_inverse = np.linalg.inv(R)
    return all(np.allclose(sum(Lam[i, j] * gamma[d] for i, d in enumerate(COORDS)),
                           R @ gamma[c] @ R_inverse, atol=1e-12)
               for j, c in enumerate(COORDS))


rotation_ok = True
for theta in np.linspace(0.0, 2 * np.pi, 9):
    R = spin_transformation("x1", "x2", theta)
    Lam = vector_matrix(R)
    expected = np.eye(8)  # the rotation by theta in the (x1, x2) block
    expected[1, 1] = expected[2, 2] = np.cos(theta)  # rows and columns x1, x2
    expected[1, 2], expected[2, 1] = np.sin(theta), -np.sin(theta)
    rotation_ok &= (rebuilds(R, Lam) and np.allclose(Lam, expected, atol=1e-12)
                    and np.allclose(Lam.T @ ETA_MATRIX @ Lam, ETA_MATRIX)
                    and abs(np.linalg.det(Lam) - 1.0) < 1e-12)
check(rotation_ok,
      "R = exp(theta S^(x1 x2)) turns the directions x1, x2 by the full angle "
      "theta; Lambda lies in SO(4,4)")
R_2pi, R_4pi = spin_transformation("x1", "x2", 2 * np.pi), spin_transformation(
    "x1", "x2", 4 * np.pi)
# Rounding leaves differences of about 1e-16, which differ between computers, so
# the cell prints only whether they are below 1e-12.
small_2pi = np.max(np.abs(R_2pi + np.eye(16))) < 1e-12
small_4pi = np.max(np.abs(R_4pi - np.eye(16))) < 1e-12
say(f"every entry of R(2 pi) + 1 is below 1e-12: {small_2pi}; "
    f"every entry of R(4 pi) - 1 is below 1e-12: {small_4pi}")
check(np.allclose(R_2pi, -np.eye(16), atol=1e-12)
      and np.allclose(R_4pi, np.eye(16), atol=1e-12)
      and np.allclose(vector_matrix(R_2pi), np.eye(8), atol=1e-12),
      "a rotation by 2 pi gives R = -1 on spinors but Lambda = 1 on vectors; by 4 pi, R "
      "= +1")
```

Out [7]:

```
PASS R = exp(theta S^(x1 x2)) turns the directions x1, x2 by the full angle theta; Lambda
    lies in SO(4,4)
every entry of R(2 pi) + 1 is below 1e-12: True; every entry of R(4 pi) - 1 is below
    1e-12: True
PASS a rotation by 2 pi gives R = -1 on spinors but Lambda = 1 on vectors; by 4 pi, R =
    +1
```

Text cell:

```
The next cell draws the half angles. Left: for the rotation in the plane
$(x1, x2)$, the vector entry $\Lambda_{x1,x1} = \cos\theta$ and the spinor quantity
$\mathrm{tr}R/16 = \cos(\theta/2)$ (the trace of $J$ is zero), for $\theta$ from 0
```

```
to $4\pi$. Right: for the boost in the plane $(x1, x4)$, $\Lambda_{x1,x1} =
\cosh\theta$ and $\mathrm{tr}R/16 = \cosh(\theta/2)$, for $\theta$ from $-3$ to 3.
The values are computed from the matrices, then compared with the formulas.
```

In [8]:

```
angles = np.linspace(0.0, 4 * np.pi, 241)
rapidities = np.linspace(-3.0, 3.0, 241)
vec_rot = [vector_matrix(spin_transformation("x1", "x2", th))[1, 1] for th in angles]
spin_rot = [np.trace(spin_transformation("x1", "x2", th)) / 16 for th in angles]
vec_boost = [vector_matrix(spin_transformation("x1", "x4", th))[1, 1]
             for th in rapidities]
spin_boost = [np.trace(spin_transformation("x1", "x4", th)) / 16 for th in rapidities]
check(np.allclose(vec_rot, np.cos(angles)) and np.allclose(spin_rot, np.cos(angles / 2))
      and np.allclose(vec_boost, np.cosh(rapidities))
      and np.allclose(spin_boost, np.cosh(rapidities / 2)),
      "vectors move with cos(theta), cosh(theta); spinors with the half angle")
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.4))
axes[0].plot(angles, vec_rot, color="#2a78d6", linewidth=2,
             label=r"vector: $\Lambda_{x1,x1} = \cos\theta$")
axes[0].plot(angles, spin_rot, color="#eb6834", linewidth=2, linestyle="--",
             label=r"spinor: $\mathrm{tr}\,R/16 = \cos(\theta/2)$")
axes[0].plot([2 * np.pi, 2 * np.pi], [1.0, -1.0], "o", color="black", markersize=7,
             label=r"at $2\pi$: vector back at $1$, spinor at $-1$")
axes[0].set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
                   ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
axes[0].set_xlabel(r"rotation angle $\theta$ in the plane $(x1, x2)$")
axes[0].set_ylabel("value")
axes[0].set_title("A rotation: the spinor needs $4\\pi$ to come back")
axes[0].legend(loc="upper center", bbox_to_anchor=(0.5, -0.17))
axes[1].plot(rapidities, vec_boost, color="#2a78d6", linewidth=2,
             label=r"vector: $\Lambda_{x1,x1} = \cosh\theta$")
axes[1].plot(rapidities, spin_boost, color="#eb6834", linewidth=2, linestyle="--",
             label=r"spinor: $\mathrm{tr}\,R/16 = \cosh(\theta/2)$")
axes[1].set_xlabel(r"rapidity $\theta$ in the plane $(x1, x4)$")
axes[1].set_title("A boost: never periodic;\nthe spinor grows half as fast")
axes[1].legend(loc="upper center", bbox_to_anchor=(0.5, -0.17))
save_figure(fig, "half_angles",
            "Vectors and spinors under a rotation and a boost. Left: for the rotation "
            "$\\exp(\\theta S^{ab})$ in the plane $(x1, x2)$, the vector entry "
            "$\\cos\\theta$ (solid) and the spinor quantity $\\mathrm{tr}\\,R/16 = "
            "\\cos(\\theta/2)$ (dashed) against the angle $\\theta$ from $0$ to "
            "$4\\pi$; at $2\\pi$ the vector is back at 1 while the spinor matrix is "
            "$-1$ (dots), and only at $4\\pi$ both are back. Right: for the boost in "
            "the plane $(x1, x4)$, $\\cosh\\theta$ and $\\cosh(\\theta/2)$ against the "
            "rapidity from $-3$ to $3$; a boost never returns. Vertical axes: pure "
            "numbers.")
```

Out [8]:

```
PASS vectors move with cos(theta), cosh(theta); spinors with the half angle
```

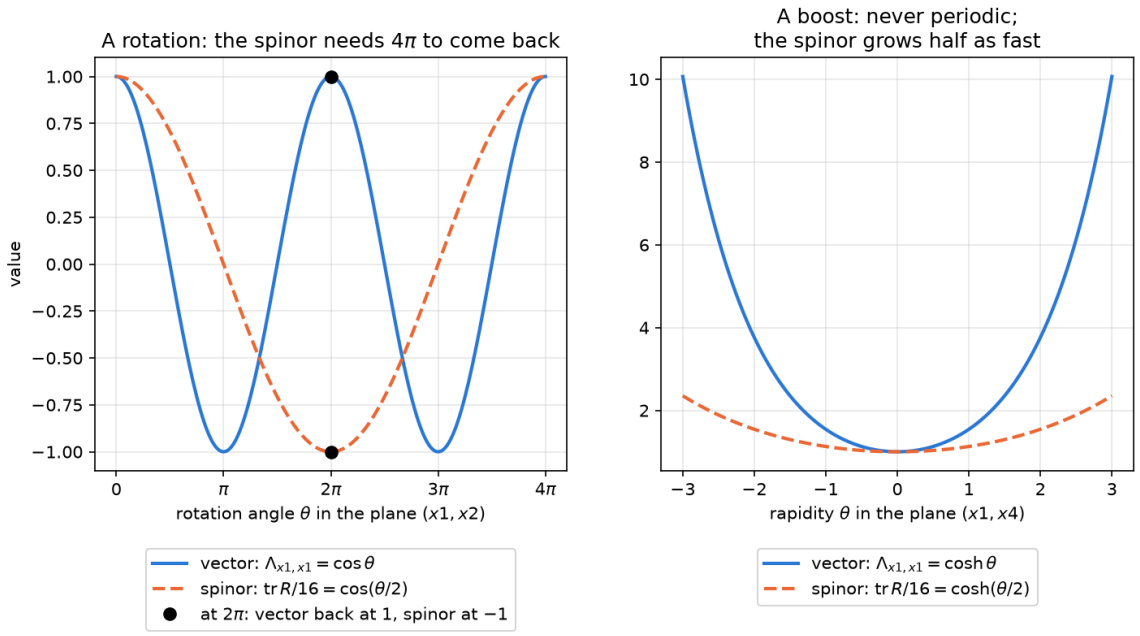

Figure 154: Vectors and spinors under a rotation and a boost. Left: for the rotation $\exp(\theta S^{ab})$ in the plane $(x1, x2)$, the vector entry $\cos\theta$ (solid) and the spinor quantity $\operatorname{tr} R/16 = \cos(\theta/2)$ (dashed) against the angle θ from 0 to 4π; at 2π the vector is back at 1 while the spinor matrix is -1 (dots), and only at 4π both are back. Right: for the boost in the plane $(x1, x4)$, $\cosh\theta$ and $\cosh(\theta/2)$ against the rapidity from -3 to 3; a boost never returns. Vertical axes: pure numbers. (Notebook 05d, figure 2.)

Out [8]:

```
Figure 05d.2 saved as Revision/textbook/figures/05d_2_half_angles.png
```

Text cell:

```
The next cell draws $R(\theta) = \exp(\theta S^{(x1\,x2)})$ as heat maps at
$\theta = 0$ (the identity), $\pi$ (then $R = \gamma^{(x1)}\gamma^{(x2)}$, because
$\cos\tfrac\pi2 = 0$ and $\sin\tfrac\pi2 = 1$), $2\pi$ (then $R = -1$) and $4\pi$
(then $R = +1$).
```

In [9]:

```
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.9))
for k, (ax, th, label) in enumerate(zip(
        axes, [0.0, np.pi, 2 * np.pi, 4 * np.pi], ["0", r"\pi", r"2\pi", r"4\pi"])):
    image = ax.imshow(spin_transformation("x1", "x2", th), cmap=SIGNS, vmin=-1,
                      vmax=1)
    ax.set_title(rf"$R(\theta)$ at $\theta = {label}$")
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("column")
    if k == 0:
        ax.set_ylabel("row")
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
check(np.allclose(spin_transformation("x1", "x2", np.pi),
                  gamma["x1"] @ gamma["x2"], atol=1e-12),
      "R(pi) = gamma^(x1) gamma^(x2) for the rotation in the plane (x1, x2)")
save_figure(fig, "rotation_matrices",
            "The spinor rotation $R(\\theta) = \\exp(\\theta S^{ab})$ in the plane "
            "$(x1, x2)$ at the angles $0$, $\\pi$, $2\\pi$ and $4\\pi$, as heat maps "
            "(column and row 1 to 16; blue $-1$, grey $0$, red $+1$). At $0$ it is "
            "the identity; at $\\pi$ it is the product "
            "$\\gamma^{(x1)}\\gamma^{(x2)}$; at $2\\pi$ it is minus the identity "
            "(every diagonal entry blue), although the directions are back where "
            "they started; only at $4\\pi$ is it the identity again.")
```

Out [9]:

```
PASS R(pi) = gamma^(x1) gamma^(x2) for the rotation in the plane (x1, x2)
```

Out [9]:

```
Figure 05d.3 saved as Revision/textbook/figures/05d_3_rotation_matrices.png
```

Text cell:

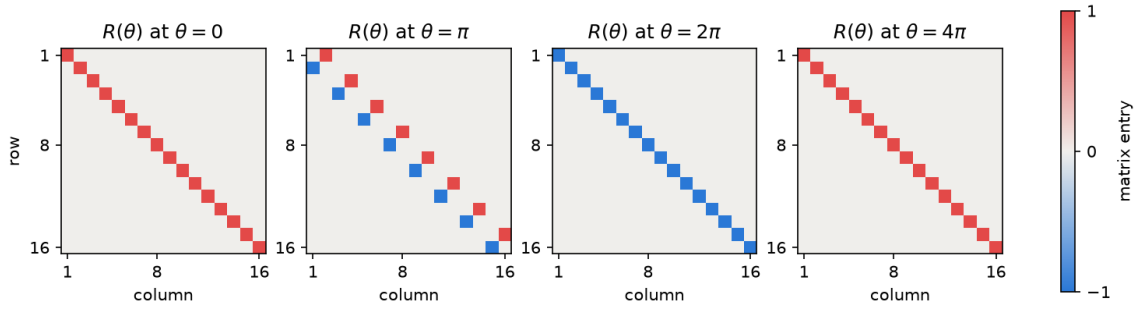

Figure 155: The spinor rotation $R(\theta) = \exp(\theta S^{ab})$ in the plane $(x1, x2)$ at the angles 0, π, 2π and 4π, as heat maps (column and row 1 to 16; blue -1, grey 0, red $+1$). At 0 it is the identity; at π it is the product $\gamma^{(x1)}\gamma^{(x2)}$; at 2π it is minus the identity (every diagonal entry blue), although the directions are back where they started; only at 4π is it the identity again. (Notebook 05d, figure 3.)

```
## 10. Circles and hyperbolas

A rotation keeps $v_1^2 + v_2^2$, a boost keeps $v_1^2 - v_4^2$ (the metric product
in that plane, because $\eta_{x4\,x4} = -1$). The next cell moves the unit vector
$e_{x1}$ with the rotation in $(x1, x2)$ for $\theta$ from 0 to $2\pi$ (it runs
around a circle), and the unit vectors $e_{x1}$ and $e_{x4}$ with the boost in
$(x1, x4)$ for $\theta$ from $-2$ to 2 (they run along the two branches of the
hyperbolas $v_1^2 - v_4^2 = +1$ and $= -1$). The vector matrices are again computed
from the spinor matrices.
```

In [10]:

```
# column k of a vector matrix is the image of e_xk; its entry 1 is v_1, entry 4 v_4
circle = np.array([vector_matrix(spin_transformation("x1", "x2", th))[:, 1]
                   for th in np.linspace(0.0, 2 * np.pi, 121)])  # images of e_x1
hyper_space = np.array([vector_matrix(spin_transformation("x1", "x4", th))[:, 1]
                        for th in np.linspace(-2.0, 2.0, 121)])  # images of e_x1
hyper_time = np.array([vector_matrix(spin_transformation("x1", "x4", th))[:, 4]
                       for th in np.linspace(-2.0, 2.0, 121)])  # images of e_x4
check(np.allclose(circle[:, 1] ** 2 + circle[:, 2] ** 2, 1.0)
      and np.allclose(hyper_space[:, 1] ** 2 - hyper_space[:, 4] ** 2, 1.0)
      and np.allclose(hyper_time[:, 1] ** 2 - hyper_time[:, 4] ** 2, -1.0),
      "rotations keep v1^2 + v2^2, boosts keep v1^2 - v4^2")
fig, axes = plt.subplots(1, 2, figsize=(11.0, 5.0))
axes[0].plot(circle[:, 1], circle[:, 2], color="#2a78d6", linewidth=2)
axes[0].plot(circle[::15, 1], circle[::15, 2], "o", color="#2a78d6", markersize=8)
axes[0].set_aspect("equal")
axes[0].set_xlabel("$v_1$ (component along $x1$)")
axes[0].set_ylabel("$v_2$ (component along $x2$)")
axes[0].set_title("rotation in $(x1, x2)$: a circle")
axes[1].plot(hyper_space[:, 1], hyper_space[:, 4], color="#2a78d6", linewidth=2,
             label=r"image of $e_{x1}$: $v_1^2 - v_4^2 = 1$")
axes[1].plot(hyper_time[:, 1], hyper_time[:, 4], color="#eb6834", linewidth=2,
             label=r"image of $e_{x4}$: $v_1^2 - v_4^2 = -1$")
axes[1].plot([-4, 4], [-4, 4], ":", color="black", linewidth=1,
             label="null lines $v_4 = \\pm v_1$")
```

```
axes[1].plot([-4, 4], [4, -4], ":", color="black", linewidth=1)
axes[1].set_xlim(-4, 4)
axes[1].set_ylim(-4, 4)
axes[1].set_aspect("equal")
axes[1].set_xlabel("$v_1$ (component along $x1$)")
axes[1].set_ylabel("$v_4$ (component along the time $x4$)")
axes[1].set_title("boost in $(x1, x4)$: hyperbolas")
axes[1].legend(loc="upper center", bbox_to_anchor=(0.5, -0.14))
save_figure(fig, "orbits",
            "Where the transformations carry a unit vector. Left: the rotation in "
            "the plane $(x1, x2)$ carries the unit vector along $x1$ around the "
            "circle $v_1^2 + v_2^2 = 1$ as the angle runs from $0$ to $2\\pi$ (dots "
            "every $\\pi/4$). Right: the boost in the plane $(x1, x4)$ carries the "
            "unit vector along $x1$ along the hyperbola $v_1^2 - v_4^2 = 1$ (blue) "
            "and the unit vector along the time $x4$ along $v_1^2 - v_4^2 = -1$ "
            "(orange), for rapidities from $-2$ to $2$; neither crosses the dotted "
            "null lines. Axes: the components of the vector (pure numbers).")
```

Out [10]:

```
PASS rotations keep v1^2 + v2^2, boosts keep v1^2 - v4^2
```

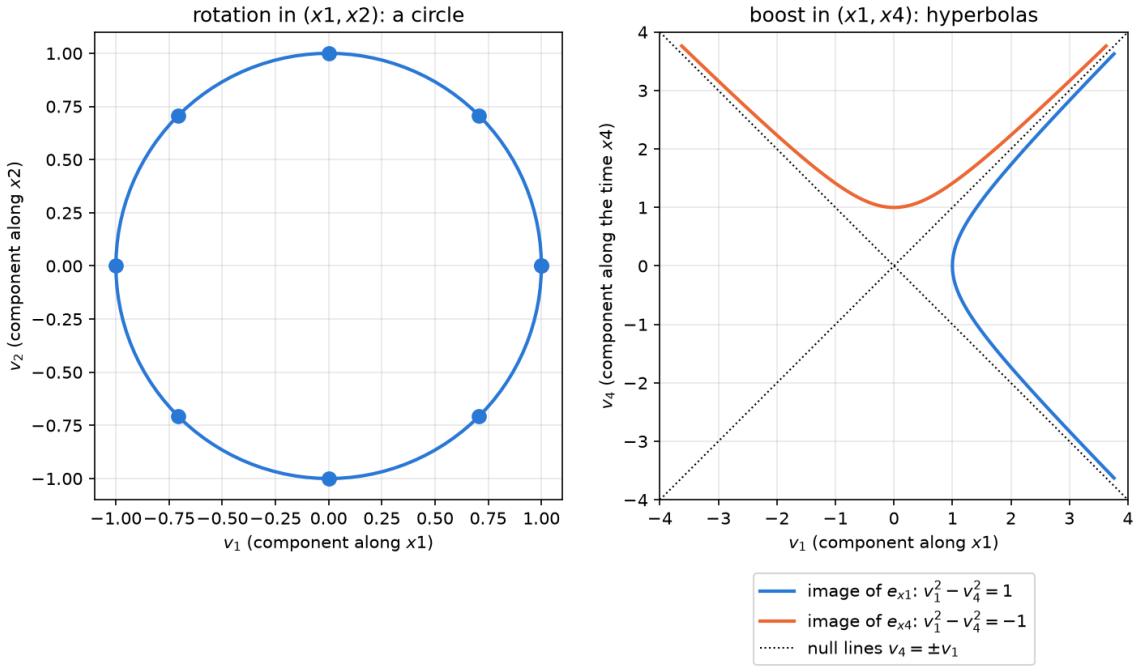

Figure 156: Where the transformations carry a unit vector. Left: the rotation in the plane $(x1, x2)$ carries the unit vector along $x1$ around the circle $v_1^2 + v_2^2 = 1$ as the angle runs from 0 to 2π (dots every $\pi/4$). Right: the boost in the plane $(x1, x4)$ carries the unit vector along $x1$ along the hyperbola $v_1^2 - v_4^2 = 1$ (blue) and the unit vector along the time $x4$ along $v_1^2 - v_4^2 = -1$ (orange), for rapidities from -2 to 2; neither crosses the dotted null lines. Axes: the components of the vector (pure numbers). (Notebook 05d, figure 4.)

Out [10]:

```
Figure 05d.4 saved as Revision/textbook/figures/05d_4_orbits.png
```

Text cell:

```
## 11. Which forms are kept: C always, B only away from the time x4

Because $(S^{ab})^TC + CS^{ab} = 0$ for every generator, every exponential keeps the
form of $C$: $R^TCR = C$. The form of $B$ (the charge density $\Psi^\dagger B\Psi$)
is different: the recorded check states that $(S^{ab})^\dagger B + BS^{ab} = 0$
holds exactly for the 21 generators that do not involve $x4$, and that for the 7
generators $S^{(x4)\,b}$ it equals $iC\gamma^b \neq 0$. The next cell checks this
generator by generator, and then for finite transformations: $R^TCR - C$ and
$R^\dagger BR - B$ ($R$ is real, so $R^\dagger = R^T$) for all 28 planes at
$\theta = 0.7$.
```

In [11]:

```
keep_B = [p for p in pairs if not (S[p].T @ B + B @ S[p]).any()]
break_B = [p for p in pairs if p not in keep_B]
rule_x4 = all(np.array_equal(S[("x4", b)].T @ B + B @ S[("x4", b)],
                             1j * (C @ gamma[b])) for b in COORDS if b != "x4")
say(f"generators with S^T B + B S = 0: {len(keep_B)}; the others: "
    + " ".join(f"({a},{b})" for a, b in break_B))
check_reproduces(len(keep_B) == 21 and all("x4" in p for p in break_B) and rule_x4
                 and recorded("wolfram", "S_preserves_B_only_off_x4"),
                 "S^dagger B + B S = 0 exactly for the 21 generators without x4; for "
                 "S^(x4 b) it equals i C gamma^b",
                 record=record_of("wolfram", "S_preserves_B_only_off_x4"))
finite_C = max(np.max(np.abs(spin_transformation(a, b, 0.7).T @ C
                             @ spin_transformation(a, b, 0.7) - C)) for a, b in pairs)
finite_B = {p: np.max(np.abs(spin_transformation(*p, 0.7).T @ B
                             @ spin_transformation(*p, 0.7) - B)) for p in pairs}
check(finite_C < 1e-12 and all((finite_B[p] < 1e-12) == (p in keep_B) for p in pairs),
      "at theta = 0.7: R^T C R = C for all 28 planes; R^T B R = B exactly for the 21 "
      "planes without x4")
```

Out [11]:

```
generators with S^T B + B S = 0: 21; the others: (x0,x4) (x1,x4) (x2,x4) (x3,x4) (x4,x5)
    (x4,x6) (x4,x7)
PASS S^dagger B + B S = 0 exactly for the 21 generators without x4; for S^(x4 b) it
    equals i C gamma^b
     reproduces Revision/algebra/reports/wolfram-algebra.json, check
    S_preserves_B_only_off_x4
PASS at theta = 0.7: R^T C R = C for all 28 planes; R^T B R = B exactly for the 21 planes
    without x4
```

Text cell:

```
The next cell draws, as functions of $\theta$, the size of the change of the form
of $B$ (the largest entry of $R^TBR - B$) for four planes: the rotations $(x1,
x2)$ and $(x5, x6)$ (no $x4$: no change), the boost $(x1, x4)$ and the rotation
$(x4, x5)$ (both involve $x4$), and the change of the form of $C$ for all four.
```

In [12]:

```
thetas = np.linspace(-3.0, 3.0, 241)
planes = [("x1", "x2"), ("x5", "x6"), ("x1", "x4"), ("x4", "x5")]
colors = ["#2a78d6", "#1baf7a", "#eb6834", "#4a3aa7"]
lines = ["-", "--", "-", "-."]
change_B = {p: [np.max(np.abs(spin_transformation(*p, th).T @ B
                              @ spin_transformation(*p, th) - B)) for th in thetas]
            for p in planes}
change_C = max(np.max(np.abs(spin_transformation(*p, th).T @ C
                             @ spin_transformation(*p, th) - C))
               for p in planes for th in thetas)
shown = "below 1e-12" if change_C < 1e-12 else f"{change_C:.2e}"
say(f"largest change of the form of C over the four planes and all angles: {shown}")
fig, ax = plt.subplots(figsize=(8.0, 4.4))
for p, color, line in zip(planes, colors, lines):
    name = "rotation" if kind[p] == 1 else "boost"
    ax.plot(thetas, change_B[p], color=color, linestyle=line, linewidth=2,
            label=f"form of $B$, {name} in $({p[0]}, {p[1]})$")
ax.plot(thetas, np.zeros_like(thetas), ":", color="black", linewidth=2,
        label="form of $C$, all four planes")
ax.set_xlabel(r"angle or rapidity $\theta$")
ax.set_ylabel("largest entry of the change")
ax.set_title(r"$R^T C R - C$ is always 0; $R^T B R - B$ only away from $x4$")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=2)
save_figure(fig, "invariant_forms",
            "How much the transformation $R = \\exp(\\theta S^{ab})$ changes the "
            "forms of $B$ and of $C$: the largest entry of $R^T B R - B$ against "
            "$\\theta$ from $-3$ to $3$ for the rotations in $(x1, x2)$ and $(x5, "
            "x6)$ (both exactly zero, on the horizontal axis), the boost in $(x1, "
            "x4)$ and the rotation in $(x4, x5)$ (both nonzero), and the largest "
            "entry of $R^T C R - C$ for all four (dotted, zero). The bilinear "
            "$\\Psi^\\dagger C\\Psi$ is invariant under all of Spin(4,4); the charge "
            "density $\\Psi^\\dagger B\\Psi$ only under the transformations that "
            "leave the time $x4$ alone.")
```

Out [12]:

```
largest change of the form of C over the four planes and all angles: below 1e-12
```

Out [12]:

```
Figure 05d.5 saved as Revision/textbook/figures/05d_5_invariant_forms.png
```

Text cell:

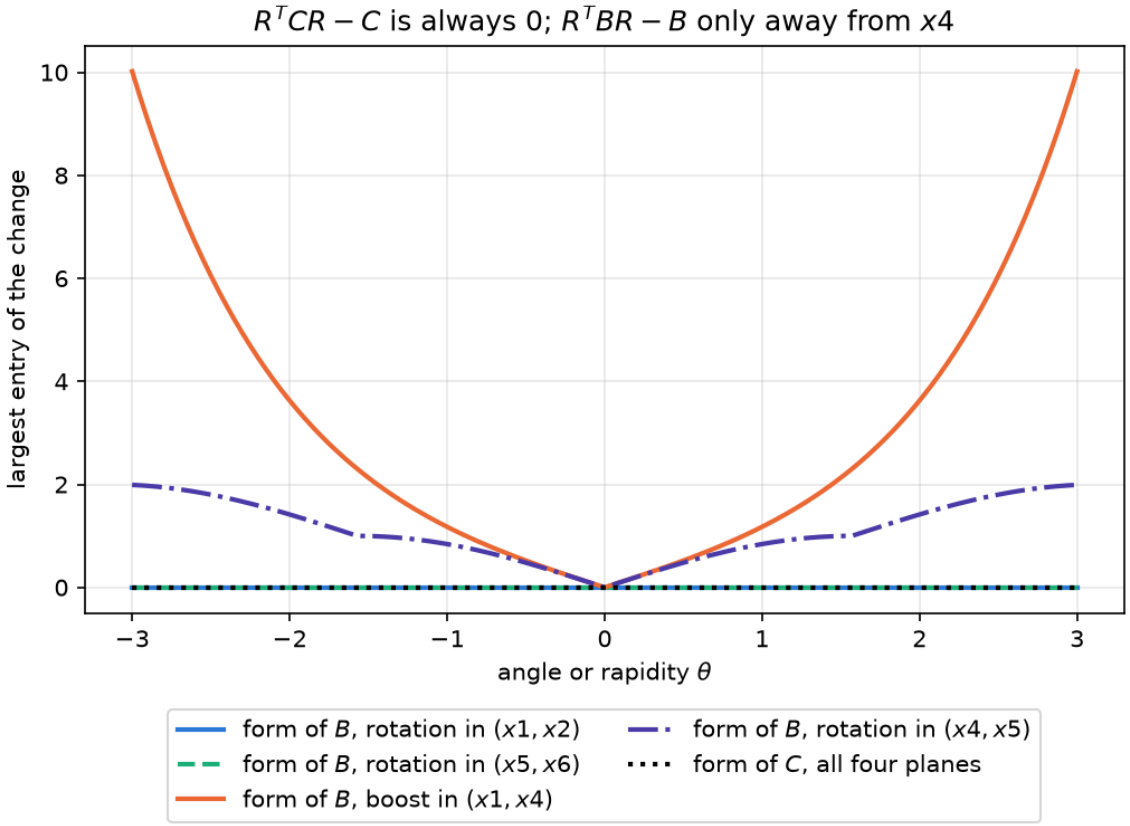

Figure 157: How much the transformation $R = \exp(\theta S^{ab})$ changes the forms of B and of C: the largest entry of $R^T BR - B$ against θ from -3 to 3 for the rotations in $(x1, x2)$ and $(x5, x6)$ (both exactly zero, on the horizontal axis), the boost in $(x1, x4)$ and the rotation in $(x4, x5)$ (both nonzero), and the largest entry of $R^T CR - C$ for all four (dotted, zero). The bilinear $\Psi^\dagger C\Psi$ is invariant under all of Spin(4,4); the charge density $\Psi^\dagger B\Psi$ only under the transformations that leave the time $x4$ alone. (Notebook 05d, figure 5.)

```
## 12. Reflections, the spinor norm, the double cover and the determinants

The next cell checks the reflection identity $-\gamma(u)\gamma(v)\gamma(u)^{-1} =
\gamma(R_uv)$ of section 4 for three unit vectors: $u = e_{x1}$ (space-like,
$\eta(u, u) = +1$), $u = e_{x4}$ (time-like, $-1$) and the tilted unit vector
$u = \cosh(0.5)\,e_{x1} + \sinh(0.5)\,e_{x4}$ (with $\eta(u, u) = \cosh^2 -
\sinh^2 = 1$), each with the fixed vector $v = (1, 2, \dots, 8)/10$, and that every
reflection has determinant $-1$ and keeps the metric.
```

In [13]:

```
def gamma_of(v):
    """gamma(v) = sum over a of v_a gamma^a."""
    return sum(v[i] * gamma[x].astype(float) for i, x in enumerate(COORDS))


def metric_product(u, v):
    """eta(u, v) = sum over a of eta_aa u_a v_a."""
    return float(u @ ETA_MATRIX @ v)


def reflection(u):
    """The 8 x 8 matrix of R_u v = v - 2 eta(u, v) u / eta(u, u)."""
    return np.eye(8) - 2.0 * np.outer(u, u @ ETA_MATRIX) / metric_product(u, u)


e = np.eye(8)  # e[i] is the unit vector along COORDS[i]
tilted = np.cosh(0.5) * e[1] + np.sinh(0.5) * e[4]  # in the plane (x1, x4)
v = np.arange(1, 9) / 10.0
reflections_ok = True
for u in (e[1], e[4], tilted):  # e_x1, e_x4 and the tilted vector
    gu = gamma_of(u)
    left = -gu @ gamma_of(v) @ np.linalg.inv(gu)
    Ru = reflection(u)
    reflections_ok &= (np.allclose(left, gamma_of(Ru @ v), atol=1e-12)
                       and abs(np.linalg.det(Ru) + 1.0) < 1e-12
                       and np.allclose(Ru.T @ ETA_MATRIX @ Ru, ETA_MATRIX))
    say(f"eta(u, u) = {metric_product(u, u):+.6f}: reflection checked")
check(reflections_ok,
      "-gamma(u) gamma(v) gamma(u)^-1 = gamma(R_u v); det R_u = -1; R_u keeps eta")
```

Out [13]:

```
eta(u, u) = +1.000000: reflection checked
eta(u, u) = -1.000000: reflection checked
eta(u, u) = +1.000000: reflection checked
PASS -gamma(u) gamma(v) gamma(u)^-1 = gamma(R_u v); det R_u = -1; R_u keeps eta
```

Text cell:

```
The next cell checks the spinor norm formula $g^TCg = (-1)^kN(g)\,C$ for five
elements: $\gamma^{(x1)}$, $\gamma^{(x4)}$, $\gamma^{(x1)}\gamma^{(x4)}$,
```

```
$\gamma(\text{tilted})\gamma^{(x5)}$ and $\Gamma$ (a product of eight gammas with
$N = (+1)^4(-1)^4 = +1$). The element $\gamma^{(x1)}\gamma^{(x4)}$ belongs to
Spin(4,4) (two factors) but has $g^TCg = -C$: since every product of exponentials
keeps $C$ (section 4), it is **not** a product of exponentials. The cell also checks
that $\Gamma$ moves every direction to its opposite ($\Lambda = -1_8$, determinant
$(-1)^8 = +1$), and the double cover along the rotation: $R(\theta + 2\pi) =
-R(\theta)$ but both give the same $\Lambda$.
```

In [14]:

```
elements = [("gamma^(x1)", [e[1]]), ("gamma^(x4)", [e[4]]),
            ("gamma^(x1) gamma^(x4)", [e[1], e[4]]),
            ("gamma(tilted) gamma^(x5)", [tilted, e[5]]),
            ("Gamma", [e[i] for i in range(8)])]
norm_ok = True
for name, factors in elements:
    g = np.eye(16)
    for u in factors:
        g = g @ gamma_of(u)
    k = len(factors)
    N = int(round(np.prod([metric_product(u, u) for u in factors])))
    predicted = (-1) ** k * N
    norm_ok &= np.allclose(g.T @ C @ g, predicted * C, atol=1e-12)
    say(f"{name:26} k = {k}, N(g) = {N:+d}: g^T C g = {predicted:+d} C")
check(norm_ok, "g^T C g = (-1)^k N(g) C for all five elements")
check(np.allclose(vector_matrix(Gamma.astype(float)), -np.eye(8), atol=1e-12),
      "Gamma moves every direction to its opposite: Lambda(Gamma) = -1 (det +1)")
cover_ok = all(
    np.allclose(spin_transformation("x1", "x2", th + 2 * np.pi),
                -spin_transformation("x1", "x2", th), atol=1e-12)
    and np.allclose(vector_matrix(spin_transformation("x1", "x2", th + 2 * np.pi)),
                    vector_matrix(spin_transformation("x1", "x2", th)), atol=1e-12)
    for th in np.linspace(0.0, 2 * np.pi, 7))
check(cover_ok, "R(theta + 2 pi) = -R(theta), and both move the vectors in the "
      "same way: two spinor matrices for every vector matrix")
```

Out [14]:

```
gamma^(x1)                k = 1, N(g) = +1: g^T C g = -1 C
gamma^(x4)                k = 1, N(g) = -1: g^T C g = +1 C
gamma^(x1) gamma^(x4)     k = 2, N(g) = -1: g^T C g = -1 C
gamma(tilted) gamma^(x5)  k = 2, N(g) = -1: g^T C g = -1 C
Gamma                     k = 8, N(g) = +1: g^T C g = +1 C
PASS g^T C g = (-1)^k N(g) C for all five elements
PASS Gamma moves every direction to its opposite: Lambda(Gamma) = -1 (det +1)
PASS R(theta + 2 pi) = -R(theta), and both move the vectors in the same way: two spinor
    matrices for every vector matrix
```

Text cell:

```
The next cell computes the determinants of the eight gammas exactly with sympy
(as $16 \times 16$ matrices). Each is $+1$ (a gamma with $\gamma\gamma = +1$ and
```

```
trace 0 has eigenvalues $+1$ and $-1$ eight times each, product $(+1)^8(-1)^8 = 1$;
one with $\gamma\gamma = -1$ has $\pm i$ eight times each, product $i^8(-i)^8 = 1$).
So every element of Pin(4,4), a product of such matrices, has determinant 1 as a
$16 \times 16$ matrix: "determinant 1" in "Spin(4,4) covers SO(4,4)" refers to the
vector matrix $\Lambda$, never to the spinor matrix.
```

In [15]:

```
import sympy as sp  # exact algebra

determinants = {x: sp.Matrix(gamma[x].tolist()).det() for x in COORDS}
say("det gamma^(x) = " + ", ".join(f"{x}: {determinants[x]}" for x in COORDS))
check(all(d == 1 for d in determinants.values()),
      "every gamma matrix has determinant +1 as a 16 x 16 matrix")
```

Out [15]:

```
det gamma^(x) = x0: 1, x1: 1, x2: 1, x3: 1, x4: 1, x5: 1, x6: 1, x7: 1
PASS every gamma matrix has determinant +1 as a 16 x 16 matrix
```

Text cell:

```
## 13. The vector matrices as pictures

The next cell draws four $8 \times 8$ vector matrices: the rotation by $\pi/3$ in
$(x1, x2)$, the boost with rapidity 1 in $(x1, x4)$, the reflection $R_u$ along
$u = e_{x1}$, and $\Lambda(\Gamma) = -1_8$. The colour scale runs from $-1.6$ to
$+1.6$ because a boost has entries $\cosh 1 \approx 1.54$.
```

In [16]:

```
pictures = [(vector_matrix(spin_transformation("x1", "x2", np.pi / 3)),
             r"rotation, $\theta = \pi/3$, $(x1, x2)$"),
            (vector_matrix(spin_transformation("x1", "x4", 1.0)),
             r"boost, $\theta = 1$, $(x1, x4)$"),
            (reflection(e[1]), r"reflection $R_u$, $u = e_{x1}$"),
            (vector_matrix(Gamma.astype(float)), r"$\Lambda(\Gamma) = -1_8$")]
fig, axes = plt.subplots(1, 4, figsize=(14.0, 4.0))
for k, (ax, (matrix, title)) in enumerate(zip(axes, pictures)):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1.6, vmax=1.6)
    for i in range(8):
        for j in range(8):
            if abs(matrix[i, j]) > 1e-12:  # write the nonzero entries
                ax.text(j, i, f"{matrix[i, j]:.2f}", ha="center", va="center",
                        fontsize=6)
    ax.set_xticks(range(8), COORDS, fontsize=7)
    ax.set_yticks(range(8), COORDS, fontsize=7)
    ax.set_title(title, fontsize=9)
    ax.grid(False)
```

```
fig.colorbar(image, ax=axes, shrink=0.8, label="matrix entry")
save_figure(fig, "vector_matrices",
            "Four 8 by 8 matrices by which transformations move the eight "
            "directions (rows and columns labelled $x0$ to $x7$; blue negative, "
            "grey 0, red positive; nonzero entries written in the squares): the "
            "rotation by $\\pi/3$ in the plane $(x1, x2)$ (entries $\\cos$ and "
            "$\\pm\\sin$ of $\\pi/3$), the boost with rapidity 1 in the plane "
            "$(x1, x4)$ (entries $\\cosh 1$ and $-\\sinh 1$), the reflection along "
            "$x1$ (one entry $-1$) and the matrix of the chirality $\\Gamma$, which "
            "reverses all eight directions.")
```

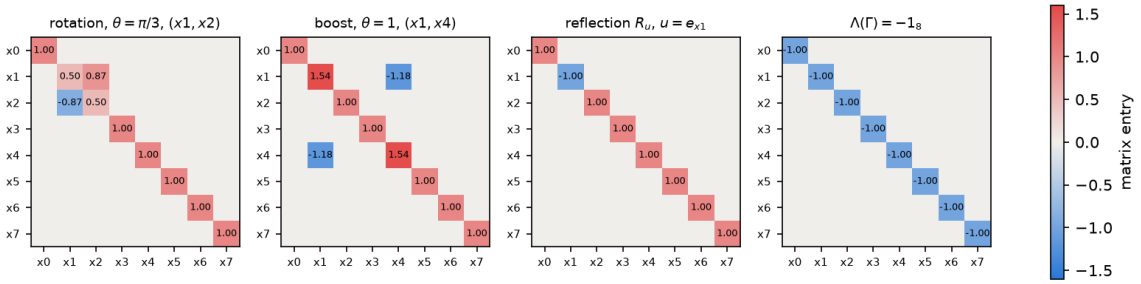

Figure 158: Four 8 by 8 matrices by which transformations move the eight directions (rows and columns labelled $x0$ to $x7$; blue negative, grey 0, red positive; nonzero entries written in the squares): the rotation by $\pi/3$ in the plane $(x1, x2)$ (entries cos and $\pm$ sin of $\pi/3$), the boost with rapidity 1 in the plane $(x1, x4)$ (entries $\cosh 1$ and $-\sinh 1$), the reflection along $x1$ (one entry -1) and the matrix of the chirality Γ, which reverses all eight directions. (Notebook 05d, figure 6.)

Out [16]:

```
Figure 05d.6 saved as Revision/textbook/figures/05d_6_vector_matrices.png
```

Text cell:

```
## 14. The last check

The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [17]:

```
FIGURES = ["05d_1_plane_types.png", "05d_2_half_angles.png",
           "05d_3_rotation_matrices.png", "05d_4_orbits.png",
           "05d_5_invariant_forms.png", "05d_6_vector_matrices.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the six figure files of notebook 05d exist")
all_checks_passed()
```

Out [17]:

```
PASS the six figure files of notebook 05d exist
ALL 20 CHECKS PASSED (notebook 05d)
```

Text cell:

```
## 15. What this notebook showed

- The 28 generators $S^{ab} = \tfrac12\gamma^a\gamma^b$ of the Revision gammas obey
  the rules of the Lie algebra so(4,4), turn the gammas like a vector, keep the form
  of $C$ and commute with $\Gamma$, as the Revision record states.
- 12 coordinate planes give rotations ($\cos$, $\sin$ of half the angle), 16 give
  boosts ($\cosh$, $\sinh$ of half the rapidity); the closed formulas agree with
  the power series of the exponential.
- The spinor matrix contains the half angle and the vector matrix the full angle:
  a rotation by $2\pi$ returns every direction but gives $R = -1$ on spinors; only
  $4\pi$ gives $R = +1$. $R$ and $-R$ move the vectors in the same way: the spinor
  group is a double cover of the group of vector transformations.
- Every spinor transformation keeps the form $\Psi^\dagger C\Psi$; the charge
  density $\Psi^\dagger B\Psi$ is kept only by the 21 generators (and their
  exponentials) that do not involve the time $x4$.
- Conjugation by a unit vector is a reflection (up to a sign), with determinant
  $-1$; the spinor norm $g^TCg = (-1)^kN(g)C$ holds; $\gamma^{(x1)}\gamma^{(x4)}$
  lies in Spin(4,4) but is not a product of exponentials; $\Gamma$ reverses all
  eight directions; every gamma has determinant $+1$ as a $16 \times 16$ matrix.
```

5.22 Line-by-line walk-through of Notebook 05d

The notebook has 17 code cells. In [1] is the set-up cell of Section 5.10 with `NOTEBOOK_ID = "05d"`; its comments repeat the instructions of Section 5.20. Docstrings are left out of the quotations and long captions are replaced by a line `...`), as in Section 5.10.

In [2], the gammas, C, Γ, B and the recorded checks.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
ETA_MATRIX = np.diag([float(ETA[x]) for x in COORDS])  # the 8 x 8 metric eta
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)
```

As in Notebook 05a, the record of the gammas is read. New is `ETA_MATRIX`, the 8×8 diagonal matrix η with floating-point entries (`np.diag` of the list of the eight signs), used for the metric product of vectors below.

```
C = gamma["x0"] @ gamma["x1"] @ gamma["x2"] @ gamma["x3"]  # the charge matrix
Gamma = I16
for x in ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]:
    Gamma = Gamma @ gamma[x]  # the chirality, the product of all eight gammas
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4)
```

C, Γ and B are built as in Sections 5.4 to 5.6; the loop multiplies the eight gammas in the author's order, starting from the identity.

```
REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    return VERDICTS[(key, name)][0] == "pass"


def record_of(key, name):
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together
```

The two reports and the helpers `recorded`, `record_of` and `check_reproduces`, exactly as in Notebook 05a (Section 5.10, In [3]).

```
def eta(a, b):
    return ETA[a] if a == b else 0


check_reproduces(all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                                    2 * eta(a, b) * I16) for a in COORDS for b in COORDS)
                 and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

`eta(a, b)` is η^{ab}: η_{aa} for equal directions and 0 otherwise. The check is the Clifford relation for all 64 pairs.

In [3], the generators and their rules.

```
S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4.0
     for a in COORDS for b in COORDS}  # all 64 ordered pairs
```

```
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 pairs


def commutator(m, n):
    return m @ n - n @ m
```

`S` holds S^{ab} for all 64 ordered pairs (including $S^{aa} = 0$), as floating-point arrays with the entries 0 and $\pm\frac{1}{2}$. Every sum and product below involves only halves and quarters, which the computer stores exactly (they are sums of powers of 2), so comparisons with `np.array_equal` are exact. `pairs` lists the 28 planes, and `commutator` is $[m, n] = mn - nm$.

```
definition_ok = all(np.array_equal(S[(a, b)], -S[(b, a)]) for a in COORDS
                    for b in COORDS) and all(
    np.array_equal(S[(a, b)], gamma[a] @ gamma[b] / 2.0) for a, b in pairs)
check_reproduces(len(pairs) == 28 and definition_ok
                 and recorded("python", "S_definition"),
                 "S^ab = -S^ba and S^ab = (1/2) gamma^a gamma^b: 28 independent "
                 "generators",
                 record=record_of("python", "S_definition"))
```

The check confirms $S^{ab} = -S^{ba}$ for all 64 pairs and $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$ for the 28 planes.

```
algebra_failures = 0
for a, b in pairs:
    for c, d in pairs:
        right = (eta(b, c) * S[(a, d)] - eta(a, c) * S[(b, d)]
                 - eta(b, d) * S[(a, c)] + eta(a, d) * S[(b, c)])
        if not np.array_equal(commutator(S[(a, b)], S[(c, d)]), right):
            algebra_failures += 1
say(f"so(4,4) rules tested for {len(pairs) ** 2} pairs; failures {algebra_failures}")
check_reproduces(algebra_failures == 0 and recorded("python", "S_lorentz_algebra")
                 and recorded("wolfram", "S_Lorentz_algebra"),
                 "[S^ab, S^cd] = eta^bc S^ad - eta^ac S^bd - eta^bd S^ac + eta^ad S^bc",
                 record=record_of("python", "S_lorentz_algebra"))
```

For each of the $28 \times 28 = 784$ pairs of planes the right side of the so(4,4) rule is assembled and compared with the commutator; failures are counted. The printed line reports 784 pairs and 0 failures.

```
vector_failures = sum(
    not np.array_equal(commutator(S[(a, b)], gamma[c]),
                       eta(b, c) * gamma[a] - eta(a, c) * gamma[b])
    for a in COORDS for b in COORDS for c in COORDS)
say(f"vector rule tested for 512 triples; failures {vector_failures}")
check_reproduces(vector_failures == 0 and recorded("python", "S_vector_action")
                 and recorded("wolfram", "S_gamma_commutator"),
                 "[S^ab, gamma^c] = eta^bc gamma^a - eta^ac gamma^b for all 512 triples",
                 record=record_of("python", "S_vector_action"))
```

The vector rule of Section 5.12 is tested for all $8 \times 8 \times 8 = 512$ triples; `sum` of true and false values counts the true ones (a failure counts 1). The printed line reports 0 failures.

In [4], the invariances of the generators.

```
check_reproduces(all(not ((S[k].T @ C + C @ S[k]).any()) for k in S)
                 and all(not commutator(Gamma, S[k]).any() for k in S)
                 and recorded("python", "S_preserves_C_and_commutes_with_Gamma"),
                 "(S^ab)^T C + C S^ab = 0 and [Gamma, S^ab] = 0 for all a, b",
                 record=record_of("python", "S_preserves_C_and_commutes_with_Gamma"))
```

For all 64 keys `k` of `S` the cell checks $(S^{ab})^T C + C S^{ab} = 0$ (so the scalar does not change to first order) and $[\Gamma, S^{ab}] = 0$ (so the halves are kept).

In [5], rotations and boosts.

```
kind = {}  # (a, b) -> +1 for a rotation, -1 for a boost
squares_ok = True
for a, b in pairs:
    J = gamma[a] @ gamma[b]
    squares_ok &= np.array_equal(J @ J, -ETA[a] * ETA[b] * I16)
    kind[(a, b)] = ETA[a] * ETA[b]
rotations = [p for p in pairs if kind[p] == 1]
boosts = [p for p in pairs if kind[p] == -1]
say(f"rotations: {len(rotations)}; boosts: {len(boosts)}")
say("rotation planes: " + " ".join(f"({a},{b})" for a, b in rotations))
check(squares_ok and len(rotations) == 12 and len(boosts) == 16,
      "(gamma^a gamma^b)^2 = -eta_aa eta_bb: 12 rotation planes and 16 boost planes")
```

For each plane $J = \gamma^a \gamma^b$ is squared and compared with $-\eta_{aa}\eta_{bb}1$; the plane is a rotation when $\eta_{aa}\eta_{bb} = +1$ and a boost otherwise. The printed lines show 12 rotations and 16 boosts and list the rotation planes: the six inside $\{x0, x1, x2, x3\}$ and the six inside $\{x4, x5, x6, x7\}$.

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
table = np.zeros((8, 8))
for (a, b), value in kind.items():
    i, j = COORDS.index(a), COORDS.index(b)
    table[i, j] = table[j, i] = value  # the same plane in both orders
fig, ax = plt.subplots(figsize=(6.4, 5.6))
ax.imshow(table, cmap=SIGNS, vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        word = "" if i == j else ("rot" if table[i, j] == 1 else "boost")
        ax.text(j, i, word, ha="center", va="center", color="white", fontsize=8,
                fontweight="bold")
for k in range(1, 8):
    ax.axhline(k - 0.5, color="white", linewidth=2)
    ax.axvline(k - 0.5, color="white", linewidth=2)
ax.set_xticks(range(8), COORDS)
```

```
ax.set_yticks(range(8), COORDS)
ax.set_title("The 28 planes: rotations (red) and boosts (blue)")
ax.grid(False)
save_figure(fig, "plane_types",
            ...)
```

The kinds are written into an 8×8 table, each plane in both orders (`COORDS.index(a)` is the position of a name in the list; `x = y = value` sets both entries). The table is drawn with the colour map of the heat maps (red $+1$, blue -1, grey 0 on the diagonal), with the word rot or boost in each square and white gaps between the squares. `05d_1_plane_types.png` shows two red 4×4 squares of rotations, for $\{x0, x1, x2, x3\}$ and for the four times, and blue boosts wherever a space-like direction meets a time-like one.

In [6], the exponential two ways.

```
def exp_series(M, terms=80):
    result = np.eye(M.shape[0])
    term = np.eye(M.shape[0])
    for k in range(1, terms):
        term = term @ M / k  # M^k / k! from M^(k-1) / (k-1)!
        result = result + term
    return result
```

`exp_series` sums the power series of the exponential up to the term $M^{79}/79!$. Each term is computed from the previous one by multiplying by M and dividing by k, because $M^k/k! = (M^{k-1}/(k-1)!)\, M/k$.

```
def spin_transformation(a, b, theta):
    J = (gamma[a] @ gamma[b]).astype(float)  # J = 2 S^ab
    if ETA[a] * ETA[b] == 1:  # J J = -1: a rotation
        return np.cos(theta / 2) * np.eye(16) + np.sin(theta / 2) * J
    return np.cosh(theta / 2) * np.eye(16) + np.sinh(theta / 2) * J  # a boost
```

`spin_transformation(a, b, theta)` is the closed formula of Section 5.18: $\cos\frac{\theta}{2}\,1 + \sin\frac{\theta}{2}\,J$ for a rotation and $\cosh\frac{\theta}{2}\,1 + \sinh\frac{\theta}{2}\,J$ for a boost.

```
tests = [(("x1", "x2"), [0.3, 1.0, 2.5, 2 * np.pi, 4 * np.pi]),
         (("x4", "x5"), [0.3, 1.0, 2.5, 2 * np.pi, 4 * np.pi]),
         (("x1", "x4"), [-1.5, 0.3, 1.0, 2.5]),
         (("x0", "x5"), [-1.5, 0.3, 1.0, 2.5])]
largest_difference = 0.0
for (a, b), angles in tests:
    for theta in angles:
        difference = np.max(np.abs(exp_series(theta * S[(a, b)])
                                   - spin_transformation(a, b, theta)))
        largest_difference = max(largest_difference, difference)
    name = "rotation" if kind[(a, b)] == 1 else "boost"
    say(f"plane ({a},{b}), a {name}: series and closed formula compared at "
        f"{len(angles)} values")
check(largest_difference < 1e-10,
```

```
      "the power series of exp(theta S^ab) equals the closed formula with half "
      "angles (difference below 1e-10)")
```

Two rotations (one of them in the plane of two times) and two boosts are tested at several angles or rapidities; for each, the largest difference of any entry between the series and the closed formula is kept in `largest_difference`. Four lines are printed, and the check requires the largest difference to be below 10^{-10} (the series is summed in floating-point numbers, and at 4π its terms first grow large before they shrink).

In [7], the vector matrix and the sign after 2π.

```
def vector_matrix(R):
    R_inverse = np.linalg.inv(R)
    Lam = np.zeros((8, 8))
    for j, c in enumerate(COORDS):
        moved = R @ gamma[c] @ R_inverse  # where the direction c is moved to
        for i, d in enumerate(COORDS):
            Lam[i, j] = ETA[d] * np.trace(gamma[d] @ moved) / 16.0
    return Lam
```

`vector_matrix(R)` computes Λ by the trace formula of Section 5.18: for each direction c it forms $R\gamma^c R^{-1}$ (`np.linalg.inv` is the inverse matrix) and fills column c with $\eta_{dd}\,\mathrm{tr}(\gamma^d R\gamma^c R^{-1})/16$ for each row d.

```
def rebuilds(R, Lam):
    R_inverse = np.linalg.inv(R)
    return all(np.allclose(sum(Lam[i, j] * gamma[d] for i, d in enumerate(COORDS)),
                           R @ gamma[c] @ R_inverse, atol=1e-12)
               for j, c in enumerate(COORDS))
```

`rebuilds` checks that $\sum_d \Lambda_{dc}\gamma^d$ really equals $R\gamma^c R^{-1}$ for every c, that is, that the moved gamma is a combination of gammas with the coefficients read off. `np.allclose(x, y, atol=1e-12)` is true when every entry of `x` differs from that of `y` by at most about 10^{-12}.

```
rotation_ok = True
for theta in np.linspace(0.0, 2 * np.pi, 9):
    R = spin_transformation("x1", "x2", theta)
    Lam = vector_matrix(R)
    expected = np.eye(8)  # the rotation by theta in the (x1, x2) block
    expected[1, 1] = expected[2, 2] = np.cos(theta)  # rows and columns x1, x2
    expected[1, 2], expected[2, 1] = np.sin(theta), -np.sin(theta)
    rotation_ok &= (rebuilds(R, Lam) and np.allclose(Lam, expected, atol=1e-12)
                    and np.allclose(Lam.T @ ETA_MATRIX @ Lam, ETA_MATRIX)
                    and abs(np.linalg.det(Lam) - 1.0) < 1e-12)
check(rotation_ok,
      "R = exp(theta S^(x1 x2)) turns the directions x1, x2 by the full angle "
      "theta; Lambda lies in SO(4,4)")
```

For nine angles from 0 to 2π the cell computes the rotation in the plane $(x1, x2)$ and its vector matrix, and compares it with the prediction of Section 5.18: the identity except for the block of

$x1$ and $x2$, which holds $\cos\theta$ on the diagonal, $\sin\theta$ in row $x1$, column $x2$, and $-\sin\theta$ in row $x2$, column $x1$. It also checks $\Lambda^T\eta\Lambda = \eta$ and $\det\Lambda = 1$ (`np.linalg.det`): Λ lies in SO(4,4).

```
R_2pi, R_4pi = spin_transformation("x1", "x2", 2 * np.pi), spin_transformation(
    "x1", "x2", 4 * np.pi)
# Rounding leaves differences of about 1e-16, which differ between computers, so
# the cell prints only whether they are below 1e-12.
small_2pi = np.max(np.abs(R_2pi + np.eye(16))) < 1e-12
small_4pi = np.max(np.abs(R_4pi - np.eye(16))) < 1e-12
say(f"every entry of R(2 pi) + 1 is below 1e-12: {small_2pi}; "
    f"every entry of R(4 pi) - 1 is below 1e-12: {small_4pi}")
check(np.allclose(R_2pi, -np.eye(16), atol=1e-12)
      and np.allclose(R_4pi, np.eye(16), atol=1e-12)
      and np.allclose(vector_matrix(R_2pi), np.eye(8), atol=1e-12),
      "a rotation by 2 pi gives R = -1 on spinors but Lambda = 1 on vectors; by 4 pi, R "
      "= +1")
```

$R(2\pi)$ and $R(4\pi)$ are computed; $\sin\pi$ is not exactly 0 in floating-point numbers, so the cell prints only whether the entries of $R(2\pi)+1$ and $R(4\pi)-1$ are below 10^{-12} (both True). The check is the double cover: $R(2\pi) = -1$ while its vector matrix is the identity, and $R(4\pi) = +1$.

In [8], the picture of the half angles.

```
angles = np.linspace(0.0, 4 * np.pi, 241)
rapidities = np.linspace(-3.0, 3.0, 241)
vec_rot = [vector_matrix(spin_transformation("x1", "x2", th))[1, 1] for th in angles]
spin_rot = [np.trace(spin_transformation("x1", "x2", th)) / 16 for th in angles]
vec_boost = [vector_matrix(spin_transformation("x1", "x4", th))[1, 1]
             for th in rapidities]
spin_boost = [np.trace(spin_transformation("x1", "x4", th)) / 16 for th in rapidities]
check(np.allclose(vec_rot, np.cos(angles)) and np.allclose(spin_rot, np.cos(angles / 2))
      and np.allclose(vec_boost, np.cosh(rapidities))
      and np.allclose(spin_boost, np.cosh(rapidities / 2)),
      "vectors move with cos(theta), cosh(theta); spinors with the half angle")
```

For 241 angles from 0 to 4π (rotation in $(x1, x2)$) and 241 rapidities from -3 to 3 (boost in $(x1, x4)$), the cell computes the vector entry $\Lambda_{x1,x1}$ and the spinor quantity $\operatorname{tr} R/16$; since $\operatorname{tr} J = 0$ (Rule 5), $\operatorname{tr} R/16$ is $\cos\frac{\theta}{2}$ or $\cosh\frac{\theta}{2}$. The check compares the four lists with $\cos\theta$, $\cos\frac{\theta}{2}$, $\cosh\theta$ and $\cosh\frac{\theta}{2}$.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.4))
axes[0].plot(angles, vec_rot, color="#2a78d6", linewidth=2,
             label=r"vector: $\Lambda_{x1,x1} = \cos\theta$")
axes[0].plot(angles, spin_rot, color="#eb6834", linewidth=2, linestyle="--",
             label=r"spinor: $\mathrm{tr}\,R/16 = \cos(\theta/2)$")
axes[0].plot([2 * np.pi, 2 * np.pi], [1.0, -1.0], "o", color="black", markersize=7,
             label=r"at $2\pi$: vector back at $1$, spinor at $-1$")
axes[0].set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
                   ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
axes[0].set_xlabel(r"rotation angle $\theta$ in the plane $(x1, x2)$")
axes[0].set_ylabel("value")
axes[0].set_title("A rotation: the spinor needs $4\\pi$ to come back")
```

```
axes[0].legend(loc="upper center", bbox_to_anchor=(0.5, -0.17))
axes[1].plot(rapidities, vec_boost, color="#2a78d6", linewidth=2,
             label=r"vector: $\Lambda_{x1,x1} = \cosh\theta$")
axes[1].plot(rapidities, spin_boost, color="#eb6834", linewidth=2, linestyle="--",
             label=r"spinor: $\mathrm{tr}\,R/16 = \cosh(\theta/2)$")
axes[1].set_xlabel(r"rapidity $\theta$ in the plane $(x1, x4)$")
axes[1].set_title("A boost: never periodic;\nthe spinor grows half as fast")
axes[1].legend(loc="upper center", bbox_to_anchor=(0.5, -0.17))
save_figure(fig, "half_angles",
            ...)
```

Two pictures: on the left the two rotation curves and two black dots at $\theta = 2\pi$ (the vector entry at 1, the spinor quantity at -1); on the right the two boost curves. The tick marks of the left horizontal axis are at multiples of π; `"\n"` in a title starts a new line; both legends go below their pictures. In `05d_2_half_angles.png` the solid cosine completes two periods over 4π while the dashed one completes one: the spinor needs 4π to come back.

In [9], the rotation matrices.

```
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.9))
for k, (ax, th, label) in enumerate(zip(
        axes, [0.0, np.pi, 2 * np.pi, 4 * np.pi], ["0", r"\pi", r"2\pi", r"4\pi"])):
    image = ax.imshow(spin_transformation("x1", "x2", th), cmap=SIGNS, vmin=-1,
                      vmax=1)
    ax.set_title(rf"$R(\theta)$ at $\theta = {label}$")
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("column")
    if k == 0:
        ax.set_ylabel("row")
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
check(np.allclose(spin_transformation("x1", "x2", np.pi),
                  gamma["x1"] @ gamma["x2"], atol=1e-12),
      "R(pi) = gamma^(x1) gamma^(x2) for the rotation in the plane (x1, x2)")
save_figure(fig, "rotation_matrices",
            ...)
```

The loop draws $R(\theta)$ at $\theta = 0$, π, 2π and 4π as heat maps (the `zip` runs through the four axes, the four angles and their labels together). The check confirms $R(\pi) = \gamma^{(x1)}\gamma^{(x2)}$ (because $\cos\frac{\pi}{2} = 0$ and $\sin\frac{\pi}{2} = 1$). In `05d_3_rotation_matrices.png` the first picture is the identity (red diagonal), the second the signed permutation $\gamma^{(x1)}\gamma^{(x2)}$, the third minus the identity (blue diagonal) and the fourth the identity again.

In [10], circles and hyperbolas.

```
# column k of a vector matrix is the image of e_xk; its entry 1 is v_1, entry 4 v_4
circle = np.array([vector_matrix(spin_transformation("x1", "x2", th))[:, 1]
                   for th in np.linspace(0.0, 2 * np.pi, 121)])  # images of e_x1
hyper_space = np.array([vector_matrix(spin_transformation("x1", "x4", th))[:, 1]
                        for th in np.linspace(-2.0, 2.0, 121)])  # images of e_x1
```

```
hyper_time = np.array([vector_matrix(spin_transformation("x1", "x4", th))[:, 4]
                       for th in np.linspace(-2.0, 2.0, 121)])  # images of e_x4
check(np.allclose(circle[:, 1] ** 2 + circle[:, 2] ** 2, 1.0)
      and np.allclose(hyper_space[:, 1] ** 2 - hyper_space[:, 4] ** 2, 1.0)
      and np.allclose(hyper_time[:, 1] ** 2 - hyper_time[:, 4] ** 2, -1.0),
      "rotations keep v1^2 + v2^2, boosts keep v1^2 - v4^2")
```

Column c of a vector matrix is the image of the basic unit vector e_c; Python counts the positions from 0, so position k is the direction xk, `[:, 1]` is the column of $x1$ and `[:, 4]` that of $x4$, and in a column the entries `[1]` and `[4]` are the components v_1 and v_4. The cell collects the images of e_{x1} under 121 rotations in $(x1, x2)$, and the images of e_{x1} and of e_{x4} under 121 boosts in $(x1, x4)$. The check confirms that the rotation keeps $v_1^2 + v_2^2 = 1$ and the boost keeps $v_1^2 - v_4^2$, equal to $+1$ for the image of e_{x1} and -1 for that of e_{x4}.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 5.0))
axes[0].plot(circle[:, 1], circle[:, 2], color="#2a78d6", linewidth=2)
axes[0].plot(circle[::15, 1], circle[::15, 2], "o", color="#2a78d6", markersize=8)
axes[0].set_aspect("equal")
axes[0].set_xlabel("$v_1$ (component along $x1$)")
axes[0].set_ylabel("$v_2$ (component along $x2$)")
axes[0].set_title("rotation in $(x1, x2)$: a circle")
axes[1].plot(hyper_space[:, 1], hyper_space[:, 4], color="#2a78d6", linewidth=2,
             label=r"image of $e_{x1}$: $v_1^2 - v_4^2 = 1$")
axes[1].plot(hyper_time[:, 1], hyper_time[:, 4], color="#eb6834", linewidth=2,
             label=r"image of $e_{x4}$: $v_1^2 - v_4^2 = -1$")
axes[1].plot([-4, 4], [-4, 4], ":", color="black", linewidth=1,
             label="null lines $v_4 = \\pm v_1$")
axes[1].plot([-4, 4], [4, -4], ":", color="black", linewidth=1)
axes[1].set_xlim(-4, 4)
axes[1].set_ylim(-4, 4)
axes[1].set_aspect("equal")
axes[1].set_xlabel("$v_1$ (component along $x1$)")
axes[1].set_ylabel("$v_4$ (component along the time $x4$)")
axes[1].set_title("boost in $(x1, x4)$: hyperbolas")
axes[1].legend(loc="upper center", bbox_to_anchor=(0.5, -0.14))
save_figure(fig, "orbits",
            ...)
```

The left picture draws the circle and a dot every 15 steps (`[::15]` takes every fifteenth row, that is every $\pi/4$); `set_aspect("equal")` gives both axes the same scale, so that a circle looks round. The right picture draws the two hyperbola branches and the two dotted **null lines** $v_4 = \pm v_1$, on which $v_1^2 - v_4^2 = 0$. In `05d_4_orbits.png` neither hyperbola crosses the null lines: a boost never turns a space-like vector into a time-like one.

In [11], which generators keep the form of B.

```
keep_B = [p for p in pairs if not (S[p].T @ B + B @ S[p]).any()]
break_B = [p for p in pairs if p not in keep_B]
rule_x4 = all(np.array_equal(S[("x4", b)].T @ B + B @ S[("x4", b)],
                             1j * (C @ gamma[b])) for b in COORDS if b != "x4")
say(f"generators with S^T B + B S = 0: {len(keep_B)}; the others: "
```

```
    + " ".join(f"({a},{b})" for a, b in break_B))
check_reproduces(len(keep_B) == 21 and all("x4" in p for p in break_B) and rule_x4
                 and recorded("wolfram", "S_preserves_B_only_off_x4"),
                 "S^dagger B + B S = 0 exactly for the 21 generators without x4; for "
                 "S^(x4 b) it equals i C gamma^b",
                 record=record_of("wolfram", "S_preserves_B_only_off_x4"))
```

`keep_B` lists the planes whose generator obeys $S^T B + BS = 0$ (S is real, so $S^T = S^\dagger$); `break_B` the others. `rule_x4` checks the formula $iC\gamma^b$ of Section 5.18 for the seven generators $S^{(x4)b}$. The printed line shows 21 planes that keep B and the seven planes with $x4$. (`"x4" in p` is true when the pair `p` contains the name `x4`.)

```
finite_C = max(np.max(np.abs(spin_transformation(a, b, 0.7).T @ C
                             @ spin_transformation(a, b, 0.7) - C)) for a, b in pairs)
finite_B = {p: np.max(np.abs(spin_transformation(*p, 0.7).T @ B
                             @ spin_transformation(*p, 0.7) - B)) for p in pairs}
check(finite_C < 1e-12 and all((finite_B[p] < 1e-12) == (p in keep_B) for p in pairs),
      "at theta = 0.7: R^T C R = C for all 28 planes; R^T B R = B exactly for the 21 "
      "planes without x4")
```

For finite transformations at $\theta = 0.7$ the cell computes the largest entry of $R^T C R - C$ over all 28 planes and of $R^T B R - B$ for each plane (`spin_transformation(*p, 0.7)` unpacks the pair `p` into the two arguments `a` and `b`). The check: the form of C is kept for all planes, that of B exactly for the 21 planes without $x4$.

In [12], the picture of the forms.

```
thetas = np.linspace(-3.0, 3.0, 241)
planes = [("x1", "x2"), ("x5", "x6"), ("x1", "x4"), ("x4", "x5")]
colors = ["#2a78d6", "#1baf7a", "#eb6834", "#4a3aa7"]
lines = ["-", "--", "-", "-."]
change_B = {p: [np.max(np.abs(spin_transformation(*p, th).T @ B
                              @ spin_transformation(*p, th) - B)) for th in thetas]
            for p in planes}
change_C = max(np.max(np.abs(spin_transformation(*p, th).T @ C
                             @ spin_transformation(*p, th) - C))
               for p in planes for th in thetas)
shown = "below 1e-12" if change_C < 1e-12 else f"{change_C:.2e}"
say(f"largest change of the form of C over the four planes and all angles: {shown}")
```

For four planes (two rotations without $x4$, a boost and a rotation with $x4$) and 241 values of θ the cell computes how much the form of B changes; for the form of C only the largest change over everything is kept and printed (below 10^{-12}; the cell prints the words instead of a number because the last digits of rounding differ between computers).

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
for p, color, line in zip(planes, colors, lines):
    name = "rotation" if kind[p] == 1 else "boost"
    ax.plot(thetas, change_B[p], color=color, linestyle=line, linewidth=2,
            label=f"form of $B$, {name} in $({p[0]}, {p[1]})$")
```

```
ax.plot(thetas, np.zeros_like(thetas), ":", color="black", linewidth=2,
        label="form of $C$, all four planes")
ax.set_xlabel(r"angle or rapidity $\theta$")
ax.set_ylabel("largest entry of the change")
ax.set_title(r"$R^T C R - C$ is always 0; $R^T B R - B$ only away from $x4$")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=2)
save_figure(fig, "invariant_forms",
            ...)
```

One curve per plane for the form of B, and a dotted line at zero for the form of C. In `05d_5_in variant_forms.png` the two rotations without $x4$ lie on the horizontal axis (no change), while the boost in $(x1, x4)$ and the rotation in $(x4, x5)$ rise away from $\theta = 0$.

In [13], reflections.

```
def gamma_of(v):
    return sum(v[i] * gamma[x].astype(float) for i, x in enumerate(COORDS))


def metric_product(u, v):
    return float(u @ ETA_MATRIX @ v)


def reflection(u):
    return np.eye(8) - 2.0 * np.outer(u, u @ ETA_MATRIX) / metric_product(u, u)
```

`gamma_of(v)` is $\gamma(v) = \sum_a v_a \gamma^a$; `metric_product(u, v)` is $\eta(u, v) = u^T \eta v$; `reflection(u)` is the 8×8 matrix of R_u: since $R_u v = v - 2u\,(u^T \eta v)/\eta(u, u)$, its matrix is $1 - 2\,u\,(\eta u)^T/\eta(u, u)$, and `np.outer(u, u @ ETA_MATRIX)` is the column u times the row $u^T \eta$.

```
e = np.eye(8)  # e[i] is the unit vector along COORDS[i]
tilted = np.cosh(0.5) * e[1] + np.sinh(0.5) * e[4]  # in the plane (x1, x4)
v = np.arange(1, 9) / 10.0
reflections_ok = True
for u in (e[1], e[4], tilted):  # e_x1, e_x4 and the tilted vector
    gu = gamma_of(u)
    left = -gu @ gamma_of(v) @ np.linalg.inv(gu)
    Ru = reflection(u)
    reflections_ok &= (np.allclose(left, gamma_of(Ru @ v), atol=1e-12)
                       and abs(np.linalg.det(Ru) + 1.0) < 1e-12
                       and np.allclose(Ru.T @ ETA_MATRIX @ Ru, ETA_MATRIX))
    say(f"eta(u, u) = {metric_product(u, u):+.6f}: reflection checked")
check(reflections_ok,
      "-gamma(u) gamma(v) gamma(u)^-1 = gamma(R_u v); det R_u = -1; R_u keeps eta")
```

Three unit vectors are tested: e_{x1} (space-like), e_{x4} (time-like) and the tilted $\cosh(0.5)\,e_{x1} + \sinh(0.5)\,e_{x4}$ (with $\eta = \cosh^2 - \sinh^2 = 1$), each with the fixed vector $v = (0.1, 0.2, \ldots, 0.8)$. For each, the key identity of Section 5.12, $\det R_u = -1$ and $R_u^T \eta R_u = \eta$ are checked, and $\eta(u, u)$ is printed: $+1$, -1, $+1$.

In [14], the spinor norm, $\Lambda(\Gamma)$ and the double cover.

```
elements = [("gamma^(x1)", [e[1]]), ("gamma^(x4)", [e[4]]),
            ("gamma^(x1) gamma^(x4)", [e[1], e[4]]),
            ("gamma(tilted) gamma^(x5)", [tilted, e[5]]),
            ("Gamma", [e[i] for i in range(8)])]
norm_ok = True
for name, factors in elements:
    g = np.eye(16)
    for u in factors:
        g = g @ gamma_of(u)
    k = len(factors)
    N = int(round(np.prod([metric_product(u, u) for u in factors])))
    predicted = (-1) ** k * N
    norm_ok &= np.allclose(g.T @ C @ g, predicted * C, atol=1e-12)
    say(f"{name:26} k = {k}, N(g) = {N:+d}: g^T C g = {predicted:+d} C")
check(norm_ok, "g^T C g = (-1)^k N(g) C for all five elements")
```

Five elements of Pin(4,4) are given by their unit-vector factors (for Γ: e_{x0} followed by $e_{x1}, \ldots, e_{x7}$). For each, the product g, the number of factors k and the spinor norm $N(g)$ (the product of the $\eta(u,u)$, rounded to a whole number) are computed, and $g^T C g$ is compared with $(-1)^k N(g)\, C$. The printed table shows in particular $g^T C g = -C$ for $\gamma^{(x1)}\gamma^{(x4)}$ and for $\gamma(\text{tilted})\gamma^{(x5)}$: two even elements that are not products of exponentials.

```
check(np.allclose(vector_matrix(Gamma.astype(float)), -np.eye(8), atol=1e-12),
      "Gamma moves every direction to its opposite: Lambda(Gamma) = -1 (det +1)")
cover_ok = all(
    np.allclose(spin_transformation("x1", "x2", th + 2 * np.pi),
                -spin_transformation("x1", "x2", th), atol=1e-12)
    and np.allclose(vector_matrix(spin_transformation("x1", "x2", th + 2 * np.pi)),
                    vector_matrix(spin_transformation("x1", "x2", th)), atol=1e-12)
    for th in np.linspace(0.0, 2 * np.pi, 7))
check(cover_ok, "R(theta + 2 pi) = -R(theta), and both move the vectors in the "
      "same way: two spinor matrices for every vector matrix")
```

Γ is even, and its vector matrix is -1_8: by (X2), $\Gamma\gamma^c\Gamma^{-1} = -\gamma^c$. Then, for seven angles, $R(\theta + 2\pi) = -R(\theta)$ and the two have the same vector matrix.

In [15], the determinants of the gammas.

```
import sympy as sp  # exact algebra

determinants = {x: sp.Matrix(gamma[x].tolist()).det() for x in COORDS}
say("det gamma^(x) = " + ", ".join(f"{x}: {determinants[x]}" for x in COORDS))
check(all(d == 1 for d in determinants.values()),
      "every gamma matrix has determinant +1 as a 16 x 16 matrix")
```

sympy computes the eight determinants exactly. The printed line shows 1 for every direction, as Rule 6 predicts.

In [16], four vector matrices as pictures.

```
pictures = [(vector_matrix(spin_transformation("x1", "x2", np.pi / 3)),
             r"rotation, $\theta = \pi/3$, $(x1, x2)$"),
```

```
            (vector_matrix(spin_transformation("x1", "x4", 1.0)),
             r"boost, $\theta = 1$, $(x1, x4)$"),
            (reflection(e[1]), r"reflection $R_u$, $u = e_{x1}$"),
            (vector_matrix(Gamma.astype(float)), r"$\Lambda(\Gamma) = -1_8$")]
fig, axes = plt.subplots(1, 4, figsize=(14.0, 4.0))
for k, (ax, (matrix, title)) in enumerate(zip(axes, pictures)):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1.6, vmax=1.6)
    for i in range(8):
        for j in range(8):
            if abs(matrix[i, j]) > 1e-12:  # write the nonzero entries
                ax.text(j, i, f"{matrix[i, j]:.2f}", ha="center", va="center",
                        fontsize=6)
    ax.set_xticks(range(8), COORDS, fontsize=7)
    ax.set_yticks(range(8), COORDS, fontsize=7)
    ax.set_title(title, fontsize=9)
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.8, label="matrix entry")
save_figure(fig, "vector_matrices",
            ...)
```

Four 8×8 matrices are drawn with the colour scale from -1.6 to 1.6 (a boost has entries $\cosh 1 \approx 1.54$), and each nonzero entry is written in its square with two decimals (`:.2f`). In `05d_6_vector_matrices.png` the rotation has 0.50 and ±0.87 (cos and sin of $\pi/3$) in the $(x1, x2)$ block, the boost 1.54 and -1.18 ($\cosh 1$ and $-\sinh 1$) in the $(x1, x4)$ block, the reflection one -1 at $x1$, and $\Lambda(\Gamma)$ is -1 on the whole diagonal.

In [17], the last check.

```
FIGURES = ["05d_1_plane_types.png", "05d_2_half_angles.png",
           "05d_3_rotation_matrices.png", "05d_4_orbits.png",
           "05d_5_invariant_forms.png", "05d_6_vector_matrices.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the six figure files of notebook 05d exist")
all_checks_passed()
```

The six figure files must exist, and the last line reads ALL 20 CHECKS PASSED (notebook 05d): one in In [2], three in In [3], one in In [4], In [5] and In [6] each, two in In [7], one each in In [8], In [9] and In [10], two in In [11], one in In [13], three in In [14], one in In [15] and one in In [17]. Six of them (In [2], the three of In [3], In [4] and the first of In [11]) reproduce recorded checks; the other fourteen are the notebook's own computations.

5.23 What do the scaled commutators generate?

The 28 scaled commutators $S^{ab} = \frac{1}{4}[\gamma^a, \gamma^b]$ are called the generators. It is tempting to say that they generate Pin(4,4). This section answers exactly what they generate, and the answer is: **not all of Pin(4,4), and not even all of Spin(4,4)**. The products of their exponentials form the piece of Pin(4,4) that is joined to 1, written $\mathrm{Spin}_0(4,4)$; Pin(4,4) consists of four such pieces; and the exponentials together with the two gammas $\gamma^{(x0)}$ and $\gamma^{(x4)}$ generate all of Pin(4,4). The proof has ten steps, (A) to (J); Notebook 05f checks each of them on the matrices.

(A) The matrices by which the generators move the directions. The vector rule of Section 5.12 can be written $[S^{ab},\gamma^c]=\sum_d M^{ab}_{dc}\gamma^d$ with the 8×8 matrix

$$M^{ab}_{dc}=\eta^{bc}\delta_{da}-\eta^{ac}\delta_{db},$$

where δ_{da} is 1 for $d=a$ and 0 otherwise (insert it: the sum over d picks $\eta^{bc}\gamma^a-\eta^{ac}\gamma^b$). For $a\neq b$ exactly two entries are nonzero: in row a, column b the entry $\eta^{bb}=\eta_{bb}$ (take $c=b$, $d=a$), and in row b, column a the entry $-\eta_{aa}$ (take $c=a$, $d=b$).

(B) A basis of so(4,4). The Lie algebra **so(4,4)** is the set of real 8×8 matrices X with $X^T\eta+\eta X=0$ (the infinitesimal form of $\Lambda^T\eta\Lambda=\eta$). Since $(\eta X)^T=X^T\eta$, the condition says that ηX is antisymmetric. An antisymmetric 8×8 matrix is fixed by its $8\cdot 7/2=28$ entries above the diagonal, so so(4,4) has dimension 28. For M^{ab} the matrix ηM^{ab} has the entry $\eta_{aa}\eta_{bb}$ in row a, column b and $-\eta_{bb}\eta_{aa}$ in row b, column a: it is antisymmetric, so every M^{ab} lies in so(4,4). The 28 matrices M^{ab} with a before b have their nonzero entries in 28 different pairs of places, so they are independent: they are a **basis** of so(4,4).

(C) Commutators go to commutators. Multiplying out shows the **Jacobi identity** $[[X,Y],Z]=[X,[Y,Z]]-[Y,[X,Z]]$ for any three matrices (both sides are $XYZ-YXZ-ZXY+ZYX$). Let $[S_1,\gamma^c]=\sum_d(M_1)_{dc}\gamma^d$ and $[S_2,\gamma^c]=\sum_d(M_2)_{dc}\gamma^d$. Then

$$[S_1,[S_2,\gamma^c]]=\sum_d(M_2)_{dc}[S_1,\gamma^d]=\sum_{d,e}(M_1)_{ed}(M_2)_{dc}\gamma^e=\sum_e(M_1M_2)_{ec}\gamma^e,$$

and the Jacobi identity gives $[[S_1,S_2],\gamma^c]=\sum_e([M_1,M_2])_{ec}\gamma^e$: **the matrix of a commutator is the commutator of the matrices.** Together with (B) and the so(4,4) rules of Section 5.18, the 28 independent S^{ab} and the 28 matrices M^{ab} have the same commutation table: the S^{ab} span an exact copy of so(4,4). (The S^{ab} are independent because they are one half times 28 different products of two gammas, which are independent by Section 5.13.)

(D) Every exponential is a product of two unit vectors. For a rotation ($\eta_{aa}\eta_{bb}=+1$), $\exp(\theta S^{ab})=\cos\frac{\theta}{2}\,1+\sin\frac{\theta}{2}\,\gamma^a\gamma^b$ (Section 5.18). Because $\gamma^a\gamma^a=\eta_{aa}1$ and $\eta_{aa}^2=1$,

$$\gamma^a\big(\eta_{aa}\cos\tfrac{\theta}{2}\,\gamma^a+\sin\tfrac{\theta}{2}\,\gamma^b\big)=\cos\tfrac{\theta}{2}\,1+\sin\tfrac{\theta}{2}\,\gamma^a\gamma^b.$$

So $\exp(\theta S^{ab})=\gamma^a\gamma(u)$ with $u=\eta_{aa}\cos\frac{\theta}{2}\,e_a+\sin\frac{\theta}{2}\,e_b$, and $\eta(u,u)=\eta_{aa}\cos^2\frac{\theta}{2}+\eta_{bb}\sin^2\frac{\theta}{2}=\eta_{aa}$ (since $\eta_{bb}=\eta_{aa}$ for a rotation). For a boost the same steps with cosh and sinh give $u=\eta_{aa}\cosh\frac{\theta}{2}\,e_a+\sinh\frac{\theta}{2}\,e_b$ and $\eta(u,u)=\eta_{aa}(\cosh^2\frac{\theta}{2}-\sinh^2\frac{\theta}{2})=\eta_{aa}$, since $\eta_{bb}=-\eta_{aa}$. Both γ^a and $\gamma(u)$ are unit vectors: **every exponential lies in Spin(4,4)**, and so does every product of exponentials.

(E) How an exponential moves the directions. Put $R(\theta)=\exp(\theta S)$ for one generator $S=S^{ab}$, and $F_c(\theta)=R\gamma^cR^{-1}$. Differentiating the power series term by term gives $dR/d\theta=SR=RS$, and $d(R^{-1})/d\theta=-SR^{-1}$ (since $R^{-1}=\exp(-\theta S)$). By the product rule of differentiation,

$$\frac{dF_c}{d\theta}=SR\gamma^cR^{-1}-R\gamma^cSR^{-1}=R\,[S,\gamma^c]\,R^{-1}=\sum_d M_{dc}F_d.$$

Writing $F_c=\sum_e\Lambda_{ec}(\theta)\gamma^e$, this says $d\Lambda/d\theta=\Lambda M$ with $\Lambda(0)=1$, whose solution is the matrix

exponential $\Lambda(\theta) = \exp(\theta M)$ (it solves the equation, because $d\exp(\theta M)/d\theta = \exp(\theta M)M$, and a linear equation of this kind has only one solution with a given starting value, a fact of the theory of differential equations quoted here). From the two entries of $M = M^{ab}$: $Me_a = -\eta_{aa}e_b$ and $Me_b = \eta_{bb}e_a$, so $MMe_a = -\eta_{aa}\eta_{bb}e_a$. Summing the power series as in Section 5.18:

$$\Lambda e_a = \cos\theta\, e_a - \eta_{aa}\sin\theta\, e_b \quad \text{(rotation)}, \qquad \Lambda e_a = \cosh\theta\, e_a - \eta_{aa}\sinh\theta\, e_b \quad \text{(boost)}.$$

(F) The forbidden band. Order the directions as $x0, x1, x2, x3$ (space-like) and then $x4, x5, x6, x7$ (time-like), and cut a vector matrix into four 4×4 blocks,

$$\Lambda = \begin{pmatrix} A & B' \\ C' & D \end{pmatrix}.$$

A is the **space block** and D the **time block** (the primes distinguish the other two blocks from the matrices B and C). In this order $\eta = \mathrm{diag}(1_4, -1_4)$, and multiplying out the blocks of $\Lambda^T\eta\Lambda = \eta$ gives, in the top-left and bottom-right places,

$$A^TA - C'^TC' = 1_4, \qquad D^TD - B'^TB' = 1_4.$$

For every column w of four numbers the first equation gives $|Aw|^2 = w^TA^TAw = |w|^2 + |C'w|^2 \geq |w|^2$, where $|w|^2 = w^Tw$ is the squared length. So every eigenvalue of the symmetric matrix A^TA is at least 1 (take w an eigenvector of length 1), and $(\det A)^2 = \det(A^TA)$, the product of these eigenvalues, is at least 1. **So** $\det A$ **is never between** -1 **and** 1; the same holds for $\det D$. This holds for every matrix of O(4,4).

(G) Products of exponentials stay above the band. Let $h = \exp(\theta_1 S_1)\cdots\exp(\theta_n S_n)$ be a product of exponentials, and $h(t) = \exp(t\theta_1 S_1)\cdots\exp(t\theta_n S_n)$ for $0 \leq t \leq 1$: a path from $h(0) = 1$ to $h(1) = h$. The entries of $\Lambda(h(t))$ are sums of products of cos, sin, cosh and sinh of multiples of t, so $\det A(t)$ is a continuous function of t. It starts at $\det 1_4 = 1$ and never enters the band $(-1, 1)$; by the intermediate value theorem (a continuous function that takes a value below -1 and one above 1 also takes every value between) it can never become negative. So $\det A \geq 1$ along the whole path, and in the same way $\det D \geq 1$. **Every product of exponentials has the sign pattern** $(+,+)$, where the **sign pattern** of g is the pair (sign of $\det A$, sign of $\det D$).

(H) Four pieces. $\Lambda(\gamma^{(x0)}) = R_{e_{x0}}$ reverses only the direction $x0$: its space block has determinant -1 and its time block $+1$, pattern $(-,+)$. $\Lambda(\gamma^{(x4)})$ reverses only $x4$: pattern $(+,-)$. $\Lambda(\gamma^{(x0)}\gamma^{(x4)})$ reverses both: $(-,-)$. For a product of exponentials h and w one of these, $\Lambda(hw) = \Lambda(h)\Lambda(w)$ (Section 5.12), and $\Lambda(w)$ is a diagonal matrix of signs: multiplying by it from the right only changes the signs of some columns of $\Lambda(h)$, so the space block of hw is A times a diagonal matrix of signs and hw has the pattern of w. Four different patterns: the four sets

$$\mathrm{Spin}_0, \quad \mathrm{Spin}_0\,\gamma^{(x0)}, \quad \mathrm{Spin}_0\,\gamma^{(x4)}, \quad \mathrm{Spin}_0\,\gamma^{(x0)}\gamma^{(x4)}$$

have no element in common, where $\mathrm{Spin}_0 = \mathrm{Spin}_0(4,4)$ is the group of all products of exponentials and $\mathrm{Spin}_0\,w$ the set of the products hw. In particular $\gamma^{(x0)}\gamma^{(x4)}$ **lies in Spin(4,4) but is not a product of exponentials**, in agreement with its spinor norm (Section 5.18).

(I) Every unit vector is a turned $\gamma^{(x0)}$ or $\gamma^{(x4)}$. Let v be a space-like unit vector, with space part σ (its components along $x0, x1, x2, x3$) and time part τ (along $x4, \ldots, x7$), so that $|\sigma|^2 - |\tau|^2 = 1$. Three stages carry e_{x0} to v:

- a boost in the plane $(x0, x4)$ carries e_{x0} to $|\sigma|\, e_{x0} + |\tau|\, e_{x4}$ (by (E), choose $\sinh\theta = -|\tau|$; then $\cosh\theta = \sqrt{1+|\tau|^2} = |\sigma|$);
- three rotations in the planes $(x4, x5)$, $(x4, x6)$, $(x4, x7)$ turn e_{x4} into $\tau/|\tau|$ and do not touch $x0$, giving $|\sigma|\, e_{x0} + \tau$;
- three rotations in the planes $(x0, x1)$, $(x0, x2)$, $(x0, x3)$ turn e_{x0} into $\sigma/|\sigma|$ and do not touch the time part, giving $\sigma + \tau = v$.

Each rotation stage turns the axis vector step by step: the first rotation puts the right component on the first other direction and keeps the rest of the length on the axis, the second does the same for the next direction, and the last leaves exactly the axis component of the target. The angles follow from (E). With h the product of these at most seven exponentials, $\Lambda(h)e_{x0} = v$, and since h is even,

$$h\gamma^{(x0)}h^{-1} = \sum_d \Lambda(h)_{d,x0}\,\gamma^d = \gamma(\Lambda(h)e_{x0}) = \gamma(v).$$

A time-like unit vector v has $|\tau|^2 - |\sigma|^2 = 1$. Three stages carry e_{x4} to v; compared with the space-like case, the boost has $\cosh\theta = |\tau|$ instead of $|\sigma|$, and the two rotation stages exchange their places:

- a boost in the plane $(x4, x0)$ carries e_{x4} to $|\tau|\, e_{x4} + |\sigma|\, e_{x0}$ (by (E) with $a = x4$ and $\eta_{aa} = -1$, $\Lambda e_{x4} = \cosh\theta\, e_{x4} + \sinh\theta\, e_{x0}$; choose $\sinh\theta = |\sigma|$; then $\cosh\theta = \sqrt{1+|\sigma|^2} = |\tau|$);
- three rotations in the planes $(x0, x1)$, $(x0, x2)$, $(x0, x3)$ turn e_{x0} into $\sigma/|\sigma|$ and do not touch $x4$, giving $|\tau|\, e_{x4} + \sigma$;
- three rotations in the planes $(x4, x5)$, $(x4, x6)$, $(x4, x7)$ turn e_{x4} into $\tau/|\tau|$ and do not touch the space part, giving $\sigma + \tau = v$.

With h the product of these at most seven exponentials, $\Lambda(h)e_{x4} = v$, and, since h is again even, the same final step with $x4$ in place of $x0$ gives

$$h\gamma^{(x4)}h^{-1} = \sum_d \Lambda(h)_{d,x4}\,\gamma^d = \gamma(\Lambda(h)e_{x4}) = \gamma(v).$$

In both cases, when the boost puts nothing on the other axis ($\tau = 0$ for a space-like v, $\sigma = 0$ for a time-like v), its angle is $\theta = 0$ and the middle stage is left out.

(J) Moving the gammas to the right. Moving γ^e through $\gamma^a\gamma^b$ costs $(-1)^2 = +1$ when e is neither a nor b, and -1 when e is one of them (Rule 1). So $\gamma^e S^{ab}(\gamma^e)^{-1} = s\, S^{ab}$ with that sign s, and, term by term in the power series,

$$\gamma^e \exp(\theta S^{ab}) = \exp(s\theta S^{ab})\,\gamma^e.$$

Now take any element $g = \gamma(v_1)\cdots\gamma(v_k)$ of Pin(4,4). Write each factor as in (I), $\gamma(v_j) = h_j\gamma^{e_j}h_j^{-1}$ with $e_j = x0$ or $x4$. Move every gamma to the right end with the rule just proved: what remains on the left is a product of exponentials, and on the right a product of gammas $\gamma^{(x0)}$ and $\gamma^{(x4)}$. With $\gamma^{(x0)}\gamma^{(x0)} = 1$, $\gamma^{(x4)}\gamma^{(x4)} = -1$ and $\gamma^{(x4)}\gamma^{(x0)} = -\gamma^{(x0)}\gamma^{(x4)}$, that product is $\pm$ one of 1, $\gamma^{(x0)}$, $\gamma^{(x4)}$, $\gamma^{(x0)}\gamma^{(x4)}$, and a sign -1 is itself the product of exponentials $\exp(2\pi S^{(x1)(x2)}) = -1$ (Section 5.18).

The answer. By (D) the exponentials lie in Pin(4,4), and $\gamma^{(x0)}$ and $\gamma^{(x4)}$ do by definition; by (I) and (J) every element of Pin(4,4) is a product of exponentials times one of the four words; by (G) and (H) the four pieces are different. So

$$\mathrm{Pin}(4,4) = \mathrm{Spin}_0 \cup \mathrm{Spin}_0\,\gamma^{(x0)} \cup \mathrm{Spin}_0\,\gamma^{(x4)} \cup \mathrm{Spin}_0\,\gamma^{(x0)}\gamma^{(x4)},$$

four pieces with no element in common, and Spin(4,4), the even elements, is the union of the first and the last piece. **The exponentials of the scaled commutators generate exactly $\mathrm{Spin}_0(4,4)$; together with $\gamma^{(x0)}$ and $\gamma^{(x4)}$ they generate all of Pin(4,4).** It is not correct to say that the scaled commutators alone generate Pin(4,4). Nothing in the representation statements of Section 5.13 changes: irreducibility under Pin(4,4) and the two inequivalent halves already hold for the smaller groups (Section 5.13).

What the group elements span. For a rotation, $\exp(\pi S^{ab}) = \gamma^a\gamma^b$ ($\cos\frac{\pi}{2} = 0$, $\sin\frac{\pi}{2} = 1$); for a boost, $(\exp(\theta S^{ab}) - \exp(-\theta S^{ab}))/(2\sinh\frac{\theta}{2}) = \gamma^a\gamma^b$. So the span of $\mathrm{Spin}_0(4,4)$ contains every product of two different gammas, and, because the span of a group contains the products of its members, every even product of different gammas: all 128 of them. Every element is even, so the span is exactly these 128 dimensions, the block-diagonal matrices. Adding one gamma adds the odd products: the span of Pin(4,4) is all 256 dimensions.

statement	**status**	**where it is verified**
the 28 S^{ab} are independent (rank 28) and span a copy of so(4,4)	PROVED; rank COMPUTED exactly	`python-algebra.json`, checks `S_definition`, `S_vector_action` and `S_lorentz_algebra`; Notebook 05f (rank 28, dimension of so(4,4) 28, the 784 commutators of the M^{ab})
(D) and (E): $\exp(\theta S^{ab}) = \gamma^a\gamma(u)$; $\Lambda = \exp(\theta M^{ab})$	PROVED above; COMPUTED (to 10^{-10} and 10^{-12})	Notebook 05f, its own computation
(F) to (H): the forbidden band, the sign pattern $(+,+)$ of $\mathrm{Spin}_0(4,4)$, the four pieces	PROVED above; COMPUTED on 200 and 240 random elements and along ten paths	Notebook 05f, its own computation
(I) and (J): every element of Pin(4,4) is in one of the four pieces	PROVED above; COMPUTED on 40 unit vectors and 12 elements	Notebook 05f, its own computation
the spans: 128 for $\mathrm{Spin}_0(4,4)$, 256 for Pin(4,4)	PROVED above; COMPUTED (singular values)	`python-algebra.json`, checks `even_products_span_M8_plus_M8` and `clifford_products_span_M16`; Notebook 05f

5.24 Example: Notebook 05f computes what the scaled commutators generate

Notebook 05f checks steps (A) to (J) one by one: the rank 28 of the generators, the 28 matrices M^{ab} and their commutators, the dimension 28 of so(4,4), the factorisation of every exponential into two unit vectors, the formula $\Lambda = \exp(\theta M)$, the block equations and the band on 200 random products of exponentials, the four sign patterns on 240 random elements, ten paths from 1 that never cross the band, the construction of (I) on 40 random unit vectors, the rule of (J), the decomposition of 12 random elements of Pin(4,4) into exponentials times one of four words, and the spans 128 and 256. All random numbers come from a generator with a fixed seed, so every run makes the same choices. It draws six figures and ends with ALL 24 CHECKS PASSED (notebook 05f).

5.25 How to run Notebook 05f

Step 1. What this notebook does and what it needs.

Notebook 05f (What the scaled commutators generate: Spin_0(4,4), and with two gammas all of Pin(4,4)) is the file `Revision/textbook/notebooks/05f_pin_generation.ipynb` of the repository Dirac_claude. It answers exactly the question whether the 28 scaled commutators S^ab = (1/4)(gamma^a gamma^b - gamma^b gamma^a) generate Pin(4,4). It checks that they are 28 independent matrices whose action on the eight directions is a basis of so(4,4) with the same commutators, writes every exponential exp(theta S^ab) as a product of two unit vectors, proves with the space and time blocks of the vector matrices that the products of exponentials form only the part Spin_0(4,4) of Pin(4,4) that is joined to 1, constructs for every unit vector v a product h of exponentials with gamma(v) = h gamma^(x0) h^-1 or h gamma^(x4) h^-1, and so writes every element of Pin(4,4) as a product of exponentials times one of 1, gamma^(x0), gamma^(x4) and gamma^(x0) gamma^(x4). Six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05f_pin_generation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05f_pin_generation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 05f_pin_generation.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/05f.captions.json`
- `Revision/textbook/figures/05f_1_generator_matrices.png`
- `Revision/textbook/figures/05f_2_two_unit_vectors.png`
- `Revision/textbook/figures/05f_3_four_pieces.png`
- `Revision/textbook/figures/05f_4_paths_and_band.png`
- `Revision/textbook/figures/05f_5_carrying_vectors.png`
- `Revision/textbook/figures/05f_6_span_ranks.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 05f exist
ALL 24 CHECKS PASSED (notebook 05f)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/05f_pin_generation.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 05f_pin_generation.ipynb
python -m nbconvert --execute --inplace 05f_pin_generation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

5.26 Notebook 05f: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/05f_pin_generation.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The 28 **scaled commutators** $S^{ab} = \tfrac14[\gamma^a, \gamma^b] =
\tfrac14(\gamma^a\gamma^b - \gamma^b\gamma^a)$ of the author's eight real
$16 \times 16$ gamma matrices are called the *generators*. Do they generate the
group Pin(4,4)? This notebook gives the exact answer, with every step written out
and checked on the matrices:

1. the $S^{ab}$ are 28 independent matrices; the way each of them moves the eight
   coordinate directions is an $8 \times 8$ matrix $M^{ab}$, and the 28 matrices
   $M^{ab}$ form a basis of the Lie algebra so(4,4) and have the same commutators
   as the $S^{ab}$;
2. every exponential $\exp(\theta S^{ab})$ is a product $\gamma^a\gamma(u)$ of two
   unit vectors, so it lies in Spin(4,4), the even part of Pin(4,4);
3. the products of exponentials form a group, written $\mathrm{Spin}_0(4,4)$, whose
   elements are all joined to $1$ by a path. It is **not** all of Pin(4,4), and not
   even all of Spin(4,4): the space block and the time block of the matrix by which
   such an element moves the directions both have determinant at least $1$, while
   $\gamma^{(x0)}$, $\gamma^{(x4)}$ and $\gamma^{(x0)}\gamma^{(x4)}$ give the other
   three combinations of signs;
4. every unit vector $\gamma(v)$ equals $h\gamma^{(x0)}h^{-1}$ or
   $h\gamma^{(x4)}h^{-1}$ with $h$ a product of at most seven exponentials (an
```

```
   explicit construction, tested on 40 random unit vectors);
5. therefore every element of Pin(4,4) is a product of exponentials times exactly
   one of $1$, $\gamma^{(x0)}$, $\gamma^{(x4)}$, $\gamma^{(x0)}\gamma^{(x4)}$
   (tested on 12 random elements): Pin(4,4) consists of four pieces, and it is
   generated by the exponentials of the scaled commutators **together with the two
   gammas** $\gamma^{(x0)}$ and $\gamma^{(x4)}$;
6. the elements of $\mathrm{Spin}_0(4,4)$ span the 128 block-diagonal matrices and
   those of Pin(4,4) all 256 matrices, the ranks recorded in the Revision record.

Every check that repeats a recorded check prints the record file and the check
name. Six teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 05f (What the scaled commutators generate: Spin_0(4,4), and with two gammas all
of Pin(4,4)) is the file ‘Revision/textbook/notebooks/05f_pin_generation.ipynb‘ of the
repository Dirac_claude. It answers exactly the question whether the 28 scaled
commutators S^ab = (1/4)(gamma^a gamma^b - gamma^b gamma^a) generate Pin(4,4). It checks
that they are 28 independent matrices whose action on the eight directions is a basis of
so(4,4) with the same commutators, writes every exponential exp(theta S^ab) as a product
of two unit vectors, proves with the space and time blocks of the vector matrices that
the products of exponentials form only the part Spin_0(4,4) of Pin(4,4) that is joined to
1, constructs for every unit vector v a product h of exponentials with gamma(v) = h
gamma^(x0) h^-1 or h gamma^(x4) h^-1, and so writes every element of Pin(4,4) as a
product of exponentials times one of 1, gamma^(x0), gamma^(x4) and gamma^(x0) gamma^(x4).
Six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy,
sympy and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):
```

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05f_pin_generation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05f_pin_generation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 05f_pin_generation.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/05f.captions.json‘
- ‘Revision/textbook/figures/05f_1_generator_matrices.png‘
- ‘Revision/textbook/figures/05f_2_two_unit_vectors.png‘
- ‘Revision/textbook/figures/05f_3_four_pieces.png‘
- ‘Revision/textbook/figures/05f_4_paths_and_band.png‘
- ‘Revision/textbook/figures/05f_5_carrying_vectors.png‘
- ‘Revision/textbook/figures/05f_6_span_ranks.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 05f exist
ALL 24 CHECKS PASSED (notebook 05f)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/05f_pin_generation.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 05f_pin_generation.ipynb
  python -m nbconvert --execute --inplace 05f_pin_generation.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers

‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 05f (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 05f (What the scaled commutators generate: Spin_0(4,4), and with two gammas
# all of Pin(4,4)) is the file Revision/textbook/notebooks/05f_pin_generation.ipynb of
# the repository Dirac_claude. It answers exactly the question whether the 28 scaled
# commutators S^ab = (1/4)(gamma^a gamma^b - gamma^b gamma^a) generate Pin(4,4). It
# checks that they are 28 independent matrices whose action on the eight directions is a
# basis of so(4,4) with the same commutators, writes every exponential exp(theta S^ab) as
# a product of two unit vectors, proves with the space and time blocks of the vector
# matrices that the products of exponentials form only the part Spin_0(4,4) of Pin(4,4)
# that is joined to 1, constructs for every unit vector v a product h of exponentials
# with gamma(v) = h gamma^(x0) h^-1 or h gamma^(x4) h^-1, and so writes every element of
# Pin(4,4) as a product of exponentials times one of 1, gamma^(x0), gamma^(x4) and
# gamma^(x0) gamma^(x4). Six teaching plots. It needs a computer with Windows 11, macOS
# or Linux, an internet connection for the installation, the program Git, and Python 3.12
# or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
```

```
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
```

```
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 05f_pin_generation.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
```

```
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 05f_pin_generation.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 05f_pin_generation.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/05f.captions.json
# - Revision/textbook/figures/05f_1_generator_matrices.png
# - Revision/textbook/figures/05f_2_two_unit_vectors.png
# - Revision/textbook/figures/05f_3_four_pieces.png
# - Revision/textbook/figures/05f_4_paths_and_band.png
# - Revision/textbook/figures/05f_5_carrying_vectors.png
# - Revision/textbook/figures/05f_6_span_ranks.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the six figure files of notebook 05f exist
#     ALL 24 CHECKS PASSED (notebook 05f)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/05f_pin_generation.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
```

```
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 05f_pin_generation.ipynb
#     python -m nbconvert --execute --inplace 05f_pin_generation.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra: the notebook was
#   opened outside the repository, or the repository is incomplete; clone the repository
#   again and open the notebook from its folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "05f"  # this notebook: chapter 05, example f


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
```

```
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
```

```
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 05f complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Commutator** $[X, Y] = XY - YX$ of two matrices.
- **Unit vector**: eight numbers $v = (v_{x0}, \dots, v_{x7})$ with metric product
```

```
  $\eta(v, v) = \sum_a \eta_{aa} v_a^2 = +1$ (*space-like*) or $-1$ (*time-like*);
  $\eta_{aa} = +1$ for $x0, x1, x2, x3$ and $-1$ for $x4, x5, x6, x7$. The basic
  unit vectors are $e_{x0}, \dots, e_{x7}$ (one entry 1, the others 0).
- $\gamma(v) = \sum_a v_a\gamma^a$; it obeys $\gamma(v)\gamma(v) = \eta(v, v)\,1$.
- **Pin(4,4)**: all products $\gamma(u_1)\gamma(u_2)\cdots\gamma(u_k)$ of unit
  vectors ($k = 0, 1, 2, \dots$). **Spin(4,4)**: those with an even number $k$ of
  factors (the *even* elements; the others are *odd*).
- **Exponential** of a matrix: $\exp(X) = 1 + X + X^2/2 + X^3/6 + \dots =
  \sum_{k \ge 0} X^k/k!$.
- **$\mathrm{Spin}_0(4,4)$**: all products of exponentials $\exp(\theta S^{ab})$.
  It is a group: the product of two products is a product, and the inverse of
  $\exp(\theta S^{ab})$ is $\exp(-\theta S^{ab})$.
- **Generated by**: the group *generated by* some matrices is the set of all
  products of these matrices and their inverses.
- **O(4,4)**: the real $8 \times 8$ matrices $\Lambda$ with $\Lambda^T\eta\Lambda =
  \eta$ (they keep the metric product); **SO(4,4)**: those with determinant $+1$.
- **so(4,4)**: the real $8 \times 8$ matrices $X$ with $X^T\eta + \eta X = 0$ (the
  *Lie algebra* of O(4,4)).
- **Rotation, boost**: the exponential $\exp(\theta S^{ab})$ of a plane of two
  directions of the same kind (rotation, angle $\theta$) or of different kinds
  (boost, rapidity $\theta$).
- **Vector matrix** $\Lambda(g)$: the $8 \times 8$ matrix with
  $\alpha(g)\gamma^c g^{-1} = \sum_d \Lambda_{dc}\gamma^d$, where $\alpha(g) = g$
  for an even and $-g$ for an odd $g$ (the *twisted* action).
- **Reflection** $R_u$ along a unit vector $u$: $R_u v = v - 2\eta(u, v)u/
  \eta(u, u)$; it reverses $u$ and keeps every direction orthogonal to $u$.
- **Space block, time block**: with the directions ordered $x0, x1, x2, x3$ (space)
  and then $x4, x5, x6, x7$ (time), a vector matrix splits into four $4 \times 4$
  blocks $\Lambda = \begin{pmatrix} A & B \\ C & D \end{pmatrix}$; $A$ is the
  *space block* and $D$ the *time block*.
- **Sign pattern** of $g$: the pair $(\mathrm{sign}\det A, \mathrm{sign}\det D)$.
- **Rank** of a list of matrices: the number of independent ones among them;
  **span**: all their combinations.
```

Text cell:

```
## 4. The physical and mathematical situation

**(A) The generators move the directions.** For $a \neq b$ the gammas anticommute,
so $S^{ab} = \tfrac12\gamma^a\gamma^b$. Using the Clifford relation
$\gamma^b\gamma^c = -\gamma^c\gamma^b + 2\eta^{bc}$ and then $\gamma^a\gamma^c =
-\gamma^c\gamma^a + 2\eta^{ac}$:

$$\gamma^a\gamma^b\gamma^c = -\gamma^a\gamma^c\gamma^b + 2\eta^{bc}\gamma^a
  = \gamma^c\gamma^a\gamma^b - 2\eta^{ac}\gamma^b + 2\eta^{bc}\gamma^a .$$

Halving gives the **vector rule** $[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a -
\eta^{ac}\gamma^b = \sum_d M^{ab}_{dc}\gamma^d$ with the $8 \times 8$ matrix

$$M^{ab}_{dc} = \eta^{bc}\delta_{da} - \eta^{ac}\delta_{db}$$

($\delta_{da}$ is 1 for $d = a$ and 0 otherwise). Exactly two entries are
nonzero: row $a$, column $b$ holds $\eta_{bb}$; row $b$, column $a$ holds
```

$-\eta_{aa}$.

(B) so(4,4). $X^T\eta + \eta X = 0$ says that ηX is antisymmetric (because $(\eta X)^T = X^T\eta$). An antisymmetric 8×8 matrix is fixed by its $8 \cdot 7/2 = 28$ entries above the diagonal, so so(4,4) has dimension 28. For M^{ab}: $(\eta M^{ab})_{ab} = \eta_{aa}\eta_{bb}$ and $(\eta M^{ab})_{ba} = -\eta_{bb}\eta_{aa}$: antisymmetric, so M^{ab} lies in so(4,4).

(C) Commutators go to commutators. Expanding both sides shows the *Jacobi identity* $[[X, Y], Z] = [X, [Y, Z]] - [Y, [X, Z]]$ (each side is $XYZ - YXZ - ZXY + ZYX$). If $[S_1, \gamma^c] = \sum_d (M_1)_{dc}\gamma^d$ and the same for S_2, then $[S_1, [S_2, \gamma^c]] = \sum_d (M_2)_{dc}[S_1, \gamma^d] = \sum_e (M_1M_2)_{ec}\gamma^e$, and the Jacobi identity gives $[[S_1, S_2], \gamma^c] = \sum_e ([M_1, M_2])_{ec}\gamma^e$: the matrix of a commutator is the commutator of the matrices. So the 28 independent S^{ab} and the 28 independent M^{ab} (a basis of so(4,4)) have the same commutation table: the S^{ab} span an exact copy of so(4,4).

(D) Each exponential is a product of two unit vectors. For a rotation ($\eta_{aa}\eta_{bb} = +1$), $(\gamma^a\gamma^b)^2 = -1$ and the power series gives $\exp(\theta S^{ab}) = \cos\tfrac\theta2 + \sin\tfrac\theta2\,\gamma^a\gamma^b$; for a boost, $(\gamma^a\gamma^b)^2 = +1$ and $\exp(\theta S^{ab}) = \cosh\tfrac\theta2 + \sinh\tfrac\theta2\,\gamma^a\gamma^b$. Because $\gamma^a\gamma^a = \eta_{aa}$ and $\eta_{aa}^2 = 1$,

$$\gamma^a\big(\eta_{aa}\cos\tfrac\theta2\,\gamma^a + \sin\tfrac\theta2\,
  \gamma^b\big) = \cos\tfrac\theta2 + \sin\tfrac\theta2\,\gamma^a\gamma^b ,$$

so $\exp(\theta S^{ab}) = \gamma^a\gamma(u)$ with $u = \eta_{aa}\cos\tfrac\theta2 \,e_a + \sin\tfrac\theta2\,e_b$ and $\eta(u, u) = \eta_{aa}\cos^2\tfrac\theta2 + \eta_{bb}\sin^2\tfrac\theta2 = \eta_{aa}$. For a boost, $u = \eta_{aa}\cosh \tfrac\theta2\,e_a + \sinh\tfrac\theta2\,e_b$ and $\eta(u, u) = \eta_{aa} (\cosh^2 - \sinh^2) = \eta_{aa}$, since $\eta_{bb} = -\eta_{aa}$. Two unit vectors: the exponential lies in Spin(4,4).

(E) How an exponential moves the directions. Put $R(\theta) = \exp(\theta S)$ and $F_c(\theta) = R\gamma^cR^{-1}$. Since $dR/d\theta = SR = RS$ and $d(R^{-1})/d\theta = -SR^{-1}$, the product rule gives $dF_c/d\theta = R[S, \gamma^c]R^{-1} = \sum_d M_{dc}F_d$, with $F_c(0) = \gamma^c$. This linear system is solved by the matrix exponential: $\Lambda(R(\theta)) = \exp(\theta M)$. From the two entries of M^{ab}: $Me_a = -\eta_{aa}e_b$ and $Me_b = \eta_{bb} e_a$, so $M^2e_a = -\eta_{aa}\eta_{bb}e_a$ and

$$\Lambda e_a = \cos\theta\, e_a - \eta_{aa}\sin\theta\, e_b \ \text{(rotation)},
  \qquad \Lambda e_a = \cosh\theta\, e_a - \eta_{aa}\sinh\theta\, e_b
  \ \text{(boost)} .$$

(F) The forbidden band. Every vector matrix keeps the metric, $\Lambda^T\eta\Lambda = \eta$. In the space/time block form, $\eta = \mathrm{diag}(1_4, -1_4)$, and the top-left block of this equation reads $A^TA - C^TC = 1_4$ (the bottom-right one $D^TD - B^TB = 1_4$). For every column w of four numbers, $w^TA^TAw = w^Tw + w^TC^TCw$, that is $|Aw|^2 = |w|^2 + |Cw|^2 \geq |w|^2$. So every eigenvalue of the symmetric matrix A^TA is at least 1, and $(\det A)^2 = \det(A^TA) \geq 1$: **$\det A$ is never between -1 and

1**, and the same holds for $\det D$.

(G) Products of exponentials stay above the band. For $h = \exp(\theta_1 S_1)\cdots\exp(\theta_n S_n)$ put $h(t) = \exp(t\theta_1 S_1)\cdots\exp(t\theta_n S_n)$, $0 \le t \le 1$: a path from $h(0) = 1$ (with $\det A = 1$) to $h(1) = h$. The entries of $\Lambda(h(t))$ are sums of products of $\cos$, $\sin$, $\cosh$, $\sinh$ of multiples of t, so $\det A(t)$ is a continuous function of t. It starts at 1 and can never enter the band, so by the intermediate value theorem it stays ≥ 1; the same for $\det D$. **Every element of $\mathrm{Spin}_0(4,4)$ has the sign pattern $(+, +)$.**

(H) Four pieces. $\Lambda(\gamma^{(x0)}) = R_{e_{x0}}$ reverses only $x0$: pattern $(-, +)$. $\Lambda(\gamma^{(x4)})$ reverses only $x4$: $(+, -)$. $\Lambda(\gamma^{(x0)}\gamma^{(x4)})$ reverses both: $(-, -)$. For h in $\mathrm{Spin}_0(4,4)$ and w one of these, $\Lambda(hw) = \Lambda(h) \Lambda(w)$, and $\Lambda(w)$ is a diagonal matrix of signs: it only changes the sign of some columns of $\Lambda(h)$, so the space block of hw is A times a diagonal matrix of signs and hw has the pattern of w. Different patterns, so the four sets Spin_0, $\mathrm{Spin}_0\gamma^{(x0)}$, $\mathrm{Spin}_0\gamma^{(x4)}$, $\mathrm{Spin}_0\gamma^{(x0)}\gamma^{(x4)}$ have no element in common. In particular $\gamma^{(x0)}\gamma^{(x4)}$ lies in Spin(4,4) but is **not** a product of exponentials.

(I) Every unit vector is a turned basic one. Let v be a space-like unit vector, with space part σ (components $x0, x1, x2, x3$) and time part τ (components $x4, \dots, x7$), so $|\sigma|^2 - |\tau|^2 = 1$. Three stages carry e_{x0} to v: a boost in the plane $(x0, x4)$ gives $|\sigma|e_{x0} + |\tau|e_{x4}$; three rotations in the planes $(x4, x5)$, $(x4, x6)$, $(x4, x7)$ turn e_{x4} into $\tau/|\tau|$ and do not touch $x0$; three rotations in the planes $(x0, x1)$, $(x0, x2)$, $(x0, x3)$ turn e_{x0} into $\sigma/|\sigma|$ and do not touch the time part. The result is $\sigma + \tau = v$. The angles follow from (E). For an even h, $h\gamma^{(x0)}h^{-1} = \gamma(\Lambda(h)e_{x0}) = \gamma(v)$. A time-like unit vector v has $|\tau|^2 - |\sigma|^2 = 1$; three stages, with the roles of the two blocks exchanged, carry e_{x4} to v: a boost in the plane $(x4, x0)$ gives $|\tau|e_{x4} + |\sigma|e_{x0}$ (by (E) with $a = x4$, $\eta_{aa} = -1$: $\sinh\theta = |\sigma|$, so $\cosh\theta = \sqrt{1 + |\sigma|^2} = |\tau|$); three rotations in the planes $(x0, x1)$, $(x0, x2)$, $(x0, x3)$ turn e_{x0} into $\sigma/|\sigma|$ and do not touch $x4$; three rotations in the planes $(x4, x5)$, $(x4, x6)$, $(x4, x7)$ turn e_{x4} into $\tau/|\tau|$ and do not touch the space part. The result is again $\sigma + \tau = v$, and $h\gamma^{(x4)}h^{-1} = \gamma(\Lambda(h)e_{x4}) = \gamma(v)$. When the boost puts nothing on the other axis ($\tau = 0$ for a space-like, $\sigma = 0$ for a time-like v), $\theta = 0$ and the middle stage is left out.

(J) Moving the gammas to the right. Moving γ^e through $\gamma^a \gamma^b$ costs $(-1)^2 = +1$ when e is neither a nor b, and -1 when e is one of them. So $\gamma^e S^{ab}(\gamma^e)^{-1} = sS^{ab}$ with that sign s, and, term by term in the power series, $\gamma^e\exp(\theta S^{ab}) = \exp(s\theta S^{ab})\gamma^e$. A product of unit vectors, each written as in (I), becomes a product of exponentials followed by a word in $\gamma^{(x0)}$ and $\gamma^{(x4)}$; with $\gamma^{(x0)}\gamma^{(x0)} = 1$, $\gamma^{(x4)}\gamma^{(x4)} = -1$, $\gamma^{(x4)}\gamma^{(x0)} = -\gamma^{(x0)}\gamma^{(x4)}$ and $-1 = \exp(2\pi S^{(x1)(x2)})$, the word becomes one of 1, $\gamma^{(x0)}$, $\gamma^{(x4)}$, $\gamma^{(x0)}\gamma^{(x4)}$.

```
**The answer.** By (D) the exponentials, and by definition $\gamma^{(x0)}$ and
$\gamma^{(x4)}$, lie in Pin(4,4); by (I) and (J) every element of Pin(4,4) is a
product of them; by (G) and (H) the four pieces are different. So

$$\mathrm{Pin}(4,4) = \mathrm{Spin}_0 \cup \mathrm{Spin}_0\gamma^{(x0)} \cup
  \mathrm{Spin}_0\gamma^{(x4)} \cup \mathrm{Spin}_0\gamma^{(x0)}\gamma^{(x4)} ,$$

and $\mathrm{Spin}(4,4)$ is the union of the first and the last piece. The
exponentials of the scaled commutators generate exactly $\mathrm{Spin}_0(4,4)$;
together with $\gamma^{(x0)}$ and $\gamma^{(x4)}$ they generate all of Pin(4,4).
```

Text cell:

```
## 5. The gammas and the recorded checks

The next cell reads the gammas (exact whole numbers) from the record
‘Revision/algebra/gammas.json‘ and the two algebra reports. It defines the lists
‘SPACE‘ and ‘TIME‘ of the four space-like and the four time-like directions, the
$8 \times 8$ metric ‘ETA_MATRIX‘, and the helpers ‘recorded(key, name)‘ (true when
the report ‘key‘, "python" or "wolfram", holds the check ‘name‘ with the verdict
pass), ‘record_of(key, name)‘ (the text printed after "reproduces") and
‘check_reproduces(condition, name, record)‘, the helper ‘check‘ for a check that
reproduces a Revision record: it lets ‘check‘ print the PASS line and the line
"reproduces ..." into a text buffer (‘contextlib.redirect_stdout‘) and sends both
lines with one ‘sys.stdout.write‘, because Jupyter delivers printed text in pieces
and one piece keeps the two lines together for the tools that read the notebook.
Then it checks the Clifford relation.
```

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
ETA_MATRIX = np.diag([float(ETA[x]) for x in COORDS])  # the 8 x 8 metric eta
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the 16 x 16 identity matrix 1
SPACE = ["x0", "x1", "x2", "x3"]  # the four space-like directions
TIME = ["x4", "x5", "x6", "x7"]  # the four time-like directions

REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
```

```
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    """True when the report key records the check name with the verdict pass."""
    return VERDICTS[(key, name)][0] == "pass"


def record_of(key, name):
    """The text printed after "reproduces": the record file and the check name."""
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


def eta(a, b):
    """eta^ab of the frame metric: eta_aa when a = b, 0 otherwise."""
    return ETA[a] if a == b else 0


check_reproduces(all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                                    2 * eta(a, b) * I16) for a in COORDS for b in COORDS)
                 and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

Out [2]:

```
PASS {gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
```

Text cell:

```
## 6. The 28 scaled commutators are independent

The next cell builds $S^{ab} = \tfrac14[\gamma^a, \gamma^b]$ for all 64 ordered
pairs (so $S^{aa} = 0$ and $S^{ba} = -S^{ab}$) as arrays of floating-point numbers;
their entries are $0$ and $\pm\tfrac12$, which the computer stores exactly. The
list ‘pairs‘ holds the 28 planes $(a, b)$ with $a$ before $b$ in the order
$x0, \dots, x7$. To test independence **exactly**, it writes each whole-number
matrix $2S^{ab} = \gamma^a\gamma^b$ as one row of 256 numbers and computes the rank
of the 28 rows with sympy’s ‘DomainMatrix‘ over the rational numbers ‘QQ‘
(Gaussian elimination with fractions): rank 28 means that no $S^{ab}$ is a
combination of the others.
```

In [3]:

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over QQ

S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4.0
     for a in COORDS for b in COORDS}  # all 64 ordered pairs; S^aa = 0
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 planes


def exact_rank(rows):
    """The rank of a list of rows of whole numbers, computed exactly over QQ."""
    whole = [[int(round(x)) for x in row] for row in rows]  # entries 0, +1, -1
    return DomainMatrix.from_list(whole, sp.QQ).rank()


rank_S = exact_rank([(gamma[a] @ gamma[b]).reshape(256) for a, b in pairs])
entries = sorted({float(v) for p in pairs for v in S[p].flat})
say(f"{len(pairs)} generators; their entries are {entries}; exact rank {rank_S}")
check_reproduces(rank_S == 28 and entries == [-0.5, 0.0, 0.5]
                 and all(np.array_equal(S[(a, b)], gamma[a] @ gamma[b] / 2.0)
                         for a, b in pairs)
                 and recorded("python", "S_definition"),
                 "the 28 S^ab = (1/2) gamma^a gamma^b are linearly independent "
                 "(exact rank 28)",
                 record=record_of("python", "S_definition"))
```

Out [3]:

```
28 generators; their entries are [-0.5, 0.0, 0.5]; exact rank 28
PASS the 28 S^ab = (1/2) gamma^a gamma^b are linearly independent (exact rank 28)
     reproduces Revision/algebra/reports/python-algebra.json, check S_definition
```

Text cell:

```
## 7. How the scaled commutators move the directions: a copy of so(4,4)

The next cell builds, for all 64 ordered pairs, the $8 \times 8$ matrix $M^{ab}$ of
section 4 (A), rows $d$ and columns $c$ in the order $x0, \dots, x7$, and checks
three things:

1. the vector rule $[S^{ab}, \gamma^c] = \sum_d M^{ab}_{dc}\gamma^d$ for the 28
   planes and the 8 directions $c$ (a recorded fact);
2. every $M^{ab}$ obeys $(M^{ab})^T\eta + \eta M^{ab} = 0$; the 28 of them are
   independent (exact rank 28); and the space so(4,4) has dimension exactly 28:
   the condition $X^T\eta + \eta X = 0$ is a system of 64 linear equations for the
   64 entries of an unknown $X$, with $(\eta X)_{rc} = \eta_{rr}X_{rc}$ and
   $(X^T\eta)_{rc} = X_{cr}\eta_{cc}$, whose solutions form a space of dimension
   64 minus the rank. So the $M^{ab}$ are a **basis** of so(4,4);
3. the commutators of the $M^{ab}$ follow the same table as those of the
   $S^{ab}$, $[X^{ab}, X^{cd}] = \eta^{bc}X^{ad} - \eta^{ac}X^{bd} -
   \eta^{bd}X^{ac} + \eta^{ad}X^{bc}$, for all $28 \times 28 = 784$ pairs (for the
   $S^{ab}$ a recorded fact).
```

In [4]:

```
def generator_matrix(a, b):
    """The 8 x 8 matrix M^ab with [S^ab, gamma^c] = sum over d of M_dc gamma^d:
    M_dc = eta^bc delta_da - eta^ac delta_db (rows d, columns c: x0, ..., x7)."""
    result = np.zeros((8, 8))
    for j, c in enumerate(COORDS):
        for i, d in enumerate(COORDS):
            result[i, j] = eta(b, c) * (d == a) - eta(a, c) * (d == b)
    return result


M = {(a, b): generator_matrix(a, b) for a in COORDS for b in COORDS}  # all 64
vector_ok = all(
    np.array_equal(S[p] @ gamma[c] - gamma[c] @ S[p],
                   sum(M[p][i, j] * gamma[d] for i, d in enumerate(COORDS)))
    for p in pairs for j, c in enumerate(COORDS))
check_reproduces(vector_ok and recorded("python", "S_vector_action")
                 and recorded("wolfram", "S_gamma_commutator"),
                 "[S^ab, gamma^c] = sum_d M^ab_dc gamma^d with M_dc = eta^bc delta_da "
                 "- eta^ac delta_db",
                 record=record_of("python", "S_vector_action"))

in_so44 = all(not (M[p].T @ ETA_MATRIX + ETA_MATRIX @ M[p]).any() for p in pairs)
rank_M = exact_rank([M[p].reshape(64) for p in pairs])
equations = []  # the 64 equations (X^T eta + eta X)_rc = 0 for the entries of X
for r in range(8):
    for c in range(8):
        row = np.zeros(64)
        row[r * 8 + c] += ETA_MATRIX[r, r]  # (eta X)_rc = eta_rr X_rc
        row[c * 8 + r] += ETA_MATRIX[c, c]  # (X^T eta)_rc = X_cr eta_cc
        equations.append(row)
dimension_so44 = 64 - exact_rank(equations)
say(f"rank of the 28 matrices M^ab: {rank_M}; dimension of so(4,4): "
    f"{dimension_so44}")
check(in_so44 and rank_M == 28 and dimension_so44 == 28,
      "every M^ab lies in so(4,4), the 28 M^ab are independent and so(4,4) has "
      "dimension 28: the M^ab are a basis of so(4,4)")


def table_failures(X):
    """How many of the 784 pairs of planes break the commutation table for the
    64 matrices X[(a, b)]."""
    failures = 0
    for a, b in pairs:
        for c, d in pairs:
            right = (eta(b, c) * X[(a, d)] - eta(a, c) * X[(b, d)]
                     - eta(b, d) * X[(a, c)] + eta(a, d) * X[(b, c)])
            if not np.array_equal(X[(a, b)] @ X[(c, d)] - X[(c, d)] @ X[(a, b)],
                                  right):
                failures += 1
    return failures
```

```
failures_S, failures_M = table_failures(S), table_failures(M)
say(f"pairs that break the table: S^ab {failures_S}, M^ab {failures_M} (of 784)")
check_reproduces(failures_S == 0 and failures_M == 0
                 and recorded("python", "S_lorentz_algebra")
                 and recorded("wolfram", "S_Lorentz_algebra"),
                 "the S^ab and the M^ab have the same commutators: the S^ab span an "
                 "exact copy of so(4,4)",
                 record=record_of("python", "S_lorentz_algebra"))
```

Out [4]:

```
PASS [S^ab, gamma^c] = sum_d M^ab_dc gamma^d with M_dc = eta^bc delta_da - eta^ac
    delta_db
     reproduces Revision/algebra/reports/python-algebra.json, check S_vector_action
rank of the 28 matrices M^ab: 28; dimension of so(4,4): 28
PASS every M^ab lies in so(4,4), the 28 M^ab are independent and so(4,4) has dimension
    28: the M^ab are a basis of so(4,4)
pairs that break the table: S^ab 0, M^ab 0 (of 784)
PASS the S^ab and the M^ab have the same commutators: the S^ab span an exact copy of
    so(4,4)
     reproduces Revision/algebra/reports/python-algebra.json, check S_lorentz_algebra
```

Text cell:

```
The next cell checks the shape of the 28 matrices that section 4 (A) predicts
(exactly two nonzero entries; antisymmetric for the 12 rotations, symmetric for the
16 boosts) and draws them, one small heat map per plane.
```

In [5]:

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
shape_ok = all(
    np.count_nonzero(M[(a, b)]) == 2
    and np.array_equal(M[(a, b)].T, -ETA[a] * ETA[b] * M[(a, b)]) for a, b in pairs)
check(shape_ok, "each M^ab has two nonzero entries; antisymmetric for the 12 "
      "rotations, symmetric for the 16 boosts")
fig, axes = plt.subplots(4, 7, figsize=(14.0, 8.8))
numbers = [x[1] for x in COORDS]  # the tick labels 0, ..., 7
for ax, (a, b) in zip(axes.flat, pairs):
    ax.imshow(M[(a, b)], cmap=SIGNS, vmin=-1, vmax=1)
    kind = "rotation" if ETA[a] * ETA[b] == 1 else "boost"
    ax.set_title(f"$M^{{{a}\\,{b}}}$, {kind}", fontsize=9)
    ax.set_xticks(range(8), numbers, fontsize=6)
    ax.set_yticks(range(8), numbers, fontsize=6)
    ax.grid(False)
fig.suptitle("The 28 matrices $M^{ab}$: how $S^{ab}$ moves the directions "
             "$x0$ to $x7$ (rows and columns numbered 0 to 7)")
save_figure(fig, "generator_matrices",
            "The 28 eight by eight matrices $M^{ab}$ by which the scaled commutators "
```

```
"$S^{ab}$ move the eight directions, one picture per plane $(a, b)$; rows "
"and columns are the directions $x0$ to $x7$, labelled by their number; "
"red $+1$, blue $-1$, grey $0$. Each has exactly two nonzero entries, in "
"row $a$ column $b$ and in row $b$ column $a$: of opposite colour "
"(antisymmetric) for the 12 rotations, of the same colour (symmetric) for "
"the 16 boosts that mix a space-like and a time-like direction. These 28 "
"matrices are a basis of so(4,4).")
```

Out [5]:

```
PASS each M^ab has two nonzero entries; antisymmetric for the 12 rotations, symmetric for
    the 16 boosts
```

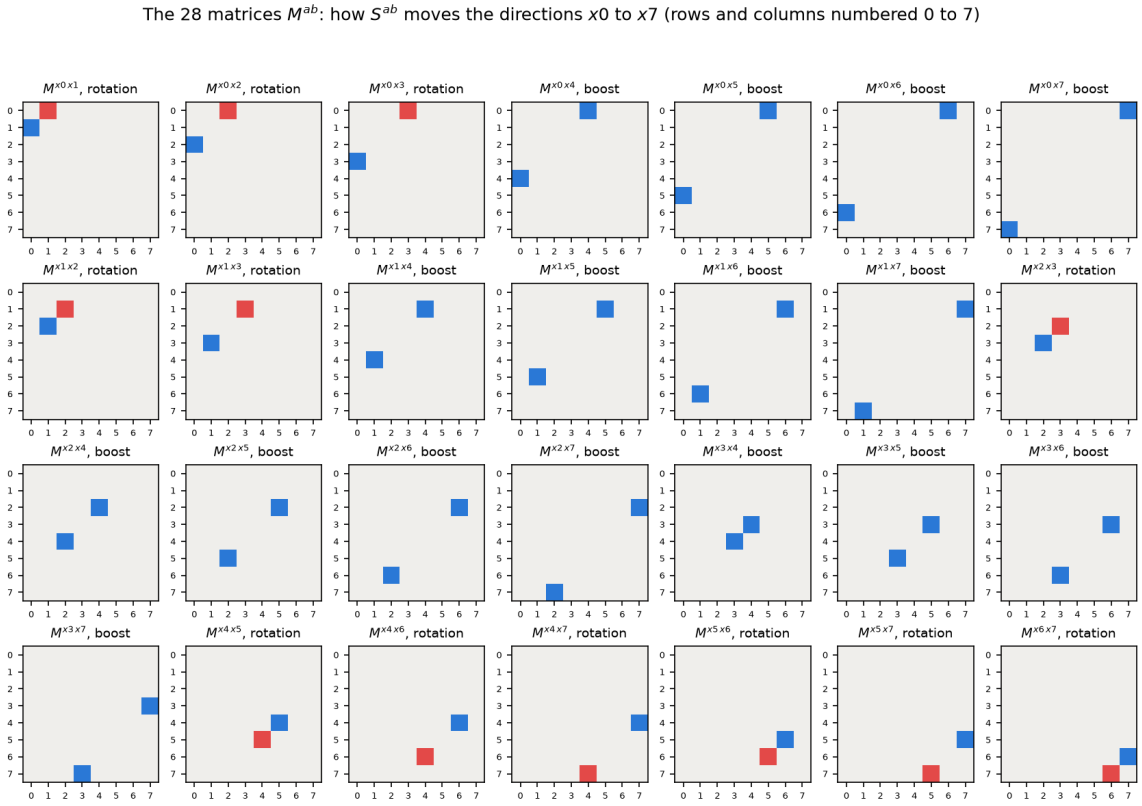

Figure 159: The 28 eight by eight matrices M^{ab} by which the scaled commutators S^{ab} move the eight directions, one picture per plane (a, b); rows and columns are the directions $x0$ to $x7$, labelled by their number; red $+1$, blue -1, grey 0. Each has exactly two nonzero entries, in row a column b and in row b column a: of opposite colour (antisymmetric) for the 12 rotations, of the same colour (symmetric) for the 16 boosts that mix a space-like and a time-like direction. These 28 matrices are a basis of so(4,4). (Notebook 05f, figure 1.)

Out [5]:

```
Figure 05f.1 saved as Revision/textbook/figures/05f_1_generator_matrices.png
```

Text cell:

```
## 8. Every exponential is a product of two unit vectors

The next cell defines ‘exponential(a, b, theta)‘, the closed formula of section 4
(D) for $\exp(\theta S^{ab})$, and compares it with the power series summed up to
60 terms for all 28 planes at $\theta = 0.9$. Then, for all 28 planes and the five
values $\theta = -2.5, -0.4, 0.9, 2.2, 5.0$, it forms the vector $u$ of section 4
(D) and checks $\exp(\theta S^{ab}) = \gamma^a\gamma(u)$ and $\eta(u, u) =
\eta_{aa}$. ‘gamma_of(v)‘ is $\gamma(v)$ and ‘metric_product(u, v)‘ is
$\eta(u, v)$; ‘E8[i]‘ is the basic unit vector of the direction ‘COORDS[i]‘.
```

In [6]:

```
def exponential(a, b, theta):
    """exp(theta S^ab) from the closed formula with half angles (any a != b)."""
    J = (gamma[a] @ gamma[b]).astype(float)  # J = 2 S^ab, J J = -eta_aa eta_bb
    if ETA[a] * ETA[b] == 1:  # J J = -1: a rotation
        return np.cos(theta / 2) * np.eye(16) + np.sin(theta / 2) * J
    return np.cosh(theta / 2) * np.eye(16) + np.sinh(theta / 2) * J  # a boost


def series_exponential(X, terms=60):
    """exp(X) = 1 + X + X^2/2 + ... summed up to X^(terms-1)/(terms-1)!."""
    result = np.eye(X.shape[0])
    term = np.eye(X.shape[0])
    for k in range(1, terms):
        term = term @ X / k  # X^k / k! from X^(k-1) / (k-1)!
        result = result + term
    return result


def gamma_of(v):
    """gamma(v) = sum over a of v_a gamma^a for a vector v of 8 numbers."""
    return sum(v[i] * gamma[x].astype(float) for i, x in enumerate(COORDS))


def metric_product(u, v):
    """eta(u, v) = sum over a of eta_aa u_a v_a."""
    return float(u @ ETA_MATRIX @ v)


def second_factor(a, b, theta):
    """The unit vector u with exp(theta S^ab) = gamma^a gamma(u) (section 4 D)."""
    i, j = COORDS.index(a), COORDS.index(b)
    if ETA[a] * ETA[b] == 1:  # a rotation
        return ETA[a] * np.cos(theta / 2) * E8[i] + np.sin(theta / 2) * E8[j]
    return ETA[a] * np.cosh(theta / 2) * E8[i] + np.sinh(theta / 2) * E8[j]


E8 = np.eye(8)  # E8[i] is the basic unit vector of the direction COORDS[i]
series_gap = max(np.max(np.abs(series_exponential(0.9 * S[p]) - exponential(*p, 0.9)))
```

```
                for p in pairs)
check(series_gap < 1e-10, "the closed formula equals the power series of "
      "exp(theta S^ab) for all 28 planes (difference below 1e-10)")
factor_ok = True
for a, b in pairs:
    for theta in [-2.5, -0.4, 0.9, 2.2, 5.0]:
        u = second_factor(a, b, theta)
        factor_ok &= (np.allclose(gamma[a] @ gamma_of(u), exponential(a, b, theta),
                                  rtol=0.0, atol=1e-10)
                      and abs(metric_product(u, u) - ETA[a]) < 1e-10)
check(factor_ok, "exp(theta S^ab) = gamma^a gamma(u) with eta(u, u) = eta_aa: a "
      "product of two unit vectors (28 planes, 5 values each)")
```

Out [6]:

```
PASS the closed formula equals the power series of exp(theta S^ab) for all 28 planes
    (difference below 1e-10)
PASS exp(theta S^ab) = gamma^a gamma(u) with eta(u, u) = eta_aa: a product of two unit
    vectors (28 planes, 5 values each)
```

Text cell:

```
The next cell draws where the second unit vector $u$ of the factorisation lies, in
the plane of its two nonzero components, for three planes: the rotation in
$(x1, x2)$ ($\theta$ from 0 to $4\pi$; $u$ runs once around the circle $u_1^2 +
u_2^2 = 1$), the boost in $(x1, x4)$ and the boost in $(x4, x0)$ ($\theta$ from
$-3$ to 3; $u$ runs along a branch of a hyperbola).
```

In [7]:

```
cases = [("x1", "x2", np.linspace(0.0, 4 * np.pi, 241)),
         ("x1", "x4", np.linspace(-3.0, 3.0, 241)),
         ("x4", "x0", np.linspace(-3.0, 3.0, 241))]
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.6))
norm_ok = True  # eta(u, u) = eta_aa along all three curves
for ax, (a, b, thetas) in zip(axes, cases):
    points = np.array([second_factor(a, b, th) for th in thetas])
    i, j = COORDS.index(a), COORDS.index(b)
    norms = [metric_product(p, p) for p in points]
    norm_ok &= max(abs(n - ETA[a]) for n in norms) < 1e-10
    ax.plot(points[:, i], points[:, j], color="#2a78d6", linewidth=2)
    marks = points[::40]  # every 40th value of theta
    ax.plot(marks[:, i], marks[:, j], "o", color="#eb6834", markersize=6)
    kind = "rotation" if ETA[a] * ETA[b] == 1 else "boost"
    ax.set_title(f"{kind} in $({a}, {b})$: $\\eta(u, u) = {norms[0]:+.0f}$")
    ax.set_xlabel(f"$u_{{{a}}}$")
    ax.set_ylabel(f"$u_{{{b}}}$")
    ax.set_aspect("equal")
    ax.set_xlim(-4.2, 4.2)
    ax.set_ylim(-4.2, 4.2)
check(norm_ok, "along the three curves eta(u, u) stays equal to eta_aa")
```

```
save_figure(fig, "two_unit_vectors",
            "The second factor $u$ in $\\exp(\\theta S^{ab}) = \\gamma^a\\gamma(u)$, "
            "drawn in the plane of its two nonzero components (axes: the components "
            "of $u$, pure numbers; orange dots every 40th value of $\\theta$). Left: "
            "the rotation in the plane $(x1, x2)$, $\\theta$ from $0$ to $4\\pi$; $u$ "
            "runs once around the unit circle. Middle and right: the boosts in the "
            "planes $(x1, x4)$ and $(x4, x0)$, $\\theta$ from $-3$ to $3$; $u$ runs "
            "along a branch of a hyperbola. In every case $\\eta(u, u)$ stays $+1$ or "
            "$-1$: each exponential is a product of two unit vectors.")
```

Out [7]:

```
PASS along the three curves eta(u, u) stays equal to eta_aa
```

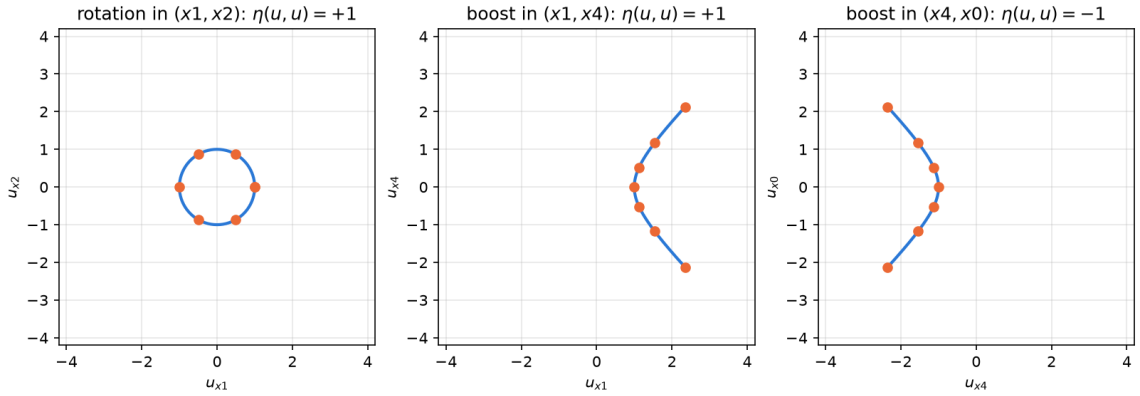

Figure 160: The second factor u in $\exp(\theta S^{ab}) = \gamma^a\gamma(u)$, drawn in the plane of its two nonzero components (axes: the components of u, pure numbers; orange dots every 40th value of θ). Left: the rotation in the plane $(x1, x2)$, θ from 0 to 4π; u runs once around the unit circle. Middle and right: the boosts in the planes $(x1, x4)$ and $(x4, x0)$, θ from -3 to 3; u runs along a branch of a hyperbola. In every case $\eta(u, u)$ stays $+1$ or -1: each exponential is a product of two unit vectors. (Notebook 05f, figure 2.)

Out [7]:

```
Figure 05f.2 saved as Revision/textbook/figures/05f_2_two_unit_vectors.png
```

Text cell:

```
## 9. The vector matrix of an exponential is exp(theta M)

The next cell defines `vector_matrix(g, odd)`, the vector matrix $\Lambda(g)$ of
the words list: it computes $\alpha(g)\gamma^cg^{-1}$ for each direction $c$ and
reads off $\Lambda_{dc} = \eta_{dd}\,\mathrm{tr}(\gamma^d\alpha(g)\gamma^cg^{-1})/16$
(because $\mathrm{tr}(\gamma^d\gamma^e) = 16\eta^{de}$). It checks, for all 28 planes
at $\theta = 0.9$, that $\Lambda(\exp(\theta S^{ab})) = \exp(\theta M^{ab})$ (the
matrix exponential summed as a power series), that it keeps the metric
($\Lambda^T\eta\Lambda = \eta$) and has determinant 1. Then it checks the turning
formulas of section 4 (E) for four planes used later: the rotations in
$(x0, x1)$ and $(x4, x5)$ and the boosts in $(x0, x4)$ and $(x4, x0)$.
```

In [8]:

```
def vector_matrix(g, odd=False):
    """Lambda(g) with alpha(g) gamma^c g^-1 = sum over d of Lambda_dc gamma^d, where
    alpha(g) = g for an even g and -g for an odd g (the twisted action)."""
    g_inverse = np.linalg.inv(g)
    sign = -1.0 if odd else 1.0
    result = np.zeros((8, 8))
    for j, c in enumerate(COORDS):
        moved = sign * (g @ gamma[c] @ g_inverse)  # alpha(g) gamma^c g^-1
        for i, d in enumerate(COORDS):
            result[i, j] = ETA[d] * np.trace(gamma[d] @ moved) / 16.0
    return result


exp_ok = True
for p in pairs:
    Lam = vector_matrix(exponential(*p, 0.9))
    exp_ok &= (np.allclose(Lam, series_exponential(0.9 * M[p]), rtol=0.0, atol=1e-12)
               and np.allclose(Lam.T @ ETA_MATRIX @ Lam, ETA_MATRIX, atol=1e-12)
               and abs(np.linalg.det(Lam) - 1.0) < 1e-12)
check(exp_ok, "the vector matrix of exp(theta S^ab) is exp(theta M^ab); it keeps "
      "the metric and has determinant 1 (28 planes)")


def turned_axis(a, b, theta):
    """Lambda e_a predicted by section 4 E for exp(theta S^ab)."""
    i, j = COORDS.index(a), COORDS.index(b)
    if ETA[a] * ETA[b] == 1:  # a rotation
        return np.cos(theta) * E8[i] - ETA[a] * np.sin(theta) * E8[j]
    return np.cosh(theta) * E8[i] - ETA[a] * np.sinh(theta) * E8[j]  # a boost


turn_ok = all(
    np.allclose(vector_matrix(exponential(a, b, th))[:, COORDS.index(a)],
                turned_axis(a, b, th), rtol=0.0, atol=1e-12)
    for a, b in [("x0", "x1"), ("x4", "x5"), ("x0", "x4"), ("x4", "x0")]
    for th in [-1.3, 0.4, 2.0])
check(turn_ok, "Lambda e_a = cos(theta) e_a - eta_aa sin(theta) e_b for a rotation "
      "and cosh(theta) e_a - eta_aa sinh(theta) e_b for a boost")
```

Out [8]:

```
PASS the vector matrix of exp(theta S^ab) is exp(theta M^ab); it keeps the metric and has
    determinant 1 (28 planes)
PASS Lambda e_a = cos(theta) e_a - eta_aa sin(theta) e_b for a rotation and cosh(theta)
    e_a - eta_aa sinh(theta) e_b for a boost
```

Text cell:

```
## 10. The forbidden band and the four pieces
```

The next cell defines ‘blocks(Lam)‘, which cuts a vector matrix into the space block A, the blocks B and C and the time block D (‘np.ix_‘ picks the rows and columns of the listed directions), and ‘random_factors(count)‘: ‘count‘ random factors (a, b, θ), each a random plane of the 28 with an angle between 0 and 2π for a rotation and a rapidity between -1 and 1 for a boost, drawn by the random-number generator with the fixed seed 12345 (so every run uses the same numbers). ‘compose(factors, t)‘ is the product $\exp(t\theta_n S_n)\cdots \exp(t\theta_1 S_1)$, in which the first factor of the list acts first. For 200 random products of six exponentials the cell checks the block equations $A^TA - C^TC = 1$ and $D^TD - B^TB = 1$ of section 4 (F) and that $\det A$ and $\det D$ are at least 1 (section 4 G).

In [9]:

```
SPACE_INDEX = [COORDS.index(x) for x in SPACE]  # the positions of x0, x1, x2, x3
TIME_INDEX = [COORDS.index(x) for x in TIME]  # the positions of x4, x5, x6, x7


def blocks(Lam):
    """The blocks A (space, space), B (space, time), C (time, space), D (time,
    time) of the 8 x 8 matrix Lam."""
    return (Lam[np.ix_(SPACE_INDEX, SPACE_INDEX)], Lam[np.ix_(SPACE_INDEX, TIME_INDEX)],
            Lam[np.ix_(TIME_INDEX, SPACE_INDEX)], Lam[np.ix_(TIME_INDEX, TIME_INDEX)])


rng = np.random.default_rng(12345)  # random numbers with a fixed seed


def random_factors(count):
    """count random factors (a, b, theta) of a product of exponentials."""
    factors = []
    for _ in range(count):
        a, b = pairs[int(rng.integers(28))]  # a random plane
        if ETA[a] * ETA[b] == 1:  # a rotation: an angle from 0 to 2 pi
            theta = rng.uniform(0.0, 2 * np.pi)
        else:  # a boost: a rapidity from -1 to 1
            theta = rng.uniform(-1.0, 1.0)
        factors.append((a, b, float(theta)))
    return factors


def compose(factors, t=1.0):
    """exp(t theta_n S_n) ... exp(t theta_1 S_1): the first factor acts first."""
    g = np.eye(16)
    for a, b, theta in factors:
        g = exponential(a, b, t * theta) @ g
    return g


block_ok, smallest = True, np.inf
for _ in range(200):
    A, B_, C_, D = blocks(vector_matrix(compose(random_factors(6))))
    block_ok &= (np.allclose(A.T @ A - C_.T @ C_, np.eye(4), atol=1e-9)
```

```
                and np.allclose(D.T @ D - B_.T @ B_, np.eye(4), atol=1e-9))
    smallest = min(smallest, np.linalg.det(A), np.linalg.det(D))
report("smallest det A or det D of 200 products of six exponentials",
       f"{smallest:.3f}")
check(block_ok and smallest > 1.0 - 1e-9,
      "200 products of exponentials: A^T A - C^T C = 1, D^T D - B^T B = 1, and "
      "det A >= 1, det D >= 1")
```

Out [9]:

```
RESULT smallest det A or det D of 200 products of six exponentials = 1.000
PASS 200 products of exponentials: A^T A - C^T C = 1, D^T D - B^T B = 1, and det A >= 1,
    det D >= 1
```

Text cell:

```
The next cell computes the vector matrices of the four words $1$, $\gamma^{(x0)}$,
$\gamma^{(x4)}$ and $\gamma^{(x0)}\gamma^{(x4)}$ (the gammas are odd, their
product even) and checks that they are the diagonal sign matrices of section 4
(H). Then, for each word $w$ and 60 random products $h$ of six exponentials, it
checks $\Lambda(hw) = \Lambda(h)\Lambda(w)$ and that $hw$ has the sign pattern of
$w$. It also records an observation that the picture shows but the proof does not
use: for each of the 240 elements $|\det A| = |\det D|$ (to $10^{-9}$). The next
cell draws all 240 points $(\det A, \det D)$. The axes are *symmetric
logarithmic*: linear between $-1$ and $1$ and logarithmic outside, so that the
forbidden band (shaded) is visible.
```

In [10]:

```
WORDS = {"1": (I16, False), "gamma^(x0)": (gamma["x0"], True),
         "gamma^(x4)": (gamma["x4"], True),
         "gamma^(x0) gamma^(x4)": (gamma["x0"] @ gamma["x4"], False)}


def pattern(Lam):
    """(sign of det A, sign of det D) of the vector matrix Lam."""
    A, _, _, D = blocks(Lam)
    return (int(np.sign(np.linalg.det(A))), int(np.sign(np.linalg.det(D))))


flip0 = np.diag([-1.0, 1, 1, 1, 1, 1, 1, 1])  # reverses x0 only (position 0)
flip4 = np.diag([1.0, 1, 1, 1, -1, 1, 1, 1])  # reverses x4 only (position 4)
expected = {"1": np.eye(8), "gamma^(x0)": flip0, "gamma^(x4)": flip4,
            "gamma^(x0) gamma^(x4)": flip0 @ flip4}
words_ok = all(np.allclose(vector_matrix(w.astype(float), odd), expected[name])
               for name, (w, odd) in WORDS.items())
for name, (w, odd) in WORDS.items():
    say(f"{name:22} sign pattern (sign det A, sign det D) = "
        f"{pattern(vector_matrix(w.astype(float), odd))}")
check(words_ok, "the vector matrices of 1, gamma^(x0), gamma^(x4), gamma^(x0) "
      "gamma^(x4) are the identity and the reversals of x0, x4 and both")
```

```
points = {}  # word -> list of (det A, det D)
pieces_ok = True
for name, (w, odd) in WORDS.items():
    points[name] = []
    Lam_w = vector_matrix(w.astype(float), odd)
    for _ in range(60):
        h = compose(random_factors(6))
        Lam = vector_matrix(h @ w, odd)
        pieces_ok &= (np.allclose(Lam, vector_matrix(h) @ Lam_w, atol=1e-9)
                      and pattern(Lam) == pattern(Lam_w))
        A, _, _, D = blocks(Lam)
        points[name].append((np.linalg.det(A), np.linalg.det(D)))
check(pieces_ok, "for 4 x 60 elements h w: Lambda(h w) = Lambda(h) Lambda(w), and h w "
      "has the sign pattern of w")
# An observation that the picture below shows and that the proof does not need:
# for every one of the 240 elements the two determinants have the same size.
equal_size = all(abs(abs(dA) - abs(dD)) < 1e-9 * max(abs(dA), 1.0)
                 for name in WORDS for dA, dD in points[name])
check(equal_size, "observed for all 240 elements: |det A| = |det D|")
```

Out [10]:

```
1                     sign pattern (sign det A, sign det D) = (1, 1)
gamma^(x0)            sign pattern (sign det A, sign det D) = (-1, 1)
gamma^(x4)            sign pattern (sign det A, sign det D) = (1, -1)
gamma^(x0) gamma^(x4)  sign pattern (sign det A, sign det D) = (-1, -1)
PASS the vector matrices of 1, gamma^(x0), gamma^(x4), gamma^(x0) gamma^(x4) are the
    identity and the reversals of x0, x4 and both
PASS for 4 x 60 elements h w: Lambda(h w) = Lambda(h) Lambda(w), and h w has the sign
    pattern of w
PASS observed for all 240 elements: |det A| = |det D|
```

Text cell:

```
The next cell draws the 240 points.
```

In [11]:

```
colors = {"1": "#2a78d6", "gamma^(x0)": "#eb6834", "gamma^(x4)": "#1baf7a",
          "gamma^(x0) gamma^(x4)": "#4a3aa7"}
labels = {"1": r"$h$ (products of exponentials)",
          "gamma^(x0)": r"$h\gamma^{(x0)}$",
          "gamma^(x4)": r"$h\gamma^{(x4)}$",
          "gamma^(x0) gamma^(x4)": r"$h\gamma^{(x0)}\gamma^{(x4)}$"}
fig, ax = plt.subplots(figsize=(7.4, 6.6))
ax.axvspan(-1.0, 1.0, color="#d8d6d2", alpha=0.6, linewidth=0)
ax.axhspan(-1.0, 1.0, color="#d8d6d2", alpha=0.6, linewidth=0)
for name in WORDS:
    values = np.array(points[name])
    ax.plot(values[:, 0], values[:, 1], "o", color=colors[name], markersize=5,
            label=labels[name])
```

```
ax.set_xscale("symlog", linthresh=1.0)
ax.set_yscale("symlog", linthresh=1.0)
ax.set_xlim(-60, 60)
ax.set_ylim(-60, 60)
ax.text(0.0, 0.0, "forbidden band:\n$|\\det| < 1$", ha="center", va="center",
        fontsize=8)
ax.set_xlabel("$\\det A$ (space block)")
ax.set_ylabel("$\\det D$ (time block)")
ax.set_title("The four pieces of Pin(4,4)")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.12), ncol=2)
save_figure(fig, "four_pieces",
            "The determinants of the space block $A$ (horizontal axis) and of the time "
            "block $D$ (vertical axis) of the vector matrices of 240 elements of "
            "Pin(4,4): 60 random products $h$ of six exponentials (blue) and $h$ "
            "times $\\gamma^{(x0)}$ (orange), $\\gamma^{(x4)}$ (green) and "
            "$\\gamma^{(x0)}\\gamma^{(x4)}$ (purple). Both axes are linear between "
            "$-1$ and $1$ and logarithmic outside; the grey bands $|\\det| < 1$ are "
            "forbidden for every element. Each kind fills one of the four corners: "
            "the products of exponentials never leave the corner where both "
            "determinants are at least 1. The points lie on the diagonals because "
            "$|\\det A| = |\\det D|$ for every element computed here.")
```

Out [11]:

```
Figure 05f.3 saved as Revision/textbook/figures/05f_3_four_pieces.png
```

Text cell:

```
## 11. Paths from 1 never cross the band

The next cell follows ten products $h$ of six exponentials along the path $h(t)$
of section 4 (G), $t = 0, 0.01, \dots, 1$, and computes $\det A(t)$; it does the
same for the odd elements $\gamma^{(x0)}h(t)$. The first family starts at $+1$ and
must stay at or above $+1$; the second starts at $-1$ and must stay at or below
$-1$. Neither can cross the band, which is why $\gamma^{(x0)}$ is not a product of
exponentials.
```

In [12]:

```
path_t = np.linspace(0.0, 1.0, 101)  # t = 0, 0.01, ..., 1
curves_h, curves_g0 = [], []  # det A(t) along h(t) and along gamma^(x0) h(t)
g0 = gamma["x0"].astype(float)
for _ in range(10):
    factors = random_factors(6)
    along_h, along_g0 = [], []
    for t in path_t:
        h = compose(factors, t)
        along_h.append(np.linalg.det(blocks(vector_matrix(h))[0]))
        along_g0.append(np.linalg.det(blocks(vector_matrix(g0 @ h, odd=True))[0]))
    curves_h.append(along_h)
    curves_g0.append(along_g0)
```

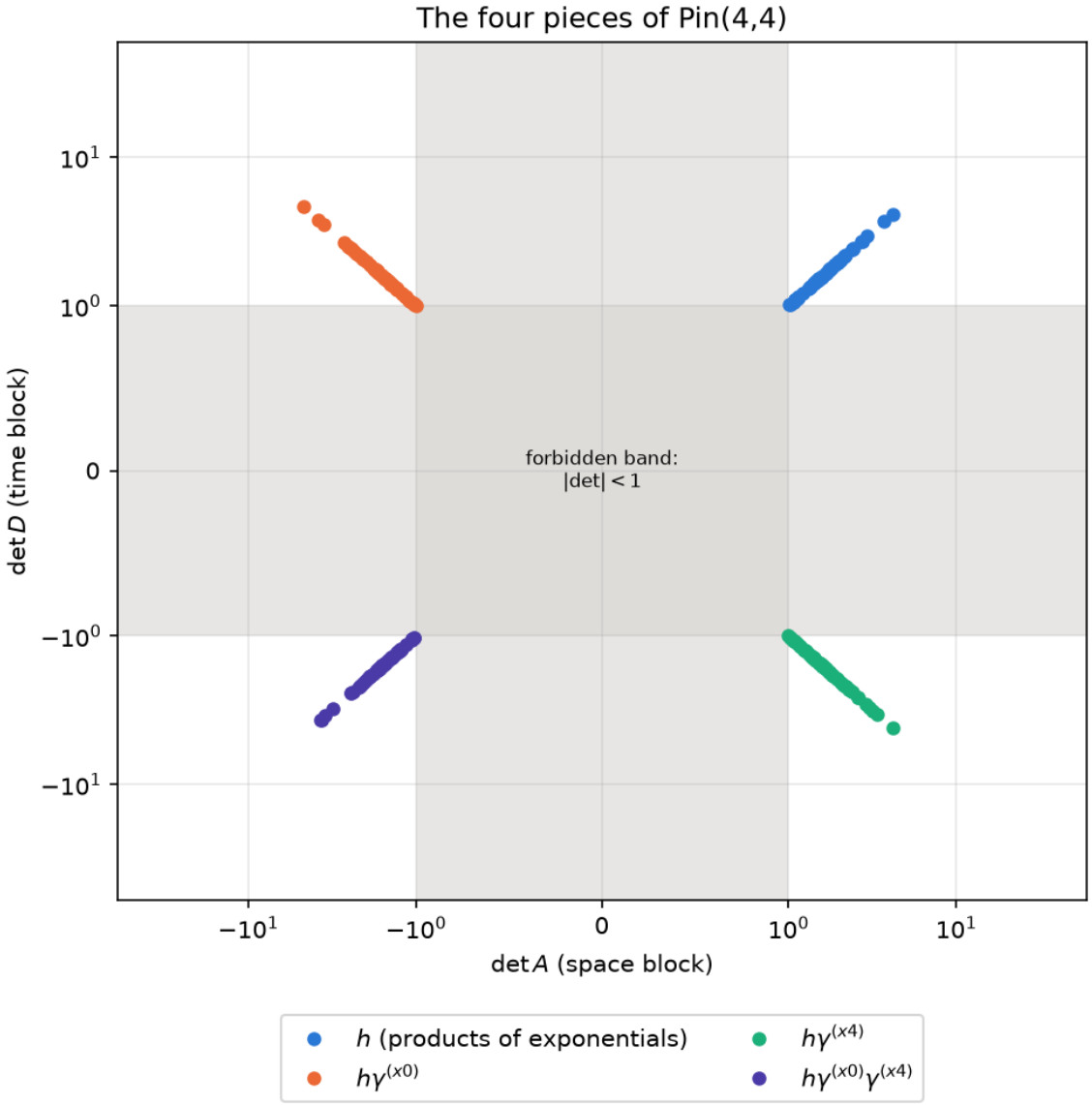

Figure 161: The determinants of the space block A (horizontal axis) and of the time block D (vertical axis) of the vector matrices of 240 elements of Pin(4,4): 60 random products h of six exponentials (blue) and h times $\gamma^{(x0)}$ (orange), $\gamma^{(x4)}$ (green) and $\gamma^{(x0)}\gamma^{(x4)}$ (purple). Both axes are linear between -1 and 1 and logarithmic outside; the grey bands $|\det| < 1$ are forbidden for every element. Each kind fills one of the four corners: the products of exponentials never leave the corner where both determinants are at least 1. The points lie on the diagonals because $|\det A| = |\det D|$ for every element computed here. (Notebook 05f, figure 3.)

```
curves_h, curves_g0 = np.array(curves_h), np.array(curves_g0)
check(np.all(curves_h > 1.0 - 1e-9) and np.all(curves_g0 < -1.0 + 1e-9)
      and np.allclose(curves_h[:, 0], 1.0) and np.allclose(curves_g0[:, 0], -1.0),
      "along ten paths t -> h(t) det A stays >= 1, and along gamma^(x0) h(t) it "
      "stays <= -1")
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.axhspan(-1.0, 1.0, color="#d8d6d2", alpha=0.6, linewidth=0)
for k in range(10):
    ax.plot(path_t, curves_h[k], color="#2a78d6", linewidth=1.5,
            label=r"$h(t)$" if k == 0 else None)
    ax.plot(path_t, curves_g0[k], color="#eb6834", linewidth=1.5, linestyle="--",
            label=r"$\gamma^{(x0)}h(t)$" if k == 0 else None)
ax.set_yscale("symlog", linthresh=1.0)
ax.set_ylim(-60, 60)
ax.text(0.5, 0.0, "forbidden band $|\\det A| < 1$", ha="center", va="center")
ax.set_xlabel("path parameter $t$ (from 1 at $t = 0$ to $h$ at $t = 1$)")
ax.set_ylabel("$\\det A$ (space block)")
ax.set_title("Ten paths of products of exponentials, and the same times "
             "$\\gamma^{(x0)}$")
ax.legend(loc="upper left")
save_figure(fig, "paths_and_band",
            "The determinant of the space block $A$ of the vector matrix along ten "
            "paths $h(t)$ from the identity ($t = 0$) to a random product $h$ of six "
            "exponentials ($t = 1$), solid blue, and along the odd elements "
            "$\\gamma^{(x0)}h(t)$, dashed orange; horizontal axis $t$, vertical axis "
            "$\\det A$ (linear between $-1$ and $1$, logarithmic outside). The blue "
            "curves start at $1$ and the orange at $-1$; the grey band $|\\det A| < "
            "1$ is forbidden for every element, so no curve can cross it: "
            "$\\gamma^{(x0)}$ is not joined to $1$ and is not a product of "
            "exponentials.")
```

Out [12]:

```
PASS along ten paths t -> h(t) det A stays >= 1, and along gamma^(x0) h(t) it stays <= -1
```

Out [12]:

```
Figure 05f.4 saved as Revision/textbook/figures/05f_4_paths_and_band.png
```

Text cell:

```
## 12. Every unit vector is a turned gamma^(x0) or gamma^(x4)

The next cell implements the construction of section 4 (I).
‘turn_factor(a, b, along_a, along_b)‘ returns the factor $(a, b, \theta)$ whose
vector matrix carries $e_a$ to the direction of $\text{along}_a e_a +
\text{along}_b e_b$: from (E), for a rotation $\theta =
\mathrm{atan2}(-\eta_{aa}\text{along}_b, \text{along}_a)$ (‘np.arctan2(y, x)‘ is
the angle of the point $(x, y)$, so $\cos\theta$ and $\sin\theta$ are proportional
to $x$ and $y$), and for a boost (which needs $\text{along}_a^2 -
\text{along}_b^2 = 1$) $\theta = \mathrm{arcsinh}(-\eta_{aa}\text{along}_b)$.
```

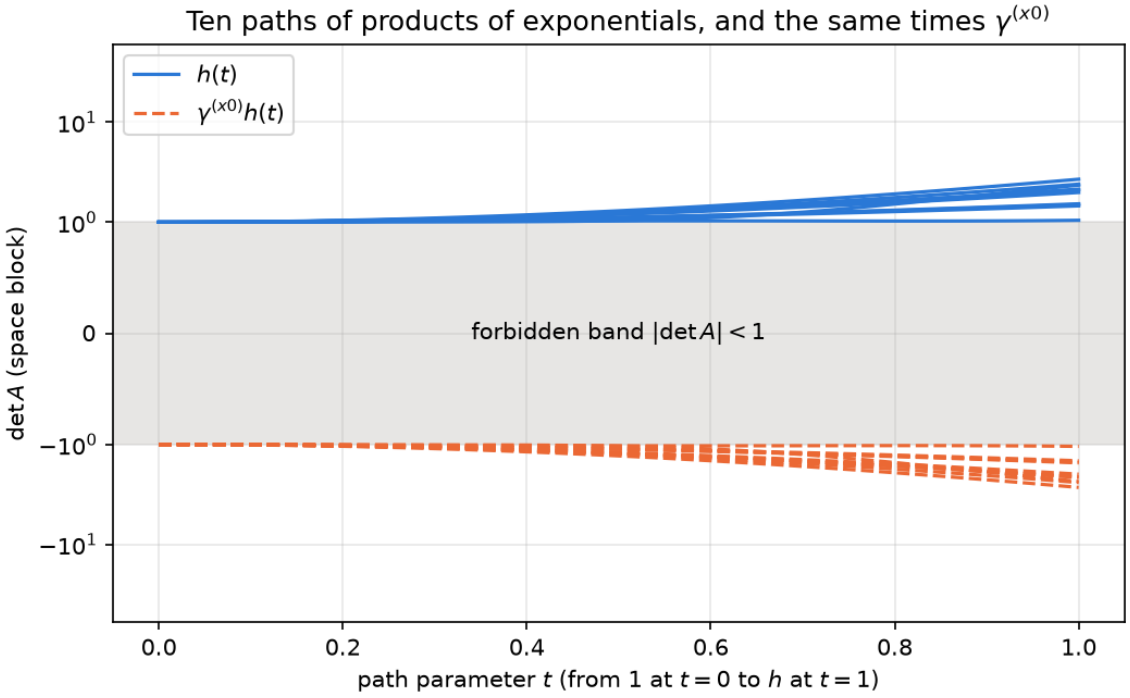

Figure 162: The determinant of the space block A of the vector matrix along ten paths $h(t)$ from the identity ($t = 0$) to a random product h of six exponentials ($t = 1$), solid blue, and along the odd elements $\gamma^{(x0)}h(t)$, dashed orange; horizontal axis t, vertical axis $\det A$ (linear between -1 and 1, logarithmic outside). The blue curves start at 1 and the orange at -1; the grey band $|\det A| < 1$ is forbidden for every element, so no curve can cross it: $\gamma^{(x0)}$ is not joined to 1 and is not a product of exponentials. (Notebook 05f, figure 4.)

‘block_rotation(axis, others, n)‘ carries e_{axis} to the unit vector n of one block by three rotations: the first turns part of the axis into the first other direction, keeping on the axis the length $\sqrt{n_{\text{axis}}^2 + \text{(squares of the remaining components)}}$, and so on; the last leaves exactly n_{axis} (which may be negative). ‘carry(v)‘ returns the factors (the first acts first) and the starting direction, $x0$ for a space-like and $x4$ for a time-like v. The cell checks the construction on 40 random unit vectors: each is a random column of 8 numbers from the normal distribution, kept when $|\eta(w, w)| \geq 0.5$ and divided by $\sqrt{|\eta(w, w)|}$. The checks: $\Lambda(h)e = v$ and, as 16×16 matrices, $h\gamma^eh^{-1} = \gamma(v)$.

In [13]:

```
def turn_factor(a, b, along_a, along_b):
    """The factor (a, b, theta) whose vector matrix carries e_a to the direction of
    along_a e_a + along_b e_b (for a boost: along_a^2 - along_b^2 = 1)."""
    if ETA[a] * ETA[b] == 1:  # a rotation
        return (a, b, float(np.arctan2(-ETA[a] * along_b, along_a)))
    return (a, b, float(np.arcsinh(-ETA[a] * along_b)))  # a boost


def block_rotation(axis, others, n):
    """Three rotations, in the planes (axis, others[k]), that carry e_axis to the
    unit vector n of its block (n: direction -> component)."""
    factors = []
    for k, b in enumerate(others):
        if k < len(others) - 1:  # keep the length of the rest on the axis
            rest = np.sqrt(n[axis] ** 2 + sum(n[o] ** 2 for o in others[k + 1:]))
        else:  # the last rotation leaves exactly n_axis on the axis
            rest = n[axis]
        factors.append(turn_factor(axis, b, rest, n[b]))
    return factors


def carry(v):
    """(factors, e): a product h of exponentials (the first factor acts first)
    with Lambda(h) e_e = v; e = "x0" for a space-like, "x4" for a time-like v."""
    part = {x: float(v[COORDS.index(x)]) for x in COORDS}
    sigma = np.sqrt(sum(part[x] ** 2 for x in SPACE))  # length of the space part
    tau = np.sqrt(sum(part[x] ** 2 for x in TIME))  # length of the time part
    space_turn = block_rotation("x0", ["x1", "x2", "x3"],
                                {x: part[x] / sigma for x in SPACE}) if sigma else []
    time_turn = block_rotation("x4", ["x5", "x6", "x7"],
                               {x: part[x] / tau for x in TIME}) if tau else []
    if metric_product(v, v) > 0:  # space-like: from e_x0
        return [turn_factor("x0", "x4", sigma, tau)] + time_turn + space_turn, "x0"
    return [turn_factor("x4", "x0", tau, sigma)] + space_turn + time_turn, "x4"


def random_unit_vector():
    """A random unit vector (|eta(w, w)| >= 0.5 before it is divided)."""
    while True:
        w = rng.normal(size=8)  # eight numbers from the normal distribution
```

```
        q = metric_product(w, w)
        if abs(q) >= 0.5:
            return w / np.sqrt(abs(q))


starts = {"x0": 0, "x4": 0}  # how many space-like and time-like vectors
carry_ok, most_factors = True, 0
for _ in range(40):
    v = random_unit_vector()
    factors, e = carry(v)
    starts[e] += 1
    most_factors = max(most_factors, len(factors))
    h = compose(factors)
    carry_ok &= (np.allclose(vector_matrix(h)[:, COORDS.index(e)], v, atol=1e-12)
                 and np.allclose(h @ gamma[e] @ np.linalg.inv(h), gamma_of(v),
                                 atol=1e-12))
say(f"space-like vectors: {starts['x0']}; time-like vectors: {starts['x4']}; "
    f"at most {most_factors} exponentials per vector")
check(carry_ok and starts["x0"] > 0 and starts["x4"] > 0 and most_factors <= 7,
      "for 40 random unit vectors v: gamma(v) = h gamma^(e) h^-1 with h a product "
      "of at most 7 exponentials and e = x0 or x4")
```

Out [13]:

```
space-like vectors: 24; time-like vectors: 16; at most 7 exponentials per vector
PASS for 40 random unit vectors v: gamma(v) = h gamma^(e) h^-1 with h a product of at
    most 7 exponentials and e = x0 or x4
```

Text cell:

```
The next cell draws the construction at work for one space-like and one
time-like random unit vector: the eight components of the moving vector after each
of the seven exponentials (step 0 is the starting vector $e_{x0}$ or $e_{x4}$; the
dots at the right are the components of $v$). It also prints the seven factors of
the space-like example and checks that the metric product of the moving vector
stays $+1$ (or $-1$) at every step.
```

In [14]:

```
examples = [random_unit_vector() for _ in range(6)]
space_example = next(v for v in examples if metric_product(v, v) > 0)
time_example = next(v for v in examples if metric_product(v, v) < 0)
line_colors = ["#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948", "#eda100",
               "#52514e", "#9e9c98"]
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.6), sharey=True)
steps_ok = True
for ax, v in zip(axes, [space_example, time_example]):
    factors, e = carry(v)
    moving = [E8[COORDS.index(e)]]  # the vector after 0, 1, ..., 7 steps
    for a, b, theta in factors:
        moving.append(vector_matrix(exponential(a, b, theta)) @ moving[-1])
```

```
    moving = np.array(moving)
    steps_ok &= np.allclose([metric_product(m, m) for m in moving], ETA[e])
    steps_ok &= np.allclose(moving[-1], v, atol=1e-12)
    if e == "x0":
        for a, b, theta in factors:
            say(f"    factor exp(theta S^({a} {b})), theta = {theta:+.4f}")
    for i, x in enumerate(COORDS):
        ax.plot(range(len(moving)), moving[:, i], "o-", color=line_colors[i],
                linewidth=1.5, markersize=4, label=x)
        ax.plot([len(moving) - 0.6], [v[i]], "D", color=line_colors[i],
                markersize=6)
    kind = "space-like" if e == "x0" else "time-like"
    ax.set_title(f"a {kind} unit vector, built from $e_{{{e}}}$")
    ax.set_xticks(range(len(moving)))
    ax.set_xlabel("number of exponentials applied")
axes[0].set_ylabel("component of the moving vector")
axes[1].legend(loc="upper center", bbox_to_anchor=(-0.05, -0.15), ncol=8)
check(steps_ok, "in both examples the moving vector keeps eta(w, w) and ends at v")
save_figure(fig, "carrying_vectors",
            "The construction that carries a basic unit vector to a given unit vector "
            "$v$ by a product of exponentials: the eight components (one colour each, "
            "legend $x0$ to $x7$) of the moving vector after $0, 1, \\dots, 7$ "
            "exponentials (horizontal axis), for a random space-like $v$ built from "
            "$e_{x0}$ (left) and a random time-like $v$ built from $e_{x4}$ (right). "
            "The first step is a boost that shares the length between $x0$ and $x4$, "
            "the next three turn one block and the last three the other; the "
            "diamonds at the right are the components of $v$, which the last step "
            "reaches. Vertical axis: the components, pure numbers.")
```

Out [14]:

```
    factor exp(theta S^(x0 x4)), theta = -1.0497
    factor exp(theta S^(x4 x5)), theta = -0.8222
    factor exp(theta S^(x4 x6)), theta = -0.6507
    factor exp(theta S^(x4 x7)), theta = +1.7526
    factor exp(theta S^(x0 x1)), theta = -0.7158
    factor exp(theta S^(x0 x2)), theta = -0.7420
    factor exp(theta S^(x0 x3)), theta = -0.1038
PASS in both examples the moving vector keeps eta(w, w) and ends at v
```

Out [14]:

```
Figure 05f.5 saved as Revision/textbook/figures/05f_5_carrying_vectors.png
```

Text cell:

```
## 13. Every element of Pin(4,4) is a product of exponentials times one of four

The next cell checks the rule of section 4 (J): $\gamma^e\exp(\theta S^{ab})
(\gamma^e)^{-1} = \exp(s\theta S^{ab})$ with $s = -1$ when $e$ is $a$ or $b$ and
$s = +1$ otherwise, for $e = x0, x4$ and all 28 planes.
```

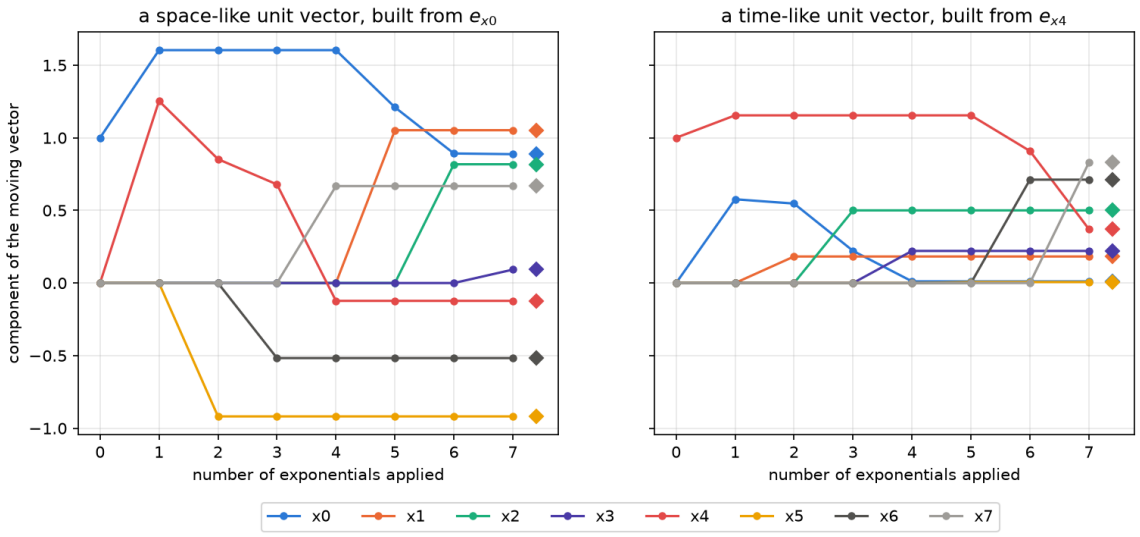

Figure 163: The construction that carries a basic unit vector to a given unit vector v by a product of exponentials: the eight components (one colour each, legend $x0$ to $x7$) of the moving vector after $0, 1, \ldots, 7$ exponentials (horizontal axis), for a random space-like v built from e_{x0} (left) and a random time-like v built from e_{x4} (right). The first step is a boost that shares the length between $x0$ and $x4$, the next three turn one block and the last three the other; the diamonds at the right are the components of v, which the last step reaches. Vertical axis: the components, pure numbers. (Notebook 05f, figure 5.)

In [15]:

```
def conjugation_sign(e, a, b):
    """The sign s with gamma^e exp(theta S^ab) (gamma^e)^-1 = exp(s theta S^ab)."""
    return -1 if e in (a, b) else 1


conjugation_ok = all(
    np.allclose(gamma[e] @ exponential(a, b, 0.8) @ np.linalg.inv(gamma[e]),
                exponential(a, b, conjugation_sign(e, a, b) * 0.8), atol=1e-12)
    for e in ("x0", "x4") for a, b in pairs)
check(conjugation_ok, "gamma^e exp(theta S^ab) (gamma^e)^-1 = exp(s theta S^ab), "
      "s = -1 if e is a or b, +1 otherwise")
```

Out [15]:

```
PASS gamma^e exp(theta S^ab) (gamma^e)^-1 = exp(s theta S^ab), s = -1 if e is a or b, +1
    otherwise
```

Text cell:

```
The next cell writes 12 random elements $g = \gamma(v_1)\cdots\gamma(v_k)$ of
Pin(4,4) ($k = 1, \dots, 6$, twice each) in the form of section 4. A *word* is a
```

list of items in the order of the matrix product, each an exponential `("exp", a, b, theta)` or a gamma `("gamma", e)`; `evaluate(word)` multiplies it out. `word_of(v)` is the word of $\gamma(v) = h\gamma^eh^{-1}$ (with h^{-1} the factors in the opposite order with the opposite angles). `move_gammas_right(word)` moves every gamma to the right end with the rule just checked, and `reduce_gammas` writes the remaining product of gammas as $\pm$ one of the four words; a sign -1 is replaced by the exponential $\exp(2\pi S^{(x1)(x2)}) = -1$. For each element the cell checks that the word equals g, that the final product of exponentials times the word w equals g (relative difference below 10^{-9}), and that g has the sign pattern of w.

In [16]:

```
FOUR = {"1": I16, "gamma^(x0)": gamma["x0"], "gamma^(x4)": gamma["x4"],
        "gamma^(x0) gamma^(x4)": gamma["x0"] @ gamma["x4"]}


def evaluate(word):
    """The matrix product of the items of the word, from left to right."""
    g = np.eye(16)
    for item in word:
        if item[0] == "exp":
            g = g @ exponential(*item[1:])
        else:
            g = g @ gamma[item[1]]
    return g


def word_of(v):
    """The word of gamma(v) = h gamma^e h^-1 (h from the construction)."""
    factors, e = carry(v)
    h_word = [("exp", a, b, th) for a, b, th in reversed(factors)]  # E_n ... E_1
    h_inverse = [("exp", a, b, -th) for a, b, th in factors]  # E_1^-1 ... E_n^-1
    return h_word + [("gamma", e)] + h_inverse


def move_gammas_right(word):
    """(exponentials, gammas) with word = exponentials followed by the gammas."""
    exponentials, waiting = [], []  # waiting: the gammas moved to the right so far
    for item in word:
        if item[0] == "gamma":
            waiting.append(item[1])
        else:  # this exponential is moved to the left of all waiting gammas
            _, a, b, theta = item
            s = 1
            for e in waiting:
                s *= conjugation_sign(e, a, b)
            exponentials.append(("exp", a, b, s * theta))
    return exponentials, waiting


def reduce_gammas(waiting):
    """(sign, name) with the product of the gammas = sign times FOUR[name]."""
```

```
    product = I16
    for e in waiting:
        product = product @ gamma[e]
    for name, w in FOUR.items():
        if np.array_equal(product, w):
            return 1, name
        if np.array_equal(product, -w):
            return -1, name
    raise ValueError("the product of the gammas is not one of the four words")


decomposition_ok = True
for number in range(12):
    k = 1 + number % 6  # the number of unit vectors
    vectors = [random_unit_vector() for _ in range(k)]
    g = np.eye(16)
    word = []
    for v in vectors:
        g = g @ gamma_of(v)
        word += word_of(v)
    size = np.max(np.abs(g))  # for relative differences
    exponentials, waiting = move_gammas_right(word)
    sign, name = reduce_gammas(waiting)
    if sign == -1:  # -1 = exp(2 pi S^(x1 x2))
        exponentials.append(("exp", "x1", "x2", 2 * np.pi))
    rebuilt = evaluate(exponentials) @ FOUR[name]
    same_pattern = (pattern(vector_matrix(g, odd=k % 2 == 1))
                    == pattern(vector_matrix(FOUR[name].astype(float), WORDS[name][1])))
    decomposition_ok &= (np.max(np.abs(evaluate(word) - g)) < 1e-9 * size
                         and np.max(np.abs(rebuilt - g)) < 1e-9 * size
                         and same_pattern and WORDS[name][1] == (k % 2 == 1))
    plural = "s" if k > 1 else " "  # "1 unit vector", "2 unit vectors"
    say(f"element {number + 1:2d}: {k} unit vector{plural} = {len(exponentials):2d} "
        f"exponentials times {name}")
check(decomposition_ok, "12 random elements of Pin(4,4): each is a product of "
      "exponentials times one of 1, gamma^(x0), gamma^(x4), gamma^(x0) gamma^(x4), "
      "with the matching sign pattern")
```

Out [16]:

```
element  1: 1 unit vector  = 14 exponentials times gamma^(x4)
element  2: 2 unit vectors = 29 exponentials times 1
element  3: 3 unit vectors = 43 exponentials times gamma^(x0)
element  4: 4 unit vectors = 56 exponentials times gamma^(x0) gamma^(x4)
element  5: 5 unit vectors = 70 exponentials times gamma^(x0)
element  6: 6 unit vectors = 84 exponentials times 1
element  7: 1 unit vector  = 14 exponentials times gamma^(x4)
element  8: 2 unit vectors = 29 exponentials times gamma^(x0) gamma^(x4)
element  9: 3 unit vectors = 43 exponentials times gamma^(x4)
element 10: 4 unit vectors = 57 exponentials times gamma^(x0) gamma^(x4)
element 11: 5 unit vectors = 71 exponentials times gamma^(x0)
element 12: 6 unit vectors = 85 exponentials times 1
PASS 12 random elements of Pin(4,4): each is a product of exponentials times one of 1,
```

```
    gamma^(x0), gamma^(x4), gamma^(x0) gamma^(x4), with the matching sign pattern
```

Text cell:

```
## 14. What the group elements span

The *span* of a group (all combinations of its elements) is closed under products,
because the product of two elements is an element. For $\mathrm{Spin}_0(4,4)$:
$\exp(\pi S^{ab}) = \gamma^a\gamma^b$ for a rotation ($\cos\tfrac\pi2 = 0$,
$\sin\tfrac\pi2 = 1$) and $(\exp(\theta S^{ab}) - \exp(-\theta S^{ab}))/
(2\sinh\tfrac\theta2) = \gamma^a\gamma^b$ for a boost, so the span contains every
$\gamma^a\gamma^b$, hence every product of an even number of gammas: all 128 even
Clifford products, which span the block-diagonal matrices. Every element is even,
so the span is exactly these 128 dimensions. With one gamma added, the span of
Pin(4,4) is all 256. The next cell checks the two formulas for all 28 planes and
then measures the rank of the first $n$ elements of two random lists, $n = 1, 2,
\dots$: 170 random products of six exponentials, and 300 random elements $hw$ with
$w$ running through the four words. The rank is computed from the *singular
values* (‘np.linalg.svd‘; a list of numbers whose count above a small threshold,
here $10^{-9}$ times the largest, is the rank).
```

In [17]:

```
formula_ok = all(
    np.allclose(exponential(a, b, np.pi), gamma[a] @ gamma[b], atol=1e-12)
    if ETA[a] * ETA[b] == 1 else
    np.allclose((exponential(a, b, 0.8) - exponential(a, b, -0.8))
                / (2 * np.sinh(0.4)), gamma[a] @ gamma[b], atol=1e-12)
    for a, b in pairs)
check(formula_ok, "exp(pi S^ab) = gamma^a gamma^b for the rotations and "
      "(exp(theta S^ab) - exp(-theta S^ab)) / (2 sinh(theta/2)) = gamma^a gamma^b "
      "for the boosts")
spin0_elements = [compose(random_factors(6)) for _ in range(170)]
four_words = list(FOUR.values())
pin_elements = [compose(random_factors(6)) @ four_words[k % 4] for k in range(300)]


def rank_curve(elements):
    """The rank of the first n elements, for n = 1, 2, ..., len(elements)."""
    ranks = []
    for n in range(1, len(elements) + 1):
        rows = np.array([g.reshape(256) for g in elements[:n]])
        singular = np.linalg.svd(rows, compute_uv=False)
        ranks.append(int(np.sum(singular > 1e-9 * singular[0])))
    return ranks


ranks_spin0, ranks_pin = rank_curve(spin0_elements), rank_curve(pin_elements)
block_diagonal = all(not g[:8, 8:].any() and not g[8:, :8].any()
                     for g in spin0_elements)
report("rank of 170 products of exponentials", ranks_spin0[-1])
report("rank of 300 elements of Pin(4,4)", ranks_pin[-1])
```

```
check_reproduces(ranks_spin0 == [min(n, 128) for n in range(1, 171)]
                 and block_diagonal
                 and recorded("python", "even_products_span_M8_plus_M8"),
                 "the products of exponentials are block diagonal and span 128 "
                 "dimensions, as the 128 even Clifford products",
                 record=record_of("python", "even_products_span_M8_plus_M8"))
check_reproduces(ranks_pin == [min(n, 256) for n in range(1, 301)]
                 and recorded("python", "clifford_products_span_M16"),
                 "the elements of Pin(4,4) span all 256 dimensions of the 16 x 16 "
                 "matrices",
                 record=record_of("python", "clifford_products_span_M16"))
```

Out [17]:

```
PASS exp(pi S^ab) = gamma^a gamma^b for the rotations and (exp(theta S^ab) - exp(-theta
    S^ab)) / (2 sinh(theta/2)) = gamma^a gamma^b for the boosts
RESULT rank of 170 products of exponentials = 128
RESULT rank of 300 elements of Pin(4,4) = 256
PASS the products of exponentials are block diagonal and span 128 dimensions, as the 128
    even Clifford products
     reproduces Revision/algebra/reports/python-algebra.json, check
    even_products_span_M8_plus_M8
PASS the elements of Pin(4,4) span all 256 dimensions of the 16 x 16 matrices
     reproduces Revision/algebra/reports/python-algebra.json, check
    clifford_products_span_M16
```

Text cell:

```
The next cell draws the two rank curves.
```

In [18]:

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.plot(range(1, 171), ranks_spin0, color="#2a78d6", linewidth=2.5,
        label=r"products of exponentials ($\mathrm{Spin}_0(4,4)$)")
ax.plot(range(1, 301), ranks_pin, color="#eb6834", linewidth=2, linestyle="--",
        label="elements of Pin(4,4) (all four pieces)")
ax.axhline(128, color="#2a78d6", linewidth=0.8, linestyle=":")
ax.axhline(256, color="#eb6834", linewidth=0.8, linestyle=":")
ax.set_xlabel("number $n$ of random group elements")
ax.set_ylabel("rank of the first $n$ elements")
ax.set_yticks([0, 64, 128, 192, 256])
ax.set_title("The span of the group elements")
ax.legend(loc="lower right")
save_figure(fig, "span_ranks",
            "The rank of the first $n$ of a list of random group elements (vertical "
            "axis) against $n$ (horizontal axis): 170 random products of six "
            "exponentials (solid blue) and 300 random elements of all four pieces of "
            "Pin(4,4) (dashed orange). Each new element adds one dimension until the "
            "rank stops at 128 for the products of exponentials (they span exactly "
            "the block-diagonal matrices) and at 256 for Pin(4,4) (all 16 by 16 "
```

```
"matrices), the two numbers recorded by the Revision record for the even "
"and for all Clifford products.")
```

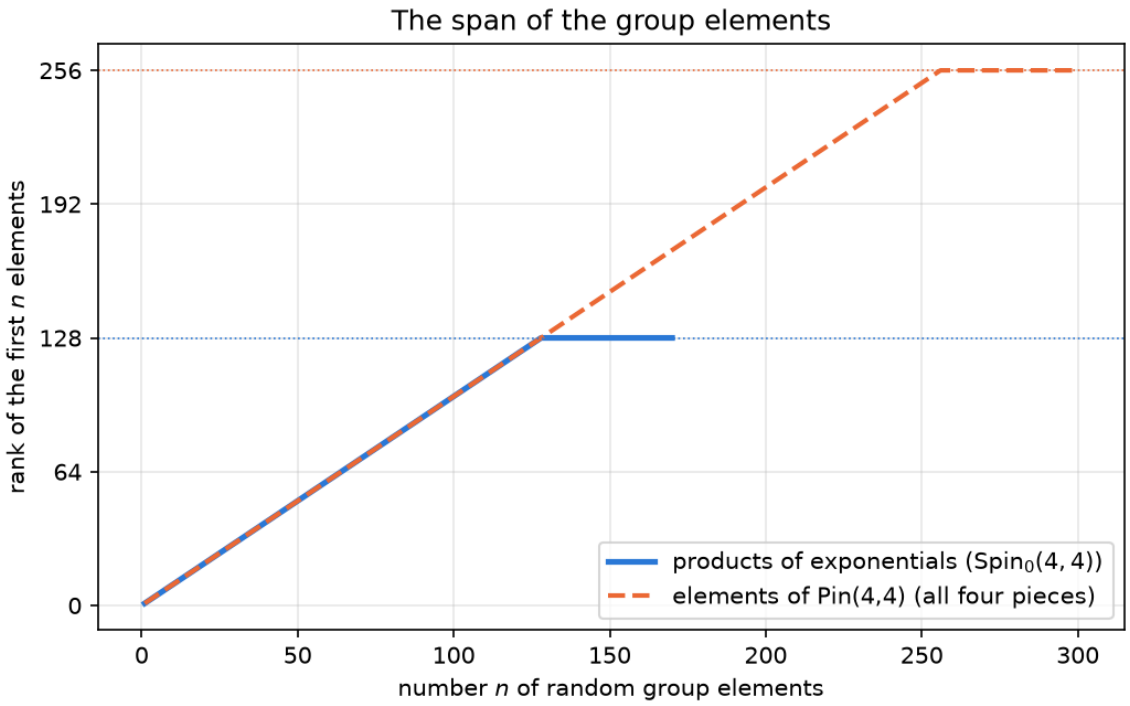

Figure 164: The rank of the first n of a list of random group elements (vertical axis) against n (horizontal axis): 170 random products of six exponentials (solid blue) and 300 random elements of all four pieces of Pin(4,4) (dashed orange). Each new element adds one dimension until the rank stops at 128 for the products of exponentials (they span exactly the block-diagonal matrices) and at 256 for Pin(4,4) (all 16 by 16 matrices), the two numbers recorded by the Revision record for the even and for all Clifford products. (Notebook 05f, figure 6.)

Out [18]:

```
Figure 05f.6 saved as Revision/textbook/figures/05f_6_span_ranks.png
```

Text cell:

```
## 15. The last check

The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [19]:

```
FIGURES = ["05f_1_generator_matrices.png", "05f_2_two_unit_vectors.png",
           "05f_3_four_pieces.png", "05f_4_paths_and_band.png",
```

```
          "05f_5_carrying_vectors.png", "05f_6_span_ranks.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the six figure files of notebook 05f exist")
all_checks_passed()
```

Out [19]:

```
PASS the six figure files of notebook 05f exist
ALL 24 CHECKS PASSED (notebook 05f)
```

Text cell:

```
## 16. What this notebook showed

- PROVED (section 4, line by line) and CHECKED on the matrices: the 28 scaled
  commutators $S^{ab} = \tfrac14[\gamma^a, \gamma^b]$ of the Revision gammas are
  independent; the matrices $M^{ab}$ by which they move the eight directions are
  a basis of so(4,4) (dimension 28) with the same commutators, so the $S^{ab}$ span
  an exact copy of so(4,4). The recorded checks S_definition, S_vector_action and
  S_lorentz_algebra are reproduced.
- PROVED and CHECKED: every $\exp(\theta S^{ab}) = \gamma^a\gamma(u)$ is a product
  of two unit vectors, so the exponentials lie in Spin(4,4); its vector matrix is
  $\exp(\theta M^{ab})$.
- PROVED: the space block and the time block of every vector matrix have
  determinants outside the band $(-1, 1)$; along a path from 1 they cannot change
  sign. So all products of exponentials, the group $\mathrm{Spin}_0(4,4)$, have
  the sign pattern $(+, +)$, while $\gamma^{(x0)}$, $\gamma^{(x4)}$ and
  $\gamma^{(x0)}\gamma^{(x4)}$ have $(-, +)$, $(+, -)$, $(-, -)$.
- PROVED by an explicit construction and CHECKED on 40 random unit vectors: every
  $\gamma(v)$ is $h\gamma^{(x0)}h^{-1}$ or $h\gamma^{(x4)}h^{-1}$ with $h$ a product
  of at most seven exponentials; CHECKED on 12 random elements: every element of
  Pin(4,4) is a product of exponentials times exactly one of $1$, $\gamma^{(x0)}$,
  $\gamma^{(x4)}$, $\gamma^{(x0)}\gamma^{(x4)}$.
- THE ANSWER: the exponentials of the scaled commutators generate exactly
  $\mathrm{Spin}_0(4,4)$, the part of Pin(4,4) joined to 1; adding the two gammas
  $\gamma^{(x0)}$ and $\gamma^{(x4)}$ generates all of Pin(4,4), which has four
  pieces, and Spin(4,4) is the union of the two even pieces. It is not correct to
  say that the scaled commutators alone generate Pin(4,4).
- COMPUTED: the elements of $\mathrm{Spin}_0(4,4)$ span the 128 block-diagonal
  matrices and those of Pin(4,4) all 256, the ranks recorded for the even and for
  all Clifford products.
```

5.27 Line-by-line walk-through of Notebook 05f

The notebook has 19 code cells. In [1] is the set-up cell of Section 5.10 with `NOTEBOOK_ID = "05f"`; its comments repeat the instructions of Section 5.25. Docstrings are left out of the quotations and long captions are replaced by a line `...`), as in Section 5.10.

In [2], the gammas and the recorded checks.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
ETA_MATRIX = np.diag([float(ETA[x]) for x in COORDS])  # the 8 x 8 metric eta
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the 16 x 16 identity matrix 1
SPACE = ["x0", "x1", "x2", "x3"]  # the four space-like directions
TIME = ["x4", "x5", "x6", "x7"]  # the four time-like directions
```

The gammas are read as in Notebook 05d; `ETA_MATRIX` is the 8×8 metric. New are the lists `SPACE` and `TIME` of the space-like and time-like directions, in the order used for the blocks of step (F).

```
REPORT_FILES = {"python": "Revision/algebra/reports/python-algebra.json",
                "wolfram": "Revision/algebra/reports/wolfram-algebra.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    return VERDICTS[(key, name)][0] == "pass"


def record_of(key, name):
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


def eta(a, b):
    return ETA[a] if a == b else 0


check_reproduces(all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                                    2 * eta(a, b) * I16) for a in COORDS for b in COORDS)
                 and recorded("python", "clifford_relation"),
                 "{gamma^a, gamma^b} = 2 eta^ab 1 for all 64 pairs",
                 record=record_of("python", "clifford_relation"))
```

The reports, the helpers and the Clifford check are those of Notebook 05d, In [2] (Section 5.22).

In [3], the 28 generators are independent.

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over QQ

S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4.0
     for a in COORDS for b in COORDS}  # all 64 ordered pairs; S^aa = 0
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 planes


def exact_rank(rows):
    whole = [[int(round(x)) for x in row] for row in rows]  # entries 0, +1, -1
    return DomainMatrix.from_list(whole, sp.QQ).rank()
```

`S` and `pairs` are those of Notebook 05d. `exact_rank` takes a list of rows whose entries are whole numbers stored as floating-point numbers, turns each entry into a Python whole number (`int(round(x))`), and computes the rank exactly over the rational numbers, as in Notebook 05b.

```
rank_S = exact_rank([(gamma[a] @ gamma[b]).reshape(256) for a, b in pairs])
entries = sorted({float(v) for p in pairs for v in S[p].flat})
say(f"{len(pairs)} generators; their entries are {entries}; exact rank {rank_S}")
check_reproduces(rank_S == 28 and entries == [-0.5, 0.0, 0.5]
                 and all(np.array_equal(S[(a, b)], gamma[a] @ gamma[b] / 2.0)
                         for a, b in pairs)
                 and recorded("python", "S_definition"),
                 "the 28 S^ab = (1/2) gamma^a gamma^b are linearly independent "
                 "(exact rank 28)",
                 record=record_of("python", "S_definition"))
```

The 28 whole-number matrices $2S^{ab} = \gamma^a\gamma^b$, each written as a row of 256 numbers, have the exact rank 28 (printed together with the entries -0.5, 0, 0.5 of the S^{ab}): no generator is a combination of the others.

In [4], the matrices M^{ab}: a copy of so(4,4).

```
def generator_matrix(a, b):
    result = np.zeros((8, 8))
    for j, c in enumerate(COORDS):
        for i, d in enumerate(COORDS):
            result[i, j] = eta(b, c) * (d == a) - eta(a, c) * (d == b)
    return result


M = {(a, b): generator_matrix(a, b) for a in COORDS for b in COORDS}  # all 64
```

`generator_matrix(a, b)` fills the 8×8 matrix of step (A), $M^{ab}_{dc} = \eta^{bc}\delta_{da} - \eta^{ac}\delta_{db}$, entry by entry: `(d == a)` is true or false, which Python counts as 1 or 0, so it plays the role of δ_{da}. `M` holds the matrices for all 64 ordered pairs.

```
vector_ok = all(
    np.array_equal(S[p] @ gamma[c] - gamma[c] @ S[p],
                   sum(M[p][i, j] * gamma[d] for i, d in enumerate(COORDS)))
    for p in pairs for j, c in enumerate(COORDS))
check_reproduces(vector_ok and recorded("python", "S_vector_action")
                 and recorded("wolfram", "S_gamma_commutator"),
                 "[S^ab, gamma^c] = sum_d M^ab_dc gamma^d with M_dc = eta^bc delta_da "
                 "- eta^ac delta_db",
                 record=record_of("python", "S_vector_action"))
```

For the 28 planes and the 8 directions c the commutator $[S^{ab}, \gamma^c]$ is compared with $\sum_d M^{ab}_{dc}\gamma^d$: the vector rule in the form of step (A).

```
in_so44 = all(not (M[p].T @ ETA_MATRIX + ETA_MATRIX @ M[p]).any() for p in pairs)
rank_M = exact_rank([M[p].reshape(64) for p in pairs])
equations = []  # the 64 equations (X^T eta + eta X)_rc = 0 for the entries of X
for r in range(8):
    for c in range(8):
        row = np.zeros(64)
        row[r * 8 + c] += ETA_MATRIX[r, r]  # (eta X)_rc = eta_rr X_rc
        row[c * 8 + r] += ETA_MATRIX[c, c]  # (X^T eta)_rc = X_cr eta_cc
        equations.append(row)
dimension_so44 = 64 - exact_rank(equations)
say(f"rank of the 28 matrices M^ab: {rank_M}; dimension of so(4,4): "
    f"{dimension_so44}")
check(in_so44 and rank_M == 28 and dimension_so44 == 28,
      "every M^ab lies in so(4,4), the 28 M^ab are independent and so(4,4) has "
      "dimension 28: the M^ab are a basis of so(4,4)")
```

Step (B) in three parts. `in_so44`: every M^{ab} obeys $(M^{ab})^T\eta + \eta M^{ab} = 0$. `rank_M`: the 28 matrices, each written as a row of 64 numbers, have rank 28. `equations`: the condition $X^T\eta + \eta X = 0$ for an unknown 8×8 matrix X is written as 64 linear equations for its 64 entries, listed row by row (the entry X_{rc} is unknown number $8r + c$). The entry (r, c) of ηX is $\eta_{rr}X_{rc}$ and that of $X^T\eta$ is $X_{cr}\eta_{cc}$, so equation (r, c) has the coefficient η_{rr} at the place of X_{rc} and η_{cc} at the place of X_{cr} (`+=` adds, so that on the diagonal, where the two places coincide, both contributions count). The dimension of so(4,4) is 64 minus the rank of these equations. The printed line shows 28 and 28.

```
def table_failures(X):
    failures = 0
    for a, b in pairs:
        for c, d in pairs:
            right = (eta(b, c) * X[(a, d)] - eta(a, c) * X[(b, d)]
                     - eta(b, d) * X[(a, c)] + eta(a, d) * X[(b, c)])
            if not np.array_equal(X[(a, b)] @ X[(c, d)] - X[(c, d)] @ X[(a, b)],
                                  right):
                failures += 1
    return failures


failures_S, failures_M = table_failures(S), table_failures(M)
say(f"pairs that break the table: S^ab {failures_S}, M^ab {failures_M} (of 784)")
```

```
check_reproduces(failures_S == 0 and failures_M == 0
                 and recorded("python", "S_lorentz_algebra")
                 and recorded("wolfram", "S_Lorentz_algebra"),
                 "the S^ab and the M^ab have the same commutators: the S^ab span an "
                 "exact copy of so(4,4)",
                 record=record_of("python", "S_lorentz_algebra"))
```

`table_failures` counts the pairs of planes for which a family of 64 matrices breaks the so(4,4) commutation table. It is applied to the S^{ab} (a recorded fact) and to the M^{ab} (step (C)): 0 failures of 784 for both.

In [5], the shape of the M^{ab}.

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
shape_ok = all(
    np.count_nonzero(M[(a, b)]) == 2
    and np.array_equal(M[(a, b)].T, -ETA[a] * ETA[b] * M[(a, b)]) for a, b in pairs)
check(shape_ok, "each M^ab has two nonzero entries; antisymmetric for the 12 "
      "rotations, symmetric for the 16 boosts")
```

Step (A) predicts two nonzero entries (η_{bb} and $-\eta_{aa}$ in mirrored places). They have opposite signs, so the matrix is antisymmetric, when $\eta_{aa}\eta_{bb} = +1$ (a rotation), and equal signs, so it is symmetric, for a boost: in one formula $(M^{ab})^T = -\eta_{aa}\eta_{bb}M^{ab}$. `np.count_nonzero` counts the nonzero entries.

```
fig, axes = plt.subplots(4, 7, figsize=(14.0, 8.8))
numbers = [x[1] for x in COORDS]  # the tick labels 0, ..., 7
for ax, (a, b) in zip(axes.flat, pairs):
    ax.imshow(M[(a, b)], cmap=SIGNS, vmin=-1, vmax=1)
    kind = "rotation" if ETA[a] * ETA[b] == 1 else "boost"
    ax.set_title(f"$M^{{{a}\\,{b}}}$, {kind}", fontsize=9)
    ax.set_xticks(range(8), numbers, fontsize=6)
    ax.set_yticks(range(8), numbers, fontsize=6)
    ax.grid(False)
fig.suptitle("The 28 matrices $M^{ab}$: how $S^{ab}$ moves the directions "
             "$x0$ to $x7$ (rows and columns numbered 0 to 7)")
save_figure(fig, "generator_matrices",
            ...)
```

A grid of 4 rows and 7 columns of small pictures, one per plane (`axes.flat` runs through the 28 picture areas). `x[1]` is the second character of a name such as `x5`, so the tick labels are the numbers 0 to 7. In the f-string of the title, three braces `{{{a}` give one literal brace followed by the value of `a`, and `\\,` a small space. `fig.suptitle` is a title over the whole figure. In `05f_1_generator_matrices.png` every small picture has exactly two coloured squares, mirrored about the diagonal: of opposite colours for the 12 rotations, of the same colour for the 16 boosts.

In [6], the exponential and its two factors.

```
def exponential(a, b, theta):
    J = (gamma[a] @ gamma[b]).astype(float)  # J = 2 S^ab, J J = -eta_aa eta_bb
```

```
    if ETA[a] * ETA[b] == 1:  # J J = -1: a rotation
        return np.cos(theta / 2) * np.eye(16) + np.sin(theta / 2) * J
    return np.cosh(theta / 2) * np.eye(16) + np.sinh(theta / 2) * J  # a boost


def series_exponential(X, terms=60):
    result = np.eye(X.shape[0])
    term = np.eye(X.shape[0])
    for k in range(1, terms):
        term = term @ X / k  # X^k / k! from X^(k-1) / (k-1)!
        result = result + term
    return result
```

`exponential` is the closed formula and `series_exponential` the power series, here summed to 60 terms; both work as `spin_transformation` and `exp_series` of Notebook 05d (Section 5.22, In [6]). `series_exponential` also serves for the 8×8 matrix exponential $\exp(\theta M)$ in In [8].

```
def gamma_of(v):
    return sum(v[i] * gamma[x].astype(float) for i, x in enumerate(COORDS))


def metric_product(u, v):
    return float(u @ ETA_MATRIX @ v)


def second_factor(a, b, theta):
    i, j = COORDS.index(a), COORDS.index(b)
    if ETA[a] * ETA[b] == 1:  # a rotation
        return ETA[a] * np.cos(theta / 2) * E8[i] + np.sin(theta / 2) * E8[j]
    return ETA[a] * np.cosh(theta / 2) * E8[i] + np.sinh(theta / 2) * E8[j]


E8 = np.eye(8)  # E8[i] is the basic unit vector of the direction COORDS[i]
```

`gamma_of` and `metric_product` are $\gamma(v)$ and $\eta(u, v)$ as in Notebook 05d. `second_factor(a, b, theta)` is the unit vector u of step (D): $\eta_{aa} \cos\frac{\theta}{2}\, e_a + \sin\frac{\theta}{2}\, e_b$ for a rotation and the same with cosh and sinh for a boost. `E8[i]` is the basic unit vector of the direction number `i`; the function uses `E8`, which is defined in the line after it, before the function is first called, so this is allowed.

```
series_gap = max(np.max(np.abs(series_exponential(0.9 * S[p]) - exponential(*p, 0.9)))
                 for p in pairs)
check(series_gap < 1e-10, "the closed formula equals the power series of "
      "exp(theta S^ab) for all 28 planes (difference below 1e-10)")
factor_ok = True
for a, b in pairs:
    for theta in [-2.5, -0.4, 0.9, 2.2, 5.0]:
        u = second_factor(a, b, theta)
        factor_ok &= (np.allclose(gamma[a] @ gamma_of(u), exponential(a, b, theta),
                                  rtol=0.0, atol=1e-10)
                      and abs(metric_product(u, u) - ETA[a]) < 1e-10)
check(factor_ok, "exp(theta S^ab) = gamma^a gamma(u) with eta(u, u) = eta_aa: a "
      "product of two unit vectors (28 planes, 5 values each)")
```

The first check compares the closed formula with the series for all 28 planes at $\theta = 0.9$. The second checks step (D) for all 28 planes at five values of θ: $\exp(\theta S^{ab}) = \gamma^a \gamma(u)$ and $\eta(u, u) = \eta_{aa}$. (`rtol=0.0, atol=1e-10` makes `np.allclose` accept only differences below 10^{-10}, without a relative tolerance.)

In [7], the picture of the second factor.

```
cases = [("x1", "x2", np.linspace(0.0, 4 * np.pi, 241)),
         ("x1", "x4", np.linspace(-3.0, 3.0, 241)),
         ("x4", "x0", np.linspace(-3.0, 3.0, 241))]
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.6))
norm_ok = True  # eta(u, u) = eta_aa along all three curves
for ax, (a, b, thetas) in zip(axes, cases):
    points = np.array([second_factor(a, b, th) for th in thetas])
    i, j = COORDS.index(a), COORDS.index(b)
    norms = [metric_product(p, p) for p in points]
    norm_ok &= max(abs(n - ETA[a]) for n in norms) < 1e-10
    ax.plot(points[:, i], points[:, j], color="#2a78d6", linewidth=2)
    marks = points[::40]  # every 40th value of theta
    ax.plot(marks[:, i], marks[:, j], "o", color="#eb6834", markersize=6)
    kind = "rotation" if ETA[a] * ETA[b] == 1 else "boost"
    ax.set_title(f"{kind} in $({a}, {b})$: $\\eta(u, u) = {norms[0]:+.0f}$")
    ax.set_xlabel(f"$u_{{{a}}}$")
    ax.set_ylabel(f"$u_{{{b}}}$")
    ax.set_aspect("equal")
    ax.set_xlim(-4.2, 4.2)
    ax.set_ylim(-4.2, 4.2)
check(norm_ok, "along the three curves eta(u, u) stays equal to eta_aa")
save_figure(fig, "two_unit_vectors",
            ...)
```

For three planes (a rotation over θ from 0 to 4π and two boosts over -3 to 3) the cell computes the second factor u at 241 values and plots its two nonzero components against each other, with an orange dot every 40th value. The check confirms $\eta(u, u) = \eta_{aa}$ along all three curves, and each title prints this value ($+1$ for the planes starting with $x1$, -1 for the plane $(x4, x0)$). In `05f_2_two_unit_vectors.png` the rotation's u runs once around the unit circle, and the boosts' u run along branches of hyperbolas.

In [8], the vector matrix of an exponential.

```
def vector_matrix(g, odd=False):
    g_inverse = np.linalg.inv(g)
    sign = -1.0 if odd else 1.0
    result = np.zeros((8, 8))
    for j, c in enumerate(COORDS):
        moved = sign * (g @ gamma[c] @ g_inverse)  # alpha(g) gamma^c g^-1
        for i, d in enumerate(COORDS):
            result[i, j] = ETA[d] * np.trace(gamma[d] @ moved) / 16.0
    return result
```

`vector_matrix(g, odd)` is the vector matrix of Section 5.12 with the twisted action: for an odd

element (`odd=True`) the moved gamma gets the sign -1. Otherwise it is the trace formula of Notebook 05d.

```
exp_ok = True
for p in pairs:
    Lam = vector_matrix(exponential(*p, 0.9))
    exp_ok &= (np.allclose(Lam, series_exponential(0.9 * M[p]), rtol=0.0, atol=1e-12)
               and np.allclose(Lam.T @ ETA_MATRIX @ Lam, ETA_MATRIX, atol=1e-12)
               and abs(np.linalg.det(Lam) - 1.0) < 1e-12)
check(exp_ok, "the vector matrix of exp(theta S^ab) is exp(theta M^ab); it keeps "
      "the metric and has determinant 1 (28 planes)")
```

Step (E) for all 28 planes at $\theta = 0.9$: the vector matrix of the spinor exponential equals the 8×8 exponential $\exp(0.9\, M^{ab})$, keeps the metric and has determinant 1.

```
def turned_axis(a, b, theta):
    i, j = COORDS.index(a), COORDS.index(b)
    if ETA[a] * ETA[b] == 1:  # a rotation
        return np.cos(theta) * E8[i] - ETA[a] * np.sin(theta) * E8[j]
    return np.cosh(theta) * E8[i] - ETA[a] * np.sinh(theta) * E8[j]  # a boost


turn_ok = all(
    np.allclose(vector_matrix(exponential(a, b, th))[:, COORDS.index(a)],
                turned_axis(a, b, th), rtol=0.0, atol=1e-12)
    for a, b in [("x0", "x1"), ("x4", "x5"), ("x0", "x4"), ("x4", "x0")]
    for th in [-1.3, 0.4, 2.0])
check(turn_ok, "Lambda e_a = cos(theta) e_a - eta_aa sin(theta) e_b for a rotation "
      "and cosh(theta) e_a - eta_aa sinh(theta) e_b for a boost")
```

`turned_axis` is the formula of step (E) for Λe_a. The check compares it with column a of the computed vector matrix for the four planes used by the construction of step (I) ($(x0, x1)$, $(x4, x5)$, $(x0, x4)$, $(x4, x0)$; note that the order of a and b matters) at three values each.

In [9], the block equations on random products.

```
SPACE_INDEX = [COORDS.index(x) for x in SPACE]  # the positions of x0, x1, x2, x3
TIME_INDEX = [COORDS.index(x) for x in TIME]  # the positions of x4, x5, x6, x7


def blocks(Lam):
    return (Lam[np.ix_(SPACE_INDEX, SPACE_INDEX)], Lam[np.ix_(SPACE_INDEX, TIME_INDEX)],
            Lam[np.ix_(TIME_INDEX, SPACE_INDEX)], Lam[np.ix_(TIME_INDEX, TIME_INDEX)])
```

`SPACE_INDEX` is the list of positions $[0, 1, 2, 3]$ of $x0, x1, x2, x3$ and `TIME_INDEX` the list $[4, 5, 6, 7]$ of $x4, x5, x6, x7$. `np.ix_(rows, columns)` picks the entries in the listed rows and columns, so `blocks` returns the four 4×4 blocks A, B', C', D of step (F).

```
rng = np.random.default_rng(12345)  # random numbers with a fixed seed
```

```
def random_factors(count):
    factors = []
    for _ in range(count):
        a, b = pairs[int(rng.integers(28))]  # a random plane
        if ETA[a] * ETA[b] == 1:  # a rotation: an angle from 0 to 2 pi
            theta = rng.uniform(0.0, 2 * np.pi)
        else:  # a boost: a rapidity from -1 to 1
            theta = rng.uniform(-1.0, 1.0)
        factors.append((a, b, float(theta)))
    return factors
```

The random-number generator starts from the fixed seed 12345; every later cell draws from the same generator in the same order, so every run makes the same choices. `random_factors(count)` draws `count` factors (a, b, θ): a plane chosen at random from the 28 (`rng.integers(28)` is a whole number from 0 to 27), with a uniformly random angle in $[0, 2\pi)$ for a rotation or a rapidity in $[-1, 1)$ for a boost (`rng.uniform(low, high)`). The loop variable `_` is a name for a value that is not used.

```
def compose(factors, t=1.0):
    g = np.eye(16)
    for a, b, theta in factors:
        g = exponential(a, b, t * theta) @ g
    return g
```

`compose(factors, t)` multiplies the exponentials $\exp(t\theta_j S_j)$ in the order of the list, each new one on the left: the result is $\exp(t\theta_n S_n)\cdots\exp(t\theta_1 S_1)$, in which the first factor of the list acts first on a column. With t running from 0 to 1 it is the path $h(t)$ of step (G).

```
block_ok, smallest = True, np.inf
for _ in range(200):
    A, B_, C_, D = blocks(vector_matrix(compose(random_factors(6))))
    block_ok &= (np.allclose(A.T @ A - C_.T @ C_, np.eye(4), atol=1e-9)
                 and np.allclose(D.T @ D - B_.T @ B_, np.eye(4), atol=1e-9))
    smallest = min(smallest, np.linalg.det(A), np.linalg.det(D))
report("smallest det A or det D of 200 products of six exponentials",
       f"{smallest:.3f}")
check(block_ok and smallest > 1.0 - 1e-9,
      "200 products of exponentials: A^T A - C^T C = 1, D^T D - B^T B = 1, and "
      "det A >= 1, det D >= 1")
```

For 200 random products of six exponentials the cell checks the two block equations of step (F) (the blocks are named `B_` and `C_` so as not to overwrite other names) and keeps the smallest of all determinants of A and D, starting from `np.inf` (infinity). The RESULT line prints it, 1.000, and the check requires it to be at least 1 (up to 10^{-9}): every product has the sign pattern $(+,+)$, step (G). The smallest value is the bound itself: one of the 200 random products is made of six rotations only (in the planes $(x1, x2)$, $(x0, x1)$, $(x4, x7)$, $(x6, x7)$, $(x2, x3)$ and $(x1, x2)$), and a rotation does not mix space with time, so $C' = B' = 0$, the block equations give $A^T A = D^T D = 1_4$, and $\det A = \det D = 1$ exactly.

In [10], the four words and the four pieces.

```
WORDS = {"1": (I16, False), "gamma^(x0)": (gamma["x0"], True),
         "gamma^(x4)": (gamma["x4"], True),
         "gamma^(x0) gamma^(x4)": (gamma["x0"] @ gamma["x4"], False)}


def pattern(Lam):
    A, _, _, D = blocks(Lam)
    return (int(np.sign(np.linalg.det(A))), int(np.sign(np.linalg.det(D))))
```

`WORDS` stores the four words 1, $\gamma^{(x0)}$, $\gamma^{(x4)}$, $\gamma^{(x0)}\gamma^{(x4)}$, each with the flag that says whether it is odd. `pattern` returns the sign pattern of a vector matrix: the signs (`np.sign`) of $\det A$ and $\det D$; the two middle blocks are not needed and are given the throw-away name `_`.

```
flip0 = np.diag([-1.0, 1, 1, 1, 1, 1, 1, 1])  # reverses x0 only (position 0)
flip4 = np.diag([1.0, 1, 1, 1, -1, 1, 1, 1])  # reverses x4 only (position 4)
expected = {"1": np.eye(8), "gamma^(x0)": flip0, "gamma^(x4)": flip4,
            "gamma^(x0) gamma^(x4)": flip0 @ flip4}
words_ok = all(np.allclose(vector_matrix(w.astype(float), odd), expected[name])
               for name, (w, odd) in WORDS.items())
for name, (w, odd) in WORDS.items():
    say(f"{name:22} sign pattern (sign det A, sign det D) = "
        f"{pattern(vector_matrix(w.astype(float), odd))}")
check(words_ok, "the vector matrices of 1, gamma^(x0), gamma^(x4), gamma^(x0) "
      "gamma^(x4) are the identity and the reversals of x0, x4 and both")
```

The predicted vector matrices of step (H) are diagonal matrices of signs. The check compares them with the computed ones (the twisted action for the two odd words), and the four printed lines show the patterns $(1,1)$, $(-1,1)$, $(1,-1)$, $(-1,-1)$.

```
points = {}  # word -> list of (det A, det D)
pieces_ok = True
for name, (w, odd) in WORDS.items():
    points[name] = []
    Lam_w = vector_matrix(w.astype(float), odd)
    for _ in range(60):
        h = compose(random_factors(6))
        Lam = vector_matrix(h @ w, odd)
        pieces_ok &= (np.allclose(Lam, vector_matrix(h) @ Lam_w, atol=1e-9)
                      and pattern(Lam) == pattern(Lam_w))
        A, _, _, D = blocks(Lam)
        points[name].append((np.linalg.det(A), np.linalg.det(D)))
check(pieces_ok, "for 4 x 60 elements h w: Lambda(h w) = Lambda(h) Lambda(w), and h w "
      "has the sign pattern of w")
```

For each word w and 60 random products h the cell checks $\Lambda(hw) = \Lambda(h)\Lambda(w)$ and that hw has the pattern of w, and stores the pair $(\det A, \det D)$ for the picture.

```
# An observation that the picture below shows and that the proof does not need:
# for every one of the 240 elements the two determinants have the same size.
equal_size = all(abs(abs(dA) - abs(dD)) < 1e-9 * max(abs(dA), 1.0)
                 for name in WORDS for dA, dD in points[name])
check(equal_size, "observed for all 240 elements: |det A| = |det D|")
```

An observation, checked but not used by the proof: for all 240 elements $|\det A| = |\det D|$, to a relative accuracy of 10^{-9}. It is COMPUTED for these 240 elements only; the book neither proves nor uses it.

In [11], the picture of the four pieces.

```
colors = {"1": "#2a78d6", "gamma^(x0)": "#eb6834", "gamma^(x4)": "#1baf7a",
          "gamma^(x0) gamma^(x4)": "#4a3aa7"}
labels = {"1": r"$h$ (products of exponentials)",
          "gamma^(x0)": r"$h\gamma^{(x0)}$",
          "gamma^(x4)": r"$h\gamma^{(x4)}$",
          "gamma^(x0) gamma^(x4)": r"$h\gamma^{(x0)}\gamma^{(x4)}$"}
fig, ax = plt.subplots(figsize=(7.4, 6.6))
ax.axvspan(-1.0, 1.0, color="#d8d6d2", alpha=0.6, linewidth=0)
ax.axhspan(-1.0, 1.0, color="#d8d6d2", alpha=0.6, linewidth=0)
for name in WORDS:
    values = np.array(points[name])
    ax.plot(values[:, 0], values[:, 1], "o", color=colors[name], markersize=5,
            label=labels[name])
ax.set_xscale("symlog", linthresh=1.0)
ax.set_yscale("symlog", linthresh=1.0)
ax.set_xlim(-60, 60)
ax.set_ylim(-60, 60)
ax.text(0.0, 0.0, "forbidden band:\n$|\\det| < 1$", ha="center", va="center",
        fontsize=8)
ax.set_xlabel("$\\det A$ (space block)")
ax.set_ylabel("$\\det D$ (time block)")
ax.set_title("The four pieces of Pin(4,4)")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.12), ncol=2)
save_figure(fig, "four_pieces",
            ...)
```

`axvspan` and `axhspan` shade the vertical and the horizontal band between -1 and 1 in grey. The 240 points are drawn in four colours, one per word. `set_xscale("symlog", linthresh=1.0)` makes an axis **symmetric logarithmic**: linear between -1 and 1 and logarithmic outside, so that values from 1 to 60 and the band are visible together. In `05f_3_four_pieces.png` each colour fills one of the four corners outside the band, and the products of exponentials (blue) never leave the corner where both determinants are at least 1.

In [12], paths from 1.

```
path_t = np.linspace(0.0, 1.0, 101)  # t = 0, 0.01, ..., 1
curves_h, curves_g0 = [], []  # det A(t) along h(t) and along gamma^(x0) h(t)
g0 = gamma["x0"].astype(float)
for _ in range(10):
    factors = random_factors(6)
    along_h, along_g0 = [], []
    for t in path_t:
        h = compose(factors, t)
        along_h.append(np.linalg.det(blocks(vector_matrix(h))[0]))
        along_g0.append(np.linalg.det(blocks(vector_matrix(g0 @ h, odd=True))[0]))
    curves_h.append(along_h)
```

```
    curves_g0.append(along_g0)
curves_h, curves_g0 = np.array(curves_h), np.array(curves_g0)
check(np.all(curves_h > 1.0 - 1e-9) and np.all(curves_g0 < -1.0 + 1e-9)
      and np.allclose(curves_h[:, 0], 1.0) and np.allclose(curves_g0[:, 0], -1.0),
      "along ten paths t -> h(t) det A stays >= 1, and along gamma^(x0) h(t) it "
      "stays <= -1")
```

For ten random products of six exponentials the cell follows the path $h(t)$ of step (G) at 101 values of t and records $\det A(t)$ (`blocks(...)[0]` is the space block), and the same for the odd elements $\gamma^{(x0)}h(t)$. The check: the first family starts at 1 and stays at or above 1, the second starts at -1 and stays at or below -1.

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.axhspan(-1.0, 1.0, color="#d8d6d2", alpha=0.6, linewidth=0)
for k in range(10):
    ax.plot(path_t, curves_h[k], color="#2a78d6", linewidth=1.5,
            label=r"$h(t)$" if k == 0 else None)
    ax.plot(path_t, curves_g0[k], color="#eb6834", linewidth=1.5, linestyle="--",
            label=r"$\gamma^{(x0)}h(t)$" if k == 0 else None)
ax.set_yscale("symlog", linthresh=1.0)
ax.set_ylim(-60, 60)
ax.text(0.5, 0.0, "forbidden band $|\\det A| < 1$", ha="center", va="center")
ax.set_xlabel("path parameter $t$ (from 1 at $t = 0$ to $h$ at $t = 1$)")
ax.set_ylabel("$\\det A$ (space block)")
ax.set_title("Ten paths of products of exponentials, and the same times "
             "$\\gamma^{(x0)}$")
ax.legend(loc="upper left")
save_figure(fig, "paths_and_band",
            ...)
```

The twenty curves are drawn over the grey band (only the first curve of each family gets a legend label; `None` means no label). In `05f_4_paths_and_band.png` the blue curves start at 1 and stay above the band, the orange ones start at -1 and stay below it: no curve crosses the band, which is why $\gamma^{(x0)}$ cannot be reached from 1 by exponentials.

In [13], every unit vector is a turned basic one.

```
def turn_factor(a, b, along_a, along_b):
    if ETA[a] * ETA[b] == 1:  # a rotation
        return (a, b, float(np.arctan2(-ETA[a] * along_b, along_a)))
    return (a, b, float(np.arcsinh(-ETA[a] * along_b)))  # a boost
```

`turn_factor` returns the factor (a, b, θ) whose vector matrix carries e_a to the direction of $\text{along}_a\, e_a + \text{along}_b\, e_b$. For a rotation, step (E) gives $\Lambda e_a = \cos\theta\, e_a - \eta_{aa} \sin\theta\, e_b$; `np.arctan2(y, x)` is the angle θ of the point (x, y) in the plane, so that $\cos\theta$ and $\sin\theta$ are x and y divided by the distance $\sqrt{x^2 + y^2}$. With $x = \text{along}_a$ and $y = -\eta_{aa}\text{along}_b$, the vector Λe_a points along $\text{along}_a\, e_a + \text{along}_b\, e_b$. For a boost, $\Lambda e_a = \cosh\theta\, e_a - \eta_{aa} \sinh\theta\, e_b$; with $\sinh\theta = -\eta_{aa}\text{along}_b$ (`np.arcsinh` is the inverse of sinh) and $\text{along}_a^2 - \text{along}_b^2 = 1$, $\cosh\theta = \sqrt{1 + \text{along}_b^2} = \text{along}_a$.

```
def block_rotation(axis, others, n):
    factors = []
    for k, b in enumerate(others):
        if k < len(others) - 1:  # keep the length of the rest on the axis
            rest = np.sqrt(n[axis] ** 2 + sum(n[o] ** 2 for o in others[k + 1:]))
        else:  # the last rotation leaves exactly n_axis on the axis
            rest = n[axis]
        factors.append(turn_factor(axis, b, rest, n[b]))
    return factors
```

`block_rotation` returns three rotations in the planes (axis, first other), (axis, second other), (axis, third other) that carry e_{axis} to the unit vector n of its block (`n` is a dictionary from direction names to components). The first rotation puts the component n_b on the first other direction and leaves on the axis the length of everything that is still to come, $\sqrt{n_{\text{axis}}^2 + n_{b_2}^2 + n_{b_3}^2}$; each later rotation acts only on what is left on the axis; the last one leaves exactly n_{axis}, which may be negative.

```
def carry(v):
    part = {x: float(v[COORDS.index(x)]) for x in COORDS}
    sigma = np.sqrt(sum(part[x] ** 2 for x in SPACE))  # length of the space part
    tau = np.sqrt(sum(part[x] ** 2 for x in TIME))  # length of the time part
    space_turn = block_rotation("x0", ["x1", "x2", "x3"],
                                {x: part[x] / sigma for x in SPACE}) if sigma else []
    time_turn = block_rotation("x4", ["x5", "x6", "x7"],
                               {x: part[x] / tau for x in TIME}) if tau else []
    if metric_product(v, v) > 0:  # space-like: from e_x0
        return [turn_factor("x0", "x4", sigma, tau)] + time_turn + space_turn, "x0"
    return [turn_factor("x4", "x0", tau, sigma)] + space_turn + time_turn, "x4"
```

`carry(v)` is the construction of step (I). `part` holds the eight components by name; `sigma` and `tau` are the lengths $|\sigma|$ and $|\tau|$ of the space part and the time part. `space_turn` turns e_{x0} into $\sigma/|\sigma|$ and `time_turn` turns e_{x4} into $\tau/|\tau|$ (an empty list when that part is zero; `if sigma else []` uses that the number 0 counts as false). For a space-like v the factors are the boost in $(x0, x4)$, then the time rotations, then the space rotations, and the starting direction is $x0$; for a time-like v the boost in $(x4, x0)$, then the space rotations, then the time rotations, starting from $x4$. The function returns the list of factors (the first acts first) together with the starting direction.

```
def random_unit_vector():
    while True:
        w = rng.normal(size=8)  # eight numbers from the normal distribution
        q = metric_product(w, w)
        if abs(q) >= 0.5:
            return w / np.sqrt(abs(q))
```

A random unit vector: eight numbers are drawn from the normal distribution (the bell curve centred at 0), and the vector is kept only when $|\eta(w, w)| \geq 0.5$ (so that it is not nearly null); `while True` repeats until that happens. Dividing by $\sqrt{|\eta(w, w)|}$ makes $\eta = +1$ or -1.

```
starts = {"x0": 0, "x4": 0}  # how many space-like and time-like vectors
carry_ok, most_factors = True, 0
```

```
for _ in range(40):
    v = random_unit_vector()
    factors, e = carry(v)
    starts[e] += 1
    most_factors = max(most_factors, len(factors))
    h = compose(factors)
    carry_ok &= (np.allclose(vector_matrix(h)[:, COORDS.index(e)], v, atol=1e-12)
                 and np.allclose(h @ gamma[e] @ np.linalg.inv(h), gamma_of(v),
                                 atol=1e-12))
say(f"space-like vectors: {starts['x0']}; time-like vectors: {starts['x4']}; "
    f"at most {most_factors} exponentials per vector")
check(carry_ok and starts["x0"] > 0 and starts["x4"] > 0 and most_factors <= 7,
      "for 40 random unit vectors v: gamma(v) = h gamma^(e) h^-1 with h a product "
      "of at most 7 exponentials and e = x0 or x4")
```

For 40 random unit vectors the cell builds h, checks that column e of its vector matrix is v and that $h\gamma^e h^{-1} = \gamma(v)$ as 16×16 matrices, and counts the space-like and time-like vectors and the largest number of factors. The printed line shows 24 space-like and 16 time-like vectors and at most 7 factors.

In [14], the construction at work.

```
examples = [random_unit_vector() for _ in range(6)]
space_example = next(v for v in examples if metric_product(v, v) > 0)
time_example = next(v for v in examples if metric_product(v, v) < 0)
line_colors = ["#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948", "#eda100",
               "#52514e", "#9e9c98"]
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.6), sharey=True)
steps_ok = True
```

Six more random unit vectors are drawn; `next(...)` takes the first space-like and the first time-like one among them. Eight colours, one per direction, and two pictures that share the vertical axis (`sharey=True`).

```
for ax, v in zip(axes, [space_example, time_example]):
    factors, e = carry(v)
    moving = [E8[COORDS.index(e)]]  # the vector after 0, 1, ..., 7 steps
    for a, b, theta in factors:
        moving.append(vector_matrix(exponential(a, b, theta)) @ moving[-1])
    moving = np.array(moving)
    steps_ok &= np.allclose([metric_product(m, m) for m in moving], ETA[e])
    steps_ok &= np.allclose(moving[-1], v, atol=1e-12)
    if e == "x0":
        for a, b, theta in factors:
            say(f"    factor exp(theta S^({a} {b})), theta = {theta:+.4f}")
```

For each example the moving vector starts at e_{x0} or e_{x4}, and each factor's vector matrix is applied to the latest vector (`moving[-1]` is the last element of the list). The cell checks that the metric product stays η_{ee} at every step and that the last vector is v, and for the space-like example it prints the seven factors with their angles to four decimals.

```
    for i, x in enumerate(COORDS):
        ax.plot(range(len(moving)), moving[:, i], "o-", color=line_colors[i],
                linewidth=1.5, markersize=4, label=x)
        ax.plot([len(moving) - 0.6], [v[i]], "D", color=line_colors[i],
                markersize=6)
    kind = "space-like" if e == "x0" else "time-like"
    ax.set_title(f"a {kind} unit vector, built from $e_{{{e}}}$")
    ax.set_xticks(range(len(moving)))
    ax.set_xlabel("number of exponentials applied")
axes[0].set_ylabel("component of the moving vector")
axes[1].legend(loc="upper center", bbox_to_anchor=(-0.05, -0.15), ncol=8)
check(steps_ok, "in both examples the moving vector keeps eta(w, w) and ends at v")
save_figure(fig, "carrying_vectors",
            ...)
```

Still inside the loop, each of the eight components is drawn against the number of steps, and the target component of v as a diamond (`"D"`) just right of the last step. In `05f_5_carrying_vectors.png` the first step shares the length between $x0$ and $x4$ (a boost), the next three steps fill one block and the last three the other, and every curve ends at its diamond.

In [15], moving a gamma through an exponential.

```
def conjugation_sign(e, a, b):
    return -1 if e in (a, b) else 1


conjugation_ok = all(
    np.allclose(gamma[e] @ exponential(a, b, 0.8) @ np.linalg.inv(gamma[e]),
                exponential(a, b, conjugation_sign(e, a, b) * 0.8), atol=1e-12)
    for e in ("x0", "x4") for a, b in pairs)
check(conjugation_ok, "gamma^e exp(theta S^ab) (gamma^e)^-1 = exp(s theta S^ab), "
      "s = -1 if e is a or b, +1 otherwise")
```

`conjugation_sign` is the sign s of step (J). The check confirms $\gamma^e \exp(\theta S^{ab})(\gamma^e)^{-1} = \exp(s\theta S^{ab})$ for $e = x0$ and $x4$, all 28 planes and $\theta = 0.8$.

In [16], twelve random elements of Pin(4,4).

```
FOUR = {"1": I16, "gamma^(x0)": gamma["x0"], "gamma^(x4)": gamma["x4"],
        "gamma^(x0) gamma^(x4)": gamma["x0"] @ gamma["x4"]}


def evaluate(word):
    g = np.eye(16)
    for item in word:
        if item[0] == "exp":
            g = g @ exponential(*item[1:])
        else:
            g = g @ gamma[item[1]]
    return g
```

A **word** is a list of items in the order of the matrix product: an exponential `("exp", a, b, theta)` or a gamma `("gamma", e)`. `evaluate` multiplies the items from left to right (`item[1:]` is

the item without its first entry, unpacked into the arguments of `exponential`). `FOUR` holds the four words as matrices.

```
def word_of(v):
    factors, e = carry(v)
    h_word = [("exp", a, b, th) for a, b, th in reversed(factors)]  # E_n ... E_1
    h_inverse = [("exp", a, b, -th) for a, b, th in factors]  # E_1^-1 ... E_n^-1
    return h_word + [("gamma", e)] + h_inverse
```

`word_of(v)` writes $\gamma(v) = h\gamma^e h^{-1}$ as a word. Since the first factor acts first, $h = E_n \cdots E_1$, which is the list of factors reversed; $h^{-1} = E_1^{-1} \cdots E_n^{-1}$, each with the opposite angle.

```
def move_gammas_right(word):
    exponentials, waiting = [], []  # waiting: the gammas moved to the right so far
    for item in word:
        if item[0] == "gamma":
            waiting.append(item[1])
        else:  # this exponential is moved to the left of all waiting gammas
            _, a, b, theta = item
            s = 1
            for e in waiting:
                s *= conjugation_sign(e, a, b)
            exponentials.append(("exp", a, b, s * theta))
    return exponentials, waiting
```

`move_gammas_right` reads the word from left to right. A gamma is put on the list `waiting`. An exponential that comes after waiting gammas is moved to their left; by step (J) each gamma it passes multiplies its angle by the sign s of that gamma. The result is the list of exponentials followed by the list of gammas, which as a word equals the original word.

```
def reduce_gammas(waiting):
    product = I16
    for e in waiting:
        product = product @ gamma[e]
    for name, w in FOUR.items():
        if np.array_equal(product, w):
            return 1, name
        if np.array_equal(product, -w):
            return -1, name
    raise ValueError("the product of the gammas is not one of the four words")
```

`reduce_gammas` multiplies the waiting gammas and finds which of the four words it equals, with which sign; if it were none of them (impossible by step (J)), the notebook would stop with an error.

```
decomposition_ok = True
for number in range(12):
    k = 1 + number % 6  # the number of unit vectors
    vectors = [random_unit_vector() for _ in range(k)]
    g = np.eye(16)
```

```
    word = []
    for v in vectors:
        g = g @ gamma_of(v)
        word += word_of(v)
    size = np.max(np.abs(g))  # for relative differences
    exponentials, waiting = move_gammas_right(word)
    sign, name = reduce_gammas(waiting)
    if sign == -1:  # -1 = exp(2 pi S^(x1 x2))
        exponentials.append(("exp", "x1", "x2", 2 * np.pi))
    rebuilt = evaluate(exponentials) @ FOUR[name]
```

Twelve elements are made from $k = 1, \ldots, 6$ random unit vectors (twice each; `number % 6` is the remainder of the number divided by 6). For each, the product g and its word are built, the gammas are moved to the right and reduced, a sign -1 is replaced by the exponential $\exp(2\pi S^{(x1)(x2)}) = -1$, and the decomposition is multiplied out again as `rebuilt`.

```
    same_pattern = (pattern(vector_matrix(g, odd=k % 2 == 1))
                    == pattern(vector_matrix(FOUR[name].astype(float), WORDS[name][1])))
    decomposition_ok &= (np.max(np.abs(evaluate(word) - g)) < 1e-9 * size
                         and np.max(np.abs(rebuilt - g)) < 1e-9 * size
                         and same_pattern and WORDS[name][1] == (k % 2 == 1))
    plural = "s" if k > 1 else " "  # "1 unit vector", "2 unit vectors"
    say(f"element {number + 1:2d}: {k} unit vector{plural} = {len(exponentials):2d} "
        f"exponentials times {name}")
check(decomposition_ok, "12 random elements of Pin(4,4): each is a product of "
      "exponentials times one of 1, gamma^(x0), gamma^(x4), gamma^(x0) gamma^(x4), "
      "with the matching sign pattern")
```

The checks for each element: the word equals g, the decomposition equals g (both to a relative accuracy of 10^{-9}, measured against the largest entry `size` of g, because the entries of g can be large), g has the sign pattern of its word, and the word is odd exactly when k is odd. The twelve printed lines show, for example, that one unit vector becomes 14 exponentials times $\gamma^{(x4)}$ (seven for h and seven for h^{-1}; both elements made of one vector happen to be time-like), that an odd number of vectors always ends with an odd word ($\gamma^{(x0)}$ or $\gamma^{(x4)}$) and an even number with an even word, and that the two elements made of two vectors end with the two different even words, 1 and $\gamma^{(x0)}\gamma^{(x4)}$.

In [17], the spans.

```
formula_ok = all(
    np.allclose(exponential(a, b, np.pi), gamma[a] @ gamma[b], atol=1e-12)
    if ETA[a] * ETA[b] == 1 else
    np.allclose((exponential(a, b, 0.8) - exponential(a, b, -0.8))
                / (2 * np.sinh(0.4)), gamma[a] @ gamma[b], atol=1e-12)
    for a, b in pairs)
check(formula_ok, "exp(pi S^ab) = gamma^a gamma^b for the rotations and "
      "(exp(theta S^ab) - exp(-theta S^ab)) / (2 sinh(theta/2)) = gamma^a gamma^b "
      "for the boosts")
```

The two formulas of the last paragraph of Section 5.23 are checked for all 28 planes (for the boosts with $\theta = 0.8$, so that $2\sinh\frac{\theta}{2} = 2\sinh 0.4$).

```
spin0_elements = [compose(random_factors(6)) for _ in range(170)]
four_words = list(FOUR.values())
pin_elements = [compose(random_factors(6)) @ four_words[k % 4] for k in range(300)]


def rank_curve(elements):
    ranks = []
    for n in range(1, len(elements) + 1):
        rows = np.array([g.reshape(256) for g in elements[:n]])
        singular = np.linalg.svd(rows, compute_uv=False)
        ranks.append(int(np.sum(singular > 1e-9 * singular[0])))
    return ranks
```

170 random products of six exponentials, and 300 random elements of all four pieces (`k % 4` runs through the four words). `rank_curve` computes, for $n = 1, 2, \ldots$, the rank of the first n elements written as rows. It uses the **singular values** of the array of rows (`np.linalg.svd(..., compute_uv=False)`): the square roots of the eigenvalues of $A^T A$ for the array A, sorted from the largest. By the argument of Section 5.13 the number of nonzero ones is the rank; here a singular value counts as nonzero when it is above 10^{-9} times the largest.

```
ranks_spin0, ranks_pin = rank_curve(spin0_elements), rank_curve(pin_elements)
block_diagonal = all(not g[:8, 8:].any() and not g[8:, :8].any()
                     for g in spin0_elements)
report("rank of 170 products of exponentials", ranks_spin0[-1])
report("rank of 300 elements of Pin(4,4)", ranks_pin[-1])
check_reproduces(ranks_spin0 == [min(n, 128) for n in range(1, 171)]
                 and block_diagonal
                 and recorded("python", "even_products_span_M8_plus_M8"),
                 "the products of exponentials are block diagonal and span 128 "
                 "dimensions, as the 128 even Clifford products",
                 record=record_of("python", "even_products_span_M8_plus_M8"))
check_reproduces(ranks_pin == [min(n, 256) for n in range(1, 301)]
                 and recorded("python", "clifford_products_span_M16"),
                 "the elements of Pin(4,4) span all 256 dimensions of the 16 x 16 "
                 "matrices",
                 record=record_of("python", "clifford_products_span_M16"))
```

The two RESULT lines print the final ranks, 128 and 256. The checks require that each new element raised the rank by one until it stopped at 128 (for the products of exponentials, which are also all block diagonal) and at 256 (for Pin(4,4)): the numbers that the record found for the even and for all products of different gammas.

In [18], the picture of the spans.

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.plot(range(1, 171), ranks_spin0, color="#2a78d6", linewidth=2.5,
        label=r"products of exponentials ($\mathrm{Spin}_0(4,4)$)")
ax.plot(range(1, 301), ranks_pin, color="#eb6834", linewidth=2, linestyle="--",
        label="elements of Pin(4,4) (all four pieces)")
ax.axhline(128, color="#2a78d6", linewidth=0.8, linestyle=":")
ax.axhline(256, color="#eb6834", linewidth=0.8, linestyle=":")
```

```
ax.set_xlabel("number $n$ of random group elements")
ax.set_ylabel("rank of the first $n$ elements")
ax.set_yticks([0, 64, 128, 192, 256])
ax.set_title("The span of the group elements")
ax.legend(loc="lower right")
save_figure(fig, "span_ranks",
            ...)
```

The two rank curves and dotted lines at 128 and 256. In `05f_6_span_ranks.png` both curves rise along the diagonal and then stop flat, the solid one at 128 and the dashed one at 256.

In [19], the last check.

```
FIGURES = ["05f_1_generator_matrices.png", "05f_2_two_unit_vectors.png",
           "05f_3_four_pieces.png", "05f_4_paths_and_band.png",
           "05f_5_carrying_vectors.png", "05f_6_span_ranks.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the six figure files of notebook 05f exist")
all_checks_passed()
```

The last line reads ALL 24 CHECKS PASSED (notebook 05f): one each in In [2] and In [3], three in In [4], one in In [5], two in In [6], one in In [7], two in In [8], one in In [9], three in In [10], one each in In [12] to In [16], three in In [17] and one in In [19]. Six of them reproduce recorded checks (In [2], In [3], the first and the third of In [4], and the last two of In [17]); the other eighteen are the notebook's own computations.

5.28 Charge conjugation is a matrix

The question. Every known particle has an **antiparticle** with the same mass and the opposite charge; the positron is the antiparticle of the electron. In a field theory the step from a solution to its antiparticle solution is made by a map called **charge conjugation**: it turns every solution of the field equation into another solution whose charge density has the opposite sign. In the usual four-dimensional Dirac theory this map contains a complex conjugation of the components together with a fixed matrix. In the author's theory one fact changes the picture: the eight gammas are real (Section 5.2). Take a **real** field, one whose 16 components are real numbers at every point, so that $\Psi^* = \Psi$. On such a field the complex conjugation $\Psi \to \Psi^*$ changes nothing at all: it is the identity map. A map that changes nothing cannot exchange matter and antimatter. So whatever charge conjugation is in this theory, it must be made by a **matrix**. This section derives the condition that such a matrix must obey together with every gamma, takes that condition as the definition of a charge-conjugation matrix (as the Revision record does), and finds every matrix that obeys it. The answer (Theorem CC below) is: exactly two, up to a factor, $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$. The Revision record that this section follows is the lead check `Revision/lead_checks/charge_conjugation_and_u1.py` with its report `Revision/lead_checks/reports/charge-conjugation-and-u1.json` (12 of 12 checks passed); Notebook 05c reproduces ten of its checks.

The field equation. Chapter 7 derives, from the Lagrangian of the Revision record, the field equation of both fields of the author,

$$\gamma^\mu D_\mu \Psi = V \Psi, \qquad V = m + U'(S).$$

Here the index μ runs over the eight coordinates and a repeated index is summed (the **sum convention** of Chapter 1); $\gamma^\mu = e^\mu{}_a \gamma^a$ are the gammas of the curved space, made from the constant gammas γ^a with the **vielbein** $e^\mu{}_a$, a set of real factors that Chapter 6 computes from the author's metric; $D_\mu = \partial_\mu + \Omega_\mu$ is the **covariant derivative**, the partial derivative ∂_μ along the coordinate μ plus a 16×16 matrix Ω_μ, the **spin connection**, which is a combination of the generators S^{ab} with real coefficients (Chapter 6); m is the mass; $U(S)$ is the self-interaction, a function of the scalar $S = \bar\Psi\Psi$, with derivative $U'(S)$; for the record's choice $U = \frac{\lambda}{2}S^2$ one has $V = m + \lambda S$. Only one property of this equation is needed now: **every matrix in it is real.** The gammas are real (Section 5.2), the vielbein factors are real, the number V is real, and the Revision record checks that every entry of every Ω_μ is a real expression (check `spinor_connection_real` of the lead report).

What we want. We look for a constant 16×16 matrix $\mathcal{C}$ such that, whenever Ψ solves the field equation, the column

$$\Psi^c = \mathcal{C}\,\bar\Psi^T$$

solves the field equation of the same form, either with the same V (**same mass**) or with $-V$ (**mass reversed**). Here $\bar\Psi = \Psi^\dagger C$ is the Dirac adjoint of Section 5.4, a row, and $\bar\Psi^T$ is that row turned into a column. This wish is not yet the definition; the definition comes after Step 4, below.

From the wish to an equation for a matrix, line by line. First the column $\bar\Psi^T$:

$$\bar\Psi^T = (\Psi^\dagger C)^T = C^T(\Psi^\dagger)^T = C\Psi^*.$$

The first step is the definition of $\bar\Psi$; the second is the rule $(XY)^T = Y^T X^T$; the third uses that the transpose of the row $\Psi^\dagger$ is the column Ψ^* of the conjugated components, and $C^T = C$ (property (C2) of Section 5.4). So $\Psi^c = \mathcal{C}C\Psi^*$. Write $M = \mathcal{C}C$; then $\Psi^c = M\Psi^*$, and since $CC = 1$ (C3), $\mathcal{C} = MC$.

- Step 1 (conjugate the equation). The complex conjugate of a product is the product of the complex conjugates, a real factor is its own conjugate, and the derivative of the conjugate is the conjugate of the derivative (the coordinates are real). So the conjugate of $\gamma^\mu D_\mu \Psi = V\Psi$ is $\gamma^\mu D_\mu \Psi^* = V\Psi^*$: **the column Ψ^* solves the same equation.**
- Step 2 (multiply by a constant matrix). Multiply from the left by M: $M\gamma^\mu D_\mu \Psi^* = V\,M\Psi^*$; the number V may stand on either side of M.
- Step 3 (a condition on M that is enough). Suppose that, with one sign s ($+1$ or -1) for all eight directions,

$$M\gamma^a = s\,\gamma^a M \qquad \text{for } a = x0, \ldots, x7.$$

Then $M\gamma^\mu = \sum_a e^\mu{}_a M\gamma^a = s\,\gamma^\mu M$, because the vielbein factors are numbers. M commutes with every product of two gammas: $M\gamma^a\gamma^b = s\,\gamma^a M\gamma^b = s^2\,\gamma^a\gamma^b M = \gamma^a\gamma^b M$ (the condition used twice, then $s^2 = 1$). The spin connection is a combination of the $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$, so $M\Omega_\mu = \Omega_\mu M$; and a constant matrix commutes with a derivative, $M\partial_\mu = \partial_\mu M$. Hence $MD_\mu = D_\mu M$.

- Step 4 (the new field). With Step 3, $M\gamma^\mu D_\mu \Psi^* = s\,\gamma^\mu M D_\mu \Psi^* = s\,\gamma^\mu D_\mu (M\Psi^*)$. So the equation of Step 2 reads $s\,\gamma^\mu D_\mu (M\Psi^*) = V\,(M\Psi^*)$. Multiplying by s and using $s^2 = 1$:

$$\gamma^\mu D_\mu (M\Psi^*) = s\,V\,(M\Psi^*).$$

The new field $M\Psi^*$ solves the equation with V when $s = +1$ and with $-V$ when $s = -1$. Because the gammas are real, $(\gamma^a)^* = \gamma^a$, and the condition of Step 3 can be written $M(\gamma^a)^* = s\,\gamma^a M$; this is the form that the Revision record solves (it would also be the right form for complex gammas). One more remark about $V = m + \lambda S$, which depends on the field through S: Section 5.29 shows that for commuting components both matrices found below give a new field with the same S as Ψ. Then $sV = s\,m + s\,\lambda S$, so the new field solves the field equation with the parameters (m, λ) when $s = +1$ and $(-m, -\lambda)$ when $s = -1$. For a free field ($\lambda = 0$) only the mass matters. (For anticommuting components the table of Section 5.29 shows that the scalar changes its sign; this affects only the self-interaction term, and Section 5.34 treats the quantised field.)

Definition. A **charge-conjugation matrix** is a constant 16×16 matrix $\mathcal{C}$ for which the matrix $M = \mathcal{C}C$ obeys the **intertwiner condition**

$$M(\gamma^a)^* = s\,\gamma^a M \qquad \text{for } a = x0, \ldots, x7,$$

with one sign s ($+1$ or -1) for all eight directions. It is called **same mass** when $s = +1$ and **mass reversed** when $s = -1$. (The name: in the words of Section 5.11, M is an intertwiner from the list of the conjugated gammas $(\gamma^a)^*$ to the list of the matrices $s\,\gamma^a$.) This is the definition of the Revision record: its checks `intertwiners_same_mass` and `intertwiners_reversed_mass` solve exactly this condition. Steps 1 to 4 prove that every charge-conjugation matrix does what we wanted: for every solution Ψ, the column $\Psi^c = M\Psi^*$ solves the field equation with sV.

Why the definition is the condition and not the wish. Steps 1 to 4 show that the condition is enough for the wish. The converse is not true in general: a matrix can turn every solution into a solution without obeying the condition. Here is an example, line by line, for $V = 0$ (mass $m = 0$ and no self-interaction, $\lambda = 0$), a case that the field equation allows. Take $M = P_- = \frac{1}{2}(1 - \Gamma)$, the chiral projector of Section 5.5, so that $\mathcal{C} = P_- C$.

1. P_- is a combination of 1 and of Γ, a product of eight gammas, so it is even, and by (X3) it commutes with every S^{ab}, hence with every Ω_μ; as a constant matrix it commutes with every ∂_μ. So $P_- D_\mu = D_\mu P_-$.
2. Section 5.5 proved $\gamma^a P_- = P_+ \gamma^a$ for every a. Multiplying by the vielbein numbers $e^\mu{}_a$ and adding over a gives $\gamma^\mu P_- = P_+ \gamma^\mu$.
3. For every solution Ψ: $\gamma^\mu D_\mu (P_- \Psi^*) = \gamma^\mu P_- D_\mu \Psi^* = P_+ \gamma^\mu D_\mu \Psi^* = P_+ (V \Psi^*) = 0$. The first step is line 1, the second is line 2, the third is Step 1 (the column Ψ^* solves the equation), the fourth is $V = 0$.

So $P_-\Psi^*$ solves the equation with $V = 0$, which is also $-V$: the matrix $P_- C$ fulfils the wish. It does not obey the condition, for either sign. Proof: Section 5.5 also gives $P_- \gamma^a = \gamma^a P_+$ (the same rule with the halves exchanged). If $P_- \gamma^a = s\,\gamma^a P_-$ held, then $\gamma^a P_+ = s\,\gamma^a P_-$; multiplying from the

left by $\eta_{aa}\gamma^a$ and using $\eta_{aa}\gamma^a\gamma^a = \eta_{aa}\eta_{aa}1 = 1$ gives $P_+ = s\,P_-$; multiplying this from the right by P_+ gives $P_+P_+ = s\,P_-P_+$, that is $P_+ = 0$ (because $P_+P_+ = P_+$ and $P_-P_+ = 0$, Section 5.5). But $P_+ = \mathrm{diag}(0, 1_8)$ is not zero: a contradiction. So with the wish as the definition, the count "exactly two" of Theorem CC would be false for $V = 0$; with the condition as the definition, it is proved below. For $V \neq 0$ this book does not study whether some matrix outside the condition fulfils the wish in the author's metric (OPEN); nothing in this chapter depends on that question, because the Revision record and this chapter use the condition.

Theorem CC (the two charge-conjugation matrices).

- (a) The solutions M of $M(\gamma^a)^* = +\gamma^a M$ for all a are the multiples of 1; the solutions of $M(\gamma^a)^* = -\gamma^a M$ for all a are the multiples of Γ.
- (b) Hence, by the definition above, there are exactly two charge-conjugation matrices, up to a factor: $\mathcal{C}_+ = C$ (same mass), with $\Psi^c = \mathcal{C}_+\bar{\Psi}^T = \Psi^*$, and $\mathcal{C}_- = \Gamma C$ (mass reversed), with $\Psi^c = \mathcal{C}_-\bar{\Psi}^T = \Gamma\Psi^*$. By Steps 1 to 4, the first maps every solution to a solution with V, the second to a solution with $-V$.
- (c) They obey $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$ and $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = +(\gamma^a)^T$ for every a; both are real and symmetric, and $\mathcal{C}_+\mathcal{C}_+ = \mathcal{C}_-\mathcal{C}_- = 1$.

Proof of (a). The gammas are real, so the condition reads $M\gamma^a = s\,\gamma^a M$. For $s = +1$, M commutes with all eight gammas, and by Theorem P of Section 5.13 it is a multiple of 1. For $s = -1$, M anticommutes with every gamma. Then ΓM commutes with every gamma:

$$\gamma^a(\Gamma M) = -\Gamma\gamma^a M = -\Gamma(-M\gamma^a) = (\Gamma M)\gamma^a.$$

The first step is (X2) of Section 5.5 (Γ anticommutes with every gamma); the second is the condition, read as $\gamma^a M = -M\gamma^a$; the third collects the two signs. By Theorem P, $\Gamma M = \lambda 1$ for some number λ; multiplying from the left by Γ and using $\Gamma\Gamma = 1$ (X1) gives $M = \lambda\Gamma$. Conversely, 1 commutes with every gamma and Γ anticommutes with every gamma (X2), so both are solutions. The two solution spaces are therefore one-dimensional, spanned by 1 and by Γ.

Proof of (b). By the definition, $\mathcal{C}$ is a charge-conjugation matrix exactly when $M = \mathcal{C}C$ solves one of the two conditions of (a); then $\mathcal{C} = MC$ (above), and by (a) M is a multiple of 1 or of Γ. $M = 1$ gives $\mathcal{C}_+ = C$ and $\Psi^c = 1\,\Psi^* = \Psi^*$; $M = \Gamma$ gives $\mathcal{C}_- = \Gamma C$ and $\Psi^c = \Gamma\Psi^*$.

Proof of (c). For $\mathcal{C}_+ = C$, with $C^{-1} = C$ (C3), $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = C\gamma^a C = -(\gamma^a)^T$ by (C5). For $\mathcal{C}_-$, the inverse of a product is the product of the inverses in the reverse order, so $\mathcal{C}_-^{-1} = C^{-1}\Gamma^{-1} = C\Gamma$ by (C3) and (X1). Then

$$\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = C\Gamma\gamma^a\Gamma C = C(-\gamma^a\Gamma)\Gamma C = -C\gamma^a C = +(\gamma^a)^T.$$

The second step is (X2), the third is $\Gamma\Gamma = 1$ (X1), the fourth is (C5). Both matrices are products of real matrices, hence real. Symmetry: $C^T = C$ is (C2); $(\Gamma C)^T = C^T\Gamma^T = C\Gamma = \Gamma C$ by $(XY)^T = Y^T X^T$, (C2), (X6) and (X5). Squares: $CC = 1$ is (C3); $\Gamma C\Gamma C = \Gamma\Gamma CC = 1$ by (X5), (X1) and (C3). □

The shape of $\mathcal{C}_-$. $\Gamma = \mathrm{diag}(-1_8, 1_8)$ and $C = \mathrm{diag}(-\sigma, \sigma)$ (Sections 5.4 and 5.5), so $\Gamma C = \mathrm{diag}(\sigma, \sigma)$: multiplying by Γ from the left reverses the sign of the rows 1 to 8 (the block rule of Section 5.5). The table of Section 5.6 lists ΓC next to C.

The same two matrices from the transposition rules. Many books define a charge-conjugation matrix by the condition $\mathcal{C}^{-1}\gamma^a\mathcal{C} = \zeta(\gamma^a)^T$ with a sign ζ, that is $\gamma^a\mathcal{C} = \zeta\,\mathcal{C}(\gamma^a)^T$. Solving this directly gives the same two matrices: the solutions X of $\gamma^a X = \zeta X(\gamma^a)^T$ (all a) are the multiples of C for $\zeta = -1$ and of ΓC for $\zeta = +1$. Proof: (C5) with $C^{-1} = C$ says $(\gamma^a)^T = -C\gamma^a C$, so the equation reads $\gamma^a X = -\zeta XC\gamma^a C$; multiplying from the right by C (and $CC = 1$) gives $\gamma^a(XC) = -\zeta(XC)\gamma^a$. By part (a), $XC = \lambda 1$ when $-\zeta = +1$ and $XC = \lambda\Gamma$ when $-\zeta = -1$; multiplying from the right by C gives $X = \lambda C$ or $X = \lambda\Gamma C$.

What $\mathcal{C}_+$ does to a real field. For every field $\mathcal{C}_+\bar{\Psi}^T$ is the column Ψ^*. For a complex field this column differs from Ψ; for a real field it is Ψ itself. So the same-mass charge conjugation leaves every real field unchanged: a real field is its own $\mathcal{C}_+$ conjugate. Section 5.29 shows what this means for the charge, and which real map is not trivial.

A worked example: the free field along the time. Take flat space, no self-interaction, and a field that depends only on the time $x4$. The field equation keeps only the term with $\mu = x4$, and in flat space $\gamma^{(x4)}$ is the constant gamma of $x4$:

$$\gamma^{(x4)}\frac{d\Psi}{dx4} = m\Psi.$$

Multiplying from the left by $-\gamma^{(x4)}$ and using $\gamma^{(x4)}\gamma^{(x4)} = -1$ gives $d\Psi/dx4 = -m\gamma^{(x4)}\Psi$. For any constant column Ψ_0 the column

$$\Psi(x4) = \cos(m\,x4)\,\Psi_0 - \sin(m\,x4)\,\gamma^{(x4)}\Psi_0$$

solves it. Check, line by line: by the chain rule, $d\cos(m\,x4)/dx4 = -m\sin(m\,x4)$ and $d\sin(m\,x4)/dx4 = m\cos(m\,x4)$, so

$$\frac{d\Psi}{dx4} = -m\sin(m\,x4)\,\Psi_0 - m\cos(m\,x4)\,\gamma^{(x4)}\Psi_0;$$

multiplying from the left by $\gamma^{(x4)}$ and using $\gamma^{(x4)}\gamma^{(x4)} = -1$ in the second term,

$$\gamma^{(x4)}\frac{d\Psi}{dx4} = -m\sin(m\,x4)\,\gamma^{(x4)}\Psi_0 + m\cos(m\,x4)\,\Psi_0 = m\Psi.$$

Now the two conjugates. Since cos, sin and $\gamma^{(x4)}$ are real,

$$\Psi^*(x4) = \cos(m\,x4)\,\Psi_0^* - \sin(m\,x4)\,\gamma^{(x4)}\Psi_0^*:$$

the same formula with the starting column Ψ_0^*, so Ψ^* solves the equation with the **same mass** m. For the other, put $\Phi_0 = \Gamma\Psi_0^*$ and use $\Gamma\gamma^{(x4)} = -\gamma^{(x4)}\Gamma$ (X2):

$$\Gamma\Psi^*(x4) = \cos(m\,x4)\,\Phi_0 + \sin(m\,x4)\,\gamma^{(x4)}\Phi_0 = \cos(-m\,x4)\,\Phi_0 - \sin(-m\,x4)\,\gamma^{(x4)}\Phi_0,$$

where the last step uses $\cos(-y) = \cos y$ and $\sin(-y) = -\sin y$. This is the solution formula with m replaced by $-m$: $\Gamma\Psi^*$ solves the equation with the **mass reversed**. Notebook 05c checks both statements exactly with sympy, for a general mass m, and numerically on 801 time points (figures 3 and 4 of the notebook).

statement	**status**	**where it is verified**
the gammas, C and the S^{ab} are real; B is purely imaginary and Hermitian	PROVED	`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, checks `representation_real` and `B_imaginary_hermitian`; Notebook 05c, In [2]
every entry of the spin connection Ω_μ is real, so $\gamma^\mu D_\mu$ is a real operator	PROVED in the record	the same report, check `spinor_connection_real`
Steps 1 to 4: a matrix M with $M(\gamma^a)^* = s\,\gamma^a M$ for all a maps every solution Ψ to the solution $M\Psi^*$ with sV	PROVED	Section 5.28 (above); the record's detail text of check `charge_conjugation_matrix_minus` states the case $s = -1$
Theorem CC (a): the solutions of $M(\gamma^a)^* = \pm\gamma^a M$ are the multiples of 1 and of Γ	PROVED; the exact solution of the 2048 equations COMPUTED	the same report, checks `intertwiners_same_mass` and `intertwiners_reversed_mass`; Notebook 05c, In [3] and In [4]
Theorem CC (b), (c): with the intertwiner condition as the definition, exactly two charge-conjugation matrices up to a factor, $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$, and their transposition rules	PROVED	the same report, checks `charge_conjugation_matrix_plus` and `charge_conjugation_matrix_minus`; Notebook 05c, In [5] and In [7]
for $V = 0$ the matrix $P_- C$ maps every solution to a solution without obeying the condition	PROVED	Section 5.28 (above)
for $V \neq 0$: does any matrix outside the condition map every solution to a solution?	OPEN (not studied; nothing in this chapter depends on it)	none
the transposition equations have only the solutions C and ΓC	PROVED; COMPUTED exactly	Notebook 05c, In [5] (its own computation)
the free solution: Ψ and Ψ^* solve with m, $\Gamma\Psi^*$ with $-m$	PROVED; COMPUTED exactly (sympy) and numerically (residuals below 10^{-12})	Notebook 05c, In [8] and In [9] (its own computation)

5.29 The bilinears, the reality conditions and real fields

Anticommuting numbers. The components of the field dirac16complex00 are ordinary (complex) numbers, which **commute**: $\Psi_r\Psi_c = \Psi_c\Psi_r$. The components of dirac16complex are **anticommuting** numbers, also called **Grassmann numbers** after the mathematician who introduced them: exchanging two of them in a product costs a sign, $\Psi_r\Psi_c = -\Psi_c\Psi_r$, and in particular $\Psi_r\Psi_r = -\Psi_r\Psi_r$, so $\Psi_r\Psi_r = 0$. Chapter 7 builds them from zero. Here only the exchange rule is used, and the conjugated components Ψ_r^* are treated as further anticommuting numbers. Write $\epsilon = +1$ for commuting and $\epsilon = -1$ for anticommuting components: exchanging two components costs the factor ϵ.

How a bilinear changes, line by line. For a fixed matrix K the **bilinear** $\Psi^\dagger K\Psi = \sum_{r,c}\Psi_r^* K_{rc}\Psi_c$ is a single number. Replace Ψ by $\Psi' = M\Psi^*$ with a real matrix M.

1. $(\Psi')^\dagger = (M\Psi^*)^\dagger = (\Psi^*)^\dagger M^\dagger = \Psi^T M^\dagger$, by the rule $(XY)^\dagger = Y^\dagger X^\dagger$ and because conjugating twice gives the number back. So $\Psi'^\dagger K\Psi' = \Psi^T M^\dagger K M\Psi^* = \sum_{r,c}\Psi_r\,(M^\dagger KM)_{rc}\,\Psi_c^*$.
2. Exchange the two factors Ψ_r and Ψ_c^*; this costs the factor ϵ: $\sum_{r,c}\epsilon\,\Psi_c^*\,(M^\dagger KM)_{rc}\,\Psi_r$.
3. Rename the summation letters, $r \leftrightarrow c$: $\sum_{r,c}\Psi_r^*\,\epsilon(M^\dagger KM)_{cr}\,\Psi_c = \Psi^\dagger K'\Psi$ with $K' = \epsilon\,(M^\dagger KM)^T$.

So **the bilinear with the matrix K turns into the bilinear with the matrix $K' = \epsilon(M^\dagger KM)^T$.** If $K' = +K$ the bilinear is kept, if $K' = -K$ it is reversed.

The signs of S and J. The scalar has $K = C$, the currents $K = -iC\gamma^a$ (Section 5.4). The two maps have $M = 1$ ($\mathcal{C}_+$) and $M = \Gamma$ ($\mathcal{C}_-$); both are real and symmetric, so $M^\dagger = M$.

- $\mathcal{C}_+$, scalar: $K' = \epsilon C^T = \epsilon C$ by (C2). So $S \to \epsilon S$.
- $\mathcal{C}_+$, currents: $(-iC\gamma^a)^T = -i(C\gamma^a)^T = -i(-C\gamma^a) = iC\gamma^a$ by (C4), so $K' = \epsilon\,iC\gamma^a = -\epsilon K$. So $J^a \to -\epsilon J^a$.
- $\mathcal{C}_-$, scalar: $\Gamma C\Gamma = \Gamma^T C\Gamma = C$ by (X6) and (X7), so $K' = \epsilon C^T = \epsilon C$. So $S \to \epsilon S$.
- $\mathcal{C}_-$, currents: $\Gamma(-iC\gamma^a)\Gamma = -i\,\Gamma^T C\gamma^a\Gamma = -i(-C\gamma^a) = iC\gamma^a$ by (X6) and (X8), and its transpose is $i(C\gamma^a)^T = -iC\gamma^a = K$ by (C4). So $K' = \epsilon K$ and $J^a \to \epsilon J^a$.

map	**components**	S **becomes**	**every** J^a **becomes**
$\mathcal{C}_+$ $(M = 1)$	commuting $(\epsilon = +1)$	$+S$	$-J^a$
$\mathcal{C}_+$ $(M = 1)$	anticommuting $(\epsilon = -1)$	$-S$	$+J^a$
$\mathcal{C}_-$ $(M = \Gamma)$	commuting $(\epsilon = +1)$	$+S$	$+J^a$
$\mathcal{C}_-$ $(M = \Gamma)$	anticommuting $(\epsilon = -1)$	$-S$	$-J^a$

This is the table that the Revision record measured (check `bilinears_under_charge_conjugation`, the part of its detail text after the word measured); Notebook 05c reproduces it exactly. For

the anticommuting rows the table is a statement about classical Grassmann components. For the quantised field the question is decided by an operator computation, which Section 5.34 carries out: after normal ordering the quantised field has exactly the signs of the anticommuting rows. The detail text of the record's check says the same: normal ordering adds no sign, because it subtracts the vacuum value, which is a number (the record makes no Fock-space computation; Notebook 05e makes one). An earlier version of that detail text said instead that normal ordering supplies one more sign for each bilinear; the derivation of Section 5.34 and the computation of Notebook 05e contradicted it, and the record was corrected on 2026-10-08, so the contradiction is resolved.

What the table means for the commuting field. For dirac16complex00, whose components are commuting complex numbers, $\mathcal{C}_+$ keeps the mass and the scalar and reverses every current, in particular the charge density $J^{(x4)} = \Psi^\dagger B \Psi$. So it turns every solution of charge Q into a solution of the same mass and charge $-Q$: it is the antiparticle map of this field. $\mathcal{C}_-$ reverses the mass and keeps the charge. Notebook 05c shows this on the free solution of Section 5.28 with a fixed complex column Ψ_0: the solution has $S = -2$ and $J^{(x4)} = -6$ at every time; Ψ^* has $S = -2$ and $J^{(x4)} = +6$; $\Gamma\Psi^*$ has $S = -2$ and $J^{(x4)} = -6$ (figure 5 of the notebook).

The reality (Majorana) conditions. A field equal to its own conjugate obeys $\Psi = M\Psi^*$; such a requirement is called a **Majorana** or **reality condition**, after the physicist who first used one. Is it a sensible requirement? Take the complex conjugate of the condition, $\Psi^* = M^*\Psi$, and put it back into the condition: $\Psi = MM^*\Psi$. If $MM^* = 1$ this is automatically true, and the condition is called **consistent**; otherwise it forces further conditions on Ψ (possibly $\Psi = 0$). For $M = 1$, $MM^* = 1$: the condition $\Psi = \Psi^*$ says that the field is real. For $M = \Gamma$, $\Gamma\Gamma^* = \Gamma\Gamma = 1$: the condition $\Psi = \Gamma\Psi^*$ says, component by component, $\Psi_r = -\Psi_r^*$ for $r = 1, \ldots, 8$ (where $\Gamma = -1$), so these components are purely imaginary, and $\Psi_r = \Psi_r^*$ for $r = 9, \ldots, 16$, which are real. Both conditions are consistent (record check `majorana_conditions_consistent`).

Do they survive the time evolution? Consistency is a statement about one moment. For the free solution of Section 5.28: if Ψ_0 is real, every factor of the solution is real and the field stays real for ever. If instead $\Psi_0 = \Gamma\Psi_0^*$, then, using $\Gamma\gamma^{(x4)} = -\gamma^{(x4)}\Gamma$ (X2) and $\Gamma\Psi_0^* = \Psi_0$,

$$\Gamma\Psi(x4)^* = \cos(m\,x4)\,\Gamma\Psi_0^* - \sin(m\,x4)\,\Gamma\gamma^{(x4)}\Psi_0^* = \cos(m\,x4)\,\Psi_0 + \sin(m\,x4)\,\gamma^{(x4)}\Psi_0,$$

and subtracting this from $\Psi(x4)$:

$$\Psi(x4) - \Gamma\Psi(x4)^* = -2\sin(m\,x4)\,\gamma^{(x4)}\Psi_0.$$

Its length is $2|\sin(m\,x4)|$ times the length of Ψ_0, because $\gamma^{(x4)}$ is a signed permutation matrix and keeps lengths. So the condition $\Psi = \Gamma\Psi^*$ holds at all times only for $m = 0$. This is what "mass reversed" means: a field equal to its own $\mathcal{C}_-$ image would have to solve the equation with V and with $-V$ at the same time (figure 7 of Notebook 05c).

Real fields. For a real field with commuting components, three facts hold.

- (F1) **The currents vanish.** $J^a = \Psi^\dagger(-iC\gamma^a)\Psi = -i\,\Psi^T A\Psi$ with $A = C\gamma^a$, which is antisymmetric by (C4). For every real column u and every antisymmetric A, $u^TAu = 0$: the number u^TAu equals its own transpose, $u^TAu = (u^TAu)^T = u^TA^Tu = -u^TAu$ (the rule $(XYZ)^T = Z^TY^TX^T$, then $A^T = -A$), and a number equal to minus itself is 0. So a real commuting field carries no current and no charge.

- (F2) $\mathcal{C}_+$ **acts as the identity.** $\Psi^c = \Psi^* = \Psi$: a real field is its own same-mass conjugate.
- (F3) **The real matrix Γ reverses the mass.** The map $\Psi \to \Gamma\Psi$ (no conjugation) takes real fields to real fields. It keeps the scalar, $(\Gamma\Psi)^T C(\Gamma\Psi) = \Psi^T\Gamma^T C\Gamma\Psi = \Psi^T C\Psi$ by (X7), and it reverses every kinetic matrix, $\Gamma^T C\gamma^a\Gamma = -C\gamma^a$ by (X8). In the field equation: Γ commutes with Ω_μ (an even matrix, (X3)) and with ∂_μ, so $\Gamma D_\mu = D_\mu\Gamma$, and Γ anticommutes with every γ^μ; hence $\gamma^\mu D_\mu(\Gamma\Psi) = -\Gamma\gamma^\mu D_\mu\Psi = -V\,(\Gamma\Psi)$. With $S(\Gamma\Psi) = S(\Psi)$ and $V = m + \lambda S$, the field $\Gamma\Psi$ solves the field equation with $(m, \lambda) \to (-m, -\lambda)$. This is the pairing theorem T1 of the Revision record, which Chapter 18 proves for the Lagrangian; the same calculation holds for complex fields, where Γ also reverses every current.

By Theorem CC, every matrix M that obeys the intertwiner condition $M(\gamma^a)^* = s\,\gamma^a M$ of Section 5.28 (one sign s for all a) is a multiple of 1 or of Γ. On a real field $\Psi^* = \Psi$, so $M\Psi^* = M\Psi$: a multiple of 1 only rescales the field, and among the matrices that obey the condition the only nontrivial one is Γ (up to a factor), which reverses the mass. (Matrices outside the condition are not charge-conjugation matrices; for $V = 0$ some of them, such as P_-, also map solutions to solutions, Section 5.28.) The Revision record states this as: for real fields the matter–antimatter map is the matrix Γ together with $m \to -m$ (check `real_fields_charge_conjugation`). A real commuting field has no charge to reverse (F1); what the map exchanges is the sign of the mass.

statement	**status**	**where it is verified**
the bilinear rule $K' = \epsilon(M^\dagger K M)^T$ and the sign table of S and J	PROVED; COMPUTED exactly	`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, check `bilinears_under_charge_conjugation` (its measured table); Notebook 05c, In [14]
on the free solution: $S = -2$ for all three fields, $J^{(x4)} = -6, +6, -6$	COMPUTED (exact at $x4 = 0$; constant to 10^{-12})	Notebook 05c, In [12] (its own computation)
both reality conditions are consistent ($MM^* = 1$)	PROVED	the same report, check `majorana_conditions_consistent`; Notebook 05c, In [16]
the real condition is kept in time; $\Psi = \Gamma\Psi^*$ is violated by twice the size of $\sin(m\,x4)$	PROVED; COMPUTED for $m = 1,\ 0.5,\ 0$	Notebook 05c, In [17] (its own computation)
real fields: (F1), (F2), (F3)	PROVED	the same report, check `real_fields_charge_conjugation`; Notebook 05c, In [18]

statement	status	where it is verified
normal ordering adds no sign: the quantised field has the signs of the anticommuting rows	PROVED (Section 5.34); COMPUTED on the Fock space; the record states the same (its detail text was corrected on 2026-10-08; the earlier remark that normal ordering adds a sign is withdrawn, so the contradiction is resolved)	Notebook 05e, In [16] to In [18]; the record's detail text of check `bilinears_under_charge_conjugation`

5.30 Example: Notebook 05c computes the charge-conjugation matrices

Notebook 05c turns Sections 5.28 and 5.29 into exact computations on the author's gammas. It reads the gammas from `Revision/algebra/gammas.json` and the lead report `Revision/lead_checks/reports/charge-conjugation-and-u1.json`; it writes the conditions $M(\gamma^a)^* = \pm\gamma^a M$ as two systems of 2048 linear equations for the 256 entries of M and solves them exactly, confirming the result with the zero eigenvalues of $A^T A$; it builds $\mathcal{C}_+$ and $\mathcal{C}_-$ and checks their transposition rules, and solves the transposition equations directly; it applies both maps to a complex column and to the free solution along $x4$, exactly with sympy and on 801 time points; it computes S and $J^{(x4)}$ of the three fields and the full sign table for commuting and anticommuting components; it checks the reality conditions and follows them in time; it treats real fields; and it checks which conjugation keeps the anticommutator of the quantised field. Ten of its checks reproduce checks of the lead report. It draws nine figures and ends with the line ALL 20 CHECKS PASSED (notebook 05c).

5.31 How to run Notebook 05c

Step 1. What this notebook does and what it needs.

Notebook 05c (The charge-conjugation matrices calC_+ = C and calC_- = Gamma C) is the file `Revision/textbook/notebooks/05c_charge_conjugation.ipynb` of the repository Dirac_claude. It derives charge conjugation as a MATRIX map: it solves exactly the linear equations M conj(gamma^a) = s gamma^a M for all 16 by 16 matrices M (s = +1: same mass, s = -1: mass reversed; conj is the complex conjugate), finds the two one-dimensional solution spaces spanned by 1 and by Gamma, builds the charge-conjugation matrices calC_+ = C and calC_- = Gamma C, checks their transposition rules, applies them to explicit solutions of the free field equation along the time x4, computes how the bilinears S and J change for commuting and anticommuting components, shows that for REAL fields calC_+ acts as the identity while the real matrix Gamma reverses the mass, and checks which map preserves the canonical anticommutator of the quantised field. Nine teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private

Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05c_charge_conjugation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05c_charge_conjugation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 05c_charge_conjugation.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- Revision/textbook/figures/05c.captions.json
- Revision/textbook/figures/05c_1_solution_spaces.png
- Revision/textbook/figures/05c_2_conjugation_matrices.png
- Revision/textbook/figures/05c_3_conjugate_solutions.png
- Revision/textbook/figures/05c_4_which_mass.png
- Revision/textbook/figures/05c_5_bilinears.png
- Revision/textbook/figures/05c_6_sign_table.png
- Revision/textbook/figures/05c_7_reality_in_time.png
- Revision/textbook/figures/05c_8_real_field.png
- Revision/textbook/figures/05c_9_quantum_b.png

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the nine figure files of notebook 05c exist
ALL 20 CHECKS PASSED (notebook 05c)
```

and the notebook must show 9 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file Revision/textbook/notebooks/05c_charge_conjugation.ipynb inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 05c_charge_conjugation.ipynb
python -m nbconvert --execute --inplace 05c_charge_conjugation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra` or in the folder `Revision/lead_checks`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

5.32 Notebook 05c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/05c_charge_conjugation.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes
```

```
**Charge conjugation is a matrix.** The author's gamma matrices are *real*. For a
*real* field ($\Psi^* = \Psi$) plain complex conjugation changes nothing: it is
the identity, and it cannot exchange matter and antimatter. Charge conjugation must
therefore be defined by a **matrix** $\mathcal{C}$:

$$\Psi^c = \mathcal{C}\, \bar\Psi^T = \mathcal{C} C \Psi^* ,$$

where $\bar\Psi = \Psi^\dagger C$ is the Dirac adjoint and $C$ the charge matrix
(the author's sigma16). As in the Revision record, a charge-conjugation matrix is
defined by a condition that $M = \mathcal{C}C$ must obey together with every gamma
(section 4 derives it); this notebook finds every such matrix exactly. It

1. solves the linear equations $M(\gamma^a)^* = s\,\gamma^a M$ ($a = x0, \dots,
   x7$) for all $16 \times 16$ matrices $M = \mathcal{C}C$: for $s = +1$ (same
   mass) the solutions are the multiples of $1$, for $s = -1$ (mass reversed) the
   multiples of $\Gamma$;
2. builds the two charge-conjugation matrices $\mathcal{C}_+ = C$ and
   $\mathcal{C}_- = \Gamma C$ and checks $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ =
   -(\gamma^a)^T$ and $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = +(\gamma^a)^T$;
3. applies them to explicit solutions of the free field equation along the time
   $x4$ and shows which mass each image solves with;
4. computes how the scalar $S = \bar\Psi\Psi$ and the current $J^a = -i\bar\Psi
   \gamma^a\Psi$ change, for commuting and for anticommuting components;
5. treats **real fields**: $J = 0$, $\mathcal{C}_+$ acts as the identity, and of
   the two matrices the one that changes a real field is $\mathcal{C}_- = \Gamma
   C$, whose map $\Psi \to \Gamma\Psi$ reverses the mass;
6. checks which map preserves the anticommutator $\{\Psi, \Psi^\dagger\} =
   B\,\delta$ of the quantised field: $\Psi \to \Gamma\Psi^{\dagger T}$ does,
   $\Psi \to \Psi^{\dagger T}$ does not.

Ten checks reproduce checks of the Revision record
`Revision/lead_checks/reports/charge-conjugation-and-u1.json`. Nine teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 05c (The charge-conjugation matrices calC_+ = C and calC_- = Gamma C) is the
file `Revision/textbook/notebooks/05c_charge_conjugation.ipynb` of the repository
Dirac_claude. It derives charge conjugation as a MATRIX map: it solves exactly the linear
equations M conj(gamma^a) = s gamma^a M for all 16 by 16 matrices M (s = +1: same mass, s
= -1: mass reversed; conj is the complex conjugate), finds the two one-dimensional
solution spaces spanned by 1 and by Gamma, builds the charge-conjugation matrices calC_+
= C and calC_- = Gamma C, checks their transposition rules, applies them to explicit
solutions of the free field equation along the time x4, computes how the bilinears S and
J change for commuting and anticommuting components, shows that for REAL fields calC_+
acts as the identity while the real matrix Gamma reverses the mass, and checks which map
preserves the canonical anticommutator of the quantised field. Nine teaching plots. It
needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
```

mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05c_charge_conjugation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05c_charge_conjugation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 05c_charge_conjugation.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/05c.captions.json‘
- ‘Revision/textbook/figures/05c_1_solution_spaces.png‘
- ‘Revision/textbook/figures/05c_2_conjugation_matrices.png‘
- ‘Revision/textbook/figures/05c_3_conjugate_solutions.png‘
- ‘Revision/textbook/figures/05c_4_which_mass.png‘
- ‘Revision/textbook/figures/05c_5_bilinears.png‘
- ‘Revision/textbook/figures/05c_6_sign_table.png‘
- ‘Revision/textbook/figures/05c_7_reality_in_time.png‘
- ‘Revision/textbook/figures/05c_8_real_field.png‘
- ‘Revision/textbook/figures/05c_9_quantum_b.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the nine figure files of notebook 05c exist
ALL 20 CHECKS PASSED (notebook 05c)
```

and the notebook must show 9 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/05c_charge_conjugation.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 05c_charge_conjugation.ipynb
      python -m nbconvert --execute --inplace 05c_charge_conjugation.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘ or in the folder ‘Revision/lead_checks‘: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 05c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 05c (The charge-conjugation matrices calC_+ = C and calC_- = Gamma C) is the
# file Revision/textbook/notebooks/05c_charge_conjugation.ipynb of the repository
# Dirac_claude. It derives charge conjugation as a MATRIX map: it solves exactly the
# linear equations M conj(gamma^a) = s gamma^a M for all 16 by 16 matrices M (s = +1:
# same mass, s = -1: mass reversed; conj is the complex conjugate), finds the two
# one-dimensional solution spaces spanned by 1 and by Gamma, builds the
# charge-conjugation matrices calC_+ = C and calC_- = Gamma C, checks their transposition
# rules, applies them to explicit solutions of the free field equation along the time x4,
# computes how the bilinears S and J change for commuting and anticommuting components,
# shows that for REAL fields calC_+ acts as the identity while the real matrix Gamma
# reverses the mass, and checks which map preserves the canonical anticommutator of the
# quantised field. Nine teaching plots. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
```

```
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
```

```
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
```

```
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 05c_charge_conjugation.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 05c_charge_conjugation.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 05c_charge_conjugation.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/05c.captions.json
# - Revision/textbook/figures/05c_1_solution_spaces.png
# - Revision/textbook/figures/05c_2_conjugation_matrices.png
# - Revision/textbook/figures/05c_3_conjugate_solutions.png
# - Revision/textbook/figures/05c_4_which_mass.png
# - Revision/textbook/figures/05c_5_bilinears.png
# - Revision/textbook/figures/05c_6_sign_table.png
# - Revision/textbook/figures/05c_7_reality_in_time.png
# - Revision/textbook/figures/05c_8_real_field.png
# - Revision/textbook/figures/05c_9_quantum_b.png
#
# It changes no other file of the repository; running it headless or saving it in
```

```
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the nine figure files of notebook 05c exist
#     ALL 20 CHECKS PASSED (notebook 05c)
#
# and the notebook must show 9 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/05c_charge_conjugation.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 05c_charge_conjugation.ipynb
#     python -m nbconvert --execute --inplace 05c_charge_conjugation.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
```

```
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra or in the folder
#   Revision/lead_checks: the notebook was opened outside the repository, or the
#   repository is incomplete; clone the repository again and open the notebook from its
#   folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "05c"  # this notebook: chapter 05, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
```

```
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
```

```
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 05c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Spinor field** $\Psi$: at every point of spacetime a column of 16 numbers
  $\Psi_1, \dots, \Psi_{16}$ (its *components*).
- **Complex conjugate** $z^*$ of $z = a + ib$ is $a - ib$; for a column or matrix,
  every entry is conjugated. A **real** field has $\Psi^* = \Psi$.
- **Transpose** $^T$, **conjugate transpose** $^\dagger$: $M^\dagger = (M^*)^T$; for
  a column $\Psi$, $\Psi^\dagger$ is the row of the conjugated components.
- **Dirac adjoint** $\bar\Psi = \Psi^\dagger C$: a row of 16 numbers.
- **Field equation** (the Euler-Lagrange equation of the Revision record):
  $(\gamma^\mu D_\mu - V)\Psi = 0$ with $V = m + U'(S)$ a real function, $m$ the
  mass, $D_\mu = \partial_\mu + \Omega_\mu$ the covariant derivative with the
  spin connection $\Omega_\mu$.
- **Intertwiner condition** $M(\gamma^a)^* = s\,\gamma^a M$ for $a = x0, \dots,
  x7$, with one sign $s$ ($+1$ or $-1$) for all eight directions: the linear
  equations for $M = \mathcal{C}C$ derived in section 4.
- **Charge-conjugation matrix** $\mathcal{C}$: a constant matrix for which
  $M = \mathcal{C}C$ obeys the intertwiner condition (the definition of the
  Revision record); **same mass** for $s = +1$, **mass reversed** for $s = -1$.
  Section 4 shows that then $\Psi^c = \mathcal{C}\bar\Psi^T$ solves the field
  equation with $V$ (same mass) or with $-V$ (mass reversed) whenever $\Psi$
  solves it.
- **Bilinear**: an expression $\Psi^\dagger K \Psi = \sum_{r,c}\Psi_r^* K_{rc}
  \Psi_c$ with a fixed matrix $K$. The **scalar** $S = \bar\Psi\Psi = \Psi^\dagger C
  \Psi$ and the **current** $J^a = -i\bar\Psi\gamma^a\Psi = \Psi^\dagger(-iC
  \gamma^a)\Psi$; its time component $J^{(x4)} = \Psi^\dagger B\Psi$ (with $B =
  -iC\gamma^{(x4)}$) is the **charge density**.
```

- **Commuting, anticommuting components**: the components of dirac16complex00 are ordinary numbers ($\Psi_r\Psi_c = \Psi_c\Psi_r$); those of dirac16complex are *anticommuting* (Grassmann) numbers, $\Psi_r\Psi_c = -\Psi_c\Psi_r$, so that exchanging two factors in a product costs a sign. We write $\epsilon = +1$ for commuting and $\epsilon = -1$ for anticommuting components.
- **Majorana (reality) condition**: the requirement $\Psi^c = \Psi$.
- **Quantised field, anticommutator**: after quantisation the components are operators with $\{\Psi_r, \Psi_c^\dagger\} = B_{rc}\,\delta$ (δ the delta function of the positions); a conjugation of the quantised field must preserve this rule.

Text cell:

```
## 4. The physical and mathematical situation

**Deriving the condition on the matrix.** Let $\Psi$ solve
$\gamma^\mu D_\mu\Psi = V\Psi$ with real $V$. In the Revision record the gammas
$\gamma^\mu = e^\mu{}_a\gamma^a$ and the spin connection $\Omega_\mu$ are real (the
lead check spinor_connection_real), so the operator $\gamma^\mu D_\mu$ is real.

1. Take the complex conjugate of the equation. Every real factor stays, so
   $\gamma^\mu D_\mu\Psi^* = V\Psi^*$: $\Psi^*$ solves the same equation.
2. Multiply from the left by a constant matrix $M$:
   $M\gamma^\mu D_\mu\Psi^* = V M\Psi^*$.
3. Suppose $M\gamma^a = s\,\gamma^a M$ for every $a$, with one sign $s = \pm1$.
   The spin connection is built from the $S^{ab} = \tfrac12\gamma^a\gamma^b$, and
   $M\gamma^a\gamma^b = s^2\gamma^a\gamma^b M = \gamma^a\gamma^b M$, so $M$
   commutes with $\Omega_\mu$ and $MD_\mu = D_\mu M$.
4. Hence $M\gamma^\mu D_\mu\Psi^* = s\,\gamma^\mu D_\mu(M\Psi^*)$, and step 2 becomes
   $s\,\gamma^\mu D_\mu(M\Psi^*) = V(M\Psi^*)$. Multiply by $s$ ($s^2 = 1$):
   $$\gamma^\mu D_\mu(M\Psi^*) = sV\,(M\Psi^*) .$$
   The new field $M\Psi^*$ solves the equation with $V$ when $s = +1$ and with $-V$
   when $s = -1$.
5. $V = m + U'(S)$ depends on the field through $S$. For commuting components and
   the two matrices found below, the new field has the same $S$ as $\Psi$ (section
   10 computes this). With $U = \tfrac\lambda2 S^2$, $V = m + \lambda S$, so the new
   field solves the field equation with the parameters $(m, \lambda)$ when $s = +1$
   and $(-m, -\lambda)$ when $s = -1$.

Because the gammas are real, $(\gamma^a)^* = \gamma^a$, and the condition reads
$M(\gamma^a)^* = s\,\gamma^a M$: $M$ commutes ($s = +1$) or anticommutes
($s = -1$) with every gamma. The textbook form is $\Psi^c = \mathcal{C}\bar\Psi^T$:
since $\bar\Psi^T = (\Psi^\dagger C)^T = C^T\Psi^* = C\Psi^*$, we have
$\Psi^c = \mathcal{C}C\Psi^*$, so $M = \mathcal{C}C$ and $\mathcal{C} = MC^{-1} =
MC$ (because $CC = 1$).

**What the equations can give.** The matrices commuting with all gammas are the
multiples of 1 (Pin(4,4) is irreducible). If $M$ anticommutes with all gammas,
then $\Gamma M$ commutes with all of them ($\Gamma$ anticommutes with every gamma),
so $\Gamma M = c\,1$ and $M = c\,\Gamma$. The notebook solves the equations exactly
and finds exactly these two one-dimensional families.

**Why the condition is the definition.** Steps 1 to 4 show that the condition is
```

```
enough for $M\Psi^*$ to be a solution; the converse is not true in general. For
$V = 0$ (mass 0, no self-interaction) the chiral projector $P_- = \frac12(1 -
\Gamma)$ is even, so it commutes with $\Omega_\mu$ and $\partial_\mu$, and
$\gamma^a P_- = P_+\gamma^a$ with $P_+ = \frac12(1 + \Gamma)$; hence
$\gamma^\mu D_\mu(P_-\Psi^*) = P_+\gamma^\mu D_\mu\Psi^* = P_+ V\Psi^* = 0$ for
every solution. So $P_-$ maps every solution to a solution, but it is neither a
multiple of 1 nor of $\Gamma$ and obeys neither condition. With the condition as
the definition (as in the Revision record) there are exactly two
charge-conjugation matrices, up to a factor.

**A field along the time.** In flat space, a field that depends only on $x4$
obeys $\gamma^{(x4)}\, d\Psi/dx4 = m\Psi$. Since $\gamma^{(x4)}\gamma^{(x4)} = -1$,
multiplying by $-\gamma^{(x4)}$ gives $d\Psi/dx4 = -m\gamma^{(x4)}\Psi$, solved by

$$\Psi(x4) = \cos(m\,x4)\,\Psi_0 - \sin(m\,x4)\,\gamma^{(x4)}\Psi_0$$

for any constant column $\Psi_0$: the derivative is $d\Psi/dx4 = -m\sin(m\,x4)
\Psi_0 - m\cos(m\,x4)\gamma^{(x4)}\Psi_0$ (the chain rule: $d\cos(m\,x4)/dx4 =
-m\sin(m\,x4)$ and $d\sin(m\,x4)/dx4 = m\cos(m\,x4)$), and $\gamma^{(x4)}$ times it
is $-m\sin(m\,x4)\gamma^{(x4)}\Psi_0 - m\cos(m\,x4)\gamma^{(x4)}\gamma^{(x4)}\Psi_0
= -m\sin(m\,x4)\gamma^{(x4)}\Psi_0 + m\cos(m\,x4)\Psi_0 = m\Psi$ (using
$\gamma^{(x4)}\gamma^{(x4)} = -1$). The notebook uses this solution to show the two
conjugations at work.
```

Text cell:

```
## 5. The gammas, C, Gamma, B and the recorded checks

The next cell reads the gammas (exact whole numbers) from
‘Revision/algebra/gammas.json‘, builds $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}
\gamma^{(x3)}$, $\Gamma = \gamma^{(x0)}\gamma^{(x1)}\cdots\gamma^{(x7)}$ and $B =
-iC\gamma^{(x4)}$, and reads the lead report. The helpers: ‘recorded(name)‘ is true
when the report holds the check ‘name‘ with the verdict pass, ‘detail(name)‘ is its
detail text, ‘RECORD‘ the report file, and ‘check_reproduces(condition, name,
record)‘ is the helper ‘check‘ for a check that reproduces a Revision record: it
lets ‘check‘ print the PASS line and the line "reproduces ..." into a text buffer
(‘contextlib.redirect_stdout‘) and sends both lines with one ‘sys.stdout.write‘,
because Jupyter delivers printed text in pieces and one piece keeps the two lines
together for the tools that read the notebook. The first two checks repeat the
recorded reality facts.
```

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
```

```
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1


def product(directions):
    """gamma^(d1) gamma^(d2) ... in the order of the list."""
    result = I16
    for d in directions:
        result = result @ gamma[d]
    return result


C = product(["x0", "x1", "x2", "x3"])  # the charge matrix (sigma16)
Gamma = product(["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"])  # the chirality
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4)
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]
S_gen = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}

RECORD = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
lead = json.loads(repository_file(RECORD).read_text(encoding="utf-8"))
LEAD = {c["name"]: (c["verdict"].lower(), c["detail"]) for c in lead["checks"]}


def recorded(name):
    """True when the lead report records the check name with the verdict pass."""
    return LEAD[name][0] == "pass"


def detail(name):
    """The detail text of the recorded check name."""
    return LEAD[name][1]


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


passed, total = lead["summary"]["passed"], lead["summary"]["total"]
say(f"the lead report holds {len(LEAD)} checks; passed: {passed} of {total}")
is_real = all(not np.iscomplexobj(gamma[x]) for x in COORDS)
check_reproduces(is_real and np.array_equal(C.T, C) and np.array_equal(C @ C, I16)
                 and all(np.isrealobj(s) for s in S_gen.values())
                 and recorded("representation_real"),
                 "the gammas, C (symmetric, C C = 1) and the S^ab are real",
                 record=f"{RECORD}, check representation_real")
check_reproduces(not np.any(B.real) and np.array_equal(B.conj().T, B)
                 and np.array_equal(B @ B, I16) and recorded("B_imaginary_hermitian"),
                 "B = -i C gamma^(x4) is purely imaginary and Hermitian, B B = 1",
                 record=f"{RECORD}, check B_imaginary_hermitian")
```

Out [2]:

```
the lead report holds 12 checks; passed: 12 of 12
PASS the gammas, C (symmetric, C C = 1) and the S^ab are real
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    representation_real
PASS B = -i C gamma^(x4) is purely imaginary and Hermitian, B B = 1
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    B_imaginary_hermitian
```

Text cell:

```
## 6. Solving for every conjugation matrix M

The next cell turns $M(\gamma^a)^* - s\,\gamma^a M = 0$ ($a = x0, \dots, x7$) into
$8 \times 256 = 2048$ linear equations for the 256 entries of $M$, read row by row.
The entry $(r, c)$ of $M G$ is $\sum_k M_{rk}G_{kc}$ (the unknown $M_{rk}$ has the
coefficient $G_{kc}$) and of $G M$ it is $\sum_k G_{rk}M_{kc}$ (the unknown $M_{kc}$
has the coefficient $G_{rk}$); collected, the coefficient matrix of $MG - sGM$ is
$\mathrm{kron}(1, G^T) - s\,\mathrm{kron}(G, 1)$. sympy's `DomainMatrix` over the
rational numbers `QQ` computes the solutions exactly (its `nullspace`). The
conjugate `np.conj(gamma[x])` is written out although it equals `gamma[x]`: the
gammas are real.
```

In [3]:

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over QQ


def conjugation_system(s):
    """Coefficient matrix of M (gamma^a)* - s gamma^a M = 0 for the 256 unknowns."""
    blocks = []
    for x in COORDS:
        G = np.conj(gamma[x]).astype(np.int64)  # (gamma^a)*, equal to gamma^a
        blocks.append(np.kron(I16, G.T) - s * np.kron(gamma[x], I16))
    return np.vstack(blocks)


def exact_solutions(A, n=16):
    """A basis of all solutions of A y = 0, each as an n x n matrix (exact)."""
    null = DomainMatrix.from_list(A.tolist(), sp.QQ).nullspace().to_Matrix()
    return [np.array(null.row(k).tolist()[0], dtype=float).reshape(n, n)
            for k in range(null.rows)]


def proportional(X, Y):
    """True when X and Y are nonzero and X is a number times Y."""
    stacked = np.array([X.reshape(-1), Y.reshape(-1)], dtype=float)
    return bool(X.any() and Y.any() and np.linalg.matrix_rank(stacked) == 1)
```

```
A_same, A_reversed = conjugation_system(+1), conjugation_system(-1)
M_same, M_reversed = exact_solutions(A_same), exact_solutions(A_reversed)
say(f"s = +1: {A_same.shape[0]} equations, {len(M_same)} independent solution(s)")
say(f"s = -1: {A_reversed.shape[0]} equations, {len(M_reversed)} independent "
    f"solution(s)")
check_reproduces(len(M_same) == 1 and proportional(M_same[0], I16)
                 and "dimension 1" in detail("intertwiners_same_mass")
                 and recorded("intertwiners_same_mass"),
                 "M (gamma^a)* = + gamma^a M: one solution, the identity (same mass)",
                 record=f"{RECORD}, check intertwiners_same_mass")
check_reproduces(len(M_reversed) == 1 and proportional(M_reversed[0], Gamma)
                 and "dimension 1" in detail("intertwiners_reversed_mass")
                 and recorded("intertwiners_reversed_mass"),
                 "M (gamma^a)* = - gamma^a M: one solution, Gamma (mass reversed)",
                 record=f"{RECORD}, check intertwiners_reversed_mass")
```

Out [3]:

```
s = +1: 2048 equations, 1 independent solution(s)
s = -1: 2048 equations, 1 independent solution(s)
PASS M (gamma^a)* = + gamma^a M: one solution, the identity (same mass)
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    intertwiners_same_mass
PASS M (gamma^a)* = - gamma^a M: one solution, Gamma (mass reversed)
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    intertwiners_reversed_mass
```

Text cell:

```
A second, numerical engine: the number of independent solutions of $Ay = 0$ equals
the number of zero eigenvalues of the symmetric matrix $A^TA$ (if $A^TAy = 0$ then
$|Ay|^2 = y^TA^TAy = 0$). The next cell computes these eigenvalues with numpy for
both signs and draws them, together with the two solutions as heat maps. A solution
is fixed only up to a factor; each is drawn with the factor that makes its entry in
row 1, column 1 equal to that of the identity and of $\Gamma$, respectively.
```

In [4]:

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])


def heat_map(ax, matrix, title, row_label=True):
    """Draw a 16 x 16 matrix with entries from -1 to +1: blue -1, grey 0, red +1.
    Rows and columns are numbered from 1; black lines separate the two halves."""
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.set_yticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
```

```
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)
    return image


eigen_same = np.linalg.eigvalsh((A_same.T @ A_same).astype(float))
eigen_reversed = np.linalg.eigvalsh((A_reversed.T @ A_reversed).astype(float))
zeros_same = int(np.sum(np.abs(eigen_same) < 1e-9))
zeros_reversed = int(np.sum(np.abs(eigen_reversed) < 1e-9))
say(f"zero eigenvalues of A^T A: s = +1: {zeros_same}; s = -1: {zeros_reversed}")
check(zeros_same == 1 and zeros_reversed == 1,
      "numerical second engine: exactly one zero eigenvalue for each sign")

fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0), width_ratios=[1, 1, 1.3])
shown_same = M_same[0] / M_same[0][0, 0]  # scaled: entry (1,1) equal to 1
shown_reversed = M_reversed[0] / M_reversed[0][0, 0] * Gamma[0, 0]  # as in Gamma
heat_map(axes[0], shown_same, "solution for $s = +1$")
image = heat_map(axes[1], shown_reversed, "solution for $s = -1$", row_label=False)
index = np.arange(1, 257)
axes[2].plot(index, eigen_same, color="#2a78d6", linewidth=2,
             label="$s = +1$ (same mass)")
axes[2].plot(index, eigen_reversed, color="#eb6834", linewidth=2, linestyle="--",
             label="$s = -1$ (mass reversed)")
axes[2].plot([1], [eigen_same[0]], "o", color="#2a78d6", markersize=9)
axes[2].plot([1], [eigen_reversed[0]], "o", color="#eb6834", markersize=9)
axes[2].set_xlabel("number of the eigenvalue (sorted)")
axes[2].set_ylabel("eigenvalue of $A^T A$")
axes[2].set_title("one zero eigenvalue for each sign")
axes[2].legend(loc="lower right")
fig.colorbar(image, ax=axes[:2], ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "solution_spaces",
            "Left and middle: the only solutions $M$ of $M(\\gamma^a)^{\\ast} = "
            "s\\,\\gamma^a M$ for all eight gammas, as heat maps (column and row 1 "
            "to 16; blue $-1$, grey $0$, red $+1$; a solution is fixed only up to a "
            "factor, and each is drawn with the factor that makes it equal to the "
            "following): for $s = +1$ the identity matrix, for $s = -1$ the "
            "chirality $\\Gamma = \\mathrm{diag}(-1, 1)$. "
            "Right: the 256 eigenvalues of $A^T A$ for the two equation systems, "
            "sorted (horizontal axis their number, vertical axis their value); "
            "each system has exactly one zero eigenvalue (dots), so each solution "
            "space is one-dimensional.")
```

Out [4]:

```
zero eigenvalues of A^T A: s = +1: 1; s = -1: 1
PASS numerical second engine: exactly one zero eigenvalue for each sign
```

Out [4]:

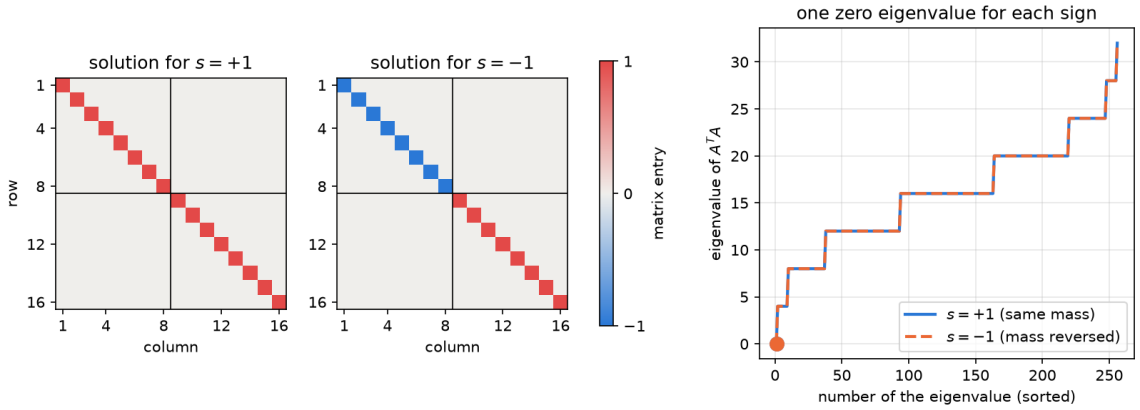

Figure 165: Left and middle: the only solutions M of $M(\gamma^a)^* = s\,\gamma^a M$ for all eight gammas, as heat maps (column and row 1 to 16; blue -1, grey 0, red $+1$; a solution is fixed only up to a factor, and each is drawn with the factor that makes it equal to the following): for $s = +1$ the identity matrix, for $s = -1$ the chirality $\Gamma = \mathrm{diag}(-1, 1)$. Right: the 256 eigenvalues of $A^T A$ for the two equation systems, sorted (horizontal axis their number, vertical axis their value); each system has exactly one zero eigenvalue (dots), so each solution space is one-dimensional. (Notebook 05c, figure 1.)

```
Figure 05c.1 saved as Revision/textbook/figures/05c_1_solution_spaces.png
```

Text cell:

```
## 7. The two charge-conjugation matrices

From $\mathcal{C} = MC$ (section 4): $M = 1$ gives $\mathcal{C}_+ = C$ and
$M = \Gamma$ gives $\mathcal{C}_- = \Gamma C$. Their transposition rules:

- $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = C\gamma^a C = -(\gamma^a)^T$ (the
  rule $C\gamma^aC^{-1} = -(\gamma^a)^T$ of the charge matrix);
- $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = C\Gamma\gamma^a\Gamma C =
  C(-\gamma^a)C = +(\gamma^a)^T$ (first $\Gamma\gamma^a\Gamma = -\gamma^a\Gamma
  \Gamma = -\gamma^a$, then the rule of $C$).

The next cell checks both rules, that both matrices are real, that
$\mathcal{C}_+$ is symmetric, and also that $\mathcal{C}_-$ is symmetric with
$\mathcal{C}_-\mathcal{C}_- = 1$. Then it solves, as an independent derivation, the
transposition equations $\gamma^a X = \zeta\, X(\gamma^a)^T$ for all $a$
directly: for $\zeta = -1$ the only solutions must be the multiples of $C$, for
$\zeta = +1$ those of $\Gamma C$.
```

In [5]:

```
calC_plus = (M_same[0] / M_same[0][0, 0]) @ C  # M = 1 (scaled), so calC_+ = C
calC_minus = Gamma @ C  # M = Gamma, so calC_- = Gamma C
inverse_plus = np.linalg.inv(calC_plus)
inverse_minus = np.linalg.inv(calC_minus)
check_reproduces(np.array_equal(calC_plus, C)
```

```
                and all(np.array_equal(inverse_plus @ gamma[x] @ calC_plus, -gamma[x].T)
                        for x in COORDS)
                and np.array_equal(calC_plus.T, calC_plus)
                and recorded("charge_conjugation_matrix_plus"),
                "calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real and "
                "symmetric",
                record=f"{RECORD}, check charge_conjugation_matrix_plus")
check_reproduces(all(np.array_equal(inverse_minus @ gamma[x] @ calC_minus, gamma[x].T)
                    for x in COORDS)
                and np.isrealobj(calC_minus)
                and np.array_equal(calC_minus.T, calC_minus)
                and np.array_equal(calC_minus @ calC_minus, I16)
                and recorded("charge_conjugation_matrix_minus"),
                "calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real",
                record=f"{RECORD}, check charge_conjugation_matrix_minus")


def transposition_system(zeta):
    """Coefficient matrix of gamma^a X - zeta X (gamma^a)^T = 0 (all a)."""
    return np.vstack([np.kron(gamma[x], I16) - zeta * np.kron(I16, gamma[x])
                      for x in COORDS])


X_minus = exact_solutions(transposition_system(-1))  # should be multiples of C
X_plus = exact_solutions(transposition_system(+1))  # should be multiples of Gamma C
say(f"zeta = -1: {len(X_minus)} solution(s); zeta = +1: {len(X_plus)} solution(s)")
check(len(X_minus) == 1 and proportional(X_minus[0], C)
      and len(X_plus) == 1 and proportional(X_plus[0], Gamma @ C),
      "the only solutions of gamma^a X = -X (gamma^a)^T are multiples of C, of "
      "gamma^a X = +X (gamma^a)^T multiples of Gamma C")
```

Out [5]:

```
PASS calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real and symmetric
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    charge_conjugation_matrix_plus
PASS calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    charge_conjugation_matrix_minus
zeta = -1: 1 solution(s); zeta = +1: 1 solution(s)
PASS the only solutions of gamma^a X = -X (gamma^a)^T are multiples of C, of gamma^a X =
    +X (gamma^a)^T multiples of Gamma C
```

Text cell:

```
The next cell draws the three matrices $\mathcal{C}_+ = C$, $\Gamma$ and
$\mathcal{C}_- = \Gamma C$. Because $\Gamma = \mathrm{diag}(-1_8, 1_8)$ and $C =
\mathrm{diag}(-\sigma, \sigma)$, the product $\Gamma C = \mathrm{diag}(\sigma,
\sigma)$: multiplying by $\Gamma$ reverses the sign of the first eight rows.
```

In [6]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], calC_plus, r"$\mathcal{C}_+ = C$")
heat_map(axes[1], Gamma, r"$\Gamma$", row_label=False)
image = heat_map(axes[2], calC_minus, r"$\mathcal{C}_- = \Gamma C$",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "conjugation_matrices",
            "Heat maps of the two charge-conjugation matrices and of the chirality "
            "that relates them: $\\mathcal{C}_+ = C$ (left, the same-mass "
            "conjugation), $\\Gamma$ (middle) and $\\mathcal{C}_- = \\Gamma C$ "
            "(right, the mass-reversing conjugation); horizontal axis the column, "
            "vertical axis the row (1 to 16); blue $-1$, grey $0$, red $+1$. "
            "$\\Gamma C$ is $C$ with the sign of rows 1 to 8 reversed, so both of "
            "its diagonal blocks equal $+\\sigma$.")
```

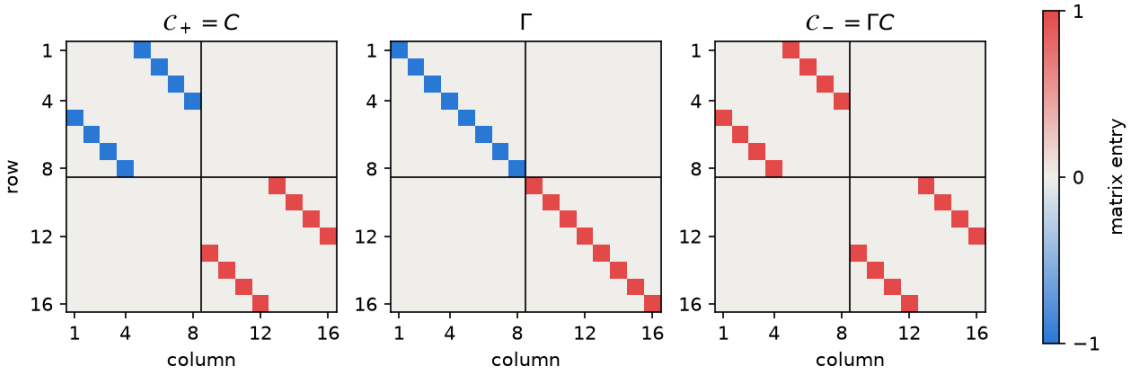

Figure 166: Heat maps of the two charge-conjugation matrices and of the chirality that relates them: $\mathcal{C}_+ = C$ (left, the same-mass conjugation), Γ (middle) and $\mathcal{C}_- = \Gamma C$ (right, the mass-reversing conjugation); horizontal axis the column, vertical axis the row (1 to 16); blue -1, grey 0, red $+1$. ΓC is C with the sign of rows 1 to 8 reversed, so both of its diagonal blocks equal $+\sigma$. (Notebook 05c, figure 2.)

Out [6]:

```
Figure 05c.2 saved as Revision/textbook/figures/05c_2_conjugation_matrices.png
```

Text cell:

```
## 8. The conjugate fields

The next cell checks the formulas of section 4 on a complex column. We take the
fixed column $\Psi_0$ with the components $(k \bmod 4) - 1 + i\,((k \bmod 3) - 1)$
for $k = 0, \dots, 15$ ($k \bmod n$ is the remainder of $k$ divided by $n$; Python
writes '%'). It computes $\bar\Psi^T = C\Psi^*$ and checks
$\mathcal{C}_+\bar\Psi^T = \Psi^*$ and $\mathcal{C}_-\bar\Psi^T = \Gamma\Psi^*$.
```

In [7]:

```
k = np.arange(16)  # 0, 1, ..., 15
PSI0 = (k % 4 - 1) + 1j * (k % 3 - 1)  # a fixed complex column with small entries
say("Psi_0 = " + ", ".join(f"{z.real:+.0f}{z.imag:+.0f}i" for z in PSI0[:8]) + ",")
say("        " + ", ".join(f"{z.real:+.0f}{z.imag:+.0f}i" for z in PSI0[8:]))
psibar_T = C @ PSI0.conj()  # (Psi^dagger C)^T = C^T Psi* = C Psi*
check(np.array_equal(calC_plus @ psibar_T, PSI0.conj())
      and np.array_equal(calC_minus @ psibar_T, Gamma @ PSI0.conj()),
      "Psi^c = calC_+ Psibar^T = Psi* and Psi^c = calC_- Psibar^T = Gamma Psi*")
```

Out [7]:

```
Psi_0 = -1-1i, +0+0i, +1+1i, +2-1i, -1+0i, +0+1i, +1-1i, +2+0i,
        -1+1i, +0-1i, +1+0i, +2+1i, -1-1i, +0+0i, +1+1i, +2-1i
PASS Psi^c = calC_+ Psibar^T = Psi* and Psi^c = calC_- Psibar^T = Gamma Psi*
```

Text cell:

```
## 9. The conjugate fields solve the equation with the mass predicted

The next cell builds, with sympy and **exactly**, the solution $\Psi(x4) =
\cos(m\,x4)\Psi_0 - \sin(m\,x4)\gamma^{(x4)}\Psi_0$ for a general real mass $m$, and
its two images $\Psi^c_+ = \Psi^*$ and $\Psi^c_- = \Gamma\Psi^*$. For each of the
three fields and each mass $+m$ and $-m$ it computes the residual
$\gamma^{(x4)} d\Phi/dx4 - (\pm m)\Phi$ (which is zero exactly when $\Phi$ solves the
equation with that mass) and expands it. Prediction: $\Psi$ and $\Psi^*$ solve
with $+m$, $\Gamma\Psi^*$ with $-m$, and no field solves with both (for
$m \neq 0$).
```

In [8]:

```
x4, m = sp.symbols("x4 m", real=True)  # the time and the mass, real numbers
psi0 = sp.Matrix([sp.Integer(int(z.real)) + sp.I * sp.Integer(int(z.imag))
                  for z in PSI0])
G4 = sp.Matrix(gamma["x4"].tolist())  # gamma^(x4) as an exact sympy matrix
GAMMA = sp.Matrix(Gamma.tolist())
psi = sp.cos(m * x4) * psi0 - sp.sin(m * x4) * G4 * psi0  # the solution
fields = {"Psi": psi, "Psi* (calC_+)": psi.conjugate(),
          "Gamma Psi* (calC_-)": GAMMA * psi.conjugate()}
solves = {}  # (field, mass sign) -> True when the residual is exactly zero
for name, phi in fields.items():
    for sign in (+1, -1):
        residual = (G4 * phi.diff(x4) - sign * m * phi).applyfunc(sp.expand)
        solves[(name, sign)] = residual == sp.zeros(16, 1)
    say(f"{name:20} solves with +m: {solves[(name, 1)]!s:5}  "
        f"with -m: {solves[(name, -1)]!s:5}")
expected = {("Psi", 1): True, ("Psi", -1): False,
            ("Psi* (calC_+)", 1): True, ("Psi* (calC_+)", -1): False,
            ("Gamma Psi* (calC_-)", 1): False, ("Gamma Psi* (calC_-)", -1): True}
check(solves == expected,
      "exactly: Psi and Psi* solve with +m, Gamma Psi* solves with -m")
```

Out [8]:

```
Psi                  solves with +m: True   with -m: False
Psi* (calC_+)        solves with +m: True   with -m: False
Gamma Psi* (calC_-)  solves with +m: False  with -m: True
PASS exactly: Psi and Psi* solve with +m, Gamma Psi* solves with -m
```

Text cell:

```
The next cell evaluates the three fields numerically for $m = 1$ (so $x4$ is
measured in units of $1/m$) on 801 points from $x4 = 0$ to $4\pi$, and the largest
size of each residual on these points. ‘np.einsum("rc,tc->tr", G, F)‘ multiplies the
matrix $G$ with the column of $F$ at every time point $t$ at once. It then draws the
first component of each field (real and imaginary part) and a table of the largest
residuals.
```

In [9]:

```
t = np.linspace(0.0, 4.0 * np.pi, 801)  # the time x4 in units of 1/m, m = 1
g4 = gamma["x4"].astype(float)
cos_t, sin_t = np.cos(t)[:, None], np.sin(t)[:, None]  # columns for broadcasting
psi_t = cos_t * PSI0 - sin_t * np.einsum("rc,c->r", g4, PSI0)  # row t: Psi(x4_t)
dpsi_t = -sin_t * PSI0 - cos_t * np.einsum("rc,c->r", g4, PSI0)  # dPsi/dx4
numeric = {"Psi": (psi_t, dpsi_t),
           "Psi* (calC_+)": (psi_t.conj(), dpsi_t.conj()),
           "Gamma Psi* (calC_-)": (psi_t.conj() @ Gamma.T, dpsi_t.conj() @ Gamma.T)}
largest = {}  # (field, sign) -> largest size of the residual over the 801 points
for name, (phi, dphi) in numeric.items():
    for sign in (+1, -1):
        residual = np.einsum("rc,tc->tr", g4, dphi) - sign * 1.0 * phi
        largest[(name, sign)] = float(np.max(np.linalg.norm(residual, axis=1)))
    shown = {sign: (f"{largest[(name, sign)]:.2f}" if largest[(name, sign)] > 1e-12
                    else "below 1e-12") for sign in (1, -1)}  # rounding-proof text
    say(f"{name:20} largest residual with +m: {shown[1]:11}, with -m: {shown[-1]}")
check(all((largest[key] < 1e-12) == solves[key] for key in solves),
      "the numerical residuals agree with the exact result")
```

Out [9]:

```
Psi                  largest residual with +m: below 1e-12, with -m: 11.83
Psi* (calC_+)        largest residual with +m: below 1e-12, with -m: 11.83
Gamma Psi* (calC_-)  largest residual with +m: 11.83      , with -m: below 1e-12
PASS the numerical residuals agree with the exact result
```

Text cell:

```
The next cell draws the first component of the three fields along $x4$: $\Psi^*$
has the same real part as $\Psi$ and the opposite imaginary part; $\Gamma\Psi^*$
has, in the first half (components 1 to 8, where $\Gamma = -1$), the opposite sign
of $\Psi^*$.
```

In [10]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
styles = {"Psi": ("#2a78d6", "-"), "Psi* (calC_+)": ("#eb6834", "--"),
          "Gamma Psi* (calC_-)": ("#1baf7a", ":")}
labels = {"Psi": r"$\Psi$", "Psi* (calC_+)": r"$\Psi^c_+ = \Psi^*$",
          "Gamma Psi* (calC_-)": r"$\Psi^c_- = \Gamma\Psi^*$"}
for name, (phi, _) in numeric.items():
    color, line = styles[name]
    axes[0].plot(t, phi[:, 0].real, color=color, linestyle=line, linewidth=2,
                 label=labels[name])
    axes[1].plot(t, phi[:, 0].imag, color=color, linestyle=line, linewidth=2,
                 label=labels[name])
for ax, part in zip(axes, ["real part", "imaginary part"]):
    ax.set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
                  ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
    ax.set_xlabel("time $x4$ (in units of $1/m$)")
    ax.set_title(f"component 1, {part}")
axes[0].set_ylabel("value of the component")
axes[1].legend(loc="upper center", bbox_to_anchor=(-0.05, -0.17), ncol=3)
save_figure(fig, "conjugate_solutions",
            "The first of the 16 components of the solution $\\Psi(x4) = "
            "\\cos(m x4)\\Psi_0 - \\sin(m x4)\\gamma^{(x4)}\\Psi_0$ of the free "
            "equation (solid) and of its two charge conjugates $\\Psi^{\\ast} = "
            "\\mathcal{C}_+\\bar\\Psi^T$ (dashed) and $\\Gamma\\Psi^{\\ast} = "
            "\\mathcal{C}_-\\bar\\Psi^T$ (dotted), real part left and imaginary "
            "part right; horizontal axis the time $x4$ in units of $1/m$, "
            "vertical axis the value (a pure number). $\\Psi^{\\ast}$ flips the sign "
            "of the imaginary part; $\\Gamma\\Psi^{\\ast}$ in addition flips the sign "
            "of every component of the first half, here component 1, so its real "
            "part is the mirror image of that of $\\Psi$ and its imaginary part lies "
            "exactly on that of $\\Psi$ (right, dotted on solid).")
```

Out [10]:

```
Figure 05c.3 saved as Revision/textbook/figures/05c_3_conjugate_solutions.png
```

Text cell:

```
The next cell draws the table of the largest residuals: a residual of zero (up to
the rounding of floating-point numbers, below $10^{-12}$) means that the field
solves the equation with that mass.
```

In [11]:

```
from matplotlib.patches import Patch  # a coloured square for a legend

names = list(numeric)
table = np.array([[largest[(name, 1)], largest[(name, -1)]] for name in names])
fig, ax = plt.subplots(figsize=(7.0, 3.6))
```

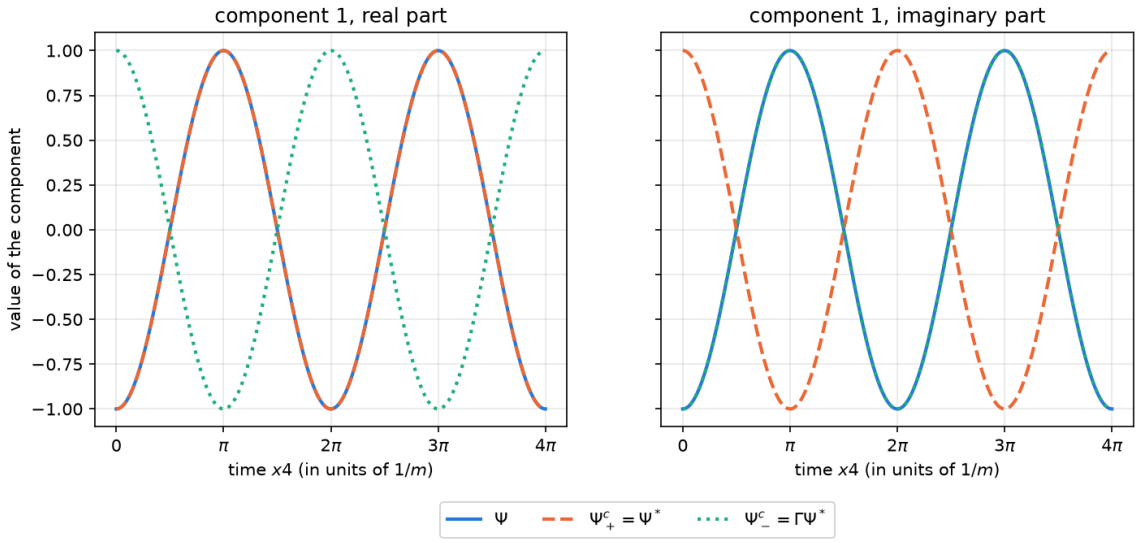

Figure 167: The first of the 16 components of the solution $\Psi(x4) = \cos(mx4)\Psi_0 - \sin(mx4)\gamma^{(x4)}\Psi_0$ of the free equation (solid) and of its two charge conjugates $\Psi^* = \mathcal{C}_+\bar{\Psi}^T$ (dashed) and $\Gamma\Psi^* = \mathcal{C}_-\bar{\Psi}^T$ (dotted), real part left and imaginary part right; horizontal axis the time $x4$ in units of $1/m$, vertical axis the value (a pure number). Ψ^* flips the sign of the imaginary part; $\Gamma\Psi^*$ in addition flips the sign of every component of the first half, here component 1, so its real part is the mirror image of that of Ψ and its imaginary part lies exactly on that of Ψ (right, dotted on solid). (Notebook 05c, figure 3.)

```
solved = table < 1e-12  # True where the field solves the equation
ax.imshow(np.where(solved, 1.0, 0.0), cmap=LinearSegmentedColormap.from_list(
    "solve", ["#f0efec", "#2a78d6"]), vmin=0, vmax=1, aspect="auto")
for r in range(3):
    for c in range(2):
        text = "solves" if solved[r, c] else f"no ({table[r, c]:.2f})"
        ax.text(c, r, text, ha="center", va="center",
                color="white" if solved[r, c] else "black", fontweight="bold")
ax.axhline(0.5, color="white", linewidth=2)
ax.axhline(1.5, color="white", linewidth=2)
ax.axvline(0.5, color="white", linewidth=2)
ax.set_xticks([0, 1], ["mass $+m$", "mass $-m$"])
ax.set_yticks(range(3), [labels[name] for name in names])
ax.set_title("Largest residual of the field equation on $0 \\leq x4 \\leq 4\\pi$")
ax.legend(handles=[Patch(color="#2a78d6", label="residual below $10^{-12}$"),
                   Patch(color="#f0efec", label="residual of order 1")],
          loc="upper center", bbox_to_anchor=(0.5, -0.14), ncol=2)
ax.grid(False)
save_figure(fig, "which_mass",
            "Which mass each field solves the free equation with: rows the solution "
            "$\\Psi$ and its two charge conjugates $\\mathcal{C}_+\\bar\\Psi^T = "
            "\\Psi^{\\ast}$ and $\\mathcal{C}_-\\bar\\Psi^T = \\Gamma\\Psi^{\\ast}$, "
            "columns the masses $+m$ and $-m$; each square "
            "gives the largest size of the residual $\\gamma^{(x4)} d\\Phi/dx4 \\mp "
            "m\\Phi$ on 801 points with $m = 1$ (blue: zero up to rounding, the field "
            "solves; grey: the residual, the field does not solve). $\\mathcal{C}_+$ "
```

```
"keeps the mass, $\\mathcal{C}_-$ reverses it, as the exact computation "
"proves.")
```

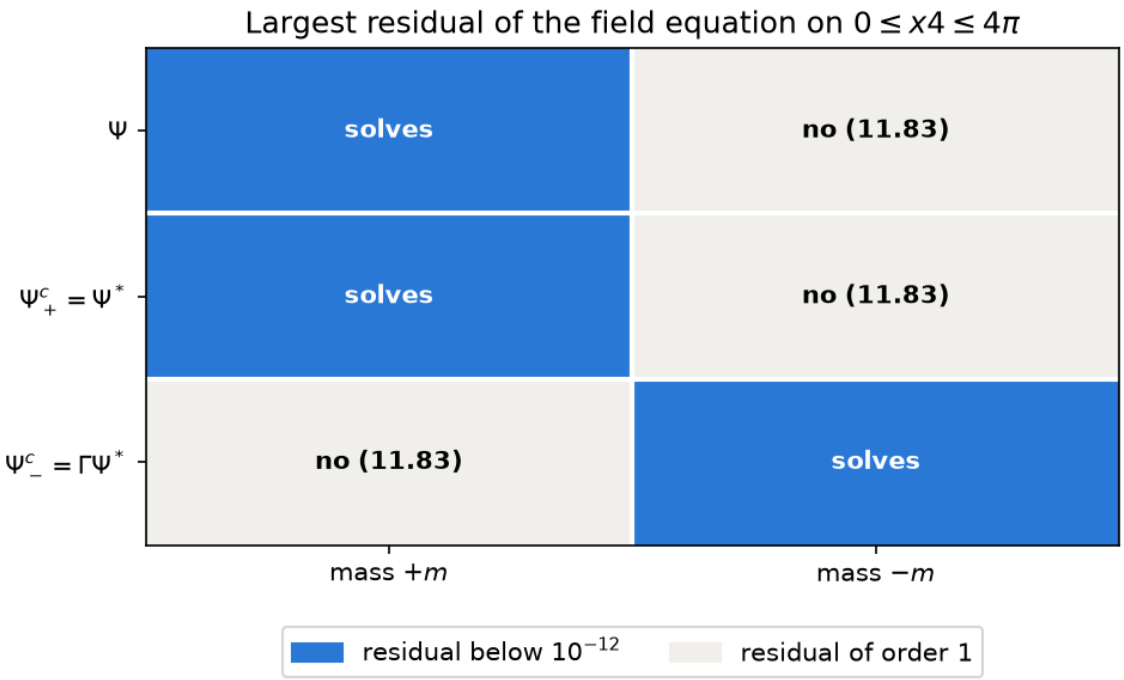

Figure 168: Which mass each field solves the free equation with: rows the solution Ψ and its two charge conjugates $\mathcal{C}_+\bar{\Psi}^T = \Psi^*$ and $\mathcal{C}_-\bar{\Psi}^T = \Gamma\Psi^*$, columns the masses $+m$ and $-m$; each square gives the largest size of the residual $\gamma^{(x4)}d\Phi/dx4 \mp m\Phi$ on 801 points with $m = 1$ (blue: zero up to rounding, the field solves; grey: the residual, the field does not solve). $\mathcal{C}_+$ keeps the mass, $\mathcal{C}_-$ reverses it, as the exact computation proves. (Notebook 05c, figure 4.)

Out [11]:

```
Figure 05c.4 saved as Revision/textbook/figures/05c_4_which_mass.png
```

Text cell:

```
## 10. The scalar S and the charge density J for commuting components

For commuting components (the field dirac16complex00) the next cell computes,
along the solution, the scalar $S = \Psi^\dagger C\Psi$ and the charge density
$J^{(x4)} = \Psi^\dagger B\Psi$ of the three fields at all 801 time points. Both
are constant along $x4$ (the derivative of $S$ is $-m\Psi^\dagger(
(\gamma^{(x4)})^TC + C\gamma^{(x4)})\Psi = 0$, because $(\gamma^{(x4)})^T =
-\gamma^{(x4)}$ commutes with $C$; the same computation with $B$ in place of $C$
gives 0 for $J^{(x4)}$, because $\gamma^{(x4)}$ also commutes with $B$). The
prediction of the Revision record for commuting components: $\Psi^*$ keeps $S$ and
reverses $J$; $\Gamma\Psi^*$ keeps $S$ and keeps $J$. The cell also checks the
exact values $S = -2$ and $J^{(x4)} = -6$ of the solution at $x4 = 0$ (where
$\Psi = \Psi_0$), which the next figure shows.
```

In [12]:

```
bilinears = {}  # field -> (S at every time point, J^(x4) at every time point)
for name, (phi, _) in numeric.items():
    S_t = np.einsum("tr,rc,tc->t", phi.conj(), C, phi).real  # Psi^dagger C Psi
    J_t = np.einsum("tr,rc,tc->t", phi.conj(), B, phi).real  # Psi^dagger B Psi
    bilinears[name] = (S_t, J_t)
    say(f"{name:20} S from {S_t.min():+.6f} to {S_t.max():+.6f}; "
        f"J^(x4) from {J_t.min():+.6f} to {J_t.max():+.6f}")
S_psi, J_psi = bilinears["Psi"][0][0], bilinears["Psi"][1][0]
report("S of the solution Psi", f"{S_psi:+.6f}")
report("J^(x4) of the solution Psi", f"{J_psi:+.6f}")
constant = all(np.ptp(v) < 1e-12 for pair in bilinears.values() for v in pair)
# The values at x4 = 0, where Psi = Psi_0, are quoted in the caption of the next
# figure. Psi_0 has whole-number real and imaginary parts and C, B have entries 0,
# +-1, +-i, so S = Psi_0^dagger C Psi_0 and J^(x4) = Psi_0^dagger B Psi_0 are
# computed exactly (small whole numbers are stored without rounding).
S_exact = PSI0.conj() @ C @ PSI0
J_exact = PSI0.conj() @ B @ PSI0
check(S_exact == -2 and J_exact == -6 and S_psi == -2 and J_psi == -6,
      "at x4 = 0 exactly: S = -2 and J^(x4) = -6 for the solution Psi")
check(constant and abs(S_psi) > 1 and abs(J_psi) > 1
      and np.allclose(bilinears["Psi* (calC_+)"][0], S_psi)
      and np.allclose(bilinears["Psi* (calC_+)"][1], -J_psi)
      and np.allclose(bilinears["Gamma Psi* (calC_-)"][0], S_psi)
      and np.allclose(bilinears["Gamma Psi* (calC_-)"][1], J_psi),
      "commuting components: Psi* keeps S and reverses J; Gamma Psi* keeps S and J")
```

Out [12]:

```
Psi                  S from -2.000000 to -2.000000; J^(x4) from -6.000000 to -6.000000
Psi* (calC_+)        S from -2.000000 to -2.000000; J^(x4) from +6.000000 to +6.000000
Gamma Psi* (calC_-)  S from -2.000000 to -2.000000; J^(x4) from -6.000000 to -6.000000
RESULT S of the solution Psi = -2.000000
RESULT J^(x4) of the solution Psi = -6.000000
PASS at x4 = 0 exactly: S = -2 and J^(x4) = -6 for the solution Psi
PASS commuting components: Psi* keeps S and reverses J; Gamma Psi* keeps S and J
```

Text cell:

```
The next cell draws these constant values as bars: one group for $S$, one for
$J^{(x4)}$.
```

In [13]:

```
fig, ax = plt.subplots(figsize=(8.0, 4.2))
width = 0.26  # the width of one bar
for j, name in enumerate(names):
    color, _ = styles[name]
    values = [bilinears[name][0][0], bilinears[name][1][0]]
```

```
    bars = ax.bar(np.arange(2) + (j - 1) * width, values, width, color=color,
                  edgecolor="white", linewidth=2, label=labels[name])
    for bar, v in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width() / 2, v + (0.3 if v > 0 else -0.7),
                f"{v:+.0f}", ha="center")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks([0, 1], [r"scalar $S = \Psi^\dagger C\Psi$",
                       r"charge density $J^{(x4)} = \Psi^\dagger B\Psi$"])
ax.set_ylabel("value (constant along $x4$)")
ax.set_ylim(-8, 8)
ax.set_title("Commuting components: what the two conjugations do to S and J")
ax.legend(loc="upper left")
save_figure(fig, "bilinears",
            "The scalar $S$ (left group) and the charge density $J^{(x4)}$ (right "
            "group) of the solution $\\Psi$ (blue) and of its charge conjugates "
            "$\\mathcal{C}_+\\bar\\Psi^T = \\Psi^{\\ast}$ (orange) and "
            "$\\mathcal{C}_-\\bar\\Psi^T = \\Gamma\\Psi^{\\ast}$ (aqua), for commuting "
            "components; vertical axis the value, which is the same at every time "
            "$x4$. All three have the same $S = -2$; $\\Psi^{\\ast}$ has the opposite "
            "charge density $+6$, $\\Gamma\\Psi^{\\ast}$ the same $-6$: for commuting "
            "components $\\mathcal{C}_+$ reverses the charge and keeps the mass, "
            "$\\mathcal{C}_-$ keeps the charge and reverses the mass.")
```

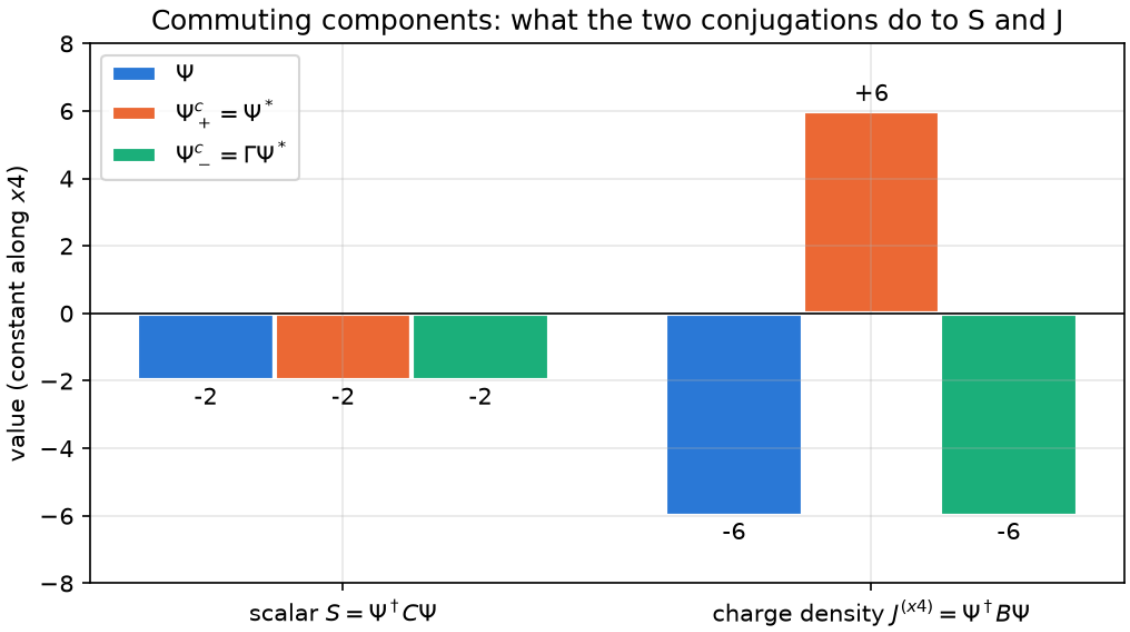

Figure 169: The scalar S (left group) and the charge density $J^{(x4)}$ (right group) of the solution Ψ (blue) and of its charge conjugates $\mathcal{C}_+\bar\Psi^T = \Psi^*$ (orange) and $\mathcal{C}_-\bar\Psi^T = \Gamma\Psi^*$ (aqua), for commuting components; vertical axis the value, which is the same at every time $x4$. All three have the same $S = -2$; Ψ^* has the opposite charge density $+6$, $\Gamma\Psi^*$ the same -6: for commuting components $\mathcal{C}_+$ reverses the charge and keeps the mass, $\mathcal{C}_-$ keeps the charge and reverses the mass. (Notebook 05c, figure 5.)

Out [13]:

```
Figure 05c.5 saved as Revision/textbook/figures/05c_5_bilinears.png
```

Text cell:

```
## 11. The sign table for commuting and anticommuting components

For a bilinear $\Psi^\dagger K\Psi = \sum_{r,c}\Psi_r^* K_{rc}\Psi_c$ and the map
$\Psi \to M\Psi^*$ with a real $M$, line by line:

1. $(M\Psi^*)^\dagger = \Psi^T M^\dagger$ (the conjugate of $\Psi^*$ is $\Psi$), so
   the bilinear becomes $\Psi^T M^\dagger K M\Psi^* = \sum_{r,c}\Psi_r\,
   (M^\dagger KM)_{rc}\,\Psi_c^*$.
2. Exchange the two factors $\Psi_r$ and $\Psi_c^*$; this costs the sign
   $\epsilon$: $\sum_{r,c}\epsilon\,\Psi_c^*(M^\dagger KM)_{rc}\Psi_r$.
3. Rename $r \leftrightarrow c$: this is $\Psi^\dagger K'\Psi$ with
   $K' = \epsilon\,(M^\dagger KM)^T$.

So the bilinear with matrix $K$ turns into the bilinear with matrix $K'$. The next
cell computes $K'$ for $K = C$ (the scalar $S$) and $K = -iC\gamma^a$ (the eight
currents $J^a$), for $M = 1$ ($\mathcal{C}_+$) and $M = \Gamma$ ($\mathcal{C}_-$)
and $\epsilon = \pm1$, and records the sign with $K' = \pm K$. It compares the
table with the signs that the Revision record measured (stored in its detail text
after the word measured).

For the *quantised* field dirac16complex the components are operators. Exchanging
two of them costs the same sign as for anticommuting numbers and leaves a constant
behind (their anticommutator); normal ordering removes that constant and nothing
else. So after normal ordering the quantised bilinears change with the signs of the
anticommuting rows of this table. Notebook 05e computes this on an explicit Fock
space. The detail text of the record's check says the same: normal ordering adds
no sign, because it subtracts the vacuum value, which is a number.
```

In [14]:

```
K_S = C.astype(complex)  # the matrix of the scalar S
K_J = [-1j * (C @ gamma[x]) for x in COORDS]  # the matrices of the currents J^a


def sign_of(M, K, eps):
    """+1 if eps (M^dagger K M)^T = K, -1 if it equals -K, 0 otherwise."""
    new = eps * (M.conj().T @ K @ M).T
    return 1 if np.array_equal(new, K) else (-1 if np.array_equal(new, -K) else 0)


measured = {}  # the record's keys, e.g. "plus,eps=1" -> [sign of S, [signs of J]]
for map_name, M in (("plus", I16), ("minus", Gamma)):
    for eps in (1, -1):
        measured[f"{map_name},eps={eps}"] = [
            sign_of(M, K_S, eps), [sign_of(M, K, eps) for K in K_J]]
for key, (sign_S, signs_J) in measured.items():
    say(f"{key:13} S -> {sign_S:+d} S;  J^a -> s J^a with s = "
        + " ".join(f"{s:+d}" for s in signs_J))
```

```
recorded_table = json.loads(detail("bilinears_under_charge_conjugation")
                            .split("measured: ", 1)[1])
check_reproduces(measured == recorded_table
                 and recorded("bilinears_under_charge_conjugation"),
                 "the signs of S and J under calC_+ and calC_-, commuting and "
                 "anticommuting, equal the recorded table",
                 record=f"{RECORD}, check bilinears_under_charge_conjugation")
```

Out [14]:

```
plus,eps=1    S -> +1 S;  J^a -> s J^a with s = -1 -1 -1 -1 -1 -1 -1 -1
plus,eps=-1   S -> -1 S;  J^a -> s J^a with s = +1 +1 +1 +1 +1 +1 +1 +1
minus,eps=1   S -> +1 S;  J^a -> s J^a with s = +1 +1 +1 +1 +1 +1 +1 +1
minus,eps=-1  S -> -1 S;  J^a -> s J^a with s = -1 -1 -1 -1 -1 -1 -1 -1
PASS the signs of S and J under calC_+ and calC_-, commuting and anticommuting, equal the
    recorded table
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    bilinears_under_charge_conjugation
```

Text cell:

```
The next cell draws the sign table: four rows (two maps, two kinds of components),
nine columns ($S$ and the eight currents).
```

In [15]:

```
row_keys = ["plus,eps=1", "plus,eps=-1", "minus,eps=1", "minus,eps=-1"]
row_names = [r"$\mathcal{C}_+$, commuting", r"$\mathcal{C}_+$, anticommuting",
             r"$\mathcal{C}_-$, commuting", r"$\mathcal{C}_-$, anticommuting"]
grid = np.array([[measured[key][0]] + measured[key][1] for key in row_keys])
fig, ax = plt.subplots(figsize=(10.0, 3.8))
ax.imshow(grid, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
for r in range(4):
    for c in range(9):
        ax.text(c, r, f"{grid[r, c]:+d}", ha="center", va="center", color="white",
                fontweight="bold")
for k in range(1, 4):
    ax.axhline(k - 0.5, color="white", linewidth=2)
for k in range(1, 9):
    ax.axvline(k - 0.5, color="white", linewidth=3 if k == 1 else 2)
ax.set_xticks(range(9), ["$S$"] + [rf"$J^{{({x})}}$" for x in COORDS])
ax.set_yticks(range(4), row_names)
ax.set_title("Sign of each bilinear after the conjugation (red $+1$ kept, blue "
             "$-1$ reversed)")
ax.grid(False)
save_figure(fig, "sign_table",
            "The sign that each bilinear acquires under the two charge "
            "conjugations, for commuting and for anticommuting (classical "
            "Grassmann) components: columns the scalar $S$ and the eight currents "
            "$J^a$, rows the map and the kind of component; red $+1$ the bilinear "
            "is kept, blue $-1$ it is reversed. $\\mathcal{C}_+$ on commuting "
```

```
"components keeps $S$ and reverses every current; $\\mathcal{C}_-$ on "
"commuting components keeps both; for anticommuting components every "
"sign is flipped once more by the exchange of two factors.")
```

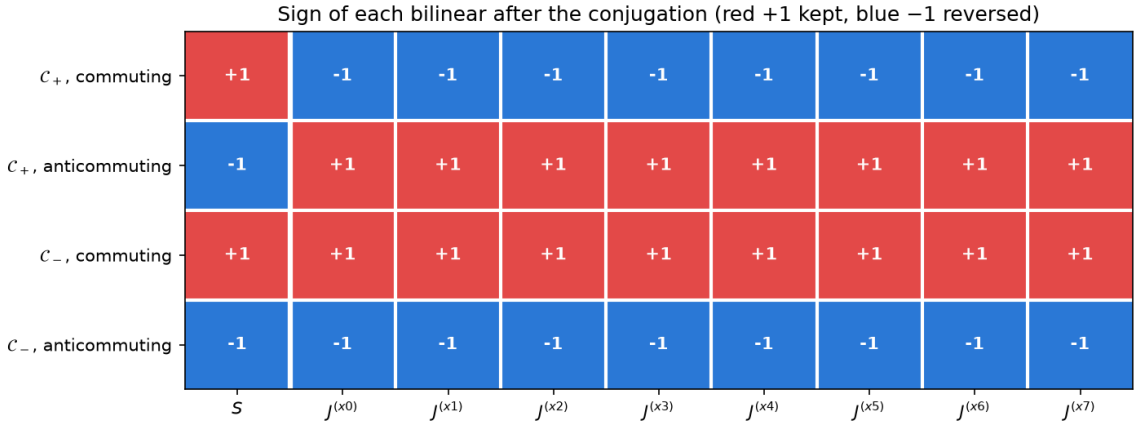

Figure 170: The sign that each bilinear acquires under the two charge conjugations, for commuting and for anticommuting (classical Grassmann) components: columns the scalar S and the eight currents J^a, rows the map and the kind of component; red $+1$ the bilinear is kept, blue -1 it is reversed. $\mathcal{C}_+$ on commuting components keeps S and reverses every current; $\mathcal{C}_-$ on commuting components keeps both; for anticommuting components every sign is flipped once more by the exchange of two factors. (Notebook 05c, figure 6.)

Out [15]:

```
Figure 05c.6 saved as Revision/textbook/figures/05c_6_sign_table.png
```

Text cell:

```
## 12. The reality (Majorana) conditions

A field equal to its own conjugate obeys $\Psi = M\Psi^*$. Taking the complex
conjugate of this condition gives $\Psi^* = M^*\Psi$, and putting it back gives
$\Psi = MM^*\Psi$. So the condition agrees with itself for every column that obeys
it (it is *consistent*) when $MM^* = 1$; otherwise it would force further
conditions, possibly $\Psi = 0$. The next cell checks $MM^* = 1$ for $M = 1$ (the
condition $\Psi = \Psi^*$: a real field) and for $M = \Gamma$ (the condition
$\Psi = \Gamma\Psi^*$: the first half imaginary, the second half real).
```

In [16]:

```
check_reproduces(np.array_equal(I16 @ np.conj(I16), I16)
                 and np.array_equal(Gamma @ np.conj(Gamma), I16)
                 and recorded("majorana_conditions_consistent"),
                 "M M* = 1 for M = 1 and M = Gamma: both reality conditions are "
                 "consistent",
                 record=f"{RECORD}, check majorana_conditions_consistent")
```

Out [16]:

```
PASS M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    majorana_conditions_consistent
```

Text cell:

```
Consistency is a statement about one moment. Is the condition kept as the field
evolves? For the solution $\Psi(x4)$ of section 9 with a starting column that obeys
the condition:

- start real ($\Psi_0^* = \Psi_0$): all factors of the solution are real, so
  $\Psi(x4)$ stays real at all times;
- start with $\Psi_0 = \Gamma\Psi_0^*$: then $\Gamma\Psi(x4)^* = \cos(m x4)\Gamma
  \Psi_0^* - \sin(m x4)\Gamma\gamma^{(x4)}\Psi_0^* = \cos(m x4)\Psi_0 + \sin(m x4)
  \gamma^{(x4)}\Psi_0$ (because $\Gamma\gamma^{(x4)} = -\gamma^{(x4)}\Gamma$), so
  $\Psi(x4) - \Gamma\Psi(x4)^* = -2\sin(m x4)\gamma^{(x4)}\Psi_0$, whose size is
  $2|\sin(m x4)|\,|\Psi_0|$ ($\gamma^{(x4)}$ is a signed permutation and keeps
  sizes).

So the second condition holds at all times only for $m = 0$. This is what
"mass reversed" means: a field equal to its $\mathcal{C}_-$ image would have to
solve the equation with $V$ and with $-V$ at once. The next cell computes both
violations for $m = 1$, $m = 0.5$ and $m = 0$ and draws them.
```

In [17]:

```
start_real = PSI0.real.astype(complex)  # obeys Psi_0* = Psi_0
start_gamma = np.concatenate([1j * PSI0.imag[:8], PSI0.real[8:]])  # first half
start_gamma = start_gamma / np.linalg.norm(start_gamma)  # imaginary, size 1
start_real = start_real / np.linalg.norm(start_real)  # size 1
check(np.allclose(start_gamma, Gamma @ start_gamma.conj())
      and np.allclose(start_real, start_real.conj()),
      "the two starting columns obey Psi_0 = Gamma Psi_0* and Psi_0 = Psi_0*")


def evolve(start, mass):
    """The solution cos(m x4) start - sin(m x4) gamma^(x4) start at all times t."""
    return (np.cos(mass * t)[:, None] * start
            - np.sin(mass * t)[:, None] * np.einsum("rc,c->r", g4, start))


violation = {}  # (condition, mass) -> size of the violation at every time
for mass in (1.0, 0.5, 0.0):
    f_real = evolve(start_real, mass)
    f_gamma = evolve(start_gamma, mass)
    violation[("real", mass)] = np.linalg.norm(f_real - f_real.conj(), axis=1)
    violation[("gamma", mass)] = np.linalg.norm(f_gamma - f_gamma.conj() @ Gamma.T,
                                                axis=1)
check(all(np.max(violation[("real", mass)]) < 1e-12 for mass in (1.0, 0.5, 0.0))
```

```
      and np.max(violation[("gamma", 0.0)]) < 1e-12
      and np.allclose(violation[("gamma", 1.0)], 2 * np.abs(np.sin(t)))
      and np.allclose(violation[("gamma", 0.5)], 2 * np.abs(np.sin(0.5 * t))),
      "Psi = Psi* is kept in time; Psi = Gamma Psi* is violated by 2 |sin(m x4)|")
fig, ax = plt.subplots(figsize=(8.0, 4.2))
ax.plot(t, violation[("gamma", 1.0)], color="#eb6834", linewidth=2,
        label=r"$|\Psi - \Gamma\Psi^*|$, $m = 1$")
ax.plot(t, violation[("gamma", 0.5)], color="#eda100", linewidth=2, linestyle="--",
        label=r"$|\Psi - \Gamma\Psi^*|$, $m = 0.5$")
ax.plot(t, violation[("gamma", 0.0)], color="#4a3aa7", linewidth=3, linestyle="-.",
        label=r"$|\Psi - \Gamma\Psi^*|$, $m = 0$")
ax.plot(t, violation[("real", 1.0)], color="#2a78d6", linewidth=2, linestyle=":",
        label=r"$|\Psi - \Psi^*|$, $m = 1$ (real start)")
ax.set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
              ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
ax.set_xlabel("time $x4$")
ax.set_ylabel("size of the violation")
ax.set_ylim(-0.1, 2.4)
ax.annotate("the dash-dotted and the dotted curve are exactly 0 at all times",
            (2.5 * np.pi, 0.0), xytext=(2.0 * np.pi, 0.35), ha="center",
            arrowprops={"arrowstyle": "->", "color": "black"},
            bbox={"facecolor": "white", "edgecolor": "#52514e"})
ax.set_title("Which reality condition survives the time evolution")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=2)
save_figure(fig, "reality_in_time",
            "How far the free solution moves away from the two reality conditions, for "
            "starting columns of size 1 that obey them: the size of $\\Psi - "
            "\\Gamma\\Psi^{\\ast}$ for the masses $m = 1$ (solid), $m = 0.5$ (dashed) "
            "and $m = 0$ (dash-dotted, zero), and the size of $\\Psi - \\Psi^{\\ast}$ "
            "for a real start (dotted, zero); horizontal axis the time $x4$, vertical "
            "axis the size (a pure number). The real field stays real; the condition "
            "$\\Psi = \\Gamma\\Psi^{\\ast}$ of the mass-reversing conjugation is "
            "violated by $2|\\sin(m x4)|$ unless $m = 0$.")
```

Out [17]:

```
PASS the two starting columns obey Psi_0 = Gamma Psi_0* and Psi_0 = Psi_0*
PASS Psi = Psi* is kept in time; Psi = Gamma Psi* is violated by 2 |sin(m x4)|
```

Out [17]:

```
Figure 05c.7 saved as Revision/textbook/figures/05c_7_reality_in_time.png
```

Text cell:

```
## 13. Real fields

For a **real** commuting field ($\Psi^* = \Psi$):

- the currents vanish identically: $J^a = -i\Psi^TC\gamma^a\Psi = 0$, because
  $C\gamma^a$ is antisymmetric and $u^TAu = 0$ for every antisymmetric $A$ (the
```

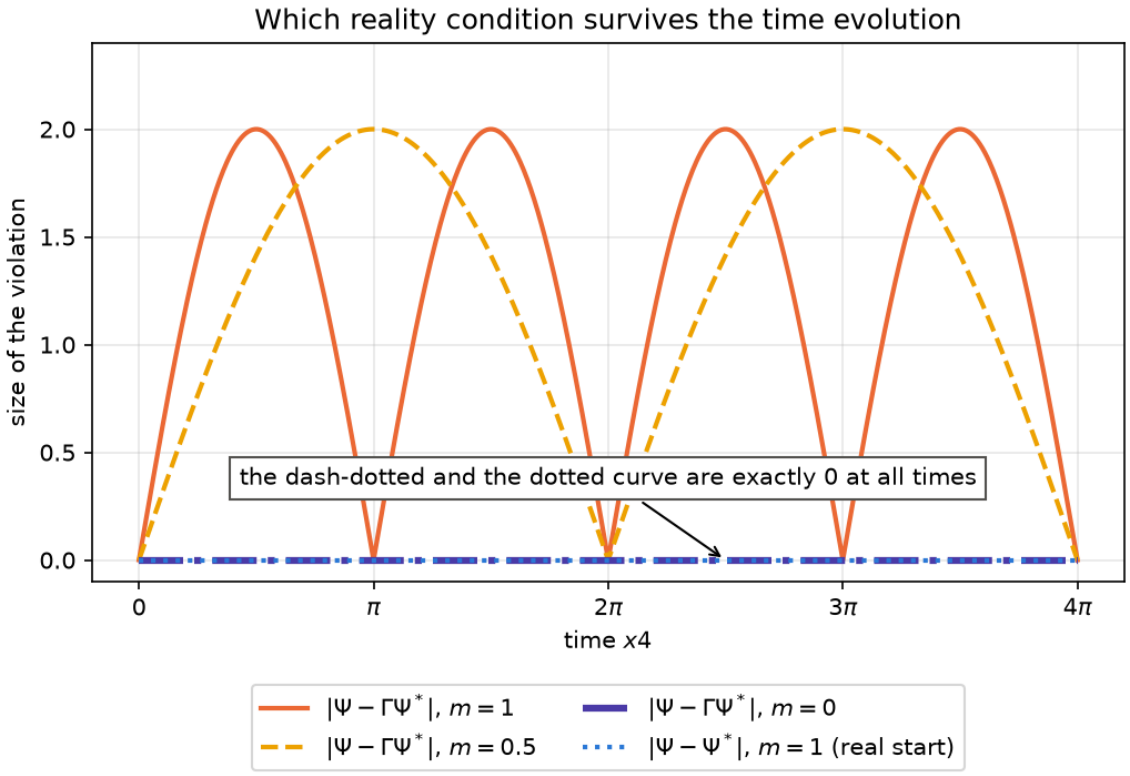

Figure 171: How far the free solution moves away from the two reality conditions, for starting columns of size 1 that obey them: the size of $\Psi - \Gamma\Psi^*$ for the masses $m = 1$ (solid), $m = 0.5$ (dashed) and $m = 0$ (dash-dotted, zero), and the size of $\Psi - \Psi^*$ for a real start (dotted, zero); horizontal axis the time $x4$, vertical axis the size (a pure number). The real field stays real; the condition $\Psi = \Gamma\Psi^*$ of the mass-reversing conjugation is violated by $2|\sin(mx4)|$ unless $m = 0$. (Notebook 05c, figure 7.)

```
  terms $u_rA_{rc}u_c$ and $u_cA_{cr}u_r$ cancel); a real field carries no U(1)
  charge;
- $\mathcal{C}_+$ acts as the identity: $\Psi^c_+ = \Psi^* = \Psi$ (a real field is
  its own conjugate);
- the **real matrix** $\Gamma$ maps real fields to real fields, keeps $S$
  ($\Gamma^TC\Gamma = C$) and reverses every kinetic matrix ($\Gamma^TC\gamma^a\Gamma
  = -C\gamma^a$): with $(m, \lambda) \to (-m, -\lambda)$ it maps solutions to
  solutions (the pairing theorem T1 of the Revision record).

The next cell checks the first statement with sympy for 16 real symbols $r_1, \dots,
r_{16}$ (all real columns at once), and the third one with exact matrices and on
the real solution $\Psi(x4)$ that starts at the real part of $\Psi_0$: $\Gamma\Psi$
solves the equation with $-m$.
```

In [18]:

```
r = sp.Matrix(sp.symbols("r1:17", real=True))  # a general real column
currents = [sp.expand((-sp.I * r.T * sp.Matrix((C @ gamma[x]).tolist()) * r)[0, 0])
            for x in COORDS]
say("J^a of a general real column, a = x0..x7: " + ", ".join(map(str, currents)))
S_change = sp.expand((GAMMA * r).T * sp.Matrix(C.tolist()) * (GAMMA * r)
                     - r.T * sp.Matrix(C.tolist()) * r)[0, 0]
kinetic_reversed = all(np.array_equal(Gamma.T @ C @ gamma[x] @ Gamma,
                                      -(C @ gamma[x])) for x in COORDS)
psi0_real = psi0.applyfunc(sp.re)  # the real part of every component of Psi_0
psi_real = sp.cos(m * x4) * psi0_real - sp.sin(m * x4) * G4 * psi0_real
image = GAMMA * psi_real  # the real field Gamma Psi
image_solves_minus = (G4 * image.diff(x4) + m * image).applyfunc(sp.expand) \
    == sp.zeros(16, 1)
check_reproduces(all(c == 0 for c in currents) and S_change == 0 and kinetic_reversed
                 and psi_real.conjugate() == psi_real and image_solves_minus
                 and recorded("real_fields_charge_conjugation"),
                 "real fields: J = 0, calC_+ is the identity, Gamma keeps S, reverses "
                 "the kinetic matrices and maps a solution with m to one with -m",
                 record=f"{RECORD}, check real_fields_charge_conjugation")
```

Out [18]:

```
J^a of a general real column, a = x0..x7: 0, 0, 0, 0, 0, 0, 0, 0
PASS real fields: J = 0, calC_+ is the identity, Gamma keeps S, reverses the kinetic
    matrices and maps a solution with m to one with -m
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    real_fields_charge_conjugation
```

Text cell:

```
The next cell draws the real solution, its $\mathcal{C}_+$ image (identical) and its
$\Gamma$ image, for one component of each half: component 1 changes sign under
$\Gamma$, component 9 does not.
```

In [19]:

```
real_t = evolve(PSI0.real.astype(complex), 1.0).real  # the real solution, m = 1
gamma_t = real_t @ Gamma.T  # Gamma Psi at every time
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
for ax, comp in zip(axes, [0, 8]):
    ax.plot(t, real_t[:, comp], color="#2a78d6", linewidth=3,
            label=r"real solution $\Psi$ (mass $m$)")
    ax.plot(t, real_t[:, comp], color="#eb6834", linewidth=2, linestyle="--",
            label=r"$\Psi^c_+ = \Psi^* = \Psi$ (identical)")
    ax.plot(t, gamma_t[:, comp], color="#1baf7a", linewidth=2, linestyle=":",
            label=r"$\Gamma\Psi$ (solves with mass $-m$)")
    ax.set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
                  ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
    ax.set_xlabel("time $x4$ (in units of $1/m$)")
    half = "first" if comp < 8 else "second"  # components 1-8 or 9-16
    ax.set_title(f"component {comp + 1} ({half} half)")
axes[0].set_ylabel("value of the component")
axes[1].text(2.0 * np.pi, 0.0, "all three curves coincide:\n"
             r"$\Gamma = +1$ on the second half", ha="center", va="center",
             bbox={"facecolor": "white", "edgecolor": "#52514e"})
axes[1].legend(loc="upper center", bbox_to_anchor=(-0.05, -0.17), ncol=3)
save_figure(fig, "real_field",
            "A real solution of the free equation (solid), its same-mass conjugate "
            "$\\Psi^{\\ast} = \\mathcal{C}_+\\bar\\Psi^T$ (dashed, lying exactly on the "
            "solid line) and its image $\\Gamma\\Psi$ under the real chirality matrix "
            "(dotted), for component 1 (left, first half) and component 9 (right, "
            "second half); horizontal axis the time $x4$ in units of $1/m$, vertical "
            "axis the value. For a real field the same-mass conjugation does nothing; "
            "of the two charge-conjugation matrices only $\\Gamma C$ changes a real "
            "field: its map $\\Psi \\to \\Gamma\\Psi$ flips the first half and "
            "solves the equation with the mass reversed.")
```

Out [19]:

```
Figure 05c.8 saved as Revision/textbook/figures/05c_8_real_field.png
```

Text cell:

```
## 14. The quantised field

After quantisation, the components of dirac16complex are operators with
$\{\Psi_r, \Psi_c^\dagger\} = B_{rc}\,\delta$. Consider the map $\Psi \to
M\Psi^{\dagger T}$ (the operator form of $M\Psi^*$) with a real $M$. Its adjoint
is $\Psi^\dagger \to (M\Psi^{\dagger T})^\dagger = \Psi^TM^\dagger$. The
anticommutator of the new operators is then $M\{\Psi^{\dagger T}, \Psi^T\}
M^\dagger = M B^T M^\dagger\,\delta$ (the anticommutator of $\Psi_c^\dagger$ with
$\Psi_r$ is $B_{rc}$, which is the entry $(c, r)$ of $B^T$). The rule is preserved
exactly when $MB^TM^\dagger = B$. The next cell checks the recorded result: it holds
for $M = \Gamma$ and fails for $M = 1$, where it gives $-B$ (because $B^T = -B$).
A factor $\lambda$ in front of $M$ multiplies $MB^TM^\dagger$ by $|\lambda|^2$, so
among the multiples of 1 and of $\Gamma$ only $\Gamma$, up to a phase factor
$e^{i\alpha}$, keeps the rule.
```

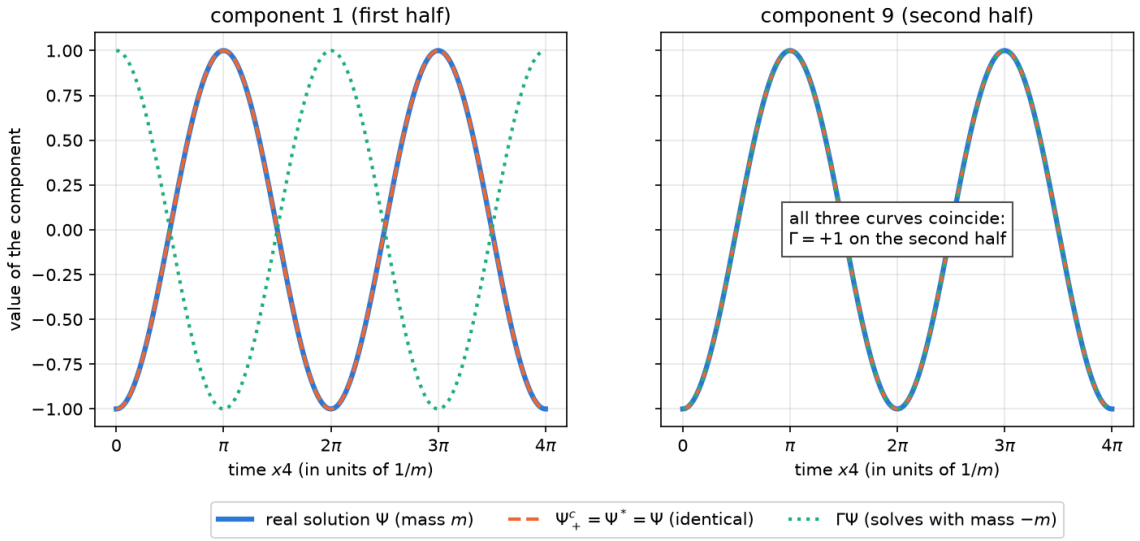

Figure 172: A real solution of the free equation (solid), its same-mass conjugate $\Psi^* = \mathcal{C}_+\bar{\Psi}^T$ (dashed, lying exactly on the solid line) and its image $\Gamma\Psi$ under the real chirality matrix (dotted), for component 1 (left, first half) and component 9 (right, second half); horizontal axis the time $x4$ in units of $1/m$, vertical axis the value. For a real field the same-mass conjugation does nothing; of the two charge-conjugation matrices only ΓC changes a real field: its map $\Psi \to \Gamma\Psi$ flips the first half and solves the equation with the mass reversed. (Notebook 05c, figure 8.)

In [20]:

```
q_plus = I16 @ B.T @ I16.conj().T  # M = 1
q_minus = Gamma @ B.T @ Gamma.conj().T  # M = Gamma
say(f"B^T = -B: {np.array_equal(B.T, -B)};  1 B^T 1 = B: "
    f"{np.array_equal(q_plus, B)};  Gamma B^T Gamma^dagger = B: "
    f"{np.array_equal(q_minus, B)}")
check_reproduces(np.array_equal(q_minus, B) and np.array_equal(q_plus, -B)
                 and recorded("quantum_charge_conjugation_unitary_type"),
                 "M B^T M^dagger = B for M = Gamma and = -B for M = 1: the conjugation "
                 "of the quantised field is Psi -> Gamma Psi^(dagger T), which reverses "
                 "the mass",
                 record=f"{RECORD}, check quantum_charge_conjugation_unitary_type")
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], B.imag, "imaginary part of $B$")
heat_map(axes[1], q_plus.imag, r"imaginary part of $1\,B^T 1$", row_label=False)
image = heat_map(axes[2], q_minus.imag, r"imaginary part of $\Gamma B^T\Gamma$",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "quantum_b",
            "The test of the canonical anticommutator: the imaginary parts of $B$ "
            "(left), of $B^T$ (middle, the result for $M = 1$) and of "
            "$\\Gamma B^T\\Gamma$ (right, the result for $M = \\Gamma$), as heat "
            "maps (column and row 1 to 16; blue $-1$, grey $0$, red $+1$; the real "
            "parts are zero). The middle picture has every colour reversed: "
            "$B^T = -B$, so $\\Psi \\to \\Psi^{\\dagger T}$ violates the rule. The "
            "right picture equals the left one: $\\Psi \\to \\Gamma\\Psi^{\\dagger "
```

```
"T}$ preserves it, and this conjugation reverses the mass.")
```

Out [20]:

```
B^T = -B: True;  1 B^T 1 = B: False;  Gamma B^T Gamma^dagger = B: True
PASS M B^T M^dagger = B for M = Gamma and = -B for M = 1: the conjugation of the
    quantised field is Psi -> Gamma Psi^(dagger T), which reverses the mass
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    quantum_charge_conjugation_unitary_type
```

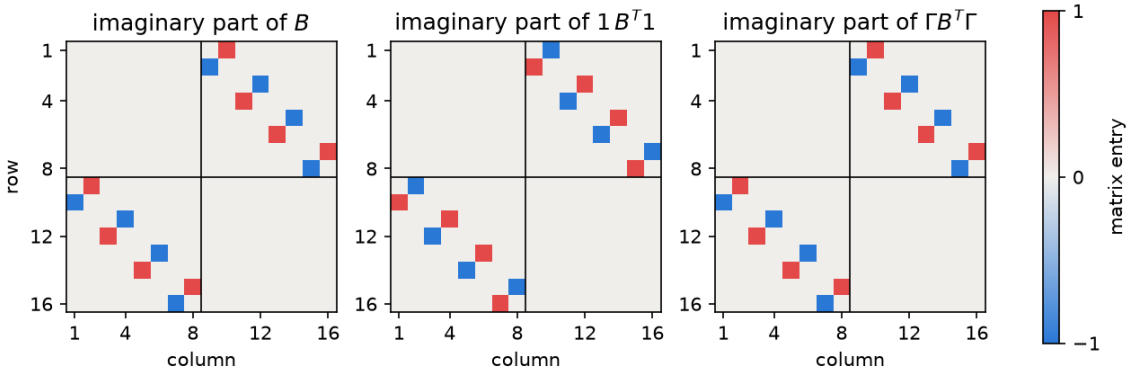

Figure 173: The test of the canonical anticommutator: the imaginary parts of B (left), of B^T (middle, the result for $M = 1$) and of $\Gamma B^T \Gamma$ (right, the result for $M = \Gamma$), as heat maps (column and row 1 to 16; blue -1, grey 0, red $+1$; the real parts are zero). The middle picture has every colour reversed: $B^T = -B$, so $\Psi \to \Psi^{\dagger T}$ violates the rule. The right picture equals the left one: $\Psi \to \Gamma\Psi^{\dagger T}$ preserves it, and this conjugation reverses the mass. (Notebook 05c, figure 9.)

Out [20]:

```
Figure 05c.9 saved as Revision/textbook/figures/05c_9_quantum_b.png
```

Text cell:

```
## 15. The last check

The last cell checks that the nine figure files exist and prints the number of
checks that passed.
```

In [21]:

```
FIGURES = ["05c_1_solution_spaces.png", "05c_2_conjugation_matrices.png",
           "05c_3_conjugate_solutions.png", "05c_4_which_mass.png",
           "05c_5_bilinears.png", "05c_6_sign_table.png",
           "05c_7_reality_in_time.png", "05c_8_real_field.png", "05c_9_quantum_b.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the nine figure files of notebook 05c exist")
all_checks_passed()
```

Out [21]:

```
PASS the nine figure files of notebook 05c exist
ALL 20 CHECKS PASSED (notebook 05c)
```

Text cell:

```
## 16. What this notebook showed

- Because the author's gammas are real, plain complex conjugation does nothing to a
  real field; charge conjugation is a **matrix** map $\Psi^c = \mathcal{C}\bar\Psi^T
  = \mathcal{C}C\Psi^*$.
- Solving $M(\gamma^a)^* = s\,\gamma^a M$ exactly: for $s = +1$ (same mass) only
  the multiples of 1, for $s = -1$ (mass reversed) only the multiples of $\Gamma$.
  Hence, with this intertwiner condition as the definition (as in the Revision
  record), exactly two charge-conjugation matrices up to a factor; each maps every
  solution to a solution (the converse is not claimed: for $V = 0$ the projector
  $P_-$ also does). They are $\mathcal{C}_+ = C$ with
  $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$, $\Psi^c = \Psi^*$; and
  $\mathcal{C}_- = \Gamma C$ with $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- =
  +(\gamma^a)^T$, $\Psi^c = \Gamma\Psi^*$. Solving the transposition equations
  directly gives the same two matrices.
- On explicit solutions: $\Psi^*$ solves the equation with the same mass,
  $\Gamma\Psi^*$ with the mass reversed (exact sympy computation and numbers).
- Commuting components: $\mathcal{C}_+$ keeps $S$ and reverses $J$;
  $\mathcal{C}_-$ keeps both. Anticommuting components: one more sign on each
  (classically), as the Revision record measured. For the quantised field the
  normal-ordered bilinears have the signs of the anticommuting components
  (computed in Notebook 05e).
- Both reality conditions are consistent ($MM^* = 1$); the real one is kept in
  time, the $\Gamma$ one only for $m = 0$.
- Real fields: $J = 0$, $\mathcal{C}_+$ is the identity, and among the matrices
  that obey the intertwiner condition the only nontrivial one is $\Gamma$ (up to a
  factor), with $(m, \lambda) \to (-m, -\lambda)$ (theorem T1).
- Quantised field: $\Psi \to \Gamma\Psi^{\dagger T}$ preserves $\{\Psi,
  \Psi^\dagger\} = B\delta$, $\Psi \to \Psi^{\dagger T}$ does not; among the
  multiples of the two matrices only $\Gamma$, up to a phase factor, keeps the
  rule, and this conjugation of the quantised field reverses the mass.
- Ten checks reproduce the Revision record
  `Revision/lead_checks/reports/charge-conjugation-and-u1.json`.
```

5.33 Line-by-line walk-through of Notebook 05c

The notebook has 21 code cells. In [1] is the set-up cell of Section 5.10 with `NOTEBOOK_ID = "05c"`; its comments repeat the instructions of Section 5.31. As in the earlier walk-throughs, the docstrings of the functions (the texts in triple quotes below a `def` line) are left out of the quotations, and a long caption given to `save_figure` is shortened to `...`; the full caption is printed under the figure in the complete text of the notebook.

In [2], the gammas, C, Γ, B and the lead report.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra
```

The modules `contextlib`, `io` and `sys` serve the helper `check_reproduces` below; numpy is loaded under its short name `np`.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1
```

These lines read the record of the gammas exactly as Notebook 05a does (Section 5.10, In [2]): `fixture` is the record, `COORDS` the eight coordinate names, `ETA` the signs η_{aa}, `gamma` the eight matrices as arrays of whole numbers (so that every product is exact) and `I16` the identity. A statement may run over two lines when the line break stands inside a bracket, as in the first line.

```
def product(directions):
    result = I16
    for d in directions:
        result = result @ gamma[d]
    return result
```

`product(["x0", "x1"])` multiplies the gammas of the listed directions in the order of the list, starting from the identity; `@` is the matrix product.

```
C = product(["x0", "x1", "x2", "x3"])  # the charge matrix (sigma16)
Gamma = product(["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"])  # the chirality
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4)
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]
S_gen = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}
```

C, Γ and $B = -iC\gamma^{(x4)}$ are built as in Sections 5.4 to 5.6 (`1j` is Python's imaginary unit i). `pairs` lists the 28 planes (a, b) with a before b (`enumerate` gives each name with its position `i`, and `COORDS[i + 1:]` is the list of the names after it), and `S_gen` holds the 28 generators $S^{ab} = \frac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$.

```
RECORD = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
lead = json.loads(repository_file(RECORD).read_text(encoding="utf-8"))
LEAD = {c["name"]: (c["verdict"].lower(), c["detail"]) for c in lead["checks"]}
```

`RECORD` is the path of the lead report. `lead` is the whole report read as a dictionary; `lead["checks"]` is its list of checks, and the dictionary comprehension stores each check under its name as the pair (verdict in small letters, detail text).

```
def recorded(name):
    return LEAD[name][0] == "pass"


def detail(name):
    return LEAD[name][1]


def check_reproduces(condition, name, record):
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together
```

`recorded(name)` is true when the lead report holds the check `name` with the verdict pass; `detail(name)` is its detail text (the second element `[1]` of the stored pair). `check_reproduces` is the helper of Notebook 05a (Section 5.10, In [3]): it lets `check` print the PASS line and the line that names the reproduced record into a text buffer and then sends both lines at once.

```
passed, total = lead["summary"]["passed"], lead["summary"]["total"]
say(f"the lead report holds {len(LEAD)} checks; passed: {passed} of {total}")
is_real = all(not np.iscomplexobj(gamma[x]) for x in COORDS)
```

The first line reads the two numbers of the report's summary; the printed line says that the report holds 12 checks, all passed. `np.iscomplexobj` is true for an array of complex numbers; `is_real` is true when none of the eight gammas is complex.

```
check_reproduces(is_real and np.array_equal(C.T, C) and np.array_equal(C @ C, I16)
                 and all(np.isrealobj(s) for s in S_gen.values())
                 and recorded("representation_real"),
                 "the gammas, C (symmetric, C C = 1) and the S^ab are real",
                 record=f"{RECORD}, check representation_real")
check_reproduces(not np.any(B.real) and np.array_equal(B.conj().T, B)
                 and np.array_equal(B @ B, I16) and recorded("B_imaginary_hermitian"),
                 "B = -i C gamma^(x4) is purely imaginary and Hermitian, B B = 1",
                 record=f"{RECORD}, check B_imaginary_hermitian")
```

The first check: the gammas are real, $C^T = C$, $CC = 1$, and every generator is real (`np.isrealobj`); this is the fact that makes plain complex conjugation do nothing on a real field. The second check: the real part of B is zero everywhere (`not np.any(B.real)`), $B^\dagger = B$ (`B.conj().T` is the conjugate transpose) and $BB = 1$. Both print PASS and the lead check they reproduce.

In [3], every conjugation matrix, solved exactly.

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over QQ


def conjugation_system(s):
    blocks = []
```

```
    for x in COORDS:
        G = np.conj(gamma[x]).astype(np.int64)  # (gamma^a)*, equal to gamma^a
        blocks.append(np.kron(I16, G.T) - s * np.kron(gamma[x], I16))
    return np.vstack(blocks)
```

`conjugation_system(s)` builds the coefficient matrix of the 2048 linear equations $M(\gamma^a)^* - s\,\gamma^a M = 0$, $a = x0, \ldots, x7$, for the 256 entries of M listed row by row. The rule of Section 5.13 for $LX - XR$ is used with the roles turned around: the entry (r, c) of MG is $\sum_k M_{rk} G_{kc}$, and the coefficient matrix of $M \to MG$ is $\mathrm{kron}(1, G^T)$; the entry (r, c) of GM is $\sum_k G_{rk} M_{kc}$, with the coefficient matrix $\mathrm{kron}(G, 1)$. `np.conj` writes out the complex conjugate although it equals the gamma (the gammas are real), so that the code says exactly what the equation says; `np.vstack` stacks the eight blocks of 256 rows.

```
def exact_solutions(A, n=16):
    null = DomainMatrix.from_list(A.tolist(), sp.QQ).nullspace().to_Matrix()
    return [np.array(null.row(k).tolist()[0], dtype=float).reshape(n, n)
            for k in range(null.rows)]
```

`exact_solutions(A)` returns a basis of all solutions of $Ay = 0$, each turned back into a 16×16 matrix, as `solution_basis` of Notebook 05b does (Section 5.17, In [3]): sympy's `DomainMatrix` over the rational numbers `QQ` finds the solutions by exact elimination, `nullspace()` returns one solution per row, and `reshape(n, n)` restores the matrix (`n=16` is a **default value**: the argument may be left out).

```
def proportional(X, Y):
    stacked = np.array([X.reshape(-1), Y.reshape(-1)], dtype=float)
    return bool(X.any() and Y.any() and np.linalg.matrix_rank(stacked) == 1)
```

`proportional(X, Y)` is true when both matrices are nonzero and one is a number times the other: written as two rows of 256 numbers (`reshape(-1)` lays out all entries in one row), they must have rank 1. `bool(...)` turns the result into a plain true or false.

```
A_same, A_reversed = conjugation_system(+1), conjugation_system(-1)
M_same, M_reversed = exact_solutions(A_same), exact_solutions(A_reversed)
say(f"s = +1: {A_same.shape[0]} equations, {len(M_same)} independent solution(s)")
say(f"s = -1: {A_reversed.shape[0]} equations, {len(M_reversed)} independent "
    f"solution(s)")
```

The two systems are built and solved. The printed lines report, for each sign, 2048 equations and 1 independent solution.

```
check_reproduces(len(M_same) == 1 and proportional(M_same[0], I16)
                 and "dimension 1" in detail("intertwiners_same_mass")
                 and recorded("intertwiners_same_mass"),
                 "M (gamma^a)* = + gamma^a M: one solution, the identity (same mass)",
                 record=f"{RECORD}, check intertwiners_same_mass")
check_reproduces(len(M_reversed) == 1 and proportional(M_reversed[0], Gamma)
```

```
        and "dimension 1" in detail("intertwiners_reversed_mass")
        and recorded("intertwiners_reversed_mass"),
        "M (gamma^a)* = - gamma^a M: one solution, Gamma (mass reversed)",
        record=f"{RECORD}, check intertwiners_reversed_mass")
```

Theorem CC (a) as two checks: for $s = +1$ exactly one solution, proportional to the identity; for $s = -1$ exactly one, proportional to Γ. Each also requires that the record's detail text contains the words `dimension 1`, the dimension the record found.

In [4], the second engine and the picture of the solutions.

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])


def heat_map(ax, matrix, title, row_label=True):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.set_yticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)
    return image
```

The colour map (blue -1, light grey 0, red $+1$) and the heat map of Notebook 05a (Section 5.10, In [6]), here always for a 16×16 matrix with the two black lines between the halves.

```
eigen_same = np.linalg.eigvalsh((A_same.T @ A_same).astype(float))
eigen_reversed = np.linalg.eigvalsh((A_reversed.T @ A_reversed).astype(float))
zeros_same = int(np.sum(np.abs(eigen_same) < 1e-9))
zeros_reversed = int(np.sum(np.abs(eigen_reversed) < 1e-9))
say(f"zero eigenvalues of A^T A: s = +1: {zeros_same}; s = -1: {zeros_reversed}")
check(zeros_same == 1 and zeros_reversed == 1,
      "numerical second engine: exactly one zero eigenvalue for each sign")
```

The second engine of Section 5.13: the number of solutions of $Ay = 0$ equals the number of zero eigenvalues of the symmetric matrix $A^T A$. `np.linalg.eigvalsh` computes the 256 eigenvalues in floating-point numbers (sorted from small to large), and the eigenvalues below 10^{-9} in size are counted. The printed line shows one for each sign, and the check requires it.

```
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0), width_ratios=[1, 1, 1.3])
shown_same = M_same[0] / M_same[0][0, 0]  # scaled: entry (1,1) equal to 1
shown_reversed = M_reversed[0] / M_reversed[0][0, 0] * Gamma[0, 0]  # as in Gamma
heat_map(axes[0], shown_same, "solution for $s = +1$")
image = heat_map(axes[1], shown_reversed, "solution for $s = -1$", row_label=False)
```

A solution is fixed only up to a factor. The solution for $s = +1$ is divided by its entry in row 1, column 1, so that this entry becomes 1, like the identity's; the solution for $s = -1$ is scaled so that this entry equals the one of Γ, which is -1. Both are drawn as heat maps.

```
index = np.arange(1, 257)
axes[2].plot(index, eigen_same, color="#2a78d6", linewidth=2,
             label="$s = +1$ (same mass)")
axes[2].plot(index, eigen_reversed, color="#eb6834", linewidth=2, linestyle="--",
             label="$s = -1$ (mass reversed)")
axes[2].plot([1], [eigen_same[0]], "o", color="#2a78d6", markersize=9)
axes[2].plot([1], [eigen_reversed[0]], "o", color="#eb6834", markersize=9)
axes[2].set_xlabel("number of the eigenvalue (sorted)")
axes[2].set_ylabel("eigenvalue of $A^T A$")
axes[2].set_title("one zero eigenvalue for each sign")
axes[2].legend(loc="lower right")
fig.colorbar(image, ax=axes[:2], ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "solution_spaces",
            ...)
```

The third picture plots the sorted eigenvalues of the two systems against their numbers 1 to 256, and a large dot on the first (smallest) eigenvalue of each, which is the zero one. The colour bar belongs to the first two pictures (`axes[:2]`). **What figure 05c.1 shows**: on the left the identity (a red diagonal), in the middle Γ (blue on the first eight diagonal places, red on the last eight), on the right two staircase curves of eigenvalues that each touch zero exactly once. The student should see that each equation system leaves exactly one free direction: the identity for the same mass, Γ for the reversed mass.

In [5], the two charge-conjugation matrices.

```
calC_plus = (M_same[0] / M_same[0][0, 0]) @ C  # M = 1 (scaled), so calC_+ = C
calC_minus = Gamma @ C  # M = Gamma, so calC_- = Gamma C
inverse_plus = np.linalg.inv(calC_plus)
inverse_minus = np.linalg.inv(calC_minus)
```

From $\mathcal{C} = MC$: the solution for $s = +1$, scaled to the identity, times C gives $\mathcal{C}_+$; ΓC is $\mathcal{C}_-$. `np.linalg.inv` computes the inverse matrices (for these signed permutation matrices the floating-point inverse is exact).

```
check_reproduces(np.array_equal(calC_plus, C)
                 and all(np.array_equal(inverse_plus @ gamma[x] @ calC_plus, -gamma[x].T)
                         for x in COORDS)
                 and np.array_equal(calC_plus.T, calC_plus)
                 and recorded("charge_conjugation_matrix_plus"),
                 "calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real and "
                 "symmetric",
                 record=f"{RECORD}, check charge_conjugation_matrix_plus")
check_reproduces(all(np.array_equal(inverse_minus @ gamma[x] @ calC_minus, gamma[x].T)
                     for x in COORDS)
                 and np.isrealobj(calC_minus)
                 and np.array_equal(calC_minus.T, calC_minus)
```

```
                and np.array_equal(calC_minus @ calC_minus, I16)
                and recorded("charge_conjugation_matrix_minus"),
                "calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real",
                record=f"{RECORD}, check charge_conjugation_matrix_minus")
```

Theorem CC (b) and (c): $\mathcal{C}_+ = C$ with $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$ for all eight directions, symmetric; $\mathcal{C}_-$ with $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = +(\gamma^a)^T$, real, symmetric and with square 1.

```
def transposition_system(zeta):
    return np.vstack([np.kron(gamma[x], I16) - zeta * np.kron(I16, gamma[x])
                      for x in COORDS])


X_minus = exact_solutions(transposition_system(-1))  # should be multiples of C
X_plus = exact_solutions(transposition_system(+1))  # should be multiples of Gamma C
say(f"zeta = -1: {len(X_minus)} solution(s); zeta = +1: {len(X_plus)} solution(s)")
check(len(X_minus) == 1 and proportional(X_minus[0], C)
      and len(X_plus) == 1 and proportional(X_plus[0], Gamma @ C),
      "the only solutions of gamma^a X = -X (gamma^a)^T are multiples of C, of "
      "gamma^a X = +X (gamma^a)^T multiples of Gamma C")
```

The independent derivation of Section 5.28 ("the same two matrices from the transposition rules"). For $\gamma^a X - \zeta X(\gamma^a)^T = 0$ the coefficient matrix is $\mathrm{kron}(\gamma^a, 1) - \zeta\,\mathrm{kron}(1, ((\gamma^a)^T)^T)$, and $((\gamma^a)^T)^T = \gamma^a$, which is why the second Kronecker factor is the gamma itself. The printed line reports one solution for each sign, and the check confirms that they are proportional to C and to ΓC.

In [6], the picture of the three matrices.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], calC_plus, r"$\mathcal{C}_+ = C$")
heat_map(axes[1], Gamma, r"$\Gamma$", row_label=False)
image = heat_map(axes[2], calC_minus, r"$\mathcal{C}_- = \Gamma C$",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "conjugation_matrices",
            ...)
```

Three heat maps and a colour bar. **What figure 05c.2 shows**: $\mathcal{C}_+ = C$ with blue squares in the pattern of σ in the top-left block and red ones in the bottom-right block; Γ, blue then red on the diagonal; and $\mathcal{C}_- = \Gamma C$, red in the pattern of σ in both diagonal blocks. The student should see that multiplying by Γ only reverses the signs of the rows 1 to 8, and that all three pictures are mirror-symmetric about the diagonal (all three matrices are symmetric).

In [7], the conjugate fields of a complex column.

```
k = np.arange(16)  # 0, 1, ..., 15
PSI0 = (k % 4 - 1) + 1j * (k % 3 - 1)  # a fixed complex column with small entries
say("Psi_0 = " + ", ".join(f"{z.real:+.0f}{z.imag:+.0f}i" for z in PSI0[:8]) + ",")
say("        " + ", ".join(f"{z.real:+.0f}{z.imag:+.0f}i" for z in PSI0[8:]))
```

A fixed complex column Ψ_0: its component number k (counted from 0) is $(k \bmod 4) - 1 + i\,((k \bmod 3) - 1)$, where `%` gives the remainder of a division. Operations on the array `k` act on all 16 entries at once. The two printed lines list the components in the form $a + bi$ (`:+.0f` writes a number with its sign and no decimals): $-1 - 1i, +0 + 0i, +1 + 1i, +2 - 1i, \ldots$.

```
psibar_T = C @ PSI0.conj()  # (Psi^dagger C)^T = C^T Psi* = C Psi*
check(np.array_equal(calC_plus @ psibar_T, PSI0.conj())
      and np.array_equal(calC_minus @ psibar_T, Gamma @ PSI0.conj()),
      "Psi^c = calC_+ Psibar^T = Psi* and Psi^c = calC_- Psibar^T = Gamma Psi*")
```

$\bar{\Psi}^T = C\Psi^*$ (Section 5.28) is computed for this column, and the check confirms the two formulas $\mathcal{C}_+\bar{\Psi}^T = \Psi^*$ and $\mathcal{C}_-\bar{\Psi}^T = \Gamma\Psi^*$ of Theorem CC (b), entry by entry.

In [8], which mass each field solves with, exactly.

```
x4, m = sp.symbols("x4 m", real=True)  # the time and the mass, real numbers
psi0 = sp.Matrix([sp.Integer(int(z.real)) + sp.I * sp.Integer(int(z.imag))
                  for z in PSI0])
G4 = sp.Matrix(gamma["x4"].tolist())  # gamma^(x4) as an exact sympy matrix
GAMMA = sp.Matrix(Gamma.tolist())
```

`sp.symbols("x4 m", real=True)` makes two sympy symbols, the time and the mass, declared real (so that sympy knows that their conjugates are themselves). `psi0` is Ψ_0 with exact whole-number real and imaginary parts (`sp.I` is sympy's i); `G4` and `GAMMA` are $\gamma^{(x4)}$ and Γ as exact sympy matrices.

```
psi = sp.cos(m * x4) * psi0 - sp.sin(m * x4) * G4 * psi0  # the solution
fields = {"Psi": psi, "Psi* (calC_+)": psi.conjugate(),
          "Gamma Psi* (calC_-)": GAMMA * psi.conjugate()}
solves = {}  # (field, mass sign) -> True when the residual is exactly zero
```

`psi` is the free solution $\Psi(x4) = \cos(m\,x4)\Psi_0 - \sin(m\,x4)\gamma^{(x4)}\Psi_0$ of Section 5.28, as a column of 16 exact expressions. `fields` collects it and its two images, Ψ^* (`conjugate()`) and $\Gamma\Psi^*$, under readable names.

```
for name, phi in fields.items():
    for sign in (+1, -1):
        residual = (G4 * phi.diff(x4) - sign * m * phi).applyfunc(sp.expand)
        solves[(name, sign)] = residual == sp.zeros(16, 1)
    say(f"{name:20} solves with +m: {solves[(name, 1)]!s:5}  "
        f"with -m: {solves[(name, -1)]!s:5}")
```

For each field Φ and each sign, the **residual** $\gamma^{(x4)}d\Phi/dx4 - (\pm m)\Phi$ is computed (`diff(x4)` differentiates every component); it is zero exactly when Φ solves the equation with that mass. `applyfunc(sp.expand)` multiplies out every component, and the comparison with the zero column `sp.zeros(16, 1)` is exact. Each printed line shows, for one field, True or False for $+m$ and $-m$ (`!s` writes the value as text, `:5` pads it to five characters).

```
expected = {("Psi", 1): True, ("Psi", -1): False,
            ("Psi* (calC_+)", 1): True, ("Psi* (calC_+)", -1): False,
            ("Gamma Psi* (calC_-)", 1): False, ("Gamma Psi* (calC_-)", -1): True}
check(solves == expected,
      "exactly: Psi and Psi* solve with +m, Gamma Psi* solves with -m")
```

The prediction of Section 5.28 is written down and compared with the six results: Ψ and Ψ^* solve with $+m$ only, $\Gamma\Psi^*$ with $-m$ only. The printed lines (True False, True False, False True) agree.

In [9], the same with numbers on 801 time points.

```
t = np.linspace(0.0, 4.0 * np.pi, 801)  # the time x4 in units of 1/m, m = 1
g4 = gamma["x4"].astype(float)
cos_t, sin_t = np.cos(t)[:, None], np.sin(t)[:, None]  # columns for broadcasting
psi_t = cos_t * PSI0 - sin_t * np.einsum("rc,c->r", g4, PSI0)  # row t: Psi(x4_t)
dpsi_t = -sin_t * PSI0 - cos_t * np.einsum("rc,c->r", g4, PSI0)  # dPsi/dx4
```

With $m = 1$ the time is measured in units of $1/m$. `t` holds 801 times from 0 to 4π. `[:, None]` turns a list of 801 numbers into a column of 801 rows, so that `cos_t * PSI0` multiplies each row of 16 components by its own number (numpy's **broadcasting**: an array with one column is repeated along the missing direction). `np.einsum("rc,c->r", g4, PSI0)` is the column $\gamma^{(x4)}\Psi_0$. So `psi_t` has one row per time, the solution at that time, and `dpsi_t` its derivative, $-\sin t\,\Psi_0 - \cos t\,\gamma^{(x4)}\Psi_0$.

```
numeric = {"Psi": (psi_t, dpsi_t),
           "Psi* (calC_+)": (psi_t.conj(), dpsi_t.conj()),
           "Gamma Psi* (calC_-)": (psi_t.conj() @ Gamma.T, dpsi_t.conj() @ Gamma.T)}
largest = {}  # (field, sign) -> largest size of the residual over the 801 points
```

The three fields with their derivatives. For a field stored as rows, multiplying every row by Γ is done by `@ Gamma.T`: a row u^T times Γ^T is the transpose of the column Γu.

```
for name, (phi, dphi) in numeric.items():
    for sign in (+1, -1):
        residual = np.einsum("rc,tc->tr", g4, dphi) - sign * 1.0 * phi
        largest[(name, sign)] = float(np.max(np.linalg.norm(residual, axis=1)))
    shown = {sign: (f"{largest[(name, sign)]:.2f}" if largest[(name, sign)] > 1e-12
                    else "below 1e-12") for sign in (1, -1)}  # rounding-proof text
    say(f"{name:20} largest residual with +m: {shown[1]:11}, with -m: {shown[-1]}")
```

For each field and sign the residual $\gamma^{(x4)}d\Phi/dx4 \mp \Phi$ is computed at all 801 times at once (`"rc,tc->tr"` multiplies $\gamma^{(x4)}$ with the column of every time t). `np.linalg.norm(..., axis=1)` is the length of the residual at each time, and the largest one is kept. A tiny residual is printed as the words below 1e-12 rather than as a number, because its last digits depend on the computer. The printed lines show below 1e-12 where the exact computation said True, and 11.83 where it said False.

```
check(all((largest[key] < 1e-12) == solves[key] for key in solves),
      "the numerical residuals agree with the exact result")
```

The check: a residual below 10^{-12} exactly where the exact computation found a solution.

In [10], the picture of the three fields.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
styles = {"Psi": ("#2a78d6", "-"), "Psi* (calC_+)": ("#eb6834", "--"),
          "Gamma Psi* (calC_-)": ("#1baf7a", ":")}
labels = {"Psi": r"$\Psi$", "Psi* (calC_+)": r"$\Psi^c_+ = \Psi^*$",
          "Gamma Psi* (calC_-)": r"$\Psi^c_- = \Gamma\Psi^*$"}
```

Two pictures that share the vertical axis (`sharey=True`), and for each field a colour, a line style (solid, dashed, dotted) and a legend label.

```
for name, (phi, _) in numeric.items():
    color, line = styles[name]
    axes[0].plot(t, phi[:, 0].real, color=color, linestyle=line, linewidth=2,
                 label=labels[name])
    axes[1].plot(t, phi[:, 0].imag, color=color, linestyle=line, linewidth=2,
                 label=labels[name])
```

For each field, the real part (left) and the imaginary part (right) of its first component (`phi[:, 0]` is column 0 of the rows, that is component 1 at every time) is drawn against the time. The derivative is not needed here, so it gets the throw-away name `_`.

```
for ax, part in zip(axes, ["real part", "imaginary part"]):
    ax.set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
                  ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
    ax.set_xlabel("time $x4$ (in units of $1/m$)")
    ax.set_title(f"component 1, {part}")
axes[0].set_ylabel("value of the component")
axes[1].legend(loc="upper center", bbox_to_anchor=(-0.05, -0.17), ncol=3)
save_figure(fig, "conjugate_solutions",
            ...)
```

Tick marks at multiples of π, axis labels and titles; one legend with three columns below both pictures. **What figure 05c.3 shows**: on the left the real part of Ψ^* (dashed) lies on that of Ψ (solid), and that of $\Gamma\Psi^*$ (dotted) is its mirror image; on the right the imaginary part of Ψ^* is the mirror image of that of Ψ, and that of $\Gamma\Psi^*$ lies on it. The reason: Ψ^* reverses the imaginary part, and Γ (which is -1 on the first half) reverses component 1 once more.

In [11], the table of the largest residuals.

```
from matplotlib.patches import Patch  # a coloured square for a legend

names = list(numeric)
table = np.array([[largest[(name, 1)], largest[(name, -1)]] for name in names])
fig, ax = plt.subplots(figsize=(7.0, 3.6))
solved = table < 1e-12  # True where the field solves the equation
ax.imshow(np.where(solved, 1.0, 0.0), cmap=LinearSegmentedColormap.from_list(
    "solve", ["#f0efec", "#2a78d6"]), vmin=0, vmax=1, aspect="auto")
```

The six largest residuals form a table of three rows (fields) and two columns (masses). `solved` is true where the residual is below 10^{-12}; `np.where(solved, 1.0, 0.0)` turns it into ones and zeros, drawn blue and grey with a two-colour map made on the spot.

```
for r in range(3):
    for c in range(2):
        text = "solves" if solved[r, c] else f"no ({table[r, c]:.2f})"
        ax.text(c, r, text, ha="center", va="center",
                color="white" if solved[r, c] else "black", fontweight="bold")
ax.axhline(0.5, color="white", linewidth=2)
ax.axhline(1.5, color="white", linewidth=2)
ax.axvline(0.5, color="white", linewidth=2)
ax.set_xticks([0, 1], ["mass $+m$", "mass $-m$"])
ax.set_yticks(range(3), [labels[name] for name in names])
ax.set_title("Largest residual of the field equation on $0 \\leq x4 \\leq 4\\pi$")
ax.legend(handles=[Patch(color="#2a78d6", label="residual below $10^{-12}$"),
                   Patch(color="#f0efec", label="residual of order 1")],
          loc="upper center", bbox_to_anchor=(0.5, -0.14), ncol=2)
ax.grid(False)
save_figure(fig, "which_mass",
            ...)
```

Each square gets the word solves or the word no with the residual; white lines separate the squares; the labels name the masses and the fields; a legend made of two coloured squares (`Patch`) goes below the picture. **What figure 05c.4 shows**: blue squares solves for Ψ and Ψ^* under $+m$ and for $\Gamma\Psi^*$ under $-m$, grey squares no (11.83) in the other three places: $\mathcal{C}_+$ keeps the mass, $\mathcal{C}_-$ reverses it.

In [12], the scalar and the charge density of the three fields.

```
bilinears = {}  # field -> (S at every time point, J^(x4) at every time point)
for name, (phi, _) in numeric.items():
    S_t = np.einsum("tr,rc,tc->t", phi.conj(), C, phi).real  # Psi^dagger C Psi
    J_t = np.einsum("tr,rc,tc->t", phi.conj(), B, phi).real  # Psi^dagger B Psi
    bilinears[name] = (S_t, J_t)
    say(f"{name:20} S from {S_t.min():+.6f} to {S_t.max():+.6f}; "
        f"J^(x4) from {J_t.min():+.6f} to {J_t.max():+.6f}")
```

For commuting components the scalar $S = \Psi^\dagger C \Psi$ and the charge density $J^{(x4)} = \Psi^\dagger B \Psi$ are computed at all 801 times (`"tr,rc,tc->t"` is $\sum_{r,c} \Psi^*_r K_{rc} \Psi_c$ for every time). Both are real (Section 5.4), and `.real` drops the imaginary part, which is zero up to rounding. Each printed line shows the smallest and the largest value over the times: S stays at -2 for all three fields, $J^{(x4)}$ at -6, $+6$ and -6. Both are constant. With $d\Psi/dx4 = -m\gamma^{(x4)}\Psi$ and $(\gamma^{(x4)}\Psi)^\dagger = \Psi^\dagger(\gamma^{(x4)})^T$, the product rule gives $dS/dx4 = -m\Psi^\dagger\big((\gamma^{(x4)})^T C + C\gamma^{(x4)}\big)\Psi$, which is 0 because $(\gamma^{(x4)})^T = -\gamma^{(x4)}$ and $\gamma^{(x4)}$ commutes with C (C1); in the same way $dJ^{(x4)}/dx4 = -m\Psi^\dagger\big((\gamma^{(x4)})^T B + B\gamma^{(x4)}\big)\Psi = 0$, because $(\gamma^{(x4)})^T B = iC\gamma^{(x4)}\gamma^{(x4)} = -iC$ and $B\gamma^{(x4)} = -iC\gamma^{(x4)}\gamma^{(x4)} = iC$.

```
S_psi, J_psi = bilinears["Psi"][0][0], bilinears["Psi"][1][0]
report("S of the solution Psi", f"{S_psi:+.6f}")
report("J^(x4) of the solution Psi", f"{J_psi:+.6f}")
constant = all(np.ptp(v) < 1e-12 for pair in bilinears.values() for v in pair)
```

The values of Ψ at the first time point, $x4 = 0$, are printed as two RESULT lines: $S = -2.000000$ and $J^{(x4)} = -6.000000$. `np.ptp` (peak to peak) is the largest minus the smallest value; `constant` is true when it is below 10^{-12} for all six lists.

```
# The values at x4 = 0, where Psi = Psi_0, are quoted in the caption of the next
# figure. Psi_0 has whole-number real and imaginary parts and C, B have entries 0,
# +-1, +-i, so S = Psi_0^dagger C Psi_0 and J^(x4) = Psi_0^dagger B Psi_0 are
# computed exactly (small whole numbers are stored without rounding).
S_exact = PSI0.conj() @ C @ PSI0
J_exact = PSI0.conj() @ B @ PSI0
check(S_exact == -2 and J_exact == -6 and S_psi == -2 and J_psi == -6,
      "at x4 = 0 exactly: S = -2 and J^(x4) = -6 for the solution Psi")
```

The four comment lines state the reason that the next sentence gives in full. At $x4 = 0$ the solution is Ψ_0, whose real and imaginary parts are whole numbers; C and B have the entries 0, ± 1, $\pm i$; so $\Psi_0^\dagger C \Psi_0$ and $\Psi_0^\dagger B \Psi_0$ are computed exactly (small whole numbers are stored without rounding). The check requires -2 and -6 exactly, the numbers quoted in the caption of the next figure.

```
check(constant and abs(S_psi) > 1 and abs(J_psi) > 1
      and np.allclose(bilinears["Psi* (calC_+)"][0], S_psi)
      and np.allclose(bilinears["Psi* (calC_+)"][1], -J_psi)
      and np.allclose(bilinears["Gamma Psi* (calC_-)"][0], S_psi)
      and np.allclose(bilinears["Gamma Psi* (calC_-)"][1], J_psi),
      "commuting components: Psi* keeps S and reverses J; Gamma Psi* keeps S and J")
```

The commuting rows of the sign table of Section 5.29, on this solution: all values constant, not zero (so that a sign can be seen), Ψ^* with the same S and the opposite $J^{(x4)}$, $\Gamma\Psi^*$ with the same S and the same $J^{(x4)}$.

In [13], the bar picture of S and J.

```
fig, ax = plt.subplots(figsize=(8.0, 4.2))
width = 0.26  # the width of one bar
for j, name in enumerate(names):
    color, _ = styles[name]
    values = [bilinears[name][0][0], bilinears[name][1][0]]
    bars = ax.bar(np.arange(2) + (j - 1) * width, values, width, color=color,
                  edgecolor="white", linewidth=2, label=labels[name])
    for bar, v in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width() / 2, v + (0.3 if v > 0 else -0.7),
                f"{v:+.0f}", ha="center")
```

Two groups of three bars: at the position 0 the scalar, at 1 the charge density; the three fields are shifted by -1, 0, $+1$ bar widths. Each bar gets its value written just above (positive) or below (negative) it; `bar.get_x() + bar.get_width() / 2` is the middle of the bar.

```
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks([0, 1], [r"scalar $S = \Psi^\dagger C\Psi$",
                       r"charge density $J^{(x4)} = \Psi^\dagger B\Psi$"])
```

```
ax.set_ylabel("value (constant along $x4$)")
ax.set_ylim(-8, 8)
ax.set_title("Commuting components: what the two conjugations do to S and J")
ax.legend(loc="upper left")
save_figure(fig, "bilinears",
            ...)
```

The zero line, the labels of the two groups, the vertical range from -8 to 8, the title and the legend. **What figure 05c.5 shows**: three bars at -2 in the left group; in the right group -6 (blue, Ψ), $+6$ (orange, Ψ^*) and -6 (aqua, $\Gamma\Psi^*$). For commuting components $\mathcal{C}_+$ reverses the charge and keeps the mass; $\mathcal{C}_-$ keeps the charge and reverses the mass.

In [14], the sign table.

```
K_S = C.astype(complex)  # the matrix of the scalar S
K_J = [-1j * (C @ gamma[x]) for x in COORDS]  # the matrices of the currents J^a


def sign_of(M, K, eps):
    new = eps * (M.conj().T @ K @ M).T
    return 1 if np.array_equal(new, K) else (-1 if np.array_equal(new, -K) else 0)
```

The matrices of the scalar and of the eight currents, and `sign_of(M, K, eps)`, which computes $K' = \epsilon(M^\dagger K M)^T$ of Section 5.29 and returns $+1$ if $K' = K$, -1 if $K' = -K$, and 0 otherwise. The comparisons are exact, because all entries are 0, ± 1 or $\pm i$.

```
measured = {}  # the record's keys, e.g. "plus,eps=1" -> [sign of S, [signs of J]]
for map_name, M in (("plus", I16), ("minus", Gamma)):
    for eps in (1, -1):
        measured[f"{map_name},eps={eps}"] = [
            sign_of(M, K_S, eps), [sign_of(M, K, eps) for K in K_J]]
for key, (sign_S, signs_J) in measured.items():
    say(f"{key:13} S -> {sign_S:+d} S;  J^a -> s J^a with s = "
        + " ".join(f"{s:+d}" for s in signs_J))
```

For the two maps ($M = 1$ and $M = \Gamma$) and the two kinds of components the signs are collected under the keys that the record uses, for example `plus,eps=1`. The four printed lines are the four rows of the table of Section 5.29: `plus,eps=1` $+1$ and eight times -1; `plus,eps=-1` -1 and eight times $+1$; `minus,eps=1` $+1$ and eight times $+1$; `minus,eps=-1` -1 and eight times -1.

```
recorded_table = json.loads(detail("bilinears_under_charge_conjugation")
                            .split("measured: ", 1)[1])
check_reproduces(measured == recorded_table
                 and recorded("bilinears_under_charge_conjugation"),
                 "the signs of S and J under calC_+ and calC_-, commuting and "
                 "anticommuting, equal the recorded table",
                 record=f"{RECORD}, check bilinears_under_charge_conjugation")
```

The record's detail text ends with the word measured, a colon and the table in JSON form. `split("measured: ", 1)[1]` takes the text after these words, and `json.loads` turns it into a dictionary

of the same shape as `measured`. The check requires the two to be equal: the notebook and the record measured the same 36 signs.

In [15], the picture of the sign table.

```
row_keys = ["plus,eps=1", "plus,eps=-1", "minus,eps=1", "minus,eps=-1"]
row_names = [r"$\mathcal{C}_+$, commuting", r"$\mathcal{C}_+$, anticommuting",
             r"$\mathcal{C}_-$, commuting", r"$\mathcal{C}_-$, anticommuting"]
grid = np.array([[measured[key][0]] + measured[key][1] for key in row_keys])
fig, ax = plt.subplots(figsize=(10.0, 3.8))
ax.imshow(grid, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
```

The four rows of signs, each the sign of S followed by the eight signs of the currents (`[a] + list` puts one element in front of a list), form a 4×9 array, drawn with the colour map of the heat maps.

```
for r in range(4):
    for c in range(9):
        ax.text(c, r, f"{grid[r, c]:+d}", ha="center", va="center", color="white",
                fontweight="bold")
for k in range(1, 4):
    ax.axhline(k - 0.5, color="white", linewidth=2)
for k in range(1, 9):
    ax.axvline(k - 0.5, color="white", linewidth=3 if k == 1 else 2)
ax.set_xticks(range(9), ["$S$"] + [rf"$J^{{({x})}}$" for x in COORDS])
ax.set_yticks(range(4), row_names)
ax.set_title("Sign of each bilinear after the conjugation (red $+1$ kept, blue "
             "$-1$ reversed)")
ax.grid(False)
save_figure(fig, "sign_table",
            ...)
```

Each square gets its sign in bold white; white lines separate the squares, with a thicker line between the column S and the currents; the columns are labelled S, $J^{(x0)}, \ldots, J^{(x7)}$ (the doubled braces of the `rf`-string give single braces) and the rows by map and kind of component. **What figure 05c.6 shows**: the row $\mathcal{C}_+$, commuting is red for S and blue for all currents; the row $\mathcal{C}_-$, commuting is red everywhere; each anticommuting row is the commuting row above it with every colour reversed. The student should see that the exchange of two anticommuting components flips every sign once more.

In [16], the reality conditions.

```
check_reproduces(np.array_equal(I16 @ np.conj(I16), I16)
                 and np.array_equal(Gamma @ np.conj(Gamma), I16)
                 and recorded("majorana_conditions_consistent"),
                 "M M* = 1 for M = 1 and M = Gamma: both reality conditions are "
                 "consistent",
                 record=f"{RECORD}, check majorana_conditions_consistent")
```

The consistency condition $MM^* = 1$ of Section 5.29 for $M = 1$ and $M = \Gamma$.

In [17], the reality conditions in time.

```
start_real = PSI0.real.astype(complex)  # obeys Psi_0* = Psi_0
start_gamma = np.concatenate([1j * PSI0.imag[:8], PSI0.real[8:]])  # first half
start_gamma = start_gamma / np.linalg.norm(start_gamma)  # imaginary, size 1
start_real = start_real / np.linalg.norm(start_real)  # size 1
check(np.allclose(start_gamma, Gamma @ start_gamma.conj())
      and np.allclose(start_real, start_real.conj()),
      "the two starting columns obey Psi_0 = Gamma Psi_0* and Psi_0 = Psi_0*")
```

Two starting columns of length 1: the real part of Ψ_0, which obeys $\Psi_0 = \Psi_0^*$; and a column whose first half is i times the imaginary parts of the first eight components of Ψ_0 and whose second half is the real parts of the last eight (`np.concatenate` joins two arrays), which obeys $\Psi_0 = \Gamma\Psi_0^*$. Dividing by `np.linalg.norm` (the length) makes the length 1. The check confirms the two conditions.

```
def evolve(start, mass):
    return (np.cos(mass * t)[:, None] * start
            - np.sin(mass * t)[:, None] * np.einsum("rc,c->r", g4, start))
```

`evolve(start, mass)` is the free solution with the given starting column and mass at all 801 times, one row per time.

```
violation = {}  # (condition, mass) -> size of the violation at every time
for mass in (1.0, 0.5, 0.0):
    f_real = evolve(start_real, mass)
    f_gamma = evolve(start_gamma, mass)
    violation[("real", mass)] = np.linalg.norm(f_real - f_real.conj(), axis=1)
    violation[("gamma", mass)] = np.linalg.norm(f_gamma - f_gamma.conj() @ Gamma.T,
                                                axis=1)
```

For the masses 1, 0.5 and 0 both solutions are computed, and at each time the length of $\Psi - \Psi^*$ (the violation of the real condition) and of $\Psi - \Gamma\Psi^*$ (the violation of the other one).

```
check(all(np.max(violation[("real", mass)]) < 1e-12 for mass in (1.0, 0.5, 0.0))
      and np.max(violation[("gamma", 0.0)]) < 1e-12
      and np.allclose(violation[("gamma", 1.0)], 2 * np.abs(np.sin(t)))
      and np.allclose(violation[("gamma", 0.5)], 2 * np.abs(np.sin(0.5 * t))),
      "Psi = Psi* is kept in time; Psi = Gamma Psi* is violated by 2 |sin(m x4)|")
```

The prediction of Section 5.29: the real condition is never violated; the other is violated by exactly $2|\sin(m\,x4)|$ (the starting column has length 1), which is zero only for $m = 0$.

```
fig, ax = plt.subplots(figsize=(8.0, 4.2))
ax.plot(t, violation[("gamma", 1.0)], color="#eb6834", linewidth=2,
        label=r"$|\Psi - \Gamma\Psi^*|$, $m = 1$")
ax.plot(t, violation[("gamma", 0.5)], color="#eda100", linewidth=2, linestyle="--",
        label=r"$|\Psi - \Gamma\Psi^*|$, $m = 0.5$")
ax.plot(t, violation[("gamma", 0.0)], color="#4a3aa7", linewidth=3, linestyle="-.",
        label=r"$|\Psi - \Gamma\Psi^*|$, $m = 0$")
ax.plot(t, violation[("real", 1.0)], color="#2a78d6", linewidth=2, linestyle=":",
        label=r"$|\Psi - \Psi^*|$, $m = 1$ (real start)")
```

Four curves: the violation of $\Psi = \Gamma\Psi^*$ for the three masses (solid, dashed, dash-dotted) and that of the real condition for $m = 1$ (dotted).

```
ax.set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
              ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
ax.set_xlabel("time $x4$")
ax.set_ylabel("size of the violation")
ax.set_ylim(-0.1, 2.4)
ax.annotate("the dash-dotted and the dotted curve are exactly 0 at all times",
            (2.5 * np.pi, 0.0), xytext=(2.0 * np.pi, 0.35), ha="center",
            arrowprops={"arrowstyle": "->", "color": "black"},
            bbox={"facecolor": "white", "edgecolor": "#52514e"})
ax.set_title("Which reality condition survives the time evolution")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=2)
save_figure(fig, "reality_in_time",
            ...)
```

Tick marks, labels, the vertical range, and an **annotation**: a text in a white box with an arrow pointing at the place $(2.5\pi, 0)$, where two curves lie on the axis and would otherwise be hard to see. **What figure 05c.7 shows**: the solid curve $2|\sin x4|$ with arches of height 2, the dashed curve $2|\sin(x4/2)|$ with arches twice as wide, and two curves lying exactly on zero: the real field stays real, and the condition of the mass-reversing conjugation survives only for $m = 0$.

In [18], real fields.

```
r = sp.Matrix(sp.symbols("r1:17", real=True))  # a general real column
currents = [sp.expand((-sp.I * r.T * sp.Matrix((C @ gamma[x]).tolist()) * r)[0, 0])
            for x in COORDS]
say("J^a of a general real column, a = x0..x7: " + ", ".join(map(str, currents)))
```

`sp.symbols("r1:17", real=True)` makes the 16 real symbols $r_1, \ldots, r_{16}$, and `r` is the column of them: all real columns at once. For each direction the current $J^a = -i\, r^T C\gamma^a r$ is multiplied out (`[0, 0]` takes the single entry of the 1×1 result). The printed line shows eight zeros: fact (F1) of Section 5.29.

```
S_change = sp.expand((GAMMA * r).T * sp.Matrix(C.tolist()) * (GAMMA * r)
                     - r.T * sp.Matrix(C.tolist()) * r)[0, 0]
kinetic_reversed = all(np.array_equal(Gamma.T @ C @ gamma[x] @ Gamma,
                                      -(C @ gamma[x])) for x in COORDS)
```

`S_change` is $S(\Gamma r) - S(r)$ for the general real column, multiplied out; it must be 0. `kinetic_reversed` checks $\Gamma^T C\gamma^a\Gamma = -C\gamma^a$ for the eight directions, (X8).

```
psi0_real = psi0.applyfunc(sp.re)  # the real part of every component of Psi_0
psi_real = sp.cos(m * x4) * psi0_real - sp.sin(m * x4) * G4 * psi0_real
image = GAMMA * psi_real  # the real field Gamma Psi
image_solves_minus = (G4 * image.diff(x4) + m * image).applyfunc(sp.expand) \
    == sp.zeros(16, 1)
```

The real free solution that starts at the real part of Ψ_0 (`sp.re` is the real part), its image $\Gamma\Psi$, and the exact test whether the image solves the equation with $-m$: the residual $\gamma^{(x4)} d\Phi/dx4 + m\Phi$ must vanish. A backslash at the end of a line continues the statement on the next line.

```
check_reproduces(all(c == 0 for c in currents) and S_change == 0 and kinetic_reversed
                 and psi_real.conjugate() == psi_real and image_solves_minus
                 and recorded("real_fields_charge_conjugation"),
                 "real fields: J = 0, calC_+ is the identity, Gamma keeps S, reverses "
                 "the kinetic matrices and maps a solution with m to one with -m",
                 record=f"{RECORD}, check real_fields_charge_conjugation")
```

The three facts (F1) to (F3) of Section 5.29 in one check: the currents vanish, the real solution equals its conjugate (so $\mathcal{C}_+$ acts as the identity on it), Γ keeps S, reverses the kinetic matrices, and maps the real solution with mass m to a real solution with mass $-m$.

In [19], the picture of a real field.

```
real_t = evolve(PSI0.real.astype(complex), 1.0).real  # the real solution, m = 1
gamma_t = real_t @ Gamma.T  # Gamma Psi at every time
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
```

The real solution for $m = 1$ at the 801 times (the starting column is not normalised here), and its image under Γ; two pictures with a shared vertical axis.

```
for ax, comp in zip(axes, [0, 8]):
    ax.plot(t, real_t[:, comp], color="#2a78d6", linewidth=3,
            label=r"real solution $\Psi$ (mass $m$)")
    ax.plot(t, real_t[:, comp], color="#eb6834", linewidth=2, linestyle="--",
            label=r"$\Psi^c_+ = \Psi^* = \Psi$ (identical)")
    ax.plot(t, gamma_t[:, comp], color="#1baf7a", linewidth=2, linestyle=":",
            label=r"$\Gamma\Psi$ (solves with mass $-m$)")
    ax.set_xticks([0, np.pi, 2 * np.pi, 3 * np.pi, 4 * np.pi],
                  ["0", r"$\pi$", r"$2\pi$", r"$3\pi$", r"$4\pi$"])
    ax.set_xlabel("time $x4$ (in units of $1/m$)")
    half = "first" if comp < 8 else "second"  # components 1-8 or 9-16
    ax.set_title(f"component {comp + 1} ({half} half)")
```

For component 1 (left; Python's index 0) and component 9 (right; index 8) three curves are drawn: the real solution (thick, solid), its $\mathcal{C}_+$ image, which is the same curve (dashed on top of it), and its Γ image (dotted). The title names the component and its half.

```
axes[0].set_ylabel("value of the component")
axes[1].text(2.0 * np.pi, 0.0, "all three curves coincide:\n"
             r"$\Gamma = +1$ on the second half", ha="center", va="center",
             bbox={"facecolor": "white", "edgecolor": "#52514e"})
axes[1].legend(loc="upper center", bbox_to_anchor=(-0.05, -0.17), ncol=3)
save_figure(fig, "real_field",
            ...)
```

A text box in the right picture explains why only one curve is visible there, and one legend goes below both pictures. **What figure 05c.8 shows**: on the left the dashed curve lies exactly on the solid one, and the dotted curve is their mirror image; on the right all three coincide. For a real field the same-mass conjugation does nothing; of the two charge-conjugation matrices, the only one that changes a real field is ΓC, whose map $\Psi \to \Gamma\Psi$ flips the first half and belongs to the mass $-m$.

In [20], the quantised field.

```
q_plus = I16 @ B.T @ I16.conj().T  # M = 1
q_minus = Gamma @ B.T @ Gamma.conj().T  # M = Gamma
say(f"B^T = -B: {np.array_equal(B.T, -B)};  1 B^T 1 = B: "
    f"{np.array_equal(q_plus, B)};  Gamma B^T Gamma^dagger = B: "
    f"{np.array_equal(q_minus, B)}")
```

The test of Section 5.34: a map $\Psi \to M\Psi^{\dagger T}$ of the quantised field keeps the canonical anticommutator exactly when $MB^TM^\dagger = B$. The cell computes $MB^TM^\dagger$ for $M = 1$ and $M = \Gamma$. The printed line shows True for $B^T = -B$, False for $M = 1$ and True for $M = \Gamma$.

```
check_reproduces(np.array_equal(q_minus, B) and np.array_equal(q_plus, -B)
                 and recorded("quantum_charge_conjugation_unitary_type"),
                 "M B^T M^dagger = B for M = Gamma and = -B for M = 1: the conjugation "
                 "of the quantised field is Psi -> Gamma Psi^(dagger T), which reverses "
                 "the mass",
                 record=f"{RECORD}, check quantum_charge_conjugation_unitary_type")
```

The check reproduces the record: $\Gamma B^T\Gamma^\dagger = B$, while $M = 1$ gives $-B$.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], B.imag, "imaginary part of $B$")
heat_map(axes[1], q_plus.imag, r"imaginary part of $1\,B^T 1$", row_label=False)
image = heat_map(axes[2], q_minus.imag, r"imaginary part of $\Gamma B^T\Gamma$",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "quantum_b",
            ...)
```

Three heat maps of imaginary parts (the real parts are zero). **What figure 05c.9 shows**: the middle picture has every colour of the left one reversed ($B^T = -B$), while the right picture equals the left one: of the two maps, only $\Psi \to \Gamma\Psi^{\dagger T}$ keeps the canonical rule.

In [21], the last check.

```
FIGURES = ["05c_1_solution_spaces.png", "05c_2_conjugation_matrices.png",
           "05c_3_conjugate_solutions.png", "05c_4_which_mass.png",
           "05c_5_bilinears.png", "05c_6_sign_table.png",
           "05c_7_reality_in_time.png", "05c_8_real_field.png", "05c_9_quantum_b.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the nine figure files of notebook 05c exist")
all_checks_passed()
```

The nine figure files must exist, and the last line reads ALL 20 CHECKS PASSED (notebook 05c): two checks in In [2], two in In [3], one in In [4], three in In [5], one in In [7], one in In [8], one in In [9], two in In [12], one in In [14], one in In [16], two in In [17], one in In [18], one in In [20] and one in In [21]. Ten of them reproduce checks of the lead report (the two of In [2], the two of In [3], the first two of In [5], and those of In [14], In [16], In [18] and In [20]); the other ten are the notebook's own computations. The two lead checks that the notebook does not repeat are `spinor_connection_real` and `u1_noether_matrix_identity`, which need the curved metric (Chapters 6 and 21); they are made at $b_4 = 1$, and `Revision/theory/reports/python-field-theory-b4.json` makes both for a general b_4 (checks `spinor_connection_real_b4` and `u1_noether_matrix_identity_b4`).

5.34 The quantised field: operators, the indefinite form and normal ordering

The field dirac16complex is **quantised** (Chapter 10 does this in full): its 16 components become operators, and a charge conjugation of the quantised field must be a map of operators that keeps the basic rule of the quantum theory. This section explains the few words of quantum theory that are needed, from zero, and then derives which conjugation survives and what it does to the scalar and to the currents. Notebook 05e computes every statement on an explicit computer model with $2^{16} = 65536$ quantum states.

Operators, anticommutators, modes. In quantum theory a **state** is a column of numbers, and an **operator** is a rule that turns a state into another state; here every operator is a (big) matrix acting on columns. Operators are multiplied by doing one after the other, and the order matters. The **anticommutator** of two operators is $\{X, Y\} = XY + YX$. A **fermion mode** is a place that holds zero or one particle. With n modes a basic state is a **pattern** of n occupations, each 0 or 1, so there are 2^n patterns; all combinations of them form the **Fock space**. The **annihilation operator** f_p empties mode p (and gives zero if it is empty), and the **creation operator** f_p^* fills it (zero if it is full), each with the sign $(-1)^{n_{<p}}$, where $n_{<p}$ is the number of occupied modes below p. Example with two modes 0 and 1: write the patterns as $|00\rangle$ (both empty), $|10\rangle$ (mode 0 full), $|01\rangle$ (mode 1 full) and $|11\rangle$ (both full). Then $f_0|11\rangle = |01\rangle$ (no mode below 0), while $f_1|11\rangle = -|10\rangle$ (mode 0 below is full). Because of this sign, operators of different modes **anticommute**; together these rules give the **canonical anticommutation relations**

$$\{f_p, f_q^*\} = \delta_{pq}, \qquad \{f_p, f_q\} = 0, \qquad \{f_p^*, f_q^*\} = 0,$$

where δ_{pq} is 1 for $p = q$ and 0 otherwise; in particular $f_p f_p = 0$: no mode holds two fermions. Check of one case: $f_0 f_1|11\rangle = f_0(-|10\rangle) = -|00\rangle$ and $f_1 f_0|11\rangle = f_1|01\rangle = |00\rangle$ (mode 0 is now empty, so no sign); the two add up to zero, $\{f_0, f_1\}|11\rangle = 0$. On the Fock space the ordinary inner product $\langle a|b\rangle = \sum_n a_n^* b_n$ is **positive**: $\langle a|a\rangle$ is a sum of squared sizes. The **Hilbert adjoint** X^* of an operator is the conjugate transpose of its matrix; f_p^* is the Hilbert adjoint of f_p. For every operator X and every state ϕ,

$$\langle\phi|\{X, X^*\}|\phi\rangle = \langle\phi|XX^*|\phi\rangle + \langle\phi|X^*X|\phi\rangle = |X^*\phi|^2 + |X\phi|^2 \geq 0,$$

because $\langle\phi|XY\phi\rangle = \langle X^*\phi|Y\phi\rangle$ (the defining property of the adjoint) and $|v|^2 = \langle v|v\rangle$ is a squared length. **The anticommutator of an operator with its Hilbert adjoint is never negative.**

Momentum, energy and the mode Hamiltonian. A few more words describe how a field changes along a direction and in time.

- A field of **one momentum** k along $x1$ is a field of the form $\Psi = \psi\, e^{ik\,x1}$: a column ψ (which may still depend on the time) times the number $e^{ik\,x1} = \cos(k\,x1) + i\sin(k\,x1)$, which goes once around the circle of complex numbers of size 1 each time $x1$ grows by $2\pi/k$. The real number k is the **momentum** of the field along $x1$. A momentum may have parts along several directions; when it has no part along the extra times $x5$, $x6$, $x7$, the field does not depend on them, and the record calls this the **good sector**. (Outside the good sector the record's check `mode_hamiltonian_good_sector` shows that the energy defined next can become imaginary and the field then grows exponentially in time; Chapters 8 and 10 treat this.)
- A column that depends on the time as $e^{-iE\,x4}$, with a real number E, has the **energy** E: the larger E, the faster the number $e^{-iE\,x4}$ goes around the circle of complex numbers of size 1 as the time $x4$ grows.

The record's example treats the field near one point of space as if the metric were flat there (its check `mode_hamiltonian_good_sector` calls these frozen-coefficient, flat-frame plane waves): the gammas of the curved space are replaced by the constant gammas and the spin connection is left out. For a free field ($V = m$) that depends only on $x4$ and $x1$, the field equation of Section 5.28 then keeps two terms, $\gamma^{(x4)}\partial_{x4}\Psi + \gamma^{(x1)}\partial_{x1}\Psi = m\Psi$. Insert $\Psi = u\,e^{i(k\,x1 - E\,x4)}$ with a constant column u. The derivative of e^{icy} with respect to y is $ic\,e^{icy}$ (differentiate $\cos(cy) + i\sin(cy)$ term by term), so ∂_{x4} brings down the factor $-iE$ and ∂_{x1} the factor ik; dividing by the nonzero number $e^{i(k\,x1 - E\,x4)}$ leaves $-iE\,\gamma^{(x4)}u + ik\,\gamma^{(x1)}u = mu$. Multiplying from the left by $-\gamma^{(x4)}$ and using $\gamma^{(x4)}\gamma^{(x4)} = -1$ gives $-iE\,u - ik\,\gamma^{(x4)}\gamma^{(x1)}u = -m\gamma^{(x4)}u$; multiplying by i gives $E\,u + k\,\gamma^{(x4)}\gamma^{(x1)}u = -im\gamma^{(x4)}u$, that is

$$E\,u = h\,u, \qquad h = -im\,\gamma^{(x4)} - k\,\gamma^{(x4)}\gamma^{(x1)}.$$

So the energies of the fields of one momentum k are the eigenvalues of the 16×16 matrix h, and the columns u are its **eigenvectors** (the columns $v \neq 0$ with $hv = Ev$, Rule 6 of Section 5.3). This matrix is called the **mode Hamiltonian** of the momentum k.

Two columns u and w are **orthonormal** when each has length 1 and they are perpendicular: $u^\dagger u = w^\dagger w = 1$ and $u^\dagger w = 0$. A list of columns is orthonormal when every two of them are.

The canonical rule of the record. The Revision record quantises dirac16complex with the canonical anticommutator on a slice of constant time $x4$,

$$\{\Psi_A(x), \Psi_C^\dagger(y)\} = B_{AC}\,\frac{\delta^7(x-y)}{\cos z},$$

where $\cos z$ is the volume factor ($z = 6Hx0$; the record writes its formulas for the author's metric at $b_4 = 1$, where $\sqrt{|g|} = b_4^{3/2}\cos z$ of Chapter 3 is $\cos z$) and $\delta^7(x-y)$ is the **delta function** of the seven other coordinates: an idealised object that is zero whenever the point x differs from the point y and whose integral over the seven coordinates is 1 (think of ever narrower and ever taller bumps of area 1). So operators at two different points of the slice anticommute, and only operators at the same point have a nonzero anticommutator (formula `quantisation` of `Revision`

`/theory/field-theory.json`; Chapter 10 derives it). For the operators of one single momentum, the record's exact Fock-space example, the delta function and the factor are absent and the rule reads $\{\Psi_A, \Psi_C^\dagger\} = B_{AC}$ for $A, C = 1, \ldots, 16$.

Why $\Psi^\dagger$ cannot be the Hilbert adjoint. Suppose $\Psi_A^\dagger$ were the Hilbert adjoint Ψ_A^*. Every diagonal entry of B is zero (the table of Section 5.6: row r of B/i never points to column r). So the rule would give $\{\Psi_A, \Psi_A^*\} = B_{AA} = 0$, and by the inequality above $|\Psi_A^*\phi|^2 + |\Psi_A\phi|^2 = 0$ for every state ϕ; a sum of two squared lengths is zero only when both are, so $\Psi_A\phi = 0$ for every ϕ: Ψ_A would be the zero operator. But row A of B contains a nonzero entry B_{AC}, and the anticommutator of the zero operator with anything is zero, not B_{AC}: a contradiction. The record makes the same argument with the column u that has $u_7 = -i/\sqrt{2}$, $u_{16} = 1/\sqrt{2}$ and zeros elsewhere, which obeys $Bu = -u$: the operator $X = \sum_A u_A^* \Psi_A$ has $\{X, X^\dagger\} = u^\dagger B u = -1 < 0$, impossible for a Hilbert adjoint (check `no_positive_inner_product` of `Revision/theory/reports/wolfram-field-theory.jso n`). So the canonical rule needs an **indefinite** inner product, one in which some states have negative squared length; a space with such a product is called a **Krein space** (Chapter 10).

The record's positive realisation. The record keeps the positive Fock space and writes $\Psi^\dagger = \chi B$, where χ is the Hilbert adjoint of Ψ. The record's example is one momentum in the good sector, with the mass $m = 3$ and the momentum $k = 4$ along $x1$. Its mode Hamiltonian, the matrix h derived above with $k = 4$, is

$$h = -im\,\gamma^{(x4)} - 4\,\gamma^{(x4)}\gamma^{(x1)}.$$

(The record gives the same matrix from the **energy density**, the energy per unit volume: it writes the energy density of one momentum k as $\Psi^\dagger(mC - iC\,k_a\gamma^a)\Psi = \chi h \Psi$, so $h = B(mC - iC\,k_a\gamma^a)$. With $BC = -iC\gamma^{(x4)}C = -i\gamma^{(x4)}$, because $C\gamma^{(x4)}C = -(\gamma^{(x4)})^T = \gamma^{(x4)}$ by (C5) and the antisymmetry of $\gamma^{(x4)}$, and with $k_{x1} = 4$ the only nonzero momentum, this is $h = -im\gamma^{(x4)} - 4i(-i\gamma^{(x4)})\gamma^{(x1)}$, the matrix above; check `mode_hamiltonian_Krein_selfadjoint` of `Revision/theory/reports/ wolfram-field-theory.json`.) Both terms are Hermitian ($\gamma^{(x4)}$ is real and antisymmetric, so $-i\gamma^{(x4)}$ is Hermitian; $\gamma^{(x4)}\gamma^{(x1)}$ is real and, by Rule 4 of Section 5.3, symmetric), they anticommute (Rule 1), and each squares to 1 ($(-i\gamma^{(x4)})^2 = -\gamma^{(x4)}\gamma^{(x4)} = 1$; $(\gamma^{(x4)}\gamma^{(x1)})^2 = -\eta_{x4\,x4}\eta_{x1\,x1}1 = 1$). Hence $hh = (m^2 + 4^2)1 = 25 \cdot 1$, and the eigenvalues of h are ± 5: the energy is $E = \sqrt{3^2 + 4^2} = 5$. Let $u_1, \ldots, u_8$ be orthonormal eigenvectors with $hu_s = 5u_s$ and $v_1, \ldots, v_8$ orthonormal eigenvectors with $hv_s = -5v_s$ (Notebook 05e constructs them). The 16 columns together are orthonormal, because an eigenvector u with the eigenvalue 5 and an eigenvector v with the eigenvalue -5 of the Hermitian matrix h are perpendicular: $5\,u^\dagger v = (hu)^\dagger v = u^\dagger h^\dagger v = u^\dagger h v = -5\,u^\dagger v$, so $10\,u^\dagger v = 0$. They form a matrix W with $W^\dagger W = 1$ (the entries of $W^\dagger W$ are the products $u^\dagger w$ of the columns, so $W^\dagger W = 1$ says exactly that the columns are orthonormal); $W^\dagger W = 1$ says that $W^\dagger$ is the inverse of the square matrix W, and then also $WW^\dagger = 1$. With eight particle modes b_s and eight antiparticle modes d_s (sixteen modes in all) the field operators are

$$\Psi_A = \sum_{s=1}^{8} \left((u_s)_A\, b_s + (v_s)_A\, d_s^*\right), \qquad \chi_A = \sum_{s=1}^{8} \left((u_s)_A^*\, b_s^* + (v_s)_A^*\, d_s\right), \qquad \Psi_A^\dagger = \sum_C \chi_C B_{CA}.$$

Line by line: by the canonical relations only the pairs (b_s, b_s^*) and (d_s^*, d_s) have a nonzero anticommutator, equal to 1, so

$$\{\Psi_A, \chi_C\} = \sum_s (u_s)_A (u_s)^*_C + \sum_s (v_s)_A (v_s)^*_C = (WW^\dagger)_{AC} = \delta_{AC},$$

and then $\{\Psi_A, \Psi^\dagger_C\} = \sum_D \{\Psi_A, \chi_D\} B_{DC} = B_{AC}$: the canonical rule holds. The record also computes the energy $\chi h \Psi$, whose value in the **vacuum** $|0\rangle$ (no mode occupied) is $-8E = -40$ (the eight negative-energy solutions form a filled **sea**), and, after **normal ordering** (the subtraction of the vacuum value, defined below), the energy $+5$ for each of the 16 one-quantum states $b^*_s|0\rangle$ and $d^*_s|0\rangle$, and the charge $\Psi^\dagger B \Psi$ equal to $+1$ for the eight particles and -1 for the eight antiparticles (check `Fock_space_good_sector_example`). Notebook 05e reproduces all of this.

Which conjugation keeps the rule. For a constant matrix M (real or complex) define the conjugated field $\Psi'_A = \sum_C M_{AC} \Psi^\dagger_C$, the operator form of $M\Psi^*$, written $\Psi' = M\Psi^{\dagger T}$. Its canonical conjugate is obtained by conjugating both sides (which conjugates the numbers): $\Psi'^\dagger_A = \sum_C M^*_{AC} \Psi_C$. Then, line by line,

$$\{\Psi'_A, \Psi'^\dagger_C\} = \sum_{D,E} M_{AD} M^*_{CE} \{\Psi^\dagger_D, \Psi_E\} = \sum_{D,E} M_{AD} B_{ED} M^*_{CE} = (MB^T M^\dagger)_{AC}.$$

The first step expands both operators (numbers come out of an anticommutator); the second uses the canonical rule $\{\Psi^\dagger_D, \Psi_E\} = \{\Psi_E, \Psi^\dagger_D\} = B_{ED}$; the third reads the sum as a matrix product, since $B_{ED} = (B^T)_{DE}$ and $M^*_{CE} = (M^\dagger)_{EC}$. **The rule is kept exactly when $MB^T M^\dagger = B$.** Now B is Hermitian and purely imaginary, so $B^T = (B^\dagger)^* = B^* = -B$. For $M = 1$: $MB^T M^\dagger = B^T = -B$: the rule is broken. For $M = \Gamma$ (real, $\Gamma^\dagger = \Gamma^T = \Gamma$):

$$\Gamma B^T \Gamma = -\Gamma B \Gamma = -(-i)\, \Gamma C \gamma^{(x4)} \Gamma = i\, \Gamma^T C \gamma^{(x4)} \Gamma = i(-C\gamma^{(x4)}) = B,$$

by $B^T = -B$, the definition of B, (X6) and (X8). The matrices of Theorem CC are the multiples $\lambda 1$ and $\lambda\Gamma$, with any complex number $\lambda \neq 0$. A factor λ gives $(\lambda M) B^T (\lambda M)^\dagger = \lambda\lambda^*\, MB^T M^\dagger = |\lambda|^2\, MB^T M^\dagger$, because the conjugate transpose turns λ into λ^*. So $\lambda 1$ gives $-|\lambda|^2 B$, which is never B (that would need $(1 + |\lambda|^2)B = 0$), and $\lambda\Gamma$ gives $|\lambda|^2 B$, which is B exactly when $|\lambda| = 1$, that is when $\lambda = e^{i\alpha}$ is a **phase factor** (a complex number of size 1). **Among the matrices of Theorem CC, only $\Psi' = \Gamma\Psi^{\dagger T}$, up to a phase factor, keeps the canonical rule** (lead check `quantum_charge_conjugation_unitary_type` for $M = \Gamma$ and $M = 1$): the conjugation of the quantised field has the type of $\mathcal{C}_-$.

The conjugated bilinear, line by line. Let $X = \Psi^\dagger K \Psi = \sum_{A,C} \Psi^\dagger_A K_{AC} \Psi_C$ and $X' = \Psi'^\dagger K \Psi'$.

1. Insert the definitions: $X' = \sum_{A,C} \sum_{D,E} M^*_{AD} \Psi_D\, K_{AC}\, M_{CE} \Psi^\dagger_E = \sum_{D,E} \Psi_D\, K'_{DE}\, \Psi^\dagger_E$ with $K' = M^\dagger K M$ (collect the numbers: $\sum_{A,C} M^*_{AD} K_{AC} M_{CE} = (M^\dagger K M)_{DE}$).

2. Exchange the two operators with the canonical rule, $\Psi_D \Psi^\dagger_E = -\Psi^\dagger_E \Psi_D + B_{DE}$: $X' = -\sum_{D,E} \Psi^\dagger_E K'_{DE} \Psi_D + \sum_{D,E} K'_{DE} B_{DE}$.

3. Rename $D \leftrightarrow E$ in the first sum, and write the second as a trace ($\sum_{D,E} K'_{DE} (B^T)_{ED} = \mathrm{tr}(K'B^T)$):

$$X' = \Psi^\dagger(-K'^T)\Psi + c, \qquad c = \mathrm{tr}(K'B^T).$$

So the conjugated bilinear is the bilinear with the matrix $-(M^\dagger K M)^T$ plus the number c (times the identity operator). The matrix $-(M^\dagger K M)^T$ is exactly the rule $\epsilon(M^\dagger K M)^T$ of Section 5.29 with $\epsilon = -1$, the rule of classical anticommuting components. When $-(M^\dagger K M)^T = sK$ with a sign s, the result is $X' = sX + c$.

The constant c. For the scalar and the currents K' is $\pm C$ or $\pm iC\gamma^a$, and $B^T = -B = iC\gamma^{(x4)}$. Then $K'B^T$ is a number times $C\,C\gamma^{(x4)} = \gamma^{(x4)}$ or times $C\gamma^a C\gamma^{(x4)} = -(\gamma^a)^T\gamma^{(x4)} = -\eta_{aa}\gamma^a\gamma^{(x4)}$ (by (C3), (C5) and the symmetry pattern). By Rule 5 of Section 5.3 the trace of a product of different gammas is zero, so $c = 0$, except when $\gamma^a\gamma^{(x4)}$ is not a product of different gammas, that is for $a = x4$: the charge density, whose matrix $-iC\gamma^{(x4)}$ is B itself. For $M = 1$, $K' = B$ and $c = \mathrm{tr}(BB^T) = -\mathrm{tr}(BB) = -\mathrm{tr}\,1 = -16$; for $M = \Gamma$, $K' = \Gamma B\Gamma = -B$ and $c = +16$.

Normal ordering removes c and nothing else. For an operator X built from two field operators, **normal ordering** is the subtraction of its vacuum value, $:X: = X - \langle 0|X|0\rangle$ (for products of b, b^*, d, d^* this is the same as moving every creation operator to the left of every annihilation operator, with a sign for each exchange). From $X' = sX + c$, the vacuum value is $\langle 0|X'|0\rangle = s\langle 0|X|0\rangle + c$, and

$$:X': = X' - \langle 0|X'|0\rangle = sX + c - s\langle 0|X|0\rangle - c = s\ :X: .$$

The constant cancels and the sign stays. **After normal ordering the quantised bilinears change with the signs of the classical anticommuting components.** The argument uses only the canonical rule and the fact that normal ordering subtracts a number, so it does not depend on the momentum of the example or on the choice of the vacuum. For the allowed map $M = \Gamma$ the anticommuting row of Section 5.29 gives $(S, J) \to (-S, -J)$: every current is reversed, so the normal-ordered charge of every quantum changes sign (particles $+1 \to -1$, antiparticles $-1 \to +1$), and the scalar is reversed, so the mass term mS becomes $(-m)S$: **the conjugation of the quantised field exchanges particles and antiparticles and reverses the mass**, as the Revision record states.

What the record says. The detail text of the lead check `bilinears_under_charge_conjugation` states the same result: normal ordering adds no sign, because it subtracts the vacuum value, a number, so that $X' = sX + c$ gives $:X': = s\ :X:$, and the quantised bilinears keep the signs of the classical anticommuting components. The record makes no Fock-space computation; Notebook 05e makes one and confirms the derivation above. For $M = 1$ the normal-ordered result is therefore $(S, J) \to (-S, +J)$, the record's measured anticommuting row. An earlier version of that detail text said instead, in parentheses, that normal ordering supplies one more sign for each bilinear, which would have given $(S, J) \to (S, -J)$ for $\mathcal{C}_+$; the derivation above and Notebook 05e contradicted it, and the record was corrected on 2026-10-08. The contradiction is resolved: the record, this derivation and the Fock-space computation agree.

statement	status	where it is verified
the canonical relations of fermion operators, 2 and 16 modes	COMPUTED exactly (2 modes) and on a random state (16 modes)	Notebook 05e, In [4] and In [6] (its own computation)

statement	status	where it is verified
$\{\Psi_A, \chi_C\} = \delta_{AC}$, $\{\Psi_A, \Psi_C^\dagger\} = B_{AC}$ in the positive realisation	PROVED; COMPUTED on 65536 states	`Revision/theory/reports/wolfram-field-theory.json`, check `Fock_space_good_sector_example`; Notebook 05e, In [8]
the canonical conjugate is not a Hilbert adjoint ($\{X, X^\dagger\} = -1$)	PROVED; COMPUTED	the same report, check `no_positive_inner_product`; Notebook 05e, In [10]
vacuum energy $-8E = -40$, energies $+5$, charges ± 1	COMPUTED (to 10^{-12})	the same report, check `Fock_space_good_sector_example`; Notebook 05e, In [12]
$MB^TM^\dagger = B$ for $M = \Gamma$, $= -B$ for $M = 1$	PROVED; COMPUTED on the Fock space	`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, check `quantum_charge_conjugation_unitary_type`; Notebook 05e, In [14]
$X' = sX + c$ with the anticommuting signs; $c = \mp 16$ for the charge density, 0 otherwise	PROVED; COMPUTED on the Fock space	Notebook 05e, In [16] and In [17]; the signs equal the anticommuting rows of the lead check `bilinears_under_charge_conjugation`
$:X':= s\ :X:;\ \Psi \to \Gamma\Psi^{\dagger T}$ reverses the charge of every quantum	PROVED; COMPUTED	Notebook 05e, In [18] and In [19] (its own computation); the detail text of the lead check `bilinears_under_charge_conjugation` states the first part (normal ordering adds no sign)

5.35 Example: Notebook 05e computes the conjugation of the quantised field

Notebook 05e builds fermion operators from zero, first for two modes as 4×4 matrices and then for sixteen modes on the Fock space of 65536 patterns, where a state is stored as a list of its nonzero amplitudes. It realises the quantised field of the record's good-sector example ($m = 3$, momentum 4 along $x1$, $E = 5$), checks the canonical rule, shows why the canonical conjugate cannot be a Hilbert adjoint, and reproduces the vacuum energy -40, the energies $+5$ and the charges ± 1 of the 16 quanta. Then it applies the two conjugations $M = 1$ and $M = \Gamma$ to the operators, measures the anticommutators, computes all nine conjugated bilinears as operators, finds the sign and the constant of $X' = sX + c$, and checks that normal ordering removes the constant and keeps the sign. It reads `Revision/algebra/gammas.json`, `Revision/theory/field-theory.json`, `Revision/theory/reports/wolfram-field-theory.json` and `Revision/lead_checks/reports/charge-conjugation-and-u1.json`. It draws eight figures and ends with the line ALL 18 CHECKS PASSED (notebook 05e).

5.36 How to run Notebook 05e

Step 1. What this notebook does and what it needs.

Notebook 05e (The quantised field: reordering operators, normal ordering and the conjugated bilinears) is the file `Revision/textbook/notebooks/05e_quantum_bilinears.ipynb` of the repository Dirac_claude. It builds the fermion Fock space of 16 modes (65536 states) of the good-sector example of the Revision record, realises the quantised field with the Hilbert adjoint chi and the canonical conjugate Psi^dagger = chi B, checks the rules {Psi_A, chi_C} = delta_AC and {Psi_A, Psi^dagger_C} = B_AC, shows why Psi^dagger cannot be the Hilbert adjoint, reproduces the recorded vacuum energy, the energies and charges of the 16 quanta and the expectation-value rule, applies the two conjugations Psi to M Psi^(dagger T) with M = 1 and M = Gamma to the operators, and computes exactly how the scalar S and the eight currents J^a change as operators: by the sign of the classical anticommuting components plus a constant, which normal ordering removes. Eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
```

```
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 05e_quantum_bilinears.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a

number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05e_quantum_bilinears.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 05e_quantum_bilinears.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/05e.captions.json`
- `Revision/textbook/figures/05e_1_two_modes.png`
- `Revision/textbook/figures/05e_2_anticommutators.png`
- `Revision/textbook/figures/05e_3_why_krein.png`
- `Revision/textbook/figures/05e_4_quanta.png`
- `Revision/textbook/figures/05e_5_conjugated_rules.png`
- `Revision/textbook/figures/05e_6_vacuum_values.png`
- `Revision/textbook/figures/05e_7_sign_tables.png`
- `Revision/textbook/figures/05e_8_conjugated_charges.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the eight figure files of notebook 05e exist
ALL 18 CHECKS PASSED (notebook 05e)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/05e_quantum_bilinears.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 05e_quantum_bilinears.ipynb
python -m nbconvert --execute --inplace 05e_quantum_bilinears.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra`, `Revision/theory` or `Revision/lead_checks`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

5.37 Notebook 05e: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/05e_quantum_bilinears.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field dirac16complex is **quantised**: its 16 components become *operators*,
and the charge-conjugation maps of the previous notebooks of this chapter act on
operators. Exchanging two operators is not free: it costs a sign and leaves a
number behind (their anticommutator). This notebook computes, on an explicit
computer model of the quantum states, what the two charge-conjugation maps do to
the scalar $S = \Psi^\dagger C\Psi$ and to the eight currents $J^a =
\Psi^\dagger(-iC\gamma^a)\Psi$ of the quantised field. It

1. builds the operators of fermions from zero (two modes, then sixteen modes with
   $2^{16} = 65536$ states);
2. realises the quantised field of the Revision record for one momentum without
   extra-time part (the record's example $m = 3$, momentum 4 along $x1$, energy
   $E = 5$), with the Hilbert adjoint $\chi$ and the canonical conjugate
   $\Psi^\dagger = \chi B$, and checks $\{\Psi_A, \chi_C\} = \delta_{AC}$ and
   $\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$;
3. shows why $\Psi^\dagger$ cannot be the Hilbert adjoint (an indefinite, Krein,
   inner product is needed), as the record proves;
4. reproduces the recorded vacuum energy $-8E = -40$, the energies $+5$ and the
   charges $\pm 1$ of the 16 quanta, and the expectation-value rule;
5. applies the two conjugations $\Psi \to M\Psi^{\dagger T}$ with $M = 1$ (the
   type of $\mathcal{C}_+$) and $M = \Gamma$ (the type of $\mathcal{C}_-$) to the
   operators: of the two, only $M = \Gamma$ keeps the canonical rule;
6. computes every conjugated bilinear as an operator: it is the original bilinear
   times a sign, plus a constant; the sign is the one of the classical
   *anticommuting* components; **normal ordering** removes the constant and
   nothing else.

Eight teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 05e (The quantised field: reordering operators, normal ordering and the
conjugated bilinears) is the file
‘Revision/textbook/notebooks/05e_quantum_bilinears.ipynb‘ of the repository Dirac_claude.
It builds the fermion Fock space of 16 modes (65536 states) of the good-sector example of
the Revision record, realises the quantised field with the Hilbert adjoint chi and the
canonical conjugate Psi^dagger = chi B, checks the rules {Psi_A, chi_C} = delta_AC and
{Psi_A, Psi^dagger_C} = B_AC, shows why Psi^dagger cannot be the Hilbert adjoint,
reproduces the recorded vacuum energy, the energies and charges of the 16 quanta and the
expectation-value rule, applies the two conjugations Psi to M Psi^(dagger T) with M = 1
and M = Gamma to the operators, and computes exactly how the scalar S and the eight
currents J^a change as operators: by the sign of the classical anticommuting components
plus a constant, which normal ordering removes. Eight teaching plots. It needs a computer
with Windows 11, macOS or Linux, an internet connection for the installation, the program
Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages
run Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv

Then close the terminal and open a NEW one (a terminal opened before the installation
does not know the new programs), and check the version of Python; it must print 3.12 or a
```

higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
```

```
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 05e_quantum_bilinears.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut

Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 05e_quantum_bilinears.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 05e_quantum_bilinears.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/05e.captions.json‘
- ‘Revision/textbook/figures/05e_1_two_modes.png‘
- ‘Revision/textbook/figures/05e_2_anticommutators.png‘
- ‘Revision/textbook/figures/05e_3_why_krein.png‘
- ‘Revision/textbook/figures/05e_4_quanta.png‘
- ‘Revision/textbook/figures/05e_5_conjugated_rules.png‘
- ‘Revision/textbook/figures/05e_6_vacuum_values.png‘
- ‘Revision/textbook/figures/05e_7_sign_tables.png‘
- ‘Revision/textbook/figures/05e_8_conjugated_charges.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the eight figure files of notebook 05e exist
ALL 18 CHECKS PASSED (notebook 05e)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started

```
  without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
  again. If the error remains, another Python installation has registered its own kernel
  called python3; with the environment active, run the following command and start
  JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from
  a copy outside the repository. Open the file
  ‘Revision/textbook/notebooks/05e_quantum_bilinears.ipynb‘ inside the folder
  Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 05e_quantum_bilinears.ipynb
      python -m nbconvert --execute --inplace 05e_quantum_bilinears.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘, ‘Revision/theory‘
  or ‘Revision/lead_checks‘: the notebook was opened outside the repository, or the
  repository is incomplete; clone the repository again and open the notebook from its
  folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 05e (the complete instructions)
#
```

```
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 05e (The quantised field: reordering operators, normal ordering and the
# conjugated bilinears) is the file
# Revision/textbook/notebooks/05e_quantum_bilinears.ipynb of the repository Dirac_claude.
# It builds the fermion Fock space of 16 modes (65536 states) of the good-sector example
# of the Revision record, realises the quantised field with the Hilbert adjoint chi and
# the canonical conjugate Psi^dagger = chi B, checks the rules {Psi_A, chi_C} = delta_AC
# and {Psi_A, Psi^dagger_C} = B_AC, shows why Psi^dagger cannot be the Hilbert adjoint,
# reproduces the recorded vacuum energy, the energies and charges of the 16 quanta and
# the expectation-value rule, applies the two conjugations Psi to M Psi^(dagger T) with M
# = 1 and M = Gamma to the operators, and computes exactly how the scalar S and the eight
# currents J^a change as operators: by the sign of the classical anticommuting components
# plus a constant, which normal ordering removes. Eight teaching plots. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
```

```
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
```

```
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 05e_quantum_bilinears.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 05e_quantum_bilinears.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
```

```
# 05e_quantum_bilinears.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/05e.captions.json
# - Revision/textbook/figures/05e_1_two_modes.png
# - Revision/textbook/figures/05e_2_anticommutators.png
# - Revision/textbook/figures/05e_3_why_krein.png
# - Revision/textbook/figures/05e_4_quanta.png
# - Revision/textbook/figures/05e_5_conjugated_rules.png
# - Revision/textbook/figures/05e_6_vacuum_values.png
# - Revision/textbook/figures/05e_7_sign_tables.png
# - Revision/textbook/figures/05e_8_conjugated_charges.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the eight figure files of notebook 05e exist
#     ALL 18 CHECKS PASSED (notebook 05e)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/05e_quantum_bilinears.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
```

```
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 05e_quantum_bilinears.ipynb
#     python -m nbconvert --execute --inplace 05e_quantum_bilinears.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra, Revision/theory or
#   Revision/lead_checks: the notebook was opened outside the repository, or the
#   repository is incomplete; clone the repository again and open the notebook from its
#   folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "05e"  # this notebook: chapter 05, example e


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
```

```
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 05e complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Operator**: a rule that turns a state into another state, here a big matrix
  that acts on columns of numbers. Operators are multiplied by doing one after the
  other; the order matters.
- **Anticommutator** of two operators: $\{X, Y\} = XY + YX$.
- **Mode, occupation, pattern**: a fermion *mode* is a place that holds zero or one
  particle. With 16 modes a basis state is a *pattern* of 16 occupations (0 or 1);
  the computer stores it as a whole number $n$ whose binary digit $p$ is the
```

```
  occupation of mode $p$ ($p = 0, \dots, 15$). There are $2^{16} = 65536$ patterns.
- **Fock space**: all combinations of the 65536 basis states. A **state** is stored
  as a dictionary `{pattern: amplitude}` that lists only the nonzero amplitudes.
- **Vacuum** $|0\rangle$: the pattern with no mode occupied (the whole number 0).
- **Annihilation and creation operators** $f_p$, $f_p^*$: $f_p$ empties mode $p$
  (and gives zero if it is empty), $f_p^*$ fills it (zero if it is full), each with
  a sign $(-1)^{(\text{number of occupied modes below } p)}$. They obey the
  **canonical anticommutation relations** $\{f_p, f_q^*\} = \delta_{pq}$,
  $\{f_p, f_q\} = 0$, $\{f_p^*, f_q^*\} = 0$ ($\delta_{pq}$ is 1 for $p = q$ and 0
  otherwise). In particular $f_pf_p = 0$: no mode holds two fermions.
- **Hilbert adjoint** $X^*$ of an operator: the conjugate transpose of its matrix
  (with the ordinary, positive inner product of the Fock space).
- **Particle and antiparticle operators**: $b_s = f_{s-1}$ for $s = 1, \dots, 8$
  (modes 0 to 7) and $d_s = f_{s+7}$ (modes 8 to 15).
- **Field operators**: $\Psi_A = \sum_s \big((u_s)_A b_s + (v_s)_A d_s^*\big)$ for
  $A = 1, \dots, 16$, with $\chi_A$ its Hilbert adjoint and the **canonical
  conjugate** $\Psi^\dagger_A = \sum_C \chi_C B_{CA}$.
- **Momentum**: a field of one momentum $k$ along $x1$ has the form
  $\psi\,e^{ik\,x1}$, a column $\psi$ times the number $e^{ik\,x1} = \cos(k\,x1) +
  i\sin(k\,x1)$; $k$ is a real number. The **good sector**: momenta without a part
  along the extra times $x5$, $x6$, $x7$ (the field does not depend on them).
- **Energy**: a column that depends on the time as $e^{-iE\,x4}$ ($E$ real) has
  the energy $E$.
- **Mode Hamiltonian** $h$: the $16 \times 16$ matrix with $E\,u = h\,u$ for the
  field $u\,e^{i(k\,x1 - E\,x4)}$ of one momentum; its eigenvalues are the energies
  of the fields of that momentum (section 8 derives it). The **energy density** is
  the energy per unit volume.
- **Orthonormal** columns: each has length 1 ($u^\dagger u = 1$) and every two are
  perpendicular ($u^\dagger w = 0$).
- **Delta function** $\delta^7(x - y)$: an idealised object that is zero whenever
  the point $x$ differs from $y$ and whose integral over the seven coordinates
  other than the time is 1.
- **Krein space**: a space with an *indefinite* inner product (some vectors have a
  negative norm).
- **Bilinear operator**: $\Psi^\dagger K\Psi = \sum_{A,C}\Psi^\dagger_A K_{AC}
  \Psi_C$ for a fixed $16 \times 16$ matrix $K$.
- **Vacuum value** $\langle 0|X|0\rangle$ of an operator $X$ and **normal ordering**
  $:\!X\!: = X - \langle 0|X|0\rangle$ (for an operator built from two field
  operators; for products of $b$, $b^*$, $d$, $d^*$ this is the same as moving every
  creation operator to the left of every annihilation operator, with a sign $-1$
  for each exchange).
- **Expectation value** $\langle q|X|q\rangle$ in a normalised state $q$.
```

Text cell:

```
## 4. The physical and mathematical situation

**The canonical rule.** The Revision record quantises dirac16complex with the
canonical anticommutator $\{\Psi_A(x), \Psi^\dagger_C(y)\} = B_{AC}\,
\delta^7(x - y)/\cos z$ on a slice $x4 = \text{const}$, $B = -iC\gamma^{(x4)}$. For
the operators of ONE momentum (the record's exact Fock-space example) the delta
function and the factor $1/\cos z$ are absent and the rule reads
$\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$.
```

Why $\Psi^\dagger$ is not the Hilbert adjoint. Every diagonal entry of B is zero. If $\Psi^\dagger_A$ were the Hilbert adjoint Ψ_A^*, then for every state ϕ: $\langle\phi|\{\Psi_A, \Psi_A^*\}|\phi\rangle = |\Psi_A^*\phi|^2 + |\Psi_A\phi|^2$ (a sum of squared lengths), and the rule would make it $B_{AA} = 0$; a sum of two squared lengths is zero only when both are zero, so $\Psi_A\phi = 0$ for every ϕ, and Ψ_A would be the zero operator. But row A of B has a nonzero entry B_{AC}, and the rule demands $\{\Psi_A, \Psi^\dagger_C\} = B_{AC} \neq 0$, while the anticommutator of the zero operator with anything is zero: a contradiction. The record makes the same argument with a vector u of the eigenspace $B u = -u$: $X = \sum_A u_A^*\Psi_A$ has $\{X, X^\dagger\} = u^\dagger Bu = -1 < 0$. The record's way out: $\Psi^\dagger = \chi B$, where χ is the Hilbert adjoint, on a positive Fock space. Line by line: $\{\Psi_A, \chi_C\} = \sum_s (u_s)_A(u_s)^*_C + \sum_s (v_s)_A(v_s)^*_C = (WW^\dagger)_{AC} = \delta_{AC}$ (the columns u_s, v_s of W are orthonormal and complete), and then $\{\Psi_A, \Psi^\dagger_C\} = \sum_D\{\Psi_A, \chi_D\}B_{DC} = B_{AC}$.

The conjugation of the quantised field. For a real matrix M put $\Psi'_A = \sum_C M_{AC}\Psi^\dagger_C$ (the operator form of $M\Psi^*$, written $M\Psi^{\dagger T}$) and, taking the conjugate of both sides (which reverses the order and conjugates numbers), $\Psi'^\dagger_A = \sum_C M_{AC}^*\Psi_C$. Then $\{\Psi'_A, \Psi'^\dagger_C\} = \sum_{D,E}M_{AD}M^*_{CE}\{\Psi^\dagger_D, \Psi_E\} = \sum_{D,E}M_{AD}B_{ED}M^*_{CE} = (MB^TM^\dagger)_{AC}$. The rule is kept exactly when $MB^TM^\dagger = B$.

The conjugated bilinear, line by line. Let $X = \Psi^\dagger K\Psi$ and $X' = \Psi'^\dagger K\Psi'$.

1. Insert the definitions: $X' = \sum_{A,C}\sum_{D,E}M^*_{AD}\Psi_D K_{AC}M_{CE} \Psi^\dagger_E = \sum_{D,E}\Psi_D\,K'_{DE}\,\Psi^\dagger_E$ with $K' = M^\dagger KM$.
2. Exchange the two operators with the canonical rule, $\Psi_D\Psi^\dagger_E = -\Psi^\dagger_E\Psi_D + B_{DE}$: $X' = -\sum_{D,E}\Psi^\dagger_E K'_{DE}\Psi_D + \sum_{D,E}K'_{DE}B_{DE}$.
3. Rename $D \leftrightarrow E$ in the first sum and write the second as a trace: $$X' = \Psi^\dagger\big(-K'^T\big)\Psi + c,\qquad c = \mathrm{tr}(K'B^T) .$$

So X' is the bilinear with the matrix $-(M^\dagger KM)^T$, plus the number c (times the identity operator). The matrix $-(M^\dagger KM)^T$ is exactly the rule for classical **anticommuting** components (the exchange of two components costs the sign $\epsilon = -1$); for commuting components the rule is $+(M^\dagger KM)^T$. When $-(M^\dagger KM)^T = sK$ with a sign s, the result is $X' = sX + c$.

Normal ordering. Subtracting vacuum values: $:\!X'\!: = X' - \langle 0|X'|0\rangle = sX + c - s\langle 0|X|0\rangle - c = s\,:\!X\!:$. The constant c cancels and the sign s stays: after normal ordering the quantised bilinears change with the signs of the classical anticommuting components. The argument uses only that normal ordering subtracts a number; it holds for every choice of vacuum.

The notebook checks every one of these statements on the Fock space.

Text cell:

5. The gammas, C, Gamma, B and the Revision records

The next cell reads the gammas (exact whole numbers) from
‘Revision/algebra/gammas.json‘ and builds $C = \gamma^{(x0)}\gamma^{(x1)}
\gamma^{(x2)}\gamma^{(x3)}$, $\Gamma = \gamma^{(x0)}\gamma^{(x1)}\cdots
\gamma^{(x7)}$ and $B = -iC\gamma^{(x4)}$. It reads the two reports whose checks
this notebook repeats and defines the helpers ‘recorded(key, name)‘ (true when
the report ‘key‘, "lead" or "wolfram", holds the check ‘name‘ with the verdict
pass), ‘detail(key, name)‘ (its detail text), ‘record_of(key, name)‘ (the text
printed after "reproduces") and ‘check_reproduces(condition, name, record)‘, the
helper ‘check‘ for a check that reproduces a Revision record: it lets ‘check‘ print
the PASS line and the line "reproduces ..." into a text buffer
(‘contextlib.redirect_stdout‘) and sends both lines with one ‘sys.stdout.write‘,
because Jupyter delivers printed text in pieces and one piece keeps the two lines
together for the tools that read the notebook. The first check confirms the
facts about B used in section 4.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1
C = gamma["x0"] @ gamma["x1"] @ gamma["x2"] @ gamma["x3"]  # the charge matrix
Gamma = I16
for x in ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]:
    Gamma = Gamma @ gamma[x]  # the chirality: the product of all eight gammas
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4), entries 0, +i, -i

REPORT_FILES = {"lead": "Revision/lead_checks/reports/charge-conjugation-and-u1.json",
                "wolfram": "Revision/theory/reports/wolfram-field-theory.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])


def recorded(key, name):
    """True when the report key records the check name with the verdict pass."""
    return VERDICTS[(key, name)][0] == "pass"


def detail(key, name):
    """The detail text of the recorded check."""
    return VERDICTS[(key, name)][1]
```

```
def record_of(key, name):
    """The text printed after "reproduces": the record file and the check name."""
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    collected = io.StringIO()
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together


say(f"{len(VERDICTS)} recorded checks were read from the two reports.")
check(np.array_equal(B.conj().T, B) and np.array_equal(B @ B, I16)
      and not np.any(np.diag(B)) and np.array_equal(B.T, -B),
      "B is Hermitian, B B = 1, B^T = -B and every diagonal entry of B is 0")
```

Out [2]:

```
96 recorded checks were read from the two reports.
PASS B is Hermitian, B B = 1, B^T = -B and every diagonal entry of B is 0
```

Text cell:

```
The next cell reads the formula ‘quantisation‘ of the Revision record
‘Revision/theory/field-theory.json‘ and prints the two statements this notebook
realises: the canonical rule and its positive representation with $\chi =
\Psi^\dagger B$ (equivalently $\Psi^\dagger = \chi B$, because $BB = 1$).
```

In [3]:

```
theory = json.loads(repository_file("Revision/theory/field-theory.json")
                    .read_text(encoding="utf-8"))
formula = next(f for f in theory["formulas"] if f["key"] == "quantisation")["wl"]
RULE = "{Psi_A(x), Psi^dagger_C(y)}_(x4 = y4) = B_AC delta^7(x - y)/Cos[z]"
POSITIVE = "positive representation chi = Psi^dagger B, {Psi_A, chi_C} = delta_AC"
say("the record states: " + RULE)
say("and: " + POSITIVE)
check(RULE in formula and POSITIVE in formula,
      "the formula quantisation of Revision/theory/field-theory.json states both rules")
```

Out [3]:

```
the record states: {Psi_A(x), Psi^dagger_C(y)}_(x4 = y4) = B_AC delta^7(x - y)/Cos[z]
and: positive representation chi = Psi^dagger B, {Psi_A, chi_C} = delta_AC
PASS the formula quantisation of Revision/theory/field-theory.json states both rules
```

Text cell:

```
## 6. Fermion operators from zero: two modes

With two modes there are four patterns: $n = 0$ (both empty, written $|00\rangle$),
$n = 1$ (mode 0 full, $|10\rangle$), $n = 2$ (mode 1 full, $|01\rangle$) and
$n = 3$ (both full, $|11\rangle$). The next cell writes $f_0$, $f_0^*$, $f_1$,
$f_1^*$ as $4 \times 4$ matrices: column $n$ holds the image of the pattern $n$.
The sign rule: acting on mode $p$ costs $-1$ for every occupied mode below $p$. So
$f_1|11\rangle = -|10\rangle$ (mode 0 below is full) while $f_0|11\rangle =
|01\rangle$. The cell checks the canonical relations and $f_pf_p = 0$, and that
$f_p^*$ is the transpose of $f_p$ (the matrices are real, so this is the Hilbert
adjoint). Without the sign, $f_0$ and $f_1$ would *commute*; with it they
anticommute.
```

In [4]:

```
def two_mode_matrix(p, filling):
    """The 4 x 4 matrix of f_p (filling=False) or f_p^* (filling=True), 2 modes."""
    matrix = np.zeros((4, 4), dtype=np.int64)
    for n in range(4):  # the pattern n: binary digit q = occupation of mode q
        full = (n >> p) & 1  # the occupation of mode p in the pattern n
        if full != filling:  # f_p needs a full mode, f_p^* an empty one
            below = bin(n & ((1 << p) - 1)).count("1")  # occupied modes below p
            matrix[n ^ (1 << p), n] = (-1) ** below  # ^ flips binary digit p
    return matrix


f2 = {p: two_mode_matrix(p, False) for p in (0, 1)}  # f_0, f_1
f2_star = {p: two_mode_matrix(p, True) for p in (0, 1)}  # f_0^*, f_1^*
Z4 = np.zeros((4, 4), dtype=np.int64)
relations_ok = all(
    np.array_equal(f2[p] @ f2_star[q] + f2_star[q] @ f2[p],
                   np.eye(4, dtype=np.int64) if p == q else Z4)
    and np.array_equal(f2[p] @ f2[q] + f2[q] @ f2[p], Z4)
    and np.array_equal(f2_star[p], f2[p].T)
    for p in (0, 1) for q in (0, 1))
# column 3 of f_1 is the image of the pattern |11>; its entry in row 1 (|10>) is -1
say(f"f_1 applied to |11>: amplitudes on |00>, |10>, |01>, |11> = "
    f"{f2[1][:, 3].tolist()}")
check(relations_ok and all(not np.any(f2[p] @ f2[p]) for p in (0, 1)),
      "two modes: {f_p, f_q^*} = delta_pq, {f_p, f_q} = 0, f_p f_p = 0, "
      "f_p^* = f_p^T")
```

Out [4]:

```
f_1 applied to |11>: amplitudes on |00>, |10>, |01>, |11> = [0, -1, 0, 0]
PASS two modes: {f_p, f_q^*} = delta_pq, {f_p, f_q} = 0, f_p f_p = 0, f_p^* = f_p^T
```

Text cell:

The next cell draws the four matrices.

In [5]:

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
PATTERNS = ["|00>", "|10>", "|01>", "|11>"]  # the patterns n = 0, 1, 2, 3
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.5))
for k, (ax, matrix, title) in enumerate(zip(
        axes, [f2[0], f2_star[0], f2[1], f2_star[1]],
        ["$f_0$ (empties mode 0)", "$f_0^*$ (fills mode 0)",
         "$f_1$ (empties mode 1)", "$f_1^*$ (fills mode 1)"])):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    for r in range(4):
        for c in range(4):
            if matrix[r, c]:
                ax.text(c, r, f"{matrix[r, c]:+d}", ha="center", va="center",
                        color="white", fontweight="bold")
    ax.set_xticks(range(4), PATTERNS)
    # the row labels only on the first picture (the rows are the same in all four)
    ax.set_yticks(range(4), PATTERNS if k == 0 else [""] * 4)
    ax.set_xlabel("from the pattern")
    if k == 0:
        ax.set_ylabel("to the pattern")
    ax.set_title(title)
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "two_modes",
            "The annihilation and creation operators of two fermion modes as 4 by 4 "
            "matrices: column the pattern acted on, row the resulting pattern "
            "(00: both modes empty, 10: mode 0 full, 01: mode 1 full, 11: both "
            "full); red $+1$, blue $-1$, grey 0. Each operator moves one pattern "
            "to another; $f_1$ acting on the full pattern 11 gives $-1$ times the "
            "pattern 10, because mode 0 below it is occupied. This sign makes "
            "operators of different modes anticommute.")
```

Out [5]:

```
Figure 05e.1 saved as Revision/textbook/figures/05e_1_two_modes.png
```

Text cell:

```
## 7. Sixteen modes: the Fock space of 65536 states

The next cell defines the operators for 16 modes. A state is a dictionary
‘{pattern: amplitude}‘; ‘annihilate(p, state)‘ is $f_p$ and ‘create(p, state)‘ is
$f_p^*$, with the sign rule of section 6; ‘combine(terms)‘ adds up states with
coefficients; ‘inner(left, right)‘ is the ordinary (positive) inner product
$\langle\text{left}|\text{right}\rangle = \sum_n \text{left}_n^*\,\text{right}_n$;
‘largest(state)‘ is the largest size of an amplitude. It then checks the canonical
```

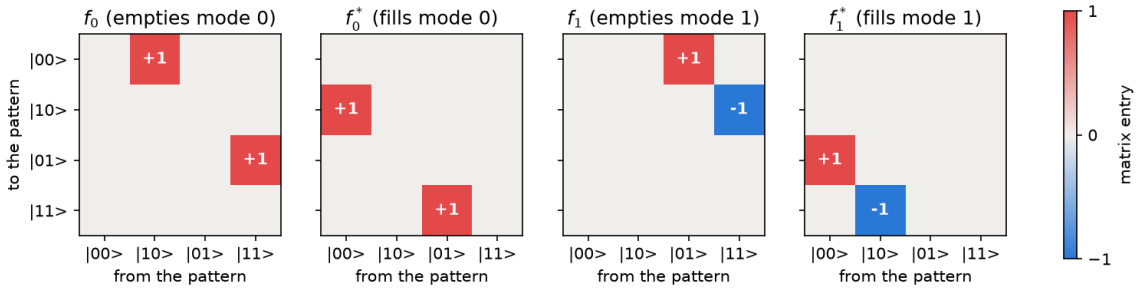

Figure 174: The annihilation and creation operators of two fermion modes as 4 by 4 matrices: column the pattern acted on, row the resulting pattern (00: both modes empty, 10: mode 0 full, 01: mode 1 full, 11: both full); red +1, blue −1, grey 0. Each operator moves one pattern to another; f_1 acting on the full pattern 11 gives −1 times the pattern 10, because mode 0 below it is occupied. This sign makes operators of different modes anticommute. (Notebook 05e, figure 1.)

```
relations $\{f_p, f_q^*\} = \delta_{pq}$ and $\{f_p, f_q\} = 0$ for all
$16 \times 16$ pairs on a random state: a dictionary with six random patterns and
random complex amplitudes, from the random-number generator with the fixed seed
12345 (so every run uses the same numbers).
```

In [6]:

```
def sign_below(n, p):
    """(-1) to the power of the number of occupied modes below mode p in pattern n."""
    return -1 if bin(n & ((1 << p) - 1)).count("1") % 2 else 1


def annihilate(p, state):
    """f_p applied to state: empties mode p (patterns with mode p empty give 0)."""
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:  # mode p is full in the pattern n
            new = n ^ (1 << p)  # the same pattern with mode p emptied
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):
    """f_p^* applied to state: fills mode p (patterns with mode p full give 0)."""
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:  # mode p is empty in the pattern n
            new = n | (1 << p)  # the same pattern with mode p filled
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def combine(terms):
    """The state sum of coefficient * state over the pairs (coefficient, state)."""
    result = {}
```

```
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result


def inner(left, right):
    """The ordinary inner product <left|right> (positive: <s|s> >= 0)."""
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    """The largest size of an amplitude of state (0 for the zero state)."""
    return max((abs(a) for a in state.values()), default=0.0)


rng = np.random.default_rng(12345)  # random numbers with a fixed seed
RANDOM_STATE = {int(n): complex(rng.normal(), rng.normal())
                for n in rng.integers(0, 2 ** 16, size=6)}  # six random patterns
worst = 0.0  # the largest violation of a canonical relation
for p in range(16):
    for q in range(16):
        mixed = combine([(1, annihilate(p, create(q, RANDOM_STATE))),
                         (1, create(q, annihilate(p, RANDOM_STATE)))])
        expected = RANDOM_STATE if p == q else {}  # delta_pq times the state
        worst = max(worst, largest(combine([(1, mixed), (-1, expected)])))
        same = combine([(1, annihilate(p, annihilate(q, RANDOM_STATE))),
                        (1, annihilate(q, annihilate(p, RANDOM_STATE)))])
        worst = max(worst, largest(same))
say(f"patterns in the random state: {len(RANDOM_STATE)}; pairs tested: 2 x 256")
check(worst < 1e-12,
      "sixteen modes: {f_p, f_q^*} = delta_pq and {f_p, f_q} = 0 on a random state")
```

Out [6]:

```
patterns in the random state: 6; pairs tested: 2 x 256
PASS sixteen modes: {f_p, f_q^*} = delta_pq and {f_p, f_q} = 0 on a random state
```

Text cell:

```
## 8. The quantised field of one good-sector momentum

The Revision record's exact Fock-space example has the mass $m = 3$ and the
momentum 4 along $x1$, so $E = \sqrt{3^2 + 4^2} = 5$; it has no extra-time
momentum (the *good sector*). It treats the metric near one point as flat
(constant gammas, no spin-connection term). Inserting $\Psi = u\,e^{i(k\,x1 -
E\,x4)}$ into $\gamma^{(x4)}\partial_{x4}\Psi + \gamma^{(x1)}\partial_{x1}\Psi =
m\Psi$ gives $-iE\gamma^{(x4)}u + ik\gamma^{(x1)}u = mu$; multiplying by
$-i\gamma^{(x4)}$ (with $\gamma^{(x4)}\gamma^{(x4)} = -1$) gives $Eu = hu$ with
the mode Hamiltonian $h = -im\gamma^{(x4)} - k\gamma^{(x4)}\gamma^{(x1)}$, here
with $k = 4$. Both matrices $-i\gamma^{(x4)}$ and $\gamma^{(x4)}\gamma^{(x1)}$ are
Hermitian ($\gamma^{(x4)}$ is real antisymmetric; $\gamma^{(x4)}\gamma^{(x1)}$ is
```

```
real symmetric), they anticommute,
and $(-i\gamma^{(x4)})^2 = (\gamma^{(x4)}\gamma^{(x1)})^2 = 1$, so $hh = (9 + 16)1
= 25\cdot 1$. The next cell builds $h$ and checks this, and finds orthonormal
eigenvectors $u_1, \dots, u_8$ ($hu_s = 5u_s$) and $v_1, \dots, v_8$
($hv_s = -5v_s$) as orthonormal columns of the projectors $\frac12(1 \pm h/E)$ (the
Gram-Schmidt procedure: subtract from each column its parts along the columns
already kept, divide by its length, keep it if the length is not zero). The 16
columns form the matrix $W$, with $W^\dagger W = WW^\dagger = 1$.
```

In [7]:

```
MASS, MOMENTUM = 3, 4  # the record's example: m = 3, momentum 4 along x1
E = float(np.sqrt(MASS ** 2 + MOMENTUM ** 2))  # the energy, 5
h = -1j * MASS * gamma["x4"] - MOMENTUM * (gamma["x4"] @ gamma["x1"])


def orthonormal_columns(P):
    """Orthonormal columns that span the range of P (Gram-Schmidt, done twice)."""
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):  # a second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e  # remove the part along e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:  # a new direction: keep it with length 1
            basis.append(v / length)
    return np.array(basis).T


U_plus = orthonormal_columns((np.eye(16) + h / E) / 2)  # u_1..u_8: energy +5
V_minus = orthonormal_columns((np.eye(16) - h / E) / 2)  # v_1..v_8: energy -5
W = np.hstack([U_plus, V_minus])  # 16 x 16: the columns u_1..u_8, v_1..v_8
report("E for m = 3 and momentum 4 along x1", f"{E:.12f}")
check(np.max(np.abs(h - h.conj().T)) < 1e-15
      and np.max(np.abs(h @ h - 25 * np.eye(16))) < 1e-13
      and U_plus.shape == (16, 8) and V_minus.shape == (16, 8)
      and np.max(np.abs(h @ U_plus - E * U_plus)) < 1e-13
      and np.max(np.abs(h @ V_minus + E * V_minus)) < 1e-13
      and np.max(np.abs(W.conj().T @ W - np.eye(16))) < 1e-13
      and np.max(np.abs(W @ W.conj().T - np.eye(16))) < 1e-13,
      "h is Hermitian, h h = 25, with 8 + 8 orthonormal complete eigenvectors")
```

Out [7]:

```
RESULT E for m = 3 and momentum 4 along x1 = 5.000000000000
PASS h is Hermitian, h h = 25, with 8 + 8 orthonormal complete eigenvectors
```

Text cell:

```
**Operators as lists of 32 numbers.** Every operator that is a sum of single
$f$'s can be stored as 32 coefficients: of the 32 *basic operators* $O_i$,
```

```
$i = 0, \dots, 31$, where $O_p = F_p$ and $O_{16+p} = F_p^*$ for $p = 0, \dots, 15$,
with $F_p = b$ (that is $f_p$) for $p < 8$ and $F_p = d^*$ (that is $f_p^*$) for
$p \geq 8$, and $F_p^*$ the Hilbert adjoint of $F_p$. Then
$\Psi_A = \sum_p W_{Ap}F_p$ is the row `PSI[A]` $= (W_{A,0}, \dots, W_{A,15}, 0,
\dots, 0)$, its Hilbert adjoint $\chi_A = \sum_p W^*_{Ap}F_p^*$ is
`CHI[A]` $= (0, \dots, 0, W^*_{A,0}, \dots, W^*_{A,15})$, and $\Psi^\dagger_A =
\sum_C\chi_C B_{CA}$ is row $A$ of $B^T$ times `CHI`. The next cell defines these
rows, `apply(row, state)` (the operator of a row applied to a state) and
`anticommutator(row1, row2, state)`. It then measures, on the random state, the
numbers $\{\Psi_A, \chi_C\}$, $\{\Psi_A, \Psi^\dagger_C\}$, $\{\Psi_A, \Psi_C\}$
and $\{\Psi^\dagger_A, \Psi^\dagger_C\}$ for all $16 \times 16$ pairs: each
anticommutator must be a number times the state; the number is read off as
$\langle\phi|\{X, Y\}\phi\rangle/\langle\phi|\phi\rangle$ and the rest must vanish.
```

In [8]:

```
def basic(i, state):
    """The basic operator O_i applied to state: O_p = F_p, O_(16+p) = F_p^*."""
    p = i % 16  # the mode
    empties = (i < 16) == (p < 8)  # F_p for p < 8 and F_p^* for p >= 8 empty mode p
    return annihilate(p, state) if empties else create(p, state)


ZEROS = np.zeros((16, 16))
PSI = np.hstack([W, ZEROS])  # row A: Psi_A = sum_p W_Ap F_p
CHI = np.hstack([ZEROS, W.conj()])  # row A: chi_A = sum_p conj(W_Ap) F_p^*
PSI_DAG = B.T @ CHI  # row A: Psi^dagger_A = sum_C chi_C B_CA


def apply(row, state):
    """The operator sum_i row_i O_i applied to state."""
    return combine([(row[i], basic(i, state)) for i in range(32)
                    if abs(row[i]) > 1e-15])


def anticommutator(row1, row2, state):
    """{X, Y} applied to state, for the operators X and Y of the two rows."""
    return combine([(1, apply(row1, apply(row2, state))),
                    (1, apply(row2, apply(row1, state)))])


def measured_rule(rows1, rows2, state):
    """The 16 x 16 numbers {X_A, Y_C} = <state|{X_A, Y_C} state>/<state|state>, and
    the largest amplitude left over when that number times the state is removed."""
    norm = inner(state, state).real
    numbers = np.zeros((16, 16), dtype=complex)
    rest = 0.0
    for A in range(16):
        for C_ in range(16):
            result = anticommutator(rows1[A], rows2[C_], state)
            numbers[A, C_] = inner(state, result) / norm
            rest = max(rest, largest(combine([(1, result),
                                              (-numbers[A, C_], state)])))
```

```
    return numbers, rest


with_chi, rest_chi = measured_rule(PSI, CHI, RANDOM_STATE)
with_dagger, rest_dagger = measured_rule(PSI, PSI_DAG, RANDOM_STATE)
psi_psi, rest_pp = measured_rule(PSI, PSI, RANDOM_STATE)
dag_dag, rest_dd = measured_rule(PSI_DAG, PSI_DAG, RANDOM_STATE)
all_rest = max(rest_chi, rest_dagger, rest_pp, rest_dd)
say("every anticommutator is a number times the state (rest below 1e-12): "
    f"{all_rest < 1e-12}")
check_reproduces(np.max(np.abs(with_chi - np.eye(16))) < 1e-12
                 and np.max(np.abs(with_dagger - B)) < 1e-12
                 and np.max(np.abs(psi_psi)) < 1e-12
                 and np.max(np.abs(dag_dag)) < 1e-12 and all_rest < 1e-12
                 and recorded("wolfram", "Fock_space_good_sector_example"),
                 "{Psi_A, chi_C} = delta_AC and {Psi_A, Psi^dagger_C} = B_AC on the "
                 "positive Fock space; {Psi, Psi} = 0 = {Psi^dagger, Psi^dagger}",
                 record=record_of("wolfram", "Fock_space_good_sector_example"))
```

Out [8]:

```
every anticommutator is a number times the state (rest below 1e-12): True
PASS {Psi_A, chi_C} = delta_AC and {Psi_A, Psi^dagger_C} = B_AC on the positive Fock
    space; {Psi, Psi} = 0 = {Psi^dagger, Psi^dagger}
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Fock_space_good_sector_example
```

Text cell:

```
The next cell draws the measured numbers $\{\Psi_A, \chi_C\}$ (real part; the
imaginary part is zero) and $\{\Psi_A, \Psi^\dagger_C\}$ (imaginary part; the real
part is zero) next to the imaginary part of $B$.
```

In [9]:

```
def heat_map(ax, matrix, title, row_label=True):
    """Draw a 16 x 16 matrix with entries from -1 to +1: blue -1, grey 0, red +1.
    Rows and columns are numbered from 1; black lines separate the two halves."""
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.set_yticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)
    return image
```

```
check(np.max(np.abs(with_chi.imag)) < 1e-12 and np.max(np.abs(with_dagger.real))
      < 1e-12, "the measured numbers are real for chi and imaginary for Psi^dagger")
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], with_chi.real, r"measured $\{\Psi_A, \chi_C\}$")
heat_map(axes[1], with_dagger.imag, r"measured $\{\Psi_A, \Psi^\dagger_C\}$ / $i$",
         row_label=False)
image = heat_map(axes[2], B.imag, r"the matrix $B$ / $i$", row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "anticommutators",
            "The anticommutators of the quantised field measured on the Fock space "
            "of 65536 states, as 16 by 16 heat maps (row $A$, column $C$, from 1 to "
            "16; blue $-1$, grey 0, red $+1$): left $\\{\\Psi_A, \\chi_C\\}$ with the "
            "Hilbert adjoint $\\chi$, which is the identity matrix; middle "
            "$\\{\\Psi_A, \\Psi^\\dagger_C\\}$ with the canonical conjugate "
            "$\\Psi^\\dagger = \\chi B$, divided by $i$; right the matrix $B$ divided "
            "by $i$. The middle and right pictures are equal: the realisation obeys "
            "the canonical rule, and its diagonal is empty.")
```

Out [9]:

```
PASS the measured numbers are real for chi and imaginary for Psi^dagger
```

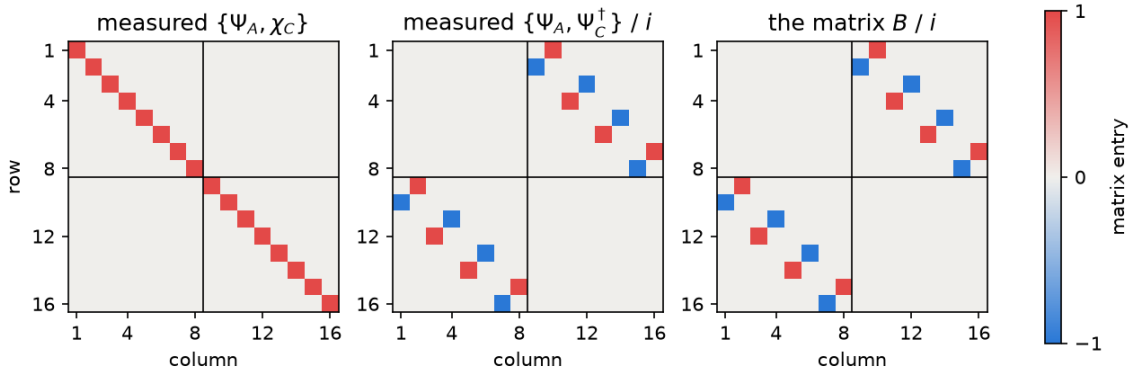

Figure 175: The anticommutators of the quantised field measured on the Fock space of 65536 states, as 16 by 16 heat maps (row A, column C, from 1 to 16; blue -1, grey 0, red $+1$): left $\{\Psi_A, \chi_C\}$ with the Hilbert adjoint χ, which is the identity matrix; middle $\{\Psi_A, \Psi^\dagger_C\}$ with the canonical conjugate $\Psi^\dagger = \chi B$, divided by i; right the matrix B divided by i. The middle and right pictures are equal: the realisation obeys the canonical rule, and its diagonal is empty. (Notebook 05e, figure 2.)

Out [9]:

```
Figure 05e.2 saved as Revision/textbook/figures/05e_2_anticommutators.png
```

Text cell:

```
## 9. Why the canonical conjugate is not the Hilbert adjoint
```

```
The record's check 'no_positive_inner_product' uses the vector $u$ with
$u_7 = -i/\sqrt2$, $u_{16} = 1/\sqrt2$ and all other entries 0, which obeys
$Bu = -u$. The next cell checks $Bu = -u$ and $u^\dagger Bu = -1$, forms
$X = \sum_A u_A^*\Psi_A$ with its Hilbert adjoint $X^* = \sum_A u_A\chi_A$ and its
canonical conjugate $X^\dagger = \sum_A u_A\Psi^\dagger_A$, and measures
$\{X, X^*\} = +1$ (positive, as for every operator and its Hilbert adjoint) and
$\{X, X^\dagger\} = -1$ (negative): the canonical conjugate cannot be the adjoint of
a positive inner product. The cell also computes the 16 eigenvalues of $B$ (eight
$+1$, eight $-1$).
```

In [10]:

```
u = np.zeros(16, dtype=complex)
u[6], u[15] = -1j / np.sqrt(2), 1 / np.sqrt(2)  # u_7 and u_16 (Python counts from 0)
X_row = u.conj() @ PSI  # X = sum_A conj(u_A) Psi_A
X_adjoint = u @ CHI  # X^* = sum_A u_A chi_A (the Hilbert adjoint)
X_dagger = u @ PSI_DAG  # X^dagger = sum_A u_A Psi^dagger_A (canonical conjugate)
norm_random = inner(RANDOM_STATE, RANDOM_STATE).real
with_adjoint = inner(RANDOM_STATE, anticommutator(X_row, X_adjoint,
                                                  RANDOM_STATE)) / norm_random
with_canonical = inner(RANDOM_STATE, anticommutator(X_row, X_dagger,
                                                    RANDOM_STATE)) / norm_random
eigen_B = np.linalg.eigvalsh(B)  # the 16 eigenvalues of the Hermitian matrix B
say(f"{{X, X^*}} = {with_adjoint.real:+.6f};  {{X, X^dagger}} = "
    f"{with_canonical.real:+.6f};  u^dagger B u = {(u.conj() @ B @ u).real:+.6f}")
check_reproduces(np.allclose(B @ u, -u) and abs(u.conj() @ B @ u + 1) < 1e-12
                 and abs(with_adjoint - 1) < 1e-12 and abs(with_canonical + 1) < 1e-12
                 and recorded("wolfram", "no_positive_inner_product"),
                 "{X, X^*} = +1 but {X, X^dagger} = u^dagger B u = -1: the canonical "
                 "conjugate needs an indefinite (Krein) inner product",
                 record=record_of("wolfram", "no_positive_inner_product"))
```

Out [10]:

```
{X, X^*} = +1.000000;  {X, X^dagger} = -1.000000;  u^dagger B u = -1.000000
PASS {X, X^*} = +1 but {X, X^dagger} = u^dagger B u = -1: the canonical conjugate needs
    an indefinite (Krein) inner product
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    no_positive_inner_product
```

Text cell:

```
The next cell draws the eigenvalues of $B$ and the two measured anticommutators.
```

In [11]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0))
axes[0].plot(np.arange(1, 17), eigen_B, "o", color="#2a78d6", markersize=8)
axes[0].axhline(0.0, color="black", linewidth=0.8)
```

```
axes[0].set_xticks([1, 4, 8, 12, 16])
axes[0].set_xlabel("number of the eigenvalue (sorted)")
axes[0].set_ylabel("eigenvalue of $B$")
axes[0].set_title("$B$: eight eigenvalues $-1$, eight $+1$")
axes[1].bar([0, 1], [with_adjoint.real, with_canonical.real],
            color=["#1baf7a", "#eb6834"], edgecolor="white", linewidth=2, width=0.5)
for x_bar, value in zip([0, 1], [with_adjoint.real, with_canonical.real]):
    axes[1].text(x_bar, value + (0.08 if value > 0 else -0.16), f"{value:+.0f}",
                 ha="center")
axes[1].axhline(0.0, color="black", linewidth=0.8)
axes[1].set_xticks([0, 1], [r"$\{X, X^*\}$ (Hilbert adjoint)",
                            r"$\{X, X^\dagger\}$ (canonical)"])
axes[1].set_ylim(-1.5, 1.5)
axes[1].set_ylabel("measured number")
axes[1].set_title("$X = u^\\dagger\\Psi$ with $Bu = -u$")
save_figure(fig, "why_krein",
            "Why the quantised field needs an indefinite (Krein) inner product. "
            "Left: the 16 eigenvalues of $B$, sorted (horizontal axis their number, "
            "vertical axis their value): eight are $-1$ and eight are $+1$. Right: "
            "for the operator $X = u^\\dagger\\Psi$ built from an eigenvector $u$ of "
            "$B$ with eigenvalue $-1$, the anticommutator with its Hilbert adjoint is "
            "$+1$ (green), as it must be for every operator in a space with a "
            "positive inner product, while the canonical rule gives $u^\\dagger Bu "
            "= -1$ (orange).")
```

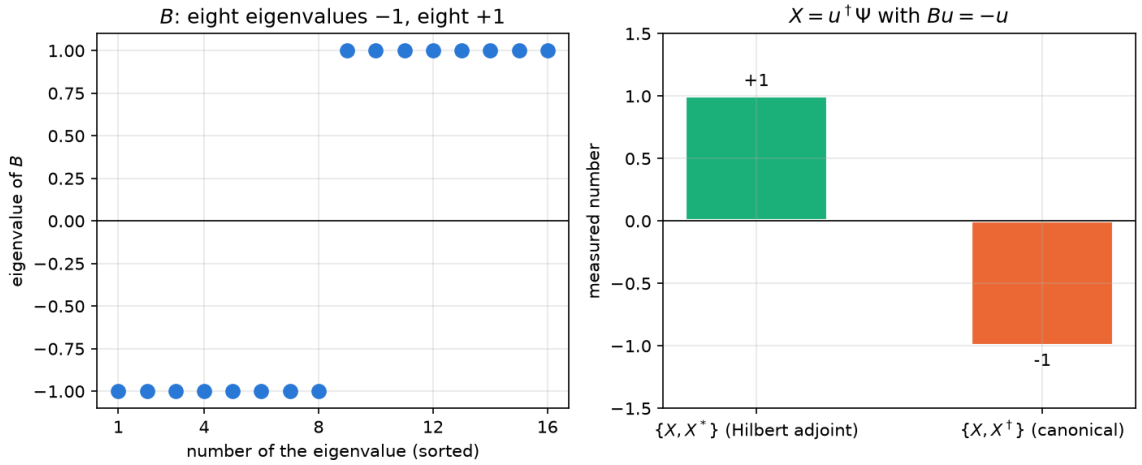

Figure 176: Why the quantised field needs an indefinite (Krein) inner product. Left: the 16 eigenvalues of B, sorted (horizontal axis their number, vertical axis their value): eight are -1 and eight are $+1$. Right: for the operator $X = u^\dagger\Psi$ built from an eigenvector u of B with eigenvalue -1, the anticommutator with its Hilbert adjoint is $+1$ (green), as it must be for every operator in a space with a positive inner product, while the canonical rule gives $u^\dagger Bu = -1$ (orange). (Notebook 05e, figure 3.)

Out [11]:

```
Figure 05e.3 saved as Revision/textbook/figures/05e_3_why_krein.png
```

Text cell:

```
## 10. The vacuum, the 16 quanta and the expectation-value rule

A **bilinear** $\sum_{A,C}L_A K_{AC}R_C$ of two operators with rows $L_A$, $R_C$
is $\sum_{i,j}Q_{ij}O_iO_j$ with the $32 \times 32$ matrix $Q = L^TKR$. The next
cell defines ‘bilinear(L, K, R, state)‘, ‘vacuum_value‘ and ‘normal_ordered_value‘
(the value in a state minus the vacuum value), and computes, as the record does:

- the energy $\chi h\Psi$: its vacuum value must be $-8E = -40$ (the filled sea),
  and after normal ordering each of the 16 quanta $b_s^*|0\rangle$, $d_s^*|0\rangle$
  must have the energy $+5$;
- the charge $J^{(x4)} = \Psi^\dagger B\Psi = \chi BB\Psi = \chi\Psi$: vacuum value 8;
  after normal ordering $+1$ for the 8 particles and $-1$ for the 8 antiparticles;
- the expectation-value rule of the record: $\langle q|:\!\Psi^\dagger M\Psi\!:|q
  \rangle = u_s^\dagger BMu_s$ for the particle $q = b_s^*|0\rangle$ and
  $-v_s^\dagger BMv_s$ for the antiparticle $q = d_s^*|0\rangle$, for the record's
  matrices $M = C$, $-iC\gamma^{(x4)}$ and $-iC\gamma^{(x1)}$.
```

In [12]:

```
VACUUM = {0: 1.0}  # the pattern 0: no particle, no antiparticle


def bilinear(L, K, R, state):
    """sum over A, C of L_A K_AC R_C applied to state (L, R: 16 rows of 32)."""
    Q = L.T @ K @ R  # 32 x 32: the coefficient of O_i O_j
    terms = []
    for j in range(32):
        if np.max(np.abs(Q[:, j])) > 1e-15:
            lowered = basic(j, state)  # O_j applied first
            if lowered:
                terms += [(Q[i, j], basic(i, lowered)) for i in range(32)
                          if abs(Q[i, j]) > 1e-15]
    return combine(terms)


def vacuum_value(L, K, R):
    """<0| sum L K R |0>."""
    return inner(VACUUM, bilinear(L, K, R, VACUUM))


QUANTA = [create(p, VACUUM) for p in range(16)]  # b_s^*|0> (p < 8), d_s^*|0>


def normal_ordered_value(L, K, R, state):
    """<state|X|state> - <0|X|0> for a normalised state."""
    return inner(state, bilinear(L, K, R, state)) - vacuum_value(L, K, R)


vacuum_energy = vacuum_value(CHI, h, PSI)
energies = np.array([normal_ordered_value(CHI, h, PSI, q) for q in QUANTA])
```

```
vacuum_charge = vacuum_value(PSI_DAG, B, PSI)
charges = np.array([normal_ordered_value(PSI_DAG, B, PSI, q) for q in QUANTA])
report("vacuum energy <0|chi h Psi|0> (the filled sea)", f"{vacuum_energy.real:.6f}")
report("vacuum charge <0|Psi^dagger B Psi|0>", f"{vacuum_charge.real:.6f}")
say("normal-ordered energies of the 16 quanta: "
    + " ".join(f"{e.real:+.0f}" for e in energies))
say("normal-ordered charges of the 16 quanta:  "
    + " ".join(f"{c.real:+.0f}" for c in charges))
rule_ok = True
for K_rule in [C, -1j * (C @ gamma["x4"]), -1j * (C @ gamma["x1"])]:
    values = [normal_ordered_value(PSI_DAG, K_rule, PSI, q) for q in QUANTA]
    predicted = ([W[:, s].conj() @ B @ K_rule @ W[:, s] for s in range(8)]
                 + [-(W[:, s].conj() @ B @ K_rule @ W[:, s]) for s in range(8, 16)])
    rule_ok &= np.max(np.abs(np.array(values) - np.array(predicted))) < 1e-12
check_reproduces(abs(vacuum_energy + 40) < 1e-12 and np.max(np.abs(energies - 5)) < 1e-12
                 and abs(vacuum_charge - 8) < 1e-12
                 and np.max(np.abs(charges - np.array([1] * 8 + [-1] * 8))) < 1e-12
                 and rule_ok and "-8 E" in detail("wolfram",
                                                  "Fock_space_good_sector_example"),
                 "vacuum energy -40, every quantum +5, charges +1 and -1, and the "
                 "expectation-value rule for M = C, -i C gamma^(x4), -i C gamma^(x1)",
                 record=record_of("wolfram", "Fock_space_good_sector_example"))
```

Out [12]:

```
RESULT vacuum energy <0|chi h Psi|0> (the filled sea) = -40.000000
RESULT vacuum charge <0|Psi^dagger B Psi|0> = 8.000000
normal-ordered energies of the 16 quanta: +5 +5 +5 +5 +5 +5 +5 +5 +5 +5 +5 +5 +5 +5 +5 +5
normal-ordered charges of the 16 quanta:  +1 +1 +1 +1 +1 +1 +1 +1 -1 -1 -1 -1 -1 -1 -1 -1
PASS vacuum energy -40, every quantum +5, charges +1 and -1, and the expectation-value
    rule for M = C, -i C gamma^(x4), -i C gamma^(x1)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Fock_space_good_sector_example
```

Text cell:

```
The next cell draws the energies and the charges of the 16 quanta before normal
ordering (the value in the state, which contains the vacuum value) and after it.
```

In [13]:

```
raw_energies = energies + vacuum_energy  # <q|chi h Psi|q>
raw_charges = charges + vacuum_charge  # <q|Psi^dagger B Psi|q>
index = np.arange(1, 17)
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for ax, raw, ordered, name in [(axes[0], raw_energies, energies, "energy"),
                               (axes[1], raw_charges, charges, "charge")]:
    ax.bar(index - 0.2, raw.real, 0.4, color="#9e9c98", label="value in the state")
    ax.bar(index + 0.2, ordered.real, 0.4, color="#2a78d6",
           label="after normal ordering")
    ax.axhline(0.0, color="black", linewidth=0.8)
```

```
    ax.axvline(8.5, color="black", linewidth=0.8, linestyle=":")
    ax.set_xticks([1, 4, 8, 9, 12, 16])
    ax.set_xlabel("quantum: 1 to 8 particles, 9 to 16 antiparticles")
    ax.set_title(f"the {name} of the 16 quanta")
axes[0].set_ylabel("value")
handles, names = axes[0].get_legend_handles_labels()  # one legend for both
fig.legend(handles, names, loc="lower center", bbox_to_anchor=(0.5, -0.1), ncol=2)
save_figure(fig, "quanta",
            "The energy $\\chi h\\Psi$ (left) and the charge $\\Psi^\\dagger B\\Psi$ "
            "(right) of the 16 one-quantum states $b_s^{\\ast}|0\\rangle$ (numbers 1 "
            "to 8) and $d_s^{\\ast}|0\\rangle$ (9 to 16) for the record's example "
            "$m = 3$, "
            "momentum 4, $E = 5$: grey the value in the state, which contains the "
            "vacuum value ($-40$ for the energy, the filled sea, and $8$ for the "
            "charge), blue the value after normal ordering. After normal ordering "
            "every quantum has the energy $+5$; particles have the charge $+1$, "
            "antiparticles $-1$.")
```

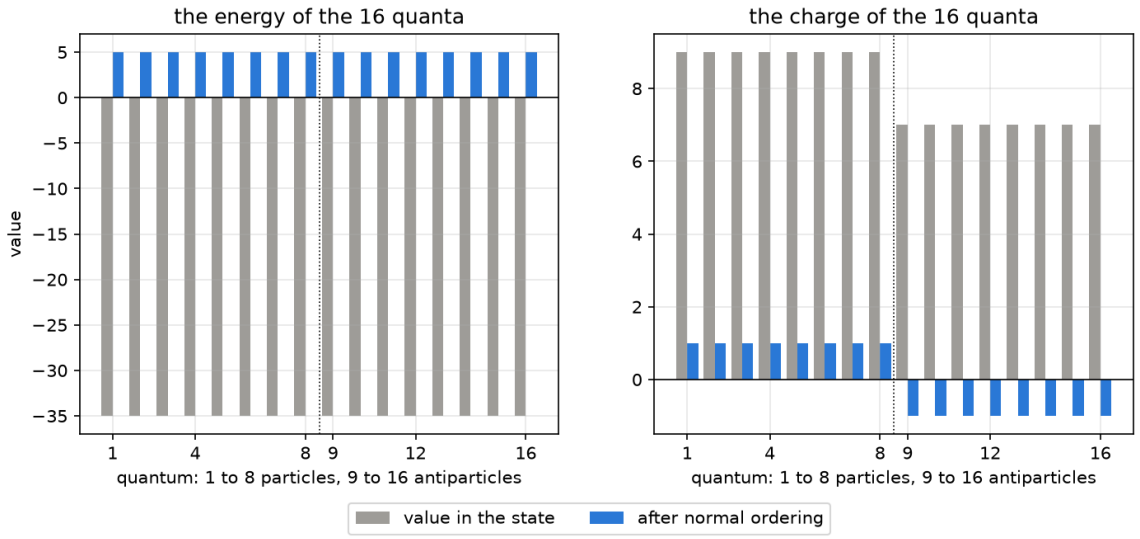

Figure 177: The energy $\chi h\Psi$ (left) and the charge $\Psi^\dagger B\Psi$ (right) of the 16 one-quantum states $b_s^*|0\rangle$ (numbers 1 to 8) and $d_s^*|0\rangle$ (9 to 16) for the record's example $m = 3$, momentum 4, $E = 5$: grey the value in the state, which contains the vacuum value (-40 for the energy, the filled sea, and 8 for the charge), blue the value after normal ordering. After normal ordering every quantum has the energy $+5$; particles have the charge $+1$, antiparticles -1. (Notebook 05e, figure 4.)

Out [13]:

```
Figure 05e.4 saved as Revision/textbook/figures/05e_4_quanta.png
```

Text cell:

```
## 11. The two conjugations of the quantised field
```

```
The next cell builds the conjugated operators of section 4 for $M = 1$ and
$M = \Gamma$: the rows of $\Psi'_A = \sum_C M_{AC}\Psi^\dagger_C$ are `M @ PSI_DAG`
and those of $\Psi'^\dagger_A = \sum_C M^*_{AC}\Psi_C$ are `M.conj() @ PSI`. It
measures $\{\Psi'_A, \Psi'^\dagger_C\}$ on the random state and compares with
$MB^TM^\dagger$, which the Revision record computed: $-B$ for $M = 1$ (the rule is
broken) and $+B$ for $M = \Gamma$ (the rule is kept).
```

In [14]:

```
MAPS = {"M = 1": I16, "M = Gamma": Gamma}  # the two conjugation matrices
conjugated = {}  # name -> (rows of Psi', rows of Psi'^dagger)
measured_conjugated = {}  # name -> the measured {Psi'_A, Psi'^dagger_C}
for name, M in MAPS.items():
    rows_prime = M @ PSI_DAG  # Psi'_A = sum_C M_AC Psi^dagger_C
    rows_prime_dagger = M.conj() @ PSI  # Psi'^dagger_A = sum_C conj(M_AC) Psi_C
    conjugated[name] = (rows_prime, rows_prime_dagger)
    numbers, rest = measured_rule(rows_prime, rows_prime_dagger, RANDOM_STATE)
    measured_conjugated[name] = numbers
    predicted = M @ B.T @ M.conj().T
    say(f"{name:9}: measured equals M B^T M^dagger: "
        f"{np.max(np.abs(numbers - predicted)) < 1e-12 and rest < 1e-12}; "
        f"it equals +B: {np.max(np.abs(numbers - B)) < 1e-12}; "
        f"-B: {np.max(np.abs(numbers + B)) < 1e-12}")
check_reproduces(np.max(np.abs(measured_conjugated["M = 1"] + B)) < 1e-12
                 and np.max(np.abs(measured_conjugated["M = Gamma"] - B)) < 1e-12
                 and recorded("lead", "quantum_charge_conjugation_unitary_type"),
                 "on the Fock space: Psi' = Psi^(dagger T) gives -B, Psi' = Gamma "
                 "Psi^(dagger T) gives +B: only M = Gamma keeps the canonical rule",
                 record=record_of("lead", "quantum_charge_conjugation_unitary_type"))
```

Out [14]:

```
M = 1    : measured equals M B^T M^dagger: True; it equals +B: False; -B: True
M = Gamma: measured equals M B^T M^dagger: True; it equals +B: True; -B: False
PASS on the Fock space: Psi' = Psi^(dagger T) gives -B, Psi' = Gamma Psi^(dagger T) gives
    +B: only M = Gamma keeps the canonical rule
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    quantum_charge_conjugation_unitary_type
```

Text cell:

```
The next cell draws the two measured rules next to $B$.
```

In [15]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], B.imag, r"$B$ / $i$ (the rule)")
heat_map(axes[1], measured_conjugated["M = 1"].imag,
         r"measured, $M = 1$, / $i$", row_label=False)
```

```
image = heat_map(axes[2], measured_conjugated["M = Gamma"].imag,
                 r"measured, $M = \Gamma$, / $i$", row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "conjugated_rules",
            "The canonical rule after the two conjugations, measured on the Fock "
            "space: the numbers $\\{\\Psi'_A, \\Psi'^\\dagger_C\\}$ divided by $i$ "
            "for $\\Psi' = \\Psi^{\\dagger T}$ (middle, the type of the same-mass "
            "matrix $\\mathcal{C}_+$) and $\\Psi' = \\Gamma\\Psi^{\\dagger T}$ "
            "(right, the type of the mass-reversing matrix $\\mathcal{C}_-$), next "
            "to $B$ divided by $i$ (left); row $A$ and column $C$ from 1 to 16; blue "
            "$-1$, grey 0, red $+1$. The middle picture has every colour reversed "
            "($-B$): that map breaks the rule. The right picture equals the left "
            "one: only $\\Gamma$ gives a conjugation of the quantised field.")
```

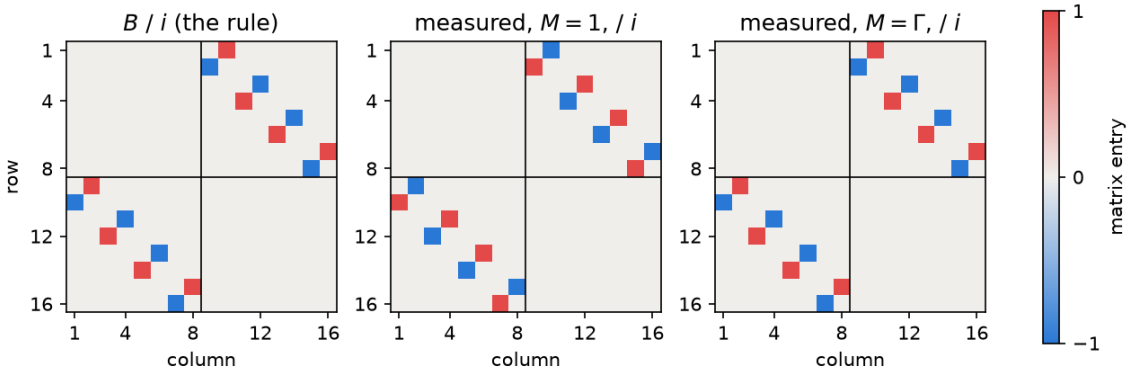

Figure 178: The canonical rule after the two conjugations, measured on the Fock space: the numbers $\{\Psi'_A, \Psi'^\dagger_C\}$ divided by i for $\Psi' = \Psi^{\dagger T}$ (middle, the type of the same-mass matrix $\mathcal{C}_+$) and $\Psi' = \Gamma\Psi^{\dagger T}$ (right, the type of the mass-reversing matrix $\mathcal{C}_-$), next to B divided by i (left); row A and column C from 1 to 16; blue -1, grey 0, red $+1$. The middle picture has every colour reversed ($-B$): that map breaks the rule. The right picture equals the left one: only Γ gives a conjugation of the quantised field. (Notebook 05e, figure 5.)

Out [15]:

```
Figure 05e.5 saved as Revision/textbook/figures/05e_5_conjugated_rules.png
```

Text cell:

```
## 12. The conjugated bilinears: a sign and a constant

The next cell computes, for the scalar ($K = C$) and the eight currents
($K = -iC\gamma^a$) and for both maps, the conjugated operator $X' =
\Psi'^\dagger K\Psi'$ and the original $X = \Psi^\dagger K\Psi$, both applied to
the random state, and finds the sign $s$ and the number $c$ with $X' = sX + c$.
Section 4 predicts $c = \mathrm{tr}(K'B^T)$ with $K' = M^\dagger KM$, and $s$ from
$-(M^\dagger KM)^T = sK$. The cell compares the measured signs with the table that
the Revision record measured for classical *anticommuting* components (its keys
‘plus,eps=-1‘ and ‘minus,eps=-1‘, stored in the detail text after the word
measured).
```

In [16]:

```
K_MATRICES = {"S": C.astype(complex)}  # the scalar S = Psi^dagger C Psi
for x in COORDS:
    K_MATRICES[f"J^({x})"] = -1j * (C @ gamma[x])  # the current J^a
BILINEAR_NAMES = list(K_MATRICES)  # "S", "J^(x0)", ..., "J^(x7)"
signs, constants = {}, {}  # (map, bilinear) -> s and c
identity_ok = True
for map_name, M in MAPS.items():
    rows_prime, rows_prime_dagger = conjugated[map_name]
    for bname, K in K_MATRICES.items():
        K_prime = M.conj().T @ K @ M  # K' = M^dagger K M
        c = np.trace(K_prime @ B.T)  # the predicted constant
        s = 1 if np.array_equal(-K_prime.T, K) else (
            -1 if np.array_equal(-K_prime.T, -K) else 0)  # -K'^T = s K
        X_prime = bilinear(rows_prime_dagger, K, rows_prime, RANDOM_STATE)
        X = bilinear(PSI_DAG, K, PSI, RANDOM_STATE)
        difference = combine([(1, X_prime), (-s, X), (-c, RANDOM_STATE)])
        identity_ok &= s != 0 and largest(difference) < 1e-10
        signs[(map_name, bname)], constants[(map_name, bname)] = s, c
for map_name in MAPS:
    say(f"{map_name:9}: s = " + " ".join(f"{signs[(map_name, b)]:+d}"
                                          for b in BILINEAR_NAMES)
        + "  (S, J^(x0), ..., J^(x7))")
    say(f"{'':9}  c = " + " ".join(f"{constants[(map_name, b)].real:+.0f}"
                                   for b in BILINEAR_NAMES))
lead_detail = detail("lead", "bilinears_under_charge_conjugation")
recorded_table = json.loads(lead_detail.split("measured: ", 1)[1])
anticommuting_rows = {"M = 1": recorded_table["plus,eps=-1"],
                      "M = Gamma": recorded_table["minus,eps=-1"]}
rows_agree = all([signs[(m_name, "S")], [signs[(m_name, b)] for b in
                                         BILINEAR_NAMES[1:]]] == row
                  for m_name, row in anticommuting_rows.items())
check(identity_ok, "X' = s X + c holds on the Fock space for all 9 bilinears and both "
      "maps, with c = tr(K' B^T)")
check_reproduces(rows_agree and recorded("lead", "bilinears_under_charge_conjugation"),
                 "the operator signs s equal the measured signs of the anticommuting "
                 "rows (eps = -1) of the recorded table",
                 record=record_of("lead", "bilinears_under_charge_conjugation"))
```

Out [16]:

```
M = 1    : s = -1 +1 +1 +1 +1 +1 +1 +1 +1  (S, J^(x0), ..., J^(x7))
           c = +0 +0 +0 +0 +0 -16 +0 +0 +0
M = Gamma: s = -1 -1 -1 -1 -1 -1 -1 -1 -1  (S, J^(x0), ..., J^(x7))
           c = +0 +0 +0 +0 +0 +16 +0 +0 +0
PASS X' = s X + c holds on the Fock space for all 9 bilinears and both maps, with c =
    tr(K' B^T)
PASS the operator signs s equal the measured signs of the anticommuting rows (eps = -1)
    of the recorded table
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    bilinears_under_charge_conjugation
```

Text cell:

```
The number $c$ is zero for every bilinear except the charge density $J^{(x4)}$
(whose matrix $-iC\gamma^{(x4)}$ is $B$): there $c = \mathrm{tr}(BB^T) = -\mathrm{tr}
(BB) = -16$ for $M = 1$ and $+16$ for $M = \Gamma$. The next cell checks this and
computes the vacuum values $\langle 0|X|0\rangle$ and $\langle 0|X'|0\rangle$ of all
bilinears: they obey $\langle 0|X'|0\rangle = s\langle 0|X|0\rangle + c$. Then it
draws them.
```

In [17]:

```
nonzero = sorted({b for (m_name, b), c in constants.items() if abs(c) > 1e-12})
check(nonzero == ["J^(x4)"] and abs(constants[("M = 1", "J^(x4)")] + 16) < 1e-12
      and abs(constants[("M = Gamma", "J^(x4)")] - 16) < 1e-12,
      "c = 0 except for the charge density J^(x4): c = -16 (M = 1), +16 (M = Gamma)")
vac_original = {b: vacuum_value(PSI_DAG, K, PSI) for b, K in K_MATRICES.items()}
vac_conjugated = {}  # (map, bilinear) -> <0|X'|0>
for map_name in MAPS:
    rows_prime, rows_prime_dagger = conjugated[map_name]
    for bname, K in K_MATRICES.items():
        vac_conjugated[(map_name, bname)] = vacuum_value(rows_prime_dagger, K,
                                                         rows_prime)
# values below 1e-12 are printed as 0 (rounding could otherwise print "-0.00")
shown = {b: (v.real if abs(v) > 1e-12 else 0.0) for b, v in vac_original.items()}
say("vacuum values <0|X|0>: " + ", ".join(f"{b} {shown[b]:+.2f}"
                                          for b in BILINEAR_NAMES))
check(all(abs(vac_conjugated[key] - signs[key] * vac_original[key[1]]
              - constants[key]) < 1e-12 for key in vac_conjugated),
      "<0|X'|0> = s <0|X|0> + c for all 9 bilinears and both maps")
positions = np.arange(len(BILINEAR_NAMES))
labels = ["$S$"] + [rf"$J^{{({x})}}$" for x in COORDS]
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.2), sharey=True)
for ax, map_name, title in [(axes[0], "M = 1", r"$\Psi' = \Psi^{\dagger T}$"),
                            (axes[1], "M = Gamma",
                             r"$\Psi' = \Gamma\Psi^{\dagger T}$")]:
    ax.bar(positions - 0.27, [vac_original[b].real for b in BILINEAR_NAMES], 0.27,
           color="#9e9c98", label=r"$\langle 0|X|0\rangle$")
    ax.bar(positions, [vac_conjugated[(map_name, b)].real for b in BILINEAR_NAMES],
           0.27, color="#2a78d6", label=r"$\langle 0|X'|0\rangle$")
    ax.bar(positions + 0.27, [constants[(map_name, b)].real for b in BILINEAR_NAMES],
           0.27, color="#eb6834", label="the constant $c$")
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, labels)
    ax.set_title(title)
    ax.legend(loc="lower left")
axes[0].set_ylabel("value")
save_figure(fig, "vacuum_values",
            "Vacuum values of the nine bilinears (horizontal axis: the scalar $S$ "
            "and the currents $J^a$) before the conjugation (grey) and after it "
            "(blue), and the constant $c$ of the identity $X' = sX + c$ (orange), "
            "for the conjugation with $M = 1$ (left) and $M = \\Gamma$ (right); "
            "vertical axis the value (a pure number). The filled sea gives nonzero "
```

```
"vacuum values; $c$ vanishes except for the charge density $J^{(x4)}$, "
"where it is $-16$ or $+16$. Normal ordering subtracts each operator's "
"own vacuum value, which removes $c$.")
```

Out [17]:

```
PASS c = 0 except for the charge density J^(x4): c = -16 (M = 1), +16 (M = Gamma)
vacuum values <0|X|0>: S -4.80, J^(x0) +0.00, J^(x1) -6.40, J^(x2) +0.00, J^(x3) +0.00,
    J^(x4) +8.00, J^(x5) +0.00, J^(x6) +0.00, J^(x7) +0.00
PASS <0|X'|0> = s <0|X|0> + c for all 9 bilinears and both maps
```

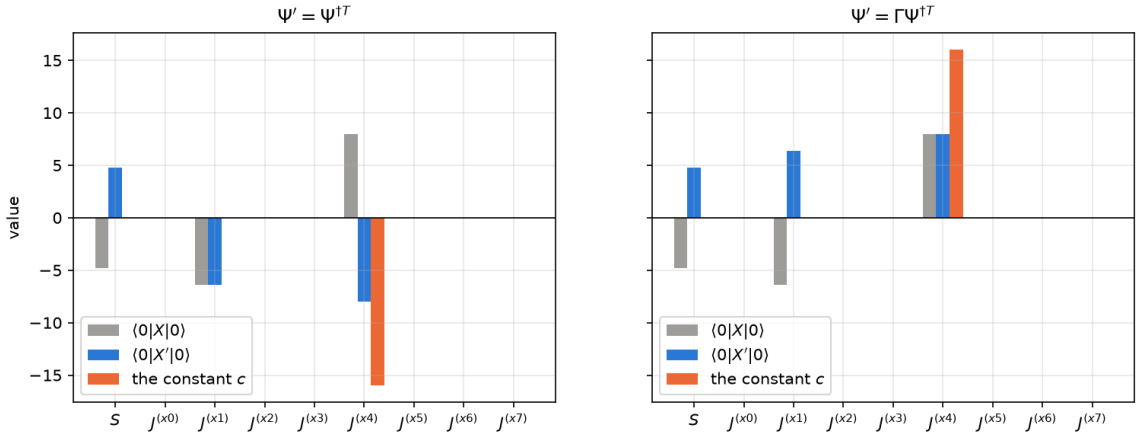

Figure 179: Vacuum values of the nine bilinears (horizontal axis: the scalar S and the currents J^a) before the conjugation (grey) and after it (blue), and the constant c of the identity $X' = sX + c$ (orange), for the conjugation with $M = 1$ (left) and $M = \Gamma$ (right); vertical axis the value (a pure number). The filled sea gives nonzero vacuum values; c vanishes except for the charge density $J^{(x4)}$, where it is -16 or $+16$. Normal ordering subtracts each operator's own vacuum value, which removes c. (Notebook 05e, figure 6.)

Out [17]:

```
Figure 05e.6 saved as Revision/textbook/figures/05e_6_vacuum_values.png
```

Text cell:

```
## 13. After normal ordering: the sign table of the quantised field

The next cell normal-orders both sides, $:\!X\!: = X - \langle 0|X|0\rangle$ and
$:\!X'\!: = X' - \langle 0|X'|0\rangle$, and checks $:\!X'\!: = s\,:\!X\!:$ on the
random state and on the 16 one-quantum states, for all bilinears and both maps.
It then computes, as the Revision record does, the classical tables:
$\epsilon(M^\dagger KM)^T$ compared with $K$, for commuting ($\epsilon = +1$) and
anticommuting ($\epsilon = -1$) components, and draws three tables: commuting,
anticommuting and the quantised field after normal ordering.
```

What the record says. The detail text of the record's check
‘bilinears_under_charge_conjugation‘ states that normal ordering adds no sign: it
subtracts the vacuum value, a number, so $X' = sX + c$ gives
$:\!X'\!: = s\,:\!X\!:$. The record makes no Fock-space computation; this cell
makes one. It confirms the statement: normal ordering removes only the constant
c, and the operator signs are exactly the measured signs of the anticommuting
rows. (Until 2026-10-08 that detail text said instead that normal ordering
supplies one more sign for each bilinear, which would give $(S, J) \to (S, -J)$
for $\mathcal{C}_+$; this computation contradicted it, and the record was
corrected.)

In [18]:

```
def apply_normal_ordered(L, K, R, state):
    """:X: = X - <0|X|0> applied to state."""
    return combine([(1, bilinear(L, K, R, state)),
                    (-vacuum_value(L, K, R), state)])


normal_ok = True
for map_name in MAPS:
    rows_prime, rows_prime_dagger = conjugated[map_name]
    for bname, K in K_MATRICES.items():
        for state in [RANDOM_STATE] + QUANTA:
            left = apply_normal_ordered(rows_prime_dagger, K, rows_prime, state)
            right = apply_normal_ordered(PSI_DAG, K, PSI, state)
            normal_ok &= largest(combine([(1, left),
                                          (-signs[(map_name, bname)], right)])) < 1e-10
check(normal_ok, ":X': = s :X: for all 9 bilinears, both maps, on the random state "
      "and the 16 quanta")


def classical_sign(M, K, eps):
    """+1 if eps (M^dagger K M)^T = K, -1 if it equals -K, 0 otherwise."""
    new = eps * (M.conj().T @ K @ M).T
    return 1 if np.array_equal(new, K) else (-1 if np.array_equal(new, -K) else 0)


tables = {"commuting components (classical)": 1,
          "anticommuting components (classical)": -1}
grids = {}
for title, eps in tables.items():
    grids[title] = np.array([[classical_sign(M, K_MATRICES[b], eps)
                              for b in BILINEAR_NAMES] for M in MAPS.values()])
grids["quantised field after normal ordering"] = np.array(
    [[signs[(m_name, b)] for b in BILINEAR_NAMES] for m_name in MAPS])
check(np.array_equal(grids["quantised field after normal ordering"],
                     grids["anticommuting components (classical)"])
      and np.array_equal(grids["commuting components (classical)"],
                         -grids["anticommuting components (classical)"]),
      "after normal ordering the quantised field has the signs of the anticommuting "
      "components; the commuting ones are the opposite")
fig, axes = plt.subplots(3, 1, figsize=(9.0, 6.6))
```

```
for ax, (title, grid) in zip(axes, grids.items()):
    ax.imshow(grid, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
    for r in range(2):
        for c_col in range(9):
            ax.text(c_col, r, f"{grid[r, c_col]:+d}", ha="center", va="center",
                    color="white", fontweight="bold")
    ax.axhline(0.5, color="white", linewidth=2)
    for k in range(1, 9):
        ax.axvline(k - 0.5, color="white", linewidth=3 if k == 1 else 2)
    ax.set_xticks(range(9), labels)
    ax.set_yticks([0, 1], [r"$\mathcal{C}_+$ type, $M = 1$",
                           r"$\mathcal{C}_-$ type, $M = \Gamma$"])
    ax.set_title(title)
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "sign_tables",
            "The sign that the scalar $S$ and the eight currents $J^a$ (columns) "
            "acquire under the two conjugations (rows: $M = 1$, the type of "
            "$\\mathcal{C}_+$, and $M = \\Gamma$, the type of $\\mathcal{C}_-$); red "
            "$+1$ kept, blue $-1$ reversed. Top: classical commuting components; "
            "middle: classical anticommuting components (the two tables measured "
            "by the Revision record); bottom: the quantised field, computed as "
            "operators on the Fock space and normal ordered. The bottom table "
            "equals the middle one: normal ordering adds no sign.")
```

Out [18]:

```
PASS :X': = s :X: for all 9 bilinears, both maps, on the random state and the 16 quanta
PASS after normal ordering the quantised field has the signs of the anticommuting
    components; the commuting ones are the opposite
```

Out [18]:

```
Figure 05e.7 saved as Revision/textbook/figures/05e_7_sign_tables.png
```

Text cell:

```
## 14. What the allowed conjugation does to the charges of the quanta

Of the two maps, only $M = \Gamma$ keeps the canonical rule (section 11). The next
cell evaluates the normal-ordered charge density $:\!J^{(x4)}\!:$ and its two
conjugates
$:\!J'^{(x4)}\!:$ in the 16 one-quantum states. With $M = \Gamma$ every charge is
reversed (particles $+1 \to -1$, antiparticles $-1 \to +1$); with $M = 1$ (which
breaks the rule) the charges are unchanged.
```

In [19]:

```
K_charge = K_MATRICES["J^(x4)"]  # equal to B
charge_values = {"original": charges.real}
```

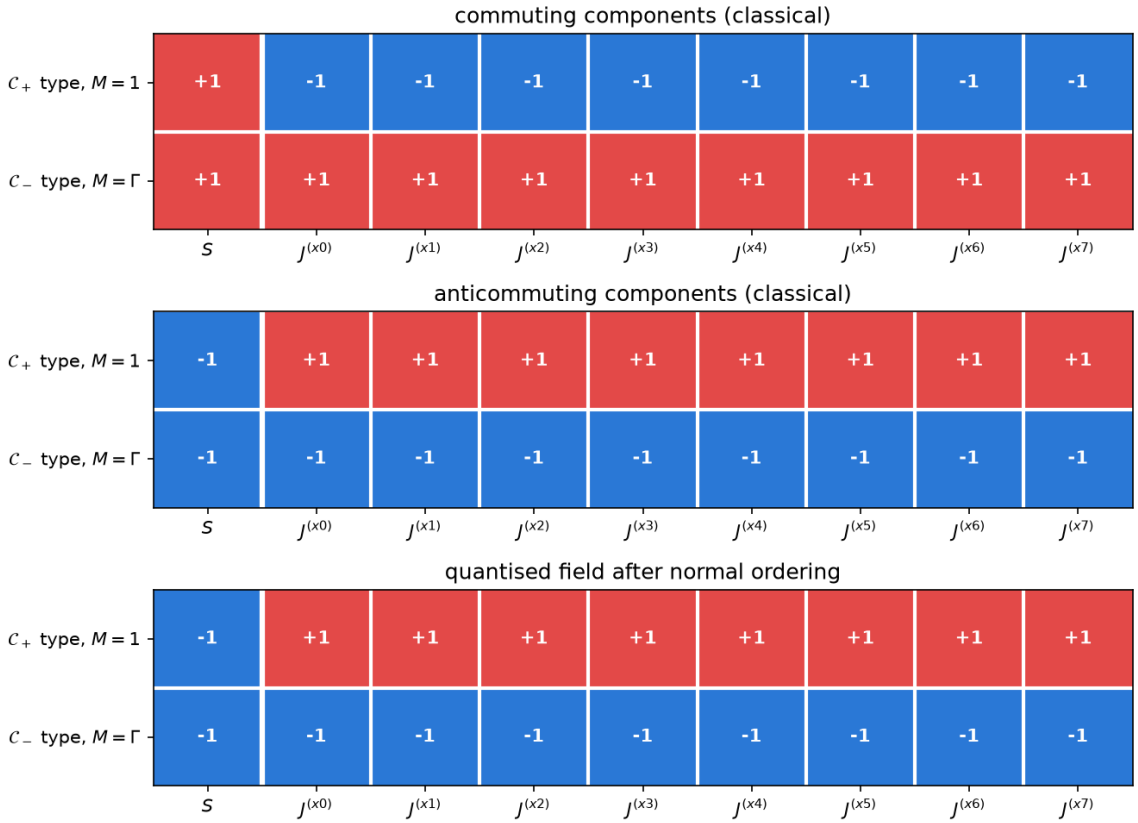

Figure 180: The sign that the scalar S and the eight currents J^a (columns) acquire under the two conjugations (rows: $M = 1$, the type of $\mathcal{C}_+$, and $M = \Gamma$, the type of $\mathcal{C}_-$); red $+1$ kept, blue -1 reversed. Top: classical commuting components; middle: classical anticommuting components (the two tables measured by the Revision record); bottom: the quantised field, computed as operators on the Fock space and normal ordered. The bottom table equals the middle one: normal ordering adds no sign. (Notebook 05e, figure 7.)

```
for map_name in MAPS:
    rows_prime, rows_prime_dagger = conjugated[map_name]
    charge_values[map_name] = np.array(
        [normal_ordered_value(rows_prime_dagger, K_charge, rows_prime, q).real
         for q in QUANTA])
check(np.array_equal(K_charge, B)
      and np.max(np.abs(charge_values["M = Gamma"] + charges.real)) < 1e-12
      and np.max(np.abs(charge_values["M = 1"] - charges.real)) < 1e-12,
      "M = Gamma reverses the normal-ordered charge of every quantum; M = 1 keeps it")
fig, ax = plt.subplots(figsize=(8.0, 4.2))
ax.plot(index, charge_values["original"], "o", color="#9e9c98", markersize=12,
        label=r"$:\!J^{(x4)}\!:$ (the field)")
ax.plot(index, charge_values["M = 1"], "s", color="#eb6834", markersize=6,
        label=r"$M = 1$ (breaks the rule)")
ax.plot(index, charge_values["M = Gamma"], "D", color="#2a78d6", markersize=7,
        label=r"$M = \Gamma$ (keeps the rule)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.axvline(8.5, color="black", linewidth=0.8, linestyle=":")
ax.set_xticks([1, 4, 8, 9, 12, 16])
ax.set_ylim(-1.6, 1.6)
ax.set_xlabel("quantum: 1 to 8 particles, 9 to 16 antiparticles")
ax.set_ylabel("normal-ordered charge")
ax.set_title("The charge of each quantum, and of its conjugates")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=3)
save_figure(fig, "conjugated_charges",
            "The normal-ordered charge density in each of the 16 one-quantum states "
            "(horizontal axis: 1 to 8 particles, 9 to 16 antiparticles; vertical axis "
            "the charge, a pure number): the field itself (grey circles: $+1$ for "
            "particles, $-1$ for antiparticles), the conjugate with $M = 1$ (orange "
            "squares, on the grey circles: unchanged) and the conjugate with $M = "
            "\\Gamma$ (blue diamonds: every charge reversed). The one of the two "
            "conjugations that keeps the canonical rule, $\\Psi \\to "
            "\\Gamma\\Psi^{\\dagger T}$, "
            "reverses the charge (and, as the Revision record states, the mass).")
```

Out [19]:

```
PASS M = Gamma reverses the normal-ordered charge of every quantum; M = 1 keeps it
```

Out [19]:

```
Figure 05e.8 saved as Revision/textbook/figures/05e_8_conjugated_charges.png
```

Text cell:

```
## 15. The last check

The last cell checks that the eight figure files exist and prints the number of
checks that passed.
```

In [20]:

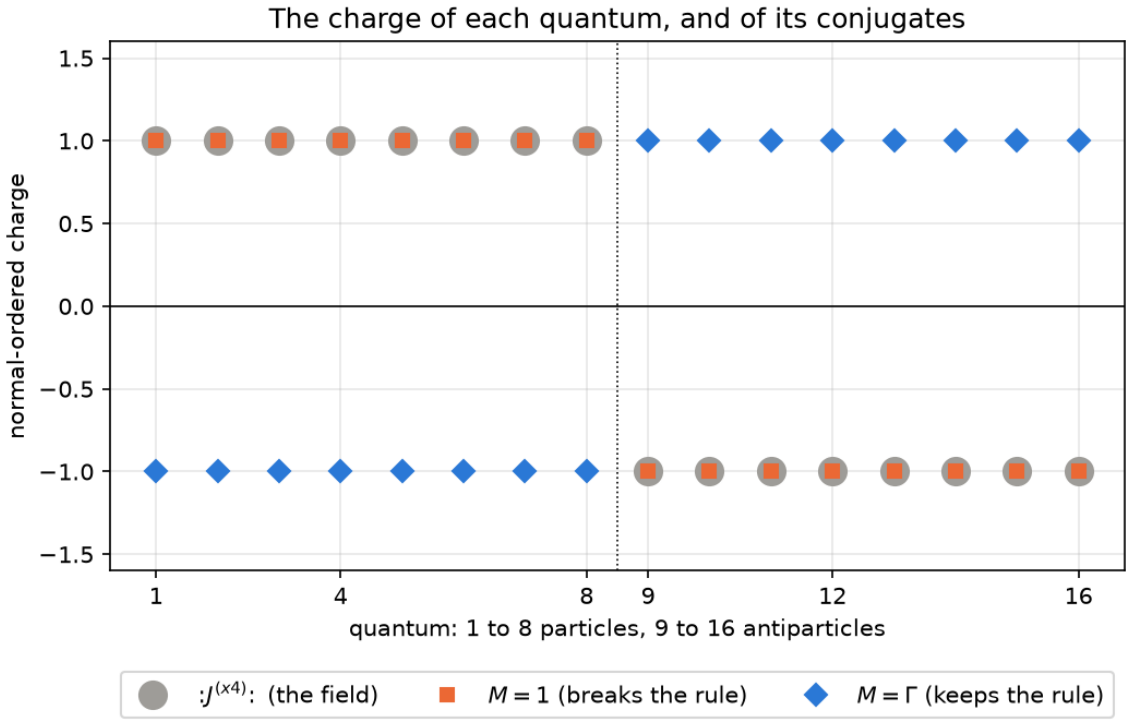

Figure 181: The normal-ordered charge density in each of the 16 one-quantum states (horizontal axis: 1 to 8 particles, 9 to 16 antiparticles; vertical axis the charge, a pure number): the field itself (grey circles: $+1$ for particles, -1 for antiparticles), the conjugate with $M = 1$ (orange squares, on the grey circles: unchanged) and the conjugate with $M = \Gamma$ (blue diamonds: every charge reversed). The one of the two conjugations that keeps the canonical rule, $\Psi \to \Gamma\Psi^{\dagger T}$, reverses the charge (and, as the Revision record states, the mass). (Notebook 05e, figure 8.)

```
FIGURES = ["05e_1_two_modes.png", "05e_2_anticommutators.png", "05e_3_why_krein.png",
           "05e_4_quanta.png", "05e_5_conjugated_rules.png",
           "05e_6_vacuum_values.png", "05e_7_sign_tables.png",
           "05e_8_conjugated_charges.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the eight figure files of notebook 05e exist")
all_checks_passed()
```

Out [20]:

```
PASS the eight figure files of notebook 05e exist
ALL 18 CHECKS PASSED (notebook 05e)
```

Text cell:

```
## 16. What this notebook showed

- PROVED (section 4, line by line) and COMPUTED (on the Fock space of 65536
  states): for the quantised field, the conjugated bilinear is
  $X' = \Psi^\dagger\big(-(M^\dagger KM)^T\big)\Psi + \mathrm{tr}(M^\dagger KMB^T)$:
  the rule of classical anticommuting components plus a constant.
- COMPUTED, reproducing the Revision record: the positive realisation
  $\Psi^\dagger = \chi B$ obeys $\{\Psi_A, \chi_C\} = \delta_{AC}$ and
  $\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$; the canonical conjugate cannot be a Hilbert
  adjoint ($\{X, X^\dagger\} = -1$); the vacuum energy is $-8E = -40$, every quantum
  has the energy $+5$ after normal ordering, particles have the charge $+1$ and
  antiparticles $-1$; the expectation-value rule holds.
- COMPUTED, reproducing the lead record: $\Psi \to \Psi^{\dagger T}$ turns the
  canonical rule into $-B$, $\Psi \to \Gamma\Psi^{\dagger T}$ keeps it.
- COMPUTED: the operator signs equal the measured signs of the anticommuting rows
  of the record's table: $M = 1$ gives $(S, J) \to (-S, +J)$, $M = \Gamma$ gives
  $(S, J) \to (-S, -J)$. The constant $c$ vanishes except for the charge density
  ($\mp 16$), and normal ordering removes it: after normal ordering the quantised
  field has exactly the anticommuting signs. This agrees with the record's check
  `bilinears_under_charge_conjugation`, whose detail text states that normal
  ordering adds no sign.
- COMPUTED: of the two conjugations, the one that keeps the canonical rule,
  $\Psi \to \Gamma\Psi^{\dagger T}$ (among the multiples of 1 and of $\Gamma$ the
  only one, up to a phase factor), reverses the normal-ordered charge of every
  quantum.
- SCOPE: one momentum without extra-time part, at one point of space, with the
  metric treated as flat near that point; the vacuum is the Fock vacuum of that
  momentum. The sign argument of section 4 uses only the
  canonical rule and the subtraction of a number, so it does not depend on these
  choices.
```

5.38 Line-by-line walk-through of Notebook 05e

The notebook has 20 code cells. In [1] is the set-up cell of Section 5.10 with `NOTEBOOK_ID = "05e"`; its comments repeat the instructions of Section 5.36. Docstrings are left out of the quotations and long captions are shortened to ..., as before.

In [2], the gammas, C, Γ, B and the records.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer io.StringIO
import sys  # sys.stdout: the channel through which the notebook prints

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1
```

The modules and the gammas, as in Notebook 05c, In [2] (Section 5.33).

```
C = gamma["x0"] @ gamma["x1"] @ gamma["x2"] @ gamma["x3"]  # the charge matrix
Gamma = I16
for x in ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]:
    Gamma = Gamma @ gamma[x]  # the chirality: the product of all eight gammas
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4), entries 0, +i, -i
```

C, Γ and B; the loop multiplies the eight gammas in the author's order, starting from the identity.

```
REPORT_FILES = {"lead": "Revision/lead_checks/reports/charge-conjugation-and-u1.json",
                "wolfram": "Revision/theory/reports/wolfram-field-theory.json"}
VERDICTS = {}  # (report key, check name) -> (verdict in lower case, detail text)
for key, path in REPORT_FILES.items():
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in report_data["checks"]:
        VERDICTS[(key, entry["name"])] = (entry["verdict"].lower(), entry["detail"])
```

Two reports are read: the lead report of charge conjugation and the Wolfram report of the field theory, which holds the record's checks of the quantisation. Every check is stored under the pair (report key, check name), as in Notebook 05a (Section 5.10, In [3]).

```
def recorded(key, name):
    return VERDICTS[(key, name)][0] == "pass"


def detail(key, name):
    return VERDICTS[(key, name)][1]


def record_of(key, name):
    return f"{REPORT_FILES[key]}, check {name}"


def check_reproduces(condition, name, record):
    collected = io.StringIO()
```

```
    with contextlib.redirect_stdout(collected):  # print into the buffer
        check(condition, name, record=record)  # stops here if the check fails
    sys.stdout.write(collected.getvalue())  # the PASS and reproduces lines together
```

The four helpers of Notebook 05b (Section 5.17, In [2]): whether a report holds a check with the verdict pass, its detail text, the text printed after reproduces, and the check that prints its two lines in one piece.

```
say(f"{len(VERDICTS)} recorded checks were read from the two reports.")
check(np.array_equal(B.conj().T, B) and np.array_equal(B @ B, I16)
      and not np.any(np.diag(B)) and np.array_equal(B.T, -B),
      "B is Hermitian, B B = 1, B^T = -B and every diagonal entry of B is 0")
```

The printed line reports 96 checks: the 12 of the lead report and the 84 of the Wolfram report. The check confirms the facts about B that Section 5.34 uses: Hermitian, square 1, $B^T = -B$, and a zero diagonal (`np.diag(B)` is the list of the diagonal entries).

In [3], the rule of the record.

```
theory = json.loads(repository_file("Revision/theory/field-theory.json")
                    .read_text(encoding="utf-8"))
formula = next(f for f in theory["formulas"] if f["key"] == "quantisation")["wl"]
RULE = "{Psi_A(x), Psi^dagger_C(y)}_(x4 = y4) = B_AC delta^7(x - y)/Cos[z]"
POSITIVE = "positive representation chi = Psi^dagger B, {Psi_A, chi_C} = delta_AC"
say("the record states: " + RULE)
say("and: " + POSITIVE)
check(RULE in formula and POSITIVE in formula,
      "the formula quantisation of Revision/theory/field-theory.json states both rules")
```

The record `Revision/theory/field-theory.json` holds a list of formulas, each with a key and a text (`"wl"`, written in the notation of the Wolfram Language). `next(f for f in ... if ...)` takes the first formula whose key is `quantisation`. The two sentences that this notebook realises, the canonical rule and its positive representation with $\chi = \Psi^\dagger B$ (equivalently $\Psi^\dagger = \chi B$, because $BB = 1$), are printed, and the check confirms that the record's text contains both, character by character.

In [4], two modes as 4×4 matrices.

```
def two_mode_matrix(p, filling):
    matrix = np.zeros((4, 4), dtype=np.int64)
    for n in range(4):  # the pattern n: binary digit q = occupation of mode q
        full = (n >> p) & 1  # the occupation of mode p in the pattern n
        if full != filling:  # f_p needs a full mode, f_p^* an empty one
            below = bin(n & ((1 << p) - 1)).count("1")  # occupied modes below p
            matrix[n ^ (1 << p), n] = (-1) ** below  # ^ flips binary digit p
    return matrix
```

The computer stores a pattern of occupations as a whole number n whose **binary digits** are the occupations: digit q (counted from 0, from the right) is the occupation of mode q. So for two modes

$n = 0, 1, 2, 3$ are $|00\rangle$, $|10\rangle$, $|01\rangle$, $|11\rangle$. The operators on binary digits: shifting n to the right by p places and taking `& 1` (the last binary digit) gives the occupation of mode p; `1` shifted to the left by p places is the number with a single 1 in digit p, and subtracting 1 from it gives the number with ones in all digits below p; `n & (...)` keeps only those digits of n; `bin(...)` writes a number in binary digits, and `.count("1")` counts the occupied modes below p. The operator `^` (exclusive or) with that single-digit number flips digit p. For each pattern n on which the operator acts (`filling=False`: f_p, which needs a full mode; `filling=True`: f_p^*, which needs an empty one), column n of the matrix gets the entry $(-1)^{\text{below}}$ in the row of the new pattern. Columns of patterns on which the operator gives zero stay zero.

```
f2 = {p: two_mode_matrix(p, False) for p in (0, 1)}  # f_0, f_1
f2_star = {p: two_mode_matrix(p, True) for p in (0, 1)}  # f_0^*, f_1^*
Z4 = np.zeros((4, 4), dtype=np.int64)
relations_ok = all(
    np.array_equal(f2[p] @ f2_star[q] + f2_star[q] @ f2[p],
                   np.eye(4, dtype=np.int64) if p == q else Z4)
    and np.array_equal(f2[p] @ f2[q] + f2[q] @ f2[p], Z4)
    and np.array_equal(f2_star[p], f2[p].T)
    for p in (0, 1) for q in (0, 1))
```

The four matrices f_0, f_1, f_0^*, f_1^*, and for all four pairs (p, q) the canonical relations $\{f_p, f_q^*\} = \delta_{pq}1$ and $\{f_p, f_q\} = 0$, and that f_p^* is the transpose of f_p (the matrices are real, so the transpose is the Hilbert adjoint).

```
# column 3 of f_1 is the image of the pattern |11>; its entry in row 1 (|10>) is -1
say(f"f_1 applied to |11>: amplitudes on |00>, |10>, |01>, |11> = "
    f"{f2[1][:, 3].tolist()}")
check(relations_ok and all(not np.any(f2[p] @ f2[p]) for p in (0, 1)),
      "two modes: {f_p, f_q^*} = delta_pq, {f_p, f_q} = 0, f_p f_p = 0, "
      "f_p^* = f_p^T")
```

The printed line is column 3 of f_1, the image of $|11\rangle$: $[0, -1, 0, 0]$, that is $f_1|11\rangle = -|10\rangle$, the example of Section 5.34. The check adds $f_p f_p = 0$.

In [5], the picture of the two-mode operators.

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
PATTERNS = ["|00>", "|10>", "|01>", "|11>"]  # the patterns n = 0, 1, 2, 3
fig, axes = plt.subplots(1, 4, figsize=(13.0, 3.5))
```

The colour map of the heat maps, the names of the four patterns and a row of four pictures.

```
for k, (ax, matrix, title) in enumerate(zip(
        axes, [f2[0], f2_star[0], f2[1], f2_star[1]],
        ["$f_0$ (empties mode 0)", "$f_0^*$ (fills mode 0)",
         "$f_1$ (empties mode 1)", "$f_1^*$ (fills mode 1)"])):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
```

```
    for r in range(4):
        for c in range(4):
            if matrix[r, c]:
                ax.text(c, r, f"{matrix[r, c]:+d}", ha="center", va="center",
                        color="white", fontweight="bold")
```

For each of the four operators: its matrix as a heat map, and its nonzero entries written in their squares.

```
    ax.set_xticks(range(4), PATTERNS)
    # the row labels only on the first picture (the rows are the same in all four)
    ax.set_yticks(range(4), PATTERNS if k == 0 else [""] * 4)
    ax.set_xlabel("from the pattern")
    if k == 0:
        ax.set_ylabel("to the pattern")
    ax.set_title(title)
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "two_modes",
            ...)
```

The columns are labelled by the pattern acted on, the rows by the resulting pattern (only in the first picture). **What figure 05e.1 shows**: each operator moves one pattern to another, so each picture has two coloured squares; all are red (+1) except one blue square in f_1 (from $|11\rangle$ to $|10\rangle$) and one in f_1^* (from $|10\rangle$ to $|11\rangle$): the sign that makes operators of different modes anticommute.

In [6], sixteen modes.

```
def sign_below(n, p):
    return -1 if bin(n & ((1 << p) - 1)).count("1") % 2 else 1
```

`sign_below(n, p)` is (-1) to the power of the number of occupied modes below p in the pattern n: -1 when that number is odd (`% 2` is its remainder after division by 2).

```
def annihilate(p, state):
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:  # mode p is full in the pattern n
            new = n ^ (1 << p)  # the same pattern with mode p emptied
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:  # mode p is empty in the pattern n
            new = n | (1 << p)  # the same pattern with mode p filled
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result
```

With 16 modes there are 65536 patterns, too many for full matrices; a state is therefore stored as a dictionary `{pattern: amplitude}` that lists only the nonzero amplitudes. `annihilate(p, state)` is f_p: for every pattern in which mode p is full it adds the amplitude, times the sign, to the pattern with mode p emptied (`result.get(new, 0)` is the amplitude collected so far, 0 if none). `create(p, state)` is f_p^*: for every pattern in which mode p is empty it fills the mode (the operator `|`, the binary or, applied with the single-digit number sets digit p to 1).

```
def combine(terms):
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result


def inner(left, right):
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    return max((abs(a) for a in state.values()), default=0.0)
```

`combine` adds states with coefficients, $\sum c_j \,\text{state}_j$. `inner(left, right)` is the positive inner product $\sum_n \text{left}_n^* \,\text{right}_n$. `largest(state)` is the largest size of an amplitude, 0 for the empty dictionary (the zero state); it measures how far a state is from zero.

```
rng = np.random.default_rng(12345)  # random numbers with a fixed seed
RANDOM_STATE = {int(n): complex(rng.normal(), rng.normal())
                for n in rng.integers(0, 2 ** 16, size=6)}  # six random patterns
```

A random test state: six random patterns (whole numbers from 0 to 65535) with random complex amplitudes whose real and imaginary parts come from the normal distribution (the bell curve centred at 0). The fixed seed 12345 makes every run use the same numbers.

```
worst = 0.0  # the largest violation of a canonical relation
for p in range(16):
    for q in range(16):
        mixed = combine([(1, annihilate(p, create(q, RANDOM_STATE))),
                         (1, create(q, annihilate(p, RANDOM_STATE)))])
        expected = RANDOM_STATE if p == q else {}  # delta_pq times the state
        worst = max(worst, largest(combine([(1, mixed), (-1, expected)])))
        same = combine([(1, annihilate(p, annihilate(q, RANDOM_STATE))),
                        (1, annihilate(q, annihilate(p, RANDOM_STATE)))])
        worst = max(worst, largest(same))
say(f"patterns in the random state: {len(RANDOM_STATE)}; pairs tested: 2 x 256")
check(worst < 1e-12,
      "sixteen modes: {f_p, f_q^*} = delta_pq and {f_p, f_q} = 0 on a random state")
```

For all 16×16 pairs (p, q): $\{f_p, f_q^*\}$ applied to the random state minus δ_{pq} times the state, and $\{f_p, f_q\}$ applied to it; `worst` keeps the largest amplitude left over. The printed line reports 6 patterns and 2×256 tests; the check requires everything left over to be below 10^{-12}.

In [7], the mode Hamiltonian and its eigenvectors.

```
MASS, MOMENTUM = 3, 4  # the record's example: m = 3, momentum 4 along x1
E = float(np.sqrt(MASS ** 2 + MOMENTUM ** 2))  # the energy, 5
h = -1j * MASS * gamma["x4"] - MOMENTUM * (gamma["x4"] @ gamma["x1"])
```

The record's example: $m = 3$, momentum 4 along $x1$, $E = \sqrt{3^2 + 4^2} = 5$, and $h = -im\gamma^{(x4)} - 4\gamma^{(x4)}\gamma^{(x1)}$ (Section 5.34).

```
def orthonormal_columns(P):
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):  # a second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e  # remove the part along e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:  # a new direction: keep it with length 1
            basis.append(v / length)
    return np.array(basis).T
```

The **Gram-Schmidt procedure**: it goes through the columns of a matrix P (the rows of `P.T`), subtracts from each column its parts along the columns already kept ($e^\dagger v$ is the size of the part of v along the unit column e), and keeps it, divided by its length, when the length that remains is not zero. The subtraction is done twice, because the first pass leaves rounding errors. The kept columns are orthonormal (of length 1 and perpendicular to each other) and span the same space as the columns of P.

```
U_plus = orthonormal_columns((np.eye(16) + h / E) / 2)  # u_1..u_8: energy +5
V_minus = orthonormal_columns((np.eye(16) - h / E) / 2)  # v_1..v_8: energy -5
W = np.hstack([U_plus, V_minus])  # 16 x 16: the columns u_1..u_8, v_1..v_8
report("E for m = 3 and momentum 4 along x1", f"{E:.12f}")
```

Because $hh = E^2\,1$, the matrices $\frac{1}{2}(1 \pm h/E)$ are projectors onto the eigenvectors with the eigenvalues $\pm E$: $h \cdot \frac{1}{2}(1 + h/E) = \frac{1}{2}(h + E\,1) = E \cdot \frac{1}{2}(1 + h/E)$, so every column of the first is an eigenvector with eigenvalue $+E$ (or zero), and in the same way for the second with $-E$. Gram-Schmidt picks eight orthonormal columns from each; `np.hstack` puts them side by side into W. The RESULT line prints $E = 5.000000000000$.

```
check(np.max(np.abs(h - h.conj().T)) < 1e-15
      and np.max(np.abs(h @ h - 25 * np.eye(16))) < 1e-13
      and U_plus.shape == (16, 8) and V_minus.shape == (16, 8)
      and np.max(np.abs(h @ U_plus - E * U_plus)) < 1e-13
      and np.max(np.abs(h @ V_minus + E * V_minus)) < 1e-13
      and np.max(np.abs(W.conj().T @ W - np.eye(16))) < 1e-13
      and np.max(np.abs(W @ W.conj().T - np.eye(16))) < 1e-13,
      "h is Hermitian, h h = 25, with 8 + 8 orthonormal complete eigenvectors")
```

The check: h is Hermitian, $hh = 25 \cdot 1$, eight columns for each energy, $hu_s = 5u_s$ and $hv_s = -5v_s$, and $W^\dagger W = WW^\dagger = 1$ (orthonormal and complete), each to a small floating-point tolerance.

In [8], the field operators and the canonical rule.

```
def basic(i, state):
    p = i % 16  # the mode
    empties = (i < 16) == (p < 8)  # F_p for p < 8 and F_p^* for p >= 8 empty mode p
    return annihilate(p, state) if empties else create(p, state)
```

Every operator used here is a combination of 32 **basic operators** O_i: $O_p = F_p$ and $O_{16+p} = F_p^*$ for $p = 0, \ldots, 15$, where $F_p = b_{p+1} = f_p$ for $p < 8$ (particles) and $F_p = d_{p-7}^* = f_p^*$ for $p \geq 8$ (antiparticles appear in Ψ through their creation operators), and F_p^* is the Hilbert adjoint of F_p. `basic(i, state)` applies O_i: the mode is $p = i \bmod 16$, and O_i empties mode p exactly when ($i < 16$ and $p < 8$) or ($i \geq 16$ and $p \geq 8$), which is what the comparison of the two truth values `(i < 16) == (p < 8)` says.

```
ZEROS = np.zeros((16, 16))
PSI = np.hstack([W, ZEROS])  # row A: Psi_A = sum_p W_Ap F_p
CHI = np.hstack([ZEROS, W.conj()])  # row A: chi_A = sum_p conj(W_Ap) F_p^*
PSI_DAG = B.T @ CHI  # row A: Psi^dagger_A = sum_C chi_C B_CA
```

An operator $\sum_i c_i O_i$ is stored as the row of its 32 coefficients. The field $\Psi_A = \sum_p W_{Ap} F_p$ (Section 5.34 with the columns of W) is row A of `PSI`: the 16 numbers $W_{A,p}$ followed by 16 zeros. Its Hilbert adjoint $\chi_A = \sum_p W_{Ap}^* F_p^*$ is row A of `CHI`. The canonical conjugate $\Psi_A^\dagger = \sum_C \chi_C B_{CA} = \sum_C (B^T)_{AC} \chi_C$ is row A of B^T times `CHI`.

```
def apply(row, state):
    return combine([(row[i], basic(i, state)) for i in range(32)
                    if abs(row[i]) > 1e-15])


def anticommutator(row1, row2, state):
    return combine([(1, apply(row1, apply(row2, state))),
                    (1, apply(row2, apply(row1, state)))])
```

`apply(row, state)` applies the operator of a row to a state (coefficients below 10^{-15} are skipped). `anticommutator(row1, row2, state)` applies $\{X, Y\} = XY + YX$.

```
def measured_rule(rows1, rows2, state):
    norm = inner(state, state).real
    numbers = np.zeros((16, 16), dtype=complex)
    rest = 0.0
    for A in range(16):
        for C_ in range(16):
            result = anticommutator(rows1[A], rows2[C_], state)
            numbers[A, C_] = inner(state, result) / norm
            rest = max(rest, largest(combine([(1, result),
                                              (-numbers[A, C_], state)])))
    return numbers, rest
```

`measured_rule` measures the 256 anticommutators $\{X_A, Y_C\}$ on a state. Each must be a number times the state; the number is read off as $\langle\phi|\{X,Y\}\phi\rangle/\langle\phi|\phi\rangle$, and `rest` keeps the largest amplitude left over after that number times the state is subtracted (it must be zero). The letter `C_` avoids overwriting the matrix `C`.

```
with_chi, rest_chi = measured_rule(PSI, CHI, RANDOM_STATE)
with_dagger, rest_dagger = measured_rule(PSI, PSI_DAG, RANDOM_STATE)
psi_psi, rest_pp = measured_rule(PSI, PSI, RANDOM_STATE)
dag_dag, rest_dd = measured_rule(PSI_DAG, PSI_DAG, RANDOM_STATE)
all_rest = max(rest_chi, rest_dagger, rest_pp, rest_dd)
say("every anticommutator is a number times the state (rest below 1e-12): "
    f"{all_rest < 1e-12}")
```

Four families of anticommutators are measured on the random state: $\{\Psi_A, \chi_C\}$, $\{\Psi_A, \Psi_C^\dagger\}$, $\{\Psi_A, \Psi_C\}$ and $\{\Psi_A^\dagger, \Psi_C^\dagger\}$. The printed line says True: each is a number times the state.

```
check_reproduces(np.max(np.abs(with_chi - np.eye(16))) < 1e-12
                 and np.max(np.abs(with_dagger - B)) < 1e-12
                 and np.max(np.abs(psi_psi)) < 1e-12
                 and np.max(np.abs(dag_dag)) < 1e-12 and all_rest < 1e-12
                 and recorded("wolfram", "Fock_space_good_sector_example"),
                 "{Psi_A, chi_C} = delta_AC and {Psi_A, Psi^dagger_C} = B_AC on the "
                 "positive Fock space; {Psi, Psi} = 0 = {Psi^dagger, Psi^dagger}",
                 record=record_of("wolfram", "Fock_space_good_sector_example"))
```

The check: the measured numbers are the identity for χ, the matrix B for $\Psi^\dagger$, and zero for the other two families, exactly as Section 5.34 derived.

In [9], the picture of the measured rules.

```
def heat_map(ax, matrix, title, row_label=True):
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.set_yticks([0, 3, 7, 11, 15], ["1", "4", "8", "12", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)
    return image
```

The heat map of Notebook 05c, In [4].

```
check(np.max(np.abs(with_chi.imag)) < 1e-12 and np.max(np.abs(with_dagger.real))
      < 1e-12, "the measured numbers are real for chi and imaginary for Psi^dagger")
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], with_chi.real, r"measured $\{\Psi_A, \chi_C\}$")
```

```
heat_map(axes[1], with_dagger.imag, r"measured $\{\Psi_A, \Psi^\dagger_C\}$ / $i$",
         row_label=False)
image = heat_map(axes[2], B.imag, r"the matrix $B$ / $i$", row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "anticommutators",
            ...)
```

Before drawing, the check confirms what the pictures show: the numbers for χ are real and those for $\Psi^\dagger$ purely imaginary, so the real part of the first and the imaginary part of the second say everything. **What figure 05e.2 shows**: on the left the identity (a red diagonal); in the middle and on the right the same pattern of red and blue squares, B/i, with an empty diagonal. The realisation obeys the canonical rule.

In [10], why the canonical conjugate is not the Hilbert adjoint.

```
u = np.zeros(16, dtype=complex)
u[6], u[15] = -1j / np.sqrt(2), 1 / np.sqrt(2)  # u_7 and u_16 (Python counts from 0)
X_row = u.conj() @ PSI  # X = sum_A conj(u_A) Psi_A
X_adjoint = u @ CHI  # X^* = sum_A u_A chi_A (the Hilbert adjoint)
X_dagger = u @ PSI_DAG  # X^dagger = sum_A u_A Psi^dagger_A (canonical conjugate)
```

The record's column u ($u_7 = -i/\sqrt{2}$, $u_{16} = 1/\sqrt{2}$), the operator $X = \sum_A u_A^* \Psi_A$ as a row (a combination of the rows of `PSI`), its Hilbert adjoint $X^* = \sum_A u_A \chi_A$ and its canonical conjugate $X^\dagger = \sum_A u_A \Psi_A^\dagger$.

```
norm_random = inner(RANDOM_STATE, RANDOM_STATE).real
with_adjoint = inner(RANDOM_STATE, anticommutator(X_row, X_adjoint,
                                                  RANDOM_STATE)) / norm_random
with_canonical = inner(RANDOM_STATE, anticommutator(X_row, X_dagger,
                                                    RANDOM_STATE)) / norm_random
eigen_B = np.linalg.eigvalsh(B)  # the 16 eigenvalues of the Hermitian matrix B
say(f"{{X, X^*}} = {with_adjoint.real:+.6f};  {{X, X^dagger}} = "
    f"{with_canonical.real:+.6f};  u^dagger B u = {(u.conj() @ B @ u).real:+.6f}")
```

The two anticommutators are measured on the random state as numbers, and the 16 eigenvalues of B are computed. In an f-string a doubled brace `{{` prints a single brace. The printed line shows $\{X, X^*\} = +1.000000$, $\{X, X^\dagger\} = -1.000000$ and $u^\dagger B u = -1.000000$.

```
check_reproduces(np.allclose(B @ u, -u) and abs(u.conj() @ B @ u + 1) < 1e-12
                 and abs(with_adjoint - 1) < 1e-12 and abs(with_canonical + 1) < 1e-12
                 and recorded("wolfram", "no_positive_inner_product"),
                 "{X, X^*} = +1 but {X, X^dagger} = u^dagger B u = -1: the canonical "
                 "conjugate needs an indefinite (Krein) inner product",
                 record=record_of("wolfram", "no_positive_inner_product"))
```

The check: $Bu = -u$, $u^\dagger B u = -1$, the anticommutator with the Hilbert adjoint is $+1$ (positive, as it must be), and the canonical one is -1: the argument of Section 5.34 on the computer.

In [11], the picture of the indefinite form.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0))
axes[0].plot(np.arange(1, 17), eigen_B, "o", color="#2a78d6", markersize=8)
axes[0].axhline(0.0, color="black", linewidth=0.8)
axes[0].set_xticks([1, 4, 8, 12, 16])
axes[0].set_xlabel("number of the eigenvalue (sorted)")
axes[0].set_ylabel("eigenvalue of $B$")
axes[0].set_title("$B$: eight eigenvalues $-1$, eight $+1$")
```

The left picture: the 16 sorted eigenvalues of B as dots.

```
axes[1].bar([0, 1], [with_adjoint.real, with_canonical.real],
            color=["#1baf7a", "#eb6834"], edgecolor="white", linewidth=2, width=0.5)
for x_bar, value in zip([0, 1], [with_adjoint.real, with_canonical.real]):
    axes[1].text(x_bar, value + (0.08 if value > 0 else -0.16), f"{value:+.0f}",
                 ha="center")
axes[1].axhline(0.0, color="black", linewidth=0.8)
axes[1].set_xticks([0, 1], [r"$\{X, X^*\}$ (Hilbert adjoint)",
                            r"$\{X, X^\dagger\}$ (canonical)"])
axes[1].set_ylim(-1.5, 1.5)
axes[1].set_ylabel("measured number")
axes[1].set_title("$X = u^\\dagger\\Psi$ with $Bu = -u$")
save_figure(fig, "why_krein",
            ...)
```

The right picture: two bars with their values written beside them. **What figure 05e.3 shows**: on the left eight dots at -1 and eight at $+1$, the signature $(8, 8)$ of B; on the right a green bar up to $+1$ (the Hilbert adjoint) and an orange bar down to -1 (the canonical conjugate). A negative value is impossible for a Hilbert adjoint, which is why the canonical rule needs an indefinite (Krein) inner product.

In [12], the vacuum, the 16 quanta and the expectation-value rule.

```
VACUUM = {0: 1.0}  # the pattern 0: no particle, no antiparticle


def bilinear(L, K, R, state):
    Q = L.T @ K @ R  # 32 x 32: the coefficient of O_i O_j
    terms = []
    for j in range(32):
        if np.max(np.abs(Q[:, j])) > 1e-15:
            lowered = basic(j, state)  # O_j applied first
            if lowered:
                terms += [(Q[i, j], basic(i, lowered)) for i in range(32)
                          if abs(Q[i, j]) > 1e-15]
    return combine(terms)
```

The vacuum is the pattern 0 with amplitude 1. A bilinear $\sum_{A,C} L_A K_{AC} R_C$ of two operators given by rows L_A and R_C is $\sum_{i,j} Q_{ij} O_i O_j$ with the 32×32 matrix $Q = L^T K R$ (insert $L_A = \sum_i L_{Ai} O_i$ and $R_C = \sum_j R_{Cj} O_j$). `bilinear(L, K, R, state)` applies it to a state: for each j with a nonzero column of Q it applies O_j first, and if the result is not the zero state (an empty dictionary counts as false), it applies every O_i with its coefficient.

```
def vacuum_value(L, K, R):
    return inner(VACUUM, bilinear(L, K, R, VACUUM))


QUANTA = [create(p, VACUUM) for p in range(16)]  # b_s^*|0> (p < 8), d_s^*|0>


def normal_ordered_value(L, K, R, state):
    return inner(state, bilinear(L, K, R, state)) - vacuum_value(L, K, R)
```

`vacuum_value` is $\langle 0|X|0\rangle$. `QUANTA` lists the 16 one-quantum states: filling mode p of the vacuum gives $b^*_{p+1}|0\rangle$ for $p < 8$ and $d^*_{p-7}|0\rangle$ for $p \geq 8$. `normal_ordered_value` is $\langle q|X|q\rangle - \langle 0|X|0\rangle$, the value of $:X:$ in a state q of length 1.

```
vacuum_energy = vacuum_value(CHI, h, PSI)
energies = np.array([normal_ordered_value(CHI, h, PSI, q) for q in QUANTA])
vacuum_charge = vacuum_value(PSI_DAG, B, PSI)
charges = np.array([normal_ordered_value(PSI_DAG, B, PSI, q) for q in QUANTA])
report("vacuum energy <0|chi h Psi|0> (the filled sea)", f"{vacuum_energy.real:.6f}")
report("vacuum charge <0|Psi^dagger B Psi|0>", f"{vacuum_charge.real:.6f}")
say("normal-ordered energies of the 16 quanta: "
    + " ".join(f"{e.real:+.0f}" for e in energies))
say("normal-ordered charges of the 16 quanta:  "
    + " ".join(f"{c.real:+.0f}" for c in charges))
```

The energy operator is $\chi h \Psi$ and the charge operator $\Psi^\dagger B \Psi$ (which equals $\chi B B \Psi = \chi \Psi$). The RESULT lines print the vacuum energy -40.000000 ($-8E$, the filled sea) and the vacuum charge 8.000000; the two printed lists show the normal-ordered energy $+5$ for all 16 quanta, and the charge $+1$ for the eight particles and -1 for the eight antiparticles.

```
rule_ok = True
for K_rule in [C, -1j * (C @ gamma["x4"]), -1j * (C @ gamma["x1"])]:
    values = [normal_ordered_value(PSI_DAG, K_rule, PSI, q) for q in QUANTA]
    predicted = ([W[:, s].conj() @ B @ K_rule @ W[:, s] for s in range(8)]
                 + [-(W[:, s].conj() @ B @ K_rule @ W[:, s]) for s in range(8, 16)])
    rule_ok &= np.max(np.abs(np.array(values) - np.array(predicted))) < 1e-12
```

The expectation-value rule of the record: $\langle q| :\Psi^\dagger K \Psi: |q\rangle = u_s^\dagger B K u_s$ for a particle and $-v_s^\dagger B K v_s$ for an antiparticle (`W[:, s]` is column s of W), tested for three of the record's matrices: the scalar, the charge density and the current along $x1$. `&=` keeps `rule_ok` true only while every test passes.

```
check_reproduces(abs(vacuum_energy + 40) < 1e-12 and np.max(np.abs(energies - 5)) < 1e-12
                 and abs(vacuum_charge - 8) < 1e-12
                 and np.max(np.abs(charges - np.array([1] * 8 + [-1] * 8))) < 1e-12
                 and rule_ok and "-8 E" in detail("wolfram",
                                                  "Fock_space_good_sector_example"),
                 "vacuum energy -40, every quantum +5, charges +1 and -1, and the "
                 "expectation-value rule for M = C, -i C gamma^(x4), -i C gamma^(x1)",
                 record=record_of("wolfram", "Fock_space_good_sector_example"))
```

The check collects the numbers and requires that the record's detail text contains `-8 E`, the vacuum energy it states.

In [13], the picture of the quanta.

```
raw_energies = energies + vacuum_energy  # <q|chi h Psi|q>
raw_charges = charges + vacuum_charge  # <q|Psi^dagger B Psi|q>
index = np.arange(1, 17)
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
```

The values in the states before normal ordering are the normal-ordered values plus the vacuum values.

```
for ax, raw, ordered, name in [(axes[0], raw_energies, energies, "energy"),
                               (axes[1], raw_charges, charges, "charge")]:
    ax.bar(index - 0.2, raw.real, 0.4, color="#9e9c98", label="value in the state")
    ax.bar(index + 0.2, ordered.real, 0.4, color="#2a78d6",
           label="after normal ordering")
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.axvline(8.5, color="black", linewidth=0.8, linestyle=":")
    ax.set_xticks([1, 4, 8, 9, 12, 16])
    ax.set_xlabel("quantum: 1 to 8 particles, 9 to 16 antiparticles")
    ax.set_title(f"the {name} of the 16 quanta")
axes[0].set_ylabel("value")
handles, names = axes[0].get_legend_handles_labels()  # one legend for both
fig.legend(handles, names, loc="lower center", bbox_to_anchor=(0.5, -0.1), ncol=2)
save_figure(fig, "quanta",
            ...)
```

For the energy (left) and the charge (right), a grey bar (value in the state) and a blue bar (after normal ordering) for each quantum, and a dotted line between particles and antiparticles; one legend for both pictures (`get_legend_handles_labels` collects the labelled bars of the first picture). **What figure 05e.4 shows**: the grey energy bars at -35 ($= -40 + 5$) and the blue ones at $+5$; the grey charge bars at 9 for particles and 7 for antiparticles, the blue ones at $+1$ and -1. The student should see that the vacuum values are large and that normal ordering subtracts them, leaving the physical numbers.

In [14], the two conjugations of the operators.

```
MAPS = {"M = 1": I16, "M = Gamma": Gamma}  # the two conjugation matrices
conjugated = {}  # name -> (rows of Psi', rows of Psi'^dagger)
measured_conjugated = {}  # name -> the measured {Psi'_A, Psi'^dagger_C}
```

The two maps of Section 5.34 and two dictionaries for the results.

```
for name, M in MAPS.items():
    rows_prime = M @ PSI_DAG  # Psi'_A = sum_C M_AC Psi^dagger_C
    rows_prime_dagger = M.conj() @ PSI  # Psi'^dagger_A = sum_C conj(M_AC) Psi_C
    conjugated[name] = (rows_prime, rows_prime_dagger)
```

```
    numbers, rest = measured_rule(rows_prime, rows_prime_dagger, RANDOM_STATE)
    measured_conjugated[name] = numbers
    predicted = M @ B.T @ M.conj().T
    say(f"{name:9}: measured equals M B^T M^dagger: "
        f"{np.max(np.abs(numbers - predicted)) < 1e-12 and rest < 1e-12}; "
        f"it equals +B: {np.max(np.abs(numbers - B)) < 1e-12}; "
        f"-B: {np.max(np.abs(numbers + B)) < 1e-12}")
```

For each map the rows of the conjugated field and of its canonical conjugate are formed (a matrix times the rows gives the rows of the combinations), the 256 anticommutators are measured on the random state and compared with the prediction $MB^TM^\dagger$. The two printed lines: for $M = 1$ the measurement equals the prediction and $-B$; for $M = \Gamma$ it equals the prediction and $+B$.

```
check_reproduces(np.max(np.abs(measured_conjugated["M = 1"] + B)) < 1e-12
                 and np.max(np.abs(measured_conjugated["M = Gamma"] - B)) < 1e-12
                 and recorded("lead", "quantum_charge_conjugation_unitary_type"),
                 "on the Fock space: Psi' = Psi^(dagger T) gives -B, Psi' = Gamma "
                 "Psi^(dagger T) gives +B: only M = Gamma keeps the canonical rule",
                 record=record_of("lead", "quantum_charge_conjugation_unitary_type"))
```

The check reproduces the lead check on actual operators: of the two maps, only $M = \Gamma$ keeps the canonical rule (Section 5.34 adds that a phase factor $e^{i\alpha}$ in front of Γ changes nothing).

In [15], the picture of the conjugated rules.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
heat_map(axes[0], B.imag, r"$B$ / $i$ (the rule)")
heat_map(axes[1], measured_conjugated["M = 1"].imag,
         r"measured, $M = 1$, / $i$", row_label=False)
image = heat_map(axes[2], measured_conjugated["M = Gamma"].imag,
                 r"measured, $M = \Gamma$, / $i$", row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "conjugated_rules",
            ...)
```

What figure 05e.5 shows: the rule B/i on the left; in the middle, measured for $M = 1$, every colour reversed ($-B$); on the right, measured for $M = \Gamma$, the same picture as on the left. Of the two matrices, only Γ gives a conjugation of the quantised field (up to a phase factor, Section 5.34).

In [16], the conjugated bilinears: a sign and a constant.

```
K_MATRICES = {"S": C.astype(complex)}  # the scalar S = Psi^dagger C Psi
for x in COORDS:
    K_MATRICES[f"J^({x})"] = -1j * (C @ gamma[x])  # the current J^a
BILINEAR_NAMES = list(K_MATRICES)  # "S", "J^(x0)", ..., "J^(x7)"
signs, constants = {}, {}  # (map, bilinear) -> s and c
identity_ok = True
```

The nine matrices: the scalar and the eight currents, under their names.

```
for map_name, M in MAPS.items():
    rows_prime, rows_prime_dagger = conjugated[map_name]
    for bname, K in K_MATRICES.items():
        K_prime = M.conj().T @ K @ M  # K' = M^dagger K M
        c = np.trace(K_prime @ B.T)  # the predicted constant
        s = 1 if np.array_equal(-K_prime.T, K) else (
            -1 if np.array_equal(-K_prime.T, -K) else 0)  # -K'^T = s K
        X_prime = bilinear(rows_prime_dagger, K, rows_prime, RANDOM_STATE)
        X = bilinear(PSI_DAG, K, PSI, RANDOM_STATE)
        difference = combine([(1, X_prime), (-s, X), (-c, RANDOM_STATE)])
        identity_ok &= s != 0 and largest(difference) < 1e-10
        signs[(map_name, bname)], constants[(map_name, bname)] = s, c
```

For each map and each of the nine bilinears: the predicted constant $c = \mathrm{tr}(K' B^T)$ and the predicted sign s with $-K'^T = sK$ (Section 5.34); then the conjugated operator $X' = \Psi'^\dagger K \Psi'$ and the original $X = \Psi^\dagger K \Psi$ are applied to the random state, and the state $X'\phi - sX\phi - c\,\phi$ must vanish. `identity_ok` stays true only if every sign is ± 1 and every difference is below 10^{-10}.

```
for map_name in MAPS:
    say(f"{map_name:9}: s = " + " ".join(f"{signs[(map_name, b)]:+d}"
                                         for b in BILINEAR_NAMES)
        + "  (S, J^(x0), ..., J^(x7))")
    say(f"{'':9}  c = " + " ".join(f"{constants[(map_name, b)].real:+.0f}"
                                   for b in BILINEAR_NAMES))
```

Two lines per map: the nine signs and the nine constants (in the second line the empty string, padded to nine characters, prints nine blanks, so that the two lines line up). The printed result: for $M = 1$ the signs -1 for S and $+1$ for all eight currents, for $M = \Gamma$ the sign -1 for all nine; the constants are 0 except -16 ($M = 1$) and $+16$ ($M = \Gamma$) for $J^{(x4)}$.

```
lead_detail = detail("lead", "bilinears_under_charge_conjugation")
recorded_table = json.loads(lead_detail.split("measured: ", 1)[1])
anticommuting_rows = {"M = 1": recorded_table["plus,eps=-1"],
                      "M = Gamma": recorded_table["minus,eps=-1"]}
rows_agree = all([signs[(m_name, "S")], [signs[(m_name, b)] for b in
                                         BILINEAR_NAMES[1:]]] == row
                 for m_name, row in anticommuting_rows.items())
```

The record's measured table is read as in Notebook 05c, In [14]; its two anticommuting rows (keys `plus,eps=-1` and `minus,eps=-1`) are compared with the operator signs, written in the same shape [sign of S, list of the eight signs of the currents].

```
check(identity_ok, "X' = s X + c holds on the Fock space for all 9 bilinears and both "
      "maps, with c = tr(K' B^T)")
check_reproduces(rows_agree and recorded("lead", "bilinears_under_charge_conjugation"),
                 "the operator signs s equal the measured signs of the anticommuting "
                 "rows (eps = -1) of the recorded table",
                 record=record_of("lead", "bilinears_under_charge_conjugation"))
```

Two checks: the operator identity $X' = sX + c$ (the derivation of Section 5.34, confirmed on the Fock space), and that the operator signs are those of the record's anticommuting rows.

In [17], the constants and the vacuum values.

```
nonzero = sorted({b for (m_name, b), c in constants.items() if abs(c) > 1e-12})
check(nonzero == ["J^(x4)"] and abs(constants[("M = 1", "J^(x4)")] + 16) < 1e-12
      and abs(constants[("M = Gamma", "J^(x4)")] - 16) < 1e-12,
      "c = 0 except for the charge density J^(x4): c = -16 (M = 1), +16 (M = Gamma)")
```

The set of bilinears with a nonzero constant (sorted, so that the printed order never changes) must be the charge density alone, with $c = -16$ and $+16$, as Section 5.34 computed.

```
vac_original = {b: vacuum_value(PSI_DAG, K, PSI) for b, K in K_MATRICES.items()}
vac_conjugated = {}  # (map, bilinear) -> <0|X'|0>
for map_name in MAPS:
    rows_prime, rows_prime_dagger = conjugated[map_name]
    for bname, K in K_MATRICES.items():
        vac_conjugated[(map_name, bname)] = vacuum_value(rows_prime_dagger, K,
                                                         rows_prime)
```

The vacuum values $\langle 0|X|0\rangle$ of the nine original bilinears and $\langle 0|X'|0\rangle$ of the 18 conjugated ones.

```
# values below 1e-12 are printed as 0 (rounding could otherwise print "-0.00")
shown = {b: (v.real if abs(v) > 1e-12 else 0.0) for b, v in vac_original.items()}
say("vacuum values <0|X|0>: " + ", ".join(f"{b} {shown[b]:+.2f}"
                                          for b in BILINEAR_NAMES))
check(all(abs(vac_conjugated[key] - signs[key] * vac_original[key[1]]
              - constants[key]) < 1e-12 for key in vac_conjugated),
      "<0|X'|0> = s <0|X|0> + c for all 9 bilinears and both maps")
```

The printed line lists the vacuum values: S -4.80, $J^{(x1)}$ -6.40, $J^{(x4)}$ $+8.00$ and 0 for the others; the filled sea of the example has a nonzero scalar and a nonzero current along its momentum $x1$. The check: $\langle 0|X'|0\rangle = s\langle 0|X|0\rangle + c$ for all 18 cases (`key[1]` is the name of the bilinear in the pair `key`).

```
positions = np.arange(len(BILINEAR_NAMES))
labels = ["$S$"] + [rf"$J^{{({x})}}$" for x in COORDS]
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.2), sharey=True)
for ax, map_name, title in [(axes[0], "M = 1", r"$\Psi' = \Psi^{\dagger T}$"),
                            (axes[1], "M = Gamma",
                             r"$\Psi' = \Gamma\Psi^{\dagger T}$")]:
    ax.bar(positions - 0.27, [vac_original[b].real for b in BILINEAR_NAMES], 0.27,
           color="#9e9c98", label=r"$\langle 0|X|0\rangle$")
    ax.bar(positions, [vac_conjugated[(map_name, b)].real for b in BILINEAR_NAMES],
           0.27, color="#2a78d6", label=r"$\langle 0|X'|0\rangle$")
    ax.bar(positions + 0.27, [constants[(map_name, b)].real for b in BILINEAR_NAMES],
           0.27, color="#eb6834", label="the constant $c$")
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, labels)
    ax.set_title(title)
```

```
    ax.legend(loc="lower left")
axes[0].set_ylabel("value")
save_figure(fig, "vacuum_values",
            ...)
```

For each map, three bars per bilinear: the vacuum value before (grey) and after (blue) the conjugation, and the constant c (orange). **What figure 05e.6 shows**: for $M = 1$ the scalar's vacuum value changes from -4.8 to $+4.8$, that of $J^{(x1)}$ stays at -6.4, and that of $J^{(x4)}$ goes from 8 to $8 - 16 = -8$; for $M = \Gamma$ all signs flip, and $J^{(x4)}$ goes from 8 to $-8 + 16 = 8$. The orange bars appear only at $J^{(x4)}$. Normal ordering subtracts each operator's own vacuum value, which removes c.

In [18], after normal ordering.

```
def apply_normal_ordered(L, K, R, state):
    return combine([(1, bilinear(L, K, R, state)),
                    (-vacuum_value(L, K, R), state)])
```

`apply_normal_ordered` applies $:X := X - \langle 0|X|0\rangle$ to a state.

```
normal_ok = True
for map_name in MAPS:
    rows_prime, rows_prime_dagger = conjugated[map_name]
    for bname, K in K_MATRICES.items():
        for state in [RANDOM_STATE] + QUANTA:
            left = apply_normal_ordered(rows_prime_dagger, K, rows_prime, state)
            right = apply_normal_ordered(PSI_DAG, K, PSI, state)
            normal_ok &= largest(combine([(1, left),
                                          (-signs[(map_name, bname)], right)])) < 1e-10
check(normal_ok, ":X': = s :X: for all 9 bilinears, both maps, on the random state "
      "and the 16 quanta")
```

For both maps, all nine bilinears and 17 states (the random state and the 16 quanta) the cell compares $:X':$ applied to the state with s times $:X:$ applied to it. The check: normal ordering removes the constant and keeps the sign, everywhere.

```
def classical_sign(M, K, eps):
    new = eps * (M.conj().T @ K @ M).T
    return 1 if np.array_equal(new, K) else (-1 if np.array_equal(new, -K) else 0)


tables = {"commuting components (classical)": 1,
          "anticommuting components (classical)": -1}
grids = {}
for title, eps in tables.items():
    grids[title] = np.array([[classical_sign(M, K_MATRICES[b], eps)
                              for b in BILINEAR_NAMES] for M in MAPS.values()])
grids["quantised field after normal ordering"] = np.array(
    [[signs[(m_name, b)] for b in BILINEAR_NAMES] for m_name in MAPS])
```

`classical_sign` is `sign_of` of Notebook 05c: the classical rule $K' = \epsilon(M^\dagger K M)^T$. Three tables of two rows (the maps) and nine columns (the bilinears) are built: classical commuting, classical anticommuting, and the operator signs of the quantised field.

```
check(np.array_equal(grids["quantised field after normal ordering"],
                     grids["anticommuting components (classical)"])
      and np.array_equal(grids["commuting components (classical)"],
                         -grids["anticommuting components (classical)"]),
      "after normal ordering the quantised field has the signs of the anticommuting "
      "components; the commuting ones are the opposite")
```

The check: the quantised table equals the classical anticommuting one, and the commuting one is its opposite.

```
fig, axes = plt.subplots(3, 1, figsize=(9.0, 6.6))
for ax, (title, grid) in zip(axes, grids.items()):
    ax.imshow(grid, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
    for r in range(2):
        for c_col in range(9):
            ax.text(c_col, r, f"{grid[r, c_col]:+d}", ha="center", va="center",
                    color="white", fontweight="bold")
    ax.axhline(0.5, color="white", linewidth=2)
    for k in range(1, 9):
        ax.axvline(k - 0.5, color="white", linewidth=3 if k == 1 else 2)
    ax.set_xticks(range(9), labels)
    ax.set_yticks([0, 1], [r"$\mathcal{C}_+$ type, $M = 1$",
                           r"$\mathcal{C}_-$ type, $M = \Gamma$"])
    ax.set_title(title)
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "sign_tables",
            ...)
```

Three sign tables, one above the other, drawn as in Notebook 05c, In [15] (`c_col` is used as the column counter, because `c` already names a constant); `fig.tight_layout()` spaces the three pictures so that their labels do not overlap. **What figure 05e.7 shows**: the top table (commuting) has a red S and blue currents for $M = 1$ and is all red for $M = \Gamma$; the middle (anticommuting) and the bottom (quantised field after normal ordering) are equal, each the top table with every colour reversed. Normal ordering adds no sign.

In [19], the charges of the quanta after the conjugation.

```
K_charge = K_MATRICES["J^(x4)"]  # equal to B
charge_values = {"original": charges.real}
for map_name in MAPS:
    rows_prime, rows_prime_dagger = conjugated[map_name]
    charge_values[map_name] = np.array(
        [normal_ordered_value(rows_prime_dagger, K_charge, rows_prime, q).real
         for q in QUANTA])
check(np.array_equal(K_charge, B)
      and np.max(np.abs(charge_values["M = Gamma"] + charges.real)) < 1e-12
      and np.max(np.abs(charge_values["M = 1"] - charges.real)) < 1e-12,
      "M = Gamma reverses the normal-ordered charge of every quantum; M = 1 keeps it")
```

The normal-ordered charge density of the conjugated field, $:J'^{(x4)}:$, is evaluated in each of the 16 one-quantum states, for both maps. The check: its matrix is B; with $M = \Gamma$ (the allowed

conjugation) every charge is reversed, particles $+1 \to -1$ and antiparticles $-1 \to +1$; with $M = 1$ (which breaks the canonical rule) every charge stays.

```
fig, ax = plt.subplots(figsize=(8.0, 4.2))
ax.plot(index, charge_values["original"], "o", color="#9e9c98", markersize=12,
        label=r"$:\!J^{(x4)}\!:$ (the field)")
ax.plot(index, charge_values["M = 1"], "s", color="#eb6834", markersize=6,
        label=r"$M = 1$ (breaks the rule)")
ax.plot(index, charge_values["M = Gamma"], "D", color="#2a78d6", markersize=7,
        label=r"$M = \Gamma$ (keeps the rule)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.axvline(8.5, color="black", linewidth=0.8, linestyle=":")
ax.set_xticks([1, 4, 8, 9, 12, 16])
ax.set_ylim(-1.6, 1.6)
ax.set_xlabel("quantum: 1 to 8 particles, 9 to 16 antiparticles")
ax.set_ylabel("normal-ordered charge")
ax.set_title("The charge of each quantum, and of its conjugates")
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.17), ncol=3)
save_figure(fig, "conjugated_charges",
            ...)
```

Large grey circles for the field, small orange squares (`"s"`) for $M = 1$ and blue diamonds (`"D"`) for $M = \Gamma$. **What figure 05e.8 shows**: the grey circles at $+1$ for quanta 1 to 8 and at -1 for 9 to 16; the orange squares sit inside the circles (unchanged); the blue diamonds are at -1 for the particles and $+1$ for the antiparticles. The one of the two conjugations that keeps the canonical rule exchanges the charges of particles and antiparticles, and, by In [16], it also reverses the scalar and with it the mass.

In [20], the last check.

```
FIGURES = ["05e_1_two_modes.png", "05e_2_anticommutators.png", "05e_3_why_krein.png",
           "05e_4_quanta.png", "05e_5_conjugated_rules.png",
           "05e_6_vacuum_values.png", "05e_7_sign_tables.png",
           "05e_8_conjugated_charges.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in FIGURES),
      "the eight figure files of notebook 05e exist")
all_checks_passed()
```

The eight figure files must exist, and the last line reads ALL 18 CHECKS PASSED (notebook 05e): one check each in In [2], In [3], In [4], In [6], In [7], In [8], In [9], In [10], In [12], In [14], In [19] and In [20], and two each in In [16], In [17] and In [18]. Five of them reproduce Revision records (In [8], In [10], In [12], In [14] and the second check of In [16]); the other thirteen are the notebook's own computations.

5.39 What the charge-conjugation matrices say, and do not say, about matter and antimatter

The request that this book answers asks for a theory that "solves matter anti-matter mysteries". The matrices of this chapter are the first tools for that question, so this section states exactly what

they establish and what they do not. Chapter 21 treats the question in full; Chapters 18 and 20 treat the pairs of universes.

What this chapter proves (exact maps between solutions).

- In this theory charge conjugation is a matrix map. With the definition of Section 5.28 (the matrix $M = \mathcal{C}C$ must obey $M(\gamma^a)^* = s\,\gamma^a M$ with one sign s for all eight gammas), there are exactly two charge-conjugation matrices, up to a factor, $\mathcal{C}_+ = C$ (same mass) and $\mathcal{C}_- = \Gamma C$ (mass reversed), and each maps every solution to a solution (Theorem CC, Section 5.28). The converse is not claimed: for $V = 0$ other matrices, such as $P_- C$, also map solutions to solutions.
- For the commuting complex field dirac16complex00, $\mathcal{C}_+$ maps every solution to a solution with the same mass and the opposite charge density (Section 5.29).
- For a real commuting field the currents vanish, $\mathcal{C}_+$ does nothing, and among the matrices that obey the intertwiner condition the only nontrivial one is Γ (up to a factor), with $(m, \lambda) \to (-m, -\lambda)$ (Section 5.29).
- For the quantised anticommuting field dirac16complex, among the matrices of Theorem CC only $\Psi \to \Gamma\Psi^{\dagger T}$, up to a phase factor, keeps the canonical anticommutator; after normal ordering it reverses the charge of every quantum and reverses the mass (Section 5.34).
- The map $\Psi \to \Gamma\Psi$ keeps the scalar and reverses every current (Section 5.5, (X7) and (X8)); it carries a solution with the parameters (m, λ) to one with $(-m, -\lambda)$ (Section 5.29, (F3); this is the input of the pairing theorem T1, which Chapter 18 proves). So a solution and its Γ image carry opposite charges, and the two together have total charge zero.
- The current obeys the local conservation law $\sum_\mu \partial_\mu(\sqrt{|g|}\, J^\mu) = 0$, $\sqrt{|g|} = b_4^{3/2} \cos z$, for every solution (record checks `u1_noether_matrix_identity` at $b_4 = 1$ and `u1_noether_matrix_identity_b4` for a general b_4, derived in Chapter 21): no process described by these field equations creates or destroys charge at any point; charge can only flow from one place to another. Hence the total charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, d^7x$ of one solution (at $b_4 = 1$; for a general b_4 it carries the factor $b_4^{3/2}$) is constant in time **if** its current flux through the boundary of the seven other directions vanishes (Section 5.6); this is a conditional statement, and its condition is not established (next list).

All of these are exact statements about **maps between sets of solutions** of the field equations.

What is not proved, and is not claimed.

- Nothing in this chapter shows that any universe is **created**, in pairs or otherwise. The maps relate solutions that are both allowed by the equations; they contain no creation process, no rate, no amplitude and no big-bang dynamics (Chapter 20).
- It is not proved that the total charge of one universe is constant in time. That needs the **no-flux condition** at the brane, $\cos z\, J^{x0} = 0$ at $z = \pi/2$, and the flux there is not zero in general (Section 5.6): exact solutions of Section 18.21, for example the one of Notebook 18c, carry charge through the brane, while the stationary members of that family do not. The condition is not derived for every solution (OPEN); the Kohn-Sham model of Chapter 14

ASSUMES it, through the Z2 mirror at the brane; the statement that no net charge is made inside one universe holds only where it is ASSUMED.

- The theory as built does **not** solve the matter–antimatter problem. The observed universe contains far more matter than antimatter (Chapter 21 explains the measurement from zero). In 1967 Sakharov showed that producing such an excess from a symmetric start needs three things: a process that changes the baryon number (roughly, the number of protons and neutrons minus the number of their antiparticles), a violation of the symmetries C and CP, and a departure from thermal equilibrium (Chapter 21 derives the three conditions). The theory as built has no baryons, no process that creates or destroys a charge at any point (the current obeys the local conservation law; the total Q of one universe is constant only under the no-flux condition at the brane, which is ASSUMED, not derived), no violation of CP built in or computed, and no computation of a departure from equilibrium. Chapter 21 lists what would have to be added.

- The idea that a universe and an anti-universe together carry zero charge belongs to a class of ideas in the published literature; one example is L. Boyle, K. Finn and N. Turok, "CPT-Symmetric Universe", Phys. Rev. Lett. 121, 251301 (2018). The pair-level statement above (a solution and its Γ image have total charge zero) is a statement of this kind about solutions. Any scenario in which our universe is one member of such a pair, or in which the pairing explains the observed excess of matter, is a **HYPOTHESIS**: nothing in this chapter or in the Revision record derives it.

5.40 What we proved, what we computed, what we assumed

PROVED (exact; the proof is in this chapter, and where a Revision record holds the fact, the report and check named confirm it independently; the notebooks named confirm it again):

- Six rules for products of different gammas (moving a gamma through a product, the square, the reverse, the transpose, the trace 0, the count of eigenvalues); the properties of $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$ ((C1) to (C7): symmetric, $CC = 1$, $C\gamma^a$ antisymmetric, $C\gamma^a C^{-1} = -(\gamma^a)^T$, signature $(8,8)$, $C = \mathrm{diag}(-\sigma, \sigma)$), of the chirality Γ ((X1) to (X9), $\Gamma = \mathrm{diag}(-1_8, 1_8)$, $\Gamma^T C\Gamma = C$, $\Gamma^T C\gamma^a\Gamma = -C\gamma^a$) and of $B = -iC\gamma^{(x4)}$ ((B1) to (B5): Hermitian, $BB = 1$, signature $(8,8)$, commuting with five gammas and anticommuting with the three of the extra times) (Sections 5.3 to 5.6; Notebook 05a; `Revision/algebra/reports/python-algebra.json` and `Revision/algebra/reports/wolfram-algebra.json`, the checks named in the tables of those sections).

- Theorem P: the 16 components carry an irreducible representation of Pin(4,4), and only the multiples of 1 commute with it; Theorem S: under Spin(4,4) they split into two irreducible, inequivalent halves of 8, and the commutant is spanned by P_- and P_+; the same holds for the 28 generators S^{ab} (Section 5.13; Notebook 05b; checks `clifford_products_span_M16`, `pin_commutant_dimension_1`, `even_products_span_M8_plus_M8`, `spin_commutant_dimension_2`, `spin_halves_irreducible`, `spin_halves_inequivalent`, `reflections_exchange_halves`).

- The so(4,4) rules, the vector rule, the key identity of reflections, the closed formulas of the exponentials, the half angle and $R(2\pi) = -1$, the invariance of the scalar under products

of exponentials, the form of B kept by exactly the 21 generators without $x4$, the spinor norm $g^T C g = (-1)^k N(g)\, C$, the double cover (the vector matrix fixes g up to the sign), and determinant $+1$ for every element of Pin(4,4) (Sections 5.12 and 5.18; Notebook 05d; checks `S_lorentz_algebra`, `S_vector_action`, `S_preserves_C_and_commutes_with_Gamma`, `S_preserves_B_only_off_x4`).

- What the scaled commutators generate: the products of their exponentials form exactly $\mathrm{Spin}_0(4,4)$, the piece of Pin(4,4) joined to 1; Pin(4,4) consists of four pieces, Spin_0, $\mathrm{Spin}_0\gamma^{(x0)}$, $\mathrm{Spin}_0\gamma^{(x4)}$ and $\mathrm{Spin}_0\gamma^{(x0)}\gamma^{(x4)}$; together with $\gamma^{(x0)}$ and $\gamma^{(x4)}$ the exponentials generate all of Pin(4,4) (Section 5.23; Notebook 05f).

- The local conservation law $\sum_\mu \partial_\mu(\sqrt{|g|}\, J^\mu) = 0$ of the record, $\sqrt{|g|} = b_4^{3/2} \cos z$, and from it the conditional statement: the total charge Q is constant in time for every solution whose current flux through the boundary of the seven other directions vanishes (Section 5.6; check `u1_noether_matrix_identity` of the lead report at $b_4 = 1$, whose detail text states that it proves the local law only, and check `u1_noether_matrix_identity_b4` of `Revision/theory/reports/python-field-theory-b4.json` for a general b_4). The flux through the brane $z = \pi/2$ is not zero in general, so the constancy of Q itself is not proved (see OPEN); it is zero for the stationary members $e^{iwx4}\chi$ of the family of exact solutions of Section 18.21, whose charge density is the same at every time (Section 5.6).

- Steps 1 to 4 of Section 5.28: every matrix M with $M(\gamma^a)^* = s\,\gamma^a M$ (one sign s for all a) maps every solution Ψ to the solution $M\Psi^*$ with sV; for $V = 0$ the matrix $P_- C$ maps every solution to a solution without obeying this condition. Theorem CC: with the condition as the definition, exactly two charge-conjugation matrices, up to a factor, $\mathcal{C}_+ = C$ (same mass, $\Psi^c = \Psi^*$) and $\mathcal{C}_- = \Gamma C$ (mass reversed, $\Psi^c = \Gamma\Psi^*$), with $\mathcal{C}_\pm^{-1}\gamma^a\mathcal{C}_\pm = \mp(\gamma^a)^T$; the sign table of the scalar and the currents for commuting and anticommuting components; both reality conditions consistent, the real one kept in time and the other only for $m = 0$; for real fields $J = 0$, $\mathcal{C}_+$ the identity and, among the matrices that obey the condition, Γ the nontrivial map with $(m, \lambda) \to (-m, -\lambda)$ (Sections 5.28 and 5.29; Notebook 05c; the lead report of charge conjugation, with the checks named in the tables of those two sections).

- For the quantised field: the canonical conjugate cannot be a Hilbert adjoint; among the matrices of Theorem CC only $\Psi \to \Gamma\Psi^{\dagger T}$, up to a phase factor, keeps the canonical anticommutator; every conjugated bilinear is $X' = sX + c$ with the sign of the classical anticommuting components and the constant $c = \mathrm{tr}(K'B^T)$; normal ordering removes c and keeps s (the detail text of the lead check `bilinears_under_charge_conjugation` states the same: normal ordering adds no sign); the allowed conjugation reverses the charge of every quantum and the mass (Section 5.34; Notebook 05e; checks `quantum_charge_conjugation_unitary_type` of the lead report and `no_positive_inner_product` of `Revision/theory/reports/wolfram-field-theory.json`).

COMPUTED (floating-point numbers, with the measured accuracy, or exact computations of the notebooks): the quadratic form of C along two paths, $\mp\sin 2t$ to 10^{-14} (05a); the exact ranks 256 and 128 of the products of different gammas, the commutant and intertwiner dimensions 1, 2, 1, 1, 0, 0 by exact ranks and again by the zero eigenvalues of $A^T A$, the halving sequence $256, 128, \ldots, 1$ and the negative control 4 (05b); the vector matrices and the closed formulas to 10^{-10} or better (05d); the sign pattern $(+,+)$ on 200 random products of exponentials (smallest determinant 1.000, reached by a product of six rotations), the four patterns on 240 random elements, ten paths that never

cross the band, the construction of step (I) on 40 random unit vectors and the decomposition of 12 random elements of Pin(4,4), and the spans 128 and 256 by singular values (05f); the exact solution spaces of the 2048 equations for each sign and the residuals of the free solution, below 10^{-12} where it solves and 11.83 where it does not, the values $S = -2$ and $J^{(x4)} = -6$, $+6$, -6 of the free solution and its images (exact at $x4 = 0$), and the violation $2|\sin(m\,x4)|$ of the condition $\Psi = \Gamma\Psi^*$ (05c); on the Fock space of 65536 states, the canonical rule to 10^{-12}, the vacuum energy -40, the energies $+5$ and the charges ±1, the vacuum values $S = -4.80$, $J^{(x1)} = -6.40$, $J^{(x4)} = 8$, the constants $c = \mp16$, and the reversed charges of the 16 quanta (05e). One observation is COMPUTED only and not used: $|\det A| = |\det D|$ for the 240 elements of Notebook 05f.

ASSUMED: the author's gammas, built from his formulas for T16 (Chapter 4), his coordinates and his metric (the input of the whole theory); the field equation $\gamma^\mu D_\mu \Psi = V\Psi$ of the Revision record (Chapter 7) and its canonical quantisation (Chapter 10); the theorem of Cartan and Dieudonné (every matrix of O(4,4) is a product of reflections), quoted without proof; three facts of analysis quoted without proof (the power series of the exponential converges for every square matrix; a linear differential equation has only one solution with a given starting value; the intermediate value theorem); and, for Notebook 05e, the scope of the example: one momentum without extra-time part, at one point of space, with the metric treated as flat near that point (the record's frozen-coefficient plane waves: constant gammas, no spin-connection term), and with the Fock vacuum of that momentum.

HYPOTHESIS and OPEN: this chapter derives no creation of universes and no explanation of the matter–antimatter asymmetry; every scenario built on the maps of this chapter is a HYPOTHESIS (Section 5.39). OPEN: the no-flux condition $\cos z\, J^{x0} = 0$ at the brane $z = \pi/2$, which a constant total charge of one universe needs; no equation of the Revision record or of this book derives it for every solution, and the field equations and the quantisation of the record impose no condition at the brane; the Kohn-Sham model of Chapter 14 ASSUMES the Z2 mirror at the brane, whose two conditions make the current along the hidden direction zero there (`Revision/kohn_sham/ks-theory.json`, entry `boundaryConditions.brane`); exact solutions of Section 18.21, for example the one of Notebook 18c, violate it, while the stationary members of that family obey it (Section 5.6); wherever a constant total charge is used, it is ASSUMED. RESOLVED on 2026-10-08: an earlier version of the detail text of the lead check `bilinears_under_charge_conjugation` said that normal ordering supplies one more sign for each bilinear; the derivation of Section 5.34 (PROVED) and the Fock-space computation of Notebook 05e (COMPUTED) contradicted it, and the record was corrected: it now states that normal ordering adds no sign, in agreement with this chapter. OPEN, not studied in this book: whether, for $V \neq 0$ in the author's metric, some matrix that does not obey the intertwiner condition maps every solution to a solution (Section 5.28; nothing in this chapter depends on it).

5.41 Exercises

Exercise 1. Use only the table of Section 5.6. (a) Read off the entries $C_{1,5}$ and $(\Gamma C)_{1,5}$, and $C_{9,13}$ and $(\Gamma C)_{9,13}$, and explain them with $\Gamma = \mathrm{diag}(-1_8, 1_8)$. (b) Show that $(\Gamma C)_{5,1} = (\Gamma C)_{1,5}$, as the symmetry of $\mathcal{C}_-$ demands. (c) Which entry of σ do the entries $(\Gamma C)_{1,5}$ and $(\Gamma C)_{9,13}$ correspond to?

Answer. (a) Row 1 of C reads -5: $C_{1,5} = -1$. Multiplying by Γ from the left multiplies row r by the diagonal entry Γ_{rr}, which is -1 for $r = 1$; so $(\Gamma C)_{1,5} = (-1)(-1) = +1$, and the table indeed shows $+5$ in row 1 of ΓC. Row 9 of C reads $+13$: $C_{9,13} = +1$; $\Gamma_{99} = +1$, so $(\Gamma C)_{9,13} = +1$, and the table

shows +13. (b) Row 5 of ΓC reads +1: $(\Gamma C)_{5,1} = +1 = (\Gamma C)_{1,5}$. (c) σ has 1_4 in its two off-diagonal blocks, so $\sigma_{1,5} = 1$. In $\Gamma C = \mathrm{diag}(\sigma, \sigma)$ the top-left block is σ itself, so $(\Gamma C)_{1,5} = \sigma_{1,5} = 1$; the bottom-right block starts at row and column 9, so $(\Gamma C)_{9,13} = \sigma_{1,5} = 1$ as well.

Exercise 2. Without a computer, show that $\mathcal{C}_+^{-1}\gamma^{(x4)}\mathcal{C}_+ = +\gamma^{(x4)}$ and $\mathcal{C}_-^{-1}\gamma^{(x4)}\mathcal{C}_- = -\gamma^{(x4)}$, and check that both agree with Theorem CC (c).

Answer. By (C1) with $\eta_{x4\,x4} = -1$, $C\gamma^{(x4)} = \gamma^{(x4)}C$. So $\mathcal{C}_+^{-1}\gamma^{(x4)}\mathcal{C}_+ = C\gamma^{(x4)}C = \gamma^{(x4)}CC = \gamma^{(x4)}$, using $CC = 1$ (C3). Theorem CC (c) says it is $-(\gamma^{(x4)})^T$, and indeed $(\gamma^{(x4)})^T = -\gamma^{(x4)}$ (the time-like gammas are antisymmetric), so $-(\gamma^{(x4)})^T = \gamma^{(x4)}$. For $\mathcal{C}_-$: $\mathcal{C}_-^{-1}\gamma^{(x4)}\mathcal{C}_- = C\Gamma\gamma^{(x4)}\Gamma C = -C\gamma^{(x4)}\Gamma\Gamma C = -C\gamma^{(x4)}C = -\gamma^{(x4)}$, by (X2), (X1) and the first part; Theorem CC (c) says $+(\gamma^{(x4)})^T = -\gamma^{(x4)}$. Both agree.

Exercise 3. Take $\Psi_0 = e_1$, the column with 1 in row 1 and 0 elsewhere. (a) Using the table of the time-like gammas in Section 5.2, find $\gamma^{(x4)}e_1$ and write the free solution $\Psi(x4)$ of Section 5.28. (b) Write Ψ^* and $\Gamma\Psi^*$ and say which mass each belongs to. (c) Compute $S = \Psi^T C\Psi$ and $J^{(x4)}$ of Ψ with the table of Section 5.6. What do the answers illustrate?

Answer. (a) $\gamma^{(x4)}e_1$ is column 1 of $\gamma^{(x4)}$. The table lists, for each row, the column of its nonzero entry; row 14 of $\gamma^{(x4)}$ reads +1, so the only nonzero entry of column 1 is +1 in row 14: $\gamma^{(x4)}e_1 = e_{14}$. So $\Psi(x4) = \cos(m\,x4)\,e_1 - \sin(m\,x4)\,e_{14}$. (b) Ψ is real, so $\Psi^* = \Psi$: the same-mass conjugate is the field itself (mass m). Γ is -1 in row 1 and $+1$ in row 14, so $\Gamma\Psi^* = -\cos(m\,x4)\,e_1 - \sin(m\,x4)\,e_{14}$. With $\Phi_0 = \Gamma e_1 = -e_1$ the solution formula with mass $-m$ gives $\cos(-m\,x4)\Phi_0 - \sin(-m\,x4)\gamma^{(x4)}\Phi_0 = -\cos(m\,x4)\,e_1 + \sin(m\,x4)\,(-e_{14})$, the same column: $\Gamma\Psi^*$ belongs to the mass $-m$. (c) $S = \cos^2(m\,x4)\,C_{1,1} - \cos(m\,x4)\sin(m\,x4)\,(C_{1,14} + C_{14,1}) + \sin^2(m\,x4)\,C_{14,14}$. Row 1 of C points to column 5 and row 14 to column 10, so all four entries are 0 and $S = 0$. In the same way row 1 of B/i points to column 10 and row 14 to column 5, so $B_{1,1} = B_{1,14} = B_{14,1} = B_{14,14} = 0$ and $J^{(x4)} = 0$. The example illustrates Section 5.29: a real field carries no charge, $\mathcal{C}_+$ does nothing to it, and the map $\Psi \to \Gamma\Psi$ of the other charge-conjugation matrix $\mathcal{C}_- = \Gamma C$ gives a different real solution with the mass reversed.

Exercise 4. Take the complex column $\Psi = e_1 + e_5 + i\,e_{10}$. (a) Compute $S = \Psi^\dagger C\Psi$ and $J^{(x4)} = \Psi^\dagger B\Psi$ with the table of Section 5.6. (b) Do the same for Ψ^* and for $\Gamma\Psi^*$, and compare with the commuting rows of the sign table of Section 5.29.

Answer. (a) $S = \sum_{r,c}\Psi_r^* C_{rc}\Psi_c$ needs the entries of C between the rows and columns 1, 5, 10. Row 1 of C points to 5 with $-$ ($C_{1,5} = -1$), row 5 to 1 with $-$ ($C_{5,1} = -1$), row 10 to 14 (not among them). So $S = \Psi_1^* C_{1,5}\Psi_5 + \Psi_5^* C_{5,1}\Psi_1 = (1)(-1)(1) + (1)(-1)(1) = -2$. For B: row 1 of B/i points to 10 with + ($B_{1,10} = +i$), row 10 to 1 with $-$ ($B_{10,1} = -i$), row 5 to 14 (not among them). So $J^{(x4)} = \Psi_1^* B_{1,10}\Psi_{10} + \Psi_{10}^* B_{10,1}\Psi_1 = (1)(i)(i) + (-i)(-i)(1) = -1 - 1 = -2$. (b) $\Psi^* = e_1 + e_5 - i\,e_{10}$: S involves only the real components 1 and 5, so $S = -2$; $J^{(x4)} = (1)(i)(-i) + (i)(-i)(1) = 1 + 1 = +2$. $\Gamma\Psi^* = -e_1 - e_5 - i\,e_{10}$ ($\Gamma = -1$ in rows 1 and 5, +1 in row 10): $S = (-1)(-1)(-1) + (-1)(-1)(-1) = -2$; $J^{(x4)} = (-1)(i)(-i) + (i)(-i)(-1) = -1 - 1 = -2$. So $\mathcal{C}_+$ keeps S and reverses $J^{(x4)}$, and $\mathcal{C}_-$ keeps both: the commuting rows of the table.

Exercise 5. For a fixed angle α, consider the condition $\Psi = e^{i\alpha}\Psi^*$. (a) Show that it is consistent in the sense of Section 5.29. (b) Find all columns that obey it. (c) Why does this not contradict the statement that there are only two charge-conjugation matrices?

Answer. (a) Here $M = e^{i\alpha}1$ and $MM^* = e^{i\alpha}e^{-i\alpha}1 = 1$: consistent. (b) Write $\Psi = e^{i\alpha/2}u$ with $u = e^{-i\alpha/2}\Psi$. Then $e^{i\alpha}\Psi^* = e^{i\alpha}e^{-i\alpha/2}u^* = e^{i\alpha/2}u^*$, so the condition reads $e^{i\alpha/2}u = e^{i\alpha/2}u^*$, that is

$u = u^*$: the solutions are the columns $e^{i\alpha/2}u$ with u real. (c) Theorem CC finds the matrices up to a factor; $e^{i\alpha}1$ is a multiple of 1. The condition is the reality condition of $\mathcal{C}_+$ for the field $e^{-i\alpha/2}\Psi$, which differs from Ψ only by a constant phase.

Exercise 6. With the sign rule of Section 5.34 compute $f_1^* f_0^* |00\rangle$ and $f_0^* f_1^* |00\rangle$ for two modes, and show that f_0^* and f_1^* anticommute on this state.

Answer. $f_0^*|00\rangle = |10\rangle$ (no mode below mode 0, sign +). Then $f_1^*|10\rangle$ fills mode 1; mode 0 below it is full, so the sign is $(-1)^1$: $f_1^*|10\rangle = -|11\rangle$. Hence $f_1^* f_0^*|00\rangle = -|11\rangle$. In the other order, $f_1^*|00\rangle = |01\rangle$ (mode 0 below is empty, sign +), and $f_0^*|01\rangle = |11\rangle$ (no mode below mode 0). Hence $f_0^* f_1^*|00\rangle = |11\rangle$. The sum is $\{f_0^*, f_1^*\}|00\rangle = -|11\rangle + |11\rangle = 0$.

Exercise 7. (a) Compute the constant $c = \mathrm{tr}(K'B^T)$ of Section 5.34 for the scalar ($K = C$) and the map $M = 1$. (b) Compute it for the charge density ($K = B$) and the map $M = \Gamma$. (c) The vacuum value of the charge density in the example of Notebook 05e is $\langle 0|J^{(x4)}|0\rangle = 8$. What is $\langle 0|J'^{(x4)}|0\rangle$ for $M = \Gamma$, and what is the normal-ordered result?

Answer. (a) $K' = C$ and $B^T = -B = iC\gamma^{(x4)}$, so $K'B^T = iCC\gamma^{(x4)} = i\gamma^{(x4)}$ by $CC = 1$, and $c = i\,\mathrm{tr}\,\gamma^{(x4)} = 0$ (Rule 5). (b) $K' = \Gamma B \Gamma = -B$ (Section 5.34), so $K'B^T = -B(-B) = BB = 1$ and $c = \mathrm{tr}\,1 = 16$. (c) For $M = \Gamma$ every sign is $s = -1$, so $\langle 0|J'^{(x4)}|0\rangle = s\langle 0|J^{(x4)}|0\rangle + c = -8 + 16 = 8$. After normal ordering, $:J'^{(x4)}: = J'^{(x4)} - 8 = (-J^{(x4)} + 16) - 8 = -(J^{(x4)} - 8) = -:J^{(x4)}:$: the constant is gone and the sign -1 stays, so every charge is reversed, as figure 8 of Notebook 05e shows.

Exercise 8. (a) Show that $\exp(\pi S^{(x4)(x5)}) = \gamma^{(x4)}\gamma^{(x5)}$. (b) Compute the spinor norm of $g = \gamma^{(x4)}\gamma^{(x5)}$ and check $g^T C g = C$. (c) Do the same for $h = \gamma^{(x1)}\gamma^{(x4)}$ and explain, with Section 5.23, why h is not a product of exponentials although it lies in Spin(4,4).

Answer. (a) $x4$ and $x5$ are both time-like, so $\eta_{x4\,x4}\eta_{x5\,x5} = +1$ and the plane is a rotation plane; the closed formula of Section 5.18 gives $\exp(\theta S^{(x4)(x5)}) = \cos\frac{\theta}{2}\,1 + \sin\frac{\theta}{2}\,\gamma^{(x4)}\gamma^{(x5)}$, and at $\theta = \pi$, $\cos\frac{\pi}{2} = 0$ and $\sin\frac{\pi}{2} = 1$. (b) g is a product of $k = 2$ unit vectors e_{x4} and e_{x5} with $\eta(e_{x4}, e_{x4}) = \eta(e_{x5}, e_{x5}) = -1$, so $N(g) = (-1)(-1) = +1$ and $g^T C g = (-1)^2 N(g)\, C = C$, as for every product of exponentials. (c) For h, $N(h) = \eta(e_{x1}, e_{x1})\,\eta(e_{x4}, e_{x4}) = (+1)(-1) = -1$, so $h^T C h = -C$. Every product of exponentials keeps C (Section 5.18), so h is not one. In the language of Section 5.23: $\Lambda(h)$ reverses $x1$ and $x4$, so its space block has determinant -1 and its time block -1; the sign pattern is $(-,-)$, the piece $\mathrm{Spin}_0\,\gamma^{(x0)}\gamma^{(x4)}$, while the products of exponentials have the pattern $(+,+)$. Since h has two factors, it is even and lies in Spin(4,4).

6 Spinors in curved 4+4 space: vielbein, spin connection and $\gamma^\mu\Omega_\mu$

The two fields of the author, dirac16complex and dirac16complex00, have sixteen components at every point of spacetime. Chapters 4 and 5 built the eight gamma matrices that act on these components and the groups Pin(4,4) and Spin(4,4) that turn them. All of that was done in flat 4+4 space, where the eight directions are the same at every point. The author's universe is not flat: 3-space inflates, the three extra times deflate exponentially, and the hidden direction is warped. This chapter teaches, from zero, how a field with sixteen components is differentiated in such a curved space, and it computes exactly what gravity puts into the field equations of the author's two fields through the canonical spin connection. The answer is one 16×16 matrix, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$, which is $3H\gamma^{(x0)}$ on the backgrounds of this book ($b_4 = 1$), and

the chapter also proves precisely what this matrix does and does not mean.

6.1 What this chapter does

The problem in plain words. A spinor field Ψ is a list of sixteen numbers at every point. The numbers are not measured in the coordinates $x_0, \ldots, x_7$ directly: they are measured with respect to eight perpendicular unit directions chosen at each point, a **frame**. In flat space one frame serves everywhere. In a curved space the frame of one point and the frame of a neighbouring point are turned with respect to each other, so the plain derivative $\partial_\mu \Psi$ mixes two things: how the field really changes, and how the frame turns. The **spin connection** measures how the frame turns, and the **covariant derivative** $D_\mu \Psi = \partial_\mu \Psi + \Omega_\mu \Psi$ removes the turning. The field equation of both fields (Chapter 7 derives it) is

$$\sum_\mu \gamma^\mu D_\mu \Psi = \big(m + U'(S)\big)\Psi,$$

and the spin connection enters it only through the one matrix $\sum_\mu \gamma^\mu \Omega_\mu$. The author asked that the field equations "always possess non-zero contributions from gravity (through the canonical spin-connection, unless we are in flat 4+4 spacetime)" (quoted from `Revision/README.md`). This chapter computes the matrix exactly and examines the request line by line.

What the chapter proves.

- The author's metric, with its two functions $a_4(x_4)$ and $b_4(x_4)$, has a diagonal vielbein (Section 6.2), 25 independent nonzero Christoffel symbols (Section 6.3) and a canonical spin connection with 12 independent nonzero components (Sections 6.4 to 6.6).
- The spinor connection Ω_μ makes the curved gammas covariantly constant (Section 6.7), and in the diagonal vielbein $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ for every $H > 0$ and all functions $a_4(x_4)$ and $b_4(x_4) > 0$: the a_4' terms along the time $x4$ of the three inflating directions and of the three deflating extra times cancel exactly, a changing b_4 leaves the term $\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$, and the six terms along the hidden direction add up to $3H\gamma^{(x0)}$ (Sections 6.8 and 6.9). For a constant b_4, as on the backgrounds of this book, the matrix is $3H\gamma^{(x0)}$.
- This matrix sends a nonzero field to zero only where $|b_4'/b_4| = 4H$, never for a constant b_4 (Section 6.10).
- A **negative control**: if the extra times inflated instead of deflating, a term $3a_4'\gamma^{(x4)}$ would survive; the cancellation of the a_4' terms is caused by the deflation (Section 6.10).
- The value of $\gamma^\mu \Omega_\mu$ belongs to the chosen frame: in another frame of the same metric the contraction is $\frac{6H-\beta}{2}(\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}) + \frac{3}{4}\frac{b_4'}{b_4}(\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)})$ and vanishes for $\beta = 6H$ when b_4 is constant (Sections 6.15 and 6.16), and a rescaling of the field removes it (Section 6.18). What no frame and no rescaling can remove is the curvature: $R^{x0}{}_{x0} = -6H^2$ for every $H > 0$ and every b_4 (Section 6.17).
- Why the record contracts the spin connection with both frame indices down: the author's notebook contracts the mixed components, which lacks a metric factor; that contraction

deletes all 16 boost parts of the connection and would give $\frac{3H}{2}\gamma^{(x0)} + \frac{3}{2}(a'_4 - \frac{b'_4}{2b_4})\gamma^{(x4)}$ (Sections 6.23 and 6.24).

The three worked examples. Each is a complete Jupyter notebook; the chapter places each one where its theory has been explained, gives the complete instructions to run it, prints its complete text and then explains every line of its code.

notebook	**what it computes**	**figures**	**checks**
06a	vielbein, Christoffel symbols, spin connection, Ω_μ, the contraction $\gamma^\mu\Omega_\mu$ direction by direction, the divergence form, the Dirac operator, the negative control, a warm-up in the plane	10	36
06b	a boosted frame, local spin covariance, the Riemann and Ricci tensors, the spinor curvature, the rescaling	6	26
06c	the mixed contraction of the author's notebook and what the missing metric factor changes	6	27

All three reproduce the Revision record where they overlap with it; every such check prints a second line that names the record file and its check. Their last lines are ALL 36 CHECKS PASSED (notebook 06a), ALL 26 CHECKS PASSED (notebook 06b) and ALL 27 CHECKS PASSED (notebook 06c).

Statuses. Every statement carries one of the five labels of Chapter 0. **PROVED**: derived exactly here, line by line, and confirmed by an exact check of a notebook; where the Revision record proves the same, the record file and its check are named. **COMPUTED**: a number obtained by a notebook, with its accuracy. **ASSUMED**: the author's metric itself (the input of the theory), two standard theorems that are quoted and named where they are used, and the illustrative histories used only in some plots: $a_4 = x_4$ with $b_4 = 1$ (and $H = 1$), which is the prescribed background of Chapter 3 with slope $A = 1$, and in one plot $b_4 = 1 + \frac{1}{2}\sin x_4$. No HYPOTHESIS enters this chapter, and nothing in it is OPEN except the choice of boundary condition at the patch end, which Section 6.24 names and Chapter 14 treats. Nothing in this chapter concerns pairs of universes, their creation, or matter and antimatter; Chapters 18 to 21 treat those questions and state exactly what is and what is not proved about them.

Notation. The eight directions are named $x0, \ldots, x7$ as in Chapter 5: $x0$ is the hidden direction, $x1, x2, x3$ are 3-space (inflating), $x4$ is the time, $x5, x6, x7$ are the three extra times (time-like, deflating exponentially). Inside formulas the coordinate values are written $x_0, \ldots, x_7$, for example

$z = 6Hx_0$ with $0 < z < \pi/2$. The letter i stands for one of the 3-space directions $x1, x2, x3$ and the letter t for one of the extra times $x5, x6, x7$ (except in the polar plane, where i is the imaginary unit, and in Section 6.16, where t is a component). The functions a_4 and $b_4 > 0$ depend on x_4 only; $a_4' = da_4/dx_4$, $a_4'' = d^2a_4/dx_4^2$, and the same for b_4. As in Chapter 3, $\alpha = a_4'$ is the expansion rate of 3-space and $\alpha_t = -a_4' + b_4'/(2b_4)$ the expansion rate of the extra times ($\alpha_t = -a_4'$ when b_4 is constant). On the backgrounds of this book $b_4 = 1$ (Chapter 3: an empty spacetime has a constant b_4, and a constant b_4 only rescales $x5, x6, x7$); every formula of this chapter holds for every b_4 unless it says otherwise. The frame metric is

$$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$$

in the order $x0, \ldots, x7$; η_{aa} is its entry for the direction a, and $\eta^{aa} = \eta_{aa}$ because every entry is ± 1. The constant gamma matrix of a direction carries the direction in parentheses, $\gamma^{(x4)}$; a gamma without parentheses, γ^{x4} or γ^μ, is a **curved gamma** (Section 6.2). Sums are written with $\sum$ as in Chapter 3. The one exception is the contraction $\gamma^\mu \Omega_\mu$: it always means $\sum_\mu \gamma^\mu \Omega_\mu$, and a single term is marked "(no sum)". Finally $s = \sin^{1/6} z$.

The record files. The Revision record files are named by their short names. The formula files `field-theory.json` (the metric at $b_4 = 1$) and `field-theory-b4.json` (the metric with $b_4(x_4)$) lie in the folder `Revision/theory`. The reports `python-field-theory.json` and `wolfram-field-theory.json` of the metric at $b_4 = 1$ and `python-field-theory-b4.json` and `wolfram-field-theory-b4.json` of the metric with b_4 (each pair written by a sympy program and by a WolframScript program that share no code), and the two scope reports `python-scope.json` and `wolfram-scope.json` (at $b_4 = 1$), lie in the folder `Revision/theory/reports`. The record `metric-b4.json` of the metric with b_4 lies in the folder `Revision/metric_b4/results`. The **lead report** is the file `emt-divergence-and-spin-connection.json` in the folder `Revision/lead_checks/reports`, written by an independent program of the lead of the Revision work for the metric at $b_4 = 1$.

6.2 A frame at every point: the vielbein

Why a spinor needs a frame. The gamma matrices obey the Clifford relation $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}1$ with the constant matrix η (Chapter 4). In a curved space the metric $g_{\mu\nu}(x)$ changes from point to point and is not η. The way out is to describe every point by its own eight reference directions, perpendicular to each other and of unit length (unit duration for a time-like direction). Measured in such a frame the metric looks exactly like η, so the constant gammas of Chapter 4 can be used unchanged. The spinor components are always measured with respect to such a frame.

Definition (vielbein). A **vielbein** (German for "many legs") is an 8×8 matrix $e^a{}_\mu(x)$ at every point with

$$g_{\mu\nu} = \sum_a \eta_{aa}\, e^a{}_\mu\, e^a{}_\nu \qquad \text{for all } \mu, \nu.$$

The upper index a is a **frame index**: it numbers the eight frame directions. The lower index μ is a **coordinate index**: it numbers the coordinates. Row a of the matrix holds the components of the frame direction a in the sense that a small coordinate step dx^μ has the frame components

$dX^a = \sum_\mu e^a{}_\mu dx^\mu$, and then $ds^2 = \sum_{\mu,\nu} g_{\mu\nu} dx^\mu dx^\nu = \sum_a \eta_{aa} (dX^a)^2$: lengths in frame components are computed with η. The **inverse vielbein** $e_a{}^\mu$ is the inverse matrix, $\sum_\mu e^a{}_\mu e_b{}^\mu = \delta^a_b$ (1 when $a = b$, 0 otherwise).

The diagonal vielbein of the author's metric. The author's metric is diagonal, $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ with the positive **vielbein factors** (Chapter 3 calls them scale factors h_a; they are the same numbers)

$$f = \big(\cot z,\ e^{a_4} s,\ e^{a_4} s,\ e^{a_4} s,\ 1,\ e^{-a_4} b_4^{1/2} s,\ e^{-a_4} b_4^{1/2} s,\ e^{-a_4} b_4^{1/2} s\big), \qquad s = \sin^{1/6} z.$$

The diagonal matrix $e^a{}_\mu = f_a\, \delta^a_\mu$ is a vielbein: in the sum $\sum_a \eta_{aa} e^a{}_\mu e^a{}_\nu$ only $a = \mu = \nu$ survives, which gives $\eta_{\mu\mu} f_\mu^2 = g_{\mu\mu}$. For example, for $\mu = \nu = x1$,

$$\eta_{11} f_1^2 = (+1)\big(e^{a_4} \sin^{1/6} z\big)^2 = e^{2a_4} \sin^{1/3} z$$

(a power of a product is the product of the powers, and $(\sin^{1/6} z)^2 = \sin^{1/3} z$), which is the author's g_{11}; for $\mu = \nu = x5$, $\eta_{55} f_5^2 = (-1) e^{-2a_4} b_4 \sin^{1/3} z$ (with $(b_4^{1/2})^2 = b_4$), the author's g_{55}; for $x0$, $(+1)\cot^2 z = g_{00}$. The inverse vielbein is diagonal too, $e_a{}^\mu = \delta^\mu_a / f_a$. PROVED; `metric-b4.json`, key `metricDiagonal`; `wolfram-field-theory-b4.json`, check `metric_is_the_authors_b4`; at $b_4 = 1$ `field-theory.json`, formulas `metric` and `vielbein_diagonal`, and the lead report, check `vielbein_reproduces_metric`; Notebook 06a, In [7].

The volume factor. The determinant of a diagonal matrix is the product of its diagonal entries, so $\det g = \prod_\mu \eta_{\mu\mu} f_\mu^2 = (+1)^4 (-1)^4 \prod_\mu f_\mu^2$, and $\sqrt{|\det g|} = f_0 f_1 \cdots f_7$. Line by line:

$$f_1 f_2 f_3 = e^{3a_4} s^3, \qquad f_5 f_6 f_7 = e^{-3a_4} b_4^{3/2} s^3$$

(three equal factors; $e^{a_4} e^{a_4} e^{a_4} = e^{3a_4}$ and $(b_4^{1/2})^3 = b_4^{3/2}$),

$$f_1 f_2 f_3\, f_4\, f_5 f_6 f_7 = e^{3a_4} e^{-3a_4} b_4^{3/2} s^6 \cdot 1 = b_4^{3/2} s^6 = b_4^{3/2} \sin z$$

(the exponentials cancel, $e^u e^{-u} = 1$, and $(\sin^{1/6} z)^6 = \sin z$),

$$\sqrt{|\det g|} = b_4^{3/2} \sin z \cdot \cot z = b_4^{3/2} \sin z \cdot \frac{\cos z}{\sin z} = b_4^{3/2} \cos z$$

(multiply by $f_0 = \cot z$ and cancel $\sin z$). The volume factor depends on the time x_4 only through $b_4^{3/2}$: what 3-space gains, the extra times lose, and for a constant b_4, in particular on the backgrounds $b_4 = 1$ where it is $\cos z$, it does not depend on the time at all. PROVED; `metric-b4.json`, key `sqrtAbsDetG`; `wolfram-field-theory-b4.json`, check `sqrt_det_g`; at $b_4 = 1$ `python-field-theory.json`, check `sqrt_det_g_equals_cos_z`, and `wolfram-field-theory.json`, check `sqrt_det_g_is_cos_z`; Notebook 06a, In [7]. This one fact will explain, in Section 6.9, why the function a_4 drops out of $\gamma^\mu \Omega_\mu$ and the function b_4 does not.

Curved gammas. With the constant frame gammas γ^a define, for every coordinate μ,

$$\gamma^\mu = \sum_a e_a{}^\mu\, \gamma^a.$$

They obey the **curved Clifford relation**. Line by line:

$$\gamma^\mu\gamma^\nu + \gamma^\nu\gamma^\mu = \sum_{a,b} e_a{}^\mu e_b{}^\nu (\gamma^a\gamma^b + \gamma^b\gamma^a)$$

(insert the definition twice; numbers commute with matrices),

$$= \sum_{a,b} e_a{}^\mu e_b{}^\nu \, 2\eta^{ab}\, 1 = 2\sum_a \eta^{aa}\, e_a{}^\mu e_a{}^\nu \, 1 = 2g^{\mu\nu}\, 1$$

(the Clifford relation; η^{ab} is zero unless $a = b$; and $g^{\mu\nu} = \sum_a \eta^{aa} e_a{}^\mu e_a{}^\nu$ is the inverse metric, as one checks for the diagonal vielbein directly: $\eta^{\mu\mu}/f_\mu^2 = 1/g_{\mu\mu}$). For the diagonal vielbein the curved gammas are the frame gammas divided by the factors, $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ (no sum):

$$\gamma^{x0} = \tan z\, \gamma^{(x0)}, \qquad \gamma^{x_i} = e^{-a_4} s^{-1} \gamma^{(x_i)}, \qquad \gamma^{x4} = \gamma^{(x4)}, \qquad \gamma^{x_t} = e^{a_4} b_4^{-1/2} s^{-1} \gamma^{(x_t)},$$

with $i = 1, 2, 3$, $t = 5, 6, 7$ and $1/\cot z = \tan z$.

A vielbein is a choice. Let $\Lambda(x)$ be an 8×8 matrix at every point with $\Lambda^T \eta \Lambda = \eta$ (an element of O(4,4), Chapter 5). Then $e'^a{}_\mu = \sum_c \Lambda^a{}_c \, e^c{}_\mu$ is another vielbein of the same metric. Line by line, in matrix form ($g = e^T \eta e$ is the definition written with matrices, e having rows a and columns μ):

$$e'^T \eta\, e' = (\Lambda e)^T \eta (\Lambda e) = e^T (\Lambda^T \eta \Lambda) e = e^T \eta\, e = g$$

(the transpose of a product is the product of the transposes in reverse order; then the defining property of Λ). Such a point-dependent turning of the frame is a **local frame rotation** (a rotation, or a boost when it mixes a space-like with a time-like direction; Chapter 5). The diagonal vielbein is the simplest choice for the author's metric, but it is one choice among infinitely many. Sections 6.15 and 6.16 show that this freedom matters.

statement	**status**	**where it is verified**
$e^a{}_\mu = f_a \delta^a_\mu$ reproduces the author's metric	PROVED	`metric-b4.json`, key `metricDiagonal`; `wolfram-field-theory-b4.json`, check `metric_is_the_authors_b4`; at $b_4 = 1$: `field-theory.json`, formulas `metric` and `vielbein_diagonal`; `python-field-theory.json`, check `metric_from_vielbein_equals_SPEC`; lead report, check `vielbein_reproduces_metric`

statement	status	where it is verified
$\sqrt{\lvert\det g\rvert} = b_4^{3/2}\cos z$; $\cos z$, independent of x_4, at $b_4 = 1$	PROVED	`metric-b4.json`, key `sqrtAbsDetG`; `wolfram-field-theory-b4.json`, check `sqrt_det_g`; at $b_4 = 1$: `python-field-theory.json`, check `sqrt_det_g_equals_cos_z`; `wolfram-field-theory.json`, check `sqrt_det_g_is_cos_z`
curved Clifford relation; another frame Λe gives the same metric	PROVED (above)	Notebook 06b checks the second for a boost, In [7]

6.3 The Christoffel symbols of the author's metric

What they are. Chapter 3 introduced the **Christoffel symbols**, the numbers that say how the coordinate directions change from point to point:

$$\Gamma^{\lambda}{}_{\mu\nu} = \tfrac{1}{2}\sum_{\rho} g^{\lambda\rho}(\partial_\mu g_{\rho\nu} + \partial_\nu g_{\rho\mu} - \partial_\rho g_{\mu\nu}).$$

They are symmetric in the two lower indices, $\Gamma^{\lambda}{}_{\mu\nu} = \Gamma^{\lambda}{}_{\nu\mu}$ (exchange μ and ν: the first two terms exchange places and the third is unchanged because g is symmetric). They enter the **coordinate covariant derivative** of a vector field, $\nabla_\mu V^\nu = \partial_\mu V^\nu + \sum_\lambda \Gamma^{\nu}{}_{\mu\lambda} V^\lambda$, the derivative that compares the vector at two points correctly although the coordinate directions change between them.

A diagonal metric. When g is diagonal, so is its inverse, $g^{\lambda\rho} = \delta^{\lambda\rho}/g_{\lambda\lambda}$, and only $\rho = \lambda$ survives in the sum. With $g_{\lambda\lambda} = \eta_{\lambda\lambda} f_\lambda^2$ there are four cases, each derived from the formula with $\rho = \lambda$ (no sums below).

(a) All three indices equal, $\lambda = \mu = \nu$:

$$\Gamma^{\lambda}{}_{\lambda\lambda} = \frac{1}{2g_{\lambda\lambda}}(\partial_\lambda g_{\lambda\lambda} + \partial_\lambda g_{\lambda\lambda} - \partial_\lambda g_{\lambda\lambda}) = \frac{\partial_\lambda g_{\lambda\lambda}}{2g_{\lambda\lambda}} = \frac{2\eta_{\lambda\lambda} f_\lambda \partial_\lambda f_\lambda}{2\eta_{\lambda\lambda} f_\lambda^2} = \partial_\lambda \ln f_\lambda$$

(two of the three terms cancel; the derivative of f^2 is $2f\,\partial f$; and $\partial f/f = \partial \ln f$).

(b) The upper index equals one lower index, $\lambda = \mu \neq \nu$:

$$\Gamma^{\lambda}{}_{\lambda\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\lambda g_{\lambda\nu} + \partial_\nu g_{\lambda\lambda} - \partial_\lambda g_{\lambda\nu}) = \frac{\partial_\nu g_{\lambda\lambda}}{2g_{\lambda\lambda}} = \partial_\nu \ln f_\lambda$$

($g_{\lambda\nu} = 0$ for $\lambda \neq \nu$; then the same steps as in (a)).

(c) The two lower indices equal, the upper one different, $\mu = \nu \neq \lambda$:

$$\Gamma^{\lambda}{}_{\mu\mu} = \frac{1}{2g_{\lambda\lambda}}(0 + 0 - \partial_\lambda g_{\mu\mu}) = -\frac{\eta_{\mu\mu}\, 2 f_\mu \partial_\lambda f_\mu}{2\eta_{\lambda\lambda} f_\lambda^2} = -\eta_{\lambda\lambda}\eta_{\mu\mu}\,\frac{f_\mu\,\partial_\lambda f_\mu}{f_\lambda^2}$$

(the off-diagonal entries $g_{\lambda\mu}$ vanish; $1/\eta_{\lambda\lambda} = \eta_{\lambda\lambda}$ because $\eta_{\lambda\lambda} = \pm 1$).

(d) All three different: every term contains an off-diagonal entry of g, so ${\Gamma^\lambda}_{\mu\nu} = 0$.

The derivatives of the factors. The author's metric depends on x_0 (through $z = 6Hx_0$) and on x_4 (through a_4 and b_4) only, so only ∂_0 and ∂_4 of the factors can be nonzero. With $\ln f_0 = \ln\cos z - \ln\sin z$, $\ln f_i = a_4 + \frac{1}{6}\ln\sin z$ for $i = 1, 2, 3$, $\ln f_4 = 0$ and $\ln f_t = -a_4 + \frac{1}{2}\ln b_4 + \frac{1}{6}\ln\sin z$ for $t = 5, 6, 7$:

$$\partial_4 \ln f_i = a_4' = \alpha, \qquad \partial_4 \ln f_t = -a_4' + \frac{b_4'}{2b_4} = \alpha_t, \qquad \partial_4 \ln f_0 = 0$$

(only $\pm a_4$ and $\frac{1}{2}\ln b_4$ depend on x_4; their derivatives are $\pm a_4'$ and $\frac{1}{2}b_4'/b_4$),

$$\partial_0 \ln f_i = \partial_0 \ln f_t = \frac{1}{6} \cdot \frac{\cos z}{\sin z} \cdot 6H = H \cot z$$

(the chain rule: the derivative of $\ln\sin z$ with respect to z is $\cos z/\sin z$, and $dz/dx_0 = 6H$),

$$\partial_0 \ln f_0 = 6H\Big(\frac{-\sin z}{\cos z} - \frac{\cos z}{\sin z}\Big) = -6H\,\frac{\sin^2 z + \cos^2 z}{\sin z\cos z} = -\frac{6H}{\sin z\cos z}$$

(the chain rule again; common denominator; $\sin^2 z + \cos^2 z = 1$).

The 25 symbols. Insert these into the four cases. With $\lambda = x0$, case (a): ${\Gamma^{x0}}_{x0\,x0} = -6H/(\sin z\cos z) = -12H/\sin 2z$ (the double-angle formula $\sin 2z = 2\sin z\cos z$). From (a) and (b) with $\lambda = x_i$ and $\lambda = x_t$:

$${\Gamma^{x_i}}_{x_i\,x4} = a_4', \qquad {\Gamma^{x_t}}_{x_t\,x4} = -a_4' + \frac{b_4'}{2b_4} = \alpha_t, \qquad {\Gamma^{x_i}}_{x_i\,x0} = {\Gamma^{x_t}}_{x_t\,x0} = H\cot z.$$

From (c) with $\lambda = x4$ ($\eta_{44} = -1$, $f_4 = 1$):

$$\Gamma^{x4}{}_{x_i\,x_i} = -(-1)(+1) f_i\,\partial_4 f_i = f_i^2\,a_4' = e^{2a_4}\sin^{1/3} z\; a_4',$$

$$\Gamma^{x4}{}_{x_t\,x_t} = -(-1)(-1) f_t\,\partial_4 f_t = -f_t^2\,\alpha_t = e^{-2a_4} b_4 \sin^{1/3} z\,\Big(a_4' - \frac{b_4'}{2b_4}\Big)$$

(for the extra times $\partial_4 f_t = \alpha_t f_t$). From (c) with $\lambda = x0$ ($\eta_{00} = +1$, $f_0 = \cot z$):

$${\Gamma^{x0}}_{x_i\,x_i} = -\frac{f_i\,\partial_0 f_i}{\cot^2 z} = -\frac{f_i^2 H\cot z}{\cot^2 z} = -f_i^2 H\tan z, \qquad {\Gamma^{x0}}_{x_t\,x_t} = +f_t^2 H\tan z$$

($\partial_0 f_i = f_i H\cot z$; $\eta_{tt} = -1$ reverses the sign for the extra times). Every other symbol vanishes: f_0 and $f_4 = 1$ do not depend on x_4, and the factors depend on no other coordinate. Counting with $\mu \le \nu$: one ${\Gamma^{x0}}_{x0x0}$, and three each of the eight families ${\Gamma^{x_i}}_{x0x_i}$, ${\Gamma^{x_i}}_{x_ix4}$, ${\Gamma^{x_t}}_{x0x_t}$, ${\Gamma^{x_t}}_{x4x_t}$, ${\Gamma^{x0}}_{x_ix_i}$, ${\Gamma^{x4}}_{x_ix_i}$, ${\Gamma^{x0}}_{x_tx_t}$, ${\Gamma^{x4}}_{x_tx_t}$: $1 + 8\cdot 3 = 25$, the same number as for $b_4 = 1$. Every nonzero symbol carries an index $x0$ or $x4$. In words: moving forward in time a 3-space direction grows at the rate

a_4' (${\Gamma^{x1}}_{x1x4} = a_4'$) and an extra-time direction grows at the rate α_t (${\Gamma^{x5}}_{x4x5} = -a_4' + b_4'/(2b_4)$): for a constant b_4 it shrinks at the same rate a_4'.

statement	status	where it is verified
the 25 independent nonzero Christoffel symbols above	PROVED	`field-theory-b4.json`, formula `christoffel_nonzero`; `wolfram-field-theory-b4.json`, check `christoffel_count`; `python-field-theory-b4.json`, check `christoffel_nonzero`; at $b_4 = 1$: `field-theory.json`, formula `christoffel_nonzero`; `python-field-theory.json`, check `christoffel_symmetric_metric_compatible`; Notebook 06a, In [9]

6.4 The canonical spin connection: the vielbein postulate and its solution

The problem. A vector field V has the coordinate components V^ν and the frame components $V^a = \sum_\nu e^a{}_\nu V^\nu$. The frame turns from point to point, so $\partial_\mu V^a$ mixes the change of V with the turning of the frame, just as $\partial_\mu V^\nu$ mixed the change of V with the bending of the coordinate lines. The cure is the same: a correction that is linear in V,

$$\nabla_\mu V^a = \partial_\mu V^a + \sum_b \omega_\mu{}^a{}_b \, V^b,$$

with coefficients $\omega_\mu{}^a{}_b(x)$, the **spin connection**. It has one coordinate index μ (the direction of the step) and two frame indices a, b (which frame direction turns towards which).

The requirement. The frame components of the covariant derivative must be the covariant derivative of the frame components: $\sum_\nu e^a{}_\nu \, \nabla_\mu V^\nu = \nabla_\mu V^a$ for every vector field V. Write both sides out. The left side is

$$\sum_\nu e^a{}_\nu \Big(\partial_\mu V^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda} V^\lambda \Big).$$

The right side is

$$\partial_\mu \Big(\sum_\lambda e^a{}_\lambda V^\lambda \Big) + \sum_b \omega_\mu{}^a{}_b \sum_\lambda e^b{}_\lambda V^\lambda = \sum_\lambda (\partial_\mu e^a{}_\lambda) V^\lambda + \sum_\lambda e^a{}_\lambda \, \partial_\mu V^\lambda + \sum_{b,\lambda} \omega_\mu{}^a{}_b \, e^b{}_\lambda V^\lambda$$

(the definition of V^a and V^b; then the product rule). The terms with $\partial_\mu V$ are the same on both sides (rename ν as λ). Subtracting the left side from the right side leaves

$$\sum_{\lambda}\Big(\partial_\mu e^{a}{}_{\lambda}-\sum_{\nu}\Gamma^{\nu}{}_{\mu\lambda}e^{a}{}_{\nu}+\sum_{b}\omega_{\mu}{}^{a}{}_{b}\,e^{b}{}_{\lambda}\Big)V^{\lambda}=0$$

(collect the coefficient of each V^λ). This must hold for every V, so every bracket vanishes. Renaming λ as ν and the summed ν as λ, this is the **vielbein postulate**:

$$\partial_\mu e^{a}{}_{\nu}-\sum_{\lambda}\Gamma^{\lambda}{}_{\mu\nu}\,e^{a}{}_{\lambda}+\sum_{b}\omega_{\mu}{}^{a}{}_{b}\,e^{b}{}_{\nu}=0\qquad\text{for all }\mu,a,\nu.$$

In eight dimensions these are $8\cdot 8\cdot 8=512$ equations.

The solution. Multiply the postulate by $e_c{}^{\nu}$ and sum over ν. The last term becomes $\sum_b \omega_\mu{}^a{}_b \sum_\nu e^b{}_\nu e_c{}^\nu = \sum_b \omega_\mu{}^a{}_b \delta^b_c = \omega_\mu{}^a{}_c$ (the inverse vielbein), so

$$\omega_{\mu}{}^{a}{}_{c}=\sum_{\nu,\lambda}e_c{}^{\nu}\,\Gamma^{\lambda}{}_{\mu\nu}\,e^{a}{}_{\lambda}-\sum_{\nu}e_c{}^{\nu}\,\partial_\mu e^{a}{}_{\nu}.$$

Differentiate $\sum_\nu e^a{}_\nu e_c{}^\nu = \delta^a_c$ (a constant) with the product rule: $\sum_\nu e_c{}^\nu \partial_\mu e^a{}_\nu = -\sum_\nu e^a{}_\nu \partial_\mu e_c{}^\nu$. Inserting this and renaming the summation letters gives the formula that the Revision record and all three notebooks use:

$$\omega_{\mu}{}^{a}{}_{b}=\sum_{\nu}e^{a}{}_{\nu}\Big(\partial_\mu e_b{}^{\nu}+\sum_{\lambda}\Gamma^{\nu}{}_{\mu\lambda}\,e_b{}^{\lambda}\Big).$$

The derivation shows that the postulate has this one solution and no other: it is the **canonical spin connection**. Its components $\omega_\mu{}^a{}_b$, with the first frame index up, are called the **mixed components**.

Lowering the first index. Define $\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b$ (no sum). **Theorem: $\omega_{\mu ab} = -\omega_{\mu ba}$.** Proof, line by line. First, $\eta_{aa}e^a{}_\nu = \sum_\sigma g_{\nu\sigma}e_a{}^\sigma$: multiply $g_{\nu\sigma} = \sum_c \eta_{cc}e^c{}_\nu e^c{}_\sigma$ by $e_a{}^\sigma$ and sum over σ; only $c=a$ survives. So

$$\omega_{\mu ab}=\sum_{\nu,\sigma}g_{\nu\sigma}\,e_a{}^{\sigma}\,\partial_\mu e_b{}^{\nu}+\sum_{\sigma,\lambda}e_a{}^{\sigma}\,\Gamma_{\sigma\mu\lambda}\,e_b{}^{\lambda},\qquad\Gamma_{\sigma\mu\lambda}=\sum_{\nu}g_{\sigma\nu}\Gamma^{\nu}{}_{\mu\lambda}$$

(insert into the formula; $\Gamma_{\sigma\mu\lambda}$ is the Christoffel symbol with its first index lowered, $\frac{1}{2}(\partial_\mu g_{\sigma\lambda}+\partial_\lambda g_{\sigma\mu}-\partial_\sigma g_{\mu\lambda})$). Second, adding the two lowered symbols with the outer indices exchanged,

$$\Gamma_{\sigma\mu\lambda}+\Gamma_{\lambda\mu\sigma}=\tfrac{1}{2}(\partial_\mu g_{\sigma\lambda}+\partial_\lambda g_{\sigma\mu}-\partial_\sigma g_{\mu\lambda}+\partial_\mu g_{\lambda\sigma}+\partial_\sigma g_{\lambda\mu}-\partial_\lambda g_{\mu\sigma})=\partial_\mu g_{\sigma\lambda}$$

(g is symmetric, so the second and sixth terms cancel, the third and fifth cancel, and the first and fourth are equal). Third, add $\omega_{\mu ab}$ and $\omega_{\mu ba}$:

$$\omega_{\mu ab}+\omega_{\mu ba}=\sum_{\nu,\sigma}g_{\nu\sigma}(e_a{}^{\sigma}\partial_\mu e_b{}^{\nu}+e_b{}^{\sigma}\partial_\mu e_a{}^{\nu})+\sum_{\sigma,\lambda}e_a{}^{\sigma}e_b{}^{\lambda}\,\partial_\mu g_{\sigma\lambda}=\partial_\mu\Big(\sum_{\sigma,\lambda}g_{\sigma\lambda}e_a{}^{\sigma}e_b{}^{\lambda}\Big)$$

(the second step: in the second Γ term rename $\sigma \leftrightarrow \lambda$ and use the identity just shown; then the three terms are the three pieces of the product rule for $g\, e_a\, e_b$). Finally $\sum_{\sigma,\lambda} g_{\sigma\lambda} e_a{}^{\sigma} e_b{}^{\lambda} = \eta_{ab}$ (the frame directions are perpendicular and of unit length), a constant, whose derivative is zero. $\square$

So for each μ the lowered connection is an antisymmetric 8×8 array: its diagonal vanishes and it has $8 \cdot 7/2 = 28$ independent entries, one for each of the 28 generators S^{ab} of Chapter 5. In words: $\omega_{\mu ab}$ is the rate at which the frame turns in the plane (a, b) when one moves along x^{μ}. The mixed components are not antisymmetric in general: $\omega_{\mu}{}^{b}{}_{a} = \eta_{bb}\omega_{\mu ba} = -\eta_{bb}\omega_{\mu ab} = -\eta_{aa}\eta_{bb}\,\omega_{\mu}{}^{a}{}_{b}$, which is antisymmetric for a pair of two space-like or two time-like directions and **symmetric** for a boost pair (Section 6.23 returns to this).

statement	**status**	**where it is verified**
the vielbein postulate has exactly the solution above	PROVED (above)	the postulate in all 512 components: `python-field-theory.json` and `wolfram-field-theory.json`, check `vielbein_postulate`; Notebook 06a, In [11]
$\omega_{\mu ab} = -\omega_{\mu ba}$	PROVED (above)	`python-field-theory.json`, check `spin_connection_antisymmetric`; `wolfram-field-theory.json`, check `omega_antisymmetric`; Notebook 06a, In [11]

6.5 A warm-up by hand: the flat plane in polar coordinates

Before eight dimensions, two. A point of the flat plane has the polar coordinates r (distance from the centre) and φ (angle). The metric is $ds^2 = dr^2 + r^2 d\varphi^2$ (Chapter 3), so $g = \mathrm{diag}(1, r^2)$, $\eta = \mathrm{diag}(1, 1)$, and the factors are $f_r = 1$, $f_\varphi = r$: the unit radial direction (frame index 0) and the unit angular direction (frame index 1).

Christoffel symbols. Case (c) of Section 6.3 with $\lambda = r$, $\mu = \varphi$: $\Gamma^{r}{}_{\varphi\varphi} = -(1)(1)\, f_\varphi \partial_r f_\varphi / f_r^2 = -r \cdot 1 = -r$. Case (b): $\Gamma^{\varphi}{}_{\varphi r} = \partial_r \ln r = 1/r$. All others vanish.

Spin connection. The formula of Section 6.4 with $\mu = \varphi$, $a = 1$ (angular), $b = 0$ (radial), where $e^{1}{}_{\varphi} = r$ and $e_0{}^{r} = 1$ are the only entries needed:

$$\omega_{\varphi}{}^{1}{}_{0} = e^{1}{}_{\varphi}\big(\partial_\varphi e_0{}^{\varphi} + \Gamma^{\varphi}{}_{\varphi r} e_0{}^{r}\big) = r\Big(0 + \frac{1}{r} \cdot 1\Big) = 1$$

(only $\nu = \varphi$ has $e^{1}{}_{\nu} \neq 0$; $e_0{}^{\varphi} = 0$; only $\lambda = r$ has $e_0{}^{\lambda} \neq 0$). With $\eta = \mathrm{diag}(1, 1)$ lowering changes nothing: $\omega_{\varphi 10} = 1$ and, by the antisymmetry, $\omega_{\varphi 01} = -1$. For $\mu = r$ everything vanishes. **Meaning:** the radial and angular unit directions at the angle φ are the Cartesian ones turned by the angle φ; moving in φ turns the frame at one radian per radian, and the spin connection records this rate.

Spinor connection. Two 2×2 matrices with the Clifford relation of the plane are the Pauli matrices

$\gamma^0 = \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $\gamma^1 = \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, with $\sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$; one checks by multiplying out that $\sigma_1\sigma_2 = i\sigma_3$ and $\sigma_2\sigma_3 = i\sigma_1$. The generator of the plane is $S^{01} = \frac{1}{4}[\sigma_1, \sigma_2] = \frac{1}{2}\sigma_1\sigma_2 = \frac{i}{2}\sigma_3$. Section 6.7 will show that the spinor connection is $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$, so

$$\Omega_\varphi = \omega_{\varphi 01} S^{01} = -\tfrac{i}{2}\,\sigma_3, \qquad \Omega_r = 0.$$

The contraction. The curved gammas are $\gamma^r = \sigma_1$ and $\gamma^\varphi = \sigma_2/r$, so

$$\sum_\mu \gamma^\mu \Omega_\mu = \frac{\sigma_2}{r}\Big(-\frac{i}{2}\sigma_3\Big) = -\frac{i}{2r}\,\sigma_2\sigma_3 = -\frac{i}{2r}\,i\sigma_1 = \frac{\sigma_1}{2r}$$

($i \cdot i = -1$). The Dirac operator of the flat plane in polar coordinates is therefore $\sigma_1(\partial_r + \frac{1}{2r}) + \frac{\sigma_2}{r}\partial_\varphi$: a term without derivative appears although the plane is flat.

But the plane is flat, and the connection can be removed. The **curvature of the spinor connection** is $F_{r\varphi} = \partial_r\Omega_\varphi - \partial_\varphi\Omega_r + \Omega_r\Omega_\varphi - \Omega_\varphi\Omega_r$; here Ω_φ is constant and $\Omega_r = 0$, so every term vanishes: $F_{r\varphi} = 0$. And with the spinor rotation $U(\varphi) = \cos\frac{\varphi}{2} + i\sin\frac{\varphi}{2}\,\sigma_3$ (Chapter 5: a rotation by the angle φ acts on spinors through the half angle),

$$\partial_\varphi U = -\tfrac{1}{2}\sin\tfrac{\varphi}{2} + \tfrac{i}{2}\cos\tfrac{\varphi}{2}\,\sigma_3 = \tfrac{i}{2}\sigma_3\Big(\cos\tfrac{\varphi}{2} + i\sin\tfrac{\varphi}{2}\,\sigma_3\Big) = \tfrac{i}{2}\sigma_3\,U$$

(differentiate each entry; then factor out $\frac{i}{2}\sigma_3$, using $\sigma_3\sigma_3 = 1$ and $i \cdot i = -1$), so $-(\partial_\varphi U)U^{-1} = -\frac{i}{2}\sigma_3 = \Omega_\varphi$.

Which rotation removes it. Section 6.15 will derive how the spinor connection changes when the frame is changed and the spinor components change by a spin transformation R: $\Omega'_\varphi = R\,\Omega_\varphi R^{-1} - (\partial_\varphi R)R^{-1}$. The Cartesian frame (the unit directions along x and y) does not turn, so its connection is 0, and the rule with $R = U$ turns this 0 into $-(\partial_\varphi U)U^{-1} = \Omega_\varphi$: U is the spinor rotation that carries the Cartesian frame into the polar frame. The way back is the inverse rotation $U^{-1} = \cos\frac{\varphi}{2} - i\sin\frac{\varphi}{2}\,\sigma_3$ (multiply out: $UU^{-1} = \cos^2\frac{\varphi}{2} + \sin^2\frac{\varphi}{2}\,\sigma_3\sigma_3 = 1$, because the two mixed terms cancel). Its derivative is

$$\partial_\varphi U^{-1} = -\tfrac{1}{2}\sin\tfrac{\varphi}{2} - \tfrac{i}{2}\cos\tfrac{\varphi}{2}\,\sigma_3 = -\tfrac{i}{2}\sigma_3\Big(\cos\tfrac{\varphi}{2} - i\sin\tfrac{\varphi}{2}\,\sigma_3\Big) = -\tfrac{i}{2}\sigma_3\,U^{-1}$$

(differentiate each entry; then factor out $-\frac{i}{2}\sigma_3$, using $\sigma_3\sigma_3 = 1$ and $i \cdot i = -1$), so $(\partial_\varphi U^{-1})\,U = -\frac{i}{2}\sigma_3$. The matrix σ_3 commutes with U and U^{-1} (both are built from 1 and σ_3), so $U^{-1}\Omega_\varphi U = \Omega_\varphi U^{-1}U = \Omega_\varphi$. The rule with $R = U^{-1}$ (whose inverse is U) therefore gives

$$\Omega'_\varphi = U^{-1}\,\Omega_\varphi\,U - (\partial_\varphi U^{-1})\,U = -\tfrac{i}{2}\sigma_3 - \Big(-\tfrac{i}{2}\sigma_3\Big) = 0,$$

and $\Omega'_r = 0$, because U does not depend on r. (The rule with $R = U$ would instead give $-\frac{i}{2}\sigma_3 - \frac{i}{2}\sigma_3 = -i\sigma_3$: the rotation U turns the polar frame further, not back.) In a flat space the spin connection is only the turning of the chosen frame, and the spinor rotation U^{-1} removes it. At $\varphi = 2\pi$ the frame is back where it started, but $U(2\pi) = \cos\pi = -1$: a spinor turns by half the

angle, the double cover of Chapter 5. Notebook 06a checks all of this in In [5] and draws it in figure 1. PROVED (exact; the notebook's checks are its own, the plane is not a Revision record).

Two lessons. First, a nonzero spin connection, and even a nonzero $\sum_\mu \gamma^\mu \Omega_\mu$, does not mean that space is curved; it may only record how the chosen frame turns. Second, the test of curvature is $F_{\mu\nu}$, not Ω_μ. Both lessons return in Sections 6.16 and 6.17.

6.6 The spin connection of the author's metric: twelve components

A diagonal vielbein, in general. Let $e^a{}_\mu = f_a \delta^a_\mu$ and $e_b{}^\nu = \delta^\nu_b / f_b$. In the formula of Section 6.4 only $\nu = a$ survives in the outer sum and only $\lambda = b$ in the inner sum:

$$\omega_\mu{}^a{}_b = f_a \Big(\partial_\mu \frac{\delta^a_b}{f_b} + \Gamma^a{}_{\mu b} \frac{1}{f_b} \Big).$$

For $a = b$ this is $f_a \big(- \partial_\mu f_a / f_a^2 + \Gamma^a{}_{\mu a} / f_a \big) = \Gamma^a{}_{\mu a} - \partial_\mu \ln f_a = 0$, by cases (a) and (b) of Section 6.3. For $a \neq b$ only the second term remains, $(f_a/f_b)\Gamma^a{}_{\mu b}$, and by case (d) it can be nonzero only when $\mu = a$ or $\mu = b$:

$$\omega_a{}^a{}_b = \frac{f_a}{f_b}\, \partial_b \ln f_a = \frac{\partial_b f_a}{f_b}, \qquad \omega_b{}^a{}_b = \frac{f_a}{f_b} \Big(- \eta_{aa}\eta_{bb} \frac{f_b\, \partial_a f_b}{f_a^2} \Big) = -\eta_{aa}\eta_{bb}\, \frac{\partial_a f_b}{f_a}$$

(no sums; case (b) with $\lambda = a$, $\nu = b$, and case (c) with $\lambda = a$, $\mu = b$). Lowering with η_{aa} gives the two forms

$$\omega_{a\,ab} = \eta_{aa}\, \frac{\partial_b f_a}{f_b}, \qquad \omega_{b\,ab} = -\eta_{bb}\, \frac{\partial_a f_b}{f_a} \qquad (a \neq b, \text{ no sum}),$$

and the antisymmetry of Section 6.4 can be seen directly: the second form with the letters a and b exchanged reads $\omega_{a\,ba} = -\eta_{aa} \partial_b f_a / f_b = -\omega_{a\,ab}$. So: **in a diagonal vielbein, $\omega_{\mu ab}$ is nonzero only when μ is one of the two frame indices, and then $\omega_{\mu\,\mu b} = \eta_{\mu\mu}\, \partial_b f_\mu / f_b$.**

The author's metric. The factors depend on x_0 and x_4 only, so b must be $x0$ or $x4$. With $\partial_4 f_i = a_4' f_i$, $\partial_4 f_t = \alpha_t f_t$, $\partial_0 f_i = H \cot z\, f_i$, $\partial_0 f_t = H \cot z\, f_t$ (Section 6.3), $f_0 = \cot z$ and $f_4 = 1$:

$$\omega_{x_i\, x_i\, x4} = (+1) \frac{a_4' f_i}{1} = a_4'\, e^{a_4} s, \qquad \omega_{x_i\, x_i\, x0} = (+1) \frac{H \cot z\, f_i}{\cot z} = H\, e^{a_4} s,$$

$$\omega_{x_t\, x_t\, x4} = (-1) \frac{\alpha_t f_t}{1} = \Big(a_4' - \frac{b_4'}{2 b_4} \Big) e^{-a_4} b_4^{1/2} s, \qquad \omega_{x_t\, x_t\, x0} = (-1) \frac{H \cot z\, f_t}{\cot z} = -H\, e^{-a_4} b_4^{1/2} s.$$

The record lists the components with the smaller index first, $a < b$ in the order $x0, \ldots, x7$; for 3-space this is $\omega_{x_i\, x0\, x_i} = -\omega_{x_i\, x_i\, x0} = -H\, e^{a_4} s$, and for the extra times $\omega_{x_t\, x4\, x_t} = -\omega_{x_t\, x_t\, x4}$ and $\omega_{x_t\, x0\, x_t} = -\omega_{x_t\, x_t\, x0}$. For $\mu = x4$ every candidate contains $\partial_b f_4 = 0$ (because $f_4 = 1$), and for $\mu = x0$ the only candidate, $b = x4$, contains $\partial_4 f_0 = 0$ (because $\cot z$ does not depend on the time); so $\omega_{x0\,ab} = \omega_{x4\,ab} = 0$. Altogether there are $3 \cdot 2 + 3 \cdot 2 = 12$ independent nonzero components:

μ	**pair** (a,b)	$\omega_{\mu ab}$	**kind of pair**
x_i $(i=1,2,3)$	$(x0,x_i)$	$-H\,e^{a_4}s$	rotation (two space-like)
x_i	$(x_i,x4)$	$a_4'\,e^{a_4}s$	boost (space-like with time-like)
x_t $(t=5,6,7)$	$(x0,x_t)$	$H\,e^{-a_4}b_4^{1/2}s$	boost (space-like with time-like)
x_t	$(x4,x_t)$	$-\big(a_4'-\frac{b_4'}{2b_4}\big)e^{-a_4}b_4^{1/2}s$	rotation (two time-like)

Each component is a rate times a factor that never vanishes for $0<z<\pi/2$: the rate is a_4' (3-space), $a_4'-b_4'/(2b_4)=-\alpha_t$ (the extra times) or H (the hidden direction), the factor $\pm e^{a_4}\sin^{1/6}z$ or $\pm e^{-a_4}b_4^{1/2}\sin^{1/6}z$. So the part made by the time dependence (3-space inflating, the extra times deflating) vanishes only if a_4 and b_4 are constant, and the part made by the hidden direction vanishes only if $H=0$, which is not a metric of the family (the entries $\sin^{1/3}(6Hx_0)$ would vanish; record check `degenerate_at_H_0`). **For every admissible $H>0$ the spin connection of the diagonal vielbein is nonzero.** Note also the signs: the extra time $x5$ has the opposite signs of the 3-space direction $x1$, because it shrinks while 3-space grows (for a constant b_4 at the same rate). Figure 06a.4 shows the arrays $\omega_{x1\,ab}$ and $\omega_{x5\,ab}$ at a sample point.

statement	**status**	**where it is verified**
exactly 12 independent nonzero $\omega_{\mu ab}$, with the values of the table	PROVED	`field-theory-b4.json`, formula `omega_nonzero`; `wolfram-field-theory-b4.json`, check `omega_components`; `python-field-theory-b4.json`, check `omega_nonzero`; at $b_4=1$: `field-theory.json`, formula `omega_nonzero`; Notebook 06a, In [11]
each is a rate (a_4', $a_4'-b_4'/(2b_4)$ or H) times a nonvanishing factor	PROVED	Notebook 06a, In [11]; at $b_4=1$: `wolfram-field-theory.json`, check `Omega_vanishes_iff_a4prime_and_H_vanish`; `python-field-theory.json`, check `nontriviality_Omega_zero_iff_flat`

6.7 The spinor connection and the covariant derivative of a spinor

What we want. A spinor Ψ has no coordinate index; its sixteen components are measured in the frame. Its covariant derivative must be of the form

$$D_\mu\Psi=\partial_\mu\Psi+\Omega_\mu\Psi$$

with eight 16×16 matrices $\Omega_\mu(x)$, the **spinor connection**. Which matrices? The gamma matrices link spinors and vectors: $\gamma^\nu\Psi$ is again a spinor, but γ^ν carries the vector index ν. The natural requirement is that the derivative treats the curved gammas as constants, as the flat derivative treats the constant gammas:

$$D_\mu\gamma^\nu = \partial_\mu\gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega_\mu, \gamma^\nu] = 0,$$

where $[A, B] = AB - BA$ is the **commutator**. The first two terms are the coordinate covariant derivative of the vector index of γ^ν, the commutator is the spinor connection acting on the matrix from the left and from the right. With this requirement the product rule holds in the form $D_\mu(\gamma^\nu\Psi) = \gamma^\nu D_\mu\Psi$, so the field equation $\sum_\mu \gamma^\mu D_\mu\Psi = (m + U')\Psi$ does not depend on where the derivative is written.

The requirement in frame form. Insert $\gamma^\nu = \sum_a e_a{}^\nu\gamma^a$ (the frame gammas are constant):

$$D_\mu\gamma^\nu = \sum_a \Big(\partial_\mu e_a{}^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda} e_a{}^\lambda\Big)\gamma^a + \sum_a e_a{}^\nu [\Omega_\mu, \gamma^a]$$

(the product rule; numbers commute with matrices). Multiply the formula of Section 6.4 by $e_c{}^\sigma$ and sum over c: since $\sum_c e_c{}^\sigma e^c{}_\nu = \delta^\sigma_\nu$, this gives $\partial_\mu e_b{}^\sigma + \sum_\lambda \Gamma^\sigma{}_{\mu\lambda} e_b{}^\lambda = \sum_c e_c{}^\sigma\, \omega_\mu{}^c{}_b$. With it the first sum becomes $\sum_{a,c} e_c{}^\nu \omega_\mu{}^c{}_a \gamma^a$, and

$$D_\mu\gamma^\nu = \sum_c e_c{}^\nu \Big([\Omega_\mu, \gamma^c] + \sum_a \omega_\mu{}^c{}_a \gamma^a\Big)$$

(rename a as c in the second sum). The inverse vielbein is invertible, so $D_\mu\gamma^\nu = 0$ for all ν holds exactly when

$$[\Omega_\mu, \gamma^c] = -\sum_b \omega_\mu{}^c{}_b\, \gamma^b \qquad \text{for all } \mu, c.$$

This is the **defining property** of the spinor connection: commuting with Ω_μ turns the gammas exactly as ω_μ turns the frame.

The solution. Theorem. $\Omega_\mu = \frac{1}{2}\sum_{a,b} \omega_{\mu ab} S^{ab}$, with the generators $S^{ab} = \frac{1}{4}[\gamma^a, \gamma^b]$ of Chapter 5, has the defining property. Proof, line by line. Chapter 5 proved the **vector rule** $[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a - \eta^{ac}\gamma^b$. So

$$[\Omega_\mu, \gamma^c] = \tfrac{1}{2}\sum_{a,b} \omega_{\mu ab}(\eta^{bc}\gamma^a - \eta^{ac}\gamma^b) = \tfrac{1}{2}\,\eta^{cc}\sum_a \omega_{\mu ac}\gamma^a - \tfrac{1}{2}\,\eta^{cc}\sum_b \omega_{\mu cb}\gamma^b$$

(a commutator is linear in each argument; η^{bc} is nonzero only for $b = c$, η^{ac} only for $a = c$). By the antisymmetry $\omega_{\mu ac} = -\omega_{\mu ca}$ the first sum is $-\frac{1}{2}\eta^{cc}\sum_a \omega_{\mu ca}\gamma^a$, the same as the second (rename a as b). Hence

$$[\Omega_\mu, \gamma^c] = -\eta^{cc}\sum_b \omega_{\mu cb}\gamma^b = -\sum_b \omega_\mu{}^c{}_b \gamma^b$$

(two equal halves; and $\eta^{cc}\omega_{\mu cb} = \eta_{cc}\eta_{cc}\omega_\mu{}^c{}_b = \omega_\mu{}^c{}_b$ because $\eta_{cc}^2 = 1$). $\square$

Uniqueness. If Ω'_μ also had the defining property, the difference $\Omega'_\mu - \Omega_\mu$ would commute with all eight gammas, and Chapter 5 proved that only multiples of the identity do (the commutant of Pin(4,4) is one-dimensional). Every S^{ab} has trace 0 (it is a multiple of a commutator, and $\mathrm{tr}(AB) = \mathrm{tr}(BA)$), so Ω_μ is the only solution with trace 0. A multiple of the identity would be an extra field of the electromagnetic kind, not part of gravity; the canonical connection has none.

Summing over $a < b$. Both $\omega_{\mu ab}$ and S^{ab} change sign when a and b are exchanged, so the terms (a, b) and (b, a) are equal, the terms $a = b$ vanish, and

$$\Omega_\mu = \tfrac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab} = \sum_{a<b}\omega_{\mu ab}S^{ab}, \qquad S^{ab} = \tfrac{1}{2}\gamma^a\gamma^b \ (a \neq b)$$

(for $a \neq b$ the two gammas anticommute, so $\gamma^a\gamma^b - \gamma^b\gamma^a = 2\gamma^a\gamma^b$).

The derivative of the Dirac adjoint. The Dirac adjoint is $\bar\Psi = \Psi^\dagger C$ (Chapter 5), and its covariant derivative is $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\,\Omega_\mu$; with this sign the number $\bar\Psi\Phi$ obeys the ordinary product rule, $(D_\mu\bar\Psi)\Phi + \bar\Psi D_\mu\Phi = \partial_\mu(\bar\Psi\Phi)$, because the two Ω_μ terms cancel. It agrees with the conjugate of $D_\mu\Psi$: Ω_μ is real and Chapter 5 proved $(S^{ab})^T C = -CS^{ab}$, so $(D_\mu\Psi)^\dagger C = \partial_\mu\Psi^\dagger C + \Psi^\dagger\Omega_\mu^T C = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$.

The author's metric. With the table of Section 6.6 and $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$:

$$\begin{aligned}\Omega_{x_i} &= e^{a_4}s\big(a'_4\,S^{x_i\,x4} + H\,S^{x_i\,x0}\big), \qquad \Omega_{x0} = \Omega_{x4} = 0,\\ \Omega_{x_t} &= -e^{-a_4}b_4^{1/2}s\Big(\big(a'_4 - \tfrac{b'_4}{2b_4}\big)S^{x4\,x_t} + H\,S^{x_t\,x0}\Big)\end{aligned}$$

(with $S^{x_i\,x0} = -S^{x0\,x_i}$ and $S^{x_t\,x0} = -S^{x0\,x_t}$, the components of the table).

No frame direction turns when one moves along the time or along the hidden direction. Each nonzero Ω_μ is a combination of two generators. A boost generator is a **symmetric** matrix and a rotation generator an **antisymmetric** one: the transpose of a product reverses the order, and the author's gammas satisfy $(\gamma^a)^T = \eta_{aa}\gamma^a$ (Chapter 5), so

$$(\gamma^a\gamma^b)^T = (\gamma^b)^T(\gamma^a)^T = \eta_{aa}\eta_{bb}\,\gamma^b\gamma^a = -\eta_{aa}\eta_{bb}\,\gamma^a\gamma^b \qquad (a \neq b),$$

which is $+\gamma^a\gamma^b$ for a boost pair ($\eta_{aa}\eta_{bb} = -1$, 16 pairs) and $-\gamma^a\gamma^b$ for a rotation pair (12 pairs). In Ω_{x1} the a'_4 part is a boost ($x1$ with $x4$) and the H part a rotation ($x1$ with $x0$); in Ω_{x5} it is the other way round. Figure 06a.5 shows both matrices.

The covariant constancy, checked. With these Ω_μ the defining property holds for all 64 pairs (μ, c) and $D_\mu\gamma^\nu = 0$ for all 64 pairs (μ, ν) (Notebook 06a, In [13]).

statement	status	where it is verified
Ω_{x_i}, Ω_{x_t} as above; $\Omega_{x0} = \Omega_{x4} = 0$	PROVED	Notebook 06a, In [13]; at $b_4 = 1$: `field-theory.json`, formula `Omega_components`; `python-field-theory.json`, check `Omega_x0_and_Omega_x4_vanish`; `wolfram-field-theory.json`, check `Omega_components`
the defining property for all 64 pairs	PROVED	Notebook 06a, In [13]; at $b_4 = 1$: `wolfram-field-theory.json`, check `S_rotates_gamma_with_omega`
$D_\mu \gamma^\nu = 0$ for all 64 pairs	PROVED	`wolfram-field-theory-b4.json`, check `gamma_covariantly_constant`; at $b_4 = 1$: `python-field-theory.json`, check `covariant_constancy_D_mu_gamma_nu`; `wolfram-field-theory.json`, check `gamma_covariantly_constant`
uniqueness up to a multiple of 1	PROVED (above, with the commutant of Chapter 5)	`python-algebra.json`, check `pin_commutant_dimension_1` (in `Revision/algebra/reports`)

6.8 The contraction $\gamma^\mu \Omega_\mu$, direction by direction

The field equation contains $\sum_\mu \gamma^\mu D_\mu \Psi = \sum_\mu \gamma^\mu \partial_\mu \Psi + \big(\sum_\mu \gamma^\mu \Omega_\mu\big)\Psi$, so the spin connection enters it only through the one 16×16 matrix $\sum_\mu \gamma^\mu \Omega_\mu$. We compute it one direction at a time.

A 3-space direction, $\mu = x1$ (the same for $x2$ and $x3$). With $\gamma^{x1} = \gamma^{(x1)}/f_1$ and $f_1 = e^{a_4} s$:

$$\gamma^{x1}\Omega_{x1} = \frac{\gamma^{(x1)}}{f_1}\, f_1 \Big(a_4' \tfrac{1}{2}\gamma^{(x1)}\gamma^{(x4)} + H\, \tfrac{1}{2}\gamma^{(x1)}\gamma^{(x0)} \Big)$$

(the formulas of Sections 6.2 and 6.7),

$$= \tfrac{1}{2} a_4' \, \gamma^{(x1)}\gamma^{(x1)}\gamma^{(x4)} + \tfrac{1}{2} H \, \gamma^{(x1)}\gamma^{(x1)}\gamma^{(x0)} = \tfrac{1}{2} a_4' \, \gamma^{(x4)} + \tfrac{1}{2} H \, \gamma^{(x0)}$$

(the factor f_1 cancels; $\gamma^{(x1)}\gamma^{(x1)} = \eta_{11} 1 = +1$ by the Clifford relation).

An extra time, $\mu = x5$ (the same for $x6$ and $x7$). With $\gamma^{x5} = \gamma^{(x5)}/f_5$ and the rate $r = a_4' - b_4'/(2b_4) = -\alpha_t$:

$$\gamma^{x5}\Omega_{x5} = \frac{\gamma^{(x5)}}{f_5}\,(-f_5)\Big(r\,\tfrac{1}{2}\gamma^{(x4)}\gamma^{(x5)} + H\,\tfrac{1}{2}\gamma^{(x5)}\gamma^{(x0)}\Big) = -\tfrac{1}{2}r\,\gamma^{(x5)}\gamma^{(x4)}\gamma^{(x5)} - \tfrac{1}{2}H\,\gamma^{(x5)}\gamma^{(x5)}\gamma^{(x0)}$$

(the factor $f_5 = e^{-a_4}b_4^{1/2}s$ cancels and leaves the sign). Now $\gamma^{(x5)}\gamma^{(x4)}\gamma^{(x5)} = -\gamma^{(x4)}\gamma^{(x5)}\gamma^{(x5)} = -\gamma^{(x4)}(-1) = \gamma^{(x4)}$ (two different gammas anticommute; $\gamma^{(x5)}\gamma^{(x5)} = \eta_{55}1 = -1$), and $\gamma^{(x5)}\gamma^{(x5)}\gamma^{(x0)} = -\gamma^{(x0)}$. Hence

$$\gamma^{x5}\Omega_{x5} = -\tfrac{1}{2}\Big(a_4' - \frac{b_4'}{2b_4}\Big)\gamma^{(x4)} + \tfrac{1}{2}H\,\gamma^{(x0)} = \Big(-\tfrac{1}{2}a_4' + \frac{b_4'}{4b_4}\Big)\gamma^{(x4)} + \tfrac{1}{2}H\,\gamma^{(x0)}.$$

The hidden direction and the time. $\Omega_{x0} = \Omega_{x4} = 0$, so $\gamma^{x0}\Omega_{x0} = \gamma^{x4}\Omega_{x4} = 0$.

The sum. Writing each term as $\alpha_\mu\gamma^{(x4)} + \beta_\mu\gamma^{(x0)}$ (these coefficients α_μ are not the expansion rates α, α_t of Section 6.3):

direction μ	α_μ **(coefficient of $\gamma^{(x4)}$)**	β_μ **(coefficient of $\gamma^{(x0)}$)**
$x0$ (hidden)	0	0
$x1$, $x2$, $x3$ (inflating)	$+\frac{1}{2}a_4'$ each	$\frac{1}{2}H$ each
$x4$ (time)	0	0
$x5$, $x6$, $x7$ (deflating)	$-\frac{1}{2}a_4' + \frac{b_4'}{4b_4}$ each	$\frac{1}{2}H$ each
total	$\frac{3}{2}a_4' - \frac{3}{2}a_4' + \frac{3}{4}\frac{b_4'}{b_4} = \frac{3}{4}\frac{b_4'}{b_4}$	$6\cdot\frac{1}{2}H = 3H$

$$\sum_\mu \gamma^\mu\Omega_\mu = 3H\,\gamma^{(x0)} + \frac{3}{4}\,\frac{b_4'}{b_4}\,\gamma^{(x4)}.$$

In words. Along the time direction the inflating 3-space and the deflating extra times push with equal and opposite strength $\pm\frac{3}{2}a_4'$, and these contributions cancel exactly, for every function $a_4(x_4)$. A changing b_4 stretches the three extra times and leaves the term $\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$. Along the hidden direction all six warped directions push the same way, and their contributions add. For a constant b_4, in particular on the backgrounds $b_4 = 1$ of this book, the result is the constant matrix $3H\gamma^{(x0)}$: it contains neither a_4 nor x_4 nor x_0.

Reading off the coefficients with traces. Notebook 06a finds α_μ and β_μ with traces (the **trace** of a square matrix is the sum of its diagonal entries). For two different gammas $\mathrm{tr}(\gamma^a\gamma^b) = 0$, while $\mathrm{tr}(\gamma^{(x4)}\gamma^{(x4)}) = \mathrm{tr}(-1) = -16$ and $\mathrm{tr}(\gamma^{(x0)}\gamma^{(x0)}) = +16$ (Chapter 4). So for $M = \alpha\gamma^{(x4)} + \beta\gamma^{(x0)}$:

$$\mathrm{tr}(M\gamma^{(x4)}) = \alpha(-16) + \beta\cdot 0, \qquad \mathrm{tr}(M\gamma^{(x0)}) = \alpha\cdot 0 + \beta\cdot 16,$$

hence $\alpha = -\mathrm{tr}(M\gamma^{(x4)})/16$ and $\beta = \mathrm{tr}(M\gamma^{(x0)})/16$. The notebook then checks that $M - \alpha\gamma^{(x4)} - \beta\gamma^{(x0)}$ is the zero matrix, so that nothing else is hidden in M.

statement	status	where it is verified
the eight terms $\gamma^\mu\Omega_\mu$ (no sum) of the table	PROVED	`python-field-theory-b4.json` and `wolfram-field-theory-b4.json`, check `gammaOmega_per_direction`; at $b_4 = 1$: `field-theory.json`, formula `gammaOmega_per_direction`; `wolfram-field-theory.json`, check `gammaOmega_x4_terms_cancel`; `python-field-theory.json`, check `time_terms_cancel_hidden_term_survives`
$\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ for every $H > 0$, a_4 and b_4; $3H\gamma^{(x0)}$ for a constant b_4	PROVED	`field-theory-b4.json`, formula `gammaOmega_total`; `wolfram-field-theory-b4.json` and `python-field-theory-b4.json`, check `gammaOmega_total`; at $b_4 = 1$: `field-theory.json`, formula `gammaOmega_total`; `wolfram-field-theory.json`, check `gammaOmega_equals_3H_gamma_x0`; `python-field-theory.json`, check `gamma_mu_Omega_mu_equals_3H_gamma_x0`; lead report, check `gamma_Omega_equals_3H_gamma0`; Notebook 06a, In [15]

6.9 Why a_4 drops out and b_4 does not: the divergence form

The cancellation of Section 6.8 is not an accident. Two facts explain it without computing Ω_μ at all.

Fact 1: for each μ separately, $\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu = 0$ (no sum). In the diagonal vielbein Ω_μ contains only generators $S^{(\mu)b}$ with $b \neq \mu$ (Section 6.6), and γ^μ is a multiple of $\gamma^{(\mu)}$. For one such generator:

$$\gamma^{(\mu)}\,\tfrac{1}{2}\gamma^{(\mu)}\gamma^{(b)} = \tfrac{1}{2}\eta_{\mu\mu}\gamma^{(b)}, \qquad \tfrac{1}{2}\gamma^{(\mu)}\gamma^{(b)}\,\gamma^{(\mu)} = -\tfrac{1}{2}\gamma^{(\mu)}\gamma^{(\mu)}\gamma^{(b)} = -\tfrac{1}{2}\eta_{\mu\mu}\gamma^{(b)}$$

(the Clifford relation; in the second product the two different gammas $\gamma^{(b)}$ and $\gamma^{(\mu)}$ are exchanged at the cost of a sign). The two products are opposite, so they anticommute, and so do the combinations.

The divergence form. Take the covariant constancy $D_\mu\gamma^\nu = 0$ of Section 6.7, put $\nu = \mu$ and sum over μ:

$$\sum_\mu \partial_\mu \gamma^\mu + \sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda}\gamma^\lambda + \sum_\mu \left(\Omega_\mu \gamma^\mu - \gamma^\mu \Omega_\mu\right) = 0.$$

The contracted Christoffel symbol of a diagonal metric is $\sum_\mu \Gamma^\mu{}_{\mu\lambda} = \sum_\mu \partial_\lambda \ln f_\mu = \partial_\lambda \ln\left(\prod_\mu f_\mu\right) = \partial_\lambda \ln\sqrt{|g|}$ (cases (a) and (b) of Section 6.3; the logarithm of a product is the sum of the logarithms; Section 6.2). So the first two sums are

$$\sum_\lambda \Big(\partial_\lambda \gamma^\lambda + \frac{\partial_\lambda \sqrt{|g|}}{\sqrt{|g|}}\gamma^\lambda\Big) = \frac{1}{\sqrt{|g|}} \sum_\lambda \partial_\lambda(\sqrt{|g|}\,\gamma^\lambda)$$

(the product rule read backwards). By Fact 1, $\Omega_\mu \gamma^\mu = -\gamma^\mu \Omega_\mu$, so the third sum is $-2\sum_\mu \gamma^\mu \Omega_\mu$. Solving for the contraction:

$$\sum_\mu \gamma^\mu \Omega_\mu = \frac{1}{2\sqrt{|g|}} \sum_\mu \partial_\mu(\sqrt{|g|}\,\gamma^\mu).$$

The right side is built from the volume factor and the vielbein alone.

Fact 2: the product structure. For the diagonal vielbein $\sqrt{|g|}\,\gamma^\mu = \left(\prod_c f_c\right)\gamma^{(\mu)}/f_\mu = \left(\prod_{c\neq\mu} f_c\right)\gamma^{(\mu)}$ (no sum over μ). The term ∂_μ of this product can be nonzero only if the product depends on its own coordinate x_μ, that is only for $\mu = x0$ or $\mu = x4$. For the hidden direction:

$$\prod_{c\neq x0} f_c = e^{3a_4} s^3 \cdot 1 \cdot e^{-3a_4} b_4^{3/2} s^3 = b_4^{3/2} s^6 = b_4^{3/2} \sin z,$$

$$\frac{1}{2b_4^{3/2}\cos z}\,\partial_0\big(b_4^{3/2}\sin z\big)\,\gamma^{(x0)} = \frac{6H\cos z}{2\cos z}\gamma^{(x0)} = 3H\,\gamma^{(x0)}$$

(the chain rule with $dz/dx_0 = 6H$; b_4 does not depend on x_0; then cancel $b_4^{3/2}\cos z$). For the time:

$$\prod_{c\neq x4} f_c = \cot z \cdot e^{3a_4} s^3 \cdot e^{-3a_4} b_4^{3/2} s^3 = b_4^{3/2} \sin z \cot z = b_4^{3/2} \cos z,$$

$$\frac{1}{2b_4^{3/2}\cos z}\,\partial_4\big(b_4^{3/2}\cos z\big)\,\gamma^{(x4)} = \frac{\frac{3}{2}b_4^{1/2}b_4'}{2b_4^{3/2}}\gamma^{(x4)} = \frac{3}{4}\,\frac{b_4'}{b_4}\,\gamma^{(x4)}$$

(Section 6.2; the factors $e^{\pm 3a_4}$ cancel, so a_4 is gone; the power rule gives $\partial_4 b_4^{3/2} = \frac{3}{2}b_4^{1/2}b_4'$). **The time direction contributes only through** b_4, and nothing when b_4 is constant. This is the deeper reason for the cancellation of the a_4' terms in Section 6.8: the 7-volume of a slice does not change with the inflation of 3-space and the deflation of the extra times, only with b_4. So the divergence form gives $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ again, by a second and much shorter route. Notebook 06a checks Fact 1 for every μ, the divergence form and the eight products (In [18]), and once more with a calculator's method, a central difference (In [19]).

statement	status	where it is verified
$\{\gamma^\mu, \Omega_\mu\} = 0$ for each μ	PROVED	Notebook 06a, In [18]; at $b_4 = 1$: `python-field-theory.json`, check `anticommutator_gamma_Ome ga_vanishes`
the divergence form equals $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$	PROVED	`wolfram-field-theory-b4.json`, check `gammaOme ga_divergence_form`; at $b_4 = 1$: `python-field-theory.json`, check `divergence_of_sqrtg_gamma`; `wolfram-field-theory.json`, check `gammaOme ga_divergence_form`; lead report, check `gamma_Om ega_divergence_formula`
the central differences reproduce $3H$ to 1.9×10^{-9} and $\frac{3}{4}b_4'/b_4$ (for $b_4 = 1 + \frac{1}{2}\sin x_4$) to 2.5×10^{-11}	COMPUTED (Notebook 06a, In [19]; the expected size of the first error is $18h^2 = 1.8 \times 10^{-9}$ for the step $h = 10^{-5}$)	Notebook 06a only

6.10 The Dirac operator written out, non-triviality and the negative control

The Dirac operator. Combine $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ with the contraction:

$$\sum_\mu \gamma^\mu D_\mu \Psi = \sum_a \frac{1}{f_a}\gamma^{(a)}\partial_a \Psi + 3H\gamma^{(x0)}\Psi + \frac{3}{4}\,\frac{b_4'}{b_4}\,\gamma^{(x4)}\Psi,$$

that is, with the factors written out,

$$\begin{aligned}&\tan z\,\gamma^{(x0)}\partial_0\Psi + e^{-a_4}s^{-1}\big(\gamma^{(x1)}\partial_1 + \gamma^{(x2)}\partial_2 + \gamma^{(x3)}\partial_3\big)\Psi + \gamma^{(x4)}\partial_4\Psi \\ &\quad + e^{a_4}b_4^{-1/2}s^{-1}\big(\gamma^{(x5)}\partial_5 + \gamma^{(x6)}\partial_6 + \gamma^{(x7)}\partial_7\big)\Psi + 3H\gamma^{(x0)}\Psi + \frac{3}{4}\,\frac{b_4'}{b_4}\,\gamma^{(x4)}\Psi.\end{aligned}$$

This is the left side of the field equation of both fields; the right side is $(m + U'(S))\Psi$ with $S = \bar\Psi\Psi$ (Chapter 7). The Revision records write out all sixteen component equations (formula `field_equation_components`, with b_4 and at $b_4 = 1$), and Notebook 06a compares each of them with its own computation (In [20]). **Where the deflation enters.** It does not enter $\gamma^\mu\Omega_\mu$, but it enters every derivative term through the factors $1/f_a$: as 3-space inflates the 3-space derivatives are weighted less and less (e^{-a_4}/s), and as the extra times deflate the extra-time derivatives are weighted more and more ($e^{a_4}b_4^{-1/2}/s$). Figure 06a.9 draws these factors.

The block form. In the chiral split $\Psi = (\psi_-, \psi_+)$ (components 1 to 8 and 9 to 16, Chapter 5) every gamma of the author has zero blocks on the diagonal, $\gamma^{(xa)} = \begin{pmatrix} 0 & \bar\tau_a \\ \tau_a & 0 \end{pmatrix}$, and $\gamma^{(x0)}$ has the 8×8 identity in both off-diagonal blocks. So $3H\gamma^{(x0)}$ maps the first eight components onto the

last eight and back with the factor $3H$, which is why its picture (figure 06a.7) shows the value 3 on two diagonal lines. Record: formula `field_equation_blocks`; `wolfram-field-theory.json`, check `block_form`.

Non-triviality. Write $M = 3H\gamma^{(x0)} + c\,\gamma^{(x4)}$ with $c = \frac{3}{4}b_4'/b_4$ for the term of the field equation. Line by line,

$$MM = 9H^2\gamma^{(x0)}\gamma^{(x0)} + 3Hc\big(\gamma^{(x0)}\gamma^{(x4)} + \gamma^{(x4)}\gamma^{(x0)}\big) + c^2\gamma^{(x4)}\gamma^{(x4)} = \big(9H^2 - c^2\big)\,1$$

(multiply out; $(\gamma^{(x0)})^2 = \eta_{00}1 = 1$, $(\gamma^{(x4)})^2 = \eta_{44}1 = -1$, and the middle bracket is the anticommutator of two different gammas, which is 0). When $9H^2 - c^2 \neq 0$, $M\Psi = 0$ gives $\Psi = MM\Psi/(9H^2 - c^2) = 0$ (multiply from the left by M and divide by the number). So **the term vanishes only for $\Psi = 0$, except at times where** $|b_4'/b_4| = 4H$ ($c = \pm 3H$). For a constant b_4, in particular on the backgrounds $b_4 = 1$, $M = 3H\gamma^{(x0)}$ and the exception never occurs: for every $H > 0$ and every field that is not identically zero, the term is present (PROVED; at $b_4 = 1$ `wolfram-field-theory.json`, checks `nontriviality_1_dirac16complex` and `nontriviality_2_dirac16complex00`, for the two fields). Where $|b_4'/b_4| = 4H$, $M = 3H(\gamma^{(x0)} \pm \gamma^{(x4)})$ is not zero but squares to zero, and its rank is 8: there it sends an 8-dimensional set of nonzero fields to zero (`python-field-theory-b4.json`, check `gravity_term_square_b4`; Notebook 06a, In [20], computes the rank exactly). Section 6.29 states precisely how far these statements reach.

The negative control: extra times that INFLATE. Is the cancellation of the a_4' terms caused by the deflation, or is it an accident of the algebra? Take $b_4 = 1$, so that only the deflation is changed, and change the metric so that the extra times inflate like 3-space: $g_{tt} = -e^{+2a_4}\sin^{1/3} z$, factors $f_t = e^{+a_4}s$. Repeat Section 6.6: now $\partial_4 f_t = +a_4' f_t$, so

$$\omega_{x_t\,x_t\,x4} = (-1)\frac{a_4' f_t}{1} = -a_4' f_t, \qquad \omega_{x_t\,x4\,x_t} = +a_4' f_t, \qquad \omega_{x_t\,x_t\,x0} = -H f_t$$

(the formula $\omega_{\mu\,\mu b} = \eta_{\mu\mu}\partial_b f_\mu / f_b$; the antisymmetry; $\partial_0 f_t$ is unchanged), and $\Omega_{x_t} = f_t(a_4' S^{x4\,x_t} - HS^{x_t\,x0})$. Repeat Section 6.8:

$$\gamma^{x5}\Omega_{x5} = \tfrac{1}{2}a_4'\,\gamma^{(x5)}\gamma^{(x4)}\gamma^{(x5)} - \tfrac{1}{2}H\,\gamma^{(x5)}\gamma^{(x5)}\gamma^{(x0)} = \tfrac{1}{2}a_4'\,\gamma^{(x4)} + \tfrac{1}{2}H\,\gamma^{(x0)}$$

(the same two products as before). Now all six warped directions contribute $+\frac{1}{2}a_4'$ along the time, and

$$\sum_\mu \gamma^\mu\Omega_\mu\Big|_{\text{control}} = 3a_4'\,\gamma^{(x4)} + 3H\,\gamma^{(x0)}.$$

The divergence form of Section 6.9 agrees. With inflating extra times the volume factor is $\sqrt{|g|} = \cot z \cdot e^{3a_4}s^3 \cdot 1 \cdot e^{3a_4}s^3 = e^{6a_4}\cos z$, which now changes with time, and

$$\frac{1}{2\sqrt{|g|}}\,\partial_4\Big(\prod_{c\neq x4} f_c\Big) = \frac{\partial_4(e^{6a_4}\cos z)}{2e^{6a_4}\cos z} = \frac{6a_4' e^{6a_4}\cos z}{2e^{6a_4}\cos z} = 3a_4'$$

(the product without $x4$ is the whole volume factor because $f_4 = 1$; the chain rule gives $\partial_4 e^{6a_4} = 6a_4' e^{6a_4}$; then cancel). **The cancellation of the a_4' terms in the author's metric is caused by the deflation.** The hidden term $3H\gamma^{(x0)}$ is the same in both cases: it comes from $x0$, not from the time dependence. PROVED; lead report, check `negative_control_inflating_extra_times`; Notebook 06a, In [22], and figure 10.

statement	status	where it is verified
the Dirac operator written out; all 16 components	PROVED	`field-theory-b4.json`, formulas `field_equation` and `field_equation_components`; `wolfram-field-theory-b4.json`, checks `Dirac_operator_explicit_G` and `Dirac_operator_explicit_C`; at $b_4 = 1$ the same formulas of `field-theory.json` and checks of `wolfram-field-theory.json`
$MM = (9H^2 - \frac{9}{16}(b_4'/b_4)^2)\,1$: the term vanishes only for $\Psi = 0$ unless $\lvert b_4'/b_4\rvert = 4H$ (never for a constant b_4); there M has rank 8	PROVED, in the diagonal vielbein (scope: Section 6.29)	`python-field-theory-b4.json`, check `gravity_term_square_b4`; at $b_4 = 1$: `wolfram-field-theory.json`, checks `nontriviality_1_dirac16complex`, `nontriviality_2_dirac16complex00`
negative control (at $b_4 = 1$): $3a_4'\gamma^{(x4)} + 3H\gamma^{(x0)}$ with inflating extra times	PROVED	lead report, check `negative_control_inflating_extra_times`

6.11 Example: Notebook 06a

Notebook 06a puts Sections 6.2 to 6.10 to work with exact algebra. It reads the author's eight gammas from the record `Revision/algebra/gammas.json`, checks the Clifford relations, and defines four general functions (Christoffel symbols, canonical spin connection, spinor connection, curved gammas) that work in any number of dimensions. It first applies them to the polar plane of Section 6.5 and draws the turning frame and the half-angle spinor rotation. Then it builds the author's metric with arbitrary functions $a_4(x_4)$ and $b_4(x_4)$ and computes, exactly: the vielbein, the volume factor, the 25 Christoffel symbols, the 12 components of the spin connection, the 512 equations of the vielbein postulate, Ω_μ, the covariant constancy of the gammas, the eight terms $\gamma^\mu\Omega_\mu$ and their sum $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$, the divergence form, the Dirac operator for sixteen arbitrary component functions, the square of the term, and the negative control. It compares every result that the Revision records also contain with the records (with b_4, and at $b_4 = 1$), draws ten figures, needs no Rust, runs in about a minute and ends with the line ALL 36 CHECKS PASSED (notebook 06a).

6.12 How to run Notebook 06a

Step 1. What this notebook does and what it needs.

Notebook 06a (Vielbein, spin connection and the term gamma^mu Omega_mu of the field equation) is the file `Revision/textbook/notebooks/06a_spin_connection.ipynb` of the repository Dirac_claude. Starting from the author's metric with the functions a4(x4) and b4(x4) and the author's real 16 x 16 gamma matrices, it computes exactly, with sympy, the diagonal vielbein, the Christoffel symbols, the canonical spin connection, the spinor connection Omega_mu, the covariant derivative of a spinor and the term gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) of the field equation; it shows that the a4' terms of the three inflating and the three deflating directions cancel while a b4' term remains (zero for constant b4, as on the backgrounds b4 = 1), checks the results against the Revision records, repeats the computation for a negative control with inflating extra times, works a two-dimensional warm-up (the polar plane), and draws ten teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
```

```
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 06a_spin_connection.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows

a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 06a_spin_connection.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 06a_spin_connection.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/06a.captions.json`
- `Revision/textbook/figures/06a_1_polar_frame_and_spin_rotation.png`
- `Revision/textbook/figures/06a_2_vielbein_factors.png`
- `Revision/textbook/figures/06a_3_christoffel_pattern.png`
- `Revision/textbook/figures/06a_4_omega_heat_maps.png`
- `Revision/textbook/figures/06a_5_spinor_connection_heat_maps.png`
- `Revision/textbook/figures/06a_6_gamma_omega_per_direction.png`
- `Revision/textbook/figures/06a_7_gamma_omega_cancellation.png`
- `Revision/textbook/figures/06a_8_divergence_form.png`
- `Revision/textbook/figures/06a_9_derivative_coefficients.png`
- `Revision/textbook/figures/06a_10_negative_control.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all ten figure files exist
ALL 36 CHECKS PASSED (notebook 06a)
```

and the notebook must show 10 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/06a_spin_connection.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 06a_spin_connection.ipynb
python -m nbconvert --execute --inplace 06a_spin_connection.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for Revision/algebra/gammas.json or for a report: the notebook reads Revision records from the repository; open it inside the folder Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a single downloaded notebook file is not enough).
- a cell runs for many minutes: the exact algebra of sympy is slow on old computers; the whole notebook needs about a minute on a 2024 laptop, and up to about two minutes while other programs use the processor. Wait, or close the other programs.

6.13 Notebook 06a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/06a_spin_connection.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A spinor field (a column of 16 numbers at every point, such as the author's fields
dirac16complex and dirac16complex00) does not see the metric directly. It sees it
through a **vielbein** (a set of eight perpendicular unit directions at every point)
and through the **spin connection** (the rate at which these directions turn from
point to point). This notebook computes both, exactly, for the author's primordial
metric with its two functions $a_4(x_4)$ and $b_4(x_4)$, and the one combination
that enters the field equation,

$$\gamma^\mu \Omega_\mu = 3 H \gamma^{(x0)} + \tfrac34\,\frac{b_4'}{b_4}\,
\gamma^{(x4)} ,$$

which is $3H\gamma^{(x0)}$ when $b_4$ is constant, as on the backgrounds of this
book ($b_4 = 1$). It

- starts with a warm-up in two dimensions (the flat plane in polar coordinates),
  where every step can be followed by hand;
- builds the diagonal vielbein of the author's metric and checks that it gives the
  metric back;
- computes the 25 independent nonzero Christoffel symbols, the canonical spin
  connection $\omega_{\mu ab}$ (12 independent nonzero components) and the spinor
  connection $\Omega_\mu$ (16 x 16 matrices built from the author's gammas);
- checks the vielbein postulate (512 equations), the antisymmetry of $\omega$ and
  the covariant constancy of the gammas (64 matrix equations);
- computes $\gamma^\mu\Omega_\mu$ direction by direction and shows that the
  $a_4'$ terms along the time $x4$ cancel between the three inflating directions
```

```
  $x1, x2, x3$ and the three deflating extra times $x5, x6, x7$, that the extra
  times leave the term $\frac34(b_4'/b_4)\gamma^{(x4)}$, and that the six terms
  along the hidden direction $x0$ add up to $3H\gamma^{(x0)}$;
- explains why the function $a_4(x_4)$ drops out and $b_4(x_4)$ does not (the
  divergence form);
- writes the covariant derivative and the Dirac operator out in full and compares
  all 16 component equations with the Revision records (with $b_4$, and at
  $b_4 = 1$);
- shows when the term vanishes for a nonzero field: never when $b_4$ is
  constant, and only where $|b_4'/b_4| = 4H$ otherwise;
- repeats the computation for a NEGATIVE CONTROL, a metric whose extra times
  inflate instead of deflating (at $b_4 = 1$): there a term
  $3 a_4' \gamma^{(x4)}$ survives;
- draws ten teaching plots.

Every result that the Revision record also contains is compared with the record,
and the check prints the record file and the name of its check.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 06a (Vielbein, spin connection and the term gamma^mu Omega_mu of the field
equation) is the file ‘Revision/textbook/notebooks/06a_spin_connection.ipynb‘ of the
repository Dirac_claude. Starting from the author’s metric with the functions a4(x4) and
b4(x4) and the author’s real 16 x 16 gamma matrices, it computes exactly, with sympy, the
diagonal vielbein, the Christoffel symbols, the canonical spin connection, the spinor
connection Omega_mu, the covariant derivative of a spinor and the term gamma^mu Omega_mu
= 3 H gamma^(x0) + (3/4)(b4’/b4) gamma^(x4) of the field equation; it shows that the a4’
terms of the three inflating and the three deflating directions cancel while a b4’ term
remains (zero for constant b4, as on the backgrounds b4 = 1), checks the results against
the Revision records, repeats the computation for a negative control with inflating extra
times, works a two-dimensional warm-up (the polar plane), and draws ten teaching plots.
It needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
```

"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in
this one window only; it changes nothing permanently.

macOS (zsh):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 06a_spin_connection.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 06a_spin_connection.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 06a_spin_connection.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/06a.captions.json‘
- ‘Revision/textbook/figures/06a_1_polar_frame_and_spin_rotation.png‘
- ‘Revision/textbook/figures/06a_2_vielbein_factors.png‘
- ‘Revision/textbook/figures/06a_3_christoffel_pattern.png‘
- ‘Revision/textbook/figures/06a_4_omega_heat_maps.png‘
- ‘Revision/textbook/figures/06a_5_spinor_connection_heat_maps.png‘
- ‘Revision/textbook/figures/06a_6_gamma_omega_per_direction.png‘
- ‘Revision/textbook/figures/06a_7_gamma_omega_cancellation.png‘
- ‘Revision/textbook/figures/06a_8_divergence_form.png‘
- ‘Revision/textbook/figures/06a_9_derivative_coefficients.png‘
- ‘Revision/textbook/figures/06a_10_negative_control.png‘

It changes no other file of the repository; running it headless or saving it in

JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all ten figure files exist
ALL 36 CHECKS PASSED (notebook 06a)
```

and the notebook must show 10 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/06a_spin_connection.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 06a_spin_connection.ipynb
  python -m nbconvert --execute --inplace 06a_spin_connection.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues

and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for Revision/algebra/gammas.json or for a report: the notebook reads Revision records from the repository; open it inside the folder Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a single downloaded notebook file is not enough).
- a cell runs for many minutes: the exact algebra of sympy is slow on old computers; the whole notebook needs about a minute on a 2024 laptop, and up to about two minutes while other programs use the processor. Wait, or close the other programs.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 06a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 06a (Vielbein, spin connection and the term gamma^mu Omega_mu of the field
# equation) is the file Revision/textbook/notebooks/06a_spin_connection.ipynb of the
# repository Dirac_claude. Starting from the author's metric with the functions a4(x4)
# and b4(x4) and the author's real 16 x 16 gamma matrices, it computes exactly, with
# sympy, the diagonal vielbein, the Christoffel symbols, the canonical spin connection,
# the spinor connection Omega_mu, the covariant derivative of a spinor and the term
# gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) of the field equation; it
# shows that the a4' terms of the three inflating and the three deflating directions
# cancel while a b4' term remains (zero for constant b4, as on the backgrounds b4 = 1),
# checks the results against the Revision records, repeats the computation for a negative
# control with inflating extra times, works a two-dimensional warm-up (the polar plane),
# and draws ten teaching plots. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
```

```
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 06a_spin_connection.ipynb
#
```

```
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 1 minute
# on a typical laptop. Scroll to the end and compare the last printed lines with the
# lines in the step "What the notebook writes and what you must see" below. To keep the
# results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the
# menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 06a_spin_connection.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 06a_spin_connection.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/06a.captions.json
# - Revision/textbook/figures/06a_1_polar_frame_and_spin_rotation.png
# - Revision/textbook/figures/06a_2_vielbein_factors.png
# - Revision/textbook/figures/06a_3_christoffel_pattern.png
# - Revision/textbook/figures/06a_4_omega_heat_maps.png
# - Revision/textbook/figures/06a_5_spinor_connection_heat_maps.png
# - Revision/textbook/figures/06a_6_gamma_omega_per_direction.png
# - Revision/textbook/figures/06a_7_gamma_omega_cancellation.png
# - Revision/textbook/figures/06a_8_divergence_form.png
# - Revision/textbook/figures/06a_9_derivative_coefficients.png
# - Revision/textbook/figures/06a_10_negative_control.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
```

```
#
#     PASS all ten figure files exist
#     ALL 36 CHECKS PASSED (notebook 06a)
#
# and the notebook must show 10 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/06a_spin_connection.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 06a_spin_connection.ipynb
#     python -m nbconvert --execute --inplace 06a_spin_connection.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json or for a report: the notebook
#   reads Revision records from the repository; open it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a
#   single downloaded notebook file is not enough).
# - a cell runs for many minutes: the exact algebra of sympy is slow on old computers;
#   the whole notebook needs about a minute on a 2024 laptop, and up to about two minutes
#   while other programs use the processor. Wait, or close the other programs.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
```

```
# ============================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "06a"  # this notebook: chapter 06, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))
```

```
def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 06a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x0, \dots, x7$: the eight numbers that name a point, with the
  author's names: $x0$ the **hidden direction**; $x1, x2, x3$ ordinary 3-space;
  $x4$ the time; $x5, x6, x7$ the three **extra times**. In Python lists they
  have the positions 0 to 7 (Python counts from 0) in the same order, so `x[0]`
  is $x0$ and `x[4]` is $x4$.
- **Metric** $g_{\mu\nu}$: the rule that turns small coordinate steps into squared
  lengths, $ds^2 = \sum_\mu g_{\mu\mu} (dx^\mu)^2$ for the author's (diagonal)
  metric. A positive $g_{\mu\mu}$ makes direction $\mu$ **space-like**, a negative
  one **time-like**.
- **Signature (4,4)**: four space-like directions ($x0, x1, x2, x3$) and four
  time-like ones ($x4, x5, x6, x7$).
- **Frame metric** $\eta_{ab}$: the constant diagonal matrix
  $\mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the order $x0, \dots, x7$.
- **Vielbein** $e^a{}_\mu$ (German "many legs"): at every point eight directions
  that are perpendicular and of unit length, so that
  $g_{\mu\nu} = \sum_{a} \eta_{aa} e^a{}_\mu e^a{}_\nu$. The index $a$ (a
  **frame index**) numbers the eight directions; $\mu$ (a **coordinate index**)
  numbers the coordinates. Its **inverse** $e_a{}^\mu$ satisfies
  $\sum_\mu e^a{}_\mu e_b{}^\mu = \delta^a_b$ (1 if $a = b$, else 0).
- **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$: the numbers that say how the
  coordinate directions change from point to point; computed from the first
  derivatives of the metric.
- **Spin connection** $\omega_\mu{}^a{}_b$: the numbers that say how the vielbein
  turns when one moves along $x^\mu$. The **canonical** one is fixed by the
  **vielbein postulate** (below). Lowering the first frame index with $\eta$ gives
  $\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b$, which is **antisymmetric**:
  $\omega_{\mu ab} = -\omega_{\mu ba}$.
- **Gamma matrices** $\gamma^a$: the author's eight real 16 x 16 matrices with
  $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\,1$ (the **Clifford
  relation**; $1$ is the 16 x 16 identity matrix). **Curved gammas**:
  $\gamma^\mu = \sum_a e_a{}^\mu \gamma^a$.
- **Commutator** $[A, B] = AB - BA$; **anticommutator** $\{A, B\} = AB + BA$.
- $S^{ab} = \frac14[\gamma^a, \gamma^b]$: the 28 matrices (one for each pair
  $a < b$) that turn spinors when the frame is turned.
```

```
- **Spinor connection** $\Omega_\mu = \frac12 \sum_{a,b}\omega_{\mu ab} S^{ab}$: a
  16 x 16 matrix for every $\mu$.
- **Covariant derivative** of a spinor: $D_\mu \Psi = \partial_\mu \Psi +
  \Omega_\mu \Psi$ ($\partial_\mu$ is the partial derivative with respect to
  $x^\mu$). **Dirac operator**: $\gamma^\mu D_\mu = \sum_\mu \gamma^\mu D_\mu$.
- **Trace** $\mathrm{tr}\,M$: the sum of the diagonal entries of a square matrix.
- **Exact** computation: with symbols and fractions (sympy), not with rounded
  decimal numbers; an exact check that an expression is zero proves it for ALL
  values of the symbols.
- **Negative control**: a deliberately changed case in which the effect must NOT
  occur (or must occur differently); it shows that a check can fail and that the
  result depends on the feature that was changed.
- **Heat map**: a picture of a matrix in which every entry is a small coloured
  square; here red is positive, blue is negative and white is zero.
```

Text cell:

```
## 4. The physical and mathematical situation

**The author's metric.** With $z = 6 H x_0$ in the interval $(0, \pi/2)$, a positive
constant $H$ and two functions of the time, $a_4(x_4)$ and $b_4(x_4) > 0$,

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4}\sin^{1/3}z,\ e^{2a_4}\sin^{1/3}z,\
e^{2a_4}\sin^{1/3}z,\ -1,\ -e^{-2a_4}b_4\sin^{1/3}z,\ -e^{-2a_4}b_4\sin^{1/3}z,\
-e^{-2a_4}b_4\sin^{1/3}z\big)$$

in the order $x0, \dots, x7$. When $a_4$ grows, lengths along $x1, x2, x3$ grow
like $e^{a_4}$ (3-space **inflates**) and lengths along the extra times
$x5, x6, x7$ shrink like $e^{-a_4}$ (the extra times **deflate exponentially**);
$b_4$ multiplies the squared lengths of the extra times. On the backgrounds of this
book $b_4 = 1$ (Chapter 3: an empty spacetime has a constant $b_4$, and a constant
$b_4$ only rescales $x5, x6, x7$); every result below holds for every $b_4$.

**The diagonal vielbein.** Because $g$ is diagonal, the vielbein can be chosen
diagonal: $e^a{}_\mu = f_a\,\delta^a_\mu$ with $f_a = \sqrt{|g_{aa}|}$:

$$f = \big(\cot z,\ e^{a_4}s,\ e^{a_4}s,\ e^{a_4}s,\ 1,\ e^{-a_4}b_4^{1/2}s,\
e^{-a_4}b_4^{1/2}s,\ e^{-a_4}b_4^{1/2}s\big), \qquad s = \sin^{1/6} z \,.$$

**The formulas used** (the standard ones of general relativity with spinors):

$$\Gamma^\lambda{}_{\mu\nu} = \tfrac12 \sum_\rho g^{\lambda\rho}\big(\partial_\mu
g_{\rho\nu} + \partial_\nu g_{\rho\mu} - \partial_\rho g_{\mu\nu}\big),$$

$$\omega_\mu{}^a{}_b = \sum_\nu e^a{}_\nu \Big(\partial_\mu e_b{}^\nu +
\sum_\lambda \Gamma^\nu{}_{\mu\lambda} e_b{}^\lambda\Big), \qquad
\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b \,,$$

$$\Omega_\mu = \tfrac12 \sum_{a,b} \omega_{\mu ab} S^{ab}, \qquad D_\mu\Psi =
\partial_\mu\Psi + \Omega_\mu\Psi, \qquad \gamma^\mu = \sum_a e_a{}^\mu\gamma^a \,.$$

The formula for $\omega$ is the unique solution of the **vielbein postulate**
$\partial_\mu e^a{}_\nu - \sum_\lambda \Gamma^\lambda{}_{\mu\nu} e^a{}_\lambda +
```

```
\sum_b \omega_\mu{}^a{}_b e^b{}_\nu = 0$: the vielbein is constant for the full
covariant derivative. The notebook checks this equation in all 512 components.

**What is PROVED here (exact, sympy) and what it means.** In the diagonal vielbein,
$\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac34(b_4'/b_4)\gamma^{(x4)}$ for every
$H > 0$ and all functions $a_4(x_4)$ and $b_4(x_4) > 0$; for a constant $b_4$ it is
$3H\gamma^{(x0)}$. This term appears in the field equation $\gamma^\mu D_\mu\Psi =
(m + U'(S))\Psi$ of both fields. Its value belongs to the diagonal vielbein: the
next notebook of this chapter (06b) shows that in another vielbein of the same
metric the contraction changes, and even vanishes, and that a rescaling of the
field removes it. What no choice can remove is the curvature of the metric.
```

Text cell:

```
## 5. The tools: sympy and the author's gamma matrices

The next cell imports numpy (numbers for the plots) and sympy (exact algebra),
reads the author's eight gamma matrices from the Revision record
`Revision/algebra/gammas.json`, and checks two of their properties: the 64
Clifford relations, and that there are eight of them, each a 16 x 16 matrix whose
entries are all real numbers. It then builds the 28 matrices $S^{ab} =
\frac14[\gamma^a, \gamma^b]$.

The first lines of the next cell change one setting of Jupyter: printed text is
sent to the screen when a cell has finished (or just before a figure is shown),
not every 0.2 seconds. This changes no result; it makes the printed text arrive in
the same pieces in every run, which the book's checking tool needs.
```

In [2]:

```
import sys  # the Python system module (here: the stream of printed text)

# Jupyter sends printed text to the screen in pieces, about every 0.2 seconds.  The
# book's checking tool reads each PASS line together with its "reproduces" line, so
# the pieces must be the same in every run: send the printed text of a cell in one
# piece when the cell ends (or just before a figure), at the latest after 600 s.
if hasattr(sys.stdout, "flush_interval"):  # true inside Jupyter only
    sys.stdout.flush_interval = 600.0
import numpy as np  # arrays of decimal numbers (only for the plots)
import sympy as sp  # exact algebra and calculus with symbols

GAMMA_FILE = "Revision/algebra/gammas.json"  # a Revision record (read only)
fixture = json.loads(repository_file(GAMMA_FILE).read_text(encoding="utf-8"))
NAMES = fixture["coordinates"]  # ["x0", ..., "x7"], the names of the directions
ETA = fixture["eta"]  # the frame metric: +1 space-like, -1 time-like
# Each gamma is stored as 16 rows of 16 whole numbers; sp.Matrix makes it an exact
# sympy matrix.  gamma[a] is the gamma of the direction NAMES[a].
gamma = [sp.Matrix(rows) for rows in fixture["gamma"]]
I16 = sp.eye(16)  # the 16 x 16 identity matrix
Z16 = sp.zeros(16, 16)  # the 16 x 16 zero matrix
say(f"directions: {NAMES}")
say(f"eta (frame metric): {ETA}")
```

```
# The Clifford relation for all 8 x 8 = 64 pairs (a, b): the anticommutator is
# 2 eta_aa times the identity when a = b and the zero matrix when a and b differ.
clifford = all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
               == (2 * ETA[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
check(clifford, "the 64 Clifford relations of the author's gammas",
      record="Revision/theory/reports/python-field-theory.json, "
             "check clifford_relations")
# eight matrices, each with 16 rows and 16 columns, and every entry a real number
check(len(gamma) == 8 and all(matrix.shape == (16, 16) for matrix in gamma)
      and all(entry.is_real for matrix in gamma for entry in matrix),
      "the eight gammas are 16 x 16 matrices with real entries",
      record="Revision/theory/reports/python-field-theory.json, check gammas_real")
# S[a][b] = (1/4)(gamma^a gamma^b - gamma^b gamma^a), for all 64 pairs (a, b).
S = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
     for a in range(8)]
```

Out [2]:

```
directions: ['x0', 'x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7']
eta (frame metric): [1, 1, 1, 1, -1, -1, -1, -1]
PASS the 64 Clifford relations of the author's gammas
     reproduces Revision/theory/reports/python-field-theory.json, check
    clifford_relations
PASS the eight gammas are 16 x 16 matrices with real entries
     reproduces Revision/theory/reports/python-field-theory.json, check gammas_real
```

Text cell:

```
The next cell defines four small helpers that the rest of the notebook uses.

- `is_zero(expr)` decides EXACTLY whether a sympy expression is zero for all values
  of its symbols. It first tries a fast route (write every sine and cosine through
  exponential functions, multiply out, cancel common factors) and then sympy's
  general `simplify`. It never says "zero" for an expression that is not zero; at
  worst it fails to recognise a zero, and then a check fails visibly.
- `matrix_is_zero(M)` applies `is_zero` to every entry of a matrix.
- `record_passed(path, name)` opens a Revision report (a JSON file) and returns
  True when the report records the check `name` as passed.
- `record_formula(key, names, formulas)` reads a formula of a record, written in
  the notation of the Wolfram Language, and turns it into a sympy expression with
  sympy's reader `parse_mathematica`. Two records are read:
  `Revision/theory/field-theory.json` (the metric at $b_4 = 1$) and
  `Revision/theory/field-theory-b4.json` (the metric with $b_4(x_4)$).
```

In [3]:

```
from sympy.parsing.mathematica import parse_mathematica  # reads Wolfram notation


def is_zero(expr):
```

```
    """True when expr is exactly zero for all values of its symbols."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    # Fast route: sin, cos, tan, ... written through exp, multiplied out, cancelled.
    if sp.cancel(sp.expand(expr.rewrite(sp.exp))) == 0:
        return True
    return sp.simplify(expr) == 0  # the slower general route


def matrix_is_zero(matrix):
    """True when every entry of the matrix is exactly zero."""
    return all(is_zero(entry) for entry in matrix)


def record_passed(path, name):
    """True when the Revision report at path records the check name as passed."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper() == "PASS"  # "PASS" or "pass"
    raise KeyError(f"{path} has no check {name}")


def read_formulas(path):
    """The formulas of a record as a dictionary key -> text (or list of texts)."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    return {item["key"]: item["wl"] for item in data["formulas"]}


THEORY_FILE = "Revision/theory/field-theory.json"  # the record at b4 = 1
THEORY_B4_FILE = "Revision/theory/field-theory-b4.json"  # the record with b4(x4)
FORMULAS = read_formulas(THEORY_FILE)
FORMULAS_B4 = read_formulas(THEORY_B4_FILE)


def wolfram_to_sympy(text, names):
    """A text in Wolfram notation as a sympy expression.  names maps the record's
    symbol names (texts) to the symbols and functions of this notebook."""
    # The record writes a4', a4'', a4(x4) and the same for b4 as Derivative[1][a4][x4],
    # Derivative[2][a4][x4], a4[x4], ...; give them short names before reading.
    for long_form, short_form in (("Derivative[1][a4][x4]", "a4p"),
                                  ("Derivative[2][a4][x4]", "a4pp"),
                                  ("Derivative[1][b4][x4]", "b4p"),
                                  ("Derivative[2][b4][x4]", "b4pp"),
                                  ("a4[x4]", "a4"), ("b4[x4]", "b4")):
        text = text.replace(long_form, short_form)
    expr = parse_mathematica(text)
    return expr.subs({sp.Symbol(name): value for name, value in names.items()},
                     simultaneous=True)


def record_formula(key, names, formulas=FORMULAS):
    """The formula key of a record (field-theory.json unless formulas says
```

```
    otherwise) as a sympy expression."""
    return wolfram_to_sympy(formulas[key], names)


REPORT_PY = "Revision/theory/reports/python-field-theory.json"
REPORT_WL = "Revision/theory/reports/wolfram-field-theory.json"
REPORT_PY_B4 = "Revision/theory/reports/python-field-theory-b4.json"
REPORT_WL_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
REPORT_LEAD = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
METRIC_B4_FILE = "Revision/metric_b4/results/metric-b4.json"
say(f"{len(FORMULAS)} formulas read from {THEORY_FILE}")
say(f"{len(FORMULAS_B4)} formulas read from {THEORY_B4_FILE}")
```

Out [3]:

```
32 formulas read from Revision/theory/field-theory.json
8 formulas read from Revision/theory/field-theory-b4.json
```

Text cell:

```
## 6. Four general formulas as Python functions

The next cell turns the formulas of Section 4 into four Python functions. They work
in ANY number of dimensions and for ANY vielbein, so the same functions serve the
two-dimensional warm-up and the author's eight-dimensional metric.

- `christoffel(g, coords)`: the Christoffel symbols of the metric matrix `g`;
  `Gam[l][m][n]` is $\Gamma^l{}_{mn}$. Because $\Gamma^l{}_{mn} = \Gamma^l{}_{nm}$,
  each pair $(m, n)$ is computed once.
- `spin_connection(e, Gam, coords, eta)`: the canonical spin connection with the
  first frame index lowered, `omega[mu][a][b]` $= \omega_{\mu ab}$. The vielbein is
  the matrix `e` with `e[a, mu]` $= e^a{}_\mu$ (row = frame index, column =
  coordinate index); its inverse matrix `E = e.inv()` has `E[mu, a]` $=
  e_a{}^\mu$, because $\sum_\mu e^a{}_\mu e_b{}^\mu = \delta^a_b$ says exactly
  `e * E = 1`.
- `spinor_connection(omega, S)`: $\Omega_\mu = \frac12\sum_{a,b}\omega_{\mu ab}
  S^{ab}$. Since both $\omega_{\mu ab}$ and $S^{ab}$ change sign when $a$ and $b$
  are exchanged, the terms $(a, b)$ and $(b, a)$ are equal, and the factor
  $\frac12$ is used up by summing over $a < b$ only.
- `curved_gammas(e, gammas)`: $\gamma^\mu = \sum_a e_a{}^\mu \gamma^a$.

Every nonzero result is simplified with sympy's `simplify`, so that it prints in a
short form.
```

In [4]:

```
def christoffel(g, coords):
    """Gam[l][m][n] = (1/2) sum_r g^lr (d_m g_rn + d_n g_rm - d_r g_mn)."""
    n = len(coords)
    g_inv = g.inv()  # the inverse metric g^lr
    Gam = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
```

```
    for l in range(n):
        for m in range(n):
            for k in range(m, n):  # Gamma^l_mk = Gamma^l_km: compute it once
                value = sum(g_inv[l, r] * (sp.diff(g[r, k], coords[m])
                                           + sp.diff(g[r, m], coords[k])
                                           - sp.diff(g[m, k], coords[r]))
                            for r in range(n) if g_inv[l, r] != 0) / 2
                value = sp.simplify(value)
                Gam[l][m][k] = value
                Gam[l][k][m] = value
    return Gam


def spin_connection(e, Gam, coords, eta):
    """omega[mu][a][b] = omega_{mu ab} = eta_aa omega_mu^a_b, where
    omega_mu^a_b = sum_nu e^a_nu (d_mu e_b^nu + sum_lam Gam^nu_{mu lam} e_b^lam)."""
    n = len(coords)
    E = e.inv()  # E[nu, b] = e_b^nu, the inverse vielbein
    omega = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for mu in range(n):
        for a in range(n):
            for b in range(n):
                mixed = 0  # will become omega_mu^a_b
                for nu in range(n):
                    if e[a, nu] == 0:
                        continue  # this term is zero
                    inner = sp.diff(E[nu, b], coords[mu]) + sum(
                        Gam[nu][mu][lam] * E[lam, b] for lam in range(n))
                    mixed += e[a, nu] * inner
                omega[mu][a][b] = sp.simplify(eta[a] * mixed)  # lower the index a
    return omega


def spinor_connection(omega, S):
    """Omega[mu] = (1/2) sum_{a,b} omega_{mu ab} S^ab = sum_{a<b} omega_{mu ab} S^ab."""
    n = len(omega)
    size = S[0][0].shape[0]  # 16 for the author's gammas, 2 in two dimensions
    Omega = []
    for mu in range(n):
        total = sp.zeros(size, size)
        for a in range(n):
            for b in range(a + 1, n):
                if omega[mu][a][b] != 0:
                    total += omega[mu][a][b] * S[a][b]
        Omega.append(total)
    return Omega


def curved_gammas(e, gammas):
    """gamma^mu = sum_a e_a^mu gamma^a for every coordinate mu."""
    E = e.inv()
    n = len(gammas)
    return [sum((E[mu, a] * gammas[a] for a in range(n)),
                sp.zeros(*gammas[0].shape)) for mu in range(n)]
```

```
say("defined: christoffel, spin_connection, spinor_connection, curved_gammas")
```

Out [4]:

```
defined: christoffel, spin_connection, spinor_connection, curved_gammas
```

Text cell:

```
## 7. A warm-up in two dimensions: the flat plane in polar coordinates

Before eight dimensions, two. A point of the flat plane has the polar coordinates
$r$ (distance from the centre) and $\varphi$ (angle). The metric is
$ds^2 = dr^2 + r^2 d\varphi^2$, so $g = \mathrm{diag}(1, r^2)$ and $\eta =
\mathrm{diag}(1, 1)$. The unit directions are the radial direction (frame index 0)
and the angular direction (frame index 1); a step $d\varphi$ has the length
$r\,d\varphi$, so the vielbein is $e = \mathrm{diag}(1, r)$. Two 2 x 2 matrices with
the Clifford relation of the plane are the Pauli matrices $\sigma_1$ and
$\sigma_2$, so $\gamma^0 = \sigma_1$ and $\gamma^1 = \sigma_2$.

The next cell computes everything with the general functions and checks the
results that can be worked by hand:
$\Gamma^r{}_{\varphi\varphi} = -r$, $\Gamma^\varphi{}_{r\varphi} = 1/r$;
$\omega_{\varphi 01} = -1$ (the frame turns by one radian per radian of
$\varphi$); $\Omega_\varphi = -\frac{i}{2}\sigma_3$; and
$\gamma^\mu\Omega_\mu = \sigma_1/(2r)$, so that the Dirac operator of the plane in
polar coordinates is $\sigma_1(\partial_r + \frac{1}{2r}) + \frac{\sigma_2}{r}
\partial_\varphi$. The plane is flat, so the curvature of the spinor connection,
$F_{r\varphi} = \partial_r\Omega_\varphi - \partial_\varphi\Omega_r +
[\Omega_r, \Omega_\varphi]$, must vanish; the cell checks that too. A change of
frame by a spin transformation $R(\varphi)$ changes the spinor connection into
$R\,\Omega_\varphi R^{-1} - (\partial_\varphi R)R^{-1}$. The Cartesian frame has
no connection, and the cell checks that $\Omega_\varphi = -(\partial_\varphi
U)U^{-1}$ for the spinor rotation $U(\varphi) = \cos\frac{\varphi}{2} +
i\sin\frac{\varphi}{2}\,\sigma_3$: this is the rule with $R = U$ applied to the
connection 0 of the Cartesian frame, so $U$ carries the Cartesian frame into the
polar frame. Finally it checks that the inverse rotation $U^{-1}$ removes the
connection: the rule with $R = U^{-1}$ gives $U^{-1}\Omega_\varphi U -
(\partial_\varphi U^{-1})U = 0$. In a flat space the spin connection is only the
turning of the chosen frame.
```

In [5]:

```
r, phi = sp.symbols("r phi", positive=True)  # polar coordinates, r > 0
sigma1 = sp.Matrix([[0, 1], [1, 0]])  # the three Pauli matrices
sigma2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
sigma3 = sp.Matrix([[1, 0], [0, -1]])
plane_gammas = [sigma1, sigma2]  # gamma^0 (radial), gamma^1 (angular)
plane_eta = [1, 1]
plane_S = [[(plane_gammas[a] * plane_gammas[b] - plane_gammas[b] * plane_gammas[a])
```

```
           / 4 for b in range(2)] for a in range(2)]
plane_g = sp.diag(1, r**2)  # ds^2 = dr^2 + r^2 dphi^2
plane_e = sp.diag(1, r)  # e^a_mu: unit radial and unit angular direction

plane_Gam = christoffel(plane_g, [r, phi])
plane_omega = spin_connection(plane_e, plane_Gam, [r, phi], plane_eta)
plane_Omega = spinor_connection(plane_omega, plane_S)
plane_gup = curved_gammas(plane_e, plane_gammas)
plane_slash = sp.simplify(plane_gup[0] * plane_Omega[0] + plane_gup[1] * plane_Omega[1])
say(f"Gamma^r_phiphi = {plane_Gam[0][1][1]},  Gamma^phi_rphi = {plane_Gam[1][0][1]}")
say(f"omega_phi01 = {plane_omega[1][0][1]},  omega_phi10 = {plane_omega[1][1][0]}")
say(f"Omega_phi = {plane_Omega[1].tolist()}")
say(f"gamma^mu Omega_mu = {plane_slash.tolist()}")
check(plane_Gam[0][1][1] == -r and plane_Gam[1][0][1] == 1 / r,
      "polar plane: Gamma^r_phiphi = -r and Gamma^phi_rphi = 1/r")
check(plane_omega[1][0][1] == -1 and plane_omega[1][1][0] == 1
      and plane_omega[0][0][1] == 0, "polar plane: omega_phi01 = -1, omega_r = 0")
check(plane_Omega[1] == -sp.I / 2 * sigma3 and plane_Omega[0] == sp.zeros(2, 2),
      "polar plane: Omega_phi = -(i/2) sigma3 and Omega_r = 0")
check(plane_slash == sigma1 / (2 * r), "polar plane: gamma^mu Omega_mu = sigma1/(2 r)")
F_rphi = (plane_Omega[1].diff(r) - plane_Omega[0].diff(phi)
          + plane_Omega[0] * plane_Omega[1] - plane_Omega[1] * plane_Omega[0])
U = sp.cos(phi / 2) * sp.eye(2) + sp.I * sp.sin(phi / 2) * sigma3  # spinor rotation
pure_gauge = -U.diff(phi) * U.inv() - plane_Omega[1]  # must be the zero matrix
removed = U.inv() * plane_Omega[1] * U - U.inv().diff(phi) * U  # frame change U^-1
check(F_rphi == sp.zeros(2, 2) and matrix_is_zero(pure_gauge)
      and matrix_is_zero(removed),
      "polar plane: flat (F_rphi = 0), Omega_phi = -(dU/dphi) U^-1, U^-1 removes it")
```

Out [5]:

```
Gamma^r_phiphi = -r,  Gamma^phi_rphi = 1/r
omega_phi01 = -1,  omega_phi10 = 1
Omega_phi = [[-I/2, 0], [0, I/2]]
gamma^mu Omega_mu = [[0, 1/(2*r)], [1/(2*r), 0]]
PASS polar plane: Gamma^r_phiphi = -r and Gamma^phi_rphi = 1/r
PASS polar plane: omega_phi01 = -1, omega_r = 0
PASS polar plane: Omega_phi = -(i/2) sigma3 and Omega_r = 0
PASS polar plane: gamma^mu Omega_mu = sigma1/(2 r)
PASS polar plane: flat (F_rphi = 0), Omega_phi = -(dU/dphi) U^-1, U^-1 removes it
```

Text cell:

```
The next cell draws two pictures of the warm-up. On the left, the unit radial
direction (black arrows) and the unit angular direction (orange arrows) at twelve
angles on two circles: the frame turns as one goes around, by exactly the angle
$\varphi$, which is why $\omega_{\varphi 01} = -1$ (one radian of frame rotation per
radian of $\varphi$; the sign records the sense of rotation). On the right, the
real part and the imaginary part of the entry $U_{11} = \cos\frac\varphi2 +
i\sin\frac\varphi2$ of the spinor rotation $U(\varphi)$ that carries the Cartesian
frame into the polar frame (its inverse $U^{-1}$ removes the connection), for
$\varphi$ from $0$ to $4\pi$: after one full turn ($\varphi = 2\pi$) $U = -1$, and
```

```
only after two full turns is $U = +1$ again. A spinor turns by half the angle of the
frame; this is the "double cover" of rotations by spinor transformations.
```

In [6]:

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.6))
angles = np.arange(12) * np.pi / 6  # 12 angles, 30 degrees apart
for radius in (1.0, 2.0):
    px, py = radius * np.cos(angles), radius * np.sin(angles)  # the points
    ax_left.quiver(px, py, np.cos(angles), np.sin(angles), color="black",
                   angles="xy", scale_units="xy", scale=3.0, width=0.006)
    ax_left.quiver(px, py, -np.sin(angles), np.cos(angles), color="tab:orange",
                   angles="xy", scale_units="xy", scale=3.0, width=0.006)
    circle = np.linspace(0.0, 2.0 * np.pi, 200)
    ax_left.plot(radius * np.cos(circle), radius * np.sin(circle), ":",
                 color="gray", linewidth=1.0)
ax_left.plot([], [], color="black", label="unit radial direction $e_{(0)}$")
ax_left.plot([], [], color="tab:orange", label="unit angular direction $e_{(1)}$")
ax_left.set_aspect("equal")
ax_left.set_xlim(-2.8, 2.8)
ax_left.set_ylim(-2.8, 3.6)  # room for the legend above the circles
ax_left.set_xlabel("Cartesian $x = r\\cos\\varphi$")
ax_left.set_ylabel("Cartesian $y = r\\sin\\varphi$")
ax_left.set_title("The polar frame turns with the angle")
ax_left.legend(loc="upper left", fontsize=8)
turn = np.linspace(0.0, 4.0 * np.pi, 400)  # phi from 0 to 4 pi
ax_right.plot(turn / np.pi, np.cos(turn / 2), label="$\\cos(\\varphi/2)$, real part")
ax_right.plot(turn / np.pi, np.sin(turn / 2), "--",
              label="$\\sin(\\varphi/2)$, imaginary part")
ax_right.set_xlabel("$\\varphi / \\pi$")
ax_right.set_ylabel("entry $U_{11}$ of the spinor rotation")
ax_right.set_ylim(-1.45, 1.2)  # room for the legend below the curves
ax_right.set_title("A spinor turns by half the angle")
ax_right.legend(loc="lower center", ncol=2, fontsize=8)
save_figure(fig, "polar_frame_and_spin_rotation",
            "Warm-up in the flat plane. Left: the unit radial direction (black) and "
            "the unit angular direction (orange) of the polar frame at twelve angles "
            "on the circles $r = 1$ and $r = 2$, drawn in Cartesian coordinates $x$, "
            "$y$ (pure numbers); the frame turns by the angle $\\varphi$ as one goes "
            "around, and the spin connection $\\omega_{\\varphi 01} = -1$ records "
            "this rate. Right: the entry $U_{11} = \\cos(\\varphi/2) + "
            "i\\sin(\\varphi/2)$ of the spinor rotation $U$ that carries the "
            "Cartesian frame into the polar frame (its inverse $U^{-1}$ removes the "
            "connection), real part (solid) and imaginary part (dashed), for "
            "$\\varphi$ from $0$ to $4\\pi$; at $\\varphi = 2\\pi$ the spinor "
            "rotation is $-1$, and only at $4\\pi$ is it $+1$ again.")
```

Out [6]:

```
Figure 06a.1 saved as Revision/textbook/figures/06a_1_polar_frame_and_spin_rotation.png
```

Text cell:

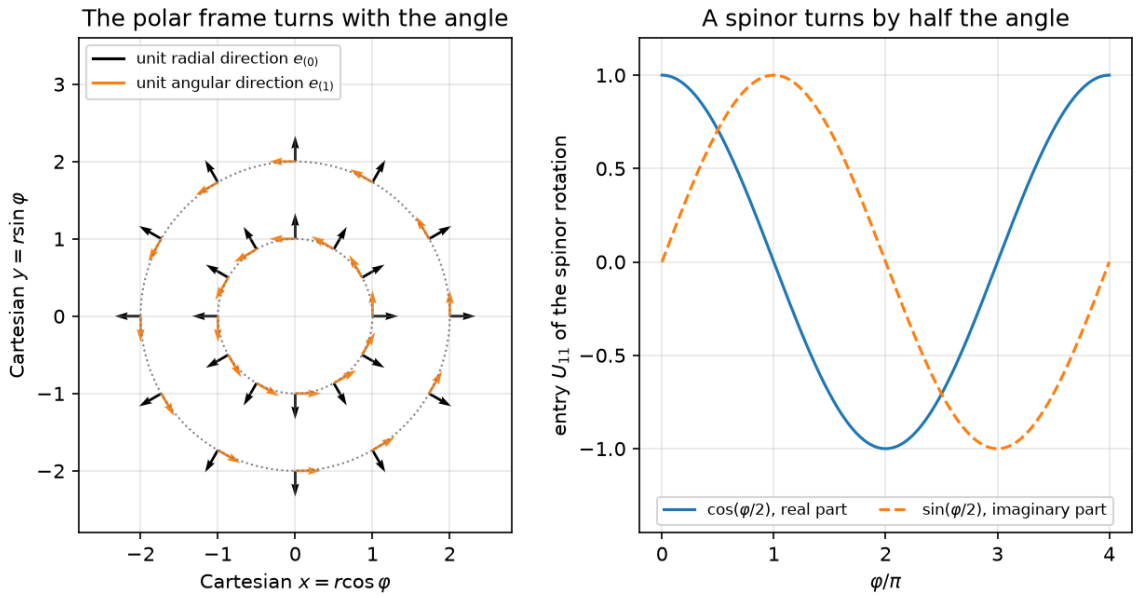

Figure 182: Warm-up in the flat plane. Left: the unit radial direction (black) and the unit angular direction (orange) of the polar frame at twelve angles on the circles $r = 1$ and $r = 2$, drawn in Cartesian coordinates x, y (pure numbers); the frame turns by the angle φ as one goes around, and the spin connection $\omega_{\varphi 01} = -1$ records this rate. Right: the entry $U_{11} = \cos(\varphi/2) + i\sin(\varphi/2)$ of the spinor rotation U that carries the Cartesian frame into the polar frame (its inverse U^{-1} removes the connection), real part (solid) and imaginary part (dashed), for φ from 0 to 4π; at $\varphi = 2\pi$ the spinor rotation is -1, and only at 4π is it $+1$ again. (Notebook 06a, figure 1.)

8. The author's metric and its diagonal vielbein

The next cell makes the symbols of the author's metric: the coordinates $x0, \dots, x7$ (real), the constant $H > 0$, the two functions $a_4(x_4)$ and $b_4(x_4) > 0$ (unknown functions: every result below holds for EVERY a_4 and b_4), and $z = 6Hx_0$. It writes the eight factors f_a of the diagonal vielbein and checks four things: that $\eta_{aa} f_a^2 = g_{aa}$ reproduces the author's metric with b_4 as recorded in `Revision/metric_b4/results/metric-b4.json` (key `metricDiagonal`); that at $b_4 = 1$ it is the metric of the record `Revision/theory/field-theory.json` (key `metric`) and the factors are its `vielbein_diagonal`; and that $\sqrt{|\det g|} = f_0 f_1 \cdots f_7 = b_4^{3/2}\cos z$ (key `sqrtAbsDetG` of `metric-b4.json`). The last fact has a meaning: the factors e^{3a_4} of the inflating 3-space and e^{-3a_4} of the deflating extra times cancel in the volume, so the 7-volume of a slice changes with the time $x4$ only through $b_4^{3/2}$, and not at all when b_4 is constant.

In [7]:

```
x = sp.symbols("x0:8", real=True)  # x[0] = x0, ..., x[4] = x4, ..., x[7] = x7
x0, x4 = x[0], x[4]  # the hidden direction and the time
H = sp.Symbol("H", positive=True)  # the author's constant H > 0
a4 = sp.Function("a4")(x4)  # the metric function a4(x4): any function of the time
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4(x4) > 0
z = 6 * H * x0  # the hidden angle z = 6 H x0, between 0 and pi/2
s = sp.sin(z) ** sp.Rational(1, 6)  # sin^(1/6) z
f = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
     + [sp.exp(-a4) * sp.sqrt(b4) * s] * 3)  # f_0, ..., f_7
g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # g_aa = eta_aa f_a^2
e = sp.diag(*f)  # the diagonal vielbein e^a_mu = f_a delta^a_mu
# The names under which the records write symbols, and our symbols for them:
RECORD_NAMES = {"H": H, "x4": x4, "x0": x0, "a4": a4, "b4": b4,
                "a4p": sp.Derivative(a4, x4), "a4pp": sp.Derivative(a4, (x4, 2)),
                "b4p": sp.Derivative(b4, x4), "b4pp": sp.Derivative(b4, (x4, 2))}
AT_B4_1 = {b4: 1}  # the background b4 = 1


def at_b4_1(expr):
    """expr at b4 = 1; doit() evaluates the derivatives of the constant 1 to 0."""
    return sp.sympify(expr).subs(AT_B4_1).doit()


metric_b4 = json.loads(repository_file(METRIC_B4_FILE).read_text(encoding="utf-8"))
diagonal_b4 = [wolfram_to_sympy(text, RECORD_NAMES)
               for text in metric_b4["metricDiagonal"]]
check(all(is_zero(g[a, a] - diagonal_b4[a]) for a in range(8))
      and record_passed(REPORT_WL_B4, "metric_is_the_authors_b4"),
      "eta_aa f_a^2 is the author's metric with b4 (all eight diagonal entries)",
      record=f"{METRIC_B4_FILE}, key metricDiagonal")
metric_record = sp.Matrix([list(row) for row in record_formula("metric", RECORD_NAMES)])
f_record = record_formula("vielbein_diagonal", RECORD_NAMES)
check(matrix_is_zero(at_b4_1(g) - metric_record)
      and all(is_zero(at_b4_1(f[a]) - f_record[a]) for a in range(8))
```

```
      and record_passed(REPORT_LEAD, "vielbein_reproduces_metric"),
      "at b4 = 1: the metric and the eight vielbein factors of the record",
      record=f"{THEORY_FILE}, formulas metric and vielbein_diagonal")
volume = sp.simplify(sp.prod(f))  # f_0 f_1 ... f_7
say(f"f_0 f_1 ... f_7 = {volume}")
volume_record = wolfram_to_sympy(metric_b4["sqrtAbsDetG"], RECORD_NAMES)
check(is_zero(volume - b4 ** sp.Rational(3, 2) * sp.cos(z))
      and is_zero(volume - volume_record) and is_zero(volume**2 - g.det())
      and record_passed(REPORT_WL_B4, "sqrt_det_g"),
      "sqrt|det g| = f_0 f_1 ... f_7 = b4^(3/2) cos z (cos z at b4 = 1)",
      record=f"{METRIC_B4_FILE}, key sqrtAbsDetG")
```

Out [7]:

```
PASS eta_aa f_a^2 is the author's metric with b4 (all eight diagonal entries)
     reproduces Revision/metric_b4/results/metric-b4.json, key metricDiagonal
PASS at b4 = 1: the metric and the eight vielbein factors of the record
     reproduces Revision/theory/field-theory.json, formulas metric and vielbein_diagonal
f_0 f_1 ... f_7 = b4(x4)**(3/2)*cos(6*H*x0)
PASS sqrt|det g| = f_0 f_1 ... f_7 = b4^(3/2) cos z (cos z at b4 = 1)
     reproduces Revision/metric_b4/results/metric-b4.json, key sqrtAbsDetG
```

Text cell:

```
The next cell draws the eight vielbein factors. The left panel shows how they
depend on the time $x4$ at the fixed hidden angle $z = \pi/4$, for one illustrative
history $a_4(x_4) = x_4$, $b_4 = 1$ in units in which $H = 1$ (an illustration
only: the results of this notebook hold for every $a_4$ and $b_4$). On a
logarithmic vertical axis the exponential factors are straight lines: the 3-space
factor $e^{a_4}s$ rises, the extra-time factor $e^{-a_4}b_4^{1/2}s$ falls at the
same rate, and their product is constant (a $b_4$ different from 1 would multiply
the falling line by $b_4^{1/2}$). The right panel shows the dependence on $z$ at
$x_4 = 0$: the factor $s = \sin^{1/6} z$ of the six warped directions, the
hidden-direction factor $\cot z$ and the volume factor $\sqrt{|\det g|} = \cos z$
(at $b_4 = 1$).
```

In [8]:

```
X4_SYMBOL = sp.Symbol("t", real=True)  # the time x4 as a plain symbol for numbers
history = {a4: X4_SYMBOL, b4: 1}  # illustrative: a4(x4) = x4, b4 = 1 (with H = 1)
f_of_time = sp.lambdify(X4_SYMBOL, [fa.subs(history).subs({H: 1, x0: sp.pi / 24})
                                    for fa in f])  # z = 6 x0 = pi/4
times = np.linspace(-2.0, 2.0, 201)
values = [np.broadcast_to(v, times.shape) for v in f_of_time(times)]
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.4))
ax_left.semilogy(times, values[1], label="$f_1 = f_2 = f_3 = e^{a_4}s$ (3-space)")
ax_left.semilogy(times, values[5], "--",
                 label="$f_5 = f_6 = f_7 = e^{-a_4}b_4^{1/2}s$ (extra times)")
ax_left.semilogy(times, values[4], ":", color="black", label="$f_4 = 1$ (time)")
ax_left.semilogy(times, values[1] * values[5], "-.", color="gray",
                 label="product $f_1 f_5 = b_4^{1/2}s^2$")
```

```
ax_left.set_xlabel("time $x_4$ (units $1/H$), history $a_4 = x_4$, $b_4 = 1$")
ax_left.set_ylabel("vielbein factor (logarithmic axis)")
ax_left.set_title("Inflating 3-space, deflating extra times")
ax_left.legend(fontsize=8)
zz = np.linspace(0.02, np.pi / 2 - 0.02, 300)  # z inside (0, pi/2)
ax_right.plot(zz, np.sin(zz) ** (1 / 6), label="$s = \\sin^{1/6} z$")
ax_right.plot(zz, 1 / np.tan(zz), "--", label="$f_0 = \\cot z$ (hidden)")
ax_right.plot(zz, np.cos(zz), ":", color="black",
              label="$\\sqrt{|\\det g|} = \\cos z$ ($b_4 = 1$)")
ax_right.set_ylim(0.0, 3.0)
ax_right.set_xlabel("hidden angle $z = 6 H x_0$ (radians)")
ax_right.set_ylabel("factor (pure number)")
ax_right.set_title("Dependence on the hidden direction")
ax_right.legend(fontsize=8)
save_figure(fig, "vielbein_factors",
            "The eight factors $f_a$ of the diagonal vielbein. Left: versus the time "
            "$x_4$ in units $1/H$ at $z = \\pi/4$, for the illustrative history "
            "$a_4 = x_4$, $b_4 = 1$, on a logarithmic axis; the 3-space factor "
            "$e^{a_4}s$ grows and the extra-time factor $e^{-a_4}b_4^{1/2}s$ shrinks "
            "at the same exponential rate, so their product $b_4^{1/2}s^2$ stays "
            "constant, and the time factor is $1$. Right: versus the hidden angle "
            "$z = 6Hx_0$ in radians at $x_4 = 0$; $s = \\sin^{1/6}z$, the hidden "
            "factor $\\cot z$, which falls from very large values to $0$, and the "
            "volume factor $\\sqrt{|\\det g|} = b_4^{3/2}\\cos z$ at $b_4 = 1$, which "
            "depends on the time only through $b_4$.")
```

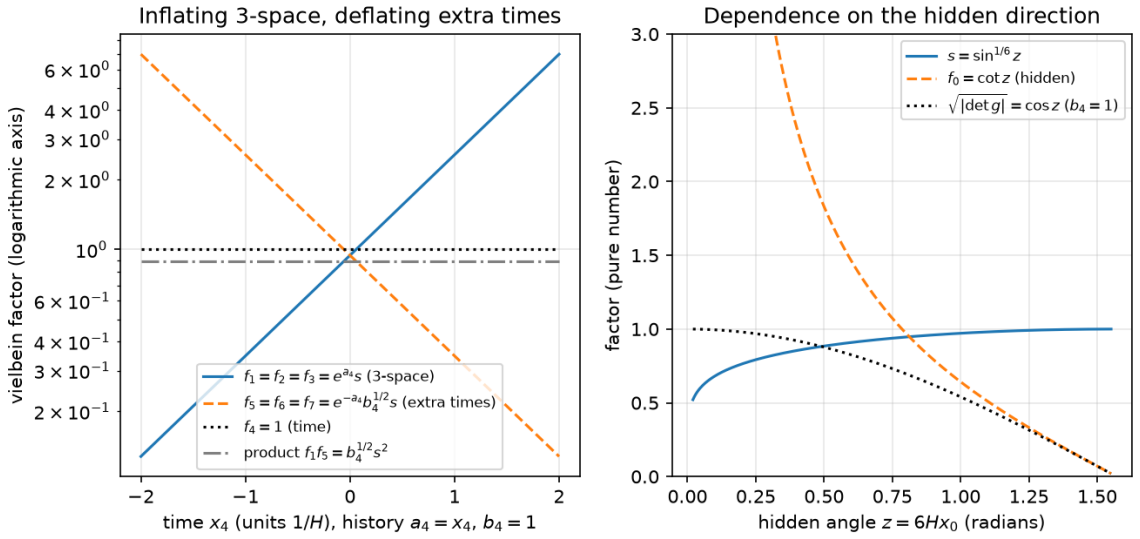

Figure 183: The eight factors f_a of the diagonal vielbein. Left: versus the time x_4 in units $1/H$ at $z = \pi/4$, for the illustrative history $a_4 = x_4$, $b_4 = 1$, on a logarithmic axis; the 3-space factor $e^{a_4}s$ grows and the extra-time factor $e^{-a_4}b_4^{1/2}s$ shrinks at the same exponential rate, so their product $b_4^{1/2}s^2$ stays constant, and the time factor is 1. Right: versus the hidden angle $z = 6Hx_0$ in radians at $x_4 = 0$; $s = \sin^{1/6} z$, the hidden factor $\cot z$, which falls from very large values to 0, and the volume factor $\sqrt{|\det g|} = b_4^{3/2}\cos z$ at $b_4 = 1$, which depends on the time only through b_4. (Notebook 06a, figure 2.)

Out [8]:

```
Figure 06a.2 saved as Revision/textbook/figures/06a_2_vielbein_factors.png
```

Text cell:

```
## 9. The Christoffel symbols of the author's metric

The next cell computes all Christoffel symbols with the general function, counts
the independent nonzero ones (those with $m \le n$; the records count 25) and
compares each of them with the list 'christoffel_nonzero' of the record with
$b_4$, and, at $b_4 = 1$, with the same list of the record of $b_4 = 1$. In these
lists an entry $\{l, m, n, \text{value}\}$ means $\Gamma^{l}{}_{mn} = $ value, with
the directions numbered from 1 in the order $x0, \dots, x7$ (the number 1 is $x0$,
5 is $x4$); Python counts from 0, so 1 is subtracted. The cell prints five symbols.
One of them is $\Gamma^{x1}{}_{x1\,x4} = a_4'$: moving forward in time, a 3-space
direction grows at the rate $a_4'$; its partner $\Gamma^{x5}{}_{x4\,x5} = -a_4' +
b_4'/(2b_4)$ says that an extra-time direction shrinks at the rate $a_4'$ and is
stretched by a changing $b_4$.
```

In [9]:

```
Gam = christoffel(g, x)
independent = {(l, m, n): Gam[l][m][n] for l in range(8) for m in range(8)
               for n in range(m, 8) if Gam[l][m][n] != 0}
report("number of independent nonzero Christoffel symbols", len(independent))


def as_dictionary(entries):
    """{l, m, n, value} entries of a record (numbered from 1) -> {(l, m, n): value}
    with Python's numbers 0 to 7."""
    return {(int(l) - 1, int(m) - 1, int(n) - 1): value for l, m, n, value in entries}


record_b4 = as_dictionary(record_formula("christoffel_nonzero", RECORD_NAMES,
                                         FORMULAS_B4))
record_b4_1 = as_dictionary(record_formula("christoffel_nonzero", RECORD_NAMES))
same_b4 = set(record_b4) == set(independent) and all(
    is_zero(independent[key] - record_b4[key]) for key in independent)
same_b4_1 = set(record_b4_1) == set(independent) and all(
    is_zero(at_b4_1(independent[key]) - record_b4_1[key]) for key in independent)
for key in [(1, 1, 4), (5, 4, 5), (1, 0, 1), (4, 5, 5), (0, 0, 0)]:
    l, m, n = key
    say(f"Gamma^{NAMES[l]}_{NAMES[m]}{NAMES[n]} = {independent[key]}")
check(len(independent) == 25 and same_b4 and same_b4_1,
      "the 25 independent nonzero Christoffel symbols equal the records (with b4, and "
      "at b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula christoffel_nonzero")
```

Out [9]:

```
RESULT number of independent nonzero Christoffel symbols = 25
Gamma^x1_x1x4 = Derivative(a4(x4), x4)
Gamma^x5_x4x5 = -Derivative(a4(x4), x4) + Derivative(b4(x4), x4)/(2*b4(x4))
Gamma^x1_x0x1 = H/tan(6*H*x0)
Gamma^x4_x5x5 = (b4(x4)*Derivative(a4(x4), x4) - Derivative(b4(x4),
    x4)/2)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)
Gamma^x0_x0x0 = -12*H/sin(12*H*x0)
PASS the 25 independent nonzero Christoffel symbols equal the records (with b4, and at b4
    = 1)
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula christoffel_nonzero
```

Text cell:

```
The next cell draws where the nonzero Christoffel symbols sit. There is one small
8 x 8 grid for each upper index $l = x0, \dots, x7$; the square in row $m$ and
column $n$ is coloured by the sign of $\Gamma^l{}_{mn}$ at the sample point
$H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ (red
positive, blue negative, white zero). The pattern shows the structure of the
metric: only the hidden direction $x0$ and the time $x4$ appear as the
"derivative" index, because the metric depends on $x_0$ and $x_4$ only.
```

In [10]:

```
SAMPLE = {H: 1, x0: sp.pi / 24}  # H = 1 and z = 6 H x0 = pi/4


def at_sample(expr):
    """The decimal value of expr at the sample point H = 1, z = pi/4, a4 = 1/2,
    a4' = 1/2, b4 = 1, b4' = 1/2, a4'' = b4'' = 0 (derivatives are replaced first,
    then the functions themselves)."""
    expr = sp.sympify(expr)
    for function, first in ((a4, sp.Rational(1, 2)), (b4, sp.Rational(1, 2))):
        expr = expr.subs(sp.Derivative(function, (x4, 2)), 0)
        expr = expr.subs(sp.Derivative(function, x4), first)
    expr = expr.subs({a4: sp.Rational(1, 2), b4: 1})
    return complex(sp.N(expr.subs(SAMPLE)))


fig, axes = plt.subplots(2, 4, figsize=(10.0, 5.6))
for l, ax in enumerate(axes.flat):
    signs = np.array([[np.sign(at_sample(Gam[l][m][n]).real) for n in range(8)]
                      for m in range(8)])
    ax.imshow(signs, cmap="RdBu_r", vmin=-1.5, vmax=1.5)
    ax.set_title(f"$\\Gamma^{{{l}}}{{}}_{{mn}}$", fontsize=9)
    ax.set_xticks(range(8), [f"{k}" for k in range(8)], fontsize=7)
    ax.set_yticks(range(8), [f"{k}" for k in range(8)], fontsize=7)
    ax.set_xlabel("$n$ (x-number)", fontsize=7)
    ax.set_ylabel("$m$ (x-number)", fontsize=7)
    ax.grid(False)
fig.suptitle("Signs of the Christoffel symbols at a sample point "
             "(red +, blue -, white 0)", fontsize=10)
```

```
fig.tight_layout()
save_figure(fig, "christoffel_pattern",
            "Where the Christoffel symbols $\\Gamma^l{}_{mn}$ of the author's metric "
            "are nonzero: one 8 x 8 grid per upper index $l = x0, \\dots, x7$, rows "
            "$m$ and columns $n$ numbered 0 to 7 for $x0$ to $x7$, each square "
            "coloured by the sign of the symbol at the sample point $H = 1$, $z = "
            "\\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ (red "
            "positive, blue negative, white zero). Every nonzero symbol has an index "
            "equal to 0 or 4: the metric depends only on the hidden coordinate $x_0$ "
            "and on the time $x_4$. The grids are symmetric because "
            "$\\Gamma^l{}_{mn} = \\Gamma^l{}_{nm}$.")
```

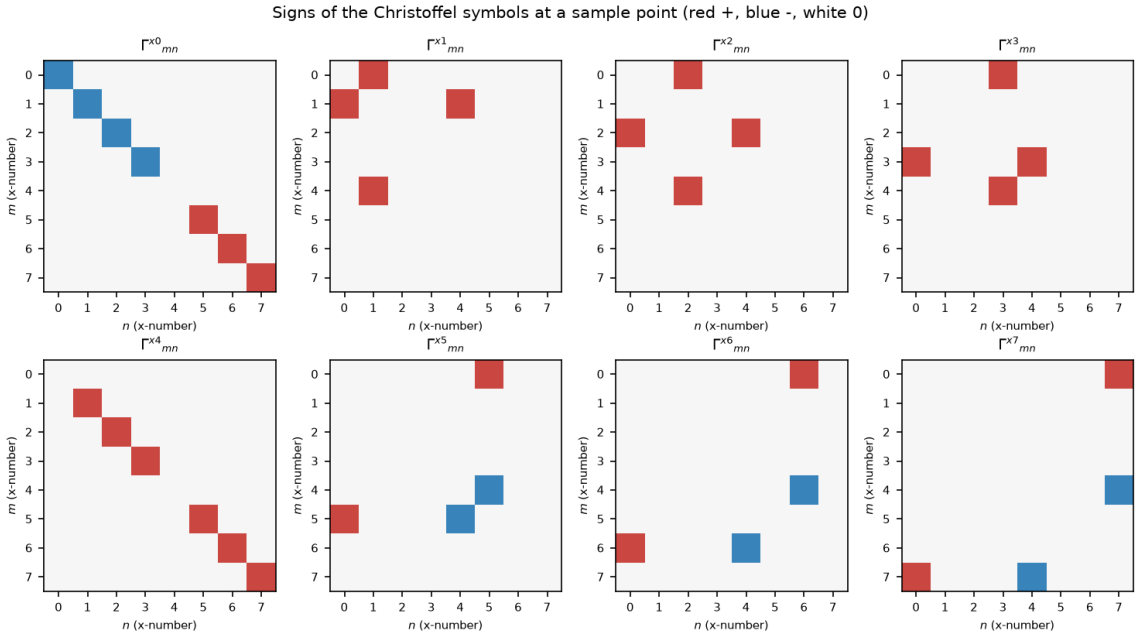

Figure 184: Where the Christoffel symbols $\Gamma^l{}_{mn}$ of the author's metric are nonzero: one 8 x 8 grid per upper index $l = x0, \dots, x7$, rows m and columns n numbered 0 to 7 for $x0$ to $x7$, each square coloured by the sign of the symbol at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ (red positive, blue negative, white zero). Every nonzero symbol has an index equal to 0 or 4: the metric depends only on the hidden coordinate x_0 and on the time x_4. The grids are symmetric because $\Gamma^l{}_{mn} = \Gamma^l{}_{nm}$. (Notebook 06a, figure 3.)

Out [10]:

```
Figure 06a.3 saved as Revision/textbook/figures/06a_3_christoffel_pattern.png
```

Text cell:

```
## 10. The canonical spin connection

The next cell computes $\omega_{\mu ab}$ with the general function and checks:
```

```
1. the vielbein postulate $\partial_\mu e^a{}_\nu - \sum_\lambda
   \Gamma^\lambda{}_{\mu\nu}e^a{}_\lambda + \sum_b \omega_\mu{}^a{}_b e^b{}_\nu = 0$
   in all $8 \cdot 8 \cdot 8 = 512$ components (here $\omega_\mu{}^a{}_b =
   \eta_{aa}\omega_{\mu ab}$, because $\eta_{aa}^2 = 1$);
2. the antisymmetry $\omega_{\mu ab} = -\omega_{\mu ba}$ (all 512 triples);
3. that exactly 12 components with $a < b$ are nonzero and equal the list
   ‘omega_nonzero‘ (entries $\{\mu, a, b, \text{value}\}$, numbered from 1 as for
   the Christoffel symbols) of the record with $b_4$, and at $b_4 = 1$ that of the
   record of $b_4 = 1$;
4. that every nonzero component is a rate times a factor that never vanishes for
   $0 < z < \pi/2$: the rate is $a_4'$ (3-space), $a_4' - b_4'/(2b_4)$ (the extra
   times) or $H$ (the hidden direction), and the factor is $\pm e^{a_4}s$ or
   $\pm e^{-a_4}b_4^{1/2}s$. So the part of the spin connection made by the time
   dependence vanishes only if $a_4$ and $b_4$ are constant, and the part made by
   the hidden direction never vanishes for $H > 0$.
```

In [11]:

```
omega = spin_connection(e, Gam, x, ETA)
postulate = [sp.diff(e[a, nu], x[mu])
             - sum(Gam[lam][mu][nu] * e[a, lam] for lam in range(8))
             + sum(ETA[a] * omega[mu][a][b] * e[b, nu] for b in range(8))
             for mu in range(8) for a in range(8) for nu in range(8)]
check(len(postulate) == 512 and all(is_zero(v) for v in postulate),
      "the vielbein postulate holds in all 512 components",
      record=f"{REPORT_PY}, check vielbein_postulate")
check(all(is_zero(omega[mu][a][b] + omega[mu][b][a])
          for mu in range(8) for a in range(8) for b in range(8)),
      "omega_mu ab = -omega_mu ba for all mu, a, b",
      record=f"{REPORT_PY}, check spin_connection_antisymmetric")
omega_nonzero = {(mu, a, b): omega[mu][a][b] for mu in range(8) for a in range(8)
                 for b in range(a + 1, 8) if omega[mu][a][b] != 0}
for (mu, a, b), value in omega_nonzero.items():
    say(f"omega_{NAMES[mu]},{NAMES[a]}{NAMES[b]} = {value}")
omega_record = as_dictionary(record_formula("omega_nonzero", RECORD_NAMES,
                                            FORMULAS_B4))
omega_record_1 = as_dictionary(record_formula("omega_nonzero", RECORD_NAMES))
check(len(omega_nonzero) == 12 and set(omega_record) == set(omega_nonzero)
      and all(is_zero(omega_nonzero[k] - omega_record[k]) for k in omega_nonzero)
      and set(omega_record_1) == set(omega_nonzero)
      and all(is_zero(at_b4_1(omega_nonzero[k]) - omega_record_1[k])
              for k in omega_nonzero),
      "the 12 independent nonzero omega_mu ab equal the records (with b4, and at "
      "b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula omega_nonzero")
a4p, b4p = sp.Derivative(a4, x4), sp.Derivative(b4, x4)
rates = (a4p, a4p - b4p / (2 * b4), H)  # 3-space, the extra times, hidden direction
factors = [sign * phys for sign in (1, -1)
           for phys in (sp.exp(a4) * s, sp.exp(-a4) * sp.sqrt(b4) * s)]
# value / (rate * factor) must simplify to exactly 1 for one choice of each.
shape_ok = all(any(sp.simplify(value / (rate * phys)) == 1 for rate in rates
                   for phys in factors) for value in omega_nonzero.values())
check(shape_ok and record_passed(REPORT_WL_B4, "omega_components"),
```

```
      "every nonzero omega is a4', a4' - b4'/(2 b4) or H times +-exp(a4) s or "
      "+-exp(-a4) b4^(1/2) s",
      record=f"{REPORT_WL_B4}, check omega_components")
```

Out [11]:

```
PASS the vielbein postulate holds in all 512 components
     reproduces Revision/theory/reports/python-field-theory.json, check
    vielbein_postulate
PASS omega_mu ab = -omega_mu ba for all mu, a, b
     reproduces Revision/theory/reports/python-field-theory.json, check
    spin_connection_antisymmetric
omega_x1,x0x1 = -H*exp(a4(x4))*sin(6*H*x0)**(1/6)
omega_x1,x1x4 = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x2,x0x2 = -H*exp(a4(x4))*sin(6*H*x0)**(1/6)
omega_x2,x2x4 = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x3,x0x3 = -H*exp(a4(x4))*sin(6*H*x0)**(1/6)
omega_x3,x3x4 = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x5,x0x5 = H*sqrt(b4(x4))*exp(-a4(x4))*sin(6*H*x0)**(1/6)
omega_x5,x4x5 = (-b4(x4)*Derivative(a4(x4), x4) + Derivative(b4(x4),
    x4)/2)*exp(-a4(x4))*sin(6*H*x0)**(1/6)/sqrt(b4(x4))
omega_x6,x0x6 = H*sqrt(b4(x4))*exp(-a4(x4))*sin(6*H*x0)**(1/6)
omega_x6,x4x6 = (-b4(x4)*Derivative(a4(x4), x4) + Derivative(b4(x4),
    x4)/2)*exp(-a4(x4))*sin(6*H*x0)**(1/6)/sqrt(b4(x4))
omega_x7,x0x7 = H*sqrt(b4(x4))*exp(-a4(x4))*sin(6*H*x0)**(1/6)
omega_x7,x4x7 = (-b4(x4)*Derivative(a4(x4), x4) + Derivative(b4(x4),
    x4)/2)*exp(-a4(x4))*sin(6*H*x0)**(1/6)/sqrt(b4(x4))
PASS the 12 independent nonzero omega_mu ab equal the records (with b4, and at b4 = 1)
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula omega_nonzero
PASS every nonzero omega is a4', a4' - b4'/(2 b4) or H times +-exp(a4) s or +-exp(-a4)
    b4^(1/2) s
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    omega_components
```

Text cell:

```
The next cell draws two of the eight 8 x 8 arrays $\omega_{\mu ab}$ at the sample
point: for $\mu = x1$ (an inflating 3-space direction) and for $\mu = x5$ (a
deflating extra time). Each array is antisymmetric: the square $(a, b)$ has the
opposite colour of the square $(b, a)$, and the diagonal is white. For $\mu = x1$
the frame direction $x1$ turns towards the time $x4$ (rate $a_4' e^{a_4}s$) and
towards the hidden direction $x0$ (rate $H e^{a_4}s$); for $\mu = x5$ the signs are
reversed, because the extra times shrink while 3-space grows (at the sample point
$a_4' - b_4'/(2b_4) = 1/4$ is still positive).
```

In [12]:

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.6))
for ax, mu in zip(axes, (1, 5)):  # the directions x1 and x5
    table = np.array([[at_sample(omega[mu][a][b]).real for b in range(8)]
```

```
                    for a in range(8)])
    limit = np.abs(table).max()
    image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
    for a in range(8):
        for b in range(8):
            if table[a, b] != 0:
                ax.text(b, a, f"{table[a, b]:.2f}", ha="center", va="center",
                        fontsize=7)
    ax.set_xticks(range(8), NAMES, fontsize=8)
    ax.set_yticks(range(8), NAMES, fontsize=8)
    ax.set_xlabel("frame index $b$")
    ax.set_ylabel("frame index $a$")
    ax.set_title(f"$\\omega_{{x{mu}\\,ab}}$ at the sample point")
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.8)
fig.tight_layout()
save_figure(fig, "omega_heat_maps",
            "The canonical spin connection $\\omega_{\\mu ab}$ of the diagonal "
            "vielbein as two heat maps, for $\\mu = x1$ (left, an inflating 3-space "
            "direction) and $\\mu = x5$ (right, a deflating extra time), at the "
            "sample point $H = 1$, $z = \\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, "
            "$b_4' = 1/2$; rows $a$ "
            "and columns $b$ are the frame directions $x0$ to $x7$, the colour and "
            "the printed number give the value (red positive, blue negative, white "
            "zero). Each array is antisymmetric, and only the pairs with $x4$ or "
            "$x0$ are nonzero; the extra time $x5$ has the opposite signs of the "
            "3-space direction $x1$.")
```

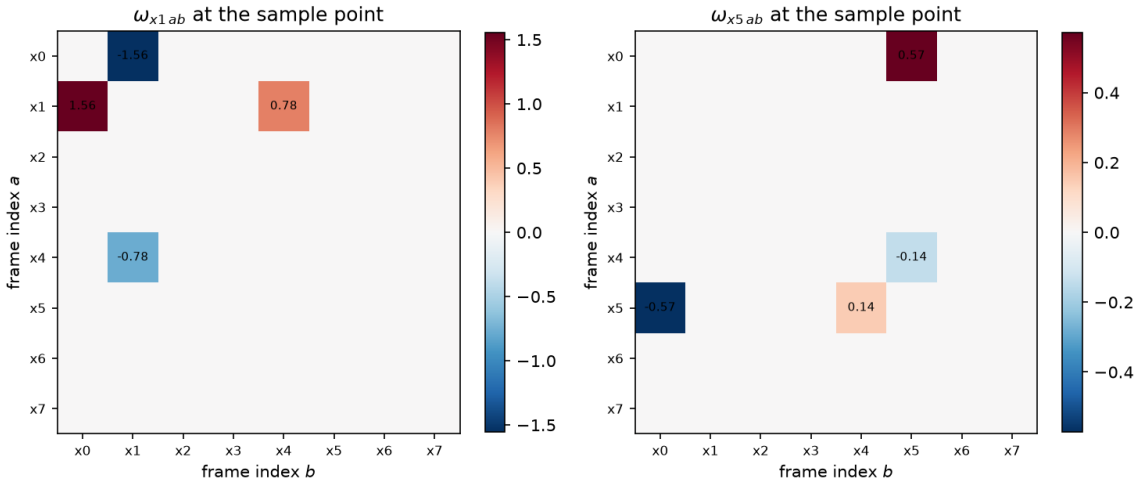

Figure 185: The canonical spin connection $\omega_{\mu ab}$ of the diagonal vielbein as two heat maps, for $\mu = x1$ (left, an inflating 3-space direction) and $\mu = x5$ (right, a deflating extra time), at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$; rows a and columns b are the frame directions $x0$ to $x7$, the colour and the printed number give the value (red positive, blue negative, white zero). Each array is antisymmetric, and only the pairs with $x4$ or $x0$ are nonzero; the extra time $x5$ has the opposite signs of the 3-space direction $x1$. (Notebook 06a, figure 4.)

Out [12]:

```
Figure 06a.4 saved as Revision/textbook/figures/06a_4_omega_heat_maps.png
```

Text cell:

```
## 11. The spinor connection $\Omega_\mu$ and the covariant constancy of the gammas

The next cell builds the eight 16 x 16 matrices $\Omega_\mu$ and the curved gammas
$\gamma^\mu = \gamma^a/f_a$ (no sum) and checks:

1. $\Omega_{x4} = \Omega_{x0} = 0$: no frame direction turns when one moves along
   the time or along the hidden direction;
2. the defining property of the spinor connection, $[\Omega_\mu, \gamma^a] =
   -\sum_b \omega_\mu{}^a{}_b \gamma^b$ for all 64 pairs $(\mu, a)$: commuting
   with $\Omega_\mu$ turns the gammas exactly as $\omega_\mu$ turns the frame;
3. the covariant constancy of the curved gammas, $D_\mu\gamma^\nu = \partial_\mu
   \gamma^\nu + \sum_\lambda\Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega_\mu,
   \gamma^\nu] = 0$ for all 64 pairs $(\mu, \nu)$, which is the vielbein postulate
   written with gammas.
```

In [13]:

```
Omega = spinor_connection(omega, S)  # eight 16 x 16 matrices
gup = curved_gammas(e, gamma)  # gamma^mu = gamma^a / f_a for mu = a
check(Omega[0] == Z16 and Omega[4] == Z16, "Omega_x0 = Omega_x4 = 0 for every b4",
      record=f"{REPORT_PY}, check Omega_x0_and_Omega_x4_vanish (b4 = 1)")
rotation_ok = all(matrix_is_zero(
    Omega[mu] * gamma[a] - gamma[a] * Omega[mu]
    + sum((ETA[a] * omega[mu][a][b] * gamma[b] for b in range(8)), Z16))
    for mu in range(8) for a in range(8))
check(rotation_ok, "[Omega_mu, gamma^a] = -omega_mu^a_b gamma^b for all 64 pairs",
      record=f"{REPORT_WL}, check S_rotates_gamma_with_omega (b4 = 1)")
constancy_ok = all(matrix_is_zero(
    gup[nu].diff(x[mu]) + sum((Gam[nu][mu][lam] * gup[lam] for lam in range(8)), Z16)
    + Omega[mu] * gup[nu] - gup[nu] * Omega[mu])
    for mu in range(8) for nu in range(8))
check(constancy_ok and record_passed(REPORT_WL_B4, "gamma_covariantly_constant"),
      "D_mu gamma^nu = 0 for all 64 pairs (mu, nu)",
      record=f"{REPORT_WL_B4}, check gamma_covariantly_constant")
say(f"Omega_x1 has {sum(1 for v in Omega[1] if v != 0)} nonzero entries of 256")
```

Out [13]:

```
PASS Omega_x0 = Omega_x4 = 0 for every b4
     reproduces Revision/theory/reports/python-field-theory.json, check
    Omega_x0_and_Omega_x4_vanish (b4 = 1)
PASS [Omega_mu, gamma^a] = -omega_mu^a_b gamma^b for all 64 pairs
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    S_rotates_gamma_with_omega (b4 = 1)
PASS D_mu gamma^nu = 0 for all 64 pairs (mu, nu)
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    gamma_covariantly_constant
Omega_x1 has 32 nonzero entries of 256
```

Text cell:

```
Each $\Omega_\mu$ is a combination of two of the matrices $S^{ab}$:
$\Omega_{x1} = e^{a_4}s\,(a_4' S^{x1\,x4} + H S^{x1\,x0})$ and $\Omega_{x5} =
-e^{-a_4}b_4^{1/2}s\,\big((a_4' - \frac{b_4'}{2b_4}) S^{x4\,x5} + H S^{x5\,x0}\big)$.
The author's gammas are real, the
four space-like ones are symmetric matrices and the four time-like ones are
antisymmetric. Because of this, $S^{ab} = \frac12\gamma^a\gamma^b$ ($a \ne b$)
is a SYMMETRIC matrix when one of $a, b$ is space-like and the other time-like
(a **boost**, 16 pairs), and an ANTISYMMETRIC matrix when both are of the same
kind (a **rotation**, 12 pairs). The next cell checks this for all 28 pairs and
then draws $\Omega_{x1}$ and $\Omega_{x5}$ as 16 x 16 heat maps at the sample
point. In $\Omega_{x1}$ the $a_4'$ part is a boost ($x1$ with the time $x4$,
symmetric) and the $H$ part a rotation ($x1$ with $x0$, antisymmetric); in
$\Omega_{x5}$ it is the other way round ($x4$ with $x5$ is a rotation of two
time-like directions, $x5$ with $x0$ a boost).
```

In [14]:

```
pattern_ok = True
for a in range(8):
    for b in range(a + 1, 8):
        boost = ETA[a] * ETA[b] == -1  # one space-like, one time-like direction
        want = S[a][b] if boost else -S[a][b]  # S^T = S (boost) or -S (rotation)
        pattern_ok = pattern_ok and S[a][b].T == want
check(pattern_ok, "S^ab is symmetric for the 16 boosts, antisymmetric for the 12 "
      "rotations")
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.6))
for ax, mu in zip(axes, (1, 5)):  # the directions x1 and x5
    table = np.array([[at_sample(v).real for v in row] for row in Omega[mu].tolist()])
    limit = np.abs(table).max()
    image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.set_xticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_yticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_xlabel("column (spinor component 1 to 16)")
    ax.set_ylabel("row (spinor component)")
    ax.set_title(f"$\\Omega_{{x{mu}}}$ at the sample point")
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.8)
fig.tight_layout()
save_figure(fig, "spinor_connection_heat_maps",
            "The spinor connection $\\Omega_{x1}$ (left) and $\\Omega_{x5}$ (right) as "
            "16 x 16 heat maps at the sample point $H = 1$, $z = \\pi/4$, $a_4 = 1/2$, "
            "$a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$; rows and columns are the 16 "
            "spinor components, red is "
            "positive, blue negative, white zero. Each matrix is a sum of two of the "
            "generators $S^{ab}$: a boost, which is a symmetric matrix (entries "
            "$(i, j)$ and $(j, i)$ of the same colour), and a rotation, which is "
            "antisymmetric (opposite colours). Both matrices map the first eight "
            "components to the first eight and the last eight to the last eight.")
```

Out [14]:

```
PASS S^ab is symmetric for the 16 boosts, antisymmetric for the 12 rotations
```

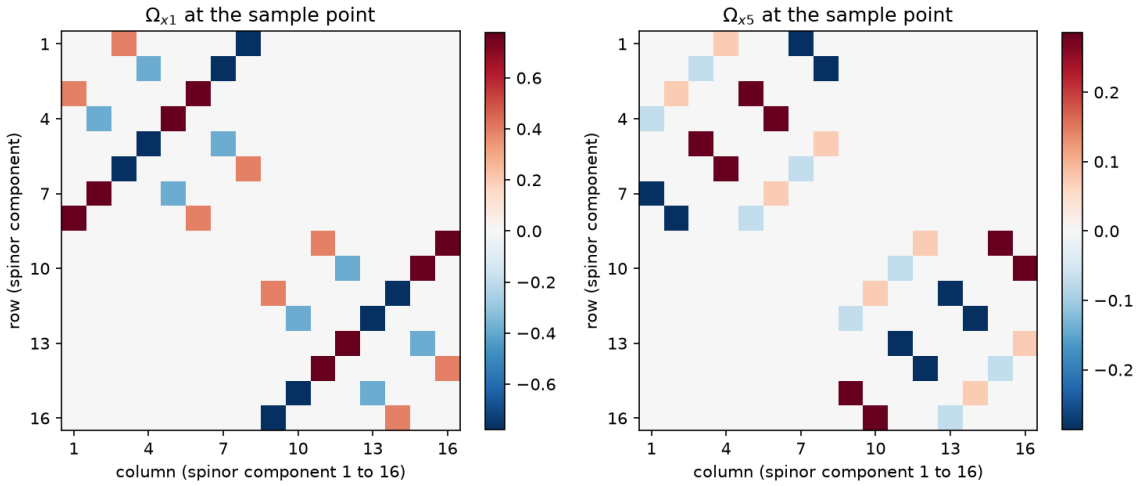

Figure 186: The spinor connection Ω_{x1} (left) and Ω_{x5} (right) as 16 x 16 heat maps at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a'_4 = 1/2$, $b_4 = 1$, $b'_4 = 1/2$; rows and columns are the 16 spinor components, red is positive, blue negative, white zero. Each matrix is a sum of two of the generators S^{ab}: a boost, which is a symmetric matrix (entries (i, j) and (j, i) of the same colour), and a rotation, which is antisymmetric (opposite colours). Both matrices map the first eight components to the first eight and the last eight to the last eight. (Notebook 06a, figure 5.)

Out [14]:

```
Figure 06a.5 saved as Revision/textbook/figures/06a_5_spinor_connection_heat_maps.png
```

Text cell:

```
## 12. The contraction $\gamma^\mu\Omega_\mu$, direction by direction

The field equation contains $\gamma^\mu D_\mu\Psi = \gamma^\mu\partial_\mu\Psi +
\gamma^\mu\Omega_\mu\Psi$, so the spin connection enters it only through the
16 x 16 matrix $\gamma^\mu\Omega_\mu$ (summed over $\mu$). The next cell computes
each term $\gamma^\mu\Omega_\mu$ (no sum) separately and writes it as
$\alpha_\mu\gamma^{(x4)} + \beta_\mu\gamma^{(x0)}$. The numbers $\alpha_\mu$ and
$\beta_\mu$ are found with traces: for two different gammas
$\mathrm{tr}(\gamma^a\gamma^b) = 0$, while $\mathrm{tr}(\gamma^{(x4)}
\gamma^{(x4)}) = -16$ and $\mathrm{tr}(\gamma^{(x0)}\gamma^{(x0)}) = +16$, so
$\alpha_\mu = -\mathrm{tr}(M\gamma^{(x4)})/16$ and $\beta_\mu =
\mathrm{tr}(M\gamma^{(x0)})/16$ for $M = \gamma^\mu\Omega_\mu$; the cell then
checks that $M - \alpha_\mu\gamma^{(x4)} - \beta_\mu\gamma^{(x0)}$ is the zero
matrix. It compares the table with the predicted values ($\alpha = a_4'/2$ for
$x1, x2, x3$, $\alpha = -a_4'/2 + b_4'/(4b_4)$ for $x5, x6, x7$, $\beta = H/2$ for
these six directions, and zero for $x0$ and $x4$; the record with $b_4$ checks the
same table), at $b_4 = 1$ with the record's table 'gammaOmega_per_direction', and
the sum with the record's 'gammaOmega_total'.
```

In [15]:

```
per_direction = []  # (alpha_mu, beta_mu) for mu = x0 ... x7
decomposed = True
for mu in range(8):
    M = (gup[mu] * Omega[mu]).applyfunc(sp.simplify)
    alpha = sp.simplify(-(M * gamma[4]).trace() / 16)  # coefficient of gamma^(x4)
    beta = sp.simplify((M * gamma[0]).trace() / 16)  # coefficient of gamma^(x0)
    decomposed = decomposed and matrix_is_zero(M - alpha * gamma[4] - beta * gamma[0])
    per_direction.append((alpha, beta))
    say(f"gamma^{NAMES[mu]} Omega_{NAMES[mu]} = ({alpha}) gamma^(x4) "
        f"+ ({beta}) gamma^(x0)")
rate_t = -a4p / 2 + b4p / (4 * b4)  # alpha of an extra time
predicted = ([(0, 0)] + [(a4p / 2, H / 2)] * 3 + [(0, 0)]
             + [(rate_t, H / 2)] * 3)  # x0, x1..x3, x4, x5..x7
check(decomposed and all(is_zero(per_direction[k][0] - predicted[k][0])
                         and is_zero(per_direction[k][1] - predicted[k][1])
                         for k in range(8))
      and record_passed(REPORT_PY_B4, "gammaOmega_per_direction")
      and record_passed(REPORT_WL_B4, "gammaOmega_per_direction"),
      "the eight terms gamma^mu Omega_mu (no sum) with b4",
      record=f"{REPORT_WL_B4}, check gammaOmega_per_direction")
table_record = record_formula("gammaOmega_per_direction", RECORD_NAMES)
check(all(is_zero(at_b4_1(per_direction[k][0]) - table_record[k][1])
          and is_zero(per_direction[k][1] - table_record[k][2]) for k in range(8)),
      "at b4 = 1 the eight terms equal the record",
      record=f"{THEORY_FILE}, formula gammaOmega_per_direction")
inflating = sp.simplify(sum(per_direction[k][0] for k in (1, 2, 3)))
deflating = sp.simplify(sum(per_direction[k][0] for k in (5, 6, 7)))
hidden = sp.simplify(sum(beta for _, beta in per_direction))
report("x4 part from the inflating x1, x2, x3", inflating)
report("x4 part from the deflating x5, x6, x7", deflating)
report("x0 part from all directions", hidden)
check(is_zero(inflating + deflating - sp.Rational(3, 4) * b4p / b4)
      and is_zero(hidden - 3 * H),
      "the a4' parts of the x4 terms cancel, (3/4) b4'/b4 remains; the x0 terms "
      "add to 3H",
      record=f"{REPORT_WL}, check gammaOmega_x4_terms_cancel (b4 = 1)")
slash = sum((gup[mu] * Omega[mu] for mu in range(8)), Z16)
gravity_term = 3 * H * gamma[0] + sp.Rational(3, 4) * b4p / b4 * gamma[4]
total_record = wolfram_to_sympy(
    FORMULAS_B4["gammaOmega_total"].replace('gamma["x0"]', "G0")
    .replace('gamma["x4"]', "G4"), RECORD_NAMES)
G0, G4 = sp.Symbol("G0"), sp.Symbol("G4")  # the record's gamma["x0"], gamma["x4"]
record_matrix = (total_record.coeff(G0) * gamma[0]
                 + total_record.coeff(G4) * gamma[4])
check(matrix_is_zero(slash - gravity_term) and matrix_is_zero(slash - record_matrix)
      and record_passed(REPORT_WL_B4, "gammaOmega_total"),
      "gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) exactly",
      record=f"{THEORY_B4_FILE}, formula gammaOmega_total")
check(matrix_is_zero(at_b4_1(slash) - 3 * H * gamma[0])
      and record_passed(REPORT_LEAD, "gamma_Omega_equals_3H_gamma0"),
      "at b4 = 1 (and for every constant b4): gamma^mu Omega_mu = 3 H gamma^(x0)",
      record=f"{REPORT_LEAD}, check gamma_Omega_equals_3H_gamma0")
```

Out [15]:

```
gamma^x0 Omega_x0 = (0) gamma^(x4) + (0) gamma^(x0)
gamma^x1 Omega_x1 = (Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x2 Omega_x2 = (Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x3 Omega_x3 = (Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x4 Omega_x4 = (0) gamma^(x4) + (0) gamma^(x0)
gamma^x5 Omega_x5 = (-Derivative(a4(x4), x4)/2 + Derivative(b4(x4), x4)/(4*b4(x4)))
    gamma^(x4) + (H/2) gamma^(x0)
gamma^x6 Omega_x6 = (-Derivative(a4(x4), x4)/2 + Derivative(b4(x4), x4)/(4*b4(x4)))
    gamma^(x4) + (H/2) gamma^(x0)
gamma^x7 Omega_x7 = (-Derivative(a4(x4), x4)/2 + Derivative(b4(x4), x4)/(4*b4(x4)))
    gamma^(x4) + (H/2) gamma^(x0)
PASS the eight terms gamma^mu Omega_mu (no sum) with b4
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    gammaOmega_per_direction
PASS at b4 = 1 the eight terms equal the record
     reproduces Revision/theory/field-theory.json, formula gammaOmega_per_direction
RESULT x4 part from the inflating x1, x2, x3 = 3*Derivative(a4(x4), x4)/2
RESULT x4 part from the deflating x5, x6, x7 = -3*Derivative(a4(x4), x4)/2 +
    3*Derivative(b4(x4), x4)/(4*b4(x4))
RESULT x0 part from all directions = 3*H
PASS the a4' parts of the x4 terms cancel, (3/4) b4'/b4 remains; the x0 terms add to 3H
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    gammaOmega_x4_terms_cancel (b4 = 1)
PASS gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) exactly
     reproduces Revision/theory/field-theory-b4.json, formula gammaOmega_total
PASS at b4 = 1 (and for every constant b4): gamma^mu Omega_mu = 3 H gamma^(x0)
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check gamma_Omega_equals_3H_gamma0
```

Text cell:

```
The next cell draws the table of the previous cell as three bar charts, one bar for
each direction $\mu$. The coefficient $\alpha_\mu$ of $\gamma^{(x4)}$ is the sum of
a part proportional to $a_4'$ and a part proportional to $b_4'/b_4$; the left chart
shows the first in units of $a_4'$, the middle chart the second in units of
$b_4'/b_4$, and the right chart the coefficient $\beta_\mu$ of $\gamma^{(x0)}$ in
units of $H$. The left bars cancel in pairs (three up, three down: total $0$); the
middle bars come only from the extra times (three times $\frac14$: total
$\frac34$); the right bars all point the same way (six times $\frac12$: total $3$).
```

In [16]:

```
a4_part = [float(sp.simplify(at_b4_1(alpha) / a4p)) for alpha, _ in per_direction]
b4_part = [float(sp.simplify(alpha.subs(a4p, 0) / (b4p / b4)))
           for alpha, _ in per_direction]  # the part with b4'/b4
beta_units = [float(sp.simplify(beta / H)) for _, beta in per_direction]  # beta / H
positions = np.arange(8)
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2), sharey=True)
colors = ["gray"] + ["tab:red"] * 3 + ["gray"] + ["tab:blue"] * 3
```

```
panels = [(a4_part, "$\\gamma^{(x4)}$, part / $a_4'$"),
          (b4_part, "$\\gamma^{(x4)}$, part / $(b_4'/b_4)$"),
          (beta_units, "$\\gamma^{(x0)}$ / $H$")]
for ax, (heights, title) in zip(axes, panels):
    ax.bar(positions, heights, color=colors, width=0.6)
    ax.set_title(f"{title}: total {sum(heights):g}", fontsize=10)
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, NAMES)
    ax.set_xlabel("direction $\\mu$")
axes[0].set_ylabel("coefficient (in the unit named in the title)")
save_figure(fig, "gamma_omega_per_direction",
            "The eight terms $\\gamma^\\mu\\Omega_\\mu$ (no sum) of the diagonal "
            "vielbein, each equal to $\\alpha_\\mu\\gamma^{(x4)} + "
            "\\beta_\\mu\\gamma^{(x0)}$; horizontal axis the direction $\\mu$ (red: "
            "inflating 3-space, blue: deflating extra times, gray: hidden direction "
            "and time). Left: the part of $\\alpha_\\mu$ proportional to $a_4'$, in "
            "units of $a_4'$: $+1/2$ for $x1, x2, x3$ and $-1/2$ for $x5, x6, x7$, "
            "total $0$. Middle: the part proportional to $b_4'/b_4$, in units of "
            "$b_4'/b_4$: $1/4$ for each extra time, total $3/4$. Right: $\\beta_\\mu$ "
            "in units of $H$: six times $1/2$, total $3$. So $\\gamma^\\mu\\Omega_\\mu "
            "= 3H\\gamma^{(x0)} + \\frac34(b_4'/b_4)\\gamma^{(x4)}$.")
```

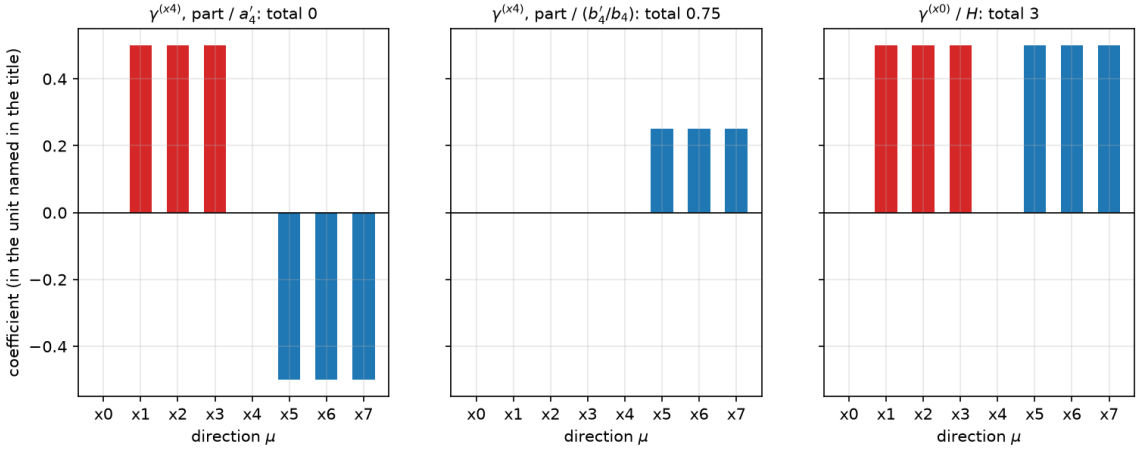

Figure 187: The eight terms $\gamma^\mu\Omega_\mu$ (no sum) of the diagonal vielbein, each equal to $\alpha_\mu\gamma^{(x4)} + \beta_\mu\gamma^{(x0)}$; horizontal axis the direction μ (red: inflating 3-space, blue: deflating extra times, gray: hidden direction and time). Left: the part of α_μ proportional to a_4', in units of a_4': $+1/2$ for $x1, x2, x3$ and $-1/2$ for $x5, x6, x7$, total 0. Middle: the part proportional to b_4'/b_4, in units of b_4'/b_4: $1/4$ for each extra time, total $3/4$. Right: β_μ in units of H: six times $1/2$, total 3. So $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$. (Notebook 06a, figure 6.)

Out [16]:

```
Figure 06a.6 saved as Revision/textbook/figures/06a_6_gamma_omega_per_direction.png
```

Text cell:

The next cell shows the same sum as pictures of the 16 x 16 matrices at the sample point ($H = 1$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$): the sum of the three inflating terms, the sum of the three deflating terms, the total $\gamma^\mu\Omega_\mu$, and $3H\gamma^{(x0)} + \frac34(b_4'/b_4)\gamma^{(x4)}$. The first two pictures contain the $\gamma^{(x4)}$ pattern with opposite colours (entries $\frac32 a_4' = 0.75$ in the first, $\frac32 a_4' - \frac34 b_4'/b_4 = 0.375$ in the second), on top of the same $\gamma^{(x0)}$ pattern (entries $\frac32 H = 1.5$); in the total only the $\gamma^{(x4)}$ entries $\frac34 b_4'/b_4 = 0.375$ remain next to $3H\gamma^{(x0)}$, and the total is the picture of the predicted matrix. The author's $\gamma^{(x0)}$ has the 8 x 8 identity matrix in its upper-right and lower-left blocks and zeros elsewhere: it maps the first eight spinor components onto the last eight and back, so $3H\gamma^{(x0)}$ shows the value 3 on two diagonal lines. The cell first checks the block form of all eight gammas against the record's list 'field_equation_blocks': each entry $\{xa, \bar t, t\}$ says that $\gamma^{(xa)}$ has the 8 x 8 block $\bar t$ in its upper-right corner, the block t in its lower-left corner and zeros in the two diagonal blocks. The record is written with curly brackets; replacing them by square brackets turns each entry into JSON, which 'json.loads' reads.

In [17]:

```
I8, Z8 = sp.eye(8), sp.zeros(8, 8)  # 8 x 8 identity and zero matrices
blocks_ok = True
for entry in FORMULAS["field_equation_blocks"]:  # eight texts {"xa", tb, t}
    name, tb, t = json.loads(entry.replace("{", "[").replace("}", "]"))
    a = NAMES.index(name)  # the position of the direction xa (0 ... 7)
    # gamma^(xa) must be the block matrix with the zero blocks on the diagonal
    blocks_ok = blocks_ok and gamma[a] == sp.BlockMatrix(
        [[Z8, sp.Matrix(tb)], [sp.Matrix(t), Z8]]).as_explicit()
check(blocks_ok and len(FORMULAS["field_equation_blocks"]) == 8,
      "the eight gammas have the block form of the record",
      record=f"{THEORY_FILE}, formula field_equation_blocks")
check(gamma[0] == sp.BlockMatrix([[Z8, I8], [I8, Z8]]).as_explicit(),
      "gamma^(x0) has the 8 x 8 identity in its two off-diagonal blocks")
panels = [
    ("sum over $x1, x2, x3$", sum((gup[k] * Omega[k] for k in (1, 2, 3)), Z16)),
    ("sum over $x5, x6, x7$", sum((gup[k] * Omega[k] for k in (5, 6, 7)), Z16)),
    ("total $\\gamma^\\mu\\Omega_\\mu$", slash),
    ("$3H\\gamma^{(x0)} + \\frac{3}{4}\\frac{b_4'}{b_4}\\gamma^{(x4)}$", gravity_term),
]
fig, axes = plt.subplots(2, 2, figsize=(8.0, 7.6))
for ax, (title, matrix) in zip(axes.flat, panels):
    table = np.array([[at_sample(v).real for v in row] for row in matrix.tolist()])
    image = ax.imshow(table, cmap="RdBu_r", vmin=-3.5, vmax=3.5)
    ax.set_title(title, fontsize=10)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.7, label="entry value")
save_figure(fig, "gamma_omega_cancellation",
            "The sum as pictures of 16 x 16 matrices at the sample point "
            "$H = 1$, $z = \\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ "
```

```
            "(red positive, blue negative, white zero; colour scale from $-3.5$ to "
            "$3.5$). Top left: the sum of the three inflating terms "
            "$\\gamma^\\mu\\Omega_\\mu$, $\\mu = x1, x2, x3$; top right: the sum of the "
            "three deflating terms, $\\mu = x5, x6, x7$; their $\\gamma^{(x4)}$ parts "
            "have opposite signs and sizes $0.75$ and $0.375$. Bottom left: the total "
            "over all eight directions; bottom right: $3H\\gamma^{(x0)} + "
            "\\frac34(b_4'/b_4)\\gamma^{(x4)}$. The two bottom pictures are identical; "
            "for a constant $b_4$ the faint $\\gamma^{(x4)}$ entries would vanish.")
```

Out [17]:

```
PASS the eight gammas have the block form of the record
     reproduces Revision/theory/field-theory.json, formula field_equation_blocks
PASS gamma^(x0) has the 8 x 8 identity in its two off-diagonal blocks
```

Out [17]:

```
Figure 06a.7 saved as Revision/textbook/figures/06a_7_gamma_omega_cancellation.png
```

Text cell:

```
## 13. Why $a_4$ drops out and $b_4$ does not: the divergence form

Two facts explain the result without computing all of $\Omega_\mu$.

**Fact 1.** For each $\mu$ separately, $\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu
= 0$ (no sum). Now sum the covariant constancy $D_\mu\gamma^\nu = 0$ over
$\mu = \nu$: $\sum_\mu\partial_\mu\gamma^\mu + \sum_{\mu,\lambda}
\Gamma^\mu{}_{\mu\lambda}\gamma^\lambda + \sum_\mu(\Omega_\mu\gamma^\mu -
\gamma^\mu\Omega_\mu) = 0$. The standard contraction $\sum_\mu
\Gamma^\mu{}_{\mu\lambda} = \partial_\lambda \ln\sqrt{|g|}$ turns the first two sums
into $\frac{1}{\sqrt{|g|}}\sum_\mu\partial_\mu(\sqrt{|g|}\,\gamma^\mu)$ (product
rule), and Fact 1 turns the last sum into $-2\sum_\mu\gamma^\mu\Omega_\mu$. Solving
for the contraction gives

$$\gamma^\mu\Omega_\mu = \frac{1}{2\sqrt{|g|}}\sum_\mu \partial_\mu\big(\sqrt{|g|}\,
\gamma^\mu\big) .$$

**Fact 2.** For the diagonal vielbein, $\sqrt{|g|}\gamma^\mu = \big(\prod_{c \ne
\mu} f_c\big)\gamma^{(\mu)}$, because $\sqrt{|g|} = \prod_c f_c$ and $\gamma^\mu =
\gamma^{(\mu)}/f_\mu$. Now $\prod_{c \ne x4} f_c = b_4^{3/2}\cos z$ (the factors
$e^{3a_4}$ and $e^{-3a_4}$ cancel, the three factors $b_4^{1/2}$ of the extra times
remain), so the time direction contributes $\frac{1}{2b_4^{3/2}\cos z}\,
\partial_{x4}(b_4^{3/2}\cos z) = \frac34\,\frac{b_4'}{b_4}$: nothing when $b_4$ is
constant. And $\prod_{c \ne x0} f_c = b_4^{3/2}\sin z$ gives
$\frac{1}{2b_4^{3/2}\cos z}\,\partial_{x0}(b_4^{3/2}\sin z) =
\frac{6H\cos z}{2\cos z} = 3H$. The other six products do not depend on their own
coordinate.

The next cell checks Fact 1 for every $\mu$, the divergence form, and the eight
products of Fact 2.
```

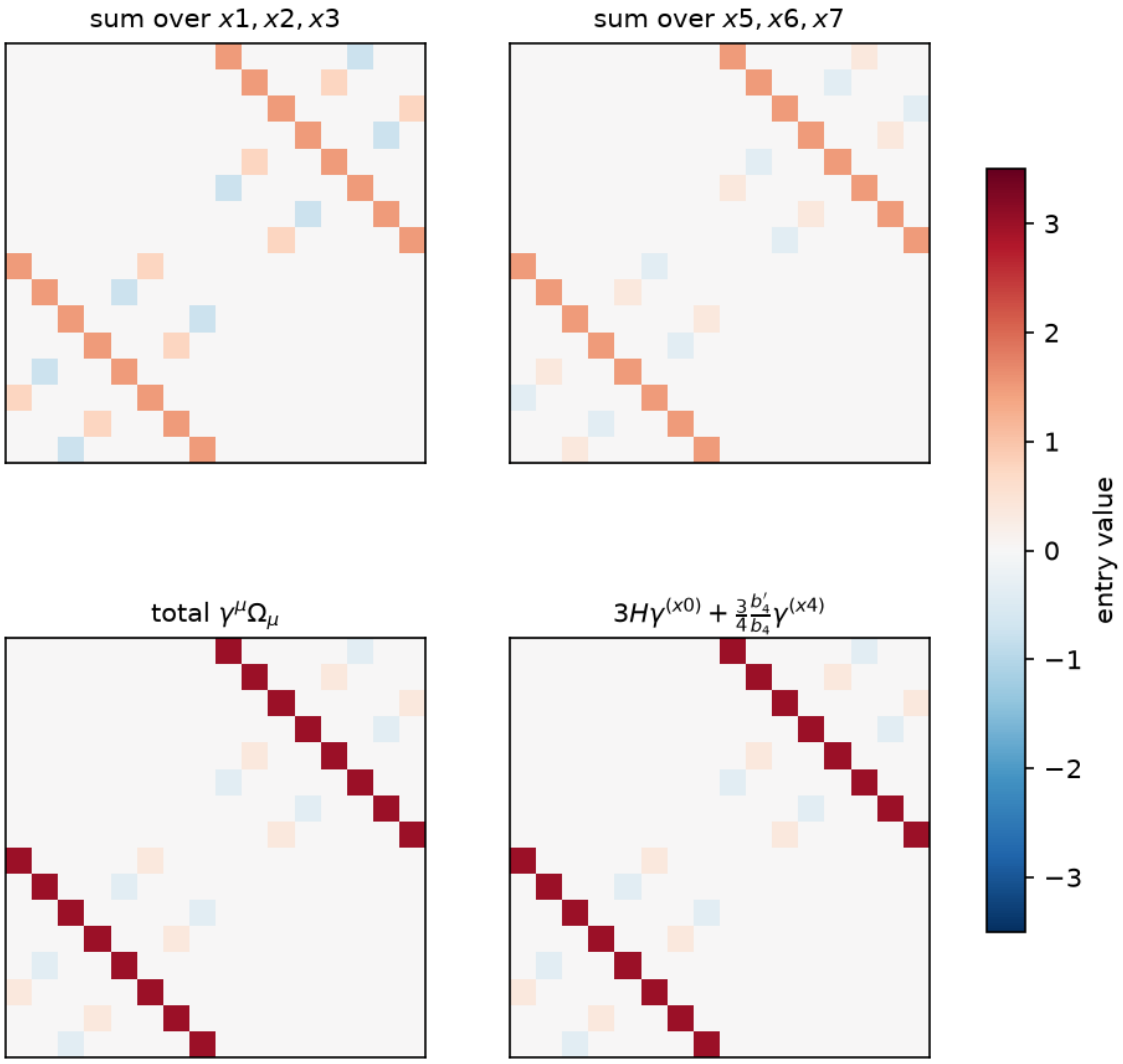

Figure 188: The sum as pictures of 16 x 16 matrices at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ (red positive, blue negative, white zero; colour scale from -3.5 to 3.5). Top left: the sum of the three inflating terms $\gamma^{\mu}\Omega_{\mu}$, $\mu = x1, x2, x3$; top right: the sum of the three deflating terms, $\mu = x5, x6, x7$; their $\gamma^{(x4)}$ parts have opposite signs and sizes 0.75 and 0.375. Bottom left: the total over all eight directions; bottom right: $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$. The two bottom pictures are identical; for a constant b_4 the faint $\gamma^{(x4)}$ entries would vanish. (Notebook 06a, figure 7.)

In [18]:

```
anticommutes = all(matrix_is_zero(gup[mu] * Omega[mu] + Omega[mu] * gup[mu])
                   for mu in range(8))
check(anticommutes, "{gamma^mu, Omega_mu} = 0 for each mu separately",
      record=f"{REPORT_PY}, check anticommutator_gamma_Omega_vanishes (b4 = 1)")
sqrt_g = b4 ** sp.Rational(3, 2) * sp.cos(z)  # sqrt|det g|
divergence = sum(((sqrt_g * gup[mu]).diff(x[mu]) for mu in range(8)), Z16) / (2 * sqrt_g)
check(matrix_is_zero(divergence - gravity_term)
      and record_passed(REPORT_WL_B4, "gammaOmega_divergence_form")
      and record_passed(REPORT_LEAD, "gamma_Omega_divergence_formula"),
      "(1/(2 sqrt|g|)) d_mu (sqrt|g| gamma^mu) = 3 H gamma^(x0) + (3/4)(b4'/b4) "
      "gamma^(x4)",
      record=f"{REPORT_WL_B4}, check gammaOmega_divergence_form")
products = [sp.simplify(sp.prod([f[c] for c in range(8) if c != b])) for b in range(8)]
for b in (0, 4):
    say(f"product of the f_c with c != {NAMES[b]}: {products[b]}")
own_coordinate = [sp.simplify(sp.diff(products[b], x[b]) / (2 * sqrt_g))
                  for b in range(8)]
say("(1/(2 sqrt|g|)) d_b (product without b), b = x0 ... x7: "
    f"{own_coordinate}")
expected_own = [3 * H, 0, 0, 0, sp.Rational(3, 4) * b4p / b4, 0, 0, 0]
check(all(is_zero(own_coordinate[b] - expected_own[b]) for b in range(8)),
      "only the hidden direction (3 H) and the time ((3/4) b4'/b4) contribute")
```

Out [18]:

```
PASS {gamma^mu, Omega_mu} = 0 for each mu separately
     reproduces Revision/theory/reports/python-field-theory.json, check
    anticommutator_gamma_Omega_vanishes (b4 = 1)
PASS (1/(2 sqrt|g|)) d_mu (sqrt|g| gamma^mu) = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4)
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    gammaOmega_divergence_form
product of the f_c with c != x0: b4(x4)**(3/2)*sin(6*H*x0)
product of the f_c with c != x4: b4(x4)**(3/2)*cos(6*H*x0)
(1/(2 sqrt|g|)) d_b (product without b), b = x0 ... x7: [3*H, 0, 0, 0,
    3*Derivative(b4(x4), x4)/(4*b4(x4)), 0, 0, 0]
PASS only the hidden direction (3 H) and the time ((3/4) b4'/b4) contribute
```

Text cell:

```
The next cell checks Fact 2 with numbers instead of symbols, as a student would with
a calculator. For the hidden direction it computes the derivative of
$b_4^{3/2}\sin z$ with respect to $x_0$ by a **central difference**,
$\big(F(x_0 + h) - F(x_0 - h)\big)/(2h)$ with the small step $h = 10^{-5}$, divides
by $2b_4^{3/2}\cos z$ and plots the result over the whole range of $z$ (with
$H = 1$; $b_4$ does not depend on $x_0$ and cancels). For the time direction it
takes one illustrative history, $b_4(x_4) = 1 + \frac12\sin x_4$ (positive, an
illustration only), computes the derivative of $b_4^{3/2}\cos z$ with respect to
$x_4$ in the same way, divides by $2b_4^{3/2}\cos z$ ($\cos z$ cancels) and
compares with the exact $\frac34 b_4'/b_4$; for $b_4 = 1$ the coefficient is
```

```
exactly $0$, drawn as a line for comparison. The differences from the exact values
are of the size $h^2$ times third derivatives (about $10^{-9}$), far below the
check limit $10^{-6}$.
```

In [19]:

```
H_value = 1.0
x0_values = np.linspace(0.002, np.pi / 12 - 0.002, 300)  # z = 6 x0 inside (0, pi/2)
step = 1e-5  # the step h of the central difference


def hidden_density(x0_value):
    return np.sin(6 * H_value * x0_value)  # sqrt|g| e_(x0)^x0 / b4^(3/2) = sin z


derivative = (hidden_density(x0_values + step)
              - hidden_density(x0_values - step)) / (2 * step)
ratio = derivative / (2 * np.cos(6 * H_value * x0_values))  # should be 3 H = 3
x4_values = np.linspace(-6.0, 6.0, 300)


def b4_history(x4_value):
    return 1.0 + 0.5 * np.sin(x4_value)  # an illustrative b4(x4) > 0


def time_density(x4_value):
    return b4_history(x4_value) ** 1.5  # sqrt|g| e_(x4)^x4 / cos z = b4^(3/2)


time_derivative = (time_density(x4_values + step)
                   - time_density(x4_values - step)) / (2 * step)
time_ratio = time_derivative / (2 * time_density(x4_values))  # (1/(2 b4^1.5)) d b4^1.5
exact_time = 0.75 * 0.5 * np.cos(x4_values) / b4_history(x4_values)  # (3/4) b4'/b4
report("largest |central difference - 3 H| over the grid",
       f"{np.max(np.abs(ratio - 3.0)):.1e}")
report("largest |central difference - (3/4) b4'/b4| over the grid",
       f"{np.max(np.abs(time_ratio - exact_time)):.1e}")
check(np.max(np.abs(ratio - 3.0)) < 1e-6
      and np.max(np.abs(time_ratio - exact_time)) < 1e-6,
      "the numerical derivatives give 3 H and (3/4) b4'/b4")
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
zz = 6 * H_value * x0_values
ax_left.plot(zz, ratio, label="$\\partial_{x0}(b_4^{3/2}\\sin z)/"
                              "(2b_4^{3/2}\\cos z)$")
ax_left.set_ylim(0.0, 3.8)
ax_left.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_left.set_ylabel("coefficient of $\\gamma^{(x0)}$ (units of $H$)")
ax_left.set_title("Hidden direction: the constant $3H$")
ax_left.legend(fontsize=8, loc="lower center")
ax_right.plot(x4_values, time_ratio,
              label="central difference, $b_4 = 1 + \\frac{1}{2}\\sin x_4$")
ax_right.plot(x4_values, exact_time, ":", color="black", linewidth=2.5,
              label="exact $\\frac{3}{4} b_4'/b_4$")
```

```
ax_right.plot(x4_values, np.zeros_like(x4_values), "--", color="gray",
              label="$b_4 = 1$: exactly $0$")
ax_right.set_ylim(-1.05, 0.6)  # room for the legend below the curves
ax_right.set_xlabel("time $x_4$ (units $1/H$)")
ax_right.set_ylabel("coefficient of $\\gamma^{(x4)}$")
ax_right.set_title("Time direction: $\\frac{3}{4} b_4'/b_4$")
ax_right.legend(fontsize=8, loc="lower center")
save_figure(fig, "divergence_form",
            "The divergence form of $\\gamma^\\mu\\Omega_\\mu$, with $H = 1$, checked "
            "by central differences with step $10^{-5}$. Left: the coefficient of "
            "$\\gamma^{(x0)}$, $\\partial_{x0}(b_4^{3/2}\\sin z)/(2b_4^{3/2}\\cos z)$, "
            "versus the hidden angle $z$ in radians: the constant $3$ for every $z$. "
            "Right: the coefficient of $\\gamma^{(x4)}$, "
            "$\\partial_{x4}(b_4^{3/2}\\cos z)/(2b_4^{3/2}\\cos z)$, versus the time "
            "$x_4$ for the illustrative history $b_4 = 1 + \\frac12\\sin x_4$ (solid), "
            "on the exact $\\frac34 b_4'/b_4$ (dotted), and $0$ for $b_4 = 1$ "
            "(dashed): the time direction contributes only through a changing $b_4$.")
```

Out [19]:

```
RESULT largest |central difference - 3 H| over the grid = 1.9e-09
RESULT largest |central difference - (3/4) b4'/b4| over the grid = 2.5e-11
PASS the numerical derivatives give 3 H and (3/4) b4'/b4
```

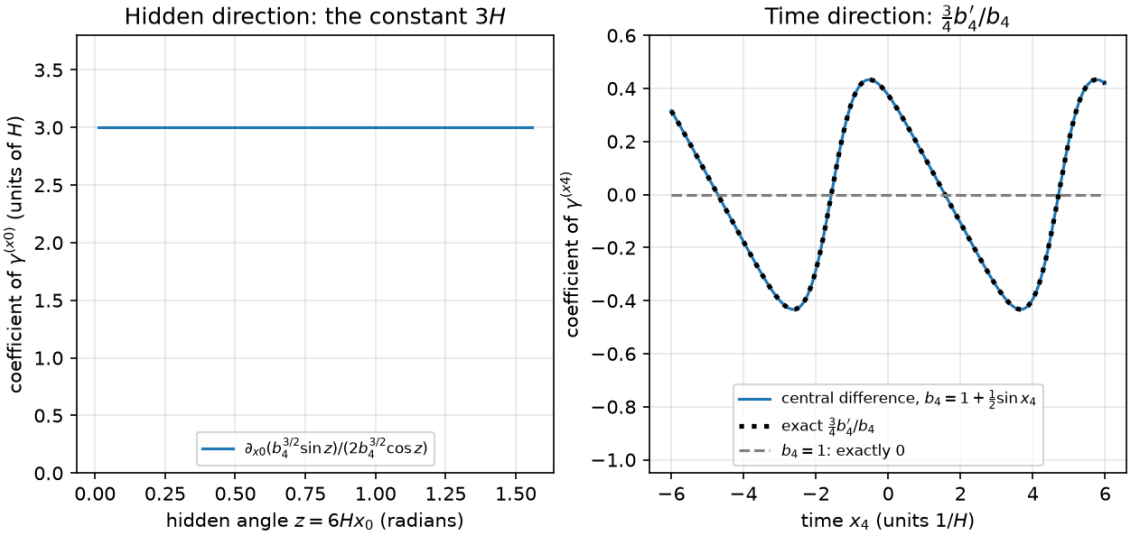

Figure 189: The divergence form of $\gamma^\mu\Omega_\mu$, with $H = 1$, checked by central differences with step 10^{-5}. Left: the coefficient of $\gamma^{(x0)}$, $\partial_{x0}(b_4^{3/2}\sin z)/(2b_4^{3/2}\cos z)$, versus the hidden angle z in radians: the constant 3 for every z. Right: the coefficient of $\gamma^{(x4)}$, $\partial_{x4}(b_4^{3/2}\cos z)/(2b_4^{3/2}\cos z)$, versus the time x_4 for the illustrative history $b_4 = 1 + \frac{1}{2}\sin x_4$ (solid), on the exact $\frac{3}{4}b_4'/b_4$ (dotted), and 0 for $b_4 = 1$ (dashed): the time direction contributes only through a changing b_4. (Notebook 06a, figure 8.)

Out [19]:

```
Figure 06a.8 saved as Revision/textbook/figures/06a_8_divergence_form.png
```

Text cell:

```
## 14. The covariant derivative of a spinor and the Dirac operator written out

The next cell takes a completely general spinor field: 16 unknown functions
$\psi_1, \dots, \psi_{16}$ of all eight coordinates. It computes the covariant
derivatives $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and the Dirac operator
$\gamma^\mu D_\mu\Psi$, and checks that

$$\gamma^\mu D_\mu\Psi = \sum_a \frac{1}{f_a}\gamma^{(a)}\partial_a\Psi +
3H\gamma^{(x0)}\Psi + \tfrac34\,\frac{b_4'}{b_4}\,\gamma^{(x4)}\Psi ,$$

that is, with the factors written out, $\tan z\,\gamma^{(x0)}\partial_0\Psi +
e^{-a_4}s^{-1}(\gamma^{(x1)}\partial_1 + \gamma^{(x2)}\partial_2 +
\gamma^{(x3)}\partial_3)\Psi + \gamma^{(x4)}\partial_4\Psi +
e^{a_4}b_4^{-1/2}s^{-1}(\gamma^{(x5)}\partial_5 + \gamma^{(x6)}\partial_6 +
\gamma^{(x7)}\partial_7)\Psi + 3H\gamma^{(x0)}\Psi +
\frac34(b_4'/b_4)\gamma^{(x4)}\Psi$. It then compares all 16 components with the
16 component equations of the record with $b_4$, and at $b_4 = 1$ with those of
the record of $b_4 = 1$ (‘field_equation_components‘; their right-hand side
$V\Psi_A$, $V = m + U'(S)$, is not needed here).

Finally it checks when the term $M\Psi$, $M = 3H\gamma^{(x0)} +
\frac34(b_4'/b_4)\gamma^{(x4)}$, can vanish for a field that is not zero. Since
$(\gamma^{(x0)})^2 = 1$, $(\gamma^{(x4)})^2 = -1$ and the two anticommute,
$MM = \big(9H^2 - \frac{9}{16}(b_4'/b_4)^2\big)\,1$. When the number in front of
$1$ is not zero, $M\Psi = 0$ gives $\Psi = 0$ (multiply by $M$ and divide by that
number): this is so for every constant $b_4$, where $M = 3H\gamma^{(x0)}$, and the
WolframScript report ‘wolfram-field-theory.json‘ records the statement for both
fields at $b_4 = 1$ (checks ‘nontriviality_1_dirac16complex‘ and
‘nontriviality_2_dirac16complex00‘). Where $|b_4'/b_4| = 4H$, however, $MM = 0$:
the cell computes that $\gamma^{(x0)} \pm \gamma^{(x4)}$ has rank 8, so there $M$
sends an 8-dimensional set of nonzero fields to zero (the record with $b_4$ checks
the same, check ‘gravity_term_square_b4‘).
```

In [20]:

```
import re  # text patterns, to rename the record's symbols

psi = sp.Matrix([sp.Function(f"psi{A}")(*x) for A in range(1, 17)])  # 16 functions
D = [psi.diff(x[mu]) + Omega[mu] * psi for mu in range(8)]  # D_mu Psi
dirac = sum((gup[mu] * D[mu] for mu in range(8)), sp.zeros(16, 1))
written_out = sum((gamma[a] * psi.diff(x[a]) / f[a] for a in range(8)),
                  sp.zeros(16, 1)) + gravity_term * psi
check(all(is_zero(v) for v in dirac - written_out)
      and record_passed(REPORT_WL_B4, "Dirac_operator_explicit_C"),
      "gamma^mu D_mu Psi = sum_a (1/f_a) gamma^a d_a Psi + 3 H gamma^(x0) Psi + "
      "(3/4)(b4'/b4) gamma^(x4) Psi",
      record=f"{REPORT_WL_B4}, check Dirac_operator_explicit_C")
# Rename the derivatives d psi_B / d x_k and then the functions psi_B as plain
```

```
# symbols dPsi<B>x<k> and Psi<B> (derivatives first: renaming psi_B first would
# turn its derivatives into derivatives of a constant symbol, which are zero).
rename_derivatives = {sp.Derivative(psi[B], x[k]): sp.Symbol(f"dPsi{B + 1}x{k}")
                      for B in range(16) for k in range(8)}
rename_functions = {psi[B]: sp.Symbol(f"Psi{B + 1}") for B in range(16)}


def same_components(equations, at_b4_equal_1):
    """True when the 16 left sides of the record equal ours (at b4 = 1 if asked)."""
    same = True
    for A, equation in enumerate(equations):
        left = equation.split("==")[0]  # the record: left side == V*Psi[A]
        left = left.replace(chr(34), "")  # chr(34) is the double-quote character
        # dd[x4, Psi[14]] (the derivative of Psi_14 by x4) becomes dPsi14x4:
        left = re.sub(r"dd\[x(\d), Psi\[(\d+)\]\]", r"dPsi\2x\1", left)
        left = re.sub(r"Psi\[(\d+)\]", r"Psi\1", left)
        record_left = wolfram_to_sympy(left, RECORD_NAMES)
        mine = dirac[A].subs(rename_derivatives).subs(rename_functions)
        if at_b4_equal_1:
            mine = at_b4_1(mine)
        same = same and is_zero(mine - record_left)
    return same


check(same_components(FORMULAS_B4["field_equation_components"], False)
      and same_components(FORMULAS["field_equation_components"], True),
      "all 16 components of the Dirac operator equal the records (with b4, and at "
      "b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula field_equation_components")
c_b4 = sp.Rational(3, 4) * b4p / b4  # the coefficient of gamma^(x4)
square = (gravity_term * gravity_term - (9 * H**2 - c_b4**2) * I16)
ranks = [(gamma[0] + sign * gamma[4]).rank() for sign in (1, -1)]
report("ranks of gamma^(x0) + gamma^(x4) and gamma^(x0) - gamma^(x4)", ranks)
both_fields = all(record_passed(REPORT_WL, name) for name in [
    "nontriviality_1_dirac16complex", "nontriviality_2_dirac16complex00"])
check(matrix_is_zero(square) and gamma[0] * gamma[0] == I16 and ranks == [8, 8]
      and both_fields and record_passed(REPORT_PY_B4, "gravity_term_square_b4"),
      "M M = (9 H^2 - (9/16)(b4'/b4)^2) 1: M Psi = 0 only for Psi = 0 unless "
      "|b4'/b4| = 4 H (always for constant b4), and there M has rank 8",
      record=f"{REPORT_PY_B4}, check gravity_term_square_b4; {REPORT_WL}, checks "
             "nontriviality_1_dirac16complex and nontriviality_2_dirac16complex00")
```

Out [20]:

```
PASS gamma^mu D_mu Psi = sum_a (1/f_a) gamma^a d_a Psi + 3 H gamma^(x0) Psi +
    (3/4)(b4'/b4) gamma^(x4) Psi
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    Dirac_operator_explicit_C
PASS all 16 components of the Dirac operator equal the records (with b4, and at b4 = 1)
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula field_equation_components
RESULT ranks of gamma^(x0) + gamma^(x4) and gamma^(x0) - gamma^(x4) = [8, 8]
PASS M M = (9 H^2 - (9/16)(b4'/b4)^2) 1: M Psi = 0 only for Psi = 0 unless |b4'/b4| = 4 H
```

```
    (always for constant b4), and there M has rank 8
     reproduces Revision/theory/reports/python-field-theory-b4.json, check
    gravity_term_square_b4; Revision/theory/reports/wolfram-field-theory.json, checks
    nontriviality_1_dirac16complex and nontriviality_2_dirac16complex00
```

Text cell:

```
The deflation does not appear in $\gamma^\mu\Omega_\mu$, but it does appear in the
Dirac operator: through the factors $1/f_a$ in front of the derivatives. The next
cell draws these factors. On the left, versus the time for the illustrative
history $a_4 = x_4$, $b_4 = 1$ (with $H = 1$, $z = \pi/4$): as 3-space inflates, the
3-space derivatives are weighted less and less ($e^{-a_4}/s$), and as the extra
times deflate, the extra-time derivatives are weighted more and more
($e^{a_4}b_4^{-1/2}/s$). On the right, versus $z$ at $x_4 = 0$: the
hidden-direction factor $\tan z$ and the common factor $1/s$ of the six warped
directions.
```

In [21]:

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
coefficient = [sp.lambdify(X4_SYMBOL, (1 / fa).subs(history).subs(
    {H: 1, x0: sp.pi / 24})) for fa in f]
ax_left.semilogy(times, coefficient[1](times),
                 label="$1/f_1 = e^{-a_4}/s$ (3-space derivatives)")
ax_left.semilogy(times, coefficient[5](times), "--",
                 label="$1/f_5 = e^{a_4}b_4^{-1/2}/s$ (extra-time derivatives)")
ax_left.semilogy(times, np.ones_like(times), ":", color="black",
                 label="$1/f_4 = 1$ (time derivative)")
ax_left.set_xlabel("time $x_4$ (units $1/H$), history $a_4 = x_4$, $b_4 = 1$")
ax_left.set_ylabel("factor in front of the derivative (log axis)")
ax_left.set_title("Where the deflation enters the field equation")
ax_left.legend(fontsize=8)
zz = np.linspace(0.05, np.pi / 2 - 0.15, 300)
ax_right.plot(zz, np.tan(zz), label="$1/f_0 = \\tan z$ (hidden derivative)")
ax_right.plot(zz, np.sin(zz) ** (-1 / 6), "--",
              label="$1/s = \\sin^{-1/6} z$ (warped directions, $a_4 = 0$)")
ax_right.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_right.set_ylabel("factor (pure number)")
ax_right.set_title("Dependence on the hidden direction")
ax_right.legend(fontsize=8)
save_figure(fig, "derivative_coefficients",
            "The factors $1/f_a$ in front of the derivatives in the Dirac operator "
            "$\\gamma^\\mu D_\\mu = \\sum_a f_a^{-1}\\gamma^{(a)}\\partial_a + "
            "3H\\gamma^{(x0)} + \\frac34(b_4'/b_4)\\gamma^{(x4)}$. Left: versus the "
            "time $x_4$ in units $1/H$ at $z = \\pi/4$ for the illustrative history "
            "$a_4 = x_4$, $b_4 = 1$, on a logarithmic axis: the 3-space factor "
            "$e^{-a_4}/s$ falls and the extra-time factor $e^{a_4}b_4^{-1/2}/s$ "
            "rises; this is how the deflation of the extra times enters the field "
            "equation. Right: versus the hidden angle $z$ in radians at "
            "$x_4 = 0$: the hidden factor $\\tan z$ and the factor $\\sin^{-1/6}z$ of "
            "the six warped directions.")
```

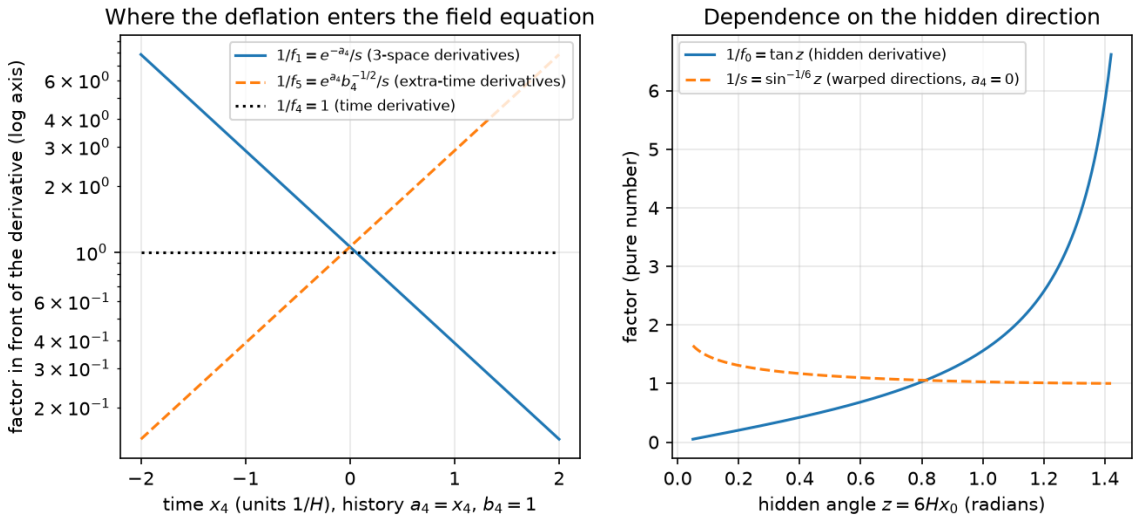

Figure 190: The factors $1/f_a$ in front of the derivatives in the Dirac operator $\gamma^\mu D_\mu = \sum_a f_a^{-1}\gamma^{(a)}\partial_a + 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$. Left: versus the time x_4 in units $1/H$ at $z = \pi/4$ for the illustrative history $a_4 = x_4$, $b_4 = 1$, on a logarithmic axis: the 3-space factor e^{-a_4}/s falls and the extra-time factor $e^{a_4} b_4^{-1/2}/s$ rises; this is how the deflation of the extra times enters the field equation. Right: versus the hidden angle z in radians at $x_4 = 0$: the hidden factor $\tan z$ and the factor $\sin^{-1/6} z$ of the six warped directions. (Notebook 06a, figure 9.)

Out [21]:

```
Figure 06a.9 saved as Revision/textbook/figures/06a_9_derivative_coefficients.png
```

Text cell:

```
## 15. The negative control: extra times that INFLATE

Is the cancellation of the $a_4'$ terms an accident of the algebra, or does it come
from the deflation? The control answers this. The next cell repeats the whole
computation (Christoffel symbols, spin connection, spinor connection, curved
gammas) for a changed metric in which the extra times inflate like 3-space,
$g_{tt} = -e^{+2a_4}\sin^{1/3}z$ for $t = x5, x6, x7$, with vielbein factors
$e^{+a_4}s$ (at $b_4 = 1$, as in the lead's record, so that only the deflation is
changed). If the deflation is the reason, a term with $a_4'$ must survive. The
record of the lead's independent check finds $3a_4'\gamma^{(x4)}$; the cell checks
that it gets the same, and that the hidden term $3H\gamma^{(x0)}$ is unchanged.
```

In [22]:

```
f_control = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
             + [sp.exp(a4) * s] * 3)  # extra times INFLATING, b4 = 1
g_control = sp.diag(*[ETA[a] * f_control[a] ** 2 for a in range(8)])
e_control = sp.diag(*f_control)
Gam_control = christoffel(g_control, x)
```

```
omega_control = spin_connection(e_control, Gam_control, x, ETA)
Omega_control = spinor_connection(omega_control, S)
gup_control = curved_gammas(e_control, gamma)
slash_control = sum((gup_control[mu] * Omega_control[mu] for mu in range(8)), Z16)
alpha_control = sp.simplify(-(slash_control * gamma[4]).trace() / 16)
beta_control = sp.simplify((slash_control * gamma[0]).trace() / 16)
report("control: coefficient of gamma^(x4)", alpha_control)
report("control: coefficient of gamma^(x0)", beta_control)
lead = json.loads(repository_file(REPORT_LEAD).read_text(encoding="utf-8"))
detail = [c["detail"] for c in lead["checks"]
          if c["name"] == "negative_control_inflating_extra_times"][0]
check(matrix_is_zero(slash_control - alpha_control * gamma[4] - beta_control * gamma[0])
      and is_zero(alpha_control - 3 * sp.Derivative(a4, x4))
      and is_zero(beta_control - 3 * H)
      and "(3*Derivative(a4(x4), x4)) gamma^(x4)" in detail
      and record_passed(REPORT_LEAD, "negative_control_inflating_extra_times"),
      "control with inflating extra times: 3 a4' gamma^(x4) + 3 H gamma^(x0)",
      record=f"{REPORT_LEAD}, check negative_control_inflating_extra_times")
```

Out [22]:

```
RESULT control: coefficient of gamma^(x4) = 3*Derivative(a4(x4), x4)
RESULT control: coefficient of gamma^(x0) = 3*H
PASS control with inflating extra times: 3 a4' gamma^(x4) + 3 H gamma^(x0)
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check negative_control_inflating_extra_times
```

Text cell:

```
The next cell draws the result of the control next to the author's case: the
coefficient of $\gamma^{(x4)}$ in $\gamma^\mu\Omega_\mu$ (left) and the coefficient
of $\gamma^{(x0)}$ (right) as functions of the rate $a_4'$ (with $H = 1$ and
$b_4 = 1$). With deflating extra times (the author's metric) the left coefficient
is zero for every $a_4'$; with inflating extra times it is $3a_4'$, a straight line
through the origin. The hidden-direction coefficient $3H$ is the same in both
cases: it comes from $x0$, not from the time dependence.
```

In [23]:

```
rates = np.linspace(-2.0, 2.0, 81)  # values of a4'
rate_symbol = sp.Symbol("rate")
author_alpha = sp.lambdify(rate_symbol, sp.simplify(at_b4_1(sum(
    a for a, _ in per_direction))).subs(a4p, rate_symbol))
control_alpha = sp.lambdify(rate_symbol, alpha_control.subs(
    sp.Derivative(a4, x4), rate_symbol))
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
ax_left.plot(rates, np.broadcast_to(author_alpha(rates), rates.shape),
             label="author: deflating extra times")
ax_left.plot(rates, np.broadcast_to(control_alpha(rates), rates.shape), "--",
             label="control: inflating extra times")
ax_left.set_xlabel("rate $a_4'$ (units of $H$)")
```

```
ax_left.set_ylabel("coefficient of $\\gamma^{(x4)}$ (units of $H$)")
ax_left.set_title("The time-direction term")
ax_left.legend(fontsize=8)
ax_right.plot(rates, np.full_like(rates, float(hidden.subs(H, 1))),
              label="author: deflating extra times")
ax_right.plot(rates, np.full_like(rates, float(beta_control.subs(H, 1))), "--",
              label="control: inflating extra times")
ax_right.set_ylim(0.0, 4.0)
ax_right.set_xlabel("rate $a_4'$ (units of $H$)")
ax_right.set_ylabel("coefficient of $\\gamma^{(x0)}$ (units of $H$)")
ax_right.set_title("The hidden-direction term (identical lines)")
ax_right.legend(fontsize=8)
save_figure(fig, "negative_control",
            "The negative control, at $b_4 = 1$. Left: the coefficient of "
            "$\\gamma^{(x4)}$ in $\\gamma^\\mu\\Omega_\\mu$ versus the rate $a_4'$, "
            "both in units of $H = 1$, for the author's metric with deflating extra "
            "times (solid, zero for every rate) and for a control metric with "
            "inflating extra times (dashed, "
            "the line $3a_4'$). Right: the coefficient of $\\gamma^{(x0)}$, $3H$ in "
            "both cases (the two lines lie on top of each other). The cancellation "
            "of the time-direction terms is caused by the deflation.")
```

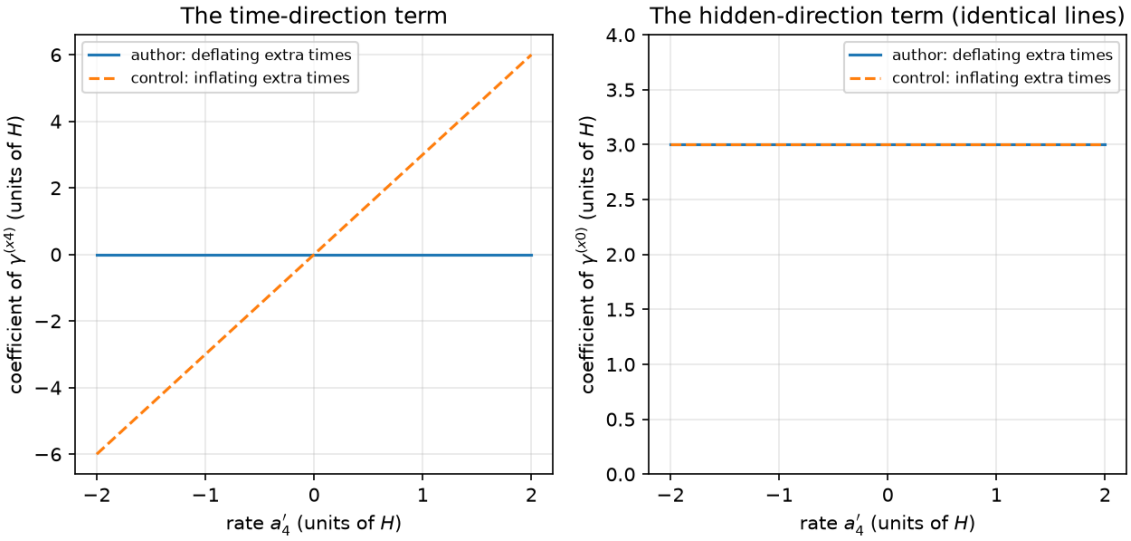

Figure 191: The negative control, at $b_4 = 1$. Left: the coefficient of $\gamma^{(x4)}$ in $\gamma^\mu\Omega_\mu$ versus the rate a_4', both in units of $H = 1$, for the author's metric with deflating extra times (solid, zero for every rate) and for a control metric with inflating extra times (dashed, the line $3a_4'$). Right: the coefficient of $\gamma^{(x0)}$, $3H$ in both cases (the two lines lie on top of each other). The cancellation of the time-direction terms is caused by the deflation. (Notebook 06a, figure 10.)

Out [23]:

```
Figure 06a.10 saved as Revision/textbook/figures/06a_10_negative_control.png
```

Text cell:

16. The last checks

Most results of this notebook are recorded several times in the Revision record, by verifiers that share no code: for the metric with b_4 the sympy checker (report ‘python-field-theory-b4.json‘) and the WolframScript checker (‘wolfram-field-theory-b4.json‘); for the background $b_4 = 1$ the sympy checker (‘python-field-theory.json‘), the WolframScript checker (‘wolfram-field-theory.json‘) and the lead’s independent check (‘emt-divergence-and-spin-connection.json‘). Each PASS line above names one of them. The last cell lists every check of these five reports that states a result reproduced here and checks that each is recorded as passed (so the record this notebook reproduces is itself a passing record). Then it checks that all ten figure files exist in the folder Revision/textbook/figures, and prints the number of checks that passed.

In [24]:

```
cited = {
    REPORT_PY_B4: ["christoffel_nonzero", "omega_nonzero", "gammaOmega_per_direction",
                   "gammaOmega_total", "gammaOmega_formula_text",
                   "gravity_term_square_b4"],
    REPORT_WL_B4: ["metric_is_the_authors_b4", "vielbein_inverse", "sqrt_det_g",
                   "christoffel_count", "vielbein_postulate", "omega_antisymmetric",
                   "omega_components", "gamma_covariantly_constant",
                   "gammaOmega_total", "gammaOmega_per_direction",
                   "gammaOmega_divergence_form", "Dirac_operator_explicit_G",
                   "Dirac_operator_explicit_C"],
    REPORT_PY: ["clifford_relations", "gammas_real", "metric_from_vielbein_equals_SPEC",
                "sqrt_det_g_equals_cos_z", "vielbein_postulate",
                "spin_connection_antisymmetric", "nontriviality_Omega_zero_iff_flat",
                "Omega_x0_and_Omega_x4_vanish", "covariant_constancy_D_mu_gamma_nu",
                "time_terms_cancel_hidden_term_survives",
                "gamma_mu_Omega_mu_equals_3H_gamma_x0",
                "anticommutator_gamma_Omega_vanishes", "divergence_of_sqrtg_gamma"],
    REPORT_WL: ["metric_is_the_authors", "sqrt_det_g_is_cos_z", "christoffel_count",
                "vielbein_postulate", "omega_antisymmetric", "omega_components",
                "Omega_components", "S_rotates_gamma_with_omega",
                "gamma_covariantly_constant", "gammaOmega_equals_3H_gamma_x0",
                "gammaOmega_x4_terms_cancel", "gammaOmega_divergence_form",
                "Dirac_operator_explicit_C", "Dirac_operator_explicit_G",
                "nontriviality_1_dirac16complex", "nontriviality_2_dirac16complex00",
                "Omega_vanishes_iff_a4prime_and_H_vanish", "block_form"],
    REPORT_LEAD: ["vielbein_reproduces_metric", "gamma_Omega_equals_3H_gamma0",
                  "gamma_Omega_divergence_formula",
                  "negative_control_inflating_extra_times"],
}
count = sum(len(names) for names in cited.values())
check(all(record_passed(path, name) for path, names in cited.items()
          for name in names),
      f"the {count} cited record checks are recorded as passed")
expected = [f"06a_{k}_{name}.png" for k, name in enumerate([
    "polar_frame_and_spin_rotation", "vielbein_factors", "christoffel_pattern",
    "omega_heat_maps", "spinor_connection_heat_maps", "gamma_omega_per_direction",
    "gamma_omega_cancellation", "divergence_form", "derivative_coefficients",
```

```
    "negative_control"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected),
      "all ten figure files exist")
all_checks_passed()
```

Out [24]:

```
PASS the 54 cited record checks are recorded as passed
PASS all ten figure files exist
ALL 36 CHECKS PASSED (notebook 06a)
```

Text cell:

```
## 17. What this notebook showed

- In the flat plane in polar coordinates the spin connection is not zero
  ($\omega_{\varphi 01} = -1$, $\gamma^\mu\Omega_\mu = \sigma_1/(2r)$), but its
  curvature is zero and a spinor rotation removes it: in flat space the spin
  connection only records how the chosen frame turns.
- For the author's metric the diagonal vielbein $f = (\cot z,\ e^{a_4}s\
  (\times 3),\ 1,\ e^{-a_4}b_4^{1/2}s\ (\times 3))$ gives the metric back, and
  $\sqrt{|\det g|} = b_4^{3/2}\cos z$ depends on the time only through $b_4$
  (PROVED, exact; reproduces the records with $b_4$ and at $b_4 = 1$).
- The metric has 25 independent nonzero Christoffel symbols, and the canonical spin
  connection has 12 independent nonzero components $\omega_{\mu ab}$, each equal to
  a rate ($a_4'$, $a_4' - b_4'/(2b_4)$ or $H$) times a factor that never vanishes;
  the vielbein postulate (512 equations), the antisymmetry and the covariant
  constancy of the gammas (64 matrix equations) hold exactly (PROVED; reproduces the
  records).
- $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac34(b_4'/b_4)\gamma^{(x4)}$ exactly,
  for every $H > 0$ and all functions $a_4(x_4)$, $b_4(x_4) > 0$ (PROVED; reproduces
  the records with $b_4$, and at $b_4 = 1$ the Python, Wolfram and lead records):
  the time-direction terms $+\frac32 a_4'$ of the three inflating directions and
  $-\frac32 a_4'$ of the three deflating extra times cancel, the extra times leave
  $\frac34 b_4'/b_4$, and the six hidden terms $\frac12 H$ add up to $3H$. The
  reason is the divergence form: the density $\sqrt{|g|}\,e_{(x4)}{}^{x4} =
  b_4^{3/2}\cos z$ changes with the time only through $b_4$. For a constant $b_4$,
  as on the backgrounds of this book, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)}$.
- The Dirac operator is $\sum_a f_a^{-1}\gamma^{(a)}\partial_a + 3H\gamma^{(x0)} +
  \frac34(b_4'/b_4)\gamma^{(x4)}$; all 16 component equations equal the records.
  The deflation enters the field equation through the factors $e^{\mp a_4}$ in
  front of the derivatives, not through $\gamma^\mu\Omega_\mu$.
- The term vanishes for a nonzero field only where $|b_4'/b_4| = 4H$ (never for a
  constant $b_4$), because its square is $(9H^2 - \frac{9}{16}(b_4'/b_4)^2)\,1$.
- NEGATIVE CONTROL (at $b_4 = 1$): with inflating extra times the term
  $3a_4'\gamma^{(x4)}$ survives (reproduces the lead's record): the cancellation
  of the $a_4'$ terms is caused by the deflation.
- Scope (stated by the record and shown in Notebook 06b): the value of
  $\gamma^\mu\Omega_\mu$ belongs to the diagonal vielbein; it changes in a boosted
  vielbein and is removed by a rescaling of the field, while the curvature of the
  metric can be removed by nothing.
```

6.14 Line-by-line walk-through of Notebook 06a

The notebook has 24 code cells, In [1] to In [24]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted and then explained. A line that starts with `#` is a **comment**, which Python skips; it is there for the reader. In a quoted figure cell a line `...` stands for the remaining lines of a long caption; every caption is printed in full under its figure in Section 6.13, and a paragraph "What Figure 06a.k shows" after the code says what to look for and why. The words of the notebook are those of its section 3 (Section 6.13), which Sections 6.2 to 6.7 defined.

In [1], the set-up cell. Its first part is the complete run instructions of Section 6.12 once more, as comment lines, so that the notebook file carries its own instructions. The code starts below the line that announces THE SET-UP. The set-up code is the same in every notebook of the book except for one line, the notebook's name; it is explained here once, and the walk-throughs of Notebooks 06b and 06c refer back to this explanation.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json` reads and writes JSON files (text files that store names, lists and numbers), `os` reads settings of the computer, `textwrap` breaks long lines, and `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on Windows, macOS and Linux. The plotting package matplotlib is loaded together with its drawing functions under the short name `plt`, and `Image` and `display` come from IPython, the part of Jupyter that runs Python; together they show a saved picture below a cell.

```
NOTEBOOK_ID = "06a"  # this notebook: chapter 06, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"06a"`. It is the only line of the set-up code that differs from notebook to notebook; the figure files and the last printed line are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
```

```
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called. The text in triple quotes below the `def` line is its **docstring**, a description that Python stores and does not execute. `Path.cwd()` is the folder in which the notebook runs and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with all of them. The `for` loop takes these folders one after the other; `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If no folder holds it, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The function is called and its result named `REPO`. It is never printed: it differs from computer to computer, while the printed output of a notebook must not. `os.environ` holds the **environment variables** of the computer (named texts that a program receives when it starts); `.get(name, defau lt)` returns the value of `TEXTBOOK_OUTPUT_ROOT` when it is set and the repository folder otherwise. When you run the notebook the variable is not set, so files are written into the repository; the book's checking tool sets it to a scratch folder, so that a check never changes the repository.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` is the full path of a repository file, used to READ a Revision record. `output_file("Revision/...")` is the full path at which a file is WRITTEN; `path.par ent.mkdir(parents=True, exist_ok=True)` first creates its folder (and any missing folder above it) and does nothing if the folder exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of the book: `str(text)` turns any value into text, and `textwrap.fill` breaks it at blanks; every line after the first starts with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings of matplotlib, so that the figures are the same on every computer whatever personal settings it has. `plt.rcParams.update({...})` then sets the size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid behind the curves. The braces make a **dictionary**, a collection of pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: each name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/06a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary `{}` and a line end into the captions file; `encoding="utf-8"` fixes how the letters are stored, and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` is called once for every figure. `FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)` returns the number already stored for this figure name, or stores and returns

one more than the number of figures so far; so the figures are numbered 1, 2, 3, and a cell that is run twice keeps its number. The file name joins the notebook id, the number and the name, for example `06a_1_polar_frame_and_spin_rotation.png`. `fig.savefig` writes the picture as a PNG file with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write exactly the same bytes. `plt.close(fig)` removes the figure from memory (Jupyter would otherwise draw it a second time). The caption is stored, and the whole dictionary of captions is written to the captions file (`json.dumps` turns it into JSON text with sorted keys). `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list**, an ordered collection written with square brackets. `check` is the function behind every check of the book. `condition` is `True` or `False`; if it is false, `raise AssertionError(...)` stops the notebook with an error that names the check (Python's own statement `assert` is not used, because Python started with the option `-O` would skip it). If it is true, the name is appended to `PASSED` and the line PASS name is printed; when the optional argument `record` names a Revision record, a second line says which record file and which check the result reproduces.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT. `all_checks_passed` prints the last line of the notebook with the number of checks that passed (`len` is the length of a list). The final statement prints the single output line of In [1].

In [2], the gammas from the record.

```
import sys  # the Python system module (here: the stream of printed text)

# Jupyter sends printed text to the screen in pieces, about every 0.2 seconds.  The
# book's checking tool reads each PASS line together with its "reproduces" line, so
# the pieces must be the same in every run: send the printed text of a cell in one
# piece when the cell ends (or just before a figure), at the latest after 600 s.
if hasattr(sys.stdout, "flush_interval"):  # true inside Jupyter only
    sys.stdout.flush_interval = 600.0
```

`sys` is the Python module that holds, among other things, `sys.stdout`, the channel through which everything printed travels. Inside Jupyter this channel collects printed text and sends it to the screen in pieces, about every 0.2 seconds. The book's checking tool reads each PASS line together with its "reproduces" line, so the pieces must be the same in every run. `hasattr(sys.stdout, "flush_interval")` is true only inside Jupyter (outside it the channel has no such setting), and then the interval is set to 600 seconds: the text of a cell is sent in one piece when the cell ends, or just before a figure is shown. This changes no result.

```
import numpy as np  # arrays of decimal numbers (only for the plots)
import sympy as sp  # exact algebra and calculus with symbols
```

numpy (short name `np`) computes with decimal numbers and is used only for the plots; sympy (short name `sp`) computes exactly with symbols, fractions and functions, and does all the mathematics of the notebook.

```
GAMMA_FILE = "Revision/algebra/gammas.json"  # a Revision record (read only)
fixture = json.loads(repository_file(GAMMA_FILE).read_text(encoding="utf-8"))
NAMES = fixture["coordinates"]  # ["x0", ..., "x7"], the names of the directions
ETA = fixture["eta"]  # the frame metric: +1 space-like, -1 time-like
```

`GAMMA_FILE` names the Revision record of the gammas. `read_text` reads the file as text and `json.loads` turns the text into Python objects: `fixture` is a dictionary whose keys are the names stored in the record. `fixture["coordinates"]` is the list `["x0", ..., "x7"]`, and `fixture["eta"]` the list of the diagonal entries of η.

```
# Each gamma is stored as 16 rows of 16 whole numbers; sp.Matrix makes it an exact
# sympy matrix.  gamma[a] is the gamma of the direction NAMES[a].
gamma = [sp.Matrix(rows) for rows in fixture["gamma"]]
I16 = sp.eye(16)  # the 16 x 16 identity matrix
Z16 = sp.zeros(16, 16)  # the 16 x 16 zero matrix
say(f"directions: {NAMES}")
say(f"eta (frame metric): {ETA}")
```

Each gamma is stored as 16 rows of 16 whole numbers; `sp.Matrix(rows)` turns it into an exact sympy matrix. The **list comprehension** `[... for rows in ...]` builds the list of the eight matrices in one line, so `gamma[0]` is $\gamma^{(x0)}$, `gamma[1]` is $\gamma^{(x1)}$ and `gamma[4]` is $\gamma^{(x4)}$ (Python counts from 0, and the record lists the gammas in the order $x0, \ldots, x7$). `sp.eye(16)` is the identity matrix and `sp.zeros(16, 16)` the zero matrix. The two `say` lines print the directions and the signs of η, the first two output lines.

```
# The Clifford relation for all 8 x 8 = 64 pairs (a, b): the anticommutator is
# 2 eta_aa times the identity when a = b and the zero matrix when a and b differ.
clifford = all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
               == (2 * ETA[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
check(clifford, "the 64 Clifford relations of the author's gammas",
      record="Revision/theory/reports/python-field-theory.json, "
             "check clifford_relations")
```

`all(... for a in range(8) for b in range(8))` is true when the expression is true for every one of the $8 \times 8 = 64$ pairs; `range(8)` is the list $0, 1, \ldots, 7$. For each pair the left side is the anticommutator $\gamma^a\gamma^b + \gamma^b\gamma^a$ (with sympy matrices `*` is the matrix product) and the right side is $2\eta_{aa}$ times the identity when $a = b$ and the zero matrix otherwise (`x if condition else y` is `x` when the condition holds and `y` otherwise; `0 * I16` is the zero matrix). `==` compares two sympy matrices entry by entry, exactly. The check prints PASS and the record it reproduces.

```
# eight matrices, each with 16 rows and 16 columns, and every entry a real number
check(len(gamma) == 8 and all(matrix.shape == (16, 16) for matrix in gamma)
      and all(entry.is_real for matrix in gamma for entry in matrix),
      "the eight gammas are 16 x 16 matrices with real entries",
      record="Revision/theory/reports/python-field-theory.json, check gammas_real")
```

The second check requires eight matrices, each of shape 16 by 16, and every entry real (`entry.is_real` is true for a whole number). `and` is true only when all its parts are true.

```
# S[a][b] = (1/4)(gamma^a gamma^b - gamma^b gamma^a), for all 64 pairs (a, b).
S = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
     for a in range(8)]
```

`S[a][b]` is the generator $S^{ab} = \frac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$ for all 64 ordered pairs, built with a nested list comprehension: for each `a` a list over `b`. The cell prints the two lines of the directions and of η, and two PASS lines with their records.

In [3], the helpers and the records.

```
from sympy.parsing.mathematica import parse_mathematica  # reads Wolfram notation
```

`parse_mathematica` is sympy's reader of formulas written in the notation of the Wolfram Language, the notation in which the records `Revision/theory/field-theory.json` and `Revision/theory/field-theory-b4.json` store their formulas.

```
def is_zero(expr):
    """True when expr is exactly zero for all values of its symbols."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    # Fast route: sin, cos, tan, ... written through exp, multiplied out, cancelled.
    if sp.cancel(sp.expand(expr.rewrite(sp.exp))) == 0:
        return True
    return sp.simplify(expr) == 0  # the slower general route
```

`is_zero(expr)` decides exactly whether an expression is zero for all values of its symbols. `sp.sympify` turns a plain number into a sympy number. If the expression is already the number 0 the answer is yes. Otherwise the fast route writes every sine, cosine and tangent through exponential functions (`rewrite(sp.exp)`; for example $\cos z = (e^{iz} + e^{-iz})/2$), multiplies everything out (`expand`) and cancels common factors of numerator and denominator (`cancel`); if that gives 0, the expression is zero. If not, the slower general `sp.simplify` decides. Both routes are exact transformations, so the function never calls a nonzero expression zero; at worst it fails to recognise a zero, and then a check fails visibly.

```
def matrix_is_zero(matrix):
    """True when every entry of the matrix is exactly zero."""
    return all(is_zero(entry) for entry in matrix)
```

`matrix_is_zero` applies `is_zero` to every entry of a matrix (a `for` over a sympy matrix runs through its entries).

```
def record_passed(path, name):
    """True when the Revision report at path records the check name as passed."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper() == "PASS"  # "PASS" or "pass"
    raise KeyError(f"{path} has no check {name}")
```

`record_passed(path, name)` reads a Revision report, a JSON file with a list `checks` whose entries have the keys `name`, `verdict` and `detail`, and returns `True` when the check `name` is recorded with the verdict PASS (`.upper()` writes the verdict in capitals, because one report writes PASS and another pass). If the report has no such check, `raise KeyError` stops the notebook: a misspelled check name can never pass silently.

```
def read_formulas(path):
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    return {item["key"]: item["wl"] for item in data["formulas"]}
```

`read_formulas(path)` reads a record that holds a list `formulas`, each with a key and the formula as Wolfram text (`wl`). The **dictionary comprehension** `{item["key"]: item["wl"] for item in ...}` builds the dictionary from key to text in one line.

```
THEORY_FILE = "Revision/theory/field-theory.json"  # the record at b4 = 1
THEORY_B4_FILE = "Revision/theory/field-theory-b4.json"  # the record with b4(x4)
FORMULAS = read_formulas(THEORY_FILE)
FORMULAS_B4 = read_formulas(THEORY_B4_FILE)
```

```
def wolfram_to_sympy(text, names):
    """A text in Wolfram notation as a sympy expression.  names maps the record's
    symbol names (texts) to the symbols and functions of this notebook."""
    # The record writes a4', a4'', a4(x4) and the same for b4 as Derivative[1][a4][x4],
```

```
    # Derivative[2][a4][x4], a4[x4], ...; give them short names before reading.
    for long_form, short_form in (("Derivative[1][a4][x4]", "a4p"),
                                  ("Derivative[2][a4][x4]", "a4pp"),
                                  ("Derivative[1][b4][x4]", "b4p"),
                                  ("Derivative[2][b4][x4]", "b4pp"),
                                  ("a4[x4]", "a4"), ("b4[x4]", "b4")):
        text = text.replace(long_form, short_form)
    expr = parse_mathematica(text)
    return expr.subs({sp.Symbol(name): value for name, value in names.items()},
                     simultaneous=True)
```

`FORMULAS` holds the formulas of the record of the metric at $b_4 = 1$ and `FORMULAS_B4` those of the record of the metric with $b_4(x_4)$. `wolfram_to_sympy(text, names)` turns a text in Wolfram notation into a sympy expression. The records write a_4', a_4'' and $a_4(x_4)$ as `Derivative[1][a4][x4]`, `Derivative[2][a4][x4]` and `a4[x4]`, and the same for b_4; the loop replaces them by the short names `a4p`, `a4pp`, `b4p`, `b4pp`, `a4` and `b4` (`text.replace(old, new)` replaces every occurrence; the derivatives come first, because they contain the text `a4[x4]`). `parse_mathematica` reads the text, and `.subs({...}, simultaneous=True)` replaces each plain symbol of the record by the symbol or function of the notebook that the dictionary `names` gives for it, all at the same time. `record_formula(key, names, formulas)` does this for one formula of a record; without the third argument it reads the record of $b_4 = 1$ (a **default value** of the argument).

```
REPORT_PY = "Revision/theory/reports/python-field-theory.json"
REPORT_WL = "Revision/theory/reports/wolfram-field-theory.json"
REPORT_PY_B4 = "Revision/theory/reports/python-field-theory-b4.json"
REPORT_WL_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
REPORT_LEAD = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
METRIC_B4_FILE = "Revision/metric_b4/results/metric-b4.json"
say(f"{len(FORMULAS)} formulas read from {THEORY_FILE}")
say(f"{len(FORMULAS_B4)} formulas read from {THEORY_B4_FILE}")
```

Names for the five Revision reports that the notebook reproduces and for the record `metric-b4.json`, and two output lines: 32 formulas read from Revision/theory/field-theory.json and 8 formulas read from Revision/theory/field-theory-b4.json.

In [4], four general formulas as Python functions. The four functions work for any number of dimensions and any vielbein, so the same code serves the plane and the author's eight-dimensional metric.

```
def christoffel(g, coords):
    """Gam[l][m][n] = (1/2) sum_r g^lr (d_m g_rn + d_n g_rm - d_r g_mn)."""
    n = len(coords)
    g_inv = g.inv()  # the inverse metric g^lr
    Gam = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for l in range(n):
        for m in range(n):
            for k in range(m, n):  # Gamma^l_mk = Gamma^l_km: compute it once
                value = sum(g_inv[l, r] * (sp.diff(g[r, k], coords[m])
                                           + sp.diff(g[r, m], coords[k])
                                           - sp.diff(g[m, k], coords[r]))
```

```
                        for r in range(n) if g_inv[l, r] != 0) / 2
                value = sp.simplify(value)
                Gam[l][m][k] = value
                Gam[l][k][m] = value
    return Gam
```

`christoffel(g, coords)` computes $\Gamma^l{}_{mn}$ by the formula of Section 6.3. `g.inv()` is the inverse matrix g^{lr}. `Gam` starts as a three-level list of zeros (`[sp.Integer(0)] * n` is a list of `n` exact zeros). The loops run over all `l` and `m` and over `k` from `m` upwards only: since $\Gamma^l{}_{mk} = \Gamma^l{}_{km}$, each symmetric pair is computed once and stored in both places. `sp.diff(g[r, k], coords[m])` is the partial derivative $\partial_m g_{rk}$; the generator expression sums over `r`, skipping the entries where the inverse metric is zero (for a diagonal metric only `r = l` remains), and `/ 2` is the factor $\frac{1}{2}$. `sp.simplify` brings each symbol into a short form.

```
def spin_connection(e, Gam, coords, eta):
    """omega[mu][a][b] = omega_{mu ab} = eta_aa omega_mu^a_b, where
    omega_mu^a_b = sum_nu e^a_nu (d_mu e_b^nu + sum_lam Gam^nu_{mu lam} e_b^lam)."""
    n = len(coords)
    E = e.inv()  # E[nu, b] = e_b^nu, the inverse vielbein
    omega = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for mu in range(n):
        for a in range(n):
            for b in range(n):
                mixed = 0  # will become omega_mu^a_b
                for nu in range(n):
                    if e[a, nu] == 0:
                        continue  # this term is zero
                    inner = sp.diff(E[nu, b], coords[mu]) + sum(
                        Gam[nu][mu][lam] * E[lam, b] for lam in range(n))
                    mixed += e[a, nu] * inner
                omega[mu][a][b] = sp.simplify(eta[a] * mixed)  # lower the index a
    return omega
```

`spin_connection(e, Gam, coords, eta)` computes the canonical spin connection by the formula of Section 6.4 and lowers its first index. The vielbein is the matrix `e` with `e[a, mu]` $= e^a{}_\mu$ (row = frame index, column = coordinate index). Its inverse matrix `E = e.inv()` has `E[mu, a]` $= e_a{}^\mu$, because $\sum_\mu e^a{}_\mu e_b{}^\mu = \delta^a_b$ says exactly `e * E` $= 1$. For each μ, a, b the variable `mixed` collects $\sum_\nu e^a{}_\nu \big(\partial_\mu e_b{}^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda} e_b{}^\lambda\big)$, the mixed component $\omega_\mu{}^a{}_b$; `continue` skips the values of ν where $e^a{}_\nu = 0$, which contribute nothing. The last line multiplies by η_{aa} (lowering the index a) and simplifies, so the function returns `omega[mu][a][b]` $= \omega_{\mu ab}$.

```
def spinor_connection(omega, S):
    """Omega[mu] = (1/2) sum_{a,b} omega_{mu ab} S^ab = sum_{a<b} omega_{mu ab} S^ab."""
    n = len(omega)
    size = S[0][0].shape[0]  # 16 for the author's gammas, 2 in two dimensions
    Omega = []
    for mu in range(n):
        total = sp.zeros(size, size)
        for a in range(n):
            for b in range(a + 1, n):
```

```
                if omega[mu][a][b] != 0:
                    total += omega[mu][a][b] * S[a][b]
        Omega.append(total)
    return Omega
```

`spinor_connection(omega, S)` builds $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$ (Section 6.7 showed that the sum over $a < b$ uses up the factor $\frac{1}{2}$). `S[0][0].shape[0]` is the size of the matrices, 16 for the author's gammas and 2 in the plane. For each μ a zero matrix is filled with the terms whose coefficient is not zero, and the eight matrices are collected in the list `Omega`.

```
def curved_gammas(e, gammas):
    """gamma^mu = sum_a e_a^mu gamma^a for every coordinate mu."""
    E = e.inv()
    n = len(gammas)
    return [sum((E[mu, a] * gammas[a] for a in range(n)),
                sp.zeros(*gammas[0].shape)) for mu in range(n)]
```

`curved_gammas(e, gammas)` returns the list of the curved gammas $\gamma^\mu = \sum_a e_a{}^\mu \gamma^a$; `sum(..., sp.zeros(...))` adds the matrices starting from a zero matrix of the right shape (`*gammas[0].shape` passes the two numbers of the shape as two arguments).

```
say("defined: christoffel, spin_connection, spinor_connection, curved_gammas")
```

One output line names the four functions.

In [5], the warm-up in the polar plane (Section 6.5).

```
r, phi = sp.symbols("r phi", positive=True)  # polar coordinates, r > 0
sigma1 = sp.Matrix([[0, 1], [1, 0]])  # the three Pauli matrices
sigma2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
sigma3 = sp.Matrix([[1, 0], [0, -1]])
plane_gammas = [sigma1, sigma2]  # gamma^0 (radial), gamma^1 (angular)
plane_eta = [1, 1]
plane_S = [[(plane_gammas[a] * plane_gammas[b] - plane_gammas[b] * plane_gammas[a])
            / 4 for b in range(2)] for a in range(2)]
plane_g = sp.diag(1, r**2)  # ds^2 = dr^2 + r^2 dphi^2
plane_e = sp.diag(1, r)  # e^a_mu: unit radial and unit angular direction
```

`sp.symbols("r phi", positive=True)` makes two symbols that sympy treats as positive numbers, the polar coordinates r and φ. The three Pauli matrices are typed entry by entry (`sp.I` is the imaginary unit i). The plane has the two gammas σ_1 (radial) and σ_2 (angular), the frame metric $\eta = (1, 1)$, the generators `plane_S` built exactly as `S` in In [2], the metric $\mathrm{diag}(1, r^2)$ and the vielbein $\mathrm{diag}(1, r)$.

```
plane_Gam = christoffel(plane_g, [r, phi])
plane_omega = spin_connection(plane_e, plane_Gam, [r, phi], plane_eta)
plane_Omega = spinor_connection(plane_omega, plane_S)
plane_gup = curved_gammas(plane_e, plane_gammas)
plane_slash = sp.simplify(plane_gup[0] * plane_Omega[0] + plane_gup[1] * plane_Omega[1])
```

The four general functions are applied to the plane. `plane_slash` is $\gamma^r\Omega_r + \gamma^\varphi\Omega_\varphi$, the contraction, simplified.

```
say(f"Gamma^r_phiphi = {plane_Gam[0][1][1]},  Gamma^phi_rphi = {plane_Gam[1][0][1]}")
say(f"omega_phi01 = {plane_omega[1][0][1]},  omega_phi10 = {plane_omega[1][1][0]}")
say(f"Omega_phi = {plane_Omega[1].tolist()}")
say(f"gamma^mu Omega_mu = {plane_slash.tolist()}")
```

Four output lines print the results: ${\Gamma^r}_{\varphi\varphi} = -r$, ${\Gamma^\varphi}_{r\varphi} = 1/r$, $\omega_{\varphi 01} = -1$, $\omega_{\varphi 10} = 1$, Ω_φ as the nested list `[[-I/2, 0], [0, I/2]]` (`.tolist()` turns a matrix into a list of rows) and the contraction `[[0, 1/(2*r)], [1/(2*r), 0]]`, which is $\sigma_1/(2r)$. Index `[1]` is φ and `[0]` is r.

```
check(plane_Gam[0][1][1] == -r and plane_Gam[1][0][1] == 1 / r,
      "polar plane: Gamma^r_phiphi = -r and Gamma^phi_rphi = 1/r")
check(plane_omega[1][0][1] == -1 and plane_omega[1][1][0] == 1
      and plane_omega[0][0][1] == 0, "polar plane: omega_phi01 = -1, omega_r = 0")
check(plane_Omega[1] == -sp.I / 2 * sigma3 and plane_Omega[0] == sp.zeros(2, 2),
      "polar plane: Omega_phi = -(i/2) sigma3 and Omega_r = 0")
check(plane_slash == sigma1 / (2 * r), "polar plane: gamma^mu Omega_mu = sigma1/(2 r)")
```

Four checks compare the results with the hand computation of Section 6.5: the two Christoffel symbols; the two components of ω_φ and the vanishing of ω_r; $\Omega_\varphi = -\frac{i}{2}\sigma_3$ and $\Omega_r = 0$; and $\sum_\mu \gamma^\mu\Omega_\mu = \sigma_1/(2r)$.

```
F_rphi = (plane_Omega[1].diff(r) - plane_Omega[0].diff(phi)
          + plane_Omega[0] * plane_Omega[1] - plane_Omega[1] * plane_Omega[0])
U = sp.cos(phi / 2) * sp.eye(2) + sp.I * sp.sin(phi / 2) * sigma3  # spinor rotation
pure_gauge = -U.diff(phi) * U.inv() - plane_Omega[1]  # must be the zero matrix
removed = U.inv() * plane_Omega[1] * U - U.inv().diff(phi) * U  # frame change U^-1
check(F_rphi == sp.zeros(2, 2) and matrix_is_zero(pure_gauge)
      and matrix_is_zero(removed),
      "polar plane: flat (F_rphi = 0), Omega_phi = -(dU/dphi) U^-1, U^-1 removes it")
```

`F_rphi` is the curvature $\partial_r\Omega_\varphi - \partial_\varphi\Omega_r + [\Omega_r, \Omega_\varphi]$ (`.diff(r)` differentiates every entry). `U` is the spinor rotation $\cos\frac{\varphi}{2} + i\sin\frac{\varphi}{2}\,\sigma_3$, and `pure_gauge` is $-(\partial_\varphi U)U^{-1} - \Omega_\varphi$ (`U.inv()` is the inverse matrix): it vanishes when Ω_φ is what the rule $R\,\Omega R^{-1} - (\partial_\varphi R)R^{-1}$ of a change of frame makes from the connection 0 of the Cartesian frame with $R = U$. `removed` is that rule applied to Ω_φ with $R = U^{-1}$, whose inverse is U: $U^{-1}\Omega_\varphi U - (\partial_\varphi U^{-1})U$, the connection of the frame obtained by turning the polar frame back with U^{-1} (Section 6.5 worked it by hand). The fifth check requires all three to be zero: the plane is flat, its spin connection is only the turning of the frame, and the inverse rotation U^{-1} removes it. The cell prints five PASS lines.

In [6], figure 1.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.6))
angles = np.arange(12) * np.pi / 6  # 12 angles, 30 degrees apart
for radius in (1.0, 2.0):
    px, py = radius * np.cos(angles), radius * np.sin(angles)  # the points
    ax_left.quiver(px, py, np.cos(angles), np.sin(angles), color="black",
```

```
                angles="xy", scale_units="xy", scale=3.0, width=0.006)
    ax_left.quiver(px, py, -np.sin(angles), np.cos(angles), color="tab:orange",
                angles="xy", scale_units="xy", scale=3.0, width=0.006)
    circle = np.linspace(0.0, 2.0 * np.pi, 200)
    ax_left.plot(radius * np.cos(circle), radius * np.sin(circle), ":",
                 color="gray", linewidth=1.0)
```

`plt.subplots(1, 2, figsize=(10.0, 4.6))` makes one figure with two panels side by side, called `ax_left` and `ax_right`. `np.arange(12) * np.pi / 6` is the list of the twelve angles $0, \pi/6, \ldots, 11\pi/6$ (30 degrees apart). For the radii 1 and 2 the points $(r\cos\varphi, r\sin\varphi)$ are computed, and `quiver` draws an arrow at each point: the unit radial direction $(\cos\varphi, \sin\varphi)$ in black and the unit angular direction $(-\sin\varphi, \cos\varphi)$ in orange. The options `angles="xy", scale_units="xy", scale=3.0` draw an arrow of length one third in the units of the axes. The dotted circle is drawn through 200 points.

```
ax_left.plot([], [], color="black", label="unit radial direction $e_{(0)}$")
ax_left.plot([], [], color="tab:orange", label="unit angular direction $e_{(1)}$")
ax_left.set_aspect("equal")
ax_left.set_xlim(-2.8, 2.8)
ax_left.set_ylim(-2.8, 3.6)  # room for the legend above the circles
ax_left.set_xlabel("Cartesian $x = r\\cos\\varphi$")
ax_left.set_ylabel("Cartesian $y = r\\sin\\varphi$")
ax_left.set_title("The polar frame turns with the angle")
ax_left.legend(loc="upper left", fontsize=8)
```

Two empty plots (`[]`, `[]`) only create the legend entries. `set_aspect("equal")` makes one unit equally long on both axes, so the circles look round. The axis limits leave room for the legend above the circles; the axes are labelled and the legend is placed in the upper left corner.

```
turn = np.linspace(0.0, 4.0 * np.pi, 400)  # phi from 0 to 4 pi
ax_right.plot(turn / np.pi, np.cos(turn / 2), label="$\\cos(\\varphi/2)$, real part")
ax_right.plot(turn / np.pi, np.sin(turn / 2), "--",
              label="$\\sin(\\varphi/2)$, imaginary part")
ax_right.set_xlabel("$\\varphi / \\pi$")
ax_right.set_ylabel("entry $U_{11}$ of the spinor rotation")
ax_right.set_ylim(-1.45, 1.2)  # room for the legend below the curves
ax_right.set_title("A spinor turns by half the angle")
ax_right.legend(loc="lower center", ncol=2, fontsize=8)
```

The right panel draws the entry $U_{11} = \cos\frac{\varphi}{2} + i\sin\frac{\varphi}{2}$ of the spinor rotation U (the rotation that carries the Cartesian frame into the polar frame; its inverse U^{-1} removes the connection, Section 6.5) for 400 values of φ from 0 to 4π: its real part $\cos(\varphi/2)$ as a solid line and its imaginary part $\sin(\varphi/2)$ dashed (the style string of two minus signs). The horizontal axis shows φ/π, so the full turn is at 2 and the double turn at 4. In the labels `\\` is a single backslash for matplotlib's formula typesetting.

```
save_figure(fig, "polar_frame_and_spin_rotation",
            "Warm-up in the flat plane. Left: the unit radial direction (black) and "
            ...
            "rotation is $-1$, and only at $4\\pi$ is it $+1$ again.")
```

`save_figure` saves the figure with its caption, shows it and prints one line with the name of the saved file, `06a_1_polar_frame_and_spin_rotation.png` in the folder `Revision/textbook/figures`.

What Figure 06a.1 shows. On the left, the black and orange arrows turn together as one goes around the circle: the polar frame at the angle φ is the Cartesian frame turned by φ. That turning is exactly what $\omega_{\varphi 01} = -1$ records, although the plane is flat; the spinor rotation $U(\varphi)$ performs the same turning on spinors, and its inverse U^{-1} turns it back and removes the connection. On the right, the solid curve $\cos(\varphi/2)$ reaches -1 at $\varphi = 2\pi$ (the value 2 on the axis), where the dashed curve is 0: after one full turn the spinor rotation is -1, and only at 4π is it $+1$ again. A spinor turns by half the angle of its frame.

In [7], the author's metric and its vielbein (Section 6.2).

```
x = sp.symbols("x0:8", real=True)  # x[0] = x0, ..., x[4] = x4, ..., x[7] = x7
x0, x4 = x[0], x[4]  # the hidden direction and the time
H = sp.Symbol("H", positive=True)  # the author's constant H > 0
a4 = sp.Function("a4")(x4)  # the metric function a4(x4): any function of the time
b4 = sp.Function("b4", positive=True)(x4)  # the metric function b4(x4) > 0
z = 6 * H * x0  # the hidden angle z = 6 H x0, between 0 and pi/2
s = sp.sin(z) ** sp.Rational(1, 6)  # sin^(1/6) z
f = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
     + [sp.exp(-a4) * sp.sqrt(b4) * s] * 3)  # f_0, ..., f_7
g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # g_aa = eta_aa f_a^2
e = sp.diag(*f)  # the diagonal vielbein e^a_mu = f_a delta^a_mu
```

`sp.symbols("x0:8", real=True)` makes the eight real symbols $x_0, \ldots, x_7$ (the range `0:8` stops before 8); `x[0]` is x_0 and `x[4]` is x_4. `H` is positive. `sp.Function("a4")(x4)` is an unknown function of x_4, and `sp.Function("b4", positive=True)(x4)` an unknown positive function of x_4: every result below holds for all functions a_4 and $b_4 > 0$. `z` is $6Hx_0$, `s` is $\sin^{1/6} z$ (`sp.Rational(1, 6)` is the exact fraction $\frac{1}{6}$). The list `f` holds the eight factors in the order $x0, \ldots, x7$: `[...] * 3` repeats a one-element list three times, `+` joins lists, and `sp.sqrt(b4)` is $b_4^{1/2}$. `g` is the diagonal matrix with the entries $\eta_{aa} f_a^2$ (`sp.diag(*list)` puts the list on the diagonal), and `e` the diagonal vielbein.

```
# The names under which the records write symbols, and our symbols for them:
RECORD_NAMES = {"H": H, "x4": x4, "x0": x0, "a4": a4, "b4": b4,
                "a4p": sp.Derivative(a4, x4), "a4pp": sp.Derivative(a4, (x4, 2)),
                "b4p": sp.Derivative(b4, x4), "b4pp": sp.Derivative(b4, (x4, 2))}
AT_B4_1 = {b4: 1}  # the background b4 = 1


def at_b4_1(expr):
    return sp.sympify(expr).subs(AT_B4_1).doit()


metric_b4 = json.loads(repository_file(METRIC_B4_FILE).read_text(encoding="utf-8"))
diagonal_b4 = [wolfram_to_sympy(text, RECORD_NAMES)
               for text in metric_b4["metricDiagonal"]]
check(all(is_zero(g[a, a] - diagonal_b4[a]) for a in range(8))
      and record_passed(REPORT_WL_B4, "metric_is_the_authors_b4"),
      "eta_aa f_a^2 is the author's metric with b4 (all eight diagonal entries)",
      record=f"{METRIC_B4_FILE}, key metricDiagonal")
```

```
metric_record = sp.Matrix([list(row) for row in record_formula("metric", RECORD_NAMES)])
f_record = record_formula("vielbein_diagonal", RECORD_NAMES)
```

The records write their symbols as plain names; `RECORD_NAMES` says which symbol or function of the notebook each name means (`sp.Derivative(a4, x4)` is a_4', `sp.Derivative(a4, (x4, 2))` is a_4'', and the same for b_4). `at_b4_1(expr)` puts $b_4 = 1$ into an expression; sympy leaves a derivative of the constant 1 written as `Derivative(1, x4)`, and `.doit()` evaluates it to 0. The record `metric-b4.json` lists the eight diagonal entries of the metric with b_4 under the key `metricDiagonal`; they are read and compared with g, and the WolframScript report with b_4 must record `metric_is_the_authors_b4` as passed. The metric of the record of $b_4 = 1$ is read as a nested list and turned into a matrix (`list(row)` makes each row a list), and that record's eight vielbein factors are read as a list.

```
check(matrix_is_zero(at_b4_1(g) - metric_record)
      and all(is_zero(at_b4_1(f[a]) - f_record[a]) for a in range(8))
      and record_passed(REPORT_LEAD, "vielbein_reproduces_metric"),
      "at b4 = 1: the metric and the eight vielbein factors of the record",
      record=f"{THEORY_FILE}, formulas metric and vielbein_diagonal")
```

The second check requires, at $b_4 = 1$, all 64 entries of g minus the record's metric to vanish, the eight factors to equal the record's, and the lead's independent report to record `vielbein_reproduces_metric` as passed.

```
volume = sp.simplify(sp.prod(f))  # f_0 f_1 ... f_7
say(f"f_0 f_1 ... f_7 = {volume}")
volume_record = wolfram_to_sympy(metric_b4["sqrtAbsDetG"], RECORD_NAMES)
check(is_zero(volume - b4 ** sp.Rational(3, 2) * sp.cos(z))
      and is_zero(volume - volume_record) and is_zero(volume**2 - g.det())
      and record_passed(REPORT_WL_B4, "sqrt_det_g"),
      "sqrt|det g| = f_0 f_1 ... f_7 = b4^(3/2) cos z (cos z at b4 = 1)",
      record=f"{METRIC_B4_FILE}, key sqrtAbsDetG")
```

`sp.prod(f)` multiplies the eight factors, and the simplified product is printed: `f_0 f_1 ... f_7 = b4(x4)**(3/2)*cos(6*H*x0)`. The check requires it to be $b_4^{3/2}\cos z$, to equal the record's `sqrtAbsDetG`, and its square to be the determinant of g (`g.det()`), so that it is $\sqrt{|\det g|}$. The cell prints three PASS lines with their records and the product.

In [8], figure 2.

```
X4_SYMBOL = sp.Symbol("t", real=True)  # the time x4 as a plain symbol for numbers
history = {a4: X4_SYMBOL, b4: 1}  # illustrative: a4(x4) = x4, b4 = 1 (with H = 1)
f_of_time = sp.lambdify(X4_SYMBOL, [fa.subs(history).subs({H: 1, x0: sp.pi / 24})
                                    for fa in f])  # z = 6 x0 = pi/4
times = np.linspace(-2.0, 2.0, 201)
values = [np.broadcast_to(v, times.shape) for v in f_of_time(times)]
```

For numbers the unknown functions must be given a form. `history` replaces $a_4(x_4)$ by the plain symbol `t` and b_4 by 1: the illustrative history $a_4 = x_4$, $b_4 = 1$ (with $H = 1$, the prescribed background of Chapter 3 with slope 1). `sp.lambdify(t, [...])` turns the eight factors, with

$H = 1$ and $x_0 = \pi/24$ (so $z = 6x_0 = \pi/4$), into one numerical function of `t` that returns the eight values. `times` is a list of 201 times from -2 to 2. The factor $f_4 = 1$ does not depend on `t`, so `lambdify` returns the single number 1 for it; `np.broadcast_to(v, times.shape)` turns such a number into a list of 201 equal values, so that every curve can be plotted.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.4))
ax_left.semilogy(times, values[1], label="$f_1 = f_2 = f_3 = e^{a_4}s$ (3-space)")
ax_left.semilogy(times, values[5], "--",
                 label="$f_5 = f_6 = f_7 = e^{-a_4}b_4^{1/2}s$ (extra times)")
ax_left.semilogy(times, values[4], ":", color="black", label="$f_4 = 1$ (time)")
ax_left.semilogy(times, values[1] * values[5], "-.", color="gray",
                 label="product $f_1 f_5 = b_4^{1/2}s^2$")
ax_left.set_xlabel("time $x_4$ (units $1/H$), history $a_4 = x_4$, $b_4 = 1$")
ax_left.set_ylabel("vielbein factor (logarithmic axis)")
ax_left.set_title("Inflating 3-space, deflating extra times")
ax_left.legend(fontsize=8)
```

`semilogy` plots with a logarithmic vertical axis, on which an exponential function is a straight line. Four curves (positions 1, 5 and 4 of the list): $f_1 = e^{a_4}s$ (solid), $f_5 = e^{-a_4}b_4^{1/2}s$ (dashed), $f_4 = 1$ (dotted, black) and the product $f_1 f_5 = b_4^{1/2}s^2$ (dash-dotted, gray).

```
zz = np.linspace(0.02, np.pi / 2 - 0.02, 300)  # z inside (0, pi/2)
ax_right.plot(zz, np.sin(zz) ** (1 / 6), label="$s = \\sin^{1/6} z$")
ax_right.plot(zz, 1 / np.tan(zz), "--", label="$f_0 = \\cot z$ (hidden)")
ax_right.plot(zz, np.cos(zz), ":", color="black",
              label="$\\sqrt{|\\det g|} = \\cos z$ ($b_4 = 1$)")
ax_right.set_ylim(0.0, 3.0)
ax_right.set_xlabel("hidden angle $z = 6 H x_0$ (radians)")
ax_right.set_ylabel("factor (pure number)")
ax_right.set_title("Dependence on the hidden direction")
ax_right.legend(fontsize=8)
```

The right panel uses 300 values of z inside $(0, \pi/2)$ (0.02 away from both ends, where $\cot z$ would be infinite) and draws $s = \sin^{1/6} z$, $f_0 = \cot z$ $(1/\tan z)$ and the volume factor $\cos z$ of $b_4 = 1$. `set_ylim(0.0, 3.0)` cuts the axis at 3, because $\cot z$ grows without bound near $z = 0$.

```
save_figure(fig, "vielbein_factors",
            "The eight factors $f_a$ of the diagonal vielbein. Left: versus the time "
            ...
            "$a_4 = x_4$, $b_4 = 1$, on a logarithmic axis; the 3-space factor "
            "$e^{a_4}s$ grows and the extra-time factor $e^{-a_4}b_4^{1/2}s$ shrinks "
            "at the same exponential rate, so their product $b_4^{1/2}s^2$ stays "
            "constant, and the time factor is $1$. Right: versus the hidden angle "
            "$z = 6Hx_0$ in radians at $x_4 = 0$; $s = \\sin^{1/6}z$, the hidden "
            "factor $\\cot z$, which falls from very large values to $0$, and the "
            "volume factor $\\sqrt{|\\det g|} = b_4^{3/2}\\cos z$ at $b_4 = 1$, which "
            "depends on the time only through $b_4$.")
```

The figure is saved with its caption; the cell prints the name of the saved file, `06a_2_vielbein_factors.png`.

What Figure 06a.2 shows. On the left, the 3-space factor rises and the extra-time factor falls along straight lines of opposite slope (on a logarithmic axis an exponential is a straight line): 3-space inflates while the extra times deflate exponentially, at the same rate. Their product is the flat gray line s^2 (here $b_4 = 1$), and the time factor is 1. On the right, as functions of the hidden angle: s rises slowly to 1, the hidden factor $\cot z$ falls from very large values near the tip $z = 0$ to 0 at the patch end $z = \pi/2$, and the volume factor $\cos z$ falls from 1 to 0. For $b_4 = 1$ the volume factor does not depend on the time at all, and in general only through $b_4^{3/2}$; Section 6.9 showed why this matters.

In [9], the Christoffel symbols (Section 6.3).

```
Gam = christoffel(g, x)
independent = {(l, m, n): Gam[l][m][n] for l in range(8) for m in range(8)
               for n in range(m, 8) if Gam[l][m][n] != 0}
report("number of independent nonzero Christoffel symbols", len(independent))
```

`christoffel(g, x)` computes all $8 \cdot 8 \cdot 8 = 512$ symbols. The dictionary `independent` keeps those with $m \le n$ (`range(m, 8)`) that are not zero, under the key `(l, m, n)`. `report` prints their number, 25.

```
def as_dictionary(entries):
    return {(int(l) - 1, int(m) - 1, int(n) - 1): value for l, m, n, value in entries}


record_b4 = as_dictionary(record_formula("christoffel_nonzero", RECORD_NAMES,
                                         FORMULAS_B4))
record_b4_1 = as_dictionary(record_formula("christoffel_nonzero", RECORD_NAMES))
same_b4 = set(record_b4) == set(independent) and all(
    is_zero(independent[key] - record_b4[key]) for key in independent)
same_b4_1 = set(record_b4_1) == set(independent) and all(
    is_zero(at_b4_1(independent[key]) - record_b4_1[key]) for key in independent)
```

The records' lists `christoffel_nonzero` have entries $\{l, m, n, \text{value}\}$ with the directions numbered from 1 in the order $x0, \ldots, x7$ (the number 1 is $x0$); `as_dictionary` subtracts 1 to get Python's positions. The list of the record with b_4 is compared with the symbols as they are, the list of the record of $b_4 = 1$ with the symbols at $b_4 = 1$. `set(record_b4) == set(independent)` requires the same 25 keys on both sides (a **set** is a collection without order), and `all(...)` requires every value to agree exactly.

```
for key in [(1, 1, 4), (5, 4, 5), (1, 0, 1), (4, 5, 5), (0, 0, 0)]:
    l, m, n = key
    say(f"Gamma^{NAMES[l]}_{NAMES[m]}{NAMES[n]} = {independent[key]}")
```

Five symbols are printed as examples: ${\Gamma^{x1}}_{x1x4} = a_4'$, ${\Gamma^{x5}}_{x4x5} = -a_4' + b_4'/(2b_4)$, ${\Gamma^{x1}}_{x0x1} = H/\tan z$, ${\Gamma^{x4}}_{x5x5} = e^{-2a_4} b_4 \sin^{1/3} z\,(a_4' - b_4'/(2b_4))$ (sympy prints it as `(b4(x4)*Derivative(a4(x4), x4) - Derivative(b4(x4), x4)/2)*exp(-2*a4(x4))*sin(6*H*x0)**(1/3)`) and ${\Gamma^{x0}}_{x0x0} = -12H/\sin 12Hx_0 = -12H/\sin 2z$, as derived in Section 6.3 (sympy prints the power $\frac{1}{3}$ as `**(1/3)` and a_4' as `Derivative(a4(x4), x4)`).

```
check(len(independent) == 25 and same_b4 and same_b4_1,
      "the 25 independent nonzero Christoffel symbols equal the records (with b4, and "
      "at b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula christoffel_nonzero")
```

The check requires 25 symbols equal to the records' (with b_4, and at $b_4 = 1$); it prints PASS and the records.

In [10], figure 3.

```
SAMPLE = {H: 1, x0: sp.pi / 24}  # H = 1 and z = 6 H x0 = pi/4


def at_sample(expr):
    """The decimal value of expr at the sample point H = 1, z = pi/4, a4 = 1/2,
    a4' = 1/2, b4 = 1, b4' = 1/2, a4'' = b4'' = 0 (derivatives are replaced first,
    then the functions themselves)."""
    expr = sp.sympify(expr)
    for function, first in ((a4, sp.Rational(1, 2)), (b4, sp.Rational(1, 2))):
        expr = expr.subs(sp.Derivative(function, (x4, 2)), 0)
        expr = expr.subs(sp.Derivative(function, x4), first)
    expr = expr.subs({a4: sp.Rational(1, 2), b4: 1})
    return complex(sp.N(expr.subs(SAMPLE)))
```

`SAMPLE` fixes $H = 1$ and $x_0 = \pi/24$, so $z = \pi/4$. `at_sample(expr)` gives the decimal value of an expression at the **sample point** $H = 1$, $z = \pi/4$, $a_4 = \frac{1}{2}$, $a_4' = \frac{1}{2}$, $b_4 = 1$, $b_4' = \frac{1}{2}$, $a_4'' = b_4'' = 0$. The order of the replacements matters: for each function the derivatives are replaced first, because replacing $a_4(x_4)$ by the number $\frac{1}{2}$ first would turn a_4' into the derivative of a constant, which is 0. `sp.N` evaluates the expression as a decimal number, and `complex(...)` makes it an ordinary Python number.

```
fig, axes = plt.subplots(2, 4, figsize=(10.0, 5.6))
for l, ax in enumerate(axes.flat):
    signs = np.array([[np.sign(at_sample(Gam[l][m][n]).real) for n in range(8)]
                      for m in range(8)])
    ax.imshow(signs, cmap="RdBu_r", vmin=-1.5, vmax=1.5)
    ax.set_title(f"$\\Gamma^{{{x{l}}}{{}}_{{mn}}$", fontsize=9)
    ax.set_xticks(range(8), [f"{k}" for k in range(8)], fontsize=7)
    ax.set_yticks(range(8), [f"{k}" for k in range(8)], fontsize=7)
    ax.set_xlabel("$n$ (x-number)", fontsize=7)
    ax.set_ylabel("$m$ (x-number)", fontsize=7)
    ax.grid(False)
fig.suptitle("Signs of the Christoffel symbols at a sample point "
             "(red +, blue -, white 0)", fontsize=10)
fig.tight_layout()
```

`plt.subplots(2, 4)` makes eight small panels in two rows, one for each upper index `l`; `enumerate(axes.flat)` numbers them 0 to 7. For each panel an 8×8 table of the signs $+1$, -1 or 0 of ${\Gamma^l}_{mn}$ at the sample point is computed (`np.sign`; `.real` takes the real part of the complex number) and drawn by `imshow` as a picture whose squares are coloured with the colour map `"RdBu_r"` (red for

positive, blue for negative, white for zero). The tick labels number the rows and columns 0 to 7, the numbers of the directions $x0$ to $x7$; `ax.grid(False)` removes the grid lines; `fig.suptitle` puts a title above all panels and `fig.tight_layout()` arranges them without overlap. In the title string the doubled braces `{{` and `}}` print single braces inside an f-string.

```
save_figure(fig, "christoffel_pattern",
            "Where the Christoffel symbols $\\Gamma^l{}_{mn}$ of the author's metric "
            ...
            "\\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ (red "
            "positive, blue negative, white zero). Every nonzero symbol has an index "
            "equal to 0 or 4: the metric depends only on the hidden coordinate $x_0$ "
            "and on the time $x_4$. The grids are symmetric because "
            "$\\Gamma^l{}_{mn} = \\Gamma^l{}_{nm}$.")
```

The figure is saved; the cell prints the name of the saved file, `06a_3_christoffel_pattern.png`.

What Figure 06a.3 shows. Every coloured square lies in row 0 or 4, in column 0 or 4, or in a panel $l = x0$ or $l = x4$: every nonzero symbol carries an index $x0$ or $x4$, because the metric depends only on the hidden coordinate and on the time. Each grid is mirror-symmetric about its diagonal, because $\Gamma^l{}_{mn} = \Gamma^l{}_{nm}$. In the panels $l = x1, x2, x3$ the two squares with the index 4 are red ($a_4' > 0$: 3-space grows), and in the panels $l = x5, x6, x7$ they are blue ($\alpha_t = -a_4' + b_4'/(2b_4) = -\frac{1}{4}$ at the sample point: the extra times shrink). The panel $l = x4$ has red squares on the diagonal for all six warped directions, and the panel $l = x0$ has blue squares for 3-space, red squares for the extra times and one blue square at $(0, 0)$, as the signs derived in Section 6.3 say.

In [11], the canonical spin connection (Sections 6.4 and 6.6).

```
omega = spin_connection(e, Gam, x, ETA)
postulate = [sp.diff(e[a, nu], x[mu])
             - sum(Gam[lam][mu][nu] * e[a, lam] for lam in range(8))
             + sum(ETA[a] * omega[mu][a][b] * e[b, nu] for b in range(8))
             for mu in range(8) for a in range(8) for nu in range(8)]
check(len(postulate) == 512 and all(is_zero(v) for v in postulate),
      "the vielbein postulate holds in all 512 components",
      record=f"{REPORT_PY}, check vielbein_postulate")
```

`omega` holds all $8^3 = 512$ components $\omega_{\mu ab}$. The list `postulate` holds the left side of the vielbein postulate for every μ, a, ν; the mixed components are recovered as $\omega_\mu{}^a{}_b = \eta_{aa}\omega_{\mu ab}$ (multiplying by η_{aa} again undoes the lowering, because $\eta_{aa}^2 = 1$). The check requires 512 entries, all exactly zero.

```
check(all(is_zero(omega[mu][a][b] + omega[mu][b][a])
          for mu in range(8) for a in range(8) for b in range(8)),
      "omega_mu ab = -omega_mu ba for all mu, a, b",
      record=f"{REPORT_PY}, check spin_connection_antisymmetric")
```

The antisymmetry $\omega_{\mu ab} + \omega_{\mu ba} = 0$, checked for all 512 triples.

```
omega_nonzero = {(mu, a, b): omega[mu][a][b] for mu in range(8) for a in range(8)
                 for b in range(a + 1, 8) if omega[mu][a][b] != 0}
for (mu, a, b), value in omega_nonzero.items():
    say(f"omega_{NAMES[mu]},{NAMES[a]}{NAMES[b]} = {value}")
```

The nonzero components with $a < b$ are collected and printed, one line each (a long line continues, indented, on the next): twelve lines such as

```
omega_x1,x1x4 = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
```

which is $a_4' e^{a_4} s$, and

```
omega_x5,x4x5 = (-b4(x4)*Derivative(a4(x4), x4) + Derivative(b4(x4),
    x4)/2)*exp(-a4(x4))*sin(6*H*x0)**(1/6)/sqrt(b4(x4))
```

which is $-(a_4' - b_4'/(2b_4))e^{-a_4} b_4^{1/2} s$: exactly the table of Section 6.6.

```
omega_record = as_dictionary(record_formula("omega_nonzero", RECORD_NAMES,
                                            FORMULAS_B4))
omega_record_1 = as_dictionary(record_formula("omega_nonzero", RECORD_NAMES))
check(len(omega_nonzero) == 12 and set(omega_record) == set(omega_nonzero)
      and all(is_zero(omega_nonzero[k] - omega_record[k]) for k in omega_nonzero)
      and set(omega_record_1) == set(omega_nonzero)
      and all(is_zero(at_b4_1(omega_nonzero[k]) - omega_record_1[k])
              for k in omega_nonzero),
      "the 12 independent nonzero omega_mu ab equal the records (with b4, and at "
      "b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula omega_nonzero")
```

The records' lists `omega_nonzero` (entries $\{\mu, a, b, \text{value}\}$) are read and renumbered; the check requires exactly 12 components, the same keys and the same values as the record with b_4, and at $b_4 = 1$ as the record of $b_4 = 1$.

```
a4p, b4p = sp.Derivative(a4, x4), sp.Derivative(b4, x4)
rates = (a4p, a4p - b4p / (2 * b4), H)  # 3-space, the extra times, hidden direction
factors = [sign * phys for sign in (1, -1)
           for phys in (sp.exp(a4) * s, sp.exp(-a4) * sp.sqrt(b4) * s)]
# value / (rate * factor) must simplify to exactly 1 for one choice of each.
shape_ok = all(any(sp.simplify(value / (rate * phys)) == 1 for rate in rates
                   for phys in factors) for value in omega_nonzero.values())
check(shape_ok and record_passed(REPORT_WL_B4, "omega_components"),
      "every nonzero omega is a4', a4' - b4'/(2 b4) or H times +-exp(a4) s or "
      "+-exp(-a4) b4^(1/2) s",
      record=f"{REPORT_WL_B4}, check omega_components")
```

The last check makes the statement of Section 6.6 precise: for every nonzero component there is a choice of `rate` (one of a_4', $a_4' - b_4'/(2b_4)$ and H) and of `phys` (one of the four factors $\pm e^{a_4} s$, $\pm e^{-a_4} b_4^{1/2} s$) such that the component divided by `rate * phys` simplifies to exactly 1. `any(...)` is true when at least one choice works. The cell prints the twelve components and four PASS lines.

In [12], figure 4.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.6))
for ax, mu in zip(axes, (1, 5)):  # the directions x1 and x5
```

```
    table = np.array([[at_sample(omega[mu][a][b]).real for b in range(8)]
                      for a in range(8)])
    limit = np.abs(table).max()
    image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
```

Two panels, for $\mu = x1$ (position 1) and $\mu = x5$ (position 5); `zip(axes, (1, 5))` pairs each panel with its μ. `table` is the 8×8 array of the values $\omega_{\mu ab}$ at the sample point. The colour scale is symmetric, from minus to plus the largest absolute value `limit`, so that white means zero.

```
    for a in range(8):
        for b in range(8):
            if table[a, b] != 0:
                ax.text(b, a, f"{table[a, b]:.2f}", ha="center", va="center",
                        fontsize=7)
```

Each nonzero value is written into its square with two decimals (`f"{...:.2f}"`), centred.

```
    ax.set_xticks(range(8), NAMES, fontsize=8)
    ax.set_yticks(range(8), NAMES, fontsize=8)
    ax.set_xlabel("frame index $b$")
    ax.set_ylabel("frame index $a$")
    ax.set_title(f"$\\omega_{{x{mu}\\,ab}}$ at the sample point")
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.8)
fig.tight_layout()
```

The tick labels are the names $x0$ to $x7$, the axes are the frame indices a (rows) and b (columns), and each panel gets its own **colour bar** (`fig.colorbar`), the scale that translates colours into numbers.

```
save_figure(fig, "omega_heat_maps",
            "The canonical spin connection $\\omega_{\\mu ab}$ of the diagonal "
            ...
            "3-space direction $x1$.")
```

The figure is saved; the cell prints the name of the saved file, `06a_4_omega_heat_maps.png`.

What Figure 06a.4 shows. For $\mu = x1$ the only nonzero squares are $(x1, x4) = 0.78$ and $(x1, x0) = 1.56$ with their mirror images -0.78 and -1.56: the antisymmetry. These numbers are $a_4' e^{a_4} s$ and $H e^{a_4} s$ at the sample point, where $e^{a_4} s = e^{1/2} \sin^{1/6}(\pi/4) = 1.556$. For $\mu = x5$ the nonzero squares are $(x4, x5) = -0.14$ and $(x5, x0) = -0.57$ with their mirror images: $-(a_4' - \frac{b_4'}{2b_4}) e^{-a_4} b_4^{1/2} s$ and $-H e^{-a_4} b_4^{1/2} s$, with $e^{-a_4} b_4^{1/2} s = 0.572$ and $a_4' - b_4'/(2b_4) = \frac{1}{4}$. The extra time has the opposite signs of the 3-space direction and smaller values, because its factor e^{-a_4} is smaller than e^{a_4} when $a_4 > 0$, and the growing b_4 ($b_4' = \frac{1}{2}$) halves its time rate.

In [13], the spinor connection and the covariant constancy (Section 6.7).

```
Omega = spinor_connection(omega, S)  # eight 16 x 16 matrices
gup = curved_gammas(e, gamma)  # gamma^mu = gamma^a / f_a for mu = a
check(Omega[0] == Z16 and Omega[4] == Z16, "Omega_x0 = Omega_x4 = 0 for every b4",
      record=f"{REPORT_PY}, check Omega_x0_and_Omega_x4_vanish (b4 = 1)")
```

`Omega` is the list of the eight matrices Ω_μ and `gup` the list of the curved gammas, $\gamma^{(\mu)}/f_\mu$. The first check requires Ω_{x0} and Ω_{x4} (positions 0 and 4) to be the zero matrix, for every b_4; the record named in its second line states this at $b_4 = 1$.

```
rotation_ok = all(matrix_is_zero(
    Omega[mu] * gamma[a] - gamma[a] * Omega[mu]
    + sum((ETA[a] * omega[mu][a][b] * gamma[b] for b in range(8)), Z16))
    for mu in range(8) for a in range(8))
check(rotation_ok, "[Omega_mu, gamma^a] = -omega_mu^a_b gamma^b for all 64 pairs",
      record=f"{REPORT_WL}, check S_rotates_gamma_with_omega (b4 = 1)")
```

The defining property $[\Omega_\mu, \gamma^a] + \sum_b \omega_\mu{}^a{}_b \gamma^b = 0$, with $\omega_\mu{}^a{}_b = \eta_{aa}\omega_{\mu ab}$, checked for all 64 pairs (μ, a).

```
constancy_ok = all(matrix_is_zero(
    gup[nu].diff(x[mu]) + sum((Gam[nu][mu][lam] * gup[lam] for lam in range(8)), Z16)
    + Omega[mu] * gup[nu] - gup[nu] * Omega[mu])
    for mu in range(8) for nu in range(8))
check(constancy_ok and record_passed(REPORT_WL_B4, "gamma_covariantly_constant"),
      "D_mu gamma^nu = 0 for all 64 pairs (mu, nu)",
      record=f"{REPORT_WL_B4}, check gamma_covariantly_constant")
```

The covariant constancy $D_\mu \gamma^\nu = \partial_\mu \gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega_\mu, \gamma^\nu] = 0$, checked for all 64 pairs (μ, ν) (`.diff(x[mu])` differentiates every entry of the matrix); the WolframScript report with b_4 must record the same.

```
say(f"Omega_x1 has {sum(1 for v in Omega[1] if v != 0)} nonzero entries of 256")
```

The last line counts the nonzero entries of Ω_{x1} and prints Omega_x1 has 32 nonzero entries of 256. Each generator $S^{ab} = \frac{1}{2}\gamma^a\gamma^b$ is one half of a signed permutation matrix, with 16 nonzero entries, and the two generators of Ω_{x1} have them in different places: $16 + 16 = 32$.

In [14], figure 5.

```
pattern_ok = True
for a in range(8):
    for b in range(a + 1, 8):
        boost = ETA[a] * ETA[b] == -1  # one space-like, one time-like direction
        want = S[a][b] if boost else -S[a][b]  # S^T = S (boost) or -S (rotation)
        pattern_ok = pattern_ok and S[a][b].T == want
check(pattern_ok, "S^ab is symmetric for the 16 boosts, antisymmetric for the 12 "
      "rotations")
```

For all 28 pairs $a < b$ the loop decides whether the pair is a boost ($\eta_{aa}\eta_{bb} = -1$) and requires `S[a][b].T` (the transpose) to equal S^{ab} for a boost and $-S^{ab}$ for a rotation, as Section 6.7 proved.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.6))
for ax, mu in zip(axes, (1, 5)):  # the directions x1 and x5
    table = np.array([[at_sample(v).real for v in row] for row in Omega[mu].tolist()])
```

```
    limit = np.abs(table).max()
    image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.set_xticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_yticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_xlabel("column (spinor component 1 to 16)")
    ax.set_ylabel("row (spinor component)")
    ax.set_title(f"$\\Omega_{{x{mu}}}$ at the sample point")
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.8)
fig.tight_layout()
```

Two heat maps, of Ω_{x1} and Ω_{x5} at the sample point (`.tolist()` gives the rows of the matrix, and `at_sample` evaluates each entry). The ticks label every third component, 1, 4, 7, 10, 13, 16.

```
save_figure(fig, "spinor_connection_heat_maps",
            "The spinor connection $\\Omega_{x1}$ (left) and $\\Omega_{x5}$ (right) as "
            ...
            "components to the first eight and the last eight to the last eight.")
```

The figure is saved; the cell prints its PASS line and the saved file name.

What Figure 06a.5 shows. Each matrix has its colours only in the upper-left and the lower-right 8×8 blocks: a generator is a product of two gammas, an even product, and maps the first eight components to the first eight and the last eight to the last eight (the two chiral halves of Chapter 5). In each matrix one set of squares appears in mirror pairs of the same colour (a symmetric matrix, the boost part) and another in mirror pairs of opposite colours (an antisymmetric matrix, the rotation part). For Ω_{x1} the boost part carries a_4' (entries ± 0.39) and the rotation part H (entries ± 0.78); for Ω_{x5} the rotation part carries $a_4' - b_4'/(2b_4)$ and the boost part H (entries ± 0.07 and ± 0.29).

In [15], the contraction direction by direction (Section 6.8).

```
per_direction = []  # (alpha_mu, beta_mu) for mu = x0 ... x7
decomposed = True
for mu in range(8):
    M = (gup[mu] * Omega[mu]).applyfunc(sp.simplify)
    alpha = sp.simplify(-(M * gamma[4]).trace() / 16)  # coefficient of gamma^(x4)
    beta = sp.simplify((M * gamma[0]).trace() / 16)  # coefficient of gamma^(x0)
    decomposed = decomposed and matrix_is_zero(M - alpha * gamma[4] - beta * gamma[0])
    per_direction.append((alpha, beta))
    say(f"gamma^{NAMES[mu]} Omega_{NAMES[mu]} = ({alpha}) gamma^(x4) "
        f"+ ({beta}) gamma^(x0)")
rate_t = -a4p / 2 + b4p / (4 * b4)  # alpha of an extra time
predicted = ([(0, 0)] + [(a4p / 2, H / 2)] * 3 + [(0, 0)]
             + [(rate_t, H / 2)] * 3)  # x0, x1..x3, x4, x5..x7
check(decomposed and all(is_zero(per_direction[k][0] - predicted[k][0])
                         and is_zero(per_direction[k][1] - predicted[k][1])
                         for k in range(8))
      and record_passed(REPORT_PY_B4, "gammaOmega_per_direction")
      and record_passed(REPORT_WL_B4, "gammaOmega_per_direction"),
      "the eight terms gamma^mu Omega_mu (no sum) with b4",
      record=f"{REPORT_WL_B4}, check gammaOmega_per_direction")
```

For each μ, `M` is the matrix $\gamma^{\mu}\Omega_{\mu}$ (no sum), simplified entry by entry (`applyfunc`). Its coefficients are read off with traces as in Section 6.8 (`.trace()` is the trace; `gamma[4]` is $\gamma^{(x4)}$ and `gamma[0]` is $\gamma^{(x0)}$), and `decomposed` stays true only if $M - \alpha\gamma^{(x4)} - \beta\gamma^{(x0)}$ is the zero matrix for every μ. Each pair $(\alpha_{\mu}, \beta_{\mu})$ is stored and printed: eight lines, the table of Section 6.8.

```
table_record = record_formula("gammaOmega_per_direction", RECORD_NAMES)
check(all(is_zero(at_b4_1(per_direction[k][0]) - table_record[k][1])
          and is_zero(per_direction[k][1] - table_record[k][2]) for k in range(8)),
      "at b4 = 1 the eight terms equal the record",
      record=f"{THEORY_FILE}, formula gammaOmega_per_direction")
```

`rate_t` is the coefficient $-\frac{1}{2}a_4' + \frac{b_4'}{4b_4}$ of an extra time, and `predicted` the table of Section 6.8 in the order $x0, \ldots, x7$. The first check compares the eight computed rows with it and requires both reports with b_4 to record `gammaOmega_per_direction` as passed. The record of $b_4 = 1$ lists the table `gammaOmega_per_direction` as $\{\mu, \alpha_{\mu}, \beta_{\mu}\}$; the second check compares all eight rows at $b_4 = 1$ with it (`table_record[k][1]` is the record's α, `[k][2]` its β).

```
inflating = sp.simplify(sum(per_direction[k][0] for k in (1, 2, 3)))
deflating = sp.simplify(sum(per_direction[k][0] for k in (5, 6, 7)))
hidden = sp.simplify(sum(beta for _, beta in per_direction))
report("x4 part from the inflating x1, x2, x3", inflating)
report("x4 part from the deflating x5, x6, x7", deflating)
report("x0 part from all directions", hidden)
check(is_zero(inflating + deflating - sp.Rational(3, 4) * b4p / b4)
      and is_zero(hidden - 3 * H),
      "the a4' parts of the x4 terms cancel, (3/4) b4'/b4 remains; the x0 terms "
      "add to 3H",
      record=f"{REPORT_WL}, check gammaOmega_x4_terms_cancel (b4 = 1)")
```

The time parts of the three inflating directions (positions 1, 2, 3) and of the three deflating directions (positions 5, 6, 7) are added separately, and all eight hidden parts together. Three RESULT lines print $\frac{3}{2}a_4'$, $-\frac{3}{2}a_4' + \frac{3}{4}b_4'/b_4$ and $3H$, and the check requires the a_4' parts of the first two to cancel, leaving $\frac{3}{4}b_4'/b_4$, and the third to be $3H$.

```
slash = sum((gup[mu] * Omega[mu] for mu in range(8)), Z16)
gravity_term = 3 * H * gamma[0] + sp.Rational(3, 4) * b4p / b4 * gamma[4]
total_record = wolfram_to_sympy(
    FORMULAS_B4["gammaOmega_total"].replace('gamma["x0"]', "G0")
    .replace('gamma["x4"]', "G4"), RECORD_NAMES)
G0, G4 = sp.Symbol("G0"), sp.Symbol("G4")  # the record's gamma["x0"], gamma["x4"]
record_matrix = (total_record.coeff(G0) * gamma[0]
                 + total_record.coeff(G4) * gamma[4])
check(matrix_is_zero(slash - gravity_term) and matrix_is_zero(slash - record_matrix)
      and record_passed(REPORT_WL_B4, "gammaOmega_total"),
      "gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) exactly",
      record=f"{THEORY_B4_FILE}, formula gammaOmega_total")
check(matrix_is_zero(at_b4_1(slash) - 3 * H * gamma[0])
      and record_passed(REPORT_LEAD, "gamma_Omega_equals_3H_gamma0"),
      "at b4 = 1 (and for every constant b4): gamma^mu Omega_mu = 3 H gamma^(x0)",
      record=f"{REPORT_LEAD}, check gamma_Omega_equals_3H_gamma0")
```

`slash` is the full sum $\sum_\mu \gamma^\mu \Omega_\mu$ and `gravity_term` the matrix $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$. The record with b_4 writes the total as text with `gamma["x0"]` and `gamma["x4"]`; these are replaced by the plain symbols `G0` and `G4` before reading, and `.coeff(G0)` and `.coeff(G4)` pick out their factors, from which `record_matrix` is built. The next check requires `slash` to equal both, for symbolic H, a_4 and b_4, and the WolframScript report with b_4 to record `gammaOmega_total` as passed. The last check puts $b_4 = 1$: then the sum is exactly $3H\gamma^{(x0)}$, and the lead's independent report must record the same. The cell prints eight lines, five PASS lines and three RESULT lines.

In [16], figure 6.

```
a4_part = [float(sp.simplify(at_b4_1(alpha) / a4p)) for alpha, _ in per_direction]
b4_part = [float(sp.simplify(alpha.subs(a4p, 0) / (b4p / b4)))
           for alpha, _ in per_direction]  # the part with b4'/b4
beta_units = [float(sp.simplify(beta / H)) for _, beta in per_direction]  # beta / H
positions = np.arange(8)
```

The coefficient of $\gamma^{(x4)}$ is split into its part with a_4' (at $b_4 = 1$, divided by a_4') and its part with b_4'/b_4 (with $a_4' = 0$, divided by b_4'/b_4); the coefficient of $\gamma^{(x0)}$ is divided by H. All are ordinary decimal numbers (`float`): $\pm\frac{1}{2}$, $\frac{1}{4}$ or 0. `positions` is the list 0 to 7, the places of the bars.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2), sharey=True)
colors = ["gray"] + ["tab:red"] * 3 + ["gray"] + ["tab:blue"] * 3
panels = [(a4_part, "$\\gamma^{(x4)}$, part / $a_4'$"),
          (b4_part, "$\\gamma^{(x4)}$, part / $(b_4'/b_4)$"),
          (beta_units, "$\\gamma^{(x0)}$ / $H$")]
for ax, (heights, title) in zip(axes, panels):
    ax.bar(positions, heights, color=colors, width=0.6)
    ax.set_title(f"{title}: total {sum(heights):g}", fontsize=10)
```

Three bar charts with a common vertical axis (`sharey=True`), one for each of the three lists in `panels`; the bars of 3-space are red, of the extra times blue, of the hidden direction and the time gray. Each title shows the total (`:g` writes a number in its shortest form).

```
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, NAMES)
    ax.set_xlabel("direction $\\mu$")
axes[0].set_ylabel("coefficient (in the unit named in the title)")
```

A black zero line, the direction names under the bars, and the axis labels.

```
save_figure(fig, "gamma_omega_per_direction",
            "The eight terms $\\gamma^\\mu\\Omega_\\mu$ (no sum) of the diagonal "
            ...
            "\\beta_\\mu\\gamma^{(x0)}$; horizontal axis the direction $\\mu$ (red: "
            "inflating 3-space, blue: deflating extra times, gray: hidden direction "
            "and time). Left: the part of $\\alpha_\\mu$ proportional to $a_4'$, in "
            "units of $a_4'$: $+1/2$ for $x1, x2, x3$ and $-1/2$ for $x5, x6, x7$, "
            "total $0$. Middle: the part proportional to $b_4'/b_4$, in units of "
            "$b_4'/b_4$: $1/4$ for each extra time, total $3/4$. Right: $\\beta_\\mu$ "
            "in units of $H$: six times $1/2$, total $3$. So $\\gamma^\\mu\\Omega_\\mu "
            "= 3H\\gamma^{(x0)} + \\frac34(b_4'/b_4)\\gamma^{(x4)}$.")
```

The figure is saved; the cell prints its file name.

What Figure 06a.6 shows. On the left, three red bars of height $+\frac{1}{2}$ and three blue bars of height $-\frac{1}{2}$: the a'_4 parts of the time-direction terms of the inflating and the deflating directions cancel, total 0. In the middle, three blue bars of height $\frac{1}{4}$: the b'_4/b_4 parts come from the extra times alone, total $\frac{3}{4}$. On the right, six bars of height $\frac{1}{2}$ all pointing up: the hidden-direction terms add, total 3. The gray directions $x0$ and $x4$ contribute nothing, because $\Omega_{x0} = \Omega_{x4} = 0$.

In [17], the block form and figure 7.

```
I8, Z8 = sp.eye(8), sp.zeros(8, 8)  # 8 x 8 identity and zero matrices
blocks_ok = True
for entry in FORMULAS["field_equation_blocks"]:  # eight texts {"xa", tb, t}
    name, tb, t = json.loads(entry.replace("{", "[").replace("}", "]"))
    a = NAMES.index(name)  # the position of the direction xa (0 ... 7)
    # gamma^(xa) must be the block matrix with the zero blocks on the diagonal
    blocks_ok = blocks_ok and gamma[a] == sp.BlockMatrix(
        [[Z8, sp.Matrix(tb)], [sp.Matrix(t), Z8]]).as_explicit()
check(blocks_ok and len(FORMULAS["field_equation_blocks"]) == 8,
      "the eight gammas have the block form of the record",
      record=f"{THEORY_FILE}, formula field_equation_blocks")
check(gamma[0] == sp.BlockMatrix([[Z8, I8], [I8, Z8]]).as_explicit(),
      "gamma^(x0) has the 8 x 8 identity in its two off-diagonal blocks")
```

The record's formula `field_equation_blocks` is a list of eight texts {"xa", $\bar\tau, \tau$} written with curly brackets. Replacing the curly brackets by square brackets turns each text into JSON, which `json.loads` reads into a name and two 8×8 lists. `NAMES.index(name)` finds the position of the direction. `sp.BlockMatrix([[Z8, tb], [t, Z8]]).as_explicit()` builds the 16×16 matrix with zero blocks on the diagonal and the two blocks off it; the first check requires all eight gammas to have this form. The second requires $\gamma^{(x0)}$ to have the 8×8 identity in both off-diagonal blocks.

```
panels = [
    ("sum over $x1, x2, x3$", sum((gup[k] * Omega[k] for k in (1, 2, 3)), Z16)),
    ("sum over $x5, x6, x7$", sum((gup[k] * Omega[k] for k in (5, 6, 7)), Z16)),
    ("total $\\gamma^\\mu\\Omega_\\mu$", slash),
    ("$3H\\gamma^{(x0)} + \\frac{3}{4}\\frac{b_4'}{b_4}\\gamma^{(x4)}$", gravity_term),
]
```

The four matrices of the figure: the sum of the three inflating terms (positions 1, 2, 3), the sum of the three deflating terms (positions 5, 6, 7), the total and $3H\gamma^{(x0)} + \frac{3}{4}(b'_4/b_4)\gamma^{(x4)}$, each with its title.

```
fig, axes = plt.subplots(2, 2, figsize=(8.0, 7.6))
for ax, (title, matrix) in zip(axes.flat, panels):
    table = np.array([[at_sample(v).real for v in row] for row in matrix.tolist()])
    image = ax.imshow(table, cmap="RdBu_r", vmin=-3.5, vmax=3.5)
    ax.set_title(title, fontsize=10)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.7, label="entry value")
```

A 2×2 grid of heat maps, all on the same colour scale from -3.5 to 3.5, without tick marks, and one common colour bar.

```
save_figure(fig, "gamma_omega_cancellation",
            "The sum as pictures of 16 x 16 matrices at the sample point "
            "$H = 1$, $z = \\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, $b_4' = 1/2$ "
            "(red positive, blue negative, white zero; colour scale from $-3.5$ to "
            "$3.5$). Top left: the sum of the three inflating terms "
            "$\\gamma^\\mu\\Omega_\\mu$, $\\mu = x1, x2, x3$; top right: the sum of the "
            "three deflating terms, $\\mu = x5, x6, x7$; their $\\gamma^{(x4)}$ parts "
            "have opposite signs and sizes $0.75$ and $0.375$. Bottom left: the total "
            "over all eight directions; bottom right: $3H\\gamma^{(x0)} + "
            ...
            "\\frac34(b_4'/b_4)\\gamma^{(x4)}$. The two bottom pictures are identical; "
            "for a constant $b_4$ the faint $\\gamma^{(x4)}$ entries would vanish.")
```

The figure is saved; the cell prints two PASS lines and the file name.

What Figure 06a.7 shows. The two upper pictures contain the same pattern of $\gamma^{(x0)}$ with the value $\frac{3}{2}H = 1.5$ (two diagonal lines in the off-diagonal blocks), and on top of it the pattern of $\gamma^{(x4)}$ with opposite colours in the two pictures: entries $\pm\frac{3}{2}a_4' = \pm 0.75$ in the first and $\pm(\frac{3}{2}a_4' - \frac{3}{4}b_4'/b_4) = \pm 0.375$ in the second. In the total (lower left) the value on the two diagonal lines is 3, and only the faint $\gamma^{(x4)}$ entries $\pm\frac{3}{4}b_4'/b_4 = \pm 0.375$ remain; the lower right picture, $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$, is identical to it. For a constant b_4 the faint entries would vanish.

In [18], the divergence form (Section 6.9).

```
anticommutes = all(matrix_is_zero(gup[mu] * Omega[mu] + Omega[mu] * gup[mu])
                   for mu in range(8))
check(anticommutes, "{gamma^mu, Omega_mu} = 0 for each mu separately",
      record=f"{REPORT_PY}, check anticommutator_gamma_Omega_vanishes (b4 = 1)")
```

Fact 1: $\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu = 0$ for each μ separately.

```
sqrt_g = b4 ** sp.Rational(3, 2) * sp.cos(z)  # sqrt|det g|
divergence = sum(((sqrt_g * gup[mu]).diff(x[mu]) for mu in range(8)), Z16) / (2 * sqrt_g)
check(matrix_is_zero(divergence - gravity_term)
      and record_passed(REPORT_WL_B4, "gammaOmega_divergence_form")
      and record_passed(REPORT_LEAD, "gamma_Omega_divergence_formula"),
      "(1/(2 sqrt|g|)) d_mu (sqrt|g| gamma^mu) = 3 H gamma^(x0) + (3/4)(b4'/b4) "
      "gamma^(x4)",
      record=f"{REPORT_WL_B4}, check gammaOmega_divergence_form")
```

`divergence` is $\frac{1}{2\sqrt{|g|}}\sum_\mu \partial_\mu(\sqrt{|g|}\gamma^\mu)$ with $\sqrt{|g|} = b_4^{3/2}\cos z$; the check requires it to equal $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$, the WolframScript report with b_4 to record the same, and the lead's report to record the form at $b_4 = 1$.

```
products = [sp.simplify(sp.prod([f[c] for c in range(8) if c != b])) for b in range(8)]
for b in (0, 4):
```

```
    say(f"product of the f_c with c != {NAMES[b]}: {products[b]}")
own_coordinate = [sp.simplify(sp.diff(products[b], x[b]) / (2 * sqrt_g))
                  for b in range(8)]
say("(1/(2 sqrt|g|)) d_b (product without b), b = x0 ... x7: "
    f"{own_coordinate}")
expected_own = [3 * H, 0, 0, 0, sp.Rational(3, 4) * b4p / b4, 0, 0, 0]
check(all(is_zero(own_coordinate[b] - expected_own[b]) for b in range(8)),
      "only the hidden direction (3 H) and the time ((3/4) b4'/b4) contribute")
```

Fact 2: `products[b]` is the product of the seven factors other than f_b. The two interesting ones are printed, `b4(x4)**(3/2)*sin(6*H*x0)` for $b = x0$ and `b4(x4)**(3/2)*cos(6*H*x0)` for $b = x4$. `own_coordinate` divides the derivative of each product with respect to its own coordinate by $2\sqrt{|g|}$; the printed list is `[3*H, 0, 0, 0, 3*Derivative(b4(x4), x4)/(4*b4(x4)), 0, 0, 0]`, and the check requires exactly this: only the hidden direction contributes, with $3H$, and the time, with $\frac{3}{4}b_4'/b_4$.

In [19], central differences and figure 8.

```
H_value = 1.0
x0_values = np.linspace(0.002, np.pi / 12 - 0.002, 300)  # z = 6 x0 inside (0, pi/2)
step = 1e-5  # the step h of the central difference
```

Numbers instead of symbols: $H = 1$, 300 values of x_0 inside $(0, \pi/12)$ (so that $z = 6x_0$ lies inside $(0, \pi/2)$), and the step $h = 10^{-5}$.

```
def hidden_density(x0_value):
    return np.sin(6 * H_value * x0_value)  # sqrt|g| e_(x0)^x0 / b4^(3/2) = sin z
```

The hidden density $\sqrt{|g|}\, {e_{(x0)}}^{x0}$ divided by $b_4^{3/2}$, $\cos z \tan z = \sin z$, as a numerical function (the factor $b_4^{3/2}$ does not depend on x_0 and cancels in the quotient).

```
derivative = (hidden_density(x0_values + step)
              - hidden_density(x0_values - step)) / (2 * step)
ratio = derivative / (2 * np.cos(6 * H_value * x0_values))  # should be 3 H = 3
x4_values = np.linspace(-6.0, 6.0, 300)


def b4_history(x4_value):
    return 1.0 + 0.5 * np.sin(x4_value)  # an illustrative b4(x4) > 0


def time_density(x4_value):
    return b4_history(x4_value) ** 1.5  # sqrt|g| e_(x4)^x4 / cos z = b4^(3/2)


time_derivative = (time_density(x4_values + step)
                   - time_density(x4_values - step)) / (2 * step)
time_ratio = time_derivative / (2 * time_density(x4_values))  # (1/(2 b4^1.5)) d b4^1.5
exact_time = 0.75 * 0.5 * np.cos(x4_values) / b4_history(x4_values)  # (3/4) b4'/b4
```

```
report("largest |central difference - 3 H| over the grid",
       f"{np.max(np.abs(ratio - 3.0)):.1e}")
report("largest |central difference - (3/4) b4'/b4| over the grid",
       f"{np.max(np.abs(time_ratio - exact_time)):.1e}")
check(np.max(np.abs(ratio - 3.0)) < 1e-6
      and np.max(np.abs(time_ratio - exact_time)) < 1e-6,
      "the numerical derivatives give 3 H and (3/4) b4'/b4")
```

The **central difference** $(F(x_0 + h) - F(x_0 - h))/(2h)$ approximates the derivative $F'(x_0)$ with an error of about $\frac{h^2}{6}F'''$; divided by $2\cos z$ it should be $3H = 3$. For the time direction `b4_history` is the illustrative $b_4(x_4) = 1 + \frac{1}{2}\sin x_4$ on 300 times from -6 to 6, and `time_density` is the time density divided by $\cos z$, $b_4^{3/2}$ (`** 1.5` is the power $\frac{3}{2}$). Its central difference, divided by $2b_4^{3/2}$, should be the exact `exact_time`, $\frac{3}{4}b_4'/b_4 = \frac{3}{4}\cdot\frac{\frac{1}{2}\cos x_4}{1+\frac{1}{2}\sin x_4}$. The two RESULT lines print the largest deviations, 1.9e-09 and 2.5e-11, and the check requires both to be below 10^{-6}. For $F = \sin 6x_0$ the error estimate is $\frac{h^2}{6}\cdot 216\cos z/(2\cos z) = 18h^2 = 1.8\times 10^{-9}$, the printed size; the time derivatives of $b_4^{3/2}$ are of size 1, so its error is about 10^{-11}.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
zz = 6 * H_value * x0_values
ax_left.plot(zz, ratio, label="$\\partial_{x0}(b_4^{3/2}\\sin z)/"
                              "(2b_4^{3/2}\\cos z)$")
ax_left.set_ylim(0.0, 3.8)
ax_left.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_left.set_ylabel("coefficient of $\\gamma^{(x0)}$ (units of $H$)")
ax_left.set_title("Hidden direction: the constant $3H$")
ax_left.legend(fontsize=8, loc="lower center")
ax_right.plot(x4_values, time_ratio,
              label="central difference, $b_4 = 1 + \\frac{1}{2}\\sin x_4$")
ax_right.plot(x4_values, exact_time, ":", color="black", linewidth=2.5,
              label="exact $\\frac{3}{4} b_4'/b_4$")
ax_right.plot(x4_values, np.zeros_like(x4_values), "--", color="gray",
              label="$b_4 = 1$: exactly $0$")
```

The left panel draws the numerical coefficient of $\gamma^{(x0)}$ versus z.

```
ax_right.set_ylim(-1.05, 0.6)  # room for the legend below the curves
ax_right.set_xlabel("time $x_4$ (units $1/H$)")
ax_right.set_ylabel("coefficient of $\\gamma^{(x4)}$")
ax_right.set_title("Time direction: $\\frac{3}{4} b_4'/b_4$")
ax_right.legend(fontsize=8, loc="lower center")
```

The right panel draws the numerical coefficient of $\gamma^{(x4)}$ for the illustrative history (solid), the exact $\frac{3}{4}b_4'/b_4$ (dotted) and the zero of $b_4 = 1$ (dashed) versus the time; `set_ylim(-1.05, 0.6)` leaves room for the legend below the curves.

```
save_figure(fig, "divergence_form",
            "The divergence form of $\\gamma^\\mu\\Omega_\\mu$, with $H = 1$, checked "
            "by central differences with step $10^{-5}$. Left: the coefficient of "
```

```
"$\\gamma^{(x0)}$, $\\partial_{x0}(b_4^{3/2}\\sin z)/(2b_4^{3/2}\\cos z)$, "
"versus the hidden angle $z$ in radians: the constant $3$ for every $z$. "
"Right: the coefficient of $\\gamma^{(x4)}$, "
"$\\partial_{x4}(b_4^{3/2}\\cos z)/(2b_4^{3/2}\\cos z)$, versus the time "
"$x_4$ for the illustrative history $b_4 = 1 + \\frac12\\sin x_4$ (solid), "
"on the exact $\\frac34 b_4'/b_4$ (dotted), and $0$ for $b_4 = 1$ "
...
"(dashed): the time direction contributes only through a changing $b_4$.")
```

The figure is saved; the cell prints the RESULT line, the PASS line and the file name.

What Figure 06a.8 shows. On the left, the solid line is flat at 3 over the whole range of z: the coefficient of $\gamma^{(x0)}$ is the constant $3H$. On the right, the solid central differences lie on the dotted exact curve $\frac{3}{4}b_4'/b_4$, which oscillates between about -0.43 and 0.43 with the illustrative b_4; for $b_4 = 1$ the coefficient is the dashed 0. The time direction contributes only through a changing b_4.

In [20], the Dirac operator written out (Section 6.10).

```
import re  # text patterns, to rename the record's symbols

psi = sp.Matrix([sp.Function(f"psi{A}")(*x) for A in range(1, 17)])  # 16 functions
D = [psi.diff(x[mu]) + Omega[mu] * psi for mu in range(8)]  # D_mu Psi
dirac = sum((gup[mu] * D[mu] for mu in range(8)), sp.zeros(16, 1))
written_out = sum((gamma[a] * psi.diff(x[a]) / f[a] for a in range(8)),
                  sp.zeros(16, 1)) + gravity_term * psi
check(all(is_zero(v) for v in dirac - written_out)
      and record_passed(REPORT_WL_B4, "Dirac_operator_explicit_C"),
      "gamma^mu D_mu Psi = sum_a (1/f_a) gamma^a d_a Psi + 3 H gamma^(x0) Psi + "
      "(3/4)(b4'/b4) gamma^(x4) Psi",
      record=f"{REPORT_WL_B4}, check Dirac_operator_explicit_C")
```

`re` is Python's module for text patterns. `psi` is a column of sixteen unknown functions $\psi_1, \ldots, \psi_{16}$ of all eight coordinates (`sp.Function(f"psi{A}")(*x)` makes the function named psi1, psi2, ... with the eight arguments). `D` is the list of the eight covariant derivatives $\partial_\mu\Psi + \Omega_\mu\Psi$, `dirac` is $\sum_\mu \gamma^\mu D_\mu\Psi$, and `written_out` is $\sum_a f_a^{-1}\gamma^{(a)}\partial_a\Psi + (3H\gamma^{(x0)} + \frac{3}{4}\frac{b_4'}{b_4}\gamma^{(x4)})\Psi$. The check requires the sixteen differences to vanish and the WolframScript report with b_4 to record `Dirac_operator_explicit_C` as passed.

```
# Rename the derivatives d psi_B / d x_k and then the functions psi_B as plain
# symbols dPsi<B>x<k> and Psi<B> (derivatives first: renaming psi_B first would
# turn its derivatives into derivatives of a constant symbol, which are zero).
rename_derivatives = {sp.Derivative(psi[B], x[k]): sp.Symbol(f"dPsi{B + 1}x{k}")
                      for B in range(16) for k in range(8)}
rename_functions = {psi[B]: sp.Symbol(f"Psi{B + 1}") for B in range(16)}
```

To compare with the records' sixteen equations, the derivatives $\partial\psi_B/\partial x_k$ are renamed as plain symbols `dPsi<B>x<k>` (with the number k of the coordinate x_k, 0 to 7) and the functions as `Psi<B>`. The order matters, as the comment says: renaming the functions first would turn their derivatives into derivatives of constants, which are zero.

```
def same_components(equations, at_b4_equal_1):
    same = True
    for A, equation in enumerate(equations):
        left = equation.split("==")[0]  # the record: left side == V*Psi[A]
        left = left.replace(chr(34), "")  # chr(34) is the double-quote character
        # dd[x4, Psi[14]] (the derivative of Psi_14 by x4) becomes dPsi14x4:
        left = re.sub(r"dd\[x(\d), Psi\[(\d+)\]\]", r"dPsi\2x\1", left)
        left = re.sub(r"Psi\[(\d+)\]", r"Psi\1", left)
        record_left = wolfram_to_sympy(left, RECORD_NAMES)
        mine = dirac[A].subs(rename_derivatives).subs(rename_functions)
        if at_b4_equal_1:
            mine = at_b4_1(mine)
        same = same and is_zero(mine - record_left)
    return same


check(same_components(FORMULAS_B4["field_equation_components"], False)
      and same_components(FORMULAS["field_equation_components"], True),
```

`same_components(equations, at_b4_equal_1)` compares the sixteen equations of one record with the notebook's. For each record equation (`enumerate` numbers them from 0) the text left of `==` is taken (the right side is $V\Psi_A$ and not needed). `chr(34)` is the double-quote character, which is removed. The pattern `dd\[x(\d), Psi\[(\d+)\]\]` matches texts such as `dd[x4, Psi[14]]`, the derivative of ψ_{14} by x_4; `\d` is a digit, `\d+` one or more digits, the parentheses capture them, and `re.sub` replaces the match by `dPsi14x4` (`\2` and `\1` insert the captured numbers). The second pattern turns `Psi[9]` into `Psi9`. The text is read with `wolfram_to_sympy`, and the notebook's component `A` with the same renaming (at $b_4 = 1$ when the second argument asks for it) must agree with it exactly.

```
      "all 16 components of the Dirac operator equal the records (with b4, and at "
      "b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula field_equation_components")
c_b4 = sp.Rational(3, 4) * b4p / b4  # the coefficient of gamma^(x4)
square = (gravity_term * gravity_term - (9 * H**2 - c_b4**2) * I16)
ranks = [(gamma[0] + sign * gamma[4]).rank() for sign in (1, -1)]
report("ranks of gamma^(x0) + gamma^(x4) and gamma^(x0) - gamma^(x4)", ranks)
both_fields = all(record_passed(REPORT_WL, name) for name in [
    "nontriviality_1_dirac16complex", "nontriviality_2_dirac16complex00"])
check(matrix_is_zero(square) and gamma[0] * gamma[0] == I16 and ranks == [8, 8]
      and both_fields and record_passed(REPORT_PY_B4, "gravity_term_square_b4"),
      "M M = (9 H^2 - (9/16)(b4'/b4)^2) 1: M Psi = 0 only for Psi = 0 unless "
      "|b4'/b4| = 4 H (always for constant b4), and there M has rank 8",
      record=f"{REPORT_PY_B4}, check gravity_term_square_b4; {REPORT_WL}, checks "
             "nontriviality_1_dirac16complex and nontriviality_2_dirac16complex00")
```

The first check requires all sixteen components to agree with both records: with b_4 as they are, and at $b_4 = 1$. Then the non-triviality of Section 6.10. `c_b4` is $\frac{3}{4}b_4'/b_4$, and `square` is $MM - (9H^2 - c^2)\,1$ for the matrix $M =$ `gravity_term`, which must be the zero matrix. `ranks` holds the exact ranks of $\gamma^{(x0)} + \gamma^{(x4)}$ and $\gamma^{(x0)} - \gamma^{(x4)}$ (`.rank()` computes the rank exactly, by elimination with fractions); the RESULT line prints `[8, 8]`. `both_fields` is `True` when the WolframScript report `wolfram-field-theory.json` (the name `REPORT_WL` of In [3]) records both of its non-triviality checks, made

at $b_4 = 1$, as passed: `nontriviality_1_dirac16complex` for the Grassmann field and `nontriviality_2_dirac16complex00` for the commuting field (`all(...)` is `True` when every item of the list is). The last check requires the square, $(\gamma^{(x0)})^2 = 1$, the two ranks 8, both non-triviality checks and the check `gravity_term_square_b4` of the sympy report with b_4: the term vanishes only for $\Psi = 0$ unless $|b_4'/b_4| = 4H$, and there M has rank 8. Its second printed line names the record checks it consulted. The cell prints three PASS lines with their records and one RESULT line.

In [21], figure 9.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
coefficient = [sp.lambdify(X4_SYMBOL, (1 / fa).subs(history).subs(
    {H: 1, x0: sp.pi / 24})) for fa in f]
```

The eight factors $1/f_a$ in front of the derivatives, as numerical functions of the time for the illustrative history $a_4 = x_4$, $b_4 = 1$, at $z = \pi/4$.

```
ax_left.semilogy(times, coefficient[1](times),
                 label="$1/f_1 = e^{-a_4}/s$ (3-space derivatives)")
ax_left.semilogy(times, coefficient[5](times), "--",
                 label="$1/f_5 = e^{a_4}b_4^{-1/2}/s$ (extra-time derivatives)")
ax_left.semilogy(times, np.ones_like(times), ":", color="black",
                 label="$1/f_4 = 1$ (time derivative)")
ax_left.set_xlabel("time $x_4$ (units $1/H$), history $a_4 = x_4$, $b_4 = 1$")
ax_left.set_ylabel("factor in front of the derivative (log axis)")
ax_left.set_title("Where the deflation enters the field equation")
ax_left.legend(fontsize=8)
```

The left panel, on a logarithmic axis: $1/f_1 = e^{-a_4}/s$ (solid), $1/f_5 = e^{a_4} b_4^{-1/2}/s$ (dashed) and $1/f_4 = 1$ (dotted).

```
zz = np.linspace(0.05, np.pi / 2 - 0.15, 300)
ax_right.plot(zz, np.tan(zz), label="$1/f_0 = \\tan z$ (hidden derivative)")
ax_right.plot(zz, np.sin(zz) ** (-1 / 6), "--",
              label="$1/s = \\sin^{-1/6} z$ (warped directions, $a_4 = 0$)")
ax_right.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_right.set_ylabel("factor (pure number)")
ax_right.set_title("Dependence on the hidden direction")
ax_right.legend(fontsize=8)
```

The right panel, versus z: $1/f_0 = \tan z$ and $1/s = \sin^{-1/6} z$; the range stops at $\pi/2 - 0.15$ because $\tan z$ grows without bound at $\pi/2$.

```
save_figure(fig, "derivative_coefficients",
            "The factors $1/f_a$ in front of the derivatives in the Dirac operator "
            ...
            "the six warped directions.")
```

The figure is saved; the cell prints the file name.

What Figure 06a.9 shows. On the left the two warped factors are straight lines of opposite slope: the factor in front of the 3-space derivatives falls and the factor in front of the extra-time derivatives rises as a_4 grows. This is how the deflation of the extra times enters the field equation, although it does not enter $\gamma^\mu \Omega_\mu$. On the right, the hidden factor $\tan z$ rises steeply towards the patch end, while $\sin^{-1/6} z$ is large only very near the tip.

In [22], the negative control (Section 6.10).

```
f_control = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
             + [sp.exp(a4) * s] * 3)  # extra times INFLATING, b4 = 1
g_control = sp.diag(*[ETA[a] * f_control[a] ** 2 for a in range(8)])
e_control = sp.diag(*f_control)
Gam_control = christoffel(g_control, x)
omega_control = spin_connection(e_control, Gam_control, x, ETA)
Omega_control = spinor_connection(omega_control, S)
gup_control = curved_gammas(e_control, gamma)
slash_control = sum((gup_control[mu] * Omega_control[mu] for mu in range(8)), Z16)
```

The same computation for the control metric at $b_4 = 1$: the factors of the extra times are $e^{+a_4} s$ instead of $e^{-a_4} s$, and everything else (metric, vielbein, Christoffel symbols, spin connection, spinor connection, curved gammas, contraction) is computed by the same four functions.

```
alpha_control = sp.simplify(-(slash_control * gamma[4]).trace() / 16)
beta_control = sp.simplify((slash_control * gamma[0]).trace() / 16)
report("control: coefficient of gamma^(x4)", alpha_control)
report("control: coefficient of gamma^(x0)", beta_control)
```

The two coefficients of the control contraction, found with traces, are printed: `3*Derivative(a4(x4), x4)` and `3*H`.

```
lead = json.loads(repository_file(REPORT_LEAD).read_text(encoding="utf-8"))
detail = [c["detail"] for c in lead["checks"]
          if c["name"] == "negative_control_inflating_extra_times"][0]
```

The lead's report is read, and `detail` is the detail text of its check `negative_control_inflating_extra_times` (the list comprehension keeps the one matching check, and `[0]` takes it).

```
check(matrix_is_zero(slash_control - alpha_control * gamma[4] - beta_control * gamma[0])
      and is_zero(alpha_control - 3 * sp.Derivative(a4, x4))
      and is_zero(beta_control - 3 * H)
      and "(3*Derivative(a4(x4), x4)) gamma^(x4)" in detail
      and record_passed(REPORT_LEAD, "negative_control_inflating_extra_times"),
      "control with inflating extra times: 3 a4' gamma^(x4) + 3 H gamma^(x0)",
      record=f"{REPORT_LEAD}, check negative_control_inflating_extra_times")
```

The check requires the control contraction to be exactly $3a_4'\gamma^{(x4)} + 3H\gamma^{(x0)}$ with nothing left over, the lead's detail text to contain the same coefficient, and the lead's check to be recorded as passed. The cell prints two RESULT lines and one PASS line.

In [23], figure 10.

```
rates = np.linspace(-2.0, 2.0, 81)  # values of a4'
rate_symbol = sp.Symbol("rate")
author_alpha = sp.lambdify(rate_symbol, sp.simplify(at_b4_1(sum(
    a for a, _ in per_direction))).subs(a4p, rate_symbol))
control_alpha = sp.lambdify(rate_symbol, alpha_control.subs(
    sp.Derivative(a4, x4), rate_symbol))
```

81 rates a_4' from -2 to 2. The total time coefficient of the author's metric at $b_4 = 1$ (0) and of the control ($3a_4'$) become numerical functions of the rate (the derivative a_4' is replaced by the plain symbol `rate`).

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
ax_left.plot(rates, np.broadcast_to(author_alpha(rates), rates.shape),
             label="author: deflating extra times")
ax_left.plot(rates, np.broadcast_to(control_alpha(rates), rates.shape), "--",
             label="control: inflating extra times")
ax_left.set_xlabel("rate $a_4'$ (units of $H$)")
ax_left.set_ylabel("coefficient of $\\gamma^{(x4)}$ (units of $H$)")
ax_left.set_title("The time-direction term")
ax_left.legend(fontsize=8)
```

The left panel: the author's coefficient (solid) and the control's (dashed) versus the rate. `np.broadcast_to` turns the author's constant 0 into a list of 81 zeros.

```
ax_right.plot(rates, np.full_like(rates, float(hidden.subs(H, 1))),
              label="author: deflating extra times")
ax_right.plot(rates, np.full_like(rates, float(beta_control.subs(H, 1))), "--",
              label="control: inflating extra times")
ax_right.set_ylim(0.0, 4.0)
ax_right.set_xlabel("rate $a_4'$ (units of $H$)")
ax_right.set_ylabel("coefficient of $\\gamma^{(x0)}$ (units of $H$)")
ax_right.set_title("The hidden-direction term (identical lines)")
ax_right.legend(fontsize=8)
```

The right panel: the hidden coefficient $3H$ with $H = 1$ for both metrics (`np.full_like` makes a list of 81 equal values).

```
save_figure(fig, "negative_control",
            "The negative control, at $b_4 = 1$. Left: the coefficient of "
            "$\\gamma^{(x4)}$ in $\\gamma^\\mu\\Omega_\\mu$ versus the rate $a_4'$, "
            "both in units of $H = 1$, for the author's metric with deflating extra "
            "times (solid, zero for every rate) and for a control metric with "
            "inflating extra times (dashed, "
            "the line $3a_4'$). Right: the coefficient of $\\gamma^{(x0)}$, $3H$ in "
            ...
            "of the time-direction terms is caused by the deflation.")
```

The figure is saved; the cell prints the file name.

What Figure 06a.10 shows. On the left, the author's line lies flat on 0 for every rate, while the control's line rises through the origin with slope 3. On the right, the two lines lie on top of each other at 3. The deflation of the extra times is exactly what cancels the a_4' terms along the time; the hidden-direction term does not care whether the extra times deflate or inflate.

In [24], the last checks.

```
cited = {
    REPORT_PY_B4: ["christoffel_nonzero", "omega_nonzero", "gammaOmega_per_direction",
                   "gammaOmega_total", "gammaOmega_formula_text",
                   "gravity_term_square_b4"],
    REPORT_WL_B4: ["metric_is_the_authors_b4", "vielbein_inverse", "sqrt_det_g",
                   "christoffel_count", "vielbein_postulate", "omega_antisymmetric",
                   "omega_components", "gamma_covariantly_constant",
                   "gammaOmega_total", "gammaOmega_per_direction",
                   "gammaOmega_divergence_form", "Dirac_operator_explicit_G",
                   "Dirac_operator_explicit_C"],
    REPORT_PY: ["clifford_relations", "gammas_real", "metric_from_vielbein_equals_SPEC",
                "sqrt_det_g_equals_cos_z", "vielbein_postulate",
                "spin_connection_antisymmetric", "nontriviality_Omega_zero_iff_flat",
                "Omega_x0_and_Omega_x4_vanish", "covariant_constancy_D_mu_gamma_nu",
                "time_terms_cancel_hidden_term_survives",
                "gamma_mu_Omega_mu_equals_3H_gamma_x0",
                "anticommutator_gamma_Omega_vanishes", "divergence_of_sqrtg_gamma"],
    REPORT_WL: ["metric_is_the_authors", "sqrt_det_g_is_cos_z", "christoffel_count",
                "vielbein_postulate", "omega_antisymmetric", "omega_components",
                "Omega_components", "S_rotates_gamma_with_omega",
                "gamma_covariantly_constant", "gammaOmega_equals_3H_gamma_x0",
                "gammaOmega_x4_terms_cancel", "gammaOmega_divergence_form",
                "Dirac_operator_explicit_C", "Dirac_operator_explicit_G",
                "nontriviality_1_dirac16complex", "nontriviality_2_dirac16complex00",
                "Omega_vanishes_iff_a4prime_and_H_vanish", "block_form"],
    REPORT_LEAD: ["vielbein_reproduces_metric", "gamma_Omega_equals_3H_gamma0",
                  "gamma_Omega_divergence_formula",
                  "negative_control_inflating_extra_times"],
}
count = sum(len(names) for names in cited.values())
check(all(record_passed(path, name) for path, names in cited.items()
          for name in names),
      f"the {count} cited record checks are recorded as passed")
```

`cited` lists, for each of the five reports, the checks that state a result reproduced in this notebook: 6 of the sympy report with b_4, 13 of the WolframScript report with b_4, 13 of the sympy report and 18 of the WolframScript report of $b_4 = 1$, and 4 of the lead's report, 54 in all. The check requires every one of them to be recorded as passed, so that the record this notebook reproduces is itself a passing record.

```
expected = [f"06a_{k}_{name}.png" for k, name in enumerate([
    "polar_frame_and_spin_rotation", "vielbein_factors", "christoffel_pattern",
    "omega_heat_maps", "spinor_connection_heat_maps", "gamma_omega_per_direction",
    "gamma_omega_cancellation", "divergence_form", "derivative_coefficients",
    "negative_control"], 1)]
```

```
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected),
      "all ten figure files exist")
all_checks_passed()
```

The names of the ten figure files (`enumerate(..., 1)` numbers them from 1); the check requires all ten to exist, and `all_checks_passed()` prints the last line, ALL 36 CHECKS PASSED (notebook 06a).

6.15 Changing the frame: local spin covariance

Section 6.2 showed that the vielbein is a choice: for every point-dependent matrix $\Lambda(x)$ with $\Lambda^T\eta\Lambda = \eta$ the frame $e' = \Lambda e$ describes the same metric. Physics must not depend on this choice. This section derives how the spin connection, the spinor connection, the curved gammas and the contraction $\gamma^\mu\Omega_\mu$ change when the frame is changed. Matrices are written without indices: for each μ, ω_μ is the 8×8 matrix with the entries $\omega_\mu{}^a{}_b$ (row a, column b), and e_ν is the column of the eight numbers $e^a{}_\nu$.

The spin connection of the new frame. In this notation the vielbein postulate of Section 6.4 reads $\partial_\mu e_\nu - \sum_\lambda \Gamma^\lambda{}_{\mu\nu} e_\lambda + \omega_\mu e_\nu = 0$. The new frame describes the same metric, so it has the same Christoffel symbols. Its postulate, with $e'_\nu = \Lambda e_\nu$:

$$\partial_\mu(\Lambda e_\nu) - \sum_\lambda \Gamma^\lambda{}_{\mu\nu}\Lambda e_\lambda + \omega'_\mu \Lambda e_\nu = (\partial_\mu\Lambda)e_\nu + \Lambda\Big(\partial_\mu e_\nu - \sum_\lambda \Gamma^\lambda{}_{\mu\nu} e_\lambda\Big) + \omega'_\mu\Lambda e_\nu$$

(the product rule; Λ taken out of the sum)

$$= (\partial_\mu\Lambda)e_\nu - \Lambda\omega_\mu e_\nu + \omega'_\mu\Lambda e_\nu = 0$$

(the old postulate replaces the bracket by $-\omega_\mu e_\nu$). The eight columns e_ν form an invertible matrix, so the matrix in front of them vanishes: $\partial_\mu\Lambda - \Lambda\omega_\mu + \omega'_\mu\Lambda = 0$, and multiplying from the right by Λ^{-1}:

$$\omega'_\mu = \Lambda\,\omega_\mu\,\Lambda^{-1} - (\partial_\mu\Lambda)\Lambda^{-1}.$$

The first term turns the old connection with the frame. The second term is new: it is the rate at which the frame is turned from point to point. The spin connection is not a tensor in its frame indices.

The spinor connection of the new frame. When the frame is turned by Λ, the spinor components change by a spin transformation $R(x)$ that **covers** Λ (Chapter 5): $R^{-1}\gamma^a R = \sum_c \Lambda^a{}_c\gamma^c$ for every a, or, inverted, $R\gamma^c R^{-1} = \sum_d (\Lambda^{-1})^c{}_d\gamma^d$. The new components are $\Psi' = R\Psi$. Covariance demands that the covariant derivative of the new components be the new components of the covariant derivative, $D'_\mu(R\Psi) = R\,D_\mu\Psi$ for every Ψ. Write it out:

$$\partial_\mu(R\Psi) + \Omega'_\mu R\Psi = (\partial_\mu R)\Psi + R\,\partial_\mu\Psi + \Omega'_\mu R\Psi \qquad \text{must equal} \qquad R\,\partial_\mu\Psi + R\,\Omega_\mu\Psi$$

(the product rule on the left). The terms $R\,\partial_\mu\Psi$ agree, so $(\partial_\mu R)\Psi + \Omega'_\mu R\Psi = R\Omega_\mu\Psi$ for every Ψ, that is $\partial_\mu R + \Omega'_\mu R = R\Omega_\mu$, and multiplying from the right by R^{-1}:

$$\Omega'_\mu = R\,\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}.$$

Theorem (local spin covariance). The canonical spinor connection of the frame $e' = \Lambda e$ is exactly this matrix. Proof, line by line. By the uniqueness of Section 6.7 it suffices to show that $X_\mu = R\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}$ has the defining property with the new connection ω'_μ, and trace 0. First part of X_μ:

$$\begin{aligned}[R\Omega_\mu R^{-1}, \gamma^a] &= R\,[\Omega_\mu, R^{-1}\gamma^a R]\,R^{-1} = \sum_c \Lambda^a{}_c\, R[\Omega_\mu, \gamma^c]R^{-1} \\ &= -\sum_{c,b} \Lambda^a{}_c\,\omega_\mu{}^c{}_b\, R\gamma^b R^{-1} = -\sum_d \left(\Lambda\omega_\mu\Lambda^{-1}\right)^a{}_d\gamma^d\end{aligned}$$

(multiply out both sides of the first equality; the covering relation; the defining property of Ω_μ; the inverted covering relation and the rule of matrix multiplication). Second part: differentiate the covering relation $R^{-1}\gamma^a R = \sum_c \Lambda^a{}_c\gamma^c$ with the product rule, using $\partial_\mu(R^{-1}) = -R^{-1}(\partial_\mu R)R^{-1}$ (differentiate $R^{-1}R = 1$):

$$-R^{-1}(\partial_\mu R)R^{-1}\gamma^a R + R^{-1}\gamma^a\,\partial_\mu R = \sum_c (\partial_\mu \Lambda^a{}_c)\gamma^c.$$

Multiply from the left by R and from the right by R^{-1}:

$$-(\partial_\mu R)R^{-1}\gamma^a + \gamma^a(\partial_\mu R)R^{-1} = \sum_c (\partial_\mu\Lambda^a{}_c)R\gamma^c R^{-1} = \sum_d \left((\partial_\mu\Lambda)\Lambda^{-1}\right)^a{}_d\gamma^d.$$

The left side is $[-(\partial_\mu R)R^{-1}, \gamma^a]$. Adding the two parts:

$$[X_\mu, \gamma^a] = -\sum_d \left(\Lambda\omega_\mu\Lambda^{-1} - (\partial_\mu\Lambda)\Lambda^{-1}\right)^a{}_d\gamma^d = -\sum_d \omega'^{\,a}_{\mu\;d}\,\gamma^d.$$

The trace of $R\Omega_\mu R^{-1}$ is that of Ω_μ, which is 0 ($\mathrm{tr}(ABA^{-1}) = \mathrm{tr}(BA^{-1}A)$). The trace of $(\partial_\mu R)R^{-1}$ is the derivative of $\ln\det R$, a standard fact of linear algebra (Jacobi's formula, quoted without proof here, ASSUMED), and $\det R = 1$ for every element of Pin(4,4) (Chapter 5), so it is 0. For the one frame change used below this trace is also computed directly. □

The curved gammas and the contraction. The new inverse vielbein is $e'^{\,\mu}_a = \sum_c e_c{}^\mu(\Lambda^{-1})^c{}_a$ (the inverse of Λe is $e^{-1}\Lambda^{-1}$). Hence

$$\gamma'^\mu = \sum_a e'^{\,\mu}_a\gamma^a = \sum_c e_c{}^\mu \sum_a (\Lambda^{-1})^c{}_a\gamma^a = \sum_c e_c{}^\mu\, R\gamma^c R^{-1} = R\,\gamma^\mu R^{-1}$$

(the inverted covering relation). Multiply with Ω'_μ and sum over μ:

$$\sum_\mu \gamma'^\mu \Omega'_\mu = \sum_\mu R\gamma^\mu R^{-1}\big(R\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}\big) = R\Big(\sum_\mu \gamma^\mu \Omega_\mu\Big)R^{-1} - \sum_\mu \gamma'^\mu (\partial_\mu R)R^{-1}$$

($R^{-1}R = 1$ in the first product; $R\gamma^\mu R^{-1} = \gamma'^\mu$ in the second). The first term only turns the old contraction; **the second term can change it completely**. The contraction $\gamma^\mu\Omega_\mu$ is **frame-dependent**.

statement	**status**	**where it is verified**
$\omega'_\mu = \Lambda\omega_\mu\Lambda^{-1} - (\partial_\mu\Lambda)\Lambda^{-1}$	PROVED (above)	the boosted frame of Section 6.16: `python-scope.json` and `wolfram-scope.json`, check `boosted_frame_canonical_connection`
local spin covariance $\Omega'_\mu = R\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}$, $\gamma'^\mu = R\gamma^\mu R^{-1}$	PROVED (above, with Jacobi's formula quoted)	Notebook 06b checks both for all eight μ in the boosted frame, In [12]

The two scope reports lie in `Revision/theory/reports`; they were written by two programs that share no code (a sympy program and a WolframScript program).

6.16 A boost in the plane of $x4$ and $x0$

The boost. Mix the time $x4$ (time-like) and the hidden direction $x0$ (space-like) with the **rapidity** b:

$$e'^{(x4)} = \cosh b\, e^{(x4)} + \sinh b\, e^{(x0)}, \qquad e'^{(x0)} = \sinh b\, e^{(x4)} + \cosh b\, e^{(x0)},$$

and keep the other six frame directions. Here $\cosh b = (e^b + e^{-b})/2$ and $\sinh b = (e^b - e^{-b})/2$ (Chapter 5), with $\cosh^2 b - \sinh^2 b = 1$. The matrix $\Lambda(b)$ is the identity except in the rows and columns of $x4$ and $x0$, where it is $\begin{pmatrix}\cosh b & \sinh b\\ \sinh b & \cosh b\end{pmatrix}$. In these two rows and columns η is $\mathrm{diag}(-1,+1)$, and

$$\Lambda^T\eta\Lambda = \begin{pmatrix}\cosh b & \sinh b\\ \sinh b & \cosh b\end{pmatrix}\begin{pmatrix}-\cosh b & -\sinh b\\ \sinh b & \cosh b\end{pmatrix}$$

(Λ is symmetric, so $\Lambda^T = \Lambda$; and $\eta\Lambda$ is Λ with the sign of its first row changed)

$$= \begin{pmatrix}-\cosh^2 b + \sinh^2 b & -\cosh b\sinh b + \sinh b\cosh b\\ -\sinh b\cosh b + \cosh b\sinh b & -\sinh^2 b + \cosh^2 b\end{pmatrix} = \begin{pmatrix}-1 & 0\\ 0 & 1\end{pmatrix}$$

(multiply row by column; then $\cosh^2 - \sinh^2 = 1$). So $\Lambda(b)$ keeps η for every b, and $e' = \Lambda(b)e$ is a vielbein of the author's metric. A rotation moves a direction along a circle; a boost moves it along a **hyperbola**, keeping $t^2 - y^2$ (figure 06b.1). Now let the rapidity grow with the time,

$$b = \beta x_4 + b_0,$$

with two constants β (the rate) and b_0 (the starting value).

The spinor boost. Put $X = \gamma^{(x4)}\gamma^{(x0)}$. Then

$$X^2 = \gamma^{(x4)}\gamma^{(x0)}\gamma^{(x4)}\gamma^{(x0)} = -\gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)}\gamma^{(x0)} = -(-1)(+1) = 1$$

(exchange the two middle factors at the cost of a sign; then the Clifford relation twice). With $c = \cosh\frac{b}{2}$ and $s = \sinh\frac{b}{2}$ define $R = c - sX$; its inverse is $R^{-1} = c + sX$, because $(c - sX)(c + sX) = c^2 - s^2X^2 = c^2 - s^2 = 1$. Chapter 5 showed that R is the spinor boost $\exp(-\frac{b}{2}X)$ in the plane $(x4, x0)$. X anticommutes with $\gamma^{(x4)}$ and with $\gamma^{(x0)}$ (moving either through the product of two gammas meets one equal and one different factor: one sign) and commutes with the other six gammas (two signs). Hence $R^{-1}\gamma^c R = \gamma^c$ for the six other directions, and for $x4$:

$$R^{-1}\gamma^{(x4)} = (c + sX)\gamma^{(x4)} = \gamma^{(x4)}(c - sX) = \gamma^{(x4)}R$$

(move $\gamma^{(x4)}$ to the left through X at the cost of a sign), so

$$\begin{aligned} R^{-1}\gamma^{(x4)}R = \gamma^{(x4)}R^2 &= \gamma^{(x4)}\big(c^2 + s^2 - 2cs\,X\big) \\ &= \cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x4)}X = \cosh b\,\gamma^{(x4)} + \sinh b\,\gamma^{(x0)} \end{aligned}$$

(multiply out $(c - sX)^2$ with $X^2 = 1$; the half-angle formulas $c^2 + s^2 = \cosh b$ and $2cs = \sinh b$ of Chapter 5; and $\gamma^{(x4)}X = \gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)} = -\gamma^{(x0)}$). In the same way $R^{-1}\gamma^{(x0)}R = \gamma^{(x0)}(\cosh b - \sinh b\,X) = \sinh b\,\gamma^{(x4)} + \cosh b\,\gamma^{(x0)}$, using $\gamma^{(x0)}X = \gamma^{(x0)}\gamma^{(x4)}\gamma^{(x0)} = -\gamma^{(x4)}$. These are the two rows of $\Lambda(b)$: R **covers the boost.**

The contraction in the boosted frame. By Sections 6.15 and 6.8, $\sum_\mu \gamma'^\mu\Omega'_\mu = R\big(3H\gamma^{(x0)} + c\,\gamma^{(x4)}\big)R^{-1} - \sum_\mu \gamma'^\mu(\partial_\mu R)R^{-1}$ with $c = \frac{3}{4}b'_4/b_4$. The first term, line by line:

$$R\gamma^{(x0)} = (c - sX)\gamma^{(x0)} = \gamma^{(x0)}(c + sX) = \gamma^{(x0)}R^{-1},$$

$$R\gamma^{(x0)}R^{-1} = \gamma^{(x0)}(c + sX)^2 = \gamma^{(x0)}(\cosh b + \sinh b\,X) = \cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}$$

(the same steps as before). Call $v = \cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}$, and $w = R\gamma^{(x4)}R^{-1} = \cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)}$ (the same steps once more; it is computed again just below); the first term is $3H\,v + c\,w$. The second term: R depends on x_4 only, through b, with $db/dx_4 = \beta$, so only $\mu = x4$ contributes:

$$\partial_4 R = \tfrac{\beta}{2}(s - cX), \qquad (\partial_4 R)R^{-1} = \tfrac{\beta}{2}(s - cX)(c + sX) = \tfrac{\beta}{2}\big(sc + s^2X - c^2X - csX^2\big) = -\tfrac{\beta}{2}X$$

(the chain rule, $\frac{d}{db}\cosh\frac{b}{2} = \frac{1}{2}\sinh\frac{b}{2}$ and $\frac{d}{db}\sinh\frac{b}{2} = \frac{1}{2}\cosh\frac{b}{2}$; multiply out; $X^2 = 1$, so $sc - cs = 0$ and $(s^2 - c^2)X = -X$). Its trace is 0, because X is a product of two different gammas. With $\gamma'^{x4} = R\gamma^{(x4)}R^{-1} = \gamma^{(x4)}(\cosh b + \sinh b\,X) = \cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)}$ (the same steps; $f_4 = 1$):

$$\gamma'^{x4}(\partial_4 R)R^{-1} = -\tfrac{\beta}{2}\big(\cosh b\,\gamma^{(x4)}X - \sinh b\,\gamma^{(x0)}X\big) = -\tfrac{\beta}{2}\big(-\cosh b\,\gamma^{(x0)} + \sinh b\,\gamma^{(x4)}\big) = \tfrac{\beta}{2}\,v$$

($\gamma^{(x4)}X = -\gamma^{(x0)}$ and $\gamma^{(x0)}X = -\gamma^{(x4)}$). Subtracting:

$$\sum_\mu \gamma'^\mu \Omega'_\mu = 3H\,v + c\,w - \tfrac{\beta}{2}\,v = \frac{6H-\beta}{2}\big(\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}\big) + \frac{3}{4}\,\frac{b'_4}{b_4}\big(\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)}\big).$$

At $\beta = b_0 = 0$ (no boost) this is $3H\gamma^{(x0)} + \frac{3}{4}(b'_4/b_4)\gamma^{(x4)}$ again. For the rate $\beta = 6H$ the first term vanishes and only $\frac{3}{4}(b'_4/b_4)\,w$ remains: **for a constant b_4, in particular on the backgrounds $b_4 = 1$, the contraction vanishes identically at the rate $\beta = 6H$**, at every time and for every function a_4. For a changing b_4 no rate removes it: v and w are independent (their coefficients of $\gamma^{(x0)}$ and $\gamma^{(x4)}$ form a matrix with the determinant $\cosh^2 b - \sinh^2 b = 1$), so the sum is zero only when both coefficients $\frac{6H-\beta}{2}$ and $\frac{3}{4}b'_4/b_4$ vanish; the rescaling of Section 6.18 removes the whole contraction.

But the spinor connection does not vanish. For $\beta = 6H$: $\Omega'_{x4} = R\Omega_{x4}R^{-1} - (\partial_4 R)R^{-1} = 0 + 3H\,X = 3H\gamma^{(x4)}\gamma^{(x0)}$, which is not zero; $\Omega'_{x_i} = R\Omega_{x_i}R^{-1}$ and $\Omega'_{x_t} = R\Omega_{x_t}R^{-1}$ are not zero because R is invertible; only $\Omega'_{x0} = R\,0\,R^{-1} - 0 = 0$. In this frame, for a constant b_4, the field equation $\sum_\mu \gamma'^\mu D'_\mu \Psi' = (m + U')\Psi'$ contains no term $\gamma'^\mu \Omega'_\mu \Psi'$ at all, although the spinor connection is present in seven of the eight directions. The canonical spin connection of the boosted frame has 13 independent nonzero components (Notebook 06b, In [7], COMPUTED exactly).

statement	**status**	**where it is verified**
the boosted frame gives the author's metric; its canonical connection obeys the vielbein postulate	PROVED	Notebook 06b, In [7] (every b_4); at $b_4 = 1$: `python-scope.json` and `wolfram-scope.json`, checks `boosted_frame_reproduces_metric`, `boosted_frame_canonical_connection`
$\gamma'^\mu\Omega'_\mu = \frac{6H-\beta}{2}(\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}) + \frac{3}{4}\frac{b'_4}{b_4}(\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)})$	PROVED	Notebook 06b, In [8]; at $b_4 = 1$: both scope reports, check `boosted_frame_gammaOmega_formula`
for $\beta = 6H$ the contraction is $\frac{3}{4}(b'_4/b_4)\,w$, zero for a constant b_4; $\Omega'_\mu \neq 0$ for $\mu = x1, \ldots, x7$	PROVED	Notebook 06b, In [10]; at $b_4 = 1$: both scope reports, check `boosted_frame_gammaOmega_vanishes`
R covers the boost; the pieces $3Hv + \frac{3}{4}\frac{b'_4}{b_4}w$ and $\frac{\beta}{2}v$	PROVED (above)	Notebook 06b, In [12] (its own checks)

6.17 What no frame can remove: the curvature

Sections 6.5 and 6.16 taught the same lesson twice: the spin connection, and even its contraction, can look different, or vanish, in another frame. What does not depend on the frame is **curvature**.

The Riemann and Ricci tensors. Chapter 3 defined the Riemann tensor

$$R^{\rho}{}_{\sigma\mu\nu} = \partial_\mu \Gamma^{\rho}{}_{\nu\sigma} - \partial_\nu \Gamma^{\rho}{}_{\mu\sigma} + \sum_{\lambda} \left(\Gamma^{\rho}{}_{\mu\lambda}\Gamma^{\lambda}{}_{\nu\sigma} - \Gamma^{\rho}{}_{\nu\lambda}\Gamma^{\lambda}{}_{\mu\sigma}\right),$$

the Ricci tensor $R_{\sigma\nu} = \sum_\rho R^{\rho}{}_{\sigma\rho\nu}$, its mixed form $R^{\mu}{}_{\nu} = \sum_\sigma g^{\mu\sigma} R_{\sigma\nu}$ and the Ricci scalar $R = \sum_\mu R^{\mu}{}_{\mu}$. A space is flat exactly when the Riemann tensor vanishes everywhere.

One component by hand: $R^{x0}{}_{x0} = -6H^2$. Put $\sigma = \nu = x0$ in the Ricci tensor and use the Christoffel symbols of Section 6.3. Write $u = \partial_0 \ln \cos z$ and $v = \Gamma^{x0}{}_{x0x0}$, and note that $\sum_\rho \Gamma^{\rho}{}_{\rho 0} = \partial_0 \ln \sqrt{|g|} = \partial_0 \ln \left(b_4^{3/2} \cos z\right) = u$ (Section 6.9; b_4 does not depend on x_0). The four terms of $R_{00} = \sum_\rho R^{\rho}{}_{0\rho 0}$ are:

$$\sum_{\rho} \partial_\rho \Gamma^{\rho}{}_{00} = \partial_0 v$$

(the only nonzero $\Gamma^{\rho}{}_{00}$ is $\rho = x0$; $\Gamma^{x4}{}_{00} = 0$ because f_0 does not depend on x_4);

$$-\partial_0 \sum_{\rho} \Gamma^{\rho}{}_{\rho 0} = -\partial_0 u$$

(the contracted symbol);

$$\sum_{\rho,\lambda} \Gamma^{\rho}{}_{\rho\lambda}\Gamma^{\lambda}{}_{00} = u\, v$$

(only $\lambda = x0$ has $\Gamma^{\lambda}{}_{00} \neq 0$);

$$-\sum_{\rho,\lambda} \Gamma^{\rho}{}_{0\lambda}\Gamma^{\lambda}{}_{\rho 0} = -\left(6H^2 \cot^2 z + v^2\right)$$

($\Gamma^{\rho}{}_{0\lambda}$ is nonzero only for $\rho = \lambda$: $H \cot z$ for the six warped directions and v for $x0$; each gives its square). With $\partial_0 = 6H \frac{d}{dz}$ (the chain rule), $u = -6H \tan z$, $\partial_0 u = -36H^2/\cos^2 z$, $v = -6H(\tan z + \cot z)$ (Section 6.3, written with $\frac{1}{\sin z \cos z} = \tan z + \cot z$) and $\partial_0 v = -36H^2\left(\frac{1}{\cos^2 z} - \frac{1}{\sin^2 z}\right)$ (the derivatives of $\tan z$ and $\cot z$ are $1/\cos^2 z$ and $-1/\sin^2 z$):

$$R_{00} = -36H^2\Big(\frac{1}{\cos^2 z} - \frac{1}{\sin^2 z}\Big) + \frac{36H^2}{\cos^2 z} + 36H^2 \tan z(\tan z + \cot z) - 6H^2 \cot^2 z - 36H^2(\tan z + \cot z)^2$$

(insert the four terms; $uv = (-6H \tan z)(-6H)(\tan z + \cot z)$)

$$= \frac{36H^2}{\sin^2 z} + 36H^2 \tan^2 z + 36H^2 - 6H^2 \cot^2 z - 36H^2\left(\tan^2 z + 2 + \cot^2 z\right)$$

(the terms with $1/\cos^2 z$ cancel; $\tan z \cot z = 1$; multiply out the square)

$$= \frac{36H^2}{\sin^2 z} - 36H^2 - 42H^2\cot^2 z = 36H^2(1+\cot^2 z) - 36H^2 - 42H^2\cot^2 z = -6H^2\cot^2 z$$

(the $\tan^2 z$ terms cancel; $1/\sin^2 z = 1+\cot^2 z$, which is $\sin^2+\cos^2 = 1$ divided by $\sin^2 z$). Finally $R^{x0}{}_{x0} = g^{00}R_{00} = \tan^2 z\cdot(-6H^2\cot^2 z) = -6H^2$ (the inverse metric is diagonal, so only R_{00} enters). It is negative for every $H>0$ and all functions a_4 and b_4: **the author's metric is never flat.**

A second component by hand: $R^{x4}{}_{x4}$. This is the component that carries the deflation, so it is worth the four lines. Put $\sigma=\nu=x4$ in the same four terms. Every $\Gamma^{\rho}{}_{44}$ vanishes (case (a) of Section 6.3 gives $\partial_4 \ln f_4 = 0$ because $f_4 = 1$, and case (c) gives a multiple of $\partial_\rho f_4 = 0$), so the first and the third term vanish:

$$\sum_{\rho} \partial_\rho \Gamma^{\rho}{}_{44} = 0, \qquad \sum_{\rho,\lambda} \Gamma^{\rho}{}_{\rho\lambda}\Gamma^{\lambda}{}_{44} = 0.$$

The contracted symbol is $\sum_\rho \Gamma^{\rho}{}_{\rho 4} = 3a_4' + 3\alpha_t + 0 + \partial_4 \ln f_0 = 3a_4' + 3\alpha_t$ (three 3-space directions, three extra times, $\Gamma^{x4}{}_{x4x4} = 0$, and $f_0 = \cot z$ does not depend on x_4; equivalently $\partial_4 \ln\sqrt{|g|} = \partial_4 \ln(b_4^{3/2}\cos z) = \frac{3}{2}b_4'/b_4$, Section 6.9), so the second term is

$$-\partial_4 \sum_{\rho} \Gamma^{\rho}{}_{\rho 4} = -3a_4'' - 3\alpha_t',$$

which vanishes for a constant b_4, where $\alpha_t = -a_4'$.

In the fourth term look at the symbols $\Gamma^{\rho}{}_{4\lambda}$ one case at a time. For $\rho = x4$ the symbol is $\partial_\lambda \ln f_4 = 0$ (case (a) for $\lambda = x4$, case (b) otherwise; $f_4 = 1$). For $\lambda = x4$ and $\rho \neq x4$ it is $\Gamma^{\rho}{}_{44} = 0$ (shown above). For ρ, λ and $x4$ all different it is 0 (case (d)). What is left is $\rho = \lambda \neq x4$, case (b): $\Gamma^{\rho}{}_{4\rho} = \partial_4 \ln f_\rho$, which is a_4' for a 3-space direction, α_t for an extra time and $\partial_4 \ln f_0 = 0$ for $x0$. So only the six terms with $\rho = \lambda$ a 3-space direction or an extra time survive, and each is the square of its symbol, because for $\lambda = \rho$ the second factor $\Gamma^{\lambda}{}_{\rho 4} = \Gamma^{\rho}{}_{\rho 4}$ equals the first, $\Gamma^{\rho}{}_{4\rho}$ (the symbols are symmetric in their two lower indices):

$$-\sum_{\rho,\lambda} \Gamma^{\rho}{}_{4\lambda}\Gamma^{\lambda}{}_{\rho 4} = -\big(3a_4'^2 + 3\alpha_t^2\big).$$

So $R_{44} = -3(a_4'^2 + a_4'') - 3(\alpha_t^2 + \alpha_t')$ (the sum of the four terms), and

$$R^{x4}{}_{x4} = g^{44}R_{44} = 3\big(a_4'^2 + a_4''\big) + 3\big(\alpha_t^2 + \alpha_t'\big),$$

with $\alpha_t = -a_4' + b_4'/(2b_4)$ and $\alpha_t' = -a_4'' + b_4''/(2b_4) - b_4'^2/(2b_4^2)$; multiplied out, $R^{x4}{}_{x4} = 6a_4'^2 - 3a_4'b_4'/b_4 + 3b_4''/(2b_4) - 3b_4'^2/(4b_4^2)$, the record's value. At $b_4 = 1$, $\alpha_t = -a_4'$ and $\alpha_t' = -a_4''$, so $R^{x4}{}_{x4} = 3a_4'^2 + 3a_4'' + 3a_4'^2 - 3a_4'' = 6a_4'^2$. The 3-space directions and the extra times enter with opposite rates $\pm a_4'$, but each enters squared, so here they add instead of cancelling.

All Ricci components. Chapter 3 computed all components by hand, from the curvatures of the 28 coordinate planes, in Section 3.25. With b_4 there are ten nonzero mixed components (record `metric-b4.json`, key `ricciMixed`): the eight on the diagonal,

$$R^{x0}{}_{x0} = -6H^2, \quad R^{x_i}{}_{x_i} = a_4'' - 6H^2 + \frac{3a_4'b_4'}{2b_4}, \quad R^{x_t}{}_{x_t} = -a_4'' - 6H^2 - \frac{3a_4'b_4'}{2b_4} + \frac{b_4''}{2b_4} + \frac{b_4'^2}{4b_4^2},$$

and $R^{x4}{}_{x4}$ above, and the two off it, $R^{x4}{}_{x0} = \frac{3}{2}H\cot z\, b_4'/b_4$ and $R^{x0}{}_{x4} = -\frac{3}{2}H\tan z\, b_4'/b_4$, which vanish for a constant b_4. The Ricci scalar is $R = 6a_4'^2 - 3a_4'b_4'/b_4 + 3b_4''/b_4 - 42H^2$. At $b_4 = 1$ these are the values of the record `field-theory.json`,

$$R^{x0}{}_{x0} = -6H^2, \qquad R^{x_i}{}_{x_i} = a_4'' - 6H^2, \qquad R^{x4}{}_{x4} = 6a_4'^2, \qquad R^{x_t}{}_{x_t} = -a_4'' - 6H^2,$$

and $R = -6H^2 + 3(a_4'' - 6H^2) + 6a_4'^2 + 3(-a_4'' - 6H^2) = 6(a_4'^2 - 7H^2)$ (add the eight components; the a_4'' terms cancel). The component $R^{x4}{}_{x4} = 6a_4'^2$ is curvature made entirely by the inflation of 3-space and the deflation of the extra times, **although $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)}$ does not contain a_4 at all**: the contraction is blind to the deflation, the metric is not. For every b_4 the contraction contains b_4 but still no a_4 (Section 6.8).

The curvature of the spinor connection. The commutator of two covariant derivatives of a spinor, line by line:

$$D_\mu D_\nu\Psi = \partial_\mu(\partial_\nu\Psi + \Omega_\nu\Psi) + \Omega_\mu(\partial_\nu\Psi + \Omega_\nu\Psi) = \partial_\mu\partial_\nu\Psi + (\partial_\mu\Omega_\nu)\Psi + \Omega_\nu\partial_\mu\Psi + \Omega_\mu\partial_\nu\Psi + \Omega_\mu\Omega_\nu\Psi$$

(the definition twice; the product rule). Subtract the same with μ and ν exchanged: $\partial_\mu\partial_\nu\Psi$ and the two middle terms are symmetric and cancel, and

$$(D_\mu D_\nu - D_\nu D_\mu)\Psi = F_{\mu\nu}\Psi, \qquad F_{\mu\nu} = \partial_\mu\Omega_\nu - \partial_\nu\Omega_\mu + [\Omega_\mu, \Omega_\nu].$$

Theorem (quoted, and checked exactly for this metric). $F_{\mu\nu} = \frac{1}{2}\sum_{a,b} R_{ab\mu\nu}S^{ab}$, where $R_{ab\mu\nu}$ is the Riemann tensor with its first two indices turned into frame indices (for the diagonal vielbein $R_{ab\mu\nu} = \eta_{aa}f_a R^a{}_{b\mu\nu}/f_b$). The general proof is a standard but long computation that this book does not reproduce; for the author's metric the identity is PROVED by exact computation in all 28 coordinate planes (`wolfram-field-theory-b4.json`, check `spin_curvature_equals_Riemann`, with b_4; at $b_4 = 1$ `python-field-theory.json`, check `spinor_curvature_equals_riemann`, and `wolfram-field-theory.json`, check `spin_curvature_equals_Riemann`; Notebook 06b, In [15]). The spinor curvature is the Riemann curvature acting on spinors.

It cannot be removed by a change of frame. By Section 6.15, $D'_\mu(R\Psi) = R\,D_\mu\Psi$ for every spinor. Apply it twice, the second time to the spinor $D_\nu\Psi$: $D'_\mu D'_\nu(R\Psi) = D'_\mu(R\,D_\nu\Psi) = R\,D_\mu D_\nu\Psi$. Subtracting the same with μ and ν exchanged gives $F'_{\mu\nu}R\Psi = R\,F_{\mu\nu}\Psi$ for every Ψ, so

$$F'_{\mu\nu} = R\,F_{\mu\nu}\,R^{-1}.$$

An invertible R can change the entries of $F_{\mu\nu}$ but cannot make a nonzero matrix zero. If some frame had $\Omega'_\mu = 0$ for all μ, its $F'_{\mu\nu}$ would vanish, hence $F_{\mu\nu}$ in every frame, hence the Riemann tensor, which contradicts $R^{x0}{}_{x0} = -6H^2$. **No frame makes the spin connection vanish.** In the diagonal vielbein 27 of the 28 planes have $F_{\mu\nu} \neq 0$ at the sample point; the plane $(x0, x4)$ has

$F_{x0\,x4} = 0$ exactly, for every b_4, because $\Omega_{x0} = \Omega_{x4} = 0$ make every term of its formula vanish (Notebook 06b, In [16], and figure 5). In the boosted frame with $\beta = 6H$, where the contraction vanished for a constant b_4, $F'_{x0\,x1} = RF_{x0\,x1}R^{-1}$ is not zero (In [15]).

statement	**status**	**where it is verified**
the ten nonzero Ricci components with b_4 (two off the diagonal) and R; at $b_4 = 1$ the eight diagonal ones and $R = 6(a_4'^2 - 7H^2)$	PROVED ($R^{x0}{}_{x0}$ and $R^{x4}{}_{x4}$ by hand above, all in Section 3.25)	`metric-b4.json`, keys `ricciMixed`, `ricciScalar`; `wolfram-field-theory-b4.json`, checks `ricci_equals_metric_b4_set`, `ricci_reduces_at_b4_1`; at $b_4 = 1$: `field-theory.json`, formulas `ricci_mixed_diagonal`, `ricci_scalar`; `wolfram-field-theory.json`, checks `ricci_mixed_components`, `ricci_scalar`; Notebook 06b, In [13]
never flat for $H > 0$: $R^{x0}{}_{x0} = -6H^2$ for every b_4	PROVED	`metric-b4.json`, key `ricciMixed`; at $b_4 = 1$: `wolfram-field-theory.json`, check `never_flat_for_H_positive`; `python-field-theory.json`, check `curvature_nonzero_flat_only_formally`
$\gamma^\mu\Omega_\mu$ contains no a_4, while $R^{x4}{}_{x4} = 6a_4'^2$ at $b_4 = 1$	PROVED	Notebook 06b, In [13]; at $b_4 = 1$: both scope reports, check `gammaOmega_blind_to_the_deflation`
$F_{\mu\nu} = \frac{1}{2}R_{ab\mu\nu}S^{ab}$ in all 28 planes	PROVED for this metric (general theorem quoted)	`wolfram-field-theory-b4.json`, check `spin_curvature_equals_Riemann`; at $b_4 = 1$: `python-field-theory.json`, check `spinor_curvature_equals_riemann`; `wolfram-field-theory.json`, check `spin_curvature_equals_Riemann`
$F' = RFR^{-1}$; in the boosted frame $F'_{x0\,x1} \neq 0$	PROVED	Notebook 06b, In [15]; at $b_4 = 1$: both scope reports, check `boosted_frame_curvature_nonzero`

6.18 Changing the field variable: the rescaling $\Psi = \sin^{-1/2}(z)\, b_4^{-3/4}\, \chi$

A frame change is one way to remove the term $\gamma^\mu\Omega_\mu$; a change of the field variable is another. Keep the diagonal vielbein, and write the field as a known function times a new field,

$$\Psi = w\,\chi, \qquad w = \sin^{-1/2}(z)\,b_4^{-3/4}.$$

On the backgrounds $b_4 = 1$ the weight is $w = \sin^{-1/2} z$, the weight of the Revision record.

The derivation. By the product rule, $\partial_\mu(w\chi) = (\partial_\mu w)\chi + w\,\partial_\mu\chi$, so

$$\sum_\mu \gamma^\mu D_\mu \Psi = w \sum_\mu \gamma^\mu \partial_\mu \chi + \Big(\sum_\mu \gamma^\mu \partial_\mu w + w \sum_\mu \gamma^\mu \Omega_\mu\Big)\chi$$

(w is a number at each point and commutes with every matrix). Only x_0 and x_4 appear in w, with $\gamma^{x0} = \tan z\,\gamma^{(x0)}$ and $\gamma^{x4} = \gamma^{(x4)}$, and $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ (Section 6.8), so the bracket is

$$\big(\tan z\,\partial_0 w + 3H\,w\big)\gamma^{(x0)} + \Big(\partial_4 w + \frac{3}{4}\,\frac{b_4'}{b_4}\,w\Big)\gamma^{(x4)}.$$

Now

$$\partial_0 \sin^{-1/2} z = -\tfrac{1}{2}\sin^{-3/2} z \cdot \cos z \cdot 6H = -3H\cos z\,\sin^{-3/2} z$$

(the power rule and the chain rule), and

$$\tan z\,\partial_0 w = \frac{\sin z}{\cos z}\big(-3H\cos z\,\sin^{-3/2} z\big)b_4^{-3/4} = -3H\sin^{-1/2}(z)\,b_4^{-3/4} = -3H\,w$$

(cancel $\cos z$; $\sin z \cdot \sin^{-3/2} z = \sin^{-1/2} z$). In the same way $\partial_4 b_4^{-3/4} = -\frac{3}{4}b_4^{-7/4}b_4'$ (the power rule and the chain rule) gives $\partial_4 w = -\frac{3}{4}(b_4'/b_4)\,w$. Both brackets are zero, $(-3Hw + 3Hw)\gamma^{(x0)} + (-\frac{3}{4}\frac{b_4'}{b_4}w + \frac{3}{4}\frac{b_4'}{b_4}w)\gamma^{(x4)} = 0$:

$$\sum_\mu \gamma^\mu D_\mu(w\chi) = w\sum_\mu \gamma^\mu \partial_\mu \chi.$$

The rescaled field equation. The bilinear is $S[\Psi] = \Psi^\dagger C \Psi$; w is real and positive, so $S[w\chi] = w^2\chi^\dagger C\chi = S[\chi]/(b_4^{3/2}\sin z)$. For the potential $U = \frac{\lambda}{2}S^2$, $U'(S) = \lambda S$, and the field equation $\sum_\mu \gamma^\mu D_\mu\Psi = (m + \lambda S[\Psi])\Psi$ becomes, after division by w,

$$\sum_\mu \gamma^\mu \partial_\mu \chi = \Big(m + \frac{\lambda\,S[\chi]}{b_4^{3/2}\sin z}\Big)\chi.$$

For $U = 0$ ($\lambda = 0$) the term $\gamma^\mu\Omega_\mu$ is removed completely. For $U \neq 0$ it reappears in another form: as the coupling $\lambda S[\chi]/(b_4^{3/2}\sin z)$, which depends on the hidden coordinate (and, through b_4, on the time). What remains of gravity in the equation of χ are the vielbein factors $1/f_a$ in front of every derivative.

statement	status	where it is verified
$\sum_\mu \gamma^\mu D_\mu(w\chi) = w\sum_\mu \gamma^\mu \partial_\mu \chi$ for $w = \sin^{-1/2}(z)\, b_4^{-3/4}$ and any 16 functions χ	PROVED	Notebook 06b, In [17]; at $b_4 = 1$ ($w = \sin^{-1/2} z$): `python-scope.json` and `wolfram-scope.json`, check `rescaling_remove s_the_connection_term`
$S[w\chi] = S[\chi]/(b_4^{3/2}\sin z)$; the rescaled equation with $U = \frac{\lambda}{2}S^2$	PROVED	Notebook 06b, In [17]; at $b_4 = 1$: both scope reports, check `rescaled_equatio n_quadratic_potential`

6.19 Example: Notebook 06b

Notebook 06b checks Sections 6.15 to 6.18 with exact algebra, for arbitrary functions $a_4(x_4)$ and $b_4(x_4)$. It rebuilds the tools of Notebook 06a (so that it runs on its own), recomputes the diagonal vielbein and its contraction $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ as a control, builds the boost matrix and checks that it keeps η, and draws the boosted frame on its hyperbolas. It then computes the canonical spin connection of the boosted frame with a symbolic rate β and starting value b_0, checks the vielbein postulate, the formula $\frac{6H-\beta}{2}(\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}) + \frac{3}{4}\frac{b_4'}{b_4}(\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)})$ and its vanishing at $\beta = 6H$ for a constant b_4, and explains it with the explicit spinor boost R. It computes the Riemann tensor, the Ricci tensor (with its two components off the diagonal) and the spinor curvature in all 28 planes, and finally the rescaling. It compares every result that the Revision records also contain with the records (the record of the metric with b_4 where it has the result, and the two scope reports of $b_4 = 1$), draws six figures, needs no Rust, runs in about two minutes and ends with the line ALL 26 CHECKS PASSED (notebook 06b).

6.20 How to run Notebook 06b

Step 1. What this notebook does and what it needs.

Notebook 06b (What depends on the frame: boosts, spin covariance, curvature, rescaling) is the file `Revision/textbook/notebooks/06b_frame_dependence.ipynb` of the repository Dirac_claude. For the author's metric with the functions a4(x4) and b4(x4) it boosts the diagonal vielbein in the (x4, x0) plane with a time-dependent rapidity and computes exactly, with sympy, the new spin connection and the new contraction gamma^mu Omega_mu = ((6 H - beta)/2)(cosh b gamma^(x0) - sinh b gamma^(x4)) + (3/4)(b4'/b4)(cosh b gamma^(x4) - sinh b gamma^(x0)), which vanishes identically for the rapidity rate beta = 6 H when b4 is constant; it verifies the local spin covariance of the connection with an explicit spinor boost, computes the Riemann and Ricci tensors and the curvature of the spinor connection, which no frame can remove, shows that the rescaling Psi = sin^(-1/2)(z) b4^(-3/4) chi removes the whole term gamma^mu Omega_mu, checks every result against the Revision records and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It

does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 06b_frame_dependence.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 06b_frame_dependence.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 06b_frame_dependence.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/06b.captions.json`
- `Revision/textbook/figures/06b_1_boost_hyperbolas.png`
- `Revision/textbook/figures/06b_2_boosted_contraction.png`
- `Revision/textbook/figures/06b_3_boosted_connection_heat_maps.png`
- `Revision/textbook/figures/06b_4_ricci_components.png`
- `Revision/textbook/figures/06b_5_spinor_curvature_norms.png`
- `Revision/textbook/figures/06b_6_rescaling.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 26 CHECKS PASSED (notebook 06b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/06b_frame_dependence.ipynb` inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 06b_frame_dependence.ipynb
python -m nbconvert --execute --inplace 06b_frame_dependence.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for Revision/algebra/gammas.json or for a report: the notebook reads Revision records from the repository; open it inside the folder Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a single downloaded notebook file is not enough).
- a cell runs for several minutes: the exact algebra of sympy with hyperbolic functions and two unknown functions is slow on old computers; the whole notebook needs about two minutes on a 2024 laptop, and up to about four minutes while other programs use the processor. Wait, or close the other programs.

6.21 Notebook 06b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/06b_frame_dependence.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The previous notebook of this chapter (06a) found $\gamma^\mu\Omega_\mu =
3H\gamma^{(x0)} + \frac34(b_4'/b_4)\gamma^{(x4)}$ for the author's metric in the
DIAGONAL vielbein ($3H\gamma^{(x0)}$ when $b_4$ is constant). A vielbein is a
choice: at every point any other set of eight perpendicular unit directions
describes the same metric. This notebook asks which results depend on that choice
and which do not. It

- builds a second vielbein of the same metric, the diagonal one **boosted** in the
  plane of the time $x4$ and the hidden direction $x0$ by a rapidity
  $b = \beta x_4 + b_0$ that grows with the time;
- computes its canonical spin connection and finds exactly
  $\gamma'^\mu\Omega'_\mu = \frac{6H - \beta}{2}\big(\cosh b\,\gamma^{(x0)} -
  \sinh b\,\gamma^{(x4)}\big) + \frac34\frac{b_4'}{b_4}\big(\cosh b\,
  \gamma^{(x4)} - \sinh b\,\gamma^{(x0)}\big)$, which VANISHES for $\beta = 6H$
  when $b_4$ is constant, although $\Omega'_\mu$ itself does not vanish;
- explains the formula with the **local spin covariance** of the connection,
  using the explicit spinor boost $R = \cosh\frac{b}{2} -
  \sinh\frac{b}{2}\,\gamma^{(x4)}\gamma^{(x0)}$;
- computes the Riemann and Ricci tensors and the curvature
  $F_{\mu\nu}$ of the spinor connection, checks $F_{\mu\nu} =
  \frac12 R_{ab\mu\nu}S^{ab}$ in all 28 coordinate planes, and shows that this
  curvature, which no choice of frame can remove, is nonzero for every $H > 0$;
- shows that the rescaled field $\chi = \sin^{1/2}(z)\,b_4^{3/4}\,\Psi$ obeys an
  equation without the term $\gamma^\mu\Omega_\mu$;
- checks every result against the Revision records (with $b_4$ where the record has
  it, and at $b_4 = 1$) and draws six teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 06b (What depends on the frame: boosts, spin covariance, curvature, rescaling)
is the file ‘Revision/textbook/notebooks/06b_frame_dependence.ipynb‘ of the repository
Dirac_claude. For the author's metric with the functions a4(x4) and b4(x4) it boosts the
diagonal vielbein in the (x4, x0) plane with a time-dependent rapidity and computes
exactly, with sympy, the new spin connection and the new contraction gammaˆmu Omega_mu =
((6 H - beta)/2)(cosh b gammaˆ(x0) - sinh b gammaˆ(x4)) + (3/4)(b4'/b4)(cosh b gammaˆ(x4)
- sinh b gammaˆ(x0)), which vanishes identically for the rapidity rate beta = 6 H when b4
is constant; it verifies the local spin covariance of the connection with an explicit
spinor boost, computes the Riemann and Ricci tensors and the curvature of the spinor
connection, which no frame can remove, shows that the rescaling Psi = sinˆ(-1/2)(z)
b4ˆ(-3/4) chi removes the whole term gammaˆmu Omega_mu, checks every result against the
Revision records and draws six teaching plots. It needs a computer with Windows 11, macOS
or Linux, an internet connection for the installation, the program Git, and Python 3.12
or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the
program that shows and runs notebooks. It does not need Rust.
```

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home

folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 06b_frame_dependence.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 06b_frame_dependence.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 06b_frame_dependence.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/06b.captions.json‘
- ‘Revision/textbook/figures/06b_1_boost_hyperbolas.png‘

- ‘Revision/textbook/figures/06b_2_boosted_contraction.png‘
- ‘Revision/textbook/figures/06b_3_boosted_connection_heat_maps.png‘
- ‘Revision/textbook/figures/06b_4_ricci_components.png‘
- ‘Revision/textbook/figures/06b_5_spinor_curvature_norms.png‘
- ‘Revision/textbook/figures/06b_6_rescaling.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 26 CHECKS PASSED (notebook 06b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/06b_frame_dependence.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
      python -m jupyterlab 06b_frame_dependence.ipynb
      python -m nbconvert --execute --inplace 06b_frame_dependence.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for Revision/algebra/gammas.json or for a report: the notebook
  reads Revision records from the repository; open it inside the folder
  Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a single
  downloaded notebook file is not enough).
- a cell runs for several minutes: the exact algebra of sympy with hyperbolic functions
  and two unknown functions is slow on old computers; the whole notebook needs about two
  minutes on a 2024 laptop, and up to about four minutes while other programs use the
  processor. Wait, or close the other programs.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 06b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 06b (What depends on the frame: boosts, spin covariance, curvature, rescaling)
# is the file Revision/textbook/notebooks/06b_frame_dependence.ipynb of the repository
# Dirac_claude. For the author's metric with the functions a4(x4) and b4(x4) it boosts
# the diagonal vielbein in the (x4, x0) plane with a time-dependent rapidity and computes
# exactly, with sympy, the new spin connection and the new contraction gamma^mu Omega_mu
# = ((6 H - beta)/2)(cosh b gamma^(x0) - sinh b gamma^(x4)) + (3/4)(b4'/b4)(cosh b
# gamma^(x4) - sinh b gamma^(x0)), which vanishes identically for the rapidity rate beta
# = 6 H when b4 is constant; it verifies the local spin covariance of the connection with
# an explicit spinor boost, computes the Riemann and Ricci tensors and the curvature of
# the spinor connection, which no frame can remove, shows that the rescaling Psi =
# sin^(-1/2)(z) b4^(-3/4) chi removes the whole term gamma^mu Omega_mu, checks every
# result against the Revision records and draws six teaching plots. It needs a computer
# with Windows 11, macOS or Linux, an internet connection for the installation, the
# program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
```

```
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
```

```
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
```

```
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 06b_frame_dependence.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 06b_frame_dependence.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 06b_frame_dependence.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/06b.captions.json
# - Revision/textbook/figures/06b_1_boost_hyperbolas.png
# - Revision/textbook/figures/06b_2_boosted_contraction.png
# - Revision/textbook/figures/06b_3_boosted_connection_heat_maps.png
# - Revision/textbook/figures/06b_4_ricci_components.png
# - Revision/textbook/figures/06b_5_spinor_curvature_norms.png
# - Revision/textbook/figures/06b_6_rescaling.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
```

```
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 26 CHECKS PASSED (notebook 06b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/06b_frame_dependence.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 06b_frame_dependence.ipynb
#     python -m nbconvert --execute --inplace 06b_frame_dependence.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json or for a report: the notebook
#   reads Revision records from the repository; open it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a
#   single downloaded notebook file is not enough).
```

```
# - a cell runs for several minutes: the exact algebra of sympy with hyperbolic functions
#   and two unknown functions is slow on old computers; the whole notebook needs about
#   two minutes on a 2024 laptop, and up to about four minutes while other programs use
#   the processor. Wait, or close the other programs.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "06b"  # this notebook: chapter 06, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
```

```
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
```

```
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 06b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Vielbein** $e^a{}_\mu$: eight perpendicular unit directions at every point
  (frame index $a$, coordinate index $\mu$), with $g_{\mu\nu} = \sum_a \eta_{aa}
  e^a{}_\mu e^a{}_\nu$; $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the
  order $x0, \dots, x7$. In Python lists the directions have the positions 0 to 7
  in the same order, so `x[0]` is $x0$ and `x[4]` is $x4$.
- **Frame rotation** (local Lorentz transformation): a new vielbein
  $e'^a{}_\mu = \sum_b \Lambda^a{}_b(x)\,e^b{}_\mu$ with a matrix $\Lambda(x)$
  that keeps $\eta$: $\Lambda^T\eta\Lambda = \eta$. It describes the same metric.
- **Boost**: a frame rotation that mixes one space-like and one time-like
  direction. In the plane of the time $x4$ (time-like) and the hidden direction
  $x0$ (space-like) it is $e'^{(x4)} = \cosh b\, e^{(x4)} + \sinh b\, e^{(x0)}$,
  $e'^{(x0)} = \sinh b\, e^{(x4)} + \cosh b\, e^{(x0)}$.
- **Rapidity** $b$: the "angle" of a boost. **Hyperbolic functions**:
  $\cosh b = (e^b + e^{-b})/2$ and $\sinh b = (e^b - e^{-b})/2$; they satisfy
  $\cosh^2 b - \sinh^2 b = 1$, just as $\cos^2 + \sin^2 = 1$ for rotations.
- **Spin connection** $\omega_{\mu ab}$, **spinor connection** $\Omega_\mu =
  \frac12\sum_{a,b}\omega_{\mu ab}S^{ab}$ with $S^{ab} = \frac14[\gamma^a,
  \gamma^b]$, **curved gammas** $\gamma^\mu = \sum_a e_a{}^\mu\gamma^a$,
  **covariant derivative** $D_\mu = \partial_\mu + \Omega_\mu$: as in Notebook
  06a. A prime marks the quantities of the boosted vielbein.
- **Spinor boost** $R(x)$: a 16 x 16 matrix that turns spinors when the frame is
  boosted; it **covers** $\Lambda$ when $R^{-1}\gamma^a R = \sum_b \Lambda^a{}_b
  \gamma^b$ for every $a$.
- **Riemann tensor** $R^\rho{}_{\sigma\mu\nu}$: the measure of curvature, built
  from the Christoffel symbols and their derivatives; it is zero everywhere only
  for a flat space. **Ricci tensor** $R_{\sigma\nu} = \sum_\rho
```

```
  R^\rho{}_{\sigma\rho\nu}$, mixed form $R^\mu{}_\nu = \sum_\sigma g^{\mu\sigma}
  R_{\sigma\nu}$, **Ricci scalar** $R = \sum_\mu R^\mu{}_\mu$.
- **Curvature of the spinor connection** $F_{\mu\nu} = \partial_\mu\Omega_\nu -
  \partial_\nu\Omega_\mu + [\Omega_\mu, \Omega_\nu]$: the 16 x 16 matrix with
  $D_\mu D_\nu\Psi - D_\nu D_\mu\Psi = F_{\mu\nu}\Psi$.
- **Frame-dependent**: a quantity whose value changes when the vielbein is
  changed (such as $\gamma^\mu\Omega_\mu$). **Frame-independent**: a statement
  that holds in every vielbein (such as "$F_{\mu\nu}$ is not zero").
- **Rescaling**: writing the field as a known function times a new field,
  $\Psi = w(x)\,\chi$, and deriving the equation that $\chi$ obeys.
- **Exact** check: sympy proves that an expression is zero for ALL values of its
  symbols. **Heat map**: a picture of a matrix, red positive, blue negative,
  white zero.
```

Text cell:

```
## 4. The physical and mathematical situation

The metric is the author's, with $z = 6Hx_0$ in $(0, \pi/2)$, $H > 0$ and two
arbitrary functions $a_4(x_4)$ and $b_4(x_4) > 0$:

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4}\sin^{1/3}z\ (\times 3),\ -1,\
-e^{-2a_4}b_4\sin^{1/3}z\ (\times 3)\big).$$

The diagonal vielbein has the factors $f = (\cot z,\ e^{a_4}s\ (\times 3),\ 1,\
e^{-a_4}b_4^{1/2}s\ (\times 3))$, $s = \sin^{1/6}z$. On the backgrounds of this
book $b_4 = 1$. The canonical spin connection of ANY vielbein $e$ is
$\omega_\mu{}^a{}_b = \sum_\nu e^a{}_\nu (\partial_\mu e_b{}^\nu +
\sum_\lambda\Gamma^\nu{}_{\mu\lambda}e_b{}^\lambda)$.

**The theorem behind this notebook (local spin covariance).** If $e' = \Lambda e$
and the spinor matrix $R(x)$ covers $\Lambda(x)$, then

$$\Omega'_\mu = R\,\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}, \qquad
\gamma'^\mu = R\,\gamma^\mu R^{-1}, \qquad F'_{\mu\nu} = R\,F_{\mu\nu}R^{-1}.$$

Multiplying the first two and summing over $\mu$ gives

$$\gamma'^\mu\Omega'_\mu = R\,(\gamma^\mu\Omega_\mu)\,R^{-1} -
\sum_\mu\gamma'^\mu(\partial_\mu R)R^{-1}.$$

The second term can cancel the first: the contraction $\gamma^\mu\Omega_\mu$ is
frame-dependent. The curvature only gets turned by $R$, so if $F_{\mu\nu} \ne 0$
in one frame, it is nonzero in every frame: no frame removes the spinor
connection itself.

**Status.** Every statement checked below is PROVED exactly (sympy, symbolic $H$,
$a_4$, $b_4$, $\beta$, $b_0$). The scope records (`python-scope.json`,
`wolfram-scope.json`) state these results for the metric at $b_4 = 1$; the notebook
reproduces them there and computes the general case itself. This is the record's
own correction of the non-triviality statement: the value of
$\gamma^\mu\Omega_\mu$ belongs to the diagonal vielbein; what is
frame-independent is the curvature.
```

Text cell:

```
## 5. The tools: sympy, the author's gammas and the general formulas

This notebook repeats the tools of Notebook 06a, so that it runs on its own. The
next cell reads the author's gammas and the matrix $C$ from
`Revision/algebra/gammas.json`, checks the 64 Clifford relations, builds the 28
matrices $S^{ab}$, and defines the exact zero test `is_zero` (first a fast route
through exponential functions, then sympy's `simplify`; it never calls a nonzero
expression zero) and `record_passed`, which reads a Revision report and tells
whether a check is recorded as passed.

The first lines of the next cell change one setting of Jupyter: printed text is
sent to the screen when a cell has finished (or just before a figure is shown),
not every 0.2 seconds. This changes no result; it makes the printed text arrive in
the same pieces in every run, which the book's checking tool needs.
```

In [2]:

```
import sys  # the Python system module (here: the stream of printed text)

# Jupyter sends printed text to the screen in pieces, about every 0.2 seconds.  The
# book's checking tool reads each PASS line together with its "reproduces" line, so
# the pieces must be the same in every run: send the printed text of a cell in one
# piece when the cell ends (or just before a figure), at the latest after 600 s.
if hasattr(sys.stdout, "flush_interval"):  # true inside Jupyter only
    sys.stdout.flush_interval = 600.0
import numpy as np  # decimal numbers for the plots
import sympy as sp  # exact algebra and calculus with symbols
from sympy.parsing.mathematica import parse_mathematica  # reads Wolfram notation

fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
NAMES = fixture["coordinates"]  # ["x0", ..., "x7"]
ETA = fixture["eta"]  # +1 space-like, -1 time-like
gamma = [sp.Matrix(rows) for rows in fixture["gamma"]]  # gamma[a] = gamma^(xa)
C = sp.Matrix(fixture["C"])  # the author's C (Dirac adjoint Psibar = Psi^dagger C)
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
check(all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
          == (2 * ETA[a] if a == b else 0) * I16
          for a in range(8) for b in range(8)),
      "the 64 Clifford relations of the author's gammas")
S = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
     for a in range(8)]


def is_zero(expr):
    """True when expr is exactly zero for all values of its symbols."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    if sp.cancel(sp.expand(expr.rewrite(sp.exp))) == 0:  # the fast route
```

```
        return True
    return sp.simplify(expr) == 0  # the slower general route


def matrix_is_zero(matrix):
    return all(is_zero(entry) for entry in matrix)


def record_passed(path, name):
    """True when the Revision report at path records the check name as passed."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper() == "PASS"
    raise KeyError(f"{path} has no check {name}")


SCOPE_PY = "Revision/theory/reports/python-scope.json"
SCOPE_WL = "Revision/theory/reports/wolfram-scope.json"
REPORT_PY = "Revision/theory/reports/python-field-theory.json"
REPORT_WL = "Revision/theory/reports/wolfram-field-theory.json"
REPORT_WL_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
THEORY_FILE = "Revision/theory/field-theory.json"
METRIC_B4_FILE = "Revision/metric_b4/results/metric-b4.json"
```

Out [2]:

```
PASS the 64 Clifford relations of the author's gammas
```

Text cell:

```
The next cell defines the four general formulas as Python functions, exactly as
in Notebook 06a: ‘christoffel(g, coords)‘ gives $\Gamma^l{}_{mn}$;
‘spin_connection(e, Gam, coords, eta)‘ gives $\omega_{\mu ab}$ for any vielbein
matrix ‘e‘ (row = frame index, column = coordinate; its inverse ‘E = e.inv()‘ has
‘E[nu, b]‘ $= e_b{}^\nu$); ‘spinor_connection(omega, S)‘ gives $\Omega_\mu =
\sum_{a<b}\omega_{\mu ab}S^{ab}$; ‘curved_gammas(e, gammas)‘ gives $\gamma^\mu =
\sum_a e_a{}^\mu\gamma^a$. For the boosted vielbein the entries contain
$\cosh$ and $\sinh$; ‘spin_connection‘ simplifies every entry, which is the
slowest step of the notebook.
```

In [3]:

```
def christoffel(g, coords):
    """Gam[l][m][n] = (1/2) sum_r g^lr (d_m g_rn + d_n g_rm - d_r g_mn)."""
    n = len(coords)
    g_inv = g.inv()
    Gam = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for l in range(n):
        for m in range(n):
            for k in range(m, n):
```

```
                value = sum(g_inv[l, r] * (sp.diff(g[r, k], coords[m])
                                           + sp.diff(g[r, m], coords[k])
                                           - sp.diff(g[m, k], coords[r]))
                            for r in range(n) if g_inv[l, r] != 0) / 2
                Gam[l][m][k] = Gam[l][k][m] = sp.simplify(value)
    return Gam


def spin_connection(e, Gam, coords, eta):
    """omega[mu][a][b] = eta_aa sum_nu e^a_nu (d_mu e_b^nu + Gam^nu_mu,lam e_b^lam)."""
    n = len(coords)
    E = e.inv()  # E[nu, b] = e_b^nu
    omega = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for mu in range(n):
        for a in range(n):
            for b in range(n):
                mixed = 0
                for nu in range(n):
                    if e[a, nu] == 0:
                        continue
                    mixed += e[a, nu] * (sp.diff(E[nu, b], coords[mu]) + sum(
                        Gam[nu][mu][lam] * E[lam, b] for lam in range(n)))
                omega[mu][a][b] = sp.simplify(eta[a] * mixed)
    return omega


def spinor_connection(omega, S):
    """Omega[mu] = sum_{a<b} omega_{mu ab} S^ab."""
    n = len(omega)
    size = S[0][0].shape[0]
    return [sum((omega[mu][a][b] * S[a][b] for a in range(n)
                 for b in range(a + 1, n) if omega[mu][a][b] != 0),
                sp.zeros(size, size)) for mu in range(n)]


def curved_gammas(e, gammas):
    """gamma^mu = sum_a e_a^mu gamma^a."""
    E = e.inv()
    return [sum((E[mu, a] * gammas[a] for a in range(len(gammas))),
                sp.zeros(*gammas[0].shape)) for mu in range(len(gammas))]


say("defined: christoffel, spin_connection, spinor_connection, curved_gammas")
```

Out [3]:

```
defined: christoffel, spin_connection, spinor_connection, curved_gammas
```

Text cell:

```
## 6. The diagonal vielbein again (the control)
```

```
The next cell makes the symbols of the metric ($x0, \dots, x7$, $H > 0$, the
functions $a_4(x_4)$ and $b_4(x_4) > 0$, $z = 6Hx_0$), the diagonal vielbein, the
Christoffel symbols, the spin connection, $\Omega_\mu$ and the curved gammas, and
checks the result of Notebook 06a, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} +
\frac34(b_4'/b_4)\gamma^{(x4)}$, which at $b_4 = 1$ is the record's
$3H\gamma^{(x0)}$. This is the CONTROL of the boosted computation: the same code,
with no boost, must give the known value. The helper `at_b4_1` puts $b_4 = 1$
into an expression and evaluates the derivatives of the constant 1 to 0.
```

In [4]:

```
x = sp.symbols("x0:8", real=True)  # x[0] = x0, ..., x[4] = x4, ..., x[7] = x7
x0, x4 = x[0], x[4]
H = sp.Symbol("H", positive=True)
a4 = sp.Function("a4")(x4)  # any function of the time
b4 = sp.Function("b4", positive=True)(x4)  # any positive function of the time
a4p, b4p = sp.Derivative(a4, x4), sp.Derivative(b4, x4)
z = 6 * H * x0
s = sp.sin(z) ** sp.Rational(1, 6)
f = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
     + [sp.exp(-a4) * sp.sqrt(b4) * s] * 3)  # f_0, ..., f_7
g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # the author's metric
e = sp.diag(*f)  # the diagonal vielbein


def at_b4_1(expr):
    """expr at b4 = 1; doit() evaluates the derivatives of the constant 1 to 0."""
    return sp.sympify(expr).subs({b4: 1}).doit()


Gam = christoffel(g, x)
omega = spin_connection(e, Gam, x, ETA)
Omega = spinor_connection(omega, S)
gup = curved_gammas(e, gamma)
slash = sum((gup[mu] * Omega[mu] for mu in range(8)), Z16)
c_b4 = sp.Rational(3, 4) * b4p / b4  # the coefficient (3/4) b4'/b4
check(matrix_is_zero(slash - 3 * H * gamma[0] - c_b4 * gamma[4])
      and matrix_is_zero(at_b4_1(slash) - 3 * H * gamma[0]),
      "control: the diagonal vielbein gives gamma^mu Omega_mu = 3 H gamma^(x0) + "
      "(3/4)(b4'/b4) gamma^(x4), 3 H gamma^(x0) at b4 = 1",
      record=f"{SCOPE_PY}, check rescaling_removes_the_connection_term (b4 = 1)")
```

Out [4]:

```
PASS control: the diagonal vielbein gives gamma^mu Omega_mu = 3 H gamma^(x0) +
    (3/4)(b4'/b4) gamma^(x4), 3 H gamma^(x0) at b4 = 1
     reproduces Revision/theory/reports/python-scope.json, check
    rescaling_removes_the_connection_term (b4 = 1)
```

Text cell:

```
## 7. A boost of the frame in the plane of $x4$ and $x0$

The next cell writes the boost matrix $\Lambda(b)$: the identity, except in the
rows and columns of $x0$ (position 0) and $x4$ (position 4), where it is
$\begin{pmatrix}\cosh b & \sinh b\\ \sinh b & \cosh b\end{pmatrix}$. It checks
that $\Lambda^T\eta\Lambda = \eta$ for EVERY rapidity $b$; in the $(x0, x4)$
block this is $\cosh^2 b - \sinh^2 b = +1$, $-\cosh^2 b + \sinh^2 b = -1$ and
$\cosh b\sinh b - \sinh b\cosh b = 0$. It also checks that the inverse boost is
the boost with $-b$.
```

In [5]:

```
def boost(b):
    """The 8 x 8 boost matrix Lambda(b) in the plane of x0 (index 0) and x4 (4)."""
    L = sp.eye(8)
    L[0, 0], L[0, 4], L[4, 0], L[4, 4] = sp.cosh(b), sp.sinh(b), sp.sinh(b), sp.cosh(b)
    return L


b_symbol = sp.Symbol("b", real=True)  # a rapidity
eta_matrix = sp.diag(*ETA)
L_test = boost(b_symbol)
check(matrix_is_zero(L_test.T * eta_matrix * L_test - eta_matrix),
      "Lambda(b)^T eta Lambda(b) = eta for every rapidity b")
check(matrix_is_zero(L_test * boost(-b_symbol) - sp.eye(8)),
      "the inverse of the boost with rapidity b is the boost with -b")
```

Out [5]:

```
PASS Lambda(b)^T eta Lambda(b) = eta for every rapidity b
PASS the inverse of the boost with rapidity b is the boost with -b
```

Text cell:

```
The next cell draws what a boost does to the two frame directions $e^{(x4)}$ and
$e^{(x0)}$, drawn with their $x4$ component horizontally and their $x0$ component
vertically, for the rapidities $b = 0, 0.5, 1$. A rotation would move them along
a circle; a boost moves them along the two **unit hyperbolas** $t^2 - y^2 = 1$
(where every vector has $\eta$-length $-1$, like the time direction) and
$y^2 - t^2 = 1$ ($\eta$-length $+1$, like the hidden direction). Both arrows tilt
towards the diagonal line $t = y$ (the "light line"), and they stay perpendicular
in the sense of $\eta$.
```

In [6]:

```
fig, ax = plt.subplots(figsize=(6.4, 6.0))
rapidity = np.linspace(-1.4, 1.4, 200)
ax.plot(np.cosh(rapidity), np.sinh(rapidity), color="tab:blue", linewidth=1.0,
        label="unit hyperbola of the time direction: $t^2 - y^2 = 1$")
```

```
ax.plot(np.sinh(rapidity), np.cosh(rapidity), color="tab:orange", linewidth=1.0,
        label="unit hyperbola of the hidden direction: $y^2 - t^2 = 1$")
ax.plot([0, 2.2], [0, 2.2], ":", color="gray", label="light line $t = y$")
for b_value, alpha in ((0.0, 1.0), (0.5, 0.7), (1.0, 0.45)):
    c_b, s_b = np.cosh(b_value), np.sinh(b_value)
    ax.annotate("", xy=(c_b, s_b), xytext=(0, 0), arrowprops=dict(
        arrowstyle="-|>", color="tab:blue", alpha=alpha, linewidth=1.6))
    ax.annotate("", xy=(s_b, c_b), xytext=(0, 0), arrowprops=dict(
        arrowstyle="-|>", color="tab:orange", alpha=alpha, linewidth=1.6))
    ax.text(c_b + 0.04, s_b - 0.12, f"$b = {b_value:g}$", fontsize=8,
            color="tab:blue")
    ax.text(s_b - 0.30, c_b + 0.06, f"$b = {b_value:g}$", fontsize=8,
            color="tab:orange")
ax.set_xlim(-0.4, 2.3)
ax.set_ylim(-0.25, 2.3)
ax.set_aspect("equal")
ax.set_xlabel("component along $e^{(x4)}$ (time), $t$")
ax.set_ylabel("component along $e^{(x0)}$ (hidden direction), $y$")
ax.set_title("A boost moves the frame along hyperbolas")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "boost_hyperbolas",
            "A boost of the frame in the plane of the time $x4$ and the hidden "
            "direction $x0$. The boosted frame directions $e'^{(x4)} = \\cosh b\\, "
            "e^{(x4)} + \\sinh b\\, e^{(x0)}$ (blue arrows) and $e'^{(x0)} = "
            "\\sinh b\\, e^{(x4)} + \\cosh b\\, e^{(x0)}$ (orange arrows) for the "
            "rapidities $b = "
            "0, 0.5, 1$ (fainter for larger $b$), drawn with their $x4$ component $t$ "
            "horizontally and their $x0$ component $y$ vertically (pure numbers). "
            "They move along the unit hyperbolas $t^2 - y^2 = 1$ and $y^2 - t^2 = 1$ "
            "and tilt symmetrically towards the light line $t = y$ (dotted).")
```

Out [6]:

```
Figure 06b.1 saved as Revision/textbook/figures/06b_1_boost_hyperbolas.png
```

Text cell:

```
## 8. The boosted vielbein and its canonical spin connection

Now the rapidity depends on the time: $b = \beta x_4 + b_0$ with two constants
$\beta$ (the rate) and $b_0$ (the starting value). The boosted vielbein is the
matrix product $e' = \Lambda(b)\,e$. The next cell checks that it describes the
author's metric (all 64 entries of $e'^T\eta e'$), computes its canonical spin
connection, and checks the vielbein postulate (512 equations) and the
antisymmetry $\omega'_{\mu ab} = -\omega'_{\mu ba}$, for every $b_4$. At $b_4 = 1$
these are the record's checks `boosted_frame_reproduces_metric` and
`boosted_frame_canonical_connection`.
```

In [7]:

```
beta, b0 = sp.symbols("beta b0", real=True)  # the rate and the start of the rapidity
b = beta * x4 + b0  # the rapidity grows with the time
```

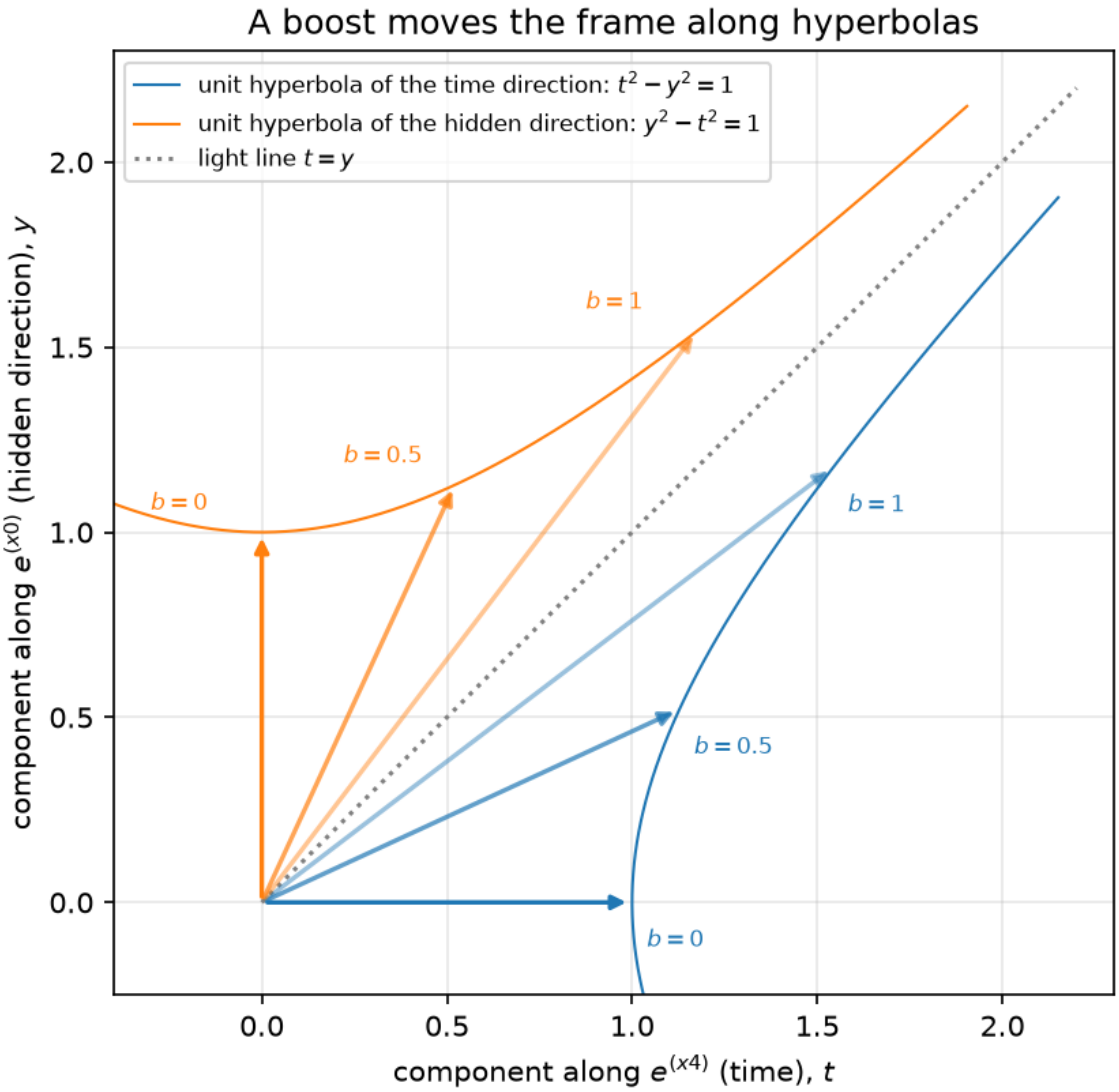

Figure 192: A boost of the frame in the plane of the time $x4$ and the hidden direction $x0$. The boosted frame directions $e'^{(x4)} = \cosh b\, e^{(x4)} + \sinh b\, e^{(x0)}$ (blue arrows) and $e'^{(x0)} = \sinh b\, e^{(x4)} + \cosh b\, e^{(x0)}$ (orange arrows) for the rapidities $b = 0, 0.5, 1$ (fainter for larger b), drawn with their $x4$ component t horizontally and their $x0$ component y vertically (pure numbers). They move along the unit hyperbolas $t^2 - y^2 = 1$ and $y^2 - t^2 = 1$ and tilt symmetrically towards the light line $t = y$ (dotted). (Notebook 06b, figure 1.)

```
L = boost(b)
e_boosted = L * e  # e'^a_mu = sum_c Lambda^a_c e^c_mu
check(matrix_is_zero(e_boosted.T * eta_matrix * e_boosted - g),
      "the boosted vielbein gives the author's metric (64 entries)",
      record=f"{SCOPE_PY}, check boosted_frame_reproduces_metric (b4 = 1)")
omega_b = spin_connection(e_boosted, Gam, x, ETA)  # omega'_{mu ab}
postulate = [sp.diff(e_boosted[a, nu], x[mu])
             - sum(Gam[lam][mu][nu] * e_boosted[a, lam] for lam in range(8))
             + sum(ETA[a] * omega_b[mu][a][c] * e_boosted[c, nu] for c in range(8))
             for mu in range(8) for a in range(8) for nu in range(8)]
antisymmetric = all(is_zero(omega_b[mu][a][c] + omega_b[mu][c][a])
                    for mu in range(8) for a in range(8) for c in range(8))
check(all(is_zero(v) for v in postulate) and antisymmetric,
      "boosted vielbein: vielbein postulate (512) and antisymmetry of omega'",
      record=f"{SCOPE_PY}, check boosted_frame_canonical_connection (b4 = 1)")
Omega_b = spinor_connection(omega_b, S)  # Omega'_mu
gup_b = curved_gammas(e_boosted, gamma)  # gamma'^mu
count_b = sum(1 for mu in range(8) for a in range(8) for c in range(a + 1, 8)
              if omega_b[mu][a][c] != 0)
say(f"nonzero components omega'_mu ab with a < b: {count_b}")
```

Out [7]:

```
PASS the boosted vielbein gives the author's metric (64 entries)
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_reproduces_metric (b4 = 1)
PASS boosted vielbein: vielbein postulate (512) and antisymmetry of omega'
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_canonical_connection (b4 = 1)
nonzero components omega'_mu ab with a < b: 13
```

Text cell:

```
## 9. The contraction in the boosted vielbein

The next cell computes $\gamma'^\mu\Omega'_\mu$ (summed over $\mu$) and compares it,
exactly, for symbolic $a_4$, $b_4$, $H$, $\beta$ and $b_0$, with

$$\gamma'^\mu\Omega'_\mu = \frac{6H - \beta}{2}\,v + \frac34\,\frac{b_4'}{b_4}\,
w, \qquad v = \cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)},\quad
w = \cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)} .$$

At $b_4 = 1$ the second term is absent, and the formula is the record's. At $b = 0$
(no boost) it gives back $3H\gamma^{(x0)} + \frac34(b_4'/b_4)\gamma^{(x4)}$. The
coefficient $(6H - \beta)/2$ already shows the surprise: for the rate
$\beta = 6H$ the first term is zero.
```

In [8]:

```
slash_b = sum((gup_b[mu] * Omega_b[mu] for mu in range(8)), Z16)
v = sp.cosh(b) * gamma[0] - sp.sinh(b) * gamma[4]  # R gamma^(x0) R^-1 (Section 11)
```

```
w = sp.cosh(b) * gamma[4] - sp.sinh(b) * gamma[0]  # R gamma^(x4) R^-1
formula = (6 * H - beta) / 2 * v + c_b4 * w
check(matrix_is_zero(slash_b - formula),
      "gamma'^mu Omega'_mu = ((6H - beta)/2) v + (3/4)(b4'/b4) w",
      record=f"{SCOPE_PY}, check boosted_frame_gammaOmega_formula (b4 = 1)")
check(matrix_is_zero(formula.subs({beta: 0, b0: 0}) - 3 * H * gamma[0]
                     - c_b4 * gamma[4])
      and matrix_is_zero(at_b4_1(formula) - (6 * H - beta) / 2 * v),
      "with no boost the formula is 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); at "
      "b4 = 1 it is ((6H - beta)/2) v")
```

Out [8]:

```
PASS gamma'^mu Omega'_mu = ((6H - beta)/2) v + (3/4)(b4'/b4) w
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_gammaOmega_formula (b4 = 1)
PASS with no boost the formula is 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); at b4 = 1 it
    is ((6H - beta)/2) v
```

Text cell:

```
The next cell draws the two coefficients of the formula at $b_4 = 1$ (the
background of this book), the coefficient of $\gamma^{(x0)}$,
$\frac{6H-\beta}{2}\cosh b$, and the coefficient of $\gamma^{(x4)}$,
$-\frac{6H-\beta}{2}\sinh b$, as functions of the time $x_4$, for $b_0 = 0$,
$H = 1$ and the four rates $\beta = 0, 3, 6, 9$. The curves are made from the
formula that the previous cell proved. For $\beta = 0$ they are the constants $3$
and $0$ of the diagonal vielbein; for $\beta = 6$ both are zero at every time; for
$\beta = 9$ they change sign.
```

In [9]:

```
times = np.linspace(-0.25, 0.25, 101)
t_symbol = sp.Symbol("t", real=True)
coefficient_x0 = (6 * H - beta) / 2 * sp.cosh(b)  # the coefficient of gamma^(x0)
coefficient_x4 = -(6 * H - beta) / 2 * sp.sinh(b)  # the coefficient of gamma^(x4)
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for rate, style in ((0, "-"), (3, "--"), (6, "-."), (9, ":")):
    values = {H: 1, beta: rate, b0: 0, x4: t_symbol}
    c0 = sp.lambdify(t_symbol, coefficient_x0.subs(values))
    c4 = sp.lambdify(t_symbol, coefficient_x4.subs(values))
    ax_left.plot(times, np.broadcast_to(c0(times), times.shape), style,
                 label=f"$\\beta = {rate}H$" if rate else "no boost")
    ax_right.plot(times, np.broadcast_to(c4(times), times.shape), style,
                  label=f"$\\beta = {rate}H$" if rate else "no boost")
ax_left.set_title("coefficient of $\\gamma^{(x0)}$")
ax_right.set_title("coefficient of $\\gamma^{(x4)}$")
for ax in (ax_left, ax_right):
    ax.set_xlabel("time $x_4$ (units $1/H$), with $b = \\beta x_4$")
    ax.legend(fontsize=8)
ax_left.set_ylabel("coefficient in $\\gamma'^\\mu\\Omega'_\\mu$ (units of $H$)")
```

```
save_figure(fig, "boosted_contraction",
            "The contraction $\\gamma'^\\mu\\Omega'_\\mu$ in the boosted vielbein with "
            "rapidity $b = \\beta x_4$, $H = 1$ and $b_4 = 1$: the coefficient of "
            "$\\gamma^{(x0)}$ (left) and of $\\gamma^{(x4)}$ (right) versus the time "
            "$x_4$ in units $1/H$, for the rates $\\beta = 0, 3H, 6H, 9H$. For "
            "$\\beta = 0$ (no boost) the coefficients are $3$ and $0$, the diagonal "
            "value $3H\\gamma^{(x0)}$ (on the right the zero lines of $\\beta = 0$ "
            "and $\\beta = 6H$ lie on top of each other); for $\\beta = 6H$ both "
            "coefficients vanish at every time; "
            "the same metric, a different frame, a different value. A changing "
            "$b_4$ would add $\\frac34(b_4'/b_4)(\\cosh b\\,\\gamma^{(x4)} - "
            "\\sinh b\\,\\gamma^{(x0)})$.")
```

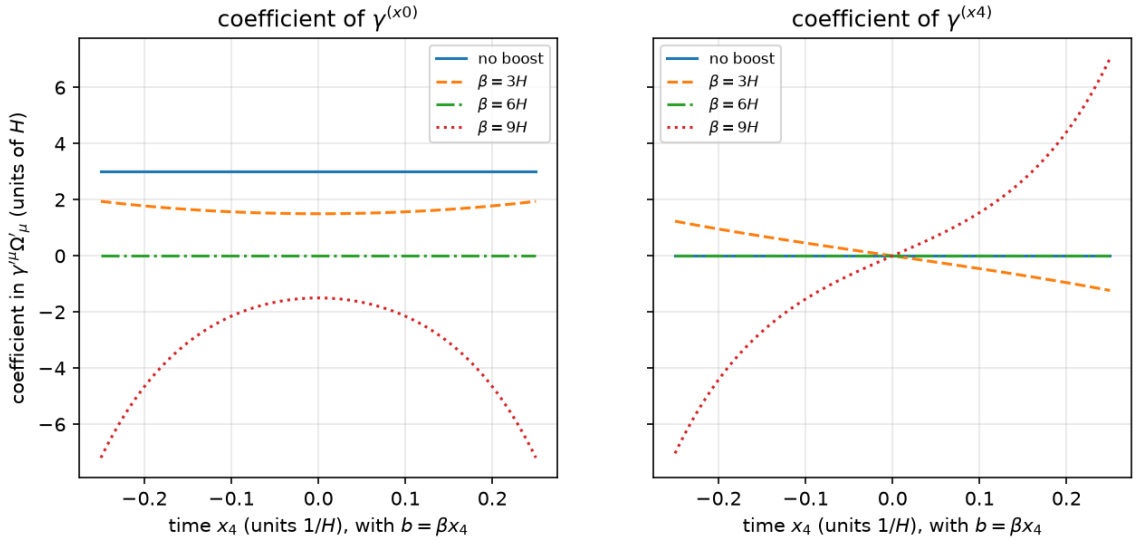

Figure 193: The contraction $\gamma'^{\mu}\Omega'_{\mu}$ in the boosted vielbein with rapidity $b = \beta x_4$, $H = 1$ and $b_4 = 1$: the coefficient of $\gamma^{(x0)}$ (left) and of $\gamma^{(x4)}$ (right) versus the time x_4 in units $1/H$, for the rates $\beta = 0, 3H, 6H, 9H$. For $\beta = 0$ (no boost) the coefficients are 3 and 0, the diagonal value $3H\gamma^{(x0)}$ (on the right the zero lines of $\beta = 0$ and $\beta = 6H$ lie on top of each other); for $\beta = 6H$ both coefficients vanish at every time; the same metric, a different frame, a different value. A changing b_4 would add $\frac{3}{4}(b_4'/b_4)(\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)})$. (Notebook 06b, figure 2.)

Out [9]:

```
Figure 06b.2 saved as Revision/textbook/figures/06b_2_boosted_contraction.png
```

Text cell:

```
## 10. The rate $\beta = 6H$: the contraction vanishes for constant $b_4$

The next cell puts $\beta = 6H$ into the boosted spin connection (substituting
after the computation is allowed, because the computation was done for every
$\beta$). It checks that $\gamma'^\mu\Omega'_\mu = \frac34(b_4'/b_4)\,w$ is all that
remains, so that the contraction is zero identically, for every $H > 0$ and every
```

```
$a_4$, exactly when $b_4$ is constant (the record's statement at $b_4 = 1$); and
that $\Omega'_\mu$ is nonzero for $\mu = x1, \dots, x7$ and zero only for
$\mu = x0$. So for a constant $b_4$ the field equation $\gamma'^\mu D'_\mu\Psi' =
(m + U')\Psi'$ in this vielbein has no term $\gamma'^\mu\Omega'_\mu\Psi'$ at all,
although the spinor connection is everywhere. (The matrices $v$ and $w$ are
independent, so no rate $\beta$ removes the term $\frac34(b_4'/b_4)w$ of a changing
$b_4$; the rescaling of Section 14 removes the whole term.)
```

In [10]:

```
six_H = {beta: 6 * H}
slash_6H = slash_b.subs(six_H)
Omega_6H = [M.subs(six_H) for M in Omega_b]
nonzero = [NAMES[mu] for mu in range(8) if not matrix_is_zero(Omega_6H[mu])]
say(f"Omega'_mu is nonzero for mu = {nonzero}")
check(matrix_is_zero(slash_6H - c_b4 * w.subs(six_H))
      and matrix_is_zero(at_b4_1(slash_6H))
      and nonzero == NAMES[1:],
      "beta = 6H: gamma'^mu Omega'_mu = (3/4)(b4'/b4) w (zero for constant b4), "
      "Omega'_mu nonzero for x1 ... x7",
      record=f"{SCOPE_PY}, check boosted_frame_gammaOmega_vanishes (b4 = 1)")
```

Out [10]:

```
Omega'_mu is nonzero for mu = ['x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7']
PASS beta = 6H: gamma'^mu Omega'_mu = (3/4)(b4'/b4) w (zero for constant b4), Omega'_mu
    nonzero for x1 ... x7
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_gammaOmega_vanishes (b4 = 1)
```

Text cell:

```
The next cell draws three 16 x 16 matrices of the boosted vielbein with
$\beta = 6H$ at the sample point $H = 1$, $b_0 = 0$, $x_4 = 1/6$ (so $b = 1$),
$z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$ on the background $b_4 = 1$: the spinor
connection $\Omega'_{x4}$ (in the diagonal vielbein $\Omega_{x4} = 0$; here it is
not, because the frame turns with time), $\Omega'_{x1}$, and the total contraction
$\gamma'^\mu\Omega'_\mu$, which is the zero matrix (white).
```

In [11]:

```
def at_sample(expr):
    """The decimal value of expr at H = 1, b0 = 0, x4 = 1/6, z = pi/4, a4 = 1/2,
    a4' = 1/2, a4'' = 0, b4 = 1 (derivatives replaced first, then a4 and b4, then
    x4)."""
    expr = at_b4_1(expr).subs(sp.Derivative(a4, (x4, 2)), 0)
    expr = expr.subs(sp.Derivative(a4, x4), sp.Rational(1, 2))
    expr = expr.subs(a4, sp.Rational(1, 2))
    expr = expr.subs({H: 1, b0: 0, x4: sp.Rational(1, 6), x0: sp.pi / 24})
```

```
    return float(sp.N(expr))


def sample_table(matrix):
    return np.array([[at_sample(v) for v in row] for row in matrix.tolist()])


panels = [("$\\Omega'_{x4}$", Omega_6H[4]), ("$\\Omega'_{x1}$", Omega_6H[1]),
          ("$\\gamma'^\\mu\\Omega'_\\mu$ (all zero)", slash_6H)]
tables = [sample_table(M) for _, M in panels]
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.6))
for ax, (title, _), table in zip(axes, panels, tables):
    limit = max(np.abs(table).max(), 1.0)  # each panel its own colour scale
    image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.8)
save_figure(fig, "boosted_connection_heat_maps",
            "The boosted vielbein with the rate $\\beta = 6H$ at the sample point "
            "$H = 1$, $b_0 = 0$, $x_4 = 1/6$ (rapidity $b = 1$), $z = \\pi/4$, $a_4 = "
            "1/2$, $a_4' = 1/2$, $b_4 = 1$, as 16 x 16 heat maps (red positive, blue "
            "negative, white zero; rows and columns are the spinor components; each "
            "panel has its own colour scale). Left: "
            "$\\Omega'_{x4}$, nonzero because the frame turns with time; middle: "
            "$\\Omega'_{x1}$; right: the contraction $\\gamma'^\\mu\\Omega'_\\mu$, the "
            "zero matrix. The spinor connection is present, its contraction is not.")
check(np.abs(tables[2]).max() < 1e-12 and np.abs(tables[0]).max() > 0.1,
      "at the sample point Omega'_x4 is nonzero and the contraction is zero")
```

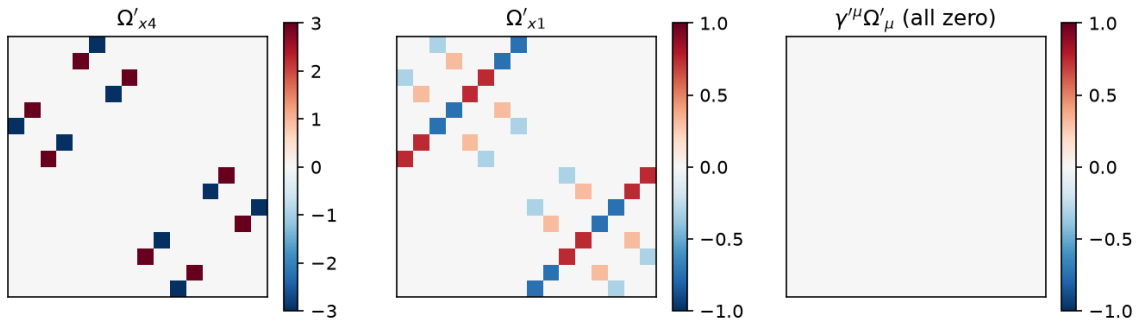

Figure 194: The boosted vielbein with the rate $\beta = 6H$ at the sample point $H = 1$, $b_0 = 0$, $x_4 = 1/6$ (rapidity $b = 1$), $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$, as 16 x 16 heat maps (red positive, blue negative, white zero; rows and columns are the spinor components; each panel has its own colour scale). Left: Ω'_{x4}, nonzero because the frame turns with time; middle: Ω'_{x1}; right: the contraction $\gamma'^\mu\Omega'_\mu$, the zero matrix. The spinor connection is present, its contraction is not. (Notebook 06b, figure 3.)

Out [11]:

```
Figure 06b.3 saved as Revision/textbook/figures/06b_3_boosted_connection_heat_maps.png
```

```
PASS at the sample point Omega'_x4 is nonzero and the contraction is zero
```

Text cell:

```
## 11. Why: the local spin covariance, with an explicit spinor boost

The next cell takes the 16 x 16 matrix $X = \gamma^{(x4)}\gamma^{(x0)}$. Since
$X^2 = -\gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)}\gamma^{(x0)} = -(-1)(+1) = 1$,
the spinor boost $R = \cosh\frac b2 - \sinh\frac b2\,X$ has the inverse
$R^{-1} = \cosh\frac b2 + \sinh\frac b2\,X$ (multiply them: $\cosh^2 -
\sinh^2 X^2 = 1$). The cell checks, exactly:

1. $R^{-1}\gamma^a R = \sum_c\Lambda^a{}_c\gamma^c$ for all eight $a$: $R$
   covers the boost $\Lambda(b)$;
2. $\gamma'^\mu = R\,\gamma^\mu R^{-1}$ and $\Omega'_\mu = R\,\Omega_\mu R^{-1} -
   (\partial_\mu R)R^{-1}$ for all eight $\mu$ (the covariance theorem);
3. the two pieces of the formula of Section 9: $R\,\gamma^{(x0)}R^{-1} = v$ and
   $R\,\gamma^{(x4)}R^{-1} = w$, so that the turned contraction is
   $R(3H\gamma^{(x0)} + \frac34\frac{b_4'}{b_4}\gamma^{(x4)})R^{-1} = 3Hv +
   \frac34\frac{b_4'}{b_4}w$, and the extra term $\sum_\mu\gamma'^\mu(\partial_\mu
   R)R^{-1} = \frac{\beta}{2}\,v$ (only $\mu = x4$ contributes, because $R$ depends
   on $x_4$ only). Their difference is the formula of Section 9.
```

In [12]:

```
X = gamma[4] * gamma[0]  # gamma^(x4) gamma^(x0)
check(X * X == I16, "X = gamma^(x4) gamma^(x0) squares to the identity")
R = sp.cosh(b / 2) * I16 - sp.sinh(b / 2) * X  # the spinor boost
R_inv = sp.cosh(b / 2) * I16 + sp.sinh(b / 2) * X
check(matrix_is_zero(R * R_inv - I16), "R R^-1 = 1")
covers = all(matrix_is_zero(R_inv * gamma[a] * R
                            - sum((L[a, c] * gamma[c] for c in range(8)), Z16))
             for a in range(8))
check(covers, "R^-1 gamma^a R = Lambda^a_c gamma^c: R covers the boost")
gammas_turn = all(matrix_is_zero(gup_b[mu] - R * gup[mu] * R_inv) for mu in range(8))
connection_rule = all(matrix_is_zero(Omega_b[mu] - (R * Omega[mu] * R_inv
                                                    - R.diff(x[mu]) * R_inv))
                      for mu in range(8))
check(gammas_turn and connection_rule,
      "gamma'^mu = R gamma^mu R^-1 and Omega'_mu = R Omega_mu R^-1 - (d_mu R) R^-1")
turned = R * slash * R_inv
extra = sum((gup_b[mu] * R.diff(x[mu]) * R_inv for mu in range(8)), Z16)
check(matrix_is_zero(R * gamma[0] * R_inv - v)
      and matrix_is_zero(R * gamma[4] * R_inv - w)
      and matrix_is_zero(turned - 3 * H * v - c_b4 * w)
      and matrix_is_zero(extra - beta / 2 * v),
      "R g0 R^-1 = v, R g4 R^-1 = w; turned term 3H v + (3/4)(b4'/b4) w; extra term "
      "(beta/2) v")
```

Out [12]:

```
PASS X = gamma^(x4) gamma^(x0) squares to the identity
PASS R R^-1 = 1
PASS R^-1 gamma^a R = Lambda^a_c gamma^c: R covers the boost
PASS gamma'^mu = R gamma^mu R^-1 and Omega'_mu = R Omega_mu R^-1 - (d_mu R) R^-1
PASS R g0 R^-1 = v, R g4 R^-1 = w; turned term 3H v + (3/4)(b4'/b4) w; extra term
    (beta/2) v
```

Text cell:

```
## 12. The curvature: the Riemann and Ricci tensors

The next cell computes the Riemann tensor of the author's metric from the
Christoffel symbols with the convention $R^\rho{}_{\sigma\mu\nu} =
\partial_\mu\Gamma^\rho{}_{\nu\sigma} - \partial_\nu\Gamma^\rho{}_{\mu\sigma} +
\sum_\lambda(\Gamma^\rho{}_{\mu\lambda}\Gamma^\lambda{}_{\nu\sigma} -
\Gamma^\rho{}_{\nu\lambda}\Gamma^\lambda{}_{\mu\sigma})$ (it keeps the components
with $\mu < \nu$; the others follow from the antisymmetry in $\mu, \nu$), then the
mixed Ricci components $R^\mu{}_\nu$ and the Ricci scalar. It compares them with
the record of the metric with $b_4$ (‘ricciMixed‘ and ‘ricciScalar‘ of
‘Revision/metric_b4/results/metric-b4.json‘, which also lists the two components
off the diagonal, $R^{x4}{}_{x0}$ and $R^{x0}{}_{x4}$, proportional to $b_4'$) and,
at $b_4 = 1$, with the record ‘field-theory.json‘ (‘ricci_mixed_diagonal‘,
‘ricci_scalar‘), whose values are

$$R^{x0}{}_{x0} = -6H^2,\quad R^{x1}{}_{x1} = R^{x2}{}_{x2} = R^{x3}{}_{x3} =
a_4'' - 6H^2,\quad R^{x4}{}_{x4} = 6a_4'^2,\quad R^{x5}{}_{x5} = R^{x6}{}_{x6} =
R^{x7}{}_{x7} = -a_4'' - 6H^2,$$

and $R = 6(a_4'^2 - 7H^2)$. The component $R^{x0}{}_{x0} = -6H^2$ holds for every
$b_4$ and is negative for every $H > 0$ and every $a_4$: the metric is never flat.
At $b_4 = 1$ the component $R^{x4}{}_{x4} = 6a_4'^2$ shows that the deflation makes
curvature, although $\gamma^\mu\Omega_\mu$ does not contain $a_4$ at all.
```

In [13]:

```
def riemann_component(r, sg, mu, nu):
    value = sp.diff(Gam[r][nu][sg], x[mu]) - sp.diff(Gam[r][mu][sg], x[nu])
    for lam in range(8):
        value += Gam[r][mu][lam] * Gam[lam][nu][sg] - Gam[r][nu][lam] * Gam[lam][mu][sg]
    return value


def is_nonzero(expr):
    """True when expr is not identically zero.  A value clearly different from zero
    at the sample point proves it (fast); otherwise the exact test decides."""
    if abs(at_sample(expr)) > 1e-9:
        return True
    return not is_zero(expr)


riemann = {}  # (r, sigma, mu, nu) -> R^r_sigma mu nu, only the nonzero ones
for r in range(8):
```

```
    for sg in range(8):
        for mu in range(8):
            for nu in range(mu + 1, 8):
                value = riemann_component(r, sg, mu, nu)
                if value != 0 and is_nonzero(value):
                    riemann[(r, sg, mu, nu)] = value
                    riemann[(r, sg, nu, mu)] = -value
report("nonzero components R^r_sigma mu nu with mu < nu", len(riemann) // 2)
ricci_lower = sp.Matrix(8, 8, lambda sg, nu: sum(
    riemann.get((r, sg, r, nu), 0) for r in range(8)))  # R_sigma nu
ricci_mixed = g.inv() * ricci_lower  # R^mu_nu = g^mu sigma R_sigma nu
names = {"H": H, "x0": x0, "x4": x4, "a4": a4, "b4": b4, "a4p": a4p, "b4p": b4p,
         "a4pp": sp.Derivative(a4, (x4, 2)), "b4pp": sp.Derivative(b4, (x4, 2))}


def wolfram_to_sympy(text):
    """A text of a record in Wolfram notation as a sympy expression."""
    for long_form, short_form in (("Derivative[1][a4][x4]", "a4p"),
                                  ("Derivative[2][a4][x4]", "a4pp"),
                                  ("Derivative[1][b4][x4]", "b4p"),
                                  ("Derivative[2][b4][x4]", "b4pp"),
                                  ("a4[x4]", "a4"), ("b4[x4]", "b4")):
        text = text.replace(long_form, short_form)
    return parse_mathematica(text).subs(
        {sp.Symbol(k): value for k, value in names.items()}, simultaneous=True)


metric_b4 = json.loads(repository_file(METRIC_B4_FILE).read_text(encoding="utf-8"))
ricci_b4 = {tuple(NAMES.index(n) for n in key.split(",")): wolfram_to_sympy(text)
            for key, text in metric_b4["ricciMixed"].items()}
nonzero_mixed = {(m, n) for m in range(8) for n in range(8)
                 if not is_zero(ricci_mixed[m, n])}
check(nonzero_mixed == set(ricci_b4)
      and all(is_zero(ricci_mixed[k] - ricci_b4[k]) for k in ricci_b4)
      and is_zero(sum(ricci_mixed[a, a] for a in range(8))
                  - wolfram_to_sympy(metric_b4["ricciScalar"])),
      "the mixed Ricci tensor (10 nonzero components, 2 off the diagonal) and R equal "
      "the record with b4",
      record=f"{METRIC_B4_FILE}, keys ricciMixed and ricciScalar")
for key in sorted(ricci_b4):
    say(f"R^{NAMES[key[0]]}_{NAMES[key[1]]} = {ricci_b4[key]}")
FORMULAS = {item["key"]: item["wl"] for item in json.loads(
    repository_file(THEORY_FILE).read_text(encoding="utf-8"))["formulas"]}
ricci_record = wolfram_to_sympy(FORMULAS["ricci_mixed_diagonal"])
ricci_scalar = sum(ricci_mixed[a, a] for a in range(8))
check(all(is_zero(at_b4_1(ricci_mixed[a, a]) - ricci_record[a]) for a in range(8))
      and is_zero(at_b4_1(ricci_scalar) - wolfram_to_sympy(FORMULAS["ricci_scalar"])),
      "at b4 = 1: the eight Ricci components and R = 6 (a4'^2 - 7 H^2) equal the record",
      record=f"{THEORY_FILE}, formulas ricci_mixed_diagonal and ricci_scalar")
# multiplied out, the entries of gamma^mu Omega_mu contain neither a4 nor its derivatives
a4_free = all(not sp.expand(entry).has(a4) for entry in slash)
check(a4_free and is_zero(ricci_mixed[0, 0] + 6 * H**2)
      and is_zero(at_b4_1(ricci_mixed[4, 4]) - 6 * a4p ** 2),
      "gamma^mu Omega_mu contains no a4, but R^x4_x4 = 6 a4'^2 at b4 = 1; "
```

```
      "R^x0_x0 = -6 H^2 for every b4",
      record=f"{SCOPE_PY}, check gammaOmega_blind_to_the_deflation (b4 = 1)")
```

Out [13]:

```
RESULT nonzero components R^r_sigma mu nu with mu < nu = 78
PASS the mixed Ricci tensor (10 nonzero components, 2 off the diagonal) and R equal the
    record with b4
     reproduces Revision/metric_b4/results/metric-b4.json, keys ricciMixed and
    ricciScalar
R^x0_x0 = -6*H**2
R^x0_x4 = -3*H*tan(6*H*x0)*Derivative(b4(x4), x4)/(2*b4(x4))
R^x1_x1 = -6*H**2 + Derivative(a4(x4), (x4, 2)) + 3*Derivative(a4(x4),
    x4)*Derivative(b4(x4), x4)/(2*b4(x4))
R^x2_x2 = -6*H**2 + Derivative(a4(x4), (x4, 2)) + 3*Derivative(a4(x4),
    x4)*Derivative(b4(x4), x4)/(2*b4(x4))
R^x3_x3 = -6*H**2 + Derivative(a4(x4), (x4, 2)) + 3*Derivative(a4(x4),
    x4)*Derivative(b4(x4), x4)/(2*b4(x4))
R^x4_x0 = 3*H*cot(6*H*x0)*Derivative(b4(x4), x4)/(2*b4(x4))
R^x4_x4 = (-6*Derivative(a4(x4), x4)*Derivative(b4(x4), x4) + 3*Derivative(b4(x4), (x4,
    2)))/(2*b4(x4)) + 6*Derivative(a4(x4), x4)**2 - 3*Derivative(b4(x4),
    x4)**2/(4*b4(x4)**2)
R^x5_x5 = (-4*(6*H**2 + Derivative(a4(x4), (x4, 2)))*b4(x4)**2 + (-6*Derivative(a4(x4),
    x4)*Derivative(b4(x4), x4) + 2*Derivative(b4(x4), (x4, 2)))*b4(x4) +
    Derivative(b4(x4), x4)**2)/(4*b4(x4)**2)
R^x6_x6 = (-4*(6*H**2 + Derivative(a4(x4), (x4, 2)))*b4(x4)**2 + (-6*Derivative(a4(x4),
    x4)*Derivative(b4(x4), x4) + 2*Derivative(b4(x4), (x4, 2)))*b4(x4) +
    Derivative(b4(x4), x4)**2)/(4*b4(x4)**2)
R^x7_x7 = (-4*(6*H**2 + Derivative(a4(x4), (x4, 2)))*b4(x4)**2 + (-6*Derivative(a4(x4),
    x4)*Derivative(b4(x4), x4) + 2*Derivative(b4(x4), (x4, 2)))*b4(x4) +
    Derivative(b4(x4), x4)**2)/(4*b4(x4)**2)
PASS at b4 = 1: the eight Ricci components and R = 6 (a4'^2 - 7 H^2) equal the record
     reproduces Revision/theory/field-theory.json, formulas ricci_mixed_diagonal and
    ricci_scalar
PASS gamma^mu Omega_mu contains no a4, but R^x4_x4 = 6 a4'^2 at b4 = 1; R^x0_x0 = -6 H^2
    for every b4
     reproduces Revision/theory/reports/python-scope.json, check
    gammaOmega_blind_to_the_deflation (b4 = 1)
```

Text cell:

```
The next cell draws the Ricci components on the background $b_4 = 1$. On the
left, the eight values $R^\mu{}_\mu$ at the sample values $H = 1$, $a_4' = 1/2$,
$a_4'' = 0$ (in units of $H^2$). On the right, three of them as functions of the
rate $a_4'$ (with $a_4'' = 0$): the hidden component $-6H^2$ does not depend on the
history, the time component $6a_4'^2$ is made entirely by the expansion and
deflation, and the 3-space component $a_4'' - 6H^2$ stays $-6H^2$ for a steady
rate ($a_4'' = 0$).
```

In [14]:

```
rate_symbol, accel_symbol = sp.symbols("rate accel", real=True)
numeric = [sp.lambdify((rate_symbol, accel_symbol), ricci_record[a].subs(
    {H: 1, sp.Derivative(a4, (x4, 2)): accel_symbol,
     sp.Derivative(a4, x4): rate_symbol})) for a in range(8)]
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
values = [float(numeric[a](0.5, 0.0)) for a in range(8)]
colors = ["black"] + ["tab:red"] * 3 + ["gray"] + ["tab:blue"] * 3
ax_left.bar(range(8), values, color=colors, width=0.6)
ax_left.axhline(0.0, color="black", linewidth=0.8)
ax_left.set_xticks(range(8), [f"$R^{{{n}}}{{}}_{{{n}}}$" for n in NAMES], fontsize=8)
ax_left.set_ylabel("value (units of $H^2$)")
ax_left.set_title("Ricci components at $a_4' = 1/2$, $a_4'' = 0$, $b_4 = 1$")
rates = np.linspace(-3.0, 3.0, 121)
for a, label, style in ((0, "$R^{x0}{}_{x0} = -6H^2$", "-"),
                        (4, "$R^{x4}{}_{x4} = 6a_4'^2$", "--"),
                        (1, "$R^{x1}{}_{x1} = a_4'' - 6H^2$", ":")):
    ax_right.plot(rates, np.broadcast_to(numeric[a](rates, 0.0), rates.shape),
                  style, label=label)
ax_right.axhline(0.0, color="black", linewidth=0.8)
ax_right.set_xlabel("rate $a_4'$ (units of $H$), with $a_4'' = 0$, $b_4 = 1$")
ax_right.set_ylabel("value (units of $H^2$)")
ax_right.set_title("Never flat: $R^{x0}{}_{x0} = -6H^2$")
ax_right.legend(fontsize=8)
save_figure(fig, "ricci_components",
            "The mixed Ricci components $R^\\mu{}_\\mu$ of the author's metric on the "
            "background $b_4 = 1$, in units of $H^2$. Left: all eight at $H = 1$, "
            "$a_4' = 1/2$, $a_4^{\\prime\\prime} = 0$ (black: hidden direction, red: "
            "3-space, gray: time, blue: extra times). Right: $R^{x0}{}_{x0} = -6H^2$ "
            "(solid), $R^{x4}{}_{x4} = 6a_4'^2$ (dashed) and $R^{x1}{}_{x1} = "
            "a_4^{\\prime\\prime} - 6H^2$ (dotted) versus the rate $a_4'$ in units of "
            "$H$ with $a_4^{\\prime\\prime} = 0$. The hidden component "
            "is negative for every history (and every $b_4$), so the metric is never "
            "flat, and the time component is curvature made by the expansion and "
            "deflation (the dotted line lies on the solid one because "
            "$a_4^{\\prime\\prime} = 0$).")
```

Out [14]:

```
Figure 06b.4 saved as Revision/textbook/figures/06b_4_ricci_components.png
```

Text cell:

```
## 13. The curvature of the spinor connection

The next cell computes, in the diagonal vielbein, the 28 matrices $F_{\mu\nu}$
($\mu < \nu$) and checks the theorem $F_{\mu\nu} = \frac12\sum_{a,b}
R_{ab\mu\nu}S^{ab}$, where $R_{ab\mu\nu} = \eta_{aa}\,f_a\,R^a{}_{b\mu\nu}/f_b$
is the Riemann tensor with two frame indices (for the diagonal vielbein), for every
$b_4$. Then it computes $F'_{x0\,x1}$ in the boosted vielbein with $\beta = 6H$
(where the contraction vanished for a constant $b_4$) and checks that it is
$R\,F_{x0\,x1}R^{-1}$ and is not zero: the spinor connection is still there and
curved.
```

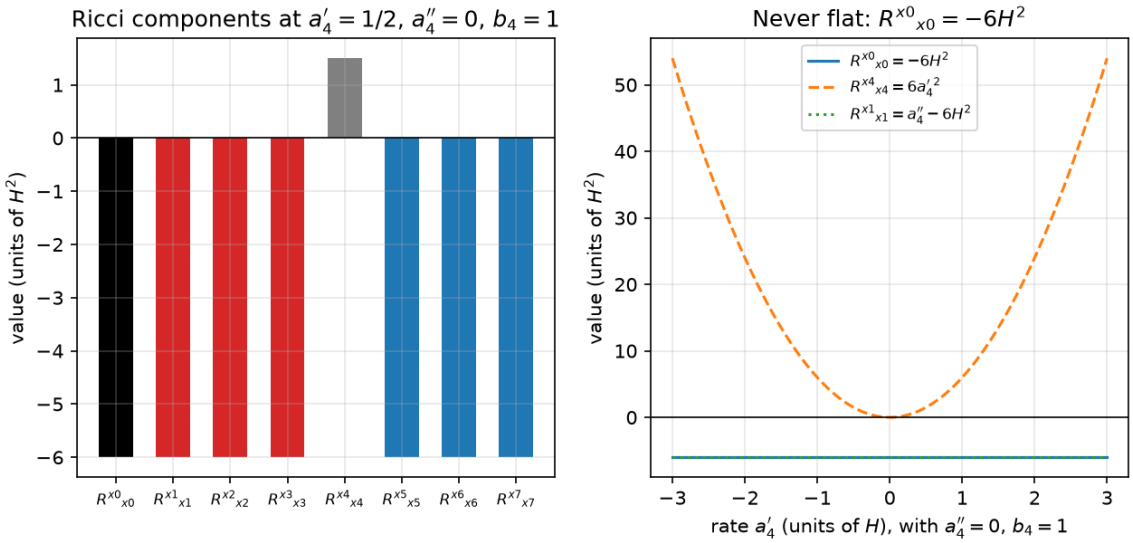

Figure 195: The mixed Ricci components $R^{\mu}{}_{\mu}$ of the author's metric on the background $b_4 = 1$, in units of H^2. Left: all eight at $H = 1$, $a_4' = 1/2$, $a_4'' = 0$ (black: hidden direction, red: 3-space, gray: time, blue: extra times). Right: $R^{x0}{}_{x0} = -6H^2$ (solid), $R^{x4}{}_{x4} = 6a_4'^2$ (dashed) and $R^{x1}{}_{x1} = a_4'' - 6H^2$ (dotted) versus the rate a_4' in units of H with $a_4'' = 0$. The hidden component is negative for every history (and every b_4), so the metric is never flat, and the time component is curvature made by the expansion and deflation (the dotted line lies on the solid one because $a_4'' = 0$). (Notebook 06b, figure 4.)

In [15]:

```
def spinor_curvature(Om, mu, nu):
    return (Om[nu].diff(x[mu]) - Om[mu].diff(x[nu])
            + Om[mu] * Om[nu] - Om[nu] * Om[mu])


F = {}
theorem_ok = True
for mu in range(8):
    for nu in range(mu + 1, 8):
        F[(mu, nu)] = spinor_curvature(Omega, mu, nu)
        from_riemann = sum((ETA[a] * f[a] * riemann[(a, c, mu, nu)] / f[c] * S[a][c] / 2
                            for a in range(8) for c in range(8)
                            if (a, c, mu, nu) in riemann), Z16)
        theorem_ok = theorem_ok and matrix_is_zero(F[(mu, nu)] - from_riemann)
check(theorem_ok and record_passed(REPORT_WL_B4, "spin_curvature_equals_Riemann"),
      "F_mu nu = (1/2) R_ab mu nu S^ab in all 28 coordinate planes",
      record=f"{REPORT_WL_B4}, check spin_curvature_equals_Riemann")
F01_boosted = spinor_curvature(Omega_6H, 0, 1)
R_6H, R_inv_6H = R.subs(six_H), R_inv.subs(six_H)
check(matrix_is_zero(F01_boosted - R_6H * F[(0, 1)] * R_inv_6H)
      and np.abs(sample_table(F01_boosted)).max() > 0.1,
      "boosted (beta = 6H): F'_x0x1 = R F_x0x1 R^-1, nonzero",
      record=f"{SCOPE_PY}, check boosted_frame_curvature_nonzero (b4 = 1)")
```

Out [15]:

```
PASS F_mu nu = (1/2) R_ab mu nu S^ab in all 28 coordinate planes
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    spin_curvature_equals_Riemann
PASS boosted (beta = 6H): F'_x0x1 = R F_x0x1 R^-1, nonzero
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_curvature_nonzero (b4 = 1)
```

Text cell:

```
The next cell draws the size of the spinor curvature in each coordinate plane: for
every pair $(\mu, \nu)$ the number $\|F_{\mu\nu}\| = \sqrt{\sum_{i,j}
(F_{\mu\nu})_{ij}^2}$ (the square root of the sum of the squares of the 256
entries) at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$,
$a_4^{\prime\prime} = 0$, $b_4 = 1$ (diagonal vielbein). In another frame
$F'_{\mu\nu} = R\,F_{\mu\nu}R^{-1}$ with an invertible matrix $R$, so a change of
frame can change the sizes, but it can never turn a coloured square white or a
white square coloured.
```

In [16]:

```
norms = np.zeros((8, 8))
for (mu, nu), matrix in F.items():
    norms[mu, nu] = norms[nu, mu] = np.sqrt((sample_table(matrix) ** 2).sum())
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(norms, cmap="Purples")
for mu in range(8):
    for nu in range(8):
        if norms[mu, nu] > 0:
            # white digits on the dark squares (the largest sizes), black elsewhere
            dark = norms[mu, nu] > 0.6 * norms.max()
            ax.text(nu, mu, f"{norms[mu, nu]:.1f}", ha="center", va="center",
                    fontsize=7, color="white" if dark else "black")
ax.set_xticks(range(8), NAMES)
ax.set_yticks(range(8), NAMES)
ax.set_xlabel("coordinate $\\nu$")
ax.set_ylabel("coordinate $\\mu$")
ax.set_title("Size of the spinor curvature $\\|F_{\\mu\\nu}\\|$")
ax.grid(False)
fig.colorbar(image, ax=ax, shrink=0.8, label="$\\|F_{\\mu\\nu}\\|$ (units of $H^2$)")
save_figure(fig, "spinor_curvature_norms",
            "The size $\\|F_{\\mu\\nu}\\|$, the square root of the sum of the squares "
            "of the 256 entries of the spinor curvature, for every pair of "
            "coordinates $(\\mu, \\nu)$ in the diagonal vielbein at the sample point "
            "$H = 1$, $z = \\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, "
            "$a_4^{\\prime\\prime} = 0$, $b_4 = 1$, in units "
            "of $H^2$ (darker is larger, white is zero). All 28 coordinate planes "
            "except the plane of the hidden direction $x0$ and the time $x4$ are "
            "curved at this point, and $F_{x0\\,x4} = 0$ exactly; $F_{\\mu\\nu} = "
            "\\frac12 R_{ab\\mu\\nu}S^{ab}$ is the Riemann curvature, which no "
```

```
            "change of frame removes.")
report("largest |F_mu nu| at the sample point (units of H^2)", f"{norms.max():.3f}")
curved = [pair for pair in F if norms[pair] > 1e-9]  # the curved planes
check(matrix_is_zero(F[(0, 4)]) and len(curved) == 27,
      "F_x0x4 = 0 exactly (every b4); the other 27 planes are curved at the sample "
      "point")
```

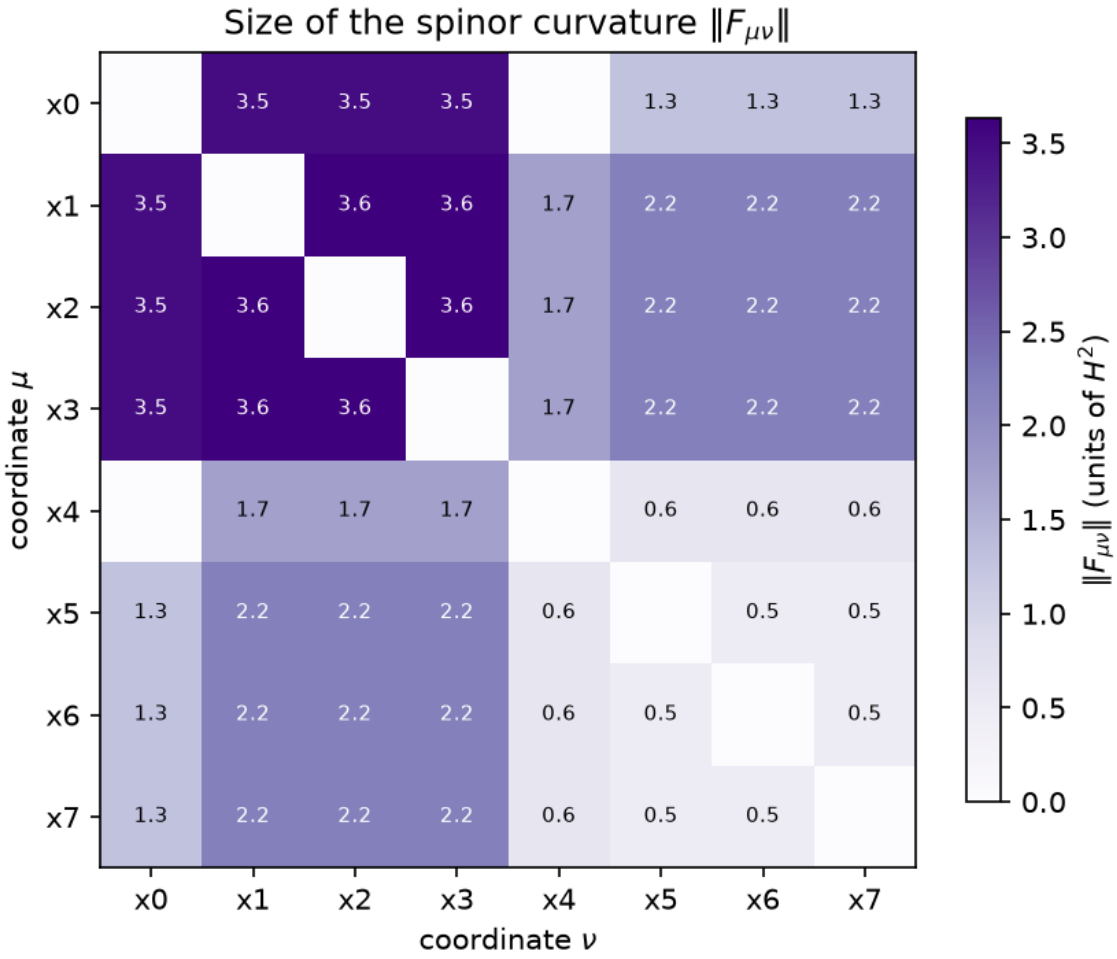

Figure 196: The size $\|F_{\mu\nu}\|$, the square root of the sum of the squares of the 256 entries of the spinor curvature, for every pair of coordinates (μ, ν) in the diagonal vielbein at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $a_4'' = 0$, $b_4 = 1$, in units of H^2 (darker is larger, white is zero). All 28 coordinate planes except the plane of the hidden direction $x0$ and the time $x4$ are curved at this point, and $F_{x0\,x4} = 0$ exactly; $F_{\mu\nu} = \frac{1}{2} R_{ab\mu\nu} S^{ab}$ is the Riemann curvature, which no change of frame removes. (Notebook 06b, figure 5.)

Out [16]:

```
Figure 06b.5 saved as Revision/textbook/figures/06b_5_spinor_curvature_norms.png
RESULT largest |F_mu nu| at the sample point (units of H^2) = 3.633
PASS F_x0x4 = 0 exactly (every b4); the other 27 planes are curved at the sample point
```

Text cell:

```
## 14. The rescaling $\Psi = \sin^{-1/2}(z)\,b_4^{-3/4}\,\chi$ removes the term

A frame change is one way to remove $\gamma^\mu\Omega_\mu$; a change of the field
variable is another. Write $\Psi = w_r\,\chi$ with $w_r = \sin^{-1/2}(z)\,
b_4^{-3/4}$. Then $\gamma^\mu D_\mu\Psi = w_r\,\gamma^\mu\partial_\mu\chi +
\big(\gamma^\mu\partial_\mu w_r + w_r\,\gamma^\mu\Omega_\mu\big)\chi$. The weight
depends on $x_0$ and $x_4$, with $\gamma^{x0} = \tan z\,\gamma^{(x0)}$ and
$\gamma^{x4} = \gamma^{(x4)}$, so the bracket is $\big(\tan z\,\partial_{x0}w_r +
3Hw_r\big)\gamma^{(x0)} + \big(\partial_{x4}w_r + \frac34\frac{b_4'}{b_4}w_r\big)
\gamma^{(x4)}$. Now $\partial_{x0}\sin^{-1/2}z = -\frac12\sin^{-3/2}z\cdot 6H\cos z$
gives $\tan z\,\partial_{x0}w_r = -3Hw_r$, and $\partial_{x4}b_4^{-3/4} =
-\frac34 b_4^{-7/4}b_4'$ gives $\partial_{x4}w_r = -\frac34\frac{b_4'}{b_4}w_r$:
the bracket is zero. At $b_4 = 1$ the weight is the record's $\sin^{-1/2}z$. The
next cell checks this as a matrix identity and, for safety, on 16 completely
general functions $\chi_1, \dots, \chi_{16}$ of all eight coordinates. It also
checks the record's remark about the potential $U = \frac{\lambda}{2}S^2$: since
$w_r$ is real, $S[\Psi] = \Psi^\dagger C\Psi = w_r^2\,S[\chi] =
S[\chi]/(b_4^{3/2}\sin z)$, so the rescaled equation is $\gamma^\mu
\partial_\mu\chi = \big(m + \lambda S[\chi]/(b_4^{3/2}\sin z)\big)\,\chi$: for
$U = 0$ the term is removed completely; for $U \ne 0$ it becomes a coupling that
depends on $x_0$ (and on $x_4$ through $b_4$).
```

In [17]:

```
w_r = sp.sin(z) ** sp.Rational(-1, 2) * b4 ** sp.Rational(-3, 4)  # the weight
bracket = sum((gup[mu] * sp.diff(w_r, x[mu]) for mu in range(8)), Z16) + w_r * slash
chi = sp.Matrix([sp.Function(f"chi{A}")(*x) for A in range(1, 17)])
psi = w_r * chi
dirac_psi = sum((gup[mu] * (psi.diff(x[mu]) + Omega[mu] * psi) for mu in range(8)),
                sp.zeros(16, 1))
dirac_chi = sum((gup[mu] * chi.diff(x[mu]) for mu in range(8)), sp.zeros(16, 1))
check(matrix_is_zero(bracket) and all(is_zero(v) for v in dirac_psi - w_r * dirac_chi),
      "gamma^mu D_mu (w chi) = w gamma^mu d_mu chi for w = sin^(-1/2) z b4^(-3/4)",
      record=f"{SCOPE_PY}, check rescaling_removes_the_connection_term (b4 = 1)")
chi_conj = sp.Matrix([sp.conjugate(v) for v in chi])  # complex conjugates
S_psi = (w_r * chi_conj).T * C * (w_r * chi)  # Psi^dagger C Psi (w is real)
S_chi = chi_conj.T * C * chi
check(is_zero(S_psi[0, 0] - S_chi[0, 0] / (b4 ** sp.Rational(3, 2) * sp.sin(z))),
      "S[w chi] = S[chi] / (b4^(3/2) sin z), S[chi] / sin z at b4 = 1",
      record=f"{SCOPE_PY}, check rescaled_equation_quadratic_potential (b4 = 1)")
```

Out [17]:

```
PASS gamma^mu D_mu (w chi) = w gamma^mu d_mu chi for w = sin^(-1/2) z b4^(-3/4)
     reproduces Revision/theory/reports/python-scope.json, check
    rescaling_removes_the_connection_term (b4 = 1)
PASS S[w chi] = S[chi] / (b4^(3/2) sin z), S[chi] / sin z at b4 = 1
     reproduces Revision/theory/reports/python-scope.json, check
    rescaled_equation_quadratic_potential (b4 = 1)
```

Text cell:

```
The next cell draws the rescaling for $H = 1$ on the background $b_4 = 1$, where
$w_r = \sin^{-1/2}z$: on the left the weight (large near the tip $z \to 0$, equal
to 1 at $z = \pi/2$); on the right the two terms of the coefficient of
$\gamma^{(x0)}$, $\tan z\,\partial_{x0}w_r$ and $3Hw_r$, computed from their sympy
formulas, and their sum, which is zero at every $z$.
```

In [18]:

```
z_values = np.linspace(0.05, np.pi / 2 - 0.02, 300)
x0_symbol = sp.Symbol("u", positive=True)
w_1 = at_b4_1(w_r)  # the weight at b4 = 1: sin^(-1/2) z
term_derivative = sp.lambdify(x0_symbol, (sp.tan(z) * sp.diff(w_1, x0)).subs(
    {H: 1}).subs(x0, x0_symbol))
term_connection = sp.lambdify(x0_symbol, (3 * H * w_1).subs({H: 1}).subs(x0, x0_symbol))
weight = sp.lambdify(x0_symbol, w_1.subs({H: 1}).subs(x0, x0_symbol))
u_values = z_values / 6.0  # x0 = z / (6 H) with H = 1
first, second = term_derivative(u_values), term_connection(u_values)
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
ax_left.plot(z_values, weight(u_values), label="$w_r = \\sin^{-1/2} z$ ($b_4 = 1$)")
ax_left.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_left.set_ylabel("weight (pure number)")
ax_left.set_title("The rescaling weight")
ax_left.legend(fontsize=8)
ax_right.plot(z_values, first, label="$\\tan z\\,\\partial_{x0} w_r$")
ax_right.plot(z_values, second, "--",
              label="$3H w_r$ (from $\\gamma^\\mu\\Omega_\\mu$)")
ax_right.plot(z_values, first + second, ":", color="black", label="sum")
ax_right.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_right.set_ylabel("coefficient of $\\gamma^{(x0)}\\chi$ (units of $H$)")
ax_right.set_title("The two terms cancel")
ax_right.legend(fontsize=8)
save_figure(fig, "rescaling",
            "The rescaling $\\Psi = w_r\\chi$ with $w_r = \\sin^{-1/2}(z)\\,"
            "b_4^{-3/4}$, drawn for $H = 1$ on the background $b_4 = 1$. Left: the "
            "weight $w_r$ versus the hidden angle $z$ in radians. Right: the two "
            "terms of the coefficient of $\\gamma^{(x0)}\\chi$ in the field "
            "equation of $\\chi$, the derivative term $\\tan z\\,\\partial_{x0}w_r = "
            "-3Hw_r$ (solid) and the spin-connection term $3Hw_r$ (dashed), in units "
            "of $H$, and their sum (dotted), which is zero at every $z$: in the "
            "variable $\\chi$ the equation has no term $3H\\gamma^{(x0)}$ (and, by "
            "the factor $b_4^{-3/4}$, no term $\\frac34(b_4'/b_4)\\gamma^{(x4)}$).")
check(np.max(np.abs(first + second)) < 1e-12, "the two terms cancel at all 300 points")
```

Out [18]:

```
Figure 06b.6 saved as Revision/textbook/figures/06b_6_rescaling.png
PASS the two terms cancel at all 300 points
```

Text cell:

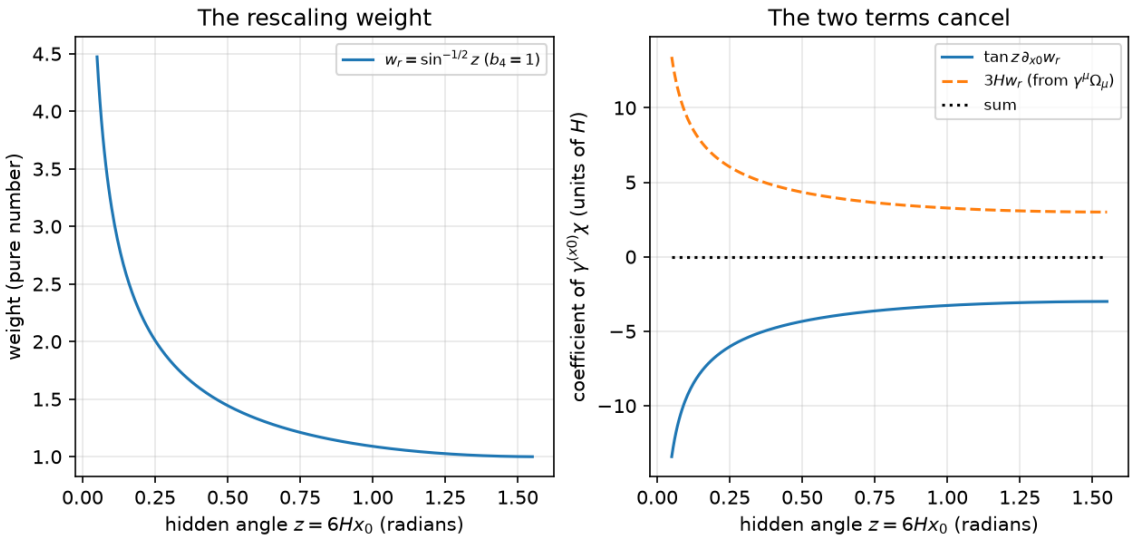

Figure 197: The rescaling $\Psi = w_r \chi$ with $w_r = \sin^{-1/2}(z)\, b_4^{-3/4}$, drawn for $H = 1$ on the background $b_4 = 1$. Left: the weight w_r versus the hidden angle z in radians. Right: the two terms of the coefficient of $\gamma^{(x0)}\chi$ in the field equation of χ, the derivative term $\tan z\, \partial_{x0} w_r = -3H w_r$ (solid) and the spin-connection term $3H w_r$ (dashed), in units of H, and their sum (dotted), which is zero at every z: in the variable χ the equation has no term $3H\gamma^{(x0)}$ (and, by the factor $b_4^{-3/4}$, no term $\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$). (Notebook 06b, figure 6.)

```
## 15. The last checks

The last cell checks that every check of the Revision reports that states a result
reproduced in this notebook is recorded there as passed (the sympy scope checker
‘python-scope.json‘, the WolframScript scope checker ‘wolfram-scope.json‘, which
shares no code with it, and the curvature checks of the field-theory reports, with
$b_4$ and at $b_4 = 1$), that all six figure files exist in the folder
Revision/textbook/figures, and prints the number of checks that passed.
```

In [19]:

```
scope_names = ["boosted_frame_reproduces_metric", "boosted_frame_canonical_connection",
               "boosted_frame_gammaOmega_formula", "boosted_frame_gammaOmega_vanishes",
               "boosted_frame_curvature_nonzero",
               "rescaling_removes_the_connection_term",
               "rescaled_equation_quadratic_potential",
               "gammaOmega_blind_to_the_deflation"]
cited = {SCOPE_PY: scope_names, SCOPE_WL: scope_names,
         REPORT_PY: ["spinor_curvature_equals_riemann",
                     "curvature_nonzero_flat_only_formally"],
         REPORT_WL: ["ricci_scalar", "ricci_mixed_components",
                     "never_flat_for_H_positive", "spin_curvature_equals_Riemann"],
         REPORT_WL_B4: ["ricci_equals_metric_b4_set", "ricci_reduces_at_b4_1",
                        "spin_curvature_equals_Riemann"]}
count = sum(len(v) for v in cited.values())
check(all(record_passed(path, name) for path, names in cited.items()
```

```
          for name in names), f"the {count} cited record checks are recorded as passed")
expected = [f"06b_{k}_{name}.png" for k, name in enumerate([
    "boost_hyperbolas", "boosted_contraction", "boosted_connection_heat_maps",
    "ricci_components", "spinor_curvature_norms", "rescaling"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected),
      "all six figure files exist")
all_checks_passed()
```

Out [19]:

```
PASS the 25 cited record checks are recorded as passed
PASS all six figure files exist
ALL 26 CHECKS PASSED (notebook 06b)
```

Text cell:

```
## 16. What this notebook showed

- A boost of the frame in the $(x4, x0)$ plane with rapidity $b = \beta x_4 + b_0$
  describes the same metric, and its canonical spin connection obeys the vielbein
  postulate. In it $\gamma'^\mu\Omega'_\mu = \frac{6H-\beta}{2}(\cosh b\,
  \gamma^{(x0)} - \sinh b\,\gamma^{(x4)}) + \frac34\frac{b_4'}{b_4}(\cosh b\,
  \gamma^{(x4)} - \sinh b\,\gamma^{(x0)})$ exactly (PROVED; at $b_4 = 1$ it
  reproduces the record). For $\beta = 6H$ and a constant $b_4$ the contraction
  vanishes identically, while $\Omega'_\mu$ is nonzero for $\mu = x1, \dots, x7$.
- The reason is the local spin covariance: with the spinor boost
  $R = \cosh\frac b2 - \sinh\frac b2\,\gamma^{(x4)}\gamma^{(x0)}$, $\Omega'_\mu =
  R\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}$, and the second term contributes
  $-\frac{\beta}{2}(\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)})$ (PROVED
  here).
- So the value of $\gamma^\mu\Omega_\mu$ in the diagonal vielbein is
  FRAME-DEPENDENT. What is frame-independent: the metric is curved for every
  $H > 0$ and every $b_4$ ($R^{x0}{}_{x0} = -6H^2$, PROVED; reproduces the
  records), the spinor curvature is $F_{\mu\nu} = \frac12 R_{ab\mu\nu}S^{ab}$ in
  all 28 planes (PROVED; reproduces the record), and it is nonzero in every frame
  because $F' = RFR^{-1}$; therefore no frame makes $\Omega_\mu$ vanish.
- The deflation makes curvature ($R^{x4}{}_{x4} = 6a_4'^2$ at $b_4 = 1$) that the
  contraction $\gamma^\mu\Omega_\mu$ does not see (PROVED; reproduces the record).
- The rescaling $\Psi = \sin^{-1/2}(z)\,b_4^{-3/4}\,\chi$ removes the term
  $\gamma^\mu\Omega_\mu$ from the field equation exactly; with
  $U = \frac\lambda2 S^2$ it becomes the coupling
  $\lambda S[\chi]/(b_4^{3/2}\sin z)$ (PROVED; at $b_4 = 1$ it reproduces the
  record).
- In short, the non-triviality of the field equations holds in the qualified sense
  of the record: the gravitational field enters through the vielbein factors of
  every derivative term and through a spin connection whose curvature cannot be
  removed, but the particular term $\gamma^\mu\Omega_\mu$ is a property of the
  chosen frame and field variables.
```

6.22 Line-by-line walk-through of Notebook 06b

The notebook has 19 code cells, In [1] to In [19]; this section explains every line of every one of them. As in Section 6.14, a line `...` in a quoted figure cell stands for the remaining lines of a long caption, printed in full in Section 6.21.

In [1], the set-up cell. Its comment lines are the run instructions of Section 6.20. Its code is, line for line, the set-up code of Notebook 06a explained at the beginning of Section 6.14, with the single difference

```
NOTEBOOK_ID = "06b"  # this notebook: chapter 06, example b
```

so that the figures are named `06b_<k>_<name>.png` and the captions file is `Revision/textbook/figures/06b.captions.json`. It prints one line, Set-up of notebook 06b complete: repository folder found, helpers defined.

In [2], the tools.

```
import sys  # the Python system module (here: the stream of printed text)

# Jupyter sends printed text to the screen in pieces, about every 0.2 seconds.  The
# book's checking tool reads each PASS line together with its "reproduces" line, so
# the pieces must be the same in every run: send the printed text of a cell in one
# piece when the cell ends (or just before a figure), at the latest after 600 s.
if hasattr(sys.stdout, "flush_interval"):  # true inside Jupyter only
    sys.stdout.flush_interval = 600.0
```

The same setting of the output channel as in In [2] of Notebook 06a: inside Jupyter the printed text of a cell is sent in one piece, so that every PASS line arrives together with its "reproduces" line.

```
import numpy as np  # decimal numbers for the plots
import sympy as sp  # exact algebra and calculus with symbols
from sympy.parsing.mathematica import parse_mathematica  # reads Wolfram notation
```

numpy for the plots, sympy for the exact algebra, and sympy's reader of Wolfram notation.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
NAMES = fixture["coordinates"]  # ["x0", ..., "x7"]
ETA = fixture["eta"]  # +1 space-like, -1 time-like
gamma = [sp.Matrix(rows) for rows in fixture["gamma"]]  # gamma[a] = gamma^(xa)
C = sp.Matrix(fixture["C"])  # the author's C (Dirac adjoint Psibar = Psi^dagger C)
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
```

The record `Revision/algebra/gammas.json` is read as in Notebook 06a: the direction names, the signs of η, and the eight gammas as exact matrices. New is `C`, the author's matrix $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$ stored in the same record (Chapter 5); In [17] needs it for the bilinear $S = \Psi^\dagger C \Psi$. `I16, Z16 = ...` gives two names at once (the identity and the zero matrix).

```
check(all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
          == (2 * ETA[a] if a == b else 0) * I16
          for a in range(8) for b in range(8)),
      "the 64 Clifford relations of the author's gammas")
S = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
     for a in range(8)]
```

The 64 Clifford relations (the same test as in Notebook 06a, In [2]) and the 64 generators S^{ab}.

```
def is_zero(expr):
    """True when expr is exactly zero for all values of its symbols."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    if sp.cancel(sp.expand(expr.rewrite(sp.exp))) == 0:  # the fast route
        return True
    return sp.simplify(expr) == 0  # the slower general route
```

The exact zero test `is_zero`, explained in Section 6.14 (Notebook 06a, In [3]): first the fast route through exponential functions, then `simplify`; it never calls a nonzero expression zero.

```
def matrix_is_zero(matrix):
    return all(is_zero(entry) for entry in matrix)


def record_passed(path, name):
    """True when the Revision report at path records the check name as passed."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper() == "PASS"
    raise KeyError(f"{path} has no check {name}")
```

`matrix_is_zero` applies the test to every entry, and `record_passed` reads a Revision report and says whether a check is recorded as passed; both as in Notebook 06a.

```
SCOPE_PY = "Revision/theory/reports/python-scope.json"
SCOPE_WL = "Revision/theory/reports/wolfram-scope.json"
REPORT_PY = "Revision/theory/reports/python-field-theory.json"
REPORT_WL = "Revision/theory/reports/wolfram-field-theory.json"
REPORT_WL_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
THEORY_FILE = "Revision/theory/field-theory.json"
METRIC_B4_FILE = "Revision/metric_b4/results/metric-b4.json"
```

The paths of the seven records that this notebook reproduces: the two scope reports, the two field-theory reports and the formula file of $b_4 = 1$, the WolframScript report of the field theory with b_4, and the record `metric-b4.json`. The cell prints one PASS line.

In [3], the four general formulas. They are those of Notebook 06a, In [4], written a little more compactly; Section 6.14 explains them line by line.

```
def christoffel(g, coords):
    """Gam[l][m][n] = (1/2) sum_r g^lr (d_m g_rn + d_n g_rm - d_r g_mn)."""
    n = len(coords)
    g_inv = g.inv()
    Gam = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for l in range(n):
        for m in range(n):
            for k in range(m, n):
                value = sum(g_inv[l, r] * (sp.diff(g[r, k], coords[m])
                                           + sp.diff(g[r, m], coords[k])
                                           - sp.diff(g[m, k], coords[r]))
                            for r in range(n) if g_inv[l, r] != 0) / 2
                Gam[l][m][k] = Gam[l][k][m] = sp.simplify(value)
    return Gam
```

The Christoffel symbols; the one difference from Notebook 06a is that the line `Gam[l][m][k] = Gam[l][k][m] = sp.simplify(value)` stores the simplified symbol in both places at once.

```
def spin_connection(e, Gam, coords, eta):
    """omega[mu][a][b] = eta_aa sum_nu e^a_nu (d_mu e_b^nu + Gam^nu_mu,lam e_b^lam)."""
    n = len(coords)
    E = e.inv()  # E[nu, b] = e_b^nu
    omega = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for mu in range(n):
        for a in range(n):
            for b in range(n):
                mixed = 0
                for nu in range(n):
                    if e[a, nu] == 0:
                        continue
                    mixed += e[a, nu] * (sp.diff(E[nu, b], coords[mu]) + sum(
                        Gam[nu][mu][lam] * E[lam, b] for lam in range(n)))
                omega[mu][a][b] = sp.simplify(eta[a] * mixed)
    return omega
```

The canonical spin connection with the first index lowered. Here it matters that the function accepts ANY vielbein matrix `e`, not only a diagonal one: In [7] gives it the boosted vielbein, which has off-diagonal entries in the rows and columns of $x4$ and $x0$, and then the inverse `E = e.inv()` and the sums over `nu` and `lam` really have several terms.

```
def spinor_connection(omega, S):
    """Omega[mu] = sum_{a<b} omega_{mu ab} S^ab."""
    n = len(omega)
    size = S[0][0].shape[0]
    return [sum((omega[mu][a][b] * S[a][b] for a in range(n)
                 for b in range(a + 1, n) if omega[mu][a][b] != 0),
                sp.zeros(size, size)) for mu in range(n)]
```

The spinor connection $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$, as one list comprehension: for each μ a sum of matrices starting from the zero matrix.

```
def curved_gammas(e, gammas):
    """gamma^mu = sum_a e_a^mu gamma^a."""
    E = e.inv()
    return [sum((E[mu, a] * gammas[a] for a in range(len(gammas))),
                sp.zeros(*gammas[0].shape)) for mu in range(len(gammas))]
```

The curved gammas, and one output line.

```
say("defined: christoffel, spin_connection, spinor_connection, curved_gammas")
```

In [4], the diagonal vielbein again (the control).

```
x = sp.symbols("x0:8", real=True)  # x[0] = x0, ..., x[4] = x4, ..., x[7] = x7
x0, x4 = x[0], x[4]
H = sp.Symbol("H", positive=True)
a4 = sp.Function("a4")(x4)  # any function of the time
b4 = sp.Function("b4", positive=True)(x4)  # any positive function of the time
a4p, b4p = sp.Derivative(a4, x4), sp.Derivative(b4, x4)
z = 6 * H * x0
s = sp.sin(z) ** sp.Rational(1, 6)
f = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
     + [sp.exp(-a4) * sp.sqrt(b4) * s] * 3)  # f_0, ..., f_7
g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # the author's metric
e = sp.diag(*f)  # the diagonal vielbein
```

The symbols, the factors, the author's metric with b_4, the diagonal vielbein and the helper `at_b4_1`, exactly as in Notebook 06a, In [7].

```
Gam = christoffel(g, x)
omega = spin_connection(e, Gam, x, ETA)
Omega = spinor_connection(omega, S)
gup = curved_gammas(e, gamma)
slash = sum((gup[mu] * Omega[mu] for mu in range(8)), Z16)
```

The Christoffel symbols, the spin connection, the spinor connection, the curved gammas and the contraction of the diagonal vielbein.

```
c_b4 = sp.Rational(3, 4) * b4p / b4  # the coefficient (3/4) b4'/b4
check(matrix_is_zero(slash - 3 * H * gamma[0] - c_b4 * gamma[4])
      and matrix_is_zero(at_b4_1(slash) - 3 * H * gamma[0]),
      "control: the diagonal vielbein gives gamma^mu Omega_mu = 3 H gamma^(x0) + "
      "(3/4)(b4'/b4) gamma^(x4), 3 H gamma^(x0) at b4 = 1",
      record=f"{SCOPE_PY}, check rescaling_removes_the_connection_term (b4 = 1)")
```

The control: the same code, with no boost, must give $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ (`c_b4` is $\frac{3}{4}b_4'/b_4$), and $3H\gamma^{(x0)}$ at $b_4 = 1$. The check names the scope report's check `rescaling_removes_the_connection_term`, whose detail records this control value at $b_4 = 1$. The cell prints one PASS line with its record.

In [5], the boost matrix (Section 6.16).

```
def boost(b):
    """The 8 x 8 boost matrix Lambda(b) in the plane of x0 (index 0) and x4 (4)."""
    L = sp.eye(8)
    L[0, 0], L[0, 4], L[4, 0], L[4, 4] = sp.cosh(b), sp.sinh(b), sp.sinh(b), sp.cosh(b)
    return L
```

`boost(b)` returns the 8×8 identity with the four entries of the rows and columns 0 ($x0$) and 4 ($x4$) replaced by $\cosh b$, $\sinh b$, $\sinh b$, $\cosh b$. The four assignments are made in one line (a tuple of four places receives a tuple of four values).

```
b_symbol = sp.Symbol("b", real=True)  # a rapidity
eta_matrix = sp.diag(*ETA)
L_test = boost(b_symbol)
```

A symbolic rapidity `b_symbol`, the matrix η as a diagonal matrix, and the boost with this rapidity.

```
check(matrix_is_zero(L_test.T * eta_matrix * L_test - eta_matrix),
      "Lambda(b)^T eta Lambda(b) = eta for every rapidity b")
check(matrix_is_zero(L_test * boost(-b_symbol) - sp.eye(8)),
      "the inverse of the boost with rapidity b is the boost with -b")
```

Two checks for every rapidity: $\Lambda(b)^T \eta \Lambda(b) - \eta = 0$ (`.T` is the transpose), and $\Lambda(b)\Lambda(-b) = 1$, so the inverse boost is the boost with $-b$. The cell prints two PASS lines.

In [6], figure 1.

```
fig, ax = plt.subplots(figsize=(6.4, 6.0))
rapidity = np.linspace(-1.4, 1.4, 200)
ax.plot(np.cosh(rapidity), np.sinh(rapidity), color="tab:blue", linewidth=1.0,
        label="unit hyperbola of the time direction: $t^2 - y^2 = 1$")
ax.plot(np.sinh(rapidity), np.cosh(rapidity), color="tab:orange", linewidth=1.0,
        label="unit hyperbola of the hidden direction: $y^2 - t^2 = 1$")
ax.plot([0, 2.2], [0, 2.2], ":", color="gray", label="light line $t = y$")
```

One square panel. 200 rapidities from -1.4 to 1.4 trace the two unit hyperbolas: $(\cosh b, \sinh b)$, where $t^2 - y^2 = 1$ (blue), and $(\sinh b, \cosh b)$, where $y^2 - t^2 = 1$ (orange); the dotted diagonal is the **light line** $t = y$.

```
for b_value, alpha in ((0.0, 1.0), (0.5, 0.7), (1.0, 0.45)):
    c_b, s_b = np.cosh(b_value), np.sinh(b_value)
    ax.annotate("", xy=(c_b, s_b), xytext=(0, 0), arrowprops=dict(
        arrowstyle="-|>", color="tab:blue", alpha=alpha, linewidth=1.6))
    ax.annotate("", xy=(s_b, c_b), xytext=(0, 0), arrowprops=dict(
        arrowstyle="-|>", color="tab:orange", alpha=alpha, linewidth=1.6))
    ax.text(c_b + 0.04, s_b - 0.12, f"$b = {b_value:g}$", fontsize=8,
            color="tab:blue")
    ax.text(s_b - 0.30, c_b + 0.06, f"$b = {b_value:g}$", fontsize=8,
            color="tab:orange")
```

For the rapidities $b = 0$, 0.5 and 1 two arrows are drawn from the origin with `annotate` (an annotation with an empty text and an arrow): the boosted time direction to $(\cosh b, \sinh b)$ and the boosted hidden direction to $(\sinh b, \cosh b)$, fainter for larger b (`alpha` is the opacity). `ax.text` writes the value of b next to each arrow tip (`{b_value:g}` writes the number in its shortest form).

```
ax.set_xlim(-0.4, 2.3)
ax.set_ylim(-0.25, 2.3)
ax.set_aspect("equal")
ax.set_xlabel("component along $e^{(x4)}$ (time), $t$")
ax.set_ylabel("component along $e^{(x0)}$ (hidden direction), $y$")
ax.set_title("A boost moves the frame along hyperbolas")
ax.legend(fontsize=8, loc="upper left")
```

The axis ranges, equal units on both axes, the labels and the legend.

```
save_figure(fig, "boost_hyperbolas",
            "A boost of the frame in the plane of the time $x4$ and the hidden "
            ...
            "and tilt symmetrically towards the light line $t = y$ (dotted).")
```

The figure is saved; the cell prints its file name.

What Figure 06b.1 shows. At $b = 0$ the two arrows point along the axes. As b grows, the blue arrow (the time direction) moves along its hyperbola away from the horizontal axis and the orange arrow (the hidden direction) along its hyperbola away from the vertical axis: both tilt symmetrically towards the light line, and both get longer on the paper, while their η-lengths stay -1 and $+1$. They remain perpendicular in the sense of η ($-\cosh b \sinh b + \sinh b \cosh b = 0$). A boost never brings the arrows back, unlike a rotation.

In [7], the boosted vielbein and its spin connection.

```
beta, b0 = sp.symbols("beta b0", real=True)  # the rate and the start of the rapidity
b = beta * x4 + b0  # the rapidity grows with the time
L = boost(b)
e_boosted = L * e  # e'^a_mu = sum_c Lambda^a_c e^c_mu
```

Two symbols, the rate β and the starting value b_0, and the rapidity $b = \beta x_4 + b_0$, which grows with the time. `e_boosted` is the matrix product $\Lambda(b)\, e$, the boosted vielbein $e'^{a}{}_{\mu} = \sum_c \Lambda^{a}{}_{c} e^{c}{}_{\mu}$.

```
check(matrix_is_zero(e_boosted.T * eta_matrix * e_boosted - g),
      "the boosted vielbein gives the author's metric (64 entries)",
      record=f"{SCOPE_PY}, check boosted_frame_reproduces_metric (b4 = 1)")
```

The check computes all 64 entries of $e'^{T} \eta e'$ and compares them with the author's metric.

```
omega_b = spin_connection(e_boosted, Gam, x, ETA)  # omega'_{mu ab}
postulate = [sp.diff(e_boosted[a, nu], x[mu])
             - sum(Gam[lam][mu][nu] * e_boosted[a, lam] for lam in range(8))
```

```
            + sum(ETA[a] * omega_b[mu][a][c] * e_boosted[c, nu] for c in range(8))
            for mu in range(8) for a in range(8) for nu in range(8)]
antisymmetric = all(is_zero(omega_b[mu][a][c] + omega_b[mu][c][a])
                    for mu in range(8) for a in range(8) for c in range(8))
check(all(is_zero(v) for v in postulate) and antisymmetric,
      "boosted vielbein: vielbein postulate (512) and antisymmetry of omega'",
      record=f"{SCOPE_PY}, check boosted_frame_canonical_connection (b4 = 1)")
```

The canonical spin connection of the boosted frame, computed by the general function (with the same Christoffel symbols `Gam`, because the metric is the same). Then the vielbein postulate in all 512 components and the antisymmetry, as in Notebook 06a, In [11], but with the boosted vielbein; one check for both. This is the slowest step of the notebook: every entry contains cosh and sinh of $\beta x_4 + b_0$, and sympy must simplify them.

```
Omega_b = spinor_connection(omega_b, S)  # Omega'_mu
gup_b = curved_gammas(e_boosted, gamma)  # gamma'^mu
count_b = sum(1 for mu in range(8) for a in range(8) for c in range(a + 1, 8)
              if omega_b[mu][a][c] != 0)
say(f"nonzero components omega'_mu ab with a < b: {count_b}")
```

The boosted spinor connection Ω'_μ, the boosted curved gammas γ'^μ, and the number of nonzero components $\omega'_{\mu ab}$ with $a < b$, printed: 13. The cell prints two PASS lines and this number.

In [8], the contraction in the boosted frame.

```
slash_b = sum((gup_b[mu] * Omega_b[mu] for mu in range(8)), Z16)
v = sp.cosh(b) * gamma[0] - sp.sinh(b) * gamma[4]  # R gamma^(x0) R^-1 (Section 11)
w = sp.cosh(b) * gamma[4] - sp.sinh(b) * gamma[0]  # R gamma^(x4) R^-1
formula = (6 * H - beta) / 2 * v + c_b4 * w
check(matrix_is_zero(slash_b - formula),
      "gamma'^mu Omega'_mu = ((6H - beta)/2) v + (3/4)(b4'/b4) w",
      record=f"{SCOPE_PY}, check boosted_frame_gammaOmega_formula (b4 = 1)")
```

`slash_b` is $\sum_\mu \gamma'^\mu \Omega'_\mu$; `v` and `w` are the matrices $\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}$ and $\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)}$ of Section 6.16 (In [12] checks that they are $R\gamma^{(x0)}R^{-1}$ and $R\gamma^{(x4)}R^{-1}$), and `formula` is $\frac{6H-\beta}{2}v + \frac{3}{4}\frac{b'_4}{b_4}w$, which at $b_4 = 1$ is the record's formula. The check requires them to be equal for symbolic a_4, b_4, H, β and b_0.

```
check(matrix_is_zero(formula.subs({beta: 0, b0: 0}) - 3 * H * gamma[0]
                     - c_b4 * gamma[4])
      and matrix_is_zero(at_b4_1(formula) - (6 * H - beta) / 2 * v),
      "with no boost the formula is 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); at "
      "b4 = 1 it is ((6H - beta)/2) v")
```

With $\beta = b_0 = 0$ the formula must give back $3H\gamma^{(x0)} + \frac{3}{4}(b'_4/b_4)\gamma^{(x4)}$ (`.subs` with a dictionary replaces both symbols), and at $b_4 = 1$ it must be $\frac{6H-\beta}{2}v$. Two PASS lines.

In [9], figure 2.

```
times = np.linspace(-0.25, 0.25, 101)
t_symbol = sp.Symbol("t", real=True)
coefficient_x0 = (6 * H - beta) / 2 * sp.cosh(b)  # the coefficient of gamma^(x0)
coefficient_x4 = -(6 * H - beta) / 2 * sp.sinh(b)  # the coefficient of gamma^(x4)
```

101 times from -0.25 to 0.25, a plain symbol `t` for the time, and the two coefficients of the formula on the background $b_4 = 1$: of $\gamma^{(x0)}$, $\frac{6H-\beta}{2}\cosh b$, and of $\gamma^{(x4)}$, $-\frac{6H-\beta}{2}\sinh b$.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for rate, style in ((0, "-"), (3, "--"), (6, "-."), (9, ":")):
    values = {H: 1, beta: rate, b0: 0, x4: t_symbol}
    c0 = sp.lambdify(t_symbol, coefficient_x0.subs(values))
    c4 = sp.lambdify(t_symbol, coefficient_x4.subs(values))
    ax_left.plot(times, np.broadcast_to(c0(times), times.shape), style,
                 label=f"$\\beta = {rate}H$" if rate else "no boost")
    ax_right.plot(times, np.broadcast_to(c4(times), times.shape), style,
                  label=f"$\\beta = {rate}H$" if rate else "no boost")
```

For the four rates $\beta = 0, 3, 6, 9$ (with $H = 1$ and $b_0 = 0$, so $b = \beta x_4$), each with its own line style, the two coefficients become numerical functions of the time and are plotted, the first on the left and the second on the right. `np.broadcast_to` makes a constant into a list of 101 values (for $\beta = 6$ both coefficients are the constant 0, for $\beta = 0$ the $\gamma^{(x0)}$ coefficient is the constant 3). The label is "no boost" for the rate 0 (the number 0 counts as false in `if rate`).

```
ax_left.set_title("coefficient of $\\gamma^{(x0)}$")
ax_right.set_title("coefficient of $\\gamma^{(x4)}$")
for ax in (ax_left, ax_right):
    ax.set_xlabel("time $x_4$ (units $1/H$), with $b = \\beta x_4$")
    ax.legend(fontsize=8)
ax_left.set_ylabel("coefficient in $\\gamma'^\\mu\\Omega'_\\mu$ (units of $H$)")
```

Titles, axis labels and legends.

```
save_figure(fig, "boosted_contraction",
            "The contraction $\\gamma'^\\mu\\Omega'_\\mu$ in the boosted vielbein with "
            ...
            "the same metric, a different frame, a different value. A changing "
            "$b_4$ would add $\\frac34(b_4'/b_4)(\\cosh b\\,\\gamma^{(x4)} - "
            "\\sinh b\\,\\gamma^{(x0)})$.")
```

The figure is saved; the cell prints its file name.

What Figure 06b.2 shows. Without boost (solid) the coefficients are the constants 3 and 0: the value $3H\gamma^{(x0)}$ of the diagonal vielbein at $b_4 = 1$. For $\beta = 3H$ (dashed) the $\gamma^{(x0)}$ coefficient is about $1.5\cosh(3x_4)$, smaller and slightly curved, and a $\gamma^{(x4)}$ coefficient appears. For $\beta = 6H$ (dash-dotted) both coefficients are zero at every time. For $\beta = 9H$ (dotted) they change sign. The same metric, a different frame, a different value of $\gamma^\mu\Omega_\mu$.

In [10], the rate $\beta = 6H$.

```
six_H = {beta: 6 * H}
slash_6H = slash_b.subs(six_H)
Omega_6H = [M.subs(six_H) for M in Omega_b]
nonzero = [NAMES[mu] for mu in range(8) if not matrix_is_zero(Omega_6H[mu])]
say(f"Omega'_mu is nonzero for mu = {nonzero}")
```

`six_H` replaces β by $6H$; substituting after the computation is allowed, because the computation was done for every β. `nonzero` lists the directions μ whose Ω'_μ is not the zero matrix and is printed: `['x1', 'x2', 'x3', 'x4', 'x5', 'x6', 'x7']`.

```
check(matrix_is_zero(slash_6H - c_b4 * w.subs(six_H))
      and matrix_is_zero(at_b4_1(slash_6H))
      and nonzero == NAMES[1:],
      "beta = 6H: gamma'^mu Omega'_mu = (3/4)(b4'/b4) w (zero for constant b4), "
      "Omega'_mu nonzero for x1 ... x7",
      record=f"{SCOPE_PY}, check boosted_frame_gammaOmega_vanishes (b4 = 1)")
```

The check requires the contraction to be $\frac{3}{4}(b_4'/b_4)\,w$ (with $\beta = 6H$ put into `w` too), to vanish identically at $b_4 = 1$, and Ω'_μ to be nonzero exactly for $x1, \ldots, x7$ (`NAMES[1:]` is the list of the names after the first). One line and one PASS line.

In [11], figure 3.

```
def at_sample(expr):
    """The decimal value of expr at H = 1, b0 = 0, x4 = 1/6, z = pi/4, a4 = 1/2,
    a4' = 1/2, a4'' = 0, b4 = 1 (derivatives replaced first, then a4 and b4, then
    x4)."""
    expr = at_b4_1(expr).subs(sp.Derivative(a4, (x4, 2)), 0)
    expr = expr.subs(sp.Derivative(a4, x4), sp.Rational(1, 2))
    expr = expr.subs(a4, sp.Rational(1, 2))
    expr = expr.subs({H: 1, b0: 0, x4: sp.Rational(1, 6), x0: sp.pi / 24})
    return float(sp.N(expr))
```

The sample point of this notebook: $H = 1$, $b_0 = 0$, $x_4 = \frac{1}{6}$ (so that $b = 6Hx_4 = 1$), $z = \pi/4$, $a_4 = \frac{1}{2}$, $a_4' = \frac{1}{2}$, $a_4'' = 0$, on the background $b_4 = 1$ (`at_b4_1` first), with the derivatives replaced first (Section 6.14, In [10] of Notebook 06a, explains why). `float` gives an ordinary decimal number.

```
def sample_table(matrix):
    return np.array([[at_sample(v) for v in row] for row in matrix.tolist()])
```

`sample_table` turns a matrix into an array of its decimal values at the sample point.

```
panels = [("$\\Omega'_{x4}$", Omega_6H[4]), ("$\\Omega'_{x1}$", Omega_6H[1]),
          ("$\\gamma'^\\mu\\Omega'_\\mu$ (all zero)", slash_6H)]
tables = [sample_table(M) for _, M in panels]
```

The three matrices of the figure: Ω'_{x4}, Ω'_{x1} and the contraction, and their tables.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.6))
for ax, (title, _), table in zip(axes, panels, tables):
    limit = max(np.abs(table).max(), 1.0)  # each panel its own colour scale
    image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.8)
```

Three heat maps side by side, each with its own symmetric colour scale; `max(..., 1.0)` keeps the scale at least ± 1, so that the zero matrix is drawn white and not with an undefined scale.

```
save_figure(fig, "boosted_connection_heat_maps",
            "The boosted vielbein with the rate $\\beta = 6H$ at the sample point "
            ...
            "zero matrix. The spinor connection is present, its contraction is not.")
```

The figure is saved.

```
check(np.abs(tables[2]).max() < 1e-12 and np.abs(tables[0]).max() > 0.1,
      "at the sample point Omega'_x4 is nonzero and the contraction is zero")
```

A numerical check of what the picture shows: the largest entry of the contraction is below 10^{-12}, and the largest entry of Ω'_{x4} is above 0.1. The cell prints the file name and one PASS line.

What Figure 06b.3 shows. At the sample point $b_4 = 1$. The left panel, Ω'_{x4}, is coloured, although $\Omega_{x4} = 0$ in the diagonal vielbein: in the boosted frame the frame turns with the time, at the rate $\beta = 6H$, and $\Omega'_{x4} = 3H\gamma^{(x4)}\gamma^{(x0)}$ (Section 6.16), a signed permutation matrix times 3 whose squares lie in the two diagonal blocks. The middle panel, Ω'_{x1}, is coloured too. The right panel is white: the contraction of the two is the zero matrix. The spinor connection is present; its contraction is not.

In [12], why: the explicit spinor boost (Sections 6.15 and 6.16).

```
X = gamma[4] * gamma[0]  # gamma^(x4) gamma^(x0)
check(X * X == I16, "X = gamma^(x4) gamma^(x0) squares to the identity")
R = sp.cosh(b / 2) * I16 - sp.sinh(b / 2) * X  # the spinor boost
R_inv = sp.cosh(b / 2) * I16 + sp.sinh(b / 2) * X
check(matrix_is_zero(R * R_inv - I16), "R R^-1 = 1")
```

$X = \gamma^{(x4)}\gamma^{(x0)}$ and the check $X^2 = 1$; the spinor boost $R = \cosh\frac{b}{2} - \sinh\frac{b}{2}X$ and its inverse $\cosh\frac{b}{2} + \sinh\frac{b}{2}X$, and the check $RR^{-1} = 1$.

```
covers = all(matrix_is_zero(R_inv * gamma[a] * R
                            - sum((L[a, c] * gamma[c] for c in range(8)), Z16))
             for a in range(8))
check(covers, "R^-1 gamma^a R = Lambda^a_c gamma^c: R covers the boost")
```

The covering relation $R^{-1}\gamma^a R = \sum_c \Lambda^a{}_c\gamma^c$ for all eight a (`L[a, c]` is $\Lambda^a{}_c$ for the rapidity $b = \beta x_4 + b_0$).

```
gammas_turn = all(matrix_is_zero(gup_b[mu] - R * gup[mu] * R_inv) for mu in range(8))
connection_rule = all(matrix_is_zero(Omega_b[mu] - (R * Omega[mu] * R_inv
                                                    - R.diff(x[mu]) * R_inv))
                      for mu in range(8))
check(gammas_turn and connection_rule,
      "gamma'^mu = R gamma^mu R^-1 and Omega'_mu = R Omega_mu R^-1 - (d_mu R) R^-1")
```

The theorem of Section 6.15 for this boost: $\gamma'^{\mu} = R\gamma^{\mu}R^{-1}$ and $\Omega'_{\mu} = R\Omega_{\mu}R^{-1} - (\partial_{\mu}R)R^{-1}$ for all eight μ (`R.diff(x[mu])` differentiates every entry of R).

```
turned = R * slash * R_inv
extra = sum((gup_b[mu] * R.diff(x[mu]) * R_inv for mu in range(8)), Z16)
check(matrix_is_zero(R * gamma[0] * R_inv - v)
      and matrix_is_zero(R * gamma[4] * R_inv - w)
      and matrix_is_zero(turned - 3 * H * v - c_b4 * w)
      and matrix_is_zero(extra - beta / 2 * v),
      "R g0 R^-1 = v, R g4 R^-1 = w; turned term 3H v + (3/4)(b4'/b4) w; extra term "
      "(beta/2) v")
```

The two pieces of the formula: `turned` is $R\left(\sum_{\mu}\gamma^{\mu}\Omega_{\mu}\right)R^{-1}$ with the contraction `slash` of In [4], and `extra` is $\sum_{\mu}\gamma'^{\mu}(\partial_{\mu}R)R^{-1}$. The check requires $R\gamma^{(x0)}R^{-1} = v$, $R\gamma^{(x4)}R^{-1} = w$, `turned` $= 3Hv + \frac{3}{4}\frac{b'_4}{b_4}w$ and `extra` $= \frac{\beta}{2}v$, as derived by hand in Section 6.16. The cell prints five PASS lines.

In [13], the Riemann and Ricci tensors (Section 6.17).

```
def riemann_component(r, sg, mu, nu):
    value = sp.diff(Gam[r][nu][sg], x[mu]) - sp.diff(Gam[r][mu][sg], x[nu])
    for lam in range(8):
        value += Gam[r][mu][lam] * Gam[lam][nu][sg] - Gam[r][nu][lam] * Gam[lam][mu][sg]
    return value
```

`riemann_component(r, sg, mu, nu)` is $R^{r}{}_{\sigma\mu\nu}$ by the formula of Section 6.17 (`sg` stands for σ).

```
def is_nonzero(expr):
    """True when expr is not identically zero.  A value clearly different from zero
    at the sample point proves it (fast); otherwise the exact test decides."""
    if abs(at_sample(expr)) > 1e-9:
        return True
    return not is_zero(expr)
```

`is_nonzero(expr)` decides whether an expression is not identically zero. If its value at the sample point is clearly different from zero, that proves it at once (a function that is not zero at one point is not the zero function); otherwise the exact test decides.

```
riemann = {}  # (r, sigma, mu, nu) -> R^r_sigma mu nu, only the nonzero ones
for r in range(8):
    for sg in range(8):
        for mu in range(8):
```

```
            for nu in range(mu + 1, 8):
                value = riemann_component(r, sg, mu, nu)
                if value != 0 and is_nonzero(value):
                    riemann[(r, sg, mu, nu)] = value
                    riemann[(r, sg, nu, mu)] = -value
report("nonzero components R^r_sigma mu nu with mu < nu", len(riemann) // 2)
```

All components with $\mu < \nu$ are computed (the others follow from the antisymmetry $R^{\rho}{}_{\sigma\nu\mu} = -R^{\rho}{}_{\sigma\mu\nu}$, which the two assignments use). `value != 0` first skips the components that are zero without any work. The RESULT line prints the number of nonzero components with $\mu < \nu$: 78 (`//` is division without remainder; the dictionary holds every component twice).

```
ricci_lower = sp.Matrix(8, 8, lambda sg, nu: sum(
    riemann.get((r, sg, r, nu), 0) for r in range(8)))  # R_sigma nu
ricci_mixed = g.inv() * ricci_lower  # R^mu_nu = g^mu sigma R_sigma nu
```

The Ricci tensor $R_{\sigma\nu} = \sum_r R^{r}{}_{\sigma r\nu}$ as an 8×8 matrix (`sp.Matrix(8, 8, function)` fills each entry with the value of the function of its row and its column; `riemann.get(key, 0)` gives 0 for a missing key), and the mixed Ricci tensor $R^{\mu}{}_{\nu} = \sum_{\sigma} g^{\mu\sigma} R_{\sigma\nu}$ as the matrix product with the inverse metric.

```
names = {"H": H, "x0": x0, "x4": x4, "a4": a4, "b4": b4, "a4p": a4p, "b4p": b4p,
         "a4pp": sp.Derivative(a4, (x4, 2)), "b4pp": sp.Derivative(b4, (x4, 2))}


def wolfram_to_sympy(text):
    for long_form, short_form in (("Derivative[1][a4][x4]", "a4p"),
                                  ("Derivative[2][a4][x4]", "a4pp"),
                                  ("Derivative[1][b4][x4]", "b4p"),
                                  ("Derivative[2][b4][x4]", "b4pp"),
                                  ("a4[x4]", "a4"), ("b4[x4]", "b4")):
        text = text.replace(long_form, short_form)
    return parse_mathematica(text).subs(
        {sp.Symbol(k): value for k, value in names.items()}, simultaneous=True)
```

`names` says which symbol or function of the notebook each plain name of the records means, and `wolfram_to_sympy(text)` reads a text of a record in Wolfram notation, as the function of the same name in Notebook 06a, In [3], here with the fixed dictionary `names`.

```
metric_b4 = json.loads(repository_file(METRIC_B4_FILE).read_text(encoding="utf-8"))
ricci_b4 = {tuple(NAMES.index(n) for n in key.split(",")): wolfram_to_sympy(text)
            for key, text in metric_b4["ricciMixed"].items()}
nonzero_mixed = {(m, n) for m in range(8) for n in range(8)
                 if not is_zero(ricci_mixed[m, n])}
check(nonzero_mixed == set(ricci_b4)
      and all(is_zero(ricci_mixed[k] - ricci_b4[k]) for k in ricci_b4)
      and is_zero(sum(ricci_mixed[a, a] for a in range(8))
                  - wolfram_to_sympy(metric_b4["ricciScalar"])),
      "the mixed Ricci tensor (10 nonzero components, 2 off the diagonal) and R equal "
      "the record with b4",
      record=f"{METRIC_B4_FILE}, keys ricciMixed and ricciScalar")
```

The record `metric-b4.json` stores the mixed Ricci tensor of the metric with b_4 under the key `ricciMixed`, as a dictionary whose keys are texts such as `"x4,x0"` (first the upper index, then the lower one) and whose values are the nonzero components; `key.split(",")` cuts a key at the comma, and `NAMES.index` turns each name into its position. `nonzero_mixed` is the set of the positions at which the computed mixed Ricci tensor is not zero. The check requires the same ten positions, the same values, and the Ricci scalar, the sum of the eight diagonal entries, equal to the record's `ricciScalar`.

```
for key in sorted(ricci_b4):
    say(f"R^{NAMES[key[0]]}_{NAMES[key[1]]} = {ricci_b4[key]}")
FORMULAS = {item["key"]: item["wl"] for item in json.loads(
    repository_file(THEORY_FILE).read_text(encoding="utf-8"))["formulas"]}
```

The loop prints the ten components in the order of their positions (`sorted` orders the pairs by the first number, then by the second): ${R^{x0}}_{x0} = -6H^2$, ${R^{0}}_{x4} = -\frac{3}{2}H \tan z\, b_4'/b_4$, the three 3-space components, ${R^{x4}}_{x0} = \frac{3}{2}H \cot z\, b_4'/b_4$, ${R^{x4}}_{x4}$ and the three extra-time components, each in sympy's form. `FORMULAS` holds the formulas of the record of $b_4 = 1$.

```
ricci_record = wolfram_to_sympy(FORMULAS["ricci_mixed_diagonal"])
ricci_scalar = sum(ricci_mixed[a, a] for a in range(8))
check(all(is_zero(at_b4_1(ricci_mixed[a, a]) - ricci_record[a]) for a in range(8))
      and is_zero(at_b4_1(ricci_scalar) - wolfram_to_sympy(FORMULAS["ricci_scalar"])),
      "at b4 = 1: the eight Ricci components and R = 6 (a4'^2 - 7 H^2) equal the record",
      record=f"{THEORY_FILE}, formulas ricci_mixed_diagonal and ricci_scalar")
```

At $b_4 = 1$ (`at_b4_1`) the eight diagonal components are compared with the record's `ricci_mixed_diagonal`, $-6H^2$, three times $-6H^2 + a_4''$, $6a_4'^2$ and three times $-6H^2 - a_4''$, and the Ricci scalar with the record's $6(a_4'^2 - 7H^2)$.

```
# multiplied out, the entries of gamma^mu Omega_mu contain neither a4 nor its derivatives
a4_free = all(not sp.expand(entry).has(a4) for entry in slash)
check(a4_free and is_zero(ricci_mixed[0, 0] + 6 * H**2)
      and is_zero(at_b4_1(ricci_mixed[4, 4]) - 6 * a4p ** 2),
      "gamma^mu Omega_mu contains no a4, but R^x4_x4 = 6 a4'^2 at b4 = 1; "
      "R^x0_x0 = -6 H^2 for every b4",
      record=f"{SCOPE_PY}, check gammaOmega_blind_to_the_deflation (b4 = 1)")
```

The last check: multiplied out (`sp.expand`), no entry of the contraction `slash` of In [4] contains a_4 (`.has(a4)` is true also for an expression with a_4', which contains a_4), while ${R^{x0}}_{x0} = -6H^2$ for every b_4 and ${R^{x4}}_{x4} = 6a_4'^2$ at $b_4 = 1$. The cell prints one RESULT line, ten component lines and three PASS lines.

In [14], figure 4.

```
rate_symbol, accel_symbol = sp.symbols("rate accel", real=True)
numeric = [sp.lambdify((rate_symbol, accel_symbol), ricci_record[a].subs(
    {H: 1, sp.Derivative(a4, (x4, 2)): accel_symbol,
     sp.Derivative(a4, x4): rate_symbol})) for a in range(8)]
```

The eight Ricci components of the record of $b_4 = 1$ as numerical functions of two plain symbols, the rate a_4' and the acceleration a_4'', with $H = 1$.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
values = [float(numeric[a](0.5, 0.0)) for a in range(8)]
colors = ["black"] + ["tab:red"] * 3 + ["gray"] + ["tab:blue"] * 3
ax_left.bar(range(8), values, color=colors, width=0.6)
ax_left.axhline(0.0, color="black", linewidth=0.8)
ax_left.set_xticks(range(8), [f"$R^{{{n}}}{{}}_{{{n}}}$" for n in NAMES], fontsize=8)
ax_left.set_ylabel("value (units of $H^2$)")
ax_left.set_title("Ricci components at $a_4' = 1/2$, $a_4'' = 0$, $b_4 = 1$")
```

The left panel: a bar chart of the eight components at $a_4' = \frac{1}{2}$, $a_4'' = 0$, coloured by kind (black hidden direction, red 3-space, gray time, blue extra times), with labels such as $R^{x1}{}_{x1}$ under the bars.

```
rates = np.linspace(-3.0, 3.0, 121)
for a, label, style in ((0, "$R^{x0}{}_{x0} = -6H^2$", "-"),
                        (4, "$R^{x4}{}_{x4} = 6a_4'^2$", "--"),
                        (1, "$R^{x1}{}_{x1} = a_4'' - 6H^2$", ":")):
    ax_right.plot(rates, np.broadcast_to(numeric[a](rates, 0.0), rates.shape),
                  style, label=label)
ax_right.axhline(0.0, color="black", linewidth=0.8)
ax_right.set_xlabel("rate $a_4'$ (units of $H$), with $a_4'' = 0$, $b_4 = 1$")
ax_right.set_ylabel("value (units of $H^2$)")
ax_right.set_title("Never flat: $R^{x0}{}_{x0} = -6H^2$")
ax_right.legend(fontsize=8)
```

The right panel: three components versus the rate a_4' from -3 to 3 with $a_4'' = 0$: $R^{x0}{}_{x0}$ (solid), $R^{x4}{}_{x4}$ (dashed) and $R^{x1}{}_{x1}$ (dotted).

```
save_figure(fig, "ricci_components",
            "The mixed Ricci components $R^\\mu{}_\\mu$ of the author's metric on the "
            "background $b_4 = 1$, in units of $H^2$. Left: all eight at $H = 1$, "
            "$a_4' = 1/2$, $a_4^{\\prime\\prime} = 0$ (black: hidden direction, red: "
            "3-space, gray: time, blue: extra times). Right: $R^{x0}{}_{x0} = -6H^2$ "
            "(solid), $R^{x4}{}_{x4} = 6a_4'^2$ (dashed) and $R^{x1}{}_{x1} = "
            ...
            "is negative for every history (and every $b_4$), so the metric is never "
            "flat, and the time component is curvature made by the expansion and "
            "deflation (the dotted line lies on the solid one because "
            "$a_4^{\\prime\\prime} = 0$).")
```

The figure is saved; the cell prints its file name.

What Figure 06b.4 shows. On the background $b_4 = 1$. On the left, the six warped bars and the hidden bar lie at -6 (with $a_4'' = 0$ the 3-space and extra-time components are both $-6H^2$), while the time bar is $6 \cdot \frac{1}{4} = 1.5$. On the right, the solid line is flat at -6 for every rate: the hidden component is negative for every history (and every b_4), so the metric is never flat. The dashed parabola $6a_4'^2$ is curvature made by the inflation and deflation alone; it vanishes only for a constant a_4. The dotted line lies on the solid one because $a_4'' = 0$.

In [15], the spinor curvature (Section 6.17).

```
def spinor_curvature(Om, mu, nu):
    return (Om[nu].diff(x[mu]) - Om[mu].diff(x[nu])
            + Om[mu] * Om[nu] - Om[nu] * Om[mu])
```

`spinor_curvature(Om, mu, nu)` is $F_{\mu\nu} = \partial_\mu \Omega_\nu - \partial_\nu \Omega_\mu + [\Omega_\mu, \Omega_\nu]$ for any list of spinor connections `Om`.

```
F = {}
theorem_ok = True
for mu in range(8):
    for nu in range(mu + 1, 8):
        F[(mu, nu)] = spinor_curvature(Omega, mu, nu)
        from_riemann = sum((ETA[a] * f[a] * riemann[(a, c, mu, nu)] / f[c] * S[a][c] / 2
                            for a in range(8) for c in range(8)
                            if (a, c, mu, nu) in riemann), Z16)
        theorem_ok = theorem_ok and matrix_is_zero(F[(mu, nu)] - from_riemann)
check(theorem_ok and record_passed(REPORT_WL_B4, "spin_curvature_equals_Riemann"),
      "F_mu nu = (1/2) R_ab mu nu S^ab in all 28 coordinate planes",
      record=f"{REPORT_WL_B4}, check spin_curvature_equals_Riemann")
```

For the 28 planes $\mu < \nu$ of the diagonal vielbein, $F_{\mu\nu}$ is computed and compared with $\frac{1}{2}\sum_{a,c} R_{ac\mu\nu} S^{ac}$, where $R_{ac\mu\nu} = \eta_{aa} f_a R^a{}_{c\mu\nu} / f_c$ (the term is added only when the Riemann component is nonzero, `if (a, c, mu, nu) in riemann`). The check requires the theorem in all 28 planes, for every b_4, and the WolframScript report with b_4 to record the same.

```
F01_boosted = spinor_curvature(Omega_6H, 0, 1)
R_6H, R_inv_6H = R.subs(six_H), R_inv.subs(six_H)
check(matrix_is_zero(F01_boosted - R_6H * F[(0, 1)] * R_inv_6H)
      and np.abs(sample_table(F01_boosted)).max() > 0.1,
      "boosted (beta = 6H): F'_x0x1 = R F_x0x1 R^-1, nonzero",
      record=f"{SCOPE_PY}, check boosted_frame_curvature_nonzero (b4 = 1)")
```

In the boosted frame with $\beta = 6H$: $F'_{x0\,x1}$ is computed from the boosted spinor connection, R and R^{-1} are taken at $\beta = 6H$, and the check requires $F'_{x0x1} = R F_{x0x1} R^{-1}$ and an entry above 0.1 at the sample point, so that it is not zero. Two PASS lines with their records.

In [16], figure 5.

```
norms = np.zeros((8, 8))
for (mu, nu), matrix in F.items():
    norms[mu, nu] = norms[nu, mu] = np.sqrt((sample_table(matrix) ** 2).sum())
```

For each plane the size $\|F_{\mu\nu}\|$, the square root of the sum of the squares of its 256 entries at the sample point (`** 2` squares every entry, `.sum()` adds them), stored symmetrically in an 8×8 array.

```
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(norms, cmap="Purples")
for mu in range(8):
    for nu in range(8):
```

```
        if norms[mu, nu] > 0:
            # white digits on the dark squares (the largest sizes), black elsewhere
            dark = norms[mu, nu] > 0.6 * norms.max()
            ax.text(nu, mu, f"{norms[mu, nu]:.1f}", ha="center", va="center",
                    fontsize=7, color="white" if dark else "black")
ax.set_xticks(range(8), NAMES)
ax.set_yticks(range(8), NAMES)
ax.set_xlabel("coordinate $\\nu$")
ax.set_ylabel("coordinate $\\mu$")
ax.set_title("Size of the spinor curvature $\\|F_{\\mu\\nu}\\|$")
ax.grid(False)
fig.colorbar(image, ax=ax, shrink=0.8, label="$\\|F_{\\mu\\nu}\\|$ (units of $H^2$)")
```

A heat map in shades of purple (white is zero), with the size written into each nonzero square with one decimal, in white on the dark squares (above 60 percent of the largest size) and in black elsewhere, and a colour bar.

```
save_figure(fig, "spinor_curvature_norms",
            "The size $\\|F_{\\mu\\nu}\\|$, the square root of the sum of the squares "
            ...
            "change of frame removes.")
```

The figure is saved.

```
report("largest |F_mu nu| at the sample point (units of H^2)", f"{norms.max():.3f}")
curved = [pair for pair in F if norms[pair] > 1e-9]  # the curved planes
check(matrix_is_zero(F[(0, 4)]) and len(curved) == 27,
      "F_x0x4 = 0 exactly (every b4); the other 27 planes are curved at the sample "
      "point")
```

The RESULT line prints the largest size, 3.633 (in units of H^2). `curved` lists the planes with a size above 10^{-9}, and the check requires $F_{x0x4} = 0$ exactly and the other 27 planes to be curved at the sample point.

What Figure 06b.5 shows. At the sample point $b_4 = 1$. Every square off the diagonal is coloured except the two of the pair $(x0, x4)$. The largest sizes, about 3.6 and 3.5, lie in the planes within 3-space and between 3-space and the hidden direction; the planes of 3-space with the extra times have about 2.2, with the time about 1.7; the planes of the extra times among themselves about 0.5, with the time 0.6 and with the hidden direction 1.3. The pattern reflects the factors e^{a_4} and e^{-a_4} at $a_4 = \frac{1}{2}$. Whatever the numbers in another frame, $F' = RFR^{-1}$ can never turn a coloured square white: this curvature is a property of the metric.

In [17], the rescaling (Section 6.18).

```
w_r = sp.sin(z) ** sp.Rational(-1, 2) * b4 ** sp.Rational(-3, 4)  # the weight
bracket = sum((gup[mu] * sp.diff(w_r, x[mu]) for mu in range(8)), Z16) + w_r * slash
```

The weight $w = \sin^{-1/2}(z)\, b_4^{-3/4}$ (named `w_r` in the code) and the bracket $\sum_\mu \gamma^\mu \partial_\mu w + w \sum_\mu \gamma^\mu \Omega_\mu$ of Section 6.18.

```
chi = sp.Matrix([sp.Function(f"chi{A}")(*x) for A in range(1, 17)])
psi = w_r * chi
dirac_psi = sum((gup[mu] * (psi.diff(x[mu]) + Omega[mu] * psi) for mu in range(8)),
                sp.zeros(16, 1))
dirac_chi = sum((gup[mu] * chi.diff(x[mu]) for mu in range(8)), sp.zeros(16, 1))
check(matrix_is_zero(bracket) and all(is_zero(v) for v in dirac_psi - w_r * dirac_chi),
      "gamma^mu D_mu (w chi) = w gamma^mu d_mu chi for w = sin^(-1/2) z b4^(-3/4)",
      record=f"{SCOPE_PY}, check rescaling_removes_the_connection_term (b4 = 1)")
```

For safety the statement is also checked on sixteen completely general functions $\chi_1, \ldots, \chi_{16}$ of all eight coordinates: `dirac_psi` is $\sum_\mu \gamma^\mu D_\mu(w\chi)$ and `dirac_chi` is $\sum_\mu \gamma^\mu \partial_\mu \chi$; the check requires the bracket to vanish and $\sum_\mu \gamma^\mu D_\mu(w\chi) - w \sum_\mu \gamma^\mu \partial_\mu \chi$ to vanish in all sixteen components.

```
chi_conj = sp.Matrix([sp.conjugate(v) for v in chi])  # complex conjugates
S_psi = (w_r * chi_conj).T * C * (w_r * chi)  # Psi^dagger C Psi (w is real)
S_chi = chi_conj.T * C * chi
check(is_zero(S_psi[0, 0] - S_chi[0, 0] / (b4 ** sp.Rational(3, 2) * sp.sin(z))),
      "S[w chi] = S[chi] / (b4^(3/2) sin z), S[chi] / sin z at b4 = 1",
      record=f"{SCOPE_PY}, check rescaled_equation_quadratic_potential (b4 = 1)")
```

The bilinear: `chi_conj` is the column of the complex conjugates χ_A^*, so `(w * chi_conj).T * C * (w * chi)` is $\Psi^\dagger C \Psi$ for $\Psi = w\chi$ (the conjugate of the real w is w), a 1×1 matrix whose only entry `[0, 0]` is the number. The check requires $S[w\chi] = S[\chi]/(b_4^{3/2} \sin z)$. Two PASS lines with their records.

In [18], figure 6.

```
z_values = np.linspace(0.05, np.pi / 2 - 0.02, 300)
x0_symbol = sp.Symbol("u", positive=True)
w_1 = at_b4_1(w_r)  # the weight at b4 = 1: sin^(-1/2) z
term_derivative = sp.lambdify(x0_symbol, (sp.tan(z) * sp.diff(w_1, x0)).subs(
    {H: 1}).subs(x0, x0_symbol))
term_connection = sp.lambdify(x0_symbol, (3 * H * w_1).subs({H: 1}).subs(x0, x0_symbol))
weight = sp.lambdify(x0_symbol, w_1.subs({H: 1}).subs(x0, x0_symbol))
u_values = z_values / 6.0  # x0 = z / (6 H) with H = 1
first, second = term_derivative(u_values), term_connection(u_values)
```

300 values of z inside $(0, \pi/2)$; `w_1` is the weight at $b_4 = 1$, $\sin^{-1/2} z$; and three numerical functions of x_0 (a positive symbol `u` replaces x_0): the derivative term $\tan z\, \partial_0 w$, the connection term $3Hw$ and the weight w, all with $H = 1$ and $b_4 = 1$. With $H = 1$, $x_0 = z/6$.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
ax_left.plot(z_values, weight(u_values), label="$w_r = \\sin^{-1/2} z$ ($b_4 = 1$)")
ax_left.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_left.set_ylabel("weight (pure number)")
ax_left.set_title("The rescaling weight")
ax_left.legend(fontsize=8)
ax_right.plot(z_values, first, label="$\\tan z\\,\\partial_{x0} w_r$")
ax_right.plot(z_values, second, "--",
              label="$3H w_r$ (from $\\gamma^\\mu\\Omega_\\mu$)")
```

```
ax_right.plot(z_values, first + second, ":", color="black", label="sum")
ax_right.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax_right.set_ylabel("coefficient of $\\gamma^{(x0)}\\chi$ (units of $H$)")
ax_right.set_title("The two terms cancel")
ax_right.legend(fontsize=8)
```

The left panel draws w; the right panel the two terms and their sum.

```
save_figure(fig, "rescaling",
            "The rescaling $\\Psi = w_r\\chi$ with $w_r = \\sin^{-1/2}(z)\\,"
            "b_4^{-3/4}$, drawn for $H = 1$ on the background $b_4 = 1$. Left: the "
            "weight $w_r$ versus the hidden angle $z$ in radians. Right: the two "
            "terms of the coefficient of $\\gamma^{(x0)}\\chi$ in the field "
            "equation of $\\chi$, the derivative term $\\tan z\\,\\partial_{x0}w_r = "
            "-3Hw_r$ (solid) and the spin-connection term $3Hw_r$ (dashed), in units "
            "of $H$, and their sum (dotted), which is zero at every $z$: in the "
            ...
            "variable $\\chi$ the equation has no term $3H\\gamma^{(x0)}$ (and, by "
            "the factor $b_4^{-3/4}$, no term $\\frac34(b_4'/b_4)\\gamma^{(x4)}$).")
```

The figure is saved.

```
check(np.max(np.abs(first + second)) < 1e-12, "the two terms cancel at all 300 points")
```

The two terms cancel at all 300 points to better than 10^{-12}. The cell prints the file name and one PASS line.

What Figure 06b.6 shows. On the background $b_4 = 1$. On the left, the weight $w = \sin^{-1/2} z$ is large near the tip $z \to 0$ and falls to 1 at the patch end. On the right, the derivative term $\tan z\, \partial_0 w = -3Hw$ (solid) is the mirror image of the connection term $3Hw$ (dashed) about the zero line, and their sum (dotted) lies on zero everywhere: in the variable χ the field equation has no term $3H\gamma^{(x0)}$ (and, by the factor $b_4^{-3/4}$ of the weight, no term $\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$).

In [19], the last checks.

```
scope_names = ["boosted_frame_reproduces_metric", "boosted_frame_canonical_connection",
               "boosted_frame_gammaOmega_formula", "boosted_frame_gammaOmega_vanishes",
               "boosted_frame_curvature_nonzero",
               "rescaling_removes_the_connection_term",
               "rescaled_equation_quadratic_potential",
               "gammaOmega_blind_to_the_deflation"]
cited = {SCOPE_PY: scope_names, SCOPE_WL: scope_names,
         REPORT_PY: ["spinor_curvature_equals_riemann",
                     "curvature_nonzero_flat_only_formally"],
         REPORT_WL: ["ricci_scalar", "ricci_mixed_components",
                     "never_flat_for_H_positive", "spin_curvature_equals_Riemann"],
         REPORT_WL_B4: ["ricci_equals_metric_b4_set", "ricci_reduces_at_b4_1",
                        "spin_curvature_equals_Riemann"]}
count = sum(len(v) for v in cited.values())
check(all(record_passed(path, name) for path, names in cited.items()
          for name in names), f"the {count} cited record checks are recorded as passed")
```

`scope_names` lists the eight scope checks that this notebook reproduces; each appears in both scope reports, the sympy one and the WolframScript one. Together with two curvature checks of the sympy field-theory report and four of the WolframScript one (both of $b_4 = 1$), and three of the WolframScript report with b_4, that makes $8 + 8 + 2 + 4 + 3 = 25$ cited checks, and the check requires all of them to be recorded as passed.

```
expected = [f"06b_{k}_{name}.png" for k, name in enumerate([
    "boost_hyperbolas", "boosted_contraction", "boosted_connection_heat_maps",
    "ricci_components", "spinor_curvature_norms", "rescaling"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected),
      "all six figure files exist")
all_checks_passed()
```

The six figure files must exist, and the last line is printed: ALL 26 CHECKS PASSED (notebook 06b).

6.23 Upper and lower frame indices: the mixed contraction

The issue. The spin connection comes out of its formula with the first frame index up, $\omega_\mu{}^a{}_b$ (the mixed components, Section 6.4), and the spinor connection is built from the components with both frame indices down, $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$ with $\omega_{\mu ab} = \eta_{aa}\omega_\mu{}^a{}_b$. The Revision record states (in `Revision/SPEC.md`, section 3, and in `Revision/docs/DIRAC16COMPLEX_FIELD_THEORY.md`, section 4) that the author's Mathematica notebook contracts the mixed components with S^{ab} instead, that this contraction lacks one factor of the frame metric, and that the record does not use it. This book has not opened the author's notebook file; it relies on the record's statement, which Notebook 06c checks in `Revision/SPEC.md`. We call that contraction

$$\Omega^{nb}_\mu = \tfrac{1}{2}\sum_{a,b}\omega_\mu{}^a{}_b\,S^{ab}$$

("nb" for notebook) and derive exactly what the missing factor changes.

The derivation, line by line. Lowering is multiplication by η_{aa}, and $\eta_{aa}\eta_{aa} = 1$, so $\omega_\mu{}^a{}_b = \eta_{aa}\omega_{\mu ab}$ (multiply $\omega_{\mu ab} = \eta_{aa}\omega_\mu{}^a{}_b$ by η_{aa}). Hence

$$\Omega^{nb}_\mu = \tfrac{1}{2}\sum_{a,b}\eta_{aa}\,\omega_{\mu ab}\,S^{ab}.$$

The terms with $a = b$ vanish ($S^{aa} = 0$). Every other term belongs to exactly one unordered pair $\{a, b\}$ with $a < b$, which contributes two terms:

$$\eta_{aa}\,\omega_{\mu ab}S^{ab} + \eta_{bb}\,\omega_{\mu ba}S^{ba} = (\eta_{aa} + \eta_{bb})\,\omega_{\mu ab}S^{ab}$$

(both factors of the second term change sign when a and b are exchanged, $\omega_{\mu ba}S^{ba} = (-\omega_{\mu ab})(-S^{ab})$). So

$$\Omega_\mu^{nb} = \sum_{a<b} \tfrac{1}{2}(\eta_{aa}+\eta_{bb})\,\omega_{\mu ab} S^{ab}, \qquad \text{while} \qquad \Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$$

(the same steps without η, Section 6.7). The factor $\frac{1}{2}(\eta_{aa}+\eta_{bb})$ is $+1$ for a pair of two space-like directions (**space-space**), -1 for a pair of two time-like directions (**time-time**) and 0 for a boost pair. In the author's frame metric there are $\binom{4}{2} = 6$ space-space pairs among $x0, x1, x2, x3$, 6 time-time pairs among $x4, x5, x6, x7$, and $4 \cdot 4 = 16$ boost pairs. Writing Ω_μ^{ss}, Ω_μ^{tt} and Ω_μ^{st} for the parts of Ω_μ that come from the three kinds of pairs:

$$\Omega_\mu^{nb} = \Omega_\mu^{ss} - \Omega_\mu^{tt}, \qquad \Omega_\mu = \Omega_\mu^{ss} + \Omega_\mu^{tt} + \Omega_\mu^{st}.$$

The mixed contraction keeps the 6 space-space parts, reverses the sign of the 6 time-time parts and deletes all 16 boost parts. In a space whose metric is positive (every $\eta_{aa} = +1$) the factor is always $+1$ and the two contractions agree: in the polar plane of Section 6.5 both give $\Omega_\varphi = -\frac{i}{2}\sigma_3$. The mistake cannot be seen in the usual textbook examples; in signature (4,4) it deletes 16 of the 28 parts.

Where it is total: the Milne wedge. Take the flat plane with one time t and one space direction y, $ds^2 = -dt^2 + dy^2$, and in the part $t > |y|$ the coordinates $\tau = \sqrt{t^2 - y^2}$ and θ with $t = \tau\cosh\theta$, $y = \tau\sinh\theta$. Line by line, $dt = \cosh\theta\, d\tau + \tau\sinh\theta\, d\theta$ and $dy = \sinh\theta\, d\tau + \tau\cosh\theta\, d\theta$ (the product rule and $\frac{d}{d\theta}\cosh\theta = \sinh\theta$, $\frac{d}{d\theta}\sinh\theta = \cosh\theta$), so

$$-dt^2 + dy^2 = -(\cosh^2\theta - \sinh^2\theta)\, d\tau^2 + \tau^2(\cosh^2\theta - \sinh^2\theta)\, d\theta^2 = -d\tau^2 + \tau^2 d\theta^2$$

(square, add; the cross terms $\mp 2\tau\cosh\theta\sinh\theta\, d\tau\, d\theta$ cancel; $\cosh^2 - \sinh^2 = 1$). This part of the flat plane is called the **Milne wedge**. Its frame metric is $\eta = \mathrm{diag}(-1, +1)$ (frame index 0 the unit time direction, 1 the unit space direction) and its vielbein factors are $f_\tau = 1$, $f_\theta = \tau$. By the formula of Section 6.6, $\omega_{\theta\,10} = \eta_{11}\,\partial_\tau f_\theta / f_\tau = 1$, so $\omega_{\theta\,01} = -1$, and the mixed components are

$$\omega_\theta{}^0{}_1 = \eta_{00}\,\omega_{\theta 01} = (-1)(-1) = 1, \qquad \omega_\theta{}^1{}_0 = \eta_{11}\,\omega_{\theta 10} = 1 :$$

symmetric, as for every boost pair (Section 6.4). Two real 2×2 matrices with $(\gamma^0)^2 = -1$, $(\gamma^1)^2 = +1$ and $\gamma^0\gamma^1 + \gamma^1\gamma^0 = 0$ are $\gamma^0 = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ and $\gamma^1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (multiply out to check), with $\gamma^0\gamma^1 = \mathrm{diag}(1, -1)$. Then

$$\Omega_\theta = \omega_{\theta 01} S^{01} = -\tfrac{1}{2}\gamma^0\gamma^1, \qquad \Omega_\theta^{nb} = \tfrac{1}{2}(\omega_\theta{}^0{}_1 S^{01} + \omega_\theta{}^1{}_0 S^{10}) = \tfrac{1}{2}(S^{01} - S^{01}) = 0.$$

The plane is flat, but the frame is boosted by the rapidity θ as one moves along θ, and the correct spinor connection records this: $\Omega_\theta = -(\partial_\theta U)U^{-1}$ for the spinor boost $U(\theta) = \cosh\frac{\theta}{2} + \sinh\frac{\theta}{2}\,\gamma^0\gamma^1 = \mathrm{diag}(e^{\theta/2}, e^{-\theta/2})$ (since $\partial_\theta U = \frac{1}{2}\gamma^0\gamma^1 U$). As in the polar plane of Section 6.5, the rule $\Omega' = R\,\Omega R^{-1} - (\partial_\theta R)R^{-1}$ of Section 6.15 with $R = U$ makes this Ω_θ from the connection 0 of the frame of t and y, so U carries that frame into the Milne frame; the inverse boost $U^{-1} = \mathrm{diag}(e^{-\theta/2}, e^{\theta/2})$ removes the connection: $\gamma^0\gamma^1$ commutes with U, so $U^{-1}\Omega_\theta U = \Omega_\theta$, and $(\partial_\theta U^{-1})U = -\frac{1}{2}\gamma^0\gamma^1$, hence

$\Omega'_\theta = -\frac{1}{2}\gamma^0\gamma^1 + \frac{1}{2}\gamma^0\gamma^1 = 0$. The mixed contraction deletes the whole connection. It also violates the divergence form of Section 6.9: with $\sqrt{|g|} = \tau$, $\gamma^\tau = \gamma^0$ and $\gamma^\theta = \gamma^1/\tau$,

$$\frac{1}{2\tau}\Big(\partial_\tau(\tau\gamma^0) + \partial_\theta(\tau\cdot\tfrac{\gamma^1}{\tau})\Big) = \frac{\gamma^0}{2\tau}, \qquad \gamma^\theta\Omega_\theta = \frac{\gamma^1}{\tau}\Big(-\tfrac{1}{2}\gamma^0\gamma^1\Big) = \frac{1}{2\tau}\gamma^0\gamma^1\gamma^1 = \frac{\gamma^0}{2\tau}$$

(in the second: exchange γ^1 and γ^0 at the cost of a sign; $(\gamma^1)^2 = 1$). The correct value equals the divergence form; the mixed contraction gives 0.

The author's metric. Of the 12 components of Section 6.6 the three $(x0, x_i)$ are space-space pairs (kept), the three $(x4, x_t)$ are time-time pairs (sign reversed), and the three $(x_i, x4)$ and the three $(x0, x_t)$ are boost pairs (deleted). So, with $r = a'_4 - b'_4/(2b_4)$ and $f_t = e^{-a_4}b_4^{1/2}s$,

$$\Omega^{nb}_{x_i} = e^{a_4}s\,H\,S^{x_i\,x0}, \qquad \Omega^{nb}_{x_t} = +f_t\,r\,S^{x4\,x_t},$$

compared with $\Omega_{x_i} = e^{a_4}s(a'_4S^{x_ix4} + HS^{x_ix0})$ and $\Omega_{x_t} = -f_t(r\,S^{x4x_t} + HS^{x_tx0})$ (Section 6.7). The contraction, by the steps of Section 6.8:

$$\gamma^{x_i}\Omega^{nb}_{x_i} = \frac{\gamma^{(x_i)}}{f_i}\,f_iH\,\tfrac{1}{2}\gamma^{(x_i)}\gamma^{(x0)} = \tfrac{H}{2}\gamma^{(x0)}, \qquad \gamma^{x_t}\Omega^{nb}_{x_t} = \frac{\gamma^{(x_t)}}{f_t}\,f_t\,r\,\tfrac{1}{2}\gamma^{(x4)}\gamma^{(x_t)} = \tfrac{r}{2}\gamma^{(x4)}$$

($(\gamma^{(x_i)})^2 = 1$; and $\gamma^{(x_t)}\gamma^{(x4)}\gamma^{(x_t)} = \gamma^{(x4)}$ as in Section 6.8). Summed over the six warped directions:

$$\sum_\mu \gamma^\mu\Omega^{nb}_\mu = \tfrac{3H}{2}\gamma^{(x0)} + \tfrac{3}{2}\Big(a'_4 - \frac{b'_4}{2b_4}\Big)\gamma^{(x4)} \quad \text{instead of} \quad \sum_\mu \gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}\,\frac{b'_4}{b_4}\,\gamma^{(x4)};$$

at $b_4 = 1$, $\frac{3H}{2}\gamma^{(x0)} + \frac{3a'_4}{2}\gamma^{(x4)}$ instead of $3H\gamma^{(x0)}$. Half of the hidden term is lost, and the a'_4 terms along the time no longer cancel: with the mixed contraction the deflation would appear in this term.

The gammas are no longer covariantly constant. Replace Ω_μ by Ω^{nb}_μ in $D_\mu\gamma^\nu$:

$$D^{nb}_\mu\gamma^\nu = \partial_\mu\gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega^{nb}_\mu, \gamma^\nu] = D_\mu\gamma^\nu + [\Omega^{nb}_\mu - \Omega_\mu, \gamma^\nu] = [\Omega^{nb}_\mu - \Omega_\mu, \gamma^\nu]$$

(add and subtract $[\Omega_\mu, \gamma^\nu]$; then $D_\mu\gamma^\nu = 0$, Section 6.7). By the vector rule, $[S^{ab}, \gamma^c]$ is nonzero only when c is a or b. (The counting below is written at $b_4 = 1$; for a general b_4 replace a'_4 in the extra-time parts by $r = a'_4 - b'_4/(2b_4)$, which changes no count.) For $\mu = x1$ the difference is $\Omega^{nb}_{x1} - \Omega_{x1} = -f_1a'_4S^{x1x4}$, the deleted boost part, and

$$[\Omega^{nb}_{x1} - \Omega_{x1}, \gamma^{x1}] = -a'_4[S^{x1x4}, \gamma^{(x1)}] = -a'_4(\eta^{x4\,x1}\gamma^{(x1)} - \eta^{x1x1}\gamma^{(x4)}) = a'_4\gamma^{(x4)}$$

(the factor f_1 of the difference cancels against the $1/f_1$ of γ^{x1}; then the vector rule $[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a - \eta^{ac}\gamma^b$ with $\eta^{x4x1} = 0$ and $\eta^{x1x1} = 1$), and in the same way, with $\eta^{x4x4} = -1$,

$$[\Omega^{nb}_{x1} - \Omega_{x1}, \gamma^{x4}] = -f_1a'_4(\eta^{x4x4}\gamma^{(x1)} - \eta^{x1x4}\gamma^{(x4)}) = f_1a'_4\gamma^{(x1)}.$$

So for each 3-space direction two pairs (μ,ν) are violated. For $\mu = x5$ the difference is $2f_5 a_4' S^{x4x5} + f_5 H S^{x5x0}$ (the time-time part counted twice, because its sign was reversed, and the deleted boost part), and the same rule gives $[\ldots,\gamma^{x5}] = -2a_4'\gamma^{(x4)} + H\gamma^{(x0)}$, $[\ldots,\gamma^{x4}] = 2f_5 a_4'\gamma^{(x5)}$ and $[\ldots,\gamma^{x0}] = \tan z\, f_5 H\gamma^{(x5)}$: three violated pairs for each extra time. Altogether $3\cdot 2 + 3\cdot 3 = 15$ of the 64 pairs. Each violated matrix is a multiple of one gamma (16 nonzero entries), except $-2a_4'\gamma^{(x4)} + H\gamma^{(x0)}$, which has 32 because $\gamma^{(x4)}$ and $\gamma^{(x0)}$ have their nonzero entries in different places (the tables of Chapter 5); so there are $6\cdot 16 + 3(16+32+16) = 288$ nonzero entries. Notebook 06c finds exactly these numbers (In [13]; COMPUTED exactly by the notebook, not a Revision record).

statement	**status**	**where it is verified**
$\Omega^{nb}_\mu = \sum_{a<b} \frac{1}{2}(\eta_{aa} + \eta_{bb})\omega_{\mu ab} S^{ab} = \Omega^{ss}_\mu - \Omega^{tt}_\mu$	PROVED (above)	Notebook 06c, In [4] and In [8] (its own exact checks)
polar plane: the contractions agree; Milne wedge: $\Omega_\theta = -\frac{1}{2}\gamma^0\gamma^1$, $\Omega^{nb} = 0$	PROVED (above)	Notebook 06c, In [6]
the record does not use the mixed contraction	the record's statement	`Revision/SPEC.md`, section 3 (checked as text by Notebook 06c, In [2])
$\gamma^\mu \Omega^{nb}_\mu = \frac{3H}{2}\gamma^{(x0)} + \frac{3}{2}(a_4' - \frac{b_4'}{2b_4})\gamma^{(x4)}$; 15 violated pairs, 288 entries	PROVED above and COMPUTED exactly by Notebook 06c; not a Revision result (it describes the contraction the record does not use)	Notebook 06c, In [10] and In [13]

6.24 Why only the value $3H$ is right

Two facts single out the correct value $3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ ($3H\gamma^{(x0)}$ at $b_4 = 1$) among all candidates.

The divergence form. Section 6.9 showed that the correct contraction equals $\frac{1}{2\sqrt{|g|}}\sum_\mu \partial_\mu(\sqrt{|g|}\gamma^\mu)$, a quantity built from the volume factor and the vielbein alone. The mixed contraction, $\frac{3H}{2}\gamma^{(x0)} + \frac{3}{2}(a_4' - \frac{b_4'}{2b_4})\gamma^{(x4)}$, does not equal it.

The hidden-direction operator. For fields that depend on x_0 only, the part of the Dirac operator along the hidden direction is $\tan z\,\gamma^{(x0)}\partial_0 + c\,\gamma^{(x0)}$, where c is the coefficient of $\gamma^{(x0)}$ in the contraction ($c = 3H$ for the correct one, $c = \frac{3}{2}H$ for the mixed one). Write $A_c q = \tan z\,\partial_0 q + c\,q$ for a function $q(x_0)$. An operator A is called **antisymmetric for the weight** $w(x_0)$ when $\int w\,p\,(Aq)\,dx_0 = -\int w\,(Ap)\,q\,dx_0$ for all functions p, q, up to **boundary terms** (values at the ends of the interval). This is the property that makes iA a Hermitian operator in quantum theory (Chapter 10), and the natural weight is the volume factor $\sqrt{|g|} = b_4^{3/2}\cos z$; along the hidden direction $b_4^{3/2}$ is a constant factor, which multiplies every line below and is left out, so the weight is $\cos z$. Compute, line by line, for two functions $p(x_0)$, $q(x_0)$ (primes are derivatives by x_0 here):

$$\partial_0(\sin z\, p\, q) = 6H\cos z\; p\, q + \sin z\,(p' q + p\, q')$$

(the product rule; $\partial_0 \sin z = 6H \cos z$),

$$\cos z\,(p\,A_c q + (A_c p)\,q) = \cos z \tan z\,(p\,q' + p' q) + 2c \cos z\; p\,q = \sin z\,(p\,q' + p' q) + 2c \cos z\; p\,q$$

(insert A_c; $\cos z \tan z = \sin z$). Subtracting the first line from the second:

$$\cos z\,(p\,A_c q + (A_c p)\,q) - \partial_0(\sin z\,p\,q) = (2c - 6H)\cos z\; p\,q.$$

For $c = 3H$ the right side vanishes: integrated over x_0, $\int \cos z\,p\,A_{3H} q\,dx_0 = -\int \cos z\,(A_{3H} p)\,q\,dx_0 + [\sin z\,p\,q]$, so A_{3H} is antisymmetric for the weight $\cos z$ up to the boundary term. This is the record's formula `hidden_direction_hermiticity`. For the mixed value $c = \frac{3}{2}H$ the term $-3H \cos z\,p\,q$ remains, which is not a boundary term (it has no derivative in front), so no boundary condition can remove it. For $c = 0$ (no spin-connection term at all) the leftover would be $-6H \cos z\,p\,q$.

Scope of this argument. The boundary term $[\sin z\,p\,q]$ vanishes at the tip $z \to 0$ but not at the patch end $z = \pi/2$, where $\sin z = 1$; there a boundary condition is needed, which Chapter 14 supplies by the ASSUMED Z_2 mirror (the choice of boundary condition is the one point of this chapter that is left OPEN here). And the role of $3H$ depends on the variables: in the rescaled field χ of Section 6.18, with the proper distance y of Chapter 3 as coordinate and the weight dy, the operator ∂_y has the same antisymmetry without any H term (the record says so in the same formula). PROVED; `field-theory.json`, formula `hidden_direction_hermiticity` (written at $b_4 = 1$); `wolfram-field-theory.json`, check `good_sector_hermiticity_curved`; both scope reports, check `good_sector_hermiticity_up_to_the_brane_flux`; Notebook 06c, In [15].

6.25 Example: Notebook 06c

Notebook 06c checks Sections 6.23 and 6.24 with exact algebra, for arbitrary functions $a_4(x_4)$ and $b_4(x_4)$. It reads the author's gammas and the formulas of the records (with b_4, and at $b_4 = 1$), and checks that `Revision/SPEC.md` contains the record's statement about the mixed contraction. It defines the general formulas again, now with the MIXED spin connection, a separate lowering function and a contraction over all ordered pairs. It sorts the 28 frame pairs into their three kinds and checks the key step of the derivation for all of them; works the polar plane and the Milne wedge; computes, for the author's metric, the correct and the mixed spinor connections and their contractions direction by direction; counts where the covariant constancy of the gammas breaks; and checks the divergence form and the antisymmetry of the hidden-direction operator for two arbitrary functions. Every correct result is compared with the Revision records; the results about the mixed contraction are the notebook's own and are labelled so. It draws six figures, needs no Rust, runs in less than a minute and ends with the line ALL 27 CHECKS PASSED (notebook 06c).

6.26 How to run Notebook 06c

Step 1. What this notebook does and what it needs.

Notebook 06c (One missing metric factor: the mixed contraction of the spin connection) is the file `Revision/textbook/notebooks/06c_mixed_contraction.ipynb` of the repository Dirac_claude.

The Revision record builds the spinor connection from the spin connection with its first frame index lowered by the frame metric eta, and records that the author's Mathematica notebook contracts the mixed components instead, which lacks this metric factor. This notebook proves exactly, with sympy, what the missing factor does: it deletes the 16 boost components of every spin connection and reverses the sign of the 6 time-time components; it changes nothing in a space with a positive metric (the polar plane) and deletes the whole connection of a flat plane with one time (the Milne wedge); in the author's metric it would give gamma^mu Omega_mu = (3 H/2) gamma^(x0) + (3/2)(a4' - b4'/(2 b4)) gamma^(x4) instead of 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4), break the covariant constancy of the gammas in 15 of the 64 pairs, and spoil the antisymmetry of the hidden-direction operator that the Revision record proves for the correct value 3 H. It checks the correct results against the Revision records and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 06c_mixed_contraction.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 45 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 06c_mixed_contraction.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 06c_mixed_contraction.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/06c.captions.json`
- `Revision/textbook/figures/06c_1_pair_kinds.png`
- `Revision/textbook/figures/06c_2_milne_frame_and_spin_boost.png`
- `Revision/textbook/figures/06c_3_per_direction_comparison.png`
- `Revision/textbook/figures/06c_4_contraction_heat_maps.png`
- `Revision/textbook/figures/06c_5_constancy_violation.png`
- `Revision/textbook/figures/06c_6_hermiticity_defect.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 27 CHECKS PASSED (notebook 06c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/06c_mixed_contraction.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 06c_mixed_contraction.ipynb
python -m nbconvert --execute --inplace 06c_mixed_contraction.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for Revision/algebra/gammas.json, Revision/SPEC.md or a report: the notebook reads Revision records from the repository; open it inside the folder Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a single downloaded notebook file is not enough).
- a cell runs for several minutes: the exact algebra of sympy is slow on old computers; the whole notebook needs less than a minute on a 2024 laptop, and up to about two minutes while other programs use the processor. Wait, or close the other programs.

6.27 Notebook 06c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/06c_mixed_contraction.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The spin connection $\omega$ carries one coordinate index and two frame indices.
The Revision record builds the spinor connection from the components with BOTH
frame indices down,

$$\Omega_\mu = \tfrac12 \sum_{a,b} \omega_{\mu ab} S^{ab}, \qquad
\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b ,$$

and it records (Revision/SPEC.md, section 3) that the author's Mathematica
notebook contracts the **mixed** components $\omega_\mu{}^a{}_b$ (first frame index
up) with $S^{ab}$ instead; that contraction lacks one factor of the frame metric
$\eta$ and is not used. This notebook shows, exactly, what the missing factor does.
We call the notebook's contraction

$$\Omega^{nb}_\mu = \tfrac12 \sum_{a,b} \omega_\mu{}^a{}_b S^{ab}$$

("nb" for notebook). The notebook

- proves that $\Omega^{nb}_\mu$ keeps the 6 space-space parts of $\Omega_\mu$,
  reverses the sign of the 6 time-time parts and DELETES the 16 boost parts;
- shows that the two contractions agree in a space whose metric is positive (the
  flat plane in polar coordinates), and that in a flat plane with one time (the
  **Milne wedge**) the notebook's contraction deletes the whole connection;
- computes, for the author's metric with $a_4(x_4)$ and $b_4(x_4)$, what the
  notebook's contraction would give: $\gamma^\mu\Omega^{nb}_\mu =
  \frac{3H}{2}\gamma^{(x0)} + \frac32(a_4' - \frac{b_4'}{2b_4})\gamma^{(x4)}$
  instead of the correct $3H\gamma^{(x0)} + \frac34\frac{b_4'}{b_4}\gamma^{(x4)}$
  (at $b_4 = 1$: $\frac{3H}{2}\gamma^{(x0)} + \frac{3a_4'}{2}\gamma^{(x4)}$
  instead of $3H\gamma^{(x0)}$);
- counts where it breaks the covariant constancy $D_\mu\gamma^\nu = 0$ of the
  gammas (15 of the 64 pairs $(\mu, \nu)$);
- shows the physical reason why only the value $3H$ is right: it makes the
  hidden-direction operator antisymmetric for the volume weight
  $\sqrt{|g|} = b_4^{3/2}\cos z$, as the Revision record proves (at $b_4 = 1$);
- draws six teaching plots.

Every correct result is compared with the Revision record. The results about
$\Omega^{nb}$ are computed by this notebook itself; they describe what the
notebook's contraction WOULD give, and the book uses them nowhere else.
```

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 06c (One missing metric factor: the mixed contraction of the spin connection) is the file ‘Revision/textbook/notebooks/06c_mixed_contraction.ipynb‘ of the repository Dirac_claude. The Revision record builds the spinor connection from the spin connection with its first frame index lowered by the frame metric eta, and records that the author’s Mathematica notebook contracts the mixed components instead, which lacks this metric factor. This notebook proves exactly, with sympy, what the missing factor does: it deletes the 16 boost components of every spin connection and reverses the sign of the 6 time-time components; it changes nothing in a space with a positive metric (the polar plane) and deletes the whole connection of a flat plane with one time (the Milne wedge); in the author’s metric it would give gamma^mu Omega_mu = (3 H/2) gamma^(x0) + (3/2)(a4’ - b4’/(2 b4)) gamma^(x4) instead of 3 H gamma^(x0) + (3/4)(b4’/b4) gamma^(x4), break the covariant constancy of the gammas in 15 of the 64 pairs, and spoil the antisymmetry of the hidden-direction operator that the Revision record proves for the correct value 3 H. It checks the correct results against the Revision records and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation

does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
```

```
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 06c_mixed_contraction.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 45 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the

notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 06c_mixed_contraction.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 06c_mixed_contraction.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/06c.captions.json‘
- ‘Revision/textbook/figures/06c_1_pair_kinds.png‘
- ‘Revision/textbook/figures/06c_2_milne_frame_and_spin_boost.png‘
- ‘Revision/textbook/figures/06c_3_per_direction_comparison.png‘
- ‘Revision/textbook/figures/06c_4_contraction_heat_maps.png‘
- ‘Revision/textbook/figures/06c_5_constancy_violation.png‘
- ‘Revision/textbook/figures/06c_6_hermiticity_defect.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 27 CHECKS PASSED (notebook 06c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it

again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/06c_mixed_contraction.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 06c_mixed_contraction.ipynb
python -m nbconvert --execute --inplace 06c_mixed_contraction.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for Revision/algebra/gammas.json, Revision/SPEC.md or a report: the notebook reads Revision records from the repository; open it inside the folder Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a single downloaded notebook file is not enough).
- a cell runs for several minutes: the exact algebra of sympy is slow on old computers; the whole notebook needs less than a minute on a 2024 laptop, and up to about two minutes while other programs use the processor. Wait, or close the other programs.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 06c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 06c (One missing metric factor: the mixed contraction of the spin connection)
# is the file Revision/textbook/notebooks/06c_mixed_contraction.ipynb of the repository
# Dirac_claude. The Revision record builds the spinor connection from the spin connection
# with its first frame index lowered by the frame metric eta, and records that the
# author's Mathematica notebook contracts the mixed components instead, which lacks this
# metric factor. This notebook proves exactly, with sympy, what the missing factor does:
# it deletes the 16 boost components of every spin connection and reverses the sign of
# the 6 time-time components; it changes nothing in a space with a positive metric (the
# polar plane) and deletes the whole connection of a flat plane with one time (the Milne
# wedge); in the author's metric it would give gamma^mu Omega_mu = (3 H/2) gamma^(x0) +
# (3/2)(a4' - b4'/(2 b4)) gamma^(x4) instead of 3 H gamma^(x0) + (3/4)(b4'/b4)
# gamma^(x4), break the covariant constancy of the gammas in 15 of the 64 pairs, and
# spoil the antisymmetry of the hidden-direction operator that the Revision record proves
# for the correct value 3 H. It checks the correct results against the Revision records
# and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
```

```
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

```
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 06c_mixed_contraction.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 45
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 06c_mixed_contraction.ipynb
```

```
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 06c_mixed_contraction.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/06c.captions.json
# - Revision/textbook/figures/06c_1_pair_kinds.png
# - Revision/textbook/figures/06c_2_milne_frame_and_spin_boost.png
# - Revision/textbook/figures/06c_3_per_direction_comparison.png
# - Revision/textbook/figures/06c_4_contraction_heat_maps.png
# - Revision/textbook/figures/06c_5_constancy_violation.png
# - Revision/textbook/figures/06c_6_hermiticity_defect.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 27 CHECKS PASSED (notebook 06c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/06c_mixed_contraction.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
```

```
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 06c_mixed_contraction.ipynb
#     python -m nbconvert --execute --inplace 06c_mixed_contraction.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for Revision/algebra/gammas.json, Revision/SPEC.md or a report:
#   the notebook reads Revision records from the repository; open it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository Dirac_claude (a
#   single downloaded notebook file is not enough).
# - a cell runs for several minutes: the exact algebra of sympy is slow on old computers;
#   the whole notebook needs less than a minute on a 2024 laptop, and up to about two
#   minutes while other programs use the processor. Wait, or close the other programs.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "06c"  # this notebook: chapter 06, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
```

```
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
```

```
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 06c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x0, \dots, x7$ with the author's names: $x0$ the **hidden
```

```
  direction**; $x1, x2, x3$ 3-space; $x4$ the time; $x5, x6, x7$ the three
  **extra times**; $z = 6Hx_0$. In Python lists they have the positions 0 to 7 in
  the same order.
- **Frame metric** $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the order
  $x0, \dots, x7$: $\eta_{aa} = +1$ for a **space-like** frame direction, $-1$ for a
  **time-like** one. Because $\eta$ is diagonal with entries $\pm1$,
  $\eta_{aa}\eta_{aa} = 1$.
- **Upper and lower frame index.** The spin connection comes out of its formula
  with the first frame index UP, $\omega_\mu{}^a{}_b$: these are the **mixed
  components**. **Lowering** the index means multiplying by $\eta$:
  $\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b$ (no sum). The lowered
  components are **antisymmetric**, $\omega_{\mu ba} = -\omega_{\mu ab}$.
- $S^{ab} = \frac14[\gamma^a, \gamma^b]$: the 28 generators (one for each pair
  $a < b$); $S^{ba} = -S^{ab}$ and $S^{aa} = 0$.
- **Kinds of pairs** $\{a, b\}$ of two different frame directions:
  **space-space** (both space-like), **time-time** (both time-like) and
  **boost pair** (one space-like and one time-like). A boost pair generates a
  boost, the other two kinds generate rotations.
- **Spinor connection** $\Omega_\mu$ (correct) and $\Omega^{nb}_\mu$ (the
  notebook's contraction). $\Omega^{ss}_\mu$, $\Omega^{tt}_\mu$, $\Omega^{st}_\mu$:
  the parts of $\Omega_\mu$ that come from the space-space, time-time and boost
  pairs.
- **Defining property** of a spinor connection: $[\Omega_\mu, \gamma^a] =
  -\sum_b \omega_\mu{}^a{}_b\gamma^b$; **covariant constancy** of the curved gammas
  $\gamma^\mu = \sum_a e_a{}^\mu\gamma^a$: $D_\mu\gamma^\nu = \partial_\mu\gamma^\nu
  + \sum_\lambda\Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega_\mu, \gamma^\nu]
  = 0$.
- **Milne wedge**: the part $t > |y|$ of the flat plane with one time $t$ and one
  space direction $y$, $ds^2 = -dt^2 + dy^2$, described by $\tau =
  \sqrt{t^2 - y^2}$ and $\theta$ with $t = \tau\cosh\theta$, $y = \tau\sinh\theta$;
  its metric is $ds^2 = -d\tau^2 + \tau^2 d\theta^2$.
- **Spinor boost**: a matrix $U$ that turns spinors when the frame is boosted.
- **Antisymmetric for a weight**: an operator $A$ acting on functions of $x_0$ is
  antisymmetric for the weight $w(x_0)$ when $\int w\,p\,(Aq)\,dx_0 = -\int
  w\,(Ap)\,q\,dx_0$ for all functions $p, q$, up to boundary terms.
- **Exact** check: sympy proves that an expression is zero for ALL values of its
  symbols. **Heat map**: a picture of a matrix, one coloured square per entry.
```

Text cell:

```
## 4. The physical and mathematical situation

**The derivation, line by line.** Start from the notebook's contraction,

$$\Omega^{nb}_\mu = \tfrac12 \sum_{a,b} \omega_\mu{}^a{}_b S^{ab} .$$

Lowering is multiplication by $\eta_{aa}$, and $\eta_{aa}\eta_{aa} = 1$, so
$\omega_\mu{}^a{}_b = \eta_{aa}\,\omega_{\mu ab}$ (multiply
$\omega_{\mu ab} = \eta_{aa}\omega_\mu{}^a{}_b$ by $\eta_{aa}$):

$$\Omega^{nb}_\mu = \tfrac12 \sum_{a,b} \eta_{aa}\,\omega_{\mu ab} S^{ab} .$$

The terms with $a = b$ vanish ($S^{aa} = 0$). Every other term belongs to exactly
```

```
one unordered pair $\{a, b\}$ with $a < b$, which contributes two terms:

$$\eta_{aa}\,\omega_{\mu ab}S^{ab} + \eta_{bb}\,\omega_{\mu ba}S^{ba} =
(\eta_{aa} + \eta_{bb})\,\omega_{\mu ab}S^{ab} ,$$

because $\omega_{\mu ba}S^{ba} = (-\omega_{\mu ab})(-S^{ab}) = \omega_{\mu ab}
S^{ab}$ (both factors change sign). Hence

$$\Omega^{nb}_\mu = \sum_{a<b} \tfrac12(\eta_{aa} + \eta_{bb})\,
\omega_{\mu ab}S^{ab} ,$$

while the same steps without $\eta$ give the correct $\Omega_\mu = \sum_{a<b}
\omega_{\mu ab}S^{ab}$. The factor $\frac12(\eta_{aa} + \eta_{bb})$ is $+1$ for a
space-space pair, $-1$ for a time-time pair and $0$ for a boost pair. So

$$\Omega^{nb}_\mu = \Omega^{ss}_\mu - \Omega^{tt}_\mu, \qquad
\Omega_\mu = \Omega^{ss}_\mu + \Omega^{tt}_\mu + \Omega^{st}_\mu .$$

In a space whose metric is positive (every $\eta_{aa} = +1$) the factor is always
$+1$ and the two contractions are the same: the mistake cannot be seen in the usual
textbook examples. In signature (4,4) it deletes 16 of the 28 parts.

**Status.** The identity above is PROVED here (exact algebra). The values of
$\Omega^{nb}$ below are COMPUTED exactly by this notebook for the author's metric
and describe the contraction that the Revision record does NOT use. The correct
values ($\Omega_\mu$, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} +
\frac34(b_4'/b_4)\gamma^{(x4)}$, $D_\mu\gamma^\nu = 0$, the antisymmetry of the
hidden-direction operator) reproduce the Revision records.
```

Text cell:

```
## 5. The tools: sympy, the author's gammas and the record

The next cell imports numpy (numbers for the plots) and sympy (exact algebra),
reads the author's eight gamma matrices from `Revision/algebra/gammas.json`, checks
the 64 Clifford relations and that they are eight real 16 x 16 matrices, builds
the matrices $S^{ab}$, and defines the exact zero test `is_zero` (first a fast route
through exponential functions, then sympy's `simplify`; it never calls a nonzero
expression zero), `matrix_is_zero` and `record_passed` (does a Revision report
record a check as passed?). It reads the formulas of
`Revision/theory/field-theory.json` and checks that `Revision/SPEC.md` (section 3)
contains the statement about the notebook's contraction.

The first lines of the next cell change one setting of Jupyter: printed text is
sent to the screen when a cell has finished (or just before a figure is shown),
not every 0.2 seconds. This changes no result; it makes the printed text arrive in
the same pieces in every run, which the book's checking tool needs.
```

In [2]:

```
import sys  # the Python system module (here: the stream of printed text)
```

```
# Jupyter sends printed text to the screen in pieces, about every 0.2 seconds.  The
# book's checking tool reads each PASS line together with its "reproduces" line, so
# the pieces must be the same in every run: send the printed text of a cell in one
# piece when the cell ends (or just before a figure), at the latest after 600 s.
if hasattr(sys.stdout, "flush_interval"):  # true inside Jupyter only
    sys.stdout.flush_interval = 600.0
import numpy as np  # decimal numbers (only for the plots)
import sympy as sp  # exact algebra and calculus with symbols

fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
NAMES = fixture["coordinates"]  # ["x0", ..., "x7"]
ETA = fixture["eta"]  # +1 space-like, -1 time-like
gamma = [sp.Matrix(rows) for rows in fixture["gamma"]]  # gamma[a] = gamma^(xa)
I16, Z16 = sp.eye(16), sp.zeros(16, 16)  # identity and zero matrix
check(all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
          == (2 * ETA[a] if a == b else 0) * I16
          for a in range(8) for b in range(8)),
      "the 64 Clifford relations of the author's gammas",
      record="Revision/theory/reports/python-field-theory.json, "
             "check clifford_relations")
check(len(gamma) == 8 and all(M.shape == (16, 16) for M in gamma)
      and all(entry.is_real for M in gamma for entry in M),
      "the eight gammas are 16 x 16 matrices with real entries",
      record="Revision/theory/reports/python-field-theory.json, check gammas_real")
S = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
     for a in range(8)]  # S[a][b] = S^ab


def is_zero(expr):
    """True when expr is exactly zero for all values of its symbols."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    if sp.cancel(sp.expand(expr.rewrite(sp.exp))) == 0:  # the fast route
        return True
    return sp.simplify(expr) == 0  # the slower general route


def matrix_is_zero(matrix):
    """True when every entry of the matrix is exactly zero."""
    return all(is_zero(entry) for entry in matrix)


def record_passed(path, name):
    """True when the Revision report at path records the check name as passed."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper() == "PASS"  # "PASS" or "pass"
    raise KeyError(f"{path} has no check {name}")


THEORY_FILE = "Revision/theory/field-theory.json"  # the record at b4 = 1
```

```
THEORY_B4_FILE = "Revision/theory/field-theory-b4.json"  # the record with b4(x4)
FORMULAS = {item["key"]: item["wl"] for item in json.loads(
    repository_file(THEORY_FILE).read_text(encoding="utf-8"))["formulas"]}
FORMULAS_B4 = {item["key"]: item["wl"] for item in json.loads(
    repository_file(THEORY_B4_FILE).read_text(encoding="utf-8"))["formulas"]}
REPORT_PY = "Revision/theory/reports/python-field-theory.json"
REPORT_WL = "Revision/theory/reports/wolfram-field-theory.json"
REPORT_WL_B4 = "Revision/theory/reports/wolfram-field-theory-b4.json"
# " ".join(text.split()) replaces every run of blanks and line breaks by one blank,
# so a sentence that the file breaks across two lines is found as one line.
spec_text = " ".join(repository_file("Revision/SPEC.md").read_text(
    encoding="utf-8").split())
statement = "contraction of the mixed omega_mu^a_b, which lacks a metric factor"
check(statement in spec_text,
      "Revision/SPEC.md section 3 records the missing metric factor")
```

Out [2]:

```
PASS the 64 Clifford relations of the author's gammas
     reproduces Revision/theory/reports/python-field-theory.json, check
    clifford_relations
PASS the eight gammas are 16 x 16 matrices with real entries
     reproduces Revision/theory/reports/python-field-theory.json, check gammas_real
PASS Revision/SPEC.md section 3 records the missing metric factor
```

Text cell:

The next cell defines the general formulas as Python functions. They work in any number of dimensions, for any vielbein matrix ‘e‘ (row = frame index, column = coordinate; its inverse ‘E = e.inv()‘ has ‘E[nu, b]‘ $= e_b{}^\nu$).

- ‘christoffel(g, coords)‘: $\Gamma^l{}_{mn}$, computed once for each pair $m \le n$.
- ‘mixed_spin_connection(e, Gam, coords)‘: the MIXED components $\omega_\mu{}^a{}_b = \sum_\nu e^a{}_\nu(\partial_\mu e_b{}^\nu + \sum_\lambda\Gamma^\nu{}_{\mu\lambda}e_b{}^\lambda)$, exactly as the formula gives them.
- ‘lower_first_index(mixed, eta)‘: $\omega_{\mu ab} = \eta_{aa}\, \omega_\mu{}^a{}_b$.
- ‘contract(coefficients, gens)‘: $\frac12\sum_{a,b}c_{\mu ab}\,S^{ab}$ for every μ, summed over ALL ordered pairs (a, b). (Notebook 06a summed over $a < b$ only, which is allowed for the antisymmetric lowered components; the mixed components are not antisymmetric, so here the full sum is needed.)
- ‘curved_gammas(e, gammas)‘: $\gamma^\mu = \sum_a e_a{}^\mu\gamma^a$.
- ‘generators(gammas)‘: the matrices $S^{ab} = \frac14[\gamma^a, \gamma^b]$.

In [3]:

```
def christoffel(g, coords):
    """Gam[l][m][n] = (1/2) sum_r g^lr (d_m g_rn + d_n g_rm - d_r g_mn)."""
    n = len(coords)
```

```
    g_inv = g.inv()  # the inverse metric
    Gam = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for l in range(n):
        for m in range(n):
            for k in range(m, n):  # Gamma^l_mk = Gamma^l_km: compute it once
                value = sum(g_inv[l, r] * (sp.diff(g[r, k], coords[m])
                                           + sp.diff(g[r, m], coords[k])
                                           - sp.diff(g[m, k], coords[r]))
                            for r in range(n) if g_inv[l, r] != 0) / 2
                Gam[l][m][k] = Gam[l][k][m] = sp.simplify(value)
    return Gam


def mixed_spin_connection(e, Gam, coords):
    """mixed[mu][a][b] = omega_mu^a_b (first frame index up)."""
    n = len(coords)
    E = e.inv()  # E[nu, b] = e_b^nu, the inverse vielbein
    mixed = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for mu in range(n):
        for a in range(n):
            for b in range(n):
                value = 0
                for nu in range(n):
                    if e[a, nu] == 0:
                        continue  # this term is zero
                    value += e[a, nu] * (sp.diff(E[nu, b], coords[mu]) + sum(
                        Gam[nu][mu][lam] * E[lam, b] for lam in range(n)))
                mixed[mu][a][b] = sp.simplify(value)
    return mixed


def lower_first_index(mixed, eta):
    """omega[mu][a][b] = eta_aa omega_mu^a_b (eta is diagonal)."""
    n = len(mixed)
    return [[[eta[a] * mixed[mu][a][b] for b in range(n)] for a in range(n)]
            for mu in range(n)]


def contract(coefficients, gens):
    """(1/2) sum over ALL ordered pairs (a, b) of coefficients[mu][a][b] S^ab."""
    n = len(coefficients)
    size = gens[0][0].shape[0]  # 16 for the author's gammas, 2 in a plane
    return [sum((coefficients[mu][a][b] * gens[a][b] / 2 for a in range(n)
                 for b in range(n) if coefficients[mu][a][b] != 0),
                sp.zeros(size, size)) for mu in range(n)]


def curved_gammas(e, gammas):
    """gamma^mu = sum_a e_a^mu gamma^a for every coordinate mu."""
    E = e.inv()
    n = len(gammas)
    return [sum((E[mu, a] * gammas[a] for a in range(n)),
                sp.zeros(*gammas[0].shape)) for mu in range(n)]
```

```
def generators(gammas):
    """S[a][b] = (1/4)(gamma^a gamma^b - gamma^b gamma^a)."""
    n = len(gammas)
    return [[(gammas[a] * gammas[b] - gammas[b] * gammas[a]) / 4 for b in range(n)]
            for a in range(n)]


say("defined: christoffel, mixed_spin_connection, lower_first_index, contract, "
    "curved_gammas, generators")
```

Out [3]:

```
defined: christoffel, mixed_spin_connection, lower_first_index, contract, curved_gammas,
    generators
```

Text cell:

```
## 6. The three kinds of frame pairs

The next cell computes the factor $\frac12(\eta_{aa} + \eta_{bb})$ of Section 4 for
every pair of frame directions of the author's frame metric, counts the 28 pairs
$a < b$ by kind (the expected counts are 6 space-space pairs among $x0, x1, x2,
x3$, 6 time-time pairs among $x4, x5, x6, x7$, and $4 \cdot 4 = 16$ boost pairs),
and checks the key step of the derivation as a matrix identity for all 28 pairs:
$\eta_{aa}\,w\,S^{ab} + \eta_{bb}\,(-w)\,S^{ba} = (\eta_{aa} + \eta_{bb})\,w\,S^{ab}$
for a symbol $w$ (which stands for $\omega_{\mu ab}$; $-w$ is $\omega_{\mu ba}$).
```

In [4]:

```
KIND = {1: "space-space", -1: "time-time", 0: "boost"}  # the factor and its name
factor = [[sp.Rational(ETA[a] + ETA[b], 2) for b in range(8)] for a in range(8)]
pairs = [(a, b) for a in range(8) for b in range(a + 1, 8)]  # the 28 pairs a < b
counts = {k: sum(1 for a, b in pairs if factor[a][b] == k) for k in KIND}
for k, name in KIND.items():
    report(f"number of {name} pairs (factor {k})", counts[k])
check(len(pairs) == 28 and counts == {1: 6, -1: 6, 0: 16},
      "28 pairs: 6 space-space (+1), 6 time-time (-1), 16 boost pairs (0)")
w = sp.Symbol("w")  # stands for omega_mu ab; then omega_mu ba = -w
check(all(matrix_is_zero(ETA[a] * w * S[a][b] + ETA[b] * (-w) * S[b][a]
                         - (ETA[a] + ETA[b]) * w * S[a][b]) for a, b in pairs),
      "eta_aa w S^ab + eta_bb (-w) S^ba = (eta_aa + eta_bb) w S^ab, all 28 pairs")
```

Out [4]:

```
RESULT number of space-space pairs (factor 1) = 6
RESULT number of time-time pairs (factor -1) = 6
RESULT number of boost pairs (factor 0) = 16
PASS 28 pairs: 6 space-space (+1), 6 time-time (-1), 16 boost pairs (0)
PASS eta_aa w S^ab + eta_bb (-w) S^ba = (eta_aa + eta_bb) w S^ab, all 28 pairs
```

Text cell:

```
The next cell draws the factor as an 8 x 8 grid: the square in row $a$ and column
$b$ shows what the notebook's contraction does to the part of the pair $\{a, b\}$:
green for KEPT (factor $+1$, space-space), orange for SIGN REVERSED (factor $-1$,
time-time) and gray for DELETED (factor $0$, boost pairs); the diagonal ($a = b$)
is white, because $S^{aa} = 0$. The gray squares form two rectangles: every
space-like direction paired with every time-like direction.
```

In [5]:

```
from matplotlib.colors import ListedColormap  # a colour map with a few colours

grid = np.full((8, 8), 3)  # 3 = the diagonal (white)
for a in range(8):
    for b in range(8):
        if a != b:
            grid[a, b] = {1: 0, -1: 1, 0: 2}[int(factor[a][b])]  # colour number
colours = ListedColormap(["tab:green", "tab:orange", "lightgray", "white"])
fig, ax = plt.subplots(figsize=(6.4, 5.6))
ax.imshow(grid, cmap=colours, vmin=-0.5, vmax=3.5)
for a in range(8):
    for b in range(8):
        if a != b:
            ax.text(b, a, f"{int(factor[a][b]):+d}" if factor[a][b] else "0",
                    ha="center", va="center", fontsize=9)
ax.set_xticks(range(8), NAMES)
ax.set_yticks(range(8), NAMES)
ax.set_xlabel("frame direction $b$")
ax.set_ylabel("frame direction $a$")
ax.set_title("Factor $(\\eta_{aa} + \\eta_{bb})/2$ of the notebook's contraction")
ax.grid(False)
for colour, label in (("tab:green", "kept: space-space (+1)"),
                      ("tab:orange", "sign reversed: time-time (-1)"),
                      ("lightgray", "deleted: boost pair (0)")):
    ax.plot([], [], "s", color=colour, markersize=9, label=label)  # legend only
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.13), ncol=2, fontsize=8)
save_figure(fig, "pair_kinds",
            "What the missing metric factor does to each pair of frame directions "
            "of the author's frame metric. Rows $a$ and columns $b$ are the frame "
            "directions $x0$ to $x7$; each square shows the factor $(\\eta_{aa} + "
            "\\eta_{bb})/2$ by which the notebook's contraction multiplies the part "
            "of the pair: green $+1$, kept (both space-like: $x0, x1, x2, x3$); "
            "orange $-1$, sign reversed (both time-like: $x4, x5, x6, x7$); gray "
            "$0$, deleted (one space-like and one time-like direction, a boost "
            "pair); white diagonal, no pair. Of the 28 pairs 6 are kept, 6 "
            "reversed and 16 deleted.")
```

Out [5]:

```
Figure 06c.1 saved as Revision/textbook/figures/06c_1_pair_kinds.png
```

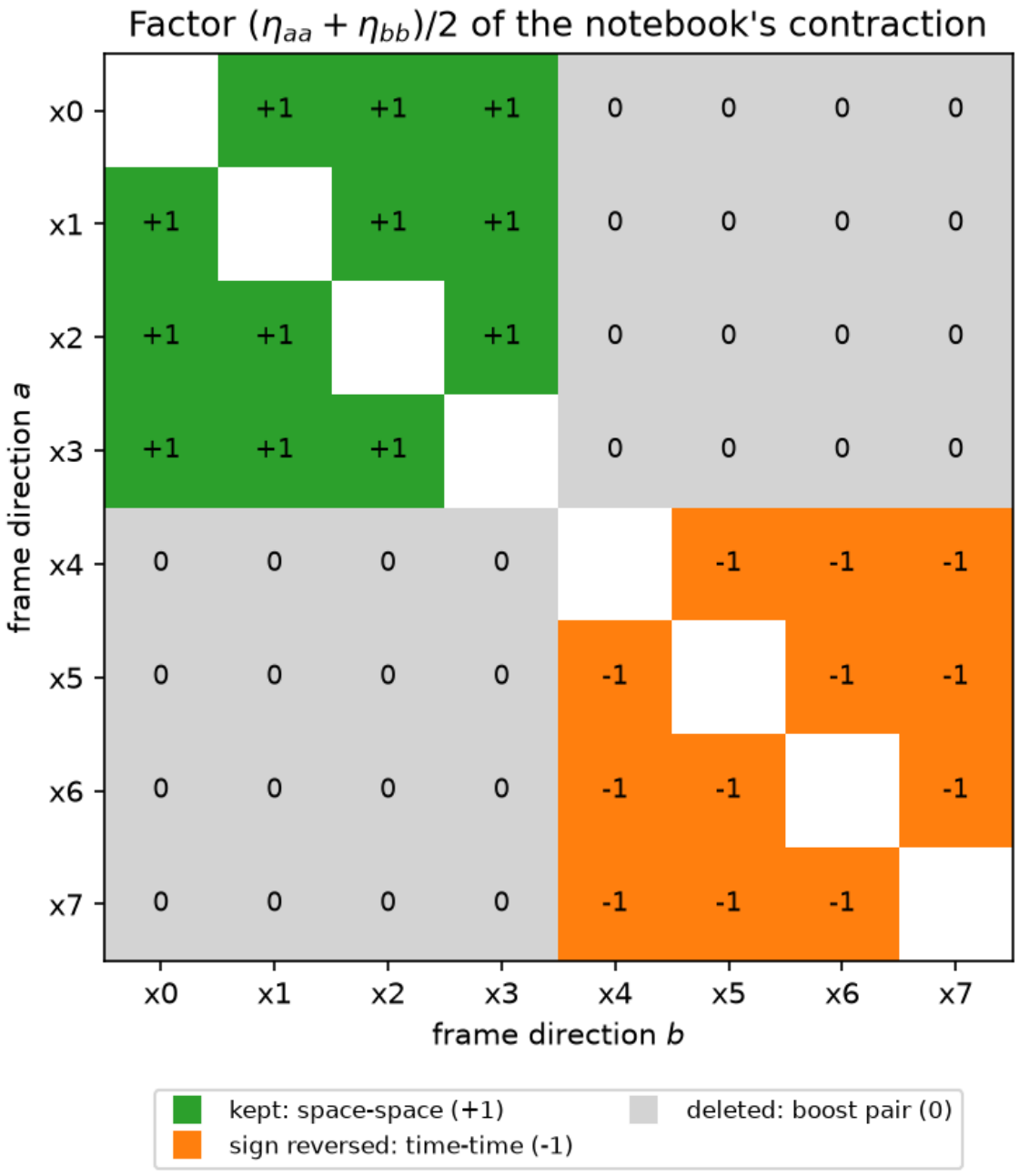

Figure 198: What the missing metric factor does to each pair of frame directions of the author's frame metric. Rows a and columns b are the frame directions $x0$ to $x7$; each square shows the factor $(\eta_{aa} + \eta_{bb})/2$ by which the notebook's contraction multiplies the part of the pair: green $+1$, kept (both space-like: $x0, x1, x2, x3$); orange -1, sign reversed (both time-like: $x4, x5, x6, x7$); gray 0, deleted (one space-like and one time-like direction, a boost pair); white diagonal, no pair. Of the 28 pairs 6 are kept, 6 reversed and 16 deleted. (Notebook 06c, figure 1.)

Text cell:

```
## 7. Two flat planes: where the mistake is invisible, and where it is total

**The polar plane** ($ds^2 = dr^2 + r^2d\varphi^2$, $\eta = \mathrm{diag}(1, 1)$,
vielbein $\mathrm{diag}(1, r)$, gammas $\sigma_1, \sigma_2$). Both frame directions
are space-like, so lowering changes nothing and the two contractions must agree;
the cell checks that both give $\Omega_\varphi = -\frac i2\sigma_3$, the result of
Notebook 06a.

**The Milne wedge** ($ds^2 = -d\tau^2 + \tau^2d\theta^2$, $\eta =
\mathrm{diag}(-1, +1)$, vielbein $\mathrm{diag}(1, \tau)$). Two REAL 2 x 2 gammas
with $(\gamma^0)^2 = -1$, $(\gamma^1)^2 = +1$, $\gamma^0\gamma^1 +
\gamma^1\gamma^0 = 0$ are $\gamma^0 = \begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix}$
and $\gamma^1 = \begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$ (the cell checks the
three relations). The only Christoffel symbols are $\Gamma^\tau{}_{\theta\theta} =
\tau$ and $\Gamma^\theta{}_{\tau\theta} = 1/\tau$, and the formula gives the mixed
components $\omega_\theta{}^0{}_1 = \omega_\theta{}^1{}_0 = 1$: SYMMETRIC, as for
every boost pair. Lowered, $\omega_{\theta 01} = -1 = -\omega_{\theta 10}$. The
plane is flat, but the frame is BOOSTED by the rapidity $\theta$ as one moves
along $\theta$ (left picture of the next figure); the correct spinor connection
$\Omega_\theta = -\frac12\gamma^0\gamma^1$ records this, and equals
$-(\partial_\theta U)U^{-1}$ for the spinor boost $U(\theta) = \cosh\frac\theta2 +
\sinh\frac\theta2\,\gamma^0\gamma^1$. A change of frame by a spin transformation
$R(\theta)$ changes the spinor connection into $R\,\Omega_\theta R^{-1} -
(\partial_\theta R)R^{-1}$; with $R = U$ applied to the connection 0 of the
Cartesian frame $(t, y)$ this gives $\Omega_\theta$, so $U$ carries the Cartesian
frame into the Milne frame, and with $R = U^{-1}$ it gives $U^{-1}\Omega_\theta U -
(\partial_\theta U^{-1})U = 0$: the inverse boost $U^{-1}$ removes the connection
(the cell checks both). The notebook's contraction gives
$\Omega^{nb} = 0$: it deletes the whole connection. The cell also checks the
divergence form $\gamma^\mu\Omega_\mu = \frac{1}{2\sqrt{|g|}}\sum_\mu
\partial_\mu(\sqrt{|g|}\gamma^\mu) = \gamma^0/(2\tau)$ (here $\sqrt{|g|} = \tau$),
which the notebook's value $0$ violates.
```

In [6]:

```
r, phi = sp.symbols("r phi", positive=True)  # the polar plane
sigma1 = sp.Matrix([[0, 1], [1, 0]])
sigma2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
sigma3 = sp.Matrix([[1, 0], [0, -1]])
polar_S = generators([sigma1, sigma2])
polar_e = sp.diag(1, r)  # unit radial and unit angular direction
polar_mixed = mixed_spin_connection(polar_e, christoffel(sp.diag(1, r**2), [r, phi]),
                                    [r, phi])
polar_Omega = contract(lower_first_index(polar_mixed, [1, 1]), polar_S)
polar_Omega_nb = contract(polar_mixed, polar_S)
check(polar_Omega == polar_Omega_nb and polar_Omega[1] == -sp.I / 2 * sigma3,
      "polar plane (eta = +1, +1): both contractions give Omega_phi = -(i/2) sigma3")

tau = sp.Symbol("tau", positive=True)  # the Milne wedge: tau > 0
theta = sp.Symbol("theta", real=True)  # the rapidity coordinate
```

```
g0 = sp.Matrix([[0, 1], [-1, 0]])  # real, squares to -1 (the time direction)
g1 = sp.Matrix([[0, 1], [1, 0]])  # real, squares to +1 (the space direction)
I2, Z2 = sp.eye(2), sp.zeros(2, 2)
check(g0 * g0 == -I2 and g1 * g1 == I2 and g0 * g1 + g1 * g0 == Z2,
      "Milne: real gammas with (g0)^2 = -1, (g1)^2 = +1, g0 g1 + g1 g0 = 0")
milne_S = generators([g0, g1])
milne_e = sp.diag(1, tau)  # unit time direction and unit space direction
milne_Gam = christoffel(sp.diag(-1, tau**2), [tau, theta])
milne_mixed = mixed_spin_connection(milne_e, milne_Gam, [tau, theta])
milne_lowered = lower_first_index(milne_mixed, [-1, 1])
say(f"Gamma^tau_thetatheta = {milne_Gam[0][1][1]}, "
    f"Gamma^theta_tautheta = {milne_Gam[1][0][1]}")
say(f"mixed: omega_theta^0_1 = {milne_mixed[1][0][1]}, "
    f"omega_theta^1_0 = {milne_mixed[1][1][0]}")
say(f"lowered: omega_theta01 = {milne_lowered[1][0][1]}, "
    f"omega_theta10 = {milne_lowered[1][1][0]}")
check(milne_mixed[1][0][1] == 1 and milne_mixed[1][1][0] == 1
      and milne_lowered[1][0][1] == -1 and milne_lowered[1][1][0] == 1,
      "Milne: mixed components symmetric (1, 1), lowered antisymmetric (-1, +1)")
milne_Omega = contract(milne_lowered, milne_S)
milne_Omega_nb = contract(milne_mixed, milne_S)
X2 = g0 * g1  # gamma^0 gamma^1, the boost generator of the plane (X2^2 = 1)
check(milne_Omega[0] == Z2 and milne_Omega[1] == -X2 / 2
      and milne_Omega_nb == [Z2, Z2],
      "Milne: Omega_theta = -(1/2) g0 g1, but the notebook's contraction gives 0")
U = sp.cosh(theta / 2) * I2 + sp.sinh(theta / 2) * X2  # the spinor boost
removed = U.inv() * milne_Omega[1] * U - U.inv().diff(theta) * U  # frame change U^-1
check(matrix_is_zero(-U.diff(theta) * U.inv() - milne_Omega[1])
      and matrix_is_zero(removed),
      "Milne: Omega_theta = -(dU/dtheta) U^-1 for the spinor boost U; U^-1 removes it")
milne_gup = curved_gammas(milne_e, [g0, g1])  # gamma^tau = g0, gamma^theta = g1/tau
slash_milne = (milne_gup[0] * milne_Omega[0] + milne_gup[1] * milne_Omega[1])
divergence_milne = ((tau * milne_gup[0]).diff(tau)
                    + (tau * milne_gup[1]).diff(theta)) / (2 * tau)
check(matrix_is_zero(slash_milne - g0 / (2 * tau))
      and matrix_is_zero(divergence_milne - slash_milne),
      "Milne: gamma^mu Omega_mu = g0/(2 tau) = the divergence form (notebook: 0)")
```

Out [6]:

```
PASS polar plane (eta = +1, +1): both contractions give Omega_phi = -(i/2) sigma3
PASS Milne: real gammas with (g0)^2 = -1, (g1)^2 = +1, g0 g1 + g1 g0 = 0
Gamma^tau_thetatheta = tau, Gamma^theta_tautheta = 1/tau
mixed: omega_theta^0_1 = 1, omega_theta^1_0 = 1
lowered: omega_theta01 = -1, omega_theta10 = 1
PASS Milne: mixed components symmetric (1, 1), lowered antisymmetric (-1, +1)
PASS Milne: Omega_theta = -(1/2) g0 g1, but the notebook's contraction gives 0
PASS Milne: Omega_theta = -(dU/dtheta) U^-1 for the spinor boost U; U^-1 removes it
PASS Milne: gamma^mu Omega_mu = g0/(2 tau) = the divergence form (notebook: 0)
```

Text cell:

The next cell draws the Milne wedge. On the left, in the flat plane with the space coordinate y horizontal and the time t vertical, the two curves $\tau = 1$ and $\tau = 2$ (hyperbolas $t^2 - y^2 = \tau^2$) and, at five values of θ, the frame: the unit time direction $e_{(0)} = (\sinh\theta, \cosh\theta)$ (blue) and the unit space direction $e_{(1)} = (\cosh\theta, \sinh\theta)$ (orange), in (y, t) components. Moving along θ the frame is boosted: both arrows tilt towards the light lines $t = \pm y$. On the right, the two diagonal entries of the spinor boost $U(\theta)$; since $\gamma^0\gamma^1 = \mathrm{diag}(1, -1)$ they are $e^{\theta/2}$ and $e^{-\theta/2}$, with $\cosh\frac\theta2$ and $\sinh\frac\theta2$ for comparison. Unlike the spinor rotation of Notebook 06a they are real and never come back: a boost has no full turn.

In [7]:

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.8))
curve = np.linspace(-1.3, 1.3, 200)  # values of theta for the hyperbolas
for radius in (1.0, 2.0):
    ax_left.plot(radius * np.sinh(curve), radius * np.cosh(curve), ":",
                 color="gray", linewidth=1.0)
    for rapidity in (-1.0, -0.5, 0.0, 0.5, 1.0):
        py, pt = radius * np.sinh(rapidity), radius * np.cosh(rapidity)  # the point
        ax_left.quiver(py, pt, np.sinh(rapidity), np.cosh(rapidity),
                       color="tab:blue", angles="xy", scale_units="xy", scale=2.5,
                       width=0.006)  # e_(0), the unit time direction
        ax_left.quiver(py, pt, np.cosh(rapidity), np.sinh(rapidity),
                       color="tab:orange", angles="xy", scale_units="xy",
                       scale=2.5, width=0.006)  # e_(1), the unit space direction
light = np.linspace(0.0, 3.6, 2)
ax_left.plot(light, light, "--", color="black", linewidth=0.8)  # t = y
ax_left.plot(-light, light, "--", color="black", linewidth=0.8,
             label="light lines $t = \\pm y$")  # t = -y
ax_left.plot([], [], color="tab:blue", label="unit time direction $e_{(0)}$")
ax_left.plot([], [], color="tab:orange", label="unit space direction $e_{(1)}$")
ax_left.set_xlim(-3.6, 3.6)
ax_left.set_ylim(0.0, 4.6)
ax_left.set_aspect("equal")
ax_left.set_xlabel("space coordinate $y = \\tau\\sinh\\theta$")
ax_left.set_ylabel("time $t = \\tau\\cosh\\theta$")
ax_left.set_title("The Milne frame is boosted along $\\theta$")
ax_left.legend(loc="upper center", fontsize=7)
rapidities = np.linspace(-4.0, 4.0, 401)
ax_right.plot(rapidities, np.exp(rapidities / 2),
              label="$U_{11} = e^{\\theta/2}$")
ax_right.plot(rapidities, np.exp(-rapidities / 2), "--",
              label="$U_{22} = e^{-\\theta/2}$")
ax_right.plot(rapidities, np.cosh(rapidities / 2), ":", color="black",
              label="$\\cosh(\\theta/2)$")
ax_right.plot(rapidities, np.sinh(rapidities / 2), "-.", color="gray",
              label="$\\sinh(\\theta/2)$")
ax_right.set_xlabel("rapidity $\\theta$")
ax_right.set_ylabel("entry of the spinor boost (pure number)")
ax_right.set_title("A spinor boost is real and grows")
ax_right.legend(fontsize=8)
```

```
save_figure(fig, "milne_frame_and_spin_boost",
            "The Milne wedge, a flat plane with one time in which the notebook's "
            "contraction deletes the whole spin connection. Left: the flat plane "
            "with the space coordinate $y$ horizontal and the time $t$ vertical "
            "(pure numbers); dotted, the curves $\\tau = 1$ and $\\tau = 2$; "
            "dashed, the light lines $t = \\pm y$; arrows, the frame at the "
            "rapidities $\\theta = -1, -0.5, 0, 0.5, 1$: unit time direction "
            "(blue) and unit space direction (orange). Along $\\theta$ the frame is "
            "boosted, and the correct spin connection $\\omega_{\\theta 01} = -1$ "
            "records this rate. Right: the diagonal entries $e^{\\theta/2}$ and "
            "$e^{-\\theta/2}$ of the spinor boost $U(\\theta)$ that carries the "
            "Cartesian frame into the Milne frame (its inverse $U^{-1}$ removes the "
            "connection), with $\\cosh(\\theta/2)$ and $\\sinh(\\theta/2)$, versus "
            "$\\theta$: real and unbounded, unlike a spinor rotation.")
```

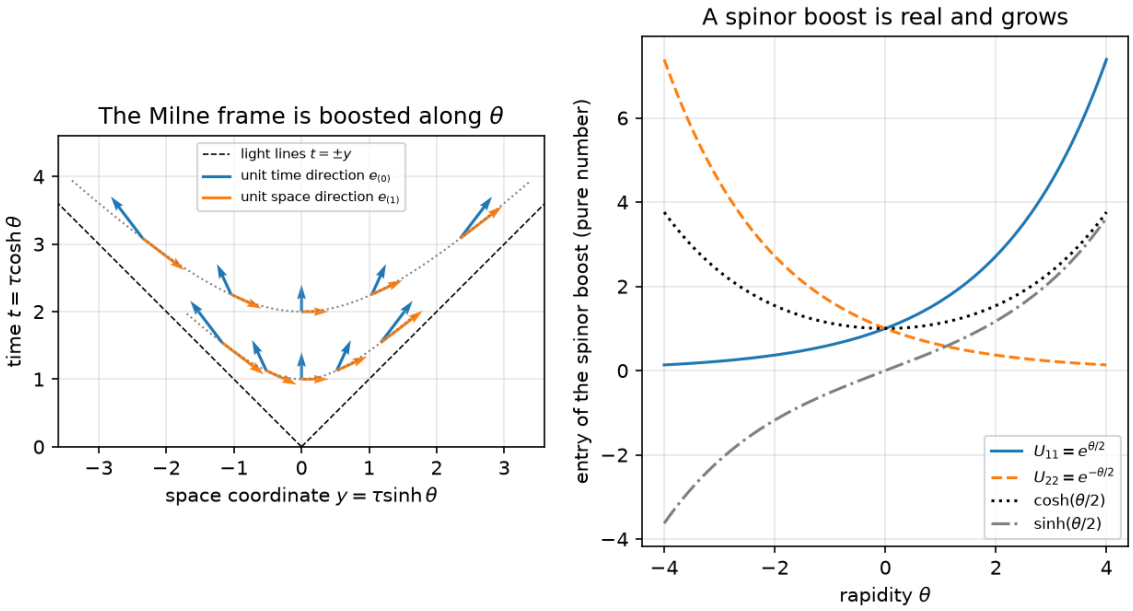

Figure 199: The Milne wedge, a flat plane with one time in which the notebook's contraction deletes the whole spin connection. Left: the flat plane with the space coordinate y horizontal and the time t vertical (pure numbers); dotted, the curves $\tau = 1$ and $\tau = 2$; dashed, the light lines $t = \pm y$; arrows, the frame at the rapidities $\theta = -1, -0.5, 0, 0.5, 1$: unit time direction (blue) and unit space direction (orange). Along θ the frame is boosted, and the correct spin connection $\omega_{\theta 01} = -1$ records this rate. Right: the diagonal entries $e^{\theta/2}$ and $e^{-\theta/2}$ of the spinor boost $U(\theta)$ that carries the Cartesian frame into the Milne frame (its inverse U^{-1} removes the connection), with $\cosh(\theta/2)$ and $\sinh(\theta/2)$, versus θ: real and unbounded, unlike a spinor rotation. (Notebook 06c, figure 2.)

Out [7]:

```
Figure 06c.2 saved as Revision/textbook/figures/06c_2_milne_frame_and_spin_boost.png
```

Text cell:

8. The author's metric: the correct and the notebook's spinor connection

The next cell builds the author's metric with its diagonal vielbein $f =
(\cot z,\ e^{a_4}s\ (\times 3),\ 1,\ e^{-a_4}b_4^{1/2}s\ (\times 3))$,
$s = \sin^{1/6}z$, for arbitrary functions $a_4(x_4)$ and $b_4(x_4) > 0$, computes
the Christoffel symbols, the MIXED spin connection and its lowered form, and
compares the 12 nonzero lowered components with the records (‘omega_nonzero‘ of
the record with b_4, and at $b_4 = 1$ of the record of $b_4 = 1$; the records
number the directions from 1 in the order $x0, \dots, x7$). Then it builds the
correct
Ω_μ, the notebook's Ω^{nb}_μ and the three parts
Ω^{ss}_μ, Ω^{tt}_μ, Ω^{st}_μ, and checks the identity
of Section 4, $\Omega^{nb}_\mu = \Omega^{ss}_\mu - \Omega^{tt}_\mu$, for all eight
μ. It prints, for each of the 12 lowered components, the kind of its pair and
what the notebook's contraction does to it.

In [8]:

```
from sympy.parsing.mathematica import parse_mathematica  # reads Wolfram notation

x = sp.symbols("x0:8", real=True)  # x[0] = x0, ..., x[4] = x4, ..., x[7] = x7
x0, x4 = x[0], x[4]  # the hidden direction and the time
H = sp.Symbol("H", positive=True)  # the author's constant H > 0
a4 = sp.Function("a4")(x4)  # any function of the time
b4 = sp.Function("b4", positive=True)(x4)  # any positive function of the time
a4p, b4p = sp.Derivative(a4, x4), sp.Derivative(b4, x4)  # a4' and b4'
rate_t = a4p - b4p / (2 * b4)  # the rate of the extra times, a4' - b4'/(2 b4)
z = 6 * H * x0  # the hidden angle
s = sp.sin(z) ** sp.Rational(1, 6)
f = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
     + [sp.exp(-a4) * sp.sqrt(b4) * s] * 3)  # f_0, ..., f_7
g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # the author's metric
e = sp.diag(*f)  # the diagonal vielbein


def at_b4_1(expr):
    """expr at b4 = 1; doit() evaluates the derivatives of the constant 1 to 0."""
    return sp.sympify(expr).subs({b4: 1}).doit()


Gam = christoffel(g, x)
mixed = mixed_spin_connection(e, Gam, x)  # omega_mu^a_b
omega = lower_first_index(mixed, ETA)  # omega_mu ab
names = {sp.Symbol("H"): H, sp.Symbol("x0"): x0, sp.Symbol("x4"): x4,
         sp.Symbol("a4"): a4, sp.Symbol("b4"): b4, sp.Symbol("a4p"): a4p,
         sp.Symbol("b4p"): b4p}


def record_omega(formulas):
    """The record's list {mu, a, b, value} (a < b, numbered from 1, Wolfram
    notation) as a dictionary with Python's numbers 0 to 7."""
    text = formulas["omega_nonzero"]
    for long_form, short_form in (("Derivative[1][a4][x4]", "a4p"),
```

```
                                ("Derivative[1][b4][x4]", "b4p"),
                                ("a4[x4]", "a4"), ("b4[x4]", "b4")):
        text = text.replace(long_form, short_form)
    return {(int(m) - 1, int(a) - 1, int(b) - 1): v.subs(names, simultaneous=True)
            for m, a, b, v in parse_mathematica(text)}


omega_record = record_omega(FORMULAS_B4)
omega_record_1 = record_omega(FORMULAS)
nonzero = {(mu, a, b): omega[mu][a][b] for mu in range(8) for a, b in pairs
           if omega[mu][a][b] != 0}
check(set(nonzero) == set(omega_record) == set(omega_record_1)
      and all(is_zero(nonzero[k] - omega_record[k])
              and is_zero(at_b4_1(nonzero[k]) - omega_record_1[k]) for k in nonzero),
      "the 12 nonzero lowered components omega_mu ab equal the records (with b4, and "
      "at b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula omega_nonzero")
ACTION = {1: "kept", -1: "sign reversed", 0: "deleted"}
for (mu, a, b) in sorted(nonzero):
    k = int(factor[a][b])
    say(f"omega_{NAMES[mu]},{NAMES[a]}{NAMES[b]}: {KIND[k]} pair, {ACTION[k]}")
Omega = contract(omega, S)  # the correct spinor connection
Omega_nb = contract(mixed, S)  # the notebook's contraction


def part(kind):
    """The part of Omega that comes from the pairs of one kind."""
    return contract([[[omega[mu][a][b] if factor[a][b] == kind else 0
                       for b in range(8)] for a in range(8)] for mu in range(8)], S)


Omega_ss, Omega_tt, Omega_st = part(1), part(-1), part(0)
check(all(matrix_is_zero(Omega[mu] - Omega_ss[mu] - Omega_tt[mu] - Omega_st[mu])
          and matrix_is_zero(Omega_nb[mu] - Omega_ss[mu] + Omega_tt[mu])
          for mu in range(8)),
      "Omega^nb_mu = Omega^ss_mu - Omega^tt_mu for all eight mu (boosts deleted)")
```

Out [8]:

```
PASS the 12 nonzero lowered components omega_mu ab equal the records (with b4, and at b4
    = 1)
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula omega_nonzero
omega_x1,x0x1: space-space pair, kept
omega_x1,x1x4: boost pair, deleted
omega_x2,x0x2: space-space pair, kept
omega_x2,x2x4: boost pair, deleted
omega_x3,x0x3: space-space pair, kept
omega_x3,x3x4: boost pair, deleted
omega_x5,x0x5: boost pair, deleted
omega_x5,x4x5: time-time pair, sign reversed
omega_x6,x0x6: boost pair, deleted
omega_x6,x4x6: time-time pair, sign reversed
```

```
omega_x7,x0x7: boost pair, deleted
omega_x7,x4x7: time-time pair, sign reversed
PASS Omega^nb_mu = Omega^ss_mu - Omega^tt_mu for all eight mu (boosts deleted)
```

Text cell:

```
The previous cell showed that for $\mu = x1$ the boost part (the pair $x1, x4$, with
the rate $a_4'$) is deleted and the rotation part (the pair $x0, x1$, with $H$) is
kept, and that for $\mu = x5$ the boost part (the pair $x0, x5$, with $H$) is
deleted and the time-time part (the pair $x4, x5$, with the rate $a_4' -
b_4'/(2b_4)$) has its sign reversed. The next cell checks the resulting closed
forms

$$\Omega^{nb}_{x1} = e^{a_4}s\,H\,S^{x1\,x0}, \qquad
\Omega^{nb}_{x5} = +e^{-a_4}b_4^{1/2}s\,\big(a_4' - \tfrac{b_4'}{2b_4}\big)\,
S^{x4\,x5},$$

compared with the correct $\Omega_{x1} = e^{a_4}s\,(a_4'S^{x1\,x4} +
HS^{x1\,x0})$ and $\Omega_{x5} = -e^{-a_4}b_4^{1/2}s\,\big((a_4' -
\frac{b_4'}{2b_4})S^{x4\,x5} + HS^{x5\,x0}\big)$ (Notebook 06a), and the same
forms for $x2, x3$ and $x6, x7$.
```

In [9]:

```
closed_ok = True
for i in (1, 2, 3):  # the 3-space directions x1, x2, x3
    closed_ok = closed_ok and matrix_is_zero(
        Omega_nb[i] - sp.exp(a4) * s * H * S[i][0])  # pair (x0, xi) kept
    closed_ok = closed_ok and matrix_is_zero(
        Omega[i] - sp.exp(a4) * s * (a4p * S[i][4] + H * S[i][0]))
f_t = sp.exp(-a4) * sp.sqrt(b4) * s  # the factor of the extra times
for t in (5, 6, 7):  # the extra times x5, x6, x7
    closed_ok = closed_ok and matrix_is_zero(
        Omega_nb[t] - f_t * rate_t * S[4][t])  # pair (x4, xt) reversed
    closed_ok = closed_ok and matrix_is_zero(
        Omega[t] + f_t * (rate_t * S[4][t] + H * S[t][0]))
check(closed_ok, "the closed forms of Omega_mu and Omega^nb_mu for x1 ... x3, "
      "x5 ... x7")
```

Out [9]:

```
PASS the closed forms of Omega_mu and Omega^nb_mu for x1 ... x3, x5 ... x7
```

Text cell:

```
## 9. What the notebook's contraction would put into the field equation

The field equation contains the 16 x 16 matrix $\gamma^\mu\Omega_\mu$ (summed over
$\mu$). The next cell computes each term $\gamma^\mu\Omega_\mu$ (no sum) for both
```

```
contractions and writes it as $\alpha_\mu\gamma^{(x4)} + \beta_\mu\gamma^{(x0)}$
(the coefficients are found with traces, as in Notebook 06a:
$\alpha_\mu = -\mathrm{tr}(M\gamma^{(x4)})/16$ and $\beta_\mu =
\mathrm{tr}(M\gamma^{(x0)})/16$, and the cell checks that nothing else is left).
By hand: $\gamma^{x1}\Omega^{nb}_{x1} = \frac{1}{f_1}\gamma^{(x1)}\cdot f_1 H\cdot
\frac12\gamma^{(x1)}\gamma^{(x0)} = \frac H2\gamma^{(x0)}$ (because
$(\gamma^{(x1)})^2 = 1$), and $\gamma^{x5}\Omega^{nb}_{x5} =
\frac{1}{f_5}\gamma^{(x5)}\cdot f_5 r_t\cdot\frac12\gamma^{(x4)}\gamma^{(x5)} =
-\frac{r_t}{2}\gamma^{(x4)}(\gamma^{(x5)})^2 = \frac{r_t}{2}\gamma^{(x4)}$ with
$r_t = a_4' - b_4'/(2b_4)$ (because $\gamma^{(x5)}$ anticommutes with
$\gamma^{(x4)}$ and $(\gamma^{(x5)})^2 = -1$). Summed:

$$\gamma^\mu\Omega^{nb}_\mu = \tfrac{3H}{2}\gamma^{(x0)} + \tfrac32\big(a_4' -
\tfrac{b_4'}{2b_4}\big)\gamma^{(x4)} \quad\text{instead of}\quad
\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \tfrac34\tfrac{b_4'}{b_4}\gamma^{(x4)} .$$

Half of the hidden term is lost, and the $a_4'$ terms along the time no longer
cancel: with the wrong contraction the deflation would appear in this term. The
cell compares the correct total with the records (‘gammaOmega_total‘ with $b_4$,
and at $b_4 = 1$).
```

In [10]:

```
gup = curved_gammas(e, gamma)  # gamma^mu = gamma^a / f_a for mu = a


def split(M):
    """(alpha, beta, rest is zero) for M = alpha gamma^(x4) + beta gamma^(x0)."""
    alpha = sp.simplify(-(M * gamma[4]).trace() / 16)  # coefficient of gamma^(x4)
    beta = sp.simplify((M * gamma[0]).trace() / 16)  # coefficient of gamma^(x0)
    return alpha, beta, matrix_is_zero(M - alpha * gamma[4] - beta * gamma[0])


correct_terms = [split(gup[mu] * Omega[mu]) for mu in range(8)]
notebook_terms = [split(gup[mu] * Omega_nb[mu]) for mu in range(8)]
for mu in range(8):
    say(f"{NAMES[mu]}: correct ({correct_terms[mu][0]}, {correct_terms[mu][1]}), "
        f"notebook ({notebook_terms[mu][0]}, {notebook_terms[mu][1]})")
want_nb = [(0, 0)] + [(0, H / 2)] * 3 + [(0, 0)] + [(rate_t / 2, 0)] * 3
check(all(term[2] for term in correct_terms + notebook_terms)
      and all(is_zero(notebook_terms[mu][0] - want_nb[mu][0])
              and is_zero(notebook_terms[mu][1] - want_nb[mu][1]) for mu in range(8)),
      "notebook's terms: (H/2) gamma^(x0) for x1, x2, x3; ((a4' - b4'/(2 b4))/2) "
      "gamma^(x4) for x5, x6, x7")
slash = sum((gup[mu] * Omega[mu] for mu in range(8)), Z16)
slash_nb = sum((gup[mu] * Omega_nb[mu] for mu in range(8)), Z16)


def record_total(formulas):
    """The record's gammaOmega_total as a 16 x 16 matrix."""
    text = formulas["gammaOmega_total"].replace(chr(34), "")  # gamma[x0], gamma[x4]
    for long_form, short_form in (("Derivative[1][b4][x4]", "b4p"),
                                  ("b4[x4]", "b4"), ("gamma[x0]", "G0"),
```

```
                                ("gamma[x4]", "G4")):
        text = text.replace(long_form, short_form)
    total = parse_mathematica(text).subs(names, simultaneous=True)
    G0, G4 = sp.Symbol("G0"), sp.Symbol("G4")
    return total.coeff(G0) * gamma[0] + total.coeff(G4) * gamma[4]


check(matrix_is_zero(slash - record_total(FORMULAS_B4))
      and matrix_is_zero(at_b4_1(slash) - record_total(FORMULAS))
      and record_passed(REPORT_PY, "gamma_mu_Omega_mu_equals_3H_gamma_x0"),
      "correct: gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); "
      "3 H gamma^(x0) at b4 = 1",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula gammaOmega_total")
report("notebook: coefficient of gamma^(x0)", sum(t[1] for t in notebook_terms))
report("notebook: coefficient of gamma^(x4)",
       sp.simplify(sum(t[0] for t in notebook_terms)))
check(matrix_is_zero(slash_nb - 3 * H / 2 * gamma[0] - 3 * rate_t / 2 * gamma[4]),
      "notebook: gamma^mu Omega^nb_mu = (3H/2) gamma^(x0) + (3/2)(a4' - b4'/(2 b4)) "
      "gamma^(x4)")
```

Out [10]:

```
x0: correct (0, 0), notebook (0, 0)
x1: correct (Derivative(a4(x4), x4)/2, H/2), notebook (0, H/2)
x2: correct (Derivative(a4(x4), x4)/2, H/2), notebook (0, H/2)
x3: correct (Derivative(a4(x4), x4)/2, H/2), notebook (0, H/2)
x4: correct (0, 0), notebook (0, 0)
x5: correct (-Derivative(a4(x4), x4)/2 + Derivative(b4(x4), x4)/(4*b4(x4)), H/2),
    notebook (Derivative(a4(x4), x4)/2 - Derivative(b4(x4), x4)/(4*b4(x4)), 0)
x6: correct (-Derivative(a4(x4), x4)/2 + Derivative(b4(x4), x4)/(4*b4(x4)), H/2),
    notebook (Derivative(a4(x4), x4)/2 - Derivative(b4(x4), x4)/(4*b4(x4)), 0)
x7: correct (-Derivative(a4(x4), x4)/2 + Derivative(b4(x4), x4)/(4*b4(x4)), H/2),
    notebook (Derivative(a4(x4), x4)/2 - Derivative(b4(x4), x4)/(4*b4(x4)), 0)
PASS notebook's terms: (H/2) gamma^(x0) for x1, x2, x3; ((a4' - b4'/(2 b4))/2) gamma^(x4)
    for x5, x6, x7
PASS correct: gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); 3 H
    gamma^(x0) at b4 = 1
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula gammaOmega_total
RESULT notebook: coefficient of gamma^(x0) = 3*H/2
RESULT notebook: coefficient of gamma^(x4) = 3*Derivative(a4(x4), x4)/2 -
    3*Derivative(b4(x4), x4)/(4*b4(x4))
PASS notebook: gamma^mu Omega^nb_mu = (3H/2) gamma^(x0) + (3/2)(a4' - b4'/(2 b4))
    gamma^(x4)
```

Text cell:

```
The next cell draws the per-direction table of the previous cell as bar charts, on
the background $b_4 = 1$: for each direction $\mu$ the coefficient of
$\gamma^{(x4)}$ in units of $a_4'$ (left) and of $\gamma^{(x0)}$ in units of $H$
(right), for the correct contraction
(dark bars) and the notebook's (light bars). On the left the correct bars cancel
```

```
in pairs ($+\frac12$ three times, $-\frac12$ three times), while the notebook's
three bars $+\frac12$ do not cancel; on the right the notebook keeps only the
three 3-space bars.
```

In [11]:

```
positions = np.arange(8)
width = 0.38  # the width of one bar; two bars side by side per direction
alpha_c = [float(sp.simplify(at_b4_1(t[0]) / a4p)) for t in correct_terms]  # b4 = 1
alpha_n = [float(sp.simplify(at_b4_1(t[0]) / a4p)) for t in notebook_terms]
beta_c = [float(sp.simplify(t[1] / H)) for t in correct_terms]  # units of H
beta_n = [float(sp.simplify(t[1] / H)) for t in notebook_terms]
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
ax_left.bar(positions - width / 2, alpha_c, width, color="tab:blue",
            label=f"correct (total {sum(alpha_c):g})")
ax_left.bar(positions + width / 2, alpha_n, width, color="lightskyblue",
            label=f"notebook (total {sum(alpha_n):g})")
ax_right.bar(positions - width / 2, beta_c, width, color="tab:red",
             label=f"correct (total {sum(beta_c):g})")
ax_right.bar(positions + width / 2, beta_n, width, color="lightsalmon",
             label=f"notebook (total {sum(beta_n):g})")
ax_left.set_title("coefficient of $\\gamma^{(x4)}$, units of $a_4'$")
ax_right.set_title("coefficient of $\\gamma^{(x0)}$, units of $H$")
for ax in (ax_left, ax_right):
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, NAMES)
    ax.set_xlabel("direction $\\mu$ of the term $\\gamma^\\mu\\Omega_\\mu$")
    ax.set_ylim(-0.8, 0.8)
    ax.legend(fontsize=8, loc="lower left")
ax_left.set_ylabel("coefficient")
save_figure(fig, "per_direction_comparison",
            "The eight terms $\\gamma^\\mu\\Omega_\\mu$ (no sum) of the author's "
            "metric at $b_4 = 1$, each of the form $\\alpha_\\mu\\gamma^{(x4)} + "
            "\\beta_\\mu\\gamma^{(x0)}$, for the correct contraction (dark bars) "
            "and for the notebook's contraction of the mixed components (light "
            "bars), versus the direction $\\mu$. Left: $\\alpha_\\mu$ in units of "
            "$a_4'$; the correct values $+1/2$ for $x1, x2, x3$ and $-1/2$ for $x5, "
            "x6, x7$ cancel, the notebook's $+1/2$ for $x5, x6, x7$ add to $3/2$. "
            "Right: $\\beta_\\mu$ in units of $H$; the correct six values $1/2$ add "
            "to $3$, the notebook keeps only three of them, total $3/2$.")
```

Out [11]:

```
Figure 06c.3 saved as Revision/textbook/figures/06c_3_per_direction_comparison.png
```

Text cell:

```
The next cell shows the two totals as pictures of 16 x 16 matrices at the sample
point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$ (red positive, blue
negative, white zero, one colour scale for both). Left: the correct
$\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)}$, the value 3 on two diagonal lines (the
```

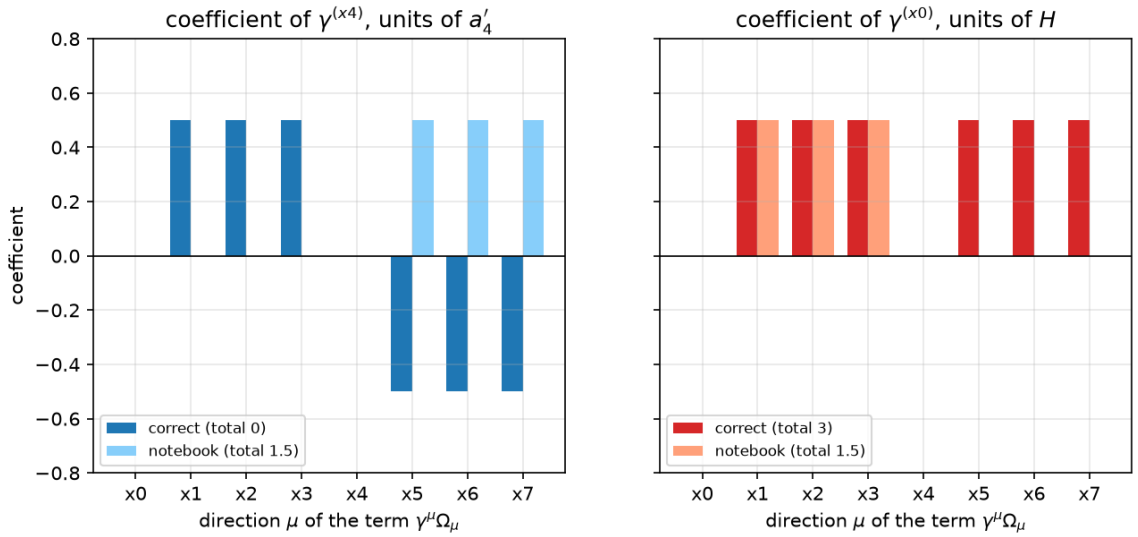

Figure 200: The eight terms $\gamma^\mu\Omega_\mu$ (no sum) of the author's metric at $b_4 = 1$, each of the form $\alpha_\mu\gamma^{(x4)} + \beta_\mu\gamma^{(x0)}$, for the correct contraction (dark bars) and for the notebook's contraction of the mixed components (light bars), versus the direction μ. Left: α_μ in units of a'_4; the correct values $+1/2$ for $x1, x2, x3$ and $-1/2$ for $x5, x6, x7$ cancel, the notebook's $+1/2$ for $x5, x6, x7$ add to $3/2$. Right: β_μ in units of H; the correct six values $1/2$ add to 3, the notebook keeps only three of them, total $3/2$. (Notebook 06c, figure 3.)

```
identity blocks of $\gamma^{(x0)}$). Right: the notebook's
$\frac{3H}{2}\gamma^{(x0)} + \frac{3a_4'}{2}\gamma^{(x4)}$: the same lines with the
value 1.5, plus the pattern of $\gamma^{(x4)}$ with entries $\pm0.75$.
```

In [12]:

```
SAMPLE = {H: 1, x0: sp.pi / 24}  # H = 1 and z = 6 H x0 = pi/4


def at_sample(expr):
    """The decimal value of expr at H = 1, z = pi/4, a4 = 1/2, a4' = 1/2, b4 = 1."""
    expr = at_b4_1(expr).subs(a4p, sp.Rational(1, 2))  # a4' first, then a4
    expr = expr.subs(a4, sp.Rational(1, 2))
    return float(sp.N(expr.subs(SAMPLE)))


def sample_table(matrix):
    """The matrix as an array of decimal numbers at the sample point."""
    return np.array([[at_sample(v) for v in row] for row in matrix.tolist()])


fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, (title, matrix) in zip(axes, (
        ("correct: $\\gamma^\\mu\\Omega_\\mu$", slash),
        ("notebook: $\\gamma^\\mu\\Omega^{nb}_\\mu$", slash_nb))):
    image = ax.imshow(sample_table(matrix), cmap="RdBu_r", vmin=-3.2, vmax=3.2)
```

```
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.85, label="entry value")
save_figure(fig, "contraction_heat_maps",
            "The total $\\gamma^\\mu\\Omega_\\mu$ as a 16 x 16 heat map at the sample "
            "point $H = 1$, $z = \\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$ (rows "
            "and columns are the spinor components; red positive, blue negative, "
            "white zero; "
            "colour scale from $-3.2$ to $3.2$). Left: the correct value "
            "$3H\\gamma^{(x0)}$, the number 3 on two diagonal lines. Right: what the "
            "notebook's contraction of the mixed components would give, "
            "$\\frac{3H}{2}\\gamma^{(x0)} + \\frac{3a_4'}{2}\\gamma^{(x4)}$: the "
            "same lines with half the value, and the pattern of $\\gamma^{(x4)}$ "
            "with entries $\\pm 0.75$, which depends on the rate $a_4'$ of the "
            "deflation.")
```

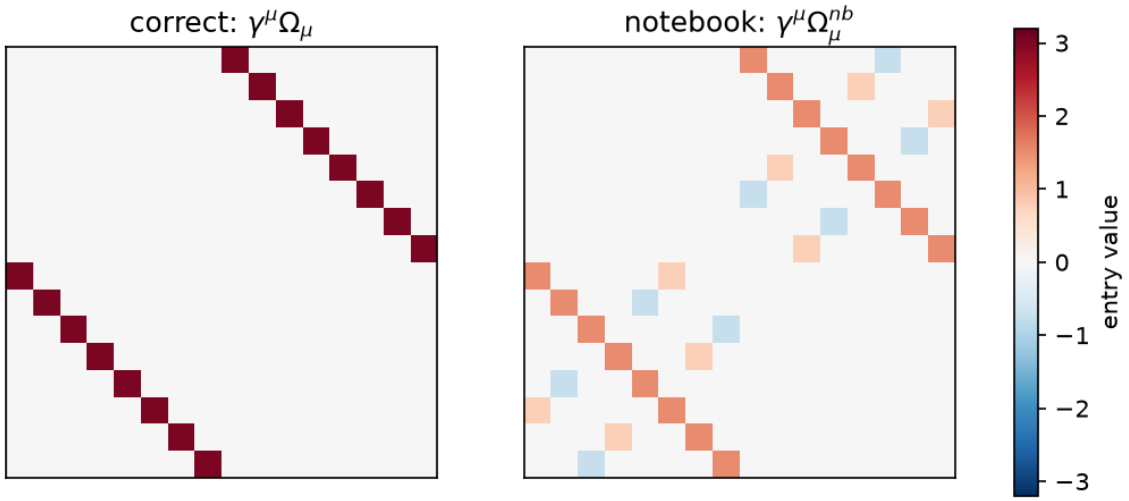

Figure 201: The total $\gamma^\mu\Omega_\mu$ as a 16 x 16 heat map at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$ (rows and columns are the spinor components; red positive, blue negative, white zero; colour scale from -3.2 to 3.2). Left: the correct value $3H\gamma^{(x0)}$, the number 3 on two diagonal lines. Right: what the notebook's contraction of the mixed components would give, $\frac{3H}{2}\gamma^{(x0)} + \frac{3a_4'}{2}\gamma^{(x4)}$: the same lines with half the value, and the pattern of $\gamma^{(x4)}$ with entries ± 0.75, which depends on the rate a_4' of the deflation. (Notebook 06c, figure 4.)

Out [12]:

```
Figure 06c.4 saved as Revision/textbook/figures/06c_4_contraction_heat_maps.png
```

Text cell:

```
## 10. The gammas are no longer covariantly constant
```

```
With the correct $\Omega_\mu$ the curved gammas are covariantly constant,
$D_\mu\gamma^\nu = 0$ for all 64 pairs $(\mu, \nu)$ (Notebook 06a; the Revision
record). Replace $\Omega_\mu$ by $\Omega^{nb}_\mu$:

$$D^{nb}_\mu\gamma^\nu = \partial_\mu\gamma^\nu +
\sum_\lambda\Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega^{nb}_\mu, \gamma^\nu]
= D_\mu\gamma^\nu + [\Omega^{nb}_\mu - \Omega_\mu, \gamma^\nu] =
[\Omega^{nb}_\mu - \Omega_\mu, \gamma^\nu] .$$

The first step adds and subtracts $[\Omega_\mu, \gamma^\nu]$; the second uses
$D_\mu\gamma^\nu = 0$. The next cell computes both versions for all 64 pairs,
checks the correct one against the record, checks this identity, counts the pairs
$(\mu, \nu)$ in which $D^{nb}_\mu\gamma^\nu$ is not zero and the number of its
nonzero entries (exact counts of this notebook), and checks that the defining
property $[\Omega_\mu, \gamma^a] = -\sum_b\omega_\mu{}^a{}_b\gamma^b$ fails for
$\Omega^{nb}$.
```

In [13]:

```
violation = {}  # (mu, nu) -> D^nb_mu gamma^nu, for the pairs where it is not zero
correct_ok, identity_ok, entries = True, True, 0
for mu in range(8):
    for nu in range(8):
        common = gup[nu].diff(x[mu]) + sum((Gam[nu][mu][lam] * gup[lam]
                                            for lam in range(8)), Z16)
        D_correct = common + Omega[mu] * gup[nu] - gup[nu] * Omega[mu]
        D_nb = common + Omega_nb[mu] * gup[nu] - gup[nu] * Omega_nb[mu]
        difference = Omega_nb[mu] - Omega[mu]
        correct_ok = correct_ok and matrix_is_zero(D_correct)
        identity_ok = identity_ok and matrix_is_zero(
            D_nb - (difference * gup[nu] - gup[nu] * difference))
        nonzero_entries = sum(1 for v in D_nb if not is_zero(v))
        if nonzero_entries:
            violation[(mu, nu)] = D_nb
            entries += nonzero_entries
check(correct_ok and record_passed(REPORT_WL_B4, "gamma_covariantly_constant"),
      "correct: D_mu gamma^nu = 0 for all 64 pairs",
      record=f"{REPORT_WL_B4}, check gamma_covariantly_constant")
check(identity_ok, "D^nb_mu gamma^nu = [Omega^nb_mu - Omega_mu, gamma^nu], all 64")
report("pairs (mu, nu) with D^nb_mu gamma^nu not zero", len(violation))
report("nonzero entries of these 16 x 16 matrices", entries)
say("violated pairs: " + ", ".join(f"({NAMES[m]},{NAMES[n]})"
                                   for m, n in sorted(violation)))
check(len(violation) == 15 and entries == 288,
      "notebook: D^nb_mu gamma^nu is not zero in 15 of 64 pairs (288 entries)")
failures = [(mu, a) for mu in range(8) for a in range(8) if not matrix_is_zero(
    Omega_nb[mu] * gamma[a] - gamma[a] * Omega_nb[mu]
    + sum((mixed[mu][a][b] * gamma[b] for b in range(8)), Z16))]
report("pairs (mu, a) where the defining property fails for Omega^nb", len(failures))
check(len(failures) > 0 and all(matrix_is_zero(
    Omega[mu] * gamma[a] - gamma[a] * Omega[mu]
    + sum((mixed[mu][a][b] * gamma[b] for b in range(8)), Z16))
    for mu in range(8) for a in range(8)),
```

```
      "the defining property holds for Omega and fails for Omega^nb",
      record=f"{REPORT_WL}, check S_rotates_gamma_with_omega")
```

Out [13]:

```
PASS correct: D_mu gamma^nu = 0 for all 64 pairs
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    gamma_covariantly_constant
PASS D^nb_mu gamma^nu = [Omega^nb_mu - Omega_mu, gamma^nu], all 64
RESULT pairs (mu, nu) with D^nb_mu gamma^nu not zero = 15
RESULT nonzero entries of these 16 x 16 matrices = 288
violated pairs: (x1,x1), (x1,x4), (x2,x2), (x2,x4), (x3,x3), (x3,x4), (x5,x0), (x5,x4),
    (x5,x5), (x6,x0), (x6,x4), (x6,x6), (x7,x0), (x7,x4), (x7,x7)
PASS notebook: D^nb_mu gamma^nu is not zero in 15 of 64 pairs (288 entries)
RESULT pairs (mu, a) where the defining property fails for Omega^nb = 15
PASS the defining property holds for Omega and fails for Omega^nb
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    S_rotates_gamma_with_omega
```

Text cell:

```
The next cell draws where the covariant constancy fails: an 8 x 8 grid with one
square for each pair $(\mu, \nu)$, coloured by the largest absolute value of the
256 entries of $D^{nb}_\mu\gamma^\nu$ at the sample point (white: zero). With the
correct connection every square would be white.
```

In [14]:

```
size = np.zeros((8, 8))
for (mu, nu), matrix in violation.items():
    size[mu, nu] = np.abs(sample_table(matrix)).max()  # the largest entry
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(size, cmap="Oranges", vmin=0.0)
for (mu, nu) in violation:
    dark = size[mu, nu] > 0.6 * size.max()  # white digits on dark squares
    ax.text(nu, mu, f"{size[mu, nu]:.2f}", ha="center", va="center", fontsize=7,
            color="white" if dark else "black")
ax.set_xticks(range(8), NAMES)
ax.set_yticks(range(8), NAMES)
ax.set_xlabel("upper index $\\nu$ of $\\gamma^\\nu$")
ax.set_ylabel("derivative index $\\mu$")
ax.set_title("Largest entry of $D^{nb}_\\mu\\gamma^\\nu$ at the sample point")
ax.grid(False)
fig.colorbar(image, ax=ax, shrink=0.8, label="largest absolute entry")
save_figure(fig, "constancy_violation",
            "Where the notebook's contraction breaks the covariant constancy of the "
            "gammas: for each pair of coordinates, the derivative index $\\mu$ "
            "(rows) and the upper index $\\nu$ (columns), the largest absolute "
            "entry of the 16 x 16 matrix $D^{nb}_\\mu\\gamma^\\nu$ at the sample "
            "point $H = 1$, $z = \\pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$ "
            "(pure numbers; white is zero). Exactly 15 of the 64 pairs are nonzero, "
```

```
"all "
"in the rows of the six warped directions; with the correct spinor "
"connection all 64 matrices vanish exactly.")
```

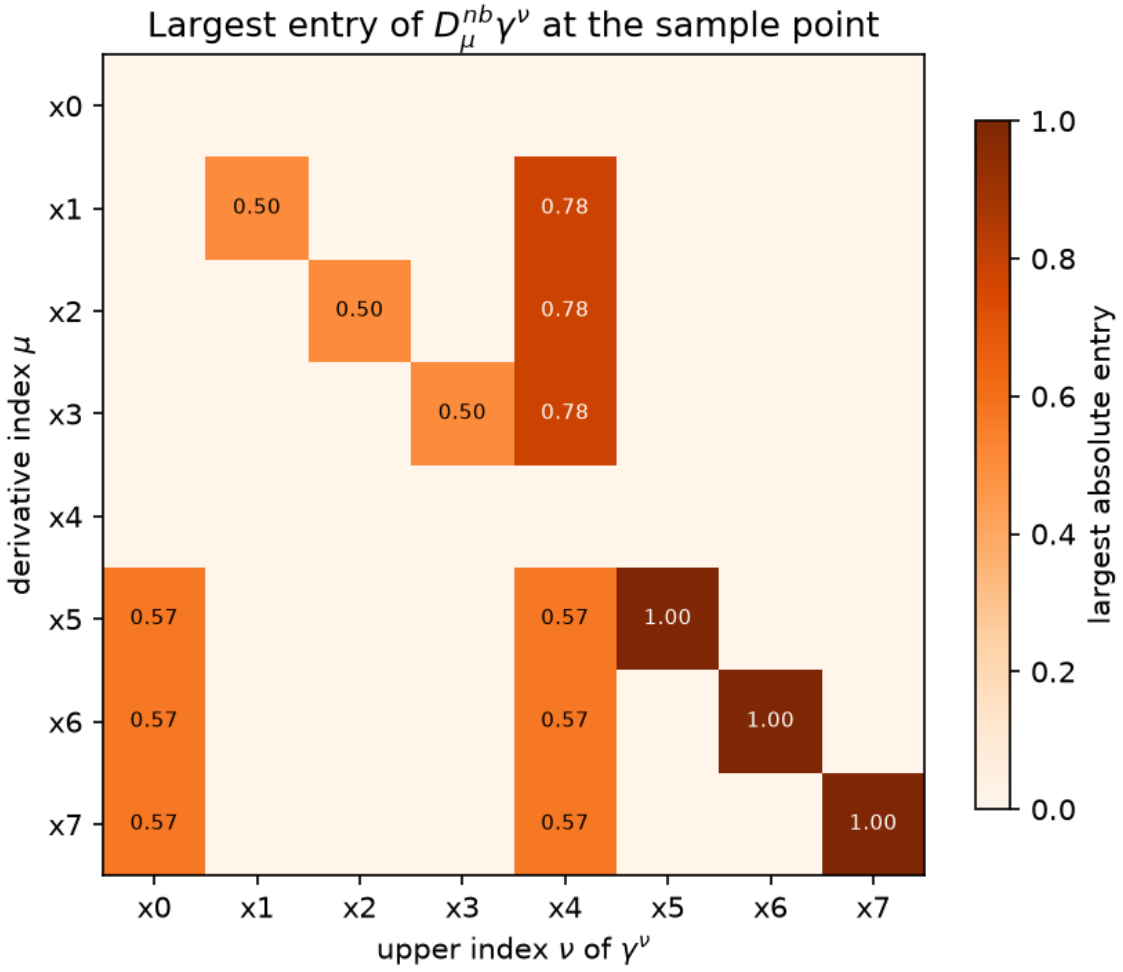

Figure 202: Where the notebook's contraction breaks the covariant constancy of the gammas: for each pair of coordinates, the derivative index μ (rows) and the upper index ν (columns), the largest absolute entry of the 16 x 16 matrix $D_\mu^{nb}\gamma^\nu$ at the sample point $H = 1$, $z = \pi/4$, $a_4 = 1/2$, $a_4' = 1/2$, $b_4 = 1$ (pure numbers; white is zero). Exactly 15 of the 64 pairs are nonzero, all in the rows of the six warped directions; with the correct spinor connection all 64 matrices vanish exactly. (Notebook 06c, figure 5.)

Out [14]:

```
Figure 06c.5 saved as Revision/textbook/figures/06c_5_constancy_violation.png
```

Text cell:

```
## 11. Why only the value $3H$ is right: the half-density term
```

```
Two facts of the Revision record single out the correct value.

**The divergence form.** Because $\{\gamma^\mu, \Omega_\mu\} = 0$ for each $\mu$
and $D_\mu\gamma^\nu = 0$, the correct contraction equals
$\frac{1}{2\sqrt{|g|}}\sum_\mu\partial_\mu(\sqrt{|g|}\,\gamma^\mu)$ (Notebook 06a),
a quantity built from the metric and the vielbein alone. The cell checks that it
equals the correct value and not the notebook's.

**The hidden-direction operator.** For fields that depend on $x_0$ only, the
derivative part of the field equation along $x0$ is the operator $A_c q =
\tan z\,\partial_{x0}q + c\,q$, where $c$ is the coefficient of $\gamma^{(x0)}$ in
$\gamma^\mu\Omega_\mu$. The volume weight is $\sqrt{|g|} = b_4^{3/2}\cos z$; along
the hidden direction the factor $b_4^{3/2}$ is a constant, which multiplies both
sides below and is left out, so the weight is $\cos z$ (the record's formula is
written at $b_4 = 1$). Compute, with the product rule and
$\partial_{x0}\sin z = 6H\cos z$:

$$\partial_{x0}(\sin z\,p\,q) = 6H\cos z\,p\,q + \sin z\,(p'q + p\,q') ,$$

$$\cos z\,\big(p\,A_cq + (A_cp)\,q\big) = \sin z\,(p\,q' + p'q) + 2c\cos z\,p\,q$$

(using $\cos z\tan z = \sin z$). Subtracting,

$$\cos z\,\big(p\,A_cq + (A_cp)\,q\big) - \partial_{x0}(\sin z\,p\,q) =
(2c - 6H)\cos z\,p\,q .$$

For $c = 3H$ the right side vanishes: integrated over $x_0$, $\int\cos z\,p\,A_cq
= -\int\cos z\,(A_cp)\,q$ plus a boundary term, so $A_{3H}$ is antisymmetric for
the weight $\cos z$; this is the record's formula 'hidden_direction_hermiticity',
and it is what makes the operator of the quantum theory Hermitian (up to boundary
conditions). For the notebook's $c = \frac{3H}{2}$ the term $-3H\cos z\,p\,q$
remains: no boundary term can absorb it. The next cell checks all of this exactly
for two arbitrary functions $p(x_0)$, $q(x_0)$.
```

In [15]:

```
sqrt_g = b4 ** sp.Rational(3, 2) * sp.cos(z)  # sqrt|det g| of the author's metric
divergence = sum(((sqrt_g * gup[mu]).diff(x[mu]) for mu in range(8)), Z16) / (2 * sqrt_g)
check(matrix_is_zero(divergence - slash) and not matrix_is_zero(divergence - slash_nb),
      "the divergence form equals the correct contraction, not the notebook's",
      record=f"{REPORT_WL_B4}, check gammaOmega_divergence_form")
p = sp.Function("p")(x0)  # two arbitrary functions of x0
q = sp.Function("q")(x0)
c = sp.Symbol("c", real=True)  # the coefficient of gamma^(x0)


def A(u):
    """The hidden-direction operator A_c u = tan z du/dx0 + c u."""
    return sp.tan(z) * u.diff(x0) + c * u


leftover = sp.cos(z) * (p * A(q) + A(p) * q) - sp.diff(sp.sin(z) * p * q, x0)
check(is_zero(leftover - (2 * c - 6 * H) * sp.cos(z) * p * q),
```

```
      "cos z (p A_c q + (A_c p) q) - d0(sin z p q) = (2c - 6H) cos z p q")
record_text = FORMULAS["hidden_direction_hermiticity"]
check(is_zero(leftover.subs(c, 3 * H)) and "Tan[z] d0 + 3 H" in record_text
      and "= d0 (Sin[z] p q)" in record_text,
      "c = 3H: A_c is antisymmetric for the weight cos z up to d0(sin z p q)",
      record=f"{THEORY_FILE}, formula hidden_direction_hermiticity")
check(is_zero(leftover.subs(c, 3 * H / 2) + 3 * H * sp.cos(z) * p * q),
      "c = 3H/2 (notebook): the leftover -3 H cos z p q remains")
```

Out [15]:

```
PASS the divergence form equals the correct contraction, not the notebook's
     reproduces Revision/theory/reports/wolfram-field-theory-b4.json, check
    gammaOmega_divergence_form
PASS cos z (p A_c q + (A_c p) q) - d0(sin z p q) = (2c - 6H) cos z p q
PASS c = 3H: A_c is antisymmetric for the weight cos z up to d0(sin z p q)
     reproduces Revision/theory/field-theory.json, formula hidden_direction_hermiticity
PASS c = 3H/2 (notebook): the leftover -3 H cos z p q remains
```

Text cell:

```
The next cell draws the leftover coefficient $(2c/H - 6)\cos z$ (the factor in
front of $H\,p\,q$) versus the hidden angle $z$ for three values of $c$: the
correct $c = 3H$ (zero for every $z$), the notebook's $c = \frac32 H$, and $c = 0$
(no spin-connection term at all).
```

In [16]:

```
zz = np.linspace(0.0, np.pi / 2, 300)  # the hidden angle z from 0 to pi/2
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.axhline(0.0, color="gray", linewidth=0.6, zorder=1)  # the zero line, underneath
for value, style, width, label in ((3.0, "-", 2.5, "correct $c = 3H$"),
                                   (1.5, "--", 1.5, "notebook $c = 3H/2$"),
                                   (0.0, ":", 1.5, "no connection term, $c = 0$")):
    ax.plot(zz, (2 * value - 6) * np.cos(zz), style, linewidth=width, label=label,
            zorder=3)  # drawn above the zero line
ax.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax.set_ylabel("leftover coefficient $(2c/H - 6)\\cos z$")
ax.set_title("Only $c = 3H$ makes the hidden operator antisymmetric")
ax.legend(fontsize=8)
save_figure(fig, "hermiticity_defect",
            "The term that spoils the antisymmetry of the hidden-direction operator "
            "$A_c = \\tan z\\,\\partial_{x0} + c$ for the volume weight $\\cos z$: "
            "the coefficient $(2c/H - 6)\\cos z$ of $H\\,p\\,q$ in $\\cos "
            "z\\,(p\\,A_c q + (A_c p)\\,q) - \\partial_{x0}(\\sin z\\,p\\,q)$, a pure "
            "number, versus the hidden angle $z$ in radians, for the correct value "
            "$c = 3H$ of the spin-connection term (solid, zero everywhere), for the "
            "notebook's value $c = 3H/2$ (dashed) and for no spin-connection term, "
            "$c = 0$ (dotted). Only the correct value leaves nothing but a boundary "
            "term; the solid line lies on the zero line.")
```

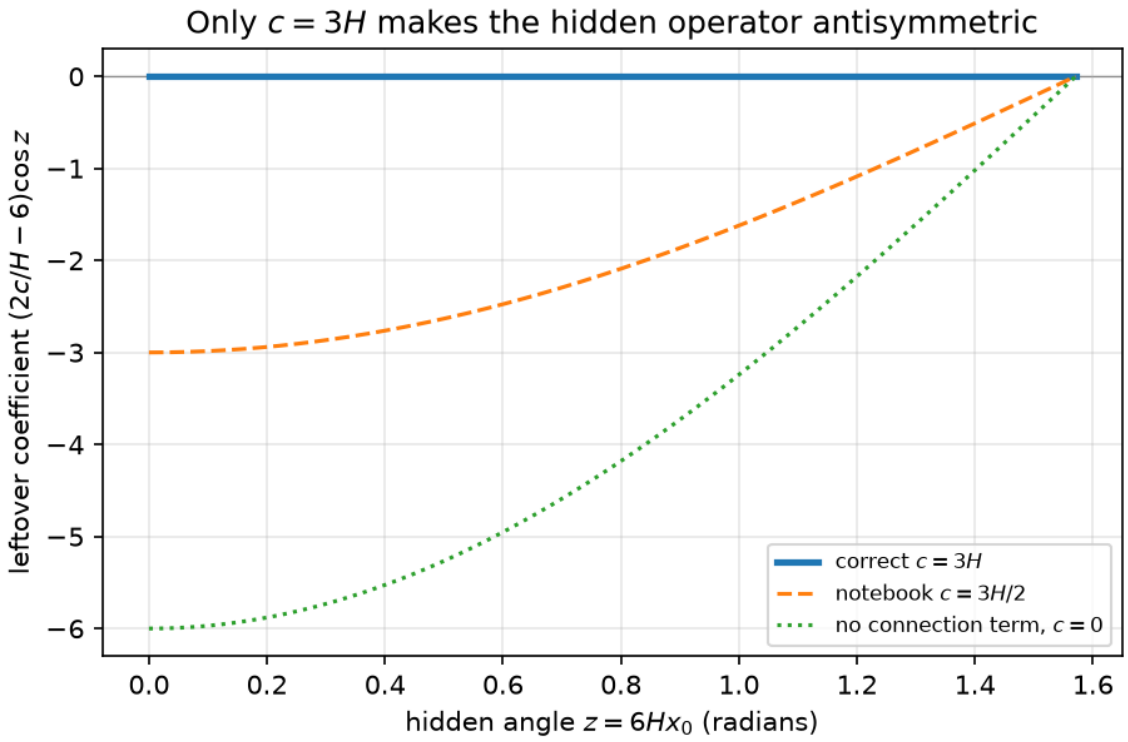

Figure 203: The term that spoils the antisymmetry of the hidden-direction operator $A_c = \tan z\,\partial_{x0} + c$ for the volume weight $\cos z$: the coefficient $(2c/H - 6)\cos z$ of $H\,p\,q$ in $\cos z\,(p\,A_c q + (A_c p)\,q) - \partial_{x0}(\sin z\,p\,q)$, a pure number, versus the hidden angle z in radians, for the correct value $c = 3H$ of the spin-connection term (solid, zero everywhere), for the notebook's value $c = 3H/2$ (dashed) and for no spin-connection term, $c = 0$ (dotted). Only the correct value leaves nothing but a boundary term; the solid line lies on the zero line. (Notebook 06c, figure 6.)

Out [16]:

```
Figure 06c.6 saved as Revision/textbook/figures/06c_6_hermiticity_defect.png
```

Text cell:

```
## 12. The last checks

The last cell checks that every check of the Revision reports that states a
correct result reproduced here is recorded there as passed, that all six figure
files exist in the folder Revision/textbook/figures, and prints the number of
checks that passed.
```

In [17]:

```
cited = {
    REPORT_PY: ["clifford_relations", "gammas_real", "vielbein_postulate",
                "spin_connection_antisymmetric", "covariant_constancy_D_mu_gamma_nu",
                "gamma_mu_Omega_mu_equals_3H_gamma_x0", "divergence_of_sqrtg_gamma"],
    REPORT_WL: ["omega_components", "S_rotates_gamma_with_omega",
                "gamma_covariantly_constant", "gammaOmega_equals_3H_gamma_x0",
                "gammaOmega_divergence_form", "good_sector_hermiticity_curved"],
    REPORT_WL_B4: ["omega_components", "gamma_covariantly_constant",
                   "gammaOmega_total", "gammaOmega_divergence_form"],
}
count = sum(len(names) for names in cited.values())
check(all(record_passed(path, name) for path, names in cited.items()
          for name in names),
      f"the {count} cited record checks are recorded as passed")
expected = [f"06c_{k}_{name}.png" for k, name in enumerate([
    "pair_kinds", "milne_frame_and_spin_boost", "per_direction_comparison",
    "contraction_heat_maps", "constancy_violation", "hermiticity_defect"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected),
      "all six figure files exist")
all_checks_passed()
```

Out [17]:

```
PASS the 17 cited record checks are recorded as passed
PASS all six figure files exist
ALL 27 CHECKS PASSED (notebook 06c)
```

Text cell:

```
## 13. What this notebook showed

- The notebook's contraction $\Omega^{nb}_\mu = \frac12\sum_{a,b}
  \omega_\mu{}^a{}_b S^{ab}$ of the MIXED components equals $\sum_{a<b}
  \frac12(\eta_{aa} + \eta_{bb})\,\omega_{\mu ab}S^{ab}$: it keeps the 6
```

```
  space-space parts, reverses the sign of the 6 time-time parts and deletes the 16
  boost parts of every spin connection (PROVED, exact).
- With a positive metric (the polar plane) the two contractions agree, so the
  mistake cannot show up there; in the Milne wedge, a flat plane with one time, the
  frame is boosted along $\theta$ and the notebook's contraction deletes the whole
  connection $\Omega_\theta = -\frac12\gamma^0\gamma^1$ (PROVED).
- For the author's metric the correct spin connection reproduces the records
  (`omega_nonzero`, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} +
  \frac34(b_4'/b_4)\gamma^{(x4)}$, $D_\mu\gamma^\nu = 0$), while the notebook's
  contraction would give $\gamma^\mu\Omega^{nb}_\mu = \frac{3H}{2}\gamma^{(x0)} +
  \frac32(a_4' - \frac{b_4'}{2b_4})\gamma^{(x4)}$ (at $b_4 = 1$:
  $\frac{3H}{2}\gamma^{(x0)} + \frac{3a_4'}{2}\gamma^{(x4)}$) and would break the
  covariant constancy of the gammas in 15 of the 64 pairs $(\mu, \nu)$, with 288
  nonzero entries (COMPUTED exactly here; not a Revision result).
- Only the correct value $3H$ equals the divergence form
  $\frac{1}{2\sqrt{|g|}}\partial_\mu(\sqrt{|g|}\gamma^\mu)$ and makes the
  hidden-direction operator $\tan z\,\partial_{x0} + 3H$ antisymmetric for the
  volume weight $\cos z$ (times the constant $b_4^{3/2}$ along $x0$) up to the
  boundary term $\partial_{x0}(\sin z\,p\,q)$
  (reproduces the record's formula `hidden_direction_hermiticity`); the
  notebook's value leaves the term $-3H\cos z\,p\,q$.
- This is why the Revision record, and this book, use the lowered components
  $\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b$ in the spinor connection.
```

6.28 Line-by-line walk-through of Notebook 06c

The notebook has 17 code cells, In [1] to In [17]; this section explains every line of every one of them. A line `...` in a quoted figure cell stands for the remaining lines of a long caption, printed in full in Section 6.27.

In [1], the set-up cell. Its comment lines are the run instructions of Section 6.26. Its code is the set-up code of Notebook 06a explained at the beginning of Section 6.14, with the single difference

```
NOTEBOOK_ID = "06c"  # this notebook: chapter 06, example c
```

It prints one line, Set-up of notebook 06c complete: repository folder found, helpers defined.

In [2], the tools and the record.

```
import sys  # the Python system module (here: the stream of printed text)

# Jupyter sends printed text to the screen in pieces, about every 0.2 seconds.  The
# book's checking tool reads each PASS line together with its "reproduces" line, so
# the pieces must be the same in every run: send the printed text of a cell in one
# piece when the cell ends (or just before a figure), at the latest after 600 s.
if hasattr(sys.stdout, "flush_interval"):  # true inside Jupyter only
    sys.stdout.flush_interval = 600.0
```

The setting of the output channel explained in Section 6.14 (Notebook 06a, In [2]).

```
import numpy as np  # decimal numbers (only for the plots)
import sympy as sp  # exact algebra and calculus with symbols
```

numpy for the plots and sympy for the exact algebra.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
NAMES = fixture["coordinates"]  # ["x0", ..., "x7"]
ETA = fixture["eta"]  # +1 space-like, -1 time-like
gamma = [sp.Matrix(rows) for rows in fixture["gamma"]]  # gamma[a] = gamma^(xa)
I16, Z16 = sp.eye(16), sp.zeros(16, 16)  # identity and zero matrix
```

The record of the gammas is read as in Notebook 06a: the names, the signs of η, the eight exact matrices, the identity and the zero matrix.

```
check(all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
          == (2 * ETA[a] if a == b else 0) * I16
          for a in range(8) for b in range(8)),
      "the 64 Clifford relations of the author's gammas",
      record="Revision/theory/reports/python-field-theory.json, "
             "check clifford_relations")
check(len(gamma) == 8 and all(M.shape == (16, 16) for M in gamma)
      and all(entry.is_real for M in gamma for entry in M),
      "the eight gammas are 16 x 16 matrices with real entries",
      record="Revision/theory/reports/python-field-theory.json, check gammas_real")
```

The two checks of Notebook 06a, In [2]: the 64 Clifford relations, and eight real 16×16 matrices, each with its record.

```
S = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
     for a in range(8)]  # S[a][b] = S^ab
```

The 64 generators S^{ab}.

```
def is_zero(expr):
    """True when expr is exactly zero for all values of its symbols."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    if sp.cancel(sp.expand(expr.rewrite(sp.exp))) == 0:  # the fast route
        return True
    return sp.simplify(expr) == 0  # the slower general route
```

The exact zero test (Section 6.14, In [3] of Notebook 06a).

```
def matrix_is_zero(matrix):
    """True when every entry of the matrix is exactly zero."""
    return all(is_zero(entry) for entry in matrix)
```

```
def record_passed(path, name):
    """True when the Revision report at path records the check name as passed."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for entry in data["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper() == "PASS"  # "PASS" or "pass"
    raise KeyError(f"{path} has no check {name}")
```

`matrix_is_zero` and `record_passed`, as in Notebook 06a.

```
THEORY_FILE = "Revision/theory/field-theory.json"  # the record at b4 = 1
THEORY_B4_FILE = "Revision/theory/field-theory-b4.json"  # the record with b4(x4)
FORMULAS = {item["key"]: item["wl"] for item in json.loads(
    repository_file(THEORY_FILE).read_text(encoding="utf-8"))["formulas"]}
FORMULAS_B4 = {item["key"]: item["wl"] for item in json.loads(
    repository_file(THEORY_B4_FILE).read_text(encoding="utf-8"))["formulas"]}
REPORT_PY = "Revision/theory/reports/python-field-theory.json"
REPORT_WL = "Revision/theory/reports/wolfram-field-theory.json"
```

The formulas of `Revision/theory/field-theory.json`, read into a dictionary from key to Wolfram text, and the paths of the two field-theory reports.

```
# " ".join(text.split()) replaces every run of blanks and line breaks by one blank,
# so a sentence that the file breaks across two lines is found as one line.
spec_text = " ".join(repository_file("Revision/SPEC.md").read_text(
    encoding="utf-8").split())
statement = "contraction of the mixed omega_mu^a_b, which lacks a metric factor"
check(statement in spec_text,
      "Revision/SPEC.md section 3 records the missing metric factor")
```

The notebook checks that the statement on which it is built is in the record. `" ".join(text.split())` replaces every run of blanks and line breaks by one blank (`split()` cuts the text at all blanks and line breaks, `" ".join` glues the pieces with single blanks), so that a sentence that the file breaks across two lines is found as one line. The check requires `Revision/SPEC.md` to contain the words "contraction of the mixed omega_mu^a_b, which lacks a metric factor" (`in` tests whether one text occurs inside another). The cell prints three PASS lines.

In [3], the general formulas, now with the mixed connection (Sections 6.4 and 6.23).

```
def christoffel(g, coords):
    """Gam[l][m][n] = (1/2) sum_r g^lr (d_m g_rn + d_n g_rm - d_r g_mn)."""
    n = len(coords)
    g_inv = g.inv()  # the inverse metric
    Gam = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for l in range(n):
        for m in range(n):
            for k in range(m, n):  # Gamma^l_mk = Gamma^l_km: compute it once
                value = sum(g_inv[l, r] * (sp.diff(g[r, k], coords[m])
```

```
                                        + sp.diff(g[r, m], coords[k])
                                        - sp.diff(g[m, k], coords[r]))
                                for r in range(n) if g_inv[l, r] != 0) / 2
                Gam[l][m][k] = Gam[l][k][m] = sp.simplify(value)
    return Gam
```

The Christoffel symbols, as in Notebook 06b, In [3].

```
def mixed_spin_connection(e, Gam, coords):
    """mixed[mu][a][b] = omega_mu^a_b (first frame index up)."""
    n = len(coords)
    E = e.inv()  # E[nu, b] = e_b^nu, the inverse vielbein
    mixed = [[[sp.Integer(0)] * n for _ in range(n)] for _ in range(n)]
    for mu in range(n):
        for a in range(n):
            for b in range(n):
                value = 0
                for nu in range(n):
                    if e[a, nu] == 0:
                        continue  # this term is zero
                    value += e[a, nu] * (sp.diff(E[nu, b], coords[mu]) + sum(
                        Gam[nu][mu][lam] * E[lam, b] for lam in range(n)))
                mixed[mu][a][b] = sp.simplify(value)
    return mixed
```

`mixed_spin_connection` is the spin connection of Notebook 06a, In [4], without the last step: it returns the MIXED components $\omega_\mu{}^a{}_b$ exactly as the formula of Section 6.4 gives them, without multiplying by η_{aa}.

```
def lower_first_index(mixed, eta):
    """omega[mu][a][b] = eta_aa omega_mu^a_b (eta is diagonal)."""
    n = len(mixed)
    return [[[eta[a] * mixed[mu][a][b] for b in range(n)] for a in range(n)]
            for mu in range(n)]
```

`lower_first_index` does the lowering separately: $\omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b$ for all μ, a, b.

```
def contract(coefficients, gens):
    """(1/2) sum over ALL ordered pairs (a, b) of coefficients[mu][a][b] S^ab."""
    n = len(coefficients)
    size = gens[0][0].shape[0]  # 16 for the author's gammas, 2 in a plane
    return [sum((coefficients[mu][a][b] * gens[a][b] / 2 for a in range(n)
                 for b in range(n) if coefficients[mu][a][b] != 0),
                sp.zeros(size, size)) for mu in range(n)]
```

`contract(coefficients, gens)` is $\frac{1}{2}\sum_{a,b} c_{\mu ab} S^{ab}$ for every μ, summed over ALL ordered pairs (a, b) (the term is skipped when the coefficient is zero). Notebook 06a summed over $a < b$ only, which is allowed for the antisymmetric lowered components; the mixed components are not antisymmetric, so here the full sum is needed, and the same function serves both contractions.

```
def curved_gammas(e, gammas):
    """gamma^mu = sum_a e_a^mu gamma^a for every coordinate mu."""
    E = e.inv()
    n = len(gammas)
    return [sum((E[mu, a] * gammas[a] for a in range(n)),
                sp.zeros(*gammas[0].shape)) for mu in range(n)]
```

The curved gammas, as before.

```
def generators(gammas):
    """S[a][b] = (1/4)(gamma^a gamma^b - gamma^b gamma^a)."""
    n = len(gammas)
    return [[(gammas[a] * gammas[b] - gammas[b] * gammas[a]) / 4 for b in range(n)]
            for a in range(n)]
```

`generators(gammas)` builds the matrices $S^{ab} = \frac{1}{4}[\gamma^a, \gamma^b]$ for any list of gammas; it is used for the 2×2 gammas of the two planes.

```
say("defined: christoffel, mixed_spin_connection, lower_first_index, contract, "
    "curved_gammas, generators")
```

One output line names the six functions.

In [4], the three kinds of frame pairs.

```
KIND = {1: "space-space", -1: "time-time", 0: "boost"}  # the factor and its name
factor = [[sp.Rational(ETA[a] + ETA[b], 2) for b in range(8)] for a in range(8)]
pairs = [(a, b) for a in range(8) for b in range(a + 1, 8)]  # the 28 pairs a < b
counts = {k: sum(1 for a, b in pairs if factor[a][b] == k) for k in KIND}
for k, name in KIND.items():
    report(f"number of {name} pairs (factor {k})", counts[k])
```

`KIND` names the three values of the factor. `factor[a][b]` is $\frac{1}{2}(\eta_{aa} + \eta_{bb})$ as an exact fraction (`sp.Rational(n, 2)`); `pairs` lists the 28 pairs $a < b$, and `counts` counts the pairs of each kind (`sum(1 for ... if ...)` counts the cases in which the condition holds). Three RESULT lines print 6, 6 and 16.

```
check(len(pairs) == 28 and counts == {1: 6, -1: 6, 0: 16},
      "28 pairs: 6 space-space (+1), 6 time-time (-1), 16 boost pairs (0)")
```

The check of the counts: 28 pairs, 6 with the factor $+1$, 6 with -1, 16 with 0.

```
w = sp.Symbol("w")  # stands for omega_mu ab; then omega_mu ba = -w
check(all(matrix_is_zero(ETA[a] * w * S[a][b] + ETA[b] * (-w) * S[b][a]
                         - (ETA[a] + ETA[b]) * w * S[a][b]) for a, b in pairs),
      "eta_aa w S^ab + eta_bb (-w) S^ba = (eta_aa + eta_bb) w S^ab, all 28 pairs")
```

The key step of the derivation of Section 6.23 as a matrix identity, for all 28 pairs: with a symbol w standing for $\omega_{\mu ab}$ (so that $\omega_{\mu ba} = -w$), $\eta_{aa}\, w\, S^{ab} + \eta_{bb}(-w) S^{ba} = (\eta_{aa} + \eta_{bb})\, w\, S^{ab}$. The cell prints three RESULT lines and two PASS lines.

In [5], figure 1.

```
from matplotlib.colors import ListedColormap  # a colour map with a few colours
```

`ListedColormap` makes a colour map out of a short list of colours: the number 0 gets the first colour, 1 the second, and so on.

```
grid = np.full((8, 8), 3)  # 3 = the diagonal (white)
for a in range(8):
    for b in range(8):
        if a != b:
            grid[a, b] = {1: 0, -1: 1, 0: 2}[int(factor[a][b])]  # colour number
```

`grid` is an 8×8 array filled with 3 (white, for the diagonal); off the diagonal the factor $+1$, -1, 0 is translated into the colour numbers 0, 1, 2 by the small dictionary `{1: 0, -1: 1, 0: 2}`.

```
colours = ListedColormap(["tab:green", "tab:orange", "lightgray", "white"])
fig, ax = plt.subplots(figsize=(6.4, 5.6))
ax.imshow(grid, cmap=colours, vmin=-0.5, vmax=3.5)
```

The four colours green, orange, light gray and white; `imshow` draws the grid with them (the limits -0.5 and 3.5 put each whole number in the middle of its colour).

```
for a in range(8):
    for b in range(8):
        if a != b:
            ax.text(b, a, f"{int(factor[a][b]):+d}" if factor[a][b] else "0",
                    ha="center", va="center", fontsize=9)
```

Each off-diagonal square gets its factor written in it, `+1`, `-1` or `0` (`:+d` writes a whole number with its sign).

```
ax.set_xticks(range(8), NAMES)
ax.set_yticks(range(8), NAMES)
ax.set_xlabel("frame direction $b$")
ax.set_ylabel("frame direction $a$")
ax.set_title("Factor $(\\eta_{aa} + \\eta_{bb})/2$ of the notebook's contraction")
ax.grid(False)
```

Labels and title; no grid lines.

```
for colour, label in (("tab:green", "kept: space-space (+1)"),
                      ("tab:orange", "sign reversed: time-time (-1)"),
                      ("lightgray", "deleted: boost pair (0)")):
    ax.plot([], [], "s", color=colour, markersize=9, label=label)  # legend only
ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.13), ncol=2, fontsize=8)
```

Three empty plots with square markers (`"s"`) only make the legend entries, and the legend is placed below the picture in two columns (`bbox_to_anchor` gives its position relative to the panel).

```
save_figure(fig, "pair_kinds",
            "What the missing metric factor does to each pair of frame directions "
            ...
            "reversed and 16 deleted.")
```

The figure is saved; the cell prints its file name.

What Figure 06c.1 shows. Green squares (+1, kept) fill the pairs among $x0, x1, x2, x3$, orange squares (-1, sign reversed) the pairs among $x4, x5, x6, x7$, and gray squares (0, deleted) form two 4×4 rectangles: every space-like direction paired with every time-like direction. Counting the upper half: 6 green, 6 orange, 16 gray. The mixed contraction deletes more than half of the spin connection.

In [6], the two flat planes (Section 6.23).

```
r, phi = sp.symbols("r phi", positive=True)  # the polar plane
sigma1 = sp.Matrix([[0, 1], [1, 0]])
sigma2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
sigma3 = sp.Matrix([[1, 0], [0, -1]])
polar_S = generators([sigma1, sigma2])
polar_e = sp.diag(1, r)  # unit radial and unit angular direction
polar_mixed = mixed_spin_connection(polar_e, christoffel(sp.diag(1, r**2), [r, phi]),
                                    [r, phi])
polar_Omega = contract(lower_first_index(polar_mixed, [1, 1]), polar_S)
polar_Omega_nb = contract(polar_mixed, polar_S)
check(polar_Omega == polar_Omega_nb and polar_Omega[1] == -sp.I / 2 * sigma3,
      "polar plane (eta = +1, +1): both contractions give Omega_phi = -(i/2) sigma3")
```

The polar plane: the Pauli matrices, their generators, the vielbein $\mathrm{diag}(1, r)$, the mixed connection of the metric $\mathrm{diag}(1, r^2)$, and both contractions, the correct one from the lowered components (with $\eta = (1, 1)$) and the mixed one. The check requires them to be equal and $\Omega_\varphi = -\frac{i}{2}\sigma_3$, as in Notebook 06a.

```
tau = sp.Symbol("tau", positive=True)  # the Milne wedge: tau > 0
theta = sp.Symbol("theta", real=True)  # the rapidity coordinate
g0 = sp.Matrix([[0, 1], [-1, 0]])  # real, squares to -1 (the time direction)
g1 = sp.Matrix([[0, 1], [1, 0]])  # real, squares to +1 (the space direction)
I2, Z2 = sp.eye(2), sp.zeros(2, 2)
check(g0 * g0 == -I2 and g1 * g1 == I2 and g0 * g1 + g1 * g0 == Z2,
      "Milne: real gammas with (g0)^2 = -1, (g1)^2 = +1, g0 g1 + g1 g0 = 0")
```

The Milne wedge: the positive symbol τ, the real symbol θ (the rapidity can be negative), the two real gammas of Section 6.23 and the check of their three relations.

```
milne_S = generators([g0, g1])
milne_e = sp.diag(1, tau)  # unit time direction and unit space direction
```

```
milne_Gam = christoffel(sp.diag(-1, tau**2), [tau, theta])
milne_mixed = mixed_spin_connection(milne_e, milne_Gam, [tau, theta])
milne_lowered = lower_first_index(milne_mixed, [-1, 1])
```

The generators, the vielbein $\mathrm{diag}(1, \tau)$, the Christoffel symbols of the metric $\mathrm{diag}(-1, \tau^2)$, the mixed connection, and its lowered form with $\eta = (-1, +1)$.

```
say(f"Gamma^tau_thetatheta = {milne_Gam[0][1][1]}, "
    f"Gamma^theta_tautheta = {milne_Gam[1][0][1]}")
say(f"mixed: omega_theta^0_1 = {milne_mixed[1][0][1]}, "
    f"omega_theta^1_0 = {milne_mixed[1][1][0]}")
say(f"lowered: omega_theta01 = {milne_lowered[1][0][1]}, "
    f"omega_theta10 = {milne_lowered[1][1][0]}")
check(milne_mixed[1][0][1] == 1 and milne_mixed[1][1][0] == 1
      and milne_lowered[1][0][1] == -1 and milne_lowered[1][1][0] == 1,
      "Milne: mixed components symmetric (1, 1), lowered antisymmetric (-1, +1)")
```

Three output lines print $\Gamma^{\tau}{}_{\theta\theta} = \tau$, $\Gamma^{\theta}{}_{\tau\theta} = 1/\tau$, the mixed components $1, 1$ and the lowered ones $-1, 1$; the check requires exactly these: mixed symmetric, lowered antisymmetric.

```
milne_Omega = contract(milne_lowered, milne_S)
milne_Omega_nb = contract(milne_mixed, milne_S)
X2 = g0 * g1  # gamma^0 gamma^1, the boost generator of the plane (X2^2 = 1)
check(milne_Omega[0] == Z2 and milne_Omega[1] == -X2 / 2
      and milne_Omega_nb == [Z2, Z2],
      "Milne: Omega_theta = -(1/2) g0 g1, but the notebook's contraction gives 0")
```

The two contractions. `X2` is $\gamma^0\gamma^1$, the boost generator of the plane times 2. The check requires $\Omega_\tau = 0$, $\Omega_\theta = -\frac{1}{2}\gamma^0\gamma^1$, and the mixed contraction to be zero in both directions (a list of two zero matrices).

```
U = sp.cosh(theta / 2) * I2 + sp.sinh(theta / 2) * X2  # the spinor boost
removed = U.inv() * milne_Omega[1] * U - U.inv().diff(theta) * U  # frame change U^-1
check(matrix_is_zero(-U.diff(theta) * U.inv() - milne_Omega[1])
      and matrix_is_zero(removed),
      "Milne: Omega_theta = -(dU/dtheta) U^-1 for the spinor boost U; U^-1 removes it")
```

The spinor boost $U(\theta) = \cosh\frac{\theta}{2} + \sinh\frac{\theta}{2}\,\gamma^0\gamma^1$. `removed` is the rule $R\,\Omega_\theta R^{-1} - (\partial_\theta R)R^{-1}$ of a change of frame (Section 6.15) with $R = U^{-1}$, whose inverse is U. The check requires $\Omega_\theta = -(\partial_\theta U)U^{-1}$ (the same rule with $R = U$ applied to the connection 0 of the frame of t and y: U carries that frame into the Milne frame) and `removed` to be zero (the inverse boost U^{-1} removes the connection), as worked by hand in Section 6.23.

```
milne_gup = curved_gammas(milne_e, [g0, g1])  # gamma^tau = g0, gamma^theta = g1/tau
slash_milne = (milne_gup[0] * milne_Omega[0] + milne_gup[1] * milne_Omega[1])
divergence_milne = ((tau * milne_gup[0]).diff(tau)
                    + (tau * milne_gup[1]).diff(theta)) / (2 * tau)
check(matrix_is_zero(slash_milne - g0 / (2 * tau))
      and matrix_is_zero(divergence_milne - slash_milne),
      "Milne: gamma^mu Omega_mu = g0/(2 tau) = the divergence form (notebook: 0)")
```

The curved gammas $\gamma^\tau = \gamma^0$, $\gamma^\theta = \gamma^1/\tau$, the correct contraction, and the divergence form with $\sqrt{|g|} = \tau$; the check requires both to be $\gamma^0/(2\tau)$, which the mixed value 0 violates. The cell prints three lines and six PASS lines.

In [7], figure 2.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.8))
curve = np.linspace(-1.3, 1.3, 200)  # values of theta for the hyperbolas
for radius in (1.0, 2.0):
    ax_left.plot(radius * np.sinh(curve), radius * np.cosh(curve), ":",
                 color="gray", linewidth=1.0)
    for rapidity in (-1.0, -0.5, 0.0, 0.5, 1.0):
        py, pt = radius * np.sinh(rapidity), radius * np.cosh(rapidity)  # the point
        ax_left.quiver(py, pt, np.sinh(rapidity), np.cosh(rapidity),
                       color="tab:blue", angles="xy", scale_units="xy", scale=2.5,
                       width=0.006)  # e_(0), the unit time direction
        ax_left.quiver(py, pt, np.cosh(rapidity), np.sinh(rapidity),
                       color="tab:orange", angles="xy", scale_units="xy",
                       scale=2.5, width=0.006)  # e_(1), the unit space direction
```

Two panels. On the left, for $\tau = 1$ and $\tau = 2$, the dotted curve $(y, t) = (\tau \sinh\theta, \tau\cosh\theta)$ for 200 values of θ, and at five rapidities $\theta = -1, -0.5, 0, 0.5, 1$ the frame: the unit time direction $(\sinh\theta, \cosh\theta)$ in blue and the unit space direction $(\cosh\theta, \sinh\theta)$ in orange, both in (y, t) components, drawn with `quiver` as in Notebook 06a.

```
light = np.linspace(0.0, 3.6, 2)
ax_left.plot(light, light, "--", color="black", linewidth=0.8)  # t = y
ax_left.plot(-light, light, "--", color="black", linewidth=0.8,
             label="light lines $t = \\pm y$")  # t = -y
ax_left.plot([], [], color="tab:blue", label="unit time direction $e_{(0)}$")
ax_left.plot([], [], color="tab:orange", label="unit space direction $e_{(1)}$")
```

The two dashed light lines $t = y$ and $t = -y$ from the origin, and two empty plots for the legend.

```
ax_left.set_xlim(-3.6, 3.6)
ax_left.set_ylim(0.0, 4.6)
ax_left.set_aspect("equal")
ax_left.set_xlabel("space coordinate $y = \\tau\\sinh\\theta$")
ax_left.set_ylabel("time $t = \\tau\\cosh\\theta$")
ax_left.set_title("The Milne frame is boosted along $\\theta$")
ax_left.legend(loc="upper center", fontsize=7)
```

Axis ranges, equal units, labels, title and legend.

```
rapidities = np.linspace(-4.0, 4.0, 401)
ax_right.plot(rapidities, np.exp(rapidities / 2),
              label="$U_{11} = e^{\\theta/2}$")
ax_right.plot(rapidities, np.exp(-rapidities / 2), "--",
              label="$U_{22} = e^{-\\theta/2}$")
```

```
ax_right.plot(rapidities, np.cosh(rapidities / 2), ":", color="black",
              label="$\\cosh(\\theta/2)$")
ax_right.plot(rapidities, np.sinh(rapidities / 2), "-.", color="gray",
              label="$\\sinh(\\theta/2)$")
ax_right.set_xlabel("rapidity $\\theta$")
ax_right.set_ylabel("entry of the spinor boost (pure number)")
ax_right.set_title("A spinor boost is real and grows")
ax_right.legend(fontsize=8)
```

The right panel: for θ from -4 to 4 the two diagonal entries of the spinor boost, $e^{\theta/2}$ and $e^{-\theta/2}$ (because $\gamma^0\gamma^1 = \mathrm{diag}(1,-1)$, $U = \mathrm{diag}(\cosh\frac{\theta}{2} + \sinh\frac{\theta}{2}, \cosh\frac{\theta}{2} - \sinh\frac{\theta}{2})$), and for comparison $\cosh\frac{\theta}{2}$ and $\sinh\frac{\theta}{2}$.

```
save_figure(fig, "milne_frame_and_spin_boost",
            "The Milne wedge, a flat plane with one time in which the notebook's "
            ...
            "$\\theta$: real and unbounded, unlike a spinor rotation.")
```

The figure is saved; the cell prints its file name.

What Figure 06c.2 shows. On the left, the frame at $\theta = 0$ is upright (time up, space to the right); moving along θ the blue and orange arrows tilt symmetrically towards the dashed light lines, as in figure 1 of Notebook 06b: the frame is boosted along θ, and the correct spin connection $\omega_{\theta 01} = -1$ records this rate, although the plane is flat. On the right, the entries $e^{\pm\theta/2}$ of the spinor boost $U(\theta)$, which carries the frame of t and y into the Milne frame (its inverse removes the connection), grow or fall without bound and never return, unlike the spinor rotation of figure 1 of Notebook 06a, which came back after 4π. The mixed contraction would delete this whole connection.

In [8], the author's metric: the correct and the mixed spinor connection (Section 6.23).

```
from sympy.parsing.mathematica import parse_mathematica  # reads Wolfram notation
```

The reader of Wolfram notation.

```
x = sp.symbols("x0:8", real=True)  # x[0] = x0, ..., x[4] = x4, ..., x[7] = x7
x0, x4 = x[0], x[4]  # the hidden direction and the time
H = sp.Symbol("H", positive=True)  # the author's constant H > 0
a4 = sp.Function("a4")(x4)  # any function of the time
b4 = sp.Function("b4", positive=True)(x4)  # any positive function of the time
a4p, b4p = sp.Derivative(a4, x4), sp.Derivative(b4, x4)  # a4' and b4'
rate_t = a4p - b4p / (2 * b4)  # the rate of the extra times, a4' - b4'/(2 b4)
z = 6 * H * x0  # the hidden angle
s = sp.sin(z) ** sp.Rational(1, 6)
f = ([sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
     + [sp.exp(-a4) * sp.sqrt(b4) * s] * 3)  # f_0, ..., f_7
g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # the author's metric
e = sp.diag(*f)  # the diagonal vielbein
```

The symbols, the factors, the metric with b_4, the diagonal vielbein and the helper `at_b4_1`, as in Notebook 06a, In [7]; `a4p` and `b4p` are short names for a_4' and b_4', and `rate_t` is the rate $a_4' - b_4'/(2b_4)$ of the extra times.

```
Gam = christoffel(g, x)
mixed = mixed_spin_connection(e, Gam, x)  # omega_mu^a_b
omega = lower_first_index(mixed, ETA)  # omega_mu ab
```

The Christoffel symbols, the mixed connection $\omega_\mu{}^a{}_b$ and the lowered connection $\omega_{\mu ab}$.

```
names = {sp.Symbol("H"): H, sp.Symbol("x0"): x0, sp.Symbol("x4"): x4,
         sp.Symbol("a4"): a4, sp.Symbol("b4"): b4, sp.Symbol("a4p"): a4p,
         sp.Symbol("b4p"): b4p}


def record_omega(formulas):
    text = formulas["omega_nonzero"]
    for long_form, short_form in (("Derivative[1][a4][x4]", "a4p"),
                                  ("Derivative[1][b4][x4]", "b4p"),
                                  ("a4[x4]", "a4"), ("b4[x4]", "b4")):
        text = text.replace(long_form, short_form)
    return {(int(m) - 1, int(a) - 1, int(b) - 1): v.subs(names, simultaneous=True)
            for m, a, b, v in parse_mathematica(text)}


omega_record = record_omega(FORMULAS_B4)
omega_record_1 = record_omega(FORMULAS)
```

`record_omega(formulas)` reads the list `omega_nonzero` of a record: its Wolfram names for a_4', b_4', a_4 and b_4 are shortened, the text is parsed, the plain symbols are replaced by the notebook's, and the directions are renumbered (the records number them from 1 in the order $x0, \ldots, x7$, Python from 0). Both records are read, the one with b_4 and the one of $b_4 = 1$.

```
nonzero = {(mu, a, b): omega[mu][a][b] for mu in range(8) for a, b in pairs
           if omega[mu][a][b] != 0}
check(set(nonzero) == set(omega_record) == set(omega_record_1)
      and all(is_zero(nonzero[k] - omega_record[k])
              and is_zero(at_b4_1(nonzero[k]) - omega_record_1[k]) for k in nonzero),
      "the 12 nonzero lowered components omega_mu ab equal the records (with b4, and "
      "at b4 = 1)",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula omega_nonzero")
```

The nonzero lowered components with $a < b$, compared with both records: the same twelve keys, and the same values as the record with b_4 and, at $b_4 = 1$, as the record of $b_4 = 1$.

```
ACTION = {1: "kept", -1: "sign reversed", 0: "deleted"}
for (mu, a, b) in sorted(nonzero):
    k = int(factor[a][b])
    say(f"omega_{NAMES[mu]},{NAMES[a]}{NAMES[b]}: {KIND[k]} pair, {ACTION[k]}")
```

For each of the twelve components (sorted by their keys) one line is printed with the kind of its pair and what the mixed contraction does to it, for example `omega_x1,x1x4:  boost pair, deleted` and `omega_x5,x4x5:  time-time pair, sign reversed`.

```
Omega = contract(omega, S)  # the correct spinor connection
Omega_nb = contract(mixed, S)  # the notebook's contraction
```

The correct spinor connection is the contraction of the lowered components, the mixed one the contraction of the mixed components; the same function `contract` makes both.

```
def part(kind):
    """The part of Omega that comes from the pairs of one kind."""
    return contract([[[omega[mu][a][b] if factor[a][b] == kind else 0
                       for b in range(8)] for a in range(8)] for mu in range(8)], S)
```

`part(kind)` keeps only the lowered components whose pair is of the given kind (all others are set to 0) and contracts them: the parts Ω^{ss}, Ω^{tt} and Ω^{st}.

```
Omega_ss, Omega_tt, Omega_st = part(1), part(-1), part(0)
check(all(matrix_is_zero(Omega[mu] - Omega_ss[mu] - Omega_tt[mu] - Omega_st[mu])
          and matrix_is_zero(Omega_nb[mu] - Omega_ss[mu] + Omega_tt[mu])
          for mu in range(8)),
      "Omega^nb_mu = Omega^ss_mu - Omega^tt_mu for all eight mu (boosts deleted)")
```

The check of the identity of Section 6.23 for all eight μ: Ω_μ is the sum of its three parts, and $\Omega^{nb}_\mu = \Omega^{ss}_\mu - \Omega^{tt}_\mu$. The cell prints twelve lines and two PASS lines.

In [9], the closed forms.

```
closed_ok = True
for i in (1, 2, 3):  # the 3-space directions x1, x2, x3
    closed_ok = closed_ok and matrix_is_zero(
        Omega_nb[i] - sp.exp(a4) * s * H * S[i][0])  # pair (x0, xi) kept
    closed_ok = closed_ok and matrix_is_zero(
        Omega[i] - sp.exp(a4) * s * (a4p * S[i][4] + H * S[i][0]))
f_t = sp.exp(-a4) * sp.sqrt(b4) * s  # the factor of the extra times
for t in (5, 6, 7):  # the extra times x5, x6, x7
    closed_ok = closed_ok and matrix_is_zero(
        Omega_nb[t] - f_t * rate_t * S[4][t])  # pair (x4, xt) reversed
    closed_ok = closed_ok and matrix_is_zero(
        Omega[t] + f_t * (rate_t * S[4][t] + H * S[t][0]))
check(closed_ok, "the closed forms of Omega_mu and Omega^nb_mu for x1 ... x3, "
      "x5 ... x7")
```

For the three 3-space directions (positions 1, 2, 3) the check requires $\Omega^{nb}_{x_i} = e^{a_4} s\, H\, S^{x_i x0}$ and $\Omega_{x_i} = e^{a_4} s\, (a'_4 S^{x_i x4} + H S^{x_i x0})$ (`S[i][0]` is $S^{x_i x0}$, `S[i][4]` is $S^{x_i x4}$); for the three extra times (positions 5, 6, 7), with `f_t` $= e^{-a_4} b_4^{1/2} s$ and $r =$ `rate_t`, $\Omega^{nb}_{x_t} = f_t\, r\, S^{x4 x_t}$ and $\Omega_{x_t} = -f_t\, (r\, S^{x4 x_t} + H S^{x_t x0})$. One PASS line.

In [10], the two contractions direction by direction.

```
gup = curved_gammas(e, gamma)  # gamma^mu = gamma^a / f_a for mu = a
```

The curved gammas.

```
def split(M):
    """(alpha, beta, rest is zero) for M = alpha gamma^(x4) + beta gamma^(x0)."""
    alpha = sp.simplify(-(M * gamma[4]).trace() / 16)  # coefficient of gamma^(x4)
    beta = sp.simplify((M * gamma[0]).trace() / 16)  # coefficient of gamma^(x0)
    return alpha, beta, matrix_is_zero(M - alpha * gamma[4] - beta * gamma[0])
```

`split(M)` returns the coefficients α of $\gamma^{(x4)}$ and β of $\gamma^{(x0)}$, found with traces as in Section 6.8, and whether nothing else is left.

```
correct_terms = [split(gup[mu] * Omega[mu]) for mu in range(8)]
notebook_terms = [split(gup[mu] * Omega_nb[mu]) for mu in range(8)]
for mu in range(8):
    say(f"{NAMES[mu]}: correct ({correct_terms[mu][0]}, {correct_terms[mu][1]}), "
        f"notebook ({notebook_terms[mu][0]}, {notebook_terms[mu][1]})")
```

Both contractions are split for each direction and printed side by side, eight lines such as `x5: correct (-Derivative(a4(x4), x4)/2, H/2), notebook (Derivative(a4(x4), x4)/2, 0)`.

```
want_nb = [(0, 0)] + [(0, H / 2)] * 3 + [(0, 0)] + [(rate_t / 2, 0)] * 3
check(all(term[2] for term in correct_terms + notebook_terms)
      and all(is_zero(notebook_terms[mu][0] - want_nb[mu][0])
              and is_zero(notebook_terms[mu][1] - want_nb[mu][1]) for mu in range(8)),
      "notebook's terms: (H/2) gamma^(x0) for x1, x2, x3; ((a' - b4'/(2 b4))/2) "
      "gamma^(x4) for x5, x6, x7")
```

The check requires every split to leave nothing over and the mixed terms to be $(0,0)$ for $x0$, $(0,\frac{H}{2})$ for $x1, x2, x3$, $(0,0)$ for $x4$ and $(\frac{1}{2}(a_4' - \frac{b_4'}{2b_4}),0)$ for $x5, x6, x7$, as derived in Section 6.23.

```
slash = sum((gup[mu] * Omega[mu] for mu in range(8)), Z16)
slash_nb = sum((gup[mu] * Omega_nb[mu] for mu in range(8)), Z16)


def record_total(formulas):
    text = formulas["gammaOmega_total"].replace(chr(34), "")  # gamma[x0], gamma[x4]
    for long_form, short_form in (("Derivative[1][b4][x4]", "b4p"),
                                  ("b4[x4]", "b4"), ("gamma[x0]", "G0"),
                                  ("gamma[x4]", "G4")):
        text = text.replace(long_form, short_form)
    total = parse_mathematica(text).subs(names, simultaneous=True)
    G0, G4 = sp.Symbol("G0"), sp.Symbol("G4")
    return total.coeff(G0) * gamma[0] + total.coeff(G4) * gamma[4]


check(matrix_is_zero(slash - record_total(FORMULAS_B4))
      and matrix_is_zero(at_b4_1(slash) - record_total(FORMULAS))
      and record_passed(REPORT_PY, "gamma_mu_Omega_mu_equals_3H_gamma_x0"),
      "correct: gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); "
      "3 H gamma^(x0) at b4 = 1",
      record=f"{THEORY_B4_FILE} and {THEORY_FILE}, formula gammaOmega_total")
```

The two totals. `sum(..., Z16)` adds the eight matrices $\gamma^\mu\Omega_\mu$, one for each direction μ, starting from the zero matrix `Z16`: `slash` is the correct total $\sum_\mu \gamma^\mu\Omega_\mu$ and `slash_nb` the total of the notebook's contraction. `record_total(formulas)` turns the formula `gammaOmega_total` of a record into a matrix. In the record with b_4 it is the text `3*H*gamma["x0"] + (3*gamma["x4"]*Derivative[1][b4][x4])/(4*b4[x4])`, in the record of $b_4 = 1$ the text `3*H*gamma["x0"]`. `chr(34)` is the double-quote character (34 is its number in the ASCII table), and `.replace(chr(34), "")` removes every double quote, as the comment says. (The code avoids writing the quoted name `gamma["x0"]` inside a Python string: that would need a string in single quotes, and the book prints straight single quotes in code as curly ones, which Python does not accept.) The loop then replaces the Wolfram names of b_4' and b_4 by short names and `gamma[x0]`, `gamma[x4]` by the plain names `G0`, `G4`, and `parse_mathematica` (In [8]) reads the text as a sympy expression; `.subs` replaces its plain symbols by the notebook's (its `H` is a new symbol that knows nothing about its sign). `.coeff(G0)` and `.coeff(G4)` pick out the factors of the two names, and the function returns that combination of $\gamma^{(x0)}$ and $\gamma^{(x4)}$. The check requires the correct total to be the record's with b_4, and at $b_4 = 1$ the record's $3H\gamma^{(x0)}$, and, through `record_passed`, the sympy report `python-field-theory.json` (the name `REPORT_PY` of In [2]) to record its check `gamma_mu_Omega_mu_equals_3H_gamma_x0` as passed. Its second printed line names the record formulas it reproduces.

```
report("notebook: coefficient of gamma^(x0)", sum(t[1] for t in notebook_terms))
report("notebook: coefficient of gamma^(x4)",
       sp.simplify(sum(t[0] for t in notebook_terms)))
check(matrix_is_zero(slash_nb - 3 * H / 2 * gamma[0] - 3 * rate_t / 2 * gamma[4]),
      "notebook: gamma^mu Omega^nb_mu = (3H/2) gamma^(x0) + (3/2)(a4' - b4'/(2 b4)) "
      "gamma^(x4)")
```

Two RESULT lines print the mixed coefficients $\frac{3H}{2}$ and $\frac{3}{2}a_4' - \frac{3}{4}b_4'/b_4$, and the last check requires the mixed total to be exactly $\frac{3H}{2}\gamma^{(x0)} + \frac{3}{2}(a_4' - \frac{b_4'}{2b_4})\gamma^{(x4)}$. The cell prints eight lines, three PASS lines and two RESULT lines.

In [11], figure 3.

```
positions = np.arange(8)
width = 0.38  # the width of one bar; two bars side by side per direction
alpha_c = [float(sp.simplify(at_b4_1(t[0]) / a4p)) for t in correct_terms]  # b4 = 1
alpha_n = [float(sp.simplify(at_b4_1(t[0]) / a4p)) for t in notebook_terms]
beta_c = [float(sp.simplify(t[1] / H)) for t in correct_terms]  # units of H
beta_n = [float(sp.simplify(t[1] / H)) for t in notebook_terms]
```

The places of the bars, the width of one bar (two bars side by side for each direction), and the coefficients in units of a_4' (on the background $b_4 = 1$, `at_b4_1`) and of H for both contractions.

```
fig, (ax_left, ax_right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
ax_left.bar(positions - width / 2, alpha_c, width, color="tab:blue",
            label=f"correct (total {sum(alpha_c):g})")
ax_left.bar(positions + width / 2, alpha_n, width, color="lightskyblue",
            label=f"notebook (total {sum(alpha_n):g})")
ax_right.bar(positions - width / 2, beta_c, width, color="tab:red",
             label=f"correct (total {sum(beta_c):g})")
ax_right.bar(positions + width / 2, beta_n, width, color="lightsalmon",
             label=f"notebook (total {sum(beta_n):g})")
```

Two bar charts with a common vertical axis: the correct bars dark (blue on the left, red on the right) and shifted half a bar to the left, the mixed bars light and shifted to the right; each label shows the total.

```
ax_left.set_title("coefficient of $\\gamma^{(x4)}$, units of $a_4'$")
ax_right.set_title("coefficient of $\\gamma^{(x0)}$, units of $H$")
for ax in (ax_left, ax_right):
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, NAMES)
    ax.set_xlabel("direction $\\mu$ of the term $\\gamma^\\mu\\Omega_\\mu$")
    ax.set_ylim(-0.8, 0.8)
    ax.legend(fontsize=8, loc="lower left")
ax_left.set_ylabel("coefficient")
```

Titles, a zero line, the direction names, the axis range from -0.8 to 0.8 and the legends.

```
save_figure(fig, "per_direction_comparison",
            "The eight terms $\\gamma^\\mu\\Omega_\\mu$ (no sum) of the author's "
            ...
            "to $3$, the notebook keeps only three of them, total $3/2$.")
```

The figure is saved; the cell prints its file name.

What Figure 06c.3 shows. On the left, the dark bars are $+\frac{1}{2}$ for $x1, x2, x3$ and $-\frac{1}{2}$ for $x5, x6, x7$ and cancel (total 0), while the light bars are zero for 3-space and $+\frac{1}{2}$ for the extra times (total $\frac{3}{2}$): the deleted boost parts of 3-space and the reversed time-time parts of the extra times destroy the cancellation. On the right, the six dark bars of height $\frac{1}{2}$ add to 3, while the mixed contraction keeps only the three of 3-space (total $\frac{3}{2}$): the boost parts $(x_t, x0)$ of the extra times are deleted.

In [12], figure 4.

```
SAMPLE = {H: 1, x0: sp.pi / 24}  # H = 1 and z = 6 H x0 = pi/4


def at_sample(expr):
    """The decimal value of expr at H = 1, z = pi/4, a4 = 1/2, a4' = 1/2, b4 = 1."""
    expr = at_b4_1(expr).subs(a4p, sp.Rational(1, 2))  # a4' first, then a4
    expr = expr.subs(a4, sp.Rational(1, 2))
    return float(sp.N(expr.subs(SAMPLE)))


def sample_table(matrix):
    """The matrix as an array of decimal numbers at the sample point."""
    return np.array([[at_sample(v) for v in row] for row in matrix.tolist()])
```

The sample point of Notebook 06a on the background $b_4 = 1$ ($H = 1$, $z = \pi/4$, $a_4 = a_4' = \frac{1}{2}$, $b_4 = 1$; here no a_4'' occurs), with $b_4 = 1$ put in first (`at_b4_1`) and then a_4' before a_4; and `sample_table`, as in Notebook 06b.

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, (title, matrix) in zip(axes, (
```

```
        ("correct: $\\gamma^\\mu\\Omega_\\mu$", slash),
        ("notebook: $\\gamma^\\mu\\Omega^{nb}_\\mu$", slash_nb))):
    image = ax.imshow(sample_table(matrix), cmap="RdBu_r", vmin=-3.2, vmax=3.2)
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])
    ax.grid(False)
fig.colorbar(image, ax=axes, shrink=0.85, label="entry value")
```

Two heat maps on one common colour scale from -3.2 to 3.2: the correct total and the mixed total.

```
save_figure(fig, "contraction_heat_maps",
            "The total $\\gamma^\\mu\\Omega_\\mu$ as a 16 x 16 heat map at the sample "
            ...
            "deflation.")
```

The figure is saved; the cell prints its file name.

What Figure 06c.4 shows. On the left, the correct $3H\gamma^{(x0)}$: the value 3 (dark red) on two diagonal lines in the off-diagonal blocks, nothing else. On the right, the same two lines with half the value, 1.5, and in addition the pattern of $\gamma^{(x4)}$ with entries ± 0.75: with the mixed contraction the field equation would contain a term proportional to the rate a_4' of the deflation.

In [13], the covariant constancy (Section 6.23).

```
violation = {}  # (mu, nu) -> D^nb_mu gamma^nu, for the pairs where it is not zero
correct_ok, identity_ok, entries = True, True, 0
for mu in range(8):
    for nu in range(8):
        common = gup[nu].diff(x[mu]) + sum((Gam[nu][mu][lam] * gup[lam]
                                            for lam in range(8)), Z16)
        D_correct = common + Omega[mu] * gup[nu] - gup[nu] * Omega[mu]
        D_nb = common + Omega_nb[mu] * gup[nu] - gup[nu] * Omega_nb[mu]
        difference = Omega_nb[mu] - Omega[mu]
        correct_ok = correct_ok and matrix_is_zero(D_correct)
        identity_ok = identity_ok and matrix_is_zero(
            D_nb - (difference * gup[nu] - gup[nu] * difference))
        nonzero_entries = sum(1 for v in D_nb if not is_zero(v))
        if nonzero_entries:
            violation[(mu, nu)] = D_nb
            entries += nonzero_entries
```

For all 64 pairs (μ, ν): `common` is the part $\partial_\mu\gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda$ that both versions share; `D_correct` adds $[\Omega_\mu, \gamma^\nu]$ and `D_nb` adds $[\Omega^{nb}_\mu, \gamma^\nu]$. Three things are recorded: whether `D_correct` is zero, whether `D_nb` equals $[\Omega^{nb}_\mu - \Omega_\mu, \gamma^\nu]$, and how many entries of `D_nb` are not zero; the nonzero matrices are kept in the dictionary `violation`, and their nonzero entries are counted in `entries`.

```
check(correct_ok and record_passed(REPORT_WL_B4, "gamma_covariantly_constant"),
      "correct: D_mu gamma^nu = 0 for all 64 pairs",
      record=f"{REPORT_WL_B4}, check gamma_covariantly_constant")
check(identity_ok, "D^nb_mu gamma^nu = [Omega^nb_mu - Omega_mu, gamma^nu], all 64")
```

Two checks: the correct version vanishes for all 64 pairs (with the record's check), and the identity of Section 6.23 holds for all 64.

```
report("pairs (mu, nu) with D^nb_mu gamma^nu not zero", len(violation))
report("nonzero entries of these 16 x 16 matrices", entries)
say("violated pairs: " + ", ".join(f"({NAMES[m]},{NAMES[n]})"
                                   for m, n in sorted(violation)))
check(len(violation) == 15 and entries == 288,
      "notebook: D^nb_mu gamma^nu is not zero in 15 of 64 pairs (288 entries)")
```

Two RESULT lines print 15 pairs and 288 entries, one line lists the violated pairs, `(x0,x0), (x0,x3), ..., (x6,x7)`, and the check requires exactly 15 and 288, the numbers derived in Section 6.23.

```
failures = [(mu, a) for mu in range(8) for a in range(8) if not matrix_is_zero(
    Omega_nb[mu] * gamma[a] - gamma[a] * Omega_nb[mu]
    + sum((mixed[mu][a][b] * gamma[b] for b in range(8)), Z16))]
report("pairs (mu, a) where the defining property fails for Omega^nb", len(failures))
check(len(failures) > 0 and all(matrix_is_zero(
    Omega[mu] * gamma[a] - gamma[a] * Omega[mu]
    + sum((mixed[mu][a][b] * gamma[b] for b in range(8)), Z16))
    for mu in range(8) for a in range(8)),
      "the defining property holds for Omega and fails for Omega^nb",
      record=f"{REPORT_WL}, check S_rotates_gamma_with_omega")
```

`failures` lists the pairs (μ, a) in which the defining property $[\Omega, \gamma^a] = -\sum_b \omega_\mu{}^a{}_b \gamma^b$ fails for Ω^{nb}; a RESULT line prints their number, 15. The check requires at least one failure for Ω^{nb} and none for the correct Ω (all 64 pairs), with the WolframScript report's check. The cell prints three PASS lines, three RESULT lines and the list.

In [14], figure 5.

```
size = np.zeros((8, 8))
for (mu, nu), matrix in violation.items():
    size[mu, nu] = np.abs(sample_table(matrix)).max()  # the largest entry
```

For each violated pair, the largest absolute entry of $D^{nb}_\mu \gamma^\nu$ at the sample point.

```
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(size, cmap="Oranges", vmin=0.0)
for (mu, nu) in violation:
    dark = size[mu, nu] > 0.6 * size.max()  # white digits on dark squares
    ax.text(nu, mu, f"{size[mu, nu]:.2f}", ha="center", va="center", fontsize=7,
            color="white" if dark else "black")
ax.set_xticks(range(8), NAMES)
ax.set_yticks(range(8), NAMES)
ax.set_xlabel("upper index $\\nu$ of $\\gamma^\\nu$")
ax.set_ylabel("derivative index $\\mu$")
ax.set_title("Largest entry of $D^{nb}_\\mu\\gamma^\\nu$ at the sample point")
ax.grid(False)
fig.colorbar(image, ax=ax, shrink=0.8, label="largest absolute entry")
```

A heat map in shades of orange with the values written into the squares (white digits on the dark squares), the derivative index μ as rows and the upper index ν as columns, and a colour bar.

```
save_figure(fig, "constancy_violation",
            "Where the notebook's contraction breaks the covariant constancy of the "
            ...
            "connection all 64 matrices vanish exactly.")
```

The figure is saved; the cell prints its file name.

What Figure 06c.5 shows. Only the rows of the six warped directions have coloured squares; the rows $x4$ and $x0$ are white because $\Omega_{x4} = \Omega_{x0} = 0$ in both versions. Each 3-space row has two squares: $(x_i, x_i) = 0.50$, which is $a'_4 = \frac{1}{2}$, and $(x_i, x4) = 0.78$, which is $f_1 a'_4 = 1.556 \cdot \frac{1}{2}$. Each extra-time row has three: $(x_t, x_t) = 1.00$, the larger of $2a'_4 = 1$ and $H = 1$, and $(x_t, x4) = (x_t, x0) = 0.57$, which is $2f_5 a'_4 = f_5 H = 0.572$ at $z = \pi/4$ ($\tan z = 1$). All values are those derived in Section 6.23. With the correct connection the whole grid would be white.

In [15], why only $3H$ is right (Section 6.24).

```
sqrt_g = b4 ** sp.Rational(3, 2) * sp.cos(z)  # sqrt|det g| of the author's metric
divergence = sum(((sqrt_g * gup[mu]).diff(x[mu]) for mu in range(8)), Z16) / (2 * sqrt_g)
check(matrix_is_zero(divergence - slash) and not matrix_is_zero(divergence - slash_nb),
      "the divergence form equals the correct contraction, not the notebook's",
      record=f"{REPORT_WL_B4}, check gammaOmega_divergence_form")
```

The divergence form with $\sqrt{|g|} = b_4^{3/2} \cos z$; the check requires it to equal the correct contraction and NOT the mixed one (`not matrix_is_zero(...)`), and the WolframScript report with b_4 to record the divergence form as passed.

```
p = sp.Function("p")(x0)  # two arbitrary functions of x0
q = sp.Function("q")(x0)
c = sp.Symbol("c", real=True)  # the coefficient of gamma^(x0)
```

Two arbitrary functions $p(x_0)$, $q(x_0)$ and a real symbol c for the coefficient of $\gamma^{(x0)}$.

```
def A(u):
    """The hidden-direction operator A_c u = tan z du/dx0 + c u."""
    return sp.tan(z) * u.diff(x0) + c * u
```

The hidden-direction operator $A_c u = \tan z \, \partial_0 u + c\, u$.

```
leftover = sp.cos(z) * (p * A(q) + A(p) * q) - sp.diff(sp.sin(z) * p * q, x0)
check(is_zero(leftover - (2 * c - 6 * H) * sp.cos(z) * p * q),
      "cos z (p A_c q + (A_c p) q) - d0(sin z p q) = (2c - 6H) cos z p q")
```

`leftover` is $\cos z\,(p\,A_c q + (A_c p)\,q) - \partial_0(\sin z\, p\, q)$, and the check requires it to be $(2c - 6H)\cos z\, p\, q$ exactly, for every c and all functions p, q.

```
record_text = FORMULAS["hidden_direction_hermiticity"]
check(is_zero(leftover.subs(c, 3 * H)) and "Tan[z] d0 + 3 H" in record_text
      and "= d0 (Sin[z] p q)" in record_text,
      "c = 3H: A_c is antisymmetric for the weight cos z up to d0(sin z p q)",
      record=f"{THEORY_FILE}, formula hidden_direction_hermiticity")
```

For $c = 3H$ the leftover vanishes; and the record's text of `hidden_direction_hermiticity` must contain the operator `Tan[z] d0 + 3 H` and the boundary term `= d0 (Sin[z] p q)`.

```
check(is_zero(leftover.subs(c, 3 * H / 2) + 3 * H * sp.cos(z) * p * q),
      "c = 3H/2 (notebook): the leftover -3 H cos z p q remains")
```

For the mixed value $c = \frac{3}{2}H$ the leftover is $-3H\cos z\,p\,q$. The cell prints four PASS lines, two with records.

In [16], figure 6.

```
zz = np.linspace(0.0, np.pi / 2, 300)  # the hidden angle z from 0 to pi/2
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.axhline(0.0, color="gray", linewidth=0.6, zorder=1)  # the zero line, underneath
for value, style, width, label in ((3.0, "-", 2.5, "correct $c = 3H$"),
                                   (1.5, "--", 1.5, "notebook $c = 3H/2$"),
                                   (0.0, ":", 1.5, "no connection term, $c = 0$")):
    ax.plot(zz, (2 * value - 6) * np.cos(zz), style, linewidth=width, label=label,
            zorder=3)  # drawn above the zero line
ax.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax.set_ylabel("leftover coefficient $(2c/H - 6)\\cos z$")
ax.set_title("Only $c = 3H$ makes the hidden operator antisymmetric")
ax.legend(fontsize=8)
```

300 values of z from 0 to $\pi/2$. A thin gray zero line is drawn first (`zorder=1` puts it underneath), then the leftover coefficient $(2c/H - 6)\cos z$ for $c = 3H$ (thick solid), $c = \frac{3}{2}H$ (dashed) and $c = 0$ (dotted), above it (`zorder=3`).

```
save_figure(fig, "hermiticity_defect",
            "The term that spoils the antisymmetry of the hidden-direction operator "
            ...
            "term; the solid line lies on the zero line.")
```

The figure is saved; the cell prints its file name.

What Figure 06c.6 shows. The thick solid line of the correct value $c = 3H$ lies on the zero line for every z: nothing is left but the boundary term. The dashed line of the mixed value starts at -3 at the tip and rises to 0 at the patch end, and the dotted line of $c = 0$ starts at -6. Only the correct coefficient makes the hidden-direction operator antisymmetric for the volume weight $\cos z$.

In [17], the last checks.

```
cited = {
    REPORT_PY: ["clifford_relations", "gammas_real", "vielbein_postulate",
```

```
                "spin_connection_antisymmetric", "covariant_constancy_D_mu_gamma_nu",
                "gamma_mu_Omega_mu_equals_3H_gamma_x0", "divergence_of_sqrtg_gamma"],
    REPORT_WL: ["omega_components", "S_rotates_gamma_with_omega",
                "gamma_covariantly_constant", "gammaOmega_equals_3H_gamma_x0",
                "gammaOmega_divergence_form", "good_sector_hermiticity_curved"],
    REPORT_WL_B4: ["omega_components", "gamma_covariantly_constant",
                   "gammaOmega_total", "gammaOmega_divergence_form"],
}
count = sum(len(names) for names in cited.values())
check(all(record_passed(path, name) for path, names in cited.items()
          for name in names),
      f"the {count} cited record checks are recorded as passed")
```

The 17 cited checks, 7 of the sympy field-theory report and 6 of the WolframScript one (both of $b_4 = 1$) and 4 of the WolframScript report with b_4, each stating a correct result reproduced here; the check requires all of them to be recorded as passed.

```
expected = [f"06c_{k}_{name}.png" for k, name in enumerate([
    "pair_kinds", "milne_frame_and_spin_boost", "per_direction_comparison",
    "contraction_heat_maps", "constancy_violation", "hermiticity_defect"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected),
      "all six figure files exist")
all_checks_passed()
```

The six figure files must exist, and the last line is printed: ALL 27 CHECKS PASSED (notebook 06c).

6.29 The exact scope of the statement that gravity enters the field equations

The author asked that the field equations of both fields "always possess non-zero contributions from gravity (through the canonical spin-connection, unless we are in flat 4+4 spacetime)". This chapter has proved several exact statements, and they must be read together. The Revision record states the conclusion in its own correction (in `Revision/README.md`): the non-triviality tests [1] and [2] hold in a qualified sense. Here is that sense, statement by statement.

What is true in the diagonal vielbein. $\sum_\mu \gamma^\mu D_\mu\Psi - \sum_\mu \gamma^\mu \partial_\mu\Psi = \big(3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}\big)\Psi$, which is nonzero for every $H > 0$, every function a_4 and every field that is not zero, except at times where $|b_4'/b_4| = 4H$; for a constant b_4, as on the backgrounds $b_4 = 1$, it is $3H\gamma^{(x0)}\Psi$ and never zero for a nonzero field (Sections 6.8 and 6.10). The record's non-triviality checks are made at $b_4 = 1$; the exception for a changing b_4 is recorded by the check `gravity_term_square_b4` of `python-field-theory-b4.json`.

Where the term comes from. The Lagrangian of Chapter 7 contains the symmetric kinetic term $\frac{1}{2}\sum_\mu \big(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi\big)$. Insert $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$ (Section 6.7):

$$\tfrac{1}{2}\sum_\mu \big(\bar\Psi\gamma^\mu\partial_\mu\Psi - (\partial_\mu\bar\Psi)\gamma^\mu\Psi\big) + \tfrac{1}{2}\sum_\mu \bar\Psi\big(\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu\big)\Psi$$

(multiply out; the two Ω terms collect into one). By Fact 1 of Section 6.9 the last bracket vanishes for each μ separately. **In the author's metric and the diagonal vielbein the canonical**

spin connection drops out of the Lagrangian completely. The term $3H\gamma^{(x0)}$ comes back in the field equation from the volume factor and the vielbein: vary the connection-free term $\sqrt{|g|}\,\frac{1}{2}\sum_\mu(\bar\Psi\gamma^\mu\partial_\mu\Psi-(\partial_\mu\bar\Psi)\gamma^\mu\Psi)$ with respect to $\bar\Psi$ by the Euler-Lagrange rule of Chapter 7 (the derivative by $\bar\Psi$ minus $\sum_\mu\partial_\mu$ of the derivative by $\partial_\mu\bar\Psi$, the two treated as independent):

$$\tfrac{1}{2}\sqrt{|g|}\sum_\mu\gamma^\mu\partial_\mu\Psi-\sum_\mu\partial_\mu\Big(-\tfrac{1}{2}\sqrt{|g|}\,\gamma^\mu\Psi\Big)=\sqrt{|g|}\sum_\mu\gamma^\mu\partial_\mu\Psi+\tfrac{1}{2}\sum_\mu\partial_\mu(\sqrt{|g|}\,\gamma^\mu)\Psi$$

(the product rule on the second term). Divided by $\sqrt{|g|}$ the last term is the divergence form of Section 6.9, $\sum_\mu\gamma^\mu\Omega_\mu\Psi=\big(3H\gamma^{(x0)}+\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}\big)\Psi$. The record calls it the **half-density term** (volume and vielbein divergence). Chapter 7 carries out the variation for both statistics with every sign. Records (at $b_4=1$; Fact 1 and the divergence form hold for every b_4, Section 6.9): both scope reports, check `connection_free_lagrangian_same_equations`; `wolfram-field-theory.json`, checks `L_spin_connection_drops_out_G` and `L_spin_connection_drops_out_C`.

What depends on choices. The value of $\gamma^\mu\Omega_\mu$ belongs to the diagonal vielbein: in the frame boosted with the rate $\beta=6H$ the contraction vanishes identically for a constant b_4 (Section 6.16). It also belongs to the field variable Ψ: for $\chi=\sin^{1/2}(z)\,b_4^{3/4}\,\Psi$ the term is absent for $U=0$ and becomes the coupling $\lambda S[\chi]/(b_4^{3/2}\sin z)$ for $U=\frac{\lambda}{2}S^2$ (Section 6.18). It contains no trace of the deflation a_4 (Section 6.8).

What does not depend on any choice.

- The spin connection vanishes in no frame: its curvature is the Riemann curvature, $F_{\mu\nu}=\frac{1}{2}R_{ab\mu\nu}S^{ab}$, and $R^{x0}{}_{x0}=-6H^2\neq0$ for every $H>0$ and every b_4 (Section 6.17). The metric is curved for every member of the family.
- The vielbein enters every derivative term of the field equation through the factors $e^{-a_4}s^{-1}$, $e^{a_4}b_4^{-1/2}s^{-1}$ and $\tan z$; this is where the inflation of 3-space and the exponential deflation of the extra times act on the fields (Section 6.10).
- The spin connection reaches the energy-momentum tensor of the fields: for a configuration that depends on the time only, the off-diagonal kinetic term $K^{x4}{}_{x1}$ has the connection part $\frac{1}{2}e^{a_4}s\,H\,\bar\Phi\gamma^{(x4)}\gamma^{(x1)}\gamma^{(x0)}\Phi$, which is not identically zero (both scope reports, check `spin_connection_in_the_energy_momentum_tensor`, at $b_4=1$; Chapter 9 treats the energy-momentum tensor).

In one sentence. Gravity enters the field equations of both fields in every frame and every choice of field variables, through the vielbein factors and through a spin connection whose curvature cannot be removed; the particular matrix $3H\gamma^{(x0)}+\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ ($3H\gamma^{(x0)}$ on the backgrounds $b_4=1$) is the form this takes in the diagonal vielbein with the variable Ψ, and it is a property of these choices, not of the physics alone. Chapter 8 returns to this scope and adds the question of well-posedness.

statement	status	where it is verified
the connection drops out of the symmetric Lagrangian; the field equations are those of the connection-free symmetric Lagrangian, with the half-density term	PROVED	both scope reports, check `connection_free_lagrangian_same_equations`; `wolfram-field-theory.json`, checks `L_spin_connection_drops_out_G`, `L_spin_connection_drops_out_C`
the value of $\gamma^\mu\Omega_\mu$ is frame-dependent and removable by a rescaling	PROVED	Sections 6.16 and 6.18 (Notebook 06b; scope reports at $b_4 = 1$)
the spin connection vanishes in no frame; the metric is never flat	PROVED	Section 6.17
the connection enters the energy-momentum tensor	PROVED	both scope reports, check `spin_connection_in_the_energy_momentum_tensor`

6.30 What we proved, what we computed, what we assumed

PROVED (exact; derived in this chapter line by line, confirmed by exact checks of the notebooks named, and, where stated, by the Revision record):

- The diagonal vielbein $f = (\cot z,\ e^{a_4}s\ (\times 3),\ 1,\ e^{-a_4}b_4^{1/2}s\ (\times 3))$ reproduces the author's metric; $\sqrt{|\det g|} = b_4^{3/2}\cos z$ depends on the time only through b_4; the curved gammas obey $\gamma^\mu\gamma^\nu + \gamma^\nu\gamma^\mu = 2g^{\mu\nu}$; every Λe with $\Lambda^T\eta\Lambda = \eta$ is another vielbein (Section 6.2; Notebook 06a, In [7]; `metric-b4.json`, keys `metricDiagonal`, `sqrtAbsDetG`; at $b_4 = 1$ `field-theory.json`, formulas `metric`, `vielbein_diagonal`, and `python-field-theory.json`, check `sqrt_det_g_equals_cos_z`).
- The 25 independent nonzero Christoffel symbols (Section 6.3; Notebook 06a, In [9]; formula `christoffel_nonzero` of both records, check `christoffel_count`).
- The vielbein postulate has exactly one solution, the canonical spin connection, and its lowered form is antisymmetric; for the author's metric it has exactly 12 independent nonzero components, each a rate (a_4', $a_4' - b_4'/(2b_4)$ or H) times a nonvanishing factor (Sections 6.4 and 6.6; Notebook 06a, In [11]; formula `omega_nonzero` of both records; checks `vielbein_postulate`, `spin_connection_antisymmetric`, `omega_components`, and at $b_4 = 1$ `Omega_vanishes_iff_a4prime_and_H_vanish`).
- $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$ is the unique traceless solution of the defining property; it makes the curved gammas covariantly constant; $\Omega_{x0} = \Omega_{x4} = 0$ (Section 6.7; Notebook 06a, In [13]; checks `S_rotates_gamma_with_omega`, `covariant_constancy_D_mu_gamma_nu`, `Omega_x0_and_Omega_x4_vanish`).
- $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ in the diagonal vielbein, for every $H > 0$, a_4 and b_4, and $3H\gamma^{(x0)}$ for a constant b_4: the time-direction terms $\pm\frac{1}{2}a_4'$ of the three inflating

and the three deflating directions cancel, the extra times leave $\frac{3}{4}b_4'/b_4$, the six hidden terms $\frac{1}{2}H$ add; equivalently, the divergence form (Sections 6.8 and 6.9; Notebook 06a, In [15] and In [18]; formulas `gammaOmega_per_direction`, `gammaOmega_total`; checks `gammaOmega_per_direction`, `gammaOmega_total`, `gammaOmega_divergence_form` with b_4, and at $b_4 = 1$ `gammaOmega_x4_terms_cancel`, `gamma_mu_Omega_mu_equals_3H_gamma_x0`, `gamma_Omega_equals_3H_gamma0`).

- The Dirac operator written out, all sixteen components; the term $M\Psi$ with $MM = (9H^2 - \frac{9}{16}(b_4'/b_4)^2)\,1$ vanishes only for $\Psi = 0$ unless $|b_4'/b_4| = 4H$, and for a constant b_4 never, for both fields (Section 6.10; Notebook 06a, In [20]; checks `Dirac_operator_explicit_G`, `Dirac_operator_explicit_C`, `gravity_term_square_b4`, and at $b_4 = 1$ `nontriviality_1_dirac16complex`, `nontriviality_2_dirac16complex00`).

- The negative control (at $b_4 = 1$): with inflating extra times the contraction is $3a_4'\gamma^{(x4)} + 3H\gamma^{(x0)}$; the cancellation of the a_4' terms is caused by the deflation (Section 6.10; Notebook 06a, In [22]; lead report, check `negative_control_inflating_extra_times`).

- Local spin covariance $\Omega_\mu' = R\Omega_\mu R^{-1} - (\partial_\mu R)R^{-1}$, $\gamma'^\mu = R\gamma^\mu R^{-1}$, $F' = RFR^{-1}$ (Sections 6.15 and 6.17; Notebook 06b, In [12] and In [15]).

- In the frame boosted with $b = \beta x_4 + b_0$, $\gamma'^\mu\Omega_\mu' = \frac{6H-\beta}{2}(\cosh b\,\gamma^{(x0)} - \sinh b\,\gamma^{(x4)}) + \frac{3}{4}\frac{b_4'}{b_4}(\cosh b\,\gamma^{(x4)} - \sinh b\,\gamma^{(x0)})$, zero for $\beta = 6H$ and a constant b_4, while $\Omega_\mu' \neq 0$ for $x1, \ldots, x7$ (Section 6.16; Notebook 06b, In [8] and In [10]; at $b_4 = 1$ both scope reports, checks `boosted_frame_gammaOmega_formula`, `boosted_frame_gammaOmega_vanishes`).

- The Ricci tensor (all components by hand in Section 3.25): with b_4 ten nonzero mixed components, two of them off the diagonal and proportional to b_4', and $R = 6a_4'^2 - 3a_4'b_4'/b_4 + 3b_4''/b_4 - 42H^2$; at $b_4 = 1$, $R = 6(a_4'^2 - 7H^2)$. By hand in Section 6.17, $R^{x0}{}_{x0} = -6H^2$ for every b_4 and $R^{x4}{}_{x4} = 3(a_4'^2 + a_4'') + 3(\alpha_t^2 + \alpha_t')$, which is $6a_4'^2$ at $b_4 = 1$: the metric is never flat, and the contraction is blind to the deflation while the metric is not; $F_{\mu\nu} = \frac{1}{2}R_{ab\mu\nu}S^{ab}$ in all 28 planes; $F'_{x0x1} \neq 0$ in the boosted frame (Section 6.17, whose table names the records that prove the same; Notebook 06b, In [13] and In [15]).

- The rescaling $\Psi = \sin^{-1/2}(z)\,b_4^{-3/4}\,\chi$ removes the term; with $U = \frac{\lambda}{2}S^2$ it becomes $\lambda S[\chi]/(b_4^{3/2}\sin z)$ (Section 6.18; Notebook 06b, In [17]; at $b_4 = 1$ checks `rescaling_removes_the_connection_term`, `rescaled_equation_quadratic_potential`).

- The mixed contraction equals $\Omega^{ss} - \Omega^{tt}$: it keeps 6, reverses 6 and deletes 16 of the 28 parts; it agrees with the correct one in the polar plane and deletes the whole connection in the Milne wedge (Section 6.23; Notebook 06c, In [4], In [6], In [8]).

- Only $c = 3H$ makes the hidden-direction operator antisymmetric for the weight $\cos z$ (times the constant $b_4^{3/2}$ along $x0$), up to the boundary term $[\sin z\, p\, q]$ (Section 6.24; Notebook 06c, In [15]; formula `hidden_direction_hermiticity`; check `good_sector_hermiticity_curved`).

- In the diagonal vielbein the canonical connection drops out of the symmetric Lagrangian, and $\gamma^\mu\Omega_\mu$ is the half-density term of its field equation (Section 6.29; check `connection_free_lagrangian_same_equations`, at $b_4 = 1$).

COMPUTED (by the notebooks):

- The central differences of Notebook 06a, In [19], reproduce the constant $3H$ to 1.9×10^{-9}, the size $18h^2$ expected for the step $h = 10^{-5}$, and $\frac{3}{4}b_4'/b_4$ for the illustrative $b_4 = 1 + \frac{1}{2}\sin x_4$ to 2.5×10^{-11}.
- The figures at the sample points: in Notebook 06a $H = 1$, $z = \pi/4$, $a_4 = a_4' = \frac{1}{2}$, $b_4 = 1$, $b_4' = \frac{1}{2}$, for example $\omega_{x1\,x1x4} = 0.78$, $\omega_{x5\,x5x0} = -0.57$, $\omega_{x5\,x4x5} = -0.14$; in Notebook 06b the same with $b_4' = 0$, the largest spinor curvature $\|F_{\mu\nu}\| = 3.633$ (in units of H^2; In [16]) with 27 of 28 planes curved there.
- Exact counts made by the notebooks themselves, not stated in the Revision record: 13 nonzero components of the boosted spin connection (Notebook 06b, In [7]); 78 nonzero Riemann components with $\mu < \nu$ (Notebook 06b, In [13]); for the mixed contraction, $\frac{3H}{2}\gamma^{(x0)} + \frac{3}{2}(a_4' - \frac{b_4'}{2b_4})\gamma^{(x4)}$, 15 violated pairs with 288 nonzero entries, and 15 pairs in which its defining property fails (Notebook 06c, In [10] and In [13]). These describe a contraction that the record does not use.

ASSUMED: the author's metric with b_4 and the roles of his coordinates (the input of the theory); the illustrative histories $a_4 = x_4$, $b_4 = 1$ with $H = 1$ in figures 2 and 9 of Notebook 06a (the prescribed background of Chapter 3) and $b_4 = 1 + \frac{1}{2}\sin x_4$ in figure 8 (no result depends on them); two standard theorems quoted without proof: Jacobi's formula for the derivative of a determinant (Section 6.15) and the general identity $F_{\mu\nu} = \frac{1}{2}R_{ab\mu\nu}S^{ab}$, which is however checked exactly for the author's metric (Section 6.17); and the record's statement about the author's notebook contraction (Section 6.23), which this book takes from `Revision/SPEC.md`.

HYPOTHESIS and OPEN: this chapter states no hypothesis. The boundary condition at the patch end $z = \pi/2$, needed for the antisymmetry of the hidden-direction operator, is not fixed here (Section 6.24); Chapter 14 assumes the Z_2 mirror. Nothing in this chapter concerns the creation of universes or matter and antimatter.

6.31 Exercises

Exercise 1 (the sphere). The unit sphere has the metric $ds^2 = d\theta^2 + \sin^2\theta\, d\varphi^2$ $(0 < \theta < \pi)$, the frame metric $\eta = \mathrm{diag}(1, 1)$, the vielbein factors $f_\theta = 1$, $f_\varphi = \sin\theta$, and the gammas $\gamma^0 = \sigma_1$, $\gamma^1 = \sigma_2$. Compute the Christoffel symbols, the spin connection, Ω_μ, $\sum_\mu \gamma^\mu\Omega_\mu$ (check it with the divergence form) and $F_{\theta\varphi}$. Is the sphere flat?

Answer. Case (c) of Section 6.3: $\Gamma^\theta{}_{\varphi\varphi} = -f_\varphi\partial_\theta f_\varphi/f_\theta^2 = -\sin\theta\cos\theta$; case (b): $\Gamma^\varphi{}_{\varphi\theta} = \partial_\theta \ln\sin\theta = \cot\theta$; all others vanish. By Section 6.6 the only nonzero component is $\omega_{\varphi\,10} = \eta_{11}\,\partial_\theta f_\varphi/f_\theta = \cos\theta$, so $\omega_{\varphi\,01} = -\cos\theta$. With $S^{01} = \frac{1}{2}\sigma_1\sigma_2 = \frac{i}{2}\sigma_3$: $\Omega_\varphi = -\cos\theta\, S^{01} = -\frac{i}{2}\cos\theta\,\sigma_3$ and $\Omega_\theta = 0$. With $\gamma^\varphi = \sigma_2/\sin\theta$: $\sum_\mu \gamma^\mu\Omega_\mu = \frac{\sigma_2}{\sin\theta}\big(-\frac{i}{2}\cos\theta\,\sigma_3\big) = -\frac{i\cot\theta}{2}\sigma_2\sigma_3 = -\frac{i\cot\theta}{2}\,i\sigma_1 = \frac{\cot\theta}{2}\sigma_1$. Divergence form with $\sqrt{|g|} = \sin\theta$: $\frac{1}{2\sin\theta}(\partial_\theta(\sin\theta\,\sigma_1) + \partial_\varphi(\sin\theta\cdot\sigma_2/\sin\theta)) = \frac{\cos\theta}{2\sin\theta}\sigma_1$, the same. Finally $F_{\theta\varphi} = \partial_\theta\Omega_\varphi - \partial_\varphi\Omega_\theta + [\Omega_\theta, \Omega_\varphi] = \partial_\theta\big(-\frac{i}{2}\cos\theta\,\sigma_3\big) = \frac{i}{2}\sin\theta\,\sigma_3 \neq 0$. The sphere is curved: unlike the polar plane of Section 6.5, here no change of frame can remove the spin connection (Section 6.17). (Check with the theorem of Section 6.17: $R^\theta{}_{\varphi\theta\varphi} = \sin^2\theta$ for the unit sphere, so $R_{01\theta\varphi} = \eta_{00}f_\theta R^\theta{}_{\varphi\theta\varphi}/f_\varphi = \sin\theta$ and $\frac{1}{2}(R_{01\theta\varphi}S^{01} + R_{10\theta\varphi}S^{10}) = \sin\theta\, S^{01} = \frac{i}{2}\sin\theta\,\sigma_3$.)

Exercise 2 (an extra time by hand). Compute $\omega_{x6\,ab}$ for the author's metric with the formula of Section 6.6 and then $\gamma^{x6}\Omega_{x6}$, without looking at Section 6.8. Why is the coefficient of $\gamma^{(x4)}$ negative?

Answer. Take b_4 constant, so that $f_6 = e^{-a_4} b_4^{1/2} s$ has $\partial_4 f_6 = -a_4' f_6$ (for a changing b_4 replace $-a_4'$ by α_t throughout); and $\partial_0 f_6 = H \cot z \, f_6$. With $\omega_{\mu\,\mu b} = \eta_{\mu\mu} \partial_b f_\mu / f_b$ and $\eta_{66} = -1$: $\omega_{x6\,x6\,x4} = (-1)(-a_4' f_6)/1 = a_4' f_6$, hence $\omega_{x6\,x4\,x6} = -a_4' f_6$; and $\omega_{x6\,x6\,x0} = (-1) H \cot z \, f_6 / \cot z = -H f_6$. So $\Omega_{x6} = -f_6 (a_4' S^{x4x6} + H S^{x6x0})$ and $\gamma^{x6} \Omega_{x6} = \frac{\gamma^{(x6)}}{f_6}(-f_6)(\frac{a_4'}{2}\gamma^{(x4)}\gamma^{(x6)} + \frac{H}{2}\gamma^{(x6)}\gamma^{(x0)}) = -\frac{a_4'}{2}\gamma^{(x6)}\gamma^{(x4)}\gamma^{(x6)} - \frac{H}{2}\gamma^{(x6)}\gamma^{(x6)}\gamma^{(x0)}$. Since $\gamma^{(x6)}\gamma^{(x4)}\gamma^{(x6)} = -\gamma^{(x4)}(\gamma^{(x6)})^2 = \gamma^{(x4)}$ and $(\gamma^{(x6)})^2 = -1$, $\gamma^{x6}\Omega_{x6} = -\frac{a_4'}{2}\gamma^{(x4)} + \frac{H}{2}\gamma^{(x0)}$. The coefficient of $\gamma^{(x4)}$ is negative because the extra time shrinks ($\partial_4 f_6 = -a_4' f_6$) while a 3-space direction grows; this is the sign that cancels the 3-space terms.

Exercise 3 (a general deflation rate). Take $b_4 = 1$ and replace the extra-time factors by $e^{-k a_4} s$ with a constant k (the author's metric has $k = 1$, the negative control $k = -1$). Use the divergence form to find the coefficient of $\gamma^{(x4)}$ in $\sum_\mu \gamma^\mu \Omega_\mu$. For which k does it vanish?

Answer. With $f_4 = 1$ the product of the other seven factors is the whole volume factor: $\sqrt{|g|} = e^{3a_4} s^3 \cdot e^{-3k a_4} s^3 \cdot \cot z = e^{3(1-k)a_4} \cos z$ ($s^6 \cot z = \sin z \cot z = \cos z$). The time contribution of the divergence form is $\frac{1}{2\sqrt{|g|}} \partial_4 \sqrt{|g|}\, \gamma^{(x4)} = \frac{3(1-k)a_4' e^{3(1-k)a_4} \cos z}{2 e^{3(1-k)a_4} \cos z} \gamma^{(x4)} = \frac{3}{2}(1-k) a_4' \gamma^{(x4)}$ (the chain rule). For $k = 1$ (the author's deflation) it is 0; for $k = -1$ it is $3a_4'$, the negative control of Section 6.10. It vanishes only for $k = 1$, exactly when the 7-volume does not change with time. (The hidden term is $3H$ for every k, because $\prod_{c \neq x0} f_c = e^{3(1-k)a_4} \sin z$ and the factor $e^{3(1-k)a_4}$ cancels in the division.)

Exercise 4 (the boosted contraction in numbers). Take $H = 1$, $b_0 = 0$ and the rate $\beta = 3$. Give the coefficients of $\gamma^{(x0)}$ and $\gamma^{(x4)}$ in $\gamma'^\mu \Omega'_\mu$ at $x_4 = 0$ and at $x_4 = \frac{1}{3}$. Which rate β gives $-3\gamma^{(x0)}$ at $x_4 = 0$?

Answer. By Section 6.16, for a constant b_4, the coefficients are $\frac{6-\beta}{2}\cosh b$ and $-\frac{6-\beta}{2}\sinh b$ with $b = \beta x_4$. Here $\frac{6-3}{2} = 1.5$. At $x_4 = 0$: $b = 0$, coefficients 1.5 and 0. At $x_4 = \frac{1}{3}$: $b = 1$, $\cosh 1 = 1.5431$, $\sinh 1 = 1.1752$, so the coefficients are $1.5 \cdot 1.5431 = 2.3146$ and $-1.5 \cdot 1.1752 = -1.7628$. At $x_4 = 0$ the coefficient of $\gamma^{(x0)}$ is $\frac{6-\beta}{2}$, which equals -3 for $\beta = 12$. A frame can thus give the contraction any multiple of $\gamma^{(x0)}$ at one instant; the value is not a property of the metric.

Exercise 5 (rescaling with a general power). Take b_4 constant and let $\Psi = \sin^\alpha(z)\, \chi$ with a constant α in the diagonal vielbein. Show that $\sum_\mu \gamma^\mu D_\mu \Psi = \sin^\alpha z \, (\sum_\mu \gamma^\mu \partial_\mu \chi + 3H(2\alpha+1)\gamma^{(x0)}\chi)$. Which α removes the term, and which leaves it unchanged?

Answer. By the product rule, and because only x_0 appears in $\sin^\alpha z$,

$$\sum_\mu \gamma^\mu D_\mu (\sin^\alpha z\, \chi) = \sin^\alpha z \sum_\mu \gamma^\mu \partial_\mu \chi + (\gamma^{x0} \partial_0 \sin^\alpha z + 3H \sin^\alpha z\, \gamma^{(x0)})\chi.$$

With $\gamma^{x0} = \tan z\, \gamma^{(x0)}$ and $\partial_0 \sin^\alpha z = 6H\alpha \sin^{\alpha-1} z \cos z$ (the power rule and the chain rule),

$$\tan z\, \partial_0 \sin^\alpha z = \frac{\sin z}{\cos z}\, 6H\alpha \sin^{\alpha-1} z \cos z = 6H\alpha \sin^\alpha z.$$

The bracket is $(6H\alpha + 3H)\sin^\alpha z\, \gamma^{(x0)} = 3H(2\alpha+1)\sin^\alpha z\, \gamma^{(x0)}$, which gives the formula. $\alpha = -\frac{1}{2}$ removes the term (Section 6.18), and $\alpha = 0$ leaves it unchanged. This is the factor $3H(2\alpha+1)$ that the record's exact solution family contains (`field-theory.json`, formula `exact_solutions`).

Exercise 6 (mixed components). Using $\omega_\mu{}^b{}_a = -\eta_{aa}\eta_{bb}\,\omega_\mu{}^a{}_b$ (Section 6.4), show that the mixed components of a time-time pair are antisymmetric and those of a boost pair symmetric. Then

compute Ω^{nb}_{x2} and Ω^{nb}_{x7} for the author's metric.

Answer. For a time-time pair $\eta_{aa}\eta_{bb} = (-1)(-1) = +1$, so $\omega_\mu{}^b{}_a = -\omega_\mu{}^a{}_b$: antisymmetric. For a boost pair $\eta_{aa}\eta_{bb} = -1$, so $\omega_\mu{}^b{}_a = +\omega_\mu{}^a{}_b$: symmetric, and in $\frac{1}{2}\sum_{a,b}\omega_\mu{}^a{}_b S^{ab}$ its two terms cancel because $S^{ba} = -S^{ab}$. For $x2$ the pair $(x2, x4)$ is a boost pair (deleted) and $(x2, x0)$ a space-space pair (kept): $\Omega^{nb}_{x2} = e^{a_4} s\, H\, S^{x2x0}$. For $x7$ the pair $(x4, x7)$ is time-time (sign reversed) and $(x0, x7)$ a boost pair (deleted): $\Omega^{nb}_{x7} = -\omega_{x7\,x4x7} S^{x4x7} = +e^{-a_4} b_4^{1/2} s\,(a_4' - \frac{b_4'}{2b_4}) S^{x4x7}$.

Exercise 7 (the hidden operator in numbers). Take $p = q = 1$ (constant functions) and integrate the identity of Section 6.24 over the whole patch $0 \le x_0 \le \pi/(12H)$. Compare the left side with the boundary term for $c = 3H$ and for $c = \frac{3}{2}H$.

Answer. With $p = q = 1$, $A_c 1 = c$, so the left side is $\int \cos z \cdot 2c\, dx_0 = 2c \int_0^{\pi/(12H)} \cos(6Hx_0)\, dx_0 = 2c \cdot \frac{1}{6H} = \frac{c}{3H}$ (the 7-volume integral of Chapter 3; the constant factor $b_4^{3/2}$ is left out on both sides). The boundary term is $[\sin z\, p\, q]_{z=0}^{z=\pi/2} = 1 - 0 = 1$. For $c = 3H$ the left side is 1, equal to the boundary term: nothing else is left. For $c = \frac{3}{2}H$ it is $\frac{1}{2}$, and the difference $\frac{1}{2} - 1 = -\frac{1}{2}$ is the integral of the leftover $-3H\cos z$, $-3H \cdot \frac{1}{6H} = -\frac{1}{2}$, which no boundary term can absorb.

Exercise 8 (which pairs break). Without a computer, explain why $D^{nb}_{x1}\gamma^{x2} = 0$ although $\Omega^{nb}_{x1} \neq \Omega_{x1}$, and why the row $x0$ of figure 5 of Notebook 06c is white.

Answer. By Section 6.23, $D^{nb}_{x1}\gamma^{x2} = [\Omega^{nb}_{x1} - \Omega_{x1}, \gamma^{x2}] = -f_1 a_4' [S^{x1x4}, \gamma^{(x2)}]/f_2$. By the vector rule $[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a - \eta^{ac}\gamma^b$, which vanishes when c is neither a nor b; here $c = x2$ is neither $x1$ nor $x4$, so the commutator is zero. The row $x0$ is white because $\Omega_{x0} = 0$ (Section 6.6) and also $\Omega^{nb}_{x0} = 0$ (it is built from the same components, all zero for $\mu = x0$), so their difference vanishes and $D^{nb}_{x0}\gamma^\nu = D_{x0}\gamma^\nu = 0$ for every ν.

7 The two fields and their Lagrangians; Grassmann numbers; Euler-Lagrange equations

This chapter writes down the laws that the two fields of the book obey in the author's primordial gravitational field. A law of this kind is not guessed equation by equation. One writes down a single function, the **Lagrangian**, and a single principle, the **principle of stationary action**, and the field equations follow from them by a fixed recipe, the **Euler-Lagrange equations**. The chapter teaches the recipe from zero, then the strange numbers (Grassmann numbers) of which the field dirac16complex is made, then the Lagrangian of the two fields, its field equations written out in the author's metric, and finally what happens to real fields and why charge conjugation is a matrix.

7.1 What this chapter does

The two fields. The book studies two fields that live on the author's eight-dimensional space-time. Both attach to every point of space-time a column of 16 complex numbers $\Psi = (\Psi_1, \ldots, \Psi_{16})$, and both change under the turnings of the eight directions in the same way, as a **spinor** of the group Pin(4,4) (Chapter 5). They differ in one respect only, the kind of numbers their 16 components are:

- **dirac16complex** has components that **anticommute**: for two of its components $\Psi_A \Psi_B = -\Psi_B \Psi_A$. Ordinary numbers cannot do this; the numbers that can are called **Grassmann**

numbers, and Sections 7.10 to 7.13 build them from nothing. The Revision record defines the components of dirac16complex as Grassmann numbers (Revision/SPEC.md, section 3); this field is later turned into a quantum field (Chapter 10).

- **dirac16complex00** has ordinary commuting complex components. It is a classical field, the 16-component analogue in eight dimensions of the four-component wave function that Dirac wrote down in 1928; it is not quantised.

The coordinates. As everywhere in this book, the eight coordinates carry the author's names $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates; x_4 is the time; x_5, x_6, x_7 are three **extra times**, time-like directions which **deflate exponentially** as 3-space inflates; x_0 is the hidden space direction. With $z = 6Hx_0$, $0 < z < \pi/2$, the author's constant $H > 0$ and a function $a_4(x_4)$ of the time, the author's metric is (Chapter 3)

$$ds^2 = e^{2a_4} \sin^{1/3} z \, (dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2 - e^{-2a_4} \sin^{1/3} z \, (dx_5^2 + dx_6^2 + dx_7^2) + \cot^2 z \, dx_0^2.$$

3-space has the scale factor $e^{a_4} \sin^{1/6} z$, the three extra times the scale factor $e^{-a_4} \sin^{1/6} z$: when a_4 grows, 3-space inflates and the extra times deflate. The signature is (4,4): x_0, x_1, x_2, x_3 are space-like (positive entries), x_4, x_5, x_6, x_7 time-like (negative entries).

What the chapter derives, in order.

- Sections 7.2 to 7.5: the action, the principle of stationary action, and the Euler-Lagrange equation, from one-variable calculus only; what changes for fields; and the first sign that the extra times are unusual: a wave along an extra time grows instead of oscillating. Notebook 07d computes every step (30 checks, 8 figures).
- Sections 7.10 to 7.13: Grassmann numbers from zero, the rules for their products, conjugation and derivatives, which bilinear forms survive for real columns, the powers of the scalar density $S = \bar{\Psi}\Psi$ (defined in Section 7.12; $S^{17} = 0$), and field equations with Grassmann numbers. Notebook 07a builds a Grassmann algebra in a few lines of Python (38 checks, 7 figures).
- Sections 7.18 to 7.23: the geometry the fields live in, the common Lagrangian of the two fields, its reality, the total divergence, why the spin connection drops out of the Lagrangian but not out of the field equation, the Euler-Lagrange equations of both fields, their explicit, block and evolution forms in the author's metric, non-triviality [1] and [2] with their exact scope, the six exact senses in which the field equations are self-consistent, and an exact family of solutions. Notebook 07b derives all of it exactly and compares it with the Revision record (47 checks, 6 figures).
- Sections 7.28 and 7.29: real fields; the author's Majorana-type Lagrangian, which is empty for real Grassmann fields (the negative control of the Revision record); and why **charge conjugation is a matrix**: for a real field, plain complex conjugation changes nothing. Notebook 07c computes the charge-conjugation matrices $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$ and the mass reversal by the real matrix Γ (23 checks, 4 figures).
- Section 7.34 lists what was proved, computed and assumed; Section 7.35 has exercises with complete answers.

The notebooks. The four notebooks appear in the order d, a, b, c, because Notebook 07d derives the recipe that the other three use. Each is complete in itself and may be run in any order. Before each notebook the book prints the complete instructions to run it, then the complete text of the notebook with its outputs and figures, then a section that explains every line of its code.

Statuses. Every statement carries one of five labels. PROVED: exact, derived here line by line and, where the Revision record contains it, verified there by a named check of a named report. COMPUTED: a number produced by a program, with its uncertainty. ASSUMED: used, not derived. HYPOTHESIS: a proposal that is being investigated and is not claimed. OPEN: a question that is not answered. The Revision reports cited in this chapter are `Revision/theory/reports/python-field-theory.json` (the sympy verifier, 70 of 70 checks pass), `Revision/theory/reports/wolfram-field-theory.json` (the Wolfram verifier, 84 of 84 checks pass), `Revision/theory/reports/python-scope.json` (14 of 14) and `Revision/theory/reports/wolfram-scope.json` (15 of 15) for the scope of the statements, the formula record `Revision/theory/field-theory.json`, and `Revision/lead_checks/reports/charge-conjugation-and-u1.json` (12 of 12) for charge conjugation. These counts are those that the reports state at the time of writing; each notebook reads the checks it relies on from the reports themselves and stops if one is missing or does not pass.

7.2 Paths, the action and the first variation

We begin with the simplest possible system, a particle that moves along a line, and learn the principle there. Everything carries over to fields in Section 7.5.

A path. A **path** (or motion) is a function $q(t)$ that gives the position q of the particle at every time t between a start time 0 and an end time T. Its **velocity** is $\dot{q} = dq/dt$ and its **acceleration** $\ddot{q} = d^2q/dt^2$ (a dot over a letter is the derivative with respect to the time t).

The Lagrangian. A **Lagrangian** is a function $L(q, \dot{q})$ of the position and the velocity. For the **oscillator**, a particle of mass 1 on a spring, it is the kinetic energy minus the potential energy:

$$L(q, \dot{q}) = \tfrac{1}{2}\dot{q}^2 - \tfrac{1}{2}\omega^2 q^2,$$

where $\omega > 0$ is a fixed number, the **angular frequency** of the spring. Here L is a function of two ordinary variables; we call them q and $\dot{q}$ because along a path we shall put the position and the velocity of the path into them. The two **partial derivatives** of L (Chapter 2) are $\partial L/\partial q = -\omega^2 q$ (the velocity held fixed) and $\partial L/\partial \dot{q} = \dot{q}$ (the position held fixed).

The action. The **action** of a path is one number for the whole path:

$$S[q] = \int_0^T L\big(q(t), \dot{q}(t)\big)\, dt.$$

At every time we put the position and the velocity of the path into L and add up (integrate) the values over the time interval. The square brackets in $S[q]$ say that S depends on the whole function q, not on one number; a rule that takes a function and gives a number is called a **functional**.

The principle of stationary action (ASSUMED: it is the starting principle of classical mechanics and of classical field theory, not derived from anything simpler). Among all paths with the same end values $q(0) = q_A$ and $q(T) = q_B$, the particle follows one whose action does not change to first

order when the path is changed a little. To make "changed a little" precise, take a fixed **shape** $\xi(t)$ with $\xi(0) = \xi(T) = 0$ (so that the changed path keeps the end values) and a small number ϵ, and form the **varied path** $q + \epsilon\xi$. Its action $S[q + \epsilon\xi]$ is an ordinary function of the one number ϵ. The **first variation** is its slope at $\epsilon = 0$:

$$S_1 = \frac{d}{d\epsilon} S[q + \epsilon\xi]\Big|_{\epsilon=0}.$$

A path is **stationary** if $S_1 = 0$ for every shape ξ; the principle says that the true motion is stationary.

For the oscillator the action is a polynomial in ϵ. Put the varied path into the Lagrangian:

$$L(q + \epsilon\xi, \dot{q} + \epsilon\dot{\xi}) = \tfrac{1}{2}(\dot{q} + \epsilon\dot{\xi})^2 - \tfrac{1}{2}\omega^2(q + \epsilon\xi)^2$$

(the velocity of the varied path is $\dot{q} + \epsilon\dot{\xi}$, because the derivative of a sum is the sum of the derivatives and the constant ϵ comes out of the derivative);

$$= \tfrac{1}{2}\dot{q}^2 + \epsilon\,\dot{q}\dot{\xi} + \tfrac{1}{2}\epsilon^2\dot{\xi}^2 - \tfrac{1}{2}\omega^2 q^2 - \epsilon\,\omega^2 q\xi - \tfrac{1}{2}\epsilon^2\omega^2\xi^2$$

(the square of a sum, $(a+b)^2 = a^2 + 2ab + b^2$, applied twice);

$$= \Big[\tfrac{1}{2}\dot{q}^2 - \tfrac{1}{2}\omega^2 q^2\Big] + \epsilon\Big[\dot{q}\dot{\xi} - \omega^2 q\xi\Big] + \epsilon^2\Big[\tfrac{1}{2}\dot{\xi}^2 - \tfrac{1}{2}\omega^2\xi^2\Big]$$

(the terms collected by the power of ϵ). Integrating from 0 to T term by term:

$$S[q + \epsilon\xi] = S_0 + S_1\,\epsilon + S_2\,\epsilon^2, \qquad S_1 = \int_0^T (\dot{q}\dot{\xi} - \omega^2 q\xi)\,dt, \qquad S_2 = \int_0^T (\tfrac{1}{2}\dot{\xi}^2 - \tfrac{1}{2}\omega^2\xi^2)\,dt,$$

with $S_0 = S[q]$. The coefficient S_1 of ϵ is exactly the first variation (the slope of a polynomial at 0 is its coefficient of the first power), and the coefficient S_2 contains only the shape ξ, not the path: it is the same for every path (PROVED; Notebook 07d checks both facts exactly, In [3]).

The example of Notebook 07d. Take $\omega = 1$, $T = 1$, $q(0) = 0$, $q(1) = 1$. The **true path** is $q(t) = \sin t / \sin 1$: it has $\ddot{q} = -\sin t/\sin 1 = -q$ (the second derivative of $\sin t$ is $-\sin t$), which is Newton's law $\ddot{q} = -\omega^2 q$ for the spring, and $q(0) = 0$, $q(1) = \sin 1/\sin 1 = 1$. Two wrong paths with the same end values are the **line** $q = t$ and the **parabola** $q = t^2$. The shapes are $\xi_n(t) = \sin(n\pi t)$ for $n = 1, 2, 3$; each vanishes at $t = 0$ and at $t = 1$, because $\sin 0 = \sin(n\pi) = 0$.

One first variation by hand. For the line, $\dot{q} = 1$, and for $\xi_1 = \sin\pi t$, $\dot{\xi}_1 = \pi\cos\pi t$:

$$S_1 = \int_0^1 (\pi\cos\pi t - t\sin\pi t)\,dt$$

(the formula for S_1 with $\omega = 1$);

$$= \big[\sin\pi t\big]_0^1 - \int_0^1 t\sin\pi t\,dt = 0 - \int_0^1 t\sin\pi t\,dt$$

(an antiderivative of $\pi\cos\pi t$ is $\sin\pi t$, which is 0 at both ends). The last integral is done by **integration by parts** (Section 7.3 derives it; here it is used with $f = t$ and $g = -\cos(\pi t)/\pi$): $\int_0^1 t\sin\pi t\,dt = \big[-t\cos(\pi t)/\pi\big]_0^1 + \int_0^1 \cos(\pi t)/\pi\,dt = 1/\pi + 0 = 1/\pi$, since $\cos\pi = -1$ and $\sin\pi = \sin 0 = 0$. So

$$S_1 = -\frac{1}{\pi} = -0.318310,$$

the value that Notebook 07d prints for the line and $n = 1$ (In [2]). The line is not stationary: its action changes to first order. For the true path the notebook finds $S_1 = 0$ exactly for all three shapes, and for the two wrong paths $S_1 \neq 0$ for all three (In [2] and In [3]; PROVED by exact integration with sympy).

A minimum, or only stationary? The coefficient S_2 decides what kind of stationary point the true path is. Take the shape $\xi_1 = \sin(\pi t/T)$ on an interval of length T. Then $\dot\xi_1 = (\pi/T)\cos(\pi t/T)$ (the chain rule), and

$$S_2 = \int_0^T \Big(\frac{\pi^2}{2T^2}\cos^2\frac{\pi t}{T} - \frac{\omega^2}{2}\sin^2\frac{\pi t}{T}\Big)dt$$

(the definition of S_2 with this shape);

$$= \frac{\pi^2}{2T^2}\cdot\frac{T}{2} - \frac{\omega^2}{2}\cdot\frac{T}{2}$$

(each of the two integrals is $T/2$: the double-angle formula $\cos 2x = 2\cos^2 x - 1$ gives $\cos^2(\pi t/T) = \frac{1}{2}(1 + \cos(2\pi t/T))$; the interval from 0 to T is one full period of $\cos(2\pi t/T)$, and indeed $\int_0^T \cos(2\pi t/T)\,dt = \big[\frac{T}{2\pi}\sin(2\pi t/T)\big]_0^T = 0$ because $\sin 2\pi = \sin 0 = 0$; so $\int_0^T \cos^2(\pi t/T)\,dt = \frac{1}{2}T = T/2$, and because $\sin^2 = 1 - \cos^2$, $\int_0^T \sin^2(\pi t/T)\,dt = T - T/2 = T/2$);

$$= \frac{T}{4}\Big(\frac{\pi^2}{T^2} - \omega^2\Big)$$

(the common factor $T/4$ taken out). For $T < \pi/\omega$ the bracket is positive, so $S_2 > 0$ and the action of $q + \epsilon\xi_1$ is $S_0 + S_2\epsilon^2$ with a positive S_2: the true path is a **minimum** along this shape. For $T > \pi/\omega$ the coefficient is negative: the true path is still stationary ($S_1 = 0$), but along ξ_1 the action is a maximum, while along faster shapes (ξ_n with large n) it is still a minimum; such a point is called a **saddle**. This is why the principle is called the principle of STATIONARY action and not of least action (PROVED; Notebook 07d checks the formula with T as a symbol and its sign at $T = 1$ and $T = 4$, In [4], and draws Figure 07d.1). With $T = 1$, $\omega = 1$ the formula gives $S_2 = (\pi^2 - 1)/4 = 2.217401$ for ξ_1; the notebook prints this value and $S_2 = \pi^2 - 1/4 = 9.619604$ for ξ_2 and $9\pi^2/4 - 1/4 = 21.956610$ for ξ_3 (In [3]).

7.3 From the principle to the Euler-Lagrange equation

Computing the action of every path is not how one finds the true path. Instead the principle is turned into an equation that the true path must satisfy at every time. This takes three lines and one lemma.

Integration by parts. The **product rule** of differentiation says $\frac{d}{dt}(fg) = \dot{f}g + f\dot{g}$ for two functions $f(t)$, $g(t)$. Integrate both sides from 0 to T. The left side is the integral of a derivative, so by the fundamental theorem of calculus it is the difference of the end values, which we write $[fg]_0^T = f(T)g(T) - f(0)g(0)$:

$$[fg]_0^T = \int_0^T \dot{f}g\,dt + \int_0^T f\dot{g}\,dt \qquad \Longrightarrow \qquad \int_0^T f\dot{g}\,dt = [fg]_0^T - \int_0^T \dot{f}g\,dt.$$

The second form moves the derivative from g to f at the price of a minus sign and the **boundary term** $[fg]_0^T$.

Line 1. Differentiate the action of the varied path with respect to ϵ:

$$\frac{d}{d\epsilon} S[q+\epsilon\xi]\Big|_{\epsilon=0} = \int_0^T \Big(\frac{\partial L}{\partial q}\,\xi + \frac{\partial L}{\partial \dot{q}}\,\dot{\xi}\Big)dt.$$

Rule: the derivative may be taken inside the integral (true for the smooth functions of this book on a finite interval; a standard theorem of calculus, ASSUMED here), and inside, L depends on ϵ through its first argument $q+\epsilon\xi$, whose derivative with respect to ϵ is ξ, and through its second argument $\dot{q}+\epsilon\dot{\xi}$, whose derivative is $\dot{\xi}$; the chain rule for a function of two variables (Chapter 2) adds the two contributions.

Line 2. Integrate the second term by parts with $f = \partial L/\partial\dot{q}$ and $g = \xi$:

$$= \int_0^T \frac{\partial L}{\partial q}\,\xi\,dt + \Big[\frac{\partial L}{\partial \dot{q}}\,\xi\Big]_0^T - \int_0^T \frac{d}{dt}\Big(\frac{\partial L}{\partial \dot{q}}\Big)\xi\,dt.$$

Line 3. The boundary term vanishes, because $\xi(0) = \xi(T) = 0$; collect the two integrals:

$$= \int_0^T E(t)\,\xi(t)\,dt, \qquad E = \frac{\partial L}{\partial q} - \frac{d}{dt}\frac{\partial L}{\partial \dot{q}}.$$

The function $E(t)$ is the **Euler-Lagrange expression** of the path. In $\partial L/\partial q$ and $\partial L/\partial\dot{q}$ the path's own position and velocity are put in after differentiating, and then $\frac{d}{dt}$ is an ordinary time derivative along the path.

The fundamental lemma. If a continuous function $E(t)$ has $\int_0^T E\xi\,dt = 0$ for every shape ξ that vanishes at the ends, then $E(t) = 0$ at every time. *Proof.* Suppose $E(t_0) > 0$ at some time t_0 between 0 and T. Because E is continuous, it stays positive on a small interval $t_0 - w < t < t_0 + w$ (if it were zero or negative arbitrarily close to t_0, its values there would not approach $E(t_0) > 0$). Take a **bump**: a smooth shape that is positive on that interval and zero outside, for example $\xi = \cos^2(\pi(t-t_0)/(2w))$ inside and 0 outside. Then $E\xi$ is positive inside the interval and zero outside, so $\int_0^T E\xi\,dt > 0$, which contradicts the assumption. The case $E(t_0) < 0$ is the same with every sign reversed. So $E(t_0) = 0$ for every t_0. □

The Euler-Lagrange equation. Putting the three lines and the lemma together: the true path satisfies

$$\frac{\partial L}{\partial q} - \frac{d}{dt}\frac{\partial L}{\partial \dot{q}} = 0$$

at every time (PROVED, from the principle of stationary action). For the oscillator, $\partial L/\partial q = -\omega^2 q$ and $\partial L/\partial \dot q = \dot q$, so $E = -\omega^2 q - \ddot q$ and the equation is $\ddot q = -\omega^2 q$: Newton's law for the spring. The true path of our example satisfies it; the line has $\ddot q = 0$ and so $E = -t$; the parabola has $\ddot q = 2$ and so $E = -2 - t^2$. Notebook 07d computes these three functions with a function `euler_lagrange` of its own and with sympy's `euler_equations` (In [5]) and checks that the first variation of every wrong path equals $\int_0^1 E\xi_n\,dt$ (for example, for the line and $n = 1$: $\int_0^1 (-t)\sin\pi t\,dt = -1/\pi$, the value of Section 7.2).

The boundary term, seen. If the shape does not vanish at the end, the boundary term of Line 2 stays. For the oscillator $\partial L/\partial \dot q = \dot q$, so $S_1 = \int_0^1 E\xi\,dt + [\dot q\,\xi]_0^1$. Take the line and the shape $\xi = t$, which is 0 at $t = 0$ but 1 at $t = 1$. By the definition, $S_1 = \int_0^1 (\dot q\dot\xi - q\xi)\,dt = \int_0^1 (1 - t^2)\,dt = 1 - \frac13 = \frac23$. The integral part is $\int_0^1 (-t)\,t\,dt = -\frac13$, and the boundary term is $\dot q(1)\xi(1) - \dot q(0)\xi(0) = 1\cdot 1 - 1\cdot 0 = 1$. Indeed $-\frac13 + 1 = \frac23$ (Notebook 07d prints these three numbers in In [5]; it does the same for the parabola: $\frac34 = -\frac54 + 2$).

The lemma made visible with narrow bumps. The ratio $\int E\xi\,dt / \int \xi\,dt$ is an **average** of E over the bump. For a bump of half-width w centred at t_0 it tends to $E(t_0)$ as w shrinks. For the parabola, whose $E(t) = -2 - t^2$ is a polynomial of degree 2, the error of the average can be computed exactly. Write $t = t_0 + s$:

$$E(t_0 + s) = E(t_0) + E'(t_0)\,s + \tfrac12 E''(t_0)\,s^2$$

(Taylor's formula, Chapter 2, which is exact for a polynomial of degree 2; here $E' = -2t_0$ and $E'' = -2$). The bump is symmetric around t_0, so the average of s over it is 0 (the contributions of s and $-s$ cancel), and

$$\text{average of } E - E(t_0) = \tfrac12 E'' \cdot (\text{average of } s^2) = -(\text{average of } s^2).$$

With $s = wu$, where u runs from -1 to 1, the average of s^2 is w^2 times the number

$$c = \frac{\int_{-1}^1 u^2\cos^2(\pi u/2)\,du}{\int_{-1}^1 \cos^2(\pi u/2)\,du}.$$

Using $\cos^2(\pi u/2) = \frac12(1 + \cos\pi u)$ (the double-angle formula), the denominator is $\int_{-1}^1 \frac12(1 + \cos\pi u)\,du = 1$ (the cosine integrates to $[\sin\pi u/\pi]_{-1}^1 = 0$). The numerator is

$$\tfrac12\int_{-1}^1 u^2\,du + \tfrac12\int_{-1}^1 u^2\cos\pi u\,du = \tfrac13 + \tfrac12\cdot\Big(-\frac{4}{\pi^2}\Big),$$

where the last integral follows from two integrations by parts: $\int_{-1}^1 u^2\cos\pi u\,du = -\frac{2}{\pi}\int_{-1}^1 u\sin\pi u\,du$ and $\int_{-1}^1 u\sin\pi u\,du = \frac{2}{\pi}$. So

$$\text{average of } E - E(t_0) = -\Big(\frac13 - \frac{2}{\pi^2}\Big)w^2 = -0.130691\,w^2,$$

independent of t_0 and exactly proportional to w^2: halving w divides the error by 4 (PROVED here). Of these steps only the replacement of $\frac12 E''$ by -1 used the parabola's $E'' = -2$: for any polynomial

E of degree 2 the same steps give the error $\frac{1}{2}E''(\frac{1}{3}-\frac{2}{\pi^2})w^2$. For the parabola and $w=0.2$ the error is $-0.130691\times 0.04=-5.228\times 10^{-3}$, the number that Notebook 07d computes numerically for all three centres $t_0=0.3,0.5,0.7$, with the ratio 4.0000 between successive half-widths (In [7], Figure 07d.3; COMPUTED with the trapezoidal rule on 200001 points, whose own error is far below the printed digits).

The action on a grid. A computer stores a path as its values $q_0,q_1,\dots,q_N$ at the **grid** times $t_n=nh$, $h=T/N$, and replaces the integral by a sum. With the velocity of each step written as the difference quotient $(q_{n+1}-q_n)/h$ and the potential energy averaged over the two ends of the step,

$$S_N=\sum_{n=0}^{N-1} h\Big[\frac{1}{2}\Big(\frac{q_{n+1}-q_n}{h}\Big)^2-\frac{\omega^2}{4}(q_n^2+q_{n+1}^2)\Big].$$

An inner value q_n $(0<n<N)$ appears in two terms of the sum: the step that ends at t_n (number $n-1$) and the step that starts there (number n). Differentiating with respect to q_n:

$$\frac{\partial S_N}{\partial q_n}=h\Big[\frac{q_n-q_{n-1}}{h^2}-\frac{q_{n+1}-q_n}{h^2}-\frac{\omega^2}{2}q_n-\frac{\omega^2}{2}q_n\Big]$$

(the chain rule on the two squares: $\frac{d}{dq_n}\frac{1}{2}\big(\frac{q_n-q_{n-1}}{h}\big)^2=\frac{q_n-q_{n-1}}{h}\cdot\frac{1}{h}$ and $\frac{d}{dq_n}\frac{1}{2}\big(\frac{q_{n+1}-q_n}{h}\big)^2=\frac{q_{n+1}-q_n}{h}\cdot(-\frac{1}{h})$; and $\frac{d}{dq_n}\frac{\omega^2}{4}q_n^2=\frac{\omega^2}{2}q_n$, once from each step);

$$=h\Big[-\frac{q_{n+1}-2q_n+q_{n-1}}{h^2}-\omega^2 q_n\Big]$$

(the terms collected). The bracket is $E=-\ddot q-\omega^2 q$ with $\ddot q$ replaced by the **second difference quotient** $(q_{n+1}-2q_n+q_{n-1})/h^2$. So "the action on the grid is stationary" means "the grid version of the Euler-Lagrange equation holds at every inner point" (PROVED; Notebook 07d checks it with sympy for $N=5$, In [8]). The second difference quotient differs from $\ddot q$ by a term of size h^2: adding the Taylor series $q(t\pm h)=q\pm h\dot q+\frac{1}{2}h^2\ddot q\pm\frac{1}{6}h^3q^{(3)}+\frac{1}{24}h^4q^{(4)}+\dots$ (Chapter 2) the odd powers cancel and $\big(q(t+h)-2q(t)+q(t-h)\big)/h^2=\ddot q+\frac{1}{12}h^2q^{(4)}+\dots$. Accordingly, the path that makes the grid action stationary approaches the true path with an error proportional to h^2: Notebook 07d solves the grid equations for $N=4,8,\dots,256$ and measures the largest errors 4.089×10^{-4} $(N=4)$ down to 1.008×10^{-7} $(N=256)$ and the observed orders 2.007, 1.978, 2.001, 2.000, 2.000, 2.000 (In [9], Figure 07d.4; COMPUTED).

7.4 Energy, total derivatives and first-order Lagrangians

The energy. For a Lagrangian $L(q,\dot q)$ that does not contain the time explicitly, the **energy** is

$$H=\dot q\,\frac{\partial L}{\partial \dot q}-L.$$

For the oscillator $H=\dot q\cdot\dot q-(\frac{1}{2}\dot q^2-\frac{1}{2}\omega^2q^2)=\frac{1}{2}\dot q^2+\frac{1}{2}\omega^2q^2$, kinetic plus potential energy. Its time derivative along any path, line by line:

$$\frac{dH}{dt} = \ddot{q}\,\dot{q} + \omega^2 q\,\dot{q}$$

(the chain rule: the derivative of $\frac{1}{2}\dot{q}^2$ is $\dot{q}\ddot{q}$, that of $\frac{1}{2}\omega^2 q^2$ is $\omega^2 q\dot{q}$);

$$= \dot{q}\,(\ddot{q} + \omega^2 q) = -\dot{q}\,E$$

(the common factor $\dot{q}$ taken out; then $E = -\ddot{q} - \omega^2 q$). So the energy is constant on every solution of $E = 0$, and it changes on a path where $E \neq 0$ and $\dot{q} \neq 0$ (PROVED; Notebook 07d checks the identity for a general path, In [10]). On the true path, $\dot{q} = \cos t/\sin 1$, so $H = \frac{1}{2}(\cos^2 t + \sin^2 t)/\sin^2 1 = 1/(2\sin^2 1) = 0.706141$, constant; on the line $H = \frac{1}{2} + \frac{1}{2}t^2$ with $dH/dt = t$; on the parabola $H = 2t^2 + \frac{1}{2}t^4$ with $dH/dt = 4t + 2t^3 = 2t(t^2 + 2)$ (all printed by In [10]; Figure 07d.5).

A total derivative changes no equation. Let $F(q, t)$ be any smooth function and add its **total derivative** along the path to the Lagrangian:

$$L' = L + \frac{d}{dt}F(q(t), t) = L + \frac{\partial F}{\partial q}\,\dot{q} + \frac{\partial F}{\partial t}$$

(the chain rule for a function of two variables, both of which change with t). Two proofs that L' has the same Euler-Lagrange equation as L:

- *By the action.* The action changes by $\int_0^T \frac{dF}{dt}dt = F(q(T), T) - F(q(0), 0)$ (the fundamental theorem of calculus), which is the same number for every path with the given end values. A change that is the same for every path cannot change which path is stationary.
- *By the expression.* For $G = \frac{dF}{dt} = F_q\dot{q} + F_t$ (subscripts denote partial derivatives): $\partial G/\partial q = F_{qq}\dot{q} + F_{qt}$ and $\partial G/\partial\dot{q} = F_q$, whose time derivative along the path is $F_{qq}\dot{q} + F_{qt}$. The difference is 0, because the order of two partial derivatives does not matter ($F_{tq} = F_{qt}$, Chapter 2). So G contributes nothing to E.

In particular a Lagrangian that is ONLY a total derivative, such as $\frac{d}{dt}(q^2) = 2q\dot{q}$, has the Euler-Lagrange expression $2\dot{q} - \frac{d}{dt}(2q) = 0$ for every path: it gives no equation at all (PROVED; Notebook 07d checks both statements with $F = tq^3$ and finds that the action changes by exactly 1 for each of the three paths, In [11]). This innocent fact decides, in Section 7.28, why the author's real Majorana-type Lagrangian describes nothing for anticommuting fields.

First-order Lagrangians. The Lagrangians of the two fields of this book contain the derivatives only to the first power. Two small examples show what such Lagrangians do.

Example A: two real variables. $L_A = \frac{1}{2}(q_0\dot{q}_1 - q_1\dot{q}_0) - \frac{1}{2}\omega(q_0^2 + q_1^2)$. With two variables there are two Euler-Lagrange expressions, one for each. For q_0: $\partial L_A/\partial q_0 = \frac{1}{2}\dot{q}_1 - \omega q_0$ and $\partial L_A/\partial\dot{q}_0 = -\frac{1}{2}q_1$, whose time derivative is $-\frac{1}{2}\dot{q}_1$; so

$$E_0 = \tfrac{1}{2}\dot{q}_1 - \omega q_0 - \big(-\tfrac{1}{2}\dot{q}_1\big) = \dot{q}_1 - \omega q_0.$$

For q_1: $\partial L_A/\partial q_1 = -\frac{1}{2}\dot{q}_0 - \omega q_1$ and $\partial L_A/\partial\dot{q}_1 = \frac{1}{2}q_0$, so $E_1 = -\frac{1}{2}\dot{q}_0 - \omega q_1 - \frac{1}{2}\dot{q}_0 = -\dot{q}_0 - \omega q_1$. The equations $\dot{q}_1 = \omega q_0$, $\dot{q}_0 = -\omega q_1$ are of FIRST order: the starting values alone fix the motion

(no starting velocity is needed). For $q_0(0) = 1$, $q_1(0) = 0$ the solution is $q_0 = \cos\omega t$, $q_1 = \sin\omega t$: indeed $\dot{q}_1 = \omega\cos\omega t = \omega q_0$ and $\dot{q}_0 = -\omega\sin\omega t = -\omega q_1$. The point (q_0, q_1) turns on a circle, counterclockwise.

Example B: one complex variable. Let $\psi = (q_0 + iq_1)/\sqrt{2}$, with the **complex conjugate** $\psi^* = (q_0 - iq_1)/\sqrt{2}$, and

$$L_B = \frac{i}{2}(\psi^*\dot{\psi} - \dot{\psi}^*\psi) - \omega\,\psi^*\psi.$$

In real variables: $\psi^*\psi = \frac{1}{2}(q_0 - iq_1)(q_0 + iq_1) = \frac{1}{2}(q_0^2 + q_1^2)$, and $\psi^*\dot{\psi} - \dot{\psi}^*\psi = \frac{1}{2}\big[(q_0 - iq_1)(\dot{q}_0 + i\dot{q}_1) - (\dot{q}_0 - i\dot{q}_1)(q_0 + iq_1)\big] = \frac{1}{2}\big[2iq_0\dot{q}_1 - 2iq_1\dot{q}_0\big] = i(q_0\dot{q}_1 - q_1\dot{q}_0)$ (multiply out; the terms $q_0\dot{q}_0$ and $q_1\dot{q}_1$ cancel). So

$$L_B = \tfrac{i}{2}\cdot i(q_0\dot{q}_1 - q_1\dot{q}_0) - \tfrac{1}{2}\omega(q_0^2 + q_1^2) = \tfrac{1}{2}(q_1\dot{q}_0 - q_0\dot{q}_1) - \tfrac{1}{2}\omega(q_0^2 + q_1^2)\,:$$

Example A with the sign of the kinetic term reversed (because $i\cdot i = -1$). It is a real Lagrangian, although it is written with complex numbers.

Varying ψ and ψ^* as if they were independent. The pair (q_0, q_1) and the pair (ψ, ψ^*) determine each other ($q_0 = (\psi + \psi^*)/\sqrt{2}$, $q_1 = (\psi - \psi^*)/(i\sqrt{2})$): the change of variables is linear and invertible. So one may use ψ and ψ^* as the two variables of the Lagrangian, and the Euler-Lagrange expressions combine in the same way. By the chain rule, $\partial/\partial\psi^* = \frac{\partial q_0}{\partial\psi^*}\partial/\partial q_0 + \frac{\partial q_1}{\partial\psi^*}\partial/\partial q_1 = \frac{1}{\sqrt{2}}(\partial/\partial q_0 + i\,\partial/\partial q_1)$, and therefore

$$E_{\psi^*} = \frac{E_{q_0} + iE_{q_1}}{\sqrt{2}}.$$

Varying ψ^* directly: $\partial L_B/\partial\psi^* = \frac{i}{2}\dot{\psi} - \omega\psi$ and $\partial L_B/\partial\dot{\psi}^* = -\frac{i}{2}\psi$, whose time derivative is $-\frac{i}{2}\dot{\psi}$; so

$$E_{\psi^*} = \tfrac{i}{2}\dot{\psi} - \omega\psi + \tfrac{i}{2}\dot{\psi} = i\dot{\psi} - \omega\psi.$$

The field equation $i\dot{\psi} = \omega\psi$ has the solution $\psi(t) = e^{-i\omega t}\psi(0)$ (the derivative of $e^{-i\omega t}$ is $-i\omega e^{-i\omega t}$, and $i\cdot(-i\omega) = \omega$): the point turns clockwise, with $|\psi|^2 = \frac{1}{2}(q_0^2 + q_1^2)$ constant (PROVED; Notebook 07d checks every statement of the two examples with ω as a symbol, In [12], and draws them in Figure 07d.6). The Lagrangians of the two fields of this book have exactly this structure: ψ becomes the column Ψ of 16 components, ψ^* becomes the row $\bar{\Psi} = \Psi^\dagger C$ (Section 7.19), and ω becomes a matrix of derivatives.

7.5 Fields: one derivative term per coordinate, and what an extra time does

A field is a function of several coordinates, for example $\phi(x_1, x_4)$ of a 3-space coordinate x_1 and the time x_4. Its Lagrangian is a function $\mathcal{L}(\phi, \partial_1\phi, \partial_4\phi)$ of the value and of the partial derivatives $\partial_\mu\phi = \partial\phi/\partial x_\mu$, called a **Lagrangian density**, and the action is its integral over all coordinates of a region. The shapes $\xi(x_1, x_4)$ now vanish on the boundary of the region. Line 1 of Section 7.3 now has one term $\frac{\partial\mathcal{L}}{\partial(\partial_\mu\phi)}\partial_\mu\xi$ for each coordinate; in Line 2 each of them is integrated by parts along its own coordinate (the other coordinates held fixed), and each boundary term vanishes because ξ

vanishes on the boundary; the fundamental lemma works with bumps in several variables in the same way. The result is the Euler-Lagrange expression of a field,

$$E = \frac{\partial \mathcal{L}}{\partial \phi} - \sum_{\mu} \frac{\partial}{\partial x_\mu} \frac{\partial \mathcal{L}}{\partial (\partial_\mu \phi)},$$

with one derivative term for each coordinate, and the field equation $E = 0$ at every point (PROVED, by the same three lines and the lemma).

A field of one space direction and time. In the flat version of the author's signature, $\eta_{11} = +1$ and $\eta_{44} = -1$, the Lagrangian density of a field of mass m is

$$\mathcal{L} = -\tfrac{1}{2}(\eta^{11}(\partial_1\phi)^2 + \eta^{44}(\partial_4\phi)^2) - \tfrac{1}{2}m^2\phi^2 = \tfrac{1}{2}(\partial_4\phi)^2 - \tfrac{1}{2}(\partial_1\phi)^2 - \tfrac{1}{2}m^2\phi^2$$

(the numbers η^{aa} are equal to η_{aa}, ± 1): kinetic energy $\frac{1}{2}(\partial_4\phi)^2$ minus a gradient energy along x_1 and a potential energy. The Euler-Lagrange expression, line by line:

$$E = \frac{\partial \mathcal{L}}{\partial \phi} - \partial_1 \frac{\partial \mathcal{L}}{\partial (\partial_1 \phi)} - \partial_4 \frac{\partial \mathcal{L}}{\partial (\partial_4 \phi)}$$

(the formula with its two coordinates);

$$= -m^2\phi - \partial_1(-\partial_1\phi) - \partial_4(\partial_4\phi)$$

(the three partial derivatives of $\mathcal{L}$, each with the other two arguments held fixed);

$$= \partial_1^2\phi - \partial_4^2\phi - m^2\phi$$

(two minus signs give a plus). So $\partial_4^2\phi = \partial_1^2\phi - m^2\phi$: the wave equation with a mass, called the **Klein-Gordon equation**. A **plane wave** $\phi = \cos(kx_1)\cos(\omega x_4)$, with **wave number** k and angular frequency ω, has $\partial_1^2\phi = -k^2\phi$ and $\partial_4^2\phi = -\omega^2\phi$, so $E = (-k^2 + \omega^2 - m^2)\phi$, which vanishes exactly when

$$\omega^2 = m^2 + k^2,$$

the **dispersion relation**: the wave oscillates in time for every k (PROVED; Notebook 07d, In [14]).

A field of one extra time and time. Replace x_1 by the extra time x_5. The author's signature makes x_5 time-like, $\eta_{55} = -1$, and the same recipe gives

$$\mathcal{L} = \tfrac{1}{2}(\partial_4\phi)^2 + \tfrac{1}{2}(\partial_5\phi)^2 - \tfrac{1}{2}m^2\phi^2, \qquad E = -\partial_5^2\phi - \partial_4^2\phi - m^2\phi.$$

The only change is the sign of the x_5 term. Try $\phi = \cos(kx_5)\,P(x_4)$. Then $\partial_5^2\phi = -k^2\phi$ and $E = \cos(kx_5)(k^2P - P'' - m^2P)$, so the field equation becomes

$$P'' = (k^2 - m^2)\,P.$$

For $k < m$ the solutions oscillate. For $k > m$ they are $\cosh(\kappa x_4)$ and $\sinh(\kappa x_4)$ with

$$\kappa = \sqrt{k^2 - m^2}$$

(the second derivative of $\cosh(\kappa x_4)$ is $\kappa^2 \cosh(\kappa x_4)$): they GROW exponentially, and the **growth rate** κ becomes as large as we like when k grows. A wave along a space direction raises ω^2 above m^2; a wave along an extra time lowers it to $m^2 - k^2$, below zero when $k > m$ (PROVED; Notebook 07d, In [15], Figures 07d.7 and 07d.8).

The same growth for the 16-component fields. The fields of this book obey a first-order equation $\gamma^{(a)}\partial_a \Psi = m\Psi$ in flat space (Section 7.21). Multiplying it by the same operator once more turns it into a second-order equation for every component, because of the Clifford relation of the author's gamma matrices (Chapter 4), $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}$. For constant numbers k_a it gives, line by line,

$$\Big(\sum_a k_a \gamma^{(a)}\Big)^2 = \sum_{a,b} k_a k_b\, \gamma^{(a)}\gamma^{(b)} = \tfrac{1}{2}\sum_{a,b} k_a k_b (\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}) = \sum_a \eta^{aa} k_a^2\, I_{16}$$

(multiply out the square of a sum; then use that the double sum does not change when the names a and b are exchanged, so it equals half the sum of itself and its exchanged form; then the Clifford relation, which leaves only $a = b$). Written out: $(k_1^2 + k_2^2 + k_3^2 - k_4^2 - k_5^2 - k_6^2 - k_7^2 + k_0^2)\, I_{16}$, where I_{16} is the 16×16 identity matrix (PROVED; Notebook 07d checks it exactly with the author's gammas, In [15]; it reproduces `Revision/theory/reports/python-field-theory.json`, check `clifford_relations`). The Revision record finds for a plane wave of the 16-component field with an extra-time momentum $k_5 = K$ the growth rate $\sqrt{K^2 - k_1^2 - k_2^2 - k_3^2 - k_0^2 - m^2}$, which has no upper bound as K grows (`Revision/theory/reports/python-scope.json`, check `extra_time_growth_rates_unbounded`); for a momentum along x_5 alone it is $\sqrt{K^2 - m^2}$, the κ above (Notebook 07d reads the formula from the record and checks this, In [15]). What this unbounded growth means for the field equations of the book (no well-posed initial-value problem for data that depend on the extra times) is the subject of Chapter 8.

7.6 Example: Notebook 07d derives the Euler-Lagrange equation

Notebook 07d puts Sections 7.2 to 7.5 to work. It computes exactly, with sympy, the action of the true path and of the two wrong paths and of their variations, and shows that only the true path has a zero first variation; it checks the formula for S_2 and the change from minimum to saddle at $T = \pi/\omega$; it computes the Euler-Lagrange expression with a function of its own and with sympy's, checks the integration by parts with and without the boundary term, the fundamental lemma with narrow bumps, the action on a grid and its second-order convergence, the energy, the total derivative, the two first-order examples, and the field equations along a space direction and along an extra time, where it reproduces the growth rate of the Revision record. It draws eight figures and ends with the line ALL 30 CHECKS PASSED (notebook 07d).

7.7 How to run Notebook 07d

Step 1. What this notebook does and what it needs.

Notebook 07d (The action principle and the Euler-Lagrange equation) is the file `Revision/textbook/notebooks/07d_action_principle.ipynb` of the repository Dirac_claude. It shows, for a reader who knows one-variable calculus, where the Euler-Lagrange equation comes from: it computes exactly the action of the true path and of two wrong paths of an oscillator and of their small variations, shows that only the true path makes the action stationary, checks the integration by parts, the fundamental lemma with narrow bumps, the action on a grid of times and its second-order convergence, the conserved energy, that a total derivative changes no equation, the first-order Lagrangians of a real pair and of one complex variable, and the field equation of a field of a space direction or of an extra time with time; it reproduces the growth rate of the Revision record for an extra-time momentum, and draws eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 07d_action_principle.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 07d_action_principle.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 07d_action_principle.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/07d.captions.json`
- `Revision/textbook/figures/07d_1_action_versus_epsilon.png`
- `Revision/textbook/figures/07d_2_paths_and_residuals.png`
- `Revision/textbook/figures/07d_3_bump_lemma.png`
- `Revision/textbook/figures/07d_4_action_on_a_grid.png`
- `Revision/textbook/figures/07d_5_energy_along_paths.png`
- `Revision/textbook/figures/07d_6_first_order_rotation.png`
- `Revision/textbook/figures/07d_7_space_and_extra_time.png`
- `Revision/textbook/figures/07d_8_dispersion.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 8 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 07d)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/07d_action_principle.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 07d_action_principle.ipynb
python -m nbconvert --execute --inplace 07d_action_principle.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/theory/reports/...": the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory/reports Revision/algebra
```

7.8 Notebook 07d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/07d_action_principle.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Notebooks 07a, 07b and 07c USE the Euler-Lagrange equation: they take a Lagrangian
and compute the expression $\partial L/\partial q - \frac{d}{dt}\,\partial L /
\partial\dot q$. This notebook shows where that expression comes from, with nothing
more than one-variable calculus and a computer that checks every step. It

- computes, exactly, the **action** (the time integral of the Lagrangian) of the
  true motion of an oscillator and of two wrong motions with the same end points,
  and of every motion changed by a small amount $\epsilon$ times a fixed shape;
  only the true motion has an action whose slope in $\epsilon$ is zero;
- shows that the true motion makes the action a minimum for short times and only
  stationary (a saddle) for long times;
- checks the integration by parts that turns the slope into the integral of the
  **Euler-Lagrange expression** times the shape, and shows the boundary term that
  appears when the shape does not vanish at the ends;
- illustrates the **fundamental lemma** with narrow bumps: a function whose
  integral against every bump is zero must be zero;
- writes the action on a grid of times, as a computer would, shows that its
  derivatives are the Euler-Lagrange expression on the grid, solves the grid
  equations and measures the error, which falls like the square of the step;
- checks that the energy is constant on the true motion and only there;
- checks that adding a total derivative to a Lagrangian changes no equation;
- derives the equations of two **first-order** Lagrangians, of the kind the fields
  of this book have: two real variables, and one complex variable $\psi$ varied
  together with its conjugate $\psi^*$ as if they were independent;
- derives the field equation of a field of one space coordinate and time, and of
  a field of one EXTRA time and time, and shows that the extra time turns
  oscillation into exponential growth, with the growth rate of the Revision record.

Every result is checked by a line that starts with PASS; the last line counts the
checks. Eight figures are drawn and saved.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 07d (The action principle and the Euler-Lagrange equation) is the file
‘Revision/textbook/notebooks/07d_action_principle.ipynb‘ of the repository Dirac_claude.
```

It shows, for a reader who knows one-variable calculus, where the Euler-Lagrange equation comes from: it computes exactly the action of the true path and of two wrong paths of an oscillator and of their small variations, shows that only the true path makes the action stationary, checks the integration by parts, the fundamental lemma with narrow bumps, the action on a grid of times and its second-order convergence, the conserved energy, that a total derivative changes no equation, the first-order Lagrangians of a real pair and of one complex variable, and the field equation of a field of a space direction or of an extra time with time; it reproduces the growth rate of the Revision record for an extra-time momentum, and draws eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
```

```
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 07d_action_principle.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
```

```
    jupyter nbconvert --to notebook --execute --inplace 07d_action_principle.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 07d_action_principle.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/07d.captions.json‘
- ‘Revision/textbook/figures/07d_1_action_versus_epsilon.png‘
- ‘Revision/textbook/figures/07d_2_paths_and_residuals.png‘
- ‘Revision/textbook/figures/07d_3_bump_lemma.png‘
- ‘Revision/textbook/figures/07d_4_action_on_a_grid.png‘
- ‘Revision/textbook/figures/07d_5_energy_along_paths.png‘
- ‘Revision/textbook/figures/07d_6_first_order_rotation.png‘
- ‘Revision/textbook/figures/07d_7_space_and_extra_time.png‘
- ‘Revision/textbook/figures/07d_8_dispersion.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS all 8 figure files of this notebook exist
    ALL 30 CHECKS PASSED (notebook 07d)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file

  ‘Revision/textbook/notebooks/07d_action_principle.ipynb‘ inside the folder
  Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 07d_action_principle.ipynb
      python -m nbconvert --execute --inplace 07d_action_principle.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/theory/reports/...": the Revision
  record files of your copy of the repository differ from the committed ones; restore
  them with the command below (run in the repository folder) and run the notebook again.

      git restore Revision/theory/reports Revision/algebra

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 07d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 07d (The action principle and the Euler-Lagrange equation) is the file
# Revision/textbook/notebooks/07d_action_principle.ipynb of the repository Dirac_claude.
# It shows, for a reader who knows one-variable calculus, where the Euler-Lagrange
# equation comes from: it computes exactly the action of the true path and of two wrong
# paths of an oscillator and of their small variations, shows that only the true path
# makes the action stationary, checks the integration by parts, the fundamental lemma
```

```
# with narrow bumps, the action on a grid of times and its second-order convergence, the
# conserved energy, that a total derivative changes no equation, the first-order
# Lagrangians of a real pair and of one complex variable, and the field equation of a
# field of a space direction or of an extra time with time; it reproduces the growth rate
# of the Revision record for an extra-time momentum, and draws eight teaching plots. It
# needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
```

```
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
```

```
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 07d_action_principle.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 07d_action_principle.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 07d_action_principle.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/07d.captions.json
```

```
# - Revision/textbook/figures/07d_1_action_versus_epsilon.png
# - Revision/textbook/figures/07d_2_paths_and_residuals.png
# - Revision/textbook/figures/07d_3_bump_lemma.png
# - Revision/textbook/figures/07d_4_action_on_a_grid.png
# - Revision/textbook/figures/07d_5_energy_along_paths.png
# - Revision/textbook/figures/07d_6_first_order_rotation.png
# - Revision/textbook/figures/07d_7_space_and_extra_time.png
# - Revision/textbook/figures/07d_8_dispersion.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 8 figure files of this notebook exist
#     ALL 30 CHECKS PASSED (notebook 07d)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/07d_action_principle.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 07d_action_principle.ipynb
```

```
#     python -m nbconvert --execute --inplace 07d_action_principle.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "ValueError: ... is not a passing check of Revision/theory/reports/...": the Revision
#   record files of your copy of the repository differ from the committed ones; restore
#   them with the command below (run in the repository folder) and run the notebook
#   again.
#     git restore Revision/theory/reports Revision/algebra
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "07d"  # this notebook: chapter 07, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
```

```
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 07d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Path** (or motion): a function $q(t)$ that gives the position $q$ of a particle
  on a line at every time $t$ between a start time and an end time. Here the times
  run from $0$ to $T$. Its **velocity** is $\dot q = dq/dt$, its **acceleration**
  $\ddot q = d^2q/dt^2$.
- **Lagrangian** $L(q, \dot q)$: a function of the position and the velocity; for
  the oscillator of this notebook $L = \frac12\dot q^2 - \frac12\omega^2q^2$, the
  kinetic energy minus the potential energy (mass 1, angular frequency $\omega$).
- **Action** $S[q] = \int_0^T L(q(t), \dot q(t))\,dt$: one number for a whole
  path. The square brackets say that $S$ depends on the whole function $q$; such a
  rule (a function in, a number out) is called a **functional**.
- **Variation**: a shape $\xi(t)$ with $\xi(0) = \xi(T) = 0$; the varied path is
  $q + \epsilon\xi$ with a small number $\epsilon$. The **first variation** is the
  slope $dS[q + \epsilon\xi]/d\epsilon$ at $\epsilon = 0$.
- **Stationary path**: a path whose first variation is zero for every variation.
```

  The **principle of stationary action** says that the true motion is stationary.
- **Euler-Lagrange expression** $E(t) = \partial L/\partial q - \frac{d}{dt}\,\partial L/\partial\dot q$, and the **Euler-Lagrange equation** $E = 0$. In $\partial L/\partial q$ the velocity is held fixed, in $\partial L/\partial\dot q$ the position.
- **Integration by parts**: the product rule $\frac{d}{dt}(fg) = \dot f g + f\dot g$ integrated from 0 to T: $\int_0^T f\dot g\,dt = [fg]_0^T - \int_0^T \dot f g\,dt$, where $[fg]_0^T = f(T)g(T) - f(0)g(0)$ is the **boundary term**.
- **Bump**: a smooth shape that is positive on a small interval and zero outside.
- **Grid**: the equally spaced times $t_n = nh$, $n = 0, 1, \dots, N$, with step $h = T/N$. **Order of convergence**: the power p in error $\approx$ constant $\times\,h^p$.
- **Energy** $H = \dot q\,\partial L/\partial\dot q - L$; for the oscillator $H = \frac12\dot q^2 + \frac12\omega^2q^2$.
- **Total derivative**: $\frac{d}{dt}F(q(t), t)$ for some function F.
- **First-order Lagrangian**: a Lagrangian that contains the velocities only to the first power, like the Lagrangians of the fields of this book.
- **Field**: a function of several coordinates, here $\phi(x_1, x_4)$ or $\phi(x_5, x_4)$, with the author's names: x_1 a direction of 3-space, x_4 the time, x_5 one of the three extra times. **Lagrangian density**: the Lagrangian of a field, integrated over all its coordinates.
- **Plane wave**: a field such as $\cos(kx_1)\cos(\omega x_4)$, with **wave number** k and angular frequency ω. **Dispersion relation**: the equation that links ω to k.

Text cell:

```
## 4. The physical and mathematical situation

**The principle.** Among all paths with the same end points, $q(0) = q_A$ and
$q(T) = q_B$, the particle follows one for which the action does not change to
first order when the path is changed a little: for every variation $\xi$,

$$\frac{d}{d\epsilon}S[q + \epsilon\xi]\Big|_{\epsilon = 0} = 0 .$$

**From the principle to the equation, line by line.**

$$\frac{d}{d\epsilon}S[q + \epsilon\xi]\Big|_{\epsilon=0} = \int_0^T\Big(
\frac{\partial L}{\partial q}\,\xi + \frac{\partial L}{\partial\dot q}\,\dot\xi
\Big)dt$$

This first line differentiates under the integral sign with the chain rule: $L$
depends on $\epsilon$ through its first argument $q + \epsilon\xi$ (derivative
$\xi$) and through its second argument $\dot q + \epsilon\dot\xi$ (derivative
$\dot\xi$).

$$= \int_0^T\frac{\partial L}{\partial q}\,\xi\,dt + \Big[\frac{\partial L}
{\partial\dot q}\,\xi\Big]_0^T - \int_0^T\frac{d}{dt}\Big(\frac{\partial L}
{\partial\dot q}\Big)\xi\,dt$$

The second line integrates the second term by parts with $f = \partial L/\partial
\dot q$ and $g = \xi$.
```

```
$$= \int_0^T E(t)\,\xi(t)\,dt,\qquad E = \frac{\partial L}{\partial q} -
\frac{d}{dt}\frac{\partial L}{\partial\dot q} .$$

The third line drops the boundary term, which is zero because $\xi(0) = \xi(T) =
0$, and collects the two integrals. Finally the **fundamental lemma**: if a
continuous $E$ had $E(t_0) > 0$ at some time $t_0$, it would be positive on a small
interval around $t_0$, and a bump $\xi$ on that interval would give $\int E\xi\,dt
> 0$; the same with signs reversed for $E(t_0) < 0$. So the true path satisfies
the **Euler-Lagrange equation** $E(t) = 0$ at every time. For the oscillator,
$\partial L/\partial q = -\omega^2q$ and $\partial L/\partial\dot q = \dot q$, so
$E = -\omega^2q - \ddot q$ and the equation is $\ddot q = -\omega^2q$.

**The example of this notebook.** $\omega = 1$, $T = 1$, $q(0) = 0$, $q(1) = 1$.
The true path is $q(t) = \sin t/\sin 1$ (it solves $\ddot q = -q$ and has the right
end values). Two wrong paths with the same end values: the straight line $q = t$
and the parabola $q = t^2$. Variations: $\xi_n(t) = \sin(n\pi t)$, $n = 1, 2, 3$,
which vanish at $t = 0$ and $t = 1$.

**Fields.** For a field $\phi$ of several coordinates the action is the integral of
a Lagrangian density over all of them, the variation $\xi$ vanishes on the
boundary of the region, and the same three lines give one derivative term for each
coordinate: $E = \partial L/\partial\phi - \sum_\mu \frac{\partial}{\partial
x_\mu}\,\partial L/\partial(\partial_\mu\phi)$. This is the expression that
Notebook 07b computes for the 16 components of the fields of this book in the
author's metric (with left derivatives for anticommuting components, Notebook 07a).
```

Text cell:

```
## 5. The action of the true path and of two wrong paths

The next cell imports the packages, wraps the set-up cell's ‘check‘ so that a PASS
line and the line "reproduces ..." that may follow it are printed by one single
‘print‘ call (Jupyter may deliver two separate prints in two pieces, and the
book's tools must see the two lines together), and defines the helper
‘reproduces‘, which makes sure that a Revision report contains a check with the
verdict PASS before a PASS line names it.

Then it computes, with exact computer algebra (sympy), the action of $q +
\epsilon\xi_n$ for each of the three paths and $n = 1, 2, 3$. Since $L$ is a
polynomial of degree 2 in $q$ and $\dot q$, the action is a polynomial of degree 2
in $\epsilon$:

$$S[q + \epsilon\xi] = S_0 + S_1\,\epsilon + S_2\,\epsilon^2 .$$

$S_1$ is the first variation (the slope at $\epsilon = 0$). The cell prints $S_1$
for every path and every $n$, exactly and as a decimal number.
```

In [2]:

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
```

```
import re  # regular expressions: read a formula from a check's detail text

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece


def revision_check(report, name):
    """The check called name of a Revision report (a JSON file): its dictionary
    (name, verdict, detail); stops if it is missing or its verdict is not PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return found[0]


def reproduces(report, name):
    """The text "<report>, check <name>" after making sure the check passes."""
    revision_check(report, name)
    return f"{report}, check {name}"


t, eps = sp.symbols("t epsilon", real=True)  # the time and the size of the change
omega, T_end = sp.Integer(1), sp.Integer(1)  # omega = 1 and the end time T = 1


def lagrangian(q, q_dot):
    """The oscillator: kinetic energy minus potential energy."""
    return q_dot**2 / 2 - omega**2 * q**2 / 2


PATHS = {"true path": sp.sin(t) / sp.sin(1), "line": t, "parabola": t**2}
SHAPES = {n: sp.sin(n * sp.pi * t) for n in (1, 2, 3)}  # xi_n, zero at t = 0, 1


def action_polynomial(path, shape):
    """S[path + eps shape] as a polynomial in eps (exact)."""
    varied = path + eps * shape
    integrand = sp.expand(lagrangian(varied, sp.diff(varied, t)))
    return sp.Poly(sp.integrate(integrand, (t, 0, T_end)), eps)


S1 = {}  # the first variations, S1[(path name, n)]
```

```
S2 = {}  # the coefficients of eps^2
for name, path in PATHS.items():
    for n, shape in SHAPES.items():
        polynomial = action_polynomial(path, shape)
        S1[(name, n)] = sp.simplify(polynomial.coeff_monomial(eps))
        S2[(name, n)] = sp.simplify(polynomial.coeff_monomial(eps**2))
        say(f"{name:9s} n = {n}: S1 = {S1[(name, n)]} = "
            f"{float(S1[(name, n)]):+.6f}")
```

Out [2]:

```
true path n = 1: S1 = 0 = +0.000000
true path n = 2: S1 = 0 = +0.000000
true path n = 3: S1 = 0 = +0.000000
line      n = 1: S1 = -1/pi = -0.318310
line      n = 2: S1 = 1/(2*pi) = +0.159155
line      n = 3: S1 = -1/(3*pi) = -0.106103
parabola  n = 1: S1 = -5/pi + 4/pi**3 = -1.462543
parabola  n = 2: S1 = 1/(2*pi) = +0.159155
parabola  n = 3: S1 = (4 - 45*pi**2)/(27*pi**3) = -0.525738
```

Text cell:

```
The next cell checks the two facts the table shows: for the true path the first
variation is exactly zero for all three shapes, for the two wrong paths it is not.
It also checks that the coefficient $S_2$ does not depend on the path: $S_2 =
\int_0^T(\frac12\dot\xi^2 - \frac12\omega^2\xi^2)\,dt$ is the action of the shape
itself, because the terms of $L$ that are quadratic in $\epsilon$ contain only
$\xi$.
```

In [3]:

```
check(all(S1[("true path", n)] == 0 for n in SHAPES),
      "the true path: the first variation is exactly 0 for xi_1, xi_2, xi_3")
check(all(S1[(name, n)] != 0 for name in ("line", "parabola") for n in SHAPES),
      "the line and the parabola: the first variation is not 0")
check(all(S2[(name, n)] == S2[("true path", n)] for name in PATHS for n in SHAPES),
      "the coefficient of eps^2 is the same for every path")
for n in SHAPES:
    coefficient = S2[("true path", n)]  # the same for every path
    report(f"S2 for xi_{n}", f"{coefficient} = {float(coefficient):.6f}")
```

Out [3]:

```
PASS the true path: the first variation is exactly 0 for xi_1, xi_2, xi_3
PASS the line and the parabola: the first variation is not 0
PASS the coefficient of eps^2 is the same for every path
RESULT S2 for xi_1 = -1/4 + pi**2/4 = 2.217401
RESULT S2 for xi_2 = -1/4 + pi**2 = 9.619604
RESULT S2 for xi_3 = -1/4 + 9*pi**2/4 = 21.956610
```

Text cell:

```
## 6. A minimum, or only stationary?

For the shape $\xi_1 = \sin(\pi t/T)$ on an interval of general length $T$, the
coefficient $S_2$ is, line by line,

$$S_2 = \int_0^T\Big(\frac{\pi^2}{2T^2}\cos^2\frac{\pi t}{T} - \frac{\omega^2}{2}
\sin^2\frac{\pi t}{T}\Big)dt = \frac{\pi^2}{2T^2}\cdot\frac{T}{2} - \frac{\omega^2}
{2}\cdot\frac{T}{2} = \frac{T}{4}\Big(\frac{\pi^2}{T^2} - \omega^2\Big).$$

The first equality inserts $\dot\xi_1 = (\pi/T)\cos(\pi t/T)$. The second uses
$\cos^2 x = \frac12(1 + \cos 2x)$ and $\sin^2 x = \frac12(1 - \cos 2x)$: the interval
from 0 to $T$ is one full period of $\cos(2\pi t/T)$, whose integral over it is 0,
so $\cos^2(\pi t/T)$ and $\sin^2(\pi t/T)$ each integrate to $T/2$. The third
collects. So $S_2 > 0$ for $T < \pi/\omega$: then the true path is a MINIMUM of the
action along this direction. For $T > \pi/\omega$ the coefficient is negative: the
true path is still stationary ($S_1 = 0$), but the action is a maximum along
$\xi_1$ and a minimum along faster shapes, a saddle. This is why the principle is
called the principle of STATIONARY action, not of least action. The next cell
checks the formula with $T$ as a symbol, then draws Figure 1.
```

In [4]:

```
T = sp.Symbol("T", positive=True)
xi_T = sp.sin(sp.pi * t / T)  # the shape xi_1 on the interval from 0 to T
S2_general = sp.integrate(sp.diff(xi_T, t) ** 2 / 2 - omega**2 * xi_T**2 / 2,
                          (t, 0, T))
say(f"S2(T) = {sp.simplify(S2_general)}")
check(sp.simplify(S2_general - T / 4 * (sp.pi**2 / T**2 - omega**2)) == 0,
      "S2 = (T/4)(pi^2/T^2 - omega^2) for the shape xi_1 on (0, T)")
check(S2_general.subs(T, 1) > 0 and S2_general.subs(T, 4) < 0,
      "S2 > 0 for T = 1 (a minimum), S2 < 0 for T = 4 > pi (a saddle)")

epsilons = np.linspace(-0.6, 0.6, 121)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
for name, style in (("true path", "-"), ("line", "--"), ("parabola", ":")):
    polynomial = action_polynomial(PATHS[name], SHAPES[1])
    values = sp.lambdify(eps, polynomial.as_expr() - polynomial.coeff_monomial(1))
    left.plot(epsilons, values(epsilons), style, label=name)
    slope = float(S1[(name, 1)])
    left.plot(epsilons, slope * epsilons, style, color="grey", linewidth=0.8)
left.axvline(0.0, color="black", linewidth=0.8)
left.set_xlabel("size of the change $\\epsilon$")
left.set_ylabel("$S[q + \\epsilon\\xi_1] - S[q]$")
left.set_title("Only the true path has slope 0")
left.legend(fontsize=8)
lengths = np.linspace(0.3, 6.0, 300)
S2_values = sp.lambdify(T, S2_general)(lengths)
right.plot(lengths, S2_values)
right.axhline(0.0, color="black", linewidth=0.8)
right.axvline(np.pi, color="red", linestyle="--", label="$T = \\pi/\\omega$")
```

```
right.set_ylim(-2.0, 4.0)
right.set_xlabel("length of the time interval $T$")
right.set_ylabel("coefficient $S_2$ of $\\epsilon^2$")
right.set_title("Minimum for $T < \\pi$, saddle for $T > \\pi$")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "action_versus_epsilon",
            "Left: the change of the action of the oscillator ($\\omega = 1$, from "
            "$t = 0$ to $1$, end values 0 and 1) when a path $q$ is replaced by "
            "$q + \\epsilon\\xi_1$, against the size $\\epsilon$ of the change, for "
            "the true path "
            "$\\sin t/\\sin 1$ (solid) and the wrong paths $t$ (dashed) and $t^2$ "
            "(dotted), with the shape $\\xi_1 = \\sin\\pi t$; the thin grey lines "
            "are the tangents at $\\epsilon = 0$. Only the true path has a "
            "horizontal tangent: its first variation is zero. Right: the "
            "coefficient $S_2 = (T/4)(\\pi^2/T^2 - \\omega^2)$ of $\\epsilon^2$ "
            "against the length $T$ of the interval; it changes sign at $T = "
            "\\pi/\\omega$ (dashed red), where the minimum becomes a saddle.")
```

Out [4]:

```
S2(T) = (-T**2 + pi**2)/(4*T)
PASS S2 = (T/4)(pi^2/T^2 - omega^2) for the shape xi_1 on (0, T)
PASS S2 > 0 for T = 1 (a minimum), S2 < 0 for T = 4 > pi (a saddle)
```

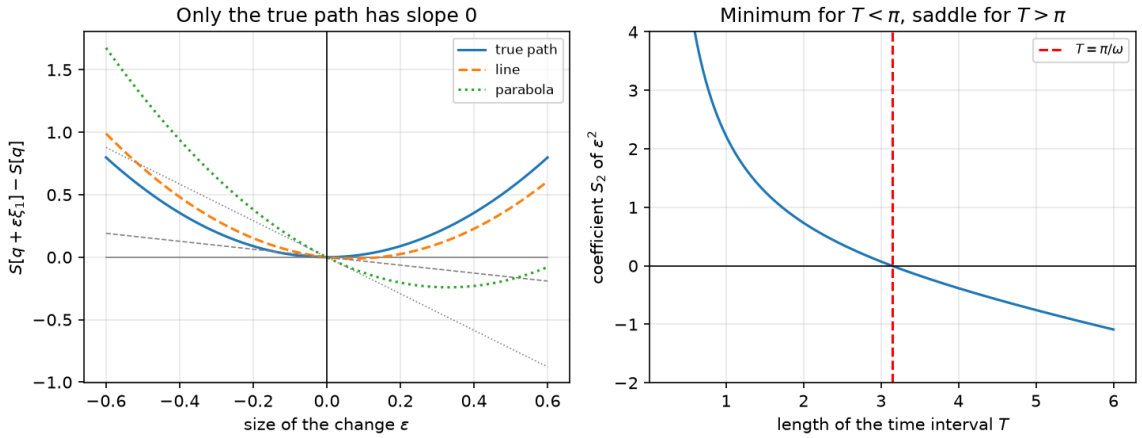

Figure 204: Left: the change of the action of the oscillator ($\omega = 1$, from $t = 0$ to 1, end values 0 and 1) when a path q is replaced by $q + \epsilon\xi_1$, against the size ϵ of the change, for the true path $\sin t/\sin 1$ (solid) and the wrong paths t (dashed) and t^2 (dotted), with the shape $\xi_1 = \sin \pi t$; the thin grey lines are the tangents at $\epsilon = 0$. Only the true path has a horizontal tangent: its first variation is zero. Right: the coefficient $S_2 = (T/4)(\pi^2/T^2 - \omega^2)$ of ϵ^2 against the length T of the interval; it changes sign at $T = \pi/\omega$ (dashed red), where the minimum becomes a saddle. (Notebook 07d, figure 1.)

Out [4]:

```
Figure 07d.1 saved as Revision/textbook/figures/07d_1_action_versus_epsilon.png
```

Text cell:

```
## 7. The first variation is the integral of $E$ times the shape

The derivation of section 4 says $S_1 = \int_0^1 E\,\xi\,dt$ when $\xi$ vanishes at
both ends, and $S_1 = \int_0^1 E\,\xi\,dt + [\dot q\,\xi]_0^1$ in general (for the
oscillator $\partial L/\partial\dot q = \dot q$). The next cell defines a function
‘euler_lagrange‘ that computes $E$ for any Lagrangian $L(q, \dot q)$ exactly as
the words of section 3 say: it replaces $q(t)$ and $\dot q(t)$ by two plain
symbols, differentiates with respect to each with the other held fixed, puts the
functions back, and differentiates the second result with respect to $t$. It
compares the result with sympy’s own ‘euler_equations‘, prints $E$ for the three
paths ($E = -\ddot q - q$), and checks the formula for $S_1$ for the two wrong
paths, both for the shapes $\xi_n$ and for the shape $\xi = t$, which does NOT
vanish at $t = 1$ and therefore produces the boundary term $\dot q(1)\cdot 1$.
```

In [5]:

```
from sympy.calculus.euler import euler_equations  # sympy’s own Euler-Lagrange


def euler_lagrange(L, functions, variable):
    """The Euler-Lagrange expressions dL/dq - d/dt dL/d(q’) of the Lagrangian L
    for each function q of the list functions of the variable."""
    values = [sp.Symbol(f"Q{i}") for i in range(len(functions))]  # stand for q
    speeds = [sp.Symbol(f"V{i}") for i in range(len(functions))]  # stand for q’
    plain = L
    for function, speed in zip(functions, speeds):
        plain = plain.subs(sp.diff(function, variable), speed)  # q’ -> V first
    for function, value in zip(functions, values):
        plain = plain.subs(function, value)  # then q -> Q
    back = {**{s: sp.diff(f, variable) for f, s in zip(functions, speeds)},
            **{v: f for f, v in zip(functions, values)}}  # Q -> q, V -> q’
    return [sp.expand(sp.diff(plain, value).subs(back)
                      - sp.diff(sp.diff(plain, speed).subs(back), variable))
            for value, speed in zip(values, speeds)]


q = sp.Function("q")(t)  # a general path
E_general = euler_lagrange(lagrangian(q, sp.diff(q, t)), [q], t)[0]
say(f"E for the oscillator: {E_general}")
sympy_E = euler_equations(lagrangian(q, sp.diff(q, t)), [q], t)[0]
check(sp.simplify(E_general - (sympy_E.lhs - sympy_E.rhs)) == 0
      and sp.simplify(E_general + sp.diff(q, t, 2) + omega**2 * q) == 0,
      "euler_lagrange gives E = -q’’ - omega^2 q, as sympy’s euler_equations")


def E_of(path):
    """E(t) of a given path (insert the path into E_general)."""
    return sp.simplify(E_general.subs(q, path).doit())
```

```
for name, path in PATHS.items():
    say(f"{name:9s}: E(t) = {E_of(path)}")
check(E_of(PATHS["true path"]) == 0, "the true path solves E = 0")
by_parts = all(sp.simplify(S1[(name, n)]
                           - sp.integrate(E_of(PATHS[name]) * SHAPES[n], (t, 0, 1)))
               == 0 for name in ("line", "parabola") for n in SHAPES)
check(by_parts, "S1 = integral of E xi_n from 0 to 1 for both wrong paths and n = 1, "
      "2, 3")
boundary_ok = True
xi_open = t  # a shape with xi(0) = 0 but xi(1) = 1
for name in ("line", "parabola"):
    path = PATHS[name]
    first = action_polynomial(path, xi_open).coeff_monomial(eps)  # its S1
    inside = sp.integrate(E_of(path) * xi_open, (t, 0, 1))
    p_xi = sp.diff(path, t) * xi_open  # (dL/dq') xi = q' xi for the oscillator
    boundary = p_xi.subs(t, 1) - p_xi.subs(t, 0)  # [q' xi] from 0 to 1
    say(f"{name}, xi = t: S1 = {sp.simplify(first)}, integral = "
        f"{sp.simplify(inside)}, boundary term = {boundary}")
    boundary_ok = boundary_ok and sp.simplify(first - inside - boundary) == 0
check(boundary_ok, "for xi = t (not zero at t = 1): S1 = integral of E xi + "
      "boundary term q'(1) xi(1)")
```

Out [5]:

```
E for the oscillator: -q(t) - Derivative(q(t), (t, 2))
PASS euler_lagrange gives E = -q'' - omega^2 q, as sympy's euler_equations
true path: E(t) = 0
line     : E(t) = -t
parabola : E(t) = -t**2 - 2
PASS the true path solves E = 0
PASS S1 = integral of E xi_n from 0 to 1 for both wrong paths and n = 1, 2, 3
line, xi = t: S1 = 2/3, integral = -1/3, boundary term = 1
parabola, xi = t: S1 = 3/4, integral = -5/4, boundary term = 2
PASS for xi = t (not zero at t = 1): S1 = integral of E xi + boundary term q'(1) xi(1)
```

Text cell:

```
Figure 2 shows the three paths and their Euler-Lagrange expressions. $E(t) =
-\ddot q - q$ is the amount by which Newton's law $\ddot q = -q$ fails at time $t$:
zero for the true path, $-t$ for the line ($\ddot q = 0$) and $-2 - t^2$ for the
parabola ($\ddot q = 2$).
```

In [6]:

```
times = np.linspace(0.0, 1.0, 201)
styles = {"true path": "-", "line": "--", "parabola": ":"}
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
for name, path in PATHS.items():
    left.plot(times, sp.lambdify(t, path)(times) * np.ones_like(times),
              styles[name], label=name)
    residual = sp.lambdify(t, E_of(path))(times) * np.ones_like(times)
```

```
    right.plot(times, residual, styles[name], label=name)
left.set_xlabel("time $t$")
left.set_ylabel("position $q(t)$")
left.set_title("Three paths with $q(0) = 0$, $q(1) = 1$")
left.legend(fontsize=8)
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("time $t$")
right.set_ylabel("$E(t) = -\\ddot q - q$")
right.set_title("Euler-Lagrange expression")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "paths_and_residuals",
            "Left: three paths of the oscillator with the same end values $q(0) = "
            "0$ and $q(1) = 1$ against the time $t$ (pure numbers): the true path "
            "$\\sin t/\\sin 1$ (solid), the line $t$ (dashed) and the parabola "
            "$t^2$ (dotted); they look alike. Right: their Euler-Lagrange "
            "expressions $E(t) = -\\ddot q - q$, the amount by which the law of "
            "motion $\\ddot q = -q$ fails: zero for the true path at every time, "
            "$-t$ for the line, $-2 - t^2$ for the parabola. The first variation of "
            "the action is the integral of $E$ times the shape of the change.")
```

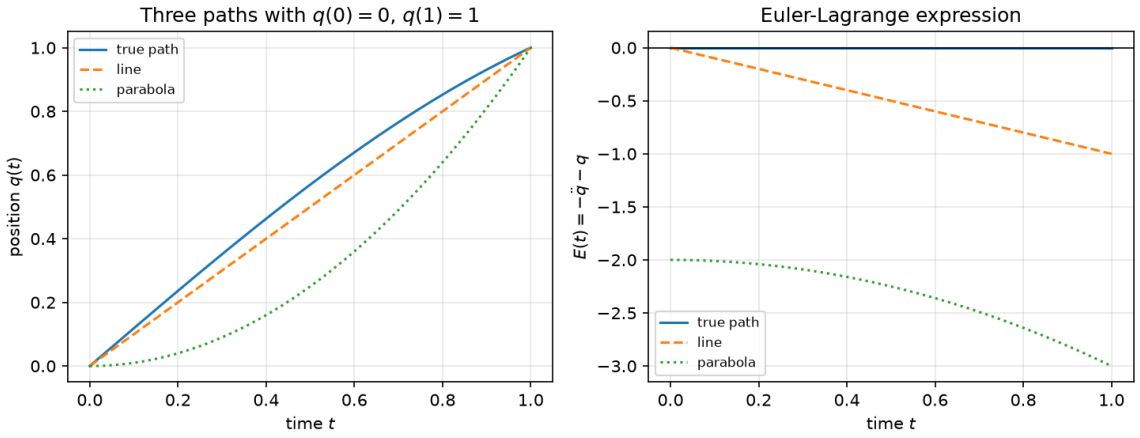

Figure 205: Left: three paths of the oscillator with the same end values $q(0) = 0$ and $q(1) = 1$ against the time t (pure numbers): the true path $\sin t / \sin 1$ (solid), the line t (dashed) and the parabola t^2 (dotted); they look alike. Right: their Euler-Lagrange expressions $E(t) = -\ddot{q} - q$, the amount by which the law of motion $\ddot{q} = -q$ fails: zero for the true path at every time, $-t$ for the line, $-2 - t^2$ for the parabola. The first variation of the action is the integral of E times the shape of the change. (Notebook 07d, figure 2.)

Out [6]:

```
Figure 07d.2 saved as Revision/textbook/figures/07d_2_paths_and_residuals.png
```

Text cell:

```
## 8. The fundamental lemma, seen with narrow bumps
```

```
Take the parabola, whose $E(t) = -2 - t^2$ is not zero, and a bump of half-width
$w$ centred at $t_0$:

$$\xi(t) = \cos^2\Big(\frac{\pi(t - t_0)}{2w}\Big)\ \text{for}\ |t - t_0| < w,
\qquad \xi(t) = 0\ \text{otherwise}.$$

The ratio $\int E\xi\,dt / \int\xi\,dt$ is an average of $E$ over the bump. As the
bump becomes narrower it tends to $E(t_0)$; so if $\int E\xi\,dt$ were zero for
every bump, $E(t_0)$ would be zero at every $t_0$. For this $E$ the error of the
average is exactly proportional to $w^2$: halving $w$ divides it by 4 (an average
over a symmetric bump removes the linear part of $E$ around $t_0$, and the
quadratic part leaves a term of size $w^2$). The next cell computes the integrals
with the trapezoidal rule on 200001 points (fine enough that its own error does not
matter), for $t_0 = 0.3, 0.5, 0.7$ and $w = 0.2, 0.1, 0.05, 0.025$, checks the
factor 4, and draws Figure 3.
```

In [7]:

```
grid = np.linspace(0.0, 1.0, 200001)  # fine grid for the integrals


def E_parabola(times_):
    return -2.0 - times_**2  # E(t) of the parabola path


def bump(times_, centre, width):
    """cos^2 bump of half-width width around centre, zero outside."""
    inside = np.abs(times_ - centre) < width
    return np.where(inside, np.cos(np.pi * (times_ - centre) / (2 * width)) ** 2, 0.0)


widths = [0.2, 0.1, 0.05, 0.025]
centres = [0.3, 0.5, 0.7]
errors = {}
for centre in centres:
    errors[centre] = []
    for width in widths:
        weight = bump(grid, centre, width)
        average = (np.trapezoid(E_parabola(grid) * weight, grid)
                   / np.trapezoid(weight, grid))
        errors[centre].append(average - E_parabola(centre))
    ratios = [errors[centre][i] / errors[centre][i + 1] for i in range(3)]
    say(f"t0 = {centre}: errors " + ", ".join(f"{e:+.3e}" for e in errors[centre])
        + "; ratios " + ", ".join(f"{r:.4f}" for r in ratios))
check(all(abs(errors[c][i] / errors[c][i + 1] - 4.0) < 1e-6
          for c in centres for i in range(3)),
      "the average of E over the bump tends to E(t0); halving w divides the error by 4")

fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(times, E_parabola(times), color="black", label="$E(t) = -2 - t^2$")
for width, marker in zip(widths[:3], ["o", "s", "^"]):
    points = np.linspace(0.25, 0.75, 6)
    averages = [np.trapezoid(E_parabola(grid) * bump(grid, c, width), grid)
```

```
                / np.trapezoid(bump(grid, c, width), grid) for c in points]
    left.plot(points, averages, marker, label=f"bump average, $w = {width}$")
shown = bump(times, 0.5, 0.2)
left.fill_between(times, -2.9, -2.9 + 0.5 * shown, color="tab:green", alpha=0.3,
                  label="a bump ($t_0 = 0.5$, $w = 0.2$)")
left.set_ylim(-3.0, -1.8)
left.set_xlabel("time $t$ (bump centre $t_0$)")
left.set_ylabel("value")
left.set_title("Averages of $E$ over bumps")
left.legend(fontsize=7, loc="upper right")
for centre, marker in zip(centres, ["o", "s", "^"]):
    right.loglog(widths, np.abs(errors[centre]), marker, label=f"$t_0 = {centre}$")
reference = np.abs(errors[0.5][0]) * (np.array(widths) / widths[0]) ** 2
right.loglog(widths, reference, "--", color="grey", label="slope 2")
right.set_xticks(widths, [str(width) for width in widths])  # plain tick labels
right.minorticks_off()  # no unlabelled minor ticks
right.set_xlabel("half-width $w$ of the bump")
right.set_ylabel("|average of $E$ minus $E(t_0)$|")
right.set_title("The error falls like $w^2$")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "bump_lemma",
            "The fundamental lemma made visible, for the parabola path with $E(t) = "
            "-2 - t^2$ (black line). Left: the average of $E$ over a bump "
            "$\\cos^2(\\pi(t - t_0)/(2w))$ of half-width $w$ (one bump is shaded "
            "green at the bottom), plotted at its centre $t_0$ for $w = 0.2$, $0.1$, "
            "$0.05$; the averages approach the curve as the bumps narrow. Right: "
            "the distance of the average from $E(t_0)$ against $w$ on logarithmic "
            "axes for three centres; the points lie on a line of slope 2 (dashed): "
            "the error is proportional to $w^2$. A function whose integral against "
            "every bump vanishes must therefore vanish everywhere.")
```

Out [7]:

```
t0 = 0.3: errors -5.228e-03, -1.307e-03, -3.267e-04, -8.168e-05; ratios 4.0000, 4.0000,
    4.0000
t0 = 0.5: errors -5.228e-03, -1.307e-03, -3.267e-04, -8.168e-05; ratios 4.0000, 4.0000,
    4.0000
t0 = 0.7: errors -5.228e-03, -1.307e-03, -3.267e-04, -8.168e-05; ratios 4.0000, 4.0000,
    4.0000
PASS the average of E over the bump tends to E(t0); halving w divides the error by 4
```

Out [7]:

```
Figure 07d.3 saved as Revision/textbook/figures/07d_3_bump_lemma.png
```

Text cell:

```
## 9. The action on a grid

A computer stores a path as its values $q_0, q_1, \dots, q_N$ at the grid times
```

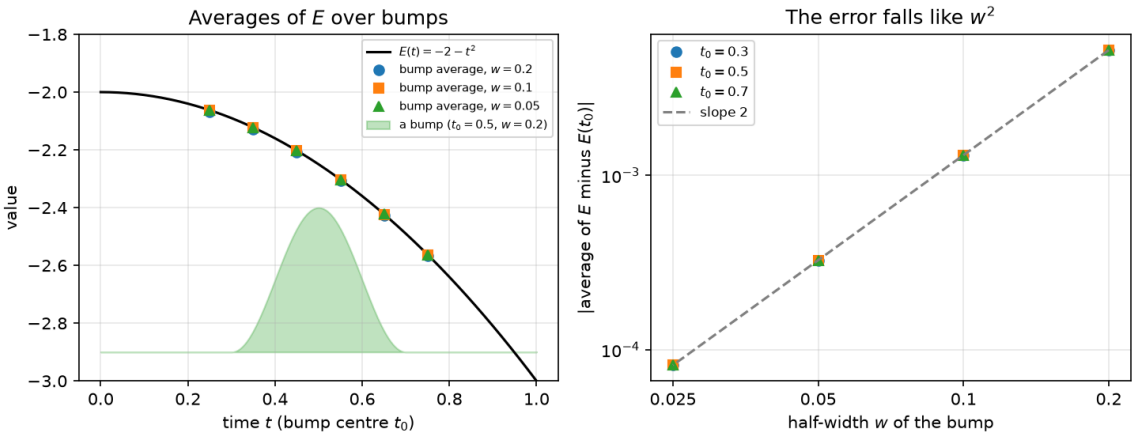

Figure 206: The fundamental lemma made visible, for the parabola path with $E(t) = -2 - t^2$ (black line). Left: the average of E over a bump $\cos^2(\pi(t - t_0)/(2w))$ of half-width w (one bump is shaded green at the bottom), plotted at its centre t_0 for $w = 0.2$, 0.1, 0.05; the averages approach the curve as the bumps narrow. Right: the distance of the average from $E(t_0)$ against w on logarithmic axes for three centres; the points lie on a line of slope 2 (dashed): the error is proportional to w^2. A function whose integral against every bump vanishes must therefore vanish everywhere. (Notebook 07d, figure 3.)

```
$t_n = nh$, $h = T/N$, and replaces the integral by a sum. With the velocity on
each step written as a difference quotient and the potential energy averaged over
the two ends of the step,

$$S_N = \sum_{n=0}^{N-1} h\Big[\frac12\Big(\frac{q_{n+1} - q_n}{h}\Big)^2 -
\frac{\omega^2}{4}\big(q_n^2 + q_{n+1}^2\big)\Big].$$

An inner value $q_n$ ($0 < n < N$) appears in two terms of the sum, the step that
ends at $t_n$ and the step that starts there. Differentiating, line by line:

$$\frac{\partial S_N}{\partial q_n} = h\Big[\frac{q_n - q_{n-1}}{h^2} -
\frac{q_{n+1} - q_n}{h^2} - \frac{\omega^2}{2}q_n - \frac{\omega^2}{2}q_n\Big]
= h\Big[-\frac{q_{n+1} - 2q_n + q_{n-1}}{h^2} - \omega^2q_n\Big].$$

The first equality takes the derivative of the two squares (chain rule: $\frac12
\cdot 2\cdot(q_n - q_{n-1})/h \cdot 1/h$ and $\frac12\cdot 2\cdot(q_{n+1} -
q_n)/h\cdot(-1/h)$) and of the two potential terms; the second collects. The
bracket is $E = -\ddot q - \omega^2q$ with $\ddot q$ replaced by the second
difference quotient. So "the action on the grid is stationary" means "the grid
version of the Euler-Lagrange equation holds at every inner point". The next cell
checks the derivative with sympy for $N = 5$ (symbols $h$, $\omega$, $q_0, \dots,
q_5$).
```

In [8]:

```
N5 = 5
h, w_sym = sp.symbols("h omega", positive=True)
qs = sp.symbols(f"q0:{N5 + 1}", real=True)  # q_0 .. q_5
```

```
S_grid = sum(h * (((qs[n + 1] - qs[n]) / h) ** 2 / 2
                  - w_sym**2 * (qs[n] ** 2 + qs[n + 1] ** 2) / 4) for n in range(N5))
grid_E = [h * (-(qs[n + 1] - 2 * qs[n] + qs[n - 1]) / h**2 - w_sym**2 * qs[n])
          for n in range(1, N5)]
check(all(sp.simplify(sp.diff(S_grid, qs[n]) - grid_E[n - 1]) == 0
          for n in range(1, N5)),
      "dS_N/dq_n = h (-(q_(n+1) - 2 q_n + q_(n-1))/h^2 - omega^2 q_n) for every "
      "inner n")
```

Out [8]:

```
PASS dS_N/dq_n = h (-(q_(n+1) - 2 q_n + q_(n-1))/h^2 - omega^2 q_n) for every inner n
```

Text cell:

```
Setting all inner derivatives to zero gives $N - 1$ linear equations for the $N - 1$
inner values (the end values $q_0 = 0$ and $q_N = 1$ are fixed). The next cell
solves them with numpy for $N = 4, 8, \dots, 256$ and measures the largest
difference from the true path $\sin t/\sin 1$ at the grid times. Each doubling of
$N$ should divide the error by about 4: the error is proportional to $h^2$ (order
2), because the second difference quotient differs from $\ddot q$ by a term of
size $h^2$. The cell prints the errors and the observed orders
$\log_2(\text{error}_N/\text{error}_{2N})$, and draws Figure 4.
```

In [9]:

```
def grid_path(N):
    """Solve the grid equations: returns the grid times and q_0 .. q_N."""
    step = 1.0 / N
    A = np.zeros((N - 1, N - 1))  # one row per inner point n = 1 .. N-1
    b = np.zeros(N - 1)
    for i in range(N - 1):
        A[i, i] = 2.0 / step**2 - 1.0  # from -(-2 q_n)/h^2 - omega^2 q_n
        if i > 0:
            A[i, i - 1] = -1.0 / step**2  # the neighbour q_(n-1)
        if i < N - 2:
            A[i, i + 1] = -1.0 / step**2  # the neighbour q_(n+1)
    b[-1] = 1.0 / step**2  # the known end value q_N = 1 moved to the right side
    inner = np.linalg.solve(A, b)
    return np.linspace(0.0, 1.0, N + 1), np.concatenate([[0.0], inner, [1.0]])


sizes = [4, 8, 16, 32, 64, 128, 256]
grid_errors = []
for N in sizes:
    nodes, values = grid_path(N)
    grid_errors.append(np.abs(values - np.sin(nodes) / np.sin(1.0)).max())
orders = [np.log2(grid_errors[i] / grid_errors[i + 1]) for i in range(len(sizes) - 1)]
say("largest errors: " + ", ".join(f"N = {N}: {e:.3e}"
                                   for N, e in zip(sizes, grid_errors)))
say("observed orders: " + ", ".join(f"{p:.3f}" for p in orders))
```

```
check(all(1.9 < p < 2.1 for p in orders),
      "the stationary grid path converges to the true path with order 2")

fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(times, np.sin(times) / np.sin(1.0), color="black", label="true path")
for N, marker in ((4, "o"), (8, "s")):
    nodes, values = grid_path(N)
    left.plot(nodes, values, marker, label=f"stationary grid path, $N = {N}$")
left.set_xlabel("time $t$")
left.set_ylabel("position $q$")
left.set_title("The action on a grid")
left.legend(fontsize=8)
steps = 1.0 / np.array(sizes)
right.loglog(steps, grid_errors, "o-", label="largest error")
right.loglog(steps, grid_errors[0] * (steps / steps[0]) ** 2, "--", color="grey",
             label="slope 2")
right.set_xlabel("step $h = 1/N$")
right.set_ylabel("largest difference from the true path")
right.set_title("Second-order convergence")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "action_on_a_grid",
            "The action on a grid of times. Left: the path that makes the grid "
            "action $S_N$ stationary, for $N = 4$ (circles) and $N = 8$ (squares) "
            "steps, and the true path $\\sin t/\\sin 1$ (black line), against the "
            "time $t$ (pure numbers); already four steps follow the true path "
            "closely. Right: the largest difference between the grid path and the "
            "true path against the step $h = 1/N$, for $N = 4$ to $256$, on "
            "logarithmic axes; the points lie on a line of slope 2 (dashed): halving "
            "the step divides the error by 4.")
```

Out [9]:

```
largest errors: N = 4: 4.089e-04, N = 8: 1.017e-04, N = 16: 2.582e-05, N = 32: 6.452e-06,
    N = 64: 1.613e-06, N = 128: 4.032e-07, N = 256: 1.008e-07
observed orders: 2.007, 1.978, 2.001, 2.000, 2.000, 2.000
PASS the stationary grid path converges to the true path with order 2
```

Out [9]:

```
Figure 07d.4 saved as Revision/textbook/figures/07d_4_action_on_a_grid.png
```

Text cell:

```
## 10. Energy is constant on the true path

The energy is $H = \dot q\,\partial L/\partial\dot q - L$; for the oscillator $H =
\dot q^2 - (\frac12\dot q^2 - \frac12\omega^2q^2) = \frac12\dot q^2 +
\frac12\omega^2q^2$, kinetic plus potential energy. Its time derivative, line by
line, for any path:
```

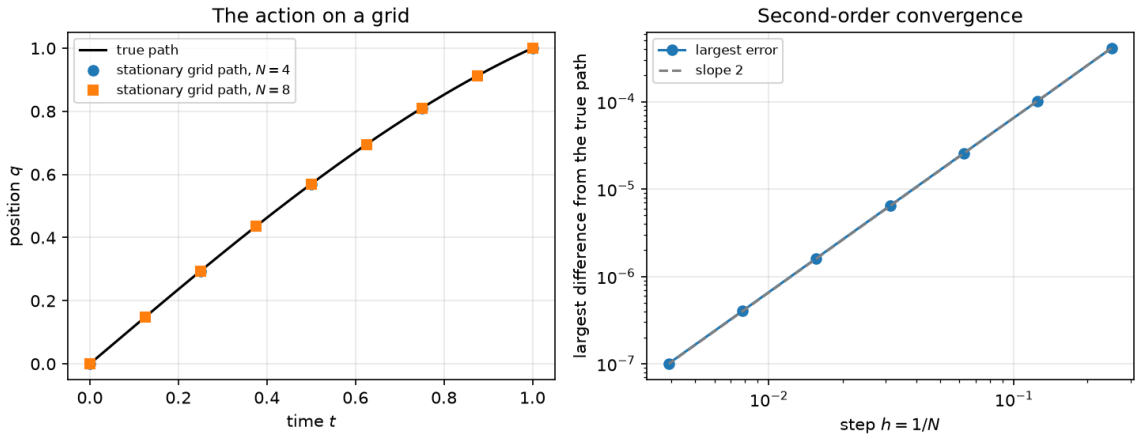

Figure 207: The action on a grid of times. Left: the path that makes the grid action S_N stationary, for $N = 4$ (circles) and $N = 8$ (squares) steps, and the true path $\sin t/\sin 1$ (black line), against the time t (pure numbers); already four steps follow the true path closely. Right: the largest difference between the grid path and the true path against the step $h = 1/N$, for $N = 4$ to 256, on logarithmic axes; the points lie on a line of slope 2 (dashed): halving the step divides the error by 4. (Notebook 07d, figure 4.)

```
$$\frac{dH}{dt} = \ddot q\,\dot q + \omega^2q\,\dot q = \dot q\,(\ddot q +
\omega^2q) = -\dot q\,E .$$

The first equality is the chain rule; the second takes out the common factor
$\dot q$; the third uses $E = -\ddot q - \omega^2q$. So $H$ is constant on every
solution of $E = 0$ and changes on a path where $E \neq 0$ (unless $\dot q = 0$).
The next cell checks the identity for a general path, computes $dH/dt$ on the three
paths and draws Figure 5.
```

In [10]:

```
def energy(path):
    """H = (1/2) q'^2 + (1/2) omega^2 q^2 of a path."""
    return sp.diff(path, t) ** 2 / 2 + omega**2 * path**2 / 2


check(sp.simplify(sp.diff(energy(q), t) + sp.diff(q, t) * E_general) == 0,
      "dH/dt = -q' E for every path")
for name, path in PATHS.items():
    say(f"{name:9s}: dH/dt = {sp.simplify(sp.diff(energy(path), t))}")
check(sp.simplify(sp.diff(energy(PATHS["true path"]), t)) == 0
      and all(sp.simplify(sp.diff(energy(PATHS[name]), t)) != 0
              for name in ("line", "parabola")),
      "H is constant on the true path and changes on the two wrong paths")
true_energy = sp.simplify(energy(PATHS["true path"]))
report("energy of the true path", f"{true_energy} = {float(true_energy):.6f}")

fig, ax = plt.subplots()
for name, path in PATHS.items():
```

```
    values = sp.lambdify(t, energy(path))(times) * np.ones_like(times)
    ax.plot(times, values, styles[name], label=name)
ax.set_xlabel("time $t$")
ax.set_ylabel("$H = \\frac{1}{2}\\dot q^2 + \\frac{1}{2}q^2$")
ax.set_title("The energy along the three paths")
ax.legend(fontsize=8)
save_figure(fig, "energy_along_paths",
            "The energy $H = \\frac{1}{2}\\dot q^2 + \\frac{1}{2}\\omega^2q^2$ of the "
            "oscillator ($\\omega = 1$) along the three paths of Figure 07d.2, against "
            "the time $t$ (pure numbers). On the true path (solid) it is constant, "
            "$1/(2\\sin^2 1) \\approx 0.706$; on the line (dashed) and the "
            "parabola (dotted) it changes, because $dH/dt = -\\dot q\\,E$ and their "
            "Euler-Lagrange expression $E$ is not zero.")
```

Out [10]:

```
PASS dH/dt = -q' E for every path
true path: dH/dt = 0
line     : dH/dt = t
parabola : dH/dt = 2*t*(t**2 + 2)
PASS H is constant on the true path and changes on the two wrong paths
RESULT energy of the true path = 1/(2*sin(1)**2) = 0.706141
```

Out [10]:

```
Figure 07d.5 saved as Revision/textbook/figures/07d_5_energy_along_paths.png
```

Text cell:

```
## 11. A total derivative changes nothing

Add to $L$ the total derivative of a function $F(q, t)$, here $F = t\,q^3$:

$$L' = L + \frac{d}{dt}F(q(t), t) = L + q^3 + 3t\,q^2\dot q .$$

The action changes by $\int_0^1 \frac{dF}{dt}\,dt = F(q(1), 1) - F(q(0), 0)$, the
same number for every path with the given end values (here $1\cdot 1^3 - 0 = 1$).
A change that is the same for every path cannot change which path is stationary,
so $L'$ has the same Euler-Lagrange equation as $L$. In particular a Lagrangian
that is ONLY a total derivative, such as $\frac{d}{dt}(q^2) = 2q\dot q$, has the
Euler-Lagrange expression $2\dot q - \frac{d}{dt}(2q) = 0$ for every path: it
gives no equation at all. (Notebook 07c meets exactly this in the author's own
Lagrangian for a real anticommuting field.) The next cell checks all three
statements.
```

In [11]:

```
F = t * q**3  # F(q, t)
L = lagrangian(q, sp.diff(q, t))
```

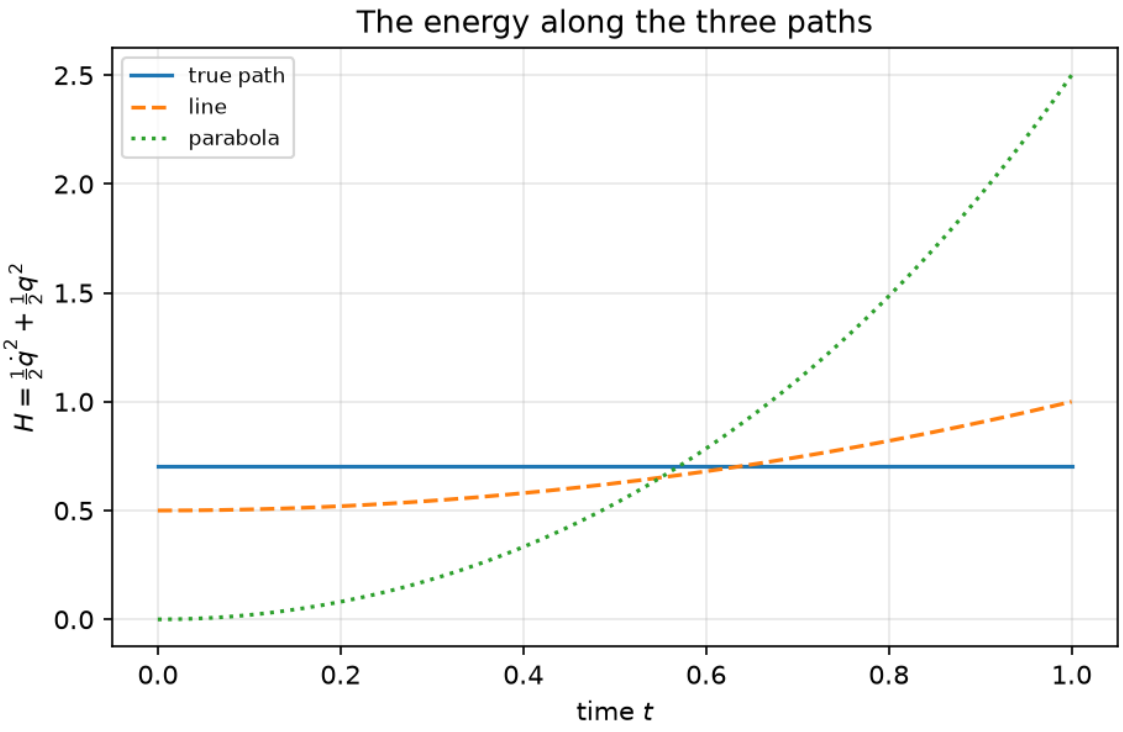

Figure 208: The energy $H = \frac{1}{2}\dot{q}^2 + \frac{1}{2}\omega^2 q^2$ of the oscillator ($\omega = 1$) along the three paths of Figure 07d.2, against the time t (pure numbers). On the true path (solid) it is constant, $1/(2\sin^2 1) \approx 0.706$; on the line (dashed) and the parabola (dotted) it changes, because $dH/dt = -\dot{q}\,E$ and their Euler-Lagrange expression E is not zero. (Notebook 07d, figure 5.)

```
E_changed = euler_lagrange(L + sp.diff(F, t), [q], t)[0]
check(sp.simplify(E_changed - E_general) == 0,
      "L + dF/dt with F = t q^3 has the same Euler-Lagrange expression as L")
E_pure = euler_lagrange(sp.diff(q**2, t), [q], t)[0]
check(E_pure == 0, "the pure total derivative d(q^2)/dt gives E = 0 for every path")
differences = [sp.simplify(sp.integrate(sp.diff(F.subs(q, path), t), (t, 0, 1)))
               for path in PATHS.values()]
say(f"S'[path] - S[path] for the three paths: {differences}")
check(all(d == 1 for d in differences),
      "the action changes by F(q(1), 1) - F(q(0), 0) = 1 for every path")
```

Out [11]:

```
PASS L + dF/dt with F = t q^3 has the same Euler-Lagrange expression as L
PASS the pure total derivative d(q^2)/dt gives E = 0 for every path
S'[path] - S[path] for the three paths: [1, 1, 1]
PASS the action changes by F(q(1), 1) - F(q(0), 0) = 1 for every path
```

Text cell:

```
## 12. First-order Lagrangians

The Lagrangians of the two fields of this book contain the derivatives only to the
first power. Two small examples show what such Lagrangians do.

**Example A: two real variables.** $L_A = \frac12(q_0\dot q_1 - q_1\dot q_0) -
\frac12\omega(q_0^2 + q_1^2)$. For $q_0$: $\partial L_A/\partial q_0 = \frac12\dot
q_1 - \omega q_0$ and $\partial L_A/\partial\dot q_0 = -\frac12 q_1$, whose time
derivative is $-\frac12\dot q_1$; so $E_0 = \frac12\dot q_1 - \omega q_0 + \frac12
\dot q_1 = \dot q_1 - \omega q_0$. In the same way $E_1 = -\dot q_0 - \omega q_1$.
The equations $\dot q_1 = \omega q_0$, $\dot q_0 = -\omega q_1$ are of FIRST order:
the starting values alone fix the motion, $q_0 = \cos\omega t$, $q_1 = \sin\omega t$
for $q_0(0) = 1$, $q_1(0) = 0$, a rotation. The kinetic part is antisymmetric in
$(q_0, q_1)$ but not a total derivative for ordinary numbers; for anticommuting
numbers it would be one (Notebook 07a, section 17).

**Example B: one complex variable.** $\psi = (q_0 + iq_1)/\sqrt2$ and

$$L_B = \frac{i}{2}\big(\psi^*\dot\psi - \dot\psi^*\psi\big) - \omega\,\psi^*\psi .$$

Multiplying out with $\psi^* = (q_0 - iq_1)/\sqrt2$ gives $\psi^*\psi = \frac12(q_0^2
+ q_1^2)$ and $\psi^*\dot\psi - \dot\psi^*\psi = i(q_0\dot q_1 - q_1\dot q_0)$, so
$L_B = \frac12(q_1\dot q_0 - q_0\dot q_1) - \frac12\omega(q_0^2 + q_1^2)$: Example A
with the sense of rotation reversed. The two complex combinations can be varied AS
IF they were independent, because $(q_0, q_1) \to (\psi, \psi^*)$ is an invertible
linear change of variables; the Euler-Lagrange expressions combine in the same way,
$E_{\psi^*} = (E_{q_0} + iE_{q_1})/\sqrt2$. Varying $\psi^*$: $\partial L_B/\partial
\psi^* = \frac{i}{2}\dot\psi - \omega\psi$ and $\partial L_B/\partial\dot\psi^* =
-\frac{i}{2}\psi$, so $E_{\psi^*} = i\dot\psi - \omega\psi$ and $i\dot\psi =
\omega\psi$, with the solution $\psi = e^{-i\omega t}\psi(0)$. The Lagrangian of
the fields of this book has exactly this structure: $\psi$ becomes the column
$\Psi$, $\psi^*$ the row $\bar\Psi = \Psi^\dagger C$, and $\omega$ a matrix of
```

```
derivatives. The next cell checks every statement of the two examples, with
$\omega$ as a symbol.
```

In [12]:

```
w = sp.Symbol("omega", positive=True)
q0, q1 = sp.Function("q0")(t), sp.Function("q1")(t)
L_A = (q0 * sp.diff(q1, t) - q1 * sp.diff(q0, t)) / 2 - w * (q0**2 + q1**2) / 2
E_A = euler_lagrange(L_A, [q0, q1], t)
say(f"Example A: E_0 = {E_A[0]},  E_1 = {E_A[1]}")
check(sp.simplify(E_A[0] - (sp.diff(q1, t) - w * q0)) == 0
      and sp.simplify(E_A[1] - (-sp.diff(q0, t) - w * q1)) == 0,
      "Example A: E_0 = q1' - omega q0, E_1 = -q0' - omega q1")
rotation = {q0: sp.cos(w * t), q1: sp.sin(w * t)}
check(all(sp.simplify(e.subs(rotation).doit()) == 0 for e in E_A),
      "Example A: q0 = cos(omega t), q1 = sin(omega t) solves both equations")

psi, psi_star = sp.Function("psi")(t), sp.Function("psistar")(t)
L_B = (sp.I / 2 * (psi_star * sp.diff(psi, t) - sp.diff(psi_star, t) * psi)
       - w * psi_star * psi)
E_B = euler_lagrange(L_B, [psi_star, psi], t)  # vary psi* first, then psi
say(f"Example B: E_psi* = {E_B[0]},  E_psi = {E_B[1]}")
complex_form = {psi: (q0 + sp.I * q1) / sp.sqrt(2), psi_star: (q0 - sp.I * q1)
                / sp.sqrt(2)}
L_B_real = (q1 * sp.diff(q0, t) - q0 * sp.diff(q1, t)) / 2 - w * (q0**2 + q1**2) / 2
check(sp.simplify(sp.expand(L_B.subs(complex_form).doit() - L_B_real)) == 0,
      "Example B in real variables is Example A with the rotation reversed")
E_real = euler_lagrange(L_B_real, [q0, q1], t)
combined = E_B[0].subs(complex_form).doit() - (E_real[0] + sp.I * E_real[1]) / sp.sqrt(2)
check(sp.simplify(sp.expand(combined)) == 0,
      "E_psi* = (E_q0 + i E_q1)/sqrt(2): varying psi* is varying q0 and q1")
check(sp.simplify(E_B[0] - (sp.I * sp.diff(psi, t) - w * psi)) == 0
      and sp.simplify(E_B[0].subs(psi, sp.exp(-sp.I * w * t)).doit()) == 0,
      "Example B: i psi' = omega psi, solved by psi = exp(-i omega t)")
```

Out [12]:

```
Example A: E_0 = -omega*q0(t) + Derivative(q1(t), t),  E_1 = -omega*q1(t) -
    Derivative(q0(t), t)
PASS Example A: E_0 = q1' - omega q0, E_1 = -q0' - omega q1
PASS Example A: q0 = cos(omega t), q1 = sin(omega t) solves both equations
Example B: E_psi* = -omega*psi(t) + I*Derivative(psi(t), t),  E_psi = -omega*psistar(t) -
    I*Derivative(psistar(t), t)
PASS Example B in real variables is Example A with the rotation reversed
PASS E_psi* = (E_q0 + i E_q1)/sqrt(2): varying psi* is varying q0 and q1
PASS Example B: i psi' = omega psi, solved by psi = exp(-i omega t)
```

Text cell:

```
Figure 6 draws the two motions in the $(q_0, q_1)$ plane for $\omega = 1$, starting
at $(1, 0)$: Example A turns counterclockwise, Example B clockwise; both stay on
```

```
the circle $q_0^2 + q_1^2 = 1$, because $|\psi|^2 = \frac12(q_0^2 + q_1^2)$ is
constant for $\psi = e^{-i\omega t}\psi(0)$.
```

In [13]:

```
turn = np.linspace(0.0, 1.5 * np.pi, 200)  # times up to three quarters of a turn
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.2))
for sense, label, style, colour in ((1.0, "Example A", "-", "tab:blue"),
                                    (-1.0, "Example B", "--", "tab:orange")):
    x, y = np.cos(turn), sense * np.sin(turn)
    left.plot(x, y, style, color=colour, label=label)
    for i in (30, 150):  # arrows that show the sense of the motion
        left.annotate("", xy=(x[i + 4], y[i + 4]), xytext=(x[i], y[i]),
                      arrowprops={"arrowstyle": "->", "lw": 1.5, "color": colour})
left.plot([1.0], [0.0], "ko")
left.text(1.05, 0.05, "start")
left.set_aspect("equal")
left.set_xlim(-1.4, 1.4)
left.set_ylim(-1.4, 1.4)
left.set_xlabel("$q_0$")
left.set_ylabel("$q_1$")
left.set_title("First-order motion: a rotation")
left.legend(fontsize=8, loc="lower left")
psi_values = np.exp(-1j * turn) / np.sqrt(2)  # psi(t) = exp(-i t) psi(0)
right.plot(turn, psi_values.real, label="real part of $\\psi$")
right.plot(turn, psi_values.imag, "--", label="imaginary part of $\\psi$")
right.plot(turn, np.abs(psi_values) ** 2, ":", color="black",
           label="$|\\psi|^2 = \\frac{1}{2}$")
right.set_xlabel("time $t$")
right.set_ylabel("value")
right.set_ylim(-0.85, 1.15)  # room for the legend above the curves
right.set_title("$\\psi(t) = e^{-it}\\psi(0)$")
right.legend(fontsize=7, loc="upper center", ncol=3)
fig.tight_layout()
save_figure(fig, "first_order_rotation",
            "First-order Lagrangians, with $\\omega = 1$. Left: the motion in the "
            "plane of the two real variables $(q_0, q_1)$, starting at $(1, 0)$ "
            "(dot), for three quarters of a turn: Example A (solid) turns "
            "counterclockwise, Example B, its complex form with the opposite sign "
            "of the kinetic term, turns clockwise (dashed); the arrows show the "
            "sense. Right: the complex variable $\\psi = (q_0 + iq_1)/\\sqrt{2}$ "
            "of Example B against the time $t$: real part $\\cos t/\\sqrt{2}$, "
            "imaginary part $-\\sin t/\\sqrt{2}$, and the constant $|\\psi|^2 = "
            "\\frac{1}{2}$.")
```

Out [13]:

```
Figure 07d.6 saved as Revision/textbook/figures/07d_6_first_order_rotation.png
```

Text cell:

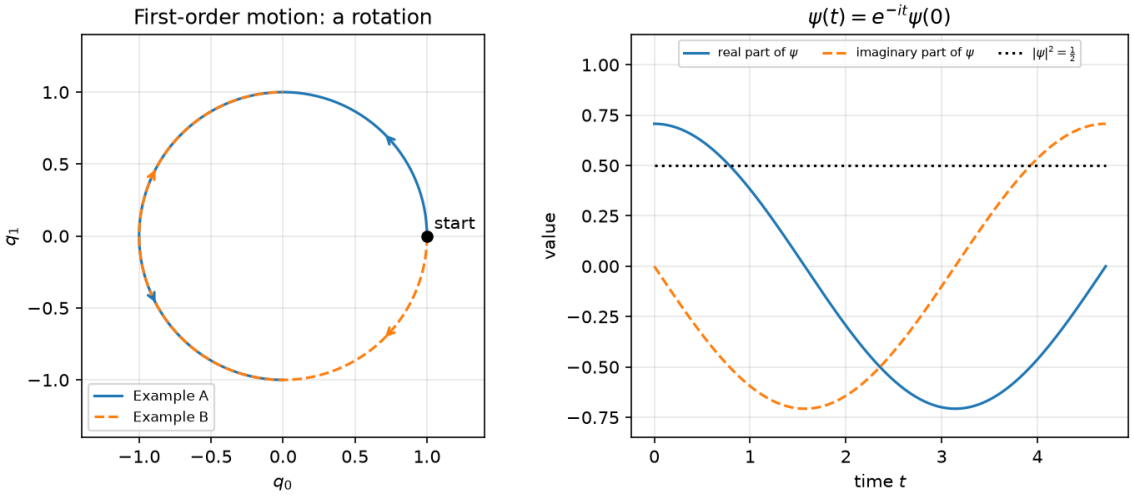

Figure 209: First-order Lagrangians, with $\omega = 1$. Left: the motion in the plane of the two real variables (q_0, q_1), starting at $(1, 0)$ (dot), for three quarters of a turn: Example A (solid) turns counterclockwise, Example B, its complex form with the opposite sign of the kinetic term, turns clockwise (dashed); the arrows show the sense. Right: the complex variable $\psi = (q_0 + iq_1)/\sqrt{2}$ of Example B against the time t: real part $\cos t/\sqrt{2}$, imaginary part $-\sin t/\sqrt{2}$, and the constant $|\psi|^2 = \frac{1}{2}$. (Notebook 07d, figure 6.)

```
## 13. A field of one space direction and time

A field $\phi(x_1, x_4)$ of a 3-space coordinate $x_1$ and the time $x_4$ has, in
the flat version of the author's signature ($\eta_{11} = +1$, $\eta_{44} = -1$),
the Lagrangian density

$$\mathcal{L} = -\frac12\big(\eta^{11}(\partial_1\phi)^2 + \eta^{44}(\partial_4
\phi)^2\big) - \frac12 m^2\phi^2 = \frac12(\partial_4\phi)^2 - \frac12(\partial_1
\phi)^2 - \frac12 m^2\phi^2,$$

kinetic minus potential energy again, with a gradient energy along $x_1$. The
Euler-Lagrange expression has one derivative term per coordinate, line by line:

$$E = \frac{\partial\mathcal{L}}{\partial\phi} - \partial_1\frac{\partial
\mathcal{L}}{\partial(\partial_1\phi)} - \partial_4\frac{\partial\mathcal{L}}
{\partial(\partial_4\phi)} = -m^2\phi - \partial_1(-\partial_1\phi) -
\partial_4(\partial_4\phi) = \partial_1^2\phi - \partial_4^2\phi - m^2\phi .$$

The first equality is the field version of section 4 (one integration by parts per
coordinate); the second inserts the three partial derivatives of $\mathcal{L}$;
the third simplifies. So $\partial_4^2\phi = \partial_1^2\phi - m^2\phi$, the wave
equation with a mass (the Klein-Gordon equation). A plane wave $\cos(kx_1)\cos(
\omega x_4)$ solves it exactly when $\omega^2 = m^2 + k^2$: it oscillates in time.

The next cell defines `field_euler_lagrange` (the same recipe as `euler_lagrange`,
with one derivative symbol per coordinate), checks the equation and the plane
wave, and checks the grid version: on a grid with steps $h_1$, $h_4$ the field
action is a sum over cells, and its derivative with respect to an inner value
```

```
$\phi_{ij}$ is $h_1h_4$ times the grid Euler-Lagrange expression (second difference
quotients in place of $\partial_1^2$ and $\partial_4^2$). The derivative is computed
numerically by the central difference $(S(\phi_{ij} + \delta) - S(\phi_{ij} -
\delta))/(2\delta)$, which is exact (up to rounding) for an action that is quadratic
in $\phi$, for a field of random values (fixed seed) on a grid of 7 by 6 points.
```

In [14]:

```
x1, x4, x5 = sp.symbols("x1 x4 x5", real=True)
m = sp.Symbol("m", positive=True)


def field_euler_lagrange(L, field, coordinates):
    """dL/dphi - sum over the coordinates of d/dx dL/d(dphi/dx)."""
    value = sp.Symbol("P")  # stands for phi
    slopes = [sp.Symbol(f"D{i}") for i in range(len(coordinates))]  # dphi/dx
    plain = L
    for x, slope in zip(coordinates, slopes):
        plain = plain.subs(sp.diff(field, x), slope)
    plain = plain.subs(field, value)
    back = {**{s: sp.diff(field, x) for x, s in zip(coordinates, slopes)},
            value: field}
    result = sp.diff(plain, value).subs(back)
    for x, slope in zip(coordinates, slopes):
        result -= sp.diff(sp.diff(plain, slope).subs(back), x)
    return sp.expand(result)


ETA = {x1: 1, x4: -1, x5: -1}  # the author's signs: x1 space-like, x4, x5 time-like
phi = sp.Function("phi")(x1, x4)
L_space = (-(ETA[x1] * sp.diff(phi, x1) ** 2 + ETA[x4] * sp.diff(phi, x4) ** 2) / 2
           - m**2 * phi**2 / 2)
E_space = field_euler_lagrange(L_space, phi, [x1, x4])
say(f"E = {E_space}")
check(sp.simplify(E_space - (sp.diff(phi, x1, 2) - sp.diff(phi, x4, 2)
                             - m**2 * phi)) == 0,
      "space and time: E = d1^2 phi - d4^2 phi - m^2 phi (Klein-Gordon)")
k, frequency = sp.symbols("k omega_k", positive=True)
wave = sp.cos(k * x1) * sp.cos(frequency * x4)
on_wave = E_space.subs(phi, wave).doit()
check(sp.simplify(on_wave.subs(frequency, sp.sqrt(m**2 + k**2))) == 0
      and sp.simplify(on_wave.subs(frequency, k)) != 0,
      "cos(k x1) cos(omega x4) solves it exactly when omega^2 = m^2 + k^2")

rng = np.random.default_rng(12345)  # random numbers with a fixed seed
h1, h4, mass = 0.3, 0.2, 1.5  # grid steps and the mass
field_values = rng.normal(size=(7, 6))  # phi_ij: i along x1, j along x4


def grid_action(f):
    """The field action on the grid: a sum over the cells."""
    d4 = (f[:-1, 1:] - f[:-1, :-1]) / h4  # time differences
    d1 = (f[1:, :-1] - f[:-1, :-1]) / h1  # space differences
```

```
    return h1 * h4 * np.sum(d4**2 / 2 - d1**2 / 2 - mass**2 * f[:-1, :-1] ** 2 / 2)


largest = 0.0
delta = 1e-3
for i in range(1, 6):
    for j in range(1, 5):
        plus, minus = field_values.copy(), field_values.copy()
        plus[i, j] += delta
        minus[i, j] -= delta
        numeric = (grid_action(plus) - grid_action(minus)) / (2 * delta)
        f_ = field_values  # a short name for the formula below
        expected = h1 * h4 * (
            (f_[i + 1, j] - 2 * f_[i, j] + f_[i - 1, j]) / h1**2  # space part
            - (f_[i, j + 1] - 2 * f_[i, j] + f_[i, j - 1]) / h4**2  # time part
            - mass**2 * f_[i, j])
        largest = max(largest, abs(numeric - expected))
say("largest difference over the 20 inner points is below 1e-9: "
    f"{largest < 1e-9}")
check(largest < 1e-9, "on the grid: dS/dphi_ij = h1 h4 times the grid "
      "Euler-Lagrange expression at every inner point")
```

Out [14]:

```
E = -m**2*phi(x1, x4) + Derivative(phi(x1, x4), (x1, 2)) - Derivative(phi(x1, x4), (x4,
    2))
PASS space and time: E = d1^2 phi - d4^2 phi - m^2 phi (Klein-Gordon)
PASS cos(k x1) cos(omega x4) solves it exactly when omega^2 = m^2 + k^2
largest difference over the 20 inner points is below 1e-9: True
PASS on the grid: dS/dphi_ij = h1 h4 times the grid Euler-Lagrange expression at every
    inner point
```

Text cell:

```
## 14. A field of one EXTRA time and time

Now replace the space coordinate $x_1$ by the extra time $x_5$. The author's
signature makes $x_5$ time-like, $\eta_{55} = -1$, and the same recipe gives

$$\mathcal{L} = \frac12(\partial_4\phi)^2 + \frac12(\partial_5\phi)^2 - \frac12
m^2\phi^2,\qquad E = -\partial_5^2\phi - \partial_4^2\phi - m^2\phi .$$

The only change is the sign of the $x_5$ term. Try $\phi = \cos(kx_5)\,P(x_4)$:
then $\partial_5^2\phi = -k^2\phi$, and $E = 0$ becomes

$$P'' = (k^2 - m^2)\,P .$$

For $k < m$ the solutions oscillate, but for $k > m$ they are $\cosh(\kappa x_4)$
and $\sinh(\kappa x_4)$ with $\kappa = \sqrt{k^2 - m^2}$: they GROW exponentially,
at a rate that becomes as large as we like when $k$ grows. A wave along a space
direction lowers nothing (its $\omega^2 = m^2 + k^2$); a wave along an extra time
lowers $\omega^2 = m^2 - k^2$ below zero.
```

The fields of this book obey a first-order equation; in flat space, multiplying it by the same operator once more gives this second-order equation for every one of the 16 components, because $(\gamma^{(a)}k_a)^2 = (\eta^{ab}k_ak_b)\,I_{16}$ (the Clifford relation). The Revision record finds the same growth rate for the 16-component field. The next cell checks the extra-time field equation and its growing solution, checks the Clifford square with the author's gammas, and reads the growth-rate formula from the Revision scope record and compares it, for a momentum along x_5 only, with $\kappa = \sqrt{k^2 - m^2}$.

In [15]:

```
phi5 = sp.Function("phi")(x5, x4)
L_extra = (-(ETA[x5] * sp.diff(phi5, x5) ** 2 + ETA[x4] * sp.diff(phi5, x4) ** 2) / 2
           - m**2 * phi5**2 / 2)
E_extra = field_euler_lagrange(L_extra, phi5, [x5, x4])
say(f"E = {E_extra}")
check(sp.simplify(E_extra - (-sp.diff(phi5, x5, 2) - sp.diff(phi5, x4, 2)
                             - m**2 * phi5)) == 0,
      "extra time and time: E = -d5^2 phi - d4^2 phi - m^2 phi")
kappa = sp.Symbol("kappa", positive=True)
growing = sp.cos(k * x5) * sp.cosh(kappa * x4)
check(sp.simplify(E_extra.subs(phi5, growing).doit()
                  .subs(kappa, sp.sqrt(k**2 - m**2))) == 0,
      "cos(k x5) cosh(kappa x4) solves it with kappa^2 = k^2 - m^2: growth for k > m")

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
G = [sp.Matrix(g) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7)
signs = [1, 1, 1, 1, -1, -1, -1, -1]  # eta in the order x0 .. x7
ks = sp.symbols("k0:8", real=True)  # a frame momentum with 8 components, k0 .. k7
slash = sum((ks[a] * G[a] for a in range(8)), sp.zeros(16, 16))  # gamma^(a) k_a
norm = sum(signs[a] * ks[a] ** 2 for a in range(8))  # eta^ab k_a k_b
check((slash * slash - norm * sp.eye(16)).applyfunc(sp.expand) == sp.zeros(16, 16),
      "(gamma^a k_a)^2 = (k0^2 + k1^2 + k2^2 + k3^2 - k4^2 - k5^2 - k6^2 - k7^2) I16",
      record=reproduces("Revision/theory/reports/python-field-theory.json",
                        "clifford_relations"))

SCOPE = "Revision/theory/reports/python-scope.json"
detail = revision_check(SCOPE, "extra_time_growth_rates_unbounded")["detail"]
formula = re.search(r"Im E = (sqrt\([^)]*\))", detail).group(1)  # the record's rate
say(f"growth rate in the record: {formula}")
K = sp.Symbol("K", positive=True)
rate = sp.sympify(formula, locals={"K": K, "m": m,
                                   **{f"k{i}": sp.Symbol(f"k{i}") for i in range(8)}})
along_x5 = rate.subs({sp.Symbol(f"k{i}"): 0 for i in (0, 1, 2, 3)}).subs(K, k)
check(sp.simplify(along_x5 - sp.sqrt(k**2 - m**2)) == 0,
      "the record's rate for a momentum along x5 only is sqrt(k^2 - m^2) = kappa",
      record=reproduces(SCOPE, "extra_time_growth_rates_unbounded"))
```

Out [15]:

```
E = -m**2*phi(x5, x4) - Derivative(phi(x5, x4), (x4, 2)) - Derivative(phi(x5, x4), (x5,
    2))
PASS extra time and time: E = -d5^2 phi - d4^2 phi - m^2 phi
PASS cos(k x5) cosh(kappa x4) solves it with kappa^2 = k^2 - m^2: growth for k > m
PASS (gamma^a k_a)^2 = (k0^2 + k1^2 + k2^2 + k3^2 - k4^2 - k5^2 - k6^2 - k7^2) I16
     reproduces Revision/theory/reports/python-field-theory.json, check
    clifford_relations
growth rate in the record: sqrt(K**2 - k0**2 - k1**2 - k2**2 - k3**2 - m**2)
PASS the record's rate for a momentum along x5 only is sqrt(k^2 - m^2) = kappa
     reproduces Revision/theory/reports/python-scope.json, check
    extra_time_growth_rates_unbounded
```

Text cell:

```
Figure 7 shows the two plane waves for $m = 1$ and $k = 2$ as maps over a
coordinate (horizontal) and the time $x_4$ (vertical): along the space direction
$x_1$ the field oscillates in time with $\omega = \sqrt5$; along the extra time
$x_5$ it grows like $\cosh(\sqrt3\,x_4)$. Figure 8 shows the dispersion relations
and the growth rate against the wave number.
```

In [16]:

```
coordinate = np.linspace(0.0, 2 * np.pi / 2.0, 121)  # one wavelength for k = 2
clock = np.linspace(0.0, 2.0, 101)  # the time x4 from 0 to 2
X, Y = np.meshgrid(coordinate, clock)
space_wave = np.cos(2.0 * X) * np.cos(np.sqrt(5.0) * Y)
extra_wave = np.cos(2.0 * X) * np.cosh(np.sqrt(3.0) * Y)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 4.2))
image = left.pcolormesh(X, Y, space_wave, cmap="coolwarm", vmin=-1, vmax=1,
                        shading="auto")
fig.colorbar(image, ax=left, shrink=0.85)
left.set_xlabel("space coordinate $x_1$")
left.set_ylabel("time $x_4$")
left.set_title("$\\cos(2x_1)\\cos(\\sqrt{5}x_4)$: oscillates")
limit = float(np.cosh(np.sqrt(3.0) * 2.0))
image = right.pcolormesh(X, Y, extra_wave, cmap="coolwarm", vmin=-limit, vmax=limit,
                         shading="auto")
fig.colorbar(image, ax=right, shrink=0.85)
right.set_xlabel("extra time $x_5$")
right.set_ylabel("time $x_4$")
right.set_title("$\\cos(2x_5)\\cosh(\\sqrt{3}x_4)$: grows")
for ax in (left, right):
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "space_and_extra_time",
            "Two exact solutions of the field equation with mass $m = 1$ and wave "
            "number $k = 2$, as colour maps over one wavelength of a coordinate "
            "(horizontal) and the time $x_4$ from 0 to 2 (vertical), pure numbers; "
            "red positive, blue negative. Left: a wave along the space direction "
            "$x_1$, $\\cos(2x_1)\\cos(\\sqrt{5}x_4)$, which oscillates in time "
            "between $-1$ and $1$. Right: a wave along the extra time $x_5$, "
            "$\\cos(2x_5)\\cosh(\\sqrt{3}x_4)$, which grows; its colour scale "
```

```
            "reaches $\\cosh(2\\sqrt{3}) \\approx 16$. The only difference in the "
            "Lagrangian is the sign of the term of the second coordinate.")

numbers = np.linspace(0.0, 3.0, 301)  # the wave number k, with m = 1
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(numbers, 1.0 + numbers**2, label="along a space direction: $m^2 + k^2$")
left.plot(numbers, 1.0 - numbers**2, "--",
          label="along an extra time: $m^2 - k^2$")
left.axhline(0.0, color="black", linewidth=0.8)
left.set_xlabel("wave number $k$")
left.set_ylabel("$\\omega^2$")
left.set_title("Dispersion relations ($m = 1$)")
left.legend(fontsize=8)
above = numbers[numbers > 1.0]
right.plot(above, np.sqrt(above**2 - 1.0), color="tab:red",
           label="$\\kappa = \\sqrt{k^2 - m^2}$")
right.plot(above, above, ":", color="grey", label="$\\kappa = k$ (large $k$)")
right.set_xlim(0.0, 3.0)
right.set_xlabel("wave number $k$ along the extra time")
right.set_ylabel("growth rate $\\kappa$")
right.set_title("Growth without upper bound")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "dispersion",
            "Left: the square $\\omega^2$ of the angular frequency of a plane wave "
            "against its wave number $k$ (pure numbers, mass $m = 1$), for a wave "
            "along a space direction ($m^2 + k^2$, always positive: oscillation) "
            "and along an extra time ($m^2 - k^2$, negative for $k > m$). Right: "
            "for $k > m$ the wave grows like $e^{\\kappa x_4}$ with the rate "
            "$\\kappa = \\sqrt{k^2 - m^2}$ (red), which approaches $k$ (dotted) and "
            "has no upper bound; the Revision record finds the same rate for the "
            "16-component field.")
```

Out [16]:

```
Figure 07d.7 saved as Revision/textbook/figures/07d_7_space_and_extra_time.png
```

Out [16]:

```
Figure 07d.8 saved as Revision/textbook/figures/07d_8_dispersion.png
```

Text cell:

```
## 15. The last check

The last cell checks that the eight figure files exist and prints the number of
checks that passed.
```

In [17]:

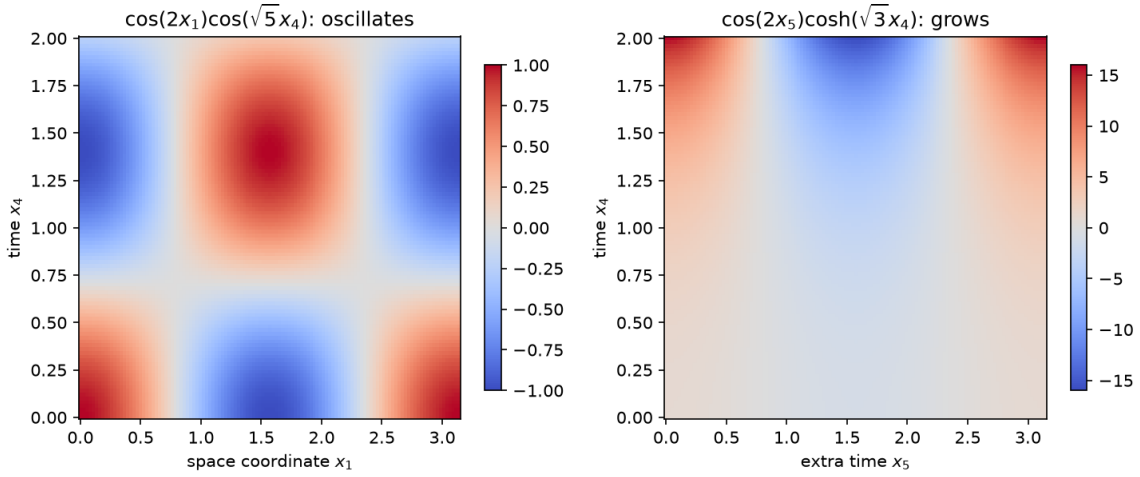

Figure 210: Two exact solutions of the field equation with mass $m = 1$ and wave number $k = 2$, as colour maps over one wavelength of a coordinate (horizontal) and the time x_4 from 0 to 2 (vertical), pure numbers; red positive, blue negative. Left: a wave along the space direction x_1, $\cos(2x_1)\cos(\sqrt{5}x_4)$, which oscillates in time between -1 and 1. Right: a wave along the extra time x_5, $\cos(2x_5)\cosh(\sqrt{3}x_4)$, which grows; its colour scale reaches $\cosh(2\sqrt{3}) \approx 16$. The only difference in the Lagrangian is the sign of the term of the second coordinate. (Notebook 07d, figure 7.)

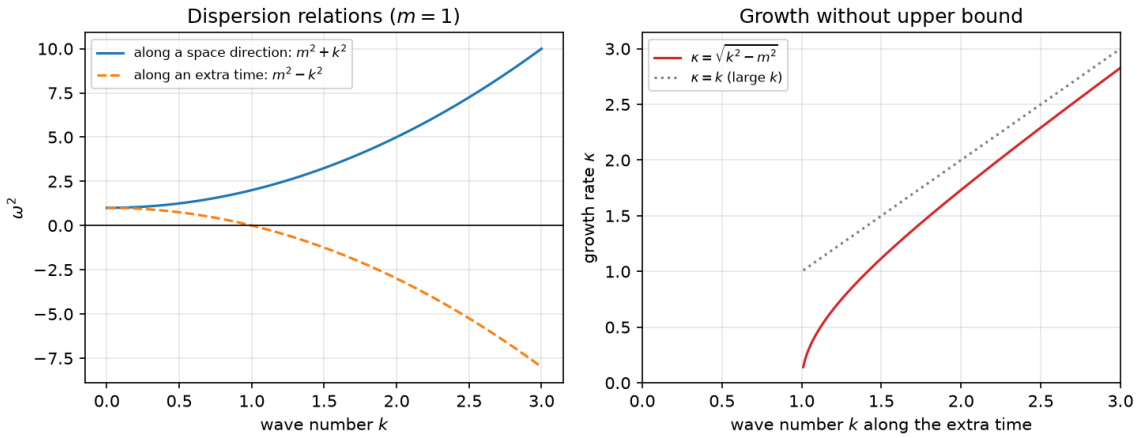

Figure 211: Left: the square ω^2 of the angular frequency of a plane wave against its wave number k (pure numbers, mass $m = 1$), for a wave along a space direction ($m^2 + k^2$, always positive: oscillation) and along an extra time ($m^2 - k^2$, negative for $k > m$). Right: for $k > m$ the wave grows like $e^{\kappa x_4}$ with the rate $\kappa = \sqrt{k^2 - m^2}$ (red), which approaches k (dotted) and has no upper bound; the Revision record finds the same rate for the 16-component field. (Notebook 07d, figure 8.)

```
names = ["action_versus_epsilon", "paths_and_residuals", "bump_lemma",
         "action_on_a_grid", "energy_along_paths", "first_order_rotation",
         "space_and_extra_time", "dispersion"]
files = [f"{FIGURE_FOLDER}/07d_{k_}_{name}.png" for k_, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 8 figure files of this notebook exist")
all_checks_passed()
```

Out [17]:

```
PASS all 8 figure files of this notebook exist
ALL 30 CHECKS PASSED (notebook 07d)
```

Text cell:

```
## 16. What this notebook showed

- The action $S[q] = \int L\,dt$ of a path changed by $\epsilon\xi$ is, for the
  oscillator, a quadratic polynomial in $\epsilon$; its slope at $\epsilon = 0$ is
  exactly zero for the true path $\sin t/\sin 1$ and not zero for the wrong paths
  $t$ and $t^2$. For short times the true path is a minimum, for times longer than
  $\pi/\omega$ only a saddle: the principle is one of STATIONARY action.
- Integration by parts turns the slope into $\int E\,\xi\,dt$ with the
  Euler-Lagrange expression $E = \partial L/\partial q - \frac{d}{dt}\partial L/
  \partial\dot q$, plus a boundary term that vanishes when $\xi$ vanishes at the
  ends; narrow bumps show that $\int E\xi\,dt = 0$ for all $\xi$ forces $E = 0$.
- On a grid of times the stationary action gives the grid Euler-Lagrange equation,
  and its solution approaches the true path with an error proportional to $h^2$.
- The energy $\frac12\dot q^2 + \frac12\omega^2q^2$ is constant exactly on the
  solutions ($dH/dt = -\dot q\,E$).
- A total derivative added to $L$ changes no equation, and a Lagrangian that is a
  total derivative gives none.
- First-order Lagrangians give first-order equations; a complex variable and its
  conjugate may be varied as if independent, and $\frac{i}{2}(\psi^*\dot\psi -
  \dot\psi^*\psi) - \omega\psi^*\psi$ gives $i\dot\psi = \omega\psi$: the structure
  of the Lagrangians of the fields of this book.
- For fields, one derivative term per coordinate appears; along a space direction
  the field oscillates ($\omega^2 = m^2 + k^2$), along an extra time it grows at the
  rate $\sqrt{k^2 - m^2}$, without upper bound: the same rate as in the Revision
  record for the 16-component field.
```

7.9 Line-by-line walk-through of Notebook 07d

The notebook has 17 code cells, In [1] to In [17]. This section explains every line of every one of them, in order. A line that starts with `#` is a **comment**: Python skips it; it is there for the reader. A text in triple quotes directly below a `def` line is a **docstring**, a description that Python stores but does not run. In the quoted code a line `...)` stands for the remaining lines of a long figure caption; every caption is printed in full under its figure in Section 7.8, and the paragraph "What Figure 07d.k shows" after the code says what to look for.

In [1], the set-up cell. Its first 237 lines are the complete run instructions of Section 7.7 again, as comment lines, so that the notebook file carries its own instructions. The code starts below the line THE SET-UP between two lines of = signs. This part is the same in every notebook of the book except for one line (the notebook's name); we explain it here once, and the walk-throughs of Notebooks 07a, 07b and 07c refer back to this explanation.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json` reads and writes the text format JSON, in which the Revision record stores its results; `os` gives access to the computer's environment; `textwrap` breaks long text into lines; `from pathlib import Path` takes the single name `Path` out of the module `pathlib`. A `Path` is the address of a file or a folder, written the same way on every operating system. The next three lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell.

```
NOTEBOOK_ID = "07d"  # this notebook: chapter 07, example d
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"07d"`. It is the only line of the set-up code that differs between notebooks.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. `Path.cwd()` is the folder in which the notebook runs and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with all of them. The `for` loop takes these folders one after the other; the operator `/` joins a folder and a name into a longer path; `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If none does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

`REPO` is the repository folder. `os.environ` holds the **environment variables** of the running program (named texts that the computer hands to it); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and otherwise the default, the repository folder written as a string. When you run the notebook the variable is not set, so the notebook writes into the repository; the book's checking tool sets it to a scratch folder, so that a check run does not change the repository.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` gives the full path of a repository file, for reading. `output_file("Revision/...")` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder that will hold it (and any missing folder above it) and does nothing if it exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters; `textwrap.fill` breaks it at blanks, and every line after the first starts with four blanks. `str(text)` turns any value into a string first.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to matplotlib's built-in settings, so that a personal settings file cannot change the figures. `plt.rcParams.update({...})` then sets, for every figure of the notebook, the size (7.0 by 4.2 inches), the size of the letters (10 points) and a faint grid (30 per cent opaque). The braces make a **dictionary**: pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
```

```
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/07d.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
```

`save_figure` is called after a figure has been drawn. `setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far; so the figures are numbered 1, 2, 3, ..., and a cell that is run twice keeps its numbers. The file name joins the notebook id, the number and the name, for example `07d_1_action_versus_epsilon.png`.

```
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`fig.savefig` writes the PNG file with 150 dots per inch, without an empty margin and without the program's name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory. The caption is stored, and the whole dictionary of captions is written into the captions file (`json.dumps` turns it into JSON text with sorted keys and one blank of indentation). `display(Image(...))` shows the saved picture below the cell; its `metadata` tells the book's tools which file it is. The last line prints where the figure was saved.

```
PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection, written with square brackets) that collects the names of the checks that pass. `check(condition, name, record=None)` is the notebook's test: `record=None` makes the third argument optional. If `condition` is false, `raise AssertionError(...)` stops the notebook with a message that names the failed check. An `if` statement is used instead of Python's `assert`, because `python -O` would skip an `assert`. Otherwise the name is appended to `PASSED` and a line `PASS <name>` is printed; when the check reproduces a Revision record, a second line `reproduces <record>` follows.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

`report` prints a key number as a line `RESULT <label> = <value> <unit>`; the expression `(f" {unit}" if unit else "")` adds the unit only when one is given. `all_checks_passed` prints the last line of the notebook with the number of passed checks, `len(PASSED)`.

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

The cell ends by printing one line. Two strings written next to each other inside the parentheses are joined into one by Python. **Out [1]:** `Set-up of notebook 07d complete:  repository folder found, helpers defined.`

In [2]: helpers, and the first variation of nine varied paths.

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import re  # regular expressions: read a formula from a check's detail text

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
```

Three modules of Python: `contextlib` can send printed text somewhere else, `io` provides an in-memory text file, `re` reads patterns in text (**regular expressions**). Then the two packages of the computation: numpy (arrays of numbers, under the short name `np`) and sympy (exact algebra with symbols, `sp`).

```
check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece
```

The first line keeps the set-up cell's `check` under a second name. The new `check` of the same name calls the old one while `contextlib.redirect_stdout(buffer)` sends everything it prints into the in-memory file `buffer`; then it prints the collected text with one single `print` (`end=""` adds no extra line end, because the collected text already ends with one). The reason: Jupyter delivers printed text in pieces, and the book's tools must see a PASS line and the "reproduces" line that may follow it as one piece.

```
def revision_check(report, name):
    """The check called name of a Revision report (a JSON file): its dictionary
    (name, verdict, detail); stops if it is missing or its verdict is not PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return found[0]


def reproduces(report, name):
    """The text "<report>, check <name>" after making sure the check passes."""
    revision_check(report, name)
    return f"{report}, check {name}"
```

`revision_check(report, name)` reads a Revision report (a JSON file) with `json.loads` and returns the entry of the check called `name` (a dictionary with the keys name, verdict and detail). The square brackets `[item for item in data["checks"] if ...]` form a **list comprehension**: the list of all entries whose name is `name`. If there is not exactly one, or its verdict, written in capitals with `.upper()`, is not PASS, the notebook stops with a `ValueError`. `reproduces(repor t, name)` makes sure that the check passes and returns the text that a PASS line prints after the word reproduces.

```
t, eps = sp.symbols("t epsilon", real=True)  # the time and the size of the change
omega, T_end = sp.Integer(1), sp.Integer(1)  # omega = 1 and the end time T = 1
```

```
def lagrangian(q, q_dot):
    """The oscillator: kinetic energy minus potential energy."""
    return q_dot**2 / 2 - omega**2 * q**2 / 2
```

`sp.symbols("t epsilon", real=True)` makes two sympy symbols, the time t and the size ϵ of the change, both real numbers. `sp.Integer(1)` is the exact number 1; here it is $\omega = 1$ and the end time $T = 1$ (called `T_end`, because the name `T` is used later for a symbol). The function `lagrangian(q, q_dot)` returns $\frac{1}{2}\dot{q}^2 - \frac{1}{2}\omega^2 q^2$; in Python `**` is the power and `/` the division.

```
PATHS = {"true path": sp.sin(t) / sp.sin(1), "line": t, "parabola": t**2}
SHAPES = {n: sp.sin(n * sp.pi * t) for n in (1, 2, 3)}  # xi_n, zero at t = 0, 1
```

`PATHS` is a dictionary from the names of the three paths to their formulas: the true path $\sin t/\sin 1$, the line t and the parabola t^2. `SHAPES` is a **dictionary comprehension**: for $n = 1, 2, 3$ it stores the shape $\xi_n = \sin(n\pi t)$ under the key n; `sp.pi` is the exact number π.

```
def action_polynomial(path, shape):
    """S[path + eps shape] as a polynomial in eps (exact)."""
    varied = path + eps * shape
    integrand = sp.expand(lagrangian(varied, sp.diff(varied, t)))
    return sp.Poly(sp.integrate(integrand, (t, 0, T_end)), eps)
```

`action_polynomial(path, shape)` forms the varied path $q + \epsilon\xi$, differentiates it with `sp.diff(varied, t)`, puts both into the Lagrangian, multiplies out with `sp.expand`, integrates exactly from 0 to T with `sp.integrate(integrand, (t, 0, T_end))` and returns the result as a polynomial in ϵ (`sp.Poly(..., eps)`), the $S_0 + S_1\epsilon + S_2\epsilon^2$ of Section 7.2.

```
S1 = {}  # the first variations, S1[(path name, n)]
S2 = {}  # the coefficients of eps^2
for name, path in PATHS.items():
    for n, shape in SHAPES.items():
        polynomial = action_polynomial(path, shape)
        S1[(name, n)] = sp.simplify(polynomial.coeff_monomial(eps))
        S2[(name, n)] = sp.simplify(polynomial.coeff_monomial(eps**2))
        say(f"{name:9s} n = {n}: S1 = {S1[(name, n)]} = "
            f"{float(S1[(name, n)]):+.6f}")
```

Two empty dictionaries collect the coefficients. The two nested loops go over the three paths and the three shapes (`PATHS.items()` gives the pairs name, formula). For each of the nine combinations the polynomial is computed; `coeff_monomial(eps)` takes the coefficient of ϵ and `coeff_monomial(eps**2)` that of ϵ^2, and `sp.simplify` brings each to its simplest form. The key `(name, n)` is a **tuple**, a fixed pair of values. The `say` line prints the name in a field of 9 characters (`{name:9s}`), the exact coefficient, and its decimal value with sign and six decimals (`{...:+.6f}`; `float` turns the exact number into a decimal one). **Out [2]:** nine lines. For the true path $S_1 = 0$ for all three shapes; for the line $-1/\pi$, $1/(2\pi)$ and $-1/(3\pi)$; for the parabola $-5/\pi + 4/\pi^3 = -1.462543$, $1/(2\pi)$ and $(4 - 45\pi^2)/(27\pi^3)$. The value $-1/\pi = -0.318310$ is the one computed by hand in Section 7.2.

In [3]: the checks of the table.

```
check(all(S1[("true path", n)] == 0 for n in SHAPES),
      "the true path: the first variation is exactly 0 for xi_1, xi_2, xi_3")
check(all(S1[(name, n)] != 0 for name in ("line", "parabola") for n in SHAPES),
      "the line and the parabola: the first variation is not 0")
check(all(S2[(name, n)] == S2[("true path", n)] for name in PATHS for n in SHAPES),
      "the coefficient of eps^2 is the same for every path")
for n in SHAPES:
    coefficient = S2[("true path", n)]  # the same for every path
    report(f"S2 for xi_{n}", f"{coefficient} = {float(coefficient):.6f}")
```

The first check uses `all(...)` over the three shapes: it is true when $S_1 = 0$ exactly for every shape of the true path. The second checks $S_1 \neq 0$ for all six combinations of a wrong path and a shape (a **generator expression** with two `for` parts goes over all of them). The third checks that S_2 is the same for every path, as Section 7.2 proved. The loop prints S_2 for each shape, exactly and to six decimals. **Out [3]:** three PASS lines and `RESULT S2 for xi_1 = -1/4 + pi**2/4 = 2.217401`, and the same for ξ_2 (9.619604) and ξ_3 (21.956610).

In [4]: minimum or saddle, and Figure 07d.1.

```
T = sp.Symbol("T", positive=True)
xi_T = sp.sin(sp.pi * t / T)  # the shape xi_1 on the interval from 0 to T
S2_general = sp.integrate(sp.diff(xi_T, t) ** 2 / 2 - omega**2 * xi_T**2 / 2,
                          (t, 0, T))
say(f"S2(T) = {sp.simplify(S2_general)}")
check(sp.simplify(S2_general - T / 4 * (sp.pi**2 / T**2 - omega**2)) == 0,
      "S2 = (T/4)(pi^2/T^2 - omega^2) for the shape xi_1 on (0, T)")
check(S2_general.subs(T, 1) > 0 and S2_general.subs(T, 4) < 0,
      "S2 > 0 for T = 1 (a minimum), S2 < 0 for T = 4 > pi (a saddle)")
```

`sp.Symbol("T", positive=True)` is the length of the interval as a positive symbol. `xi_T` is the shape $\sin(\pi t/T)$, and `S2_general` the exact integral $\int_0^T (\frac{1}{2}\dot{\xi}^2 - \frac{1}{2}\omega^2\xi^2)\,dt$. The `say` line prints it (sympy writes it as `(-T**2 + pi**2)/(4*T)`, which is $\frac{T}{4}(\frac{\pi^2}{T^2} - 1)$ with $\omega = 1$). The first check compares it with the formula of Section 7.2: the difference simplifies to 0. The second evaluates it with `.subs(T, 1)` and `.subs(T, 4)` and checks the signs: positive at $T = 1$ (minimum), negative at $T = 4 > \pi$ (saddle).

```
epsilons = np.linspace(-0.6, 0.6, 121)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
for name, style in (("true path", "-"), ("line", "--"), ("parabola", ":")):
    polynomial = action_polynomial(PATHS[name], SHAPES[1])
    values = sp.lambdify(eps, polynomial.as_expr() - polynomial.coeff_monomial(1))
    left.plot(epsilons, values(epsilons), style, label=name)
    slope = float(S1[(name, 1)])
    left.plot(epsilons, slope * epsilons, style, color="grey", linewidth=0.8)
left.axvline(0.0, color="black", linewidth=0.8)
left.set_xlabel("size of the change $\\epsilon$")
left.set_ylabel("$S[q + \\epsilon\\xi_1] - S[q]$")
left.set_title("Only the true path has slope 0")
left.legend(fontsize=8)
```

`np.linspace(-0.6, 0.6, 121)` is 121 equally spaced values of ϵ. `plt.subplots(1, 2, figsize=(9.8, 3.9))` makes a figure with one row of two panels, `left` and `right`. For each path with its line style (solid `"-"`, dashed, written as two minus signs, and dotted `":"`), the polynomial with the shape ξ_1 is computed again; `polynomial.as_expr() - polynomial.coeff_monomial(1)` is $S - S_0 = S_1\epsilon + S_2\epsilon^2$; `sp.lambdify(eps, ...)` turns this formula into a numerical function, which `values(epsilons)` evaluates at all 121 values at once. The curve is drawn with its label, and its tangent at $\epsilon = 0$, the straight line $S_1\epsilon$, is drawn thin and grey. `axvline(0.0)` draws a vertical line at $\epsilon = 0$. The axis labels use LaTeX inside `$...$`; in a Python string a backslash is written twice. `legend` draws the key of the line styles.

```
lengths = np.linspace(0.3, 6.0, 300)
S2_values = sp.lambdify(T, S2_general)(lengths)
right.plot(lengths, S2_values)
right.axhline(0.0, color="black", linewidth=0.8)
right.axvline(np.pi, color="red", linestyle="--", label="$T = \\pi/\\omega$")
right.set_ylim(-2.0, 4.0)
right.set_xlabel("length of the time interval $T$")
right.set_ylabel("coefficient $S_2$ of $\\epsilon^2$")
right.set_title("Minimum for $T < \\pi$, saddle for $T > \\pi$")
right.legend(fontsize=8)
fig.tight_layout()
```

The right panel: 300 interval lengths from 0.3 to 6.0, the coefficient $S_2(T)$ evaluated on them, a black line at 0 and a red dashed vertical line at $T = \pi$ (`np.pi`), the place where S_2 changes sign. `set_ylim(-2.0, 4.0)` fixes the vertical range. `fig.tight_layout()` arranges the panels so that nothing overlaps.

```
save_figure(fig, "action_versus_epsilon",
            "Left: the change of the action of the oscillator ($\\omega = 1$, from "
            ...)
```

`save_figure` saves and shows the figure with its caption. **Out [4]:** `S2(T) = (-T**2 + pi**2)/(4*T)`, two PASS lines, the figure and the line `Figure 07d.1 saved as Revision/textbook/figures/07d_1_action_versus_epsilon.png`.

What Figure 07d.1 shows. In the left panel the solid curve of the true path touches its horizontal tangent at $\epsilon = 0$: changing the true path a little changes its action only to second order. The dashed and dotted curves of the line and the parabola cross $\epsilon = 0$ with a slope (their grey tangents are tilted): their action changes to first order, so they are not stationary. All three curves are parabolas of the same shape, shifted, because S_2 is the same for every path. In the right panel S_2 is positive for short intervals and changes sign at $T = \pi$ (red line): beyond it the true path is a saddle, not a minimum.

In [5]: the Euler-Lagrange expression and the integration by parts.

```
from sympy.calculus.euler import euler_equations  # sympy's own Euler-Lagrange
```

This line imports sympy's own function `euler_equations`, used below as an independent comparison.

```
def euler_lagrange(L, functions, variable):
    """The Euler-Lagrange expressions dL/dq - d/dt dL/d(q') of the Lagrangian L
    for each function q of the list functions of the variable."""
    values = [sp.Symbol(f"Q{i}") for i in range(len(functions))]  # stand for q
    speeds = [sp.Symbol(f"V{i}") for i in range(len(functions))]  # stand for q'
    plain = L
    for function, speed in zip(functions, speeds):
        plain = plain.subs(sp.diff(function, variable), speed)  # q' -> V first
    for function, value in zip(functions, values):
        plain = plain.subs(function, value)  # then q -> Q
    back = {**{s: sp.diff(f, variable) for f, s in zip(functions, speeds)},
            **{v: f for f, v in zip(functions, values)}}  # Q -> q, V -> q'
    return [sp.expand(sp.diff(plain, value).subs(back)
                      - sp.diff(sp.diff(plain, speed).subs(back), variable))
            for value, speed in zip(values, speeds)]
```

`euler_lagrange(L, functions, variable)` computes $\partial L/\partial q - \frac{d}{dt}\partial L/\partial \dot{q}$ for each function q in the list `functions`, exactly as Section 7.3 says. A partial derivative with respect to q must hold $\dot{q}$ fixed, but in sympy $q(t)$ and its derivative are tied together. So the function first replaces them by plain symbols: `values` are symbols `Q0`, `Q1`, ... standing for the functions and `speeds` are `V0`, `V1`, ... standing for their derivatives (`range(len(functions))` counts the functions; `zip` walks through two lists side by side). The first loop replaces each derivative $\dot{q}$ by its speed symbol (this must come first, because $\dot{q}$ contains q); the second replaces each q by its value symbol. The dictionary `back` undoes both replacements (the notation `{**a, **b}` merges two dictionaries). The returned list holds, for each function, the derivative of the plain Lagrangian with respect to its value symbol, put back into functions, minus the time derivative of the derivative with respect to its speed symbol, also put back; `sp.expand` multiplies out.

```
q = sp.Function("q")(t)  # a general path
E_general = euler_lagrange(lagrangian(q, sp.diff(q, t)), [q], t)[0]
say(f"E for the oscillator: {E_general}")
sympy_E = euler_equations(lagrangian(q, sp.diff(q, t)), [q], t)[0]
check(sp.simplify(E_general - (sympy_E.lhs - sympy_E.rhs)) == 0
      and sp.simplify(E_general + sp.diff(q, t, 2) + omega**2 * q) == 0,
      "euler_lagrange gives E = -q'' - omega^2 q, as sympy's euler_equations")
```

`sp.Function("q")(t)` is an unknown function $q(t)$, a general path. `E_general` is its Euler-Lagrange expression for the oscillator. `euler_equations` returns sympy's own equation in the form left side = right side (`lhs`, `rhs`); the check requires that the two results agree and that `E_general` equals $-\ddot{q} - \omega^2 q$ (`sp.diff(q, t, 2)` is the second derivative).

```
def E_of(path):
    """E(t) of a given path (insert the path into E_general)."""
    return sp.simplify(E_general.subs(q, path).doit())


for name, path in PATHS.items():
    say(f"{name:9s}: E(t) = {E_of(path)}")
check(E_of(PATHS["true path"]) == 0, "the true path solves E = 0")
```

`E_of(path)` puts a given path into `E_general` (`.subs(q, path)` replaces the unknown function, `.doit()` carries out the derivatives) and simplifies. The loop prints $E(t)$ for the three paths, and the check confirms that the true path gives exactly 0.

```
by_parts = all(sp.simplify(S1[(name, n)]
                           - sp.integrate(E_of(PATHS[name]) * SHAPES[n], (t, 0, 1)))
               == 0 for name in ("line", "parabola") for n in SHAPES)
check(by_parts, "S1 = integral of E xi_n from 0 to 1 for both wrong paths and n = 1, "
      "2, 3")
```

`by_parts` is true when, for both wrong paths and all three shapes, the first variation S_1 of In [2] equals the exact integral $\int_0^1 E\,\xi_n\,dt$: Line 3 of Section 7.3 for shapes that vanish at both ends.

```
boundary_ok = True
xi_open = t  # a shape with xi(0) = 0 but xi(1) = 1
for name in ("line", "parabola"):
    path = PATHS[name]
    first = action_polynomial(path, xi_open).coeff_monomial(eps)  # its S1
    inside = sp.integrate(E_of(path) * xi_open, (t, 0, 1))
    p_xi = sp.diff(path, t) * xi_open  # (dL/dq') xi = q' xi for the oscillator
    boundary = p_xi.subs(t, 1) - p_xi.subs(t, 0)  # [q' xi] from 0 to 1
    say(f"{name}, xi = t: S1 = {sp.simplify(first)}, integral = "
        f"{sp.simplify(inside)}, boundary term = {boundary}")
    boundary_ok = boundary_ok and sp.simplify(first - inside - boundary) == 0
check(boundary_ok, "for xi = t (not zero at t = 1): S1 = integral of E xi + "
      "boundary term q'(1) xi(1)")
```

Now the shape $\xi = t$, which is 1 at $t = 1$. For each wrong path: `first` is its S_1 (the coefficient of ϵ), `inside` is $\int_0^1 E\xi\,dt$, `p_xi` is $\dot{q}\,\xi$ (because $\partial L/\partial\dot{q} = \dot{q}$ for the oscillator), and `boundary` is $[\dot{q}\,\xi]_0^1$, the value at 1 minus the value at 0. The line printed shows the three numbers; `boundary_ok` stays true only if S_1 equals the integral plus the boundary term for both paths. **Out [5]:** `E for th e oscillator:  -q(t) - Derivative(q(t), (t, 2))`, the expressions 0, $-t$ and $-t^2 - 2$ of the three paths, the lines `line, xi = t:  S1 = 2/3, integral = -1/3, boundary term = 1` and `parabola, xi = t:  S1 = 3/4, integral = -5/4, boundary term = 2`, and four PASS lines.

In [6]: Figure 07d.2.

```
times = np.linspace(0.0, 1.0, 201)
styles = {"true path": "-", "line": "--", "parabola": ":"}
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
for name, path in PATHS.items():
    left.plot(times, sp.lambdify(t, path)(times) * np.ones_like(times),
              styles[name], label=name)
    residual = sp.lambdify(t, E_of(path))(times) * np.ones_like(times)
    right.plot(times, residual, styles[name], label=name)
left.set_xlabel("time $t$")
left.set_ylabel("position $q(t)$")
left.set_title("Three paths with $q(0) = 0$, $q(1) = 1$")
left.legend(fontsize=8)
right.axhline(0.0, color="black", linewidth=0.8)
```

```
right.set_xlabel("time $t$")
right.set_ylabel("$E(t) = -\\ddot q - q$")
right.set_title("Euler-Lagrange expression")
right.legend(fontsize=8)
fig.tight_layout()
```

201 times from 0 to 1 and a dictionary of line styles. For each path the left panel draws $q(t)$ and the right panel $E(t)$; `sp.lambdify(t, ...)` makes numerical functions of the formulas. The factor `np.ones_like(times)`, an array of ones of the same length, turns a constant result (the 0 of the true path, which `lambdify` returns as the single number 0) into an array that can be drawn. Labels, titles, a black zero line and the legends follow.

```
save_figure(fig, "paths_and_residuals",
            "Left: three paths of the oscillator with the same end values $q(0) = "
            ...)
```

Out [6]: the figure and its saved file `07d_2_paths_and_residuals.png`. **What Figure 07d.2 shows.** The three paths in the left panel look alike; nothing in their shapes shows which one is the true motion. The right panel tells them apart at once: $E(t)$, the amount by which Newton's law $\ddot q = -q$ fails at time t, is zero at every time only for the true path, $-t$ for the line and $-2 - t^2$ for the parabola.

In [7]: narrow bumps and Figure 07d.3.

```
grid = np.linspace(0.0, 1.0, 200001)  # fine grid for the integrals


def E_parabola(times_):
    return -2.0 - times_**2  # E(t) of the parabola path


def bump(times_, centre, width):
    """cos^2 bump of half-width width around centre, zero outside."""
    inside = np.abs(times_ - centre) < width
    return np.where(inside, np.cos(np.pi * (times_ - centre) / (2 * width)) ** 2, 0.0)
```

`grid` is 200001 equally spaced times from 0 to 1, fine enough that the integrals below are accurate to far more digits than are printed. `E_parabola` is $E(t) = -2 - t^2$ of the parabola path, for an array of times. `bump(times_, centre, width)` is the shape $\cos^2(\pi(t - t_0)/(2w))$ inside $|t - t_0| < w$ and 0 outside: `inside` is an array of true and false values, and `np.where(inside, a, b)` takes `a` where it is true and `b` elsewhere.

```
widths = [0.2, 0.1, 0.05, 0.025]
centres = [0.3, 0.5, 0.7]
errors = {}
for centre in centres:
    errors[centre] = []
    for width in widths:
        weight = bump(grid, centre, width)
```

```
        average = (np.trapezoid(E_parabola(grid) * weight, grid)
                   / np.trapezoid(weight, grid))
        errors[centre].append(average - E_parabola(centre))
    ratios = [errors[centre][i] / errors[centre][i + 1] for i in range(3)]
    say(f"t0 = {centre}: errors " + ", ".join(f"{e:+.3e}" for e in errors[centre])
        + "; ratios " + ", ".join(f"{r:.4f}" for r in ratios))
check(all(abs(errors[c][i] / errors[c][i + 1] - 4.0) < 1e-6
          for c in centres for i in range(3)),
      "the average of E over the bump tends to E(t0); halving w divides the error by 4")
```

Four half-widths and three centres. For each centre and width, `weight` is the bump on the grid and `average` is $\int E\xi\,dt / \int \xi\,dt$; `np.trapezoid(f, grid)` computes an integral with the trapezoidal rule (Chapter 2). The error, the average minus $E(t_0)$, is appended to a list. `ratios` divides each error by the next one (half the width). The `say` line prints the errors in **scientific notation** (`{e:+.3e}` gives three decimals and a power of ten) and the ratios with four decimals; `", ".join(...)` joins the texts with commas. The check requires that every ratio equals 4 within 10^{-6}. **Out [7]:** for each of the three centres the errors -5.228×10^{-3}, -1.307×10^{-3}, -3.267×10^{-4}, -8.168×10^{-5} and the ratios 4.0000, 4.0000, 4.0000, then the PASS line. These are exactly $-0.130691\,w^2$ (Section 7.3), the same for every centre because $E'' = -2$ is the same everywhere.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(times, E_parabola(times), color="black", label="$E(t) = -2 - t^2$")
for width, marker in zip(widths[:3], ["o", "s", "^"]):
    points = np.linspace(0.25, 0.75, 6)
    averages = [np.trapezoid(E_parabola(grid) * bump(grid, c, width), grid)
                / np.trapezoid(bump(grid, c, width), grid) for c in points]
    left.plot(points, averages, marker, label=f"bump average, $w = {width}$")
shown = bump(times, 0.5, 0.2)
left.fill_between(times, -2.9, -2.9 + 0.5 * shown, color="tab:green", alpha=0.3,
                  label="a bump ($t_0 = 0.5$, $w = 0.2$)")
left.set_ylim(-3.0, -1.8)
left.set_xlabel("time $t$ (bump centre $t_0$)")
left.set_ylabel("value")
left.set_title("Averages of $E$ over bumps")
left.legend(fontsize=7, loc="upper right")
```

The left panel draws $E(t)$ in black and, for the three larger widths with three marker shapes (`zip(widths[:3], ["o", "s", "^"])` pairs each width with circles, squares or triangles), the bump averages at six centres between 0.25 and 0.75. One bump ($t_0 = 0.5$, $w = 0.2$) is shown as a green shaded area near the bottom: `fill_between(times, -2.9, -2.9 + 0.5 * shown, ...)` fills the area between the level -2.9 and a curve that rises by half the bump's height, `alpha=0.3` makes it transparent.

```
for centre, marker in zip(centres, ["o", "s", "^"]):
    right.loglog(widths, np.abs(errors[centre]), marker, label=f"$t_0 = {centre}$")
reference = np.abs(errors[0.5][0]) * (np.array(widths) / widths[0]) ** 2
right.loglog(widths, reference, "--", color="grey", label="slope 2")
right.set_xticks(widths, [str(width) for width in widths])  # plain tick labels
right.minorticks_off()  # no unlabelled minor ticks
right.set_xlabel("half-width $w$ of the bump")
```

```
right.set_ylabel("|average of $E$ minus $E(t_0)$|")
right.set_title("The error falls like $w^2$")
right.legend(fontsize=8)
fig.tight_layout()
```

The right panel draws the size of the errors (`np.abs`) against the width on **logarithmic axes** (`loglog`), one marker shape per centre, and a grey dashed reference line proportional to w^2, which on these axes is a straight line of slope 2. `set_xticks` writes the four widths as plain numbers under the axis, and `minorticks_off()` removes the unlabelled small ticks.

```
save_figure(fig, "bump_lemma",
            "The fundamental lemma made visible, for the parabola path with $E(t) = "
            ...)
```

What Figure 07d.3 shows. In the left panel the markers move onto the black curve as the bumps narrow: averaging E over a narrower and narrower bump recovers its value at the centre. In the right panel the points of all three centres lie on one line of slope 2: the distance falls like w^2. So if $\int E\xi\,dt$ vanished for every bump, every average would be 0, and so would E at every point: the fundamental lemma.

In [8]: the derivative of the grid action.

```
N5 = 5
h, w_sym = sp.symbols("h omega", positive=True)
qs = sp.symbols(f"q0:{N5 + 1}", real=True)  # q_0 .. q_5
S_grid = sum(h * (((qs[n + 1] - qs[n]) / h) ** 2 / 2
                  - w_sym**2 * (qs[n] ** 2 + qs[n + 1] ** 2) / 4) for n in range(N5))
grid_E = [h * (-(qs[n + 1] - 2 * qs[n] + qs[n - 1]) / h**2 - w_sym**2 * qs[n])
          for n in range(1, N5)]
check(all(sp.simplify(sp.diff(S_grid, qs[n]) - grid_E[n - 1]) == 0
          for n in range(1, N5)),
      "dS_N/dq_n = h (-(q_(n+1) - 2 q_n + q_(n-1))/h^2 - omega^2 q_n) for every "
      "inner n")
```

$N = 5$ steps. `h` and `w_sym` are positive symbols for the step and for ω; `sp.symbols(f"q0:{N5 + 1}", real=True)` makes the six symbols `q0` to `q5` (the notation `q0:6` means "q0 up to but not including q6"). `S_grid` is the grid action S_N of Section 7.3, a sum over the five steps. `grid_E` is the list of the expected derivatives $h[-(q_{n+1} - 2q_n + q_{n-1})/h^2 - \omega^2 q_n]$ for the inner points $n = 1, \ldots, 4$. The check differentiates `S_grid` with respect to each inner value and compares. **Out [8]:** one PASS line.

In [9]: solving the grid equations, and Figure 07d.4.

```
def grid_path(N):
    """Solve the grid equations: returns the grid times and q_0 .. q_N."""
    step = 1.0 / N
    A = np.zeros((N - 1, N - 1))  # one row per inner point n = 1 .. N-1
    b = np.zeros(N - 1)
    for i in range(N - 1):
```

```
        A[i, i] = 2.0 / step**2 - 1.0  # from -(-2 q_n)/h^2 - omega^2 q_n
        if i > 0:
            A[i, i - 1] = -1.0 / step**2  # the neighbour q_(n-1)
        if i < N - 2:
            A[i, i + 1] = -1.0 / step**2  # the neighbour q_(n+1)
    b[-1] = 1.0 / step**2  # the known end value q_N = 1 moved to the right side
    inner = np.linalg.solve(A, b)
    return np.linspace(0.0, 1.0, N + 1), np.concatenate([[0.0], inner, [1.0]])
```

`grid_path(N)` solves the $N-1$ linear equations "the bracket is zero at every inner point" for $\omega = 1$, $T = 1$, $q_0 = 0$, $q_N = 1$. Multiplying the bracket by -1, the equation at the inner point n is $-q_{n-1}/h^2 + (2/h^2 - 1)q_n - q_{n+1}/h^2 = 0$. The matrix `A` holds these coefficients: row `i` belongs to the inner point $n = i + 1$; its diagonal entry is $2/h^2 - 1$ and its neighbours $-1/h^2$ (the `if` lines leave out neighbours that are end points). The known end value $q_N = 1$ appears only in the last row, as $-1/h^2$ times 1; moved to the right side it gives `b[-1] = 1.0 / step**2` (the index `-1` means the last entry). `np.linalg.solve(A, b)` solves the equations, and the function returns the $N+1$ grid times and the values with the two end values added in front and behind (`np.concatenate`).

```
sizes = [4, 8, 16, 32, 64, 128, 256]
grid_errors = []
for N in sizes:
    nodes, values = grid_path(N)
    grid_errors.append(np.abs(values - np.sin(nodes) / np.sin(1.0)).max())
orders = [np.log2(grid_errors[i] / grid_errors[i + 1]) for i in range(len(sizes) - 1)]
say("largest errors: " + ", ".join(f"N = {N}: {e:.3e}"
                                   for N, e in zip(sizes, grid_errors)))
say("observed orders: " + ", ".join(f"{p:.3f}" for p in orders))
check(all(1.9 < p < 2.1 for p in orders),
      "the stationary grid path converges to the true path with order 2")
```

For $N = 4, 8, \ldots, 256$ the largest distance between the grid path and the true path $\sin t / \sin 1$ at the grid times is stored. `np.log2(e_N/e_{2N})` is the observed order: if the error is Ch^p, halving h divides it by 2^p, and the logarithm to base 2 of the ratio is p (Chapter 2). The check requires every observed order between 1.9 and 2.1. **Out [9]:** the errors 4.089×10^{-4} ($N = 4$), 1.017×10^{-4}, 2.582×10^{-5}, 6.452×10^{-6}, 1.613×10^{-6}, 4.032×10^{-7}, 1.008×10^{-7} ($N = 256$), the orders 2.007, 1.978, 2.001, 2.000, 2.000, 2.000 and the PASS line.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(times, np.sin(times) / np.sin(1.0), color="black", label="true path")
for N, marker in ((4, "o"), (8, "s")):
    nodes, values = grid_path(N)
    left.plot(nodes, values, marker, label=f"stationary grid path, $N = {N}$")
left.set_xlabel("time $t$")
left.set_ylabel("position $q$")
left.set_title("The action on a grid")
left.legend(fontsize=8)
steps = 1.0 / np.array(sizes)
right.loglog(steps, grid_errors, "o-", label="largest error")
right.loglog(steps, grid_errors[0] * (steps / steps[0]) ** 2, "--", color="grey",
             label="slope 2")
```

```
right.set_xlabel("step $h = 1/N$")
right.set_ylabel("largest difference from the true path")
right.set_title("Second-order convergence")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "action_on_a_grid",
...)
```

The left panel draws the true path as a black line and, for $N = 4$ (circles) and $N = 8$ (squares), the points of the stationary grid path, computed again with `grid_path`. For the right panel `steps` is the array of the steps $h = 1/N$ (`np.array` turns the list of sizes into an array, so that `1.0 /` divides every entry); the largest errors are drawn against the steps on logarithmic axes, as dots joined by lines, and the grey dashed reference line is the first error times $(h/h_0)^2$, a line of slope 2 through the first point. Labels, titles and legends follow, and `save_figure` saves the figure.

What Figure 07d.4 shows. Already four steps give a grid path whose points lie on the true path to the eye. On the right the errors lie on a line parallel to the slope-2 guide: halving the step divides the error by 4. So the principle of stationary action can be used directly on a computer, and the grid solution converges to the true motion.

In [10]: the energy, and Figure 07d.5.

```
def energy(path):
    """H = (1/2) q'^2 + (1/2) omega^2 q^2 of a path."""
    return sp.diff(path, t) ** 2 / 2 + omega**2 * path**2 / 2


check(sp.simplify(sp.diff(energy(q), t) + sp.diff(q, t) * E_general) == 0,
      "dH/dt = -q' E for every path")
for name, path in PATHS.items():
    say(f"{name:9s}: dH/dt = {sp.simplify(sp.diff(energy(path), t))}")
check(sp.simplify(sp.diff(energy(PATHS["true path"]), t)) == 0
      and all(sp.simplify(sp.diff(energy(PATHS[name]), t)) != 0
              for name in ("line", "parabola")),
      "H is constant on the true path and changes on the two wrong paths")
true_energy = sp.simplify(energy(PATHS["true path"]))
report("energy of the true path", f"{true_energy} = {float(true_energy):.6f}")
```

`energy(path)` is $H = \frac{1}{2}\dot{q}^2 + \frac{1}{2}\omega^2 q^2$. The first check is the identity $dH/dt = -\dot{q}\,E$ of Section 7.4 for a general path (the sum $dH/dt + \dot{q}E$ simplifies to 0). The loop prints dH/dt on the three paths; the second check requires 0 for the true path and a nonzero result for the two wrong ones. `report` prints the energy of the true path exactly and as a decimal. **Out [10]:** two PASS lines, `true path:  dH/dt = 0`, `line:  dH/dt = t`, `parabola:  dH/dt = 2*t*(t**2 + 2)` and `RESULT energy of the true path = 1/(2*sin(1)**2) = 0.706141`.

```
fig, ax = plt.subplots()
for name, path in PATHS.items():
    values = sp.lambdify(t, energy(path))(times) * np.ones_like(times)
    ax.plot(times, values, styles[name], label=name)
ax.set_xlabel("time $t$")
ax.set_ylabel("$H = \\frac{1}{2}\\dot q^2 + \\frac{1}{2}q^2$")
```

```
ax.set_title("The energy along the three paths")
ax.legend(fontsize=8)
save_figure(fig, "energy_along_paths",
...)
```

One panel: the energy along each path against the time. **What Figure 07d.5 shows.** The solid line of the true path is flat at $1/(2\sin^2 1) \approx 0.706$; the energies of the line and the parabola change, because their E is not zero.

In [11]: a total derivative changes nothing.

```
F = t * q**3  # F(q, t)
L = lagrangian(q, sp.diff(q, t))
E_changed = euler_lagrange(L + sp.diff(F, t), [q], t)[0]
check(sp.simplify(E_changed - E_general) == 0,
      "L + dF/dt with F = t q^3 has the same Euler-Lagrange expression as L")
E_pure = euler_lagrange(sp.diff(q**2, t), [q], t)[0]
check(E_pure == 0, "the pure total derivative d(q^2)/dt gives E = 0 for every path")
differences = [sp.simplify(sp.integrate(sp.diff(F.subs(q, path), t), (t, 0, 1)))
               for path in PATHS.values()]
say(f"S'[path] - S[path] for the three paths: {differences}")
check(all(d == 1 for d in differences),
      "the action changes by F(q(1), 1) - F(q(0), 0) = 1 for every path")
```

`F` is $F(q,t) = t\,q^3$ for the general path `q` of In [5]. `E_changed` is the Euler-Lagrange expression of $L + dF/dt$ (`sp.diff(F, t)` is the total derivative along the path, because `q` is a function of `t`); the first check requires it to equal that of L. `E_pure` is the expression of the pure total derivative $\frac{d}{dt}q^2$; the second check requires it to be 0. `differences` holds, for each of the three paths, the change $\int_0^1 \frac{dF}{dt}\,dt$ of the action; it is printed, and the third check requires it to be 1 for every path. **Out [11]:** three PASS lines and `S'[path] - S[path] for the three paths:  [1, 1, 1]`.

In [12]: the two first-order Lagrangians.

```
w = sp.Symbol("omega", positive=True)
q0, q1 = sp.Function("q0")(t), sp.Function("q1")(t)
L_A = (q0 * sp.diff(q1, t) - q1 * sp.diff(q0, t)) / 2 - w * (q0**2 + q1**2) / 2
E_A = euler_lagrange(L_A, [q0, q1], t)
say(f"Example A: E_0 = {E_A[0]},  E_1 = {E_A[1]}")
check(sp.simplify(E_A[0] - (sp.diff(q1, t) - w * q0)) == 0
      and sp.simplify(E_A[1] - (-sp.diff(q0, t) - w * q1)) == 0,
      "Example A: E_0 = q1' - omega q0, E_1 = -q0' - omega q1")
rotation = {q0: sp.cos(w * t), q1: sp.sin(w * t)}
check(all(sp.simplify(e.subs(rotation).doit()) == 0 for e in E_A),
      "Example A: q0 = cos(omega t), q1 = sin(omega t) solves both equations")
```

`w` is ω as a positive symbol and `q0`, `q1` are two unknown functions of t. `L_A` is Example A of Section 7.4, and `E_A` the list of its two Euler-Lagrange expressions (the function `euler_lagrange` of In [5] works for several functions at once). The first check compares them with $\dot q_1 - \omega q_0$ and $-\dot q_0 - \omega q_1$, the second puts in the rotation $q_0 = \cos\omega t$, $q_1 = \sin\omega t$ and requires both to vanish.

```
psi, psi_star = sp.Function("psi")(t), sp.Function("psistar")(t)
L_B = (sp.I / 2 * (psi_star * sp.diff(psi, t) - sp.diff(psi_star, t) * psi)
       - w * psi_star * psi)
E_B = euler_lagrange(L_B, [psi_star, psi], t)  # vary psi* first, then psi
say(f"Example B: E_psi* = {E_B[0]},  E_psi = {E_B[1]}")
complex_form = {psi: (q0 + sp.I * q1) / sp.sqrt(2), psi_star: (q0 - sp.I * q1)
                / sp.sqrt(2)}
L_B_real = (q1 * sp.diff(q0, t) - q0 * sp.diff(q1, t)) / 2 - w * (q0**2 + q1**2) / 2
check(sp.simplify(sp.expand(L_B.subs(complex_form).doit() - L_B_real)) == 0,
      "Example B in real variables is Example A with the rotation reversed")
```

`psi` and `psi_star` are two unknown functions ψ and ψ^*, varied as independent functions; `sp.I` is the imaginary unit i. `L_B` is Example B. `E_B` lists the expressions for ψ^* (first) and ψ. `complex_form` is the dictionary that writes ψ and ψ^* in terms of q_0 and q_1, and `L_B_real` is the real form of Section 7.4; the check requires `L_B` with the substitution to equal it.

```
E_real = euler_lagrange(L_B_real, [q0, q1], t)
combined = E_B[0].subs(complex_form).doit() - (E_real[0] + sp.I * E_real[1]) / sp.sqrt(2)
check(sp.simplify(sp.expand(combined)) == 0,
      "E_psi* = (E_q0 + i E_q1)/sqrt(2): varying psi* is varying q0 and q1")
check(sp.simplify(E_B[0] - (sp.I * sp.diff(psi, t) - w * psi)) == 0
      and sp.simplify(E_B[0].subs(psi, sp.exp(-sp.I * w * t)).doit()) == 0,
      "Example B: i psi' = omega psi, solved by psi = exp(-i omega t)")
```

`E_real` are the expressions of the real form. `combined` is E_{ψ^*} written in q_0, q_1 minus $(E_{q_0} + iE_{q_1})/\sqrt{2}$; the check requires it to vanish: varying ψ^* is the same as varying q_0 and q_1. The last check compares E_{ψ^*} with $i\dot{\psi} - \omega\psi$ and puts in $\psi = e^{-i\omega t}$. **Out [12]:** the four expressions and five PASS lines.

In [13]: Figure 07d.6.

```
turn = np.linspace(0.0, 1.5 * np.pi, 200)  # times up to three quarters of a turn
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.2))
for sense, label, style, colour in ((1.0, "Example A", "-", "tab:blue"),
                                    (-1.0, "Example B", "--", "tab:orange")):
    x, y = np.cos(turn), sense * np.sin(turn)
    left.plot(x, y, style, color=colour, label=label)
    for i in (30, 150):  # arrows that show the sense of the motion
        left.annotate("", xy=(x[i + 4], y[i + 4]), xytext=(x[i], y[i]),
                      arrowprops={"arrowstyle": "->", "lw": 1.5, "color": colour})
left.plot([1.0], [0.0], "ko")
left.text(1.05, 0.05, "start")
left.set_aspect("equal")
left.set_xlim(-1.4, 1.4)
left.set_ylim(-1.4, 1.4)
left.set_xlabel("$q_0$")
left.set_ylabel("$q_1$")
left.set_title("First-order motion: a rotation")
left.legend(fontsize=8, loc="lower left")
```

`turn` holds 200 times from 0 to 1.5π, three quarters of a turn for $\omega = 1$. The loop draws both motions in the plane (q_0, q_1): `sense` is $+1$ for Example A ($q_1 = \sin t$) and -1 for Example B ($q_1 = -\sin t$).

For each, two arrows show the sense of motion: `annotate("", xy=..., xytext=...)` draws an arrow from the point at index `i` to the point four indices later, with the arrow style given in `arrowprops`. A black dot marks the start $(1, 0)$; `set_aspect("equal")` makes one unit equally long on both axes, so the circle looks round.

```
psi_values = np.exp(-1j * turn) / np.sqrt(2)  # psi(t) = exp(-i t) psi(0)
right.plot(turn, psi_values.real, label="real part of $\\psi$")
right.plot(turn, psi_values.imag, "--", label="imaginary part of $\\psi$")
right.plot(turn, np.abs(psi_values) ** 2, ":", color="black",
           label="$|\\psi|^2 = \\frac{1}{2}$")
right.set_xlabel("time $t$")
right.set_ylabel("value")
right.set_ylim(-0.85, 1.15)  # room for the legend above the curves
right.set_title("$\\psi(t) = e^{-it}\\psi(0)$")
right.legend(fontsize=7, loc="upper center", ncol=3)
fig.tight_layout()
save_figure(fig, "first_order_rotation",
...)
```

The right panel draws $\psi(t) = e^{-it}/\sqrt{2}$ (`1j` is Python's imaginary unit; `.real` and `.imag` are the real and imaginary parts) and $|\psi|^2$. **What Figure 07d.6 shows.** Both first-order motions run on the unit circle; Example A turns counterclockwise and Example B, whose kinetic term has the opposite sign, clockwise. On the right the real and imaginary parts of ψ oscillate a quarter period apart while $|\psi|^2 = \frac{1}{2}$ stays constant.

In [14]: a field of a space direction and time.

```
x1, x4, x5 = sp.symbols("x1 x4 x5", real=True)
m = sp.Symbol("m", positive=True)
```

Three real coordinate symbols x_1, x_4, x_5 and the mass m as a positive symbol.

```
def field_euler_lagrange(L, field, coordinates):
    """dL/dphi - sum over the coordinates of d/dx dL/d(dphi/dx)."""
    value = sp.Symbol("P")  # stands for phi
    slopes = [sp.Symbol(f"D{i}") for i in range(len(coordinates))]  # dphi/dx
    plain = L
    for x, slope in zip(coordinates, slopes):
        plain = plain.subs(sp.diff(field, x), slope)
    plain = plain.subs(field, value)
    back = {**{s: sp.diff(field, x) for x, s in zip(coordinates, slopes)},
            value: field}
    result = sp.diff(plain, value).subs(back)
    for x, slope in zip(coordinates, slopes):
        result -= sp.diff(sp.diff(plain, slope).subs(back), x)
    return sp.expand(result)
```

`field_euler_lagrange(L, field, coordinates)` is the recipe of Section 7.5: like `euler_lagrange` of In [5], it replaces the field's derivatives by the symbols `D0`, `D1`, ... (one per coordinate) and then the field by `P`; it differentiates with respect to `P`, puts the functions back, and subtracts

for each coordinate the derivative along that coordinate of the derivative with respect to its slope symbol.

```
ETA = {x1: 1, x4: -1, x5: -1}  # the author's signs: x1 space-like, x4, x5 time-like
phi = sp.Function("phi")(x1, x4)
L_space = (-(ETA[x1] * sp.diff(phi, x1) ** 2 + ETA[x4] * sp.diff(phi, x4) ** 2) / 2
           - m**2 * phi**2 / 2)
E_space = field_euler_lagrange(L_space, phi, [x1, x4])
say(f"E = {E_space}")
check(sp.simplify(E_space - (sp.diff(phi, x1, 2) - sp.diff(phi, x4, 2)
                             - m**2 * phi)) == 0,
      "space and time: E = d1^2 phi - d4^2 phi - m^2 phi (Klein-Gordon)")
k, frequency = sp.symbols("k omega_k", positive=True)
wave = sp.cos(k * x1) * sp.cos(frequency * x4)
on_wave = E_space.subs(phi, wave).doit()
check(sp.simplify(on_wave.subs(frequency, sp.sqrt(m**2 + k**2))) == 0
      and sp.simplify(on_wave.subs(frequency, k)) != 0,
      "cos(k x1) cos(omega x4) solves it exactly when omega^2 = m^2 + k^2")
```

`ETA` holds the author's signs for the three coordinates used: $+1$ for x_1, -1 for x_4 and x_5. `phi` is an unknown field $\phi(x_1, x_4)$, and `L_space` the Lagrangian density of Section 7.5. The first check compares the Euler-Lagrange expression with $\partial_1^2\phi - \partial_4^2\phi - m^2\phi$. For the plane wave $\cos(kx_1)\cos(\omega x_4)$, the second check requires $E = 0$ when $\omega = \sqrt{m^2 + k^2}$ and $E \neq 0$ for the wrong value $\omega = k$.

```
rng = np.random.default_rng(12345)  # random numbers with a fixed seed
h1, h4, mass = 0.3, 0.2, 1.5  # grid steps and the mass
field_values = rng.normal(size=(7, 6))  # phi_ij: i along x1, j along x4


def grid_action(f):
    """The field action on the grid: a sum over the cells."""
    d4 = (f[:-1, 1:] - f[:-1, :-1]) / h4  # time differences
    d1 = (f[1:, :-1] - f[:-1, :-1]) / h1  # space differences
    return h1 * h4 * np.sum(d4**2 / 2 - d1**2 / 2 - mass**2 * f[:-1, :-1] ** 2 / 2)
```

Now the same on a grid of 7 by 6 points with steps $h_1 = 0.3$ and $h_4 = 0.2$ and mass 1.5. `rng` is a random-number generator with a fixed **seed** 12345, so that every run draws the same numbers; `rng.normal(size=(7, 6))` gives a table of random field values ϕ_{ij}. `grid_action(f)` is the action on the grid: in each cell the time difference `d4` and the space difference `d1` (array slices such as `f[:-1, 1:]` take all rows but the last and all columns but the first), and the sum of $h_1 h_4[\frac{1}{2}d_4^2 - \frac{1}{2}d_1^2 - \frac{1}{2}m^2\phi^2]$.

```
largest = 0.0
delta = 1e-3
for i in range(1, 6):
    for j in range(1, 5):
        plus, minus = field_values.copy(), field_values.copy()
        plus[i, j] += delta
        minus[i, j] -= delta
        numeric = (grid_action(plus) - grid_action(minus)) / (2 * delta)
        f_ = field_values  # a short name for the formula below
```

```
        expected = h1 * h4 * (
            (f_[i + 1, j] - 2 * f_[i, j] + f_[i - 1, j]) / h1**2  # space part
            - (f_[i, j + 1] - 2 * f_[i, j] + f_[i, j - 1]) / h4**2  # time part
            - mass**2 * f_[i, j])
        largest = max(largest, abs(numeric - expected))
say("largest difference over the 20 inner points is below 1e-9: "
    f"{largest < 1e-9}")
check(largest < 1e-9, "on the grid: dS/dphi_ij = h1 h4 times the grid "
      "Euler-Lagrange expression at every inner point")
```

For each of the 20 inner points, the derivative of the grid action with respect to ϕ_{ij} is computed numerically by the **central difference** $(S(\phi_{ij}+\delta) - S(\phi_{ij}-\delta))/(2\delta)$ with $\delta = 10^{-3}$ (`.copy()` makes independent copies of the table, so that only one entry is changed). For an action that is quadratic in ϕ this is exact up to rounding. `expected` is $h_1 h_4$ times the grid Euler-Lagrange expression (second difference quotients in place of ∂_1^2 and ∂_4^2). The largest difference is printed as a true or false statement and checked to be below 10^{-9}. **Out [14]:** the expression `E = -m**2*phi(x1, x4) + Derivative(...)  - Derivative(...)`, the PASS lines and `largest difference over the 20 inner points is below 1e-9:  True`.

In [15]: a field of an extra time and time, and the record's growth rate.

```
phi5 = sp.Function("phi")(x5, x4)
L_extra = (-(ETA[x5] * sp.diff(phi5, x5) ** 2 + ETA[x4] * sp.diff(phi5, x4) ** 2) / 2
           - m**2 * phi5**2 / 2)
E_extra = field_euler_lagrange(L_extra, phi5, [x5, x4])
say(f"E = {E_extra}")
check(sp.simplify(E_extra - (-sp.diff(phi5, x5, 2) - sp.diff(phi5, x4, 2)
                             - m**2 * phi5)) == 0,
      "extra time and time: E = -d5^2 phi - d4^2 phi - m^2 phi")
kappa = sp.Symbol("kappa", positive=True)
growing = sp.cos(k * x5) * sp.cosh(kappa * x4)
check(sp.simplify(E_extra.subs(phi5, growing).doit()
                  .subs(kappa, sp.sqrt(k**2 - m**2))) == 0,
      "cos(k x5) cosh(kappa x4) solves it with kappa^2 = k^2 - m^2: growth for k > m")
```

`phi5` is an unknown field $\phi(x_5, x_4)$ and `L_extra` its Lagrangian density with the sign $\eta_{55} = -1$. The first check compares the expression with $-\partial_5^2\phi - \partial_4^2\phi - m^2\phi$; the second puts in $\cos(kx_5)\cosh(\kappa x_4)$ and requires $E = 0$ when $\kappa = \sqrt{k^2 - m^2}$.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
G = [sp.Matrix(g) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7)
signs = [1, 1, 1, 1, -1, -1, -1, -1]  # eta in the order x0 .. x7
ks = sp.symbols("k0:8", real=True)  # a frame momentum with 8 components, k0 .. k7
slash = sum((ks[a] * G[a] for a in range(8)), sp.zeros(16, 16))  # gamma^(a) k_a
norm = sum(signs[a] * ks[a] ** 2 for a in range(8))  # eta^ab k_a k_b
check((slash * slash - norm * sp.eye(16)).applyfunc(sp.expand) == sp.zeros(16, 16),
      "(gamma^a k_a)^2 = (k0^2 + k1^2 + k2^2 + k3^2 - k4^2 - k5^2 - k6^2 - k7^2) I16",
      record=reproduces("Revision/theory/reports/python-field-theory.json",
                        "clifford_relations"))
```

The notebook now reads the author's gammas from the Revision record `Revision/algebra/gammas.json` (the key `"gamma"` holds the eight 16×16 integer matrices in the order $x_0, \ldots, x_7$) and turns them into exact sympy matrices. `signs` is η in the same order, `ks` eight real symbols $k_0, \ldots, k_7$. `slash` is $\sum_a k_a \gamma^{(a)}$ (the sum starts from the zero matrix `sp.zeros(16, 16)`), and `norm` is $\sum_a \eta^{aa} k_a^2$. The check multiplies out `slash * slash - norm * I16` entry by entry (`applyfunc(sp.expand)`) and requires the zero matrix; its second line names the Revision check it reproduces.

```
SCOPE = "Revision/theory/reports/python-scope.json"
detail = revision_check(SCOPE, "extra_time_growth_rates_unbounded")["detail"]
formula = re.search(r"Im E = (sqrt\([^)]*\))", detail).group(1)  # the record's rate
say(f"growth rate in the record: {formula}")
K = sp.Symbol("K", positive=True)
rate = sp.sympify(formula, locals={"K": K, "m": m,
                                   **{f"k{i}": sp.Symbol(f"k{i}") for i in range(8)}})
along_x5 = rate.subs({sp.Symbol(f"k{i}"): 0 for i in (0, 1, 2, 3)}).subs(K, k)
check(sp.simplify(along_x5 - sp.sqrt(k**2 - m**2)) == 0,
      "the record's rate for a momentum along x5 only is sqrt(k^2 - m^2) = kappa",
      record=reproduces(SCOPE, "extra_time_growth_rates_unbounded"))
```

`detail` is the detail text of the scope record's check `extra_time_growth_rates_unbounded`. `re.search(r"Im E = (sqrt\([^)]*\))", detail)` finds in it the text that follows "Im E = ", a `sqrt(...)` up to the first closing parenthesis; `.group(1)` is that text, printed as `sqrt(K**2 - k0**2 - k1**2 - k2**2 - k3**2 - m**2)` ($k_0, \ldots, k_3$ are the momenta along the space-like directions $x_0, \ldots, x_3$, K the size of the momentum along the extra times). `sp.sympify` turns it into a formula (the dictionary `locals` tells sympy which names are which symbols). Setting $k_0 = k_1 = k_2 = k_3 = 0$ and $K = k$ gives the rate for a momentum along x_5 alone, and the check compares it with $\sqrt{k^2 - m^2}$. **Out [15]:** the expression, four PASS lines, two of them with the "reproduces" line, and the formula of the record.

In [16]: Figures 07d.7 and 07d.8.

```
coordinate = np.linspace(0.0, 2 * np.pi / 2.0, 121)  # one wavelength for k = 2
clock = np.linspace(0.0, 2.0, 101)  # the time x4 from 0 to 2
X, Y = np.meshgrid(coordinate, clock)
space_wave = np.cos(2.0 * X) * np.cos(np.sqrt(5.0) * Y)
extra_wave = np.cos(2.0 * X) * np.cosh(np.sqrt(3.0) * Y)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 4.2))
image = left.pcolormesh(X, Y, space_wave, cmap="coolwarm", vmin=-1, vmax=1,
                        shading="auto")
fig.colorbar(image, ax=left, shrink=0.85)
left.set_xlabel("space coordinate $x_1$")
left.set_ylabel("time $x_4$")
left.set_title("$\\cos(2x_1)\\cos(\\sqrt{5}x_4)$: oscillates")
limit = float(np.cosh(np.sqrt(3.0) * 2.0))
image = right.pcolormesh(X, Y, extra_wave, cmap="coolwarm", vmin=-limit, vmax=limit,
                         shading="auto")
fig.colorbar(image, ax=right, shrink=0.85)
right.set_xlabel("extra time $x_5$")
right.set_ylabel("time $x_4$")
right.set_title("$\\cos(2x_5)\\cosh(\\sqrt{3}x_4)$: grows")
for ax in (left, right):
    ax.grid(False)
```

`coordinate` is one wavelength $2\pi/k = \pi$ of a coordinate for $k = 2$, `clock` the time from 0 to 2. `np.meshgrid` makes two tables `X`, `Y` that hold the coordinate and the time at every point of a 121 by 101 grid. `space_wave` is $\cos(2x_1)\cos(\sqrt{5}x_4)$ (for $m = 1$, $k = 2$: $\omega = \sqrt{1+4}$), `extra_wave` is $\cos(2x_5)\cosh(\sqrt{3}x_4)$ ($\kappa = \sqrt{4-1}$). `pcolormesh` draws each table as a colour map (red positive, blue negative in the colour scheme `coolwarm`); `colorbar` adds the colour scale. The right panel's scale reaches $\cosh(2\sqrt{3}) \approx 16$ (`limit`). `ax.grid(False)` switches the grid lines off over the colour maps.

```
fig.tight_layout()
save_figure(fig, "space_and_extra_time",
            "Two exact solutions of the field equation with mass $m = 1$ and wave "
            ...)
```

What Figure 07d.7 shows. On the left the colours repeat in time: the wave along the space direction oscillates between -1 and 1. On the right the colours grow stronger with time: the wave along the extra time grows like $\cosh(\sqrt{3}x_4)$. The two Lagrangians differ only by the sign of one term.

```
numbers = np.linspace(0.0, 3.0, 301)  # the wave number k, with m = 1
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(numbers, 1.0 + numbers**2, label="along a space direction: $m^2 + k^2$")
left.plot(numbers, 1.0 - numbers**2, "--",
          label="along an extra time: $m^2 - k^2$")
left.axhline(0.0, color="black", linewidth=0.8)
left.set_xlabel("wave number $k$")
left.set_ylabel("$\\omega^2$")
left.set_title("Dispersion relations ($m = 1$)")
left.legend(fontsize=8)
above = numbers[numbers > 1.0]
right.plot(above, np.sqrt(above**2 - 1.0), color="tab:red",
           label="$\\kappa = \\sqrt{k^2 - m^2}$")
right.plot(above, above, ":", color="grey", label="$\\kappa = k$ (large $k$)")
right.set_xlim(0.0, 3.0)
right.set_xlabel("wave number $k$ along the extra time")
right.set_ylabel("growth rate $\\kappa$")
right.set_title("Growth without upper bound")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "dispersion",
...)
```

The second figure: on the left $\omega^2 = 1 + k^2$ and $\omega^2 = 1 - k^2$ against the wave number k from 0 to 3; on the right, for $k > 1$ (`numbers[numbers > 1.0]` keeps only those values), the growth rate $\kappa = \sqrt{k^2 - 1}$ and the dotted line $\kappa = k$ that it approaches. **Out [16]:** the two figures and their saved files.

What Figure 07d.8 shows. Along a space direction ω^2 is always positive (oscillation); along an extra time it falls below zero for $k > m$, and then the wave grows at the rate κ, which keeps growing with k and has no upper bound: the same rate as in the Revision record for the 16-component field.

In [17]: the last check.

```
names = ["action_versus_epsilon", "paths_and_residuals", "bump_lemma",
         "action_on_a_grid", "energy_along_paths", "first_order_rotation",
         "space_and_extra_time", "dispersion"]
files = [f"{FIGURE_FOLDER}/07d_{k_}_{name}.png" for k_, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 8 figure files of this notebook exist")
all_checks_passed()
```

The list of the eight figure names, and the list of their file paths built with `enumerate(names, 1)`, which numbers them from 1. The check requires that every file exists in the output folder; `all_checks_passed()` prints the last line. **Out [17]:** `PASS all 8 figure files of this notebook exist` and `ALL 30 CHECKS PASSED (notebook 07d)`.

7.10 Grassmann numbers from zero

Why anticommuting numbers. Electrons, and the quanta of the field dirac16complex, are **fermions**: two of them never occupy the same state (the Pauli principle). In the quantum theory this is expressed by field operators that **anticommute**, $XY = -YX$ (Chapter 10). The classical field from which such operators are obtained is therefore given values that anticommute: the Revision record defines the components of dirac16complex in this way (ASSUMED here as the definition of the field; Revision/SPEC.md, section 3). Such values are not measured; they are bookkeeping symbols with exact rules, and every statement about them is a statement about these rules.

Generators and the defining rule. Take n symbols $\theta_0, \theta_1, \ldots, \theta_{n-1}$, called **generators**. The one rule is

$$\theta_i \theta_j = -\theta_j \theta_i \qquad \text{for all } i, j.$$

Ordinary numbers are allowed as factors and commute with everything. Setting $j = i$ in the rule gives $\theta_i\theta_i = -\theta_i\theta_i$, that is $2\theta_i\theta_i = 0$ (add $\theta_i\theta_i$ to both sides), so

$$\theta_i^2 = 0.$$

A generator times itself is zero. This is the algebraic form of the Pauli principle.

Monomials and the sign of a reordering. A product of generators can be put into increasing order of the indices by exchanging neighbours that stand in the wrong order, one pair at a time; every exchange costs a factor -1 by the defining rule. Example: $\theta_2\theta_0\theta_1$. Exchange θ_2 and θ_0: $-\theta_0\theta_2\theta_1$. Exchange θ_2 and θ_1: $+\theta_0\theta_1\theta_2$. Two exchanges, sign $+1$. Another: $\theta_1\theta_0 = -\theta_0\theta_1$, one exchange, sign -1. If a generator occurs twice, the two copies can be brought next to each other, and the product is 0. A product of different generators in increasing order, such as $\theta_0\theta_2\theta_5$, is called a **monomial**; the empty product is the number 1; the number of generators in a monomial is its **degree**. Every product of generators is therefore 0, or $+1$ or -1 times exactly one monomial. The sign does not depend on the order in which the exchanges are made: every way of sorting a list by exchanging neighbours uses a number of exchanges of the same parity (even or odd), because each exchange of two neighbours changes the number of pairs that stand in the wrong order by exactly one. (Notebook 07a, In [2], computes the sign by sorting, and In [3] by counting the pairs in the wrong order; the two agree on 2000 random examples.)

The Grassmann algebra. The **Grassmann algebra** of the n generators is the set of all sums of ordinary numbers times monomials, with the defining rule. Its elements are called **Grassmann numbers**, and the ordinary numbers in front of the monomials are their **coefficients**. Two elements are added by adding the coefficients of equal monomials, and multiplied by multiplying every term of the first with every term of the second, each product of monomials being reordered with its sign. Example with two generators: for $F = a + b\theta_0 + c\theta_1 + d\theta_0\theta_1$ and $F' = a' + b'\theta_0 + c'\theta_1 + d'\theta_0\theta_1$,

$$FF' = aa' + (ab' + ba')\theta_0 + (ac' + ca')\theta_1 + (ad' + da' + bc' - cb')\,\theta_0\theta_1.$$

Rule for each term: $b\theta_0\, c'\theta_1 = bc'\,\theta_0\theta_1$ (already in order); $c\theta_1\, b'\theta_0 = cb'\,\theta_1\theta_0 = -cb'\,\theta_0\theta_1$ (one exchange); every product with a repeated generator, such as $b\theta_0\, b'\theta_0$ or $b\theta_0\, d'\theta_0\theta_1$, vanishes. (PROVED; Notebook 07a, In [5].)

How big is a Grassmann algebra? The product $(1+\theta_0)(1+\theta_1)\cdots(1+\theta_{n-1})$, multiplied out, contains every monomial exactly once, with coefficient $+1$: from each bracket we take either 1 or θ_i, and the chosen generators come out in increasing order, so no reordering is needed. So the number of monomials, called the **dimension** of the algebra, is 2^n (each generator is either in a monomial or not: two choices, n times). The number of monomials of degree k is the number of ways to choose k of the n generators, the **binomial coefficient** $\binom{n}{k} = \frac{n!}{k!(n-k)!}$, where $n! = 1\cdot 2\cdots n$ is the **factorial** ($0! = 1$). For $n = 8$ these are 1, 8, 28, 56, 70, 56, 28, 8, 1, which add up to $2^8 = 256$ (PROVED; Notebook 07a counts the monomials for $n = 0$ to 12, In [6], Figure 07a.1). For ordinary commuting symbols there would be infinitely many monomials ($q, q^2, q^3, \ldots$ are all different); anticommuting symbols give a finite algebra.

The multiplication table of three generators. With three generators the eight monomials are $1, \theta_0, \theta_1, \theta_2, \theta_0\theta_1, \theta_0\theta_2, \theta_1\theta_2, \theta_0\theta_1\theta_2$. A product of two of them is nonzero exactly when each of the three generators is in the left factor, in the right factor, or in neither (in both would repeat it): three possibilities for each of three generators, $3^3 = 27$ nonzero products among the $8 \times 8 = 64$ (PROVED; Notebook 07a, In [7], Figure 07a.2).

7.11 Even and odd elements, conjugation and derivatives

Even and odd. Moving a monomial X of degree k past a monomial Y of degree l (with no common generator) moves each of the k generators of X past each of the l generators of Y: kl exchanges. So

$$YX = (-1)^{kl}\, XY.$$

An element is **even** if all its monomials have even degree, **odd** if all have odd degree. If k or l is even, kl is even and the sign is $+1$: an even element commutes with everything. If both are odd, the sign is -1: two odd elements anticommute (PROVED; Notebook 07a measures the sign for $k, l = 0$ to 4, In [8], Figure 07a.3). The Lagrangian of dirac16complex contains the components in pairs, so it is even and behaves like an ordinary number in the order of factors. One more consequence: every odd element squares to zero. For example $(3\theta_0+5\theta_1)^2 = 9\theta_0^2+15\theta_0\theta_1+15\theta_1\theta_0+25\theta_1^2 = 15\theta_0\theta_1-15\theta_0\theta_1 = 0$ (Notebook 07a, In [5]).

Complex Grassmann numbers and conjugation. The components of dirac16complex are complex, and complex numbers have conjugates. For complex Grassmann numbers the generators

come in pairs, θ_k and its **conjugate** $\bar\theta_k = \theta_k^*$, two different generators. The **conjugation** $F \mapsto F^*$ of the algebra is defined by three rules: it exchanges the two members of every pair ($\theta_k^* = \bar\theta_k$, $\bar\theta_k^* = \theta_k$), it replaces every coefficient by its complex conjugate, and it REVERSES the order of every product:

$$(FG)^* = G^* F^*, \qquad \text{for example} \qquad (\theta_1\theta_2)^* = \theta_2^*\theta_1^* = \bar\theta_2\bar\theta_1.$$

Why reverse? With this rule the product of a generator with its conjugate is real: $(\bar\theta\theta)^* = \theta^*\bar\theta^* = \bar\theta\theta$. Without the reversal we would get $\bar\theta^*\theta^* = \theta\bar\theta = -\bar\theta\theta$, and the analogue of $|z|^2 = z^*z$ would not be real. An element with $F^* = F$ is called **real**. Applying the conjugation twice gives back F (the order is reversed twice, the pairs exchanged twice, the coefficients conjugated twice). (PROVED; Notebook 07a checks these rules on the test algebra of the Wolfram verifier and on the 32 generators of a 16-component field, In [9]; they reproduce `Revision/theory/reports/wolfram-field-theory.json`, check `grassmann_algebra_structure`, and `Revision/theory/reports/python-field-theory.json`, check `superalgebra_axioms`.)

Left and right derivatives. To vary a Lagrangian we must differentiate with respect to a generator. Because the order matters, there are two derivatives. The **left derivative** $\partial_L F/\partial\theta_k$: in every monomial that contains θ_k, move θ_k to the far left, collecting a -1 for every generator it passes, and then delete it; monomials without θ_k give nothing. The **right derivative** $\partial_R F/\partial\theta_k$ moves θ_k to the far right instead. In a sorted monomial of degree n in which θ_k stands at position p (counting from 0), moving it left passes p generators and moving it right passes $n-1-p$. Worked by hand for $F = \theta_0\theta_1$:

$$\frac{\partial_L F}{\partial\theta_0} = \theta_1, \qquad \frac{\partial_L F}{\partial\theta_1} = -\theta_0, \qquad \frac{\partial_R F}{\partial\theta_1} = \theta_0, \qquad \frac{\partial_R F}{\partial\theta_0} = -\theta_1.$$

(The second: write $F = -\theta_1\theta_0$, then delete θ_1. The fourth: $F = -\theta_1\theta_0$ has θ_0 on the right.) For an EVEN monomial the two signs differ by $(-1)^{(n-1-p)-p} = (-1)^{n-1}\cdot(-1)^{-2p} = (-1)^{n-1} = -1$, because $n-1$ is odd. So $\partial_R F = -\partial_L F$ for every even F (PROVED; Notebook 07a, In [10], also checks the derivative rules of the sympy record and reproduces its check `superalgebra_axioms`).

7.12 Bilinear forms, real columns and the powers of S

When is $\Psi^\dagger M\Psi$ real? Take a column Ψ of n complex Grassmann components with generators $\psi_A = \Psi_A$ and the conjugate row $\Psi^\dagger$ with generators $\chi_A = \psi_A^*$ (the **dagger** $\dagger$ means transpose and conjugate). For a matrix M of ordinary numbers the **bilinear form** $\Psi^\dagger M\Psi = \sum_{A,B}\chi_A M_{AB}\psi_B$ contains one factor from $\Psi^\dagger$ and one from Ψ in every term. Its conjugate, line by line:

$$\left(\Psi^\dagger M\Psi\right)^* = \sum_{A,B}\left(\chi_A M_{AB}\psi_B\right)^*$$

(the conjugate of a sum is the sum of the conjugates);

$$= \sum_{A,B} M_{AB}^*\,\psi_B^*\chi_A^* = \sum_{A,B} M_{AB}^*\,\chi_B\psi_A$$

(the conjugation reverses the order of the two factors and conjugates the coefficient; then $\psi_B^* = \chi_B$ and $\chi_A^* = \psi_A$);

$$= \sum_{B,A} \chi_B \, (M^\dagger)_{BA} \, \psi_A = \Psi^\dagger M^\dagger \Psi$$

(the coefficient is an ordinary number and may stand anywhere; $(M^\dagger)_{BA} = M^*_{AB}$ by the definition of the dagger; the names of the two summation indices are exchanged). So the form is real exactly when M is **Hermitian**, $M^\dagger = M$. For ordinary complex numbers the same result holds for the opposite reason: there nothing is reordered, because ordinary numbers commute. The reversal of the order built into the conjugation is what makes the two cases come out the same (PROVED; Notebook 07a checks it for $n = 4$ with a random Hermitian and a random non-Hermitian matrix, In [11]).

Real columns keep opposite halves of a matrix. Now take a REAL column $\Theta = (\theta_0, \ldots, \theta_{15})^T$ of 16 real Grassmann generators (no conjugates at all) and the form $\Theta^T M \Theta = \sum_{A,B} \theta_A M_{AB} \theta_B$, where ^{T} is the **transpose** (rows and columns exchanged). Exchanging the two odd factors and renaming the indices, line by line:

$$\Theta^T M \Theta = \sum_{A,B} \theta_A M_{AB} \theta_B = - \sum_{A,B} \theta_B M_{AB} \theta_A$$

(the anticommutation of θ_A and θ_B; for $A = B$ both sides are 0);

$$= - \sum_{A,B} \theta_A M_{BA} \theta_B = - \sum_{A,B} \theta_A (M^T)_{AB} \theta_B = -\Theta^T M^T \Theta$$

(the names A and B exchanged; then $(M^T)_{AB} = M_{BA}$). Write M as the sum of its **symmetric part** $\frac{1}{2}(M + M^T)$ and its **antisymmetric part** $\frac{1}{2}(M - M^T)$. The identity says that the symmetric part gives zero. So: **for a real Grassmann column only the antisymmetric part of M survives, and $\Theta^T M \Theta = 0$ for every symmetric M.** For a real column q of ordinary numbers it is the other way round: there $q^T M q = q^T M^T q$ (no sign when the factors are exchanged), so only the symmetric part survives and $q^T M q = 0$ for every antisymmetric M (PROVED).

The author's matrix C. The author's 16×16 matrix $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ (his σ_{16}; Chapter 5) is real and symmetric with $C^2 = 1$, and every $C\gamma^{(a)}$ is real and antisymmetric (`Revision /theory/reports/python-field-theory.json`, check `C_properties`). By the rule just proved: for a real Grassmann column $\Theta^T C \Theta = 0$, while every $\Theta^T C \gamma^{(a)} \Theta$ survives; for real ordinary numbers $q^T C q$ survives and every $q^T C \gamma^{(a)} q = 0$. Notebook 07a computes the nine forms with $M = C$ and $M = C\gamma^{(a)}$ for both kinds of numbers and counts the surviving monomials: $[0, 8, 8, 8, 8, 8, 8, 8, 8]$ for real Grassmann numbers and $[8, 0, 0, 0, 0, 0, 0, 0, 0]$ for real ordinary numbers (In [12] and In [13], Figures 07a.4 and 07a.5; the Grassmann statement is part of `Revision/theory/reports/wolfram-field-theory.json`, check `Majorana_Lg_total_derivative_grassmann`, the commuting one part of its check `Majorana_Lg_commuting_control`). Each of the 8 surviving monomials of $\Theta^T C \gamma^{(a)} \Theta$ has the coefficient ± 2: every row of the signed permutation matrix $C\gamma^{(a)}$ has one entry ± 1, and the two mirror entries $M_{AB} = -M_{BA}$ give $\theta_A M_{AB} \theta_B + \theta_B M_{BA} \theta_A = M_{AB}(\theta_A \theta_B - \theta_B \theta_A) = 2 M_{AB} \theta_A \theta_B$.

The scalar density and its powers. At one point, the field dirac16complex consists of 16 complex Grassmann numbers: 32 generators $\chi_A = \Psi_A^*$ and $\psi_A = \Psi_A$. The **scalar density** is

$$S = \Psi^\dagger C \Psi = \sum_{A,B} \chi_A C_{AB} \psi_B.$$

C is a **signed permutation matrix**: every row and every column has exactly one nonzero entry, ± 1 (Chapter 5). If row A has its entry in column $\pi(A)$, then

$$S = \sum_A C_{A\pi(A)}\, P_A, \qquad P_A = \chi_A \psi_{\pi(A)},$$

a sum of 16 **pairs** that share no generator (different rows give different χ's, and different columns different ψ's). Each pair is even, so the pairs commute with each other, and $P_A^2 = \chi_A \psi_{\pi(A)} \chi_A \psi_{\pi(A)} = 0$ (the generator χ_A repeats). Now take the k-th power. Multiplying out $S^k = S \cdot S \cdots S$ gives a sum over all ordered choices of k pairs, one from each factor. A choice with a repeated pair gives 0. A choice of k different pairs gives the product of those pairs, and since the pairs commute, all $k!$ orders of the same k pairs give the same product. Each such product is ± 1 times one monomial of degree $2k$, and different sets of pairs give different monomials. So

$$S^k = k! \sum_{\{A_1 < \cdots < A_k\}} C_{A_1\pi(A_1)} \cdots C_{A_k\pi(A_k)}\, P_{A_1} \cdots P_{A_k} :$$

S^k has $\binom{16}{k}$ monomials, each with the coefficient $\pm k!$; S^{16} is a single monomial (all 32 generators), and $S^{17} = 0$, because 17 different pairs do not exist (PROVED; Notebook 07a computes all 17 powers with its program: 16, 120, 560, 1820, 4368, 8008, 11440, 12870, 11440, 8008, 4368, 1820, 560, 120, 16, 1, 0 monomials, In [14], Figure 07a.6). Consequence for the Lagrangian: every function of S is a polynomial of degree at most 16, so the potential $U(S) = \frac{\lambda}{2} S^2$ is a choice among finitely many possible terms. For ordinary commuting components the same S has powers that never vanish; there S^k has $\binom{16+k-1}{k}$ monomials (the number of ways to choose k of the 16 pairs when a pair may be chosen several times): 16, 136, 816 for $k = 1, 2, 3$ (Notebook 07a checks these three with sympy, In [14]). The difference $136 - 120 = 16$ for $k = 2$ will reappear in Section 7.20: it is the number of squares P_A^2 that vanish for Grassmann numbers.

The test algebra of the Revision record. The Wolfram verifier of the Revision record does not give every component its own generators. It uses a Grassmann algebra with only two generators θ_1, θ_2 and their conjugates $\bar\theta_1, \bar\theta_2$, and writes each component as $\Psi_A = c_{A1}\theta_1 + c_{A2}\theta_2$ with ordinary complex coefficients c_{Ak} (which may depend on the coordinates). This keeps the computation small and is still a genuine Grassmann field. Then $\Psi_A^* = c_{A1}^* \bar\theta_1 + c_{A2}^* \bar\theta_2$ (a single generator is not reordered), and

$$S = \sum_{A,B} \Psi_A^* C_{AB} \Psi_B = \sum_{k,l} \bar\theta_k \theta_l \Big(\sum_{A,B} c_{Ak}^* C_{AB} c_{Bl} \Big)$$

(multiply out and collect by the pair of generators). So S lives on the four monomials $\bar\theta_k \theta_l$ (it is even), S^2 only on the one monomial that contains all four generators, and $S^3 = 0$, because it would need six different generators out of four. Hence in this test algebra the most general

potential is $U = u_1 S + \frac{\lambda}{2} S^2$ (PROVED; Notebook 07a checks it with random coefficients, In [16], and reproduces `Revision/theory/reports/wolfram-field-theory.json`, check `grassmann_algebra_structure`).

7.13 Field equations with Grassmann numbers

Jets. A field depends on coordinates. To vary a Lagrangian we treat the value of the field and its derivatives at one point as independent symbols; such a set of symbols is called a **jet**. For a Grassmann field each of them is a generator. The **total derivative** d/dx acts on a jet by raising each generator to its derivative ($\theta \to \theta'$, $\theta' \to \theta''$) in each place in turn, with the product rule; d/dx is even (it does not change the number of odd factors), so it needs no extra sign, but the raised generator may have to be sorted back into place.

The Grassmann oscillator. Take one complex Grassmann field $\theta(t)$ of one variable t and the Lagrangian

$$L = \tfrac{i}{2}(\bar\theta\theta' - \bar\theta'\theta) - \omega\,\bar\theta\theta,$$

the Grassmann copy of Example B of Section 7.4 (a prime is d/dt). It is real: $(\bar\theta\theta')^* = \theta'^*\bar\theta^* = \bar\theta'\theta$, so the bracket is $X - X^*$ with $X = \bar\theta\theta'$, and $\big(\tfrac{i}{2}(X - X^*)\big)^* = -\tfrac{i}{2}(X^* - X) = \tfrac{i}{2}(X - X^*)$ (the conjugate of i is $-i$); and $(\bar\theta\theta)^* = \bar\theta\theta$. Varying $\bar\theta$ with LEFT derivatives:

$$\mathcal{E} = \frac{\partial_L L}{\partial\bar\theta} - \frac{d}{dt}\frac{\partial_L L}{\partial\bar\theta'} = \Big(\tfrac{i}{2}\theta' - \omega\theta\Big) - \frac{d}{dt}\Big(-\tfrac{i}{2}\theta\Big) = i\theta' - \omega\theta.$$

The first term: $\bar\theta$ already stands on the left in $\bar\theta\theta'$ and $\bar\theta\theta$, so deleting it costs no sign. The second: $\bar\theta'$ stands on the left in $-\tfrac{i}{2}\bar\theta'\theta$, its left derivative is $-\tfrac{i}{2}\theta$, whose d/dt is $-\tfrac{i}{2}\theta'$. The field equation $i\theta' = \omega\theta$ has the same form as for an ordinary complex oscillator (PROVED; Notebook 07a, In [17]). Its solution is $\theta(t) = e^{-i\omega t}\theta_0$ with a constant generator θ_0: an ordinary complex function times a fixed generator (In [18], Figure 07a.7). The anticommuting nature sits in θ_0, not in the time dependence. The lesson, used again in Section 7.21: if every conjugate factor is written on the left of every non-conjugate factor, and left derivatives are taken with respect to the conjugates, field equations with Grassmann numbers keep their ordinary form.

A first-order kinetic term of REAL Grassmann fields gives no equation. Now two real Grassmann fields $\theta_0(x)$, $\theta_1(x)$ of one coordinate x, and the first-order kinetic term with the antisymmetric matrix $A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$:

$$K_A = \theta^T A\,\theta' = \theta_0\theta_1' - \theta_1\theta_0'.$$

Line by line:

$$K_A = \theta_0\theta_1' + \theta_0'\theta_1$$

(in the second term $\theta_1\theta_0' = -\theta_0'\theta_1$, one exchange of two odd factors);

$$= (\theta_0\theta_1)'$$

(the product rule read backwards); and $\theta^T A\theta = \theta_0\theta_1 - \theta_1\theta_0 = 2\theta_0\theta_1$, so

$$K_A = \tfrac{1}{2}(\theta^T A\theta)',$$

a **total derivative**. By Section 7.4 it gives no field equation: both Euler-Lagrange expressions vanish identically. With the symmetric identity matrix instead, $K_I = \theta_0\theta_0' + \theta_1\theta_1'$ is not a total derivative ($(\theta_0\theta_0)' = 0$ because $\theta_0^2 = 0$), and its Euler-Lagrange expression for θ_0 is $\theta_0' - \frac{d}{dx}(-\theta_0) = 2\theta_0'$ (the left derivative of $\theta_0\theta_0'$ with respect to θ_0' passes θ_0, hence the $-\theta_0$). For ORDINARY numbers q_0, q_1 the roles are exchanged: $K_A = q_0q_1' - q_1q_0'$ gives the nonzero expressions $E_0 = q_1' - \frac{d}{dx}(-q_1) = 2q_1'$ and $E_1 = -q_0' - \frac{d}{dx}q_0 = -2q_0'$ (PROVED; Notebook 07a, In [19]). This is the one-dimensional model of the negative control of Section 7.28: a real anticommuting field whose kinetic matrices $C\gamma^\mu$ are antisymmetric has no field equation at all. That is why the Lagrangian of dirac16complex is written for a COMPLEX field, with the row $\bar\Psi = \Psi^\dagger C$ in place of a transpose.

7.14 Example: Notebook 07a builds a Grassmann algebra by hand

Notebook 07a builds the algebra of Sections 7.10 to 7.13 from nothing but its rules, in a few lines of plain Python: a sorting function that returns the sign of a reordering, a faster sign rule for the product of two sorted monomials, and a class `Grassmann` whose elements are dictionaries from monomials to coefficients. It checks the defining rules and the product of two general elements, counts the 2^n monomials, draws the multiplication table of three generators and the commutation signs $(-1)^{kl}$, builds the conjugation and the two derivatives, checks the reality rule for bilinear forms, shows with the author's matrix C (read from the Revision record) which bilinear forms survive for real columns of the two kinds of numbers, computes all powers of the scalar density of a 16-component field up to $S^{17} = 0$, checks the test algebra of the Wolfram verifier, derives the Grassmann oscillator and shows that the first-order kinetic term of real Grassmann fields is a total derivative. It draws seven figures and ends with the line ALL 38 CHECKS PASSED (notebook 07a).

7.15 How to run Notebook 07a

Step 1. What this notebook does and what it needs.

Notebook 07a (A Grassmann algebra by hand) is the file `Revision/textbook/notebooks/07a_grassmann_algebra.ipynb` of the repository Dirac_claude. It builds a Grassmann algebra (numbers that anticommute) from its rules in a few lines of plain Python, checks the rules, the order-reversing conjugation and the left and right derivatives, shows which bilinear forms of a real anticommuting column survive with the author's charge matrix C, counts the monomials of every power of the scalar density S of a 16-component complex Grassmann field (the 17th power is zero), reproduces the Grassmann-algebra facts of the Revision record, derives two Euler-Lagrange equations with Grassmann variables and draws seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6,

sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 07a_grassmann_algebra.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 07a_grassmann_algebra.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 07a_grassmann_algebra.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/07a.captions.json`
- `Revision/textbook/figures/07a_1_dimension_and_degrees.png`
- `Revision/textbook/figures/07a_2_multiplication_signs.png`
- `Revision/textbook/figures/07a_3_commutation_signs.png`
- `Revision/textbook/figures/07a_4_bilinear_survivors.png`
- `Revision/textbook/figures/07a_5_symmetric_antisymmetric.png`
- `Revision/textbook/figures/07a_6_powers_of_s.png`
- `Revision/textbook/figures/07a_7_oscillator_solution.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 7 figure files of this notebook exist
ALL 38 CHECKS PASSED (notebook 07a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/07a_grassmann_algebra.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 07a_grassmann_algebra.ipynb
python -m nbconvert --execute --inplace 07a_grassmann_algebra.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/theory/reports/...": the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory/reports Revision/algebra
```

7.16 Notebook 07a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/07a_grassmann_algebra.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field dirac16complex of this book has 16 components that **anticommute**: for
two of its components $\Psi_A \Psi_B = -\Psi_B \Psi_A$. Ordinary numbers cannot do
that. This notebook builds, from nothing but its rules, the algebra of such numbers
(a **Grassmann algebra**) in a few lines of plain Python, and then uses it. It

- writes a program that multiplies anticommuting symbols and checks the rules
  $\theta_i \theta_j = -\theta_j \theta_i$ and $\theta_i^2 = 0$;
- counts how many independent products a Grassmann algebra has ($2^n$ for $n$
  symbols) and draws the multiplication table of three symbols;
- shows which elements commute and which anticommute (even and odd elements);
- builds the **conjugation**, which reverses the order of a product, and the
  **left and right derivatives**;
- shows that a bilinear form $\Psi^\dagger M \Psi$ is real exactly when the matrix
  $M$ is Hermitian, for anticommuting components as for ordinary numbers;
- shows, with the author's matrix $C$ read from the Revision record, that for a
  REAL anticommuting column the form $\Theta^T C \Theta$ is zero while every
  $\Theta^T C \gamma^{(a)} \Theta$ survives, and that for real ordinary numbers it is
  the other way round;
- computes every power of the scalar density $S = \bar\Psi \Psi = \Psi^\dagger C
  \Psi$ of a 16-component complex anticommuting field: $S^k$ has
  $\binom{16}{k}$ monomials, each with coefficient $\pm k!$, and $S^{17} = 0$;
- reproduces the Grassmann-algebra facts that the Revision record verifies
  (Revision/theory/reports/python-field-theory.json and
  Revision/theory/reports/wolfram-field-theory.json);
- derives the Euler-Lagrange equation of a small anticommuting oscillator, and
  shows that a first-order kinetic term of REAL anticommuting fields gives no
  field equation at all.

Every result is checked by a line that starts with PASS; the last line counts the
checks. Seven figures are drawn and saved.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 07a (A Grassmann algebra by hand) is the file
‘Revision/textbook/notebooks/07a_grassmann_algebra.ipynb‘ of the repository Dirac_claude.
It builds a Grassmann algebra (numbers that anticommute) from its rules in a few lines of
plain Python, checks the rules, the order-reversing conjugation and the left and right
derivatives, shows which bilinear forms of a real anticommuting column survive with the
author's charge matrix C, counts the monomials of every power of the scalar density S of
a 16-component complex Grassmann field (the 17th power is zero), reproduces the
Grassmann-algebra facts of the Revision record, derives two Euler-Lagrange equations with
Grassmann variables and draws seven teaching plots. It needs a computer with Windows 11,
macOS or Linux, an internet connection for the installation, the program Git, and Python
3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at
exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
```

Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then

needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 07a_grassmann_algebra.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 07a_grassmann_algebra.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 07a_grassmann_algebra.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/07a.captions.json‘

- ‘Revision/textbook/figures/07a_1_dimension_and_degrees.png‘
- ‘Revision/textbook/figures/07a_2_multiplication_signs.png‘
- ‘Revision/textbook/figures/07a_3_commutation_signs.png‘
- ‘Revision/textbook/figures/07a_4_bilinear_survivors.png‘
- ‘Revision/textbook/figures/07a_5_symmetric_antisymmetric.png‘
- ‘Revision/textbook/figures/07a_6_powers_of_s.png‘
- ‘Revision/textbook/figures/07a_7_oscillator_solution.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 7 figure files of this notebook exist
ALL 38 CHECKS PASSED (notebook 07a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/07a_grassmann_algebra.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command

below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 07a_grassmann_algebra.ipynb
python -m nbconvert --execute --inplace 07a_grassmann_algebra.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/theory/reports/...": the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory/reports Revision/algebra
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 07a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 07a (A Grassmann algebra by hand) is the file
# Revision/textbook/notebooks/07a_grassmann_algebra.ipynb of the repository Dirac_claude.
# It builds a Grassmann algebra (numbers that anticommute) from its rules in a few lines
# of plain Python, checks the rules, the order-reversing conjugation and the left and
# right derivatives, shows which bilinear forms of a real anticommuting column survive
# with the author’s charge matrix C, counts the monomials of every power of the scalar
# density S of a 16-component complex Grassmann field (the 17th power is zero),
# reproduces the Grassmann-algebra facts of the Revision record, derives two
# Euler-Lagrange equations with Grassmann variables and draws seven teaching plots. It
# needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
```

```
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
```

```
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
```

```
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 07a_grassmann_algebra.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 07a_grassmann_algebra.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 07a_grassmann_algebra.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/07a.captions.json
# - Revision/textbook/figures/07a_1_dimension_and_degrees.png
# - Revision/textbook/figures/07a_2_multiplication_signs.png
# - Revision/textbook/figures/07a_3_commutation_signs.png
# - Revision/textbook/figures/07a_4_bilinear_survivors.png
# - Revision/textbook/figures/07a_5_symmetric_antisymmetric.png
# - Revision/textbook/figures/07a_6_powers_of_s.png
# - Revision/textbook/figures/07a_7_oscillator_solution.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
```

```
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 7 figure files of this notebook exist
#     ALL 38 CHECKS PASSED (notebook 07a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/07a_grassmann_algebra.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 07a_grassmann_algebra.ipynb
#     python -m nbconvert --execute --inplace 07a_grassmann_algebra.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "ValueError: ... is not a passing check of Revision/theory/reports/...": the Revision
#   record files of your copy of the repository differ from the committed ones; restore
#   them with the command below (run in the repository folder) and run the notebook
#   again.
#     git restore Revision/theory/reports Revision/algebra
#
# ------------------------------------------------------------------------------------
```

```
# ============================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ============================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "07a"  # this notebook: chapter 07, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
```

```
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 07a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Generator**: one of the anticommuting symbols $\theta_0, \theta_1, \dots,
  \theta_{n-1}$ from which everything is built. In the program a generator is
  named by its number $0, 1, \dots, n-1$.
- **Anticommute**: $xy = -yx$. Two different generators anticommute; a generator
  times itself is zero, because $\theta\theta = -\theta\theta$ means
  $2\theta\theta = 0$.
- **Monomial**: a product of different generators written in increasing order,
  such as $\theta_0 \theta_2 \theta_5$; the empty product is the number 1. Its
  **degree** is the number of generators in it.
- **Grassmann algebra**: all sums of ordinary numbers times monomials, with the
  rules above. An element of it is called a **Grassmann number**. The ordinary
  numbers in front of the monomials are the **coefficients**.
- **Sign of a reordering**: putting the factors of a product into increasing order
  by exchanging neighbours; every exchange of two generators gives a factor $-1$.
  The total factor is $+1$ for an even and $-1$ for an odd number of exchanges.
- **Even, odd**: an element is even if all its monomials have even degree, odd if
  all have odd degree. Even elements commute with everything.
- **Conjugation**: the rule that turns an element $F$ into $F^*$. For complex
  Grassmann numbers the generators come in pairs $\theta$ and $\bar\theta =
  \theta^*$; the conjugation exchanges the two members of every pair, replaces
  every coefficient by its complex conjugate and REVERSES the order of every
  product: $(FG)^* = G^* F^*$. An element with $F^* = F$ is called **real**.
- **Left (right) derivative** $\partial_L F / \partial \theta_k$
  ($\partial_R F / \partial \theta_k$): move $\theta_k$ to the far left (right)
  of every monomial that contains it, collecting a $-1$ for every generator it
  passes, and then delete it.
- **Matrix, transpose, symmetric, antisymmetric, Hermitian**: a table of numbers;
  $M^T$ has rows and columns exchanged; $M$ is symmetric if $M^T = M$,
  antisymmetric if $M^T = -M$, Hermitian if $M^\dagger = M$, where $M^\dagger$ is
  the transpose with every entry complex-conjugated.
- **Bilinear form**: an expression $\sum_{A,B} x_A M_{AB} y_B$ that contains one
  factor from the column $x$ and one from the column $y$ in every term.
```

- **The matrix C**: the author's 16×16 matrix $C = \gamma^{(x_0)} \gamma^{(x_1)} \gamma^{(x_2)} \gamma^{(x_3)}$ (his σ_{16}), read from the Revision file Revision/algebra/gammas.json together with the eight gamma matrices $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$.
- **Scalar density** $S = \bar\Psi\Psi = \Psi^\dagger C \Psi$: the bilinear that the mass term and the potential of the Lagrangian contain.
- **Euler-Lagrange equation**: the field equation obtained from a Lagrangian L by requiring that the action does not change, to first order, when the field is changed a little.
- **Python dictionary**: a table that maps keys to values; here monomials (tuples of generator numbers) to coefficients.

Text cell:

```
## 4. The physical and mathematical situation

**Why anticommuting numbers.** Electrons and the quanta of the field dirac16complex
are fermions: two of them never occupy the same state (the Pauli principle). In the
quantum theory this is expressed by field operators that anticommute. The
classical field from which such operators are obtained is therefore given values
that anticommute: the Revision record defines the components of dirac16complex as
Grassmann numbers (Revision/SPEC.md, section 3). Grassmann numbers are not
measured; they are bookkeeping symbols with exact rules, and every statement of
this notebook is a statement about those rules.

**The two fields of the book.** dirac16complex is a column $\Psi$ of 16 complex
Grassmann numbers $\Psi_1, \dots, \Psi_{16}$ (one complex generator pair per
component and point); dirac16complex00 is a column of 16 ordinary complex numbers.
Both have the same Lagrangian,

$$L = \sqrt{|g|}\,\Big[\tfrac12\big(\bar\Psi\gamma^\mu D_\mu\Psi -
(D_\mu\bar\Psi)\gamma^\mu\Psi\big) - m S - \tfrac{\lambda}{2} S^2\Big],
\qquad S = \bar\Psi\Psi,\quad \bar\Psi = \Psi^\dagger C,$$

(Notebook 07b derives its field equations). The difference between the two fields
is only the kind of numbers, and this notebook shows where that difference
matters: in products of a symbol with itself ($\theta^2 = 0$), in the order of
factors, in which bilinear forms vanish, and in the powers of $S$.

**Rules.** For generators $\theta_0, \dots, \theta_{n-1}$:

$$\theta_i \theta_j = -\theta_j \theta_i \quad\text{for all } i, j,
\qquad\text{hence}\qquad \theta_i \theta_i = 0 .$$

Ordinary numbers commute with every generator. Every product of generators can be
reordered into increasing order; each exchange of two neighbours gives a factor
$-1$. So every element is a unique sum of numbers times the $2^n$ monomials
$\theta_{i_1}\theta_{i_2}\cdots\theta_{i_k}$ with $i_1 < i_2 < \dots < i_k$.
This is exactly how the program stores an element: a dictionary from sorted
tuples $(i_1, \dots, i_k)$ to coefficients.
```

Text cell:

5. The sign of a reordering

The next cell imports the packages and wraps the set-up cell's ‘check‘ so that a PASS line and the line "reproduces ..." that may follow it are printed by one single ‘print‘ call (Jupyter sends printed text in pieces; two separate prints can arrive in two pieces, and the book's tools then do not see that the two lines belong together). Then it defines the function ‘sort_with_sign‘. It sorts a list of generator numbers by exchanging neighbours that stand in the wrong order (the method called bubble sort), counts the exchanges, and returns the sign $(-1)^{\text{exchanges}}$ together with the sorted tuple. If a generator occurs twice, the product contains $\theta_i\theta_i = 0$ and the sign is 0. The cell prints six examples and checks three of them by hand:
$\theta_1\theta_0 = -\theta_0\theta_1$ (one exchange),
$\theta_2\theta_0\theta_1 = +\theta_0\theta_1\theta_2$ (two exchanges) and
$\theta_0\theta_0 = 0$.

In [2]:

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import math  # factorials and binomial coefficients
from bisect import bisect_right  # counts the entries of a sorted tuple that are <= y
from collections import Counter  # counts how often each value occurs

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece


def sort_with_sign(generators):
    """Sort generator numbers by exchanging neighbours (bubble sort).
    Return (sign, sorted tuple): sign = +1 or -1 for an even or odd number of
    exchanges, and sign = 0 if a generator occurs twice (the product is zero)."""
    items = list(generators)  # a copy that we may reorder
    sign = 1
    for end in range(len(items) - 1, 0, -1):  # pass over items[0 .. end]
        for i in range(end):  # every neighbour pair of this pass
            if items[i] > items[i + 1]:  # a pair in the wrong order:
                items[i], items[i + 1] = items[i + 1], items[i]  # exchange it,
                sign = -sign  # which costs one factor -1
    if any(items[i] == items[i + 1] for i in range(len(items) - 1)):
        return 0, tuple(items)  # a repeated generator: theta_i theta_i = 0
    return sign, tuple(items)
```

```
for example in [(0, 1), (1, 0), (2, 0, 1), (2, 1, 0), (0, 0), (3, 1, 2, 0)]:
    sign, ordered = sort_with_sign(example)
    say(f"generators {example} -> sign {sign:+d}, sorted {ordered}")
check(sort_with_sign((1, 0)) == (-1, (0, 1)), "theta_1 theta_0 = -theta_0 theta_1")
check(sort_with_sign((2, 0, 1)) == (1, (0, 1, 2)),
      "theta_2 theta_0 theta_1 = +theta_0 theta_1 theta_2 (two exchanges)")
check(sort_with_sign((0, 0))[0] == 0, "theta_0 theta_0 = 0")
```

Out [2]:

```
generators (0, 1) -> sign +1, sorted (0, 1)
generators (1, 0) -> sign -1, sorted (0, 1)
generators (2, 0, 1) -> sign +1, sorted (0, 1, 2)
generators (2, 1, 0) -> sign -1, sorted (0, 1, 2)
generators (0, 0) -> sign +0, sorted (0, 0)
generators (3, 1, 2, 0) -> sign -1, sorted (0, 1, 2, 3)
PASS theta_1 theta_0 = -theta_0 theta_1
PASS theta_2 theta_0 theta_1 = +theta_0 theta_1 theta_2 (two exchanges)
PASS theta_0 theta_0 = 0
```

Text cell:

```
## 6. A Grassmann algebra in a few lines of Python

Multiplying two monomials means writing one after the other and sorting. Both are
already sorted, so there is a faster way to get the sign: when the left monomial
is placed before the right one, every pair ($x$ from the left, $y$ from the right)
with $x > y$ is in the wrong order and needs exactly one exchange to be repaired.
So the sign is $(-1)^N$ with $N$ the number of such pairs. The function
‘shuffle_sign‘ counts them with ‘bisect_right‘ (which finds how many entries of a
sorted tuple are at most $y$). The next cell defines it and compares it with the
slow ‘sort_with_sign‘ on 2000 random pairs of monomials.
```

In [3]:

```
def shuffle_sign(left, right):
    """The sign of the product of the sorted monomials left and right: 0 if they
    share a generator, otherwise (-1)**N with N the number of pairs (x in left,
    y in right) with x > y, which is the number of neighbour exchanges needed."""
    if set(left) & set(right):  # a common generator: the product holds theta^2
        return 0
    # len(left) - bisect_right(left, y) entries of left are larger than y
    pairs = sum(len(left) - bisect_right(left, y) for y in right)
    return -1 if pairs % 2 else 1


rng = np.random.default_rng(12345)  # random numbers with a fixed seed: same every run
agree = 0
for trial in range(2000):
```

```
    # two random sorted monomials with generators from 0 .. 9
    left = tuple(sorted(rng.choice(10, size=rng.integers(0, 5), replace=False)))
    right = tuple(sorted(rng.choice(10, size=rng.integers(0, 5), replace=False)))
    agree += shuffle_sign(left, right) == sort_with_sign(left + right)[0]
report("random pairs of monomials on which the two sign rules agree", f"{agree} of 2000")
check(agree == 2000, "shuffle_sign equals sort_with_sign on 2000 random pairs")
```

Out [3]:

```
RESULT random pairs of monomials on which the two sign rules agree = 2000 of 2000
PASS shuffle_sign equals sort_with_sign on 2000 random pairs
```

Text cell:

```
Now the algebra itself. An element is stored as a dictionary ‘terms‘ that maps
every monomial (a sorted tuple; ‘()‘ is the number 1) to its coefficient. Adding
two elements adds the coefficients of equal monomials; multiplying multiplies every
term of the first by every term of the second, with the sign of ‘shuffle_sign‘.
A coefficient that becomes zero is deleted, so an element is zero exactly when its
dictionary is empty. The coefficients may be integers or exact sympy numbers and
expressions; a sympy coefficient is multiplied out (‘sp.expand‘) whenever it
changes, so that terms that are equal also look equal and cancel.
```

In [4]:

```
class Grassmann:
    """An element of a Grassmann algebra: self.terms maps every monomial (a sorted
    tuple of generator numbers; () is the number 1) to its coefficient."""

    def __init__(self, terms=None):
        self.terms = {}
        for monomial, coefficient in (terms or {}).items():
            self.add(monomial, coefficient)

    def add(self, monomial, coefficient):
        """Add coefficient times monomial to this element (in place)."""
        total = self.terms.get(monomial, 0) + coefficient
        if isinstance(total, sp.Basic):  # an exact sympy number or expression:
            total = sp.expand(total)  # multiply out, so that equal terms cancel
        if total == 0:
            self.terms.pop(monomial, None)  # a zero coefficient is not stored
        else:
            self.terms[monomial] = total

    def __add__(self, other):  # self + other
        result = Grassmann(self.terms)
        for monomial, coefficient in other.terms.items():
            result.add(monomial, coefficient)
        return result

    def scaled(self, factor):  # factor * self, factor an ordinary number
```

```
        return Grassmann({m: factor * c for m, c in self.terms.items()})

    def __sub__(self, other):  # self - other
        return self + other.scaled(-1)

    def __mul__(self, other):  # self * other: every term times every term
        result = Grassmann()
        for left, a in self.terms.items():
            for right, b in other.terms.items():
                sign = shuffle_sign(left, right)
                if sign != 0:
                    result.add(tuple(sorted(left + right)), sign * a * b)
        return result

    def is_zero(self):
        return not self.terms  # True when no monomial is left


def number(value):
    """The ordinary number value as an element of the algebra."""
    return Grassmann({(): value})


def theta(i):
    """The generator number i."""
    return Grassmann({(i,): 1})


say("The class Grassmann and the helpers number and theta are defined.")
```

Out [4]:

```
The class Grassmann and the helpers number and theta are defined.
```

Text cell:

```
The next cell checks the two defining rules and one consequence: every odd element
squares to zero, for example $(3\theta_0 + 5\theta_1)^2 = 15\,\theta_0\theta_1 +
15\,\theta_1\theta_0 = 0$. Then it multiplies two general elements of the algebra
with two generators, $F = a + b\theta_0 + c\theta_1 + d\theta_0\theta_1$ and
$F' = a' + b'\theta_0 + c'\theta_1 + d'\theta_0\theta_1$ (the primes are written
a2, b2, c2, d2 in the code), and compares with the product worked out by hand:

$$FF' = aa' + (ab' + ba')\theta_0 + (ac' + ca')\theta_1 + (ad' + da' + bc' -
cb')\,\theta_0\theta_1 .$$

The term $-cb'$ comes from $c\theta_1\, b'\theta_0 = cb'\,\theta_1\theta_0 =
-cb'\,\theta_0\theta_1$; every product with a repeated generator vanishes.
```

In [5]:

```
t0, t1, t2 = theta(0), theta(1), theta(2)
check((t0 * t1 + t1 * t0).is_zero(), "theta_0 theta_1 + theta_1 theta_0 = 0")
check((t0 * t0).is_zero() and (t2 * t2).is_zero(), "a generator squares to zero")
odd = t0.scaled(3) + t1.scaled(5)  # the odd element 3 theta_0 + 5 theta_1
check((odd * odd).is_zero(), "(3 theta_0 + 5 theta_1)^2 = 0")

a, b, c, d, a2, b2, c2, d2 = sp.symbols("a b c d a2 b2 c2 d2")  # 8 ordinary numbers
F = number(a) + t0.scaled(b) + t1.scaled(c) + (t0 * t1).scaled(d)
F2 = number(a2) + t0.scaled(b2) + t1.scaled(c2) + (t0 * t1).scaled(d2)
by_hand = {(): a * a2, (0,): a * b2 + b * a2, (1,): a * c2 + c * a2,
           (0, 1): a * d2 + d * a2 + b * c2 - c * b2}
product = F * F2
same = sorted(product.terms) == sorted(by_hand) and all(
    sp.expand(product.terms[key] - value) == 0 for key, value in by_hand.items())
say(f"coefficient of theta_0 theta_1 in F F': {product.terms[(0, 1)]}")
check(same, "the product of two general elements of a 2-generator algebra")
```

Out [5]:

```
PASS theta_0 theta_1 + theta_1 theta_0 = 0
PASS a generator squares to zero
PASS (3 theta_0 + 5 theta_1)^2 = 0
coefficient of theta_0 theta_1 in F F': a*d2 + a2*d + b*c2 - b2*c
PASS the product of two general elements of a 2-generator algebra
```

Text cell:

```
## 7. How big is a Grassmann algebra?

The product $(1 + \theta_0)(1 + \theta_1)\cdots(1 + \theta_{n-1})$ contains every
monomial exactly once, with coefficient $+1$: from each bracket we take either 1 or
$\theta_i$, and the factors come out in increasing order. So the number of its
terms is the number of monomials, the **dimension** of the algebra, which should be
$2^n$ (each generator is either in a monomial or not). The number of monomials of
degree $k$ should be the binomial coefficient $\binom{n}{k}$, the number of ways to
choose $k$ of the $n$ generators. The next cell computes the product for
$n = 0, 1, \dots, 12$, checks both counts, and draws them (Figure 1). For ordinary
commuting symbols there would be infinitely many monomials ($q^2, q^3, \dots$ are
all different); anticommuting symbols give a finite algebra.
```

In [6]:

```
sizes = []  # the number of monomials for n = 0, 1, ..., 12
for n in range(13):
    everything = number(1)
    for i in range(n):
        everything = everything * (number(1) + theta(i))  # (1 + theta_i)
    sizes.append(len(everything.terms))
    if n == 8:
        degrees8 = Counter(len(monomial) for monomial in everything.terms)
        ones8 = all(value == 1 for value in everything.terms.values())
```

```
report("number of monomials for n = 12 generators", sizes[12])
check(sizes == [2 ** n for n in range(13)], "a Grassmann algebra with n generators "
      "has 2^n monomials (n = 0 to 12)")
check(ones8 and all(degrees8[k] == math.comb(8, k) for k in range(9)),
      "for n = 8 there are binom(8, k) monomials of degree k, each coefficient +1")

fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.semilogy(range(13), sizes, "o", label="counted by the program")
left.semilogy(range(13), [2 ** n for n in range(13)], "-", label="$2^n$")
left.set_xlabel("number of generators $n$")
left.set_ylabel("number of monomials")
left.set_title("Dimension of the algebra")
left.legend()
right.bar(range(9), [degrees8[k] for k in range(9)], color="tab:orange")
right.set_xlabel("degree $k$ (generators in the monomial)")
right.set_ylabel("number of monomials")
right.set_title("Degrees for $n = 8$: $\\binom{8}{k}$")
fig.tight_layout()
save_figure(fig, "dimension_and_degrees",
            "Left: the number of monomials of a Grassmann algebra with $n$ "
            "generators, counted by the program (dots), and the formula $2^n$ "
            "(line), for $n = 0$ to $12$; horizontal axis $n$, vertical axis the "
            "count on a logarithmic scale, on which $2^n$ is a straight line. "
            "Right: for $n = 8$ the number of monomials of each degree $k$, the "
            "binomial coefficients 1, 8, 28, 56, 70, 56, 28, 8, 1, which add up to "
            "$2^8 = 256$. The algebra is finite because no generator can occur twice "
            "in a nonzero product.")
```

Out [6]:

```
RESULT number of monomials for n = 12 generators = 4096
PASS a Grassmann algebra with n generators has 2^n monomials (n = 0 to 12)
PASS for n = 8 there are binom(8, k) monomials of degree k, each coefficient +1
```

Out [6]:

```
Figure 07a.1 saved as Revision/textbook/figures/07a_1_dimension_and_degrees.png
```

Text cell:

```
## 8. The multiplication table of three generators

With three generators the algebra has the eight monomials $1, \theta_0, \theta_1,
\theta_2, \theta_0\theta_1, \theta_0\theta_2, \theta_1\theta_2,
\theta_0\theta_1\theta_2$. The next cell computes the sign of the product of every
monomial (row) with every monomial (column): $+1$, $-1$, or 0 when they share a
generator. A product is nonzero when each of the three generators is in the row
monomial, in the column monomial, or in neither: $3^3 = 27$ possibilities. The
cell checks this count and draws the table as a heat map (Figure 2).
```

In [7]:

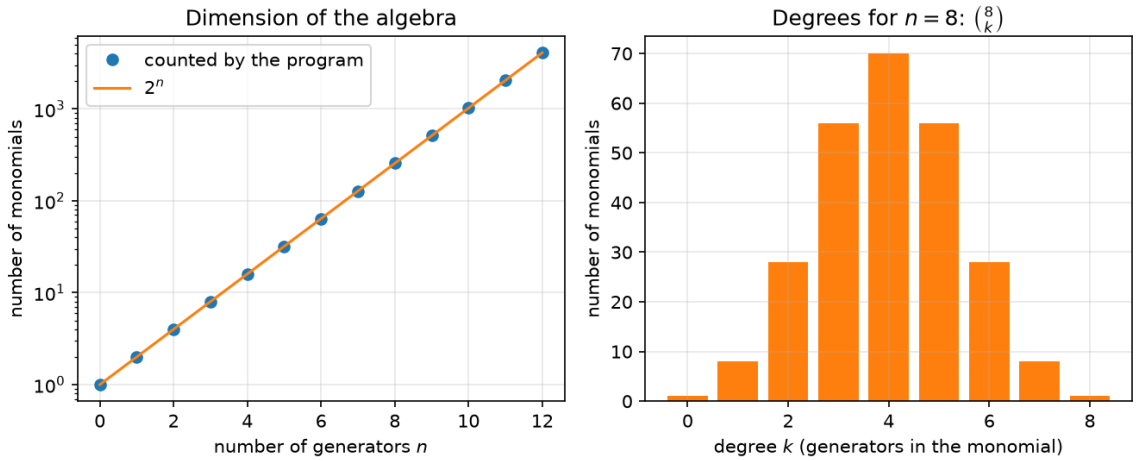

Figure 212: Left: the number of monomials of a Grassmann algebra with n generators, counted by the program (dots), and the formula 2^n (line), for $n = 0$ to 12; horizontal axis n, vertical axis the count on a logarithmic scale, on which 2^n is a straight line. Right: for $n = 8$ the number of monomials of each degree k, the binomial coefficients 1, 8, 28, 56, 70, 56, 28, 8, 1, which add up to $2^8 = 256$. The algebra is finite because no generator can occur twice in a nonzero product. (Notebook 07a, figure 1.)

```
basis3 = [(), (0,), (1,), (2,), (0, 1), (0, 2), (1, 2), (0, 1, 2)]
names3 = ["$1$"] + ["$" + "".join(f"\\theta_{i}" for i in m) + "$" for m in basis3[1:]]
table = np.array([[shuffle_sign(row, column) for column in basis3] for row in basis3])
nonzero = int(np.count_nonzero(table))
report("nonzero products among the 64 products of the 8 monomials", nonzero)
check(nonzero == 27, "27 = 3^3 of the 64 products of monomials of 3 generators are "
      "nonzero")
check(np.array_equal(table[1:4, 1:4], -table[1:4, 1:4].T),
      "the table of the three generators is antisymmetric")

fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(table, cmap="coolwarm", vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        ax.text(j, i, {1: "+", -1: "-", 0: "0"}[int(table[i, j])], ha="center",
                va="center", fontsize=12)
ax.set_xticks(range(8), names3, rotation=45)
ax.set_yticks(range(8), names3)
ax.set_xlabel("right factor")
ax.set_ylabel("left factor")
ax.set_title("Sign of (left factor) times (right factor)")
ax.grid(False)
fig.colorbar(image, ax=ax, ticks=[-1, 0, 1], shrink=0.8)
save_figure(fig, "multiplication_signs",
            "The multiplication table of the 8 monomials of a Grassmann algebra with "
            "three generators: row = left factor, column = right factor; the entry "
            "is the sign with which the product equals the sorted monomial "
            "(red $+1$, blue $-1$), and 0 (grey) when the two factors share a "
            "generator. The 27 nonzero entries are the ways of distributing the "
```

```
        "three generators over the two factors. The block of the three "
        "generators is antisymmetric: $\\theta_i\\theta_j = -\\theta_j\\theta_i$.")
```

Out [7]:

```
RESULT nonzero products among the 64 products of the 8 monomials = 27
PASS 27 = 3^3 of the 64 products of monomials of 3 generators are nonzero
PASS the table of the three generators is antisymmetric
```

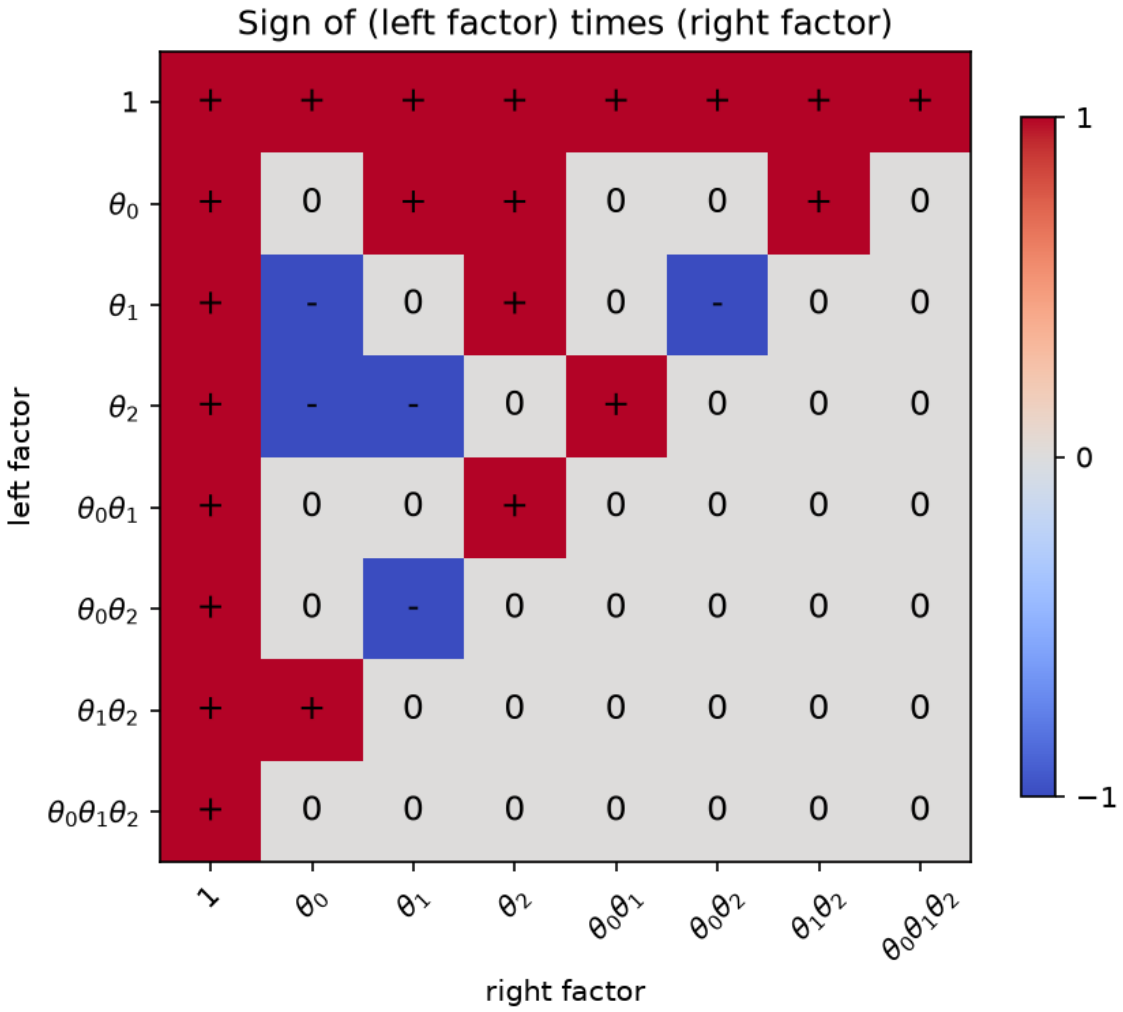

Figure 213: The multiplication table of the 8 monomials of a Grassmann algebra with three generators: row = left factor, column = right factor; the entry is the sign with which the product equals the sorted monomial (red +1, blue −1), and 0 (grey) when the two factors share a generator. The 27 nonzero entries are the ways of distributing the three generators over the two factors. The block of the three generators is antisymmetric: $\theta_i\theta_j = -\theta_j\theta_i$. (Notebook 07a, figure 2.)

Out [7]:

```
Figure 07a.2 saved as Revision/textbook/figures/07a_2_multiplication_signs.png
```

Text cell:

```
## 9. Even and odd elements

Moving a monomial $X$ of degree $k$ past a monomial $Y$ of degree $l$ (with no
common generator) moves each of the $k$ generators past each of the $l$ others:
$kl$ exchanges. So

$$YX = (-1)^{kl}\, XY .$$

If $k$ or $l$ is even, the sign is $+1$: an even element commutes with everything.
If both are odd, the sign is $-1$: odd elements anticommute. The next cell measures
the sign for $k, l = 0, \dots, 4$ with $X = \theta_0\cdots\theta_{k-1}$ and $Y =
\theta_5\cdots\theta_{5+l-1}$, checks it against $(-1)^{kl}$ and draws Figure 3.
This is why the Lagrangian of dirac16complex, which contains the components in
pairs (it is even), behaves like an ordinary number in the order of factors.
```

In [8]:

```
signs = np.zeros((5, 5), dtype=int)
for k in range(5):
    for l in range(5):
        X = number(1)
        for i in range(k):
            X = X * theta(i)  # theta_0 ... theta_(k-1)
        Y = number(1)
        for i in range(l):
            Y = Y * theta(5 + i)  # theta_5 ... theta_(4+l)
        XY, YX = X * Y, Y * X
        (monomial, value), = XY.terms.items()  # XY is one monomial
        signs[k, l] = YX.terms[monomial] // value  # +1 or -1
expected = np.array([[(-1) ** (k * l) for l in range(5)] for k in range(5)])
check(np.array_equal(signs, expected), "YX = (-1)^(k l) XY for degrees k, l = 0 .. 4")

fig, ax = plt.subplots(figsize=(5.6, 4.6))
image = ax.imshow(signs, cmap="coolwarm", vmin=-1, vmax=1)
for k in range(5):
    for l in range(5):
        ax.text(l, k, f"{signs[k, l]:+d}", ha="center", va="center", fontsize=12)
ax.set_xticks(range(5))
ax.set_yticks(range(5))
ax.set_xlabel("degree $l$ of $Y$")
ax.set_ylabel("degree $k$ of $X$")
ax.set_title("$YX = (\\pm 1)\\, XY$")
ax.grid(False)
fig.colorbar(image, ax=ax, ticks=[-1, 1], shrink=0.8)
save_figure(fig, "commutation_signs",
            "The sign in $YX = \\pm XY$ for a monomial $X$ of degree $k$ (row) and "
            "a monomial $Y$ of degree $l$ (column) without a common generator, "
            "measured by the program for $k, l = 0$ to $4$. Red $+1$: they "
            "commute; blue $-1$: they anticommute. The pattern is $(-1)^{kl}$: only "
            "two odd monomials anticommute, and an even monomial (rows and columns "
            "0, 2, 4) commutes with everything.")
```

Out [8]:

```
PASS YX = (-1)^(k l) XY for degrees k, l = 0 .. 4
```

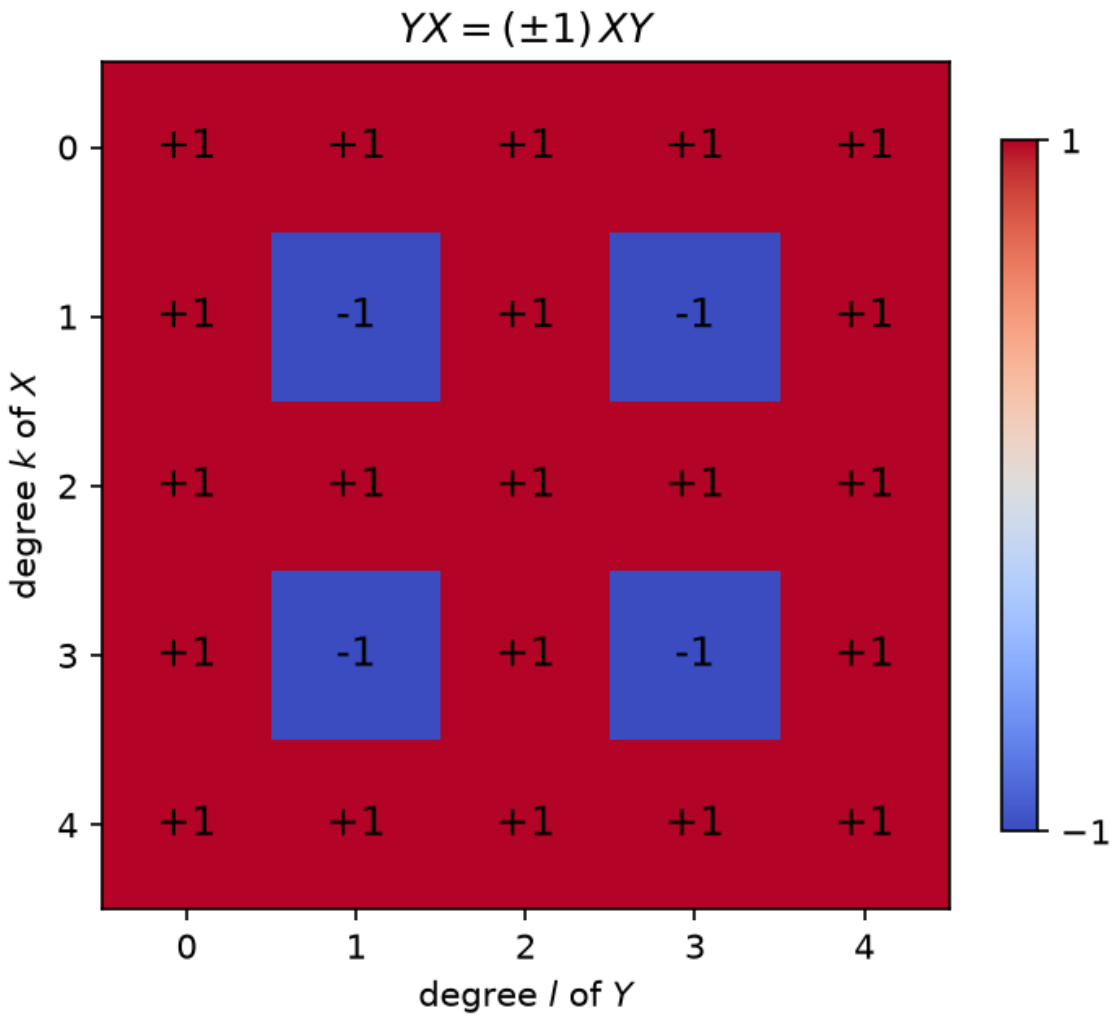

Figure 214: The sign in $YX = \pm XY$ for a monomial X of degree k (row) and a monomial Y of degree l (column) without a common generator, measured by the program for $k, l = 0$ to 4. Red $+1$: they commute; blue -1: they anticommute. The pattern is $(-1)^{kl}$: only two odd monomials anticommute, and an even monomial (rows and columns 0, 2, 4) commutes with everything. (Notebook 07a, figure 3.)

Out [8]:

```
Figure 07a.3 saved as Revision/textbook/figures/07a_3_commutation_signs.png
```

Text cell:

```
## 10. Conjugation reverses the order

For complex Grassmann numbers the generators come in pairs, $\theta_k$ and
$\bar\theta_k = \theta_k^*$. The conjugation exchanges the members of every pair,
conjugates every coefficient, and reverses every product:

$$(\theta_1\theta_2)^* = \theta_2^*\theta_1^* = \bar\theta_2\bar\theta_1 .$$

Why reverse? With this rule the conjugate of $\bar\theta\theta$ is
$\theta^*\bar\theta^* = \bar\theta\theta$: the product of a complex Grassmann
number with its conjugate is real, as for ordinary numbers. The function
‘conjugate‘ below takes a list ‘partner‘, with ‘partner[g]‘ the generator that
the conjugation assigns to generator ‘g‘, reverses each monomial, replaces each
generator by its partner, sorts back with ‘sort_with_sign‘ and conjugates the
coefficient with sympy.

The cell first defines the helper ‘reproduces(report, name)‘: it reads the Revision
report (a JSON file), makes sure that it contains the check ‘name‘ with the verdict
PASS, and returns the text that the PASS line prints after "reproduces". The cell
then checks the rules that the Revision record checks for its own algebras. For
the Wolfram test algebra (generators $\theta_1, \theta_2$ and
$\bar\theta_1, \bar\theta_2$, numbered 0, 1, 2, 3 here): anticommutation,
$\theta_i^2 = 0$, $(\theta_1\theta_2)^* = \bar\theta_2\bar\theta_1$ and
$(F^*)^* = F$. For the jet super-algebra of the sympy record, with 16 complex
components $\psi_A$ and their conjugates $\chi_A = \psi_A^*$ (numbered $A$ and
$16 + A$ here): $\psi\psi = 0$, $\psi_A\psi_B = -\psi_B\psi_A$,
$(\chi_A\psi_B)^* = \chi_B\psi_A$ (order reversal) and $(i\chi_A\psi_A)^* =
-i\chi_A\psi_A$.
```

In [9]:

```
def reproduces(report, name):
    """The text "<report>, check <name>" for a PASS line, after making sure that
    the Revision report (a JSON file) contains the check name with verdict PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return f"{report}, check {name}"


WLREP = "Revision/theory/reports/wolfram-field-theory.json"
PYREP = "Revision/theory/reports/python-field-theory.json"


def conjugate(element, partner):
    """The conjugate of element: generator g -> partner[g], the order of every
    product reversed, every coefficient complex-conjugated."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        images = [partner[g] for g in reversed(monomial)]  # reversed order
        sign, ordered = sort_with_sign(images)  # back to increasing order
        if sign != 0:
```

```
            result.add(ordered, sign * sp.conjugate(coefficient))
    return result


# the Wolfram test algebra: 0 = theta_1, 1 = theta_2, 2 = thetabar_1, 3 = thetabar_2
PAIR4 = [2, 3, 0, 1]
th1, th2, tb1, tb2 = theta(0), theta(1), theta(2), theta(3)
general = (number(sp.Rational(1, 2) + 2 * sp.I) + th1.scaled(3 - sp.I)
           + (th2 * tb1).scaled(-4 * sp.I) + (th1 * th2 * tb2).scaled(7))
check((th1 * th2 + th2 * th1).is_zero() and (th1 * th1).is_zero()
      and (conjugate(th1 * th2, PAIR4) - tb2 * tb1).is_zero()
      and (conjugate(conjugate(general, PAIR4), PAIR4) - general).is_zero(),
      "anticommutation, theta^2 = 0, (theta_1 theta_2)* = thetabar_2 thetabar_1, "
      "(F*)* = F",
      record=reproduces(WLREP, "grassmann_algebra_structure"))

# the jet super-algebra of the sympy record: psi_A = generator A, chi_A = 16 + A
PAIR32 = [16 + A for A in range(16)] + list(range(16))
psi = [theta(A) for A in range(16)]
chi = [theta(16 + A) for A in range(16)]
rules = ((psi[0] * psi[0]).is_zero() and (psi[0] * psi[1] + psi[1] * psi[0]).is_zero()
         and (conjugate(chi[0] * psi[1], PAIR32) - chi[1] * psi[0]).is_zero()
         and (conjugate((chi[0] * psi[0]).scaled(sp.I), PAIR32)
              + (chi[0] * psi[0]).scaled(sp.I)).is_zero())
check(rules, "psi psi = 0, psi_A psi_B = -psi_B psi_A, (chi_A psi_B)* = chi_B "
      "psi_A, (i chi_A psi_A)* = -i chi_A psi_A",
      record=reproduces(PYREP, "superalgebra_axioms"))
check((conjugate(chi[3] * psi[3], PAIR32) - chi[3] * psi[3]).is_zero(),
      "chi_A psi_A (a component times its conjugate) is real")
```

Out [9]:

```
PASS anticommutation, theta^2 = 0, (theta_1 theta_2)* = thetabar_2 thetabar_1, (F*)* = F
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    grassmann_algebra_structure
PASS psi psi = 0, psi_A psi_B = -psi_B psi_A, (chi_A psi_B)* = chi_B psi_A, (i chi_A
    psi_A)* = -i chi_A psi_A
     reproduces Revision/theory/reports/python-field-theory.json, check
    superalgebra_axioms
PASS chi_A psi_A (a component times its conjugate) is real
```

Text cell:

```
## 11. Left and right derivatives

The left derivative $\partial_L F/\partial\theta_k$ moves $\theta_k$ to the far
left of each monomial that contains it (a $-1$ for every generator passed) and
deletes it; the right derivative moves it to the far right. In a sorted monomial
$\theta_k$ stands at a position $p$ (counting from 0); moving it left passes $p$
generators, moving it right passes $\text{degree} - 1 - p$. Examples worked by
hand for $F = \theta_0\theta_1$: $\partial_L F/\partial\theta_0 = \theta_1$,
$\partial_L F/\partial\theta_1 = -\theta_0$ (write $F = -\theta_1\theta_0$),
```

$\partial_R F/\partial\theta_1 = \theta_0$, $\partial_R F/\partial\theta_0 =
-\theta_1$.

For an EVEN monomial of degree n the two signs differ by
$(-1)^{(n-1-p) - p} = (-1)^{n-1} = -1$, so $\partial_R F = -\partial_L F$ for
every even F. The next cell defines both derivatives, checks the four examples,
the rule for even elements on a random even element, and the derivative rules of
the sympy record: for $x = \chi_0$, $a = \psi_0$, $y = \psi_1$ and $p = xay$:
$\partial_L p/\partial y = \partial_R p/\partial y = xa$ and $\partial_L
p/\partial a = \partial_R p/\partial a = -xy$; for $q = xa$: $\partial_L
q/\partial a = -x$ and $\partial_R q/\partial a = +x$.

In [10]:

```
def left_derivative(element, k):
    """d_L element / d theta_k: move theta_k to the far left, then delete it."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        if k in monomial:
            p = monomial.index(k)  # the number of generators to its left
            result.add(monomial[:p] + monomial[p + 1:], (-1) ** p * coefficient)
    return result


def right_derivative(element, k):
    """d_R element / d theta_k: move theta_k to the far right, then delete it."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        if k in monomial:
            p = monomial.index(k)
            passed = len(monomial) - 1 - p  # generators to its right
            result.add(monomial[:p] + monomial[p + 1:], (-1) ** passed * coefficient)
    return result


F = t0 * t1
check((left_derivative(F, 0) - t1).is_zero() and (left_derivative(F, 1) + t0).is_zero()
      and (right_derivative(F, 1) - t0).is_zero()
      and (right_derivative(F, 0) + t1).is_zero(),
      "the four derivatives of theta_0 theta_1 worked by hand")
even = Grassmann()
for trial in range(40):  # a random even element of the algebra of 8 generators
    size = 2 * int(rng.integers(1, 4))  # degree 2, 4 or 6
    monomial = tuple(sorted(rng.choice(8, size=size, replace=False)))
    even.add(monomial, int(rng.integers(-9, 10)))
check(all((right_derivative(even, k) + left_derivative(even, k)).is_zero()
          for k in range(8)), "d_R F = -d_L F for a random even element F")
x, a_, y = chi[0], psi[0], psi[1]  # x = chi_0 (generator 16), a = psi_0, y = psi_1
p_, q_ = x * a_ * y, x * a_
rules = ((left_derivative(p_, 1) - x * a_).is_zero()
         and (right_derivative(p_, 1) - x * a_).is_zero()
         and (left_derivative(p_, 0) + x * y).is_zero()
         and (right_derivative(p_, 0) + x * y).is_zero()
```

```
         and (left_derivative(q_, 0) + x).is_zero()
         and (right_derivative(q_, 0) - x).is_zero())
check(rules, "left and right derivatives with the Grassmann signs",
      record=reproduces(PYREP, "superalgebra_axioms"))
```

Out [10]:

```
PASS the four derivatives of theta_0 theta_1 worked by hand
PASS d_R F = -d_L F for a random even element F
PASS left and right derivatives with the Grassmann signs
     reproduces Revision/theory/reports/python-field-theory.json, check
    superalgebra_axioms
```

Text cell:

```
## 12. Bilinear forms: when is $\Psi^\dagger M \Psi$ real?

Take a column $\Psi$ of $n$ complex Grassmann numbers (generators $\psi_A$, numbered
$A$) and its conjugate row $\Psi^\dagger$ (generators $\chi_A = \psi_A^*$, numbered
$n + A$). For a matrix $M$ of ordinary numbers the conjugation rules give, in two
lines,

$$\big(\Psi^\dagger M \Psi\big)^* = \sum_{A,B} \big(\chi_A M_{AB} \psi_B\big)^*
= \sum_{A,B} M_{AB}^*\, \chi_B \psi_A = \Psi^\dagger M^\dagger \Psi .$$

The first equality is the definition of the bilinear form; the second is the
conjugation rule $(\chi_A\psi_B)^* = \psi_B^*\chi_A^* = \chi_B\psi_A$ (order
reversed, partners exchanged) with the conjugated coefficient; the third renames
the summation indices and uses $(M^\dagger)_{BA} = M_{AB}^*$. So the form is real
exactly when $M$ is Hermitian, the same rule as for ordinary complex numbers. The
reversal of the order is what makes the rule come out the same. The next cell
checks this with $n = 4$ for a random Hermitian $M$ (real) and a random
non-Hermitian $M$ (not real).
```

In [11]:

```
def bilinear(row, M, column):
    """sum over A, B of row[A] M[A, B] column[B] (one factor from each side)."""
    total = Grassmann()
    for A in range(len(row)):
        for B in range(len(column)):
            if M[A, B] != 0:
                total = total + (row[A] * column[B]).scaled(M[A, B])
    return total


n4 = 4
psi4 = [theta(A) for A in range(n4)]  # psi_A: generators 0 .. 3
chi4 = [theta(n4 + A) for A in range(n4)]  # chi_A = psi_A*: generators 4 .. 7
PAIR8 = [n4 + A for A in range(n4)] + list(range(n4))
random_matrix = sp.Matrix(n4, n4, lambda i, j: int(rng.integers(-3, 4))
```

```
                          + sp.I * int(rng.integers(-3, 4)))
hermitian = random_matrix + random_matrix.H  # M + M^dagger is Hermitian
form_h = bilinear(chi4, hermitian, psi4)
form_n = bilinear(chi4, random_matrix, psi4)
check((conjugate(form_h, PAIR8) - form_h).is_zero(),
      "Psi^dagger M Psi is real for a Hermitian M")
not_hermitian = random_matrix.H != random_matrix  # True: M^dagger differs from M
check(not_hermitian and not (conjugate(form_n, PAIR8) - form_n).is_zero(),
      "Psi^dagger M Psi is not real for a non-Hermitian M")
```

Out [11]:

```
PASS Psi^dagger M Psi is real for a Hermitian M
PASS Psi^dagger M Psi is not real for a non-Hermitian M
```

Text cell:

```
## 13. Real Grassmann columns and the author's matrix $C$

Now take a REAL column $\Theta = (\theta_0, \dots, \theta_{15})^T$ of 16 real
Grassmann generators (no conjugates at all) and the form $\Theta^T M \Theta =
\sum_{A,B} \theta_A M_{AB} \theta_B$. Exchanging the two odd factors and renaming
the indices gives, line by line,

$$\Theta^T M \Theta = \sum_{A,B} \theta_A M_{AB} \theta_B = -\sum_{A,B} \theta_B
M_{AB} \theta_A = -\sum_{A,B} \theta_A (M^T)_{AB} \theta_B = -\Theta^T M^T \Theta .$$

The second step is the anticommutation of $\theta_A$ and $\theta_B$; the third
renames $A \leftrightarrow B$ and uses $(M^T)_{AB} = M_{BA}$. Hence the symmetric
part of $M$ drops out: **for a real Grassmann column only the antisymmetric part of
$M$ survives, and $\Theta^T M \Theta = 0$ for every symmetric $M$.** For a real
column $q$ of ordinary numbers it is the other way round: $q^T M q = q^T M^T q$, so
only the symmetric part survives and $q^T M q = 0$ for every antisymmetric $M$.

The author's matrix $C$ is symmetric and each $C\gamma^{(a)}$ is antisymmetric.
The next cell reads $C$ and the eight gamma matrices from the Revision record
Revision/algebra/gammas.json, checks these two properties, and computes the nine
forms $\Theta^T C \Theta$ and $\Theta^T C \gamma^{(a)} \Theta$ ($a = x_0, \dots,
x_7$) both for real Grassmann $\Theta$ (with the program) and for real ordinary
numbers $q$ (with sympy), counting the monomials that survive.
```

In [12]:

```
gammas_file = "Revision/algebra/gammas.json"
fixture = json.loads(repository_file(gammas_file).read_text(encoding="utf-8"))
G = [np.array(g, dtype=int) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7)
C = np.array(fixture["C"], dtype=int)  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
check(np.array_equal(C, G[0] @ G[1] @ G[2] @ G[3]), "C = gamma^(x0) gamma^(x1) "
      "gamma^(x2) gamma^(x3) (read from Revision/algebra/gammas.json)")
check(np.array_equal(C.T, C) and np.array_equal(C @ C, np.eye(16, dtype=int))
      and all(np.array_equal((C @ g).T, -(C @ g)) for g in G),
```

```
      "C is symmetric, C^2 = 1, and every C gamma^(a) is antisymmetric",
      record=reproduces(PYREP, "C_properties"))

Theta = [theta(A) for A in range(16)]  # a real Grassmann column
q = sp.Matrix(sp.symbols("q0:16"))  # a real column of ordinary numbers
labels = ["$C$"] + [f"$C\\gamma^{{(x_{a})}}$" for a in range(8)]
matrices = [sp.Matrix(C)] + [sp.Matrix(C @ g) for g in G]  # exact sympy matrices
grassmann_counts, commuting_counts = [], []
for M in matrices:
    grassmann_counts.append(len(bilinear(Theta, M, Theta).terms))
    form = sp.expand((q.T * M * q)[0, 0])
    commuting_counts.append(0 if form == 0 else len(sp.Add.make_args(form)))
say(f"monomials, real Grassmann: {grassmann_counts}")
say(f"monomials, real commuting: {commuting_counts}")
```

Out [12]:

```
PASS C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3) (read from
    Revision/algebra/gammas.json)
PASS C is symmetric, C^2 = 1, and every C gamma^(a) is antisymmetric
     reproduces Revision/theory/reports/python-field-theory.json, check C_properties
monomials, real Grassmann: [0, 8, 8, 8, 8, 8, 8, 8, 8]
monomials, real commuting: [8, 0, 0, 0, 0, 0, 0, 0, 0]
```

Text cell:

```
The next cell checks the pattern and the coefficients: $\Theta^T C \Theta$ has no
monomial, while each $\Theta^T C\gamma^{(a)}\Theta$ has 8 monomials with
coefficients $\pm 2$ (each antisymmetric pair of entries $\pm1$ of $C\gamma^{(a)}$
gives $\theta_A\theta_B - \theta_B\theta_A = 2\theta_A\theta_B$). For real ordinary
numbers $q^T C q$ has 8 monomials and every $q^T C\gamma^{(a)} q$ vanishes. The
first of these facts is part of the Revision check that the Majorana-type
Lagrangian of a real anticommuting field has no mass term; the last is part of the
check that its contrast for commuting components states. Figure 4 shows the
counts and Figure 5 the matrices $C$ and $C\gamma^{(x_1)}$.
```

In [13]:

```
two = all(abs(value) == 2 for M in matrices[1:]
          for value in bilinear(Theta, M, Theta).terms.values())
check(grassmann_counts == [0] + [8] * 8 and two, "real Grassmann: Theta^T C Theta "
      "= 0; each Theta^T C gamma^(a) Theta has 8 monomials with coefficients +-2",
      record=reproduces(WLREP, "Majorana_Lg_total_derivative_grassmann"))
check(commuting_counts == [8] + [0] * 8, "real commuting: q^T C q has 8 monomials, "
      "every q^T C gamma^(a) q is 0",
      record=reproduces(WLREP, "Majorana_Lg_commuting_control"))

fig, ax = plt.subplots(figsize=(8.4, 4.0))
positions = np.arange(len(labels))
ax.bar(positions - 0.2, grassmann_counts, width=0.4, label="real Grassmann column")
ax.bar(positions + 0.2, commuting_counts, width=0.4, label="real ordinary numbers")
```

```
ax.set_xticks(positions, labels)
ax.set_ylabel("surviving monomials")
ax.set_ylim(0, 10.5)  # room for the legend above the bars
ax.set_title("Which bilinear forms survive for a real column")
ax.legend(loc="upper center", ncol=2)
save_figure(fig, "bilinear_survivors",
            "The number of monomials that survive in the bilinear form $v^T M v$ "
            "of a real 16-component column $v$, for the nine matrices $M = C$ "
            "and $M = C\\gamma^{(a)}$, $a = x_0, \\dots, x_7$ (horizontal axis), "
            "when $v$ consists of real Grassmann generators (blue) or of real "
            "ordinary numbers (orange). The symmetric $C$ survives only for "
            "ordinary numbers, the antisymmetric $C\\gamma^{(a)}$ only for "
            "Grassmann numbers: the two kinds of numbers keep opposite halves of a "
            "matrix.")

fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, M, title in zip(axes, [C, C @ G[1]],
                        ["$C$ (symmetric)", "$C\\gamma^{(x_1)}$ (antisymmetric)"]):
    image = ax.imshow(M, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_xticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_yticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_xlabel("column $B$")
    ax.set_ylabel("row $A$")
    ax.set_title(title)
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8)
save_figure(fig, "symmetric_antisymmetric",
            "The $16 \\times 16$ matrices $C$ (left) and $C\\gamma^{(x_1)}$ (right) "
            "read from the Revision record; row $A$ and column $B$ run from 1 to "
            "16; red $+1$, blue $-1$, grey 0. Each row has exactly one nonzero "
            "entry (both are signed permutation matrices). Mirroring $C$ in its "
            "diagonal gives the same colours (symmetric); mirroring "
            "$C\\gamma^{(x_1)}$ swaps red and blue (antisymmetric).")
```

Out [13]:

```
PASS real Grassmann: Theta^T C Theta = 0; each Theta^T C gamma^(a) Theta has 8 monomials
    with coefficients +-2
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Majorana_Lg_total_derivative_grassmann
PASS real commuting: q^T C q has 8 monomials, every q^T C gamma^(a) q is 0
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Majorana_Lg_commuting_control
```

Out [13]:

```
Figure 07a.4 saved as Revision/textbook/figures/07a_4_bilinear_survivors.png
```

Out [13]:

```
Figure 07a.5 saved as Revision/textbook/figures/07a_5_symmetric_antisymmetric.png
```

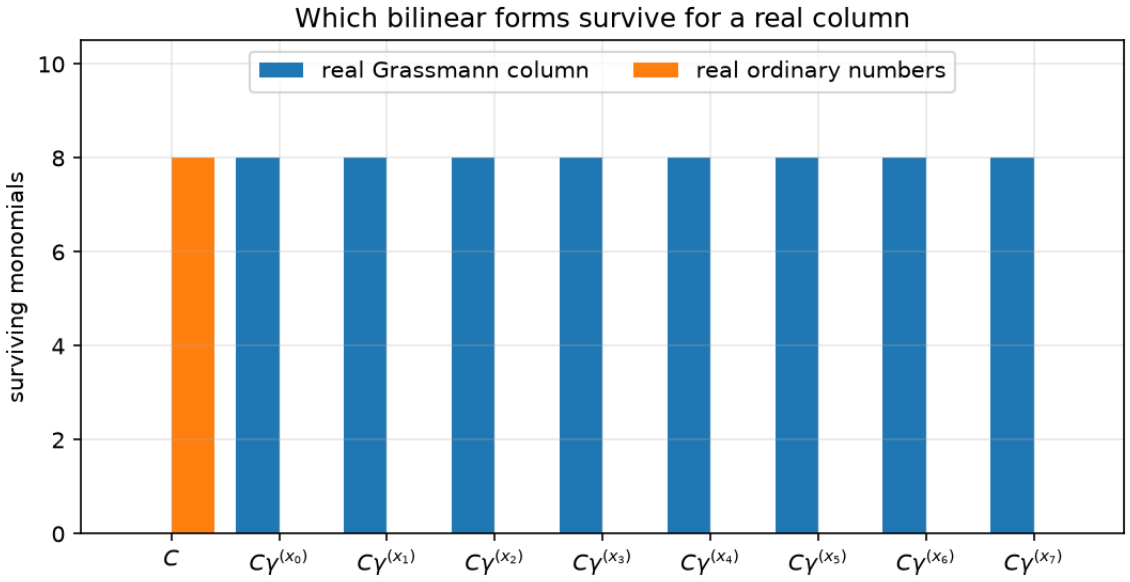

Figure 215: The number of monomials that survive in the bilinear form $v^T M v$ of a real 16-component column v, for the nine matrices $M = C$ and $M = C\gamma^{(a)}$, $a = x_0, \ldots, x_7$ (horizontal axis), when v consists of real Grassmann generators (blue) or of real ordinary numbers (orange). The symmetric C survives only for ordinary numbers, the antisymmetric $C\gamma^{(a)}$ only for Grassmann numbers: the two kinds of numbers keep opposite halves of a matrix. (Notebook 07a, figure 4.)

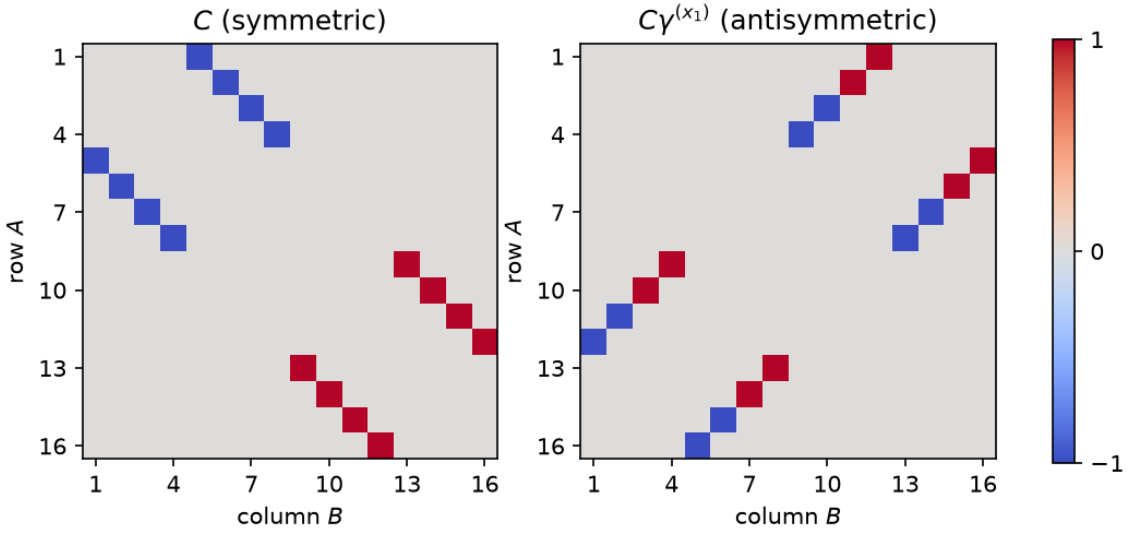

Figure 216: The 16×16 matrices C (left) and $C\gamma^{(x_1)}$ (right) read from the Revision record; row A and column B run from 1 to 16; red $+1$, blue -1, grey 0. Each row has exactly one nonzero entry (both are signed permutation matrices). Mirroring C in its diagonal gives the same colours (symmetric); mirroring $C\gamma^{(x_1)}$ swaps red and blue (antisymmetric). (Notebook 07a, figure 5.)

Text cell:

```
## 14. The scalar density $S$ of a 16-component field and its powers

At one point the field dirac16complex consists of 16 complex Grassmann numbers. We
use 32 generators: $\chi_A = \Psi_A^*$ (numbers 0 to 15) and $\psi_A = \Psi_A$
(numbers 16 to 31). The scalar density is

$$S = \Psi^\dagger C \Psi = \sum_{A,B} \chi_A C_{AB} \psi_B .$$

Every row of $C$ has exactly one entry $\pm1$, in the column $\pi(A)$, so $S =
\sum_A C_{A\pi(A)}\, P_A$ is a sum of 16 pairs $P_A = \chi_A\psi_{\pi(A)}$ that
share no generator. Each pair is even, so the pairs commute with each other, and
$P_A^2 = 0$. In $S^k$ only products of $k$ DIFFERENT pairs survive, and each such
product appears $k!$ times (once for every order of its factors). Hence $S^k$ has
$\binom{16}{k}$ monomials with coefficients $\pm k!$, $S^{16}$ is a single
monomial, and $S^{17} = 0$. Consequence for the Lagrangian: every function of $S$
is a polynomial of degree at most 16, so the potential $U(S) = \frac{\lambda}{2}
S^2$ is a choice among finitely many terms. For ordinary commuting numbers the
same $S$ has powers that never vanish; $S^k$ then has $\binom{16 + k - 1}{k}$
monomials (the number of ways to choose $k$ pairs with repetition).

The next cell computes $S, S^2, \dots, S^{17}$ with the program (about one second),
checks the counts and the coefficients, checks the commuting counts with sympy for
$k \le 3$, and draws Figure 6.
```

In [14]:

```
S = Grassmann()
for A in range(16):
    for B in range(16):
        if C[A, B] != 0:
            S = S + (theta(A) * theta(16 + B)).scaled(int(C[A, B]))  # chi_A C psi_B
power = number(1)
counts, sizes_ok = [], True
for k in range(1, 18):
    power = power * S  # S^k
    counts.append(len(power.terms))
    sizes_ok = sizes_ok and all(abs(value) == math.factorial(k)
                                for value in power.terms.values())
report("monomials of S^k for k = 1 .. 8", counts[:8])
report("monomials of S^k for k = 9 .. 17", counts[8:])
check(counts[:16] == [math.comb(16, k) for k in range(1, 17)] and sizes_ok,
      "S^k has binom(16, k) monomials with coefficients +-k! (k = 1 .. 16)")
check(counts[16] == 0, "S^17 = 0: every function of S is a polynomial of degree <= 16")

# ordinary (commuting) stand-ins for the 16 chi_A and the 16 psi_A
chi_c, psi_c = sp.symbols("chi0:16"), sp.symbols("psi0:16")
S_commuting = sum(int(C[A, B]) * chi_c[A] * psi_c[B]
                  for A in range(16) for B in range(16) if C[A, B] != 0)
commuting = [len(sp.Add.make_args(sp.expand(S_commuting ** k))) for k in (1, 2, 3)]
check(commuting == [math.comb(16 + k - 1, k) for k in (1, 2, 3)],
      "commuting numbers: S^k has binom(15 + k, k) monomials (k = 1, 2, 3)")
```

Out [14]:

```
RESULT monomials of S^k for k = 1 .. 8 = [16, 120, 560, 1820, 4368, 8008, 11440, 12870]
RESULT monomials of S^k for k = 9 .. 17 = [11440, 8008, 4368, 1820, 560, 120, 16, 1, 0]
PASS S^k has binom(16, k) monomials with coefficients +-k! (k = 1 .. 16)
PASS S^17 = 0: every function of S is a polynomial of degree <= 16
PASS commuting numbers: S^k has binom(15 + k, k) monomials (k = 1, 2, 3)
```

Text cell:

```
Figure 6 compares the two kinds of numbers. The next cell draws it.
```

In [15]:

```
ks = np.arange(1, 18)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.0))
left.bar(ks[:16], counts[:16], color="tab:blue", label="Grassmann (computed)")
left.plot(ks, [math.comb(15 + k, k) for k in ks], "s--", color="tab:orange",
          label="ordinary numbers $\\binom{15+k}{k}$")
left.set_yscale("log")
left.set_xticks(range(1, 18, 2))
left.set_xlabel("power $k$")
left.set_ylabel("number of monomials of $S^k$")
left.set_title("$S^{17} = 0$ for Grassmann numbers")
left.legend(fontsize=8)
right.semilogy(ks[:16], [math.factorial(k) for k in ks[:16]], "o-", color="tab:green")
right.set_xticks(range(1, 17, 3))
right.set_xlabel("power $k$")
right.set_ylabel("size of every coefficient of $S^k$")
right.set_title("Coefficients $\\pm k!$")
fig.tight_layout()
save_figure(fig, "powers_of_s",
            "Left: the number of monomials of $S^k$, the $k$-th power of the scalar "
            "density $S = \\Psi^\\dagger C\\Psi$ of a 16-component field at one "
            "point, for $k = 1$ to $17$ (logarithmic vertical axis). Blue bars: "
            "complex Grassmann components, computed by the program; the counts "
            "rise and fall as $\\binom{16}{k}$ and $S^{17} = 0$ (no bar). Orange "
            "squares: ordinary commuting components, $\\binom{15+k}{k}$, which "
            "grow without end. Right: every coefficient of $S^k$ is $\\pm k!$, "
            "from 1 to $16! \\approx 2.1 \\times 10^{13}$.")
```

Out [15]:

```
Figure 07a.6 saved as Revision/textbook/figures/07a_6_powers_of_s.png
```

Text cell:

```
## 15. The Revision test algebra with two generators
```

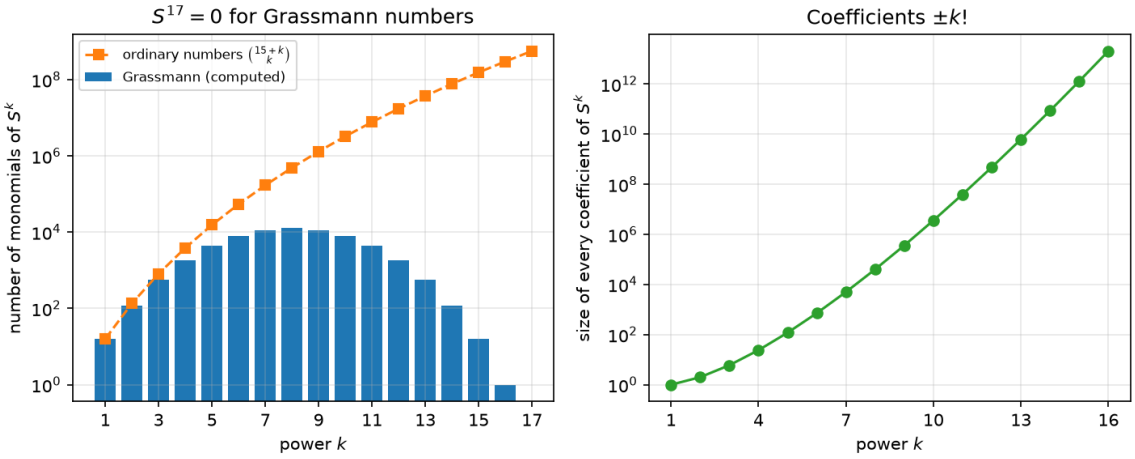

Figure 217: Left: the number of monomials of S^k, the k-th power of the scalar density $S = \Psi^\dagger C \Psi$ of a 16-component field at one point, for $k = 1$ to 17 (logarithmic vertical axis). Blue bars: complex Grassmann components, computed by the program; the counts rise and fall as $\binom{16}{k}$ and $S^{17} = 0$ (no bar). Orange squares: ordinary commuting components, $\binom{15+k}{k}$, which grow without end. Right: every coefficient of S^k is $\pm k!$, from 1 to 16! $\approx 2.1 \times 10^{13}$. (Notebook 07a, figure 6.)

```
The Wolfram verifier of the Revision record does not give every component its own
generator. It uses an algebra with only two generators $\theta_1, \theta_2$ and
their conjugates $\bar\theta_1, \bar\theta_2$, and writes each component as
$\Psi_A = c_{A1}\theta_1 + c_{A2}\theta_2$ with ordinary complex coefficients. That
keeps the computation small and is still a genuine Grassmann field. Then

$$S = \Psi^\dagger C \Psi = \sum_{k,l} \bar\theta_k\theta_l\, \big(c_k^\dagger C
c_l\big),$$

so $S$ lives only on the four monomials $\bar\theta_k\theta_l$ (it is even),
$S^2$ only on $\theta_1\theta_2\bar\theta_1\bar\theta_2$, and $S^3 = 0$ because it
would need six different generators out of four. Hence in this test algebra the
most general potential is $U = u_1 S + \frac{\lambda}{2} S^2$. The next cell
builds such a field with random Gaussian-integer coefficients (fixed seed) and
checks all of this, and that $S$ is real ($C$ is real and symmetric, hence
Hermitian).
```

In [16]:

```
coefficients = [[int(rng.integers(-3, 4)) + sp.I * int(rng.integers(-3, 4))
                 for k in range(2)] for A in range(16)]
Psi = [th1.scaled(coefficients[A][0]) + th2.scaled(coefficients[A][1])
       for A in range(16)]
Psi_dagger = [conjugate(component, PAIR4) for component in Psi]
S2 = bilinear(Psi_dagger, sp.Matrix(C), Psi)  # S in the test algebra
shapes = sorted(S2.terms)
say(f"monomials of S (generator numbers): {shapes}")
say(f"monomials of S^2: {sorted((S2 * S2).terms)}")
check(all(len(m) == 2 and m[0] in (0, 1) and m[1] in (2, 3) for m in shapes)
      and (conjugate(S2, PAIR4) - S2).is_zero(),
```

```
      "S is a real even element on the monomials thetabar_k theta_l")
check(sorted((S2 * S2).terms) == [(0, 1, 2, 3)] and (S2 * S2 * S2).is_zero(),
      "S^2 != 0 lives on theta_1 theta_2 thetabar_1 thetabar_2 and S^3 = 0",
      record=reproduces(WLREP, "grassmann_algebra_structure"))
```

Out [16]:

```
monomials of S (generator numbers): [(0, 2), (0, 3), (1, 2), (1, 3)]
monomials of S^2: [(0, 1, 2, 3)]
PASS S is a real even element on the monomials thetabar_k theta_l
PASS S^2 != 0 lives on theta_1 theta_2 thetabar_1 thetabar_2 and S^3 = 0
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    grassmann_algebra_structure
```

Text cell:

```
## 16. Field equations with Grassmann numbers: the oscillator

A field depends on a coordinate. To vary a Lagrangian we treat the value of a
variable and its derivative at one point as independent symbols (they are called
**jets**). Here the variable is a complex Grassmann number $\theta(t)$ of one
variable $t$; the generators are $\bar\theta$ (number 0), $\theta$ (1),
$\bar\theta'$ (2), $\theta'$ (3), and the second derivatives
$\bar\theta^{\prime\prime}$ (4), $\theta^{\prime\prime}$ (5); the prime is $d/dt$.
The **total derivative** $d/dt$ raises each generator by 2 (from $\theta$ to
$\theta'$, from $\theta'$ to $\theta^{\prime\prime}$) in each place, with the
product rule; it is even, so it needs no extra sign, but the raised generator may
have to be sorted back into place.

The Lagrangian of the oscillator,

$$L = \tfrac{i}{2}\big(\bar\theta\theta' - \bar\theta'\theta\big) -
\omega\,\bar\theta\theta ,$$

is real. Varying $\bar\theta$ from the left gives the Euler-Lagrange expression

$$\mathcal{E} = \frac{\partial_L L}{\partial\bar\theta} - \frac{d}{dt}\,
\frac{\partial_L L}{\partial\bar\theta'}
= \Big(\tfrac{i}{2}\theta' - \omega\theta\Big) - \frac{d}{dt}\Big(-\tfrac{i}{2}
\theta\Big) = i\theta' - \omega\theta .$$

The first term: $\bar\theta$ already stands on the left of $\bar\theta\theta'$
and $\bar\theta\theta$; the second: $-\frac{i}{2}\bar\theta'\theta$ has
$\bar\theta'$ on the left, its derivative is $-\frac{i}{2}\theta$, whose $d/dt$ is
$-\frac{i}{2}\theta'$. The field equation $i\theta' = \omega\theta$ has the same
form as for an ordinary complex oscillator. The next cell computes $\mathcal{E}$
with the program and checks that $L$ is real.
```

In [17]:

```
def total_derivative(element, step):
    """d/dt on jets: every generator g in every place becomes g + step (from a
```

```
    variable to its derivative), with the product rule; the raised monomial is
    sorted back with its sign."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        for place, g in enumerate(monomial):
            raised = monomial[:place] + (g + step,) + monomial[place + 1:]
            sign, ordered = sort_with_sign(raised)
            if sign != 0:
                result.add(ordered, sign * coefficient)
    return result


omega = sp.Symbol("omega", positive=True)  # the angular frequency
bar, th, bar_d, th_d = theta(0), theta(1), theta(2), theta(3)
L_osc = ((bar * th_d - bar_d * th).scaled(sp.I / 2) - (bar * th).scaled(omega))
E_osc = left_derivative(L_osc, 0) - total_derivative(left_derivative(L_osc, 2), 2)
PAIR6 = [1, 0, 3, 2, 5, 4]  # theta-bar <-> theta for the value and two derivatives
say(f"Euler-Lagrange expression (monomial: coefficient): {E_osc.terms}")
check((E_osc - (th_d.scaled(sp.I) - th.scaled(omega))).is_zero(),
      "the oscillator's Euler-Lagrange expression is i theta' - omega theta")
check((conjugate(L_osc, PAIR6) - L_osc).is_zero(), "the oscillator Lagrangian is real")
```

Out [17]:

```
Euler-Lagrange expression (monomial: coefficient): {(3,): I, (1,): -omega}
PASS the oscillator's Euler-Lagrange expression is i theta' - omega theta
PASS the oscillator Lagrangian is real
```

Text cell:

```
The solution of $i\theta' = \omega\theta$ is $\theta(t) = e^{-i\omega t}\,\theta_0$
with a constant Grassmann generator $\theta_0$: an ordinary complex function times a
fixed generator. Indeed $i\theta' = i(-i\omega)e^{-i\omega t}\theta_0 = \omega
\theta$. The next cell checks this with sympy (the generator is a common factor and
drops out) and plots the ordinary function $e^{-i\omega t}$ for $\omega = 2$
(Figure 7).
```

In [18]:

```
t = sp.Symbol("t", real=True)
solution = sp.exp(-sp.I * omega * t)  # theta(t) = solution * theta_0
residual = sp.simplify(sp.I * sp.diff(solution, t) - omega * solution)
check(residual == 0, "theta(t) = exp(-i omega t) theta_0 solves i theta' = omega theta")

times = np.linspace(0.0, 2.0 * np.pi, 400)
values = np.exp(-2.0j * times)  # omega = 2
fig, ax = plt.subplots()
ax.plot(times, values.real, label="real part $\\cos 2t$")
ax.plot(times, values.imag, "--", label="imaginary part $-\\sin 2t$")
ax.plot(times, np.abs(values), ":", color="black", label="absolute value 1")
ax.set_xlabel("time $t$")
```

```
ax.set_ylabel("coefficient of $\\theta_0$")
ax.set_title("$\\theta(t) = e^{-2it}\\,\\theta_0$")
ax.set_ylim(-1.75, 1.25)  # room for the legend below the curves
ax.legend(loc="lower center", ncol=3, fontsize=9)
save_figure(fig, "oscillator_solution",
            "The solution $\\theta(t) = e^{-i\\omega t}\\theta_0$ of the Grassmann "
            "oscillator for $\\omega = 2$ is an ordinary function times a fixed "
            "generator $\\theta_0$; the plot shows that function: its real part "
            "$\\cos 2t$ (solid), its imaginary part $-\\sin 2t$ (dashed) and its "
            "absolute value 1 (dotted), for $t$ from 0 to $2\\pi$ (pure numbers). "
            "The anticommuting nature sits in $\\theta_0$, not in the time "
            "dependence.")
```

Out [18]:

```
PASS theta(t) = exp(-i omega t) theta_0 solves i theta' = omega theta
```

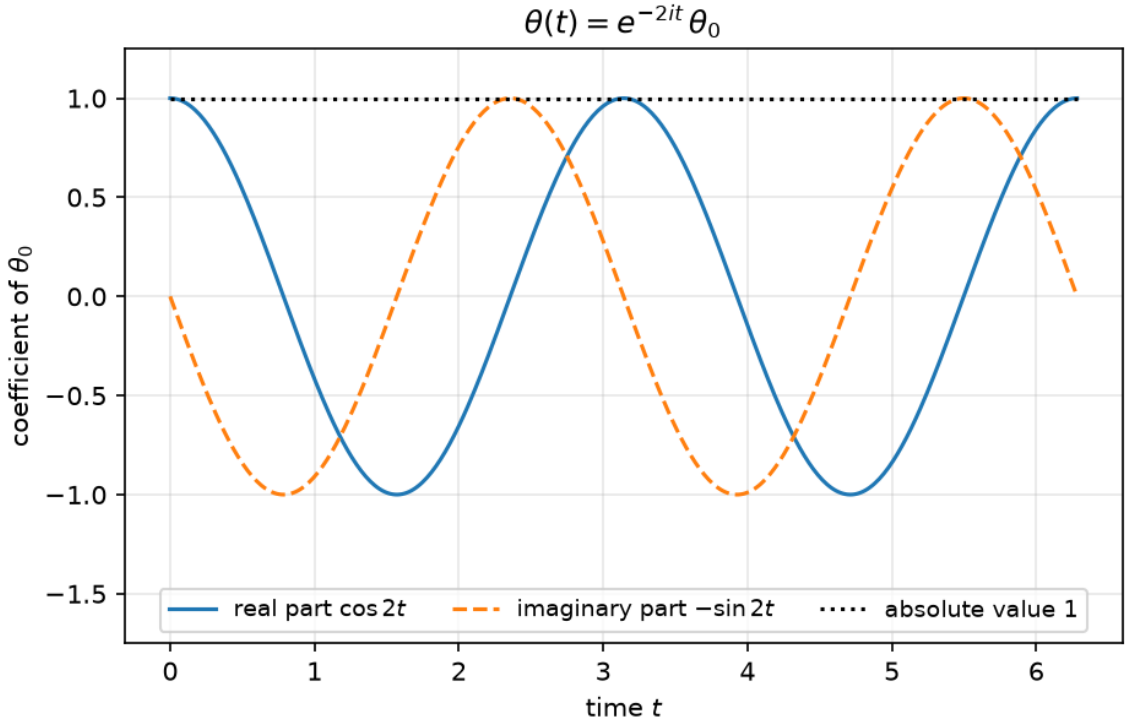

Figure 218: The solution $\theta(t) = e^{-i\omega t}\theta_0$ of the Grassmann oscillator for $\omega = 2$ is an ordinary function times a fixed generator θ_0; the plot shows that function: its real part $\cos 2t$ (solid), its imaginary part $-\sin 2t$ (dashed) and its absolute value 1 (dotted), for t from 0 to 2π (pure numbers). The anticommuting nature sits in θ_0, not in the time dependence. (Notebook 07a, figure 7.)

Out [18]:

```
Figure 07a.7 saved as Revision/textbook/figures/07a_7_oscillator_solution.png
```

Text cell:

```
## 17. A first-order kinetic term of real Grassmann fields

Now two REAL Grassmann fields $\theta_0(x)$, $\theta_1(x)$ of one coordinate $x$
(generators: $\theta_0$ = 0, $\theta_1$ = 1, $\theta_0'$ = 2, $\theta_1'$ = 3,
$\theta_0^{\prime\prime}$ = 4, $\theta_1^{\prime\prime}$ = 5) and the first-order
kinetic term with the antisymmetric matrix $A = \begin{pmatrix} 0 & 1 \\ -1 & 0
\end{pmatrix}$:

$$K_A = \theta^T A\, \theta' = \theta_0\theta_1' - \theta_1\theta_0' .$$

Moving $\theta_0'$ to the left in the second term (one exchange) gives
$K_A = \theta_0\theta_1' + \theta_0'\theta_1 = (\theta_0\theta_1)'$ by the
product rule, and $\theta^T A\theta = \theta_0\theta_1 - \theta_1\theta_0 =
2\theta_0\theta_1$. So $K_A = \frac12(\theta^T A\theta)'$ is a **total
derivative**, and a total derivative gives no field equation: both
Euler-Lagrange expressions vanish identically. With the symmetric identity matrix
instead, $K_I = \theta_0\theta_0' + \theta_1\theta_1'$ is not a derivative and its
Euler-Lagrange expression for $\theta_0$ is $\theta_0' - \frac{d}{dx}(-\theta_0) =
2\theta_0'$. For ORDINARY numbers $q_0, q_1$ the roles are exchanged: $K_A$ gives
the nonzero expressions $2q_1'$ and $-2q_0'$. This is the one-dimensional model of
the Majorana-type negative control of the Revision record (a real anticommuting
field with the antisymmetric kinetic matrices $C\gamma^\mu$ has no field equation),
which Notebook 07c computes in the author's metric. The next cell checks all of
it, with sympy for the ordinary numbers.
```

In [19]:

```
th0, th1_, th0_d, th1_d = theta(0), theta(1), theta(2), theta(3)
K_A = th0 * th1_d - th1_ * th0_d  # theta^T A theta' with A = ((0, 1), (-1, 0))
K_I = th0 * th0_d + th1_ * th1_d  # theta^T I theta'


def euler_lagrange(L, field):
    """dL/d(field) - d/dx dL/d(field') with left derivatives (field = 0 or 1)."""
    return left_derivative(L, field) - total_derivative(left_derivative(L, field + 2), 2)


check((K_A - total_derivative((th0 * th1_ - th1_ * th0).scaled(sp.Rational(1, 2)),
                              2)).is_zero(),
      "K_A = (1/2) d/dx (theta^T A theta): a total derivative")
check(euler_lagrange(K_A, 0).is_zero() and euler_lagrange(K_A, 1).is_zero(),
      "real Grassmann fields: both Euler-Lagrange expressions of K_A vanish")
check((euler_lagrange(K_I, 0) - th0_d.scaled(2)).is_zero(),
      "real Grassmann fields: K_I gives the expression 2 theta_0'")

from sympy.calculus.euler import euler_equations  # sympy's Euler-Lagrange

xs = sp.Symbol("x", real=True)
q0, q1 = sp.Function("q0")(xs), sp.Function("q1")(xs)  # two ordinary functions
K_A_commuting = q0 * sp.diff(q1, xs) - q1 * sp.diff(q0, xs)
equations = euler_equations(K_A_commuting, [q0, q1], xs)
expressions = [sp.simplify(equation.lhs - equation.rhs) for equation in equations]
say(f"ordinary numbers, Euler-Lagrange expressions of K_A: {expressions}")
```

```
check(sp.simplify(expressions[0] - 2 * sp.diff(q1, xs)) == 0
      and sp.simplify(expressions[1] + 2 * sp.diff(q0, xs)) == 0,
      "ordinary numbers: K_A gives the nonzero expressions 2 q1' and -2 q0'")
```

Out [19]:

```
PASS K_A = (1/2) d/dx (theta^T A theta): a total derivative
PASS real Grassmann fields: both Euler-Lagrange expressions of K_A vanish
PASS real Grassmann fields: K_I gives the expression 2 theta_0'
ordinary numbers, Euler-Lagrange expressions of K_A: [2*Derivative(q1(x), x),
    -2*Derivative(q0(x), x)]
PASS ordinary numbers: K_A gives the nonzero expressions 2 q1' and -2 q0'
```

Text cell:

```
## 18. The last check

The last cell checks that the seven figure files exist and prints the number of
checks that passed.
```

In [20]:

```
names = ["dimension_and_degrees", "multiplication_signs", "commutation_signs",
         "bilinear_survivors", "symmetric_antisymmetric", "powers_of_s",
         "oscillator_solution"]
files = [f"{FIGURE_FOLDER}/07a_{k}_{name}.png" for k, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 7 figure files of this notebook exist")
all_checks_passed()
```

Out [20]:

```
PASS all 7 figure files of this notebook exist
ALL 38 CHECKS PASSED (notebook 07a)
```

Text cell:

```
## 19. What this notebook showed

- A Grassmann algebra is completely fixed by $\theta_i\theta_j = -\theta_j\theta_i$:
  a product is the sorted monomial times the sign $(-1)^{\text{exchanges}}$, or 0
  when a generator repeats. With $n$ generators there are exactly $2^n$
  monomials, $\binom{n}{k}$ of degree $k$.
- Even elements commute with everything, odd ones anticommute
  ($YX = (-1)^{kl}XY$); the Lagrangian, which contains the field in pairs, is even.
- The conjugation reverses the order of products; with that rule a bilinear form
  $\Psi^\dagger M\Psi$ is real exactly when $M$ is Hermitian, for Grassmann and for
  ordinary numbers alike. For even elements the right derivative is minus the left
```

```
  derivative.
- For a REAL column the two kinds of numbers keep opposite halves of a matrix: real
  Grassmann numbers keep the antisymmetric part ($\Theta^T C\Theta = 0$, every
  $\Theta^T C\gamma^{(a)}\Theta$ survives), real ordinary numbers the symmetric
  part.
- The scalar density $S$ of the 16-component complex Grassmann field satisfies
  $S^{17} = 0$; $S^k$ has $\binom{16}{k}$ monomials with coefficients $\pm k!$. In
  the two-generator test algebra of the Revision record $S^3 = 0$ already.
- With Grassmann numbers written with every conjugate on the left, field equations
  keep their ordinary form (the oscillator gives $i\theta' = \omega\theta$); but a
  first-order kinetic term of REAL Grassmann fields with an antisymmetric matrix is
  a total derivative and gives no field equation. This is why the Lagrangian of
  dirac16complex uses the complex field with $\bar\Psi = \Psi^\dagger C$.
- The rules reproduce the Grassmann-algebra checks of the Revision record
  (superalgebra_axioms, grassmann_algebra_structure, C_properties, and the parts of
  Majorana_Lg_total_derivative_grassmann and Majorana_Lg_commuting_control that
  concern the bilinear forms).
```

7.17 Line-by-line walk-through of Notebook 07a

The notebook has 20 code cells, In [1] to In [20]. This section explains every line of every one of them, with the conventions of Section 7.9 (comments, docstrings, and `...`) for the rest of a long caption, which is printed in full under its figure in Section 7.16).

In [1], the set-up cell. It is the set-up cell of Notebook 07d with one difference: the line `NOTEBOOK_ID = "07a"`. Its first part repeats the complete run instructions of Section 7.15 as comment lines; its code is explained line by line in Section 7.9. **Out [1]:** `Set-up of notebook 07a complete: repository folder found, helpers defined.`

In [2]: the sign of a reordering.

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import math  # factorials and binomial coefficients
from bisect import bisect_right  # counts the entries of a sorted tuple that are <= y
from collections import Counter  # counts how often each value occurs

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
```

The modules used by the notebook: `contextlib` and `io` for the combined printing of a PASS line (explained below), `math` for factorials and binomial coefficients, `bisect_right` from the module `bisect` (it finds how many entries of a sorted tuple are at most a given number), `Counter` from `collections` (it counts how often each value occurs), numpy and sympy.

```
check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
```

```
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece
```

The same wrapper of `check` as in Notebook 07d (Section 7.9, In [2]): the set-up cell's check runs while its printed text is collected in `buffer`, and the collected text is then printed in one piece, so that a PASS line and its "reproduces" line reach the book's tools together.

```
def sort_with_sign(generators):
    """Sort generator numbers by exchanging neighbours (bubble sort).
    Return (sign, sorted tuple): sign = +1 or -1 for an even or odd number of
    exchanges, and sign = 0 if a generator occurs twice (the product is zero)."""
    items = list(generators)  # a copy that we may reorder
    sign = 1
    for end in range(len(items) - 1, 0, -1):  # pass over items[0 .. end]
        for i in range(end):  # every neighbour pair of this pass
            if items[i] > items[i + 1]:  # a pair in the wrong order:
                items[i], items[i + 1] = items[i + 1], items[i]  # exchange it,
                sign = -sign  # which costs one factor -1
    if any(items[i] == items[i + 1] for i in range(len(items) - 1)):
        return 0, tuple(items)  # a repeated generator: theta_i theta_i = 0
    return sign, tuple(items)
```

`sort_with_sign(generators)` sorts a list of generator numbers by the method called **bubble sort** and counts the exchanges. `items` is a copy of the input that may be reordered, and `sign` starts at +1. The outer loop makes passes over shorter and shorter beginnings of the list (`range(len(items) - 1, 0, -1)` counts down from the last position to 1); in each pass the inner loop looks at every neighbour pair, and when a pair stands in the wrong order it is exchanged (`items[i], items[i + 1] = items[i + 1], items[i]` swaps two entries in one line) and the sign is reversed. After a pass the largest remaining number has bubbled to the end, so the list is sorted after the last pass. If two equal neighbours remain, a generator occurs twice and the function returns the sign 0 (the product contains $\theta_i\theta_i = 0$); otherwise it returns the sign and the sorted tuple.

```
for example in [(0, 1), (1, 0), (2, 0, 1), (2, 1, 0), (0, 0), (3, 1, 2, 0)]:
    sign, ordered = sort_with_sign(example)
    say(f"generators {example} -> sign {sign:+d}, sorted {ordered}")
check(sort_with_sign((1, 0)) == (-1, (0, 1)), "theta_1 theta_0 = -theta_0 theta_1")
check(sort_with_sign((2, 0, 1)) == (1, (0, 1, 2)),
      "theta_2 theta_0 theta_1 = +theta_0 theta_1 theta_2 (two exchanges)")
check(sort_with_sign((0, 0))[0] == 0, "theta_0 theta_0 = 0")
```

Six examples are sorted and printed (`{sign:+d}` prints a whole number with its sign). The three checks are the examples of Section 7.10: one exchange for $\theta_1\theta_0$, two for $\theta_2\theta_0\theta_1$, and 0 for $\theta_0\theta_0$. **Out [2]:** six lines such as `generators (2, 0, 1) -> sign +1, sorted (0, 1, 2)` and `generators (3, 1, 2, 0) -> sign -1, sorted (0, 1, 2, 3)` (five exchanges), and three PASS lines.

In [3]: a faster sign rule.

```
def shuffle_sign(left, right):
    """The sign of the product of the sorted monomials left and right: 0 if they
    share a generator, otherwise (-1)**N with N the number of pairs (x in left,
    y in right) with x > y, which is the number of neighbour exchanges needed."""
    if set(left) & set(right):  # a common generator: the product holds theta^2
        return 0
    # len(left) - bisect_right(left, y) entries of left are larger than y
    pairs = sum(len(left) - bisect_right(left, y) for y in right)
    return -1 if pairs % 2 else 1
```

`shuffle_sign(left, right)` gives the sign of the product of two monomials that are already sorted. `set(left) & set(right)` is the set of the generators that both contain; if it is not empty, the product holds a square and the sign is 0. Otherwise, placing `left` before `right`, each pair (x from the left, y from the right) with $x > y$ stands in the wrong order and needs exactly one exchange, and no other pair does. For each y of the right factor, `bisect_right(left, y)` is the number of entries of `left` that are at most y, so `len(left) - bisect_right(left, y)` is the number larger than y. Their total `pairs` decides the sign: -1 if it is odd (`pairs % 2` is the remainder after division by 2), $+1$ if even.

```
rng = np.random.default_rng(12345)  # random numbers with a fixed seed: same every run
agree = 0
for trial in range(2000):
    # two random sorted monomials with generators from 0 .. 9
    left = tuple(sorted(rng.choice(10, size=rng.integers(0, 5), replace=False)))
    right = tuple(sorted(rng.choice(10, size=rng.integers(0, 5), replace=False)))
    agree += shuffle_sign(left, right) == sort_with_sign(left + right)[0]
report("random pairs of monomials on which the two sign rules agree", f"{agree} of 2000")
check(agree == 2000, "shuffle_sign equals sort_with_sign on 2000 random pairs")
```

A random-number generator with the fixed seed 12345. In each of 2000 trials two random sorted monomials are drawn: `rng.integers(0, 5)` is a random degree from 0 to 4 and `rng.choice(10, size=..., replace=False)` that many different generators out of 0 to 9; `sorted` and `tuple` put them in increasing order. `agree` counts the trials in which the fast rule gives the same sign as sorting the joined list (`left + right` joins two tuples; a true comparison counts as 1). **Out [3]:** `RESULT random pairs of monomials on which the two sign rules agree = 2000 of 2000` and the PASS line.

In [4]: the algebra itself.

```
class Grassmann:
    """An element of a Grassmann algebra: self.terms maps every monomial (a sorted
    tuple of generator numbers; () is the number 1) to its coefficient."""

    def __init__(self, terms=None):
        self.terms = {}
        for monomial, coefficient in (terms or {}).items():
            self.add(monomial, coefficient)
```

A **class** is a recipe for a new kind of object. Every object of the class `Grassmann` holds a dictionary `self.terms` from monomials (sorted tuples; `()` is the number 1) to coefficients. `__init__` is the

function that runs when an object is made: it starts with an empty dictionary and adds the given terms one by one (`terms or {}` uses an empty dictionary when none is given).

```
    def add(self, monomial, coefficient):
        """Add coefficient times monomial to this element (in place)."""
        total = self.terms.get(monomial, 0) + coefficient
        if isinstance(total, sp.Basic):  # an exact sympy number or expression:
            total = sp.expand(total)  # multiply out, so that equal terms cancel
        if total == 0:
            self.terms.pop(monomial, None)  # a zero coefficient is not stored
        else:
            self.terms[monomial] = total
```

`add` adds a coefficient times a monomial to the object itself. `self.terms.get(monomial, 0)` is the stored coefficient or 0. If the sum is an exact sympy expression (`isinstance(total, sp.Basic)`), it is multiplied out with `sp.expand`, so that terms that are equal also look equal and cancel. A coefficient that has become zero is removed (`pop(monomial, None)` removes the key if it is there), so an element is zero exactly when its dictionary is empty.

```
    def __add__(self, other):  # self + other
        result = Grassmann(self.terms)
        for monomial, coefficient in other.terms.items():
            result.add(monomial, coefficient)
        return result

    def scaled(self, factor):  # factor * self, factor an ordinary number
        return Grassmann({m: factor * c for m, c in self.terms.items()})

    def __sub__(self, other):  # self - other
        return self + other.scaled(-1)
```

`__add__` defines what `+` does for two elements: a copy of the first with every term of the second added. `scaled(factor)` multiplies every coefficient by an ordinary number. `__sub__` defines `-` as adding the second element multiplied by -1.

```
    def __mul__(self, other):  # self * other: every term times every term
        result = Grassmann()
        for left, a in self.terms.items():
            for right, b in other.terms.items():
                sign = shuffle_sign(left, right)
                if sign != 0:
                    result.add(tuple(sorted(left + right)), sign * a * b)
        return result

    def is_zero(self):
        return not self.terms  # True when no monomial is left
```

`__mul__` defines `*`: every term of the first element times every term of the second. For two monomials `left` and `right` the sign comes from `shuffle_sign`; if it is not 0, the joined and sorted monomial gets the product of the two coefficients times the sign. `is_zero` is true when no monomial is left.

```
def number(value):
    """The ordinary number value as an element of the algebra."""
    return Grassmann({(): value})


def theta(i):
    """The generator number i."""
    return Grassmann({(i,): 1})


say("The class Grassmann and the helpers number and theta are defined.")
```

Two helpers: `number(value)` is an ordinary number as an element (the monomial `()` with that coefficient), and `theta(i)` is the generator number `i` (the monomial `(i,)` with coefficient 1; the comma makes a tuple of one entry). **Out [4]:** `The class Grassmann and the helpers number and theta are defined.`

In [5]: the defining rules and a product worked by hand.

```
t0, t1, t2 = theta(0), theta(1), theta(2)
check((t0 * t1 + t1 * t0).is_zero(), "theta_0 theta_1 + theta_1 theta_0 = 0")
check((t0 * t0).is_zero() and (t2 * t2).is_zero(), "a generator squares to zero")
odd = t0.scaled(3) + t1.scaled(5)  # the odd element 3 theta_0 + 5 theta_1
check((odd * odd).is_zero(), "(3 theta_0 + 5 theta_1)^2 = 0")
```

`t0`, `t1`, `t2` are the generators $\theta_0, \theta_1, \theta_2$. The three checks: $\theta_0\theta_1 + \theta_1\theta_0 = 0$; $\theta_0^2 = \theta_2^2 = 0$; and $(3\theta_0 + 5\theta_1)^2 = 0$ (Section 7.11).

```
a, b, c, d, a2, b2, c2, d2 = sp.symbols("a b c d a2 b2 c2 d2")  # 8 ordinary numbers
F = number(a) + t0.scaled(b) + t1.scaled(c) + (t0 * t1).scaled(d)
F2 = number(a2) + t0.scaled(b2) + t1.scaled(c2) + (t0 * t1).scaled(d2)
by_hand = {(): a * a2, (0,): a * b2 + b * a2, (1,): a * c2 + c * a2,
           (0, 1): a * d2 + d * a2 + b * c2 - c * b2}
product = F * F2
same = sorted(product.terms) == sorted(by_hand) and all(
    sp.expand(product.terms[key] - value) == 0 for key, value in by_hand.items())
say(f"coefficient of theta_0 theta_1 in F F': {product.terms[(0, 1)]}")
check(same, "the product of two general elements of a 2-generator algebra")
```

Eight ordinary sympy symbols for the coefficients of F and F' (the primes are written a2, b2, c2, d2). `F` and `F2` are the two general elements of Section 7.10, `by_hand` the dictionary of the product worked out there. `same` is true when the computed product has exactly the same monomials (`sorted(product.terms)` lists them in order) and every coefficient agrees after multiplying out. **Out [5]:** four PASS lines and the coefficient `a*d2 + a2*d + b*c2 - b2*c` of $\theta_0\theta_1$, with the minus sign of the exchange.

In [6]: the dimension, and Figure 07a.1.

```
sizes = []  # the number of monomials for n = 0, 1, ..., 12
for n in range(13):
```

```
    everything = number(1)
    for i in range(n):
        everything = everything * (number(1) + theta(i))  # (1 + theta_i)
    sizes.append(len(everything.terms))
    if n == 8:
        degrees8 = Counter(len(monomial) for monomial in everything.terms)
        ones8 = all(value == 1 for value in everything.terms.values())
report("number of monomials for n = 12 generators", sizes[12])
check(sizes == [2 ** n for n in range(13)], "a Grassmann algebra with n generators "
      "has 2^n monomials (n = 0 to 12)")
check(ones8 and all(degrees8[k] == math.comb(8, k) for k in range(9)),
      "for n = 8 there are binom(8, k) monomials of degree k, each coefficient +1")
```

For $n = 0$ to 12 the product $(1+\theta_0)\cdots(1+\theta_{n-1})$ is computed, starting from the number 1, and the number of its monomials is stored. For $n = 8$, `Counter(len(monomial) ...)` counts the monomials of each degree and `ones8` records whether every coefficient is $+1$. The first check compares the counts with 2^n, the second the degrees with `math.comb(8, k)` $= \binom{8}{k}$. **Out [6]:** `RESULT number of monomials for n = 12 generators = 4096` and two PASS lines.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.semilogy(range(13), sizes, "o", label="counted by the program")
left.semilogy(range(13), [2 ** n for n in range(13)], "-", label="$2^n$")
left.set_xlabel("number of generators $n$")
left.set_ylabel("number of monomials")
left.set_title("Dimension of the algebra")
left.legend()
right.bar(range(9), [degrees8[k] for k in range(9)], color="tab:orange")
right.set_xlabel("degree $k$ (generators in the monomial)")
right.set_ylabel("number of monomials")
right.set_title("Degrees for $n = 8$: $\\binom{8}{k}$")
fig.tight_layout()
save_figure(fig, "dimension_and_degrees",
...)
```

Left: the counts as dots and 2^n as a line on a logarithmic vertical axis (`semilogy`). Right: a bar chart (`bar`) of the number of monomials of each degree for $n = 8$. **What Figure 07a.1 shows.** On the logarithmic axis 2^n is a straight line and the dots lie on it: each new generator doubles the algebra. The bars are the binomial coefficients 1, 8, 28, 56, 70, 56, 28, 8, 1, symmetric around degree 4.

In [7]: the multiplication table, and Figure 07a.2.

```
basis3 = [(), (0,), (1,), (2,), (0, 1), (0, 2), (1, 2), (0, 1, 2)]
names3 = ["$1$"] + ["$" + "".join(f"\\theta_{i}" for i in m) + "$" for m in basis3[1:]]
table = np.array([[shuffle_sign(row, column) for column in basis3] for row in basis3])
nonzero = int(np.count_nonzero(table))
report("nonzero products among the 64 products of the 8 monomials", nonzero)
check(nonzero == 27, "27 = 3^3 of the 64 products of monomials of 3 generators are "
      "nonzero")
check(np.array_equal(table[1:4, 1:4], -table[1:4, 1:4].T),
      "the table of the three generators is antisymmetric")
```

`basis3` lists the eight monomials of three generators, `names3` their labels for the plot (`"".join( ...)` joins the texts θ_i of one monomial; a doubled backslash in a Python string is one backslash for LaTeX). `table` is the 8×8 array of the signs of all products, row = left factor, column = right factor. `np.count_nonzero` counts the nonzero entries. The first check requires 27; the second checks that the block of the three single generators (rows and columns 1 to 3; `table[1:4, 1:4]`) is antisymmetric. **Out [7]:** `RESULT nonzero products among the 64 products of the 8 monomials = 27` and two PASS lines.

```
fig, ax = plt.subplots(figsize=(6.4, 5.4))
image = ax.imshow(table, cmap="coolwarm", vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        ax.text(j, i, {1: "+", -1: "-", 0: "0"}[int(table[i, j])], ha="center",
                va="center", fontsize=12)
ax.set_xticks(range(8), names3, rotation=45)
ax.set_yticks(range(8), names3)
ax.set_xlabel("right factor")
ax.set_ylabel("left factor")
ax.set_title("Sign of (left factor) times (right factor)")
ax.grid(False)
fig.colorbar(image, ax=ax, ticks=[-1, 0, 1], shrink=0.8)
save_figure(fig, "multiplication_signs",
...)
```

`imshow` draws the table as a **heat map**, one coloured square per entry (red $+1$, blue -1, grey 0), and the two loops write $+$, $-$ or 0 into each square (`ha` and `va` centre the text). The ticks carry the monomial names, rotated by 45 degrees under the horizontal axis. **What Figure 07a.2 shows.** The first row and column (the factor 1) are all $+$; every product with a shared generator is grey; the 3×3 block of single generators has $+$ above and $-$ below the grey diagonal: $\theta_i\theta_j = -\theta_j\theta_i$.

In [8]: even and odd, and Figure 07a.3.

```
signs = np.zeros((5, 5), dtype=int)
for k in range(5):
    for l in range(5):
        X = number(1)
        for i in range(k):
            X = X * theta(i)  # theta_0 ... theta_(k-1)
        Y = number(1)
        for i in range(l):
            Y = Y * theta(5 + i)  # theta_5 ... theta_(4+l)
        XY, YX = X * Y, Y * X
        (monomial, value), = XY.terms.items()  # XY is one monomial
        signs[k, l] = YX.terms[monomial] // value  # +1 or -1
expected = np.array([[(-1) ** (k * l) for l in range(5)] for k in range(5)])
check(np.array_equal(signs, expected), "YX = (-1)^(k l) XY for degrees k, l = 0 .. 4")
```

For degrees $k, l = 0$ to 4, `X` is the monomial $\theta_0 \cdots \theta_{k-1}$ and `Y` the monomial $\theta_5 \cdots \theta_{4+l}$ (no common generator). Both products `X * Y` and `Y * X` are formed. `(monomial, value), = XY.terms.it ems()` unpacks the single term of `XY` (the trailing comma says "exactly one item"), and the sign

is the coefficient of the same monomial in `YX` divided by `value` (`//` divides whole numbers). The check compares the table of signs with $(-1)^{kl}$. **Out [8]:** one PASS line.

```
fig, ax = plt.subplots(figsize=(5.6, 4.6))
image = ax.imshow(signs, cmap="coolwarm", vmin=-1, vmax=1)
for k in range(5):
    for l in range(5):
        ax.text(l, k, f"{signs[k, l]:+d}", ha="center", va="center", fontsize=12)
ax.set_xticks(range(5))
ax.set_yticks(range(5))
ax.set_xlabel("degree $l$ of $Y$")
ax.set_ylabel("degree $k$ of $X$")
ax.set_title("$YX = (\\pm 1)\\, XY$")
ax.grid(False)
fig.colorbar(image, ax=ax, ticks=[-1, 1], shrink=0.8)
save_figure(fig, "commutation_signs",
...)
```

A heat map of the 5×5 signs with the numbers written in. **What Figure 07a.3 shows.** Only the squares where both degrees are odd (1 and 3) are blue: two odd monomials anticommute. Every row and column of even degree is red: an even monomial commutes with everything.

In [9]: the conjugation.

```
def reproduces(report, name):
    """The text "<report>, check <name>" for a PASS line, after making sure that
    the Revision report (a JSON file) contains the check name with verdict PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return f"{report}, check {name}"


WLREP = "Revision/theory/reports/wolfram-field-theory.json"
PYREP = "Revision/theory/reports/python-field-theory.json"
```

`reproduces(report, name)` reads a Revision report, makes sure that it holds exactly one check of that name with the verdict PASS, and returns the text that a PASS line prints after "reproduces" (the same helper as in Notebook 07d). `WLREP` and `PYREP` are the paths of the Wolfram and the sympy field-theory reports.

```
def conjugate(element, partner):
    """The conjugate of element: generator g -> partner[g], the order of every
    product reversed, every coefficient complex-conjugated."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        images = [partner[g] for g in reversed(monomial)]  # reversed order
        sign, ordered = sort_with_sign(images)  # back to increasing order
        if sign != 0:
            result.add(ordered, sign * sp.conjugate(coefficient))
    return result
```

`conjugate(element, partner)` implements the three rules of Section 7.11. For every term, `reversed(monomial)` runs through the generators in reverse order and `partner[g]` replaces each by its partner; `sort_with_sign` sorts the result back and gives its sign; the coefficient is conjugated with `sp.conjugate`.

```
# the Wolfram test algebra: 0 = theta_1, 1 = theta_2, 2 = thetabar_1, 3 = thetabar_2
PAIR4 = [2, 3, 0, 1]
th1, th2, tb1, tb2 = theta(0), theta(1), theta(2), theta(3)
general = (number(sp.Rational(1, 2) + 2 * sp.I) + th1.scaled(3 - sp.I)
           + (th2 * tb1).scaled(-4 * sp.I) + (th1 * th2 * tb2).scaled(7))
check((th1 * th2 + th2 * th1).is_zero() and (th1 * th1).is_zero()
      and (conjugate(th1 * th2, PAIR4) - tb2 * tb1).is_zero()
      and (conjugate(conjugate(general, PAIR4), PAIR4) - general).is_zero(),
      "anticommutation, theta^2 = 0, (theta_1 theta_2)* = thetabar_2 thetabar_1, "
      "(F*)* = F",
      record=reproduces(WLREP, "grassmann_algebra_structure"))
```

The test algebra of the Wolfram verifier: generators 0 and 1 are θ_1, θ_2 and 2 and 3 are $\bar{\theta}_1, \bar{\theta}_2$, so the list `PAIR4 = [2, 3, 0, 1]` gives each generator its partner. `general` is an element with complex coefficients ($\frac{1}{2} + 2i$, $3 - i$, $-4i$, 7; `sp.Rational(1, 2)` is the exact fraction $\frac{1}{2}$). The check combines four statements with `and`: anticommutation, $\theta_1^2 = 0$, $(\theta_1\theta_2)^* = \bar{\theta}_2\bar{\theta}_1$, and $(F^*)^* = F$.

```
# the jet super-algebra of the sympy record: psi_A = generator A, chi_A = 16 + A
PAIR32 = [16 + A for A in range(16)] + list(range(16))
psi = [theta(A) for A in range(16)]
chi = [theta(16 + A) for A in range(16)]
rules = ((psi[0] * psi[0]).is_zero() and (psi[0] * psi[1] + psi[1] * psi[0]).is_zero()
         and (conjugate(chi[0] * psi[1], PAIR32) - chi[1] * psi[0]).is_zero()
         and (conjugate((chi[0] * psi[0]).scaled(sp.I), PAIR32)
              + (chi[0] * psi[0]).scaled(sp.I)).is_zero())
check(rules, "psi psi = 0, psi_A psi_B = -psi_B psi_A, (chi_A psi_B)* = chi_B "
      "psi_A, (i chi_A psi_A)* = -i chi_A psi_A",
      record=reproduces(PYREP, "superalgebra_axioms"))
check((conjugate(chi[3] * psi[3], PAIR32) - chi[3] * psi[3]).is_zero(),
      "chi_A psi_A (a component times its conjugate) is real")
```

The jet algebra of the sympy record for one point of a 16-component field: ψ_A is generator A (0 to 15) and $\chi_A = \psi_A^*$ is generator $16 + A$, so `PAIR32` sends A to $16 + A$ and back. `rules` combines the four rules of the record's check `superalgebra_axioms`: $\psi\psi = 0$, $\psi_A\psi_B = -\psi_B\psi_A$, $(\chi_A\psi_B)^* = \chi_B\psi_A$ (order reversal) and $(i\chi_A\psi_A)^* = -i\chi_A\psi_A$. The last check: $\chi_A\psi_A$ is real. **Out [9]:** three PASS lines; the first two each followed by a "reproduces" line naming `wolfram-field-theory.json`, check `grassmann_algebra_structure`, and `python-field-theory.json`, check `superalgebra_axioms`.

In [10]: left and right derivatives.

```
def left_derivative(element, k):
    """d_L element / d theta_k: move theta_k to the far left, then delete it."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        if k in monomial:
```

```
            p = monomial.index(k)  # the number of generators to its left
            result.add(monomial[:p] + monomial[p + 1:], (-1) ** p * coefficient)
    return result


def right_derivative(element, k):
    """d_R element / d theta_k: move theta_k to the far right, then delete it."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        if k in monomial:
            p = monomial.index(k)
            passed = len(monomial) - 1 - p  # generators to its right
            result.add(monomial[:p] + monomial[p + 1:], (-1) ** passed * coefficient)
    return result
```

`left_derivative(element, k)`: for every monomial that contains generator `k`, `monomial.index(k)` is its position `p`, which is the number of generators to its left; the generator is deleted (`monomial[:p] + monomial[p + 1:]` joins the parts before and after it) and the coefficient multiplied by $(-1)^p$. `right_derivative` does the same with the number of generators to its right, `len(monomial) - 1 - p`.

```
F = t0 * t1
check((left_derivative(F, 0) - t1).is_zero() and (left_derivative(F, 1) + t0).is_zero()
      and (right_derivative(F, 1) - t0).is_zero()
      and (right_derivative(F, 0) + t1).is_zero(),
      "the four derivatives of theta_0 theta_1 worked by hand")
even = Grassmann()
for trial in range(40):  # a random even element of the algebra of 8 generators
    size = 2 * int(rng.integers(1, 4))  # degree 2, 4 or 6
    monomial = tuple(sorted(rng.choice(8, size=size, replace=False)))
    even.add(monomial, int(rng.integers(-9, 10)))
check(all((right_derivative(even, k) + left_derivative(even, k)).is_zero()
          for k in range(8)), "d_R F = -d_L F for a random even element F")
```

The four derivatives of $\theta_0\theta_1$ worked by hand in Section 7.11 are checked. Then a random even element of the algebra of 8 generators is built from 40 random monomials of degree 2, 4 or 6 (`2 * int(rng.integers(1, 4))`) with random whole coefficients from -9 to 9 (`rng.integers(-9, 10)`; a repeated monomial simply adds up); the check requires $\partial_R F = -\partial_L F$ for all 8 generators.

```
x, a_, y = chi[0], psi[0], psi[1]  # x = chi_0 (generator 16), a = psi_0, y = psi_1
p_, q_ = x * a_ * y, x * a_
rules = ((left_derivative(p_, 1) - x * a_).is_zero()
         and (right_derivative(p_, 1) - x * a_).is_zero()
         and (left_derivative(p_, 0) + x * y).is_zero()
         and (right_derivative(p_, 0) + x * y).is_zero()
         and (left_derivative(q_, 0) + x).is_zero()
         and (right_derivative(q_, 0) - x).is_zero())
check(rules, "left and right derivatives with the Grassmann signs",
      record=reproduces(PYREP, "superalgebra_axioms"))
```

The derivative rules of the sympy record, with $x = \chi_0$, $a = \psi_0$, $y = \psi_1$ and $p = xay$, $q = xa$:

$\partial_L p/\partial y = \partial_R p/\partial y = xa$, $\partial_L p/\partial a = \partial_R p/\partial a = -xy$, $\partial_L q/\partial a = -x$ and $\partial_R q/\partial a = +x$ (generators 1 and 0 are y and a). **Out [10]:** three PASS lines, the last with its "reproduces" line.

In [11]: when is a bilinear form real?

```
def bilinear(row, M, column):
    """sum over A, B of row[A] M[A, B] column[B] (one factor from each side)."""
    total = Grassmann()
    for A in range(len(row)):
        for B in range(len(column)):
            if M[A, B] != 0:
                total = total + (row[A] * column[B]).scaled(M[A, B])
    return total
```

`bilinear(row, M, column)` forms $\sum_{A,B} \mathrm{row}_A M_{AB} \mathrm{column}_B$, skipping the zero entries of M.

```
n4 = 4
psi4 = [theta(A) for A in range(n4)]  # psi_A: generators 0 .. 3
chi4 = [theta(n4 + A) for A in range(n4)]  # chi_A = psi_A*: generators 4 .. 7
PAIR8 = [n4 + A for A in range(n4)] + list(range(n4))
random_matrix = sp.Matrix(n4, n4, lambda i, j: int(rng.integers(-3, 4))
                          + sp.I * int(rng.integers(-3, 4)))
hermitian = random_matrix + random_matrix.H  # M + M^dagger is Hermitian
form_h = bilinear(chi4, hermitian, psi4)
form_n = bilinear(chi4, random_matrix, psi4)
check((conjugate(form_h, PAIR8) - form_h).is_zero(),
      "Psi^dagger M Psi is real for a Hermitian M")
not_hermitian = random_matrix.H != random_matrix  # True: M^dagger differs from M
check(not_hermitian and not (conjugate(form_n, PAIR8) - form_n).is_zero(),
      "Psi^dagger M Psi is not real for a non-Hermitian M")
```

$n = 4$: the ψ_A are generators 0 to 3, the χ_A generators 4 to 7, and `PAIR8` pairs them. `random_matrix` is a 4×4 sympy matrix with random **Gaussian integers** (whole numbers $a + bi$ with a, b from -3 to 3; the `lambda i, j: ...` is a small unnamed function that gives the entry in row `i` and column `j`). `random_matrix.H` is its conjugate transpose $M^\dagger$, so `hermitian` $= M + M^\dagger$ is Hermitian. The first check: $\Psi^\dagger M \Psi$ with the Hermitian matrix equals its conjugate. The second: the random matrix is not Hermitian, and its form is not real. **Out [11]:** two PASS lines.

In [12]: the author's C and the real columns.

```
gammas_file = "Revision/algebra/gammas.json"
fixture = json.loads(repository_file(gammas_file).read_text(encoding="utf-8"))
G = [np.array(g, dtype=int) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7)
C = np.array(fixture["C"], dtype=int)  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
check(np.array_equal(C, G[0] @ G[1] @ G[2] @ G[3]), "C = gamma^(x0) gamma^(x1) "
      "gamma^(x2) gamma^(x3) (read from Revision/algebra/gammas.json)")
check(np.array_equal(C.T, C) and np.array_equal(C @ C, np.eye(16, dtype=int))
      and all(np.array_equal((C @ g).T, -(C @ g)) for g in G),
      "C is symmetric, C^2 = 1, and every C gamma^(a) is antisymmetric",
      record=reproduces(PYREP, "C_properties"))
```

The notebook reads the Revision record `Revision/algebra/gammas.json`: `G` holds the eight gammas $\gamma^{(x_0)}, \ldots, \gamma^{(x_7)}$ as integer arrays (`G[k]` is $\gamma^{(x_k)}$: `G[0]` is $\gamma^{(x_0)}$, `G[7]` is $\gamma^{(x_7)}$) and `C` the matrix C. The first check recomputes $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ (`@` is the matrix product of numpy). The second checks that C is symmetric (`C.T` is the transpose), $C^2 = 1$ (`np.eye(16)` is the identity), and every $C\gamma^{(a)}$ is antisymmetric; it names the record's check `C_properties`.

```
Theta = [theta(A) for A in range(16)]  # a real Grassmann column
q = sp.Matrix(sp.symbols("q0:16"))  # a real column of ordinary numbers
labels = ["$C$"] + [f"$C\\gamma^{{(x_{a})}}$" for a in range(8)]
matrices = [sp.Matrix(C)] + [sp.Matrix(C @ g) for g in G]  # exact sympy matrices
grassmann_counts, commuting_counts = [], []
for M in matrices:
    grassmann_counts.append(len(bilinear(Theta, M, Theta).terms))
    form = sp.expand((q.T * M * q)[0, 0])
    commuting_counts.append(0 if form == 0 else len(sp.Add.make_args(form)))
say(f"monomials, real Grassmann: {grassmann_counts}")
say(f"monomials, real commuting: {commuting_counts}")
```

`Theta` is a real Grassmann column of 16 generators, `q` a column of 16 ordinary sympy symbols. `labels` are the plot labels of the nine matrices (in an f-string a doubled brace `{{` prints one brace). `matrices` holds C and the eight $C\gamma^{(a)}$ as exact sympy matrices. For each, the number of surviving monomials is counted: for Grassmann numbers with the class `Grassmann`, for ordinary numbers by expanding the sympy expression $q^T M q$ and counting its terms (`sp.Add.make_args` splits a sum into its terms; a zero form has no terms). **Out [12]:** two PASS lines (the second followed by its "reproduces" line, which names the check `C_properties`), and `monomials, real Grassmann: [0, 8, 8, 8, 8, 8, 8, 8, 8]`, `monomials, real commuting: [8, 0, 0, 0, 0, 0, 0, 0, 0]`.

In [13]: the pattern and its figures.

```
two = all(abs(value) == 2 for M in matrices[1:]
          for value in bilinear(Theta, M, Theta).terms.values())
check(grassmann_counts == [0] + [8] * 8 and two, "real Grassmann: Theta^T C Theta "
      "= 0; each Theta^T C gamma^(a) Theta has 8 monomials with coefficients +-2",
      record=reproduces(WLREP, "Majorana_Lg_total_derivative_grassmann"))
check(commuting_counts == [8] + [0] * 8, "real commuting: q^T C q has 8 monomials, "
      "every q^T C gamma^(a) q is 0",
      record=reproduces(WLREP, "Majorana_Lg_commuting_control"))
```

`two` is true when every coefficient of every surviving Grassmann form is ± 2. The first check requires the counts $[0, 8, \ldots, 8]$ and the coefficients ± 2 and names the Wolfram check `Majorana_Lg_total_derivative_grassmann`, of which this is the part about the bilinear forms; the second requires the commuting counts $[8, 0, \ldots, 0]$ and names the Wolfram check `Majorana_Lg_commuting_control`. **Out [13]:** two PASS lines with their "reproduces" lines, and two figures.

```
fig, ax = plt.subplots(figsize=(8.4, 4.0))
positions = np.arange(len(labels))
ax.bar(positions - 0.2, grassmann_counts, width=0.4, label="real Grassmann column")
ax.bar(positions + 0.2, commuting_counts, width=0.4, label="real ordinary numbers")
ax.set_xticks(positions, labels)
```

```
ax.set_ylabel("surviving monomials")
ax.set_ylim(0, 10.5)  # room for the legend above the bars
ax.set_title("Which bilinear forms survive for a real column")
ax.legend(loc="upper center", ncol=2)
save_figure(fig, "bilinear_survivors",
...)
```

Two bars per matrix, shifted left and right by 0.2 (`positions - 0.2`, `positions + 0.2`), for the two kinds of numbers. **What Figure 07a.4 shows.** The blue bars (real Grassmann columns) are zero for C and 8 for every $C\gamma^{(a)}$; the orange bars (real ordinary numbers) are 8 for C and zero for every $C\gamma^{(a)}$. The two kinds of numbers keep opposite halves of a matrix.

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, M, title in zip(axes, [C, C @ G[1]],
                        ["$C$ (symmetric)", "$C\\gamma^{(x_1)}$ (antisymmetric)"]):
    image = ax.imshow(M, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_xticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_yticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
    ax.set_xlabel("column $B$")
    ax.set_ylabel("row $A$")
    ax.set_title(title)
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8)
save_figure(fig, "symmetric_antisymmetric",
...)
```

Heat maps of C and $C\gamma^{(x_1)}$ (`C @ G[1]`), with ticks every three rows labelled 1, 4, 7, ... (rows and columns counted from 1, as in the book), and one common colour scale for both panels (`fig.colorbar(image, ax=axes, ...)`). **What Figure 07a.5 shows.** Each row of both matrices has exactly one coloured square. Mirroring C in its diagonal gives the same colours (symmetric); mirroring $C\gamma^{(x_1)}$ turns every red square into blue and every blue into red (antisymmetric).

In [14]: the powers of the scalar density.

```
S = Grassmann()
for A in range(16):
    for B in range(16):
        if C[A, B] != 0:
            S = S + (theta(A) * theta(16 + B)).scaled(int(C[A, B]))  # chi_A C psi_B
```

$S = \sum_{A,B} \chi_A C_{AB} \psi_B$ is built from the 16 nonzero entries of C; here the χ_A are generators 0 to 15 and the ψ_B generators 16 to 31.

```
power = number(1)
counts, sizes_ok = [], True
for k in range(1, 18):
    power = power * S  # S^k
    counts.append(len(power.terms))
    sizes_ok = sizes_ok and all(abs(value) == math.factorial(k)
                                for value in power.terms.values())
```

```
report("monomials of S^k for k = 1 .. 8", counts[:8])
report("monomials of S^k for k = 9 .. 17", counts[8:])
check(counts[:16] == [math.comb(16, k) for k in range(1, 17)] and sizes_ok,
      "S^k has binom(16, k) monomials with coefficients +-k! (k = 1 .. 16)")
check(counts[16] == 0, "S^17 = 0: every function of S is a polynomial of degree <= 16")
```

`power` starts at 1 and is multiplied by S seventeen times. After each multiplication the number of monomials is stored, and `sizes_ok` stays true only while every coefficient of S^k has the size $k!$ (`math.factorial(k)`). The two `report` lines print the counts; `counts[:8]` is the first eight entries and `counts[8:]` the rest. The first check compares the counts for $k = 1$ to 16 with $\binom{16}{k}$, the second requires $S^{17} = 0$. The computation takes about one second.

```
# ordinary (commuting) stand-ins for the 16 chi_A and the 16 psi_A
chi_c, psi_c = sp.symbols("chi0:16"), sp.symbols("psi0:16")
S_commuting = sum(int(C[A, B]) * chi_c[A] * psi_c[B]
                  for A in range(16) for B in range(16) if C[A, B] != 0)
commuting = [len(sp.Add.make_args(sp.expand(S_commuting ** k))) for k in (1, 2, 3)]
check(commuting == [math.comb(16 + k - 1, k) for k in (1, 2, 3)],
      "commuting numbers: S^k has binom(15 + k, k) monomials (k = 1, 2, 3)")
```

The same S with 32 ordinary sympy symbols `chi0` to `chi15` and `psi0` to `psi15` in place of the generators. Its powers $k = 1, 2, 3$ are multiplied out and their terms counted; the check compares with $\binom{15+k}{k}$. **Out [14]:** `RESULT monomials of S^k for k = 1 ..  8 = [16, 120, 560, 1820, 4368, 8008, 11440, 12870]`, `RESULT monomials of S^k for k = 9 ..  17 = [11440, 8008, 4368, 1820, 560, 120, 16, 1, 0]` and three PASS lines.

In [15]: Figure 07a.6.

```
ks = np.arange(1, 18)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.0))
left.bar(ks[:16], counts[:16], color="tab:blue", label="Grassmann (computed)")
left.plot(ks, [math.comb(15 + k, k) for k in ks], "s--", color="tab:orange",
          label="ordinary numbers $\\binom{15+k}{k}$")
left.set_yscale("log")
left.set_xticks(range(1, 18, 2))
left.set_xlabel("power $k$")
left.set_ylabel("number of monomials of $S^k$")
left.set_title("$S^{17} = 0$ for Grassmann numbers")
left.legend(fontsize=8)
right.semilogy(ks[:16], [math.factorial(k) for k in ks[:16]], "o-", color="tab:green")
right.set_xticks(range(1, 17, 3))
right.set_xlabel("power $k$")
right.set_ylabel("size of every coefficient of $S^k$")
right.set_title("Coefficients $\\pm k!$")
fig.tight_layout()
save_figure(fig, "powers_of_s",
...)
```

Left: blue bars of the Grassmann counts for $k = 1$ to 16 and orange squares joined by a dashed line for the commuting counts $\binom{15+k}{k}$ for $k = 1$ to 17, on a logarithmic axis (`set_yscale("log")`).

Right: the size $k!$ of the coefficients. **What Figure 07a.6 shows.** The Grassmann counts rise to 12870 at $k = 8$ and fall back symmetrically to 1 at $k = 16$, with no bar at $k = 17$; the commuting counts keep growing. On the right, the coefficients grow from 1 to 16! $\approx 2.1 \times 10^{13}$.

In [16]: the test algebra with two generators.

```
coefficients = [[int(rng.integers(-3, 4)) + sp.I * int(rng.integers(-3, 4))
                 for k in range(2)] for A in range(16)]
Psi = [th1.scaled(coefficients[A][0]) + th2.scaled(coefficients[A][1])
       for A in range(16)]
Psi_dagger = [conjugate(component, PAIR4) for component in Psi]
S2 = bilinear(Psi_dagger, sp.Matrix(C), Psi)  # S in the test algebra
shapes = sorted(S2.terms)
say(f"monomials of S (generator numbers): {shapes}")
say(f"monomials of S^2: {sorted((S2 * S2).terms)}")
check(all(len(m) == 2 and m[0] in (0, 1) and m[1] in (2, 3) for m in shapes)
      and (conjugate(S2, PAIR4) - S2).is_zero(),
      "S is a real even element on the monomials thetabar_k theta_l")
check(sorted((S2 * S2).terms) == [(0, 1, 2, 3)] and (S2 * S2 * S2).is_zero(),
      "S^2 != 0 lives on theta_1 theta_2 thetabar_1 thetabar_2 and S^3 = 0",
      record=reproduces(WLREP, "grassmann_algebra_structure"))
```

`coefficients` holds, for each of the 16 components, two random Gaussian integers c_{A1}, c_{A2}; `Psi` is the column of the components $c_{A1}\theta_1 + c_{A2}\theta_2$ (generators 0 and 1 of the test algebra of In [9]), and `Psi_dagger` the conjugates. `S2` is $S = \Psi^\dagger C \Psi$ in this algebra (the name says "S of the two-generator algebra"). The two `say` lines print the monomials of S and of S^2 as tuples of generator numbers. The first check: every monomial of S holds exactly one generator θ_l (number 0 or 1) and one conjugate $\bar{\theta}_k$ (number 2 or 3), which sorting puts second, and S is real. The second: S^2 lives on the single monomial (0, 1, 2, 3) and $S^3 = 0$. **Out [16]:** `monomials of S (generator numbers): [(0, 2), (0, 3), (1, 2), (1, 3)]`, `monomials of S^2: [(0, 1, 2, 3)]` and two PASS lines, the second with its "reproduces" line.

In [17]: the Grassmann oscillator.

```
def total_derivative(element, step):
    """d/dt on jets: every generator g in every place becomes g + step (from a
    variable to its derivative), with the product rule; the raised monomial is
    sorted back with its sign."""
    result = Grassmann()
    for monomial, coefficient in element.terms.items():
        for place, g in enumerate(monomial):
            raised = monomial[:place] + (g + step,) + monomial[place + 1:]
            sign, ordered = sort_with_sign(raised)
            if sign != 0:
                result.add(ordered, sign * coefficient)
    return result
```

`total_derivative(element, step)` is d/dt on jets. In the numbering of this cell the generators of the value come first and their derivatives have numbers larger by `step` (here 2), so raising a generator `g` to its derivative means replacing it by `g + step`. For every monomial and every place in it, the raised monomial is formed, sorted back with its sign, and added (the product rule adds one term per place).

```
omega = sp.Symbol("omega", positive=True)  # the angular frequency
bar, th, bar_d, th_d = theta(0), theta(1), theta(2), theta(3)
L_osc = ((bar * th_d - bar_d * th).scaled(sp.I / 2) - (bar * th).scaled(omega))
E_osc = left_derivative(L_osc, 0) - total_derivative(left_derivative(L_osc, 2), 2)
PAIR6 = [1, 0, 3, 2, 5, 4]  # theta-bar <-> theta for the value and two derivatives
say(f"Euler-Lagrange expression (monomial: coefficient): {E_osc.terms}")
check((E_osc - (th_d.scaled(sp.I) - th.scaled(omega))).is_zero(),
      "the oscillator's Euler-Lagrange expression is i theta' - omega theta")
check((conjugate(L_osc, PAIR6) - L_osc).is_zero(), "the oscillator Lagrangian is real")
```

`omega` is a positive symbol. The generators are $\bar\theta$ (0), θ (1), $\bar\theta'$ (2), θ' (3); `L_osc` is the Lagrangian of Section 7.13. `E_osc` is the Euler-Lagrange expression with left derivatives: the left derivative with respect to $\bar\theta$ minus the total derivative of the left derivative with respect to $\bar\theta'$. `PAIR6` pairs $\bar\theta \leftrightarrow \theta$ for the value and two derivatives (generators 4 and 5 are the second derivatives). The `say` line prints the expression as its dictionary of terms; the checks compare it with $i\theta' - \omega\theta$ and test that L is real. **Out [17]:** `Euler-Lagrange expression (monomial: coefficient): {(3,): I, (1, ): -omega}` (that is $i\theta' - \omega\theta$) and two PASS lines.

In [18]: the solution, and Figure 07a.7.

```
t = sp.Symbol("t", real=True)
solution = sp.exp(-sp.I * omega * t)  # theta(t) = solution * theta_0
residual = sp.simplify(sp.I * sp.diff(solution, t) - omega * solution)
check(residual == 0, "theta(t) = exp(-i omega t) theta_0 solves i theta' = omega theta")
```

The coefficient function $e^{-i\omega t}$ of the solution $\theta(t) = e^{-i\omega t}\theta_0$; the residual $i\frac{d}{dt}e^{-i\omega t} - \omega e^{-i\omega t}$ simplifies to 0 (the generator θ_0 is a common factor and drops out).

```
times = np.linspace(0.0, 2.0 * np.pi, 400)
values = np.exp(-2.0j * times)  # omega = 2
fig, ax = plt.subplots()
ax.plot(times, values.real, label="real part $\\cos 2t$")
ax.plot(times, values.imag, "--", label="imaginary part $-\\sin 2t$")
ax.plot(times, np.abs(values), ":", color="black", label="absolute value 1")
ax.set_xlabel("time $t$")
ax.set_ylabel("coefficient of $\\theta_0$")
ax.set_title("$\\theta(t) = e^{-2it}\\,\\theta_0$")
ax.set_ylim(-1.75, 1.25)  # room for the legend below the curves
ax.legend(loc="lower center", ncol=3, fontsize=9)
save_figure(fig, "oscillator_solution",
...)
```

400 times from 0 to 2π, the values of e^{-2it} ($\omega = 2$), and one panel with the real part, the imaginary part and the absolute value; `set_ylim(-1.75, 1.25)` leaves room for the legend below the curves. **What Figure 07a.7 shows.** The function in front of θ_0 is an ordinary complex function of absolute value 1 that turns with angular frequency 2: its real part is $\cos 2t$ and its imaginary part $-\sin 2t$. The anticommuting nature of the solution sits entirely in the constant generator θ_0.

In [19]: the first-order kinetic term of real fields.

```
th0, th1_, th0_d, th1_d = theta(0), theta(1), theta(2), theta(3)
K_A = th0 * th1_d - th1_ * th0_d  # theta^T A theta' with A = ((0, 1), (-1, 0))
K_I = th0 * th0_d + th1_ * th1_d  # theta^T I theta'
```

Four generators: θ_0 (0), θ_1 (1), θ_0' (2), θ_1' (3); the variable names `th1_` and `th0_d`, `th1_d` avoid clashing with names used earlier. `K_A` $= \theta^T A \theta'$ and `K_I` $= \theta^T \theta'$ of Section 7.13.

```
def euler_lagrange(L, field):
    """dL/d(field) - d/dx dL/d(field') with left derivatives (field = 0 or 1)."""
    return left_derivative(L, field) - total_derivative(left_derivative(L, field + 2), 2)
```

`euler_lagrange(L, field)` is the Euler-Lagrange expression of field 0 or 1 with left derivatives; the derivative of field `field` is generator `field + 2`.

```
check((K_A - total_derivative((th0 * th1_ - th1_ * th0).scaled(sp.Rational(1, 2)),
                              2)).is_zero(),
      "K_A = (1/2) d/dx (theta^T A theta): a total derivative")
check(euler_lagrange(K_A, 0).is_zero() and euler_lagrange(K_A, 1).is_zero(),
      "real Grassmann fields: both Euler-Lagrange expressions of K_A vanish")
check((euler_lagrange(K_I, 0) - th0_d.scaled(2)).is_zero(),
      "real Grassmann fields: K_I gives the expression 2 theta_0'")
```

Three checks: K_A equals $\frac{1}{2}\frac{d}{dx}(\theta^T A \theta)$ with $\theta^T A \theta = \theta_0\theta_1 - \theta_1\theta_0$; both Euler-Lagrange expressions of K_A vanish; the expression of K_I for θ_0 is $2\theta_0'$.

```
from sympy.calculus.euler import euler_equations  # sympy's Euler-Lagrange

xs = sp.Symbol("x", real=True)
q0, q1 = sp.Function("q0")(xs), sp.Function("q1")(xs)  # two ordinary functions
K_A_commuting = q0 * sp.diff(q1, xs) - q1 * sp.diff(q0, xs)
equations = euler_equations(K_A_commuting, [q0, q1], xs)
expressions = [sp.simplify(equation.lhs - equation.rhs) for equation in equations]
say(f"ordinary numbers, Euler-Lagrange expressions of K_A: {expressions}")
check(sp.simplify(expressions[0] - 2 * sp.diff(q1, xs)) == 0
      and sp.simplify(expressions[1] + 2 * sp.diff(q0, xs)) == 0,
      "ordinary numbers: K_A gives the nonzero expressions 2 q1' and -2 q0'")
```

The same kinetic term for two ordinary functions $q_0(x)$, $q_1(x)$, with sympy's `euler_equations`; the list `expressions` holds left side minus right side of each equation. The check requires $2q_1'$ and $-2q_0'$.
Out [19]: four PASS lines and `ordinary numbers, Euler-Lagrange expressions of K_A: [2* Derivative(q1(x), x), -2*Derivative(q0(x), x)]`.

In [20]: the last check.

```
names = ["dimension_and_degrees", "multiplication_signs", "commutation_signs",
         "bilinear_survivors", "symmetric_antisymmetric", "powers_of_s",
         "oscillator_solution"]
files = [f"{FIGURE_FOLDER}/07a_{k}_{name}.png" for k, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 7 figure files of this notebook exist")
all_checks_passed()
```

The seven figure files are listed and checked, and the last line is printed. **Out [20]:** `PASS all 7 figure files of this notebook exist` and `ALL 38 CHECKS PASSED (notebook 07a)`.

7.18 The geometry the fields live in

The Lagrangian of the two fields uses five geometric objects of the author's metric: the frame, the volume factor, the coordinate gammas, the Christoffel symbols and the spin connection. Chapter 3 introduced the first four and Chapter 6 treats the spin connection in full; this section collects what the field equations need, with the derivations that Notebook 07b repeats exactly.

The frame and the volume factor. The author's metric is diagonal, $g_{aa} = \eta_{aa} f_a^2$ (no sum), with the frame metric $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the order $x_0, \ldots, x_7$ and the **frame factors**

$$f_0 = \cot z, \qquad f_1 = f_2 = f_3 = e^{a_4} \sin^{1/6} z, \qquad f_4 = 1, \qquad f_5 = f_6 = f_7 = e^{-a_4} b_4^{1/2} \sin^{1/6} z.$$

Here $b_4(x_4) > 0$ is the second function of the author's metric (Chapter 3); the backgrounds of the record have $b_4 = 1$. The **diagonal frame** (or **vielbein**, "many legs") $e^a{}_\mu = f_a \delta^a_\mu$ is a set of eight local unit directions, one along each coordinate; a column of numbers written in this frame, such as the 16 components of a spinor, is measured with these unit directions. For a diagonal metric $|\det g|$ is the product of the $|g_{aa}| = f_a^2$, so the **volume factor** is

$$\sqrt{|g|} = f_0 \cdot f_1 f_2 f_3 \cdot f_4 \cdot f_5 f_6 f_7 = \frac{\cos z}{\sin z} \cdot e^{3a_4} \sin^{1/2} z \cdot 1 \cdot e^{-3a_4} b_4^{3/2} \sin^{1/2} z = b_4^{3/2} \cos z$$

(insert the factors; $e^{3a_4} e^{-3a_4} = 1$; $\sin^{1/2} z \sin^{1/2} z = \sin z$ cancels against $1/\sin z$). It depends on the time only through $b_4^{3/2}$: the inflation e^{3a_4} of 3-space and the deflation e^{-3a_4} of the extra times compensate exactly, and on the backgrounds $b_4 = 1$ the volume factor $\cos z$ does not depend on the time at all (PROVED; Notebook 07b, In [4]; the table at the end of this section names the Revision checks of this and the following statements).

Exact bookkeeping with a few symbols. Notebook 07b writes every coefficient with the symbols $E = e^{a_4}$, $s = \sin^{1/6} z$, $c = \cos z$, $A_1 = a_4'$, $A_2 = a_4''$, $A_3 = a_4'''$, b_4 and its derivatives b_4', b_4'', b_4''', and H, m, λ (a prime is d/dx_4). Then $f_0 = c/s^6$ (because $\sin z = s^6$), $f_{1,2,3} = Es$ and $f_{5,6,7} = s\sqrt{b_4}/E$. Their derivatives follow from the chain rule: $\partial_4 E = EA_1$, $\partial_4 b_4 = b_4'$; $\partial_0 s = \frac{1}{6} \sin^{-5/6} z \cdot \cos z \cdot 6H = Hc/s^5$ (the power rule, the derivative $\cos z$ of $\sin z$, then $dz/dx_0 = 6H$); $\partial_0 c = -\sin z \cdot 6H = -6Hs^6$; nothing depends on $x_1, x_2, x_3, x_5, x_6, x_7$. Because $\cos^2 z + \sin^2 z = 1$, the symbols obey $c^2 = 1 - s^{12}$; replacing c^2 by $1 - s^{12}$ decides exactly whether an expression is zero.

The coordinate gammas. The author's eight frame gammas $\gamma^{(x_0)}, \ldots, \gamma^{(x_7)}$ (Chapter 4) obey $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}$. The **coordinate gammas** are $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ (the frame gamma of the same direction divided by its frame factor); they obey $\gamma^\mu\gamma^\nu + \gamma^\nu\gamma^\mu = 2g^{\mu\nu}$ with the inverse metric $g^{\mu\mu} = \eta_{\mu\mu}/f_\mu^2$.

The Christoffel symbols. The **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$ say how the coordinate directions turn from point to point (Chapter 3). For a diagonal metric the general formula keeps only a few terms:

$$\Gamma^{\lambda}{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}\big(\delta_{\lambda\nu}\,\partial_\mu g_{\lambda\lambda} + \delta_{\lambda\mu}\,\partial_\nu g_{\lambda\lambda} - \delta_{\mu\nu}\,\partial_\lambda g_{\mu\mu}\big).$$

Four examples, each in one line. $\Gamma^{x_1}{}_{x_1x_4} = \partial_4 g_{11}/(2g_{11}) = \partial_4(E^2s^2)/(2E^2s^2) = 2E^2A_1s^2/(2E^2s^2) = a_4'$: 3-space stretches at the rate a_4'. $\Gamma^{x_5}{}_{x_4x_5} = \partial_4 g_{55}/(2g_{55})$ with $g_{55} = -b_4s^2/E^2$, and $\partial_4(-b_4s^2E^{-2}) = -s^2E^{-2}(b_4' - 2A_1b_4)$, so $\Gamma^{x_5}{}_{x_4x_5} = (b_4' - 2A_1b_4)/(2b_4) = -a_4' + b_4'/(2b_4)$: the extra times shrink at this rate, which is $-a_4'$, the rate of 3-space with the opposite sign, at $b_4 = 1$. We write $\alpha_t = -a_4' + b_4'/(2b_4)$ for it. $\Gamma^{x_1}{}_{x_0x_1} = \partial_0(E^2s^2)/(2E^2s^2) = \partial_0 s/s = Hc/s^6 = H\cot z$. $\Gamma^{x_0}{}_{x_0x_0} = \frac{1}{2}\partial_0 \ln g_{00}$ with $g_{00} = c^2/s^{12}$, so $\frac{1}{2}(2\partial_0 c/c - 12\,\partial_0 s/s) = -6Hs^6/c - 6Hc/s^6 = -6H(s^{12}+c^2)/(cs^6) = -6H/(\sin z\cos z)$. The metric has 25 independent nonzero Christoffel symbols (COMPUTED exactly by Notebook 07b, In [7], and compared there with the formula records `Revision/theory/field-theory-b4.json` and, at $b_4 = 1$, `Revision/theory/field-theory.json`).

Why a spinor needs a connection. The 16 components of a spinor are measured in the local frame. Because the frame turns from point to point (that is what the Christoffel symbols describe), the plain derivative $\partial_\mu\Psi$ compares components measured in two different frames, and an equation built from it would depend on the arbitrary choice of frames. The **canonical spin connection** $\omega_\mu{}^a{}_b$ records how the frame turns; it is fixed by the **vielbein postulate** $\partial_\mu e^a{}_\nu - \Gamma^\lambda{}_{\mu\nu}e^a{}_\lambda + \omega_\mu{}^a{}_b\,e^b{}_\nu = 0$ (the frame is carried along consistently with the metric). For the diagonal frame its solution is

$$\omega_\mu{}^a{}_b = f_a\Big(\delta_{ab}\,\partial_\mu\frac{1}{f_b} + \frac{\Gamma^a{}_{\mu b}}{f_b}\Big), \qquad \omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b,$$

and the lowered $\omega_{\mu ab}$ is antisymmetric in a, b. For $a \neq b$ it is $\eta_{aa}f_a\Gamma^a{}_{\mu b}/f_b$. Two examples: $\omega_{x_1(x_1)(x_4)} = (+1)\cdot Es\cdot a_4'/1 = a_4'e^{a_4}\sin^{1/6} z$, and $\omega_{x_5(x_4)(x_5)} = (-1)\cdot 1\cdot\Gamma^{x_4}{}_{x_5x_5}/f_5$ with $\Gamma^{x_4}{}_{x_5x_5} = -\partial_4 g_{55}/(2g_{44}) = -f_5^2\alpha_t$, which gives $f_5^2\alpha_t/f_5 = \alpha_t f_5 = -(a_4' - \frac{b_4'}{2b_4})e^{-a_4}b_4^{1/2}\sin^{1/6} z$. Exactly 12 components with $a < b$ are nonzero (COMPUTED exactly by Notebook 07b, In [8]):

$$\begin{aligned}
&\omega_{x_i(x_i)(x_4)} = a_4'e^{a_4}\sin^{1/6} z, && \omega_{x_i(x_0)(x_i)} = -He^{a_4}\sin^{1/6} z,\\
&\omega_{x_t(x_4)(x_t)} = -\Big(a_4' - \frac{b_4'}{2b_4}\Big)e^{-a_4}b_4^{1/2}\sin^{1/6} z, && \omega_{x_t(x_0)(x_t)} = He^{-a_4}b_4^{1/2}\sin^{1/6} z,
\end{aligned}$$

for $i = 1,2,3$ and $t = 5,6,7$. Each is a rate (a_4', $a_4' - b_4'/(2b_4)$ or H) times a factor that never vanishes for $0 < z < \pi/2$.

The spinor connection and the covariant derivatives. With the **spin generators** $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$, which equal $\frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ for $a \neq b$ (the two products differ only in sign) and 0 for $a = b$ (Chapter 5), the **spinor connection** is

$$\Omega_\mu = \tfrac{1}{2}\,\omega_{\mu ab}S^{ab} = \sum_{a<b}\omega_{\mu ab}S^{ab}$$

(the two halves of the double sum are equal, because both $\omega_{\mu ab}$ and S^{ab} change sign when a and b are exchanged). The **covariant derivatives** of a column and of a row are

$$D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi, \qquad D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\,\Omega_\mu.$$

In the author's metric $\Omega_{x_0} = \Omega_{x_4} = 0$, and for $i = 1, 2, 3$ and $t = 5, 6, 7$, with $r = a_4' - b_4'/(2b_4) = -\alpha_t$:

$$\Omega_{x_i} = \tfrac{1}{2} e^{a_4} \sin^{1/6} z \, \big(a_4' \gamma^{(x_i)} \gamma^{(x_4)} + H \gamma^{(x_i)} \gamma^{(x_0)}\big),$$
$$\Omega_{x_t} = -\tfrac{1}{2} e^{-a_4} b_4^{1/2} \sin^{1/6} z \, \big(r \, \gamma^{(x_4)} \gamma^{(x_t)} + H \gamma^{(x_t)} \gamma^{(x_0)}\big).$$

The term $\gamma^\mu \Omega_\mu$, direction by direction. The field equation will contain the matrix $\gamma^\mu \Omega_\mu$ (summed over μ). For an inflating direction x_i:

$$\gamma^{x_i} \Omega_{x_i} = \frac{\gamma^{(x_i)}}{e^{a_4} \sin^{1/6} z} \cdot \tfrac{1}{2} e^{a_4} \sin^{1/6} z \, \big(a_4' \gamma^{(x_i)} \gamma^{(x_4)} + H \gamma^{(x_i)} \gamma^{(x_0)}\big) = \tfrac{1}{2} \big(a_4' \gamma^{(x_4)} + H \gamma^{(x_0)}\big)$$

(the frame factors cancel; then $\gamma^{(x_i)} \gamma^{(x_i)} = \eta_{ii} = +1$). For a deflating extra time x_t:

$$\begin{aligned} \gamma^{x_t} \Omega_{x_t} &= \frac{\gamma^{(x_t)}}{f_t} \cdot \Big(-\tfrac{1}{2} f_t \Big) \big(r \, \gamma^{(x_4)} \gamma^{(x_t)} + H \gamma^{(x_t)} \gamma^{(x_0)}\big) \\ &= -\tfrac{1}{2} \big(r \, \gamma^{(x_t)} \gamma^{(x_4)} \gamma^{(x_t)} + H \gamma^{(x_t)} \gamma^{(x_t)} \gamma^{(x_0)}\big) \end{aligned}$$

(the frame factors cancel), and $\gamma^{(x_t)} \gamma^{(x_4)} \gamma^{(x_t)} = -\gamma^{(x_4)} \gamma^{(x_t)} \gamma^{(x_t)} = -\gamma^{(x_4)} \cdot (-1) = \gamma^{(x_4)}$ (exchange two different gammas, then $\gamma^{(x_t)} \gamma^{(x_t)} = \eta_{tt} = -1$), and $\gamma^{(x_t)} \gamma^{(x_t)} \gamma^{(x_0)} = -\gamma^{(x_0)}$; so

$$\gamma^{x_t} \Omega_{x_t} = -\tfrac{1}{2} \big(r \, \gamma^{(x_4)} - H \gamma^{(x_0)}\big) = -\tfrac{1}{2} \Big(a_4' - \frac{b_4'}{2b_4}\Big) \gamma^{(x_4)} + \tfrac{1}{2} H \gamma^{(x_0)}.$$

Summing over the three inflating directions, the three deflating extra times and the two directions x_0, x_4 (which give 0):

$$\gamma^\mu \Omega_\mu = 3 \cdot \tfrac{1}{2} a_4' \gamma^{(x_4)} - 3 \cdot \tfrac{1}{2} \Big(a_4' - \frac{b_4'}{2b_4}\Big) \gamma^{(x_4)} + 6 \cdot \tfrac{1}{2} H \gamma^{(x_0)} = 3H \gamma^{(x_0)} + \frac{3}{4} \frac{b_4'}{b_4} \gamma^{(x_4)}.$$

The a_4' terms of the three inflating directions and the three deflating extra times cancel exactly, because 3-space inflates at the rate at which the extra times deflate; the extra times leave the term $\frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ of their factor $b_4^{1/2}$; the hidden-direction terms of all six directions add up. The result does not contain a_4 at all, for every function $a_4(x_4)$; on the backgrounds $b_4 = 1$ (and for every constant b_4) it is $3H\gamma^{(x_0)}$ (PROVED; Notebook 07b, In [9], Figure 07b.2). We call $M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ the **gravitational term**.

Two more facts about Ω_μ. (i) For each direction separately, γ^μ and Ω_μ anticommute: $\gamma^\mu \Omega_\mu + \Omega_\mu \gamma^\mu = 0$ (no sum). The reason: Ω_μ contains only products $\gamma^{(\mu)} \gamma^{(b)}$ that share the index μ with $\gamma^{(\mu)}$, and $\gamma^{(\mu)} \gamma^{(\mu)} \gamma^{(b)} = \eta_{\mu\mu} \gamma^{(b)}$ while $\gamma^{(\mu)} \gamma^{(b)} \gamma^{(\mu)} = -\gamma^{(\mu)} \gamma^{(\mu)} \gamma^{(b)} = -\eta_{\mu\mu} \gamma^{(b)}$; the two add to 0. (ii) The same matrix is a derivative of the volume factor and the coordinate gammas:

$$\begin{aligned} \frac{1}{2\sqrt{|g|}} \, \partial_\mu \big(\sqrt{|g|} \, \gamma^\mu\big) &= \frac{1}{2 b_4^{3/2} \cos z} \Big[\partial_0 \Big(b_4^{3/2} \cos z \cdot \frac{\sin z}{\cos z} \Big) \gamma^{(x_0)} + \partial_4 \big(b_4^{3/2} \cos z\big) \gamma^{(x_4)} \Big] \\ &= 3H \gamma^{(x_0)} + \frac{3}{4} \frac{b_4'}{b_4} \gamma^{(x_4)} = \gamma^\mu \Omega_\mu. \end{aligned}$$

The first equality: $\sqrt{|g|}\gamma^\mu = b_4^{3/2}\cos z\,\gamma^{(\mu)}/f_\mu$ depends only on x_4 and x_0, so only $\mu = x_0$ (with $1/f_0 = \tan z = \sin z/\cos z$) and $\mu = x_4$ (with $f_4 = 1$) contribute. The second: $\cos z \cdot \sin z/\cos z = \sin z$, whose derivative along x_0 is $6H\cos z$, which gives $6Hb_4^{3/2}\cos z/(2b_4^{3/2}\cos z) = 3H$; and $\partial_4 b_4^{3/2} = \frac{3}{2}b_4^{1/2}b_4'$, which gives $\frac{3}{2}b_4^{1/2}b_4'/(2b_4^{3/2}) = \frac{3}{4}b_4'/b_4$. So $\partial_\mu(\sqrt{|g|}\gamma^\mu) = 2\sqrt{|g|}\,\gamma^\mu\Omega_\mu$ in this metric (PROVED; Notebook 07b, In [11]). Both facts are used in the next three sections.

Where the statements of this section are verified. The reports are named by their file names; the theory reports lie in the folder `Revision/theory/reports`. The formula records are `Revision/theory/field-theory-b4.json` (the metric with b_4) and `Revision/theory/field-theory.json` (the metric at $b_4 = 1$); the reports without "b4" in their name check the metric at $b_4 = 1$.

statement	status	where it is verified
$\sqrt{\|g\|} = b_4^{3/2}\cos z$ ($\cos z$ at $b_4 = 1$)	PROVED; Notebook 07b, In [4]	`wolfram-field-theory-b4.json`, check `sqrt_det_g`; at $b_4 = 1$ `python-field-theory.json`, check `sqrt_det_g_equals_cos_z`, and `wolfram-field-theory.json`, check `sqrt_det_g_is_cos_z`
25 independent nonzero Christoffel symbols	COMPUTED exactly; In [7]	`python-field-theory-b4.json`, check `christoffel_nonzero`; `wolfram-field-theory-b4.json`, check `christoffel_count`; formula records, key `christoffel_nonzero`; at $b_4 = 1$ `python-field-theory.json`, check `christoffel_symmetric_metric_compatible`
the vielbein postulate; the 12 nonzero components of $\omega_{\mu ab}$	COMPUTED exactly; In [8]	`wolfram-field-theory-b4.json`, checks `vielbein_postulate` and `omega_components`; `python-field-theory-b4.json`, check `omega_nonzero`; formula records, key `omega_nonzero`; at $b_4 = 1$ `python-field-theory.json`, check `vielbein_postulate`

statement	status	where it is verified
$\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$; the a_4' terms cancel, the hidden terms add	PROVED; In [9]	`python-field-theory-b4.json`, checks `gammaOmega_per_direction` and `gammaOmega_total`; `wolfram-field-theory-b4.json`, checks `gammaOmega_per_direction` and `gammaOmega_total`; at $b_4 = 1$ `python-field-theory.json`, checks `gamma_mu_Omega_mu_equals_3H_gamma_x0` and `time_terms_cancel_hidden_term_survives`, and `wolfram-field-theory.json`, checks `gammaOmega_equals_3H_gamma_x0` and `gammaOmega_x4_terms_cancel`
$\gamma^\mu \Omega_\mu + \Omega_\mu \gamma^\mu = 0$ for each μ; $\partial_\mu(\sqrt{\lvert g\rvert}\gamma^\mu) = 2\sqrt{\lvert g\rvert}\gamma^\mu \Omega_\mu$	PROVED; In [11]	`wolfram-field-theory-b4.json`, check `gammaOmega_divergence_form`; at $b_4 = 1$ `python-field-theory.json`, checks `anticommutator_gamma_Omega_vanishes` and `divergence_of_sqrtg_gamma`

7.19 The two fields and their common Lagrangian

Why the row contains C. A Lagrangian must not depend on the choice of the local frames, so it must be built from combinations of the field that do not change when the frames are turned. Under a turning of the frames the column changes as $\Psi \to R\Psi$ with a real 16×16 **spin transformation** R (Chapter 5). The simplest number, $\Psi^\dagger\Psi = \sum_A |\Psi_A|^2$, becomes $\Psi^\dagger R^T R \Psi$, which equals $\Psi^\dagger\Psi$ only if $R^T R = 1$: true for rotations, false for boosts (Chapter 5). The author's matrix C cures this: every spin generator satisfies $(S^{ab})^T C = -C S^{ab}$ (`python-field-theory.json`, check `S_definition_and_C_S_antisymmetric`), and therefore $R^T C R = C$ for every spin transformation (Chapter 5), so

$$\Psi^\dagger C \Psi \to \Psi^\dagger R^T C R\, \Psi = \Psi^\dagger C \Psi.$$

The row $\bar\Psi = \Psi^\dagger C$ is called the **Dirac adjoint** of Ψ, and $S = \bar\Psi\Psi$ the **scalar density**. In the same way the eight numbers $\bar\Psi\gamma^{(a)}\Psi$ change like the components of a vector under the turnings, and contracted with a covariant derivative they give the frame-independent combination $\bar\Psi\gamma^\mu D_\mu \Psi$.

The Lagrangian. For both fields the Revision record uses the same Lagrangian density (Revision/SPEC.md, section 3; ASSUMED as the definition of the two theories):

$$\mathcal{L} = \sqrt{|g|}\,\Big[\tfrac{1}{2}(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi) - m\,S - U(S)\Big], \qquad \bar\Psi = \Psi^\dagger C, \quad S = \bar\Psi\Psi, \quad U(S) = \tfrac{\lambda}{2}S^2,$$

with a sum over $\mu = x_0, \ldots, x_7$. Read term by term:

- $\sqrt{|g|} = b_4^{3/2}\cos z$ turns the coordinate volume into proper volume, so that the action does not depend on the coordinates.
- The **kinetic term** $K = \frac{1}{2}(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi)$ contains each derivative to the first power, like the first-order Lagrangians of Section 7.4. It is **symmetrised**: the derivative acts once on Ψ and once on $\bar\Psi$, with opposite signs (Section 7.20 shows why).
- The **mass term** $-mS = -m\bar\Psi\Psi$ is linear in the mass m (a real number) and quadratic in the field: every term contains one component of $\Psi^\dagger$ and one of Ψ.
- The **interaction** $-U(S) = -\frac{\lambda}{2}S^2$ with a real coupling λ is quartic in the field. For dirac16complex it must be a polynomial in S (Section 7.12: $S^{17} = 0$); for dirac16complex00 any real function of S would be allowed, and the record keeps the same U so that the two theories differ only in the statistics. We write $V = m + U'(S) = m + \lambda S$, the **effective mass**.
- Gravity enters through $\sqrt{|g|}$, through the coordinate gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$, and through the spinor connection in D_μ.

The structure is that of Example B of Section 7.4: Ψ plays the role of ψ, the row $\bar\Psi$ that of ψ^*, and the kinetic term is symmetrised in the same way. The factor i of Example B is not needed, because the matrices $C\gamma^{(a)}$ are antisymmetric (next section).

The two statistics. For dirac16complex the 16 components Ψ_A and the 16 conjugates Ψ_A^* are 32 Grassmann generators at each point, with the rules of Sections 7.10 and 7.11; for dirac16complex00 they are ordinary complex numbers. Every term of $\mathcal{L}$ is written with all factors of $\Psi^\dagger$ (or of its derivatives) to the LEFT of all factors of Ψ, so the formula can be read for both kinds of numbers without any reordering.

7.20 Reality, the total divergence, and the connection that drops out of $\mathcal{L}$

The lemma for two columns. For two columns Ψ and Φ (for example $\Phi = D_\mu\Psi$), both commuting or both Grassmann, and a matrix M of ordinary numbers,

$$\big(\Psi^\dagger M\Phi\big)^* = \Phi^\dagger M^\dagger\Psi.$$

The proof is the three-line computation of Section 7.12 with Φ in place of the second Ψ: for Grassmann components the conjugation reverses the order of the two factors; for commuting components nothing needs reordering (PROVED).

The Lagrangian is real. Call $K_u = \bar\Psi\gamma^\mu D_\mu\Psi = \Psi^\dagger(C\gamma^\mu)(D_\mu\Psi)$ the **unsymmetrised** kinetic term. Line by line:

$$K_u^* = (D_\mu\Psi)^\dagger (C\gamma^\mu)^\dagger \Psi$$

(the lemma with $M = C\gamma^\mu$ and $\Phi = D_\mu\Psi$);

$$= -(D_\mu\Psi)^\dagger C\gamma^\mu \Psi$$

($C\gamma^\mu = C\gamma^{(\mu)}/f_\mu$ is real and antisymmetric, so its dagger is its transpose, which is minus itself);

$$= -(D_\mu\bar\Psi)\gamma^\mu\Psi$$

(because $(D_\mu\Psi)^\dagger C = \partial_\mu\Psi^\dagger C + \Psi^\dagger\Omega_\mu^T C = \partial_\mu\bar\Psi - \Psi^\dagger C\Omega_\mu = D_\mu\bar\Psi$: the matrix Ω_μ is real, and $\Omega_\mu^T C = -C\Omega_\mu$ since Ω_μ is a combination of the S^{ab} with real coefficients and $(S^{ab})^T C = -CS^{ab}$). So the kinetic term $K = \frac{1}{2}(K_u - (D_\mu\bar\Psi)\gamma^\mu\Psi) = \frac{1}{2}(K_u + K_u^*)$ is real. Next $S^* = \Psi^\dagger C^\dagger\Psi = S$, because C is real and symmetric (the lemma with $M = C$). For Grassmann components S is even, so $(S^2)^* = S^*S^* = S^2$, and $U(S) = \frac{\lambda}{2}S^2$ is real; for commuting components S is a real number. Finally m, λ and $\sqrt{|g|}$ are real. Hence $\mathcal{L}^* = \mathcal{L}$ for both statistics (PROVED; Notebook 07b, In [15]; `python-field-theory.json`, checks `grassmann_lagrangian_real` and `commuting_lagrangian_real`; `wolfram-field-theory.json`, checks `L_real_G` and `L_real_C`). The unsymmetrised form $\sqrt{|g|}\,[K_u - mS - U]$ is NOT real: its imaginary part is the difference $K_u - K_u^*$, which is not zero for a general field (checks `grassmann_controls_not_vacuous` and `commuting_controls_not_vacuous`); this control shows that the reality test is not empty. A real Lagrangian matters: the equations obtained by varying $\Psi^\dagger$ and by varying Ψ are then conjugates of each other (Section 7.21), so there are exactly as many equations as unknown components.

The two forms differ by a total divergence. The product rule along x_μ, applied to the three factors of $\sqrt{|g|}\,\bar\Psi\gamma^\mu\Psi$ (with the matrix $\sqrt{|g|}\gamma^\mu$ as one factor), gives

$$\partial_\mu(\sqrt{|g|}\,\bar\Psi\gamma^\mu\Psi) = \sqrt{|g|}\,(\partial_\mu\bar\Psi)\gamma^\mu\Psi + \bar\Psi\,\partial_\mu(\sqrt{|g|}\gamma^\mu)\Psi + \sqrt{|g|}\,\bar\Psi\gamma^\mu\partial_\mu\Psi$$

(for Grassmann components the derivative is even and needs no sign; the order of the factors is kept);

$$= \sqrt{|g|}\,[(\partial_\mu\bar\Psi)\gamma^\mu\Psi + 2\bar\Psi\gamma^\mu\Omega_\mu\Psi + \bar\Psi\gamma^\mu\partial_\mu\Psi]$$

(the identity $\partial_\mu(\sqrt{|g|}\gamma^\mu) = 2\sqrt{|g|}\gamma^\mu\Omega_\mu$ of Section 7.18);

$$= \sqrt{|g|}\,[(\partial_\mu\bar\Psi - \bar\Psi\Omega_\mu)\gamma^\mu\Psi + \bar\Psi\gamma^\mu(\partial_\mu\Psi + \Omega_\mu\Psi)] = \sqrt{|g|}\,[(D_\mu\bar\Psi)\gamma^\mu\Psi + \bar\Psi\gamma^\mu D_\mu\Psi]$$

(split $2\gamma^\mu\Omega_\mu = \gamma^\mu\Omega_\mu - \Omega_\mu\gamma^\mu$, using $\gamma^\mu\Omega_\mu = -\Omega_\mu\gamma^\mu$ for each μ; then the definitions of the covariant derivatives). Now $K = \frac{1}{2}(K_u - (D_\mu\bar\Psi)\gamma^\mu\Psi) = K_u - \frac{1}{2}(K_u + (D_\mu\bar\Psi)\gamma^\mu\Psi)$, so

$$\mathcal{L} = \sqrt{|g|}\,[\bar\Psi\gamma^\mu D_\mu\Psi - mS - U(S)] - \tfrac{1}{2}\,\partial_\mu(\sqrt{|g|}\,\bar\Psi\gamma^\mu\Psi).$$

The two forms differ by a total divergence (the field version of the total derivative of Section 7.4) and give the same field equations; only the symmetrised one is real (PROVED; Notebook 07b, In

[15]; checks `grassmann_total_divergence_relation` and `commuting_total_divergence_relation`; Wolfram `L_total_divergence_to_unsymmetrised_G` and `_C`).

The spin connection drops out of $\mathcal{L}$ in this metric. The terms of K that contain Ω_μ are $\frac{1}{2}\bar{\Psi}\gamma^\mu\Omega_\mu\Psi$ from the first half and $-\frac{1}{2}(-\bar{\Psi}\Omega_\mu)\gamma^\mu\Psi = +\frac{1}{2}\bar{\Psi}\Omega_\mu\gamma^\mu\Psi$ from the second, together

$$\tfrac{1}{2}\,\bar{\Psi}\big(\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu\big)\Psi = 0,$$

because γ^μ and Ω_μ anticommute for each μ (Section 7.18). So in the author's metric

$$\mathcal{L} = b_4^{3/2}\cos z\,\Big[\tfrac{1}{2}\sum_{a=0}^{7}\frac{1}{f_a}\big(\bar{\Psi}\gamma^{(a)}\partial_a\Psi - \partial_a\bar{\Psi}\,\gamma^{(a)}\Psi\big) - mS - U(S)\Big]$$

contains no spin connection at all (PROVED; Notebook 07b, In [15]; Wolfram checks `L_spin_connection_drops_out_G` and `L_spin_connection_drops_out_C`). Gravity is still present in $\mathcal{L}$, through $b_4^{3/2}\cos z$ and the frame factors $1/f_a$. Section 7.21 shows that the connection comes back in the field equation, through the derivative of $\sqrt{|g|}\gamma^\mu$.

Counting the terms of $\mathcal{L}$ at one point. Written in the jet symbols of Section 7.13 (the 16 components, their 16 conjugates and their first derivatives at one point), the Lagrangian is a sum of monomials. Every $C\gamma^{(a)}$ is a signed permutation matrix, so in each of the 8 directions $\bar{\Psi}\gamma^{(a)}\partial_a\Psi$ gives 16 monomials $\Psi_A^*\,\partial_a\Psi_B$ and $\partial_a\bar{\Psi}\gamma^{(a)}\Psi$ another 16: $8\times 32 = 256$ kinetic monomials. The mass term gives 16 (one for each nonzero entry of C). The quartic term $\frac{\lambda}{2}S^2$ is $\frac{\lambda}{2}\big(\sum_A c_A P_A\big)^2$ with the 16 pairs P_A of Section 7.12: the $\binom{16}{2} = 120$ products of two different pairs survive for both statistics, while the 16 squares P_A^2 survive only for commuting numbers. So $\mathcal{L}$ has $256 + 16 + 120 = 392$ monomials for dirac16complex and $256 + 16 + 136 = 408$ for dirac16complex00 (COMPUTED by Notebook 07b, In [13] and In [14], Figure 07b.3; the counts reproduce those written in `python-field-theory.json`, checks `grassmann_lagrangian_real` and `commuting_lagrangian_real`).

7.21 The Euler-Lagrange equations of both fields

The derivation by hand, for the commuting field. Write $V = m + \lambda S$ and, with $\chi_A = \Psi_A^*$ the components of $\Psi^\dagger$,

$$\frac{\mathcal{L}}{\sqrt{|g|}} = \tfrac{1}{2}\,\chi C\gamma^\mu\big(\partial_\mu\Psi + \Omega_\mu\Psi\big) - \tfrac{1}{2}\big(\partial_\mu\chi\,C - \chi C\Omega_\mu\big)\gamma^\mu\Psi - m\,\chi C\Psi - \tfrac{\lambda}{2}\big(\chi C\Psi\big)^2,$$

where χ is the row $\Psi^\dagger$. Vary the component χ_A; by Section 7.4 the components and their conjugates may be varied as if they were independent. The Euler-Lagrange expression of a field (Section 7.5) needs two partial derivatives.

- Step 1. $\partial\mathcal{L}/\partial\chi_A = \sqrt{|g|}\,\big[\frac{1}{2}(C\gamma^\mu D_\mu\Psi)_A + \frac{1}{2}(C\Omega_\mu\gamma^\mu\Psi)_A - V(C\Psi)_A\big]$. Rule: a term χX with a column X has the derivative X_A with respect to χ_A; the second half contributes through its part $+\frac{1}{2}\chi C\Omega_\mu\gamma^\mu\Psi$; and the chain rule gives $\partial\big(\frac{\lambda}{2}S^2\big)/\partial\chi_A = \lambda S\,(C\Psi)_A$.
- Step 2. $\partial\mathcal{L}/\partial(\partial_\mu\chi_A) = -\frac{1}{2}\sqrt{|g|}\,(C\gamma^\mu\Psi)_A$. Rule: only the second half contains $\partial_\mu\chi$.

- Step 3. The Euler-Lagrange expression is Step 1 minus ∂_μ of Step 2, that is Step 1 plus $\frac{1}{2}\partial_\mu(\sqrt{|g|}\,C\gamma^\mu\Psi)_A$.
- Step 4. By the product rule and Section 7.18, $\partial_\mu(\sqrt{|g|}\gamma^\mu\Psi) = \partial_\mu(\sqrt{|g|}\gamma^\mu)\Psi + \sqrt{|g|}\gamma^\mu\partial_\mu\Psi = 2\sqrt{|g|}\,\gamma^\mu\Omega_\mu\Psi + \sqrt{|g|}\,\gamma^\mu\partial_\mu\Psi$ (C is constant and comes out of the derivative).
- Step 5. Add: the expression is $\sqrt{|g|}\,\big(C\,[\frac{1}{2}\gamma^\mu D_\mu\Psi + \frac{1}{2}\Omega_\mu\gamma^\mu\Psi + \gamma^\mu\Omega_\mu\Psi + \frac{1}{2}\gamma^\mu\partial_\mu\Psi - V\Psi]\big)_A$. With $\Omega_\mu\gamma^\mu = -\gamma^\mu\Omega_\mu$ the second and third terms give $\frac{1}{2}\gamma^\mu\Omega_\mu\Psi$, and $\frac{1}{2}\gamma^\mu\Omega_\mu\Psi + \frac{1}{2}\gamma^\mu\partial_\mu\Psi = \frac{1}{2}\gamma^\mu D_\mu\Psi$. So the expression is $\sqrt{|g|}\,\big(C\,(\gamma^\mu D_\mu\Psi - V\Psi)\big)_A$.
- Step 6. $\sqrt{|g|} = b_4^{3/2}\cos z \neq 0$ and C is invertible ($C^2 = 1$), so the 16 expressions vanish exactly when

$$\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi.$$

This is the **field equation** of both fields (for a general potential, $m + U'(S)$ in place of $m + \lambda S$). The spin connection, which was absent from $\mathcal{L}$, is present in it: Step 4 brought it back through $\partial_\mu(\sqrt{|g|}\gamma^\mu)$.

Grassmann components. For dirac16complex the LEFT derivative with respect to χ_A is used (Section 7.11). In every term of $\mathcal{L}$ the factor χ (or $\partial_\mu\chi$) stands on the far left, so moving it there costs no sign, and S is even, so the chain rule of Step 1 holds unchanged. Steps 1 to 6 go through word for word: the field equation is the same for both statistics (PROVED; Notebook 07b, In [16], with the control that the comparison with m replaced by $-m$ fails).

The adjoint equation. Varying Ψ instead (for Grassmann components with RIGHT derivatives, because Ψ stands on the far right) gives in the same way

$$(D_\mu\bar\Psi)\gamma^\mu = -(m + \lambda S)\bar\Psi.$$

It is not a new equation: it is the **Dirac conjugate** of the field equation. With the residual $\mathcal{E} = \gamma^\mu D_\mu\Psi - V\Psi$, line by line:

$$\mathcal{E}^\dagger C = (D_\mu\Psi)^\dagger(\gamma^\mu)^T C - V\Psi^\dagger C$$

(the dagger of a product reverses it; γ^μ and V are real; for Grassmann components the conjugation reverses the order as well);

$$= -(D_\mu\Psi)^\dagger C\gamma^\mu - V\bar\Psi = -(D_\mu\bar\Psi)\gamma^\mu - V\bar\Psi$$

($(\gamma^\mu)^T C = -C\gamma^\mu$, because $C\gamma^\mu$ is antisymmetric and C symmetric; then $(D_\mu\Psi)^\dagger C = D_\mu\bar\Psi$ as in Section 7.20). So $\mathcal{E}^\dagger C = -\big[(D_\mu\bar\Psi)\gamma^\mu + V\bar\Psi\big]$: the residual of the adjoint equation is minus the Dirac conjugate of the residual of the field equation. One equation holds exactly when the other does, and the two form one system (PROVED; Notebook 07b, In [16]).

Where the statements of this section are verified (in the folder `Revision/theory/reports`; the sympy names start with `grassmann_` or `commuting_` for the two statistics, the Wolfram names end in `_G` or `_C`):

statement	status	where it is verified
varying $\Psi^\dagger$ gives $\gamma^\mu D_\mu \Psi = (m + \lambda S)\Psi$, both statistics	PROVED; Notebook 07b, In [16]	`python-field-theory.json`, checks `grassmann_euler_lagrange_psibar_variation` and `commuting_euler_lagrange_psibar_variation`; `wolfram-field-theory.json`, checks `EL_Psibar_G` and `EL_Psibar_C`
varying Ψ gives the adjoint equation	PROVED; In [16]	`python-field-theory.json`, checks `grassmann_euler_lagrange_psi_variation` and `commuting_euler_lagrange_psi_variation`; `wolfram-field-theory.json`, checks `EL_Psi_G` and `EL_Psi_C`
the adjoint equation is the Dirac conjugate of the field equation	PROVED; In [16]	`python-field-theory.json`, checks `grassmann_adjoint_equation_is_conjugate` and `commuting_adjoint_equation_is_conjugate`; `wolfram-field-theory.json`, checks `adjoint_equation_is_Dirac_conjugate_G` and `adjoint_equation_is_Dirac_conjugate_C`

7.22 The equations in the author's metric: explicit, block and evolution forms

The explicit form. Since $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ and $\gamma^\mu \Omega_\mu = M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ (Section 7.18), the Dirac operator is $\gamma^\mu D_\mu \Psi = \sum_a \frac{1}{f_a}\gamma^{(a)}\partial_a \Psi + M\Psi$, and the field equation in the author's metric reads

$$\tan z\, \gamma^{(x_0)}\partial_0 \Psi + \frac{1}{e^{a_4}\sin^{1/6} z}\sum_{i=1}^{3}\gamma^{(x_i)}\partial_i\Psi + \gamma^{(x_4)}\partial_4\Psi + \frac{e^{a_4}}{b_4^{1/2}\sin^{1/6} z}\sum_{t=5}^{7}\gamma^{(x_t)}\partial_t\Psi + 3H\gamma^{(x_0)}\Psi + \frac{3}{4}\frac{b_4'}{b_4}\gamma^{(x_4)}\Psi = (m+\lambda S)\Psi$$

(PROVED; Notebook 07b, In [17]; `wolfram-field-theory-b4.json`, checks `Dirac_operator_explicit_G` and `Dirac_operator_explicit_C`; formula records, key `field_equation`; at $b_4 = 1$ the same checks of `wolfram-field-theory.json`). The coefficient $e^{a_4} b_4^{-1/2}\sin^{-1/6} z$ of the extra-time derivatives GROWS as the extra times deflate, while that of the 3-space derivatives shrinks.

One component equation, read off the gamma tables. Every frame gamma is a signed permutation matrix: row 1 of $\gamma^{(x_0)}, \gamma^{(x_1)}, \ldots, \gamma^{(x_7)}$ has its single entry in the columns $+9, -16, +15, -14, -14, +15, +16, +9$ (the sign of the entry and its column; the tables of Chapter 5, read from the record `Revision/algebra/gammas.json`). So the first of the 16 component equations is

$$\tan z\, \partial_0 \Psi_9 + \frac{-\partial_1 \Psi_{16} + \partial_2 \Psi_{15} - \partial_3 \Psi_{14}}{e^{a_4} \sin^{1/6} z} - \partial_4 \Psi_{14} + \frac{e^{a_4}(\partial_5 \Psi_{15} + \partial_6 \Psi_{16} + \partial_7 \Psi_9)}{b_4^{1/2} \sin^{1/6} z} + 3H\Psi_9 - \frac{3}{4}\frac{b_4'}{b_4}\Psi_{14} = (m + \lambda S)\Psi_1.$$

Notebook 07b prints this equation in its own symbols and the same equation read from the formula record (`Revision/theory/field-theory-b4.json`, key `field_equation_components`), and checks that all 16 equations agree term by term with it and, at $b_4 = 1$, with the record `Revision/theory/field-theory.json` (In [17]). Each equation contains four components, each with derivatives along two directions (here Ψ_9 along x_0 and x_7, Ψ_{16} along x_1 and x_6, Ψ_{15} along x_2 and x_5, Ψ_{14} along x_3 and x_4), and the gravitational term $3H\Psi_B$ stands next to the hidden-direction derivative $\tan z\, \partial_0 \Psi_B$ of the same component, the term $-\frac{3}{4}(b_4'/b_4)\Psi_B$ next to the time derivative $-\partial_4 \Psi_B$ (Figure 07b.4).

The chiral block form. Split Ψ into its upper half ψ_- (components 1 to 8, where the chirality matrix $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)} \cdots \gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$ is -1; Chapter 5) and its lower half ψ_+ (components 9 to 16, $\Gamma = +1$). Every frame gamma anticommutes with Γ, and therefore has the block form

$$\gamma^{(a)} = \begin{pmatrix} 0 & \bar{\tau}_a \\ \tau_a & 0 \end{pmatrix}$$

with 8×8 blocks $\bar{\tau}_a$ (upper right) and τ_a (lower left); for the hidden direction $\bar{\tau}_{x_0} = \tau_{x_0} = I_8$. (Why: write $\gamma^{(a)}$ in 8×8 blocks P, Q, R, T; then $\Gamma\gamma^{(a)} = \begin{pmatrix} -P & -Q \\ R & T \end{pmatrix}$ and $\gamma^{(a)}\Gamma = \begin{pmatrix} -P & Q \\ -R & T \end{pmatrix}$, and $\Gamma\gamma^{(a)} = -\gamma^{(a)}\Gamma$ forces $P = 0$ and $T = 0$.) A block matrix times the column (ψ_-, ψ_+) gives $(\bar{\tau}_a\psi_+, \tau_a\psi_-)$: the upper rows see only the lower half. The term $V\Psi$ keeps each half in its own rows. So the field equation is the pair

$$\sum_a \frac{1}{f_a}\bar{\tau}_a\, \partial_a \psi_+ + 3H\psi_+ + \tfrac{3}{4}\tfrac{b_4'}{b_4}\bar{\tau}_{x_4}\psi_+ = V\psi_-, \qquad \sum_a \frac{1}{f_a}\tau_a\, \partial_a \psi_- + 3H\psi_- + \tfrac{3}{4}\tfrac{b_4'}{b_4}\tau_{x_4}\psi_- = V\psi_+ :$$

the derivatives and the gravitational term of one half are balanced by the mass term of the other half (PROVED; Notebook 07b, In [19]; Wolfram check `block_form`; formula record key `field_equation_blocks`).

The evolution form. Write the explicit equation with the x_4 term separately ($f_4 = 1$), with the gravitational term $M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$, and solve it for $\partial_4\Psi$, line by line:

$$\gamma^{(x_4)}\partial_4\Psi + \sum_{a \neq x_4} \frac{1}{f_a}\gamma^{(a)}\partial_a\Psi + M\Psi = V\Psi$$

(the field equation);

$$\gamma^{(x_4)}\partial_4\Psi = V\Psi - \sum_{a\neq x_4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi - M\Psi$$

(every other term moved to the right side);

$$\partial_4\Psi = -\gamma^{(x_4)}\Big[V\Psi - \sum_{a\neq x_4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi - M\Psi\Big]$$

(multiplied from the left by $-\gamma^{(x_4)}$, using $-\gamma^{(x_4)}\gamma^{(x_4)} = 1$ because $(\gamma^{(x_4)})^2 = \eta_{44} = -1$). So the time derivative of every component is fixed by the field and its derivatives along the other seven directions at the same time x_4: the slices $x_4 = \text{const}$ are **non-characteristic** (PROVED; Notebook 07b, In [19]; `wolfram-field-theory-b4.json`, checks `evolution_form_G` and `evolution_form_C`; the part $\frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ of M gives $-\frac{3}{4}(b_4'/b_4)\Psi$ on the right, formula record key `evolution_form`). Scope, from the Revision scope record: this does NOT make the initial-value problem well posed. For data that depend on the extra times the growth rates of the modes have no upper bound (Section 7.5; `Revision/theory/reports/python-scope.json`, check `extra_time_growth_rates_unbounded`), so small changes of the data can grow without limit in any time; Chapter 8 treats this.

7.23 Non-triviality [1] and [2], their exact scope, and an exact family of solutions

The author's requirement. The author asked that the Lagrangians of both fields be **non-trivial**: their Euler-Lagrange equations must always contain nonzero contributions from gravity through the canonical spin connection, unless space-time is flat 4+4 space ([1] for dirac16complex, [2] for dirac16complex00), and must be self-consistent.

What "self-consistent" means for field equations. Here the word has a meaning of its own, different from its meaning for a Kohn-Sham state in Chapters 13 to 15 (there a state is self-consistent when the potential computed from the density of its orbitals is the potential that produced them, a fixed point). Field equations are **self-consistent** when they do not contradict one another or the geometry in which they live: the equations obtained by varying different fields form one system, and the further equations that follow from them (by differentiating them, or by combining them with identities of the metric) hold without any extra condition on the field or on the metric (this plain-word summary is the book's; the exact content is the list of senses below). The specification of the Revision record (`Revision/SPEC.md`, section 3) names three parts: the integrability of the field equations, the conserved current and the Bianchi-type identity of the energy-momentum tensor. The record itself (`Revision/docs/DIRAC16COMPLEX_FIELD_THEORY.md`, section 8, "Self-consistency") states six exact senses. This book treats them as follows. The record proves (iii), (v) and (vi) for the metric at $b_4 = 1$; for a general b_4 the record contains (i), (ii) and the local law of (iv), while the Noether identities of (v), the Lichnerowicz identity of (iii) and the exact solutions of (vi) were NOT recomputed for a general b_4 (`Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md`; `claude_failures.md`, C1).

(i) One system. The adjoint equation, obtained by varying Ψ, is the Dirac conjugate of the field equation, obtained by varying $\Psi^\dagger$ (PROVED in Section 7.21; Notebook 07b, In [16]; checks `adjoint_equation_is_Dirac_conjugate_G` and `grassmann_adjoint_equation_is_conjugate`).

(ii) An evolution form, but no well-posed initial-value problem. The field equation can be

solved for the time derivative $\partial_4\Psi$, so the slices $x_4 = \text{const}$ are non-characteristic (PROVED in Section 7.22; check `evolution_form_G`), and the Heisenberg equation of the quantised field reproduces this evolution form (Section 10.12; checks `Heisenberg_equation_reproduces_field_equation` and `hamiltonian_form_and_heisenberg_equation`). For data that depend on the extra times the initial-value problem is NOT well posed (Section 7.5 and Sections 8.16 to 8.18; check `extra_time_growth_rates_unbounded`).

(iii) Integrability. A first-order equation implies a second-order one. Take $U = 0$, so that the right side is $m\Psi$ with a constant number m. Line by line: $\gamma^\nu D_\nu(\gamma^\mu D_\mu\Psi) = \gamma^\nu D_\nu(m\Psi)$ (apply the operator $\gamma^\nu D_\nu$ to both sides of the field equation); $= m\,\gamma^\nu D_\nu\Psi$ (the number m is constant, and D_ν is linear); $= m^2\Psi$ (the field equation once more). So every solution obeys $(\gamma^\mu D_\mu)^2\Psi = m^2\Psi$. The Revision record proves the **Lichnerowicz identity** for the square of the Dirac operator in the author's metric, for every Ψ, solution or not:

$$(\gamma^\mu D_\mu)^2\Psi = g^{\mu\nu}\big(D_\mu D_\nu\Psi - \Gamma^\lambda{}_{\mu\nu} D_\lambda\Psi\big) - \frac{R}{4}\Psi, \qquad R = 6\big((a_4')^2 - 7H^2\big) \text{ at } b_4 = 1.$$

Here $D_\mu D_\nu\Psi$ is D_μ applied to the column $D_\nu\Psi$; the term with the Christoffel symbols $\Gamma^\lambda{}_{\mu\nu}$ (Section 3.15; not the matrix Γ) corrects for the index ν, as for the covariant derivative of a vector; the sum with the inverse metric $g^{\mu\nu}$ makes the whole first term the curved-space form of the wave operator; and R is the Ricci scalar of the author's metric (Section 3.17; its value is derived in Sections 3.25 and 8.7; with b_4 it is $R = 6a_4'^2 - 3a_4'b_4'/b_4 + 3b_4''/b_4 - 42H^2$, formula record key `ricci_scalar`). In flat 4+4 space with Cartesian coordinates ($D_\mu = \partial_\mu$, no Christoffel symbols, $R = 0$) the identity reads $(\gamma^{(a)}\partial_a)^2 = \sum_a \eta^{aa}\partial_a^2$, the operator form of the Clifford square of Section 7.5. Combined with the line above, every solution obeys the second-order equation $g^{\mu\nu}(D_\mu D_\nu\Psi - \Gamma^\lambda{}_{\mu\nu}D_\lambda\Psi) - \frac{R}{4}\Psi = m^2\Psi$, in which the curvature appears only as the term $-\frac{R}{4}\Psi$; in the record's words, the second-order consequence of the field equations is consistent with the curvature. Status: the identity is PROVED in the Revision record for the metric at $b_4 = 1$ (checks `Lichnerowicz_identity_G` of `wolfram-field-theory.json`, `lichnerowicz_identity_on_fields` and `lichnerowicz_contraction` of `python-field-theory.json`); it is quoted here and NOT derived in this book. It is a general identity of the square of the Dirac operator, but the record has not checked it for the metric with a changing b_4.

(iv) A conserved current. The phase change $\Psi \to e^{i\alpha}\Psi$ leaves the Lagrangian unchanged, and the quantity that belongs to this symmetry is the **current** $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, whose time component is the charge density (Section 7.28). It is real and obeys the local law $\nabla_\mu J^\mu = 0$, that is $\sum_\mu \partial_\mu(\sqrt{|g|}\,J^\mu) = 0$ with $\sqrt{|g|} = b_4^{3/2}\cos z$, on every solution (PROVED; derived line by line in Section 21.17; checks `current_conservation_identity_G` and `grassmann_current_conservation` at $b_4 = 1$, and `u1_noether_matrix_identity_b4` of `python-field-theory-b4.json` for every b_4). The total charge of a slice of constant x_4 is constant only if no charge flows through the brane $z = \pi/2$; this no-flux condition is ASSUMED, not derived (Sections 21.17 and 21.18).

(v) The Bianchi-type identity of the energy-momentum tensor. The invariance of the action under changes of coordinates and under local frame rotations gives identities that hold for every field, solution or not (like the Bianchi identity of the curvature, Section 3.17); on every solution they make the energy-momentum tensor symmetric and conserved, $\nabla_\mu T^\mu{}_\nu = 0$ (PROVED in the Revision record for the metric at $b_4 = 1$, checks `Noether_identity_diffeomorphisms_G`, `Noether_identity_local_Lorentz_G` and `conservation_on_shell_general`; Chapter 9 uses it without its proof and derives what it says for a diagonal tensor, Sections 9.9 to 9.11).

(vi) Exact solutions as test points. The exact family derived at the end of this section solves the field equation for every a_4 and every constant b_4 (PROVED here; Notebook 07b, In [21]); the record also verifies that T and J are conserved exactly at these solutions, and states the scope of this test: there the pressures vanish, so the general proof is the pair of identities of (v) (checks `exact_solution_x4_x0_G` and `exact_solution_family_x4_x0`).

The non-triviality theorem below answers the first half of the author's requirement, (i) to (vi) the second half. Self-consistency in these senses is a property of the equations: it does not make the initial-value problem well posed (ii), and it imposes no boundary condition at $z = \pi/2$ (iv).

Theorem (non-triviality [1] and [2] in the diagonal frame). Hypotheses: the author's metric with $H > 0$ and $0 < z < \pi/2$; any function $a_4(x_4)$, a constant one included; a constant $b_4 > 0$ (as on the backgrounds $b_4 = 1$; a changing b_4 is treated after the proof); real m and λ; the diagonal frame and its canonical spin connection; a field Ψ of either statistics that is not identically zero. Then the field-equation residual computed with the spin connection minus the one computed with $\Omega_\mu = 0$ is

$$\gamma^\mu \Omega_\mu \Psi = 3H\gamma^{(x_0)}\Psi \neq 0.$$

Proof. The difference is $\gamma^\mu(D_\mu - \partial_\mu)\Psi = \gamma^\mu\Omega_\mu\Psi$ by the definition of D_μ, and $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ for a constant b_4 by Section 7.18. The matrix $\gamma^{(x_0)}$ is a signed permutation matrix with $(\gamma^{(x_0)})^2 = 1$, so every component of $\gamma^{(x_0)}\Psi$ is $\pm$ a component of Ψ, and $3H\gamma^{(x_0)}\Psi = 0$ forces $\Psi = 0$ when $H > 0$. $\square$ (PROVED; Notebook 07b, In [20], checks the difference in all 16 components for both statistics.) Moreover, Ω_μ vanishes for all μ only if $H = 0$, because six of its 12 nonzero components are H times a factor that never vanishes (and the other six are a_4' or $a_4' - b_4'/(2b_4)$ times such a factor); $H = 0$ is not a member of the author's family (it is a degenerate limit in which g_{11} and g_{55} go to 0 and g_{00} to infinity), and the metric is curved for every $H > 0$, since $R^{x_0}{}_{x_0} = -6H^2$ (the checks are listed in the table at the end of this section).

A changing b_4. Then the difference is $M\Psi$ with $M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$. Because $\gamma^{(x_0)}$ and $\gamma^{(x_4)}$ anticommute and $(\gamma^{(x_0)})^2 = 1$, $(\gamma^{(x_4)})^2 = -1$, the cross terms cancel and $MM = (9H^2 - \frac{9}{16}(b_4'/b_4)^2)1$. So $M\Psi = 0$ forces $\Psi = 0$ at every time where $|b_4'/b_4| \neq 4H$; at a time where $|b_4'/b_4| = 4H$ the matrix M has rank 8 and $M\Psi$ vanishes for the fields in its null space (PROVED; `python-field-theory-b4.json`, check `gravity_term_square_b4`; Notebook 07b, In [20], checks the difference $M\Psi$ in all 16 components). The theorem, as the record states it, is the case of a constant b_4.

The exact scope of the theorem. The value $3H\gamma^{(x_0)}$ belongs to the diagonal frame and to the field variables Ψ; read without these hypotheses the theorem would overstate [1] and [2]. The Revision record states, and its scope reports verify exactly:

- In another frame of the same metric, boosted in the (x_4, x_0) plane with the rapidity $6Hx_4 + b_0$, the term $\gamma^\mu\Omega_\mu$ vanishes identically (for a constant b_4; Section 6.16).
- Within the diagonal frame the rescaling $\Psi = \sin^{-1/2} z\, \chi$ removes it (derived below for a constant b_4; for a changing b_4 the rescaling $\Psi = \sin^{-1/2} z\, b_4^{-3/4}\chi$ removes all of M, Section 6.18).
- The deflation contributes nothing to $\gamma^\mu\Omega_\mu$: a_4 enters the field equation only through the frame factors $e^{-a_4}\sin^{-1/6} z$ and $e^{a_4}b_4^{-1/2}\sin^{-1/6} z$ of the derivative terms. And since the connection

drops out of $\mathcal{L}$ (Section 7.20), the Euler-Lagrange equations are those of the connection-free symmetric Lagrangian; $M = \frac{1}{2\sqrt{|g|}}\partial_\mu(\sqrt{|g|}\gamma^\mu)$ is its volume-and-frame term.

- What is frame-independent: Ω_μ itself vanishes in no frame, because its curvature is the Riemann tensor of the metric, which is not zero; and the frame factors enter every derivative term. In this qualified sense [1] and [2] hold; Chapter 8 discusses the frame dependence further.

The rescaling, line by line. Put $\Psi = \sin^{-1/2} z\,\chi$. Take b_4 constant. Only the x_0 term of the Dirac operator acts on the factor $\sin^{-1/2} z$, so by the product rule

$$\gamma^\mu D_\mu \Psi = \sin^{-1/2} z\,\gamma^\mu D_\mu \chi + \tan z\,\gamma^{(x_0)}\big(\partial_0 \sin^{-1/2} z\big)\chi.$$

The derivative is $\partial_0 \sin^{-1/2} z = -\frac{1}{2}\sin^{-3/2} z\cos z \cdot 6H$ (the power rule and the chain rule with $dz/dx_0 = 6H$), so the last term is

$$\frac{\sin z}{\cos z}\cdot\big(-3H\sin^{-3/2} z\cos z\big)\gamma^{(x_0)}\chi = -3H\sin^{-1/2} z\,\gamma^{(x_0)}\chi.$$

And $\gamma^\mu D_\mu\chi = \gamma^\mu\partial_\mu\chi + 3H\gamma^{(x_0)}\chi$. The two $3H$ terms cancel:

$$\gamma^\mu D_\mu \Psi = \sin^{-1/2} z\,\gamma^\mu\partial_\mu\chi.$$

For $U = 0$ the equation for χ therefore has no spin-connection term. For $U = \frac{\lambda}{2}S^2$ the scalar density is $S[\Psi] = S[\chi]/\sin z$ (the factor is real), and the term becomes the x_0-dependent coupling $\lambda S[\chi]/\sin z$ in $\gamma^\mu\partial_\mu\chi = (m + \lambda S[\chi]/\sin z)\chi$.

An exact family of solutions. Look for solutions that depend only on the time x_4 and the hidden direction x_0, of the form $\Psi = \sin^\alpha z\,P(x_4)\,\chi_0$ with a constant column χ_0, a number α and a 16×16 matrix function P. Take $\lambda = 0$. Line by line:

$$\gamma^{(x_4)}\partial_4\Psi + \tan z\,\gamma^{(x_0)}\partial_0\Psi + 3H\gamma^{(x_0)}\Psi = m\Psi$$

(the explicit equation for a constant b_4; the derivatives along the other six directions vanish, which is why the result holds for EVERY a_4);

$$\sin^\alpha z\,\big[\gamma^{(x_4)}P' + (6H\alpha + 3H)\gamma^{(x_0)}P\big]\chi_0 = m\sin^\alpha z\,P\chi_0$$

(because $\tan z\,\partial_0\sin^\alpha z = \frac{\sin z}{\cos z}\,\alpha\sin^{\alpha-1} z\cos z\cdot 6H = 6H\alpha\sin^\alpha z$);

$$P' = -\gamma^{(x_4)}\big(m - 3H(2\alpha+1)\gamma^{(x_0)}\big)P = MP, \qquad M = -m\gamma^{(x_4)} + 3H(2\alpha+1)\gamma^{(x_4)}\gamma^{(x_0)}$$

(divide by $\sin^\alpha z$, require the bracket for every χ_0, and multiply by $-\gamma^{(x_4)}$, using $(\gamma^{(x_4)})^2 = -1$). The matrix M squares to a number: with $\beta = 3H(2\alpha+1)$,

$$M^2 = m^2(\gamma^{(x_4)})^2 - m\beta\big(\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)} + \gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_4)}\big) + \beta^2\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)} = -m^2 + 0 + \beta^2 = k^2$$

with $k^2 = 9H^2(2\alpha+1)^2 - m^2$ (the middle bracket is $-\gamma^{(x_0)} + \gamma^{(x_0)} = 0$, because $\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_4)} = -\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)} = \gamma^{(x_0)}$; the last product is $-\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_0)} = -(-1)(1) = 1$). Therefore

$$P(x_4) = \cosh(kx_4) + \frac{\sinh(kx_4)}{k} M$$

solves $P' = MP$ with $P(0) = 1$: $P' = k\sinh(kx_4) + \cosh(kx_4)M$ and $MP = \cosh(kx_4)M + \frac{\sinh(kx_4)}{k}k^2$, the same. Moreover $M^TC + CM = 0$ (from $(\gamma^{(x_4)})^TC = -C\gamma^{(x_4)}$ and $\gamma^{(x_4)}C = C\gamma^{(x_4)}$, $\gamma^{(x_0)}C = -C\gamma^{(x_0)}$), which gives $\frac{d}{dx_4}(P^TCP) = P^T(M^TC + CM)P = 0$, so $P^TCP = C$ and the scalar density $S = \sin^{2\alpha} z\, \chi_0^\dagger C\chi_0$ does not change with x_4 (P is real) (PROVED; Notebook 07b, In [21]). Two members, with the illustration values $H = 1$, $m = 2$, $\chi_0 = e_1 + e_5$ (components 1 and 5 equal to 1):

- $\alpha = 0$: $k^2 = 9 - 4 = 5$, and P grows like $e^{\sqrt{5}x_4}$. This is an x_0-independent growing mode, which exists without a boundary condition at $z = \pi/2$ whenever $m^2 < 9H^2$ (the scope record, table below). Its scalar density is $\chi_0^T C\chi_0 = 2C_{1,5} = -2$.
- $\alpha = -\frac{1}{2}$: $2\alpha + 1 = 0$, $M = -m\gamma^{(x_4)}$, $k^2 = -m^2 = -4$, so $\cosh(kx_4) = \cos 2x_4$ and $\sinh(kx_4)/k = \sin(2x_4)/2$: the solution oscillates. This is the rescaling $\Psi = \sin^{-1/2} z\, \chi$ above, in which the $3H$ term disappears. Its scalar density at $z = \pi/4$ is $-2/\sin z = -2\sqrt{2} = -2.828427$.

Notebook 07b evaluates both members at $z = \pi/4$ for x_4 from 0 to 2 and finds $S = -2.000000$ and -2.828427, constant along x_4 (In [22], Figure 07b.5; COMPUTED). Put into the equation WITHOUT the term $3H\gamma^{(x_0)}\Psi$, the growing solution leaves the residual $3H\gamma^{(x_0)}\Psi$, whose largest component is $3H$ times the largest component of Ψ (In [24], Figure 07b.6): the gravitational term is needed for the solution. These solutions are test points; at them the pressures vanish, so they are a weak test of the energy-momentum tensor, whose conservation is proved in general in Chapter 9.

Where the statements of this section are verified (the reports in the folder `Revision/theory/reports`):

statement	status	where it is verified
the gravitational term $3H\gamma^{(x_0)}\Psi$ of the field equation is nonzero for a constant b_4, both statistics	PROVED; Notebook 07b, In [20]	`wolfram-field-theory.json`, checks `nontriviality_1_dirac16complex` and `nontriviality_2_dirac16complex00` (at $b_4 = 1$)
with a changing b_4: $MM = (9H^2 - \frac{9}{16}(b_4'/b_4)^2)1$, so $M\Psi \neq 0$ unless $\lvert b_4'/b_4\rvert = 4H$	PROVED	`python-field-theory-b4.json`, check `gravity_term_square_b4`

statement	status	where it is verified
$\Omega_\mu = 0$ only for $a_4' = 0$ and $H = 0$ (at $b_4 = 1$); $H = 0$ degenerate; curved for every $H > 0$	PROVED	`python-field-theory.json`, check `nontriviality_Omega_zero_iff_flat`; `wolfram-field-theory.json`, checks `Omega_vanishes_iff_a4prime_and_H_vanish`, `degenerate_at_H_0` and `never_flat_for_H_positive`
the boosted frame removes $\gamma^\mu \Omega_\mu$ (constant b_4)	PROVED (quoted)	`wolfram-scope.json` and `python-scope.json`, check `boosted_frame_gammaOmega_vanishes`
the rescaling $\Psi = \sin^{-1/2} z\, \chi$ removes it; the coupling $\lambda S[\chi]/\sin z$ (constant b_4; with b_4 see Section 6.18)	PROVED here	scope reports, checks `rescaling_removes_the_connection_term` and `rescaled_equation_quadratic_potential`
$\gamma^\mu \Omega_\mu$ does not see the deflation; the equations are those of the connection-free Lagrangian	PROVED (quoted)	scope reports, checks `gammaOmega_blind_to_the_deflation` and `connection_free_lagrangian_same_equations`
Ω_μ vanishes in no frame	PROVED (quoted)	`wolfram-scope.json`, check `boosted_frame_curvature_nonzero`; `wolfram-field-theory.json`, check `spin_curvature_equals_Riemann`
the exact family of solutions, $M^2 = k^2$, $M^T C + C M = 0$	PROVED; In [21]	`python-field-theory.json`, check `exact_solution_family_x4_x0`
growing x_0-independent modes without a boundary condition at $z = \pi/2$ when $m^2 < 9H^2$	PROVED (quoted); In [21]	`python-scope.json`, check `good_sector_x0_independent_modes_without_boundary_condition`
self-consistency (ii): the Heisenberg equation reproduces the evolution form	PROVED (quoted; derived in Section 10.12)	`wolfram-field-theory.json`, check `Heisenberg_equation_reproduces_field_equation`; `python-field-theory.json`, check `hamiltonian_form_and_heisenberg_equation`

statement	status	where it is verified
self-consistency (iii): the Lichnerowicz identity, $R = 6((a_4')^2 - 7H^2)$ (at $b_4 = 1$)	PROVED in the record at $b_4 = 1$, quoted, NOT derived in this book (R derived in Sections 3.25 and 8.7)	`wolfram-field-theory.json`, check `Lichnerowicz_identity_G`; `python-field-theory.json`, checks `lichnerowicz_identity_on_fields` and `lichnerowicz_contraction`
self-consistency (iv): the local law $\nabla_\mu J^\mu = 0$	PROVED (derived in Section 21.17)	`wolfram-field-theory.json`, check `current_conservation_identity_G`; `python-field-theory.json`, check `grassman_current_conservation`
self-consistency (v): the Noether identities; T symmetric and conserved on shell	PROVED in the record, quoted (used in Chapter 9)	`wolfram-field-theory.json`, checks `Noether_identity_diffeomorphisms_G`, `Noether_identity_local_Lorentz_G` and `conservation_on_shell_general`
self-consistency (vi): T and J conserved at the exact solutions	PROVED (quoted)	`wolfram-field-theory.json`, check `exact_solution_x4_x0_G`

7.24 Example: Notebook 07b derives the field equations in the author's metric

Notebook 07b derives everything of Sections 7.18 to 7.23 exactly, with the computer doing every step of algebra that a person would do by hand. It reads the author's gammas and C from the Revision record; builds the metric from its frame and checks $\sqrt{|g|} = \cos z$; computes the 25 Christoffel symbols, the 12 components of the spin connection and $\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ direction by direction; writes the Lagrangian of both fields in an exact jet algebra (392 and 408 monomials); proves its reality, the total divergence and the absence of the connection; derives the 16 Euler-Lagrange equations of both statistics and compares them term by term with the formula record; checks the block and evolution forms, non-triviality, and the exact family of solutions. Every result that the Revision record also contains is checked against it. It draws six figures and ends with the line ALL 47 CHECKS PASSED (notebook 07b).

7.25 How to run Notebook 07b

Step 1. What this notebook does and what it needs.

Notebook 07b (The Euler-Lagrange equations in the author's metric) is the file `Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb` of the repository Dirac_claude. It builds the author's primordial metric, its diagonal frame, Christoffel symbols and canonical spin connection from the Revision gamma matrices with exact computer algebra, shows that the spin-connection term of the field equation is 3 H gamma^(x0) because the time terms of the three inflating and the three deflating directions cancel, writes the Lagrangian of both fields in an exact jet algebra

(392 monomials for dirac16complex, 408 for dirac16complex00), proves that it is real, derives the 16 Euler-Lagrange equations for both statistics, compares them term by term with the Revision formula record, checks their chiral block form and their evolution form, verifies an exact family of solutions, and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 07b_euler_lagrange_metric.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 07b_euler_lagrange_metric.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 07b_euler_lagrange_metric.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/07b.captions.json`
- `Revision/textbook/figures/07b_1_scale_factors.png`
- `Revision/textbook/figures/07b_2_gamma_omega_per_direction.png`
- `Revision/textbook/figures/07b_3_lagrangian_monomials.png`
- `Revision/textbook/figures/07b_4_coupling_map.png`
- `Revision/textbook/figures/07b_5_exact_solutions.png`
- `Revision/textbook/figures/07b_6_residuals.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 47 CHECKS PASSED (notebook 07b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 07b_euler_lagrange_metric.ipynb
python -m nbconvert --execute --inplace 07b_euler_lagrange_metric.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/theory/reports/..." or a check that names Revision/theory/field-theory.json fails: the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory Revision/algebra
```

7.26 Notebook 07b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the

cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook derives the field equations of the two fields of the book in the
author's primordial gravitational field, exactly, with the computer doing every
step of algebra that a person would do by hand. It

- reads the author's gamma matrices and the matrix $C$ from the Revision record and
  checks the Clifford relation;
- builds the author's metric from its diagonal frame (vielbein), checks that the
  volume factor is $\sqrt{|g|} = \cos z$, and plots the scale factors of 3-space
  and of the three deflating extra times;
- computes the 25 nonzero Christoffel symbols, the 12 nonzero components of the
  canonical spin connection and the spinor connection $\Omega_\mu$;
- shows that the spin-connection term of the field equation is
  $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$: the time terms of the three inflating
  and the three deflating directions cancel, the hidden-direction terms add up;
- writes the Lagrangian of both fields in an exact "jet" algebra: 392 monomials
  for dirac16complex (anticommuting) and 408 for dirac16complex00 (commuting), the
  difference being 16 squares $(\bar\Psi_A\Psi_B)^2$ that vanish for Grassmann
  numbers;
- proves that the Lagrangian is real, that it differs from the unsymmetrised form
  by a total divergence, and that the spin connection drops out of it in this
  metric;
- derives the 16 Euler-Lagrange equations for both statistics and shows that they
  are the same: $\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi$;
- writes them out in the author's metric and compares every one of the 16 component
  equations term by term with the Revision formula record
  Revision/theory/field-theory.json;
- checks two more forms of the same equations: the chiral block form (each
  equation relates the two halves of $\Psi$) and the evolution form, which gives
  the time derivative $\partial_4\Psi$ from the field at one time;
- verifies an exact family of solutions and shows numerically that it fails when
  the spin-connection term $3H\gamma^{(x_0)}$ is left out.

Every result that the Revision record also contains is checked against it; such a
PASS line is followed by a line "reproduces <record file>, check <name>".
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 07b (The Euler-Lagrange equations in the author's metric) is the file
`Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb` of the repository
Dirac_claude. It builds the author's primordial metric, its diagonal frame, Christoffel
symbols and canonical spin connection from the Revision gamma matrices with exact
computer algebra, shows that the spin-connection term of the field equation is 3 H
```

gamma^(x0) because the time terms of the three inflating and the three deflating directions cancel, writes the Lagrangian of both fields in an exact jet algebra (392 monomials for dirac16complex, 408 for dirac16complex00), proves that it is real, derives the 16 Euler-Lagrange equations for both statistics, compares them term by term with the Revision formula record, checks their chiral block form and their evolution form, verifies an exact family of solutions, and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of

‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 07b_euler_lagrange_metric.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 30 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.

**Step 6. Or run the notebook without a browser (headless).**

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 07b_euler_lagrange_metric.ipynb

This runs every cell from the top to the bottom and saves the results into the notebook
```

file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 07b_euler_lagrange_metric.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/07b.captions.json‘
- ‘Revision/textbook/figures/07b_1_scale_factors.png‘
- ‘Revision/textbook/figures/07b_2_gamma_omega_per_direction.png‘
- ‘Revision/textbook/figures/07b_3_lagrangian_monomials.png‘
- ‘Revision/textbook/figures/07b_4_coupling_map.png‘
- ‘Revision/textbook/figures/07b_5_exact_solutions.png‘
- ‘Revision/textbook/figures/07b_6_residuals.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 6 figure files of this notebook exist
ALL 47 CHECKS PASSED (notebook 07b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with

  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 07b_euler_lagrange_metric.ipynb
python -m nbconvert --execute --inplace 07b_euler_lagrange_metric.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/theory/reports/..." or a check that names Revision/theory/field-theory.json fails: the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory Revision/algebra
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 07b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 07b (The Euler-Lagrange equations in the author's metric) is the file
# Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb of the repository
# Dirac_claude. It builds the author's primordial metric, its diagonal frame, Christoffel
# symbols and canonical spin connection from the Revision gamma matrices with exact
# computer algebra, shows that the spin-connection term of the field equation is 3 H
# gamma^(x0) because the time terms of the three inflating and the three deflating
# directions cancel, writes the Lagrangian of both fields in an exact jet algebra (392
# monomials for dirac16complex, 408 for dirac16complex00), proves that it is real,
# derives the 16 Euler-Lagrange equations for both statistics, compares them term by term
# with the Revision formula record, checks their chiral block form and their evolution
```

```
# form, verifies an exact family of solutions, and draws six teaching plots. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
```

```
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
```

```
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 07b_euler_lagrange_metric.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 07b_euler_lagrange_metric.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 07b_euler_lagrange_metric.ipynb (in the same folder, without running the cells again)
# to read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/07b.captions.json
# - Revision/textbook/figures/07b_1_scale_factors.png
# - Revision/textbook/figures/07b_2_gamma_omega_per_direction.png
# - Revision/textbook/figures/07b_3_lagrangian_monomials.png
# - Revision/textbook/figures/07b_4_coupling_map.png
```

```
# - Revision/textbook/figures/07b_5_exact_solutions.png
# - Revision/textbook/figures/07b_6_residuals.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 6 figure files of this notebook exist
#     ALL 47 CHECKS PASSED (notebook 07b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/07b_euler_lagrange_metric.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 07b_euler_lagrange_metric.ipynb
#     python -m nbconvert --execute --inplace 07b_euler_lagrange_metric.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
```

```
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "ValueError: ... is not a passing check of Revision/theory/reports/..." or a check
#   that names Revision/theory/field-theory.json fails: the Revision record files of your
#   copy of the repository differ from the committed ones; restore them with the command
#   below (run in the repository folder) and run the notebook again.
#     git restore Revision/theory Revision/algebra
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "07b"  # this notebook: chapter 07, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative
```

```
def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
```

```
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 07b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$, named as the author names them: $x_1, x_2,
  x_3$ ordinary 3-space; $x_4$ the time; $x_5, x_6, x_7$ the three **extra
  times** (time-like directions that shrink, "deflate", as $a_4$ grows); $x_0$ the
  **hidden** space direction. In the program they are list positions 0 to 7.
- **Metric** $g_{\mu\nu}$: the table that gives lengths and time intervals,
  $ds^2 = \sum g_{\mu\nu} dx_\mu dx_\nu$. The author's metric is diagonal. Its
  **signature** (4,4): four space-like directions ($x_0, x_1, x_2, x_3$, positive
  entries) and four time-like ones ($x_4, \dots, x_7$, negative entries).
- $z = 6Hx_0$, with the author's constant $H > 0$ and $0 < z < \pi/2$; $a_4(x_4)$ is
  the free function of time in the metric; $a_4' = da_4/dx_4$.
- **Scale factor**: the number by which a coordinate distance is multiplied to give
  a real (proper) distance: $e^{a_4}\sin^{1/6}z$ for 3-space, $e^{-a_4}\sin^{1/6}z$
  for the extra times. When $a_4$ grows, 3-space inflates and the extra times
  deflate.
- **Vielbein (frame)** $e^a{}_\mu = f_a\delta^a_\mu$: a set of eight local unit
  directions; here simply the square roots $f_a = \sqrt{|g_{aa}|}$.
- **Frame gamma matrices** $\gamma^{(a)}$: the author's eight real $16\times16$
  matrices with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}$,
  $\eta = \mathrm{diag}(1,1,1,1,-1,-1,-1,-1)$ (in the order $x_0, \dots, x_7$).
  **Coordinate gammas** $\gamma^\mu =
```

```
  \gamma^{(\mu)}/f_\mu$.
- **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$: the numbers that say how the
  coordinate directions turn from point to point.
- **Canonical spin connection** $\omega_{\mu ab}$ and **spinor connection**
  $\Omega_\mu = \frac12\omega_{\mu ab}S^{ab}$, $S^{ab} = \frac14[\gamma^{(a)},
  \gamma^{(b)}]$: they say how the frame turns, and how a spinor must be turned
  with it. **Covariant derivative** $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$.
- **Lagrangian density** $L$: the function whose integral (the action) must not
  change to first order when the field is varied; that requirement is the
  **Euler-Lagrange equation**, the field equation.
- **Statistics**: whether the components commute (dirac16complex00) or
  anticommute (dirac16complex, Grassmann numbers).
- **Jet**: the value of a field component and its derivatives at one point,
  treated as independent symbols ($\psi_A$, $\partial_\mu\psi_A$, ...). The
  **total derivative** $d/dx_\mu$ acts on jets by raising the derivative order.
- **Coefficient ring**: the symbols $E = e^{a_4}$, $s = \sin^{1/6}z$, $c = \cos z$,
  $A_1 = a_4'$ and the second and third derivatives $A_2 = a_4^{\prime\prime}$,
  $A_3 = a_4^{\prime\prime\prime}$, together with $H$, $m$, $\lambda$, in which
  every coefficient is written; with the rule $c^2 = 1 - s^{12}$ it allows an
  exact test whether an expression is zero.
- **Monomial**: a product of jets with a coefficient, such as
  $E\,s\,\chi_3\,\partial_1\psi_7$.
- **On shell**: on solutions of the field equation.
```

Text cell:

```
## 4. The physical and mathematical situation

**The author's metric** (Revision/SPEC.md, section 1), $z = 6Hx_0 \in (0,
\pi/2)$:

$$ds^2 = e^{2a_4}\sin^{1/3}z\,(dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2
- e^{-2a_4}\sin^{1/3}z\,(dx_5^2 + dx_6^2 + dx_7^2) + \cot^2 z\, dx_0^2 .$$

3-space ($x_1, x_2, x_3$) has the scale factor $e^{a_4}\sin^{1/6}z$, the three
extra times ($x_5, x_6, x_7$) the scale factor $e^{-a_4}\sin^{1/6}z$: when $a_4$
increases, 3-space inflates and the extra times DEFLATE exponentially. Every
equation of this notebook holds for every function $a_4(x_4)$. Only the plots use a
definite history, the linear member $a_4 = AHx_4$ with $A = 1$ and $H = 1$; this
history is prescribed (a choice made for the picture, deflation being the choice of
sign $A > 0$), not a result.

**The two fields** have the same Lagrangian (Revision/SPEC.md, section 3):

$$L = \sqrt{|g|}\,\Big[\tfrac12\big(\bar\Psi\gamma^\mu D_\mu\Psi -
(D_\mu\bar\Psi)\gamma^\mu\Psi\big) - mS - \tfrac{\lambda}{2}S^2\Big],\quad
\bar\Psi = \Psi^\dagger C,\quad S = \bar\Psi\Psi,\quad D_\mu\bar\Psi =
\partial_\mu\bar\Psi - \bar\Psi\Omega_\mu .$$

For dirac16complex the 16 components are complex Grassmann numbers, for
dirac16complex00 ordinary complex numbers.

**What we expect.** Varying $\Psi^\dagger$ should give the field equation
```

```
$\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi$ for both statistics, and in the
author's metric

$$e^{-a_4}\sin^{-1/6}z\sum_{i=1}^{3}\gamma^{(x_i)}\partial_i\Psi
+ \gamma^{(x_4)}\partial_4\Psi + e^{a_4}\sin^{-1/6}z\sum_{t=5}^{7}
\gamma^{(x_t)}\partial_t\Psi + \tan z\,\gamma^{(x_0)}\partial_0\Psi
+ 3H\gamma^{(x_0)}\Psi = (m + \lambda S)\Psi .$$

**How the computer proves it.** Every coefficient is written with the symbols of
the coefficient ring; derivatives along $x_4$ and $x_0$ follow fixed rules
(section 6); and a coefficient is exactly zero when its numerator vanishes after
$c^2$ is replaced by $1 - s^{12}$. The field components and their derivatives at a
point are the generators of a jet algebra that can be commuting or anticommuting
(section 12). Nothing is approximated, except in the two numerical plots at the end.
```

Text cell:

```
## 5. The gamma matrices and $C$ from the Revision record

The next cell imports the packages and wraps the set-up cell's 'check' so that a
PASS line and the line "reproduces ..." that follows it are printed by one single
'print' call (Jupyter sends printed text in pieces; two separate prints can arrive
in two pieces, and the book's tools then do not see that the two lines belong
together). It then reads the eight frame gamma matrices
$\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$ and the matrix $C = \gamma^{(x_0)}
\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ from Revision/algebra/gammas.json as
exact sympy matrices, builds the 28 spin generators $S^{ab} = \frac14(\gamma^{(a)}
\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$, and defines a helper that looks up a
check of a Revision report and makes sure that its verdict is PASS. It then checks
the Clifford relation for all 64 pairs and the properties of $C$.
```

In [2]:

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import re  # regular expressions: used to read the Wolfram formula record

import numpy as np  # numbers for the plots
import sympy as sp  # exact algebra

check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece
```

```
GAMMAS = "Revision/algebra/gammas.json"
FORMULAS = "Revision/theory/field-theory.json"  # the formulas at b4 = 1
FORMULAS_B4 = "Revision/theory/field-theory-b4.json"  # the formulas with b4(x4)
PYREP_B4 = "Revision/theory/reports/python-field-theory-b4.json"
PYREP = "Revision/theory/reports/python-field-theory.json"
WLREP = "Revision/theory/reports/wolfram-field-theory.json"
SCOPE = "Revision/theory/reports/python-scope.json"


def revision_check(report, name):
    """The check called name of the Revision report (a JSON file): a dictionary
    with its name, verdict and detail; stops if it is missing or not PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return found[0]


def reproduces(report, name):
    """The text "<report>, check <name>" for a PASS line, after making sure that
    the Revision report contains the check name with the verdict PASS."""
    revision_check(report, name)
    return f"{report}, check {name}"


fixture = json.loads(repository_file(GAMMAS).read_text(encoding="utf-8"))
G = [sp.Matrix(g) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7), integers
C = sp.Matrix(fixture["C"])  # the author's sigma16
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # eta in the order x0 .. x7
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
SAB = [[(G[a] * G[b] - G[b] * G[a]) / 4 for b in range(8)] for a in range(8)]
X0, X4 = 0, 4  # list positions of the hidden direction x0 and of the time x4
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
clifford = all(G[a] * G[b] + G[b] * G[a] == 2 * (ETA[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
check(clifford, "{gamma^a, gamma^b} = 2 eta^ab I16 for all 64 pairs",
      record=reproduces(PYREP, "clifford_relations"))
c_ok = (C == G[0] * G[1] * G[2] * G[3] and C.T == C and C * C == I16
        and all((C * g).T == -(C * g) for g in G))
check(c_ok, "C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3), symmetric, C^2 = 1, "
      "every C gamma^a antisymmetric", record=reproduces(PYREP, "C_properties"))
```

Out [2]:

```
PASS {gamma^a, gamma^b} = 2 eta^ab I16 for all 64 pairs
     reproduces Revision/theory/reports/python-field-theory.json, check
    clifford_relations
PASS C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3), symmetric, C^2 = 1, every C gamma^a
    antisymmetric
     reproduces Revision/theory/reports/python-field-theory.json, check C_properties
```

Text cell:

```
## 6. Exact symbols for the metric functions (the coefficient ring)

Every coefficient will be written with the symbols $E = e^{a_4(x_4)}$, $s =
\sin^{1/6}z$, $c = \cos z$, $A_1 = a_4'$, $A_2 = a_4^{\prime\prime}$, $A_3 =
a_4^{\prime\prime\prime}$, the second function of the metric $b_4(x_4) > 0$ and its
derivatives (the symbols `b4`, `b4p`, `b4pp`, `b4ppp`), $H$, $m$ and $\lambda$.
Their derivatives follow from the chain rule, line by line:

- $\partial_4 E = e^{a_4}a_4' = E A_1$, $\partial_4 A_1 = A_2$, $\partial_4 A_2 =
  A_3$ (the derivative of $e^{a_4(x_4)}$ is $e^{a_4}$ times the derivative of the
  exponent); $\partial_4 b_4 = b_4'$, $\partial_4 b_4' = b_4''$, $\partial_4 b_4''
  = b_4^{\prime\prime\prime}$;
- $\partial_0 s = \frac16\sin^{-5/6}z\,\cos z \cdot 6H = H c/s^5$ (the power rule,
  then the derivative $\cos z$ of $\sin z$, then $dz/dx_0 = 6H$, and finally
  $\sin^{-5/6}z = s^{-5}$);
- $\partial_0 c = -\sin z \cdot 6H = -6Hs^6$ (because $\sin z = s^6$);
- no symbol depends on $x_1, x_2, x_3, x_5, x_6, x_7$.

The function `cd(expression, mu)` applies these rules (the product and chain rules
are done by sympy's `diff`). Since $\cos^2z + \sin^2z = 1$, the symbols obey
$c^2 = 1 - s^{12}$; the function `is_zero` brings an expression to one fraction and
replaces $c^2$ by $1 - s^{12}$ in its numerator (by polynomial division), which
decides exactly whether it is zero. The function `to_physical` turns ring
expressions back into functions of $x_4$ and $x_0$. The cell checks the derivative
rules against sympy's own derivatives of the physical functions.
```

In [3]:

```
E, s, c = sp.symbols("E s c", positive=True)  # e^a4, sin(z)^(1/6), cos z
A1, A2, A3 = sp.symbols("A1 A2 A3", real=True)  # 1st, 2nd, 3rd derivative of a4
b4 = sp.Symbol("b4", positive=True)  # the function b4(x4) > 0 of the metric
b4p, b4pp, b4ppp = sp.symbols("b4p b4pp b4ppp", real=True)  # its 1st to 3rd derivative
AT_B4_1 = {b4: 1, b4p: 0, b4pp: 0, b4ppp: 0}  # the background b4 = 1
H = sp.Symbol("H", positive=True)  # the author's constant
m, lam = sp.symbols("m lambda", real=True)  # the mass and the coupling


def cd(expression, mu):
    """The derivative of a ring expression along coordinate mu (0 .. 7)."""
    expression = sp.sympify(expression)
    if mu == X4:
        return (sp.diff(expression, E) * E * A1 + sp.diff(expression, A1) * A2
                + sp.diff(expression, A2) * A3 + sp.diff(expression, b4) * b4p
                + sp.diff(expression, b4p) * b4pp + sp.diff(expression, b4pp) * b4ppp)
    if mu == X0:
        return (sp.diff(expression, s) * H * c / s**5
                + sp.diff(expression, c) * (-6 * H * s**6))
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7


def is_zero(expression):
    """Exact test: the numerator vanishes after c^2 -> 1 - s^12."""
    numerator, _ = sp.fraction(sp.together(sp.sympify(expression)))
```

```
    numerator = sp.expand(numerator)
    if numerator == 0:
        return True
    remainder = sp.rem(sp.Poly(numerator, c), sp.Poly(c**2 - 1 + s**12, c))
    return sp.expand(remainder.as_expr()) == 0


x4, x0 = sp.symbols("x4 x0", real=True)
a4 = sp.Function("a4")(x4)  # the free function of the metric
b4f = sp.Function("b4")(x4)  # the second function of the metric
z = 6 * H * x0


def to_physical(expression):
    """Ring symbols -> the functions they stand for."""
    return sp.sympify(expression).subs(
        {E: sp.exp(a4), s: sp.sin(z) ** sp.Rational(1, 6), c: sp.cos(z),
         A1: a4.diff(x4), A2: a4.diff(x4, 2), A3: a4.diff(x4, 3),
         b4: b4f, b4p: b4f.diff(x4), b4pp: b4f.diff(x4, 2), b4ppp: b4f.diff(x4, 3)})


rules_ok = all(
    sp.simplify(to_physical(cd(v, mu)) - sp.diff(to_physical(v), variable)) == 0
    for v in (E, s, c, A1, A2, b4, b4p, b4pp)
    for mu, variable in ((X4, x4), (X0, x0)))
check(rules_ok, "the derivative rules of the ring agree with sympy's derivatives")
check(is_zero(c**2 + s**12 - 1) and not is_zero(c) and not is_zero(s - c),
      "the exact zero test: c^2 + s^12 - 1 is zero, c and s - c are not")
```

Out [3]:

```
PASS the derivative rules of the ring agree with sympy's derivatives
PASS the exact zero test: c^2 + s^12 - 1 is zero, c and s - c are not
```

Text cell:

```
## 7. The metric, its frame and the volume factor

The diagonal frame has $f_0 = c/s^6 = \cot z$, $f_1 = f_2 = f_3 = Es$, $f_4 = 1$,
$f_5 = f_6 = f_7 = s\sqrt{b_4}/E$, and the metric is $g_{aa} = \eta_{aa}f_a^2$. The
volume factor is the product of the $f_a$, worked out line by line:

$$\sqrt{|g|} = f_0 \cdot f_1 f_2 f_3 \cdot f_4 \cdot f_5 f_6 f_7
= \frac{c}{s^6} \cdot E^3s^3 \cdot 1 \cdot \frac{s^3b_4^{3/2}}{E^3} = b_4^{3/2}c
= b_4^{3/2}\cos z .$$

The first equality: for a diagonal metric $|\det g|$ is the product of the
$f_a^2$; the second inserts the $f_a$; the third cancels $E^3$ against $E^{-3}$
and $s^6$ against $s^{-6}$. The inflation $E^3$ of 3-space and the deflation
$E^{-3}$ of the extra times compensate: the 7-volume depends on the time only
through $b_4^{3/2}$, and not at all on the backgrounds $b_4 = 1$. The next cell
checks that the frame gives the author's metric entry by entry and that
$\sqrt{|g|} = b_4^{3/2}\cos z$, which is $\cos z$ at $b_4 = 1$.
```

In [4]:

```
f = [c / s**6] + [E * s] * 3 + [sp.Integer(1)] + [s * sp.sqrt(b4) / E] * 3  # frame
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal metric entries
sqrt_g = sp.prod(f)  # the volume factor
third = sp.sin(z) ** sp.Rational(1, 3)
author = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * third] * 3 + [-1]
          + [-sp.exp(-2 * a4) * b4f * third] * 3)  # the author's metric with b4
check(all(sp.simplify(to_physical(g[a]) - author[a]) == 0 for a in range(8)),
      "the diagonal frame reproduces the author's metric with b4 entry by entry",
      record=reproduces(PYREP, "metric_from_vielbein_equals_SPEC") + " (at b4 = 1)")
say(f"sqrt|g| = {sqrt_g} (ring) = {to_physical(sqrt_g)}")
check(sqrt_g == b4 ** sp.Rational(3, 2) * c and sqrt_g.subs(AT_B4_1) == c,
      "sqrt|g| = b4^(3/2) cos z; cos z, independent of x4, at b4 = 1",
      record=reproduces(WLREP, "sqrt_det_g_is_cos_z") + " (at b4 = 1)")
```

Out [4]:

```
PASS the diagonal frame reproduces the author's metric with b4 entry by entry
     reproduces Revision/theory/reports/python-field-theory.json, check
    metric_from_vielbein_equals_SPEC (at b4 = 1)
sqrt|g| = b4**(3/2)*c (ring) = b4(x4)**(3/2)*cos(6*H*x0)
PASS sqrt|g| = b4^(3/2) cos z; cos z, independent of x4, at b4 = 1
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    sqrt_det_g_is_cos_z (at b4 = 1)
```

Text cell:

```
The formula record Revision/theory/field-theory.json stores the formulas of the
Wolfram verifier as text in the Wolfram Language (for example
‘E^a4[x4]*Sin[6*H*x0]^(1/6)‘). The next cell defines a small translator: it
rewrites such a text into sympy syntax (square brackets to round ones, ‘^‘ to
‘**‘, ‘Sin‘ to ‘sin‘, ‘a4[x4]‘, ‘b4[x4]‘ and their derivatives to symbols), lets
sympy read it, and then puts it into the ring: $a_4 \to \ln E$ (so $e^{a_4} \to
E$), $b_4$ and its derivatives to ‘b4‘, ‘b4p‘, ‘b4pp‘, $\sin z \to s^6$, $\cos z
\to c$, $\tan z \to s^6/c$, $\cot z \to c/s^6$, $\sec z \to 1/c$, $\csc z \to
1/s^6$ and $\csc 2z \to 1/(2s^6c)$ (from $\sin 2z = 2\sin z\cos z$). Then a record
formula and our own result can be compared with ‘is_zero‘. Two formula records are
read: ‘record‘ (the metric at $b_4 = 1$, Revision/theory/field-theory.json) and
‘record_b4‘ (the metric with $b_4$, Revision/theory/field-theory-b4.json). A result
of this notebook is compared with the first after ‘.subs(AT_B4_1)‘, which puts
$b_4 = 1$ and $b_4' = b_4'' = b_4^{\prime\prime\prime} = 0$. The cell compares the
frame and the
volume factor with the record of $b_4 = 1$.
```

In [5]:

```
record = {item["key"]: item["wl"] for item in
          json.loads(repository_file(FORMULAS).read_text(encoding="utf-8"))["formulas"]}
record_b4 = {item["key"]: item["wl"] for item in json.loads(
```

```
    repository_file(FORMULAS_B4).read_text(encoding="utf-8"))["formulas"]}
a4x, A1w = sp.symbols("a4x A1w", real=True)  # a4(x4) and a4'(x4) in the record
b4x = sp.Symbol("b4x", positive=True)  # b4(x4) in the record
B1w, B2w = sp.symbols("B1w B2w", real=True)  # b4'(x4) and b4''(x4) in the record


def from_wolfram(text):
    """Read a Wolfram InputForm text of the formula record into sympy."""
    t = text.replace("Derivative[1][a4][x4]", "A1w").replace("a4[x4]", "a4x")
    t = t.replace("Derivative[1][b4][x4]", "B1w").replace("Derivative[2][b4][x4]", "B2w")
    t = t.replace("b4[x4]", "b4x")
    t = re.sub(r'dd\["x(\d)",\s*Psi\[(\d+)\]\]', r"d\1_\2", t)  # d_a Psi_B
    t = re.sub(r"Psi\[(\d+)\]", r"P\1", t)  # Psi_B
    for name in ("Sin", "Cos", "Tan", "Cot", "Sec", "Csc", "Sqrt"):
        t = t.replace(name + "[", name.lower() + "(")
    t = t.replace("[", "(").replace("]", ")").replace("{", "[").replace("}", "]")
    t = t.replace("^", "**")
    names = {"E": sp.E, "H": H, "x0": x0, "a4x": a4x, "A1w": A1w, "b4x": b4x,
             "B1w": B1w, "B2w": B2w}
    for symbol in set(re.findall(r"\b(d\d_\d+|P\d+)\b", t)):
        names[symbol] = sp.Symbol(symbol)
    return sp.sympify(t, locals=names)


RING = {sp.sin(z): s**6, sp.cos(z): c, sp.tan(z): s**6 / c, sp.cot(z): c / s**6,
        sp.sec(z): 1 / c, sp.csc(z): 1 / s**6, sp.csc(2 * z): 1 / (2 * s**6 * c)}


def to_ring(expression):
    """A record formula in the ring symbols E, s, c, A1, b4, b4p, b4pp."""
    return (sp.sympify(expression).subs(a4x, sp.log(E)).subs(A1w, A1)
            .subs({b4x: b4, B1w: b4p, B2w: b4pp}).subs(RING))


frame_record = from_wolfram(record["vielbein_diagonal"])
say(f"record vielbein_diagonal in the ring: {[to_ring(v) for v in frame_record]}")
check(all(is_zero(to_ring(v) - f[a].subs(AT_B4_1)) for a, v in enumerate(frame_record))
      and is_zero(to_ring(from_wolfram(record["sqrt_det_g"])) - sqrt_g.subs(AT_B4_1)),
      "at b4 = 1 the frame factors and sqrt|g| equal the formula record",
      record=f"{FORMULAS}, formulas vielbein_diagonal and sqrt_det_g")
```

Out [5]:

```
record vielbein_diagonal in the ring: [c/s**6, E*s, E*s, E*s, 1, s/E, s/E, s/E]
PASS at b4 = 1 the frame factors and sqrt|g| equal the formula record
     reproduces Revision/theory/field-theory.json, formulas vielbein_diagonal and
    sqrt_det_g
```

Text cell:

```
Figure 1 shows what the metric does, for the illustration $H = 1$, the
deflating history $a_4 = Ax_4H$ with $A = 1$ and the background $b_4 = 1$. Left, at $z
```

```
= \pi/4$: the scale
factor of 3-space grows like $e^{x_4}$, that of the extra times shrinks like
$e^{-x_4}$, and their product is constant. Right, at $x_4 = 0$: the dependence on
the hidden direction, $\sin^{1/6}z$ (all six scale factors), $\cot z$ (the frame
factor of $x_0$) and the volume factor $\cos z$.
```

In [6]:

```
times = np.linspace(0.0, 3.0, 301)  # x4 from 0 to 3 (H = 1, A = 1: a4 = x4)
warp = np.sin(np.pi / 4) ** (1 / 6)  # sin^(1/6) z at z = pi/4
angles = np.linspace(0.01, np.pi / 2 - 0.01, 300)  # z inside (0, pi/2)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
left.semilogy(times, np.exp(times) * warp, label="3-space $e^{a_4}\\sin^{1/6}z$")
left.semilogy(times, np.exp(-times) * warp, "--",
              label="extra times $e^{-a_4}\\sin^{1/6}z$")
left.semilogy(times, np.exp(times) * np.exp(-times) * warp**2, ":", color="black",
              label="their product")
left.set_xlabel("time $x_4$ (with $a_4 = x_4$)")
left.set_ylabel("scale factor")
left.set_title("Inflation and deflation at $z = \\pi/4$")
left.legend(fontsize=8)
right.plot(angles, np.sin(angles) ** (1 / 6), label="$\\sin^{1/6}z$")
right.plot(angles, np.cos(angles), "--", label="$\\sqrt{|g|} = \\cos z$")
right.plot(angles, 1 / np.tan(angles), ":", label="$f_0 = \\cot z$")
right.set_ylim(0, 3)
right.set_xlabel("$z = 6Hx_0$")
right.set_ylabel("value")
right.set_title("Dependence on the hidden direction")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "scale_factors",
            "The author's metric along the deflating history $a_4 = AHx_4$ "
            "with the illustration values $A = 1$, $H = 1$ on the background "
            "$b_4 = 1$ (a prescribed history, not a result). Left, at $z = \\pi/4$, "
            "against the time "
            "$x_4$ (logarithmic vertical axis, pure numbers): the scale factor "
            "of 3-space $e^{a_4}\\sin^{1/6}z$ grows, that of the three extra "
            "times $e^{-a_4}\\sin^{1/6}z$ shrinks exponentially, and their "
            "product stays constant. Right, at $x_4 = 0$, against $z = 6Hx_0$ "
            "from 0 to $\\pi/2$: the common warp $\\sin^{1/6}z$, the volume "
            "factor $\\sqrt{|g|} = \\cos z$ and the frame factor $\\cot z$ of the "
            "hidden direction.")
```

Out [6]:

```
Figure 07b.1 saved as Revision/textbook/figures/07b_1_scale_factors.png
```

Text cell:

```
## 8. The Christoffel symbols
```

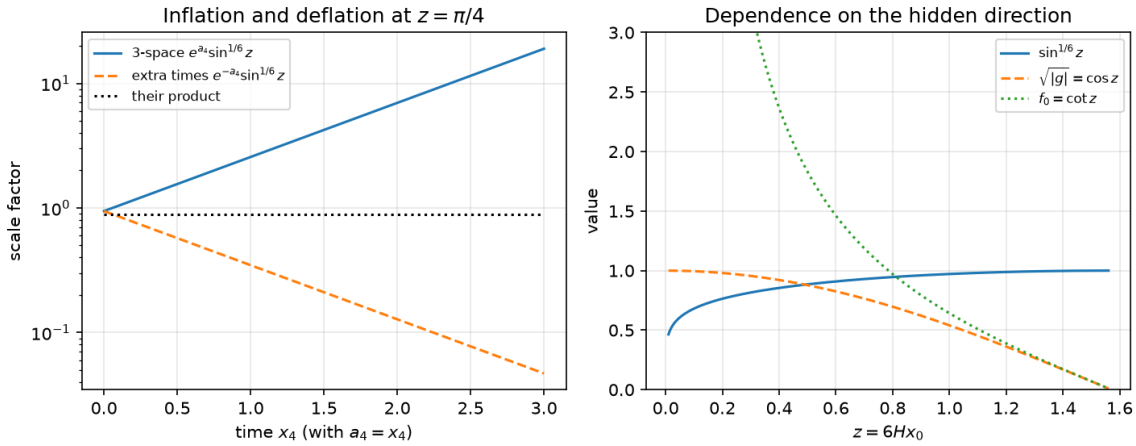

Figure 219: The author's metric along the deflating history $a_4 = AHx_4$ with the illustration values $A = 1$, $H = 1$ on the background $b_4 = 1$ (a prescribed history, not a result). Left, at $z = \pi/4$, against the time x_4 (logarithmic vertical axis, pure numbers): the scale factor of 3-space $e^{a_4} \sin^{1/6} z$ grows, that of the three extra times $e^{-a_4} \sin^{1/6} z$ shrinks exponentially, and their product stays constant. Right, at $x_4 = 0$, against $z = 6Hx_0$ from 0 to $\pi/2$: the common warp $\sin^{1/6} z$, the volume factor $\sqrt{|g|} = \cos z$ and the frame factor $\cot z$ of the hidden direction. (Notebook 07b, figure 1.)

```
For a metric that is diagonal, the general formula $\Gamma^\lambda{}_{\mu\nu} =
\frac12 g^{\lambda\rho}(\partial_\mu g_{\rho\nu} + \partial_\nu g_{\rho\mu} -
\partial_\rho g_{\mu\nu})$ keeps only $\rho = \lambda$, because $g^{\lambda\rho} =
0$ otherwise; then $\partial_\mu g_{\lambda\nu}$ is nonzero only for $\nu =
\lambda$, $\partial_\nu g_{\lambda\mu}$ only for $\mu = \lambda$, and
$\partial_\lambda g_{\mu\nu}$ only for $\mu = \nu$. So

$$\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}\big(\delta_{\lambda\nu}
\partial_\mu g_{\lambda\lambda} + \delta_{\lambda\mu}\partial_\nu g_{\lambda\lambda}
- \delta_{\mu\nu}\partial_\lambda g_{\mu\mu}\big),$$

which the next cell evaluates for all $8^3$ index triples. It counts the
independent nonzero symbols ($\mu \le \nu$), compares all of them with both formula
records, and prints four of them in the ring: $\Gamma^{x_1}{}_{x_1x_4} = a_4'$
(3-space stretches with time), $\Gamma^{x_5}{}_{x_4x_5} = -a_4' + b_4'/(2b_4)$ (the
extra times shrink; $-a_4'$ at $b_4 = 1$), $\Gamma^{x_1}{}_{x_0x_1} = H\cot z$ and
$\Gamma^{x_0}{}_{x_0x_0}$.
```

In [7]:

```
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]  # Gam[l][mu][nu]
for l in range(8):
    for mu in range(8):
        for nu in range(8):
            value = 0
            if l == nu:
                value += cd(g[l], mu)
            if l == mu:
                value += cd(g[l], nu)
```

```
            if mu == nu:
                value -= cd(g[mu], l)
            if value != 0:
                Gam[l][mu][nu] = sp.expand(value / (2 * g[l]))
nonzero = [(l, mu, nu) for l in range(8) for mu in range(8) for nu in range(mu, 8)
           if not is_zero(Gam[l][mu][nu])]
report("independent nonzero Christoffel symbols", len(nonzero))
for l, mu, nu in [(1, 1, 4), (5, 4, 5), (1, 0, 1), (0, 0, 0)]:
    say(f"Gamma^{NAMES[l]}_{NAMES[mu]}{NAMES[nu]} = {sp.factor(Gam[l][mu][nu])}")
recorded = int(re.search(r"(\d+) independent nonzero",
                         revision_check(PYREP, "christoffel_symmetric_metric_"
                                               "compatible")["detail"]).group(1))
check(len(nonzero) == recorded == 25, "25 independent nonzero Christoffel symbols",
      record=reproduces(PYREP, "christoffel_symmetric_metric_compatible"))
# the records number the directions from 1 in the order x0 .. x7: subtract 1
listed = from_wolfram(record_b4["christoffel_nonzero"])
listed_1 = from_wolfram(record["christoffel_nonzero"])
check(len(listed) == len(listed_1) == 25
      and all(is_zero(to_ring(v) - Gam[l - 1][mu - 1][nu - 1])
              for l, mu, nu, v in listed)
      and all(is_zero(to_ring(v) - Gam[l - 1][mu - 1][nu - 1].subs(AT_B4_1))
              for l, mu, nu, v in listed_1),
      "every Christoffel symbol equals the formula record with b4, and at b4 = 1 "
      "the record of b4 = 1",
      record=f"{FORMULAS_B4} and {FORMULAS}, formula christoffel_nonzero")
```

Out [7]:

```
RESULT independent nonzero Christoffel symbols = 25
Gamma^x1_x1x4 = A1
Gamma^x5_x4x5 = -(2*A1*b4 - b4p)/(2*b4)
Gamma^x1_x0x1 = H*c/s**6
Gamma^x0_x0x0 = -6*H*(c**2 + s**12)/(c*s**6)
PASS 25 independent nonzero Christoffel symbols
     reproduces Revision/theory/reports/python-field-theory.json, check
    christoffel_symmetric_metric_compatible
PASS every Christoffel symbol equals the formula record with b4, and at b4 = 1 the record
    of b4 = 1
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula christoffel_nonzero
```

Text cell:

```
## 9. The canonical spin connection

The canonical spin connection is fixed by the **vielbein postulate**
$\partial_\mu e^a{}_\nu - \Gamma^\lambda{}_{\mu\nu}e^a{}_\lambda +
\omega_\mu{}^a{}_b\,e^b{}_\nu = 0$ (the frame is carried along consistently with
the metric). Its solution is $\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu
e_b{}^\nu + \Gamma^\nu{}_{\mu\lambda}e_b{}^\lambda)$ with the inverse frame
$e_b{}^\nu = \delta_b^\nu/f_b$; for our diagonal frame this is
```

```
$$\omega_\mu{}^a{}_b = f_a\Big(\delta_{ab}\,\partial_\mu\frac{1}{f_b} +
\frac{\Gamma^a{}_{\mu b}}{f_b}\Big),\qquad \omega_{\mu ab} =
\eta_{aa}\,\omega_\mu{}^a{}_b .$$

The next cell computes it, checks that $\omega_{\mu ab} = -\omega_{\mu ba}$ and that
the vielbein postulate holds in all 512 components, counts the nonzero components
with $a < b$ (there should be 12), compares them with the formula record and prints
them.
```

In [8]:

```
om = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]  # om[mu][a][b]
for mu in range(8):
    for a in range(8):
        for b in range(8):
            mixed = f[a] * ((cd(1 / f[b], mu) if a == b else 0) + Gam[a][mu][b] / f[b])
            om[mu][a][b] = sp.expand(ETA[a] * mixed)  # lowered with eta_aa
antisymmetric = all(is_zero(om[mu][a][b] + om[mu][b][a])
                    for mu in range(8) for a in range(8) for b in range(8))
postulate = all(is_zero((cd(f[a], mu) if a == nu else 0) - Gam[a][mu][nu] * f[a]
                        + ETA[a] * om[mu][a][nu] * f[nu])
                for mu in range(8) for a in range(8) for nu in range(8))
check(antisymmetric and postulate, "omega_mu ab = -omega_mu ba and the vielbein "
      "postulate holds in all 512 components",
      record=reproduces(PYREP, "vielbein_postulate"))
pairs = [(mu, a, b) for mu in range(8) for a in range(8) for b in range(a + 1, 8)
         if not is_zero(om[mu][a][b])]
for mu, a, b in pairs:
    say(f"omega_{NAMES[mu]} ({NAMES[a]})({NAMES[b]}) = {sp.factor(om[mu][a][b])}")
listed = from_wolfram(record_b4["omega_nonzero"])
listed_1 = from_wolfram(record["omega_nonzero"])
check(len(pairs) == 12 and len(listed) == len(listed_1) == 12
      and all(is_zero(to_ring(v) - om[mu - 1][a - 1][b - 1])
              for mu, a, b, v in listed)
      and all(is_zero(to_ring(v) - om[mu - 1][a - 1][b - 1].subs(AT_B4_1))
              for mu, a, b, v in listed_1),
      "exactly 12 nonzero components, equal to the formula record with b4, and at "
      "b4 = 1 to the record of b4 = 1", record=f"{FORMULAS_B4}, formula "
      "omega_nonzero; " + reproduces(WLREP, "omega_components") + " (at b4 = 1)")
```

Out [8]:

```
PASS omega_mu ab = -omega_mu ba and the vielbein postulate holds in all 512 components
     reproduces Revision/theory/reports/python-field-theory.json, check
    vielbein_postulate
omega_x1 (x0)(x1) = -E*H*s
omega_x1 (x1)(x4) = A1*E*s
omega_x2 (x0)(x2) = -E*H*s
omega_x2 (x2)(x4) = A1*E*s
omega_x3 (x0)(x3) = -E*H*s
omega_x3 (x3)(x4) = A1*E*s
omega_x5 (x0)(x5) = H*sqrt(b4)*s/E
```

```
omega_x5 (x4)(x5) = -s*(2*A1*b4 - b4p)/(2*E*sqrt(b4))
omega_x6 (x0)(x6) = H*sqrt(b4)*s/E
omega_x6 (x4)(x6) = -s*(2*A1*b4 - b4p)/(2*E*sqrt(b4))
omega_x7 (x0)(x7) = H*sqrt(b4)*s/E
omega_x7 (x4)(x7) = -s*(2*A1*b4 - b4p)/(2*E*sqrt(b4))
PASS exactly 12 nonzero components, equal to the formula record with b4, and at b4 = 1 to
    the record of b4 = 1
     reproduces Revision/theory/field-theory-b4.json, formula omega_nonzero;
    Revision/theory/reports/wolfram-field-theory.json, check omega_components (at b4 = 1)
```

Text cell:

```
Each of the 12 components printed above is a rate, $a_4'$, $a_4' - b_4'/(2b_4)$
(the extra times) or $H$, times a factor $\pm Es$ or $\pm s\sqrt{b_4}/E$ that never
vanishes. Six of them are $H$ times such a factor: the connection never vanishes,
because $H = 0$ is not a member of the author's family.
```

Text cell:

```
## 10. The spinor connection and the term $\gamma^\mu\Omega_\mu$

The spinor connection is $\Omega_\mu = \frac12\omega_{\mu ab}S^{ab} =
\sum_{a<b}\omega_{\mu ab}S^{ab}$ (the two halves of the double sum are equal
because both $\omega_{\mu ab}$ and $S^{ab}$ change sign when $a$ and $b$ are
exchanged). The coordinate gammas are $\gamma^\mu = \gamma^{(\mu)}/f_\mu$. The next
cell computes, for every direction $\mu$ separately (no sum), the matrix
$\gamma^\mu\Omega_\mu$ and writes it as $\alpha_\mu\gamma^{(x_4)} +
\beta_\mu\gamma^{(x_0)}$. The two numbers are found with traces, because
$\mathrm{tr}(\gamma^{(x_4)}\gamma^{(x_4)}) = -16$, $\mathrm{tr}(\gamma^{(x_0)}
\gamma^{(x_0)}) = 16$ and $\mathrm{tr}(\gamma^{(x_4)}\gamma^{(x_0)}) = 0$; the cell
checks that nothing else is left over. The hand calculation for an inflating
direction $i$, line by line:

$$\gamma^{x_i}\Omega_{x_i} = \frac{\gamma^{(i)}}{Es}\cdot\frac{Es}{2}\big(A_1
\gamma^{(i)}\gamma^{(x_4)} + H\gamma^{(i)}\gamma^{(x_0)}\big) = \frac12\big(A_1
\gamma^{(x_4)} + H\gamma^{(x_0)}\big),$$

where the factors $Es$ cancel and $(\gamma^{(i)})^2 = +1$. For a deflating extra
time $t$ the frame factor is $f_t = s\sqrt{b_4}/E$ and its rate is $\alpha_t =
\partial_4 f_t/f_t = -A_1 + b_4'/(2b_4)$; $\Omega_{x_t} = \frac{f_t}{2}(\alpha_t
\gamma^{(x_4)}\gamma^{(t)} - H\gamma^{(t)}\gamma^{(x_0)})$, and with
$(\gamma^{(t)})^2 = -1$ and $\gamma^{(t)}\gamma^{(x_4)}\gamma^{(t)} =
+\gamma^{(x_4)}$ one gets $\gamma^{x_t}\Omega_{x_t} = \frac12(\alpha_t\gamma^{(x_4)}
+ H\gamma^{(x_0)})$, which is $\frac12(-A_1\gamma^{(x_4)} + H\gamma^{(x_0)})$ at
$b_4 = 1$. The record with $b_4$ stores only the sum; the cell compares the eight
terms at $b_4 = 1$ with the record of $b_4 = 1$.
```

In [9]:

```
Om = []  # Omega_mu, mu = x0 .. x7
for mu in range(8):
```

```
    M = sp.zeros(16, 16)
    for a in range(8):
        for b in range(a + 1, 8):
            if om[mu][a][b] != 0:
                M += om[mu][a][b] * SAB[a][b]
    Om.append(M.applyfunc(sp.expand))
gam = [G[a] / f[a] for a in range(8)]  # the coordinate gammas gamma^mu
alpha, beta, rest_zero = [], [], True
for mu in range(8):
    M = (gam[mu] * Om[mu]).applyfunc(sp.expand)  # gamma^mu Omega_mu, no sum
    alpha.append(sp.expand(-(M * G[X4]).trace() / 16))  # coefficient of gamma^(x4)
    beta.append(sp.expand((M * G[X0]).trace() / 16))  # coefficient of gamma^(x0)
    rest_zero = rest_zero and all(
        is_zero(v) for v in (M - alpha[-1] * G[X4] - beta[-1] * G[X0]))
for mu in range(8):
    say(f"gamma^{NAMES[mu]} Omega_{NAMES[mu]} = ({alpha[mu]}) gamma^(x4) + "
        f"({beta[mu]}) gamma^(x0)")
listed = from_wolfram(record["gammaOmega_per_direction"])
check(rest_zero and all(is_zero(to_ring(al) - alpha[mu - 1].subs(AT_B4_1))
                        and is_zero(to_ring(be) - beta[mu - 1].subs(AT_B4_1))
                        for mu, al, be in listed),
      "per direction gamma^mu Omega_mu = alpha gamma^(x4) + beta gamma^(x0); at "
      "b4 = 1 equal to the formula record",
      record=f"{FORMULAS}, formula gammaOmega_per_direction")
total = sum((gam[mu] * Om[mu] for mu in range(8)), Z16)
GRAV = 3 * H * G[X0] + 3 * b4p / (4 * b4) * G[X4]  # the gravitational term
check(all(is_zero(v) for v in (total - GRAV))
      and all(is_zero(v) for v in (total.subs(AT_B4_1) - 3 * H * G[X0])),
      "gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) (summed over "
      "mu); 3 H gamma^(x0) at b4 = 1",
      record=reproduces(PYREP_B4, "gammaOmega_total") + "; "
      + reproduces(PYREP, "gamma_mu_Omega_mu_equals_3H_gamma_x0") + " (at b4 = 1)")
check(sp.expand(sum(alpha[1:4])) == sp.Rational(3, 2) * A1
      and sp.expand(sum(alpha[5:8]) + sp.Rational(3, 2) * A1
                    - sp.Rational(3, 4) * b4p / b4) == 0
      and sp.expand(sum(alpha[5:8]).subs(AT_B4_1)) == -sp.Rational(3, 2) * A1
      and sp.expand(sum(beta)) == 3 * H,
      "the a4' terms cancel (+3 A1/2 inflating, -3 A1/2 deflating; the extra times "
      "leave (3/4) b4'/b4, zero at b4 = 1), the six x0 terms add to 3 H",
      record=reproduces(PYREP, "time_terms_cancel_hidden_term_survives")
      + " (at b4 = 1)")
```

Out [9]:

```
gamma^x0 Omega_x0 = (0) gamma^(x4) + (0) gamma^(x0)
gamma^x1 Omega_x1 = (A1/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x2 Omega_x2 = (A1/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x3 Omega_x3 = (A1/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x4 Omega_x4 = (0) gamma^(x4) + (0) gamma^(x0)
gamma^x5 Omega_x5 = (-A1/2 + b4p/(4*b4)) gamma^(x4) + (H/2) gamma^(x0)
gamma^x6 Omega_x6 = (-A1/2 + b4p/(4*b4)) gamma^(x4) + (H/2) gamma^(x0)
gamma^x7 Omega_x7 = (-A1/2 + b4p/(4*b4)) gamma^(x4) + (H/2) gamma^(x0)
PASS per direction gamma^mu Omega_mu = alpha gamma^(x4) + beta gamma^(x0); at b4 = 1
```

```
    equal to the formula record
     reproduces Revision/theory/field-theory.json, formula gammaOmega_per_direction
PASS gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) (summed over mu); 3 H
    gamma^(x0) at b4 = 1
     reproduces Revision/theory/reports/python-field-theory-b4.json, check
    gammaOmega_total; Revision/theory/reports/python-field-theory.json, check
    gamma_mu_Omega_mu_equals_3H_gamma_x0 (at b4 = 1)
PASS the a4' terms cancel (+3 A1/2 inflating, -3 A1/2 deflating; the extra times leave
    (3/4) b4'/b4, zero at b4 = 1), the six x0 terms add to 3 H
     reproduces Revision/theory/reports/python-field-theory.json, check
    time_terms_cancel_hidden_term_survives (at b4 = 1)
```

Text cell:

```
Figure 2 shows the cancellation on the background $b_4 = 1$: the coefficient of
$\gamma^{(x_4)}$ (in units of $a_4'$) is $+\frac12$ for each inflating direction and
$-\frac12$ for each deflating extra time, so its running sum climbs to $\frac32$ and
falls back to 0; the
coefficient of $\gamma^{(x_0)}$ (in units of $H$) is $+\frac12$ for all six and adds
up to 3. The deflation of the extra times is exactly what cancels the time term of
the inflation.
```

In [10]:

```
a_units = [float(sp.expand(v.subs(AT_B4_1) / A1)) if v != 0 else 0.0
           for v in alpha]  # in units of a4', at b4 = 1
b_units = [float(sp.expand(v / H)) if v != 0 else 0.0 for v in beta]  # H units
positions = np.arange(8)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9), sharey=True)
for ax, values, unit, title in [
        (left, a_units, "$a_4'$", "coefficient of $\\gamma^{(x_4)}$"),
        (right, b_units, "$H$", "coefficient of $\\gamma^{(x_0)}$")]:
    ax.bar(positions, values, color=["grey"] + ["tab:blue"] * 3 + ["grey"]
           + ["tab:red"] * 3, label="one direction")
    ax.plot(positions, np.cumsum(values), "ko-", label="running sum")
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, [f"$x_{k}$" for k in range(8)])
    ax.set_xlabel("direction $\\mu$ (blue 3-space, red extra times)")
    ax.set_title(f"{title} (units of {unit})")
    ax.legend(fontsize=8, loc="upper left")
left.set_ylabel("coefficient")
fig.tight_layout()
save_figure(fig, "gamma_omega_per_direction",
            "The spin-connection term $\\gamma^\\mu\\Omega_\\mu$ of the field "
            "equation, direction by direction (horizontal axis $\\mu = x_0$ to "
            "$x_7$; blue: inflating 3-space, red: deflating extra times, grey: "
            "$x_0$ and $x_4$, which give 0), on the background $b_4 = 1$. Left: "
            "the coefficient of "
            "$\\gamma^{(x_4)}$ in units of $a_4'$; it is $+1/2$ for each "
            "inflating and $-1/2$ for each deflating direction, so the running "
            "sum (black) returns to 0. Right: the coefficient of $\\gamma^{(x_0)}$ "
            "in units of $H$; all six are $+1/2$ and the sum is 3. Hence "
```

```
"$\\gamma^\\mu\\Omega_\\mu = 3H\\gamma^{(x_0)}$ for every $a_4$ at "
"$b_4 = 1$; a changing $b_4$ adds $\\frac{3}{4}(b_4'/b_4)"
"\\gamma^{(x_4)}$.")
```

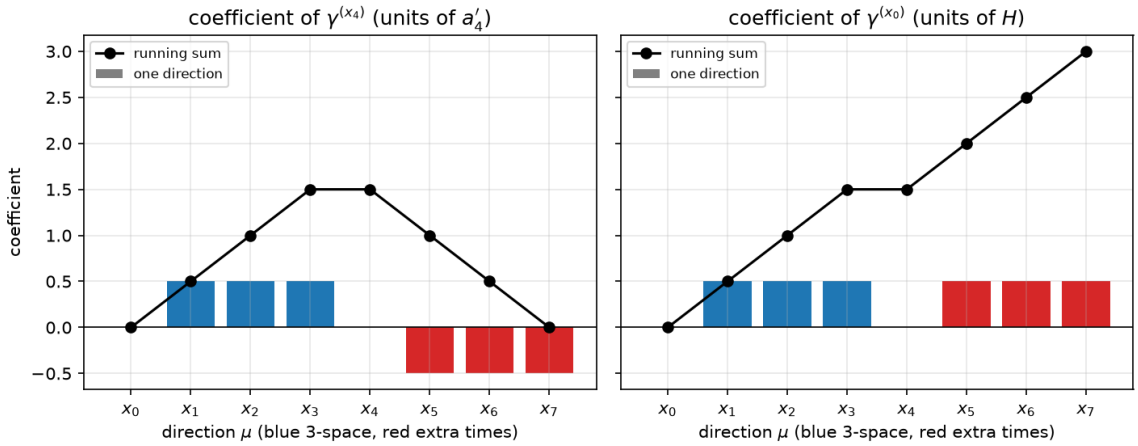

Figure 220: The spin-connection term $\gamma^\mu\Omega_\mu$ of the field equation, direction by direction (horizontal axis $\mu = x_0$ to x_7; blue: inflating 3-space, red: deflating extra times, grey: x_0 and x_4, which give 0), on the background $b_4 = 1$. Left: the coefficient of $\gamma^{(x_4)}$ in units of a_4'; it is $+1/2$ for each inflating and $-1/2$ for each deflating direction, so the running sum (black) returns to 0. Right: the coefficient of $\gamma^{(x_0)}$ in units of H; all six are $+1/2$ and the sum is 3. Hence $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ for every a_4 at $b_4 = 1$; a changing b_4 adds $\frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$. (Notebook 07b, figure 2.)

Out [10]:

```
Figure 07b.2 saved as Revision/textbook/figures/07b_2_gamma_omega_per_direction.png
```

Text cell:

```
## 11. Why the connection drops out of $L$ but not out of the field equation

$\Omega_\mu$ contains only generators $S^{(\mu)b}$ that share the index $\mu$ with
$\gamma^{(\mu)}$, and $\gamma^{(\mu)}$ anticommutes with such an $S^{(\mu)b}$. So
$\{\gamma^\mu, \Omega_\mu\} = 0$ for each $\mu$ separately. The connection enters
the symmetrised kinetic term only as $\frac12\bar\Psi\{\gamma^\mu,\Omega_\mu\}\Psi$
(the $\Omega$ parts of the two halves of $K$), so it drops out of $L$ in this
metric. It comes back through the variation: the derivative of $\sqrt{|g|}
\gamma^\mu$ appears when the action is varied (section 15), and

$$\frac{1}{2\sqrt{|g|}}\,\partial_\mu\big(\sqrt{|g|}\,\gamma^\mu\big)
= \frac{1}{2b_4^{3/2}c}\Big[\partial_0\Big(b_4^{3/2}c\,\frac{s^6}{c}\Big)\gamma^{(x_0)}
+ \partial_4\big(b_4^{3/2}c\big)\gamma^{(x_4)}\Big]
= 3H\gamma^{(x_0)} + \frac34\,\frac{b_4'}{b_4}\,\gamma^{(x_4)} .$$

The first equality: only $\mu = x_0$ and $\mu = x_4$ contribute, because nothing
depends on the other six coordinates; $\gamma^{x_0} = \gamma^{(x_0)}/f_0 =
(s^6/c)\gamma^{(x_0)}$ and $\gamma^{x_4} = \gamma^{(x_4)}$. The second:
```

```
$c\cdot s^6/c = s^6 = \sin z$ and $\partial_0\sin z = 6H\cos z$, so the first term
is $6Hb_4^{3/2}c/(2b_4^{3/2}c) = 3H$; and $\partial_4 b_4^{3/2} = \frac32
b_4^{1/2}b_4'$, so the second is $\frac34 b_4'/b_4$ (zero at $b_4 = 1$). This
equals $\gamma^\mu\Omega_\mu$: the surviving term is the half-density (volume and
frame divergence) term. The next cell checks both statements.
```

In [11]:

```
anti = all(all(is_zero(v) for v in (gam[mu] * Om[mu] + Om[mu] * gam[mu]))
           for mu in range(8))
check(anti, "{gamma^mu, Omega_mu} = 0 for each mu: the connection drops out of L",
      record=reproduces(PYREP, "anticommutator_gamma_Omega_vanishes"))
divergence = sum((((sqrt_g * gam[mu]).applyfunc(lambda v: cd(v, mu)))
                  for mu in range(8)), Z16) / (2 * sqrt_g)
check(all(is_zero(v) for v in (divergence - GRAV)),
      "(1/(2 sqrt|g|)) d_mu (sqrt|g| gamma^mu) = 3 H gamma^(x0) + (3/4)(b4'/b4) "
      "gamma^(x4)", record=reproduces(PYREP, "divergence_of_sqrtg_gamma")
      + " (at b4 = 1)")
```

Out [11]:

```
PASS {gamma^mu, Omega_mu} = 0 for each mu: the connection drops out of L
     reproduces Revision/theory/reports/python-field-theory.json, check
    anticommutator_gamma_Omega_vanishes
PASS (1/(2 sqrt|g|)) d_mu (sqrt|g| gamma^mu) = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4)
     reproduces Revision/theory/reports/python-field-theory.json, check
    divergence_of_sqrtg_gamma (at b4 = 1)
```

Text cell:

```
## 12. Fields at one point: a jet algebra for both statistics

To vary the Lagrangian we need the field components and their derivatives at one
point as symbols. A **generator** is a triple `(kind, A, derivatives)`: `kind` 0
for a component $\chi_A = \Psi_A^*$ of $\Psi^\dagger$, `kind` 1 for a component
$\psi_A = \Psi_A$ of $\Psi$; `A` = 0 to 15; `derivatives` a sorted tuple of
coordinate positions (empty for the value itself, `(4,)` for $\partial_4$). An
element is a dictionary from sorted tuples of generators (monomials) to ring
coefficients, as in Notebook 07a, with one switch: `odd = True` makes the
generators anticommute (dirac16complex), `odd = False` makes them commute
(dirac16complex00).

- Products sort the generators; for odd generators every exchange gives $-1$ and a
  repeated generator gives 0; for even ones nothing changes sign.
- The conjugation reverses each product, exchanges $\chi_A \leftrightarrow \psi_A$
  (keeping the derivatives) and replaces $i$ by $-i$.
- The left (right) derivative with respect to a generator moves it to the far left
  (right) with the Grassmann signs; for commuting generators both are the ordinary
  partial derivative ($x^n \to n x^{n-1}$).
- The total derivative $d/dx_\mu$ differentiates the coefficient with `cd` and
  raises the derivative tuple of each generator in turn (product rule).
```

The next cell defines the class ‘Jet‘ and the vector helpers.

In [12]:

```
CHI, PSI = 0, 1  # the two kinds: components of Psi^dagger and of Psi


def gen(kind, A, derivatives=()):
    """The generator: component A of chi (kind 0) or psi (kind 1), differentiated
    along the coordinate positions in derivatives."""
    return (kind, A, tuple(sorted(derivatives)))


def sort_sign(keys, odd):
    """Sorted tuple of generators and the sign; (None, 0) for a zero product."""
    items, sign = list(keys), 1
    for end in range(len(items) - 1, 0, -1):  # bubble sort, counting exchanges
        for i in range(end):
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                sign = -sign
    if not odd:
        return tuple(items), 1  # commuting generators: no signs, squares allowed
    if any(items[i] == items[i + 1] for i in range(len(items) - 1)):
        return None, 0  # a repeated Grassmann generator: the product is zero
    return tuple(items), sign


class Jet:
    """An element of the jet algebra: self.terms maps monomials to coefficients."""

    def __init__(self, odd, terms=None):
        self.odd, self.terms = odd, {}
        for monomial, coefficient in (terms or {}).items():
            self.add(monomial, coefficient)

    def add(self, monomial, coefficient):
        total = sp.expand(self.terms.get(monomial, 0) + coefficient)
        if total == 0:
            self.terms.pop(monomial, None)
        else:
            self.terms[monomial] = total

    def __add__(self, other):
        result = Jet(self.odd, self.terms)
        for monomial, coefficient in other.terms.items():
            result.add(monomial, coefficient)
        return result

    def __sub__(self, other):
        return self + other.times(-1)
```

```
def times(self, factor):  # an ordinary (ring) factor
    return Jet(self.odd, {k: factor * v for k, v in self.terms.items()})

def __mul__(self, other):
    result = Jet(self.odd)
    for k1, v1 in self.terms.items():
        for k2, v2 in other.terms.items():
            monomial, sign = sort_sign(k1 + k2, self.odd)
            if monomial is not None:
                result.add(monomial, sign * v1 * v2)
    return result

def conj(self):
    result = Jet(self.odd)
    for monomial, coefficient in self.terms.items():
        images = [(1 - kind, A, d) for kind, A, d in reversed(monomial)]
        ordered, sign = sort_sign(images, self.odd)
        if ordered is not None:
            result.add(ordered, sign * sp.sympify(coefficient).subs(sp.I, -sp.I))
    return result

def lderiv(self, key):
    result = Jet(self.odd)
    for monomial, coefficient in self.terms.items():
        if key in monomial:
            p = monomial.index(key)
            factor = (-1) ** p if self.odd else monomial.count(key)
            result.add(monomial[:p] + monomial[p + 1:], factor * coefficient)
    return result

def rderiv(self, key):
    if not self.odd:
        return self.lderiv(key)
    result = Jet(self.odd)
    for monomial, coefficient in self.terms.items():
        if key in monomial:
            p = monomial.index(key)
            factor = (-1) ** (len(monomial) - 1 - p)
            result.add(monomial[:p] + monomial[p + 1:], factor * coefficient)
    return result

def total(self, mu):
    result = Jet(self.odd)
    for monomial, coefficient in self.terms.items():
        derivative = cd(coefficient, mu)
        if derivative != 0:
            result.add(monomial, derivative)
        for p, (kind, A, d) in enumerate(monomial):
            raised = (kind, A, tuple(sorted(d + (mu,))))
            ordered, sign = sort_sign(monomial[:p] + (raised,) + monomial[p + 1:],
                                      self.odd)
            if ordered is not None:
                result.add(ordered, sign * coefficient)
    return result
```

```
    def is_zero(self):
        return all(is_zero(v) for v in self.terms.values())


def column(odd, kind, derivatives=()):
    """The 16 generators of chi or psi (or of one of their derivatives)."""
    return [Jet(odd, {(gen(kind, A, derivatives),): 1}) for A in range(16)]


def matvec(M, v, odd):
    """The column M v."""
    out = [Jet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[i] = out[i] + v[j].times(M[i, j])
    return out


def vecmat(v, M, odd):
    """The row v M."""
    out = [Jet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[j] = out[j] + v[i].times(M[i, j])
    return out


def dot(row, col, odd):
    """The number row col (row first)."""
    result = Jet(odd)
    for x, y in zip(row, col):
        result = result + x * y
    return result


say("The jet algebra and the vector helpers are defined.")
```

Out [12]:

```
The jet algebra and the vector helpers are defined.
```

Text cell:

```
## 13. The Lagrangian of both fields

The next cell builds, for each statistics, the jets $\psi = \Psi$, $\chi =
\Psi^\dagger$, the row $\bar\Psi = \chi C$, the covariant derivatives $D_\mu\psi =
\partial_\mu\psi + \Omega_\mu\psi$ and $D_\mu\bar\Psi = \partial_\mu\bar\Psi -
\bar\Psi\Omega_\mu$, the kinetic term $K = \frac12\sum_\mu(\bar\Psi\gamma^\mu
```

```
D_\mu\psi - D_\mu\bar\Psi\gamma^\mu\psi)$, $S = \bar\Psi\psi$, and

$$L = \sqrt{|g|}\,\big(K - mS - \tfrac{\lambda}{2}S^2\big).$$

It also builds the unsymmetrised form $L_u = \sqrt{|g|}(\bar\Psi\gamma^\mu
D_\mu\psi - mS - \frac{\lambda}{2}S^2)$ and the residual of the expected field
equation, $\mathcal{E} = \gamma^\mu D_\mu\psi - (m + \lambda S)\psi$, and counts
the monomials of $L$ by type: kinetic ($8 \times 32 = 256$: in each direction 16
terms $\chi\,\partial\psi$ and 16 terms $\partial\chi\,\psi$), mass (16, one for
each nonzero entry of $C$), and quartic. In the quartic term $S^2 = \sum_{A,B}
P_AP_B$ with the 16 pairs $P_A = \chi_A C_{A\pi(A)}\psi_{\pi(A)}$: for commuting
numbers the 16 squares $P_A^2$ and the $\binom{16}{2} = 120$ products of two
different pairs survive (136), for Grassmann numbers the squares vanish (120).
```

In [13]:

```
def build(odd, connection):
    """Lagrangian, unsymmetrised Lagrangian, field-equation residual and pieces
    for one statistics (odd True: Grassmann) and one spinor connection."""
    psi, chi = column(odd, PSI), column(odd, CHI)
    psibar = vecmat(chi, C, odd)  # Psibar = Psi^dagger C
    Dpsi = [[x + y for x, y in zip(column(odd, PSI, (mu,)),
                                   matvec(connection[mu], psi, odd))]
            for mu in range(8)]
    Dpsibar = [[x - y for x, y in zip(vecmat(column(odd, CHI, (mu,)), C, odd),
                                      vecmat(psibar, connection[mu], odd))]
               for mu in range(8)]
    K, Ku = Jet(odd), Jet(odd)
    for mu in range(8):
        forward = dot(psibar, matvec(gam[mu], Dpsi[mu], odd), odd)
        K = K + forward - dot(vecmat(Dpsibar[mu], gam[mu], odd), psi, odd)
        Ku = Ku + forward
    S = dot(psibar, psi, odd)
    potential = S.times(m) + (S * S).times(lam / 2)  # m S + (lambda/2) S^2
    L = (K.times(sp.Rational(1, 2)) - potential).times(sqrt_g)
    Lu = (Ku - potential).times(sqrt_g)
    dirac = [Jet(odd) for _ in range(16)]  # gamma^mu D_mu psi
    for mu in range(8):
        dirac = [x + y for x, y in zip(dirac, matvec(gam[mu], Dpsi[mu], odd))]
    V = Jet(odd, {(): m}) + S.times(lam)  # m + lambda S
    residual = [dirac[A] - V * psi[A] for A in range(16)]
    bar_residual = [Jet(odd)] * 16  # -(D_mu Psibar gamma^mu + V Psibar)
    for mu in range(8):
        bar_residual = [x - y for x, y in zip(bar_residual,
                                              vecmat(Dpsibar[mu], gam[mu], odd))]
    bar_residual = [x - V * y for x, y in zip(bar_residual, psibar)]
    return {"L": L, "Lu": Lu, "E": residual, "Ebar": bar_residual, "S": S,
            "psi": psi, "psibar": psibar, "dirac": dirac}


STATS = {"grassmann": True, "commuting": False}  # statistics -> odd
fields = {name: build(odd, Om) for name, odd in STATS.items()}
kinds = {}
```

```
for name, data in fields.items():
    monomials = data["L"].terms
    kinetic = sum(1 for k in monomials if any(d for (_, _, d) in k))
    quartic = sum(1 for k in monomials if len(k) == 4)
    kinds[name] = (kinetic, len(monomials) - kinetic - quartic, quartic)
    report(f"monomials of L, {name}", f"{len(monomials)} = {kinetic} kinetic + "
           f"{kinds[name][1]} mass + {quartic} quartic")
```

Out [13]:

```
RESULT monomials of L, grassmann = 392 = 256 kinetic + 16 mass + 120 quartic
RESULT monomials of L, commuting = 408 = 256 kinetic + 16 mass + 136 quartic
```

Text cell:

```
The next cell compares the two counts with the numbers that the sympy verifier of
the Revision record wrote into the details of its reality checks, and draws them
by type (Figure 3).
```

In [14]:

```
for name in STATS:
    detail = revision_check(PYREP, f"{name}_lagrangian_real")["detail"]
    recorded = int(re.search(r"\((\d+) monomials", detail).group(1))
    check(len(fields[name]["L"].terms) == recorded,
          f"{name}: L has {recorded} monomials, as recorded",
          record=reproduces(PYREP, f"{name}_lagrangian_real"))
check(kinds["grassmann"] == (256, 16, 120) and kinds["commuting"] == (256, 16, 136),
      "the difference is the 16 squares of the pairs in S^2 (136 - 120)")

fig, ax = plt.subplots(figsize=(7.0, 4.2))
labels = ["dirac16complex\n(Grassmann)", "dirac16complex00\n(commuting)"]
bottoms = np.zeros(2)
for index, (part, colour) in enumerate([("kinetic", "tab:blue"), ("mass", "tab:green"),
                                        ("quartic", "tab:orange")]):
    values = np.array([kinds["grassmann"][index], kinds["commuting"][index]])
    ax.bar(labels, values, bottom=bottoms, color=colour, label=part, width=0.5)
    for x, (value, bottom) in enumerate(zip(values, bottoms)):
        ax.text(x, bottom + value / 2, str(value), ha="center", va="center")
    bottoms = bottoms + values
for x, total in enumerate(bottoms):
    ax.text(x, total + 6, f"total {int(total)}", ha="center")
ax.set_ylim(0, 520)  # room above the bars for the legend
ax.set_ylabel("number of monomials of $L$")
ax.set_title("The Lagrangian at one point, by type of term")
ax.legend(loc="upper center", ncol=3, fontsize=8)
save_figure(fig, "lagrangian_monomials",
            "The number of monomials of the Lagrangian $L$ of the two fields in the "
            "author's metric, written in the jet algebra (the field components and "
            "their first derivatives at one point as generators), split into "
            "kinetic terms (256 for both), mass terms (16 for both) and quartic "
```

```
        "terms from $\\frac{\\lambda}{2}S^2$. The only difference between the "
        "two statistics: for commuting components the 16 squares of the pairs "
        "$\\bar\\Psi_A\\Psi_B$ survive (136 quartic terms), for anticommuting "
        "components they vanish (120); totals 408 and 392, the numbers of the "
        "Revision record.")
```

Out [14]:

```
PASS grassmann: L has 392 monomials, as recorded
     reproduces Revision/theory/reports/python-field-theory.json, check
    grassmann_lagrangian_real
PASS commuting: L has 408 monomials, as recorded
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_lagrangian_real
PASS the difference is the 16 squares of the pairs in S^2 (136 - 120)
```

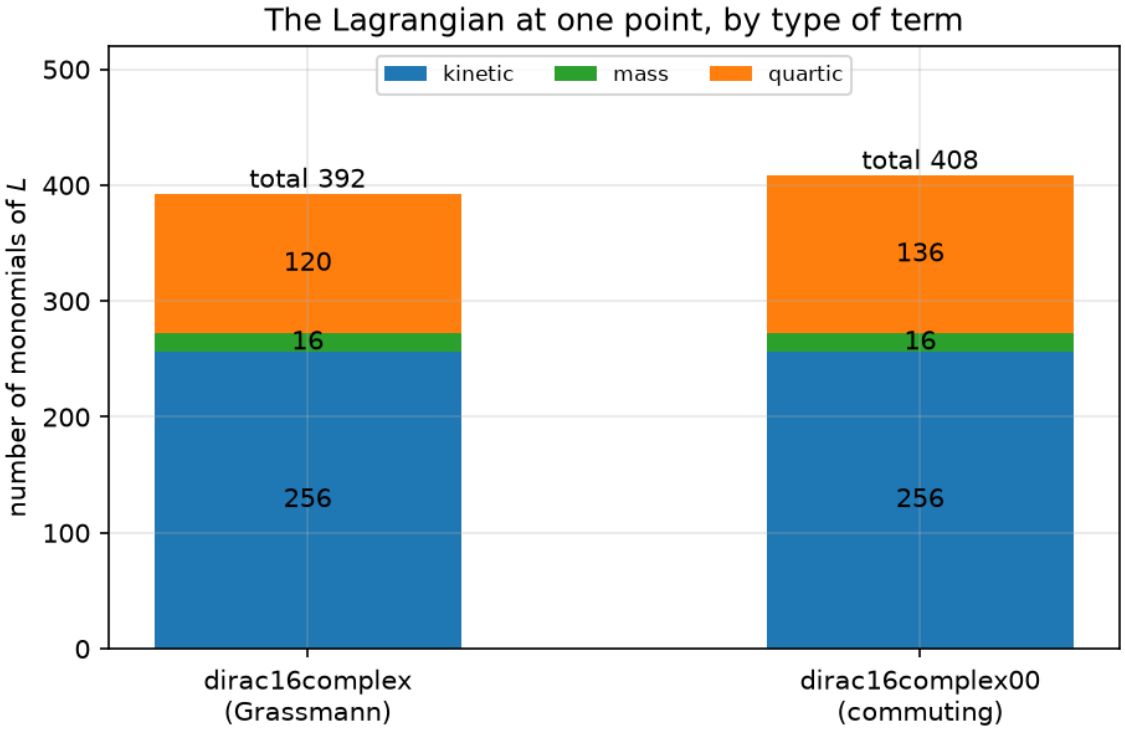

Figure 221: The number of monomials of the Lagrangian L of the two fields in the author's metric, written in the jet algebra (the field components and their first derivatives at one point as generators), split into kinetic terms (256 for both), mass terms (16 for both) and quartic terms from $\frac{\lambda}{2}S^2$. The only difference between the two statistics: for commuting components the 16 squares of the pairs $\bar{\Psi}_A\Psi_B$ survive (136 quartic terms), for anticommuting components they vanish (120); totals 408 and 392, the numbers of the Revision record. (Notebook 07b, figure 3.)

Out [14]:

```
Figure 07b.3 saved as Revision/textbook/figures/07b_3_lagrangian_monomials.png
```

Text cell:

```
## 14. Reality, the total divergence, and the connection that drops out

Three exact statements, for both statistics:

1. $L^* = L$: the Lagrangian is real. The proof by hand uses that $C\gamma^{(a)}$
   is real and antisymmetric (so the two halves of $K$ are complex conjugates of
   each other) and that $C$ is real and symmetric (so $S^* = S$). The control: the
   unsymmetrised $L_u$ is NOT real, so the test is not empty.
2. $L = L_u - \frac12\partial_\mu\big(\sqrt{|g|}\,\bar\Psi\gamma^\mu\Psi\big)$:
   the two forms differ by a total divergence and give the same field equations.
3. In this metric $L$ computed with $\Omega_\mu$ equals $L$ computed with
   $\Omega_\mu$ set to zero (section 11).

The next cell checks all three and names the Revision checks that state them.
```

In [15]:

```
no_connection = [Z16] * 8  # Omega_mu = 0
for name, odd in STATS.items():
    data = fields[name]
    L, Lu = data["L"], data["Lu"]
    check((L.conj() - L).is_zero() and not (Lu.conj() - Lu).is_zero(),
          f"{name}: L is real, the unsymmetrised L_u is not",
          record=reproduces(PYREP, f"{name}_controls_not_vacuous"))
    current = Jet(odd)
    for mu in range(8):
        current = current + dot(data["psibar"], matvec(gam[mu], data["psi"], odd),
                                odd).times(sqrt_g).total(mu)
    check((L - Lu + current.times(sp.Rational(1, 2))).is_zero(),
          f"{name}: L = L_u - (1/2) d_mu (sqrt|g| Psibar gamma^mu Psi)",
          record=reproduces(PYREP, f"{name}_total_divergence_relation"))
    flat_connection = build(odd, no_connection)["L"]
    tag = "G" if odd else "C"
    check((L - flat_connection).is_zero(),
          f"{name}: L with Omega equals L with Omega = 0 in this metric",
          record=reproduces(WLREP, f"L_spin_connection_drops_out_{tag}"))
```

Out [15]:

```
PASS grassmann: L is real, the unsymmetrised L_u is not
     reproduces Revision/theory/reports/python-field-theory.json, check
    grassmann_controls_not_vacuous
PASS grassmann: L = L_u - (1/2) d_mu (sqrt|g| Psibar gamma^mu Psi)
     reproduces Revision/theory/reports/python-field-theory.json, check
    grassmann_total_divergence_relation
PASS grassmann: L with Omega equals L with Omega = 0 in this metric
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    L_spin_connection_drops_out_G
PASS commuting: L is real, the unsymmetrised L_u is not
```

```
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_controls_not_vacuous
PASS commuting: L = L_u - (1/2) d_mu (sqrt|g| Psibar gamma^mu Psi)
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_total_divergence_relation
PASS commuting: L with Omega equals L with Omega = 0 in this metric
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    L_spin_connection_drops_out_C
```

Text cell:

```
## 15. The Euler-Lagrange equations

**The derivation by hand** (commuting field; the Grassmann case follows below).
Write $V = m + \lambda S$ and $L/\sqrt{|g|} = \frac12\Psi^\dagger C\gamma^\mu(
\partial_\mu\Psi + \Omega_\mu\Psi) - \frac12(\partial_\mu\Psi^\dagger C -
\Psi^\dagger C\Omega_\mu)\gamma^\mu\Psi - m\Psi^\dagger C\Psi -
\frac{\lambda}{2}(\Psi^\dagger C\Psi)^2$. Vary the component $\Psi_A^* = \chi_A$.

- Step 1. $\partial L/\partial\chi_A = \sqrt{|g|}\big[\frac12(C\gamma^\mu
  D_\mu\Psi)_A + \frac12(C\Omega_\mu\gamma^\mu\Psi)_A - V(C\Psi)_A\big]$ (every
  term with an undifferentiated $\chi$ contributes its column entry $A$; the chain
  rule gives $\partial(\frac{\lambda}{2}S^2)/\partial\chi_A = \lambda S(C\Psi)_A$).
- Step 2. $\partial L/\partial(\partial_\mu\chi_A) = -\frac12\sqrt{|g|}\,
  (C\gamma^\mu\Psi)_A$ (only the second term contains $\partial_\mu\chi$).
- Step 3. The Euler-Lagrange expression is Step 1 minus $\partial_\mu$ of Step 2,
  that is Step 1 plus $\frac12\partial_\mu(\sqrt{|g|}\,C\gamma^\mu\Psi)_A$.
- Step 4. By the product rule and section 11, $\partial_\mu(\sqrt{|g|}\gamma^\mu
  \Psi) = 2\sqrt{|g|}\gamma^\mu\Omega_\mu\Psi + \sqrt{|g|}\gamma^\mu\partial_\mu
  \Psi$, and $\gamma^\mu\Omega_\mu = -\Omega_\mu\gamma^\mu$ for each $\mu$.
- Step 5. Adding: the terms $\pm\frac12 C\Omega_\mu\gamma^\mu\Psi$ cancel, and
  $\frac12C\gamma^\mu D_\mu\Psi + \frac12 C\gamma^\mu\Omega_\mu\Psi + \frac12
  C\gamma^\mu\partial_\mu\Psi = C\gamma^\mu D_\mu\Psi$. So the expression is
  $\sqrt{|g|}\,\big(C(\gamma^\mu D_\mu\Psi - V\Psi)\big)_A$.
- Step 6. $\sqrt{|g|} = b_4^{3/2}\cos z \ne 0$ and $C$ is invertible ($C^2 = 1$), so the
  16 expressions vanish exactly when $\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi$.

**Grassmann components.** The left derivative with respect to $\chi_A$ is used. In
every term of $L$ the factor $\chi$ (or $\partial_\mu\chi$) stands on the far left,
so moving it there costs no sign, and $S$ is even; Steps 1 to 6 go through word for
word. Varying $\psi_B$ with the right derivative gives the adjoint equation
$D_\mu\bar\Psi\gamma^\mu = -(m + \lambda S)\bar\Psi$, which is the Dirac conjugate
of the first one.

The next cell lets the computer do all of this: for every $A$ it computes
$\partial_L L/\partial\chi_A - \sum_\mu \frac{d}{dx_\mu}\partial_L L/\partial(
\partial_\mu\chi_A)$ and compares it with $\sqrt{|g|}(C\mathcal{E})_A$; as a control
it repeats the comparison with $m$ replaced by $-m$, which must fail.
```

In [16]:

```
def euler_lagrange_chi(L, A):
    """dL/dchi_A - d_mu dL/d(d_mu chi_A), left derivatives."""
```

```
    result = L.lderiv(gen(CHI, A))
    for mu in range(8):
        result = result - L.lderiv(gen(CHI, A, (mu,))).total(mu)
    return result


def euler_lagrange_psi(L, B):
    """dL/dpsi_B - d_mu dL/d(d_mu psi_B), right derivatives."""
    result = L.rderiv(gen(PSI, B))
    for mu in range(8):
        result = result - L.rderiv(gen(PSI, B, (mu,))).total(mu)
    return result


for name, odd in STATS.items():
    data = fields[name]
    L, residual, psi = data["L"], data["E"], data["psi"]
    expected = matvec(C, residual, odd)  # C times the residual
    ok = all((euler_lagrange_chi(L, A) - expected[A].times(sqrt_g)).is_zero()
             for A in range(16))
    wrong = matvec(C, [residual[B] + psi[B].times(2 * m) for B in range(16)], odd)
    control = not (euler_lagrange_chi(L, 0) - wrong[0].times(sqrt_g)).is_zero()
    check(ok and control, f"{name}: varying Psi^dagger gives gamma^mu D_mu Psi = "
          "(m + lambda S) Psi in all 16 components (control m -> -m fails)",
          record=reproduces(PYREP, f"{name}_euler_lagrange_psibar_variation"))
    ok = all((euler_lagrange_psi(L, B) - data["Ebar"][B].times(sqrt_g)).is_zero()
             for B in range(16))
    check(ok, f"{name}: varying Psi gives D_mu Psibar gamma^mu = -(m + lambda S) "
          "Psibar", record=reproduces(PYREP, f"{name}_euler_lagrange_psi_variation"))
    conjugate_row = vecmat([x.conj() for x in residual], C, odd)
    check(all((conjugate_row[B] - data["Ebar"][B]).is_zero() for B in range(16)),
          f"{name}: the adjoint equation is the Dirac conjugate of the field equation",
          record=reproduces(PYREP, f"{name}_adjoint_equation_is_conjugate"))
```

Out [16]:

```
PASS grassmann: varying Psi^dagger gives gamma^mu D_mu Psi = (m + lambda S) Psi in all 16
    components (control m -> -m fails)
     reproduces Revision/theory/reports/python-field-theory.json, check
    grassmann_euler_lagrange_psibar_variation
PASS grassmann: varying Psi gives D_mu Psibar gamma^mu = -(m + lambda S) Psibar
     reproduces Revision/theory/reports/python-field-theory.json, check
    grassmann_euler_lagrange_psi_variation
PASS grassmann: the adjoint equation is the Dirac conjugate of the field equation
     reproduces Revision/theory/reports/python-field-theory.json, check
    grassmann_adjoint_equation_is_conjugate
PASS commuting: varying Psi^dagger gives gamma^mu D_mu Psi = (m + lambda S) Psi in all 16
    components (control m -> -m fails)
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_euler_lagrange_psibar_variation
PASS commuting: varying Psi gives D_mu Psibar gamma^mu = -(m + lambda S) Psibar
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_euler_lagrange_psi_variation
```

```
PASS commuting: the adjoint equation is the Dirac conjugate of the field equation
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_adjoint_equation_is_conjugate
```

Text cell:

```
## 16. The equations written out in the author's metric

Since $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ and $\gamma^\mu\Omega_\mu = 3H
\gamma^{(x_0)} + \frac34(b_4'/b_4)\gamma^{(x_4)}$, the Dirac operator is

$$\gamma^\mu D_\mu\Psi = \sum_{a=0}^{7}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi +
3H\gamma^{(x_0)}\Psi + \frac34\frac{b_4'}{b_4}\gamma^{(x_4)}\Psi,$$
$$\frac{1}{f_a} = \frac{s^6}{c} = \tan z\ (x_0),\ \frac{1}{Es}\ (x_1, x_2, x_3),\
1\ (x_4),\ \frac{E}{s\sqrt{b_4}}\ (x_5, x_6, x_7).$$

The coefficient of the extra-time derivatives, $e^{a_4}b_4^{-1/2}\sin^{-1/6}z$,
grows as the extra times deflate. The next cell checks this form for both
statistics, then turns each of our 16 component equations into an ordinary sympy
expression in the symbols `d<a>_<B>` (for $\partial_a\Psi_B$, with $a = 0, \dots,
7$ for $x_0, \dots, x_7$) and `P<B>` (for $\Psi_B$), and compares it with the
corresponding equation of both formula records (key `field_equation_components`,
left-hand sides), term by term: as it is with the record with $b_4$, and at
$b_4 = 1$ with the record of $b_4 = 1$. It prints the first equation in both
forms.
```

In [17]:

```
for name, odd in STATS.items():
    data = fields[name]
    explicit = matvec(GRAV, data["psi"], odd)  # (3 H gamma^(x0) + ...) psi
    for a in range(8):
        explicit = [x + y for x, y in zip(explicit, matvec(
            G[a] / f[a], column(odd, PSI, (a,)), odd))]
    tag = "G" if odd else "C"
    check(all((data["dirac"][A] - explicit[A]).is_zero() for A in range(16)),
          f"{name}: gamma^mu D_mu Psi = sum_a (1/f_a) gamma^(a) d_a Psi + 3 H "
          "gamma^(x0) Psi + (3/4)(b4'/b4) gamma^(x4) Psi",
          record=reproduces(WLREP, f"Dirac_operator_explicit_{tag}") + " (at b4 = 1)")


def as_sympy(element):
    """A linear jet element as a sympy expression in d<a>_<B> and P<B>."""
    total = 0
    for ((kind, B, d),), coefficient in element.terms.items():
        symbol = f"d{d[0]}_{B + 1}" if d else f"P{B + 1}"  # d0 .. d7: x0 .. x7
        total += coefficient * sp.Symbol(symbol)
    return total


ours = [as_sympy(x) for x in fields["grassmann"]["dirac"]]
theirs = [to_ring(from_wolfram(text.split("==")[0]))
```

```
          for text in record_b4["field_equation_components"]]
theirs_1 = [to_ring(from_wolfram(text.split("==")[0]))
            for text in record["field_equation_components"]]
say(f"equation 1, this notebook: {ours[0]} = V P1")
say(f"equation 1, formula record: {theirs[0]} = V P1")
check(len(theirs) == len(theirs_1) == 16
      and all(is_zero(x - y) for x, y in zip(ours, theirs))
      and all(is_zero(x.subs(AT_B4_1) - y) for x, y in zip(ours, theirs_1)),
      "all 16 component equations equal the formula record with b4 term by term, "
      "and at b4 = 1 the record of b4 = 1",
      record=f"{FORMULAS_B4} and {FORMULAS}, formula field_equation_components")
```

Out [17]:

```
PASS grassmann: gamma^mu D_mu Psi = sum_a (1/f_a) gamma^(a) d_a Psi + 3 H gamma^(x0) Psi
    + (3/4)(b4'/b4) gamma^(x4) Psi
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Dirac_operator_explicit_G (at b4 = 1)
PASS commuting: gamma^mu D_mu Psi = sum_a (1/f_a) gamma^(a) d_a Psi + 3 H gamma^(x0) Psi
    + (3/4)(b4'/b4) gamma^(x4) Psi
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Dirac_operator_explicit_C (at b4 = 1)
equation 1, this notebook: E*d5_15/(sqrt(b4)*s) + E*d6_16/(sqrt(b4)*s) +
    E*d7_9/(sqrt(b4)*s) + 3*H*P9 - 3*P14*b4p/(4*b4) - d4_14 + d0_9*s**6/c - d1_16/(E*s) +
    d2_15/(E*s) - d3_14/(E*s) = V P1
equation 1, formula record: E*d5_15/(sqrt(b4)*s) + E*d6_16/(sqrt(b4)*s) +
    E*d7_9/(sqrt(b4)*s) + 3*H*P9 - 3*P14*b4p/(4*b4) - d4_14 + d0_9*s**6/c - d1_16/(E*s) +
    d2_15/(E*s) - d3_14/(E*s) = V P1
PASS all 16 component equations equal the formula record with b4 term by term, and at b4
    = 1 the record of b4 = 1
     reproduces Revision/theory/field-theory-b4.json and Revision/theory/field-
    theory.json, formula field_equation_components
```

Text cell:

```
Figure 4 shows which components are coupled. Every frame gamma is a signed
permutation matrix, so each equation contains one derivative term per direction. A
square of the map (row $A$ = equation, column $B$ = component) is coloured by the
number of derivatives $\partial_a\Psi_B$ in equation $A$ and labelled with the
directions $a$. Each equation contains only four components, each with two
directions, and the equations of rows 1 to 8 (chirality $\Gamma = -1$) contain only
components 9 to 16 ($\Gamma = +1$) and back: every gamma is block off-diagonal in
the chiral split. The term $3H\gamma^{(x_0)}\Psi$ sits in the squares labelled
with 0, next to $\tan z\,\partial_0$ of the same component, and the term
$\frac34(b_4'/b_4)\gamma^{(x_4)}\Psi$ in the squares labelled with 4.
```

In [18]:

```
counts = np.zeros((16, 16), dtype=int)
tags = [["" for _ in range(16)] for _ in range(16)]
for a in range(8):
```

```
    for A in range(16):
        for B in range(16):
            if G[a][A, B] != 0:
                counts[A, B] += 1
                tags[A][B] += str(a)  # the digit k names the direction xk
four_twos = all(np.count_nonzero(counts[A]) == 4 and counts[A].max() == 2
                for A in range(16))  # 4 components, each with 2 directions
check(four_twos and int(counts.sum()) == 128
      and not counts[:8, :8].any() and not counts[8:, 8:].any(),
      "each equation couples 4 components with 2 directions each, across the chiral "
      "halves")

fig, ax = plt.subplots(figsize=(7.4, 6.6))
image = ax.imshow(counts, cmap="Blues", vmin=0, vmax=2)
for A in range(16):
    for B in range(16):
        if tags[A][B]:
            ax.text(B, A, tags[A][B], ha="center", va="center", fontsize=7,
                    color="white" if counts[A, B] == 2 else "black")
ax.axhline(7.5, color="red", linewidth=1)
ax.axvline(7.5, color="red", linewidth=1)
ax.set_xticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
ax.set_yticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
ax.set_xlabel("component $B$ whose derivative appears")
ax.set_ylabel("equation $A$")
ax.set_title("Which derivatives $\\partial_a\\Psi_B$ enter equation $A$")
ax.grid(False)
fig.colorbar(image, ax=ax, ticks=[0, 1, 2], shrink=0.8,
             label="number of directions $a$")
save_figure(fig, "coupling_map",
            "The coupling map of the 16 component field equations in the author's "
            "metric: row $A$ is the equation $(\\gamma^\\mu D_\\mu\\Psi)_A = "
            "V\\Psi_A$, column $B$ a component; a square is coloured by the number "
            "of directions $a$ whose derivative $\\partial_a\\Psi_B$ appears in "
            "equation $A$, and the digits name those directions ($k$ for "
            "$x_k$). Every equation contains 4 components with 2 "
            "directions each. The red lines separate the chiral halves: rows 1 to 8 "
            "involve only components 9 to 16 and rows 9 to 16 only components 1 to "
            "8. The term $3H\\gamma^{(x_0)}\\Psi$ appears in the squares that "
            "contain the digit 0, the term $\\frac{3}{4}(b_4'/b_4)"
            "\\gamma^{(x_4)}\\Psi$ in those that contain the digit 4.")
```

Out [18]:

```
PASS each equation couples 4 components with 2 directions each, across the chiral halves
```

Out [18]:

```
Figure 07b.4 saved as Revision/textbook/figures/07b_4_coupling_map.png
```

Text cell:

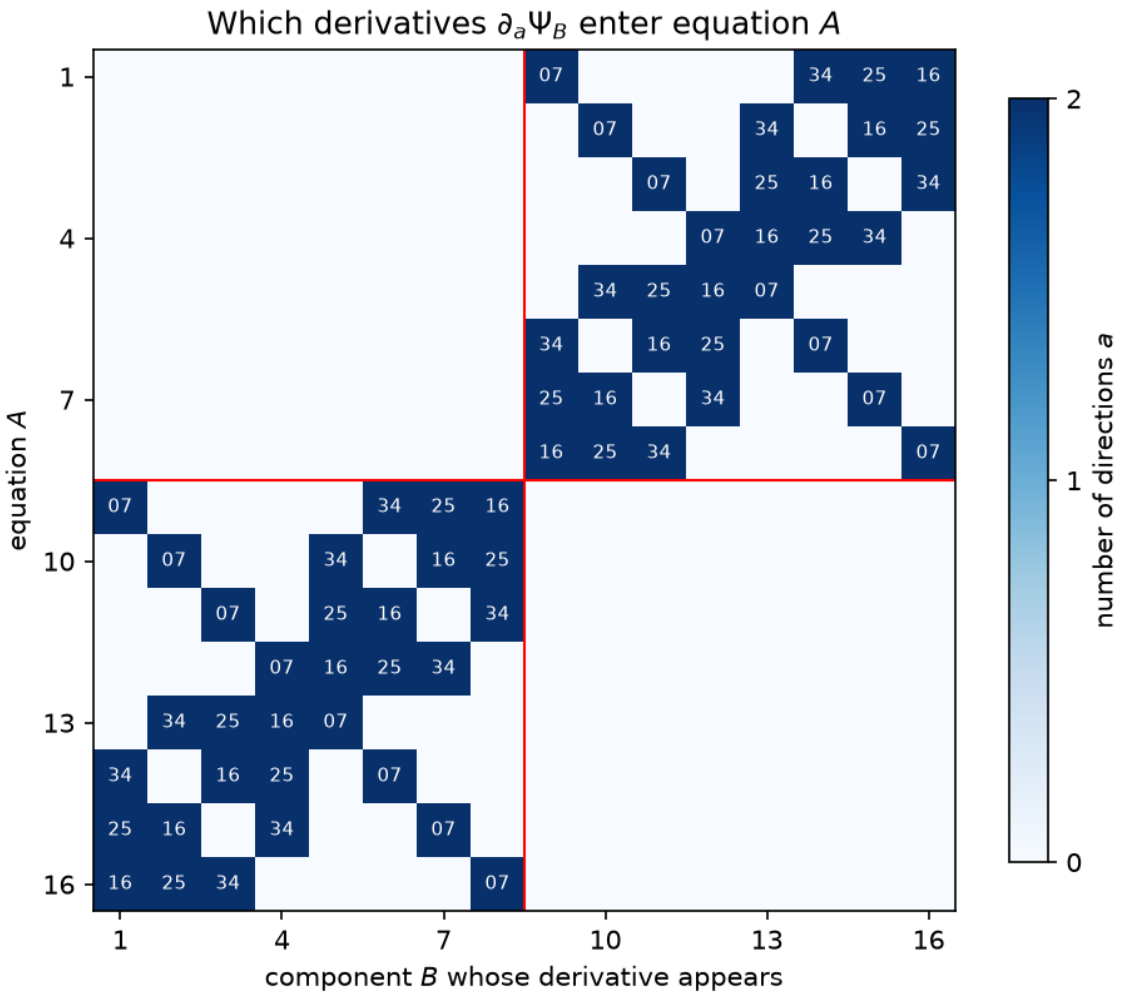

Figure 222: The coupling map of the 16 component field equations in the author's metric: row A is the equation $(\gamma^\mu D_\mu \Psi)_A = V \Psi_A$, column B a component; a square is coloured by the number of directions a whose derivative $\partial_a \Psi_B$ appears in equation A, and the digits name those directions (k for x_k). Every equation contains 4 components with 2 directions each. The red lines separate the chiral halves: rows 1 to 8 involve only components 9 to 16 and rows 9 to 16 only components 1 to 8. The term $3H\gamma^{(x_0)}\Psi$ appears in the squares that contain the digit 0, the term $\frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}\Psi$ in those that contain the digit 4. (Notebook 07b, figure 4.)

17. The chiral blocks and the evolution form

Two more ways of writing the same 16 equations; both are stated in the Revision formula record.

The chiral blocks. Split Ψ into its upper half ψ_- (components 1 to 8, where the chirality matrix $\Gamma = \mathrm{diag}(-I_8, I_8)$ is -1) and its lower half ψ_+ (components 9 to 16, where $\Gamma = +1$). Every frame gamma is block off-diagonal,

$$\gamma^{(a)} = \begin{pmatrix} 0 & \bar\tau_a \\ \tau_a & 0 \end{pmatrix},$$

with 8×8 blocks $\bar\tau_a$ (upper right) and τ_a (lower left), and $\bar\tau_{x_0} = \tau_{x_0} = I_8$. Multiplying a block matrix by the column (ψ_-, ψ_+) gives $(\bar\tau_a\psi_+, \tau_a\psi_-)$: the upper rows see only the lower half and the lower rows only the upper half. The mass term $V\Psi$, $V = m + \lambda S$, keeps each half in its own rows. So the field equation is the pair

$$\sum_a\frac{1}{f_a}\bar\tau_a\partial_a\psi_+ + 3H\bar\tau_{x_0}\psi_+ + \tfrac34\tfrac{b_4'}{b_4}\bar\tau_{x_4}\psi_+ = V\psi_-,\qquad \sum_a\frac{1}{f_a}\tau_a\partial_a\psi_- + 3H\tau_{x_0}\psi_- + \tfrac34\tfrac{b_4'}{b_4}\tau_{x_4}\psi_- = V\psi_+ .$$

The derivatives and the gravitational term of one half are balanced by the mass term of the other half: the mass term and the gravitational term both couple the two chiral halves (the coupling map of Figure 4 shows the same fact).

The evolution form. Write the explicit equation of section 16 with the x_4 term separately ($f_4 = 1$), write $M = 3H\gamma^{(x_0)} + \frac34(b_4'/b_4)\gamma^{(x_4)}$ for the gravitational term, and solve for $\partial_4\Psi$, line by line:

$$\gamma^{(x_4)}\partial_4\Psi + \sum_{a\neq4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi + M\Psi = V\Psi$$
$$\gamma^{(x_4)}\partial_4\Psi = V\Psi - \sum_{a\neq4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi - M\Psi$$
$$\partial_4\Psi = -\gamma^{(x_4)}\Big[V\Psi - \sum_{a\neq4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi - M\Psi\Big] .$$

(The part $\frac34(b_4'/b_4)\gamma^{(x_4)}$ of M gives $-\frac34(b_4'/b_4)\Psi$ on the right, as in the record's ‘evolution_form‘.)

The second line moves every other term to the right side; the third multiplies from the left by $-\gamma^{(x_4)}$ and uses $-\gamma^{(x_4)}\gamma^{(x_4)} = 1$ (because $(\gamma^{(x_4)})^2 = -1$). So the time derivative of every component is fixed by the field and its derivatives along the other seven directions at the same time x_4: the slices $x_4 = $ const are **non-characteristic** (the coefficient $\gamma^{(x_4)}$ of ∂_4 is invertible; $g^{44} = -1 \neq 0$). Scope, from the Revision scope record: this does NOT make the initial-value problem well posed. For data that depend on the extra times the growth rates of the modes have no upper bound (check extra_time_growth_rates_unbounded of Revision/theory/reports/python-scope.json).

```
The next cell reads the blocks from the formula record and compares them with the
gammas, checks $\Gamma = \mathrm{diag}(-I_8, I_8)$, and verifies the evolution form
exactly for both statistics: with $R$ the right side of the third line,
$\gamma^{(x_4)}(\partial_4\Psi - R)$ must equal the field-equation residual
$\mathcal{E} = \gamma^\mu D_\mu\Psi - (m + \lambda S)\Psi$ in all 16 components.
```

In [19]:

```
blocks = {}  # direction name -> (tau-bar_a, tau_a)
for text in record["field_equation_blocks"]:
    # the Wolfram list {"x1", {{...}}, {{...}}} is the JSON list ["x1", [[...]], [[...]]]
    name, upper, lower = json.loads(text.replace("{", "[").replace("}", "]"))
    blocks[name] = (sp.Matrix(upper), sp.Matrix(lower))
GAMMA = sp.Matrix(fixture["Gamma"])  # the chirality matrix of Revision/algebra
Z8 = sp.zeros(8, 8)
blocks_ok = sorted(blocks) == NAMES and GAMMA == sp.diag(*([-1] * 8 + [1] * 8))
for a in range(8):
    upper, lower = blocks[NAMES[a]]
    blocks_ok = (blocks_ok and G[a][:8, :8] == Z8 and G[a][8:, 8:] == Z8  # zero
                 and G[a][:8, 8:] == upper and G[a][8:, :8] == lower)  # the blocks
blocks_ok = blocks_ok and blocks["x0"][0] == sp.eye(8) and blocks["x0"][1] == sp.eye(8)
check(blocks_ok, "Gamma = diag(-I8, I8); every gamma^(a) is block off-diagonal with the "
      "blocks of the formula record; tau-bar_x0 = tau_x0 = I8",
      record=reproduces(WLREP, "block_form"))

for name, odd in STATS.items():
    data = fields[name]
    psi = data["psi"]
    V = Jet(odd, {(): m}) + data["S"].times(lam)  # m + lambda S
    others = matvec(GRAV, psi, odd)  # the gravitational term M Psi, then the sum
    for a in range(8):
        if a != X4:  # (1/f_a) gamma^(a) d_a Psi for the seven other directions
            others = [x + y for x, y in zip(others, matvec(
                G[a] / f[a], column(odd, PSI, (a,)), odd))]
    bracket = [V * psi[A] - others[A] for A in range(16)]  # the square bracket
    right = matvec(-G[X4], bracket, odd)  # R = -gamma^(x4) [ ... ]
    difference = [x - y for x, y in zip(column(odd, PSI, (X4,)), right)]  # d4 Psi - R
    left = matvec(G[X4], difference, odd)  # gamma^(x4) (d4 Psi - R)
    tag = "G" if odd else "C"
    check(all((left[A] - data["E"][A]).is_zero() for A in range(16)),
          f"{name}: the evolution form d4 Psi = R is the field equation "
          "(gamma^(x4) (d4 Psi - R) = E in all 16 components)",
          record=reproduces(WLREP, f"evolution_form_{tag}") + " (at b4 = 1)")
```

Out [19]:

```
PASS Gamma = diag(-I8, I8); every gamma^(a) is block off-diagonal with the blocks of the
    formula record; tau-bar_x0 = tau_x0 = I8
     reproduces Revision/theory/reports/wolfram-field-theory.json, check block_form
PASS grassmann: the evolution form d4 Psi = R is the field equation (gamma^(x4) (d4 Psi -
    R) = E in all 16 components)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check evolution_form_G
```

```
    (at b4 = 1)
PASS commuting: the evolution form d4 Psi = R is the field equation (gamma^(x4) (d4 Psi -
    R) = E in all 16 components)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check evolution_form_C
    (at b4 = 1)
```

Text cell:

```
## 18. Non-triviality: the gravitational term in the field equation

The difference between the field-equation residual computed with the canonical
spin connection and the one computed with $\Omega_\mu = 0$ is $\gamma^\mu\Omega_\mu
\Psi = M\Psi$, $M = 3H\gamma^{(x_0)} + \frac34(b_4'/b_4)\gamma^{(x_4)}$. Because
$\gamma^{(x_0)}$ and $\gamma^{(x_4)}$ anticommute, $MM = (9H^2 - \frac{9}{16}
(b_4'/b_4)^2)\,1$: $M\Psi$ vanishes only for $\Psi = 0$, except at times where
$|b_4'/b_4| = 4H$ (the check ‘gravity_term_square_b4‘ of
Revision/theory/reports/python-field-theory-b4.json). For a constant $b_4$, as on
the backgrounds $b_4 = 1$, $M = 3H\gamma^{(x_0)}$, a signed permutation matrix with
$(\gamma^{(x_0)})^2 = 1$, and $M\Psi$ vanishes only for $\Psi = 0$ when $H > 0$.
So in the diagonal frame and in the variables $\Psi$ the Euler-Lagrange equations
of both fields always contain a gravitational term through the spin connection (the
Revision record’s non-triviality tests [1] and [2]). Its exact scope, stated in the
Revision record: the value $3H\gamma^{(x_0)}$ belongs to this frame and these
variables (it vanishes in a frame boosted in the $(x_4, x_0)$ plane, and the
rescaling $\Psi = \sin^{-1/2}z\,\chi$ removes it), while the frame-independent
facts are that $\Omega_\mu$ itself vanishes in no frame (the metric is curved for
$H > 0$) and that the frame factors $1/f_a$ enter every derivative term. The next
cell checks the difference for both statistics.
```

In [20]:

```
for name, odd in STATS.items():
    data = fields[name]
    without = build(odd, no_connection)["E"]  # residual with Omega = 0
    gravity = matvec(GRAV, data["psi"], odd)  # M Psi
    ok = all((data["E"][A] - without[A] - gravity[A]).is_zero() for A in range(16))
    hidden = matvec(3 * H * G[X0], data["psi"], odd)  # M Psi at b4 = 1
    nonzero = all(len(hidden[A].terms) == 1 for A in range(16))
    number = "1_dirac16complex" if odd else "2_dirac16complex00"
    check(ok and nonzero, f"{name}: residual(Omega) - residual(Omega = 0) = M Psi "
          "with M = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); at b4 = 1 it is "
          "3 H gamma^(x0) Psi, nonzero in all 16 components",
          record=reproduces(WLREP, f"nontriviality_{number}") + " (at b4 = 1); "
          + reproduces(PYREP_B4, "gravity_term_square_b4"))
```

Out [20]:

```
PASS grassmann: residual(Omega) - residual(Omega = 0) = M Psi with M = 3 H gamma^(x0) +
    (3/4)(b4'/b4) gamma^(x4); at b4 = 1 it is 3 H gamma^(x0) Psi, nonzero in all 16
    components
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
```

```
    nontriviality_1_dirac16complex (at b4 = 1); Revision/theory/reports/python-field-
    theory-b4.json, check gravity_term_square_b4
PASS commuting: residual(Omega) - residual(Omega = 0) = M Psi with M = 3 H gamma^(x0) +
    (3/4)(b4'/b4) gamma^(x4); at b4 = 1 it is 3 H gamma^(x0) Psi, nonzero in all 16
    components
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    nontriviality_2_dirac16complex00 (at b4 = 1); Revision/theory/reports/python-field-
    theory-b4.json, check gravity_term_square_b4
```

Text cell:

```
## 19. An exact family of solutions

Look for solutions that depend only on the time $x_4$ and the hidden direction
$x_0$: $\Psi = \sin^\alpha z\,P(x_4)\,\chi_0$ with a constant column $\chi_0$ and a
number $\alpha$, for a constant $b_4$ (as on the backgrounds $b_4 = 1$; a changing
$b_4$ adds the term $\frac34(b_4'/b_4)\gamma^{(x_4)}\Psi$ and changes the family).
With $U = 0$ ($\lambda = 0$) the field equation becomes, line by line,

$$\gamma^{(x_4)}\partial_4\Psi + \tan z\,\gamma^{(x_0)}\partial_0\Psi + 3H
\gamma^{(x_0)}\Psi = m\Psi$$
$$\sin^\alpha z\,\big[\gamma^{(x_4)}P' + (6H\alpha + 3H)\gamma^{(x_0)}P\big]\chi_0
= m\sin^\alpha z\,P\chi_0$$
$$P' = -\gamma^{(x_4)}\big(m - 3H(2\alpha + 1)\gamma^{(x_0)}\big)P = MP,\qquad M =
-m\gamma^{(x_4)} + 3H(2\alpha + 1)\gamma^{(x_4)}\gamma^{(x_0)} .$$

The first line keeps the terms that act on a function of $x_4$ and $x_0$ (the other
derivatives vanish, which is why the result holds for EVERY $a_4$ and every
constant $b_4$). The second uses
$\tan z\,\partial_0\sin^\alpha z = \frac{\sin z}{\cos z}\,\alpha\sin^{\alpha-1}z
\cos z\cdot 6H = 6H\alpha\sin^\alpha z$. The third multiplies by
$-\gamma^{(x_4)}$ (since $(\gamma^{(x_4)})^2 = -1$). Because $\gamma^{(x_4)}$ and
$\gamma^{(x_0)}$ anticommute, $M^2 = k^2$ with $k^2 = 9H^2(2\alpha + 1)^2 - m^2$, so

$$P(x_4) = \cosh(kx_4) + \frac{\sinh(kx_4)}{k}M$$

solves $P' = MP$ with $P(0) = 1$. Moreover $M^TC + CM = 0$, so $S = \Psi^\dagger C
\Psi = \sin^{2\alpha}z\,\chi_0^\dagger C\chi_0$ does not change with $x_4$. The
next cell verifies the family exactly, writing $\cosh(kx_4)$ and $\sinh(kx_4)$ as
symbols ‘ch‘, ‘sh‘ with $\partial_4\,\mathrm{ch} = k\,\mathrm{sh}$, $\partial_4\,
\mathrm{sh} = k\,\mathrm{ch}$, and $\sin^\alpha z = s^{6\alpha}$; then it checks the
matrix identity behind the growing modes of the Revision scope record.
```

In [21]:

```
al, k, ch, sh = sp.symbols("alpha k ch sh", real=True)
chi0 = sp.Matrix(sp.symbols("q1:17"))  # an arbitrary constant column
M = -m * G[X4] + 3 * H * (2 * al + 1) * G[X4] * G[X0]
k_squared = 9 * H**2 * (2 * al + 1) ** 2 - m**2
check((M * M - k_squared * I16).applyfunc(sp.expand) == Z16
      and (M.T * C + C * M).applyfunc(sp.expand) == Z16,
      "M^2 = k^2 with k^2 = 9 H^2 (2 alpha + 1)^2 - m^2, and M^T C + C M = 0")
```

```
def d4(v):  # derivative along x4 with ch' = k sh, sh' = k ch
    return v.diff(ch) * k * sh + v.diff(sh) * k * ch


def d0(v):  # derivative along x0 in the ring (s^(6 alpha) included)
    return v.diff(s) * H * c / s**5 + v.diff(c) * (-6 * H * s**6)


Psi = s ** (6 * al) * (ch * I16 + sh / k * M) * chi0  # sin^alpha z P(x4) chi0
residual = (G[X4] * Psi.applyfunc(d4) + (s**6 / c) * G[X0] * Psi.applyfunc(d0)
            + 3 * H * G[X0] * Psi - m * Psi)


def vanishes(v):
    """Multiply by k and divide by sin^alpha z = s^(6 alpha), multiply out, use
    k^2 = 9 H^2 (2 alpha + 1)^2 - m^2: is the result zero?"""
    v = sp.expand(v * k / s ** (6 * al))
    return sp.expand(v.subs(k**2, k_squared)) == 0


without_term = residual - 3 * H * G[X0] * Psi  # the same without 3 H gamma^(x0)
check(all(vanishes(v) for v in residual)
      and not all(vanishes(v) for v in without_term),
      "Psi = sin^alpha z (cosh k x4 + sinh k x4 / k M) chi0 solves the field "
      "equation for every alpha, every a4 and constant b4 (and fails without "
      "3 H gamma^(x0))",
      record=reproduces(PYREP, "exact_solution_family_x4_x0"))
A_matrix = -sp.I * m * G[X4] + 3 * sp.I * H * G[X4] * G[X0]
check((A_matrix * A_matrix - (m**2 - 9 * H**2) * I16).applyfunc(sp.expand) == Z16,
      "the mode matrix A = -i m gamma^(x4) + 3 i H gamma^(x4) gamma^(x0) has A^2 = "
      "(m^2 - 9 H^2) I16", record=reproduces(
          SCOPE, "good_sector_x0_independent_modes_without_boundary_condition"))
```

Out [21]:

```
PASS M^2 = k^2 with k^2 = 9 H^2 (2 alpha + 1)^2 - m^2, and M^T C + C M = 0
PASS Psi = sin^alpha z (cosh k x4 + sinh k x4 / k M) chi0 solves the field equation for
    every alpha, every a4 and constant b4 (and fails without 3 H gamma^(x0))
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_solution_family_x4_x0
PASS the mode matrix A = -i m gamma^(x4) + 3 i H gamma^(x4) gamma^(x0) has A^2 = (m^2 - 9
    H^2) I16
     reproduces Revision/theory/reports/python-scope.json, check
    good_sector_x0_independent_modes_without_boundary_condition
```

Text cell:

```
Two members of the family, with the illustration values $H = 1$, $m = 2$ and the
real column $\chi_0 = e_1 + e_5$ (components 1 and 5 equal to 1, all others 0; it
has $\chi_0^T C\chi_0 = 2C_{1,5} = -2$):
```

```
- $\alpha = 0$: $k^2 = 9 - 4 = 5$, so $P$ grows like $e^{\sqrt5\,x_4}$. This is
  the $x_0$-independent growing mode that the Revision scope record finds without
  a boundary condition at $z = \pi/2$ whenever $m^2 < 9H^2$ (the mode matrix
  $A$ above has $A^2 = (m^2 - 9H^2) = -5$, imaginary frequencies $\pm i\sqrt5$).
- $\alpha = -\frac12$: $2\alpha + 1 = 0$, $M = -m\gamma^{(x_4)}$ and $k^2 = -m^2 =
  -4$, so $\cosh(kx_4) = \cos 2x_4$ and $\sinh(kx_4)/k = \sin(2x_4)/2$: the
  solution oscillates. This is the rescaling $\Psi = \sin^{-1/2}z\,\chi$ of the
  Revision record, in which the term $3H\gamma^{(x_0)}$ disappears.

The next cell evaluates both with numpy at $z = \pi/4$ for $x_4$ from 0 to 2, and
the scalar density $S = \Psi^T C\Psi$, which must stay constant (Figure 5).
```

In [22]:

```
Gn = [np.array(gm.tolist(), dtype=float) for gm in G]  # the gammas as float arrays
Cn = np.array(C.tolist(), dtype=float)
chi_n = np.zeros(16)
chi_n[[0, 4]] = 1.0  # e_1 + e_5
zeta = np.pi / 4  # the point z = pi/4 of the hidden direction
xs4 = np.linspace(0.0, 2.0, 201)


def member(alpha_value, mass=2.0, h=1.0):
    """Psi(x4) at z = pi/4 for the family member alpha (columns = times)."""
    Mn = -mass * Gn[X4] + 3 * h * (2 * alpha_value + 1) * Gn[X4] @ Gn[X0]
    ksq = 9 * h**2 * (2 * alpha_value + 1) ** 2 - mass**2
    kk = np.sqrt(complex(ksq))  # real for ksq > 0, imaginary for ksq < 0
    values = [(np.cosh(kk * t) * np.eye(16) + np.sinh(kk * t) / kk * Mn) @ chi_n
              for t in xs4]
    return np.sin(zeta) ** alpha_value * np.real(np.array(values).T)


growing, oscillating = member(0.0), member(-0.5)
S_growing = np.einsum("it,ij,jt->t", growing, Cn, growing)  # Psi^T C Psi
S_oscillating = np.einsum("it,ij,jt->t", oscillating, Cn, oscillating)
report("S of the alpha = 0 member", f"{S_growing[0]:.6f}")
report("S of the alpha = -1/2 member", f"{S_oscillating[0]:.6f}")
check(np.allclose(S_growing, -2.0, atol=1e-9 * np.abs(growing).max() ** 2)
      and np.allclose(S_oscillating, -2.0 / np.sin(zeta), atol=1e-9),
      "S is constant along x4 for both members (-2 and -2/sin z)")
```

Out [22]:

```
RESULT S of the alpha = 0 member = -2.000000
RESULT S of the alpha = -1/2 member = -2.828427
PASS S is constant along x4 for both members (-2 and -2/sin z)
```

Text cell:

```
The next cell draws Figure 5: the nonzero components of the two members against
$x_4$ (the growing one on a logarithmic axis), and the constant scalar density.
```

In [23]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.6))
for A in range(16):
    if np.abs(growing[A]).max() > 1e-12:  # draw only the nonzero components
        axes[0].semilogy(xs4, np.abs(growing[A]), label=f"$|\\Psi_{{{A + 1}}}|$")
    if np.abs(oscillating[A]).max() > 1e-12:
        axes[1].plot(xs4, oscillating[A], label=f"$\\Psi_{{{A + 1}}}$")
axes[0].set_title("$\\alpha = 0$: growth like $e^{\\sqrt{5}x_4}$")
axes[1].set_title("$\\alpha = -1/2$: oscillation")
axes[2].plot(xs4, S_growing, label="$\\alpha = 0$")
axes[2].plot(xs4, S_oscillating, "--", label="$\\alpha = -1/2$")
axes[2].set_ylim(-3.5, 0.5)
axes[2].set_title("scalar density $S = \\Psi^T C\\Psi$")
for ax in axes:
    ax.set_xlabel("time $x_4$")
    ax.legend(fontsize=7)
axes[0].set_ylabel("value at $z = \\pi/4$")
fig.tight_layout()
save_figure(fig, "exact_solutions",
            "Two exact solutions $\\Psi = \\sin^\\alpha z\\,(\\cosh kx_4 + "
            "\\sinh(kx_4)/k\\,M)\\chi_0$ of the field equation with $\\lambda = 0$, "
            "for the illustration values $H = 1$, $m = 2$, $\\chi_0 = e_1 + e_5$, "
            "at $z = \\pi/4$, against the time $x_4$ from 0 to 2 (pure numbers). "
            "Left: $\\alpha = 0$, $k = \\sqrt{5}$; the nonzero components grow "
            "exponentially (logarithmic axis), the growing mode that exists without "
            "a boundary condition at $z = \\pi/2$ when $m^2 < 9H^2$. Middle: "
            "$\\alpha = -1/2$, $k = 2i$; the components oscillate. Right: the "
            "scalar density $S$ of both stays exactly constant ($-2$ and "
            "$-2/\\sin z$), because $M^TC + CM = 0$. The solutions hold for every "
            "$a_4(x_4)$.")
```

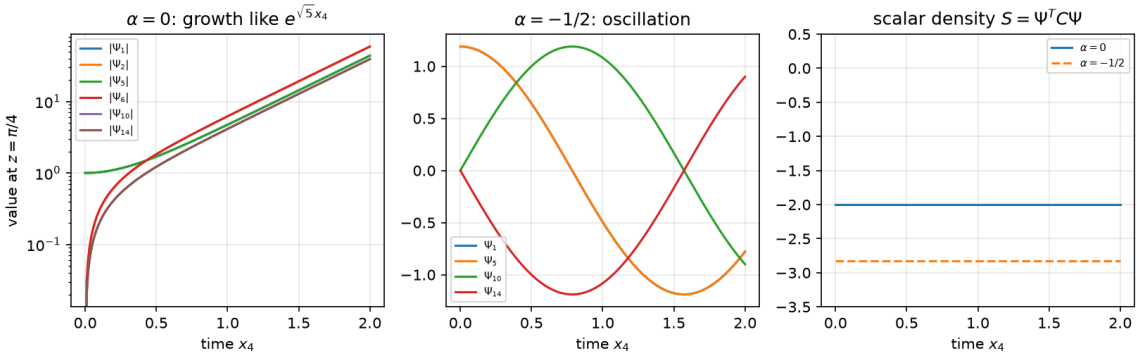

Figure 223: Two exact solutions $\Psi = \sin^\alpha z\,(\cosh kx_4 + \sinh(kx_4)/k\,M)\chi_0$ of the field equation with $\lambda = 0$, for the illustration values $H = 1$, $m = 2$, $\chi_0 = e_1 + e_5$, at $z = \pi/4$, against the time x_4 from 0 to 2 (pure numbers). Left: $\alpha = 0$, $k = \sqrt{5}$; the nonzero components grow exponentially (logarithmic axis), the growing mode that exists without a boundary condition at $z = \pi/2$ when $m^2 < 9H^2$. Middle: $\alpha = -1/2$, $k = 2i$; the components oscillate. Right: the scalar density S of both stays exactly constant (-2 and $-2/\sin z$), because $M^TC + CM = 0$. The solutions hold for every $a_4(x_4)$. (Notebook 07b, figure 5.)

Out [23]:

```
Figure 07b.5 saved as Revision/textbook/figures/07b_5_exact_solutions.png
```

Text cell:

```
Finally a numerical view of non-triviality. The next cell evaluates the residual
$\gamma^{(x_4)}\partial_4\Psi + \tan z\,\gamma^{(x_0)}\partial_0\Psi + 3H\gamma^{(x_0)}
\Psi - m\Psi$ of the growing member ($\alpha = 0$, so $\partial_0\Psi = 0$) with the
exact derivative $\partial_4\Psi = (k\sinh(kx_4) + \cosh(kx_4)M)\chi_0$, once for
the full equation and twice for wrong equations: without the spin-connection term
$3H\gamma^{(x_0)}\Psi$, and with the wrong sign of the mass. The full equation is
satisfied to rounding (about $10^{-16}$ of the size of $\Psi$); the wrong ones fail
by amounts that grow with $\Psi$ (Figure 6).
```

In [24]:

```
Mn = -2.0 * Gn[X4] + 3.0 * Gn[X4] @ Gn[X0]  # alpha = 0, m = 2, H = 1
kk = np.sqrt(5.0)
psi_t = np.array([(np.cosh(kk * t) * np.eye(16) + np.sinh(kk * t) / kk * Mn) @ chi_n
                  for t in xs4])
dpsi_t = np.array([(kk * np.sinh(kk * t) * np.eye(16) + np.cosh(kk * t) * Mn) @ chi_n
                   for t in xs4])
full = np.array([Gn[X4] @ dp + 3.0 * Gn[X0] @ p - 2.0 * p
                 for p, dp in zip(psi_t, dpsi_t)])
no_gravity = np.array([Gn[X4] @ dp - 2.0 * p for p, dp in zip(psi_t, dpsi_t)])
wrong_mass = np.array([Gn[X4] @ dp + 3.0 * Gn[X0] @ p + 2.0 * p
                       for p, dp in zip(psi_t, dpsi_t)])
size = np.abs(psi_t).max(axis=1)  # the size of Psi at each time
relative_full = np.abs(full).max(axis=1) / size
check(relative_full.max() < 1e-12, "the full field equation holds to rounding "
      "(relative residual below 1e-12 at all 201 times)")
check(np.allclose(np.abs(no_gravity).max(axis=1), 3.0 * size),
      "without the term 3 H gamma^(x0) Psi the residual is 3 H |Psi|")

fig, ax = plt.subplots()
ax.semilogy(xs4, np.abs(no_gravity).max(axis=1),
            label="without $3H\\gamma^{(x_0)}\\Psi$")
ax.semilogy(xs4, np.abs(wrong_mass).max(axis=1), "--", label="mass $m \\to -m$")
ax.semilogy(xs4, size, ":", color="black", label="size of $\\Psi$")
ax.set_xlabel("time $x_4$")
ax.set_ylabel("largest component of the residual")
ax.set_title("The exact solution fails the wrong equations")
ax.legend(fontsize=8)
save_figure(fig, "residuals",
            "The largest component of the residual of the growing exact solution "
            "($\\alpha = 0$, $H = 1$, $m = 2$, $\\chi_0 = e_1 + e_5$) against the "
            "time $x_4$ (logarithmic vertical axis), when it is put into two WRONG "
            "equations: the field equation without the spin-connection term "
            "$3H\\gamma^{(x_0)}\\Psi$ (solid; it equals $3H$ times the largest "
            "component of $\\Psi$, dotted) and the equation with the wrong sign of "
```

```
        "the mass (dashed). In the correct equation the residual is zero up to "
        "rounding (relative size below $10^{-12}$, not drawn on this axis). "
        "The gravitational term is needed for the solution.")
```

Out [24]:

```
PASS the full field equation holds to rounding (relative residual below 1e-12 at all 201
    times)
PASS without the term 3 H gamma^(x0) Psi the residual is 3 H |Psi|
```

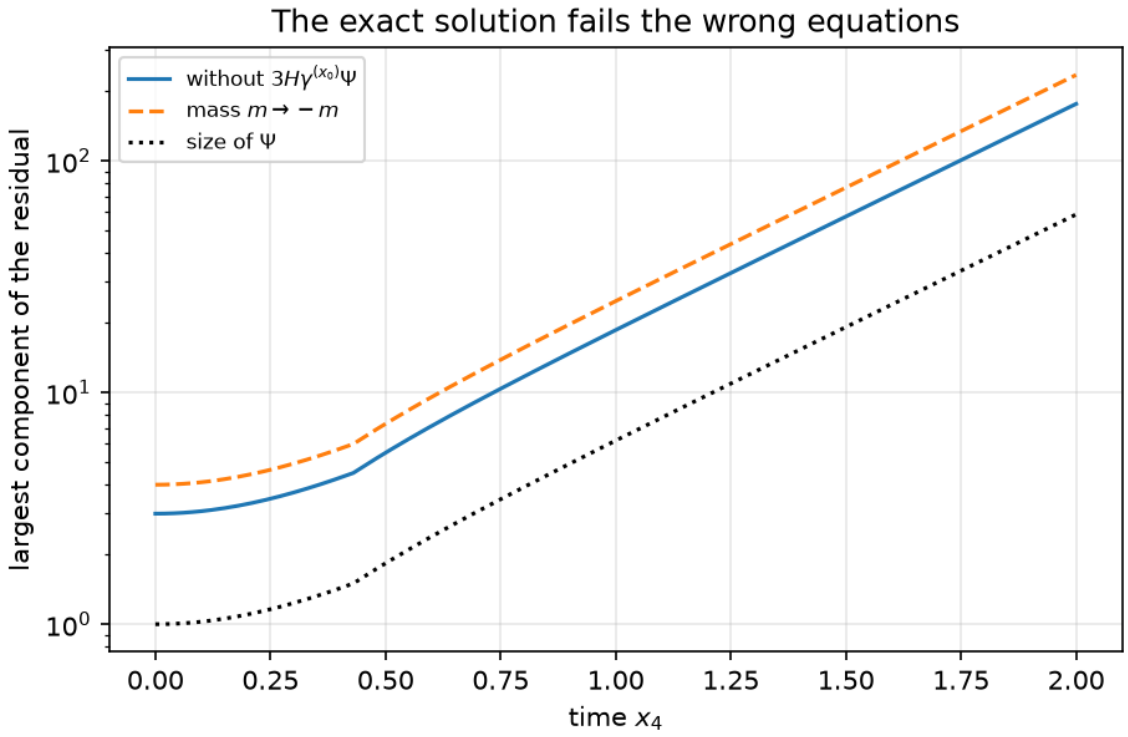

Figure 224: The largest component of the residual of the growing exact solution ($\alpha = 0$, $H = 1$, $m = 2$, $\chi_0 = e_1 + e_5$) against the time x_4 (logarithmic vertical axis), when it is put into two WRONG equations: the field equation without the spin-connection term $3H\gamma^{(x_0)}\Psi$ (solid; it equals $3H$ times the largest component of Ψ, dotted) and the equation with the wrong sign of the mass (dashed). In the correct equation the residual is zero up to rounding (relative size below 10^{-12}, not drawn on this axis). The gravitational term is needed for the solution. (Notebook 07b, figure 6.)

Out [24]:

```
Figure 07b.6 saved as Revision/textbook/figures/07b_6_residuals.png
```

Text cell:

```
## 20. The last check
```

The last cell checks that the six figure files exist and prints the number of checks that passed.

In [25]:

```
names = ["scale_factors", "gamma_omega_per_direction", "lagrangian_monomials",
         "coupling_map", "exact_solutions", "residuals"]
files = [f"{FIGURE_FOLDER}/07b_{k}_{name}.png" for k, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

Out [25]:

```
PASS all 6 figure files of this notebook exist
ALL 47 CHECKS PASSED (notebook 07b)
```

Text cell:

```
## 21. What this notebook showed

- The diagonal frame $f = (\cot z,\ e^{a_4}\sin^{1/6}z\ (\times3),\ 1,\
  e^{-a_4}b_4^{1/2}\sin^{1/6}z\ (\times3))$ gives exactly the author's metric with
  $b_4$, with $\sqrt{|g|} = b_4^{3/2}\cos z$: the inflation of 3-space and the
  deflation of the extra times compensate, and at $b_4 = 1$ the volume factor
  $\cos z$ does not depend on time.
- The metric has 25 independent nonzero Christoffel symbols; the canonical spin
  connection has 12 nonzero components, each a rate ($a_4'$, $a_4' -
  b_4'/(2b_4)$ or $H$) times a nonvanishing factor. Both agree with the formula
  record with $b_4$ and, at $b_4 = 1$, with the record of $b_4 = 1$.
- Direction by direction, $\gamma^\mu\Omega_\mu$ has a time part $\pm\frac12a_4'
  \gamma^{(x_4)}$ that cancels between the three inflating and the three deflating
  directions (the extra times leave $\frac34(b_4'/b_4)\gamma^{(x_4)}$), and a hidden
  part $\frac12H\gamma^{(x_0)}$ that adds up: $\gamma^\mu\Omega_\mu =
  3H\gamma^{(x_0)} + \frac34(b_4'/b_4)\gamma^{(x_4)}$, which is $3H\gamma^{(x_0)}$
  for every $a_4$ at $b_4 = 1$.
- The Lagrangian of both fields is real, differs from the unsymmetrised form by a
  total divergence, and in this metric does not contain the spin connection at all
  ($\{\gamma^\mu, \Omega_\mu\} = 0$); written in jets it has 392 monomials for
  dirac16complex and 408 for dirac16complex00, the difference being the 16 squares
  that vanish for Grassmann numbers.
- Its Euler-Lagrange equations are the same for both statistics:
  $\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi$ and its Dirac conjugate. In the
  author's metric the 16 component equations contain the frame factors $\tan z$,
  $e^{-a_4}\sin^{-1/6}z$, $1$, $e^{a_4}b_4^{-1/2}\sin^{-1/6}z$ and the
  gravitational term $M\Psi$, $M = 3H\gamma^{(x_0)} +
  \frac34(b_4'/b_4)\gamma^{(x_4)}$; they agree term by term with the Revision
  formula record with $b_4$, and at $b_4 = 1$ with the record of $b_4 = 1$. Each
  equation couples four components across the chiral halves: every gamma is block
  off-diagonal, and the mass term and the gravitational term both couple the halves
  $\psi_-$ and $\psi_+$. $M\Psi$ vanishes only for $\Psi = 0$, except at times
```

```
  where $|b_4'/b_4| = 4H$; for a constant $b_4$ never.
- Solved for the time derivative, $\partial_4\Psi = -\gamma^{(x_4)}[V\Psi -
  \sum_{a\neq4}f_a^{-1}\gamma^{(a)}\partial_a\Psi - M\Psi]$: the slices $x_4 = $
  const are non-characteristic, which does not by itself make the initial-value
  problem well posed (the extra-time growth rates are unbounded).
- The family $\Psi = \sin^\alpha z(\cosh kx_4 + \sinh(kx_4)/k\,M)\chi_0$ solves the
  equations exactly for every $a_4$ and every constant $b_4$, with constant $S$; for
  $\alpha = 0$ and $m^2
  < 9H^2$ it grows exponentially, and it fails the equation from which the
  spin-connection term is removed.
- Every statement that the Revision record also contains was reproduced; the scope
  of the gravitational term (frame and variable dependence) is the one stated in
  the Revision record.
```

7.27 Line-by-line walk-through of Notebook 07b

The notebook has 25 code cells, In [1] to In [25]. This section explains every line of every one of them, with the conventions of Section 7.9; every caption is printed in full under its figure in Section 7.26.

In [1], the set-up cell. It is the set-up cell of Notebook 07d with the one line `NOTEBOOK_ID = "07b"`; its first part repeats the complete run instructions of Section 7.25 as comment lines, and its code is explained line by line in Section 7.9. **Out [1]:** `Set-up of notebook 07b complete: repository folder found, helpers defined.`

In [2]: the gammas and C from the Revision record.

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import re  # regular expressions: used to read the Wolfram formula record

import numpy as np  # numbers for the plots
import sympy as sp  # exact algebra

check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece
```

The modules (as in Notebook 07d; `re` is used here to read formulas and numbers from the record's texts) and the same wrapper of `check` that prints a PASS line and its "reproduces" line in one piece (Section 7.9, In [2]).

```
GAMMAS = "Revision/algebra/gammas.json"
FORMULAS = "Revision/theory/field-theory.json"  # the formulas at b4 = 1
```

```
FORMULAS_B4 = "Revision/theory/field-theory-b4.json"  # the formulas with b4(x4)
PYREP_B4 = "Revision/theory/reports/python-field-theory-b4.json"
PYREP = "Revision/theory/reports/python-field-theory.json"
WLREP = "Revision/theory/reports/wolfram-field-theory.json"
SCOPE = "Revision/theory/reports/python-scope.json"
```

The repository paths of the five Revision files the notebook reads: the gammas, the formula record of the Wolfram verifier, the sympy and the Wolfram field-theory reports, and the sympy scope report.

```
def revision_check(report, name):
    """The check called name of the Revision report (a JSON file): a dictionary
    with its name, verdict and detail; stops if it is missing or not PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return found[0]


def reproduces(report, name):
    """The text "<report>, check <name>" for a PASS line, after making sure that
    the Revision report contains the check name with the verdict PASS."""
    revision_check(report, name)
    return f"{report}, check {name}"
```

`revision_check` returns the entry of a named check of a report and stops if it is missing or does not pass; `reproduces` uses it and returns the text for the "reproduces" line. Both are explained in Section 7.9 (In [2] of Notebook 07d).

```
fixture = json.loads(repository_file(GAMMAS).read_text(encoding="utf-8"))
G = [sp.Matrix(g) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7), integers
C = sp.Matrix(fixture["C"])  # the author's sigma16
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # eta in the order x0 .. x7
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
SAB = [[(G[a] * G[b] - G[b] * G[a]) / 4 for b in range(8)] for a in range(8)]
X0, X4 = 0, 4  # list positions of the hidden direction x0 and of the time x4
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
```

The gammas are read as exact sympy matrices: `G[0]` to `G[7]` are $\gamma^{(x_0)}$ to $\gamma^{(x_7)}$, `C` is C. `ETA` is η in the order $x_0, \ldots, x_7$; `I16` and `Z16` are the 16×16 identity and zero matrices. `SAB[a][b]` is the spin generator $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$, built for all 64 pairs with a nested list comprehension. `X0, X4 = 0, 4` are the list positions of x_0 and x_4 (positions count from 0, so position k is the direction x_k), and `NAMES` the printed names of the eight coordinates. `FORMULAS_B4` and `PYREP_B4` name the formula record and the sympy report of the metric with b_4.

```
clifford = all(G[a] * G[b] + G[b] * G[a] == 2 * (ETA[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
check(clifford, "{gamma^a, gamma^b} = 2 eta^ab I16 for all 64 pairs",
```

```
      record=reproduces(PYREP, "clifford_relations"))
c_ok = (C == G[0] * G[1] * G[2] * G[3] and C.T == C and C * C == I16
        and all((C * g).T == -(C * g) for g in G))
check(c_ok, "C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3), symmetric, C^2 = 1, "
      "every C gamma^a antisymmetric", record=reproduces(PYREP, "C_properties"))
```

`clifford` is true when $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab} I_{16}$ for all 64 pairs (the expression `(ETA[a] if a == b else 0)` is η^{ab}). `c_ok` checks $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, $C^T = C$, $C^2 = 1$ and that every $C\gamma^{(a)}$ is antisymmetric. **Out [2]:** two PASS lines with their "reproduces" lines (`clifford_relations` and `C_properties` of the sympy report).

In [3]: the coefficient ring.

```
E, s, c = sp.symbols("E s c", positive=True)  # e^a4, sin(z)^(1/6), cos z
A1, A2, A3 = sp.symbols("A1 A2 A3", real=True)  # 1st, 2nd, 3rd derivative of a4
b4 = sp.Symbol("b4", positive=True)  # the function b4(x4) > 0 of the metric
b4p, b4pp, b4ppp = sp.symbols("b4p b4pp b4ppp", real=True)  # its 1st to 3rd derivative
AT_B4_1 = {b4: 1, b4p: 0, b4pp: 0, b4ppp: 0}  # the background b4 = 1
H = sp.Symbol("H", positive=True)  # the author's constant
m, lam = sp.symbols("m lambda", real=True)  # the mass and the coupling
```

The symbols of Section 7.18: `E`, `s`, `c` (positive), `A1`, `A2`, `A3` (the first three derivatives of a_4), `b4` (positive) and `b4p`, `b4pp`, `b4ppp` (the first three derivatives of b_4), `H` (positive), and `m`, `lam` (λ; the name `lambda` is reserved in Python). `AT_B4_1` is the substitution that puts in the background $b_4 = 1$: with `.subs(AT_B4_1)` a result for a general b_4 becomes the result at $b_4 = 1$, which is compared with the record of $b_4 = 1$.

```
def cd(expression, mu):
    """The derivative of a ring expression along coordinate mu (0 .. 7)."""
    expression = sp.sympify(expression)
    if mu == X4:
        return (sp.diff(expression, E) * E * A1 + sp.diff(expression, A1) * A2
                + sp.diff(expression, A2) * A3 + sp.diff(expression, b4) * b4p
                + sp.diff(expression, b4p) * b4pp + sp.diff(expression, b4pp) * b4ppp)
    if mu == X0:
        return (sp.diff(expression, s) * H * c / s**5
                + sp.diff(expression, c) * (-6 * H * s**6))
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7
```

`cd(expression, mu)` differentiates a ring expression along the coordinate at list position `mu`. Along x_4: by the chain rule, through E ($\partial_4 E = EA_1$), through A_1 ($\partial_4 A_1 = A_2$), through A_2 ($\partial_4 A_2 = A_3$), and in the same way through b_4, b_4' and b_4''. Along x_0: through s ($\partial_0 s = Hc/s^5$) and through c ($\partial_0 c = -6Hs^6$). Along every other coordinate: 0.

```
def is_zero(expression):
    """Exact test: the numerator vanishes after c^2 -> 1 - s^12."""
    numerator, _ = sp.fraction(sp.together(sp.sympify(expression)))
    numerator = sp.expand(numerator)
    if numerator == 0:
```

```
        return True
    remainder = sp.rem(sp.Poly(numerator, c), sp.Poly(c**2 - 1 + s**12, c))
    return sp.expand(remainder.as_expr()) == 0
```

`is_zero(expression)` decides exactly whether a ring expression is zero. `sp.together` brings it to one fraction and `sp.fraction` returns its numerator and denominator; the numerator is multiplied out. If it is not already 0, it is divided as a polynomial in c by $c^2 - 1 + s^{12}$ (`sp.rem` gives the remainder of the division); the remainder no longer contains c^2, and it vanishes exactly when the expression is zero.

```
x4, x0 = sp.symbols("x4 x0", real=True)
a4 = sp.Function("a4")(x4)  # the free function of the metric
b4f = sp.Function("b4")(x4)  # the second function of the metric
z = 6 * H * x0


def to_physical(expression):
    """Ring symbols -> the functions they stand for."""
    return sp.sympify(expression).subs(
        {E: sp.exp(a4), s: sp.sin(z) ** sp.Rational(1, 6), c: sp.cos(z),
         A1: a4.diff(x4), A2: a4.diff(x4, 2), A3: a4.diff(x4, 3),
         b4: b4f, b4p: b4f.diff(x4), b4pp: b4f.diff(x4, 2), b4ppp: b4f.diff(x4, 3)})
```

The physical variables: `x4`, `x0`, the unknown functions $a_4(x_4)$ and $b_4(x_4)$ (`b4f`) and $z = 6Hx_0$. `to_physical` replaces the ring symbols by the functions they stand for.

```
rules_ok = all(
    sp.simplify(to_physical(cd(v, mu)) - sp.diff(to_physical(v), variable)) == 0
    for v in (E, s, c, A1, A2, b4, b4p, b4pp)
    for mu, variable in ((X4, x4), (X0, x0)))
check(rules_ok, "the derivative rules of the ring agree with sympy's derivatives")
check(is_zero(c**2 + s**12 - 1) and not is_zero(c) and not is_zero(s - c),
      "the exact zero test: c^2 + s^12 - 1 is zero, c and s - c are not")
```

The first check compares, for the five symbols E, s, c, A_1, A_2 and the two coordinates, the ring derivative put back into functions with sympy's own derivative of the functions. The second checks the zero test on three examples: $c^2 + s^{12} - 1$ is zero, c and $s - c$ are not. **Out [3]:** two PASS lines.

In [4]: the metric and the volume factor.

```
f = [c / s**6] + [E * s] * 3 + [sp.Integer(1)] + [s * sp.sqrt(b4) / E] * 3  # frame
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal metric entries
sqrt_g = sp.prod(f)  # the volume factor
third = sp.sin(z) ** sp.Rational(1, 3)
author = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * third] * 3 + [-1]
          + [-sp.exp(-2 * a4) * b4f * third] * 3)  # the author's metric with b4
check(all(sp.simplify(to_physical(g[a]) - author[a]) == 0 for a in range(8)),
      "the diagonal frame reproduces the author's metric with b4 entry by entry",
      record=reproduces(PYREP, "metric_from_vielbein_equals_SPEC") + " (at b4 = 1)")
say(f"sqrt|g| = {sqrt_g} (ring) = {to_physical(sqrt_g)}")
```

```
check(sqrt_g == b4 ** sp.Rational(3, 2) * c and sqrt_g.subs(AT_B4_1) == c,
      "sqrt|g| = b4^(3/2) cos z; cos z, independent of x4, at b4 = 1",
      record=reproduces(WLREP, "sqrt_det_g_is_cos_z") + " (at b4 = 1)")
```

`f` is the list of the eight frame factors in the order $x_0, \dots, x_7$ (`[E * s] * 3` repeats a one-entry list three times), `g` the diagonal metric entries $\eta_{aa} f_a^2$, and `sqrt_g` their product $\sqrt{|g|}$ (`sp.prod`). `author` is the author's metric with b_4, typed from the author's notebook `Dirac_Einstein-Lovelock_4+4_Nash.nb` (Chapter 3). The first check compares the eight entries; the `say` line prints the volume factor in the ring and as a function; the second check requires it to be $b_4^{3/2} c$, and c at $b_4 = 1$. **Out [4]:** two PASS lines with "reproduces" lines and `sqrt|g| = b4**(3/2)*c (ring) = b4(x4)**(3/2)*cos(6*H*x0)`.

In [5]: reading the formula record.

```
record = {item["key"]: item["wl"] for item in
          json.loads(repository_file(FORMULAS).read_text(encoding="utf-8"))["formulas"]}
record_b4 = {item["key"]: item["wl"] for item in json.loads(
    repository_file(FORMULAS_B4).read_text(encoding="utf-8"))["formulas"]}
a4x, A1w = sp.symbols("a4x A1w", real=True)  # a4(x4) and a4'(x4) in the record
b4x = sp.Symbol("b4x", positive=True)  # b4(x4) in the record
B1w, B2w = sp.symbols("B1w B2w", real=True)  # b4'(x4) and b4''(x4) in the record
```

`record` is a dictionary from the keys of the formula record `Revision/theory/field-theory.json` (the metric at $b_4 = 1$) to their Wolfram-language texts (the field `wl` of each entry), and `record_b4` the same for `Revision/theory/field-theory-b4.json` (the metric with b_4). `a4x` and `A1w` are symbols for $a_4(x_4)$ and $a_4'(x_4)$ as they appear in those texts, `b4x`, `B1w` and `B2w` for $b_4(x_4)$, $b_4'(x_4)$ and $b_4''(x_4)$.

```
def from_wolfram(text):
    """Read a Wolfram InputForm text of the formula record into sympy."""
    t = text.replace("Derivative[1][a4][x4]", "A1w").replace("a4[x4]", "a4x")
    t = t.replace("Derivative[1][b4][x4]", "B1w").replace("Derivative[2][b4][x4]", "B2w")
    t = t.replace("b4[x4]", "b4x")
    t = re.sub(r'dd\["x(\d)",\s*Psi\[(\d+)\]\]', r"d\1_\2", t)  # d_a Psi_B
    t = re.sub(r"Psi\[(\d+)\]", r"P\1", t)  # Psi_B
    for name in ("Sin", "Cos", "Tan", "Cot", "Sec", "Csc", "Sqrt"):
        t = t.replace(name + "[", name.lower() + "(")
    t = t.replace("[", "(").replace("]", ")").replace("{", "[").replace("}", "]")
    t = t.replace("^", "**")
    names = {"E": sp.E, "H": H, "x0": x0, "a4x": a4x, "A1w": A1w, "b4x": b4x,
             "B1w": B1w, "B2w": B2w}
    for symbol in set(re.findall(r"\b(d\d_\d+|P\d+)\b", t)):
        names[symbol] = sp.Symbol(symbol)
    return sp.sympify(t, locals=names)
```

`from_wolfram(text)` turns a Wolfram-language text into a sympy expression in five steps: the derivatives and the functions a_4 and b_4 are replaced by the symbols `A1w`, `a4x`, `B1w`, `B2w` and `b4x`; a regular expression replaces each derivative `dd["x5", Psi[15]]` by the name `d5_15` (the pattern `(\d)` captures a digit and `\1`, `\2` put the captured parts back); each `Psi[9]` becomes `P9`; the

function names `Sin` ... `Csc` and `Sqrt` become `sin` ... `csc` and `sqrt`; square brackets become round ones, braces (Wolfram lists) square ones (Python lists), and `^` becomes `**`. The dictionary `names` tells sympy what each name means; the loop adds a plain symbol for every derivative name and component name found in the text (`set(...)` removes repetitions). `sp.sympify` reads the result.

```
RING = {sp.sin(z): s**6, sp.cos(z): c, sp.tan(z): s**6 / c, sp.cot(z): c / s**6,
        sp.sec(z): 1 / c, sp.csc(z): 1 / s**6, sp.csc(2 * z): 1 / (2 * s**6 * c)}


def to_ring(expression):
    """A record formula in the ring symbols E, s, c, A1, b4, b4p, b4pp."""
    return (sp.sympify(expression).subs(a4x, sp.log(E)).subs(A1w, A1)
            .subs({b4x: b4, B1w: b4p, B2w: b4pp}).subs(RING))
```

`RING` replaces the trigonometric functions of z by ring expressions: $\sin z = s^6$, $\cos z = c$, $\tan z = s^6/c$, $\cot z = c/s^6$, $\sec z = 1/c$, $\csc z = 1/s^6$ and $\csc 2z = 1/(2s^6c)$ (because $\sin 2z = 2\sin z \cos z$). `to_ring` puts a record formula into the ring: $a_4 \to \ln E$ (so $e^{a_4} \to E$), $a_4' \to A_1$, and the replacements of `RING`.

```
frame_record = from_wolfram(record["vielbein_diagonal"])
say(f"record vielbein_diagonal in the ring: {[to_ring(v) for v in frame_record]}")
check(all(is_zero(to_ring(v) - f[a].subs(AT_B4_1)) for a, v in enumerate(frame_record))
      and is_zero(to_ring(from_wolfram(record["sqrt_det_g"])) - sqrt_g.subs(AT_B4_1)),
      "at b4 = 1 the frame factors and sqrt|g| equal the formula record",
      record=f"{FORMULAS}, formulas vielbein_diagonal and sqrt_det_g")
```

The record's frame factors (key `vielbein_diagonal`, a record of $b_4 = 1$) are put into the ring and printed; the check compares each with our `f[a]` at $b_4 = 1$ and the record's volume factor with ours. The `record=` text names the record file and the two formula keys. **Out [5]:** `record vielbein_diagonal in the ring: [c/s**6, E*s, E*s, E*s, 1, s/E, s/E, s/E]` and the PASS line with its "reproduces" line.

In [6]: Figure 07b.1.

```
times = np.linspace(0.0, 3.0, 301)  # x4 from 0 to 3 (H = 1, A = 1: a4 = x4)
warp = np.sin(np.pi / 4) ** (1 / 6)  # sin^(1/6) z at z = pi/4
angles = np.linspace(0.01, np.pi / 2 - 0.01, 300)  # z inside (0, pi/2)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
left.semilogy(times, np.exp(times) * warp, label="3-space $e^{a_4}\\sin^{1/6}z$")
left.semilogy(times, np.exp(-times) * warp, "--",
              label="extra times $e^{-a_4}\\sin^{1/6}z$")
left.semilogy(times, np.exp(times) * np.exp(-times) * warp**2, ":", color="black",
              label="their product")
left.set_xlabel("time $x_4$ (with $a_4 = x_4$)")
left.set_ylabel("scale factor")
left.set_title("Inflation and deflation at $z = \\pi/4$")
left.legend(fontsize=8)
right.plot(angles, np.sin(angles) ** (1 / 6), label="$\\sin^{1/6}z$")
right.plot(angles, np.cos(angles), "--", label="$\\sqrt{|g|} = \\cos z$")
right.plot(angles, 1 / np.tan(angles), ":", label="$f_0 = \\cot z$")
```

```
right.set_ylim(0, 3)
right.set_xlabel("$z = 6Hx_0$")
right.set_ylabel("value")
right.set_title("Dependence on the hidden direction")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "scale_factors",
...)
```

`times` is x_4 from 0 to 3 (with $H = 1$, $A = 1$ the history is $a_4 = x_4$); `warp` is $\sin^{1/6} z$ at $z = \pi/4$; `angles` is z from just above 0 to just below $\pi/2$. The left panel draws, on a logarithmic axis, the scale factor of 3-space $e^{x_4} \sin^{1/6} z$, that of the extra times $e^{-x_4} \sin^{1/6} z$ and their product; the right panel draws $\sin^{1/6} z$, $\cos z$ and $\cot z$ against z, with the vertical range cut at 3 because $\cot z$ grows without bound near $z = 0$. **What Figure 07b.1 shows.** On the logarithmic axis the 3-space factor is a rising straight line and the extra-time factor a falling one: exponential inflation and exponential deflation, with a constant product. On the right, $\sin^{1/6} z$ rises steeply near the tip $z = 0$ and is almost 1 elsewhere; the volume factor $\cos z$ falls to 0 at $z = \pi/2$, where the frame factor $\cot z$ of the hidden direction also vanishes. The history $a_4 = x_4$ is an illustration chosen for the picture, not a result: every equation of the notebook holds for every $a_4(x_4)$.

In [7]: the Christoffel symbols.

```
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]  # Gam[l][mu][nu]
for l in range(8):
    for mu in range(8):
        for nu in range(8):
            value = 0
            if l == nu:
                value += cd(g[l], mu)
            if l == mu:
                value += cd(g[l], nu)
            if mu == nu:
                value -= cd(g[mu], l)
            if value != 0:
                Gam[l][mu][nu] = sp.expand(value / (2 * g[l]))
```

`Gam[l][mu][nu]` is $\Gamma^{\lambda}{}_{\mu\nu}$, a three-index table of zeros to start with (`[sp.Integer(0)] * 8` is a list of eight zeros; the underscore `_` is a loop variable whose value is not used). The three `if` lines add the three terms of the diagonal formula of Section 7.18, and a nonzero result is divided by $2g_{\lambda\lambda}$ and multiplied out.

```
nonzero = [(l, mu, nu) for l in range(8) for mu in range(8) for nu in range(mu, 8)
           if not is_zero(Gam[l][mu][nu])]
report("independent nonzero Christoffel symbols", len(nonzero))
for l, mu, nu in [(1, 1, 4), (5, 4, 5), (1, 0, 1), (0, 0, 0)]:
    say(f"Gamma^{NAMES[l]}_{NAMES[mu]}{NAMES[nu]} = {sp.factor(Gam[l][mu][nu])}")
```

`nonzero` lists the index triples with $\mu \le \nu$ (the symbols are symmetric in μ and ν) whose symbol is not zero; its length is reported. Four of them are printed, factored with `sp.factor`.

```
recorded = int(re.search(r"(\d+) independent nonzero",
                         revision_check(PYREP, "christoffel_symmetric_metric_"
                                               "compatible")["detail"]).group(1))
check(len(nonzero) == recorded == 25, "25 independent nonzero Christoffel symbols",
      record=reproduces(PYREP, "christoffel_symmetric_metric_compatible"))
# the records number the directions from 1 in the order x0 .. x7: subtract 1
listed = from_wolfram(record_b4["christoffel_nonzero"])
listed_1 = from_wolfram(record["christoffel_nonzero"])
check(len(listed) == len(listed_1) == 25
      and all(is_zero(to_ring(v) - Gam[l - 1][mu - 1][nu - 1])
              for l, mu, nu, v in listed)
      and all(is_zero(to_ring(v) - Gam[l - 1][mu - 1][nu - 1].subs(AT_B4_1))
              for l, mu, nu, v in listed_1),
      "every Christoffel symbol equals the formula record with b4, and at b4 = 1 "
      "the record of b4 = 1",
      record=f"{FORMULAS_B4} and {FORMULAS}, formula christoffel_nonzero")
```

`recorded` is the number that the detail text of the sympy check `christoffel_symmetric_metric_compatible` states ("25 independent nonzero"), read with a regular expression (`(\d+)` captures the digits) and turned into a whole number with `int`. The first check requires our count to equal it and 25. `listed` is the list of all nonzero symbols of the record with b_4 (key `christoffel_nonzero`), `listed_1` that of the record of $b_4 = 1$, each entry a list λ, μ, ν, value with the directions counted from 1 in the order $x_0, \ldots, x_7$; the second check compares each value of the first with ours and each value of the second with ours at $b_4 = 1$ (`Gam[l - 1][mu - 1][nu - 1]` shifts to positions counted from 0). **Out [7]:** `RESULT independent nonzero Christoffel symbols = 25`, the four symbols `Gamma^x1_x1x4 = A1`, `Gamma^x5_x4x5 = -(2*A1*b4 - b4p)/(2*b4)`, `Gamma^x1_x0x1 = H*c/s**6`, `Gamma^x0_x0x0 = -6*H*(c**2 + s**12)/(c*s**6)` (the values derived in Section 7.18; with $c^2 + s^{12} = 1$ the last is $-6H/(\sin z \cos z)$), and two PASS lines with their "reproduces" lines.

In [8]: the canonical spin connection.

```
om = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]  # om[mu][a][b]
for mu in range(8):
    for a in range(8):
        for b in range(8):
            mixed = f[a] * ((cd(1 / f[b], mu) if a == b else 0) + Gam[a][mu][b] / f[b])
            om[mu][a][b] = sp.expand(ETA[a] * mixed)  # lowered with eta_aa
```

`om[mu][a][b]` is $\omega_{\mu ab}$: the formula $f_a(\delta_{ab}\partial_\mu(1/f_b) + \Gamma^a{}_{\mu b}/f_b)$ of Section 7.18 for the mixed connection, lowered with η_{aa}.

```
antisymmetric = all(is_zero(om[mu][a][b] + om[mu][b][a])
                    for mu in range(8) for a in range(8) for b in range(8))
postulate = all(is_zero((cd(f[a], mu) if a == nu else 0) - Gam[a][mu][nu] * f[a]
                        + ETA[a] * om[mu][a][nu] * f[nu])
                for mu in range(8) for a in range(8) for nu in range(8))
check(antisymmetric and postulate, "omega_mu ab = -omega_mu ba and the vielbein "
      "postulate holds in all 512 components",
      record=reproduces(PYREP, "vielbein_postulate"))
```

`antisymmetric` checks $\omega_{\mu ab} + \omega_{\mu ba} = 0$ for all 512 index triples. `postulate` checks the vielbein postulate for the diagonal frame in all 512 components: $\partial_\mu e^a{}_\nu$ (nonzero only for $\nu = a$, where it is $\partial_\mu f_a$), minus $\Gamma^a{}_{\mu\nu} f_a$, plus $\omega_\mu{}^a{}_\nu f_\nu$ with $\omega_\mu{}^a{}_\nu = \eta_{aa}\omega_{\mu a\nu}$.

```
pairs = [(mu, a, b) for mu in range(8) for a in range(8) for b in range(a + 1, 8)
         if not is_zero(om[mu][a][b])]
for mu, a, b in pairs:
    say(f"omega_{NAMES[mu]} ({NAMES[a]})({NAMES[b]}) = {sp.factor(om[mu][a][b])}")
listed = from_wolfram(record_b4["omega_nonzero"])
listed_1 = from_wolfram(record["omega_nonzero"])
check(len(pairs) == 12 and len(listed) == len(listed_1) == 12
      and all(is_zero(to_ring(v) - om[mu - 1][a - 1][b - 1])
              for mu, a, b, v in listed)
      and all(is_zero(to_ring(v) - om[mu - 1][a - 1][b - 1].subs(AT_B4_1))
              for mu, a, b, v in listed_1),
      "exactly 12 nonzero components, equal to the formula record with b4, and at "
      "b4 = 1 to the record of b4 = 1", record=f"{FORMULAS_B4}, formula "
      "omega_nonzero; " + reproduces(WLREP, "omega_components") + " (at b4 = 1)")
```

`pairs` lists the nonzero components with $a < b$, which are printed. `listed` and `listed_1` are the lists of the two records (key `omega_nonzero`); the check requires exactly 12 components, equal to those of the record with b_4, and at $b_4 = 1$ to those of the record of $b_4 = 1$. **Out [8]:** a PASS line with its "reproduces" line, the twelve components (for example `omega_x1 (x1)(x4) = A1*E*s` and `omega_x5 (x4)(x5) = -s*(2*A1*b4 - b4p)/(2*E*sqrt(b4))`), and a second PASS line with its "reproduces" line (the formula record with b_4 and the Wolfram check `omega_components`).

In [9]: the spinor connection and $\gamma^\mu\Omega_\mu$.

```
Om = []  # Omega_mu, mu = x0 .. x7
for mu in range(8):
    M = sp.zeros(16, 16)
    for a in range(8):
        for b in range(a + 1, 8):
            if om[mu][a][b] != 0:
                M += om[mu][a][b] * SAB[a][b]
    Om.append(M.applyfunc(sp.expand))
```

`Om[mu]` is $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$, a 16×16 matrix, built for each of the eight directions and multiplied out entry by entry.

```
gam = [G[a] / f[a] for a in range(8)]  # the coordinate gammas gamma^mu
alpha, beta, rest_zero = [], [], True
for mu in range(8):
    M = (gam[mu] * Om[mu]).applyfunc(sp.expand)  # gamma^mu Omega_mu, no sum
    alpha.append(sp.expand(-(M * G[X4]).trace() / 16))  # coefficient of gamma^(x4)
    beta.append(sp.expand((M * G[X0]).trace() / 16))  # coefficient of gamma^(x0)
    rest_zero = rest_zero and all(
        is_zero(v) for v in (M - alpha[-1] * G[X4] - beta[-1] * G[X0]))
```

`gam` holds the coordinate gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$. For each direction separately (no sum), `M` is $\gamma^\mu\Omega_\mu$, and it is written as $\alpha_\mu\gamma^{(x_4)} + \beta_\mu\gamma^{(x_0)}$. The two numbers come from **traces** (the trace of a

matrix is the sum of its diagonal entries): because $\mathrm{tr}(\gamma^{(x_4)}\gamma^{(x_4)}) = -16$, $\mathrm{tr}(\gamma^{(x_0)}\gamma^{(x_0)}) = 16$ and $\mathrm{tr}(\gamma^{(x_4)}\gamma^{(x_0)}) = 0$, the trace of $M\gamma^{(x_4)}$ is $-16\alpha_\mu$ and the trace of $M\gamma^{(x_0)}$ is $16\beta_\mu$. `rest_zero` stays true only if nothing else is left over: every entry of $M - \alpha_\mu\gamma^{(x_4)} - \beta_\mu\gamma^{(x_0)}$ is zero.

```
for mu in range(8):
    say(f"gamma^{NAMES[mu]} Omega_{NAMES[mu]} = ({alpha[mu]}) gamma^(x4) + "
        f"({beta[mu]}) gamma^(x0)")
listed = from_wolfram(record["gammaOmega_per_direction"])
check(rest_zero and all(is_zero(to_ring(al) - alpha[mu - 1].subs(AT_B4_1))
                        and is_zero(to_ring(be) - beta[mu - 1].subs(AT_B4_1))
                        for mu, al, be in listed),
      "per direction gamma^mu Omega_mu = alpha gamma^(x4) + beta gamma^(x0); at "
      "b4 = 1 equal to the formula record",
      record=f"{FORMULAS}, formula gammaOmega_per_direction")
```

The eight decompositions are printed. The check compares each pair (α_μ, β_μ) at $b_4 = 1$ with the list of the record of $b_4 = 1$ (key `gammaOmega_per_direction`; the record with b_4 stores only the sum).

```
total = sum((gam[mu] * Om[mu] for mu in range(8)), Z16)
GRAV = 3 * H * G[X0] + 3 * b4p / (4 * b4) * G[X4]  # the gravitational term
check(all(is_zero(v) for v in (total - GRAV))
      and all(is_zero(v) for v in (total.subs(AT_B4_1) - 3 * H * G[X0])),
      "gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4) (summed over "
      "mu); 3 H gamma^(x0) at b4 = 1",
      record=reproduces(PYREP_B4, "gammaOmega_total") + "; "
      + reproduces(PYREP, "gamma_mu_Omega_mu_equals_3H_gamma_x0") + " (at b4 = 1)")
check(sp.expand(sum(alpha[1:4])) == sp.Rational(3, 2) * A1
      and sp.expand(sum(alpha[5:8]) + sp.Rational(3, 2) * A1
                    - sp.Rational(3, 4) * b4p / b4) == 0
      and sp.expand(sum(alpha[5:8]).subs(AT_B4_1)) == -sp.Rational(3, 2) * A1
      and sp.expand(sum(beta)) == 3 * H,
      "the a4' terms cancel (+3 A1/2 inflating, -3 A1/2 deflating; the extra times "
      "leave (3/4) b4'/b4, zero at b4 = 1), the six x0 terms add to 3 H",
      record=reproduces(PYREP, "time_terms_cancel_hidden_term_survives")
      + " (at b4 = 1)")
```

`total` is the sum over μ and `GRAV` the gravitational term $M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$; the first check requires `total` to be M, and $3H\gamma^{(x_0)}$ at $b_4 = 1$. The second checks the cancellation: the α of the three inflating directions (positions 1 to 3, `alpha[1:4]`) add to $\frac{3}{2}A_1$, those of the three extra times (positions 5 to 7, `alpha[5:8]`) to $-\frac{3}{2}A_1 + \frac{3}{4}b_4'/b_4$, which is $-\frac{3}{2}A_1$ at $b_4 = 1$, and all β add to $3H$. **Out [9]:** the eight lines `gamma^x0 Omega_x0 = (0) gamma^(x4) + (0) gamma^(x0)`, `gamma^x1 Omega_x1 = (A1/2) gamma^(x4) + (H/2) gamma^(x0) ...` `gamma^x5 Omega_x5 = (-A1/2 + b4p/(4*b4)) gamma^(x4) + (H/2) gamma^(x0) ...`, with zeros for x_0 and x_4, and three PASS lines with their "reproduces" lines.

In [10]: Figure 07b.2.

```
a_units = [float(sp.expand(v.subs(AT_B4_1) / A1)) if v != 0 else 0.0
           for v in alpha]  # in units of a4', at b4 = 1
b_units = [float(sp.expand(v / H)) if v != 0 else 0.0 for v in beta]  # H units
```

```
positions = np.arange(8)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9), sharey=True)
for ax, values, unit, title in [
        (left, a_units, "$a_4'$", "coefficient of $\\gamma^{(x_4)}$"),
        (right, b_units, "$H$", "coefficient of $\\gamma^{(x_0)}$")]:
    ax.bar(positions, values, color=["grey"] + ["tab:blue"] * 3 + ["grey"]
           + ["tab:red"] * 3, label="one direction")
    ax.plot(positions, np.cumsum(values), "ko-", label="running sum")
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(positions, [f"$x_{k}$" for k in range(8)])
    ax.set_xlabel("direction $\\mu$ (blue 3-space, red extra times)")
    ax.set_title(f"{title} (units of {unit})")
    ax.legend(fontsize=8, loc="upper left")
left.set_ylabel("coefficient")
fig.tight_layout()
save_figure(fig, "gamma_omega_per_direction",
...)
```

`a_units` and `b_units` are the coefficients in units of a_4' (at $b_4 = 1$) and of H (the zeros of x_0 and x_4 become 0.0). Two panels with a shared vertical axis (`sharey=True`); the loop draws in each a bar chart with grey bars for x_0 and x_4, blue for 3-space and red for the extra times, and the **running sum** (`np.cumsum`, the sum of the first one, two, three, ... values) as black dots joined by lines. **What Figure 07b.2 shows.** On the left the bars are $+\frac{1}{2}$ (blue) and $-\frac{1}{2}$ (red): the running sum climbs to $\frac{3}{2}$ and falls back to 0, the time terms cancel. On the right all six bars are $+\frac{1}{2}$ and the running sum ends at 3: $\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ at $b_4 = 1$.

In [11]: why the connection drops out of $\mathcal{L}$ but not out of the equation.

```
anti = all(all(is_zero(v) for v in (gam[mu] * Om[mu] + Om[mu] * gam[mu]))
           for mu in range(8))
check(anti, "{gamma^mu, Omega_mu} = 0 for each mu: the connection drops out of L",
      record=reproduces(PYREP, "anticommutator_gamma_Omega_vanishes"))
divergence = sum((((sqrt_g * gam[mu]).applyfunc(lambda v: cd(v, mu)))
                  for mu in range(8)), Z16) / (2 * sqrt_g)
check(all(is_zero(v) for v in (divergence - GRAV)),
      "(1/(2 sqrt|g|)) d_mu (sqrt|g| gamma^mu) = 3 H gamma^(x0) + (3/4)(b4'/b4) "
      "gamma^(x4)", record=reproduces(PYREP, "divergence_of_sqrtg_gamma")
      + " (at b4 = 1)")
```

`anti` checks $\gamma^\mu \Omega_\mu + \Omega_\mu \gamma^\mu = 0$ for each direction. `divergence` is $\frac{1}{2\sqrt{|g|}} \sum_\mu \partial_\mu(\sqrt{|g|}\gamma^\mu)$: `applyfun c(lambda v: cd(v, mu))` differentiates every entry of the matrix $\sqrt{|g|}\gamma^\mu$ along x_μ. The second check requires $M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$. **Out [11]:** two PASS lines with their "reproduces" lines.

In [12]: a jet algebra for both statistics.

```
CHI, PSI = 0, 1  # the two kinds: components of Psi^dagger and of Psi


def gen(kind, A, derivatives=()):
    """The generator: component A of chi (kind 0) or psi (kind 1), differentiated
```

```
    along the coordinate positions in derivatives."""
    return (kind, A, tuple(sorted(derivatives)))
```

A generator of the jet algebra is a triple `(kind, A, derivatives)`: kind 0 for a component $\chi_A = \Psi^*_A$ of $\Psi^\dagger$, kind 1 for a component ψ_A of Ψ; `A` from 0 to 15; `derivatives` a sorted tuple of coordinate positions (empty for the value itself, `(4,)` for ∂_4). `gen` builds such a triple.

```
def sort_sign(keys, odd):
    """Sorted tuple of generators and the sign; (None, 0) for a zero product."""
    items, sign = list(keys), 1
    for end in range(len(items) - 1, 0, -1):  # bubble sort, counting exchanges
        for i in range(end):
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                sign = -sign
    if not odd:
        return tuple(items), 1  # commuting generators: no signs, squares allowed
    if any(items[i] == items[i + 1] for i in range(len(items) - 1)):
        return None, 0  # a repeated Grassmann generator: the product is zero
    return tuple(items), sign
```

`sort_sign(keys, odd)` sorts a tuple of generators with the bubble sort of Notebook 07a and counts the exchanges (Python compares triples entry by entry, so the order is first by kind, then by component, then by derivatives). For commuting generators (`odd` false) there is no sign and squares are allowed; for Grassmann generators a repeated generator gives the zero product, `(None, 0)`.

```
class Jet:
    """An element of the jet algebra: self.terms maps monomials to coefficients."""

    def __init__(self, odd, terms=None):
        self.odd, self.terms = odd, {}
        for monomial, coefficient in (terms or {}).items():
            self.add(monomial, coefficient)

    def add(self, monomial, coefficient):
        total = sp.expand(self.terms.get(monomial, 0) + coefficient)
        if total == 0:
            self.terms.pop(monomial, None)
        else:
            self.terms[monomial] = total
```

The class `Jet`: an element remembers whether its generators are odd and holds a dictionary from monomials to ring coefficients. `add` adds a coefficient to a monomial, multiplies out and removes zeros, as in Notebook 07a.

```
    def __add__(self, other):
        result = Jet(self.odd, self.terms)
        for monomial, coefficient in other.terms.items():
            result.add(monomial, coefficient)
        return result
```

```
    def __sub__(self, other):
        return self + other.times(-1)

    def times(self, factor):  # an ordinary (ring) factor
        return Jet(self.odd, {k: factor * v for k, v in self.terms.items()})
```

Addition, subtraction, and `times(factor)`, the multiplication by an ordinary ring factor.

```
    def __mul__(self, other):
        result = Jet(self.odd)
        for k1, v1 in self.terms.items():
            for k2, v2 in other.terms.items():
                monomial, sign = sort_sign(k1 + k2, self.odd)
                if monomial is not None:
                    result.add(monomial, sign * v1 * v2)
        return result
```

The product: every term times every term, the joined monomial sorted with its sign.

```
    def conj(self):
        result = Jet(self.odd)
        for monomial, coefficient in self.terms.items():
            images = [(1 - kind, A, d) for kind, A, d in reversed(monomial)]
            ordered, sign = sort_sign(images, self.odd)
            if ordered is not None:
                result.add(ordered, sign * sp.sympify(coefficient).subs(sp.I, -sp.I))
        return result
```

`conj` is the conjugation of Section 7.11: the monomial is reversed, each generator's kind is exchanged ($\chi_A \leftrightarrow \psi_A$, `1 - kind`) while its component and derivatives are kept, the result is sorted with its sign (for commuting generators no sign), and i is replaced by $-i$ in the coefficient (all other symbols are real).

```
    def lderiv(self, key):
        result = Jet(self.odd)
        for monomial, coefficient in self.terms.items():
            if key in monomial:
                p = monomial.index(key)
                factor = (-1) ** p if self.odd else monomial.count(key)
                result.add(monomial[:p] + monomial[p + 1:], factor * coefficient)
        return result

    def rderiv(self, key):
        if not self.odd:
            return self.lderiv(key)
        result = Jet(self.odd)
        for monomial, coefficient in self.terms.items():
            if key in monomial:
                p = monomial.index(key)
```

```
                factor = (-1) ** (len(monomial) - 1 - p)
                result.add(monomial[:p] + monomial[p + 1:], factor * coefficient)
        return result
```

`lderiv(key)` is the left derivative: for Grassmann generators the sign $(-1)^p$ of the position; for commuting generators the ordinary derivative, which brings down the power (`monomial.count(key)` is the number of times the generator occurs, the n of $x^n \to nx^{n-1}$). `rderiv(key)` is the right derivative; for commuting generators it is the same as the left one.

```
    def total(self, mu):
        result = Jet(self.odd)
        for monomial, coefficient in self.terms.items():
            derivative = cd(coefficient, mu)
            if derivative != 0:
                result.add(monomial, derivative)
            for p, (kind, A, d) in enumerate(monomial):
                raised = (kind, A, tuple(sorted(d + (mu,))))
                ordered, sign = sort_sign(monomial[:p] + (raised,) + monomial[p + 1:],
                                          self.odd)
                if ordered is not None:
                    result.add(ordered, sign * coefficient)
        return result

    def is_zero(self):
        return all(is_zero(v) for v in self.terms.values())
```

`total(mu)` is the total derivative d/dx_μ on jets: the coefficient is differentiated with `cd` (the explicit dependence on x_4 and x_0), and every generator of the monomial in turn gets the additional derivative μ (its derivative tuple grows by `mu`), with the product rule and the sign of sorting back. `is_zero` uses the exact zero test of In [3] on every coefficient.

```
def column(odd, kind, derivatives=()):
    """The 16 generators of chi or psi (or of one of their derivatives)."""
    return [Jet(odd, {(gen(kind, A, derivatives),): 1}) for A in range(16)]


def matvec(M, v, odd):
    """The column M v."""
    out = [Jet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[i] = out[i] + v[j].times(M[i, j])
    return out


def vecmat(v, M, odd):
    """The row v M."""
    out = [Jet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
```

```
            if M[i, j] != 0:
                out[j] = out[j] + v[i].times(M[i, j])
    return out


def dot(row, col, odd):
    """The number row col (row first)."""
    result = Jet(odd)
    for x, y in zip(row, col):
        result = result + x * y
    return result


say("The jet algebra and the vector helpers are defined.")
```

Vector helpers: `column(odd, kind, derivatives)` is the list of the 16 generators of χ or ψ (or of one of their derivatives) as elements; `matvec(M, v, odd)` is the column Mv, `vecmat(v, M, odd)` the row vM (each skips the zero entries of the matrix), and `dot(row, col, odd)` the number $\sum_A \mathrm{row}_A \mathrm{col}_A$ with the row factor first. **Out [12]:** `The jet algebra and the vector helpers are defined.`

In [13]: the Lagrangian of both fields.

```
def build(odd, connection):
    """Lagrangian, unsymmetrised Lagrangian, field-equation residual and pieces
    for one statistics (odd True: Grassmann) and one spinor connection."""
    psi, chi = column(odd, PSI), column(odd, CHI)
    psibar = vecmat(chi, C, odd)  # Psibar = Psi^dagger C
    Dpsi = [[x + y for x, y in zip(column(odd, PSI, (mu,)),
                                   matvec(connection[mu], psi, odd))]
            for mu in range(8)]
    Dpsibar = [[x - y for x, y in zip(vecmat(column(odd, CHI, (mu,)), C, odd),
                                      vecmat(psibar, connection[mu], odd))]
               for mu in range(8)]
```

`build(odd, connection)` builds everything for one statistics and one spinor connection (a list of eight matrices). `psi` and `chi` are the columns of generators; `psibar` is the row $\bar\Psi = \chi C$. `Dpsi[mu]` is $D_\mu\psi = \partial_\mu\psi + \Omega_\mu\psi$ and `Dpsibar[mu]` is $D_\mu\bar\Psi = \partial_\mu\chi\, C - \bar\Psi\Omega_\mu$, each a list of 16 elements.

```
    K, Ku = Jet(odd), Jet(odd)
    for mu in range(8):
        forward = dot(psibar, matvec(gam[mu], Dpsi[mu], odd), odd)
        K = K + forward - dot(vecmat(Dpsibar[mu], gam[mu], odd), psi, odd)
        Ku = Ku + forward
    S = dot(psibar, psi, odd)
    potential = S.times(m) + (S * S).times(lam / 2)  # m S + (lambda/2) S^2
    L = (K.times(sp.Rational(1, 2)) - potential).times(sqrt_g)
    Lu = (Ku - potential).times(sqrt_g)
```

`K` collects $\sum_\mu(\bar\Psi\gamma^\mu D_\mu\psi - D_\mu\bar\Psi\gamma^\mu\psi)$ and `Ku` the unsymmetrised $\sum_\mu \bar\Psi\gamma^\mu D_\mu\psi$ (`forward` is the first product, computed once). `S` is $\bar\Psi\psi$, `potential` is $mS + \frac{\lambda}{2}S^2$. `L` is $\sqrt{|g|}(\frac{1}{2}K - mS - \frac{\lambda}{2}S^2)$, `Lu` the unsymmetrised form.

```
    dirac = [Jet(odd) for _ in range(16)]  # gamma^mu D_mu psi
    for mu in range(8):
        dirac = [x + y for x, y in zip(dirac, matvec(gam[mu], Dpsi[mu], odd))]
    V = Jet(odd, {(): m}) + S.times(lam)  # m + lambda S
    residual = [dirac[A] - V * psi[A] for A in range(16)]
    bar_residual = [Jet(odd)] * 16  # -(D_mu Psibar gamma^mu + V Psibar)
    for mu in range(8):
        bar_residual = [x - y for x, y in zip(bar_residual,
                                              vecmat(Dpsibar[mu], gam[mu], odd))]
    bar_residual = [x - V * y for x, y in zip(bar_residual, psibar)]
    return {"L": L, "Lu": Lu, "E": residual, "Ebar": bar_residual, "S": S,
            "psi": psi, "psibar": psibar, "dirac": dirac}
```

`dirac` is the column $\gamma^\mu D_\mu \psi$; `V` is the element $m + \lambda S$; `residual` is the column $\mathcal{E} = \gamma^\mu D_\mu \psi - V\psi$. `bar_residual` is the row $-(D_\mu \bar{\Psi}\gamma^\mu + V\bar{\Psi})$. The function returns all pieces in a dictionary.

```
STATS = {"grassmann": True, "commuting": False}  # statistics -> odd
fields = {name: build(odd, Om) for name, odd in STATS.items()}
kinds = {}
for name, data in fields.items():
    monomials = data["L"].terms
    kinetic = sum(1 for k in monomials if any(d for (_, _, d) in k))
    quartic = sum(1 for k in monomials if len(k) == 4)
    kinds[name] = (kinetic, len(monomials) - kinetic - quartic, quartic)
    report(f"monomials of L, {name}", f"{len(monomials)} = {kinetic} kinetic + "
           f"{kinds[name][1]} mass + {quartic} quartic")
```

`STATS` maps the two names to the switch `odd`. `fields` holds the pieces for both statistics with the canonical connection `Om`. For each, the monomials of $\mathcal{L}$ are sorted by type: kinetic if some generator carries a derivative (`any(d for (_, _, d) in k)` is true when a derivative tuple is not empty), quartic if the monomial has four generators, and mass otherwise. **Out [13]:** `RESULT monomials of L, grassmann = 392 = 256 kinetic + 16 mass + 120 quartic` and `RESULT monomials of L, commuting = 408 = 256 kinetic + 16 mass + 136 quartic`.

In [14]: the counts against the record, and Figure 07b.3.

```
for name in STATS:
    detail = revision_check(PYREP, f"{name}_lagrangian_real")["detail"]
    recorded = int(re.search(r"\((\d+) monomials", detail).group(1))
    check(len(fields[name]["L"].terms) == recorded,
          f"{name}: L has {recorded} monomials, as recorded",
          record=reproduces(PYREP, f"{name}_lagrangian_real"))
check(kinds["grassmann"] == (256, 16, 120) and kinds["commuting"] == (256, 16, 136),
      "the difference is the 16 squares of the pairs in S^2 (136 - 120)")
```

For each statistics the number of monomials is read from the detail text of the record's reality check ("(392 monomials" and "(408 monomials") and compared. The last check is the difference of Section 7.20: 136 minus 120 quartic monomials, the 16 squares of the pairs.

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
labels = ["dirac16complex\n(Grassmann)", "dirac16complex00\n(commuting)"]
```

```
bottoms = np.zeros(2)
for index, (part, colour) in enumerate([("kinetic", "tab:blue"), ("mass", "tab:green"),
                                        ("quartic", "tab:orange")]):
    values = np.array([kinds["grassmann"][index], kinds["commuting"][index]])
    ax.bar(labels, values, bottom=bottoms, color=colour, label=part, width=0.5)
    for x, (value, bottom) in enumerate(zip(values, bottoms)):
        ax.text(x, bottom + value / 2, str(value), ha="center", va="center")
    bottoms = bottoms + values
for x, total in enumerate(bottoms):
    ax.text(x, total + 6, f"total {int(total)}", ha="center")
ax.set_ylim(0, 520)  # room above the bars for the legend
ax.set_ylabel("number of monomials of $L$")
ax.set_title("The Lagrangian at one point, by type of term")
ax.legend(loc="upper center", ncol=3, fontsize=8)
save_figure(fig, "lagrangian_monomials",
...)
```

A **stacked bar chart**: for each field the bars of the three types are drawn on top of each other (`bottom=bottoms` starts each new bar where the previous one ended), the counts are written into the bars and the totals above them (`"\n"` in a label is a line break). **Out [14]:** three PASS lines (the first two with "reproduces" lines) and the figure. **What Figure 07b.3 shows.** The kinetic (blue) and mass (green) parts are equal for the two fields; the only difference is the quartic part (orange), 120 against 136: the 16 squares $(\bar{\Psi}_A \Psi_B)^2$ vanish for Grassmann numbers.

In [15]: reality, the total divergence and the connection that drops out.

```
no_connection = [Z16] * 8  # Omega_mu = 0
for name, odd in STATS.items():
    data = fields[name]
    L, Lu = data["L"], data["Lu"]
    check((L.conj() - L).is_zero() and not (Lu.conj() - Lu).is_zero(),
          f"{name}: L is real, the unsymmetrised L_u is not",
          record=reproduces(PYREP, f"{name}_controls_not_vacuous"))
    current = Jet(odd)
    for mu in range(8):
        current = current + dot(data["psibar"], matvec(gam[mu], data["psi"], odd),
                                odd).times(sqrt_g).total(mu)
    check((L - Lu + current.times(sp.Rational(1, 2))).is_zero(),
          f"{name}: L = L_u - (1/2) d_mu (sqrt|g| Psibar gamma^mu Psi)",
          record=reproduces(PYREP, f"{name}_total_divergence_relation"))
    flat_connection = build(odd, no_connection)["L"]
    tag = "G" if odd else "C"
    check((L - flat_connection).is_zero(),
          f"{name}: L with Omega equals L with Omega = 0 in this metric",
          record=reproduces(WLREP, f"L_spin_connection_drops_out_{tag}"))
```

`no_connection` is the list of eight zero matrices ($\Omega_\mu = 0$). For each statistics: the first check requires $\mathcal{L}^* = \mathcal{L}$ and that the unsymmetrised form is not real (the control). `current` is $\sum_\mu \frac{d}{dx_\mu}(\sqrt{|g|}\,\bar{\Psi}\gamma^\mu\psi)$, built with the total derivative of the jet algebra; the second check is the relation $\mathcal{L} = \mathcal{L}_u - \frac{1}{2}\cdot$ current of Section 7.20. `flat_connection` is the Lagrangian built with $\Omega_\mu = 0$; the third check requires it to equal $\mathcal{L}$. `tag` is `"G"` or `"C"`, the ending of the Wolfram check names. **Out [15]:** six PASS lines, each with its "reproduces" line.

In [16]: the Euler-Lagrange equations.

```
def euler_lagrange_chi(L, A):
    """dL/dchi_A - d_mu dL/d(d_mu chi_A), left derivatives."""
    result = L.lderiv(gen(CHI, A))
    for mu in range(8):
        result = result - L.lderiv(gen(CHI, A, (mu,))).total(mu)
    return result


def euler_lagrange_psi(L, B):
    """dL/dpsi_B - d_mu dL/d(d_mu psi_B), right derivatives."""
    result = L.rderiv(gen(PSI, B))
    for mu in range(8):
        result = result - L.rderiv(gen(PSI, B, (mu,))).total(mu)
    return result
```

`euler_lagrange_chi(L, A)` is $\partial_L \mathcal{L}/\partial\chi_A - \sum_\mu \frac{d}{dx_\mu}\partial_L \mathcal{L}/\partial(\partial_\mu \chi_A)$ with left derivatives; `euler_lagrange_psi(L, B)` is the same for ψ_B with right derivatives.

```
for name, odd in STATS.items():
    data = fields[name]
    L, residual, psi = data["L"], data["E"], data["psi"]
    expected = matvec(C, residual, odd)  # C times the residual
    ok = all((euler_lagrange_chi(L, A) - expected[A].times(sqrt_g)).is_zero()
             for A in range(16))
    wrong = matvec(C, [residual[B] + psi[B].times(2 * m) for B in range(16)], odd)
    control = not (euler_lagrange_chi(L, 0) - wrong[0].times(sqrt_g)).is_zero()
    check(ok and control, f"{name}: varying Psi^dagger gives gamma^mu D_mu Psi = "
          "(m + lambda S) Psi in all 16 components (control m -> -m fails)",
          record=reproduces(PYREP, f"{name}_euler_lagrange_psibar_variation"))
    ok = all((euler_lagrange_psi(L, B) - data["Ebar"][B].times(sqrt_g)).is_zero()
             for B in range(16))
    check(ok, f"{name}: varying Psi gives D_mu Psibar gamma^mu = -(m + lambda S) "
          "Psibar", record=reproduces(PYREP, f"{name}_euler_lagrange_psi_variation"))
    conjugate_row = vecmat([x.conj() for x in residual], C, odd)
    check(all((conjugate_row[B] - data["Ebar"][B]).is_zero() for B in range(16)),
          f"{name}: the adjoint equation is the Dirac conjugate of the field equation",
          record=reproduces(PYREP, f"{name}_adjoint_equation_is_conjugate"))
```

For each statistics: `expected` is the column $C\mathcal{E}$; the first check requires the Euler-Lagrange expression of every χ_A to equal $\sqrt{|g|}(C\mathcal{E})_A$ (Steps 1 to 6 of Section 7.21), and, as a control, that the comparison with m replaced by $-m$ fails (`wrong` adds $2m\psi$ to the residual; only component 0 is tested, which suffices to show the failure). The second check: the expressions for ψ_B equal $\sqrt{|g|}$ times the adjoint residual. The third: the conjugate of the residual, as a row times C (`conjugate_row`), equals the adjoint residual. **Out [16]:** six PASS lines, each with its "reproduces" line.

In [17]: the equations written out in the author's metric.

```
for name, odd in STATS.items():
    data = fields[name]
```

```
    explicit = matvec(GRAV, data["psi"], odd)  # (3 H gamma^(x0) + ...) psi
    for a in range(8):
        explicit = [x + y for x, y in zip(explicit, matvec(
            G[a] / f[a], column(odd, PSI, (a,)), odd))]
    tag = "G" if odd else "C"
    check(all((data["dirac"][A] - explicit[A]).is_zero() for A in range(16)),
          f"{name}: gamma^mu D_mu Psi = sum_a (1/f_a) gamma^(a) d_a Psi + 3 H "
          "gamma^(x0) Psi + (3/4)(b4'/b4) gamma^(x4) Psi",
          record=reproduces(WLREP, f"Dirac_operator_explicit_{tag}") + " (at b4 = 1)")
```

For each statistics `explicit` is the column $M\psi + \sum_a \frac{1}{f_a}\gamma^{(a)}\partial_a\psi$ (`GRAV` is M); the check requires it to equal $\gamma^\mu D_\mu\psi$ in all 16 components.

```
def as_sympy(element):
    """A linear jet element as a sympy expression in d<a>_<B> and P<B>."""
    total = 0
    for ((kind, B, d),), coefficient in element.terms.items():
        symbol = f"d{d[0]}_{B + 1}" if d else f"P{B + 1}"  # d0 .. d7: x0 .. x7
        total += coefficient * sp.Symbol(symbol)
    return total
```

`as_sympy(element)` turns a linear jet element into an ordinary sympy expression: each monomial has exactly one generator (the unpacking `((kind, B, d),)` requires this), which becomes the symbol `d<a>_<B>` for $\partial_a\Psi_B$ or `P<B>` for Ψ_B, counted from 1 like the record.

```
ours = [as_sympy(x) for x in fields["grassmann"]["dirac"]]
theirs = [to_ring(from_wolfram(text.split("==")[0]))
          for text in record_b4["field_equation_components"]]
theirs_1 = [to_ring(from_wolfram(text.split("==")[0]))
            for text in record["field_equation_components"]]
say(f"equation 1, this notebook: {ours[0]} = V P1")
say(f"equation 1, formula record: {theirs[0]} = V P1")
check(len(theirs) == len(theirs_1) == 16
      and all(is_zero(x - y) for x, y in zip(ours, theirs))
      and all(is_zero(x.subs(AT_B4_1) - y) for x, y in zip(ours, theirs_1)),
      "all 16 component equations equal the formula record with b4 term by term, "
      "and at b4 = 1 the record of b4 = 1",
      record=f"{FORMULAS_B4} and {FORMULAS}, formula field_equation_components")
```

`as_sympy` names the derivative of component B along the direction at position a `d<a>_<B>` (so `d0_9` is $\partial_0\Psi_9$, as in the records). `ours` are our 16 Dirac-operator components, `theirs` and `theirs_1` the left sides of the 16 component equations of the record with b_4 and of the record of $b_4 = 1$ (each text is split at `==` and its first part translated and put into the ring). The first equation is printed in both forms, and the check compares all 16 with the exact zero test: ours with the first record, and ours at $b_4 = 1$ with the second. **Out [17]:** two PASS lines with their "reproduces" lines, the first equation twice (identical, the equation of Section 7.22 in ring symbols), and the third PASS line. The first equation is printed as

```
equation 1, this notebook: E*d5_15/(sqrt(b4)*s) + E*d6_16/(sqrt(b4)*s) +
    E*d7_9/(sqrt(b4)*s) + 3*H*P9 - 3*P14*b4p/(4*b4) - d4_14 + d0_9*s**6/c - d1_16/(E*s) +
    d2_15/(E*s) - d3_14/(E*s) = V P1
```

In [18]: the coupling map, Figure 07b.4.

```
counts = np.zeros((16, 16), dtype=int)
tags = [["" for _ in range(16)] for _ in range(16)]
for a in range(8):
    for A in range(16):
        for B in range(16):
            if G[a][A, B] != 0:
                counts[A, B] += 1
                tags[A][B] += str(a)  # the digit k names the direction xk
four_twos = all(np.count_nonzero(counts[A]) == 4 and counts[A].max() == 2
                for A in range(16))  # 4 components, each with 2 directions
check(four_twos and int(counts.sum()) == 128
      and not counts[:8, :8].any() and not counts[8:, 8:].any(),
      "each equation couples 4 components with 2 directions each, across the chiral "
      "halves")
```

`counts[A, B]` counts the directions a whose gamma has a nonzero entry in row A, column B, that is the directions whose derivative of Ψ_B appears in equation A; `tags[A][B]` collects their numbers as a text. The check: every row has 4 nonzero entries, each 2; 128 derivative terms in total (8×16); and the two diagonal 8×8 blocks are empty (`counts[:8, :8]` is the upper left block): every equation couples the two chiral halves.

```
fig, ax = plt.subplots(figsize=(7.4, 6.6))
image = ax.imshow(counts, cmap="Blues", vmin=0, vmax=2)
for A in range(16):
    for B in range(16):
        if tags[A][B]:
            ax.text(B, A, tags[A][B], ha="center", va="center", fontsize=7,
                    color="white" if counts[A, B] == 2 else "black")
ax.axhline(7.5, color="red", linewidth=1)
ax.axvline(7.5, color="red", linewidth=1)
ax.set_xticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
ax.set_yticks(range(0, 16, 3), [str(k + 1) for k in range(0, 16, 3)])
ax.set_xlabel("component $B$ whose derivative appears")
ax.set_ylabel("equation $A$")
ax.set_title("Which derivatives $\\partial_a\\Psi_B$ enter equation $A$")
ax.grid(False)
fig.colorbar(image, ax=ax, ticks=[0, 1, 2], shrink=0.8,
             label="number of directions $a$")
save_figure(fig, "coupling_map",
...)
```

A heat map of `counts` in shades of blue, with the direction numbers written in (white on the darker squares) and red lines between rows 8 and 9 and columns 8 and 9 (`axhline(7.5)` lies between positions 7 and 8). **What Figure 07b.4 shows.** Each row has four coloured squares, each labelled with two direction numbers; the upper left and lower right quarters are empty: the equations of the upper half contain only the lower half of the field, and the other way round. The digit 0 marks the components that carry both $\tan z\,\partial_0$ and the term $3H$, the digit 4 those that carry ∂_4 and the term $\frac{3}{4}(b_4'/b_4)$ (each digit k names the direction x_k).

In [19]: the chiral blocks and the evolution form.

```
blocks = {}  # direction name -> (tau-bar_a, tau_a)
for text in record["field_equation_blocks"]:
    # the Wolfram list {"x1", {{...}}, {{...}}} is the JSON list ["x1", [[...]], [[...]]]
    name, upper, lower = json.loads(text.replace("{", "[").replace("}", "]"))
    blocks[name] = (sp.Matrix(upper), sp.Matrix(lower))
GAMMA = sp.Matrix(fixture["Gamma"])  # the chirality matrix of Revision/algebra
Z8 = sp.zeros(8, 8)
blocks_ok = sorted(blocks) == NAMES and GAMMA == sp.diag(*([-1] * 8 + [1] * 8))
for a in range(8):
    upper, lower = blocks[NAMES[a]]
    blocks_ok = (blocks_ok and G[a][:8, :8] == Z8 and G[a][8:, 8:] == Z8  # zero
                 and G[a][:8, 8:] == upper and G[a][8:, :8] == lower)  # the blocks
blocks_ok = blocks_ok and blocks["x0"][0] == sp.eye(8) and blocks["x0"][1] == sp.eye(8)
check(blocks_ok, "Gamma = diag(-I8, I8); every gamma^(a) is block off-diagonal with the "
      "blocks of the formula record; tau-bar_x0 = tau_x0 = I8",
      record=reproduces(WLREP, "block_form"))
```

The record's key `field_equation_blocks` holds, for each direction, its name and its two 8×8 blocks as a Wolfram list; replacing braces by square brackets makes it a JSON list, which `json.loads` reads. `GAMMA` is the chirality matrix of the record, `Z8` the 8×8 zero matrix. The check requires: the eight direction names; $\Gamma = \mathrm{diag}(-I_8, I_8)$ (`sp.diag(*list)` builds a diagonal matrix from a list); for every gamma the two diagonal blocks zero and the off-diagonal blocks equal to the record's (`G[a][:8, 8:]` is the upper right block); and $\bar{\tau}_{x_0} = \tau_{x_0} = I_8$.

```
for name, odd in STATS.items():
    data = fields[name]
    psi = data["psi"]
    V = Jet(odd, {(): m}) + data["S"].times(lam)  # m + lambda S
    others = matvec(GRAV, psi, odd)  # the gravitational term M Psi, then the sum
    for a in range(8):
        if a != X4:  # (1/f_a) gamma^(a) d_a Psi for the seven other directions
            others = [x + y for x, y in zip(others, matvec(
                G[a] / f[a], column(odd, PSI, (a,)), odd))]
    bracket = [V * psi[A] - others[A] for A in range(16)]  # the square bracket
    right = matvec(-G[X4], bracket, odd)  # R = -gamma^(x4) [ ... ]
    difference = [x - y for x, y in zip(column(odd, PSI, (X4,)), right)]  # d4 Psi - R
    left = matvec(G[X4], difference, odd)  # gamma^(x4) (d4 Psi - R)
    tag = "G" if odd else "C"
    check(all((left[A] - data["E"][A]).is_zero() for A in range(16)),
          f"{name}: the evolution form d4 Psi = R is the field equation "
          "(gamma^(x4) (d4 Psi - R) = E in all 16 components)",
          record=reproduces(WLREP, f"evolution_form_{tag}") + " (at b4 = 1)")
```

For each statistics: `others` is $M\psi + \sum_{a \neq x_4} \frac{1}{f_a} \gamma^{(a)} \partial_a \psi$, `bracket` is $V\psi -$ others, `right` is $R = -\gamma^{(x_4)}[\dots]$, `difference` is $\partial_4 \psi - R$ and `left` is $\gamma^{(x_4)}(\partial_4 \psi - R)$. The check requires `left` to equal the residual $\mathcal{E}$ in all 16 components: the evolution form is the field equation. **Out [19]:** three PASS lines with their "reproduces" lines.

In [20]: non-triviality.

```
for name, odd in STATS.items():
    data = fields[name]
```

```
    without = build(odd, no_connection)["E"]  # residual with Omega = 0
    gravity = matvec(GRAV, data["psi"], odd)  # M Psi
    ok = all((data["E"][A] - without[A] - gravity[A]).is_zero() for A in range(16))
    hidden = matvec(3 * H * G[X0], data["psi"], odd)  # M Psi at b4 = 1
    nonzero = all(len(hidden[A].terms) == 1 for A in range(16))
    number = "1_dirac16complex" if odd else "2_dirac16complex00"
    check(ok and nonzero, f"{name}: residual(Omega) - residual(Omega = 0) = M Psi "
          "with M = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4); at b4 = 1 it is "
          "3 H gamma^(x0) Psi, nonzero in all 16 components",
          record=reproduces(WLREP, f"nontriviality_{number}") + " (at b4 = 1); "
          + reproduces(PYREP_B4, "gravity_term_square_b4"))
```

For each statistics, `without` is the residual computed with $\Omega_\mu = 0$, `gravity` the column $M\psi$ and `hidden` the column $3H\gamma^{(x_0)}\psi$, which is $M\psi$ at $b_4 = 1$. The check requires residual(with Ω) minus residual(without) to equal `gravity` in all 16 components, and every component of `hidden` to be one nonzero monomial. **Out [20]:** two PASS lines with their "reproduces" lines (Wolfram checks `nontriviality_1_dirac16complex` and `nontriviality_2_dirac16complex00` at $b_4 = 1$, and the sympy check `gravity_term_square_b4` for a changing b_4).

In [21]: the exact family of solutions.

```
al, k, ch, sh = sp.symbols("alpha k ch sh", real=True)
chi0 = sp.Matrix(sp.symbols("q1:17"))  # an arbitrary constant column
M = -m * G[X4] + 3 * H * (2 * al + 1) * G[X4] * G[X0]
k_squared = 9 * H**2 * (2 * al + 1) ** 2 - m**2
check((M * M - k_squared * I16).applyfunc(sp.expand) == Z16
      and (M.T * C + C * M).applyfunc(sp.expand) == Z16,
      "M^2 = k^2 with k^2 = 9 H^2 (2 alpha + 1)^2 - m^2, and M^T C + C M = 0")
```

Symbols for α, k and the two functions $\cosh(kx_4)$, $\sinh(kx_4)$ (written `ch`, `sh`); `chi0` is a column of 16 arbitrary symbols. `M` and `k_squared` are those of Section 7.23. The check: $M^2 = k^2$ and $M^T C + CM = 0$.

```
def d4(v):  # derivative along x4 with ch' = k sh, sh' = k ch
    return v.diff(ch) * k * sh + v.diff(sh) * k * ch


def d0(v):  # derivative along x0 in the ring (s^(6 alpha) included)
    return v.diff(s) * H * c / s**5 + v.diff(c) * (-6 * H * s**6)


Psi = s ** (6 * al) * (ch * I16 + sh / k * M) * chi0  # sin^alpha z P(x4) chi0
residual = (G[X4] * Psi.applyfunc(d4) + (s**6 / c) * G[X0] * Psi.applyfunc(d0)
            + 3 * H * G[X0] * Psi - m * Psi)
```

`d4` differentiates along x_4 with $\partial_4 \mathrm{ch} = k\,\mathrm{sh}$ and $\partial_4 \mathrm{sh} = k\,\mathrm{ch}$; `d0` along x_0 in the ring (the factor $\sin^\alpha z$ is written $s^{6\alpha}$, whose derivative sympy handles). `Psi` is the family $\sin^\alpha z\,(\cosh kx_4 + \sinh(kx_4)/k\,M)\chi_0$ and `residual` the left side minus the right side of the explicit equation (the other six derivative terms vanish for a field of x_4 and x_0 only).

```
def vanishes(v):
    """Multiply by k and divide by sin^alpha z = s^(6 alpha), multiply out, use
    k^2 = 9 H^2 (2 alpha + 1)^2 - m^2: is the result zero?"""
    v = sp.expand(v * k / s ** (6 * al))
    return sp.expand(v.subs(k**2, k_squared)) == 0


without_term = residual - 3 * H * G[X0] * Psi  # the same without 3 H gamma^(x0)
check(all(vanishes(v) for v in residual)
      and not all(vanishes(v) for v in without_term),
      "Psi = sin^alpha z (cosh k x4 + sinh k x4 / k M) chi0 solves the field "
      "equation for every alpha, every a4 and constant b4 (and fails without "
      "3 H gamma^(x0))",
      record=reproduces(PYREP, "exact_solution_family_x4_x0"))
```

`vanishes(v)` multiplies a residual entry by k, divides by $s^{6\alpha}$, multiplies out, replaces k^2 by its value, and tests for zero. The check requires all 16 entries of the residual to vanish, and at least one entry of the residual without the term $3H\gamma^{(x_0)}\Psi$ not to vanish.

```
A_matrix = -sp.I * m * G[X4] + 3 * sp.I * H * G[X4] * G[X0]
check((A_matrix * A_matrix - (m**2 - 9 * H**2) * I16).applyfunc(sp.expand) == Z16,
      "the mode matrix A = -i m gamma^(x4) + 3 i H gamma^(x4) gamma^(x0) has A^2 = "
      "(m^2 - 9 H^2) I16", record=reproduces(
          SCOPE, "good_sector_x0_independent_modes_without_boundary_condition"))
```

The mode matrix of the scope record, $A = -im\gamma^{(x_4)} + 3iH\gamma^{(x_4)}\gamma^{(x_0)}$ (the matrix that $i\partial_4$ applies to the x_0-independent fields), squares to $(m^2 - 9H^2)I_{16}$. **Out [21]:** three PASS lines, the last two with their "reproduces" lines.

In [22]: two members of the family, numerically.

```
Gn = [np.array(gm.tolist(), dtype=float) for gm in G]  # the gammas as float arrays
Cn = np.array(C.tolist(), dtype=float)
chi_n = np.zeros(16)
chi_n[[0, 4]] = 1.0  # e_1 + e_5
zeta = np.pi / 4  # the point z = pi/4 of the hidden direction
xs4 = np.linspace(0.0, 2.0, 201)
```

The gammas and C as floating-point numpy arrays; `chi_n` is $e_1 + e_5$ (positions 0 and 4 set to 1); the point $z = \pi/4$ and 201 times from 0 to 2.

```
def member(alpha_value, mass=2.0, h=1.0):
    """Psi(x4) at z = pi/4 for the family member alpha (columns = times)."""
    Mn = -mass * Gn[X4] + 3 * h * (2 * alpha_value + 1) * Gn[X4] @ Gn[X0]
    ksq = 9 * h**2 * (2 * alpha_value + 1) ** 2 - mass**2
    kk = np.sqrt(complex(ksq))  # real for ksq > 0, imaginary for ksq < 0
    values = [(np.cosh(kk * t) * np.eye(16) + np.sinh(kk * t) / kk * Mn) @ chi_n
              for t in xs4]
    return np.sin(zeta) ** alpha_value * np.real(np.array(values).T)
```

`member(alpha_value)` evaluates the family at $z = \pi/4$ with $m = 2$, $H = 1$. `np.sqrt(complex(ksq))` is real for $k^2 > 0$ and imaginary for $k^2 < 0$; `np.cosh` and `np.sinh` of an imaginary argument give cos and i sin, so the formula works in both cases; `np.real` drops the zero imaginary parts. The result is a table with one column per time (`.T` transposes).

```
growing, oscillating = member(0.0), member(-0.5)
S_growing = np.einsum("it,ij,jt->t", growing, Cn, growing)  # Psi^T C Psi
S_oscillating = np.einsum("it,ij,jt->t", oscillating, Cn, oscillating)
report("S of the alpha = 0 member", f"{S_growing[0]:.6f}")
report("S of the alpha = -1/2 member", f"{S_oscillating[0]:.6f}")
check(np.allclose(S_growing, -2.0, atol=1e-9 * np.abs(growing).max() ** 2)
      and np.allclose(S_oscillating, -2.0 / np.sin(zeta), atol=1e-9),
      "S is constant along x4 for both members (-2 and -2/sin z)")
```

The growing member ($\alpha = 0$) and the oscillating member ($\alpha = -\frac{1}{2}$). `np.einsum("it,ij,jt->t", ...)` computes $\Psi^T C \Psi$ at every time t (a sum over the repeated indices i and j). The check requires the constant values -2 and $-2/\sin z$ (for the growing member the tolerance grows with the size of Ψ, because rounding errors grow with it). **Out [22]:** `RESULT S of the alpha = 0 member = -2.000000`, `RESULT S of the alpha = -1/2 member = -2.828427` and the PASS line.

In [23]: Figure 07b.5.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.6))
for A in range(16):
    if np.abs(growing[A]).max() > 1e-12:  # draw only the nonzero components
        axes[0].semilogy(xs4, np.abs(growing[A]), label=f"$|\\Psi_{{{A + 1}}}|$")
    if np.abs(oscillating[A]).max() > 1e-12:
        axes[1].plot(xs4, oscillating[A], label=f"$\\Psi_{{{A + 1}}}$")
axes[0].set_title("$\\alpha = 0$: growth like $e^{\\sqrt{5}x_4}$")
axes[1].set_title("$\\alpha = -1/2$: oscillation")
axes[2].plot(xs4, S_growing, label="$\\alpha = 0$")
axes[2].plot(xs4, S_oscillating, "--", label="$\\alpha = -1/2$")
axes[2].set_ylim(-3.5, 0.5)
axes[2].set_title("scalar density $S = \\Psi^T C\\Psi$")
for ax in axes:
    ax.set_xlabel("time $x_4$")
    ax.legend(fontsize=7)
axes[0].set_ylabel("value at $z = \\pi/4$")
fig.tight_layout()
save_figure(fig, "exact_solutions",
...)
```

Three panels: the size of the nonzero components of the growing member on a logarithmic axis, the nonzero components of the oscillating member, and the two scalar densities. `f"$|\\Psi_{{{A + 1}}}|$"` writes the component number as a LaTeX subscript (three braces: two give one literal brace, the third encloses the value). **What Figure 07b.5 shows.** Left: straight rising lines on the logarithmic axis, growth like $e^{\sqrt{5}x_4}$. Middle: oscillations of period π (angular frequency 2). Right: both scalar densities are flat lines, at -2 and at -2.83.

In [24]: the exact solution fails the wrong equations, Figure 07b.6.

```
Mn = -2.0 * Gn[X4] + 3.0 * Gn[X4] @ Gn[X0]  # alpha = 0, m = 2, H = 1
kk = np.sqrt(5.0)
psi_t = np.array([(np.cosh(kk * t) * np.eye(16) + np.sinh(kk * t) / kk * Mn) @ chi_n
                  for t in xs4])
dpsi_t = np.array([(kk * np.sinh(kk * t) * np.eye(16) + np.cosh(kk * t) * Mn) @ chi_n
                   for t in xs4])
full = np.array([Gn[X4] @ dp + 3.0 * Gn[X0] @ p - 2.0 * p
                 for p, dp in zip(psi_t, dpsi_t)])
no_gravity = np.array([Gn[X4] @ dp - 2.0 * p for p, dp in zip(psi_t, dpsi_t)])
wrong_mass = np.array([Gn[X4] @ dp + 3.0 * Gn[X0] @ p + 2.0 * p
                       for p, dp in zip(psi_t, dpsi_t)])
size = np.abs(psi_t).max(axis=1)  # the size of Psi at each time
relative_full = np.abs(full).max(axis=1) / size
check(relative_full.max() < 1e-12, "the full field equation holds to rounding "
      "(relative residual below 1e-12 at all 201 times)")
check(np.allclose(np.abs(no_gravity).max(axis=1), 3.0 * size),
      "without the term 3 H gamma^(x0) Psi the residual is 3 H |Psi|")
```

For the growing member, `psi_t` and `dpsi_t` are Ψ and its exact derivative $\partial_4\Psi = (k\sinh(kx_4) + \cosh(kx_4)M)\chi_0$ at the 201 times ($\partial_0\Psi = 0$ for $\alpha = 0$). `full` is the residual $\gamma^{(x_4)}\partial_4\Psi + 3H\gamma^{(x_0)}\Psi - m\Psi$, `no_gravity` the residual without the gravitational term, `wrong_mass` the residual of the equation with the mass m replaced by $-m$ (the term $-m\Psi$ becomes $+m\Psi$). `size` is the largest component of Ψ at each time. The first check: the full residual is below 10^{-12} of the size at every time; the second: without the gravitational term the residual is $3H$ times the size.

```
fig, ax = plt.subplots()
ax.semilogy(xs4, np.abs(no_gravity).max(axis=1),
            label="without $3H\\gamma^{(x_0)}\\Psi$")
ax.semilogy(xs4, np.abs(wrong_mass).max(axis=1), "--", label="mass $m \\to -m$")
ax.semilogy(xs4, size, ":", color="black", label="size of $\\Psi$")
ax.set_xlabel("time $x_4$")
ax.set_ylabel("largest component of the residual")
ax.set_title("The exact solution fails the wrong equations")
ax.legend(fontsize=8)
save_figure(fig, "residuals",
...)
```

One panel with the two wrong residuals and the size of Ψ on a logarithmic axis. **Out [24]:** two PASS lines and the figure. **What Figure 07b.6 shows.** Both wrong residuals grow as fast as the solution itself (parallel lines on the logarithmic axis); the residual without the gravitational term lies a factor 3 above the size of Ψ. The correct residual, below 10^{-12} of the size, is too small to be drawn.

In [25]: the last check.

```
names = ["scale_factors", "gamma_omega_per_direction", "lagrangian_monomials",
         "coupling_map", "exact_solutions", "residuals"]
files = [f"{FIGURE_FOLDER}/07b_{k}_{name}.png" for k, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 6 figure files of this notebook exist")
all_checks_passed()
```

The six figure files are checked and the last line printed. **Out [25]:** `PASS all 6 figure files of this notebook exist` and `ALL 47 CHECKS PASSED (notebook 07b)`.

7.28 Real fields and the Majorana-type negative control

Real fields. A **real field** is a column whose components are their own conjugates: $\Phi^* = \Phi$ for ordinary numbers, and for Grassmann components a column of real generators, $\theta^* = \theta$ (no separate conjugate generators at all). For a real column the row $\Phi^\dagger$ is just the transpose Φ^T. Real fields matter in this book for two reasons: the author's own notebook wrote its Lagrangian for a real column, and the question "what is charge conjugation?" has a surprising answer for real fields (Section 7.29).

The author's Majorana-type Lagrangian. The author's notebook used a real column Θ of 16 components and, with the canonical connection, the Lagrangian

$$L_g = \sqrt{|g|}\,[\Theta^T C\gamma^\mu D_\mu\Theta + (\text{a number})\,\Theta^T C\Theta],$$

built with the transpose Θ^T instead of the Dirac adjoint (a Lagrangian of this kind is called **Majorana-type**, after the physicist Ettore Majorana, who studied real spinor fields). If the components anticommute, as the components of dirac16complex do, this Lagrangian describes nothing:

- Its mass-type term vanishes: $\Theta^T C\Theta = 0$, because C is symmetric (Section 7.12).
- Its kinetic term is a total derivative. Three identities show it. First, for an antisymmetric matrix A (here $A = C\gamma^\mu$, which may depend on the coordinates) and an odd column, $(\partial_\mu\Theta)^T A\Theta = \Theta^T A\,\partial_\mu\Theta$: exchanging the two odd factors gives a minus sign, and the transpose of A another one. So the product rule gives $\partial_\mu(\Theta^T A\Theta) = 2\,\Theta^T A\,\partial_\mu\Theta + \Theta^T(\partial_\mu A)\Theta$, that is

$$\Theta^T C\gamma^\mu\partial_\mu\Theta = \tfrac{1}{2}\partial_\mu(\Theta^T C\gamma^\mu\Theta) - \tfrac{1}{2}\Theta^T\partial_\mu(C\gamma^\mu)\Theta.$$

Second, in $\Theta^T C\gamma^\mu\Omega_\mu\Theta$ only the antisymmetric part of the matrix $N = C\gamma^\mu\Omega_\mu$ survives (Section 7.12), and $N^T = \Omega_\mu^T(\gamma^\mu)^T C = \Omega_\mu^T(-C\gamma^\mu) = C\Omega_\mu\gamma^\mu$ (using $(\gamma^\mu)^T C = -C\gamma^\mu$ and $\Omega_\mu^T C = -C\Omega_\mu$), so $\frac{1}{2}(N - N^T) = \frac{1}{2}C[\gamma^\mu, \Omega_\mu]$ with the **commutator** $[X, Y] = XY - YX$:

$$\Theta^T C\gamma^\mu\Omega_\mu\Theta = \tfrac{1}{2}\Theta^T C[\gamma^\mu, \Omega_\mu]\Theta.$$

Third, multiply both by $\sqrt{|g|}$, use $\sqrt{|g|}\,\partial_\mu X = \partial_\mu(\sqrt{|g|}X) - (\partial_\mu\sqrt{|g|})X$ and collect:

$$\sqrt{|g|}\,\Theta^T C\gamma^\mu D_\mu\Theta = \tfrac{1}{2}\partial_\mu(\sqrt{|g|}\,\Theta^T C\gamma^\mu\Theta) - \tfrac{1}{2}\Theta^T C\Big(\partial_\mu(\sqrt{|g|}\gamma^\mu) - \sqrt{|g|}\,[\gamma^\mu, \Omega_\mu]\Big)\Theta.$$

The last bracket is zero: in the author's metric $[\gamma^\mu, \Omega_\mu] = 2\gamma^\mu\Omega_\mu$ (they anticommute) and $\partial_\mu(\sqrt{|g|}\gamma^\mu) = 2\sqrt{|g|}\gamma^\mu\Omega_\mu$ (Section 7.18); in a general metric this is the covariant constancy of the gammas (Wolfram check `gamma_covariantly_constant`). So L_g is the total derivative of

$\frac{1}{2}\sqrt{|g|}\,\Theta^T C\gamma^\mu\Theta$, and by Section 7.4 all its Euler-Lagrange expressions vanish identically: no field equation at all (PROVED; Notebook 07c computes it in the author's metric, In [5] and In [6]: L_g has 136 monomials, it equals its total-derivative form exactly, and 0 of its 16 Euler-Lagrange expressions are nonzero; the Revision checks are listed in the table at the end of this section).

This is the **negative control** of the Revision record: a test designed to show what does NOT work. It is the reason why the Revision Lagrangian of dirac16complex is written for a complex field with $\bar\Psi = \Psi^\dagger C$ (Section 7.19), and Section 7.13 showed the same mechanism with two components.

Real commuting fields. For ordinary numbers the roles are exchanged (Section 7.12): $\Phi^T C\gamma^\mu\Phi = 0$, while $\Phi^T C\Phi$ survives, and the kinetic term is not a total derivative. Varying Φ_A in $\sqrt{|g|}\,\Phi^T C\gamma^\mu D_\mu\Phi$: the derivative with respect to Φ_A is $\sqrt{|g|}\,[(C\gamma^\mu\partial_\mu\Phi)_A + (C\gamma^\mu\Omega_\mu\Phi)_A + (N^T\Phi)_A]$ (each factor Φ contributes once), the derivative with respect to $\partial_\mu\Phi_A$ is $\sqrt{|g|}\,((C\gamma^\mu)^T\Phi)_A = -\sqrt{|g|}(C\gamma^\mu\Phi)_A$, and with $N^T = C\Omega_\mu\gamma^\mu$ and $\partial_\mu(\sqrt{|g|}\gamma^\mu) = 2\sqrt{|g|}\gamma^\mu\Omega_\mu$ the Euler-Lagrange expression becomes

$$\sqrt{|g|}\,C\big[2\gamma^\mu\partial_\mu\Phi + \gamma^\mu\Omega_\mu\Phi + \Omega_\mu\gamma^\mu\Phi + 2\gamma^\mu\Omega_\mu\Phi\big] = 2\sqrt{|g|}\,C\gamma^\mu D_\mu\Phi$$

(using $\Omega_\mu\gamma^\mu = -\gamma^\mu\Omega_\mu$): a massless field equation $\gamma^\mu D_\mu\Phi = 0$ in all 16 components (PROVED; Notebook 07c, In [7]; `python-field-theory.json`, check `negative_control_majorana_commuting_contrast`, "16 of 16 components"; `wolfram-field-theory.json`, check `Majorana_Lg_commuting_control`).

The Revision Lagrangian for a real commuting field. Put $\Psi = \Phi$ real and commuting into the Lagrangian of Section 7.19. Then $\bar\Phi = \Phi^T C$, and the second half of the kinetic term equals the first: $(D_\mu\bar\Phi)\gamma^\mu\Phi = (D_\mu\Phi)^T C\gamma^\mu\Phi = \Phi^T(C\gamma^\mu)^T D_\mu\Phi = -\Phi^T C\gamma^\mu D_\mu\Phi$ (a single number equals its own transpose, and $C\gamma^\mu$ is antisymmetric). So $K = \Phi^T C\gamma^\mu D_\mu\Phi$ and

$$\mathcal{L}[\Phi] = \sqrt{|g|}\,\big[\Phi^T C\gamma^\mu D_\mu\Phi - m\,\Phi^T C\Phi - \tfrac{\lambda}{2}(\Phi^T C\Phi)^2\big]\,:$$

the author's L_g with a mass term and the interaction. Its Euler-Lagrange expressions are $2\sqrt{|g|}\,\big(C(\gamma^\mu D_\mu\Phi - (m+\lambda S)\Phi)\big)_A$ (the kinetic part as just computed; $\partial(\Phi^T C\Phi)/\partial\Phi_A = 2(C\Phi)_A$ because C is symmetric): a real commuting field obeys the same field equation as the complex one (PROVED; Notebook 07c, In [12]). Because the gammas, C and the spinor connection are real (`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, checks `representation_real` and `spinor_connection_real`), the operator $\gamma^\mu D_\mu$ is real, the complex conjugate of a solution is again a solution, and real fields are a consistent restriction of dirac16complex00.

A real field carries no charge. The phase change $\Psi \to e^{i\alpha}\Psi$ leaves the Lagrangian unchanged (every term contains as many factors of $\Psi^\dagger$ as of Ψ), and the conserved quantity that belongs to it is the **current** $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, whose time component $J^{x_4} = \Psi^\dagger B\Psi$, with $B = -iC\gamma^{(x_4)}$, is the **charge density** (Chapter 5; Chapter 21 proves its conservation). For a real commuting field $J^\mu = -i\Phi^T C\gamma^\mu\Phi = 0$ identically, because $C\gamma^\mu$ is antisymmetric (Section 7.12): a real field has no charge (PROVED; Notebook 07c, In [11]; `charge-conjugation-and-u1.json`, check `real_fields_charge_conjugation`).

Where the negative control is verified (the theory reports in the folder `Revision/theory/reports`):

statement	status	where it is verified
L_g of a real Grassmann field is a total derivative: no field equation; $\Theta^T C \Theta = 0$	PROVED; Notebook 07c, In [5] and In [6]	`python-field-theory.json`, check `negative_control_majorana_grassmann_total_derivative`; `wolfram-field-theory.json`, check `Majorana_Lg_total_derivative_grassmann`
L_g of a real commuting field gives $2\sqrt{\lvert g\rvert}\, C\gamma^\mu D_\mu \Phi$ in 16 of 16 components	PROVED; In [7]	`python-field-theory.json`, check `negative_control_majorana_commuting_contrast`; `wolfram-field-theory.json`, check `Majorana_Lg_commuting_control`

7.29 Charge conjugation is a matrix

What charge conjugation must do. A **charge conjugation** is a map that turns a field into a field of the opposite charge, the map that exchanges matter and antimatter. For a spinor field it is a MATRIX map

$$\Psi^c = \mathcal{C}\, \bar{\Psi}^T,$$

with a fixed 16×16 matrix $\mathcal{C}$, the **charge-conjugation matrix**. Since $\bar{\Psi}^T = (\Psi^\dagger C)^T = C^T \Psi^* = C\Psi^*$ (C is symmetric), it reads $\Psi^c = M\Psi^*$ with the constant matrix $M = \mathcal{C}C$. It must turn every solution of the field equation into a solution, of the same equation or of the equation with the mass reversed. The author's gammas are REAL. So for a REAL field $\Psi^* = \Psi$: the plain complex conjugate of a real field is the field itself, and plain complex conjugation cannot exchange anything. Everything therefore rests on the matrix.

The condition on M, line by line. Let Ψ solve $(\gamma^\mu D_\mu - V)\Psi = 0$ with a real V. Then

$$(\gamma^\mu D_\mu - V)\Psi^* = 0$$

(take the complex conjugate: γ^μ, Ω_μ and V are real, so only Ψ changes);

$$M(\gamma^\mu D_\mu - V)M^{-1}\,(M\Psi^*) = 0$$

(insert $M^{-1}M = 1$ in front of Ψ^* and multiply from the left by M). If $M\gamma^{(a)}M^{-1} = s\gamma^{(a)}$ for all eight a, with $s = +1$ or $s = -1$, then $M\Omega_\mu M^{-1} = \Omega_\mu$ (each S^{ab} is a product of two gammas, and $s^2 = 1$), M commutes with ∂_μ (it is constant), and $M(\gamma^\mu D_\mu - V)M^{-1} = s\gamma^\mu D_\mu - V = s(\gamma^\mu D_\mu - sV)$. So $M\Psi^*$ solves the equation with V replaced by sV: the same mass for $s = +1$, the reversed mass for $s = -1$. We must find all matrices M with

$$M\gamma^{(a)} = s\,\gamma^{(a)}M \qquad \text{for all } a.$$

All solutions, line by line. The 256 products $\gamma_I = \gamma^{(a_1)} \cdots \gamma^{(a_k)}$ with $a_1 < \cdots < a_k$ (all subsets I of the eight directions, the empty product being 1) are a **basis** of all 16×16 matrices: every matrix is exactly one combination $\sum_I c_I \gamma_I$. *Proof.* Each gamma is a signed permutation matrix with $(\gamma^{(a)})^T = (\gamma^{(a)})^{-1}$, so $\gamma_I^T = \gamma_I^{-1}$ and $\gamma_I^T \gamma_J = \pm\gamma_K$, where K is the set of directions in exactly one of I, J (the shared gammas square to ± 1). The trace of γ_K is 0 for every nonempty K: if K has an even number of elements, take a in K; by Rule 1 of Chapter 5 γ_K anticommutes with $\gamma^{(a)}$, so $\operatorname{tr}\gamma_K = \operatorname{tr}\big((\gamma^{(a)})^{-1}\gamma_K\gamma^{(a)}\big) = -\operatorname{tr}\gamma_K$ (the trace does not change when a matrix is moved from the front to the back of a product); if K has an odd number of elements fewer than 8, take a not in K, and the same argument applies. So $\operatorname{tr}(\gamma_I^T\gamma_J) = 16\,\delta_{IJ}$: the 256 matrices are **orthogonal** (for the scalar product $\operatorname{tr}(X^TY)$ of matrices), hence independent, and 256 independent matrices span the 256-dimensional space of 16×16 matrices. □ (Notebook 07c checks the orthogonality with exact integers, In [8].)

Next, each basis product either commutes or anticommutes with a given $\gamma^{(a)}$: moving $\gamma^{(a)}$ through the k factors of γ_I costs $(-1)^k$ if a is not in I and $(-1)^{k-1}$ if it is (it passes $k-1$ other factors and commutes with itself) (Rule 1 of Chapter 5). So $\gamma^{(a)}\gamma_I(\gamma^{(a)})^{-1} = \epsilon_I(a)\gamma_I$ with a sign $\epsilon_I(a)$. Write $M = \sum_I c_I\gamma_I$. The condition $\gamma^{(a)}M(\gamma^{(a)})^{-1} = sM$ (the condition above, multiplied by $(\gamma^{(a)})^{-1}$ from the right and divided by s, since $1/s = s$) becomes $\sum_I \epsilon_I(a)c_I\gamma_I = \sum_I sc_I\gamma_I$, and because the γ_I are independent, $\epsilon_I(a)c_I = sc_I$ for every I and every a. So $c_I \neq 0$ only for products that have the sign s with ALL eight gammas.

Count, for a product of k gammas, the number of gammas it commutes with. If k is even, it commutes with the $8-k$ gammas not in it and anticommutes with the k in it: $8-k$. If k is odd, it anticommutes with those not in it and commutes with those in it: k. Commuting with all eight needs $8-k=8$ ($k=0$, the identity) or $k=8$ with k odd (impossible): only the identity. Anticommuting with all eight needs $8-k=0$ ($k=8$, the product of all eight, which is $\pm\Gamma$) or $k=0$ with k odd (impossible): only Γ. Hence (PROVED; Notebook 07c, In [8] and In [9], Figure 07c.2; `charge-conjugation-and-u1.json`, checks `intertwiners_same_mass` and `intertwiners_reversed_mass`):

$$M = c\cdot 1 \quad (s=+1,\ \text{same mass}), \qquad M = c\cdot\Gamma \quad (s=-1,\ \text{mass reversed}),$$

with a constant c. With $M = \mathcal{C}C$ and $C^{-1} = C$ (because $C^2 = 1$) the two **charge-conjugation matrices** are, up to a constant factor,

$$\mathcal{C}_+ = C, \quad \Psi^c = \Psi^*; \qquad\qquad \mathcal{C}_- = \Gamma C, \quad \Psi^c = \Gamma\Psi^*.$$

Their defining properties. For $\mathcal{C}_+ = C$, line by line:

$$\mathcal{C}_+^{-1}\gamma^{(a)}\mathcal{C}_+ = (C\gamma^{(a)})\,C = -(C\gamma^{(a)})^TC = -(\gamma^{(a)})^TC^TC = -(\gamma^{(a)})^T$$

($C^{-1} = C$; $C\gamma^{(a)}$ is antisymmetric; the transpose of a product is the product of the transposes in reversed order; $C^TC = C^2 = 1$). For $\mathcal{C}_- = \Gamma C$: $\mathcal{C}_-^{-1} = C\Gamma$ (both square to 1), and $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}$, so $\mathcal{C}_-^{-1}\gamma^{(a)}\mathcal{C}_- = C\Gamma\gamma^{(a)}\Gamma C = -C\gamma^{(a)}C = +(\gamma^{(a)})^T$. Both matrices are real (PROVED; Notebook 07c, In [10]; checks `charge_conjugation_matrix_plus` and `charge_conjugation_matrix_minus`). A **Majorana (reality) condition** $\Psi = M\Psi^*$ is consistent only if $MM^* = 1$ (apply it twice: $\Psi^* = M^*\Psi$, so $\Psi = MM^*\Psi$); this holds for $M = 1$ and for $M = \Gamma$ (real, $\Gamma^2 = 1$), so both reality

conditions are consistent (check `majorana_conditions_consistent`). Chapter 5 derives the same matrices from the algebra alone.

What they do to a real field. For a real column Φ:

$$\Phi^c_+ = \mathcal{C}_+\bar{\Phi}^T = C\,(\Phi^T C)^T = CC\Phi = \Phi, \qquad \Phi^c_- = \mathcal{C}_-\bar{\Phi}^T = \Gamma CC\Phi = \Gamma\Phi.$$

$\mathcal{C}_+$ does nothing to a real field: a real field is its own conjugate, in agreement with $J^\mu = 0$ (Section 7.28). The only nontrivial map of real fields is the REAL matrix $\Gamma = \mathrm{diag}(-I_8, I_8)$. It keeps the scalar density and reverses every kinetic matrix:

$$\Gamma^T C\Gamma = C, \qquad \Gamma^T C\gamma^{(a)}\Gamma = -C\gamma^{(a)}$$

($\Gamma^T = \Gamma$ is diagonal; it commutes with C, a product of four gammas, and anticommutes with $\gamma^{(a)}$; and $\Gamma^2 = 1$). It also commutes with every Ω_μ (a combination of products of two gammas), so $D_\mu(\Gamma\Phi) = \Gamma D_\mu\Phi$. Therefore $K[\Gamma\Phi] = -K[\Phi]$ and $S[\Gamma\Phi] = S[\Phi]$, and

$$\mathcal{L}_{m,\lambda}[\Gamma\Phi] = \sqrt{|g|}\,\big[-K - mS - \tfrac{\lambda}{2}S^2\big] = -\sqrt{|g|}\,\big[K - (-m)S - \tfrac{(-\lambda)}{2}S^2\big] = -\mathcal{L}_{-m,-\lambda}[\Phi]\,:$$

the matrix Γ, together with $(m,\lambda) \to (-m,-\lambda)$, maps solutions to solutions; a real solution of mass m goes to a real solution of mass $-m$ (PROVED; Notebook 07c, In [11] and In [12], and numerically on an exact solution, In [13], Figure 07c.4; `charge-conjugation-and-u1.json`, check `real_fields_charge_conjugation`). This is the form that the pairing theorem T1 takes for real fields; Chapter 18 proves T1 for both fields in every gravitational field. For a COMPLEX field the same matrix map $\Psi \to \Gamma\Psi$ (without any conjugation) also reverses the current: since Γ is real, $(\Gamma\Psi)^\dagger = \Psi^\dagger\Gamma^T$, and $J^a[\Gamma\Psi] = -i\Psi^\dagger\Gamma^T C\gamma^{(a)}\Gamma\Psi = +i\Psi^\dagger C\gamma^{(a)}\Psi = -J^a[\Psi]$ by the identity just proved; for a real field both sides are zero.

The signs of S and J under the two conjugations. For a complex field and $\Psi' = M\Psi^*$ with the real $M = 1$ or $M = \Gamma$, the row is $\Psi'^\dagger = \Psi^T M^T$, and both matrices satisfy $M^T C M = C$, while $M^T C\gamma^{(a)} M = \pm C\gamma^{(a)}$ (+ for $M = 1$, − for $M = \Gamma$). So

$$S' = \Psi^T C\Psi^* = \sum_{A,B} \Psi_A C_{AB}\Psi^*_B = \sigma \sum_{A,B} \Psi^*_B C_{BA}\Psi_A = \sigma S$$

(exchange the two factors, with $\sigma = +1$ for commuting and $\sigma = -1$ for Grassmann components; then $C_{AB} = C_{BA}$), and in the same way, with the antisymmetric $C\gamma^{(a)}$, $-i\Psi^T C\gamma^{(a)}\Psi^* = -\sigma J^a$ for $M = 1$, and the opposite sign for $M = \Gamma$. In a table:

map	**commuting components**	**Grassmann components (classical)**
$\Psi \to \Psi^*$ (matrix $\mathcal{C}_+$)	$(S,J) \to (S,-J)$	$(S,J) \to (-S,+J)$
$\Psi \to \Gamma\Psi^*$ (matrix $\mathcal{C}_-$)	$(S,J) \to (S,+J)$	$(S,J) \to (-S,-J)$

(PROVED here; it reproduces `charge-conjugation-and-u1.json`, check `bilinears_under_charge_conjugation`.) For the commuting complex field dirac16complex00 the map built with $\mathcal{C}_+$

keeps the mass and reverses the charge. For the Grassmann field the classical signs differ. The record states that **normal ordering** (the subtraction of the vacuum value of an operator, its value in the state with no quanta; Section 5.34) adds no sign: it subtracts a number, so a bilinear that changes as $X \to sX + c$ has the normal-ordered form changing as $:X: \to s\ :X:$, and the quantised bilinears keep the signs of the classical Grassmann column (the record makes no Fock-space computation for this; Section 5.34 confirms it on the Fock space). The record also states that for the QUANTISED field the conjugation that preserves the canonical anticommutator $\{\Psi, \Psi^\dagger\} = B\delta$ is $\Psi \to \Gamma\Psi^{\dagger T}$, which reverses the mass (both quoted here from the same report, checks `bilinears_under_charge_conjugation` and `quantum_charge_conjugation_unitary_type`; Section 5.34 derives both; Sections 10.15 and 21.25 derive the second, and Section 21.26 confirms the first for the charge).

What this does and does not say. The two charge-conjugation matrices and the mass reversal by Γ are exact maps between the solutions of two equations (PROVED, with the checks named above). They do not say how any solution comes about: nothing in this chapter says that solutions, or universes, are created, in pairs or otherwise. Nor do they explain the observed excess of matter over antimatter: as built, the theory has no baryons, no violation of baryon number, no CP violation and no departure from equilibrium, the ingredients that such an explanation needs; Chapter 21 treats this question from zero, and Chapter 20 states exactly what the pairing theorems do and do not establish.

7.30 Example: Notebook 07c: real fields and the charge-conjugation matrices

Notebook 07c computes Sections 7.28 and 7.29 exactly. It reads the gammas, C and Γ from the Revision record and checks that they are real and that B is imaginary and Hermitian; rebuilds the spin connection of the author's metric and checks that it is real; builds a jet algebra for a real field; computes the Majorana-type Lagrangian for real Grassmann and real commuting components (no field equation in the first case, 16 equations in the second); finds all matrices that turn solutions into solutions from the 256 products of gammas; checks the two charge-conjugation matrices and the two reality conditions; shows that $\mathcal{C}_+$ is the identity on a real field and that $J = 0$; and checks the mass reversal by Γ in the author's metric and on an exact solution. Every result that the Revision record also contains is checked against it. It draws four figures and ends with the line ALL 23 CHECKS PASSED (notebook 07c).

7.31 How to run Notebook 07c

Step 1. What this notebook does and what it needs.

Notebook 07c (Real fields: the Majorana-type control and the matrix Gamma) is the file `Revision/textbook/notebooks/07c_real_fields.ipynb` of the repository Dirac_claude. It shows in the author's metric, with exact computer algebra, that the Majorana-type Lagrangian of the author's notebook is a total derivative with no field equation for a real anticommuting 16-component field but not for real commuting components; that for a real field plain complex conjugation is the identity and the conserved current vanishes; that every matrix that maps solutions to solutions in the conjugated form is a multiple of the identity (same mass) or of the chirality matrix Gamma (mass reversed), which gives the two charge-conjugation matrices C and Gamma C; and that the real matrix Gamma maps every real solution of mass m to a real solution of mass minus m; four

teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 07c_real_fields.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 07c_real_fields.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 07c_real_fields.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/07c.captions.json`
- `Revision/textbook/figures/07c_1_majorana_control.png`
- `Revision/textbook/figures/07c_2_commutation_pattern.png`
- `Revision/textbook/figures/07c_3_gamma_map_matrices.png`
- `Revision/textbook/figures/07c_4_mass_reversal.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 4 figure files of this notebook exist
ALL 23 CHECKS PASSED (notebook 07c)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/07c_real_fields.ipynb` inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 07c_real_fields.ipynb
python -m nbconvert --execute --inplace 07c_real_fields.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/...": the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory Revision/algebra Revision/lead_checks
```

7.32 Notebook 07c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/07c_real_fields.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook is about REAL fields: columns of 16 components that are their own
complex conjugates. They matter for two reasons. The author's own notebook wrote its
Lagrangian for a real column; and the question "what is charge conjugation?" has a
surprising answer for real fields. The notebook

- computes, in the author's metric, the Majorana-type Lagrangian $L_g = \sqrt{|g|}\,
  \Theta^T C\gamma^\mu D_\mu\Theta$ of the author's notebook for a real column
  $\Theta$ of anticommuting (Grassmann) components and shows that it is a total
  derivative: all 16 Euler-Lagrange expressions vanish, there is no field equation;
- shows that for real COMMUTING components the same $L_g$ is not a total
  derivative: it gives a field equation in all 16 components;
- shows that the Revision Lagrangian restricted to a real commuting field gives the
  field equation $\gamma^\mu D_\mu\Phi = (m + \lambda S)\Phi$ and that its conserved
  current vanishes identically;
- finds ALL matrices $M$ that turn solutions $\Psi$ into solutions $M\Psi^*$: only
  multiples of the identity (same mass) and of the chirality matrix $\Gamma$ (mass
  reversed). Hence the two charge-conjugation matrices are $\mathcal{C}_+ = C$ and
  $\mathcal{C}_- = \Gamma C$;
- shows that for a real field plain complex conjugation, and with it
  $\mathcal{C}_+$, acts as the identity, so it cannot exchange matter and
  antimatter, while the real matrix $\Gamma$ maps every real solution of mass $m$ to
  a real solution of mass $-m$.

Every result that the Revision record also contains is checked against it
(Revision/theory/reports/ and Revision/lead_checks/reports/). Four figures are
drawn.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 07c (Real fields: the Majorana-type control and the matrix Gamma) is the file
‘Revision/textbook/notebooks/07c_real_fields.ipynb‘ of the repository Dirac_claude. It
shows in the author's metric, with exact computer algebra, that the Majorana-type
Lagrangian of the author's notebook is a total derivative with no field equation for a
real anticommuting 16-component field but not for real commuting components; that for a
real field plain complex conjugation is the identity and the conserved current vanishes;
that every matrix that maps solutions to solutions in the conjugated form is a multiple
of the identity (same mass) or of the chirality matrix Gamma (mass reversed), which gives
the two charge-conjugation matrices C and Gamma C; and that the real matrix Gamma maps
every real solution of mass m to a real solution of mass minus m; four teaching plots. It
needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
does not need Rust.
```

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home

folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 07c_real_fields.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 07c_real_fields.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 07c_real_fields.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/07c.captions.json‘
- ‘Revision/textbook/figures/07c_1_majorana_control.png‘
- ‘Revision/textbook/figures/07c_2_commutation_pattern.png‘

- ‘Revision/textbook/figures/07c_3_gamma_map_matrices.png‘
- ‘Revision/textbook/figures/07c_4_mass_reversal.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all 4 figure files of this notebook exist
ALL 23 CHECKS PASSED (notebook 07c)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/07c_real_fields.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 07c_real_fields.ipynb
  python -m nbconvert --execute --inplace 07c_real_fields.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event

  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "ValueError: ... is not a passing check of Revision/...": the Revision record files of your copy of the repository differ from the committed ones; restore them with the command below (run in the repository folder) and run the notebook again.

```
git restore Revision/theory Revision/algebra Revision/lead_checks
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 07c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 07c (Real fields: the Majorana-type control and the matrix Gamma) is the file
# Revision/textbook/notebooks/07c_real_fields.ipynb of the repository Dirac_claude. It
# shows in the author’s metric, with exact computer algebra, that the Majorana-type
# Lagrangian of the author’s notebook is a total derivative with no field equation for a
# real anticommuting 16-component field but not for real commuting components; that for a
# real field plain complex conjugation is the identity and the conserved current
# vanishes; that every matrix that maps solutions to solutions in the conjugated form is
# a multiple of the identity (same mass) or of the chirality matrix Gamma (mass
# reversed), which gives the two charge-conjugation matrices C and Gamma C; and that the
# real matrix Gamma maps every real solution of mass m to a real solution of mass minus
# m; four teaching plots. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
```

```
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
```

```
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 07c_real_fields.ipynb
```

```
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 07c_real_fields.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 07c_real_fields.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/07c.captions.json
# - Revision/textbook/figures/07c_1_majorana_control.png
# - Revision/textbook/figures/07c_2_commutation_pattern.png
# - Revision/textbook/figures/07c_3_gamma_map_matrices.png
# - Revision/textbook/figures/07c_4_mass_reversal.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all 4 figure files of this notebook exist
#     ALL 23 CHECKS PASSED (notebook 07c)
#
# and the notebook must show 4 figures below the cells that draw them.
```

```
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/07c_real_fields.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 07c_real_fields.ipynb
#     python -m nbconvert --execute --inplace 07c_real_fields.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "ValueError: ... is not a passing check of Revision/...": the Revision record files
#   of your copy of the repository differ from the committed ones; restore them with the
#   command below (run in the repository folder) and run the notebook again.
#     git restore Revision/theory Revision/algebra Revision/lead_checks
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
```

```
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "07c"  # this notebook: chapter 07, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
```

```
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
```

```
    "defined.")
```

Out [1]:

```
Set-up of notebook 07c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Real field**: a column $\Phi$ whose 16 components are real ($\Phi^* = \Phi$);
  for Grassmann components, a column of real generators ($\theta^* = \theta$).
- **Complex conjugation**: replacing every number by its complex conjugate
  ($i \to -i$). On a real column it changes nothing: it is the identity.
- **Charge conjugation**: a map that turns a field into a field of the opposite
  charge. For a spinor field it is a MATRIX map $\Psi^c = \mathcal{C}\,
  \bar\Psi^T$ with a fixed $16\times16$ matrix $\mathcal{C}$, the
  **charge-conjugation matrix**; since $\bar\Psi^T = (\Psi^\dagger C)^T =
  C\Psi^*$, it is $\Psi^c = \mathcal{C}C\,\Psi^*$.
- **Chirality matrix** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$:
  the product of all eight gammas; it anticommutes with every $\gamma^{(a)}$ and
  $\Gamma^2 = 1$.
- **Majorana (reality) condition**: a condition $\Psi = M\Psi^*$ that makes a
  field real in a generalised sense; it is consistent only if $MM^* = 1$.
- **Majorana-type Lagrangian** $L_g$: the Lagrangian of the author's notebook, built
  with the transpose $\Theta^T$ of a real column instead of the Dirac adjoint
  $\bar\Psi = \Psi^\dagger C$.
- **Total derivative**: an expression $\sum_\mu \frac{d}{dx_\mu}V^\mu$; a Lagrangian
  that is a total derivative gives no field equation (its Euler-Lagrange expressions
  vanish identically).
- **Negative control**: a test designed to show that something does NOT work, here
  the Majorana-type Lagrangian; it explains why the Revision record uses the complex
  Dirac-type Lagrangian.
- **Current** $J^a = -i\bar\Psi\gamma^{(a)}\Psi$: the conserved current of the
  phase symmetry $\Psi \to e^{i\alpha}\Psi$; its time component gives the charge.
- **Commute, anticommute**: $XY = YX$ or $XY = -YX$ for two matrices.
- **Jet**, **total derivative on jets**, **coefficient ring**: as in Notebook 07b:
  the field components and their derivatives at one point are symbols, and every
  coefficient is written with $E = e^{a_4}$, $s = \sin^{1/6}z$, $c = \cos z$,
  $A_1 = a_4'$, $A_2$, $A_3$ (higher derivatives of $a_4$), $H$, $m$, $\lambda$.
```

Text cell:

```
## 4. The physical and mathematical situation

**The author's Lagrangian.** The author's notebook used a real column of 16
components and the Lagrangian $L_g = \sqrt{|g|}\,[\Theta^T C\gamma^\mu D_\mu\Theta +
(\text{number})\,\Theta^T C\Theta]$. If the components anticommute, as the
components of a fermion field must, then $\Theta^T C\Theta = 0$ because $C$ is
symmetric, and $\Theta^T C\gamma^\mu D_\mu\Theta$ is a total derivative because
$C\gamma^{(a)}$ is antisymmetric (Notebook 07a showed both rules in small form). So
```

```
$L_g$ describes nothing: this is the negative control of the Revision record, and
the reason why the Revision Lagrangian uses the complex field and $\bar\Psi =
\Psi^\dagger C$.

**Charge conjugation is a matrix.** The author's gammas are real. For a REAL
field, $\Psi^* = \Psi$, so plain complex conjugation changes nothing and cannot be
charge conjugation. Charge conjugation is the matrix map
$\Psi^c = \mathcal{C}C\,\Psi^*$, which must send every solution of $(\gamma^\mu
D_\mu - V)\Psi = 0$ ($V = m + U'(S)$, real) to a solution of the equation with $sV$,
where $s = +1$ (same mass) or $s = -1$ (mass reversed). Conjugating the equation and
multiplying by $M = \mathcal{C}C$ shows, line by line, what $M$ must do:

$$(\gamma^\mu D_\mu - V)\Psi = 0 \;\Rightarrow\; (\gamma^\mu D_\mu - V)\Psi^* = 0
\;\Rightarrow\; M(\gamma^\mu D_\mu - V)M^{-1}\,(M\Psi^*) = 0 .$$

The first step takes the complex conjugate (the gammas, $\Omega_\mu$ and $V$ are
real); the second inserts $M^{-1}M = 1$ and multiplies by $M$. The new equation is
the equation with $sV$ exactly when $M\gamma^{(a)}M^{-1} = s\gamma^{(a)}$ for every
$a$ ($\Omega_\mu$ is built from products of two gammas and then follows). Section 9
finds all such $M$: multiples of $1$ ($s = +1$) and of $\Gamma$ ($s = -1$). Hence

$$\mathcal{C}_+ = C^{-1} = C,\qquad \Psi^c = \Psi^*;\qquad\qquad
\mathcal{C}_- = \Gamma C^{-1} = \Gamma C,\qquad \Psi^c = \Gamma\Psi^* .$$

For a real field the first is the identity (a real field is its own conjugate, it
has no charge to reverse), and the second is the real matrix $\Gamma$, which maps a
solution of mass $m$ to a solution of mass $-m$. This is a map between solutions of
two equations; it says nothing about how either solution comes about.
```

Text cell:

```
## 5. The gamma matrices, $C$, $\Gamma$ and $B$ are real or imaginary

The next cell imports the packages, wraps the set-up cell's ‘check‘ so that a PASS
line and its "reproduces" line are printed by one single ‘print‘ call (Jupyter
sends printed text in pieces, and the book's tools must see the two lines
together), defines the helper ‘reproduces‘ (it makes sure that a Revision report
contains a check with the verdict PASS before a PASS line names it), and reads the
gammas, $C$ and $\Gamma$ from Revision/algebra/gammas.json. It checks that all of
them are real, that $C$ is symmetric with $C^2 = 1$, that $\Gamma = \gamma^{(x_0)}
\gamma^{(x_1)}\cdots\gamma^{(x_7)}$ anticommutes with every gamma and $\Gamma^2 =
1$, and that the charge matrix $B = -iC\gamma^{(x_4)}$ is purely imaginary and
Hermitian with $B^2 = 1$. Finally it takes a column of 16 real symbols and checks
that complex conjugation leaves it unchanged.
```

In [2]:

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import re  # regular expressions: read a number from a check's detail text
from itertools import combinations  # all subsets of the eight gammas
```

```
import numpy as np  # integer and floating-point arrays
import sympy as sp  # exact algebra

check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece


PYREP = "Revision/theory/reports/python-field-theory.json"
WLREP = "Revision/theory/reports/wolfram-field-theory.json"
LEAD = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
PYREP_B4 = "Revision/theory/reports/python-field-theory-b4.json"  # the metric with b4


def revision_check(report, name):
    """The check called name of a Revision report: its dictionary (name, verdict,
    detail); stops if it is missing or its verdict is not PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return found[0]


def reproduces(report, name):
    """The text "<report>, check <name>" after making sure the check passes."""
    revision_check(report, name)
    return f"{report}, check {name}"


fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
G = [sp.Matrix(g) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7)
C = sp.Matrix(fixture["C"])
GAMMA = G[0] * G[1] * G[2] * G[3] * G[4] * G[5] * G[6] * G[7]  # chirality
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
X0, X4 = 0, 4  # list positions of x0 and x4
B = -sp.I * C * G[X4]
real = all(v.is_real for M in G + [C, GAMMA] for v in M)
check(real and C.T == C and C * C == I16 and GAMMA == sp.Matrix(fixture["Gamma"])
      and GAMMA * GAMMA == I16 and all(GAMMA * g == -g * GAMMA for g in G),
      "the gammas, C and Gamma are real; C^T = C, C^2 = 1; Gamma^2 = 1 and Gamma "
      "anticommutes with every gamma", record=reproduces(LEAD, "representation_real"))
check(all(sp.re(v) == 0 for v in B) and B.H == B and B * B == I16,
      "B = -i C gamma^(x4) is purely imaginary and Hermitian, B^2 = 1",
      record=reproduces(LEAD, "B_imaginary_hermitian"))
r = sp.Matrix(sp.symbols("r1:17", real=True))  # a real column of 16 numbers
```

```
check(r.conjugate() == r, "complex conjugation leaves a real column unchanged: on "
      "a real field it is the identity")
```

Out [2]:

```
PASS the gammas, C and Gamma are real; C^T = C, C^2 = 1; Gamma^2 = 1 and Gamma
    anticommutes with every gamma
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    representation_real
PASS B = -i C gamma^(x4) is purely imaginary and Hermitian, B^2 = 1
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    B_imaginary_hermitian
PASS complex conjugation leaves a real column unchanged: on a real field it is the
    identity
```

Text cell:

```
## 6. The author's metric and its spin connection, in brief

The next cell rebuilds, exactly as Notebook 07b does step by step, the coefficient
ring, the diagonal frame $f = (c/s^6, Es, Es, Es, 1, s\sqrt{b_4}/E, s\sqrt{b_4}/E,
s\sqrt{b_4}/E)$ of the metric with $b_4$ (in the order $x_0, \dots, x_7$; the
symbol `b4` stands for $b_4(x_4)$ and `b4p`, `b4pp`, `b4ppp` for its first three
derivatives), the Christoffel symbols of
the diagonal metric, the canonical spin connection $\omega_{\mu ab}$, the spinor
connection $\Omega_\mu = \sum_{a<b}\omega_{\mu ab}S^{ab}$ and the coordinate gammas
$\gamma^\mu = \gamma^{(\mu)}/f_\mu$. It then checks two things, for arbitrary
functions $a_4$ and $b_4 > 0$. First, every entry of every $\Omega_\mu$ and
$\gamma^\mu$ is real (real gammas times the real functions $e^{\pm a_4}$,
$\sin^{1/6}z$, $\cot z$, $b_4^{1/2}$, $a_4'$, $b_4'$ and $H$; no imaginary unit $i$
anywhere): the operator $\gamma^\mu D_\mu$ is REAL, so the complex conjugate of a
solution is again a solution, and real fields are a consistent restriction. Second,
as a consistency check, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)} +
\frac34(b_4'/b_4)\gamma^{(x_4)}$, the value of the Revision record with $b_4$, which
is $3H\gamma^{(x_0)}$ at $b_4 = 1$.
```

In [3]:

```
E, s, c = sp.symbols("E s c", positive=True)  # e^a4, sin(z)^(1/6), cos z
A1, A2, A3 = sp.symbols("A1 A2 A3", real=True)  # derivatives of a4 (1st to 3rd)
b4 = sp.Symbol("b4", positive=True)  # b4(x4) > 0
b4p, b4pp, b4ppp = sp.symbols("b4p b4pp b4ppp", real=True)  # its 1st to 3rd derivative
H = sp.Symbol("H", positive=True)
m, lam = sp.symbols("m lambda", real=True)
ETA = [1, 1, 1, 1, -1, -1, -1, -1]


def cd(expression, mu):
    """The derivative of a ring expression along coordinate mu (0 .. 7)."""
    expression = sp.sympify(expression)
    if mu == X4:
```

```
        return (sp.diff(expression, E) * E * A1 + sp.diff(expression, A1) * A2
                + sp.diff(expression, A2) * A3 + sp.diff(expression, b4) * b4p
                + sp.diff(expression, b4p) * b4pp + sp.diff(expression, b4pp) * b4ppp)
    if mu == X0:
        return (sp.diff(expression, s) * H * c / s**5
                + sp.diff(expression, c) * (-6 * H * s**6))
    return sp.Integer(0)


def is_zero(expression):
    """Exact test: the numerator vanishes after c^2 -> 1 - s^12."""
    numerator, _ = sp.fraction(sp.together(sp.sympify(expression)))
    numerator = sp.expand(numerator)
    if numerator == 0:
        return True
    remainder = sp.rem(sp.Poly(numerator, c), sp.Poly(c**2 - 1 + s**12, c))
    return sp.expand(remainder.as_expr()) == 0


f = [c / s**6] + [E * s] * 3 + [sp.Integer(1)] + [s * sp.sqrt(b4) / E] * 3  # frame
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal metric
sqrt_g = sp.prod(f)  # = b4^(3/2) c = b4^(3/2) cos z
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for l in range(8):
    for mu in range(8):
        for nu in range(8):
            value = ((cd(g[l], mu) if l == nu else 0) + (cd(g[l], nu) if l == mu else 0)
                     - (cd(g[mu], l) if mu == nu else 0))
            if value != 0:
                Gam[l][mu][nu] = sp.expand(value / (2 * g[l]))
SAB = [[(G[a] * G[b] - G[b] * G[a]) / 4 for b in range(8)] for a in range(8)]
Om = []  # the spinor connection Omega_mu
for mu in range(8):
    M = sp.zeros(16, 16)
    for a in range(8):
        for b in range(a + 1, 8):
            mixed = f[a] * Gam[a][mu][b] / f[b]  # omega_mu^a_b for a != b
            if mixed != 0:
                M += sp.expand(ETA[a] * mixed) * SAB[a][b]
    Om.append(M.applyfunc(sp.expand))
gam = [G[a] / f[a] for a in range(8)]  # the coordinate gammas
# real: no imaginary unit, and sympy does not find the entry non-real
real_entries = all(not v.has(sp.I) and v.is_real is not False
                   for M in Om + gam for v in M)
check(real_entries, "every entry of every Omega_mu and gamma^mu is real, for every "
      "b4: gamma^mu D_mu is a real operator",
      record=reproduces(PYREP_B4, "spinor_connection_real_b4") + "; at b4 = 1 "
      + reproduces(LEAD, "spinor_connection_real"))
total = sum((gam[mu] * Om[mu] for mu in range(8)), Z16)
at_b4_1 = {b4: 1, b4p: 0, b4pp: 0, b4ppp: 0}  # the background b4 = 1
check(all(is_zero(v) for v in (total - 3 * H * G[X0] - 3 * b4p / (4 * b4) * G[X4]))
      and all(is_zero(v) for v in (total.subs(at_b4_1) - 3 * H * G[X0])),
      "gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4), and "
      "3 H gamma^(x0) at b4 = 1: the values of the Revision records",
```

```
      record=reproduces(PYREP_B4, "gammaOmega_total") + "; at b4 = 1 "
      + reproduces(PYREP, "gamma_mu_Omega_mu_equals_3H_gamma_x0"))
```

Out [3]:

```
PASS every entry of every Omega_mu and gamma^mu is real, for every b4: gamma^mu D_mu is a
    real operator
     reproduces Revision/theory/reports/python-field-theory-b4.json, check
    spinor_connection_real_b4; at b4 = 1 Revision/lead_checks/reports/charge-conjugation-
    and-u1.json, check spinor_connection_real
PASS gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4), and 3 H gamma^(x0) at
    b4 = 1: the values of the Revision records
     reproduces Revision/theory/reports/python-field-theory-b4.json, check
    gammaOmega_total; at b4 = 1 Revision/theory/reports/python-field-theory.json, check
    gamma_mu_Omega_mu_equals_3H_gamma_x0
```

Text cell:

```
In the cell above only the off-diagonal entries $a \ne b$ of the spin connection
are needed, because $S^{aa} = 0$; for $a \ne b$ the formula of Notebook 07b,
$\omega_\mu{}^a{}_b = f_a(\delta_{ab}\partial_\mu(1/f_b) + \Gamma^a{}_{\mu b}/f_b)$,
reduces to $f_a\Gamma^a{}_{\mu b}/f_b$.
```

Text cell:

```
## 7. A jet algebra for a real field

A real field has no separate conjugate components: the generators are just the
components $\theta_A$ and their derivatives, written `(A, derivatives)`. The class
`RealJet` below is the jet algebra of Notebook 07b without conjugation, with the
switch `odd` (True: anticommuting, False: commuting). The helpers `column`,
`matvec` (matrix times column), `vecmat` (row times matrix) and `dot` (row times
column) build vectors of jets.
```

In [4]:

```
def sort_sign(keys, odd):
    """Sorted tuple of generators and the sign; (None, 0) for a zero product."""
    items, sign = list(keys), 1
    for end in range(len(items) - 1, 0, -1):  # bubble sort, counting exchanges
        for i in range(end):
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                sign = -sign
    if not odd:
        return tuple(items), 1
    if any(items[i] == items[i + 1] for i in range(len(items) - 1)):
        return None, 0
    return tuple(items), sign
```

```
class RealJet:
    """An element of the jet algebra of a real field: monomial -> coefficient."""

    def __init__(self, odd, terms=None):
        self.odd, self.terms = odd, {}
        for monomial, coefficient in (terms or {}).items():
            self.add(monomial, coefficient)

    def add(self, monomial, coefficient):
        total = sp.expand(self.terms.get(monomial, 0) + coefficient)
        if total == 0:
            self.terms.pop(monomial, None)
        else:
            self.terms[monomial] = total

    def __add__(self, other):
        result = RealJet(self.odd, self.terms)
        for monomial, coefficient in other.terms.items():
            result.add(monomial, coefficient)
        return result

    def __sub__(self, other):
        return self + other.times(-1)

    def times(self, factor):
        return RealJet(self.odd, {k: factor * v for k, v in self.terms.items()})

    def __mul__(self, other):
        result = RealJet(self.odd)
        for k1, v1 in self.terms.items():
            for k2, v2 in other.terms.items():
                monomial, sign = sort_sign(k1 + k2, self.odd)
                if monomial is not None:
                    result.add(monomial, sign * v1 * v2)
        return result

    def lderiv(self, key):
        result = RealJet(self.odd)
        for monomial, coefficient in self.terms.items():
            if key in monomial:
                p = monomial.index(key)
                factor = (-1) ** p if self.odd else monomial.count(key)
                result.add(monomial[:p] + monomial[p + 1:], factor * coefficient)
        return result

    def total(self, mu):
        result = RealJet(self.odd)
        for monomial, coefficient in self.terms.items():
            derivative = cd(coefficient, mu)
            if derivative != 0:
                result.add(monomial, derivative)
            for p, (A, d) in enumerate(monomial):
```

```
                raised = (A, tuple(sorted(d + (mu,))))
                ordered, sign = sort_sign(monomial[:p] + (raised,) + monomial[p + 1:],
                                          self.odd)
                if ordered is not None:
                    result.add(ordered, sign * coefficient)
        return result

    def is_zero(self):
        return all(is_zero(v) for v in self.terms.values())


def column(odd, derivatives=()):
    """The 16 generators theta_A (or one of their derivatives)."""
    return [RealJet(odd, {((A, tuple(sorted(derivatives))),): 1}) for A in range(16)]


def matvec(M, v, odd):
    out = [RealJet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[i] = out[i] + v[j].times(M[i, j])
    return out


def vecmat(v, M, odd):
    out = [RealJet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[j] = out[j] + v[i].times(M[i, j])
    return out


def dot(row, col, odd):
    result = RealJet(odd)
    for x, y in zip(row, col):
        result = result + x * y
    return result


say("The jet algebra of a real field is defined.")
```

Out [4]:

```
The jet algebra of a real field is defined.
```

Text cell:

```
## 8. The Majorana-type negative control

The next cell builds, for both kinds of numbers, the Majorana-type Lagrangian
```

```
$$L_g = \sqrt{|g|}\sum_\mu \Theta^T C\gamma^\mu\big(\partial_\mu\Theta +
\Omega_\mu\Theta\big)$$

and its candidate total-derivative form $\frac12\sum_\mu \frac{d}{dx_\mu}\big(
\sqrt{|g|}\,\Theta^T C\gamma^\mu\Theta\big)$, the 16 Euler-Lagrange expressions
$\mathcal{E}_A = \partial_L L_g/\partial\theta_A - \sum_\mu\frac{d}{dx_\mu}
\partial_L L_g/\partial(\partial_\mu\theta_A)$, and the mass-type term $\Theta^T
C\Theta$. Why the anticommuting case is a total derivative, in three lines
(Notebook 07a showed the two rules used):

$$\Theta^T C\gamma^\mu\partial_\mu\Theta = \tfrac12\partial_\mu\big(\Theta^T
C\gamma^\mu\Theta\big) - \tfrac12\Theta^T\partial_\mu\big(C\gamma^\mu\big)\Theta,
\qquad \Theta^T C\gamma^\mu\Omega_\mu\Theta = \tfrac12\Theta^T C[\gamma^\mu,
\Omega_\mu]\Theta,$$
$$\sqrt{|g|}\,\Theta^T C\gamma^\mu D_\mu\Theta = \tfrac12\partial_\mu\big(\sqrt{|g|}
\,\Theta^T C\gamma^\mu\Theta\big) - \tfrac12\Theta^T C\big(\partial_\mu(\sqrt{|g|}
\gamma^\mu) - \sqrt{|g|}[\gamma^\mu, \Omega_\mu]\big)\Theta .$$

The first identity: the product rule, and exchanging the two odd factors in
$(\partial_\mu\Theta)^T C\gamma^\mu\Theta$ with the antisymmetric $C\gamma^\mu$
makes it equal to $\Theta^T C\gamma^\mu\partial_\mu\Theta$. The second: only the
antisymmetric part of $C\gamma^\mu\Omega_\mu$ survives in a real Grassmann form, and
it is $\frac12 C[\gamma^\mu, \Omega_\mu]$. The third: multiply by $\sqrt{|g|}$ and
collect. The last bracket is zero by the divergence identity
$\partial_\mu(\sqrt{|g|}\gamma^\mu) = \sqrt{|g|}[\gamma^\mu, \Omega_\mu]$ (the
covariant constancy of the gammas). So $L_g$ is the total derivative of
$\frac12\sqrt{|g|}\,\Theta^T C\gamma^\mu\Theta$.
```

In [5]:

```
results = {}
for name, odd in (("grassmann", True), ("commuting", False)):
    theta = column(odd)
    row = vecmat(theta, C, odd)  # Theta^T C
    Lg = RealJet(odd)
    derivative_form = RealJet(odd)
    for mu in range(8):
        D_theta = [x + y for x, y in zip(column(odd, (mu,)),
                                         matvec(Om[mu], theta, odd))]
        Lg = Lg + dot(row, matvec(gam[mu], D_theta, odd), odd)
        current = dot(row, matvec(gam[mu], theta, odd), odd).times(sqrt_g / 2)
        derivative_form = derivative_form + current.total(mu)
    Lg = Lg.times(sqrt_g)
    euler_lagrange = []
    for A in range(16):
        value = Lg.lderiv((A, ()))
        for mu in range(8):
            value = value - Lg.lderiv((A, (mu,))).total(mu)
        euler_lagrange.append(value)
    results[name] = {"Lg": Lg, "derivative_form": derivative_form,
                     "EL": euler_lagrange, "mass": dot(row, theta, odd),
                     "theta": theta}
```

```
    nonzero = sum(1 for e in euler_lagrange if not e.is_zero())
    report(f"{name}: monomials of Lg", len(Lg.terms))
    report(f"{name}: nonzero Euler-Lagrange expressions", f"{nonzero} of 16")
```

Out [5]:

```
RESULT grassmann: monomials of Lg = 144
RESULT grassmann: nonzero Euler-Lagrange expressions = 0 of 16
RESULT commuting: monomials of Lg = 128
RESULT commuting: nonzero Euler-Lagrange expressions = 16 of 16
```

Text cell:

```
The next cell checks the anticommuting case against both Revision verifiers: $L_g$
is not zero, it equals its total-derivative form exactly, all 16 Euler-Lagrange
expressions vanish, and $\Theta^T C\Theta = 0$.
```

In [6]:

```
grassmann = results["grassmann"]
check(len(grassmann["Lg"].terms) > 0
      and (grassmann["Lg"] - grassmann["derivative_form"]).is_zero(),
      "real Grassmann field: Lg = (1/2) d_mu (sqrt|g| Theta^T C gamma^mu Theta), not 0",
      record=reproduces(PYREP, "negative_control_majorana_grassmann_total_derivative"))
check(all(e.is_zero() for e in grassmann["EL"]) and grassmann["mass"].is_zero(),
      "real Grassmann field: all 16 Euler-Lagrange expressions vanish and Theta^T C "
      "Theta = 0", record=reproduces(WLREP, "Majorana_Lg_total_derivative_grassmann"))
```

Out [6]:

```
PASS real Grassmann field: Lg = (1/2) d_mu (sqrt|g| Theta^T C gamma^mu Theta), not 0
     reproduces Revision/theory/reports/python-field-theory.json, check
    negative_control_majorana_grassmann_total_derivative
PASS real Grassmann field: all 16 Euler-Lagrange expressions vanish and Theta^T C Theta =
    0
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Majorana_Lg_total_derivative_grassmann
```

Text cell:

```
For real COMMUTING components the same $L_g$ is a genuine Lagrangian: the
Euler-Lagrange expression is $2\sqrt{|g|}\,(C\gamma^\mu D_\mu\Phi)_A$ (a massless
field equation), nonzero in all 16 components, while $\Phi^T C\gamma^\mu\Phi = 0$
for every $\mu$. The next cell checks this, reads the count "16 of 16" from the
detail of the sympy record's check, and draws Figure 1.
```

In [7]:

```
commuting = results["commuting"]
phi = commuting["theta"]
operator = [RealJet(False) for _ in range(16)]  # gamma^mu D_mu Phi
for mu in range(8):
    D_phi = [x + y for x, y in zip(column(False, (mu,)), matvec(Om[mu], phi, False))]
    operator = [x + y for x, y in zip(operator, matvec(gam[mu], D_phi, False))]
expected = matvec(C * 2 * sqrt_g, operator, False)  # 2 sqrt|g| C gamma^mu D_mu Phi
nonzero = sum(1 for e in commuting["EL"] if not e.is_zero())
detail = revision_check(PYREP, "negative_control_majorana_commuting_contrast")["detail"]
recorded = int(re.search(r"in (\d+) of 16 components", detail).group(1))
check(nonzero == recorded == 16, "real commuting field: Lg gives nonzero "
      "Euler-Lagrange expressions in 16 of 16 components",
      record=reproduces(PYREP, "negative_control_majorana_commuting_contrast"))
vector_zero = all(dot(vecmat(phi, C, False), matvec(G[a], phi, False), False)
                  .is_zero() for a in range(8))
check(all((e - x).is_zero() for e, x in zip(commuting["EL"], expected))
      and vector_zero, "real commuting field: the expression is 2 sqrt|g| (C "
      "gamma^mu D_mu Phi)_A and Phi^T C gamma^(a) Phi = 0",
      record=reproduces(WLREP, "Majorana_Lg_commuting_control"))

fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
labels = ["real Grassmann", "real commuting"]
left.bar(labels, [len(grassmann["Lg"].terms), len(commuting["Lg"].terms)],
         color=["tab:blue", "tab:orange"])
left.set_ylabel("monomials of $L_g$")
left.set_title("$L_g$ is not zero in either case")
counts = [sum(1 for e in grassmann["EL"] if not e.is_zero()), nonzero]
right.bar(labels, counts, color=["tab:blue", "tab:orange"])
right.set_ylim(0, 17)
right.set_ylabel("nonzero Euler-Lagrange expressions")
right.set_title("Field equations from $L_g$")
for x, value in enumerate(counts):
    right.text(x, value + 0.4, f"{value} of 16", ha="center")
fig.tight_layout()
save_figure(fig, "majorana_control",
            "The Majorana-type Lagrangian $L_g = \\sqrt{|g|}\\,\\Theta^T C\\gamma^\\mu "
            "D_\\mu\\Theta$ of the author's notebook in the author's metric, for a "
            "real column of 16 anticommuting (blue) or commuting (orange) "
            "components. Left: the number of monomials of $L_g$ in the jet algebra; "
            "it is not zero in either case. Right: the number of its 16 "
            "Euler-Lagrange expressions that are not zero. For anticommuting "
            "components $L_g$ is a total derivative and gives no field equation at "
            "all (0 of 16); for commuting components it gives a field equation in "
            "every component (16 of 16).")
```

Out [7]:

```
PASS real commuting field: Lg gives nonzero Euler-Lagrange expressions in 16 of 16
    components
     reproduces Revision/theory/reports/python-field-theory.json, check
    negative_control_majorana_commuting_contrast
PASS real commuting field: the expression is 2 sqrt|g| (C gamma^mu D_mu Phi)_A and Phi^T
    C gamma^(a) Phi = 0
```

```
 reproduces Revision/theory/reports/wolfram-field-theory.json, check
Majorana_Lg_commuting_control
```

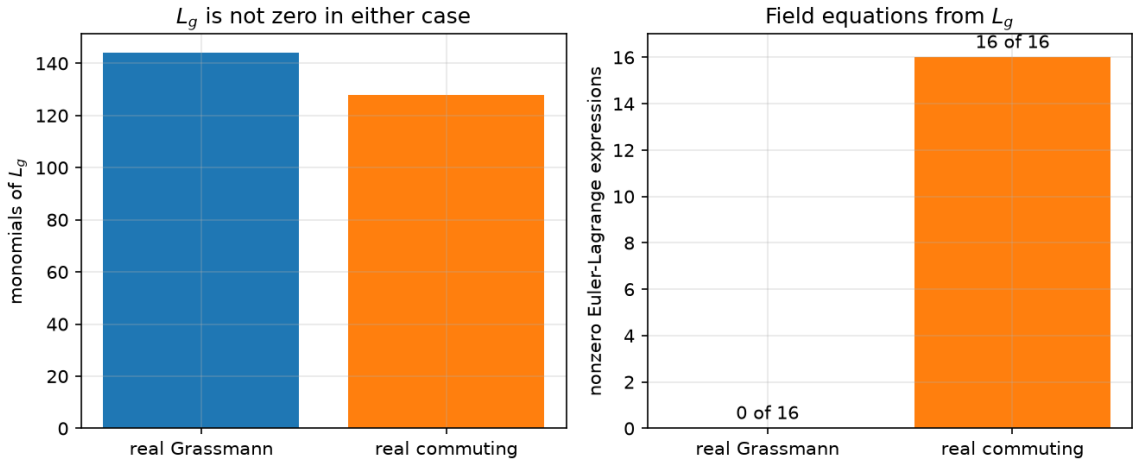

Figure 225: The Majorana-type Lagrangian $L_g = \sqrt{|g|}\,\Theta^T C\gamma^\mu D_\mu\Theta$ of the author's notebook in the author's metric, for a real column of 16 anticommuting (blue) or commuting (orange) components. Left: the number of monomials of L_g in the jet algebra; it is not zero in either case. Right: the number of its 16 Euler-Lagrange expressions that are not zero. For anticommuting components L_g is a total derivative and gives no field equation at all (0 of 16); for commuting components it gives a field equation in every component (16 of 16). (Notebook 07c, figure 1.)

Out [7]:

```
Figure 07c.1 saved as Revision/textbook/figures/07c_1_majorana_control.png
```

Text cell:

```
## 9. Which matrices turn solutions into solutions?

We look for all $16\times16$ matrices $M$ with $M\gamma^{(a)} = s\,\gamma^{(a)}M$
for all eight $a$ (for real gammas $(\gamma^{(a)})^* = \gamma^{(a)}$, so this is the
condition of section 4). The 256 products $\gamma_I = \gamma^{(a_1)}\cdots
\gamma^{(a_k)}$ with $a_1 < \dots < a_k$ (all subsets $I$ of the eight gammas,
including the empty product 1) are a basis of all $16\times16$ matrices, because
they are mutually orthogonal: $\mathrm{tr}(\gamma_I^T\gamma_J) = 16\,\delta_{IJ}$
(the next cell checks this exactly with integers), and 256 orthogonal nonzero
matrices in the 256-dimensional space of $16\times16$ matrices form a basis.

Each basis product either commutes or anticommutes with a given $\gamma^{(a)}$:
moving $\gamma^{(a)}$ through the $k$ factors costs $(-1)^k$ if $a \notin I$ and
$(-1)^{k-1}$ if $a \in I$ (it passes $k - 1$ other factors and commutes with
itself). Write $M = \sum_I c_I\gamma_I$; since $\gamma^{(a)}\gamma_I(\gamma^{(a)})^{-1}
= \pm\gamma_I$ and the $\gamma_I$ are independent, the condition holds exactly when
every $\gamma_I$ with $c_I \ne 0$ has the sign $s$ for every $a$. So the solution
space is spanned by the products that commute with ALL eight gammas ($s = +1$) or
anticommute with ALL eight ($s = -1$). The next cell counts, for each product, the
number of gammas it commutes with, and finds which products qualify.
```

In [8]:

```
Gn = [np.array(gm.tolist(), dtype=np.int64) for gm in G]  # exact integer arrays
products, commute_counts, degrees = [], [], []
for k in range(9):
    for subset in combinations(range(8), k):
        P = np.eye(16, dtype=np.int64)
        for a in subset:
            P = P @ Gn[a]  # gamma_(a1) ... gamma_(ak)
        products.append(P)
        degrees.append(k)
        commute_counts.append(sum(1 for a in range(8)
                                  if np.array_equal(P @ Gn[a], Gn[a] @ P)))
flat = np.array([P.reshape(256) for P in products])  # one row per product
gram = flat @ flat.T  # all traces tr(gamma_I^T gamma_J), exact integers
check(len(products) == 256 and np.array_equal(gram, 16 * np.eye(256, dtype=np.int64)),
      "the 256 products of gammas are orthogonal: a basis of all 16 x 16 matrices")
same = [i for i in range(256) if commute_counts[i] == 8]  # commute with all
reversed_ = [i for i in range(256) if commute_counts[i] == 0]  # anticommute with all
report("products that commute with all 8 gammas (degree)", [degrees[i] for i in same])
report("products that anticommute with all 8 gammas (degree)",
       [degrees[i] for i in reversed_])
GAMMAn = np.array(GAMMA.tolist(), dtype=np.int64)
check(len(same) == 1 and np.array_equal(products[same[0]], np.eye(16, dtype=np.int64)),
      "s = +1: the solutions M form a 1-dimensional space, spanned by the identity",
      record=reproduces(LEAD, "intertwiners_same_mass"))
check(len(reversed_) == 1 and (np.array_equal(products[reversed_[0]], GAMMAn)
                               or np.array_equal(products[reversed_[0]], -GAMMAn)),
      "s = -1: the solutions M form a 1-dimensional space, spanned by Gamma",
      record=reproduces(LEAD, "intertwiners_reversed_mass"))
```

Out [8]:

```
PASS the 256 products of gammas are orthogonal: a basis of all 16 x 16 matrices
RESULT products that commute with all 8 gammas (degree) = [0]
RESULT products that anticommute with all 8 gammas (degree) = [8]
PASS s = +1: the solutions M form a 1-dimensional space, spanned by the identity
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    intertwiners_same_mass
PASS s = -1: the solutions M form a 1-dimensional space, spanned by Gamma
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    intertwiners_reversed_mass
```

Text cell:

```
The product of all eight gammas in the order $x_0, \dots, x_7$ is $\Gamma$ (in
another order of the factors it may be $-\Gamma$), so the second space is spanned by
$\Gamma$. Figure 2 shows the whole pattern: for a product of $k$ gammas, the number
of gammas it commutes with is $k$ when $k$ is odd and $8 - k$ when $k$ is even. Only
the empty product ($k = 0$) commutes with all eight, and only the full product ($k
= 8$) with none.
```

In [9]:

```
table = np.zeros((9, 9), dtype=int)  # rows: degree k, columns: number commuted with
for k, j in zip(degrees, commute_counts):
    table[k, j] += 1
binomials = [1, 8, 28, 56, 70, 56, 28, 8, 1]  # the number of products of k gammas
check(all(table[k, k if k % 2 else 8 - k] == binomials[k] for k in range(9)),
      "a product of k gammas commutes with k of them (k odd) or 8 - k (k even)")

fig, ax = plt.subplots(figsize=(6.6, 5.4))
shown = np.where(table > 0, np.log10(np.maximum(table, 1)) + 0.3, 0.0)
image = ax.imshow(shown, cmap="Purples", vmin=0, vmax=2.3)
for k in range(9):
    for j in range(9):
        if table[k, j]:
            ax.text(j, k, str(table[k, j]), ha="center", va="center",
                    color="white" if table[k, j] > 20 else "black")
ax.add_patch(plt.Rectangle((7.5, -0.5), 1, 1, fill=False, edgecolor="red",
                           linewidth=2))
ax.add_patch(plt.Rectangle((-0.5, 7.5), 1, 1, fill=False, edgecolor="red",
                           linewidth=2))
ax.set_xticks(range(9))
ax.set_yticks(range(9))
ax.set_xlabel("number of the 8 gammas that the product commutes with")
ax.set_ylabel("number $k$ of gamma factors in the product")
ax.set_title("The 256 products of gammas")
ax.grid(False)
save_figure(fig, "commutation_pattern",
            "The 256 products $\\gamma_I$ of the author's gamma matrices (a basis "
            "of all $16 \\times 16$ matrices), counted by the number $k$ of factors "
            "(rows) and by the number of the eight gammas with which they commute "
            "(columns); the numbers are the binomial coefficients. Only two squares "
            "(red frames) lie in the outer columns: the identity ($k = 0$) commutes "
            "with all eight, and the chirality matrix $\\Gamma$ ($k = 8$) "
            "anticommutes with all eight. Hence every matrix $M$ with $M\\gamma^{(a)} "
            "= \\pm\\gamma^{(a)}M$ for all $a$ is a multiple of 1 or of $\\Gamma$.")
```

Out [9]:

```
PASS a product of k gammas commutes with k of them (k odd) or 8 - k (k even)
```

Out [9]:

```
Figure 07c.2 saved as Revision/textbook/figures/07c_2_commutation_pattern.png
```

Text cell:

```
## 10. The two charge-conjugation matrices

With $M = \mathcal{C}C$ and $C^{-1} = C$: $\mathcal{C}_+ = C$ (from $M = 1$) and
```

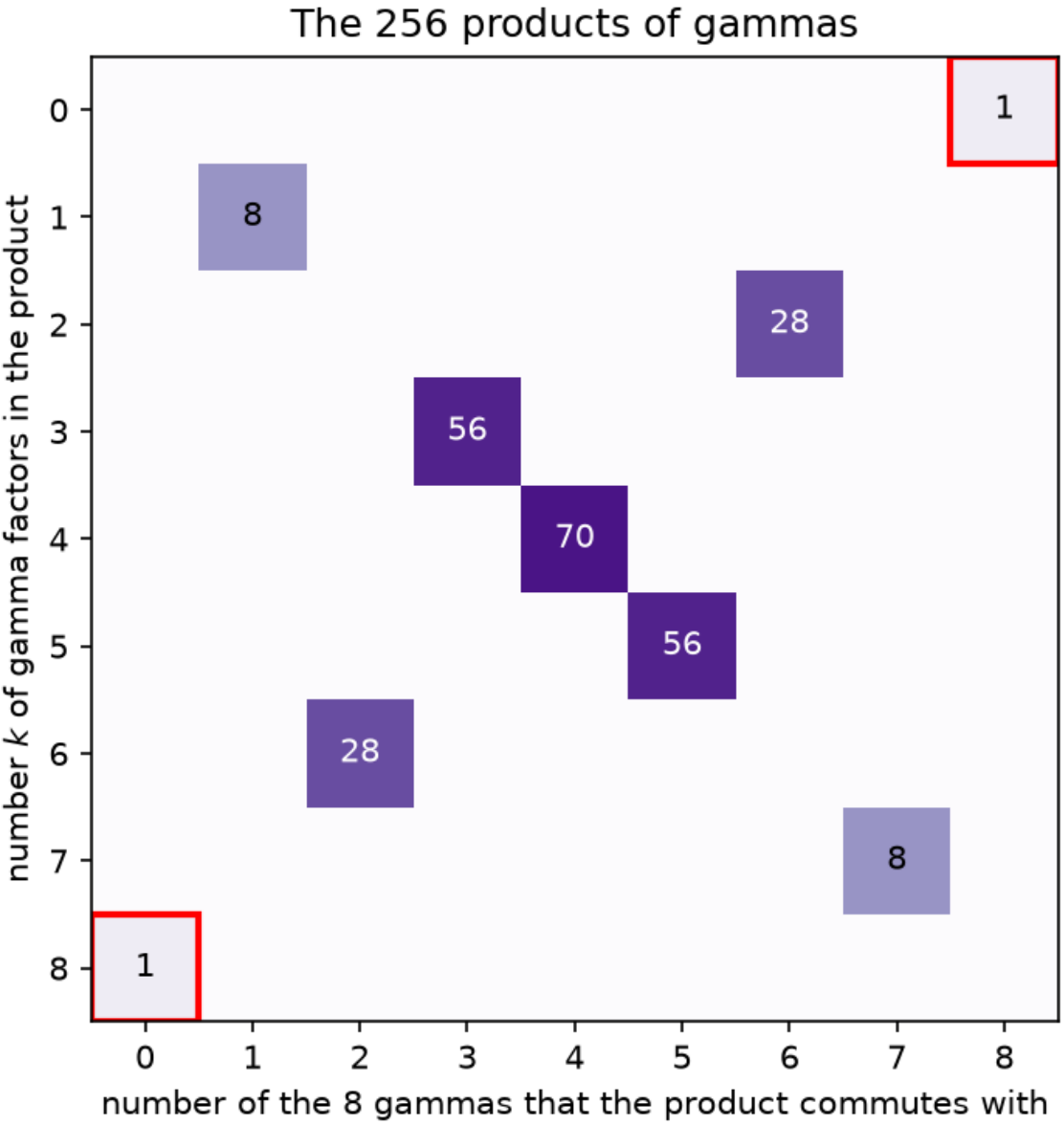

Figure 226: The 256 products γ_I of the author's gamma matrices (a basis of all 16×16 matrices), counted by the number k of factors (rows) and by the number of the eight gammas with which they commute (columns); the numbers are the binomial coefficients. Only two squares (red frames) lie in the outer columns: the identity ($k = 0$) commutes with all eight, and the chirality matrix Γ ($k = 8$) anticommutes with all eight. Hence every matrix M with $M\gamma^{(a)} = \pm\gamma^{(a)}M$ for all a is a multiple of 1 or of Γ. (Notebook 07c, figure 2.)

```
$\mathcal{C}_- = \Gamma C$ (from $M = \Gamma$). The defining properties of a
charge-conjugation matrix, $\mathcal{C}^{-1}\gamma^{(a)}\mathcal{C} =
\mp(\gamma^{(a)})^T$, follow line by line for $\mathcal{C}_+ = C$:

$$C^{-1}\gamma^{(a)}C = (C\gamma^{(a)})\,C = -(C\gamma^{(a)})^T C
= -(\gamma^{(a)})^T C^T C = -(\gamma^{(a)})^T .$$

The first step uses $C^{-1} = C$ (because $C^2 = 1$); the second that
$C\gamma^{(a)}$ is antisymmetric; the third that the transpose of a product is the
product of the transposes in reversed order; the last that $C^TC = C^2 = 1$. For
$\mathcal{C}_- = \Gamma C$: $\mathcal{C}_-^{-1} = C\Gamma$, and $\Gamma\gamma^{(a)}
\Gamma = -\gamma^{(a)}$ (anticommute, then $\Gamma^2 = 1$), so
$\mathcal{C}_-^{-1}\gamma^{(a)}\mathcal{C}_- = -C\gamma^{(a)}C =
+(\gamma^{(a)})^T$. Both matrices are real, and both reality conditions $\Psi =
M\Psi^*$ are consistent because $MM^* = 1$ for $M = 1$ and for $M = \Gamma$
($\Gamma$ is real and $\Gamma^2 = 1$). The next cell checks all of this exactly.
```

In [10]:

```
C_plus, C_minus = C, GAMMA * C
check(all(C_plus.inv() * g * C_plus == -g.T for g in G) and C_plus.T == C_plus,
      "calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real symmetric",
      record=reproduces(LEAD, "charge_conjugation_matrix_plus"))
check(all(C_minus.inv() * g * C_minus == g.T for g in G)
      and all(v.is_real for v in C_minus),
      "calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real",
      record=reproduces(LEAD, "charge_conjugation_matrix_minus"))
check(I16 * I16.conjugate() == I16 and GAMMA * GAMMA.conjugate() == I16,
      "M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent",
      record=reproduces(LEAD, "majorana_conditions_consistent"))
```

Out [10]:

```
PASS calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real symmetric
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    charge_conjugation_matrix_plus
PASS calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    charge_conjugation_matrix_minus
PASS M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    majorana_conditions_consistent
```

Text cell:

```
## 11. Real fields: what charge conjugation does to them

For a real column $\Phi$ ($\Phi^* = \Phi$, so $\Phi^\dagger = \Phi^T$):

$$\Phi^c_+ = \mathcal{C}_+\bar\Phi^T = C\,(\Phi^T C)^T = C\,C^T\Phi = \Phi,
\qquad \Phi^c_- = \mathcal{C}_-\bar\Phi^T = \Gamma C\,C\Phi = \Gamma\Phi .$$
```

```
So $\mathcal{C}_+$ does nothing to a real field: a real field is its own
conjugate. This matches the current: $J^a = -i\Phi^T C\gamma^{(a)}\Phi = 0$
identically, because $C\gamma^{(a)}$ is antisymmetric and the components commute; a
real commuting field carries no charge. The only nontrivial real map is the matrix
$\Gamma$. It keeps the scalar density and reverses every kinetic matrix:

$$\Gamma^T C\Gamma = C,\qquad \Gamma^T C\gamma^{(a)}\Gamma = -C\gamma^{(a)} .$$

(Both from $\Gamma^T = \Gamma$, which commutes with $C$, and anticommutes with
$\gamma^{(a)}$.) Therefore, for real fields, $L_{m,\lambda}[\Gamma\Phi] =
-L_{-m,-\lambda}[\Phi]$: the matrix $\Gamma$ together with $(m, \lambda) \to (-m,
-\lambda)$ maps solutions to solutions. The next cell checks the two conjugates,
the vanishing current and the two matrix identities, and draws Figure 3.
```

In [11]:

```
phi_bar_T = (r.T * C).T  # (Phi^T C)^T for the real column r
check((C_plus * phi_bar_T - r).applyfunc(sp.expand) == sp.zeros(16, 1)
      and (C_minus * phi_bar_T - GAMMA * r).applyfunc(sp.expand) == sp.zeros(16, 1),
      "for a real field Phi: calC_+ gives Phi itself, calC_- gives Gamma Phi")
currents = [sp.expand((-sp.I * r.T * C * g * r)[0, 0]) for g in G]
kinetic_reversed = all(GAMMA.T * C * g * GAMMA == -(C * g) for g in G)
check(all(j == 0 for j in currents) and GAMMA.T * C * GAMMA == C and kinetic_reversed,
      "real commuting field: J^a = 0 for every a; Gamma^T C Gamma = C and Gamma^T C "
      "gamma^a Gamma = -C gamma^a",
      record=reproduces(LEAD, "real_fields_charge_conjugation"))

numbers = [np.array(M.tolist(), dtype=float) for M in
           (C, GAMMA.T * C * GAMMA, C * G[X4], GAMMA.T * C * G[X4] * GAMMA)]
titles = ["$C$", "$\\Gamma^T C\\,\\Gamma$ (equal)", "$C\\gamma^{(x_4)}$",
          "$\\Gamma^T C\\gamma^{(x_4)}\\Gamma$ (opposite)"]
fig, axes = plt.subplots(2, 2, figsize=(7.6, 7.4))
for ax, M, title in zip(axes.flat, numbers, titles):
    image = ax.imshow(M, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.6)
save_figure(fig, "gamma_map_matrices",
            "What the real matrix $\\Gamma$ does to the two kinds of terms of the "
            "Lagrangian of a real field ($16 \\times 16$ matrices, red $+1$, blue "
            "$-1$, grey 0; rows and columns 1 to 16). Top: the matrix $C$ of the "
            "scalar density $S = \\Phi^T C\\Phi$ and $\\Gamma^T C\\Gamma$, which is "
            "the same: $S$ is kept. Bottom: the kinetic matrix $C\\gamma^{(x_4)}$ "
            "and $\\Gamma^T C\\gamma^{(x_4)}\\Gamma$, which has every colour "
            "swapped: the kinetic term changes sign (the same holds for all eight "
            "directions). So the Lagrangian of $\\Gamma\\Phi$ with $(m, \\lambda)$ "
            "is minus the Lagrangian of $\\Phi$ with $(-m, -\\lambda)$.")
```

Out [11]:

```
PASS for a real field Phi: calC_+ gives Phi itself, calC_- gives Gamma Phi
PASS real commuting field: J^a = 0 for every a; Gamma^T C Gamma = C and Gamma^T C gamma^a
    Gamma = -C gamma^a
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    real_fields_charge_conjugation
```

Out [11]:

```
Figure 07c.3 saved as Revision/textbook/figures/07c_3_gamma_map_matrices.png
```

Text cell:

```
## 12. The mass reversal in the author's metric

The next cell checks the statement $L_{m,\lambda}[\Gamma\Phi] =
-L_{-m,-\lambda}[\Phi]$ for a real commuting field in the author's metric, exactly,
with the jet algebra. The Lagrangian of a real field is the Revision Lagrangian with
$\Psi^\dagger = \Phi^T$: $\bar\Phi = \Phi^T C$, $D_\mu\bar\Phi = \partial_\mu
\Phi^T C - \Phi^T C\Omega_\mu$. The field $\Gamma\Phi$ has the components
$\sum_B\Gamma_{AB}\theta_B$ and the derivatives $\sum_B\Gamma_{AB}\partial_\mu
\theta_B$ ($\Gamma$ is constant). The cell also checks that the Euler-Lagrange
expressions of the real Lagrangian are $2\sqrt{|g|}\,(C(\gamma^\mu D_\mu\Phi - (m +
\lambda S)\Phi))_A$: a real commuting field obeys the same field equation as the
complex one.
```

In [12]:

```
def real_lagrangian(columns, mass, coupling):
    """sqrt|g| [K - mass S - (coupling/2) S^2] of the real field whose value and
    derivatives are columns(derivatives)."""
    field = columns(())
    bar = vecmat(field, C, False)  # Phi^T C
    K = RealJet(False)
    for mu in range(8):
        D_field = [x + y for x, y in zip(columns((mu,)), matvec(Om[mu], field, False))]
        D_bar = [x - y for x, y in zip(vecmat(columns((mu,)), C, False),
                                       vecmat(bar, Om[mu], False))]
        K = K + dot(bar, matvec(gam[mu], D_field, False), False)
        K = K - dot(vecmat(D_bar, gam[mu], False), field, False)
    S = dot(bar, field, False)
    return (K.times(sp.Rational(1, 2)) - S.times(mass)
            - (S * S).times(coupling / 2)).times(sqrt_g)


def plain(derivatives):  # the components of Phi
    return column(False, derivatives)


def mapped(derivatives):  # the components of Gamma Phi
    return matvec(GAMMA, column(False, derivatives), False)
```

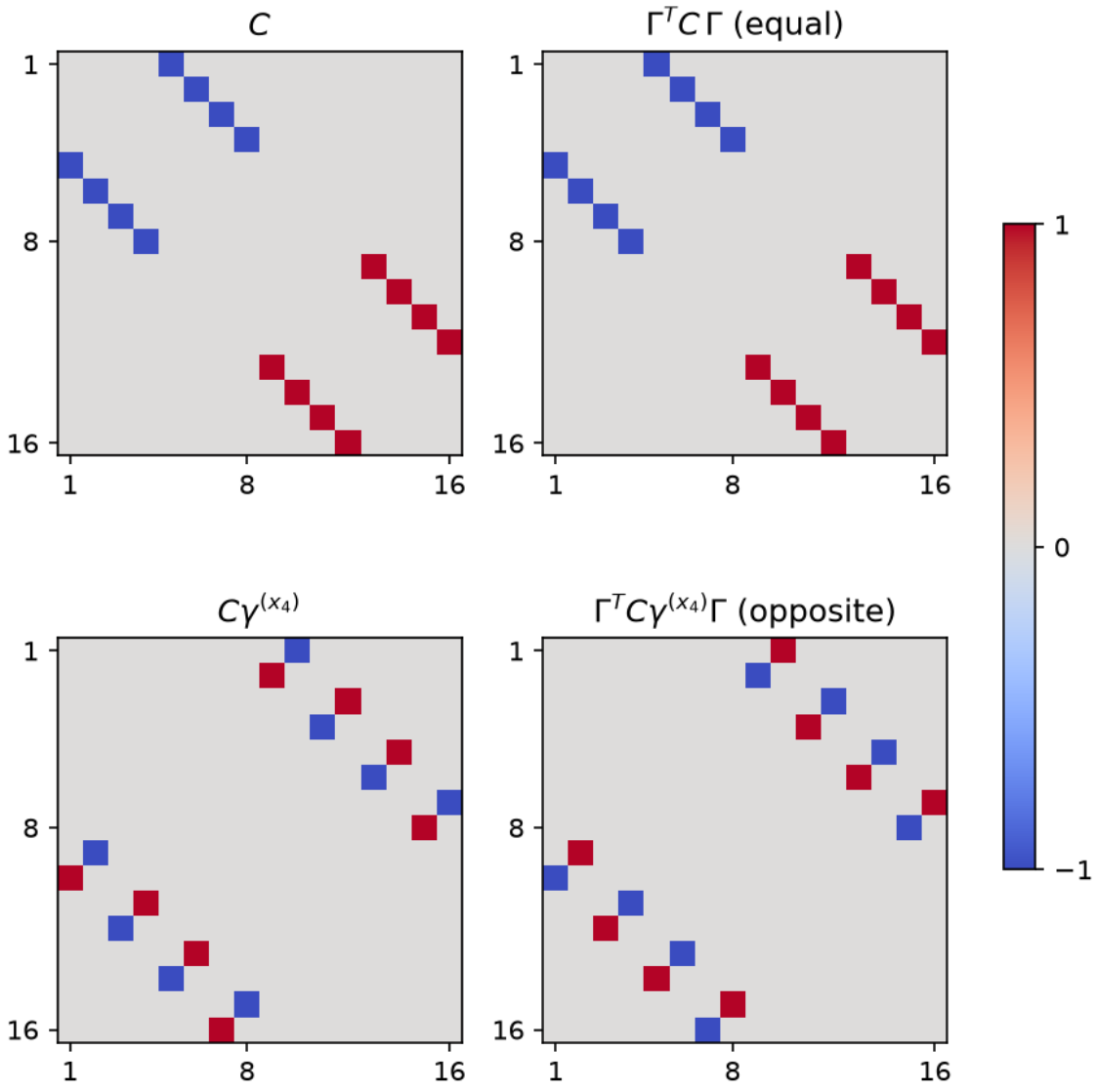

Figure 227: What the real matrix Γ does to the two kinds of terms of the Lagrangian of a real field (16×16 matrices, red $+1$, blue -1, grey 0; rows and columns 1 to 16). Top: the matrix C of the scalar density $S = \Phi^T C \Phi$ and $\Gamma^T C \Gamma$, which is the same: S is kept. Bottom: the kinetic matrix $C\gamma^{(x_4)}$ and $\Gamma^T C \gamma^{(x_4)} \Gamma$, which has every colour swapped: the kinetic term changes sign (the same holds for all eight directions). So the Lagrangian of $\Gamma\Phi$ with (m, λ) is minus the Lagrangian of Φ with $(-m, -\lambda)$. (Notebook 07c, figure 3.)

```
L_mapped = real_lagrangian(mapped, m, lam)
L_reversed = real_lagrangian(plain, -m, -lam)
check((L_mapped + L_reversed).is_zero(), "real field in the author's metric: "
      "L_(m, lambda)[Gamma Phi] = -L_(-m, -lambda)[Phi]")
L_real = real_lagrangian(plain, m, lam)
S_real = dot(vecmat(phi, C, False), phi, False)
residual = [x - (RealJet(False, {(): m}) + S_real.times(lam)) * y
            for x, y in zip(operator, phi)]  # gamma D Phi - (m + lambda S) Phi
target = matvec(C * 2 * sqrt_g, residual, False)
ok = True
for A in range(16):
    value = L_real.lderiv((A, ()))
    for mu in range(8):
        value = value - L_real.lderiv((A, (mu,))).total(mu)
    ok = ok and (value - target[A]).is_zero()
check(ok, "real commuting field: Euler-Lagrange expressions 2 sqrt|g| (C (gamma^mu "
      "D_mu Phi - (m + lambda S) Phi))_A")
```

Out [12]:

```
PASS real field in the author's metric: L_(m, lambda)[Gamma Phi] = -L_(-m, -lambda)[Phi]
PASS real commuting field: Euler-Lagrange expressions 2 sqrt|g| (C (gamma^mu D_mu Phi -
    (m + lambda S) Phi))_A
```

Text cell:

```
**A real solution and its image.** Notebook 07b verified the exact family $\Psi =
\sin^\alpha z\,(\cosh kx_4 + \sinh(kx_4)/k\,M)\chi_0$ with $M = -m\gamma^{(x_4)} +
3H(2\alpha + 1)\gamma^{(x_4)}\gamma^{(x_0)}$ on the background $b_4 = 1$ (for a
changing $b_4$ the extra term $\frac34(b_4'/b_4)\gamma^{(x_4)}$ of the field equation
changes the family; this cell stays at $b_4 = 1$). For a real $\chi_0$ and real $k$ it is
a real solution. Because $\Gamma$ anticommutes with $\gamma^{(x_4)}$ and commutes
with $\gamma^{(x_4)}\gamma^{(x_0)}$, $\Gamma M_m\Gamma = M_{-m}$: the image $\Gamma
\Psi$ is the member of the family with mass $-m$ and the column $\Gamma\chi_0$. The
next cell checks $\Gamma M_m\Gamma = M_{-m}$ exactly, then evaluates, for the
illustration values $H = 1$, $m = 2$, $\alpha = 0$ ($k = \sqrt5$), $\chi_0 = e_1 +
2e_{13}$, $z = \pi/4$ and $x_4$ from 0 to 2, the residuals
$\gamma^{(x_4)}\partial_4\Psi + 3H\gamma^{(x_0)}\Psi - \mu\Psi$ of $\Phi$ and of
$\Gamma\Phi$ in the equations with $\mu = +m$ and $\mu = -m$ (Figure 4).
```

In [13]:

```
mass_symbol = sp.Symbol("mu", real=True)


def M_of(mass):
    return -mass * G[X4] + 3 * H * G[X4] * G[X0]  # alpha = 0, b4 = 1


check((GAMMA * M_of(mass_symbol) * GAMMA - M_of(-mass_symbol)).applyfunc(sp.expand)
```

```
      == Z16, "Gamma M_m Gamma = M_(-m): Gamma maps the family of mass m to mass -m")

Gf = [np.array(gm.tolist(), dtype=float) for gm in G]
GAMMAf = np.array(GAMMA.tolist(), dtype=float)
chi0 = np.zeros(16)
chi0[0], chi0[12] = 1.0, 2.0  # e_1 + 2 e_13: one component in each chiral half
times = np.linspace(0.0, 2.0, 201)
Mf = -2.0 * Gf[X4] + 3.0 * Gf[X4] @ Gf[X0]  # m = 2, H = 1, b4 = 1
kk = np.sqrt(5.0)
Phi = np.array([(np.cosh(kk * t) * np.eye(16) + np.sinh(kk * t) / kk * Mf) @ chi0
                for t in times])
dPhi = np.array([(kk * np.sinh(kk * t) * np.eye(16) + np.cosh(kk * t) * Mf) @ chi0
                 for t in times])


def residual_size(values, derivatives, mass):
    """The largest component of gamma4 d4 Psi + 3 gamma0 Psi - mass Psi."""
    return np.array([np.abs(Gf[X4] @ d + 3.0 * Gf[X0] @ v - mass * v).max()
                     for v, d in zip(values, derivatives)])


image_values, image_derivatives = Phi @ GAMMAf.T, dPhi @ GAMMAf.T  # Gamma Phi
size = np.abs(Phi).max(axis=1)
phi_plus = residual_size(Phi, dPhi, 2.0) / size
image_minus = residual_size(image_values, image_derivatives, -2.0) / size
image_plus = residual_size(image_values, image_derivatives, 2.0)
check(phi_plus.max() < 1e-12 and image_minus.max() < 1e-12
      and np.allclose(image_plus, 4.0 * size),
      "numerically: Phi solves the mass +2 equation, Gamma Phi the mass -2 equation "
      "(relative residuals below 1e-12); Gamma Phi fails the +2 equation by 2 m |Phi|")
```

Out [13]:

```
PASS Gamma M_m Gamma = M_(-m): Gamma maps the family of mass m to mass -m
PASS numerically: Phi solves the mass +2 equation, Gamma Phi the mass -2 equation
    (relative residuals below 1e-12); Gamma Phi fails the +2 equation by 2 m |Phi|
```

Text cell:

```
Figure 4: left, two components of $\Phi$ and of $\Gamma\Phi$ ($\Gamma =
\mathrm{diag}(-1, \dots, -1, 1, \dots, 1)$ flips the sign of components 1 to 8 and
keeps 9 to 16); right, the residuals of the image in the wrong and in the right
equation.
```

In [14]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(times, Phi[:, 0], label="$\\Phi_1$")
left.plot(times, image_values[:, 0], "--", label="$(\\Gamma\\Phi)_1 = -\\Phi_1$")
left.plot(times, Phi[:, 12], label="$\\Phi_{13}$")
left.plot(times, image_values[:, 12], ":", label="$(\\Gamma\\Phi)_{13} = \\Phi_{13}$")
```

```
left.set_xlabel("time $x_4$")
left.set_ylabel("component at $z = \\pi/4$")
left.set_title("A real solution and its image")
left.legend(fontsize=8)
right.semilogy(times, image_plus, label="$\\Gamma\\Phi$ in the mass $+m$ equation")
right.semilogy(times, size, ":", color="black", label="size of $\\Phi$")
right.set_xlabel("time $x_4$")
right.set_ylabel("largest residual component")
right.set_title("$\\Gamma\\Phi$ solves the mass $-m$ equation")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "mass_reversal",
            "The real matrix $\\Gamma$ reverses the mass. Left: components 1 and 13 "
            "of the real exact solution $\\Phi$ (illustration values $H = 1$, "
            "$m = 2$, $\\alpha = 0$, $\\chi_0 = e_1 + 2e_{13}$, at $z = \\pi/4$) and "
            "of its image $\\Gamma\\Phi$ against the time $x_4$: $\\Gamma$ reverses "
            "the sign of the components 1 to 8 and keeps 9 to 16. Right "
            "(logarithmic axis): the largest component of the residual of "
            "$\\Gamma\\Phi$ in the equation with mass $+m$ (solid), which is $2m$ "
            "times the size of $\\Phi$ (dotted); in the equation with mass $-m$ the "
            "residual is zero up to rounding (below $10^{-12}$ of the size, not "
            "drawn). A real field has no charge; $\\Gamma$ relates solutions of "
            "masses $m$ and $-m$.")
```

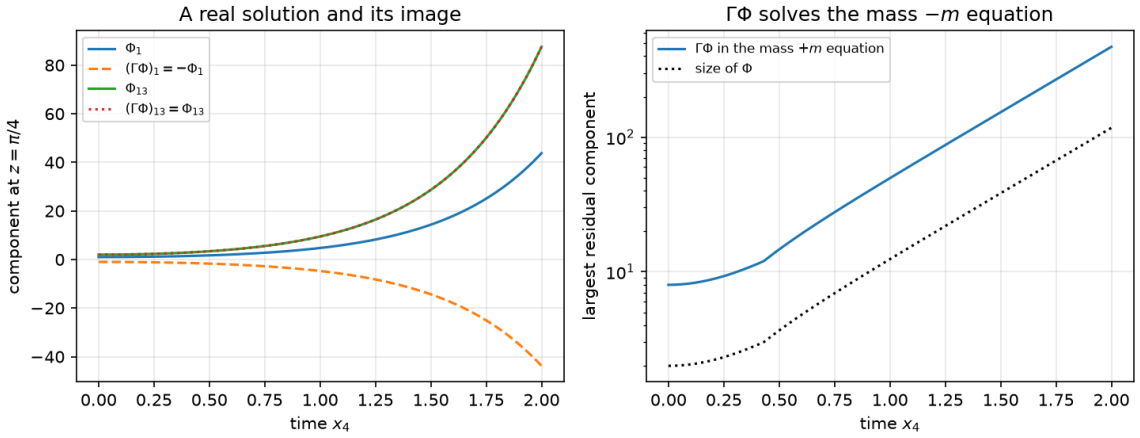

Figure 228: The real matrix Γ reverses the mass. Left: components 1 and 13 of the real exact solution Φ (illustration values $H = 1$, $m = 2$, $\alpha = 0$, $\chi_0 = e_1 + 2e_{13}$, at $z = \pi/4$) and of its image $\Gamma\Phi$ against the time x_4: Γ reverses the sign of the components 1 to 8 and keeps 9 to 16. Right (logarithmic axis): the largest component of the residual of $\Gamma\Phi$ in the equation with mass $+m$ (solid), which is $2m$ times the size of Φ (dotted); in the equation with mass $-m$ the residual is zero up to rounding (below 10^{-12} of the size, not drawn). A real field has no charge; Γ relates solutions of masses m and $-m$. (Notebook 07c, figure 4.)

Out [14]:

```
Figure 07c.4 saved as Revision/textbook/figures/07c_4_mass_reversal.png
```

Text cell:

```
## 13. The last check

The last cell checks that the four figure files exist and prints the number of
checks that passed.
```

In [15]:

```
names = ["majorana_control", "commutation_pattern", "gamma_map_matrices",
         "mass_reversal"]
files = [f"{FIGURE_FOLDER}/07c_{k}_{name}.png" for k, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 4 figure files of this notebook exist")
all_checks_passed()
```

Out [15]:

```
PASS all 4 figure files of this notebook exist
ALL 23 CHECKS PASSED (notebook 07c)
```

Text cell:

```
## 14. What this notebook showed

- The Majorana-type Lagrangian $L_g = \sqrt{|g|}\,\Theta^T C\gamma^\mu D_\mu\Theta$
  of the author's notebook is, for a real anticommuting 16-component field in the
  author's metric, exactly the total derivative of $\frac12\sqrt{|g|}\,\Theta^T
  C\gamma^\mu\Theta$: no field equation; its mass term $\Theta^T C\Theta$ vanishes.
  For real commuting components it gives a field equation in all 16 components.
  This negative control is why the Revision Lagrangian uses the complex field with
  $\bar\Psi = \Psi^\dagger C$.
- The author's gammas, $C$, $\Gamma$ and the spin connection are real; the field
  equation is real; real fields are a consistent restriction, and the real
  Lagrangian gives $\gamma^\mu D_\mu\Phi = (m + \lambda S)\Phi$.
- Charge conjugation is a MATRIX map. Every matrix $M$ with $M\gamma^{(a)} =
  s\gamma^{(a)}M$ for all $a$ is a multiple of 1 ($s = +1$) or of $\Gamma$ ($s =
  -1$), because among the 256 products of gammas only the identity commutes and
  only $\Gamma$ anticommutes with all eight. The two charge-conjugation matrices are
  $\mathcal{C}_+ = C$ (same mass, $\Psi^c = \Psi^*$) and $\mathcal{C}_- = \Gamma C$
  (mass reversed, $\Psi^c = \Gamma\Psi^*$); both reality conditions are consistent.
- For a REAL field plain complex conjugation is the identity, and so is
  $\mathcal{C}_+$: a real field is its own conjugate and its current $J^a$ vanishes.
  The nontrivial real map is the matrix $\Gamma$ with $(m, \lambda) \to (-m,
  -\lambda)$: $L_{m,\lambda}[\Gamma\Phi] = -L_{-m,-\lambda}[\Phi]$, and $\Gamma$ maps
  the real solutions of mass $m$ onto real solutions of mass $-m$.
- These are exact maps between solutions of two equations (PROVED, with the checks
  named above). They do not say that such solutions are created, in pairs or
  otherwise.
```

7.33 Line-by-line walk-through of Notebook 07c

The notebook has 15 code cells, In [1] to In [15]. This section explains every line of every one of them, with the conventions of Section 7.9; every caption is printed in full under its figure in Section 7.32.

In [1], the set-up cell. It is the set-up cell of Notebook 07d with the one line `NOTEBOOK_ID = "07c"`; its first part repeats the complete run instructions of Section 7.31 as comment lines, and its code is explained line by line in Section 7.9. **Out [1]:** `Set-up of notebook 07c complete: repository folder found, helpers defined.`

In [2]: the matrices are real or imaginary.

```
import contextlib  # redirect printed text into a buffer
import io  # an in-memory text file (the buffer)
import re  # regular expressions: read a number from a check's detail text
from itertools import combinations  # all subsets of the eight gammas

import numpy as np  # integer and floating-point arrays
import sympy as sp  # exact algebra

check_of_the_setup = check  # the helper check of the set-up cell


def check(condition, name, record=None):
    """The set-up cell's check, with its PASS line and its "reproduces" line
    printed by ONE print call, so that Jupyter delivers them together."""
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # collect what check prints
        check_of_the_setup(condition, name, record)
    print(buffer.getvalue(), end="")  # and print it in one piece
```

The modules (as in Notebook 07d, plus `combinations` from `itertools`, which lists all subsets of a given size) and the wrapper of `check` that prints a PASS line and its "reproduces" line in one piece (Section 7.9, In [2]).

```
PYREP = "Revision/theory/reports/python-field-theory.json"
WLREP = "Revision/theory/reports/wolfram-field-theory.json"
LEAD = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
PYREP_B4 = "Revision/theory/reports/python-field-theory-b4.json"  # the metric with b4


def revision_check(report, name):
    """The check called name of a Revision report: its dictionary (name, verdict,
    detail); stops if it is missing or its verdict is not PASS."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    found = [item for item in data["checks"] if item["name"] == name]
    if len(found) != 1 or found[0]["verdict"].upper() != "PASS":
        raise ValueError(f"{name} is not a passing check of {report}")
    return found[0]
```

```
def reproduces(report, name):
    """The text "<report>, check <name>" after making sure the check passes."""
    revision_check(report, name)
    return f"{report}, check {name}"
```

The paths of the three Revision reports used (the sympy and Wolfram field-theory reports and the lead's charge-conjugation report), and the helpers `revision_check` and `reproduces` of Section 7.9.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
G = [sp.Matrix(g) for g in fixture["gamma"]]  # gamma^(x0) .. gamma^(x7)
C = sp.Matrix(fixture["C"])
GAMMA = G[0] * G[1] * G[2] * G[3] * G[4] * G[5] * G[6] * G[7]  # chirality
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
X0, X4 = 0, 4  # list positions of x0 and x4
B = -sp.I * C * G[X4]
```

The gammas and C read from `Revision/algebra/gammas.json` as exact sympy matrices. `GAMMA` is the chirality matrix computed as the product $\gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$ in the author's order; `I16`, `Z16` the identity and zero matrices; `X0`, `X4` the list positions of x_0 and x_4; `B` is $-iC\gamma^{(x_4)}$.

```
real = all(v.is_real for M in G + [C, GAMMA] for v in M)
check(real and C.T == C and C * C == I16 and GAMMA == sp.Matrix(fixture["Gamma"])
      and GAMMA * GAMMA == I16 and all(GAMMA * g == -g * GAMMA for g in G),
      "the gammas, C and Gamma are real; C^T = C, C^2 = 1; Gamma^2 = 1 and Gamma "
      "anticommutes with every gamma", record=reproduces(LEAD, "representation_real"))
check(all(sp.re(v) == 0 for v in B) and B.H == B and B * B == I16,
      "B = -i C gamma^(x4) is purely imaginary and Hermitian, B^2 = 1",
      record=reproduces(LEAD, "B_imaginary_hermitian"))
```

`real` is true when every entry of every gamma, of C and of Γ is a real number (`v.is_real`). The first check adds: $C^T = C$, $C^2 = 1$, Γ equal to the record's matrix (key `"Gamma"`), $\Gamma^2 = 1$, and Γ anticommutes with every gamma. The second: every entry of B has zero real part (`sp.re(v) == 0`), $B^\dagger = B$ (`B.H` is the conjugate transpose) and $B^2 = 1$.

```
r = sp.Matrix(sp.symbols("r1:17", real=True))  # a real column of 16 numbers
check(r.conjugate() == r, "complex conjugation leaves a real column unchanged: on "
      "a real field it is the identity")
```

`r` is a column of 16 real symbols; its complex conjugate (`r.conjugate()`) equals itself: on a real field, complex conjugation is the identity. **Out [2]:** three PASS lines, the first two with "reproduces" lines naming `charge-conjugation-and-u1.json`, checks `representation_real` and `B_imaginary_hermitian`.

In [3]: the metric and its spin connection, in brief.

```
E, s, c = sp.symbols("E s c", positive=True)  # e^a4, sin(z)^(1/6), cos z
A1, A2, A3 = sp.symbols("A1 A2 A3", real=True)  # derivatives of a4 (1st to 3rd)
```

```
b4 = sp.Symbol("b4", positive=True)  # b4(x4) > 0
b4p, b4pp, b4ppp = sp.symbols("b4p b4pp b4ppp", real=True)  # its 1st to 3rd derivative
H = sp.Symbol("H", positive=True)
m, lam = sp.symbols("m lambda", real=True)
ETA = [1, 1, 1, 1, -1, -1, -1, -1]
```

The ring symbols and η, as in Notebook 07b (Section 7.27, In [3]).

```
def cd(expression, mu):
    """The derivative of a ring expression along coordinate mu (0 .. 7)."""
    expression = sp.sympify(expression)
    if mu == X4:
        return (sp.diff(expression, E) * E * A1 + sp.diff(expression, A1) * A2
                + sp.diff(expression, A2) * A3 + sp.diff(expression, b4) * b4p
                + sp.diff(expression, b4p) * b4pp + sp.diff(expression, b4pp) * b4ppp)
    if mu == X0:
        return (sp.diff(expression, s) * H * c / s**5
                + sp.diff(expression, c) * (-6 * H * s**6))
    return sp.Integer(0)


def is_zero(expression):
    """Exact test: the numerator vanishes after c^2 -> 1 - s^12."""
    numerator, _ = sp.fraction(sp.together(sp.sympify(expression)))
    numerator = sp.expand(numerator)
    if numerator == 0:
        return True
    remainder = sp.rem(sp.Poly(numerator, c), sp.Poly(c**2 - 1 + s**12, c))
    return sp.expand(remainder.as_expr()) == 0
```

The ring derivative `cd` and the exact zero test `is_zero`, the same functions as in Notebook 07b (explained in Section 7.27, In [3]).

```
f = [c / s**6] + [E * s] * 3 + [sp.Integer(1)] + [s * sp.sqrt(b4) / E] * 3  # frame
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal metric
sqrt_g = sp.prod(f)  # = b4^(3/2) c = b4^(3/2) cos z
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for l in range(8):
    for mu in range(8):
        for nu in range(8):
            value = ((cd(g[l], mu) if l == nu else 0) + (cd(g[l], nu) if l == mu else 0)
                     - (cd(g[mu], l) if mu == nu else 0))
            if value != 0:
                Gam[l][mu][nu] = sp.expand(value / (2 * g[l]))
```

The frame factors `f` (with b_4, in the order $x_0, \ldots, x_7$), the metric entries `g`, the volume factor $\sqrt{|g|}$ (which is $b_4^{3/2} c = b_4^{3/2} \cos z$), and the Christoffel symbols of the diagonal formula, written here with conditional expressions (`(cd(g[l], mu) if l == nu else 0)` is the first term, and so on) instead of three `if` statements.

```
SAB = [[(G[a] * G[b] - G[b] * G[a]) / 4 for b in range(8)] for a in range(8)]
Om = []  # the spinor connection Omega_mu
for mu in range(8):
    M = sp.zeros(16, 16)
    for a in range(8):
        for b in range(a + 1, 8):
            mixed = f[a] * Gam[a][mu][b] / f[b]  # omega_mu^a_b for a != b
            if mixed != 0:
                M += sp.expand(ETA[a] * mixed) * SAB[a][b]
    Om.append(M.applyfunc(sp.expand))
gam = [G[a] / f[a] for a in range(8)]  # the coordinate gammas
```

The spin generators S^{ab} and the spinor connection $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$. Only $a \neq b$ is needed (because $S^{aa} = 0$), and for $a \neq b$ the formula of Section 7.18 reduces to $\omega_{\mu ab} = \eta_{aa} f_a \Gamma^a{}_{\mu b} / f_b$ (`mixed` is the part before η_{aa}). `gam` holds the coordinate gammas $\gamma^{(\mu)}/f_\mu$.

```
# real: no imaginary unit, and sympy does not find the entry non-real
real_entries = all(not v.has(sp.I) and v.is_real is not False
                   for M in Om + gam for v in M)
check(real_entries, "every entry of every Omega_mu and gamma^mu is real, for every "
      "b4: gamma^mu D_mu is a real operator",
      record=reproduces(PYREP_B4, "spinor_connection_real_b4") + "; at b4 = 1 "
      + reproduces(LEAD, "spinor_connection_real"))
total = sum((gam[mu] * Om[mu] for mu in range(8)), Z16)
at_b4_1 = {b4: 1, b4p: 0, b4pp: 0, b4ppp: 0}  # the background b4 = 1
check(all(is_zero(v) for v in (total - 3 * H * G[X0] - 3 * b4p / (4 * b4) * G[X4]))
      and all(is_zero(v) for v in (total.subs(at_b4_1) - 3 * H * G[X0])),
      "gamma^mu Omega_mu = 3 H gamma^(x0) + (3/4)(b4'/b4) gamma^(x4), and "
      "3 H gamma^(x0) at b4 = 1: the values of the Revision records",
      record=reproduces(PYREP_B4, "gammaOmega_total") + "; at b4 = 1 "
      + reproduces(PYREP, "gamma_mu_Omega_mu_equals_3H_gamma_x0"))
```

`real_entries` is true when no entry of any Ω_μ or γ^μ contains the imaginary unit and sympy does not find any entry non-real (`v.is_real is not False`: sympy answers true, false, or "unknown", and only an explicit false would fail). So $\gamma^\mu D_\mu$ is a real operator, for every b_4. The second check is the consistency check $\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$, and $3H\gamma^{(x_0)}$ at $b_4 = 1$ (`at_b4_1` puts in $b_4 = 1$). **Out [3]:** two PASS lines with their "reproduces" lines (`spinor_connection_real_b4` and `gammaOmega_total` of the sympy report with b_4; at $b_4 = 1$ `spinor_connection_real` of the lead's report and `gamma_mu_Omega_mu_equals_3H_gamma_x0` of the sympy report).

In [4]: a jet algebra for a real field.

```
def sort_sign(keys, odd):
    """Sorted tuple of generators and the sign; (None, 0) for a zero product."""
    items, sign = list(keys), 1
    for end in range(len(items) - 1, 0, -1):  # bubble sort, counting exchanges
        for i in range(end):
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                sign = -sign
    if not odd:
```

```
        return tuple(items), 1
    if any(items[i] == items[i + 1] for i in range(len(items) - 1)):
        return None, 0
    return tuple(items), sign
```

`sort_sign` is the bubble sort with the Grassmann sign of Notebook 07b (Section 7.27, In [12]).

```
class RealJet:
    """An element of the jet algebra of a real field: monomial -> coefficient."""

    def __init__(self, odd, terms=None):
        self.odd, self.terms = odd, {}
        for monomial, coefficient in (terms or {}).items():
            self.add(monomial, coefficient)

    def add(self, monomial, coefficient):
        total = sp.expand(self.terms.get(monomial, 0) + coefficient)
        if total == 0:
            self.terms.pop(monomial, None)
        else:
            self.terms[monomial] = total

    def __add__(self, other):
        result = RealJet(self.odd, self.terms)
        for monomial, coefficient in other.terms.items():
            result.add(monomial, coefficient)
        return result

    def __sub__(self, other):
        return self + other.times(-1)

    def times(self, factor):
        return RealJet(self.odd, {k: factor * v for k, v in self.terms.items()})

    def __mul__(self, other):
        result = RealJet(self.odd)
        for k1, v1 in self.terms.items():
            for k2, v2 in other.terms.items():
                monomial, sign = sort_sign(k1 + k2, self.odd)
                if monomial is not None:
                    result.add(monomial, sign * v1 * v2)
        return result
```

The class `RealJet` is the class `Jet` of Notebook 07b without the conjugation: a real field has no separate conjugate components, so a generator is a pair `(A, derivatives)` (component and derivative tuple), and there is no kind. `add`, addition, subtraction, `times` and the product work exactly as there.

```
    def lderiv(self, key):
        result = RealJet(self.odd)
        for monomial, coefficient in self.terms.items():
```

```
            if key in monomial:
                p = monomial.index(key)
                factor = (-1) ** p if self.odd else monomial.count(key)
                result.add(monomial[:p] + monomial[p + 1:], factor * coefficient)
        return result

    def total(self, mu):
        result = RealJet(self.odd)
        for monomial, coefficient in self.terms.items():
            derivative = cd(coefficient, mu)
            if derivative != 0:
                result.add(monomial, derivative)
            for p, (A, d) in enumerate(monomial):
                raised = (A, tuple(sorted(d + (mu,))))
                ordered, sign = sort_sign(monomial[:p] + (raised,) + monomial[p + 1:],
                                          self.odd)
                if ordered is not None:
                    result.add(ordered, sign * coefficient)
        return result

    def is_zero(self):
        return all(is_zero(v) for v in self.terms.values())
```

The left derivative (with the Grassmann sign $(-1)^p$, or the power for commuting generators), the total derivative d/dx_μ (the coefficient differentiated with `cd`, each generator raised in turn), and the exact zero test of an element; all as in Notebook 07b.

```
def column(odd, derivatives=()):
    """The 16 generators theta_A (or one of their derivatives)."""
    return [RealJet(odd, {((A, tuple(sorted(derivatives))),): 1}) for A in range(16)]


def matvec(M, v, odd):
    out = [RealJet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[i] = out[i] + v[j].times(M[i, j])
    return out


def vecmat(v, M, odd):
    out = [RealJet(odd) for _ in range(16)]
    for i in range(16):
        for j in range(16):
            if M[i, j] != 0:
                out[j] = out[j] + v[i].times(M[i, j])
    return out


def dot(row, col, odd):
    result = RealJet(odd)
    for x, y in zip(row, col):
```

```
        result = result + x * y
    return result


say("The jet algebra of a real field is defined.")
```

The vector helpers `column` (the 16 generators θ_A or one of their derivatives), `matvec`, `vecmat` and `dot`, as in Notebook 07b. **Out [4]:** `The jet algebra of a real field is defined.`

In [5]: the Majorana-type Lagrangian for both kinds of numbers.

```
results = {}
for name, odd in (("grassmann", True), ("commuting", False)):
    theta = column(odd)
    row = vecmat(theta, C, odd)  # Theta^T C
    Lg = RealJet(odd)
    derivative_form = RealJet(odd)
    for mu in range(8):
        D_theta = [x + y for x, y in zip(column(odd, (mu,)),
                                         matvec(Om[mu], theta, odd))]
        Lg = Lg + dot(row, matvec(gam[mu], D_theta, odd), odd)
        current = dot(row, matvec(gam[mu], theta, odd), odd).times(sqrt_g / 2)
        derivative_form = derivative_form + current.total(mu)
    Lg = Lg.times(sqrt_g)
```

For real Grassmann and real commuting components: `theta` is the column of the 16 generators and `row` the row $\Theta^T C$. For each direction, `D_theta` is $\partial_\mu\Theta + \Omega_\mu\Theta$, `Lg` collects $\Theta^T C\gamma^\mu D_\mu\Theta$, and `derivative_form` collects $\frac{d}{dx_\mu}\big(\frac{1}{2}\sqrt{|g|}\,\Theta^T C\gamma^\mu\Theta\big)$, the candidate total-derivative form of Section 7.28. Finally `Lg` is multiplied by $\sqrt{|g|}$.

```
    euler_lagrange = []
    for A in range(16):
        value = Lg.lderiv((A, ()))
        for mu in range(8):
            value = value - Lg.lderiv((A, (mu,))).total(mu)
        euler_lagrange.append(value)
    results[name] = {"Lg": Lg, "derivative_form": derivative_form,
                     "EL": euler_lagrange, "mass": dot(row, theta, odd),
                     "theta": theta}
    nonzero = sum(1 for e in euler_lagrange if not e.is_zero())
    report(f"{name}: monomials of Lg", len(Lg.terms))
    report(f"{name}: nonzero Euler-Lagrange expressions", f"{nonzero} of 16")
```

The 16 Euler-Lagrange expressions with left derivatives, stored with the other pieces in the dictionary `results`; the mass-type term $\Theta^T C\Theta$ is stored too. The two `report` lines print the number of monomials of L_g and how many Euler-Lagrange expressions are not zero. **Out [5]:** four lines,

```
RESULT grassmann: monomials of Lg = 144
RESULT grassmann: nonzero Euler-Lagrange expressions = 0 of 16
RESULT commuting: monomials of Lg = 128
RESULT commuting: nonzero Euler-Lagrange expressions = 16 of 16
```

In [6]: the anticommuting case against the record.

```
grassmann = results["grassmann"]
check(len(grassmann["Lg"].terms) > 0
      and (grassmann["Lg"] - grassmann["derivative_form"]).is_zero(),
      "real Grassmann field: Lg = (1/2) d_mu (sqrt|g| Theta^T C gamma^mu Theta), not 0",
      record=reproduces(PYREP, "negative_control_majorana_grassmann_total_derivative"))
check(all(e.is_zero() for e in grassmann["EL"]) and grassmann["mass"].is_zero(),
      "real Grassmann field: all 16 Euler-Lagrange expressions vanish and Theta^T C "
      "Theta = 0", record=reproduces(WLREP, "Majorana_Lg_total_derivative_grassmann"))
```

The first check: L_g is not zero and equals its total-derivative form exactly. The second: all 16 Euler-Lagrange expressions vanish and $\Theta^T C \Theta = 0$. **Out [6]:** two PASS lines with their "reproduces" lines (`negative_control_majorana_grassmann_total_derivative` of the sympy report, `Majorana_Lg_total_derivative_grassmann` of the Wolfram report).

In [7]: the commuting case, and Figure 07c.1.

```
commuting = results["commuting"]
phi = commuting["theta"]
operator = [RealJet(False) for _ in range(16)]  # gamma^mu D_mu Phi
for mu in range(8):
    D_phi = [x + y for x, y in zip(column(False, (mu,)), matvec(Om[mu], phi, False))]
    operator = [x + y for x, y in zip(operator, matvec(gam[mu], D_phi, False))]
expected = matvec(C * 2 * sqrt_g, operator, False)  # 2 sqrt|g| C gamma^mu D_mu Phi
nonzero = sum(1 for e in commuting["EL"] if not e.is_zero())
detail = revision_check(PYREP, "negative_control_majorana_commuting_contrast")["detail"]
recorded = int(re.search(r"in (\d+) of 16 components", detail).group(1))
check(nonzero == recorded == 16, "real commuting field: Lg gives nonzero "
      "Euler-Lagrange expressions in 16 of 16 components",
      record=reproduces(PYREP, "negative_control_majorana_commuting_contrast"))
vector_zero = all(dot(vecmat(phi, C, False), matvec(G[a], phi, False), False)
                  .is_zero() for a in range(8))
check(all((e - x).is_zero() for e, x in zip(commuting["EL"], expected))
      and vector_zero, "real commuting field: the expression is 2 sqrt|g| (C "
      "gamma^mu D_mu Phi)_A and Phi^T C gamma^(a) Phi = 0",
      record=reproduces(WLREP, "Majorana_Lg_commuting_control"))
```

`operator` is the column $\gamma^\mu D_\mu \Phi$ for the real commuting field, `expected` the column $2\sqrt{|g|}\, C\gamma^\mu D_\mu \Phi$ of Section 7.28. `recorded` is the number read from the detail text of the sympy check ("in 16 of 16 components"). The first check: 16 nonzero expressions, as recorded. `vector_zero` is true when $\Phi^T C \gamma^{(a)} \Phi = 0$ for all eight a. The second check: every expression equals the expected one, and the vector forms vanish.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
labels = ["real Grassmann", "real commuting"]
left.bar(labels, [len(grassmann["Lg"].terms), len(commuting["Lg"].terms)],
         color=["tab:blue", "tab:orange"])
left.set_ylabel("monomials of $L_g$")
left.set_title("$L_g$ is not zero in either case")
counts = [sum(1 for e in grassmann["EL"] if not e.is_zero()), nonzero]
```

```
right.bar(labels, counts, color=["tab:blue", "tab:orange"])
right.set_ylim(0, 17)
right.set_ylabel("nonzero Euler-Lagrange expressions")
right.set_title("Field equations from $L_g$")
for x, value in enumerate(counts):
    right.text(x, value + 0.4, f"{value} of 16", ha="center")
fig.tight_layout()
save_figure(fig, "majorana_control",
...)
```

Two bar charts: the number of monomials of L_g, and the number of nonzero Euler-Lagrange expressions with the text "0 of 16" and "16 of 16" written above the bars. **Out [7]:** two PASS lines with their "reproduces" lines and the figure. **What Figure 07c.1 shows.** The left panel shows that L_g is not zero for either kind of numbers (144 and 128 monomials for a general b_4). The right panel shows the decisive difference: for anticommuting components no Euler-Lagrange expression survives, L_g gives no field equation; for commuting components all 16 do.

In [8]: all matrices that turn solutions into solutions.

```
Gn = [np.array(gm.tolist(), dtype=np.int64) for gm in G]  # exact integer arrays
products, commute_counts, degrees = [], [], []
for k in range(9):
    for subset in combinations(range(8), k):
        P = np.eye(16, dtype=np.int64)
        for a in subset:
            P = P @ Gn[a]  # gamma_(a1) ... gamma_(ak)
        products.append(P)
        degrees.append(k)
        commute_counts.append(sum(1 for a in range(8)
                                  if np.array_equal(P @ Gn[a], Gn[a] @ P)))
flat = np.array([P.reshape(256) for P in products])  # one row per product
gram = flat @ flat.T  # all traces tr(gamma_I^T gamma_J), exact integers
check(len(products) == 256 and np.array_equal(gram, 16 * np.eye(256, dtype=np.int64)),
      "the 256 products of gammas are orthogonal: a basis of all 16 x 16 matrices")
```

`Gn` holds the gammas as exact 64-bit integer arrays. The loops run over all subsets of the eight directions, by size $k = 0$ to 8 (`combinations(range(8), k)` lists the subsets of size k in increasing order); `P` is the product of the gammas of the subset, starting from the identity. For each product the notebook stores it, its degree, and the number of gammas it commutes with. `flat` writes each product as one row of 256 numbers (`reshape(256)`), and `gram = flat @ flat.T` is the table of all traces $\mathrm{tr}(\gamma_I^T \gamma_J)$ (the sum of the products of corresponding entries is exactly this trace). The check: 256 products, and the table is 16 times the 256×256 identity.

```
same = [i for i in range(256) if commute_counts[i] == 8]  # commute with all
reversed_ = [i for i in range(256) if commute_counts[i] == 0]  # anticommute with all
report("products that commute with all 8 gammas (degree)", [degrees[i] for i in same])
report("products that anticommute with all 8 gammas (degree)",
       [degrees[i] for i in reversed_])
GAMMAn = np.array(GAMMA.tolist(), dtype=np.int64)
check(len(same) == 1 and np.array_equal(products[same[0]], np.eye(16, dtype=np.int64)),
      "s = +1: the solutions M form a 1-dimensional space, spanned by the identity",
```

```
      record=reproduces(LEAD, "intertwiners_same_mass"))
check(len(reversed_) == 1 and (np.array_equal(products[reversed_[0]], GAMMAn)
                               or np.array_equal(products[reversed_[0]], -GAMMAn)),
      "s = -1: the solutions M form a 1-dimensional space, spanned by Gamma",
      record=reproduces(LEAD, "intertwiners_reversed_mass"))
```

`same` lists the products that commute with all eight gammas, `reversed_` those that commute with none (they anticommute with all eight); the trailing underscore avoids the name of Python's function `reversed`. Their degrees are reported. The two checks: exactly one product commutes with all, and it is the identity; exactly one anticommutes with all, and it is $\pm\Gamma$. **Out [8]:** a PASS line, `RESULT products that commute with all 8 gammas (degree) = [0]`, `RESULT p roducts that anticommute with all 8 gammas (degree) = [8]`, and two PASS lines with their "reproduces" lines (`intertwiners_same_mass` and `intertwiners_reversed_mass`).

In [9]: the commutation pattern, Figure 07c.2.

```
table = np.zeros((9, 9), dtype=int)  # rows: degree k, columns: number commuted with
for k, j in zip(degrees, commute_counts):
    table[k, j] += 1
binomials = [1, 8, 28, 56, 70, 56, 28, 8, 1]  # the number of products of k gammas
check(all(table[k, k if k % 2 else 8 - k] == binomials[k] for k in range(9)),
      "a product of k gammas commutes with k of them (k odd) or 8 - k (k even)")
```

`table[k, j]` counts the products of k gammas that commute with j of the eight. `binomials` are the numbers $\binom{8}{k}$ of products of k gammas. The check: all $\binom{8}{k}$ products of k gammas commute with exactly k gammas for odd k and $8-k$ for even k (Section 7.29).

```
fig, ax = plt.subplots(figsize=(6.6, 5.4))
shown = np.where(table > 0, np.log10(np.maximum(table, 1)) + 0.3, 0.0)
image = ax.imshow(shown, cmap="Purples", vmin=0, vmax=2.3)
for k in range(9):
    for j in range(9):
        if table[k, j]:
            ax.text(j, k, str(table[k, j]), ha="center", va="center",
                    color="white" if table[k, j] > 20 else "black")
ax.add_patch(plt.Rectangle((7.5, -0.5), 1, 1, fill=False, edgecolor="red",
                           linewidth=2))
ax.add_patch(plt.Rectangle((-0.5, 7.5), 1, 1, fill=False, edgecolor="red",
                           linewidth=2))
ax.set_xticks(range(9))
ax.set_yticks(range(9))
ax.set_xlabel("number of the 8 gammas that the product commutes with")
ax.set_ylabel("number $k$ of gamma factors in the product")
ax.set_title("The 256 products of gammas")
ax.grid(False)
save_figure(fig, "commutation_pattern",
...)
```

A heat map of the table on a logarithmic colour scale (`np.log10` of the counts, shifted so that 1 is still visible, and 0 left white), with the counts written in, and two red frames (`plt.Rectangle`)

around the two squares of the outer columns. **Out [9]:** a PASS line and the figure. **What Figure 07c.2 shows.** The counts form two diagonals: odd products on the line "commutes with k", even products on the line "commutes with $8-k$". Only two squares reach the outer columns: the identity (row 0, column 8) and Γ (row 8, column 0). Every matrix that commutes or anticommutes with all eight gammas is therefore a multiple of 1 or of Γ.

In [10]: the two charge-conjugation matrices.

```
C_plus, C_minus = C, GAMMA * C
check(all(C_plus.inv() * g * C_plus == -g.T for g in G) and C_plus.T == C_plus,
      "calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real symmetric",
      record=reproduces(LEAD, "charge_conjugation_matrix_plus"))
check(all(C_minus.inv() * g * C_minus == g.T for g in G)
      and all(v.is_real for v in C_minus),
      "calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real",
      record=reproduces(LEAD, "charge_conjugation_matrix_minus"))
check(I16 * I16.conjugate() == I16 and GAMMA * GAMMA.conjugate() == I16,
      "M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent",
      record=reproduces(LEAD, "majorana_conditions_consistent"))
```

$\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$. The first check: $\mathcal{C}_+^{-1}\gamma^{(a)}\mathcal{C}_+ = -(\gamma^{(a)})^T$ for every gamma (`C_plus.inv()` is the inverse matrix), and $\mathcal{C}_+$ is symmetric. The second: $\mathcal{C}_-^{-1}\gamma^{(a)}\mathcal{C}_- = +(\gamma^{(a)})^T$, and $\mathcal{C}_-$ is real. The third: $MM^* = 1$ for $M = 1$ and $M = \Gamma$. **Out [10]:** three PASS lines with their "reproduces" lines (`charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus`, `majorana_conditions_consistent`).

In [11]: real fields under charge conjugation, Figure 07c.3.

```
phi_bar_T = (r.T * C).T  # (Phi^T C)^T for the real column r
check((C_plus * phi_bar_T - r).applyfunc(sp.expand) == sp.zeros(16, 1)
      and (C_minus * phi_bar_T - GAMMA * r).applyfunc(sp.expand) == sp.zeros(16, 1),
      "for a real field Phi: calC_+ gives Phi itself, calC_- gives Gamma Phi")
currents = [sp.expand((-sp.I * r.T * C * g * r)[0, 0]) for g in G]
kinetic_reversed = all(GAMMA.T * C * g * GAMMA == -(C * g) for g in G)
check(all(j == 0 for j in currents) and GAMMA.T * C * GAMMA == C and kinetic_reversed,
      "real commuting field: J^a = 0 for every a; Gamma^T C Gamma = C and Gamma^T C "
      "gamma^a Gamma = -C gamma^a",
      record=reproduces(LEAD, "real_fields_charge_conjugation"))
```

`phi_bar_T` is $\bar{\Phi}^T = (\Phi^T C)^T$ for the real column `r` of In [2]. The first check: $\mathcal{C}_+\bar{\Phi}^T = \Phi$ and $\mathcal{C}_-\bar{\Phi}^T = \Gamma\Phi$. `currents` are the eight components $J^a = -i\Phi^T C\gamma^{(a)}\Phi$ for the real commuting column; `kinetic_reversed` tests $\Gamma^T C\gamma^{(a)}\Gamma = -C\gamma^{(a)}$ for all a. The second check: every $J^a = 0$, $\Gamma^T C\Gamma = C$, and the kinetic matrices are reversed.

```
numbers = [np.array(M.tolist(), dtype=float) for M in
           (C, GAMMA.T * C * GAMMA, C * G[X4], GAMMA.T * C * G[X4] * GAMMA)]
titles = ["$C$", "$\\Gamma^T C\\,\\Gamma$ (equal)", "$C\\gamma^{(x_4)}$",
          "$\\Gamma^T C\\gamma^{(x_4)}\\Gamma$ (opposite)"]
fig, axes = plt.subplots(2, 2, figsize=(7.6, 7.4))
for ax, M, title in zip(axes.flat, numbers, titles):
```

```
    image = ax.imshow(M, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.grid(False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.6)
save_figure(fig, "gamma_map_matrices",
...)
```

Four heat maps in a 2×2 arrangement (`axes.flat` runs through the four panels): C, $\Gamma^T C \Gamma$, $C\gamma^{(x_4)}$ and $\Gamma^T C \gamma^{(x_4)} \Gamma$, with one common colour scale. **Out [11]:** two PASS lines (the second with its "reproduces" line, `real_fields_charge_conjugation`) and the figure. **What Figure 07c.3 shows.** The two upper maps are identical: Γ keeps the scalar density. The two lower maps have the same pattern with every colour swapped: Γ reverses the kinetic matrix of x_4 (and, as the check showed, of all eight directions).

In [12]: the mass reversal in the author's metric.

```
def real_lagrangian(columns, mass, coupling):
    """sqrt|g| [K - mass S - (coupling/2) S^2] of the real field whose value and
    derivatives are columns(derivatives)."""
    field = columns(())
    bar = vecmat(field, C, False)  # Phi^T C
    K = RealJet(False)
    for mu in range(8):
        D_field = [x + y for x, y in zip(columns((mu,)), matvec(Om[mu], field, False))]
        D_bar = [x - y for x, y in zip(vecmat(columns((mu,)), C, False),
                                       vecmat(bar, Om[mu], False))]
        K = K + dot(bar, matvec(gam[mu], D_field, False), False)
        K = K - dot(vecmat(D_bar, gam[mu], False), field, False)
    S = dot(bar, field, False)
    return (K.times(sp.Rational(1, 2)) - S.times(mass)
            - (S * S).times(coupling / 2)).times(sqrt_g)
```

`real_lagrangian(columns, mass, coupling)` builds the Revision Lagrangian of a real commuting field whose value and derivatives are given by the function `columns(derivatives)`: the row $\bar{\Phi} = \Phi^T C$, the two halves of the kinetic term with the covariant derivatives $D_\mu \Phi$ and $D_\mu \bar{\Phi} = \partial_\mu \Phi^T C - \bar{\Phi} \Omega_\mu$, the scalar density, and $\sqrt{|g|}[\frac{1}{2}K - mS - \frac{\lambda}{2}S^2]$ with the given mass and coupling.

```
def plain(derivatives):  # the components of Phi
    return column(False, derivatives)


def mapped(derivatives):  # the components of Gamma Phi
    return matvec(GAMMA, column(False, derivatives), False)
```

Two ways to give the field: `plain` returns the components of Φ (or their derivatives), `mapped` those of $\Gamma\Phi$, that is $\sum_B \Gamma_{AB} \theta_B$ and the same for each derivative (Γ is constant).

```
L_mapped = real_lagrangian(mapped, m, lam)
L_reversed = real_lagrangian(plain, -m, -lam)
```

```
check((L_mapped + L_reversed).is_zero(), "real field in the author's metric: "
      "L_(m, lambda)[Gamma Phi] = -L_(-m, -lambda)[Phi]")
```

`L_mapped` is $\mathcal{L}_{m,\lambda}[\Gamma\Phi]$ and `L_reversed` is $\mathcal{L}_{-m,-\lambda}[\Phi]$; the check requires their sum to vanish exactly.

```
L_real = real_lagrangian(plain, m, lam)
S_real = dot(vecmat(phi, C, False), phi, False)
residual = [x - (RealJet(False, {(): m}) + S_real.times(lam)) * y
           for x, y in zip(operator, phi)]  # gamma D Phi - (m + lambda S) Phi
target = matvec(C * 2 * sqrt_g, residual, False)
ok = True
for A in range(16):
    value = L_real.lderiv((A, ()))
    for mu in range(8):
        value = value - L_real.lderiv((A, (mu,))).total(mu)
    ok = ok and (value - target[A]).is_zero()
check(ok, "real commuting field: Euler-Lagrange expressions 2 sqrt|g| (C (gamma^mu "
      "D_mu Phi - (m + lambda S) Phi))_A")
```

`L_real` is $\mathcal{L}_{m,\lambda}[\Phi]$ and `S_real` the scalar density $\Phi^T C\Phi$. `residual` is the column $\gamma^\mu D_\mu\Phi-(m+\lambda S)\Phi$ (with `operator` from In [7]) and `target` is $2\sqrt{|g|}\,C$ times it. The loop computes the 16 Euler-Lagrange expressions of `L_real` and compares each with `target`. **Out [12]:** two PASS lines.

In [13]: a real solution and its image.

```
mass_symbol = sp.Symbol("mu", real=True)


def M_of(mass):
    return -mass * G[X4] + 3 * H * G[X4] * G[X0]  # alpha = 0, b4 = 1


check((GAMMA * M_of(mass_symbol) * GAMMA - M_of(-mass_symbol)).applyfunc(sp.expand)
      == Z16, "Gamma M_m Gamma = M_(-m): Gamma maps the family of mass m to mass -m")
```

`mass_symbol` is a real symbol μ for the mass, and `M_of(mass)` the matrix M of the exact family of Section 7.23 with $\alpha = 0$. The check: $\Gamma M_\mu \Gamma = M_{-\mu}$ exactly, so Γ maps the member of mass μ to the member of mass $-\mu$.

```
Gf = [np.array(gm.tolist(), dtype=float) for gm in G]
GAMMAf = np.array(GAMMA.tolist(), dtype=float)
chi0 = np.zeros(16)
chi0[0], chi0[12] = 1.0, 2.0  # e_1 + 2 e_13: one component in each chiral half
times = np.linspace(0.0, 2.0, 201)
Mf = -2.0 * Gf[X4] + 3.0 * Gf[X4] @ Gf[X0]  # m = 2, H = 1, b4 = 1
kk = np.sqrt(5.0)
Phi = np.array([(np.cosh(kk * t) * np.eye(16) + np.sinh(kk * t) / kk * Mf) @ chi0
                for t in times])
dPhi = np.array([(kk * np.sinh(kk * t) * np.eye(16) + np.cosh(kk * t) * Mf) @ chi0
                 for t in times])
```

The gammas and Γ as floating-point arrays; `chi0` is $e_1 + 2e_{13}$ (one component in each chiral half); 201 times from 0 to 2; `Mf` is M for $m = 2$, $H = 1$, and $k = \sqrt{5}$. `Phi` and `dPhi` are the real solution $\Phi = (\cosh kx_4 + \sinh(kx_4)/k\, M)\chi_0$ and its exact time derivative, one row per time.

```
def residual_size(values, derivatives, mass):
    """The largest component of gamma4 d4 Psi + 3 gamma0 Psi - mass Psi."""
    return np.array([np.abs(Gf[X4] @ d + 3.0 * Gf[X0] @ v - mass * v).max()
                     for v, d in zip(values, derivatives)])
```

`residual_size(values, derivatives, mass)` returns, at every time, the largest component of $\gamma^{(x_4)}\partial_4\Psi + 3\gamma^{(x_0)}\Psi - \text{mass}\,\Psi$, the residual of the field equation for a field of x_4 alone ($H = 1$).

```
image_values, image_derivatives = Phi @ GAMMAf.T, dPhi @ GAMMAf.T  # Gamma Phi
size = np.abs(Phi).max(axis=1)
phi_plus = residual_size(Phi, dPhi, 2.0) / size
image_minus = residual_size(image_values, image_derivatives, -2.0) / size
image_plus = residual_size(image_values, image_derivatives, 2.0)
check(phi_plus.max() < 1e-12 and image_minus.max() < 1e-12
      and np.allclose(image_plus, 4.0 * size),
      "numerically: Phi solves the mass +2 equation, Gamma Phi the mass -2 equation "
      "(relative residuals below 1e-12); Gamma Phi fails the +2 equation by 2 m |Phi|")
```

The image $\Gamma\Phi$ and its derivative (`Phi @ GAMMAf.T` applies Γ to every row). `phi_plus` is the relative residual of Φ in the equation of mass $+2$, `image_minus` that of $\Gamma\Phi$ in the equation of mass -2, and `image_plus` the residual of $\Gamma\Phi$ in the equation of mass $+2$. The check: the first two are below 10^{-12} at all times, and the third equals $2m = 4$ times the size of Φ (because $\Gamma\Phi$ solves the $-m$ equation, its residual in the $+m$ equation is $-2m\,\Gamma\Phi$). **Out [13]:** two PASS lines.

In [14]: Figure 07c.4.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.8, 3.9))
left.plot(times, Phi[:, 0], label="$\\Phi_1$")
left.plot(times, image_values[:, 0], "--", label="$(\\Gamma\\Phi)_1 = -\\Phi_1$")
left.plot(times, Phi[:, 12], label="$\\Phi_{13}$")
left.plot(times, image_values[:, 12], ":", label="$(\\Gamma\\Phi)_{13} = \\Phi_{13}$")
left.set_xlabel("time $x_4$")
left.set_ylabel("component at $z = \\pi/4$")
left.set_title("A real solution and its image")
left.legend(fontsize=8)
right.semilogy(times, image_plus, label="$\\Gamma\\Phi$ in the mass $+m$ equation")
right.semilogy(times, size, ":", color="black", label="size of $\\Phi$")
right.set_xlabel("time $x_4$")
right.set_ylabel("largest residual component")
right.set_title("$\\Gamma\\Phi$ solves the mass $-m$ equation")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "mass_reversal",
...)
```

Left: components 1 and 13 of Φ and of $\Gamma\Phi$ against the time; right: the residual of $\Gamma\Phi$ in the wrong equation and the size of Φ on a logarithmic axis. **What Figure 07c.4 shows.** Component 1 of the

image is minus component 1 of Φ (the two curves are mirror images), component 13 is unchanged (the curves coincide): $\Gamma = \mathrm{diag}(-I_8, I_8)$. On the right the residual of $\Gamma\Phi$ in the equation of mass $+m$ runs parallel to the size of Φ, a factor 4 above it; in the equation of mass $-m$ its residual is zero up to rounding. A real field carries no charge; the real matrix Γ relates its solutions of mass m to solutions of mass $-m$.

In [15]: the last check.

```
names = ["majorana_control", "commutation_pattern", "gamma_map_matrices",
         "mass_reversal"]
files = [f"{FIGURE_FOLDER}/07c_{k}_{name}.png" for k, name in enumerate(names, 1)]
check(all(output_file(path).is_file() for path in files),
      "all 4 figure files of this notebook exist")
all_checks_passed()
```

The four figure files are checked and the last line printed. **Out [15]:** `PASS all 4 figure files of this notebook exist` and `ALL 23 CHECKS PASSED (notebook 07c)`.

7.34 What we proved, what we computed, what we assumed

PROVED in this chapter (each derivation written out line by line; each is also confirmed by a check of a notebook, and where the Revision record contains it, by the check of the Revision record named in the table after this list):

- the principle of stationary action gives the Euler-Lagrange equation $\partial L/\partial q - \frac{d}{dt}\partial L/\partial \dot{q} = 0$ (Line 1 by the chain rule, Line 2 by integration by parts, Line 3 because the shapes vanish at the ends, and the fundamental lemma); for the oscillator the action of a varied path is $S_0 + S_1\epsilon + S_2\epsilon^2$ with S_2 independent of the path, $S_2 = \frac{T}{4}(\frac{\pi^2}{T^2} - \omega^2)$ for the slowest shape, so the true path is a minimum for $T < \pi/\omega$ and a saddle beyond; the error of a bump average of the parabola path's $E = -2 - t^2$ (with $E'' = -2$) is $-(\frac{1}{3} - \frac{2}{\pi^2})w^2$, and for any quadratic E it is $\frac{1}{2}E''(\frac{1}{3} - \frac{2}{\pi^2})w^2$; the derivative of the grid action is h times the grid Euler-Lagrange expression; $dH/dt = -\dot{q}E$; a total derivative changes no equation; a complex variable and its conjugate may be varied as if independent, $E_{\psi^*} = (E_{q_0} + iE_{q_1})/\sqrt{2}$ (Sections 7.2 to 7.4);
- the Euler-Lagrange expression of a field has one derivative term per coordinate; along a space direction a plane wave oscillates with $\omega^2 = m^2 + k^2$, along an extra time it grows with the rate $\kappa = \sqrt{k^2 - m^2}$ for $k > m$; $(\sum_a k_a\gamma^{(a)})^2 = \sum_a \eta^{aa}k_a^2\, I_{16}$, so the 16-component field has the same growth rate as the record (Section 7.5);
- the rules of Grassmann numbers: $\theta_i^2 = 0$; the sign of a reordering; 2^n monomials, $\binom{n}{k}$ of degree k; $YX = (-1)^{kl}XY$; the conjugation reverses products; $\partial_R F = -\partial_L F$ for even F; $(\Psi^\dagger M\Psi)^* = \Psi^\dagger M^\dagger\Psi$ for both statistics; a real Grassmann column keeps only the antisymmetric part of a matrix and a real commuting column only the symmetric part; S^k has $\binom{16}{k}$ monomials with coefficients $\pm k!$ and $S^{17} = 0$; in the two-generator test algebra $S^3 = 0$; a first-order kinetic term of real Grassmann fields with an antisymmetric matrix is a total derivative (Sections 7.10 to 7.13);
- $\sqrt{|g|} = b_4^{3/2}\cos z$, independent of the time at $b_4 = 1$; $\gamma^{x_i}\Omega_{x_i} = \frac{1}{2}(a_4'\gamma^{(x_4)} + H\gamma^{(x_0)})$ for 3-space and $\gamma^{x_t}\Omega_{x_t} = \frac{1}{2}(-(a_4' - \frac{b_4'}{2b_4})\gamma^{(x_4)} + H\gamma^{(x_0)})$ for the extra times, so $\gamma^\mu\Omega_\mu = M =$

$3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)} = \frac{1}{2\sqrt{|g|}}\partial_\mu(\sqrt{|g|}\gamma^\mu)$, which is $3H\gamma^{(x_0)}$ at $b_4 = 1$: the a_4' terms cancel because the extra times deflate; $\{\gamma^\mu, \Omega_\mu\} = 0$ for each μ (Section 7.18);

- the Lagrangian of both fields is real; it differs from the unsymmetrised form by the total divergence $\frac{1}{2}\partial_\mu(\sqrt{|g|}\bar{\Psi}\gamma^\mu\Psi)$; in the author's metric the spin connection drops out of it (Section 7.20);
- its Euler-Lagrange equations are, for both statistics, $\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi$ and the Dirac-conjugate equation $(D_\mu\bar{\Psi})\gamma^\mu = -(m + \lambda S)\bar{\Psi}$; written out in the author's metric they have the frame factors $\tan z$, $e^{-a_4}\sin^{-1/6} z$, 1, $e^{a_4}b_4^{-1/2}\sin^{-1/6} z$ and the term $M\Psi$; they have a chiral block form and an evolution form with non-characteristic slices $x_4 = \text{const}$ (Sections 7.21 and 7.22);
- non-triviality [1] and [2] in the diagonal frame: for a constant b_4 (as on the backgrounds $b_4 = 1$) the gravitational term $3H\gamma^{(x_0)}\Psi$ is nonzero for every $\Psi \neq 0$ and every $H > 0$; for a changing b_4, $MM = (9H^2 - \frac{9}{16}(b_4'/b_4)^2)1$ and $M\Psi \neq 0$ except at times where $|b_4'/b_4| = 4H$; Ω_μ vanishes only for $H = 0$; with the exact scope that the value of the term depends on the frame and on the variables (the rescaling $\Psi = \sin^{-1/2} z\,\chi$ removes it for a constant b_4, derived here), while Ω_μ vanishes in no frame (Section 7.23);
- the exact family $\Psi = \sin^\alpha z\,(\cosh kx_4 + \sinh(kx_4)/k\,M)\chi_0$ with $M^2 = k^2 = 9H^2(2\alpha+1)^2 - m^2$ and constant S solves the field equation for every a_4 and every constant b_4 when $\lambda = 0$ (Section 7.23);
- the author's Majorana-type Lagrangian is a total derivative for real Grassmann fields (no field equation) and gives $2\sqrt{|g|}C\gamma^\mu D_\mu\Phi$ for real commuting fields; the Revision Lagrangian of a real commuting field gives the same field equation as the complex one, and its current vanishes (Section 7.28);
- charge conjugation is a MATRIX: every matrix M with $M\gamma^{(a)} = s\gamma^{(a)}M$ for all a is a multiple of 1 ($s = +1$) or of Γ ($s = -1$), because the 256 products of gammas are an orthogonal basis and only 1 and Γ commute or anticommute with all eight; hence $\mathcal{C}_+ = C$ with $\mathcal{C}_+^{-1}\gamma^{(a)}\mathcal{C}_+ = -(\gamma^{(a)})^T$ and $\mathcal{C}_- = \Gamma C$ with $\mathcal{C}_-^{-1}\gamma^{(a)}\mathcal{C}_- = +(\gamma^{(a)})^T$; both reality conditions are consistent; for a real field $\mathcal{C}_+$ is the identity and the nontrivial real map is Γ, with $\mathcal{L}_{m,\lambda}[\Gamma\Phi] = -\mathcal{L}_{-m,-\lambda}[\Phi]$; the signs of S and J under the two conjugations for both statistics (Section 7.29).

The checks of the Revision record that state the same results (the theory reports lie in the folder `Revision/theory/reports`, the lead's report is `Revision/lead_checks/reports/charge-conjugation-and-u1.json`):

result	**sections**	**Revision checks**
the Clifford square and the unbounded extra-time growth rate	7.5	`python-field-theory.json`, check `clifford_relations`; `python-scope.json`, check `extra_time_growth_rates_unbounded`

result	sections	Revision checks
the Grassmann rules and the matrix C	7.10 to 7.13	`wolfram-field-theory.json`, check `grassmann_algebra_structure`; `python-field-theory.json`, checks `superalgebra_axioms` and `C_properties`
the geometry and $\gamma^{\mu}\Omega_{\mu} = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$	7.18	the table at the end of Section 7.18
reality, total divergence, connection absent from $\mathcal{L}$	7.20	`python-field-theory.json`, checks `grassmann_lagrangian_real`, `commuting_lagrangian_real`, `grassmann_total_divergence_relation` and `commuting_total_divergence_relation`; `wolfram-field-theory.json`, checks `L_spin_connection_drops_out_G` and `L_spin_connection_drops_out_C`
the Euler-Lagrange equations and the adjoint equation	7.21	the table at the end of Section 7.21
explicit, block and evolution forms	7.22	`wolfram-field-theory.json`, checks `Dirac_operator_explicit_G`, `Dirac_operator_explicit_C`, `block_form`, `evolution_form_G` and `evolution_form_C`; formula record `Revision/theory/field-theory.json`, keys `field_equation_components` and `field_equation_blocks`
non-triviality, its scope, the exact family; the six senses of self-consistency	7.23	the table at the end of Section 7.23
the Majorana-type negative control; $J = 0$ for real fields	7.28	the table in Section 7.28; lead check `real_fields_charge_conjugation`

result	sections	Revision checks
the charge-conjugation matrices and the sign table	7.29	lead checks `representation_real`, `spinor_connection_real`, `intertwiners_same_mass`, `intertwiners_reversed_mass`, `charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus`, `majorana_conditions_consistent`, `real_fields_charge_conjugation` and `bilinears_under_charge_conjugation`

COMPUTED by the notebooks (each number in the cell named; exact unless an uncertainty is given):

- Notebook 07d: the first variations of the nine varied paths, for example $S_1 = -1/\pi = -0.318310$ for the line (In [2]); $S_2 = 2.217401$, 9.619604, 21.956610 (In [3]); the bump errors -5.228×10^{-3} to -8.168×10^{-5} with ratios 4.0000 (In [7], trapezoidal rule on 200001 points); the grid errors 4.089×10^{-4} ($N = 4$) to 1.008×10^{-7} ($N = 256$) with orders 2.007, 1.978, 2.001, 2.000, 2.000, 2.000 (In [9]); the energy $1/(2\sin^2 1) = 0.706141$ of the true path (In [10]); the grid derivative of the field action equal to $h_1 h_4$ times the grid Euler-Lagrange expression to better than 10^{-9} at 20 inner points (In [14]); the record's growth rate for a momentum along x_5 (In [15]).

- Notebook 07a: 2000 of 2000 random sign comparisons (In [3]); 4096 monomials for $n = 12$ (In [6]); 27 nonzero products of the 64 (In [7]); the surviving monomials $[0, 8, \ldots, 8]$ and $[8, 0, \ldots, 0]$ (In [12]); the counts 16, 120, 560, 1820, 4368, 8008, 11440, 12870, 11440, 8008, 4368, 1820, 560, 120, 16, 1, 0 of S^1 to S^{17} (In [14]).

- Notebook 07b: 25 Christoffel symbols and 12 connection components, equal to the formula record (In [7], In [8]); 392 and 408 monomials of $\mathcal{L}$, equal to the counts written in the record (In [13], In [14]); all 16 component equations equal to the formula record term by term (In [17]); $S = -2.000000$ and -2.828427 for the two exact solutions, constant along x_4, and the full residual below 10^{-12} of the size of Ψ at 201 times (In [22], In [24]).

- Notebook 07c: 136 and 128 monomials of L_g, with 0 and 16 nonzero Euler-Lagrange expressions (In [5]); the 256 products orthogonal with the trace table $16 \cdot 1$, one product commuting with all gammas (degree 0) and one anticommuting with all (degree 8) (In [8]); the real solution of mass $+2$ and its image of mass -2 with relative residuals below 10^{-12} (In [13]).

ASSUMED (used, not derived here):

- the principle of stationary action, the starting principle of classical mechanics and field theory; standard theorems of calculus (differentiation under the integral sign, the fundamental theorem, Taylor's formula, the equality of mixed partial derivatives);

- the definition of the two fields by the Revision record (Revision/SPEC.md, section 3): dirac16complex has complex Grassmann components, dirac16complex00 ordinary complex components; the form of their common Lagrangian (the symmetrised kinetic term, the Dirac adjoint with C, the mass term $-mS$ and the potential $U = \frac{\lambda}{2}S^2$) is a choice of the theory, not a derivation; the method of varying jets (the field and its derivatives at a point treated as independent symbols);

- the author's gammas, C, Γ, B and S^{ab} with their properties (Chapters 4 and 5; read from `Revision/algebra/gammas.json` and checked again in the notebooks), the spin transformations and $R^T C R = C$ (Chapter 5), and the vielbein postulate as the definition of the canonical spin connection (Chapter 6);

- the illustration values ($H = 1$, $m = 2$, χ_0, $z = \pi/4$) and, in Figure 07b.1 only, the linear history $a_4 = AHx_4$ with $A = 1$ and $b_4 = 1$: a prescribed history chosen for the picture, not a result (every equation of the chapter holds for every $a_4(x_4)$; those that hold only for a constant b_4 say so);

- the second function $b_4(x_4) > 0$ of the author's metric of 2026-10-14; the backgrounds of the Revision record have $b_4 = 1$ (`Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md`);

- quoted from the record and derived in later chapters: the general covariant constancy of the gammas (`gamma_covariantly_constant`), the frame and scope statements of the scope reports, the boundary behaviour at $z = \pi/2$ and the growing modes without a boundary condition there (`good_sector_x0_independent_modes_without_boundary_condition`), the quantum statements about conjugation (normal ordering adds no sign, from the detail text of `bilinears_under_charge_conjugation`, and `quantum_charge_conjugation_unitary_type`; Chapters 5, 8, 10 and 21), and the parts (ii) and (iv) of self-consistency (the Heisenberg equation, Chapter 10; the local law of the current, Chapter 21);

- quoted from the record and NOT derived in this book: the Lichnerowicz identity of self-consistency (iii) (`Lichnerowicz_identity_G`), and the Noether identities of self-consistency (v) (`Noether_identity_diffeomorphisms_G`, `Noether_identity_local_Lorentz_G`), which Chapter 9 uses without their proof; the total charge is constant only under the ASSUMED no-flux condition at $z = \pi/2$ (Chapter 21).

HYPOTHESIS. No hypothesis enters this chapter. In particular the chapter does not use the author's hypothesis that the big bang creates universes in pairs; the mass reversal by Γ is an exact map between solutions and says nothing about creation (Chapter 20).

OPEN. The chapter leaves open what the unbounded growth of extra-time modes means for the time evolution of the fields (no well-posed initial-value problem is claimed; Chapter 8), which boundary condition belongs at $z = \pi/2$ (none is imposed here), and, for the matter-antimatter question, everything that Chapter 21 lists as needed beyond this theory.

7.35 Exercises

Exercise 1. For the oscillator with $\omega = 1$, $T = 1$, $q(0) = 0$, $q(1) = 1$, compute the action of the line $q = t$ and of the true path $q = \sin t / \sin 1$. Which is smaller, and why must it be so?

Answer. Line: $\dot q = 1$, $L = \frac12 - \frac12 t^2$, so $S = \int_0^1 (\frac12 - \frac12 t^2)\,dt = \frac12 - \frac16 = \frac13 = 0.333333$. True path: $\dot q = \cos t/\sin 1$, so $L = \frac{\cos^2 t - \sin^2 t}{2\sin^2 1} = \frac{\cos 2t}{2\sin^2 1}$ (the double-angle formula), and $S = \frac{1}{2\sin^2 1}\big[\frac12 \sin 2t\big]_0^1 = \frac{\sin 2}{4\sin^2 1} = \frac{2\sin 1\cos 1}{4\sin^2 1} = \frac12 \cot 1 = 0.321046$. The true path has the smaller action. It must: the line is the true path plus the shape $\xi = t - \sin t/\sin 1$, which vanishes at both ends, so by Section 7.2 $S[\text{line}] = S_0 + S_1 + S_2[\xi]$ with $S_1 = 0$ (the true path is stationary) and $S_2[\xi] = \int_0^1 (\frac12 \dot\xi^2 - \frac12 \xi^2)\,dt$. For the shapes $\sin(n\pi t)$ this coefficient is $(n^2\pi^2 - 1)/4 > 0$ (Section 7.2 with $T = 1$, $\omega = 1$; for $n = 1, 2, 3$ the values 2.217401, 9.619604, 21.956610 of Notebook 07d), and every smooth shape that vanishes at both ends is a combination of these sine shapes (a Fourier sine series; quoted here without proof), whose cross terms integrate to zero; so for $T = 1 < \pi$ the coefficient is positive for every nonzero such shape. Indeed $S[\text{line}] - S[\text{true}] = 0.333333 - 0.321046 = 0.012287 > 0$.

Exercise 2. Compute the first variation of the parabola $q = t^2$ for the shape $\xi_2 = \sin 2\pi t$ in two ways: from the definition $S_1 = \int_0^1 (\dot q\dot\xi - q\xi)\,dt$ and from $S_1 = \int_0^1 E\xi\,dt$ with $E = -2 - t^2$. Compare with Notebook 07d.

Answer. Second way first: $\int_0^1 \sin 2\pi t\,dt = 0$ (a whole period), and $\int_0^1 t^2 \sin 2\pi t\,dt = \big[-\frac{t^2\cos 2\pi t}{2\pi}\big]_0^1 + \frac{1}{\pi}\int_0^1 t\cos 2\pi t\,dt = -\frac{1}{2\pi} + \frac{1}{\pi}\Big(\big[\frac{t\sin 2\pi t}{2\pi}\big]_0^1 - \int_0^1 \frac{\sin 2\pi t}{2\pi}dt\Big) = -\frac{1}{2\pi} + 0$ (integration by parts twice; $\cos 2\pi = 1$, $\sin 2\pi = 0$). So $S_1 = -2\cdot 0 - (-\frac{1}{2\pi}) = \frac{1}{2\pi}$. First way: $\dot q = 2t$, $\dot\xi = 2\pi\cos 2\pi t$, so $S_1 = \int_0^1 4\pi t\cos 2\pi t\,dt - \int_0^1 t^2\sin 2\pi t\,dt = 4\pi\cdot 0 + \frac{1}{2\pi} = \frac{1}{2\pi}$, the same. Notebook 07d prints `parabola n = 2:  S1 = 1/(2*pi) = +0.159155` (In [2]).

Exercise 3. Let $F = 1 + 2\theta_0 + 3\theta_1\theta_2$ and $G = \theta_0 - \theta_1$. Compute FG and GF, and explain the difference with the rule $YX = (-1)^{kl}XY$.

Answer. $FG = \theta_0 - \theta_1 + 2\theta_0\theta_0 - 2\theta_0\theta_1 + 3\theta_1\theta_2\theta_0 - 3\theta_1\theta_2\theta_1$. Here $\theta_0\theta_0 = 0$; $\theta_1\theta_2\theta_1 = -\theta_1\theta_1\theta_2 = 0$; and $\theta_1\theta_2\theta_0 = +\theta_0\theta_1\theta_2$ (two exchanges). So $FG = \theta_0 - \theta_1 - 2\theta_0\theta_1 + 3\theta_0\theta_1\theta_2$. $GF = \theta_0 + 2\theta_0\theta_0 + 3\theta_0\theta_1\theta_2 - \theta_1 - 2\theta_1\theta_0 - 3\theta_1\theta_1\theta_2 = \theta_0 - \theta_1 + 2\theta_0\theta_1 + 3\theta_0\theta_1\theta_2$. The difference $FG - GF = -4\theta_0\theta_1$ comes only from the odd part $2\theta_0$ of F times the odd G: two odd elements anticommute ($k = l = 1$), while the even part $1 + 3\theta_1\theta_2$ commutes with everything ($k = 0$ or 2).

Exercise 4. For the odd monomial $F = \theta_0\theta_1\theta_2$ compute $\partial_L F/\partial\theta_1$ and $\partial_R F/\partial\theta_1$. Show that for every odd element the two derivatives are equal.

Answer. θ_1 stands at position $p = 1$ of a monomial of degree $n = 3$. Moving it left passes one generator: $\partial_L F/\partial\theta_1 = -\theta_0\theta_2$. Moving it right passes $n - 1 - p = 1$ generator: $\partial_R F/\partial\theta_1 = -\theta_0\theta_2$. In general the two signs differ by $(-1)^{(n-1-p)-p} = (-1)^{n-1}$; for odd n the exponent $n - 1$ is even, so the two derivatives of every odd monomial, and hence of every odd element, are equal (for even elements they are opposite, Section 7.11).

Exercise 5. A field with two complex Grassmann components has $S = \chi_0\psi_0 - \chi_1\psi_1$ (the matrix $C = \mathrm{diag}(1, -1)$). Compute S^2, S^3 and the "exponential" $1 + S + \frac12 S^2 + \frac16 S^3 + \dots$. Check the counts $\binom{2}{k}$ and the coefficients $\pm k!$ of Section 7.12.

Answer. The pairs $P_0 = \chi_0\psi_0$ and $P_1 = \chi_1\psi_1$ are even, commute and square to zero. $S^2 = P_0P_0 - P_0P_1 - P_1P_0 + P_1P_1 = -2P_0P_1 = -2\chi_0\psi_0\chi_1\psi_1$: one monomial ($\binom{2}{2} = 1$) with coefficient of size $2 = 2!$. $S^3 = S\cdot S^2 = -2(P_0 - P_1)P_0P_1 = 0$, because every term repeats a pair. So the series stops: $e^S = 1 + \chi_0\psi_0 - \chi_1\psi_1 - \chi_0\psi_0\chi_1\psi_1$, an exact finite sum. S itself has $\binom{2}{1} = 2$ monomials with coefficients $\pm 1 = \pm 1!$.

Exercise 6. For two real components and $M = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, compute $\Theta^T M\Theta$ for a real Grassmann

column $\Theta = (\theta_0, \theta_1)^T$ and $q^T M q$ for a real column of ordinary numbers. Which part of M does each keep?

Answer. $\Theta^T M \Theta = \theta_0\theta_0 + 2\theta_0\theta_1 + 3\theta_1\theta_0 + 4\theta_1\theta_1 = 2\theta_0\theta_1 - 3\theta_0\theta_1 = -\theta_0\theta_1$. The antisymmetric part of M is $\frac{1}{2}(M - M^T) = \begin{pmatrix} 0 & -\frac{1}{2} \\ \frac{1}{2} & 0 \end{pmatrix}$, which gives $-\frac{1}{2}\theta_0\theta_1 + \frac{1}{2}\theta_1\theta_0 = -\theta_0\theta_1$, the same: only the antisymmetric part survives. For ordinary numbers $q^T M q = q_0^2 + 2q_0q_1 + 3q_1q_0 + 4q_1^2 = q_0^2 + 5q_0q_1 + 4q_1^2$, which is the form of the symmetric part $\begin{pmatrix} 1 & \frac{5}{2} \\ \frac{5}{2} & 4 \end{pmatrix}$: only the symmetric part survives.

Exercise 7. Take $b_4 = 1$ and suppose the extra times INFLATED like 3-space, with the scale factor $e^{a_4} \sin^{1/6} z$ (that is $g_{55} = -e^{2a_4} \sin^{1/3} z$). Compute $\gamma^{x_5}\Omega_{x_5}$ and $\gamma^\mu \Omega_\mu$. What does this show about the author's metric?

Answer. Now $f_5 = Es$ and $g_{55} = -E^2 s^2$, so $\partial_4 g_{55} = -2E^2 s^2 A_1$. The Christoffel symbol and the connection component are

$$\Gamma^{x_4}{}_{x_5 x_5} = -\frac{\partial_4 g_{55}}{2g_{44}} = -\frac{-2E^2 s^2 A_1}{-2} = -E^2 s^2 A_1,$$
$$\omega_{x_5 (x_4)(x_5)} = \frac{\eta_{44} f_4 \, \Gamma^{x_4}{}_{x_5 x_5}}{f_5} = \frac{E^2 s^2 A_1}{Es} = Es A_1,$$

so the a_4' part of Ω_{x_5} is $Es A_1 S^{(x_4)(x_5)} = \frac{1}{2} Es A_1 \gamma^{(x_4)}\gamma^{(x_5)}$, and the a_4' part of $\gamma^{x_5}\Omega_{x_5}$ is

$$\frac{\gamma^{(x_5)}}{Es} \cdot \tfrac{1}{2} Es A_1 \gamma^{(x_4)}\gamma^{(x_5)} = \tfrac{1}{2} A_1 \gamma^{(x_5)}\gamma^{(x_4)}\gamma^{(x_5)} = \tfrac{1}{2} A_1 \gamma^{(x_4)}$$

(as in Section 7.18, $\gamma^{(x_5)}\gamma^{(x_4)}\gamma^{(x_5)} = \gamma^{(x_4)}$). The H part is unchanged, $\frac{1}{2} H \gamma^{(x_0)}$, because the warp $\sin^{1/6} z$ is the same. So each of the six directions now contributes $+\frac{1}{2} a_4' \gamma^{(x_4)}$, and $\gamma^\mu \Omega_\mu = 3a_4' \gamma^{(x_4)} + 3H\gamma^{(x_0)}$. The cancellation of the time terms in the author's metric is due to the DEFLATION of the extra times. This is the negative control of the lead's independent check `negative_control_inflating_extra_times` in the report `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`.

Exercise 8. In flat space with $m = 2$, a plane wave of the 16-component field has the frame momenta $k_0 = 1$ along the hidden direction and $k_7 = 3$ along an extra time, all others zero. Does it oscillate or grow, and at what rate?

Answer. By the Clifford square of Section 7.5, $\omega^2 = m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2 = 4 + 1 - 9 = -4 < 0$: it grows, with the rate $\sqrt{4} = 2$. This agrees with the record's formula $\sqrt{K^2 - k_1^2 - k_2^2 - k_3^2 - k_0^2 - m^2} = \sqrt{9 - 1 - 4} = 2$ with the extra-time momentum $K = 3$ (the record writes it for x_5; the three extra times enter the formula in the same way).

Exercise 9. The reality condition with the matrix Γ is $\Psi = \Gamma \Psi^*$. What does it say about the 16 components? Is it consistent?

Answer. $\Gamma = \mathrm{diag}(-I_8, I_8)$, so the condition reads $\Psi_A = -\Psi_A^*$ for $A = 1, \ldots, 8$ and $\Psi_A = \Psi_A^*$ for $A = 9, \ldots, 16$: the upper eight components are purely imaginary and the lower eight real. It is consistent: applying it twice gives $\Psi = \Gamma(\Gamma\Psi^*)^* = \Gamma\Gamma\Psi = \Psi$, because Γ is real and $\Gamma^2 = 1$ (the condition $MM^* = 1$ of Section 7.29).

Exercise 10. Show that for $U = \frac{\lambda}{2}S^2$ the map Γ does NOT relate $\mathcal{L}_{m,\lambda}$ to $-\mathcal{L}_{-m,\lambda}$ when $\lambda \neq 0$: compute $\mathcal{L}_{m,\lambda}[\Gamma\Phi] + \mathcal{L}_{-m,\lambda}[\Phi]$ for a real commuting field.

Answer. By Section 7.29, $K[\Gamma\Phi] = -K[\Phi]$ and $S[\Gamma\Phi] = S[\Phi]$, so $\mathcal{L}_{m,\lambda}[\Gamma\Phi] = \sqrt{|g|}\,(-K - mS - \frac{\lambda}{2}S^2)$, while $\mathcal{L}_{-m,\lambda}[\Phi] = \sqrt{|g|}\,(K + mS - \frac{\lambda}{2}S^2)$. The sum is $-\lambda\sqrt{|g|}\,S^2$, which is not zero when $\lambda \neq 0$ and $S \neq 0$. The potential does not change sign under Γ, so the coupling must be reversed together with the mass: $(m, \lambda) \to (-m, -\lambda)$, as in Section 7.29.

8 Non-triviality, its exact scope, and well-posedness

When the author coupled the fields dirac16complex and dirac16complex00 to his primordial gravitational field, he asked for a test: the field equations must contain gravity through the canonical spin connection, unless spacetime is flat. This chapter answers that request exactly, and it answers a second question that every field equation must face before anyone can trust its solutions: does the equation decide the future from the present in a stable way? Both answers are more interesting than a plain yes. The gravitational term of the field equation is $3H\gamma^{(0)}\Psi$ in the natural frame on the backgrounds $b_4 = 1$ of the Revision record ($(3H\gamma^{(0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(4)})\Psi$ for the general metric with b_4), but a different, equally valid frame makes it vanish, and a change of the field variables removes it too; what no choice removes is the spin connection itself, because its curvature is the curvature of spacetime. And the field equation, with its four time-like directions, is not a well-posed evolution equation: waves along the three extra times grow, with growth rates that have no upper bound, and the deflation of the extra times pushes every such wave into growth. (This is proved exactly in flat 4+4 space and, for the author's metric, with coefficients frozen at one point; carrying it over to the author's varying coefficients needs a standard theorem that this chapter quotes without proof. Section 8.18 gives each status.) Four notebooks compute every statement anew from the definitions and reproduce the Revision record: Notebook 08c (the term in two frames), Notebook 08a (the growth rates), Notebook 08b (the onset of growth along the deflating history) and Notebook 08d (the rescaling and the two ends of the hidden direction).

8.1 What this chapter does

Why we need this chapter. The author's task, quoted in the file `Revision/README.md` of the repository, says of the two Lagrangians: "[1]- the Lagrangian for dirac16complex must be non-trivial (i.e., the Euler-Lagrange equations for dirac16complex always possess non-zero contributions from gravity (through the canonical spin-connection, unless we are in flat 4+4 spacetime), must be self consistent, must be checked and verified)", and "[2]" asks the same of dirac16complex00. Two questions hide in this request. First: what exactly is "the contribution from gravity through the canonical spin connection", and which part of it is a property of the field equation itself rather than of the way we chose to write the equation down? Second: a field equation is useful only if, given the field now, it determines the field later, and determines it stably, so that a tiny change now makes only a tiny change later. Is that true here? This chapter answers both questions exactly, and says precisely where each answer stops.

The answers in brief. Each item carries its label (the labels are explained at the end of this section).

- In the diagonal frame of the author's metric the gravitational term of the field equation is $\gamma^\mu \Omega_\mu \Psi = 3H\gamma^{(0)}\Psi$, for every history $a_4(x_4)$ and a constant b_4, as on the backgrounds $b_4 = 1$ (for a changing b_4 the term $\frac{3}{4}(b_4'/b_4)\gamma^{(4)}\Psi$ is added; Chapters 6 and 7). The time-direction pieces of the three inflating and the three deflating directions cancel; the six hidden-direction pieces add up to $3H$. The term is not zero for any field $\Psi \neq 0$ (PROVED; Sections 8.4 to 8.6; for a changing b_4 except at times where $|b_4'/b_4| = 4H$, Section 7.23).

- The spin connection vanishes only in the formal flat limit $a_4' = 0$ and $H = 0$ (at $b_4 = 1$; for every b_4 it vanishes only for $H = 0$), which is not a member of the author's family of metrics (PROVED; Section 8.6).

- The value $3H\gamma^{(0)}$ belongs to one frame and to one choice of field variables. In a frame boosted in the (x_4, x_0) plane with the rapidity $6Hx_4$ the term is identically zero (for a constant b_4), and the rescaling $\Psi = \sin^{-1/2} z\, \chi$ ($\sin^{-1/2} z\, b_4^{-3/4}\chi$ for a changing b_4) removes it in the diagonal frame (PROVED; Sections 8.9 and 8.30; Sections 6.16 and 6.18).

- What no choice removes: the spin connection itself vanishes in no frame, because its curvature is the curvature of spacetime, which is not zero for $H > 0$; and the frame factors $e^{-a_4}\sin^{-1/6} z$, $e^{a_4} b_4^{-1/2}\sin^{-1/6} z$ and $\tan z$ multiply every derivative in the field equation except the one along the time x_4 (whose factor is $1/f_4 = 1$). This is the exact, frame-independent content of the tests [1] and [2] (PROVED; Sections 8.7 and 8.10). The Revision record states this as a correction for the author: the tests hold in this qualified sense.

- The field equation can be solved for the time derivative $\partial_4\Psi$, but its initial-value problem is NOT well posed for data that depend on the extra times x_5, x_6, x_7: such waves grow, and their growth rates have no upper bound (PROVED in flat 4+4 space and, for the author's metric, with frozen coefficients, a MODEL; for the author's varying coefficients it follows from the Lax-Mizohata theorem, a standard theorem quoted without proof, so that step is ASSUMED; Sections 8.16 to 8.19).

- Along the deflating history every wave along an extra time eventually starts to grow (PROVED with frozen coefficients; followed numerically through the onset in a labelled model, COMPUTED; Sections 8.24 and 8.25).

- Even without extra-time dependence, the hidden direction has an open end at $z = \pi/2$; there the Revision record imposes no boundary condition, and fields that do not depend on x_0 grow when $m < 3H$, fed through that end. Which boundary condition is physical is OPEN (Sections 8.31 and 8.32).

What the chapter does, section by section. Section 8.2 sets the stage. Sections 8.3 to 8.11 compute the spin connection, the gravitational term, its frame dependence and the curvature, with Notebook 08c as the worked example (Sections 8.12 to 8.15). Sections 8.16 to 8.19 define well-posedness and show, with plane waves, that it fails; Notebook 08a is the example (Sections 8.20 to 8.23). Sections 8.24 and 8.25 follow waves along the deflating history; Notebook 08b is the example (Sections 8.26 to 8.29). Sections 8.30 to 8.32 change the field variables and study the two ends of the hidden direction; Notebook 08d is the example (Sections 8.33 to 8.36). Section 8.37 collects the exact scope in one table, Section 8.38 lists what we proved, computed and assumed, and Section 8.39 has the exercises.

The four notebooks.

notebook	what it computes	checks and figures	Revision records it reproduces
08c	the Christoffel symbols and the canonical spin connection of the diagonal frame and of a boosted frame, from the definitions; what cancels and what survives in $\gamma^{\mu}\Omega_{\mu}$; the connection vanishes only in flat space; the term in two frames; the curvature in both frames; the connection in the energy-momentum tensor	32 checks, 5 figures	`python-field-theory.json`, `wolfram-field-theory.json`, `python-scope.json` (all in `Revision/theory/reports/`), `Revision/kohn_sham/results/parameters.json`
08a	the mode matrix of a plane wave and its square; the growth rates of the extra-time waves; the Hadamard amplification; the Krein form	30 checks, 6 figures	`python-scope.json`, `wolfram-scope.json`, `python-field-theory.json`, `wolfram-field-theory.json`, `Revision/algebra/reports/python-algebra.json`, `Revision/pairing/reports/python-pairing.json`
08b	the scale factors along the deflating history; the frame momenta of a wave and its onset time; one wave followed through the onset with RK4 and compared with WKB formulas	25 checks, 5 figures	`parameters.json`, `python-field-theory.json`, `python-scope.json`
08d	the rescaling that removes the term; an exact family of solutions; the growing modes that do not depend on x_0; the boundary term at the patch end	23 checks, 5 figures	`python-field-theory.json`, `Revision/theory/field-theory.json`, `python-scope.json`, `parameters.json`

The chapter places the notebooks in the order in which it uses them: 08c, 08a, 08b, 08d.

How this chapter relates to Chapter 6. Chapter 6 introduces the vielbein, the canonical spin

connection, the contraction $\gamma^\mu \Omega_\mu = 3H\gamma^{(0)}$, and, with its Notebook 06b, a first look at the boosted frame, the curvature and the rescaling; it also proves local spin covariance in general. This chapter recomputes everything it needs, so that it can be read on its own, and goes further: what cancels and what survives direction by direction, the proof that the connection vanishes only in the flat limit, the boosted term derived from the uniqueness of the connection, the curvature compared in two frames, the exact family with its growing finite-norm members, the flux through the patch end, and, above all, the question of well-posedness, which Chapter 6 does not treat.

The metric with b_4. The author's metric of 2026-10-14 has a second function $b_4(x_4) > 0$ in the extra-time entries (Chapter 3; `Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md`). The Revision record's scope reports and the backgrounds of its numerical work have $b_4 = 1$, and the four notebooks of this chapter compare with those reports, so their exact checks are made at $b_4 = 1$; Notebooks 06a and 06b and Chapter 7 treat a general b_4. Where a statement of this chapter changes for a changing b_4, the change is stated with it.

Labels. As everywhere in the book, every statement carries one of five labels. **PROVED** means exact: derived here line by line and confirmed by an exact check of a notebook, and, where the Revision record proves the same, the record file and its check are named. **COMPUTED** means a number obtained numerically by a notebook, with its measured accuracy. **ASSUMED** marks the inputs we choose or quote: standard theorems quoted without proof (each is named where it is used), the deflating history $a_4 = AHx_4$, which the Revision record calls a PRESCRIBED BACKGROUND, and two approximations that the notebooks label MODEL (frozen coefficients, and the local-frame model of Section 8.25). **OPEN** marks questions this record does not answer. No HYPOTHESIS enters this chapter. Nothing in this chapter concerns pairs of universes, their creation, or matter and antimatter; those questions belong to Chapters 18 to 21, which state precisely what is and what is not proved about them.

8.2 The stage: coordinates, metric, frame and the field equation

Coordinates. A point of the author's spacetime has eight coordinates, named as the author names them: x_1, x_2, x_3 are ordinary 3-space; x_4 is the time in which everything evolves; x_5, x_6, x_7 are the three **extra times**, directions that behave like time (they are time-like) and that deflate exponentially; x_0 is the **hidden** space direction. We write $z = 6Hx_0$, where $H > 0$ is a constant of the author; z runs from 0 (the **tip**) to $\pi/2$ (the **patch end**). In formulas the letter i always stands for one of the 3-space directions x_1, x_2, x_3 and the letter t for one of the extra times x_5, x_6, x_7. In Python, lists are counted from 0, like the author's coordinates: position k holds x_k, so position 0 is the hidden coordinate x_0 and position 4 the time x_4.

The metric. The author's metric is diagonal (Chapter 3). We write it with eight positive **scale factors** $f_0, \dots, f_7$:

$$g = \mathrm{diag}(f_0^2,\ f_1^2,\ f_2^2,\ f_3^2,\ -f_4^2,\ -f_5^2,\ -f_6^2,\ -f_7^2),$$

$$f_0 = \cot z, \quad f_1 = f_2 = f_3 = e^{a_4} \sin^{1/6} z, \quad f_4 = 1, \quad f_5 = f_6 = f_7 = e^{-a_4} b_4^{1/2} \sin^{1/6} z,$$

with $b_4 = 1$ on the backgrounds of the record (and in the checks of this chapter's notebooks).

A small step dx_μ along the direction x_μ has the proper length (the length measured with the metric) $f_\mu|dx_\mu|$. The function $a_4(x_4)$, the **metric function**, increases with the time: then the 3-space factor e^{a_4} grows (3-space **inflates**) and the extra-time factor e^{-a_4} shrinks (the extra times **deflate**). We write $a_4' = da_4/dx_4$ and $a_4'' = d^2a_4/dx_4^2$. With the **frame metric** $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ (in the order $x_0,\dots,x_7$) every diagonal entry is $g_{\mu\mu} = \eta_{\mu\mu}f_\mu^2$. The signs say which directions are space-like (+: x_0,x_1,x_2,x_3) and which are time-like (−: x_4,x_5,x_6,x_7); there are four of each, the signature (4,4). The product of the eight factors is the volume factor, $\sqrt{|g|} = f_0f_1\cdots f_7 = b_4^{3/2}\cos z$, which is $\cos z$ at $b_4 = 1$; it does not depend on the time there, because the inflation e^{3a_4} of 3-space and the deflation e^{-3a_4} of the extra times cancel (PROVED; `Revision/theory/reports/python-field-theory.json`, check `sqrt_det_g_equals_cos_z`; for b_4, `wolfram-field-theory-b4.json`, check `sqrt_det_g`). The value $H = 0$ is not a member of the author's family: at fixed x_0 the factor $\sin^{1/3} z$ goes to 0 and $\cot^2 z$ to infinity, so the metric degenerates (`Revision/theory/reports/wolfram-field-theory.json`, check `degenerate_at_H_0`). Every statement of this chapter assumes $H > 0$ and $0 < z < \pi/2$.

The diagonal frame. A **frame** (also called a **vielbein**) is a set of eight vectors, one for each direction $a = 0,\dots,7$, that have length 1 and are perpendicular to each other when measured with the metric. Its components ${e^a}_\mu$ (frame direction a, coordinate direction μ) obey $\sum_a \eta_{aa}{e^a}_\mu{e^a}_\nu = g_{\mu\nu}$. The **diagonal frame** is ${e^a}_\mu = f_a\delta^a_\mu$, where δ^a_μ is 1 for $a = \mu$ and 0 otherwise; its inverse is ${E_a}^\mu = \delta^\mu_a/f_a$. Indeed $\sum_a \eta_{aa}f_a\delta^a_\mu f_a\delta^a_\nu$ is zero for $\mu \neq \nu$ and $\eta_{\mu\mu}f_\mu^2 = g_{\mu\mu}$ for $\mu = \nu$.

The gamma matrices. The book uses the author's eight **real** 16×16 gamma matrices (Chapter 4), stored in the Revision file `Revision/algebra/gammas.json` in the order $x_0,\dots,x_7$; we write them $\gamma^{(0)},\dots,\gamma^{(7)}$ ($\gamma^{(4)}$ means the gamma of the direction x_4, and so on). Their entries are 0, 1 and −1. They obey the **Clifford relations**

$$\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}\,I_{16},$$

where I_{16} is the 16×16 unit matrix and $\eta^{ab} = \eta_{ab}$ (PROVED; `Revision/algebra/reports/python-algebra.json`, check `clifford_relation`). Two consequences are used in almost every line of this chapter: the square of each gamma is $\eta_{aa}I_{16}$, that is $+I_{16}$ for x_0,x_1,x_2,x_3 and $-I_{16}$ for x_4,x_5,x_6,x_7; and two different gammas **anticommute**, $\gamma^{(a)}\gamma^{(b)} = -\gamma^{(b)}\gamma^{(a)}$ for $a \neq b$. A third fact: the gammas of the space-like directions are symmetric matrices and those of the time-like directions antisymmetric, $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$ (the T means the transpose: rows become columns). The **coordinate gammas** are $\gamma^\mu = \sum_a {E_a}^\mu\gamma^{(a)}$; in the diagonal frame $\gamma^\mu = \gamma^{(\mu)}/f_\mu$. The matrix $C = \gamma^{(0)}\gamma^{(1)}\gamma^{(2)}\gamma^{(3)}$ (Chapter 5) and the matrix $B = -iC\gamma^{(4)}$ (Chapters 5 and 10) appear in the notebooks; B is Hermitian, $B^2 = I_{16}$, with eight eigenvalues +1 and eight −1.

The two fields and their field equation. dirac16complex is a column Ψ of 16 complex **anticommuting** (Grassmann) components; dirac16complex00 is a column of 16 ordinary complex numbers (Chapter 7). Their Lagrangians have the same form, and so do their field equations (Chapter 7 derives them):

$$\gamma^\mu D_\mu\Psi = \big(m + U'(S)\big)\Psi, \qquad D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi.$$

Here ∂_μ is the partial derivative with respect to x_μ, the sum over $\mu = 0,\dots,7$ is understood (a repeated upper and lower index is summed), m is the mass, $S = \bar\Psi\Psi = \Psi^\dagger C\Psi$ is the scalar built

from the field and its **Dirac adjoint** $\bar\Psi = \Psi^\dagger C$ (the dagger means: transpose and take the complex conjugate of every entry), $U(S) = \frac{\lambda}{2}S^2$ is the potential, and $U'(S) = \lambda S$. The matrices Ω_μ are the **spinor connection**, built from the canonical spin connection in Section 8.4. Everything in this chapter is a statement about this matrix equation; it holds for both fields (for the Grassmann field, "$\Psi \neq 0$" means a configuration that is not identically zero). Written out in the diagonal frame at $b_4 = 1$ the equation reads (`Revision/theory/field-theory.json`, formula key `field_equation`; Section 8.5 derives the last term on the left; for a general b_4 the extra-time factor is $e^{a_4} b_4^{-1/2} \sin^{-1/6} z$ and the term $\frac{3}{4}(b_4'/b_4)\gamma^{(4)}\Psi$ is added on the left, `Revision/theory/field-theory-b4.json`, key `field_equation`):

$$
\begin{aligned}
&e^{-a_4} \sin^{-1/6} z \,\big(\gamma^{(1)}\partial_1 + \gamma^{(2)}\partial_2 + \gamma^{(3)}\partial_3\big)\Psi + \gamma^{(4)}\partial_4\Psi \\
&\quad + e^{a_4} \sin^{-1/6} z \,\big(\gamma^{(5)}\partial_5 + \gamma^{(6)}\partial_6 + \gamma^{(7)}\partial_7\big)\Psi + \tan z\, \gamma^{(0)}\partial_0\Psi + 3H\gamma^{(0)}\Psi = \big(m + U'(S)\big)\Psi .
\end{aligned}
$$

The derivative terms are $\sum_\mu \gamma^\mu \partial_\mu \Psi = \sum_\mu (\gamma^{(\mu)}/f_\mu)\partial_\mu\Psi$, with $1/f_0 = \tan z$, $1/f_i = e^{-a_4}\sin^{-1/6} z$, $1/f_4 = 1$ and $1/f_t = e^{a_4} b_4^{-1/2}\sin^{-1/6} z$ ($e^{a_4}\sin^{-1/6} z$ at $b_4 = 1$). These factors are the **frame factors**: they carry the metric into every derivative term.

Units. H, m and every momentum are measured in one unit, an inverse length, and the coordinates in its inverse; every printed number is a pure number in this unit. The examples use $H = m = 1$, the values of the Kohn-Sham record.

8.3 Frames, and why a frame is not unique

Why spinors need a frame. The gamma matrices are attached to the eight orthonormal frame directions, not to the coordinates: the Clifford relations $\{\gamma^{(a)}, \gamma^{(b)}\} = 2\eta^{ab}$ involve the fixed numbers η^{ab}, while the metric $g_{\mu\nu}$ changes from point to point. (The curly bracket $\{A, B\} = AB + BA$ is the **anticommutator**.) To write a spinor field equation in curved space one therefore needs a frame at every point, and the frame vectors turn from point to point; the canonical spin connection measures how they turn (Section 8.4).

A frame is not unique. At every point one may replace the eight frame vectors by combinations of them, $e'^a{}_\mu = \sum_b \Lambda^a{}_b e^b{}_\mu$, with a matrix Λ that keeps the frame metric: $\sum_a \eta_{aa}\Lambda^a{}_b\Lambda^a{}_c = \eta_{bc}$. The new frame describes the same metric. Line by line:

$$
\begin{aligned}
\sum_a \eta_{aa} e'^a{}_\mu e'^a{}_\nu &= \sum_a \eta_{aa}\Big(\sum_b \Lambda^a{}_b e^b{}_\mu\Big)\Big(\sum_c \Lambda^a{}_c e^c{}_\nu\Big) \\
&= \sum_{b,c}\Big(\sum_a \eta_{aa}\Lambda^a{}_b\Lambda^a{}_c\Big) e^b{}_\mu e^c{}_\nu \\
&= \sum_{b,c} \eta_{bc}\, e^b{}_\mu e^c{}_\nu = g_{\mu\nu}.
\end{aligned}
$$

The first line inserts the new frame; the second reorders the finite sums; the third uses the condition on Λ and then the frame condition of the old frame.

The boost in the (x_4, x_0) plane. The **hyperbolic functions** are $\cosh b = (e^b + e^{-b})/2$ and $\sinh b = (e^b - e^{-b})/2$. They obey

$$\cosh^2 b - \sinh^2 b = \frac{(e^b + e^{-b})^2 - (e^b - e^{-b})^2}{4} = \frac{4e^b e^{-b}}{4} = 1,$$

where the middle step expands both squares ($e^{2b} + 2 + e^{-2b}$ minus $e^{2b} - 2 + e^{-2b}$ leaves 4). The **boost** with **rapidity** b mixes the time-like frame vector of x_4 and the space-like frame vector of x_0:

$$e'^{(4)} = \cosh b\, e^{(4)} + \sinh b\, e^{(0)}, \qquad e'^{(0)} = \sinh b\, e^{(4)} + \cosh b\, e^{(0)},$$

and keeps the other six. It keeps the frame metric: the two mixed vectors enter the metric as $-e'^{(4)}e'^{(4)} + e'^{(0)}e'^{(0)}$ (products of components at the same μ and ν), and

$$\begin{aligned} -e'^{(4)}e'^{(4)} + e'^{(0)}e'^{(0)} &= -\big(\cosh^2 b\, e^{(4)}e^{(4)} + 2\cosh b \sinh b\, e^{(4)}e^{(0)} + \sinh^2 b\, e^{(0)}e^{(0)}\big) \\ &\quad + \big(\sinh^2 b\, e^{(4)}e^{(4)} + 2\sinh b \cosh b\, e^{(4)}e^{(0)} + \cosh^2 b\, e^{(0)}e^{(0)}\big) \\ &= -(\cosh^2 b - \sinh^2 b)\, e^{(4)}e^{(4)} + (\cosh^2 b - \sinh^2 b)\, e^{(0)}e^{(0)} \\ &= -e^{(4)}e^{(4)} + e^{(0)}e^{(0)}. \end{aligned}$$

The first two lines multiply out the two squares (here $e^{(4)}e^{(0)}$ stands for the symmetric combination of the two orders), the third collects the terms (the mixed terms cancel), and the fourth uses $\cosh^2 b - \sinh^2 b = 1$. The rapidity may change from point to point; we take $b = \beta x_4 + b_0$ with two constants β (the **rapidity rate**) and b_0, a different boost at every time. Every such frame describes the author's metric exactly (PROVED; `Revision/theory/reports/python-scope.json`, check `boosted_frame_reproduces_metric`, and the same check of `wolfram-scope.json`).

The principle. A change of frame changes the spin connection and the coordinate gammas, and the field changes with it, $\Psi' = S\Psi$ with an invertible 16×16 matrix $S(x)$ (Section 8.9 finds S for the boost). A statement about the field equation is a property of the equation itself only if it holds in every frame. A statement that holds in one frame only describes how we chose to write the equation.

8.4 The Christoffel symbols and the canonical spin connection of the diagonal frame

Christoffel symbols. The **Christoffel symbols** ${\Gamma^\lambda}_{\mu\nu}$ say how the coordinate directions turn from point to point (Chapter 3). In this section and the next five the metric is taken at $b_4 = 1$, as in the record's scope reports and in Notebook 08c; Sections 7.18 and 6.3 to 6.8 give the same objects for a general b_4, where the extra-time rate $-a'_4$ becomes $-a'_4 + b'_4/(2b_4)$ and the frame factor of the extra times gets the factor $b_4^{1/2}$. For a diagonal metric they are

$${\Gamma^\lambda}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu}),$$

and they are symmetric in the two lower indices. The author's metric depends only on x_4 (through a_4) and on x_0 (through z), and $dz/dx_0 = 6H$. It has 25 independent nonzero symbols (PROVED; `Revision/theory/reports/python-field-theory.json`, check `christoffel_symmetric_metric_compatible`):

symbol	value	symbol	value
${\Gamma^{x_i}}_{x_i x_4}$	a_4'	${\Gamma^{x_i}}_{x_i x_0}$	$H\cot z$
${\Gamma^{x_t}}_{x_t x_4}$	$-a_4'$	${\Gamma^{x_t}}_{x_t x_0}$	$H\cot z$
${\Gamma^{x_4}}_{x_i x_i}$	$e^{2a_4}\sin^{1/3} z\, a_4'$	${\Gamma^{x_4}}_{x_t x_t}$	$e^{-2a_4}\sin^{1/3} z\, a_4'$
${\Gamma^{x_0}}_{x_i x_i}$	$-e^{2a_4}H\sin^{4/3} z/\cos z$	${\Gamma^{x_0}}_{x_t x_t}$	$e^{-2a_4}H\sin^{4/3} z/\cos z$
${\Gamma^{x_0}}_{x_0 x_0}$	$-6H/(\sin z\cos z)$		

(each of the eight entries of the first four rows stands for three symbols, one for each value of i or of t: 24 symbols, plus the last one: 25). Chapter 3 derives all of them from the metric, and Notebook 08c computes all of them again; here we repeat the three that this chapter needs most. First, ${\Gamma^{x_1}}_{x_1 x_4}$: only the term $\partial_4 g_{11}$ survives, and $g_{11} = e^{2a_4}\sin^{1/3} z$ has $\partial_4 g_{11} = 2a_4' g_{11}$ (chain rule), so ${\Gamma^{x_1}}_{x_1 x_4} = 2a_4' g_{11}/(2g_{11}) = a_4'$. Second, ${\Gamma^{x_1}}_{x_1 x_0} = \partial_0 g_{11}/(2g_{11})$, and $\partial_0 \sin^{1/3} z = \frac{1}{3}\sin^{-2/3} z\cos z \cdot 6H$, so ${\Gamma^{x_1}}_{x_1 x_0} = \frac{1}{2}\cdot\frac{1}{3}\cdot\frac{\cos z}{\sin z}\cdot 6H = H\cot z$. Third, ${\Gamma^{x_5}}_{x_5 x_4} = \partial_4 g_{55}/(2g_{55})$ with $g_{55} = -e^{-2a_4}\sin^{1/3} z$, $\partial_4 g_{55} = -2a_4' g_{55}$, so ${\Gamma^{x_5}}_{x_5 x_4} = -a_4'$: the sign is opposite to that of 3-space because the extra times deflate. A useful rule follows the same way: for a diagonal metric, ${\Gamma^{\mu}}_{\mu\nu} = \partial_\nu g_{\mu\mu}/(2g_{\mu\mu}) = \partial_\nu \ln f_\mu$ (no sum over μ), because $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ gives $\partial_\nu g_{\mu\mu}/(2g_{\mu\mu}) = 2\eta_{\mu\mu} f_\mu \partial_\nu f_\mu/(2\eta_{\mu\mu} f_\mu^2) = \partial_\nu f_\mu/f_\mu$.

The canonical spin connection. The **canonical spin connection** of a frame is (Chapter 6)

$$\omega_\mu{}^a{}_b = \sum_\nu e^a{}_\nu \Big(\partial_\mu E_b{}^\nu + \sum_\lambda {\Gamma^\nu}_{\mu\lambda} E_b{}^\lambda\Big), \qquad \omega_{\mu ab} = \eta_{aa}\,\omega_\mu{}^a{}_b.$$

It obeys the **vielbein postulate** $\partial_\mu e^a{}_\nu - \sum_\lambda {\Gamma^\lambda}_{\mu\nu} e^a{}_\lambda + \sum_b \omega_\mu{}^a{}_b e^b{}_\nu = 0$, and $\omega_{\mu ab}$ is antisymmetric in a, b (PROVED for the diagonal frame; `python-field-theory.json`, checks `vielbein_postulate` and `spin_connection_antisymmetric`). For the diagonal frame we compute it line by line:

$$\begin{aligned}\omega_\mu{}^a{}_b &= \sum_\nu f_a \delta^a_\nu \Big(\partial_\mu \frac{\delta^\nu_b}{f_b} + \sum_\lambda {\Gamma^\nu}_{\mu\lambda}\frac{\delta^\lambda_b}{f_b}\Big)\\ &= f_a\Big(\partial_\mu \frac{\delta^a_b}{f_b} + \frac{{\Gamma^a}_{\mu b}}{f_b}\Big).\end{aligned}$$

The first line inserts $e^a{}_\nu = f_a\delta^a_\nu$ and $E_b{}^\nu = \delta^\nu_b/f_b$; the second carries out the sums (the delta keeps only $\nu = a$ and $\lambda = b$). Two cases:

- $a = b$: $\omega_\mu{}^a{}_a = f_a\partial_\mu(1/f_a) + {\Gamma^a}_{\mu a} = -\partial_\mu \ln f_a + \partial_\mu \ln f_a = 0$, by the derivative of $1/f$ ($f\,\partial(1/f) = -\partial f/f$) and the rule ${\Gamma^a}_{\mu a} = \partial_\mu \ln f_a$ above.
- $a \neq b$: $\delta^a_b = 0$, so $\omega_\mu{}^a{}_b = (f_a/f_b)\,{\Gamma^a}_{\mu b}$.

So a component with $a \neq b$ is nonzero exactly when the Christoffel symbol ${\Gamma^a}_{\mu b}$ is. The table shows that for $\mu = x_4$ and $\mu = x_0$ every symbol ${\Gamma^a}_{\mu b}$ with $a \neq b$ vanishes (the nonzero symbols with a lower index x_4 or x_0 all have $a = b$, like ${\Gamma^{x_i}}_{x_i x_4}$), so $\omega_{x_4} = \omega_{x_0} = 0$. For $\mu = x_i$ the symbols with $a \neq b$ are ${\Gamma^{x_i}}_{x_i x_4}$, ${\Gamma^{x_i}}_{x_i x_0}$, ${\Gamma^{x_4}}_{x_i x_i}$ and ${\Gamma^{x_0}}_{x_i x_i}$; for $\mu = x_t$ the same with t. We compute one member of each antisymmetric pair, with $a < b$, line by line:

- $\omega_{x_i}{}^{(i)}{}_{(4)} = (f_i/f_4)\,{\Gamma^{x_i}}_{x_i x_4} = e^{a_4}\sin^{1/6} z\cdot a_4'$, and lowering with $\eta_{ii} = +1$: $\omega_{x_i\,(i)(4)} = a_4' e^{a_4}\sin^{1/6} z$.

- $\omega_{x_i}{}^{(i)}{}_{(0)} = (f_i/f_0)\,\Gamma^{x_i}{}_{x_i x_0} = e^{a_4}\sin^{1/6} z\,\tan z \cdot H\cot z = He^{a_4}\sin^{1/6} z$ (because $\tan z\cot z = 1$), so $\omega_{x_i\,(i)(0)} = He^{a_4}\sin^{1/6} z$.
- $\omega_{x_t}{}^{(4)}{}_{(t)} = (f_4/f_t)\,\Gamma^{x_4}{}_{x_t x_t} = e^{a_4}\sin^{-1/6} z\cdot e^{-2a_4}\sin^{1/3} z\,a_4' = a_4' e^{-a_4}\sin^{1/6} z$, and lowering with $\eta_{44} = -1$: $\omega_{x_t\,(4)(t)} = -a_4' e^{-a_4}\sin^{1/6} z$.
- $\omega_{x_t}{}^{(t)}{}_{(0)} = (f_t/f_0)\,\Gamma^{x_t}{}_{x_t x_0} = e^{-a_4}\sin^{1/6} z\,\tan z\cdot H\cot z = He^{-a_4}\sin^{1/6} z$, and lowering with $\eta_{tt} = -1$: $\omega_{x_t\,(t)(0)} = -He^{-a_4}\sin^{1/6} z$.

Antisymmetry can be seen on one example: $\omega_{x_i}{}^{(4)}{}_{(i)} = (f_4/f_i)\Gamma^{x_4}{}_{x_i x_i} = e^{-a_4}\sin^{-1/6} z\cdot e^{2a_4}\sin^{1/3} z\,a_4' = a_4' e^{a_4}\sin^{1/6} z$, and lowering with $\eta_{44} = -1$ gives $\omega_{x_i\,(4)(i)} = -a_4' e^{a_4}\sin^{1/6} z = -\omega_{x_i\,(i)(4)}$. With three values of i and three of t this gives exactly 12 independent nonzero components $\omega_{\mu ab}$ with $a<b$ (PROVED; `Revision/theory/reports/wolfram-field-theory.json`, check `omega_components`).

The spinor connection. The **generators** $S^{ab} = \frac{1}{4}[\gamma^{(a)},\gamma^{(b)}]$, with the **commutator** $[A,B] = AB - BA$, are, for $a\neq b$, $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)}) = \frac{1}{4}(2\gamma^{(a)}\gamma^{(b)}) = \frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ (because the two gammas anticommute). The spinor connection is $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab} = \sum_{a<b}\omega_{\mu ab}S^{ab}$ (the pairs (a,b) and (b,a) give equal terms, because both $\omega_{\mu ab}$ and S^{ab} change sign). Inserting the components:

$$\begin{aligned}
\Omega_{x_i} &= \omega_{x_i(i)(4)}S^{i4} + \omega_{x_i(i)(0)}S^{i0} = \tfrac{1}{2}e^{a_4}\sin^{1/6} z\,\big(a_4'\gamma^{(i)}\gamma^{(4)} + H\gamma^{(i)}\gamma^{(0)}\big),\\
\Omega_{x_t} &= \omega_{x_t(4)(t)}S^{4t} + \omega_{x_t(t)(0)}S^{t0} = -\tfrac{1}{2}e^{-a_4}\sin^{1/6} z\,\big(a_4'\gamma^{(4)}\gamma^{(t)} + H\gamma^{(t)}\gamma^{(0)}\big),\\
\Omega_{x_4} &= \Omega_{x_0} = 0.
\end{aligned}$$

These are the formulas of the record (`Revision/theory/field-theory.json`, formula key `Omega_components`). Two exact properties of this connection are used below. It makes the gammas **covariantly constant**: $\partial_\mu\gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega_\mu,\gamma^\nu] = 0$ for all 64 pairs (μ,ν) (PROVED; `python-field-theory.json`, check `covariant_constancy_D_mu_gamma_nu`). And its **curvature** $F_{\mu\nu} = \partial_\mu\Omega_\nu - \partial_\nu\Omega_\mu + [\Omega_\mu,\Omega_\nu]$ equals $\frac{1}{4}\sum_{\rho,\sigma}R_{\rho\sigma\mu\nu}\gamma^\rho\gamma^\sigma$, where $R_{\rho\sigma\mu\nu}$ is the Riemann tensor of the metric with its first index lowered (PROVED; `python-field-theory.json`, check `spinor_curvature_equals_riemann`, and `wolfram-field-theory.json`, check `spin_curvature_equals_Riemann`).

8.5 What cancels and what survives: the term $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$

The gravitational term of the field equation is $\gamma^\mu D_\mu\Psi - \gamma^\mu\partial_\mu\Psi = \gamma^\mu\Omega_\mu\Psi$. We compute the matrix $\gamma^\mu\Omega_\mu$ direction by direction (no sum), then add.

A 3-space direction x_i. Line by line:

$$\begin{aligned}
\gamma^{x_i}\Omega_{x_i} &= \frac{\gamma^{(i)}}{e^{a_4}\sin^{1/6} z}\cdot\tfrac{1}{2}e^{a_4}\sin^{1/6} z\,\big(a_4'\gamma^{(i)}\gamma^{(4)} + H\gamma^{(i)}\gamma^{(0)}\big)\\
&= \tfrac{1}{2}\big(a_4'\gamma^{(i)}\gamma^{(i)}\gamma^{(4)} + H\gamma^{(i)}\gamma^{(i)}\gamma^{(0)}\big)\\
&= \tfrac{1}{2}a_4'\gamma^{(4)} + \tfrac{1}{2}H\gamma^{(0)}.
\end{aligned}$$

The first line inserts $\gamma^{x_i} = \gamma^{(i)}/f_i$ and Ω_{x_i}; the second cancels the scale factor against its inverse; the third uses $(\gamma^{(i)})^2 = +I_{16}$ (a space-like direction).

An extra time x_t. Line by line:

$$\begin{aligned}\gamma^{x_t}\Omega_{x_t} &= \frac{\gamma^{(t)}}{e^{-a_4}\sin^{1/6} z}\cdot\Big(-\tfrac{1}{2}e^{-a_4}\sin^{1/6} z\Big)\big(a_4'\gamma^{(4)}\gamma^{(t)}+H\gamma^{(t)}\gamma^{(0)}\big)\\ &= -\tfrac{1}{2}\big(a_4'\gamma^{(t)}\gamma^{(4)}\gamma^{(t)}+H\gamma^{(t)}\gamma^{(t)}\gamma^{(0)}\big)\\ &= -\tfrac{1}{2}\big(-a_4'\gamma^{(4)}\gamma^{(t)}\gamma^{(t)}+H\gamma^{(t)}\gamma^{(t)}\gamma^{(0)}\big)\\ &= -\tfrac{1}{2}\big(a_4'\gamma^{(4)}-H\gamma^{(0)}\big) = -\tfrac{1}{2}a_4'\gamma^{(4)}+\tfrac{1}{2}H\gamma^{(0)}.\end{aligned}$$

The first line inserts $\gamma^{x_t}=\gamma^{(t)}/f_t$ and Ω_{x_t}; the second cancels the scale factors; the third moves $\gamma^{(t)}$ to the right past $\gamma^{(4)}$, which changes the sign (they anticommute); the fourth uses $(\gamma^{(t)})^2=-I_{16}$ (a time-like direction) in both terms.

The time x_4 and the hidden direction x_0. $\Omega_{x_4}=\Omega_{x_0}=0$, so $\gamma^{x_4}\Omega_{x_4}=\gamma^{x_0}\Omega_{x_0}=0$.

The sum. The coefficient of $\gamma^{(4)}$ is $3\cdot\frac{1}{2}a_4'+3\cdot(-\frac{1}{2}a_4')=0$: the three inflating directions and the three deflating extra times cancel exactly. The coefficient of $\gamma^{(0)}$ is $6\cdot\frac{1}{2}H=3H$: the six directions that carry the warp $\sin^{1/6} z$ add up. Hence

$$\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$$

for every history $a_4(x_4)$, every $H>0$ and every point. This is PROVED, and the Revision record proves it twice: the sympy report `python-field-theory.json` has the checks `time_terms_cancel_hidden_term_survives` and `gamma_mu_Omega_mu_equals_3H_gamma_x0`, and the independent Wolfram report `wolfram-field-theory.json` has the checks `gammaOmega_x4_terms_cancel` and `gammaOmega_equals_3H_gamma_x0`. The cancellation happens because 3-space inflates at exactly the rate at which the extra times deflate: the same balance makes the volume factor $\cos z$ independent of the time. Exercise 2 of Section 8.39 shows what would happen if the extra times inflated too. For a changing b_4 the extra times deflate at the rate $a_4'-b_4'/(2b_4)$, and the sum is $3H\gamma^{(0)}+\frac{3}{4}(b_4'/b_4)\gamma^{(4)}$ (Section 7.18; `python-field-theory-b4.json`, check `gammaOmega_total`).

The divergence form. The same matrix has a second, shorter formula. The rule $\Gamma^\mu{}_{\mu\nu}=\partial_\nu\ln f_\mu$ of Section 8.4, summed over μ, gives $\sum_\mu\Gamma^\mu{}_{\mu\nu}=\partial_\nu\ln(f_0\cdots f_7)=\partial_\nu\ln\sqrt{|g|}$. Now sum the covariant constancy of Section 8.4 over $\mu=\nu$:

$$\begin{aligned}0 &= \sum_\mu\Big(\partial_\mu\gamma^\mu+\sum_\lambda\Gamma^\mu{}_{\mu\lambda}\gamma^\lambda+\Omega_\mu\gamma^\mu-\gamma^\mu\Omega_\mu\Big)\\ &= \sum_\mu\partial_\mu\gamma^\mu+\sum_\lambda(\partial_\lambda\ln\sqrt{|g|})\gamma^\lambda+\sum_\mu(\Omega_\mu\gamma^\mu-\gamma^\mu\Omega_\mu)\\ &= \frac{1}{\sqrt{|g|}}\sum_\mu\partial_\mu(\sqrt{|g|}\gamma^\mu)-2\sum_\mu\gamma^\mu\Omega_\mu.\end{aligned}$$

The first line is the covariant constancy with $\nu=\mu$, summed; the second uses the summed rule for the Christoffel symbols; the third combines the first two sums by the product rule ($\partial(\sqrt{|g|}\gamma)=\sqrt{|g|}\,\partial\gamma+(\partial\sqrt{|g|})\gamma$, divided by $\sqrt{|g|}$) and uses $\Omega_\mu\gamma^\mu=-\gamma^\mu\Omega_\mu$ for each μ (Section 8.8 proves this anticommutation). Therefore

$$\gamma^\mu \Omega_\mu = \frac{1}{2\sqrt{|g|}} \sum_\mu \partial_\mu (\sqrt{|g|}\gamma^\mu).$$

For the author's metric only $\mu = x_0$ contributes: $\sqrt{|g|}\gamma^{x_0} = \cos z \tan z\, \gamma^{(0)} = \sin z\, \gamma^{(0)}$, whose x_0 derivative is $6H \cos z\, \gamma^{(0)}$, and dividing by $2\cos z$ gives $3H\gamma^{(0)}$ again; the term with $\mu = x_4$ is $\partial_4(\cos z\, \gamma^{(4)}) = 0$ because $\cos z$ does not depend on the time (for a changing b_4, $\sqrt{|g|} = b_4^{3/2} \cos z$ and this term gives $\frac{3}{4}(b_4'/b_4)\gamma^{(4)}$), and the other six vanish because nothing depends on $x_1, x_2, x_3, x_5, x_6, x_7$ (PROVED; `python-field-theory.json`, check `divergence_of_sqrtg_gamma`). In words: the term is half the divergence of the coordinate gammas weighted with the volume factor, a **half-density term**.

8.6 Non-triviality: the connection vanishes only in flat space

The connection is zero only in the flat limit. Each of the 12 components of Section 8.4 is a_4' times one of the factors $\pm e^{\pm a_4} \sin^{1/6} z$ (six components), or H times one of them (the other six). These factors are never zero for $0 < z < \pi/2$: an exponential is never zero, and $\sin z > 0$ there. The 28 matrices S^{ab} with $a < b$ are **linearly independent**: no combination of them with coefficients that are not all zero gives the zero matrix (Notebook 08c checks it: written as 28 rows of $16 \times 16 = 256$ numbers they have rank 28). So, line by line:

1. $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$ is zero exactly when all its coefficients $\omega_{\mu ab}$ are zero (linear independence).
2. If $a_4' \neq 0$, the six components of the form $a_4' \times$(factor) are not zero; if $H \neq 0$, the six of the form $H \times$(factor) are not zero.
3. Therefore $\Omega_\mu = 0$ for every μ if and only if $a_4' = 0$ and $H = 0$.

The case $H = 0$ is the formal flat limit; it is not a member of the author's family (Section 8.2). For every metric of the family the canonical spin connection is therefore not zero. This is PROVED; in the Revision record it is the check `nontriviality_Omega_zero_iff_flat` of the sympy report `python-field-theory.json` and the check `Omega_vanishes_iff_a4prime_and_H_vanish` of the Wolfram report `wolfram-field-theory.json`.

The term is not zero for any field that is not zero. Suppose $3H\gamma^{(0)}\Psi = 0$ at some point. Multiply from the left by $\gamma^{(0)}/(3H)$ (allowed, since $H > 0$): $\gamma^{(0)}\gamma^{(0)}\Psi = 0$, and $(\gamma^{(0)})^2 = I_{16}$ gives $\Psi = 0$. So in the diagonal frame the gravitational term $3H\gamma^{(0)}\Psi$ is nonzero wherever the field is nonzero (PROVED; `wolfram-field-theory.json`, checks `nontriviality_1_dirac16complex` and `nontriviality_2_dirac16complex00`).

What this proves, and what it does not. Read literally, in the diagonal frame and for the field variables Ψ, the tests [1] and [2] hold: the field equations always contain the nonzero gravitational term $3H\gamma^{(0)}\Psi$, and the spin connection vanishes only in flat 4+4 space. Sections 8.7 to 8.10 and 8.30 make precise which parts of this statement survive a change of frame or of variables.

8.7 The deflation is invisible in the term, but not in the curvature

No a_4 in the term. The result $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$ contains neither a_4 nor a_4' nor a_4'': the deflation of the extra times leaves no trace in this term (PROVED; `Revision/theory/reports/python-scope.json`, check `gammaOmega_blind_to_the_deflation`). Along the deflating history the separate pieces do change: the components $\omega_{x_1\,(1)(4)}$ and $\omega_{x_1\,(1)(8)}$ of the inflating direction grow like e^{a_4}, those of a deflating extra time shrink like e^{-a_4}. Contracted with the coordinate gammas, which carry the inverse factors, each piece becomes constant, and the pieces then cancel; Notebook 08c draws this (Figure 08c.2).

The geometry of the a_4 sector is curved. The mixed Ricci tensor of the author's metric (Chapter 3) is diagonal, with

$$R^{x_i}{}_{x_i} = a_4'' - 6H^2, \quad R^{x_4}{}_{x_4} = 6(a_4')^2, \quad R^{x_t}{}_{x_t} = -a_4'' - 6H^2, \quad R^{x_0}{}_{x_0} = -6H^2,$$

and the Ricci scalar is $R = 3(a_4'' - 6H^2) + 6(a_4')^2 + 3(-a_4'' - 6H^2) - 6H^2 = 6\big((a_4')^2 - 7H^2\big)$ (adding the eight diagonal entries: the a_4'' terms cancel and $-18H^2 - 18H^2 - 6H^2 = -42H^2$). (PROVED; `wolfram-field-theory.json`, check `ricci_mixed_components`, and `python-field-theory.json`, check `curvature_nonzero_flat_only_formally`.) Since $R^{x_0}{}_{x_0} = -6H^2 < 0$ for every $H > 0$ and every a_4, the metric is never flat (`wolfram-field-theory.json`, check `never_flat_for_H_positive`). And $R^{x_4}{}_{x_4} = 6(a_4')^2$ is not zero whenever a_4 changes: the deflation is real curvature, which the term $\gamma^\mu\Omega_\mu$ does not see.

Where the deflation does enter. It enters the field equation through the frame factors of the derivative terms (Section 8.2): the 3-space derivatives carry $e^{-a_4}\sin^{-1/6} z$, which shrinks as 3-space inflates, and the extra-time derivatives carry $e^{a_4}b_4^{-1/2}\sin^{-1/6} z$ ($e^{a_4}\sin^{-1/6} z$ at $b_4 = 1$), which grows as the extra times deflate. Sections 8.24 and 8.25 show what this growth does to waves along the extra times.

8.8 The connection drops out of the Lagrangian but not out of the field equation

An anticommutation for each direction. For each μ separately (no sum), $\{\gamma^\mu, \Omega_\mu\} = 0$. For $\mu = x_i$ it suffices to check the two products in Ω_{x_i} against $\gamma^{(i)}$:

$$\gamma^{(i)}\big(\gamma^{(i)}\gamma^{(4)}\big) + \big(\gamma^{(i)}\gamma^{(4)}\big)\gamma^{(i)} = \gamma^{(4)} + \gamma^{(i)}\gamma^{(4)}\gamma^{(i)} = \gamma^{(4)} - \gamma^{(4)}\gamma^{(i)}\gamma^{(i)} = \gamma^{(4)} - \gamma^{(4)} = 0,$$

using $(\gamma^{(i)})^2 = I_{16}$, then moving $\gamma^{(i)}$ past $\gamma^{(4)}$ (one sign change), then $(\gamma^{(i)})^2 = I_{16}$ again. For the second product the same three steps read $\gamma^{(i)}\gamma^{(i)}\gamma^{(0)} + \gamma^{(i)}\gamma^{(0)}\gamma^{(i)} = \gamma^{(0)} - \gamma^{(0)}\gamma^{(i)}\gamma^{(i)} = \gamma^{(0)} - \gamma^{(0)} = 0$. For $\mu = x_t$ the products are $\gamma^{(4)}\gamma^{(t)}$ and $\gamma^{(t)}\gamma^{(0)}$: $\gamma^{(t)}\gamma^{(4)}\gamma^{(t)} + \gamma^{(4)}\gamma^{(t)}\gamma^{(t)} = -\gamma^{(4)}\gamma^{(t)}\gamma^{(t)} + \gamma^{(4)}\gamma^{(t)}\gamma^{(t)} = 0$, and $\gamma^{(t)}\gamma^{(t)}\gamma^{(0)} + \gamma^{(t)}\gamma^{(0)}\gamma^{(t)} = \gamma^{(t)}\gamma^{(t)}\gamma^{(0)} - \gamma^{(t)}\gamma^{(t)}\gamma^{(0)} = 0$. For $\mu = x_4, x_0$ the connection is zero. (PROVED; `python-scope.json`, check `connection_free_lagrangian_same_equations`, and `python-field-theory.json`, check `anticommutator_gamma_Omega_vanishes`.)

The connection drops out of the Lagrangian. The kinetic part of the Lagrangian (Chapter 7) is $\sqrt{|g|}\cdot\frac{1}{2}\big(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi\big)$ with $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$. Its connection part is, line by line,

$$\tfrac{1}{2}\big(\bar\Psi\gamma^\mu\Omega_\mu\Psi - (-\bar\Psi\Omega_\mu)\gamma^\mu\Psi\big) = \tfrac{1}{2}\bar\Psi\big(\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu\big)\Psi = \tfrac{1}{2}\bar\Psi\{\gamma^\mu, \Omega_\mu\}\Psi = 0.$$

The first step keeps only the Ω parts of the two covariant derivatives; the second collects them between $\bar\Psi$ and Ψ; the third is the anticommutation just proved, for each μ. So in this metric the Lagrangian with the canonical spin connection equals the Lagrangian with the connection left out (PROVED; `python-scope.json`, check `connection_free_lagrangian_same_equations`).

But the field equation keeps the term. Take the connection-free symmetric Lagrangian of the commuting field, $\mathcal{L}_0 = \sqrt{|g|}\big[\tfrac{1}{2}(\bar\Psi\gamma^\mu\partial_\mu\Psi - \partial_\mu\bar\Psi\gamma^\mu\Psi) - mS - U(S)\big]$, and vary it with respect to $\bar\Psi$, treating the components of $\bar\Psi$ and of Ψ as independent variables (Chapter 7 explains why this is allowed). The Euler-Lagrange expression is $\partial\mathcal{L}_0/\partial\bar\Psi - \sum_\mu \partial_\mu(\partial\mathcal{L}_0/\partial(\partial_\mu\bar\Psi))$. Line by line:

$$\begin{aligned}
\frac{\partial\mathcal{L}_0}{\partial\bar\Psi} &= \sqrt{|g|}\big[\tfrac{1}{2}\gamma^\mu\partial_\mu\Psi - m\Psi - U'(S)\Psi\big],\\
\frac{\partial\mathcal{L}_0}{\partial(\partial_\mu\bar\Psi)} &= -\tfrac{1}{2}\sqrt{|g|}\gamma^\mu\Psi,\\
\frac{\partial\mathcal{L}_0}{\partial\bar\Psi} - \sum_\mu \partial_\mu\Big(\frac{\partial\mathcal{L}_0}{\partial(\partial_\mu\bar\Psi)}\Big) &= \sqrt{|g|}\big[\tfrac{1}{2}\gamma^\mu\partial_\mu\Psi - (m+U')\Psi\big] + \tfrac{1}{2}\sum_\mu \partial_\mu(\sqrt{|g|}\gamma^\mu)\Psi + \tfrac{1}{2}\sqrt{|g|}\gamma^\mu\partial_\mu\Psi\\
&= \sqrt{|g|}\Big[\gamma^\mu\partial_\mu\Psi + \frac{1}{2\sqrt{|g|}}\sum_\mu \partial_\mu(\sqrt{|g|}\gamma^\mu)\Psi - (m+U')\Psi\Big].
\end{aligned}$$

The first line differentiates the terms that contain $\bar\Psi$ without a derivative ($\partial S/\partial\bar\Psi = \Psi$ and $\partial U/\partial\bar\Psi = U'(S)\Psi$); the second differentiates the one term that contains $\partial_\mu\bar\Psi$; the third subtracts the derivative of the second line, using the product rule $\partial_\mu(\sqrt{|g|}\gamma^\mu\Psi) = \partial_\mu(\sqrt{|g|}\gamma^\mu)\Psi + \sqrt{|g|}\gamma^\mu\partial_\mu\Psi$; the fourth collects the two halves of $\gamma^\mu\partial_\mu\Psi$. Setting it to zero and using the divergence form of Section 8.5 gives exactly $\gamma^\mu\partial_\mu\Psi + 3H\gamma^{(0)}\Psi = (m+U')\Psi$, the field equation of Section 8.2. (The record proves the same for the Grassmann field with left derivatives: `python-field-theory.json`, check `grassmann_euler_lagrange_psibar_variation`.)

So the gravitational term $3H\gamma^{(0)}\Psi$ is at the same time the term that the canonical spin connection produces and the half-density term that the volume factor produces when a Lagrangian without any spin connection is varied (PROVED; `python-scope.json`, check `connection_free_lagrangian_same_equations`, both engines). This is the first hint that the value $3H\gamma^{(0)}$ is not a deep property of gravity's coupling: the next two sections make that precise.

8.9 Another frame: the boosted frame and its term

Take the boosted frame of Section 8.3, with rapidity $b = \beta x_4 + b_0$. Its canonical spin connection (computed from the definition of Section 8.4 with the new frame) satisfies the vielbein postulate and is antisymmetric (PROVED; `python-scope.json`, check `boosted_frame_canonical_connection`). The record then finds, exactly, for every a_4, every $H > 0$ and every β (at $b_4 = 1$; for a general b_4 the term $\tfrac{3}{4}(b_4'/b_4)(\cosh b\,\gamma^{(4)} - \sinh b\,\gamma^{(0)})$ is added, Section 6.16):

$$\gamma'^\mu\Omega'_\mu = \frac{6H-\beta}{2}\big(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)}\big)$$

(PROVED; `python-scope.json`, check `boosted_frame_gammaOmega_formula`, and the same check of `wolfram-scope.json`). Notebook 08c computes it from the definitions, with sympy. Here we derive it by a shorter route that also explains it.

Step 1: the matrix that moves the gammas. Let $J = \gamma^{(4)}\gamma^{(0)}$. Its square is $J^2 = \gamma^{(4)}\gamma^{(0)}\gamma^{(4)}\gamma^{(0)} = -\gamma^{(4)}\gamma^{(4)}\gamma^{(0)}\gamma^{(0)} = -(-I_{16})(I_{16}) = I_{16}$ (anticommute the middle pair, then use the two squares). For a matrix with $J^2 = I_{16}$ the exponential series splits into even and odd powers, and $e^{\theta J} = \cosh\theta\, I_{16} + \sinh\theta\, J$. Define

$$S = e^{-(b/2)J} = \cosh\tfrac{b}{2}\, I_{16} - \sinh\tfrac{b}{2}\, J, \qquad S^{-1} = e^{(b/2)J} = \cosh\tfrac{b}{2}\, I_{16} + \sinh\tfrac{b}{2}\, J.$$

Step 2: what S does to each gamma. J commutes with $\gamma^{(a)}$ for $a \neq 4, 8$ (moving $\gamma^{(a)}$ past the two factors of J changes the sign twice), so $S\gamma^{(a)}S^{-1} = \gamma^{(a)}$ for those six directions. J anticommutes with $\gamma^{(4)}$ and with $\gamma^{(0)}$. To see it for $\gamma^{(4)}$, move $\gamma^{(4)}$ from the left of $J = \gamma^{(4)}\gamma^{(0)}$ to its right: passing the factor $\gamma^{(4)}$ (itself) changes nothing, and passing the factor $\gamma^{(0)}$ changes the sign once, so $\gamma^{(4)}J = \gamma^{(4)}\gamma^{(4)}\gamma^{(0)} = -\gamma^{(4)}\gamma^{(0)}\gamma^{(4)} = -J\gamma^{(4)}$. For $\gamma^{(0)}$ it is the same with the roles exchanged: passing $\gamma^{(4)}$ changes the sign once and passing $\gamma^{(0)}$ changes nothing, so $\gamma^{(0)}J = \gamma^{(0)}\gamma^{(4)}\gamma^{(0)} = -\gamma^{(4)}\gamma^{(0)}\gamma^{(0)} = -J\gamma^{(0)}$. Because $\gamma^{(4)}$ anticommutes with J, it turns every power J^n into $(-J)^n$ when it is moved past it, so $\gamma^{(4)}e^{\theta J} = e^{-\theta J}\gamma^{(4)}$ for every number θ; in particular $S\gamma^{(4)} = e^{-(b/2)J}\gamma^{(4)} = \gamma^{(4)}e^{(b/2)J} = \gamma^{(4)}S^{-1}$, and line by line

$$\begin{aligned} S\gamma^{(4)}S^{-1} &= \gamma^{(4)}S^{-1}S^{-1} = \gamma^{(4)}e^{bJ} = \gamma^{(4)}\big(\cosh b\, I_{16} + \sinh b\, \gamma^{(4)}\gamma^{(0)}\big) \\ &= \cosh b\, \gamma^{(4)} + \sinh b\, (\gamma^{(4)})^2\gamma^{(0)} = \cosh b\, \gamma^{(4)} - \sinh b\, \gamma^{(0)}. \end{aligned}$$

The first line moves S to the right of $\gamma^{(4)}$ (it becomes S^{-1}), multiplies the two exponentials ($e^{(b/2)J}e^{(b/2)J} = e^{bJ}$) and writes out e^{bJ}; the second multiplies out and uses $(\gamma^{(4)})^2 = -I_{16}$. In the same way $S\gamma^{(0)}S^{-1} = \gamma^{(0)}e^{bJ} = \cosh b\, \gamma^{(0)} + \sinh b\, \gamma^{(0)}\gamma^{(4)}\gamma^{(0)} = \cosh b\, \gamma^{(0)} - \sinh b\, \gamma^{(4)}$ (here $\gamma^{(0)}\gamma^{(4)}\gamma^{(0)} = -\gamma^{(4)}(\gamma^{(0)})^2 = -\gamma^{(4)}$).

Step 3: the coordinate gammas of the boosted frame. The boosted frame is $e' = Le$, with the boost matrix L; its inverse is $E' = EL^{-1}$ (the inverse of a product is the product of the inverses in reverse order), and L^{-1} is the boost with $-b$. So $\gamma'^{\mu} = \sum_a E'_a{}^{\mu}\gamma^{(a)} = \sum_b E_b{}^{\mu}\tilde{\gamma}^{(b)}$, where $\tilde{\gamma}^{(4)} = \cosh b\, \gamma^{(4)} - \sinh b\, \gamma^{(0)}$, $\tilde{\gamma}^{(0)} = \cosh b\, \gamma^{(0)} - \sinh b\, \gamma^{(4)}$ and $\tilde{\gamma}^{(a)} = \gamma^{(a)}$ otherwise. By Step 2, $\tilde{\gamma}^{(b)} = S\gamma^{(b)}S^{-1}$ for every b, and therefore $\gamma'^{\mu} = S\gamma^{\mu}S^{-1}$.

Step 4: the connection of the boosted frame. Chapter 6 proves in general (in its section on local spin covariance) that the canonical spinor connection of the turned frame is $S\Omega_\mu S^{-1} - (\partial_\mu S)S^{-1}$. Here is a shorter argument for the same result. It rests on two facts: the canonical spin connection of every frame makes that frame's coordinate gammas covariantly constant (a consequence of the vielbein postulate), and it is a combination of the S^{ab}, so its trace (the sum of its diagonal entries) is zero. Both are established in Chapter 6; we quote them here, and the direct computation of the record and of Notebook 08c confirms their consequence. We show that $\Omega'_\mu = S\Omega_\mu S^{-1} - (\partial_\mu S)S^{-1}$ has both properties, and that only one matrix has them. First, covariant constancy, line by line (with $\partial_\mu S^{-1} = -S^{-1}(\partial_\mu S)S^{-1}$, which follows from differentiating $SS^{-1} = I_{16}$):

$$\begin{aligned} \partial_\mu\gamma'^{\nu} &= (\partial_\mu S)\gamma^{\nu}S^{-1} + S(\partial_\mu\gamma^{\nu})S^{-1} - S\gamma^{\nu}S^{-1}(\partial_\mu S)S^{-1}, \\ [\Omega'_\mu, \gamma'^{\nu}] &= S[\Omega_\mu, \gamma^{\nu}]S^{-1} - (\partial_\mu S)\gamma^{\nu}S^{-1} + S\gamma^{\nu}S^{-1}(\partial_\mu S)S^{-1}, \\ \partial_\mu\gamma'^{\nu} + \sum_\lambda \Gamma^{\nu}{}_{\mu\lambda}\gamma'^{\lambda} + [\Omega'_\mu, \gamma'^{\nu}] &= S\Big(\partial_\mu\gamma^{\nu} + \sum_\lambda \Gamma^{\nu}{}_{\mu\lambda}\gamma^{\lambda} + [\Omega_\mu, \gamma^{\nu}]\Big)S^{-1} = 0. \end{aligned}$$

The first line is the product rule for $S\gamma^{\nu}S^{-1}$; the second inserts Ω'_μ and γ'^{ν} into the commutator and

multiplies out; the third adds the two lines and the Christoffel term (Γ belongs to the metric, the same in both frames), sees the four terms with $\partial_\mu S$ cancel in pairs, and uses the covariant constancy of the diagonal frame. Second, the trace: $\mathrm{tr}(S\Omega_\mu S^{-1}) = \mathrm{tr}\,\Omega_\mu = 0$, and $(\partial_\mu S)S^{-1}$ is a multiple of J (Step 5 below), whose trace is zero (the trace of a product of two different gammas is zero: $\mathrm{tr}(\gamma^{(4)}\gamma^{(0)}) = \mathrm{tr}(\gamma^{(0)}\gamma^{(4)}) = -\mathrm{tr}(\gamma^{(4)}\gamma^{(0)})$, by the cyclic rule of the trace and then anticommutation). Third, uniqueness: if two traceless matrices Ω'_1, Ω'_2 both make the γ'^ν covariantly constant, their difference commutes with every γ'^ν; then $S^{-1}(\Omega'_1 - \Omega'_2)S$ commutes with every $\gamma^{(a)}$, so it is a multiple of I_{16} (the gammas form an irreducible representation, Chapter 5; `Revision/algebra/reports/python-algebra.json`, check `pin_commutant_dimension_1`), and a traceless multiple of I_{16} is zero. Hence the canonical connection of the boosted frame is $\Omega'_\mu = S\Omega_\mu S^{-1} - (\partial_\mu S)S^{-1}$.

Step 5: the term. Line by line:

$$
\begin{aligned}
\gamma'^\mu\Omega'_\mu &= S\gamma^\mu S^{-1}\big(S\Omega_\mu S^{-1} - (\partial_\mu S)S^{-1}\big) = S\big(\gamma^\mu\Omega_\mu - \gamma^\mu S^{-1}\partial_\mu S\big)S^{-1},\\
S^{-1}\partial_\mu S &= -\tfrac{\beta}{2}J \text{ for } \mu = x_4, \quad 0 \text{ otherwise},\\
\gamma^{x_4}S^{-1}\partial_4 S &= \gamma^{(4)}\big(-\tfrac{\beta}{2}\gamma^{(4)}\gamma^{(0)}\big) = -\tfrac{\beta}{2}(\gamma^{(4)})^2\gamma^{(0)} = \tfrac{\beta}{2}\gamma^{(0)},\\
\gamma'^\mu\Omega'_\mu &= S\big(3H\gamma^{(0)} - \tfrac{\beta}{2}\gamma^{(0)}\big)S^{-1} = \frac{6H-\beta}{2}\,S\gamma^{(0)}S^{-1} = \frac{6H-\beta}{2}\big(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)}\big).
\end{aligned}
$$

The first line inserts Steps 3 and 4 and cancels $S^{-1}S$; the second differentiates $S = e^{-(b/2)J}$ with $\partial_4 b = \beta$ (only b depends on the coordinates, and only on x_4); the third uses $\gamma^{x_4} = \gamma^{(4)}$ ($f_4 = 1$) and $(\gamma^{(4)})^2 = -I_{16}$; the fourth uses $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$ (Section 8.5) and Step 2. This is the record's formula.

The consequence. For $\beta = 6H$, that is with the rapidity $b = 6Hx_4 + b_0$, the term $\gamma'^\mu\Omega'_\mu$ is identically zero, for every $H > 0$ and every history a_4, although the metric is the author's curved metric (PROVED; `python-scope.json`, check `boosted_frame_gammaOmega_vanishes`, both engines). The connection itself is not zero in that frame: $\Omega'_\mu \neq 0$ for $\mu = x_0, \ldots, x_6$ (zero only for x_7; the same check). So the sentence "the field equation contains the gravitational term $3H\gamma^{(0)}\Psi$" is true in the diagonal frame and false in this frame: it is a statement about one frame, not about the field equation as such. In the boosted frame gravity enters the field equation through the boosted frame vectors $E'^{\,\mu}_a$ alone.

8.10 What no frame removes: the curvature of the connection

The curvature in two frames. The curvature of the spinor connection equals the Riemann tensor contracted with two coordinate gammas, $F_{\mu\nu} = \frac{1}{4}\sum_{\rho,\sigma} R_{\rho\sigma\mu\nu}\gamma^\rho\gamma^\sigma$ (Section 8.4). This identity holds in every frame (a standard theorem, quoted without proof here as in Chapter 6, so ASSUMED in general; the record proves it for this metric in the diagonal frame, and Notebook 08c checks it in the diagonal and in the boosted frame for $(\mu,\nu) = (x_1, x_0)$). Chapter 6 also derives the relation below directly from local spin covariance, without this identity. The Riemann tensor belongs to the metric and is the same in both frames, and $\gamma'^\rho = S\gamma^\rho S^{-1}$ (Section 8.9). Hence, line by line,

$$
F'_{\mu\nu} = \tfrac{1}{4}\sum_{\rho,\sigma} R_{\rho\sigma\mu\nu}\, S\gamma^\rho S^{-1}S\gamma^\sigma S^{-1} = S\Big(\tfrac{1}{4}\sum_{\rho,\sigma} R_{\rho\sigma\mu\nu}\gamma^\rho\gamma^\sigma\Big)S^{-1} = SF_{\mu\nu}S^{-1}.
$$

It is never zero. In the diagonal frame Notebook 08c finds $F_{x_1x_0} = c_{14}\,\gamma^{(1)}\gamma^{(4)} + c_{10}\,\gamma^{(1)}\gamma^{(0)}$ with

$$c_{14} = -\frac{H a_4' e^{a_4}\cos z}{2\sin^{5/6} z}, \qquad c_{10} = -\frac{H^2 e^{a_4}\cos z}{2\sin^{5/6} z}.$$

The coefficient c_{10} is never zero for $H > 0$ and $0 < z < \pi/2$, and $\gamma^{(1)}\gamma^{(4)}$ and $\gamma^{(1)}\gamma^{(0)}$ are linearly independent, so $F_{x_1x_0} \neq 0$ for every history a_4. Since S is invertible, $F'_{x_1x_0} = SF_{x_1x_0}S^{-1} \neq 0$ too. If the connection Ω'_μ vanished at all points of a region in some frame, its curvature would vanish there; it does not. So the spin connection vanishes in no frame of this kind (PROVED; `python-scope.json`, check `boosted_frame_curvature_nonzero`, both engines, which argues in general: the curvature of the connection is $\frac{1}{2}R_{ab\mu\nu}S^{ab}$ in every frame, and the metric is curved for $H > 0$).

Two frames, one curvature. The square of $F_{x_1x_0}$ is the same in both frames, $F^2 = (c_{14}^2 - c_{10}^2)I_{16} = H^2((a_4')^2 - H^2)e^{2a_4}\cos^2 z/(4\sin^{5/3} z)\cdot I_{16}$: the products $\gamma^{(1)}\gamma^{(4)}$ and $\gamma^{(1)}\gamma^{(0)}$ square to $+I_{16}$ and $-I_{16}$ and anticommute, so the cross terms cancel, and $F'^2 = SF^2S^{-1} = F^2$ because F^2 is a multiple of I_{16}. On the canonical history $a_4 = x_4$ (where $a_4' = H = 1$) this square is zero although F is not: F is then **nilpotent** (its square is the zero matrix, like the 2×2 matrix with rows $(0,1)$ and $(0,0)$), and in the boosted frame with $\beta = 6H$ it is $F' = e^{-b}F$ (Exercise 6 of Section 8.39 derives this).

The frame-independent content of non-triviality. Putting Sections 8.6 to 8.10 together: the canonical spin connection is nonzero in every frame, because its curvature is the curvature of spacetime, which is nonzero for every $H > 0$; and the frame factors $e^{\mp a_4}\sin^{-1/6} z$ and $\tan z$, none of which is identically 1, multiply every derivative term of the field equation except the one along the time x_4 (the derivatives along $x_1, x_2, x_3, x_5, x_6, x_7, x_0$; the factor of ∂_4 is $1/f_4 = 1$). These two facts do not depend on any choice. The value $3H\gamma^{(0)}$ of the term $\gamma^\mu\Omega_\mu$ does.

8.11 The connection in the energy-momentum tensor

The spin connection drops out of the Lagrangian (Section 8.8), but it does reach the energy-momentum tensor, the source of gravity (Chapter 9). For a field that depends only on the time, $\Phi = \Phi(x_4)$, the entry $K^{x_4}{}_{x_1} = \frac{1}{2}(\bar\Phi\gamma^{x_4}D_{x_1}\Phi - (D_{x_1}\bar\Phi)\gamma^{x_4}\Phi)$ has no derivative part ($\partial_1\Phi = 0$), and its connection part is $\frac{1}{2}\bar\Phi\{\gamma^{x_4}, \Omega_{x_1}\}\Phi$. Line by line, with $\gamma^{x_4} = \gamma^{(4)}$ and $\Omega_{x_1} = \frac{1}{2}f_1(a_4'\gamma^{(1)}\gamma^{(4)} + H\gamma^{(1)}\gamma^{(0)})$, $f_1 = e^{a_4}\sin^{1/6} z$:

$$\begin{aligned}
&\gamma^{(4)}\gamma^{(1)}\gamma^{(4)} + \gamma^{(1)}\gamma^{(4)}\gamma^{(4)} = -\gamma^{(1)}\gamma^{(4)}\gamma^{(4)} + \gamma^{(1)}\gamma^{(4)}\gamma^{(4)} = 0,\\
&\gamma^{(4)}\gamma^{(1)}\gamma^{(0)} + \gamma^{(1)}\gamma^{(0)}\gamma^{(4)} = \gamma^{(4)}\gamma^{(1)}\gamma^{(0)} + \gamma^{(4)}\gamma^{(1)}\gamma^{(0)} = 2\gamma^{(4)}\gamma^{(1)}\gamma^{(0)},\\
&\qquad \tfrac{1}{2}\{\gamma^{x_4}, \Omega_{x_1}\} = \tfrac{1}{2}\cdot\tfrac{1}{2}f_1\big(a_4'\cdot 0 + H\cdot 2\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\big) = \tfrac{1}{2}e^{a_4}\sin^{1/6} z\, H\,\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}.
\end{aligned}$$

The first line moves the first $\gamma^{(4)}$ past $\gamma^{(1)}$ (one sign) and finds that the a_4' part cancels; the second moves $\gamma^{(4)}$ in the second product to the front past two gammas (two signs); the third collects. The bilinear matrix $C\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}$ is not the zero matrix, so this entry is not zero in general (PROVED; `python-scope.json`, check `spin_connection_in_the_energy_momentum_tensor`, both engines). Chapter 9 uses this entry (Section 9.12).

8.12 Example: the term in two frames, computed from the definitions

Notebook 08c puts Sections 8.3 to 8.11 to work, exactly, with sympy, for a symbolic history $a_4(x_4)$ and symbolic H, β, b_0. It builds the scale factors and the metric and compares them with the record; computes all Christoffel symbols and the canonical spin connection of the diagonal frame and checks the vielbein postulate for all 512 components; shows that the connection vanishes only for $a_4' = 0$ and $H = 0$; computes $\gamma^\mu \Omega_\mu$ direction by direction and draws what cancels and what survives; computes the Ricci components and shows that the term contains no a_4; builds the boosted frame and finds $\gamma'^\mu \Omega'_\mu$; computes the spinor curvature in both frames; and checks the connection part of the energy-momentum tensor. It needs no Rust and runs in one to two minutes; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/08c_frame_dependence.PROVENANCE.md`. It ends with the line ALL 32 CHECKS PASSED (notebook 08c).

8.13 How to run Notebook 08c

Step 1. What this notebook does and what it needs.

Notebook 08c (The spin-connection term depends on the frame) is the file `Revision/textbook/notebooks/08c_frame_dependence.ipynb` of the repository Dirac_claude. From the author's metric and gamma matrices it computes exactly (sympy) the Christoffel symbols, the canonical spin connection of the diagonal frame and of a frame boosted in the (x4, x0) plane, and the term gamma^mu Omega_mu of the field equation; it shows that the connection vanishes only in the formal flat limit, which pieces cancel and which survive (3 H gamma^(x0) in the diagonal frame), that the term contains no a4 although the a4 sector is curved, that it vanishes identically in the frame boosted with rapidity 6 H x4 + b0, that the spin connection itself vanishes in no frame (its curvature is the Riemann tensor), and that it enters the energy-momentum tensor; it draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
```

```
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08c_frame_dependence.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08c_frame_dependence.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 08c_frame_dependence.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/08c.captions.json`
- `Revision/textbook/figures/08c_1_cancel_and_survive.png`
- `Revision/textbook/figures/08c_2_pieces_along_history.png`
- `Revision/textbook/figures/08c_3_boost_size.png`
- `Revision/textbook/figures/08c_4_heat_maps.png`
- `Revision/textbook/figures/08c_5_curvature_maps.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08c_5_curvature_maps.png exists
ALL 32 CHECKS PASSED (notebook 08c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/08c_frame_dependence.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 08c_frame_dependence.ipynb
python -m nbconvert --execute --inplace 08c_frame_dependence.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- A cell of sections 7, 11 or 12 runs for more than a minute: these cells simplify hundreds of exact expressions with sympy; on a slow computer each can take a few minutes. Wait until the star in the brackets to the left of the cell turns into a number.

8.14 Notebook 08c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/08c_frame_dependence.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field equation of dirac16complex (and of dirac16complex00) contains the term
$\gamma^\mu\Omega_\mu\Psi$, where $\Omega_\mu$ is the canonical spin connection. In
the diagonal frame of the author's metric this term is $3H\gamma^{(0)}\Psi$. This
notebook computes everything from the definitions, exactly, with sympy, and shows
what that value means and what it does not mean. It

- builds the author's metric from the diagonal vielbein, its Christoffel symbols
  (25 independent nonzero ones) and the canonical spin connection (12 independent
  nonzero components), and checks the vielbein postulate for all 512 components;
- shows that the connection vanishes only in the formal flat limit $a_4' = 0$,
  $H = 0$ (the core of non-triviality) and that the term $3H\gamma^{(0)}\Psi$ is
  not zero for any field $\Psi \neq 0$;
- shows, direction by direction, which pieces of $\gamma^\mu\Omega_\mu$ cancel (the
  time-direction pieces of the three inflating and the three deflating
  directions) and which survive (the hidden-direction pieces, $3H\gamma^{(0)}$);
- shows that the term contains no $a_4$, although the curvature does
  ($R^{x_4}{}_{x_4} = 6(a_4')^2$): the deflation of the extra times is invisible
  in this term;
- repeats everything in a second, equally valid frame, boosted in the
  $(x_4, x_0)$ plane, finds $\gamma'^\mu\Omega'_\mu = \frac{6H - \beta}{2}
```

```
  (\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$, and so a frame in which the
  term is identically ZERO;
- shows that the spin connection itself is not zero in that frame (nor in any
  other): its curvature is the Riemann tensor, which is not zero;
- shows that the spin connection enters the energy-momentum tensor.

Every exact statement reproduces a check of a Revision record. Five teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 08c (The spin-connection term depends on the frame) is the file
‘Revision/textbook/notebooks/08c_frame_dependence.ipynb‘ of the repository Dirac_claude.
From the author’s metric and gamma matrices it computes exactly (sympy) the Christoffel
symbols, the canonical spin connection of the diagonal frame and of a frame boosted in
the (x4, x0) plane, and the term gamma^mu Omega_mu of the field equation; it shows that
the connection vanishes only in the formal flat limit, which pieces cancel and which
survive (3 H gamma^(x0) in the diagonal frame), that the term contains no a4 although the
a4 sector is curved, that it vanishes identically in the frame boosted with rapidity 6 H
x4 + b0, that the spin connection itself vanishes in no frame (its curvature is the
Riemann tensor), and that it enters the energy-momentum tensor; it draws five teaching
plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for
the installation, the program Git, and Python 3.12 or newer (the notebooks were built
with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:
```

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08c_frame_dependence.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,

copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08c_frame_dependence.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 08c_frame_dependence.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/08c.captions.json‘
- ‘Revision/textbook/figures/08c_1_cancel_and_survive.png‘
- ‘Revision/textbook/figures/08c_2_pieces_along_history.png‘
- ‘Revision/textbook/figures/08c_3_boost_size.png‘
- ‘Revision/textbook/figures/08c_4_heat_maps.png‘
- ‘Revision/textbook/figures/08c_5_curvature_maps.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08c_5_curvature_maps.png exists
ALL 32 CHECKS PASSED (notebook 08c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/08c_frame_dependence.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 08c_frame_dependence.ipynb
      python -m nbconvert --execute --inplace 08c_frame_dependence.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- A cell of sections 7, 11 or 12 runs for more than a minute: these cells simplify hundreds of exact expressions with sympy; on a slow computer each can take a few minutes. Wait until the star in the brackets to the left of the cell turns into a number.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 08c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 08c (The spin-connection term depends on the frame) is the file
# Revision/textbook/notebooks/08c_frame_dependence.ipynb of the repository Dirac_claude.
# From the author's metric and gamma matrices it computes exactly (sympy) the Christoffel
# symbols, the canonical spin connection of the diagonal frame and of a frame boosted in
# the (x4, x0) plane, and the term gamma^mu Omega_mu of the field equation; it shows that
# the connection vanishes only in the formal flat limit, which pieces cancel and which
# survive (3 H gamma^(x0) in the diagonal frame), that the term contains no a4 although
# the a4 sector is curved, that it vanishes identically in the frame boosted with
# rapidity 6 H x4 + b0, that the spin connection itself vanishes in no frame (its
# curvature is the Riemann tensor), and that it enters the energy-momentum tensor; it
# draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
```

```
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
```

```
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 08c_frame_dependence.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 08c_frame_dependence.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
```

```
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 08c_frame_dependence.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/08c.captions.json
# - Revision/textbook/figures/08c_1_cancel_and_survive.png
# - Revision/textbook/figures/08c_2_pieces_along_history.png
# - Revision/textbook/figures/08c_3_boost_size.png
# - Revision/textbook/figures/08c_4_heat_maps.png
# - Revision/textbook/figures/08c_5_curvature_maps.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 08c_5_curvature_maps.png exists
#     ALL 32 CHECKS PASSED (notebook 08c)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/08c_frame_dependence.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
```

```
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 08c_frame_dependence.ipynb
#     python -m nbconvert --execute --inplace 08c_frame_dependence.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - A cell of sections 7, 11 or 12 runs for more than a minute: these cells simplify
#   hundreds of exact expressions with sympy; on a slow computer each can take a few
#   minutes. Wait until the star in the brackets to the left of the cell turns into a
#   number.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "08c"  # this notebook: chapter 08, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
```

```
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 08c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ ordinary space (inflating);
  $x_4$ the time; $x_5, x_6, x_7$ the three extra times (time-like, DEFLATING:
  their scale factor is $e^{-a_4}\sin^{1/6}z$); $x_0$ the hidden direction,
  $z = 6Hx_0$ between $0$ and $\pi/2$. $a_4(x_4)$ is any function of the time.
- **Metric** $g_{\mu\nu}$: the table of numbers that turns coordinate steps into
  lengths, $ds^2 = \sum g_{\mu\nu}dx_\mu dx_\nu$. The author's metric is diagonal:
  $g = \mathrm{diag}(e^{2a_4}\sin^{1/3}z\ (3\times), -1, -e^{-2a_4}\sin^{1/3}z\
```

```
  (3\times), \cot^2 z)$.
- **Frame (vielbein)** $e^a{}_\mu$: eight vectors, one per direction $a$, that are
  orthonormal for the metric: $\sum_{a} \eta_{aa}e^a{}_\mu e^a{}_\nu = g_{\mu\nu}$
  with $\eta = \mathrm{diag}(+,+,+,+,-,-,-,-)$. The *diagonal frame* has
  $e^a{}_\mu = f_a\delta^a_\mu$ with $f_a = \sqrt{|g_{aa}|}$. Its inverse is
  $E_a{}^\mu$ (written `E[mu, a]` in the code).
- **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$: the numbers that say how the
  coordinate directions turn from point to point; for a diagonal metric
  $\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\mu
  g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu})$.
- **Canonical spin connection** $\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu
  E_b{}^\nu + \Gamma^\nu{}_{\mu\lambda}E_b{}^\lambda)$: how the frame vectors turn;
  it obeys the *vielbein postulate* $\partial_\mu e^a{}_\nu -
  \Gamma^\lambda{}_{\mu\nu}e^a{}_\lambda + \omega_\mu{}^a{}_b e^b{}_\nu = 0$, and
  $\omega_{\mu ab} = \eta_{aa}\omega_\mu{}^a{}_b$ is antisymmetric in $a, b$.
- **Spinor connection** $\Omega_\mu = \frac12\sum_{a,b}\omega_{\mu ab}S^{ab}$ with
  $S^{ab} = \frac14[\gamma^{(a)}, \gamma^{(b)}]$; $D_\mu\Psi = \partial_\mu\Psi +
  \Omega_\mu\Psi$ and $\gamma^\mu = \sum_a E_a{}^\mu\gamma^{(a)}$.
- **Boost**: a change of frame that mixes a time-like and a space-like frame vector
  with $\cosh b$ and $\sinh b$ ($b$ is the *rapidity*); because $\cosh^2 b -
  \sinh^2 b = 1$ the new vectors are again orthonormal, so the new frame describes
  the SAME metric.
- **Riemann tensor** $R^\rho{}_{\sigma\mu\nu}$ (curvature) and its traces: the Ricci
  tensor $R_{\sigma\nu} = \sum_\rho R^\rho{}_{\sigma\rho\nu}$, its mixed form
  $R^\mu{}_\nu$ (for a diagonal metric $R^a{}_a = R_{aa}/g_{aa}$) and the Ricci
  scalar $R = \sum_a R^a{}_a$. A space is flat exactly when the Riemann tensor
  vanishes.
- **Spinor curvature** $F_{\mu\nu} = \partial_\mu\Omega_\nu - \partial_\nu\Omega_\mu
  + [\Omega_\mu, \Omega_\nu]$, where $[A, B] = AB - BA$ (the *commutator*); the
  record proves $F_{\mu\nu} = \frac14\sum R_{\rho\sigma\mu\nu}\gamma^\rho
  \gamma^\sigma$.
- **Anticommutator** $\{A, B\} = AB + BA$; **trace** of a matrix: the sum of its
  diagonal.
```

Text cell:

```
## 4. The physical and mathematical situation

Coupling a spinor field to gravity needs a frame, because the gamma matrices
$\gamma^{(a)}$ belong to the orthonormal frame directions, not to the coordinates.
The frame is not unique: at every point one may turn (rotate or boost) the eight
frame vectors among themselves, and the metric stays the same. The spin connection
changes under such a turn, and so may $\gamma^\mu\Omega_\mu$. A statement is a
property of the field equation only if it holds in every frame (together with the
matching change of the field, $\Psi' = S\Psi$ with a constant-or-varying $16
\times 16$ matrix $S$).

The plan: compute $\gamma^\mu\Omega_\mu$ in the diagonal frame (result
$3H\gamma^{(0)}$, sections 7 to 10), then in the boosted frame
$e'^{(4)} = \cosh b\,e^{(4)} + \sinh b\,e^{(0)}$, $e'^{(0)} = \sinh b\,e^{(4)} +
\cosh b\,e^{(0)}$, other $e'^{(a)} = e^{(a)}$, with rapidity $b = \beta x_4 + b_0$
(section 11), and finally the curvature (section 12), which no frame can remove.
```

```
Every symbol is exact: $H > 0$, $\beta$, $b_0$ are letters, and $a_4(x_4)$ is an
unspecified function; so every check holds for EVERY history $a_4$. The second
function $b_4(x_4)$ of the author's metric is put equal to 1 here, the background
of the Revision record's scope reports with which this notebook compares; Notebook
06b (Chapter 6) repeats the frame dependence for a general $b_4$, where the
diagonal frame gives $3H\gamma^{(0)} + \frac34(b_4'/b_4)\gamma^{(4)}$. Only the plots
put numbers in: $H = 1$, $z = \pi/4$, and the canonical history $a_4 = AHx_4$ with
$A = 1$ of the Kohn-Sham record (a prescribed background).
```

Text cell:

```
## 5. The metric from the diagonal frame

The next cell first defines two helpers for the checks that reproduce a Revision
record. `record_says(report, name, ...)` opens the report (a JSON file with a list
of checks, each with a name, a verdict and a detail text) and is true when the
check `name` is there with the verdict pass and its detail text contains every
further piece of text given (for example a formula that this notebook computes).
`check_record(condition, title, report, names, ...)` is the helper `check` for such
a result (`names` is one check name or a list of names): it passes only if the
notebook's own computation (`condition`) is right AND the record says the same.
Then the cell reads the gamma matrices, defines the symbols, builds the scale
factors $f_a$ and the metric $g_{aa} = \eta_{aa}f_a^2$, and checks that this is the
author's metric entry by entry (and that the record prints the same eight entries)
and that $\sqrt{|g|} = f_0 f_1 \cdots f_7 = \cos z$. The helper `is_zero` asks sympy
to simplify an expression twice (the second time with its trigonometric simplifier
`fu`) and says whether it is exactly zero.
```

In [2]:

```
import numpy as np  # numbers and matrices (for the plots)
import sympy as sp  # exact algebra with symbols

REPORTS = {}  # report file -> {check name: (verdict, detail)}, each read once


def record_says(report_file, check_name, *pieces):
    """True when the Revision report records the check check_name with the verdict
    pass and its detail text contains every given piece of text."""
    if report_file not in REPORTS:  # read the report the first time it is needed
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        REPORTS[report_file] = {entry["name"]: (entry["verdict"].lower(),
                                                entry["detail"])
                                for entry in data["checks"]}
    verdict, detail = REPORTS[report_file][check_name]
    return verdict == "pass" and all(piece in detail for piece in pieces)


def check_record(condition, title, report_file, check_names, *pieces):
    """check() for a result that reproduces Revision checks (one name or a list of
    names): it passes only if condition is true AND the report records every one
    of them as passed, with every piece of text in the detail of the first one."""
```

```
    names = [check_names] if isinstance(check_names, str) else list(check_names)
    on_record = record_says(report_file, names[0], *pieces) and all(
        record_says(report_file, name) for name in names[1:])
    word = "check" if len(names) == 1 else "checks"
    check(condition and on_record, title,
          record=f"{report_file}, {word} " + " and ".join(names))


gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
G = [sp.Matrix(rows) for rows in gammas_record["gamma"]]  # gamma^(x0) ... (x7)
ETA = gammas_record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1]
C = sp.Matrix(gammas_record["C"])  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)

x = sp.symbols("x0:8", real=True)  # the coordinates; x[k] is xk
H = sp.Symbol("H", positive=True)
beta, b0 = sp.symbols("beta b0", real=True)  # the boost: rapidity beta x4 + b0
a4 = sp.Function("a4")(x[4])  # the metric function: ANY function of x4
z = 6 * H * x[0]
sixth = sp.sin(z) ** sp.Rational(1, 6)  # sin(z)^(1/6)
f = [sp.cot(z)] + [sp.exp(a4) * sixth] * 3 + [sp.Integer(1)] \
    + [sp.exp(-a4) * sixth] * 3  # the scale factors f_0, ..., f_7 (b4 = 1)
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal entries g_aa
third = sp.sin(z) ** sp.Rational(1, 3)
authors_metric = [sp.cot(z) ** 2] + [sp.exp(2 * a4) * third] * 3 + [-1] \
    + [-sp.exp(-2 * a4) * third] * 3  # the author's metric at b4 = 1


def is_zero(expr):
    """True when sympy shows that expr is exactly zero (two simplifications)."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    simpler = sp.simplify(expr)
    return simpler == 0 or sp.simplify(sp.fu(simpler)) == 0


# The record prints each entry as sympy writes it, e.g. "g_x0x0 = cot(6*H*x0)**2".
metric_texts = [f"g_x{a}x{a} = {sp.sstr(authors_metric[a])}" for a in range(8)]
say(metric_texts[0])
say(metric_texts[4])
check_record(all(is_zero(g[a] - authors_metric[a]) for a in range(8)),
             "eta_aa f_a^2 is the author's metric (all 8 diagonal entries)",
             "Revision/theory/reports/python-field-theory.json",
             "metric_from_vielbein_equals_SPEC", *metric_texts)
check_record(is_zero(sp.prod(f) - sp.cos(z)), "sqrt|g| = f0 f1 ... f7 = cos z (b4 = 1)",
             "Revision/theory/reports/python-field-theory.json",
             "sqrt_det_g_equals_cos_z", "cos(6 H x0)")
```

Out [2]:

```
g_x0x0 = cot(6*H*x0)**2
g_x4x4 = -1
```

```
PASS eta_aa f_a^2 is the author's metric (all 8 diagonal entries)
     reproduces Revision/theory/reports/python-field-theory.json, check
    metric_from_vielbein_equals_SPEC
PASS sqrt|g| = f0 f1 ... f7 = cos z (b4 = 1)
     reproduces Revision/theory/reports/python-field-theory.json, check
    sqrt_det_g_equals_cos_z
```

Text cell:

```
## 6. The Christoffel symbols

The next cell computes all $8 \times 8 \times 8$ symbols
$\Gamma^\lambda{}_{\mu\nu}$ (stored as `Gam[lam][mu][nu]`) with the formula for a
diagonal metric, counts the independent nonzero ones (those with $\mu \le \nu$,
because $\Gamma^\lambda{}_{\mu\nu} = \Gamma^\lambda{}_{\nu\mu}$), and prints four of
them. sympy writes $a_4'$ as `Derivative(a4(x4), x4)`.
```

In [3]:

```
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for lam in range(8):
    for mu in range(8):
        for nu in range(8):
            value = 0  # the bracket of the formula; g is diagonal, so only these
            if lam == nu:  # three cases contribute
                value += sp.diff(g[lam], x[mu])
            if lam == mu:
                value += sp.diff(g[lam], x[nu])
            if mu == nu:
                value -= sp.diff(g[mu], x[lam])
            if value != 0:
                Gam[lam][mu][nu] = sp.simplify(value / (2 * g[lam]))
independent = sum(1 for lam in range(8) for mu in range(8) for nu in range(mu, 8)
                  if Gam[lam][mu][nu] != 0)
report("independent nonzero Christoffel symbols", independent)
for lam, mu, nu in ((1, 1, 4), (5, 4, 5), (1, 0, 1), (0, 0, 0)):
    say(f"Gamma^x{lam}_x{mu} x{nu} = {Gam[lam][mu][nu]}")
check_record(independent == 25, "25 independent nonzero Christoffel symbols",
             "Revision/theory/reports/python-field-theory.json",
             "christoffel_symmetric_metric_compatible",
             f"{independent} independent nonzero symbols")
```

Out [3]:

```
RESULT independent nonzero Christoffel symbols = 25
Gamma^x1_x1 x4 = Derivative(a4(x4), x4)
Gamma^x5_x4 x5 = -Derivative(a4(x4), x4)
Gamma^x1_x0 x1 = H/tan(6*H*x0)
Gamma^x0_x0 x0 = -12*H/sin(12*H*x0)
PASS 25 independent nonzero Christoffel symbols
     reproduces Revision/theory/reports/python-field-theory.json, check
    christoffel_symmetric_metric_compatible
```

Text cell:

```
## 7. The canonical spin connection of the diagonal frame

The function ‘frame(b)‘ below builds the frame boosted by the rapidity $b$ (for
$b = 0$ the diagonal frame), its inverse, the mixed connection
$\omega_\mu{}^a{}_c$ (stored as ‘omega[mu, a, c]‘), the spinor connections
$\Omega_\mu$ and the matrices $\gamma^\mu$. ‘postulate_failures‘ counts the
components of the vielbein postulate that are NOT zero. The next cell applies them
to the diagonal frame, checks the postulate (512 components) and the antisymmetry,
and prints the 12 independent nonzero components $\omega_{\mu ab}$ ($a < b$).
```

In [4]:

```
S_AB = [[(G[a] * G[c] - G[c] * G[a]) / 4 for c in range(8)] for a in range(8)]


def boost(b):
    """The boost by the rapidity b in the frame plane (x4, x0), and its inverse."""
    L, L_inverse = sp.eye(8), sp.eye(8)
    L[4, 4] = L[0, 0] = L_inverse[4, 4] = L_inverse[0, 0] = sp.cosh(b)
    L[4, 0] = L[0, 4] = sp.sinh(b)
    L_inverse[4, 0] = L_inverse[0, 4] = -sp.sinh(b)
    return L, L_inverse


def frame(b):
    """Vielbein e[a, mu], inverse E[mu, a], connection omega[mu, a, c] (mixed),
    Omega_mu and gamma^mu of the diagonal frame boosted by the rapidity b."""
    L, L_inverse = boost(b)
    e = L * sp.diag(*f)
    E = sp.diag(*[1 / value for value in f]) * L_inverse
    omega = {}
    for mu in range(8):
        for a in range(8):
            for c in range(8):
                value = 0  # e^a_nu (d_mu E_c^nu + Gamma^nu_mu,lam E_c^lam)
                for nu in range(8):
                    if e[a, nu] == 0:
                        continue  # skip the terms that are zero anyway
                    term = sp.diff(E[nu, c], x[mu])
                    for lam in range(8):
                        if Gam[nu][mu][lam] != 0 and E[lam, c] != 0:
                            term += Gam[nu][mu][lam] * E[lam, c]
                    value += e[a, nu] * term
                omega[mu, a, c] = sp.simplify(value)
    Omega = []
    for mu in range(8):
        matrix = sp.zeros(16, 16)
        for a in range(8):
            for c in range(a + 1, 8):  # (1/2) sum over all a, c = sum over a < c
                matrix += ETA[a] * omega[mu, a, c] * S_AB[a][c]
```

```
        Omega.append(matrix)
    gamma_up = [sum((E[mu, a] * G[a] for a in range(8)), sp.zeros(16, 16))
                for mu in range(8)]
    return {"e": e, "E": E, "omega": omega, "Omega": Omega, "gamma": gamma_up}


def postulate_failures(fr):
    """How many of the 512 components of the vielbein postulate are not zero."""
    failures = 0
    for a in range(8):
        for mu in range(8):
            for nu in range(8):
                value = sp.diff(fr["e"][a, nu], x[mu]) \
                    - sum(Gam[lam][mu][nu] * fr["e"][a, lam] for lam in range(8)) \
                    + sum(fr["omega"][mu, a, c] * fr["e"][c, nu] for c in range(8))
                failures += 0 if is_zero(value) else 1
    return failures


def antisymmetric(fr):
    """omega_mu,ac = eta_aa omega_mu^a_c is antisymmetric in a, c for every mu."""
    return all(is_zero(ETA[a] * fr["omega"][mu, a, c] + ETA[c] * fr["omega"][mu, c, a])
               for mu in range(8) for a in range(8) for c in range(8))


diagonal = frame(sp.Integer(0))
check_record(postulate_failures(diagonal) == 0 and antisymmetric(diagonal),
             "diagonal frame: vielbein postulate (512 components), omega antisymmetric",
             "Revision/theory/reports/python-field-theory.json",
             ["vielbein_postulate", "spin_connection_antisymmetric"],
             "(512 components)")
nonzero = [(mu, a, c) for mu in range(8) for a in range(8) for c in range(a + 1, 8)
           if not is_zero(diagonal["omega"][mu, a, c])]
omega_diagonal = diagonal["omega"]
for mu, a, c in nonzero:
    say(f"omega_x{mu},(x{a})(x{c}) = "
        f"{sp.simplify(ETA[a] * omega_diagonal[mu, a, c])}")
check_record(len(nonzero) == 12, "12 independent nonzero components omega_mu,ab (a < b)",
             "Revision/theory/reports/wolfram-field-theory.json", "omega_components",
             f"exactly {len(nonzero)} independent nonzero omega_mu,ab "
             "(a < b in the order x0..x7)")
```

Out [4]:

```
PASS diagonal frame: vielbein postulate (512 components), omega antisymmetric
     reproduces Revision/theory/reports/python-field-theory.json, checks
    vielbein_postulate and spin_connection_antisymmetric
omega_x1,(x0)(x1) = -H*exp(a4(x4))*sin(6*H*x0)**(1/6)
omega_x1,(x1)(x4) = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x2,(x0)(x2) = -H*exp(a4(x4))*sin(6*H*x0)**(1/6)
omega_x2,(x2)(x4) = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x3,(x0)(x3) = -H*exp(a4(x4))*sin(6*H*x0)**(1/6)
omega_x3,(x3)(x4) = exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
```

```
omega_x5,(x0)(x5) = H*exp(-a4(x4))*sin(6*H*x0)**(1/6)
omega_x5,(x4)(x5) = -exp(-a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x6,(x0)(x6) = H*exp(-a4(x4))*sin(6*H*x0)**(1/6)
omega_x6,(x4)(x6) = -exp(-a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
omega_x7,(x0)(x7) = H*exp(-a4(x4))*sin(6*H*x0)**(1/6)
omega_x7,(x4)(x7) = -exp(-a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)
PASS 12 independent nonzero components omega_mu,ab (a < b)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check omega_components
```

Text cell:

```
## 8. The connection vanishes only in flat space: non-triviality

Every one of the 12 components printed above is $a_4'$ times, or $H$ times, one of
the four factors $\pm e^{\pm a_4}\sin^{1/6}z$, and these factors are never zero for
$0 < z < \pi/2$ (an exponential is never zero, and $\sin z > 0$ there). The 28
matrices $S^{ab}$ ($a < b$) are linearly independent: written as rows of
$16 \times 16 = 256$ numbers they have rank 28 (rank: the number of independent
rows). So $\Omega_\mu = \frac12\sum\omega_{\mu ab}S^{ab}$ is zero exactly when all
its coefficients are. The next cell finds, for each of the 12 components, which of
the two shapes it has (it writes `A1` for $a_4'$, the name the Revision record
uses), counts six of each, computes the rank of the $S^{ab}$, and so checks, line
by line:

1. each component is $a_4'\cdot(\text{factor})$ or $H\cdot(\text{factor})$, six of
   each, with a factor that is never zero;
2. if $a_4' \neq 0$, the six components of the first kind are not zero; if
   $H \neq 0$, the six of the second kind are not zero;
3. so $\Omega_\mu = 0$ for every $\mu$ if and only if $a_4' = 0$ AND $H = 0$, the
   formal flat limit. $H = 0$ is not a member of the author's family (the metric
   degenerates there), so for the author's metric the connection never vanishes.

This is the core of the non-triviality statements [1] and [2] of the Revision
record.
```

In [5]:

```
a4_prime = sp.diff(a4, x[4])  # a4' = d a4 / d x4
A1 = sp.Symbol("A1", real=True)  # a letter that stands for a4'
allowed = [sign * sp.exp(power * a4) * sixth for sign in (1, -1) for power in (1, -1)]
kinds = []  # for each component: "A1" (a4' times a factor) or "H" (H times a factor)
for mu, a, c in nonzero:
    value = sp.simplify(ETA[a] * omega_diagonal[mu, a, c]).subs(a4_prime, A1)
    found = [str(coupling) for coupling in (A1, H) for factor in allowed
             if is_zero(value - coupling * factor)]  # which shape fits
    kinds.append(found[0] if len(found) == 1 else "no shape")
n_A1, n_H = kinds.count("A1"), kinds.count("H")
report("components of the form a4' x factor and H x factor", f"{n_A1}, {n_H}")
S_rows = np.array([[float(v) for v in S_AB[a][c]] for a in range(8)
                   for c in range(a + 1, 8)])  # 28 rows of 256 numbers
rank_S = int(np.linalg.matrix_rank(S_rows))
report("rank of the 28 matrices S^ab written as rows of 256 numbers", rank_S)
```

```
# a4' != 0 makes the six "A1" components nonzero, H != 0 the six "H" components;
# with independent S^ab, Omega vanishes for every mu only when a4' = 0 and H = 0.
check_record(n_A1 == 6 and n_H == 6 and rank_S == 28,
             "Omega_mu = 0 for every mu iff a4' = 0 and H = 0 (formal flat limit)",
             "Revision/theory/reports/python-field-theory.json",
             "nontriviality_Omega_zero_iff_flat",
             "the S^ab are linearly independent",
             "every Omega_mu vanishes iff a4' = 0 AND H = 0")
```

Out [5]:

```
RESULT components of the form a4' x factor and H x factor = 6, 6
RESULT rank of the 28 matrices S^ab written as rows of 256 numbers = 28
PASS Omega_mu = 0 for every mu iff a4' = 0 and H = 0 (formal flat limit)
     reproduces Revision/theory/reports/python-field-theory.json, check
    nontriviality_Omega_zero_iff_flat
```

Text cell:

```
## 9. What cancels and what survives

For each direction $\mu$ separately (no sum) the product $\gamma^{x_\mu}
\Omega_{x_\mu}$ turns out to be a combination of $\gamma^{(4)}$ and $\gamma^{(0)}$
only. The coefficient of $\gamma^{(0)}$ in a matrix $M$ is $\mathrm{tr}(\gamma^{(0)}
M)/16$ and that of $\gamma^{(4)}$ is $-\mathrm{tr}(\gamma^{(4)}M)/16$ (because
$(\gamma^{(0)})^2 = 1$, $(\gamma^{(4)})^2 = -1$, and the trace of a product of two
different gammas is 0). The next cell computes both coefficients for every
direction, checks that nothing else is left, checks the sum
$\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$, the *divergence form*
$\frac{1}{2\sqrt{|g|}}\sum_\mu\partial_\mu(\sqrt{|g|}\gamma^\mu) = 3H\gamma^{(0)}$,
and $\{\gamma^\mu, \Omega_\mu\} = 0$ for each $\mu$ (which makes the connection
drop out of the symmetrised Lagrangian), and draws the coefficients. It prints the
three partial sums in the record's notation (`A1` for $a_4'$) and compares them
with the record's text. It also checks $(\gamma^{(0)})^2 = I_{16}$: then
$\gamma^{(0)}$ can be undone (multiply by $\gamma^{(0)}$ again), so
$3H\gamma^{(0)}\Psi = 0$ only when $\Psi = 0$; in the diagonal frame the
gravitational term is not zero for any field that is not zero (the statement [1]
of the Revision record, whose scope sections 11 and 12 below make precise).
```

In [6]:

```
per_direction = [(diagonal["gamma"][mu] * diagonal["Omega"][mu]).applyfunc(sp.simplify)
                 for mu in range(8)]
coefficient_4 = [sp.simplify(-(G[4] * M).trace() / 16) for M in per_direction]
coefficient_0 = [sp.simplify((G[0] * M).trace() / 16) for M in per_direction]
nothing_else = all(
    all(is_zero(entry) for entry in
        per_direction[mu] - coefficient_4[mu] * G[4] - coefficient_0[mu] * G[0])
    for mu in range(8))
for mu in range(8):
    say(f"gamma^x{mu} Omega_x{mu} = ({coefficient_4[mu]}) gamma^(x4) "
```

```
        f"+ ({coefficient_0[mu]}) gamma^(x0)")
check(nothing_else, "each gamma^mu Omega_mu (no sum) is a combination of gamma^(x4) "
      "and gamma^(x0)")
THEORY = "Revision/theory/reports/python-field-theory.json"  # the sympy record
# The record writes A1 for a4'; the partial sums in the record's notation:
inflating_sum = sp.simplify(sum(coefficient_4[1:4])).subs(a4_prime, A1)  # 3*A1/2
deflating_sum = sp.simplify(sum(coefficient_4[5:8])).subs(a4_prime, A1)  # -3*A1/2
hidden_sum = sp.simplify(sum(coefficient_0))  # 3*H
say(f"sums: inflating {inflating_sum}, deflating {deflating_sum}, hidden {hidden_sum}")
check_record([sp.simplify(c4 / a4_prime) for c4 in coefficient_4]
             == [0] + [sp.Rational(1, 2)] * 3 + [0] + [-sp.Rational(1, 2)] * 3
             and [sp.simplify(c0 / H) for c0 in coefficient_0]
             == [0] + [sp.Rational(1, 2)] * 3 + [0] + [sp.Rational(1, 2)] * 3,
             "per direction: +a4'/2 and -a4'/2 cancel, six times H/2 add up",
             THEORY, "time_terms_cancel_hidden_term_survives",
             f"inflating sum {inflating_sum}, deflating sum {deflating_sum}",
             f"the six hidden-direction terms add to {hidden_sum} gamma^(x0)")
total = sum(per_direction, sp.zeros(16, 16))
check_record(all(is_zero(entry) for entry in total - 3 * H * G[0]),
             "gamma^mu Omega_mu = 3 H gamma^(x0) in the diagonal frame",
             THEORY, "gamma_mu_Omega_mu_equals_3H_gamma_x0",
             "gamma^mu Omega_mu = 3 H gamma^(x0) exactly")
# gamma^(x0) squares to 1, so it is invertible: 3 H gamma^(x0) Psi = 0 only for
# Psi = 0. The gravitational term is not zero for any field Psi that is not zero.
check_record(G[0] * G[0] == sp.eye(16),
             "(gamma^(x0))^2 = 1: 3 H gamma^(x0) Psi != 0 for every Psi != 0 (H > 0)",
             "Revision/theory/reports/wolfram-field-theory.json",
             "nontriviality_1_dirac16complex",
             "every Psi != 0 (gamma^(x0) is invertible)")
sqrt_g = sp.cos(z)  # sqrt|g| = b4^(3/2) cos z, at b4 = 1
divergence = sum((sp.diff(sqrt_g * diagonal["gamma"][mu], x[mu]) for mu in range(8)),
                 sp.zeros(16, 16)) / (2 * sqrt_g)
check_record(all(is_zero(entry) for entry in divergence - 3 * H * G[0]),
             "(1/(2 sqrt|g|)) d_mu(sqrt|g| gamma^mu) = 3 H gamma^(x0) (divergence "
             "form)", THEORY, "divergence_of_sqrtg_gamma",
             "= 3 H gamma^(x0) = gamma^mu Omega_mu")
check_record(all(all(is_zero(entry) for entry in
                     diagonal["gamma"][mu] * diagonal["Omega"][mu]
                     + diagonal["Omega"][mu] * diagonal["gamma"][mu])
                 for mu in range(8)),
             "{gamma^mu, Omega_mu} = 0 for each mu: Omega drops out of the Lagrangian",
             "Revision/theory/reports/python-scope.json",
             "connection_free_lagrangian_same_equations",
             "{gamma^mu, Omega_mu} = 0 for each mu separately")

names = [f"$x_{mu}$" for mu in range(8)] + ["sum"]
values_4 = [float(sp.simplify(c4 / a4_prime)) for c4 in coefficient_4]
values_0 = [float(sp.simplify(c0 / H)) for c0 in coefficient_0]
values_4.append(sum(values_4))
values_0.append(sum(values_0))
colours = ["gray"] + ["tab:blue"] * 3 + ["gray"] + ["tab:red"] * 3 + ["black"]
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), sharey=True)
left.bar(names, values_4, color=colours)
left.set_title("coefficient of $\\gamma^{(4)}$, in units of $a_4'$")
```

```
left.set_ylabel("coefficient")
right.bar(names, values_0, color=colours)
right.set_title("coefficient of $\\gamma^{(0)}$, in units of $H$")
for panel in (left, right):
    panel.axhline(0.0, color="black", linewidth=0.8)
    panel.set_xlabel("direction $\\mu$ of $\\gamma^{\\mu}\\Omega_{\\mu}$")
save_figure(fig, "cancel_and_survive",
            "The term $\\gamma^\\mu\\Omega_\\mu$ of the field equation in the "
            "diagonal frame, direction by direction (no sum) and summed (black). "
            "Left: the coefficient of $\\gamma^{(4)}$ in units of $a_4'$; the three "
            "inflating directions $x_1, x_2, x_3$ (blue) give $+1/2$ each, the three "
            "deflating extra times $x_5, x_6, x_7$ (red) give $-1/2$ each, $x_4$ and "
            "$x_0$ (gray) give 0, and the sum is 0. Right: the coefficient of "
            "$\\gamma^{(0)}$ in units of $H$; the same six directions give $+1/2$ "
            "each and the sum is $3$. Exact values from sympy, valid for every "
            "history $a_4$ and every $H > 0$, on the background $b_4 = 1$.")
```

Out [6]:

```
gamma^x0 Omega_x0 = (0) gamma^(x4) + (0) gamma^(x0)
gamma^x1 Omega_x1 = (Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x2 Omega_x2 = (Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x3 Omega_x3 = (Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x4 Omega_x4 = (0) gamma^(x4) + (0) gamma^(x0)
gamma^x5 Omega_x5 = (-Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x6 Omega_x6 = (-Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
gamma^x7 Omega_x7 = (-Derivative(a4(x4), x4)/2) gamma^(x4) + (H/2) gamma^(x0)
PASS each gamma^mu Omega_mu (no sum) is a combination of gamma^(x4) and gamma^(x0)
sums: inflating 3*A1/2, deflating -3*A1/2, hidden 3*H
PASS per direction: +a4'/2 and -a4'/2 cancel, six times H/2 add up
     reproduces Revision/theory/reports/python-field-theory.json, check
    time_terms_cancel_hidden_term_survives
PASS gamma^mu Omega_mu = 3 H gamma^(x0) in the diagonal frame
     reproduces Revision/theory/reports/python-field-theory.json, check
    gamma_mu_Omega_mu_equals_3H_gamma_x0
PASS (gamma^(x0))^2 = 1: 3 H gamma^(x0) Psi != 0 for every Psi != 0 (H > 0)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    nontriviality_1_dirac16complex
PASS (1/(2 sqrt|g|)) d_mu(sqrt|g| gamma^mu) = 3 H gamma^(x0) (divergence form)
     reproduces Revision/theory/reports/python-field-theory.json, check
    divergence_of_sqrtg_gamma
PASS {gamma^mu, Omega_mu} = 0 for each mu: Omega drops out of the Lagrangian
     reproduces Revision/theory/reports/python-scope.json, check
    connection_free_lagrangian_same_equations
```

Out [6]:

```
Figure 08c.1 saved as Revision/textbook/figures/08c_1_cancel_and_survive.png
```

Text cell:

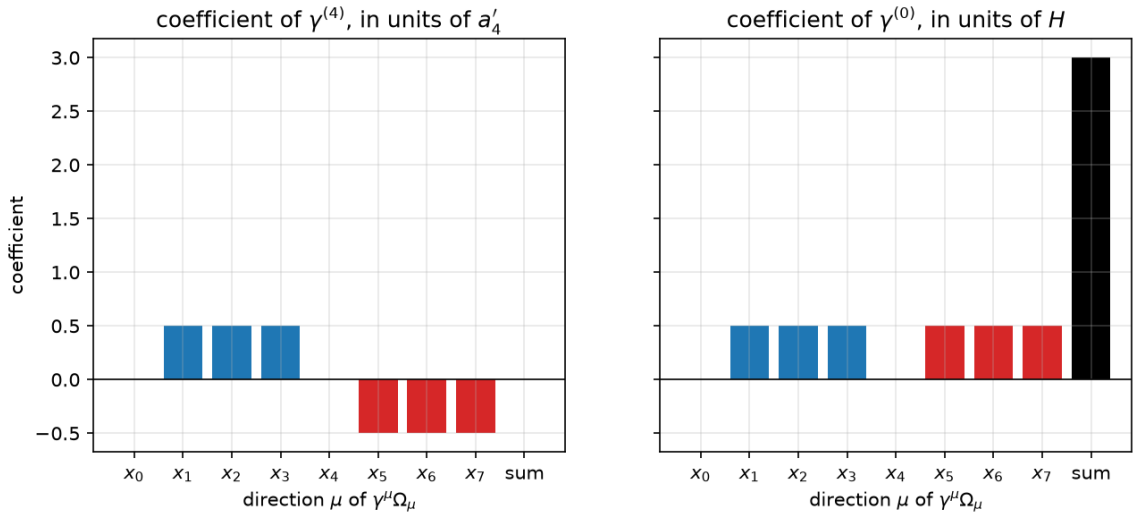

Figure 229: The term $\gamma^\mu\Omega_\mu$ of the field equation in the diagonal frame, direction by direction (no sum) and summed (black). Left: the coefficient of $\gamma^{(4)}$ in units of a_4'; the three inflating directions x_1, x_2, x_3 (blue) give $+1/2$ each, the three deflating extra times x_5, x_6, x_7 (red) give $-1/2$ each, x_4 and x_0 (gray) give 0, and the sum is 0. Right: the coefficient of $\gamma^{(0)}$ in units of H; the same six directions give $+1/2$ each and the sum is 3. Exact values from sympy, valid for every history a_4 and every $H > 0$, on the background $b_4 = 1$. (Notebook 08c, figure 1.)

```
## 10. The deflation is invisible in this term, but not in the curvature

The term $3H\gamma^{(0)}$ contains no $a_4$: the deflation of the extra times does
not show up in it. The geometry of the $a_4$ sector is nevertheless curved. The
next cell computes the Riemann tensor from the Christoffel symbols,
$R^\rho{}_{\sigma\mu\nu} = \partial_\mu\Gamma^\rho{}_{\nu\sigma} -
\partial_\nu\Gamma^\rho{}_{\mu\sigma} + \sum_\lambda(\Gamma^\rho{}_{\mu\lambda}
\Gamma^\lambda{}_{\nu\sigma} - \Gamma^\rho{}_{\nu\lambda}\Gamma^\lambda{}_{\mu\sigma})$,
and the diagonal Ricci components $R^{x_a}{}_{x_a}$, and checks them against the
record: $a_4'' - 6H^2$ (3-space), $6(a_4')^2$ (time), $-a_4'' - 6H^2$ (extra
times), $-6H^2$ (hidden direction), and $R = 6((a_4')^2 - 7H^2)$. Then it draws,
along the canonical history $a_4 = x_4$ at $z = \pi/4$ and $H = 1$, two pieces of
the connection (they change exponentially) and the two partial sums of the
$\gamma^{(4)}$ coefficients (they stay constant and cancel).
```

In [7]:

```
def riemann(rho, sigma, mu, nu):
    """R^rho_sigma,mu,nu from the Christoffel symbols (not simplified)."""
    value = sp.diff(Gam[rho][nu][sigma], x[mu]) - sp.diff(Gam[rho][mu][sigma], x[nu])
    for lam in range(8):
        value += Gam[rho][mu][lam] * Gam[lam][nu][sigma] \
            - Gam[rho][nu][lam] * Gam[lam][mu][sigma]
    return value


ricci = [sum(riemann(rho, a, rho, a) for rho in range(8)) / g[a] for a in range(8)]
```

```
a4_second = sp.diff(a4, x[4], 2)
expected = [-6 * H**2] + [a4_second - 6 * H**2] * 3 + [6 * a4_prime**2] \
    + [-a4_second - 6 * H**2] * 3
check_record(all(is_zero(ricci[a] - expected[a]) for a in range(8)),
             "R^x1_x1 = a4'' - 6H^2, R^x4_x4 = 6 a4'^2, R^x5_x5 = -a4'' - 6H^2, "
             "R^x0_x0 = -6H^2", "Revision/theory/reports/wolfram-field-theory.json",
             "ricci_mixed_components", "R^x0_x0 = -6 H^2",
             "R^x1_x1 = R^x2_x2 = R^x3_x3 = a4'' - 6 H^2", "R^x4_x4 = 6 a4'^2",
             "R^x5_x5 = R^x6_x6 = R^x7_x7 = -a4'' - 6 H^2")
ricci_scalar = sp.factor(sp.simplify(sum(ricci)).subs(a4_prime, A1))  # A1 for a4'
say(f"Ricci scalar in the record's notation: R = {ricci_scalar}")
check_record(is_zero(sum(ricci) - 6 * (a4_prime**2 - 7 * H**2)),
             "Ricci scalar R = 6 (a4'^2 - 7 H^2): curved for every H > 0",
             THEORY, "curvature_nonzero_flat_only_formally",
             f"Ricci scalar R = {sp.sstr(ricci_scalar)}")
check_record(not total.applyfunc(sp.simplify).has(a4) and not is_zero(ricci[4]),
             "gamma^mu Omega_mu contains no a4, although R^x4_x4 = 6 a4'^2 is not zero",
             "Revision/theory/reports/python-scope.json",
             "gammaOmega_blind_to_the_deflation",
             "contains neither e^a4 nor a4' nor a4''", "R^x4_x4 = 6 a4'^2")

parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
A_history = parameters["physics"]["historyA"]  # the canonical history a4 = A H x4
history = sp.Lambda(x[4], A_history * x[4])  # with H = 1: a4 = x4


def along_history(expr):
    """A numpy function of x4 for expr with H = 1, a4 = A x4, z = pi/4."""
    concrete = expr.replace(sp.Function("a4"), history).doit()
    concrete = concrete.subs({H: 1, x[0]: sp.pi / 24})  # z = 6 x0 = pi/4
    return sp.lambdify(x[4], concrete, "numpy")


times = np.linspace(0.0, 4.0, 201)
pieces = {"$\\omega_{x_1,(1)(4)} = a_4'e^{a_4}\\sin^{1/6}z$": diagonal["omega"][1, 1, 4],
          "$\\omega_{x_1,(1)(0)} = He^{a_4}\\sin^{1/6}z$": diagonal["omega"][1, 1, 0],
          "$|\\omega_{x_5,(4)(5)}| = a_4'e^{-a_4}\\sin^{1/6}z$":
              -ETA[4] * diagonal["omega"][5, 4, 5],
          "$|\\omega_{x_5,(5)(0)}| = He^{-a_4}\\sin^{1/6}z$":
              ETA[5] * diagonal["omega"][5, 5, 0]}
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
for (label, expr), style in zip(pieces.items(), ("-", "--", "-", "--")):
    left.plot(times, np.abs(along_history(expr)(times)) * np.ones_like(times),
              style, label=label)
left.set_yscale("log")
left.set_xlabel("time $x_4$")
left.set_ylabel("size of the component")
left.set_title("Pieces of the connection")
left.legend(fontsize=7)
inflating = along_history(sum(coefficient_4[1:4]))(times) * np.ones_like(times)
deflating = along_history(sum(coefficient_4[5:8]))(times) * np.ones_like(times)
right.plot(times, inflating, color="tab:blue", label="$x_1, x_2, x_3$: $+3a_4'/2$")
right.plot(times, deflating, color="tab:red", label="$x_5, x_6, x_7$: $-3a_4'/2$")
```

```
right.plot(times, inflating + deflating, color="black", label="sum: $0$")
right.set_ylim(-2.0, 2.0)
right.set_xlabel("time $x_4$")
right.set_ylabel("coefficient of $\\gamma^{(4)}$")
right.set_title("Their contraction: constant, cancels")
right.legend(fontsize=8);
save_figure(fig, "pieces_along_history",
            "Along the canonical history $a_4 = x_4$ of the Kohn-Sham record ($A = "
            "H = 1$, a prescribed background) at $z = \\pi/4$, against the time "
            "$x_4$. Left, on a logarithmic axis: four components of the canonical "
            "spin connection of the diagonal frame; those of the inflating "
            "direction $x_1$ grow like $e^{a_4}$, those of the deflating extra time "
            "$x_5$ shrink like $e^{-a_4}$; the dashed ones (factor $H$) lie on the "
            "solid ones (factor $a_4'$) because here $a_4' = AH = H = 1$. Right: "
            "the coefficient of $\\gamma^{(4)}$ in $\\gamma^\\mu\\Omega_\\mu$ summed "
            "over the three inflating "
            "directions (blue, $+3a_4'/2$) and over the three extra times (red, "
            "$-3a_4'/2$): the frame factors cancel the exponentials, both partial "
            "sums stay constant, and they cancel exactly (black).")
```

Out [7]:

```
PASS R^x1_x1 = a4'' - 6H^2, R^x4_x4 = 6 a4'^2, R^x5_x5 = -a4'' - 6H^2, R^x0_x0 = -6H^2
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    ricci_mixed_components
Ricci scalar in the record's notation: R = 6*(A1**2 - 7*H**2)
PASS Ricci scalar R = 6 (a4'^2 - 7 H^2): curved for every H > 0
     reproduces Revision/theory/reports/python-field-theory.json, check
    curvature_nonzero_flat_only_formally
PASS gamma^mu Omega_mu contains no a4, although R^x4_x4 = 6 a4'^2 is not zero
     reproduces Revision/theory/reports/python-scope.json, check
    gammaOmega_blind_to_the_deflation
```

Out [7]:

```
Figure 08c.2 saved as Revision/textbook/figures/08c_2_pieces_along_history.png
```

Text cell:

```
## 11. The boosted frame: the term depends on the frame

The next cell builds the frame boosted by the rapidity $b = \beta x_4 + b_0$ with
symbolic $\beta$ and $b_0$. It checks that the new frame gives the SAME metric
($\sum_a\eta_{aa}e'^a{}_\mu e'^a{}_\nu = g_{\mu\nu}$ for all 64 pairs) and that its
canonical connection obeys the vielbein postulate, and then that
$\gamma'^\mu\Omega'_\mu = \frac{6H - \beta}{2}(\cosh b\,\gamma^{(0)} -
\sinh b\,\gamma^{(4)})$ exactly. For $\beta = 6H$ this is ZERO for every $H$ and
every $a_4$, although the spinor connections $\Omega'_\mu$ themselves are not zero
(for $\mu = x_1, \dots, x_7$). The 28 matrices $S^{ab}$ ($a < b$) are linearly
independent (rank 28, checked in section 8), so $\Omega'_\mu$ is zero exactly when
all its coefficients $\omega'_{\mu ab}$ are. Every check also compares with the
```

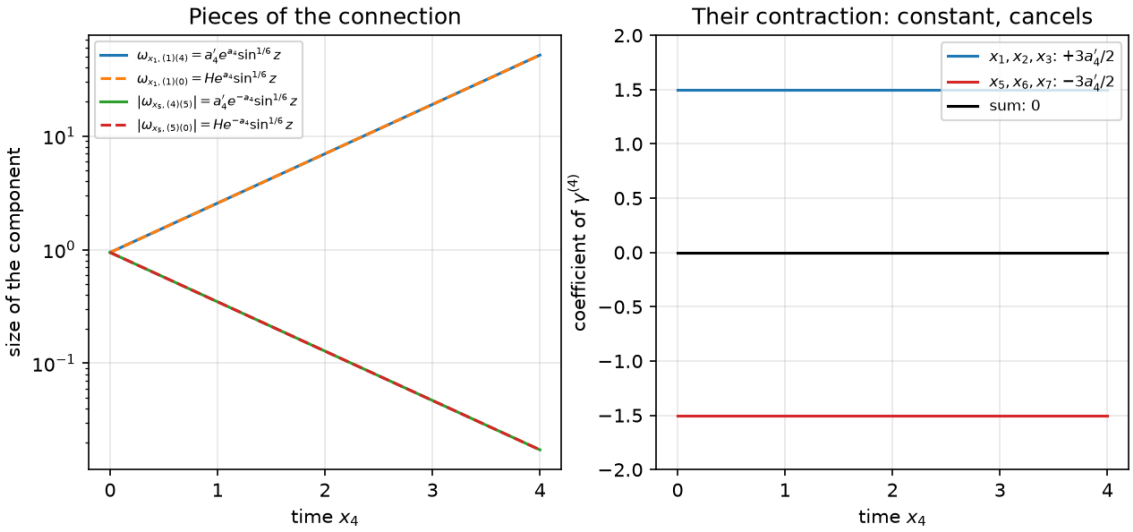

Figure 230: Along the canonical history $a_4 = x_4$ of the Kohn-Sham record ($A = H = 1$, a prescribed background) at $z = \pi/4$, against the time x_4. Left, on a logarithmic axis: four components of the canonical spin connection of the diagonal frame; those of the inflating direction x_1 grow like e^{a_4}, those of the deflating extra time x_5 shrink like e^{-a_4}; the dashed ones (factor H) lie on the solid ones (factor a_4') because here $a_4' = AH = H = 1$. Right: the coefficient of $\gamma^{(4)}$ in $\gamma^\mu \Omega_\mu$ summed over the three inflating directions (blue, $+3a_4'/2$) and over the three extra times (red, $-3a_4'/2$): the frame factors cancel the exponentials, both partial sums stay constant, and they cancel exactly (black). (Notebook 08c, figure 2.)

```
text of the Revision record (for example the formula of
$\gamma'^\mu\Omega'_\mu$ and the list of the directions with $\Omega'_\mu \neq 0$).
```

In [8]:

```
rapidity = beta * x[4] + b0
boosted = frame(rapidity)
e_b = boosted["e"]
same_metric = all(
    is_zero(sum(ETA[a] * e_b[a, mu] * e_b[a, nu] for a in range(8))
            - (g[mu] if mu == nu else 0)) for mu in range(8) for nu in range(8))
SCOPE = "Revision/theory/reports/python-scope.json"  # the record of these checks
check_record(same_metric and all(is_zero(v) for v in e_b * boosted["E"] - sp.eye(8)),
             "the boosted frame gives the same metric (64 entries); e' E' = 1",
             SCOPE, "boosted_frame_reproduces_metric",
             "equals the author's metric for all 64 (mu, nu)")
failures = postulate_failures(boosted)  # the number of nonzero components
check_record(failures == 0 and antisymmetric(boosted),
             "boosted frame: vielbein postulate (512 components), omega' antisymmetric",
             SCOPE, "boosted_frame_canonical_connection",
             f"holds for all 512 (a, mu, nu) ({failures} failures)")
total_boosted = sum((boosted["gamma"][mu] * boosted["Omega"][mu] for mu in range(8)),
                    sp.zeros(16, 16))
formula = (6 * H - beta) / 2 * (sp.cosh(rapidity) * G[0] - sp.sinh(rapidity) * G[4])
check_record(all(is_zero(entry) for entry in total_boosted - formula),
             "gamma'^mu Omega'_mu = ((6H - beta)/2) (cosh b gamma^(x0) - sinh b "
             "gamma^(x4))", SCOPE, "boosted_frame_gammaOmega_formula",
             "gamma'^mu Omega'_mu = ((6 H - beta)/2) (cosh b gamma^(x0) - sinh b "
             "gamma^(x4))")
at_6H = {beta: 6 * H}
nonzero_Omega = [mu for mu in range(8) if not all(
    is_zero(ETA[a] * boosted["omega"][mu, a, c].subs(at_6H))
    for a in range(8) for c in range(a + 1, 8))]
nonzero_text = ", ".join(f"x{mu}" for mu in nonzero_Omega)  # "x1, x2, ..."
say("beta = 6H: Omega'_mu is nonzero for mu = " + nonzero_text)
check_record(all(is_zero(entry.subs(at_6H)) for entry in total_boosted - formula)
             and formula.subs(at_6H) == sp.zeros(16, 16)
             and nonzero_Omega == [1, 2, 3, 4, 5, 6, 7],
             "beta = 6H: gamma'^mu Omega'_mu = 0 identically, Omega'_mu != 0 for "
             "x1..x7", SCOPE, "boosted_frame_gammaOmega_vanishes", "IDENTICALLY ZERO",
             f"nonzero for mu = {nonzero_text} (zero only for x0)")
```

Out [8]:

```
PASS the boosted frame gives the same metric (64 entries); e' E' = 1
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_reproduces_metric
PASS boosted frame: vielbein postulate (512 components), omega' antisymmetric
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_canonical_connection
PASS gamma'^mu Omega'_mu = ((6H - beta)/2) (cosh b gamma^(x0) - sinh b gamma^(x4))
     reproduces Revision/theory/reports/python-scope.json, check
```

```
    boosted_frame_gammaOmega_formula
beta = 6H: Omega'_mu is nonzero for mu = x1, x2, x3, x4, x5, x6, x7
PASS beta = 6H: gamma'^mu Omega'_mu = 0 identically, Omega'_mu != 0 for x1..x7
     reproduces Revision/theory/reports/python-scope.json, check
    boosted_frame_gammaOmega_vanishes
```

Text cell:

```
The next cell draws the size of the term, $\lVert M\rVert = \sqrt{\mathrm{tr}
(M^TM)/16}$ (for $M = \gamma^{(0)}$ this is 1), for $H = 1$: on the left against
$\beta$ at $x_4 = 0$, $b_0 = 0$; on the right against $x_4$ for four values of
$\beta$. The cell evaluates the formula, which the previous cell showed to be
equal, entry by entry, to the computed matrix $\gamma'^\mu\Omega'_\mu$.
```

In [9]:

```
term = sp.lambdify((beta, x[4], b0), formula.subs(H, 1), "numpy")


def size(matrix):
    """sqrt(tr(M^T M)/16): the size of a 16 x 16 matrix, 1 for gamma^(x0)."""
    matrix = np.array(matrix, dtype=float)
    return np.sqrt(np.trace(matrix.T @ matrix) / 16)


beta_values = np.linspace(0.0, 12.0, 241)
sizes_beta = [size(term(bv, 0.0, 0.0)) for bv in beta_values]
check(abs(size(term(0.0, 0.0, 0.0)) - 3.0) < 1e-12 and size(term(6.0, 0.7, 0.3)) == 0,
      "size 3 in the diagonal frame (3 H gamma^(x0), H = 1), 0 for beta = 6H")
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(beta_values, sizes_beta)
left.plot([0.0, 6.0], [3.0, 0.0], "o", color="black")
left.annotate("diagonal frame: $3H\\gamma^{(0)}$", (0.0, 3.0), (1.0, 3.3))
left.annotate("$\\beta = 6H$: zero", (6.0, 0.0), (6.6, 0.4))
left.set_xlabel("rapidity rate $\\beta$ (units of $H$)")
left.set_ylabel("size of $\\gamma'^\\mu\\Omega'_\\mu$ (units of $H$)")
left.set_title("At $x_4 = 0$, $b_0 = 0$")
left.set_ylim(-0.2, 4.0)
times = np.linspace(0.0, 0.5, 101)
for bv in (0.0, 3.0, 6.0, 9.0):
    right.plot(times, [size(term(bv, t, 0.0)) for t in times],
               label=f"$\\beta = {bv:g}H$")
right.set_yscale("symlog", linthresh=0.1)
right.set_xlabel("time $x_4$ (units of $1/H$)")
right.set_ylabel("size (units of $H$)")
right.set_title("Against time, $b_0 = 0$")
right.legend(fontsize=8);
save_figure(fig, "boost_size",
            "The size $\\sqrt{\\mathrm{tr}(M^TM)/16}$ of the term $M = "
            "\\gamma'^\\mu\\Omega'_\\mu$ of the field equation in the frame boosted "
            "in the $(x_4, x_0)$ plane with rapidity $b = \\beta x_4 + b_0$, for "
            "$H = 1$; sizes in units of $H$. Left: against the rapidity rate "
```

```
"$\\beta$ at $x_4 = 0$, $b_0 = 0$; the diagonal frame ($\\beta = 0$) "
"gives $3$, the value of $3H\\gamma^{(0)}$, and the size $|6H - "
"\\beta|/2$ falls to zero at $\\beta = 6H$. Right: against the time "
"$x_4$ (symmetric logarithmic axis) for $\\beta = 0, 3H, 6H, 9H$; for "
"$\\beta = 6H$ the term is zero at all times. The metric is the same in "
"every one of these frames.")
```

Out [9]:

```
PASS size 3 in the diagonal frame (3 H gamma^(x0), H = 1), 0 for beta = 6H
```

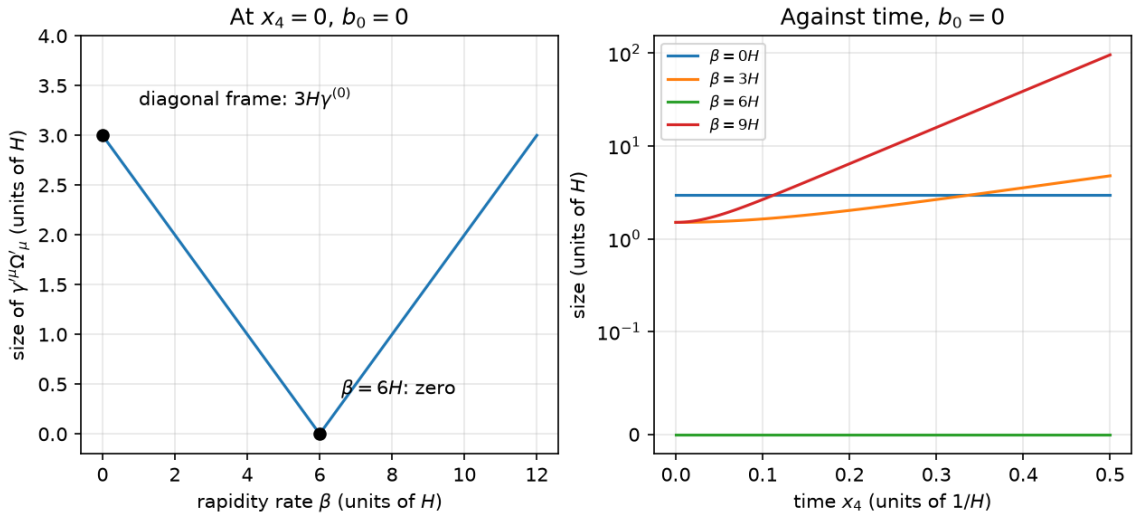

Figure 231: The size $\sqrt{\mathrm{tr}(M^T M)/16}$ of the term $M = \gamma'^{\mu}\Omega'_{\mu}$ of the field equation in the frame boosted in the (x_4, x_0) plane with rapidity $b = \beta x_4 + b_0$, for $H = 1$; sizes in units of H. Left: against the rapidity rate β at $x_4 = 0$, $b_0 = 0$; the diagonal frame ($\beta = 0$) gives 3, the value of $3H\gamma^{(0)}$, and the size $|6H - \beta|/2$ falls to zero at $\beta = 6H$. Right: against the time x_4 (symmetric logarithmic axis) for $\beta = 0, 3H, 6H, 9H$; for $\beta = 6H$ the term is zero at all times. The metric is the same in every one of these frames. (Notebook 08c, figure 3.)

Out [9]:

```
Figure 08c.3 saved as Revision/textbook/figures/08c_3_boost_size.png
```

Text cell:

```
The next cell draws the $16 \times 16$ matrices themselves as colour maps (red
positive, blue negative, white zero): $\gamma^\mu\Omega_\mu$ of the diagonal frame,
$\gamma'^\mu\Omega'_\mu$ for $\beta = 3H$ at $x_4 = 0.3$, and for $\beta = 6H$
(zero), with $H = 1$, $b_0 = 0$.
```

In [10]:

```
maps = [("diagonal frame", term(0.0, 0.3, 0.0)),
        ("boosted, $\\beta = 3H$", term(3.0, 0.3, 0.0)),
        ("boosted, $\\beta = 6H$", term(6.0, 0.3, 0.0))]
fig, panels = plt.subplots(1, 3, figsize=(11.0, 3.9))
for panel, (title, matrix) in zip(panels, maps):
    image = panel.imshow(np.array(matrix, dtype=float), cmap="RdBu_r", vmin=-3.5,
                         vmax=3.5)
    panel.set_title(title, fontsize=10)
    panel.set_xticks([0, 5, 10, 15])
    panel.set_yticks([0, 5, 10, 15])
    panel.set_xlabel("column")
    panel.grid(False)  # no grid lines over the coloured squares
panels[0].set_ylabel("row")
fig.colorbar(image, ax=list(panels), shrink=0.8, label="entry (units of $H$)")
save_figure(fig, "heat_maps",
            "The $16 \\times 16$ matrix $\\gamma^\\mu\\Omega_\\mu$ of the field "
            "equation as a colour map (rows and columns numbered from 0; red "
            "positive, blue negative, white zero; entries in units of $H$, $H = 1$, "
            "time $x_4 = 0.3$, $b_0 = 0$). Left: diagonal frame, $3H\\gamma^{(0)}$, "
            "sixteen entries equal to 3. Middle: frame boosted with $\\beta = 3H$, "
            "$\\frac{3H}{2}(\\cosh b\\,\\gamma^{(0)} - \\sinh b\\,\\gamma^{(4)})$ "
            "with $b = 0.9$. Right: frame boosted with $\\beta = 6H$, the zero "
            "matrix. Same metric, same field equation, different frames.")
```

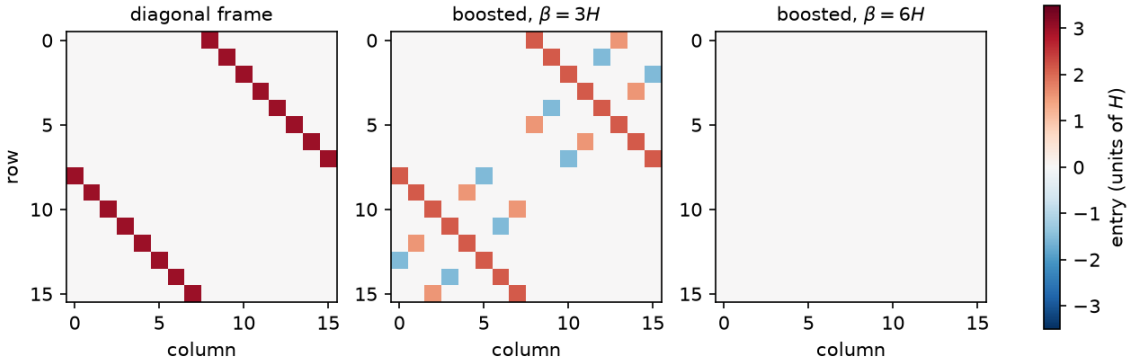

Figure 232: The 16×16 matrix $\gamma^\mu \Omega_\mu$ of the field equation as a colour map (rows and columns numbered from 0; red positive, blue negative, white zero; entries in units of H, $H = 1$, time $x_4 = 0.3$, $b_0 = 0$). Left: diagonal frame, $3H\gamma^{(0)}$, sixteen entries equal to 3. Middle: frame boosted with $\beta = 3H$, $\frac{3H}{2}(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$ with $b = 0.9$. Right: frame boosted with $\beta = 6H$, the zero matrix. Same metric, same field equation, different frames. (Notebook 08c, figure 4.)

Out [10]:

```
Figure 08c.4 saved as Revision/textbook/figures/08c_4_heat_maps.png
```

Text cell:

```
## 12. The connection itself vanishes in no frame: the curvature
```

If Ω'_μ were zero in some frame, the spinor curvature $F_{\mu\nu} = \partial_\mu\Omega_\nu - \partial_\nu\Omega_\mu + [\Omega_\mu, \Omega_\nu]$ would be zero there. The next cell computes $F_{x_1x_0}$ in the diagonal frame and in the frame boosted with $\beta = 6H$ (where $\gamma'^\mu\Omega'_\mu = 0$) and checks:

1. in both frames $F_{x_1x_0} = \frac14\sum_{\rho,\sigma}R_{\rho\sigma x_1x_0} \gamma^\rho\gamma^\sigma$ with $R_{\rho\sigma\mu\nu} = g_{\rho\rho}R^\rho{}_{\sigma\mu\nu}$: the curvature of the connection is the Riemann tensor of the metric;
2. in both frames $F_{x_1x_0} = c_{14}\,\gamma^{(1)}\gamma^{(4)} + c_{10}\,\gamma^{(1)}\gamma^{(0)}$ with two numbers c_{14}, c_{10} (found as traces, like the coefficients of section 9, and printed);
3. hence $F_{x_1x_0}^2 = (c_{14}^2 - c_{10}^2)\,I_{16}$, because $(\gamma^{(1)}\gamma^{(4)})^2 = +1$, $(\gamma^{(1)}\gamma^{(0)})^2 = -1$ and the two products anticommute; and $c_{14}^2 - c_{10}^2$ is the SAME in both frames, $H^2\big((a_4')^2 - H^2\big)e^{2a_4}\cos^2 z/(4\sin^{5/3}z)$. It must be: the matrices γ^ρ and γ'^ρ of the two frames obey the same relations $\{\gamma^\rho, \gamma^\sigma\} = 2g^{\rho\sigma}$, so (because the 16-dimensional representation is irreducible; Pauli's theorem) there is an invertible 16×16 matrix S with $\gamma'^\rho = S\gamma^\rho S^{-1}$, and then $F' = SFS^{-1}$ and $F'^2 = SF^2S^{-1} = F^2$;
4. $F'_{x_1x_0}$ is not zero.

Since the Riemann tensor does not depend on the frame and is not zero, no frame can remove Ω_μ.

In [11]:

```
def spinor_curvature(fr):
    """F_x1x0 = d_1 Omega_0 - d_0 Omega_1 + [Omega_1, Omega_0]."""
    O1, O0 = fr["Omega"][1], fr["Omega"][0]
    return sp.diff(O0, x[1]) - sp.diff(O1, x[0]) + O1 * O0 - O0 * O1


def riemann_side(fr):
    """(1/4) sum R_rho,sigma,x1,x0 gamma^rho gamma^sigma in the frame fr."""
    result = sp.zeros(16, 16)
    for rho in range(8):
        for sigma in range(8):
            value = g[rho] * riemann(rho, sigma, 1, 0)  # lower the first index
            if value != 0:
                result += value * fr["gamma"][rho] * fr["gamma"][sigma] / 4
    return result


P14, P10 = G[1] * G[4], G[1] * G[0]  # gamma^(1) gamma^(4) and gamma^(1) gamma^(0)


def two_coefficients(F):
    """c14, c10 with F = c14 P14 + c10 P10 (P14^2 = 1, P10^2 = -1), and whether
    nothing else is left."""
    c14 = sp.simplify((P14 * F).trace() / 16)
    c10 = sp.simplify(-(P10 * F).trace() / 16)
```

```
    return c14, c10, all(is_zero(v) for v in F - c14 * P14 - c10 * P10)


zero_frame = {"Omega": [M.subs(at_6H) for M in boosted["Omega"]],  # beta = 6H
              "gamma": [M.subs(at_6H) for M in boosted["gamma"]]}
F_diagonal = spinor_curvature(diagonal)
F_zero = spinor_curvature(zero_frame)
check_record(all(is_zero(v) for v in F_diagonal - riemann_side(diagonal)),
             "diagonal frame: F_x1x0 = (1/4) R_rho,sigma,x1,x0 gamma^rho gamma^sigma",
             THEORY, "spinor_curvature_equals_riemann",
             "= (1/4) R_{rho sigma mu nu} gamma^rho gamma^sigma")
check(all(is_zero(v) for v in F_zero - riemann_side(zero_frame)),
      "boosted frame (beta = 6H): F'_x1x0 = (1/4) R gamma'^rho gamma'^sigma")
c14, c10, only_two = two_coefficients(F_diagonal)
c14_b, c10_b, only_two_b = two_coefficients(F_zero)
say(f"diagonal frame: c14 = {c14}")
say(f"diagonal frame: c10 = {c10}")
say(f"boosted frame:  c14 = {c14_b}")
say(f"boosted frame:  c10 = {c10_b}")
check(only_two and only_two_b,
      "in both frames F_x1x0 = c14 gamma^(1) gamma^(4) + c10 gamma^(1) gamma^(0)")
kappa_F = H**2 * (a4_prime**2 - H**2) * sp.exp(2 * a4) * sp.cos(z) ** 2 \
    / (4 * sp.sin(z) ** sp.Rational(5, 3))
check(is_zero(c14**2 - c10**2 - kappa_F) and is_zero(c14_b**2 - c10_b**2 - kappa_F),
      "F^2 = F'^2 = H^2 (a4'^2 - H^2) e^(2 a4) cos^2 z / (4 sin^(5/3) z) I16 in "
      "both frames")
entries_nonzero = sum(1 for v in F_zero if not is_zero(v))
report("nonzero entries of F'_x1x0 in the frame with gamma'^mu Omega'_mu = 0",
       entries_nonzero)
check_record(entries_nonzero > 0, "beta = 6H: the spinor curvature F'_x1x0 is not "
             "zero", "Revision/theory/reports/python-scope.json",
             "boosted_frame_curvature_nonzero",
             "F'_x1x0 = d_x1 Omega'_x0 - d_x0 Omega'_x1 + [Omega'_x1, Omega'_x0] is "
             "nonzero")
```

Out [11]:

```
PASS diagonal frame: F_x1x0 = (1/4) R_rho,sigma,x1,x0 gamma^rho gamma^sigma
     reproduces Revision/theory/reports/python-field-theory.json, check
    spinor_curvature_equals_riemann
PASS boosted frame (beta = 6H): F'_x1x0 = (1/4) R gamma'^rho gamma'^sigma
diagonal frame: c14 = -H*exp(a4(x4))*cos(6*H*x0)*Derivative(a4(x4),
    x4)/(2*sin(6*H*x0)**(5/6))
diagonal frame: c10 = -H**2*exp(a4(x4))*cos(6*H*x0)/(2*sin(6*H*x0)**(5/6))
boosted frame:  c14 = H*(H*sinh(6*H*x4 + b0) - cosh(6*H*x4 + b0)*Derivative(a4(x4),
    x4))*exp(a4(x4))*cos(6*H*x0)/(2*sin(6*H*x0)**(5/6))
boosted frame:  c10 = -H*(H*cosh(6*H*x4 + b0) - sinh(6*H*x4 + b0)*Derivative(a4(x4),
    x4))*exp(a4(x4))*cos(6*H*x0)/(2*sin(6*H*x0)**(5/6))
PASS in both frames F_x1x0 = c14 gamma^(1) gamma^(4) + c10 gamma^(1) gamma^(0)
PASS F^2 = F'^2 = H^2 (a4'^2 - H^2) e^(2 a4) cos^2 z / (4 sin^(5/3) z) I16 in both frames
RESULT nonzero entries of F'_x1x0 in the frame with gamma'^mu Omega'_mu = 0 = 32
PASS beta = 6H: the spinor curvature F'_x1x0 is not zero
     reproduces Revision/theory/reports/python-scope.json, check
```

```
    boosted_frame_curvature_nonzero
```

Text cell:

```
The next cell draws $F_{x_1x_0}$ in both frames as colour maps, at $H = 1$,
$z = \pi/4$, $x_4 = 0.5$, $b_0 = 0$, along the canonical history $a_4 = x_4$. On
this history $a_4' = AH = H$, so $c_{14}^2 - c_{10}^2 = 0$: the matrix $F_{x_1x_0}$
is NOT zero, but its square is zero (a *nilpotent* matrix, like the $2 \times 2$
matrix with rows $(0, 1)$ and $(0, 0)$); similar matrices have the same rank, and
the cell checks that both have rank 8. Moreover, with $a_4' = H$ the printed
boosted coefficients are $e^{-b}$ times the diagonal ones ($\cosh b - \sinh b =
e^{-b}$), so $F' = e^{-b}F$ at this point, with $b = 6Hx_4 = 3$; the cell checks
this too. The entries of the right map are therefore about $e^{-3} \approx 0.05$
times those of the left one; each map has its own colour scale.
```

In [12]:

```
def at_point(matrix):
    """The numbers of a symbolic matrix at H = 1, a4 = x4, z = pi/4, x4 = 0.5."""
    concrete = matrix.replace(sp.Function("a4"), history).doit()
    concrete = concrete.subs({H: 1, x[0]: sp.pi / 24, x[4]: sp.Rational(1, 2), b0: 0})
    return np.array(concrete.evalf(), dtype=float)


F_numbers = [("diagonal frame: $F_{x_1x_0}$", at_point(F_diagonal)),
             ("boosted, $\\beta = 6H$: $F'_{x_1x_0}$", at_point(F_zero))]
report("largest entry of F_x1x0 at the point (diagonal, boosted)",
       ", ".join(f"{np.abs(matrix).max():.6f}" for _, matrix in F_numbers))
ranks = [int(np.linalg.matrix_rank(matrix)) for _, matrix in F_numbers]
squares = [np.abs(matrix @ matrix).max() for _, matrix in F_numbers]
report("rank of F_x1x0 at the point (diagonal, boosted)", ranks)
check(ranks == [8, 8] and max(squares) < 1e-14 and
      np.allclose(F_numbers[1][1], np.exp(-3.0) * F_numbers[0][1], atol=1e-14),
      "a4 = x4: F and F' have rank 8 and square 0, and F' = e^(-3) F at x4 = 0.5")
fig, panels = plt.subplots(1, 2, figsize=(10.0, 4.2))
for panel, (title, matrix) in zip(panels, F_numbers):
    largest = np.abs(matrix).max()  # each panel has its own colour scale
    image = panel.imshow(matrix, cmap="RdBu_r", vmin=-largest, vmax=largest)
    fig.colorbar(image, ax=panel, shrink=0.85, label="entry (units of $H^2$)")
    panel.set_title(title, fontsize=10)
    panel.set_xticks([0, 5, 10, 15])
    panel.set_yticks([0, 5, 10, 15])
    panel.set_xlabel("column")
    panel.grid(False)  # no grid lines over the coloured squares
panels[0].set_ylabel("row")
save_figure(fig, "curvature_maps",
            "The spinor curvature $F_{x_1x_0} = \\partial_1\\Omega_0 - "
            "\\partial_0\\Omega_1 + \\Omega_1\\Omega_0 - \\Omega_0\\Omega_1$ as a "
            "$16 \\times 16$ colour map (red positive, blue negative, white zero; "
            "units of $H^2$) at $H = 1$, $z = \\pi/4$, $x_4 = 0.5$ along the history "
            "$a_4 = x_4$; each panel has its own colour scale. Left: diagonal "
            "frame. Right: the frame boosted with $\\beta = 6H$, in which the term "
```

```
"$\\gamma'^\\mu\\Omega'_\\mu$ of the field equation is identically zero; "
"there the curvature is $e^{-3}$ times the left one. Neither is zero: "
"the curvature equals one quarter of the Riemann tensor contracted "
"with two gammas, so no choice of frame can make the spin connection "
"vanish. On this history both matrices have rank 8 and square zero.")
```

Out [12]:

```
RESULT largest entry of F_x1x0 at the point (diagonal, boosted) = 0.778093, 0.038739
RESULT rank of F_x1x0 at the point (diagonal, boosted) = [8, 8]
PASS a4 = x4: F and F' have rank 8 and square 0, and F' = e^(-3) F at x4 = 0.5
```

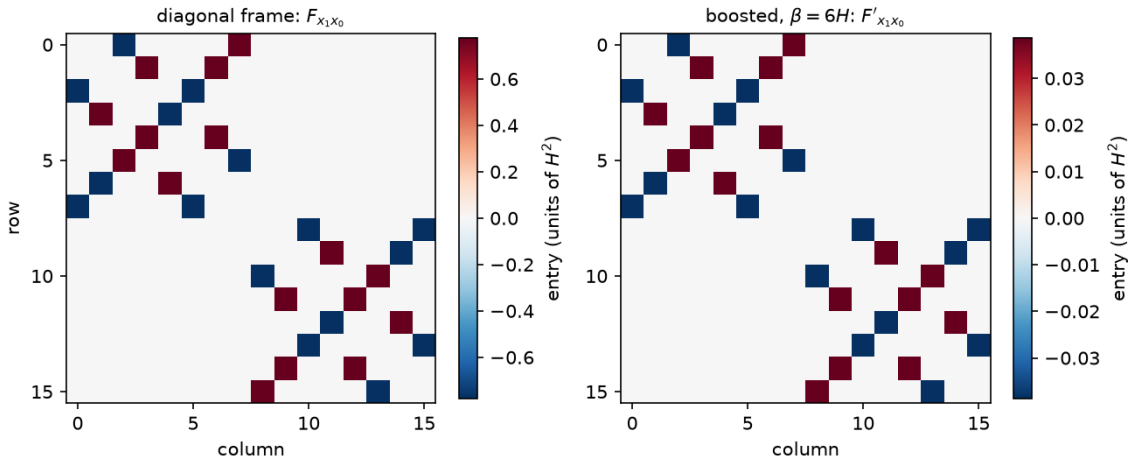

Figure 233: The spinor curvature $F_{x_1x_0} = \partial_1\Omega_0 - \partial_0\Omega_1 + \Omega_1\Omega_0 - \Omega_0\Omega_1$ as a 16×16 colour map (red positive, blue negative, white zero; units of H^2) at $H = 1$, $z = \pi/4$, $x_4 = 0.5$ along the history $a_4 = x_4$; each panel has its own colour scale. Left: diagonal frame. Right: the frame boosted with $\beta = 6H$, in which the term $\gamma'^\mu\Omega'_\mu$ of the field equation is identically zero; there the curvature is e^{-3} times the left one. Neither is zero: the curvature equals one quarter of the Riemann tensor contracted with two gammas, so no choice of frame can make the spin connection vanish. On this history both matrices have rank 8 and square zero. (Notebook 08c, figure 5.)

Out [12]:

```
Figure 08c.5 saved as Revision/textbook/figures/08c_5_curvature_maps.png
```

Text cell:

```
## 13. The spin connection in the energy-momentum tensor

Although the connection drops out of the symmetrised Lagrangian (section 9), it
appears in the energy-momentum tensor. For a field that depends only on $x_4$,
the derivative part of $K^{x_4}{}_{x_1} = \frac12(\bar\Phi\gamma^{x_4}D_{x_1}\Phi -
(D_{x_1}\bar\Phi)\gamma^{x_4}\Phi)$ vanishes and the connection part is
$\frac12\bar\Phi\{\gamma^{x_4}, \Omega_{x_1}\}\Phi$. The next cell checks
```

```
$\frac12\{\gamma^{x_4}, \Omega_{x_1}\} = \frac12e^{a_4}\sin^{1/6}z\,H\,\gamma^{(4)}
\gamma^{(1)}\gamma^{(0)}$ (the $a_4'$ part cancels) and that the bilinear matrix
$C\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}$ is not zero.
```

In [13]:

```
g4_up, Omega_1 = diagonal["gamma"][4], diagonal["Omega"][1]
connection_part = (g4_up * Omega_1 + Omega_1 * g4_up) / 2
target = sp.exp(a4) * sixth * H * G[4] * G[1] * G[0] / 2
check_record(all(is_zero(v) for v in connection_part - target)
             and C * G[4] * G[1] * G[0] != sp.zeros(16, 16),
             "(1/2){gamma^x4, Omega_x1} = (1/2) e^a4 sin^(1/6) z H gamma^(4) "
             "gamma^(1) gamma^(0), and C gamma^(4) gamma^(1) gamma^(0) != 0",
             "Revision/theory/reports/python-scope.json",
             "spin_connection_in_the_energy_momentum_tensor",
             "(1/2) e^a4 sin^(1/6) z H Phibar gamma^(x4) gamma^(x1) gamma^(x0) Phi",
             "C gamma^(x4) gamma^(x1) gamma^(x0) != 0")
```

Out [13]:

```
PASS (1/2){gamma^x4, Omega_x1} = (1/2) e^a4 sin^(1/6) z H gamma^(4) gamma^(1) gamma^(0),
    and C gamma^(4) gamma^(1) gamma^(0) != 0
     reproduces Revision/theory/reports/python-scope.json, check
    spin_connection_in_the_energy_momentum_tensor
```

Text cell:

```
## 14. The last check

The last cell checks that the five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [14]:

```
for name in ("08c_1_cancel_and_survive.png", "08c_2_pieces_along_history.png",
             "08c_3_boost_size.png", "08c_4_heat_maps.png", "08c_5_curvature_maps.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [14]:

```
PASS the figure file 08c_1_cancel_and_survive.png exists
PASS the figure file 08c_2_pieces_along_history.png exists
PASS the figure file 08c_3_boost_size.png exists
PASS the figure file 08c_4_heat_maps.png exists
PASS the figure file 08c_5_curvature_maps.png exists
ALL 32 CHECKS PASSED (notebook 08c)
```

Text cell:

```
## 15. What this notebook showed

- On the background $b_4 = 1$: the canonical spin connection of the diagonal frame
  has 12 independent nonzero components, each $a_4'$ or $H$ times a factor that
  never vanishes, so
  $\Omega_\mu = 0$ for every $\mu$ only in the formal flat limit $a_4' = 0$,
  $H = 0$, which is not a member of the author's family (PROVED; Revision check
  `nontriviality_Omega_zero_iff_flat`).
- In the diagonal frame the term of the field equation is $\gamma^\mu\Omega_\mu =
  3H\gamma^{(0)}$ for EVERY history $a_4$ (PROVED, exact): the time-direction
  pieces $\pm a_4'/2$ of the three inflating and the three deflating directions
  cancel, the six hidden-direction pieces $H/2$ add up; equivalently it is the
  half-density term $\frac{1}{2\sqrt{|g|}}\partial_\mu(\sqrt{|g|}\gamma^\mu)$, and
  $\{\gamma^\mu, \Omega_\mu\} = 0$ for each $\mu$, so the connection drops out of
  the symmetrised Lagrangian. Because $(\gamma^{(0)})^2 = I_{16}$, the term
  $3H\gamma^{(0)}\Psi$ is not zero for any field $\Psi \neq 0$ (PROVED).
- The term contains no $a_4$, although $R^{x_4}{}_{x_4} = 6(a_4')^2$: the
  deflation of the extra times enters the field equation only through the frame
  factors $e^{\mp a_4}\sin^{-1/6}z$ of the derivative terms (PROVED). For a general
  $b_4$ the term is $3H\gamma^{(0)} + \frac34(b_4'/b_4)\gamma^{(4)}$ and the
  extra-time factor $e^{a_4}b_4^{-1/2}\sin^{-1/6}z$ (Chapter 6, Notebook 06a).
- The value $3H\gamma^{(0)}$ belongs to the diagonal frame: in the frame boosted in
  the $(x_4, x_0)$ plane with rapidity $\beta x_4 + b_0$ the term is
  $\frac{6H - \beta}{2}(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$, and for
  $\beta = 6H$ it is identically ZERO (PROVED). So "the field equation contains the
  gravitational term $3H\gamma^{(0)}$" is a statement about one frame and one
  choice of field variables, not about the field equation as such.
- What no frame removes: the spin connection itself. Its curvature equals
  $\frac14 R_{\rho\sigma\mu\nu}\gamma^\rho\gamma^\sigma$ in every frame (checked in
  two frames for $(\mu, \nu) = (x_1, x_0)$), and the Riemann tensor is not zero
  ($R^{x_0}{}_{x_0} = -6H^2$, $R = 6((a_4')^2 - 7H^2)$): the metric is curved for
  every $H > 0$ (PROVED). Together with the frame factors in every derivative
  term, this is the frame-independent content of the non-triviality statements
  [1] and [2].
- The connection also enters the energy-momentum tensor (PROVED for the component
  $K^{x_4}{}_{x_1}$ of a field that depends only on $x_4$).
```

8.15 Line-by-line walk-through of Notebook 08c

The notebook has 14 code cells, In [1] to In [14]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted exactly and then explained. A line that starts with `#` is a **comment**, which Python skips; it is there for the reader. In the quoted code a line `...` stands for the remaining lines of a long figure caption; every caption is printed in full under its figure in Section 8.14, and a paragraph "What Figure 08c.k shows" after the code says what to look for. Because Notebook 08c is the first notebook of this chapter, its set-up cell and its two record helpers are explained here in full; the walk-throughs of Notebooks 08a, 08b and 08d refer back to this explanation.

In [1], the set-up cell. The first part of the cell, down to the two lines of `-` and `=` signs, is the complete run instructions of Section 8.13 again, as comment lines, so that the notebook file carries

its own instructions. The code starts below the line that announces THE SET-UP. It computes no physics, and it is the same in every notebook of the book except for one line, the notebook's name.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of an installed package) so that the cell can use it. `json` reads and writes the text format JSON, in which the Revision record stores its results (a JSON file holds names and values: numbers, texts, lists and dictionaries); `os` gives access to the computer's environment; `textwrap` breaks long text into lines; `from pathlib import Path` takes the single name `Path` out of the module `pathlib`. A `Path` is the address of a file or a folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "08c"  # this notebook: chapter 08, example c
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell. A **variable** is a name for a value; the last line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"08c"`. The figure files are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. The text in triple quotes under the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it, and `[here, *here.parents]` is the list that starts with `here` and continues with all of them (the star unpacks one list into another). The `for` loop takes these folders one after the other; the operator `/` joins a folder and a name into a longer path; `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If no folder does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

`REPO` is the repository folder found by the function. An **environment variable** is a named text that the computer hands to every program it starts; `os.environ.get(name, default)` returns its value, or the default when it is not set. So `OUTPUT_ROOT`, the folder below which files are written, is the repository, unless the book's checking tool names a scratch folder; then a check run never changes the repository.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`repository_file` gives the path of a repository file for reading. `output_file` gives the path at which to write a file and first creates its folder: `path.parent` is the folder of the file, and `mkdir(parents=True, exist_ok=True)` makes it together with any missing folders above it and does nothing if it exists already. `say` prints a text broken into lines of at most 89 characters; `str(text)` turns any value into a string first, and the continuation lines are indented by four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

matplotlib reads personal settings from the computer it runs on; `rcdefaults()` throws them away, so that every computer draws the same figures. `rcParams` is matplotlib's dictionary of settings (a **dictionary** is a table of pairs key: value in curly brackets); the update sets the standard figure size (7 by 4.2 inches), the font size, and a faint grid in every plot.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
```

```
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: the names in curly brackets are replaced by their values, so `CAPTION_FILE` is `Revision/textbook/figures/08c.captions.json`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes the text `{}` and a line end into the captions file, in the character encoding UTF-8 and with the line end `\n` on every operating system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
```

`setdefault(name, value)` returns the number already stored for this name, or stores and returns `value`, here one more than the number of figures so far; so the figures are numbered 1, 2, 3, ... in the order in which they are first saved, and a cell that is run twice keeps its number. The next two lines build the file name, for example `08c_1_cancel_and_survive.png`, and its path relative to the repository.

```
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`fig.savefig` writes the picture as a PNG file with the options explained in the comments. `plt.close(fig)` removes the figure from matplotlib's memory, so that Jupyter does not draw it again at the end of the cell. The caption is stored in the dictionary, and the whole dictionary is written into the captions file as JSON text (`json.dumps` turns it into a string; `indent=1` puts every entry on its own line and `sort_keys=True` sorts the entries, so that the file is always the same). `display(Image(...))` shows the saved picture below the cell; the `metadata` tells the book's tools which figure it is. The last line prints, for example, `Figure 08c.1 saved as Revision/textbook/figures/08c_1_cancel_and_survive.png`.

```
PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (a sequence of values in square brackets). `check` is the notebook's test: if the condition is false, `raise AssertionError` stops the notebook with a message that names the check; otherwise the name is appended to `PASSED` and a line `PASS name` is printed, followed, when the argument `record` is given, by a line `reproduces ...` that names the Revision record and its check. `None` is Python's value for "nothing given".

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with `RESULT`; the expression `(f" {unit}" if u nit else "")` adds the unit only when one is given. `all_checks_passed` prints the last line of the notebook, with `len(PASSED)`, the number of checks that passed. The last statement prints the only output of the cell, `Set-up of notebook 08c complete:  repository folder found, helpers defined.`

In [2], the record helpers, the gammas, the symbols and the metric.

```
import numpy as np  # numbers and matrices (for the plots)
import sympy as sp  # exact algebra with symbols

REPORTS = {}  # report file -> {check name: (verdict, detail)}, each read once
```

numpy (short name `np`) computes with arrays of floating-point numbers; sympy (short name `sp`) computes exactly with symbols, fractions and functions. `REPORTS` is an empty dictionary that will hold each Revision report after it has been read once.

```
def record_says(report_file, check_name, *pieces):
    """True when the Revision report records the check check_name with the verdict
```

```
    pass and its detail text contains every given piece of text."""
    if report_file not in REPORTS:  # read the report the first time it is needed
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        REPORTS[report_file] = {entry["name"]: (entry["verdict"].lower(),
                                                entry["detail"])
                                for entry in data["checks"]}
    verdict, detail = REPORTS[report_file][check_name]
    return verdict == "pass" and all(piece in detail for piece in pieces)
```

A Revision report is a JSON file with a list `checks`; each entry has a `name`, a `verdict` (`PASS` or `FAIL`, written in capitals by some verifiers and in small letters by others) and a `detail` text. `*pieces` collects any number of further arguments into a tuple (a fixed list). The first time a report is needed, `json.loads` turns its text into Python values, and the **dictionary comprehension** in curly brackets builds the table name: (verdict in small letters, detail) for every entry. Then the function looks up the check and returns true only if its verdict is `pass` and every piece of text occurs in the detail (`piece in detail` asks whether one string is part of another; `all(...)` is true when every item is). A check name that the report does not contain makes the lookup fail with an error, which also stops the notebook.

```
def check_record(condition, title, report_file, check_names, *pieces):
    """check() for a result that reproduces Revision checks (one name or a list of
    names): it passes only if condition is true AND the report records every one
    of them as passed, with every piece of text in the detail of the first one."""
    names = [check_names] if isinstance(check_names, str) else list(check_names)
    on_record = record_says(report_file, names[0], *pieces) and all(
        record_says(report_file, name) for name in names[1:])
    word = "check" if len(names) == 1 else "checks"
    check(condition and on_record, title,
          record=f"{report_file}, {word} " + " and ".join(names))
```

`check_record` is `check` for a result that the Revision record also contains. `isinstance(check_names, str)` asks whether a single name (a string) was given; then it is put into a one-element list, otherwise the given names are turned into a list. The record must report the first check as passed with every piece of text in its detail, and every further check as passed. The check passes only if the notebook's own computation (`condition`) holds AND the record says so; the `record=` line then names the file and the check or checks (`" and ".join(names)` joins the names with the word `and`). So a PASS line with a `reproduces` line means two independent things agree: this notebook's computation and the Revision verifier.

```
gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
G = [sp.Matrix(rows) for rows in gammas_record["gamma"]]  # gamma^(x0) ... (x7)
ETA = gammas_record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1]
C = sp.Matrix(gammas_record["C"])  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
```

The file `gammas.json` of the Revision record holds the eight gamma matrices as lists of rows of whole numbers, the frame metric `eta` and the matrix `C`. The **list comprehension** `[sp.Matrix(rows) for rows in ...]` turns each gamma into an exact sympy matrix; `G[k]` is $\gamma^{(k)}$: `G[0]` is $\gamma^{(0)}$, `G[1]` is $\gamma^{(1)}$ and `G[4]` is $\gamma^{(4)}$.

```
x = sp.symbols("x0:8", real=True)  # the coordinates; x[k] is xk
H = sp.Symbol("H", positive=True)
beta, b0 = sp.symbols("beta b0", real=True)  # the boost: rapidity beta x4 + b0
a4 = sp.Function("a4")(x[4])  # the metric function: ANY function of x4
z = 6 * H * x[0]
sixth = sp.sin(z) ** sp.Rational(1, 6)  # sin(z)^(1/6)
```

`sp.symbols("x0:8")` makes the eight symbols `x0`, ..., `x7` (the range 0 to 7; the end 8 is not included), so `x[k]` is x_k; `real=True` tells sympy they are real numbers, and `positive=True` that $H > 0$. `sp.Function("a4")(x[4])` is an unknown function $a_4(x_4)$: nothing is assumed about it, so every check below holds for every history. `z` is the expression $6Hx_0$ and `sixth` the expression $\sin^{1/6} z$ (`sp.Rational(1, 6)` is the exact fraction 1/6, not the decimal number 0.1666...).

```
f = [sp.cot(z)] + [sp.exp(a4) * sixth] * 3 + [sp.Integer(1)] \
    + [sp.exp(-a4) * sixth] * 3  # the scale factors f_0, ..., f_7 (b4 = 1)
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal entries g_aa
third = sp.sin(z) ** sp.Rational(1, 3)
authors_metric = [sp.cot(z) ** 2] + [sp.exp(2 * a4) * third] * 3 + [-1] \
    + [-sp.exp(-2 * a4) * third] * 3  # the author's metric at b4 = 1
```

`[value] * 3` is a list with three copies of the value, and `+` joins lists; so `f` is the list of the eight scale factors of Section 8.2 (the backslash at the end of a line continues the statement on the next line). `g` holds the diagonal entries $g_{aa} = \eta_{aa} f_a^2$ (`range(8)` runs through 0, 1, ..., 7). `authors_metric` is the author's metric typed directly from his formula, as a second, independent copy.

```
def is_zero(expr):
    """True when sympy shows that expr is exactly zero (two simplifications)."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    simpler = sp.simplify(expr)
    return simpler == 0 or sp.simplify(sp.fu(simpler)) == 0
```

`is_zero` decides whether an exact expression is zero. `sp.sympify` turns plain Python numbers into sympy numbers; if the expression is already the number 0, the answer is yes. Otherwise sympy's general simplifier `sp.simplify` is tried, and if that does not reach 0, the trigonometric simplifier `sp.fu` followed by `simplify` again. (An answer "no" would only mean that sympy did not find a proof; every check of the notebook that uses `is_zero` needs the answer "yes", or that a quantity is NOT zero; for the latter the notebook also prints the expression, so that the reader can see it.)

```
# The record prints each entry as sympy writes it, e.g. "g_x0x0 = cot(6*H*x0)**2".
metric_texts = [f"g_x{a}x{a} = {sp.sstr(authors_metric[a])}" for a in range(8)]
say(metric_texts[0])
say(metric_texts[4])
check_record(all(is_zero(g[a] - authors_metric[a]) for a in range(8)),
             "eta_aa f_a^2 is the author's metric (all 8 diagonal entries)",
             "Revision/theory/reports/python-field-theory.json",
             "metric_from_vielbein_equals_SPEC", *metric_texts)
check_record(is_zero(sp.prod(f) - sp.cos(z)), "sqrt|g| = f0 f1 ... f7 = cos z (b4 = 1)",
             "Revision/theory/reports/python-field-theory.json",
             "sqrt_det_g_equals_cos_z", "cos(6 H x0)")
```

`sp.sstr` writes an expression as sympy's text (powers with `**`). The list `metric_texts` holds the eight entries in the form in which the record prints them, for example `g_x1x1 = exp(2*a4(x4))*sin(6*H*x0)**(1/3)`; two of them are printed. The first check compares the eight entries $\eta_{aa} f_a^2$ with the author's entries exactly, and it requires the record's check `metric_from_vielbein_equals_SPEC` to contain all eight texts (`*metric_texts` passes the list as eight separate arguments). The second check multiplies the eight scale factors (`sp.prod`) and compares with $\cos z$. Output: the two entries and two PASS lines, each with its `reproduces` line.

In [3], the Christoffel symbols.

```
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for lam in range(8):
    for mu in range(8):
        for nu in range(8):
            value = 0  # the bracket of the formula; g is diagonal, so only these
            if lam == nu:  # three cases contribute
                value += sp.diff(g[lam], x[mu])
            if lam == mu:
                value += sp.diff(g[lam], x[nu])
            if mu == nu:
                value -= sp.diff(g[mu], x[lam])
            if value != 0:
                Gam[lam][mu][nu] = sp.simplify(value / (2 * g[lam]))
```

`Gam` is an $8 \times 8 \times 8$ table of exact zeros (a list of 8 lists of 8 lists of 8 zeros; `_` is a loop name that is not used). The three nested loops run over all 512 index triples. For a diagonal metric $g_{\lambda\nu}$ is nonzero only for $\lambda = \nu$, so the bracket $\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu}$ of the formula of Section 8.4 has at most the three terms that the three `if` lines add (`sp.diff(expr, x[mu])` is the exact partial derivative; `+=` adds to a variable, `-=` subtracts). If the bracket is not zero, the symbol is the bracket divided by $2g_{\lambda\lambda}$, simplified.

```
independent = sum(1 for lam in range(8) for mu in range(8) for nu in range(mu, 8)
                  if Gam[lam][mu][nu] != 0)
report("independent nonzero Christoffel symbols", independent)
for lam, mu, nu in ((1, 1, 4), (5, 4, 5), (1, 0, 1), (0, 0, 0)):
    say(f"Gamma^x{lam}_x{mu} x{nu} = {Gam[lam][mu][nu]}")
check_record(independent == 25, "25 independent nonzero Christoffel symbols",
             "Revision/theory/reports/python-field-theory.json",
             "christoffel_symmetric_metric_compatible",
             f"{independent} independent nonzero symbols")
```

The **generator expression** counts 1 for every nonzero symbol with $\mu \le \nu$ (the symbols are symmetric in the two lower indices, so the pairs with $\nu < \mu$ are not new); the count is printed as a RESULT line: 25. The loop prints four symbols (index positions from 0, printed as names from 1): ${\Gamma^{x_1}}_{x_1 x_4} = a_4'$, which sympy writes `Derivative(a4(x4), x4)`; ${\Gamma^{x_5}}_{x_4 x_5} = -a_4'$; ${\Gamma^{x_1}}_{x_1 x_0} = H/\tan z = H \cot z$; and ${\Gamma^{x_0}}_{x_0 x_0} = -12H/\sin(2z)$, which is the table's $-6H/(\sin z \cos z)$ because $\sin 2z = 2 \sin z \cos z$. The check requires 25 and the record's text "25 independent nonzero symbols".

In [4], the frame, its connection and the vielbein postulate.

```
S_AB = [[(G[a] * G[c] - G[c] * G[a]) / 4 for c in range(8)] for a in range(8)]


def boost(b):
    """The boost by the rapidity b in the frame plane (x4, x0), and its inverse."""
    L, L_inverse = sp.eye(8), sp.eye(8)
    L[4, 4] = L[0, 0] = L_inverse[4, 4] = L_inverse[0, 0] = sp.cosh(b)
    L[4, 0] = L[0, 4] = sp.sinh(b)
    L_inverse[4, 0] = L_inverse[0, 4] = -sp.sinh(b)
    return L, L_inverse
```

`S_AB[a][c]` is the generator $S^{ac} = \frac{1}{4}[\gamma^{(a)}, \gamma^{(c)}]$ for all 64 pairs (the letter `c` is used for the second frame index because `b` is the rapidity). `boost(b)` starts from two 8×8 unit matrices (`sp.eye(8)`) and fills the entries of the (x_4, x_0) plane (rows and columns 4 and 0): $\cosh b$ on the diagonal and $\sinh b$ off it for the boost L of Section 8.3, and $-\sinh b$ off the diagonal for its inverse, the boost with $-b$ (multiply the two: $\cosh^2 b - \sinh^2 b = 1$ on the diagonal, $-\cosh b \sinh b + \sinh b \cosh b = 0$ off it).

```
def frame(b):
    """Vielbein e[a, mu], inverse E[mu, a], connection omega[mu, a, c] (mixed),
    Omega_mu and gamma^mu of the diagonal frame boosted by the rapidity b."""
    L, L_inverse = boost(b)
    e = L * sp.diag(*f)
    E = sp.diag(*[1 / value for value in f]) * L_inverse
```

`frame(b)` builds everything for the frame boosted by b (for $b = 0$, the diagonal frame). `sp.diag(*f)` is the diagonal matrix with the scale factors on its diagonal, the diagonal frame ${e^a}_\mu = f_a \delta^a_\mu$; multiplying by L from the left combines its rows, the frame vectors, exactly as in Section 8.3. The inverse is the product of the inverses in reverse order: the diagonal matrix of the $1/f_a$ times L^{-1}; `E[mu, a]` is ${E_a}^\mu$.

```
    omega = {}
    for mu in range(8):
        for a in range(8):
            for c in range(8):
                value = 0  # e^a_nu (d_mu E_c^nu + Gamma^nu_mu,lam E_c^lam)
                for nu in range(8):
                    if e[a, nu] == 0:
                        continue  # skip the terms that are zero anyway
                    term = sp.diff(E[nu, c], x[mu])
                    for lam in range(8):
                        if Gam[nu][mu][lam] != 0 and E[lam, c] != 0:
                            term += Gam[nu][mu][lam] * E[lam, c]
                    value += e[a, nu] * term
                omega[mu, a, c] = sp.simplify(value)
```

This is the definition of the canonical spin connection of Section 8.4, ${\omega_\mu}^a{}_c = \sum_\nu {e^a}_\nu(\partial_\mu {E_c}^\nu + \sum_\lambda {\Gamma^\nu}_{\mu\lambda} {E_c}^\lambda)$, written as four nested loops. `continue` jumps to the next ν when ${e^a}_\nu = 0$, and the inner `if` skips products with a zero factor; both only save time. The result is stored in the dictionary `omega` under the key `(mu, a, c)`; it is the connection with the first frame index up (the "mixed" form).

```
    Omega = []
    for mu in range(8):
        matrix = sp.zeros(16, 16)
        for a in range(8):
            for c in range(a + 1, 8):  # (1/2) sum over all a, c = sum over a < c
                matrix += ETA[a] * omega[mu, a, c] * S_AB[a][c]
        Omega.append(matrix)
    gamma_up = [sum((E[mu, a] * G[a] for a in range(8)), sp.zeros(16, 16))
                for mu in range(8)]
    return {"e": e, "E": E, "omega": omega, "Omega": Omega, "gamma": gamma_up}
```

For each μ the spinor connection $\Omega_\mu = \sum_{a<c} \omega_{\mu ac} S^{ac}$ is built from a 16×16 zero matrix (`sp.zeros(16, 16)`); `ETA[a] * omega[mu, a, c]` lowers the first frame index, $\omega_{\mu ac} = \eta_{aa} \omega_\mu{}^a{}_c$. The coordinate gammas are $\gamma^\mu = \sum_a E_a{}^\mu \gamma^{(a)}$ (the sum starts from the zero matrix given as the second argument of `sum`). The function returns a dictionary with the five objects.

```
def postulate_failures(fr):
    """How many of the 512 components of the vielbein postulate are not zero."""
    failures = 0
    for a in range(8):
        for mu in range(8):
            for nu in range(8):
                value = sp.diff(fr["e"][a, nu], x[mu]) \
                    - sum(Gam[lam][mu][nu] * fr["e"][a, lam] for lam in range(8)) \
                    + sum(fr["omega"][mu, a, c] * fr["e"][c, nu] for c in range(8))
                failures += 0 if is_zero(value) else 1
    return failures
```

`postulate_failures` evaluates the vielbein postulate $\partial_\mu e^a{}_\nu - \sum_\lambda \Gamma^\lambda{}_{\mu\nu} e^a{}_\lambda + \sum_c \omega_\mu{}^a{}_c e^c{}_\nu$ for all $8 \times 8 \times 8 = 512$ index triples and counts the ones that are not zero (`0 if ... else 1` is 0 when the component vanishes and 1 otherwise).

```
def antisymmetric(fr):
    """omega_mu,ac = eta_aa omega_mu^a_c is antisymmetric in a, c for every mu."""
    return all(is_zero(ETA[a] * fr["omega"][mu, a, c] + ETA[c] * fr["omega"][mu, c, a])
               for mu in range(8) for a in range(8) for c in range(8))


diagonal = frame(sp.Integer(0))
check_record(postulate_failures(diagonal) == 0 and antisymmetric(diagonal),
             "diagonal frame: vielbein postulate (512 components), omega antisymmetric",
             "Revision/theory/reports/python-field-theory.json",
             ["vielbein_postulate", "spin_connection_antisymmetric"],
             "(512 components)")
```

`antisymmetric` checks $\omega_{\mu ac} + \omega_{\mu ca} = 0$ for all 512 triples. Then the diagonal frame is built with $b = 0$ (`sp.Integer(0)` is sympy's exact zero) and the first check of the cell requires zero failures, antisymmetry, and the two record checks `vielbein_postulate` (whose detail must contain "(512 components)") and `spin_connection_antisymmetric`.

```
nonzero = [(mu, a, c) for mu in range(8) for a in range(8) for c in range(a + 1, 8)
           if not is_zero(diagonal["omega"][mu, a, c])]
omega_diagonal = diagonal["omega"]
for mu, a, c in nonzero:
    say(f"omega_x{mu},(x{a})(x{c}) = "
        f"{sp.simplify(ETA[a] * omega_diagonal[mu, a, c])}")
check_record(len(nonzero) == 12, "12 independent nonzero components omega_mu,ab (a < b)",
             "Revision/theory/reports/wolfram-field-theory.json", "omega_components",
             f"exactly {len(nonzero)} independent nonzero omega_mu,ab "
             "(a < b in the order x0..x7)")
```

`nonzero` lists every index triple (μ, a, c) with $a < c$ whose component is not zero, and the loop prints each lowered component $\omega_{\mu ac}$. The output shows exactly the 12 components of Section 8.4, in sympy's notation: for x_1, x_2, x_3 the pair `exp(a4(x4))*sin(6*H*x0)**(1/6)*Derivative(a4(x4), x4)` and `H*exp(a4(x4))*sin(6*H*x0)**(1/6)`, and for x_5, x_6, x_7 the same with e^{-a_4} and a minus sign. The check requires 12 and the Wolfram record's text with the number 12.

In [5], the connection vanishes only in flat space.

```
a4_prime = sp.diff(a4, x[4])  # a4' = d a4 / d x4
A1 = sp.Symbol("A1", real=True)  # a letter that stands for a4'
allowed = [sign * sp.exp(power * a4) * sixth for sign in (1, -1) for power in (1, -1)]
kinds = []  # for each component: "A1" (a4' times a factor) or "H" (H times a factor)
```

`a4_prime` is a_4'. `A1` is a plain symbol that the cell puts in place of a_4' (the Revision record writes `A1` for a_4'). `allowed` is the list of the four factors $\pm e^{\pm a_4} \sin^{1/6} z$ (two loops: two signs times two powers). `kinds` will record, for each of the 12 components, which shape it has.

```
for mu, a, c in nonzero:
    value = sp.simplify(ETA[a] * omega_diagonal[mu, a, c]).subs(a4_prime, A1)
    found = [str(coupling) for coupling in (A1, H) for factor in allowed
             if is_zero(value - coupling * factor)]  # which shape fits
    kinds.append(found[0] if len(found) == 1 else "no shape")
n_A1, n_H = kinds.count("A1"), kinds.count("H")
report("components of the form a4' x factor and H x factor", f"{n_A1}, {n_H}")
```

For each nonzero component, `.subs(a4_prime, A1)` replaces a_4' by the letter `A1`, and `found` lists the names of the couplings (a_4' or H) for which the component equals coupling times one allowed factor. Exactly one shape must fit; otherwise the component is marked `"no shape"`. `count` counts the two kinds. Output: `RESULT components of the form a4' x factor and H x factor = 6, 6`.

```
S_rows = np.array([[float(v) for v in S_AB[a][c]] for a in range(8)
                   for c in range(a + 1, 8)])  # 28 rows of 256 numbers
rank_S = int(np.linalg.matrix_rank(S_rows))
report("rank of the 28 matrices S^ab written as rows of 256 numbers", rank_S)
```

Each of the 28 generators S^{ac} with $a < c$ is written as one row of its 256 entries (iterating over a sympy matrix gives its entries row by row; `float` turns each into a decimal number). `np.linal`

`g.matrix_rank` computes the **rank** of the 28×256 array: the number of independent rows. Output: 28, so the 28 generators are linearly independent.

```
# a4' != 0 makes the six "A1" components nonzero, H != 0 the six "H" components;
# with independent S^ab, Omega vanishes for every mu only when a4' = 0 and H = 0.
check_record(n_A1 == 6 and n_H == 6 and rank_S == 28,
             "Omega_mu = 0 for every mu iff a4' = 0 and H = 0 (formal flat limit)",
             "Revision/theory/reports/python-field-theory.json",
             "nontriviality_Omega_zero_iff_flat",
             "the S^ab are linearly independent",
             "every Omega_mu vanishes iff a4' = 0 AND H = 0")
```

The comment repeats the three-line argument of Section 8.6, and the check combines its three ingredients (six components of each kind, rank 28) with the record's check `nontriviality_Omega_zero_iff_flat`, whose detail must contain the two quoted sentences.

In [6], what cancels and what survives.

```
per_direction = [(diagonal["gamma"][mu] * diagonal["Omega"][mu]).applyfunc(sp.simplify)
                 for mu in range(8)]
coefficient_4 = [sp.simplify(-(G[4] * M).trace() / 16) for M in per_direction]
coefficient_0 = [sp.simplify((G[0] * M).trace() / 16) for M in per_direction]
```

`per_direction[mu]` is the product $\gamma^{x_\mu}\Omega_{x_\mu}$ (no sum); `.applyfunc(sp.simplify)` simplifies each of its 256 entries. The **trace** (`.trace()`) of a matrix is the sum of its diagonal entries. If $M = c_4\gamma^{(4)} + c_0\gamma^{(0)}$, then $\mathrm{tr}(\gamma^{(0)}M) = c_4\,\mathrm{tr}(\gamma^{(0)}\gamma^{(4)}) + c_0\,\mathrm{tr}(I_{16}) = 16c_0$ and $\mathrm{tr}(\gamma^{(4)}M) = c_4\,\mathrm{tr}(-I_{16}) + c_0\,\mathrm{tr}(\gamma^{(4)}\gamma^{(0)}) = -16c_4$, because the trace of a product of two different gammas is zero (Section 8.9, Step 4). So the two lists hold the coefficients c_4 and c_0 of every direction.

```
nothing_else = all(
    all(is_zero(entry) for entry in
        per_direction[mu] - coefficient_4[mu] * G[4] - coefficient_0[mu] * G[0])
    for mu in range(8))
for mu in range(8):
    say(f"gamma^x{mu} Omega_x{mu} = ({coefficient_4[mu]}) gamma^(x4) "
        f"+ ({coefficient_0[mu]}) gamma^(x0)")
check(nothing_else, "each gamma^mu Omega_mu (no sum) is a combination of gamma^(x4) "
      "and gamma^(x0)")
```

`nothing_else` subtracts $c_4\gamma^{(4)} + c_0\gamma^{(0)}$ from each product and asks whether every entry of the remainder is zero: the trace formula is valid only if nothing else is left. The loop prints the eight products: $(a_4'/2)\gamma^{(4)} + (H/2)\gamma^{(0)}$ for x_1, x_2, x_3; zero for x_4 and x_0; $(-a_4'/2)\gamma^{(4)} + (H/2)\gamma^{(0)}$ for x_5, x_6, x_7, exactly the results of Section 8.5. The check prints its PASS line.

```
THEORY = "Revision/theory/reports/python-field-theory.json"  # the sympy record
# The record writes A1 for a4'; the partial sums in the record's notation:
inflating_sum = sp.simplify(sum(coefficient_4[1:4])).subs(a4_prime, A1)  # 3*A1/2
deflating_sum = sp.simplify(sum(coefficient_4[5:8])).subs(a4_prime, A1)  # -3*A1/2
hidden_sum = sp.simplify(sum(coefficient_0))  # 3*H
say(f"sums: inflating {inflating_sum}, deflating {deflating_sum}, hidden {hidden_sum}")
```

`THEORY` is a short name for the path of the sympy field-theory report. `coefficient_4[1:4]` is the part of the list with positions 1, 2, 3 (the directions x_1, x_2, x_3) and `coefficient_4[5:8]` the part with positions 5, 6, 7 (x_5, x_6, x_7). The three partial sums are printed in the record's notation: `sums: inflating 3*A1/2, deflating -3*A1/2, hidden 3*H`.

```
check_record([sp.simplify(c4 / a4_prime) for c4 in coefficient_4]
             == [0] + [sp.Rational(1, 2)] * 3 + [0] + [-sp.Rational(1, 2)] * 3
             and [sp.simplify(c0 / H) for c0 in coefficient_0]
             == [0] + [sp.Rational(1, 2)] * 3 + [0] + [sp.Rational(1, 2)] * 3,
             "per direction: +a4'/2 and -a4'/2 cancel, six times H/2 add up",
             THEORY, "time_terms_cancel_hidden_term_survives",
             f"inflating sum {inflating_sum}, deflating sum {deflating_sum}",
             f"the six hidden-direction terms add to {hidden_sum} gamma^(x0)")
```

The coefficients of $\gamma^{(4)}$ are divided by a_4' and must be exactly $\frac{1}{2}$ for x_1, x_2, x_3, then 0 for x_4, $-\frac{1}{2}$ for x_5, x_6, x_7 and 0 for x_0; the coefficients of $\gamma^{(0)}$ are divided by H and must be $\frac{1}{2}$ for the same six directions and 0 for x_4 and x_0. The record's check must contain the printed partial sums.

```
total = sum(per_direction, sp.zeros(16, 16))
check_record(all(is_zero(entry) for entry in total - 3 * H * G[0]),
             "gamma^mu Omega_mu = 3 H gamma^(x0) in the diagonal frame",
             THEORY, "gamma_mu_Omega_mu_equals_3H_gamma_x0",
             "gamma^mu Omega_mu = 3 H gamma^(x0) exactly")
# gamma^(x0) squares to 1, so it is invertible: 3 H gamma^(x0) Psi = 0 only for
# Psi = 0. The gravitational term is not zero for any field Psi that is not zero.
check_record(G[0] * G[0] == sp.eye(16),
             "(gamma^(x0))^2 = 1: 3 H gamma^(x0) Psi != 0 for every Psi != 0 (H > 0)",
             "Revision/theory/reports/wolfram-field-theory.json",
             "nontriviality_1_dirac16complex",
             "every Psi != 0 (gamma^(x0) is invertible)")
```

`total` adds the eight products: the matrix $\gamma^\mu \Omega_\mu$. The first check compares it entry by entry with $3H\gamma^{(0)}$. The second checks $(\gamma^{(0)})^2 = I_{16}$, the argument of Section 8.6 that $3H\gamma^{(0)}\Psi = 0$ only for $\Psi = 0$, together with the Wolfram record's non-triviality check.

```
sqrt_g = sp.cos(z)  # sqrt|g| = b4^(3/2) cos z, at b4 = 1
divergence = sum((sp.diff(sqrt_g * diagonal["gamma"][mu], x[mu]) for mu in range(8)),
                 sp.zeros(16, 16)) / (2 * sqrt_g)
check_record(all(is_zero(entry) for entry in divergence - 3 * H * G[0]),
             "(1/(2 sqrt|g|)) d_mu(sqrt|g| gamma^mu) = 3 H gamma^(x0) (divergence "
             "form)", THEORY, "divergence_of_sqrtg_gamma",
             "= 3 H gamma^(x0) = gamma^mu Omega_mu")
```

`divergence` is the half-density term $\frac{1}{2\sqrt{|g|}} \sum_\mu \partial_\mu(\sqrt{|g|}\gamma^\mu)$ of Section 8.5, computed with $\sqrt{|g|} = \cos z$ (sympy differentiates a matrix entry by entry); the check compares it with $3H\gamma^{(0)}$.

```
check_record(all(all(is_zero(entry) for entry in
                     diagonal["gamma"][mu] * diagonal["Omega"][mu]
```

```
            + diagonal["Omega"][mu] * diagonal["gamma"][mu])
        for mu in range(8)),
    "{gamma^mu, Omega_mu} = 0 for each mu: Omega drops out of the Lagrangian",
    "Revision/theory/reports/python-scope.json",
    "connection_free_lagrangian_same_equations",
    "{gamma^mu, Omega_mu} = 0 for each mu separately")
```

This check computes the anticommutator $\{\gamma^\mu, \Omega_\mu\}$ for each of the eight directions separately and requires every entry to be zero: the fact of Section 8.8 that removes the connection from the Lagrangian.

```
names = [f"$x_{mu}$" for mu in range(8)] + ["sum"]
values_4 = [float(sp.simplify(c4 / a4_prime)) for c4 in coefficient_4]
values_0 = [float(sp.simplify(c0 / H)) for c0 in coefficient_0]
values_4.append(sum(values_4))
values_0.append(sum(values_0))
colours = ["gray"] + ["tab:blue"] * 3 + ["gray"] + ["tab:red"] * 3 + ["black"]
```

For the bar chart: nine bar labels (the directions, written as LaTeX for matplotlib, and `"sum"`), the coefficients in units of a_4' and of H as decimal numbers, each list extended by its sum (`append` adds one element at the end), and nine colours (blue for 3-space, red for the extra times, gray for x_4 and x_0, black for the sum).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), sharey=True)
left.bar(names, values_4, color=colours)
left.set_title("coefficient of $\\gamma^{(4)}$, in units of $a_4'$")
left.set_ylabel("coefficient")
right.bar(names, values_0, color=colours)
right.set_title("coefficient of $\\gamma^{(0)}$, in units of $H$")
for panel in (left, right):
    panel.axhline(0.0, color="black", linewidth=0.8)
    panel.set_xlabel("direction $\\mu$ of $\\gamma^{\\mu}\\Omega_{\\mu}$")
save_figure(fig, "cancel_and_survive",
            "The term $\\gamma^\\mu\\Omega_\\mu$ of the field equation in the "
            ...
            "history $a_4$ and every $H > 0$, on the background $b_4 = 1$.")
```

`plt.subplots(1, 2, ...)` makes a figure with one row of two panels (axes), `left` and `right`, 10 by 4 inches, with a shared vertical axis. `bar` draws one bar per label with the given heights and colours. The titles and labels are LaTeX text between dollar signs (in a Python string a backslash is written twice). `axhline(0.0, ...)` draws a horizontal line at zero in both panels. `save_figure` saves the figure as Figure 08c.1 with its caption.

What Figure 08c.1 shows. Two bar charts with the eight directions $x_0, \ldots, x_7$ and their sum on the horizontal axis; the vertical axis is a pure number. Left: the coefficient of $\gamma^{(4)}$ in units of a_4': three blue bars at $+\frac{1}{2}$ (3-space), a gray bar at 0 (x_4), three red bars at $-\frac{1}{2}$ (the extra times), a gray bar at 0 (x_0), and a black sum bar at 0. Right: the coefficient of $\gamma^{(0)}$ in units of H: six bars at $+\frac{1}{2}$ and a sum bar at 3. The student should see the cancellation of the time-direction pieces (because the extra times deflate at the rate at which 3-space inflates) and the addition of the six hidden-direction pieces. The values are exact and hold for every history.

In [7], the curvature, and the pieces along the history.

```
def riemann(rho, sigma, mu, nu):
    """R^rho_sigma,mu,nu from the Christoffel symbols (not simplified)."""
    value = sp.diff(Gam[rho][nu][sigma], x[mu]) - sp.diff(Gam[rho][mu][sigma], x[nu])
    for lam in range(8):
        value += Gam[rho][mu][lam] * Gam[lam][nu][sigma] \
            - Gam[rho][nu][lam] * Gam[lam][mu][sigma]
    return value
```

`riemann` evaluates the Riemann tensor $R^{\rho}{}_{\sigma\mu\nu} = \partial_\mu \Gamma^{\rho}{}_{\nu\sigma} - \partial_\nu \Gamma^{\rho}{}_{\mu\sigma} + \sum_\lambda (\Gamma^{\rho}{}_{\mu\lambda}\Gamma^{\lambda}{}_{\nu\sigma} - \Gamma^{\rho}{}_{\nu\lambda}\Gamma^{\lambda}{}_{\mu\sigma})$ (Chapter 3) for one index quadruple.

```
ricci = [sum(riemann(rho, a, rho, a) for rho in range(8)) / g[a] for a in range(8)]
a4_second = sp.diff(a4, x[4], 2)
expected = [-6 * H**2] + [a4_second - 6 * H**2] * 3 + [6 * a4_prime**2] \
    + [-a4_second - 6 * H**2] * 3
check_record(all(is_zero(ricci[a] - expected[a]) for a in range(8)),
             "R^x1_x1 = a4'' - 6H^2, R^x4_x4 = 6 a4'^2, R^x5_x5 = -a4'' - 6H^2, "
             "R^x0_x0 = -6H^2", "Revision/theory/reports/wolfram-field-theory.json",
             "ricci_mixed_components", "R^x0_x0 = -6 H^2",
             "R^x1_x1 = R^x2_x2 = R^x3_x3 = a4'' - 6 H^2", "R^x4_x4 = 6 a4'^2",
             "R^x5_x5 = R^x6_x6 = R^x7_x7 = -a4'' - 6 H^2")
```

The Ricci tensor is $R_{aa} = \sum_\rho R^{\rho}{}_{a\rho a}$, and for a diagonal metric its mixed form is $R^{a}{}_{a} = R_{aa}/g_{aa}$; `ricci` holds the eight diagonal entries. `sp.diff(a4, x[4], 2)` is a_4''. The check compares them with the values of Section 8.7 and with the four texts of the Wolfram record.

```
ricci_scalar = sp.factor(sp.simplify(sum(ricci)).subs(a4_prime, A1))  # A1 for a4'
say(f"Ricci scalar in the record's notation: R = {ricci_scalar}")
check_record(is_zero(sum(ricci) - 6 * (a4_prime**2 - 7 * H**2)),
             "Ricci scalar R = 6 (a4'^2 - 7 H^2): curved for every H > 0",
             THEORY, "curvature_nonzero_flat_only_formally",
             f"Ricci scalar R = {sp.sstr(ricci_scalar)}")
check_record(not total.applyfunc(sp.simplify).has(a4) and not is_zero(ricci[4]),
             "gamma^mu Omega_mu contains no a4, although R^x4_x4 = 6 a4'^2 is not zero",
             "Revision/theory/reports/python-scope.json",
             "gammaOmega_blind_to_the_deflation",
             "contains neither e^a4 nor a4' nor a4''", "R^x4_x4 = 6 a4'^2")
```

The Ricci scalar is the sum of the eight entries; `sp.factor` writes it as a product, and it is printed as `R = 6*(A1**2 - 7*H**2)`. The first check compares it with $6((a_4')^2 - 7H^2)$ and with the record's text. The second check asks whether the simplified matrix $\gamma^\mu \Omega_\mu$ contains the function a_4 at all (`.has(a4)` is true if a_4 or a derivative of it occurs anywhere in it) and requires that $R^{x_4}{}_{x_4}$ is not zero.

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
A_history = parameters["physics"]["historyA"]  # the canonical history a4 = A H x4
history = sp.Lambda(x[4], A_history * x[4])  # with H = 1: a4 = x4
```

The parameters of the Revision Kohn-Sham record hold the constant A of the canonical history $a_4 = AHx_4$ (here $A = 1$), the prescribed background of the book's Kohn-Sham chapters. `sp.Lambda(x[4], A_history * x[4])` is the function $x_4 \mapsto Ax_4$, which is a_4 when $H = 1$.

```
def along_history(expr):
    """A numpy function of x4 for expr with H = 1, a4 = A x4, z = pi/4."""
    concrete = expr.replace(sp.Function("a4"), history).doit()
    concrete = concrete.subs({H: 1, x[0]: sp.pi / 24})  # z = 6 x0 = pi/4
    return sp.lambdify(x[4], concrete, "numpy")
```

`along_history` turns an exact expression into a numerical function of the time: `replace` puts the history in place of the unknown function a_4, `doit()` carries out the derivatives that then become computable ($a_4' = A$), `subs` puts in $H = 1$ and $x_0 = \pi/24$, so that $z = 6x_0 = \pi/4$, and `sp.lambdify` makes a numpy function of x_4.

```
times = np.linspace(0.0, 4.0, 201)
pieces = {"$\\omega_{x_1,(1)(4)} = a_4'e^{a_4}\\sin^{1/6}z$": diagonal["omega"][1, 1, 4],
          "$\\omega_{x_1,(1)(0)} = He^{a_4}\\sin^{1/6}z$": diagonal["omega"][1, 1, 0],
          "$|\\omega_{x_5,(4)(5)}| = a_4'e^{-a_4}\\sin^{1/6}z$":
              -ETA[4] * diagonal["omega"][5, 4, 5],
          "$|\\omega_{x_5,(5)(0)}| = He^{-a_4}\\sin^{1/6}z$":
              ETA[5] * diagonal["omega"][5, 5, 0]}
```

`np.linspace(0.0, 4.0, 201)` is an array of 201 equally spaced times from 0 to 4. The dictionary `pieces` maps four legend labels to four components of the connection: the two of the inflating direction x_1 and the two of the extra time x_5 (the factors `-ETA[3]` and `ETA[4]` turn the stored mixed components into the sizes named in the labels; the plot takes absolute values anyway).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
for (label, expr), style in zip(pieces.items(), ("-", "--", "-", "--")):
    left.plot(times, np.abs(along_history(expr)(times)) * np.ones_like(times),
              style, label=label)
left.set_yscale("log")
left.set_xlabel("time $x_4$")
left.set_ylabel("size of the component")
left.set_title("Pieces of the connection")
left.legend(fontsize=7)
```

`zip` pairs each (label, expression) with a line style: one minus sign in quotes for a solid line, two minus signs for a dashed one. Each component is evaluated along the history at the 201 times; `np.abs` takes the size, and multiplying by `np.ones_like(times)` (an array of ones) makes sure that an expression that happens to be constant is still drawn as a full curve. The vertical axis is logarithmic (`set_yscale("log")`), so an exponential is a straight line.

```
inflating = along_history(sum(coefficient_4[1:4]))(times) * np.ones_like(times)
deflating = along_history(sum(coefficient_4[5:8]))(times) * np.ones_like(times)
right.plot(times, inflating, color="tab:blue", label="$x_1, x_2, x_3$: $+3a_4'/2$")
right.plot(times, deflating, color="tab:red", label="$x_5, x_6, x_7$: $-3a_4'/2$")
```

```
right.plot(times, inflating + deflating, color="black", label="sum: $0$")
right.set_ylim(-2.0, 2.0)
right.set_xlabel("time $x_4$")
right.set_ylabel("coefficient of $\\gamma^{(4)}$")
right.set_title("Their contraction: constant, cancels")
right.legend(fontsize=8);
save_figure(fig, "pieces_along_history",
            "Along the canonical history $a_4 = x_4$ of the Kohn-Sham record ($A = "
            ...
            "sums stay constant, and they cancel exactly (black).")
```

The right panel draws the two partial sums of the $\gamma^{(4)}$ coefficients (3-space and extra times) along the history, and their sum; `set_ylim` fixes the vertical range. The semicolon after `right.legend(fontsize=8)` stops Jupyter from printing the value of that line (it would print an object with a memory address that changes from run to run). The figure is saved as Figure 08c.2. The cell prints three PASS lines, each with its `reproduces` line, and the figure line.

What Figure 08c.2 shows. Both panels have the time x_4 (units of $1/H$, with $H = 1$) on the horizontal axis, along the history $a_4 = x_4$ at $z = \pi/4$. Left, on a logarithmic vertical axis, four components of the canonical spin connection (pure numbers in units of H): the two of the inflating direction x_1 rise along a straight line (they grow like e^{a_4}), the two of the deflating extra time x_5 fall along a straight line (they shrink like e^{-a_4}); the dashed and the solid curves coincide because here $a_4' = AH = 1 = H$. Right: the coefficient of $\gamma^{(4)}$ in $\gamma^\mu \Omega_\mu$, summed over 3-space (blue, $+\frac{3}{2}$) and over the extra times (red, $-\frac{3}{2}$), and their sum (black, 0): three horizontal lines. The student should see that the pieces of the connection change exponentially while their contraction with the coordinate gammas does not change at all, and cancels: the deflation is invisible in the term.

In [8], the boosted frame.

```
rapidity = beta * x[4] + b0
boosted = frame(rapidity)
e_b = boosted["e"]
same_metric = all(
    is_zero(sum(ETA[a] * e_b[a, mu] * e_b[a, nu] for a in range(8))
            - (g[mu] if mu == nu else 0)) for mu in range(8) for nu in range(8))
SCOPE = "Revision/theory/reports/python-scope.json"  # the record of these checks
check_record(same_metric and all(is_zero(v) for v in e_b * boosted["E"] - sp.eye(8)),
             "the boosted frame gives the same metric (64 entries); e' E' = 1",
             SCOPE, "boosted_frame_reproduces_metric",
             "equals the author's metric for all 64 (mu, nu)")
```

The rapidity is the expression $b = \beta x_4 + b_0$ with symbolic β and b_0, and `frame(rapidity)` builds the boosted frame, its connection and its gammas exactly. `same_metric` checks, for all 64 pairs (μ, ν), that $\sum_a \eta_{aa} e'^a{}_\mu e'^a{}_\nu$ is $g_{\mu\mu}$ on the diagonal and 0 off it (the expression `(g[mu] if mu == nu else 0)` chooses). The check also verifies that E' is the inverse of e' ($e'E' = I_8$). `SCOPE` is a short name for the path of the scope report.

```
failures = postulate_failures(boosted)  # the number of nonzero components
check_record(failures == 0 and antisymmetric(boosted),
             "boosted frame: vielbein postulate (512 components), omega' antisymmetric",
```

```
            SCOPE, "boosted_frame_canonical_connection",
            f"holds for all 512 (a, mu, nu) ({failures} failures)")
```

The vielbein postulate and the antisymmetry for the boosted frame; the record's text must contain the same number of failures, 0.

```
total_boosted = sum((boosted["gamma"][mu] * boosted["Omega"][mu] for mu in range(8)),
                    sp.zeros(16, 16))
formula = (6 * H - beta) / 2 * (sp.cosh(rapidity) * G[0] - sp.sinh(rapidity) * G[4])
check_record(all(is_zero(entry) for entry in total_boosted - formula),
             "gamma'^mu Omega'_mu = ((6H - beta)/2) (cosh b gamma^(x0) - sinh b "
             "gamma^(x4))", SCOPE, "boosted_frame_gammaOmega_formula",
             "gamma'^mu Omega'_mu = ((6 H - beta)/2) (cosh b gamma^(x0) - sinh b "
             "gamma^(x4))")
```

`total_boosted` is $\gamma'^{\mu}\Omega'_{\mu}$ computed from the definitions; `formula` is the result of Section 8.9; the check compares all 256 entries.

```
at_6H = {beta: 6 * H}
nonzero_Omega = [mu for mu in range(8) if not all(
    is_zero(ETA[a] * boosted["omega"][mu, a, c].subs(at_6H))
    for a in range(8) for c in range(a + 1, 8))]
nonzero_text = ", ".join(f"x{mu}" for mu in nonzero_Omega)  # "x1, x2, ..."
say("beta = 6H: Omega'_mu is nonzero for mu = " + nonzero_text)
```

`at_6H` is the substitution $\beta = 6H$. `nonzero_Omega` lists the directions μ for which at least one component $\omega'_{\mu ac}$ is not zero at $\beta = 6H$; because the S^{ac} are linearly independent (In [5]), these are exactly the directions with $\Omega'_{\mu} \neq 0$. The line printed: `beta = 6H: Omega'_mu is nonzero for mu = x1, x2, x3, x4, x5, x6, x7`.

```
check_record(all(is_zero(entry.subs(at_6H)) for entry in total_boosted - formula)
             and formula.subs(at_6H) == sp.zeros(16, 16)
             and nonzero_Omega == [1, 2, 3, 4, 5, 6, 7],
             "beta = 6H: gamma'^mu Omega'_mu = 0 identically, Omega'_mu != 0 for "
             "x1..x7", SCOPE, "boosted_frame_gammaOmega_vanishes", "IDENTICALLY ZERO",
             f"nonzero for mu = {nonzero_text} (zero only for x0)")
```

The last check of the cell: at $\beta = 6H$ the computed term equals the formula, the formula is the zero matrix, and the connection is nonzero in the seven directions $x_1, \ldots, x_7$; the record must say "IDENTICALLY ZERO" and list the same directions. The cell prints four PASS lines and the line about Ω'_{μ}.

In [9], the size of the term in the boosted frames.

```
term = sp.lambdify((beta, x[4], b0), formula.subs(H, 1), "numpy")


def size(matrix):
    """sqrt(tr(M^T M)/16): the size of a 16 x 16 matrix, 1 for gamma^(x0)."""
    matrix = np.array(matrix, dtype=float)
    return np.sqrt(np.trace(matrix.T @ matrix) / 16)
```

`term` is a numerical function of (β, x_4, b_0) that returns the 16×16 matrix $\gamma'^{\mu}\Omega'_{\mu}$ at $H = 1$, from the formula that In [8] proved equal to the computed matrix. `size` measures a matrix by $\sqrt{\mathrm{tr}(M^T M)/16}$: $\mathrm{tr}(M^T M)$ is the sum of the squares of all 256 entries, so the size is the square root of that sum divided by 16 (for $\gamma^{(0)}$, a matrix with sixteen entries 1 and no others, it is 1); `@` is numpy's matrix product and `.T` the transpose.

```
beta_values = np.linspace(0.0, 12.0, 241)
sizes_beta = [size(term(bv, 0.0, 0.0)) for bv in beta_values]
check(abs(size(term(0.0, 0.0, 0.0)) - 3.0) < 1e-12 and size(term(6.0, 0.7, 0.3)) == 0,
      "size 3 in the diagonal frame (3 H gamma^(x0), H = 1), 0 for beta = 6H")
```

The sizes at 241 rapidity rates from 0 to 12, at $x_4 = 0$, $b_0 = 0$. The check: the diagonal frame ($\beta = 0$) gives size 3 to 10^{-12} (the term is $3\gamma^{(0)}$), and $\beta = 6H = 6$ gives exactly 0 at another point ($x_4 = 0.7$, $b_0 = 0.3$), because the factor $(6H - \beta)/2$ is exactly zero.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(beta_values, sizes_beta)
left.plot([0.0, 6.0], [3.0, 0.0], "o", color="black")
left.annotate("diagonal frame: $3H\\gamma^{(0)}$", (0.0, 3.0), (1.0, 3.3))
left.annotate("$\\beta = 6H$: zero", (6.0, 0.0), (6.6, 0.4))
left.set_xlabel("rapidity rate $\\beta$ (units of $H$)")
left.set_ylabel("size of $\\gamma'^\\mu\\Omega'_\\mu$ (units of $H$)")
left.set_title("At $x_4 = 0$, $b_0 = 0$")
left.set_ylim(-0.2, 4.0)
```

The left panel draws the size against β, marks the two special points with black dots (`"o"`), and writes a text next to each (`annotate(text, point, text position)`).

```
times = np.linspace(0.0, 0.5, 101)
for bv in (0.0, 3.0, 6.0, 9.0):
    right.plot(times, [size(term(bv, t, 0.0)) for t in times],
               label=f"$\\beta = {bv:g}H$")
right.set_yscale("symlog", linthresh=0.1)
right.set_xlabel("time $x_4$ (units of $1/H$)")
right.set_ylabel("size (units of $H$)")
right.set_title("Against time, $b_0 = 0$")
right.legend(fontsize=8);
save_figure(fig, "boost_size",
            "The size $\\sqrt{\\mathrm{tr}(M^TM)/16}$ of the term $M = "
            ...
            "every one of these frames.")
```

The right panel draws the size against the time from 0 to 0.5 for four rapidity rates (`{bv:g}` prints a number without unnecessary zeros, so `6.0` becomes `6`). The vertical axis is **symmetric logarithmic** (`"symlog"`): linear between -0.1 and 0.1 and logarithmic outside, so that the curve at exactly 0 can be drawn together with the growing ones. The figure is Figure 08c.3. Output: one PASS line and the figure line.

What Figure 08c.3 shows. Left: the size of $\gamma'^{\mu}\Omega'_{\mu}$ (units of H) against the rapidity rate β (units of H) at $x_4 = 0$: a V-shaped line, $|6H - \beta|/2$, from 3 at $\beta = 0$ (the diagonal frame, black dot)

down to 0 at $\beta = 6H$ (black dot) and up again. Right: the size against the time x_4 (units of $1/H$) for $\beta = 0, 3H, 6H, 9H$: constant 3 for the diagonal frame, growing for $\beta = 3H$ and $9H$ (the factor $\sqrt{\cosh 2b}$ of Exercise 5 in Section 8.39), and zero at all times for $\beta = 6H$. The student should see that the same metric and the same field equation give, in different frames, terms of every size including zero.

In [10], the term as colour maps.

```
maps = [("diagonal frame", term(0.0, 0.3, 0.0)),
        ("boosted, $\\beta = 3H$", term(3.0, 0.3, 0.0)),
        ("boosted, $\\beta = 6H$", term(6.0, 0.3, 0.0))]
fig, panels = plt.subplots(1, 3, figsize=(11.0, 3.9))
```

Three (title, matrix) pairs at $x_4 = 0.3$, $b_0 = 0$, and a figure with three panels.

```
for panel, (title, matrix) in zip(panels, maps):
    image = panel.imshow(np.array(matrix, dtype=float), cmap="RdBu_r", vmin=-3.5,
                         vmax=3.5)
    panel.set_title(title, fontsize=10)
    panel.set_xticks([0, 5, 10, 15])
    panel.set_yticks([0, 5, 10, 15])
    panel.set_xlabel("column")
    panel.grid(False)  # no grid lines over the coloured squares
panels[0].set_ylabel("row")
fig.colorbar(image, ax=list(panels), shrink=0.8, label="entry (units of $H$)")
save_figure(fig, "heat_maps",
            "The $16 \\times 16$ matrix $\\gamma^\\mu\\Omega_\\mu$ of the field "
            ...
            "matrix. Same metric, same field equation, different frames.")
```

`imshow` draws a matrix as a grid of coloured squares; the colour map `"RdBu_r"` runs from blue (negative) through white (zero) to red (positive), on the same scale -3.5 to 3.5 in all three panels (`vmin`, `vmax`). The ticks mark rows and columns 0, 5, 10, 15. One colour bar serves all three panels. The figure is Figure 08c.4; the cell prints only the figure line.

What Figure 08c.4 shows. Three 16×16 colour maps (rows and columns numbered from 0; entries in units of H). Left, the diagonal frame: $3H\gamma^{(0)}$, sixteen red squares of value 3, in rows 0 to 7 at columns 8 to 15 and in rows 8 to 15 at columns 0 to 7, along two diagonal lines ($\gamma^{(0)}$ has the entries 1 at the places (row, column) $= (k, k+8)$ and $(k+8, k)$, $k = 0, \ldots, 7$, and no others). Middle, the frame boosted with $\beta = 3H$ at $b = 0.9$: $\frac{3H}{2}(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$, more nonzero entries of both signs. Right, $\beta = 6H$: a white square, the zero matrix. The student should see one metric and one field equation producing three different matrices in three frames.

In [11], the spinor curvature in two frames.

```
def spinor_curvature(fr):
    """F_x1x0 = d_1 Omega_0 - d_0 Omega_1 + [Omega_1, Omega_0]."""
    O1, O0 = fr["Omega"][1], fr["Omega"][0]
    return sp.diff(O0, x[1]) - sp.diff(O1, x[0]) + O1 * O0 - O0 * O1
```

`spinor_curvature` computes the curvature component $F_{x_1x_0} = \partial_1\Omega_0 - \partial_0\Omega_1 + [\Omega_1, \Omega_0]$ of a frame.

```
def riemann_side(fr):
    """(1/4) sum R_rho,sigma,x1,x0 gamma^rho gamma^sigma in the frame fr."""
    result = sp.zeros(16, 16)
    for rho in range(8):
        for sigma in range(8):
            value = g[rho] * riemann(rho, sigma, 1, 0)  # lower the first index
            if value != 0:
                result += value * fr["gamma"][rho] * fr["gamma"][sigma] / 4
    return result
```

`riemann_side` computes the other side of the identity of Section 8.10, $\frac{1}{4}\sum_{\rho,\sigma} R_{\rho\sigma x_1 x_0}\gamma^\rho\gamma^\sigma$ with the coordinate gammas of the given frame; for a diagonal metric, lowering the first index multiplies by $g_{\rho\rho}$.

```
P14, P10 = G[1] * G[4], G[1] * G[0]  # gamma^(1) gamma^(4) and gamma^(1) gamma^(0)


def two_coefficients(F):
    """c14, c10 with F = c14 P14 + c10 P10 (P14^2 = 1, P10^2 = -1), and whether
    nothing else is left."""
    c14 = sp.simplify((P14 * F).trace() / 16)
    c10 = sp.simplify(-(P10 * F).trace() / 16)
    return c14, c10, all(is_zero(v) for v in F - c14 * P14 - c10 * P10)
```

`P14` and `P10` are the products $\gamma^{(1)}\gamma^{(4)}$ and $\gamma^{(1)}\gamma^{(0)}$. Their squares are $+I_{16}$ and $-I_{16}$ ($\gamma^{(1)}\gamma^{(4)}\gamma^{(1)}\gamma^{(4)} = -(\gamma^{(1)})^2(\gamma^{(4)})^2 = -(1)(-1) = 1$, and $\gamma^{(1)}\gamma^{(0)}\gamma^{(1)}\gamma^{(0)} = -(\gamma^{(1)})^2(\gamma^{(0)})^2 = -1$), and the trace of their product is zero, so the two coefficients are found by traces exactly as in In [6]; the third returned value says whether F is exactly $c_{14}P_{14} + c_{10}P_{10}$.

```
zero_frame = {"Omega": [M.subs(at_6H) for M in boosted["Omega"]],  # beta = 6H
              "gamma": [M.subs(at_6H) for M in boosted["gamma"]]}
F_diagonal = spinor_curvature(diagonal)
F_zero = spinor_curvature(zero_frame)
check_record(all(is_zero(v) for v in F_diagonal - riemann_side(diagonal)),
             "diagonal frame: F_x1x0 = (1/4) R_rho,sigma,x1,x0 gamma^rho gamma^sigma",
             THEORY, "spinor_curvature_equals_riemann",
             "= (1/4) R_{rho sigma mu nu} gamma^rho gamma^sigma")
check(all(is_zero(v) for v in F_zero - riemann_side(zero_frame)),
      "boosted frame (beta = 6H): F'_x1x0 = (1/4) R gamma'^rho gamma'^sigma")
```

`zero_frame` is the boosted frame at $\beta = 6H$, the frame in which $\gamma'^\mu\Omega'_\mu = 0$ (only its connection and gammas are needed). The two curvatures are computed, and the identity $F = \frac{1}{4}R\gamma\gamma$ is checked in both frames; the first check also requires the record's check `spinor_curvature_equals_riemann`.

```
c14, c10, only_two = two_coefficients(F_diagonal)
c14_b, c10_b, only_two_b = two_coefficients(F_zero)
say(f"diagonal frame: c14 = {c14}")
say(f"diagonal frame: c10 = {c10}")
say(f"boosted frame:  c14 = {c14_b}")
```

```
say(f"boosted frame:  c10 = {c10_b}")
check(only_two and only_two_b,
      "in both frames F_x1x0 = c14 gamma^(1) gamma^(4) + c10 gamma^(1) gamma^(0)")
```

The four coefficients are printed in sympy's notation. In the diagonal frame they are $c_{14} = -Ha_4' e^{a_4}\cos z/(2\sin^{5/6} z)$ and $c_{10} = -H^2 e^{a_4}\cos z/(2\sin^{5/6} z)$, the values quoted in Section 8.10; in the boosted frame they contain $\cosh(6Hx_4 + b_0)$ and $\sinh(6Hx_4 + b_0)$. The check confirms that nothing else is left in either frame.

```
kappa_F = H**2 * (a4_prime**2 - H**2) * sp.exp(2 * a4) * sp.cos(z) ** 2 \
    / (4 * sp.sin(z) ** sp.Rational(5, 3))
check(is_zero(c14**2 - c10**2 - kappa_F) and is_zero(c14_b**2 - c10_b**2 - kappa_F),
      "F^2 = F'^2 = H^2 (a4'^2 - H^2) e^(2 a4) cos^2 z / (4 sin^(5/3) z) I16 in "
      "both frames")
```

`kappa_F` is the number $H^2((a_4')^2 - H^2)e^{2a_4}\cos^2 z/(4\sin^{5/3} z)$ of Section 8.10, and the check confirms that $c_{14}^2 - c_{10}^2$ equals it in both frames: the square of the curvature is the same in both frames.

```
entries_nonzero = sum(1 for v in F_zero if not is_zero(v))
report("nonzero entries of F'_x1x0 in the frame with gamma'^mu Omega'_mu = 0",
       entries_nonzero)
check_record(entries_nonzero > 0, "beta = 6H: the spinor curvature F'_x1x0 is not "
             "zero", "Revision/theory/reports/python-scope.json",
             "boosted_frame_curvature_nonzero",
             "F'_x1x0 = d_x1 Omega'_x0 - d_x0 Omega'_x1 + [Omega'_x1, Omega'_x0] is "
             "nonzero")
```

The number of nonzero entries of $F'_{x_1x_0}$ in the frame where the term vanishes is counted and printed: 32. The last check of the cell requires at least one, together with the record's check `boosted_frame_curvature_nonzero`. The cell prints five PASS lines (two with `reproduces` lines), the four coefficients and the RESULT line.

In [12], the curvature as colour maps.

```
def at_point(matrix):
    """The numbers of a symbolic matrix at H = 1, a4 = x4, z = pi/4, x4 = 0.5."""
    concrete = matrix.replace(sp.Function("a4"), history).doit()
    concrete = concrete.subs({H: 1, x[0]: sp.pi / 24, x[4]: sp.Rational(1, 2), b0: 0})
    return np.array(concrete.evalf(), dtype=float)
```

`at_point` puts the history, $H = 1$, $z = \pi/4$, $x_4 = \frac{1}{2}$ and $b_0 = 0$ into a symbolic matrix, evaluates it as decimal numbers (`evalf`) and returns a numpy array.

```
F_numbers = [("diagonal frame: $F_{x_1x_0}$", at_point(F_diagonal)),
             ("boosted, $\\beta = 6H$: $F'_{x_1x_0}$", at_point(F_zero))]
report("largest entry of F_x1x0 at the point (diagonal, boosted)",
       ", ".join(f"{np.abs(matrix).max():.6f}" for _, matrix in F_numbers))
ranks = [int(np.linalg.matrix_rank(matrix)) for _, matrix in F_numbers]
squares = [np.abs(matrix @ matrix).max() for _, matrix in F_numbers]
report("rank of F_x1x0 at the point (diagonal, boosted)", ranks)
```

The two curvature matrices at the point. The first RESULT line prints the largest entry of each (`:.6f` prints six decimals): 0.778093 and 0.038739. `ranks` holds their ranks and `squares` the largest entry of each square F^2; the second RESULT line prints the ranks, `[8, 8]`.

```
check(ranks == [8, 8] and max(squares) < 1e-14 and
      np.allclose(F_numbers[1][1], np.exp(-3.0) * F_numbers[0][1], atol=1e-14),
      "a4 = x4: F and F' have rank 8 and square 0, and F' = e^(-3) F at x4 = 0.5")
```

On this history $a_4' = H$, so $c_{14}^2 - c_{10}^2 = 0$ and $F^2 = 0$: the check requires rank 8 for both, squares below 10^{-14} (rounding), and $F' = e^{-3}F$ entry by entry ($b = 6Hx_4 = 3$; `np.allclose` compares two arrays within the tolerance `atol`). Indeed $0.778093 \cdot e^{-3} = 0.038739$.

```
fig, panels = plt.subplots(1, 2, figsize=(10.0, 4.2))
for panel, (title, matrix) in zip(panels, F_numbers):
    largest = np.abs(matrix).max()  # each panel has its own colour scale
    image = panel.imshow(matrix, cmap="RdBu_r", vmin=-largest, vmax=largest)
    fig.colorbar(image, ax=panel, shrink=0.85, label="entry (units of $H^2$)")
    panel.set_title(title, fontsize=10)
    panel.set_xticks([0, 5, 10, 15])
    panel.set_yticks([0, 5, 10, 15])
    panel.set_xlabel("column")
    panel.grid(False)  # no grid lines over the coloured squares
panels[0].set_ylabel("row")
save_figure(fig, "curvature_maps",
            "The spinor curvature $F_{x_1x_0} = \\partial_1\\Omega_0 - "
            "\\partial_0\\Omega_1 + \\Omega_1\\Omega_0 - \\Omega_0\\Omega_1$ as a "
            ...
            "vanish. On this history both matrices have rank 8 and square zero.")
```

Two colour maps, each with its own symmetric colour scale from minus to plus its largest entry and its own colour bar. The figure is Figure 08c.5. Output: two RESULT lines, one PASS line, the figure line.

What Figure 08c.5 shows. Two 16×16 colour maps of the curvature $F_{x_1x_0}$ (entries in units of H^2) at $H = 1$, $z = \pi/4$, $x_4 = 0.5$ on the history $a_4 = x_4$. Left: the diagonal frame, entries up to 0.778. Right: the frame boosted with $\beta = 6H$, where the term of the field equation is zero; the pattern is the same and the entries are e^{-3} times smaller (up to 0.0387; the colour scales differ). The student should see that the curvature is not zero in the frame in which the term vanishes: no frame can make the spin connection vanish.

In [13], the connection in the energy-momentum tensor.

```
g4_up, Omega_1 = diagonal["gamma"][4], diagonal["Omega"][1]
connection_part = (g4_up * Omega_1 + Omega_1 * g4_up) / 2
target = sp.exp(a4) * sixth * H * G[4] * G[1] * G[0] / 2
check_record(all(is_zero(v) for v in connection_part - target)
             and C * G[4] * G[1] * G[0] != sp.zeros(16, 16),
             "(1/2){gamma^x4, Omega_x1} = (1/2) e^a4 sin^(1/6) z H gamma^(4) "
             "gamma^(1) gamma^(0), and C gamma^(4) gamma^(1) gamma^(0) != 0",
             "Revision/theory/reports/python-scope.json",
```

```
            "spin_connection_in_the_energy_momentum_tensor",
            "(1/2) e^a4 sin^(1/6) z H Phibar gamma^(x4) gamma^(x1) gamma^(x0) Phi",
            "C gamma^(x4) gamma^(x1) gamma^(x0) != 0")
```

`connection_part` is $\frac{1}{2}\{\gamma^{x_4}, \Omega_{x_1}\}$ in the diagonal frame, `target` the result of Section 8.11. The check compares them entry by entry, requires the bilinear matrix $C\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}$ to be nonzero (`!=` means "is not equal to"), and requires the record's check with its two texts. Output: one PASS line with its `reproduces` line.

In [14], the last check.

```
for name in ("08c_1_cancel_and_survive.png", "08c_2_pieces_along_history.png",
             "08c_3_boost_size.png", "08c_4_heat_maps.png", "08c_5_curvature_maps.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The loop checks that each of the five figure files exists, and the last line prints the count. The checks come from the cells as follows: two in In [2], one in In [3], two in In [4], one in In [5], six in In [6], three in In [7], four in In [8], one in In [9], five in In [11], one each in In [12] and In [13], and five here; together 32, and the last line reads `ALL 32 CHECKS PASSED (notebook 08c)`.

8.16 Well-posedness: what a good evolution equation must do

The initial-value problem. A field equation that contains the time derivative $\partial_4\Psi$ is used like this: one gives the field on the slice $x_4 = 0$ (a seven-dimensional set with the coordinates $x_1, x_2, x_3, x_5, x_6, x_7, x_0$), the **data**, and asks for the field at later times. This is the **initial-value problem**, also called the **Cauchy problem**. The French mathematician Jacques Hadamard called such a problem **well posed** if three things hold: a solution exists; it is unique; and it depends **continuously** on the data, which means: for every later time and every tolerance, data that are close enough to each other give solutions that stay within that tolerance of each other up to that time. "Close" is measured by a **norm**, a number that measures the size of a function (for example its largest value, or the square root of the integral of its square). The third condition is what makes an equation useful for prediction: data are never known exactly, and an equation in which an arbitrarily small error of the data can produce an arbitrarily large error of the solution predicts nothing.

A toy example with two time directions. Take a function $\varphi(x_4, x_5)$ of the time x_4 and of one extra time x_5, and the equation

$$\partial_4^2\varphi = -\partial_5^2\varphi.$$

Compare it with the ordinary wave equation $\partial_4^2\varphi = +\partial_1^2\varphi$ along a space direction: the only difference is the minus sign, which appears because x_5 is time-like like x_4 (we show below that the field equation of this book has exactly this sign). The function $\varphi = \epsilon\cosh(Kx_4)\cos(Kx_5)$, with any numbers $\epsilon > 0$ and $K > 0$, solves it. Line by line:

$$\partial_4^2\varphi = \epsilon K^2\cosh(Kx_4)\cos(Kx_5) = K^2\varphi,$$
$$\partial_5^2\varphi = -\epsilon K^2\cosh(Kx_4)\cos(Kx_5) = -K^2\varphi,$$
$$-\partial_5^2\varphi = K^2\varphi = \partial_4^2\varphi.$$

The first line differentiates twice with respect to x_4 ($\frac{d}{dx}\cosh(Kx) = K\sinh(Kx)$ and $\frac{d}{dx}\sinh(Kx) = K\cosh(Kx)$); the second twice with respect to x_5 ($\frac{d^2}{dx^2}\cos(Kx) = -K^2\cos(Kx)$); the third compares. Its data are $\varphi(0, x_5) = \epsilon\cos(Kx_5)$ and $\partial_4\varphi(0, x_5) = 0$ (because $\cosh 0 = 1$ and $\sinh 0 = 0$): their largest value is ϵ, as small as we like. At the time $x_4 = 1$ the largest value of the solution is $\epsilon\cosh K$. The ratio, $\cosh K$, grows without bound as K grows ($\cosh 10 = 11013.2$, $\cosh 20 = 2.43 \times 10^8$). So however small we require the data to be, some data of that size produce a solution larger than any given number after one unit of time: the solution does not depend continuously on the data, and the problem is not well posed. For the ordinary wave equation the same data give $\epsilon\cos(Kx_4)\cos(Kx_1)$, which never exceeds ϵ.

The same sign in our field equation. In flat 4+4 space (all frame factors equal to 1, no gravitational term, $U = 0$) the field equation is $\sum_a \gamma^{(a)}\partial_a\Psi = m\Psi$. Applying the operator twice, line by line:

$$\begin{aligned}\Big(\sum_a \gamma^{(a)}\partial_a\Big)\Big(\sum_b \gamma^{(b)}\partial_b\Big)\Psi &= \sum_{a,b}\gamma^{(a)}\gamma^{(b)}\partial_a\partial_b\Psi \\ &= \sum_a(\gamma^{(a)})^2\partial_a^2\Psi + \sum_{a<b}\big(\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}\big)\partial_a\partial_b\Psi \\ &= (\partial_1^2 + \partial_2^2 + \partial_3^2 + \partial_0^2 - \partial_4^2 - \partial_5^2 - \partial_6^2 - \partial_7^2)\Psi.\end{aligned}$$

The first line multiplies out (the gammas are constant matrices); the second separates the terms with $a = b$ from the pairs $a \neq b$, using $\partial_a\partial_b = \partial_b\partial_a$; the third uses $(\gamma^{(a)})^2 = \eta_{aa}I_{16}$ and the anticommutation of different gammas. On the other side, applying the operator to $m\Psi$ gives $m\sum_a \gamma^{(a)}\partial_a\Psi = m^2\Psi$. Solving for the second time derivative:

$$\partial_4^2\Psi = (\partial_1^2 + \partial_2^2 + \partial_3^2 + \partial_0^2 - \partial_5^2 - \partial_6^2 - \partial_7^2 - m^2)\Psi.$$

The three extra-time derivatives enter with the sign of the toy example. This is the root of everything that follows.

Non-characteristic slices. In the author's metric the field equation of Section 8.2 can be solved for $\partial_4\Psi$. Write it as $\gamma^{(4)}\partial_4\Psi + R = V\Psi$, where R holds all the other terms on the left and $V = m + U'(S)$. Multiplying from the left by $\gamma^{(4)}$ and using $(\gamma^{(4)})^2 = -I_{16}$ gives $-\partial_4\Psi + \gamma^{(4)}R = \gamma^{(4)}V\Psi$, so

$$\partial_4\Psi = -\gamma^{(4)}\Big[V\Psi - \sum_{a\neq 4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Psi - 3H\gamma^{(0)}\Psi\Big]$$

(PROVED; `Revision/theory/reports/wolfram-field-theory.json`, check `evolution_form_G`). A slice is called **non-characteristic** when the matrix in front of the time derivative is invertible, as here: the equation then tells how the field changes in time from its values on the slice. This is necessary for a good evolution equation, but not sufficient: the toy example can be solved for

$\partial_4^2\varphi$ too. Sections 8.17 and 8.18 show that the field equation fails the third condition of Hadamard: exactly in flat 4+4 space and with frozen coefficients, and, through a standard theorem quoted without proof, in the author's metric itself (Section 8.18 states each status).

8.17 Plane waves with frozen coefficients and the mode matrix

Frozen coefficients. In the author's metric the frame factors $1/f_a$ change from point to point and in time. Near one point and for a short time one may replace them by their values at that point; this is the method of **frozen coefficients**, a device for studying an equation near one point (labelled MODEL in the notebooks, ASSUMED in the five-label scheme). It does not say that the extra times stop deflating: in the author's metric they always deflate, and Section 8.24 follows a wave along the deflating history. In flat 4+4 space the coefficients are constant and nothing is frozen. With frozen coefficients the factors are absorbed into the **frame momenta** k_a, and the term $3H\gamma^{(0)}$ is left out; it is absent exactly in the field variables χ of Section 8.30.

Plane waves. A **plane wave** is a field $\Psi = u\, e^{i(k_1x_1+k_2x_2+k_3x_3+k_5x_5+k_6x_6+k_7x_7+k_0x_0)-iEx_4}$ with a constant column u of 16 complex numbers. The number k_a is the **momentum** (or wave number) along x_a: radians of phase per unit length; E is the **frequency**. Inserting it into $\sum_a \gamma^{(a)}\partial_a\Psi = m\Psi$, line by line:

1. $\gamma^{(4)}(-iE)u + \sum_{a\neq 4} ik_a\gamma^{(a)}u = mu$, because ∂_4 of the exponential gives the factor $-iE$ and ∂_a gives ik_a (the common exponential is divided out).
2. Multiply from the left by $\gamma^{(4)}$ and use $(\gamma^{(4)})^2 = -I_{16}$: $iEu + i\sum_{a\neq 4} k_a\gamma^{(4)}\gamma^{(a)}u = m\gamma^{(4)}u$.
3. Multiply by $-i$ (and $(-i)(i) = 1$) and solve for Eu: $Eu = h_k u$ with the **mode matrix**

$$h_k = -im\gamma^{(4)} - \gamma^{(4)}\sum_{a\neq 4} k_a\gamma^{(a)}.$$

So E is an **eigenvalue** of h_k and u an **eigenvector**: $h_k u = Eu$ with $u \neq 0$. The set of all eigenvectors of one eigenvalue (with the zero column) is its **eigenspace**, and the number of independent ones is its **dimension**.

The square of the mode matrix. Write $K = \sum_{a\neq 4} k_a\gamma^{(a)}$, so that $h_k = -\gamma^{(4)}(im + K)$. Since $\gamma^{(4)}$ anticommutes with every $\gamma^{(a)}$ in K, $K\gamma^{(4)} = -\gamma^{(4)}K$ and therefore $(im+K)\gamma^{(4)} = \gamma^{(4)}(im-K)$. Line by line:

$$\begin{aligned} h_k^2 &= \gamma^{(4)}(im+K)\gamma^{(4)}(im+K) = \gamma^{(4)}\gamma^{(4)}(im-K)(im+K) \\ &= -\big((im)^2 + imK - imK - K^2\big) = m^2 + K^2, \\ K^2 &= \sum_{a\neq 4} k_a^2(\gamma^{(a)})^2 + \sum_{a<b} k_a k_b\big(\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}\big) = \big(k_1^2+k_2^2+k_3^2+k_0^2-k_5^2-k_6^2-k_7^2\big)I_{16}. \end{aligned}$$

The first line uses $(-1)^2 = 1$ and moves $(im + K)$ past $\gamma^{(4)}$; the second uses $(\gamma^{(4)})^2 = -I_{16}$ and multiplies out (the number im commutes with K); the third line computes K^2 as in Section 8.16. Hence

$$h_k^2 = E^2\, I_{16}, \qquad E^2 = m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2$$

(PROVED; `Revision/theory/reports/python-scope.json` and `wolfram-scope.json`, check `extra_time_growth_rates_unbounded`). The extra-time momenta enter with a MINUS sign, because the extra times are time-like. Every eigenvalue λ of h_k obeys $\lambda^2 = E^2$ (apply $h_k^2 = E^2$ to an eigenvector), so the eigenvalues are $+E$ and $-E$. The **trace** of h_k (the sum of its diagonal entries, which is also the sum of its eigenvalues, each counted as often as it occurs) is zero, because the trace of $\gamma^{(4)}$ and of each product of two different gammas is zero. With n_+ eigenvalues $+E$ and n_- eigenvalues $-E$, $n_+ + n_- = 16$ and $n_+E - n_-E = 0$; for $E \neq 0$ this gives $n_+ = n_- = 8$.

When is the mode matrix Hermitian? A matrix is **Hermitian** if it equals its conjugate transpose, $M^\dagger = M$, and **anti-Hermitian** if $M^\dagger = -M$. A Hermitian matrix has only real eigenvalues. The gammas are real with $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$. So $(-i\gamma^{(4)})^\dagger = i(\gamma^{(4)})^T = -i\gamma^{(4)}$: the mass term is Hermitian. For $a \neq 4$, $(\gamma^{(4)}\gamma^{(a)})^T = (\gamma^{(a)})^T(\gamma^{(4)})^T = \eta_{aa}\gamma^{(a)}(-\gamma^{(4)}) = \eta_{aa}\gamma^{(4)}\gamma^{(a)}$ (the last step anticommutes the two gammas). For a space-like direction ($\eta_{aa} = +1$) the real matrix $\gamma^{(4)}\gamma^{(a)}$ is symmetric, hence Hermitian; for an extra time ($\eta_{aa} = -1$) it is antisymmetric, hence anti-Hermitian. So the anti-Hermitian part of h_k is exactly its extra-time part, $-\gamma^{(4)}(k_5\gamma^{(5)} + k_6\gamma^{(6)} + k_7\gamma^{(7)})$, and h_k is Hermitian exactly when $k_5 = k_6 = k_7 = 0$. Fields that do not depend on the extra times are called the **good sector**. In the good sector every frequency is real, and h_k commutes with B (PROVED; `python-field-theory.json`, check `good_sector_spectrum_and_B_sectors`).

An exact example. Take $m = 1$, $k_5 = 2$ and all other momenta 0. Then $E^2 = 1 - 4 = -3$, and the eigenvalues are $E = \pm i\sqrt{3}$, eight each (PROVED; `python-field-theory.json`, check `extra_time_modes_grow`, whose detail lists the eigenvalues `-sqrt(3)*I` and `sqrt(3)*I`).

8.18 Growth without bound: the Cauchy problem is not well posed

Growth. When $E^2 < 0$, the frequencies are $E = \pm i\kappa$ with the **growth rate** $\kappa = \sqrt{-E^2} > 0$. A wave $u\, e^{-iEx_4}$ with $E = i\kappa$ behaves in time like $e^{-i(i\kappa)x_4} = e^{\kappa x_4}$: it grows exponentially (and the one with $E = -i\kappa$ decays). In the example above, $\kappa = \sqrt{3}$. With an extra-time momentum $k_5 = K$ and a space momentum k_1 (all others 0), $E^2 = m^2 + k_1^2 - K^2$, and the waves grow as soon as $K^2 > m^2 + k_1^2$, with

$$\kappa = \sqrt{K^2 - m^2 - k_1^2}.$$

No upper bound. Write $c^2 = m^2 + k_1^2$ with $c \geq 0$. For $K \geq c$ the rate satisfies $\kappa \geq K - c$: both sides are not negative, and squaring, $(K-c)^2 = K^2 - 2Kc + c^2 \leq K^2 - c^2 = \kappa^2$ is the same as $2c^2 \leq 2Kc$, which holds because $c \leq K$. So κ grows at least like K, without bound: for every rate, however large, some extra-time waves grow faster (PROVED; `python-scope.json`, check `extra_time_growth_rates_unbounded`, both engines; Notebook 08a prints $\kappa = 999.9995$ for $m = 1$, $K = 1000$).

The solution in time. The equation in time of a plane wave is $i\, du/dx_4 = h_k u$, solved by $u(x_4) = e^{-ih_k x_4}u(0)$, where the **matrix exponential** is the series $e^A = I + A + A^2/2! + A^3/3! + \ldots$. Because $h_k^2 = E^2 I_{16}$, every even power is $h_k^{2j} = E^{2j} I_{16}$ and every odd power $h_k^{2j+1} = E^{2j} h_k$. Sorting the series:

$$
\begin{aligned}
e^{-ih_k x_4} &= \sum_{j\ge 0} \frac{(-i)^{2j} E^{2j} x_4^{2j}}{(2j)!} I_{16} + \sum_{j\ge 0} \frac{(-i)^{2j+1} E^{2j} x_4^{2j+1}}{(2j+1)!} h_k \\
&= \sum_{j\ge 0} \frac{(-1)^j (Ex_4)^{2j}}{(2j)!} I_{16} - \frac{i}{E} \sum_{j\ge 0} \frac{(-1)^j (Ex_4)^{2j+1}}{(2j+1)!} h_k \\
&= \cos(Ex_4)\, I_{16} - i\, \frac{\sin(Ex_4)}{E}\, h_k .
\end{aligned}
$$

The first line separates even and odd powers; the second uses $(-i)^{2j} = (-1)^j$ and $(-i)^{2j+1} = -i(-1)^j$ and takes $1/E$ out of the odd sum; the third recognises the series of the cosine and of the sine. For $E = i\kappa$ this is $\cosh(\kappa x_4) I_{16} - i\frac{\sinh(\kappa x_4)}{\kappa} h_k$ (because $\cos(i\theta) = \cosh\theta$ and $\sin(i\theta)/i = \sinh\theta$), and for $E = 0$ it is the limit $I_{16} - i h_k x_4$.

Hadamard's third condition fails. Take as data at $x_4 = 0$ one growing wave, $\epsilon\, e^{iKx_5} u_+$, where u_+ is an eigenvector of h_k (with $k_5 = K$) for the eigenvalue $+i\kappa$, of size $u_+^\dagger u_+ = 1$. At the time x_4 the solution is $\epsilon\, e^{\kappa x_4} e^{iKx_5} u_+$. To measure the data we may even use a norm that punishes wiggly data: the **Sobolev norm of order** s counts, for one wave of momentum K and amplitude ϵ, the size $\epsilon(1+K^2)^{s/2}$ (each order of derivative brings a factor of about K). The ratio of the size of the solution at $x_4 = 1$ to the size of the data is then

$$
\frac{\epsilon\, e^{\kappa}}{\epsilon(1+K^2)^{s/2}} = e^{\kappa}(1+K^2)^{-s/2}, \qquad \ln(\text{ratio}) = \kappa - \tfrac{s}{2}\ln(1+K^2) \ge K - c - \tfrac{s}{2}\ln(1+K^2),
$$

the same for every ϵ. The right-hand side grows without bound as K grows, because the logarithm grows more slowly than any multiple of K. So for every order s data as small as we like give, after one unit of time, solutions as large as we like: the solution does not depend continuously on the data. So, in flat 4+4 space and with frozen coefficients, the initial-value problem is NOT well posed in the sense of Hadamard for data that depend on the extra times, although the slices $x_4 = \text{const}$ are non-characteristic (PROVED in this setting; `python-scope.json`, check `extra_time_growth_rates_unbounded`, both engines, whose detail names exactly this setting: "flat 4+4 space (or frozen coefficients)"). With $m = 1$ the ratio first exceeds 10^6 at $K = 13.9$ for $s = 0$, at $K = 19.9$ for $s = 2$, at $K = 27.1$ for $s = 4$ and at $K = 44.2$ for $s = 8$ (COMPUTED by Notebook 08a on a grid of step 0.1): a stronger norm only delays the rise.

From frozen coefficients to the author's metric (an ASSUMED step). In the author's metric the frame factors change from point to point, and a plane wave is not an exact solution of the field equation; so the argument above, by itself, proves nothing about the field equation in that metric. The step is made by a standard theorem of the theory of partial differential equations, the **Lax-Mizohata theorem** (P. D. Lax, 1957; S. Mizohata, 1961), which this book quotes without proof, so it is ASSUMED here. It says: if the initial-value problem of a linear system of first-order equations with smooth coefficients is well posed near a point, then at that point the frequencies of its **principal part** (the derivative terms alone, with their coefficients taken at that point) are real for every real momentum. For the field equation with $U = 0$ (then the equation is linear) the principal part at a point is the mode matrix of Section 8.17 without the mass term, with the frame momenta of that point: a wave with the coordinate momentum q_a along x_a has there the frame momentum $k_a = q_a/f_a$, with the frame factors of that point (the term $3H\gamma^{(0)}$ contains no derivative, so it does not belong to the principal part). By the formula of Section 8.17 with $m = 0$, its frequencies obey

$E^2 = k_1^2 + k_2^2 + k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2$, which is negative for a wave along an extra time alone ($k_5 \neq 0$, all other momenta zero), at every point. So, granted the theorem, the initial-value problem of the field equation in the author's metric is not well posed for data that depend on the extra times. Status: the statements in flat 4+4 space and with frozen coefficients are PROVED, and they are what the Revision record checks; the step to the author's varying coefficients rests on the quoted theorem (ASSUMED) and is not checked by the record; for $U \neq 0$ nothing more is claimed.

8.19 The Krein form is conserved; the growing waves carry Krein form zero

Two ways to measure a column. The ordinary size of a column u is the **Hilbert norm** $u^\dagger u = |u_1|^2 + \cdots + |u_{16}|^2$. The **Krein form** is $u^\dagger B u$ with the matrix $B = -iC\gamma^{(4)}$ of the record ($B^\dagger = B$, $B^2 = I_{16}$, eight eigenvalues $+1$ and eight -1; Chapters 5 and 10). Because B has negative eigenvalues, $u^\dagger B u$ can be negative or zero for $u \neq 0$. Chapter 10 explains why the canonical quantisation of dirac16complex forces this indefinite form; here we need only one property.

The Krein form is conserved. The mode matrix satisfies $Bh_k = h_k^\dagger B$ for every momentum (PROVED; `python-field-theory.json`, check `mode_hamiltonian_B_selfadjoint_dispersion`; a matrix with this property is called **self-adjoint for the Krein form**). For a solution of $du/dx_4 = -ih_k u$, line by line:

$$\begin{aligned}\frac{d}{dx_4}(u^\dagger B u) &= \Big(\frac{du}{dx_4}\Big)^\dagger B u + u^\dagger B \frac{du}{dx_4} = (-ih_k u)^\dagger B u + u^\dagger B(-ih_k u)\\ &= i\,u^\dagger h_k^\dagger B u - i\,u^\dagger B h_k u = i\,u^\dagger(h_k^\dagger B - B h_k)u = 0.\end{aligned}$$

The first step is the product rule; the second inserts the equation; the third uses $(-ih_k u)^\dagger = u^\dagger h_k^\dagger (-i)^* = i\,u^\dagger h_k^\dagger$; the last uses $Bh_k = h_k^\dagger B$. The Hilbert norm is not conserved: the same computation gives $\frac{d}{dx_4}(u^\dagger u) = i\,u^\dagger(h_k^\dagger - h_k)u$, which is not zero when h_k is not Hermitian, that is when there is extra-time momentum.

The growing waves carry Krein form zero. Let u and v be two eigenvectors of h_k for the same growing eigenvalue $i\kappa$. Then $u(x_4) = e^{\kappa x_4}u$ and $v(x_4) = e^{\kappa x_4}v$ solve the equation, and the computation above, done with two different columns, shows that $v(x_4)^\dagger B u(x_4) = e^{2\kappa x_4}\, v^\dagger B u$ is constant in time. A constant that equals $e^{2\kappa x_4}$ times a fixed number, with $\kappa > 0$, must be zero: $v^\dagger B u = 0$. So the Krein form vanishes identically on the eigenspace of a growing frequency: the eigenspace is **Krein-neutral** (PROVED by the argument just given; the pairing record proves the same and checks it on exact samples, `Revision/pairing/reports/python-pairing.json`, check `Q.one_particle_complex_frequency_Krein_neutral`). On an eigenspace of a real frequency $E \neq 0$, by contrast, the Krein form has four positive and four negative directions: its **inertia** is (4,4) (PROVED in the pairing record for every real frequency; `python-pairing.json`, checks `Q.one_particle_Krein_inertia_proof` and, for an exact sample, `Q.one_particle_Krein_inertia`). This is how the Hilbert norm can grow while the Krein form stays constant: the positive and the negative parts of the Krein form grow together and cancel.

8.20 Example: growth rates and Hadamard ill-posedness

Notebook 08a puts Sections 8.16 to 8.19 to work. It reads the author's gammas, checks the 64 Clifford relations and $(\gamma^{(4)})^2 = -I_{16}$; builds the mode matrix with symbolic mass and momenta and proves

$h_k^2 = E^2 I_{16}$ exactly, comparing with both Revision verifiers; finds the anti-Hermitian part; checks the exact example $m = 1$, $k_5 = 2$; computes the growth rate against the extra-time momentum and maps it in the plane of k_1 and k_5; solves the equation in time in two independent ways (the closed formula and RK4); computes the Hadamard amplification for four Sobolev orders; checks the conservation of the Krein form, the Krein-neutral growing eigenspace and the inertia (4,4) of a real-frequency eigenspace; and follows the eigenvalues into the complex plane. It needs no Rust and runs in well under a minute; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/08a_growth_rates.PROVENANCE.md`. It ends with the line ALL 30 CHECKS PASSED (notebook 08a).

8.21 How to run Notebook 08a

Step 1. What this notebook does and what it needs.

Notebook 08a (Growth rates of the extra-time waves and Hadamard ill-posedness) is the file `Revision/textbook/notebooks/08a_growth_rates.ipynb` of the repository Dirac_claude. It builds the 16 x 16 mode matrix of a plane wave of the field equation with frozen coefficients from the author's gamma matrices, proves exactly that its square is E^2 times the unit matrix, computes the growth rate of the waves that move along an extra time, shows that this rate has no upper bound (so, in flat 4 + 4 space and with frozen coefficients, the initial-value problem is not well posed in the sense of Hadamard), checks that the Krein form is conserved while the ordinary norm grows, and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08a_growth_rates.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08a_growth_rates.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 08a_growth_rates.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/08a.captions.json`
- `Revision/textbook/figures/08a_1_growth_rate_vs_momentum.png`
- `Revision/textbook/figures/08a_2_growth_map.png`
- `Revision/textbook/figures/08a_3_norm_in_time.png`
- `Revision/textbook/figures/08a_4_hadamard_ratio.png`
- `Revision/textbook/figures/08a_5_krein_norm.png`
- `Revision/textbook/figures/08a_6_eigenvalue_paths.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08a_6_eigenvalue_paths.png exists
ALL 30 CHECKS PASSED (notebook 08a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/08a_growth_rates.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 08a_growth_rates.ipynb
python -m nbconvert --execute --inplace 08a_growth_rates.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- FileNotFoundError naming Revision/algebra/gammas.json: the notebook reads this Revision record of the repository. Check that the folder Revision/algebra of your copy of the repository contains the file gammas.json (clone the repository again if it does not) and that you opened the notebook from its folder Revision/textbook/notebooks.

8.22 Notebook 08a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/08a_growth_rates.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field equation of dirac16complex (and of dirac16complex00, which has the same
equation) in the author's 4 + 4 dimensional primordial universe has FOUR time-like
directions: the time $x_4$ and the three extra times $x_5, x_6, x_7$. This notebook
shows what that means for the waves of the field. It

- reads the author's sixteen-by-sixteen gamma matrices from the Revision record and
  checks the 64 Clifford relations;
- builds the *mode matrix* $h_k$ of a plane wave with frozen coefficients and proves
  exactly (with sympy) that $h_k^2 = E^2\,I_{16}$ with
  $E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$;
- shows that a wave with a large momentum $K$ along an extra time does not oscillate
  but GROWS like $e^{\kappa x_4}$ with the growth rate
  $\kappa = \sqrt{K^2 - m^2 - k_0^2 - k_1^2 - k_2^2 - k_3^2}$, and that $\kappa$ has
  no upper bound;
- solves the wave equation in time in two independent ways (an exact formula and a
  step-by-step numerical method) and compares them;
- shows that therefore a tiny change of the starting values can make an arbitrarily
  large change later: the initial-value problem is NOT well posed in the sense of
  Hadamard (for starting values that depend on the extra times; exactly in flat
  4 + 4 space, and with frozen coefficients in the author's metric);
- checks that the Krein form $u^\dagger B u$ stays constant while the ordinary size
  $u^\dagger u$ grows, and that the growing waves have Krein form zero;
- shows that without extra-time momentum (the *good sector*) every frequency is real.

Every exact statement reproduces a check of a Revision record, named in the PASS
line. The notebook draws six teaching plots.
```

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 08a (Growth rates of the extra-time waves and Hadamard ill-posedness) is the file ‘Revision/textbook/notebooks/08a_growth_rates.ipynb‘ of the repository Dirac_claude. It builds the 16 x 16 mode matrix of a plane wave of the field equation with frozen coefficients from the author’s gamma matrices, proves exactly that its square is E^2 times the unit matrix, computes the growth rate of the waves that move along an extra time, shows that this rate has no upper bound (so, in flat 4 + 4 space and with frozen coefficients, the initial-value problem is not well posed in the sense of Hadamard), checks that the Krein form is conserved while the ordinary norm grows, and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08a_growth_rates.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08a_growth_rates.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 08a_growth_rates.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/08a.captions.json‘
- ‘Revision/textbook/figures/08a_1_growth_rate_vs_momentum.png‘
- ‘Revision/textbook/figures/08a_2_growth_map.png‘
- ‘Revision/textbook/figures/08a_3_norm_in_time.png‘
- ‘Revision/textbook/figures/08a_4_hadamard_ratio.png‘
- ‘Revision/textbook/figures/08a_5_krein_norm.png‘
- ‘Revision/textbook/figures/08a_6_eigenvalue_paths.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08a_6_eigenvalue_paths.png exists
ALL 30 CHECKS PASSED (notebook 08a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/08a_growth_rates.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 08a_growth_rates.ipynb
python -m nbconvert --execute --inplace 08a_growth_rates.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- FileNotFoundError naming Revision/algebra/gammas.json: the notebook reads this Revision record of the repository. Check that the folder Revision/algebra of your copy of the repository contains the file gammas.json (clone the repository again if it does not) and that you opened the notebook from its folder Revision/textbook/notebooks.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 08a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 08a (Growth rates of the extra-time waves and Hadamard ill-posedness) is the
# file Revision/textbook/notebooks/08a_growth_rates.ipynb of the repository Dirac_claude.
# It builds the 16 x 16 mode matrix of a plane wave of the field equation with frozen
# coefficients from the author’s gamma matrices, proves exactly that its square is E^2
# times the unit matrix, computes the growth rate of the waves that move along an extra
# time, shows that this rate has no upper bound (so, in flat 4 + 4 space and with frozen
```

```
# coefficients, the initial-value problem is not well posed in the sense of Hadamard),
# checks that the Krein form is conserved while the ordinary norm grows, and draws six
# teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
# shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
```

```
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
```

```
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 08a_growth_rates.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 08a_growth_rates.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 08a_growth_rates.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/08a.captions.json
# - Revision/textbook/figures/08a_1_growth_rate_vs_momentum.png
# - Revision/textbook/figures/08a_2_growth_map.png
# - Revision/textbook/figures/08a_3_norm_in_time.png
```

```
# - Revision/textbook/figures/08a_4_hadamard_ratio.png
# - Revision/textbook/figures/08a_5_krein_norm.png
# - Revision/textbook/figures/08a_6_eigenvalue_paths.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 08a_6_eigenvalue_paths.png exists
#     ALL 30 CHECKS PASSED (notebook 08a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/08a_growth_rates.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 08a_growth_rates.ipynb
#     python -m nbconvert --execute --inplace 08a_growth_rates.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
```

```
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - FileNotFoundError naming Revision/algebra/gammas.json: the notebook reads this
#   Revision record of the repository. Check that the folder Revision/algebra of your
#   copy of the repository contains the file gammas.json (clone the repository again if
#   it does not) and that you opened the notebook from its folder
#   Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "08a"  # this notebook: chapter 08, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative
```

```
def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
```

```
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 08a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$ (the author's names): $x_1, x_2, x_3$ ordinary
  space (it inflates); $x_4$ the time; $x_5, x_6, x_7$ the three *extra times*,
  time-like directions that DEFLATE exponentially (their lengths carry the factor
  $e^{-a_4}\sin^{1/6} z$ with $a_4$ increasing); $x_0$ the hidden space direction,
  $z = 6 H x_0$ between $0$ and $\pi/2$.
- **Gamma matrices** $\gamma^{(0)}, \dots, \gamma^{(7)}$: sixteen-by-sixteen matrices
  of the numbers $0, 1, -1$ with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}
  = 2\eta^{ab} I_{16}$ (the *Clifford relation*), $\eta = \mathrm{diag}(+1, +1, +1,
  +1, -1, -1, -1, -1)$ in the order $x_0, \dots, x_7$: a gamma squares to $+1$ for a
  space-like direction and to
  $-1$ for a time-like one. $I_{16}$ is the unit matrix.
- **Plane wave**: a field of the form $\Psi = u\, e^{i(k_0 x_0 + \dots + k_7 x_7)
  - i E x_4}$ (no $k_4$ term) with a constant column $u$ of 16 complex numbers.
  $k_a$ is the *wave number* or *momentum* along $x_a$ (radians of phase per unit
  length) and $E$ the *frequency*.
- **Frozen coefficients**: in a curved space the factors in front of the derivatives
  change from point to point and in time; near one point and for a short time one
  replaces them by their values at that point. The momenta measured with these
  factors are the *frame momenta* $k_a$. This is a device of the analysis near one
  point (labelled MODEL); it does NOT say that the extra times stop deflating: in
```

the author's metric they always deflate exponentially, and Notebook 08b follows a wave along the deflating history.
- **Mode matrix** h_k: the 16×16 complex matrix with $E u = h_k u$.
- **Eigenvalue, eigenvector**: E and $u \neq 0$ with $h_k u = E u$; the *eigenspace* of E is the set of all such u; its *dimension* is the number of independent ones.
- **Oscillation and growth**: e^{-iEx_4} oscillates (its size stays 1) when E is real; when $E = \pm i\kappa$ with $\kappa > 0$ it is $e^{\pm\kappa x_4}$, which grows or decays exponentially. κ is the **growth rate**.
- **Conjugate transpose** $M^\dagger$: transpose the matrix and replace every entry by its complex conjugate. **Hermitian**: $M^\dagger = M$; a Hermitian matrix has only real eigenvalues.
- **Hilbert norm** $u^\dagger u = |u_1|^2 + \dots + |u_{16}|^2$ (the ordinary size squared) and **Krein form** $u^\dagger B u$ with the matrix $B = -iC\gamma^{(4)}$ of the record ($B^\dagger = B$, $B^2 = 1$, eight eigenvalues $+1$ and eight -1); the Krein form can be negative or zero for $u \neq 0$.
- **Good sector**: the fields that do not depend on x_5, x_6, x_7 ($k_5 = k_6 = k_7 = 0$).
- **Initial-value (Cauchy) problem**: find the field at later times x_4 from its values at $x_4 = 0$ (the *data*). It is **well posed in the sense of Hadamard** if a solution exists, is unique, and depends *continuously* on the data: a small enough change of the data changes the solution at a later time only a little.
- **Sobolev norm of order s**: a way to measure the size of data that also counts how fast they wiggle; for one wave of momentum K and amplitude ϵ it is $\epsilon\,(1 + K^2)^{s/2}$. Larger s punishes wiggly data more.
- **RK4**: the classical fourth-order Runge-Kutta method, a step-by-step numerical method for equations $du/dt = F(t, u)$.

Text cell:

```
## 4. The physical and mathematical situation

The field equation (with $U = 0$) in the author's metric, in the diagonal frame, is

$$\tan z\,\gamma^{(0)}\partial_0\Psi + e^{-a_4}\sin^{-1/6}z\,\gamma^{(i)}\partial_i\Psi
+ \gamma^{(4)}\partial_4\Psi + e^{a_4}b_4^{-1/2}\sin^{-1/6}z\,\gamma^{(t)}\partial_t\Psi
+ M\Psi = m\Psi$$

(sums over $i = 1, 2, 3$ and $t = 5, 6, 7$; $b_4(x_4) > 0$ is the second function
of the author's metric, and the backgrounds of the record have $b_4 = 1$). $M =
3H\gamma^{(0)} + \frac34(b_4'/b_4)\gamma^{(4)}$ is the gravitational term (Chapter
6); it has no derivative, so it does not change the growth of short waves. It
disappears exactly in the variables $\chi = \sin^{1/2}z\,b_4^{3/4}\Psi$ (Revision
check 'rescaling_removes_the_connection_term', at $b_4 = 1$). With frozen
coefficients the factors $e^{-a_4}\sin^{-1/6}z$, $e^{a_4}b_4^{-1/2}\sin^{-1/6}z$ and
$\tan z$ become constants and are absorbed into the frame momenta $k_a$; in flat
4 + 4 space they are exactly 1. (Freezing them is a local MODEL: along the history
$a_4(x_4)$ the extra-time factor $e^{a_4}$ keeps growing, so the frame momentum of
a given wave along an extra time keeps growing too; Notebook 08b computes that on
the background $b_4 = 1$.) So for a plane wave the equation reads, line by
line:

1. $\gamma^{(4)}(-iE)u + \sum_{a \neq 4} i k_a \gamma^{(a)} u = m u$, because
```

```
   $\partial_4$ gives the factor $-iE$ and $\partial_a$ the factor $i k_a$.
2. Multiply from the left by $\gamma^{(4)}$ and use $(\gamma^{(4)})^2 = -1$:
   $iE u + i\sum_a k_a \gamma^{(4)}\gamma^{(a)} u = m\gamma^{(4)} u$.
3. Multiply by $-i$ and solve for $E u$:
   $E u = h_k u$ with $h_k = -i m\gamma^{(4)} - \gamma^{(4)}\sum_{a \neq 4}
   k_a\gamma^{(a)}$.

Squaring $h_k$ with the Clifford relation gives $h_k^2 = E^2 I_{16}$ with
$E^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$: the
extra-time momenta enter with a MINUS sign, because the extra times are time-like.
When $k_5^2 + k_6^2 + k_7^2 > m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2$, $E^2 < 0$, the
frequencies are $\pm i\kappa$, and half of the waves grow like $e^{\kappa x_4}$.

Units: $m$, every $k_a$ and $\kappa$ are measured in the same unit (an inverse
length) and $x_4$ in its inverse; every number below is a pure number in this unit.
```

Text cell:

```
## 5. The author's gamma matrices

The next cell first defines two helpers for the checks that reproduce a Revision
record. `record_says(report, name, ...)` opens the report (a JSON file with a list
of checks, each with a name, a verdict and a detail text) and is true when the
check `name` is there with the verdict pass and its detail text contains every
further piece of text given (for example a number that this notebook computes).
`check_record(condition, title, report, name, ...)` is the helper `check` for such
a result: it passes only if the notebook's own computation (`condition`) is right
AND the record says the same. Then the cell reads the record
`Revision/algebra/gammas.json`, turns the eight gamma matrices and the matrix $B$
into numpy arrays and checks the Clifford relation for all $8 \times 8 = 64$
ordered pairs $(a, b)$ with exact integer arithmetic. In Python lists count from 0,
like the author's coordinates: $\gamma^{(x_0)}$ is `GAMMA[0]` and $\gamma^{(x_4)}$
is `GAMMA[4]`. Last, it
checks $(\gamma^{(x_4)})^2 = -I_{16}$: multiplying the field equation by
$-\gamma^{(x_4)}$ solves it for $\partial_4\Psi$, so the field equation says how the
field changes in the time $x_4$ (the slices $x_4 = $ const are *non-characteristic*).
Section 12 shows that this does NOT make the initial-value problem well posed.
```

In [2]:

```
import numpy as np  # arrays of numbers, matrices, linear algebra
import sympy as sp  # exact algebra with symbols

REPORTS = {}  # report file -> {check name: (verdict, detail)}, each read once


def record_says(report_file, check_name, *pieces):
    """True when the Revision report records the check check_name with the verdict
    pass and its detail text contains every given piece of text."""
    if report_file not in REPORTS:  # read the report the first time it is needed
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        REPORTS[report_file] = {entry["name"]: (entry["verdict"].lower(),
```

```
                                            entry["detail"])
                                for entry in data["checks"]}
    verdict, detail = REPORTS[report_file][check_name]
    return verdict == "pass" and all(piece in detail for piece in pieces)


def check_record(condition, title, report_file, check_name, *pieces):
    """check() for a result that reproduces the Revision check check_name: it passes
    only if condition is true AND the report records check_name as passed, with
    every piece of text (values computed here) in its detail."""
    on_record = record_says(report_file, check_name, *pieces)
    check(condition and on_record, title,
          record=f"{report_file}, check {check_name}")


gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
# The record stores every matrix as a list of rows of whole numbers.
GAMMA = [np.array(rows, dtype=int) for rows in gammas_record["gamma"]]
ETA = gammas_record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1] in the order x0 ... x7
# B = -i C gamma^(x4) is stored as its real part and its imaginary part.
B = np.array(gammas_record["B"]["re"]) + 1j * np.array(gammas_record["B"]["im"])
I16 = np.eye(16, dtype=int)  # the 16 x 16 unit matrix
say(f"eta = {ETA}; each gamma is a {GAMMA[0].shape[0]} x {GAMMA[0].shape[1]} matrix")
clifford_ok = all(
    np.array_equal(GAMMA[a] @ GAMMA[b] + GAMMA[b] @ GAMMA[a],
                   2 * ETA[a] * (a == b) * I16)  # (a == b) is 1 or 0
    for a in range(8) for b in range(8))
check_record(clifford_ok, "the 64 Clifford relations {gamma^a, gamma^b} = 2 eta^ab I16",
             "Revision/algebra/reports/python-algebra.json", "clifford_relation",
             "for all 64 ordered pairs")
check(np.allclose(B, B.conj().T) and np.allclose(B @ B, np.eye(16)),
      "B is Hermitian and B^2 = 1")
# (gamma^(x4))^2 = -1: the field equation can be solved for d4 Psi (multiply it by
# -gamma^(x4)), so the slices x4 = const are non-characteristic.
check_record(np.array_equal(GAMMA[4] @ GAMMA[4], -I16),
             "(gamma^(x4))^2 = -1: the slices x4 = const are non-characteristic",
             "Revision/theory/reports/wolfram-field-theory.json", "evolution_form_G",
             "the slices x4 = const are non-characteristic")
```

Out [2]:

```
eta = [1, 1, 1, 1, -1, -1, -1, -1]; each gamma is a 16 x 16 matrix
PASS the 64 Clifford relations {gamma^a, gamma^b} = 2 eta^ab I16
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
PASS B is Hermitian and B^2 = 1
PASS (gamma^(x4))^2 = -1: the slices x4 = const are non-characteristic
     reproduces Revision/theory/reports/wolfram-field-theory.json, check evolution_form_G
```

Text cell:

```
## 6. The mode matrix and its square, exactly
```

```
The next cell builds $h_k = -i m\gamma^{(4)} - \gamma^{(4)}\sum_{a \neq 4}
k_a\gamma^{(a)}$ with sympy, where $m$ and the seven momenta are symbols (letters
that stand for any real numbers), and checks three exact statements: $h_k^2 = E^2
I_{16}$ with the $E^2$ of section 4; the trace of $h_k$ (the sum of its diagonal) is
0, so its eigenvalues $+E$ and $-E$ occur equally often (8 times each); and
$B h_k = h_k^\dagger B$, which says that $h_k$ is *self-adjoint for the Krein form*
(section 13 uses it). sympy prints a power with two stars: `k1**2` means $k_1^2$.
The Revision record was checked by two independent programs, one in Python (sympy)
and one in the Wolfram language; the cell checks that the reports of BOTH contain
exactly the $E^2$ computed here.
```

In [3]:

```
m = sp.Symbol("m", real=True)  # the mass
k = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}  # momenta
G = [sp.Matrix(rows) for rows in gammas_record["gamma"]]  # the gammas, exact
B_exact = sp.I * sp.Matrix(gammas_record["B"]["im"])  # B is purely imaginary
g4 = G[4]  # gamma^(x4)
momentum_part = sp.zeros(16, 16)
for a, k_a in k.items():  # a runs over 0, 1, 2, 3, 5, 6, 7
    momentum_part += k_a * g4 * G[a]  # k_a gamma^(x4) gamma^(xa)
h = -sp.I * m * g4 - momentum_part  # the mode matrix h_k
E2 = m**2 + k[0]**2 + k[1]**2 + k[2]**2 + k[3]**2 - k[5]**2 - k[6]**2 - k[7]**2
say(f"E^2 = {E2}")
square_minus_E2 = (h * h - E2 * sp.eye(16)).applyfunc(sp.expand)
square_ok = square_minus_E2 == sp.zeros(16, 16)
# The two independent Revision verifiers print E^2 in their own notations: sympy
# writes powers with **, the Wolfram verifier with ^. Both texts must hold this E^2.
E2_sympy_text = sp.sstr(E2)  # "k0**2 + k1**2 + ... + m**2"
E2_wolfram_text = E2_sympy_text.replace("**", "^")  # "k0^2 + k1^2 + ... + m^2"
check_record(square_ok,
             "h_k^2 = E^2 I16, E^2 = m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 "
             "- k7^2", "Revision/theory/reports/python-scope.json",
             "extra_time_growth_rates_unbounded", f"h_k^2 = ({E2_sympy_text}) I16")
check_record(square_ok, "the same E^2 in the independent Wolfram verifier",
             "Revision/theory/reports/wolfram-scope.json",
             "extra_time_growth_rates_unbounded", f"h_k^2 = ({E2_wolfram_text}) I16")
check(sp.expand(h.trace()) == 0, "the trace of h_k is 0 (eigenvalues +E, -E, 8 each)")
krein = (B_exact * h - h.H * B_exact).applyfunc(sp.expand)  # .H: conjugate transpose
check_record(krein == sp.zeros(16, 16), "B h_k = h_k^dagger B (Krein self-adjoint)",
             "Revision/theory/reports/python-field-theory.json",
             "mode_hamiltonian_B_selfadjoint_dispersion", "B h = h^dagger B")
```

Out [3]:

```
E^2 = k0**2 + k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + m**2
PASS h_k^2 = E^2 I16, E^2 = m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 - k7^2
     reproduces Revision/theory/reports/python-scope.json, check
    extra_time_growth_rates_unbounded
PASS the same E^2 in the independent Wolfram verifier
     reproduces Revision/theory/reports/wolfram-scope.json, check
    extra_time_growth_rates_unbounded
```

```
PASS the trace of h_k is 0 (eigenvalues +E, -E, 8 each)
PASS B h_k = h_k^dagger B (Krein self-adjoint)
     reproduces Revision/theory/reports/python-field-theory.json, check
    mode_hamiltonian_B_selfadjoint_dispersion
```

Text cell:

```
## 7. When is the mode matrix Hermitian?

Split $h_k$ into its Hermitian part $(h_k + h_k^\dagger)/2$ and its anti-Hermitian
part $(h_k - h_k^\dagger)/2$. The next cell shows that the anti-Hermitian part is
exactly $-\gamma^{(4)}(k_5\gamma^{(5)} + k_6\gamma^{(6)} + k_7\gamma^{(7)})$, the
extra-time part, and that its square is $-(k_5^2 + k_6^2 + k_7^2) I_{16}$. A matrix
whose square is a negative number times $I_{16}$ is zero only when that number is
zero: $h_k$ is Hermitian exactly when there is no extra-time momentum.
```

In [4]:

```
anti = ((h - h.H) / 2).applyfunc(sp.expand)  # the anti-Hermitian part of h_k
extra_time_part = -(k[5] * g4 * G[5] + k[6] * g4 * G[6] + k[7] * g4 * G[7])
check((anti - extra_time_part).applyfunc(sp.expand) == sp.zeros(16, 16),
      "the anti-Hermitian part of h_k is the extra-time part")
anti_square = (anti * anti).applyfunc(sp.expand)
check(anti_square == (-(k[5]**2 + k[6]**2 + k[7]**2) * sp.eye(16)).applyfunc(
    sp.expand), "its square is -(k5^2 + k6^2 + k7^2) I16")
```

Out [4]:

```
PASS the anti-Hermitian part of h_k is the extra-time part
PASS its square is -(k5^2 + k6^2 + k7^2) I16
```

Text cell:

```
## 8. An exact example: $m = 1$, $k_5 = 2$

Put $m = 1$, $k_5 = 2$ and every other momentum 0. Then $E^2 = 1 - 4 = -3$, so the
eigenvalues must be $\pm i\sqrt3$. The next cell checks $h^2 = -3 I_{16}$ exactly,
computes the dimension of the eigenspace of $+i\sqrt3$ exactly (16 minus the rank of
$h - i\sqrt3\,I_{16}$), and prints the numerical eigenvalues. The waves of the
eigenvalue $+i\sqrt3$ behave like $e^{-i(i\sqrt3)x_4} = e^{\sqrt3\,x_4}$: they grow.
The Revision record lists the same two exact eigenvalues in sympy's notation,
`-sqrt(3)*I` and `sqrt(3)*I` (`I` is sympy's $i$); the check compares with that text.
```

In [5]:

```
sample = {m: 1, k[5]: 2, k[0]: 0, k[1]: 0, k[2]: 0, k[3]: 0, k[6]: 0, k[7]: 0}
h_sample = h.subs(sample)  # an exact matrix of numbers
check(h_sample * h_sample == -3 * sp.eye(16), "m = 1, k5 = 2: h^2 = -3 I16")
```

```
dimension = 16 - (h_sample - sp.I * sp.sqrt(3) * sp.eye(16)).rank()
report("dimension of the eigenspace of +i sqrt(3)", dimension)
eigenvalues = np.linalg.eigvals(np.array(h_sample, dtype=complex))
# Round to 9 digits and count how often each value occurs (sorted, so the printed
# order is always the same).
rounded = sorted({complex(0.0, round(v.imag, 9)) for v in eigenvalues},
                 key=lambda v: v.imag)  # the real parts are zero (checked below)
for value in rounded:
    count = int(np.sum(np.abs(eigenvalues - value) < 1e-6))
    say(f"eigenvalue {value.imag:+.9f} i occurs {count} times")
# The record lists the exact eigenvalues as sympy writes them, sorted as text:
exact_text = sorted(str(sign * sp.sqrt(3) * sp.I) for sign in (1, -1))
say("exact eigenvalues as sympy writes them: " + ", ".join(exact_text))
check_record(dimension == 8 and np.max(np.abs(eigenvalues.real)) < 1e-9
             and all(abs(abs(v.imag) - 3**0.5) < 1e-9 for v in eigenvalues),
             "m = 1, k5 = 2: eigenvalues +i sqrt(3) and -i sqrt(3), 8 each",
             "Revision/theory/reports/python-field-theory.json",
             "extra_time_modes_grow", f"m = 1, k5 = 2: eigenvalues {exact_text}")
```

Out [5]:

```
PASS m = 1, k5 = 2: h^2 = -3 I16
RESULT dimension of the eigenspace of +i sqrt(3) = 8
eigenvalue -1.732050808 i occurs 8 times
eigenvalue +1.732050808 i occurs 8 times
exact eigenvalues as sympy writes them: -sqrt(3)*I, sqrt(3)*I
PASS m = 1, k5 = 2: eigenvalues +i sqrt(3) and -i sqrt(3), 8 each
     reproduces Revision/theory/reports/python-field-theory.json, check
    extra_time_modes_grow
```

Text cell:

```
## 9. The growth rate versus the extra-time momentum

From now on the computations are numerical (numpy). The function ‘mode_matrix‘ makes
$h_k$ from a mass and a dictionary of momenta. With only an extra-time momentum
$k_5 = K$ (and possibly a space momentum $k_1$), $E^2 = m^2 + k_1^2 - K^2$; the
growth rate is $\kappa = \sqrt{K^2 - m^2 - k_1^2}$ when this is positive and 0
otherwise. The next cell compares this formula with the largest imaginary part of
the numerically computed eigenvalues at 61 momenta (to $10^{-10}$; at the single
point $K = m$, where $h_k^2 = 0$, a computer finds the double eigenvalue 0 only to
about $10^{-8}$, so there the tolerance is $10^{-6}$), and draws $\kappa$ against
$K$: on the left for three masses, on the right for three space momenta $k_1$.
```

In [6]:

```
def mode_matrix(mass, momenta):
    """h_k = -i m gamma^(x4) - gamma^(x4) sum_a k_a gamma^(xa); momenta: {a: k_a}."""
    g4n = GAMMA[4]
    result = -1j * mass * g4n.astype(complex)
    for a, k_a in momenta.items():  # a is the direction: GAMMA[a] is gamma^(xa)
```

```
        result = result - k_a * (g4n @ GAMMA[a])
    return result


def growth_rate(mass, K, k1=0.0):
    """kappa = sqrt(K^2 - m^2 - k1^2) where positive, else 0 (numpy arrays too)."""
    return np.sqrt(np.maximum(K**2 - mass**2 - k1**2, 0.0))


K_values = np.linspace(0.0, 6.0, 61)  # extra-time momenta 0, 0.1, ..., 6
numerical = np.array([np.max(np.linalg.eigvals(mode_matrix(1.0, {5: K})).imag)
                      for K in K_values])  # the largest growth rate, m = 1
deviation = np.abs(numerical - growth_rate(1.0, K_values))
at_threshold = np.abs(K_values - 1.0) < 1e-12  # K = m exactly, where h_k^2 = 0
check(np.max(deviation[~at_threshold]) < 1e-10 and np.max(deviation) < 1e-6,
      "growth rate from the eigenvalues = sqrt(K^2 - m^2) (m = 1)")
report("growth rate at m = 1, K = 2 (sqrt 3)", f"{growth_rate(1.0, 2.0):.9f}")
report("growth rate at m = 1, K = 1000", f"{growth_rate(1.0, 1000.0):.6f}")
check_record(growth_rate(1.0, 1000.0) > 999.0 and growth_rate(1.0, 1e6) > 999999.0,
             "the growth rate has no upper bound (kappa/K tends to 1)",
             "Revision/theory/reports/python-scope.json",
             "extra_time_growth_rates_unbounded", "which has no upper bound")

K_fine = np.linspace(0.0, 6.0, 601)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for mass, style in ((0.5, "-"), (1.0, "--"), (2.0, ":")):
    left.plot(K_fine, growth_rate(mass, K_fine), style, label=f"$m = {mass}$")
left.plot(K_values[::3], numerical[::3], "o", markersize=4, color="black",
          label="eigenvalues, $m = 1$")
left.plot(K_fine, K_fine, color="gray", linewidth=0.8, label="$\\kappa = K$")
left.set_xlabel("extra-time momentum $K = k_5$")
left.set_ylabel("growth rate $\\kappa$")
left.set_title("$k_1 = 0$: three masses")
left.legend(loc="upper left")
for k1, style in ((0.0, "-"), (2.0, "--"), (4.0, ":")):
    right.plot(K_fine, growth_rate(1.0, K_fine, k1), style, label=f"$k_1 = {k1}$")
right.plot(K_fine, K_fine, color="gray", linewidth=0.8, label="$\\kappa = K$")
right.set_xlabel("extra-time momentum $K = k_5$")
right.set_title("$m = 1$: three space momenta $k_1$")
right.legend(loc="upper left")
save_figure(fig, "growth_rate_vs_momentum",
            "The growth rate $\\kappa = \\sqrt{K^2 - m^2 - k_1^2}$ of the growing "
            "waves against the momentum $K = k_5$ along the extra time $x_5$ "
            "(frozen coefficients); both axes in the same unit, an inverse length. "
            "Left: masses $m = 0.5, 1, 2$ with $k_1 = 0$, and the black dots are "
            "the largest imaginary parts of the computed eigenvalues of the mode "
            "matrix for $m = 1$. Right: $m = 1$ with space momenta $k_1 = 0, 2, 4$. "
            "Below the threshold $K^2 = m^2 + k_1^2$ the waves oscillate and "
            "$\\kappa = 0$; above it every curve approaches the gray line "
            "$\\kappa = K$, so the rate has no upper bound.")
```

Out [6]:

```
PASS growth rate from the eigenvalues = sqrt(K^2 - m^2) (m = 1)
RESULT growth rate at m = 1, K = 2 (sqrt 3) = 1.732050808
RESULT growth rate at m = 1, K = 1000 = 999.999500
PASS the growth rate has no upper bound (kappa/K tends to 1)
     reproduces Revision/theory/reports/python-scope.json, check
    extra_time_growth_rates_unbounded
```

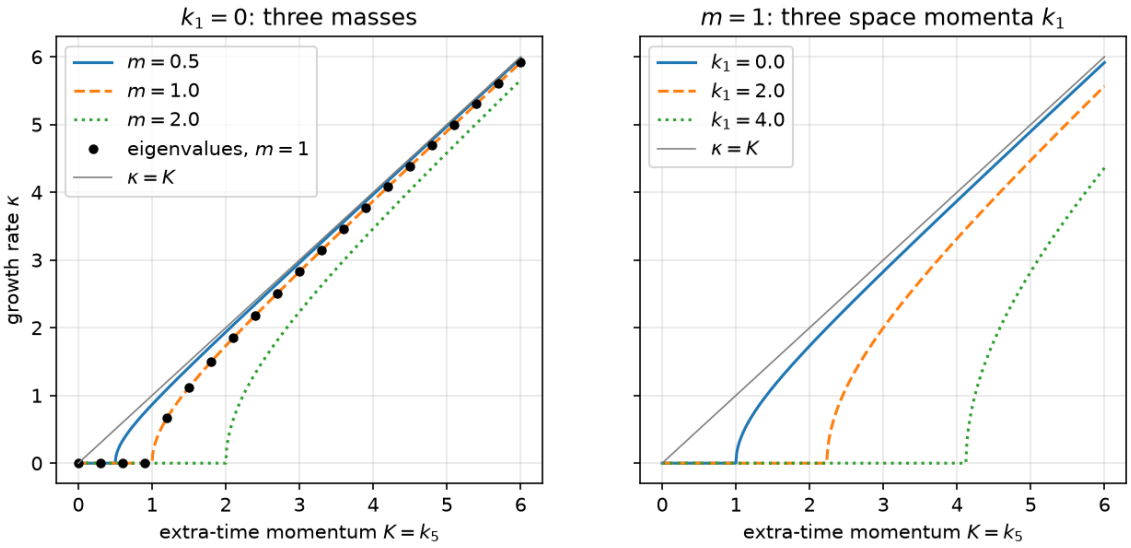

Figure 234: The growth rate $\kappa = \sqrt{K^2 - m^2 - k_1^2}$ of the growing waves against the momentum $K = k_5$ along the extra time x_5 (frozen coefficients); both axes in the same unit, an inverse length. Left: masses $m = 0.5, 1, 2$ with $k_1 = 0$, and the black dots are the largest imaginary parts of the computed eigenvalues of the mode matrix for $m = 1$. Right: $m = 1$ with space momenta $k_1 = 0, 2, 4$. Below the threshold $K^2 = m^2 + k_1^2$ the waves oscillate and $\kappa = 0$; above it every curve approaches the gray line $\kappa = K$, so the rate has no upper bound. (Notebook 08a, figure 1.)

Out [6]:

```
Figure 08a.1 saved as Revision/textbook/figures/08a_1_growth_rate_vs_momentum.png
```

Text cell:

```
## 10. Where the waves oscillate and where they grow

The next cell computes the growth rate on a grid of $201 \times 201$ points of the
plane of the space momentum $k_1$ and the extra-time momentum $k_5$ (with $m = 1$)
and draws it as a colour map. The white region is where $E^2 > 0$ (oscillation);
its border is the curve $k_5^2 - k_1^2 = 1$ (a hyperbola), and the contour lines
are the curves of equal growth rate $\kappa = 1, 2, 3, 4$.
```

In [7]:

```
k1_grid, k5_grid = np.meshgrid(np.linspace(-5.0, 5.0, 201), np.linspace(-5.0, 5.0, 201))
E2_grid = 1.0 + k1_grid**2 - k5_grid**2  # m = 1
```

```
kappa_grid = np.sqrt(np.maximum(-E2_grid, 0.0))
growing_fraction = float(np.mean(E2_grid < 0.0))
report("fraction of the grid where the waves grow", f"{growing_fraction:.4f}")
check(abs(kappa_grid[100, 100]) == 0.0 and abs(kappa_grid[200, 100] - 24**0.5) < 1e-12,
      "kappa = 0 at k = 0 and kappa = sqrt(24) at k1 = 0, k5 = 5")

fig, ax = plt.subplots(figsize=(6.0, 5.0))
shown = np.where(E2_grid < 0.0, kappa_grid, np.nan)  # nan: not coloured (white)
image = ax.pcolormesh(k1_grid, k5_grid, shown, cmap="viridis", shading="auto")
fig.colorbar(image, ax=ax, label="growth rate $\\kappa$")
lines = ax.contour(k1_grid, k5_grid, kappa_grid, levels=[1, 2, 3, 4], colors="white",
                   linewidths=0.8)
ax.clabel(lines, fmt="%d", fontsize=8)
ax.contour(k1_grid, k5_grid, E2_grid, levels=[0.0], colors="black", linewidths=1.2)
ax.text(2.6, 0.0, "oscillation\n$E^2 > 0$", ha="center", va="center")
ax.set_xlabel("space momentum $k_1$")
ax.set_ylabel("extra-time momentum $k_5$")
ax.set_title("Growth rate of the waves, $m = 1$")
save_figure(fig, "growth_map",
            "The growth rate $\\kappa$ in the plane of the space momentum $k_1$ "
            "(horizontal) and the extra-time momentum $k_5$ (vertical) for $m = 1$, "
            "both in the same unit, an inverse length; frozen coefficients. White: "
            "$E^2 = 1 + k_1^2 - k_5^2 > 0$, the waves oscillate. Coloured: the waves "
            "grow; the black curve $k_5^2 - k_1^2 = 1$ is the border, and the white "
            "contour lines mark $\\kappa = 1, 2, 3, 4$. The two coloured regions "
            "reach to infinity: however large $k_1$ is, a large enough $k_5$ "
            "makes the waves grow.")
```

Out [7]:

```
RESULT fraction of the grid where the waves grow = 0.4439
PASS kappa = 0 at k = 0 and kappa = sqrt(24) at k1 = 0, k5 = 5
```

Out [7]:

```
Figure 08a.2 saved as Revision/textbook/figures/08a_2_growth_map.png
```

Text cell:

```
## 11. Solving in time, in two independent ways

The equation in time is $i\,du/dx_4 = h_k u$. Because $h_k^2 = E^2 I_{16}$, the
exponential series of $e^{-ih_k x_4}$ collapses to
$u(x_4) = \big(\cos(Ex_4) I_{16} - i\,\frac{\sin(Ex_4)}{E}\,h_k\big)\,u(0)$:
the even powers of $h_k$ are powers of $E^2$ and give the cosine, the odd ones give
$h_k$ times the sine. For $E = i\kappa$ this is $\cosh(\kappa x_4) I_{16} -
i\,\frac{\sinh(\kappa x_4)}{\kappa}h_k$, and for $E = 0$ it is $I_{16} - i h_k x_4$
(the limit). The next cell implements this formula (‘evolve‘) and, independently,
the RK4 method with 3000 steps (‘rk4‘), starts both from the same random column
(fixed seed, so every run uses the same numbers) and compares them at $x_4 = 3$.
```

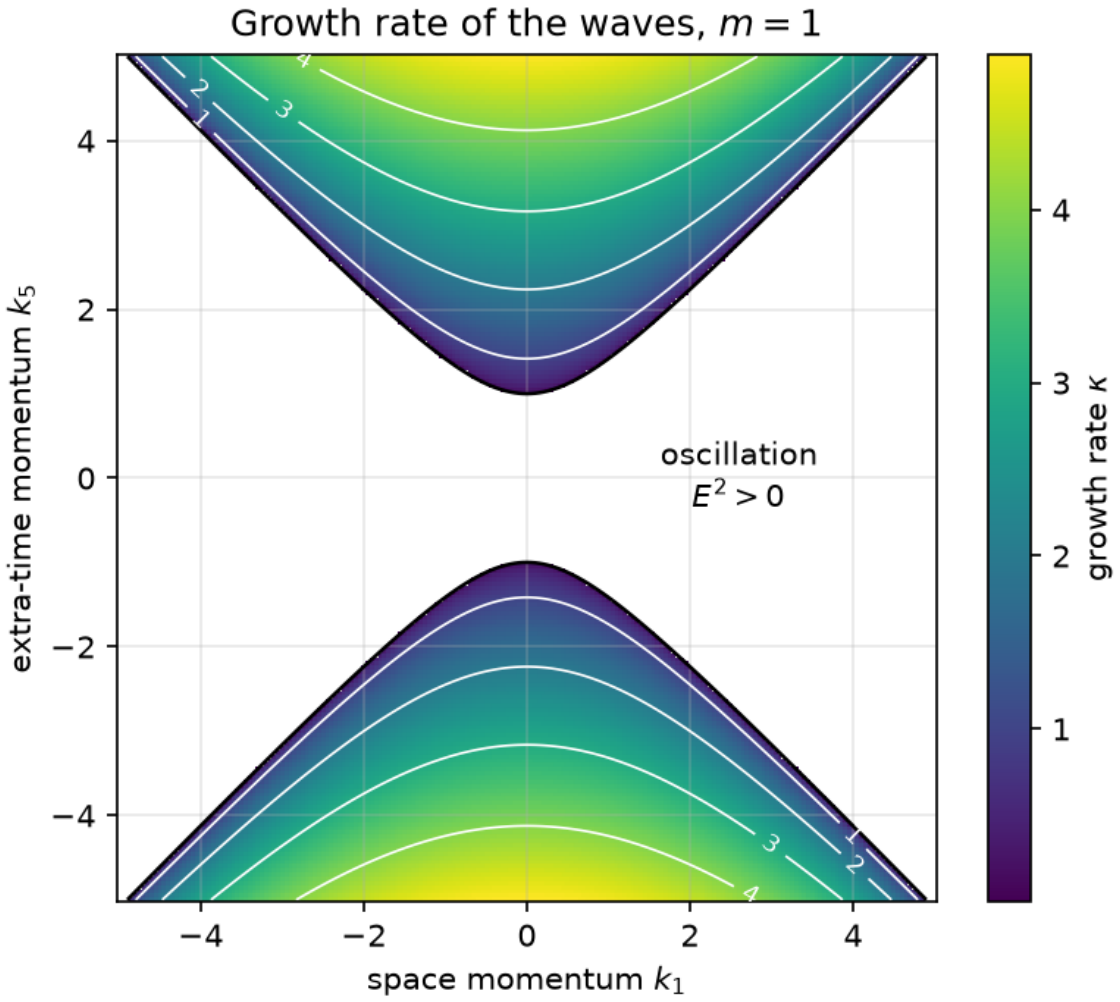

Figure 235: The growth rate κ in the plane of the space momentum k_1 (horizontal) and the extra-time momentum k_5 (vertical) for $m = 1$, both in the same unit, an inverse length; frozen coefficients. White: $E^2 = 1 + k_1^2 - k_5^2 > 0$, the waves oscillate. Coloured: the waves grow; the black curve $k_5^2 - k_1^2 = 1$ is the border, and the white contour lines mark $\kappa = 1, 2, 3, 4$. The two coloured regions reach to infinity: however large k_1 is, a large enough k_5 makes the waves grow. (Notebook 08a, figure 2.)

In [8]:

```
def evolve(hk, E2_value, x4, u0):
    """u(x4) = (cos(E x4) - i sin(E x4)/E h_k) u0, with E = sqrt(E^2) (complex)."""
    if E2_value == 0.0:
        return u0 - 1j * x4 * (hk @ u0)  # the limit E -> 0
    E = np.sqrt(complex(E2_value))  # E is imaginary when E^2 < 0
    return np.cos(E * x4) * u0 - 1j * np.sin(E * x4) / E * (hk @ u0)


def rk4(hk, x4_end, u0, steps):
    """The classical RK4 method for du/dx4 = -i h_k u from 0 to x4_end."""
    step = x4_end / steps
    u = u0.astype(complex)
    for _ in range(steps):
        s1 = -1j * (hk @ u)
        s2 = -1j * (hk @ (u + step / 2 * s1))
        s3 = -1j * (hk @ (u + step / 2 * s2))
        s4 = -1j * (hk @ (u + step * s3))
        u = u + step / 6 * (s1 + 2 * s2 + 2 * s3 + s4)
    return u


rng = np.random.default_rng(12345)  # fixed seed: the same numbers in every run
u_start = rng.normal(size=16) + 1j * rng.normal(size=16)
u_start = u_start / np.sqrt(np.vdot(u_start, u_start).real)  # u^dagger u = 1
worst = 0.0
for K in (0.5, 1.0, 2.0, 3.0):
    hk = mode_matrix(1.0, {5: K})
    exact = evolve(hk, 1.0 - K**2, 3.0, u_start)
    stepped = rk4(hk, 3.0, u_start, 3000)
    relative = np.linalg.norm(exact - stepped) / np.linalg.norm(exact)
    worst = max(worst, relative)
    say(f"K = {K}: u^dagger u at x4 = 3 is {np.vdot(exact, exact).real:.6e}")
check(worst < 1e-9, "the closed formula and RK4 agree at x4 = 3 (relative < 1e-9)")
```

Out [8]:

```
K = 0.5: u^dagger u at x4 = 3 is 1.347676e+00
K = 1.0: u^dagger u at x4 = 3 is 2.006346e+01
K = 2.0: u^dagger u at x4 = 3 is 1.880098e+04
K = 3.0: u^dagger u at x4 = 3 is 1.086955e+07
PASS the closed formula and RK4 agree at x4 = 3 (relative < 1e-9)
```

Text cell:

```
The next cell draws $\ln(u^\dagger u)$ against $x_4$ for five extra-time momenta
($m = 1$). For $K < 1$ the size only oscillates; at $K = 1$ ($E = 0$) it grows like
a power of $x_4$; for $K > 1$ the curves become straight lines of slope $2\kappa$
(dashed), because $u^\dagger u$ grows like $e^{2\kappa x_4}$.
```

In [9]:

```
x4_values = np.linspace(0.0, 3.0, 301)
fig, ax = plt.subplots()
for K in (0.5, 1.0, 2.0, 3.0, 5.0):
    hk = mode_matrix(1.0, {5: K})
    sizes = [np.vdot(v, v).real for v in
             (evolve(hk, 1.0 - K**2, x4, u_start) for x4 in x4_values)]
    ax.plot(x4_values, np.log(sizes), label=f"$K = {K}$")
    if K > 1.0:  # the asymptotic straight line of slope 2 kappa
        kappa = growth_rate(1.0, K)
        ax.plot(x4_values, np.log(sizes[-1]) + 2 * kappa * (x4_values - 3.0), "--",
                color="gray", linewidth=0.8)
# An empty line that only adds the dashed straight lines to the legend:
ax.plot([], [], "--", color="gray", linewidth=0.8, label="slope $2\\kappa$")
ax.set_xlabel("time $x_4$")
ax.set_ylabel("$\\ln(u^\\dagger u)$")
ax.set_title("Size of a wave in time, $m = 1$")
ax.legend();
save_figure(fig, "norm_in_time",
            "The natural logarithm of the size $u^\\dagger u$ of a wave against the "
            "time $x_4$ for $m = 1$ and five extra-time momenta $K$, all started from "
            "the same random column of size 1; time and momenta in units of the "
            "inverse of $m$ and of $m$. For $K = 0.5$ the size stays bounded; for "
            "$K = 1$ ($E = 0$) it grows like a power of $x_4$; for $K = 2, 3, 5$ it "
            "grows exponentially and the curves approach straight lines of slope "
            "$2\\kappa$ (gray dashed), $2\\sqrt3$, $2\\sqrt8$ and $2\\sqrt{24}$.")
```

Out [9]:

```
Figure 08a.3 saved as Revision/textbook/figures/08a_3_norm_in_time.png
```

Text cell:

```
## 12. Hadamard: no continuous dependence on the data

Take as data at $x_4 = 0$ one growing wave, $\epsilon\,e^{iKx_5}u_+$, where $u_+$
is an eigenvector of $h_k$ for $+i\kappa$ with $u_+^\dagger u_+ = 1$. Its Sobolev
size of order $s$ is $\epsilon(1 + K^2)^{s/2}$. At $x_4 = 1$ the solution is
$\epsilon\,e^{\kappa}e^{iKx_5}u_+$, of size $\epsilon\,e^{\kappa}$. The ratio
(size of the solution at $x_4 = 1$) / (size of the data) is therefore
$e^{\kappa(K)}(1 + K^2)^{-s/2}$, the same for every $\epsilon$. Because $\kappa(K)$
grows like $K$, the exponential beats every power: for every $s$ the ratio is
unbounded. So data as small as we like (in any Sobolev norm) can give solutions as
large as we like after one unit of time: the solution does not depend continuously
on the data. The next cell finds $u_+$ numerically, confirms that it grows exactly
like $e^{\kappa x_4}$, and draws $\log_{10}$ of the ratio for $s = 0, 2, 4, 8$.
```

In [10]:

```
def growing_eigenvector(mass, K):
    """A unit eigenvector of h_k (k5 = K) for the eigenvalue +i kappa."""
    hk = mode_matrix(mass, {5: K})
```

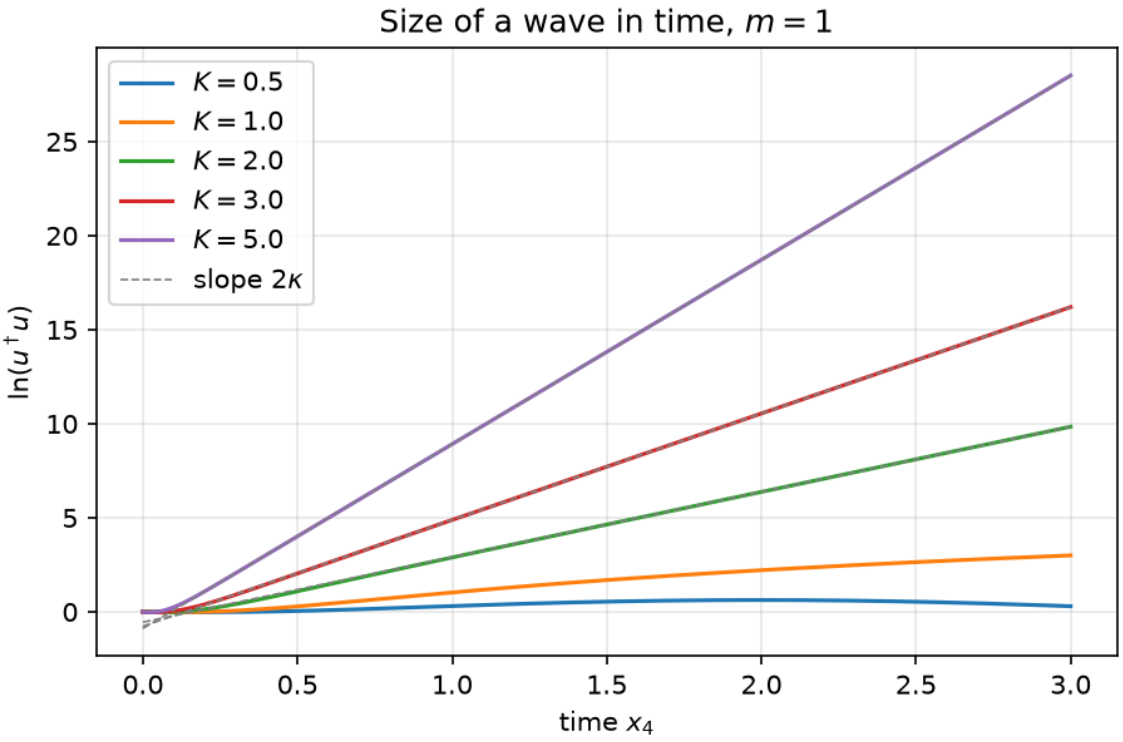

Figure 236: The natural logarithm of the size $u^\dagger u$ of a wave against the time x_4 for $m = 1$ and five extra-time momenta K, all started from the same random column of size 1; time and momenta in units of the inverse of m and of m. For $K = 0.5$ the size stays bounded; for $K = 1$ ($E = 0$) it grows like a power of x_4; for $K = 2, 3, 5$ it grows exponentially and the curves approach straight lines of slope 2κ (gray dashed), $2\sqrt{3}$, $2\sqrt{8}$ and $2\sqrt{24}$. (Notebook 08a, figure 3.)

```
    kappa = growth_rate(mass, K)
    # The null space of h_k - i kappa I: the right singular vector of the smallest
    # singular value (numpy sorts the singular values from large to small).
    _, _, vh = np.linalg.svd(hk - 1j * kappa * np.eye(16))
    u_plus = vh[-1].conj()
    return hk, kappa, u_plus / np.linalg.norm(u_plus)


hk, kappa, u_plus = growing_eigenvector(1.0, 4.0)
later = evolve(hk, 1.0 - 4.0**2, 1.0, u_plus)
check(np.linalg.norm(hk @ u_plus - 1j * kappa * u_plus) < 1e-12
      and abs(np.linalg.norm(later) - np.exp(kappa)) < 1e-9 * np.exp(kappa),
      "a growing eigenvector grows exactly like e^(kappa x4) (K = 4)")

K_axis = np.linspace(0.0, 200.0, 2001)
fig, ax = plt.subplots()
for s in (0, 2, 4, 8):
    log_ratio = (growth_rate(1.0, K_axis) - s / 2 * np.log(1 + K_axis**2)) / np.log(10)
    ax.plot(K_axis, log_ratio, label=f"$s = {s}$")
    crossing = K_axis[np.argmax(log_ratio > 6.0)]  # first K with ratio above 10^6
    report(f"s = {s}: first K (step 0.1) with ratio above 10^6", f"{crossing:.1f}")
    check(log_ratio[-1] > 50.0, f"s = {s}: the ratio exceeds 10^50 at K = 200")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("extra-time momentum $K$")
ax.set_ylabel("$\\log_{10}$(solution at $x_4 = 1$ / data)")
ax.set_title("Amplification after one unit of time, $m = 1$")
ax.legend();
save_figure(fig, "hadamard_ratio",
            "The base-10 logarithm of the amplification, the size of the solution "
            "at $x_4 = 1$ divided by the size of the data at $x_4 = 0$, for one "
            "growing wave of extra-time momentum $K$ and $m = 1$, with the data "
            "measured in the Sobolev norm of order $s = 0, 2, 4, 8$; horizontal "
            "axis $K$ in units of $m$. The amplification is $e^{\\kappa}(1 + "
            "K^2)^{-s/2}$: a stronger norm only delays the rise, and every curve "
            "grows without bound, so arbitrarily small data give arbitrarily large "
            "solutions and the problem is not well posed in the sense of Hadamard.")
```

Out [10]:

```
PASS a growing eigenvector grows exactly like e^(kappa x4) (K = 4)
RESULT s = 0: first K (step 0.1) with ratio above 10^6 = 13.9
PASS s = 0: the ratio exceeds 10^50 at K = 200
RESULT s = 2: first K (step 0.1) with ratio above 10^6 = 19.9
PASS s = 2: the ratio exceeds 10^50 at K = 200
RESULT s = 4: first K (step 0.1) with ratio above 10^6 = 27.1
PASS s = 4: the ratio exceeds 10^50 at K = 200
RESULT s = 8: first K (step 0.1) with ratio above 10^6 = 44.2
PASS s = 8: the ratio exceeds 10^50 at K = 200
```

Out [10]:

```
Figure 08a.4 saved as Revision/textbook/figures/08a_4_hadamard_ratio.png
```

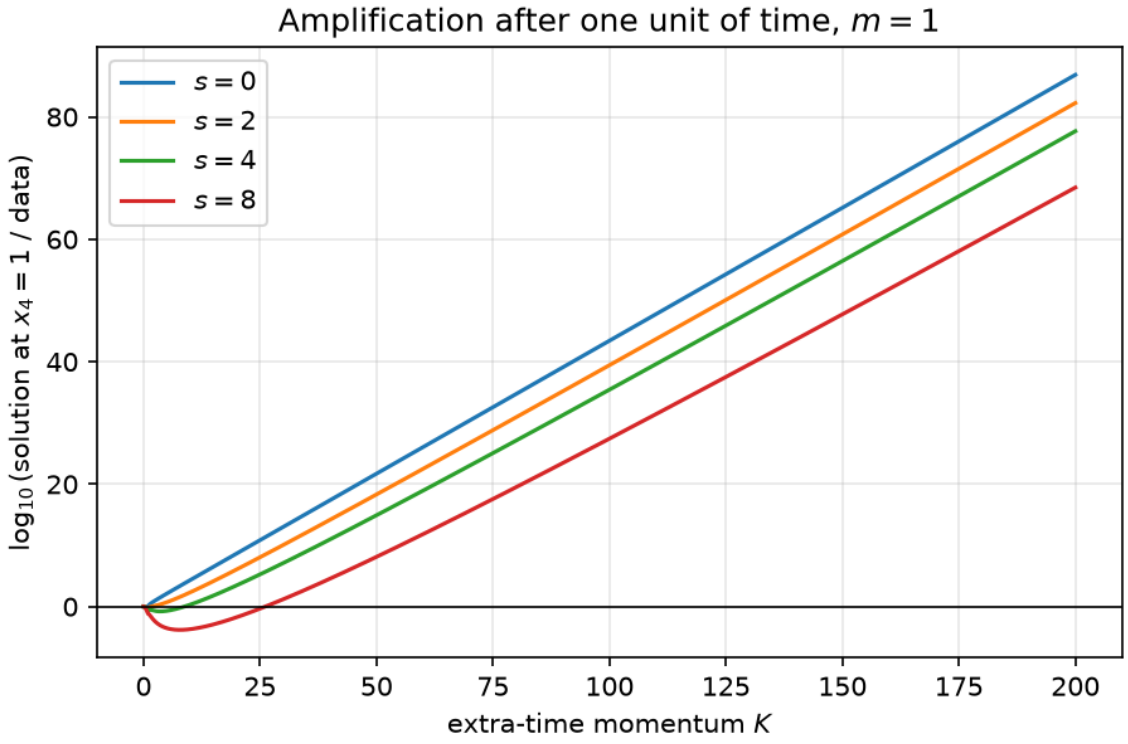

Figure 237: The base-10 logarithm of the amplification, the size of the solution at $x_4 = 1$ divided by the size of the data at $x_4 = 0$, for one growing wave of extra-time momentum K and $m = 1$, with the data measured in the Sobolev norm of order $s = 0, 2, 4, 8$; horizontal axis K in units of m. The amplification is $e^{\kappa}(1 + K^2)^{-s/2}$: a stronger norm only delays the rise, and every curve grows without bound, so arbitrarily small data give arbitrarily large solutions and the problem is not well posed in the sense of Hadamard. (Notebook 08a, figure 4.)

Text cell:

```
## 13. The Krein form is conserved; the growing waves have Krein form zero

From $B h_k = h_k^\dagger B$ (section 6): $\frac{d}{dx_4}(u^\dagger B u) = i\,u^\dagger
(h_k^\dagger B - B h_k)u = 0$, so the Krein form never changes, while
$\frac{d}{dx_4}(u^\dagger u) = i\,u^\dagger(h_k^\dagger - h_k)u$ is not zero when
$h_k$ is not Hermitian. The next cell follows a random wave with $m = 1$, $K = 2$
up to $x_4 = 4$ and compares both, then checks two exact statements of the pairing
record on its samples: the eight-dimensional eigenspace of a growing frequency
($m = 1$, $k_5 = 2$) carries the Krein form zero (every $u^\dagger B v$ vanishes on
it), while an eigenspace of a real frequency ($m = 1$, $k_1 = 2$, $k_0 = 2$, $E = 3$)
carries a Krein form with four positive and four negative directions (*inertia*
(4, 4)). Both checks compare the dimensions and the inertia found here with the
text of the pairing record (which calls the frequency $w$).
```

In [11]:

```
hk = mode_matrix(1.0, {5: 2.0})
times = np.linspace(0.0, 4.0, 401)
path = [evolve(hk, 1.0 - 4.0, x4, u_start) for x4 in times]
hilbert = np.array([np.vdot(v, v).real for v in path])  # u^dagger u
krein_values = np.array([np.vdot(v, B @ v) for v in path])  # u^dagger B u
drift = np.max(np.abs(krein_values - krein_values[0]) / np.maximum(hilbert, 1.0))
report("Krein form at x4 = 0", f"{krein_values[0].real:+.9f}")
report("u^dagger u at x4 = 4", f"{hilbert[-1]:.6e}")
imaginary = np.max(np.abs(krein_values.imag) / np.maximum(hilbert, 1.0))
check(drift < 1e-12 and imaginary < 1e-12,
      "the Krein form u^dagger B u is constant (drift / max(u^dagger u, 1) < 1e-12)")


def eigenspace(hk, value):
    """An orthonormal basis (columns) of the eigenspace of hk for value."""
    _, singular, vh = np.linalg.svd(hk - value * np.eye(16))
    return vh[singular < 1e-9].conj().T


V_grow = eigenspace(mode_matrix(1.0, {5: 2.0}), 1j * 3**0.5)
form_grow = V_grow.conj().T @ B @ V_grow  # the Krein form on that eigenspace
check_record(V_grow.shape[1] == 8 and np.max(np.abs(form_grow)) < 1e-12,
             "m = 1, k5 = 2: the 8-dim eigenspace of +i sqrt(3) is Krein-neutral",
             "Revision/pairing/reports/python-pairing.json",
             "Q.one_particle_complex_frequency_Krein_neutral",
             "(0, 0, 0, 0, 2, 0, 0): w^2 = -3, eigenspace dimensions "
             f"[{V_grow.shape[1]}, {V_grow.shape[1]}], B-form identically zero: True")
V_real = eigenspace(mode_matrix(1.0, {0: 2.0, 1: 2.0}), 3.0)
form_real = np.linalg.eigvalsh(V_real.conj().T @ B @ V_real)  # Hermitian 8 x 8
inertia = (int(np.sum(form_real > 1e-9)), int(np.sum(form_real < -1e-9)))
report("Krein inertia of the eigenspace of E = 3 (m = 1, k0 = k1 = 2)", inertia)
check_record(V_real.shape[1] == 8 and inertia == (4, 4),
             "m = 1, k0 = k1 = 2: the eigenspace of E = 3 has Krein inertia (4, 4)",
             "Revision/pairing/reports/python-pairing.json",
```

```
             "Q.one_particle_Krein_inertia",
             f"w = 3: dim {V_real.shape[1]}, Krein inertia ({inertia[0]},{inertia[1]})")

fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(times, np.log(hilbert))
left.set_xlabel("time $x_4$")
left.set_ylabel("$\\ln(u^\\dagger u)$")
left.set_title("Hilbert norm: grows")
right.plot(times, krein_values.real)
right.axhline(krein_values[0].real, color="gray", linestyle="--", linewidth=0.8)
right.set_ylim(krein_values[0].real - 0.5, krein_values[0].real + 0.5)
right.set_xlabel("time $x_4$")
right.set_ylabel("$u^\\dagger B u$")
right.set_title("Krein form: constant")
save_figure(fig, "krein_norm",
            "One wave with $m = 1$ and extra-time momentum $K = 2$ followed from "
            "$x_4 = 0$ to $x_4 = 4$ (time in units of $1/m$). Left: the natural "
            "logarithm of its Hilbert norm $u^\\dagger u$, which grows like "
            "$e^{2\\sqrt3\\,x_4}$. Right: its Krein form $u^\\dagger B u$, which "
            "stays exactly at its starting value (gray dashed line). The growth "
            "is possible because the growing waves have Krein form zero: the "
            "positive and the negative parts of the Krein form grow together.")
```

Out [11]:

```
RESULT Krein form at x4 = 0 = +0.129919115
RESULT u^dagger u at x4 = 4 = 6.006621e+05
PASS the Krein form u^dagger B u is constant (drift / max(u^dagger u, 1) < 1e-12)
PASS m = 1, k5 = 2: the 8-dim eigenspace of +i sqrt(3) is Krein-neutral
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_complex_frequency_Krein_neutral
RESULT Krein inertia of the eigenspace of E = 3 (m = 1, k0 = k1 = 2) = (4, 4)
PASS m = 1, k0 = k1 = 2: the eigenspace of E = 3 has Krein inertia (4, 4)
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_Krein_inertia
```

Out [11]:

```
Figure 08a.5 saved as Revision/textbook/figures/08a_5_krein_norm.png
```

Text cell:

```
## 14. The good sector: every frequency is real

In the good sector ($k_5 = k_6 = k_7 = 0$) $h_k$ is Hermitian (section 7), so its
eigenvalues are real. The next cell checks this for a random good-sector momentum,
checks that $B$ commutes with $h_k$ there, and then follows the eigenvalues
$\pm\sqrt{1 + k_1^2 - K^2}$ in the complex plane along two paths: a growing space
momentum $k_1$ from 0 to 2.5 (good sector) and a growing extra-time momentum $K$
from 0 to 2.5. Along the first path the eigenvalues move outwards on the real axis;
along the second they meet at 0 when $K = 1$ and then leave the real axis.
```

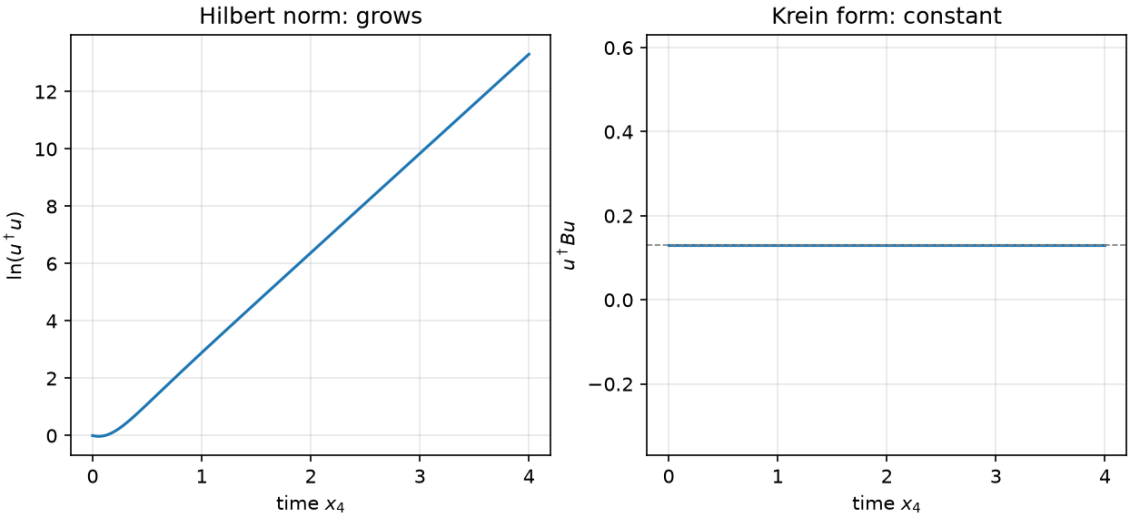

Figure 238: One wave with $m = 1$ and extra-time momentum $K = 2$ followed from $x_4 = 0$ to $x_4 = 4$ (time in units of $1/m$). Left: the natural logarithm of its Hilbert norm $u^\dagger u$, which grows like $e^{2\sqrt{3}\,x_4}$. Right: its Krein form $u^\dagger B u$, which stays exactly at its starting value (gray dashed line). The growth is possible because the growing waves have Krein form zero: the positive and the negative parts of the Krein form grow together. (Notebook 08a, figure 5.)

In [12]:

```
good = mode_matrix(1.3, {0: 2.2, 1: 0.4, 2: -1.1, 3: 0.7})  # no k5, k6, k7
good_eigenvalues = np.linalg.eigvals(good)
E_good = np.sqrt(1.3**2 + 0.4**2 + 1.1**2 + 0.7**2 + 2.2**2)
check_record(np.allclose(good, good.conj().T) and np.allclose(good @ B, B @ good)
             and np.max(np.abs(np.abs(good_eigenvalues) - E_good)) < 1e-12
             and np.max(np.abs(good_eigenvalues.imag)) < 1e-12,
             "good sector: h_k Hermitian, [B, h_k] = 0, eigenvalues real +-E",
             "Revision/theory/reports/python-field-theory.json",
             "good_sector_spectrum_and_B_sectors", "h is Hermitian", "[B, h] = 0")

path_values = np.linspace(0.0, 2.5, 26)  # the momenta 0, 0.1, ..., 2.5
colours = np.repeat(path_values, 16)  # each matrix has 16 eigenvalues
fig, panels = plt.subplots(1, 2, figsize=(10.0, 4.6), sharey=True)
for panel, direction, name in ((panels[0], 1, "space momentum $k_1$ (good sector)"),
                               (panels[1], 5, "extra-time momentum $K = k_5$")):
    points = np.concatenate([np.linalg.eigvals(mode_matrix(1.0, {direction: p}))
                             for p in path_values])
    dots = panel.scatter(points.real, points.imag, c=colours, cmap="plasma", s=22)
    panel.set_xlim(-3.0, 3.0)
    panel.set_ylim(-3.0, 3.0)
    panel.set_aspect("equal")
    panel.set_xlabel("real part of the eigenvalue $E$")
    panel.set_title(name)
panels[0].set_ylabel("imaginary part of the eigenvalue $E$")
fig.colorbar(dots, ax=panels, label="momentum (0 to 2.5)")
save_figure(fig, "eigenvalue_paths",
            "The eigenvalues $E$ of the mode matrix for $m = 1$ in the complex "
```

```
"plane (horizontal: real part, vertical: imaginary part, in units of "
"$m$), coloured by the momentum from 0 (dark) to 2.5 (bright). Left: a "
"growing space momentum $k_1$ (good sector); the eigenvalues "
"$\\pm\\sqrt{1 + k_1^2}$ move outwards along the real axis and stay "
"real. Right: a growing extra-time momentum $K$; the eigenvalues "
"$\\pm\\sqrt{1 - K^2}$ move inwards, meet at 0 for $K = 1$ and then "
"leave along the imaginary axis as $\\pm i\\sqrt{K^2 - 1}$: growth.")
```

Out [12]:

```
PASS good sector: h_k Hermitian, [B, h_k] = 0, eigenvalues real +-E
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_spectrum_and_B_sectors
```

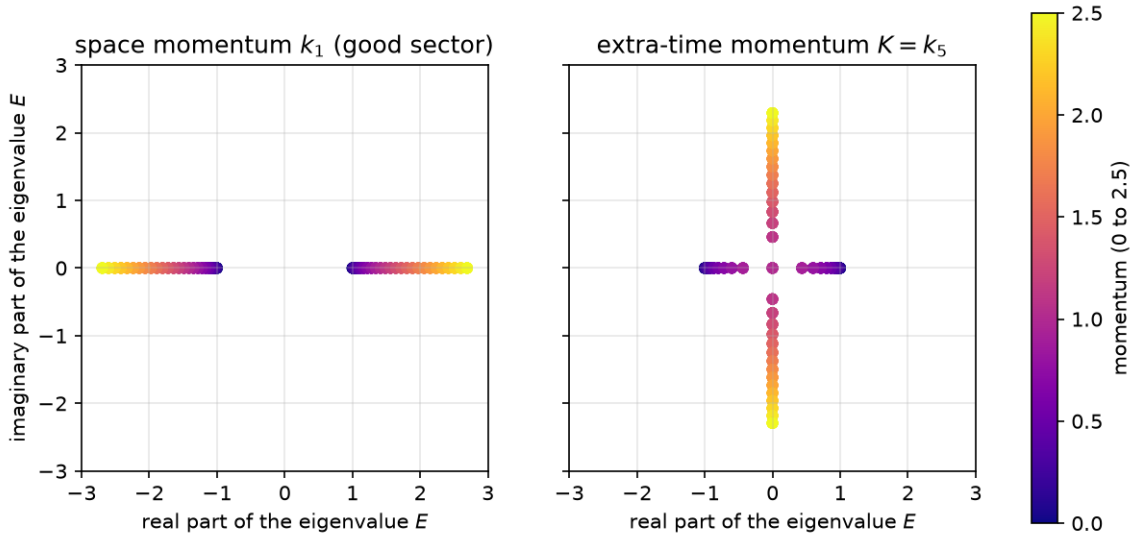

Figure 239: The eigenvalues E of the mode matrix for $m = 1$ in the complex plane (horizontal: real part, vertical: imaginary part, in units of m), coloured by the momentum from 0 (dark) to 2.5 (bright). Left: a growing space momentum k_1 (good sector); the eigenvalues $\pm\sqrt{1 + k_1^2}$ move outwards along the real axis and stay real. Right: a growing extra-time momentum K; the eigenvalues $\pm\sqrt{1 - K^2}$ move inwards, meet at 0 for $K = 1$ and then leave along the imaginary axis as $\pm i\sqrt{K^2 - 1}$: growth. (Notebook 08a, figure 6.)

Out [12]:

```
Figure 08a.6 saved as Revision/textbook/figures/08a_6_eigenvalue_paths.png
```

Text cell:

```
## 15. The last check

The last cell checks that the six figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [13]:

```
for name in ("08a_1_growth_rate_vs_momentum.png", "08a_2_growth_map.png",
             "08a_3_norm_in_time.png", "08a_4_hadamard_ratio.png",
             "08a_5_krein_norm.png", "08a_6_eigenvalue_paths.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [13]:

```
PASS the figure file 08a_1_growth_rate_vs_momentum.png exists
PASS the figure file 08a_2_growth_map.png exists
PASS the figure file 08a_3_norm_in_time.png exists
PASS the figure file 08a_4_hadamard_ratio.png exists
PASS the figure file 08a_5_krein_norm.png exists
PASS the figure file 08a_6_eigenvalue_paths.png exists
ALL 30 CHECKS PASSED (notebook 08a)
```

Text cell:

```
## 16. What this notebook showed

- With the author's gamma matrices the mode matrix of a plane wave (frozen
  coefficients) satisfies $h_k^2 = E^2 I_{16}$ exactly, with
  $E^2 = m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2$ (PROVED, exact
  sympy; Revision check `extra_time_growth_rates_unbounded`).
- $h_k$ is Hermitian exactly when there is no extra-time momentum; in the good
  sector every frequency is real (PROVED).
- With enough extra-time momentum the frequencies are $\pm i\kappa$ and half of the
  waves grow like $e^{\kappa x_4}$; for $m = 1$, $k_5 = 2$ the rate is $\sqrt3$
  (PROVED; check `extra_time_modes_grow`).
- The growth rate $\kappa = \sqrt{K^2 - m^2 - k_1^2 - \dots}$ has no upper bound
  (PROVED). The amplification $e^{\kappa}(1 + K^2)^{-s/2}$ after one unit of time is
  unbounded in every Sobolev norm, so the initial-value problem for data that
  depend on the extra times is NOT well posed in the sense of Hadamard, although
  the slices $x_4 = $ const are non-characteristic (the equation can be solved for
  $\partial_4\Psi$).
- The Krein form $u^\dagger B u$ is conserved by every wave, also by the growing
  ones; the growing eigenspaces are Krein-neutral and the real-frequency ones have
  inertia (4, 4) (PROVED in the pairing record; reproduced here on its samples).
- Scope: these are statements with frozen coefficients (or in flat 4 + 4 space). In
  the author's metric the extra times deflate, so the frame momentum of a wave
  along an extra time grows with time; that is the subject of the next example of
  this chapter (Notebook 08b).
```

8.23 Line-by-line walk-through of Notebook 08a

The notebook has 13 code cells, In [1] to In [13]. As in Section 8.15, each line or small group of lines is quoted and explained, and a line `...` in quoted code stands for the remaining lines of a figure caption, which is printed in full under its figure in Section 8.22.

In [1], the set-up cell. It is the set-up cell explained line by line in Section 8.15, with one difference: the line `NOTEBOOK_ID = "08a" # this notebook: chapter 08, example a`. Its comment lines are the run instructions of Section 8.21. Output: `Set-up of notebook 08a complete: repository folder found, helpers defined.`

In [2], the record helpers and the author's gamma matrices.

```
import numpy as np  # arrays of numbers, matrices, linear algebra
import sympy as sp  # exact algebra with symbols

REPORTS = {}  # report file -> {check name: (verdict, detail)}, each read once
```

The two packages and the empty table of reports, as in Section 8.15.

```
def record_says(report_file, check_name, *pieces):
    ...
    verdict, detail = REPORTS[report_file][check_name]
    return verdict == "pass" and all(piece in detail for piece in pieces)


def check_record(condition, title, report_file, check_name, *pieces):
    """check() for a result that reproduces the Revision check check_name: it passes
    only if condition is true AND the report records check_name as passed, with
    every piece of text (values computed here) in its detail."""
    on_record = record_says(report_file, check_name, *pieces)
    check(condition and on_record, title,
          record=f"{report_file}, check {check_name}")
```

`record_says` is exactly the function of Notebook 08c (its body, shortened here to `...`, is quoted and explained in Section 8.15). `check_record` is the simpler form of the 08c helper: it takes one check name only. It passes only if the notebook's own condition is true AND the record reports the check as passed with every given piece of text in its detail.

```
gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
# The record stores every matrix as a list of rows of whole numbers.
GAMMA = [np.array(rows, dtype=int) for rows in gammas_record["gamma"]]
ETA = gammas_record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1] in the order x0 ... x7
# B = -i C gamma^(x4) is stored as its real part and its imaginary part.
B = np.array(gammas_record["B"]["re"]) + 1j * np.array(gammas_record["B"]["im"])
I16 = np.eye(16, dtype=int)  # the 16 x 16 unit matrix
say(f"eta = {ETA}; each gamma is a {GAMMA[0].shape[0]} x {GAMMA[0].shape[1]} matrix")
```

The gammas are read from the record and turned into numpy arrays of whole numbers (`dtype=int`), so that the products below are exact. `ETA` is the list of the eight signs. The matrix B is complex; the record stores its real part (`"re"`) and its imaginary part (`"im"`) separately, and the line adds them, `1j` being Python's imaginary unit i. `np.eye(16, dtype=int)` is the unit matrix. The `say` line prints the signs and the shape of a gamma (`.shape` gives the number of rows and columns): `eta = [1, 1, 1, 1, -1, -1, -1, -1]; each gamma is a 16 x 16 matrix`.

```
clifford_ok = all(
    np.array_equal(GAMMA[a] @ GAMMA[b] + GAMMA[b] @ GAMMA[a],
                   2 * ETA[a] * (a == b) * I16)  # (a == b) is 1 or 0
    for a in range(8) for b in range(8))
check_record(clifford_ok, "the 64 Clifford relations {gamma^a, gamma^b} = 2 eta^ab I16",
             "Revision/algebra/reports/python-algebra.json", "clifford_relation",
             "for all 64 ordered pairs")
```

For all 64 ordered pairs (a, b) the anticommutator $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}$ (`@` is the matrix product) is compared, entry by entry and exactly (`np.array_equal`), with $2\eta_{aa} I_{16}$ when $a = b$ and the zero matrix otherwise: in arithmetic, the truth value `(a == b)` counts as 1 or 0. The check also requires the algebra record's check `clifford_relation`.

```
check(np.allclose(B, B.conj().T) and np.allclose(B @ B, np.eye(16)),
      "B is Hermitian and B^2 = 1")
# (gamma^(x4))^2 = -1: the field equation can be solved for d4 Psi (multiply it by
# -gamma^(x4)), so the slices x4 = const are non-characteristic.
check_record(np.array_equal(GAMMA[4] @ GAMMA[4], -I16),
             "(gamma^(x4))^2 = -1: the slices x4 = const are non-characteristic",
             "Revision/theory/reports/wolfram-field-theory.json", "evolution_form_G",
             "the slices x4 = const are non-characteristic")
```

`B.conj().T` is the conjugate transpose $B^\dagger$; `np.allclose` compares two arrays up to rounding. The first check confirms $B^\dagger = B$ and $B^2 = I_{16}$. The second confirms $(\gamma^{(4)})^2 = -I_{16}$, which is the reason the field equation can be solved for $\partial_4\Psi$ (Section 8.16), together with the Wolfram record's check `evolution_form_G`. Output: the line about `eta` and three PASS lines, two with `reproduces` lines.

In [3], the mode matrix and its square, exactly.

```
m = sp.Symbol("m", real=True)  # the mass
k = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}  # momenta
G = [sp.Matrix(rows) for rows in gammas_record["gamma"]]  # the gammas, exact
B_exact = sp.I * sp.Matrix(gammas_record["B"]["im"])  # B is purely imaginary
g4 = G[4]  # gamma^(x4)
```

The mass and the seven momenta are exact symbols (a dictionary that maps each direction number a to the symbol k_a; there is no k_4). `G` holds the gammas as exact sympy matrices. The real part of B is zero (because $C\gamma^{(4)}$ is real), so `B_exact` is i times the stored imaginary part. `g4` is $\gamma^{(4)}$.

```
momentum_part = sp.zeros(16, 16)
for a, k_a in k.items():  # a runs over 0, 1, 2, 3, 5, 6, 7
    momentum_part += k_a * g4 * G[a]  # k_a gamma^(x4) gamma^(xa)
h = -sp.I * m * g4 - momentum_part  # the mode matrix h_k
E2 = m**2 + k[0]**2 + k[1]**2 + k[2]**2 + k[3]**2 - k[5]**2 - k[6]**2 - k[7]**2
say(f"E^2 = {E2}")
```

`k.items()` gives the pairs (direction, symbol); `G[a]` is the gamma of direction x_a (lists count from 0, like the coordinates). The loop adds up $\sum_{a\neq 4} k_a\gamma^{(4)}\gamma^{(a)}$, and `h` is the mode matrix $h_k = -im\gamma^{(4)} - \gamma^{(4)}\sum_a k_a\gamma^{(a)}$ of Section 8.17 (`sp.I` is the exact i). `E2` is the expected E^2; it is

printed in sympy's order: `E^2 = k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + k8**2 + m**2`.

```
square_minus_E2 = (h * h - E2 * sp.eye(16)).applyfunc(sp.expand)
square_ok = square_minus_E2 == sp.zeros(16, 16)
# The two independent Revision verifiers print E^2 in their own notations: sympy
# writes powers with **, the Wolfram verifier with ^. Both texts must hold this E^2.
E2_sympy_text = sp.sstr(E2)  # "k0**2 + k1**2 + ... + m**2"
E2_wolfram_text = E2_sympy_text.replace("**", "^")  # "k0^2 + k1^2 + ... + m^2"
```

The matrix $h_k^2 - E^2 I_{16}$ is computed and every entry multiplied out (`sp.expand`); `square_ok` is true when all 256 entries are exactly zero. The two texts are E^2 as the sympy record writes it and as the Wolfram record writes it (`replace` swaps `**` for `^`).

```
check_record(square_ok,
             "h_k^2 = E^2 I16, E^2 = m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 "
             "- k7^2", "Revision/theory/reports/python-scope.json",
             "extra_time_growth_rates_unbounded", f"h_k^2 = ({E2_sympy_text}) I16")
check_record(square_ok, "the same E^2 in the independent Wolfram verifier",
             "Revision/theory/reports/wolfram-scope.json",
             "extra_time_growth_rates_unbounded", f"h_k^2 = ({E2_wolfram_text}) I16")
check(sp.expand(h.trace()) == 0, "the trace of h_k is 0 (eigenvalues +E, -E, 8 each)")
```

Two checks of the same exact statement against the two independent Revision verifiers, whose details must contain exactly this E^2 in their notations; and the check that the trace of h_k is zero, which, with $h_k^2 = E^2$, gives eight eigenvalues $+E$ and eight $-E$ (Section 8.17).

```
krein = (B_exact * h - h.H * B_exact).applyfunc(sp.expand)  # .H: conjugate transpose
check_record(krein == sp.zeros(16, 16), "B h_k = h_k^dagger B (Krein self-adjoint)",
             "Revision/theory/reports/python-field-theory.json",
             "mode_hamiltonian_B_selfadjoint_dispersion", "B h = h^dagger B")
```

`h.H` is sympy's conjugate transpose $h_k^\dagger$. The check confirms $Bh_k = h_k^\dagger B$ for symbolic mass and momenta, the property that conserves the Krein form (Section 8.19). Output: the line with E^2 and four PASS lines, three with `reproduces` lines.

In [4], when is the mode matrix Hermitian?

```
anti = ((h - h.H) / 2).applyfunc(sp.expand)  # the anti-Hermitian part of h_k
extra_time_part = -(k[5] * g4 * G[5] + k[6] * g4 * G[6] + k[7] * g4 * G[7])
check((anti - extra_time_part).applyfunc(sp.expand) == sp.zeros(16, 16),
      "the anti-Hermitian part of h_k is the extra-time part")
anti_square = (anti * anti).applyfunc(sp.expand)
check(anti_square == (-(k[5]**2 + k[6]**2 + k[7]**2) * sp.eye(16)).applyfunc(
    sp.expand), "its square is -(k5^2 + k6^2 + k7^2) I16")
```

Every matrix is the sum of its Hermitian part $(h + h^\dagger)/2$ and its anti-Hermitian part $(h - h^\dagger)/2$. The first check shows that the anti-Hermitian part is exactly the extra-time part $-\gamma^{(4)}(k_5\gamma^{(5)} +$

$k_6\gamma^{(6)} + k_7\gamma^{(7)}$) (Section 8.17). The second shows that its square is $-(k_5^2 + k_6^2 + k_7^2)I_{16}$; a matrix whose square is a negative number times I_{16} is the zero matrix only when that number is zero, so h_k is Hermitian exactly when there is no extra-time momentum. Output: two PASS lines.

In [5], the exact example $m = 1$, $k_5 = 2$.

```
sample = {m: 1, k[5]: 2, k[0]: 0, k[1]: 0, k[2]: 0, k[3]: 0, k[6]: 0, k[7]: 0}
h_sample = h.subs(sample)  # an exact matrix of numbers
check(h_sample * h_sample == -3 * sp.eye(16), "m = 1, k5 = 2: h^2 = -3 I16")
dimension = 16 - (h_sample - sp.I * sp.sqrt(3) * sp.eye(16)).rank()
report("dimension of the eigenspace of +i sqrt(3)", dimension)
```

The substitution puts in $m = 1$, $k_5 = 2$ and zero for the other momenta. The check confirms $h^2 = -3I_{16}$. The dimension of the eigenspace of $+i\sqrt{3}$ is the number of independent solutions of $(h - i\sqrt{3}I)u = 0$, which is 16 minus the rank of that matrix (a standard fact of linear algebra, Chapter 1); sympy computes the rank exactly. RESULT: 8.

```
eigenvalues = np.linalg.eigvals(np.array(h_sample, dtype=complex))
# Round to 9 digits and count how often each value occurs (sorted, so the printed
# order is always the same).
rounded = sorted({complex(0.0, round(v.imag, 9)) for v in eigenvalues},
                 key=lambda v: v.imag)  # the real parts are zero (checked below)
for value in rounded:
    count = int(np.sum(np.abs(eigenvalues - value) < 1e-6))
    say(f"eigenvalue {value.imag:+.9f} i occurs {count} times")
```

numpy computes the 16 eigenvalues numerically (`np.linalg.eigvals`, on a complex array). The **set** in curly brackets keeps each rounded value once; `sorted(..., key=lambda v: v.imag)` orders them by their imaginary part (a `lambda` is a one-line function). For each distinct value the loop counts how many eigenvalues lie within 10^{-6} of it (`np.sum` of true values counts them) and prints, for example, `eigenvalue -1.732050808 i occurs 8 times` (`:+.9f` prints the sign and nine decimals).

```
# The record lists the exact eigenvalues as sympy writes them, sorted as text:
exact_text = sorted(str(sign * sp.sqrt(3) * sp.I) for sign in (1, -1))
say("exact eigenvalues as sympy writes them: " + ", ".join(exact_text))
check_record(dimension == 8 and np.max(np.abs(eigenvalues.real)) < 1e-9
             and all(abs(abs(v.imag) - 3**0.5) < 1e-9 for v in eigenvalues),
             "m = 1, k5 = 2: eigenvalues +i sqrt(3) and -i sqrt(3), 8 each",
             "Revision/theory/reports/python-field-theory.json",
             "extra_time_modes_grow", f"m = 1, k5 = 2: eigenvalues {exact_text}")
```

`exact_text` is the list of the two exact eigenvalues as sympy writes them, `-sqrt(3)*I` and `sqrt(3)*I`, sorted as texts. The check requires the eigenspace dimension 8, real parts below 10^{-9}, every imaginary part within 10^{-9} of $\pm\sqrt{3}$, and the record's text that starts with `m = 1, k5 = 2: eigenvalues` followed by this list (the f-string writes the Python list with its square brackets and quotation marks, exactly as the record does). Output: two PASS lines, one RESULT line, the two counted eigenvalues and the exact text.

In [6], the growth rate against the extra-time momentum.

```
def mode_matrix(mass, momenta):
    """h_k = -i m gamma^(x4) - gamma^(x4) sum_a k_a gamma^(xa); momenta: {a: k_a}."""
    g4n = GAMMA[4]
    result = -1j * mass * g4n.astype(complex)
    for a, k_a in momenta.items():  # a is the direction: GAMMA[a] is gamma^(xa)
        result = result - k_a * (g4n @ GAMMA[a])
    return result


def growth_rate(mass, K, k1=0.0):
    """kappa = sqrt(K^2 - m^2 - k1^2) where positive, else 0 (numpy arrays too)."""
    return np.sqrt(np.maximum(K**2 - mass**2 - k1**2, 0.0))
```

`mode_matrix` builds the numerical mode matrix for a mass and a dictionary of momenta, for example `{5: 2.0}` for $k_5 = 2$ (`.astype(complex)` makes the integer matrix complex, so that $-i$ can multiply it). `growth_rate` is the formula $\kappa = \sqrt{K^2 - m^2 - k_1^2}$ of Section 8.18 where the bracket is positive, and 0 where it is not (`np.maximum` takes the larger of the bracket and 0, entry by entry, so the function also works on whole arrays of K).

```
K_values = np.linspace(0.0, 6.0, 61)  # extra-time momenta 0, 0.1, ..., 6
numerical = np.array([np.max(np.linalg.eigvals(mode_matrix(1.0, {5: K})).imag)
                      for K in K_values])  # the largest growth rate, m = 1
deviation = np.abs(numerical - growth_rate(1.0, K_values))
at_threshold = np.abs(K_values - 1.0) < 1e-12  # K = m exactly, where h_k^2 = 0
check(np.max(deviation[~at_threshold]) < 1e-10 and np.max(deviation) < 1e-6,
      "growth rate from the eigenvalues = sqrt(K^2 - m^2) (m = 1)")
```

For 61 momenta $K = 0, 0.1, \ldots, 6$ the largest imaginary part of the 16 eigenvalues of h_k (with $m = 1$, $k_5 = K$) is the numerical growth rate. `deviation` holds the differences from the formula. `at_threshold` marks the single point $K = 1 = m$, where $h_k^2 = 0$: a matrix whose square is zero has the double eigenvalue 0, which a computer finds only to about 10^{-8}. `~` means "not", so `deviation[~at_threshold]` are the deviations at the other 60 points; the check requires them below 10^{-10}, and the one at the threshold below 10^{-6}.

```
report("growth rate at m = 1, K = 2 (sqrt 3)", f"{growth_rate(1.0, 2.0):.9f}")
report("growth rate at m = 1, K = 1000", f"{growth_rate(1.0, 1000.0):.6f}")
check_record(growth_rate(1.0, 1000.0) > 999.0 and growth_rate(1.0, 1e6) > 999999.0,
             "the growth rate has no upper bound (kappa/K tends to 1)",
             "Revision/theory/reports/python-scope.json",
             "extra_time_growth_rates_unbounded", "which has no upper bound")
```

Two RESULT lines: the rate $\sqrt{3} = 1.732050808$ at $K = 2$, and 999.999500 at $K = 1000$. The check illustrates the unbounded growth at $K = 1000$ and $K = 10^6$ (`1e6`), and requires the record's words "which has no upper bound"; the proof is the inequality $\kappa \geq K - c$ of Section 8.18.

```
K_fine = np.linspace(0.0, 6.0, 601)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for mass, style in ((0.5, "-"), (1.0, "--"), (2.0, ":")):
    left.plot(K_fine, growth_rate(mass, K_fine), style, label=f"$m = {mass}$")
```

```
left.plot(K_values[::3], numerical[::3], "o", markersize=4, color="black",
          label="eigenvalues, $m = 1$")
left.plot(K_fine, K_fine, color="gray", linewidth=0.8, label="$\\kappa = K$")
left.set_xlabel("extra-time momentum $K = k_5$")
left.set_ylabel("growth rate $\\kappa$")
left.set_title("$k_1 = 0$: three masses")
left.legend(loc="upper left")
```

The left panel draws the formula for three masses on a fine grid (601 points) with solid, dashed and dotted lines; the black dots are every third numerical rate of $m = 1$ (`[::3]` takes every third element); the gray line is $\kappa = K$, which every curve approaches.

```
for k1, style in ((0.0, "-"), (2.0, "--"), (4.0, ":")):
    right.plot(K_fine, growth_rate(1.0, K_fine, k1), style, label=f"$k_1 = {k1}$")
right.plot(K_fine, K_fine, color="gray", linewidth=0.8, label="$\\kappa = K$")
right.set_xlabel("extra-time momentum $K = k_5$")
right.set_title("$m = 1$: three space momenta $k_1$")
right.legend(loc="upper left")
save_figure(fig, "growth_rate_vs_momentum",
            "The growth rate $\\kappa = \\sqrt{K^2 - m^2 - k_1^2}$ of the growing "
            ...
            "$\\kappa = K$, so the rate has no upper bound.")
```

The right panel draws the formula for $m = 1$ and three space momenta, with the same gray line. The figure is Figure 08a.1. Output: one PASS line, two RESULT lines, one PASS line with its `reproduces` line, and the figure line.

What Figure 08a.1 shows. Both panels plot the growth rate κ (vertical) against the extra-time momentum $K = k_5$ (horizontal), both in the same unit, an inverse length. Left: masses $m = 0.5, 1, 2$; each curve is zero up to the threshold $K = m$, then rises steeply and bends towards the gray line $\kappa = K$; the black dots, the numerical eigenvalues for $m = 1$, lie exactly on the dashed curve. Right: $m = 1$ with $k_1 = 0, 2, 4$: a space momentum moves the threshold to $K = \sqrt{1 + k_1^2}$ but does not stop the rise. The student should see that every curve approaches $\kappa = K$, so the rate has no upper bound.

In [7], where the waves oscillate and where they grow.

```
k1_grid, k5_grid = np.meshgrid(np.linspace(-5.0, 5.0, 201), np.linspace(-5.0, 5.0, 201))
E2_grid = 1.0 + k1_grid**2 - k5_grid**2  # m = 1
kappa_grid = np.sqrt(np.maximum(-E2_grid, 0.0))
growing_fraction = float(np.mean(E2_grid < 0.0))
report("fraction of the grid where the waves grow", f"{growing_fraction:.4f}")
check(abs(kappa_grid[100, 100]) == 0.0 and abs(kappa_grid[200, 100] - 24**0.5) < 1e-12,
      "kappa = 0 at k = 0 and kappa = sqrt(24) at k1 = 0, k5 = 5")
```

`np.meshgrid` makes two 201×201 arrays: at row r and column c, `k1_grid` holds the c-th value of k_1 and `k5_grid` the r-th value of k_5, each running from -5 to 5 in steps of 0.05. `E2_grid` is $E^2 = 1 + k_1^2 - k_5^2$ at every grid point and `kappa_grid` the growth rate. `np.mean` of the true/false array `E2_grid < 0.0` is the fraction of grid points where the waves grow: RESULT 0.4439. The check tests two points: row 100, column 100 is $k_1 = k_5 = 0$, where $\kappa = 0$; row 200, column 100 is $k_5 = 5$, $k_1 = 0$, where $\kappa = \sqrt{25 - 1} = \sqrt{24}$.

```
fig, ax = plt.subplots(figsize=(6.0, 5.0))
shown = np.where(E2_grid < 0.0, kappa_grid, np.nan)  # nan: not coloured (white)
image = ax.pcolormesh(k1_grid, k5_grid, shown, cmap="viridis", shading="auto")
fig.colorbar(image, ax=ax, label="growth rate $\\kappa$")
lines = ax.contour(k1_grid, k5_grid, kappa_grid, levels=[1, 2, 3, 4], colors="white",
                   linewidths=0.8)
ax.clabel(lines, fmt="%d", fontsize=8)
ax.contour(k1_grid, k5_grid, E2_grid, levels=[0.0], colors="black", linewidths=1.2)
ax.text(2.6, 0.0, "oscillation\n$E^2 > 0$", ha="center", va="center")
ax.set_xlabel("space momentum $k_1$")
ax.set_ylabel("extra-time momentum $k_5$")
ax.set_title("Growth rate of the waves, $m = 1$")
save_figure(fig, "growth_map",
            "The growth rate $\\kappa$ in the plane of the space momentum $k_1$ "
            ...
            "makes the waves grow.")
```

`np.where(condition, a, b)` takes `a` where the condition holds and `b` elsewhere; `np.nan` ("not a number") is left uncoloured, so the oscillating region stays white. `pcolormesh` colours the grid with the colour map `"viridis"`; `contour` draws the curves of equal growth rate $\kappa = 1, 2, 3, 4$ in white, and `clabel` writes their values on them (`"%d"`: as whole numbers); a second `contour` draws the border $E^2 = 0$ in black; `text` writes a label at $(2.6, 0)$ (`\n` starts a new line). The figure is Figure 08a.2. Output: one RESULT line, one PASS line, the figure line.

What Figure 08a.2 shows. The plane of the space momentum k_1 (horizontal) and the extra-time momentum k_5 (vertical), both from -5 to 5 in units of the inverse length, for $m = 1$. The white region around the horizontal axis is where $E^2 = 1 + k_1^2 - k_5^2 > 0$: the waves oscillate. Above and below it, bounded by the black hyperbola $k_5^2 - k_1^2 = 1$, the colour shows the growth rate, with white contour lines at $\kappa = 1, 2, 3, 4$. The student should see that the two coloured regions reach to infinity: however large k_1 is, a large enough k_5 makes the waves grow.

In [8], solving in time, in two independent ways.

```
def evolve(hk, E2_value, x4, u0):
    """u(x4) = (cos(E x4) - i sin(E x4)/E h_k) u0, with E = sqrt(E^2) (complex)."""
    if E2_value == 0.0:
        return u0 - 1j * x4 * (hk @ u0)  # the limit E -> 0
    E = np.sqrt(complex(E2_value))  # E is imaginary when E^2 < 0
    return np.cos(E * x4) * u0 - 1j * np.sin(E * x4) / E * (hk @ u0)
```

`evolve` is the closed formula of Section 8.18: for $E^2 = 0$ the limit $u_0 - ix_4 h_k u_0$, otherwise $\cos(Ex_4)u_0 - i\frac{\sin(Ex_4)}{E} h_k u_0$, where `np.sqrt(complex(...))` gives the imaginary $E = i\kappa$ when $E^2 < 0$ (numpy's cosine and sine of an imaginary number are then the hyperbolic functions).

```
def rk4(hk, x4_end, u0, steps):
    """The classical RK4 method for du/dx4 = -i h_k u from 0 to x4_end."""
    step = x4_end / steps
    u = u0.astype(complex)
    for _ in range(steps):
        s1 = -1j * (hk @ u)
        s2 = -1j * (hk @ (u + step / 2 * s1))
```

```
        s3 = -1j * (hk @ (u + step / 2 * s2))
        s4 = -1j * (hk @ (u + step * s3))
        u = u + step / 6 * (s1 + 2 * s2 + 2 * s3 + s4)
    return u
```

`rk4` is the classical fourth-order Runge-Kutta method of Chapter 2 for $du/dx_4 = -ih_k u$: in each step it evaluates the right-hand side four times (at the start, twice at the middle, at the end) and advances with the weighted average $\frac{1}{6}(s_1 + 2s_2 + 2s_3 + s_4)$. It knows nothing of the closed formula, so the two are independent.

```
rng = np.random.default_rng(12345)  # fixed seed: the same numbers in every run
u_start = rng.normal(size=16) + 1j * rng.normal(size=16)
u_start = u_start / np.sqrt(np.vdot(u_start, u_start).real)  # u^dagger u = 1
```

A random-number generator with the fixed **seed** 12345 produces the same "random" numbers in every run. The starting column has 16 complex entries whose real and imaginary parts are drawn from the normal (bell-shaped) distribution; dividing by $\sqrt{u^\dagger u}$ makes its size 1 (`np.vdot(a, b)` is $a^\dagger b$).

```
worst = 0.0
for K in (0.5, 1.0, 2.0, 3.0):
    hk = mode_matrix(1.0, {5: K})
    exact = evolve(hk, 1.0 - K**2, 3.0, u_start)
    stepped = rk4(hk, 3.0, u_start, 3000)
    relative = np.linalg.norm(exact - stepped) / np.linalg.norm(exact)
    worst = max(worst, relative)
    say(f"K = {K}: u^dagger u at x4 = 3 is {np.vdot(exact, exact).real:.6e}")
check(worst < 1e-9, "the closed formula and RK4 agree at x4 = 3 (relative < 1e-9)")
```

For four extra-time momenta (with $m = 1$, so $E^2 = 1 - K^2$) the wave is followed to $x_4 = 3$ both ways, RK4 with 3000 steps. The relative difference (the length of the difference divided by the length of the exact column; `np.linalg.norm` is the length $\sqrt{u^\dagger u}$) is recorded, and the largest one must be below 10^{-9}. The printed sizes at $x_4 = 3$ (`:.6e` prints in powers of ten) are 1.347676 for $K = 0.5$ (oscillation), 20.06346 for $K = 1$ ($E = 0$, growth like a power of x_4), 1.880098×10^4 for $K = 2$ and 1.086955×10^7 for $K = 3$ (COMPUTED). Output: four lines and one PASS line.

In [9], the size of a wave in time.

```
x4_values = np.linspace(0.0, 3.0, 301)
fig, ax = plt.subplots()
for K in (0.5, 1.0, 2.0, 3.0, 5.0):
    hk = mode_matrix(1.0, {5: K})
    sizes = [np.vdot(v, v).real for v in
             (evolve(hk, 1.0 - K**2, x4, u_start) for x4 in x4_values)]
    ax.plot(x4_values, np.log(sizes), label=f"$K = {K}$")
```

For five momenta the size $u^\dagger u$ is computed with the closed formula at 301 times from 0 to 3, and its natural logarithm is drawn (`np.log`).

```
    if K > 1.0:  # the asymptotic straight line of slope 2 kappa
        kappa = growth_rate(1.0, K)
        ax.plot(x4_values, np.log(sizes[-1]) + 2 * kappa * (x4_values - 3.0), "--",
                color="gray", linewidth=0.8)
# An empty line that only adds the dashed straight lines to the legend:
ax.plot([], [], "--", color="gray", linewidth=0.8, label="slope $2\\kappa$")
ax.set_xlabel("time $x_4$")
ax.set_ylabel("$\\ln(u^\\dagger u)$")
ax.set_title("Size of a wave in time, $m = 1$")
ax.legend();
save_figure(fig, "norm_in_time",
            "The natural logarithm of the size $u^\\dagger u$ of a wave against the "
            ...
            "$2\\kappa$ (gray dashed), $2\\sqrt3$, $2\\sqrt8$ and $2\\sqrt{24}$.")
```

For the growing waves a gray dashed straight line of slope 2κ is drawn through the last point (`sizes[-1]` is the last element): $u^\dagger u$ grows like $e^{2\kappa x_4}$ (the size squared of $e^{\kappa x_4}$), so its logarithm becomes a straight line of slope 2κ. The empty plot only adds the gray line to the legend. The figure is Figure 08a.3; the cell prints only the figure line.

What Figure 08a.3 shows. The natural logarithm of $u^\dagger u$ (a pure number) against the time x_4 (units of $1/m$), for $m = 1$ and $K = 0.5, 1, 2, 3, 5$, all started from the same column of size 1 ($\ln 1 = 0$). For $K = 0.5$ the curve stays near 0 (oscillation); for $K = 1$ it rises slowly (a power of x_4); for $K = 2, 3, 5$ it soon runs along a straight line of slope $2\sqrt{3}$, $2\sqrt{8}$ and $2\sqrt{24}$ (gray dashed). The student should see exponential growth whose rate increases with the extra-time momentum.

In [10], Hadamard's amplification.

```
def growing_eigenvector(mass, K):
    """A unit eigenvector of h_k (k5 = K) for the eigenvalue +i kappa."""
    hk = mode_matrix(mass, {5: K})
    kappa = growth_rate(mass, K)
    # The null space of h_k - i kappa I: the right singular vector of the smallest
    # singular value (numpy sorts the singular values from large to small).
    _, _, vh = np.linalg.svd(hk - 1j * kappa * np.eye(16))
    u_plus = vh[-1].conj()
    return hk, kappa, u_plus / np.linalg.norm(u_plus)
```

`growing_eigenvector` finds a column u_+ with $h_k u_+ = i\kappa u_+$. The **singular value decomposition** (`np.linalg.svd`) writes a matrix A as $U\Sigma V^\dagger$ with a list of numbers Σ (the singular values, sorted from large to small) and two matrices U and V whose columns have length 1 and are perpendicular; a column of V whose singular value is zero solves $Au = 0$. numpy returns $V^\dagger$ as `vh`, so the last row of `vh`, conjugated, is the last column of V. The column is divided by its length.

```
hk, kappa, u_plus = growing_eigenvector(1.0, 4.0)
later = evolve(hk, 1.0 - 4.0**2, 1.0, u_plus)
check(np.linalg.norm(hk @ u_plus - 1j * kappa * u_plus) < 1e-12
      and abs(np.linalg.norm(later) - np.exp(kappa)) < 1e-9 * np.exp(kappa),
      "a growing eigenvector grows exactly like e^(kappa x4) (K = 4)")
```

For $m = 1$, $K = 4$ ($\kappa = \sqrt{15}$) the check confirms that u_+ is an eigenvector to 10^{-12} and that its length at $x_4 = 1$ is e^κ to a relative 10^{-9}.

```
K_axis = np.linspace(0.0, 200.0, 2001)
fig, ax = plt.subplots()
for s in (0, 2, 4, 8):
    log_ratio = (growth_rate(1.0, K_axis) - s / 2 * np.log(1 + K_axis**2)) / np.log(10)
    ax.plot(K_axis, log_ratio, label=f"$s = {s}$")
    crossing = K_axis[np.argmax(log_ratio > 6.0)]  # first K with ratio above 10^6
    report(f"s = {s}: first K (step 0.1) with ratio above 10^6", f"{crossing:.1f}")
    check(log_ratio[-1] > 50.0, f"s = {s}: the ratio exceeds 10^50 at K = 200")
```

For K from 0 to 200 in steps of 0.1 and four Sobolev orders, `log_ratio` is the base-10 logarithm of the amplification, $(\kappa - \frac{s}{2}\ln(1+K^2))/\ln 10$ (Section 8.18). `np.argmax` of a true/false array gives the position of the first true value, so `crossing` is the first K at which the ratio exceeds 10^6; it is printed (13.9, 19.9, 27.1, 44.2). Each check requires the ratio to exceed 10^{50} at $K = 200$ (`log_rati o[-1]` is the last value).

```
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("extra-time momentum $K$")
ax.set_ylabel("$\\log_{10}$(solution at $x_4 = 1$ / data)")
ax.set_title("Amplification after one unit of time, $m = 1$")
ax.legend();
save_figure(fig, "hadamard_ratio",
            "The base-10 logarithm of the amplification, the size of the solution "
            ...
            "solutions and the problem is not well posed in the sense of Hadamard.")
```

Axis labels, a black line at 0 (amplification 1), and Figure 08a.4. Output: one PASS line, four RESULT lines with four PASS lines, the figure line.

What Figure 08a.4 shows. The base-10 logarithm of the amplification after one unit of time (vertical; a pure number) against the extra-time momentum K from 0 to 200 (horizontal, units of m), for the Sobolev orders $s = 0, 2, 4, 8$. The curves of the larger orders first dip below 0 (for $s = 8$ down to about -4, back above 0 near $K = 26$, where $\sqrt{K^2-1} = 4\ln(1+K^2)$), because a strong norm punishes the wiggles of the data; then every curve rises along a nearly straight line of slope about $1/\ln 10 \approx 0.43$, without bound (at $K = 200$ the values lie between about 68 and 87). The student should see that a stronger norm only delays the rise: arbitrarily small data give arbitrarily large solutions, the failure of Hadamard's third condition.

In [11], the Krein form.

```
hk = mode_matrix(1.0, {5: 2.0})
times = np.linspace(0.0, 4.0, 401)
path = [evolve(hk, 1.0 - 4.0, x4, u_start) for x4 in times]
hilbert = np.array([np.vdot(v, v).real for v in path])  # u^dagger u
krein_values = np.array([np.vdot(v, B @ v) for v in path])  # u^dagger B u
drift = np.max(np.abs(krein_values - krein_values[0]) / np.maximum(hilbert, 1.0))
report("Krein form at x4 = 0", f"{krein_values[0].real:+.9f}")
report("u^dagger u at x4 = 4", f"{hilbert[-1]:.6e}")
imaginary = np.max(np.abs(krein_values.imag) / np.maximum(hilbert, 1.0))
check(drift < 1e-12 and imaginary < 1e-12,
      "the Krein form u^dagger B u is constant (drift / max(u^dagger u, 1) < 1e-12)")
```

The random column of In [8] is followed with $m = 1$, $K = 2$ to $x_4 = 4$ at 401 times; `hilbert` holds $u^\dagger u$ and `krein_values` holds $u^\dagger B u$. The **drift** is the largest change of the Krein form, divided by $\max(u^\dagger u, 1)$: rounding errors grow with the size of the numbers, so the change is measured relative to it. The two RESULT lines print the Krein form at the start, $+0.129919115$, and $u^\dagger u = 6.006621 \times 10^5$ at $x_4 = 4$. The check requires the drift and the imaginary part of the Krein form (a Hermitian form gives real values) below 10^{-12}.

```
def eigenspace(hk, value):
    """An orthonormal basis (columns) of the eigenspace of hk for value."""
    _, singular, vh = np.linalg.svd(hk - value * np.eye(16))
    return vh[singular < 1e-9].conj().T
```

`eigenspace` returns, as columns, all right singular vectors of $h_k - \lambda I$ whose singular value is below 10^{-9}: an **orthonormal basis** (columns of length 1, perpendicular to each other) of the eigenspace of λ. `vh[singular < 1e-9]` selects the rows of `vh` with small singular values.

```
V_grow = eigenspace(mode_matrix(1.0, {5: 2.0}), 1j * 3**0.5)
form_grow = V_grow.conj().T @ B @ V_grow  # the Krein form on that eigenspace
check_record(V_grow.shape[1] == 8 and np.max(np.abs(form_grow)) < 1e-12,
             "m = 1, k5 = 2: the 8-dim eigenspace of +i sqrt(3) is Krein-neutral",
             "Revision/pairing/reports/python-pairing.json",
             "Q.one_particle_complex_frequency_Krein_neutral",
             "(0, 0, 0, 0, 2, 0, 0): w^2 = -3, eigenspace dimensions "
             f"[{V_grow.shape[1]}, {V_grow.shape[1]}], B-form identically zero: True")
```

The eigenspace of the growing frequency $+i\sqrt{3}$ ($m = 1$, $k_5 = 2$) has 8 basis columns. The 8×8 matrix `form_grow` holds every value $v^\dagger B u$ for two basis columns; the check requires dimension 8 and all 64 values below 10^{-12} (the Krein-neutrality of Section 8.19), and the pairing record's text for the same sample (it lists the momenta in the order $k_1, k_2, k_3, k_5, k_6, k_7, k_0$ and calls the frequency w).

```
V_real = eigenspace(mode_matrix(1.0, {0: 2.0, 1: 2.0}), 3.0)
form_real = np.linalg.eigvalsh(V_real.conj().T @ B @ V_real)  # Hermitian 8 x 8
inertia = (int(np.sum(form_real > 1e-9)), int(np.sum(form_real < -1e-9)))
report("Krein inertia of the eigenspace of E = 3 (m = 1, k0 = k1 = 2)", inertia)
check_record(V_real.shape[1] == 8 and inertia == (4, 4),
             "m = 1, k0 = k1 = 2: the eigenspace of E = 3 has Krein inertia (4, 4)",
             "Revision/pairing/reports/python-pairing.json",
             "Q.one_particle_Krein_inertia",
             f"w = 3: dim {V_real.shape[1]}, Krein inertia ({inertia[0]},{inertia[1]})")
```

For the good-sector wave $m = 1$, $k_1 = k_0 = 2$ ($E^2 = 1 + 4 + 4 = 9$) the eigenspace of $E = 3$ is found; the Krein form restricted to it is a Hermitian 8×8 matrix, whose eigenvalues `np.linalg.eigvalsh` computes. Counting the positive and the negative ones gives the inertia, printed as `(4, 4)`; the check also requires the pairing record's text.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(times, np.log(hilbert))
```

```
left.set_xlabel("time $x_4$")
left.set_ylabel("$\\ln(u^\\dagger u)$")
left.set_title("Hilbert norm: grows")
right.plot(times, krein_values.real)
right.axhline(krein_values[0].real, color="gray", linestyle="--", linewidth=0.8)
right.set_ylim(krein_values[0].real - 0.5, krein_values[0].real + 0.5)
right.set_xlabel("time $x_4$")
right.set_ylabel("$u^\\dagger B u$")
right.set_title("Krein form: constant")
save_figure(fig, "krein_norm",
            "One wave with $m = 1$ and extra-time momentum $K = 2$ followed from "
            ...
            "positive and the negative parts of the Krein form grow together.")
```

Left: $\ln(u^\dagger u)$ against the time. Right: the Krein form, with a gray dashed line at its starting value and a vertical range of ± 0.5 around it. The figure is Figure 08a.5. Output: three RESULT lines, three PASS lines (two with `reproduces` lines), the figure line.

What Figure 08a.5 shows. One wave ($m = 1$, $K = 2$) from $x_4 = 0$ to 4 (units of $1/m$). Left: the logarithm of its Hilbert norm rises, after a short start, along a straight line of slope $2\sqrt{3}$, to about $\ln(6.0 \times 10^5) \approx 13.3$. Right: its Krein form (a pure number) is a flat line on the gray dashed line at 0.1299. The student should see the ordinary size growing by almost six powers of ten while the Krein form does not move, possible because the growing waves carry Krein form zero.

In [12], the good sector, and the eigenvalue paths.

```
good = mode_matrix(1.3, {0: 2.2, 1: 0.4, 2: -1.1, 3: 0.7})  # no k5, k6, k7
good_eigenvalues = np.linalg.eigvals(good)
E_good = np.sqrt(1.3**2 + 0.4**2 + 1.1**2 + 0.7**2 + 2.2**2)
check_record(np.allclose(good, good.conj().T) and np.allclose(good @ B, B @ good)
             and np.max(np.abs(np.abs(good_eigenvalues) - E_good)) < 1e-12
             and np.max(np.abs(good_eigenvalues.imag)) < 1e-12,
             "good sector: h_k Hermitian, [B, h_k] = 0, eigenvalues real +-E",
             "Revision/theory/reports/python-field-theory.json",
             "good_sector_spectrum_and_B_sectors", "h is Hermitian", "[B, h] = 0")
```

A good-sector mode matrix with $m = 1.3$ and four space momenta (no extra-time momentum). The check requires: h_k Hermitian, commuting with B, every eigenvalue of size $E = \sqrt{m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2}$ and with imaginary part below 10^{-12}, so the eigenvalues are $\pm E$, real; and the record's texts.

```
path_values = np.linspace(0.0, 2.5, 26)  # the momenta 0, 0.1, ..., 2.5
colours = np.repeat(path_values, 16)  # each matrix has 16 eigenvalues
fig, panels = plt.subplots(1, 2, figsize=(10.0, 4.6), sharey=True)
for panel, direction, name in ((panels[0], 1, "space momentum $k_1$ (good sector)"),
                               (panels[1], 5, "extra-time momentum $K = k_5$")):
    points = np.concatenate([np.linalg.eigvals(mode_matrix(1.0, {direction: p}))
                             for p in path_values])
    dots = panel.scatter(points.real, points.imag, c=colours, cmap="plasma", s=22)
```

For 26 momenta from 0 to 2.5, the 16 eigenvalues of h_k ($m = 1$) are computed with the momentum along x_1 (left panel) or along x_5 (right panel); `np.concatenate` joins the 26 lists into one. `np.repea`

`t(path_values, 16)` repeats each momentum 16 times, so that every eigenvalue gets the colour of its momentum. `scatter` draws them as dots in the complex plane (real part horizontal, imaginary part vertical).

```
    panel.set_xlim(-3.0, 3.0)
    panel.set_ylim(-3.0, 3.0)
    panel.set_aspect("equal")
    panel.set_xlabel("real part of the eigenvalue $E$")
    panel.set_title(name)
panels[0].set_ylabel("imaginary part of the eigenvalue $E$")
fig.colorbar(dots, ax=panels, label="momentum (0 to 2.5)")
save_figure(fig, "eigenvalue_paths",
            "The eigenvalues $E$ of the mode matrix for $m = 1$ in the complex "
            ...
            "leave along the imaginary axis as $\\pm i\\sqrt{K^2 - 1}$: growth.")
```

Equal ranges and equal scales on both axes (`set_aspect("equal")`), labels, one colour bar for both panels, and Figure 08a.6. Output: one PASS line with its `reproduces` line, the figure line.

What Figure 08a.6 shows. The eigenvalues E of the mode matrix for $m = 1$ in the complex plane (horizontal: real part; vertical: imaginary part; units of m), coloured from dark (momentum 0) to bright (momentum 2.5). Left, a growing space momentum: the eigenvalues $\pm\sqrt{1 + k_1^2}$ start at ± 1 and move outwards along the real axis; they stay real. Right, a growing extra-time momentum: the eigenvalues $\pm\sqrt{1 - K^2}$ move inwards along the real axis, meet at 0 for $K = 1$, and then leave along the imaginary axis as $\pm i\sqrt{K^2 - 1}$. The student should see the moment at which oscillation turns into growth.

In [13], the last check.

```
for name in ("08a_1_growth_rate_vs_momentum.png", "08a_2_growth_map.png",
             "08a_3_norm_in_time.png", "08a_4_hadamard_ratio.png",
             "08a_5_krein_norm.png", "08a_6_eigenvalue_paths.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Six checks that the figure files exist, and the last line. The 30 checks come from the cells as follows: three in In [2], four in In [3], two each in In [4], In [5] and In [6], one each in In [7] and In [8], five in In [10], three in In [11], one in In [12] and six here (In [9] has none). The last line reads `ALL 30 C HECKS PASSED (notebook 08a)`.

8.24 The deflating history pushes every extra-time wave into growth

Sections 8.17 to 8.19 froze the coefficients. In the author's metric they are not frozen: the extra times deflate, and this changes the momentum that a wave has along them. This section follows the change exactly.

The hidden coordinate y. A second way to label the points of the hidden direction is $y = \ln(\sin z)/(6H)$. At the patch end $z = \pi/2$, $\sin z = 1$ and $y = 0$; towards the tip $z \to 0$, $\sin z \to 0$

and $y \to -\infty$. Two facts make y useful, each derived in one line. First, $\sin z = e^{6Hy}$ (undo the logarithm), so $\sin^{1/6} z = e^{Hy}$ and $\sin^{1/3} z = e^{2Hy}$. Second, $dy/dx_0 = \frac{1}{6H} \cdot \frac{\cos z}{\sin z} \cdot 6H = \cot z$ (chain rule with $dz/dx_0 = 6H$), so $\cot^2 z\, dx_0^2 = dy^2$. The author's metric therefore reads

$$ds^2 = e^{2Hy}\big(e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 + dx_7^2)\big) - dx_4^2 + dy^2,$$

and y measures proper length along the hidden direction. The Kohn-Sham record cuts the tip at $y = -L$ with $L = 3$ (`Revision/kohn_sham/results/parameters.json`, entry `physics.L_tipCut off`).

The history. The examples use the history $a_4 = AHx_4$ with $A = 1$ and $H = m = 1$, the canonical history of the Kohn-Sham record (`parameters.json`, entries `physics.historyA`, `physics.H`, `physics.m`). It is ASSUMED: the record calls it a PRESCRIBED BACKGROUND, chosen and not solved for, because the Kohn-Sham states along it do not satisfy the source conditions of the a_4 equations (Chapter 17). Along it the 3-space scale factor $e^{a_4}e^{Hy}$ grows exponentially, the extra-time scale factor $e^{-a_4}e^{Hy}$ shrinks exponentially, and the product of the six, $e^{6Hy} = \sin z$, does not change.

Wave numbers that do not change. The coefficients of the field equation in the author's metric do not depend on $x_1, x_2, x_3, x_5, x_6, x_7$. So a field of the form $\Psi = e^{i(q_1x_1 + q_5x_5)}\psi(x_4, y)$ keeps this form for all times: every derivative ∂_1 or ∂_5 acting on it gives the factor iq_1 or iq_5, the common exponential divides out, and the equation for ψ contains neither x_1 nor x_5. The numbers q_1 and q_5 are **coordinate wave numbers**: radians of phase per unit of the coordinate. They are exact constants of the motion.

Frame momenta that do change. A frame momentum counts radians per unit of PROPER length; a coordinate length Δx_a has the proper length $f_a \Delta x_a$, so $k_{(a)} = q_a/f_a$. Line by line:

1. $k_{(1)} = q_1/f_1 = q_1 e^{-a_4} e^{-Hy}$, because $f_1 = e^{a_4} \sin^{1/6} z = e^{a_4} e^{Hy}$;
2. $k_{(5)} = q_5/f_5 = q_5 e^{a_4} e^{-Hy}$, because $f_5 = e^{-a_4} \sin^{1/6} z = e^{-a_4} e^{Hy}$;
3. with frozen coefficients at the time x_4 (Section 8.17), $E^2 = m^2 + k_{(1)}^2 - k_{(5)}^2 = m^2 + q_1^2 e^{-2Hy} e^{-2a_4} - q_5^2 e^{-2Hy} e^{2a_4}$.

As the extra times deflate (a_4 grows), a fixed number of wiggles per unit of x_5 means more and more wiggles per unit of proper length: the frame momentum $k_{(5)}$ grows like e^{a_4}, while $k_{(1)}$ shrinks like e^{-a_4} because 3-space inflates.

The onset of growth. Write $X = e^{2a_4}$ and $w = e^{-Hy}$. Then $e^{-2a_4} = 1/X$ and $E^2 = m^2 + q_1^2 w^2/X - q_5^2 w^2 X$. As a function of $X > 0$ it decreases: its derivative $-q_1^2 w^2/X^2 - q_5^2 w^2$ is negative for $q_5 \neq 0$. So along the history (where X grows with the time) E^2 crosses zero exactly once and stays negative afterwards; the moment of the crossing is the **onset**. To find it, line by line:

$$
\begin{aligned}
0 &= m^2 + \frac{q_1^2 w^2}{X} - q_5^2 w^2 X, \\
0 &= q_5^2 w^2 X^2 - m^2 X - q_1^2 w^2, \\
X^* &= \frac{m^2 + \sqrt{m^4 + 4 q_1^2 q_5^2 w^4}}{2 q_5^2 w^2}, \\
x_4^* &= \frac{\ln X^*}{2AH}.
\end{aligned}
$$

The first line sets $E^2 = 0$; the second multiplies by $-X$ (allowed, $X > 0$); the third is the quadratic formula, with the root that is positive (the other root, with the minus sign, is not positive, because the square root is at least m^2, and $X = e^{2a_4}$ must be positive); the fourth undoes $X = e^{2a_4} = e^{2AHx_4}$. For a wave without space momentum, $q_1 = 0$, the third line is $X^* = m^2/(q_5^2 w^2)$, and the onset time simplifies to

$$
x_4^* = \frac{\ln(m/q_5) - \ln w}{AH} = \frac{Hy + \ln(m/q_5)}{AH}.
$$

At $y = 0$ with $A = H = m = 1$ this is $\ln(1/q_5)$: $x_4^* = \ln 20 = 2.995732$ for $q_5 = 0.05$ and $\ln 10 = 2.302585$ for $q_5 = 0.1$ (Notebook 08b prints both). The onset time is finite for every $q_5 > 0$: a smaller wave number only delays it, logarithmically. So every wave along an extra time eventually stops oscillating and starts to grow, and after the onset its growth rate $\kappa = \sqrt{k_{(5)}^2 - m^2 - k_{(1)}^2}$ keeps increasing, since $k_{(5)}$ grows without bound (PROVED with frozen coefficients at each instant; this is the local-frame statement of the record, `Revision/theory/reports/python-field-theory.json`, check `extra_time_modes_grow`: "every extra-time mode eventually enters the growing regime"). Restricting the theory to the good sector, the fields that do not depend on the extra times, removes this growth; it is a choice, not a consequence of the equations.

8.25 Following one wave through the onset: the local-frame model and WKB

The local-frame model (labelled MODEL; ASSUMED). The field also depends on y, and the factor e^{-Hy} in the frame momenta makes the exact equation couple neighbouring values of y. The **local-frame model** holds the hidden position fixed (it freezes the coefficients in y only), works in the variables χ of Section 8.30 (in which the term $3H\gamma^{(0)}$ is absent exactly), takes no momentum along the hidden direction, and keeps the exact time dependence of the frame momenta:

$$
i\,\frac{du}{dx_4} = h(x_4)\,u, \qquad h(x_4) = -im\gamma^{(4)} - \gamma^{(4)}\big(k_{(1)}(x_4)\gamma^{(1)} + k_{(5)}(x_4)\gamma^{(5)}\big).
$$

Its solutions are not exact solutions of the field equation; they show how a wave behaves near one hidden position as the extra times deflate. The position enters only through $q_5 e^{-Hy}$, so the runs use $y = 0$. At every instant $h(x_4)^2 = E^2(x_4) I_{16}$ with the E^2 of Section 8.24 (Section 8.17 with the frame momenta of that instant).

A clean starting column. Take $q_1 = 0$. The matrix $C = \gamma^{(0)}\gamma^{(1)}\gamma^{(2)}\gamma^{(3)}$ commutes with $\gamma^{(4)}$ and with $\gamma^{(5)}$ (moving either of them through the four factors of C changes the sign four times), hence with $h(x_4)$. A column u can therefore be an eigenvector of $h(0)$ for the positive frequency

$E_0 = \sqrt{m^2 - q_5^2}$ and at the same time of C for the eigenvalue $\sigma = +1$ or -1 (C is real, symmetric, and $C^2 = I_{16}$). For such a column of size $u^\dagger u = 1$ the Krein form is $u^\dagger B u = \sigma E_0/m$. Line by line:

1. $B = -iC\gamma^{(4)} = C(-i\gamma^{(4)})$, and since C is Hermitian with $Cu = \sigma u$, $u^\dagger C = \sigma u^\dagger$; so $u^\dagger B u = \sigma\, u^\dagger(-i\gamma^{(4)})u$.
2. $E_0 = E_0 u^\dagger u = u^\dagger h(0) u = m\, u^\dagger(-i\gamma^{(4)})u - q_5\, u^\dagger\gamma^{(4)}\gamma^{(5)}u$ (at $x_4 = 0$, $a_4 = 0$ and $k_{(5)} = q_5$ at $y = 0$).
3. $-i\gamma^{(4)}$ is Hermitian, so $u^\dagger(-i\gamma^{(4)})u$ is real; $\gamma^{(4)}\gamma^{(5)}$ is real and antisymmetric, so $u^\dagger\gamma^{(4)}\gamma^{(5)}u$ is purely imaginary (its complex conjugate is $u^T\gamma^{(4)}\gamma^{(5)}u^* = u^\dagger(\gamma^{(4)}\gamma^{(5)})^T u = -u^\dagger\gamma^{(4)}\gamma^{(5)}u$, a number being equal to its own transpose).
4. E_0 is real, so the imaginary term vanishes, $u^\dagger(-i\gamma^{(4)})u = E_0/m$, and by line 1, $u^\dagger B u = \sigma E_0/m$.

The notebook builds such columns with two **projectors** (matrices P with $P^2 = P$): $\Lambda_+ = \frac{1}{2}(I_{16} + h(0)/E_0)$ keeps the part of a column with eigenvalue $+E_0$, and $\frac{1}{2}(I_{16} + \sigma C)$ the part with $C = \sigma$; they commute, so their product applied to the first unit column gives a column with both properties.

The leading WKB approximation. After the onset write $Q = k_{(5)} = q_5 e^{AHx_4}$ (at $y = 0$) and $\kappa = \sqrt{Q^2 - m^2}$. When κ changes slowly, the wave follows the growing eigenvalue of $-ih$ at each instant; this is the **WKB approximation** (after Wentzel, Kramers and Brillouin). Line by line:

1. At each instant the growing eigenvalue of $-ih$ is $+\kappa$, so the size grows at the rate $\frac{d}{dx_4}\ln(u^\dagger u) \approx 2\kappa$, and $\ln(u^\dagger u) \approx 2W +$ const with $W(x_4) = \int \kappa\, dx_4$.
2. Since $dQ/dx_4 = AH\,Q$, the substitution $dx_4 = dQ/(AHQ)$ gives $W = \frac{1}{AH}\int \frac{\sqrt{Q^2-m^2}}{Q}\, dQ$.
3. This integral is $W = \frac{1}{AH}(\sqrt{Q^2 - m^2} - m \arccos\frac{m}{Q})$. To check it, differentiate the bracket with respect to Q: $\frac{d}{dQ}\sqrt{Q^2-m^2} = \frac{Q}{\kappa}$, and $\frac{d}{dQ}\arccos\frac{m}{Q} = -\frac{1}{\sqrt{1-m^2/Q^2}}\cdot(-\frac{m}{Q^2}) = \frac{m}{Q\kappa}$; so the derivative of the bracket is $\frac{Q}{\kappa} - \frac{m^2}{Q\kappa} = \frac{Q^2-m^2}{Q\kappa} = \frac{\kappa}{Q}$, the integrand.

The first-order WKB correction. The matrices $A_4 = -i\gamma^{(4)}$ and $M = -i\gamma^{(4)}\gamma^{(5)}$ are Hermitian, square to I_{16} ($(-i\gamma^{(4)})^2 = -(\gamma^{(4)})^2 = I_{16}$; $(\gamma^{(4)}\gamma^{(5)})^2 = -(\gamma^{(4)})^2(\gamma^{(5)})^2 = -I_{16}$, so $M^2 = I_{16}$) and anticommute, and with $q_1 = 0$, $h = mA_4 - iQM$. Line by line:

1. Take a column w with $A_4 w = w$; then $A_4(Mw) = -MA_4 w = -Mw$, and h maps the plane spanned by w and Mw into itself: $hw = mw - iQMw$ and $h(Mw) = -mMw - iQw$. For $u = v_1 w + v_2 Mw$ the equation $du/dx_4 = -ihu$ becomes $v' = \begin{pmatrix} -im & -Q \\ -Q & im \end{pmatrix} v$ (there are 8 such planes).
2. This symmetric 2×2 matrix has the eigenvalues $\pm\kappa$ (its determinant condition is $(-im - \lambda)(im - \lambda) - Q^2 = \lambda^2 + m^2 - Q^2 = 0$) and, for $+\kappa$, the eigenvector $r = (Q, -\kappa - im)$ (the first row gives $(-im - \kappa)Q - Q(-\kappa - im) = 0$).

3. Keep only the growing part, $v \approx c\,r$. Then $v' = c'r + cr' \approx \kappa cr$; multiplying from the left by the row r^T and writing $n = r^T r$ gives $c'n + c\,r^T r' = \kappa cn$, and $r^T r' = n'/2$ (the derivative of $r^T r$ is $2r^T r'$ for a symmetric product). So $c'/c = \kappa - n'/(2n)$.

4. $n = Q^2 + (\kappa + im)^2 = Q^2 + \kappa^2 - m^2 + 2im\kappa = 2\kappa(\kappa + im)$, using $Q^2 - m^2 = \kappa^2$; its size is $|n| = 2\kappa\sqrt{\kappa^2 + m^2} = 2\kappa Q$.

5. Integrating line 3, $\ln c = W - \frac{1}{2}\ln n +$ const, so $|c|^2 \propto e^{2W}/|n| = e^{2W}/(2\kappa Q)$. The size of the column is $|c|^2 r^\dagger r = |c|^2(Q^2 + \kappa^2 + m^2) = |c|^2 \cdot 2Q^2$, so $u^\dagger u \propto e^{2W} Q/\kappa$, and

$$\ln(u^\dagger u) \approx 2W + \ln\frac{Q}{\kappa} + \text{const}, \qquad \frac{d}{dx_4}\ln(u^\dagger u) \approx 2\kappa - \frac{AH\,m^2}{\kappa^2},$$

because $\frac{d}{dx_4}\ln Q = AH$ and $\frac{d}{dx_4}\ln\kappa = \frac{Q}{\kappa^2}\frac{dQ}{dx_4} = AH\frac{Q^2}{\kappa^2}$, whose difference is $AH\frac{\kappa^2 - Q^2}{\kappa^2} = -\frac{AHm^2}{\kappa^2}$.

What the computation shows (COMPUTED). Both approximations fail at the onset itself, where $\kappa \to 0$, so Notebook 08b compares increases over a window from 1.5 to 3 time units after the onset (the unknown constants drop out of increases). For $q_5 = 0.05$ the RK4 solution of the model increases $\ln(u^\dagger u)$ by 31.0009 over the window; the leading WKB formula gives 31.0328 (relative difference 1.03×10^{-3}) and the first-order formula 31.0085 (relative difference 2.48×10^{-4}); for $q_5 = 0.1$ the numbers are 30.9993, 31.0312 and 31.0069. Late in the run the growth rate computed from the solution is 40.09851 and the first-order prediction 40.09868 (agreement to 10^{-5}). The RK4 solution itself is accurate to 10^{-8} (relative), and halving the step divides its error by 15.89, close to the value 16 of a fourth-order method. Before the onset the size does not stay exactly 1: with extra-time momentum the mode matrix is not Hermitian, so the Hilbert norm is not conserved (Section 8.19), and the size $u^\dagger u$, which starts at 1, has grown to 1.52 ($q_5 = 0.05$) and 1.54 ($q_5 = 0.1$) when the onset is reached. Three time units after the onset $\ln(u^\dagger u) = 37.071$ for $q_5 = 0.05$, that is $u^\dagger u \approx 1.26 \times 10^{16}$, while the Krein form stays constant to rounding (its change divided by $\max(u^\dagger u, 1)$ stays below 10^{-9}). Because the rate $2\kappa \approx 2q_5 e^{AHx_4}$ itself grows exponentially, the growth is faster than any exponential.

8.26 Example: the onset of growth along the deflating history

Notebook 08b puts Sections 8.24 and 8.25 to work along the prescribed history of the Kohn-Sham record. It reads A, H, m and L from the record; checks exactly that $dy/dx_0 = \cot z$, $e^{Hy} = \sin^{1/6} z$ and that the product of the scale factors is $\cos z$ for every a_4; draws the scale factors along the history; computes the frame momenta and the exact onset time and compares it with a numerical root search; shows that the onset time is finite for every $q_5 > 0$; builds the model's mode matrix, checks $h(x_4)^2 = E^2(x_4) I_{16}$ against the record's formula, chooses clean starting columns, follows two waves through the onset with RK4 (with a convergence test), and compares with both WKB formulas; and checks the Krein form. It needs no Rust and runs in well under a minute; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/08b_deflation_onset.PROVENANCE.md`. It ends with the line ALL 25 CHECKS PASSED (notebook 08b).

8.27 How to run Notebook 08b

Step 1. What this notebook does and what it needs.

Notebook 08b (The deflating extra times drive every extra-time wave into growth) is the file `Revision/textbook/notebooks/08b_deflation_onset.ipynb` of the repository Dirac_claude. Along the deflating history a4 = A H x4 of the Kohn-Sham record (A = 1, H = m = 1, a prescribed background) it computes the scale factors and the frame momenta of a wave with fixed wave numbers along x1 and x5, the exact time at which such a wave turns from oscillation to growth, and shows that every wave along an extra time reaches this onset; it then follows one wave through the onset with the local-frame model (RK4, convergence of order 4) and compares the growth with the WKB formulas, checks the Krein form, and draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08b_deflation_onset.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08b_deflation_onset.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 08b_deflation_onset.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/08b.captions.json`
- `Revision/textbook/figures/08b_1_scale_factors.png`
- `Revision/textbook/figures/08b_2_frame_momenta.png`
- `Revision/textbook/figures/08b_3_onset_times.png`
- `Revision/textbook/figures/08b_4_growth_along_history.png`
- `Revision/textbook/figures/08b_5_rate_and_krein.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08b_5_rate_and_krein.png exists
ALL 25 CHECKS PASSED (notebook 08b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/08b_deflation_onset.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 08b_deflation_onset.ipynb
python -m nbconvert --execute --inplace 08b_deflation_onset.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- FileNotFoundError naming Revision/kohn_sham/results/parameters.json: the notebook reads this Revision record of the repository. Check that your copy of the repository contains the folder Revision/kohn_sham/results with the file parameters.json (clone the repository again if it does not) and that you opened the notebook from its folder Revision/textbook/notebooks.

8.28 Notebook 08b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/08b_deflation_onset.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

In the author's metric the three extra times $x_5, x_6, x_7$ DEFLATE: their lengths
carry the factor $e^{-a_4}b_4^{1/2}\sin^{1/6}z$, and $a_4$ increases with the time
$x_4$ ($b_4(x_4) > 0$ is the second function of the metric; this notebook works on
the background $b_4 = 1$ of the record, where the factor is $e^{-a_4}\sin^{1/6}z$).
A wave that wiggles a fixed number of times per unit of the coordinate $x_5$
therefore wiggles more and more often per unit of PROPER length (length measured
with the metric): its frame momentum grows like $e^{a_4}$. This notebook uses the
deflating history of the Kohn-Sham record, $a_4 = A H x_4$ with $A = 1$ and
$H = m = 1$ (a PRESCRIBED background, not a solution of the $a_4$ equations), and

- draws the scale factors of 3-space (inflating) and of the extra times (deflating)
  and checks exactly that the volume factor $\sqrt{|g|} = b_4^{3/2}\cos z$ is
  $\cos z$ at $b_4 = 1$ and does not change in time;
- computes the frame momenta of a wave with fixed wave numbers $q_1$ along $x_1$
  and $q_5$ along $x_5$, and the exact time $x_4^\ast$ at which $E^2 = m^2 +
  k_{(1)}^2 - k_{(5)}^2$ turns negative (the *onset*), checked against a numerical
  root search;
- shows that every wave with $q_5 \neq 0$ reaches the onset at a finite time: every
  wave along an extra time eventually grows (the Revision statement
  'extra_time_modes_grow');
- follows one wave through the onset with the local-frame model (explained in
  section 4), solved with RK4 (fourth-order convergence checked), and compares its
  growth with the WKB formulas; the growth is faster than any exponential;
- checks that the Krein form stays constant while the ordinary size grows by a
  factor of about $10^{16}$.

It draws five teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 08b (The deflating extra times drive every extra-time wave into growth) is the
file 'Revision/textbook/notebooks/08b_deflation_onset.ipynb' of the repository
Dirac_claude. Along the deflating history a4 = A H x4 of the Kohn-Sham record (A = 1, H =
m = 1, a prescribed background) it computes the scale factors and the frame momenta of a
wave with fixed wave numbers along x1 and x5, the exact time at which such a wave turns
```

from oscillation to growth, and shows that every wave along an extra time reaches this onset; it then follows one wave through the onset with the local-frame model (RK4, convergence of order 4) and compares the growth with the WKB formulas, checks the Krein form, and draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 08b_deflation_onset.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 08b_deflation_onset.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp]

Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 08b_deflation_onset.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/08b.captions.json‘
- ‘Revision/textbook/figures/08b_1_scale_factors.png‘
- ‘Revision/textbook/figures/08b_2_frame_momenta.png‘
- ‘Revision/textbook/figures/08b_3_onset_times.png‘
- ‘Revision/textbook/figures/08b_4_growth_along_history.png‘
- ‘Revision/textbook/figures/08b_5_rate_and_krein.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08b_5_rate_and_krein.png exists
ALL 25 CHECKS PASSED (notebook 08b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/08b_deflation_onset.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt

  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 08b_deflation_onset.ipynb
      python -m nbconvert --execute --inplace 08b_deflation_onset.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- FileNotFoundError naming Revision/kohn_sham/results/parameters.json: the notebook reads
  this Revision record of the repository. Check that your copy of the repository contains
  the folder Revision/kohn_sham/results with the file parameters.json (clone the
  repository again if it does not) and that you opened the notebook from its folder
  Revision/textbook/notebooks.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 08b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 08b (The deflating extra times drive every extra-time wave into growth) is the
# file Revision/textbook/notebooks/08b_deflation_onset.ipynb of the repository
# Dirac_claude. Along the deflating history a4 = A H x4 of the Kohn-Sham record (A = 1, H
# = m = 1, a prescribed background) it computes the scale factors and the frame momenta
# of a wave with fixed wave numbers along x1 and x5, the exact time at which such a wave
# turns from oscillation to growth, and shows that every wave along an extra time reaches
# this onset; it then follows one wave through the onset with the local-frame model (RK4,
# convergence of order 4) and compares the growth with the WKB formulas, checks the Krein
# form, and draws five teaching plots. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
```

```
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
```

```
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
```

```
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 08b_deflation_onset.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 08b_deflation_onset.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 08b_deflation_onset.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/08b.captions.json
# - Revision/textbook/figures/08b_1_scale_factors.png
# - Revision/textbook/figures/08b_2_frame_momenta.png
# - Revision/textbook/figures/08b_3_onset_times.png
# - Revision/textbook/figures/08b_4_growth_along_history.png
# - Revision/textbook/figures/08b_5_rate_and_krein.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
```

```
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 08b_5_rate_and_krein.png exists
#     ALL 25 CHECKS PASSED (notebook 08b)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/08b_deflation_onset.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 08b_deflation_onset.ipynb
#     python -m nbconvert --execute --inplace 08b_deflation_onset.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - FileNotFoundError naming Revision/kohn_sham/results/parameters.json: the notebook
#   reads this Revision record of the repository. Check that your copy of the repository
#   contains the folder Revision/kohn_sham/results with the file parameters.json (clone
#   the repository again if it does not) and that you opened the notebook from its folder
#   Revision/textbook/notebooks.
```

```
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "08b"  # this notebook: chapter 08, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
```

```
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 08b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ ordinary space; $x_4$ the
  time; $x_5, x_6, x_7$ the three extra times (time-like, deflating); $x_0$ the
  hidden direction, $z = 6Hx_0$ between $0$ and $\pi/2$.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: a second way to label the points of
  the hidden direction. $y = 0$ at $z = \pi/2$ (the *patch end*, where the author's
  coordinate patch ends) and $y \to -\infty$ at the *tip* $z \to 0$; the Kohn-Sham
  record cuts the tip at $y = -L$ with $L = 3$. In this coordinate
  $\sin^{1/6}z = e^{Hy}$.
- **Scale factor**: the factor by which a coordinate length must be multiplied to
  give the proper length; $e^{a_4}\sin^{1/6}z$ for $x_1, x_2, x_3$ and
  $e^{-a_4}b_4^{1/2}\sin^{1/6}z$ for $x_5, x_6, x_7$ ($e^{-a_4}\sin^{1/6}z$ at
  $b_4 = 1$). **Inflating** means growing,
  **deflating** shrinking.
- **History** $a_4(x_4)$: how $a_4$ changes with time. Here $a_4 = A H x_4$ with
  $A = 1$ (the canonical history of the Kohn-Sham record). It is PRESCRIBED: chosen,
  not solved for (the record shows that the Kohn-Sham states do not satisfy the
  source conditions of the $a_4$ equations).
- **Coordinate wave number** $q_a$ and **frame momentum** $k_{(a)} = q_a/f_a$, where
  $f_a$ is the scale factor: the wave $e^{iq_a x_a}$ has $q_a$ radians of phase per
  unit of coordinate length and $k_{(a)}$ radians per unit of proper length.
- **Mode matrix** $h = -im\gamma^{(4)} - \gamma^{(4)}\sum_a k_{(a)}\gamma^{(a)}$:
  with frozen coefficients a wave $u\,e^{-iEx_4}$ needs $hu = Eu$, and $h^2 = E^2$
  with $E^2 = m^2 + k_{(1)}^2 - k_{(5)}^2$ when only $k_{(1)}$ and $k_{(5)}$ are
  nonzero. **Onset**: the time at which $E^2$ turns negative.
- **Local-frame model**: the equation $i\,du/dx_4 = h(x_4)u$ in which the frame
  momenta change with $x_4$ as the history prescribes, while the hidden position is
  held fixed (section 4).
- **WKB approximation** (after Wentzel, Kramers and Brillouin): when the growth rate
  $\kappa$ changes slowly, the size grows like $e^{2W}$ with $W = \int\kappa\,dx_4$.
- **Krein form** $u^\dagger B u$ with the matrix $B = -iC\gamma^{(4)}$ of the record
```

  ($B^\dagger = B$, $B^2 = 1$); **Hilbert norm** $u^\dagger u$.
- **RK4**: the classical fourth-order Runge-Kutta method; *fourth order* means that
  halving the step divides the error by about $2^4 = 16$.

Text cell:

```
## 4. The physical and mathematical situation

In the hidden coordinate $y$ the author's metric reads
$ds^2 = e^{2Hy}\big(e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2
+ dx_7^2)\big) - dx_4^2 + dy^2$, because $\sin^{1/3}z = e^{2Hy}$ and $dy = \cot z\,
dx_0$. Its coefficients do not depend on $x_1, x_2, x_3, x_5, x_6, x_7$, so a wave
$e^{i(q_1x_1 + q_5x_5)}$ keeps its coordinate wave numbers $q_1, q_5$ exactly for
all times. Its frame momenta are, line by line:

1. $k_{(1)} = q_1/f_1 = q_1\,e^{-a_4}e^{-Hy}$ (the factor of $x_1$ is
   $f_1 = e^{a_4}e^{Hy}$);
2. $k_{(5)} = q_5/f_5 = q_5\,e^{a_4}e^{-Hy}$ (the factor of $x_5$ is
   $f_5 = e^{-a_4}e^{Hy}$);
3. with frozen coefficients $E^2 = m^2 + k_{(1)}^2 - k_{(5)}^2
   = m^2 + q_1^2e^{-2Hy}e^{-2a_4} - q_5^2e^{-2Hy}e^{2a_4}$.

As $a_4$ grows, the last term grows like $e^{2a_4}$ and the middle one decays like
$e^{-2a_4}$: $E^2$ must turn negative. With $X = e^{2a_4}$ and $w = e^{-Hy}$ the
onset $E^2 = 0$ is the quadratic equation $q_5^2w^2X^2 - m^2X - q_1^2w^2 = 0$, whose
positive root is $X^\ast = \big(m^2 + \sqrt{m^4 + 4q_1^2q_5^2w^4}\big)/(2q_5^2w^2)$;
the onset time is $x_4^\ast = \ln(X^\ast)/(2AH)$. For $q_1 = 0$ this is
$x_4^\ast = \big(Hy + \ln(m/q_5)\big)/(AH)$.

THE MODEL (labelled). The field also depends on $y$, and the factor $e^{-Hy}$ in
the frame momenta makes the exact equation couple neighbouring values of $y$. The
*local-frame model* holds the hidden position fixed (it freezes the coefficients
in $y$, the WKB statement of the Revision record), works in the variables
$\chi = \sin^{1/2}z\,\Psi$ (in which the term $3H\gamma^{(0)}$ of the diagonal
frame is absent exactly), takes no momentum along the hidden direction, and keeps
the exact time dependence of the frame momenta: $i\,du/dx_4 = h(x_4)u$ with
$h(x_4) = -im\gamma^{(4)} - \gamma^{(4)}\big(k_{(1)}(x_4)\gamma^{(1)} +
k_{(5)}(x_4)\gamma^{(5)}\big)$. Its solutions are not exact solutions of the field
equation; they show how a wave behaves near one hidden position. The position
enters only through $q_5e^{-Hy}$, so the runs below use $y = 0$.

Units: $H = m = 1$, as in the Kohn-Sham record; times in units of $1/m$, momenta
in units of $m$.
```

Text cell:

```
## 5. The deflating history of the Kohn-Sham record

The next cell first defines two helpers for the checks that reproduce a Revision
record. `record_says(report, name, ...)` opens the report (a JSON file with a list
of checks, each with a name, a verdict and a detail text) and is true when the
check `name` is there with the verdict pass and its detail text contains every
```

further piece of text given. ‘check_record(condition, title, report, name, ...)‘ is the helper ‘check‘ for such a result: it passes only if the notebook’s own computation (‘condition‘) is right AND the record says the same. Then the cell reads the parameters of the Kohn-Sham record, takes from it H, m, the history constant A and the tip cutoff L, prints how the record describes the history and its status, and checks the values $A = H = m = 1$, $L = 3$.

In [2]:

```
import numpy as np  # arrays of numbers, matrices, linear algebra
import sympy as sp  # exact algebra with symbols

REPORTS = {}  # report file -> {check name: (verdict, detail)}, each read once


def record_says(report_file, check_name, *pieces):
    """True when the Revision report records the check check_name with the verdict
    pass and its detail text contains every given piece of text."""
    if report_file not in REPORTS:  # read the report the first time it is needed
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        REPORTS[report_file] = {entry["name"]: (entry["verdict"].lower(),
                                                entry["detail"])
                                for entry in data["checks"]}
    verdict, detail = REPORTS[report_file][check_name]
    return verdict == "pass" and all(piece in detail for piece in pieces)


def check_record(condition, title, report_file, check_name, *pieces):
    """check() for a result that reproduces the Revision check check_name: it passes
    only if condition is true AND the report records check_name as passed, with
    every piece of text (values computed here) in its detail."""
    on_record = record_says(report_file, check_name, *pieces)
    check(condition and on_record, title,
          record=f"{report_file}, check {check_name}")


parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
H = parameters["physics"]["H"]  # the author’s constant H
mass = parameters["physics"]["m"]  # the mass m
A = parameters["physics"]["historyA"]  # the history a4 = A H x4
L_tip = parameters["physics"]["L_tipCutoff"]  # the record cuts the tip at y = -L
say("history: " + parameters["theoryInputs"]["adiabaticityHistory"])
say("status: " + parameters["conventions"]["history"].split(". ")[0] + ".")
check(H == 1.0 and mass == 1.0 and A == 1.0 and L_tip == 3.0,
      "the canonical history: A = 1, H = 1, m = 1, tip cutoff L = 3",
      record="Revision/kohn_sham/results/parameters.json, physics.historyA, "
             "physics.H, physics.m, physics.L_tipCutoff")


def a4_of(x4):
    """The prescribed deflating history a4 = A H x4."""
    return A * H * x4
```

Out [2]:

```
history: a4 = A H x4 (A = 1 canonical), a4' = A H; slices a4,0 = a4(x4); PRESCRIBED
    BACKGROUND (see historyStatus)
status: PRESCRIBED BACKGROUND: the history a4 = A H x4 is prescribed, not solved for.
PASS the canonical history: A = 1, H = 1, m = 1, tip cutoff L = 3
     reproduces Revision/kohn_sham/results/parameters.json, physics.historyA, physics.H,
    physics.m, physics.L_tipCutoff
```

Text cell:

```
## 6. The scale factors and the constant volume

The next cell first checks three exact statements with sympy: $dy/dx_0 = \cot z$
for $y = \ln(\sin z)/(6H)$; $e^{Hy} = \sin^{1/6}z$; and the product of the eight
scale factors, $\cot z \cdot (e^{a_4}\sin^{1/6}z)^3 \cdot 1 \cdot
(e^{-a_4}b_4^{1/2}\sin^{1/6}z)^3$, equals $b_4^{3/2}\cos z$ for every $a_4$, and
$\cos z$ at $b_4 = 1$: the inflation of 3-space and the deflation of the extra
times compensate exactly. Then it draws the scale factors along the
history at two hidden positions.
```

In [3]:

```
x0, H_symbol = sp.symbols("x0 H", positive=True)
a4_symbol = sp.Symbol("a4", real=True)
z = 6 * H_symbol * x0
y_of_x0 = sp.log(sp.sin(z)) / (6 * H_symbol)  # the hidden coordinate y
check(sp.simplify(sp.diff(y_of_x0, x0) - sp.cot(z)) == 0, "dy/dx0 = cot z")
check(sp.simplify(sp.exp(H_symbol * y_of_x0) - sp.sin(z) ** sp.Rational(1, 6)) == 0,
      "e^(H y) = sin(z)^(1/6)")
space_factor = sp.exp(a4_symbol) * sp.sin(z) ** sp.Rational(1, 6)  # x1, x2, x3
b4_symbol = sp.Symbol("b4", positive=True)  # the second function of the metric
extra_factor = (sp.exp(-a4_symbol) * sp.sqrt(b4_symbol)
                * sp.sin(z) ** sp.Rational(1, 6))  # x5, x6, x7
volume = sp.cot(z) * space_factor**3 * 1 * extra_factor**3  # x4 has the factor 1
check_record(sp.simplify(volume - b4_symbol ** sp.Rational(3, 2) * sp.cos(z)) == 0
             and sp.simplify(volume.subs(b4_symbol, 1) - sp.cos(z)) == 0,
             "sqrt|g| = product of the scale factors = b4^(3/2) cos z, for every a4; "
             "cos z at b4 = 1",
             "Revision/theory/reports/python-field-theory.json",
             "sqrt_det_g_equals_cos_z", "cos(6 H x0)")

times = np.linspace(0.0, 6.0, 601)
fig, ax = plt.subplots()
for y, style in ((0.0, "-"), (-1.0, "--")):
    warp = np.exp(H * y)  # sin^(1/6) z = e^(H y)
    ax.plot(times, np.exp(a4_of(times)) * warp, style, color="tab:blue",
            label=f"3-space $e^{{a_4}}e^{{Hy}}$, $y = {y}$")
    ax.plot(times, np.exp(-a4_of(times)) * warp, style, color="tab:red",
            label=f"extra times $e^{{-a_4}}e^{{Hy}}$, $y = {y}$")
    ax.plot(times, np.full_like(times, warp**6), style, color="black",
```

```
            linewidth=0.8, label=f"product of the six, $e^{{6Hy}}$, $y = {y}$")
ax.set_yscale("log")
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("scale factor")
ax.set_title("Scale factors along the history $a_4 = x_4$")
ax.legend(fontsize=7, loc="upper left");
save_figure(fig, "scale_factors",
            "The scale factors of the author's metric along the prescribed "
            "deflating history $a_4 = A H x_4$ ($A = H = 1$) against the time $x_4$ "
            "in units of $1/m$, on a logarithmic vertical axis, at the patch end "
            "$y = 0$ (solid) and at $y = -1$ (dashed). Blue: 3-space, "
            "$e^{a_4}e^{Hy}$, a rising straight line (exponential inflation). Red: "
            "the extra times, $e^{-a_4}e^{Hy}$, a falling straight line "
            "(exponential deflation). Black: the product of the three spatial and "
            "the three extra-time factors, $e^{6Hy} = \\sin z$, which does not "
            "change in time.")
```

Out [3]:

```
PASS dy/dx0 = cot z
PASS e^(H y) = sin(z)^(1/6)
PASS sqrt|g| = product of the scale factors = b4^(3/2) cos z, for every a4; cos z at b4 =
    1
     reproduces Revision/theory/reports/python-field-theory.json, check
    sqrt_det_g_equals_cos_z
```

Out [3]:

```
Figure 08b.1 saved as Revision/textbook/figures/08b_1_scale_factors.png
```

Text cell:

```
## 7. Frame momenta and the onset of growth

The next cell defines the frame momenta of section 4, the local $E^2$, and the
exact onset time; it checks the onset time against a numerical root search
(bisection: halve an interval in which $E^2$ changes sign 200 times) for several
waves, and checks the simple form $x_4^\ast = \ln(m/q_5)$ for $q_1 = 0$, $y = 0$.
```

In [4]:

```
def frame_momenta(x4, q1, q5, y):
    """(k_(1), k_(5)) of the wave exp(i (q1 x1 + q5 x5)) at time x4 and position y."""
    w = np.exp(-H * y)  # e^(-H y) = sin^(-1/6) z
    return q1 * np.exp(-a4_of(x4)) * w, q5 * np.exp(a4_of(x4)) * w


def local_E2(x4, q1, q5, y):
    """E^2 = m^2 + k_(1)^2 - k_(5)^2 with frozen coefficients."""
```

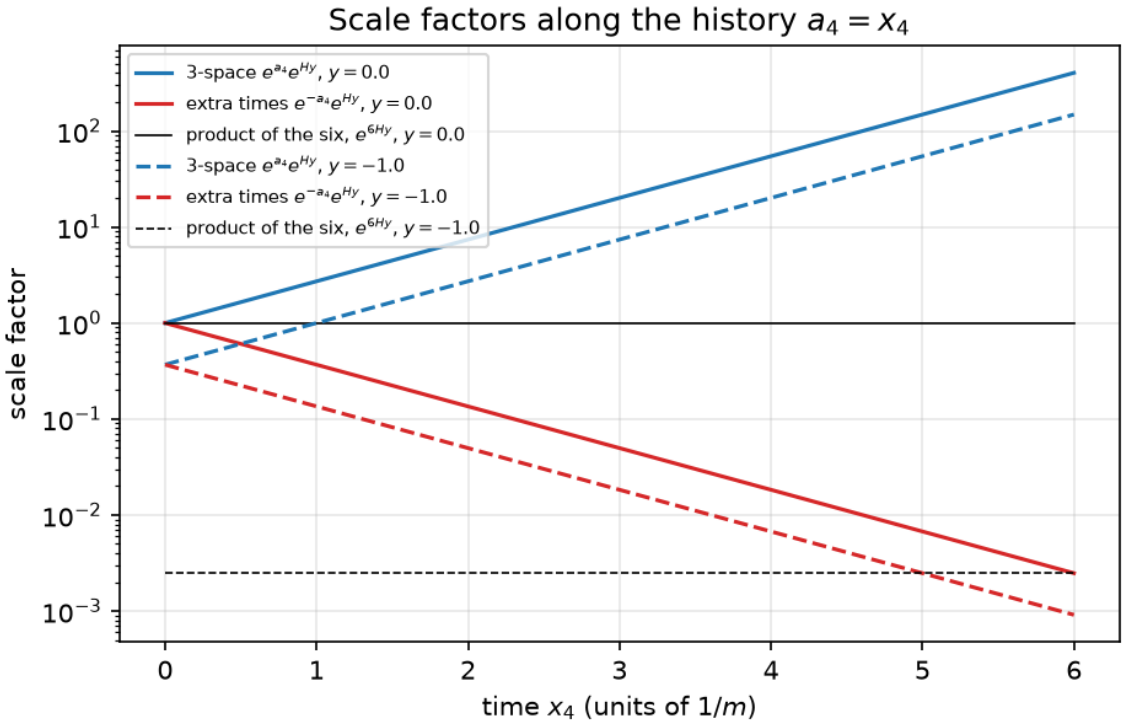

Figure 240: The scale factors of the author's metric along the prescribed deflating history $a_4 = AHx_4$ ($A = H = 1$) against the time x_4 in units of $1/m$, on a logarithmic vertical axis, at the patch end $y = 0$ (solid) and at $y = -1$ (dashed). Blue: 3-space, $e^{a_4}e^{Hy}$, a rising straight line (exponential inflation). Red: the extra times, $e^{-a_4}e^{Hy}$, a falling straight line (exponential deflation). Black: the product of the three spatial and the three extra-time factors, $e^{6Hy} = \sin z$, which does not change in time. (Notebook 08b, figure 1.)

```
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return mass**2 + k1**2 - k5**2


def onset_time(q1, q5, y):
    """The exact time at which E^2 = 0 (positive root of the quadratic in X)."""
    w = np.exp(-H * y)
    X = (mass**2 + np.sqrt(mass**4 + 4 * q1**2 * q5**2 * w**4)) / (2 * q5**2 * w**2)
    return np.log(X) / (2 * A * H)


def bisection(q1, q5, y, low=-30.0, high=30.0):
    """The time at which E^2 changes sign, by halving [low, high] 200 times."""
    for _ in range(200):
        middle = (low + high) / 2
        if local_E2(middle, q1, q5, y) > 0:  # still oscillating: onset is later
            low = middle
        else:
            high = middle
    return (low + high) / 2


worst = max(abs(onset_time(q1, q5, y) - bisection(q1, q5, y))
            for q1 in (0.0, 0.5, 1.0, 3.0) for q5 in (0.01, 0.05, 0.1, 0.4)
            for y in (0.0, -1.0, -3.0))
check(worst < 1e-12, "the exact onset time equals the bisection result (48 waves)")
check(abs(onset_time(0.0, 0.05, 0.0) - np.log(1 / 0.05)) < 1e-14,
      "q1 = 0, y = 0: onset time = ln(m/q5)")
for q5 in (0.05, 0.1):
    report(f"onset time for q1 = 0, q5 = {q5}, y = 0", f"{onset_time(0.0, q5, 0.0):.6f}")
```

Out [4]:

```
PASS the exact onset time equals the bisection result (48 waves)
PASS q1 = 0, y = 0: onset time = ln(m/q5)
RESULT onset time for q1 = 0, q5 = 0.05, y = 0 = 2.995732
RESULT onset time for q1 = 0, q5 = 0.1, y = 0 = 2.302585
```

Text cell:

```
The next cell draws the two frame momenta of a wave with $q_1 = q_5 = 0.1$ (left)
and the local $E^2$ of several waves (right) along the history; the dots mark the
onset times computed above.
```

In [5]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
k1_path, k5_path = frame_momenta(times, 0.1, 0.1, 0.0)
left.plot(times, k5_path, color="tab:red", label="extra time: $k_{(5)} = q_5e^{a_4}$")
left.plot(times, k1_path, color="tab:blue", label="3-space: $k_{(1)} = q_1e^{-a_4}$")
left.axhline(mass, color="black", linewidth=0.8, linestyle=":", label="$m$")
```

```
left.set_yscale("log")
left.set_xlabel("time $x_4$")
left.set_ylabel("frame momentum (units of $m$)")
left.set_title("$q_1 = q_5 = 0.1$, $y = 0$")
left.legend(fontsize=8)
for q1, q5, style in ((0.0, 0.02, "-"), (0.0, 0.05, "-"), (0.0, 0.1, "-"),
                      (0.0, 0.2, "-"), (1.0, 0.05, "--")):
    right.plot(times, local_E2(times, q1, q5, 0.0), style,
               label=f"$q_1 = {q1}$, $q_5 = {q5}$")
    t_star = onset_time(q1, q5, 0.0)
    right.plot([t_star], [0.0], "o", color="black", markersize=4)
right.axhline(0.0, color="black", linewidth=0.8)
right.set_ylim(-4.0, 2.5)
right.set_xlabel("time $x_4$")
right.set_ylabel("$E^2 = m^2 + k_{(1)}^2 - k_{(5)}^2$")
right.set_title("Local $E^2$ along the history, $y = 0$")
right.legend(fontsize=8, loc="lower left");
save_figure(fig, "frame_momenta",
            "Left: the frame momenta of one wave with coordinate wave numbers "
            "$q_1 = q_5 = 0.1$ at $y = 0$ against the time $x_4$, on a logarithmic "
            "axis in units of $m$: the extra-time momentum $k_{(5)} = q_5e^{a_4}$ "
            "grows exponentially because the extra times deflate, the space "
            "momentum $k_{(1)} = q_1e^{-a_4}$ shrinks because 3-space inflates; the "
            "dotted line is the mass $m = 1$. Right: the local $E^2$ of five waves "
            "along the history; each curve crosses zero once, at its onset time "
            "(black dot), and stays negative afterwards: the wave stops oscillating "
            "and starts to grow. The dashed wave has also $q_1 = 1$: its $E^2$ "
            "starts higher, but the space term dies away like $e^{-2a_4}$ and its "
            "onset is almost that of the same wave with $q_1 = 0$.")
```

Out [5]:

```
Figure 08b.2 saved as Revision/textbook/figures/08b_2_frame_momenta.png
```

Text cell:

```
## 8. Every wave along an extra time reaches the onset

The onset time $x_4^\ast$ is finite for EVERY $q_5 > 0$: the formula of section 4
has $q_5$ only in a denominator inside a logarithm, so a smaller $q_5$ only delays
the onset logarithmically, and a deeper position (more negative $y$) or a nonzero
$q_1$ changes it by a bounded amount. The next cell evaluates the onset time on a
grid of $q_5$ from $10^{-4}$ to $1$, checks that it is finite and decreasing in
$q_5$, and draws it for three hidden positions and for $q_1 = 1$.
```

In [6]:

```
q5_grid = np.logspace(-4.0, 0.0, 200)  # 10^-4 ... 1, equally spaced on a log axis
fig, ax = plt.subplots()
all_finite_and_decreasing = True
for q1, y, style in ((0.0, 0.0, "-"), (0.0, -1.0, "--"), (0.0, -L_tip, ":"),
```

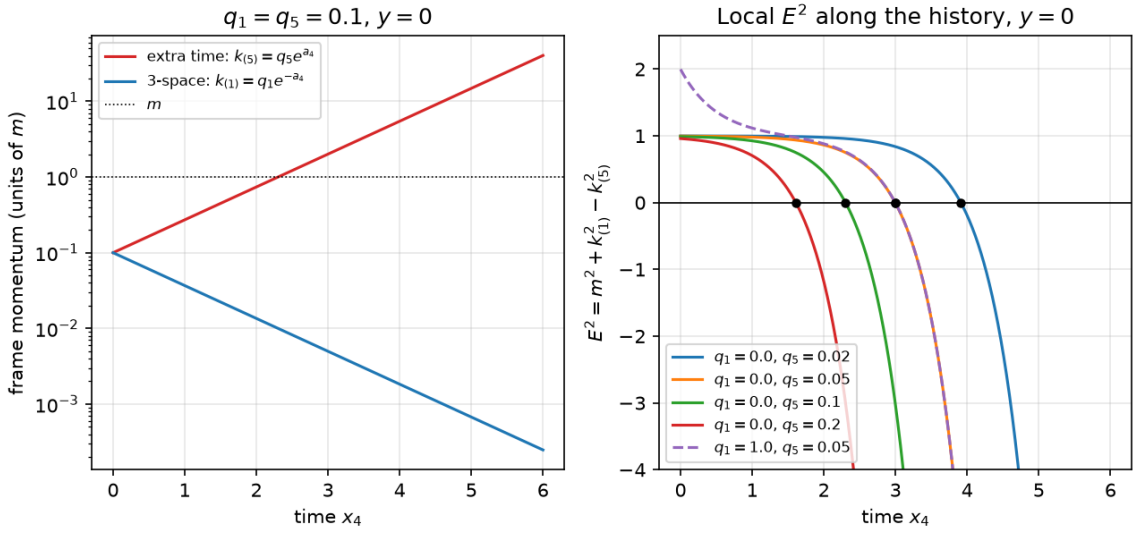

Figure 241: Left: the frame momenta of one wave with coordinate wave numbers $q_1 = q_5 = 0.1$ at $y = 0$ against the time x_4, on a logarithmic axis in units of m: the extra-time momentum $k_{(5)} = q_5 e^{a_4}$ grows exponentially because the extra times deflate, the space momentum $k_{(1)} = q_1 e^{-a_4}$ shrinks because 3-space inflates; the dotted line is the mass $m = 1$. Right: the local E^2 of five waves along the history; each curve crosses zero once, at its onset time (black dot), and stays negative afterwards: the wave stops oscillating and starts to grow. The dashed wave has also $q_1 = 1$: its E^2 starts higher, but the space term dies away like e^{-2a_4} and its onset is almost that of the same wave with $q_1 = 0$. (Notebook 08b, figure 2.)

```
                    (1.0, 0.0, "-.")):
    onsets = onset_time(q1, q5_grid, y)
    all_finite_and_decreasing &= bool(np.all(np.isfinite(onsets))
                                      and np.all(np.diff(onsets) < 0))
    ax.plot(q5_grid, onsets, style, label=f"$q_1 = {q1}$, $y = {y}$")
ax.set_xscale("log")
ax.set_xlabel("coordinate wave number $q_5$ along the extra time $x_5$")
ax.set_ylabel("onset time $x_4^\\ast$ (units of $1/m$)")
ax.set_title("Every extra-time wave reaches the onset")
ax.legend();
save_figure(fig, "onset_times",
            "The onset time $x_4^\\ast$ at which a wave with coordinate wave number "
            "$q_5$ along the extra time $x_5$ stops oscillating and starts to grow, "
            "against $q_5$ from $10^{-4}$ to $1$ on a logarithmic axis, along the "
            "history $a_4 = x_4$ with $m = H = 1$, at the hidden positions $y = 0$ "
            "(the patch end), $y = -1$ and $y = -3$ (the tip cutoff of the "
            "Kohn-Sham record), and for a wave that also has $q_1 = 1$. Every curve "
            "is finite and rises only like $\\ln(1/q_5)$: however small $q_5$ is, "
            "the deflation of the extra times eventually drives the wave into "
            "growth; a negative onset time means that the wave grows already at "
            "$x_4 = 0$.")
check_record(all_finite_and_decreasing,
             "the onset time is finite for every q5 > 0 on the grid and decreases "
             "with q5", "Revision/theory/reports/python-field-theory.json",
             "extra_time_modes_grow",
             "every extra-time mode eventually enters the growing regime")
```

Out [6]:

```
Figure 08b.3 saved as Revision/textbook/figures/08b_3_onset_times.png
PASS the onset time is finite for every q5 > 0 on the grid and decreases with q5
     reproduces Revision/theory/reports/python-field-theory.json, check
    extra_time_modes_grow
```

Text cell:

```
## 9. Following one wave through the onset: the local-frame model

The next cell reads the gamma matrices and builds the time-dependent mode matrix
$h(x_4)$ of the local-frame model. It checks at five instants, for a wave with
$q_1 = 0.3$ and $q_5 = 0.1$, that $h(x_4)^2 = E^2(x_4)\,I_{16}$, where $E^2$ is the
formula of the Revision record for frozen coefficients, $E^2 = m^2 + k_1^2 + k_2^2 +
k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2$, taken at the frame momenta of that instant
(and checks that the record holds exactly this formula). Then it chooses the
starting column for $q_1 = 0$.
With $q_1 = 0$ the matrix $C = \gamma^{(0)}\gamma^{(1)}\gamma^{(2)}\gamma^{(3)}$
commutes with $h$ ($\gamma^{(4)}$ and $\gamma^{(5)}$ each anticommute with all four
factors of $C$), so there are columns that are eigenvectors of $h(0)$ for the
positive frequency $E_0 = \sqrt{m^2 - q_5^2}$ AND of $C$ for $+1$ or $-1$. The
projectors $\Lambda_+ = (1 + h(0)/E_0)/2$ and $(1 \pm C)/2$ produce them from the
first unit column. For such a column the Krein form is $u^\dagger B u = \pm E_0/m$:
$B = -iC\gamma^{(4)} = C(-i\gamma^{(4)})$ gives $u^\dagger B u = \pm u^\dagger
```

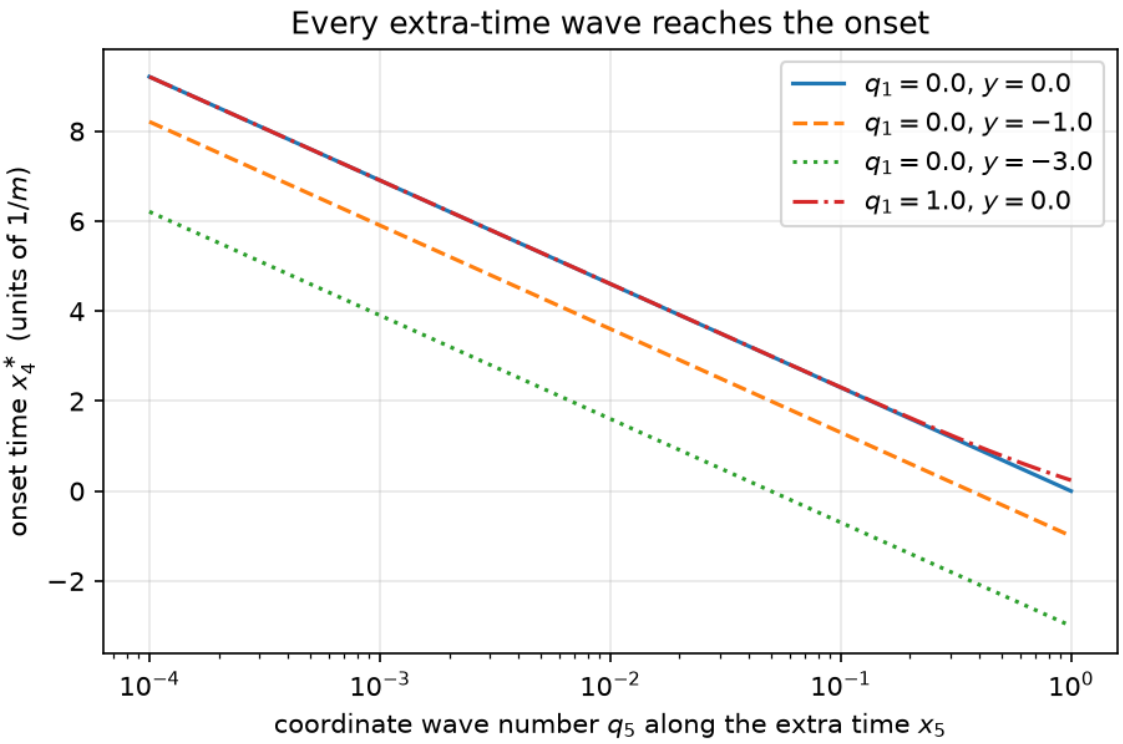

Figure 242: The onset time x_4^* at which a wave with coordinate wave number q_5 along the extra time x_5 stops oscillating and starts to grow, against q_5 from 10^{-4} to 1 on a logarithmic axis, along the history $a_4 = x_4$ with $m = H = 1$, at the hidden positions $y = 0$ (the patch end), $y = -1$ and $y = -3$ (the tip cutoff of the Kohn-Sham record), and for a wave that also has $q_1 = 1$. Every curve is finite and rises only like $\ln(1/q_5)$: however small q_5 is, the deflation of the extra times eventually drives the wave into growth; a negative onset time means that the wave grows already at $x_4 = 0$. (Notebook 08b, figure 3.)

```
(-i\gamma^{(4)})u$, and $E_0 = u^\dagger h u = m\,u^\dagger(-i\gamma^{(4)})u -
q_5\,u^\dagger\gamma^{(4)}\gamma^{(5)}u$, where the first term is real and the
second imaginary ($\gamma^{(4)}\gamma^{(5)}$ is real and antisymmetric), so the
second term vanishes. The cell checks all of this.
```

In [7]:

```
gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
GAMMA = [np.array(rows, dtype=int) for rows in gammas_record["gamma"]]
C = np.array(gammas_record["C"], dtype=int)  # C = g^(x0) g^(x1) g^(x2) g^(x3)
B = np.array(gammas_record["B"]["re"]) + 1j * np.array(gammas_record["B"]["im"])
g4 = GAMMA[4].astype(complex)  # gamma^(x4)
g4g1 = (GAMMA[4] @ GAMMA[1]).astype(complex)  # gamma^(x4) gamma^(x1)
g4g5 = (GAMMA[4] @ GAMMA[5]).astype(complex)  # gamma^(x4) gamma^(x5)
check(np.array_equal(C @ GAMMA[4], GAMMA[4] @ C) and
      np.array_equal(C @ GAMMA[5], GAMMA[5] @ C),
      "C commutes with gamma^(x4) and gamma^(x5)")


def h_local(x4, q1, q5, y):
    """The mode matrix of the local-frame model at time x4."""
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return -1j * mass * g4 - k1 * g4g1 - k5 * g4g5


# At every instant h(x4)^2 = E^2(x4) I16, where E^2 is the frozen-coefficient E^2 of
# the record with k1 = k_(1)(x4), k5 = k_(5)(x4) and all other momenta 0.
m_s = sp.Symbol("m", real=True)
k_s = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}
E2_record = m_s**2 + sum(k_s[a] ** 2 for a in (0, 1, 2, 3)) \
    - sum(k_s[a] ** 2 for a in (5, 6, 7))
squares_ok = True
for x4 in (0.0, 1.0, 2.0, 3.0, 4.0):
    k1_now, k5_now = frame_momenta(x4, 0.3, 0.1, 0.0)  # q1 = 0.3, q5 = 0.1, y = 0
    values = {symbol: 0 for symbol in k_s.values()}  # every momentum 0 ...
    values.update({m_s: mass, k_s[1]: k1_now, k_s[5]: k5_now})  # ... but these
    E2_now = float(E2_record.subs(values))
    h_now = h_local(x4, 0.3, 0.1, 0.0)
    squares_ok &= bool(np.allclose(h_now @ h_now, E2_now * np.eye(16), atol=1e-12)
                       and abs(E2_now - local_E2(x4, 0.3, 0.1, 0.0)) < 1e-12)
check_record(squares_ok, "x4 = 0, 1, 2, 3, 4: h(x4)^2 = E^2(x4) I16 (q1 = 0.3, "
             "q5 = 0.1)", "Revision/theory/reports/python-scope.json",
             "extra_time_growth_rates_unbounded", f"h_k^2 = ({sp.sstr(E2_record)}) I16")


def starting_column(q5, sign):
    """A unit column with h(0) u = E0 u and C u = sign u (q1 = 0, y = 0)."""
    h0 = h_local(0.0, 0.0, q5, 0.0)
    E0 = np.sqrt(mass**2 - q5**2)
    first = np.zeros(16, dtype=complex)
    first[0] = 1.0  # the first unit column
    u = (np.eye(16) + h0 / E0) @ ((np.eye(16) + sign * C) @ first) / 4
```

```
    return u / np.linalg.norm(u), h0, E0


starts = {}
for q5, sign in ((0.05, +1), (0.1, -1)):
    u0, h0, E0 = starting_column(q5, sign)
    starts[q5] = u0
    krein0 = np.vdot(u0, B @ u0)  # u^dagger B u
    report(f"q5 = {q5}: E0, Krein form of the start",
           f"{E0:.9f}, {krein0.real:+.9f}")
    sign_text = "+" if sign > 0 else "-"  # the sign as a character
    check(np.linalg.norm(h0 @ u0 - E0 * u0) < 1e-14 and
          np.linalg.norm(C @ u0 - sign * u0) < 1e-14 and
          abs(krein0 - sign * E0 / mass) < 1e-14,
          f"q5 = {q5}: h(0) u = E0 u, C u = {sign_text}u, "
          f"Krein form = {sign_text}E0/m")
```

Out [7]:

```
PASS C commutes with gamma^(x4) and gamma^(x5)
PASS x4 = 0, 1, 2, 3, 4: h(x4)^2 = E^2(x4) I16 (q1 = 0.3, q5 = 0.1)
     reproduces Revision/theory/reports/python-scope.json, check
    extra_time_growth_rates_unbounded
RESULT q5 = 0.05: E0, Krein form of the start = 0.998749218, +0.998749218
PASS q5 = 0.05: h(0) u = E0 u, C u = +u, Krein form = +E0/m
RESULT q5 = 0.1: E0, Krein form of the start = 0.994987437, -0.994987437
PASS q5 = 0.1: h(0) u = E0 u, C u = -u, Krein form = -E0/m
```

Text cell:

```
The next cell solves $du/dx_4 = -i\,h(x_4)u$ with RK4 from $x_4 = 0$ to three
time units after the onset, for $q_5 = 0.05$ and $q_5 = 0.1$, with 12000 steps,
and records $u^\dagger u$ and $u^\dagger B u$ after every step. It prints the size
$u^\dagger u$ at the onset (at the first step after it) and $\ln(u^\dagger u)$ at
the end. Before the onset the size does not stay exactly 1: with extra-time
momentum the mode matrix is not Hermitian, so the Hilbert norm is not conserved.
For $q_5 = 0.05$ it repeats the run with 6000 and 24000 steps: the differences of
the end values must shrink by a factor close to 16 (fourth order).
```

In [8]:

```
def rk4_history(u0, q5, x4_end, steps):
    """RK4 for du/dx4 = -i h(x4) u (q1 = 0, y = 0); returns the times, u^dagger u,
    u^dagger B u after every step, and the final column."""
    step = x4_end / steps
    u = u0.copy()
    times_out = [0.0]
    hilbert = [np.vdot(u, u).real]
    krein = [np.vdot(u, B @ u)]

    def rate(x4, v):  # the right-hand side -i h(x4) v
```

```
        return -1j * (h_local(x4, 0.0, q5, 0.0) @ v)

    for n in range(steps):
        x4 = n * step
        s1 = rate(x4, u)
        s2 = rate(x4 + step / 2, u + step / 2 * s1)
        s3 = rate(x4 + step / 2, u + step / 2 * s2)
        s4 = rate(x4 + step, u + step * s3)
        u = u + step / 6 * (s1 + 2 * s2 + 2 * s3 + s4)
        times_out.append((n + 1) * step)
        hilbert.append(np.vdot(u, u).real)
        krein.append(np.vdot(u, B @ u))
    return np.array(times_out), np.array(hilbert), np.array(krein), u


runs = {}
for q5 in (0.05, 0.1):
    x4_end = onset_time(0.0, q5, 0.0) + 3.0
    runs[q5] = rk4_history(starts[q5], q5, x4_end, 12000)
    at_onset = np.searchsorted(runs[q5][0], onset_time(0.0, q5, 0.0))  # its index
    report(f"q5 = {q5}: u^dagger u at the onset", f"{runs[q5][1][at_onset]:.2f}")
    final_size = runs[q5][1][-1]  # u^dagger u at the end of the run
    report(f"q5 = {q5}: ln(u^dagger u) at the onset + 3", f"{np.log(final_size):.6f}")
x4_end = onset_time(0.0, 0.05, 0.0) + 3.0
end_6000 = rk4_history(starts[0.05], 0.05, x4_end, 6000)[3]
end_24000 = rk4_history(starts[0.05], 0.05, x4_end, 24000)[3]
end_12000 = runs[0.05][3]
ratio = np.linalg.norm(end_6000 - end_12000) / np.linalg.norm(end_12000 - end_24000)
relative_error = np.linalg.norm(end_12000 - end_24000) / np.linalg.norm(end_24000)
report("RK4 error ratio (6000 vs 12000) / (12000 vs 24000) steps", f"{ratio:.2f}")
check(14.0 < ratio < 18.0, "RK4 converges with order 4 (error ratio near 16)")
check(relative_error < 1e-8, "the 12000-step run is accurate to 1e-8 (relative)")
```

Out [8]:

```
RESULT q5 = 0.05: u^dagger u at the onset = 1.52
RESULT q5 = 0.05: ln(u^dagger u) at the onset + 3 = 37.071245
RESULT q5 = 0.1: u^dagger u at the onset = 1.54
RESULT q5 = 0.1: ln(u^dagger u) at the onset + 3 = 37.113916
RESULT RK4 error ratio (6000 vs 12000) / (12000 vs 24000) steps = 15.89
PASS RK4 converges with order 4 (error ratio near 16)
PASS the 12000-step run is accurate to 1e-8 (relative)
```

Text cell:

```
## 10. The WKB formulas and the comparison

After the onset, write $Q = k_{(5)} = q_5e^{AHx_4}$ and
$\kappa = \sqrt{Q^2 - m^2}$. Line by line:

1. Leading order: at each instant the growing eigenvalue of $-ih$ is $+\kappa$, so
   $\frac{d}{dx_4}\ln(u^\dagger u) \approx 2\kappa$ and $\ln(u^\dagger u) \approx
```

```
   2W + $ const with $W(x_4) = \int\kappa\,dx_4$.
2. Because $dQ/dx_4 = AH\,Q$, the substitution $dx_4 = dQ/(AHQ)$ gives
   $W = \frac{1}{AH}\int\frac{\sqrt{Q^2 - m^2}}{Q}dQ = \frac{1}{AH}\Big(\sqrt{Q^2 -
   m^2} - m\arccos\frac{m}{Q}\Big)$ (differentiate the bracket: $\frac{Q}{\kappa} -
   \frac{m^2}{Q\kappa} = \frac{\kappa}{Q}$).
3. First order: the matrices $A_4 = -i\gamma^{(4)}$ and $M = -i\gamma^{(4)}
   \gamma^{(5)}$ are Hermitian, square to 1 and anticommute, and $h = mA_4 - iQM$.
   In each of 8 planes spanned by a column $w$ with $A_4w = w$ and by $Mw$ the
   equation is $v' = \begin{pmatrix}-im & -Q\\ -Q & im\end{pmatrix}v$.
4. This symmetric matrix has the eigenvector $r = (Q, -\kappa - im)$ for $+\kappa$.
   Multiplying the equation by $r^T$ and keeping only the growing part $v \approx
   c\,r$ gives $c'/c = \kappa - n'/(2n)$ with $n = r^Tr = 2\kappa(\kappa + im)$.
5. Hence $|c|^2 \propto e^{2W}/|n| = e^{2W}/(2\kappa Q)$, and $u^\dagger u \approx
   |c|^2\,r^\dagger r = |c|^2\,2Q^2 \propto e^{2W}Q/\kappa$: $\ln(u^\dagger u)
   \approx 2W + \ln(Q/\kappa) + $ const, and the rate is $2\kappa + \frac{d}{dx_4}
   \ln\frac{Q}{\kappa} = 2\kappa - AH\,m^2/\kappa^2$.

Both approximations fail at the onset itself ($\kappa \to 0$), so the comparison
uses increases over a window from 1.5 to 3 time units after the onset (the
unknown constants drop out of increases). The next cell computes them.
```

In [9]:

```
def Q_of(x4, q5):
    return q5 * np.exp(a4_of(x4))  # the frame momentum k_(5) at y = 0


def kappa_of(x4, q5):
    return np.sqrt(Q_of(x4, q5) ** 2 - mass**2)


def W_of(x4, q5):
    Q = Q_of(x4, q5)
    return (np.sqrt(Q**2 - mass**2) - mass * np.arccos(mass / Q)) / (A * H)


comparison = {}
for q5 in (0.05, 0.1):
    x4s, hilbert, krein, _ = runs[q5]
    start = np.searchsorted(x4s, onset_time(0.0, q5, 0.0) + 1.5)  # window start
    xa, xb = x4s[start], x4s[-1]
    increase = np.log(hilbert[-1]) - np.log(hilbert[start])
    leading = 2 * (W_of(xb, q5) - W_of(xa, q5))
    first = leading + np.log(Q_of(xb, q5) / kappa_of(xb, q5)) \
        - np.log(Q_of(xa, q5) / kappa_of(xa, q5))
    comparison[q5] = (xa, increase, leading, first)
    say(f"q5 = {q5}: window {xa:.4f} to {xb:.4f}; increase of ln(u^dagger u) "
        f"{increase:.4f}, leading WKB {leading:.4f} (relative difference "
        f"{abs(increase - leading) / increase:.2e}), first order {first:.4f} "
        f"(relative difference {abs(increase - first) / increase:.2e})")
    check(abs(increase - leading) / increase < 2e-3,
          f"q5 = {q5}: leading WKB agrees to 2e-3 over the window")
    check(abs(increase - first) < abs(increase - leading) and
```

```
          abs(increase - first) / increase < 5e-4,
          f"q5 = {q5}: first-order WKB is better and agrees to 5e-4")
x4s, hilbert, _, _ = runs[0.05]
step = x4s[1] - x4s[0]
late_rate = (np.log(hilbert[-1]) - np.log(hilbert[-3])) / (2 * step)  # centred
kappa_late = kappa_of(x4s[-2], 0.05)  # kappa at the middle of the centred difference
predicted = 2 * kappa_late - A * H * mass**2 / kappa_late**2
report("q5 = 0.05: computed late rate, predicted 2 kappa - m^2/kappa^2",
       f"{late_rate:.5f}, {predicted:.5f}")
check(abs(late_rate - predicted) / predicted < 1e-5,
      "the late growth rate equals 2 kappa - A H m^2/kappa^2 to 1e-5")
```

Out [9]:

```
q5 = 0.05: window 4.4958 to 5.9957; increase of ln(u^dagger u) 31.0009, leading WKB
    31.0328 (relative difference 1.03e-03), first order 31.0085 (relative difference
    2.48e-04)
PASS q5 = 0.05: leading WKB agrees to 2e-3 over the window
PASS q5 = 0.05: first-order WKB is better and agrees to 5e-4
q5 = 0.1: window 3.8028 to 5.3026; increase of ln(u^dagger u) 30.9993, leading WKB
    31.0312 (relative difference 1.03e-03), first order 31.0069 (relative difference
    2.48e-04)
PASS q5 = 0.1: leading WKB agrees to 2e-3 over the window
PASS q5 = 0.1: first-order WKB is better and agrees to 5e-4
RESULT q5 = 0.05: computed late rate, predicted 2 kappa - m^2/kappa^2 = 40.09851,
    40.09868
PASS the late growth rate equals 2 kappa - A H m^2/kappa^2 to 1e-5
```

Text cell:

```
The next cell draws $\ln(u^\dagger u)$ of both runs with the leading WKB curve $2W$
(shifted to agree at the window start), and the onset times.
```

In [10]:

```
fig, ax = plt.subplots()
for q5, colour in ((0.05, "tab:blue"), (0.1, "tab:orange")):
    x4s, hilbert, _, _ = runs[q5]
    xa, _, _, _ = comparison[q5]
    ax.plot(x4s, np.log(hilbert), color=colour, label=f"RK4, $q_5 = {q5}$")
    after = x4s >= onset_time(0.0, q5, 0.0) + 0.05
    shift = np.log(hilbert[np.searchsorted(x4s, xa)]) - 2 * W_of(xa, q5)
    ax.plot(x4s[after], 2 * W_of(x4s[after], q5) + shift, "--", color="black",
            linewidth=0.9)
    ax.axvline(onset_time(0.0, q5, 0.0), color=colour, linestyle=":", linewidth=0.9)
ax.plot([], [], "--", color="black", linewidth=0.9, label="WKB $2W$ + const")
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("$\\ln(u^\\dagger u)$")
ax.set_title("A wave along an extra time, $a_4 = x_4$, $y = 0$")
ax.legend();
save_figure(fig, "growth_along_history",
```

```
"The natural logarithm of the size $u^\\dagger u$ of a wave with "
"coordinate wave number $q_5 = 0.05$ (blue) and $q_5 = 0.1$ (orange) "
"along the extra time $x_5$, solved with RK4 in the local-frame model "
"along the prescribed deflating history $a_4 = x_4$ ($m = H = 1$, "
"$y = 0$), against the time $x_4$ in units of $1/m$. Before the onset "
"(dotted vertical lines, $x_4^\\ast = \\ln 20$ and $\\ln 10$) the wave "
"oscillates and its size grows only slowly, to about 1.5 at the onset "
"(the mode matrix is not Hermitian); after the onset it grows faster "
"and faster, by about 16 powers of ten in three time units, following "
"the WKB curve $2W$ (black dashed, matched 1.5 units after the onset).")
```

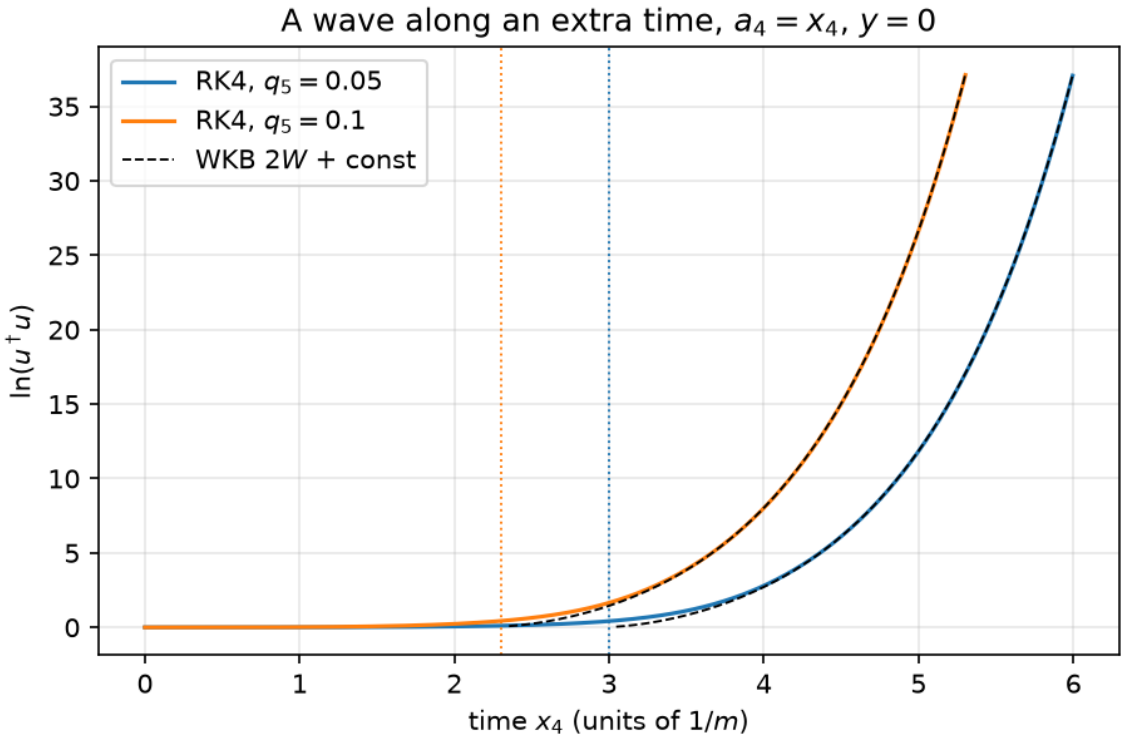

Figure 243: The natural logarithm of the size $u^\dagger u$ of a wave with coordinate wave number $q_5 = 0.05$ (blue) and $q_5 = 0.1$ (orange) along the extra time x_5, solved with RK4 in the local-frame model along the prescribed deflating history $a_4 = x_4$ ($m = H = 1$, $y = 0$), against the time x_4 in units of $1/m$. Before the onset (dotted vertical lines, $x_4^* = \ln 20$ and $\ln 10$) the wave oscillates and its size grows only slowly, to about 1.5 at the onset (the mode matrix is not Hermitian); after the onset it grows faster and faster, by about 16 powers of ten in three time units, following the WKB curve $2W$ (black dashed, matched 1.5 units after the onset). (Notebook 08b, figure 4.)

Out [10]:

```
Figure 08b.4 saved as Revision/textbook/figures/08b_4_growth_along_history.png
```

Text cell:

```
The last figure shows, for $q_5 = 0.05$, the growth RATE $\frac{d}{dx_4}
\ln(u^\dagger u)$ (computed from the RK4 values by centred differences) together
```

```
with the WKB rates $2\kappa$ and $2\kappa - m^2/\kappa^2$, and the change of the
Krein form divided by $\max(u^\dagger u, 1)$. The cell checks that this change
stays below $10^{-9}$.
```

In [11]:

```
x4s, hilbert, krein, _ = runs[0.05]
step = x4s[1] - x4s[0]
computed_rate = (np.log(hilbert[2:]) - np.log(hilbert[:-2])) / (2 * step)
middle = x4s[1:-1]
after = middle > onset_time(0.0, 0.05, 0.0) + 0.02
drift = np.abs(krein - krein[0]) / np.maximum(hilbert, 1.0)
check(drift.max() < 1e-9, "the Krein form is conserved (drift below 1e-9)")
check(hilbert[-1] > 1e15, "the Hilbert norm grows by more than 10^15")

fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
fig.subplots_adjust(wspace=0.35)  # room for the label of the right panel
left.plot(middle, computed_rate, color="tab:blue", label="RK4 (centred differences)")
left.plot(middle[after], 2 * kappa_of(middle[after], 0.05), "--", color="black",
          label="$2\\kappa$")
left.plot(middle[after], 2 * kappa_of(middle[after], 0.05)
          - mass**2 / kappa_of(middle[after], 0.05) ** 2, ":", color="tab:red",
          label="$2\\kappa - m^2/\\kappa^2$")
left.set_ylim(-5.0, 45.0)
left.set_xlabel("time $x_4$")
left.set_ylabel("growth rate $d\\ln(u^\\dagger u)/dx_4$")
left.set_title("Growth rate, $q_5 = 0.05$")
left.legend(fontsize=8)
right.semilogy(x4s[1:], np.maximum(drift[1:], 1e-18), color="tab:green")
right.set_xlabel("time $x_4$")
right.set_ylabel("$|\\Delta(u^\\dagger Bu)| / \\max(u^\\dagger u, 1)$")
right.set_title("Krein form: change (rounding only)")
save_figure(fig, "rate_and_krein",
            "Left: the growth rate $d\\ln(u^\\dagger u)/dx_4$ of the wave with "
            "$q_5 = 0.05$ against the time $x_4$ (units of $1/m$ and $m$), computed "
            "from the RK4 solution (blue), with the leading WKB rate $2\\kappa$ "
            "(black dashed) and the first-order rate $2\\kappa - m^2/\\kappa^2$ "
            "(red dotted); before the onset the rate stays below 1, after it the "
            "rate itself grows exponentially, so the growth is faster than any "
            "exponential, and far from the onset the first-order rate is accurate. "
            "Right: the change of the Krein form $u^\\dagger Bu$ divided by "
            "$\\max(u^\\dagger u, 1)$, on a logarithmic axis (exact zeros are drawn "
            "at $10^{-18}$); it stays at the level of rounding errors, far below "
            "$10^{-9}$.")
```

Out [11]:

```
PASS the Krein form is conserved (drift below 1e-9)
PASS the Hilbert norm grows by more than 10^15
```

Out [11]:

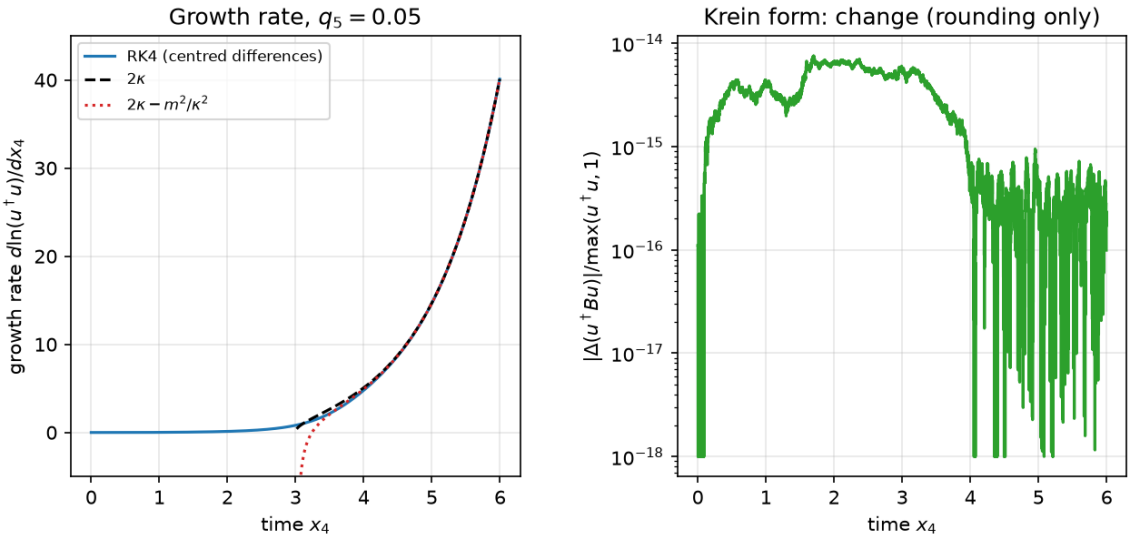

Figure 244: Left: the growth rate $d\ln(u^\dagger u)/dx_4$ of the wave with $q_5 = 0.05$ against the time x_4 (units of $1/m$ and m), computed from the RK4 solution (blue), with the leading WKB rate 2κ (black dashed) and the first-order rate $2\kappa - m^2/\kappa^2$ (red dotted); before the onset the rate stays below 1, after it the rate itself grows exponentially, so the growth is faster than any exponential, and far from the onset the first-order rate is accurate. Right: the change of the Krein form $u^\dagger Bu$ divided by $\max(u^\dagger u, 1)$, on a logarithmic axis (exact zeros are drawn at 10^{-18}); it stays at the level of rounding errors, far below 10^{-9}. (Notebook 08b, figure 5.)

```
Figure 08b.5 saved as Revision/textbook/figures/08b_5_rate_and_krein.png
```

Text cell:

```
## 11. The last check

The last cell checks that the five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [12]:

```
for name in ("08b_1_scale_factors.png", "08b_2_frame_momenta.png",
             "08b_3_onset_times.png", "08b_4_growth_along_history.png",
             "08b_5_rate_and_krein.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [12]:

```
PASS the figure file 08b_1_scale_factors.png exists
PASS the figure file 08b_2_frame_momenta.png exists
```

```
PASS the figure file 08b_3_onset_times.png exists
PASS the figure file 08b_4_growth_along_history.png exists
PASS the figure file 08b_5_rate_and_krein.png exists
ALL 25 CHECKS PASSED (notebook 08b)
```

Text cell:

```
## 12. What this notebook showed

- Along the prescribed deflating history $a_4 = AHx_4$ of the Kohn-Sham record
  ($A = H = m = 1$) 3-space inflates like $e^{a_4}$ and the extra times deflate like
  $e^{-a_4}$, while the volume factor $\sqrt{|g|} = \cos z$ (at $b_4 = 1$; $b_4^{3/2}
  \cos z$ in general) stays constant (PROVED,
  exact sympy; Revision check ‘sqrt_det_g_equals_cos_z‘).
- A wave $e^{i(q_1x_1 + q_5x_5)}$ keeps its coordinate wave numbers; its frame
  momentum along the extra time grows like $q_5e^{a_4}e^{-Hy}$ and along 3-space it
  shrinks like $q_1e^{-a_4}e^{-Hy}$ (PROVED).
- With frozen coefficients its $E^2$ turns negative at the onset time
  $x_4^\ast = \ln(X^\ast)/(2AH)$, finite for every $q_5 > 0$: every wave along an
  extra time eventually enters the growing regime (PROVED for the frozen
  coefficients; the Revision statement ‘extra_time_modes_grow‘ is this local-frame
  statement).
- In the local-frame model (MODEL: hidden position frozen; not an exact solution of
  the field equation) a wave started with positive frequency oscillates until the
  onset, while its size grows only slowly (to about 1.5 at the onset, because with
  extra-time momentum the mode matrix is not Hermitian), and then grows faster than
  any exponential, by about $10^{16}$ in three time units; the growth follows the
  WKB formulas (COMPUTED: leading order to about $10^{-3}$, first order to about
  $2.5 \times 10^{-4}$ over the window, late rate to $10^{-5}$), and the Krein form
  is conserved to rounding.
- Consequence (with Notebook 08a): in flat 4 + 4 space and with frozen
  coefficients the initial-value problem is not well posed for data that depend on
  the extra times, and the deflation makes every such wave reach the growing
  regime. Only the good sector (no dependence on $x_5, x_6, x_7$) is free of this
  growth; restricting to it is a choice, not a result.
```

8.29 Line-by-line walk-through of Notebook 08b

The notebook has 12 code cells, In [1] to In [12]. The conventions are those of Section 8.15; captions shortened to ... are printed in full in Section 8.28.

In [1], the set-up cell. It is the set-up cell explained in Section 8.15, with the line `NOTEBOOK_ID = "08b" # this notebook: chapter 08, example b`; its comment lines are the run instructions of Section 8.27. Output: `Set-up of notebook 08b complete: repository folder found, helpers defined.`

In [2], the record helpers and the history of the Kohn-Sham record. The cell starts with the imports of numpy and sympy, the empty table `REPORTS` and the two helpers `record_says` and `check_record`, word for word those of Notebook 08a (Section 8.23; `check_record` takes one check name). Then:

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
H = parameters["physics"]["H"]  # the author's constant H
mass = parameters["physics"]["m"]  # the mass m
A = parameters["physics"]["historyA"]  # the history a4 = A H x4
L_tip = parameters["physics"]["L_tipCutoff"]  # the record cuts the tip at y = -L
```

The parameters of the Kohn-Sham record are read; the four numbers are taken from its part `physics` (a dictionary inside the dictionary; two square brackets reach into it).

```
say("history: " + parameters["theoryInputs"]["adiabaticityHistory"])
say("status: " + parameters["conventions"]["history"].split(". ")[0] + ".")
check(H == 1.0 and mass == 1.0 and A == 1.0 and L_tip == 3.0,
      "the canonical history: A = 1, H = 1, m = 1, tip cutoff L = 3",
      record="Revision/kohn_sham/results/parameters.json, physics.historyA, "
             "physics.H, physics.m, physics.L_tipCutoff")
```

The first line prints the record's description of the history: `a4 = A H x4 (A = 1 canonical), a4' = A H; slices a4,0 = a4(x4); PRESCRIBED BACKGROUND (see historyStatus)`. The second prints the first sentence of the record's statement of its status (`split(".  ")` cuts the text at every full stop followed by a blank, `[0]` takes the first piece, and the full stop is added back): `PRESCRIBED BACKGROUND: the history a4 = A H x4 is prescribed, not solved for.` The check confirms the four values and names the record entries it reproduces.

```
def a4_of(x4):
    """The prescribed deflating history a4 = A H x4."""
    return A * H * x4
```

`a4_of` returns $a_4 = AHx_4$; it works for a single time and for a whole numpy array of times. Output of the cell: the two lines and one PASS line with its `reproduces` line.

In [3], the scale factors and the constant volume.

```
x0, H_symbol = sp.symbols("x0 H", positive=True)
a4_symbol = sp.Symbol("a4", real=True)
z = 6 * H_symbol * x0
y_of_x0 = sp.log(sp.sin(z)) / (6 * H_symbol)  # the hidden coordinate y
check(sp.simplify(sp.diff(y_of_x0, x0) - sp.cot(z)) == 0, "dy/dx0 = cot z")
check(sp.simplify(sp.exp(H_symbol * y_of_x0) - sp.sin(z) ** sp.Rational(1, 6)) == 0,
      "e^(H y) = sin(z)^(1/6)")
```

Exact symbols for x_0, H and a_4 (here a plain number-like symbol, because only its value enters). `y_of_x0` is $y = \ln(\sin z)/(6H)$. The two checks are the two facts of Section 8.24: $dy/dx_0 = \cot z$ and $e^{Hy} = \sin^{1/6} z$.

```
space_factor = sp.exp(a4_symbol) * sp.sin(z) ** sp.Rational(1, 6)  # x1, x2, x3
b4_symbol = sp.Symbol("b4", positive=True)  # the second function of the metric
extra_factor = (sp.exp(-a4_symbol) * sp.sqrt(b4_symbol)
```

```
                * sp.sin(z) ** sp.Rational(1, 6))  # x5, x6, x7
volume = sp.cot(z) * space_factor**3 * 1 * extra_factor**3  # x4 has the factor 1
check_record(sp.simplify(volume - b4_symbol ** sp.Rational(3, 2) * sp.cos(z)) == 0
             and sp.simplify(volume.subs(b4_symbol, 1) - sp.cos(z)) == 0,
             "sqrt|g| = product of the scale factors = b4^(3/2) cos z, for every a4; "
             "cos z at b4 = 1",
             "Revision/theory/reports/python-field-theory.json",
             "sqrt_det_g_equals_cos_z", "cos(6 H x0)")
```

The product of the eight scale factors is compared with $b_4^{3/2}\cos z$ for a symbolic a_4 and a symbol `b4_symbol` for b_4, and at $b_4 = 1$ with $\cos z$, together with the record's check, whose detail contains `cos(6 H x0)`.

```
times = np.linspace(0.0, 6.0, 601)
fig, ax = plt.subplots()
for y, style in ((0.0, "-"), (-1.0, "--")):
    warp = np.exp(H * y)  # sin^(1/6) z = e^(H y)
    ax.plot(times, np.exp(a4_of(times)) * warp, style, color="tab:blue",
            label=f"3-space $e^{{a_4}}e^{{Hy}}$, $y = {y}$")
    ax.plot(times, np.exp(-a4_of(times)) * warp, style, color="tab:red",
            label=f"extra times $e^{{-a_4}}e^{{Hy}}$, $y = {y}$")
    ax.plot(times, np.full_like(times, warp**6), style, color="black",
            linewidth=0.8, label=f"product of the six, $e^{{6Hy}}$, $y = {y}$")
```

601 times from 0 to 6. For the two hidden positions $y = 0$ (solid lines) and $y = -1$ (dashed), the cell draws the 3-space factor $e^{a_4}e^{Hy}$ (blue), the extra-time factor $e^{-a_4}e^{Hy}$ (red) and the product of the six, e^{6Hy} (black; `np.full_like(times, value)` is an array of the same length filled with the value). In an f-string a curly bracket that must appear in the text is written twice, so `e^{{a_4}}` prints as `e^{a_4}`.

```
ax.set_yscale("log")
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("scale factor")
ax.set_title("Scale factors along the history $a_4 = x_4$")
ax.legend(fontsize=7, loc="upper left");
save_figure(fig, "scale_factors",
            "The scale factors of the author's metric along the prescribed "
            ...
            "change in time.")
```

A logarithmic vertical axis, labels, a small legend, and Figure 08b.1. Output: three PASS lines (one with its `reproduces` line) and the figure line.

What Figure 08b.1 shows. The scale factors (pure numbers, logarithmic vertical axis) against the time x_4 from 0 to 6 (units of $1/m$), along $a_4 = x_4$, at $y = 0$ (solid) and $y = -1$ (dashed). Blue: 3-space, a rising straight line from e^{Hy} to $e^6 e^{Hy}$. Red: the extra times, a falling straight line from e^{Hy} to $e^{-6}e^{Hy}$. Black: the product of the six, a horizontal line at e^{6Hy} (1 at $y = 0$, e^{-6} at $y = -1$). The student should see exponential inflation of 3-space, exponential deflation of the extra times, and a product that does not change in time.

In [4], frame momenta and the onset.

```
def frame_momenta(x4, q1, q5, y):
    """(k_(1), k_(5)) of the wave exp(i (q1 x1 + q5 x5)) at time x4 and position y."""
    w = np.exp(-H * y)  # e^(-H y) = sin^(-1/6) z
    return q1 * np.exp(-a4_of(x4)) * w, q5 * np.exp(a4_of(x4)) * w


def local_E2(x4, q1, q5, y):
    """E^2 = m^2 + k_(1)^2 - k_(5)^2 with frozen coefficients."""
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return mass**2 + k1**2 - k5**2
```

`frame_momenta` returns the pair $(k_{(1)}, k_{(5)})$ of Section 8.24, lines 1 and 2; `local_E2` returns E^2 of line 3 (a function may return two values, which the caller unpacks into two names).

```
def onset_time(q1, q5, y):
    """The exact time at which E^2 = 0 (positive root of the quadratic in X)."""
    w = np.exp(-H * y)
    X = (mass**2 + np.sqrt(mass**4 + 4 * q1**2 * q5**2 * w**4)) / (2 * q5**2 * w**2)
    return np.log(X) / (2 * A * H)
```

`onset_time` is the formula $x_4^* = \ln(X^*)/(2AH)$ with the positive root X^* of Section 8.24.

```
def bisection(q1, q5, y, low=-30.0, high=30.0):
    """The time at which E^2 changes sign, by halving [low, high] 200 times."""
    for _ in range(200):
        middle = (low + high) / 2
        if local_E2(middle, q1, q5, y) > 0:  # still oscillating: onset is later
            low = middle
        else:
            high = middle
    return (low + high) / 2
```

`bisection` finds the same time independently, by the **bisection method**: start with an interval $[-30, 30]$ in which E^2 changes sign; look at the middle; if E^2 is still positive there, the onset is later and the middle becomes the new left end, otherwise the new right end; after 200 halvings the interval is far smaller than the rounding of the numbers, and its middle is returned. (`low=-30.0` gives a default value used when the caller does not pass one.)

```
worst = max(abs(onset_time(q1, q5, y) - bisection(q1, q5, y))
            for q1 in (0.0, 0.5, 1.0, 3.0) for q5 in (0.01, 0.05, 0.1, 0.4)
            for y in (0.0, -1.0, -3.0))
check(worst < 1e-12, "the exact onset time equals the bisection result (48 waves)")
check(abs(onset_time(0.0, 0.05, 0.0) - np.log(1 / 0.05)) < 1e-14,
      "q1 = 0, y = 0: onset time = ln(m/q5)")
for q5 in (0.05, 0.1):
    report(f"onset time for q1 = 0, q5 = {q5}, y = 0", f"{onset_time(0.0, q5, 0.0):.6f}")
```

The two methods are compared for $4 \times 4 \times 3 = 48$ waves (four values of q_1, four of q_5, three positions); the largest difference must be below 10^{-12}. The second check is the simple form $x_4^* = \ln(m/q_5)$ for

$q_1 = 0$, $y = 0$ (with $m = 1$). The RESULT lines print 2.995732 ($\ln 20$) and 2.302585 ($\ln 10$). Output: two PASS lines and two RESULT lines.

In [5], the frame momenta and the local E^2 along the history.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
k1_path, k5_path = frame_momenta(times, 0.1, 0.1, 0.0)
left.plot(times, k5_path, color="tab:red", label="extra time: $k_{(5)} = q_5e^{a_4}$")
left.plot(times, k1_path, color="tab:blue", label="3-space: $k_{(1)} = q_1e^{-a_4}$")
left.axhline(mass, color="black", linewidth=0.8, linestyle=":", label="$m$")
left.set_yscale("log")
left.set_xlabel("time $x_4$")
left.set_ylabel("frame momentum (units of $m$)")
left.set_title("$q_1 = q_5 = 0.1$, $y = 0$")
left.legend(fontsize=8)
```

The left panel draws the two frame momenta of the wave with $q_1 = q_5 = 0.1$ at $y = 0$ along the 601 times, on a logarithmic axis, with a dotted line at the mass.

```
for q1, q5, style in ((0.0, 0.02, "-"), (0.0, 0.05, "-"), (0.0, 0.1, "-"),
                      (0.0, 0.2, "-"), (1.0, 0.05, "--")):
    right.plot(times, local_E2(times, q1, q5, 0.0), style,
               label=f"$q_1 = {q1}$, $q_5 = {q5}$")
    t_star = onset_time(q1, q5, 0.0)
    right.plot([t_star], [0.0], "o", color="black", markersize=4)
right.axhline(0.0, color="black", linewidth=0.8)
right.set_ylim(-4.0, 2.5)
right.set_xlabel("time $x_4$")
right.set_ylabel("$E^2 = m^2 + k_{(1)}^2 - k_{(5)}^2$")
right.set_title("Local $E^2$ along the history, $y = 0$")
right.legend(fontsize=8, loc="lower left");
save_figure(fig, "frame_momenta",
            "Left: the frame momenta of one wave with coordinate wave numbers "
            ...
            "onset is almost that of the same wave with $q_1 = 0$.")
```

The right panel draws the local E^2 of five waves (four with $q_1 = 0$, one dashed with $q_1 = 1$) and marks each onset time with a black dot on the zero line. The figure is Figure 08b.2; the cell prints only the figure line.

What Figure 08b.2 shows. Left: the frame momenta (units of m, logarithmic axis) against the time x_4 for $q_1 = q_5 = 0.1$ at $y = 0$: the red line $k_{(5)} = 0.1e^{x_4}$ rises and crosses the dotted mass line at $x_4 = \ln 10 \approx 2.3$; the blue line $k_{(1)} = 0.1e^{-x_4}$ falls. Right: the local E^2 (units of m^2) of five waves: each starts near 1, decreases, crosses zero once at its onset (black dot) and stays negative; smaller q_5 cross later. The dashed wave with $q_1 = 1$ starts at $E^2 = 1 + 1 - 0.0025 \approx 2$, but its space term dies away like e^{-2a_4}, and its onset (2.997) is almost that of the wave with $q_1 = 0$, $q_5 = 0.05$ (2.996). The student should see that the deflation drives every wave from oscillation into growth.

In [6], every wave along an extra time reaches the onset.

```
q5_grid = np.logspace(-4.0, 0.0, 200)  # 10^-4 ... 1, equally spaced on a log axis
fig, ax = plt.subplots()
```

```
all_finite_and_decreasing = True
for q1, y, style in ((0.0, 0.0, "-"), (0.0, -1.0, "--"), (0.0, -L_tip, ":"),
                     (1.0, 0.0, "-.")):
    onsets = onset_time(q1, q5_grid, y)
    all_finite_and_decreasing &= bool(np.all(np.isfinite(onsets))
                                      and np.all(np.diff(onsets) < 0))
    ax.plot(q5_grid, onsets, style, label=f"$q_1 = {q1}$, $y = {y}$")
```

`np.logspace(-4.0, 0.0, 200)` gives 200 values of q_5 from 10^{-4} to 1, equally spaced on a logarithmic axis. For four cases (three hidden positions, including the tip cutoff $y = -3$, and one wave with $q_1 = 1$) the onset times are computed for the whole grid at once. `np.isfinite` is true for an ordinary number (not infinite, not "not a number"); `np.diff` gives the differences of neighbours, which must all be negative (the onset time decreases as q_5 grows). `&=` keeps the variable true only while every case passes.

```
ax.set_xscale("log")
ax.set_xlabel("coordinate wave number $q_5$ along the extra time $x_5$")
ax.set_ylabel("onset time $x_4^\\ast$ (units of $1/m$)")
ax.set_title("Every extra-time wave reaches the onset")
ax.legend();
save_figure(fig, "onset_times",
            "The onset time $x_4^\\ast$ at which a wave with coordinate wave number "
            ...
            "$x_4 = 0$.")
check_record(all_finite_and_decreasing,
             "the onset time is finite for every q5 > 0 on the grid and decreases "
             "with q5", "Revision/theory/reports/python-field-theory.json",
             "extra_time_modes_grow",
             "every extra-time mode eventually enters the growing regime")
```

A logarithmic horizontal axis, labels, Figure 08b.3, and the check, which requires the record's sentence "every extra-time mode eventually enters the growing regime". Output: the figure line and one PASS line with its `reproduces` line.

What Figure 08b.3 shows. The onset time x_4^* (vertical, units of $1/m$) against the coordinate wave number q_5 from 10^{-4} to 1 (horizontal, logarithmic). Each curve is a falling, nearly straight line: on a logarithmic q_5 axis, $x_4^* = (Hy + \ln(m/q_5))/(AH)$ is a straight line of slope -1 per factor e. At $q_5 = 10^{-4}$ and $y = 0$ the onset is at $\ln 10^4 \approx 9.2$; the curve for $y = -1$ lies one unit lower and that for $y = -3$ three units lower (for large q_5 these are negative: such waves grow already at $x_4 = 0$); the curve with $q_1 = 1$ lies on top of the $y = 0$ curve except at the largest q_5. The student should see that no curve goes to infinity: however small q_5 is, the onset comes.

In [7], the model's mode matrix and the starting columns.

```
gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
GAMMA = [np.array(rows, dtype=int) for rows in gammas_record["gamma"]]
C = np.array(gammas_record["C"], dtype=int)  # C = g^(x0) g^(x1) g^(x2) g^(x3)
B = np.array(gammas_record["B"]["re"]) + 1j * np.array(gammas_record["B"]["im"])
g4 = GAMMA[4].astype(complex)  # gamma^(x4)
g4g1 = (GAMMA[4] @ GAMMA[1]).astype(complex)  # gamma^(x4) gamma^(x1)
```

```
g4g5 = (GAMMA[4] @ GAMMA[5]).astype(complex)  # gamma^(x4) gamma^(x5)
check(np.array_equal(C @ GAMMA[4], GAMMA[4] @ C) and
      np.array_equal(C @ GAMMA[5], GAMMA[5] @ C),
      "C commutes with gamma^(x4) and gamma^(x5)")
```

The gammas, C and B are read as in Notebook 08a; the three matrices of the model, $\gamma^{(4)}$, $\gamma^{(4)}\gamma^{(1)}$ and $\gamma^{(4)}\gamma^{(5)}$, are prepared as complex arrays. The check confirms that C commutes with $\gamma^{(4)}$ and $\gamma^{(5)}$ (Section 8.25).

```
def h_local(x4, q1, q5, y):
    """The mode matrix of the local-frame model at time x4."""
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return -1j * mass * g4 - k1 * g4g1 - k5 * g4g5
```

`h_local` is the model's mode matrix $h(x_4) = -im\gamma^{(4)} - k_{(1)}(x_4)\gamma^{(4)}\gamma^{(1)} - k_{(5)}(x_4)\gamma^{(4)}\gamma^{(5)}$.

```
# At every instant h(x4)^2 = E^2(x4) I16, where E^2 is the frozen-coefficient E^2 of
# the record with k1 = k_(1)(x4), k5 = k_(5)(x4) and all other momenta 0.
m_s = sp.Symbol("m", real=True)
k_s = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}
E2_record = m_s**2 + sum(k_s[a] ** 2 for a in (0, 1, 2, 3)) \
    - sum(k_s[a] ** 2 for a in (5, 6, 7))
```

`E2_record` is the record's general formula $E^2 = m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2$, built with exact symbols.

```
squares_ok = True
for x4 in (0.0, 1.0, 2.0, 3.0, 4.0):
    k1_now, k5_now = frame_momenta(x4, 0.3, 0.1, 0.0)  # q1 = 0.3, q5 = 0.1, y = 0
    values = {symbol: 0 for symbol in k_s.values()}  # every momentum 0 ...
    values.update({m_s: mass, k_s[1]: k1_now, k_s[5]: k5_now})  # ... but these
    E2_now = float(E2_record.subs(values))
    h_now = h_local(x4, 0.3, 0.1, 0.0)
    squares_ok &= bool(np.allclose(h_now @ h_now, E2_now * np.eye(16), atol=1e-12)
                       and abs(E2_now - local_E2(x4, 0.3, 0.1, 0.0)) < 1e-12)
check_record(squares_ok, "x4 = 0, 1, 2, 3, 4: h(x4)^2 = E^2(x4) I16 (q1 = 0.3, "
             "q5 = 0.1)", "Revision/theory/reports/python-scope.json",
             "extra_time_growth_rates_unbounded", f"h_k^2 = ({sp.sstr(E2_record)}) I16")
```

At five instants, for the wave $q_1 = 0.3$, $q_5 = 0.1$, the record's formula is evaluated with the frame momenta of that instant (a dictionary sets every momentum to 0, and `update` then sets the mass, k_1 and k_5), and the square of the model's matrix is compared with $E^2 I_{16}$ to 10^{-12}; the value must also agree with `local_E2`. The check requires the record's text with the general E^2.

```
def starting_column(q5, sign):
    """A unit column with h(0) u = E0 u and C u = sign u (q1 = 0, y = 0)."""
    h0 = h_local(0.0, 0.0, q5, 0.0)
    E0 = np.sqrt(mass**2 - q5**2)
```

```
    first = np.zeros(16, dtype=complex)
    first[0] = 1.0  # the first unit column
    u = (np.eye(16) + h0 / E0) @ ((np.eye(16) + sign * C) @ first) / 4
    return u / np.linalg.norm(u), h0, E0
```

`starting_column` applies the two projectors of Section 8.25, $\frac{1}{2}(I+\sigma C)$ and $\frac{1}{2}(I+h(0)/E_0)$ (together the factor $\frac{1}{4}$), to the first unit column (a column of zeros with a 1 in position 0) and divides by the length.

```
starts = {}
for q5, sign in ((0.05, +1), (0.1, -1)):
    u0, h0, E0 = starting_column(q5, sign)
    starts[q5] = u0
    krein0 = np.vdot(u0, B @ u0)  # u^dagger B u
    report(f"q5 = {q5}: E0, Krein form of the start",
           f"{E0:.9f}, {krein0.real:+.9f}")
    sign_text = "+" if sign > 0 else "-"  # the sign as a character
    check(np.linalg.norm(h0 @ u0 - E0 * u0) < 1e-14 and
          np.linalg.norm(C @ u0 - sign * u0) < 1e-14 and
          abs(krein0 - sign * E0 / mass) < 1e-14,
          f"q5 = {q5}: h(0) u = E0 u, C u = {sign_text}u, "
          f"Krein form = {sign_text}E0/m")
```

Two starting columns are made, for $q_5 = 0.05$ with $\sigma = +1$ and for $q_5 = 0.1$ with $\sigma = -1$, and stored in the dictionary `starts`. For each, the RESULT line prints E_0 and the Krein form (0.998749218 and +0.998749218; 0.994987437 and −0.994987437), and the check confirms to 10^{-14} the three properties: eigenvector of $h(0)$ for E_0, eigenvector of C for σ, Krein form $\sigma E_0/m$. Output: three PASS lines (one with its `reproduces` line) and two RESULT lines.

In [8], RK4 through the onset.

```
def rk4_history(u0, q5, x4_end, steps):
    """RK4 for du/dx4 = -i h(x4) u (q1 = 0, y = 0); returns the times, u^dagger u,
    u^dagger B u after every step, and the final column."""
    step = x4_end / steps
    u = u0.copy()
    times_out = [0.0]
    hilbert = [np.vdot(u, u).real]
    krein = [np.vdot(u, B @ u)]

    def rate(x4, v):  # the right-hand side -i h(x4) v
        return -1j * (h_local(x4, 0.0, q5, 0.0) @ v)
```

`rk4_history` solves the model with RK4 and records, after every step, the time, the Hilbert norm and the Krein form (`u0.copy()` makes a copy, so that the starting column itself is not changed). The inner function `rate` is the right-hand side $-ih(x_4)v$; unlike In [8] of Notebook 08a the matrix now depends on the time.

```
    for n in range(steps):
        x4 = n * step
```

```
        s1 = rate(x4, u)
        s2 = rate(x4 + step / 2, u + step / 2 * s1)
        s3 = rate(x4 + step / 2, u + step / 2 * s2)
        s4 = rate(x4 + step, u + step * s3)
        u = u + step / 6 * (s1 + 2 * s2 + 2 * s3 + s4)
        times_out.append((n + 1) * step)
        hilbert.append(np.vdot(u, u).real)
        krein.append(np.vdot(u, B @ u))
    return np.array(times_out), np.array(hilbert), np.array(krein), u
```

The classical RK4 step for an equation whose right-hand side depends on the time: the four slopes are taken at the start, twice at the middle and at the end of the step. The function returns three arrays and the final column.

```
runs = {}
for q5 in (0.05, 0.1):
    x4_end = onset_time(0.0, q5, 0.0) + 3.0
    runs[q5] = rk4_history(starts[q5], q5, x4_end, 12000)
    at_onset = np.searchsorted(runs[q5][0], onset_time(0.0, q5, 0.0))  # its index
    report(f"q5 = {q5}: u^dagger u at the onset", f"{runs[q5][1][at_onset]:.2f}")
    final_size = runs[q5][1][-1]  # u^dagger u at the end of the run
    report(f"q5 = {q5}: ln(u^dagger u) at the onset + 3", f"{np.log(final_size):.6f}")
```

Each wave is followed from $x_4 = 0$ to three time units after its onset with 12000 steps; the results are stored in `runs` (for each wave the four results of `rk4_history`: `runs[q5][0]` is the array of the times, `runs[q5][1]` the array of the sizes $u^\dagger u$). `np.searchsorted(runs[q5][0], onset_time(0.0, q5, 0.0))` is the position of the first recorded time that is not earlier than the onset time, that is the first step at or after the onset; the first RESULT line of each wave prints the size $u^\dagger u$ there, with two decimals: 1.52 for $q_5 = 0.05$ and 1.54 for $q_5 = 0.1$. The size started at 1, so before the onset it grows only slowly, by about half (the mode matrix is not Hermitian, so the size is not conserved; Section 8.19). The second RESULT line of each wave prints $\ln(u^\dagger u)$ at the end of the run: 37.071245 and 37.113916.

```
x4_end = onset_time(0.0, 0.05, 0.0) + 3.0
end_6000 = rk4_history(starts[0.05], 0.05, x4_end, 6000)[3]
end_24000 = rk4_history(starts[0.05], 0.05, x4_end, 24000)[3]
end_12000 = runs[0.05][3]
ratio = np.linalg.norm(end_6000 - end_12000) / np.linalg.norm(end_12000 - end_24000)
relative_error = np.linalg.norm(end_12000 - end_24000) / np.linalg.norm(end_24000)
report("RK4 error ratio (6000 vs 12000) / (12000 vs 24000) steps", f"{ratio:.2f}")
check(14.0 < ratio < 18.0, "RK4 converges with order 4 (error ratio near 16)")
check(relative_error < 1e-8, "the 12000-step run is accurate to 1e-8 (relative)")
```

The convergence test: the run with $q_5 = 0.05$ is repeated with 6000 and 24000 steps (`[3]` takes the final column). For a fourth-order method the difference between 6000 and 12000 steps is about $2^4 = 16$ times the difference between 12000 and 24000 steps; the ratio printed is 15.89, and the check requires it between 14 and 18. The difference between 12000 and 24000 steps, relative to the length of the column, estimates the error of the 12000-step run; it must be below 10^{-8}. Output of the whole cell: five RESULT lines and two PASS lines.

In [9], the WKB formulas and the comparison.

```
def Q_of(x4, q5):
    return q5 * np.exp(a4_of(x4))  # the frame momentum k_(5) at y = 0


def kappa_of(x4, q5):
    return np.sqrt(Q_of(x4, q5) ** 2 - mass**2)


def W_of(x4, q5):
    Q = Q_of(x4, q5)
    return (np.sqrt(Q**2 - mass**2) - mass * np.arccos(mass / Q)) / (A * H)
```

The three functions of Section 8.25: $Q = q_5 e^{a_4}$, $\kappa = \sqrt{Q^2 - m^2}$ and $W = \frac{1}{AH}(\sqrt{Q^2 - m^2} - m \arccos(m/Q))$ (`np.arccos` is the inverse cosine).

```
comparison = {}
for q5 in (0.05, 0.1):
    x4s, hilbert, krein, _ = runs[q5]
    start = np.searchsorted(x4s, onset_time(0.0, q5, 0.0) + 1.5)  # window start
    xa, xb = x4s[start], x4s[-1]
    increase = np.log(hilbert[-1]) - np.log(hilbert[start])
    leading = 2 * (W_of(xb, q5) - W_of(xa, q5))
    first = leading + np.log(Q_of(xb, q5) / kappa_of(xb, q5)) \
        - np.log(Q_of(xa, q5) / kappa_of(xa, q5))
    comparison[q5] = (xa, increase, leading, first)
```

For each run the window starts at the first recorded time 1.5 units after the onset (`np.searchsorted` finds the position at which a value would be inserted into a sorted array) and ends at the last time. `increase` is the increase of $\ln(u^\dagger u)$ over the window in the RK4 solution; `leading` is the leading WKB increase $2(W(x_b) - W(x_a))$; `first` adds the first-order correction $\ln(Q/\kappa)$ at the two ends. The four numbers are stored for the plot.

```
    say(f"q5 = {q5}: window {xa:.4f} to {xb:.4f}; increase of ln(u^dagger u) "
        f"{increase:.4f}, leading WKB {leading:.4f} (relative difference "
        f"{abs(increase - leading) / increase:.2e}), first order {first:.4f} "
        f"(relative difference {abs(increase - first) / increase:.2e})")
    check(abs(increase - leading) / increase < 2e-3,
          f"q5 = {q5}: leading WKB agrees to 2e-3 over the window")
    check(abs(increase - first) < abs(increase - leading) and
          abs(increase - first) / increase < 5e-4,
          f"q5 = {q5}: first-order WKB is better and agrees to 5e-4")
```

The comparison is printed (the numbers quoted in Section 8.25), and two checks per wave require the leading formula to agree to 2×10^{-3} and the first-order formula to be better and to agree to 5×10^{-4}.

```
x4s, hilbert, _, _ = runs[0.05]
step = x4s[1] - x4s[0]
late_rate = (np.log(hilbert[-1]) - np.log(hilbert[-3])) / (2 * step)  # centred
kappa_late = kappa_of(x4s[-2], 0.05)  # kappa at the middle of the centred difference
```

```
predicted = 2 * kappa_late - A * H * mass**2 / kappa_late**2
report("q5 = 0.05: computed late rate, predicted 2 kappa - m^2/kappa^2",
       f"{late_rate:.5f}, {predicted:.5f}")
check(abs(late_rate - predicted) / predicted < 1e-5,
      "the late growth rate equals 2 kappa - A H m^2/kappa^2 to 1e-5")
```

The growth rate at the end of the run is computed by a **centred difference**, the change of $\ln(u^\dagger u)$ over the last two steps divided by their length, which approximates the derivative at the middle point; the first-order prediction is evaluated at that middle point. RESULT: 40.09851 and 40.09868; the check requires a relative agreement better than 10^{-5}. Output: two comparison texts, five PASS lines and one RESULT line.

In [10], the growth of the size along the history.

```
fig, ax = plt.subplots()
for q5, colour in ((0.05, "tab:blue"), (0.1, "tab:orange")):
    x4s, hilbert, _, _ = runs[q5]
    xa, _, _, _ = comparison[q5]
    ax.plot(x4s, np.log(hilbert), color=colour, label=f"RK4, $q_5 = {q5}$")
    after = x4s >= onset_time(0.0, q5, 0.0) + 0.05
    shift = np.log(hilbert[np.searchsorted(x4s, xa)]) - 2 * W_of(xa, q5)
    ax.plot(x4s[after], 2 * W_of(x4s[after], q5) + shift, "--", color="black",
            linewidth=0.9)
    ax.axvline(onset_time(0.0, q5, 0.0), color=colour, linestyle=":", linewidth=0.9)
```

For both runs: $\ln(u^\dagger u)$ of the RK4 solution; the leading WKB curve $2W$, drawn from 0.05 units after the onset (`after` is a true/false array that selects those times) and shifted by a constant so that it agrees with the solution at the window start; and a dotted vertical line at the onset (`axvline`).

```
ax.plot([], [], "--", color="black", linewidth=0.9, label="WKB $2W$ + const")
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("$\\ln(u^\\dagger u)$")
ax.set_title("A wave along an extra time, $a_4 = x_4$, $y = 0$")
ax.legend();
save_figure(fig, "growth_along_history",
            "The natural logarithm of the size $u^\\dagger u$ of a wave with "
            ...
            "the WKB curve $2W$ (black dashed, matched 1.5 units after the onset).")
```

An empty dashed line for the legend, labels, and Figure 08b.4; the cell prints only the figure line.

What Figure 08b.4 shows. The natural logarithm of $u^\dagger u$ (vertical, pure number) against the time x_4 (units of $1/m$) for the two waves $q_5 = 0.05$ (blue) and $q_5 = 0.1$ (orange). Before its onset (dotted vertical line at $\ln 10 \approx 2.30$ or $\ln 20 \approx 3.00$) each curve rises only slowly, from 0 to about 0.4 ($\ln 1.52 = 0.42$, $\ln 1.54 = 0.43$): the wave oscillates and its size grows only by about half, because with extra-time momentum the mode matrix is not Hermitian and the size is not conserved. After the onset the curve bends upwards ever more steeply and reaches about 37 three units later (a factor of about 10^{16}), following the black dashed WKB curve. The student should see the switch from oscillation to growth at the predicted time, and growth that accelerates.

In [11], the growth rate and the Krein form.

```
x4s, hilbert, krein, _ = runs[0.05]
step = x4s[1] - x4s[0]
computed_rate = (np.log(hilbert[2:]) - np.log(hilbert[:-2])) / (2 * step)
middle = x4s[1:-1]
after = middle > onset_time(0.0, 0.05, 0.0) + 0.02
drift = np.abs(krein - krein[0]) / np.maximum(hilbert, 1.0)
check(drift.max() < 1e-9, "the Krein form is conserved (drift below 1e-9)")
check(hilbert[-1] > 1e15, "the Hilbert norm grows by more than 10^15")
```

For the run with $q_5 = 0.05$: the growth rate at every inner time by centred differences (`hilbert[2:]` drops the first two values, `hilbert[:-2]` the last two, so the two arrays are shifted by two steps; `middle` are the middle times); `after` selects the times just after the onset; `drift` is the change of the Krein form divided by $\max(u^\dagger u, 1)$. The checks require the drift below 10^{-9} and a final size above 10^{15}.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
fig.subplots_adjust(wspace=0.35)  # room for the label of the right panel
left.plot(middle, computed_rate, color="tab:blue", label="RK4 (centred differences)")
left.plot(middle[after], 2 * kappa_of(middle[after], 0.05), "--", color="black",
          label="$2\\kappa$")
left.plot(middle[after], 2 * kappa_of(middle[after], 0.05)
          - mass**2 / kappa_of(middle[after], 0.05) ** 2, ":", color="tab:red",
          label="$2\\kappa - m^2/\\kappa^2$")
left.set_ylim(-5.0, 45.0)
left.set_xlabel("time $x_4$")
left.set_ylabel("growth rate $d\\ln(u^\\dagger u)/dx_4$")
left.set_title("Growth rate, $q_5 = 0.05$")
left.legend(fontsize=8)
```

The left panel draws the computed growth rate with the two WKB rates (`subplots_adjust(wspace=0.35)` widens the gap between the panels).

```
right.semilogy(x4s[1:], np.maximum(drift[1:], 1e-18), color="tab:green")
right.set_xlabel("time $x_4$")
right.set_ylabel("$|\\Delta(u^\\dagger Bu)| / \\max(u^\\dagger u, 1)$")
right.set_title("Krein form: change (rounding only)")
save_figure(fig, "rate_and_krein",
            "Left: the growth rate $d\\ln(u^\\dagger u)/dx_4$ of the wave with "
            ...
            "$10^{-9}$.")
```

The right panel draws the drift on a logarithmic axis (`semilogy`); a logarithm cannot show an exact zero, so zeros are raised to 10^{-18} (`np.maximum`). The figure is Figure 08b.5. Output: two PASS lines and the figure line.

What Figure 08b.5 shows. Left: the growth rate $d\ln(u^\dagger u)/dx_4$ (units of m) against the time x_4 (units of $1/m$): small before the onset at $x_4 \approx 3$ (it starts at 0 and stays below 1; on the vertical scale up to 45 this looks almost flat), then rising ever more steeply to about 40; the black dashed curve 2κ and the red dotted curve $2\kappa - m^2/\kappa^2$ start at the onset, where both fail ($\kappa \to 0$), and the computed rate approaches the red curve as κ grows. Right: the change of the Krein form relative to

$\max(u^\dagger u, 1)$, on a logarithmic axis: it stays below about 10^{-14} (the drops to 10^{-18} are exact zeros), the level of rounding errors. The student should see that the rate itself grows exponentially (growth faster than any exponential) while the Krein form is conserved.

In [12], the last check.

```
for name in ("08b_1_scale_factors.png", "08b_2_frame_momenta.png",
             "08b_3_onset_times.png", "08b_4_growth_along_history.png",
             "08b_5_rate_and_krein.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Five checks that the figure files exist, and the last line. The 25 checks come from the cells as follows: one in In [2], three in In [3], two in In [4], one in In [6], four in In [7], two in In [8], five in In [9], two in In [11] and five here (In [5] and In [10] have none). The last line reads `ALL 25 CHECKS PASSED (notebook 08b)`.

8.30 Changing the field variables: the rescaling $\Psi = \sin^{-1/2} z\,\chi$

Section 8.9 changed the frame. This section keeps the diagonal frame and changes the field variables instead: we write the field as a known function times a new field, $\Psi = w\,\chi$ with $w = \sin^{-1/2} z$, and ask which equation χ obeys.

The derivative of the factor. w depends only on x_0. Line by line:

$$\partial_0 w = -\tfrac{1}{2}\sin^{-3/2} z \cdot \cos z \cdot 6H = -3H \sin^{-3/2} z \cos z,$$
$$\tan z\,\partial_0 w = \frac{\sin z}{\cos z} \cdot \big(-3H \sin^{-3/2} z \cos z\big) = -3H \sin^{-1/2} z = -3H\,w.$$

The first line is the chain rule (the power rule for $\sin^{-1/2}$, the derivative $\cos z$ of $\sin z$, and $dz/dx_0 = 6H$); the second multiplies by $\tan z = \sin z/\cos z$ and combines the powers of $\sin z$ ($\sin z \cdot \sin^{-3/2} z = \sin^{-1/2} z$).

The term disappears. For $\mu \neq 8$, $\partial_\mu(w\chi) = w\,\partial_\mu\chi$, because w does not depend on x_μ. For $\mu = 8$ the product rule gives $\partial_0(w\chi) = (\partial_0 w)\chi + w\,\partial_0\chi$. So, line by line,

$$\begin{aligned}\gamma^\mu D_\mu(w\chi) &= \sum_\mu \gamma^\mu \partial_\mu(w\chi) + 3H\gamma^{(0)} w\chi \\ &= w\sum_\mu \gamma^\mu \partial_\mu\chi + \tan z\,\gamma^{(0)}(\partial_0 w)\chi + 3H\gamma^{(0)} w\chi \\ &= w\sum_\mu \gamma^\mu \partial_\mu\chi + \big(-3Hw + 3Hw\big)\gamma^{(0)}\chi = w\sum_\mu \gamma^\mu \partial_\mu\chi.\end{aligned}$$

The first line uses $\gamma^\mu D_\mu = \gamma^\mu\partial_\mu + \gamma^\mu\Omega_\mu$ and $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$ (Section 8.5); the second applies the two derivative rules, with $\gamma^{x_0} = \tan z\,\gamma^{(0)}$; the third inserts $\tan z\,\partial_0 w = -3Hw$. For every column χ of 16 functions the spin-connection term is gone: in the variables χ the derivative part of the field equation is $w\,\gamma^\mu\partial_\mu\chi$, with no term $3H\gamma^{(0)}$ (PROVED; `Revision/theory/reports/python-s`

`cope.json`, check `rescaling_removes_the_connection_term`, both engines). The frame factors $e^{\mp a_4}\sin^{-1/6} z$ and $\tan z$ stay: they are part of γ^μ.

The potential. S is quadratic in the field and w is real, so $S[w\chi] = (w\chi)^\dagger C(w\chi) = w^2\,\chi^\dagger C\chi = S[\chi]/\sin z$ (because $w^2 = \sin^{-1} z$). With $U = \frac{\lambda}{2}S^2$, $U'(S[\Psi]) = \lambda S[\Psi] = \lambda S[\chi]/\sin z$. The field equation $\gamma^\mu D_\mu\Psi = (m+\lambda S[\Psi])\Psi$ becomes $w\,\gamma^\mu\partial_\mu\chi = (m+\lambda S[\chi]/\sin z)\,w\chi$, and dividing by $w > 0$:

$$\gamma^\mu\partial_\mu\chi = \Big(m + \frac{\lambda S[\chi]}{\sin z}\Big)\chi.$$

For $U = 0$ the term $3H\gamma^{(0)}$ is removed completely; for $U \neq 0$ it reappears in a new form, as the x_0-dependent coupling $\lambda S[\chi]/\sin z$ (PROVED; `python-scope.json`, check `rescaled_equation_quadratic_potential`, both engines).

The weight moves into the measure. The norm of a field along the hidden direction is $\int \cos z\,\Psi^\dagger\Psi\,dx_0$ ($\cos z = \sqrt{|g|}$ is the volume factor). In the new variables $\cos z\,\Psi^\dagger\Psi\,dx_0 = \cos z\,w^2\,\chi^\dagger\chi\,dx_0 = \cot z\,\chi^\dagger\chi\,dx_0 = \chi^\dagger\chi\,dy$, by $dy = \cot z\,dx_0$ (Section 8.24). So the norm of Ψ is the plain y-integral of $\chi^\dagger\chi$.

The scope of "the field equation contains $3H\gamma^{(0)}$". Together with Section 8.9: the value $3H\gamma^{(0)}$ belongs to one frame (the diagonal one) AND to one choice of field variables (Ψ). A boosted frame removes it, and so does the rescaling (for a changing b_4 the rescaling $\Psi = \sin^{-1/2} z\,b_4^{-3/4}\chi$ removes the whole term $3H\gamma^{(0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(4)}$, Section 6.18). Interpretation (labelled as such, not a theorem): in the variables Ψ with the measure $\cos z\,dx_0$ the term makes the hidden-direction operator $\tan z\,\partial_0 + 3H$ antisymmetric up to a boundary term (Section 8.32); in the variables χ with the measure dy the plain ∂_y has the same property without any H term. The role of $3H$ therefore depends on the variables and the measure.

8.31 The two ends of the hidden direction and an exact family of solutions

The two ends. Proper length along the hidden direction is $\sqrt{g_{00}}\,dx_0 = \cot z\,dx_0 = dy$. The **tip** $z \to 0$ is $y \to -\infty$: infinitely far from every point. The **patch end** $z = \pi/2$ is $y = 0$: at the finite proper distance $|y|$ from the point y. At the patch end $g_{00} = \cot^2 z = 0$ and $\sqrt{|g|} = \cos z = 0$: the surface is degenerate, and the Revision record claims nothing about it and imposes no boundary condition there. (The Kohn-Sham record ASSUMES a Z_2 **brane** at $z = \pi/2$. A **brane** is a boundary surface of spacetime, here the surface $z = \pi/2$ where the author's coordinate patch ends; "Z_2" says that the patch is glued there to a mirror copy of itself, and the mirror fixes a boundary condition for the field at that surface. Chapter 14 works it out. This chapter does not use it.)

An exact family. For $U = 0$ and a field that depends only on x_4 and x_0, the field equation of Section 8.2 is $\gamma^{(4)}\partial_4\Psi + \tan z\,\gamma^{(0)}\partial_0\Psi + 3H\gamma^{(0)}\Psi = m\Psi$. The record gives a family of exact solutions (`Revision/theory/reports/python-field-theory.json`, check `exact_solution_family_x4_x0`):

$$\Psi = \sin^\alpha z\; e^{Mx_4}\chi_0, \qquad M = -m\gamma^{(4)} + c\,\gamma^{(4)}\gamma^{(0)}, \qquad c = 3H(2\alpha+1),$$

with any real exponent α and any constant column χ_0. We derive it line by line. First, M^2:

$$
\begin{aligned}
M^2 &= m^2(\gamma^{(4)})^2 - mc\big((\gamma^{(4)})^2\gamma^{(0)} + \gamma^{(4)}\gamma^{(0)}\gamma^{(4)}\big) + c^2\gamma^{(4)}\gamma^{(0)}\gamma^{(4)}\gamma^{(0)} \\
&= -m^2 - mc\big(-\gamma^{(0)} + \gamma^{(0)}\big) - c^2(\gamma^{(4)})^2(\gamma^{(0)})^2 \\
&= (c^2 - m^2)\, I_{16} = k^2 I_{16}, \qquad k^2 = 9H^2(2\alpha+1)^2 - m^2.
\end{aligned}
$$

The first line multiplies out; the second uses $(\gamma^{(4)})^2 = -I_{16}$, $\gamma^{(4)}\gamma^{(0)}\gamma^{(4)} = -(\gamma^{(4)})^2\gamma^{(0)} = \gamma^{(0)}$ and, in the last term, one anticommutation and the two squares. As in Section 8.18, $M^2 = k^2 I_{16}$ makes the exponential series collapse: $e^{Mx_4} = \cosh(kx_4)I_{16} + \frac{\sinh(kx_4)}{k}M$. Second, the equation. With $s = \sin z$: $\partial_0 s^\alpha = \alpha s^{\alpha-1}\cos z \cdot 6H$, so $\tan z\, \partial_0 s^\alpha = 6H\alpha\, s^\alpha$; and $\partial_4 e^{Mx_4} = Me^{Mx_4}$. Inserting, and dividing by the common factor s^α:

$$
\begin{aligned}
&\gamma^{(4)}Me^{Mx_4}\chi_0 + 6H\alpha\,\gamma^{(0)}e^{Mx_4}\chi_0 + 3H\gamma^{(0)}e^{Mx_4}\chi_0 = \big(\gamma^{(4)}M + c\,\gamma^{(0)}\big)e^{Mx_4}\chi_0, \\
&\gamma^{(4)}M = -m(\gamma^{(4)})^2 + c(\gamma^{(4)})^2\gamma^{(0)} = m - c\,\gamma^{(0)}, \\
&\big(\gamma^{(4)}M + c\,\gamma^{(0)}\big)e^{Mx_4}\chi_0 = m\,e^{Mx_4}\chi_0.
\end{aligned}
$$

The first line collects the two $\gamma^{(0)}$ terms ($6H\alpha + 3H = 3H(2\alpha+1) = c$); the second multiplies out $\gamma^{(4)}M$; the third inserts it: the left side equals $m\Psi$ after the factor s^α is put back. So every member solves the field equation, for every history a_4 (nothing in the solution depends on a_4, because the term $3H\gamma^{(0)}$ does not; PROVED).

Which members grow. A member grows like e^{kx_4} when $k^2 > 0$, that is when $3H|2\alpha+1| > m$, and oscillates when $k^2 < 0$. The member $\alpha = -\frac{1}{2}$ has $c = 0$, $M = -m\gamma^{(4)}$ without any H, and $k^2 = -m^2$: it oscillates with the frequency m. It is the rescaled field of Section 8.30, $\Psi = \sin^{-1/2} z\,\chi$ with χ independent of x_0 (PROVED; `python-scope.json`, check `rescaled_equation_quadratic_potential`, whose detail states this member). The member $\alpha = 0$, the fields that do not depend on x_0, has $k^2 = 9H^2 - m^2$ and grows exactly when $m < 3H$.

Which members have a finite norm. The norm of a member along the hidden direction is a function of x_4 times the integral of $\cos z\,\sin^{2\alpha} z$ over x_0 from the tip to the patch end (where $z = 6Hx_0 = \pi/2$, that is $x_0 = \pi/(12H)$). We substitute $s = \sin z$, so that $ds = 6H\cos z\, dx_0$. Line by line:

$$
\int_0^{\pi/(12H)} \cos z\,\sin^{2\alpha} z\, dx_0 = \frac{1}{6H}\int_0^1 s^{2\alpha}\, ds = \frac{1}{6H}\Big[\frac{s^{2\alpha+1}}{2\alpha+1}\Big]_0^1 = \frac{1}{6H(2\alpha+1)} \quad (2\alpha+1 > 0).
$$

The first step changes the variable (s runs from 0 to 1 as x_0 runs over the patch); the second takes the antiderivative of a power; the third evaluates it, and $s^{2\alpha+1} \to 0$ at $s = 0$ exactly when $2\alpha+1 > 0$. For $2\alpha + 1 \le 0$ the integral diverges at the tip; at $\alpha = -\frac{1}{2}$ it is $\int ds/s$, which grows like $\ln(1/\epsilon)$ as the lower limit ϵ goes to 0. So the member without H has an infinite norm, and the growing members with a finite norm are those with $2\alpha + 1 > m/(3H)$ and $\alpha > -\frac{1}{2}$.

For every mass some member grows and has a finite norm. Take $\alpha = m/(3H)$, which is $> -\frac{1}{2}$. Then $2\alpha + 1 = (2m + 3H)/(3H)$, $c = 2m + 3H$, and $k^2 = (2m + 3H)^2 - m^2 = (2m + 3H - m)(2m + 3H + m) = (m + 3H)(3m + 3H) = 3(m + H)(m + 3H) > 0$ (a difference of two squares). The record states the case $\alpha = 0$, $m < 3H$; the statement for every mass is this book's own exact consequence of the record's family, proved here and checked by Notebook 08d (PROVED).

8.32 The boundary term at the patch end and the growing modes of the good sector

The fields that do not depend on x_0. Such a field of the good sector depends only on x_4, and the field equation, multiplied by $\gamma^{(4)}$ as in Section 8.16, becomes $i\,\partial_4\Psi = A\Psi$ with the constant matrix $A = -im\gamma^{(4)} + 3iH\gamma^{(4)}\gamma^{(0)}$, which is i times the matrix M of the family at $\alpha = 0$. Its square is $A^2 = -M^2 = (m^2 - 9H^2)I_{16}$, so for $m < 3H$ its eigenvalues are $\pm i\sqrt{9H^2 - m^2}$, eight each, and half of the modes grow; at $m = H = 1$ the eigenvalues are $\pm 2\sqrt{2}\,i$. A is not Hermitian (its second term is anti-Hermitian, see below). These modes have the finite norm factor $\int_0^{\pi/(12H)} \cos(6Hx_0)\,dx_0 = \frac{1}{6H}$ (the case $\alpha = 0$ above). (PROVED; `python-scope.json`, check `good_sector_x0_independent_modes_without_boundary_condition`, both engines.) So even the good sector, without any extra-time dependence, contains growing modes of finite norm, as long as nothing is imposed at the patch end.

The mode operator. For fields of x_4 and x_0 (and $U = 0$), multiplying the field equation $\gamma^{(4)}\partial_4\Psi + \gamma^{(0)}(\tan z\,\partial_0 + 3H)\Psi = m\Psi$ from the left by $\gamma^{(4)}$ gives $-\partial_4\Psi + \gamma^{(4)}\gamma^{(0)}(\tan z\,\partial_0 + 3H)\Psi = m\gamma^{(4)}\Psi$, and multiplying by $-i$ and rearranging: $i\,\partial_4\Psi = h\Psi$ with the **mode operator**

$$h = -im\gamma^{(4)} + i\gamma^{(4)}\gamma^{(0)}\big(\tan z\,\partial_0 + 3H\big) = A + P\,\partial_0, \qquad P = i\tan z\,\gamma^{(4)}\gamma^{(0)}.$$

Three matrix facts. Let $M_0 = i\gamma^{(4)}\gamma^{(0)}$. The real matrix $\gamma^{(4)}\gamma^{(0)}$ is symmetric: $(\gamma^{(4)}\gamma^{(0)})^T = (\gamma^{(0)})^T(\gamma^{(4)})^T = \gamma^{(0)}(-\gamma^{(4)}) = \gamma^{(4)}\gamma^{(0)}$. Hence:

1. $P^\dagger = -i\tan z\,(\gamma^{(4)}\gamma^{(0)})^T = -P$: the coefficient of ∂_0 is anti-Hermitian.
2. $\cos z\,P = i\sin z\,\gamma^{(4)}\gamma^{(0)} = \sin z\,M_0$.
3. $(-im\gamma^{(4)})^\dagger = im(\gamma^{(4)})^T = -im\gamma^{(4)}$ (Hermitian) and $(3iH\gamma^{(4)}\gamma^{(0)})^\dagger = -3iH\gamma^{(4)}\gamma^{(0)}$ (anti-Hermitian), so $A - A^\dagger = 6iH\gamma^{(4)}\gamma^{(0)} = 6H\,M_0$, and $\cos z\,(A - A^\dagger) = 6H\cos z\,M_0 = (\partial_0 \sin z)\,M_0$.

The boundary identity. For any two columns $u(x_0)$, $v(x_0)$, line by line (a prime is ∂_0 here):

$$\begin{aligned}\cos z\big[u^\dagger(hv) - (hu)^\dagger v\big] &= \cos z\,u^\dagger(A - A^\dagger)v + \cos z\big[u^\dagger Pv' - u'^\dagger P^\dagger v\big]\\ &= \cos z\,u^\dagger(A - A^\dagger)v + \cos z\big[u^\dagger Pv' + u'^\dagger Pv\big]\\ &= (\partial_0\sin z)\,u^\dagger M_0 v + \sin z\big[u^\dagger M_0 v' + u'^\dagger M_0 v\big] = \partial_0\big(\sin z\,u^\dagger M_0 v\big).\end{aligned}$$

The first line inserts $h = A + P\partial_0$ and uses $(hu)^\dagger = u^\dagger A^\dagger + u'^\dagger P^\dagger$; the second uses fact 1; the third uses facts 3 and 2 and then recognises the product rule for the three factors $\sin z$, $u^\dagger$ and v (M_0 is constant). (PROVED; `python-scope.json`, check `good_sector_hermiticity_up_to_the_brane_flux`, both engines.) Integrated over the hidden direction, the left side is the failure of h to be symmetric for the norm $\int \cos z\,u^\dagger v\,dx_0$, and the right side gives the **boundary term** (or **flux**) $[\sin z\,u^\dagger M_0 v]$, taken between the tip and the patch end. At the tip $\sin z \to 0$; at the patch end $\sin z = 1$, and the flux need not vanish: for a column with $\gamma^{(4)}\gamma^{(0)}u = u$ (such columns exist, because $(\gamma^{(4)}\gamma^{(0)})^2 = -(\gamma^{(4)})^2(\gamma^{(0)})^2 = I_{16}$ and the matrix is not $\pm I_{16}$), $u^\dagger M_0 u = i\,u^\dagger u \neq 0$; the record's example has $u^\dagger u = 2$ and $u^\dagger M_0 u = 2i$.

The norm changes only through the patch end. For a solution of $i\,\partial_4\Psi = h\Psi$ let $N(x_4) = \int_0^{\pi/(12H)} \cos z\,\Psi^\dagger\Psi\,dx_0$. Line by line:

$$\begin{aligned}\frac{dN}{dx_4} &= \int \cos z\big[(\partial_4\Psi)^\dagger\Psi + \Psi^\dagger\partial_4\Psi\big]dx_0 = \int \cos z\big[(-ih\Psi)^\dagger\Psi + \Psi^\dagger(-ih\Psi)\big]dx_0 \\ &= -i\int \cos z\big[\Psi^\dagger(h\Psi) - (h\Psi)^\dagger\Psi\big]dx_0 = -i\big[\sin z\,\Psi^\dagger M_0\Psi\big]_{\text{tip}}^{\text{end}} \\ &= -i\,\Psi^\dagger M_0\Psi\big|_{z=\pi/2} = \Psi^\dagger\gamma^{(4)}\gamma^{(0)}\Psi\big|_{z=\pi/2}.\end{aligned}$$

The first line differentiates under the integral and inserts the equation; the second collects the factors $\mp i$ and uses the boundary identity with $u = v = \Psi$ (the integral of a derivative is the difference of its end values); the third uses $\sin z = 0$ at the tip and 1 at the end, and $-i \cdot i = 1$. The norm of a solution changes ONLY by the flux through the patch end. For a field that does not depend on x_0, $N = \Psi^\dagger\Psi/(6H)$. Notebook 08d follows such a growing mode at $m = H = 1$: its norm first dips (the flux is briefly negative) and then grows by a factor $N(1.5)/N(0) = 2094.786$ in 1.5 time units, and dN/dx_4 equals the flux at every time to rounding (COMPUTED).

What this means (labelled). Interpretation: the growth of the good-sector modes is fed through the patch end. A boundary condition at $z = \pi/2$ that stops the flux would make h symmetric and would change the spectrum; which condition is physical is OPEN. Without one, the initial-value problem of the good sector has growing solutions: this is the second way, besides the extra-time waves of Sections 8.17 to 8.25, in which the field equation as recorded fails to define a stable evolution.

8.33 Example: the rescaling and the modes of the hidden direction

Notebook 08d puts Sections 8.30 to 8.32 to work, exactly, with sympy. It takes the spin connection of the diagonal frame from the record's formula and verifies it (the gammas are covariantly constant); proves the rescaling for sixteen arbitrary component functions; draws the rescaling factor and the hidden coordinate y with the tip cutoff of the Kohn-Sham record; verifies the exact family for symbolic α, m, H and every a_4, and maps which members grow and which have a finite norm; checks the matrix A of the modes that do not depend on x_0; proves the boundary identity on explicit columns; and follows a growing mode whose norm changes exactly by the flux through the patch end. It needs no Rust and runs in one to two minutes; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/08d_rescaling_modes.PROVENANCE.md`. It ends with the line ALL 23 CHECKS PASSED (notebook 08d).

8.34 How to run Notebook 08d

Step 1. What this notebook does and what it needs.

Notebook 08d (The rescaling that removes the term, and the growing modes at the patch end) is the file `Revision/textbook/notebooks/08d_rescaling_modes.ipynb` of the repository Dirac_claude. It takes the spin connection of the diagonal frame from the Revision record and verifies it, proves exactly that the rescaling Psi = sin(z)^(-1/2) chi removes the term 3 H gamma^(x0) from the field equation (for 16 arbitrary component functions) and turns the quadratic potential into the

coupling lambda S/sin z, verifies the exact family of solutions sin(z)^alpha exp(M x4) chi0, finds which of them grow and which have a finite norm, verifies the growing x0-independent modes of the good sector, proves that the norm of the hidden direction changes only through a boundary term at the patch end z = pi/2, and draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
```

```
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08d_rescaling_modes.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08d_rescaling_modes.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 08d_rescaling_modes.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/08d.captions.json`
- `Revision/textbook/figures/08d_1_rescaling_factors.png`
- `Revision/textbook/figures/08d_2_family_k_squared.png`
- `Revision/textbook/figures/08d_3_growing_finite_norm.png`
- `Revision/textbook/figures/08d_4_patch_end_modes.png`
- `Revision/textbook/figures/08d_5_norm_and_flux.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08d_5_norm_and_flux.png exists
ALL 23 CHECKS PASSED (notebook 08d)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/08d_rescaling_modes.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 08d_rescaling_modes.ipynb
python -m nbconvert --execute --inplace 08d_rescaling_modes.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- A cell of section 6 or 7 runs for more than a minute: sympy simplifies the field equation for sixteen arbitrary functions there; on a slow computer this can take a few minutes. Wait until the star in the brackets to the left of the cell turns into a number.

8.35 Notebook 08d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/08d_rescaling_modes.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

In the diagonal frame the field equation of dirac16complex (and of dirac16complex00)
in the author's metric contains the term $3H\gamma^{(0)}\Psi$ (on the background
$b_4 = 1$ of the Revision record's scope reports, with which this notebook compares;
for a general $b_4(x_4)$ the term is $(3H\gamma^{(0)} + \frac34(b_4'/b_4)
\gamma^{(4)})\Psi$ and the rescaling $\sin^{-1/2}z\,b_4^{-3/4}$ removes it, Notebook
06b). This notebook shows,
exactly, that a change of the field VARIABLES removes it, and then studies the
hidden direction $x_0$ near its two ends: the tip $z \to 0$ and the patch end
$z = \pi/2$. It

- takes the spin connection of the diagonal frame from the Revision record and
  verifies it (the gammas are covariantly constant), and recomputes
  $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$;
- proves that for $\Psi = \sin^{-1/2}z\,\chi$ the field equation becomes
  $\gamma^\mu\partial_\mu\chi = (m + U')\chi$ with no spin-connection term, for
  sixteen arbitrary component functions, and that the quadratic potential turns
  into the coupling $\lambda S[\chi]/\sin z$;
- verifies the exact family of solutions $\Psi = \sin^\alpha z\,e^{Mx_4}\chi_0$ with
  $k^2 = 9H^2(2\alpha + 1)^2 - m^2$, and finds which members grow ($k^2 > 0$) and
  which have a finite norm at the tip ($\alpha > -\frac12$);
- verifies the growing $x_0$-independent modes of the good sector ($m < 3H$) and
  their finite norm;
- proves that the norm in the hidden direction changes only through a boundary
  term at the patch end $z = \pi/2$, and shows a growing mode whose norm grows
  exactly as fast as this boundary term feeds it.

Every exact statement reproduces a check of a Revision record or is labelled as
this notebook's own exact computation. Five teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 08d (The rescaling that removes the term, and the growing modes at the patch
end) is the file 'Revision/textbook/notebooks/08d_rescaling_modes.ipynb' of the
repository Dirac_claude. It takes the spin connection of the diagonal frame from the
Revision record and verifies it, proves exactly that the rescaling Psi = sin(z)^(-1/2)
chi removes the term 3 H gamma^(x0) from the field equation (for 16 arbitrary component
functions) and turns the quadratic potential into the coupling lambda S/sin z, verifies
the exact family of solutions sin(z)^alpha exp(M x4) chi0, finds which of them grow and
which have a finite norm, verifies the growing x0-independent modes of the good sector,
proves that the norm of the hidden direction changes only through a boundary term at the
patch end z = pi/2, and draws five teaching plots. It needs a computer with Windows 11,
macOS or Linux, an internet connection for the installation, the program Git, and Python
3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at
exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
Jupyter, the program that shows and runs notebooks. It does not need Rust.
```

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home

folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 08d_rescaling_modes.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 08d_rescaling_modes.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 08d_rescaling_modes.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/08d.captions.json‘
- ‘Revision/textbook/figures/08d_1_rescaling_factors.png‘

- ‘Revision/textbook/figures/08d_2_family_k_squared.png‘
- ‘Revision/textbook/figures/08d_3_growing_finite_norm.png‘
- ‘Revision/textbook/figures/08d_4_patch_end_modes.png‘
- ‘Revision/textbook/figures/08d_5_norm_and_flux.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 08d_5_norm_and_flux.png exists
ALL 23 CHECKS PASSED (notebook 08d)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/08d_rescaling_modes.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 08d_rescaling_modes.ipynb
  python -m nbconvert --execute --inplace 08d_rescaling_modes.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- A cell of section 6 or 7 runs for more than a minute: sympy simplifies the field equation for sixteen arbitrary functions there; on a slow computer this can take a few minutes. Wait until the star in the brackets to the left of the cell turns into a number.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 08d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 08d (The rescaling that removes the term, and the growing modes at the patch
# end) is the file Revision/textbook/notebooks/08d_rescaling_modes.ipynb of the
# repository Dirac_claude. It takes the spin connection of the diagonal frame from the
# Revision record and verifies it, proves exactly that the rescaling Psi = sin(z)^(-1/2)
# chi removes the term 3 H gamma^(x0) from the field equation (for 16 arbitrary component
# functions) and turns the quadratic potential into the coupling lambda S/sin z, verifies
# the exact family of solutions sin(z)^alpha exp(M x4) chi0, finds which of them grow and
# which have a finite norm, verifies the growing x0-independent modes of the good sector,
# proves that the norm of the hidden direction changes only through a boundary term at
# the patch end z = pi/2, and draws five teaching plots. It needs a computer with Windows
# 11, macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
```

```
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
```

```
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 08d_rescaling_modes.ipynb
```

```
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 08d_rescaling_modes.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 08d_rescaling_modes.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/08d.captions.json
# - Revision/textbook/figures/08d_1_rescaling_factors.png
# - Revision/textbook/figures/08d_2_family_k_squared.png
# - Revision/textbook/figures/08d_3_growing_finite_norm.png
# - Revision/textbook/figures/08d_4_patch_end_modes.png
# - Revision/textbook/figures/08d_5_norm_and_flux.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 08d_5_norm_and_flux.png exists
#     ALL 23 CHECKS PASSED (notebook 08d)
#
```

```
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/08d_rescaling_modes.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 08d_rescaling_modes.ipynb
#     python -m nbconvert --execute --inplace 08d_rescaling_modes.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - A cell of section 6 or 7 runs for more than a minute: sympy simplifies the field
#   equation for sixteen arbitrary functions there; on a slow computer this can take a
#   few minutes. Wait until the star in the brackets to the left of the cell turns into a
#   number.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
```

```
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "08d"  # this notebook: chapter 08, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
```

```
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 08d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ ordinary space (inflating);
  $x_4$ the time; $x_5, x_6, x_7$ the three extra times (time-like, DEFLATING);
  $x_0$ the hidden direction, $z = 6Hx_0$ between $0$ (the *tip*) and $\pi/2$ (the
  *patch end*, at $x_0 = \pi/(12H)$).
- **Diagonal frame**, scale factors $f_a$: $f_{1,2,3} = e^{a_4}\sin^{1/6}z$,
  $f_4 = 1$, $f_{5,6,7} = e^{-a_4}\sin^{1/6}z$, $f_0 = \cot z$; $\gamma^\mu =
  \gamma^{(\mu)}/f_\mu$.
- **Spin connection** $\Omega_\mu$ and **covariant derivative** $D_\mu\Psi =
  \partial_\mu\Psi + \Omega_\mu\Psi$; the gammas are *covariantly constant*:
  $\partial_\mu\gamma^\nu + \sum_\lambda\Gamma^\nu{}_{\mu\lambda}\gamma^\lambda +
  \Omega_\mu\gamma^\nu - \gamma^\nu\Omega_\mu = 0$ ($\Gamma$: Christoffel symbols).
  This property fixes $\Omega_\mu$ and is used to verify it.
- **Rescaling** (a change of field variables): $\Psi = w\,\chi$ with the function
  $w = \sin^{-1/2}z$.
- **Bilinear** $S = \bar\Psi\Psi = \Psi^\dagger C\Psi$ and the quadratic potential
  $U = \frac{\lambda}{2}S^2$, $U' = \lambda S$.
- **Norm** of a field in the hidden direction: $\int \cos z\,\Psi^\dagger\Psi\,dx_0$
  ($\cos z = \sqrt{|g|}$ is the volume factor). It is *finite* when the integral
  does not diverge at the ends.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$, $dy = \cot z\,dx_0$; $y = 0$ at the
  patch end and $y \to -\infty$ at the tip.
- **Good sector**: fields that do not depend on the extra times $x_5, x_6, x_7$.
- **Mode operator** $h$: the field equation written as $i\,\partial_4\Psi = h\Psi$;
  for fields that depend only on $x_4$ and $x_0$ (and $U = 0$),
  $h = -im\gamma^{(4)} + i\gamma^{(4)}\gamma^{(0)}(\tan z\,\partial_0 + 3H)$.
- **Hermitian, anti-Hermitian**: $M^\dagger = M$ or $M^\dagger = -M$
  ($M^\dagger$: the conjugate transpose).
- **Boundary term (flux)**: a total derivative $\partial_0(\dots)$ whose integral is
  the difference of its values at the two ends.
- **$Z_2$ brane**: a boundary surface of spacetime, here the patch end
  $z = \pi/2$, at which the patch is glued to a mirror copy of itself; the mirror
  fixes a boundary condition for the field there. The Kohn-Sham record ASSUMES
  one; this notebook does not use it.
```

Text cell:

```
## 4. The physical and mathematical situation

For a field that depends only on $x_4$ and $x_0$ the field equation (with $U = 0$)
is $\gamma^{(4)}\partial_4\Psi + \tan z\,\gamma^{(0)}\partial_0\Psi +
```

```
3H\gamma^{(0)}\Psi = m\Psi$. The rescaling works because, line by line,

1. $\partial_0\sin^{-1/2}z = -\frac12\sin^{-3/2}z\cos z \cdot 6H$ (chain rule, with
   $dz/dx_0 = 6H$);
2. so $\tan z\,\partial_0\sin^{-1/2}z = -3H\,\sin^{-1/2}z$ (because $\tan z\cos z =
   \sin z$);
3. so $\tan z\,\gamma^{(0)}\partial_0(w\chi) + 3H\gamma^{(0)}w\chi =
   w\tan z\,\gamma^{(0)}\partial_0\chi$: the product rule produces exactly
   $-3H\gamma^{(0)}w\chi$, which cancels the spin-connection term.

The weight moves into the measure: $\cos z\,\Psi^\dagger\Psi\,dx_0 = \cos z\,
\sin^{-1}z\,\chi^\dagger\chi\,dx_0 = \chi^\dagger\chi\,dy$.

Proper lengths along the hidden direction are $\sqrt{g_{00}}\,dx_0 = \cot z\,dx_0 =
dy$. So the *patch end* $z = \pi/2$ ($y = 0$) lies at the finite proper distance
$|y|$ from every point, while the tip $z \to 0$ ($y \to -\infty$) is infinitely far
away. The Revision
record imposes no boundary condition at $z = \pi/2$ (the Kohn-Sham record ASSUMES a
$Z_2$ brane there). Without one, the mode operator is symmetric only up to a
boundary term at $z = \pi/2$, and the good sector contains growing modes: this is
the second way (besides the extra-time waves) in which the initial-value problem
fails to be well posed.

Every symbol is exact ($H > 0$, $m$, $\alpha$, $\lambda$ letters; $a_4(x_4)$ any
function). The plots use $H = 1$.
```

Text cell:

```
## 5. The spin connection from the record, verified

The next cell first defines two helpers for the checks that reproduce a Revision
record. ‘record_says(report, name, ...)‘ opens the report (a JSON file with a list
of checks, each with a name, a verdict and a detail text) and is true when the
check ‘name‘ is there with the verdict pass and its detail text contains every
further piece of text given (for example a value that this notebook computes).
‘check_record(condition, title, report, name, ...)‘ is the helper ‘check‘ for such
a result: it passes only if the notebook’s own computation (‘condition‘) is right
AND the record says the same. Then the cell reads the gammas and the formula
record ‘Omega_components‘ of
‘Revision/theory/field-theory.json‘ (written in the Wolfram language: ‘g[xi]‘ is
$\gamma^{(x_i)}$, ‘a4’[x4]‘ is $a_4’$), enters the same formula in sympy, computes
the Christoffel symbols of the author’s metric, and checks that the gammas are
covariantly constant for all 64 pairs $(\mu, \nu)$ and that $\gamma^\mu\Omega_\mu =
3H\gamma^{(0)}$. The helper ‘is_zero‘ asks sympy to simplify twice (the second time
with its trigonometric simplifier ‘fu‘).
```

In [2]:

```
import numpy as np  # numbers and matrices (for the plots)
import sympy as sp  # exact algebra with symbols

REPORTS = {}  # report file -> {check name: (verdict, detail)}, each read once
```

```
def record_says(report_file, check_name, *pieces):
    """True when the Revision report records the check check_name with the verdict
    pass and its detail text contains every given piece of text."""
    if report_file not in REPORTS:  # read the report the first time it is needed
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        REPORTS[report_file] = {entry["name"]: (entry["verdict"].lower(),
                                                entry["detail"])
                                for entry in data["checks"]}
    verdict, detail = REPORTS[report_file][check_name]
    return verdict == "pass" and all(piece in detail for piece in pieces)


def check_record(condition, title, report_file, check_name, *pieces):
    """check() for a result that reproduces the Revision check check_name: it passes
    only if condition is true AND the report records check_name as passed, with
    every piece of text (values computed here) in its detail."""
    on_record = record_says(report_file, check_name, *pieces)
    check(condition and on_record, title,
          record=f"{report_file}, check {check_name}")


THEORY = "Revision/theory/reports/python-field-theory.json"  # the sympy records
SCOPE = "Revision/theory/reports/python-scope.json"
gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
G = [sp.Matrix(rows) for rows in gammas_record["gamma"]]  # gamma^(x0) ... (x7)
ETA = gammas_record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1]
C = sp.Matrix(gammas_record["C"])  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
B = sp.Matrix(gammas_record["B"]["re"]) + sp.I * sp.Matrix(gammas_record["B"]["im"])
theory = json.loads(
    repository_file("Revision/theory/field-theory.json").read_text(encoding="utf-8"))
formulas = {entry["key"]: entry for entry in theory["formulas"]}
say("record Omega_components: " + formulas["Omega_components"]["wl"])

x = sp.symbols("x0:8", real=True)  # the coordinates; x[k] is xk
H = sp.Symbol("H", positive=True)
m = sp.Symbol("m", real=True)
a4 = sp.Function("a4")(x[4])  # any function of the time x4
a4_prime = sp.diff(a4, x[4])
z = 6 * H * x[0]
sixth = sp.sin(z) ** sp.Rational(1, 6)
f = [sp.cot(z)] + [sp.exp(a4) * sixth] * 3 + [sp.Integer(1)] \
    + [sp.exp(-a4) * sixth] * 3  # the frame factors at b4 = 1
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal metric
gamma_up = [G[mu] / f[mu] for mu in range(8)]  # gamma^mu = gamma^(mu) / f_mu
Omega = [sp.zeros(16, 16) for _ in range(8)]
for i in (1, 2, 3):  # x1, x2, x3 (inflating)
    Omega[i] = sp.exp(a4) * sixth * (a4_prime * G[i] * G[4] + H * G[i] * G[0]) / 2
for t in (5, 6, 7):  # x5, x6, x7 (deflating extra times)
    Omega[t] = -sp.exp(-a4) * sixth * (a4_prime * G[4] * G[t] + H * G[t] * G[0]) / 2
```

```
def is_zero(expr):
    """True when sympy shows that expr is exactly zero (two simplifications)."""
    expr = sp.sympify(expr)
    if expr == 0:
        return True
    simpler = sp.simplify(expr)
    return simpler == 0 or sp.simplify(sp.fu(simpler)) == 0


Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for lam in range(8):  # Christoffel symbols of the diagonal metric
    for mu in range(8):
        for nu in range(8):
            value = 0
            if lam == nu:
                value += sp.diff(g[lam], x[mu])
            if lam == mu:
                value += sp.diff(g[lam], x[nu])
            if mu == nu:
                value -= sp.diff(g[mu], x[lam])
            if value != 0:
                Gam[lam][mu][nu] = sp.simplify(value / (2 * g[lam]))
constant = all(
    all(is_zero(v) for v in sp.diff(gamma_up[nu], x[mu])
        + sum((Gam[nu][mu][lam] * gamma_up[lam] for lam in range(8)), sp.zeros(16, 16))
        + Omega[mu] * gamma_up[nu] - gamma_up[nu] * Omega[mu])
    for mu in range(8) for nu in range(8))
check_record(constant, "the record's Omega_mu makes every gamma^nu covariantly "
             "constant (64 pairs)", THEORY, "covariant_constancy_D_mu_gamma_nu",
             "= 0 for all 64 (mu, nu)")
gamma_Omega = sum((gamma_up[mu] * Omega[mu] for mu in range(8)), sp.zeros(16, 16))
check_record(all(is_zero(v) for v in gamma_Omega - 3 * H * G[0]),
             "gamma^mu Omega_mu = 3 H gamma^(x0) (diagonal frame)",
             THEORY, "gamma_mu_Omega_mu_equals_3H_gamma_x0",
             "gamma^mu Omega_mu = 3 H gamma^(x0) exactly")
```

Out [2]:

```
record Omega_components: Omega_xi = (1/2) E^a4[x4] Sin[6 H x0]^(1/6) (a4'[x4] g[xi].g[x4]
    + H g[xi].g[x0]) (i = 1, 2, 3); Omega_xt = -(1/2) E^-a4[x4] Sin[6 H x0]^(1/6)
    (a4'[x4] g[x4].g[xt] + H g[xt].g[x0]) (t = 5, 6, 7); Omega_x0 = Omega_x4 = 0; g[xa] =
    gamma^(xa) (frame gammas of the fixture)
PASS the record's Omega_mu makes every gamma^nu covariantly constant (64 pairs)
     reproduces Revision/theory/reports/python-field-theory.json, check
    covariant_constancy_D_mu_gamma_nu
PASS gamma^mu Omega_mu = 3 H gamma^(x0) (diagonal frame)
     reproduces Revision/theory/reports/python-field-theory.json, check
    gamma_mu_Omega_mu_equals_3H_gamma_x0
```

Text cell:

```
## 6. The rescaling removes the term
```

```
The next cell checks the one-line identity $\tan z\,\partial_0\sin^{-1/2}z =
-3H\sin^{-1/2}z$, and then the full statement on sixteen ARBITRARY component
functions $\chi_A(x_0, \dots, x_7)$ (sympy ‘Function‘ objects, about which nothing
is assumed): $\gamma^\mu D_\mu(w\chi) - w\,\gamma^\mu\partial_\mu\chi = 0$ for all 16
components. Then it checks the potential: $S[w\chi] = w^2S[\chi] = S[\chi]/\sin z$
($w$ is real), so $\lambda S[\Psi]\Psi = w\,\frac{\lambda S[\chi]}{\sin z}\chi$, and
the measure: $\cos z\,w^2 = \cot z = dy/dx_0$.
```

In [3]:

```
w = sp.sin(z) ** sp.Rational(-1, 2)  # the rescaling factor sin(z)^(-1/2)
check_record(is_zero(sp.tan(z) * sp.diff(w, x[0]) + 3 * H * w),
             "tan z d0 sin(z)^(-1/2) = -3 H sin(z)^(-1/2)", SCOPE,
             "rescaling_removes_the_connection_term",
             "(tan z (-3 H sin^(-1/2) z) + 3 H sin^(-1/2) z) gamma^(x0) chi = 0")
chi = sp.Matrix([sp.Function(f"chi{A}")(*x) for A in range(1, 17)])  # arbitrary
Psi = w * chi
D_Psi = sum((gamma_up[mu] * (sp.diff(Psi, x[mu]) + Omega[mu] * Psi)
             for mu in range(8)), sp.zeros(16, 1))  # gamma^mu D_mu Psi
D_chi = sum((gamma_up[mu] * sp.diff(chi, x[mu]) for mu in range(8)),
            sp.zeros(16, 1))  # gamma^mu d_mu chi (no connection)
check_record(all(is_zero(v) for v in D_Psi - w * D_chi),
             "gamma^mu D_mu (w chi) = w gamma^mu d_mu chi for 16 arbitrary functions",
             SCOPE, "rescaling_removes_the_connection_term",
             "the field equation has no spin-connection term: gamma^mu d_mu chi = "
             "(m + U'(S)) chi")
lam, S_chi = sp.symbols("lambda S_chi", real=True)  # S_chi stands for S[chi]
S_Psi = w**2 * S_chi  # S[w chi] = (w chi)^dagger C (w chi) = w^2 S[chi]
check_record(is_zero(lam * S_Psi - lam * S_chi / sp.sin(z)),
             "U = (lambda/2) S^2: lambda S[Psi] = lambda S[chi]/sin z (the new "
             "coupling)", SCOPE, "rescaled_equation_quadratic_potential",
             "gamma^mu d_mu chi = (m + lambda S[chi]/sin z) chi")
y_of_x0 = sp.log(sp.sin(z)) / (6 * H)
check(is_zero(sp.cos(z) * w**2 - sp.cot(z)) and
      is_zero(sp.diff(y_of_x0, x[0]) - sp.cot(z)),
      "cos z w^2 = cot z = dy/dx0: the norm of Psi is the y-norm of chi")
```

Out [3]:

```
PASS tan z d0 sin(z)^(-1/2) = -3 H sin(z)^(-1/2)
     reproduces Revision/theory/reports/python-scope.json, check
    rescaling_removes_the_connection_term
PASS gamma^mu D_mu (w chi) = w gamma^mu d_mu chi for 16 arbitrary functions
     reproduces Revision/theory/reports/python-scope.json, check
    rescaling_removes_the_connection_term
PASS U = (lambda/2) S^2: lambda S[Psi] = lambda S[chi]/sin z (the new coupling)
     reproduces Revision/theory/reports/python-scope.json, check
    rescaled_equation_quadratic_potential
PASS cos z w^2 = cot z = dy/dx0: the norm of Psi is the y-norm of chi
```

Text cell:

```
The next cell draws the rescaling factor $w = \sin^{-1/2}z$, the new coupling factor
$1/\sin z$ of the potential, and the hidden coordinate $y = \ln(\sin z)/(6H)$, for
$H = 1$. It reads $H$ and the tip cutoff $L$ of the Kohn-Sham record (that record
cuts the hidden direction at $y = -L$) and computes where the cutoff lies in $z$:
$y = -L$ means $\ln(\sin z) = -6HL$, so $\sin z = e^{-6HL}$ and
$z_{\rm cut} = \arcsin(e^{-6HL})$. This number is tiny, so the right panel uses a
logarithmic $z$ axis: on it $y$ is almost a straight line, because $\sin z \approx
z$ for small $z$ and then $y \approx \ln(z)/(6H)$.
```

In [4]:

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
H_record = parameters["physics"]["H"]  # the author's constant H of the record
L_tip = parameters["physics"]["L_tipCutoff"]  # the record cuts the tip at y = -L
z_cut = np.arcsin(np.exp(-6 * H_record * L_tip))  # the z at which y = -L
report("H and the tip cutoff L of the Kohn-Sham record", f"{H_record}, {L_tip}")
report("z at the tip cutoff, arcsin(e^(-6 H L))", f"{z_cut:.6e}")
check(H_record == 1.0 and abs(np.log(np.sin(z_cut)) / (6 * H_record) + L_tip) < 1e-12,
      "the record has H = 1, and y(z_cut) = -L",
      record="Revision/kohn_sham/results/parameters.json, physics.H, "
             "physics.L_tipCutoff")
zs = np.linspace(0.01, np.pi / 2, 400)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(zs, np.sin(zs) ** -0.5, label="rescaling factor $\\sin^{-1/2}z$")
left.plot(zs, 1 / np.sin(zs), "--", label="coupling factor $1/\\sin z$")
left.set_yscale("log")
left.set_xlabel("$z = 6Hx_0$ (tip at 0, patch end at $\\pi/2$)")
left.set_ylabel("factor")
left.set_title("The rescaling $\\Psi = \\sin^{-1/2}z\\,\\chi$")
left.legend()
mantissa, exponent = f"{z_cut:.1e}".split("e")  # "1.5e-08" -> "1.5" and "-08"
exponent = int(exponent)  # -8, for the caption
z_log = np.logspace(-10.0, np.log10(np.pi / 2), 400)  # 10^-10 ... pi/2, log spaced
right.semilogx(z_log, np.log(np.sin(z_log)) / (6 * H_record), color="tab:green")
right.axhline(-L_tip, color="gray", linestyle=":", linewidth=0.9)
right.plot([z_cut], [-L_tip], "o", color="black")  # where y reaches the cutoff
right.text(2e-8, -L_tip + 0.15, "tip cutoff $y = -L$ of the Kohn-Sham record",
           fontsize=8)
right.set_xlabel("$z = 6Hx_0$ (logarithmic axis)")
right.set_ylabel("$y = \\ln(\\sin z)/(6H)$")
right.set_title("The hidden coordinate $y$, $H = 1$")
save_figure(fig, "rescaling_factors",
            "Left, on a logarithmic axis, against $z = 6Hx_0$ from the tip (near 0) "
            "to the patch end ($\\pi/2$): the rescaling factor $\\sin^{-1/2}z$ "
            "(solid) of $\\Psi = \\sin^{-1/2}z\\,\\chi$, which removes the term "
            "$3H\\gamma^{(0)}$, and the factor $1/\\sin z$ (dashed) of the new "
            "coupling $\\lambda S_\\chi/\\sin z$; both equal 1 at the patch end and "
            "diverge at the tip. Right: the hidden coordinate $y = \\ln(\\sin z)/"
            "(6H)$ for $H = 1$, in which the norm of $\\Psi$ is $\\int\\chi^\\dagger"
            "\\chi\\,dy$, against $z$ on a logarithmic axis; $y = 0$ at the patch "
            "end and $y \\to -\\infty$ at the tip. Dotted: the cutoff "
            f"$y = -L = {-L_tip:g}$ of the Kohn-Sham record, reached only at "
```

```
        f"$z = \\arcsin(e^{{{-6 * H_record * L_tip:g}}}) \\approx {mantissa} "
        f"\\times 10^{{{exponent}}}$ (black dot): almost the whole range of $y$ "
        "lies in a tiny neighbourhood of the tip.")
```

Out [4]:

```
RESULT H and the tip cutoff L of the Kohn-Sham record = 1.0, 3.0
RESULT z at the tip cutoff, arcsin(e^(-6 H L)) = 1.522998e-08
PASS the record has H = 1, and y(z_cut) = -L
     reproduces Revision/kohn_sham/results/parameters.json, physics.H,
    physics.L_tipCutoff
```

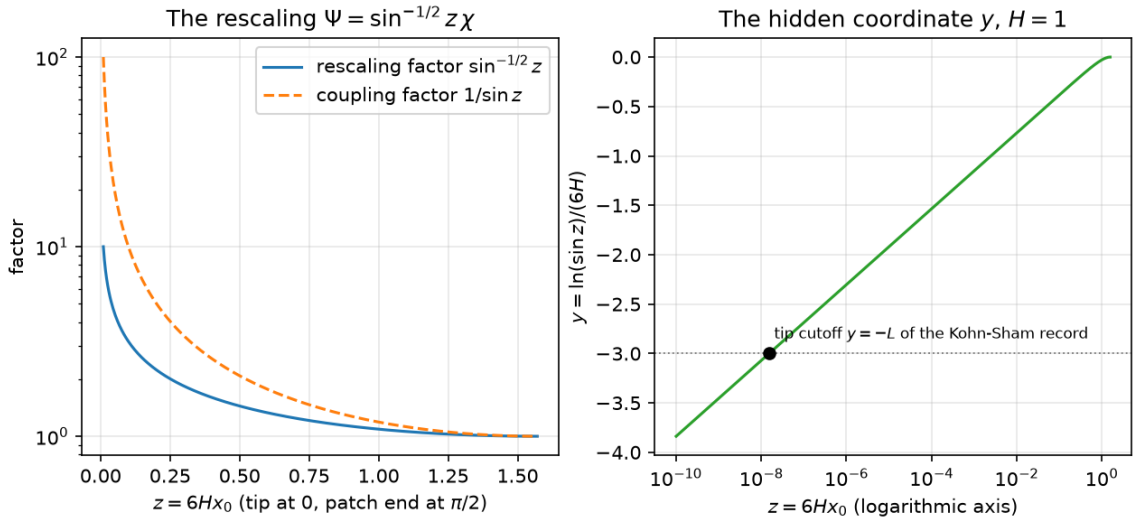

Figure 245: Left, on a logarithmic axis, against $z = 6Hx_0$ from the tip (near 0) to the patch end ($\pi/2$): the rescaling factor $\sin^{-1/2} z$ (solid) of $\Psi = \sin^{-1/2} z\,\chi$, which removes the term $3H\gamma^{(0)}$, and the factor $1/\sin z$ (dashed) of the new coupling $\lambda S_\chi/\sin z$; both equal 1 at the patch end and diverge at the tip. Right: the hidden coordinate $y = \ln(\sin z)/(6H)$ for $H = 1$, in which the norm of Ψ is $\int \chi^\dagger \chi\, dy$, against z on a logarithmic axis; $y = 0$ at the patch end and $y \to -\infty$ at the tip. Dotted: the cutoff $y = -L = -3$ of the Kohn-Sham record, reached only at $z = \arcsin(e^{-18}) \approx 1.5 \times 10^{-8}$ (black dot): almost the whole range of y lies in a tiny neighbourhood of the tip. (Notebook 08d, figure 1.)

Out [4]:

```
Figure 08d.1 saved as Revision/textbook/figures/08d_1_rescaling_factors.png
```

Text cell:

```
## 7. An exact family of solutions

For $U = 0$ and a field that depends only on $x_4$ and $x_0$ the record gives the
family $\Psi = \sin^\alpha z\,\big(\cosh(kx_4) + \frac{\sinh(kx_4)}{k}M\big)\chi_0$
```

```
with a constant column $\chi_0$, $M = -m\gamma^{(4)} + c\,\gamma^{(4)}\gamma^{(0)}$,
$c = 3H(2\alpha + 1)$ and $k^2 = c^2 - m^2$. The bracket is $e^{Mx_4}$, because
$M^2 = k^2 I_{16}$. The next cell checks $M^2 = (c^2 - m^2)I_{16}$ and
$M^TC + CM = 0$ (so $S$ does not change in time), and then that $\Psi$ solves the
full field equation $\gamma^\mu D_\mu\Psi = m\Psi$ (all eight directions, with the
spin connection of section 5) for symbolic $\alpha$, $m$, $H$ and every $a_4$.
For $\alpha = -\frac12$: $c = 0$, $M = -m\gamma^{(4)}$ contains no $H$, and
$k^2 = -m^2$ (oscillation with frequency $m$): this is the rescaling of section 6
at work.
```

In [5]:

```
alpha = sp.Symbol("alpha", real=True)
c = 3 * H * (2 * alpha + 1)
k = sp.sqrt(c**2 - m**2)  # k^2 = 9 H^2 (2 alpha + 1)^2 - m^2
M = -m * G[4] + c * G[4] * G[0]
check_record((M * M - (c**2 - m**2) * sp.eye(16)).applyfunc(sp.expand)
             == sp.zeros(16, 16)
             and (M.T * C + C * M).applyfunc(sp.expand) == sp.zeros(16, 16),
             "M^2 = (9 H^2 (2 alpha + 1)^2 - m^2) I16 and M^T C + C M = 0",
             THEORY, "exact_solution_family_x4_x0",
             "M = -m gamma^(x4) + 3 H (2 alpha + 1) gamma^(x4) gamma^(x0)",
             "k^2 = 9 H^2 (2 alpha + 1)^2 - m^2", "(M^2 = k^2 I16, M^T C + C M = 0")
chi0 = sp.Matrix(sp.symbols("q1:17"))  # 16 arbitrary constants
family = sp.sin(z) ** alpha * (sp.cosh(k * x[4]) * chi0
                               + sp.sinh(k * x[4]) / k * (M * chi0))
residual = sum((gamma_up[mu] * (sp.diff(family, x[mu]) + Omega[mu] * family)
                for mu in range(8)), sp.zeros(16, 1)) - m * family
check_record(all(is_zero(v) for v in residual),
             "Psi = sin(z)^alpha (cosh(k x4) + sinh(k x4)/k M) chi0 solves the field "
             "equation", THEORY, "exact_solution_family_x4_x0",
             "solves gamma^mu D_mu Psi = m Psi exactly for every a4(x4) and every "
             "alpha")
M_half = M.subs(alpha, -sp.Rational(1, 2))
check_record(not M_half.has(H)
             and sp.expand(k.subs(alpha, -sp.Rational(1, 2)) ** 2) == -m**2,
             "alpha = -1/2: M = -m gamma^(x4) has no H and k^2 = -m^2 (oscillation)",
             SCOPE, "rescaled_equation_quadratic_potential",
             "= -m gamma^(x4) and k^2 = -m^2 (no H)")
```

Out [5]:

```
PASS M^2 = (9 H^2 (2 alpha + 1)^2 - m^2) I16 and M^T C + C M = 0
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_solution_family_x4_x0
PASS Psi = sin(z)^alpha (cosh(k x4) + sinh(k x4)/k M) chi0 solves the field equation
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_solution_family_x4_x0
PASS alpha = -1/2: M = -m gamma^(x4) has no H and k^2 = -m^2 (oscillation)
     reproduces Revision/theory/reports/python-scope.json, check
    rescaled_equation_quadratic_potential
```

Text cell:

```
## 8. Which members grow, and which have a finite norm

A member GROWS when $k^2 > 0$, that is when $3H|2\alpha + 1| > m$. Its norm in the
hidden direction is $\int_0^{\pi/(12H)}\cos z\,\sin^{2\alpha}z\,dx_0$ times a
function of $x_4$; with $s = \sin z$, $ds = 6H\cos z\,dx_0$, this is
$\frac{1}{6H}\int_0^1 s^{2\alpha}ds$, which is $\frac{1}{6H(2\alpha + 1)}$ for
$\alpha > -\frac12$ and infinite for $\alpha \le -\frac12$ (at $\alpha = -\frac12$
the integral of $1/s$ diverges like $\ln$ at the tip). So the H-free member
$\alpha = -\frac12$ has an infinite norm, and the growing members with a finite norm
are those with $2\alpha + 1 > m/(3H)$. For EVERY mass $m > 0$ there are such
members: $\alpha = m/(3H)$ gives $k^2 = (2m + 3H)^2 - m^2 = 3(m + H)(m + 3H) > 0$.
The Revision record states the $x_0$-independent case $\alpha = 0$, which grows when
$m < 3H$; the statement for every mass is this notebook's own exact consequence of
the record's family. The next cell checks the integrals and this example, and draws
$k^2$ against $\alpha$ for three masses.
```

In [6]:

```
s, p, eps = sp.symbols("s p epsilon", positive=True)
check(sp.integrate(s ** (p - 1), (s, 0, 1)) == 1 / p and
      sp.limit(sp.integrate(1 / s, (s, eps, 1)), eps, 0, "+") == sp.oo,
      "int_0^1 s^(2 alpha) ds = 1/(2 alpha + 1) for 2 alpha + 1 > 0; diverges at -1/2")
k2_every_mass = sp.factor(sp.expand((c**2 - m**2).subs(alpha, m / (3 * H))))
say(f"k^2 at alpha = m/(3H): {k2_every_mass}")
check(sp.expand(k2_every_mass - 3 * (m + H) * (m + 3 * H)) == 0,
      "alpha = m/(3H) > -1/2 gives k^2 = 3 (m + H)(m + 3H) > 0 for every m > 0")

alphas = np.linspace(-2.0, 1.0, 601)
fig, ax = plt.subplots()
for mass, style in ((1.0, "-"), (2.0, "--"), (4.0, ":")):
    ax.plot(alphas, 9 * (2 * alphas + 1) ** 2 - mass**2, style, label=f"$m = {mass:g}H$")
    ax.plot([-0.5, 0.0], [-mass**2, 9 - mass**2], "o", color="black", markersize=4)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.axvspan(-2.0, -0.5, color="gray", alpha=0.15)
ax.text(-1.4, 66, "infinite norm\nat the tip", fontsize=8)
ax.text(0.05, 66, "grows where\n$k^2 > 0$", fontsize=8)
ax.set_ylim(-20.0, 80.0)
ax.set_xlabel("exponent $\\alpha$ of $\\sin^{\\alpha}z$")
ax.set_ylabel("$k^2 = 9H^2(2\\alpha + 1)^2 - m^2$ (units of $H^2$)")
ax.set_title("The exact family $\\sin^\\alpha z\\,e^{Mx_4}\\chi_0$, $H = 1$")
ax.legend(loc="upper center");
save_figure(fig, "family_k_squared",
            "The number $k^2 = 9H^2(2\\alpha + 1)^2 - m^2$ of the exact family of "
            "solutions $\\sin^\\alpha z\\,e^{Mx_4}\\chi_0$ against the exponent "
            "$\\alpha$, for the masses $m = H, 2H, 4H$ ($H = 1$; vertical axis in "
            "units of $H^2$). Where $k^2 > 0$ the member grows like $e^{kx_4}$, "
            "where $k^2 < 0$ it oscillates. The black dots mark $\\alpha = -1/2$ "
            "($k^2 = -m^2$, no $H$: the rescaled field) and $\\alpha = 0$ ($k^2 = "
            "9H^2 - m^2$, the $x_0$-independent modes). In the gray region "
            "$\\alpha \\le -1/2$ the norm diverges at the tip; to the right of it "
```

```
        "every parabola becomes positive, so for every mass some member with a "
        "finite norm grows.")
```

Out [6]:

```
PASS int_0^1 s^(2 alpha) ds = 1/(2 alpha + 1) for 2 alpha + 1 > 0; diverges at -1/2
k^2 at alpha = m/(3H): 3*(H + m)*(3*H + m)
PASS alpha = m/(3H) > -1/2 gives k^2 = 3 (m + H)(m + 3H) > 0 for every m > 0
```

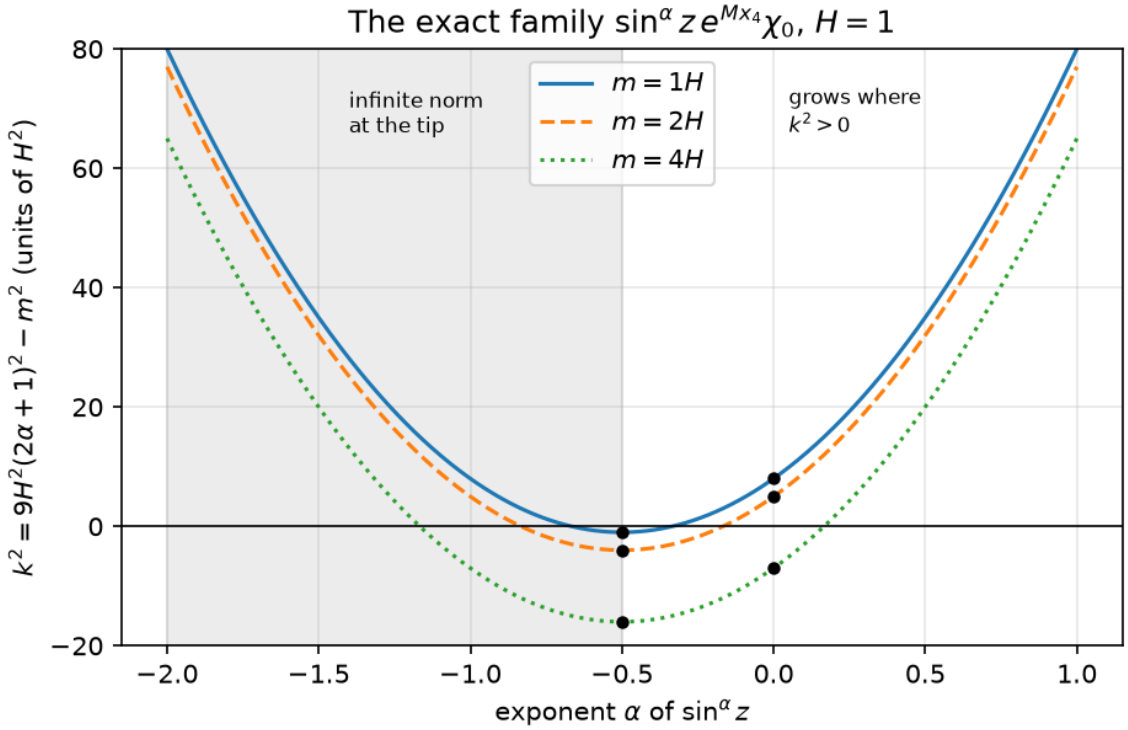

Figure 246: The number $k^2 = 9H^2(2\alpha + 1)^2 - m^2$ of the exact family of solutions $\sin^\alpha z\, e^{Mx_4}\chi_0$ against the exponent α, for the masses $m = H, 2H, 4H$ ($H = 1$; vertical axis in units of H^2). Where $k^2 > 0$ the member grows like e^{kx_4}, where $k^2 < 0$ it oscillates. The black dots mark $\alpha = -1/2$ ($k^2 = -m^2$, no H: the rescaled field) and $\alpha = 0$ ($k^2 = 9H^2 - m^2$, the x_0-independent modes). In the gray region $\alpha \le -1/2$ the norm diverges at the tip; to the right of it every parabola becomes positive, so for every mass some member with a finite norm grows. (Notebook 08d, figure 2.)

Out [6]:

```
Figure 08d.2 saved as Revision/textbook/figures/08d_2_family_k_squared.png
```

Text cell:

```
The next cell draws the same information in the plane of $\alpha$ and the mass
$m/H$: the region of growing members with a finite norm is $\alpha > -\frac12$ and
$2\alpha + 1 > m/(3H)$. The horizontal line $\alpha = 0$ crosses it exactly for
$m < 3H$, the case stated in the Revision record.
```

In [7]:

```
masses = np.linspace(0.0, 6.0, 301)
A_grid, M_grid = np.meshgrid(np.linspace(-1.0, 1.5, 251), masses)
grows = 9 * (2 * A_grid + 1) ** 2 - M_grid**2 > 0
finite = A_grid > -0.5
region = np.where(grows & finite, 2, np.where(grows, 1, 0))  # 2, 1 or 0
fig, ax = plt.subplots(figsize=(6.4, 4.6))
ax.contourf(M_grid, A_grid, region, levels=[-0.5, 0.5, 1.5, 2.5],
            colors=["white", "#d9d9d9", "#9ecae1"])
ax.plot(masses, (masses / 3 - 1) / 2, color="black", linewidth=1.0)
ax.axhline(-0.5, color="gray", linestyle="--", linewidth=0.9)
ax.axhline(0.0, color="tab:red", linewidth=1.2)
ax.plot([3.0], [0.0], "o", color="tab:red")
ax.text(0.2, 1.2, "grows, finite norm", fontsize=9)
ax.text(4.2, -0.2, "oscillates", fontsize=9)
ax.text(0.2, -0.9, "grows, infinite norm", fontsize=9)
ax.text(3.1, 0.05, "$\\alpha = 0$: grows for $m < 3H$", fontsize=8, color="tab:red")
ax.set_xlabel("mass $m$ (units of $H$)")
ax.set_ylabel("exponent $\\alpha$")
ax.set_title("Members of the family that grow and have a finite norm")
fraction = float(np.mean(grows & finite))
report("fraction of the drawn (m, alpha) rectangle that grows with finite norm",
       f"{fraction:.4f}")
check(all(np.any(grows[i] & finite[i]) for i in range(len(masses))),
      "for every drawn mass some alpha gives a growing finite-norm member")
save_figure(fig, "growing_finite_norm",
            "The plane of the mass $m$ (horizontal, units of $H$) and the exponent "
            "$\\alpha$ (vertical) of the exact family $\\sin^\\alpha z\\,"
            "e^{Mx_4}\\chi_0$. Blue: the member grows ($k^2 > 0$) and has a finite "
            "norm at the tip ($\\alpha > -1/2$, above the dashed line). Gray: it "
            "grows but its norm diverges. White: it oscillates. The black line is "
            "$2\\alpha + 1 = m/(3H)$. The red line $\\alpha = 0$ (the $x_0$-"
            "independent modes) is blue exactly for $m < 3H$; above the black line "
            "every mass has growing finite-norm members.")
```

Out [7]:

```
RESULT fraction of the drawn (m, alpha) rectangle that grows with finite norm = 0.5996
PASS for every drawn mass some alpha gives a growing finite-norm member
```

Out [7]:

```
Figure 08d.3 saved as Revision/textbook/figures/08d_3_growing_finite_norm.png
```

Text cell:

```
## 9. The $x_0$-independent modes of the good sector

On fields that depend only on $x_4$ (good sector, $\alpha = 0$) the mode operator
```

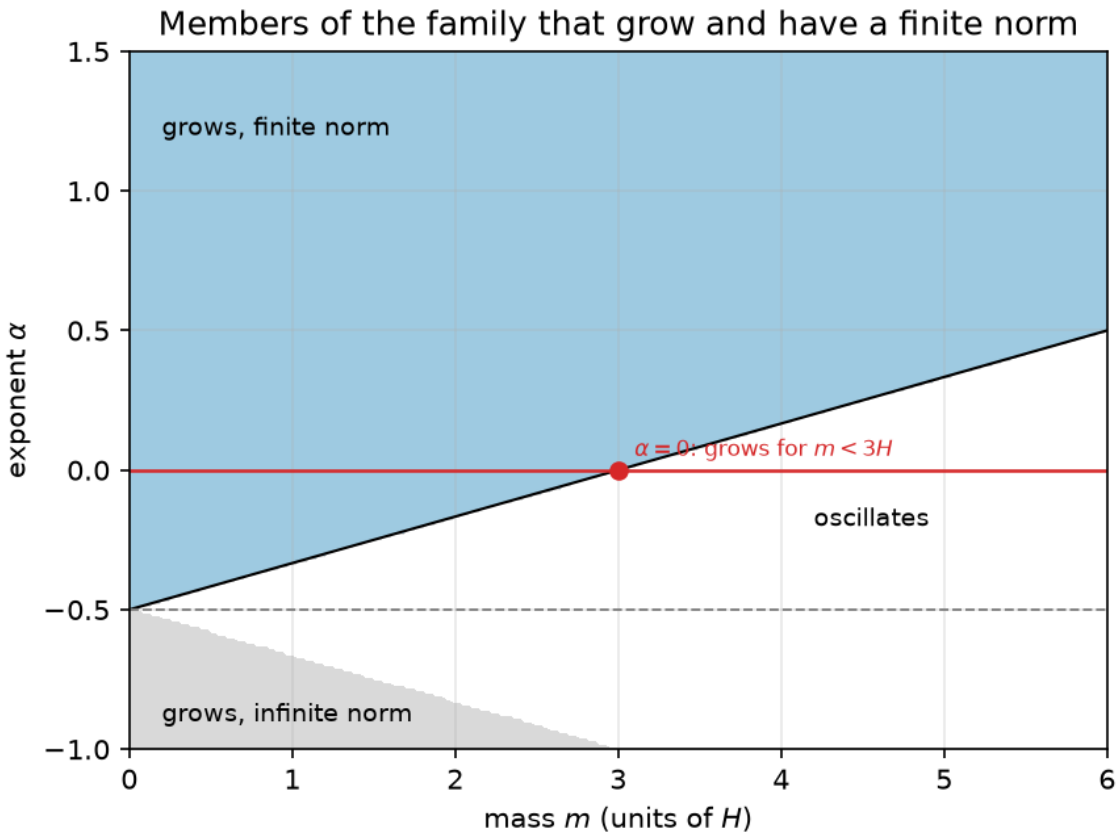

Figure 247: The plane of the mass m (horizontal, units of H) and the exponent α (vertical) of the exact family $\sin^{\alpha} z\, e^{Mx_4}\chi_0$. Blue: the member grows ($k^2 > 0$) and has a finite norm at the tip ($\alpha > -1/2$, above the dashed line). Gray: it grows but its norm diverges. White: it oscillates. The black line is $2\alpha + 1 = m/(3H)$. The red line $\alpha = 0$ (the x_0-independent modes) is blue exactly for $m < 3H$; above the black line every mass has growing finite-norm members. (Notebook 08d, figure 3.)

```
acts as the matrix $A = -im\gamma^{(4)} + 3iH\gamma^{(4)}\gamma^{(0)}$, and
$i\,\partial_4\Psi = A\Psi$ is the same as $\partial_4\Psi = M\Psi$ at $\alpha = 0$,
because $A = iM$. The next cell checks $A = iM|_{\alpha = 0}$, that $A$ is not
Hermitian, that $A^2 = (m^2 - 9H^2)I_{16}$, that at $m = H = 1$ the eigenvalues are
$\pm 2\sqrt2\,i$ (eight each; a solution with $e^{-i(2\sqrt2 i)x_4} = e^{2\sqrt2
x_4}$ grows), and that such a mode has the finite norm factor
$\int_0^{\pi/(12H)}\cos(6Hx_0)\,dx_0 = \frac{1}{6H}$.
```

In [8]:

```
A = -sp.I * m * G[4] + 3 * sp.I * H * G[4] * G[0]
A_squared = (A * A).applyfunc(sp.expand)
A_one = A.subs({m: 1, H: 1})
dimension = 16 - (A_one - 2 * sp.sqrt(2) * sp.I * sp.eye(16)).rank()
norm_factor = sp.integrate(sp.cos(6 * H * x[0]), (x[0], 0, sp.pi / (12 * H)))
report("dimension of the eigenspace of +2 sqrt(2) i at m = H = 1", dimension)
report("int_0^(pi/(12H)) cos(6 H x0) dx0", norm_factor)
all_true = ((A - sp.I * M.subs(alpha, 0)).applyfunc(sp.expand) == sp.zeros(16, 16)
            and A != A.H
            and A_squared == ((m**2 - 9 * H**2) * sp.eye(16)).applyfunc(sp.expand)
            and A_one * A_one == -8 * sp.eye(16) and A_one.trace() == 0
            and dimension == 8 and sp.simplify(norm_factor - 1 / (6 * H)) == 0)
# The record states the same matrix, square, eigenvalues and norm in its text:
check_record(all_true,
             "A = -i m g4 + 3 i H g4 g0: not Hermitian, A^2 = (m^2 - 9H^2) I16, "
             "eigenvalues +-2 sqrt(2) i at m = H = 1, finite norm 1/(6H)",
             SCOPE, "good_sector_x0_independent_modes_without_boundary_condition",
             f"cos(6 H x0) dx0 = {sp.sstr(norm_factor)})",
             "A = -i m gamma^(x4) + 3 i H gamma^(x4) gamma^(x0)",
             "A^2 = (m^2 - 9 H^2) I16 exactly",
             f"the eigenvalues are +-2 sqrt(2) i ({dimension} each)")
```

Out [8]:

```
RESULT dimension of the eigenspace of +2 sqrt(2) i at m = H = 1 = 8
RESULT int_0^(pi/(12H)) cos(6 H x0) dx0 = 1/(6*H)
PASS A = -i m g4 + 3 i H g4 g0: not Hermitian, A^2 = (m^2 - 9H^2) I16, eigenvalues +-2
    sqrt(2) i at m = H = 1, finite norm 1/(6H)
     reproduces Revision/theory/reports/python-scope.json, check
    good_sector_x0_independent_modes_without_boundary_condition
```

Text cell:

```
The next cell draws, for $H = 1$, the two eigenvalues $\pm\sqrt{m^2 - 9H^2}$ of $A$
in the complex plane as the mass grows from 0 to 5 (left), and the growth rates
$k = \sqrt{9H^2(2\alpha + 1)^2 - m^2}$ of three members of the family against the
mass (right).
```

In [9]:

```
A_number = sp.lambdify((m, H), A, "numpy")
mass_path = np.linspace(0.0, 5.0, 51)
points = np.concatenate([np.linalg.eigvals(np.array(A_number(mv, 1.0), dtype=complex))
                         for mv in mass_path])
largest_growth = max(abs(points.imag))
report("largest |imaginary part| on the path (reached at m = 0)",
       f"{largest_growth:.9f}")
check(abs(largest_growth - 3.0) < 1e-9, "at m = 0 the growth rate is 3H")
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
dots = left.scatter(points.real, points.imag, c=np.repeat(mass_path, 16),
                    cmap="plasma", s=20)
fig.colorbar(dots, ax=left, label="mass $m$ (units of $H$)")
left.set_aspect("equal")
left.set_xlim(-4.5, 4.5)
left.set_ylim(-4.5, 4.5)
left.set_xlabel("real part")
left.set_ylabel("imaginary part")
left.set_title("Eigenvalues of $A$, $H = 1$")
mass_fine = np.linspace(0.0, 5.0, 501)  # a finer grid for the smooth curves
for a_value, style in ((0.0, "-"), (0.5, "--"), (1.0, ":")):
    rate = np.sqrt(np.maximum(9 * (2 * a_value + 1) ** 2 - mass_fine**2, 0.0))
    right.plot(mass_fine, rate, style, label=f"$\\alpha = {a_value:g}$")
right.axvline(3.0, color="gray", linewidth=0.8)
right.set_xlabel("mass $m$ (units of $H$)")
right.set_ylabel("growth rate $k$ (units of $H$)")
right.set_title("Growth rates of the family")
right.legend();
save_figure(fig, "patch_end_modes",
            "Left: the eigenvalues $\\pm\\sqrt{m^2 - 9H^2}$ of the matrix $A = "
            "-im\\gamma^{(4)} + 3iH\\gamma^{(4)}\\gamma^{(0)}$, which acts on the "
            "$x_0$-independent fields of the good sector, in the complex plane, "
            "coloured by the mass $m$ from 0 to $5H$ ($H = 1$). For $m < 3H$ they "
            "lie on the imaginary axis (the mode $e^{-iEx_4}$ grows, at most like "
            "$e^{3Hx_4}$ at $m = 0$); at $m = 3H$ they meet at 0; for $m > 3H$ they "
            "are real. Right: the growth rate $k = \\sqrt{9H^2(2\\alpha + 1)^2 - "
            "m^2}$ of the members $\\alpha = 0, 0.5, 1$ of the exact family against "
            "$m$; members with larger $\\alpha$ still grow when $m > 3H$ (gray "
            "line).")
```

Out [9]:

```
RESULT largest |imaginary part| on the path (reached at m = 0) = 3.000000000
PASS at m = 0 the growth rate is 3H
```

Out [9]:

```
Figure 08d.4 saved as Revision/textbook/figures/08d_4_patch_end_modes.png
```

Text cell:

```
## 10. The boundary term at the patch end
```

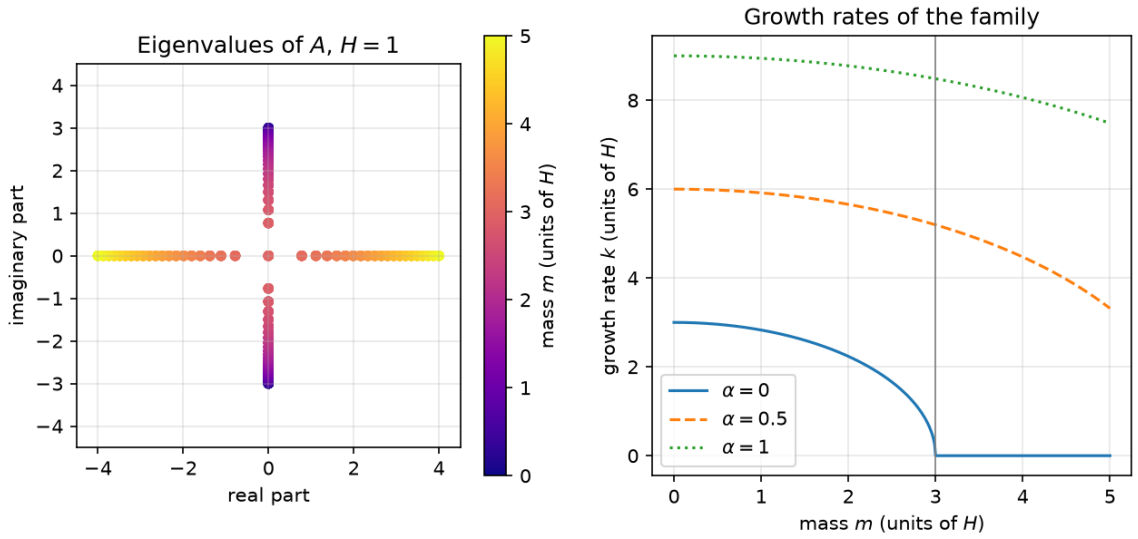

Figure 248: Left: the eigenvalues $\pm\sqrt{m^2 - 9H^2}$ of the matrix $A = -im\gamma^{(4)} + 3iH\gamma^{(4)}\gamma^{(0)}$, which acts on the x_0-independent fields of the good sector, in the complex plane, coloured by the mass m from 0 to $5H$ ($H = 1$). For $m < 3H$ they lie on the imaginary axis (the mode e^{-iEx_4} grows, at most like e^{3Hx_4} at $m = 0$); at $m = 3H$ they meet at 0; for $m > 3H$ they are real. Right: the growth rate $k = \sqrt{9H^2(2\alpha + 1)^2 - m^2}$ of the members $\alpha = 0, 0.5, 1$ of the exact family against m; members with larger α still grow when $m > 3H$ (gray line). (Notebook 08d, figure 4.)

```
For fields of $x_4$ and $x_0$ the mode operator is $h = -im\gamma^{(4)} +
i\gamma^{(4)}\gamma^{(0)}(\tan z\,\partial_0 + 3H)$. With $M_0 = i\gamma^{(4)}
\gamma^{(0)}$ the record proves, for all columns $u(x_0)$, $v(x_0)$,
$$\cos z\,\big[u^\dagger(hv) - (hu)^\dagger v\big] = \partial_0\big(\sin z\,
u^\dagger M_0 v\big).$$
It follows from three matrix facts: $\cos z\cdot(i\tan z\,\gamma^{(4)}\gamma^{(0)})
= \sin z\,M_0$ (the coefficient of $\partial_0$); $\cos z\,(A - A^\dagger) =
(\partial_0\sin z)\,M_0$ for the algebraic part $A$; and the coefficient of
$\partial_0$ is anti-Hermitian. The next cell checks the three facts, the identity
itself for two explicit columns of polynomials with complex coefficients, and the
value at the patch end: for $u$ with $\gamma^{(4)}\gamma^{(0)}u = u$,
$u^\dagger M_0u = i\,u^\dagger u \neq 0$. So $h$ is symmetric for the norm
$\int\cos z\,u^\dagger v\,dx_0$ only up to $[\sin z\,u^\dagger M_0v]$, which
vanishes at the tip ($\sin z \to 0$) but not at $z = \pi/2$ ($\sin z = 1$).
```

In [10]:

```
M0 = sp.I * G[4] * G[0]
d0_coefficient = sp.I * sp.tan(z) * G[4] * G[0]  # the matrix in front of d0 in h
fact_1 = all(is_zero(v) for v in sp.cos(z) * d0_coefficient - sp.sin(z) * M0)
fact_2 = all(is_zero(v) for v in sp.cos(z) * (A - A.H) - sp.diff(sp.sin(z), x[0]) * M0)
fact_3 = (d0_coefficient + d0_coefficient.H).applyfunc(sp.simplify) == sp.zeros(16, 16)
X0 = x[0]
u_col = sp.Matrix([(A_ + 1) + sp.I * (2 * A_ - 3) * X0 + (A_ - 5) * X0**2 / 3
                   for A_ in range(16)])  # a column of polynomials in x0
v_col = sp.Matrix([(3 - A_) * X0 + sp.I * (A_ + 2) + sp.I * A_ * X0**3 / 7
```

```
                  for A_ in range(16)])


def h_of(column):
    """The mode operator h applied to a column of functions of x0 (U = 0)."""
    return -sp.I * m * G[4] * column + sp.I * G[4] * G[0] * (
        sp.tan(z) * sp.diff(column, X0) + 3 * H * column)


left_side = sp.cos(z) * ((u_col.H * h_of(v_col))[0] - (h_of(u_col).H * v_col)[0])
right_side = sp.diff(sp.sin(z) * (u_col.H * M0 * v_col)[0], X0)
u0 = (G[4] * G[0] - sp.eye(16)).nullspace()[0]  # gamma^(4) gamma^(0) u0 = u0
flux = sp.simplify((u0.H * M0 * u0)[0])
size_u0 = (u0.H * u0)[0]
report("u0^dagger M0 u0 and u0^dagger u0", f"{flux}, {size_u0}")
# The record states the same identity and the same flux (sympy writes i as I):
check_record(fact_1 and fact_2 and fact_3 and is_zero(left_side - right_side)
             and flux == 2 * sp.I and size_u0 == 2,
             "cos z (u^dagger h v - (h u)^dagger v) = d0(sin z u^dagger M0 v); at "
             "z = pi/2 the flux u0^dagger M0 u0 = 2 i is not zero",
             SCOPE, "good_sector_hermiticity_up_to_the_brane_flux",
             "cos z [u^dagger (h v) - (h u)^dagger v] = d_x0(sin z u^dagger M0 v)",
             f"u^dagger M0 u = {sp.sstr(flux)} (|u|^2 = {sp.sstr(size_u0)})")
```

Out [10]:

```
RESULT u0^dagger M0 u0 and u0^dagger u0 = 2*I, 2
PASS cos z (u^dagger h v - (h u)^dagger v) = d0(sin z u^dagger M0 v); at z = pi/2 the
    flux u0^dagger M0 u0 = 2 i is not zero
     reproduces Revision/theory/reports/python-scope.json, check
    good_sector_hermiticity_up_to_the_brane_flux
```

Text cell:

```
## 11. A growing mode is fed through the patch end

Integrate the identity of section 10 over the hidden direction. For a solution of
$i\,\partial_4\Psi = h\Psi$ the norm $N(x_4) = \int_0^{\pi/(12H)}\cos z\,
\Psi^\dagger\Psi\,dx_0$ changes, line by line, as
$\frac{dN}{dx_4} = \int\cos z\,\big[(-ih\Psi)^\dagger\Psi + \Psi^\dagger(-ih\Psi)
\big]dx_0 = -i\int\cos z\,\big[\Psi^\dagger h\Psi - (h\Psi)^\dagger\Psi\big]dx_0
= -i\big[\sin z\,\Psi^\dagger M_0\Psi\big]_{\rm tip}^{\rm end} =
\Psi^\dagger\gamma^{(4)}\gamma^{(0)}\Psi\big|_{z = \pi/2}$:
the norm changes ONLY through the patch end. The next cell follows the
$x_0$-independent mode $\Psi = e^{-iAx_4}\chi_0$ with $m = H = 1$ (so
$N = \Psi^\dagger\Psi/(6H)$) from a fixed random column $\chi_0$, computes
$dN/dx_4 = 2\,\mathrm{Re}\big(\Psi^\dagger(-iA\Psi)\big)/(6H)$ and the flux
$\Psi^\dagger\gamma^{(4)}\gamma^{(0)}\Psi$, checks that they agree, and draws them.
Interpretation (labelled): the growth is fed through the patch end; a boundary
condition at $z = \pi/2$ that stops this flux would change the spectrum, and which
condition is physical is a question this notebook does not decide.
```

In [11]:

```
A_num = np.array(A_number(1.0, 1.0), dtype=complex)  # m = H = 1
g4g0 = np.array(G[4] * G[0], dtype=float)  # real symmetric
rng = np.random.default_rng(12345)  # fixed seed: the same column in every run
chi_start = rng.normal(size=16) + 1j * rng.normal(size=16)
chi_start = chi_start / np.linalg.norm(chi_start)
omega = np.sqrt(complex(1.0 - 9.0))  # A^2 = (m^2 - 9 H^2) = -8: omega = 2 sqrt(2) i
times = np.linspace(0.0, 1.5, 151)
norms, slopes, fluxes = [], [], []
for x4 in times:
    state = np.cos(omega * x4) * chi_start \
        - 1j * np.sin(omega * x4) / omega * (A_num @ chi_start)  # e^(-i A x4) chi
    norms.append(np.vdot(state, state).real / 6.0)  # N = Psi^dagger Psi / (6 H)
    slopes.append(2 * np.vdot(state, -1j * (A_num @ state)).real / 6.0)  # dN/dx4
    fluxes.append(np.vdot(state, g4g0 @ state).real)  # flux at z = pi/2
norms, slopes, fluxes = map(np.array, (norms, slopes, fluxes))
agreement = np.max(np.abs(slopes - fluxes) / np.maximum(np.abs(fluxes), 1.0))
check(agreement < 1e-12, "dN/dx4 equals the flux through the patch end at every x4")
report("N(1.5) / N(0)", f"{norms[-1] / norms[0]:.6e}")
check(norms[-1] / norms[0] > 100.0, "the norm grows by more than a factor 100")

fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.semilogy(times, norms)
left.set_xlabel("time $x_4$ (units of $1/H$)")
left.set_ylabel("norm $N(x_4)$")
left.set_title("The $x_0$-independent mode, $m = H = 1$")
right.plot(times, slopes, label="$dN/dx_4$")
right.plot(times[::10], fluxes[::10], "o", color="black", markersize=4,
           label="flux at $z = \\pi/2$")
right.axhline(0.0, color="black", linewidth=0.8)
right.set_yscale("symlog", linthresh=1.0)  # linear between -1 and 1, log outside
right.set_ylim(-1.5, 3.0e3)
right.set_xlabel("time $x_4$ (units of $1/H$)")
right.set_ylabel("rate (units of $H$)")
right.set_title("Rate of change of the norm = flux")
right.legend();
save_figure(fig, "norm_and_flux",
            "An $x_0$-independent mode of the good sector, $\\Psi = e^{-iAx_4}"
            "\\chi_0$ with $m = H = 1$ and a fixed random start $\\chi_0$, against "
            "the time $x_4$ in units of $1/H$. Left, on a logarithmic axis: its "
            "norm $N = \\int\\cos z\\,\\Psi^\\dagger\\Psi\\,dx_0$, which first dips "
            "and then grows like $e^{4\\sqrt2\\,x_4}$. Right, on an axis that is "
            "linear between $-1$ and $1$ and logarithmic outside: the rate of "
            "change $dN/dx_4$ (line) and the flux $\\Psi^\\dagger\\gamma^{(4)}"
            "\\gamma^{(0)}\\Psi$ through the patch end $z = \\pi/2$ (dots). They "
            "coincide at every time, also in the short first phase in which the "
            "flux is negative and the norm decreases: the norm changes only "
            "through the patch end, where the Revision record imposes no boundary "
            "condition.")
```

Out [11]:

```
PASS dN/dx4 equals the flux through the patch end at every x4
RESULT N(1.5) / N(0) = 2.094786e+03
PASS the norm grows by more than a factor 100
```

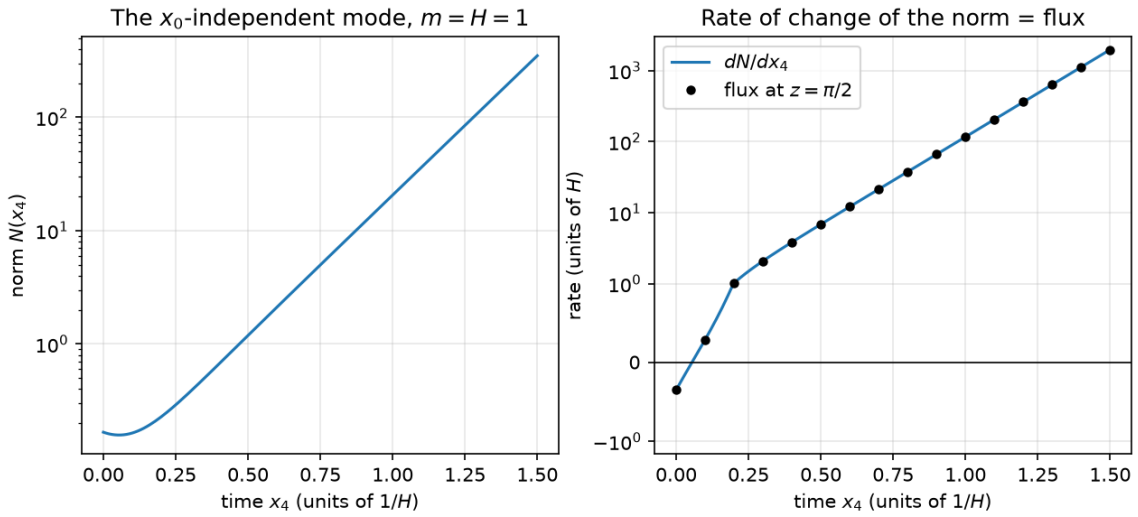

Figure 249: An x_0-independent mode of the good sector, $\Psi = e^{-iAx_4}\chi_0$ with $m = H = 1$ and a fixed random start χ_0, against the time x_4 in units of $1/H$. Left, on a logarithmic axis: its norm $N = \int \cos z\, \Psi^\dagger \Psi\, dx_0$, which first dips and then grows like $e^{4\sqrt{2}\,x_4}$. Right, on an axis that is linear between -1 and 1 and logarithmic outside: the rate of change dN/dx_4 (line) and the flux $\Psi^\dagger \gamma^{(4)} \gamma^{(0)} \Psi$ through the patch end $z = \pi/2$ (dots). They coincide at every time, also in the short first phase in which the flux is negative and the norm decreases: the norm changes only through the patch end, where the Revision record imposes no boundary condition. (Notebook 08d, figure 5.)

Out [11]:

```
Figure 08d.5 saved as Revision/textbook/figures/08d_5_norm_and_flux.png
```

Text cell:

```
## 12. The last check

The last cell checks that the five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [12]:

```
for name in ("08d_1_rescaling_factors.png", "08d_2_family_k_squared.png",
             "08d_3_growing_finite_norm.png", "08d_4_patch_end_modes.png",
             "08d_5_norm_and_flux.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [12]:

```
PASS the figure file 08d_1_rescaling_factors.png exists
PASS the figure file 08d_2_family_k_squared.png exists
PASS the figure file 08d_3_growing_finite_norm.png exists
PASS the figure file 08d_4_patch_end_modes.png exists
PASS the figure file 08d_5_norm_and_flux.png exists
ALL 23 CHECKS PASSED (notebook 08d)
```

Text cell:

```
## 13. What this notebook showed

- The spin connection of the record makes the gammas covariantly constant, and
  $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$ in the diagonal frame (PROVED).
- The rescaling $\Psi = \sin^{-1/2}z\,\chi$ removes this term exactly: $\gamma^\mu
  D_\mu\Psi = \sin^{-1/2}z\,\gamma^\mu\partial_\mu\chi$ for every $\chi$, so for
  $U = 0$ the equation for $\chi$ has no spin-connection term, and for $U =
  \frac{\lambda}{2}S^2$ the term reappears as the $x_0$-dependent coupling
  $\lambda S[\chi]/\sin z$ (PROVED). The weight moves into the measure: the norm of
  $\Psi$ is $\int\chi^\dagger\chi\,dy$. Together with the frame dependence of
  the term, this is the exact scope of the statement "the field equation contains
  $3H\gamma^{(0)}$": it belongs to one frame AND one choice of field variables.
- The exact family $\sin^\alpha z\,e^{Mx_4}\chi_0$ solves the field equation for
  every $a_4$ (PROVED); its member $\alpha = -\frac12$ contains no $H$. Members with
  $3H|2\alpha + 1| > m$ grow; members with $\alpha > -\frac12$ have a finite norm at
  the tip; for every mass some member does both (PROVED here; the record states the
  case $\alpha = 0$, $m < 3H$).
- The $x_0$-independent modes of the good sector grow for $m < 3H$ (at $m = H = 1$
  with rate $2\sqrt2$) and have a finite norm (PROVED; Revision check
  `good_sector_x0_independent_modes_without_boundary_condition`).
- The mode operator is symmetric only up to the boundary term $[\sin z\,u^\dagger
  M_0v]$, which does not vanish at the patch end $z = \pi/2$ (PROVED), and the norm
  of the growing mode changes exactly by the flux through the patch end (COMPUTED,
  to rounding). Without a boundary condition at $z = \pi/2$ the initial-value
  problem of the good sector therefore has growing solutions; which boundary
  condition is physical is OPEN here (the Kohn-Sham record ASSUMES a $Z_2$ brane).
```

8.36 Line-by-line walk-through of Notebook 08d

The notebook has 12 code cells, In [1] to In [12]. The conventions are those of Section 8.15; captions shortened to `...` are printed in full in Section 8.35.

In [1], the set-up cell. It is the set-up cell explained in Section 8.15, with the line `NOTEBOOK_ID = "08d" # this notebook: chapter 08, example d`; its comment lines are the run instructions of Section 8.34. Output: `Set-up of notebook 08d complete: repository folder found, helpers defined.`

In [2], the spin connection from the record, verified. The cell starts with the imports of numpy and sympy, the empty table `REPORTS` and the helpers `record_says` and `check_record`, word for word those of Notebook 08a (Section 8.23; `check_record` takes one check name). Then:

```
THEORY = "Revision/theory/reports/python-field-theory.json"  # the sympy records
SCOPE = "Revision/theory/reports/python-scope.json"
gammas_record = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
G = [sp.Matrix(rows) for rows in gammas_record["gamma"]]  # gamma^(x0) ... (x7)
ETA = gammas_record["eta"]  # [1, 1, 1, 1, -1, -1, -1, -1]
C = sp.Matrix(gammas_record["C"])  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
B = sp.Matrix(gammas_record["B"]["re"]) + sp.I * sp.Matrix(gammas_record["B"]["im"])
```

Short names for the paths of the two sympy reports, and the gammas, the signs, C and B as exact sympy matrices.

```
theory = json.loads(
    repository_file("Revision/theory/field-theory.json").read_text(encoding="utf-8"))
formulas = {entry["key"]: entry for entry in theory["formulas"]}
say("record Omega_components: " + formulas["Omega_components"]["wl"])
```

The formula file of the field-theory record holds a list `formulas`, each with a `key` and the formula as text; the dictionary comprehension makes a table key: entry. The `say` line prints the record's formula of the spinor connection in the Wolfram language (`g[xi]` is $\gamma^{(x_i)}$, `a4'[x4]` is a_4', `E^a4[x4]` is e^{a_4}), which the next lines enter in sympy.

```
x = sp.symbols("x0:8", real=True)  # the coordinates; x[k] is xk
H = sp.Symbol("H", positive=True)
m = sp.Symbol("m", real=True)
a4 = sp.Function("a4")(x[4])  # any function of the time x4
a4_prime = sp.diff(a4, x[4])
z = 6 * H * x[0]
sixth = sp.sin(z) ** sp.Rational(1, 6)
f = [sp.cot(z)] + [sp.exp(a4) * sixth] * 3 + [sp.Integer(1)] \
    + [sp.exp(-a4) * sixth] * 3  # the frame factors at b4 = 1
g = [ETA[a] * f[a] ** 2 for a in range(8)]  # the diagonal metric
gamma_up = [G[mu] / f[mu] for mu in range(8)]  # gamma^mu = gamma^(mu) / f_mu
```

The symbols, the unknown history $a_4(x_4)$ and its derivative, z, $\sin^{1/6} z$, the scale factors, the diagonal metric and the coordinate gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ of the diagonal frame, as in Notebook 08c.

```
Omega = [sp.zeros(16, 16) for _ in range(8)]
for i in (1, 2, 3):  # x1, x2, x3 (inflating)
    Omega[i] = sp.exp(a4) * sixth * (a4_prime * G[i] * G[4] + H * G[i] * G[0]) / 2
for t in (5, 6, 7):  # x5, x6, x7 (deflating extra times)
    Omega[t] = -sp.exp(-a4) * sixth * (a4_prime * G[4] * G[t] + H * G[t] * G[0]) / 2
```

The record's formula entered in sympy: $\Omega_{x_i} = \frac{1}{2} e^{a_4} \sin^{1/6} z (a_4' \gamma^{(i)} \gamma^{(4)} + H \gamma^{(i)} \gamma^{(0)})$, $\Omega_{x_t} = -\frac{1}{2} e^{-a_4} \sin^{1/6} z (a_4' \gamma^{(4)} \gamma^{(t)} + H \gamma^{(t)} \gamma^{(0)})$, and zero for x_4 and x_0 (the starting zero matrices are kept). These are the formulas derived in Section 8.4.

```
def is_zero(expr):
    ...
Gam = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for lam in range(8):  # Christoffel symbols of the diagonal metric
    ...
                Gam[lam][mu][nu] = sp.simplify(value / (2 * g[lam]))
```

`is_zero` and the Christoffel symbols are computed exactly as in Notebook 08c (In [2] and In [3]; the lines shortened to `...` here are the same lines, explained in Section 8.15).

```
constant = all(
    all(is_zero(v) for v in sp.diff(gamma_up[nu], x[mu])
        + sum((Gam[nu][mu][lam] * gamma_up[lam] for lam in range(8)), sp.zeros(16, 16))
        + Omega[mu] * gamma_up[nu] - gamma_up[nu] * Omega[mu])
    for mu in range(8) for nu in range(8))
check_record(constant, "the record's Omega_mu makes every gamma^nu covariantly "
             "constant (64 pairs)", THEORY, "covariant_constancy_D_mu_gamma_nu",
             "= 0 for all 64 (mu, nu)")
```

For all 64 pairs (μ, ν) the covariant derivative of the coordinate gamma, $\partial_\mu \gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + \Omega_\mu \gamma^\nu - \gamma^\nu \Omega_\mu$, is computed and every entry must vanish. This verifies the record's connection independently of the definition used in Notebook 08c: covariant constancy fixes the connection (Section 8.9, Step 4).

```
gamma_Omega = sum((gamma_up[mu] * Omega[mu] for mu in range(8)), sp.zeros(16, 16))
check_record(all(is_zero(v) for v in gamma_Omega - 3 * H * G[0]),
             "gamma^mu Omega_mu = 3 H gamma^(x0) (diagonal frame)",
             THEORY, "gamma_mu_Omega_mu_equals_3H_gamma_x0",
             "gamma^mu Omega_mu = 3 H gamma^(x0) exactly")
```

The sum $\gamma^\mu \Omega_\mu$, compared with $3H\gamma^{(0)}$. Output: the formula line and two PASS lines with their `reproduces` lines.

In [3], the rescaling removes the term.

```
w = sp.sin(z) ** sp.Rational(-1, 2)  # the rescaling factor sin(z)^(-1/2)
check_record(is_zero(sp.tan(z) * sp.diff(w, x[0]) + 3 * H * w),
             "tan z d0 sin(z)^(-1/2) = -3 H sin(z)^(-1/2)", SCOPE,
             "rescaling_removes_the_connection_term",
             "(tan z (-3 H sin^(-1/2) z) + 3 H sin^(-1/2) z) gamma^(x0) chi = 0")
```

The one-line identity $\tan z \, \partial_0 w = -3Hw$ of Section 8.30, with the record's text of the same identity.

```
chi = sp.Matrix([sp.Function(f"chi{A}")(*x) for A in range(1, 17)])  # arbitrary
Psi = w * chi
D_Psi = sum((gamma_up[mu] * (sp.diff(Psi, x[mu]) + Omega[mu] * Psi)
             for mu in range(8)), sp.zeros(16, 1))  # gamma^mu D_mu Psi
D_chi = sum((gamma_up[mu] * sp.diff(chi, x[mu]) for mu in range(8)),
            sp.zeros(16, 1))  # gamma^mu d_mu chi (no connection)
```

```
check_record(all(is_zero(v) for v in D_Psi - w * D_chi),
             "gamma^mu D_mu (w chi) = w gamma^mu d_mu chi for 16 arbitrary functions",
             SCOPE, "rescaling_removes_the_connection_term",
             "the field equation has no spin-connection term: gamma^mu d_mu chi = "
             "(m + U'(S)) chi")
```

`chi` is a column of sixteen unknown functions $\chi_1, \ldots, \chi_{16}$ of all eight coordinates (`*x` passes the eight symbols as arguments); nothing is assumed about them. `Psi` is $w\chi$; `D_Psi` is $\gamma^\mu D_\mu \Psi$ with the connection, `D_chi` is $\gamma^\mu \partial_\mu \chi$ without it (both start from a 16×1 zero column). The check requires $\gamma^\mu D_\mu(w\chi) - w\gamma^\mu \partial_\mu \chi = 0$ in all 16 components, the statement of Section 8.30 for every field.

```
lam, S_chi = sp.symbols("lambda S_chi", real=True)  # S_chi stands for S[chi]
S_Psi = w**2 * S_chi  # S[w chi] = (w chi)^dagger C (w chi) = w^2 S[chi]
check_record(is_zero(lam * S_Psi - lam * S_chi / sp.sin(z)),
             "U = (lambda/2) S^2: lambda S[Psi] = lambda S[chi]/sin z (the new "
             "coupling)", SCOPE, "rescaled_equation_quadratic_potential",
             "gamma^mu d_mu chi = (m + lambda S[chi]/sin z) chi")
y_of_x0 = sp.log(sp.sin(z)) / (6 * H)
check(is_zero(sp.cos(z) * w**2 - sp.cot(z)) and
      is_zero(sp.diff(y_of_x0, x[0]) - sp.cot(z)),
      "cos z w^2 = cot z = dy/dx0: the norm of Psi is the y-norm of chi")
```

The potential: with a symbol `S_chi` for $S[\chi]$, $\lambda S[\Psi] = \lambda w^2 S[\chi]$ is compared with $\lambda S[\chi]/\sin z$. The measure: $\cos z\, w^2 = \cot z = dy/dx_0$. Output: four PASS lines, three with `reproduces` lines.

In [4], the rescaling factors and the hidden coordinate.

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
H_record = parameters["physics"]["H"]  # the author's constant H of the record
L_tip = parameters["physics"]["L_tipCutoff"]  # the record cuts the tip at y = -L
z_cut = np.arcsin(np.exp(-6 * H_record * L_tip))  # the z at which y = -L
report("H and the tip cutoff L of the Kohn-Sham record", f"{H_record}, {L_tip}")
report("z at the tip cutoff, arcsin(e^(-6 H L))", f"{z_cut:.6e}")
check(H_record == 1.0 and abs(np.log(np.sin(z_cut)) / (6 * H_record) + L_tip) < 1e-12,
      "the record has H = 1, and y(z_cut) = -L",
      record="Revision/kohn_sham/results/parameters.json, physics.H, "
             "physics.L_tipCutoff")
```

H and the tip cutoff L are read from the Kohn-Sham record. The cutoff $y = -L$ means $\ln(\sin z) = -6HL$, so $\sin z = e^{-6HL}$ and $z_{\mathrm{cut}} = \arcsin(e^{-6HL})$ (`np.arcsin` is the inverse sine). The RESULT lines print `1.0, 3.0` and $z_{\mathrm{cut}} = 1.522998 \times 10^{-8}$; the check confirms $H = 1$ and that $y(z_{\mathrm{cut}}) = -L$ to 10^{-12}.

```
zs = np.linspace(0.01, np.pi / 2, 400)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(zs, np.sin(zs) ** -0.5, label="rescaling factor $\\sin^{-1/2}z$")
left.plot(zs, 1 / np.sin(zs), "--", label="coupling factor $1/\\sin z$")
left.set_yscale("log")
left.set_xlabel("$z = 6Hx_0$ (tip at 0, patch end at $\\pi/2$)")
```

```
left.set_ylabel("factor")
left.set_title("The rescaling $\\Psi = \\sin^{-1/2}z\\,\\chi$")
left.legend()
```

The left panel draws the two factors $\sin^{-1/2} z$ and $1/\sin z$ for 400 values of z from 0.01 to $\pi/2$ on a logarithmic axis.

```
mantissa, exponent = f"{z_cut:.1e}".split("e")  # "1.5e-08" -> "1.5" and "-08"
exponent = int(exponent)  # -8, for the caption
z_log = np.logspace(-10.0, np.log10(np.pi / 2), 400)  # 10^-10 ... pi/2, log spaced
right.semilogx(z_log, np.log(np.sin(z_log)) / (6 * H_record), color="tab:green")
right.axhline(-L_tip, color="gray", linestyle=":", linewidth=0.9)
right.plot([z_cut], [-L_tip], "o", color="black")  # where y reaches the cutoff
right.text(2e-8, -L_tip + 0.15, "tip cutoff $y = -L$ of the Kohn-Sham record",
           fontsize=8)
```

The cutoff value is written with one decimal in powers of ten and split at the letter `e` into the two parts that the caption prints (`1.5` and `-8`). The right panel draws $y = \ln(\sin z)/(6H)$ for 400 values of z from 10^{-10} to $\pi/2$, equally spaced on a logarithmic axis (`semilogx`; `np.log10` is the base-10 logarithm), a dotted line at $y = -L$, a black dot where y reaches it, and a label.

```
right.set_xlabel("$z = 6Hx_0$ (logarithmic axis)")
right.set_ylabel("$y = \\ln(\\sin z)/(6H)$")
right.set_title("The hidden coordinate $y$, $H = 1$")
save_figure(fig, "rescaling_factors",
            "Left, on a logarithmic axis, against $z = 6Hx_0$ from the tip (near 0) "
            ...
            "lies in a tiny neighbourhood of the tip.")
```

Labels, a title, and Figure 08d.1; the caption is an f-string, so the values read from the record ($y = -3$, $\arcsin(e^{-18}) \approx 1.5 \times 10^{-8}$) are put in when the cell runs. Output: two RESULT lines, one PASS line with its `reproduces` line, the figure line.

What Figure 08d.1 shows. Left: the two factors (pure numbers, logarithmic vertical axis) against z from 0.01 to $\pi/2$: the rescaling factor $\sin^{-1/2} z$ falls from 10 to 1, the coupling factor $1/\sin z$ from 100 to 1; both equal 1 at the patch end and grow without bound towards the tip. Right: the hidden coordinate y (vertical, units of $1/H$) against z on a logarithmic axis from 10^{-10} to $\pi/2$: an almost straight line from about -3.84 up to 0 (because $\sin z \approx z$ for small z, $y \approx \ln(z)/6$ is a straight line on this axis), with the cutoff $y = -3$ reached at $z \approx 1.5 \times 10^{-8}$. The student should see that almost the whole range of y used by the Kohn-Sham record lies in a tiny neighbourhood of the tip.

In [5], the exact family.

```
alpha = sp.Symbol("alpha", real=True)
c = 3 * H * (2 * alpha + 1)
k = sp.sqrt(c**2 - m**2)  # k^2 = 9 H^2 (2 alpha + 1)^2 - m^2
M = -m * G[4] + c * G[4] * G[0]
check_record((M * M - (c**2 - m**2) * sp.eye(16)).applyfunc(sp.expand)
             == sp.zeros(16, 16)
```

```
            and (M.T * C + C * M).applyfunc(sp.expand) == sp.zeros(16, 16),
            "M^2 = (9 H^2 (2 alpha + 1)^2 - m^2) I16 and M^T C + C M = 0",
            THEORY, "exact_solution_family_x4_x0",
            "M = -m gamma^(x4) + 3 H (2 alpha + 1) gamma^(x4) gamma^(x0)",
            "k^2 = 9 H^2 (2 alpha + 1)^2 - m^2", "(M^2 = k^2 I16, M^T C + C M = 0")
```

The exponent α, the number $c = 3H(2\alpha+1)$, k and the matrix M of Section 8.31. The check confirms $M^2 = (c^2 - m^2)I_{16}$ (derived in Section 8.31) and $M^T C + CM = 0$; the second property means that the scalar $S = \Psi^\dagger C \Psi$ of a member does not change in time, because $\frac{d}{dx_4}(e^{M^T x_4} C e^{M x_4}) = e^{M^T x_4}(M^T C + CM)e^{M x_4} = 0$ (M is real). The record's detail must contain the three texts.

```
chi0 = sp.Matrix(sp.symbols("q1:17"))  # 16 arbitrary constants
family = sp.sin(z) ** alpha * (sp.cosh(k * x[4]) * chi0
                               + sp.sinh(k * x[4]) / k * (M * chi0))
residual = sum((gamma_up[mu] * (sp.diff(family, x[mu]) + Omega[mu] * family)
                for mu in range(8)), sp.zeros(16, 1)) - m * family
check_record(all(is_zero(v) for v in residual),
            "Psi = sin(z)^alpha (cosh(k x4) + sinh(k x4)/k M) chi0 solves the field "
            "equation", THEORY, "exact_solution_family_x4_x0",
            "solves gamma^mu D_mu Psi = m Psi exactly for every a4(x4) and every "
            "alpha")
```

χ_0 is a column of sixteen symbols `q1`, ..., `q16` (arbitrary constants). `family` is the member $\sin^\alpha z(\cosh(kx_4)\chi_0 + \frac{\sinh(kx_4)}{k} M\chi_0)$, and `residual` is $\gamma^\mu D_\mu \Psi - m\Psi$ with the full connection, summed over all eight directions; every entry must vanish, for symbolic α, m, H and every a_4.

```
M_half = M.subs(alpha, -sp.Rational(1, 2))
check_record(not M_half.has(H)
            and sp.expand(k.subs(alpha, -sp.Rational(1, 2)) ** 2) == -m**2,
            "alpha = -1/2: M = -m gamma^(x4) has no H and k^2 = -m^2 (oscillation)",
            SCOPE, "rescaled_equation_quadratic_potential",
            "= -m gamma^(x4) and k^2 = -m^2 (no H)")
```

At $\alpha = -\frac{1}{2}$, M contains no H and $k^2 = -m^2$: the rescaled field of Section 8.30. Output: three PASS lines with their `reproduces` lines.

In [6], which members grow and which have a finite norm.

```
s, p, eps = sp.symbols("s p epsilon", positive=True)
check(sp.integrate(s ** (p - 1), (s, 0, 1)) == 1 / p and
      sp.limit(sp.integrate(1 / s, (s, eps, 1)), eps, 0, "+") == sp.oo,
      "int_0^1 s^(2 alpha) ds = 1/(2 alpha + 1) for 2 alpha + 1 > 0; diverges at -1/2")
```

With $p = 2\alpha + 1 > 0$, sympy integrates $s^{p-1} = s^{2\alpha}$ from 0 to 1 and finds $1/p$; and the integral of $1/s$ from ϵ to 1 goes to infinity (`sp.oo`) as $\epsilon \to 0$ from above (`"+"`): the two integrals of Section 8.31.

```
k2_every_mass = sp.factor(sp.expand((c**2 - m**2).subs(alpha, m / (3 * H))))
say(f"k^2 at alpha = m/(3H): {k2_every_mass}")
check(sp.expand(k2_every_mass - 3 * (m + H) * (m + 3 * H)) == 0,
      "alpha = m/(3H) > -1/2 gives k^2 = 3 (m + H)(m + 3H) > 0 for every m > 0")
```

At $\alpha = m/(3H)$ sympy factors k^2 and prints `3*(H + m)*(3*H + m)`; the check compares with $3(m+H)(m+3H)$.

```
alphas = np.linspace(-2.0, 1.0, 601)
fig, ax = plt.subplots()
for mass, style in ((1.0, "-"), (2.0, "--"), (4.0, ":")):
    ax.plot(alphas, 9 * (2 * alphas + 1) ** 2 - mass**2, style, label=f"$m = {mass:g}H$")
    ax.plot([-0.5, 0.0], [-mass**2, 9 - mass**2], "o", color="black", markersize=4)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.axvspan(-2.0, -0.5, color="gray", alpha=0.15)
ax.text(-1.4, 66, "infinite norm\nat the tip", fontsize=8)
ax.text(0.05, 66, "grows where\n$k^2 > 0$", fontsize=8)
```

For three masses ($H = 1$) the parabola $k^2 = 9(2\alpha+1)^2 - m^2$ is drawn against α from -2 to 1, with black dots at $\alpha = -\frac{1}{2}$ ($k^2 = -m^2$) and $\alpha = 0$ ($k^2 = 9 - m^2$); `axvspan` shades the band $\alpha \le -\frac{1}{2}$ light gray (here `alpha=0.15` is matplotlib's transparency, not the exponent).

```
ax.set_ylim(-20.0, 80.0)
ax.set_xlabel("exponent $\\alpha$ of $\\sin^{\\alpha}z$")
ax.set_ylabel("$k^2 = 9H^2(2\\alpha + 1)^2 - m^2$ (units of $H^2$)")
ax.set_title("The exact family $\\sin^\\alpha z\\,e^{Mx_4}\\chi_0$, $H = 1$")
ax.legend(loc="upper center");
save_figure(fig, "family_k_squared",
            "The number $k^2 = 9H^2(2\\alpha + 1)^2 - m^2$ of the exact family of "
            ...
            "finite norm grows.")
```

The vertical range, labels, and Figure 08d.2. Output: two PASS lines, the factored k^2, the figure line.

What Figure 08d.2 shows. The number k^2 (vertical, units of H^2) against the exponent α (horizontal) for $m = H, 2H, 4H$: three upward parabolas with their lowest point $-m^2$ at $\alpha = -\frac{1}{2}$ (black dots at -1, -4, -16); at $\alpha = 0$ the dots are at 8, 5 and -7. The shaded band $\alpha \le -\frac{1}{2}$ is where the norm diverges at the tip. The student should see that to the right of the band every parabola eventually becomes positive: for every mass, some members with a finite norm grow; and that the x_0-independent member $\alpha = 0$ grows for $m = H$ and $2H$ but not for $4H$.

In [7], the plane of mass and exponent.

```
masses = np.linspace(0.0, 6.0, 301)
A_grid, M_grid = np.meshgrid(np.linspace(-1.0, 1.5, 251), masses)
grows = 9 * (2 * A_grid + 1) ** 2 - M_grid**2 > 0
finite = A_grid > -0.5
region = np.where(grows & finite, 2, np.where(grows, 1, 0))  # 2, 1 or 0
```

A grid of 301 masses (0 to $6H$) and 251 exponents (-1 to 1.5). `grows` and `finite` are true/false arrays; `&` is "and" for such arrays. `region` is 2 where a member grows with a finite norm, 1 where it grows with an infinite norm, and 0 where it oscillates.

```
fig, ax = plt.subplots(figsize=(6.4, 4.6))
ax.contourf(M_grid, A_grid, region, levels=[-0.5, 0.5, 1.5, 2.5],
            colors=["white", "#d9d9d9", "#9ecae1"])
ax.plot(masses, (masses / 3 - 1) / 2, color="black", linewidth=1.0)
ax.axhline(-0.5, color="gray", linestyle="--", linewidth=0.9)
ax.axhline(0.0, color="tab:red", linewidth=1.2)
ax.plot([3.0], [0.0], "o", color="tab:red")
ax.text(0.2, 1.2, "grows, finite norm", fontsize=9)
ax.text(4.2, -0.2, "oscillates", fontsize=9)
ax.text(0.2, -0.9, "grows, infinite norm", fontsize=9)
ax.text(3.1, 0.05, "$\\alpha = 0$: grows for $m < 3H$", fontsize=8, color="tab:red")
ax.set_xlabel("mass $m$ (units of $H$)")
ax.set_ylabel("exponent $\\alpha$")
ax.set_title("Members of the family that grow and have a finite norm")
```

`contourf` fills the three regions with white, light gray and light blue (colours given as hexadecimal codes, `"#9ecae1"`). The black line is $2\alpha + 1 = m/(3H)$, that is $\alpha = (m/3 - 1)/2$ for $H = 1$; the gray dashed line is $\alpha = -\frac{1}{2}$; the red line is $\alpha = 0$, with a red dot at $m = 3H$; four labels.

```
fraction = float(np.mean(grows & finite))
report("fraction of the drawn (m, alpha) rectangle that grows with finite norm",
       f"{fraction:.4f}")
check(all(np.any(grows[i] & finite[i]) for i in range(len(masses))),
      "for every drawn mass some alpha gives a growing finite-norm member")
save_figure(fig, "growing_finite_norm",
            "The plane of the mass $m$ (horizontal, units of $H$) and the exponent "
            ...
            "every mass has growing finite-norm members.")
```

The fraction of the drawn rectangle where members grow with a finite norm is printed (0.5996). The check requires, for each of the 301 masses (each row `i` of the grid), at least one exponent with a growing finite-norm member (`np.any` is true when at least one entry is). Figure 08d.3. Output: one RESULT line, one PASS line, the figure line.

What Figure 08d.3 shows. The plane of the mass m (horizontal, 0 to $6H$) and the exponent α (vertical, -1 to 1.5). Light blue, above the rising black line $2\alpha + 1 = m/(3H)$: growing members with a finite norm. Light gray, a wedge below the dashed line $\alpha = -\frac{1}{2}$ at small masses: growing members whose norm diverges at the tip. White: oscillating members. The red line $\alpha = 0$ lies in the blue region exactly for $m < 3H$ (red dot). The student should see that every vertical line (every mass) meets the blue region.

In [8], the modes that do not depend on x_0.

```
A = -sp.I * m * G[4] + 3 * sp.I * H * G[4] * G[0]
A_squared = (A * A).applyfunc(sp.expand)
A_one = A.subs({m: 1, H: 1})
dimension = 16 - (A_one - 2 * sp.sqrt(2) * sp.I * sp.eye(16)).rank()
norm_factor = sp.integrate(sp.cos(6 * H * x[0]), (x[0], 0, sp.pi / (12 * H)))
report("dimension of the eigenspace of +2 sqrt(2) i at m = H = 1", dimension)
report("int_0^(pi/(12H)) cos(6 H x0) dx0", norm_factor)
```

The matrix $A = -im\gamma^{(4)} + 3iH\gamma^{(4)}\gamma^{(0)}$ of Section 8.32, its square, its value at $m = H = 1$, the dimension of the eigenspace of $+2\sqrt{2}\,i$ (16 minus a rank, as in Notebook 08a), and the norm factor $\int_0^{\pi/(12H)} \cos(6Hx_0)\,dx_0$ integrated exactly. RESULT lines: 8 and `1/(6*H)`.

```
all_true = ((A - sp.I * M.subs(alpha, 0)).applyfunc(sp.expand) == sp.zeros(16, 16)
            and A != A.H
            and A_squared == ((m**2 - 9 * H**2) * sp.eye(16)).applyfunc(sp.expand)
            and A_one * A_one == -8 * sp.eye(16) and A_one.trace() == 0
            and dimension == 8 and sp.simplify(norm_factor - 1 / (6 * H)) == 0)
# The record states the same matrix, square, eigenvalues and norm in its text:
check_record(all_true,
             "A = -i m g4 + 3 i H g4 g0: not Hermitian, A^2 = (m^2 - 9H^2) I16, "
             "eigenvalues +-2 sqrt(2) i at m = H = 1, finite norm 1/(6H)",
             SCOPE, "good_sector_x0_independent_modes_without_boundary_condition",
             f"cos(6 H x0) dx0 = {sp.sstr(norm_factor)})",
             "A = -i m gamma^(x4) + 3 i H gamma^(x4) gamma^(x0)",
             "A^2 = (m^2 - 9 H^2) I16 exactly",
             f"the eigenvalues are +-2 sqrt(2) i ({dimension} each)")
```

Seven facts in one condition: $A = iM$ at $\alpha = 0$; A is not Hermitian ($A \neq A^\dagger$); $A^2 = (m^2 - 9H^2)I_{16}$; at $m = H = 1$, $A^2 = -8I_{16}$ and the trace is 0 (so the eigenvalues are $\pm 2\sqrt{2}\,i$, eight each, as in Section 8.17); the eigenspace dimension is 8; and the norm factor is $1/(6H)$. The record's detail must contain the four texts. Output: two RESULT lines and one PASS line with its `reproduces` line.

In [9], the eigenvalues of A against the mass.

```
A_number = sp.lambdify((m, H), A, "numpy")
mass_path = np.linspace(0.0, 5.0, 51)
points = np.concatenate([np.linalg.eigvals(np.array(A_number(mv, 1.0), dtype=complex))
                         for mv in mass_path])
largest_growth = max(abs(points.imag))
report("largest |imaginary part| on the path (reached at m = 0)",
       f"{largest_growth:.9f}")
check(abs(largest_growth - 3.0) < 1e-9, "at m = 0 the growth rate is 3H")
```

`A_number` is a numerical function of m and H. For 51 masses from 0 to 5 (with $H = 1$) the 16 eigenvalues are computed and joined; the largest imaginary part, reached at $m = 0$ where the eigenvalues are $\pm 3i$, is printed as 3.000000000, and the check requires $3H$ to 10^{-9}.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
dots = left.scatter(points.real, points.imag, c=np.repeat(mass_path, 16),
                    cmap="plasma", s=20)
fig.colorbar(dots, ax=left, label="mass $m$ (units of $H$)")
left.set_aspect("equal")
left.set_xlim(-4.5, 4.5)
left.set_ylim(-4.5, 4.5)
left.set_xlabel("real part")
left.set_ylabel("imaginary part")
left.set_title("Eigenvalues of $A$, $H = 1$")
```

The left panel draws the eigenvalues in the complex plane, coloured by the mass, with equal scales on both axes.

```
mass_fine = np.linspace(0.0, 5.0, 501)  # a finer grid for the smooth curves
for a_value, style in ((0.0, "-"), (0.5, "--"), (1.0, ":")):
    rate = np.sqrt(np.maximum(9 * (2 * a_value + 1) ** 2 - mass_fine**2, 0.0))
    right.plot(mass_fine, rate, style, label=f"$\\alpha = {a_value:g}$")
right.axvline(3.0, color="gray", linewidth=0.8)
right.set_xlabel("mass $m$ (units of $H$)")
right.set_ylabel("growth rate $k$ (units of $H$)")
right.set_title("Growth rates of the family")
right.legend();
save_figure(fig, "patch_end_modes",
            "Left: the eigenvalues $\\pm\\sqrt{m^2 - 9H^2}$ of the matrix $A = "
            ...
            "line).")
```

The right panel draws the growth rate $k = \sqrt{9(2\alpha+1)^2 - m^2}$ (0 where the bracket is negative) of the members $\alpha = 0, 0.5, 1$ against the mass, with a gray vertical line at $m = 3H$. Figure 08d.4. Output: one RESULT line, one PASS line, the figure line.

What Figure 08d.4 shows. Left: the eigenvalues of A (units of H) in the complex plane, coloured by the mass from 0 to $5H$: for small masses they sit on the imaginary axis at $\pm 3i$ (dark), move towards 0 as m grows, meet at 0 for $m = 3H$, and then move out along the real axis to ± 4 at $m = 5H$ ($\sqrt{25-9} = 4$). Right: the growth rates (units of H) against the mass: the member $\alpha = 0$ falls from 3 to 0 at $m = 3H$ (gray line) and stays 0; the members $\alpha = 0.5$ and $\alpha = 1$ start at 6 and 9 and still grow at $m = 5H$ (about 3.3 and 7.5). The student should see that the x_0-independent modes stop growing above $m = 3H$, but members that vary along x_0 do not.

In [10], the boundary term at the patch end.

```
M0 = sp.I * G[4] * G[0]
d0_coefficient = sp.I * sp.tan(z) * G[4] * G[0]  # the matrix in front of d0 in h
fact_1 = all(is_zero(v) for v in sp.cos(z) * d0_coefficient - sp.sin(z) * M0)
fact_2 = all(is_zero(v) for v in sp.cos(z) * (A - A.H) - sp.diff(sp.sin(z), x[0]) * M0)
fact_3 = (d0_coefficient + d0_coefficient.H).applyfunc(sp.simplify) == sp.zeros(16, 16)
```

$M_0 = i\gamma^{(4)}\gamma^{(0)}$ and the coefficient P of ∂_0 in h. The three facts of Section 8.32, in the order of the notebook: $\cos z\, P = \sin z\, M_0$; $\cos z\,(A - A^\dagger) = (\partial_0 \sin z) M_0$; and $P + P^\dagger = 0$.

```
X0 = x[0]
u_col = sp.Matrix([(A_ + 1) + sp.I * (2 * A_ - 3) * X0 + (A_ - 5) * X0**2 / 3
                   for A_ in range(16)])  # a column of polynomials in x0
v_col = sp.Matrix([(3 - A_) * X0 + sp.I * (A_ + 2) + sp.I * A_ * X0**3 / 7
                   for A_ in range(16)])


def h_of(column):
    """The mode operator h applied to a column of functions of x0 (U = 0)."""
    return -sp.I * m * G[4] * column + sp.I * G[4] * G[0] * (
        sp.tan(z) * sp.diff(column, X0) + 3 * H * column)
```

Two explicit columns of polynomials in x_0 with complex coefficients that differ from entry to entry (the loop variable `A_` runs from 0 to 15; its name has an underscore so that it does not overwrite the matrix `A`), and the mode operator h applied to a column.

```
left_side = sp.cos(z) * ((u_col.H * h_of(v_col))[0] - (h_of(u_col).H * v_col)[0])
right_side = sp.diff(sp.sin(z) * (u_col.H * M0 * v_col)[0], X0)
u0 = (G[4] * G[0] - sp.eye(16)).nullspace()[0]  # gamma^(4) gamma^(0) u0 = u0
flux = sp.simplify((u0.H * M0 * u0)[0])
size_u0 = (u0.H * u0)[0]
report("u0^dagger M0 u0 and u0^dagger u0", f"{flux}, {size_u0}")
```

The two sides of the boundary identity for these columns (a row times a column is a 1×1 matrix, and `[0]` takes its entry). `nullspace()` returns a list of independent columns u with $(\gamma^{(4)}\gamma^{(0)} - I)u = 0$; the first one is `u0`. RESULT: $u_0^\dagger M_0 u_0 = 2i$ (sympy writes `2*I`) and $u_0^\dagger u_0 = 2$.

```
# The record states the same identity and the same flux (sympy writes i as I):
check_record(fact_1 and fact_2 and fact_3 and is_zero(left_side - right_side)
             and flux == 2 * sp.I and size_u0 == 2,
             "cos z (u^dagger h v - (h u)^dagger v) = d0(sin z u^dagger M0 v); at "
             "z = pi/2 the flux u0^dagger M0 u0 = 2 i is not zero",
             SCOPE, "good_sector_hermiticity_up_to_the_brane_flux",
             "cos z [u^dagger (h v) - (h u)^dagger v] = d_x0(sin z u^dagger M0 v)",
             f"u^dagger M0 u = {sp.sstr(flux)} (|u|^2 = {sp.sstr(size_u0)})")
```

The check combines the three facts, the identity for the two columns, and the flux; the record's detail must contain the identity and the same flux. (The record proves the identity for all columns; here it is confirmed on one explicit pair, and the three facts are what make it general, Section 8.32.) Output: one RESULT line and one PASS line with its `reproduces` line.

In [11], a growing mode is fed through the patch end.

```
A_num = np.array(A_number(1.0, 1.0), dtype=complex)  # m = H = 1
g4g0 = np.array(G[4] * G[0], dtype=float)  # real symmetric
rng = np.random.default_rng(12345)  # fixed seed: the same column in every run
chi_start = rng.normal(size=16) + 1j * rng.normal(size=16)
chi_start = chi_start / np.linalg.norm(chi_start)
omega = np.sqrt(complex(1.0 - 9.0))  # A^2 = (m^2 - 9 H^2) = -8: omega = 2 sqrt(2) i
times = np.linspace(0.0, 1.5, 151)
```

The matrix A at $m = H = 1$, the real symmetric matrix $\gamma^{(4)}\gamma^{(0)}$, a random starting column of size 1 (fixed seed, as in Notebook 08a), the frequency $\omega = \sqrt{m^2 - 9H^2} = \sqrt{-8} = 2\sqrt{2}\,i$ (here the name `omega` is a frequency, not the spin connection), and 151 times from 0 to 1.5.

```
norms, slopes, fluxes = [], [], []
for x4 in times:
    state = np.cos(omega * x4) * chi_start \
        - 1j * np.sin(omega * x4) / omega * (A_num @ chi_start)  # e^(-i A x4) chi
    norms.append(np.vdot(state, state).real / 6.0)  # N = Psi^dagger Psi / (6 H)
    slopes.append(2 * np.vdot(state, -1j * (A_num @ state)).real / 6.0)  # dN/dx4
    fluxes.append(np.vdot(state, g4g0 @ state).real)  # flux at z = pi/2
norms, slopes, fluxes = map(np.array, (norms, slopes, fluxes))
```

At each time the mode $\Psi = e^{-iAx_4}\chi_0$ is computed with the closed formula of Section 8.18 (valid because $A^2 = \omega^2 I_{16}$). Three numbers are recorded: the norm $N = \Psi^\dagger\Psi/(6H)$; its rate of change $dN/dx_4 = 2\,\mathrm{Re}(\Psi^\dagger(-iA\Psi))/(6H)$ (the derivative of $\Psi^\dagger\Psi$ is twice the real part of $\Psi^\dagger\partial_4\Psi$, and $\partial_4\Psi = -iA\Psi$); and the flux $\Psi^\dagger\gamma^{(4)}\gamma^{(0)}\Psi$ at the patch end (a field that does not depend on x_0 has the same value there). `map(np.array, ...)` turns the three lists into arrays.

```
agreement = np.max(np.abs(slopes - fluxes) / np.maximum(np.abs(fluxes), 1.0))
check(agreement < 1e-12, "dN/dx4 equals the flux through the patch end at every x4")
report("N(1.5) / N(0)", f"{norms[-1] / norms[0]:.6e}")
check(norms[-1] / norms[0] > 100.0, "the norm grows by more than a factor 100")
```

The check requires dN/dx_4 to equal the flux at all 151 times, to 10^{-12} relative to $\max(|\text{flux}|, 1)$: the result of Section 8.32. The RESULT line prints $N(1.5)/N(0) = 2.094786 \times 10^3$, and the second check requires growth by more than 100.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.semilogy(times, norms)
left.set_xlabel("time $x_4$ (units of $1/H$)")
left.set_ylabel("norm $N(x_4)$")
left.set_title("The $x_0$-independent mode, $m = H = 1$")
right.plot(times, slopes, label="$dN/dx_4$")
right.plot(times[::10], fluxes[::10], "o", color="black", markersize=4,
           label="flux at $z = \\pi/2$")
right.axhline(0.0, color="black", linewidth=0.8)
right.set_yscale("symlog", linthresh=1.0)  # linear between -1 and 1, log outside
right.set_ylim(-1.5, 3.0e3)
right.set_xlabel("time $x_4$ (units of $1/H$)")
right.set_ylabel("rate (units of $H$)")
right.set_title("Rate of change of the norm = flux")
right.legend();
save_figure(fig, "norm_and_flux",
            "An $x_0$-independent mode of the good sector, $\\Psi = e^{-iAx_4}"
            ...
            "condition.")
```

Left: the norm on a logarithmic axis. Right: dN/dx_4 as a line and every tenth flux value as a black dot, on a symmetric logarithmic axis that is linear between -1 and 1, so that the short negative phase is visible. Figure 08d.5. Output: two PASS lines, one RESULT line, the figure line.

What Figure 08d.5 shows. Left: the norm N (logarithmic vertical axis) against the time x_4 from 0 to 1.5 (units of $1/H$): it starts at $1/6$, dips a little during the first tenth of a unit, and then rises along a straight line (growth like $e^{4\sqrt{2}\,x_4}$, twice the rate $2\sqrt{2}$ of the column) to about 350. Right: the rate of change dN/dx_4 (line) and the flux through the patch end (dots), in units of H: slightly negative at the start, zero near $x_4 \approx 0.05$, then rising to about 2000. The student should see the dots lie exactly on the line: the norm changes only through the patch end, where the Revision record imposes no boundary condition.

In [12], the last check.

```
for name in ("08d_1_rescaling_factors.png", "08d_2_family_k_squared.png",
             "08d_3_growing_finite_norm.png", "08d_4_patch_end_modes.png",
```

```
            "08d_5_norm_and_flux.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Five checks that the figure files exist, and the last line. The 23 checks come from the cells as follows: two in In [2], four in In [3], one in In [4], three in In [5], two in In [6], one each in In [7], In [8], In [9] and In [10], two in In [11] and five here. The last line reads `ALL 23 CHECKS PASSED (notebook 08d)`.

8.37 The exact scope of non-triviality and of the evolution, in one table

Every statement of the chapter in one place. The last column names the Revision check, the short name of its report file in brackets, and the notebook that reproduces it. The report files are these:

```
Revision/theory/reports/python-field-theory.json     (python-field-theory)
Revision/theory/reports/wolfram-field-theory.json    (wolfram-field-theory)
Revision/theory/reports/python-scope.json            (python-scope)
Revision/theory/reports/wolfram-scope.json           (wolfram-scope)
Revision/pairing/reports/python-pairing.json         (python-pairing)
Revision/theory/reports/python-field-theory-b4.json  (python-field-theory-b4)
```

The rows that concern the metric are statements at $b_4 = 1$, as in the scope reports (Section 8.1, "The metric with b_4"); the last two rows give the general-b_4 forms.

statement	**status**	**Revision check; notebook**
diagonal frame: $\gamma^\mu \Omega_\mu = 3H\gamma^{(0)}$ for every a_4; the $\pm a_4'/2$ pieces cancel, six pieces $H/2$ add	PROVED	`gamma_mu_Omega_mu_equals_3H_gamma_x0`, `time_terms_cancel_hidden_term_survives` (python-field-theory); 08c
$3H\gamma^{(0)}\Psi \neq 0$ for every $\Psi \neq 0$	PROVED	`nontriviality_1_dirac16complex`, `nontriviality_2_dirac16complex00` (wolfram-field-theory); 08c
$\Omega_\mu = 0$ for every μ only if $a_4' = 0$ and $H = 0$ (not a member of the family)	PROVED	`nontriviality_Omega_zero_iff_flat` (python-field-theory); 08c
the term contains no a_4, although $R^{x_4}{}_{x_4} = 6(a_4')^2$	PROVED	`gammaOmega_blind_to_the_deflation` (python-scope); 08c
the connection drops out of the Lagrangian; the term is the half-density term	PROVED	`connection_free_lagrangian_same_equations` (python-scope); 08c

statement	status	Revision check; notebook
boosted frame: $\gamma'^{\mu}\Omega'_{\mu} = \frac{6H-\beta}{2}(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$, zero for $\beta = 6H$	PROVED	`boosted_frame_gammaOmega_formula`, `boosted_frame_gammaOmega_vanishes` (python-scope, wolfram-scope); 08c
the spin connection vanishes in no frame: its curvature is the Riemann tensor, nonzero for $H > 0$	PROVED	`boosted_frame_curvature_nonzero` (python-scope); `spinor_curvature_equals_riemann` (python-field-theory); 08c
the connection enters the energy-momentum tensor	PROVED	`spin_connection_in_the_energy_momentum_tensor` (python-scope); 08c
the rescaling $\Psi = \sin^{-1/2} z\,\chi$ removes the term; for $U \neq 0$ it becomes $\lambda S[\chi]/\sin z$	PROVED	`rescaling_removes_the_connection_term`, `rescaled_equation_quadratic_potential` (python-scope); 08d
the slices $x_4 = \text{const}$ are non-characteristic	PROVED	`evolution_form_G` (wolfram-field-theory); 08a
general b_4: $\gamma^{\mu}\Omega_{\mu} = 3H\gamma^{(0)} + \frac{3}{4}(b'_4/b_4)\gamma^{(4)}$	PROVED	`gammaOmega_total` (python-field-theory-b4); Notebook 06a, Chapter 7
general b_4: $MM = (9H^2 - \frac{9}{16}(b'_4/b_4)^2)I_{16}$, so $M\Psi \neq 0$ for $\Psi \neq 0$ unless $\lvert b'_4/b_4\rvert = 4H$	PROVED	`gravity_term_square_b4` (python-field-theory-b4); Notebook 06a, Section 7.23
$h_k^2 = E^2 I_{16}$; the growth rates of extra-time waves have no upper bound; in flat 4+4 space and with frozen coefficients the Cauchy problem is not well posed in Hadamard's sense	PROVED	`extra_time_growth_rates_unbounded` (python-scope, wolfram-scope); 08a
in the author's metric itself (varying coefficients, $U = 0$) the Cauchy problem is not well posed for data that depend on the extra times	ASSUMED (a quoted theorem)	not checked by the record; the Lax-Mizohata theorem, quoted without proof, applied to the frequencies of the principal part (Section 8.18)
the Krein form is conserved; growing eigenspaces are Krein-neutral; real-frequency eigenspaces have inertia (4,4)	PROVED	`mode_hamiltonian_B_selfadjoint_dispersion` (python-field-theory); `Q.one_particle_complex_frequency_Krein_neutral`, `Q.one_particle_Krein_inertia_proof` (python-pairing); 08a

statement	status	Revision check; notebook
along the deflating history every extra-time wave reaches the onset of growth (frozen coefficients at each instant)	PROVED	`extra_time_modes_grow` (python-field-theory); 08b
through the onset the growth follows the WKB formulas and is faster than any exponential (local-frame model)	COMPUTED	08b: agreement 1.03×10^{-3} (leading), 2.48×10^{-4} (first order), late rate 10^{-5}
the exact family $\sin^{\alpha} z\, e^{M x_4} \chi_0$; for every mass a growing member with finite norm	PROVED	`exact_solution_family_x4_x0` (python-field-theory); 08d
the x_0-independent good-sector modes grow for $m < 3H$; the mode operator is symmetric only up to the flux at $z = \pi/2$	PROVED	`good_sector_x0_independent_modes_without_boundary_condition`, `good_sector_hermiticity_up_to_the_brane_flux` (python-scope); 08d
the history $a_4 = AHx_4$ $(A = 1)$	ASSUMED	prescribed background (`Revision/kohn_sham/results/parameters.json`)
frozen coefficients; the local-frame model	ASSUMED	labelled MODEL in 08a and 08b
which boundary condition at $z = \pi/2$ is physical; a well-posed formulation of the full field equation	OPEN	none imposed in the record

8.38 What we proved, what we computed, what we assumed

PROVED in this chapter, line by line (and in the Revision record, with the file and check named in the section):

- The canonical spin connection of the diagonal frame: the rule $\omega_\mu{}^a{}_b = (f_a/f_b)\Gamma^a{}_{\mu b}$ for $a \neq b$, its 12 independent nonzero components, and the spinor connections Ω_{x_i}, Ω_{x_t}, $\Omega_{x_4} = \Omega_{x_0} = 0$ (Section 8.4).
- $\gamma^\mu \Omega_\mu = 3H\gamma^{(0)}$, direction by direction: the time-direction pieces of the three inflating and the three deflating directions cancel, the six hidden-direction pieces add; and the divergence form $\gamma^\mu \Omega_\mu = \frac{1}{2\sqrt{|g|}} \partial_\mu (\sqrt{|g|}\gamma^\mu)$ (Section 8.5).
- $\Omega_\mu = 0$ for every μ if and only if $a_4' = 0$ and $H = 0$; $3H\gamma^{(0)}\Psi \neq 0$ for every $\Psi \neq 0$ (Section 8.6). The term contains no a_4, while $R^{x_4}{}_{x_4} = 6(a_4')^2$ and $R = 6((a_4')^2 - 7H^2)$ (Section 8.7).
- $\{\gamma^\mu, \Omega_\mu\} = 0$ for each μ; the connection drops out of the Lagrangian, and the Euler-Lagrange

equation of the connection-free symmetric Lagrangian contains the same term as its half-density term (Section 8.8).

- The boost keeps the metric; the spin transformation $S = e^{-(b/2)\gamma^{(4)}\gamma^{(0)}}$ moves the gammas of the boosted frame; the connection of the boosted frame and the formula $\gamma'^{\mu}\Omega'_{\mu} = \frac{6H-\beta}{2}(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$, zero for $\beta = 6H$ (Sections 8.3 and 8.9).

- $F' = SFS^{-1}$; $F_{x_1 x_0} \neq 0$ in every frame of this kind; the square of the curvature is the same in both frames (Section 8.10). The connection part $\frac{1}{2}e^{a_4}\sin^{1/6} z\, H\, \gamma^{(4)}\gamma^{(1)}\gamma^{(0)}$ of $K^{x_4}{}_{x_1}$ (Section 8.11).

- The Hadamard toy example; the second-order form with the extra-time derivatives of the "wrong" sign; the evolution form (Section 8.16). The mode matrix, $h_k^2 = E^2 I_{16}$, its eigenvalues $\pm E$ eight each, its anti-Hermitian part (Section 8.17). Unbounded growth rates ($\kappa \geq K - c$), the closed solution in time, and the failure of continuous dependence in every Sobolev norm, in flat 4+4 space and with frozen coefficients (Section 8.18); the frequencies of the principal part, which are not real for a wave along an extra time at any point of the author's metric (Section 8.18). Conservation of the Krein form and the Krein-neutrality of the growing eigenspaces (Section 8.19).

- Conserved coordinate wave numbers, growing frame momenta, the exact onset time and its finiteness for every $q_5 > 0$ (Section 8.24). The Krein form $\sigma E_0/m$ of the starting columns and the leading and first-order WKB formulas (Section 8.25).

- The rescaling removes the term for every field, and turns the quadratic potential into $\lambda S[\chi]/\sin z$; the norm becomes $\int \chi^{\dagger}\chi\, dy$ (Section 8.30). The exact family, its growth condition, its finite-norm condition, and a growing finite-norm member for every mass (Section 8.31). The matrix A and its growing modes; the boundary identity; the change of the norm only through the flux at the patch end (Section 8.32).

PROVED in the Revision record and used here without its proof: the Clifford relations and the reality and symmetry of the author's gammas (Chapter 4); the irreducibility of the 16-dimensional representation (commutant dimension 1; Chapter 5); the 25 Christoffel symbols and the Ricci components (Chapter 3); the vielbein postulate and the covariant constancy of the gammas in the diagonal frame; the curvature identity $F = \frac{1}{4}R\gamma\gamma$ in the diagonal frame; $Bh_k = h_k^{\dagger}B$; the Krein inertia (4,4) of every real-frequency eigenspace; the Euler-Lagrange equations of the Grassmann field; and the direct computation of the boosted frame's connection.

COMPUTED by the notebooks (every exact statement is also checked there, and every number reproduces the record where they overlap): Notebook 08c, 32 checks, computes everything of Sections 8.4 to 8.11 exactly with sympy, including the curvature in both frames, and at one point of the history $a_4 = x_4$ the largest curvature entries 0.778093 and 0.038739 ($= e^{-3} \cdot 0.778093$) and ranks 8. Notebook 08a, 30 checks, confirms $h_k^2 = E^2$ exactly, the growth rates against the eigenvalues to 10^{-10}, the closed formula against RK4 to 10^{-9}, the Hadamard ratio (first above 10^6 at $K = 13.9, 19.9, 27.1, 44.2$ for $s = 0, 2, 4, 8$), the Krein drift below 10^{-12}. Notebook 08b, 25 checks, confirms the onset times against bisection to 10^{-12}, RK4 with the error ratio 15.89 and accuracy 10^{-8}, the WKB agreement (1.03×10^{-3}, 2.48×10^{-4}, late rate 10^{-5}), the slow growth of the size before the onset (to $u^{\dagger}u = 1.52$ and 1.54 at the onset), and growth to $u^{\dagger}u \approx 1.26 \times 10^{16}$ with the Krein drift below 10^{-9}. Notebook 08d, 23 checks, proves the rescaling for arbitrary functions, the

family for symbolic α, the boundary identity on explicit columns, and finds $N(1.5)/N(0) = 2094.786$ with dN/dx_4 equal to the flux to 10^{-12}. Run times: 08a and 08b run in well under a minute, 08c and 08d in one to two minutes; the provenance file of each notebook (in the folder `Revision/textbook/notebooks`, named after the notebook, for example `08c_frame_dependence.PROVENANCE.md`) records the times measured on the build computer in its build and check runs, which depend on how busy that computer is.

ASSUMED, chosen, or an interpretation. The history $a_4 = AHx_4$ with $A = H = m = 1$ is a PRESCRIBED BACKGROUND (the Kohn-Sham record's own label); every exact identity of the chapter holds for every history. The second function of the metric is put equal to $b_4 = 1$ in every exact check of this chapter, as in the Revision record's scope reports; the general-b_4 forms of the gravitational term and of the non-triviality statement are quoted from Chapters 6 and 7 (Sections 8.1 and 8.37). Frozen coefficients (Sections 8.17 to 8.19) and the local-frame model (Section 8.25) are labelled MODEL: they describe a wave near one point, not exact solutions of the field equation in the author's metric. Facts taken from Chapter 6 (where local spin covariance is proved): the canonical spin connection of every frame makes its gammas covariantly constant and is a combination of the S^{ab}. Standard theorems quoted without proof: the Lax-Mizohata theorem, which carries the failure of well-posedness from frozen coefficients to the author's varying coefficients (Section 8.18; so that conclusion is ASSUMED, not PROVED, and the record does not check it); the general curvature identity $F = \frac{1}{4}R\gamma\gamma$ in every frame (proved for this metric by the record); the trace of a matrix is the sum of its eigenvalues; the dimension of an eigenspace is 16 minus the rank of $h - \lambda I$. Treating $\bar{\Psi}$ and Ψ as independent variables in the Euler-Lagrange equations is the method of Chapter 7. Two statements are interpretations, labelled as such: that the role of the term $3H$ depends on the variables and the measure (Section 8.30), and that the growth of the good-sector modes is "fed through the patch end" (Section 8.32).

OPEN. Which boundary condition at the patch end $z = \pi/2$ is physical (the Kohn-Sham record ASSUMES a Z_2 brane there; this record imposes none). Whether and how the full field equation can be given a well-posed evolution: restricting to the good sector removes the extra-time growth but is a choice, not a consequence, and even the good sector needs a boundary condition. A positive-norm quantum theory of the full field with the growing modes present, and a stable vacuum, are not constructed (Chapter 10 states what is). Whether the frame-dependent term $3H\gamma^{(0)}$ has any observable effect is not claimed.

8.39 Exercises

Exercise 1. At $b_4 = 1$, compute $\gamma^{x_6}\Omega_{x_6}$ directly from $\Omega_{x_6} = -\frac{1}{2}e^{-a_4}\sin^{1/6} z\,(a_4'\gamma^{(4)}\gamma^{(6)} + H\gamma^{(6)}\gamma^{(0)})$, line by line.

Answer. $\gamma^{x_6} = \gamma^{(6)}/f_6$ with $f_6 = e^{-a_4}\sin^{1/6} z$. Then $\gamma^{x_6}\Omega_{x_6} = -\frac{1}{2}(a_4'\gamma^{(6)}\gamma^{(4)}\gamma^{(6)} + H\gamma^{(6)}\gamma^{(6)}\gamma^{(0)})$ (the scale factors cancel). Anticommuting $\gamma^{(6)}$ past $\gamma^{(4)}$: $\gamma^{(6)}\gamma^{(4)}\gamma^{(6)} = -\gamma^{(4)}(\gamma^{(6)})^2 = -\gamma^{(4)}(-I_{16}) = \gamma^{(4)}$; and $(\gamma^{(6)})^2 = -I_{16}$. So $\gamma^{x_6}\Omega_{x_6} = -\frac{1}{2}(a_4'\gamma^{(4)} - H\gamma^{(0)}) = -\frac{1}{2}a_4'\gamma^{(4)} + \frac{1}{2}H\gamma^{(0)}$, the value of every extra time in Section 8.5.

Exercise 2. Suppose the extra times inflated like 3-space, $f_5 = f_6 = f_7 = e^{+a_4}\sin^{1/6} z$ (this is NOT the author's metric). Compute $\gamma^{x_t}\Omega_{x_t}$ and $\gamma^{\mu}\Omega_{\mu}$, and confirm the result with the divergence form.

Answer. Now $g_{tt} = -e^{2a_4}\sin^{1/3} z$, so $\Gamma^{x_t}{}_{x_t x_4} = \partial_4 \ln f_t = +a_4'$ and $\Gamma^{x_4}{}_{x_t x_t} = -\partial_4 g_{tt}/(2g_{44}) = -(2a_4' g_{tt})/(-2) = a_4' g_{tt} = -a_4' f_t^2$. By the rule of Section 8.4, $\omega_{x_t}{}^{(4)}{}_{(t)} = (f_4/f_t)\Gamma^{x_4}{}_{x_t x_t} = -a_4' f_t$,

and lowering with $\eta_{44} = -1$, $\omega_{x_t\,(4)(t)} = +a_4' f_t$ (the opposite sign of the author's metric). The H component is unchanged, $\omega_{x_t\,(t)(0)} = -H f_t$. So $\Omega_{x_t} = \frac{1}{2} f_t (a_4' \gamma^{(4)}\gamma^{(t)} - H\gamma^{(t)}\gamma^{(0)})$, and $\gamma^{x_t}\Omega_{x_t} = \frac{1}{2}(a_4'\gamma^{(t)}\gamma^{(4)}\gamma^{(t)} - H(\gamma^{(t)})^2\gamma^{(0)}) = \frac{1}{2}(a_4'\gamma^{(4)} + H\gamma^{(0)})$, the same as a 3-space direction. The six directions then give $\gamma^\mu\Omega_\mu = 3a_4'\gamma^{(4)} + 3H\gamma^{(0)}$: nothing cancels. Divergence form: $\sqrt{|g|} = e^{6a_4}\cos z$, so $\frac{1}{2\sqrt{|g|}}\partial_4(\sqrt{|g|}\gamma^{(4)}) = \frac{6a_4' e^{6a_4}\cos z}{2e^{6a_4}\cos z}\gamma^{(4)} = 3a_4'\gamma^{(4)}$, and $\frac{1}{2\sqrt{|g|}}\partial_0(e^{6a_4}\sin z\,\gamma^{(0)}) = 3H\gamma^{(0)}$. The cancellation of Section 8.5 happens because in the author's metric the volume factor does not depend on the time.

Exercise 3. (a) Show that $\varphi = \epsilon\cosh(Kx_4)\cos(Kx_5)$ solves $\partial_4^2\varphi = -\partial_5^2\varphi$. (b) For data of largest value ϵ, compute the ratio of the largest value of the solution at $x_4 = 1$ to ϵ for $K = 10$ and $K = 20$. (c) Show that $\varphi = \epsilon\cos(Kx_4)\cos(Kx_1)$ solves the ordinary wave equation $\partial_4^2\varphi = \partial_1^2\varphi$, and compare.

Answer. (a) Section 8.16: both sides equal $\epsilon K^2\cosh(Kx_4)\cos(Kx_5)$. (b) The ratio is $\cosh K$: $\cosh 10 = 11013.2$ and $\cosh 20 = 2.4258\times 10^8$. Doubling K multiplies the amplification by about $e^{10} \approx 22000$; data of the same size give ever larger solutions. (c) $\partial_4^2\varphi = -\epsilon K^2\cos(Kx_4)\cos(Kx_1) = \partial_1^2\varphi$; the largest value stays ϵ at every time, for every K: the ordinary wave equation depends continuously on its data.

Exercise 4. With frozen coefficients and $m = 1$, decide whether the waves oscillate or grow, and give the frequency or the growth rate: (a) $k_1 = 2$, $k_5 = 3$; (b) $k_5 = 2$, $k_0 = 1$; (c) $k_6 = k_7 = 0.5$. (d) With $k_1 = 4$, how large must k_5 be for growth? Compare with Figure 08a.1.

Answer. $E^2 = m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2 - k_5^2 - k_6^2 - k_7^2$. (a) $E^2 = 1 + 4 - 9 = -4$: growth with $\kappa = 2$. (b) $E^2 = 1 + 1 - 4 = -2$: growth with $\kappa = \sqrt{2} = 1.414214$. (c) $E^2 = 1 - 0.25 - 0.25 = 0.5$: oscillation with $E = \pm 0.707107$. (d) Growth needs $k_5^2 > 1 + 16 = 17$, that is $k_5 > \sqrt{17} = 4.123106$; in the right panel of Figure 08a.1 the dotted curve ($k_1 = 4$) leaves zero there.

Exercise 5. Show that the size $\sqrt{\mathrm{tr}(M^T M)/16}$ of $M = c\,(\cosh b\,\gamma^{(0)} - \sinh b\,\gamma^{(4)})$ is $|c|\sqrt{\cosh 2b}$. Evaluate it for the boosted frame with $\beta = 3H$, $H = 1$, $b_0 = 0$ at $x_4 = 0$, 0.3 and 0.5, and for $\beta = 9H$ at $x_4 = 0.5$, and compare with Figure 08c.3.

Answer. $(\gamma^{(0)})^T = \gamma^{(0)}$ and $(\gamma^{(4)})^T = -\gamma^{(4)}$, so $M^T = c(\cosh b\,\gamma^{(0)} + \sinh b\,\gamma^{(4)})$ and $M^T M = c^2(\cosh^2 b\,(\gamma^{(0)})^2 - \sinh^2 b\,(\gamma^{(4)})^2 + \cosh b\sinh b\,(\gamma^{(4)}\gamma^{(0)} - \gamma^{(0)}\gamma^{(4)})) = c^2((\cosh^2 b + \sinh^2 b)I_{16} + 2\cosh b\sinh b\,\gamma^{(4)}\gamma^{(0)})$. The trace of $\gamma^{(4)}\gamma^{(0)}$ is zero, so $\mathrm{tr}(M^T M)/16 = c^2(\cosh^2 b + \sinh^2 b) = c^2\cosh 2b$ (because $\cosh^2 b + \sinh^2 b = \frac{(e^b+e^{-b})^2+(e^b-e^{-b})^2}{4} = \frac{e^{2b}+e^{-2b}}{2}$). For $\beta = 3H$, $c = (6-3)/2 = 1.5$ and $b = 3x_4$: at $x_4 = 0$ the size is 1.5; at $x_4 = 0.3$ ($b = 0.9$) it is $1.5\sqrt{\cosh 1.8} = 2.644204$; at $x_4 = 0.5$ ($b = 1.5$) it is $1.5\sqrt{\cosh 3} = 4.759437$. For $\beta = 9H$, $|c| = 1.5$, $b = 4.5$: $1.5\sqrt{\cosh 9} = 95.477587$. The orange and red curves of the right panel of Figure 08c.3 end near 4.8 and 95.

Exercise 6. On the history $a_4 = x_4$ ($a_4' = H = 1$), use the coefficients c_{14}, c_{10} printed by Notebook 08c in both frames to show that $F' = e^{-b}F$ and $F^2 = 0$; check the largest entries 0.778093 and 0.038739 at $x_4 = 0.5$.

Answer. Write $Q = e^{a_4}\cos z/(2\sin^{5/6} z)$. Diagonal frame: $c_{14} = -Ha_4'Q = -H^2Q$ and $c_{10} = -H^2Q$, equal. Boosted frame ($b = 6Hx_4 + b_0$): $c_{14}' = H(H\sinh b - a_4'\cosh b)Q = H^2(\sinh b - \cosh b)Q = -H^2e^{-b}Q$ and $c_{10}' = -H(H\cosh b - a_4'\sinh b)Q = -H^2(\cosh b - \sinh b)Q = -H^2e^{-b}Q$, using $\cosh b - \sinh b = e^{-b}$. Both coefficients are e^{-b} times the diagonal ones, and both frames use the same matrices $\gamma^{(1)}\gamma^{(4)}$, $\gamma^{(1)}\gamma^{(0)}$, so $F' = e^{-b}F$. Since $c_{14} = c_{10}$, $F^2 = (c_{14}^2 - c_{10}^2)I_{16} = 0$. At $x_4 = 0.5$, $b_0 = 0$: $b = 3$ and $0.778093\cdot e^{-3} = 0.778093\times 0.049787 = 0.038739$, the printed value.

Exercise 7. Along $a_4 = x_4$ with $m = H = 1$, compute the onset times of the waves (a) $q_1 = 0$, $q_5 = 0.01$, $y = 0$; (b) the same at $y = -3$; (c) $q_1 = 0$, $q_5 = 10^{-4}$, $y = 0$; (d) $q_1 = 1$, $q_5 = 0.05$, $y = 0$.

Answer. For $q_1 = 0$, $x_4^* = (Hy + \ln(m/q_5))/(AH) = y + \ln(1/q_5)$. (a) $\ln 100 = 4.605170$. (b) $-3 + 4.605170 = 1.605170$. (c) $\ln 10^4 = 9.210340$, the left end of the top curve in Figure 08b.3. (d) $w = 1$, $X^* = (1 + \sqrt{1 + 4 \cdot 1 \cdot 0.0025})/(2 \cdot 0.0025) = (1 + \sqrt{1.01})/0.005 = 400.997512$, and $x_4^* = \frac{1}{2} \ln X^* = 2.996978$, slightly later than the 2.995732 of the same wave without space momentum: the space term dies away like e^{-2a_4} and hardly matters.

Exercise 8. For the rescaling $\Psi = \sin^p z\, \chi$ with any real power p, find the term that replaces $3H\gamma^{(0)}$ in the equation for χ, and the power that removes it. What does the norm become for that power?

Answer. $\tan z\, \partial_0 \sin^p z = \tan z \cdot p \sin^{p-1} z \cos z \cdot 6H = 6Hp \sin^p z$. As in Section 8.30, $\gamma^\mu D_\mu(\sin^p z\, \chi) = \sin^p z (\gamma^\mu \partial_\mu \chi + (6Hp + 3H)\gamma^{(0)}\chi)$, so the term becomes $3H(2p+1)\gamma^{(0)}$. It vanishes exactly for $p = -\frac{1}{2}$. For $p = 0$ it is the original $3H\gamma^{(0)}$. For $p = -\frac{1}{2}$ the norm is $\int \cos z\, \sin^{-1} z\, \chi^\dagger \chi\, dx_0 = \int \chi^\dagger \chi\, dy$. (The exact family of Section 8.31 is this computation with $\chi = e^{Mx_4}\chi_0$: its $c = 3H(2\alpha + 1)$ is the same factor.)

Exercise 9. For $m = H = 1$ take the member $\alpha = m/(3H) = \frac{1}{3}$ of the exact family. Compute k^2 in two ways, the growth factor of e^{Mx_4} over one unit of time for large kx_4, and the norm factor $\int_0^{\pi/(12H)} \cos z \sin^{2\alpha} z\, dx_0$.

Answer. $k^2 = 9H^2(2\alpha + 1)^2 - m^2 = 9 \cdot (5/3)^2 - 1 = 25 - 1 = 24$, and also $3(m + H)(m + 3H) = 3 \cdot 2 \cdot 4 = 24$. So $k = 2\sqrt{6} = 4.898979$, and the member grows by the factor $e^k = 134.15$ per unit of time once cosh and sinh are both close to $\frac{1}{2}e^{kx_4}$. The norm factor is $\frac{1}{6H(2\alpha+1)} = \frac{1}{6 \cdot 5/3} = 0.1$: finite. This member varies along x_0 like $\sin^{1/3} z$. Unlike the x_0-independent member, whose growth needs $m < 3H$, the members $\alpha = m/(3H)$ grow for every mass.

Exercise 10. Prove the Krein-neutrality of a growing eigenvector algebraically: if $h_k u = i\kappa u$ with $\kappa > 0$ and $Bh_k = h_k^\dagger B$, then $u^\dagger B u = 0$.

Answer. Compute $u^\dagger B h_k u$ in two ways. First, $h_k u = i\kappa u$ gives $u^\dagger B h_k u = i\kappa\, u^\dagger B u$. Second, $Bh_k = h_k^\dagger B$ gives $u^\dagger B h_k u = u^\dagger h_k^\dagger B u = (h_k u)^\dagger B u = (i\kappa u)^\dagger B u = -i\kappa\, u^\dagger B u$ (the dagger conjugates the number $i\kappa$). Equating: $i\kappa\, u^\dagger B u = -i\kappa\, u^\dagger B u$, so $2i\kappa\, u^\dagger B u = 0$, and since $\kappa \neq 0$, $u^\dagger B u = 0$. The same computation with two eigenvectors u, v of $i\kappa$ gives $v^\dagger B u = 0$: the Krein form vanishes on the whole eigenspace. For a real eigenvalue E the two sides are $E\, u^\dagger B u$ and $E\, u^\dagger B u$, and nothing follows; that is why the real-frequency eigenspaces can carry the inertia (4,4).

9 The energy-momentum tensor, pressures, equations of state, conservation identities

Gravity has to be told where the energy is. In Einstein's theory, and in the Einstein-Lovelock theory that Chapter 12 uses for the author's metric, the right-hand side of the field equations is a table of numbers called the energy-momentum tensor. It says, at every point, how much energy there is in a unit of volume, how hard the matter pushes in each direction (the pressures), and how energy and momentum flow. This chapter builds that table for the field dirac16complex00 in the author's primordial metric, splits it into a kinetic part and a potential part, defines the energy density, the pressures of 3-space, of the extra times and of the hidden direction, and the equations of state, and derives the two exact conservation identities that the table obeys: one along the time x_4, which

describes how energy flows between the inflating 3-space and the deflating extra times, and one along the hidden direction x_0. Three notebooks compute every statement anew from the Revision record: Notebook 09a builds the tensor at one point and derives the two identities, Notebook 09b studies the condensates (fields that depend on the time only) and their equations of state, and Notebook 09c computes the complete tensor of a condensate, including the entries off the diagonal that the spin connection creates.

9.1 What this chapter does

Why we need this chapter. Cosmologists describe the matter of the universe, at each moment, by two numbers: the **energy density** ρ (energy per unit volume) and the **pressure** p (the push per unit area, which is also energy per unit volume). Their ratio $w = p/\rho$ is called the **equation of state**. Three values are famous: dust (matter at rest whose particles do not push on each other) has $w = 0$, radiation (light) has $w = 1/3$, and a cosmological constant (the simplest model of dark energy) has $w = -1$. **Dark energy** is the name for the unknown cause of the observed speeding-up of the expansion of the universe, and **dark matter** for the unseen matter that is detected only through its gravity. The author's two hypotheses, called Hypothesis (for dirac16complex) and Hypothesis00 (for dirac16complex00), say that these fields might provide a time-varying equation of state for dark energy or dark matter (they are HYPOTHESES; the Revision record investigates them and establishes neither, Chapter 22; this chapter does not test them). Before anyone can test such a statement, one must know exactly what ρ and p are for these fields in the author's eight-dimensional metric. That is what this chapter derives.

What it does. In eight dimensions there is one energy density and seven pressures, one for each direction other than the time x_4. They come in three families: the pressure p_3 of the three directions of ordinary space, the pressure p_t of the three extra times, and the pressure p_0 of the hidden direction. The chapter

- defines the energy-momentum tensor by the response of the action to a change of the metric, and derives from this definition, line by line, its diagonal entries $T^{\mu}{}_{\mu} = L_0 - K_\mu$ (Section 9.5);
- states the tensor that a variation of the vielbein gives and its symmetric part, the complete tensor used in this book, says when the two agree, and proves that the complete tensor is real and that its lowered form is symmetric (Section 9.6);
- splits the energy density and the pressures into kinetic and potential parts and defines the equations of state (Section 9.7);
- derives the trace identity and the values that the field equation imposes (Section 9.8);
- derives the covariant divergence of a diagonal tensor in the author's metric and from it the two conservation identities (Sections 9.9 to 9.11);
- shows where the spin connection, which drops out of the Lagrangian, does enter the tensor (Section 9.12);
- builds the exact condensates, computes their equation of state $w = x/(2 + x)$ with $x = \lambda S/m$, and shows that the energy of the commuting field has no lower bound (Sections 9.18 to 9.21);

- computes the complete tensor of a condensate, with 42 entries off the diagonal, and the exact condensates whose tensor is diagonal (Section 9.26).

What it does not do. The equation of state that an observer living in 3-space would infer (after integrating over the hidden direction and the extra times), its change in time and any comparison with supernova data (the measured brightness of distant exploding stars, from which astronomers infer the equation of state of dark energy) are not computed in this chapter (Section 9.19 quotes the record's result for a condensate). The Revision record computes them in `Revision/dark_sector/` (written up in `Revision/docs/DARK_SECTOR_HYPOTHESES.md`; COMPUTED, and taught in Chapter 22) under an ASSUMED normalisation of the observer's energy density over the extra times (three definitions, called A, B and C), compares them with the supernova fit values of a document of the author (the record calls them the Unite values), and establishes neither hypothesis. Which normalisation, if any, describes a physical observer, and a fit to the supernova data themselves, are OPEN. The anticommuting field dirac16complex obeys the same formulas in its Grassmann algebra; for the quantised field the record defines the tensor as a normal-ordered operator, on a positive space of states constructed only for single good-sector momenta with frozen coefficients, and for $\lambda \neq 0$ it verifies the operator form of the on-shell identity only in a finite model under Wick ordering (Section 9.13 explains these words and states these limits exactly); this chapter computes numbers only for the commuting field dirac16complex00.

Labels. As everywhere in the book, every statement carries one of five labels: PROVED (exact; the Revision report file and the name of its check are given), COMPUTED (a number of a notebook, with its accuracy), ASSUMED, HYPOTHESIS, OPEN.

9.2 The stage: coordinates, metric and the field dirac16complex00

Coordinates. The eight coordinates are named as the author names them: x_1, x_2, x_3 are ordinary 3-space; x_4 is the time in which everything evolves; x_5, x_6, x_7 are the three **extra times**, which behave like time and deflate exponentially; x_0 is the **hidden** space direction. We write $z = 6Hx_0$, where $H > 0$ is the author's constant; z lies between 0 and $\pi/2$. In formulas the letter i always stands for one of the 3-space directions x_1, x_2, x_3, and the letter t for one of the extra times x_5, x_6, x_7.

The metric. The author's metric is diagonal (Chapter 3). We write it with eight positive **vielbein factors** $f_0, \ldots, f_7$:

$$g = \mathrm{diag}(f_0^2,\, f_1^2,\, f_2^2,\, f_3^2,\, -f_4^2,\, -f_5^2,\, -f_6^2,\, -f_7^2),$$

$$f_0 = \cot z, \quad f_1 = f_2 = f_3 = e^{a_4} \sin^{1/6} z, \quad f_4 = 1, \quad f_5 = f_6 = f_7 = e^{-a_4} \sin^{1/6} z.$$

These are the factors on the backgrounds $b_4 = 1$ of the Revision record. The author's metric of 2026-10-14 has a second function $b_4(x_4) > 0$, and then $f_5 = f_6 = f_7 = e^{-a_4} b_4^{1/2} \sin^{1/6} z$ and $\sqrt{|g|} = b_4^{3/2} \cos z$ (Chapters 3, 6 and 7). This chapter works at $b_4 = 1$, as the record's energy-momentum checks do (`Revision/theory/reports/wolfram-field-theory.json`); where the record has the tensor or its identities for a general b_4, the section says so (Sections 9.10 and 9.11). So $g_{x_1x_1} = e^{2a_4} \sin^{1/3} z$, $g_{x_4x_4} = -1$, $g_{x_5x_5} = -e^{-2a_4} \sin^{1/3} z$ and $g_{x_0x_0} = \cot^2 z$, exactly the author's entries. A small step dx_μ along the direction x_μ has the proper length $f_\mu\, |dx_\mu|$; that is

the meaning of f_μ. With the frame metric $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the order $x_0,\dots,x_7$ every entry is $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$. The function $a_4(x_4)$ grows with the time: then the 3-space factor e^{a_4} grows (3-space inflates) and the extra-time factor e^{-a_4} shrinks (the extra times deflate). We write $a_4' = da_4/dx_4$. The examples of this chapter take $a_4 = AHx_4$ with $A = 1$ (the deflating history that the Kohn-Sham part of the book uses as a prescribed background) and, in Notebook 09c, the curved history $a_4 = 0.3x_4 + 0.05x_4^2$; every identity of the chapter holds for every function a_4. Which a_4 the field equations of gravity allow, and with which source, is the subject of Chapter 12.

The volume factor. The product of the eight factors is the volume factor $\sqrt{|g|}$ (the square root of the absolute value of the determinant of the diagonal matrix g, which is the product of its diagonal entries):

$$\begin{aligned}\sqrt{|g|} &= f_1 f_2 f_3\, f_4\, f_5 f_6 f_7\, f_0 \\ &= \left(e^{a_4}\sin^{1/6} z\right)^3 \cdot 1 \cdot \left(e^{-a_4}\sin^{1/6} z\right)^3 \cdot \cot z \\ &= e^{3a_4}\sin^{1/2} z \cdot e^{-3a_4}\sin^{1/2} z \cdot \frac{\cos z}{\sin z} \\ &= \cos z.\end{aligned}$$

The first line is the definition; the second inserts the factors; the third multiplies the powers $((e^{a_4})^3 = e^{3a_4}$, $(\sin^{1/6} z)^3 = \sin^{1/2} z)$ and writes $\cot z = \cos z/\sin z$; the fourth uses $e^{3a_4}e^{-3a_4} = 1$ and $\sin^{1/2} z \sin^{1/2} z = \sin z$. The volume factor does not depend on the time x_4: the growth of 3-space and the shrinking of the extra times cancel exactly (PROVED; `Revision/theory/reports/python-field-theory.json`, check `sqrt_det_g_equals_cos_z`).

The gamma matrices. The book uses the author's eight **real** 16×16 gamma matrices (Chapter 4), stored in the Revision fixture `Revision/algebra/gammas.json` in the order $x_0,\dots,x_7$. We write them $\gamma^{(0)},\dots,\gamma^{(7)}$ ($\gamma^{(4)}$ means $\gamma^{(x_4)}$, and so on). They obey the Clifford relations $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}\cdot 1$. Two consequences are used again and again in this chapter: the square of each gamma is η_{aa} times the unit matrix ($+1$ for x_0, x_1, x_2, x_3 and -1 for $x_4,\dots,x_7$), and two different gammas anticommute, $\gamma^{(a)}\gamma^{(b)} = -\gamma^{(b)}\gamma^{(a)}$ for $a \neq b$. The matrix $C = \gamma^{(0)}\gamma^{(1)}\gamma^{(2)}\gamma^{(3)}$ (Chapter 5) is real and symmetric, $C^2 = 1$, and every product $C\gamma^{(a)}$ is antisymmetric: $(C\gamma^{(a)})^T = -C\gamma^{(a)}$ (PROVED; `Revision/algebra/reports/python-algebra.json`, checks `clifford_relation`, `C_real_symmetric_involution` and `C_gamma_antisymmetric`). The **coordinate gammas** are $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ (no sum), and with a lower index $\gamma_\mu = g_{\mu\mu}\gamma^\mu = \eta_{\mu\mu} f_\mu \gamma^{(\mu)}$.

The field. dirac16complex00 is a column Φ of 16 ordinary (commuting) complex numbers at every point (Chapter 7). Its **Dirac adjoint** is the row $\bar\Phi = \Phi^\dagger C$ (the dagger means: transpose and take the complex conjugate of every entry), and the **scalar** is the number $S = \bar\Phi\Phi = \Phi^\dagger C\Phi$. A number of the form $\bar\Phi M\Phi$, with a fixed 16×16 matrix M, is called a **bilinear** of the field (it contains the field twice: once in the row $\bar\Phi$ and once in the column Φ); S is the simplest bilinear, with $M = 1$. The potential is $U(S) = \frac{\lambda}{2}S^2$, so $U'(S) = \lambda S$; m is the mass.

The spin connection and the covariant derivative (Chapter 6). For the diagonal vielbein of this metric the spin connection is

$$\begin{aligned}\Omega_{x_i} &= \tfrac{1}{2} f_i \left(a_4'\,\gamma^{(i)}\gamma^{(4)} + H\,\gamma^{(i)}\gamma^{(0)}\right), \qquad i = 1,2,3, \\ \Omega_{x_t} &= -\tfrac{1}{2} f_t \left(a_4'\,\gamma^{(4)}\gamma^{(t)} + H\,\gamma^{(t)}\gamma^{(0)}\right), \qquad t = 5,6,7, \\ \Omega_{x_4} &= \Omega_{x_0} = 0,\end{aligned}$$

with $f_i = e^{a_4}\sin^{1/6} z$ and $f_t = e^{-a_4}\sin^{1/6} z$ (Revision record `Revision/theory/field-theory.json`, formula `Omega_components`). The covariant derivatives are $D_\mu\Phi = \partial_\mu\Phi + \Omega_\mu\Phi$ and $D_\mu\bar\Phi = \partial_\mu\bar\Phi - \bar\Phi\,\Omega_\mu$, where ∂_μ is the partial derivative with respect to x_μ.

The Lagrangian. The Lagrangian density is $\mathcal{L} = \sqrt{|g|}\,L_0$ with

$$L_0 = \tfrac{1}{2}\sum_\mu \left(\bar\Phi\gamma^\mu D_\mu\Phi - (D_\mu\bar\Phi)\gamma^\mu\Phi\right) - mS - U(S),$$

and its Euler-Lagrange equation is the **field equation** $\gamma^\mu D_\mu\Phi = V\Phi$ with the **effective mass** $V = m + U'(S) = m + \lambda S$ (Chapter 7). Written out in the author's metric (record formula `field_equation`) it reads

$$\sum_{a=1}^{8}\frac{1}{f_a}\,\gamma^{(a)}\partial_a\Phi + 3H\gamma^{(0)}\Phi = V\Phi,$$

and because $(\gamma^{(4)})^2 = -1$ and $f_4 = 1$ it can be solved for the time derivative (the **evolution form**; multiply by $-\gamma^{(4)}$):

$$\partial_4\Phi = -\gamma^{(4)}\Big[V\Phi - \sum_{a\neq 4}\frac{1}{f_a}\,\gamma^{(a)}\partial_a\Phi - 3H\gamma^{(0)}\Phi\Big].$$

A configuration that satisfies the field equation is said to be **on shell**; one that need not satisfy it is **off shell**. Some identities of this chapter hold off shell (for every configuration), others only on shell; each statement says which.

9.3 Two exact facts about the spin connection of this metric

Two facts about Ω_μ decide where the spin connection appears in the energy-momentum tensor. Both are proved here line by line.

Fact 1: for every direction separately, $\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu = 0$ (no sum). For x_4 and x_0 this is clear, because $\Omega_{x_4} = \Omega_{x_0} = 0$. Take a 3-space direction x_i:

$$\begin{aligned}\gamma^{x_i}\Omega_{x_i} &= \frac{1}{f_i}\gamma^{(i)}\cdot\tfrac{1}{2}f_i(a_4'\gamma^{(i)}\gamma^{(4)} + H\gamma^{(i)}\gamma^{(0)})\\ &= \tfrac{1}{2}(a_4'\,\gamma^{(i)}\gamma^{(i)}\gamma^{(4)} + H\,\gamma^{(i)}\gamma^{(i)}\gamma^{(0)})\\ &= \tfrac{1}{2}(a_4'\gamma^{(4)} + H\gamma^{(0)}).\end{aligned}$$

The first line inserts $\gamma^{x_i} = \gamma^{(i)}/f_i$ and the formula for Ω_{x_i}; the second cancels $1/f_i$ against f_i and multiplies out the bracket; the third uses $\gamma^{(i)}\gamma^{(i)} = \eta_{ii} = +1$. In the other order:

$$\begin{aligned}\Omega_{x_i}\gamma^{x_i} &= \tfrac{1}{2}(a_4'\,\gamma^{(i)}\gamma^{(4)}\gamma^{(i)} + H\,\gamma^{(i)}\gamma^{(0)}\gamma^{(i)})\\ &= \tfrac{1}{2}(-a_4'\,\gamma^{(i)}\gamma^{(i)}\gamma^{(4)} - H\,\gamma^{(i)}\gamma^{(i)}\gamma^{(0)})\\ &= -\tfrac{1}{2}(a_4'\gamma^{(4)} + H\gamma^{(0)}).\end{aligned}$$

The first line is the same insertion; the second exchanges the last two gammas of each product, which costs a minus sign because different gammas anticommute ($\gamma^{(4)}\gamma^{(i)} = -\gamma^{(i)}\gamma^{(4)}$, $\gamma^{(0)}\gamma^{(i)} = -\gamma^{(i)}\gamma^{(0)}$); the third uses $\gamma^{(i)}\gamma^{(i)} = 1$ again. Adding the two results gives zero. Now an extra time x_t, with $\Omega_{x_t} = -\frac{1}{2} f_t (a_4' \gamma^{(4)}\gamma^{(t)} + H\gamma^{(t)}\gamma^{(0)})$:

$$\begin{aligned}\gamma^{x_t}\Omega_{x_t} &= -\tfrac{1}{2}\big(a_4'\,\gamma^{(t)}\gamma^{(4)}\gamma^{(t)} + H\,\gamma^{(t)}\gamma^{(t)}\gamma^{(0)}\big)\\ &= -\tfrac{1}{2}\big(-a_4'\,\gamma^{(t)}\gamma^{(t)}\gamma^{(4)} + H\,\gamma^{(t)}\gamma^{(t)}\gamma^{(0)}\big)\\ &= -\tfrac{1}{2}\big(a_4'\gamma^{(4)} - H\gamma^{(0)}\big) = -\tfrac{a_4'}{2}\gamma^{(4)} + \tfrac{H}{2}\gamma^{(0)}.\end{aligned}$$

The first line inserts $\gamma^{x_t} = \gamma^{(t)}/f_t$ and cancels f_t; the second exchanges $\gamma^{(4)}$ and $\gamma^{(t)}$ in the first product (a minus sign); the third uses $\gamma^{(t)}\gamma^{(t)} = \eta_{tt} = -1$ in both products and multiplies out. In the same way

$$\Omega_{x_t}\gamma^{x_t} = -\tfrac{1}{2}\big(a_4'\,\gamma^{(4)}\gamma^{(t)}\gamma^{(t)} + H\,\gamma^{(t)}\gamma^{(0)}\gamma^{(t)}\big) = -\tfrac{1}{2}\big(-a_4'\gamma^{(4)} + H\gamma^{(0)}\big) = \tfrac{a_4'}{2}\gamma^{(4)} - \tfrac{H}{2}\gamma^{(0)},$$

where the middle step uses $\gamma^{(t)}\gamma^{(t)} = -1$ in the first product and, in the second, $\gamma^{(0)}\gamma^{(t)} = -\gamma^{(t)}\gamma^{(0)}$ followed by $\gamma^{(t)}\gamma^{(t)} = -1$ (two minus signs). The sum is again zero. Fact 1 is PROVED (Revision record `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `gamma_mu_anticommutes_with_Omega_mu_no_sum`).

Fact 2: summed over the directions, $\gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$. Add the results above over the three 3-space directions and the three extra times (the directions x_4 and x_0 give zero):

$$\sum_\mu \gamma^\mu\Omega_\mu = 3\cdot\tfrac{1}{2}\big(a_4'\gamma^{(4)} + H\gamma^{(0)}\big) + 3\cdot\big(-\tfrac{a_4'}{2}\gamma^{(4)} + \tfrac{H}{2}\gamma^{(0)}\big) = \big(\tfrac{3a_4'}{2} - \tfrac{3a_4'}{2}\big)\gamma^{(4)} + \big(\tfrac{3H}{2} + \tfrac{3H}{2}\big)\gamma^{(0)} = 3H\gamma^{(0)}.$$

The terms with a_4' cancel: the three inflating directions and the three deflating directions contribute equal and opposite amounts. The cancellation is due to the deflation. The lead's independent check computes the same sum for a metric in which the extra times inflate like 3-space and finds that a term $3a_4'\gamma^{(4)}$ survives. Both statements are PROVED in the Revision record:

statement	**Revision report**	**check**
$\sum_\mu \gamma^\mu\Omega_\mu = 3H\gamma^{(0)}$	`Revision/theory/reports/python-field-theory.json`	`gamma_mu_Omega_mu_equals_3H_gamma_x0`
the same, computed independently by the lead	`Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`	`gamma_Omega_equals_3H_gamma0`
with inflating extra times a term $3a_4'\gamma^{(4)}$ survives	`Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`	`negative_control_inflating_extra_times`

What Fact 1 does to the Lagrangian. Insert $D_\mu\Phi = \partial_\mu\Phi + \Omega_\mu\Phi$ and $D_\mu\bar\Phi = \partial_\mu\bar\Phi - \bar\Phi\Omega_\mu$ into one term of L_0:

$$\tfrac{1}{2}(\bar\Phi\gamma^\mu D_\mu\Phi - (D_\mu\bar\Phi)\gamma^\mu\Phi) = \tfrac{1}{2}(\bar\Phi\gamma^\mu\partial_\mu\Phi - \partial_\mu\bar\Phi\,\gamma^\mu\Phi) + \tfrac{1}{2}\,\bar\Phi(\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu)\Phi$$
$$= \frac{1}{2f_\mu}(\bar\Phi\gamma^{(\mu)}\partial_\mu\Phi - \partial_\mu\bar\Phi\,\gamma^{(\mu)}\Phi) = K_\mu.$$

The first line collects the terms with Ω_μ (the term $+\bar\Phi\Omega_\mu\gamma^\mu\Phi$ comes from $-(-\bar\Phi\Omega_\mu)\gamma^\mu\Phi$); the second drops them by Fact 1 and writes $\gamma^\mu = \gamma^{(\mu)}/f_\mu$. The number K_μ (no sum) is called the **kinetic term of the direction** x_μ: the part of the Lagrangian that holds the derivative along x_μ. So

$$L_0 = \sum_\mu K_\mu - mS - U(S), \qquad K_\mu = \frac{1}{2f_\mu}(\bar\Phi\gamma^{(\mu)}\partial_\mu\Phi - \partial_\mu\bar\Phi\,\gamma^{(\mu)}\Phi).$$

The spin connection has disappeared from the Lagrangian; it reaches the field equation only through Fact 2 (the term $3H\gamma^{(0)}\Phi$), and the energy-momentum tensor only off its diagonal (Section 9.12).

9.4 What an energy-momentum tensor is

A table of 8×8 numbers. The energy-momentum tensor $T^\nu{}_\mu$ is, at every point, a table of 64 numbers, one for each ordered pair of directions (ν,μ). We write the upper index ν as the row and the lower index μ as the column. The entries on the diagonal ($\nu=\mu$) are the energy density and the pressures; the entries off the diagonal ($\nu\neq\mu$) describe flows: the entries with one index x_4 hold the density of momentum and the flow of energy along the other direction, and the entries with two indices other than x_4 hold shearing stresses. Lowering the upper index with the metric gives $T_{\nu\mu} = g_{\nu\nu}T^\nu{}_\mu$ (the metric is diagonal, so no sum is needed).

A familiar example. In ordinary four-dimensional flat space with the coordinates (x_1,x_2,x_3,x_4), x_4 the time and the metric $\mathrm{diag}(1,1,1,-1)$, a gas at rest with energy density ρ and pressure p has $T^\nu{}_\mu = \mathrm{diag}(p,p,p,-\rho)$. The minus sign in front of ρ comes from $g_{x_4x_4}=-1$: lowering the index gives $T_{x_4x_4} = g_{x_4x_4}T^{x_4}{}_{x_4} = \rho$, the energy density, as in every book on relativity.

The conventions of the book (Revision record, `Revision/SPEC.md` section 4; record formula `equation_of_state_definitions`): the energy density and the pressures are

$$\rho = -T^{x_4}{}_{x_4}, \qquad p_\mu = T^\mu{}_\mu \ \text{(no sum, } \mu\neq x_4), \qquad p_3 = T^{x_1}{}_{x_1}, \quad p_t = T^{x_5}{}_{x_5}, \quad p_0 = T^{x_0}{}_{x_0},$$

and the **equations of state** are the ratios

$$w_3 = \frac{p_3}{\rho}, \qquad w_t = \frac{p_t}{\rho}, \qquad w_0 = \frac{p_0}{\rho}.$$

For a configuration that looks the same in the three directions of 3-space (such a configuration is called **isotropic** in 3-space), $T^{x_1}{}_{x_1} = T^{x_2}{}_{x_2} = T^{x_3}{}_{x_3} = p_3$, and likewise p_t for a configuration isotropic in the extra times. Along an extra time, which is time-like, the entry $T^{x_t}{}_{x_t}$ is called a pressure by definition; it has no everyday meaning.

The definition. The **action** is the integral of the Lagrangian density over the eight coordinates, $S_{\mathrm{act}} = \int \mathcal{L}\, d^8x$. The energy-momentum tensor is defined by how the action responds when the metric is changed a little, everywhere inside a bounded region:

$$\delta S_{\text{act}} = \tfrac{1}{2} \int \sqrt{|g|}\, T^{\mu\nu}\, \delta g_{\mu\nu}\, d^8x$$

for every small change $\delta g_{\mu\nu}$ (record normalisation; `Revision/docs/DIRAC16COMPLEX00_FIELD_THEORY.md` section 8). Here δ means "the first-order change": if a quantity changes from Q to $Q + \varepsilon Q_1 + \varepsilon^2 Q_2 + \dots$ when a small number ε is switched on, then $\delta Q = \varepsilon Q_1$, and the terms with ε^2 and higher powers are dropped. For a spinor field the metric is changed through the vielbein factors, with the 16 components of Φ held fixed (the record's definition is $T^{\nu}{}_{\mu} = e^{b}{}_{\mu}\, \frac{1}{\sqrt{|g|}}\, \delta S_{\text{act}}/\delta e^{b}{}_{\nu}$; check `T_vielbein_variation_closed_form_C` of `Revision/theory/reports/wolfram-field-theory.json` computes it for all 64 entries).

What the definition fixes, and what it does not. The metric is symmetric, $g_{\mu\nu} = g_{\nu\mu}$, so every change $\delta g_{\mu\nu}$ is symmetric too. Split any table $T^{\mu\nu}$ into its symmetric part $\frac{1}{2}(T^{\mu\nu} + T^{\nu\mu})$ and its antisymmetric part $A^{\mu\nu} = \frac{1}{2}(T^{\mu\nu} - T^{\nu\mu})$. In the double sum $\sum_{\mu,\nu} A^{\mu\nu}\, \delta g_{\mu\nu}$ the term (μ, ν) and the term (ν, μ) cancel, because $A^{\nu\mu} = -A^{\mu\nu}$ while $\delta g_{\nu\mu} = \delta g_{\mu\nu}$; so the antisymmetric part never enters the formula above, and the formula fixes only the **symmetric part** of $T^{\mu\nu}$. The vielbein carries more information than the metric (it also fixes the orientation of the frame), and the tensor obtained by varying the vielbein can have an antisymmetric part as well. Section 9.6 states what that part is and which tensor this book uses.

9.5 The diagonal entries, derived from the definition

We now derive the diagonal entries from the definition, using only the product rule and the first terms of a power series.

The change. Choose one direction x_b and a small function $\varepsilon(x)$ that vanishes outside a bounded region. Change the vielbein factor of that direction, $f_b \to f_b\,(1 + \varepsilon)$, keep the other seven factors and keep Φ fixed. The metric is still diagonal. Its entry g_{bb} changes by

$$\delta g_{bb} = \eta_{bb}(f_b^2(1+\varepsilon)^2 - f_b^2) = \eta_{bb} f_b^2\,(2\varepsilon + \varepsilon^2) \;\to\; 2\, g_{bb}\, \varepsilon.$$

The first step writes the new entry minus the old one; the second multiplies out $(1+\varepsilon)^2 = 1 + 2\varepsilon + \varepsilon^2$; the arrow keeps the first order and uses $\eta_{bb} f_b^2 = g_{bb}$. Every other entry is unchanged.

What the definition says. Insert this change into the definition. Only the term $\mu = \nu = b$ of the double sum survives:

$$\delta S_{\text{act}} = \tfrac{1}{2} \int \sqrt{|g|}\, T^{bb} \cdot 2 g_{bb}\, \varepsilon\, d^8x = \int \sqrt{|g|}\, T^{b}{}_{b}\, \varepsilon\, d^8x \qquad \text{(no sum over } b\text{)},$$

because lowering the second index with the diagonal metric gives $T^{bb} g_{bb} = T^{b}{}_{b}$.

What the Lagrangian says. For every diagonal vielbein, not only the author's, the spin connection drops out of the Lagrangian (the Revision record proves this for an arbitrary diagonal vielbein; `Revision/theory/reports/python-field-theory.json`, check `commuting_emt_equals_vielbein_variation_diagonal`). So, with the same kinetic terms as in Section 9.3,

$$\mathcal{L} = (f_0 f_1 \cdots f_7)\Big[\sum_a \frac{1}{2f_a} X_a - mS - U(S)\Big], \qquad X_a = \bar{\Phi}\gamma^{(a)}\partial_a\Phi - \partial_a\bar{\Phi}\,\gamma^{(a)}\Phi.$$

The numbers X_a, S and $U(S)$ contain no vielbein factor (Φ is held fixed and the frame gammas $\gamma^{(a)}$ are constant matrices), so only two places change: the product $f_0\cdots f_7$ becomes $(1+\varepsilon)f_0\cdots f_7$, and the coefficient $1/(2f_b)$ becomes $1/(2f_b(1+\varepsilon))$. To first order $1/(1+\varepsilon) = 1-\varepsilon$ (the series $1/(1+\varepsilon) = 1-\varepsilon+\varepsilon^2-\ldots$, cut after the first order). Therefore, with $K_a = X_a/(2f_a)$,

$$\begin{aligned}\mathcal{L} \;&\to (1+\varepsilon)\sqrt{|g|}\,\Big[\sum_{a\neq b} K_a + (1-\varepsilon)K_b - mS - U\Big] \\ &= (1+\varepsilon)\sqrt{|g|}\,[L_0 - \varepsilon K_b] \\ &= \sqrt{|g|}\,[L_0 + \varepsilon L_0 - \varepsilon K_b] + (\text{terms with } \varepsilon^2),\end{aligned}$$

where the second line uses $\sum_a K_a - mS - U = L_0$, and the third multiplies out. The first-order change is $\delta\mathcal{L} = \sqrt{|g|}\,(L_0 - K_b)\,\varepsilon$, hence

$$\delta S_{\text{act}} = \int \sqrt{|g|}\;(L_0 - K_b)\,\varepsilon\; d^8x.$$

Comparison. The two expressions for δS_{act} agree for every small function ε. If $\int A\,\varepsilon\,d^8x = \int B\,\varepsilon\,d^8x$ for every such ε, then $A = B$ at every point (choose ε concentrated near any point you like; this is the fundamental lemma of the calculus of variations). So

$$T^b{}_b = L_0 - K_b \qquad (\text{no sum},\ b = x_0,\ldots,x_7).$$

This is the record's formula (PROVED; `Revision/theory/reports/wolfram-field-theory.json`, check `T_diagonal_components_C`, and `Revision/theory/field-theory.json`, formula `EMT_diagonal`). It holds off shell: the derivation never used the field equation.

9.6 The whole tensor: its formula, its reality and its symmetry

The tensor from the vielbein variation. The off-diagonal entries need a change of the vielbein that is not diagonal, and with it the change of the spin connection; this calculation is long, and the Revision record carries it out exactly for all 64 entries (PROVED; `Revision/theory/reports/wolfram-field-theory.json`, check `T_vielbein_variation_closed_form_C`; record formula `T_variation` of `Revision/theory/field-theory.json`). Its result, which we call the **variation tensor** T_{var}, is

$$T_{\text{var}}{}^{\nu}{}_{\mu} = \delta^{\nu}_{\mu}\,L_0 - \tfrac{1}{2}\Big(\bar{\Phi}\gamma^{\nu}D_{\mu}\Phi - D_{\mu}\bar{\Phi}\,\gamma^{\nu}\Phi\Big) - \tfrac{1}{4}\sum_{\rho,\lambda} g_{\mu\rho}\,\nabla_{\lambda}\Big(\bar{\Phi}\{\gamma^{\lambda},\Sigma^{\nu\rho}\}\Phi\Big), \qquad \Sigma^{\nu\rho} = \tfrac{1}{4}(\gamma^{\nu}\gamma^{\rho} - \gamma^{\rho}\gamma^{\nu}),$$

where δ^{ν}_{μ} is 1 for $\nu = \mu$ and 0 otherwise, $\{A,B\} = AB + BA$ is the anticommutator and ∇_{λ} is the covariant derivative (the derivative corrected by Christoffel symbols, Section 9.9). The last

term comes from the change of the spin connection; the record calls the bilinear in it the totally antisymmetric **spin density** (it changes sign when two of its three indices are exchanged). We take this formula from the record and do not derive it. Because of its last term, the variation tensor is in general not symmetric: its lowered form $g_{\nu\nu}T_{\mathrm{var}}{}^{\nu}{}_{\mu}$ has an antisymmetric part.

Its symmetric part: the Belinfante tensor. The record proves that the symmetric part of the variation tensor (with both indices raised, $\frac{1}{2}(T^{\nu\rho}_{\mathrm{var}} + T^{\rho\nu}_{\mathrm{var}})$) is, for all 36 pairs $\nu \le \rho$ and for every configuration, solution or not, the **symmetric (Belinfante) energy-momentum tensor** (named after the physicist F. J. Belinfante, who showed how to make the energy-momentum tensor of a spinor field symmetric) (PROVED; same report, check `T_symmetric_part_Belinfante_C`; record formula `T_symmetric`):

$$T^{\nu}{}_{\mu} = \delta^{\nu}_{\mu}\, L_0 - \tfrac{1}{4}\Big(\bar{\Phi}\gamma^{\nu} D_{\mu}\Phi - D_{\mu}\bar{\Phi}\,\gamma^{\nu}\Phi + \bar{\Phi}\gamma_{\mu} D^{\nu}\Phi - D^{\nu}\bar{\Phi}\,\gamma_{\mu}\Phi\Big),$$

where $\gamma_{\mu} = g_{\mu\mu}\gamma^{\mu}$ and $D^{\nu} = g^{\nu\nu}D_{\nu}$ with $g^{\nu\nu} = 1/g_{\nu\nu}$.

When the two tensors agree. On the diagonal they always agree: the diagonal of an antisymmetric table is zero, so $T_{\mathrm{var}}{}^{\mu}{}_{\mu} = T^{\mu}{}_{\mu}$ for every configuration (the 8 entries of Section 9.5). Off the diagonal they differ by the antisymmetric part of T_{var}, and the record proves that this difference is proportional to the field equation: on every solution of the field equation the variation tensor is symmetric and equals $T^{\nu}{}_{\mu}$ in all 64 entries, while for a configuration that is not a solution, in general, only the 8 diagonal entries agree (PROVED; `Revision/theory/reports/python-field-theory.json`, check `commuting_emt_equals_general_vielbein_variation_on_shell`, which compares all 64 entries of a general first-order vielbein variation around the author's metric). The reason is the invariance of the action under a rotation of the frame at fixed metric (local Lorentz invariance, in the record's words).

Which tensor the book uses. This book, like the record's formulas for the energy density and the pressures, uses the symmetric tensor $T^{\nu}{}_{\mu}$ above as "the" energy-momentum tensor: on a solution it is the variation tensor, and for a configuration that is not a solution it is the symmetric part of the variation tensor. Notebook 09a evaluates it for random values of Φ and its derivatives, which do not solve the field equation; that is why its table $T_{\nu\mu}$ comes out symmetric, while the variation tensor of values that do not solve the field equation is in general not symmetric. Notebooks 09b and 09c evaluate it on exact solutions, where the two tensors are the same. We check two consequences of the formula here.

Its diagonal agrees with Section 9.5. Set $\nu = \mu$. Then $\gamma_{\mu}D^{\mu} = g_{\mu\mu}\gamma^{\mu}g^{\mu\mu}D_{\mu} = \gamma^{\mu}D_{\mu}$, so the last two terms in the bracket equal the first two, and

$$\begin{aligned} T^{\mu}{}_{\mu} &= L_0 - \tfrac{1}{4}\cdot 2(\bar{\Phi}\gamma^{\mu}D_{\mu}\Phi - D_{\mu}\bar{\Phi}\,\gamma^{\mu}\Phi) \\ &= L_0 - \tfrac{1}{2}(\bar{\Phi}\gamma^{\mu}\partial_{\mu}\Phi - \partial_{\mu}\bar{\Phi}\,\gamma^{\mu}\Phi) - \tfrac{1}{2}\,\bar{\Phi}\{\gamma^{\mu}, \Omega_{\mu}\}\Phi \\ &= L_0 - K_{\mu}. \end{aligned}$$

The first line uses $\delta^{\mu}_{\mu} = 1$ and the equality of the two pairs; the second inserts the covariant derivatives as in Section 9.3 ($\{A, B\} = AB + BA$ is the anticommutator); the third uses Fact 1 and the definition of K_{μ}. This is the result of Section 9.5, as it must be.

Symmetry. Multiply by $g_{\nu\nu}$. Because $g_{\nu\nu}\gamma^{\nu} = \gamma_{\nu}$ and $g_{\nu\nu}D^{\nu} = D_{\nu}$,

$$T_{\nu\mu} = g_{\nu\nu}T^{\nu}{}_{\mu} = g_{\nu\mu}L_0 - \tfrac{1}{4}\Big(\bar{\Phi}\gamma_\nu D_\mu\Phi - D_\mu\bar{\Phi}\,\gamma_\nu\Phi + \bar{\Phi}\gamma_\mu D_\nu\Phi - D_\nu\bar{\Phi}\,\gamma_\mu\Phi\Big),$$

and this expression is unchanged when ν and μ are exchanged (the first pair of terms becomes the second pair and the reverse; $g_{\nu\mu}$ is symmetric). So the lowered tensor is symmetric, $T_{\nu\mu} = T_{\mu\nu}$, while the mixed table $T^{\nu}{}_{\mu}$ is in general not symmetric as printed (PROVED; `Revision/theory/reports/python-field-theory.json`, check `commuting_emt_symmetric`).

Reality. Every entry is a real number. The proof needs three facts. (a) $D_\nu\bar{\Phi} = (D_\nu\Phi)^\dagger C$: indeed $(D_\nu\Phi)^\dagger C = \partial_\nu\Phi^\dagger C + \Phi^\dagger\Omega_\nu^T C$ (Ω_ν is real, so its dagger is its transpose), and $\Omega_\nu^T C = -C\Omega_\nu$, because Ω_ν is a sum of products $\gamma^{(a)}\gamma^{(b)}$ with $a \neq b$ and real coefficients, and $(\gamma^{(a)}\gamma^{(b)})^T C = \gamma^{(b)T}\gamma^{(a)T}C = -\gamma^{(b)T}C\gamma^{(a)} = C\gamma^{(b)}\gamma^{(a)} = -C\gamma^{(a)}\gamma^{(b)}$ (each step uses $\gamma^{(a)T}C = -C\gamma^{(a)}$, the transpose of the antisymmetry of $C\gamma^{(a)}$, and the last step the anticommutation of different gammas); so $(D_\nu\Phi)^\dagger C = \partial_\nu\bar{\Phi} - \bar{\Phi}\Omega_\nu = D_\nu\bar{\Phi}$. (b) Call $Y = \bar{\Phi}\gamma_\mu D_\nu\Phi = \Phi^\dagger(C\gamma_\mu)D_\nu\Phi$, a single number. Its complex conjugate is $Y^* = \Phi^T(C\gamma_\mu)(D_\nu\Phi)^*$ (the matrix $C\gamma_\mu$ is real); a single number equals its own transpose, so $Y^* = (D_\nu\Phi)^\dagger(C\gamma_\mu)^T\Phi = -(D_\nu\Phi)^\dagger C\gamma_\mu\Phi = -D_\nu\bar{\Phi}\,\gamma_\mu\Phi$, by the antisymmetry of $C\gamma_\mu$ and by (a). (c) Hence each pair in the bracket has the form $Y + Y^*$, which is twice the real part of Y, a real number. The same argument with ∂ in place of D shows that each K_μ is real, and $S^* = \Phi^T C\Phi^* = \Phi^\dagger C^T\Phi = S$ shows that S is real. So L_0 and all 64 entries are real (PROVED; `Revision/theory/reports/wolfram-field-theory.json`, check `L_real_C`; Notebook 09a confirms it numerically).

9.7 Energy density, pressures, kinetic and potential parts, equations of state

The energy density. With $\mu = x_4$ in $T^{\mu}{}_{\mu} = L_0 - K_\mu$:

$$\begin{aligned}\rho &= -T^{x_4}{}_{x_4} = -(L_0 - K_4)\\ &= -\Big(\sum_\mu K_\mu - mS - U - K_4\Big)\\ &= -\sum_{\mu\neq x_4} K_\mu + mS + U(S).\end{aligned}$$

The first line is the definition and Section 9.5; the second inserts $L_0 = \sum_\mu K_\mu - mS - U$; the third removes K_4 from the sum, where it cancels, and distributes the minus sign.

The pressures. For every direction $\mu \neq x_4$, in the same way,

$$p_\mu = T^{\mu}{}_{\mu} = L_0 - K_\mu = \sum_{\nu\neq\mu} K_\nu - mS - U(S).$$

Kinetic and potential parts. The terms with derivatives (the K's) form the **kinetic part**; the terms without derivatives, $mS + U(S)$, form the **potential part**. The record's split (formula `EMT_kinetic_potential`) gives:

quantity	kinetic part	potential part	total
energy density $\rho = -T^{x_4}{}_{x_4}$	$-\sum_{\mu\neq x_4} K_\mu$	$mS+U$	$-\sum_{\mu\neq x_4} K_\mu + mS + U$
3-space pressure $p_3 = T^{x_1}{}_{x_1}$	$\sum_{\mu\neq x_1} K_\mu$	$-(mS+U)$	$\sum_{\mu\neq x_1} K_\mu - mS - U$
extra-time pressure $p_t = T^{x_5}{}_{x_5}$	$\sum_{\mu\neq x_5} K_\mu$	$-(mS+U)$	$\sum_{\mu\neq x_5} K_\mu - mS - U$
hidden pressure $p_0 = T^{x_0}{}_{x_0}$	$\sum_{\mu\neq x_0} K_\mu$	$-(mS+U)$	$\sum_{\mu\neq x_0} K_\mu - mS - U$

Four remarks.

- The time derivative $\partial_4\Phi$ does not occur in ρ: the kinetic term K_4 cancels. As for Dirac's electron field, the field equation is of first order in time, and its energy is fixed by the derivatives along the other seven directions and by the potential.
- Each kinetic term carries the weight $1/f_\mu$: $e^{-a_4}\sin^{-1/6} z$ along 3-space (it shrinks as 3-space inflates), 1 along the time, $e^{a_4}\sin^{-1/6} z$ along the extra times (it grows as they deflate) and $\tan z$ along the hidden direction. Figure 09a.1 of Notebook 09a draws these weights.
- The potential part is the same, $-(mS+U)$, in all seven pressures, and $+(mS+U)$ in the energy density.
- Neither the K_μ nor S has a fixed sign, so ρ can be negative (Section 9.21).

All of this is PROVED in the record (`Revision/theory/reports/wolfram-field-theory.json`, check `T_diagonal_components_C`; `Revision/theory/field-theory.json`, formulas `EMT_diagonal` and `EMT_kinetic_potential`) and holds off shell.

9.8 The trace, and what the field equation adds

The trace off shell. The **trace** is the sum of the diagonal entries. Summing $T^\mu{}_\mu = L_0 - K_\mu$ over the eight directions:

$$\begin{aligned}\sum_\mu T^\mu{}_\mu &= 8L_0 - \sum_\mu K_\mu \\ &= 8\Big(\sum_\mu K_\mu - mS - U\Big) - \sum_\mu K_\mu \\ &= 7\sum_\mu K_\mu - 8(mS+U).\end{aligned}$$

The first line adds eight copies of L_0 and subtracts every K_μ once; the second inserts L_0; the third collects the kinetic terms (PROVED; check `EMT_trace_C` of `Revision/theory/reports/wolfram-field-theory.json`).

The kinetic sum. Define the **field-equation expression** $E = \gamma^\mu D_\mu\Phi - V\Phi$ and its adjoint $\bar{E} = (D_\mu\bar{\Phi})\gamma^\mu + V\bar{\Phi}$ (sums over μ); a configuration is on shell exactly when $E = 0$, and then $\bar{E} = 0$ too. Multiply E by $\bar{\Phi}$ from the left and $\bar{E}$ by Φ from the right:

$$\bar{\Phi}E = \sum_\mu \bar{\Phi}\gamma^\mu D_\mu \Phi - VS, \qquad \bar{E}\Phi = \sum_\mu D_\mu \bar{\Phi}\,\gamma^\mu \Phi + VS.$$

Subtract the second from the first and use Section 9.3 (the sum of the differences $\bar{\Phi}\gamma^\mu D_\mu\Phi - D_\mu\bar{\Phi}\gamma^\mu\Phi$ is $2\sum_\mu K_\mu$):

$$\bar{\Phi}E - \bar{E}\Phi = 2\sum_\mu K_\mu - 2VS, \qquad \text{so} \qquad \sum_\mu K_\mu = VS + \tfrac{1}{2}(\bar{\Phi}E - \bar{E}\Phi).$$

This **kinetic-sum identity** holds off shell (PROVED; check `kinetic_sum_on_shell_C`).

On shell. With $E = 0$ and $\bar{E} = 0$, line by line:

$$\begin{aligned}
\sum_\mu K_\mu &= VS = (m + U'(S))S, \\
L_0 &= \sum_\mu K_\mu - mS - U = mS + SU' - mS - U = SU'(S) - U(S), \\
\sum_\mu T^\mu{}_\mu &= 7(m + U')S - 8(mS + U) = -mS + 7SU'(S) - 8U(S).
\end{aligned}$$

The first line is the kinetic sum with $E = \bar{E} = 0$; the second inserts it into L_0 and cancels mS; the third inserts it into the trace. For $U = \frac{\lambda}{2}S^2$, $SU' = \lambda S^2$ and $8U = 4\lambda S^2$, so the trace on shell is $-mS + 7\lambda S^2 - 4\lambda S^2 = -mS + 3\lambda S^2$ (PROVED; `Revision/theory/reports/python-field-theory.json`, check `commuting_trace_on_shell`).

9.9 The covariant divergence of a diagonal tensor

Why a divergence. In flat space, energy that is not created or destroyed obeys a **conservation law**: the change in time of the energy density equals minus the outflow through the walls. In curved space the derivatives must be corrected by the **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$, which say how the coordinate directions turn and stretch from point to point (Chapter 3). The corrected derivative summed over the upper index is the **covariant divergence**

$$\nabla_\mu T^\mu{}_\nu = \sum_\mu \partial_\mu T^\mu{}_\nu + \sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda}\, T^\lambda{}_\nu - \sum_{\mu,\lambda} \Gamma^\lambda{}_{\mu\nu}\, T^\mu{}_\lambda,$$

one number for each of the eight values of ν. **Conservation** means $\nabla_\mu T^\mu{}_\nu = 0$ for all eight. On every solution of the field equation the tensor is conserved: this is a theorem, proved in the record from the invariance of the action under a change of coordinates (the **Noether identity**; PROVED, see the table below). We do not repeat that proof. We ask instead: what does conservation say about a tensor of the simplest kind?

statement	Revision report	check
the Noether identity of coordinate invariance (off shell)	`Revision/theory/reports/wolfram-field-theory.json`	`Noether_identity_diffeomorphisms_C`
$\nabla_\mu T^\mu{}_\nu = 0$ for every solution	`Revision/theory/reports/wolfram-field-theory.json`	`conservation_on_shell_general`
the same for dirac16complex00, exact, all eight ν	`Revision/theory/reports/python-field-theory.json`	`commuting_emt_conservation_on_shell`
a wrong tensor is not conserved (negative control)	`Revision/theory/reports/python-field-theory.json`	`commuting_emt_conservation_negative_control`

The Christoffel symbols of the author's metric. For a metric the symbols are $\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2}\sum_\rho g^{\lambda\rho}(\partial_\mu g_{\rho\nu} + \partial_\nu g_{\rho\mu} - \partial_\rho g_{\mu\nu})$. For a diagonal metric only $\rho = \lambda$ survives and

$$\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu}) \qquad \text{(no sum over } \lambda\text{)}.$$

The symbol is symmetric in μ and ν. Six of them, computed line by line (the entries depend only on x_4, through a_4, and on x_0, through $z = 6Hx_0$, whose derivative is $dz/dx_0 = 6H$):

- $\Gamma^{x_1}{}_{x_1x_4} = \frac{1}{2g_{11}}\partial_4 g_{11} = \frac{1}{2}\,\partial_4 \ln g_{11} = \frac{1}{2}\,\partial_4(2a_4 + \frac{1}{3}\ln\sin z) = a_4'$; the first step keeps the only nonzero derivative, the second uses $\partial(\ln Q) = \partial Q/Q$, the third writes $\ln g_{11} = 2a_4 + \frac{1}{3}\ln\sin z$, and the last differentiates ($\sin z$ does not depend on x_4).
- $\Gamma^{x_4}{}_{x_1x_1} = \frac{1}{2g_{44}}(-\partial_4 g_{11}) = \frac{1}{-2}(-2a_4' e^{2a_4}\sin^{1/3} z) = a_4'\, e^{2a_4}\sin^{1/3} z$; only the third term of the bracket survives ($g_{41} = 0$), $g_{44} = -1$, and the chain rule gives $\partial_4 e^{2a_4} = 2a_4' e^{2a_4}$.
- $\Gamma^{x_0}{}_{x_1x_1} = \frac{1}{2g_{00}}(-\partial_0 g_{11}) = -\frac{\tan^2 z}{2}\, e^{2a_4}\cdot\frac{1}{3}\sin^{-2/3} z\cos z\cdot 6H = -He^{2a_4}\,\frac{\sin^{4/3} z}{\cos z}$; here $1/g_{00} = \tan^2 z$, the chain rule gives $\partial_0 \sin^{1/3} z = \frac{1}{3}\sin^{-2/3} z\cos z\cdot 6H$, and $\tan^2 z\,\sin^{-2/3} z\cos z = \sin^{4/3} z/\cos z$.
- $\Gamma^{x_0}{}_{x_0x_0} = \frac{1}{2}\,\partial_0 \ln\cot^2 z = \partial_0 \ln\cot z = 6H\cdot\frac{-1/\sin^2 z}{\cot z} = -\frac{6H}{\sin z\cos z} = -\frac{12H}{\sin(12Hx_0)}$, using $\frac{d}{dz}\cot z = -1/\sin^2 z$ and $2\sin z\cos z = \sin 2z$.
- $\Gamma^{x_4}{}_{x_5x_5} = \frac{1}{2g_{44}}(-\partial_4 g_{55}) = \frac{1}{-2}(-2a_4' e^{-2a_4}\sin^{1/3} z) = a_4'\, e^{-2a_4}\sin^{1/3} z$; only the third term of the bracket survives ($g_{45} = 0$), $g_{44} = -1$, and the chain rule gives $\partial_4 g_{55} = \partial_4(-e^{-2a_4}\sin^{1/3} z) = -(-2a_4')e^{-2a_4}\sin^{1/3} z = 2a_4' e^{-2a_4}\sin^{1/3} z$ (two minus signs: the one of the entry and the one of the exponent $-2a_4$).
- $\Gamma^{x_0}{}_{x_5x_5} = \frac{1}{2g_{00}}(-\partial_0 g_{55}) = \frac{\tan^2 z}{2}\, e^{-2a_4}\cdot\frac{1}{3}\sin^{-2/3} z\cos z\cdot 6H = He^{-2a_4}\,\frac{\sin^{4/3} z}{\cos z}$; here $-\partial_0 g_{55} = +\partial_0(e^{-2a_4}\sin^{1/3} z)$ because the entry g_{55} carries a minus sign, and the rest is the computation of $\Gamma^{x_0}{}_{x_1x_1}$; the sign is opposite to that of $\Gamma^{x_0}{}_{x_1x_1}$ because g_{55} and g_{11} have opposite signs.

The three remaining kinds, $\Gamma^{x_i}{}_{x_ix_0}$, $\Gamma^{x_t}{}_{x_4x_t}$ and $\Gamma^{x_t}{}_{x_tx_0}$, have the form of the first bullet, a logarithmic derivative of one diagonal entry; the lemma below gives each of them in one line, and Exercise 1

works two of them out in full. The directions x_2, x_3 give the same values as x_1, and x_6, x_7 the same values as x_5, because their entries of the metric are equal. The complete list of the nonzero symbols with $\mu \le \nu$ is (25 symbols: four for each of the six directions x_i and x_t, and one more; why there are exactly 25 is shown after the lemma below):

symbol	value	symbol	value
$\Gamma^{x_i}{}_{x_i x_4}$	a_4'	$\Gamma^{x_t}{}_{x_4 x_t}$	$-a_4'$
$\Gamma^{x_i}{}_{x_i x_0}$	$H\cot z$	$\Gamma^{x_t}{}_{x_t x_0}$	$H\cot z$
$\Gamma^{x_4}{}_{x_i x_i}$	$a_4' e^{2a_4}\sin^{1/3} z$	$\Gamma^{x_4}{}_{x_t x_t}$	$a_4' e^{-2a_4}\sin^{1/3} z$
$\Gamma^{x_0}{}_{x_i x_i}$	$-He^{2a_4}\sin^{4/3} z/\cos z$	$\Gamma^{x_0}{}_{x_t x_t}$	$He^{-2a_4}\sin^{4/3} z/\cos z$
$\Gamma^{x_0}{}_{x_0 x_0}$	$-6H/(\sin z\cos z)$	(none)	(none)

(PROVED; `Revision/theory/field-theory.json`, formula `christoffel_nonzero`, and `Revision/theory/reports/wolfram-field-theory.json`, check `christoffel_count`. Notebook 09a computes the 25 symbols with sympy and compares them with the record one by one.)

A lemma for diagonal metrics. Set $\lambda = \mu$ in the formula above (no sum):

$$\Gamma^{\mu}{}_{\mu\nu} = \frac{1}{2g_{\mu\mu}}\left(\partial_\mu g_{\mu\nu} + \partial_\nu g_{\mu\mu} - \partial_\mu g_{\mu\nu}\right) = \frac{\partial_\nu g_{\mu\mu}}{2g_{\mu\mu}} = \frac{2\eta_{\mu\mu} f_\mu\, \partial_\nu f_\mu}{2\eta_{\mu\mu} f_\mu^2} = \partial_\nu \ln f_\mu .$$

The first and the third term of the bracket cancel; then $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ and the chain rule $\partial_\nu f_\mu^2 = 2 f_\mu \partial_\nu f_\mu$; finally $\partial_\nu f_\mu / f_\mu = \partial_\nu \ln f_\mu$. Summed over μ this gives $\sum_\mu \Gamma^{\mu}{}_{\mu\nu} = \partial_\nu \ln(f_0 \cdots f_7) = \partial_\nu \ln\sqrt{|g|}$. For example, $\Gamma^{x_1}{}_{x_1 x_0} = \partial_0 \ln f_1 = \partial_0 (a_4 + \frac{1}{6}\ln\sin z) = \frac{1}{6}\cdot\frac{\cos z}{\sin z}\cdot 6H = H\cot z$, by the chain rule with $dz/dx_0 = 6H$.

Why there are exactly 25 symbols. Sort the symbols by how many of the three indices λ, μ, ν are equal, and use the diagonal formula. Remember that the entries of the metric depend only on x_4 and x_0, that $g_{44} = -1$ is a constant, and that $g_{00} = \cot^2 z$ does not depend on x_4.

- Three different indices: every entry in the bracket, $g_{\lambda\nu}$, $g_{\lambda\mu}$ and $g_{\mu\nu}$, lies off the diagonal and is zero, so the symbol is zero.
- $\mu = \nu \neq \lambda$: the first two terms of the bracket contain $g_{\lambda\mu} = 0$, so $\Gamma^{\lambda}{}_{\mu\mu} = -\partial_\lambda g_{\mu\mu}/(2g_{\lambda\lambda})$. It is nonzero only when $g_{\mu\mu}$ depends on x_λ: λ is x_4 or x_0, and μ is one of the six directions x_i, x_t, whose entries depend on both ($\mu = x_4$ is excluded because g_{44} is constant, and $\mu = x_0$ with $\lambda = x_4$ because g_{00} does not depend on x_4). That makes $2 \cdot 6 = 12$ symbols.
- λ equal to exactly one of μ, ν (stored as $\Gamma^{\lambda}{}_{\lambda\nu}$ or $\Gamma^{\lambda}{}_{\nu\lambda}$, whichever has the smaller index first): by the lemma it is $\partial_\nu \ln f_\lambda$, nonzero only when f_λ depends on x_ν, that is for λ one of the six directions x_i, x_t and $\nu = x_4$ or x_0 ($f_4 = 1$ is constant, and $f_0 = \cot z$ does not depend on x_4). That makes another $6 \cdot 2 = 12$ symbols.
- All three equal: $\Gamma^{\lambda}{}_{\lambda\lambda} = \partial_\lambda \ln f_\lambda$ by the lemma, nonzero only for $\lambda = x_0$, since $f_0 = \cot z$ is the only factor that depends on its own coordinate. One symbol.

The total is $12 + 12 + 1 = 25$, the count of the table and of the record.

The divergence of a diagonal tensor. Now let $T^{\mu}{}_{\nu}$ be diagonal: $T^{\mu}{}_{\nu} = 0$ for $\mu \neq \nu$. In each of the three sums of the divergence only the terms with a diagonal entry survive:

- first sum: only $\mu=\nu$, giving $\partial_\nu T^\nu{}_\nu$;
- second sum: only $\lambda=\nu$, giving $\sum_\mu \Gamma^\mu{}_{\mu\nu} T^\nu{}_\nu = \sum_\mu (\partial_\nu \ln f_\mu)\, T^\nu{}_\nu$;
- third sum: only $\lambda=\mu$, giving $\sum_\mu \Gamma^\mu{}_{\mu\nu} T^\mu{}_\mu = \sum_\mu (\partial_\nu \ln f_\mu)\, T^\mu{}_\mu$.

Both corrections use only the lemma. Subtracting the third from the second:

$$\nabla_\mu T^\mu{}_\nu = \partial_\nu T^\nu{}_\nu + \sum_\mu (\partial_\nu \ln f_\mu)(T^\nu{}_\nu - T^\mu{}_\mu) \qquad (\text{diagonal } T,\ \text{no sum over } \nu).$$

This formula holds for every diagonal metric and every diagonal tensor. We apply it to the author's metric and to

$$T^\mu{}_\nu = \operatorname{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0),$$

whose four entries are any functions of x_4 and x_0. The logarithmic derivatives we need are, from $\ln f_i = a_4 + \frac{1}{6}\ln\sin z$, $\ln f_t = -a_4 + \frac{1}{6}\ln\sin z$, $\ln f_4 = 0$ and $\ln f_0 = \ln\cot z$:

$$\begin{aligned} &\partial_4 \ln f_i = a_4', \qquad \partial_4 \ln f_t = -a_4', \qquad \partial_4 \ln f_4 = \partial_4 \ln f_0 = 0, \\ &\partial_0 \ln f_i = \partial_0 \ln f_t = \tfrac{1}{6}\cdot\frac{\cos z}{\sin z}\cdot 6H = H\cot z, \qquad \partial_0 \ln f_4 = 0. \end{aligned}$$

The component $\nu = x_4$. Here $T^{x_4}{}_{x_4} = -\rho$:

$$\begin{aligned} \nabla_\mu T^\mu{}_{x_4} &= \partial_4(-\rho) + 3a_4'(-\rho - p_3) + 3(-a_4')(-\rho - p_t) \\ &= -\partial_4\rho - 3a_4'\rho - 3a_4'p_3 + 3a_4'\rho + 3a_4'p_t \\ &= -\partial_4\rho - 3a_4'(p_3 - p_t). \end{aligned}$$

The first line takes the three 3-space directions (weight a_4', entry p_3) and the three extra times (weight $-a_4'$, entry p_t); the directions x_4 and x_0 have weight 0. The second multiplies out; the third cancels $\mp 3a_4'\rho$ and factors.

The component $\nu = x_0$. Here $T^{x_0}{}_{x_0} = p_0$. The direction $\mu = x_0$ gives $(p_0 - p_0) = 0$ whatever its weight, and $\mu = x_4$ has weight 0:

$$\nabla_\mu T^\mu{}_{x_0} = \partial_0 p_0 + 3H\cot z\,(p_0 - p_3) + 3H\cot z\,(p_0 - p_t) = \partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t).$$

The other six components. For $\nu = x_1$ (and likewise x_2, x_3, x_5, x_6, x_7) every term contains a derivative ∂_1 of something that depends only on x_4 and x_0; all vanish: $\nabla_\mu T^\mu{}_{x_1} = 0$ identically.

These are exactly the record's statements (PROVED; the table lists the record files and checks). Notebook 09a derives them again with sympy, from the 25 Christoffel symbols and the general divergence formula, without using the lemma.

statement	Revision record	formula or check
all eight components of the divergence of a diagonal tensor	`Revision/theory/field-theory.json`	formula `energy_exchange`
the same, verified with the Wolfram Language and with sympy	`Revision/theory/reports/wolfram-field-theory.json` and `python-field-theory.json`	`energy_exchange_equation`
the x_4 component, computed independently by the lead	`Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`	`divergence_x4_component`
the x_0 component, computed independently by the lead	the same report	`divergence_x0_component`
the other six components vanish	the same report	`divergence_other_components_zero`

9.10 The x4 identity: energy exchange between 3-space and the extra times

Conservation of a diagonal tensor along x_4 is therefore

$$\frac{\partial \rho}{\partial x_4} = -3a_4' \, (p_3 - p_t).$$

What it says. On a deflating history ($a_4' > 0$) the energy density falls when the 3-space pressure exceeds the extra-time pressure, rises when it is smaller, and stays constant only when $p_3 = p_t$. Energy flows between 3-space and the extra times. Reversing the history ($a_4' \to -a_4'$: the extra times inflate and 3-space deflates) reverses the flow. Note what is absent: in ordinary cosmology the energy density of a gas always falls as space expands, $d\rho/dt = -3\,(\dot{a}/a)\,(\rho + p)$ with the scale factor a, because the volume grows. Here the volume factor $\sqrt{|g|} = \cos z$ does not change with x_4, so no term with ρ itself appears; only the difference of the two pressures drives the energy.

With a changing b_4. For the metric with $b_4(x_4)$ the volume factor is $b_4^{3/2} \cos z$, and the extra times shrink at the rate $a_4' - b_4'/(2b_4)$. The Revision record computes the divergence of a diagonal tensor for this metric (`Revision/theory/field-theory-b4.json`, formula `energy_exchange`; `wolfram-field-theory-b4.json`, check `energy_exchange_equation_b4`): the x_4 component becomes

$$\nabla_\mu T^\mu{}_{x_4} = -\frac{\partial \rho}{\partial x_4} - 3a_4' \, (p_3 - p_t) - \frac{3}{2}\,\frac{b_4'}{b_4}\,(p_t + \rho),$$

so a changing b_4 adds a term of the ordinary cosmological kind, with $\rho + p_t$, for the three extra times; the x_0 component is unchanged. The derivation of this section and Notebook 09a are for $b_4 = 1$.

A first-law reading (an interpretation, not an extra result). At fixed x_0 take a small box of coordinate size 1 along each of the seven directions other than x_4. Its 3-space volume is $V_3 = f_1 f_2 f_3 = e^{3a_4} \sin^{1/2} z$, its extra-time volume $V_t = f_5 f_6 f_7 = e^{-3a_4} \sin^{1/2} z$, and its 7-volume $V_7 = V_3 V_t f_8 = \cos z$ does not change with x_4. The first law of thermodynamics says that the energy

of a box changes by minus the pressure times the change of its volume. Applied separately to the two families of directions, with each change of volume taken relative to the volume of its family,

$$\frac{d}{dx_4}(\rho V_7) = -p_3\,\frac{V_7}{V_3}\frac{dV_3}{dx_4} - p_t\,\frac{V_7}{V_t}\frac{dV_t}{dx_4}.$$

Divide by the constant V_7 and insert $\frac{1}{V_3}\frac{dV_3}{dx_4} = 3a_4'$ and $\frac{1}{V_t}\frac{dV_t}{dx_4} = -3a_4'$ (the logarithmic derivatives of $e^{\pm 3a_4}$): the result is $d\rho/dx_4 = -3a_4'p_3 + 3a_4'p_t$, the conservation identity. While 3-space inflates ($dV_3 > 0$), a positive p_3 takes energy out of the box; while the extra times deflate ($dV_t < 0$), a positive p_t puts energy in. Notebook 09a checks with sympy that this reading is exactly the identity, and Figure 09a.4 draws the three volumes.

Toy fluids (an ILLUSTRATION, not solutions of the field equations of this book). Suppose a source had pressures proportional to its energy density, $p_3 = w_3\rho$ and $p_t = w_t\rho$ with constants w_3, w_t, on the history $a_4 = AHx_4$ ($a_4' = AH$). The identity becomes $d\rho/dx_4 = -3AH(w_3 - w_t)\,\rho$: the rate of change of ρ is a constant times ρ. The function $\rho_0 e^{cx_4}$ has the derivative $c\,\rho_0 e^{cx_4}$, so the solution with $\rho(0) = \rho_0$ is

$$\rho(x_4) = \rho_0\, e^{-3AH(w_3 - w_t)x_4}.$$

With $A = 1$, $H = 0.25$ and $w_3 - w_t = 1/3$, at $x_4 = 8$ the exponent is $-3 \cdot 0.25 \cdot \frac{1}{3} \cdot 8 = -2$ and $\rho(8)/\rho(0) = e^{-2} = 0.135335$, the number that Notebook 09b computes with the Runge-Kutta method. The condensates of Section 9.18 have $p_3 = p_t$ and exchange no energy at all.

9.11 The x0 identity: the balance along the hidden direction

Conservation along x_0 is

$$\frac{\partial p_0}{\partial x_0} = -3H\cot z\,(2p_0 - p_3 - p_t).$$

It is a balance of pressures, like the balance of pressure and weight in a column of air: a change of the hidden pressure along x_0 must be paid for by the difference between p_0 and the mean of the other two pressures.

Entries that do not depend on x_0. Then the left side is zero, and because $\cot z > 0$ for $0 < z < \pi/2$ the bracket must vanish:

$$p_0 = \tfrac{1}{2}(p_3 + p_t).$$

The field equations of gravity force the same condition, $p_3 + p_t = 2p_0$, for every source with which the author's metric solves them (Chapter 12 derives these equations). Add the record's 3-space equation (right-hand side κp_3) and its extra-time equation (right-hand side κp_t): every term with a_4'' appears in the two left-hand sides with opposite signs and cancels, and what remains is exactly twice the left-hand side of the hidden equation (right-hand side κp_0). Hence $\kappa(p_3 + p_t) = 2\kappa p_0$, and for a coupling $\kappa \neq 0$, $p_3 + p_t = 2p_0$ (PROVED; the table names the record's entries and checks). The conservation identity and the field equations of gravity are therefore consistent.

statement	Revision record	entry or check
the 3-space, extra-time and hidden equations	`Revision/field_equations_a4/a4-equations.json`	entry `generalSource`: `space_x1_eq_x2_eq_x3`, `extraTime_x5_eq_x6_eq_x7`, `hidden_x0`
the condition $p_3 + p_t = 2p_0$	`Revision/field_equations_a4/a4-equations.json`	`generalSource.algebraic_condition`
the identity of the Lovelock tensors behind it	`Revision/field_equations_a4/reports/wolfram-a4-report.json`	`algebraic_identity_x1_plus_x5_minus_2x0`
the same, verified independently with sympy	`Revision/field_equations_a4/reports/python-a4-report.json`	`algebraic_identity`

All solutions when $p_3 + p_t = P$ is a constant. Write $q = p_0 - P/2$. Then $2p_0 - P = 2q$, and with $dz = 6H\,dx_0$ the identity becomes

$$6H\frac{dq}{dz} = -3H\cot z \cdot 2q, \qquad \text{that is} \qquad \frac{dq}{dz} + \cot z\; q = 0.$$

Multiply by $\sin z$: $\sin z\,\frac{dq}{dz} + \cos z\; q = \frac{d}{dz}(q\sin z) = 0$ (the product rule read backwards). So $q\sin z$ is a constant c, and every solution is

$$p_0 = \frac{P}{2} + \frac{c}{\sin z}.$$

Only $c = 0$ gives a profile independent of x_0; every other profile grows in size like $1/\sin z$ towards the tip $z = 0$ (Figure 09a.5).

In the Kohn-Sham coordinate. The Kohn-Sham solver of Chapter 15 uses the hidden coordinate $y = \ln(\sin z)/(6H)$, for which $dy/dx_0 = \frac{1}{6H}\cdot\frac{\cos z}{\sin z}\cdot 6H = \cot z$. By the chain rule $\partial_0 p_0 = \cot z\, dp_0/dy$, so the x_0 component of the divergence is $\cot z\,[dp_0/dy + 6Hp_0 - 3H(p_3 + p_t)]$, the form that solver checks (PROVED; `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`, checks `ks_coordinate_jacobian` and `ks_coordinate_form`).

9.12 Where the spin connection does enter the tensor

The spin connection drops out of the Lagrangian and of the diagonal entries (Fact 1). It does enter entries off the diagonal. Take a **homogeneous** configuration: one that depends on the time x_4 only, so that $\partial_\mu\Phi = 0$ for every $\mu \neq x_4$. Consider the piece

$$K^{x_4}{}_{x_1} = \tfrac{1}{2}\big(\bar\Phi\gamma^{x_4}D_{x_1}\Phi - D_{x_1}\bar\Phi\,\gamma^{x_4}\Phi\big)$$

of the entry $T^{x_4}{}_{x_1}$. Line by line:

$$K^{x_4}{}_{x_1} = \tfrac{1}{2}\big(\bar\Phi\gamma^{(4)}\Omega_{x_1}\Phi + \bar\Phi\,\Omega_{x_1}\gamma^{(4)}\Phi\big) = \tfrac{1}{2}\,\bar\Phi\{\gamma^{(4)},\Omega_{x_1}\}\Phi,$$
$$\{\gamma^{(4)},\gamma^{(1)}\gamma^{(4)}\} = \gamma^{(4)}\gamma^{(1)}\gamma^{(4)} + \gamma^{(1)}\gamma^{(4)}\gamma^{(4)} = -\gamma^{(1)}\gamma^{(4)}\gamma^{(4)} + \gamma^{(1)}\gamma^{(4)}\gamma^{(4)} = 0,$$
$$\{\gamma^{(4)},\gamma^{(1)}\gamma^{(0)}\} = \gamma^{(4)}\gamma^{(1)}\gamma^{(0)} + \gamma^{(1)}\gamma^{(0)}\gamma^{(4)} = 2\gamma^{(4)}\gamma^{(1)}\gamma^{(0)},$$
$$K^{x_4}{}_{x_1} = \tfrac{1}{2}\cdot\tfrac{1}{2}f_1\big(a_4'\cdot 0 + H\cdot 2\,\bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi\big) = \tfrac{1}{2}\,e^{a_4}\sin^{1/6} z\; H\;\bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi.$$

The first line uses $\partial_1\Phi = 0$, so $D_{x_1}\Phi = \Omega_{x_1}\Phi$ and $D_{x_1}\bar\Phi = -\bar\Phi\Omega_{x_1}$, and $\gamma^{x_4} = \gamma^{(4)}$ ($f_4 = 1$). The second moves $\gamma^{(4)}$ past $\gamma^{(1)}$ (a minus sign). The third moves $\gamma^{(4)}$ in the second product to the front past $\gamma^{(0)}$ and then past $\gamma^{(1)}$ (two minus signs, so no sign change). The fourth inserts $\Omega_{x_1} = \frac{1}{2}f_1(a_4'\gamma^{(1)}\gamma^{(4)} + H\gamma^{(1)}\gamma^{(0)})$ and the two anticommutators. The a_4' part cancels, the H part survives, and the three-gamma bilinear $\bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi$ is not zero in general: the canonical spin connection contributes to the energy-momentum tensor even though it drops out of the Lagrangian (PROVED; `Revision/theory/reports/python-scope.json` and `wolfram-scope.json`, check `spin_connection_in_the_energy_momentum_tensor`). Section 9.26 and Notebook 09c compute all such entries of a condensate.

9.13 The same tensor for dirac16complex

Everything in Sections 9.3 to 9.12 is algebra with the bilinears $\bar\Phi M\Phi$ and $\bar\Phi M\partial\Phi$, in which the row $\bar\Phi$ always stands to the left of the column. For the anticommuting field dirac16complex (Chapter 7) the same formulas hold, with every bilinear an even element of the Grassmann algebra (a sum of products of two anticommuting numbers; such a product commutes with every element of the algebra, as an ordinary number does); the Revision record verifies them (PROVED; for example the checks `T_diagonal_components_G`, `EMT_trace_G` and `kinetic_sum_on_shell_G` of `Revision/theory/reports/wolfram-field-theory.json` and the check `grassmann_homogeneous_on_shell_rho_p` of `Revision/theory/reports/python-field-theory.json`). After canonical quantisation (Chapter 10; the field becomes an **operator**, a rule that acts on the quantum states) the record defines the tensor as an operator too: every bilinear is replaced by the **normal-ordered** product of the quantised field (the product rearranged so that the operators that remove a particle stand to the right of those that create one, which gives the empty state the energy zero; Chapter 10 defines it exactly), $\hat T^{\nu}{}_{\mu} = :T^{\nu}{}_{\mu}[\hat\Psi,\hat{\bar\Psi}]:$, normal ordered with respect to the waves of the **good sector** (the waves that do not depend on the extra times x_5, x_6, x_7), and the numbers are **expectation values** (the average value that measurements of the operator give in a given quantum state) (record `Revision/docs/DIRAC16COMPLEX_FIELD_THEORY.md`, section 12). Three limits of the record must be stated.

1. The space of quantum states with positive norms on which this normal ordering rests (the positive **Fock space** of Chapter 10, built from states with definite numbers of particles) is constructed and checked only for single good-sector momenta with frozen coefficients, that is, for plane waves of a flat frame at one point, in which the deflation of the extra times does not enter (PROVED for these examples; the table below names the checks). In the author's curved metric the record imposes no boundary condition at the patch end $z = \pi/2$, and without one the good sector has complex frequencies (waves that grow or decay in time instead of oscillating), for $U = 0$ whenever $m^2 < 9H^2$ (PROVED). A space of states with positive norms for the full field is not established (OPEN).

2. For $\lambda \neq 0$ the operator form of the on-shell identity $\sum_\mu \langle :K_\mu: \rangle = \langle :(m + U')S: \rangle$ is verified only in a finite model: the Fock space of one good-sector plane-wave mode set with frozen coefficients (flat frame, volume 1, 16 modes, 2^{16} states). There it holds as an exact operator identity, for the solution of the interacting operator field equation, if and only if the potential and the tensor are both **Wick ordered** (the whole product of field operators is normal ordered at once: the potential as $\frac{\lambda}{2}:SS:$, not as $\frac{\lambda}{2}:S::S:$, and the tensor as a Wick product, not by subtracting its value in the empty state); with every other ordering tested it fails, already in expectation values (PROVED in that model; its report, named in the table below, passes 21 of 21 checks). For the field on a whole slice (many momenta), for the curved x_0 dependence and for the extra-time sector the operator identity is OPEN (the field-equations record of Chapter 12 ASSUMES it), and symmetry and conservation of the quartic operator are not verified.

3. Numerical expectation values of the tensor for many-fermion states are computed only in the Kohn-Sham approximation of Part IV. The finite model of limit 2 gives exact values on the **occupation states** of its one mode set at rest (momentum $k = 0$). An occupation state has each of the 16 modes either empty or holding one **quantum** (one excitation of the quantised field); Chapter 10 splits the 16 modes into 8 particle modes and 8 **antiparticle** modes (an antiparticle has the opposite charge of a particle, and both have positive energy). On an occupation state with n quanta, particles and antiparticles counted together, $\rho = mn + \frac{\lambda}{2}n(n-1)$ and $p = \frac{\lambda}{2}n(n-1)$, so that $\langle :U(S): \rangle = \frac{\lambda}{2}n(n-1)$ differs from $U(\langle :S: \rangle) = \frac{\lambda}{2}n^2$ for $n \geq 1$ and $\lambda \neq 0$ (PROVED in that model; the table below names the check). For example, one particle and one antiparticle at rest ($n = 2$) give $\rho = 2m + \lambda$ and $p = \lambda$, and two particles and one antiparticle ($n = 3$) give $\rho = 3m + 3\lambda$ and $p = 3\lambda$. The model gives no numerical expectation value for a physical many-fermion state.

The records of the three limits:

statement	**Revision report**	**check**
positive Fock space for single good-sector momenta with frozen coefficients	`Revision/theory/reports/wolfram-field-theory.json`	`mode_hamiltonian_good_sector`, `Fock_space_good_sector_example`
the same with sympy, for another momentum	`Revision/theory/reports/python-field-theory.json`	`good_sector_positive_fock_realisation`
complex frequencies of the curved good sector without a boundary condition at $z = \pi/2$	`Revision/theory/reports/wolfram-scope.json` and `python-scope.json`	`good_sector_hermiticity_up_to_the_brane_flux`, `good_sector_x0_independent_modes_without_boundary_condition`
finite model, $\lambda \neq 0$: the identity holds when both are Wick ordered	`Revision/theory/fock_quartic/reports/fock-quartic.json`	`trace_identity_operator_identity_wick`
finite model: it fails for every other ordering tested	the same report	`trace_identity_fails_for_every_other_combination`

statement	Revision report	check
finite model, occupation states with n quanta at rest: $\rho = mn + \frac{\lambda}{2}n(n-1)$, $p = \frac{\lambda}{2}n(n-1)$	the same report	`homogeneous_expectation_values_are_not_U_of_expectation`

This chapter therefore computes numbers only for the commuting field dirac16complex00.

9.14 Example: the tensor at one point and the two identities

The first worked example puts Sections 9.3 to 9.12 on the computer. At one point of the deflating history ($H = 0.25$, $a_4 = AHx_4$ with $A = 1$, so $a_4 = 0.5$ and $a_4' = 0.25$ at $x_4 = 2$, and $z = \pi/4$) it builds the coordinate gammas and the spin connection from the Revision fixture and formula, draws a configuration of dirac16complex00 at random, computes its 64 entries $T^{\nu}{}_{\mu}$ literally from the formula of Section 9.6, and checks every identity of Sections 9.5 to 9.8, first off shell and then after putting the configuration on shell with the evolution form. It checks the connection term of Section 9.12. Then it switches to exact symbols: it computes the 25 Christoffel symbols with sympy, compares them with the record, derives the eight components of the divergence of a diagonal tensor and compares them with the record's formula `energy_exchange` and with the lead's checks, checks the first-law reading and the profiles of the x_0 balance, and draws five figures.

9.15 How to run Notebook 09a

Step 1. What this notebook does and what it needs.

Notebook 09a (The energy-momentum tensor and its conservation identities) is the file `Revision/textbook/notebooks/09a_emt_and_identities.ipynb` of the repository Dirac_claude. It builds the energy-momentum tensor of the commuting field dirac16complex00 at one point of the author's metric from the Revision gamma matrices and spin connection, checks that the tensor is real and symmetric, that its diagonal entries are the energy density and the pressures with their kinetic and potential parts, and the trace and on-shell identities; it then derives with sympy the Christoffel symbols of the metric and the two conservation identities of a diagonal tensor (the energy exchange between 3-space and the extra times along x4, and the balance along the hidden direction x0), reads the first one as a first law of thermodynamics, and draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at

the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 09a_emt_and_identities.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 09a_emt_and_identities.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 09a_emt_and_identities.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/09a.captions.json`

- `Revision/textbook/figures/09a_1_kinetic_weights.png`
- `Revision/textbook/figures/09a_2_tensor_heat_map.png`
- `Revision/textbook/figures/09a_3_diagonal_parts.png`
- `Revision/textbook/figures/09a_4_volumes_first_law.png`
- `Revision/textbook/figures/09a_5_hidden_balance.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the five figures of this notebook are saved and captioned
ALL 27 CHECKS PASSED (notebook 09a)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/09a_emt_and_identities.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 09a_emt_and_identities.ipynb
python -m nbconvert --execute --inplace 09a_emt_and_identities.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "KeyError" with the words "has no check": the notebook asks a Revision report for the verdict of one of its checks, and the report in your copy of the repository does not contain that check: your copy is older or newer than the notebook. Run `git pull` in the repository folder, then run the notebook again.
- The cell of the Christoffel symbols runs for more than a minute: sympy simplifies 288 expressions there; on a slow computer this takes longer; wait until the cell has finished (its number appears in the margin) before you run the next cell.

9.16 Notebook 09a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/09a_emt_and_identities.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook is the first worked example of the chapter on the energy-momentum
tensor. It works with the commuting field dirac16complex00 (16 complex numbers at
every point) in the author's primordial metric, and it

- builds, at one chosen point of the deflating history, the coordinate gamma
  matrices, the spin connection and the energy-momentum tensor $T^\nu{}_\mu$ (an
```

```
  $8 \times 8$ table of numbers) of a field configuration chosen at random;
- checks that the tensor is real and symmetric, that its diagonal entries are
  $T^\mu{}_\mu = L_0 - K_\mu$, and computes from them the energy density $\rho$,
  the pressures $p_3$ (3-space), $p_t$ (extra times) and $p_0$ (hidden direction)
  with their kinetic and potential parts;
- checks the trace identity and the kinetic-sum identity, and what they become
  when the configuration satisfies the field equation (on shell);
- shows where the spin connection does enter the tensor (an off-diagonal entry);
- derives exactly, with sympy, the 25 Christoffel symbols of the metric and the
  covariant divergence of a diagonal tensor: the two conservation identities
  $\partial_4\rho = -3a_4'(p_3 - p_t)$ and
  $\partial_0 p_0 = -3H\cot z (2p_0 - p_3 - p_t)$;
- reads the first identity as a first law of thermodynamics for the 7-volume and
  the second one as a balance of pressures along the hidden direction;
- compares every result that the Revision record also contains with that record
  and draws five teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 09a (The energy-momentum tensor and its conservation identities) is the file
‘Revision/textbook/notebooks/09a_emt_and_identities.ipynb‘ of the repository
Dirac_claude. It builds the energy-momentum tensor of the commuting field
dirac16complex00 at one point of the author’s metric from the Revision gamma matrices and
spin connection, checks that the tensor is real and symmetric, that its diagonal entries
are the energy density and the pressures with their kinetic and potential parts, and the
trace and on-shell identities; it then derives with sympy the Christoffel symbols of the
metric and the two conservation identities of a diagonal tensor (the energy exchange
between 3-space and the extra times along x4, and the balance along the hidden direction
x0), reads the first one as a first law of thermodynamics, and draws five teaching plots.
It needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".
```

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
```

```
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 09a_emt_and_identities.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 09a_emt_and_identities.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 09a_emt_and_identities.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/09a.captions.json‘
- ‘Revision/textbook/figures/09a_1_kinetic_weights.png‘
- ‘Revision/textbook/figures/09a_2_tensor_heat_map.png‘
- ‘Revision/textbook/figures/09a_3_diagonal_parts.png‘
- ‘Revision/textbook/figures/09a_4_volumes_first_law.png‘
- ‘Revision/textbook/figures/09a_5_hidden_balance.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS the five figures of this notebook are saved and captioned
    ALL 27 CHECKS PASSED (notebook 09a)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/09a_emt_and_identities.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 09a_emt_and_identities.ipynb
      python -m nbconvert --execute --inplace 09a_emt_and_identities.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "KeyError" with the words "has no check": the notebook asks a Revision report for the verdict of one of its checks, and the report in your copy of the repository does not

  contain that check: your copy is older or newer than the notebook. Run ‘git pull‘ in
  the repository folder, then run the notebook again.
- The cell of the Christoffel symbols runs for more than a minute: sympy simplifies 288
  expressions there; on a slow computer this takes longer; wait until the cell has
  finished (its number appears in the margin) before you run the next cell.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 09a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 09a (The energy-momentum tensor and its conservation identities) is the file
# Revision/textbook/notebooks/09a_emt_and_identities.ipynb of the repository
# Dirac_claude. It builds the energy-momentum tensor of the commuting field
# dirac16complex00 at one point of the author's metric from the Revision gamma matrices
# and spin connection, checks that the tensor is real and symmetric, that its diagonal
# entries are the energy density and the pressures with their kinetic and potential
# parts, and the trace and on-shell identities; it then derives with sympy the
# Christoffel symbols of the metric and the two conservation identities of a diagonal
# tensor (the energy exchange between 3-space and the extra times along x4, and the
# balance along the hidden direction x0), reads the first one as a first law of
# thermodynamics, and draws five teaching plots. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
```

```
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
```

```
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 09a_emt_and_identities.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
```

```
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 09a_emt_and_identities.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 09a_emt_and_identities.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/09a.captions.json
# - Revision/textbook/figures/09a_1_kinetic_weights.png
# - Revision/textbook/figures/09a_2_tensor_heat_map.png
# - Revision/textbook/figures/09a_3_diagonal_parts.png
# - Revision/textbook/figures/09a_4_volumes_first_law.png
# - Revision/textbook/figures/09a_5_hidden_balance.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the five figures of this notebook are saved and captioned
#     ALL 27 CHECKS PASSED (notebook 09a)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
```

```
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/09a_emt_and_identities.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 09a_emt_and_identities.ipynb
#     python -m nbconvert --execute --inplace 09a_emt_and_identities.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "KeyError" with the words "has no check": the notebook asks a Revision report for the
#   verdict of one of its checks, and the report in your copy of the repository does not
#   contain that check: your copy is older or newer than the notebook. Run git pull in
#   the repository folder, then run the notebook again.
# - The cell of the Christoffel symbols runs for more than a minute: sympy simplifies 288
#   expressions there; on a slow computer this takes longer; wait until the cell has
#   finished (its number appears in the margin) before you run the next cell.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "09a"  # this notebook: chapter 09, example a
```

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 09a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates**: $x_1, x_2, x_3$ are ordinary 3-space; $x_4$ is the time;
  $x_5, x_6, x_7$ are the three extra times; $x_0$ is the hidden space direction.
  In the code, index 0 means $x_1$, index 3 means $x_4$ and index 7 means $x_0$.
- **Tensor** $T^\nu{}_\mu$: a table of $8 \times 8$ numbers at every point, one
  for each pair of directions $(\nu, \mu)$. The upper index $\nu$ is the row, the
  lower index $\mu$ the column. Lowering the upper index with the metric gives
  $T_{\nu\mu} = g_{\nu\nu} T^\nu{}_\mu$ (the metric here is diagonal).
- **Energy-momentum tensor**: the tensor that says how much energy and momentum
  the field has and how they flow. Its diagonal entries are the energy density
  and the pressures; its off-diagonal entries are flows of energy and momentum.
- **Energy density** $\rho = -T^{x_4}{}_{x_4}$: energy per unit (proper) volume.
  **Pressure** in direction $\mu$: $p_\mu = T^\mu{}_\mu$ (no sum).
  **Equation of state**: the ratio $w = p/\rho$.
- **Kinetic term** $K_\mu$: the part of the Lagrangian that holds the derivative
  along the direction $\mu$; **potential energy**: $mS + U(S)$, the part without
  derivatives.
- **Trace**: the sum of the diagonal entries, $T^\mu{}_\mu$ summed over $\mu$.
- **Jet at a point**: the values of the field and of its eight first derivatives
  at one point. The energy-momentum tensor needs nothing else.
- **On shell / off shell**: a configuration is on shell when it satisfies the
  field equation, off shell when it need not.
- **Spin connection** $\Omega_\mu$: the $16 \times 16$ matrix that turns the
  ordinary derivative of the spinor into the covariant one,
  $D_\mu\Phi = \partial_\mu\Phi + \Omega_\mu\Phi$.
- **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$: the numbers that turn
  ordinary derivatives of tensors into covariant ones.
- **Covariant divergence** $\nabla_\mu T^\mu{}_\nu$: the derivative of the
  tensor summed over its upper index, corrected by the Christoffel symbols.
  **Conservation** means $\nabla_\mu T^\mu{}_\nu = 0$.
- **Heat map**: a picture of a table of numbers in which each entry is a coloured
  square; the colour scale says which number each colour means.
```

Text cell:

```
## 4. The physical and mathematical situation

**The metric.** The author's primordial metric is diagonal,
$g = \mathrm{diag}(f_1^2, f_2^2, f_3^2, -f_4^2, -f_5^2, -f_6^2, -f_7^2, f_0^2)$,
with the vielbein factors
$f_1 = f_2 = f_3 = e^{a_4}\sin^{1/6}z$ (3-space, inflating as $a_4$ grows),
$f_4 = 1$ (the time), $f_5 = f_6 = f_7 = e^{-a_4}\sin^{1/6}z$ (the extra times,
deflating exponentially), $f_0 = \cot z$, where $z = 6Hx_0$ lies between $0$ and
$\pi/2$, $H > 0$ is the author's constant and $a_4$ depends on $x_4$. The volume
factor is $\sqrt{|g|} = f_0 f_1 \cdots f_7 = \cos z$: it does not depend on $x_4$.
The Kohn-Sham part of the book uses the deflating history $a_4 = AHx_4$ with
$A > 0$ as a prescribed background; this notebook takes its point on that
```

history.

The field and its Lagrangian. dirac16complex00 is a column Φ of 16
complex numbers at every point. With $\bar\Phi = \Phi^\dagger C$, $S = \bar\Phi\Phi$
and $U(S) = \frac{\lambda}{2}S^2$, the Lagrangian per unit volume is
$L_0 = \frac12\sum_\mu(\bar\Phi\gamma^\mu D_\mu\Phi
- (D_\mu\bar\Phi)\gamma^\mu\Phi) - mS - U(S)$, with the coordinate gammas
$\gamma^\mu = \gamma^{(\mu)}/f_\mu$,
$D_\mu\Phi = \partial_\mu\Phi + \Omega_\mu\Phi$ and
$D_\mu\bar\Phi = \partial_\mu\bar\Phi - \bar\Phi\Omega_\mu$.

The energy-momentum tensor (Revision record, ‘Revision/theory/field-theory.json‘,
formula ‘T_symmetric‘):
$T^\nu{}_\mu = \delta^\nu_\mu L_0 - \frac14(\bar\Phi\gamma^\nu D_\mu\Phi
- D_\mu\bar\Phi \gamma^\nu\Phi + \bar\Phi\gamma_\mu D^\nu\Phi
- D^\nu\bar\Phi \gamma_\mu\Phi)$, with $\gamma_\mu = g_{\mu\mu}\gamma^\mu$ and
$D^\nu = g^{\nu\nu}D_\nu$. The sign convention is $\rho = -T^{x_4}{}_{x_4}$ and
$p_\mu = T^\mu{}_\mu$ (no sum) for $\mu \neq x_4$. This symmetric (Belinfante)
tensor is the symmetric part of the tensor that a variation of the vielbein
gives (record formula ‘T_variation‘, which has an extra spin-density term); the
two are equal on every solution of the field equation, and for other
configurations they agree in general only on the diagonal (record check
‘commuting_emt_equals_general_vielbein_variation_on_shell‘). This notebook uses
the symmetric tensor ‘T_symmetric‘; so the lowered table $T_{\nu\mu}$ computed
below for random values, which do not solve the field equation, is symmetric by
construction.

The diagonal entries. With the kinetic term of direction μ,
$K_\mu = \frac{1}{2f_\mu}(\bar\Phi\gamma^{(\mu)}\partial_\mu\Phi
- \partial_\mu\bar\Phi \gamma^{(\mu)}\Phi)$, the record states
$L_0 = \sum_\mu K_\mu - mS - U$ and $T^\mu{}_\mu = L_0 - K_\mu$, so
$\rho = -\sum_{\mu \neq x_4}K_\mu + mS + U$ and
$p_\mu = \sum_{\nu \neq \mu}K_\nu - mS - U$.

Conservation. On shell $\nabla_\mu T^\mu{}_\nu = 0$ (a theorem of the record,
proved from the invariance of the action under changes of coordinates). For a
diagonal tensor $\mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0)$ whose
entries depend on x_4 and x_0, this notebook derives the two conditions it
becomes.

Both fields. The same formulas hold for the anticommuting field
dirac16complex, whose bilinears are even elements of a Grassmann algebra (the
record verifies them, for example in the checks ‘T_diagonal_components_G‘,
‘EMT_trace_G‘ and ‘kinetic_sum_on_shell_G‘ of
‘Revision/theory/reports/wolfram-field-theory.json‘). After quantisation the record
defines the tensor as a normal-ordered operator; the positive space of quantum
states this needs is constructed only for single good-sector momenta (waves that
do not depend on the extra times) with frozen coefficients, and for
$\lambda \neq 0$ the operator form of the on-shell identity
$\sum_\mu\langle{:}K_\mu{:}\rangle = \langle{:}(m + U')S{:}\rangle$ is verified
only in a finite model of one such mode set, and there only when the potential
and the tensor are Wick (normal) ordered as whole products
(‘Revision/theory/fock_quartic/reports/fock-quartic.json‘, 21 of 21 checks); for
the field on a whole slice it is OPEN. This notebook computes with ordinary

```
complex numbers, that is, for dirac16complex00.
```

Text cell:

```
## 5. The gamma matrices and the matrix C

The next cell reads the Revision fixture ‘Revision/algebra/gammas.json‘, which
holds the author’s real $16 \times 16$ gamma matrices in the order
$x_0, \dots, x_7$, the frame metric $\eta$ and the matrix
$C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$. It also defines the
helper ‘recorded(path, name)‘, which opens a Revision report and returns the
verdict of one of its checks; every check of this notebook that reproduces a
Revision check asks the report for that verdict too.
```

In [2]:

```
import numpy as np  # arrays of numbers, matrices, linear algebra

fixture = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
# gamma[a] is the frame matrix gamma^(xa): gamma[0] = x0, ..., gamma[7] = x7.
gamma = [np.array(rows, dtype=float) for rows in fixture["gamma"]]
eta = np.array(fixture["eta"], dtype=float)  # (+1, +1, +1, +1, -1, -1, -1, -1)
C = np.array(fixture["C"], dtype=float)  # gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
I16 = np.eye(16)  # the 16 x 16 unit matrix
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # the author’s coordinates


def recorded(path, name):
    """The verdict (PASS or FAIL) of the check name in the Revision report path."""
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for item in report_data["checks"]:
        if item["name"] == name:
            return item["verdict"].upper()  # some reports write "pass"
    raise KeyError(f"{path} has no check {name}")


ALGEBRA = "Revision/algebra/reports/python-algebra.json"
# {gamma^a, gamma^b} = gamma^a gamma^b + gamma^b gamma^a must be 2 eta^ab times 1.
clifford = all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                              2 * eta[a] * (a == b) * I16)
               for a in range(8) for b in range(8))
check(clifford and recorded(ALGEBRA, "clifford_relation") == "PASS",
      "the 64 Clifford relations {gamma^a, gamma^b} = 2 eta^ab",
      record=f"{ALGEBRA}, check clifford_relation")
# C is symmetric, C times C is 1, and every C gamma^a is antisymmetric.
c_ok = np.array_equal(C, C.T) and np.array_equal(C @ C, I16) and all(
    np.array_equal((C @ g).T, -(C @ g)) for g in gamma)
check(c_ok and recorded(ALGEBRA, "C_real_symmetric_involution") == "PASS"
      and recorded(ALGEBRA, "C_gamma_antisymmetric") == "PASS",
      "C is real symmetric, C^2 = 1, every C gamma^a is antisymmetric",
      record=f"{ALGEBRA}, checks C_real_symmetric_involution and "
             "C_gamma_antisymmetric")
```

Out [2]:

```
PASS the 64 Clifford relations {gamma^a, gamma^b} = 2 eta^ab
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
PASS C is real symmetric, C^2 = 1, every C gamma^a is antisymmetric
     reproduces Revision/algebra/reports/python-algebra.json, checks
    C_real_symmetric_involution and C_gamma_antisymmetric
```

Text cell:

```
## 6. The author's metric at one point and the weights of the kinetic terms

The next cell fixes the point at which the tensor is built. The author's
constant is $H = 0.25$ (all quantities are pure numbers: lengths in a unit of
our choice, $H$, $m$ and $\lambda$ in the matching powers of it). The history is
the deflating member $a_4 = AHx_4$ with $A = 1$, so $a_4' = AH = 0.25$; the point
is $x_4 = 2$ (hence $a_4 = 0.5$) and $z = \pi/4$. The function `frame_factors`
returns the eight factors $f_a$; the cell checks that $\eta_{aa}f_a^2$ is the
author's metric and that $f_0 f_1 \cdots f_7 = \cos z$.

In the energy-momentum tensor every derivative along $x_\mu$ carries the weight
$1/f_\mu$: $e^{-a_4}\sin^{-1/6}z$ along 3-space, $1$ along the time,
$e^{a_4}\sin^{-1/6}z$ along the extra times and $\tan z$ along the hidden
direction. The figure shows these weights against $a_4$ and against $z$.
```

In [3]:

```
H = 0.25  # the author's constant H
A = 1.0  # the history a4 = A H x4; A > 0: the extra times deflate
m = 1.0  # the mass m of the field
lam = 0.5  # the strength lambda of the potential U(S) = (lambda/2) S^2
X4 = 2.0  # the time x4 of the point
a4 = A * H * X4  # a4 at the point: 0.5
a4p = A * H  # its derivative da4/dx4 (written a4p, "a4 prime"): 0.25
z = np.pi / 4  # z = 6 H x0 at the point


def frame_factors(a4_value, z_value):
    """The eight factors f_a of the diagonal vielbein, in the order x0, ..., x7."""
    space = np.exp(a4_value) * np.sin(z_value) ** (1 / 6)  # e^a4 sin^(1/6) z
    extra = np.exp(-a4_value) * np.sin(z_value) ** (1 / 6)  # e^(-a4) sin^(1/6) z
    return np.array([1 / np.tan(z_value)] + [space] * 3 + [1.0] + [extra] * 3)


f = frame_factors(a4, z)
g_diag = eta * f ** 2  # the diagonal entries g_mumu = eta_mumu f_mu^2
# The author's metric written out: e^(2 a4) sin^(1/3) z, -1, -e^(-2 a4) sin^(1/3) z,
# cot^2 z.
authors = np.array([1 / np.tan(z) ** 2] + [np.exp(2 * a4) * np.sin(z) ** (1 / 3)] * 3
                   + [-1.0] + [-np.exp(-2 * a4) * np.sin(z) ** (1 / 3)] * 3)
THEORY_PY = "Revision/theory/reports/python-field-theory.json"
```

```
check(np.allclose(g_diag, authors, rtol=1e-14, atol=0)
      and recorded(THEORY_PY, "metric_from_vielbein_equals_SPEC") == "PASS",
      "eta_aa f_a^2 is the author's metric at the point",
      record=f"{THEORY_PY}, check metric_from_vielbein_equals_SPEC")
check(abs(np.prod(f) - np.cos(z)) < 1e-14
      and recorded(THEORY_PY, "sqrt_det_g_equals_cos_z") == "PASS",
      "the volume factor f0 f1 ... f7 equals cos z",
      record=f"{THEORY_PY}, check sqrt_det_g_equals_cos_z")
for name, value in zip(NAMES, f):
    report(f"f_{name}", f"{value:.6f}")
```

Out [3]:

```
PASS eta_aa f_a^2 is the author's metric at the point
     reproduces Revision/theory/reports/python-field-theory.json, check
    metric_from_vielbein_equals_SPEC
PASS the volume factor f0 f1 ... f7 equals cos z
     reproduces Revision/theory/reports/python-field-theory.json, check
    sqrt_det_g_equals_cos_z
RESULT f_x0 = 1.000000
RESULT f_x1 = 1.556186
RESULT f_x2 = 1.556186
RESULT f_x3 = 1.556186
RESULT f_x4 = 1.000000
RESULT f_x5 = 0.572489
RESULT f_x6 = 0.572489
RESULT f_x7 = 0.572489
```

Text cell:

```
The next cell draws the weights $1/f_\mu$. Left: against $a_4$ from $-2$ to $2$
at $z = \pi/4$; the 3-space weight falls like $e^{-a_4}$ and the extra-time
weight rises like $e^{a_4}$, so as the extra times deflate their derivatives
count more and more (the time weight and the hidden weight are both 1 there, so
their two lines coincide). Right: against $z$ at $a_4 = 0.5$; the
hidden-direction weight $\tan z$ grows without bound as $z$ approaches $\pi/2$.
```

In [4]:

```
from matplotlib.ticker import NullFormatter  # a tick label that prints nothing

a4_axis = np.linspace(-2.0, 2.0, 401)  # values of a4 for the left panel
z_axis = np.linspace(0.02, 1.50, 400)  # values of z for the right panel
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
weights_a4 = np.array([1 / frame_factors(value, z) for value in a4_axis])
weights_z = np.array([1 / frame_factors(a4, value) for value in z_axis])
for panel, axis, weights in ((left, a4_axis, weights_a4), (right, z_axis, weights_z)):
    panel.plot(axis, weights[:, 1], label="3-space $x_1, x_2, x_3$")
    panel.plot(axis, weights[:, 4], "--", label="time $x_4$")
    panel.plot(axis, weights[:, 5], label="extra times $x_5, x_6, x_7$")
    panel.plot(axis, weights[:, 0], ":", label="hidden direction $x_0$")
```

```
    panel.set_yscale("log")  # equal factors are equal distances on this axis
    panel.yaxis.set_minor_formatter(NullFormatter())  # no labels on minor ticks
    panel.set_ylabel("weight $1/f_\\mu$")
ticks = [0.03, 0.1, 0.2, 0.5, 1, 2, 5, 10]  # plain numbers on the vertical axes
left.set_yticks(ticks[2:7], [str(t) for t in ticks[2:7]])
right.set_yticks(ticks, [str(t) for t in ticks])
left.set_xlabel("$a_4$ (at $z = \\pi/4$)")
right.set_xlabel("$z = 6 H x_0$ (at $a_4 = 0.5$)")
left.set_title("Weights against $a_4$")
right.set_title("Weights against $z$")
left.legend(fontsize=8)
save_figure(fig, "kinetic_weights",
            "The weights $1/f_\\mu$ with which a derivative along each direction "
            "enters the energy-momentum tensor, on a logarithmic vertical axis "
            "(pure numbers). Left: against $a_4$ from $-2$ to $2$ at $z = \\pi/4$; "
            "the 3-space weight $e^{-a_4}\\sin^{-1/6}z$ falls and the extra-time "
            "weight $e^{a_4}\\sin^{-1/6}z$ rises, so derivatives along the "
            "deflating extra times count more as $a_4$ grows; the time weight is "
            "$1$ and the hidden weight is $\\tan z = 1$, so these two lines lie on "
            "top of each other. Right: against $z$ at "
            "$a_4 = 0.5$; the hidden weight $\\tan z$ grows without bound near "
            "$z = \\pi/2$.")
```

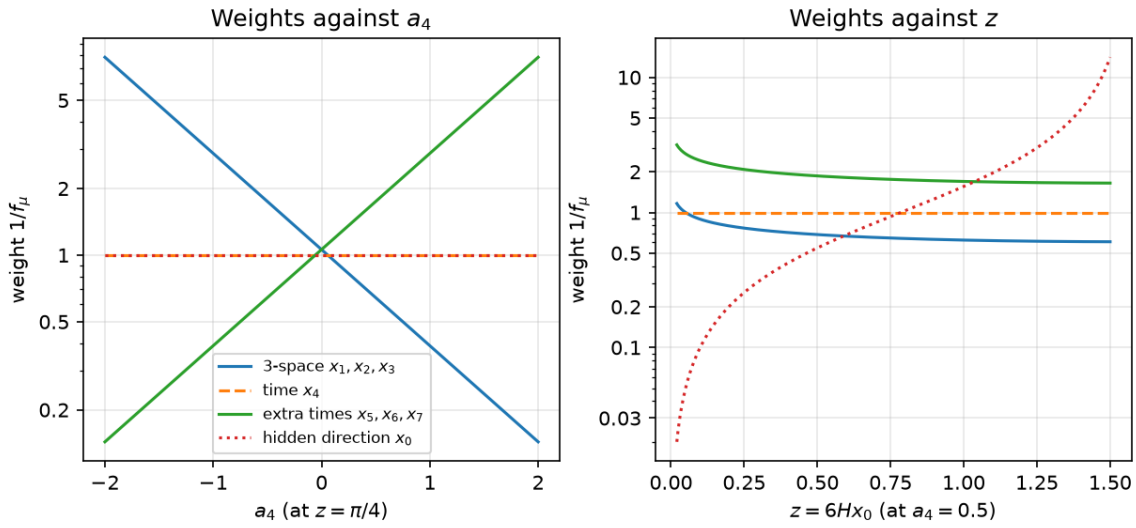

Figure 250: The weights $1/f_\mu$ with which a derivative along each direction enters the energy-momentum tensor, on a logarithmic vertical axis (pure numbers). Left: against a_4 from -2 to 2 at $z = \pi/4$; the 3-space weight $e^{-a_4} \sin^{-1/6} z$ falls and the extra-time weight $e^{a_4} \sin^{-1/6} z$ rises, so derivatives along the deflating extra times count more as a_4 grows; the time weight is 1 and the hidden weight is $\tan z = 1$, so these two lines lie on top of each other. Right: against z at $a_4 = 0.5$; the hidden weight $\tan z$ grows without bound near $z = \pi/2$. (Notebook 09a, figure 1.)

Out [4]:

```
Figure 09a.1 saved as Revision/textbook/figures/09a_1_kinetic_weights.png
```

Text cell:

```
## 7. The spin connection at the point

The record gives the spin connection of the diagonal vielbein in closed form
(‘Revision/theory/field-theory.json‘, formula ‘Omega_components‘):
$\Omega_{x_i} = \frac12 e^{a_4}\sin^{1/6}z (a_4'\gamma^{(i)}\gamma^{(4)}
+ H\gamma^{(i)}\gamma^{(0)})$ for $i = 1, 2, 3$,
$\Omega_{x_t} = -\frac12 e^{-a_4}\sin^{1/6}z (a_4'\gamma^{(4)}\gamma^{(t)}
+ H\gamma^{(t)}\gamma^{(0)})$ for $t = 5, 6, 7$, and
$\Omega_{x_4} = \Omega_{x_0} = 0$. The next cell builds these eight matrices and
the coordinate gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$, and confirms two exact
facts of the record at this point: for each direction separately
$\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu = 0$ (no sum), which is why no spin
connection survives in the diagonal entries of the tensor; and summed over the
directions $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$, the term of the field
equation.
```

In [5]:

```
def spin_connection(a4_value, a4p_value, z_value):
    """The eight 16 x 16 matrices Omega_mu of the record's formula Omega_components."""
    s = np.sin(z_value) ** (1 / 6)  # sin^(1/6) z
    omega = [np.zeros((16, 16)) for _ in range(8)]  # Omega_x4 = Omega_x0 = 0
    for i in range(1, 4):  # the three 3-space directions x1, x2, x3
        omega[i] = 0.5 * np.exp(a4_value) * s * (
            a4p_value * gamma[i] @ gamma[4] + H * gamma[i] @ gamma[0])
    for t in range(5, 8):  # the three extra times x5, x6, x7
        omega[t] = -0.5 * np.exp(-a4_value) * s * (
            a4p_value * gamma[4] @ gamma[t] + H * gamma[t] @ gamma[0])
    return omega


Omega = spin_connection(a4, a4p, z)
gamma_up = [gamma[mu] / f[mu] for mu in range(8)]  # coordinate gammas gamma^mu
gamma_down = [g_diag[mu] * gamma_up[mu] for mu in range(8)]  # gamma_mu
# {gamma^mu, Omega_mu} for each mu separately: the largest entry of all eight.
anti = max(np.abs(gamma_up[mu] @ Omega[mu] + Omega[mu] @ gamma_up[mu]).max()
           for mu in range(8))
A4_REPORT = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
check(anti < 1e-14
      and recorded(A4_REPORT, "gamma_mu_anticommutes_with_Omega_mu_no_sum") == "PASS",
      "{gamma^mu, Omega_mu} = 0 for each direction mu (no sum)",
      record=f"{A4_REPORT}, check gamma_mu_anticommutes_with_Omega_mu_no_sum")
total = sum(gamma_up[mu] @ Omega[mu] for mu in range(8))  # gamma^mu Omega_mu, summed
LEAD = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
check(np.abs(total - 3 * H * gamma[0]).max() < 1e-14
      and recorded(THEORY_PY, "gamma_mu_Omega_mu_equals_3H_gamma_x0") == "PASS"
      and recorded(LEAD, "gamma_Omega_equals_3H_gamma0") == "PASS",
      "gamma^mu Omega_mu = 3 H gamma^(x0) at the point",
      record=f"{THEORY_PY}, check gamma_mu_Omega_mu_equals_3H_gamma_x0; {LEAD}, "
             "check gamma_Omega_equals_3H_gamma0")
```

Out [5]:

```
PASS {gamma^mu, Omega_mu} = 0 for each direction mu (no sum)
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    gamma_mu_anticommutes_with_Omega_mu_no_sum
PASS gamma^mu Omega_mu = 3 H gamma^(x0) at the point
     reproduces Revision/theory/reports/python-field-theory.json, check
    gamma_mu_Omega_mu_equals_3H_gamma_x0; Revision/lead_checks/reports/emt-divergence-
    and-spin-connection.json, check gamma_Omega_equals_3H_gamma0
```

Text cell:

```
## 8. A field configuration at the point and its energy-momentum tensor

The tensor at a point needs only the jet there: the 16 complex values of $\Phi$
and the $8 \times 16$ complex values of its first derivatives
$\partial_\mu\Phi$. The next cell defines two functions. ‘bar‘ returns the Dirac
adjoint $\bar\Phi = \Phi^\dagger C$ as a row of 16 numbers. ‘energy_momentum‘
follows the formula of section 4 literally: from the jet it builds
$D_\mu\Phi$ and $D_\mu\bar\Phi$, the Lagrangian $L_0$ and the 64 entries
$T^\nu{}_\mu$ (row $\nu$, column $\mu$), and also the eight kinetic terms
$K_\mu$ from their own formula (without the spin connection), so that the two
can be compared later. The cell only defines the functions; it prints nothing.
```

In [6]:

```
def bar(vector):
    """The Dirac adjoint: the row vector vector^dagger C."""
    return vector.conj() @ C


def energy_momentum(phi, dphi, f_values, gup, gdown, omega):
    """S, U, L0, the kinetic terms K_mu and the tensor T (T[nu, mu] = T^nu_mu)."""
    g_values = eta * f_values ** 2  # g_mumu
    S = (bar(phi) @ phi).real  # S = Phibar Phi (a real number)
    U = lam / 2 * S ** 2  # U(S) = (lambda/2) S^2
    D = [dphi[mu] + omega[mu] @ phi for mu in range(8)]  # D_mu Phi
    Dbar = [bar(dphi[mu]) - bar(phi) @ omega[mu] for mu in range(8)]  # D_mu Phibar
    L0 = 0.5 * sum(bar(phi) @ gup[mu] @ D[mu] - Dbar[mu] @ gup[mu] @ phi
                   for mu in range(8)) - m * S - U
    T = np.zeros((8, 8), dtype=complex)
    for nu in range(8):
        for mu in range(8):
            bracket = (bar(phi) @ gup[nu] @ D[mu] - Dbar[mu] @ gup[nu] @ phi
                       # gamma_mu D^nu = gamma_mu D_nu / g_nunu
                       + (bar(phi) @ gdown[mu] @ D[nu]
                          - Dbar[nu] @ gdown[mu] @ phi) / g_values[nu])
            T[nu, mu] = (nu == mu) * L0 - bracket / 4
    # K_mu = (1/(2 f_mu)) (Phibar gamma^(mu) d_mu Phi - d_mu Phibar gamma^(mu) Phi)
    K = np.array([(bar(phi) @ gamma[mu] @ dphi[mu] - bar(dphi[mu]) @ gamma[mu] @ phi)
                  / (2 * f_values[mu]) for mu in range(8)])
    return S, U, L0, K, T
```

Text cell:

```
The next cell draws the jet at random, with a fixed seed so that every run draws
the same numbers. The jet need not satisfy the field equation: the identities of
sections 8 to 10 hold off shell. The cell computes the tensor and checks that
every entry is real (its imaginary part is rounding noise) and that
$T_{\nu\mu} = g_{\nu\nu}T^\nu{}_\mu$ is symmetric.
```

In [7]:

```
rng = np.random.default_rng(12345)  # a fixed seed: the same numbers in every run
Phi = rng.normal(size=16) + 1j * rng.normal(size=16)  # the 16 values of Phi
dPhi = rng.normal(size=(8, 16)) + 1j * rng.normal(size=(8, 16))  # d_mu Phi
# np.roll moves the last row to the front: the eight rows are the derivatives along
# x0, x1, ..., x7, and the numbers are those of the earlier edition of this notebook.
dPhi = np.roll(dPhi, 1, axis=0)
S, U, L0, K, T = energy_momentum(Phi, dPhi, f, gamma_up, gamma_down, Omega)
largest_imaginary = max(np.abs(T.imag).max(), np.abs(K.imag).max(), abs(L0.imag))
check(largest_imaginary < 1e-12, "every entry of T, every K_mu and L0 are real")
T, K, L0 = T.real, K.real, L0.real  # keep the real parts
T_low = g_diag[:, None] * T  # T_numu = g_nunu T^nu_mu
check(np.abs(T_low - T_low.T).max() < 1e-12
      and recorded(THEORY_PY, "commuting_emt_symmetric") == "PASS",
      "T_numu is symmetric (64 entries, 28 pairs compared)",
      record=f"{THEORY_PY}, check commuting_emt_symmetric")
report("S = Phibar Phi", f"{S:.6f}")
report("L0", f"{L0:.6f}")
```

Out [7]:

```
PASS every entry of T, every K_mu and L0 are real
PASS T_numu is symmetric (64 entries, 28 pairs compared)
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_emt_symmetric
RESULT S = Phibar Phi = -6.527875
RESULT L0 = 13.616346
```

Text cell:

```
The next cell draws the 64 entries $T^\nu{}_\mu$ as a heat map: row $\nu$,
column $\mu$, red for positive and blue for negative numbers, with the value
written in each square. The diagonal holds the energy density (with a minus
sign, at $x_4$) and the seven pressures; the other squares are flows of energy
and momentum, which a random configuration has in every direction.
```

In [8]:

```
fig, ax = plt.subplots(figsize=(6.6, 5.6))
largest = np.abs(T).max()  # the colour scale runs from -largest to +largest
```

```
picture = ax.imshow(T, cmap="RdBu_r", vmin=-largest, vmax=largest)
for nu in range(8):
    for mu in range(8):
        # white digits on dark squares, black digits on light ones
        colour = "white" if abs(T[nu, mu]) > 0.6 * largest else "black"
        ax.text(mu, nu, f"{T[nu, mu]:.1f}", ha="center", va="center", fontsize=7,
                color=colour)
ax.set_xticks(range(8), [f"${n[0]}_{n[1]}$" for n in NAMES])
ax.set_yticks(range(8), [f"${n[0]}_{n[1]}$" for n in NAMES])
ax.set_xlabel("lower index $\\mu$ (column)")
ax.set_ylabel("upper index $\\nu$ (row)")
ax.set_title("$T^\\nu{}_\\mu$ of one random configuration at one point")
ax.grid(False)  # no grid lines across the squares
fig.colorbar(picture, ax=ax, label="value of $T^\\nu{}_\\mu$ (energy per volume)")
save_figure(fig, "tensor_heat_map",
            "Heat map of the energy-momentum tensor $T^\\nu{}_\\mu$ of a random "
            "configuration of dirac16complex00 at the point $a_4 = 0.5$, "
            "$a_4' = 0.25$, $z = \\pi/4$ of the deflating history, with $H = 0.25$, "
            "$m = 1$, $\\lambda = 0.5$; row $\\nu$ and column $\\mu$ run over "
            "$x_0, \\dots, x_7$; colour and printed number give the value, in units "
            "of energy per unit volume. The diagonal holds $-\\rho$ at $x_4$ and "
            "the seven pressures; the off-diagonal squares are flows of energy and "
            "momentum. The table is not symmetric as printed, because the upper "
            "index is raised with the metric; $g_{\\nu\\nu}T^\\nu{}_\\mu$ is "
            "symmetric.")
```

Out [8]:

```
Figure 09a.2 saved as Revision/textbook/figures/09a_2_tensor_heat_map.png
```

Text cell:

```
## 9. The diagonal entries: energy density, pressures, kinetic and potential parts

The next cell checks the record's statement $T^\mu{}_\mu = L_0 - K_\mu$ for all
eight directions and $L_0 = \sum_\mu K_\mu - mS - U$. It then computes the energy
density and the pressures and splits each into its kinetic part (made of the
$K_\nu$) and its potential part ($mS + U$):
$\rho = \rho_{\rm kin} + \rho_{\rm pot}$ with
$\rho_{\rm kin} = -\sum_{\mu \neq x_4}K_\mu$, $\rho_{\rm pot} = mS + U$, and
$p_\mu = \sum_{\nu \neq \mu}K_\nu - (mS + U)$. A random configuration is not
isotropic, so the three 3-space pressures differ, and so do the three extra-time
pressures. Its energy density may come out negative: the energy of
dirac16complex00 has no lower bound (Revision record
‘Revision/theory/reports/python-scope.json‘, check
‘commuting_field_energy_unbounded_below‘), because both $S$ and the kinetic
terms can have either sign.
```

In [9]:

```
THEORY_WL = "Revision/theory/reports/wolfram-field-theory.json"
check(np.abs(np.diag(T) - (L0 - K)).max() < 1e-12
```

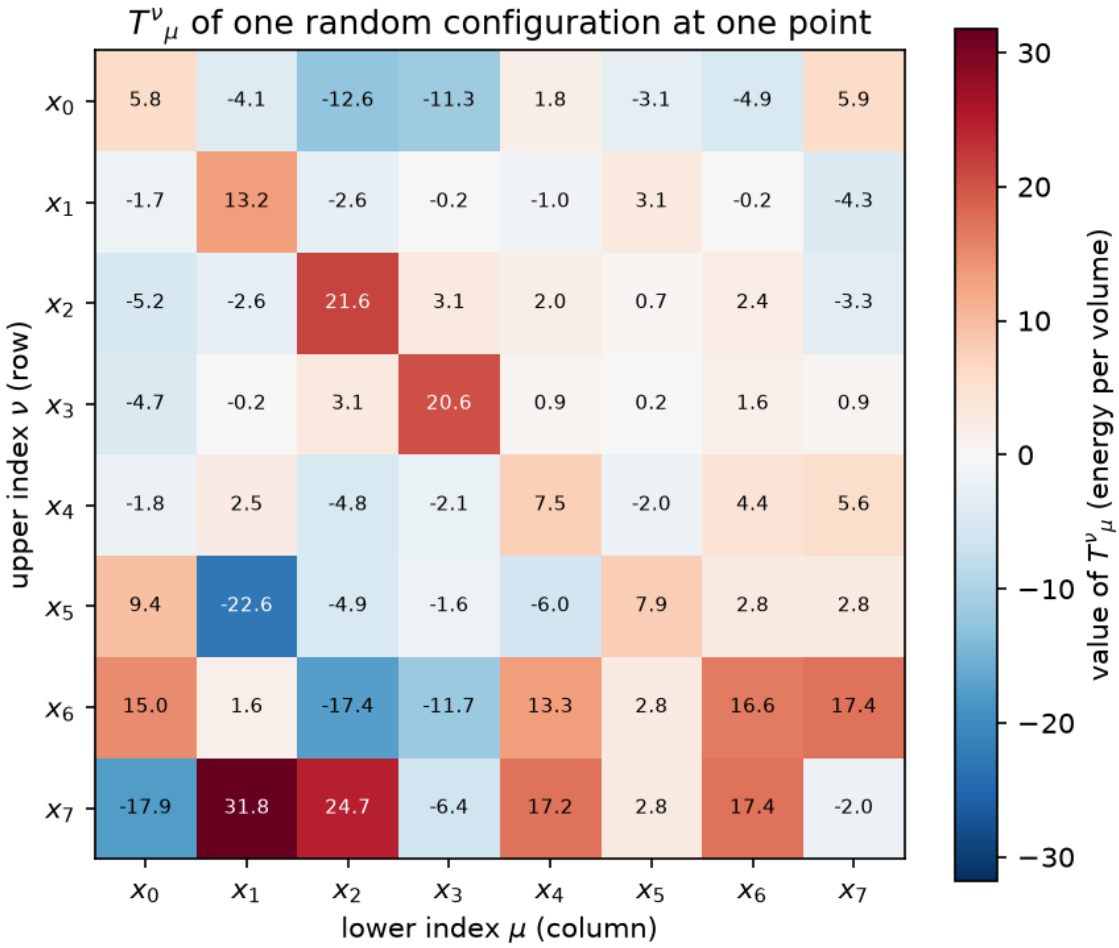

Figure 251: Heat map of the energy-momentum tensor $T^{\nu}{}_{\mu}$ of a random configuration of dirac16complex00 at the point $a_4 = 0.5$, $a_4' = 0.25$, $z = \pi/4$ of the deflating history, with $H = 0.25$, $m = 1$, $\lambda = 0.5$; row ν and column μ run over $x_0, \ldots, x_7$; colour and printed number give the value, in units of energy per unit volume. The diagonal holds $-\rho$ at x_4 and the seven pressures; the off-diagonal squares are flows of energy and momentum. The table is not symmetric as printed, because the upper index is raised with the metric; $g_{\nu\nu}T^{\nu}{}_{\mu}$ is symmetric. (Notebook 09a, figure 2.)

```
      and abs(L0 - (K.sum() - m * S - U)) < 1e-12
      and recorded(THEORY_WL, "T_diagonal_components_C") == "PASS",
      "T^mu_mu = L0 - K_mu for all eight mu, and L0 = sum K - m S - U",
      record=f"{THEORY_WL}, check T_diagonal_components_C")
potential = m * S + U  # the potential energy density m S + U(S)
# kinetic part of T^mu_mu: the sum of the K_nu with nu different from mu
kinetic_parts = np.array([K.sum() - K[mu] for mu in range(8)])
rho = -T[4, 4]  # the energy density
rho_kin = -(K.sum() - K[4])  # minus the sum of the K_mu with mu different from x4
check(abs(rho - (rho_kin + potential)) < 1e-12
      and np.abs(np.diag(T) - (kinetic_parts - potential)).max() < 1e-12,
      "rho = rho_kin + rho_pot and p_mu = (sum of the other K) - (m S + U)")
report("rho = -T^x4_x4", f"{rho:.6f}")
report("rho_kin", f"{rho_kin:.6f}")
report("rho_pot = m S + U", f"{potential:.6f}")
for mu in (0, 1, 2, 3, 5, 6, 7):
    report(f"pressure T^{NAMES[mu]}_{NAMES[mu]}", f"{T[mu, mu]:.6f}")
```

Out [9]:

```
PASS T^mu_mu = L0 - K_mu for all eight mu, and L0 = sum K - m S - U
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    T_diagonal_components_C
PASS rho = rho_kin + rho_pot and p_mu = (sum of the other K) - (m S + U)
RESULT rho = -T^x4_x4 = -7.531406
RESULT rho_kin = -11.656818
RESULT rho_pot = m S + U = 4.125412
RESULT pressure T^x0_x0 = 5.772126
RESULT pressure T^x1_x1 = 13.160483
RESULT pressure T^x2_x2 = 21.578866
RESULT pressure T^x3_x3 = 20.599843
RESULT pressure T^x5_x5 = 7.938088
RESULT pressure T^x6_x6 = 16.584797
RESULT pressure T^x7_x7 = -1.976597
```

Text cell:

```
The next cell draws, for each direction, the kinetic part, the potential part and
their sum, the diagonal entry $T^\mu{}_\mu$. The potential part is the same,
$-(mS + U)$, in every direction; the kinetic parts differ.
```

In [10]:

```
positions = np.arange(8)  # one group of bars per direction
fig, ax = plt.subplots(figsize=(8.0, 4.2))
ax.bar(positions - 0.27, kinetic_parts, width=0.27, label="kinetic part")
ax.bar(positions, [-potential] * 8, width=0.27, label="potential part $-(mS + U)$")
ax.bar(positions + 0.27, np.diag(T), width=0.27, label="total $T^\\mu{}_\\mu$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(positions, [f"${n[0]}_{n[1]}$" for n in NAMES])
ax.set_xlabel("direction $\\mu$")
```

```
ax.set_ylabel("value (energy per unit volume)")
ax.set_title("Diagonal entries $T^\\mu{}_\\mu$ and their two parts")
ax.legend(fontsize=8)
save_figure(fig, "diagonal_parts",
            "The eight diagonal entries $T^\\mu{}_\\mu$ (right bar of each group) "
            "of the random configuration of the previous figure, each split into "
            "its kinetic part, the sum of the kinetic terms $K_\\nu$ of the other "
            "seven directions (left bar), and its potential part $-(mS + U)$ "
            "(middle bar), in units of energy per unit volume. The potential part "
            "is the same in every direction; the entry at $x_4$ is $-\\rho$, and "
            "the others are the pressures $p_3$ (three of them), $p_t$ (three) "
            "and $p_0$.")
```

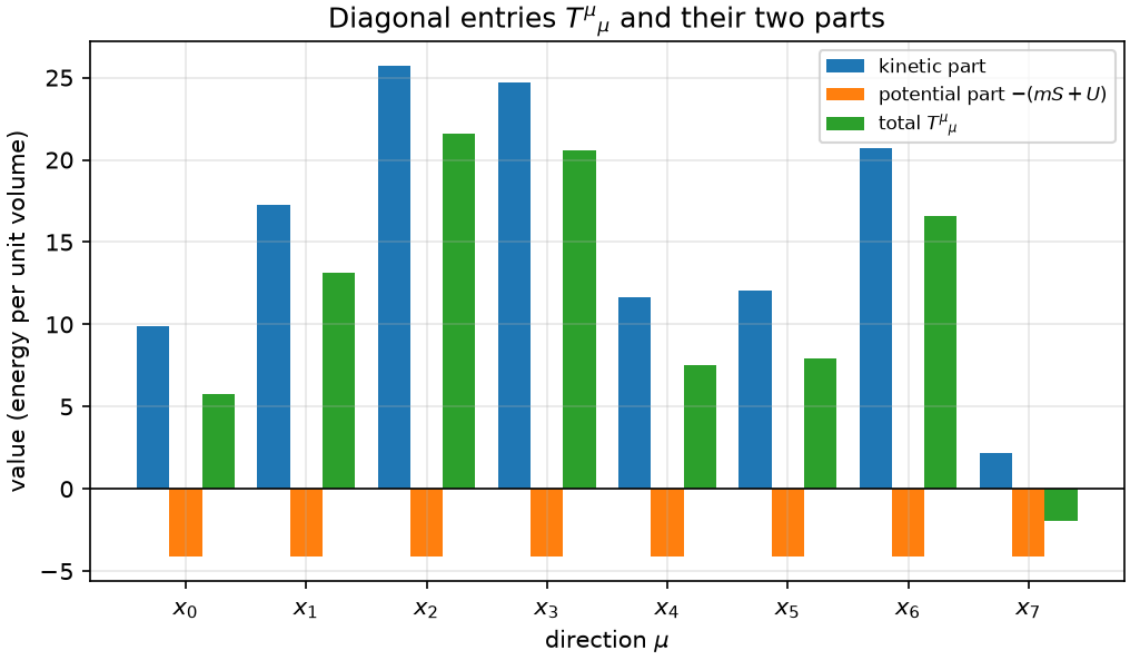

Figure 252: The eight diagonal entries $T^\mu{}_\mu$ (right bar of each group) of the random configuration of the previous figure, each split into its kinetic part, the sum of the kinetic terms K_ν of the other seven directions (left bar), and its potential part $-(mS+U)$ (middle bar), in units of energy per unit volume. The potential part is the same in every direction; the entry at x_4 is $-\rho$, and the others are the pressures p_3 (three of them), p_t (three) and p_0. (Notebook 09a, figure 3.)

Out [10]:

```
Figure 09a.3 saved as Revision/textbook/figures/09a_3_diagonal_parts.png
```

Text cell:

```
## 10. The trace, the kinetic sum, and the configuration put on shell

Summing $T^\mu{}_\mu = L_0 - K_\mu$ over the eight directions gives
```

```
$8L_0 - \sum_\mu K_\mu = 7\sum_\mu K_\mu - 8(mS + U)$ (the record's trace
identity). The record also states, for every configuration, the kinetic-sum
identity $\sum_\mu K_\mu = VS + \frac12(\bar\Phi E - \bar E\Phi)$ with
$V = m + U'(S) = m + \lambda S$, the field-equation expression
$E = \gamma^\mu D_\mu\Phi - V\Phi$ and its adjoint
$\bar E = (D_\mu\bar\Phi)\gamma^\mu + V\bar\Phi$. The next cell checks both.

Then it puts the configuration on shell at the point: the field equation can be
solved for the time derivative (its evolution form),
$\partial_4\Phi = -\gamma^{(4)}\big[V\Phi - \sum_{a \neq 4}\frac{1}{f_a}\gamma^{(a)}
\partial_a\Phi - 3H\gamma^{(0)}\Phi\big]$; the cell replaces the random
$\partial_4\Phi$ by this one, checks that $E = 0$, and checks the on-shell values
$\sum_\mu K_\mu = VS$, $L_0 = SU' - U$ and
$T^\mu{}_\mu = -mS + 7SU' - 8U = -mS + 3\lambda S^2$.
```

In [11]:

```
V = m + lam * S  # V = m + U'(S), with U'(S) = lambda S
E = sum(gamma_up[mu] @ (dPhi[mu] + Omega[mu] @ Phi) for mu in range(8)) - V * Phi
E_bar = sum((bar(dPhi[mu]) - bar(Phi) @ Omega[mu]) @ gamma_up[mu]
            for mu in range(8)) + V * bar(Phi)
check(abs(np.trace(T) - (7 * K.sum() - 8 * (m * S + U))) < 1e-11
      and recorded(THEORY_WL, "EMT_trace_C") == "PASS",
      "trace: sum of T^mu_mu = 7 sum K - 8 (m S + U), off shell",
      record=f"{THEORY_WL}, check EMT_trace_C")
kinetic_sum = V * S + 0.5 * (bar(Phi) @ E - E_bar @ Phi)
check(abs(K.sum() - kinetic_sum) < 1e-11
      and recorded(THEORY_WL, "kinetic_sum_on_shell_C") == "PASS",
      "sum K = V S + (1/2)(Phibar E - Ebar Phi), off shell",
      record=f"{THEORY_WL}, check kinetic_sum_on_shell_C")
report("|E| of the random configuration (it is off shell)",
       f"{np.linalg.norm(E):.6f}")
# The evolution form of the field equation gives d4 Phi from the other derivatives.
rest = sum(gamma[a] @ dPhi[a] / f[a] for a in range(8) if a != 4)
dPhi_on = dPhi.copy()
dPhi_on[4] = -gamma[4] @ (V * Phi - rest - 3 * H * gamma[0] @ Phi)
E_on = sum(gamma_up[mu] @ (dPhi_on[mu] + Omega[mu] @ Phi) for mu in range(8)) - V * Phi
S_on, U_on, L0_on, K_on, T_on = energy_momentum(Phi, dPhi_on, f, gamma_up, gamma_down,
                                                Omega)
check(np.abs(E_on).max() < 1e-12, "with d4 Phi from the evolution form, E = 0")
check(abs(K_on.real.sum() - V * S) < 1e-11
      and abs(L0_on.real - (S * lam * S - U)) < 1e-11
      and abs(np.trace(T_on.real) - (-m * S + 3 * lam * S ** 2)) < 1e-10
      and recorded(THEORY_PY, "commuting_trace_on_shell") == "PASS",
      "on shell: sum K = V S, L0 = S U' - U, trace = -m S + 3 lambda S^2",
      record=f"{THEORY_PY}, check commuting_trace_on_shell")
report("trace on shell, -m S + 3 lambda S^2", f"{np.trace(T_on.real):.6f}")
```

Out [11]:

```
PASS trace: sum of T^mu_mu = 7 sum K - 8 (m S + U), off shell
     reproduces Revision/theory/reports/wolfram-field-theory.json, check EMT_trace_C
```

```
PASS sum K = V S + (1/2)(Phibar E - Ebar Phi), off shell
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    kinetic_sum_on_shell_C
RESULT |E| of the random configuration (it is off shell) = 19.335808
PASS with d4 Phi from the evolution form, E = 0
PASS on shell: sum K = V S, L0 = S U' - U, trace = -m S + 3 lambda S^2
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_trace_on_shell
RESULT trace on shell, -m S + 3 lambda S^2 = 70.447596
```

Text cell:

```
## 11. Where the spin connection does enter the tensor

The spin connection drops out of the Lagrangian and of the diagonal entries.
It does enter off-diagonal entries. Take a homogeneous configuration (it depends
on $x_4$ only, so $\partial_\mu\Phi = 0$ for $\mu \neq x_4$). In the piece
$K^{x_4}{}_{x_1} = \frac12(\bar\Phi\gamma^{x_4}D_{x_1}\Phi
- D_{x_1}\bar\Phi \gamma^{x_4}\Phi)$ of $T^{x_4}{}_{x_1}$ the derivative part
vanishes and only the connection part
$\frac12\bar\Phi\{\gamma^{x_4}, \Omega_{x_1}\}\Phi$ is left; the record states
that it equals $\frac12 e^{a_4}\sin^{1/6}z H \bar\Phi\gamma^{(4)}\gamma^{(1)}
\gamma^{(0)}\Phi$ (the $a_4'$ part cancels). The next cell checks this at the
point.
```

In [12]:

```
dPhi_hom = np.zeros((8, 16), dtype=complex)  # homogeneous: only d4 Phi is nonzero
dPhi_hom[4] = rng.normal(size=16) + 1j * rng.normal(size=16)
D_x1 = dPhi_hom[1] + Omega[1] @ Phi  # D_x1 Phi = Omega_x1 Phi here
Dbar_x1 = bar(dPhi_hom[1]) - bar(Phi) @ Omega[1]  # D_x1 Phibar = -Phibar Omega_x1
K_41 = 0.5 * (bar(Phi) @ gamma_up[4] @ D_x1 - Dbar_x1 @ gamma_up[4] @ Phi)
expected = 0.5 * np.exp(a4) * np.sin(z) ** (1 / 6) * H * (
    bar(Phi) @ gamma[4] @ gamma[1] @ gamma[0] @ Phi)
SCOPE_PY = "Revision/theory/reports/python-scope.json"
SCOPE_WL = "Revision/theory/reports/wolfram-scope.json"
check(abs(K_41 - expected) < 1e-12 and abs(K_41) > 0.01
      and recorded(SCOPE_PY, "spin_connection_in_the_energy_momentum_tensor") == "PASS"
      and recorded(SCOPE_WL, "spin_connection_in_the_energy_momentum_tensor") == "PASS",
      "K^x4_x1 of a homogeneous configuration is the nonzero connection term",
      record=f"{SCOPE_PY} and {SCOPE_WL}, check "
             "spin_connection_in_the_energy_momentum_tensor")
report("K^x4_x1 (homogeneous configuration)", f"{K_41.real:.6f}")
```

Out [12]:

```
PASS K^x4_x1 of a homogeneous configuration is the nonzero connection term
     reproduces Revision/theory/reports/python-scope.json and
    Revision/theory/reports/wolfram-scope.json, check
    spin_connection_in_the_energy_momentum_tensor
RESULT K^x4_x1 (homogeneous configuration) = -0.203311
```

Text cell:

```
## 12. The Christoffel symbols of the author's metric (exact, with sympy)

From here on the notebook computes exactly with symbols. For a diagonal metric
the Christoffel symbols are
$\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\mu
g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu})$
(no sum over $\lambda$). The next cell writes the author's metric with symbols
(a general function $a_4(x_4)$, the symbol $H$) and computes all the symbols with
$\mu \le \nu$ (the symbol is symmetric in $\mu$ and $\nu$), keeping the nonzero
ones in a dictionary. It prints how many it found.
```

In [13]:

```
import sympy as sp  # exact algebra and calculus with symbols

x = sp.symbols("x0:8", real=True)  # the coordinates x0, ..., x7 (x[0] ... x[7])
Hs = sp.symbols("H", positive=True)  # the author's constant, as a symbol
a4_function = sp.Function("a4")(x[4])  # a general function a4(x4)
zs = 6 * Hs * x[0]  # z = 6 H x0
space_entry = sp.exp(2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
extra_entry = -sp.exp(-2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
g = sp.diag(*([sp.cot(zs) ** 2] + [space_entry] * 3 + [-1] + [extra_entry] * 3))
christoffel = {}  # (lambda, mu, nu) -> Gamma^lambda_mu nu, only the nonzero ones
for l in range(8):
    for i in range(8):
        for j in range(i, 8):  # mu <= nu
            value = sp.simplify((sp.diff(g[l, j], x[i]) + sp.diff(g[l, i], x[j])
                                 - sp.diff(g[i, j], x[l])) / (2 * g[l, l]))
            if value != 0:
                christoffel[(l, i, j)] = value
say(f"{len(christoffel)} nonzero Christoffel symbols with mu <= nu")
```

Out [13]:

```
25 nonzero Christoffel symbols with mu <= nu
```

Text cell:

```
The record lists exactly 25 such symbols (‘Revision/theory/field-theory.json‘,
formula ‘christoffel_nonzero‘, written in the Wolfram Language). The next cell
translates that text into sympy's notation with the function ‘from_wolfram‘, by
plain text replacements (for example ‘Sin[‘ becomes ‘sin(‘ and
‘Derivative[1][a4][x4]‘ becomes ‘a4p‘); the function ‘plain‘ writes our own
symbols with the same plain names ‘a4‘ and ‘a4p‘. The cell compares the two lists
symbol by symbol and prints the 25 symbols.
```

In [14]:

```
a4s, a4ps = sp.symbols("a4 a4p", real=True)  # plain symbols for a4 and da4/dx4


def plain(expression):
    """Replace da4/dx4 by the symbol a4p and a4(x4) by the symbol a4."""
    return expression.subs(sp.Derivative(a4_function, x[4]), a4ps).subs(
        a4_function, a4s)


WOLFRAM_TO_SYMPY = [  # (Wolfram text, sympy text), applied in this order
    ("Derivative[1][a4][x4]", "a4p"), ("Derivative[1, 0][rho][x4, x0]", "rho_4"),
    ("Derivative[0, 1][p0][x4, x0]", "p0_0"), ("[x4, x0]", ""), ("a4[x4]", "a4"),
    ("Sin[", "sin("), ("Cos[", "cos("), ("Cot[", "cot("), ("Sec[", "sec("),
    ("Csc[", "csc("), ("E^", "E**"), ("^", "**"), ("[", "("), ("]", ")"),
    ("{", "["), ("}", "]")]


def from_wolfram(text):
    """Translate a formula of the record from the Wolfram Language into sympy."""
    for old, new in WOLFRAM_TO_SYMPY:
        text = text.replace(old, new)
    return text


FORMULAS = {item["key"]: item["wl"] for item in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
NAMES_SYMPY = {"a4": a4s, "a4p": a4ps, "H": Hs, "x0": x[0], "E": sp.E}
record_list = sp.sympify(from_wolfram(FORMULAS["christoffel_nonzero"]),
                         locals=NAMES_SYMPY)
# The record counts the coordinates 1 ... 8; Python counts 0 ... 7.
record_symbols = {(int(e[0]) - 1, int(e[1]) - 1, int(e[2]) - 1): e[3]
                  for e in record_list}
same = sorted(record_symbols) == sorted(christoffel) and all(
    sp.simplify(plain(christoffel[key]) - record_symbols[key]) == 0
    for key in christoffel)
check(len(christoffel) == 25 and same
      and recorded(THEORY_WL, "christoffel_count") == "PASS",
      "25 nonzero Christoffel symbols, equal to the record's list one by one",
      record=f"{THEORY_WL}, check christoffel_count (formula christoffel_nonzero)")
for (l, i, j), value in sorted(christoffel.items()):
    print(f"Gamma^{NAMES[l]}_({NAMES[i]} {NAMES[j]}) = {plain(value)}")
```

Out [14]:

```
PASS 25 nonzero Christoffel symbols, equal to the record's list one by one
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    christoffel_count (formula christoffel_nonzero)
Gamma^x0_(x0 x0) = -12*H/sin(12*H*x0)
Gamma^x0_(x1 x1) = -H*exp(2*a4)*sin(6*H*x0)**(4/3)/cos(6*H*x0)
Gamma^x0_(x2 x2) = -H*exp(2*a4)*sin(6*H*x0)**(4/3)/cos(6*H*x0)
Gamma^x0_(x3 x3) = -H*exp(2*a4)*sin(6*H*x0)**(4/3)/cos(6*H*x0)
Gamma^x0_(x5 x5) = H*exp(-2*a4)*sin(6*H*x0)**(4/3)/cos(6*H*x0)
Gamma^x0_(x6 x6) = H*exp(-2*a4)*sin(6*H*x0)**(4/3)/cos(6*H*x0)
```

```
Gamma^x0_(x7 x7) = H*exp(-2*a4)*sin(6*H*x0)**(4/3)/cos(6*H*x0)
Gamma^x1_(x0 x1) = H/tan(6*H*x0)
Gamma^x1_(x1 x4) = a4p
Gamma^x2_(x0 x2) = H/tan(6*H*x0)
Gamma^x2_(x2 x4) = a4p
Gamma^x3_(x0 x3) = H/tan(6*H*x0)
Gamma^x3_(x3 x4) = a4p
Gamma^x4_(x1 x1) = a4p*exp(2*a4)*sin(6*H*x0)**(1/3)
Gamma^x4_(x2 x2) = a4p*exp(2*a4)*sin(6*H*x0)**(1/3)
Gamma^x4_(x3 x3) = a4p*exp(2*a4)*sin(6*H*x0)**(1/3)
Gamma^x4_(x5 x5) = a4p*exp(-2*a4)*sin(6*H*x0)**(1/3)
Gamma^x4_(x6 x6) = a4p*exp(-2*a4)*sin(6*H*x0)**(1/3)
Gamma^x4_(x7 x7) = a4p*exp(-2*a4)*sin(6*H*x0)**(1/3)
Gamma^x5_(x0 x5) = H/tan(6*H*x0)
Gamma^x5_(x4 x5) = -a4p
Gamma^x6_(x0 x6) = H/tan(6*H*x0)
Gamma^x6_(x4 x6) = -a4p
Gamma^x7_(x0 x7) = H/tan(6*H*x0)
Gamma^x7_(x4 x7) = -a4p
```

Text cell:

```
## 13. The covariant divergence of a diagonal tensor: the two identities

For a tensor $T^\mu{}_\nu$ the covariant divergence is
$\nabla_\mu T^\mu{}_\nu = \partial_\mu T^\mu{}_\nu + \Gamma^\mu{}_{\mu\lambda}
T^\lambda{}_\nu - \Gamma^\lambda{}_{\mu\nu}T^\mu{}_\lambda$ (sums over $\mu$ and
$\lambda$). The next cell takes the diagonal tensor
$\mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0)$ whose four entries are
arbitrary functions of $x_4$ and $x_0$ and computes the eight components with the
Christoffel symbols of the previous section. It prints which components are not
identically zero.
```

In [15]:

```
rho_f, p3_f, pt_f, p0_f = [sp.Function(n)(x[4], x[0]) for n in ("rho", "p3", "pt",
                                                                "p0")]
T_diag = sp.diag(p0_f, p3_f, p3_f, p3_f, -rho_f, pt_f, pt_f, pt_f)  # T^mu_nu


def gamma_symbol(l, i, j):
    """Gamma^l_ij for any order of i and j (zero when it is not in the list)."""
    return christoffel.get((l, min(i, j), max(i, j)), 0)


divergence = []
for nu in range(8):
    value = sum(sp.diff(T_diag[mu, nu], x[mu]) for mu in range(8))
    value += sum(gamma_symbol(mu, mu, l) * T_diag[l, nu]
                 for mu in range(8) for l in range(8))
    value -= sum(gamma_symbol(l, mu, nu) * T_diag[mu, l]
                 for mu in range(8) for l in range(8))
```

```
    divergence.append(sp.simplify(value))
nonzero = [NAMES[nu] for nu in range(8) if divergence[nu] != 0]
say("components of the divergence that are not identically zero: nu = "
    + ", ".join(nonzero))
```

Out [15]:

```
components of the divergence that are not identically zero: nu = x0, x4
```

Text cell:

```
The record (formula ‘energy_exchange‘, and the lead’s independent check) says:
$\nabla_\mu T^\mu{}_{x_4} = -\partial_4\rho - 3a_4’(p_3 - p_t)$,
$\nabla_\mu T^\mu{}_{x_0} = \partial_0 p_0 + 3H\cot z (2p_0 - p_3 - p_t)$, and
the other six components vanish identically. The next cell writes our eight
components with the record’s plain names (‘rho_4‘ for $\partial_4\rho$, ‘p0_0‘
for $\partial_0 p_0$), translates the record’s formula, and checks all eight
components against it and against these two lines.
```

In [16]:

```
rho_4, p0_0, p3s, pts, p0s = sp.symbols("rho_4 p0_0 p3 pt p0", real=True)


def plain_t(expression):
    """Write the derivatives and functions as the plain symbols of the record."""
    expression = expression.subs({sp.Derivative(rho_f, x[4]): rho_4,
                                  sp.Derivative(p0_f, x[0]): p0_0})
    expression = expression.subs({p3_f: p3s, pt_f: pts, p0_f: p0s,
                                  rho_f: sp.Symbol("rho", real=True)})
    return plain(expression)


NAMES_T = dict(NAMES_SYMPY, rho_4=rho_4, p0_0=p0_0, p3=p3s, pt=pts, p0=p0s)
record_divergence = sp.sympify(from_wolfram(FORMULAS["energy_exchange"][0]),
                               locals=NAMES_T)
agree = all(sp.simplify(plain_t(divergence[n]) - record_divergence[n]) == 0
            for n in range(8))
x4_line = -rho_4 - 3 * a4ps * (p3s - pts)
x0_line = p0_0 + 3 * Hs * sp.cot(zs) * (2 * p0s - p3s - pts)
check(agree and recorded(THEORY_WL, "energy_exchange_equation") == "PASS"
      and recorded(THEORY_PY, "energy_exchange_equation") == "PASS",
      "the eight components agree with the record formula energy_exchange",
      record=f"{THEORY_WL} and {THEORY_PY}, check energy_exchange_equation")
check(sp.simplify(plain_t(divergence[4]) - x4_line) == 0
      and recorded(LEAD, "divergence_x4_component") == "PASS",
      "nabla_mu T^mu_x4 = -d4 rho - 3 a4p (p3 - p_t)",
      record=f"{LEAD}, check divergence_x4_component")
check(sp.simplify(plain_t(divergence[0]) - x0_line) == 0
      and recorded(LEAD, "divergence_x0_component") == "PASS",
      "nabla_mu T^mu_x0 = d0 p0 + 3 H cot z (2 p0 - p3 - p_t)",
```

```
      record=f"{LEAD}, check divergence_x0_component")
check(all(divergence[n] == 0 for n in (1, 2, 3, 5, 6, 7))
      and recorded(LEAD, "divergence_other_components_zero") == "PASS",
      "the components x1, x2, x3, x5, x6, x7 vanish identically",
      record=f"{LEAD}, check divergence_other_components_zero")
print("nabla_mu T^mu_x4 =", x4_line)  # rho_4 = d rho/d x4
print("nabla_mu T^mu_x0 =", x0_line)  # p0_0 = d p0/d x0
```

Out [16]:

```
PASS the eight components agree with the record formula energy_exchange
     reproduces Revision/theory/reports/wolfram-field-theory.json and
    Revision/theory/reports/python-field-theory.json, check energy_exchange_equation
PASS nabla_mu T^mu_x4 = -d4 rho - 3 a4p (p3 - p_t)
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check divergence_x4_component
PASS nabla_mu T^mu_x0 = d0 p0 + 3 H cot z (2 p0 - p3 - p_t)
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check divergence_x0_component
PASS the components x1, x2, x3, x5, x6, x7 vanish identically
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check divergence_other_components_zero
nabla_mu T^mu_x4 = -3*a4p*(p3 - pt) - rho_4
nabla_mu T^mu_x0 = 3*H*(2*p0 - p3 - pt)*cot(6*H*x0) + p0_0
```

Text cell:

```
## 14. The x4 identity read as a first law

Conservation, $\nabla_\mu T^\mu{}_{x_4} = 0$, says
$\partial_4\rho = -3a_4'(p_3 - p_t)$. Here is why. At fixed $x_0$ take a small
box of coordinate size 1 in each of the seven directions other than $x_4$. Its
3-space volume is $V_3 = f_1 f_2 f_3 = e^{3a_4}\sin^{1/2}z$, its extra-time
volume $V_t = f_5 f_6 f_7 = e^{-3a_4}\sin^{1/2}z$, and its 7-volume
$V_7 = V_3 V_t f_0 = \cos z$ does not change with $x_4$. The first law of
thermodynamics (energy changes by minus pressure times change of volume), applied
to each family of directions, would say that the energy $\rho V_7$ in the box
changes as
$\frac{d}{dx_4}(\rho V_7) = -p_3\frac{V_7}{V_3}\frac{dV_3}{dx_4}
- p_t\frac{V_7}{V_t}\frac{dV_t}{dx_4}$. The next cell checks with sympy that this
is exactly the conservation identity. This is a reading of the identity (an
interpretation, not an extra result): dividing by the constant $V_7$,
$d\rho = -p_3\,dV_3/V_3 - p_t\,dV_t/V_t$. While 3-space inflates ($dV_3 > 0$) a
positive $p_3$ takes energy out of the box; while the extra times deflate
($dV_t < 0$) a positive $p_t$ puts energy in. The energy density stays constant
only if $p_3 = p_t$.
```

In [17]:

```
f_space = sp.exp(a4_function) * sp.sin(zs) ** sp.Rational(1, 6)  # f1 = f2 = f3
f_extra = sp.exp(-a4_function) * sp.sin(zs) ** sp.Rational(1, 6)  # f5 = f6 = f7
```

```
V3 = f_space ** 3  # e^(3 a4) sin^(1/2) z
Vt = f_extra ** 3  # e^(-3 a4) sin^(1/2) z
V7 = V3 * Vt * sp.cot(zs)  # times f0 = cot z (and f4 = 1)
check(sp.simplify(V7 - sp.cos(zs)) == 0, "V3 Vt f0 = cos z, independent of x4")
first_law = (sp.diff(rho_f * V7, x[4]) + p3_f * V7 / V3 * sp.diff(V3, x[4])
             + pt_f * V7 / Vt * sp.diff(Vt, x[4]))
# The first law says first_law = 0; it must be -V7 times the x4 divergence.
check(sp.simplify(first_law + V7 * divergence[4]) == 0,
      "d(rho V7)/dx4 + p3 (V7/V3) dV3/dx4 + p_t (V7/Vt) dVt/dx4 = -V7 nabla_mu T^mu_x4")
```

Out [17]:

```
PASS V3 Vt f0 = cos z, independent of x4
PASS d(rho V7)/dx4 + p3 (V7/V3) dV3/dx4 + p_t (V7/Vt) dVt/dx4 = -V7 nabla_mu T^mu_x4
```

Text cell:

```
The next cell draws the three volumes along the deflating history
$a_4 = AHx_4$ ($A = 1$, $H = 0.25$) for $x_4$ from $0$ to $8$, each divided by its
value at $x_4 = 0$: $V_3$ grows like $e^{3AHx_4}$, $V_t$ shrinks like
$e^{-3AHx_4}$, and $V_7$ stays 1.
```

In [18]:

```
x4_axis = np.linspace(0.0, 8.0, 401)  # the time x4
growth = np.exp(3 * A * H * x4_axis)  # V3(x4)/V3(0) = e^(3 A H x4)
fig, ax = plt.subplots()
ax.plot(x4_axis, growth, label="3-space volume $V_3$ (inflates)")
ax.plot(x4_axis, 1 / growth, "--", label="extra-time volume $V_t$ (deflates)")
ax.plot(x4_axis, growth / growth, ":", color="black",
        label="7-volume $V_7 = V_3 V_t \\cot z$ (constant)")
ax.set_yscale("log")
ax.set_xlabel("time $x_4$")
ax.set_ylabel("volume divided by its value at $x_4 = 0$")
ax.set_title("Volumes along the history $a_4 = A H x_4$ ($A = 1$, $H = 0.25$)")
ax.legend()
save_figure(fig, "volumes_first_law",
            "The 3-space volume $V_3 = e^{3a_4}\\sin^{1/2}z$ (solid), the "
            "extra-time volume $V_t = e^{-3a_4}\\sin^{1/2}z$ (dashed) and the "
            "7-volume $V_7 = \\cos z$ (dotted) of a coordinate box at fixed $x_0$, "
            "along the deflating history $a_4 = AHx_4$ with $A = 1$, $H = 0.25$, "
            "for $x_4$ from $0$ to $8$, each divided by its value at $x_4 = 0$; "
            "logarithmic vertical axis. The growth of $V_3$ and the shrinking of "
            "$V_t$ cancel exactly. Read as a first law, the conservation identity "
            "$d\\rho/dx_4 = -3a_4'(p_3 - p_t)$ says: while 3-space inflates a "
            "positive $p_3$ takes energy out of the box, while the extra times "
            "deflate a positive $p_t$ puts energy in, and the two balance only "
            "when $p_3 = p_t$.")
```

Out [18]:

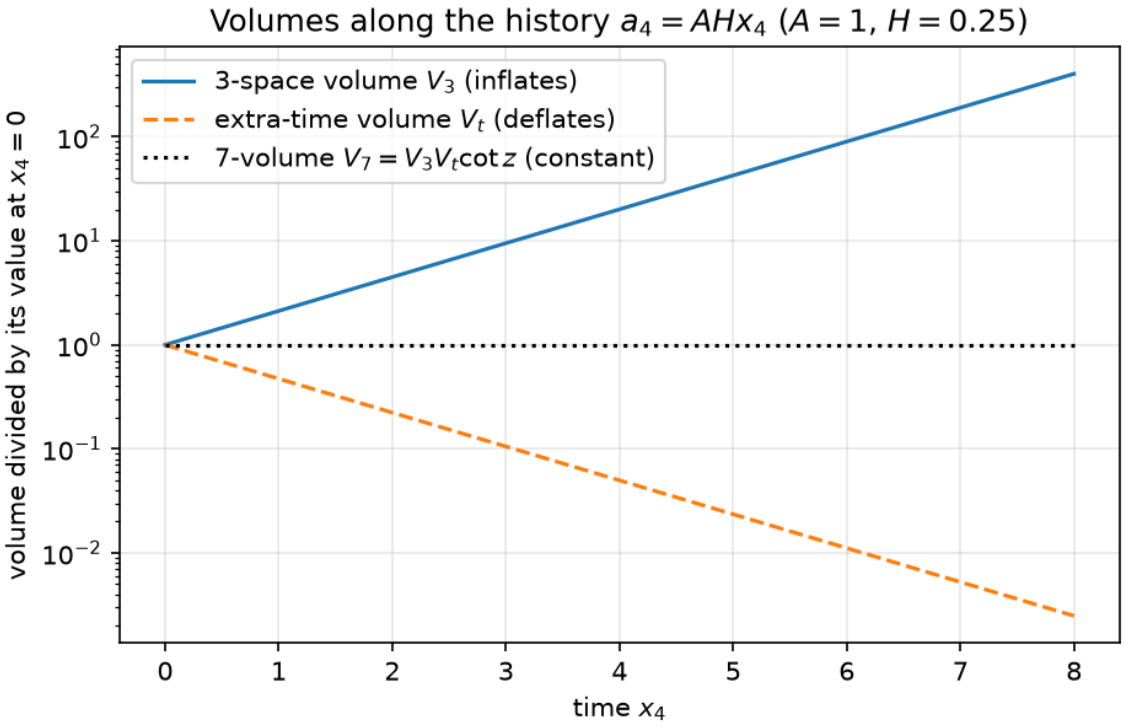

Figure 253: The 3-space volume $V_3 = e^{3a_4} \sin^{1/2} z$ (solid), the extra-time volume $V_t = e^{-3a_4} \sin^{1/2} z$ (dashed) and the 7-volume $V_7 = \cos z$ (dotted) of a coordinate box at fixed x_0, along the deflating history $a_4 = AHx_4$ with $A = 1$, $H = 0.25$, for x_4 from 0 to 8, each divided by its value at $x_4 = 0$; logarithmic vertical axis. The growth of V_3 and the shrinking of V_t cancel exactly. Read as a first law, the conservation identity $d\rho/dx_4 = -3a_4'(p_3 - p_t)$ says: while 3-space inflates a positive p_3 takes energy out of the box, while the extra times deflate a positive p_t puts energy in, and the two balance only when $p_3 = p_t$. (Notebook 09a, figure 4.)

```
Figure 09a.4 saved as Revision/textbook/figures/09a_4_volumes_first_law.png
```

Text cell:

```
## 15. The x0 identity: the balance along the hidden direction

Conservation along $x_0$ says
$\partial_0 p_0 = -3H\cot z (2p_0 - p_3 - p_t)$, a balance of pressures like
the balance of pressure and weight in a column of air. If the entries do not
depend on $x_0$, it forces $p_0 = (p_3 + p_t)/2$. If $p_3 + p_t = P$ is a
constant, every solution is $p_0 = P/2 + c/\sin z$ with a constant $c$; only
$c = 0$ is independent of $x_0$. The next cell checks this with sympy, then the
same identity in the hidden coordinate $y = \ln(\sin z)/(6H)$ used by the
Kohn-Sham solver of the book: $dy/dx_0 = \cot z$, and the $x_0$ component is
$\cot z [p_0'(y) + 6Hp_0 - 3H(p_3 + p_t)]$ (the lead's checks).
```

In [19]:

```
P, c = sp.symbols("P c", real=True)  # P = p3 + p_t (a constant) and the constant c
profile = P / 2 + c / sp.sin(zs)  # p0 as a function of x0 (through z)
balance = sp.diff(profile, x[0]) + 3 * Hs * sp.cot(zs) * (2 * profile - P)
check(sp.simplify(balance) == 0,
      "p0 = P/2 + c/sin z solves the x0 identity for every c")
y = sp.symbols("y", real=True)
dy_dx0 = sp.diff(sp.log(sp.sin(zs)) / (6 * Hs), x[0])  # y = ln(sin z)/(6 H)
check(sp.simplify(dy_dx0 - sp.cot(zs)) == 0
      and recorded(LEAD, "ks_coordinate_jacobian") == "PASS",
      "y = ln(sin z)/(6 H) has dy/dx0 = cot z",
      record=f"{LEAD}, check ks_coordinate_jacobian")
z_of_y = sp.asin(sp.exp(6 * Hs * y))  # sin z = e^(6 H y), z between 0 and pi/2
P0, P3, PT = [sp.Function(n)(y) for n in ("P0", "P3", "PT")]
in_x0 = sp.cot(z_of_y) * sp.diff(P0, y) + 3 * Hs * sp.cot(z_of_y) * (2 * P0 - P3 - PT)
in_y = sp.cot(z_of_y) * (sp.diff(P0, y) + 6 * Hs * P0 - 3 * Hs * (P3 + PT))
check(sp.simplify(in_x0 - in_y) == 0
      and recorded(LEAD, "ks_coordinate_form") == "PASS",
      "x0 component = cot z [p0'(y) + 6 H p0 - 3 H (p3 + p_t)]",
      record=f"{LEAD}, check ks_coordinate_form")
```

Out [19]:

```
PASS p0 = P/2 + c/sin z solves the x0 identity for every c
PASS y = ln(sin z)/(6 H) has dy/dx0 = cot z
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check ks_coordinate_jacobian
PASS x0 component = cot z [p0'(y) + 6 H p0 - 3 H (p3 + p_t)]
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check ks_coordinate_form
```

Text cell:

The next cell draws the profiles $p_0 = P/2 + c/\sin z$ for $P = 1$ and five values of c, and checks numerically on the drawn grid (with numpy's ‘np.gradient‘, a finite-difference derivative) that each profile satisfies the balance: the left side minus the right side is small compared with the terms.

In [20]:

```
z_grid = np.linspace(0.1, np.pi / 2 - 0.01, 2001)  # z from 0.1 to just below pi/2
x0_grid = z_grid / (6 * H)  # the matching x0 = z/(6 H)
fig, ax = plt.subplots()
worst = 0.0  # the largest relative residual of the balance on the grid
for c_value in (-0.2, -0.1, 0.0, 0.1, 0.2):
    p0_values = 0.5 + c_value / np.sin(z_grid)  # P = 1
    slope = np.gradient(p0_values, x0_grid)  # d p0/d x0, by finite differences
    rhs = -3 * H / np.tan(z_grid) * (2 * p0_values - 1.0)
    # The size of the terms; 1 (the size of P) is added, so that for the flat
    # profile c = 0, whose terms are zero, rounding noise is not divided by zero.
    scale = 1.0 + np.abs(slope).max() + np.abs(rhs).max()
    # [5:-5] leaves out five points at each end, where np.gradient is less exact.
    worst = max(worst, np.abs(slope - rhs)[5:-5].max() / scale)
    style = "-" if c_value == 0.0 else "--"
    ax.plot(z_grid, p0_values, style, label=f"$c = {c_value:g}$")  # g: short form
check(worst < 1e-4, "the drawn profiles satisfy the x0 balance on the grid")
ax.set_ylim(-2.0, 3.0)
ax.set_xlabel("$z = 6 H x_0$")
ax.set_ylabel("$p_0$ (with $p_3 + p_t = 1$)")
ax.set_title("Pressure profiles allowed by the balance along $x_0$")
ax.legend()
save_figure(fig, "hidden_balance",
            "The hidden-direction pressure profiles $p_0 = P/2 + c/\\sin z$ that "
            "satisfy the conservation identity along $x_0$, "
            "$\\partial_0 p_0 = -3H\\cot z\\,(2p_0 - p_3 - p_t)$, when "
            "$p_3 + p_t = P = 1$ is constant, for $c = -0.2, -0.1, 0, 0.1, 0.2$; "
            "horizontal axis $z = 6Hx_0$ from $0.1$ to just below $\\pi/2$, "
            "vertical axis $p_0$ in units of energy per unit volume. Only $c = 0$ "
            "(solid line) is independent of $x_0$, and it has $p_0 = P/2 = "
            "(p_3 + p_t)/2$; every other profile grows in size like $1/\\sin z$ "
            "towards the tip $z = 0$ (upwards for $c > 0$, downwards for $c < 0$).")
report("largest relative residual of the balance on the grid", f"{worst:.1e}")
```

Out [20]:

```
PASS the drawn profiles satisfy the x0 balance on the grid
```

Out [20]:

```
Figure 09a.5 saved as Revision/textbook/figures/09a_5_hidden_balance.png
RESULT largest relative residual of the balance on the grid = 2.3e-05
```

Text cell:

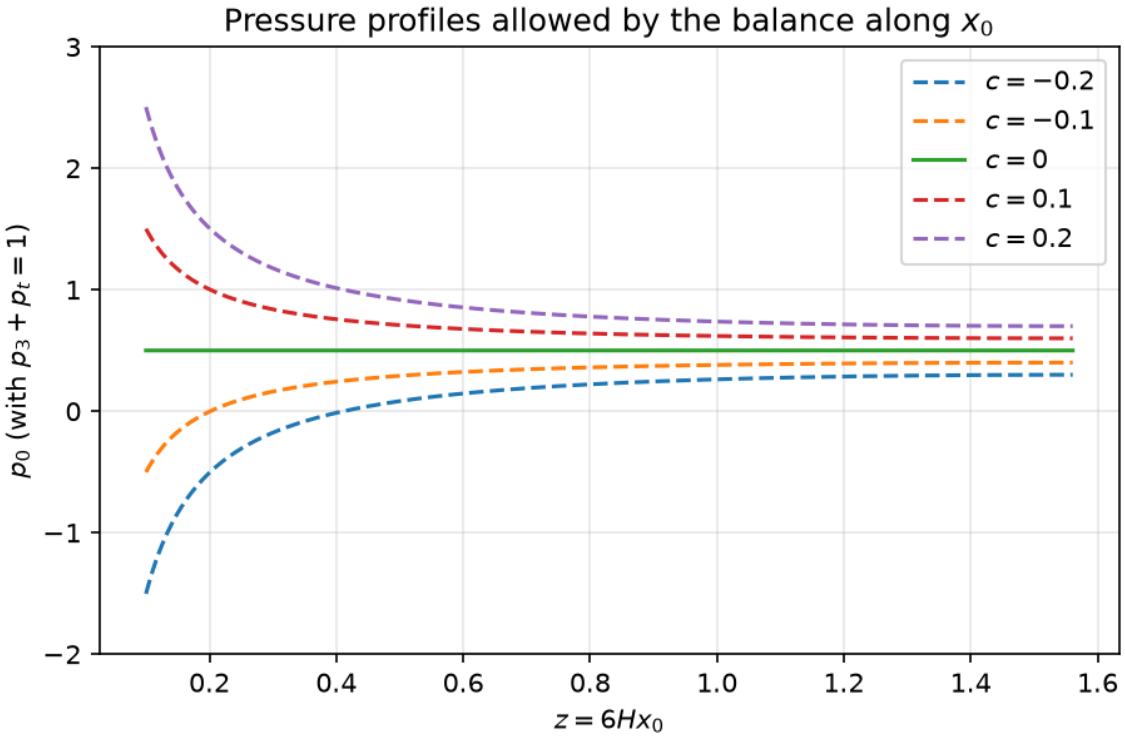

Figure 254: The hidden-direction pressure profiles $p_0 = P/2 + c/\sin z$ that satisfy the conservation identity along x_0, $\partial_0 p_0 = -3H\cot z\,(2p_0 - p_3 - p_t)$, when $p_3 + p_t = P = 1$ is constant, for $c = -0.2, -0.1, 0, 0.1, 0.2$; horizontal axis $z = 6Hx_0$ from 0.1 to just below $\pi/2$, vertical axis p_0 in units of energy per unit volume. Only $c = 0$ (solid line) is independent of x_0, and it has $p_0 = P/2 = (p_3 + p_t)/2$; every other profile grows in size like $1/\sin z$ towards the tip $z = 0$ (upwards for $c > 0$, downwards for $c < 0$). (Notebook 09a, figure 5.)

```
## 16. The last check

The last cell checks that the five figures are saved with their captions and
prints the number of checks that passed.
```

In [21]:

```
captions = json.loads(output_file(CAPTION_FILE).read_text(encoding="utf-8"))
expected_files = ["09a_1_kinetic_weights.png", "09a_2_tensor_heat_map.png",
                  "09a_3_diagonal_parts.png", "09a_4_volumes_first_law.png",
                  "09a_5_hidden_balance.png"]
check(sorted(captions) == expected_files and all(
    output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected_files),
    "the five figures of this notebook are saved and captioned")
all_checks_passed()
```

Out [21]:

```
PASS the five figures of this notebook are saved and captioned
ALL 27 CHECKS PASSED (notebook 09a)
```

Text cell:

```
## 17. What this notebook showed

- At a point of the deflating history the energy-momentum tensor of
  dirac16complex00 is a real $8 \times 8$ table whose lowered form is symmetric;
  its diagonal entries are $T^\mu{}_\mu = L_0 - K_\mu$, so the energy density is
  $\rho = -\sum_{\mu \neq x_4}K_\mu + mS + U$ and each pressure is the sum of the
  other seven kinetic terms minus $mS + U$ (exact statements of the record,
  confirmed here numerically).
- The trace is $7\sum_\mu K_\mu - 8(mS + U)$ for every configuration; on shell
  $\sum_\mu K_\mu = (m + U')S$, $L_0 = SU' - U$ and the trace is
  $-mS + 3\lambda S^2$.
- The spin connection drops out of the diagonal entries, because
  $\{\gamma^\mu, \Omega_\mu\} = 0$ for each direction, but it enters off-diagonal
  entries such as $T^{x_4}{}_{x_1}$; summed, $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$.
- The author's metric has 25 nonzero Christoffel symbols, the record's list
  exactly, and conservation of a diagonal tensor is exactly two conditions:
  $\partial_4\rho = -3a_4'(p_3 - p_t)$ (energy exchange between 3-space and the
  extra times, a first law for the constant 7-volume) and
  $\partial_0 p_0 = -3H\cot z (2p_0 - p_3 - p_t)$ (a balance along the hidden
  direction, which forces $p_0 = (p_3 + p_t)/2$ for $x_0$-independent entries).
- Status: the identities are PROVED in the record (exact, Wolfram and sympy);
  this notebook re-derives the two conservation identities exactly and confirms
  the tensor identities numerically at one point. The parameter values are
  choices of this notebook, not results.
```

9.17 Line-by-line walk-through of Notebook 09a

The notebook has 21 code cells, In [1] to In [21]. This section explains every line of every one of them, in order. Code that is printed again here is quoted exactly, except that a long figure caption is shortened to a line "...)" (it is printed in full in Section 9.16, under its figure) and that the texts in triple quotes that document a function (its docstring, explained under In [1]) and the comment lines inside the function `save_figure` are left out.

In [1], the set-up cell. Every line that starts with `#` is a **comment**, which Python skips. The first part of the cell, down to the lines of `-` and `=` signs, is the complete run instructions of Section 9.15 again, as comments, so that the notebook file carries its own instructions. The code starts after the heading THE SET-UP; it computes no physics and is the same in every notebook of the book, except for the name of the notebook.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "09a"  # this notebook: chapter 09, example a
```

`import` loads a **module** (a part of Python or of an installed package) so that the cell can use it; `from pathlib import Path` takes only the name `Path`, the address of a file or folder, written the same way on every operating system. `plt` is the usual short name of matplotlib's drawing functions, and `Image` and `display` show a saved picture below a cell. The last line gives the **variable** `NOTEBOOK_ID` the **string** (a text in quotes) `"09a"`; the figure files are named after it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called (the notebook's version also has a **docstring**, the text in triple quotes under the `def` line, which only documents it; it is left out here). Jupyter runs a notebook in the folder that holds it, `Path.cwd()`, and `.resolve()` writes it as a complete address. The `for` loop walks up from that folder through its parents; the operator `/` joins folder and file names, and `.is_file()` asks whether the file exists. The first folder that contains `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If none qualifies, `raise` stops the notebook with an error message that says what to do.

```
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
```

```
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`REPO` is the repository folder; it is never printed, because it differs from computer to computer. `OUTPUT_ROOT` is where files are written: the repository, unless the **environment variable** `TEXTBOOK_OUTPUT_ROOT` (a named text that the computer hands to a program) names another folder; the book's checking tool sets it, so that a check never changes the repository. `repository_file` gives the path of a repository file for reading; `output_file` gives the path at which to write a file and first creates its folder (`mkdir`, where `exist_ok=True` does nothing if the folder exists). `say` prints a text in lines of at most 89 characters (the width of a page of the book), indenting the continuation lines by four blanks. (The comment lines above `REPO` and `OUTPUT_ROOT` and the docstrings of the three functions are left out here; they say the same in words.)

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

The first two lines make every figure look the same on every computer: the built-in settings, a figure size of 7.0 by 4.2 inches, 10-point letters and a faint grid. The braces `{...}` make a **dictionary** (pairs `key: value`). A string that starts with `f` is an **f-string**: each name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/09a.captions.json"`. The last line starts that file as an empty dictionary `{}`; `newline="\n"` writes the same line end on every operating system.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
```

```
    relative = f"{FIGURE_FOLDER}/{file_name}"
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` (shown without its docstring and its comment lines) numbers the figures 1, 2, 3, ... in the order of their first appearance (`setdefault` returns the number stored for a name, or stores and returns the next one), saves the picture as a PNG file at 150 dots per inch with the empty margin cut away and without a program name inside the file (so that two runs write the same bytes), closes it, stores the caption in the captions file (`json.dumps` turns the dictionary into text with the keys sorted), shows the saved picture below the cell and prints one line saying where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`PASSED` is an empty **list** (an ordered collection, in square brackets). `check` is the function behind every check: `condition` is either `True` or `False`; if it is false, the notebook stops with an `AssertionError` that names the check; if it is true, the name is appended to `PASSED` and the line `PASS name` is printed, followed, when the check reproduces a Revision record, by a line `reproduces <file>, check <name>`. (An `if` statement is used instead of Python's `assert`, which Python started with the option `-O` would skip.) `report` prints a key number as a line that starts with `RESULT`. `all_checks_passed` prints the last line of the notebook with the number of checks that passed. The cell prints one line: `Set-up of notebook 09a complete:  repository folder found, helpers defined.`

In [2], the gamma matrices and the matrix C.

```
import numpy as np  # arrays of numbers, matrices, linear algebra

fixture = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
# gamma[a] is the frame matrix gamma^(xa): gamma[0] = x0, ..., gamma[7] = x7.
gamma = [np.array(rows, dtype=float) for rows in fixture["gamma"]]
eta = np.array(fixture["eta"], dtype=float)  # (+1, +1, +1, +1, -1, -1, -1, -1)
C = np.array(fixture["C"], dtype=float)  # gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
I16 = np.eye(16)  # the 16 x 16 unit matrix
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # the author's coordinates
```

numpy is the package for arrays of numbers. `read_text` reads the fixture file as text, and `json.loads` turns the text into Python objects: here a dictionary whose entry `"gamma"` is a list of eight matrices, each a list of 16 rows of 16 numbers. The line `gamma = [... for rows in fixture["gamma"]]` is a **list comprehension**: it builds a list by evaluating the expression before `for` for each item; `np.array(rows, dtype=float)` turns one matrix into a numpy array of real (floating-point) numbers. Python counts from 0, like the author's coordinates, so `gamma[k]` is $\gamma^{(k)}$: `gamma[0]` is $\gamma^{(0)}$, `gamma[4]` is $\gamma^{(4)}$ and `gamma[7]` is $\gamma^{(7)}$. The eight matrices are real; their entries are the whole numbers 0, 1 and -1. `eta` holds the eight diagonal entries of η, `C` the matrix C, `I16` the 16×16 unit matrix, and `NAMES` the names of the coordinates for printing.

```
def recorded(path, name):
    """The verdict (PASS or FAIL) of the check name in the Revision report path."""
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for item in report_data["checks"]:
        if item["name"] == name:
            return item["verdict"].upper()  # some reports write "pass"
    raise KeyError(f"{path} has no check {name}")
```

A Revision report is a JSON file with a list `"checks"`; each check has a `"name"` and a `"verdict"`. `recorded` opens the report, looks for the check of the given name and returns its verdict in capital letters (`.upper()`, because some reports write `pass`). If the name is missing it stops with a `KeyError`. Every check of the notebook that reproduces a record also asks the record for its own verdict, so a notebook check passes only when the record agrees.

```
ALGEBRA = "Revision/algebra/reports/python-algebra.json"
# {gamma^a, gamma^b} = gamma^a gamma^b + gamma^b gamma^a must be 2 eta^ab times 1.
clifford = all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                              2 * eta[a] * (a == b) * I16)
               for a in range(8) for b in range(8))
check(clifford and recorded(ALGEBRA, "clifford_relation") == "PASS",
      "the 64 Clifford relations {gamma^a, gamma^b} = 2 eta^ab",
      record=f"{ALGEBRA}, check clifford_relation")
```

The operator `@` is the matrix product. `(a == b)` is `True` (counted as 1) when $a = b$ and `False` (counted as 0) otherwise, so `2 * eta[a] * (a == b) * I16` is $2\eta^{ab} \cdot 1$ (for this diagonal η, $\eta^{aa} = \eta_{aa}$). `np.array_equal` asks whether two arrays are equal entry by entry, exactly: the gammas hold whole

numbers, so no rounding occurs. `all(... for a in range(8) for b in range(8))` is true only when the comparison holds for all 64 pairs; `range(8)` is 0, 1, ..., 7. The check passes when all 64 Clifford relations hold and the record's check `clifford_relation` passed.

```
# C is symmetric, C times C is 1, and every C gamma^a is antisymmetric.
c_ok = np.array_equal(C, C.T) and np.array_equal(C @ C, I16) and all(
    np.array_equal((C @ g).T, -(C @ g)) for g in gamma)
check(c_ok and recorded(ALGEBRA, "C_real_symmetric_involution") == "PASS"
      and recorded(ALGEBRA, "C_gamma_antisymmetric") == "PASS",
      "C is real symmetric, C^2 = 1, every C gamma^a is antisymmetric",
      record=f"{ALGEBRA}, checks C_real_symmetric_involution and "
             "C_gamma_antisymmetric")
```

`.T` is the transpose. The three conditions are $C^T = C$, $C^2 = 1$ and $(C\gamma^{(a)})^T = -C\gamma^{(a)}$ for each of the eight gammas (`for g in gamma` takes them in turn): the facts used in Sections 9.6 and 9.18. The cell prints the two PASS lines with their record lines.

In [3], the author's metric at one point.

```
H = 0.25  # the author's constant H
A = 1.0  # the history a4 = A H x4; A > 0: the extra times deflate
m = 1.0  # the mass m of the field
lam = 0.5  # the strength lambda of the potential U(S) = (lambda/2) S^2
X4 = 2.0  # the time x4 of the point
a4 = A * H * X4  # a4 at the point: 0.5
a4p = A * H  # its derivative da4/dx4 (written a4p, "a4 prime"): 0.25
z = np.pi / 4  # z = 6 H x0 at the point
```

These lines choose the numbers of the example: $H = 0.25$, the deflating history $a_4 = AHx_4$ with $A = 1$, the mass $m = 1$, the strength $\lambda = 0.5$ (written `lam`, because `lambda` is a reserved word of Python), the time $x_4 = 2$, hence $a_4 = 0.5$ and $a_4' = AH = 0.25$, and $z = \pi/4$ (`np.pi` is π). They are choices, not results.

```
def frame_factors(a4_value, z_value):
    """The eight factors f_a of the diagonal vielbein, in the order x0, ..., x7."""
    space = np.exp(a4_value) * np.sin(z_value) ** (1 / 6)  # e^a4 sin^(1/6) z
    extra = np.exp(-a4_value) * np.sin(z_value) ** (1 / 6)  # e^(-a4) sin^(1/6) z
    return np.array([1 / np.tan(z_value)] + [space] * 3 + [1.0] + [extra] * 3)


f = frame_factors(a4, z)
g_diag = eta * f ** 2  # the diagonal entries g_mumu = eta_mumu f_mu^2
```

`frame_factors` returns the eight factors f_a of Section 9.2. `**` means "to the power", so `np.sin(z_value) ** (1 / 6)` is $\sin^{1/6} z$. `[space] * 3` is a list of three copies of `space`, and `+` joins lists, so the returned array is $(f_1, f_1, f_1, 1, f_5, f_5, f_5, \cot z)$ with $\cot z = 1/\tan z$. Then `f` holds the factors at the point, and `g_diag` the eight diagonal entries $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$; arithmetic on arrays acts entry by entry.

```
# The author's metric written out: e^(2 a4) sin^(1/3) z, -1, -e^(-2 a4) sin^(1/3) z,
# cot^2 z.
authors = np.array([1 / np.tan(z) ** 2] + [np.exp(2 * a4) * np.sin(z) ** (1 / 3)] * 3
                   + [-1.0] + [-np.exp(-2 * a4) * np.sin(z) ** (1 / 3)] * 3)
THEORY_PY = "Revision/theory/reports/python-field-theory.json"
check(np.allclose(g_diag, authors, rtol=1e-14, atol=0)
      and recorded(THEORY_PY, "metric_from_vielbein_equals_SPEC") == "PASS",
      "eta_aa f_a^2 is the author's metric at the point",
      record=f"{THEORY_PY}, check metric_from_vielbein_equals_SPEC")
check(abs(np.prod(f) - np.cos(z)) < 1e-14
      and recorded(THEORY_PY, "sqrt_det_g_equals_cos_z") == "PASS",
      "the volume factor f0 f1 ... f7 equals cos z",
      record=f"{THEORY_PY}, check sqrt_det_g_equals_cos_z")
for name, value in zip(NAMES, f):
    report(f"f_{name}", f"{value:.6f}")
```

The two comment lines say what the next line builds: `authors` writes the author's eight metric entries directly, copied from the author's matrix and not from the factors f_a, so that the comparison is a real test. `np.allclose(x, y, rtol=1e-14, atol=0)` is true when every entry of `x` differs from the entry of `y` by less than 10^{-14} times its size (a relative tolerance; floating-point numbers carry about 16 significant digits). The second check compares the product of the eight factors (`np.prod`) with $\cos z$, the computation of Section 9.2. `zip(NAMES, f)` pairs each name with its factor, and `report` prints each with six decimals (the format `:.6f`): $f_{x_1} = e^{0.5}\sin^{1/6}(\pi/4) = 1.6487213 \cdot 0.9438743 = 1.556186$, $f_{x_4} = 1$, $f_{x_5} = e^{-0.5}\sin^{1/6}(\pi/4) = 0.6065307 \cdot 0.9438743 = 0.572489$ and $f_{x_0} = \cot(\pi/4) = 1$ (each factor written with seven decimals and each product rounded to six, the precision that the notebook prints; with factors rounded to four decimals the sixth decimal of a product would come out wrong).

In [4], the weights of the kinetic terms (Figure 09a.1).

```
from matplotlib.ticker import NullFormatter  # a tick label that prints nothing

a4_axis = np.linspace(-2.0, 2.0, 401)  # values of a4 for the left panel
z_axis = np.linspace(0.02, 1.50, 400)  # values of z for the right panel
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
weights_a4 = np.array([1 / frame_factors(value, z) for value in a4_axis])
weights_z = np.array([1 / frame_factors(a4, value) for value in z_axis])
```

`np.linspace(a, b, n)` makes `n` equally spaced numbers from `a` to `b`. `plt.subplots(1, 2, ...)` makes a figure with one row of two panels, named `left` and `right`, 9.6 by 3.9 inches. `weights_a4` is a table with one row per value of a_4 and eight columns, the weights $1/f_\mu$ at $z = \pi/4$; `weights_z` is the same against z at $a_4 = 0.5$.

```
for panel, axis, weights in ((left, a4_axis, weights_a4), (right, z_axis, weights_z)):
    panel.plot(axis, weights[:, 1], label="3-space $x_1, x_2, x_3$")
    panel.plot(axis, weights[:, 4], "--", label="time $x_4$")
    panel.plot(axis, weights[:, 5], label="extra times $x_5, x_6, x_7$")
    panel.plot(axis, weights[:, 0], ":", label="hidden direction $x_0$")
    panel.set_yscale("log")  # equal factors are equal distances on this axis
    panel.yaxis.set_minor_formatter(NullFormatter())  # no labels on minor ticks
    panel.set_ylabel("weight $1/f_\\mu$")
```

The loop runs twice, once per panel. `weights[:, 1]` is column 1 of the table (all rows), the weight of x_1; columns 4, 5 and 0 are x_4, x_5 and x_0. The style string of two hyphens draws a dashed line and `":"` a dotted one; `label` is the name in the legend; text between dollar signs is typeset as mathematics, and a backslash inside a Python string is written twice. On a **logarithmic axis** equal ratios are equal distances, so a function $e^{\pm a_4}$ appears as a straight line. The minor ticks get no labels.

```
ticks = [0.03, 0.1, 0.2, 0.5, 1, 2, 5, 10]  # plain numbers on the vertical axes
left.set_yticks(ticks[2:7], [str(t) for t in ticks[2:7]])
right.set_yticks(ticks, [str(t) for t in ticks])
left.set_xlabel("$a_4$ (at $z = \\pi/4$)")
right.set_xlabel("$z = 6 H x_0$ (at $a_4 = 0.5$)")
left.set_title("Weights against $a_4$")
right.set_title("Weights against $z$")
left.legend(fontsize=8)
save_figure(fig, "kinetic_weights",
            "The weights $1/f_\\mu$ with which a derivative along each direction "
            ...)
```

These lines put plain numbers on the vertical axes (`ticks[2:7]` is the part of the list from position 2 up to, not including, position 7), label the axes, set the titles, draw the legend in the left panel and save the figure as `09a_1_kinetic_weights.png` with its caption. **What Figure 09a.1 shows.** Both panels plot the weight $1/f_\mu$ (a pure number) on a logarithmic vertical axis. Left, against a_4: the 3-space weight $e^{-a_4}\sin^{-1/6} z$ falls and the extra-time weight $e^{a_4}\sin^{-1/6} z$ rises, two straight lines of opposite slope; the time weight 1 and the hidden weight $\tan(\pi/4) = 1$ lie on top of each other. The student should see that as the extra times deflate, derivatives along them count more and more in ρ and in the pressures. Right, against z: the hidden weight $\tan z$ grows without bound near $z = \pi/2$.

In [5], the spin connection at the point.

```
def spin_connection(a4_value, a4p_value, z_value):
    """The eight 16 x 16 matrices Omega_mu of the record's formula Omega_components."""
    s = np.sin(z_value) ** (1 / 6)  # sin^(1/6) z
    omega = [np.zeros((16, 16)) for _ in range(8)]  # Omega_x4 = Omega_x0 = 0
    for i in range(1, 4):  # the three 3-space directions x1, x2, x3
        omega[i] = 0.5 * np.exp(a4_value) * s * (
            a4p_value * gamma[i] @ gamma[4] + H * gamma[i] @ gamma[0])
    for t in range(5, 8):  # the three extra times x5, x6, x7
        omega[t] = -0.5 * np.exp(-a4_value) * s * (
            a4p_value * gamma[4] @ gamma[t] + H * gamma[t] @ gamma[0])
    return omega
```

The function writes the formula of Section 9.2 literally. It starts with eight zero matrices (`np.zeros ((16, 16))`; the name `_` is used for a loop counter that is not needed), which stay zero for x_4 and x_0, then fills the 3-space entries $\Omega_{x_i} = \frac{1}{2}e^{a_4}\sin^{1/6} z\,(a_4'\gamma^{(i)}\gamma^{(4)} + H\gamma^{(i)}\gamma^{(0)})$ at the indices 1, 2, 3 (`range(1, 4)`) and the extra-time entries $\Omega_{x_t} = -\frac{1}{2}e^{-a_4}\sin^{1/6} z\,(a_4'\gamma^{(4)}\gamma^{(t)} + H\gamma^{(t)}\gamma^{(0)})$ at the indices 5, 6, 7 (`range(5, 8)` is 5, 6, 7): the index is the direction.

```
Omega = spin_connection(a4, a4p, z)
gamma_up = [gamma[mu] / f[mu] for mu in range(8)]  # coordinate gammas gamma^mu
```

```
gamma_down = [g_diag[mu] * gamma_up[mu] for mu in range(8)]  # gamma_mu
# {gamma^mu, Omega_mu} for each mu separately: the largest entry of all eight.
anti = max(np.abs(gamma_up[mu] @ Omega[mu] + Omega[mu] @ gamma_up[mu]).max()
           for mu in range(8))
A4_REPORT = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
check(anti < 1e-14
      and recorded(A4_REPORT, "gamma_mu_anticommutes_with_Omega_mu_no_sum") == "PASS",
      "{gamma^mu, Omega_mu} = 0 for each direction mu (no sum)",
      record=f"{A4_REPORT}, check gamma_mu_anticommutes_with_Omega_mu_no_sum")
```

`Omega` holds the eight matrices at the point; `gamma_up` the coordinate gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ and `gamma_down` the lowered ones $\gamma_\mu = g_{\mu\mu}\gamma^\mu$. For each direction the cell computes the anticommutator $\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu$ and takes the largest absolute value of its 256 entries (`np.abs(...).max()`); `anti` is the largest of these eight numbers. Fact 1 of Section 9.3 says it is zero; the check allows rounding errors below 10^{-14}.

```
total = sum(gamma_up[mu] @ Omega[mu] for mu in range(8))  # gamma^mu Omega_mu, summed
LEAD = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
check(np.abs(total - 3 * H * gamma[0]).max() < 1e-14
      and recorded(THEORY_PY, "gamma_mu_Omega_mu_equals_3H_gamma_x0") == "PASS"
      and recorded(LEAD, "gamma_Omega_equals_3H_gamma0") == "PASS",
      "gamma^mu Omega_mu = 3 H gamma^(x0) at the point",
      record=f"{THEORY_PY}, check gamma_mu_Omega_mu_equals_3H_gamma_x0; {LEAD}, "
             "check gamma_Omega_equals_3H_gamma0")
```

`sum(...)` adds the eight products $\gamma^\mu\Omega_\mu$, and the check compares the result with $3H\gamma^{(0)} = 0.75\,\gamma^{(0)}$: Fact 2, which the theory record and the lead's independent check both state.

In [6], the Dirac adjoint and the energy-momentum tensor.

```
def bar(vector):
    """The Dirac adjoint: the row vector vector^dagger C."""
    return vector.conj() @ C
```

`.conj()` takes the complex conjugate of every entry. For a one-dimensional numpy array, `@` with a matrix on its right treats the array as a row, so `vector.conj() @ C` is the row $\Phi^\dagger C = \bar{\Phi}$.

```
def energy_momentum(phi, dphi, f_values, gup, gdown, omega):
    """S, U, L0, the kinetic terms K_mu and the tensor T (T[nu, mu] = T^nu_mu)."""
    g_values = eta * f_values ** 2  # g_mumu
    S = (bar(phi) @ phi).real  # S = Phibar Phi (a real number)
    U = lam / 2 * S ** 2  # U(S) = (lambda/2) S^2
    D = [dphi[mu] + omega[mu] @ phi for mu in range(8)]  # D_mu Phi
    Dbar = [bar(dphi[mu]) - bar(phi) @ omega[mu] for mu in range(8)]  # D_mu Phibar
    L0 = 0.5 * sum(bar(phi) @ gup[mu] @ D[mu] - Dbar[mu] @ gup[mu] @ phi
                   for mu in range(8)) - m * S - U
```

The arguments are the 16 values `phi` of Φ at the point, the 8×16 table `dphi` whose row `mu` holds $\partial_\mu\Phi$, the factors, the coordinate gammas with upper and lower index, and the spin connection. `S`

is $\bar\Phi\Phi$; it is real (Section 9.6), and `.real` keeps the real part, dropping a rounding remainder. `D` and `Dbar` are the lists of the eight covariant derivatives $D_\mu\Phi = \partial_\mu\Phi + \Omega_\mu\Phi$ and $D_\mu\bar\Phi = \partial_\mu\bar\Phi - \bar\Phi\Omega_\mu$ (`bar(dphi[mu])` is $\partial_\mu\bar\Phi$, because C is the same at every point). `L0` is the formula of Section 9.2, with the spin connection included: the cell computes it the long way, and a later check compares it with $\sum_\mu K_\mu - mS - U$.

```
T = np.zeros((8, 8), dtype=complex)
for nu in range(8):
    for mu in range(8):
        bracket = (bar(phi) @ gup[nu] @ D[mu] - Dbar[mu] @ gup[nu] @ phi
                   # gamma_mu D^nu = gamma_mu D_nu / g_nunu
                   + (bar(phi) @ gdown[mu] @ D[nu]
                      - Dbar[nu] @ gdown[mu] @ phi) / g_values[nu])
        T[nu, mu] = (nu == mu) * L0 - bracket / 4
```

The two loops fill the 8×8 table `T`, row `nu` and column `mu`, with the formula of Section 9.6: `bracket` is $\bar\Phi\gamma^\nu D_\mu\Phi - D_\mu\bar\Phi\gamma^\nu\Phi + \bar\Phi\gamma_\mu D^\nu\Phi - D^\nu\bar\Phi\gamma_\mu\Phi$, where $D^\nu = D_\nu / g_{\nu\nu}$ is the division by `g_values[nu]`, and `(nu == mu) * L0` is $\delta^\nu_\mu L_0$. The table is complex at first (`dtype=complex`), because the field is complex; it must come out real.

```
# K_mu = (1/(2 f_mu)) (Phibar gamma^(mu) d_mu Phi - d_mu Phibar gamma^(mu) Phi)
K = np.array([(bar(phi) @ gamma[mu] @ dphi[mu] - bar(dphi[mu]) @ gamma[mu] @ phi)
              / (2 * f_values[mu]) for mu in range(8)])
return S, U, L0, K, T
```

`K` holds the eight kinetic terms computed from their own formula, with ordinary derivatives and frame gammas and without any spin connection, so that the record's statements $T^\mu{}_\mu = L_0 - K_\mu$ and $L_0 = \sum_\mu K_\mu - mS - U$ can be tested independently. The function returns five results; the cell prints nothing.

In [7], a random configuration and its tensor.

```
rng = np.random.default_rng(12345)  # a fixed seed: the same numbers in every run
Phi = rng.normal(size=16) + 1j * rng.normal(size=16)  # the 16 values of Phi
dPhi = rng.normal(size=(8, 16)) + 1j * rng.normal(size=(8, 16))  # d_mu Phi
# np.roll moves the last row to the front: the eight rows are the derivatives along
# x0, x1, ..., x7, and the numbers are those of the earlier edition of this notebook.
dPhi = np.roll(dPhi, 1, axis=0)
S, U, L0, K, T = energy_momentum(Phi, dPhi, f, gamma_up, gamma_down, Omega)
```

`np.random.default_rng(12345)` makes a generator of random numbers; the fixed **seed** 12345 makes it produce the same numbers in every run, so the notebook's output never changes. `rng.normal(size=16)` draws 16 numbers from the bell-shaped normal distribution; `1j` is Python's imaginary unit i, so `Phi` is a random complex column, and `dPhi` a random 8×16 table of first derivatives, row k for the direction x_k. `np.roll(dPhi, 1, axis=0)` moves the last row of the drawn table to the front; it changes no number, only which direction each row belongs to, so that the configuration is the same as in the first edition of this notebook (which listed the directions in another order) and every printed number below is unchanged. The **jet** at the point, the values of the field and of its eight first derivatives, is all the tensor needs; it need not satisfy the field equation. The last line computes everything and names the five results.

```
largest_imaginary = max(np.abs(T.imag).max(), np.abs(K.imag).max(), abs(L0.imag))
check(largest_imaginary < 1e-12, "every entry of T, every K_mu and L0 are real")
T, K, L0 = T.real, K.real, L0.real  # keep the real parts
T_low = g_diag[:, None] * T  # T_numu = g_nunu T^nu_mu
check(np.abs(T_low - T_low.T).max() < 1e-12
      and recorded(THEORY_PY, "commuting_emt_symmetric") == "PASS",
      "T_numu is symmetric (64 entries, 28 pairs compared)",
      record=f"{THEORY_PY}, check commuting_emt_symmetric")
report("S = Phibar Phi", f"{S:.6f}")
report("L0", f"{L0:.6f}")
```

`.imag` is the imaginary part. The first check confirms the reality of Section 9.6: the largest imaginary part is rounding noise, below 10^{-12}. Then only the real parts are kept. `g_diag[:, None] * T` multiplies row ν of the table by $g_{\nu\nu}$ (`[:, None]` turns the eight numbers into a column, which numpy repeats across the eight columns of `T`), giving $T_{\nu\mu}$; the second check compares it with its transpose, the symmetry of Section 9.6 (the 56 entries off the diagonal form 28 pairs). The random values do not solve the field equation, and the table is symmetric because the notebook computes the symmetric (Belinfante) tensor; the tensor of the vielbein variation is in general not symmetric for values that do not solve the field equation (Section 9.6). The cell prints $S = -6.527875$ (negative: S has no fixed sign) and $L_0 = 13.616346$.

In [8], the heat map of the tensor (Figure 09a.2).

```
fig, ax = plt.subplots(figsize=(6.6, 5.6))
largest = np.abs(T).max()  # the colour scale runs from -largest to +largest
picture = ax.imshow(T, cmap="RdBu_r", vmin=-largest, vmax=largest)
for nu in range(8):
    for mu in range(8):
        # white digits on dark squares, black digits on light ones
        colour = "white" if abs(T[nu, mu]) > 0.6 * largest else "black"
        ax.text(mu, nu, f"{T[nu, mu]:.1f}", ha="center", va="center", fontsize=7,
                color=colour)
```

`ax.imshow` draws the table as a **heat map**, a grid of coloured squares: row ν from top to bottom, column μ from left to right. The colour map `"RdBu_r"` runs from blue (negative) through white (zero) to red (positive), with a scale symmetric about zero. The loops write each value with one decimal in its square (`ha` and `va` centre the text), white on dark squares and black on light ones (`x if condition else y` chooses one of two values).

```
ax.set_xticks(range(8), [f"${n[0]}_{n[1]}$" for n in NAMES])
ax.set_yticks(range(8), [f"${n[0]}_{n[1]}$" for n in NAMES])
ax.set_xlabel("lower index $\\mu$ (column)")
ax.set_ylabel("upper index $\\nu$ (row)")
ax.set_title("$T^\\nu{}_\\mu$ of one random configuration at one point")
ax.grid(False)  # no grid lines across the squares
fig.colorbar(picture, ax=ax, label="value of $T^\\nu{}_\\mu$ (energy per volume)")
save_figure(fig, "tensor_heat_map",
            "Heat map of the energy-momentum tensor $T^\\nu{}_\\mu$ of a random "
            ...)
```

The tick labels turn a name such as `"x1"` into x_1 (`n[0]` is its letter, `n[1]` its digit). The colour bar beside the map says which value each colour means. **What Figure 09a.2 shows.** The 64 entries of the tensor of one random configuration, in units of energy per unit volume. The diagonal holds $-\rho$ at (x_4, x_4) and the seven pressures; every square off the diagonal is coloured, because a random configuration has flows of energy and momentum in every direction. The student should notice that the printed table is not symmetric: the upper index is raised with the metric, and only $g_{\nu\nu}T^{\nu}{}_{\mu}$ is symmetric.

In [9], the diagonal entries.

```
THEORY_WL = "Revision/theory/reports/wolfram-field-theory.json"
check(np.abs(np.diag(T) - (L0 - K)).max() < 1e-12
      and abs(L0 - (K.sum() - m * S - U)) < 1e-12
      and recorded(THEORY_WL, "T_diagonal_components_C") == "PASS",
      "T^mu_mu = L0 - K_mu for all eight mu, and L0 = sum K - m S - U",
      record=f"{THEORY_WL}, check T_diagonal_components_C")
```

`np.diag(T)` is the list of the eight diagonal entries, and `L0 - K` the list of the numbers $L_0 - K_\mu$. The check confirms Section 9.5 for all eight directions, and that the L_0 computed with the spin connection equals $\sum_\mu K_\mu - mS - U$ computed without it (Section 9.3).

```
potential = m * S + U  # the potential energy density m S + U(S)
# kinetic part of T^mu_mu: the sum of the K_nu with nu different from mu
kinetic_parts = np.array([K.sum() - K[mu] for mu in range(8)])
rho = -T[4, 4]  # the energy density
rho_kin = -(K.sum() - K[4])  # minus the sum of the K_mu with mu different from x4
check(abs(rho - (rho_kin + potential)) < 1e-12
      and np.abs(np.diag(T) - (kinetic_parts - potential)).max() < 1e-12,
      "rho = rho_kin + rho_pot and p_mu = (sum of the other K) - (m S + U)")
```

These lines build the split of Section 9.7: `potential` is $mS + U$; `kinetic_parts[mu]` is $\sum_{\nu\neq\mu} K_\nu$; `rho` is $-T^{x_4}{}_{x_4}$ (row and column 3 are x_4); `rho_kin` is $-\sum_{\mu\neq x_4} K_\mu$. The check confirms $\rho = \rho_{\text{kin}} + \rho_{\text{pot}}$ and, for every direction, that the diagonal entry is its kinetic part minus $mS + U$.

```
report("rho = -T^x4_x4", f"{rho:.6f}")
report("rho_kin", f"{rho_kin:.6f}")
report("rho_pot = m S + U", f"{potential:.6f}")
for mu in (0, 1, 2, 3, 5, 6, 7):
    report(f"pressure T^{NAMES[mu]}_{NAMES[mu]}", f"{T[mu, mu]:.6f}")
```

The cell prints $\rho = -7.531406$, made of $\rho_{\text{kin}} = -11.656818$ and $\rho_{\text{pot}} = 4.125412$ (indeed $-11.656818 + 4.125412 = -7.531406$), and the seven pressures, from $T^{x_7}{}_{x_7} = -1.976597$ to $T^{x_2}{}_{x_2} = 21.578866$. The energy density of this random configuration is negative, and the three 3-space pressures differ from each other (as do the three extra-time pressures): a random configuration is not isotropic.

In [10], the diagonal entries and their two parts (Figure 09a.3).

```
positions = np.arange(8)  # one group of bars per direction
fig, ax = plt.subplots(figsize=(8.0, 4.2))
```

```
ax.bar(positions - 0.27, kinetic_parts, width=0.27, label="kinetic part")
ax.bar(positions, [-potential] * 8, width=0.27, label="potential part $-(mS + U)$")
ax.bar(positions + 0.27, np.diag(T), width=0.27, label="total $T^\\mu{}_\\mu$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(positions, [f"${n[0]}_{n[1]}$" for n in NAMES])
ax.set_xlabel("direction $\\mu$")
ax.set_ylabel("value (energy per unit volume)")
ax.set_title("Diagonal entries $T^\\mu{}_\\mu$ and their two parts")
ax.legend(fontsize=8)
save_figure(fig, "diagonal_parts",
            "The eight diagonal entries $T^\\mu{}_\\mu$ (right bar of each group) "
            ...)
```

`np.arange(8)` is 0, 1, ..., 7. `ax.bar(x, heights, width=...)` draws bars; the three calls place three bars side by side for each direction, shifted by -0.27, 0 and $+0.27$. `ax.axhline(0.0, ...)` draws a horizontal line at zero. **What Figure 09a.3 shows.** For each of the eight directions: the kinetic part $\sum_{\nu\neq\mu} K_\nu$ (left bar), the potential part $-(mS+U)$ (middle bar, the same in every group) and their sum $T^\mu{}_\mu$ (right bar), in units of energy per unit volume. The group at x_4 shows $-\rho$. The student should see that only the kinetic parts make the directions differ.

In [11], the trace, the kinetic sum, and the configuration put on shell.

```
V = m + lam * S  # V = m + U'(S), with U'(S) = lambda S
E = sum(gamma_up[mu] @ (dPhi[mu] + Omega[mu] @ Phi) for mu in range(8)) - V * Phi
E_bar = sum((bar(dPhi[mu]) - bar(Phi) @ Omega[mu]) @ gamma_up[mu]
            for mu in range(8)) + V * bar(Phi)
```

These lines compute the effective mass $V = m + \lambda S$, the field-equation expression $E = \gamma^\mu D_\mu \Phi - V\Phi$ (a column of 16 numbers) and its adjoint $\bar{E} = (D_\mu \bar{\Phi})\gamma^\mu + V\bar{\Phi}$ (a row), exactly as defined in Section 9.8.

```
check(abs(np.trace(T) - (7 * K.sum() - 8 * (m * S + U))) < 1e-11
      and recorded(THEORY_WL, "EMT_trace_C") == "PASS",
      "trace: sum of T^mu_mu = 7 sum K - 8 (m S + U), off shell",
      record=f"{THEORY_WL}, check EMT_trace_C")
kinetic_sum = V * S + 0.5 * (bar(Phi) @ E - E_bar @ Phi)
check(abs(K.sum() - kinetic_sum) < 1e-11
      and recorded(THEORY_WL, "kinetic_sum_on_shell_C") == "PASS",
      "sum K = V S + (1/2)(Phibar E - Ebar Phi), off shell",
      record=f"{THEORY_WL}, check kinetic_sum_on_shell_C")
report("|E| of the random configuration (it is off shell)",
       f"{np.linalg.norm(E):.6f}")
```

`np.trace(T)` is the sum of the diagonal. The two checks confirm, for the random (off-shell) configuration, the trace identity and the kinetic-sum identity of Section 9.8. `np.linalg.norm(E)` is the length of the column E, the square root of the sum of the squared sizes of its 16 entries; it is 19.335808, far from zero: the random configuration does not satisfy the field equation.

```
# The evolution form of the field equation gives d4 Phi from the other derivatives.
rest = sum(gamma[a] @ dPhi[a] / f[a] for a in range(8) if a != 4)
```

```
dPhi_on = dPhi.copy()
dPhi_on[4] = -gamma[4] @ (V * Phi - rest - 3 * H * gamma[0] @ Phi)
E_on = sum(gamma_up[mu] @ (dPhi_on[mu] + Omega[mu] @ Phi) for mu in range(8)) - V * Phi
S_on, U_on, L0_on, K_on, T_on = energy_momentum(Phi, dPhi_on, f, gamma_up, gamma_down,
                                                Omega)
check(np.abs(E_on).max() < 1e-12, "with d4 Phi from the evolution form, E = 0")
```

Now the configuration is put on shell at the point. `rest` is $\sum_{a\neq 4} \frac{1}{f_a}\gamma^{(a)}\partial_a\Phi$ (the `if a != 3` in the comprehension skips x_4). `dPhi.copy()` makes a separate copy of the table, so that the random one is kept, and its row 3 is replaced by the evolution form of Section 9.2, $\partial_4\Phi = -\gamma^{(4)}[V\Phi - \text{rest} - 3H\gamma^{(0)}\Phi]$. The value of Φ and the other seven derivatives are unchanged, so S and V are unchanged. The field-equation expression is computed again, and the tensor of the new jet; the check confirms that now $E = 0$ up to rounding.

```
check(abs(K_on.real.sum() - V * S) < 1e-11
      and abs(L0_on.real - (S * lam * S - U)) < 1e-11
      and abs(np.trace(T_on.real) - (-m * S + 3 * lam * S ** 2)) < 1e-10
      and recorded(THEORY_PY, "commuting_trace_on_shell") == "PASS",
      "on shell: sum K = V S, L0 = S U' - U, trace = -m S + 3 lambda S^2",
      record=f"{THEORY_PY}, check commuting_trace_on_shell")
report("trace on shell, -m S + 3 lambda S^2", f"{np.trace(T_on.real):.6f}")
```

The check confirms the three on-shell values of Section 9.8: $\sum_\mu K_\mu = VS$, $L_0 = SU' - U$ (here `S * lam * S` is $SU'(S)$) and the trace $-mS + 3\lambda S^2$, which is printed: 70.447596. By hand, with the value of S that the notebook holds before rounding, $S = -6.5278747$: $-1\cdot(-6.5278747) + 3\cdot 0.5\cdot 6.5278747^2 = 6.527875 + 63.919721 = 70.447596$. (With the printed, rounded value $S = -6.527875$ the second term would come out as 63.919728 and the sum as 70.447603; the difference of $7\cdot 10^{-6}$ is the effect of rounding S in its seventh digit, multiplied by the slope $|-1+3S| \approx 20.6$ of the trace as a function of S.)

In [12], the spin connection in the entry $T^{x_4}{}_{x_1}$.

```
dPhi_hom = np.zeros((8, 16), dtype=complex)  # homogeneous: only d4 Phi is nonzero
dPhi_hom[4] = rng.normal(size=16) + 1j * rng.normal(size=16)
D_x1 = dPhi_hom[1] + Omega[1] @ Phi  # D_x1 Phi = Omega_x1 Phi here
Dbar_x1 = bar(dPhi_hom[1]) - bar(Phi) @ Omega[1]  # D_x1 Phibar = -Phibar Omega_x1
K_41 = 0.5 * (bar(Phi) @ gamma_up[4] @ D_x1 - Dbar_x1 @ gamma_up[4] @ Phi)
expected = 0.5 * np.exp(a4) * np.sin(z) ** (1 / 6) * H * (
    bar(Phi) @ gamma[4] @ gamma[1] @ gamma[0] @ Phi)
```

A homogeneous jet has only the time derivative: the table `dPhi_hom` is zero except row 3, which is drawn at random. Then $D_{x_1}\Phi = \Omega_{x_1}\Phi$ and $D_{x_1}\bar\Phi = -\bar\Phi\Omega_{x_1}$ (the derivative parts are zero), `K_41` is $K^{x_4}{}_{x_1} = \frac{1}{2}(\bar\Phi\gamma^{x_4}D_{x_1}\Phi - D_{x_1}\bar\Phi\gamma^{x_4}\Phi)$, and `expected` is the result of Section 9.12, $\frac{1}{2}e^{a_4}\sin^{1/6} z\, H\, \bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi$.

```
SCOPE_PY = "Revision/theory/reports/python-scope.json"
SCOPE_WL = "Revision/theory/reports/wolfram-scope.json"
check(abs(K_41 - expected) < 1e-12 and abs(K_41) > 0.01
      and recorded(SCOPE_PY, "spin_connection_in_the_energy_momentum_tensor") == "PASS"
```

```
      and recorded(SCOPE_WL, "spin_connection_in_the_energy_momentum_tensor") == "PASS",
      "K^x4_x1 of a homogeneous configuration is the nonzero connection term",
      record=f"{SCOPE_PY} and {SCOPE_WL}, check "
             "spin_connection_in_the_energy_momentum_tensor")
report("K^x4_x1 (homogeneous configuration)", f"{K_41.real:.6f}")
```

The check requires the two numbers to agree and, so that the test is not empty, the result to be clearly nonzero (larger than 0.01 in size). It prints $K^{x_4}{}_{x_1} = -0.203311$.

In [13], the Christoffel symbols with sympy.

```
import sympy as sp  # exact algebra and calculus with symbols

x = sp.symbols("x0:8", real=True)  # the coordinates x0, ..., x7 (x[0] ... x[7])
Hs = sp.symbols("H", positive=True)  # the author's constant, as a symbol
a4_function = sp.Function("a4")(x[4])  # a general function a4(x4)
zs = 6 * Hs * x[0]  # z = 6 H x0
space_entry = sp.exp(2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
extra_entry = -sp.exp(-2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
g = sp.diag(*([sp.cot(zs) ** 2] + [space_entry] * 3 + [-1] + [extra_entry] * 3))
```

From here on the notebook computes exactly, with symbols instead of numbers. `sp.symbols("x0:8", real=True)` makes the eight real symbols $x_0, \ldots, x_7$, stored as `x[0]` to `x[7]`; `Hs` is the symbol H, declared positive. `sp.Function("a4")(x[4])` is an unknown function $a_4(x_4)$: the results hold for every history. `sp.Rational(1, 3)` is the exact fraction $1/3$ (the Python number `1 / 3` would be rounded). `sp.diag(*list)` builds the diagonal matrix with the list on its diagonal (the star hands the items of the list over as separate arguments): the author's metric, symbol by symbol.

```
christoffel = {}  # (lambda, mu, nu) -> Gamma^lambda_mu nu, only the nonzero ones
for l in range(8):
    for i in range(8):
        for j in range(i, 8):  # mu <= nu
            value = sp.simplify((sp.diff(g[l, j], x[i]) + sp.diff(g[l, i], x[j])
                                 - sp.diff(g[i, j], x[l])) / (2 * g[l, l]))
            if value != 0:
                christoffel[(l, i, j)] = value
say(f"{len(christoffel)} nonzero Christoffel symbols with mu <= nu")
```

The three loops take every λ (`l`) and every pair $\mu \le \nu$ (`i`, `j`; `range(i, 8)` starts at `i`) and compute the diagonal-metric formula of Section 9.9, $\frac{1}{2g_{\lambda\lambda}}(\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu})$; `sp.diff(expr, x[i])` is the partial derivative and `sp.simplify` brings the result to a simplest form. The nonzero symbols are stored in the dictionary `christoffel` under the key `(l, i, j)`. There are $8 \cdot 36 = 288$ expressions to simplify, which takes a few seconds. The cell prints `25 nonzero Christoffel symbols with mu <= nu`, the count of Section 9.9.

In [14], comparison with the record's list.

```
a4s, a4ps = sp.symbols("a4 a4p", real=True)  # plain symbols for a4 and da4/dx4
```

```
def plain(expression):
    """Replace da4/dx4 by the symbol a4p and a4(x4) by the symbol a4."""
    return expression.subs(sp.Derivative(a4_function, x[4]), a4ps).subs(
        a4_function, a4s)
```

The record writes a_4 and a_4' as plain names. `plain` makes our symbols comparable: `.subs(old, new)` replaces every occurrence of `old` by `new`; the derivative is replaced first (replacing $a_4(x_4)$ first would destroy it).

```
WOLFRAM_TO_SYMPY = [  # (Wolfram text, sympy text), applied in this order
    ("Derivative[1][a4][x4]", "a4p"), ("Derivative[1, 0][rho][x4, x0]", "rho_4"),
    ("Derivative[0, 1][p0][x4, x0]", "p0_0"), ("[x4, x0]", ""), ("a4[x4]", "a4"),
    ("Sin[", "sin("), ("Cos[", "cos("), ("Cot[", "cot("), ("Sec[", "sec("),
    ("Csc[", "csc("), ("E^", "E**"), ("^", "**"), ("[", "("), ("]", ")"),
    ("{", "["), ("}", "]")]


def from_wolfram(text):
    """Translate a formula of the record from the Wolfram Language into sympy."""
    for old, new in WOLFRAM_TO_SYMPY:
        text = text.replace(old, new)
    return text
```

The record's formulas are written in the Wolfram Language, which uses square brackets for the arguments of functions (`Sin[z]`), `^` for powers and braces for lists. `from_wolfram` translates such a text into sympy's notation by plain text replacements, in the order of the list: the long derivative names first, then the function names, then the single brackets and braces.

```
FORMULAS = {item["key"]: item["wl"] for item in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
NAMES_SYMPY = {"a4": a4s, "a4p": a4ps, "H": Hs, "x0": x[0], "E": sp.E}
record_list = sp.sympify(from_wolfram(FORMULAS["christoffel_nonzero"]),
                         locals=NAMES_SYMPY)
# The record counts the coordinates 1 ... 8; Python counts 0 ... 7.
record_symbols = {(int(e[0]) - 1, int(e[1]) - 1, int(e[2]) - 1): e[3]
                  for e in record_list}
```

`FORMULAS` is a **dictionary comprehension**: it maps the key of every formula of the record file to its Wolfram text. `sp.sympify(text, locals=...)` reads a translated text as a sympy expression, with the names `a4`, `a4p`, `H`, `x0` and `E` (Euler's number e) bound to our symbols. The record's list consists of entries $\{\lambda, \mu, \nu, \Gamma^{\lambda}{}_{\mu\nu}\}$ with the coordinates counted from 1; `record_symbols` stores them under keys counted from 0.

```
same = sorted(record_symbols) == sorted(christoffel) and all(
    sp.simplify(plain(christoffel[key]) - record_symbols[key]) == 0
    for key in christoffel)
check(len(christoffel) == 25 and same
      and recorded(THEORY_WL, "christoffel_count") == "PASS",
      "25 nonzero Christoffel symbols, equal to the record's list one by one",
```

```
      record=f"{THEORY_WL}, check christoffel_count (formula christoffel_nonzero)")
for (l, i, j), value in sorted(christoffel.items()):
    print(f"Gamma^{NAMES[l]}_({NAMES[i]} {NAMES[j]}) = {plain(value)}")
```

`same` is true when both dictionaries have the same 25 keys (`sorted` lists the keys in order) and, for each key, the difference of the two symbols simplifies to exactly 0. The loop prints the 25 symbols in order; sympy writes cot as `1/tan` and sec z as `1/cos`. The printed list is the table of Section 9.9; for example `Gamma^x0_(x0 x0) = -12*H/sin(12*H*x0)`.

In [15], the covariant divergence of a diagonal tensor.

```
rho_f, p3_f, pt_f, p0_f = [sp.Function(n)(x[4], x[0]) for n in ("rho", "p3", "pt",
                                                                "p0")]
T_diag = sp.diag(p0_f, p3_f, p3_f, p3_f, -rho_f, pt_f, pt_f, pt_f)  # T^mu_nu


def gamma_symbol(l, i, j):
    """Gamma^l_ij for any order of i and j (zero when it is not in the list)."""
    return christoffel.get((l, min(i, j), max(i, j)), 0)
```

`rho_f`, `p3_f`, `pt_f` and `p0_f` are four unknown functions of x_4 and x_0, and `T_diag` is the diagonal tensor $\mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0)$ of Section 9.9. `gamma_symbol(l, i, j)` returns $\Gamma^{l}{}_{ij}$ for either order of `i` and `j` (the symbol is symmetric, and the dictionary stores only $i \le j$); `.get(key, 0)` returns 0 for a symbol that is not stored.

```
divergence = []
for nu in range(8):
    value = sum(sp.diff(T_diag[mu, nu], x[mu]) for mu in range(8))
    value += sum(gamma_symbol(mu, mu, l) * T_diag[l, nu]
                 for mu in range(8) for l in range(8))
    value -= sum(gamma_symbol(l, mu, nu) * T_diag[mu, l]
                 for mu in range(8) for l in range(8))
    divergence.append(sp.simplify(value))
nonzero = [NAMES[nu] for nu in range(8) if divergence[nu] != 0]
say("components of the divergence that are not identically zero: nu = "
    + ", ".join(nonzero))
```

For each ν the loop builds the three sums of the general formula of Section 9.9 literally (without the lemma): $\sum_\mu \partial_\mu T^\mu{}_\nu$, plus $\sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda} T^\lambda{}_\nu$, minus $\sum_{\mu,\lambda} \Gamma^\lambda{}_{\mu\nu} T^\mu{}_\lambda$ (`+=` adds to `value`, `-=` subtracts from it), simplifies the result and appends it to the list. `", ".join(...)` writes the names of the nonzero components separated by commas. The cell prints `nu = x4, x0`: the other six components vanish identically.

In [16], comparison with the record's identities.

```
rho_4, p0_0, p3s, pts, p0s = sp.symbols("rho_4 p0_0 p3 pt p0", real=True)


def plain_t(expression):
    """Write the derivatives and functions as the plain symbols of the record."""
```

```
    expression = expression.subs({sp.Derivative(rho_f, x[4]): rho_4,
                                  sp.Derivative(p0_f, x[0]): p0_0})
    expression = expression.subs({p3_f: p3s, pt_f: pts, p0_f: p0s,
                                  rho_f: sp.Symbol("rho", real=True)})
    return plain(expression)
```

`plain_t` writes our results with the record's plain names: $\partial_4\rho$ as `rho_4`, $\partial_0 p_0$ as `p0_0`, and the four functions as plain symbols; `.subs` also accepts a dictionary of replacements.

```
NAMES_T = dict(NAMES_SYMPY, rho_4=rho_4, p0_0=p0_0, p3=p3s, pt=pts, p0=p0s)
record_divergence = sp.sympify(from_wolfram(FORMULAS["energy_exchange"][0]),
                               locals=NAMES_T)
agree = all(sp.simplify(plain_t(divergence[n]) - record_divergence[n]) == 0
            for n in range(8))
x4_line = -rho_4 - 3 * a4ps * (p3s - pts)
x0_line = p0_0 + 3 * Hs * sp.cot(zs) * (2 * p0s - p3s - pts)
```

`NAMES_T` extends the dictionary of names. The record's formula `energy_exchange` is a list of three texts; the first one (`[0]`) is the list of all eight components of the divergence, which `record_divergence` reads. `agree` is true when each of our eight components equals the record's, exactly. `x4_line` and `x0_line` are the two results of Section 9.9, typed by hand.

```
check(agree and recorded(THEORY_WL, "energy_exchange_equation") == "PASS"
      and recorded(THEORY_PY, "energy_exchange_equation") == "PASS",
      "the eight components agree with the record formula energy_exchange",
      record=f"{THEORY_WL} and {THEORY_PY}, check energy_exchange_equation")
check(sp.simplify(plain_t(divergence[4]) - x4_line) == 0
      and recorded(LEAD, "divergence_x4_component") == "PASS",
      "nabla_mu T^mu_x4 = -d4 rho - 3 a4p (p3 - p_t)",
      record=f"{LEAD}, check divergence_x4_component")
check(sp.simplify(plain_t(divergence[0]) - x0_line) == 0
      and recorded(LEAD, "divergence_x0_component") == "PASS",
      "nabla_mu T^mu_x0 = d0 p0 + 3 H cot z (2 p0 - p3 - p_t)",
      record=f"{LEAD}, check divergence_x0_component")
check(all(divergence[n] == 0 for n in (1, 2, 3, 5, 6, 7))
      and recorded(LEAD, "divergence_other_components_zero") == "PASS",
      "the components x1, x2, x3, x5, x6, x7 vanish identically",
      record=f"{LEAD}, check divergence_other_components_zero")
print("nabla_mu T^mu_x4 =", x4_line)  # rho_4 = d rho/d x4
print("nabla_mu T^mu_x0 =", x0_line)  # p0_0 = d p0/d x0
```

Four checks: all eight components agree with the record's formula (whose check passed in both engines); the x_4 component and the x_0 component equal the hand-derived lines (and the lead's independent checks passed); and the other six components are zero. The last two lines print the two identities in sympy's spelling, `-3*a4p*(p3 - pt) - rho_4` and `3*H*(-p3 + 2*p0 - pt)*cot(6*H*x0) + p0_8`.

In [17], the first-law reading.

```
f_space = sp.exp(a4_function) * sp.sin(zs) ** sp.Rational(1, 6)  # f1 = f2 = f3
f_extra = sp.exp(-a4_function) * sp.sin(zs) ** sp.Rational(1, 6)  # f5 = f6 = f7
```

```
V3 = f_space ** 3  # e^(3 a4) sin^(1/2) z
Vt = f_extra ** 3  # e^(-3 a4) sin^(1/2) z
V7 = V3 * Vt * sp.cot(zs)  # times f0 = cot z (and f4 = 1)
check(sp.simplify(V7 - sp.cos(zs)) == 0, "V3 Vt f0 = cos z, independent of x4")
```

These lines build the three volumes of Section 9.10 with symbols and check $V_7 = \cos z$ exactly.

```
first_law = (sp.diff(rho_f * V7, x[4]) + p3_f * V7 / V3 * sp.diff(V3, x[4])
             + pt_f * V7 / Vt * sp.diff(Vt, x[4]))
# The first law says first_law = 0; it must be -V7 times the x4 divergence.
check(sp.simplify(first_law + V7 * divergence[4]) == 0,
      "d(rho V7)/dx4 + p3 (V7/V3) dV3/dx4 + p_t (V7/Vt) dVt/dx4 = -V7 nabla_mu T^mu_x4")
```

`first_law` is the first law of Section 9.10 with everything on one side, $\frac{d}{dx_4}(\rho V_7) + p_3 \frac{V_7}{V_3}\frac{dV_3}{dx_4} + p_t \frac{V_7}{V_t}\frac{dV_t}{dx_4}$. The check shows that it equals $-V_7$ times the x_4 component of the divergence: setting one to zero is the same as setting the other to zero. The reading is an interpretation of an exact identity, not an extra result.

In [18], the volumes along the history (Figure 09a.4).

```
x4_axis = np.linspace(0.0, 8.0, 401)  # the time x4
growth = np.exp(3 * A * H * x4_axis)  # V3(x4)/V3(0) = e^(3 A H x4)
fig, ax = plt.subplots()
ax.plot(x4_axis, growth, label="3-space volume $V_3$ (inflates)")
ax.plot(x4_axis, 1 / growth, "--", label="extra-time volume $V_t$ (deflates)")
ax.plot(x4_axis, growth / growth, ":", color="black",
        label="7-volume $V_7 = V_3 V_t \\cot z$ (constant)")
ax.set_yscale("log")
ax.set_xlabel("time $x_4$")
ax.set_ylabel("volume divided by its value at $x_4 = 0$")
ax.set_title("Volumes along the history $a_4 = A H x_4$ ($A = 1$, $H = 0.25$)")
ax.legend()
save_figure(fig, "volumes_first_law",
            "The 3-space volume $V_3 = e^{3a_4}\\sin^{1/2}z$ (solid), the "
            ...)
```

On the history $a_4 = AHx_4$ the ratio $V_3(x_4)/V_3(0)$ is e^{3AHx_4}, the ratio for V_t is its inverse, and the ratio for V_7 is 1 (`growth / growth`). **What Figure 09a.4 shows.** The three volume ratios (pure numbers) against the time x_4 from 0 to 8 on a logarithmic axis: V_3 rises along a straight line to $e^6 \approx 403$, V_t falls along the mirror line to e^{-6}, and V_7 stays at 1. The student should see that the extra times deflate, exponentially, and that their shrinking exactly compensates the growth of 3-space; this is why the energy density changes only through the difference $p_3 - p_t$.

In [19], the x0 identity with sympy.

```
P, c = sp.symbols("P c", real=True)  # P = p3 + p_t (a constant) and the constant c
profile = P / 2 + c / sp.sin(zs)  # p0 as a function of x0 (through z)
balance = sp.diff(profile, x[0]) + 3 * Hs * sp.cot(zs) * (2 * profile - P)
check(sp.simplify(balance) == 0,
      "p0 = P/2 + c/sin z solves the x0 identity for every c")
```

`balance` is the x_0 component of the divergence for $p_0 = P/2 + c/\sin z$ and $p_3 + p_t = P$; the check confirms that it is zero for every c, the solution found in Section 9.11.

```
y = sp.symbols("y", real=True)
dy_dx0 = sp.diff(sp.log(sp.sin(zs)) / (6 * Hs), x[0])  # y = ln(sin z)/(6 H)
check(sp.simplify(dy_dx0 - sp.cot(zs)) == 0
      and recorded(LEAD, "ks_coordinate_jacobian") == "PASS",
      "y = ln(sin z)/(6 H) has dy/dx0 = cot z",
      record=f"{LEAD}, check ks_coordinate_jacobian")
```

This check differentiates $y = \ln(\sin z)/(6H)$ with respect to x_0 and confirms $dy/dx_0 = \cot z$.

```
z_of_y = sp.asin(sp.exp(6 * Hs * y))  # sin z = e^(6 H y), z between 0 and pi/2
P0, P3, PT = [sp.Function(n)(y) for n in ("P0", "P3", "PT")]
in_x0 = sp.cot(z_of_y) * sp.diff(P0, y) + 3 * Hs * sp.cot(z_of_y) * (2 * P0 - P3 - PT)
in_y = sp.cot(z_of_y) * (sp.diff(P0, y) + 6 * Hs * P0 - 3 * Hs * (P3 + PT))
check(sp.simplify(in_x0 - in_y) == 0
      and recorded(LEAD, "ks_coordinate_form") == "PASS",
      "x0 component = cot z [p0'(y) + 6 H p0 - 3 H (p3 + p_t)]",
      record=f"{LEAD}, check ks_coordinate_form")
```

Solving $\sin z = e^{6Hy}$ for z gives $z = \arcsin(e^{6Hy})$ (`sp.asin`). `P0`, `P3` and `PT` are the three pressures as functions of y. `in_x0` is the x_0 component with $\partial_0 p_0 = \cot z \, dp_0/dy$, and `in_y` the form of Section 9.11; the check confirms that they are the same expression.

In [20], the pressure profiles of the x0 balance (Figure 09a.5).

```
z_grid = np.linspace(0.1, np.pi / 2 - 0.01, 2001)  # z from 0.1 to just below pi/2
x0_grid = z_grid / (6 * H)  # the matching x0 = z/(6 H)
fig, ax = plt.subplots()
worst = 0.0  # the largest relative residual of the balance on the grid
for c_value in (-0.2, -0.1, 0.0, 0.1, 0.2):
    p0_values = 0.5 + c_value / np.sin(z_grid)  # P = 1
    slope = np.gradient(p0_values, x0_grid)  # d p0/d x0, by finite differences
    rhs = -3 * H / np.tan(z_grid) * (2 * p0_values - 1.0)
```

The grid has 2001 values of z between 0.1 and $\pi/2 - 0.01$, and the matching $x_0 = z/(6H)$. For each of five constants c the loop computes the profile $p_0 = 1/2 + c/\sin z$ (so $P = 1$), its slope dp_0/dx_0 by **finite differences** (`np.gradient` approximates a derivative by the difference of neighbouring values divided by their distance), and the right-hand side $-3H \cot z \, (2p_0 - 1)$ of the balance.

```
    # The size of the terms; 1 (the size of P) is added, so that for the flat
    # profile c = 0, whose terms are zero, rounding noise is not divided by zero.
    scale = 1.0 + np.abs(slope).max() + np.abs(rhs).max()
    # [5:-5] leaves out five points at each end, where np.gradient is less exact.
    worst = max(worst, np.abs(slope - rhs)[5:-5].max() / scale)
    style = "-" if c_value == 0.0 else "--"
    ax.plot(z_grid, p0_values, style, label=f"$c = {c_value:g}$")  # g: short form
check(worst < 1e-4, "the drawn profiles satisfy the x0 balance on the grid")
```

The residual, slope minus right-hand side, is divided by the size of the terms; `[5:-5]` drops five points at each end of the grid, and `worst` keeps the largest relative residual of the five profiles. The profile with $c = 0$ is drawn solid, the others dashed; the format `:g` writes a number in its shortest form. The check requires the residual to be below 10^{-4}: it measures the error of the finite differences, not a failure of the identity.

```
ax.set_ylim(-2.0, 3.0)
ax.set_xlabel("$z = 6 H x_0$")
ax.set_ylabel("$p_0$ (with $p_3 + p_t = 1$)")
ax.set_title("Pressure profiles allowed by the balance along $x_0$")
ax.legend()
save_figure(fig, "hidden_balance",
            "The hidden-direction pressure profiles $p_0 = P/2 + c/\\sin z$ that "
            "satisfy the conservation identity along $x_0$, "
            "$\\partial_0 p_0 = -3H\\cot z\\,(2p_0 - p_3 - p_t)$, when "
            ...)
report("largest relative residual of the balance on the grid", f"{worst:.1e}")
```

These lines fix the vertical range, label the axes, save the figure and print the residual, $2.3 \cdot 10^{-5}$ (the format `.1e` writes one decimal and a power of ten). **What Figure 09a.5 shows.** The hidden pressure p_0 (energy per unit volume) against z from 0.1 to just below $\pi/2$ for five values of c. Only the solid line $c = 0$ is flat, at $p_0 = P/2 = 0.5$; every other profile grows in size like $1/\sin z$ towards the tip $z = 0$, upwards for $c > 0$ and downwards for $c < 0$. The student should see that an x_0-independent tensor must have $p_0 = (p_3 + p_t)/2$.

In [21], the last check.

```
captions = json.loads(output_file(CAPTION_FILE).read_text(encoding="utf-8"))
expected_files = ["09a_1_kinetic_weights.png", "09a_2_tensor_heat_map.png",
                  "09a_3_diagonal_parts.png", "09a_4_volumes_first_law.png",
                  "09a_5_hidden_balance.png"]
check(sorted(captions) == expected_files and all(
    output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected_files),
    "the five figures of this notebook are saved and captioned")
all_checks_passed()
```

The cell reads the captions file back, checks that it holds exactly the five figure names (sorted) and that the five files exist, and prints the last line, `ALL 27 CHECKS PASSED (notebook 09a)`: two checks each in In [2], In [3], In [5], In [7] and In [9], four in In [11], one each in In [12] and In [14], four in In [16], two in In [17], three in In [19], and one each in In [20] and In [21].

9.18 Condensates: exact solutions that depend on the time only

A **condensate** (also called a **homogeneous configuration**) is a field that is the same at every place: it depends on the time x_4 only, not on $x_1, x_2, x_3, x_5, x_6, x_7, x_0$. Condensates are the simplest candidates for a smooth cosmological fluid, and the field equation can be solved for them exactly.

The field equation of a condensate. In the explicit field equation of Section 9.2 every derivative term with $a \neq 4$ vanishes, and $f_4 = 1$:

$$\gamma^{(4)}\partial_4\Phi + 3H\gamma^{(0)}\Phi = V\Phi, \qquad V = m + \lambda S.$$

Multiply from the left by $-\gamma^{(4)}$. Because $(\gamma^{(4)})^2 = -1$, the first term becomes $-\gamma^{(4)}\gamma^{(4)}\partial_4\Phi = \partial_4\Phi$, the second $-3H\gamma^{(4)}\gamma^{(0)}\Phi$, and the right side $-V\gamma^{(4)}\Phi$. Moving the second term to the right:

$$\partial_4\Phi = M\Phi, \qquad M = -V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}.$$

Neither a_4 nor x_0 appears: the same condensate solves the field equation on every history $a_4(x_4)$, the deflating one included. (This is the effect of Section 9.3: the connection enters only as $3H\gamma^{(0)}$, which contains no a_4.)

The scalar S is constant. The equation is nonlinear, because V contains $S = \Phi^\dagger C\Phi$. But S does not change. By the product rule,

$$\frac{dS}{dx_4} = (\partial_4\Phi)^\dagger C\Phi + \Phi^\dagger C\,\partial_4\Phi = (M\Phi)^\dagger C\Phi + \Phi^\dagger CM\Phi = \Phi^\dagger\big(M^TC + CM\big)\Phi,$$

where the last step uses $(M\Phi)^\dagger = \Phi^\dagger M^\dagger$ and $M^\dagger = M^T$ (M is real). Now compute M^TC. From the antisymmetry of $C\gamma^{(a)}$ and the symmetry of C: $\gamma^{(a)T}C = (C\gamma^{(a)})^T = -C\gamma^{(a)}$. Hence

$$\begin{aligned}
\gamma^{(4)T}C &= -C\gamma^{(4)},\\
\big(\gamma^{(4)}\gamma^{(0)}\big)^TC &= \gamma^{(0)T}\gamma^{(4)T}C = -\gamma^{(0)T}C\gamma^{(4)} = C\gamma^{(0)}\gamma^{(4)} = -C\gamma^{(4)}\gamma^{(0)},\\
M^TC &= -V\gamma^{(4)T}C + 3H\big(\gamma^{(4)}\gamma^{(0)}\big)^TC = VC\gamma^{(4)} - 3HC\gamma^{(4)}\gamma^{(0)} = -CM.
\end{aligned}$$

The first line is the rule just stated; the second applies it twice (the transpose of a product is the product of the transposes in reverse order) and then uses $\gamma^{(0)}\gamma^{(4)} = -\gamma^{(4)}\gamma^{(0)}$; the third inserts both. So $M^TC + CM = 0$ and $dS/dx_4 = 0$: S keeps its initial value $S_0 = \chi^\dagger C\chi$, where $\chi = \Phi(0)$. Then $V = m + \lambda S_0$ is a constant, and M is a constant matrix.

The square of M. Multiply out $M^2 = (-V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)})(-V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)})$:

$$\begin{aligned}
M^2 &= V^2\gamma^{(4)}\gamma^{(4)} - 3HV\big(\gamma^{(4)}\gamma^{(4)}\gamma^{(0)} + \gamma^{(4)}\gamma^{(0)}\gamma^{(4)}\big) + 9H^2\gamma^{(4)}\gamma^{(0)}\gamma^{(4)}\gamma^{(0)}\\
&= -V^2 - 3HV\big(-\gamma^{(0)} + \gamma^{(0)}\big) + 9H^2\cdot 1\\
&= \big(9H^2 - V^2\big)\cdot 1 = k^2\cdot 1, \qquad k^2 = 9H^2 - V^2.
\end{aligned}$$

The first line multiplies out the four products; the second uses $\gamma^{(4)}\gamma^{(4)} = -1$, then $\gamma^{(4)}\gamma^{(0)}\gamma^{(4)} = -\gamma^{(4)}\gamma^{(4)}\gamma^{(0)} = \gamma^{(0)}$, and $\gamma^{(4)}\gamma^{(0)}\gamma^{(4)}\gamma^{(0)} = -\gamma^{(4)}\gamma^{(4)}\gamma^{(0)}\gamma^{(0)} = -(-1)(+1) = 1$; the third collects.

The solution. For a number k the exponential series $e^{kx} = \sum_n (kx)^n/n!$ splits into its even and odd terms, $\cosh(kx) = \sum_j (kx)^{2j}/(2j)!$ and $\sinh(kx) = \sum_j (kx)^{2j+1}/(2j+1)!$. The same series with the matrix Mx_4 in place of kx defines the **matrix exponential** e^{Mx_4}, and $e^{Mx_4}\chi$ solves $\partial_4\Phi = M\Phi$ with $\Phi(0) = \chi$. Because $M^2 = k^2$, every even power is $M^{2j} = k^{2j}$ and every odd power is $M^{2j+1} = k^{2j}M$, so the even terms sum to $\cosh(kx_4)$ and the odd terms to $\frac{\sinh(kx_4)}{k}M$:

$$\Phi(x_4) = \Big(\cosh(kx_4) + \frac{\sinh(kx_4)}{k}\,M\Big)\chi.$$

A direct check, without series: at $x_4 = 0$ this is χ; its derivative is $\big(k\sinh(kx_4) + \cosh(kx_4)M\big)\chi$; and $M\Phi = \big(\cosh(kx_4)M + \frac{\sinh(kx_4)}{k}M^2\big)\chi = \big(\cosh(kx_4)M + k\sinh(kx_4)\big)\chi$ (using $M^2 = k^2$), the same. Three cases:

- $k^2 < 0$, that is $|V| > 3H$: $k = i\omega$ with $\omega = \sqrt{V^2 - 9H^2}$; since $\cosh(i\omega x) = \cos(\omega x)$ and $\sinh(i\omega x)/(i\omega) = \sin(\omega x)/\omega$, the condensate **oscillates** with the angular frequency ω;
- $k^2 > 0$, that is $|V| < 3H$: the solution contains e^{kx_4} and **grows**;
- $k^2 = 0$: the limit $\Phi = \chi + x_4 M\chi$.

These are the record's exact nonlinear homogeneous solutions (PROVED; record formula `exact_solutions` of `Revision/theory/field-theory.json`, and the checks in the table).

statement	**Revision report**	**check**
$M^2 = k^2 \cdot 1$ and $M^T C + CM = 0$	`Revision/theory/reports/wolfram-field-theory.json`	`solution_matrix_square`
the condensate solves the field equation, $S = S_0$, and its ρ, p	`Revision/theory/reports/wolfram-field-theory.json`	`exact_solution_nonlinear_homogeneous_C`
the same, verified independently with sympy	`Revision/theory/reports/python-field-theory.json`	`exact_nonlinear_homogeneous_solution`

9.19 The energy density, pressure and equation of state of a condensate

The kinetic terms. For a condensate $\partial_\mu\Phi = 0$ for $\mu \neq x_4$, so every kinetic term except K_4 is zero. On shell the kinetic sum of Section 9.8 gives $\sum_\mu K_\mu = VS$, hence $K_4 = VS$.

Energy density and pressures. Insert into Section 9.7:

$$\rho = -\sum_{\mu \neq x_4} K_\mu + mS + U = 0 + mS + U(S),$$
$$p_\mu = \sum_{\nu \neq \mu} K_\nu - mS - U = K_4 - mS - U = (m + U')S - mS - U = S\,U'(S) - U(S) \qquad (\mu \neq x_4).$$

In the first line all seven kinetic terms vanish; in the second the only kinetic term left in the sum is K_4 (because $\mu \neq x_4$), which is $VS = (m + U')S$. The pressure is the same in all seven directions, $p_3 = p_t = p_0 = p$. The kinetic part of the energy density is zero, the kinetic part of every pressure is VS, the potential parts are $\pm(mS + U)$. For $U = \frac{\lambda}{2}S^2$, with $SU' = \lambda S^2$:

$$\rho = mS + \tfrac{\lambda}{2}S^2, \qquad p = \lambda S^2 - \tfrac{\lambda}{2}S^2 = \tfrac{\lambda}{2}S^2.$$

(PROVED; `Revision/theory/reports/python-field-theory.json`, check `commuting_homogeneous_on_shell_rho_p`; record formula `EMT_homogeneous_on_shell`.) The trace is $-\rho + 7p =$

$-mS - \frac{\lambda}{2}S^2 + \frac{7\lambda}{2}S^2 = -mS + 3\lambda S^2$, as Section 9.8 requires. Because S is constant, ρ and p are constant in time; this agrees with the energy exchange of Section 9.10, since $p_3 = p_t$ gives $d\rho/dx_4 = 0$.

No x_4-x_0 flow. The record gives the entry (formula `EMT_offdiagonal_x4_x0`)

$$T^{x_4}{}_{x_0} = -\tfrac{1}{4}(B_{40} - \cot z\, B_{04}), \qquad B_{40} = \bar\Phi\gamma^{(4)}\partial_0\Phi - \partial_0\bar\Phi\,\gamma^{(4)}\Phi, \qquad B_{04} = \bar\Phi\gamma^{(0)}\partial_4\Phi - \partial_4\bar\Phi\,\gamma^{(0)}\Phi.$$

For a condensate $B_{40} = 0$ (no x_0 derivative). For B_{04}: $\partial_4\Phi = M\Phi$ and $\partial_4\bar\Phi = (M\Phi)^\dagger C = \Phi^\dagger M^T C = -\Phi^\dagger C M = -\bar\Phi M$, so

$$B_{04} = \bar\Phi\gamma^{(0)}M\Phi + \bar\Phi M\gamma^{(0)}\Phi = \bar\Phi\{\gamma^{(0)}, M\}\Phi = \bar\Phi(-V\{\gamma^{(0)}, \gamma^{(4)}\} + 3H\{\gamma^{(0)}, \gamma^{(4)}\gamma^{(0)}\})\Phi = 0,$$

because $\{\gamma^{(0)}, \gamma^{(4)}\} = 2\eta^{84} = 0$ and $\{\gamma^{(0)}, \gamma^{(4)}\gamma^{(0)}\} = \gamma^{(0)}\gamma^{(4)}\gamma^{(0)} + \gamma^{(4)}\gamma^{(0)}\gamma^{(0)} = -\gamma^{(4)} + \gamma^{(4)} = 0$. So $T^{x_4}{}_{x_0} = 0$, and $T^{x_0}{}_{x_4} = -\tan^2 z\, T^{x_4}{}_{x_0} = 0$ (PROVED; check `commuting_T_x4x0_homogeneous`). The other entries off the diagonal are not zero in general (Section 9.26).

The equation of state. For $S \neq 0$, divide the numerator and the denominator of p/ρ by $mS/2$:

$$w = \frac{p}{\rho} = \frac{\frac{\lambda}{2}S^2}{mS + \frac{\lambda}{2}S^2} = \frac{\lambda S/m}{2 + \lambda S/m} = \frac{x}{2+x}, \qquad x = \frac{\lambda S}{m}.$$

(The record writes it $w = \lambda S/(2m + \lambda S)$.) It depends on m, λ and S only through x, and the effective mass is $V = m + \lambda S = m(1 + x)$. Its special values, each by elementary algebra:

$x = \lambda S/m$	w	**why**
$x = 0$ ($\lambda = 0$)	$w = 0$ (like dust)	$p = 0$
$x = 1$	$w = 1/3$ (like radiation)	$1/(2+1)$
$x = -1$	$w = -1$ (like a cosmological constant)	$-1/(2-1)$; here $V = m(1+x) = 0$
$-2 < x < -1$	$w < -1$ (called **phantom**)	$2 + x > 0$, and $x < -(2+x)$ means $x < -1$
$x = -2$	not defined	$\rho = mS(1 + x/2) = 0$
$x < -2$ (with $m > 0$, $S > 0$)	$w > 1$	$\rho < 0$ and $p < 0$; $x/(2+x) > 1$ means $x < 2 + x$ after multiplying by the negative $2 + x$
$x \to \pm\infty$	$w \to 1$	$x/(2+x) = 1/(1 + 2/x)$

What these numbers are, and what they are not. They are the ratios p/ρ of the eight-dimensional tensor of exact solutions, constant in time (COMPUTED in Notebook 09b on twelve exact solutions; the formula is PROVED in the record). They are not the equation of state that an observer in 3-space would infer: that observer sees neither the extra times nor the hidden direction as space, and to relate the two one must integrate over x_5, x_6, x_7, x_0 and follow the deflation. This chapter makes no such computation. The Revision dark-sector record makes it under an ASSUMED

normalisation of the observer's energy density (the definitions A, B and C of Chapter 22): for a homogeneous condensate of either field, the equation of state w_{eff} that the observer infers from the dilution of that density is 0 (dust-like) under A and B and -1 under C, constant in time and independent of x wherever $\rho \neq 0$, because $p_3 = p_t$ (PROVED in that record; `Revision/docs/DARK_SECTOR_HYPOTHESES.md`, sections 3.2 and 13.1). Which definition, if any, describes a physical observer, and a fit to supernova data, are OPEN. In particular, the phantom range $w < -1$ of this table is not a claim about dark energy: the commuting field has negative energies (Section 9.21), and any use of them for a phantom equation of state is a HYPOTHESIS, not a result (the record's dark-sector study obtains a crossing of $w = -1$ only in a model with a component of negative classical energy whose share was chosen; Chapter 22).

9.20 Where condensates oscillate and where they grow

With $V = m(1 + x)$ the number $k^2 = 9H^2 - V^2 = 9H^2 - m^2(1 + x)^2$ is positive exactly when $|1 + x| < 3H/|m|$. In the plane of the two pure numbers x and $3H/|m|$ the growing condensates fill the wedge between the lines $3H/|m| = |1 + x|$; outside it they oscillate. The equation of state depends on x alone, so every line of constant w is vertical in this plane. At $x = -1$ ($w = -1$) the effective mass vanishes, $k^2 = 9H^2 > 0$, and the condensate grows like e^{3Hx_4}: every condensate with $w = -1$ grows (for $H > 0$).

The two condensates of Notebook 09b have $m = 1$, $H = 0.25$ (so $3H = 0.75$) and $S_0 = 1$:

- $\lambda = 0.5$: $V = 1.5 > 0.75$, $k^2 = 0.5625 - 2.25 = -1.6875$; it oscillates with $\omega = \sqrt{1.6875} = 1.299$, that is with the period $2\pi/\omega = 4.84$;
- $\lambda = -0.5$: $V = 0.5 < 0.75$, $k^2 = 0.5625 - 0.25 = 0.3125$, $k = 0.559$; it grows: for large x_4 its size $\Phi^\dagger\Phi$ grows like e^{2kx_4}, and the exponential factor alone reaches $e^{2 \cdot 0.559 \cdot 12} \approx 7 \cdot 10^5$ at $x_4 = 12$ (Figure 09b.2 shows a growth by more than a factor 100000).

In both cases S stays exactly at 1. A growing Φ with a constant $S = \Phi^\dagger C \Phi$ is possible because C has eight eigenvalues $+1$ and eight -1: the growth happens along directions in which the indefinite form $\Phi^\dagger C \Phi$ does not grow. Whether growing condensates are acceptable physically is the question of well-posedness and boundary conditions discussed in Chapter 8; this chapter only computes their tensor.

9.21 The energy of the commuting field has no lower bound

S has no sign. C is real and symmetric with $C^2 = 1$, so its eigenvalues are $+1$ or -1; the record writes $C = \text{diag}(-\sigma, \sigma)$ (the block $-\sigma$ in the upper left corner, σ in the lower right, zeros elsewhere) with a real symmetric 8×8 matrix σ that obeys $\sigma^2 = 1$ and has trace 0. The eigenvalues of σ are therefore $+1$ or -1 (if $\sigma v = \mu v$ then $v = \sigma^2 v = \mu^2 v$, so $\mu^2 = 1$), and because the trace of a symmetric matrix is the sum of its eigenvalues, trace 0 means four eigenvalues $+1$ and four -1. The block $-\sigma$ has the same eigenvalues with the signs exchanged, again four of each. So C has eight eigenvalues $+1$ and eight -1 (PROVED; see the table at the end of this section). For an eigenvector χ with eigenvalue -1, $S_0 = \chi^\dagger C \chi = -\chi^\dagger \chi < 0$. A condensate with $\lambda = 0$ has $\rho = mS_0$: for $m > 0$ its energy density is negative whenever $S_0 < 0$.

Plane waves of positive frequency with negative energy. Take flat 4+4 space (all $f_a = 1$, no spin connection), $U = 0$, and a **plane wave** $\Phi = u\, e^{i(k_1x_1+k_2x_2+k_3x_3+k_0x_0-Ex_4)}$ with a constant column u, no dependence on the extra times, and a positive frequency E. Then $\partial_a\Phi = ik_a\Phi$ for $a = 1, 2, 3, 8$, $\partial_4\Phi = -iE\Phi$, and $\partial_a\bar\Phi = -ik_a\bar\Phi$ (complex conjugation turns i into $-i$). Line by line:

$$
\begin{aligned}
K_a &= \tfrac{1}{2}\big(\bar\Phi\gamma^{(a)}(ik_a\Phi) - (-ik_a\bar\Phi)\gamma^{(a)}\Phi\big) = ik_a\,\bar\Phi\gamma^{(a)}\Phi \qquad (a = 1, 2, 3, 8),\\
\rho &= -\sum_{a\neq x_4} K_a + mS = \bar\Phi\Big(m - i\sum_a k_a\gamma^{(a)}\Big)\Phi,\\
mu &= -iE\gamma^{(4)}u + i\sum_a k_a\gamma^{(a)}u \quad\Longrightarrow\quad \Big(m - i\sum_a k_a\gamma^{(a)}\Big)\Phi = -iE\gamma^{(4)}\Phi,\\
\rho &= \bar\Phi\big(-iE\gamma^{(4)}\big)\Phi = E\,\Phi^\dagger\big(-iC\gamma^{(4)}\big)\Phi = E\,\Phi^\dagger B\Phi = E\,u^\dagger Bu.
\end{aligned}
$$

The first line inserts the derivatives into K_a (the two terms are equal); the second inserts K_a into ρ (Section 9.7, with $U = 0$; the extra-time terms are zero); the third is the flat field equation $\sum_a \gamma^{(a)}\partial_a\Phi = m\Phi$ for the plane wave, rearranged; the fourth inserts it, writes $\bar\Phi = \Phi^\dagger C$, uses the matrix $B = -iC\gamma^{(4)}$ of Chapter 5, and the fact that the exponential factor has size 1. B is Hermitian with $B^2 = 1$, with eight eigenvalues $+1$ and eight -1. The field equation multiplied by $-i\gamma^{(4)}$ reads $Eu = hu$ with the matrix $h = m\beta + \sum_a k_a\alpha^a$, $\beta = -i\gamma^{(4)}$, $\alpha^a = -\gamma^{(4)}\gamma^{(a)}$; for $m = 2$ and $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $h^2 = (4+16+1+4+0)\cdot 1 = 25$, so $E = \pm 5$. The record proves that h commutes with B and that on the 8-dimensional space of solutions with $E = 5$ the form $u^\dagger Bu$ has four positive and four negative directions (**Krein inertia** (4,4)): there are unit columns u with $Bu = -u$, whose waves have $\rho = -5$, and with $Bu = +u$, whose waves have $\rho = +5$ (PROVED; the table lists the checks). Multiplying such a wave by a large constant makes ρ as negative as we like: the classical energy of dirac16complex00 has no lower bound, already for $U = 0$.

statement	**Revision report**	**check**
h commutes with B; waves with $\rho = -5$ and $\rho = +5$ at $E = 5$	`Revision/theory/reports/python-scope.json` and `wolfram-scope.json`	`commuting_field_energy_unbounded_below`
every space of solutions of one real frequency has Krein inertia (4,4)	`Revision/pairing/reports/python-pairing.json`	`Q.one_particle_Krein_inertia_proof`
C has eight eigenvalues $+1$ and eight -1	`Revision/algebra/reports/python-algebra.json` and `wolfram-algebra.json`	`C_equals_notebook_sigma16`, `sigma8_involution`

This is the classical form of the **spin-statistics** problem of a commuting field with a first-order Lagrangian. The **spin-statistics rule** of ordinary four-dimensional quantum field theory says that particles of half-integer **spin** (the spin is the intrinsic angular momentum of a particle, measured in units of Planck's constant divided by 2π; the electron has spin $\frac{1}{2}$), which are the fermions of Chapter 7, must be described by anticommuting fields; described by commuting fields, as dirac16complex00 is, the energy of a spinor field has no lower bound, and this is exactly what the plane waves above show. (The rule is a theorem of ordinary four-dimensional physics; this book does not prove it, and for 4+4 dimensions it claims only the statements of this section.) It must be remembered in every reading of the author's Hypothesis00 (for example a phantom equation of state), and it is why the anticommuting, quantised field dirac16complex is treated separately (Chapter 10).

In Part V the chirality map Γ sends every solution with parameters (m, λ) to a solution with $(-m, -\lambda)$ and reverses the sign of the energy-momentum tensor (theorem T1; PROVED, `Revision/pairing/reports/python-pairing.json`, checks `T1.metric.commuting.euler_lagrange_map` and `T1.metric.commuting.emt`; Chapter 18 gives the proof). That theorem is an exact map between sets of solutions: it does not prove that any universe is created, in pairs or otherwise, and it gives no creation process, rate or amplitude.

9.22 Example: condensates

The second worked example builds the exact condensates of Section 9.18 and measures their energy-momentum tensor. It first shows with 20000 random columns that $S_0 = \chi^\dagger C \chi$ takes both signs; checks $M^2 = k^2$ and $M^T C + CM = 0$ for 18 pairs (V, H); builds one oscillating ($\lambda = 0.5$) and one growing ($\lambda = -0.5$) condensate with $m = 1$, $H = 0.25$ and $S_0 = 1$ and checks the field equation and the constancy of S at 401 times; computes from the solutions K_4, ρ, p and B_{04} and checks the values of Section 9.19; measures $w = p/\rho$ on twelve exact solutions and compares it with $x/(2 + x)$; draws the map of Section 9.20; confirms that a condensate keeps its energy density along the deflating history and, as an ILLUSTRATION only, solves the energy exchange of toy fluids with the Runge-Kutta method; and reproduces the negative energies of Section 9.21. It draws seven figures.

9.23 How to run Notebook 09b

Step 1. What this notebook does and what it needs.

Notebook 09b (Homogeneous condensates: energy, pressure, equation of state and energy exchange) is the file `Revision/textbook/notebooks/09b_condensates.ipynb` of the repository Dirac_claude. It builds the exact homogeneous solutions (condensates) of the commuting field dirac16complex00 with the potential U = (lambda/2) S^2 in the author's metric from the Revision gamma matrices, checks that S stays constant and that the energy density and pressure are rho = m S + U and p = S U' - U in every direction, measures the equation of state w = p/rho of actual solutions against lambda S/m, maps where condensates oscillate or grow, shows with assumed toy fluids how energy is exchanged between 3-space and the extra times when the two pressures differ (and that a condensate exchanges none), and shows that the energy of the commuting field has no lower bound, in seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its

first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook,

skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 09b_condensates.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 09b_condensates.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 09b_condensates.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/09b.captions.json`
- `Revision/textbook/figures/09b_1_sign_of_s.png`
- `Revision/textbook/figures/09b_2_condensate_solutions.png`
- `Revision/textbook/figures/09b_3_energy_and_pressure.png`

- `Revision/textbook/figures/09b_4_equation_of_state.png`
- `Revision/textbook/figures/09b_5_regime_map.png`
- `Revision/textbook/figures/09b_6_energy_exchange.png`
- `Revision/textbook/figures/09b_7_energy_sign.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the seven figures of this notebook are saved and captioned
ALL 33 CHECKS PASSED (notebook 09b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/09b_condensates.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 09b_condensates.ipynb
python -m nbconvert --execute --inplace 09b_condensates.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "KeyError" with the words "has no check": the notebook asks a Revision report for the verdict of one of its checks, and the report in your copy of the repository does not contain that check: your copy is older or newer than the notebook. Run `git pull` in the repository folder, then run the notebook again.

9.24 Notebook 09b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/09b_condensates.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A *condensate* is a configuration of the field that is the same at every place:
it depends on the time $x_4$ only. This notebook builds the exact condensates of
the commuting field dirac16complex00 with the potential
$U(S) = \frac{\lambda}{2}S^2$ in the author's metric and studies their
energy-momentum tensor. It

- shows that the number $S = \bar\Phi\Phi$ can have either sign (a histogram);
- builds the exact solution
  $\Phi(x_4) = (\cosh(kx_4) + \frac{\sinh(kx_4)}{k}M)\chi$ of the Revision
  record, checks the field equation and that $S$ stays constant, for one
```

```
  condensate that oscillates and one that grows;
- computes from the solution the energy density $\rho$ and the pressure $p$ and
  checks the record's values $\rho = mS + U$ and $p = SU' - U$, the same in all
  seven directions, and the vanishing of the $x_4$-$x_0$ component;
- measures the equation of state $w = p/\rho$ of actual solutions and compares it
  with $w = \lambda S/(2m + \lambda S)$; finds where $w = -1$;
- maps where condensates oscillate and where they grow;
- shows with toy fluids (an ILLUSTRATION, not solutions of the field equations)
  how energy flows between 3-space and the extra times when $p_3 \neq p_t$, and
  that a condensate, with $p_3 = p_t$, exchanges none;
- shows that the energy of the commuting field has no lower bound;
- draws seven teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 09b (Homogeneous condensates: energy, pressure, equation of state and energy
exchange) is the file ‘Revision/textbook/notebooks/09b_condensates.ipynb‘ of the
repository Dirac_claude. It builds the exact homogeneous solutions (condensates) of the
commuting field dirac16complex00 with the potential U = (lambda/2) S^2 in the author's
metric from the Revision gamma matrices, checks that S stays constant and that the energy
density and pressure are rho = m S + U and p = S U' - U in every direction, measures the
equation of state w = p/rho of actual solutions against lambda S/m, maps where
condensates oscillate or grow, shows with assumed toy fluids how energy is exchanged
between 3-space and the extra times when the two pressures differ (and that a condensate
exchanges none), and shows that the energy of the commuting field has no lower bound, in
seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):
```

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 09b_condensates.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 09b_condensates.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 09b_condensates.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/09b.captions.json‘
- ‘Revision/textbook/figures/09b_1_sign_of_s.png‘
- ‘Revision/textbook/figures/09b_2_condensate_solutions.png‘
- ‘Revision/textbook/figures/09b_3_energy_and_pressure.png‘
- ‘Revision/textbook/figures/09b_4_equation_of_state.png‘
- ‘Revision/textbook/figures/09b_5_regime_map.png‘
- ‘Revision/textbook/figures/09b_6_energy_exchange.png‘
- ‘Revision/textbook/figures/09b_7_energy_sign.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the seven figures of this notebook are saved and captioned
ALL 33 CHECKS PASSED (notebook 09b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/09b_condensates.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

  ```
  python -m jupyterlab 09b_condensates.ipynb
  python -m nbconvert --execute --inplace 09b_condensates.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "KeyError" with the words "has no check": the notebook asks a Revision report for the verdict of one of its checks, and the report in your copy of the repository does not contain that check: your copy is older or newer than the notebook. Run `git pull` in the repository folder, then run the notebook again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 09b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 09b (Homogeneous condensates: energy, pressure, equation of state and energy
# exchange) is the file Revision/textbook/notebooks/09b_condensates.ipynb of the
# repository Dirac_claude. It builds the exact homogeneous solutions (condensates) of the
# commuting field dirac16complex00 with the potential U = (lambda/2) S^2 in the author's
# metric from the Revision gamma matrices, checks that S stays constant and that the
# energy density and pressure are rho = m S + U and p = S U' - U in every direction,
# measures the equation of state w = p/rho of actual solutions against lambda S/m, maps
# where condensates oscillate or grow, shows with assumed toy fluids how energy is
# exchanged between 3-space and the extra times when the two pressures differ (and that a
# condensate exchanges none), and shows that the energy of the commuting field has no
# lower bound, in seven teaching plots. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
```

```
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
```

```
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 09b_condensates.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
```

```
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 09b_condensates.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 09b_condensates.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/09b.captions.json
# - Revision/textbook/figures/09b_1_sign_of_s.png
# - Revision/textbook/figures/09b_2_condensate_solutions.png
# - Revision/textbook/figures/09b_3_energy_and_pressure.png
# - Revision/textbook/figures/09b_4_equation_of_state.png
# - Revision/textbook/figures/09b_5_regime_map.png
# - Revision/textbook/figures/09b_6_energy_exchange.png
# - Revision/textbook/figures/09b_7_energy_sign.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the seven figures of this notebook are saved and captioned
#     ALL 33 CHECKS PASSED (notebook 09b)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
```

```
#   Revision/textbook/notebooks/09b_condensates.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 09b_condensates.ipynb
#     python -m nbconvert --execute --inplace 09b_condensates.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "KeyError" with the words "has no check": the notebook asks a Revision report for the
#   verdict of one of its checks, and the report in your copy of the repository does not
#   contain that check: your copy is older or newer than the notebook. Run git pull in
#   the repository folder, then run the notebook again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "09b"  # this notebook: chapter 09, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
```

```
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
```

```
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 09b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook
```

- **Condensate (homogeneous configuration)**: a field that depends only on the time x_4, not on $x_1, x_2, x_3, x_5, x_6, x_7, x_0$.
- **Effective mass** $V = m + U'(S) = m + \lambda S$: the number that multiplies Φ on the right side of the field equation.
- **Matrix exponential**: for a square matrix M the matrix $e^{Mx_4} = 1 + Mx_4 + \frac{1}{2}M^2x_4^2 + \dots$; the column $e^{Mx_4}\chi$ solves $\frac{d}{dx_4}\Phi = M\Phi$ with $\Phi(0) = \chi$. When $M^2 = k^2$ times the unit matrix, the series sums to $\cosh(kx_4) + \frac{\sinh(kx_4)}{k}M$.
- **Oscillating / growing**: when $k^2 < 0$ the number k is imaginary, $\cosh(kx_4)$ becomes a cosine and the solution oscillates; when $k^2 > 0$ it contains e^{kx_4} and grows.
- **Energy density** ρ, **pressure** p, **equation of state** $w = p/\rho$. Dust has $w = 0$, radiation $w = 1/3$, a cosmological constant $w = -1$; a value $w < -1$ is called *phantom*.
- **Energy exchange**: the change of the energy density in time caused by work done by the inflating 3-space and the deflating extra times.
- **Toy fluid (ILLUSTRATION)**: a source with assumed pressures $p_3 = w_3\rho$ and $p_t = w_t\rho$, used only to show what the conservation identity does; it is not a solution of the field equations of this book.
- **Runge-Kutta method (RK4)**: a step-by-step method that solves a differential equation numerically with four slope evaluations per step.
- **Histogram**: a bar chart that counts how many numbers fall into each of a row of equal intervals.
- **Krein (indefinite) form**: the form $\Phi^\dagger B\Phi$ built with the matrix B, which has eight eigenvalues $+1$ and eight eigenvalues -1; it can be positive or negative.

Text cell:

```
## 4. The physical and mathematical situation

**The field equation of a condensate.** In the author's metric the field
equation of dirac16complex00 is
$\sum_a\frac{1}{f_a}\gamma^{(a)}\partial_a\Phi + 3H\gamma^{(0)}\Phi = V\Phi$ with
$V = m + \lambda S$. For a condensate every derivative except $\partial_4$ is
zero, and $f_4 = 1$, so the equation is
$\gamma^{(4)}\partial_4\Phi + 3H\gamma^{(0)}\Phi = V\Phi$. Multiplying by
$-\gamma^{(4)}$ and using $(\gamma^{(4)})^2 = -1$ gives
$\partial_4\Phi = M\Phi$ with $M = -V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}$.
Neither $a_4$ nor $x_0$ appears: the same condensate solves the equation on
every history $a_4(x_4)$, the deflating one included.

**The exact solution** (Revision record, `Revision/theory/field-theory.json`,
formula `exact_solutions`): $M^2 = k^2$ with $k^2 = 9H^2 - V^2$, and
$M^TC + CM = 0$, so $S = \Phi^\dagger C\Phi$ does not change with $x_4$; then $V$
is a constant and $\Phi = (\cosh(kx_4) + \frac{\sinh(kx_4)}{k}M)\chi$
with $S = S_0 = \chi^\dagger C\chi$.

**Its energy-momentum tensor** (formula `EMT_homogeneous_on_shell`): only $K_4$
is nonzero, $K_4 = VS$ on shell, so $\rho = mS + U$ and
$p_3 = p_t = p_0 = SU' - U$; for $U = \frac{\lambda}{2}S^2$:
$\rho = mS + \frac{\lambda}{2}S^2$, $p = \frac{\lambda}{2}S^2$ and
```

```
$w = \lambda S/(2m + \lambda S)$.

**Energy exchange** (formula `energy_exchange`): conservation gives
$d\rho/dx_4 = -3a_4'(p_3 - p_t)$. A condensate has $p_3 = p_t$, so its energy
density is constant along every history.

**What is not computed here.** The equation of state that an observer living in
3-space would infer (after integrating over the hidden direction and the extra
times), its time dependence and its comparison with supernova fit values are not
computed in this notebook. The Revision record computes them in
`Revision/dark_sector/` under an ASSUMED normalisation of the observer's energy
density (definitions A, B and C; `Revision/docs/DARK_SECTOR_HYPOTHESES.md`,
taught in Chapter 22); which normalisation, if any, describes a physical
observer, and a fit to supernova data, are OPEN. The $w$ of this notebook is the
ratio $p/\rho$ of the 8-dimensional tensor. The condensate's off-diagonal entries,
which are not zero in general, are not used here.
```

Text cell:

```
## 5. The matrices C and B, and the sign of S

The next cell reads the Revision fixture (the gammas $\gamma^{(x_0)}, \dots,
\gamma^{(x_7)}$, the matrix $C$ and the matrix $B = -iC\gamma^{(4)}$) and defines
the helper `recorded(path, name)`, which returns the verdict of a check in a
Revision report. The record says that $C = \mathrm{diag}(-\sigma, \sigma)$ with
an $8 \times 8$ matrix $\sigma$ that squares to 1 and has trace 0; so $C$ has
eight eigenvalues $+1$ and eight $-1$, and $S_0 = \chi^\dagger C\chi$ can be
positive or negative. The cell computes the eigenvalues of $C$ and the values of
$S_0$ for 20000 random columns $\chi$ of length 1.
```

In [2]:

```
import numpy as np  # arrays of numbers, matrices, linear algebra

fixture = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
gamma = [np.array(rows, dtype=float) for rows in fixture["gamma"]]  # x0, ..., x7
C = np.array(fixture["C"], dtype=float)  # gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
B = (np.array(fixture["B"]["re"], dtype=float)  # B = -i C gamma^(x4), stored as its
     + 1j * np.array(fixture["B"]["im"], dtype=float))  # real and imaginary parts
I16 = np.eye(16)  # the 16 x 16 unit matrix
g4, g0 = gamma[4], gamma[0]  # gamma^(x4) and gamma^(x0)


def recorded(path, name):
    """The verdict (PASS or FAIL) of the check name in the Revision report path."""
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for item in report_data["checks"]:
        if item["name"] == name:
            return item["verdict"].upper()  # some reports write "pass"
    raise KeyError(f"{path} has no check {name}")
```

```
ALGEBRA_PY = "Revision/algebra/reports/python-algebra.json"
ALGEBRA_WL = "Revision/algebra/reports/wolfram-algebra.json"
eigenvalues_C = np.linalg.eigvalsh(C)  # C is symmetric: eigvalsh, sorted ascending
check(np.allclose(eigenvalues_C, [-1.0] * 8 + [1.0] * 8, atol=1e-12)
      and recorded(ALGEBRA_PY, "C_equals_notebook_sigma16") == "PASS"
      and recorded(ALGEBRA_WL, "sigma8_involution") == "PASS",
      "C has eight eigenvalues -1 and eight eigenvalues +1",
      record=f"{ALGEBRA_PY}, check C_equals_notebook_sigma16; {ALGEBRA_WL}, check "
             "sigma8_involution")
rng = np.random.default_rng(2026)  # a fixed seed: the same numbers in every run
chis = rng.normal(size=(20000, 16)) + 1j * rng.normal(size=(20000, 16))
chis /= np.linalg.norm(chis, axis=1)[:, None]  # every row now has length 1
# S0 = chi^dagger C chi for every row at once (einsum sums over the indices i, j).
S0_values = np.einsum("ni,ij,nj->n", chis.conj(), C, chis).real
negative_share = float(np.mean(S0_values < 0))  # the fraction of negative S0
check(S0_values.min() < 0 < S0_values.max() and np.abs(S0_values).max() <= 1 + 1e-12,
      "S0 = chi^dagger C chi takes both signs and lies between -1 and 1")
report("fraction of the 20000 random chi with S0 < 0", f"{negative_share:.4f}")
```

Out [2]:

```
PASS C has eight eigenvalues -1 and eight eigenvalues +1
     reproduces Revision/algebra/reports/python-algebra.json, check
    C_equals_notebook_sigma16; Revision/algebra/reports/wolfram-algebra.json, check
    sigma8_involution
PASS S0 = chi^dagger C chi takes both signs and lies between -1 and 1
RESULT fraction of the 20000 random chi with S0 < 0 = 0.4983
```

Text cell:

```
The next cell draws the histogram of the 20000 values of $S_0$. A condensate
with $\lambda = 0$ has $\rho = mS_0$, so for $m > 0$ about half of these random
condensates have a negative energy density.
```

In [3]:

```
fig, ax = plt.subplots()
ax.hist(S0_values, bins=60, range=(-1.0, 1.0), color="tab:blue", edgecolor="white")
ax.axvline(0.0, color="black", linewidth=1.0)
ax.set_xlabel("$S_0 = \\chi^\\dagger C \\chi$ for a random $\\chi$ of length 1")
ax.set_ylabel("number of the 20000 random $\\chi$")
ax.set_title("The scalar $S = \\bar\\Phi\\Phi$ has no sign")
save_figure(fig, "sign_of_s",
            "Histogram of $S_0 = \\chi^\\dagger C\\chi$ for 20000 random complex "
            "columns $\\chi$ of length 1 (fixed seed); horizontal axis $S_0$ (a pure "
            "number between $-1$ and $1$, because the eigenvalues of $C$ are $\\pm 1$), "
            "vertical axis the number of $\\chi$ in each of 60 intervals. The values "
            "are spread symmetrically about $0$: about half are negative. For a "
            "condensate with $\\lambda = 0$ the energy density is $\\rho = mS_0$, so "
            "its sign is the sign of $S_0$.")
```

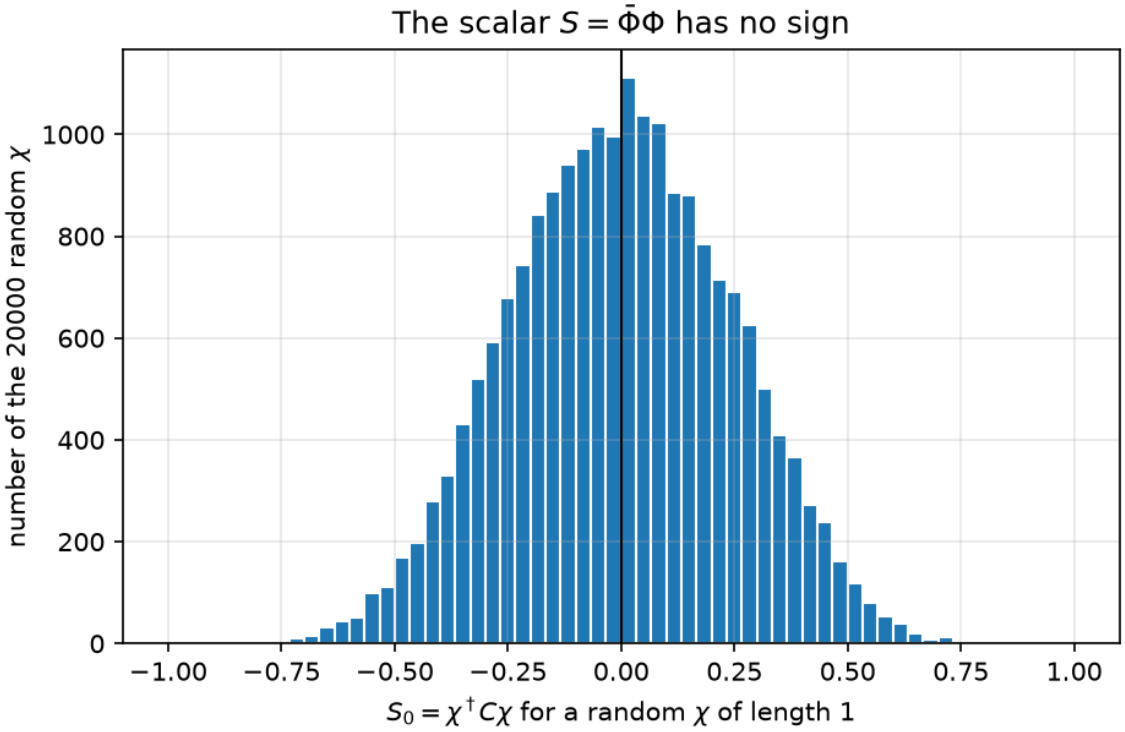

Figure 255: Histogram of $S_0 = \chi^\dagger C\chi$ for 20000 random complex columns χ of length 1 (fixed seed); horizontal axis S_0 (a pure number between -1 and 1, because the eigenvalues of C are ± 1), vertical axis the number of χ in each of 60 intervals. The values are spread symmetrically about 0: about half are negative. For a condensate with $\lambda = 0$ the energy density is $\rho = mS_0$, so its sign is the sign of S_0. (Notebook 09b, figure 1.)

Out [3]:

```
Figure 09b.1 saved as Revision/textbook/figures/09b_1_sign_of_s.png
```

Text cell:

```
## 6. The matrix M and its square

The next cell defines the function ‘condensate_matrix(V, H)‘, which returns
$M = -V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}$ and $k^2 = 9H^2 - V^2$, and
checks for several values of $V$ and $H$ the two facts of the record:
$M^2 = k^2 \cdot 1$ and $M^TC + CM = 0$. The second fact makes $S$ constant:
$\frac{d}{dx_4}(\Phi^\dagger C\Phi) = \Phi^\dagger(M^TC + CM)\Phi = 0$ (here $M$ is
real, so $M^\dagger = M^T$).
```

In [4]:

```
def condensate_matrix(V, H):
    """M = -V gamma^(x4) + 3 H gamma^(x4) gamma^(x0) and k^2 = 9 H^2 - V^2."""
    return -V * g4 + 3 * H * g4 @ g0, 9 * H ** 2 - V ** 2


THEORY_WL = "Revision/theory/reports/wolfram-field-theory.json"
THEORY_PY = "Revision/theory/reports/python-field-theory.json"
worst_square, worst_c = 0.0, 0.0  # the largest deviations found
for V_test in (-2.0, -0.5, 0.0, 0.75, 1.5, 3.0):
    for H_test in (0.1, 0.25, 1.0):
        M_test, k2_test = condensate_matrix(V_test, H_test)
        worst_square = max(worst_square, np.abs(M_test @ M_test - k2_test * I16).max())
        worst_c = max(worst_c, np.abs(M_test.T @ C + C @ M_test).max())
check(worst_square < 1e-12 and worst_c < 1e-12
      and recorded(THEORY_WL, "solution_matrix_square") == "PASS",
      "M^2 = (9 H^2 - V^2) times 1 and M^T C + C M = 0 for 18 pairs (V, H)",
      record=f"{THEORY_WL}, check solution_matrix_square")
```

Out [4]:

```
PASS M^2 = (9 H^2 - V^2) times 1 and M^T C + C M = 0 for 18 pairs (V, H)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    solution_matrix_square
```

Text cell:

```
## 7. Two exact condensates: one oscillates, one grows

The next cell fixes $m = 1$ and $H = 0.25$ and a column $\chi$ with
$S_0 = \chi^\dagger C\chi = 1$ (it draws random columns with a fixed seed until
$S_0 > 0$ and divides by $\sqrt{S_0}$). With $\lambda = 0.5$ the effective mass is
$V = 1.5 > 3H = 0.75$, so $k^2 = 9H^2 - V^2 < 0$: the condensate oscillates.
```

```
With $\lambda = -0.5$ it is $V = 0.5 < 0.75$, so $k^2 > 0$: the condensate grows.
The cell also defines the function ‘condensate(x4, chi, M, k2)‘, which returns
$\Phi(x_4)$ and its derivative
$\Phi'(x_4) = k\sinh(kx_4)\chi + \cosh(kx_4)M\chi$; numpy computes $\cosh$ and
$\sinh$ of the imaginary $k$ correctly when $k$ is stored as a complex number.
It prints the value of $S_0$ after the division, which must be 1.
```

In [5]:

```
m, H = 1.0, 0.25  # the mass and the author's constant
rng = np.random.default_rng(7)
while True:  # draw until S0 > 0 (the first draw usually succeeds)
    chi = rng.normal(size=16) + 1j * rng.normal(size=16)
    S0 = (chi.conj() @ C @ chi).real
    if S0 > 0:
        break
chi = chi / np.sqrt(S0)  # now chi^dagger C chi = 1


def condensate(x4, chi_value, M, k2):
    """Phi(x4) and dPhi/dx4 of the exact condensate with Phi(0) = chi_value."""
    if k2 == 0.0:  # cosh(k x4) -> 1 and sinh(k x4)/k -> x4 when k -> 0
        return chi_value + x4 * M @ chi_value, M @ chi_value
    k = np.sqrt(complex(k2))  # k is imaginary when k2 < 0
    phi = np.cosh(k * x4) * chi_value + np.sinh(k * x4) / k * (M @ chi_value)
    dphi = k * np.sinh(k * x4) * chi_value + np.cosh(k * x4) * (M @ chi_value)
    return phi, dphi


S0_now = (chi.conj() @ C @ chi).real  # 1 up to rounding
report("S0 = chi^dagger C chi after the division", f"{S0_now:.6f}")
```

Out [5]:

```
RESULT S0 = chi^dagger C chi after the division = 1.000000
```

Text cell:

```
The next cell builds the two condensates at 401 times $x_4$ from $0$ to $12$ and
checks at every time the field equation
$\gamma^{(4)}\Phi' + 3H\gamma^{(0)}\Phi = (m + \lambda S)\Phi$ and that $S$ stays
equal to 1. It keeps the values of $\Phi$ and $S$ for the figure that follows.
```

In [6]:

```
times = np.linspace(0.0, 12.0, 401)  # the times x4 at which the solution is tested
EXAMPLES = {"oscillating": 0.5, "growing": -0.5}  # name -> lambda
solutions = {}  # name -> (Phi at every time, S at every time)
for name, lam in EXAMPLES.items():
```

```
    V = m + lam * 1.0  # S0 = 1
    M, k2 = condensate_matrix(V, H)
    phis, S_values, worst_residual = [], [], 0.0
    for x4 in times:
        phi, dphi = condensate(x4, chi, M, k2)
        S_now = (phi.conj() @ C @ phi).real
        residual = g4 @ dphi + 3 * H * g0 @ phi - (m + lam * S_now) * phi
        # The residual relative to the size of the solution at this time.  Rounding
        # errors grow with the solution (S is a difference of large numbers when
        # Phi grows), so the tolerance below is 1e-9, not 1e-15.
        worst_residual = max(worst_residual,
                             np.abs(residual).max() / max(1.0, np.abs(phi).max()))
        phis.append(phi)
        S_values.append(S_now)
    solutions[name] = (np.array(phis), np.array(S_values))
    report(f"{name}: lambda = {lam}, V = m + lambda S0", f"{V:.4f}")
    report(f"{name}: k^2 = 9 H^2 - V^2", f"{k2:.4f}")
    check(worst_residual < 1e-9,
          f"{name}: the field equation holds at 401 times")
    check(np.abs(np.array(S_values) - 1.0).max() < 1e-9,
          f"{name}: S stays equal to S0 = 1 at 401 times")
check(recorded(THEORY_WL, "exact_solution_nonlinear_homogeneous_C") == "PASS"
      and recorded(THEORY_PY, "exact_nonlinear_homogeneous_solution") == "PASS",
      "both condensates are the record's exact nonlinear homogeneous solution",
      record=f"{THEORY_WL}, check exact_solution_nonlinear_homogeneous_C; "
             f"{THEORY_PY}, check exact_nonlinear_homogeneous_solution")
```

Out [6]:

```
RESULT oscillating: lambda = 0.5, V = m + lambda S0 = 1.5000
RESULT oscillating: k^2 = 9 H^2 - V^2 = -1.6875
PASS oscillating: the field equation holds at 401 times
PASS oscillating: S stays equal to S0 = 1 at 401 times
RESULT growing: lambda = -0.5, V = m + lambda S0 = 0.5000
RESULT growing: k^2 = 9 H^2 - V^2 = 0.3125
PASS growing: the field equation holds at 401 times
PASS growing: S stays equal to S0 = 1 at 401 times
PASS both condensates are the record's exact nonlinear homogeneous solution
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    exact_solution_nonlinear_homogeneous_C; Revision/theory/reports/python-field-
    theory.json, check exact_nonlinear_homogeneous_solution
```

Text cell:

```
The next cell draws the two condensates. Left: the real parts of three of the 16
components of the oscillating condensate, and $S$. Right: the size
$\Phi^\dagger\Phi$ of the growing condensate (logarithmic axis) and $S$. In both
cases $S$ stays exactly 1, even where $\Phi^\dagger\Phi$ grows by a factor of
more than 100000: the growth happens in directions in which the indefinite form
$\Phi^\dagger C\Phi$ does not grow.
```

In [7]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
phis, S_values = solutions["oscillating"]
for component in (0, 5, 10):  # three of the 16 components (rows 1, 6 and 11)
    left.plot(times, phis[:, component].real,
              label=f"Re $\\Phi_{{{component + 1}}}$")
left.plot(times, S_values, "k--", label="$S = \\bar\\Phi\\Phi$")
left.set_xlabel("time $x_4$")
left.set_ylabel("value")
left.set_ylim(-0.8, 1.35)  # room for the legend above the curves
left.set_title("Oscillating: $\\lambda = 0.5$, $V = 1.5$")
left.legend(fontsize=7, loc="upper center", ncol=4)
phis, S_values = solutions["growing"]
right.plot(times, np.einsum("ti,ti->t", phis.conj(), phis).real,
           label="$\\Phi^\\dagger\\Phi$")
right.plot(times, S_values, "k--", label="$S = \\bar\\Phi\\Phi$")
right.set_yscale("log")
right.set_xlabel("time $x_4$")
right.set_title("Growing: $\\lambda = -0.5$, $V = 0.5$")
right.legend(fontsize=8)
save_figure(fig, "condensate_solutions",
            "Two exact condensates of dirac16complex00 with $m = 1$, $H = 0.25$ and "
            "$S_0 = 1$, for $x_4$ from $0$ to $12$. Left, $\\lambda = 0.5$, so the "
            "effective mass $V = 1.5$ exceeds $3H = 0.75$: the real parts of the "
            "components 1, 6 and 11 of $\\Phi$ oscillate and $S$ (dashed) stays $1$. "
            "Right, $\\lambda = -0.5$, so $V = 0.5 < 3H$: the size "
            "$\\Phi^\\dagger\\Phi$ grows like $e^{2kx_4}$ with $k = 0.559$ "
            "(logarithmic axis) while $S$ (dashed) stays exactly $1$. All values "
            "are pure numbers.")
```

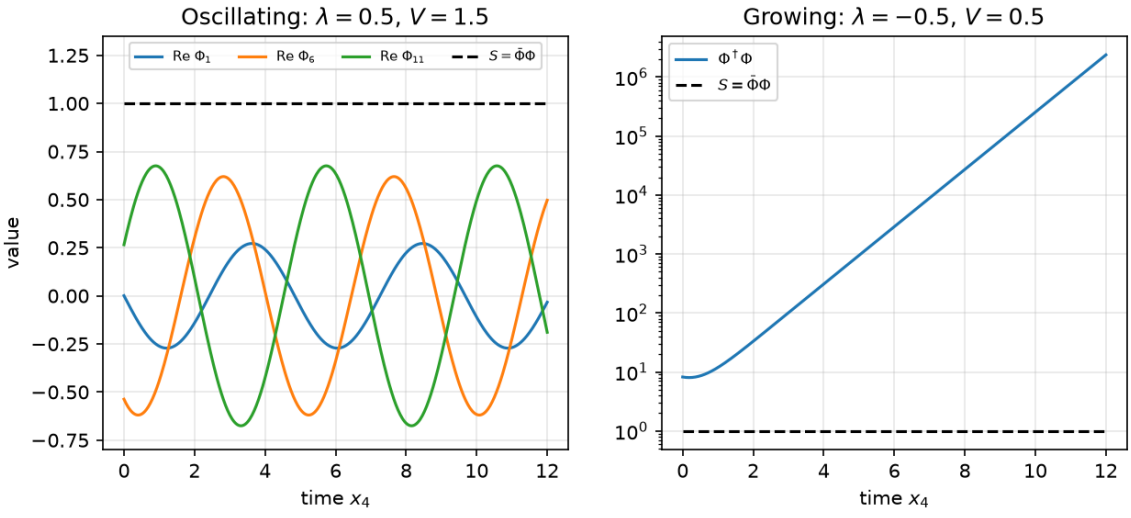

Figure 256: Two exact condensates of dirac16complex00 with $m = 1$, $H = 0.25$ and $S_0 = 1$, for x_4 from 0 to 12. Left, $\lambda = 0.5$, so the effective mass $V = 1.5$ exceeds $3H = 0.75$: the real parts of the components 1, 6 and 11 of Φ oscillate and S (dashed) stays 1. Right, $\lambda = -0.5$, so $V = 0.5 < 3H$: the size $\Phi^\dagger\Phi$ grows like e^{2kx_4} with $k = 0.559$ (logarithmic axis) while S (dashed) stays exactly 1. All values are pure numbers. (Notebook 09b, figure 2.)

Out [7]:

```
Figure 09b.2 saved as Revision/textbook/figures/09b_2_condensate_solutions.png
```

Text cell:

```
## 8. Energy density, pressures, kinetic and potential parts of the condensates

For a condensate every kinetic term except $K_4$ vanishes, because every other
derivative is zero. The next cell computes, at every tested time,
$K_4 = \frac12(\bar\Phi\gamma^{(4)}\Phi' - \bar\Phi'\gamma^{(4)}\Phi)$ from the
solution, the Lagrangian $L_0 = K_4 - mS - U$, the energy density
$\rho = -T^{x_4}{}_{x_4} = K_4 - L_0$ and the pressures
$p_\mu = T^\mu{}_\mu = L_0 - K_\mu = L_0$ for $\mu \neq x_4$, which are equal in
all seven directions. It checks $K_4 = VS$, $\rho = mS + \frac{\lambda}{2}S^2$ and
$p = \frac{\lambda}{2}S^2$, hence the kinetic and potential parts
($\rho_{\rm kin} = 0$, $p_{\rm kin} = K_4 = VS$, $p_{\rm pot} = -(mS + U)$), the trace
$-\rho + 7p = -mS + 3\lambda S^2$, and that the $x_4$-$x_0$ component vanishes:
$T^{x_4}{}_{x_0} = -\frac14(B_{40} - \cot z B_{04})$ with
$B_{40} = 0$ (no $x_0$ derivative) and
$B_{04} = \bar\Phi\gamma^{(0)}\Phi' - \bar\Phi'\gamma^{(0)}\Phi$, which must be 0.
```

In [8]:

```
def homogeneous_tensor(phi, dphi, lam):
    """K4, rho, p and B04 of a condensate at one time."""
    S = (phi.conj() @ C @ phi).real
    U = lam / 2 * S ** 2
    K4 = 0.5 * (phi.conj() @ C @ g4 @ dphi - dphi.conj() @ C @ g4 @ phi)
    L0 = K4.real - m * S - U  # the other seven kinetic terms are zero
    B04 = phi.conj() @ C @ g0 @ dphi - dphi.conj() @ C @ g0 @ phi
    return S, U, K4, K4.real - L0, L0, B04  # rho = K4 - L0, p = L0


values = {}  # name -> (rho, p) of the condensate
for name, lam in EXAMPLES.items():
    V = m + lam * 1.0
    M, k2 = condensate_matrix(V, H)
    rhos, ps, worst = [], [], 0.0
    for x4 in times:
        phi, dphi = condensate(x4, chi, M, k2)
        S, U, K4, rho, p, B04 = homogeneous_tensor(phi, dphi, lam)
        size = max(1.0, (phi.conj() @ phi).real)  # for relative tolerances
        worst = max(worst, abs(K4.imag) / size, abs(K4.real - V * S) / size,
                    abs(B04) / size)
        rhos.append(rho)
        ps.append(p)
    rhos, ps = np.array(rhos), np.array(ps)
    values[name] = (rhos.mean(), ps.mean())
    check(worst < 1e-12, f"{name}: K4 is real, K4 = V S, and B04 = 0 (T^x4_x0 = 0)")
    check(np.abs(rhos - (m + lam / 2)).max() < 1e-9
```

```
          and np.abs(ps - lam / 2).max() < 1e-9,
          f"{name}: rho = m S + lambda S^2/2 and p = lambda S^2/2 at 401 times")
    check(abs(-values[name][0] + 7 * values[name][1] - (-m + 3 * lam)) < 1e-9,
          f"{name}: trace -rho + 7 p = -m S + 3 lambda S^2")
    report(f"{name}: rho", f"{values[name][0]:.6f}")
    report(f"{name}: p (all seven directions)", f"{values[name][1]:.6f}")
    report(f"{name}: w = p/rho", f"{values[name][1] / values[name][0]:.6f}")
check(recorded(THEORY_PY, "commuting_homogeneous_on_shell_rho_p") == "PASS"
      and recorded(THEORY_PY, "commuting_T_x4x0_homogeneous") == "PASS"
      and recorded(THEORY_PY, "commuting_trace_on_shell") == "PASS",
      "these are the record's homogeneous values and its vanishing x4-x0 component",
      record=f"{THEORY_PY}, checks commuting_homogeneous_on_shell_rho_p, "
             "commuting_T_x4x0_homogeneous and commuting_trace_on_shell")
```

Out [8]:

```
PASS oscillating: K4 is real, K4 = V S, and B04 = 0 (T^x4_x0 = 0)
PASS oscillating: rho = m S + lambda S^2/2 and p = lambda S^2/2 at 401 times
PASS oscillating: trace -rho + 7 p = -m S + 3 lambda S^2
RESULT oscillating: rho = 1.250000
RESULT oscillating: p (all seven directions) = 0.250000
RESULT oscillating: w = p/rho = 0.200000
PASS growing: K4 is real, K4 = V S, and B04 = 0 (T^x4_x0 = 0)
PASS growing: rho = m S + lambda S^2/2 and p = lambda S^2/2 at 401 times
PASS growing: trace -rho + 7 p = -m S + 3 lambda S^2
RESULT growing: rho = 0.750000
RESULT growing: p (all seven directions) = -0.250000
RESULT growing: w = p/rho = -0.333333
PASS these are the record's homogeneous values and its vanishing x4-x0 component
     reproduces Revision/theory/reports/python-field-theory.json, checks
    commuting_homogeneous_on_shell_rho_p, commuting_T_x4x0_homogeneous and
    commuting_trace_on_shell
```

Text cell:

```
## 9. Energy density and pressure against S

The next cell draws, for $m = 1$ and $\lambda = 0.5$, the energy density
$\rho = mS + \frac{\lambda}{2}S^2$ and the pressure $p = \frac{\lambda}{2}S^2$ of
a condensate against $S$, with the two parts of the pressure: the kinetic part
$p_{\rm kin} = VS = (m + \lambda S)S$ and the potential part
$p_{\rm pot} = -(mS + U)$. The energy density is zero at $S = 0$ and at
$S = -2m/\lambda = -4$ and negative between them. The cell also checks that the
two parts add up to $p$ at every drawn $S$.
```

In [9]:

```
lam = 0.5
S_axis = np.linspace(-6.0, 2.0, 401)
rho_axis = m * S_axis + lam / 2 * S_axis ** 2  # rho = m S + U
p_axis = lam / 2 * S_axis ** 2  # p = S U' - U
```

```
p_kin = (m + lam * S_axis) * S_axis  # V S
p_pot = -(m * S_axis + lam / 2 * S_axis ** 2)  # -(m S + U)
check(np.abs(p_kin + p_pot - p_axis).max() < 1e-12,
      "p = p_kin + p_pot = V S - (m S + U) at every drawn S")
fig, ax = plt.subplots()
ax.plot(S_axis, rho_axis, label="energy density $\\rho = mS + \\lambda S^2/2$")
ax.plot(S_axis, p_axis, label="pressure $p = \\lambda S^2/2$")
ax.plot(S_axis, p_kin, "--", label="kinetic part of $p$: $(m + \\lambda S)S$")
ax.plot(S_axis, p_pot, ":", label="potential part of $p$: $-(mS + U)$")
ax.plot([0.0, -2 * m / lam], [0.0, 0.0], "ko", label="$\\rho = 0$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("$S = \\bar\\Phi\\Phi$")
ax.set_ylabel("energy per unit volume")
ax.set_title("Condensate: $\\rho$ and $p$ against $S$ ($m = 1$, $\\lambda = 0.5$)")
ax.legend(fontsize=8)
save_figure(fig, "energy_and_pressure",
            "Energy density $\\rho = mS + \\lambda S^2/2$ (solid), pressure "
            "$p = \\lambda S^2/2$ (solid) and the two parts of the pressure, the "
            "kinetic part $(m + \\lambda S)S$ (dashed) and the potential part "
            "$-(mS + U)$ (dotted), of a condensate of dirac16complex00 with $m = 1$, "
            "$\\lambda = 0.5$, against $S$ from $-6$ to $2$; vertical axis in units "
            "of energy per unit volume. The energy density vanishes at $S = 0$ and "
            "$S = -2m/\\lambda = -4$ (black dots) and is negative between them; the "
            "pressure is the same in all seven directions.")
```

Out [9]:

```
PASS p = p_kin + p_pot = V S - (m S + U) at every drawn S
```

Out [9]:

```
Figure 09b.3 saved as Revision/textbook/figures/09b_3_energy_and_pressure.png
```

Text cell:

```
## 10. The equation of state of actual solutions

The ratio $w = p/\rho$ depends on $m$, $\lambda$ and $S$ only through
$x = \lambda S/m$: $w = \frac{\lambda S^2/2}{mS + \lambda S^2/2} = \frac{x}{2 + x}$.
The next cell does not use this formula to compute $w$. For twelve values of $x$
it builds the exact condensate with $m = 1$, $S_0 = 1$ and $\lambda = x$, takes
$\Phi$ and $\Phi'$ at the time $x_4 = 1.3$, computes $\rho$ and $p$ from them as
in section 8, and only then compares $p/\rho$ with $x/(2 + x)$. It also finds the
special values: $w = 0$ at $x = 0$ (dust-like), $w = -1$ at $x = -1$, where the
effective mass $V = m + \lambda S$ is zero, $w < -1$ (phantom) for
$-2 < x < -1$, and $w \to 1$ as $x$ grows. At $x = -2$ the energy density is zero
and $w$ is not defined. For $x < -2$ (with $m > 0$ and $S > 0$) the energy
density and the pressure are both negative, and $w > 1$.
```

In [10]:

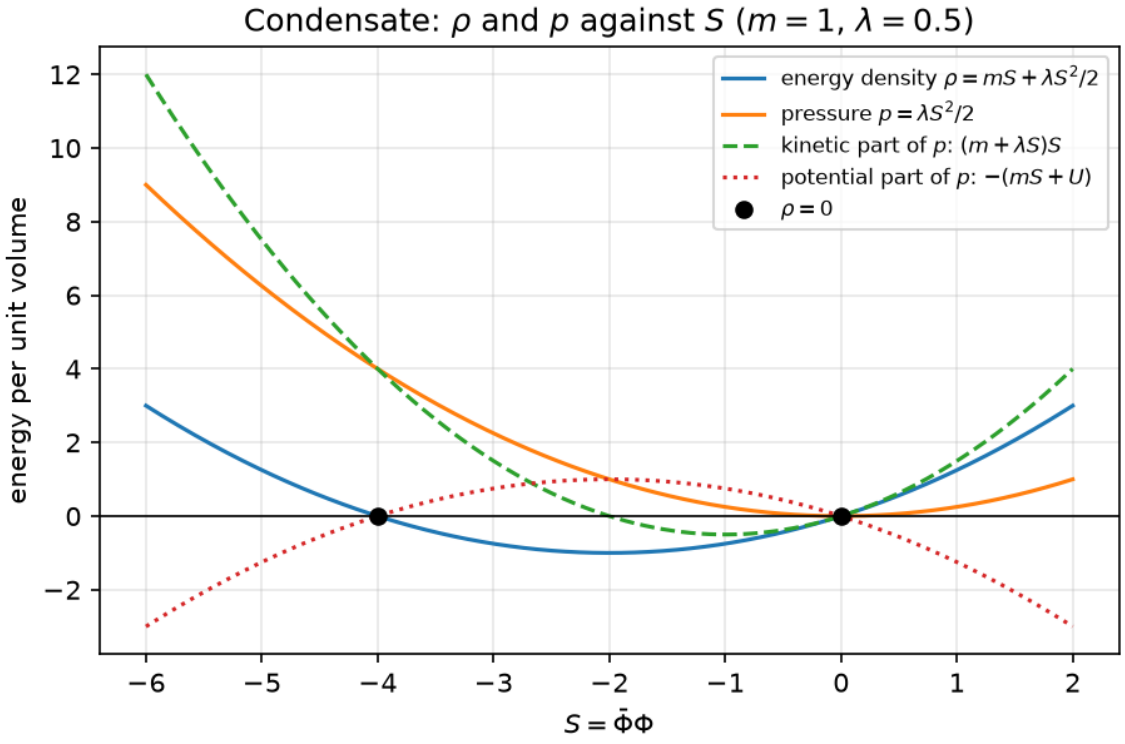

Figure 257: Energy density $\rho = mS + \lambda S^2/2$ (solid), pressure $p = \lambda S^2/2$ (solid) and the two parts of the pressure, the kinetic part $(m + \lambda S)S$ (dashed) and the potential part $-(mS + U)$ (dotted), of a condensate of dirac16complex00 with $m = 1$, $\lambda = 0.5$, against S from -6 to 2; vertical axis in units of energy per unit volume. The energy density vanishes at $S = 0$ and $S = -2m/\lambda = -4$ (black dots) and is negative between them; the pressure is the same in all seven directions. (Notebook 09b, figure 3.)

```
x_measured = np.array([-3.5, -3.0, -2.5, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 2.0, 3.0,
                       4.0])  # values of x = lambda S/m (here lambda = x)
rho_measured, p_measured = [], []  # rho and p of each of the twelve condensates
for x_value in x_measured:
    V = m + x_value * 1.0  # lambda = x_value, S0 = 1
    M, k2 = condensate_matrix(V, H)
    phi, dphi = condensate(1.3, chi, M, k2)
    S, U, K4, rho, p, B04 = homogeneous_tensor(phi, dphi, x_value)
    rho_measured.append(rho)
    p_measured.append(p)
rho_measured, p_measured = np.array(rho_measured), np.array(p_measured)
w_measured = p_measured / rho_measured  # the equation of state of each condensate
w_formula = x_measured / (2 + x_measured)
check(np.abs(w_measured - w_formula).max() < 1e-9,
      "w = p/rho of 12 exact condensates equals x/(2 + x), x = lambda S/m")
at_minus_one = w_measured[list(x_measured).index(-1.0)]
check(abs(at_minus_one + 1.0) < 1e-12,
      "w = -1 exactly where the effective mass m + lambda S vanishes (x = -1)")
below = x_measured < -2  # the three condensates with x < -2
check(np.all(rho_measured[below] < 0) and np.all(p_measured[below] < 0)
      and np.all(w_measured[below] > 1),
      "for x < -2: rho < 0, p < 0 and w > 1")
for x_value, w_value in zip(x_measured, w_measured):
    say(f"x = lambda S/m = {x_value:5.1f}:  w = p/rho = {w_value: .6f}")
```

Out [10]:

```
PASS w = p/rho of 12 exact condensates equals x/(2 + x), x = lambda S/m
PASS w = -1 exactly where the effective mass m + lambda S vanishes (x = -1)
PASS for x < -2: rho < 0, p < 0 and w > 1
x = lambda S/m =  -3.5:  w = p/rho =  2.333333
x = lambda S/m =  -3.0:  w = p/rho =  3.000000
x = lambda S/m =  -2.5:  w = p/rho =  5.000000
x = lambda S/m =  -1.5:  w = p/rho = -3.000000
x = lambda S/m =  -1.0:  w = p/rho = -1.000000
x = lambda S/m =  -0.5:  w = p/rho = -0.333333
x = lambda S/m =   0.0:  w = p/rho =  0.000000
x = lambda S/m =   0.5:  w = p/rho =  0.200000
x = lambda S/m =   1.0:  w = p/rho =  0.333333
x = lambda S/m =   2.0:  w = p/rho =  0.500000
x = lambda S/m =   3.0:  w = p/rho =  0.600000
x = lambda S/m =   4.0:  w = p/rho =  0.666667
```

Text cell:

```
The next cell draws $w = x/(2 + x)$ as a curve, the twelve measured values as
points, and marks the special lines.
```

In [11]:

```
fig, ax = plt.subplots()
for piece in (np.linspace(-6.0, -2.05, 300), np.linspace(-1.95, 6.0, 500)):
```

```
    ax.plot(piece, piece / (2 + piece), color="tab:blue")  # two branches
ax.plot(x_measured, w_measured, "o", color="tab:red",
        label="measured on exact condensates")
ax.axvline(-2.0, color="gray", linestyle=":", label="$\\rho = 0$ ($x = -2$)")
ax.axhline(-1.0, color="tab:green", linestyle="--", label="$w = -1$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.axhspan(-6.0, -1.0, xmin=0.0, xmax=1.0, color="tab:green", alpha=0.08)
ax.set_ylim(-6.0, 6.0)
ax.set_xlabel("$x = \\lambda S / m$")
ax.set_ylabel("$w = p/\\rho$")
ax.set_title("Equation of state of a condensate: $w = x/(2 + x)$")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "equation_of_state",
            "Equation of state $w = p/\\rho$ of a condensate of dirac16complex00 "
            "against $x = \\lambda S/m$ (pure numbers): the curve is "
            "$w = x/(2 + x)$, the red points are $p/\\rho$ computed from twelve exact "
            "solutions ($m = 1$, $H = 0.25$, $S_0 = 1$, $\\lambda = x$) and lie on "
            "it. $w = 0$ at $x = 0$, $w = -1$ (dashed) at $x = -1$, where the "
            "effective mass $m + \\lambda S$ vanishes, $w < -1$ (shaded) for "
            "$-2 < x < -1$, $w$ is undefined at $x = -2$ (dotted) where $\\rho = 0$, "
            "$w > 1$ for $x < -2$, where $\\rho$ and $p$ are both negative, "
            "and $w$ approaches $1$ for large $x$. This is the 8-dimensional "
            "ratio, not the equation of state a 3-space observer would infer.")
```

Out [11]:

```
Figure 09b.4 saved as Revision/textbook/figures/09b_4_equation_of_state.png
```

Text cell:

```
## 11. Where condensates oscillate and where they grow

A condensate oscillates when $k^2 = 9H^2 - V^2 < 0$, that is when
$|1 + x| > 3H/|m|$ with $x = \lambda S/m$, and grows when $|1 + x| < 3H/|m|$
(because $V = m(1 + x)$). The equation of state depends on $x$ alone, so on the
map of the plane $(x, 3H/|m|)$ every line of constant $w$ is vertical. The next
cell draws the map, the lines $w = -1$, $0$, $1/3$ and the two condensates of
section 7, and checks that each of them lies in the region the map gives.
```

In [12]:

```
x_grid = np.linspace(-4.0, 3.0, 701)  # x = lambda S/m
h_grid = np.linspace(0.0, 3.0, 301)  # h = 3 H/|m|
X, Hgrid = np.meshgrid(x_grid, h_grid)  # every pair (x, h) of the two grids
growing_region = (np.abs(1 + X) < Hgrid).astype(float)  # 1 where k^2 > 0
fig, ax = plt.subplots()
ax.contourf(X, Hgrid, growing_region, levels=[-0.5, 0.5, 1.5],
            colors=["#dbe9f6", "#f6d5d5"])
for label, x_line in (("$w = -1$", -1.0), ("$w = 0$", 0.0), ("$w = 1/3$", 1.0)):
    ax.axvline(x_line, color="black", linestyle="--", linewidth=0.8)
    ax.text(x_line + 0.05, 2.8, label, fontsize=8)
```

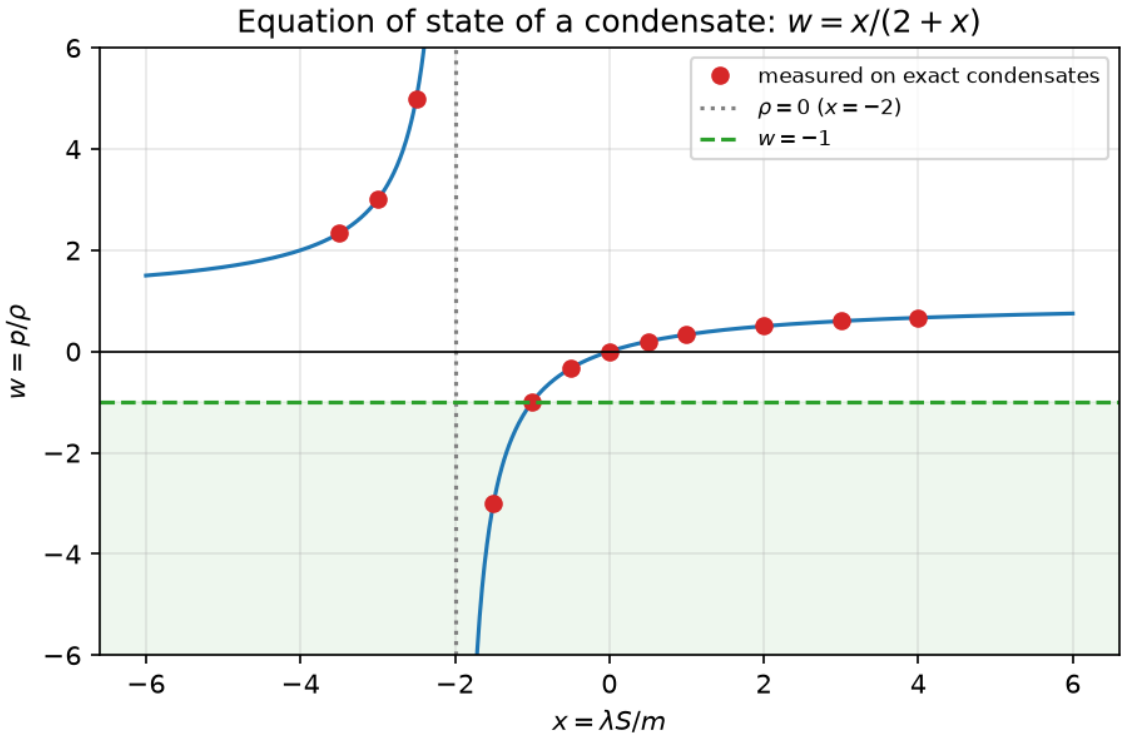

Figure 258: Equation of state $w = p/\rho$ of a condensate of dirac16complex00 against $x = \lambda S/m$ (pure numbers): the curve is $w = x/(2+x)$, the red points are p/ρ computed from twelve exact solutions ($m = 1$, $H = 0.25$, $S_0 = 1$, $\lambda = x$) and lie on it. $w = 0$ at $x = 0$, $w = -1$ (dashed) at $x = -1$, where the effective mass $m + \lambda S$ vanishes, $w < -1$ (shaded) for $-2 < x < -1$, w is undefined at $x = -2$ (dotted) where $\rho = 0$, $w > 1$ for $x < -2$, where ρ and p are both negative, and w approaches 1 for large x. This is the 8-dimensional ratio, not the equation of state a 3-space observer would infer. (Notebook 09b, figure 4.)

```
for name, lam_value in EXAMPLES.items():
    ax.plot([lam_value * 1.0 / m], [3 * H / abs(m)], "ko")
    ax.text(lam_value / m + 0.08, 3 * H / abs(m) - 0.15, name, fontsize=8)
ax.text(-3.8, 0.3, "oscillating ($k^2 < 0$)", fontsize=9)
ax.text(-2.9, 2.3, "growing\n($k^2 > 0$)", fontsize=9)
ax.set_xlabel("$x = \\lambda S / m$")
ax.set_ylabel("$3H/|m|$")
ax.set_title("Condensates: oscillating (blue) or growing (red)")
ax.grid(False)
save_figure(fig, "regime_map",
            "Map of the condensates of dirac16complex00 in the plane of "
            "$x = \\lambda S/m$ (horizontal) and $3H/|m|$ (vertical), pure numbers. "
            "In the red wedge $|1 + x| < 3H/|m|$ the number $k^2 = 9H^2 - "
            "(m + \\lambda S)^2$ is positive and the condensate grows; in the blue "
            "region it oscillates. The dashed vertical lines are the equations of "
            "state $w = -1$ ($x = -1$), $w = 0$ and $w = 1/3$ ($x = 1$); $w$ "
            "depends on $x$ only. The black dots are the two condensates of the "
            "second figure ($H = 0.25$, $m = 1$). Every condensate with $w = -1$ "
            "lies in the growing wedge.")
for name, lam_value in EXAMPLES.items():
    grows = abs(1 + lam_value / m) < 3 * H / abs(m)
    check(grows == (name == "growing"),
          f"the {name} condensate lies in the {name} region of the map")
```

Out [12]:

```
Figure 09b.5 saved as Revision/textbook/figures/09b_5_regime_map.png
PASS the oscillating condensate lies in the oscillating region of the map
PASS the growing condensate lies in the growing region of the map
```

Text cell:

```
## 12. Energy exchange between 3-space and the extra times

Conservation says $d\rho/dx_4 = -3a_4'(p_3 - p_t)$. A condensate has $p_3 = p_t$,
so along every history its energy density stays constant; the next cell confirms
this on the oscillating condensate along the deflating history $a_4 = AHx_4$
($A = 1$): the solution does not depend on $a_4$, and $\rho$ is the same at all
401 times.
```

In [13]:

```
A = 1.0  # the deflating history a4 = A H x4
M, k2 = condensate_matrix(m + 0.5 * 1.0, H)  # the oscillating condensate, S0 = 1
# rho at every time; homogeneous_tensor returns (S, U, K4, rho, p, B04): index 3.
# The condensate does not contain a4, so these are its values on this history too.
rho_at_times = np.array([homogeneous_tensor(*condensate(x4, chi, M, k2), 0.5)[3]
                         for x4 in times])
check(np.abs(rho_at_times - rho_at_times[0]).max() < 1e-9,
      "the condensate has p3 = p_t, so its rho is constant along the history")
```

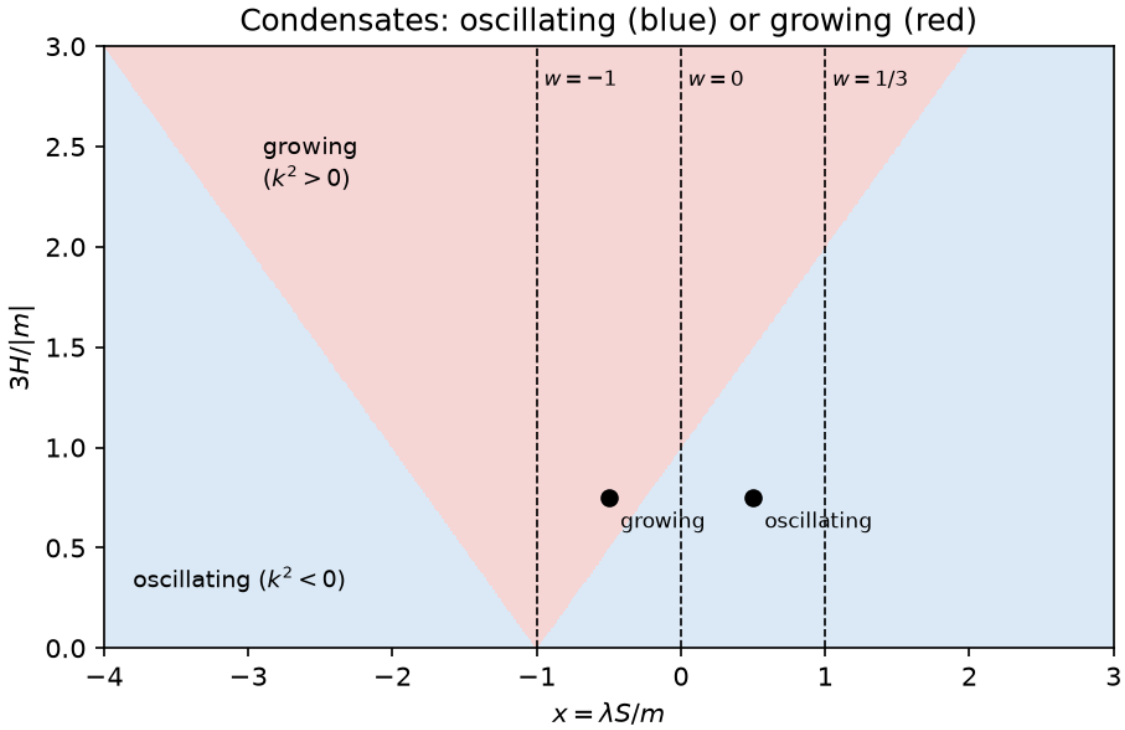

Figure 259: Map of the condensates of dirac16complex00 in the plane of $x = \lambda S/m$ (horizontal) and $3H/|m|$ (vertical), pure numbers. In the red wedge $|1+x| < 3H/|m|$ the number $k^2 = 9H^2-(m+\lambda S)^2$ is positive and the condensate grows; in the blue region it oscillates. The dashed vertical lines are the equations of state $w = -1$ ($x = -1$), $w = 0$ and $w = 1/3$ ($x = 1$); w depends on x only. The black dots are the two condensates of the second figure ($H = 0.25$, $m = 1$). Every condensate with $w = -1$ lies in the growing wedge. (Notebook 09b, figure 5.)

Out [13]:

```
PASS the condensate has p3 = p_t, so its rho is constant along the history
```

Text cell:

```
Then, as an ILLUSTRATION only, the next cell takes toy fluids with assumed
pressures $p_3 = w_3\rho$ and $p_t = w_t\rho$ (constant $w_3$, $w_t$; these are
not solutions of the field equations of this book). The identity becomes
$d\rho/dx_4 = -3AH(w_3 - w_t)\rho$, whose solution is
$\rho = \rho_0 e^{-3AH(w_3 - w_t)x_4}$. The cell solves it numerically with the
Runge-Kutta method (RK4, step $0.05$) for five values of $\Delta w = w_3 - w_t$,
compares with the exact solution (and checks that halving the step divides the
error by about $2^4 = 16$, as it must for a fourth-order method), and checks that
reversing the history ($A \to -A$: the extra times inflate, 3-space deflates)
reverses the flow.
```

In [14]:

```
def rk4(rate, y0, step, count):
    """Solve dy/dx = rate(y) from y(0) = y0 with count Runge-Kutta steps."""
    y, path = y0, [y0]
    for _ in range(count):
        s1 = rate(y)
        s2 = rate(y + step / 2 * s1)
        s3 = rate(y + step / 2 * s2)
        s4 = rate(y + step * s3)
        y = y + step / 6 * (s1 + 2 * s2 + 2 * s3 + s4)
        path.append(y)
    return np.array(path)


STEP, COUNT = 0.05, 160  # x4 from 0 to 8
toy_times = STEP * np.arange(COUNT + 1)
DELTAS = (-2 / 3, -1 / 3, 0.0, 1 / 3, 2 / 3)  # Delta w = w3 - w_t
toy = {}  # Delta w -> rho(x4)/rho0 along A = +1
errors = {STEP: 0.0, STEP / 2: 0.0}  # the largest relative error for two steps
for delta in DELTAS:
    for step in errors:
        count = round(8.0 / step)  # the number of steps from x4 = 0 to 8
        path = rk4(lambda r, d=delta: -3 * A * H * d * r, 1.0, step, count)
        exact = np.exp(-3 * A * H * delta * step * np.arange(count + 1))
        errors[step] = max(errors[step], np.abs(path / exact - 1).max())
        if step == STEP:
            toy[delta] = path
check(errors[STEP] < 1e-7,
      "RK4 (step 0.05) reproduces rho0 exp(-3 A H Delta w x4) for 5 toy fluids")
ratio = errors[STEP] / errors[STEP / 2]  # RK4: half the step, 1/16 of the error
check(14 < ratio < 18, "halving the RK4 step divides the error by about 2^4 = 16")
report("largest relative RK4 error, step 0.05", f"{errors[STEP]:.2e}")
report("error ratio of the steps 0.05 and 0.025", f"{ratio:.2f}")
```

```
reversed_path = rk4(lambda r: -3 * (-A) * H * (1 / 3) * r, 1.0, STEP, COUNT)
check(np.abs(reversed_path * toy[1 / 3] - 1).max() < 1e-8,
      "A -> -A reverses the energy flow (the product of the two paths is 1)")
report("toy fluid Delta w = 1/3: rho(8)/rho(0) along A = 1", f"{toy[1 / 3][-1]:.6f}")
```

Out [14]:

```
PASS RK4 (step 0.05) reproduces rho0 exp(-3 A H Delta w x4) for 5 toy fluids
PASS halving the RK4 step divides the error by about 2^4 = 16
RESULT largest relative RK4 error, step 0.05 = 1.33e-08
RESULT error ratio of the steps 0.05 and 0.025 = 16.17
PASS A -> -A reverses the energy flow (the product of the two paths is 1)
RESULT toy fluid Delta w = 1/3: rho(8)/rho(0) along A = 1 = 0.135335
```

Text cell:

```
The next cell draws the energy densities of the toy fluids and of the condensate
(left), and for the toy fluid with $(w_3, w_t) = (1/3, 2/3)$, that is
$\Delta w = -1/3$, the two contributions to $d\rho/dx_4$ (right): the term of
the inflating 3-space, $-3a_4'p_3$, which takes energy out, and the term of the
deflating extra times, $+3a_4'p_t$, which puts energy in. Here the second is
twice as large as the first, so $\rho$ grows. For the condensate the two would
be equal and opposite at every time. The cell checks that their sum is the
slope $d\rho/dx_4$ of the drawn path.
```

In [15]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
fig.subplots_adjust(wspace=0.35)  # more room between the two panels
for delta in DELTAS:
    left.plot(toy_times, toy[delta], label=f"toy, $\\Delta w = {delta:+.2f}$")
left.plot(times[times <= 8.0], rho_at_times[times <= 8.0] / rho_at_times[0], "k:",
          linewidth=2.0, label="condensate ($p_3 = p_t$)")
left.set_yscale("log")
left.set_xlabel("time $x_4$")
left.set_ylabel("$\\rho(x_4) / \\rho(0)$")
left.set_title("Energy density along $a_4 = AHx_4$ ($A = 1$)")
left.legend(fontsize=7)
rho_toy = toy[-1 / 3]  # Delta w = -1/3: w3 = 1/3, w_t = 2/3
work_space = -3 * A * H * (1 / 3) * rho_toy  # -3 a4' p3 with p3 = rho/3
work_extra = 3 * A * H * (2 / 3) * rho_toy  # +3 a4' p_t with p_t = 2 rho/3
right.plot(toy_times, work_space, label="3-space: $-3a_4' p_3$")
right.plot(toy_times, work_extra, "--", label="extra times: $+3a_4' p_t$")
right.plot(toy_times, np.gradient(rho_toy, toy_times), ":", color="black",
           label="$d\\rho/dx_4$ (finite differences)")
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("time $x_4$")
right.set_ylabel("rate of change of $\\rho$")
right.set_title("Toy fluid $(w_3, w_t) = (1/3, 2/3)$")
right.legend(fontsize=7)
save_figure(fig, "energy_exchange",
```

```
            "Energy exchange along the deflating history $a_4 = AHx_4$, $A = 1$, "
            "$H = 0.25$, for $x_4$ from $0$ to $8$. Left, logarithmic axis: "
            "$\\rho(x_4)/\\rho(0)$ of five toy fluids with assumed pressures "
            "$p_3 = w_3\\rho$, $p_t = w_t\\rho$ and $\\Delta w = w_3 - w_t$ from "
            "$-2/3$ to $2/3$ (an illustration, not solutions of the field "
            "equations), solved with RK4: $\\rho$ falls when $p_3 > p_t$ and rises "
            "when $p_3 < p_t$; the exact condensate (dotted) has $p_3 = p_t$ and "
            "constant $\\rho$. Right: for the toy fluid $(w_3, w_t) = (1/3, 2/3)$ "
            "the term of the inflating 3-space $-3a_4'p_3$ (negative: it takes "
            "energy out), the term of the deflating extra times $+3a_4'p_t$ "
            "(positive and twice as large: it puts energy in) and $d\\rho/dx_4$ "
            "(dotted), which is their sum, so $\\rho$ grows; vertical axis in units "
            "of energy per unit volume per unit time.")
# [2:-2] leaves out two points at each end, where np.gradient is less exact.
check(np.abs(np.gradient(rho_toy, toy_times) - (work_space + work_extra))[2:-2].max()
      < 1e-3, "d rho/dx4 equals the sum of the two work terms on the drawn path")
```

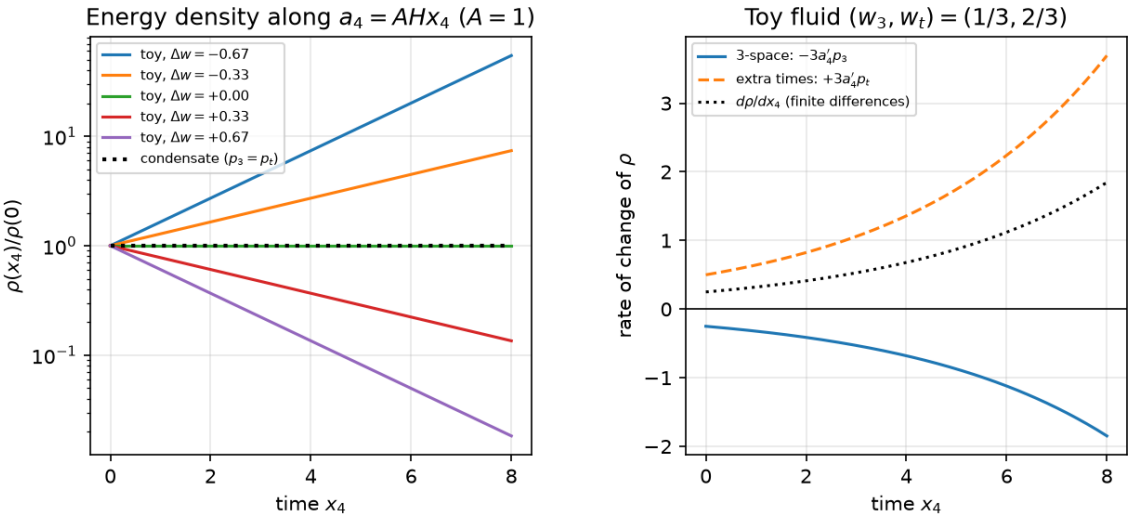

Figure 260: Energy exchange along the deflating history $a_4 = AHx_4$, $A = 1$, $H = 0.25$, for x_4 from 0 to 8. Left, logarithmic axis: $\rho(x_4)/\rho(0)$ of five toy fluids with assumed pressures $p_3 = w_3\rho$, $p_t = w_t\rho$ and $\Delta w = w_3 - w_t$ from $-2/3$ to $2/3$ (an illustration, not solutions of the field equations), solved with RK4: ρ falls when $p_3 > p_t$ and rises when $p_3 < p_t$; the exact condensate (dotted) has $p_3 = p_t$ and constant ρ. Right: for the toy fluid $(w_3, w_t) = (1/3, 2/3)$ the term of the inflating 3-space $-3a_4'p_3$ (negative: it takes energy out), the term of the deflating extra times $+3a_4'p_t$ (positive and twice as large: it puts energy in) and $d\rho/dx_4$ (dotted), which is their sum, so ρ grows; vertical axis in units of energy per unit volume per unit time. (Notebook 09b, figure 6.)

Out [15]:

```
Figure 09b.6 saved as Revision/textbook/figures/09b_6_energy_exchange.png
PASS d rho/dx4 equals the sum of the two work terms on the drawn path
```

Text cell:

```
## 13. The energy of the commuting field has no lower bound

The record proves that the classical energy of dirac16complex00 is unbounded
below already for $U = 0$, with an exact example: in flat 4+4 space (all
$f_a = 1$, no spin connection) with $m = 2$ and the momentum
$(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, the plane waves
$\Phi = u e^{i(k_1x_1 + k_2x_2 + k_0x_0 - Ex_4)}$ with $E = 5$ solve the field
equation when $u$ lies in the 8-dimensional eigenspace of the matrix
$h = m\beta + \sum_a k_a\alpha^a$ ($\beta = -i\gamma^{(4)}$,
$\alpha^a = -\gamma^{(4)}\gamma^{(a)}$) with eigenvalue 5, and the energy density
is $\rho = E u^\dagger Bu$. On that eigenspace the form $u^\dagger Bu$ has four
positive and four negative directions (Krein inertia (4,4)). The next cell
builds $h$, checks that it is Hermitian, that $h^2 = 25$ and that it commutes
with $B$, finds its eigenvalues and the eigenspace with eigenvalue 5, and the
inertia of the form $u^\dagger Bu$ on that eigenspace (the eigenvalues of the
$8 \times 8$ matrix of the form).
```

In [16]:

```
m_flat, E_flat = 2.0, 5.0
momentum = {0: 4.0, 1: 1.0, 2: 2.0, 3: 0.0}  # k0, k1, k2, k3 (x5, x6, x7: zero)
beta = -1j * g4
h = m_flat * beta + sum(k_a * (-g4 @ gamma[a]) for a, k_a in momentum.items())
check(np.abs(h - h.conj().T).max() < 1e-12 and np.abs(h @ h - 25 * I16).max() < 1e-12
      and np.abs(B @ h - h @ B).max() < 1e-12,
      "h is Hermitian, h^2 = 25, and h commutes with B")
energies, vectors = np.linalg.eigh(h)  # eigenvalues in ascending order
U5 = vectors[:, energies > 0]  # the eight columns with eigenvalue +5
form = U5.conj().T @ B @ U5  # u^dagger B u on the eigenspace, as an 8 x 8 matrix
inertia, directions = np.linalg.eigh(form)
PAIRING = "Revision/pairing/reports/python-pairing.json"
check(np.allclose(energies, [-5.0] * 8 + [5.0] * 8, atol=1e-12)
      and np.allclose(inertia, [-1.0] * 4 + [1.0] * 4, atol=1e-12)
      and recorded(PAIRING, "Q.one_particle_Krein_inertia_proof") == "PASS",
      "E = +-5 (eight each); u^dagger B u has inertia (4,4) on the E = 5 space",
      record=f"{PAIRING}, check Q.one_particle_Krein_inertia_proof (every real "
             "frequency)")
```

Out [16]:

```
PASS h is Hermitian, h^2 = 25, and h commutes with B
PASS E = +-5 (eight each); u^dagger B u has inertia (4,4) on the E = 5 space
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_Krein_inertia_proof (every real frequency)
```

Text cell:

```
The next cell defines `flat_energy_density(u)`, which computes
$\rho = -\sum_{a \neq x_4}K_a + mS$ of the plane wave from its kinetic terms
($\partial_a\Phi = ik_a\Phi$), takes the two eigenvectors of the form with
$Bu = -u$ and $Bu = +u$, checks that each plane wave solves the field equation
```

```
$\sum_a\gamma^{(a)}\partial_a\Phi = m\Phi$ of flat space, and checks the record's
values $\rho = -5$ and $\rho = +5$.
```

In [17]:

```
def flat_energy_density(u):
    """rho = -sum over a != x4 of K_a + m S for the plane wave u e^(i(k.x - E x4))."""
    S = (u.conj() @ C @ u).real
    kinetic = 0.0
    for a, k_a in momentum.items():  # d_a Phi = i k_a Phi
        d_u = 1j * k_a * u
        kinetic += 0.5 * (u.conj() @ C @ gamma[a] @ d_u - d_u.conj() @ C @ gamma[a] @ u)
    return (-kinetic + m_flat * S).real


rho_pair = []
# column 0 of directions has the eigenvalue -1 of the form, column 7 the value +1
for column, b_value in ((0, -1.0), (7, 1.0)):
    u = U5 @ directions[:, column]  # a unit vector of the eigenspace, B u = b_value u
    d4_u = -1j * E_flat * u  # d4 Phi = -i E Phi
    field = (g4 @ d4_u + sum(gamma[a] @ (1j * k_a * u) for a, k_a in momentum.items())
             - m_flat * u)  # sum_a gamma^(a) d_a Phi - m Phi
    check(np.abs(field).max() < 1e-12 and np.abs(B @ u - b_value * u).max() < 1e-12,
          f"the plane wave with B u = {b_value:+.0f} u solves the field equation")
    rho_pair.append(flat_energy_density(u))
SCOPE_PY = "Revision/theory/reports/python-scope.json"
SCOPE_WL = "Revision/theory/reports/wolfram-scope.json"
check(abs(rho_pair[0] + 5.0) < 1e-12 and abs(rho_pair[1] - 5.0) < 1e-12
      and recorded(SCOPE_PY, "commuting_field_energy_unbounded_below") == "PASS"
      and recorded(SCOPE_WL, "commuting_field_energy_unbounded_below") == "PASS",
      "rho = -5 for B u = -u and rho = +5 for B u = +u (|u| = 1)",
      record=f"{SCOPE_PY} and {SCOPE_WL}, check commuting_field_energy_unbounded_below")
```

Out [17]:

```
PASS the plane wave with B u = -1 u solves the field equation
PASS the plane wave with B u = +1 u solves the field equation
PASS rho = -5 for B u = -u and rho = +5 for B u = +u (|u| = 1)
     reproduces Revision/theory/reports/python-scope.json and
    Revision/theory/reports/wolfram-scope.json, check
    commuting_field_energy_unbounded_below
```

Text cell:

```
The next cell draws 5000 random unit vectors $u$ of the eigenspace (random
combinations of its eight columns, each divided by its length), computes the
energy density of each plane wave from its kinetic terms, and checks that it
equals $E\,u^\dagger Bu$ every time and takes both signs.
```

In [18]:

```
samples = rng.normal(size=(5000, 8)) + 1j * rng.normal(size=(5000, 8))
samples /= np.linalg.norm(samples, axis=1)[:, None]
us = samples @ U5.T  # 5000 random unit vectors of the eigenspace
charges = np.einsum("ni,ij,nj->n", us.conj(), B, us).real  # u^dagger B u
rho_samples = np.array([flat_energy_density(u) for u in us])
check(np.abs(rho_samples - E_flat * charges).max() < 1e-12
      and rho_samples.min() < 0 < rho_samples.max(),
      "rho = E u^dagger B u for all 5000 random u of the eigenspace; both signs")
report("smallest and largest rho of the 5000 samples",
       f"{rho_samples.min():.4f} and {rho_samples.max():.4f}")
```

Out [18]:

```
PASS rho = E u^dagger B u for all 5000 random u of the eigenspace; both signs
RESULT smallest and largest rho of the 5000 samples = -4.3821 and 4.4892
```

Text cell:

```
The next cell draws the histogram of the 5000 energy densities.
```

In [19]:

```
fig, ax = plt.subplots()
ax.hist(rho_samples, bins=50, range=(-5.0, 5.0), color="tab:purple", edgecolor="white")
ax.axvline(0.0, color="black", linewidth=1.0)
ax.set_xlabel("energy density $\\rho = E\\,u^\\dagger B u$ of the plane wave")
ax.set_ylabel("number of the 5000 random $u$")
ax.set_title("Waves of positive frequency $E = 5$: $\\rho$ has both signs")
save_figure(fig, "energy_sign",
            "Histogram of the energy density $\\rho = -\\sum_{a \\neq x_4} K_a + mS$ "
            "of 5000 plane waves $u\\,e^{i(k \\cdot x - Ex_4)}$ of dirac16complex00 "
            "in flat 4+4 space with $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ "
            "and positive frequency $E = 5$, for random unit vectors $u$ of the "
            "8-dimensional solution space; horizontal axis $\\rho$ in units of "
            "energy per unit volume, vertical axis the count in each of 50 "
            "intervals. Every value equals $E\\,u^\\dagger Bu$; about half are "
            "negative. The possible values fill the whole interval from $-5$ "
            "(reached for $Bu = -u$) to $5$ (for $Bu = u$), but random vectors "
            "seldom come near the two ends: positive frequency does not mean "
            "positive energy for this commuting field.")
```

Out [19]:

```
Figure 09b.7 saved as Revision/textbook/figures/09b_7_energy_sign.png
```

Text cell:

```
## 14. The last check

The last cell checks that the seven figures are saved with their captions and
prints the number of checks that passed.
```

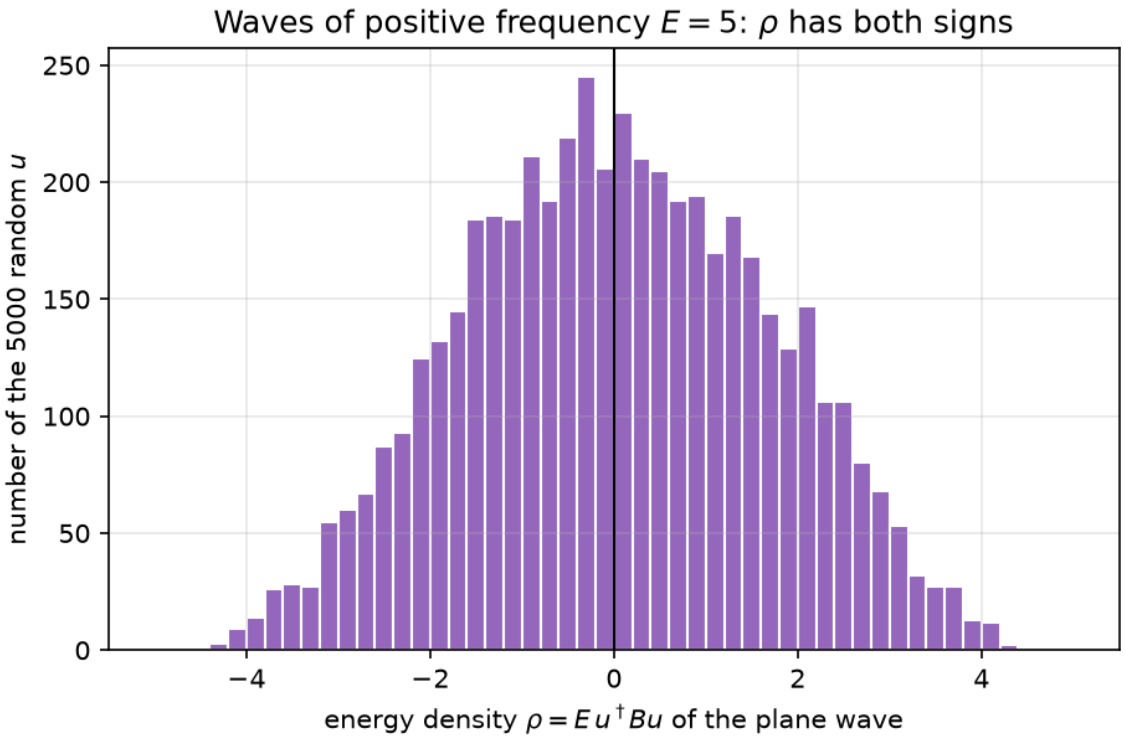

Figure 261: Histogram of the energy density $\rho = -\sum_{a \neq x_4} K_a + mS$ of 5000 plane waves $u\, e^{i(k\cdot x - Ex_4)}$ of dirac16complex00 in flat 4+4 space with $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ and positive frequency $E = 5$, for random unit vectors u of the 8-dimensional solution space; horizontal axis ρ in units of energy per unit volume, vertical axis the count in each of 50 intervals. Every value equals $E\,u^\dagger Bu$; about half are negative. The possible values fill the whole interval from -5 (reached for $Bu = -u$) to 5 (for $Bu = u$), but random vectors seldom come near the two ends: positive frequency does not mean positive energy for this commuting field. (Notebook 09b, figure 7.)

In [20]:

```
captions = json.loads(output_file(CAPTION_FILE).read_text(encoding="utf-8"))
expected_files = ["09b_1_sign_of_s.png", "09b_2_condensate_solutions.png",
                  "09b_3_energy_and_pressure.png", "09b_4_equation_of_state.png",
                  "09b_5_regime_map.png", "09b_6_energy_exchange.png",
                  "09b_7_energy_sign.png"]
check(sorted(captions) == expected_files and all(
    output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected_files),
    "the seven figures of this notebook are saved and captioned")
all_checks_passed()
```

Out [20]:

```
PASS the seven figures of this notebook are saved and captioned
ALL 33 CHECKS PASSED (notebook 09b)
```

Text cell:

```
## 15. What this notebook showed

- The exact condensates of dirac16complex00,
  $\Phi = (\cosh(kx_4) + \frac{\sinh(kx_4)}{k}M)\chi$ with
  $k^2 = 9H^2 - (m + \lambda S_0)^2$, solve the field equation on every history
  $a_4$; $S$ stays equal to $S_0$, whether the condensate oscillates
  ($k^2 < 0$) or grows ($k^2 > 0$).
- Their energy density and pressure are $\rho = mS + \frac{\lambda}{2}S^2$ and
  $p = \frac{\lambda}{2}S^2$, the same in all seven directions; the kinetic
  energy density is zero, the kinetic pressure is $(m + \lambda S)S$; the
  $x_4$-$x_0$ component vanishes (record values, computed here anew from the
  solutions).
- The equation of state is $w = x/(2 + x)$ with $x = \lambda S/m$, measured on
  twelve exact solutions; $w = -1$ exactly where the effective mass vanishes, and
  such condensates always grow. These $w$ are constant in time and are
  8-dimensional ratios; what a 3-space observer would infer rests on an ASSUMED
  normalisation of its energy density (Chapter 22).
- Because $p_3 = p_t$, a condensate exchanges no energy between 3-space and the
  extra times; toy fluids with $p_3 \neq p_t$ (an illustration) show the exchange
  $d\rho/dx_4 = -3a_4'(p_3 - p_t)$ and its reversal under $A \to -A$.
- $S$ and the energy density of the commuting field have no sign: half of all
  random condensates with $\lambda = 0$ have $\rho < 0$, and positive-frequency
  plane waves have $\rho$ anywhere between $-E$ and $E$ (record:
  `commuting_field_energy_unbounded_below`).
- Status: the exact condensates, their $\rho$ and $p$, the vanishing $x_4$-$x_0$
  component, the energy-exchange identity and the unbounded energy are PROVED in
  the Revision record (exact, Wolfram and sympy); this notebook re-computes them
  numerically (COMPUTED). The toy fluids are an ILLUSTRATION, not solutions. The
  equation of state of a 3-space observer is not computed here; the Revision
  record computes it under the ASSUMED definitions A, B and C (Chapter 22), and
  which definition describes a physical observer is OPEN. The parameter values
  are choices of this notebook.
```

9.25 Line-by-line walk-through of Notebook 09b

The notebook has 20 code cells, In [1] to In [20]. As in Section 9.17, long figure captions are shortened to a line "...)" and docstrings are left out here; the captions are printed in full in Section 9.24.

In [1], the set-up cell. It is word for word the set-up cell of Notebook 09a, explained line by line in Section 9.17, with two differences: its comment lines are the run instructions of Notebook 09b (Section 9.23), and its line `NOTEBOOK_ID = "09b"` names this notebook, so the figures are saved as `09b_<k>_<name>.png` and their captions in `09b.captions.json`. It prints `Set-up of notebook 09b complete:  repository folder found, helpers defined.`

In [2], the matrices C and B, and the sign of S.

```
import numpy as np  # arrays of numbers, matrices, linear algebra

fixture = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
gamma = [np.array(rows, dtype=float) for rows in fixture["gamma"]]  # x0, ..., x7
C = np.array(fixture["C"], dtype=float)  # gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
B = (np.array(fixture["B"]["re"], dtype=float)  # B = -i C gamma^(x4), stored as its
     + 1j * np.array(fixture["B"]["im"], dtype=float))  # real and imaginary parts
I16 = np.eye(16)  # the 16 x 16 unit matrix
g4, g0 = gamma[4], gamma[0]  # gamma^(x4) and gamma^(x0)
```

As in Notebook 09a, the fixture gives the eight real gammas (`gamma[0]` is $\gamma^{(0)}$, ..., `gamma[7]` is $\gamma^{(7)}$) and C. The matrix $B = -iC\gamma^{(4)}$ is complex; JSON has no complex numbers, so the fixture stores its real part and its imaginary part separately, and the cell adds them as `re + 1j * im`. The last line gives $\gamma^{(4)}$ and $\gamma^{(0)}$ the short names `g4` and `g8` (two names assigned at once from two values).

```
def recorded(path, name):
    """The verdict (PASS or FAIL) of the check name in the Revision report path."""
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for item in report_data["checks"]:
        if item["name"] == name:
            return item["verdict"].upper()  # some reports write "pass"
    raise KeyError(f"{path} has no check {name}")
```

The same helper as in Notebook 09a: the verdict of one check of a Revision report.

```
ALGEBRA_PY = "Revision/algebra/reports/python-algebra.json"
ALGEBRA_WL = "Revision/algebra/reports/wolfram-algebra.json"
eigenvalues_C = np.linalg.eigvalsh(C)  # C is symmetric: eigvalsh, sorted ascending
check(np.allclose(eigenvalues_C, [-1.0] * 8 + [1.0] * 8, atol=1e-12)
      and recorded(ALGEBRA_PY, "C_equals_notebook_sigma16") == "PASS"
      and recorded(ALGEBRA_WL, "sigma8_involution") == "PASS",
      "C has eight eigenvalues -1 and eight eigenvalues +1",
      record=f"{ALGEBRA_PY}, check C_equals_notebook_sigma16; {ALGEBRA_WL}, check "
             "sigma8_involution")
```

An **eigenvalue** of a matrix M is a number μ for which $Mv = \mu v$ has a solution column $v \neq 0$ (an **eigenvector**). `np.linalg.eigvalsh` computes the eigenvalues of a symmetric (or Hermitian) matrix and returns them sorted from the smallest to the largest. The check compares them with eight -1 followed by eight $+1$ (the list `[-1.0] * 8 + [1.0] * 8`), with an absolute tolerance of 10^{-12}: the fact of Section 9.21.

```
rng = np.random.default_rng(2026)  # a fixed seed: the same numbers in every run
chis = rng.normal(size=(20000, 16)) + 1j * rng.normal(size=(20000, 16))
chis /= np.linalg.norm(chis, axis=1)[:, None]  # every row now has length 1
# S0 = chi^dagger C chi for every row at once (einsum sums over the indices i, j).
S0_values = np.einsum("ni,ij,nj->n", chis.conj(), C, chis).real
negative_share = float(np.mean(S0_values < 0))  # the fraction of negative S0
check(S0_values.min() < 0 < S0_values.max() and np.abs(S0_values).max() <= 1 + 1e-12,
      "S0 = chi^dagger C chi takes both signs and lies between -1 and 1")
report("fraction of the 20000 random chi with S0 < 0", f"{negative_share:.4f}")
```

`chis` is a table of 20000 random complex columns χ, one per row. `np.linalg.norm(chis, axis=1)` computes the length of each row, and `/=` divides each row by its length (the `[:, None]` makes the 20000 lengths a column so that each row is divided by its own length): every χ now has length 1. `np.einsum("ni,ij,nj->n", ...)` is Einstein's summation written as a recipe: for every row n it computes $\sum_{i,j} \chi^*_{ni} C_{ij} \chi_{nj} = \chi_n^\dagger C \chi_n$, all 20000 values of S_0 at once. `S0_values < 0` is an array of `True` and `False`, and its mean (`np.mean`, with `True` counted as 1) is the fraction of negative values. The check requires values of both signs (`a < 0 < b` means $a < 0$ and $0 < b$) and all values between -1 and 1, as they must be for a unit column and eigenvalues ± 1. It prints the fraction 0.4983: about half.

In [3], the histogram of S_0 (Figure 09b.1).

```
fig, ax = plt.subplots()
ax.hist(S0_values, bins=60, range=(-1.0, 1.0), color="tab:blue", edgecolor="white")
ax.axvline(0.0, color="black", linewidth=1.0)
ax.set_xlabel("$S_0 = \\chi^\\dagger C \\chi$ for a random $\\chi$ of length 1")
ax.set_ylabel("number of the 20000 random $\\chi$")
ax.set_title("The scalar $S = \\bar\\Phi\\Phi$ has no sign")
save_figure(fig, "sign_of_s",
            "Histogram of $S_0 = \\chi^\\dagger C\\chi$ for 20000 random complex "
            ...)
```

A **histogram** is a bar chart that counts how many numbers fall into each of a row of equal intervals: `ax.hist` uses 60 intervals between -1 and 1. `ax.axvline(0.0, ...)` draws a vertical line at zero. **What Figure 09b.1 shows.** The number of random unit columns (vertical axis) whose S_0 falls into each interval (horizontal axis, a pure number between -1 and 1). The histogram is symmetric about zero, roughly a hill: about half of the values are negative. The student should see that S has no sign; for a condensate with $\lambda = 0$, $\rho = mS_0$ has the sign of S_0.

In [4], the matrix M and its square.

```
def condensate_matrix(V, H):
    """M = -V gamma^(x4) + 3 H gamma^(x4) gamma^(x0) and k^2 = 9 H^2 - V^2."""
    return -V * g4 + 3 * H * g4 @ g0, 9 * H ** 2 - V ** 2
```

The function returns two results, the matrix M of Section 9.18 and the number $k^2 = 9H^2 - V^2$.

```
THEORY_WL = "Revision/theory/reports/wolfram-field-theory.json"
THEORY_PY = "Revision/theory/reports/python-field-theory.json"
worst_square, worst_c = 0.0, 0.0  # the largest deviations found
for V_test in (-2.0, -0.5, 0.0, 0.75, 1.5, 3.0):
    for H_test in (0.1, 0.25, 1.0):
        M_test, k2_test = condensate_matrix(V_test, H_test)
        worst_square = max(worst_square, np.abs(M_test @ M_test - k2_test * I16).max())
        worst_c = max(worst_c, np.abs(M_test.T @ C + C @ M_test).max())
check(worst_square < 1e-12 and worst_c < 1e-12
      and recorded(THEORY_WL, "solution_matrix_square") == "PASS",
      "M^2 = (9 H^2 - V^2) times 1 and M^T C + C M = 0 for 18 pairs (V, H)",
      record=f"{THEORY_WL}, check solution_matrix_square")
```

For six values of V and three of H (18 pairs) the loops compute the largest entry of $M^2 - k^2 \cdot 1$ and of $M^T C + CM$, and keep the largest over all pairs. The check confirms the two facts derived in Section 9.18 up to rounding.

In [5], one column χ with $S_0 = 1$, and the solution formula.

```
m, H = 1.0, 0.25  # the mass and the author's constant
rng = np.random.default_rng(7)
while True:  # draw until S0 > 0 (the first draw usually succeeds)
    chi = rng.normal(size=16) + 1j * rng.normal(size=16)
    S0 = (chi.conj() @ C @ chi).real
    if S0 > 0:
        break
chi = chi / np.sqrt(S0)  # now chi^dagger C chi = 1
```

The cell fixes $m = 1$ and $H = 0.25$, starts a new random generator with the seed 7, and repeats (`while True:` loops until `break`) drawing a random column until its S_0 is positive. Dividing χ by $\sqrt{S_0}$ divides $S_0 = \chi^\dagger C \chi$ by S_0, so afterwards $S_0 = 1$.

```
def condensate(x4, chi_value, M, k2):
    """Phi(x4) and dPhi/dx4 of the exact condensate with Phi(0) = chi_value."""
    if k2 == 0.0:  # cosh(k x4) -> 1 and sinh(k x4)/k -> x4 when k -> 0
        return chi_value + x4 * M @ chi_value, M @ chi_value
    k = np.sqrt(complex(k2))  # k is imaginary when k2 < 0
    phi = np.cosh(k * x4) * chi_value + np.sinh(k * x4) / k * (M @ chi_value)
    dphi = k * np.sinh(k * x4) * chi_value + np.cosh(k * x4) * (M @ chi_value)
    return phi, dphi
```

`condensate` returns $\Phi(x_4) = (\cosh(kx_4) + \frac{\sinh(kx_4)}{k} M)\chi$ and its derivative $\Phi'(x_4) = k \sinh(kx_4)\chi + \cosh(kx_4) M\chi$ (Section 9.18). The special case $k^2 = 0$ uses the limit $\Phi = \chi + x_4 M\chi$. `complex(k2)` turns k^2 into a complex number, so that `np.sqrt` returns an imaginary k when $k^2 < 0$ (for a real negative number it would fail); numpy computes cosh and sinh of complex arguments correctly.

```
S0_now = (chi.conj() @ C @ chi).real  # 1 up to rounding
report("S0 = chi^dagger C chi after the division", f"{S0_now:.6f}")
```

The cell prints `1.000000`.

In [6], two exact condensates and the field equation at 401 times.

```
times = np.linspace(0.0, 12.0, 401)  # the times x4 at which the solution is tested
EXAMPLES = {"oscillating": 0.5, "growing": -0.5}  # name -> lambda
solutions = {}  # name -> (Phi at every time, S at every time)
for name, lam in EXAMPLES.items():
    V = m + lam * 1.0  # S0 = 1
    M, k2 = condensate_matrix(V, H)
    phis, S_values, worst_residual = [], [], 0.0
```

The two examples are stored in a dictionary from their names to their λ; `.items()` gives the pairs. For each, $V = m + \lambda S_0$ with $S_0 = 1$, and M, k^2 follow. Three empty containers collect the values of Φ, the values of S and the largest residual.

```
    for x4 in times:
        phi, dphi = condensate(x4, chi, M, k2)
        S_now = (phi.conj() @ C @ phi).real
        residual = g4 @ dphi + 3 * H * g0 @ phi - (m + lam * S_now) * phi
        # The residual relative to the size of the solution at this time.  Rounding
        # errors grow with the solution (S is a difference of large numbers when
        # Phi grows), so the tolerance below is 1e-9, not 1e-15.
        worst_residual = max(worst_residual,
                             np.abs(residual).max() / max(1.0, np.abs(phi).max()))
        phis.append(phi)
        S_values.append(S_now)
    solutions[name] = (np.array(phis), np.array(S_values))
```

At each of the 401 times the inner loop evaluates the solution, its S, and the **residual** of the field equation $\gamma^{(4)}\Phi' + 3H\gamma^{(0)}\Phi - (m + \lambda S)\Phi$, which must vanish; note that it uses S computed at that time, so the test also covers the nonlinearity. The residual is divided by the size of the solution (at least 1), because rounding errors grow with a growing solution. The values are appended (`.append`) to the lists and finally stored in `solutions`.

```
    report(f"{name}: lambda = {lam}, V = m + lambda S0", f"{V:.4f}")
    report(f"{name}: k^2 = 9 H^2 - V^2", f"{k2:.4f}")
    check(worst_residual < 1e-9,
          f"{name}: the field equation holds at 401 times")
    check(np.abs(np.array(S_values) - 1.0).max() < 1e-9,
          f"{name}: S stays equal to S0 = 1 at 401 times")
check(recorded(THEORY_WL, "exact_solution_nonlinear_homogeneous_C") == "PASS"
      and recorded(THEORY_PY, "exact_nonlinear_homogeneous_solution") == "PASS",
      "both condensates are the record's exact nonlinear homogeneous solution",
      record=f"{THEORY_WL}, check exact_solution_nonlinear_homogeneous_C; "
             f"{THEORY_PY}, check exact_nonlinear_homogeneous_solution")
```

For each example the cell prints V and k^2 (oscillating: $V = 1.5$, $k^2 = -1.6875$; growing: $V = 0.5$, $k^2 = 0.3125$, the numbers of Section 9.20) and checks the field equation and the constancy of S at

all 401 times; after the loop, one more check confirms that the record proved these solutions exactly. Five PASS lines in all.

In [7], the two condensates (Figure 09b.2).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
phis, S_values = solutions["oscillating"]
for component in (0, 5, 10):  # three of the 16 components (rows 1, 6 and 11)
    left.plot(times, phis[:, component].real,
              label=f"Re $\\Phi_{{{component + 1}}}$")
left.plot(times, S_values, "k--", label="$S = \\bar\\Phi\\Phi$")
left.set_xlabel("time $x_4$")
left.set_ylabel("value")
left.set_ylim(-0.8, 1.35)  # room for the legend above the curves
left.set_title("Oscillating: $\\lambda = 0.5$, $V = 1.5$")
left.legend(fontsize=7, loc="upper center", ncol=4)
```

The left panel draws the real parts (`.real`) of the components 1, 6 and 11 of the oscillating condensate against the time, and S as a black dashed line (the style string is the letter `k`, which means black, followed by two hyphens). In the f-string, a doubled brace prints a single brace, so the label of component 1 reads Re Φ_1. The legend is placed at the top in four columns.

```
phis, S_values = solutions["growing"]
right.plot(times, np.einsum("ti,ti->t", phis.conj(), phis).real,
           label="$\\Phi^\\dagger\\Phi$")
right.plot(times, S_values, "k--", label="$S = \\bar\\Phi\\Phi$")
right.set_yscale("log")
right.set_xlabel("time $x_4$")
right.set_title("Growing: $\\lambda = -0.5$, $V = 0.5$")
right.legend(fontsize=8)
save_figure(fig, "condensate_solutions",
            "Two exact condensates of dirac16complex00 with $m = 1$, $H = 0.25$ and "
            ...)
```

The right panel draws, for the growing condensate, the size $\Phi^\dagger\Phi = \sum_i |\Phi_i|^2$ at every time (the recipe `"ti,ti->t"` sums over the components `i` for each time `t`) and S, on a logarithmic axis. **What Figure 09b.2 shows.** Left: three components oscillate with the period $2\pi/\omega = 4.84$ while S stays at 1. Right: $\Phi^\dagger\Phi$ rises, soon along a straight line on the logarithmic axis, like e^{2kx_4} with $k = 0.559$, by more than a factor 100000, while S stays exactly 1. The student should see that a constant S does not mean a constant field: the indefinite form $\Phi^\dagger C\Phi$ can stay fixed while the field grows.

In [8], the energy density and pressure of the condensates.

```
def homogeneous_tensor(phi, dphi, lam):
    """K4, rho, p and B04 of a condensate at one time."""
    S = (phi.conj() @ C @ phi).real
    U = lam / 2 * S ** 2
    K4 = 0.5 * (phi.conj() @ C @ g4 @ dphi - dphi.conj() @ C @ g4 @ phi)
    L0 = K4.real - m * S - U  # the other seven kinetic terms are zero
    B04 = phi.conj() @ C @ g0 @ dphi - dphi.conj() @ C @ g0 @ phi
    return S, U, K4, K4.real - L0, L0, B04  # rho = K4 - L0, p = L0
```

The function computes from Φ and Φ' at one time: S, U, the kinetic term $K_4 = \frac{1}{2}(\bar{\Phi}\gamma^{(4)}\Phi' - \bar{\Phi}'\gamma^{(4)}\Phi)$ (with $f_4 = 1$; `phi.conj() @ C` is $\bar{\Phi}$), $L_0 = K_4 - mS - U$ (the other seven kinetic terms vanish), and $B_{04} = \bar{\Phi}\gamma^{(0)}\Phi' - \bar{\Phi}'\gamma^{(0)}\Phi$. It returns six values; the fourth is $\rho = -T^{x_4}{}_{x_4} = -(L_0 - K_4) = K_4 - L_0$ and the fifth is $p = T^{\mu}{}_{\mu} = L_0 - K_\mu = L_0$ for every $\mu \neq x_4$.

```
values = {}  # name -> (rho, p) of the condensate
for name, lam in EXAMPLES.items():
    V = m + lam * 1.0
    M, k2 = condensate_matrix(V, H)
    rhos, ps, worst = [], [], 0.0
    for x4 in times:
        phi, dphi = condensate(x4, chi, M, k2)
        S, U, K4, rho, p, B04 = homogeneous_tensor(phi, dphi, lam)
        size = max(1.0, (phi.conj() @ phi).real)  # for relative tolerances
        worst = max(worst, abs(K4.imag) / size, abs(K4.real - V * S) / size,
                    abs(B04) / size)
        rhos.append(rho)
        ps.append(p)
    rhos, ps = np.array(rhos), np.array(ps)
    values[name] = (rhos.mean(), ps.mean())
```

For both condensates and all 401 times the loop evaluates the tensor and keeps the largest of three numbers that must vanish: the imaginary part of K_4, the difference $K_4 - VS$, and B_{04}, each relative to the size $\Phi^\dagger\Phi$ (at least 1). The energy densities and pressures are collected, and their means are stored.

```
    check(worst < 1e-12, f"{name}: K4 is real, K4 = V S, and B04 = 0 (T^x4_x0 = 0)")
    check(np.abs(rhos - (m + lam / 2)).max() < 1e-9
          and np.abs(ps - lam / 2).max() < 1e-9,
          f"{name}: rho = m S + lambda S^2/2 and p = lambda S^2/2 at 401 times")
    check(abs(-values[name][0] + 7 * values[name][1] - (-m + 3 * lam)) < 1e-9,
          f"{name}: trace -rho + 7 p = -m S + 3 lambda S^2")
    report(f"{name}: rho", f"{values[name][0]:.6f}")
    report(f"{name}: p (all seven directions)", f"{values[name][1]:.6f}")
    report(f"{name}: w = p/rho", f"{values[name][1] / values[name][0]:.6f}")
```

Three checks per condensate: K_4 is real and equals VS and $B_{04} = 0$ (so $T^{x_4}{}_{x_0} = 0$, Section 9.19); $\rho = m + \lambda/2$ and $p = \lambda/2$ at all 401 times (the values $mS + \frac{\lambda}{2}S^2$ and $\frac{\lambda}{2}S^2$ with $S = 1$); and the trace $-\rho + 7p = -mS + 3\lambda S^2$. The printed values are: oscillating, $\rho = 1.25$, $p = 0.25$, $w = 0.2$; growing, $\rho = 0.75$, $p = -0.25$, $w = -0.333333$. (By hand: $1 + 0.5/2 = 1.25$, $0.5/2 = 0.25$, $0.25/1.25 = 0.2$; $1 - 0.25 = 0.75$, $-0.25/0.75 = -1/3$.)

```
check(recorded(THEORY_PY, "commuting_homogeneous_on_shell_rho_p") == "PASS"
      and recorded(THEORY_PY, "commuting_T_x4x0_homogeneous") == "PASS"
      and recorded(THEORY_PY, "commuting_trace_on_shell") == "PASS",
      "these are the record's homogeneous values and its vanishing x4-x0 component",
      record=f"{THEORY_PY}, checks commuting_homogeneous_on_shell_rho_p, "
             "commuting_T_x4x0_homogeneous and commuting_trace_on_shell")
```

The last check confirms that the record proved these statements exactly. Seven PASS lines in all.

In [9], energy density and pressure against S (Figure 09b.3).

```
lam = 0.5
S_axis = np.linspace(-6.0, 2.0, 401)
rho_axis = m * S_axis + lam / 2 * S_axis ** 2  # rho = m S + U
p_axis = lam / 2 * S_axis ** 2  # p = S U' - U
p_kin = (m + lam * S_axis) * S_axis  # V S
p_pot = -(m * S_axis + lam / 2 * S_axis ** 2)  # -(m S + U)
check(np.abs(p_kin + p_pot - p_axis).max() < 1e-12,
      "p = p_kin + p_pot = V S - (m S + U) at every drawn S")
```

With $m = 1$ and $\lambda = 0.5$ these lines tabulate, for 401 values of S from -6 to 2, the energy density, the pressure and its two parts, and check that the parts add up to the pressure.

```
fig, ax = plt.subplots()
ax.plot(S_axis, rho_axis, label="energy density $\\rho = mS + \\lambda S^2/2$")
ax.plot(S_axis, p_axis, label="pressure $p = \\lambda S^2/2$")
ax.plot(S_axis, p_kin, "--", label="kinetic part of $p$: $(m + \\lambda S)S$")
ax.plot(S_axis, p_pot, ":", label="potential part of $p$: $-(mS + U)$")
ax.plot([0.0, -2 * m / lam], [0.0, 0.0], "ko", label="$\\rho = 0$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("$S = \\bar\\Phi\\Phi$")
ax.set_ylabel("energy per unit volume")
ax.set_title("Condensate: $\\rho$ and $p$ against $S$ ($m = 1$, $\\lambda = 0.5$)")
ax.legend(fontsize=8)
save_figure(fig, "energy_and_pressure",
            "Energy density $\\rho = mS + \\lambda S^2/2$ (solid), pressure "
            ...)
```

Four curves and two black dots (`"ko"`: black circles) at the zeros $S = 0$ and $S = -2m/\lambda = -4$ of ρ. **What Figure 09b.3 shows.** Against S (horizontal, a pure number) in units of energy per unit volume: the energy density, a parabola through 0 and -4 with its lowest point -1 at $S = -2$; the pressure, a parabola that touches zero at $S = 0$; the kinetic part of the pressure (dashed) and its potential part (dotted). The student should see that the energy density is negative for $-4 < S < 0$ and that the pressure is the sum of a kinetic and a potential part that partly cancel.

In [10], the equation of state of twelve exact condensates.

```
x_measured = np.array([-3.5, -3.0, -2.5, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 2.0, 3.0,
                       4.0])  # values of x = lambda S/m (here lambda = x)
rho_measured, p_measured = [], []  # rho and p of each of the twelve condensates
for x_value in x_measured:
    V = m + x_value * 1.0  # lambda = x_value, S0 = 1
    M, k2 = condensate_matrix(V, H)
    phi, dphi = condensate(1.3, chi, M, k2)
    S, U, K4, rho, p, B04 = homogeneous_tensor(phi, dphi, x_value)
    rho_measured.append(rho)
    p_measured.append(p)
rho_measured, p_measured = np.array(rho_measured), np.array(p_measured)
w_measured = p_measured / rho_measured  # the equation of state of each condensate
w_formula = x_measured / (2 + x_measured)
```

With $m = 1$ and $S_0 = 1$ the number $x = \lambda S/m$ equals λ, so the cell builds, for twelve values of x, the exact condensate with $\lambda = x$, takes Φ and Φ' at the time $x_4 = 1.3$, and computes ρ and p from them as in In [8]; the formula $w = x/(2 + x)$ is used only afterwards, for the comparison. The value $x = -2$ is left out, because there $\rho = 0$.

```
check(np.abs(w_measured - w_formula).max() < 1e-9,
      "w = p/rho of 12 exact condensates equals x/(2 + x), x = lambda S/m")
at_minus_one = w_measured[list(x_measured).index(-1.0)]
check(abs(at_minus_one + 1.0) < 1e-12,
      "w = -1 exactly where the effective mass m + lambda S vanishes (x = -1)")
below = x_measured < -2  # the three condensates with x < -2
check(np.all(rho_measured[below] < 0) and np.all(p_measured[below] < 0)
      and np.all(w_measured[below] > 1),
      "for x < -2: rho < 0, p < 0 and w > 1")
for x_value, w_value in zip(x_measured, w_measured):
    say(f"x = lambda S/m = {x_value:5.1f}:  w = p/rho = {w_value: .6f}")
```

Three checks: the twelve measured values agree with $x/(2 + x)$; the value at $x = -1$ (`list(... ).index(-1.0)` finds its position) is -1; and the three condensates with $x < -2$ (`below` is an array of `True`/`False` that selects them) have negative ρ, negative p and $w > 1$ (`np.all` is true when every selected value satisfies the condition). The loop prints the table: for example $x = -1.5$ gives $w = -3$, $x = 1$ gives $w = 1/3$, $x = -2.5$ gives $w = 5$. The format `{x_value:5.1f}` prints with one decimal in five places and `{w_value:  .6f}` leaves a blank in place of a plus sign, so the columns line up.

In [11], the equation of state against x (Figure 09b.4).

```
fig, ax = plt.subplots()
for piece in (np.linspace(-6.0, -2.05, 300), np.linspace(-1.95, 6.0, 500)):
    ax.plot(piece, piece / (2 + piece), color="tab:blue")  # two branches
ax.plot(x_measured, w_measured, "o", color="tab:red",
        label="measured on exact condensates")
ax.axvline(-2.0, color="gray", linestyle=":", label="$\\rho = 0$ ($x = -2$)")
ax.axhline(-1.0, color="tab:green", linestyle="--", label="$w = -1$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.axhspan(-6.0, -1.0, xmin=0.0, xmax=1.0, color="tab:green", alpha=0.08)
ax.set_ylim(-6.0, 6.0)
ax.set_xlabel("$x = \\lambda S / m$")
ax.set_ylabel("$w = p/\\rho$")
ax.set_title("Equation of state of a condensate: $w = x/(2 + x)$")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "equation_of_state",
            "Equation of state $w = p/\\rho$ of a condensate of dirac16complex00 "
            ...)
```

The curve $w = x/(2 + x)$ is drawn in two pieces that stop short of $x = -2$, where it jumps from $+\infty$ to $-\infty$. The twelve measured values are red dots; a dotted vertical line marks $x = -2$, a dashed horizontal line $w = -1$, and `ax.axhspan` shades the band $w < -1$ faintly (`alpha=0.08` is 8 per cent opaque). **What Figure 09b.4 shows.** The equation of state (vertical) against x (horizontal), both pure numbers. The red dots lie on the curve. The student should read off: $w = 0$ at $x = 0$; $w = -1$

at $x = -1$; the phantom band $w < -1$ for $-2 < x < -1$; $w > 1$ for $x < -2$, where ρ and p are both negative; and $w \to 1$ for large x. This is the eight-dimensional ratio, not the equation of state an observer in 3-space would infer (that one rests on an ASSUMED definition of the observer's energy density; Section 9.19).

In [12], where condensates oscillate and where they grow (Figure 09b.5).

```
x_grid = np.linspace(-4.0, 3.0, 701)  # x = lambda S/m
h_grid = np.linspace(0.0, 3.0, 301)  # h = 3 H/|m|
X, Hgrid = np.meshgrid(x_grid, h_grid)  # every pair (x, h) of the two grids
growing_region = (np.abs(1 + X) < Hgrid).astype(float)  # 1 where k^2 > 0
fig, ax = plt.subplots()
ax.contourf(X, Hgrid, growing_region, levels=[-0.5, 0.5, 1.5],
            colors=["#dbe9f6", "#f6d5d5"])
```

`np.meshgrid` makes two tables that together list every pair (x, h) of the two grids, with $h = 3H/|m|$. The condition $|1 + x| < h$ of Section 9.20 is evaluated at every pair and turned into 1 (growing) or 0 (oscillating) by `.astype(float)`. `ax.contourf` colours the plane by these values: the levels -0.5, 0.5, 1.5 separate 0 from 1, and the two colours are a light blue and a light red, given as hexadecimal colour codes.

```
for label, x_line in (("$w = -1$", -1.0), ("$w = 0$", 0.0), ("$w = 1/3$", 1.0)):
    ax.axvline(x_line, color="black", linestyle="--", linewidth=0.8)
    ax.text(x_line + 0.05, 2.8, label, fontsize=8)
for name, lam_value in EXAMPLES.items():
    ax.plot([lam_value * 1.0 / m], [3 * H / abs(m)], "ko")
    ax.text(lam_value / m + 0.08, 3 * H / abs(m) - 0.15, name, fontsize=8)
ax.text(-3.8, 0.3, "oscillating ($k^2 < 0$)", fontsize=9)
ax.text(-2.9, 2.3, "growing\n($k^2 > 0$)", fontsize=9)
ax.set_xlabel("$x = \\lambda S / m$")
ax.set_ylabel("$3H/|m|$")
ax.set_title("Condensates: oscillating (blue) or growing (red)")
ax.grid(False)
save_figure(fig, "regime_map",
            "Map of the condensates of dirac16complex00 in the plane of "
            ...)
```

The first loop draws the vertical lines of constant w at $x = -1$, 0 and 1 with their labels; the second marks the two condensates of In [6] as black dots at $(x, 3H/|m|) = (0.5, 0.75)$ and $(-0.5, 0.75)$ with their names; two more labels name the regions (`\n` starts a new line). **What Figure 09b.5 shows.** The plane of x (horizontal) and $3H/|m|$ (vertical), both pure numbers: a red wedge with its tip at $x = -1$ on the horizontal axis, opening upwards, where condensates grow, and the blue rest, where they oscillate. The student should see that the line $w = -1$ runs through the middle of the wedge: every condensate with $w = -1$ grows.

```
for name, lam_value in EXAMPLES.items():
    grows = abs(1 + lam_value / m) < 3 * H / abs(m)
    check(grows == (name == "growing"),
          f"the {name} condensate lies in the {name} region of the map")
```

The two checks confirm that the map puts each condensate of In [6] into the region of its name.

In [13], a condensate keeps its energy along the deflating history.

```
A = 1.0  # the deflating history a4 = A H x4
M, k2 = condensate_matrix(m + 0.5 * 1.0, H)  # the oscillating condensate, S0 = 1
# rho at every time; homogeneous_tensor returns (S, U, K4, rho, p, B04): index 3.
# The condensate does not contain a4, so these are its values on this history too.
rho_at_times = np.array([homogeneous_tensor(*condensate(x4, chi, M, k2), 0.5)[3]
                         for x4 in times])
check(np.abs(rho_at_times - rho_at_times[0]).max() < 1e-9,
      "the condensate has p3 = p_t, so its rho is constant along the history")
```

`A = 1` names the deflating history $a_4 = AHx_4$. The oscillating condensate does not contain a_4 (Section 9.18), so it is a solution on this history too. `condensate(...)` returns the pair (Φ, Φ'), and the star in `homogeneous_tensor(*condensate(...), 0.5)` hands the two over as the first two arguments; `[3]` picks the fourth result, ρ. The check confirms that ρ is the same at all 401 times, as $p_3 = p_t$ and the identity of Section 9.10 require.

In [14], toy fluids solved with the Runge-Kutta method (an ILLUSTRATION).

```
def rk4(rate, y0, step, count):
    """Solve dy/dx = rate(y) from y(0) = y0 with count Runge-Kutta steps."""
    y, path = y0, [y0]
    for _ in range(count):
        s1 = rate(y)
        s2 = rate(y + step / 2 * s1)
        s3 = rate(y + step / 2 * s2)
        s4 = rate(y + step * s3)
        y = y + step / 6 * (s1 + 2 * s2 + 2 * s3 + s4)
        path.append(y)
    return np.array(path)
```

The **Runge-Kutta method** of fourth order (RK4, Chapter 2) solves an equation $dy/dx = \text{rate}(y)$ step by step: from the value y it evaluates four slopes (at the start, twice at the middle of the step, and at the end) and moves on by the step times their weighted mean $(s_1 + 2s_2 + 2s_3 + s_4)/6$. The function returns the whole path, the starting value included.

```
STEP, COUNT = 0.05, 160  # x4 from 0 to 8
toy_times = STEP * np.arange(COUNT + 1)
DELTAS = (-2 / 3, -1 / 3, 0.0, 1 / 3, 2 / 3)  # Delta w = w3 - w_t
toy = {}  # Delta w -> rho(x4)/rho0 along A = +1
errors = {STEP: 0.0, STEP / 2: 0.0}  # the largest relative error for two steps
```

160 steps of 0.05 cover x_4 from 0 to 8; `toy_times` holds the 161 times. Five toy fluids are labelled by $\Delta w = w_3 - w_t$. `toy` will hold their paths, and `errors` the largest relative error for the step 0.05 and for half of it.

```
for delta in DELTAS:
    for step in errors:
```

```
        count = round(8.0 / step)  # the number of steps from x4 = 0 to 8
        path = rk4(lambda r, d=delta: -3 * A * H * d * r, 1.0, step, count)
        exact = np.exp(-3 * A * H * delta * step * np.arange(count + 1))
        errors[step] = max(errors[step], np.abs(path / exact - 1).max())
        if step == STEP:
            toy[delta] = path
```

For each toy fluid and each of the two steps, the cell solves $d\rho/dx_4 = -3AH\,\Delta w\,\rho$ from $\rho(0) = 1$ (Section 9.10). `lambda r, d=delta: ...` is a short nameless function of `r`; the extra argument `d=delta` freezes the current value of `delta` inside it. `exact` is the exact solution $e^{-3AH\Delta w\,x_4}$ at the same times, and the largest relative error of the path is kept. The paths with the step 0.05 are stored for the figure.

```
check(errors[STEP] < 1e-7,
      "RK4 (step 0.05) reproduces rho0 exp(-3 A H Delta w x4) for 5 toy fluids")
ratio = errors[STEP] / errors[STEP / 2]  # RK4: half the step, 1/16 of the error
check(14 < ratio < 18, "halving the RK4 step divides the error by about 2^4 = 16")
report("largest relative RK4 error, step 0.05", f"{errors[STEP]:.2e}")
report("error ratio of the steps 0.05 and 0.025", f"{ratio:.2f}")
```

The first check requires the error with the step 0.05 to be below 10^{-7} (it is $1.33 \cdot 10^{-8}$). The error of a fourth-order method is proportional to the fourth power of the step, so halving the step must divide it by about $2^4 = 16$; the measured ratio is 16.17, and the second check accepts 14 to 18.

```
reversed_path = rk4(lambda r: -3 * (-A) * H * (1 / 3) * r, 1.0, STEP, COUNT)
check(np.abs(reversed_path * toy[1 / 3] - 1).max() < 1e-8,
      "A -> -A reverses the energy flow (the product of the two paths is 1)")
report("toy fluid Delta w = 1/3: rho(8)/rho(0) along A = 1", f"{toy[1 / 3][-1]:.6f}")
```

The reversed history ($A \to -A$: the extra times inflate and 3-space deflates) gives the path $e^{+3AH\Delta w\,x_4}$, the inverse of the original: the check requires their product to be 1 at every time. The cell prints $\rho(8)/\rho(0) = 0.135335 = e^{-2}$ for $\Delta w = 1/3$, the number computed by hand in Section 9.10. (`toy[1 / 3][-1]` is the last value of the path; the index -1 counts from the end.)

In [15], the energy exchange (Figure 09b.6).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
fig.subplots_adjust(wspace=0.35)  # more room between the two panels
for delta in DELTAS:
    left.plot(toy_times, toy[delta], label=f"toy, $\\Delta w = {delta:+.2f}$")
left.plot(times[times <= 8.0], rho_at_times[times <= 8.0] / rho_at_times[0], "k:",
          linewidth=2.0, label="condensate ($p_3 = p_t$)")
left.set_yscale("log")
left.set_xlabel("time $x_4$")
left.set_ylabel("$\\rho(x_4) / \\rho(0)$")
left.set_title("Energy density along $a_4 = AHx_4$ ($A = 1$)")
left.legend(fontsize=7)
```

The left panel draws the five toy paths (the format `+.2f` prints the sign and two decimals) and, as a thick black dotted line, the condensate's $\rho(x_4)/\rho(0)$ for the times up to 8 (`times <= 8.0` selects them), on a logarithmic axis.

```
rho_toy = toy[-1 / 3]  # Delta w = -1/3: w3 = 1/3, w_t = 2/3
work_space = -3 * A * H * (1 / 3) * rho_toy  # -3 a4' p3 with p3 = rho/3
work_extra = 3 * A * H * (2 / 3) * rho_toy  # +3 a4' p_t with p_t = 2 rho/3
right.plot(toy_times, work_space, label="3-space: $-3a_4' p_3$")
right.plot(toy_times, work_extra, "--", label="extra times: $+3a_4' p_t$")
right.plot(toy_times, np.gradient(rho_toy, toy_times), ":", color="black",
           label="$d\\rho/dx_4$ (finite differences)")
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("time $x_4$")
right.set_ylabel("rate of change of $\\rho$")
right.set_title("Toy fluid $(w_3, w_t) = (1/3, 2/3)$")
right.legend(fontsize=7)
save_figure(fig, "energy_exchange",
            "Energy exchange along the deflating history $a_4 = AHx_4$, $A = 1$, "
            ...)
```

For the toy fluid with $w_3 = 1/3$ and $w_t = 2/3$ the right panel draws the two terms of $d\rho/dx_4 = -3a_4'p_3 + 3a_4'p_t$ separately, and the slope of the path by finite differences. **What Figure 09b.6 shows.** Left, the energy density divided by its starting value (a pure number, logarithmic axis) against x_4 from 0 to 8: the toy fluids with $p_3 > p_t$ ($\Delta w > 0$) lose energy, those with $p_3 < p_t$ gain energy, $\Delta w = 0$ and the exact condensate stay constant. Right, rates in units of energy per unit volume per unit time: the 3-space term is negative (inflating 3-space takes energy out), the extra-time term is positive and twice as large (the deflating extra times put energy in), and the dotted slope is their sum. The student should see the energy exchange of Section 9.10 at work, remembering that the toy fluids are an illustration, not solutions.

```
# [2:-2] leaves out two points at each end, where np.gradient is less exact.
check(np.abs(np.gradient(rho_toy, toy_times) - (work_space + work_extra))[2:-2].max()
      < 1e-3, "d rho/dx4 equals the sum of the two work terms on the drawn path")
```

The check confirms that the finite-difference slope equals the sum of the two terms within 10^{-3}.

In [16], negative energies: the plane waves of flat space.

```
m_flat, E_flat = 2.0, 5.0
momentum = {0: 4.0, 1: 1.0, 2: 2.0, 3: 0.0}  # k0, k1, k2, k3 (x5, x6, x7: zero)
beta = -1j * g4
h = m_flat * beta + sum(k_a * (-g4 @ gamma[a]) for a, k_a in momentum.items())
check(np.abs(h - h.conj().T).max() < 1e-12 and np.abs(h @ h - 25 * I16).max() < 1e-12
      and np.abs(B @ h - h @ B).max() < 1e-12,
      "h is Hermitian, h^2 = 25, and h commutes with B")
```

The cell sets up the example of Section 9.21: $m = 2$, the momenta $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ stored in a dictionary from the index of the direction to its momentum, $\beta = -i\gamma^{(4)}$ and $h = m\beta + \sum_a k_a \alpha^a$ with $\alpha^a = -\gamma^{(4)}\gamma^{(a)}$. The check confirms that h is **Hermitian** (equal to its own dagger, `h.conj().T`), that $h^2 = 25$ and that $hB = Bh$.

```
energies, vectors = np.linalg.eigh(h)  # eigenvalues in ascending order
U5 = vectors[:, energies > 0]  # the eight columns with eigenvalue +5
form = U5.conj().T @ B @ U5  # u^dagger B u on the eigenspace, as an 8 x 8 matrix
inertia, directions = np.linalg.eigh(form)
```

`np.linalg.eigh` returns the eigenvalues of a Hermitian matrix in ascending order and, as the columns of a second matrix, unit eigenvectors that are perpendicular to each other. `energies > 0` selects the eight columns with $E = +5$: `U5` is a 16×8 matrix whose columns span the space of solutions with $E = 5$. Every unit column of that space is $u = U_5 c$ with a unit column c of eight numbers, and then $u^\dagger B u = c^\dagger (U_5^\dagger B U_5) c$: the 8×8 matrix `form` describes the form $u^\dagger B u$ on the space. Its eigenvalues `inertia` and unit eigenvectors `directions` give its positive and negative directions.

```
PAIRING = "Revision/pairing/reports/python-pairing.json"
check(np.allclose(energies, [-5.0] * 8 + [5.0] * 8, atol=1e-12)
      and np.allclose(inertia, [-1.0] * 4 + [1.0] * 4, atol=1e-12)
      and recorded(PAIRING, "Q.one_particle_Krein_inertia_proof") == "PASS",
      "E = +-5 (eight each); u^dagger B u has inertia (4,4) on the E = 5 space",
      record=f"{PAIRING}, check Q.one_particle_Krein_inertia_proof (every real "
             "frequency)")
```

The check confirms the frequencies ± 5, eight each, and that the form has four eigenvalues -1 and four $+1$: Krein inertia (4,4), which the pairing record proves for every real frequency.

In [17], the energy density of two plane waves.

```
def flat_energy_density(u):
    """rho = -sum over a != x4 of K_a + m S for the plane wave u e^(i(k.x - E x4))."""
    S = (u.conj() @ C @ u).real
    kinetic = 0.0
    for a, k_a in momentum.items():  # d_a Phi = i k_a Phi
        d_u = 1j * k_a * u
        kinetic += 0.5 * (u.conj() @ C @ gamma[a] @ d_u - d_u.conj() @ C @ gamma[a] @ u)
    return (-kinetic + m_flat * S).real
```

The function computes $\rho = -\sum_{a \neq x_4} K_a + mS$ from its definition (Section 9.7, flat space, $U = 0$), with $\partial_a \Phi = i k_a \Phi$; the common exponential factor has size 1 and drops out of every bilinear, so the column u can stand for Φ.

```
rho_pair = []
# column 0 of directions has the eigenvalue -1 of the form, column 7 the value +1
for column, b_value in ((0, -1.0), (7, 1.0)):
    u = U5 @ directions[:, column]  # a unit vector of the eigenspace, B u = b_value u
    d4_u = -1j * E_flat * u  # d4 Phi = -i E Phi
    field = (g4 @ d4_u + sum(gamma[a] @ (1j * k_a * u) for a, k_a in momentum.items())
             - m_flat * u)  # sum_a gamma^(a) d_a Phi - m Phi
    check(np.abs(field).max() < 1e-12 and np.abs(B @ u - b_value * u).max() < 1e-12,
          f"the plane wave with B u = {b_value:+.0f} u solves the field equation")
    rho_pair.append(flat_energy_density(u))
```

For the two extreme directions of the form (the first eigenvector, eigenvalue -1, and the last, eigenvalue $+1$) the loop builds the unit column u of the solution space, checks that the plane wave solves the flat field equation $\sum_a \gamma^{(a)} \partial_a \Phi = m \Phi$ and that $B u = \mp u$, and computes its energy density.

```
SCOPE_PY = "Revision/theory/reports/python-scope.json"
SCOPE_WL = "Revision/theory/reports/wolfram-scope.json"
check(abs(rho_pair[0] + 5.0) < 1e-12 and abs(rho_pair[1] - 5.0) < 1e-12
      and recorded(SCOPE_PY, "commuting_field_energy_unbounded_below") == "PASS"
      and recorded(SCOPE_WL, "commuting_field_energy_unbounded_below") == "PASS",
      "rho = -5 for B u = -u and rho = +5 for B u = +u (|u| = 1)",
      record=f"{SCOPE_PY} and {SCOPE_WL}, check commuting_field_energy_unbounded_below")
```

The check confirms the record's values: the wave with $Bu = -u$ has $\rho = -5$, the wave with $Bu = +u$ has $\rho = +5$, both at the positive frequency $E = 5$, in agreement with $\rho = E\,u^\dagger Bu$ of Section 9.21.

In [18], 5000 random waves of positive frequency.

```
samples = rng.normal(size=(5000, 8)) + 1j * rng.normal(size=(5000, 8))
samples /= np.linalg.norm(samples, axis=1)[:, None]
us = samples @ U5.T  # 5000 random unit vectors of the eigenspace
charges = np.einsum("ni,ij,nj->n", us.conj(), B, us).real  # u^dagger B u
rho_samples = np.array([flat_energy_density(u) for u in us])
check(np.abs(rho_samples - E_flat * charges).max() < 1e-12
      and rho_samples.min() < 0 < rho_samples.max(),
      "rho = E u^dagger B u for all 5000 random u of the eigenspace; both signs")
report("smallest and largest rho of the 5000 samples",
       f"{rho_samples.min():.4f} and {rho_samples.max():.4f}")
```

5000 random unit columns c of eight numbers give 5000 random unit columns $u = U_5 c$ of the solution space (`samples @ U5.T` computes all of them at once, one per row). For each, `charges` holds $u^\dagger Bu$ and `rho_samples` the energy density computed from the kinetic terms. The check confirms $\rho = E\,u^\dagger Bu$ every time and that both signs occur. The smallest and largest values are -4.3821 and 4.4892.

In [19], the histogram of the 5000 energy densities (Figure 09b.7).

```
fig, ax = plt.subplots()
ax.hist(rho_samples, bins=50, range=(-5.0, 5.0), color="tab:purple", edgecolor="white")
ax.axvline(0.0, color="black", linewidth=1.0)
ax.set_xlabel("energy density $\\rho = E\\,u^\\dagger B u$ of the plane wave")
ax.set_ylabel("number of the 5000 random $u$")
ax.set_title("Waves of positive frequency $E = 5$: $\\rho$ has both signs")
save_figure(fig, "energy_sign",
            "Histogram of the energy density $\\rho = -\\sum_{a \\neq x_4} K_a + mS$ "
            ...)
```

What Figure 09b.7 shows. The number of waves (vertical) whose energy density, in units of energy per unit volume, falls into each of 50 intervals between -5 and 5 (horizontal). The values are spread symmetrically about zero, and about half are negative. The possible values fill the whole interval from -5 (reached for $Bu = -u$) to 5 (for $Bu = u$), but random columns seldom come near the two ends. The student should see that for this commuting field a positive frequency does not mean a positive energy.

In [20], the last check.

```
captions = json.loads(output_file(CAPTION_FILE).read_text(encoding="utf-8"))
expected_files = ["09b_1_sign_of_s.png", "09b_2_condensate_solutions.png",
                  "09b_3_energy_and_pressure.png", "09b_4_equation_of_state.png",
                  "09b_5_regime_map.png", "09b_6_energy_exchange.png",
                  "09b_7_energy_sign.png"]
check(sorted(captions) == expected_files and all(
    output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected_files),
    "the seven figures of this notebook are saved and captioned")
all_checks_passed()
```

As in Notebook 09a: the seven figures exist and are captioned, and the last line is `ALL 33 CHECKS PASSED (notebook 09b)`: two checks in In [2], one in In [4], five in In [6], seven in In [8], one in In [9], three in In [10], two in In [12], one in In [13], three in In [14], one in In [15], two in In [16], three in In [17], one in In [18] and one in In [20].

9.26 The full tensor of a condensate

A condensate has the same pressure in all seven directions. That does not yet make it a **perfect fluid at rest**, which has a diagonal energy-momentum tensor: Section 9.12 showed that the spin connection puts entries off the diagonal. For a condensate $\partial_\mu \Phi = 0$ for $\mu \neq x_4$, but $D_\mu \Phi = \Omega_\mu \Phi$ is not zero along 3-space and the extra times, because Ω_{x_i} and Ω_{x_t} are not zero.

The structure, from the record. Off the diagonal the tensor of Section 9.6 is $T^\nu{}_\mu = -K^{(\nu}{}_{\mu)}$ with

$$K^\nu{}_\mu = \tfrac{1}{2}\big(\bar{\Phi}\gamma^\nu D_\mu \Phi - D_\mu \bar{\Phi}\,\gamma^\nu \Phi\big), \qquad K^{(\nu}{}_{\mu)} = \tfrac{1}{2}\big(K^\nu{}_\mu + g^{\nu\nu} g_{\mu\mu} K^\mu{}_\nu\big),$$

because the first pair of terms in the bracket of Section 9.6 is $2K^\nu{}_\mu$ and the second pair is $g_{\mu\mu} g^{\nu\nu} \cdot 2K^\mu{}_\nu$ (move the metric factors of γ_μ and D^ν in front). The Revision record proves three facts for every exact condensate (PROVED; the table names the record files and checks; the coefficients themselves are stored in the record file `Revision/field_equations_a4/a4-equations.json` under the entry `offDiagonalKinetic`):

statement	Revision report	check
the diagonal of the tensor of a condensate	`Revision/field_equations_a4/reports/wolfram-a4-report.json`	`condensate_kinetic_tensor_diagonal`
the same, with the author's gammas	`Revision/field_equations_a4/reports/python-a4-report.json`	`authorT16_condensate_kinetic_diagonal`
the entries off the diagonal are multiples of three-gamma bilinears	`Revision/field_equations_a4/reports/wolfram-a4-report.json`	`condensate_offdiagonal_are_three_gamma_bilinears`

statement	Revision report	check
the same, with the author's gammas	`Revision/field_equations_a4/reports/python-a4-report.json`	`authorT16_condensate_offdiagonal_three_gamma`
the stored coefficients are the computed ones	`Revision/field_equations_a4/reports/python-a4-report.json`	`json_offdiagonal_coefficients`

The three facts are:

- the diagonal is $T^{x_4}{}_{x_4} = -VS + L_0$ and $T^{\mu}{}_{\mu} = L_0$ otherwise (Section 9.19), and $T^{x_4}{}_{x_0} = T^{x_0}{}_{x_4} = 0$;
- exactly 42 of the 56 entries off the diagonal are nonzero in general: the 12 flows $T^{x_4}{}_{x_j}$ and $T^{x_j}{}_{x_4}$ (x_j any of the six directions $x_1, x_2, x_3, x_5, x_6, x_7$), whose coefficients contain H and not a_4'; the 18 entries $T^{x_i}{}_{x_t}$ and $T^{x_t}{}_{x_i}$ between 3-space and the extra times; and the 12 entries $T^{x_j}{}_{x_0}$ and $T^{x_0}{}_{x_j}$; the last 30 are proportional to a_4' (the 14 entries inside 3-space, inside the extra times, and x_4-x_0 vanish);
- each nonzero entry is a fixed number (depending on a_4, a_4', z and H, not on V) times one of 15 **three-gamma bilinears** $B_{abc} = \bar{\Phi}\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$, with $\{a,b,c\} = \{j,4,8\}$ (six of them) or $\{i,4,t\}$ with i in 3-space and t an extra time (nine of them).

One entry, line by line. Take $T^{x_1}{}_{x_5}$. Both $K^{x_1}{}_{x_5}$ and $K^{x_5}{}_{x_1}$ contain no derivative of Φ (there is none along x_1 or x_5), so, exactly as in Section 9.12, each is $\frac{1}{2}\bar{\Phi}\{\gamma^{\nu}, \Omega_{\mu}\}\Phi$. With $\gamma^{x_1} = \gamma^{(1)}/f_1$, $\gamma^{x_5} = \gamma^{(5)}/f_5$ and the anticommutators of a gamma with a product of two other gammas, $\{\gamma^{(a)}, \gamma^{(b)}\gamma^{(c)}\} = 2\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}$ (move $\gamma^{(a)}$ from the right end to the front past two gammas: two minus signs):

$$
\begin{aligned}
K^{x_1}{}_{x_5} &= \tfrac{1}{2}\cdot\frac{1}{f_1}\cdot\Big(-\frac{f_5}{2}\Big)\,\bar{\Phi}\big(a_4'\{\gamma^{(1)},\gamma^{(4)}\gamma^{(5)}\} + H\{\gamma^{(1)},\gamma^{(5)}\gamma^{(0)}\}\big)\Phi \\
&= -\frac{f_5}{2f_1}\big(a_4' B_{145} + H B_{158}\big), \\
K^{x_5}{}_{x_1} &= \tfrac{1}{2}\cdot\frac{1}{f_5}\cdot\frac{f_1}{2}\,\bar{\Phi}\big(a_4'\{\gamma^{(5)},\gamma^{(1)}\gamma^{(4)}\} + H\{\gamma^{(5)},\gamma^{(1)}\gamma^{(0)}\}\big)\Phi \\
&= \frac{f_1}{2f_5}\big(a_4' B_{145} - H B_{158}\big), \\
g^{11}g_{55}K^{x_5}{}_{x_1} &= \frac{1}{f_1^2}\big(-f_5^2\big)\frac{f_1}{2f_5}\big(a_4' B_{145} - H B_{158}\big) = -\frac{f_5}{2f_1}\big(a_4' B_{145} - H B_{158}\big), \\
T^{x_1}{}_{x_5} &= -\tfrac{1}{2}\Big(K^{x_1}{}_{x_5} + g^{11}g_{55}K^{x_5}{}_{x_1}\Big) = \tfrac{1}{2}\cdot\frac{f_5}{2f_1}\cdot 2a_4' B_{145} = \tfrac{1}{2}\,e^{-2a_4}\,a_4'\,\bar{\Phi}\gamma^{(1)}\gamma^{(4)}\gamma^{(5)}\Phi.
\end{aligned}
$$

The first line inserts $\gamma^{x_1} = \gamma^{(1)}/f_1$ and Ω_{x_5}; the second replaces each anticommutator by twice the product of its three gammas, which are already in the order $1,4,5$ and $1,5,8$, and cancels

the factors 2 and $\frac{1}{2}$. The third line inserts $\gamma^{x_5} = \gamma^{(5)}/f_5$ and Ω_{x_1}; the fourth does the same and reorders the three gammas into the order $1,4,5$ and $1,5,8$ ($\gamma^{(5)}\gamma^{(1)}\gamma^{(4)} = \gamma^{(1)}\gamma^{(4)}\gamma^{(5)}$ after two exchanges; $\gamma^{(5)}\gamma^{(1)}\gamma^{(0)} = -\gamma^{(1)}\gamma^{(5)}\gamma^{(0)}$ after one). The fifth multiplies by $g^{11}g_{55} = (1/f_1^2)(-f_5^2)$; the sixth adds, so that the H terms cancel and the a_4' terms double, and uses $f_5/f_1 = e^{-2a_4}$. The entry is proportional to the deflation rate a_4': it exists only because the extra times deflate (or, for $a_4' < 0$, inflate). The coefficient $\frac{1}{2}e^{-2a_4}a_4'$ is exactly the record's (its table lists $K^{x_i}{}_{x_t}$ with the coefficient $-\frac{1}{2}e^{-2a_4}a_4'$, and $T = -K$); at $a_4 = 0.5$, $a_4' = 0.25$ it is $0.5 \cdot e^{-1} \cdot 0.25 = 0.045985$, the number Notebook 09c prints for $T^{x_1}{}_{x_5}$.

A second entry: the momentum flow $T^{x_4}{}_{x_1}$. Section 9.12 computed the piece $K^{x_4}{}_{x_1} = \frac{1}{2}f_1 H\,\bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi$, with the coefficient $\frac{1}{2}$. The record's coefficient of the whole entry is $\frac{7}{4}$. The difference is the second piece of the symmetrised tensor, $g^{44}g_{11}K^{x_1}{}_{x_4}$, which contains the time derivative of the condensate. Line by line, with $\partial_4\Phi = M\Phi$ and $\partial_4\bar\Phi = -\bar\Phi M$ (Section 9.19) and $\Omega_{x_4} = 0$, so that $D_{x_4} = \partial_4$:

$$
\begin{aligned}
K^{x_4}{}_{x_1} &= \tfrac{1}{2}f_1 H\,\bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi = -\tfrac{1}{2}f_1 H\,B_{148},\\
K^{x_1}{}_{x_4} &= \tfrac{1}{2}\big(\bar\Phi\gamma^{x_1}M\Phi + \bar\Phi M\gamma^{x_1}\Phi\big) = \frac{1}{2f_1}\,\bar\Phi\{\gamma^{(1)}, M\}\Phi,\\
\{\gamma^{(1)}, M\} &= -V\{\gamma^{(1)},\gamma^{(4)}\} + 3H\{\gamma^{(1)},\gamma^{(4)}\gamma^{(0)}\} = 0 + 6H\,\gamma^{(1)}\gamma^{(4)}\gamma^{(0)},\\
K^{x_1}{}_{x_4} &= \frac{3H}{f_1}\,B_{148}, \qquad g^{44}g_{11}K^{x_1}{}_{x_4} = (-1)\,f_1^2\cdot\frac{3H}{f_1}\,B_{148} = -3f_1 H\,B_{148},\\
T^{x_4}{}_{x_1} &= -\tfrac{1}{2}\Big(K^{x_4}{}_{x_1} + g^{44}g_{11}K^{x_1}{}_{x_4}\Big) = -\tfrac{1}{2}\Big(-\tfrac{1}{2}-3\Big)f_1 H\,B_{148} = \tfrac{7}{4}\,e^{a_4}\sin^{1/6}z\;H\,B_{148}.
\end{aligned}
$$

The first line is the result of Section 9.12, with $\gamma^{(4)}\gamma^{(1)}\gamma^{(0)} = -\gamma^{(1)}\gamma^{(4)}\gamma^{(0)}$ (one exchange). The second inserts the derivatives of the condensate into the definition of $K^{x_1}{}_{x_4}$ and $\gamma^{x_1} = \gamma^{(1)}/f_1$. The third inserts $M = -V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}$: two different gammas anticommute, so $\{\gamma^{(1)},\gamma^{(4)}\} = 0$, and the anticommutator with a product of two other gammas is twice the product of the three. The fourth collects, and multiplies by $g^{44}g_{11} = (1/g_{44})\,g_{11} = (-1)f_1^2$. The fifth adds the two pieces, with $-\frac{1}{2}(-\frac{1}{2}-3) = \frac{7}{4}$, and writes $f_1 = e^{a_4}\sin^{1/6}z$. Three things follow. The effective mass V has dropped out. The coefficient contains H and not a_4', so the 12 momentum flows remain when $a_4' = 0$. And the larger part, 3 of the $\frac{7}{2}$ inside the bracket, comes from the term $3H\gamma^{(4)}\gamma^{(0)}$ of M, that is from the gravitational term $3H\gamma^{(0)}$ of the field equation (Fact 2 of Section 9.3). At $a_4 = 0.5$, $z = \pi/4$, $H = 1$ the coefficient is $\frac{7}{4}\cdot 1.6487213\cdot 0.9438743 = 2.723325$ (rounded to six decimals); the record writes the symmetrised K with the coefficient -2.723325, and Notebook 09c prints `T^x4_x1 = -(-2.723325) x its bilinear`.

Condensates with a diagonal tensor. The record also constructs exact condensates for which all 15 bilinears vanish, so that their tensor is diagonal at every point of every history: a perfect fluid at rest, with $T^{x_4}{}_{x_4} = -VS + L_0$ and pressure L_0. The construction uses the three matrices $P_1 = \gamma^{(1)}\gamma^{(5)}$, $P_2 = \gamma^{(2)}\gamma^{(6)}$, $P_3 = \gamma^{(3)}\gamma^{(7)}$, which pair each 3-space direction with an extra time. Each squares to 1 ($\gamma^{(1)}\gamma^{(5)}\gamma^{(1)}\gamma^{(5)} = -\gamma^{(1)}\gamma^{(1)}\gamma^{(5)}\gamma^{(5)} = -(1)(-1) = 1$), they commute with each other and with M (moving a product of two gammas past another gamma with a different index costs two minus signs), and each anticommutes with C ($C = \gamma^{(0)}\gamma^{(1)}\gamma^{(2)}\gamma^{(3)}$ contains $\gamma^{(k)}$ but not $\gamma^{(k+4)}$, so moving P_k through C costs three minus signs for $\gamma^{(k)}$ and four for $\gamma^{(k+4)}$). When $V^2 > 9H^2$, let $\omega = \sqrt{V^2 - 9H^2}$; then $M^2 = -\omega^2$ (Section 9.18, with $k^2 = 9H^2 - V^2 = -\omega^2$). Four steps build the condensate.

- **Step 1: two spaces of dimension 8.** For any column u put $u_- = \frac{1}{2}(u + \frac{i}{\omega}Mu)$ and $u_+ = \frac{1}{2}(u - \frac{i}{\omega}Mu)$. Then $u_- + u_+ = u$, and $Mu_- = \frac{1}{2}(Mu + \frac{i}{\omega}M^2u) = \frac{1}{2}(Mu - i\omega u) = -i\omega\, u_-$ (insert $M^2 = -\omega^2$, then take out the factor $-i\omega$, using $-i\omega \cdot \frac{i}{\omega} = 1$); in the same way $Mu_+ = +i\omega\, u_+$. So every column is the sum of a column of the space E_- of the eigenvalue $-i\omega$ and a column of the space E_+ of the eigenvalue $+i\omega$; a nonzero column cannot lie in both (it cannot have two different eigenvalues), so the dimensions of E_- and E_+ add up to 16. M is real, so for $Mu = -i\omega u$ the complex conjugate column $\bar{u}$ obeys $M\bar{u} = \overline{Mu} = \overline{-i\omega u} = +i\omega\,\bar{u}$: complex conjugation carries E_- into E_+ and E_+ into E_-, and it keeps independent columns independent, so the two spaces have the same dimension, $16/2 = 8$.

- **Step 2: eight sectors.** P_1, P_2, P_3 commute with M, so they map E_- into itself (if $Mu = -i\omega u$ then $M(P_k u) = P_k Mu = -i\omega\, P_k u$). Because $P_k^2 = 1$, the matrix $\frac{1}{2}(1 + \varepsilon P_k)$, for a sign $\varepsilon = \pm 1$, keeps the columns with $P_k u = \varepsilon u$ and sends those with $P_k u = -\varepsilon u$ to zero (it is a **projector**), and $\frac{1}{2}(1 + \frac{i}{\omega}M)$, which turns u into u_-, keeps E_- and sends E_+ to zero. For three signs $\varepsilon_1, \varepsilon_2, \varepsilon_3$ the product $\Pi = \frac{1}{2}(1 + \frac{i}{\omega}M) \cdot \frac{1}{2}(1 + \varepsilon_1 P_1) \cdot \frac{1}{2}(1 + \varepsilon_2 P_2) \cdot \frac{1}{2}(1 + \varepsilon_3 P_3)$ of these four commuting projectors keeps exactly the columns of E_- with $P_1 = \varepsilon_1$, $P_2 = \varepsilon_2$, $P_3 = \varepsilon_3$ (a **sector** of E_-; there are $2^3 = 8$ of them) and sends every other column to zero; it obeys $\Pi^2 = \Pi$.

- **Step 3: every sector has dimension 1.** For a matrix with $\Pi^2 = \Pi$ every column splits as $u = \Pi u + (u - \Pi u)$, a part that Π keeps ($\Pi\Pi u = \Pi u$) and a part that it sends to zero ($\Pi(u - \Pi u) = 0$). In a basis made of kept and of removed columns Π is diagonal with entries 1 and 0, so its **trace** (the sum of its diagonal entries, which is the same in every basis because $\mathrm{tr}(AB) = \mathrm{tr}(BA)$) is the dimension of the kept space. Multiply out Π: it is $\frac{1}{16}$ times the unit matrix plus terms that are each a number times a product of 1 to 8 different frame gammas (the indices of P_k are k and $k+4$, those of $M = -V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}$ are 4 and 8: all different). A product X of n different frame gammas has trace 0: choose a gamma $\gamma^{(b)}$ among its factors if n is even, and one that is not among them if n is odd (then $n \le 7$, so one is left); moving $\gamma^{(b)}$ through X costs $n - 1$ minus signs in the first case and n in the second, an odd number either way, so $X\gamma^{(b)} = -\gamma^{(b)}X$; then $\mathrm{tr}X = \mathrm{tr}\big(X\gamma^{(b)}(\gamma^{(b)})^{-1}\big) = \mathrm{tr}\big((\gamma^{(b)})^{-1}X\gamma^{(b)}\big) = -\mathrm{tr}\big((\gamma^{(b)})^{-1}\gamma^{(b)}X\big) = -\mathrm{tr}X$ (the inverse exists because $(\gamma^{(b)})^2 = \pm 1$; the second step is $\mathrm{tr}(AB) = \mathrm{tr}(BA)$), and a number equal to minus itself is 0. Only the unit matrix contributes, $\mathrm{tr}\,\Pi = 16/16 = 1$, and every one of the eight sectors of E_- has dimension 1.

- **Step 4: the condensate.** Take a unit column v_1 (a column of length 1) in the sector $P_1 = P_2 = P_3 = -1$ and a unit column v_2 in the sector $P_1 = P_2 = P_3 = +1$. By Step 3 each is unique up to a factor $e^{i\theta}$ of size 1. Put $c = \overline{v_1^\dagger C v_2}$ (the bar over a number means its complex conjugate) and $\Phi_0 = v_1 + cv_2$, a column of E_-. Other choices of the factors change nothing that matters: replacing v_1 by $e^{i\alpha}v_1$ and v_2 by $e^{i\beta}v_2$ replaces c by $e^{i(\alpha-\beta)}c$ and Φ_0 by $e^{i\alpha}\Phi_0$, which leaves every bilinear unchanged. Since $M\Phi_0 = -i\omega\Phi_0$, the column $\Phi = e^{-i\omega x_4}\Phi_0$ solves $\partial_4\Phi = M\Phi$: an exact condensate that oscillates with the angular frequency ω.

A **witness** is an explicit example that proves that something exists. For three choices of (V, H) the record builds this condensate with exact arithmetic and proves that all 15 of its bilinears vanish (and with them every off-diagonal entry, for every a_4 and a_4'); each of the three is a witness: it proves that condensates with a diagonal tensor exist. Notebook 09c builds the same three and confirms numerically the dimensions 8 and 1 of Steps 1 and 3. The record's three witnesses

have $(V, H) = (5, 1)$, $(5, 4/3)$ and $(-5, 1)$, so $\omega = \sqrt{25-9} = 4$, $\sqrt{25-16} = 3$ and 4 (PROVED; check `condensate_diagonal_witness_exact`; the record writes the effective mass V as M and the frequency as w). Their S is never negative: C anticommutes with P_1, which is real and symmetric, so for $P_1 v_1 = -v_1$

$$v_1^\dagger C v_1 = -v_1^\dagger C P_1 v_1 = v_1^\dagger P_1 C v_1 = (P_1 v_1)^\dagger C v_1 = -v_1^\dagger C v_1,$$

hence $v_1^\dagger C v_1 = 0$ (a number equal to minus itself is zero), and likewise $v_2^\dagger C v_2 = 0$; with $s = v_1^\dagger C v_2$ one has $v_2^\dagger C v_1 = \bar{s}$ (C is real and symmetric) and $c = \bar{s}$, so $S = \Phi_0^\dagger C \Phi_0 = 0 + c\,s + \bar{c}\,\bar{s} + 0 = 2|s|^2 \geq 0$. (The first step replaces v_1 by $-P_1 v_1$, the second uses $CP_1 = -P_1 C$, the third $P_1^\dagger = P_1$, the fourth $P_1 v_1 = -v_1$.)

What this means for the field equations of gravity. The field equations of a_4 (Chapter 12) have one equation for each of the 64 entries. For every entry off the diagonal the gravitational side (the left-hand side) vanishes identically, so the equation reads $0 = \kappa T^\nu{}_\mu$, and for a coupling $\kappa \neq 0$ every off-diagonal entry of the source must vanish (PROVED; the table below names the record's entries and checks). For a condensate the record turns this into conditions on the 15 bilinears: the six B_{j48} must vanish, and on a deflating history ($a_4' \neq 0$) the nine B_{i4t} as well. So on a deflating history a condensate meets the off-diagonal conditions only if all 15 bilinears vanish, as for the witnesses.

statement	Revision record	entry or check
the off-diagonal equations read $0 = \kappa T^\nu{}_\mu$	`Revision/field_equations_a4/a4-equations.json`	`generalSource.offDiagonal_x4x0`, `generalSource.offDiagonal_x0x4`, `generalSource.offDiagonal_other`
their left-hand sides vanish identically	`Revision/field_equations_a4/reports/wolfram-a4-report.json`	`independent_components`
the same, verified independently with sympy	`Revision/field_equations_a4/reports/python-a4-report.json`	`other_components_vanish`
the conditions on the 15 bilinears of a condensate	`Revision/field_equations_a4/a4-equations.json`	`fields.dirac16complex00.offDiagonalConditions`
the evolution equation $a_4'' F(a_4') = \kappa(p_3 - p_t)$	`Revision/field_equations_a4/a4-equations.json`	`generalSource.evolution_x1_minus_x5`, `generalSource.evolution_F`
the left-hand side factorises as $a_4'' F(a_4')$	`Revision/field_equations_a4/reports/wolfram-a4-report.json`	`evolution_factorises_a4pp_times_F`
F is not the zero polynomial unless $\alpha_1 = \alpha_2 = \alpha_3 = 0$	both reports of the folder `Revision/field_equations_a4/reports`	`evolution_F_not_identically_zero`

statement	Revision record	entry or check
the theorem of the linear history	`Revision/field_equations_a4/a4-equations.json`	`fields.dirac16complex00.theoremLinear`
the linear history: A enters only through even powers	`Revision/field_equations_a4/a4-equations.json`	`linearMember`

The theorem of the linear history (PROVED; record entry `theoremLinear`, see the table). Its hypotheses are four: (1) Φ is a homogeneous condensate of the commuting field that solves its field equation; (2) it meets the off-diagonal conditions above; (3) a_4 is twice continuously differentiable (a_4' and a_4'' exist and are continuous functions of x_4); (4) the three Lovelock couplings $\alpha_1, \alpha_2, \alpha_3$ of Chapter 12 are not all zero. Its conclusion: if the author's metric with this a_4 solves the field equations of gravity with the condensate as the source, then $a_4 = AHx_4 + a_0$ with real constants A and a_0. The argument uses, besides (3) and (4), the equality $p_3 = p_t$ of a condensate and one equation of the record:

- the evolution equation, the 3-space equation minus the extra-time equation, is $a_4'' F(a_4') = \kappa(p_3 - p_t)$ with $F = 2\alpha_1 - 48(a_4')^2\alpha_2 + 720(a_4')^4\alpha_3 - 80\alpha_2H^2 + 864(a_4')^2\alpha_3H^2 + 720\alpha_3H^4$ (see the table);
- a condensate has $p_3 = p_t$ (Section 9.19: all seven pressures equal $SU' - U$), so $a_4'' F(a_4') = 0$ at every time;
- F is a polynomial in a_4' that is not the zero polynomial: its coefficient of $(a_4')^4$ is $720\alpha_3$; if $\alpha_3 = 0$, its coefficient of $(a_4')^2$ is $-48\alpha_2$; if $\alpha_2 = \alpha_3 = 0$, its constant term is $2\alpha_1$; by (4) one of these is not zero, so F has at most four roots (see the table);
- suppose $a_4'' \neq 0$ at some time; since a_4'' is continuous it stays different from zero on a small interval around that time, where a_4' is then strictly increasing or strictly decreasing and takes infinitely many different values; but $a_4'' F(a_4') = 0$ forces $F(a_4') = 0$ on that interval, so F would have infinitely many roots, which is impossible; hence $a_4'' = 0$ at every time;
- so a_4' is a constant, which we write AH, and integrating once more gives $a_4 = AHx_4 + a_0$.

Deflation is a choice of sign. The equations of the linear history contain A only through A^2, A^4 and A^6 (record entry `linearMember`), so they do not change when A is replaced by $-A$: extra times that deflate ($A > 0$, the author's history), extra times that inflate ($A < 0$) and the static case ($A = 0$) are allowed on exactly the same footing. That the author's extra times deflate is a choice of sign, an initial condition; the equations do not select it (record entry `theoremLinear`; Chapter 12).

What is and is not known about a condensate as the source. The Revision record contains no check that combines a witness with the remaining equations of gravity for definite values of κ, Λ, m, λ and A. Chapter 12 makes that combination for Einstein gravity: Notebook 12d constructs, with exact arithmetic, condensates whose 15 bilinears vanish and chooses m, λ and Λ so that all the Einstein equations hold on the deflating history $a_4 = Hx_4$; that is a computation of the book under stated ASSUMED inputs, not a Revision record, and Chapter 12 gives its status. For Einstein-Lovelock gravity with α_2 or α_3 different from zero no such solution is constructed, in the record or in this book (OPEN).

Conservation of the full tensor. The Noether identity proves $\nabla_\mu T^\mu{}_\nu = 0$ for every solution, the 42 off-diagonal entries included (Section 9.9). Notebook 09c tests it numerically on a condensate along the curved history $a_4 = 0.3x_4 + 0.05x_4^2$, with a **negative control**: a configuration built so that the test must fail, which shows that the test can detect an error. The control is the condensate of an equation without the gravitational term $3H\gamma^{(0)}$ ($M = -V\gamma^{(4)}$). Its S is constant and its $K_4 = \frac{1}{2}\bar{\Phi}\{\gamma^{(4)}, M\}\Phi = \frac{1}{2}(-V)(-2)S = VS$ is the same as for the true condensate, so its energy density and pressures, and with them the energy balance (x_4) and the hidden balance (x_0), are unchanged; but it does not solve the field equation of the author's metric, and its six momentum balances along 3-space and the extra times fail.

9.27 Example: the full tensor of a condensate

The third worked example computes all 64 entries of the tensor of exact condensates. It shows the tensor of a generic condensate (effective mass $V = 5$, $H = 1$, at the point $a_4 = 0.5$, $a_4' = 0.25$, $z = \pi/4$) as a heat map and counts its nonzero off-diagonal entries (42, and 12 when $a_4' = 0$); shows how three entries depend on a_4'; checks that each of the 42 entries is a fixed multiple of one of 15 bilinears and that the multiple is exactly minus the record's coefficient; builds the record's three witnesses, checks the dimensions 8 and 1 of their construction, checks that their tensor is diagonal at 36 points and that their $S = 2|v_1^\dagger C v_2|^2$, and reproduces the record's frequencies and values of S; and tests the conservation of the full tensor with finite differences along a curved history, with the negative control. In this notebook the matrix of the condensate equation is called $A = -\gamma^{(4)}(V - 3H\gamma^{(0)})$; it is the matrix M of Section 9.18 (multiply out: $-V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}$), and it has nothing to do with the constant A of the history $a_4 = AHx_4$. The angular frequency of the witnesses is called ω (in the code `freq`), to keep it apart from the equation of state w. The notebook draws five figures.

9.28 How to run Notebook 09c

Step 1. What this notebook does and what it needs.

Notebook 09c (The full energy-momentum tensor of a condensate) is the file `Revision/textbook/notebooks/09c_condensate_tensor.ipynb` of the repository Dirac_claude. It computes all 64 entries of the energy-momentum tensor of exact condensates of the commuting field dirac16complex00 in the author's metric, including the off-diagonal entries that the spin connection produces, finds the 42 nonzero off-diagonal entries of a generic condensate and checks that each is a multiple of one of 15 three-gamma bilinears with exactly the coefficient that the record lists for it, builds the exact condensates whose tensor is diagonal and reproduces their frequencies omega and their S from the record (S up to one common factor), and tests the conservation of the full tensor along a curved history with finite differences of shrinking step, with a negative control that leaves out the gravitational term of the field equation; five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 09c_condensate_tensor.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 09c_condensate_tensor.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 09c_condensate_tensor.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/09c.captions.json`
- `Revision/textbook/figures/09c_1_generic_condensate.png`
- `Revision/textbook/figures/09c_2_offdiagonal_entries.png`
- `Revision/textbook/figures/09c_3_witness_condensate.png`
- `Revision/textbook/figures/09c_4_conservation_convergence.png`
- `Revision/textbook/figures/09c_5_divergence_components.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the five figures of this notebook are saved and captioned
ALL 25 CHECKS PASSED (notebook 09c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/09c_condensate_tensor.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 09c_condensate_tensor.ipynb
python -m nbconvert --execute --inplace 09c_condensate_tensor.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "KeyError" with the words "has no check": the notebook asks a Revision report for the verdict of one of its checks, and the report in your copy of the repository does not contain that check: your copy is older or newer than the notebook. Run `git pull` in the repository folder, then run the notebook again.

9.29 Notebook 09c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/09c_condensate_tensor.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A condensate (a field that depends on the time $x_4$ only) has the same energy
density and pressure in every direction. That does not yet make it a *perfect
fluid*: a perfect fluid at rest has a diagonal energy-momentum tensor, and the
spin connection of the author's metric puts entries off the diagonal. This
notebook computes all 64 entries of the tensor of exact condensates of
dirac16complex00 and
```

```
- shows the tensor of a generic condensate as a heat map and counts its nonzero
  off-diagonal entries: 42 on the deflating history, 12 when $a_4' = 0$;
- shows how these entries depend on the deflation rate $a_4'$;
- checks that each of the 42 entries is a multiple of one of 15 *three-gamma
  bilinears* $\bar\Phi\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$, with exactly the
  coefficient that the Revision record lists for it;
- builds the exact condensates of the Revision record for which all 15
  bilinears vanish, and checks that their tensor is diagonal at 36 points with
  different $a_4$, $a_4'$ and $z$ (the record states it for all of them);
- shows why $S$ of these condensates is never negative, and reproduces the
  record's numbers for them: their frequencies $\omega = 4, 3, 4$ exactly, and
  its three values of $S$ up to one common factor (the record's columns are not
  of length 1);
- tests the conservation $\nabla_\mu T^\mu{}_\nu = 0$ of the full tensor of a
  condensate along a curved (non-linear) history with finite differences whose
  step shrinks, and contrasts it with a configuration that leaves out the
  gravitational term $3H\gamma^{(0)}$ of the field equation (a negative control);
- draws five teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 09c (The full energy-momentum tensor of a condensate) is the file
‘Revision/textbook/notebooks/09c_condensate_tensor.ipynb‘ of the repository Dirac_claude.
It computes all 64 entries of the energy-momentum tensor of exact condensates of the
commuting field dirac16complex00 in the author's metric, including the off-diagonal
entries that the spin connection produces, finds the 42 nonzero off-diagonal entries of a
generic condensate and checks that each is a multiple of one of 15 three-gamma bilinears
with exactly the coefficient that the record lists for it, builds the exact condensates
whose tensor is diagonal and reproduces their frequencies omega and their S from the
record (S up to one common factor), and tests the conservation of the full tensor along a
curved history with finite differences of shrinking step, with a negative control that
leaves out the gravitational term of the field equation; five teaching plots. It needs a
computer with Windows 11, macOS or Linux, an internet connection for the installation,
the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
```

"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in
this one window only; it changes nothing permanently.

macOS (zsh):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 09c_condensate_tensor.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 09c_condensate_tensor.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 09c_condensate_tensor.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/09c.captions.json‘
- ‘Revision/textbook/figures/09c_1_generic_condensate.png‘
- ‘Revision/textbook/figures/09c_2_offdiagonal_entries.png‘
- ‘Revision/textbook/figures/09c_3_witness_condensate.png‘
- ‘Revision/textbook/figures/09c_4_conservation_convergence.png‘
- ‘Revision/textbook/figures/09c_5_divergence_components.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the five figures of this notebook are saved and captioned
ALL 25 CHECKS PASSED (notebook 09c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/09c_condensate_tensor.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

  ```
  python -m jupyterlab 09c_condensate_tensor.ipynb
  python -m nbconvert --execute --inplace 09c_condensate_tensor.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

- "KeyError" with the words "has no check": the notebook asks a Revision report for the verdict of one of its checks, and the report in your copy of the repository does not contain that check: your copy is older or newer than the notebook. Run ‘git pull‘ in the repository folder, then run the notebook again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 09c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 09c (The full energy-momentum tensor of a condensate) is the file
# Revision/textbook/notebooks/09c_condensate_tensor.ipynb of the repository Dirac_claude.
# It computes all 64 entries of the energy-momentum tensor of exact condensates of the
# commuting field dirac16complex00 in the author's metric, including the off-diagonal
# entries that the spin connection produces, finds the 42 nonzero off-diagonal entries of
# a generic condensate and checks that each is a multiple of one of 15 three-gamma
# bilinears with exactly the coefficient that the record lists for it, builds the exact
# condensates whose tensor is diagonal and reproduces their frequencies omega and their S
# from the record (S up to one common factor), and tests the conservation of the full
# tensor along a curved history with finite differences of shrinking step, with a
# negative control that leaves out the gravitational term of the field equation; five
# teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
# shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
```

```
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
```

```
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 09c_condensate_tensor.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
```

```
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 09c_condensate_tensor.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 09c_condensate_tensor.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/09c.captions.json
# - Revision/textbook/figures/09c_1_generic_condensate.png
# - Revision/textbook/figures/09c_2_offdiagonal_entries.png
# - Revision/textbook/figures/09c_3_witness_condensate.png
# - Revision/textbook/figures/09c_4_conservation_convergence.png
# - Revision/textbook/figures/09c_5_divergence_components.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the five figures of this notebook are saved and captioned
#     ALL 25 CHECKS PASSED (notebook 09c)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
```

```
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/09c_condensate_tensor.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 09c_condensate_tensor.ipynb
#     python -m nbconvert --execute --inplace 09c_condensate_tensor.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "KeyError" with the words "has no check": the notebook asks a Revision report for the
#   verdict of one of its checks, and the report in your copy of the repository does not
#   contain that check: your copy is older or newer than the notebook. Run git pull in
#   the repository folder, then run the notebook again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "09c"  # this notebook: chapter 09, example c


def find_repository_root():
```

```
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
```

```
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 09c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Condensate**: a configuration that depends only on the time $x_4$.
- **Diagonal / off-diagonal entry**: the entry $T^\nu{}_\mu$ with $\nu = \mu$ /
  with $\nu \neq \mu$. Off-diagonal entries describe flows of energy and momentum
  (for example $T^{x_4}{}_{x_1}$ is a flow of momentum along $x_1$) and stresses
  that shear.
- **Perfect fluid (at rest)**: matter whose energy-momentum tensor is diagonal,
  with the energy density and the pressures on the diagonal.
- **Three-gamma bilinear**: the number $\bar\Phi\gamma^{(a)}\gamma^{(b)}
  \gamma^{(c)}\Phi$ for three different directions $a$, $b$, $c$.
- **Eigenvector, joint eigenvector**: a column $v$ with $Mv = \mu v$ for a
  number $\mu$ (the eigenvalue); a joint eigenvector of several matrices is an
  eigenvector of each of them.
- **Witness**: an explicit example that proves that something exists.
- **Trace** of a square matrix: the sum of its diagonal entries.
- **Rank** of a matrix: the number of its independent columns.
- **Projector**: a matrix $\Pi$ with $\Pi^2 = \Pi$; it keeps some columns
  unchanged and sends others to zero, and its trace is the dimension of the
  space it keeps.
- **Sector**: the columns $v$ of one eigenvalue space of $A$ with fixed signs
  $P_1v = \pm v$, $P_2v = \pm v$, $P_3v = \pm v$ (the matrices $P_k$ are defined
  in section 8).
- **Frequency** $\omega$ (the Greek letter omega): the number in the factor
  $e^{-i\omega x_4} = \cos(\omega x_4) - i\sin(\omega x_4)$; a field with this
  factor repeats itself after the time $2\pi/\omega$. It has nothing to do with
  the equation of state $w = p/\rho$ of the other examples of this chapter.
- **Finite difference**: the derivative approximated by
  $\frac{F(x + h) - F(x - h)}{2h}$; its error falls like $h^2$ when the step $h$
  shrinks, until rounding errors take over.
- **Covariant divergence**: $\nabla_\mu T^\mu{}_\nu = \partial_\mu T^\mu{}_\nu
  + \Gamma^\mu{}_{\mu\lambda}T^\lambda{}_\nu - \Gamma^\lambda{}_{\mu\nu}
  T^\mu{}_\lambda$, with the Christoffel symbols $\Gamma$.
- **Negative control**: a test case built so that the check must FAIL; it shows
  that the check can detect an error.
```

Text cell:

```
## 4. The physical and mathematical situation

**The condensates.** For a field that depends on $x_4$ only, the field equation
of dirac16complex00 in the author's metric reduces to $\partial_4\Phi = A\Phi$
with $A = -\gamma^{(4)}(V - 3H\gamma^{(0)})$ and the effective mass
$V = m + \lambda S$; $S$ is constant, and the exact solution is
$\Phi = e^{Ax_4}\chi$. It does not contain $a_4$: it solves the equation on every
history $a_4(x_4)$.

**The tensor.** $T^\nu{}_\mu = \delta^\nu_\mu L_0 - \frac14(\bar\Phi\gamma^\nu
D_\mu\Phi - D_\mu\bar\Phi \gamma^\nu\Phi + \bar\Phi\gamma_\mu D^\nu\Phi
- D^\nu\bar\Phi \gamma_\mu\Phi)$ with $D_\mu\Phi = \partial_\mu\Phi
```

```
+ \Omega_\mu\Phi$. For a condensate $\partial_\mu\Phi = 0$ for $\mu \neq x_4$,
but $D_\mu\Phi = \Omega_\mu\Phi$ is not zero along 3-space and the extra times,
because the spin connection $\Omega_{x_i}$, $\Omega_{x_t}$ is not zero. This is
what fills the off-diagonal entries.

**The record** (‘Revision/field_equations_a4/reports/wolfram-a4-report.json‘ and
the independent ‘python-a4-report.json‘): for a condensate the diagonal is
$T^{x_4}{}_{x_4} = -VS + L_0$ and $T^\mu{}_\mu = L_0$ otherwise, and
$T^{x_4}{}_{x_0} = T^{x_0}{}_{x_4} = 0$; 42 ordered pairs $(\nu, \mu)$ carry
nonzero off-diagonal entries, each a multiple of one of the 15 bilinears with
$\{a, b, c\} = \{i, x_4, x_0\}$ ($i$ any of $x_1, x_2, x_3, x_5, x_6, x_7$) or
$\{i, j, x_4\}$ ($i$ in 3-space, $j$ an extra time); and exact witnesses exist
for which all 15 vanish. The field equations for $a_4$ require, among other
conditions, that every off-diagonal entry of the source vanishes (for a
coupling $\kappa \neq 0$; record
‘Revision/field_equations_a4/a4-equations.json‘, entries
‘generalSource.offDiagonal_other‘ and
‘fields.dirac16complex00.offDiagonalConditions‘); on the deflating history a
condensate meets this one condition only if all 15 bilinears vanish, as for the
witnesses.

**Conservation.** The record proves $\nabla_\mu T^\mu{}_\nu = 0$ for every
solution (the Noether identity of coordinate invariance) and checks it exactly
on the condensate (‘exact_solution_nonlinear_homogeneous_C‘). This notebook
tests it numerically, for all eight components and with the off-diagonal entries
included, along the non-linear history $a_4 = 0.3x_4 + 0.05x_4^2$.
```

Text cell:

```
## 5. The gammas, the geometry and the tensor at a point

The next cell reads the Revision gammas and $C$ from the fixture
‘Revision/algebra/gammas.json‘ and defines two helpers for the Revision
reports: ‘record_entry(path, name)‘ returns the whole entry of one check (its
name, its verdict and its detail text), and ‘recorded(path, name)‘ returns only
its verdict (PASS or FAIL).
```

In [2]:

```
import numpy as np  # arrays of numbers, matrices, linear algebra

fixture = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
gamma = [np.array(rows, dtype=float) for rows in fixture["gamma"]]  # x0, ..., x7
eta = np.array(fixture["eta"], dtype=float)  # (+1, +1, +1, +1, -1, -1, -1, -1)
C = np.array(fixture["C"], dtype=float)  # gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
I16 = np.eye(16)  # the 16 x 16 unit matrix
g4, g0 = gamma[4], gamma[0]  # gamma^(x4) and gamma^(x0)
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # the author's coordinates


def record_entry(path, name):
```

```
    """The entry (name, verdict, detail) of the check name in the report path."""
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for item in report_data["checks"]:
        if item["name"] == name:
            return item
    raise KeyError(f"{path} has no check {name}")


def recorded(path, name):
    """The verdict (PASS or FAIL) of the check name in the Revision report path."""
    return record_entry(path, name)["verdict"].upper()  # some reports write "pass"


say("The gammas, C and the two report helpers are ready.")
```

Out [2]:

```
The gammas, C and the two report helpers are ready.
```

Text cell:

```
The next cell defines the geometry at one point, the same functions as in the
first example of this chapter: ‘frame_factors(a4, z)‘ returns the eight vielbein
factors $f_a$ in the order $x_0, \dots, x_7$ ($\cot z$, $e^{a_4}\sin^{1/6}z$ three
times, $1$, $e^{-a_4}\sin^{1/6}z$ three times; the background $b_4 = 1$), and
‘spin_connection(a4, a4p, z, H)‘ returns the eight
$16 \times 16$ matrices $\Omega_\mu$ of the record’s formula ‘Omega_components‘:
$\Omega_{x_i} = \frac12 e^{a_4}\sin^{1/6}z (a_4’\gamma^{(i)}\gamma^{(4)}
+ H\gamma^{(i)}\gamma^{(0)})$, $\Omega_{x_t} = -\frac12 e^{-a_4}\sin^{1/6}z
(a_4’\gamma^{(4)}\gamma^{(t)} + H\gamma^{(t)}\gamma^{(0)})$ and
$\Omega_{x_4} = \Omega_{x_0} = 0$.
```

In [3]:

```
def frame_factors(a4, z):
    """The eight factors f_a of the diagonal vielbein, in the order x0, ..., x7."""
    space = np.exp(a4) * np.sin(z) ** (1 / 6)  # 3-space: e^a4 sin^(1/6) z
    extra = np.exp(-a4) * np.sin(z) ** (1 / 6)  # extra times: e^(-a4) sin^(1/6) z
    return np.array([1 / np.tan(z)] + [space] * 3 + [1.0] + [extra] * 3)


def spin_connection(a4, a4p, z, H):
    """The eight matrices Omega_mu (record formula Omega_components)."""
    s = np.sin(z) ** (1 / 6)  # sin^(1/6) z
    omega = [np.zeros((16, 16)) for _ in range(8)]  # Omega_x4 = Omega_x0 = 0
    for i in range(1, 4):  # the three 3-space directions x1, x2, x3
        omega[i] = 0.5 * np.exp(a4) * s * (a4p * gamma[i] @ g4 + H * gamma[i] @ g0)
    for t in range(5, 8):  # the three extra times x5, x6, x7
        omega[t] = -0.5 * np.exp(-a4) * s * (a4p * g4 @ gamma[t] + H * gamma[t] @ g0)
    return omega
```

```
say("frame_factors and spin_connection are defined.")
```

Out [3]:

```
frame_factors and spin_connection are defined.
```

Text cell:

```
The next cell defines ‘bar(vector)‘, the Dirac adjoint $\bar\Phi = \Phi^\dagger C$,
and ‘energy_momentum(phi, dphi, m, lam, a4, a4p, z, H)‘, which returns the 64
entries $T^\nu{}_\mu$ (row $\nu$, column $\mu$) of the record’s tensor
$T^\nu{}_\mu = \delta^\nu_\mu L_0 - \frac14(\bar\Phi\gamma^\nu D_\mu\Phi
- D_\mu\bar\Phi \gamma^\nu\Phi + \bar\Phi\gamma_\mu D^\nu\Phi
- D^\nu\bar\Phi \gamma_\mu\Phi)$ from the jet of the field at one point (the 16
values of $\Phi$ and the eight columns $\partial_\mu\Phi$). The tensor of a
configuration of the commuting field is real; the function stops with an error
if the imaginary parts are larger than rounding noise.
```

In [4]:

```
def bar(vector):
    """The Dirac adjoint vector^dagger C."""
    return vector.conj() @ C


def energy_momentum(phi, dphi, m, lam, a4, a4p, z, H):
    """The 64 entries T[nu, mu] = T^nu_mu at one point (real parts)."""
    f = frame_factors(a4, z)
    g_values = eta * f ** 2  # g_mumu
    omega = spin_connection(a4, a4p, z, H)
    gup = [gamma[mu] / f[mu] for mu in range(8)]  # gamma^mu
    S = (bar(phi) @ phi).real
    D = [dphi[mu] + omega[mu] @ phi for mu in range(8)]  # D_mu Phi
    Dbar = [bar(dphi[mu]) - bar(phi) @ omega[mu] for mu in range(8)]  # D_mu Phibar
    L0 = 0.5 * sum(bar(phi) @ gup[mu] @ D[mu] - Dbar[mu] @ gup[mu] @ phi
                   for mu in range(8)) - m * S - lam / 2 * S ** 2
    T = np.zeros((8, 8), dtype=complex)
    for nu in range(8):
        for mu in range(8):
            bracket = (bar(phi) @ gup[nu] @ D[mu] - Dbar[mu] @ gup[nu] @ phi
                       + g_values[mu] / g_values[nu]  # gamma_mu D^nu
                       * (bar(phi) @ gup[mu] @ D[nu] - Dbar[nu] @ gup[mu] @ phi))
            T[nu, mu] = (nu == mu) * L0 - bracket / 4
    if np.abs(T.imag).max() > 1e-9 * max(1.0, np.abs(T).max()):
        raise ValueError("the tensor is not real")  # it must be real
    return T.real


say("bar and energy_momentum are defined.")
```

Out [4]:

```
bar and energy_momentum are defined.
```

Text cell:

```
## 6. The tensor of a generic condensate

The next cell takes the effective mass $V = 5$ ($m = 5$, $\lambda = 0$), the
constant $H = 1$, a random column $\chi$ of length 1 (fixed seed), and the point
$a_4 = 0.5$, $a_4' = 0.25$, $z = \pi/4$ of a deflating history. The condensate's
jet at the point is $\Phi = \chi$ and $\partial_4\Phi = A\chi$ (the solution at
$x_4 = 0$; any other time gives another column $\chi$ with the same $S$). The
cell computes the 64 entries, checks the record's diagonal ($T^{x_4}{}_{x_4} =
-VS$ and the other diagonal entries $0$, because $\lambda = 0$ makes $L_0 = 0$ on
shell) and $T^{x_4}{}_{x_0} = T^{x_0}{}_{x_4} = 0$, counts the nonzero
off-diagonal entries (42), and counts them again with $a_4' = 0$ (12: only the
flows $T^{x_4}{}_{x_j}$ and $T^{x_j}{}_{x_4}$ remain, where $x_j$ is one of the
six directions $x_1, x_2, x_3, x_5, x_6, x_7$).
```

In [5]:

```
V, H = 5.0, 1.0  # effective mass (m = 5, lambda = 0) and the author's constant
POINT = {"a4": 0.5, "a4p": 0.25, "z": np.pi / 4}  # a point of a deflating history
rng = np.random.default_rng(11)  # a fixed seed
chi = rng.normal(size=16) + 1j * rng.normal(size=16)
chi /= np.linalg.norm(chi)  # length 1


def condensate_tensor(column, a4, a4p, z, V_value=V, H_value=H):
    """T of the condensate (lambda = 0, m = V_value) whose jet at the point is
    Phi = column and d4 Phi = A column, A = -gamma^(x4) (V - 3 H gamma^(x0))."""
    A_value = -g4 @ (V_value * I16 - 3 * H_value * g0)
    dphi = np.zeros((8, 16), dtype=complex)
    dphi[4] = A_value @ column  # the only nonzero derivative
    return energy_momentum(column, dphi, V_value, 0.0, a4, a4p, z, H_value)


def off_diagonal_pairs(T):
    """The ordered pairs (nu, mu), nu != mu, whose entry is not zero."""
    tolerance = 1e-10 * max(1.0, np.abs(T).max())
    return [(nu, mu) for nu in range(8) for mu in range(8)
            if nu != mu and abs(T[nu, mu]) > tolerance]


T_generic = condensate_tensor(chi, **POINT)
S_chi = (bar(chi) @ chi).real
expected_diagonal = np.zeros(8)
expected_diagonal[4] = -V * S_chi
A4_WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
A4_PY = "Revision/field_equations_a4/reports/python-a4-report.json"
```

```
check(np.abs(np.diag(T_generic) - expected_diagonal).max() < 1e-12
      and abs(T_generic[4, 0]) < 1e-12 and abs(T_generic[0, 4]) < 1e-12
      and recorded(A4_WL, "condensate_kinetic_tensor_diagonal") == "PASS"
      and recorded(A4_PY, "authorT16_condensate_kinetic_diagonal") == "PASS",
      "diagonal (0, 0, 0, 0, -V S, 0, 0, 0) and T^x4_x0 = T^x0_x4 = 0",
      record=f"{A4_WL}, check condensate_kinetic_tensor_diagonal; {A4_PY}, check "
             "authorT16_condensate_kinetic_diagonal")
pairs = off_diagonal_pairs(T_generic)
# the same condensate and point with a4p = 0 (a4 not changing at this moment)
pairs_constant_a4 = off_diagonal_pairs(
    condensate_tensor(chi, POINT["a4"], 0.0, POINT["z"]))
check(len(pairs) == 42, "a generic condensate has 42 nonzero off-diagonal entries")
momentum_flows = sorted([(4, i) for i in (1, 2, 3, 5, 6, 7)]
                        + [(i, 4) for i in (1, 2, 3, 5, 6, 7)])
check(sorted(pairs_constant_a4) == momentum_flows,
      "with a4p = 0 only the 12 flows T^x4_xj, T^xj_x4 (j = 1, 2, 3, 5, 6, 7) remain")
report("S of the random column", f"{S_chi:.6f}")
report("T^x4_x4 = -V S", f"{T_generic[4, 4]:.6f}")
```

Out [5]:

```
PASS diagonal (0, 0, 0, 0, -V S, 0, 0, 0) and T^x4_x0 = T^x0_x4 = 0
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_kinetic_tensor_diagonal;
    Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_kinetic_diagonal
PASS a generic condensate has 42 nonzero off-diagonal entries
PASS with a4p = 0 only the 12 flows T^x4_xj, T^xj_x4 (j = 1, 2, 3, 5, 6, 7) remain
RESULT S of the random column = 0.293667
RESULT T^x4_x4 = -V S = -1.468334
```

Text cell:

```
The next cell draws the 64 entries of this generic condensate as a heat map.
```

In [6]:

```
def heat_map(ax, T, title, largest):
    """Draw T as coloured squares with the value printed in each."""
    picture = ax.imshow(T, cmap="RdBu_r", vmin=-largest, vmax=largest)
    for nu in range(8):
        for mu in range(8):
            text = f"{T[nu, mu]:.2f}"
            if text == "-0.00":  # a tiny negative number: print it as 0.00
                text = "0.00"
            # white digits on dark squares, black digits on light ones
            colour = "white" if abs(T[nu, mu]) > 0.6 * largest else "black"
            ax.text(mu, nu, text, ha="center", va="center", fontsize=6,
                    color=colour)
    ticks = [f"${n[0]}_{n[1]}$" for n in NAMES]
    ax.set_xticks(range(8), ticks)
```

```
    ax.set_yticks(range(8), ticks)
    ax.set_xlabel("lower index $\\mu$ (column)")
    ax.set_ylabel("upper index $\\nu$ (row)")
    ax.set_title(title)
    ax.grid(False)
    return picture


SCALE = 2.0  # the colour scale runs from -2 to 2 in both heat maps of this notebook
fig, ax = plt.subplots(figsize=(6.6, 5.6))
picture = heat_map(ax, T_generic, "Generic condensate: $T^\\nu{}_\\mu$", SCALE)
fig.colorbar(picture, ax=ax, label="value (energy per unit volume)", extend="both")
save_figure(fig, "generic_condensate",
            "Heat map of the 64 entries $T^\\nu{}_\\mu$ (row $\\nu$, column $\\mu$, "
            "both running over $x_0, \\dots, x_7$) of a generic exact condensate of "
            "dirac16complex00 with effective mass $V = 5$, $\\lambda = 0$, $H = 1$, "
            "a random unit column $\\chi$, at the point $a_4 = 0.5$, $a_4' = 0.25$, "
            "$z = \\pi/4$; colour scale from $-2$ to $2$ (entries beyond it are "
            "shown in the darkest colour and printed), in units of energy per unit "
            "volume. The diagonal holds only $-VS$ at $x_4$; 42 off-diagonal "
            "squares are nonzero, all of them produced by the spin connection. The "
            "squares $(x_4, x_0)$ and $(x_0, x_4)$ are zero.")
```

Out [6]:

```
Figure 09c.1 saved as Revision/textbook/figures/09c_1_generic_condensate.png
```

Text cell:

```
## 7. How the off-diagonal entries depend on the deflation rate

The next cell computes three off-diagonal entries of the same condensate at the
same point for 41 values of $a_4'$ from $-1$ to $1$: the momentum flow
$T^{x_4}{}_{x_1}$, the 3-space/extra-time entry $T^{x_1}{}_{x_5}$ and the
3-space/hidden entry $T^{x_1}{}_{x_0}$. The first does not depend on $a_4'$ (it
comes from the $H$ part of the spin connection); the other two are proportional
to $a_4'$, so they appear only because the extra times deflate (or, for
$a_4' < 0$, inflate). The cell checks both statements.
```

In [7]:

```
rates = np.linspace(-1.0, 1.0, 41)  # values of a4p
entries = np.array([[T[4, 1], T[1, 5], T[1, 0]] for T in (
    condensate_tensor(chi, POINT["a4"], rate, POINT["z"]) for rate in rates)])
check(np.abs(entries[:, 0] - entries[20, 0]).max() < 1e-12 and abs(entries[20, 0]) > 0.1,
      "T^x4_x1 does not depend on a4p and is not zero")
slopes = entries[-1, 1:] / rates[-1]  # the slope of each of the other two entries
check(np.abs(entries[:, 1:] - np.outer(rates, slopes)).max() < 1e-12
      and np.abs(slopes).min() > 1e-3,
      "T^x1_x5 and T^x1_x0 are proportional to a4p (zero only at a4p = 0)")
report("T^x4_x1", f"{entries[20, 0]:.6f}")
```

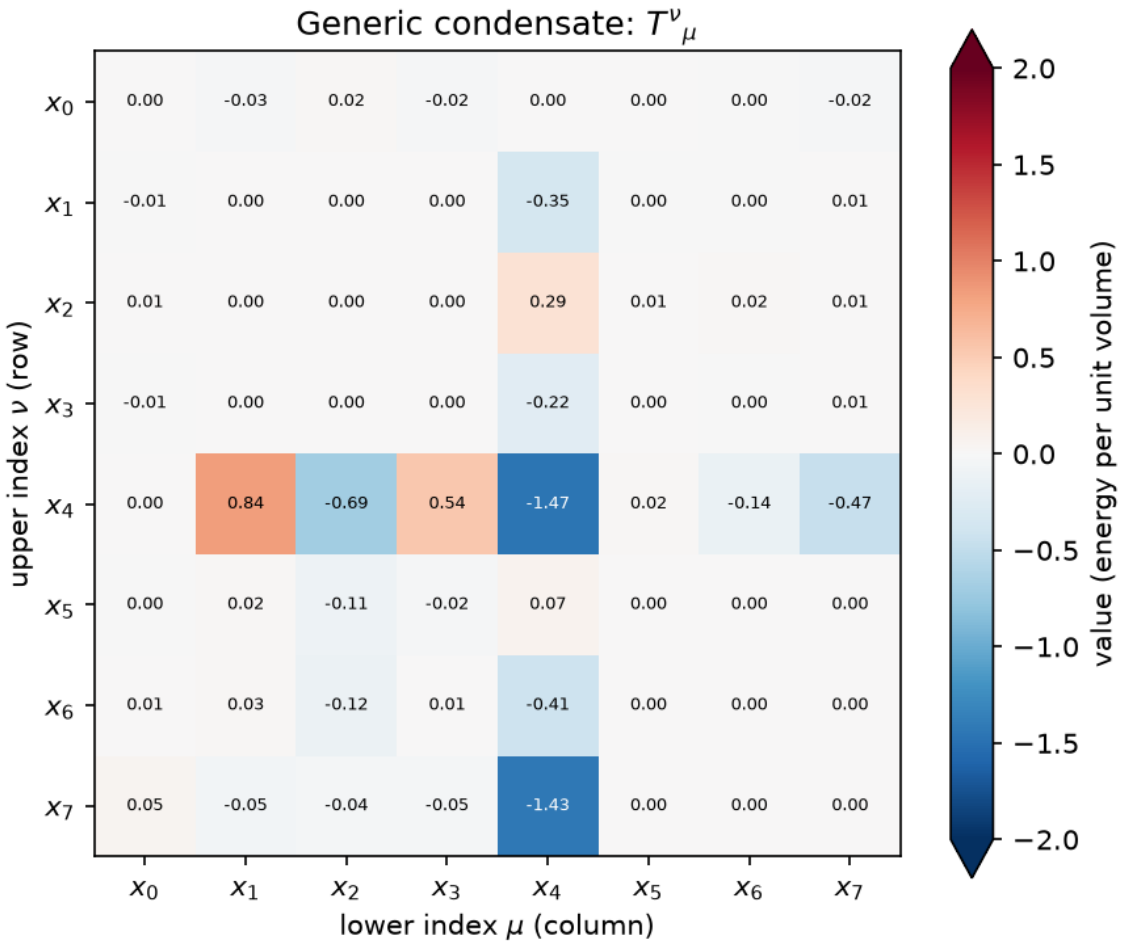

Figure 262: Heat map of the 64 entries $T^{\nu}{}_{\mu}$ (row ν, column μ, both running over $x_0, \ldots, x_7$) of a generic exact condensate of dirac16complex00 with effective mass $V = 5$, $\lambda = 0$, $H = 1$, a random unit column χ, at the point $a_4 = 0.5$, $a_4' = 0.25$, $z = \pi/4$; colour scale from -2 to 2 (entries beyond it are shown in the darkest colour and printed), in units of energy per unit volume. The diagonal holds only $-VS$ at x_4; 42 off-diagonal squares are nonzero, all of them produced by the spin connection. The squares (x_4, x_0) and (x_0, x_4) are zero. (Notebook 09c, figure 1.)

```
report("slopes d T^x1_x5/d a4p and d T^x1_x0/d a4p",
       f"{slopes[0]:.6f} and {slopes[1]:.6f}")
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
fig.subplots_adjust(wspace=0.35)
left.plot(rates, entries[:, 0], label="$T^{x_4}{}_{x_1}$ (momentum flow)")
left.set_ylim(0.0, 1.2 * entries[20, 0])
left.set_xlabel("deflation rate $a_4'$")
left.set_ylabel("entry (energy per unit volume)")
left.set_title("Comes from $H$: independent of $a_4'$")
left.legend(fontsize=8)
right.plot(rates, entries[:, 1], label="$T^{x_1}{}_{x_5}$")
right.plot(rates, entries[:, 2], "--", label="$T^{x_1}{}_{x_0}$")
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("deflation rate $a_4'$")
right.set_title("Proportional to $a_4'$")
right.legend(fontsize=8)
save_figure(fig, "offdiagonal_entries",
            "Three off-diagonal entries of the generic condensate of the previous "
            "figure ($V = 5$, $H = 1$, $a_4 = 0.5$, $z = \\pi/4$) against the "
            "deflation rate $a_4'$ from $-1$ to $1$, in units of energy per unit "
            "volume. Left: the momentum flow $T^{x_4}{}_{x_1}$ comes from the $H$ "
            "part of the spin connection and does not depend on $a_4'$. Right: "
            "$T^{x_1}{}_{x_5}$ (solid) and $T^{x_1}{}_{x_0}$ (dashed) are "
            "proportional to $a_4'$; at $a_4' = 0$ they vanish, and only the 12 "
            "flows $T^{x_4}{}_{x_j}$ and $T^{x_j}{}_{x_4}$ remain ($x_j$ one of "
            "$x_1, x_2, x_3, x_5, x_6, x_7$).")
```

Out [7]:

```
PASS T^x4_x1 does not depend on a4p and is not zero
PASS T^x1_x5 and T^x1_x0 are proportional to a4p (zero only at a4p = 0)
RESULT T^x4_x1 = 0.835532
RESULT slopes d T^x1_x5/d a4p and d T^x1_x0/d a4p = -0.012942 and -0.049288
```

Out [7]:

```
Figure 09c.2 saved as Revision/textbook/figures/09c_2_offdiagonal_entries.png
```

Text cell:

```
## 8. The 15 three-gamma bilinears and the condensates with a diagonal tensor

**Each entry is one bilinear.** The record says that each of the 42 nonzero
off-diagonal entries is a multiple of one bilinear
$\bar\Phi\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$: for the pair $(\nu, \mu)$
the three directions are $\nu$, $\mu$ and the missing one of $x_4$, $x_0$ (or
$x_4$ when neither occurs). The next cell checks this by computing the ratio of
each entry to its bilinear for three different random columns: the ratio must
be the same number for all three (it depends on the point and on $V$, $H$, not on
the column). It also checks that exactly 15 bilinears occur.
```

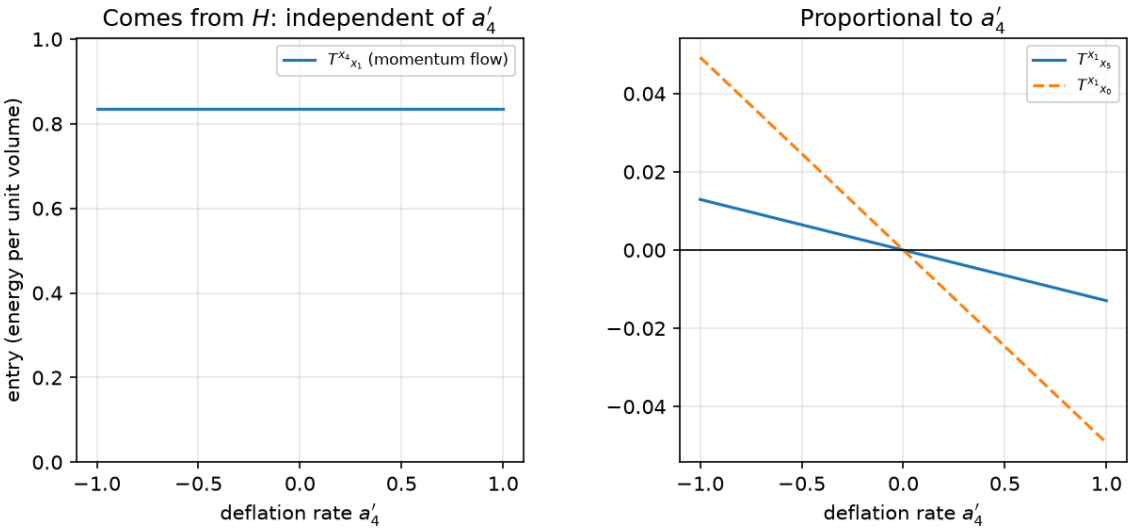

Figure 263: Three off-diagonal entries of the generic condensate of the previous figure ($V = 5$, $H = 1$, $a_4 = 0.5$, $z = \pi/4$) against the deflation rate a_4' from -1 to 1, in units of energy per unit volume. Left: the momentum flow $T^{x_4}{}_{x_1}$ comes from the H part of the spin connection and does not depend on a_4'. Right: $T^{x_1}{}_{x_5}$ (solid) and $T^{x_1}{}_{x_0}$ (dashed) are proportional to a_4'; at $a_4' = 0$ they vanish, and only the 12 flows $T^{x_4}{}_{x_j}$ and $T^{x_j}{}_{x_4}$ remain (x_j one of $x_1, x_2, x_3, x_5, x_6, x_7$). (Notebook 09c, figure 2.)

```
**The witnesses.** The record builds condensates for which all 15 bilinears
vanish. The three matrices $P_1 = \gamma^{(1)}\gamma^{(5)}$,
$P_2 = \gamma^{(2)}\gamma^{(6)}$, $P_3 = \gamma^{(3)}\gamma^{(7)}$ square to 1 and
commute with each other and with $A$. Let $\omega = \sqrt{V^2 - 9H^2}$ (a real
number when $V^2 > 9H^2$); then $A^2 = (9H^2 - V^2) \cdot 1 = -\omega^2 \cdot 1$,
and $A$ has the eigenvalue $-i\omega$ on an 8-dimensional space: every column
$u$ is the sum of $\frac12(u + \frac{i}{\omega}Au)$, with eigenvalue $-i\omega$,
and $\frac12(u - \frac{i}{\omega}Au)$, with eigenvalue $+i\omega$, and because
$A$ is real, complex conjugation carries the one space onto the other, so both
have dimension $16/2 = 8$. The three $P_k$ split that space into eight sectors
(one for each choice of the signs of $P_1$, $P_2$, $P_3$). The matrix that keeps
one sector, $\frac12(1 + \frac{i}{\omega}A) \cdot \frac12(1 \pm P_1) \cdot
\frac12(1 \pm P_2) \cdot \frac12(1 \pm P_3)$, has trace 1, because every
product of different gammas has trace 0; so every sector has dimension 1. In
the sector $P_1 = P_2 = P_3 = -1$ take a unit vector $v_1$ and in the sector
$P_1 = P_2 = P_3 = +1$ a unit vector $v_2$ (each unique up to a factor of size
1, which does not change the bilinears), put $c = \overline{v_1^\dagger Cv_2}$
(the bar means the complex conjugate number) and $\Phi_0 = v_1 + cv_2$. The
condensate is $\Phi = e^{-i\omega x_4}\Phi_0$: it oscillates with the frequency
$\omega$. The record gives three such witnesses, $(V, H) = (5, 1)$, $(5, 4/3)$,
$(-5, 1)$, with $\omega = 4$, $3$, $4$ (the record writes the effective mass $V$
as $M$ and the frequency $\omega$ as w).
```

In [8]:

```
def bilinear_directions(nu, mu):
    """The three directions (a, b, c) of the bilinear that carries T^nu_mu."""
```

```
    directions = {nu, mu}
    if 4 in directions and 0 not in directions:
        directions.add(0)  # add x0
    elif 0 in directions and 4 not in directions:
        directions.add(4)  # add x4
    elif 4 not in directions and 0 not in directions:
        directions.add(4)  # add x4
    return tuple(sorted(directions))


def bilinear(column, directions):
    a, b, c = directions
    return bar(column) @ gamma[a] @ gamma[b] @ gamma[c] @ column


columns = [rng.normal(size=16) + 1j * rng.normal(size=16) for _ in range(3)]
tensors = [condensate_tensor(column, **POINT) for column in columns]
worst_ratio, used = 0.0, set()
for nu, mu in pairs:
    directions = bilinear_directions(nu, mu)
    used.add(directions)
    ratios = [tensors[n][nu, mu] / bilinear(columns[n], directions) for n in range(3)]
    worst_ratio = max(worst_ratio,
                      max(abs(r - ratios[0]) for r in ratios) / abs(ratios[0]))
check(worst_ratio < 1e-9 and len(used) == 15
      and recorded(A4_WL, "condensate_offdiagonal_are_three_gamma_bilinears") == "PASS"
      and recorded(A4_PY, "authorT16_condensate_offdiagonal_three_gamma") == "PASS",
      "each of the 42 entries is a fixed multiple of one of 15 three-gamma bilinears",
      record=f"{A4_WL}, check condensate_offdiagonal_are_three_gamma_bilinears; "
             f"{A4_PY}, check authorT16_condensate_offdiagonal_three_gamma")
for directions in sorted(used):
    say("bilinear Phibar gamma^(" + ") gamma^(".join(NAMES[d] for d in directions)
        + ") Phi")
```

Out [8]:

```
PASS each of the 42 entries is a fixed multiple of one of 15 three-gamma bilinears
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_offdiagonal_are_three_gamma_bilinears;
    Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_offdiagonal_three_gamma
bilinear Phibar gamma^(x0) gamma^(x1) gamma^(x4) Phi
bilinear Phibar gamma^(x0) gamma^(x2) gamma^(x4) Phi
bilinear Phibar gamma^(x0) gamma^(x3) gamma^(x4) Phi
bilinear Phibar gamma^(x0) gamma^(x4) gamma^(x5) Phi
bilinear Phibar gamma^(x0) gamma^(x4) gamma^(x6) Phi
bilinear Phibar gamma^(x0) gamma^(x4) gamma^(x7) Phi
bilinear Phibar gamma^(x1) gamma^(x4) gamma^(x5) Phi
bilinear Phibar gamma^(x1) gamma^(x4) gamma^(x6) Phi
bilinear Phibar gamma^(x1) gamma^(x4) gamma^(x7) Phi
bilinear Phibar gamma^(x2) gamma^(x4) gamma^(x5) Phi
bilinear Phibar gamma^(x2) gamma^(x4) gamma^(x6) Phi
bilinear Phibar gamma^(x2) gamma^(x4) gamma^(x7) Phi
```

```
bilinear Phibar gamma^(x3) gamma^(x4) gamma^(x5) Phi
bilinear Phibar gamma^(x3) gamma^(x4) gamma^(x6) Phi
bilinear Phibar gamma^(x3) gamma^(x4) gamma^(x7) Phi
```

Text cell:

```
**The coefficients of the record.** The record
‘Revision/field_equations_a4/a4-equations.json‘ lists, for each of the 42
entries, its bilinear and its coefficient as a formula in the Wolfram Language,
in the names ‘cc‘ $= \cot z$, ‘a4v‘ $= a_4$, ‘ad1‘ $= a_4’$ and ‘H‘, for the
symmetrised kinetic tensor $K^{(\nu}{}_{\mu)}$, the average of
$K^\nu{}_\mu = \frac12(\bar\Phi\gamma^\nu D_\mu\Phi - D_\mu\bar\Phi\gamma^\nu\Phi)$
and of $g^{\nu\nu}g_{\mu\mu}K^\mu{}_\nu$ (the same with the two indices
exchanged, then moved back to their places with the metric). Off the diagonal the
tensor is $T^\nu{}_\mu = -K^{(\nu}{}_{\mu)}$, so every entry must equal minus
the coefficient times the bilinear. The next cell translates each formula into
sympy (‘E^x‘ becomes ‘E**x‘, ‘^‘ becomes ‘**‘), evaluates it at the point
$a_4 = 0.5$, $a_4’ = 0.25$, $z = \pi/4$, $H = 1$, and compares, for the generic
condensate and for the three random columns, all 42 entries. The coefficients
contain $V$ nowhere: they are the same for every effective mass. The cell prints
one entry of each of the ten groups of equal coefficients.
```

In [9]:

```
import re  # regular expressions: patterns that find pieces of a text

import sympy as sp  # exact algebra with symbols

A4_RECORD = "Revision/field_equations_a4/a4-equations.json"
record_entries = json.loads(repository_file(A4_RECORD).read_text(encoding="utf-8"))[
    "fields"]["dirac16complex00"]["offDiagonalKinetic"]
cc, a4v, ad1, H_symbol = sp.symbols("cc a4v ad1 H", real=True)
AT_POINT = {cc: 1 / np.tan(POINT["z"]), a4v: POINT["a4"], ad1: POINT["a4p"],
            H_symbol: H}
worst_coefficient, groups = 0.0, {}  # groups: formula text -> its first entry
for item in record_entries:
    # "K^x1_x4 (symmetrised)": upper index x1 (row nu), lower index x4 (column mu)
    nu, mu = [int(digit) for digit in re.findall(r"x(\d)", item["component"])]
    term = item["terms"][0]  # every entry of the record has exactly one term
    directions = [int(digit) for digit in re.findall(r"x(\d)", term["bilinear"])]
    formula = term["coefficient"]["input"]  # the Wolfram Language text
    coefficient = float(sp.sympify(
        formula.replace("E^", "E**").replace("^", "**"),
        locals={"E": sp.E, "cc": cc, "a4v": a4v, "ad1": ad1, "H": H_symbol},
    ).subs(AT_POINT))
    for column, T in zip([chi] + columns, [T_generic] + tensors):
        predicted = -coefficient * bilinear(column, tuple(directions))
        worst_coefficient = max(worst_coefficient, abs(T[nu, mu] - predicted))
    groups.setdefault(formula, (nu, mu, coefficient))
check(len(record_entries) == 42 and len(groups) == 10 and worst_coefficient < 1e-12
      and recorded(A4_PY, "json_offdiagonal_coefficients") == "PASS"
      and recorded(A4_PY, "offdiagonal_coefficients_representation_independent")
```

```
      == "PASS",
      "all 42 entries are -(record coefficient) x bilinear, for 4 columns",
      record=f"{A4_RECORD}, offDiagonalKinetic; {A4_PY}, checks "
             "json_offdiagonal_coefficients and "
             "offdiagonal_coefficients_representation_independent")
for nu, mu, coefficient in groups.values():
    say(f"T^{NAMES[nu]}_{NAMES[mu]} = -({coefficient:+.6f}) x its bilinear")
```

Out [9]:

```
PASS all 42 entries are -(record coefficient) x bilinear, for 4 columns
     reproduces Revision/field_equations_a4/a4-equations.json, offDiagonalKinetic;
    Revision/field_equations_a4/reports/python-a4-report.json, checks
    json_offdiagonal_coefficients and offdiagonal_coefficients_representation_independent
T^x0_x1 = -(+0.097262) x its bilinear
T^x0_x5 = -(-0.035781) x its bilinear
T^x1_x0 = -(+0.040162) x its bilinear
T^x1_x4 = -(+1.124544) x its bilinear
T^x1_x5 = -(-0.045985) x its bilinear
T^x4_x1 = -(-2.723325) x its bilinear
T^x4_x5 = -(-1.001855) x its bilinear
T^x5_x0 = -(+0.109172) x its bilinear
T^x5_x1 = -(+0.339785) x its bilinear
T^x5_x4 = -(-3.056829) x its bilinear
```

Text cell:

```
The next cell defines the function ‘witness(V_w, H_w)‘. It finds the
8-dimensional space of $A$ for the eigenvalue $-i\omega$ (as the null space of
$A + i\omega$, from a singular value decomposition), then inside it the joint
eigenvectors $v_1$, $v_2$ (with the projector
$\frac18(1 \pm P_1)(1 \pm P_2)(1 \pm P_3)$, which keeps exactly the vectors with
$P_1 = P_2 = P_3 = \pm1$), each of length 1, and returns $\omega$, $A$, $v_1$,
$v_2$ and $\Phi_0 = v_1 + cv_2$ (in the code the frequency $\omega$ is called
‘freq‘). The cell builds the three witnesses and prints their frequencies
$\omega = \sqrt{V^2 - 9H^2}$. It then checks the two dimensions of the
construction with numbers: each of the $2^8 - 1 = 255$ products of 1 to 8
different gammas has trace 0; for each witness the space of $A$ for $-i\omega$
has dimension 8 (the rank of $A + i\omega$ is $16 - 8$); and the matrix that keeps
one sector has trace 1 for all eight choices of the three signs.
```

In [10]:

```
import itertools  # all combinations and all sign choices of a list

P = [gamma[i] @ gamma[i + 4] for i in range(1, 4)]  # gamma^(x1) gamma^(x5), ...


def witness(V_w, H_w):
    """The frequency omega (here freq), A, the unit vectors v1, v2 and the column
    Phi0 of the record's diagonal witness for (V_w, H_w)."""
```

```
    A_w = -g4 @ (V_w * I16 - 3 * H_w * g0)  # d4 Phi = A Phi
    freq = np.sqrt(V_w ** 2 - 9 * H_w ** 2)  # the frequency omega
    # The null space of A + i omega: the rows of the third SVD factor whose
    # singular value is zero, complex conjugated and written as columns.
    _, singular, rows = np.linalg.svd(A_w + 1j * freq * I16)
    space = rows[singular < 1e-9].conj().T  # 16 x 8: the eigenspace for -i omega
    found = []
    for sign in (-1, 1):
        projector = (I16 + sign * P[0]) @ (I16 + sign * P[1]) @ (I16 + sign * P[2]) / 8
        left_vectors, values, _ = np.linalg.svd(projector @ space)
        if np.sum(values > 1e-9) != 1:
            raise ValueError("the joint eigenvector is not unique")
        found.append(left_vectors[:, 0])  # a unit vector
    v1, v2 = found
    c = np.conj(bar(v1) @ v2)  # c = conj(v1^dagger C v2)
    return freq, A_w, v1, v2, v1 + c * v2


WITNESSES = [(5.0, 1.0), (5.0, 4 / 3), (-5.0, 1.0)]  # the record's (V, H)
witnesses = {pair: witness(*pair) for pair in WITNESSES}  # (V, H) -> the five
for (V_w, H_w), found in witnesses.items():
    say(f"witness (V, H) = ({V_w:g}, {H_w:.4g}): frequency omega = "
        f"sqrt(V^2 - 9 H^2) = {found[0]:.6f}")


def product_of(indices):
    """The product gamma^(a) gamma^(b) ... of the frame gammas with these indices."""
    result = I16
    for index in indices:
        result = result @ gamma[index]
    return result


subsets = [chosen for size in range(1, 9)
           for chosen in itertools.combinations(range(8), size)]  # 255 index sets
traceless = all(abs(np.trace(product_of(chosen))) < 1e-12 for chosen in subsets)
dimensions_hold = True
for (V_w, H_w), (freq, A_w, v1, v2, phi0) in witnesses.items():
    # the dimension of the space of -i omega: 16 minus the rank of A + i omega
    dimension = 16 - np.linalg.matrix_rank(A_w + 1j * freq * I16, tol=1e-9)
    for signs in itertools.product((-1, 1), repeat=3):  # the eight sectors
        keeper = (I16 + 1j / freq * A_w) / 2  # keeps the space of -i omega
        for sign, P_k in zip(signs, P):
            keeper = keeper @ (I16 + sign * P_k) / 2  # keeps P_k = sign
        dimensions_hold = dimensions_hold and abs(np.trace(keeper) - 1) < 1e-12
    dimensions_hold = dimensions_hold and dimension == 8
check(len(subsets) == 255 and traceless and dimensions_hold,
      "255 gamma products have trace 0; space of -i omega: dim 8; each sector: dim 1")
```

Out [10]:

```
witness (V, H) = (5, 1): frequency omega = sqrt(V^2 - 9 H^2) = 4.000000
witness (V, H) = (5, 1.333): frequency omega = sqrt(V^2 - 9 H^2) = 3.000000
```

```
witness (V, H) = (-5, 1): frequency omega = sqrt(V^2 - 9 H^2) = 4.000000
PASS 255 gamma products have trace 0; space of -i omega: dim 8; each sector: dim 1
```

Text cell:

```
The next cell checks each witness: that $A\Phi_0 = -i\omega\Phi_0$ (so
$\Phi = e^{-i\omega x_4}\Phi_0$ is an exact condensate), that all 15 bilinears
vanish, that $S \neq 0$, and that the tensor is diagonal at 36 points: every
combination of $a_4 \in \{-1, 0, 0.5\}$, $a_4' \in \{-0.5, 0, 0.25, 1\}$ and
$z \in \{0.3, 0.8, 1.3\}$. It prints $S$ of each witness (with $v_1$ and $v_2$ of
length 1).
```

In [11]:

```
grid = [(a4, a4p, z) for a4 in (-1.0, 0.0, 0.5) for a4p in (-0.5, 0.0, 0.25, 1.0)
        for z in (0.3, 0.8, 1.3)]  # 3 x 4 x 3 = 36 points
witness_columns = {}  # (V, H) -> Phi0
unit_S = {}  # (V, H) -> S of Phi0 built from v1 and v2 of length 1
for (V_w, H_w), (freq, A_w, v1, v2, phi0) in witnesses.items():
    S_w = (bar(phi0) @ phi0).real
    largest_bilinear = max(abs(bilinear(phi0, d)) for d in used)
    worst_off = 0.0  # the largest off-diagonal entry at the 36 points
    for a4, a4p, z in grid:
        T = condensate_tensor(phi0, a4, a4p, z, V_w, H_w)
        worst_off = max(worst_off, np.abs(T - np.diag(np.diag(T))).max())
    check(np.abs(A_w @ phi0 + 1j * freq * phi0).max() < 1e-12
          and largest_bilinear < 1e-12 and abs(S_w) > 0.1 and worst_off < 1e-12,
          f"witness (V, H) = ({V_w:g}, {H_w:.4g}): omega = {freq:g}, the 15 "
          "bilinears vanish, S != 0, T diagonal at 36 points")
    witness_columns[(V_w, H_w)] = phi0
    unit_S[(V_w, H_w)] = S_w
    report(f"witness ({V_w:g}, {H_w:.4g}): omega, and S for unit v1, v2",
           f"{freq:g} and {S_w:.6f}")
```

Out [11]:

```
PASS witness (V, H) = (5, 1): omega = 4, the 15 bilinears vanish, S != 0, T diagonal at
    36 points
RESULT witness (5, 1): omega, and S for unit v1, v2 = 4 and 1.280000
PASS witness (V, H) = (5, 1.333): omega = 3, the 15 bilinears vanish, S != 0, T diagonal
    at 36 points
RESULT witness (5, 1.333): omega, and S for unit v1, v2 = 3 and 0.720000
PASS witness (V, H) = (-5, 1): omega = 4, the 15 bilinears vanish, S != 0, T diagonal at
    36 points
RESULT witness (-5, 1): omega, and S for unit v1, v2 = 4 and 1.280000
```

Text cell:

```
**Why $S$ of a witness is never negative.** $C$ anticommutes with each $P_k$ (it
contains $\gamma^{(x_k)}$ but not $\gamma^{(x_{k+4})}$), and each $P_k$ is real
```

```
and symmetric, so $P_k^\dagger = P_k$. Take $P_1v_1 = -v_1$. Then, line by line:
$v_1^\dagger Cv_1 = -v_1^\dagger CP_1v_1$ (replace $v_1$ by $-P_1v_1$)
$= v_1^\dagger P_1Cv_1$ ($CP_1 = -P_1C$) $= (P_1v_1)^\dagger Cv_1$
($P_1^\dagger = P_1$) $= -v_1^\dagger Cv_1$ ($P_1v_1 = -v_1$). A number equal to
minus itself is zero: $v_1^\dagger Cv_1 = 0$, and in the same way
$v_2^\dagger Cv_2 = 0$. Write $s = v_1^\dagger Cv_2$; then
$v_2^\dagger Cv_1 = \bar s$ ($C$ is real and symmetric) and $c = \bar s$, so
$S = \Phi_0^\dagger C\Phi_0 = v_1^\dagger Cv_1 + c\,s + \bar c\,\bar s
+ |c|^2 v_2^\dagger Cv_2 = 0 + |s|^2 + |s|^2 + 0 = 2|s|^2 \geq 0$. The next cell
checks each step numerically for the three witnesses. It also checks an
observation (COMPUTED for these three witnesses, not proved here): with $v_1$,
$v_2$ of length 1, $|s| = |\omega/V|$, so $S = 2\omega^2/V^2$; this gives
$2 \cdot 16/25 = 1.28$ and $2 \cdot 9/25 = 0.72$.
```

In [12]:

```
anticommute = all(np.abs(C @ P_k + P_k @ C).max() < 1e-12 for P_k in P)
symmetric = all(np.array_equal(P_k, P_k.T) for P_k in P)  # real: P^dagger = P^T
steps_hold, observation = True, True
for (V_w, H_w), (freq, A_w, v1, v2, phi0) in witnesses.items():
    s = bar(v1) @ v2  # s = v1^dagger C v2
    S_unit = unit_S[(V_w, H_w)]  # S of Phi0 with unit v1, v2 (previous cell)
    steps_hold = (steps_hold and abs(bar(v1) @ v1) < 1e-12
                  and abs(bar(v2) @ v2) < 1e-12
                  and abs(S_unit - 2 * abs(s) ** 2) < 1e-12)
    observation = observation and abs(S_unit - 2 * freq ** 2 / V_w ** 2) < 1e-12
check(anticommute and symmetric and steps_hold,
      "C P_k = -P_k C, P_k symmetric, v^dagger C v = 0, so S = 2 |v1^dagger C v2|^2")
check(observation,
      "observed: S = 2 omega^2 / V^2 for the three witnesses (unit v1, v2)")
```

Out [12]:

```
PASS C P_k = -P_k C, P_k symmetric, v^dagger C v = 0, so S = 2 |v1^dagger C v2|^2
PASS observed: S = 2 omega^2 / V^2 for the three witnesses (unit v1, v2)
```

Text cell:

```
**The numbers of the record.** The Wolfram report of the record lists, in the
detail text of its check ‘condensate_diagonal_witness_exact‘, the frequencies
$\omega$ (written w there) and the values of $S$ of the three witnesses. The
record’s verifier builds the witnesses with exact columns $v_1$, $v_2$ that are
not of length 1; since $S = 2|v_1^\dagger Cv_2|^2$ grows with the lengths of
$v_1$ and $v_2$, its $S$ cannot equal ours, while $\omega = \sqrt{V^2 - 9H^2}$
must agree exactly. The next cell reads the numbers out of the detail text (with
a regular expression, a pattern that finds text), prints them, checks $\omega$,
and divides each $S$ of the record by our $S$: the three quotients must be one
and the same number, so that the record’s three values are reproduced up to one
common factor, which the cell prints. That factor is the product of the squared
lengths of the record’s columns, a choice of its verifier, not a physical
number.
```

In [13]:

```
detail = record_entry(A4_WL, "condensate_diagonal_witness_exact")["detail"]
# the frequencies, which the record writes "w = sqrt(M^2 - 9 H^2) = {4, 3, 4}"
freq_text = re.search(r"9 H\^2\) = \{([^}]*)\}", detail).group(1)  # "4, 3, 4"
S_text = re.search(r"S = \{([^}]*)\}", detail).group(1)  # the three S values
record_freq = np.array([float(value) for value in freq_text.split(",")])
record_S = np.array([float(value) for value in S_text.split(",")])
our_freq = np.array([witnesses[pair][0] for pair in WITNESSES])
our_S = np.array([unit_S[pair] for pair in WITNESSES])
check(np.abs(our_freq - record_freq).max() < 1e-12
      and recorded(A4_WL, "condensate_diagonal_witness_exact") == "PASS"
      and recorded(A4_PY, "authorT16_condensate_witness") == "PASS",
      f"omega = {freq_text} for the three witnesses, as in the record",
      record=f"{A4_WL}, check condensate_diagonal_witness_exact; {A4_PY}, check "
             "authorT16_condensate_witness")
quotients = record_S / our_S  # S of the record divided by S of unit v1, v2
check(np.abs(quotients / quotients[0] - 1).max() < 1e-9 and np.all(record_S > 0),
      f"S = {S_text} of the record are one common multiple of ours",
      record=f"{A4_WL}, check condensate_diagonal_witness_exact")
report("S of the record divided by S of unit v1, v2 (all three)",
       f"{quotients[0]:.1f}")
```

Out [13]:

```
PASS omega = 4, 3, 4 for the three witnesses, as in the record
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_diagonal_witness_exact;
    Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_witness
PASS S = 51200, 28800, 51200 of the record are one common multiple of ours
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_diagonal_witness_exact
RESULT S of the record divided by S of unit v1, v2 (all three) = 40000.0
```

Text cell:

```
The next cell draws the tensor of the witness $(V, H) = (5, 1)$ at the same point
and with the same colour scale as the generic condensate: only one square is
coloured. (The value of $S$, and hence of $-VS$, depends on the lengths of $v_1$
and $v_2$; here both have length 1, so $S = 1.28$ and $-VS = -6.4$.)
```

In [14]:

```
T_witness = condensate_tensor(witness_columns[(5.0, 1.0)], **POINT)
fig, ax = plt.subplots(figsize=(6.6, 5.6))
picture = heat_map(ax, T_witness, "Witness condensate: $T^\\nu{}_\\mu$", SCALE)
fig.colorbar(picture, ax=ax, label="value (energy per unit volume)", extend="both")
save_figure(fig, "witness_condensate",
            "Heat map of the 64 entries $T^\\nu{}_\\mu$ of the exact witness "
```

```
            "condensate $\\Phi = e^{-4ix_4}\\Phi_0$ of the Revision record with "
            "$V = 5$, $H = 1$, $\\lambda = 0$, at the same point "
            "($a_4 = 0.5$, $a_4' = 0.25$, $z = \\pi/4$) and with the same colour "
            "scale as the generic condensate, in units of energy per unit volume. "
            "All 15 three-gamma bilinears of $\\Phi_0$ vanish, so every off-diagonal "
            "entry is zero (by the record, at every point of every history): this "
            "condensate is a perfect fluid at rest, with $T^{x_4}{}_{x_4} = -VS$ "
            "and zero pressure.")
check(len(off_diagonal_pairs(T_witness)) == 0 and abs(T_witness[4, 4]) > 0.1,
      "the drawn witness tensor is diagonal with T^x4_x4 = -V S != 0")
```

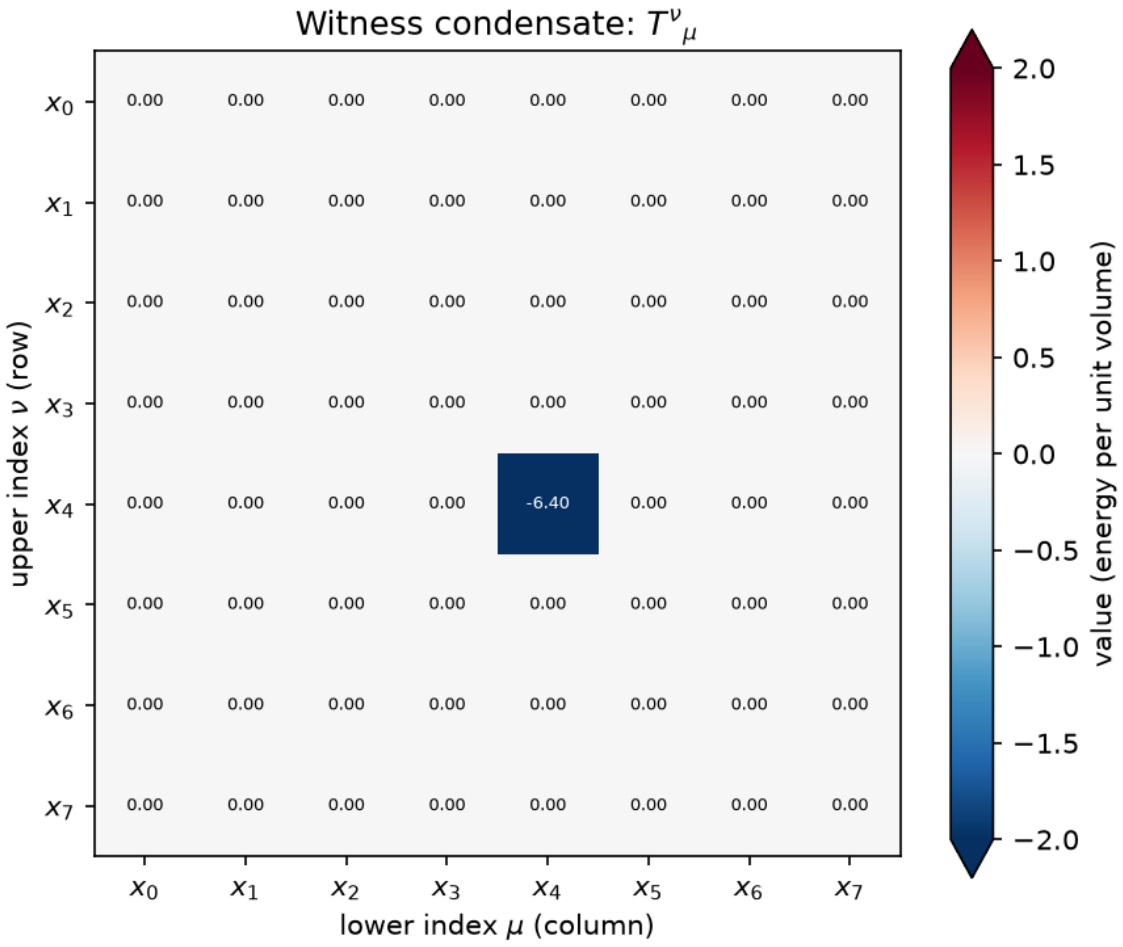

Figure 264: Heat map of the 64 entries $T^{\nu}{}_{\mu}$ of the exact witness condensate $\Phi = e^{-4ix_4}\Phi_0$ of the Revision record with $V = 5$, $H = 1$, $\lambda = 0$, at the same point ($a_4 = 0.5$, $a_4' = 0.25$, $z = \pi/4$) and with the same colour scale as the generic condensate, in units of energy per unit volume. All 15 three-gamma bilinears of Φ_0 vanish, so every off-diagonal entry is zero (by the record, at every point of every history): this condensate is a perfect fluid at rest, with $T^{x_4}{}_{x_4} = -VS$ and zero pressure. (Notebook 09c, figure 3.)

Out [14]:

```
Figure 09c.3 saved as Revision/textbook/figures/09c_3_witness_condensate.png
```

```
PASS the drawn witness tensor is diagonal with T^x4_x4 = -V S != 0
```

Text cell:

```
## 9. Conservation of the full tensor, tested with finite differences

The full tensor of a condensate, off-diagonal entries included, must satisfy
$\nabla_\mu T^\mu{}_\nu = 0$. Because the condensate solves the field equation on
every history, the test uses a non-linear one, $a_4 = 0.3x_4 + 0.05x_4^2$
($a_4' = 0.3 + 0.1x_4$), with $m = 1$, $\lambda = 0.5$, $H = 0.25$ and a random
unit column $\chi$. The tensor depends on $x_4$ (through $\Phi$, $a_4$ and $a_4'$)
and on $x_0$ (through $z$); the derivatives $\partial_4 T$ and $\partial_0 T$ are
taken by central finite differences with step $h$.

The next cell computes the 25 Christoffel symbols of the metric with sympy
(the formula $\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}
(\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu}
- \partial_\lambda g_{\mu\nu})$ of a diagonal metric), puts the history into
them and turns each into a fast numpy function of $(x_4, x_0, H)$ with
`sp.lambdify`.
```

In [15]:

```
x = sp.symbols("x0:8", real=True)  # sympy was imported in section 8
Hs = sp.symbols("H", positive=True)
a4_function = sp.Function("a4")(x[4])
zs = 6 * Hs * x[0]
space_entry = sp.exp(2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
extra_entry = -sp.exp(-2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
g = sp.diag(*([sp.cot(zs) ** 2] + [space_entry] * 3 + [-1] + [extra_entry] * 3))
history = sp.Rational(3, 10) * x[4] + sp.Rational(1, 20) * x[4] ** 2  # a4(x4)
christoffel = {}  # (l, i, j) with i <= j -> a numpy function of (x4, x0, H)
for l in range(8):
    for i in range(8):
        for j in range(i, 8):
            value = sp.simplify((sp.diff(g[l, j], x[i]) + sp.diff(g[l, i], x[j])
                                 - sp.diff(g[i, j], x[l])) / (2 * g[l, l]))
            if value != 0:  # put the history in, then make a numpy function
                christoffel[(l, i, j)] = sp.lambdify(
                    (x[4], x[0], Hs), value.subs(a4_function, history).doit(), "numpy")
check(len(christoffel) == 25, "25 nonzero Christoffel symbols (i <= j)")
```

Out [15]:

```
PASS 25 nonzero Christoffel symbols (i <= j)
```

Text cell:

```
The next cell builds the two condensates of the test and the divergence. The
true condensate uses $M = -V\gamma^{(4)} + 3H\gamma^{(4)}\gamma^{(0)}$; the
```

negative control uses $M = -V\gamma^{(4)}$, the condensate of an equation without the gravitational term $3H\gamma^{(0)}$. Its S is also constant, but it does not solve the field equation of the author's metric, so its tensor must not be conserved. The function `field(M, k2, x4)` returns $\Phi(x_4)$ and $\partial_4\Phi = M\Phi$, `tensor_at(M, k2, x4, x0)` the 64 entries of the tensor at a point of the history, and `divergence(M, k2, x4, x0, h)` the eight components of $\nabla_\mu T^\mu{}_\nu$, with the derivatives $\partial_4 T$ and $\partial_0 T$ replaced by central finite differences of step h (the tensor of a condensate depends on x_4 and x_0 only). The cell only defines; it prints the effective mass V of the test.

In [16]:

```
m9, lam9, H9 = 1.0, 0.5, 0.25  # the parameters of this section
chi9 = rng.normal(size=16) + 1j * rng.normal(size=16)
chi9 /= np.linalg.norm(chi9)
V9 = m9 + lam9 * (bar(chi9) @ chi9).real  # the effective mass
M_TRUE = -V9 * g4 + 3 * H9 * g4 @ g0  # the true condensate: M^2 = 9 H^2 - V^2
M_CONTROL = -V9 * g4  # without the term 3 H gamma^(x0): M^2 = -V^2
K2_TRUE, K2_CONTROL = 9 * H9 ** 2 - V9 ** 2, -V9 ** 2


def field(M, k2, x4):
    """Phi(x4) and dPhi/dx4 = M Phi(x4) for Phi(0) = chi9 (M^2 = k2)."""
    k = np.sqrt(complex(k2))
    phi = np.cosh(k * x4) * chi9 + np.sinh(k * x4) / k * (M @ chi9)
    return phi, M @ phi


def tensor_at(M, k2, x4, x0):
    phi, d4 = field(M, k2, x4)
    dphi = np.zeros((8, 16), dtype=complex)
    dphi[4] = d4
    return energy_momentum(phi, dphi, m9, lam9, 0.3 * x4 + 0.05 * x4 ** 2,
                           0.3 + 0.1 * x4, 6 * H9 * x0, H9)


def divergence(M, k2, x4, x0, h):
    """The eight components of nabla_mu T^mu_nu, with central differences of step h,
    and the size of the tensor (its largest entry)."""
    T = tensor_at(M, k2, x4, x0)
    d4T = (tensor_at(M, k2, x4 + h, x0) - tensor_at(M, k2, x4 - h, x0)) / (2 * h)
    d0T = (tensor_at(M, k2, x4, x0 + h) - tensor_at(M, k2, x4, x0 - h)) / (2 * h)
    G = np.zeros((8, 8, 8))  # G[l, i, j] = Gamma^l_ij
    for (l, i, j), function in christoffel.items():
        G[l, i, j] = G[l, j, i] = function(x4, x0, H9)
    result = np.zeros(8)
    for nu in range(8):
        result[nu] = (d4T[4, nu] + d0T[0, nu]  # d_mu T^mu_nu (only x4 and x0)
                      + np.einsum("mml,l->", G, T[:, nu])  # Gamma^mu_mu l T^l_nu
                      - np.einsum("lm,ml->", G[:, :, nu], T))  # Gamma^l_mu nu T^mu_l
    return result, np.abs(T).max()
```

```
report("effective mass V = m + lambda S of the test condensate", f"{V9:.6f}")
```

Out [16]:

```
RESULT effective mass V = m + lambda S of the test condensate = 0.883522
```

Text cell:

```
The next cell computes the largest of the eight components of the divergence for
eleven steps $h$ from $0.2$ down to $0.0002$ at the point $x_4 = 1$, $z = 0.5$,
for the true condensate and for the control. For the true condensate the result
must fall like $h^2$ (each halving of $h$ divides it by about 4): the exact
divergence is zero and only the error of the finite differences is left. For the
control it must approach a nonzero value. The cell also checks the true
condensate at two more points with $h = 0.001$ and $h = 0.0001$: the divergence
must shrink about a hundredfold and end below $10^{-5}$ of the size of the tensor.
```

In [17]:

```
steps = 0.2 / 2.0 ** np.arange(11)  # 0.2, 0.1, ..., about 0.0002
X4_TEST, X8_TEST = 1.0, 0.5 / (6 * H9)  # x4 = 1 and z = 0.5
true_sizes, control_sizes = [], []
for h in steps:
    div_true, size_true = divergence(M_TRUE, K2_TRUE, X4_TEST, X8_TEST, h)
    div_control, _ = divergence(M_CONTROL, K2_CONTROL, X4_TEST, X8_TEST, h)
    true_sizes.append(np.abs(div_true).max())
    control_sizes.append(np.abs(div_control).max())
true_sizes, control_sizes = np.array(true_sizes), np.array(control_sizes)
ratios = true_sizes[:6] / true_sizes[1:7]  # halving h from 0.2 to 0.003125
check(np.all((ratios > 3.6) & (ratios < 4.4)),
      "true condensate: halving h divides the divergence by about 4 (error h^2)")
check(true_sizes[8] < 1e-6 * size_true,
      "true condensate: the divergence falls below 1e-6 of the tensor's size")
check(abs(control_sizes[-1] / control_sizes[-2] - 1) < 1e-3
      and control_sizes[-1] > 1e-2 * size_true,
      "control: the divergence tends to a nonzero value (it is not conserved)")
for x4_point, z_point in ((2.0, 1.0), (0.5, 1.2)):
    x0_point = z_point / (6 * H9)
    coarse, size_other = divergence(M_TRUE, K2_TRUE, x4_point, x0_point, 1e-3)
    fine, _ = divergence(M_TRUE, K2_TRUE, x4_point, x0_point, 1e-4)
    shrink = np.abs(coarse).max() / np.abs(fine).max()  # about 10^2 for an h^2 error
    check(90 < shrink < 110 and np.abs(fine).max() < 1e-5 * size_other,
          f"true condensate conserved at x4 = {x4_point}, z = {z_point} "
          "(h = 0.001 and 0.0001)")
WL = "Revision/theory/reports/wolfram-field-theory.json"
check(recorded(WL, "exact_solution_nonlinear_homogeneous_C") == "PASS"
      and recorded(WL, "conservation_on_shell_general") == "PASS",
      "the record proves the conservation that these numbers confirm",
      record=f"{WL}, checks exact_solution_nonlinear_homogeneous_C and "
             "conservation_on_shell_general")
```

```
for h, t_size, c_size in zip(steps[::2], true_sizes[::2], control_sizes[::2]):
    say(f"h = {h:.5f}:  true condensate {t_size:.2e},  control {c_size:.4f}")
```

Out [17]:

```
PASS true condensate: halving h divides the divergence by about 4 (error h^2)
PASS true condensate: the divergence falls below 1e-6 of the tensor's size
PASS control: the divergence tends to a nonzero value (it is not conserved)
PASS true condensate conserved at x4 = 2.0, z = 1.0 (h = 0.001 and 0.0001)
PASS true condensate conserved at x4 = 0.5, z = 1.2 (h = 0.001 and 0.0001)
PASS the record proves the conservation that these numbers confirm
     reproduces Revision/theory/reports/wolfram-field-theory.json, checks
    exact_solution_nonlinear_homogeneous_C and conservation_on_shell_general
h = 0.20000:  true condensate 5.06e-03,  control 0.0729
h = 0.05000:  true condensate 2.94e-04,  control 0.0679
h = 0.01250:  true condensate 1.83e-05,  control 0.0676
h = 0.00313:  true condensate 1.14e-06,  control 0.0676
h = 0.00078:  true condensate 7.16e-08,  control 0.0676
h = 0.00020:  true condensate 4.47e-09,  control 0.0676
```

Text cell:

```
The next cell draws the two series on logarithmic axes, with a line proportional
to $h^2$ for comparison.
```

In [18]:

```
fig, ax = plt.subplots()
ax.loglog(steps, true_sizes, "o-", label="true condensate")
ax.loglog(steps, control_sizes, "s-", label="control (no $3H\\gamma^{(0)}$ term)")
ax.loglog(steps, true_sizes[0] * (steps / steps[0]) ** 2, "k:",
          label="proportional to $h^2$")
ax.set_xlabel("finite-difference step $h$")
ax.set_ylabel("largest $|\\nabla_\\mu T^\\mu{}_\\nu|$")
ax.set_title("Conservation of the full tensor of a condensate")
ax.legend()
save_figure(fig, "conservation_convergence",
            "The largest of the eight components of the covariant divergence "
            "$\\nabla_\\mu T^\\mu{}_\\nu$ of the full tensor of a condensate "
            "($m = 1$, $\\lambda = 0.5$, $H = 0.25$, history "
            "$a_4 = 0.3x_4 + 0.05x_4^2$, point $x_4 = 1$, $z = 0.5$), computed with "
            "central finite differences of step $h$, against $h$ from $0.2$ to "
            "$0.0002$; both axes logarithmic, the divergence in units of energy per "
            "unit volume per unit length. For the true condensate (circles) it "
            "falls like $h^2$ (dotted line): the exact divergence is zero. For the "
            "control (squares), which leaves out the term $3H\\gamma^{(0)}$ of the "
            "field equation, it stays near $0.07$: its tensor is not conserved.")
```

Out [18]:

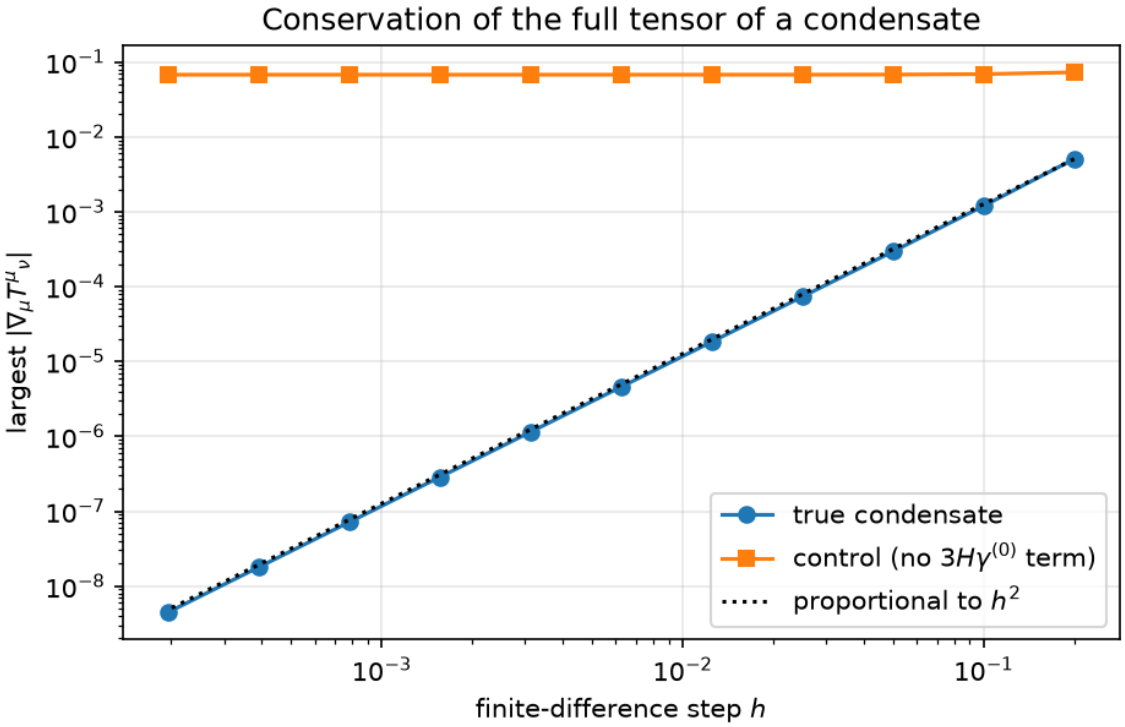

Figure 265: The largest of the eight components of the covariant divergence $\nabla_\mu T^\mu{}_\nu$ of the full tensor of a condensate ($m = 1$, $\lambda = 0.5$, $H = 0.25$, history $a_4 = 0.3x_4 + 0.05x_4^2$, point $x_4 = 1$, $z = 0.5$), computed with central finite differences of step h, against h from 0.2 to 0.0002; both axes logarithmic, the divergence in units of energy per unit volume per unit length. For the true condensate (circles) it falls like h^2 (dotted line): the exact divergence is zero. For the control (squares), which leaves out the term $3H\gamma^{(0)}$ of the field equation, it stays near 0.07: its tensor is not conserved. (Notebook 09c, figure 4.)

```
Figure 09c.4 saved as Revision/textbook/figures/09c_4_conservation_convergence.png
```

Text cell:

```
The next cell draws the eight components separately at the step $h = 0.001$, for
the true condensate and for the control, as bars on a logarithmic axis, and
checks what the picture shows: the control fails in the six momentum components
(along 3-space and the extra times), while its energy component $\nu = x_4$ and
its hidden component $\nu = x_0$ vanish. The control's energy density and
pressures are the same as those of the true condensate (its $S$ is constant and
$p_3 = p_t$), so its energy balance holds; it is the momentum balance that the
missing term $3H\gamma^{(0)}$ breaks.
```

In [19]:

```
div_true, _ = divergence(M_TRUE, K2_TRUE, X4_TEST, X8_TEST, 1e-3)
div_control, _ = divergence(M_CONTROL, K2_CONTROL, X4_TEST, X8_TEST, 1e-3)
positions = np.arange(8)
fig, ax = plt.subplots()
floor = 1e-16  # zero components are drawn at this height on the logarithmic axis
ax.bar(positions - 0.2, np.abs(div_true) + floor, width=0.4, label="true condensate")
ax.bar(positions + 0.2, np.abs(div_control) + floor, width=0.4,
       label="control (no $3H\\gamma^{(0)}$ term)")
ax.set_yscale("log")
ax.set_ylim(1e-16, 10.0)
ax.set_xticks(positions, [f"$\\nu = {n[0]}_{n[1]}$" for n in NAMES], fontsize=8)
ax.set_ylabel("$|\\nabla_\\mu T^\\mu{}_\\nu|$ (with $h = 0.001$)")
ax.set_title("The eight components of the divergence")
ax.legend(fontsize=8)
save_figure(fig, "divergence_components",
            "The eight components $|\\nabla_\\mu T^\\mu{}_\\nu|$, $\\nu = x_0, \\dots, "
            "x_7$, of the divergence of the full tensor of the condensate of the "
            "previous figure, with finite differences of step $h = 0.001$, on a "
            "logarithmic axis (components that vanish exactly are drawn at "
            "$10^{-16}$). The true condensate (left bars) has every component at the "
            "level of the finite-difference error, below $10^{-6}$. The control "
            "(right bars) has its six momentum components $\\nu = x_1, x_2, x_3, "
            "x_5, x_6, x_7$ between about $0.01$ and $0.07$, while its components "
            "$\\nu = x_4$ and $x_0$ vanish like those of the true condensate: the "
            "control keeps the energy balance but breaks the momentum balance along "
            "3-space and the extra times.")
momentum = [1, 2, 3, 5, 6, 7]  # the components nu = x1, x2, x3, x5, x6, x7
check(np.abs(div_control).max() > 1e3 * np.abs(div_true).max()
      and np.abs(div_control[momentum]).min() > 1e-3
      and max(abs(div_control[4]), abs(div_control[0])) < 1e-9,
      "control: its six momentum components are nonzero (over 1000 times the true "
      "condensate's), its x4 and x0 components vanish")
```

Out [19]:

```
Figure 09c.5 saved as Revision/textbook/figures/09c_5_divergence_components.png
PASS control: its six momentum components are nonzero (over 1000 times the true
    condensate's), its x4 and x0 components vanish
```

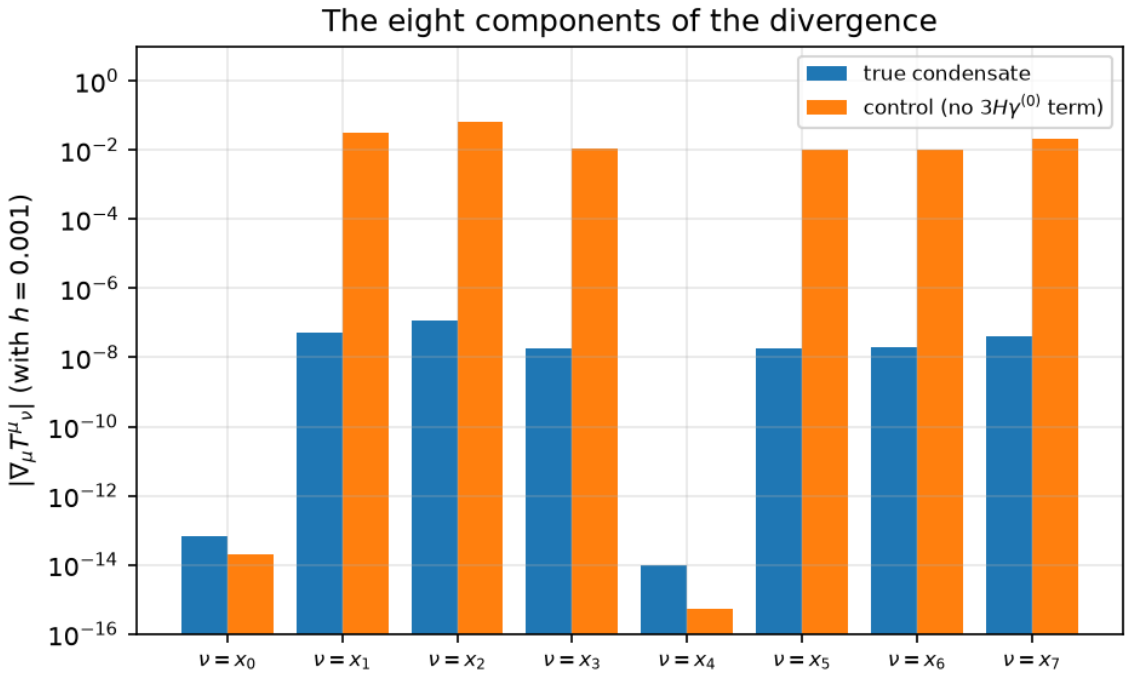

Figure 266: The eight components $|\nabla_\mu T^\mu{}_\nu|$, $\nu = x_0, \ldots, x_7$, of the divergence of the full tensor of the condensate of the previous figure, with finite differences of step $h = 0.001$, on a logarithmic axis (components that vanish exactly are drawn at 10^{-16}). The true condensate (left bars) has every component at the level of the finite-difference error, below 10^{-6}. The control (right bars) has its six momentum components $\nu = x_1, x_2, x_3, x_5, x_6, x_7$ between about 0.01 and 0.07, while its components $\nu = x_4$ and x_0 vanish like those of the true condensate: the control keeps the energy balance but breaks the momentum balance along 3-space and the extra times. (Notebook 09c, figure 5.)

Text cell:

```
## 10. The last check

The last cell checks that the five figures are saved with their captions and
prints the number of checks that passed.
```

In [20]:

```
captions = json.loads(output_file(CAPTION_FILE).read_text(encoding="utf-8"))
expected_files = ["09c_1_generic_condensate.png", "09c_2_offdiagonal_entries.png",
                  "09c_3_witness_condensate.png",
                  "09c_4_conservation_convergence.png",
                  "09c_5_divergence_components.png"]
check(sorted(captions) == expected_files and all(
    output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected_files),
    "the five figures of this notebook are saved and captioned")
all_checks_passed()
```

Out [20]:

```
PASS the five figures of this notebook are saved and captioned
ALL 25 CHECKS PASSED (notebook 09c)
```

Text cell:

```
## 11. What this notebook showed

- A generic exact condensate of dirac16complex00 has the diagonal of a perfect
  fluid ($-VS$ at $x_4$, the same pressure elsewhere, zero for $\lambda = 0$) but
  42 nonzero off-diagonal entries, all produced by the spin connection: the 12
  flows $T^{x_4}{}_{x_j}$ and $T^{x_j}{}_{x_4}$ ($x_j$ one of the six directions
  of 3-space and of the extra times), which come from $H$, and 30 more that are
  proportional to the deflation rate $a_4'$. Its $x_4$-$x_0$ entries vanish.
- Each off-diagonal entry is a fixed multiple of one of 15 three-gamma bilinears
  (the record's statement, confirmed with three random columns), and the
  multiple is exactly minus the coefficient that the record lists for the
  entry (all 42 entries, ten groups of equal coefficients, four columns).
- The record's witnesses, built from joint eigenvectors of
  $\gamma^{(1)}\gamma^{(5)}$, $\gamma^{(2)}\gamma^{(6)}$, $\gamma^{(3)}\gamma^{(7)}$,
  make all 15 bilinears vanish: their tensor is diagonal (the record shows it for
  every $a_4$ and $a_4'$; this notebook checked 36 points). The space of the
  eigenvalue $-i\omega$ has dimension 8 and each of its eight sectors dimension
  1, so $v_1$ and $v_2$ are unique up to a factor of size 1. Their
  $S = 2|v_1^\dagger Cv_2|^2$ is never negative; the record's frequencies
  $\omega = 4, 3, 4$ are reproduced exactly and its three values of $S$ up to one
  common factor (the factor printed above; the record's columns are not of
  length 1).
  On the deflating history only condensates whose 15 bilinears all vanish, such
  as these, meet the condition of the $a_4$ field equations that the
```

```
  off-diagonal entries of the source vanish (one condition among several).
- The full tensor of a condensate is conserved, $\nabla_\mu T^\mu{}_\nu = 0$,
  along a curved history: the finite-difference divergence falls like $h^2$. A
  configuration that leaves out the gravitational term $3H\gamma^{(0)}$ keeps the
  energy balance but breaks the six momentum balances along 3-space and the
  extra times: the term is needed for the conservation of the tensor.
- Status: the statements are PROVED in the Revision record (exact, Wolfram and
  sympy); this notebook confirms them numerically (COMPUTED). The formula
  $S = 2\omega^2/V^2$ for unit $v_1$, $v_2$ is an observation for the three
  witnesses (COMPUTED, not proved). The parameter values are choices of this
  notebook.
```

9.30 Line-by-line walk-through of Notebook 09c

The notebook has 20 code cells, In [1] to In [20]. As in Section 9.17, long figure captions are shortened to a line "...)" and docstrings are left out here; the captions are printed in full in Section 9.29.

In [1], the set-up cell. It is word for word the set-up cell of Notebook 09a, explained line by line in Section 9.17, except that its comment lines are the run instructions of Notebook 09c (Section 9.28) and its line `NOTEBOOK_ID = "09c"` names this notebook. It prints `Set-up of notebook 09c complete:  repository folder found, helpers defined.`

In [2], the gammas and two report helpers.

```
import numpy as np  # arrays of numbers, matrices, linear algebra

fixture = json.loads(
    repository_file("Revision/algebra/gammas.json").read_text(encoding="utf-8"))
gamma = [np.array(rows, dtype=float) for rows in fixture["gamma"]]  # x0, ..., x7
eta = np.array(fixture["eta"], dtype=float)  # (+1, +1, +1, +1, -1, -1, -1, -1)
C = np.array(fixture["C"], dtype=float)  # gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
I16 = np.eye(16)  # the 16 x 16 unit matrix
g4, g0 = gamma[4], gamma[0]  # gamma^(x4) and gamma^(x0)
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # the author's coordinates
```

These lines read the eight real gammas, η and C from the fixture, as in In [2] of Notebooks 09a and 09b, and name $\gamma^{(4)}$, $\gamma^{(0)}$, the unit matrix and the coordinates.

```
def record_entry(path, name):
    """The entry (name, verdict, detail) of the check name in the report path."""
    report_data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    for item in report_data["checks"]:
        if item["name"] == name:
            return item
    raise KeyError(f"{path} has no check {name}")


def recorded(path, name):
    """The verdict (PASS or FAIL) of the check name in the Revision report path."""
    return record_entry(path, name)["verdict"].upper()  # some reports write "pass"
```

```
say("The gammas, C and the two report helpers are ready.")
```

`record_entry` returns the whole entry of one check of a report (its name, its verdict and its **detail**, a text in which the record writes what it computed); In [13] reads numbers out of a detail text. `recorded` returns only the verdict, as in the other notebooks. The cell prints one line.

In [3], the geometry at a point.

```
def frame_factors(a4, z):
    """The eight factors f_a of the diagonal vielbein, in the order x0, ..., x7."""
    space = np.exp(a4) * np.sin(z) ** (1 / 6)  # 3-space: e^a4 sin^(1/6) z
    extra = np.exp(-a4) * np.sin(z) ** (1 / 6)  # extra times: e^(-a4) sin^(1/6) z
    return np.array([1 / np.tan(z)] + [space] * 3 + [1.0] + [extra] * 3)


def spin_connection(a4, a4p, z, H):
    """The eight matrices Omega_mu (record formula Omega_components)."""
    s = np.sin(z) ** (1 / 6)  # sin^(1/6) z
    omega = [np.zeros((16, 16)) for _ in range(8)]  # Omega_x4 = Omega_x0 = 0
    for i in range(1, 4):  # the three 3-space directions x1, x2, x3
        omega[i] = 0.5 * np.exp(a4) * s * (a4p * gamma[i] @ g4 + H * gamma[i] @ g0)
    for t in range(5, 8):  # the three extra times x5, x6, x7
        omega[t] = -0.5 * np.exp(-a4) * s * (a4p * g4 @ gamma[t] + H * gamma[t] @ g0)
    return omega


say("frame_factors and spin_connection are defined.")
```

The same two functions as in In [3] and In [5] of Notebook 09a (explained in Section 9.17), with one change: H is now an argument of `spin_connection`, because this notebook uses several values of H.

In [4], the Dirac adjoint and the tensor at a point.

```
def bar(vector):
    """The Dirac adjoint vector^dagger C."""
    return vector.conj() @ C


def energy_momentum(phi, dphi, m, lam, a4, a4p, z, H):
    """The 64 entries T[nu, mu] = T^nu_mu at one point (real parts)."""
    f = frame_factors(a4, z)
    g_values = eta * f ** 2  # g_mumu
    omega = spin_connection(a4, a4p, z, H)
    gup = [gamma[mu] / f[mu] for mu in range(8)]  # gamma^mu
    S = (bar(phi) @ phi).real
    D = [dphi[mu] + omega[mu] @ phi for mu in range(8)]  # D_mu Phi
    Dbar = [bar(dphi[mu]) - bar(phi) @ omega[mu] for mu in range(8)]  # D_mu Phibar
    L0 = 0.5 * sum(bar(phi) @ gup[mu] @ D[mu] - Dbar[mu] @ gup[mu] @ phi
                   for mu in range(8)) - m * S - lam / 2 * S ** 2
```

`bar` is the Dirac adjoint, as before. This `energy_momentum` takes the jet and the parameters m, λ, a_4, a_4', z and H, and builds everything at the point itself: the factors, the metric entries, the spin connection, the coordinate gammas, S, the covariant derivatives and L_0, each line as in In [6] of Notebook 09a.

```
    T = np.zeros((8, 8), dtype=complex)
    for nu in range(8):
        for mu in range(8):
            bracket = (bar(phi) @ gup[nu] @ D[mu] - Dbar[mu] @ gup[nu] @ phi
                       + g_values[mu] / g_values[nu]  # gamma_mu D^nu
                       * (bar(phi) @ gup[mu] @ D[nu] - Dbar[nu] @ gup[mu] @ phi))
            T[nu, mu] = (nu == mu) * L0 - bracket / 4
    if np.abs(T.imag).max() > 1e-9 * max(1.0, np.abs(T).max()):
        raise ValueError("the tensor is not real")  # it must be real
    return T.real


say("bar and energy_momentum are defined.")
```

The 64 entries are the formula of Section 9.6, with $\gamma_\mu D^\nu$ written as $(g_{\mu\mu}/g_{\nu\nu})\gamma^\mu D_\nu$. The function stops with a `ValueError` if the imaginary parts exceed rounding noise (the tensor must be real, Section 9.6), and otherwise returns the real parts.

In [5], the tensor of a generic condensate.

```
V, H = 5.0, 1.0  # effective mass (m = 5, lambda = 0) and the author's constant
POINT = {"a4": 0.5, "a4p": 0.25, "z": np.pi / 4}  # a point of a deflating history
rng = np.random.default_rng(11)  # a fixed seed
chi = rng.normal(size=16) + 1j * rng.normal(size=16)
chi /= np.linalg.norm(chi)  # length 1
```

The example has $V = 5$ (with $m = 5$ and $\lambda = 0$) and $H = 1$, so $V^2 > 9H^2$ and the condensate oscillates. `POINT` is a dictionary with the point $a_4 = 0.5$, $a_4' = 0.25$, $z = \pi/4$. The column χ is random (seed 11) and divided by its length.

```
def condensate_tensor(column, a4, a4p, z, V_value=V, H_value=H):
    """T of the condensate (lambda = 0, m = V_value) whose jet at the point is
    Phi = column and d4 Phi = A column, A = -gamma^(x4) (V - 3 H gamma^(x0))."""
    A_value = -g4 @ (V_value * I16 - 3 * H_value * g0)
    dphi = np.zeros((8, 16), dtype=complex)
    dphi[4] = A_value @ column  # the only nonzero derivative
    return energy_momentum(column, dphi, V_value, 0.0, a4, a4p, z, H_value)
```

The jet of a condensate at a point is $\Phi = \chi$ and $\partial_4\Phi = A\chi$ (the solution at $x_4 = 0$; at any other time the solution is another column with the same S, so nothing is lost), all other derivatives zero. `V_value=V` gives an argument a **default value**, used when the call leaves it out. With $\lambda = 0$ the mass m equals V.

```
def off_diagonal_pairs(T):
    """The ordered pairs (nu, mu), nu != mu, whose entry is not zero."""
```

```
    tolerance = 1e-10 * max(1.0, np.abs(T).max())
    return [(nu, mu) for nu in range(8) for mu in range(8)
            if nu != mu and abs(T[nu, mu]) > tolerance]
```

This function lists the pairs (ν, μ) off the diagonal whose entry is larger than rounding noise (10^{-10} times the largest entry, or 10^{-10}).

```
T_generic = condensate_tensor(chi, **POINT)
S_chi = (bar(chi) @ chi).real
expected_diagonal = np.zeros(8)
expected_diagonal[4] = -V * S_chi
A4_WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
A4_PY = "Revision/field_equations_a4/reports/python-a4-report.json"
check(np.abs(np.diag(T_generic) - expected_diagonal).max() < 1e-12
      and abs(T_generic[4, 0]) < 1e-12 and abs(T_generic[0, 4]) < 1e-12
      and recorded(A4_WL, "condensate_kinetic_tensor_diagonal") == "PASS"
      and recorded(A4_PY, "authorT16_condensate_kinetic_diagonal") == "PASS",
      "diagonal (0, 0, 0, 0, -V S, 0, 0, 0) and T^x4_x0 = T^x0_x4 = 0",
      record=f"{A4_WL}, check condensate_kinetic_tensor_diagonal; {A4_PY}, check "
             "authorT16_condensate_kinetic_diagonal")
```

`**POINT` hands the three entries of the dictionary over as the named arguments `a4`, `a4p` and `z`. With $\lambda = 0$, $L_0 = SU' - U = 0$ on shell, so the diagonal of Section 9.26 is $(0, 0, 0, -VS, 0, 0, 0, 0)$; the check confirms it, and $T^{x_4}{}_{x_0} = T^{x_0}{}_{x_4} = 0$.

```
pairs = off_diagonal_pairs(T_generic)
# the same condensate and point with a4p = 0 (a4 not changing at this moment)
pairs_constant_a4 = off_diagonal_pairs(
    condensate_tensor(chi, POINT["a4"], 0.0, POINT["z"]))
check(len(pairs) == 42, "a generic condensate has 42 nonzero off-diagonal entries")
momentum_flows = sorted([(4, i) for i in (1, 2, 3, 5, 6, 7)]
                        + [(i, 4) for i in (1, 2, 3, 5, 6, 7)])
check(sorted(pairs_constant_a4) == momentum_flows,
      "with a4p = 0 only the 12 flows T^x4_xj, T^xj_x4 (j = 1, 2, 3, 5, 6, 7) remain")
report("S of the random column", f"{S_chi:.6f}")
report("T^x4_x4 = -V S", f"{T_generic[4, 4]:.6f}")
```

The cell counts the nonzero entries off the diagonal (42) and repeats the count with $a_4' = 0$: then exactly the 12 pairs (x_4, x_j) and (x_j, x_4) with x_j one of the six directions of 3-space and the extra times remain (`momentum_flows`). It prints $S = 0.293667$ and $T^{x_4}{}_{x_4} = -VS = -1.468334$.

In [6], the heat map of the generic condensate (Figure 09c.1).

```
def heat_map(ax, T, title, largest):
    """Draw T as coloured squares with the value printed in each."""
    picture = ax.imshow(T, cmap="RdBu_r", vmin=-largest, vmax=largest)
    for nu in range(8):
        for mu in range(8):
            text = f"{T[nu, mu]:.2f}"
            if text == "-0.00":  # a tiny negative number: print it as 0.00
```

```
                text = "0.00"
            # white digits on dark squares, black digits on light ones
            colour = "white" if abs(T[nu, mu]) > 0.6 * largest else "black"
            ax.text(mu, nu, text, ha="center", va="center", fontsize=6,
                    color=colour)
    ticks = [f"${n[0]}_{n[1]}$" for n in NAMES]
    ax.set_xticks(range(8), ticks)
    ax.set_yticks(range(8), ticks)
    ax.set_xlabel("lower index $\\mu$ (column)")
    ax.set_ylabel("upper index $\\nu$ (row)")
    ax.set_title(title)
    ax.grid(False)
    return picture
```

`heat_map` draws a table as in In [8] of Notebook 09a, with two decimals in each square; a rounding remainder such as -0.0000001 would print as `-0.00`, which the `if` replaces by `0.00`. The function returns the picture, for the colour bar.

```
SCALE = 2.0  # the colour scale runs from -2 to 2 in both heat maps of this notebook
fig, ax = plt.subplots(figsize=(6.6, 5.6))
picture = heat_map(ax, T_generic, "Generic condensate: $T^\\nu{}_\\mu$", SCALE)
fig.colorbar(picture, ax=ax, label="value (energy per unit volume)", extend="both")
save_figure(fig, "generic_condensate",
            "Heat map of the 64 entries $T^\\nu{}_\\mu$ (row $\\nu$, column $\\mu$, "
            ...)
```

Both heat maps of the notebook use the fixed colour scale from -2 to 2, so they can be compared; `extend="both"` adds arrows to the colour bar for values beyond the scale. **What Figure 09c.1 shows.** The 64 entries of the generic condensate's tensor, in units of energy per unit volume. The only diagonal entry that is not zero is $-VS = -1.47$ at (x_4, x_4); 42 squares off the diagonal are coloured, all of them produced by the spin connection; the squares (x_4, x_0) and (x_0, x_4) are zero, and so are the squares inside 3-space and inside the extra times. The student should see that equal pressures do not make a condensate a perfect fluid.

In [7], the dependence on the deflation rate (Figure 09c.2).

```
rates = np.linspace(-1.0, 1.0, 41)  # values of a4p
entries = np.array([[T[4, 1], T[1, 5], T[1, 0]] for T in (
    condensate_tensor(chi, POINT["a4"], rate, POINT["z"]) for rate in rates)])
check(np.abs(entries[:, 0] - entries[20, 0]).max() < 1e-12 and abs(entries[20, 0]) > 0.1,
      "T^x4_x1 does not depend on a4p and is not zero")
slopes = entries[-1, 1:] / rates[-1]  # the slope of each of the other two entries
check(np.abs(entries[:, 1:] - np.outer(rates, slopes)).max() < 1e-12
      and np.abs(slopes).min() > 1e-3,
      "T^x1_x5 and T^x1_x0 are proportional to a4p (zero only at a4p = 0)")
report("T^x4_x1", f"{entries[20, 0]:.6f}")
report("slopes d T^x1_x5/d a4p and d T^x1_x0/d a4p",
       f"{slopes[0]:.6f} and {slopes[1]:.6f}")
```

For 41 values of a_4' from -1 to 1 the cell computes the tensor of the same condensate at the same point and keeps three entries: $T^{x_4}{}_{x_1}$ (`T[4, 1]`), $T^{x_1}{}_{x_5}$ (`T[1, 5]`) and $T^{x_1}{}_{x_0}$ (`T[1, 0]`). The first

check confirms that $T^{x_4}{}_{x_1}$ is the same for every a_4' (it equals its value at the middle of the list, `entries[20, 0]`, where $a_4' = 0$) and is not zero. The second computes the slopes of the other two entries from the last value ($a_4' = 1$) and confirms that each entry is its slope times a_4' at all 41 values (`np.outer(rates, slopes)` is the table of all products), with slopes that are not zero. It prints $T^{x_4}{}_{x_1} = 0.835532$ and the slopes -0.012942 and -0.049288.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 3.9))
fig.subplots_adjust(wspace=0.35)
left.plot(rates, entries[:, 0], label="$T^{x_4}{}_{x_1}$ (momentum flow)")
left.set_ylim(0.0, 1.2 * entries[20, 0])
left.set_xlabel("deflation rate $a_4'$")
left.set_ylabel("entry (energy per unit volume)")
left.set_title("Comes from $H$: independent of $a_4'$")
left.legend(fontsize=8)
right.plot(rates, entries[:, 1], label="$T^{x_1}{}_{x_5}$")
right.plot(rates, entries[:, 2], "--", label="$T^{x_1}{}_{x_0}$")
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("deflation rate $a_4'$")
right.set_title("Proportional to $a_4'$")
right.legend(fontsize=8)
save_figure(fig, "offdiagonal_entries",
            "Three off-diagonal entries of the generic condensate of the previous "
            ...)
```

What Figure 09c.2 shows. Left: the momentum flow $T^{x_4}{}_{x_1}$ (energy per unit volume) against a_4' is a horizontal line at 0.835532, the value printed by the cell: it comes from the H part of the spin connection and from the term $3H\gamma^{(4)}\gamma^{(0)}$ of the condensate's time derivative (Section 9.26). Right: $T^{x_1}{}_{x_5}$ (solid) and $T^{x_1}{}_{x_0}$ (dashed) are straight lines through zero. The student should see that 30 of the 42 entries exist only because $a_4' \neq 0$, as the hand computation of $T^{x_1}{}_{x_5}$ in Section 9.26 showed.

In [8], the 15 three-gamma bilinears.

```
def bilinear_directions(nu, mu):
    """The three directions (a, b, c) of the bilinear that carries T^nu_mu."""
    directions = {nu, mu}
    if 4 in directions and 0 not in directions:
        directions.add(0)  # add x0
    elif 0 in directions and 4 not in directions:
        directions.add(4)  # add x4
    elif 4 not in directions and 0 not in directions:
        directions.add(4)  # add x4
    return tuple(sorted(directions))


def bilinear(column, directions):
    a, b, c = directions
    return bar(column) @ gamma[a] @ gamma[b] @ gamma[c] @ column
```

`bilinear_directions` encodes the rule of the record: the bilinear of the entry (ν, μ) has the directions ν, μ and the missing one of x_4, x_0 (or x_4 when neither occurs). `{nu, mu}` is a **set** (a collection without order and without repetitions); `.add` puts an element in; `elif` means "else

if"; `tuple(sorted(...))` returns the three directions in increasing order. `bilinear` computes $\bar{\Phi}\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$.

```
columns = [rng.normal(size=16) + 1j * rng.normal(size=16) for _ in range(3)]
tensors = [condensate_tensor(column, **POINT) for column in columns]
worst_ratio, used = 0.0, set()
for nu, mu in pairs:
    directions = bilinear_directions(nu, mu)
    used.add(directions)
    ratios = [tensors[n][nu, mu] / bilinear(columns[n], directions) for n in range(3)]
    worst_ratio = max(worst_ratio,
                      max(abs(r - ratios[0]) for r in ratios) / abs(ratios[0]))
```

Three more random columns give three more condensates at the same point. For each of the 42 pairs the loop divides the entry by its bilinear for each of the three columns; if the entry is a fixed multiple of the bilinear, the three ratios are equal. `worst_ratio` keeps the largest relative spread, and `used` collects the bilinears that occur.

```
check(worst_ratio < 1e-9 and len(used) == 15
      and recorded(A4_WL, "condensate_offdiagonal_are_three_gamma_bilinears") == "PASS"
      and recorded(A4_PY, "authorT16_condensate_offdiagonal_three_gamma") == "PASS",
      "each of the 42 entries is a fixed multiple of one of 15 three-gamma bilinears",
      record=f"{A4_WL}, check condensate_offdiagonal_are_three_gamma_bilinears; "
             f"{A4_PY}, check authorT16_condensate_offdiagonal_three_gamma")
for directions in sorted(used):
    say("bilinear Phibar gamma^(" + ") gamma^(".join(NAMES[d] for d in directions)
        + ") Phi")
```

The check confirms equal ratios and exactly 15 bilinears; the loop prints them: the nine $\bar{\Phi}\gamma^{(x_i)}\gamma^{(x_4)}\gamma^{(x_t)}\Phi$ and the six $\bar{\Phi}\gamma^{(x_j)}\gamma^{(x_4)}\gamma^{(x_0)}\Phi$ (written with the directions in increasing order).

In [9], the coefficients of the record.

```
import re  # regular expressions: patterns that find pieces of a text

import sympy as sp  # exact algebra with symbols

A4_RECORD = "Revision/field_equations_a4/a4-equations.json"
record_entries = json.loads(repository_file(A4_RECORD).read_text(encoding="utf-8"))[
    "fields"]["dirac16complex00"]["offDiagonalKinetic"]
cc, a4v, ad1, H_symbol = sp.symbols("cc a4v ad1 H", real=True)
AT_POINT = {cc: 1 / np.tan(POINT["z"]), a4v: POINT["a4"], ad1: POINT["a4p"],
            H_symbol: H}
worst_coefficient, groups = 0.0, {}  # groups: formula text -> its first entry
```

A **regular expression** is a pattern that finds pieces of a text; the module `re` provides them. The record file holds, under `fields`, `dirac16complex00`, `offDiagonalKinetic`, the list of the 42 entries with their bilinears and coefficients. The coefficients are Wolfram-Language formulas in the names `cc` ($\cot z$), `a4v` (a_4), `ad1` (a_4') and `H`; `AT_POINT` gives these names the values of the point.

```
for item in record_entries:
    # "K^x1_x4 (symmetrised)": upper index x1 (row nu), lower index x4 (column mu)
    nu, mu = [int(digit) for digit in re.findall(r"x(\d)", item["component"])]
    term = item["terms"][0]  # every entry of the record has exactly one term
    directions = [int(digit) for digit in re.findall(r"x(\d)", term["bilinear"])]
    formula = term["coefficient"]["input"]  # the Wolfram Language text
```

For each entry of the record, `re.findall(r"x(\d)", text)` finds every `x` followed by a digit and returns the digits (`\d` means one digit, the parentheses mark the part to return; the `r` before the string keeps the backslash). The component name gives ν and μ and the bilinear's name gives its three directions, all counted from 0. `formula` is the coefficient's text.

```
    coefficient = float(sp.sympify(
        formula.replace("E^", "E**").replace("^", "**"),
        locals={"E": sp.E, "cc": cc, "a4v": a4v, "ad1": ad1, "H": H_symbol},
    ).subs(AT_POINT))
    for column, T in zip([chi] + columns, [T_generic] + tensors):
        predicted = -coefficient * bilinear(column, tuple(directions))
        worst_coefficient = max(worst_coefficient, abs(T[nu, mu] - predicted))
    groups.setdefault(formula, (nu, mu, coefficient))
```

The formula is translated (`E^x` becomes `E**x`, and `^` becomes `**`), read by sympy, evaluated at the point (`.subs(AT_POINT)`) and turned into an ordinary number (`float`). For the generic column and the three random columns, the entry predicted by the record is minus the coefficient times the bilinear (off the diagonal $T = -K^{(\nu}{}_{\mu)}$, Section 9.26); the largest difference from the computed entry is kept. `groups` remembers the first entry of each distinct formula.

```
check(len(record_entries) == 42 and len(groups) == 10 and worst_coefficient < 1e-12
      and recorded(A4_PY, "json_offdiagonal_coefficients") == "PASS"
      and recorded(A4_PY, "offdiagonal_coefficients_representation_independent")
      == "PASS",
      "all 42 entries are -(record coefficient) x bilinear, for 4 columns",
      record=f"{A4_RECORD}, offDiagonalKinetic; {A4_PY}, checks "
             "json_offdiagonal_coefficients and "
             "offdiagonal_coefficients_representation_independent")
for nu, mu, coefficient in groups.values():
    say(f"T^{NAMES[nu]}_{NAMES[mu]} = -({coefficient:+.6f}) x its bilinear")
```

The check confirms 42 entries in the record, ten distinct coefficient formulas, and agreement within 10^{-12} for all 42 entries and four columns. The loop prints one entry of each group; for example `T^x1_x5 = -(-0.045985) x its bilinear`, the coefficient derived by hand in Section 9.26, and `T^x4_x1 = -(-2.723325) x its bilinear`, from the record's coefficient $-\frac{7}{4}e^{a_4}H\sin^{1/6} z$ ($\frac{7}{4}\cdot 1.6487213\cdot 0.9438743 = 2.723325$, rounded to six decimals), also derived by hand in Section 9.26. Each printed line names one entry of its group; the other entries of the group have the same coefficient with another 3-space index x_i or extra-time index x_t.

In [10], the record's witnesses.

```
import itertools  # all combinations and all sign choices of a list

P = [gamma[i] @ gamma[i + 4] for i in range(1, 4)]  # gamma^(x1) gamma^(x5), ...
```

The module `itertools` of Python's standard library provides the two tools used at the end of the cell: all combinations of a list and all choices of signs. `P` is the list of the three matrices $P_1 = \gamma^{(1)}\gamma^{(5)}$, $P_2 = \gamma^{(2)}\gamma^{(6)}$, $P_3 = \gamma^{(3)}\gamma^{(7)}$ of Section 9.26.

```
def witness(V_w, H_w):
    """The frequency omega (here freq), A, the unit vectors v1, v2 and the column
    Phi0 of the record's diagonal witness for (V_w, H_w)."""
    A_w = -g4 @ (V_w * I16 - 3 * H_w * g0)  # d4 Phi = A Phi
    freq = np.sqrt(V_w ** 2 - 9 * H_w ** 2)  # the frequency omega
    # The null space of A + i omega: the rows of the third SVD factor whose
    # singular value is zero, complex conjugated and written as columns.
    _, singular, rows = np.linalg.svd(A_w + 1j * freq * I16)
    space = rows[singular < 1e-9].conj().T  # 16 x 8: the eigenspace for -i omega
```

The function builds the matrix A and the angular frequency $\omega = \sqrt{V^2 - 9H^2}$. The columns v with $Av = -i\omega v$ are the solutions of $(A + i\omega)v = 0$, the **null space** of $A + i\omega$. The **singular value decomposition** (`np.linalg.svd`) writes any matrix as a product $U\,\mathrm{diag}(s)\,W$ of two matrices with perpendicular unit rows or columns and a list of non-negative numbers s, the singular values; the rows of W whose singular value is zero span the null space (after complex conjugation, written as columns). `rows[singular < 1e-9]` selects these rows; there are eight.

```
    found = []
    for sign in (-1, 1):
        projector = (I16 + sign * P[0]) @ (I16 + sign * P[1]) @ (I16 + sign * P[2]) / 8
        left_vectors, values, _ = np.linalg.svd(projector @ space)
        if np.sum(values > 1e-9) != 1:
            raise ValueError("the joint eigenvector is not unique")
        found.append(left_vectors[:, 0])  # a unit vector
    v1, v2 = found
    c = np.conj(bar(v1) @ v2)  # c = conj(v1^dagger C v2)
    return freq, A_w, v1, v2, v1 + c * v2
```

Because $P_k^2 = 1$, the matrix $\frac{1}{2}(1 \pm P_k)$ keeps the part of a column with $P_k = \pm 1$ and removes the rest (a **projector**); the product of the three, $\frac{1}{8}(1 \pm P_1)(1 \pm P_2)(1 \pm P_3)$, keeps exactly the columns with $P_1 = P_2 = P_3 = \pm 1$. Applied to the eight columns of the eigenspace it leaves a single direction (the check `np.sum(values > 1e-9) != 1` stops the notebook otherwise), and the first left factor of its singular value decomposition is a unit column along it: v_1 for the sign -1, v_2 for $+1$. Then $c = \overline{v_1^\dagger C v_2}$ and $\Phi_0 = v_1 + c v_2$. The function returns five results.

```
WITNESSES = [(5.0, 1.0), (5.0, 4 / 3), (-5.0, 1.0)]  # the record's (V, H)
witnesses = {pair: witness(*pair) for pair in WITNESSES}  # (V, H) -> the five
for (V_w, H_w), found in witnesses.items():
    say(f"witness (V, H) = ({V_w:g}, {H_w:.4g}): frequency omega = "
        f"sqrt(V^2 - 9 H^2) = {found[0]:.6f}")
```

The record's three pairs (V, H); `witness(*pair)` hands the two numbers over as two arguments. The cell prints the frequencies 4, 3 and 4 ($H = 4/3$ is printed as `1.333` by the format `.4g`, four significant digits).

```
def product_of(indices):
    """The product gamma^(a) gamma^(b) ... of the frame gammas with these indices."""
    result = I16
    for index in indices:
        result = result @ gamma[index]
    return result


subsets = [chosen for size in range(1, 9)
           for chosen in itertools.combinations(range(8), size)]  # 255 index sets
traceless = all(abs(np.trace(product_of(chosen))) < 1e-12 for chosen in subsets)
```

The rest of the cell checks Steps 1 and 3 of the construction in Section 9.26 with numbers. `product_of` multiplies the frame gammas whose indices it is given, from left to right, starting from the unit matrix. `itertools.combinations(range(8), size)` lists every choice of `size` different indices out of $0, \ldots, 7$, each in increasing order; for `size` from 1 to 8 these are $8+28+56+70+56+28+8+1 = 255 = 2^8 - 1$ choices. `np.trace` adds the diagonal entries of a matrix, and `traceless` is true when every one of the 255 products has trace 0 up to rounding, the lemma of Step 3.

```
dimensions_hold = True
for (V_w, H_w), (freq, A_w, v1, v2, phi0) in witnesses.items():
    # the dimension of the space of -i omega: 16 minus the rank of A + i omega
    dimension = 16 - np.linalg.matrix_rank(A_w + 1j * freq * I16, tol=1e-9)
    for signs in itertools.product((-1, 1), repeat=3):  # the eight sectors
        keeper = (I16 + 1j / freq * A_w) / 2  # keeps the space of -i omega
        for sign, P_k in zip(signs, P):
            keeper = keeper @ (I16 + sign * P_k) / 2  # keeps P_k = sign
        dimensions_hold = dimensions_hold and abs(np.trace(keeper) - 1) < 1e-12
    dimensions_hold = dimensions_hold and dimension == 8
check(len(subsets) == 255 and traceless and dimensions_hold,
      "255 gamma products have trace 0; space of -i omega: dim 8; each sector: dim 1")
```

For each witness, the **rank** of a matrix (`np.linalg.matrix_rank`, the number of its independent columns, counted here with the tolerance 10^{-9} for rounding) gives the dimension of the space of the eigenvalue $-i\omega$: the columns v with $(A + i\omega)v = 0$ form a space of dimension 16 minus the rank of $A + i\omega$ (Step 1 says 8). `itertools.product((-1, 1), repeat=3)` lists the eight choices of three signs $(\varepsilon_1, \varepsilon_2, \varepsilon_3)$; for each, `keeper` is the matrix $\Pi = \frac{1}{2}(1 + \frac{i}{\omega}A) \cdot \frac{1}{2}(1 + \varepsilon_1 P_1) \cdot \frac{1}{2}(1 + \varepsilon_2 P_2) \cdot \frac{1}{2}(1 + \varepsilon_3 P_3)$ of Step 2, built factor by factor (`1j / freq * A_w` is $\frac{i}{\omega}A$), and its trace must be 1 (Step 3). The check confirms the 255 traces, the dimension 8 and the 24 sector traces (eight sectors for each of the three witnesses), and prints `PASS 255 gamma products have trace 0; space of -i omega: dim 8; each sector: dim 1`.

In [11], the witnesses are condensates with a diagonal tensor.

```
grid = [(a4, a4p, z) for a4 in (-1.0, 0.0, 0.5) for a4p in (-0.5, 0.0, 0.25, 1.0)
        for z in (0.3, 0.8, 1.3)]  # 3 x 4 x 3 = 36 points
witness_columns = {}  # (V, H) -> Phi0
unit_S = {}  # (V, H) -> S of Phi0 built from v1 and v2 of length 1
for (V_w, H_w), (freq, A_w, v1, v2, phi0) in witnesses.items():
    S_w = (bar(phi0) @ phi0).real
```

```
    largest_bilinear = max(abs(bilinear(phi0, d)) for d in used)
    worst_off = 0.0  # the largest off-diagonal entry at the 36 points
    for a4, a4p, z in grid:
        T = condensate_tensor(phi0, a4, a4p, z, V_w, H_w)
        worst_off = max(worst_off, np.abs(T - np.diag(np.diag(T))).max())
```

`grid` lists 36 points: every combination of three values of a_4, four of a_4' (deflating, static and inflating) and three of z. For each witness the loop computes S, the largest of its 15 bilinears, and, at the 36 points, the largest entry off the diagonal of its tensor (`np.diag(np.diag(T))` is the diagonal matrix made of the diagonal of `T`, so `T - np.diag(np.diag(T))` keeps only the entries off the diagonal).

```
    check(np.abs(A_w @ phi0 + 1j * freq * phi0).max() < 1e-12
          and largest_bilinear < 1e-12 and abs(S_w) > 0.1 and worst_off < 1e-12,
          f"witness (V, H) = ({V_w:g}, {H_w:.4g}): omega = {freq:g}, the 15 "
          "bilinears vanish, S != 0, T diagonal at 36 points")
    witness_columns[(V_w, H_w)] = phi0
    unit_S[(V_w, H_w)] = S_w
    report(f"witness ({V_w:g}, {H_w:.4g}): omega, and S for unit v1, v2",
           f"{freq:g} and {S_w:.6f}")
```

Each check confirms four things: $A\Phi_0 = -i\omega\Phi_0$, so $\Phi = e^{-i\omega x_4}\Phi_0$ is an exact condensate; all 15 bilinears vanish; $S \neq 0$; and the tensor is diagonal at all 36 points. The cell prints $S = 1.28$, 0.72 and 1.28.

In [12], why S of a witness is never negative.

```
anticommute = all(np.abs(C @ P_k + P_k @ C).max() < 1e-12 for P_k in P)
symmetric = all(np.array_equal(P_k, P_k.T) for P_k in P)  # real: P^dagger = P^T
steps_hold, observation = True, True
for (V_w, H_w), (freq, A_w, v1, v2, phi0) in witnesses.items():
    s = bar(v1) @ v2  # s = v1^dagger C v2
    S_unit = unit_S[(V_w, H_w)]  # S of Phi0 with unit v1, v2 (previous cell)
    steps_hold = (steps_hold and abs(bar(v1) @ v1) < 1e-12
                  and abs(bar(v2) @ v2) < 1e-12
                  and abs(S_unit - 2 * abs(s) ** 2) < 1e-12)
    observation = observation and abs(S_unit - 2 * freq ** 2 / V_w ** 2) < 1e-12
```

These lines check each step of the proof of Section 9.26: $CP_k = -P_kC$ and P_k symmetric for the three P_k; and, for each witness, $v_1^\dagger C v_1 = 0$, $v_2^\dagger C v_2 = 0$ and $S = 2|v_1^\dagger C v_2|^2$. `observation` tests a further formula, $S = 2\omega^2/V^2$.

```
check(anticommute and symmetric and steps_hold,
      "C P_k = -P_k C, P_k symmetric, v^dagger C v = 0, so S = 2 |v1^dagger C v2|^2")
check(observation,
      "observed: S = 2 omega^2 / V^2 for the three witnesses (unit v1, v2)")
```

The first check confirms the steps of the proof. The second confirms the observation $S = 2\omega^2/V^2$ for these three witnesses ($2 \cdot 16/25 = 1.28$ and $2 \cdot 9/25 = 0.72$); this formula is COMPUTED for three cases and not proved here.

In [13], the numbers of the record.

```
detail = record_entry(A4_WL, "condensate_diagonal_witness_exact")["detail"]
# the frequencies, which the record writes "w = sqrt(M^2 - 9 H^2) = {4, 3, 4}"
freq_text = re.search(r"9 H\^2\) = \{([^}]*)\}", detail).group(1)  # "4, 3, 4"
S_text = re.search(r"S = \{([^}]*)\}", detail).group(1)  # the three S values
record_freq = np.array([float(value) for value in freq_text.split(",")])
record_S = np.array([float(value) for value in S_text.split(",")])
our_freq = np.array([witnesses[pair][0] for pair in WITNESSES])
our_S = np.array([unit_S[pair] for pair in WITNESSES])
```

The record's Wolfram report writes the witnesses' numbers into the detail text of its check `condensate_diagonal_witness_exact`. `re.search(pattern, text)` finds the first place where the pattern matches, and `.group(1)` returns the part inside the first parentheses of the pattern. In the first pattern a backslash makes `^`, `)`, `{` and `}` ordinary characters, and `[^}]*` means "any characters except a closing brace"; so `freq_text` is the text between the braces after `9 H^2) =`, namely `4, 3, 4`, and `S_text` the text between the braces after `S =`. `.split(",")` cuts each text at the commas and `float` turns the pieces into numbers. `our_freq` and `our_S` are the notebook's values in the same order.

```
check(np.abs(our_freq - record_freq).max() < 1e-12
      and recorded(A4_WL, "condensate_diagonal_witness_exact") == "PASS"
      and recorded(A4_PY, "authorT16_condensate_witness") == "PASS",
      f"omega = {freq_text} for the three witnesses, as in the record",
      record=f"{A4_WL}, check condensate_diagonal_witness_exact; {A4_PY}, check "
             "authorT16_condensate_witness")
quotients = record_S / our_S  # S of the record divided by S of unit v1, v2
check(np.abs(quotients / quotients[0] - 1).max() < 1e-9 and np.all(record_S > 0),
      f"S = {S_text} of the record are one common multiple of ours",
      record=f"{A4_WL}, check condensate_diagonal_witness_exact")
report("S of the record divided by S of unit v1, v2 (all three)",
       f"{quotients[0]:.1f}")
```

The frequencies must agree exactly: $\omega = \sqrt{V^2 - 9H^2}$ does not depend on any choice. The values of S cannot agree, because the record builds the witnesses with exact columns v_1, v_2 that are not of length 1, and $S = 2|v_1^\dagger C v_2|^2$ grows with their lengths (multiplying v_1 by a number a and v_2 by b multiplies S by $|a|^2|b|^2$). So the second check divides each recorded S by ours and requires the three quotients to be one and the same number; the record's values are then reproduced up to one common factor, which the cell prints. The factor is not physics: it is the product of the squared lengths of the record's columns, a choice of the record's verifier, and it changes whenever the verifier picks other columns. The Wolfram report `Revision/field_equations_a4/reports/wolfram-a4-report.json` in its version of 2026-10-08 (52 checks), whose verifier builds the columns from the author's gamma matrices, lists $S = 51200, 28800, 51200$; the quotients are $51200/1.28 = 40000$ for the first and the third witness and $28800/0.72 = 40000$ for the second, and the cell prints 40000.0. The version of 2026-10-01 (47 checks) built its columns from another set of 16×16 gamma matrices that obey the same Clifford relations and listed $S = 204800, 115200, 204800$, the common factor 160000 (the record file `Revision/field_equations_a4/README.md` states both): the factor changed with the columns, while the frequencies, $S \neq 0$ and the vanishing of the 15 bilinears did not. The

notebook reads the numbers from the report, so a later version of the report that lists other values of S makes the cell print those values and their quotient instead.

In [14], the heat map of a witness (Figure 09c.3).

```
T_witness = condensate_tensor(witness_columns[(5.0, 1.0)], **POINT)
fig, ax = plt.subplots(figsize=(6.6, 5.6))
picture = heat_map(ax, T_witness, "Witness condensate: $T^\\nu{}_\\mu$", SCALE)
fig.colorbar(picture, ax=ax, label="value (energy per unit volume)", extend="both")
save_figure(fig, "witness_condensate",
            "Heat map of the 64 entries $T^\\nu{}_\\mu$ of the exact witness "
            ...)
check(len(off_diagonal_pairs(T_witness)) == 0 and abs(T_witness[4, 4]) > 0.1,
      "the drawn witness tensor is diagonal with T^x4_x4 = -V S != 0")
```

The witness with $(V, H) = (5, 1)$ is drawn at the same point and with the same colour scale as the generic condensate. **What Figure 09c.3 shows.** All 64 entries, in units of energy per unit volume: only the square (x_4, x_4) is coloured, with $-VS = -5 \cdot 1.28 = -6.40$ (beyond the colour scale, so it has the darkest blue); every other entry is zero, the pressure included ($\lambda = 0$). The student should compare it with Figure 09c.1: this condensate is a perfect fluid at rest. The check confirms that no entry off the diagonal is nonzero.

In [15], the Christoffel symbols along a curved history.

```
x = sp.symbols("x0:8", real=True)  # sympy was imported in section 8
Hs = sp.symbols("H", positive=True)
a4_function = sp.Function("a4")(x[4])
zs = 6 * Hs * x[0]
space_entry = sp.exp(2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
extra_entry = -sp.exp(-2 * a4_function) * sp.sin(zs) ** sp.Rational(1, 3)
g = sp.diag(*([sp.cot(zs) ** 2] + [space_entry] * 3 + [-1] + [extra_entry] * 3))
history = sp.Rational(3, 10) * x[4] + sp.Rational(1, 20) * x[4] ** 2  # a4(x4)
```

The metric with symbols, as in In [13] of Notebook 09a, and the history $a_4 = \frac{3}{10}x_4 + \frac{1}{20}x_4^2$ with exact fractions. It is curved (not linear), increasing for $x_4 > -3$: the extra times deflate at a growing rate $a_4' = 0.3 + 0.1x_4$.

```
christoffel = {}  # (l, i, j) with i <= j -> a numpy function of (x4, x0, H)
for l in range(8):
    for i in range(8):
        for j in range(i, 8):
            value = sp.simplify((sp.diff(g[l, j], x[i]) + sp.diff(g[l, i], x[j])
                                 - sp.diff(g[i, j], x[l])) / (2 * g[l, l]))
            if value != 0:  # put the history in, then make a numpy function
                christoffel[(l, i, j)] = sp.lambdify(
                    (x[4], x[0], Hs), value.subs(a4_function, history).doit(), "numpy")
check(len(christoffel) == 25, "25 nonzero Christoffel symbols (i <= j)")
```

The 25 symbols are computed as in Notebook 09a. Then the history is put in (`.subs(a4_function, history)` replaces $a_4(x_4)$; `.doit()` carries out the derivative a_4' that is now a derivative of a known

function), and `sp.lambdify((x4, x0, H), expression, "numpy")` turns the exact expression into a fast numpy function of the three numbers. The check confirms the count, 25.

In [16], the test condensate, the control and the divergence.

```
m9, lam9, H9 = 1.0, 0.5, 0.25  # the parameters of this section
chi9 = rng.normal(size=16) + 1j * rng.normal(size=16)
chi9 /= np.linalg.norm(chi9)
V9 = m9 + lam9 * (bar(chi9) @ chi9).real  # the effective mass
M_TRUE = -V9 * g4 + 3 * H9 * g4 @ g0  # the true condensate: M^2 = 9 H^2 - V^2
M_CONTROL = -V9 * g4  # without the term 3 H gamma^(x0): M^2 = -V^2
K2_TRUE, K2_CONTROL = 9 * H9 ** 2 - V9 ** 2, -V9 ** 2
```

The test uses $m = 1$, $\lambda = 0.5$, $H = 0.25$ and a new random unit column. The true condensate has the matrix M of Section 9.18; the control leaves out $3H\gamma^{(4)}\gamma^{(0)}$, so its matrix squares to $-V^2$ (the same computation as in Section 9.18 with $H = 0$).

```
def field(M, k2, x4):
    """Phi(x4) and dPhi/dx4 = M Phi(x4) for Phi(0) = chi9 (M^2 = k2)."""
    k = np.sqrt(complex(k2))
    phi = np.cosh(k * x4) * chi9 + np.sinh(k * x4) / k * (M @ chi9)
    return phi, M @ phi


def tensor_at(M, k2, x4, x0):
    phi, d4 = field(M, k2, x4)
    dphi = np.zeros((8, 16), dtype=complex)
    dphi[4] = d4
    return energy_momentum(phi, dphi, m9, lam9, 0.3 * x4 + 0.05 * x4 ** 2,
                           0.3 + 0.1 * x4, 6 * H9 * x0, H9)
```

`field` returns $\Phi(x_4)$ and $\partial_4\Phi = M\Phi$ for either matrix. `tensor_at` returns the 64 entries at the point (x_4, x_0) of the curved history: $a_4 = 0.3x_4 + 0.05x_4^2$, $a_4' = 0.3 + 0.1x_4$ and $z = 6Hx_0$.

```
def divergence(M, k2, x4, x0, h):
    """The eight components of nabla_mu T^mu_nu, with central differences of step h,
    and the size of the tensor (its largest entry)."""
    T = tensor_at(M, k2, x4, x0)
    d4T = (tensor_at(M, k2, x4 + h, x0) - tensor_at(M, k2, x4 - h, x0)) / (2 * h)
    d0T = (tensor_at(M, k2, x4, x0 + h) - tensor_at(M, k2, x4, x0 - h)) / (2 * h)
```

The tensor of a condensate depends on x_4 and x_0 only, so its divergence needs $\partial_4 T$ and $\partial_0 T$. They are replaced by **central finite differences**, $\partial F \approx (F(x+h) - F(x-h))/(2h)$, whose error falls like h^2 as the step h shrinks (the Taylor series: the terms of first and third order of $F(x+h)$ and $F(x-h)$ cancel in the difference, up to the error term $\frac{h^2}{6}F'''$).

```
    G = np.zeros((8, 8, 8))  # G[l, i, j] = Gamma^l_ij
    for (l, i, j), function in christoffel.items():
        G[l, i, j] = G[l, j, i] = function(x4, x0, H9)
```

```
    result = np.zeros(8)
    for nu in range(8):
        result[nu] = (d4T[4, nu] + d0T[0, nu]  # d_mu T^mu_nu (only x4 and x0)
                      + np.einsum("mml,l->", G, T[:, nu])  # Gamma^mu_mu l T^l_nu
                      - np.einsum("lm,ml->", G[:, :, nu], T))  # Gamma^l_mu nu T^mu_l
    return result, np.abs(T).max()


report("effective mass V = m + lambda S of the test condensate", f"{V9:.6f}")
```

`G` is the $8 \times 8 \times 8$ table of all Christoffel symbols at the point (both orders of the lower indices are filled). For each ν the divergence is the formula of Section 9.9: the first sum has only the terms $\partial_4 T^{x_4}{}_\nu$ and $\partial_0 T^{x_0}{}_\nu$; `np.einsum("mml,l->", G, T[:, nu])` is $\sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda} T^\lambda{}_\nu$ (a repeated letter is summed; `->` with nothing after it means "sum everything to one number"), and `np.einsum("lm,ml->", G[:, :, nu], T)` is $\sum_{\lambda,\mu} \Gamma^\lambda{}_{\mu\nu} T^\mu{}_\lambda$. All entries off the diagonal are included. The function returns the eight components and the size of the tensor. The cell prints $V = 0.883522$.

In [17], the divergence as the step shrinks.

```
steps = 0.2 / 2.0 ** np.arange(11)  # 0.2, 0.1, ..., about 0.0002
X4_TEST, X8_TEST = 1.0, 0.5 / (6 * H9)  # x4 = 1 and z = 0.5
true_sizes, control_sizes = [], []
for h in steps:
    div_true, size_true = divergence(M_TRUE, K2_TRUE, X4_TEST, X8_TEST, h)
    div_control, _ = divergence(M_CONTROL, K2_CONTROL, X4_TEST, X8_TEST, h)
    true_sizes.append(np.abs(div_true).max())
    control_sizes.append(np.abs(div_control).max())
true_sizes, control_sizes = np.array(true_sizes), np.array(control_sizes)
```

Eleven steps $h = 0.2/2^n$, $n = 0, \ldots, 10$, from 0.2 down to about 0.0002. At the point $x_4 = 1$, $z = 0.5$ the loop computes the largest of the eight components of the divergence for the true condensate and for the control.

```
ratios = true_sizes[:6] / true_sizes[1:7]  # halving h from 0.2 to 0.003125
check(np.all((ratios > 3.6) & (ratios < 4.4)),
      "true condensate: halving h divides the divergence by about 4 (error h^2)")
check(true_sizes[8] < 1e-6 * size_true,
      "true condensate: the divergence falls below 1e-6 of the tensor's size")
check(abs(control_sizes[-1] / control_sizes[-2] - 1) < 1e-3
      and control_sizes[-1] > 1e-2 * size_true,
      "control: the divergence tends to a nonzero value (it is not conserved)")
```

Three checks. If the exact divergence of the true condensate is zero, what remains is the error of the finite differences, proportional to h^2: halving h must divide it by about 4 (`&` means "and" for arrays; the ratios of six successive halvings must lie between 3.6 and 4.4). At $h = 0.2/2^8 \approx 0.00078$ it must be below 10^{-6} of the size of the tensor. For the control the divergence must settle at a nonzero value: its last two values agree to 10^{-3} and exceed 10^{-2} of the size of the tensor.

```
for x4_point, z_point in ((2.0, 1.0), (0.5, 1.2)):
    x0_point = z_point / (6 * H9)
```

```
    coarse, size_other = divergence(M_TRUE, K2_TRUE, x4_point, x0_point, 1e-3)
    fine, _ = divergence(M_TRUE, K2_TRUE, x4_point, x0_point, 1e-4)
    shrink = np.abs(coarse).max() / np.abs(fine).max()  # about 10^2 for an h^2 error
    check(90 < shrink < 110 and np.abs(fine).max() < 1e-5 * size_other,
          f"true condensate conserved at x4 = {x4_point}, z = {z_point} "
          "(h = 0.001 and 0.0001)")
```

Two more points of the history. Dividing the step by 10 must divide an error proportional to h^2 by about 100, and the finer result must be below 10^{-5} of the size of the tensor.

```
WL = "Revision/theory/reports/wolfram-field-theory.json"
check(recorded(WL, "exact_solution_nonlinear_homogeneous_C") == "PASS"
      and recorded(WL, "conservation_on_shell_general") == "PASS",
      "the record proves the conservation that these numbers confirm",
      record=f"{WL}, checks exact_solution_nonlinear_homogeneous_C and "
             "conservation_on_shell_general")
for h, t_size, c_size in zip(steps[::2], true_sizes[::2], control_sizes[::2]):
    say(f"h = {h:.5f}:  true condensate {t_size:.2e},  control {c_size:.4f}")
```

The last check confirms that the record proves the conservation exactly. The loop prints every second step (`[::2]` takes every second entry): the true condensate's divergence falls from $5.06 \cdot 10^{-3}$ at $h = 0.2$ to $4.47 \cdot 10^{-9}$ at $h = 0.0002$, by about 16 for each factor 4 in h, while the control's settles at 0.0676. Six PASS lines in all.

In [18], the convergence (Figure 09c.4).

```
fig, ax = plt.subplots()
ax.loglog(steps, true_sizes, "o-", label="true condensate")
ax.loglog(steps, control_sizes, "s-", label="control (no $3H\\gamma^{(0)}$ term)")
ax.loglog(steps, true_sizes[0] * (steps / steps[0]) ** 2, "k:",
          label="proportional to $h^2$")
ax.set_xlabel("finite-difference step $h$")
ax.set_ylabel("largest $|\\nabla_\\mu T^\\mu{}_\\nu|$")
ax.set_title("Conservation of the full tensor of a condensate")
ax.legend()
save_figure(fig, "conservation_convergence",
            "The largest of the eight components of the covariant divergence "
            ...)
```

`ax.loglog` draws with logarithmic scales on both axes; `"o-"` draws circles joined by lines, `"s-"` squares. The dotted line is proportional to h^2. **What Figure 09c.4 shows.** The largest component of the divergence (energy per unit volume per unit length) against the step h, both axes logarithmic. The circles of the true condensate fall along the dotted h^2 line over six powers of ten, so the exact divergence is zero; the squares of the control stay near 0.07. The student should see how a convergence plot separates an error of the method (which shrinks with h) from a real failure (which does not).

In [19], the eight components (Figure 09c.5).

```
div_true, _ = divergence(M_TRUE, K2_TRUE, X4_TEST, X8_TEST, 1e-3)
div_control, _ = divergence(M_CONTROL, K2_CONTROL, X4_TEST, X8_TEST, 1e-3)
```

```
positions = np.arange(8)
fig, ax = plt.subplots()
floor = 1e-16  # zero components are drawn at this height on the logarithmic axis
ax.bar(positions - 0.2, np.abs(div_true) + floor, width=0.4, label="true condensate")
ax.bar(positions + 0.2, np.abs(div_control) + floor, width=0.4,
       label="control (no $3H\\gamma^{(0)}$ term)")
ax.set_yscale("log")
ax.set_ylim(1e-16, 10.0)
ax.set_xticks(positions, [f"$\\nu = {n[0]}_{n[1]}$" for n in NAMES], fontsize=8)
ax.set_ylabel("$|\\nabla_\\mu T^\\mu{}_\\nu|$ (with $h = 0.001$)")
ax.set_title("The eight components of the divergence")
ax.legend(fontsize=8)
save_figure(fig, "divergence_components",
            "The eight components $|\\nabla_\\mu T^\\mu{}_\\nu|$, $\\nu = x_0, \\dots, "
            "x_7$, of the divergence of the full tensor of the condensate of the "
            ...)
```

At $h = 0.001$ the cell draws the size of each of the eight components for both configurations as bars on a logarithmic axis; 10^{-16} is added so that an exact zero, which a logarithmic axis cannot show, is drawn at the bottom. **What Figure 09c.5 shows.** For $\nu = x_0, \dots, x_7$: the true condensate's bars all lie at the level of the finite-difference error, below 10^{-6}; the control's bars for the six momentum components $\nu = x_1, x_2, x_3, x_5, x_6, x_7$ stand between about 0.01 and 0.07, while its components $\nu = x_4$ and x_0 vanish. The student should see what Section 9.26 explained: the control keeps the energy and hidden balances (its ρ and pressures are those of the true condensate) but breaks the momentum balances; the gravitational term $3H\gamma^{(0)}$ is needed for the conservation of the tensor.

```
momentum = [1, 2, 3, 5, 6, 7]  # the components nu = x1, x2, x3, x5, x6, x7
check(np.abs(div_control).max() > 1e3 * np.abs(div_true).max()
      and np.abs(div_control[momentum]).min() > 1e-3
      and max(abs(div_control[4]), abs(div_control[0])) < 1e-9,
      "control: its six momentum components are nonzero (over 1000 times the true "
      "condensate's), its x4 and x0 components vanish")
```

The check confirms what the figure shows: the control's divergence is more than 1000 times the true condensate's, each of its six momentum components exceeds 10^{-3}, and its x_4 and x_0 components are below 10^{-9}.

In [20], the last check.

```
captions = json.loads(output_file(CAPTION_FILE).read_text(encoding="utf-8"))
expected_files = ["09c_1_generic_condensate.png", "09c_2_offdiagonal_entries.png",
                  "09c_3_witness_condensate.png",
                  "09c_4_conservation_convergence.png",
                  "09c_5_divergence_components.png"]
check(sorted(captions) == expected_files and all(
    output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in expected_files),
    "the five figures of this notebook are saved and captioned")
all_checks_passed()
```

The five figures exist and are captioned, and the last line is `ALL 25 CHECKS PASSED (notebook 09c)`: three checks in In [5], two in In [7], one each in In [8], In [9] and In [10], three in In [11], two

in In [12], two in In [13], one each in In [14] and In [15], six in In [17], and one each in In [19] and In [20].

9.31 What we proved, what we computed, what we assumed

PROVED in this chapter, line by line (and in the Revision record, with the file and check named in the section):

- For the diagonal vielbein of the author's metric, $\{\gamma^\mu, \Omega_\mu\} = 0$ for each direction and $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(0)}$; the a_4' terms cancel because the extra times deflate (Section 9.3).
- From the definition of the energy-momentum tensor by the variation of the action, the diagonal entries $T^\mu{}_\mu = L_0 - K_\mu$ (Section 9.5); the reality of every entry and the symmetry of $T_{\nu\mu}$ (Section 9.6).
- $\rho = -\sum_{\mu\neq x_4} K_\mu + mS + U$, $p_\mu = \sum_{\nu\neq\mu} K_\nu - mS - U$, with their kinetic and potential parts (Section 9.7); the trace $7\sum_\mu K_\mu - 8(mS+U)$ and the kinetic-sum identity off shell, and on shell $\sum_\mu K_\mu = (m+U')S$, $L_0 = SU' - U$ and the trace $-mS + 3\lambda S^2$ (Section 9.8).
- The 25 Christoffel symbols; the lemma $\Gamma^\mu{}_{\mu\nu} = \partial_\nu \ln f_\mu$; the divergence of a diagonal tensor; the two conservation identities $\partial_4\rho = -3a_4'(p_3 - p_t)$ and $\partial_0 p_0 = -3H\cot z\,(2p_0 - p_3 - p_t)$; the vanishing of the other six components (Section 9.9); the profiles $p_0 = P/2 + c/\sin z$ and the form in the coordinate y (Section 9.11).
- The connection term $K^{x_4}{}_{x_1} = \frac{1}{2}e^{a_4}\sin^{1/6} z\, H\,\bar\Phi\gamma^{(4)}\gamma^{(1)}\gamma^{(0)}\Phi$ of a homogeneous configuration (Section 9.12) and the entries $T^{x_1}{}_{x_5} = \frac{1}{2}e^{-2a_4}a_4'\,\bar\Phi\gamma^{(1)}\gamma^{(4)}\gamma^{(5)}\Phi$ and $T^{x_4}{}_{x_1} = \frac{7}{4}e^{a_4}\sin^{1/6} z\, H\,\bar\Phi\gamma^{(1)}\gamma^{(4)}\gamma^{(0)}\Phi$ of a condensate (Section 9.26), equal to the record's coefficients.
- The exact condensates, the constancy of S, $M^2 = 9H^2 - V^2$; their $\rho = mS + U$, $p = SU' - U$ in all seven directions, $T^{x_4}{}_{x_0} = 0$, and $w = x/(2+x)$ with its special values (Sections 9.18 to 9.20); $\rho = E\,u^\dagger B u$ for plane waves of flat space (Section 9.21); in the construction of the witnesses, the dimension 8 of the space of the eigenvalue $-i\omega$ and the dimension 1 of each of its eight sectors, and $S = 2|v_1^\dagger C v_2|^2 \geq 0$ (Section 9.26); the argument of the theorem of the linear history from the record's evolution equation (Section 9.26).

PROVED in the Revision record and used here without its proof: the vielbein-variation tensor with its spin-density term; the complete (Belinfante) tensor as its symmetric part for every configuration, and the equality of the two tensors on shell in all 64 entries (off shell, in general, only the 8 diagonal entries agree); the drop-out of the spin connection from the Lagrangian of every diagonal vielbein; the conservation $\nabla_\mu T^\mu{}_\nu = 0$ of every solution (Noether identity); the same formulas for the Grassmann field dirac16complex; the positive Fock realisation of the quantised field for single good-sector momenta with frozen coefficients, and the complex frequencies of the curved good sector without a boundary condition at $z = \pi/2$ (for $U = 0$ whenever $m^2 < 9H^2$); in the finite Fock model of one good-sector mode set with frozen coefficients, the operator form of the on-shell identity for $\lambda \neq 0$ when the potential and the tensor are both Wick ordered, its failure for every other ordering tested, and the exact values $\rho = mn + \frac{\lambda}{2}n(n-1)$ and $p = \frac{\lambda}{2}n(n-1)$ on the occupation states with n quanta at rest (Section 9.13); the 42 off-diagonal entries of a condensate as multiples of 15 bilinears with the listed coefficients, the diagonal witnesses with $\omega = 4, 3, 4$; the Krein inertia

(4,4) of the plane waves; the vanishing of every off-diagonal entry of the source and the condition $p_3 + p_t = 2p_0$, both imposed by the field equations of gravity; and the theorem that a condensate source that meets the off-diagonal conditions allows only the linear history $a_4 = AHx_4 + a_0$ (under its four hypotheses, Section 9.26), with either sign of A; and, in the dark-sector record, the equation of state w_{eff} that a 3-space observer infers for a homogeneous condensate: 0 under the definitions A and B of the observer's energy density and -1 under C (Section 9.19; the definitions themselves are ASSUMED).

COMPUTED by the notebooks (every number reproduces the record where they overlap): Notebook 09a, 27 checks, confirms the tensor identities at one point to 10^{-11} or better, re-derives the 25 Christoffel symbols and the two identities exactly, and checks the x_0 profiles on a grid to a relative residual of $2.3 \cdot 10^{-5}$; Notebook 09b, 33 checks, confirms the condensates at 401 times to 10^{-9}, measures w on twelve exact solutions (agreement with $x/(2+x)$ to 10^{-9}), solves the toy fluids with RK4 (relative error $1.33 \cdot 10^{-8}$, error ratio 16.17 for a halved step) and reproduces $\rho = \pm 5$; Notebook 09c, 25 checks, confirms the 42 entries against the record's coefficients to 10^{-12}, the dimensions 8 and 1 of the construction of the witnesses, the diagonal tensor of the witnesses at 36 points, the record's frequencies exactly and its values of S up to one common factor (40000 for the Wolfram report of 2026-10-08), and the conservation of the full tensor along a curved history (an error falling like h^2, down to $4.5 \cdot 10^{-9}$), with a negative control. The observation $S = 2\omega^2/V^2$ for the three witnesses is COMPUTED, not proved.

ASSUMED, chosen, or an interpretation. The sign conventions $\rho = -T^{x_4}{}_{x_4}$ and $p_\mu = T^\mu{}_\mu$ are definitions. The histories $a_4 = AHx_4$ ($A = 1$) and $a_4 = 0.3x_4 + 0.05x_4^2$ and every parameter value of the examples are choices; every identity holds for every history. The first-law reading of the x_4 identity is an interpretation, not an extra result. The toy fluids are an ILLUSTRATION, not solutions of the field equations. The normalisations A, B and C of the observer's energy density, quoted in Section 9.19 from the dark-sector record, are ASSUMPTIONS of that record.

OPEN. Which normalisation of the observer's energy density over the extra times, if any, describes a physical observer in 3-space, and a fit to supernova data (the Revision record computes the equation of state that such an observer would infer, its change in time, fits of the CPL form $w(a) = w_0 + w_a(1-a)$, a straight line in the 3-space scale factor a named after Chevallier, Polarski and Linder, and the comparison with the Unite values, all under the ASSUMED definitions A, B and C; Chapter 22, COMPUTED); the operator (normal-ordered) form of the on-shell kinetic-sum identity $\sum_\mu \langle{:}K_\mu{:}\rangle = \langle{:}(m + U')S{:}\rangle$ for $\lambda \neq 0$ for the field on a whole slice (Section 9.13; it is verified only in the finite Fock model of one good-sector mode set with frozen coefficients, and there only under Wick ordering), and with it the operator form of the on-shell values of L_0 and of the trace, and the symmetry and conservation of the quartic operator; a space of quantum states with positive norms for the full quantised field (positive Fock spaces are constructed only for single good-sector momenta with frozen coefficients); a condensate that realises the deflating history together with all the field equations of Einstein-Lovelock gravity with α_2 or α_3 different from zero, for definite values of the constants (for Einstein gravity Chapter 12 constructs such solutions, under the ASSUMED inputs it states). Any reading of the negative energies of the commuting field as a phantom dark energy is a HYPOTHESIS, not a result.

Not claimed. Nothing in this chapter concerns the creation of universes or the matter-antimatter asymmetry. The only link to Part V is the sign reversal of the tensor under the chirality map Γ (theorem T1, Section 9.21), an exact map between sets of solutions in a fixed gravitational field; it proves no creation process, rate or amplitude (Chapters 18 to 21 state what the pairing theorems

prove and what they do not).

9.32 Exercises

Exercise 1. Use the lemma $\Gamma^{\mu}{}_{\mu\nu} = \partial_\nu \ln f_\mu$ of Section 9.9 to compute $\Gamma^{x_5}{}_{x_5 x_4}$ and $\Gamma^{x_5}{}_{x_5 x_0}$, and compare with the table.

Answer. $\ln f_5 = \ln\left(e^{-a_4}\sin^{1/6} z\right) = -a_4 + \frac{1}{6}\ln\sin z$. Differentiating with respect to x_4: only $-a_4$ depends on x_4, so $\Gamma^{x_5}{}_{x_5 x_4} = -a_4'$. Differentiating with respect to x_0 with the chain rule ($dz/dx_0 = 6H$): $\Gamma^{x_5}{}_{x_5 x_0} = \frac{1}{6}\cdot\frac{\cos z}{\sin z}\cdot 6H = H\cot z$. The table lists $\Gamma^{x_t}{}_{x_4 x_t} = -a_4'$ (the symbol is symmetric in its lower indices) and $\Gamma^{x_t}{}_{x_t x_0} = H\cot z$.

Exercise 2. Suppose the extra times inflated like 3-space, with the factors $f_5 = f_6 = f_7 = e^{+a_4}\sin^{1/6} z$ (this is not the author's metric). Use the divergence formula for a diagonal tensor to find the x_4 identity, and explain the difference from Section 9.10.

Answer. Now $\partial_4 \ln f_\mu = a_4'$ for all six directions $x_1, x_2, x_3, x_5, x_6, x_7$, and 0 for x_4 and x_0. With $T^{x_4}{}_{x_4} = -\rho$: $\nabla_\mu T^{\mu}{}_{x_4} = -\partial_4\rho + 3a_4'(-\rho - p_3) + 3a_4'(-\rho - p_t) = -\partial_4\rho - 3a_4'(2\rho + p_3 + p_t)$. Conservation gives $\partial_4\rho = -3a_4'(2\rho + p_3 + p_t)$. This has the form of the familiar cosmological law $d\rho/dt = -3(\dot a/a)(\rho + p)$, once for each family of three directions: the volume $\sqrt{|g|} = e^{6a_4}\cos z$ grows, and the energy density is diluted. In the author's metric the volume is constant, the two terms $\mp 3a_4'\rho$ cancel, and only the difference $p_3 - p_t$ remains.

Exercise 3. A condensate has $m = 1$, $S = 1$ and $\lambda = 0.5$. Compute ρ, p, w and the trace in two ways. Repeat for $\lambda = -0.5$, and compare with the numbers printed by Notebook 09b.

Answer. $\lambda = 0.5$: $\rho = mS + \frac{\lambda}{2}S^2 = 1 + 0.25 = 1.25$; $p = \frac{\lambda}{2}S^2 = 0.25$; $w = 0.25/1.25 = 0.2$ (also $x/(2+x)$ with $x = 0.5$: $0.5/2.5 = 0.2$). Trace: $-\rho + 7p = -1.25 + 1.75 = 0.5$, and $-mS + 3\lambda S^2 = -1 + 1.5 = 0.5$. $\lambda = -0.5$: $\rho = 1 - 0.25 = 0.75$, $p = -0.25$, $w = -1/3$ (and $x/(2+x) = -0.5/1.5$); trace $-0.75 - 1.75 = -2.5 = -1 - 1.5$. Notebook 09b prints $\rho = 1.25$, $p = 0.25$, $w = 0.2$ and $\rho = 0.75$, $p = -0.25$, $w = -0.333333$.

Exercise 4. For $m > 0$ and $\lambda \neq 0$, find the value of S for which a condensate has $w = -1$, compute its ρ and p, and decide the sign of ρ. Does this condensate oscillate or grow?

Answer. $w = -1$ requires $x = \lambda S/m = -1$, that is $S = -m/\lambda$. Then $\rho = mS + \frac{\lambda}{2}S^2 = -\frac{m^2}{\lambda} + \frac{\lambda}{2}\cdot\frac{m^2}{\lambda^2} = -\frac{m^2}{\lambda} + \frac{m^2}{2\lambda} = -\frac{m^2}{2\lambda}$ and $p = \frac{\lambda}{2}S^2 = \frac{m^2}{2\lambda}$, so indeed $p = -\rho$. The energy density is positive only for $\lambda < 0$ (then $S = -m/\lambda > 0$); for $\lambda > 0$ it is negative. The effective mass is $V = m + \lambda S = 0$, so $k^2 = 9H^2 - 0 = 9H^2 > 0$: the condensate grows, like e^{3Hx_4}.

Exercise 5. A toy fluid (an illustration) has $w_3 - w_t = 1/3$ on the history $a_4 = AHx_4$ with $H = 0.25$. Compute $\rho(8)/\rho(0)$ for $A = 1$ and for $A = -1$, and their product.

Answer. $\rho(x_4)/\rho(0) = e^{-3AH(w_3 - w_t)x_4}$. For $A = 1$ the exponent at $x_4 = 8$ is $-3\cdot 0.25\cdot\frac{1}{3}\cdot 8 = -2$, so the ratio is $e^{-2} = 0.135335$, the number of Notebook 09b. For $A = -1$ it is $e^{2} = 7.389056$. The product is $e^{-2}e^{2} = 1$: reversing the history reverses the energy flow.

Exercise 6. Suppose $p_3 + p_t = P$ is a constant and the hidden pressure p_0 stays finite as $z \to 0$. Show that $p_0 = P/2$ everywhere.

Answer. By Section 9.11 every solution of the x_0 balance is $p_0 = P/2 + c/\sin z$ with a constant c. As $z \to 0$, $\sin z \to 0$, so $c/\sin z$ grows without bound unless $c = 0$. A finite p_0 therefore needs $c = 0$,

and then $p_0 = P/2$ at every z.

Exercise 7. Show that the kinetic term K_μ is a real number.

Answer. Let $Y = \bar{\Phi}\gamma^{(\mu)}\partial_\mu\Phi = \Phi^\dagger(C\gamma^{(\mu)})\partial_\mu\Phi$. Since $C\gamma^{(\mu)}$ is real, $Y^* = \Phi^T(C\gamma^{(\mu)})(\partial_\mu\Phi)^*$. A single number equals its transpose, so $Y^* = (\partial_\mu\Phi)^\dagger(C\gamma^{(\mu)})^T\Phi$; since $C\gamma^{(\mu)}$ is antisymmetric, $Y^* = -(\partial_\mu\Phi)^\dagger C\gamma^{(\mu)}\Phi = -\partial_\mu\bar{\Phi}\,\gamma^{(\mu)}\Phi$ (because $\partial_\mu\bar{\Phi} = (\partial_\mu\Phi)^\dagger C$). Hence $\bar{\Phi}\gamma^{(\mu)}\partial_\mu\Phi - \partial_\mu\bar{\Phi}\gamma^{(\mu)}\Phi = Y + Y^*$, which is twice the real part of Y, and $K_\mu = (Y + Y^*)/(2f_\mu)$ is real.

Exercise 8. Use the derivation of Section 9.26 to show that the entry $T^{x_1}{}_{x_5}$ of a condensate vanishes at every moment at which $a_4' = 0$, and evaluate its coefficient at $a_4 = 0.5$, $a_4' = 0.25$. Which number of Notebook 09c does it reproduce?

Answer. Section 9.26 gives $T^{x_1}{}_{x_5} = \frac{1}{2}e^{-2a_4}a_4'\,\bar{\Phi}\gamma^{(1)}\gamma^{(4)}\gamma^{(5)}\Phi$, which is zero when $a_4' = 0$ (the H terms of $K^{x_1}{}_{x_5}$ and of $g^{11}g_{55}K^{x_5}{}_{x_1}$ cancel exactly). At $a_4 = 0.5$, $a_4' = 0.25$ the coefficient is $\frac{1}{2}e^{-1}\cdot 0.25 = 0.5\cdot 0.367879\cdot 0.25 = 0.045985$. Notebook 09c prints `T^x1_x5 = -(-0.045985) x its bilinear` in In [9], and In [7] shows that $T^{x_1}{}_{x_5}$ is proportional to a_4'.

Exercise 9. For the plane wave of Section 9.21 with $m = 2$ and $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, find the positive frequency E and the energy density of a wave whose unit column satisfies $Bu = -u$. What is the energy density of the same wave multiplied by 10?

Answer. $h^2 = (m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2)\cdot 1 = (4+1+4+0+16)\cdot 1 = 25$, so $E = 5$. With $\rho = E\,u^\dagger Bu$ and $u^\dagger Bu = -u^\dagger u = -1$: $\rho = -5$. Multiplying the wave by 10 multiplies every bilinear by $10^2 = 100$: $\rho = -500$. There is no lower bound.

10 Canonical quantisation in 4+4: Krein space and the good sector

Chapters 7 to 9 treated the field dirac16complex as a classical field: 16 complex anticommuting numbers $\Psi_1, \ldots, \Psi_{16}$ at every point of the author's 4+4 dimensional universe, with a Lagrangian, a field equation and an energy-momentum tensor. This chapter makes the field a quantum field. It follows the standard recipe, canonical quantisation with the time x_4, step by step, and finds what is new in signature (4,4): the quantum rule contains the matrix B, which has eight positive and eight negative eigenvalues, so the space of quantum states cannot carry an ordinary positive inner product when the field operators are read in the ordinary way; it is a Krein space. In the good sector (waves that do not depend on the three extra times) a positive state space is constructed and checked for each single momentum with frozen coefficients (flat-frame plane waves, in which the deflation of the extra times does not enter), with particles and antiparticles of positive energy. The chapter shows exactly how far this goes, where it stops, and what the quantum theory says about the pairing of universes of masses $+m$ and $-m$.

10.1 What this chapter does

Why quantise. Dirac's equation of 1928 describes one electron as a classical wave. It has waves of negative energy, and a classical theory cannot say why an electron does not fall into them. Quantum field theory answers: the field becomes a collection of operators that create and destroy particles, the negative-energy waves describe antiparticles, and every particle and every antiparticle has positive energy. The same recipe is applied here to dirac16complex, the author's fermion field with 16

complex anticommuting components (a spinor of Pin(4,4), Chapter 5). The second field of the theory, dirac16complex00, with 16 commuting components, is a classical ("semi-classical") field in the author's definition and is not quantised; Section 10.22 shows what that means for its energy.

What happens in signature (4,4). The recipe has three steps: read off from the Lagrangian the term with the time derivative; turn it into an anticommutator of the field operators; find a space of quantum states on which operators with this anticommutator act. In ordinary 3+1 dimensional physics the anticommutator is the identity matrix and the state space is an ordinary Hilbert space. Here the anticommutator is the matrix $B = -iC\gamma^{(x_4)}$, and the chapter proves that no space with a positive inner product carries it when $\Psi^\dagger$ is read as the ordinary adjoint. That is the central fact of the chapter, and everything else is built around it:

- one-particle waves: the evolution of a plane wave conserves the Krein form $u^\dagger Bu$, its frequencies are real or imaginary according to the sign of $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$, and at every real frequency the waves of one frequency carry four directions of positive and four of negative Krein form (Sections 10.2 to 10.6, Notebook 10a);
- quantum states, fermion modes and the Fock space from zero, the canonical rule, the theorem that forces the Krein space, and the conjugation of the quantised field (Sections 10.11 to 10.15, Notebook 10b);
- the good sector and its positive Fock space (one momentum with frozen coefficients), the Dirac sea and normal ordering, the energy-momentum tensor operator, and the contrast with the classical commuting field (Sections 10.20 to 10.22, Notebook 10c);
- the curved good sector of the author's metric, where Hermiticity needs a boundary condition at the patch end $z = \pi/2$ (Sections 10.27 and 10.28, Notebook 10d);
- the proof that no invariant choice of the charge density is positive, and the symmetries that keep the canonical rule (Sections 10.33 and 10.34, Notebook 10e);
- the same structure along the deflating history of the extra times, where every wave with extra-time momentum eventually grows (Section 10.39, Notebook 10f);
- the quantum reading of the pairing of universes of masses $+m$ and $-m$, with an exact list of what it does not establish (Sections 10.44 and 10.45, Notebook 10g).

The chapter closes with "What we proved, what we computed, what we assumed" (Section 10.50), with exercises (Section 10.51) and with their complete answers (Section 10.52).

The seven worked examples. Each is a complete Jupyter notebook; each is complete in itself, so you may run them in any order.

notebook	what it computes	checks	figures
10a	one-particle spectra and Krein inertia	28	6
10b	the canonical rule on a Fock space; the Krein space	25	5

notebook	what it computes	checks	figures
10c	the positive Fock space of one good-sector momentum	14	5
10d	the curved good sector along the hidden direction	16	5
10e	invariant forms and Krein-unitary generators	15	5
10f	the Krein structure along the deflating history	19	5
10g	the quantum reading of the pairing	30	5

Notation. The author's coordinates are written $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates (scale factor $e^{a_4} \sin^{1/6} z$); x_4 is the time; x_5, x_6, x_7 are the three extra times, which are time-like and DEFLATE EXPONENTIALLY (scale factor $e^{-a_4} \sin^{1/6} z$ with a_4 increasing); x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$ and $H > 0$ the author's constant. The flat frame metric is $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \ldots, x_7$. The gamma matrices are the author's eight real 16×16 matrices; the one of the direction x_a is written $\gamma^{(x_a)}$, and they obey the Clifford relation $\gamma^{(x_a)}\gamma^{(x_b)} + \gamma^{(x_b)}\gamma^{(x_a)} = 2\eta_{ab} I_{16}$ (Chapter 4), where I_{16} is the 16×16 identity matrix and η_{ab} is η_{aa} for $a = b$ and 0 otherwise. So a space-like gamma (x_0, x_1, x_2, x_3) squares to $+I_{16}$, a time-like one $(x_4, \ldots, x_7)$ to $-I_{16}$, and two different gammas anticommute. Chapter 5 built from them the charge matrix $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ (the author's sigma16), the chirality $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)} \cdots \gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$ and the matrix $B = -iC\gamma^{(x_4)}$. For a matrix M, M^T is its transpose, M^* the matrix of the complex conjugate entries and $M^\dagger = (M^T)^*$ its conjugate transpose; for a column u, $u^\dagger$ is the row of the conjugated entries. The letter m is the mass. The words used for the status of a statement are those of the whole book: PROVED (exact, with the verifying record and check), COMPUTED (a numerical result with its accuracy), ASSUMED, HYPOTHESIS and OPEN.

The records this chapter uses. Every formula and number comes from these Revision records or from the chapter's own notebooks, which reproduce the records where they overlap:

record	what it holds
`Revision/algebra/gammas.json`	the eight gammas, C and B
`Revision/algebra/reports/`	the algebra checks: `python-algebra.json`, `wolfram-algebra.json`
`Revision/theory/reports/`	the field-theory checks: `python-field-theory.json`, `wolfram-field-theory.json`, `python-scope.json`, `wolfram-scope.json`
`Revision/theory/field-theory.json`	the formulas of the field theory
`Revision/pairing/pairing-theory.json`	the pairing theorems and their data
`Revision/pairing/reports/`	the pairing checks: `python-pairing.json`, `wolfram-pairing.json`

record	what it holds
`Revision/lead_checks/reports/`	`charge-conjugation-and-u1.json` (12 checks)
`Revision/kohn_sham/results/parameters.json`	the parameters of the deflating history
`Revision/field_equations_a4/reports/ks-source-conditions.json`	the status of that history: a prescribed background
`Revision/docs/DIRAC16COMPLEX_FIELD_THEORY.md`	the theory document of the record (its sections 11 and 12: quantisation, the energy-momentum tensor operator)
`Revision/docs/PAIR_CREATION_PROOFS.md`	the proof document of the pairing (its section 6: the quantum reading Q)

A report is a file in which a verifying program has recorded each check with its name, its verdict and a detail text; every check cited in this chapter has the verdict PASS. Below, a report is named by its file name only, for example "`python-field-theory.json`, check `canonical_anticommutator_B`".

10.2 The matrix B and the Krein form

The charge. Chapter 5 showed that the current of the field is $J^\mu = -i\bar\Psi\gamma^\mu\Psi$ with $\bar\Psi = \Psi^\dagger C$, and that its time component is the charge density

$$J^{(x_4)} = -i\Psi^\dagger C\gamma^{(x_4)}\Psi = \Psi^\dagger B\Psi, \qquad B = -iC\gamma^{(x_4)}.$$

The Revision record proves the **local conservation law** $\partial_\mu(\cos z\, J^\mu) = 0$ (for the author's metric with a changing b_4 the weight is $\sqrt{|g|} = b_4^{3/2}\cos z$: `python-field-theory-b4.json`, check `u1_noether_matrix_identity_b4`) (summed over the eight directions μ, as in Chapter 1) for every solution of the field equation (`charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity`), and that $J^{(x_4)}$ is the density of the total charge $Q = \int \cos z\, \Psi^\dagger B\Psi\, d^7x$, the integral over a slice $x_4 =$ const (`wolfram-field-theory.json`, check `charge_density_is_Krein_form_G`). A local law does not by itself make Q constant. Write it as $\partial_4(\cos z\, J^{(x_4)}) = -\sum_{a\neq 4}\partial_a(\cos z\, J^{(x_a)})$ and integrate over the slice: the left side gives dQ/dx_4, and each term on the right is the integral of a derivative along one direction, which by the fundamental theorem of calculus equals the difference of the values at the two ends of that direction. So Q changes by exactly the charge that flows out through the boundary of the slice, and it is constant only when this **flux** vanishes. In flat space it vanishes for fields that die away at large distances. In the author's patch the boundary includes the patch end $z = \pi/2$, and there the flux vanishes only under a boundary condition, an ASSUMED no-flux condition that the quantisation of the Revision record does not impose; without it the constancy of Q is OPEN (Revision theory document, section 11). Sections 10.27 and 10.28 show an exact solution whose charge Q grows from 1/6 to 454.007. For two columns u and v of 16 complex numbers the number $u^\dagger Bv$ is called their **Krein form**, and $u^\dagger Bu$ the **Krein norm** of u (a name, not a length: it can be negative).

Five properties of B, derived line by line. We use only the Clifford relation, the reality of the gammas, $C^T = C$, $CC = I_{16}$ and $(\gamma^{(x_4)})^T = -\gamma^{(x_4)}$ (Chapter 5).

(B1) $C\gamma^{(x_4)} = \gamma^{(x_4)}C$. Move $\gamma^{(x_4)}$ from the right of $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ to its left, one factor at a time:

$$\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}\gamma^{(x_4)} = (-1)^4\,\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}.$$

Each of the four steps exchanges $\gamma^{(x_4)}$ with a different gamma, which costs a sign by the Clifford relation; four signs give $+1$.

(B2) B is purely imaginary: $C\gamma^{(x_4)}$ is a product of real matrices, and B is $-i$ times it.

(B3) B is Hermitian, $B^\dagger = B$:

$$B^\dagger = (+i)\,(C\gamma^{(x_4)})^T = (+i)\,(\gamma^{(x_4)})^T C^T = (+i)(-\gamma^{(x_4)})C = -i\,\gamma^{(x_4)}C = -i\,C\gamma^{(x_4)} = B.$$

The first step conjugates the number $-i$ and transposes the real matrix; the second uses $(MN)^T = N^T M^T$; the third uses $(\gamma^{(x_4)})^T = -\gamma^{(x_4)}$ and $C^T = C$; the fourth collects the signs; the fifth is (B1).

(B4) $BB = I_{16}$:

$$BB = (-i)^2\,C\gamma^{(x_4)}C\gamma^{(x_4)} = -\,CC\,\gamma^{(x_4)}\gamma^{(x_4)} = -\,I_{16}\,(-I_{16}) = I_{16}.$$

The first step uses $(-i)^2 = -1$; the second moves the middle C to the left by (B1); the third uses $CC = I_{16}$ and $\gamma^{(x_4)}\gamma^{(x_4)} = \eta_{44}I_{16} = -I_{16}$.

(B5) $\operatorname{tr} B = 0$, and B has eight eigenvalues $+1$ and eight -1. The trace of a matrix is the sum of its diagonal entries; it does not change when the factors of a product are rotated, $\operatorname{tr}(XY) = \operatorname{tr}(YX)$. The chirality Γ anticommutes with every gamma (it is the product of all eight; moving one gamma through it passes seven different gammas and itself, sign $(-1)^7 = -1$), so it anticommutes with the product of the five gammas in $C\gamma^{(x_4)}$, and $\Gamma\Gamma = I_{16}$. Hence

$$\operatorname{tr} B = \operatorname{tr}(\Gamma\Gamma B) = \operatorname{tr}(\Gamma B\Gamma) = \operatorname{tr}(-B\Gamma\Gamma) = -\operatorname{tr} B,$$

so $\operatorname{tr} B = 0$. The first step inserts $\Gamma\Gamma = I_{16}$, the second rotates the first factor Γ to the end, the third moves Γ through B at the cost of a sign. If $Bu = \lambda u$ with $u \neq 0$, then $u = BBu = \lambda^2 u$, so $\lambda^2 = 1$ and $\lambda = \pm 1$; the trace is the sum of the 16 eigenvalues, and a sum of sixteen numbers ± 1 is 0 only with eight of each sign. A Hermitian matrix with p positive and n negative eigenvalues has the **signature** (p, n): B has signature (8,8).

So the Krein norm $u^\dagger Bu$ is a real number for every column (because B is Hermitian, $(u^\dagger Bu)^* = u^\dagger B^\dagger u = u^\dagger Bu$), and it is positive for the eight eigenvectors of $+1$ and negative for the eight eigenvectors of -1. Unlike the charge density $\psi^\dagger\psi$ of Dirac's electron, the charge density of dirac16complex takes both signs.

B as a table. B is i times a real signed permutation matrix: every row has exactly one nonzero entry, $+i$ or $-i$. The following table, computed from the record `Revision/algebra/gammas.json`, gives for every row r the column c of its nonzero entry and its value; so $(Bu)_r = B_{rc}\,u_c$.

row r	**column** c	**entry** B_{rc}	**row** r	**column** c	**entry** B_{rc}
1	10	$+i$	9	2	$+i$
2	9	$-i$	10	1	$-i$
3	12	$-i$	11	4	$-i$

row r	column c	entry B_{rc}	row r	column c	entry B_{rc}
4	11	$+i$	12	3	$+i$
5	14	$-i$	13	6	$-i$
6	13	$+i$	14	5	$+i$
7	16	$+i$	15	8	$+i$
8	15	$-i$	16	7	$-i$

Which gammas commute with B. B is $-i$ times the product of the five gammas of x_0, x_1, x_2, x_3, x_4. A gamma among these five passes four different gammas and itself when it is moved through the product, sign $(-1)^4 = +1$: it commutes with B. A gamma of x_5, x_6, x_7 passes five different gammas, sign $(-1)^5 = -1$: it anticommutes with B. Hence B commutes with $\gamma^{(x_1)}, \gamma^{(x_2)}, \gamma^{(x_3)}, \gamma^{(x_4)}, \gamma^{(x_0)}$ and anticommutes with $\gamma^{(x_5)}, \gamma^{(x_6)}, \gamma^{(x_7)}$, and by the same counting $\Gamma B \Gamma = -B$.

statement	status	where it is verified
B purely imaginary, Hermitian, $BB = I_{16}$, $\operatorname{tr} B = 0$, signature (8,8)	PROVED	`python-field-theory.json`, check `B_properties`; `wolfram-algebra.json`, check `B_signature_8_8`
B commutes with the gammas of x_1, x_2, x_3, x_4, x_0, anticommutes with those of x_5, x_6, x_7	PROVED	`python-algebra.json`, check `B_gamma_relations`
the local conservation law $\partial_\mu(\cos z\, J^\mu) = 0$ on shell; $J^{(x_4)} = \Psi^\dagger B \Psi$	PROVED	`charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity`; `wolfram-field-theory.json`, check `charge_density_is_Krein_form_G`
the total charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, d^7x$ is constant in time	OPEN (true only when no charge flows through the boundary; at $z = \pi/2$ this is an ASSUMED no-flux condition that the quantisation record does not impose, Sections 10.27 and 10.28)	Revision theory document, section 11

10.3 One plane wave: the mode Hamiltonian

We first study one wave of the field at a time, as a classical wave, before any quantisation. Take $U = 0$ (no self-interaction) and flat 4+4 space. Flat space is not the author's universe. The end of this section defines the **frozen-coefficient model** of the author's universe, which has the same equation with the derivatives divided by scale factors; it is an ASSUMPTION, because it also leaves out two terms of the author's field equation. The field equation in flat space is (Chapter 7)

$$\gamma^{(x_4)}\partial_4\Psi + \sum_{a\neq 4}\gamma^{(x_a)}\partial_a\Psi = m\Psi,$$

where ∂_a is the derivative with respect to x_a and the sum runs over the seven directions $a = 1,2,3,5,6,7,8$ of a slice $x_4 = \text{const}$.

Step 1. Multiply from the left by $\gamma^{(x_4)}$ and use $\gamma^{(x_4)}\gamma^{(x_4)} = -I_{16}$:

$$-\partial_4\Psi + \sum_{a\neq 4}\gamma^{(x_4)}\gamma^{(x_a)}\partial_a\Psi = m\gamma^{(x_4)}\Psi.$$

Step 2. Add $\partial_4\Psi - m\gamma^{(x_4)}\Psi$ to both sides and exchange the two sides:

$$\partial_4\Psi = -m\gamma^{(x_4)}\Psi + \sum_{a\neq 4}\gamma^{(x_4)}\gamma^{(x_a)}\partial_a\Psi.$$

Step 3. A **plane wave** is $\Psi = u(x_4)\, e^{i\sum_{a\neq 4} k_a x_a}$ with a column u that depends on the time only and real numbers k_a, the **momenta** (wave numbers) along the slice. The derivative of $e^{ik_a x_a}$ with respect to x_a is $ik_a e^{ik_a x_a}$, so each ∂_a with $a \neq 4$ becomes the factor ik_a; then the common exponential cancels:

$$\frac{du}{dx_4} = -m\gamma^{(x_4)}u + i\sum_{a\neq 4} k_a\gamma^{(x_4)}\gamma^{(x_a)}u.$$

Step 4. Multiply by i and use $i\cdot i = -1$:

$$i\frac{du}{dx_4} = h\,u, \qquad h = -im\gamma^{(x_4)} - \gamma^{(x_4)}\sum_{a\neq 4} k_a\gamma^{(x_a)}.$$

The 16×16 matrix h is the **mode Hamiltonian** of the wave. The equation $i\,du/dx_4 = hu$ has the form of the Schrödinger equation of quantum mechanics; its solutions with $u(x_4) = u_0 e^{-iwx_4}$ are the eigenvectors $hu_0 = wu_0$, and the eigenvalue w is the **frequency** of the wave. A real w means oscillation; an imaginary $w = i\kappa$ means $e^{-iwx_4} = e^{\kappa x_4}$, growth.

Frame momenta and the deflating extra times. In the author's metric each derivative of the field equation is divided by the scale factor f_a of its direction (Chapter 6): $f_a = e^{a_4}\sin^{1/6} z$ for x_1, x_2, x_3, $f_4 = 1$, $f_a = e^{-a_4}\sin^{1/6} z$ for x_5, x_6, x_7 and $f_0 = \cot z$ (record `Revision/theory/field-theory.json`, formula `vielbein_diagonal`). A wave $e^{iq_a x_a}$ with the **coordinate momentum** q_a therefore enters the equation with the **frame momentum** $k_a = q_a/f_a$. Along 3-space $k_a = q_a e^{-a_4}\sin^{-1/6} z$ shrinks as 3-space inflates; along an extra time $k_a = q_a e^{a_4}\sin^{-1/6} z$ GROWS as the extra times deflate.

The frozen-coefficient model, and what it leaves out. The model evaluates the scale factors at one instant and one hidden position, so that the equation of that instant has constant coefficients and the plane waves above solve it with the frame momenta k_a; the extra times are never treated as static (Section 10.39 and Notebook 10f follow the frame momenta along the deflating history). But the author's field equation also contains two terms of the hidden

direction (Section 10.27; record `Revision/theory/field-theory.json`, formula `field_equation`): the derivative $\tan z\,\gamma^{(x_0)}\partial_0\Psi$, whose coefficient changes along x_0, and the spin-connection term $3H\gamma^{(x_0)}\Psi$ (Chapter 6). The model leaves both out, so it is an ASSUMPTION, an approximation whose error this chapter does not estimate. The connection term matters. Kept in the equation, it adds to h the matrix $h_c = 3iH\gamma^{(x_4)}\gamma^{(x_0)}$ (multiply $3H\gamma^{(x_0)}\Psi$ by $\gamma^{(x_4)}$ and by i, as in Steps 1 to 4). The real matrix $\gamma^{(x_4)}\gamma^{(x_0)}$ is symmetric (Chapter 5: $(\gamma^{(x_0)})^T = \gamma^{(x_0)}$ and $(\gamma^{(x_4)})^T = -\gamma^{(x_4)}$), $(\gamma^{(x_4)}\gamma^{(x_0)})^T = (\gamma^{(x_0)})^T(\gamma^{(x_4)})^T = \gamma^{(x_0)}(-\gamma^{(x_4)}) = \gamma^{(x_4)}\gamma^{(x_0)}$, so $h_c^\dagger = -h_c$; and h_c commutes with B, because $\gamma^{(x_4)}$ and $\gamma^{(x_0)}$ do (Section 10.2). Section 10.4 proves $Bh = h^\dagger B$ for the flat h; for $h + h_c$ this becomes

$$B(h+h_c) - (h+h_c)^\dagger B = (Bh - h^\dagger B) + Bh_c + h_cB = 2Bh_c = 6iH\,B\gamma^{(x_4)}\gamma^{(x_0)} \neq 0,$$

by $h_c^\dagger = -h_c$ and $h_cB = Bh_c$; the right side is not zero because B, $\gamma^{(x_4)}$ and $\gamma^{(x_0)}$ are invertible. So with the connection term the Krein form of a single wave is no longer conserved, Theorem 10.1 below does not apply, and the frequencies can be complex: Section 10.28 finds $\pm 2\sqrt{2}\,i$ for the waves that do not depend on x_0 at $m = H = 1$. Everything in Sections 10.4 to 10.6 and 10.20 holds exactly in flat 4+4 space and in the frozen-coefficient model; Sections 10.27 and 10.28 show what the terms of the hidden direction change.

10.4 The square of h, the Krein form, and three kinds of waves

The square. Write h as a sum of eight terms, $h = mE_0 + \sum_{a\neq 4} k_a E_a$, with $E_0 = -i\gamma^{(x_4)}$ and $E_a = -\gamma^{(x_4)}\gamma^{(x_a)}$. Three facts follow from the Clifford relation:

$$E_0E_0 = (-i)^2\gamma^{(x_4)}\gamma^{(x_4)} = -(-I_{16}) = I_{16},$$

$$E_aE_a = \gamma^{(x_4)}\gamma^{(x_a)}\gamma^{(x_4)}\gamma^{(x_a)} = -\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_a)}\gamma^{(x_a)} = -(-I_{16})(\eta_{aa}I_{16}) = \eta_{aa}I_{16},$$

where the second step exchanges the middle factors $\gamma^{(x_a)}\gamma^{(x_4)}$ at the cost of a sign; and any two different E's anticommute, for example

$$E_0E_a + E_aE_0 = i\big(\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_a)} + \gamma^{(x_4)}\gamma^{(x_a)}\gamma^{(x_4)}\big) = i\big(\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_a)} - \gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_a)}\big) = 0,$$

and in the same way $E_aE_b + E_bE_a = -\gamma^{(x_4)}\gamma^{(x_4)}(\gamma^{(x_a)}\gamma^{(x_b)} + \gamma^{(x_b)}\gamma^{(x_a)}) = 0$ for $a \neq b$. Now multiply out hh: the products of a term with itself give m^2I_{16} and $k_a^2\eta_{aa}I_{16}$, and the products of two different terms come in pairs $c_ic_j(E_iE_j + E_jE_i) = 0$. Therefore

$$h^2 = w^2 I_{16}, \qquad w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2.$$

The space-like momenta enter with $+$, the extra-time momenta with $-$ (because $\eta_{aa} = -1$ for $a = 5, 6, 7$). Every eigenvalue w' of h obeys $w'^2 = w^2$ (if $hu = w'u$ then $w^2u = hhu = w'^2u$), so the eigenvalues are $+w$ and $-w$; since $\operatorname{tr} h = 0$ (the traces of $\gamma^{(x_4)}$ and of $\gamma^{(x_4)}\gamma^{(x_a)}$ vanish, by the

argument of (B5) for a product of an odd number of gammas and by $\mathrm{tr}(\gamma^{(x_4)}\gamma^{(x_a)}) = \mathrm{tr}(\gamma^{(x_a)}\gamma^{(x_4)}) = -\mathrm{tr}(\gamma^{(x_4)}\gamma^{(x_a)})$ for two), each occurs eight times when $w \neq 0$.

Hermitian and anti-Hermitian parts. A matrix is **Hermitian** if $M^\dagger = M$ and **anti-Hermitian** if $M^\dagger = -M$. Using $(\gamma^{(x_a)})^T = \eta_{aa}\gamma^{(x_a)}$ (Chapter 5):

$$(-i\gamma^{(x_4)})^\dagger = i(\gamma^{(x_4)})^T = -i\gamma^{(x_4)},$$

$$(\gamma^{(x_4)}\gamma^{(x_a)})^\dagger = (\gamma^{(x_a)})^T(\gamma^{(x_4)})^T = -\eta_{aa}\gamma^{(x_a)}\gamma^{(x_4)} = \eta_{aa}\gamma^{(x_4)}\gamma^{(x_a)}.$$

So $-i\gamma^{(x_4)}$ and $\gamma^{(x_4)}\gamma^{(x_a)}$ for the space-like $a = 1, 2, 3, 8$ are Hermitian, while $\gamma^{(x_4)}\gamma^{(x_a)}$ for the extra times $a = 5, 6, 7$ is anti-Hermitian. Write $h = h_H + h_A$ with

$$h_H = -im\gamma^{(x_4)} - \gamma^{(x_4)}\big(k_1\gamma^{(x_1)} + k_2\gamma^{(x_2)} + k_3\gamma^{(x_3)} + k_0\gamma^{(x_0)}\big),$$

$$h_A = -\gamma^{(x_4)}\big(k_5\gamma^{(x_5)} + k_6\gamma^{(x_6)} + k_7\gamma^{(x_7)}\big).$$

h_H is Hermitian and h_A anti-Hermitian. By Section 10.2, $\gamma^{(x_4)}$ and the four space-like gammas commute with B, so h_H commutes with B; the gammas of x_5, x_6, x_7 anticommute with B while $\gamma^{(x_4)}$ commutes, so h_A anticommutes with B. Two consequences:

- h is Hermitian, and commutes with B, exactly when $k_5 = k_6 = k_7 = 0$: in the **good sector**, the waves that do not depend on the extra times;
- for all momenta, $h^\dagger B = (h_H - h_A)B = Bh_H + Bh_A = Bh$. In words, h is **Krein self-adjoint**: $Bh = h^\dagger B$.

The Krein form is conserved. Let $u(x_4)$ solve $i\,du/dx_4 = hu$, so $du/dx_4 = -ihu$ and, taking the conjugate transpose, $du^\dagger/dx_4 = iu^\dagger h^\dagger$. By the product rule

$$\frac{d}{dx_4}(u^\dagger Bu) = \frac{du^\dagger}{dx_4}Bu + u^\dagger B\frac{du}{dx_4} = iu^\dagger h^\dagger Bu - iu^\dagger Bhu = i\,u^\dagger(h^\dagger B - Bh)u = 0.$$

The last step is $h^\dagger B = Bh$. The Krein norm of every wave is constant in time, also when the wave grows; the ordinary squared length $u^\dagger u$ changes by $d(u^\dagger u)/dx_4 = iu^\dagger(h^\dagger - h)u = -2iu^\dagger h_A u$, which vanishes in the good sector only.

Three kinds of waves. Because $h^2 = w^2 I_{16}$, the exponential series of the solution collapses. The solution is $u(x_4) = e^{-ihx_4}u(0)$ with $e^X = I_{16} + X + X^2/2! + \ldots$; the even powers of $-ihx_4$ are $(-1)^j w^{2j} x_4^{2j} I_{16}$ and the odd powers are $-i(-1)^j w^{2j} x_4^{2j+1} h$, so

$$e^{-ihx_4} = \cos(wx_4)\, I_{16} - i\,\frac{\sin(wx_4)}{w}\, h,$$

by the power series of the cosine and the sine. Three cases follow:

- $w^2 > 0$: real frequencies $\pm w$, the wave oscillates;
- $w^2 = 0$ with $h \neq 0$: $h^2 = 0$ and $e^{-ihx_4} = I_{16} - ihx_4$, the wave grows linearly;
- $w^2 < 0$, that is $k_5^2 + k_6^2 + k_7^2 > m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2$: $w = i\kappa$ with $\kappa = \sqrt{-w^2}$, and since $\cos(i\kappa x_4) = \cosh(\kappa x_4)$ and $\sin(i\kappa x_4)/(i\kappa) = \sinh(\kappa x_4)/\kappa$, half of the waves grow like $e^{\kappa x_4}$.

A large enough extra-time momentum always produces growth, and the growth rate κ has no upper bound as that momentum grows: the initial-value problem with data that depend on the extra times is not well posed (Chapter 8).

statement	**status**	**where it is verified**
$h^2 = w^2 I_{16}$ for all real m, k	PROVED	`wolfram-pairing.json`, check `Q_one_particle_flat_dispersion`; `python-field-theory.json`, check `mode_hamiltonian_B_selfadjoint_dispersion`
$Bh = h^\dagger B$ for all real m, k	PROVED	`python-field-theory.json`, check `mode_hamiltonian_B_selfadjoint_dispersion`; `wolfram-field-theory.json`, check `mode_hamiltonian_Krein_selfadjoint`
h Hermitian and $[B, h] = 0$ exactly in the good sector	PROVED	`wolfram-field-theory.json`, check `mode_hamiltonian_good_sector`; `python-field-theory.json`, check `good_sector_spectrum_and_B_sectors`
growth for $w^2 < 0$, rates unbounded	PROVED	`python-field-theory.json`, check `extra_time_modes_grow`; `python-scope.json`, check `extra_time_growth_rates_unbounded`

10.5 Krein inertia of the eigenspaces

Projectors onto the eigenspaces. An **eigenspace** of h is the set of all columns u with $hu = w'u$ for one eigenvalue w' (with the zero column). For $w \neq 0$ define

$$P_\pm = \tfrac{1}{2}(I_{16} \pm h/w).$$

Then $P_\pm P_\pm = \tfrac{1}{4}(I_{16} \pm 2h/w + h^2/w^2) = \tfrac{1}{4}(2I_{16} \pm 2h/w) = P_\pm$, by $h^2 = w^2 I_{16}$: $P_\pm$ is a **projector** (a matrix with $PP = P$, which maps every column into one subspace, its range, and leaves the

columns of that range unchanged). And $hP_\pm = \frac{1}{2}(h \pm w^2/w\, I_{16}) = \pm w P_\pm$, so the range of P_+ is the eigenspace of $+w$ and the range of P_- that of $-w$. Since $P_+ + P_- = I_{16}$, every column is the sum of a column of each.

Krein inertia. Choose a basis $v_1, \ldots, v_d$ of a subspace and form the **Gram matrix** $G_{ij} = v_i^\dagger B v_j$. It is Hermitian; its numbers of positive, negative and zero eigenvalues do not depend on the basis chosen (a fact of linear algebra called Sylvester's law of inertia), and they form the **Krein inertia** (p, n, z) of the subspace. It says how many independent waves of the subspace carry positive, negative and zero Krein norm. A subspace on which $u^\dagger B v = 0$ for all its members has inertia $(0, 0, d)$ and is called **Krein-neutral**.

Theorem 10.1 (Krein inertia of one-particle waves). In flat 4+4 space (or in the frozen-coefficient model of Section 10.3, an ASSUMPTION that leaves out the connection term), for every real frequency, $w^2 > 0$, with or without extra-time momentum, each of the two eigenspaces of h has dimension 8 and Krein inertia (4,4). For every imaginary frequency both eigenspaces are Krein-neutral, and for $w = 0$ the range of h is Krein-neutral.

statement	status	where it is verified
Theorem 10.1, real frequencies: inertia (4,4)	PROVED	`wolfram-pairing.json`, check `Q_one_particle_Krein_inertia_real_frequencies`; `python-pairing.json`, check `Q.one_particle_Krein_inertia_proof`
Theorem 10.1, imaginary and zero frequencies: Krein-neutral	PROVED	`wolfram-pairing.json`, check `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`; `python-pairing.json`, check `Q.one_particle_complex_frequency_Krein_neutral`

Proof, step (i): the two eigenspaces are Krein-orthogonal. Let $w > 0$ be real, so $P_+^\dagger = \frac{1}{2}(I_{16} + h^\dagger/w)$. Then

$$4w^2\, P_+^\dagger B P_- = (wI_{16} + h^\dagger)\, B\, (wI_{16} - h) = w^2 B - wBh + wh^\dagger B - h^\dagger B h.$$

The first step multiplies each projector by $2w$; the second multiplies out. By $h^\dagger B = Bh$ the two middle terms cancel, and $h^\dagger B h = Bhh = w^2 B$, so the right side is $w^2 B - w^2 B = 0$. A wave of frequency $+w$ and one of $-w$ have zero mixed Krein form. In the same way $P_+^\dagger B P_+ = B P_+$. Because B is invertible and the two eigenspaces together span all 16 directions, B cannot vanish on either eigenspace: if a column u of the $+w$ eigenspace had $u^\dagger B v = 0$ for all v in that eigenspace, it would also have it for all v in the other (step (i)), hence for all columns, so $Bu = 0$ and $u = BBu = 0$. Such a form is called **nondegenerate** on the eigenspace: its Gram matrix has no zero eigenvalue.

Step (ii): in the good sector the inertia is (4,4). There $Bh = hB$, so B commutes with $P_\pm$ and maps each eigenspace into itself. In the good sector h is also Hermitian (Section 10.4), so P_+

is Hermitian, and its range, the $+w$ eigenspace, has dimension $\operatorname{tr} P_+ = \frac{1}{2}(16 + \operatorname{tr} h/w) = 8$ (the trace of a projector is the dimension of its range; $\operatorname{tr} h = 0$ because h is a combination of $\gamma^{(x_4)}$ and of products of two different gammas, which have trace 0 by the argument given below for Bh). Choose an orthonormal basis $v_1, \ldots, v_8$ of this eigenspace ($v_i^\dagger v_j$ is 1 for $i = j$ and 0 otherwise); then $P_+ = \sum_i v_i v_i^\dagger$. Because Bv_j lies in the same eigenspace, $Bv_j = \sum_i v_i \, (v_i^\dagger B v_j) = \sum_i v_i G_{ij}$: the Gram matrix $G_{ij} = v_i^\dagger B v_j$ of the Krein form in this basis is exactly the matrix of B restricted to the eigenspace. So its eight eigenvalues are eigenvalues of B, each $+1$ or -1 (because $BB = I_{16}$), and the Krein inertia counts how many are $+1$ and how many -1. Their sum is the trace of the Gram matrix, $\sum_i v_i^\dagger B v_i = \operatorname{tr}(B \sum_i v_i v_i^\dagger) = \operatorname{tr}(BP_+)$, by $\operatorname{tr}(XY) = \operatorname{tr}(YX)$ for the column v_i and the row $v_i^\dagger B$. Now

$$\operatorname{tr}(BP_\pm) = \tfrac{1}{2} \operatorname{tr} B \pm \tfrac{1}{2w} \operatorname{tr}(Bh) = 0,$$

because $\operatorname{tr} B = 0$ (Section 10.2) and $Bh = mC - i \sum_a k_a C \gamma^{(x_a)}$ is a combination of C (a product of four different gammas) and of products of three or five different gammas, all of trace 0: a product of an odd number of different gammas has trace 0 by the argument of (B5), and a product of an even number k of different gammas has trace 0 because moving its first factor to the end does not change the trace but costs the sign $(-1)^{k-1} = -1$. Eight numbers ± 1 with the sum 0 are four $+1$ and four -1: inertia (4,4).

Step (iii): every real frequency. Start in the good sector and turn on the extra-time momenta slowly, keeping m, k_1, k_2, k_3, k_0 fixed. As long as $w^2 > 0$ the projectors $P_\pm$ change continuously, and by step (i) the Gram matrix of each eigenspace never has a zero eigenvalue. An eigenvalue of a continuously changing Gram matrix can change its sign only by passing through 0, which is forbidden, so the inertia stays (4,4). Every real-frequency point can be reached in this way (lowering k_5, k_6, k_7 to 0 only increases w^2).

Imaginary frequencies. Let w be an eigenvalue that is not real, and $hu = wu$, $hv = wv$. Compute $u^\dagger B h v$ in two ways: it is $w \, u^\dagger B v$; and with $Bh = h^\dagger B$ it is $u^\dagger h^\dagger B v = (hu)^\dagger B v = w^* \, u^\dagger B v$. Subtracting, $(w - w^*) \, u^\dagger B v = 0$, and $w - w^* \neq 0$, so $u^\dagger B v = 0$ on the whole eigenspace. *Zero frequency:* $h^2 = 0$ and $(hx)^\dagger B (hy) = x^\dagger h^\dagger B h y = x^\dagger B h h y = 0$. QED.

What it means. In flat 4+4 space the waves of every real frequency mix positive and negative charge half and half; the good sector removes the complex frequencies, but not this indefiniteness of the one-particle waves. The growing waves carry no charge at all: their eigenspaces are neutral.

10.6 The universes of masses $+m$ and $-m$ have the same one-particle spectrum

The pairing theorem T1 of the Revision record (proved in Chapter 18) relates the field of mass m to the field of mass $-m$ by the chirality matrix: if Ψ solves the equations with (m, λ), then $\Gamma\Psi$ solves them with $(-m, -\lambda)$. What does this do to the one-particle waves? Write $h_m(k)$ for the mode Hamiltonian of mass m and momenta k.

The chirality map. Γ anticommutes with every gamma and $\Gamma\Gamma = I_{16}$, so

$$\Gamma \gamma^{(x_4)} \Gamma = -\gamma^{(x_4)} \Gamma\Gamma = -\gamma^{(x_4)},$$

$$\Gamma\gamma^{(x_4)}\gamma^{(x_a)}\Gamma = (\Gamma\gamma^{(x_4)}\Gamma)(\Gamma\gamma^{(x_a)}\Gamma) = (-\gamma^{(x_4)})(-\gamma^{(x_a)}) = \gamma^{(x_4)}\gamma^{(x_a)},$$

where the second identity inserts $\Gamma\Gamma = I_{16}$ between the two gammas. Insert into $h_m(k)$:

$$\Gamma h_m(k)\Gamma = +im\gamma^{(x_4)} - \gamma^{(x_4)}\sum_{a\neq 4} k_a\gamma^{(x_a)} = h_{-m}(k).$$

The mirror map. In the same way, with $\gamma^{(x_0)}\gamma^{(x_0)} = I_{16}$: $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)} = -\gamma^{(x_4)}$; $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_a)}\gamma^{(x_0)} = \gamma^{(x_4)}\gamma^{(x_a)}$ for $a \neq 4, 8$ (two sign changes); and $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_0)} = -\gamma^{(x_4)}\gamma^{(x_0)}$ (one sign change). So $\gamma^{(x_0)}h_m(k)\gamma^{(x_0)} = h_{-m}(R_0k)$, where R_0k is k with k_0 replaced by $-k_0$.

Equal spectra. If $h_mu = w'u$, then $h_{-m}(\Gamma u) = \Gamma h_m\Gamma\Gamma u = \Gamma h_m u = w'\,\Gamma u$: the matrix Γ carries every eigenvector of h_m to an eigenvector of h_{-m} with the same eigenvalue. Matrices related in this way ($N = SMS^{-1}$) are called **similar**, and similar matrices have the same eigenvalues. So the one-particle spectra of the universes of masses $+m$ and $-m$ are IDENTICAL, not opposite; the same follows from w^2, which contains the mass only as m^2. (PROVED: `wolfram-pairing.json`, checks `Q_one_particle_maps` and `Q_one_particle_Krein_signatures`; `python-pairing.json`, check `Q.one_particle_maps`.) This is statement Q4 of the quantum reading of the Revision record; Sections 10.44 and 10.45 discuss the whole quantum reading and what it does not establish.

10.7 Example: Notebook 10a computes the one-particle spectra and their Krein inertia

Notebook 10a reads the author's gammas from the record `Revision/algebra/gammas.json`, builds C and B, checks the properties of Section 10.2, builds the mode Hamiltonian of Section 10.3, proves exactly with sympy that $h^2 = w^2 I_{16}$ and $Bh = h^\dagger B$ (Section 10.4), follows the eigenvalues as the extra-time momentum grows, computes the Krein inertia of the eigenspaces for the eight samples stored in the pairing record and for a scan across the threshold of growth, repeats steps (i) and (ii) of the proof of Theorem 10.1 exactly, and checks the identities of Section 10.6. It draws six figures and ends with the line ALL 28 CHECKS PASSED (notebook 10a).

10.8 How to run Notebook 10a

Step 1. What this notebook does and what it needs.

Notebook 10a (One-particle spectra and Krein signatures) is the file `Revision/textbook/notebooks/10a_krein_spectra.ipynb` of the repository Dirac_claude. It builds the Krein matrix B from the author's gamma matrices, the one-particle mode Hamiltonian h of a plane wave in flat 4+4 space, proves exactly with sympy that h squared is the squared frequency times the identity and that B h equals h dagger B, computes the eigenvalues of h as the momentum along an extra time grows, the Krein inertia of every eigenspace (4 positive and 4 negative directions at every real frequency, none at an imaginary frequency), checks that the universes of masses +m and -m have the same one-particle spectrum, reproduces the recorded samples of the Revision pairing record, and draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with

Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10a_krein_spectra.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10a_krein_spectra.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 10a_krein_spectra.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10a.captions.json`
- `Revision/textbook/figures/10a_1_gamma4_c_and_b.png`
- `Revision/textbook/figures/10a_2_frequency_squared.png`
- `Revision/textbook/figures/10a_3_eigenvalue_flow.png`
- `Revision/textbook/figures/10a_4_krein_inertia_scan.png`
- `Revision/textbook/figures/10a_5_krein_gram_matrices.png`
- `Revision/textbook/figures/10a_6_plus_minus_mass_spectra.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10a_6_plus_minus_mass_spectra.png exists
ALL 28 CHECKS PASSED (notebook 10a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10a_krein_spectra.ipynb` inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10a_krein_spectra.ipynb
python -m nbconvert --execute --inplace 10a_krein_spectra.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.9 Notebook 10a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10a_krein_spectra.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when

it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook studies ONE wave of the field dirac16complex at a time: a plane wave
with fixed momenta along the seven directions of a slice $x_4 = $ const, in flat
4+4 space (or in the frozen-coefficient model of the author's universe, which has
the same equation but is an ASSUMPTION; section 4 says exactly what it leaves out).
It

- builds, from the author's gamma matrices, the matrix $B = -iC\gamma^{(x_4)}$ that
  appears in the charge density $\Psi^\dagger B \Psi$ and later in the canonical
  anticommutator, and checks that $B$ is Hermitian, squares to 1 and has eight
  eigenvalues $+1$ and eight eigenvalues $-1$;
- writes the field equation of one plane wave as $i\,du/dx_4 = h\,u$ with a
  $16 \times 16$ matrix $h$, the *mode Hamiltonian*;
- proves with exact algebra (sympy) that $h^2 = w^2 I_{16}$ with
  $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$, and that
  $B h = h^\dagger B$;
- follows the eigenvalues of $h$ as the momentum $k_5$ along the extra time $x_5$
  grows: they are real (oscillation) while $w^2 > 0$ and imaginary (growth) when
  $w^2 < 0$;
- computes the *Krein inertia* of every eigenspace: how many directions of positive
  and of negative charge it contains. The answer is 4 and 4 at every real
  frequency, and 0 and 0 (a *neutral* eigenspace) at every imaginary frequency;
- checks that the masses $+m$ and $-m$ give the SAME one-particle spectrum;
- reproduces every recorded number of the Revision record that it touches, and
  draws six teaching figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 10a (One-particle spectra and Krein signatures) is the file
‘Revision/textbook/notebooks/10a_krein_spectra.ipynb‘ of the repository Dirac_claude. It
builds the Krein matrix B from the author's gamma matrices, the one-particle mode
Hamiltonian h of a plane wave in flat 4+4 space, proves exactly with sympy that h squared
is the squared frequency times the identity and that B h equals h dagger B, computes the
eigenvalues of h as the momentum along an extra time grows, the Krein inertia of every
eigenspace (4 positive and 4 negative directions at every real frequency, none at an
imaginary frequency), checks that the universes of masses +m and -m have the same
one-particle spectrum, reproduces the recorded samples of the Revision pairing record,
and draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
notebooks were built with Python 3.14.5) with these packages at exactly these versions:
numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
```

shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then

needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 10a_krein_spectra.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 10a_krein_spectra.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10a_krein_spectra.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10a.captions.json‘

- ‘Revision/textbook/figures/10a_1_gamma4_c_and_b.png‘
- ‘Revision/textbook/figures/10a_2_frequency_squared.png‘
- ‘Revision/textbook/figures/10a_3_eigenvalue_flow.png‘
- ‘Revision/textbook/figures/10a_4_krein_inertia_scan.png‘
- ‘Revision/textbook/figures/10a_5_krein_gram_matrices.png‘
- ‘Revision/textbook/figures/10a_6_plus_minus_mass_spectra.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10a_6_plus_minus_mass_spectra.png exists
ALL 28 CHECKS PASSED (notebook 10a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10a_krein_spectra.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
      python -m jupyterlab 10a_krein_spectra.ipynb
      python -m nbconvert --execute --inplace 10a_krein_spectra.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
  the Revision record: the data it starts from and the report files whose checks it
  reproduces (each cited check must still exist there with the verdict PASS). It must be
  opened inside the folder Revision/textbook/notebooks of a complete copy of the
  repository (a notebook copied alone to another folder cannot find them). Clone the
  repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the
  Revision record no longer holds the check that the notebook names, or holds it with
  another verdict: the copy of the repository is incomplete or was changed. Clone the
  repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 10a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 10a (One-particle spectra and Krein signatures) is the file
# Revision/textbook/notebooks/10a_krein_spectra.ipynb of the repository Dirac_claude. It
# builds the Krein matrix B from the author's gamma matrices, the one-particle mode
# Hamiltonian h of a plane wave in flat 4+4 space, proves exactly with sympy that h
# squared is the squared frequency times the identity and that B h equals h dagger B,
# computes the eigenvalues of h as the momentum along an extra time grows, the Krein
# inertia of every eigenspace (4 positive and 4 negative directions at every real
# frequency, none at an imaginary frequency), checks that the universes of masses +m and
# -m have the same one-particle spectrum, reproduces the recorded samples of the Revision
# pairing record, and draws six teaching figures. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
```

```
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
```

```
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
```

```
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10a_krein_spectra.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10a_krein_spectra.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10a_krein_spectra.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10a.captions.json
# - Revision/textbook/figures/10a_1_gamma4_c_and_b.png
# - Revision/textbook/figures/10a_2_frequency_squared.png
# - Revision/textbook/figures/10a_3_eigenvalue_flow.png
# - Revision/textbook/figures/10a_4_krein_inertia_scan.png
# - Revision/textbook/figures/10a_5_krein_gram_matrices.png
# - Revision/textbook/figures/10a_6_plus_minus_mass_spectra.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
```

```
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 10a_6_plus_minus_mass_spectra.png exists
#     ALL 28 CHECKS PASSED (notebook 10a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10a_krein_spectra.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10a_krein_spectra.ipynb
#     python -m nbconvert --execute --inplace 10a_krein_spectra.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
```

```
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10a"  # this notebook: chapter 10, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
```

```
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
```

```
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Matrix, entry, identity**: a matrix is a table of numbers; $M_{AB}$ is the entry
  in row $A$ and column $B$. $I_{16}$ (in code `np.eye(16)`) is the $16 \times 16$
  identity matrix: 1 on the diagonal, 0 elsewhere.
- **Complex conjugate, transpose, dagger**: $z^*$ changes $i$ into $-i$; $M^T$
  exchanges rows and columns; $M^\dagger = (M^T)^*$ is the *conjugate transpose*
  (in code `M.conj().T`). For a column $u$, $u^\dagger$ is the row of the
  conjugated entries, and $u^\dagger v = \sum_A u_A^* v_A$.
- **Hermitian**: $M^\dagger = M$. A Hermitian matrix has real eigenvalues.
- **Eigenvalue, eigenvector, eigenspace**: if $M u = \lambda u$ with $u \neq 0$,
  then $\lambda$ is an eigenvalue and $u$ an eigenvector; all eigenvectors of one
  eigenvalue (with the zero column) form its eigenspace.
- **Projector**: a matrix $P$ with $P^2 = P$. It maps every column into one
  subspace (its *range*) and leaves the columns of that subspace unchanged.
- **Trace**: $\mathrm{tr}\,M$, the sum of the diagonal entries; it equals the sum
  of the eigenvalues.
- **Krein form**: the number $u^\dagger B v$ for two columns $u, v$. Because $B$ is
  Hermitian, $u^\dagger B u$ is real, but it can be positive, negative or zero. In
  the theory it is the charge of the wave $u$.
- **Signature, inertia**: a Hermitian matrix with $p$ positive, $n$ negative and $z$
  zero eigenvalues has inertia $(p, n, z)$; the signature of $B$ is $(8, 8)$. The
  *Krein inertia of a subspace* is the inertia of the Krein form restricted to it.
- **Neutral subspace**: a subspace on which $u^\dagger B v = 0$ for all $u, v$ in it.
- **Plane wave, momentum, frequency**: $\Psi = u\,e^{i(k\cdot x - w x_4)}$ with a
  constant column $u$; the numbers $k_a$ are the momenta (wave numbers) along the
  slice directions and $w$ is the frequency. A real $w$ means oscillation, an
  imaginary $w = i\kappa$ means growth like $e^{\kappa x_4}$.
```

```
- **Good sector**: waves that do not depend on the extra times $x_5, x_6, x_7$,
  that is $k_5 = k_6 = k_7 = 0$.
- **Chirality** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$: the
  product of all eight gammas in the author's order.
- **sympy, numpy**: sympy computes EXACTLY with symbols (a proof for all values);
  numpy computes with floating-point numbers (about 16 digits), so its checks
  allow a tiny tolerance such as $10^{-10}$.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's coordinates are $x_1, x_2, x_3$ (ordinary 3-space), $x_4$ (the time),
$x_5, x_6, x_7$ (the three EXTRA TIMES, which deflate exponentially in the author's
metric) and $x_0$ (the hidden space direction). The frame metric is
$\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \dots, x_7$:
the gammas obey $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} =
2\eta^{ab} I_{16}$, so $(\gamma^{(a)})^2 = +1$ for $x_0, x_1, x_2, x_3$ and $-1$
for $x_4, \dots, x_7$.

**From the field equation to the mode Hamiltonian, line by line.** With $U = 0$ and
without gravity (flat 4+4 space, or the frozen-coefficient model described below)
the field equation of dirac16complex is

$$\gamma^{(x_4)}\partial_4\Psi + \sum_{a \neq 4}\gamma^{(x_a)}\partial_a\Psi = m\Psi .$$

Multiply from the left by $\gamma^{(x_4)}$ and use $(\gamma^{(x_4)})^2 = -1$:

$$-\partial_4\Psi + \sum_{a \neq 4}\gamma^{(x_4)}\gamma^{(x_a)}\partial_a\Psi =
m\gamma^{(x_4)}\Psi .$$

Move $\partial_4\Psi$ to the right-hand side and everything else to the left, then
exchange the two sides:

$$\partial_4\Psi = -m\gamma^{(x_4)}\Psi + \sum_{a \neq 4}
\gamma^{(x_4)}\gamma^{(x_a)}\partial_a\Psi .$$

For a plane wave $\Psi = u(x_4)\,e^{i\sum_{a \neq 4} k_a x_a}$ every $\partial_a$
with $a \neq 4$ becomes the factor $i k_a$ (the derivative of $e^{ik_a x_a}$ is
$ik_a e^{ik_a x_a}$); cancel the common exponential:

$$\frac{du}{dx_4} = -m\gamma^{(x_4)}u + i\sum_{a \neq 4} k_a
\gamma^{(x_4)}\gamma^{(x_a)}u .$$

Multiply by $i$ (and use $i \cdot i = -1$):

$$i\frac{du}{dx_4} = h\,u,\qquad h = -im\gamma^{(x_4)} - \gamma^{(x_4)}
\sum_{a \neq 4} k_a\gamma^{(x_a)} .$$

**Frame momenta and the deflating extra times.** The numbers $k_a$ are momenta in
the local frame. In the author's metric the field equation contains the derivative
along $x_a$ divided by the scale factor $f_a$ of that direction (Revision theory
record): $f_a = e^{a_4}\sin^{1/6}z$ for $x_1, x_2, x_3$ and $f_a =
```

e^{-a_4}\sin^{1/6}z$ for the extra times x_5, x_6, x_7. A wave $e^{iq_ax_a}$ with the coordinate momentum q_a therefore has the frame momentum $k_a = q_a/f_a$: along 3-space $k_a = q_ae^{-a_4}\sin^{-1/6}z$ shrinks as 3-space inflates, and along an extra time $k_a = q_ae^{a_4}\sin^{-1/6}z$ GROWS as the extra times deflate (a_4 increasing). The *frozen-coefficient model* evaluates these factors at one time x_4 and one hidden position, so that the wave equation of that instant has constant coefficients, AND it leaves out the two terms of the hidden direction that the author's field equation also contains: the derivative $\tan z\, \gamma^{(x_0)}\partial_0\Psi$ with its coefficient that changes along x_0, and the spin-connection term $3H\gamma^{(x_0)}\Psi$ (Revision theory record, formula 'field_equation'). The model is therefore an ASSUMPTION. Kept in the equation, the connection term would add $3iH\gamma^{(x_4)}\gamma^{(x_0)}$ to the matrix h above; that matrix commutes with B and is anti-Hermitian, so it would spoil the identity $Bh = h^\dagger B$ that this notebook proves, and the frequencies could become complex. The extra times are not static: the notebook "The Krein structure along the deflating history" of this chapter follows the frame momenta, and everything this notebook computes, along the deflating history.

The Krein form is conserved. The charge density of the field is $\Psi^\dagger B\Psi$ with $B = -iC\gamma^{(x_4)}$, $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ (the author's sigma16). From $i\,du/dx_4 = hu$ we get $du/dx_4 = -ihu$ and, taking the dagger, $du^\dagger/dx_4 = i\,u^\dagger h^\dagger$. By the product rule

$$\frac{d}{dx_4}(u^\dagger B\,u) = i\,u^\dagger h^\dagger B\,u - i\,u^\dagger B h\,u = i\,u^\dagger(h^\dagger B - B h)u\ ,$$

which is zero for every u exactly when $B\,h = h^\dagger B$. The notebook proves this identity. The ordinary squared length $u^\dagger u$, on the other hand, is conserved only when h is Hermitian.

Krein inertia. Because $h^2 = w^2\,I_{16}$, for $w \neq 0$ the two matrices $P_\pm = \frac12(I_{16} \pm h/w)$ are projectors onto the eigenspaces of $\pm w$ (check: $P_\pm^2 = \frac14(I \pm 2h/w + h^2/w^2) = \frac14(2I \pm 2h/w) = P_\pm$, and $hP_\pm = \frac12(h \pm w^2/w) = \pm w\,P_\pm$). The Krein inertia of an eigenspace tells how many of its independent waves carry positive and how many negative charge. Every statement here is a statement about one-particle waves; the quantum field built from them is the subject of the later notebooks of this chapter.

Text cell:

5. The gamma matrices and the Krein matrix B

The next cell reads the author's gamma matrices from the Revision record 'Revision/algebra/gammas.json' (16 x 16 matrices of whole numbers), stores them as 'gamma[1]', ..., 'gamma[0]' so that 'gamma[a]' is $\gamma^{(x_a)}$, and checks the 64 Clifford relations exactly (whole numbers, no rounding).

It also defines 'check_record(condition, name, record)'. It does what 'check(condition, name, record=record)' of the set-up cell does (stop with an error if the condition is false, otherwise print the PASS line and the line naming the Revision record that the check reproduces), but it first collects the two printed lines and then prints them in one piece. Jupyter sends printed text to the screen

in pieces whose boundaries depend on timing; printing the two lines in one piece
keeps them together in every run, so that the stored output of the notebook is the
same every time. Before it checks anything, ‘check_record‘ asks the helper
‘record_says_pass(record)‘ whether the cited record still says what the notebook
claims: a record name of the form ‘<report file>, check <check name>‘ names a check
of a Revision report, and the helper opens that report (each file only once, kept
in the dictionary ‘REPORT_CHECKS‘) and answers True only if a check of that name
exists there with the verdict PASS. If not, the notebook stops with an error
instead of printing an outdated claim. A record name without ‘, check ‘ names a data
entry of a file that the notebook reads and compares itself.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
# gamma[a] is the matrix gamma^(x_a) of the coordinate x_a, a = 1, ..., 8.
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
eta = {a: fixture["eta"][a] for a in range(8)}  # +1 space-like, -1 time-like
identity = np.eye(16, dtype=int)
say(f"eta = {[eta[a] for a in range(8)]} for x0, ..., x7")
clifford_ok = all(
```

```
    np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                   2 * (eta[a] if a == b else 0) * identity)
    for a in range(8) for b in range(8))  # all 64 pairs (a, b)
check(clifford_ok,
      "the 64 Clifford relations hold exactly (eta = diag(+,+,+,+,-,-,-,-))")
```

Out [2]:

```
eta = [1, 1, 1, 1, -1, -1, -1, -1] for x0, ..., x7
PASS the 64 Clifford relations hold exactly (eta = diag(+,+,+,+,-,-,-,-))
```

Text cell:

```
The next cell builds $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$
and $B = -iC\gamma^{(x_4)}$ (in Python the imaginary unit is written `1j`), and
compares $B$ with the matrix `B` stored in the same record (its real and imaginary
parts are stored separately). Then it checks the four properties of $B$ that the
chapter uses: $B$ is purely imaginary, $B^\dagger = B$, $B^2 = I_{16}$, and its
trace is 0. Since $B^2 = I$, every eigenvalue $\lambda$ obeys $\lambda^2 = 1$, so it is
$+1$ or $-1$; the trace 0 (the sum of the eigenvalues) then forces eight of each:
the signature $(8, 8)$. The cell confirms this by computing the eigenvalues.
```

In [3]:

```
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4), a complex 16 x 16 matrix
B_file = np.array(fixture["B"]["re"]) + 1j * np.array(fixture["B"]["im"])
check_record(np.array_equal(B, B_file),
             "B = -i C gamma^(x4) equals the matrix B of the record",
             record="Revision/algebra/gammas.json, entry B")
purely_imaginary = np.all(B.real == 0)  # every entry is 0, +i or -i
hermitian = np.array_equal(B, B.conj().T)  # B^dagger = B
squares_to_one = np.array_equal(B @ B, np.eye(16))  # B^2 = I
trace_B = np.trace(B)  # the sum of the diagonal entries
eigenvalues_B = np.linalg.eigvalsh(B)  # the 16 real eigenvalues, sorted
n_plus = int(np.sum(eigenvalues_B > 0.5))  # how many are +1
n_minus = int(np.sum(eigenvalues_B < -0.5))  # how many are -1
report("trace of B", int(round(abs(trace_B))))
report("eigenvalues of B equal to +1 and to -1", f"{n_plus} and {n_minus}")
check_record(purely_imaginary and hermitian and squares_to_one and trace_B == 0
             and (n_plus, n_minus) == (8, 8),
             "B is imaginary and Hermitian, B^2 = I, tr B = 0, signature (8, 8)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "B_properties")
```

Out [3]:

```
PASS B = -i C gamma^(x4) equals the matrix B of the record
     reproduces Revision/algebra/gammas.json, entry B
```

```
RESULT trace of B = 0
RESULT eigenvalues of B equal to +1 and to -1 = 8 and 8
PASS B is imaginary and Hermitian, B^2 = I, tr B = 0, signature (8, 8)
     reproduces Revision/theory/reports/python-field-theory.json, check B_properties
```

Text cell:

```
The next cell defines the colours of all figures of this notebook (a blue, an
orange and a green that stay distinguishable for colour-blind readers, and a
blue-grey-red colour scale for matrices: blue for $-1$, light grey for 0, red for
$+1$) and a small function that draws a matrix as a coloured square grid, a *heat
map*. It checks what the picture will show: $\gamma^{(x_4)}$ is antisymmetric
($M^T = -M$), $C$ is symmetric ($M^T = M$), the imaginary part of $B$ is
antisymmetric, and each of the three has exactly one nonzero entry in every row
and every column. Why the imaginary part of $B$ must be antisymmetric: write
$B = iY$ with a real matrix $Y$; then $B^\dagger = (iY)^{*T} = -iY^T$, and
$B^\dagger = B$ means $-iY^T = iY$, that is $Y^T = -Y$. Then it draws
$\gamma^{(x_4)}$, $C$ and the imaginary part of $B$ side by side.
```

In [4]:

```
from matplotlib.colors import LinearSegmentedColormap  # colour scales

BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
# A diverging colour scale: dark blue (negative), light grey (zero), red (positive).
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])
# A sequential colour scale for sizes: almost white (zero) to dark blue (large).
SEQUENTIAL = LinearSegmentedColormap.from_list(
    "white_blue", ["#fcfcfb", "#86b6ef", "#184f95"])


def draw_matrix(ax, matrix, title, cmap=DIVERGING, vmin=-1.0, vmax=1.0):
    """Draw a real matrix as a heat map with rows and columns numbered from 1."""
    image = ax.imshow(matrix, cmap=cmap, vmin=vmin, vmax=vmax)
    size = matrix.shape[0]
    ticks = list(range(0, size, 3))  # every third row and column is labelled
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.grid(False)  # no grid lines on top of the squares
    ax.set_title(title)
    return image


# A purely imaginary B = i Y (Y real) is Hermitian exactly when -i Y^T = i Y,
# that is when Y^T = -Y: the imaginary part of B must be antisymmetric.
one_per_row = all(np.count_nonzero(M, axis=1).tolist() == [1] * 16
                  and np.count_nonzero(M, axis=0).tolist() == [1] * 16
                  for M in (gamma[4], C, B.imag))  # signed permutation matrices
check(np.array_equal(gamma[4].T, -gamma[4]) and np.array_equal(C.T, C)
      and np.array_equal(B.imag.T, -B.imag) and one_per_row,
      "gamma^(x4) and Im B are antisymmetric, C is symmetric, all signed permutations")
```

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], gamma[4], "$\\gamma^{(x_4)}$ (real, antisymmetric)")
draw_matrix(axes[1], C, "$C$ (real, symmetric)")
image = draw_matrix(axes[2], B.imag, "imaginary part of $B = -iC\\gamma^{(x_4)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "gamma4_c_and_b",
            "Heat maps of three 16 by 16 matrices: the time gamma $\\gamma^{(x_4)}$ "
            "(left), the matrix $C = \\gamma^{(x_0)}\\gamma^{(x_1)}\\gamma^{(x_2)}"
            "\\gamma^{(x_3)}$ (middle) and the imaginary part of $B = -iC"
            "\\gamma^{(x_4)}$ (right; its real part is zero). Horizontal axis: "
            "column number, vertical axis: row number; red is $+1$, blue is $-1$, "
            "light grey is 0. Each row and each column has exactly one nonzero "
            "entry: all three are signed permutation matrices, so $B$ is $i$ times "
            "a real signed permutation. $C$ is symmetric about the diagonal; "
            "$\\gamma^{(x_4)}$ and the imaginary part of $B$ are antisymmetric "
            "(mirror entries have opposite colours), as they must be: a Hermitian "
            "matrix whose entries are purely imaginary is $i$ times a real "
            "antisymmetric matrix.")
```

Out [4]:

```
PASS gamma^(x4) and Im B are antisymmetric, C is symmetric, all signed permutations
```

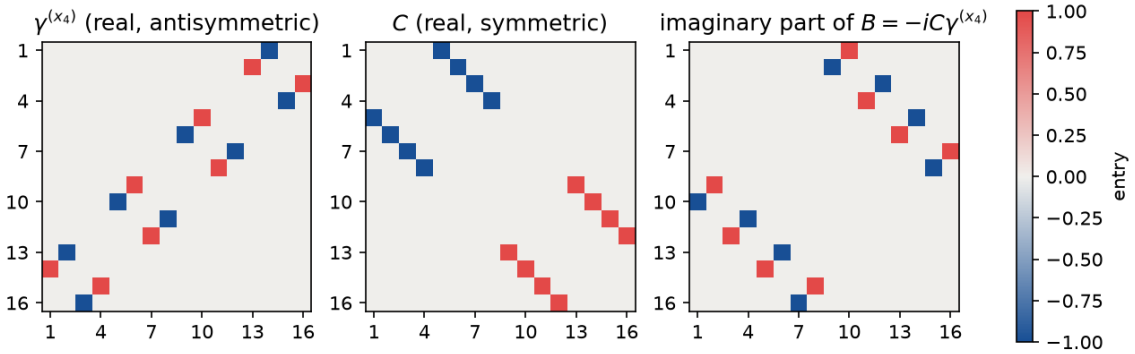

Figure 267: Heat maps of three 16 by 16 matrices: the time gamma $\gamma^{(x_4)}$ (left), the matrix $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ (middle) and the imaginary part of $B = -iC\gamma^{(x_4)}$ (right; its real part is zero). Horizontal axis: column number, vertical axis: row number; red is $+1$, blue is -1, light grey is 0. Each row and each column has exactly one nonzero entry: all three are signed permutation matrices, so B is i times a real signed permutation. C is symmetric about the diagonal; $\gamma^{(x_4)}$ and the imaginary part of B are antisymmetric (mirror entries have opposite colours), as they must be: a Hermitian matrix whose entries are purely imaginary is i times a real antisymmetric matrix. (Notebook 10a, figure 1.)

Out [4]:

```
Figure 10a.1 saved as Revision/textbook/figures/10a_1_gamma4_c_and_b.png
```

Text cell:

```
## 6. The mode Hamiltonian and its square

The next cell defines two functions. ‘mode_hamiltonian(m, k)‘ returns the numpy
matrix $h = -im\gamma^{(x_4)} - \gamma^{(x_4)}\sum_a k_a\gamma^{(x_a)}$, where ‘k‘ is a
dictionary that gives the momentum $k_a$ for some of the slice directions
$a = 1, 2, 3, 5, 6, 7, 8$ (missing directions have momentum 0).
‘frequency_squared(m, k)‘ returns $w^2 = m^2 + \sum_a \eta_{aa}k_a^2$: the
space-like momenta enter with $+$, the extra-time momenta with $-$.
```

In [5]:

```
def mode_hamiltonian(m, k):
    """h = -i m gamma^(x4) - gamma^(x4) sum_a k_a gamma^(x_a), a numpy matrix;
    k is a dictionary {a: k_a} for some of the directions a = 1, 2, 3, 5, 6, 7, 8."""
    h = -1j * m * gamma[4]
    for a, k_a in k.items():
        h = h - k_a * (gamma[4] @ gamma[a])
    return h


def frequency_squared(m, k):
    """w^2 = m^2 + k0^2 + k1^2 + k2^2 + k3^2 - k5^2 - k6^2 - k7^2."""
    return m ** 2 + sum(eta[a] * k_a ** 2 for a, k_a in k.items())


example = {0: 1.3, 1: 0.3, 2: -1.1, 3: 0.7, 5: 0.4, 6: 0.2, 7: -0.9}  # any numbers
h_example = mode_hamiltonian(1.7, example)
w2_example = frequency_squared(1.7, example)
report("w^2 for m = 1.7 and the example momenta", f"{w2_example:.6f}")
error = np.max(np.abs(h_example @ h_example - w2_example * np.eye(16)))
report("largest entry of h^2 - w^2 I (rounding only)", f"{error:.1e}")
check(error < 1e-12, "numerical example: h^2 = w^2 I at one generic point")
```

Out [5]:

```
RESULT w^2 for m = 1.7 and the example momenta = 5.360000
RESULT largest entry of h^2 - w^2 I (rounding only) = 8.9e-16
PASS numerical example: h^2 = w^2 I at one generic point
```

Text cell:

```
A numerical example is not a proof. The next cell repeats the computation EXACTLY
with sympy, for symbols $m, k_1, k_2, k_3, k_5, k_6, k_7, k_0$ that stand for any
real numbers. It checks two identities for all values at once:

- $h^2 = w^2 I_{16}$ (so every eigenvalue of $h$ is $+w$ or $-w$);
- $B h = h^\dagger B$ (so the Krein form $u^\dagger B u$ of every wave is conserved,
  as Section 4 showed).

Why $h^2 = w^2 I$ holds: $(-im\gamma^{(x_4)})^2 = -m^2(\gamma^{(x_4)})^2 = m^2$;
$(\gamma^{(x_4)}\gamma^{(x_a)})^2 = -(\gamma^{(x_4)})^2(\gamma^{(x_a)})^2 = \eta_{aa}$
```

```
for $a \neq 4$ (move the second $\gamma^{(x_4)}$ to the left past $\gamma^{(x_a)}$,
which costs a sign); and all mixed terms cancel in pairs because the matrices
$\gamma^{(x_4)}$ and $\gamma^{(x_4)}\gamma^{(x_a)}$ anticommute with each other.
```

In [6]:

```
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
C_exact = g[0] * g[1] * g[2] * g[3]
B_exact = -sp.I * C_exact * g[4]
m = sp.Symbol("m", real=True)  # the mass: any real number
k_sym = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}


def mode_hamiltonian_exact(mass, k):
    """The same h as mode_hamiltonian, as an exact sympy matrix."""
    h = -sp.I * mass * g[4]
    for a, k_a in k.items():
        h = h - k_a * g[4] * g[a]
    return h


zero = sp.zeros(16, 16)
h_sym = mode_hamiltonian_exact(m, k_sym)
w2_sym = m ** 2 + sum(eta[a] * k_sym[a] ** 2 for a in k_sym)
say(f"w^2 = {w2_sym}")
square_ok = (h_sym * h_sym - w2_sym * sp.eye(16)).applyfunc(sp.expand) == zero
check_record(square_ok, "exact: h^2 = w^2 I for all real m and k",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_one_particle_flat_dispersion")
krein_ok = (B_exact * h_sym - h_sym.H * B_exact).applyfunc(sp.expand) == zero
check_record(krein_ok,
             "exact: B h = h^dagger B for all real m and k (Krein self-adjoint)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "mode_hamiltonian_B_selfadjoint_dispersion")
```

Out [6]:

```
w^2 = k0**2 + k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + m**2
PASS exact: h^2 = w^2 I for all real m and k
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_one_particle_flat_dispersion
PASS exact: B h = h^dagger B for all real m and k (Krein self-adjoint)
     reproduces Revision/theory/reports/python-field-theory.json, check
    mode_hamiltonian_B_selfadjoint_dispersion
```

Text cell:

```
When is $h$ Hermitian, and when does it commute with $B$? The next cell computes
the exact matrices $h - h^\dagger$ and $Bh - hB$ and lists the symbols that appear
in them. Only $k_5, k_6, k_7$ appear: both matrices vanish exactly when there is no
momentum along the extra times, the *good sector*. (The reason: $\gamma^{(x_4)}
```

```
\gamma^{(x_a)}$ is Hermitian for the space-like directions and anti-Hermitian for
the extra times, and it commutes with $B$ for $a = 1, 2, 3, 8$ and anticommutes
with $B$ for $a = 5, 6, 7$.)
```

In [7]:

```
not_hermitian_part = (h_sym - h_sym.H).applyfunc(sp.expand)
commutator = (B_exact * h_sym - h_sym * B_exact).applyfunc(sp.expand)
symbols_1 = sorted(str(s) for s in not_hermitian_part.free_symbols)
symbols_2 = sorted(str(s) for s in commutator.free_symbols)
say(f"symbols in h - h^dagger: {symbols_1}")
say(f"symbols in B h - h B:    {symbols_2}")
good_sector = {k_sym[5]: 0, k_sym[6]: 0, k_sym[7]: 0}  # no extra-time momentum
check_record(symbols_1 == ["k5", "k6", "k7"]
             and not_hermitian_part.subs(good_sector) == zero,
             "exact: h is Hermitian exactly in the good sector k5 = k6 = k7 = 0",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "mode_hamiltonian_good_sector")
check_record(symbols_2 == ["k5", "k6", "k7"]
             and commutator.subs(good_sector) == zero,
             "exact: B h = h B exactly in the good sector",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")
```

Out [7]:

```
symbols in h - h^dagger: ['k5', 'k6', 'k7']
symbols in B h - h B:    ['k5', 'k6', 'k7']
PASS exact: h is Hermitian exactly in the good sector k5 = k6 = k7 = 0
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    mode_hamiltonian_good_sector
PASS exact: B h = h B exactly in the good sector
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_spectrum_and_B_sectors
```

Text cell:

```
## 7. The three cases: real, zero and imaginary frequency

Since $h^2 = w^2 I_{16}$, everything depends on the sign of
$w^2 = m^2 + k_s^2 - k_t^2$, where $k_s^2 = k_0^2 + k_1^2 + k_2^2 + k_3^2$ collects the
space-like momenta and $k_t^2 = k_5^2 + k_6^2 + k_7^2$ the extra-time momenta:

- $w^2 > 0$: the frequencies $\pm w$ are real and the wave oscillates;
- $w^2 = 0$: $h^2 = 0$ and the wave grows linearly in $x_4$;
- $w^2 < 0$: the frequencies are $\pm i\kappa$ with $\kappa = \sqrt{-w^2}$, and one
  half of the waves grows like $e^{\kappa x_4}$.

The next cell draws $w^2$ against $k_5$ (the only extra-time momentum here) for
$m = 1$ and three values of $k_s$, and checks that the zero crossing lies at
$k_5 = \sqrt{m^2 + k_s^2}$.
```

In [8]:

```
k5_values = np.linspace(0.0, 3.0, 301)  # 301 values of k5 from 0 to 3
fig, ax = plt.subplots()
for k_s, colour in ((0.0, BLUE), (1.0, ORANGE), (2.0, GREEN)):
    w2_curve = [frequency_squared(1.0, {1: k_s, 5: k5}) for k5 in k5_values]
    ax.plot(k5_values, w2_curve, color=colour, linewidth=2,
            label=f"$k_s = {k_s:.0f}$")
    crossing = np.sqrt(1.0 + k_s ** 2)  # where w^2 = 0
    ax.plot([crossing], [0.0], "o", color=colour, markersize=8)
ax.axhline(0.0, color=GREY, linewidth=1)
ax.text(0.1, 3.0, "$w^2 > 0$: real frequency, the wave oscillates", color=GREY)
ax.text(0.1, -6.0, "$w^2 < 0$: imaginary frequency, the wave grows", color=GREY)
ax.set_xlabel("momentum $k_5$ along the extra time $x_5$")
ax.set_ylabel("$w^2 = m^2 + k_s^2 - k_5^2$")
ax.set_title("The squared frequency of a plane wave, $m = 1$")
ax.legend(title="space-like momentum")
save_figure(fig, "frequency_squared",
            "The squared frequency $w^2 = m^2 + k_s^2 - k_5^2$ of a plane wave with "
            "mass $m = 1$, plotted against the momentum $k_5$ along the extra time "
            "$x_5$ (horizontal axis, units of $m$) for the space-like momentum "
            "$k_s = 0$, 1 and 2 (the three curves). Vertical axis: $w^2$ in units "
            "of $m^2$. Above the grey line the frequency is real and the wave "
            "oscillates; below it the frequency is imaginary and the wave grows. "
            "The dots mark $k_5 = \\sqrt{m^2 + k_s^2}$, where the wave changes "
            "from oscillation to growth: a large enough extra-time momentum always "
            "wins.")
check(all(abs(frequency_squared(1.0, {1: k_s, 5: np.sqrt(1.0 + k_s ** 2)})) < 1e-12
          for k_s in (0.0, 1.0, 2.0)),
      "w^2 = 0 exactly at k5 = sqrt(m^2 + k_s^2)")
```

Out [8]:

```
Figure 10a.2 saved as Revision/textbook/figures/10a_2_frequency_squared.png
PASS w^2 = 0 exactly at k5 = sqrt(m^2 + k_s^2)
```

Text cell:

```
## 8. The eigenvalues as the extra-time momentum grows

The next cell computes the 16 eigenvalues of $h$ numerically (numpy's
`np.linalg.eigvals`) for $m = 1$, no space-like momentum, and $k_5$ from 0 to 3, and
plots their real and imaginary parts. The formula predicts eight eigenvalues
$+\sqrt{1 - k_5^2}$ and eight $-\sqrt{1 - k_5^2}$ for $k_5 < 1$, and eight $+i\kappa$
and eight $-i\kappa$ with $\kappa = \sqrt{k_5^2 - 1}$ for $k_5 > 1$. At $k_5 = 1$
exactly the matrix cannot be diagonalised and the numerical eigenvalues are only
accurate to about $10^{-8}$, so the check leaves out the points with
$|w^2| < 0.05$.
```

In [9]:

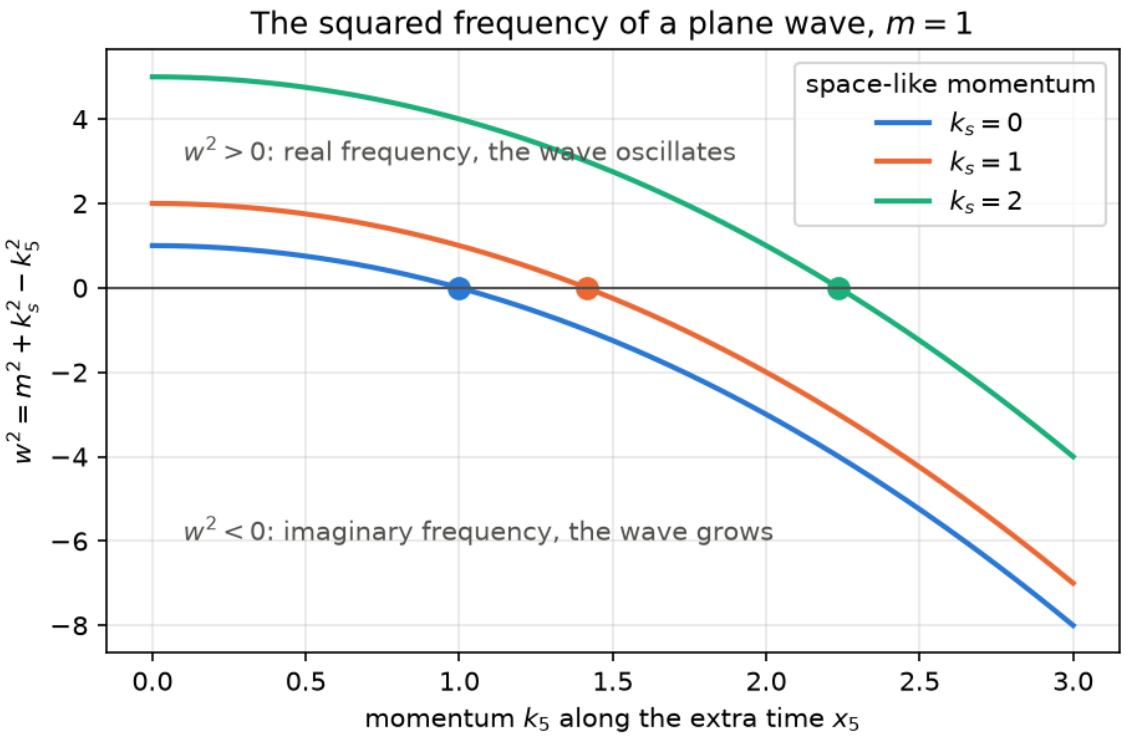

Figure 268: The squared frequency $w^2 = m^2 + k_s^2 - k_5^2$ of a plane wave with mass $m = 1$, plotted against the momentum k_5 along the extra time x_5 (horizontal axis, units of m) for the space-like momentum $k_s = 0$, 1 and 2 (the three curves). Vertical axis: w^2 in units of m^2. Above the grey line the frequency is real and the wave oscillates; below it the frequency is imaginary and the wave grows. The dots mark $k_5 = \sqrt{m^2 + k_s^2}$, where the wave changes from oscillation to growth: a large enough extra-time momentum always wins. (Notebook 10a, figure 2.)

```
real_parts, imaginary_parts, worst = [], [], 0.0
for k5 in k5_values:
    eigenvalues = np.linalg.eigvals(mode_hamiltonian(1.0, {5: k5}))
    real_parts.append(np.sort(eigenvalues.real))
    imaginary_parts.append(np.sort(eigenvalues.imag))
    w2 = 1.0 - k5 ** 2
    if abs(w2) > 0.05:  # away from the point where h cannot be diagonalised
        w = np.sqrt(complex(w2))  # sqrt of a negative number is imaginary
        predicted = np.array([w] * 8 + [-w] * 8)
        # compare the sorted real parts and the sorted imaginary parts separately
        worst = max(worst,
                    np.max(np.abs(np.sort(eigenvalues.real) - np.sort(predicted.real))),
                    np.max(np.abs(np.sort(eigenvalues.imag) - np.sort(predicted.imag))))
report("largest difference from the formula +-w (8 times each)", f"{worst:.1e}")
check(worst < 1e-10, "the 16 eigenvalues of h are +w and -w, eight times each")

fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0), sharex=True)
axes[0].plot(k5_values, np.array(real_parts), color=BLUE, linewidth=2)
axes[1].plot(k5_values, np.array(imaginary_parts), color=ORANGE, linewidth=2)
for ax, what in ((axes[0], "real part"), (axes[1], "imaginary part")):
    ax.axvline(1.0, color=GREY, linestyle=":", linewidth=1)
    ax.set_xlabel("momentum $k_5$ along the extra time $x_5$ ($m = 1$)")
    ax.set_ylabel(f"{what} of the eigenvalues of $h$")
axes[0].set_title("oscillation: real frequencies $\\pm w$")
axes[1].set_title("growth: imaginary frequencies $\\pm i\\kappa$")
save_figure(fig, "eigenvalue_flow",
            "The 16 eigenvalues of the mode Hamiltonian $h$ for mass $m = 1$ and "
            "only the extra-time momentum $k_5$ (horizontal axis, units of $m$): "
            "their real parts (left, blue) and imaginary parts (right, orange), "
            "vertical axes in units of $m$; each curve carries eight eigenvalues. "
            "For $k_5 < 1$ the eigenvalues are the real numbers $\\pm\\sqrt{1 - "
            "k_5^2}$, which move together and meet at 0 at $k_5 = 1$ (dotted line); "
            "for $k_5 > 1$ they leave the real axis as $\\pm i\\sqrt{k_5^2 - 1}$, "
            "and the waves with $+i\\kappa$ grow like $e^{\\kappa x_4}$.")
```

Out [9]:

```
RESULT largest difference from the formula +-w (8 times each) = 3.6e-15
PASS the 16 eigenvalues of h are +w and -w, eight times each
```

Out [9]:

```
Figure 10a.3 saved as Revision/textbook/figures/10a_3_eigenvalue_flow.png
```

Text cell:

```
## 9. Krein inertia of the eigenspaces: the recorded samples

The next cell defines three functions.

- ‘orthonormal_basis(P)‘ returns an orthonormal basis of the range of a matrix $P$
```

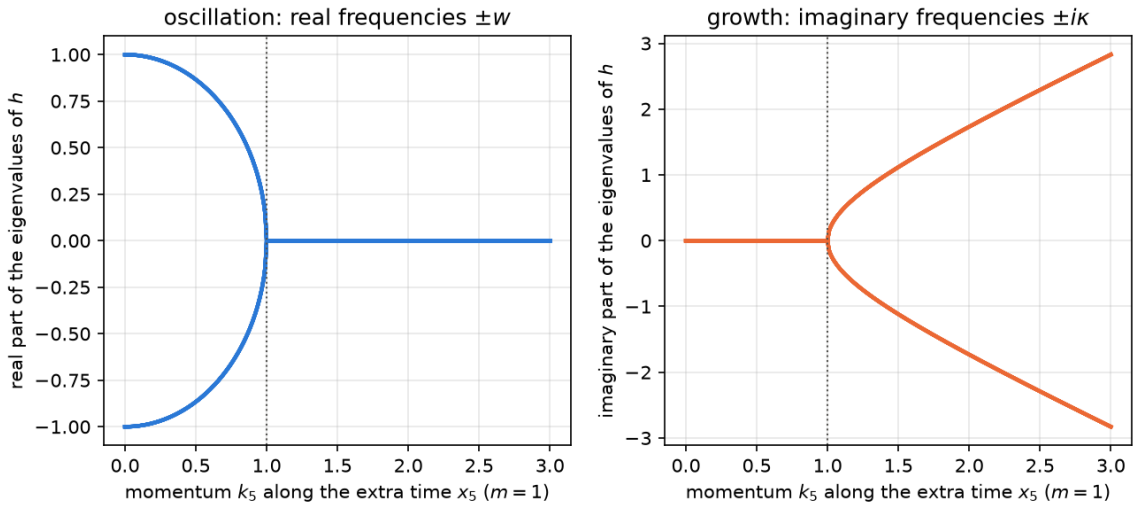

Figure 269: The 16 eigenvalues of the mode Hamiltonian h for mass $m = 1$ and only the extra-time momentum k_5 (horizontal axis, units of m): their real parts (left, blue) and imaginary parts (right, orange), vertical axes in units of m; each curve carries eight eigenvalues. For $k_5 < 1$ the eigenvalues are the real numbers $\pm\sqrt{1 - k_5^2}$, which move together and meet at 0 at $k_5 = 1$ (dotted line); for $k_5 > 1$ they leave the real axis as $\pm i\sqrt{k_5^2 - 1}$, and the waves with $+i\kappa$ grow like $e^{\kappa x_4}$. (Notebook 10a, figure 3.)

```
  by the Gram-Schmidt method: take the columns of $P$ in order, remove from each
  its parts along the basis vectors found so far, and keep it (divided by its
  length) if something is left. The procedure is repeated once to remove rounding
  errors.
- ‘eigenspace(m, k, sign)‘ returns such a basis of the eigenspace of $+w$
  (‘sign = +1‘) or $-w$ (‘sign = -1‘), as the range of the projector
  $P_\pm = \frac12(I \pm h/w)$ of Section 4. For $w^2 < 0$ the number $w$ is
  $i\kappa$, and the same formula gives the eigenspaces of $\pm i\kappa$.
- ‘krein_inertia(V)‘ computes the Gram matrix $G = V^\dagger B V$ (entry $(i, j)$ is
  the Krein form of the basis vectors $i$ and $j$) and counts its positive,
  negative and zero eigenvalues (zero means smaller than $10^{-8}$). It also returns
  the smallest size of a nonzero eigenvalue, to show that no eigenvalue is close
  to the borderline $10^{-8}$.
```

In [10]:

```
def orthonormal_basis(P):
    """Orthonormal columns that span the range of P (Gram-Schmidt, done twice)."""
    basis = []
    for column in P.T:  # the columns of P, in order
        v = column.astype(complex)
        for _ in range(2):  # the second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e  # remove the part of v along e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:  # v is not a combination of the earlier columns
```

```
            basis.append(v / length)
    return np.array(basis).T  # the basis vectors as the columns of a matrix


def eigenspace(m, k, sign):
    """Orthonormal basis of the eigenspace of h for the eigenvalue sign * w."""
    h = mode_hamiltonian(m, k)
    w = np.sqrt(complex(frequency_squared(m, k)))  # w, or i kappa when w^2 < 0
    P = (np.eye(16) + sign * h / w) / 2  # the projector P_+ or P_-
    return orthonormal_basis(P)


def krein_inertia(V):
    """(positive, negative, zero) eigenvalues of V^dagger B V, and the smallest
    size of a nonzero one (None when all are zero)."""
    G = V.conj().T @ B @ V  # the Krein form on the span of the columns of V
    values = np.linalg.eigvalsh(G)
    nonzero = np.abs(values[np.abs(values) > 1e-8])
    counts = (int(np.sum(values > 1e-8)), int(np.sum(values < -1e-8)),
              int(np.sum(np.abs(values) <= 1e-8)))
    return counts, (float(nonzero.min()) if nonzero.size else None)
```

Text cell:

```
The Revision pairing record ‘Revision/pairing/pairing-theory.json‘ stores eight
samples (entry ‘data‘, ‘one_particle_flat‘, ‘samples‘): for each a mass $m$, a
momentum list $(k_0, \dots, k_7)$ (the entry $k_4$ is not used), the frequency $w$
(written in the notation of the Wolfram Language, for example ‘Sqrt[2]‘), the
dimensions of the two eigenspaces and their Krein inertia. The next cell reads them,
recomputes every number, prints one line per sample and checks that everything
agrees.
```

In [11]:

```
pairing = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
samples = pairing["data"]["one_particle_flat"]["samples"]
all_agree = True
say("   m  nonzero momenta         w      dims   inertia(+w)  inertia(-w)")
for sample in samples:
    m_value, k_list = sample["m"], sample["k"]
    k = {a: k_list[a] for a in (0, 1, 2, 3, 5, 6, 7) if k_list[a] != 0}
    # "Sqrt[2]" (Wolfram notation) -> the number sqrt(2)
    w_recorded = float(sp.sympify(sample["w"].replace("Sqrt[", "sqrt(")
                                  .replace("]", ")")))
    w = np.sqrt(frequency_squared(m_value, k))
    found = []
    for sign in (+1, -1):
        V = eigenspace(m_value, k, sign)
        counts, _ = krein_inertia(V)
        found.append((V.shape[1], counts))
    agree = (abs(w - w_recorded) < 1e-12
```

```
            and found[0][0] == sample["dim_plus_w"]
            and found[1][0] == sample["dim_minus_w"]
            and list(found[0][1][:2]) == sample["B_inertia_plus_w"]
            and list(found[1][1][:2]) == sample["B_inertia_minus_w"]
            and found[0][1][2] == 0 and found[1][1][2] == 0)
    all_agree = all_agree and agree
    momenta = ", ".join(f"k{a}={v}" for a, v in k.items()) or "none"
    say(f"{m_value:4d}  {momenta:22s} {w:7.4f}   {found[0][0]},{found[1][0]}    "
        f"{found[0][1][:2]}       {found[1][1][:2]}")
check_record(all_agree,
            "all 8 recorded samples: w, dimensions 8 and 8, Krein inertia (4, 4)",
            record="Revision/pairing/pairing-theory.json, data one_particle_flat")
```

Out [11]:

```
   m  nonzero momenta         w      dims   inertia(+w)  inertia(-w)
   2  k0=4, k1=1, k2=2        5.0000   8,8    (4, 4)       (4, 4)
  -2  k0=4, k1=1, k2=2        5.0000   8,8    (4, 4)       (4, 4)
   3  none                    3.0000   8,8    (4, 4)       (4, 4)
  -3  none                    3.0000   8,8    (4, 4)       (4, 4)
   1  k1=1                    1.4142   8,8    (4, 4)       (4, 4)
  -1  k1=1                    1.4142   8,8    (4, 4)       (4, 4)
   2  k5=1                    1.7321   8,8    (4, 4)       (4, 4)
  -2  k5=1                    1.7321   8,8    (4, 4)       (4, 4)
PASS all 8 recorded samples: w, dimensions 8 and 8, Krein inertia (4, 4)
     reproduces Revision/pairing/pairing-theory.json, data one_particle_flat
```

Text cell:

```
Two more recorded statements. The sympy pairing verifier uses the sample $m = 1$,
$k_1 = 2$, $k_0 = 2$ (so $w = \sqrt{1 + 4 + 4} = 3$). Its imaginary-frequency samples
are $m = 1$ with $k_5 = 2$ ($w^2 = -3$) and with $k_1 = 1$, $k_5 = 2$ ($w^2 = -2$):
there both eigenspaces must be *neutral*, inertia $(0, 0, 8)$. For $m = 1$, $k_5 = 1$
the frequency is zero: $h^2 = 0$, the range of $h$ has dimension 8, and since
$(hx)^\dagger B (hy) = x^\dagger h^\dagger B h y = x^\dagger B h^2 y = 0$, the range of
$h$ is neutral as well. The next cell checks all of this.
```

In [12]:

```
sample_3 = {0: 2, 1: 2}
inertias_3 = [krein_inertia(eigenspace(1, sample_3, s))[0] for s in (+1, -1)]
report("w for m = 1, k0 = 2, k1 = 2", f"{np.sqrt(frequency_squared(1, sample_3)):.6f}")
say(f"inertia of the +w and -w eigenspaces: {inertias_3}")
check_record(inertias_3 == [(4, 4, 0), (4, 4, 0)],
            "m = 1, k0 = 2, k1 = 2 (w = 3): inertia (4, 4) on both eigenspaces",
            record="Revision/pairing/reports/python-pairing.json, check "
                   "Q.one_particle_Krein_inertia")
neutral_ok = True
for k in ({5: 2}, {1: 1, 5: 2}):
    spaces = [eigenspace(1, k, s) for s in (+1, -1)]
    inertias = [krein_inertia(V)[0] for V in spaces]
```

```
    say(f"m = 1, momenta {k}: w^2 = {frequency_squared(1, k)}, dimensions "
        f"{[V.shape[1] for V in spaces]}, inertia {inertias}")
    neutral_ok = neutral_ok and inertias == [(0, 0, 8), (0, 0, 8)]
h_zero = mode_hamiltonian(1, {5: 1})  # w^2 = 1 - 1 = 0
rank_h = int(np.linalg.matrix_rank(h_zero))
nilpotent = np.max(np.abs(h_zero @ h_zero)) < 1e-12  # h^2 = 0
range_neutral = np.max(np.abs(h_zero.conj().T @ B @ h_zero)) < 1e-12
say(f"m = 1, k5 = 1: w^2 = 0, h^2 = 0: {nilpotent}, rank of h = {rank_h}, "
    f"range of h neutral: {range_neutral}")
check_record(neutral_ok and nilpotent and rank_h == 8 and range_neutral,
             "imaginary and zero frequencies: the eigenspaces are Krein-neutral",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_complex_frequency_Krein_neutral")
```

Out [12]:

```
RESULT w for m = 1, k0 = 2, k1 = 2 = 3.000000
inertia of the +w and -w eigenspaces: [(4, 4, 0), (4, 4, 0)]
PASS m = 1, k0 = 2, k1 = 2 (w = 3): inertia (4, 4) on both eigenspaces
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_Krein_inertia
m = 1, momenta {5: 2}: w^2 = -3, dimensions [8, 8], inertia [(0, 0, 8), (0, 0, 8)]
m = 1, momenta {1: 1, 5: 2}: w^2 = -2, dimensions [8, 8], inertia [(0, 0, 8), (0, 0, 8)]
m = 1, k5 = 1: w^2 = 0, h^2 = 0: True, rank of h = 8, range of h neutral: True
PASS imaginary and zero frequencies: the eigenspaces are Krein-neutral
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_complex_frequency_Krein_neutral
```

Text cell:

```
## 10. Why the inertia is (4, 4) at EVERY real frequency

Samples are not a proof. The Revision record proves the statement for every real
frequency in three steps, which the next cells repeat.

**Step (i).** For real $w > 0$: $P_+^\dagger B P_- = 0$ and $P_+^\dagger B P_+ =
BP_+$. Line by line, with $P_\pm = \frac12(I \pm h/w)$ and $P_+^\dagger = \frac12(I +
h^\dagger/w)$ ($w$ is real):

$$4w^2\,P_+^\dagger B P_- = (wI + h^\dagger)B(wI - h) = w^2 B - wBh + wh^\dagger B -
h^\dagger B h .$$

Use $h^\dagger B = Bh$ twice: the two middle terms cancel, and $h^\dagger B h = B h h
= w^2 B$, so the right-hand side is $w^2B - w^2B = 0$. Hence the two eigenspaces
are Krein-orthogonal: a wave of frequency $+w$ and one of $-w$ have zero mixed
Krein form. Since $B$ is invertible and the two eigenspaces together span all 16
directions, $B$ cannot vanish on either of them (it is *nondegenerate* on each).

**Step (ii).** In the good sector $Bh = hB$, so $\mathrm{tr}(BP_\pm) = \frac12
\mathrm{tr}B \pm \frac{1}{2w}\mathrm{tr}(Bh)$, and both traces are 0. The trace of
$BP_+$ is the sum of the eigenvalues of $B$ on the $+w$ eigenspace (in the good
sector $B$ maps this eigenspace into itself), each $+1$ or $-1$; a sum of eight
```

```
such numbers is 0 only with four of each: inertia $(4, 4)$.

**Step (iii).** Turning on the extra-time momenta slowly at fixed $m, k_1, k_2, k_3,
k_0$ keeps $w^2$ positive as long as $k_t^2 < m^2 + k_s^2$, the projectors change
continuously, and the Krein form stays nondegenerate on their ranges by step (i).
An eigenvalue of the Gram matrix could change sign only by passing through 0, which
nondegeneracy forbids: the inertia stays $(4, 4)$ at every real frequency.

The next cell checks the identities of steps (i) and (ii) exactly with sympy. The
letter $w$ is a positive symbol, and after expanding, every $w^2$ is replaced by
$m^2 + k_s^2 - k_t^2$.
```

In [13]:

```
w = sp.Symbol("w", positive=True)
I16 = sp.eye(16)


def replace_w_squared(matrix):
    """Expand every entry and replace w^2 by m^2 + k_s^2 - k_t^2."""
    return matrix.applyfunc(lambda e: sp.expand(sp.expand(e).subs(w ** 2, w2_sym)))


step_i_a = replace_w_squared((w * I16 + h_sym.H) * B_exact * (w * I16 - h_sym))
step_i_b = replace_w_squared((w * I16 + h_sym.H) * B_exact * (w * I16 + h_sym)
                             - 2 * w * B_exact * (w * I16 + h_sym))
check_record(step_i_a == zero and step_i_b == zero,
             "exact step (i): 4 w^2 P+^dagger B P- = 0 and P+^dagger B P+ = B P+",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_Krein_inertia_proof")
trace_B_h = sp.expand((B_exact * h_sym).trace())
say(f"exact: tr B = {B_exact.trace()}, tr(B h) = {trace_B_h}")
check(B_exact.trace() == 0 and trace_B_h == 0,
      "exact step (ii): tr B = tr(B h) = 0, so tr(B P+) = tr(B P-) = 0")
```

Out [13]:

```
PASS exact step (i): 4 w^2 P+^dagger B P- = 0 and P+^dagger B P+ = B P+
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_Krein_inertia_proof
exact: tr B = 0, tr(B h) = 0
PASS exact step (ii): tr B = tr(B h) = 0, so tr(B P+) = tr(B P-) = 0
```

Text cell:

```
Step (iii) is a statement about continuity. The next cell shows it at work: for
$m = 1$, the space-like momentum $k_1 = 0.5$ and the extra-time momentum $k_5$ from 0
to 2.5 it computes the Krein inertia of the $+w$ eigenspace. The threshold is
$k_5 = \sqrt{1.25} \approx 1.118$; points closer than 0.02 to it are skipped (there
the eigenspaces merge). Below the threshold the inertia must be $(4, 4, 0)$ at every
point, above it $(0, 0, 8)$. The cell also records the smallest size of a nonzero
```

eigenvalue of the Gram matrix: it shrinks towards the threshold but never reaches 0
before it.

In [14]:

```
threshold = np.sqrt(1.25)
scan = [k5 for k5 in np.linspace(0.0, 2.5, 251) if abs(k5 - threshold) > 0.02]
counts_scan, margins = [], []
for k5 in scan:
    counts, margin = krein_inertia(eigenspace(1.0, {1: 0.5, 5: k5}, +1))
    counts_scan.append(counts)
    if margin is not None:
        margins.append(margin)
below = [c for k5, c in zip(scan, counts_scan) if k5 < threshold]
above = [c for k5, c in zip(scan, counts_scan) if k5 > threshold]
report("points below and above the threshold", f"{len(below)} and {len(above)}")
report("smallest nonzero Gram eigenvalue below the threshold", f"{min(margins):.4f}")
check(set(below) == {(4, 4, 0)} and set(above) == {(0, 0, 8)},
      "scan: inertia (4, 4) at every real frequency, (0, 0, 8) at every imaginary "
      "one")

counts_array = np.array(counts_scan)  # one row (positive, negative, zero) per k5
fig, ax = plt.subplots()
width = 0.0105  # a little wider than the step 0.01 between the k5 values (no gaps)
# Stacked bars: the blue part (positive directions) starts at 0, the orange part
# (negative directions) on top of it, the green part (neutral) on top of both.
ax.bar(scan, counts_array[:, 0], width=width, color=BLUE,
       label="positive charge $u^\\dagger B u > 0$")
ax.bar(scan, counts_array[:, 1], width=width, bottom=counts_array[:, 0],
       color=ORANGE, label="negative charge $u^\\dagger B u < 0$")
ax.bar(scan, counts_array[:, 2], width=width,
       bottom=counts_array[:, 0] + counts_array[:, 1], color=GREEN,
       label="neutral (the form vanishes)")
ax.axvline(threshold, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("momentum $k_5$ along the extra time ($m = 1$, $k_1 = 0.5$)")
ax.set_ylabel("directions in the $+w$ eigenspace")
ax.set_yticks(range(0, 9))
ax.set_ylim(0.0, 10.5)  # room for the legend above the bars
ax.set_title("Krein inertia of one eigenspace across the threshold")
ax.legend(loc="upper center", ncol=3, fontsize=8)
save_figure(fig, "krein_inertia_scan",
            "The Krein inertia of the eigenspace of $+w$ of the mode Hamiltonian "
            "for $m = 1$, $k_1 = 0.5$ and the extra-time momentum $k_5$ from 0 to "
            "2.5 (horizontal axis, units of $m$). Each thin bar splits the eight "
            "independent directions of the eigenspace (vertical axis) into those "
            "of positive charge (blue), negative charge (orange) and zero charge "
            "for every pair (green). Left of the threshold $k_5 = \\sqrt{1.25}$ "
            "(dotted line) the frequency is real and the split is always 4 and 4; "
            "right of it the frequency is imaginary and the whole eigenspace is "
            "neutral: the growing waves carry no charge.")
```

Out [14]:

```
RESULT points below and above the threshold = 110 and 137
RESULT smallest nonzero Gram eigenvalue below the threshold = 0.2225
PASS scan: inertia (4, 4) at every real frequency, (0, 0, 8) at every imaginary one
```

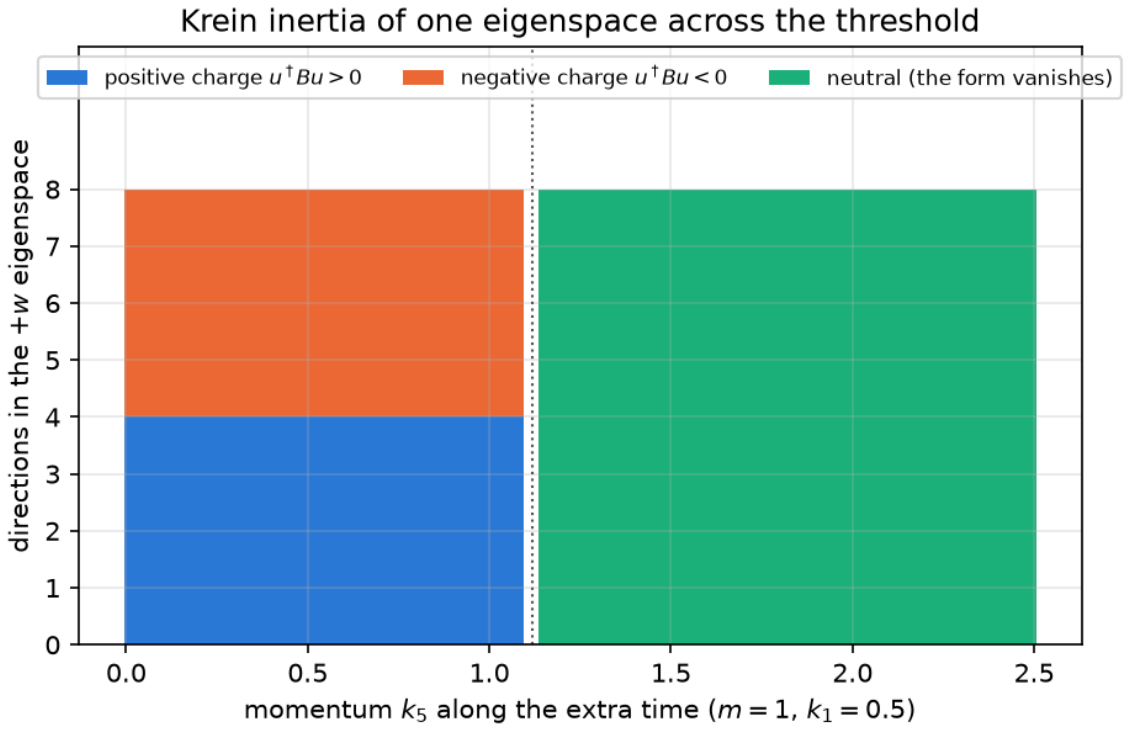

Figure 270: The Krein inertia of the eigenspace of $+w$ of the mode Hamiltonian for $m = 1$, $k_1 = 0.5$ and the extra-time momentum k_5 from 0 to 2.5 (horizontal axis, units of m). Each thin bar splits the eight independent directions of the eigenspace (vertical axis) into those of positive charge (blue), negative charge (orange) and zero charge for every pair (green). Left of the threshold $k_5 = \sqrt{1.25}$ (dotted line) the frequency is real and the split is always 4 and 4; right of it the frequency is imaginary and the whole eigenspace is neutral: the growing waves carry no charge. (Notebook 10a, figure 4.)

Out [14]:

```
Figure 10a.4 saved as Revision/textbook/figures/10a_4_krein_inertia_scan.png
```

Text cell:

```
## 11. Three pictures of the Krein form on the eigenvectors

The next cell puts the 8 basis vectors of the $+w$ eigenspace and the 8 of the $-w$
eigenspace side by side as the 16 columns of a matrix $W$, and draws the sizes of the
entries of the Gram matrix $W^\dagger B W$ for three waves:

- the good-sector sample $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ along $(x_0,
x_1, x_2, x_3)$,
```

```
  $w = 5$;
- $m = 2$ with the extra-time momentum $k_5 = 1$: still a real frequency,
  $w = \sqrt3$;
- $m = 1$, $k_5 = 2$: an imaginary frequency, $w = i\sqrt3$.

In the first two pictures only the two diagonal $8 \times 8$ blocks are nonzero
(the eigenspaces are Krein-orthogonal, step (i)); in the third the diagonal blocks
vanish (each eigenspace is neutral) and only the off-diagonal blocks remain: a
growing wave has nonzero Krein form only with a decaying one.
```

In [15]:

```
cases = [(2, {0: 4, 1: 1, 2: 2}, "$m = 2$, $k = (4, 1, 2, 0)$: $w = 5$"),
         (2, {5: 1}, "$m = 2$, $k_5 = 1$: $w = \\sqrt{3}$"),
         (1, {5: 2}, "$m = 1$, $k_5 = 2$: $w = i\\sqrt{3}$")]
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2))
block_sizes = []
for ax, (m_value, k, title) in zip(axes, cases):
    W = np.hstack([eigenspace(m_value, k, +1), eigenspace(m_value, k, -1)])
    gram = np.abs(W.conj().T @ B @ W)  # sizes of the Krein form between columns
    image = draw_matrix(ax, gram, title, cmap=SEQUENTIAL, vmin=0.0, vmax=1.0)
    ax.axhline(7.5, color=GREY, linewidth=1)  # the border between the two
    ax.axvline(7.5, color=GREY, linewidth=1)  # eigenspaces (rows/columns 8 and 9)
    diagonal_blocks = max(gram[:8, :8].max(), gram[8:, 8:].max())
    off_blocks = max(gram[:8, 8:].max(), gram[8:, :8].max())
    block_sizes.append((diagonal_blocks, off_blocks))
    say(f"case {len(block_sizes)}: largest entry in the diagonal blocks "
        f"{diagonal_blocks:.3f}; off-diagonal blocks below 1e-12: {off_blocks < 1e-12}")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="size of the entry")
save_figure(fig, "krein_gram_matrices",
            "Sizes of the entries of the Gram matrix $W^\\dagger B W$, where the "
            "first 8 columns of $W$ span the eigenspace of $+w$ and the last 8 that "
            "of $-w$ (horizontal and vertical axes: column and row number; white is "
            "0, dark blue is 1; grey lines separate the two eigenspaces). Left: a "
            "good-sector wave with $w = 5$; middle: a wave with extra-time momentum "
            "and real $w = \\sqrt{3}$; right: a wave with imaginary $w = "
            "i\\sqrt{3}$. For real frequencies only the diagonal blocks are filled: "
            "the two eigenspaces are Krein-orthogonal. For the imaginary frequency "
            "the diagonal blocks are empty: each eigenspace is neutral.")
check(block_sizes[0][1] < 1e-12 and block_sizes[1][1] < 1e-12
      and block_sizes[2][0] < 1e-12 and block_sizes[2][1] > 0.1,
      "real w: the eigenspaces are Krein-orthogonal; imaginary w: each is neutral")
```

Out [15]:

```
case 1: largest entry in the diagonal blocks 0.894; off-diagonal blocks below 1e-12: True
case 2: largest entry in the diagonal blocks 0.866; off-diagonal blocks below 1e-12: True
case 3: largest entry in the diagonal blocks 0.000; off-diagonal blocks below 1e-12:
    False
```

Out [15]:

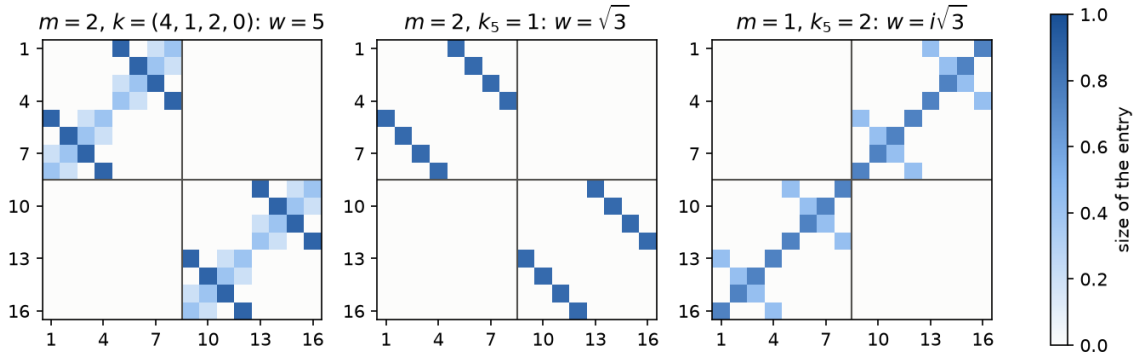

Figure 271: Sizes of the entries of the Gram matrix $W^\dagger B W$, where the first 8 columns of W span the eigenspace of $+w$ and the last 8 that of $-w$ (horizontal and vertical axes: column and row number; white is 0, dark blue is 1; grey lines separate the two eigenspaces). Left: a good-sector wave with $w = 5$; middle: a wave with extra-time momentum and real $w = \sqrt{3}$; right: a wave with imaginary $w = i\sqrt{3}$. For real frequencies only the diagonal blocks are filled: the two eigenspaces are Krein-orthogonal. For the imaginary frequency the diagonal blocks are empty: each eigenspace is neutral. (Notebook 10a, figure 5.)

```
Figure 10a.5 saved as Revision/textbook/figures/10a_5_krein_gram_matrices.png
PASS real w: the eigenspaces are Krein-orthogonal; imaginary w: each is neutral
```

Text cell:

```
## 12. The universes of masses $+m$ and $-m$

The pairing theorem T1 of the Revision record relates the field of mass $m$ to the
field of mass $-m$ through the chirality matrix $\Gamma$. At the one-particle level
the record proves (statement Q4 of the quantum reading):

- $\Gamma h_m(k)\Gamma = h_{-m}(k)$, because $\Gamma$ anticommutes with
  $\gamma^{(x_4)}$ and commutes with every product $\gamma^{(x_4)}\gamma^{(x_a)}$;
- $\gamma^{(x_0)} h_m(k)\gamma^{(x_0)} = h_{-m}(R_0 k)$, where $R_0$ reverses the
  hidden momentum $k_0$.

Since $\Gamma^2 = I$ and $(\gamma^{(x_0)})^2 = I$, these are *similarity*
transformations: $h_{-m}$ has exactly the eigenvalues of $h_m$. The one-particle
spectra of the two universes are IDENTICAL, not opposite. The next cell checks both
identities exactly.
```

In [16]:

```
Gamma_exact = g[0] * g[1] * g[2] * g[3] * g[4] * g[5] * g[6] * g[7]
h_minus = mode_hamiltonian_exact(-m, k_sym)  # the same momenta, mass -m
k_reflected = dict(k_sym)
k_reflected[0] = -k_sym[0]  # R_0: k0 -> -k0
h_minus_reflected = mode_hamiltonian_exact(-m, k_reflected)
chirality_ok = (Gamma_exact * h_sym * Gamma_exact - h_minus).applyfunc(
```

```
    sp.expand) == zero
mirror_ok = (g[0] * h_sym * g[0] - h_minus_reflected).applyfunc(sp.expand) == zero
check_record(Gamma_exact * Gamma_exact == I16 and chirality_ok,
             "exact: Gamma h_m(k) Gamma = h_-m(k)",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_maps")
check_record(mirror_ok, "exact: gamma^(x0) h_m(k) gamma^(x0) = h_-m(R_0 k)",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_one_particle_maps")
```

Out [16]:

```
PASS exact: Gamma h_m(k) Gamma = h_-m(k)
     reproduces Revision/pairing/reports/python-pairing.json, check Q.one_particle_maps
PASS exact: gamma^(x0) h_m(k) gamma^(x0) = h_-m(R_0 k)
     reproduces Revision/pairing/reports/wolfram-pairing.json, check Q_one_particle_maps
```

Text cell:

```
The Revision theory record also states how the good-sector spectrum splits between
the two eigenspaces of $B$. In the good sector $B$ commutes with $h$, so the
projector $P_B = \frac12(I + B)$ onto the eigenvalue $+1$ of $B$ (8 directions)
commutes with $h$, and $h$ acts inside that 8-dimensional space. For the exact
example $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $w = 5$, the matrix $P_B h P_B$
must have the
eigenvalue $+5$ four times, $-5$ four times and $0$ eight times (the 0 belongs to the
other 8 directions, which $P_B$ removes). The next cell checks this and then draws
the spectra of $h_m$ and $h_{-m}$ for $m = 2$ against $k_1$ and, at fixed $k_1 = 1$,
against $m$ from $-3$ to $3$.
```

In [17]:

```
h_sample = mode_hamiltonian(2, {0: 4, 1: 1, 2: 2})
P_B = (np.eye(16) + B) / 2  # projector onto the eigenvalue +1 of B
sector_values = np.round(np.linalg.eigvals(P_B @ h_sample @ P_B).real, 10)
multiplicities = {v: int(np.sum(sector_values == v)) for v in (5.0, -5.0, 0.0)}
say(f"eigenvalues of P_B h P_B and how often each occurs: {multiplicities}")
check_record(multiplicities == {5.0: 4, -5.0: 4, 0.0: 8},
             "good sector, m = 2, k = (4, 1, 2, 0): on B = +1 energies +5, -5 (4 each)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")

k1_values = np.linspace(0.0, 4.0, 41)
plus_spectra = [np.sort(np.linalg.eigvals(mode_hamiltonian(2.0, {1: x})).real)
                for x in k1_values]
minus_spectra = [np.sort(np.linalg.eigvals(mode_hamiltonian(-2.0, {1: x})).real)
                 for x in k1_values]
mass_values = np.linspace(-3.0, 3.0, 61)
mass_spectra = [np.sort(np.linalg.eigvals(mode_hamiltonian(x, {1: 1.0})).real)
                for x in mass_values]
difference = np.max(np.abs(np.array(plus_spectra) - np.array(minus_spectra)))
```

```
report("largest difference between the spectra of m = 2 and m = -2", f"{difference:.1e}")
check(difference < 1e-10, "numerical: the spectra of h_m and h_-m coincide")

fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0))
axes[0].plot(k1_values, np.array(plus_spectra)[:, [0, 15]], color=BLUE, linewidth=2)
axes[0].plot(k1_values, np.array(minus_spectra)[:, [0, 15]], "o", color=ORANGE,
             markersize=5, fillstyle="none")
axes[0].plot([], [], color=BLUE, linewidth=2, label="$m = +2$ (line)")
axes[0].plot([], [], "o", color=ORANGE, fillstyle="none", label="$m = -2$ (circles)")
axes[0].set_xlabel("momentum $k_1$ (with $m = \\pm 2$)")
axes[0].set_ylabel("eigenvalues $\\pm w$ of $h$")
axes[0].set_title("same spectrum for $+m$ and $-m$")
axes[0].legend(loc="center left")
axes[1].plot(mass_values, np.array(mass_spectra)[:, [0, 15]], color=GREEN,
             linewidth=2)
axes[1].set_xlabel("mass $m$ (with $k_1 = 1$)")
axes[1].set_ylabel("eigenvalues $\\pm\\sqrt{m^2 + k_1^2}$ of $h$")
axes[1].set_title("the spectrum is even in $m$")
save_figure(fig, "plus_minus_mass_spectra",
            "Left: the eigenvalues $\\pm w = \\pm\\sqrt{m^2 + k_1^2}$ of the mode "
            "Hamiltonian for $m = +2$ (blue lines) and $m = -2$ (orange circles), "
            "computed separately, against the momentum $k_1$ (horizontal axis); "
            "the circles sit exactly on the lines. Right: the same eigenvalues at "
            "$k_1 = 1$ against the mass $m$ from $-3$ to $3$ (horizontal axis); "
            "the green curves are mirror images in the vertical axis $m = 0$. "
            "Vertical axes: eigenvalue (same units as $m$). The universes of "
            "masses $+m$ and $-m$ have identical, not opposite, one-particle "
            "spectra.")
```

Out [17]:

```
eigenvalues of P_B h P_B and how often each occurs: {5.0: 4, -5.0: 4, 0.0: 8}
PASS good sector, m = 2, k = (4, 1, 2, 0): on B = +1 energies +5, -5 (4 each)
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_spectrum_and_B_sectors
RESULT largest difference between the spectra of m = 2 and m = -2 = 0.0e+00
PASS numerical: the spectra of h_m and h_-m coincide
```

Out [17]:

```
Figure 10a.6 saved as Revision/textbook/figures/10a_6_plus_minus_mass_spectra.png
```

Text cell:

```
## 13. The last check

The last cell checks that all six figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [18]:

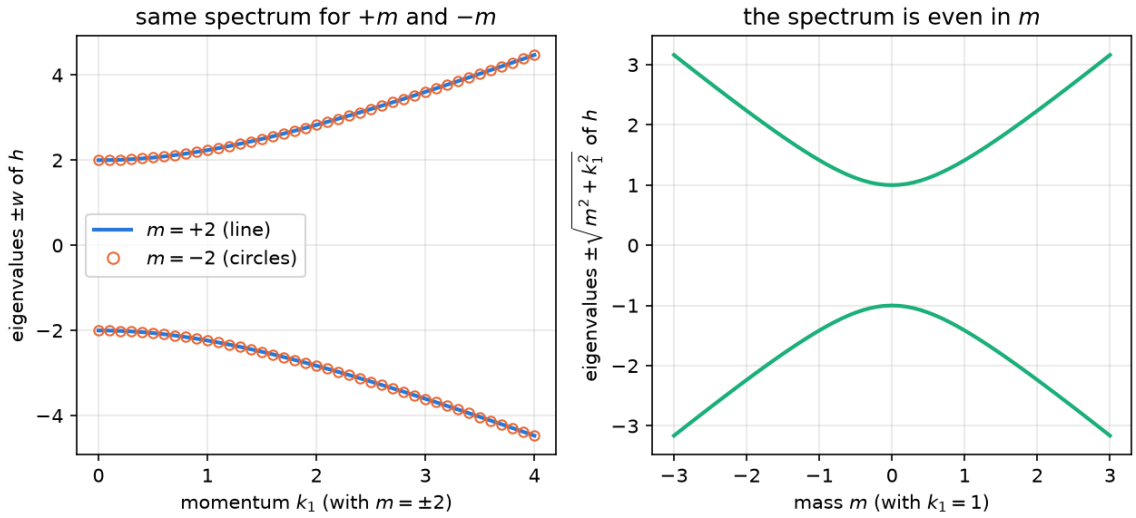

Figure 272: Left: the eigenvalues $\pm w = \pm\sqrt{m^2 + k_1^2}$ of the mode Hamiltonian for $m = +2$ (blue lines) and $m = -2$ (orange circles), computed separately, against the momentum k_1 (horizontal axis); the circles sit exactly on the lines. Right: the same eigenvalues at $k_1 = 1$ against the mass m from -3 to 3 (horizontal axis); the green curves are mirror images in the vertical axis $m = 0$. Vertical axes: eigenvalue (same units as m). The universes of masses $+m$ and $-m$ have identical, not opposite, one-particle spectra. (Notebook 10a, figure 6.)

```
for name in ("10a_1_gamma4_c_and_b.png", "10a_2_frequency_squared.png",
             "10a_3_eigenvalue_flow.png", "10a_4_krein_inertia_scan.png",
             "10a_5_krein_gram_matrices.png", "10a_6_plus_minus_mass_spectra.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [18]:

```
PASS the figure file 10a_1_gamma4_c_and_b.png exists
PASS the figure file 10a_2_frequency_squared.png exists
PASS the figure file 10a_3_eigenvalue_flow.png exists
PASS the figure file 10a_4_krein_inertia_scan.png exists
PASS the figure file 10a_5_krein_gram_matrices.png exists
PASS the figure file 10a_6_plus_minus_mass_spectra.png exists
ALL 28 CHECKS PASSED (notebook 10a)
```

Text cell:

```
## 14. What this notebook showed

- PROVED (exactly, with sympy, for all real $m$ and momenta): the mode Hamiltonian
  of a plane wave obeys $h^2 = w^2 I_{16}$ with $w^2 = m^2 + k_1^2 + k_2^2 + k_3^2 +
  k_0^2 - k_5^2 - k_6^2 - k_7^2$, and $Bh = h^\dagger B$, so the Krein form
  $u^\dagger B u$ (the charge) of every wave is conserved, also of a growing one;
```

```
  $h$ is Hermitian and commutes with $B$ exactly in the good sector.
- PROVED (steps (i) and (ii) exactly, step (iii) by continuity; flat 4+4 space or
  in the frozen-coefficient model, an ASSUMPTION): every eigenspace of a REAL
  frequency has Krein inertia
  $(4, 4)$; every eigenspace of an imaginary
  frequency, and the range of $h$ at zero frequency, is Krein-neutral. So the
  positive and negative charges are mixed half and half at every real frequency;
  the good sector does not remove this indefiniteness of the one-particle waves.
- PROVED (exactly): $\Gamma h_m\Gamma = h_{-m}$ and $\gamma^{(x_0)}h_m(k)\gamma^{(x_0)}
  = h_{-m}(R_8k)$: the universes of masses $+m$ and $-m$ have identical
  one-particle spectra.
- REPRODUCED: the properties of $B$ (signature $(8, 8)$), the eight recorded
  samples of the pairing record, the recorded Krein inertia and neutrality samples,
  and the good-sector split $+5$ (4 times), $-5$ (4 times) on $B = +1$.
- These are statements about one-particle waves in flat 4+4 space or in the
  frozen-coefficient model, which leaves out the hidden-direction terms of the
  author's field equation (an ASSUMPTION). How a positive quantum state space is
  built from them (and where that is possible) is the subject of the next notebooks
  of this chapter.
```

10.10 Line-by-line walk-through of Notebook 10a

The notebook has 18 code cells, In [1] to In [18]. This section explains every line of each of them. Python, the language of the notebooks, is read from top to bottom; a line that starts with `#` is a **comment** for the reader, which Python skips, and the text after `#` on a line of code is a comment too. Where a cell defines a function, the **docstring** (the text in triple quotes below the `def` line, which only describes the function) is left out of the quotations; it is printed in Section 10.9.

In [1], the set-up cell. Its first 247 lines are comments: they repeat, word for word, the run instructions of Section 10.8 (Python skips them; they are there so that the notebook carries its own instructions, so this comment block is different in every notebook), and end with the heading `# THE SET-UP (the same in every notebook of this book; it computes no physics)` between two lines of `=` signs. The code starts below that heading. The words "the same in every notebook of this book" in the heading are not exactly true; here is what is the same. The code below the heading is the same in all seven notebooks of this chapter, except for the line that sets `NOTEBOOK_ID`. In the whole book, that line differs from notebook to notebook, and the notebooks that run a Rust program also import `shutil` and `subprocess` and define one more helper, `rust_program`, which this chapter does not need.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module**, a collection of ready-made functions, so that the code can use it. The four modules come with Python. `json` reads and writes JSON files, the format of the Revision records; `os` gives access to the operating system; `textwrap` breaks a long text into lines. `from pathlib imp ort Path` takes only the name `Path` out of the module `pathlib`: a `Path` is the address of a file or a folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

matplotlib is the package that draws the figures; its drawing functions are loaded under the short name `plt`. `Image` and `display` come from IPython, the part of Jupyter that runs Python code; together they show a picture file below a cell.

```
NOTEBOOK_ID = "10a"  # this notebook: chapter 10, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text between quotes) `"10a"`. The figure files and the last printed line of the notebook use it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs each time it is called. `Path.cwd()` is the folder in which the notebook runs (Jupyter runs a notebook in the folder that holds it), and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with them. The `for` loop takes these folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds the file `Revision/textbook/requirements.txt` is the repository, and `return` hands it back to the caller. If no folder qualifies, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The two comment lines say what the next line does and why. That line calls the function and names its result `REPO`. It is never printed, because it differs from computer to computer while the printed output of a notebook must be the same everywhere. The three comment lines that follow explain the last line, which chooses the folder below which files are written. `os.environ` holds the **environment variables**, named texts that a program receives from the computer; `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set, and otherwise the repository folder (`str(REPO)` writes it as a text). When you run the notebook the variable is not set, so files go into the repository; the book's checking tool sets it to a scratch folder, so that a check never changes the repository.

```
def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` is the complete path of a file of the repository; the notebook uses it to READ the Revision records. `output_file("Revision/...")` is the path at which to WRITE a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder of the file, and every missing folder above it, and does nothing if it already exists.

```
def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of this book: `textwrap.fill` breaks the text at blanks, and every line after the first starts with four blanks. `str(text)` turns a number or a list into text first.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings of matplotlib, so that a personal settings file on your computer cannot change the figures. `plt.rcParams.update({...})` then sets four of them: the size of a figure (7.0 by 4.2 inches), the size of the letters (10 points), and a faint grid behind every plot (`"grid.alpha": 0.3` makes it 30 percent opaque). The braces make a **dictionary**, a collection of pairs `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with the letter `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is the text `Revision/textbook/figures/10a.captions.json`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary, `{}` and a line end, into the captions file, which `save_figure` fills later; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` writes the same line end on every operating system, as the two comment lines above it say.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
```

```
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far (`len` is the number of entries); so the figures are numbered 1, 2, 3, and a cell that is run again keeps its numbers. The file name joins the notebook id, the number and the name, for example `10a_1_gamma4_c_and_b.png`. The three comment lines explain the options of the next line: `fig.savefig` writes the picture as a PNG file with 150 dots per inch (`dpi=150`), cuts away the empty margin (`bbox_inches="tight"`) and stores no program name in the file (`metadata={"Software": None}`), so that two runs write exactly the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time at the end of the cell. The caption is stored, and the whole dictionary of captions is written to the captions file; `json.dumps` turns the dictionary into JSON text with sorted keys. `display(Image(...))` shows the saved picture below the cell, with a label that the book's tools read, and the last line prints where the figure was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list**, an ordered collection written in square brackets. `check` is the function behind every check of the book. `condition` is a truth value, `True` or `False`. If it is false, `raise AssertionError(...)` stops the notebook with an error message that names the check (an `if` statement is used instead of Python's `assert`, because Python started with the option O, for optimise, would skip an `assert`). If it is true, the name is appended to `PASSED` and the line PASS name is printed; when the optional third argument `record` names a Revision record, a second line says which record and check the result reproduces.

```
def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))
```

```
def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; the unit is added only when one is given (`a if condition else b` is `a` when the condition holds and `b` otherwise). `all_checks_passed` prints the last line of the notebook with the number of checks that passed. The last statement prints the single output line of In [1]. Two strings written next to each other, as here, are joined into one.

In [2], the gammas and the Clifford relation.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols
```

Three more modules of Python are loaded, and the two mathematical packages of the book: numpy (short name `np`) computes with arrays of numbers, sympy (short name `sp`) computes exactly with symbols.

```
REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"
```

`REPORT_CHECKS` starts as an empty dictionary; it will hold, for every report file that the notebook cites, the verdicts of the report's checks, stored under the check names. `record_says_pass(record)` answers one question: does the cited record still say what the notebook claims? A record name such as `"Revision/theory/reports/python-field-theory.json, check B_properties"` is a file name and a check name joined by the text `, check `. `record.partition(", check ")` cuts the record name at the first occurrence of that text into three pieces: the part before it (`path`), the text itself (`separator`) and the part after it (`check_name`). If the text does not occur, `separator` is empty and the record names a data entry of a file, such as `"Revision/algebra/gammas.json, entry B"`, which the notebook reads and compares itself; the answer is then True. Otherwise the report file is

read, once: `json.loads(...)["checks"]` is the list of its checks, each a dictionary with the keys `name`, `verdict` and `detail`, and the dictionary comprehension stores every verdict under its check name in capital letters (`.upper()`; some reports write `pass` in small letters). The answer is True only if a check of that name exists and its verdict is `PASS`; `.get` gives `None` for a missing name, and `None == "PASS"` is False.

```
def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output
```

`check_record` first asks `record_says_pass`. If the answer is False, `raise AssertionError(...)` stops the notebook with an error message that names the record, in the same way as a failed check: the notebook never prints that it reproduces a record check that has been renamed, removed or turned to FAIL. Otherwise `check_record` does what `check(condition, name, record=record)` does, but prints the PASS line and the reproduces line in one piece. `io.StringIO()` is a **text buffer**, a piece of memory that collects printed text. Inside the `with` block everything that is printed goes into the buffer instead of the screen; `check` prints its two lines there (or stops the notebook if the condition is false). Then `sys.stdout.write` sends the collected text to the screen at once. Jupyter delivers printed text in pieces whose boundaries depend on timing; printing the two lines as one piece keeps the stored output of the notebook the same in every run.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
# gamma[a] is the matrix gamma^(x_a) of the coordinate x_a, a = 1, ..., 8.
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
eta = {a: fixture["eta"][a] for a in range(8)}  # +1 space-like, -1 time-like
identity = np.eye(16, dtype=int)
say(f"eta = {[eta[a] for a in range(8)]} for x0, ..., x7")
```

`read_text` reads the record file as text, and `json.loads` turns that text into Python objects: `fixture` is a dictionary whose keys are the names stored in the record. `fixture["gamma"]` is a list of eight matrices, each a list of 16 rows of 16 whole numbers, in the order $x_0, \ldots, x_7$; Python counts list positions from 0, like the author's coordinates, so the matrix of x_a is at position `a`. The **dictionary comprehension** `{a: ... for a in range(8)}` builds a dictionary in one line: for $a = 0, \ldots, 7$ (`range(8)` runs from 0 up to, but not including, 8) it stores the matrix as a numpy array of whole numbers (`dtype=int`), so that every product below is exact. `eta` stores the eight signs of the frame metric in the same way, and `np.eye(16, dtype=int)` is the identity matrix I_{16}. The `say` line prints the list of the eight signs; its output, `eta = [1, 1, 1, 1, -1, -1, -1, -1] for x0, ..., x7`, is the frame metric η of the notation paragraph of Section 10.1.

```
clifford_ok = all(
    np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                   2 * (eta[a] if a == b else 0) * identity)
    for a in range(8) for b in range(8))  # all 64 pairs (a, b)
check(clifford_ok,
      "the 64 Clifford relations hold exactly (eta = diag(+,+,+,+,-,-,-,-))")
```

In Python the sign `@` multiplies two matrices. For each of the $8 \times 8 = 64$ pairs (a, b) the expression compares $\gamma^{(x_a)}\gamma^{(x_b)} + \gamma^{(x_b)}\gamma^{(x_a)}$ with $2\eta_{ab} I_{16}$ (the factor is η_{aa} when $a = b$ and 0 otherwise); `np.array_equal` is true only when all 256 entries agree. `all(...)` is true when the comparison holds for every pair. The check prints PASS: the Clifford relation of Section 10.1 holds exactly.

In [3], the matrices C and B.

```
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4), a complex 16 x 16 matrix
B_file = np.array(fixture["B"]["re"]) + 1j * np.array(fixture["B"]["im"])
check_record(np.array_equal(B, B_file),
             "B = -i C gamma^(x4) equals the matrix B of the record",
             record="Revision/algebra/gammas.json, entry B")
```

The first two lines build $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ and $B = -iC\gamma^{(x_4)}$; Python writes the imaginary unit i as `1j`. The record stores B too, its real and imaginary parts as two tables of numbers; `B_file` puts them together. The check compares the two matrices entry by entry and reproduces the record's entry `B`.

```
purely_imaginary = np.all(B.real == 0)  # every entry is 0, +i or -i
hermitian = np.array_equal(B, B.conj().T)  # B^dagger = B
squares_to_one = np.array_equal(B @ B, np.eye(16))  # B^2 = I
trace_B = np.trace(B)  # the sum of the diagonal entries
eigenvalues_B = np.linalg.eigvalsh(B)  # the 16 real eigenvalues, sorted
n_plus = int(np.sum(eigenvalues_B > 0.5))  # how many are +1
n_minus = int(np.sum(eigenvalues_B < -0.5))  # how many are -1
```

These lines test the properties (B2) to (B5) of Section 10.2. `B.real == 0` is a table of truth values, and `np.all` is true when all are true. `B.conj().T` is the conjugate transpose $B^\dagger$ (`.conj()` conjugates every entry, `.T` transposes). `np.trace` is the trace. `np.linalg.eigvalsh` computes the eigenvalues of a Hermitian matrix, which are real, sorted from the smallest; `eigenvalues_B > 0.5` marks those that are $+1$, `np.sum` counts the marks, and `int` writes the count as an ordinary whole number.

```
report("trace of B", int(round(abs(trace_B))))
report("eigenvalues of B equal to +1 and to -1", f"{n_plus} and {n_minus}")
check_record(purely_imaginary and hermitian and squares_to_one and trace_B == 0
             and (n_plus, n_minus) == (8, 8),
             "B is imaginary and Hermitian, B^2 = I, tr B = 0, signature (8, 8)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "B_properties")
```

The two RESULT lines print the trace, 0, and the counts, 8 and 8. The check requires all four properties and the signature (8,8), and reproduces the check `B_properties` of `python-field-theory.json`.

In [4], a heat map of three matrices.

```
from matplotlib.colors import LinearSegmentedColormap  # colour scales
```

```
BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
# A diverging colour scale: dark blue (negative), light grey (zero), red (positive).
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])
# A sequential colour scale for sizes: almost white (zero) to dark blue (large).
SEQUENTIAL = LinearSegmentedColormap.from_list(
    "white_blue", ["#fcfcfb", "#86b6ef", "#184f95"])
```

A colour is written as a text `"#rrggbb"`, three two-digit numbers in base 16 for its red, green and blue parts. The first line of assignments gives four colours four names at once (Python assigns the values on the right to the names on the left in order); the colours stay distinguishable for colour-blind readers. A **colour scale** turns a number into a colour: `DIVERGING` runs from dark blue for -1 through light grey for 0 to red for $+1$, and `SEQUENTIAL` from almost white for 0 to dark blue for large values. `from_list` builds a scale that blends smoothly between the listed colours.

```
def draw_matrix(ax, matrix, title, cmap=DIVERGING, vmin=-1.0, vmax=1.0):
    image = ax.imshow(matrix, cmap=cmap, vmin=vmin, vmax=vmax)
    size = matrix.shape[0]
    ticks = list(range(0, size, 3))  # every third row and column is labelled
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.grid(False)  # no grid lines on top of the squares
    ax.set_title(title)
    return image
```

A **heat map** draws a matrix as a grid of coloured squares, one for each entry. `ax` is one drawing area (matplotlib calls it an axes); `ax.imshow` draws the matrix with the colour scale `cmap`, where `vmin` and `vmax` are the numbers that get the two end colours. `matrix.shape[0]` is the number of rows. The tick marks are put at every third row and column, labelled from 1 (the computer counts from 0, the book from 1). The grid is switched off because it would cut through the squares. The function returns the drawn image, which a colour bar needs.

```
# A purely imaginary B = i Y (Y real) is Hermitian exactly when -i Y^T = i Y,
# that is when Y^T = -Y: the imaginary part of B must be antisymmetric.
one_per_row = all(np.count_nonzero(M, axis=1).tolist() == [1] * 16
                  and np.count_nonzero(M, axis=0).tolist() == [1] * 16
                  for M in (gamma[4], C, B.imag))  # signed permutation matrices
check(np.array_equal(gamma[4].T, -gamma[4]) and np.array_equal(C.T, C)
      and np.array_equal(B.imag.T, -B.imag) and one_per_row,
      "gamma^(x4) and Im B are antisymmetric, C is symmetric, all signed permutations")
```

`np.count_nonzero(M, axis=1)` counts the nonzero entries in each row of M and `axis=0` in each column; `.tolist()` turns the result into a list, and `[1] * 16` is the list of sixteen ones. So `one_per_row` is true when each of the three matrices $\gamma^{(x_4)}$, C and the imaginary part of B has exactly one nonzero entry in every row and every column. The check also tests that $\gamma^{(x_4)}$ is antisymmetric ($M^T = -M$), C symmetric, and the imaginary part of B antisymmetric. The two comment lines at the top give the reason for the last fact: if $B = iY$ with Y real, then $B^\dagger = -iY^T$, and $B^\dagger = B$ means $Y^T = -Y$.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], gamma[4], "$\\gamma^{(x_4)}$ (real, antisymmetric)")
draw_matrix(axes[1], C, "$C$ (real, symmetric)")
image = draw_matrix(axes[2], B.imag, "imaginary part of $B = -iC\\gamma^{(x_4)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "gamma4_c_and_b",
...)
```

(The caption text, the third argument of `save_figure`, is shortened here to three dots; it is printed in full under the figure in Section 10.9.) `plt.subplots(1, 3, ...)` makes a figure with one row of three drawing areas, 11 by 4 inches. The titles are written in LaTeX, the mathematical notation of the book; inside a Python string a backslash is written twice. `fig.colorbar` adds the colour bar on the right, shared by the three maps. Figure 1 of Notebook 10a shows the result: in each of the three heat maps every row and every column has exactly one coloured square; C is the same after reflection in the diagonal, while $\gamma^{(x_4)}$ and the imaginary part of B change colour under that reflection. This is the picture of the table of B in Section 10.2.

In [5], the mode Hamiltonian as numbers.

```
def mode_hamiltonian(m, k):
    h = -1j * m * gamma[4]
    for a, k_a in k.items():
        h = h - k_a * (gamma[4] @ gamma[a])
    return h


def frequency_squared(m, k):
    return m ** 2 + sum(eta[a] * k_a ** 2 for a, k_a in k.items())
```

`mode_hamiltonian(m, k)` builds $h = -im\gamma^{(x_4)} - \gamma^{(x_4)} \sum_a k_a \gamma^{(x_a)}$ of Section 10.3; `k` is a dictionary that gives the momenta of some of the directions, for example `{1: 0.3, 5: 0.4}`, and `k.items()` runs through its pairs (direction, momentum). Directions that are not listed have momentum 0. `frequency_squared(m, k)` returns $w^2 = m^2 + \sum_a \eta_{aa} k_a^2$ (`**` is the power in Python): the space-like momenta enter with $+$, the extra-time momenta with $-$.

```
example = {0: 1.3, 1: 0.3, 2: -1.1, 3: 0.7, 5: 0.4, 6: 0.2, 7: -0.9}  # any numbers
h_example = mode_hamiltonian(1.7, example)
w2_example = frequency_squared(1.7, example)
report("w^2 for m = 1.7 and the example momenta", f"{w2_example:.6f}")
error = np.max(np.abs(h_example @ h_example - w2_example * np.eye(16)))
report("largest entry of h^2 - w^2 I (rounding only)", f"{error:.1e}")
check(error < 1e-12, "numerical example: h^2 = w^2 I at one generic point")
```

For the mass 1.7 and momenta along all seven slice directions the cell computes $w^2 = 2.89 + (0.09 + 1.21 + 0.49 + 1.69) - (0.16 + 0.04 + 0.81) = 5.36$, printed with six decimals (`:.6f`), and the largest size of an entry of $h^2 - w^2 I_{16}$ (`np.abs` takes sizes, `np.max` the largest), printed in the exponent form `8.9e-16`, which means 8.9×10^{-16}: only rounding. numpy computes with about 16 significant digits, so its checks allow a tiny tolerance.

In [6], the same, exactly.

```
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
C_exact = g[0] * g[1] * g[2] * g[3]
B_exact = -sp.I * C_exact * g[4]
m = sp.Symbol("m", real=True)  # the mass: any real number
k_sym = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}
```

The same gammas are stored as sympy matrices, which hold exact whole numbers; for sympy matrices `*` is the matrix product and `sp.I` is the imaginary unit. `sp.Symbol("m", real=True)` is a letter m that stands for any real number, and `k_sym` holds the seven letters $k_1, k_2, k_3, k_5, k_6, k_7, k_0$. A computation with these letters holds for all their values at once: it is a proof.

```
def mode_hamiltonian_exact(mass, k):
    h = -sp.I * mass * g[4]
    for a, k_a in k.items():
        h = h - k_a * g[4] * g[a]
    return h


zero = sp.zeros(16, 16)
h_sym = mode_hamiltonian_exact(m, k_sym)
w2_sym = m ** 2 + sum(eta[a] * k_sym[a] ** 2 for a in k_sym)
say(f"w^2 = {w2_sym}")
```

The function builds h exactly; `h_sym` is h for the letters, `w2_sym` the expression w^2, which the cell prints: `k0**2 + k1**2 + k2**2 + k3**2 - k5**2 - k6**2 - k7**2 + m**2` (sympy writes the power as `**`).

```
square_ok = (h_sym * h_sym - w2_sym * sp.eye(16)).applyfunc(sp.expand) == zero
check_record(square_ok, "exact: h^2 = w^2 I for all real m and k",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_one_particle_flat_dispersion")
krein_ok = (B_exact * h_sym - h_sym.H * B_exact).applyfunc(sp.expand) == zero
check_record(krein_ok,
             "exact: B h = h^dagger B for all real m and k (Krein self-adjoint)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "mode_hamiltonian_B_selfadjoint_dispersion")
```

`applyfunc(sp.expand)` multiplies out every entry of a matrix, so that an entry that is zero for all values of the letters becomes literally 0; comparing with the zero matrix then decides the identity. The first check proves $h^2 = w^2 I_{16}$ (Section 10.4) and reproduces the Wolfram check `Q_one_particle_flat_dispersion`; the second proves $Bh = h^\dagger B$ (`.H` is sympy's conjugate transpose; the letters are declared real, so their conjugates are themselves) and reproduces the sympy record check `mode_hamiltonian_B_selfadjoint_dispersion`.

In [7], where h stops being Hermitian.

```
not_hermitian_part = (h_sym - h_sym.H).applyfunc(sp.expand)
commutator = (B_exact * h_sym - h_sym * B_exact).applyfunc(sp.expand)
symbols_1 = sorted(str(s) for s in not_hermitian_part.free_symbols)
```

```
symbols_2 = sorted(str(s) for s in commutator.free_symbols)
say(f"symbols in h - h^dagger: {symbols_1}")
say(f"symbols in B h - h B:    {symbols_2}")
```

The cell computes the exact matrices $h - h^\dagger$ and $Bh - hB$. `.free_symbols` is the set of the letters that occur in an expression; `sorted(str(s) for s in ...)` writes them as a sorted list of names (a set has no fixed order, a sorted list has). Both printed lists are `['k5', 'k6', 'k7']`: only the extra-time momenta make h non-Hermitian and make it fail to commute with B, as Section 10.4 derived from $h = h_H + h_A$.

```
good_sector = {k_sym[5]: 0, k_sym[6]: 0, k_sym[7]: 0}  # no extra-time momentum
check_record(symbols_1 == ["k5", "k6", "k7"]
             and not_hermitian_part.subs(good_sector) == zero,
             "exact: h is Hermitian exactly in the good sector k5 = k6 = k7 = 0",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "mode_hamiltonian_good_sector")
check_record(symbols_2 == ["k5", "k6", "k7"]
             and commutator.subs(good_sector) == zero,
             "exact: B h = h B exactly in the good sector",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")
```

`.subs(good_sector)` replaces k_5, k_6, k_7 by 0. The two checks require that only these letters occur and that both matrices vanish when they are 0, and reproduce the two record checks named.

In [8], the squared frequency against the extra-time momentum.

```
k5_values = np.linspace(0.0, 3.0, 301)  # 301 values of k5 from 0 to 3
fig, ax = plt.subplots()
for k_s, colour in ((0.0, BLUE), (1.0, ORANGE), (2.0, GREEN)):
    w2_curve = [frequency_squared(1.0, {1: k_s, 5: k5}) for k5 in k5_values]
    ax.plot(k5_values, w2_curve, color=colour, linewidth=2,
            label=f"$k_s = {k_s:.0f}$")
    crossing = np.sqrt(1.0 + k_s ** 2)  # where w^2 = 0
    ax.plot([crossing], [0.0], "o", color=colour, markersize=8)
```

`np.linspace(0.0, 3.0, 301)` is a list of 301 equally spaced numbers from 0 to 3. For the mass 1 and three space-like momenta $k_s = 0, 1, 2$ (put along x_1) the loop computes $w^2 = 1 + k_s^2 - k_5^2$ at every k_5 (a **list comprehension** `[... for k5 in k5_values]` builds the list in one line), draws it as a line (`ax.plot(x, y, ...)`), and marks with a dot (`"o"`) the point $k_5 = \sqrt{1 + k_s^2}$ where $w^2 = 0$.

```
ax.axhline(0.0, color=GREY, linewidth=1)
ax.text(0.1, 3.0, "$w^2 > 0$: real frequency, the wave oscillates", color=GREY)
ax.text(0.1, -6.0, "$w^2 < 0$: imaginary frequency, the wave grows", color=GREY)
ax.set_xlabel("momentum $k_5$ along the extra time $x_5$")
ax.set_ylabel("$w^2 = m^2 + k_s^2 - k_5^2$")
ax.set_title("The squared frequency of a plane wave, $m = 1$")
ax.legend(title="space-like momentum")
save_figure(fig, "frequency_squared",
...)
```

`axhline` draws a horizontal line at height 0; `ax.text` writes a text at a point of the plot; the labels name the axes; `ax.legend` lists the three curves with their labels. Figure 2 of Notebook 10a shows three downward parabolas, one for each k_s, each crossing zero at its dot: above the grey line the wave oscillates, below it the wave grows. A larger space-like momentum moves the crossing to the right, but every curve crosses: a large enough extra-time momentum always wins.

```
check(all(abs(frequency_squared(1.0, {1: k_s, 5: np.sqrt(1.0 + k_s ** 2)})) < 1e-12
          for k_s in (0.0, 1.0, 2.0)),
      "w^2 = 0 exactly at k5 = sqrt(m^2 + k_s^2)")
```

The check confirms that w^2 vanishes at the three dots, up to rounding.

In [9], the eigenvalues as k_5 grows.

```
real_parts, imaginary_parts, worst = [], [], 0.0
for k5 in k5_values:
    eigenvalues = np.linalg.eigvals(mode_hamiltonian(1.0, {5: k5}))
    real_parts.append(np.sort(eigenvalues.real))
    imaginary_parts.append(np.sort(eigenvalues.imag))
```

For each of the 301 values of k_5 (mass 1, no other momentum) `np.linalg.eigvals` computes the 16 eigenvalues of h, which may be complex numbers; their real parts and their imaginary parts are sorted and stored for the figure.

```
    w2 = 1.0 - k5 ** 2
    if abs(w2) > 0.05:  # away from the point where h cannot be diagonalised
        w = np.sqrt(complex(w2))  # sqrt of a negative number is imaginary
        predicted = np.array([w] * 8 + [-w] * 8)
        # compare the sorted real parts and the sorted imaginary parts separately
        worst = max(worst,
                    np.max(np.abs(np.sort(eigenvalues.real) - np.sort(predicted.real))),
                    np.max(np.abs(np.sort(eigenvalues.imag) - np.sort(predicted.imag))))
report("largest difference from the formula +-w (8 times each)", f"{worst:.1e}")
check(worst < 1e-10, "the 16 eigenvalues of h are +w and -w, eight times each")
```

The predicted eigenvalues are $+w$ eight times and $-w$ eight times; `complex(w2)` makes w^2 a complex number, so that `np.sqrt` of a negative number gives the imaginary $i\kappa$ instead of an error. At $k_5 = 1$ exactly, $h^2 = 0$ but $h \neq 0$: such a matrix has no basis of eigenvectors, and numerical eigenvalues are then only accurate to about 10^{-8}; points with $|w^2| < 0.05$ are therefore left out of the comparison. `worst` keeps the largest difference seen; it is printed as `3.6e-15`, so the formula holds to rounding.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0), sharex=True)
axes[0].plot(k5_values, np.array(real_parts), color=BLUE, linewidth=2)
axes[1].plot(k5_values, np.array(imaginary_parts), color=ORANGE, linewidth=2)
for ax, what in ((axes[0], "real part"), (axes[1], "imaginary part")):
    ax.axvline(1.0, color=GREY, linestyle=":", linewidth=1)
    ax.set_xlabel("momentum $k_5$ along the extra time $x_5$ ($m = 1$)")
    ax.set_ylabel(f"{what} of the eigenvalues of $h$")
axes[0].set_title("oscillation: real frequencies $\\pm w$")
```

```
axes[1].set_title("growth: imaginary frequencies $\\pm i\\kappa$")
save_figure(fig, "eigenvalue_flow",
...)
```

`np.array(real_parts)` is a table with one row per k_5 and 16 columns; `plot` draws each column as a curve. `sharex=True` gives both panels the same horizontal axis, and `axvline` draws a dotted vertical line at $k_5 = 1$. Figure 3 of Notebook 10a shows the real parts $\pm\sqrt{1 - k_5^2}$ closing in to 0 at $k_5 = 1$ and staying 0 afterwards, while the imaginary parts are 0 before and open up as $\pm\sqrt{k_5^2 - 1}$ afterwards: oscillation turns into growth. Each curve carries eight eigenvalues on top of each other.

In [10], three functions for the Krein inertia.

```
def orthonormal_basis(P):
    basis = []
    for column in P.T:  # the columns of P, in order
        v = column.astype(complex)
        for _ in range(2):  # the second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e  # remove the part of v along e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:  # v is not a combination of the earlier columns
            basis.append(v / length)
    return np.array(basis).T  # the basis vectors as the columns of a matrix
```

This is the **Gram-Schmidt method**. Two columns are **orthonormal** when each has length 1 and $e^\dagger v = 0$ between them. Going through the columns of P in order (`P.T` lists the columns of P as rows), the function removes from each column v its part along every basis vector e found so far ($e^\dagger v$ is the size of that part, and `e.conj() @ v` computes it), and keeps what is left, divided by its length $\sqrt{v^\dagger v}$, if anything is left. The removal is done twice, because the first pass leaves tiny rounding errors. A column that is a combination of the earlier ones leaves nothing (a length below 10^{-8}) and is skipped. The result is a matrix whose columns are an orthonormal basis of the range of P; `for _ in range(2)` repeats a block twice without needing the counter.

```
def eigenspace(m, k, sign):
    h = mode_hamiltonian(m, k)
    w = np.sqrt(complex(frequency_squared(m, k)))  # w, or i kappa when w^2 < 0
    P = (np.eye(16) + sign * h / w) / 2  # the projector P_+ or P_-
    return orthonormal_basis(P)
```

`eigenspace(m, k, sign)` builds the projector $P_\pm = \frac{1}{2}(I_{16} \pm h/w)$ of Section 10.5 and returns an orthonormal basis of its range, the eigenspace of $+w$ (`sign = +1`) or $-w$ (`sign = -1`). For $w^2 < 0$ the number w is $i\kappa$, and the same formula gives the eigenspaces of $\pm i\kappa$.

```
def krein_inertia(V):
    G = V.conj().T @ B @ V  # the Krein form on the span of the columns of V
    values = np.linalg.eigvalsh(G)
    nonzero = np.abs(values[np.abs(values) > 1e-8])
    counts = (int(np.sum(values > 1e-8)), int(np.sum(values < -1e-8)),
              int(np.sum(np.abs(values) <= 1e-8)))
    return counts, (float(nonzero.min()) if nonzero.size else None)
```

`krein_inertia(V)` forms the Gram matrix $G = V^\dagger B V$ of the columns of V (Section 10.5) and counts its positive, negative and zero eigenvalues, "zero" meaning smaller in size than 10^{-8}. It returns the triple (p, n, z) and, in addition, the smallest size of a nonzero eigenvalue (or `None`, Python's word for "nothing", when all are zero), to show that no eigenvalue lies near the borderline 10^{-8}. `values[np.abs(values) > 1e-8]` picks out the eigenvalues that are not zero.

In [11], the eight recorded samples.

```
pairing = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
samples = pairing["data"]["one_particle_flat"]["samples"]
all_agree = True
say("   m  nonzero momenta         w      dims   inertia(+w)  inertia(-w)")
```

The cell reads the pairing record and takes the list of eight samples stored under `data`, `one_part icle_flat`, `samples`. Each sample is a dictionary with a mass `m`, a list `k` of eight momenta $(k_0, \ldots, k_7)$ (the entry k_4 is not used), the frequency `w` as text, the dimensions of the two eigenspaces and their Krein inertia. The `say` line prints the header of the table.

```
for sample in samples:
    m_value, k_list = sample["m"], sample["k"]
    k = {a: k_list[a] for a in (0, 1, 2, 3, 5, 6, 7) if k_list[a] != 0}
    # "Sqrt[2]" (Wolfram notation) -> the number sqrt(2)
    w_recorded = float(sp.sympify(sample["w"].replace("Sqrt[", "sqrt(")
                                  .replace("]", ")")))
    w = np.sqrt(frequency_squared(m_value, k))
```

For each sample the loop builds the dictionary of the nonzero momenta (the `if` inside the comprehension skips the zeros). The record writes the frequency in the notation of the Wolfram Language, for example `Sqrt[2]`; `.replace` turns it into the sympy notation `sqrt(2)`, `sp.sympify` reads that text as an exact number, and `float` gives its decimal value. `w` is the frequency recomputed from m and k.

```
    found = []
    for sign in (+1, -1):
        V = eigenspace(m_value, k, sign)
        counts, _ = krein_inertia(V)
        found.append((V.shape[1], counts))
```

For both eigenspaces the loop computes a basis, its dimension (`V.shape[1]`, the number of columns) and its Krein inertia; the underscore receives the second result of `krein_inertia`, which is not needed here.

```
    agree = (abs(w - w_recorded) < 1e-12
             and found[0][0] == sample["dim_plus_w"]
             and found[1][0] == sample["dim_minus_w"]
             and list(found[0][1][:2]) == sample["B_inertia_plus_w"]
             and list(found[1][1][:2]) == sample["B_inertia_minus_w"]
             and found[0][1][2] == 0 and found[1][1][2] == 0)
    all_agree = all_agree and agree
```

`agree` requires the frequency, both dimensions and both inertias to equal the recorded ones, and no zero eigenvalue (`[:2]` takes the first two numbers of a triple, `[2]` the third). `all_agree` stays true only if every sample agrees.

```
    momenta = ", ".join(f"k{a}={v}" for a, v in k.items()) or "none"
    say(f"{m_value:4d}  {momenta:22s} {w:7.4f}   {found[0][0]},{found[1][0]}    "
        f"{found[0][1][:2]}       {found[1][1][:2]}")
check_record(all_agree,
             "all 8 recorded samples: w, dimensions 8 and 8, Krein inertia (4, 4)",
             record="Revision/pairing/pairing-theory.json, data one_particle_flat")
```

`", ".join(...)` writes the momenta as one text separated by commas, or the word `none` when there is no momentum (an empty text counts as false, so `or` takes the second value). The format codes fix the widths of the columns: `:4d` a whole number in 4 places, `:22s` a text in 22 places, `:7.4f` a decimal number in 7 places with 4 decimals. The printed table has one line per sample: for the masses ± 2 with $(k_0, k_1, k_2) = (4, 1, 2)$ the frequency is $5 = \sqrt{4 + 16 + 1 + 4}$; for ± 3 at rest it is 3; for ± 1 with $k_1 = 1$ it is $1.4142 = \sqrt{2}$; for ± 2 with the extra-time momentum $k_5 = 1$ it is $1.7321 = \sqrt{3}$. Every line shows the dimensions 8 and 8 and the inertia (4, 4) on both eigenspaces, the same for $+m$ and $-m$, and the check reproduces the record's data `one_particle_flat`.

In [12], one more real-frequency sample and the neutral eigenspaces.

```
sample_3 = {0: 2, 1: 2}
inertias_3 = [krein_inertia(eigenspace(1, sample_3, s))[0] for s in (+1, -1)]
report("w for m = 1, k0 = 2, k1 = 2", f"{np.sqrt(frequency_squared(1, sample_3)):.6f}")
say(f"inertia of the +w and -w eigenspaces: {inertias_3}")
check_record(inertias_3 == [(4, 4, 0), (4, 4, 0)],
             "m = 1, k0 = 2, k1 = 2 (w = 3): inertia (4, 4) on both eigenspaces",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_Krein_inertia")
```

The sympy pairing verifier uses the sample $m = 1$, $k_1 = 2$, $k_0 = 2$, so $w = \sqrt{1 + 4 + 4} = 3$, printed as `3.000000`. The inertia of both eigenspaces is printed as `[(4, 4, 0), (4, 4, 0)]` and reproduces the check `Q.one_particle_Krein_inertia`.

```
neutral_ok = True
for k in ({5: 2}, {1: 1, 5: 2}):
    spaces = [eigenspace(1, k, s) for s in (+1, -1)]
    inertias = [krein_inertia(V)[0] for V in spaces]
    say(f"m = 1, momenta {k}: w^2 = {frequency_squared(1, k)}, dimensions "
        f"{[V.shape[1] for V in spaces]}, inertia {inertias}")
    neutral_ok = neutral_ok and inertias == [(0, 0, 8), (0, 0, 8)]
```

Two imaginary-frequency samples of the record: $m = 1$ with $k_5 = 2$ ($w^2 = 1 - 4 = -3$) and with $k_1 = 1$, $k_5 = 2$ ($w^2 = 1 + 1 - 4 = -2$). The printed lines show dimensions 8 and 8 and the inertia $(0, 0, 8)$ for both eigenspaces: they are Krein-neutral, as the last part of the proof of Theorem 10.1 says.

```
h_zero = mode_hamiltonian(1, {5: 1})  # w^2 = 1 - 1 = 0
rank_h = int(np.linalg.matrix_rank(h_zero))
```

```
nilpotent = np.max(np.abs(h_zero @ h_zero)) < 1e-12  # h^2 = 0
range_neutral = np.max(np.abs(h_zero.conj().T @ B @ h_zero)) < 1e-12
say(f"m = 1, k5 = 1: w^2 = 0, h^2 = 0: {nilpotent}, rank of h = {rank_h}, "
    f"range of h neutral: {range_neutral}")
check_record(neutral_ok and nilpotent and rank_h == 8 and range_neutral,
             "imaginary and zero frequencies: the eigenspaces are Krein-neutral",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_complex_frequency_Krein_neutral")
```

The zero-frequency sample $m = 1$, $k_5 = 1$: `np.linalg.matrix_rank` is the **rank** of a matrix, the number of its independent columns. The cell checks $h^2 = 0$, rank 8, and that $h^\dagger B h = 0$, which says that the columns of h (and so its whole range) have zero Krein form with each other. The printed line shows `True`, 8 and `True`, and the check reproduces the record.

In [13], steps (i) and (ii) of the proof, exactly.

```
w = sp.Symbol("w", positive=True)
I16 = sp.eye(16)


def replace_w_squared(matrix):
    return matrix.applyfunc(lambda e: sp.expand(sp.expand(e).subs(w ** 2, w2_sym)))
```

`w` is now a positive letter. A **lambda** is a function written in one line without a name: `lambda e: ...` takes an entry `e`, multiplies it out, replaces every w^2 by $m^2 + k_s^2 - k_t^2$ (the expression `w2_sym` of In [6]) and multiplies out again. `replace_w_squared` does this for every entry of a matrix.

```
step_i_a = replace_w_squared((w * I16 + h_sym.H) * B_exact * (w * I16 - h_sym))
step_i_b = replace_w_squared((w * I16 + h_sym.H) * B_exact * (w * I16 + h_sym)
                             - 2 * w * B_exact * (w * I16 + h_sym))
check_record(step_i_a == zero and step_i_b == zero,
             "exact step (i): 4 w^2 P+^dagger B P- = 0 and P+^dagger B P+ = B P+",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_Krein_inertia_proof")
```

`step_i_a` is $(wI + h^\dagger)B(wI - h) = 4w^2 P_+^\dagger B P_-$, and `step_i_b` is $(wI + h^\dagger)B(wI + h) - 2wB(wI + h)$, which is $4w^2(P_+^\dagger B P_+ - B P_+)$. Both vanish exactly, which is step (i) of Theorem 10.1, and the check reproduces `Q.one_particle_Krein_inertia_proof`.

```
trace_B_h = sp.expand((B_exact * h_sym).trace())
say(f"exact: tr B = {B_exact.trace()}, tr(B h) = {trace_B_h}")
check(B_exact.trace() == 0 and trace_B_h == 0,
      "exact step (ii): tr B = tr(B h) = 0, so tr(B P+) = tr(B P-) = 0")
```

The exact traces of B and of Bh are printed as 0 and 0; with them $\mathrm{tr}(BP_\pm) = 0$, step (ii).

In [14], step (iii) at work: a scan across the threshold.

```
threshold = np.sqrt(1.25)
scan = [k5 for k5 in np.linspace(0.0, 2.5, 251) if abs(k5 - threshold) > 0.02]
```

```
counts_scan, margins = [], []
for k5 in scan:
    counts, margin = krein_inertia(eigenspace(1.0, {1: 0.5, 5: k5}, +1))
    counts_scan.append(counts)
    if margin is not None:
        margins.append(margin)
```

For $m = 1$ and $k_1 = 0.5$ the frequency is real while $k_5 < \sqrt{1 + 0.25} = \sqrt{1.25} \approx 1.118$. The list `scan` holds 251 values of k_5 from 0 to 2.5 without those closer than 0.02 to the threshold (there the two eigenspaces merge and the numbers become inaccurate). For each value the loop stores the inertia of the $+w$ eigenspace and the smallest size of a nonzero Gram eigenvalue.

```
below = [c for k5, c in zip(scan, counts_scan) if k5 < threshold]
above = [c for k5, c in zip(scan, counts_scan) if k5 > threshold]
report("points below and above the threshold", f"{len(below)} and {len(above)}")
report("smallest nonzero Gram eigenvalue below the threshold", f"{min(margins):.4f}")
check(set(below) == {(4, 4, 0)} and set(above) == {(0, 0, 8)},
      "scan: inertia (4, 4) at every real frequency, (0, 0, 8) at every imaginary "
      "one")
```

`zip` pairs each k_5 with its inertia. The cell prints 110 points below and 137 above the threshold, and the smallest nonzero Gram eigenvalue below the threshold, 0.2225: well away from 0, as step (iii) requires. `set(below)` is the set of the different inertias found below the threshold; the check requires it to be the single triple $(4, 4, 0)$, and above the threshold the single triple $(0, 0, 8)$.

```
counts_array = np.array(counts_scan)  # one row (positive, negative, zero) per k5
fig, ax = plt.subplots()
width = 0.0105  # a little wider than the step 0.01 between the k5 values (no gaps)
# Stacked bars: the blue part (positive directions) starts at 0, the orange part
# (negative directions) on top of it, the green part (neutral) on top of both.
ax.bar(scan, counts_array[:, 0], width=width, color=BLUE,
       label="positive charge $u^\\dagger B u > 0$")
ax.bar(scan, counts_array[:, 1], width=width, bottom=counts_array[:, 0],
       color=ORANGE, label="negative charge $u^\\dagger B u < 0$")
ax.bar(scan, counts_array[:, 2], width=width,
       bottom=counts_array[:, 0] + counts_array[:, 1], color=GREEN,
       label="neutral (the form vanishes)")
```

`counts_array[:, 0]` is the first column of the table (the positive counts), and so on. `ax.bar` draws one thin bar at each k_5 (the variable `width`, 0.0105, is a little more than the step 0.01 between neighbouring values of k_5, so that the bars touch); as the two comment lines say, `bottom=` stacks the second kind of bar on top of the first and the third on top of both, so each full bar has the height 8, split by colour into positive, negative and neutral directions.

```
ax.axvline(threshold, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("momentum $k_5$ along the extra time ($m = 1$, $k_1 = 0.5$)")
ax.set_ylabel("directions in the $+w$ eigenspace")
ax.set_yticks(range(0, 9))
ax.set_ylim(0.0, 10.5)  # room for the legend above the bars
```

```
ax.set_title("Krein inertia of one eigenspace across the threshold")
ax.legend(loc="upper center", ncol=3, fontsize=8)
save_figure(fig, "krein_inertia_scan",
...)
```

The dotted line marks the threshold; the vertical axis is labelled 0 to 8 and extended to 10.5 to leave room for the legend, which is written in three columns at the top. Figure 4 of Notebook 10a shows bars that are half blue and half orange left of the threshold and entirely green right of it: the waves of real frequency mix the two signs of charge four and four, the growing waves carry no charge.

In [15], the Gram matrices of three waves.

```
cases = [(2, {0: 4, 1: 1, 2: 2}, "$m = 2$, $k = (4, 1, 2, 0)$: $w = 5$"),
         (2, {5: 1}, "$m = 2$, $k_5 = 1$: $w = \\sqrt{3}$"),
         (1, {5: 2}, "$m = 1$, $k_5 = 2$: $w = i\\sqrt{3}$")]
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2))
block_sizes = []
```

Three waves are compared: a good-sector wave with $w = 5$, a wave with extra-time momentum and a real frequency $w = \sqrt{4-1} = \sqrt{3}$, and a wave with an imaginary frequency $w = \sqrt{1-4} = i\sqrt{3}$. Each entry of `cases` holds the mass, the momenta and the title of its panel.

```
for ax, (m_value, k, title) in zip(axes, cases):
    W = np.hstack([eigenspace(m_value, k, +1), eigenspace(m_value, k, -1)])
    gram = np.abs(W.conj().T @ B @ W)  # sizes of the Krein form between columns
    image = draw_matrix(ax, gram, title, cmap=SEQUENTIAL, vmin=0.0, vmax=1.0)
    ax.axhline(7.5, color=GREY, linewidth=1)  # the border between the two
    ax.axvline(7.5, color=GREY, linewidth=1)  # eigenspaces (rows/columns 8 and 9)
```

`np.hstack` puts the 8 basis columns of the $+w$ eigenspace and the 8 of the $-w$ eigenspace side by side into one 16×16 matrix W. `gram` holds the sizes of the entries of $W^\dagger B W$, the Krein form between every pair of these columns, drawn with the white-to-blue scale; grey lines separate the two eigenspaces.

```
    diagonal_blocks = max(gram[:8, :8].max(), gram[8:, 8:].max())
    off_blocks = max(gram[:8, 8:].max(), gram[8:, :8].max())
    block_sizes.append((diagonal_blocks, off_blocks))
    say(f"case {len(block_sizes)}: largest entry in the diagonal blocks "
        f"{diagonal_blocks:.3f}; off-diagonal blocks below 1e-12: {off_blocks < 1e-12}")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="size of the entry")
save_figure(fig, "krein_gram_matrices",
...)
```

`gram[:8, :8]` is the upper left 8×8 block (rows and columns 1 to 8), `gram[8:, 8:]` the lower right one, and the two others are the off-diagonal blocks. The printed lines show, for the first two cases, diagonal blocks with entries up to 0.894 and 0.866 and empty off-diagonal blocks (`True`), and for the third case empty diagonal blocks (`0.000`) and filled off-diagonal blocks (`False`). Figure 5 of Notebook 10a shows exactly this: for a real frequency the two eigenspaces are Krein-orthogonal (step (i)); for an imaginary frequency each eigenspace is neutral, and a growing wave has a nonzero Krein form only with a decaying one.

```
check(block_sizes[0][1] < 1e-12 and block_sizes[1][1] < 1e-12
      and block_sizes[2][0] < 1e-12 and block_sizes[2][1] > 0.1,
      "real w: the eigenspaces are Krein-orthogonal; imaginary w: each is neutral")
```

The check states the same in numbers.

In [16], the universes of masses $+m$ and $-m$.

```
Gamma_exact = g[0] * g[1] * g[2] * g[3] * g[4] * g[5] * g[6] * g[7]
h_minus = mode_hamiltonian_exact(-m, k_sym)  # the same momenta, mass -m
k_reflected = dict(k_sym)
k_reflected[0] = -k_sym[0]  # R_0: k0 -> -k0
h_minus_reflected = mode_hamiltonian_exact(-m, k_reflected)
```

The exact chirality Γ is the product of the eight gammas in the author's order. `h_minus` is $h_{-m}(k)$; `dict(k_sym)` makes a copy of the dictionary of momenta, in which k_0 is replaced by $-k_0$, and `h_minus_reflected` is $h_{-m}(R_0 k)$.

```
chirality_ok = (Gamma_exact * h_sym * Gamma_exact - h_minus).applyfunc(
    sp.expand) == zero
mirror_ok = (g[0] * h_sym * g[0] - h_minus_reflected).applyfunc(sp.expand) == zero
check_record(Gamma_exact * Gamma_exact == I16 and chirality_ok,
             "exact: Gamma h_m(k) Gamma = h_-m(k)",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_maps")
check_record(mirror_ok, "exact: gamma^(x0) h_m(k) gamma^(x0) = h_-m(R_0 k)",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_one_particle_maps")
```

The two identities of Section 10.6 are checked exactly, for all m and k, with $\Gamma\Gamma = I_{16}$; they reproduce the sympy and the Wolfram record.

In [17], the B-sectors of the good sector and the two spectra.

```
h_sample = mode_hamiltonian(2, {0: 4, 1: 1, 2: 2})
P_B = (np.eye(16) + B) / 2  # projector onto the eigenvalue +1 of B
sector_values = np.round(np.linalg.eigvals(P_B @ h_sample @ P_B).real, 10)
multiplicities = {v: int(np.sum(sector_values == v)) for v in (5.0, -5.0, 0.0)}
say(f"eigenvalues of P_B h P_B and how often each occurs: {multiplicities}")
check_record(multiplicities == {5.0: 4, -5.0: 4, 0.0: 8},
             "good sector, m = 2, k = (4, 1, 2, 0): on B = +1 energies +5, -5 (4 each)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")
```

In the good sector B commutes with h, so the projector $P_B = \frac{1}{2}(I_{16} + B)$ onto the eight columns with $Bu = u$ commutes with h, and h acts inside that 8-dimensional space. The eigenvalues of $P_B h P_B$ are those of h on that space, together with 0 for the eight directions that P_B removes. `np.round(..., 10)` rounds them to 10 decimals so that equal values compare equal. The printed dictionary `{5.0: 4, -5.0: 4, 0.0: 8}` says: on the $B = +1$ sector the energies are $+5$ and -5, four of each, as step (ii) of Theorem 10.1 requires; this is the record's exact example.

```
k1_values = np.linspace(0.0, 4.0, 41)
plus_spectra = [np.sort(np.linalg.eigvals(mode_hamiltonian(2.0, {1: x})).real)
                for x in k1_values]
minus_spectra = [np.sort(np.linalg.eigvals(mode_hamiltonian(-2.0, {1: x})).real)
                 for x in k1_values]
mass_values = np.linspace(-3.0, 3.0, 61)
mass_spectra = [np.sort(np.linalg.eigvals(mode_hamiltonian(x, {1: 1.0})).real)
                for x in mass_values]
difference = np.max(np.abs(np.array(plus_spectra) - np.array(minus_spectra)))
report("largest difference between the spectra of m = 2 and m = -2", f"{difference:.1e}")
check(difference < 1e-10, "numerical: the spectra of h_m and h_-m coincide")
```

The sorted eigenvalues of h are computed for $m = +2$ and $m = -2$ at 41 values of k_1 from 0 to 4, and at $k_1 = 1$ for 61 masses from -3 to 3. The largest difference between the spectra of $+2$ and -2 is printed as `0.0e+00`: identical.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0))
axes[0].plot(k1_values, np.array(plus_spectra)[:, [0, 15]], color=BLUE, linewidth=2)
axes[0].plot(k1_values, np.array(minus_spectra)[:, [0, 15]], "o", color=ORANGE,
             markersize=5, fillstyle="none")
axes[0].plot([], [], color=BLUE, linewidth=2, label="$m = +2$ (line)")
axes[0].plot([], [], "o", color=ORANGE, fillstyle="none", label="$m = -2$ (circles)")
```

`[:, [0, 15]]` picks the smallest and the largest eigenvalue, $-w$ and $+w$ (the other fourteen lie on top of these two). The spectra of $+2$ are drawn as lines, those of -2 as open circles (`fillstyle="none"`). The two calls with empty lists draw nothing; they only create the entries of the legend.

```
axes[0].set_xlabel("momentum $k_1$ (with $m = \\pm 2$)")
axes[0].set_ylabel("eigenvalues $\\pm w$ of $h$")
axes[0].set_title("same spectrum for $+m$ and $-m$")
axes[0].legend(loc="center left")
axes[1].plot(mass_values, np.array(mass_spectra)[:, [0, 15]], color=GREEN,
             linewidth=2)
axes[1].set_xlabel("mass $m$ (with $k_1 = 1$)")
axes[1].set_ylabel("eigenvalues $\\pm\\sqrt{m^2 + k_1^2}$ of $h$")
axes[1].set_title("the spectrum is even in $m$")
save_figure(fig, "plus_minus_mass_spectra",
...)
```

Figure 6 of Notebook 10a shows on the left the circles of $m = -2$ sitting exactly on the lines of $m = +2$, and on the right the eigenvalues $\pm\sqrt{m^2 + 1}$ against the mass: curves that are mirror images in the vertical axis $m = 0$. The one-particle spectra of the universes of masses $+m$ and $-m$ are identical, not opposite.

In [18], the last check.

```
for name in ("10a_1_gamma4_c_and_b.png", "10a_2_frequency_squared.png",
             "10a_3_eigenvalue_flow.png", "10a_4_krein_inertia_scan.png",
             "10a_5_krein_gram_matrices.png", "10a_6_plus_minus_mass_spectra.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The loop checks that each of the six figure files exists in the figure folder, and `all_checks_passed()` prints the last line, ALL 28 CHECKS PASSED (notebook 10a): the six figure checks of this cell and the 22 checks of the cells before.

10.11 Quantum states, operators and fermion modes from zero

Sections 10.2 to 10.6 treated one wave as a column u of 16 complex numbers. A quantum field is something else: its components become *operators* that act on *quantum states*. This section introduces the few notions that are needed, from zero, with small worked examples; Notebook 10b then builds all of them in Python.

States and the inner product. In the examples of this chapter a **quantum state** is a column $\psi = (\psi_1, \ldots, \psi_n)$ of n complex numbers. The **inner product** of two states ϕ and ψ is the complex number

$$\langle \phi | \psi \rangle = \sum_{j=1}^{n} \phi_j^* \psi_j .$$

For $\phi = \psi$ every term is $\psi_j^* \psi_j = |\psi_j|^2 \geq 0$, so $\langle \psi | \psi \rangle > 0$ for every state ψ that is not the zero column: the inner product is **positive**. The number $\|\psi\| = \sqrt{\langle \psi | \psi \rangle}$ is the **length** of ψ, and a state of length 1 is **normalised**. A space of states with a positive inner product is a **Hilbert space**. Positivity is what quantum mechanics needs for probabilities: for normalised ϕ and ψ the number $|\langle \phi | \psi \rangle|^2$ is the probability of finding ϕ in the state ψ, and a probability cannot be negative.

Operators and the Hilbert adjoint. An **operator** X maps every state to a state and respects sums and multiples, $X(a\phi + b\psi) = aX\phi + bX\psi$; here it is a matrix. The **Hilbert adjoint** of X is the operator X^* with

$$\langle \phi | X\psi \rangle = \langle X^* \phi | \psi \rangle \quad \text{for all states } \phi, \psi .$$

For the inner product above X^* is the conjugate transpose of the matrix X: $\langle \phi | X\psi \rangle = \phi^\dagger X \psi = (X^\dagger \phi)^\dagger \psi$. In this chapter the star on an operator always means the Hilbert adjoint, and the dagger on a field operator is kept for the *canonical conjugate* of Section 10.12; for numbers, matrices and columns the star and the dagger keep their meaning of Section 10.1 (complex conjugate, conjugate transpose). One consequence of the definition is used again and again. Put $\psi = X^* \phi$:

$$\langle \phi | X X^* \phi \rangle = \langle X^* \phi | X^* \phi \rangle = \|X^* \phi\|^2 \geq 0 .$$

The first step is the definition of X^* with $\psi = X^* \phi$; the second is the definition of the length. An operator with $X^* = X$ is **Hermitian**; its expectation values $\langle \phi | X \phi \rangle$ are real.

Commutator, anticommutator and the Heisenberg equation. For two operators, $[X, Y] = XY - YX$ is the **commutator** and $\{X, Y\} = XY + YX$ the **anticommutator**. We shall need the identity

$$[XY, Z] = X\{Y, Z\} - \{X, Z\}Y .$$

Proof: the right side is $X(YZ+ZY)-(XZ+ZX)Y = XYZ+XZY-XZY-ZXY = XYZ-ZXY$, by multiplying out and cancelling XZY, and $XYZ - ZXY$ is $[XY, Z]$ by the definition of the commutator. The energy operator H (the **Hamiltonian**) moves the system in time. In the Heisenberg picture the states stay fixed and an operator O changes with the time x_4 as $O(x_4) = e^{iHx_4} O e^{-iHx_4}$. Differentiating with the product rule, and using that H commutes with $e^{\pm iHx_4}$ (both are power series in H), gives the **Heisenberg equation**

$$\frac{dO}{dx_4} = iH\,O(x_4) - i\,O(x_4)H = i\,[H, O(x_4)].$$

One fermion mode and the Pauli principle. Let b be an operator with

$$\{b, b^*\} = 1, \qquad b\,b = 0.$$

Then also $b^*b^* = (bb)^* = 0$. The **number operator** $N = b^*b$ obeys

$$NN = b^*bb^*b = b^*(1 - b^*b)b = b^*b - b^*b^*bb = N.$$

The second step replaces bb^* by $1 - b^*b$ (the anticommutator); the third multiplies out; the fourth uses $b^*b^* = 0$. If $Nv = nv$ for a state $v \neq 0$, then $nv = Nv = NNv = n^2v$, so $n^2 = n$: $n = 0$ or $n = 1$. A fermion mode is either empty or occupied once. This is the **Pauli principle**, and it follows from the anticommutator alone. Example: on the two states $|0\rangle = (1, 0)$ (empty) and $|1\rangle = (0, 1)$ (occupied) the matrices

$$b = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad b^* = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$$

give $bb^* = \mathrm{diag}(1, 0)$ and $b^*b = \mathrm{diag}(0, 1)$, whose sum is the identity; $b|1\rangle = |0\rangle$ (it **annihilates** the fermion), $b^*|0\rangle = |1\rangle$ (it **creates** one), and $b^*|1\rangle = 0$: a second fermion cannot be created.

Many modes, patterns and the sign. With n modes $f_0, \ldots, f_{n-1}$ the rules are

$$\{f_p, f_q^*\} = \delta_{pq}, \qquad \{f_p, f_q\} = 0,$$

where δ_{pq} is 1 for $p = q$ and 0 otherwise. A basis state is a **pattern** of occupations $(o_0, o_1, \ldots, o_{n-1})$, each o_p being 0 (empty) or 1 (occupied); there are 2^n patterns. The pattern with every mode empty is the **vacuum** $|0\rangle$. The **Fock space** is the set of all superpositions of patterns, with the positive inner product in which the patterns are orthonormal. The operators act on a pattern as follows:

- f_p: if mode p is occupied it empties it and multiplies by $(-1)^s$, where s is the number of occupied modes below p (the modes $0, \ldots, p-1$); if mode p is empty the result is 0;
- f_p^*: if mode p is empty it fills it and multiplies by the same $(-1)^s$; if it is occupied the result is 0.

Why the sign: take two modes and write a pattern as $|o_0 o_1\rangle$. Then $f_0|11\rangle = |01\rangle$ (no occupied mode below 0), and $f_1|01\rangle = |00\rangle$ (mode 0 is empty), so $f_1 f_0|11\rangle = |00\rangle$. In the other order $f_1|11\rangle = -|10\rangle$ (one occupied mode, mode 0, lies below mode 1), and $f_0|10\rangle = |00\rangle$, so $f_0 f_1|11\rangle = -|00\rangle$. The sum is $\{f_0, f_1\}|11\rangle = 0$, as the rules require; without the sign it would be $2|00\rangle$. The rule $\{f_p, f_p^*\} = 1$ holds on every pattern: if mode p is empty, f_p^* fills it with a sign $(-1)^s$, f_p empties it again with the same sign (the modes below p have not changed), and $f_p^* f_p$ gives 0; the sum is $(-1)^{2s} = 1$ times the pattern. If mode p is occupied, the roles of the two products are exchanged.

A pattern as one whole number. The notebooks store the pattern $(o_0, \ldots, o_{n-1})$ as the whole number $n_{\text{pattern}} = \sum_p o_p 2^p$, whose binary digit number p (counted from 0, the last digit first) is o_p. For example, with modes 0 and 3 occupied the number is $1 + 8 = 9$, binary 1001. The number of occupied modes below p is the number of digits 1 among the binary digits $0, \ldots, p-1$. A state is stored as a Python dictionary that pairs every pattern of nonzero amplitude with its amplitude; the vacuum is the dictionary with the pattern 0 and the amplitude 1. With 16 modes there are $2^{16} = 65536$ patterns, but a state is stored by the few patterns it contains.

10.12 The canonical rule from the Lagrangian

To **quantise** dirac16complex means to replace its 16 components Ψ_A by operators and to fix their anticommutators. *Canonical quantisation* reads the rule off the Lagrangian, with the time x_4 as the evolution time. The slices $x_4 =$ const are seven-dimensional, with the coordinates $x_1, x_2, x_3, x_5, x_6, x_7, x_0$.

The time-derivative term, line by line. In the author's metric the Lagrangian density of dirac16complex is (Chapter 7, record `Revision/theory/field-theory.json`)

$$\mathcal{L} = \cos z \left[\tfrac{1}{2} \sum_a f_a^{-1} \big(\bar{\Psi} \gamma^{(x_a)} \partial_a \Psi - \partial_a \bar{\Psi} \gamma^{(x_a)} \Psi \big) - mS - U(S) \right],$$

with $\sqrt{|g|} = \cos z$ (on the backgrounds $b_4 = 1$ of the record, on which this chapter works; $b_4^{3/2} \cos z$ for the general metric of Chapter 3), the scale factors f_a of Section 10.3 ($f_4 = 1$), $\bar{\Psi} = \Psi^\dagger C$ and $S = \bar{\Psi}\Psi$; in this metric the spin connection drops out of the Lagrangian (`wolfram-field-theory.json`, check `L_spin_connection_drops_out_G`). The only terms with a derivative in x_4 are those with $a = 4$:

$$\tfrac{1}{2} \cos z \, \big(\Psi^\dagger C \gamma^{(x_4)} \partial_4 \Psi - \partial_4 \Psi^\dagger C \gamma^{(x_4)} \Psi \big).$$

This line writes out $\bar{\Psi} = \Psi^\dagger C$. Now $C\gamma^{(x_4)} = iB$, because the definition $B = -iC\gamma^{(x_4)}$ multiplied by i gives $iB = C\gamma^{(x_4)}$. Insert it:

$$\tfrac{i}{2} \cos z \, \big(\Psi^\dagger B \, \partial_4 \Psi - \partial_4 \Psi^\dagger B \, \Psi \big).$$

Add the total derivative $\frac{i}{2} \partial_4 (\cos z \, \Psi^\dagger B \Psi)$. A total derivative does not change the field equations (its integral over the time depends only on the end values, which the variation keeps fixed; Chapter 7). Since $\cos z$ does not depend on x_4, the product rule gives $\frac{i}{2} \partial_4 (\cos z \, \Psi^\dagger B \Psi) = \frac{i}{2} \cos z \, (\partial_4 \Psi^\dagger B \Psi + \Psi^\dagger B \partial_4 \Psi)$. Added to the line above, the two terms with $\partial_4 \Psi^\dagger$ cancel and the two with $\partial_4 \Psi$ add:

$$i\cos z\,\Psi^\dagger B\,\partial_4\Psi = \Psi^\dagger K\,\partial_4\Psi, \qquad K = i\cos z\,B.$$

The rest of the Lagrangian has no x_4-derivative; the Revision record writes it as $-\mathcal{H}$ with the **Hamiltonian density** $\mathcal{H} = \cos z\,[\Psi^\dagger(mC - \sum_{\mu\neq x_4} C\gamma^\mu D_\mu)\Psi + U(S)]$, so that $\mathcal{L} = \Psi^\dagger K\partial_4\Psi - \mathcal{H}$ exactly, up to the total derivative (`python-field-theory.json`, check `hamiltonian_form_and_heisenberg_equation`). In words: the time-derivative term contains the matrix B, and the momentum conjugate to Ψ_A is $\pi_A = i\cos z\,(\Psi^\dagger B)_A$, which depends on $\Psi^\dagger$ only, as for Dirac's electron (`python-field-theory.json`, check `canonical_momentum`).

The rule, line by line. Take a Lagrangian of the form $\Psi^\dagger K\partial_4\Psi - \Psi^\dagger h'\Psi$ with constant matrices K and h' (one point of the slice; the derivatives along the slice are inside h'). Varying $\Psi^\dagger$ gives the classical equation

$$K\,\partial_4\Psi = h'\Psi, \qquad \text{that is} \qquad \partial_4\Psi = K^{-1}h'\Psi.$$

In the quantum theory suppose that the operators obey $\{\Psi_A, \Psi_C^\dagger\} = A_{AC}$ with an unknown matrix A, and $\{\Psi_A, \Psi_C\} = 0$. The energy operator is $H = \Psi^\dagger h'\Psi = \sum_{A,C}\Psi_A^\dagger h'_{AC}\Psi_C$. For one component Ψ_c:

$$[H,\Psi_c] = \sum_{A,C} h'_{AC}\,[\Psi_A^\dagger\Psi_C, \Psi_c] = \sum_{A,C} h'_{AC}\big(\Psi_A^\dagger\{\Psi_C,\Psi_c\} - \{\Psi_A^\dagger,\Psi_c\}\Psi_C\big).$$

The first step takes the numbers h'_{AC} out of the commutator; the second is the identity $[XY,Z] = X\{Y,Z\} - \{X,Z\}Y$ of Section 10.11 with $X = \Psi_A^\dagger$, $Y = \Psi_C$, $Z = \Psi_c$. Now $\{\Psi_C,\Psi_c\} = 0$ and $\{\Psi_A^\dagger,\Psi_c\} = \{\Psi_c,\Psi_A^\dagger\} = A_{cA}$, so

$$[H,\Psi_c] = -\sum_{A,C} A_{cA}h'_{AC}\Psi_C = -(Ah'\Psi)_c,$$

by the definition of a matrix product. The Heisenberg equation then reads

$$\partial_4\Psi = i[H,\Psi] = -iAh'\Psi.$$

It agrees with the classical equation $\partial_4\Psi = K^{-1}h'\Psi$ for every h' exactly when $-iA = K^{-1}$, that is $A = iK^{-1}$. With $K = i\cos z\,B$, $B^{-1} = B$ (Section 10.2) and $1/i = -i$:

$$iK^{-1} = i\cdot\frac{1}{i\cos z}\,B^{-1} = \frac{B}{\cos z}.$$

From one point to the whole slice. So far the slice was a single point. To pass to the whole slice, divide it into many small cells, each with the coordinate volume ΔV (the product of its seven coordinate widths), and keep one value $\Psi(x)$ of the field per cell x. The Lagrangian of the slice is then the sum over the cells of ΔV times the density, so the time-derivative term of the cell x is $\Psi(x)^\dagger(K\Delta V)\,\partial_4\Psi(x)$, with $K = i\cos z\,B$ at the position of that cell; this term couples no two different cells (the derivatives along the slice, which do couple neighbouring cells, sit in h'). The rule just derived holds for any number of components: take as the components all 16 components of

all cells. Their time-derivative kernel is block diagonal, with one block $K\Delta V$ per cell, so its inverse is block diagonal with the blocks $(K\Delta V)^{-1}$, and the rule $A = iK^{-1}$ gives

$$\{\Psi_A(x), \Psi_C^\dagger(y)\} = i\,(K\Delta V)^{-1}_{AC}\,\delta_{xy} = B_{AC}\,\frac{\delta_{xy}}{\cos z\,\Delta V},$$

where δ_{xy} is 1 for the same cell and 0 for two different cells: components of different cells anticommute. The second step is $iK^{-1} = B/\cos z$ from above, divided by ΔV. As the cells shrink, $\delta_{xy}/\Delta V$ becomes the seven-dimensional **delta function** $\delta^7(x-y)$. It is defined by what it does in an integral: it is zero for $x \neq y$, and for every smooth function f on the slice

$$\int f(y)\,\delta^7(x-y)\,d^7y = f(x),$$

just as the sum over the cells $\sum_y \Delta V\, f(y)\,\delta_{xy}/\Delta V = f(x)$ picks out the cell x. It is not an ordinary function (no ordinary function is zero everywhere but at one point and still has the integral 1) but a rule for integrals, and in the theory it only appears inside such integrals. So the **canonical anticommutator** of dirac16complex on a slice $x_4 = \text{const}$ is

$$\{\Psi_A(x), \Psi_C^\dagger(y)\} = B_{AC}\,\frac{\delta^7(x-y)}{\cos z}.$$

At one point the factor $1/\cos z$ is a positive number, which changes no sign; the notebooks therefore work with $\{\Psi_A, \Psi_C^\dagger\} = B_{AC}$.

The Heisenberg equation is the wave equation. For a plane wave with frame momenta k_a along the slice, at one point with frozen coefficients, the derivatives D_a in $\mathcal{H}$ become ik_a and

$$h' = mC - i\sum_{a\neq 4} k_a\, C\gamma^{(x_a)}.$$

Multiply by $B = -iC\gamma^{(x_4)}$:

$$Bh' = -im\,C\gamma^{(x_4)}C - \sum_{a\neq 4} k_a\, C\gamma^{(x_4)}C\gamma^{(x_a)} = -im\gamma^{(x_4)} - \gamma^{(x_4)}\sum_{a\neq 4} k_a\gamma^{(x_a)} = h.$$

The first step multiplies out $((-i)(-i) = -1)$; the second moves C through $\gamma^{(x_4)}$ (they commute, (B1) of Section 10.2) and uses $CC = I_{16}$; the result is the mode Hamiltonian h of Section 10.3. With $A = B$ the Heisenberg equation is $\partial_4\Psi = -iBh'\Psi = -ih\Psi$, that is $i\,\partial_4\Psi = h\Psi$: the quantum field obeys the classical wave equation of the field.

Why anticommutators. The rule uses anticommutators because dirac16complex is a fermion field by the author's definition: its components are anticommuting numbers. In ordinary 3+1 dimensional physics this choice is not free: the **spin-statistics theorem** of quantum field theory shows that, if the energy is to be bounded below and measurements at two points that no signal can connect are not to disturb each other, fields of half-integer spin (such as the electron's) must be quantised with anticommutators and fields of whole-number spin with commutators. No such theorem is proved for signature (4,4); here the anticommutator is an ASSUMPTION, part of the definition of the field.

statement	status	where it is verified
the time-derivative term is $\Psi^\dagger K \partial_4 \Psi$ with $K = i\cos z\, B$; $\pi_A = i\cos z\,(\Psi^\dagger B)_A$	PROVED	`wolfram-field-theory.json`, checks `canonical_momentum` and `first_order_form_and_anticommutator`; `python-field-theory.json`, check `canonical_momentum`
$\{\Psi_A(x), \Psi^\dagger_C(y)\} = B_{AC}\,\delta^7(x-y)/\cos z$	PROVED	`python-field-theory.json`, check `canonical_anticommutator_B`; `python-pairing.json`, check `Q.canonical_anticommutator`
the Heisenberg equation reproduces the field equation (general U)	PROVED	`wolfram-field-theory.json`, check `Heisenberg_equation_reproduces_field_equation`; `python-field-theory.json`, check `hamiltonian_form_and_heisenberg_equation`
anticommutators (not commutators) for dirac16complex	ASSUMED	part of the definition of the field as a fermion field; no spin-statistics theorem for signature (4,4) is proved

10.13 The canonical conjugate is not the Hilbert adjoint: the Krein space

In ordinary quantum theory the canonical conjugate $\Psi^\dagger$ is the Hilbert adjoint of Ψ. In signature (4,4) this is impossible.

Theorem 10.2 (the canonical rule forces a Krein space). There is no space of states with a positive inner product on which operators Ψ_A obey $\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$ with $\Psi^\dagger_A$ the Hilbert adjoint of Ψ_A. (PROVED: `python-field-theory.json` and `wolfram-field-theory.json`, check `no_positive_inner_product`.)

Proof. Take the column u with $u_7 = -i/\sqrt{2}$, $u_{16} = 1/\sqrt{2}$ and all other entries 0 (the column named by the record). By the table of B in Section 10.2, $(Bu)_7 = B_{7,16}u_{16} = i/\sqrt{2} = -u_7$ and $(Bu)_{16} = B_{16,7}u_7 = (-i)(-i/\sqrt{2}) = -1/\sqrt{2} = -u_{16}$, so $Bu = -u$; and $u^\dagger u = \frac{1}{2} + \frac{1}{2} = 1$, so $u^\dagger B u = -u^\dagger u = -1$. Define the operator $X = \sum_A u^*_A \Psi_A$ and its canonical conjugate $X^\dagger = \sum_A u_A \Psi^\dagger_A$. Then

$$\{X, X^\dagger\} = \sum_{A,C} u^*_A u_C \{\Psi_A, \Psi^\dagger_C\} = \sum_{A,C} u^*_A B_{AC} u_C = u^\dagger B u = -1.$$

The first step takes the numbers out of the anticommutator; the second is the canonical rule; the third is the definition of $u^\dagger B u$. Suppose now that $X^\dagger$ were the Hilbert adjoint X^*. Then for every normalised state ϕ, by the consequence of the definition of the adjoint in Section 10.11,

$$\langle\phi|\{X,X^*\}\phi\rangle = \langle\phi|XX^*\phi\rangle + \langle\phi|X^*X\phi\rangle = \|X^*\phi\|^2 + \|X\phi\|^2 \geq 0,$$

while $\{X,X^*\} = -1$ gives $\langle\phi|(-1)\phi\rangle = -1$. A number cannot be both ≥ 0 and -1. QED.

What remains: a Krein space. The canonical rule itself is consistent; what fails is only its reading with a positive inner product. A **Krein space** is a space with a Hermitian form $[u,v]$ that is *nondegenerate* (no nonzero u has $[u,v]=0$ for all v) but *indefinite* (it takes both signs), together with a matrix J, the **fundamental symmetry**, with $J^\dagger = J = J^{-1}$, such that $[u,Jv]$ is a positive inner product. For the one-particle columns of dirac16complex, $[u,v] = u^\dagger Bv$ and $J = B$: indeed $[u,Bv] = u^\dagger BBv = u^\dagger v$, the ordinary positive inner product. The theorem says that the canonical rule turns the state space of the field into a Krein space with fundamental symmetry B, whenever $\Psi^\dagger$ is read as the adjoint.

10.14 Two realisations and the expectation-value rule

The Revision record describes two honest ways to realise the canonical rule on a concrete space of states.

(a) The positive realisation. Keep a POSITIVE Fock space, with 16 modes $f_0,\ldots,f_{15}$, and let $\Psi_A = f_{A-1}$ and $\chi_A = f^*_{A-1}$, the Hilbert adjoint. Then *define* the canonical conjugate as

$$\Psi^\dagger_A = \sum_C \chi_C B_{CA}, \qquad \text{briefly} \qquad \Psi^\dagger = \chi B.$$

Line by line,

$$\{\Psi_A, \Psi^\dagger_C\} = \sum_D \{\Psi_A,\chi_D\}B_{DC} = \sum_D \delta_{AD}B_{DC} = B_{AC}.$$

The first step takes the numbers B_{DC} out of the anticommutator; the second is the Fock rule $\{f_{A-1}, f^*_{D-1}\} = \delta_{AD}$; the third keeps the one term $D = A$. So the canonical rule holds on a positive space, but $\Psi^\dagger$ is not the Hilbert adjoint: for the column u of Theorem 10.2,

$$X^\dagger = \sum_A u_A \Psi^\dagger_A = \sum_{A,C} u_A\chi_C B_{CA} = \sum_C (Bu)_C\chi_C = -\sum_C u_C\chi_C = -X^*,$$

where the third step collects $\sum_A B_{CA}u_A = (Bu)_C$, the fourth uses $Bu = -u$, and the last recognises $X^* = \sum_C u_C\chi_C$, the Hilbert adjoint of $X = \sum_C u^*_C\Psi_C$. So $\{X,X^*\} = +1$ (positive, as in every Hilbert space) while $\{X,X^\dagger\} = -1$ (the canonical value): the two differ by a sign. This is the form the Revision record uses for the good sector (Section 10.20).

(b) The Krein-Fock realisation. Keep $\Psi^\dagger$ as the conjugate and let the modes carry Krein norms ± 1. Choose 16 columns $u_1,\ldots,u_{16}$ with $u^\dagger_n Bu_{n'} = \epsilon_n\delta_{nn'}$, $\epsilon_n = +1$ or -1 (*Krein-orthonormal* modes). Write U for the matrix with these columns and $E = \mathrm{diag}(\epsilon_1,\ldots,\epsilon_{16})$, so that $U^\dagger BU = E$. Then also $UEU^\dagger = B$: multiply $U^\dagger BU = E$ from the left by UE, which gives $UEU^\dagger BU = UEE = U$ (because $EE = I_{16}$); multiply from the right by U^{-1}: $UEU^\dagger B = I_{16}$; multiply from the right by B:

$UEU^\dagger = B$ (because $BB = I_{16}$). Take operators b_n with $\{b_n, b^\dagger_{n'}\} = \epsilon_n \delta_{nn'}$ and put $\Psi = \sum_n u_n b_n$, $\Psi^\dagger = \sum_n u^\dagger_n b^\dagger_n$. Then

$$\{\Psi_A, \Psi^\dagger_C\} = \sum_{n,n'} (u_n)_A (u_{n'})^*_C \{b_n, b^\dagger_{n'}\} = \sum_n \epsilon_n (u_n)_A (u_n)^*_C = (UEU^\dagger)_{AC} = B_{AC} :$$

the same canonical rule. The simplest choice takes 8 eigenvectors of B with eigenvalue $+1$ ($\epsilon = +1$) and 8 with eigenvalue -1 ($\epsilon = -1$). A **Krein boost** of a pair, $u'_1 = \cosh t\, u_1 + \sinh t\, u_9$ and $u'_9 = \sinh t\, u_1 + \cosh t\, u_9$ (with $Bu_1 = u_1$, $Bu_9 = -u_9$, both of length 1 and orthogonal), keeps the Krein norms, because $\cosh^2 t - \sinh^2 t = 1$, and keeps the two modes Krein-orthogonal, because $\cosh t \sinh t - \sinh t \cosh t = 0$; but it changes the ordinary squared length to $\cosh^2 t + \sinh^2 t$. The record uses $\cosh t = \frac{5}{4}$, $\sinh t = \frac{3}{4}$ ($\frac{25}{16} - \frac{9}{16} = 1$), so the boosted mode has the ordinary squared length $\frac{25}{16} + \frac{9}{16} = \frac{34}{16} = 2.125$.

The expectation-value rule, line by line. In the one-mode state $b^\dagger_n|0\rangle$ (with $b_{n'}|0\rangle = 0$ for all n') the observable $\Psi^\dagger M \Psi = \sum_{k,k'} (u^\dagger_k M u_{k'})\, b^\dagger_k b_{k'}$ has the vacuum value

$$\langle 0|b_n (\Psi^\dagger M \Psi) b^\dagger_n|0\rangle = \sum_{k,k'} (u^\dagger_k M u_{k'}) \langle 0|b_n b^\dagger_k b_{k'} b^\dagger_n|0\rangle = \sum_{k,k'} (u^\dagger_k M u_{k'})\, \epsilon_n \delta_{k'n}\, \epsilon_n \delta_{kn} = u^\dagger_n M u_n .$$

The first step inserts the expansion; the second uses $b_{k'} b^\dagger_n|0\rangle = \{b_{k'}, b^\dagger_n\}|0\rangle = \epsilon_n \delta_{k'n}|0\rangle$ (the term $b^\dagger_n b_{k'}|0\rangle$ vanishes) and in the same way $\langle 0|b_n b^\dagger_k|0\rangle = \epsilon_n \delta_{kn}$; the third uses $\epsilon_n^2 = 1$. The Krein norm of the state is $\langle 0|b_n b^\dagger_n|0\rangle = \epsilon_n$, so the **normalised** expectation value is

$$\frac{\langle 0|b_n (\Psi^\dagger M \Psi) b^\dagger_n|0\rangle}{\langle 0|b_n b^\dagger_n|0\rangle} = \epsilon_n\, u^\dagger_n M u_n .$$

For a mode that is an eigenvector of B, $Bu_n = \epsilon_n u_n$, the conjugate transpose gives $u^\dagger_n B = \epsilon_n u^\dagger_n$ (B is Hermitian and ϵ_n real), so $\epsilon_n u^\dagger_n M u_n = u^\dagger_n B M u_n$, the form in which the Revision specification states the rule. For a Krein-boosted mode $Bu_n \neq \epsilon_n u_n$, and the form $u^\dagger_n B M u_n$ is wrong; the general rule is $\epsilon_n u^\dagger_n M u_n$ (`python-field-theory.json`, check `expectation_value_rule`).

10.15 Maps that keep the canonical rule, and the conjugation of the quantised field

Linear maps. If $\Psi' = M\Psi$ with a constant matrix M, the conjugate is $\Psi'^\dagger = \Psi^\dagger M^\dagger$, that is $\Psi'^\dagger_C = \sum_F \Psi^\dagger_F M^*_{CF}$, and

$$\{\Psi'_A, \Psi'^\dagger_C\} = \sum_{D,F} M_{AD} M^*_{CF} \{\Psi_D, \Psi^\dagger_F\} = \sum_{D,F} M_{AD} B_{DF} M^*_{CF} = (MBM^\dagger)_{AC} .$$

The map keeps the canonical rule exactly when $MBM^\dagger = B$. The Revision pairing record lists $MBM^\dagger = \sigma B$ with a sign σ for 17 maps (`Revision/pairing/pairing-theory.json`, data `Krein_signs_M_B_Mdagger`; `wolfram-pairing.json`, check `Q_Krein_metric_of_images`). The signs follow from Section 10.2 in three lines. For a gamma, $(\gamma^{(x_a)})^\dagger = (\gamma^{(x_a)})^T = \eta_{aa} \gamma^{(x_a)}$ (the

gammas are real; the space-like ones are symmetric, the time-like ones antisymmetric) and $\gamma^{(x_a)}B = s_a B\gamma^{(x_a)}$ with $s_a = +1$ for $a = 1,2,3,4,8$ and $s_a = -1$ for $a = 5,6,7$; so

$$\gamma^{(x_a)}B(\gamma^{(x_a)})^\dagger = \eta_{aa}\,\gamma^{(x_a)}B\gamma^{(x_a)} = \eta_{aa}s_a\,B\gamma^{(x_a)}\gamma^{(x_a)} = \eta_{aa}s_a\eta_{aa}\,B = s_aB.$$

For the chirality, Γ is real and symmetric and $\Gamma B\Gamma = -B$ (Section 10.2), so $\Gamma B\Gamma^\dagger = -B$. For the reflection $P_a = \Gamma\gamma^{(x_a)}$ the two signs multiply: $\sigma = -s_a$. The table:

map M	**sign** σ **in** $MBM^\dagger = \sigma B$
Γ	-1
$\gamma^{(x_1)}, \gamma^{(x_2)}, \gamma^{(x_3)}, \gamma^{(x_4)}, \gamma^{(x_0)}$	$+1$
$\gamma^{(x_5)}, \gamma^{(x_6)}, \gamma^{(x_7)}$	-1
P_1, P_2, P_3, P_4, P_0	-1
P_5, P_6, P_7	$+1$

The chirality image $\Gamma\Psi$ of the pairing theorem T1 therefore carries the Krein metric $-B$ (statement Q1 of the quantum reading of the pairing record, Section 10.44), while the mirror image $\gamma^{(x_0)}\Psi$ of theorem T2 keeps $+B$ (statement Q5).

The conjugation of the quantised field. Charge conjugation is a MATRIX map (Chapter 5): the two charge-conjugation matrices are $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$, and for a classical field they give $\Psi^c = \mathcal{C}_+\bar\Psi^T = CC\Psi^* = \Psi^*$ (same mass) and $\Psi^c = \mathcal{C}_-\bar\Psi^T = \Gamma\Psi^*$ (mass reversed), since $\bar\Psi^T = (\Psi^\dagger C)^T = C^T\Psi^* = C\Psi^*$. (For a REAL field $\Psi^* = \Psi$, so the first map is the identity: a real field is its own conjugate, and plain complex conjugation does nothing to it; the nontrivial real map is Γ, Chapter 5.) For the QUANTISED field the role of Ψ^* is played by the column of conjugate operators $\Psi^{\dagger T}$, and the candidate conjugations are $\Psi^c = M\Psi^{\dagger T}$, that is $\Psi^c_A = \sum_D M_{AD}\Psi^\dagger_D$. Its conjugate is $\Psi^{c\dagger}_C = \sum_F M^*_{CF}\Psi_F$ (the conjugate of $\Psi^\dagger$ is Ψ). Line by line,

$$\{\Psi^c_A, \Psi^{c\dagger}_C\} = \sum_{D,F} M_{AD}M^*_{CF}\{\Psi^\dagger_D, \Psi_F\} = \sum_{D,F} M_{AD}B_{FD}M^*_{CF} = (MB^TM^\dagger)_{AC}.$$

The second step uses $\{\Psi^\dagger_D, \Psi_F\} = \{\Psi_F, \Psi^\dagger_D\} = B_{FD}$, and the third $B_{FD} = (B^T)_{DF}$. Because B is Hermitian and purely imaginary, $B^T = (B^\dagger)^* = B^* = -B$. Hence:

- $M = I_{16}$ (the $\mathcal{C}_+$ type, $\Psi \to \Psi^{\dagger T}$): $MB^TM^\dagger = -B$, the sign of the canonical rule is REVERSED;
- $M = \Gamma$ (the $\mathcal{C}_-$ type, $\Psi \to \Gamma\Psi^{\dagger T}$): $\Gamma B^T\Gamma^\dagger = -\Gamma B\Gamma = +B$, the canonical rule is KEPT.

So the conjugation of the quantised field that preserves $\{\Psi, \Psi^\dagger\} = B\delta$ is $\Psi \to \Gamma\Psi^{\dagger T}$, and it reverses the mass: the gammas are real, so the conjugate of the field equation $\gamma^\mu\partial_\mu\Psi = m\Psi$ (flat space, $U = 0$) is $\gamma^\mu\partial_\mu\Psi^* = m\Psi^*$, and since Γ anticommutes with every gamma, $\gamma^\mu\partial_\mu(\Gamma\Psi^*) = -\Gamma\gamma^\mu\partial_\mu\Psi^* = -m\,\Gamma\Psi^*$: the conjugated field obeys the equation of mass $-m$ (`charge-conjugation-and-u1.json`, check `quantum_charge_conjugation_unitary_type`). Chapter 21 uses this result for matter and antimatter.

statement	status	where it is verified
Theorem 10.2: no positive inner product with $\Psi^\dagger$ the adjoint	PROVED	`python-field-theory.json` and `wolfram-field-theory.json`, check `no_positive_inner_product`
positive realisation $\Psi^\dagger = \chi B$ gives $\{\Psi, \Psi^\dagger\} = B$	PROVED	`python-field-theory.json`, check `good_sector_positive_fock_realisation`; Notebook 10b on a Fock space
expectation-value rule $\epsilon_n u_n^\dagger M u_n$; $u_n^\dagger B M u_n$ only for eigenvectors of B	PROVED	`python-field-theory.json`, check `expectation_value_rule`
the 17 signs σ of $MBM^\dagger = \sigma B$	PROVED	`pairing-theory.json`, data `Krein_signs_M_B_Mdagger`; `wolfram-pairing.json`, check `Q_Krein_metric_of_images`; `python-pairing.json`, check `compare.theory.krein_signs`
$\Psi \to \Gamma \Psi^{\dagger T}$ keeps B and reverses the mass; $M = I_{16}$ gives $-B$	PROVED	`charge-conjugation-and-u1.json`, check `quantum_charge_conjugation_unitary_type`

10.16 Example: Notebook 10b builds the canonical rule on a Fock space

Notebook 10b repeats the derivation of $iK^{-1} = B/\cos z$ exactly with sympy, builds the fermionic Fock space of Section 10.11 in a few lines of Python, realises on it the canonical rule $\{\Psi_A, \Psi_C^\dagger\} = B_{AC}$ with $\Psi^\dagger = \chi B$ (Section 10.14), checks that the Heisenberg equation gives the classical wave equation (Section 10.12), checks Theorem 10.2 on the recorded column and makes the positivity visible for 300 random states, builds the Krein-Fock realisation with a Krein-boosted pair and checks the expectation-value rule for 60 random matrices, and reproduces the 17 signs of Section 10.15 and the conjugation $\Psi \to \Gamma \Psi^{\dagger T}$. It draws five figures and ends with the line ALL 25 CHECKS PASSED (notebook 10b).

10.17 How to run Notebook 10b

Step 1. What this notebook does and what it needs.

Notebook 10b (The canonical anticommutator forces a Krein space) is the file `Revision/textbook/notebooks/10b_canonical_krein.ipynb` of the repository Dirac_claude. It derives the canonical anticommutator of dirac16complex from the time derivative term of the Lagrangian (exactly, with sympy), builds a fermionic Fock space of the 16 components in a few lines of Python and checks on it the anticommutator, the Heisenberg equation and the theorem that the canonical conjugate cannot be the Hilbert adjoint in a space with a positive inner product; it then builds the Krein-Fock realisation with modes of negative Krein norm, including a Krein-boosted pair, checks

the expectation-value rule, and computes which matrix maps keep the anticommutator: the Krein signs of the chirality, of the eight gammas and of the eight reflections, and the conjugation of the quantised field that preserves the anticommutator (the matrix Gamma, which reverses the mass); five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10b_canonical_krein.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10b_canonical_krein.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 10b_canonical_krein.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10b.captions.json`
- `Revision/textbook/figures/10b_1_positivity.png`
- `Revision/textbook/figures/10b_2_krein_norms_random.png`
- `Revision/textbook/figures/10b_3_krein_plane.png`
- `Revision/textbook/figures/10b_4_expectation_rule.png`
- `Revision/textbook/figures/10b_5_krein_signs.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10b_5_krein_signs.png exists
ALL 25 CHECKS PASSED (notebook 10b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10b_canonical_krein.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10b_canonical_krein.ipynb
python -m nbconvert --execute --inplace 10b_canonical_krein.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.18 Notebook 10b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10b_canonical_krein.ipynb`: every cell in order, with the output that the stored, executed notebook

holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

To *quantise* the field dirac16complex means to turn its 16 components $\Psi_A$
($A = 1, \dots, 16$) into operators on a space of states, with a rule for the
anticommutator $\{\Psi_A, \Psi^\dagger_C\} = \Psi_A\Psi^\dagger_C +
\Psi^\dagger_C\Psi_A$. This notebook

- derives that rule from the Lagrangian of the field (canonical quantisation with
  the time $x_4$): $\{\Psi_A(x), \Psi^\dagger_C(y)\} = B_{AC}\,\delta^7(x - y)/\cos z$
  with the matrix $B = -iC\gamma^{(x_4)}$;
- builds a *fermionic Fock space* (the quantum states of 16 fermion modes) in a few
  lines of Python, and checks on it the anticommutator and that the quantum
  equation of motion (the Heisenberg equation) gives back the classical wave
  equation of the field;
- checks the theorem that $\Psi^\dagger$ cannot be the ordinary (Hilbert) adjoint of
  $\Psi$ in a space whose inner product is positive: the state space must carry an
  indefinite inner product, a *Krein space*;
- builds the Krein-Fock realisation, with modes of negative Krein norm and a
  Krein-boosted pair, and checks the expectation-value rule of the Revision record;
- computes which matrix maps $\Psi \to M\Psi$ keep the anticommutator, and which
  conjugation of the quantised field keeps it: $\Psi \to \Gamma\Psi^{\dagger T}$
  with the chirality matrix $\Gamma$ (it reverses the mass); the map with $M = 1$
  would turn $B$ into $-B$;
- reproduces the corresponding checks of the Revision record and draws five
  teaching figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 10b (The canonical anticommutator forces a Krein space) is the file
‘Revision/textbook/notebooks/10b_canonical_krein.ipynb‘ of the repository Dirac_claude.
It derives the canonical anticommutator of dirac16complex from the time derivative term
of the Lagrangian (exactly, with sympy), builds a fermionic Fock space of the 16
components in a few lines of Python and checks on it the anticommutator, the Heisenberg
equation and the theorem that the canonical conjugate cannot be the Hilbert adjoint in a
space with a positive inner product; it then builds the Krein-Fock realisation with modes
of negative Krein norm, including a Krein-boosted pair, checks the expectation-value
rule, and computes which matrix maps keep the anticommutator: the Krein signs of the
chirality, of the eight gammas and of the eight reflections, and the conjugation of the
quantised field that preserves the anticommutator (the matrix Gamma, which reverses the
mass); five teaching figures. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
```

notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10b_canonical_krein.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10b_canonical_krein.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10b_canonical_krein.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10b.captions.json‘
- ‘Revision/textbook/figures/10b_1_positivity.png‘
- ‘Revision/textbook/figures/10b_2_krein_norms_random.png‘
- ‘Revision/textbook/figures/10b_3_krein_plane.png‘
- ‘Revision/textbook/figures/10b_4_expectation_rule.png‘
- ‘Revision/textbook/figures/10b_5_krein_signs.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10b_5_krein_signs.png exists
ALL 25 CHECKS PASSED (notebook 10b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10b_canonical_krein.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs).

Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 10b_canonical_krein.ipynb
python -m nbconvert --execute --inplace 10b_canonical_krein.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 10b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 10b (The canonical anticommutator forces a Krein space) is the file
# Revision/textbook/notebooks/10b_canonical_krein.ipynb of the repository Dirac_claude.
# It derives the canonical anticommutator of dirac16complex from the time derivative term
# of the Lagrangian (exactly, with sympy), builds a fermionic Fock space of the 16
# components in a few lines of Python and checks on it the anticommutator, the Heisenberg
# equation and the theorem that the canonical conjugate cannot be the Hilbert adjoint in
# a space with a positive inner product; it then builds the Krein-Fock realisation with
# modes of negative Krein norm, including a Krein-boosted pair, checks the
# expectation-value rule, and computes which matrix maps keep the anticommutator: the
# Krein signs of the chirality, of the eight gammas and of the eight reflections, and the
# conjugation of the quantised field that preserves the anticommutator (the matrix Gamma,
# which reverses the mass); five teaching figures. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
```

```
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
```

```
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
```

```
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10b_canonical_krein.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10b_canonical_krein.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10b_canonical_krein.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10b.captions.json
# - Revision/textbook/figures/10b_1_positivity.png
# - Revision/textbook/figures/10b_2_krein_norms_random.png
# - Revision/textbook/figures/10b_3_krein_plane.png
# - Revision/textbook/figures/10b_4_expectation_rule.png
# - Revision/textbook/figures/10b_5_krein_signs.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
```

```
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 10b_5_krein_signs.png exists
#     ALL 25 CHECKS PASSED (notebook 10b)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10b_canonical_krein.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10b_canonical_krein.ipynb
#     python -m nbconvert --execute --inplace 10b_canonical_krein.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
```

```
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10b"  # this notebook: chapter 10, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative
```

```
def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
```

```
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **State, inner product, Hilbert space**: a quantum state is a vector; the inner
  product $\langle\phi|\psi\rangle = \sum_n \phi_n^*\psi_n$ of two states is a
  complex number, and $\langle\psi|\psi\rangle > 0$ for every state $\psi \neq 0$
  (it is *positive*). A space of states with a positive inner product is a
  *Hilbert space*. Positivity is what makes probabilities possible.
- **Operator, Hilbert adjoint**: an operator maps states to states. The Hilbert
  adjoint $X^*$ of $X$ is the operator with $\langle\phi|X\psi\rangle =
  \langle X^*\phi|\psi\rangle$ for all states; for a matrix it is the conjugate
  transpose.
- **Anticommutator**: $\{X, Y\} = XY + YX$.
- **Fermion mode, occupation, vacuum**: a fermion mode is either empty or occupied
  once (the Pauli principle). With 16 modes a basis state is a *pattern* of 16
  occupations (0 or 1); there are $2^{16} = 65536$ patterns. The *vacuum* is the
  pattern with every mode empty. The annihilation operator $f_p$ empties mode $p$
  and the creation operator $f_p^*$ fills it, with a sign explained in Section 6.
- **Fock space**: all superpositions of the patterns, with the positive inner
  product in which the patterns are orthonormal.
- **Canonical conjugate**: the operator $\Psi^\dagger_A$ that the Lagrangian pairs
  with $\Psi_A$; the question of the notebook is whether it equals the Hilbert
  adjoint $\Psi_A^*$.
- **Krein form, Krein space, fundamental symmetry**: a Krein space is a space with an
  indefinite but nondegenerate Hermitian form, here $[u, v] = u^\dagger B v$, and a
```

```
  matrix $J$ (here $J = B$) with $J^\dagger = J = J^{-1}$ such that
  $[u, Jv] = u^\dagger v$ is positive. $J$ is the *fundamental symmetry*.
- **Krein-orthonormal modes**: columns $u_n$ with $u_n^\dagger B u_m = \epsilon_n
  \delta_{nm}$, $\epsilon_n = +1$ or $-1$. A *Krein boost* mixes a mode of Krein
  norm $+1$ with one of $-1$ by the numbers $\cosh t$ and $\sinh t$; it keeps the
  Krein form but changes the ordinary length.
- **Pattern as a whole number**: the program stores a pattern as one whole number
  $n$ whose binary digit number $p$ (counted from 0) is the occupation of mode $p$;
  `n >> p & 1` reads that digit.
```

Text cell:

```
## 4. The physical and mathematical situation

**The time-derivative term of the Lagrangian, line by line.** The Lagrangian of
dirac16complex in the author's metric (the Revision theory record) is
$\mathcal{L} = \cos z\,\big[\frac12\sum_a f_a^{-1}(\bar\Psi\gamma^{(x_a)}\partial_a
\Psi - \partial_a\bar\Psi\gamma^{(x_a)}\Psi) - mS - U(S)\big]$ with $z = 6Hx_0$,
$\bar\Psi = \Psi^\dagger C$ and $f_4 = 1$. Its only terms with a derivative in the
time $x_4$ are

$$\tfrac12\cos z\,\big(\Psi^\dagger C\gamma^{(x_4)}\partial_4\Psi -
\partial_4\Psi^\dagger C\gamma^{(x_4)}\Psi\big).$$

The notebook checks that $C\gamma^{(x_4)} = iB$ (this is the definition
$B = -iC\gamma^{(x_4)}$ multiplied by $i$). Insert it:

$$\tfrac{i}{2}\cos z\,\big(\Psi^\dagger B\,\partial_4\Psi - \partial_4\Psi^\dagger
B\,\Psi\big).$$

Add the total derivative $\frac{i}{2}\partial_4(\cos z\,\Psi^\dagger B\Psi)$, which
does not change the field equations; since $\cos z$ does not depend on $x_4$, the
product rule gives $\frac{i}{2}\cos z(\partial_4\Psi^\dagger B\Psi +
\Psi^\dagger B\partial_4\Psi)$, and the sum is

$$i\cos z\,\Psi^\dagger B\,\partial_4\Psi = \Psi^\dagger K\,\partial_4\Psi,\qquad
K = i\cos z\,B .$$

**The rule.** For a Lagrangian of the first-order form $\Psi^\dagger K\partial_4\Psi
- \Psi^\dagger h'\Psi$ the classical equation is $K\partial_4\Psi = h'\Psi$. If the
quantum operators obey $\{\Psi_A, \Psi^\dagger_C\} = A_{AC}$ (and $\{\Psi_A,
\Psi_C\} = 0$), the operator $H = \Psi^\dagger h'\Psi$ satisfies
$[H, \Psi_c] = -(Ah'\Psi)_c$, so the Heisenberg equation $\partial_4\Psi =
i[H, \Psi]$ reads $\partial_4\Psi = -iAh'\Psi$. Agreement with the classical
equation $\partial_4\Psi = K^{-1}h'\Psi$ for every $h'$ requires $-iA = K^{-1}$,
that is $A = iK^{-1}$. With $K = i\cos z\,B$ and $B^{-1} = B$:

$$iK^{-1} = i\cdot\frac{1}{i\cos z}B^{-1} = \frac{B}{\cos z},\qquad
\{\Psi_A(x), \Psi^\dagger_C(y)\} = B_{AC}\,\frac{\delta^7(x - y)}{\cos z}.$$

(On a slice $x_4 = $ const the components at different points anticommute; the
seven-dimensional delta $\delta^7(x - y)$ expresses this. In the Fock-space
computations of this notebook one point is kept, so the anticommutator is the
```

```
matrix $B$ itself, up to the positive factor $1/\cos z$ that does not change any
sign.)

**The obstruction.** $B$ has the eigenvalue $-1$. For a column $u$ with $Bu = -u$
and $u^\dagger u = 1$, the operator $X = \sum_A u_A^*\Psi_A$ would have
$\{X, X^\dagger\} = u^\dagger Bu = -1$. If $X^\dagger$ were the Hilbert adjoint,
then for every state $\phi$ we would have $\langle\phi|\{X, X^\dagger\}|\phi\rangle =
\lVert X^\dagger\phi\rVert^2 + \lVert X\phi\rVert^2 \geq 0$, a contradiction. So the
canonical conjugate is not the Hilbert adjoint: the honest statement is that the
canonical rule defines a Krein space.
```

Text cell:

```
## 5. The time-derivative kernel and the canonical rule, exactly

The next cell reads the gammas, builds $C$, $B$ and the chirality $\Gamma =
\gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$ with numpy (whole numbers and the
imaginary unit `1j`), and checks $C\gamma^{(x_4)} = iB$ and $B^{-1} = B$. Then it
repeats the rule $iK^{-1} = B/\cos z$ with sympy, keeping $\cos z$ as a positive
symbol `c`.

It also defines `check_record(condition, name, record)`, which does what
`check(condition, name, record=record)` of the set-up cell does but prints the PASS
line and the line naming the reproduced Revision record in one piece (Jupyter
sends printed text to the screen in pieces whose boundaries depend on timing;
printing them in one piece keeps the stored output the same in every run).
Before it checks anything, `check_record` asks the helper `record_says_pass` whether
the cited record still says PASS: for a record name of the form
`<report file>, check <check name>` it opens that report (each file once, kept in
the dictionary `REPORT_CHECKS`) and stops the notebook with an error unless a check
of that name exists there with the verdict PASS; a record name without `, check `
names a data entry that the notebook reads and compares itself.
```

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
```

```
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])  # the chirality matrix
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
check(np.array_equal(C @ gamma[4], 1j * B), "C gamma^(x4) = i B")
check(np.array_equal(B @ B, np.eye(16)), "B B = I, so the inverse of B is B")

c = sp.Symbol("c", positive=True)  # c stands for cos z, positive on the patch
B_exact = sp.Matrix(16, 16, lambda i, j: sp.I * int(round(B[i, j].imag)))
K = sp.I * c * B_exact  # the kernel of the time-derivative term
rule = (sp.I * K.inv()).applyfunc(sp.simplify)  # the anticommutator matrix i K^-1
check_record(rule == B_exact / c,
             "exact: i K^(-1) = B / cos z with K = i cos z B",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "first_order_form_and_anticommutator")
```

Out [2]:

```
PASS C gamma^(x4) = i B
PASS B B = I, so the inverse of B is B
PASS exact: i K^(-1) = B / cos z with K = i cos z B
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    first_order_form_and_anticommutator
```

Text cell:

```
## 6. A fermionic Fock space in a few lines

The next cell builds the Fock space of 16 fermion modes $p = 0, 1, \dots, 15$ (mode
$p$ will carry the component $\Psi_{p+1}$). A state is stored as a Python
dictionary `{pattern: amplitude}` that lists only the patterns with a nonzero
amplitude; the vacuum is `{0: 1}` (the pattern 0 has every mode empty).

- `annihilate(p, state)` is $f_p$: in every pattern where mode $p$ is occupied it
```

```
  empties the mode; patterns where it is empty give nothing.
- `create(p, state)` is $f_p^*$: in every pattern where mode $p$ is empty it fills
  the mode.
- Both multiply by $(-1)^{\text{(number of occupied modes below } p)}$. This sign is
  what makes different modes ANTIcommute: exchanging the order in which two modes
  are filled changes the sign of the state.
- `add` forms combinations of states, `inner` is the positive inner product of two
  states, `largest` the largest size of an amplitude (to measure how far a state is
  from zero).
```

In [3]:

```
def sign_below(n, p):
    """(-1) to the power of the number of occupied modes below mode p in pattern n."""
    occupied_below = bin(n & ((1 << p) - 1)).count("1")  # count the 1-digits below p
    return -1 if occupied_below % 2 else 1


def annihilate(p, state):
    """f_p applied to a state {pattern: amplitude}."""
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:  # mode p is occupied in the pattern n
            new = n ^ (1 << p)  # the same pattern with mode p emptied
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):
    """f_p^* (the Hilbert adjoint of f_p) applied to a state."""
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:  # mode p is empty in the pattern n
            new = n | (1 << p)  # the same pattern with mode p filled
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def add(*terms):
    """The state sum of coefficient * state over the pairs (coefficient, state)."""
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result


def inner(left, right):
    """The positive inner product <left|right>."""
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
```

```
    """The largest size of an amplitude of the state (0 for the zero state)."""
    return max((abs(a) for a in state.values()), default=0.0)


VACUUM = {0: 1.0}
rng = np.random.default_rng(12345)  # random numbers with a fixed seed


def random_state(patterns=6):
    """A normalised superposition of a few random patterns."""
    state = {}
    for n in rng.integers(0, 2 ** 16, size=patterns):
        state[int(n)] = complex(rng.normal(), rng.normal())
    length = np.sqrt(inner(state, state).real)
    return {n: a / length for n, a in state.items()}


phi = random_state()
worst = 0.0
for p in range(16):
    for q in range(16):
        # {f_p, f_q^*} phi = f_p f_q^* phi + f_q^* f_p phi must be phi (p = q) or 0
        both = add((1, annihilate(p, create(q, phi))),
                   (1, create(q, annihilate(p, phi))))
        expected = phi if p == q else {}
        worst = max(worst, largest(add((1, both), (-1, expected))))
        # {f_p, f_q} phi must be 0
        pair = add((1, annihilate(p, annihilate(q, phi))),
                   (1, annihilate(q, annihilate(p, phi))))
        worst = max(worst, largest(pair))
report("largest violation of the 512 anticommutation relations", f"{worst:.1e}")
check(worst < 1e-14, "the Fock operators obey {f_p, f_q^*} = delta_pq, {f_p, f_q} = 0")
```

Out [3]:

```
RESULT largest violation of the 512 anticommutation relations = 0.0e+00
PASS the Fock operators obey {f_p, f_q^*} = delta_pq, {f_p, f_q} = 0
```

Text cell:

```
## 7. The canonical field on the Fock space

In this positive Fock space the field component $\Psi_A$ is the annihilator $f_{A-1}$
and its Hilbert adjoint is $\chi_A = f_{A-1}^*$. The CANONICAL conjugate is defined as
$\Psi^\dagger = \chi B$, that is $\Psi^\dagger_A = \sum_C \chi_C B_{CA}$. Then

$$\{\Psi_A, \Psi^\dagger_C\} = \sum_D \{f_{A-1}, f_{D-1}^*\}B_{DC} = B_{AC},$$

the canonical rule of Section 5 (at one point). The next cell checks this for all
256 pairs $(A, C)$ on a random state. Then it checks the Heisenberg equation for a
plane wave: with the Hamiltonian-density matrix $h' = mC - i\sum_a k_a C\gamma^{(x_a)}$
(the matrix of $\Psi^\dagger(\ldots)\Psi$ in the Hamiltonian of a plane wave) the
```

```
operator $H = \Psi^\dagger h'\Psi = \chi(Bh')\Psi$ must satisfy $[H, \Psi_c] =
-(Bh'\Psi)_c$, and $Bh'$ must be the mode Hamiltonian $h = -im\gamma^{(x_4)} -
\gamma^{(x_4)}\sum_a k_a\gamma^{(x_a)}$. Together: $i\,\partial_4\Psi = i \cdot
i[H, \Psi] = h\Psi$, the classical wave equation of the field.
```

In [4]:

```
def psi(A, state):
    """Psi_A = f_(A-1), for the components A = 1, ..., 16."""
    return annihilate(A - 1, state)


def psi_dagger(A, state):
    """The canonical conjugate Psi^dagger_A = sum_C chi_C B_CA, chi_C = f_(C-1)^*."""
    return add(*[(B[C - 1, A - 1], create(C - 1, state))
                 for C in range(1, 17) if B[C - 1, A - 1] != 0])


worst = 0.0
for A in range(1, 17):
    for C_index in range(1, 17):
        both = add((1, psi(A, psi_dagger(C_index, phi))),
                   (1, psi_dagger(C_index, psi(A, phi))))
        worst = max(worst, largest(add((1, both), (-B[A - 1, C_index - 1], phi))))
report("largest violation of {Psi_A, Psi^dagger_C} = B_AC", f"{worst:.1e}")
check_record(worst < 1e-14, "on the Fock space {Psi_A, Psi^dagger_C} = B_AC (256 pairs)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "canonical_anticommutator_B")

m_value = 1.3  # any mass and momenta (here with momenta along the extra times too)
k = {0: 1.1, 1: 0.4, 2: -0.7, 3: 0.2, 5: 0.9, 6: -0.3, 7: 0.5}
h_prime = m_value * C - 1j * sum(k_a * (C @ gamma[a]) for a, k_a in k.items())
h_mode = -1j * m_value * gamma[4] - sum(k_a * (gamma[4] @ gamma[a])
                                        for a, k_a in k.items())
check(np.max(np.abs(B @ h_prime - h_mode)) < 1e-14,
      "B (m C - i sum_a k_a C gamma^a) equals the mode Hamiltonian h")


def hamiltonian(state):
    """H = Psi^dagger h' Psi = sum_(C,D) (B h')_CD chi_C Psi_D applied to a state."""
    M = B @ h_prime
    return add(*[(M[c_, d_], create(c_, annihilate(d_, state)))
                 for c_ in range(16) for d_ in range(16) if abs(M[c_, d_]) > 0])


worst = 0.0
for c_ in range(16):
    commutator = add((1, hamiltonian(annihilate(c_, phi))),
                     (-1, annihilate(c_, hamiltonian(phi))))  # [H, Psi_c] phi
    expected = add(*[(-h_mode[c_, d_], annihilate(d_, phi)) for d_ in range(16)])
    worst = max(worst, largest(add((1, commutator), (-1, expected))))
report("largest violation of [H, Psi_c] = -(h Psi)_c", f"{worst:.1e}")
check_record(worst < 1e-12,
```

```
            "Heisenberg: i d4 Psi = h Psi, the classical wave equation",
            record="Revision/theory/reports/wolfram-field-theory.json, check "
                   "Heisenberg_equation_reproduces_field_equation")
```

Out [4]:

```
RESULT largest violation of {Psi_A, Psi^dagger_C} = B_AC = 0.0e+00
PASS on the Fock space {Psi_A, Psi^dagger_C} = B_AC (256 pairs)
     reproduces Revision/theory/reports/python-field-theory.json, check
    canonical_anticommutator_B
PASS B (m C - i sum_a k_a C gamma^a) equals the mode Hamiltonian h
RESULT largest violation of [H, Psi_c] = -(h Psi)_c = 1.6e-17
PASS Heisenberg: i d4 Psi = h Psi, the classical wave equation
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Heisenberg_equation_reproduces_field_equation
```

Text cell:

```
## 8. The theorem: the canonical conjugate is not the Hilbert adjoint

The Revision record names an exact column with $Bu = -u$: $u_7 = -i/\sqrt2$,
$u_{16} = 1/\sqrt2$, all other entries 0. The next cell checks $Bu = -u$ and
$u^\dagger Bu = -1$ and builds three operators on the Fock space:
$X = \sum_A u_A^*\Psi_A$, its Hilbert adjoint $X^* = \sum_A u_A\chi_A$, and its
canonical conjugate $X^\dagger = \sum_A u_A\Psi^\dagger_A$. It checks
$\{X, X^*\}\phi = \phi$ (positive, as in every Hilbert space) and $\{X, X^\dagger\}
\phi = -\phi$ (the canonical value $u^\dagger Bu = -1$). So $X^\dagger = -X^*$ here:
the canonical conjugate differs from the Hilbert adjoint.
```

In [5]:

```
u = np.zeros(16, dtype=complex)
u[6] = -1j / np.sqrt(2)  # u_7 (the computer counts from 0)
u[15] = 1 / np.sqrt(2)  # u_16
krein_norm_u = (u.conj() @ B @ u).real
report("u^dagger u and u^dagger B u", f"{(u.conj() @ u).real:.12f} and "
       f"{krein_norm_u:.12f}")


def X(state):  # X = sum_A conj(u_A) Psi_A
    return add(*[(np.conj(u[A - 1]), psi(A, state)) for A in range(1, 17)])


def X_hilbert(state):  # X^* = sum_A u_A chi_A (the Hilbert adjoint of X)
    return add(*[(u[A - 1], create(A - 1, state)) for A in range(1, 17)])


def X_canonical(state):  # X^dagger = sum_A u_A Psi^dagger_A (canonical conjugate)
    return add(*[(u[A - 1], psi_dagger(A, state)) for A in range(1, 17)])
```

```
hilbert = add((1, X(X_hilbert(phi))), (1, X_hilbert(X(phi))))
canonical = add((1, X(X_canonical(phi))), (1, X_canonical(X(phi))))
hilbert_ok = largest(add((1, hilbert), (-1, phi))) < 1e-14  # {X, X^*} = +1
canonical_ok = largest(add((1, canonical), (1, phi))) < 1e-14  # {X, X^dagger} = -1
check_record(np.max(np.abs(B @ u + u)) < 1e-15 and abs(krein_norm_u + 1) < 1e-15
             and hilbert_ok and canonical_ok,
             "B u = -u: {X, X^*} = +1 but the canonical {X, X^dagger} = -1",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "no_positive_inner_product")
```

Out [5]:

```
RESULT u^dagger u and u^dagger B u = 1.000000000000 and -1.000000000000
PASS B u = -u: {X, X^*} = +1 but the canonical {X, X^dagger} = -1
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    no_positive_inner_product
```

Text cell:

```
The next cell makes the positivity visible. For 300 random normalised states $\phi$
it computes the two lengths $\lVert X\phi\rVert^2$ and $\lVert X^*\phi\rVert^2$.
Each is a squared length, never negative, and their sum is $\langle\phi|\{X,X^*\}|
\phi\rangle = 1$: every point lies on the line $x + y = 1$ in the positive quarter
of the plane. The canonical rule would need the sum $-1$ instead (the dashed line),
which no state of any Hilbert space can reach. The same cell also shows how
indefinite the Krein form is: for 5000 random unit columns $v$ (normal random
numbers, then divided by the length) the ordinary length is always 1, but the
Krein norm $v^\dagger Bv$ is spread between $-1$ and $1$.
```

In [6]:

```
lengths = []
for _ in range(300):
    state = random_state(patterns=4)
    x_part, x_star_part = X(state), X_hilbert(state)
    lengths.append((inner(x_part, x_part).real, inner(x_star_part, x_star_part).real))
lengths = np.array(lengths)
sums = lengths.sum(axis=1)
report("smallest and largest |X phi|^2 + |X^* phi|^2",
       f"{sums.min():.12f} and {sums.max():.12f}")
check(np.all(lengths >= 0) and np.max(np.abs(sums - 1)) < 1e-12,
      "positivity: |X phi|^2 + |X^* phi|^2 = 1 for every random state")

columns = rng.normal(size=(5000, 16)) + 1j * rng.normal(size=(5000, 16))
columns = columns / np.linalg.norm(columns, axis=1, keepdims=True)  # unit length
krein_norms = np.einsum("ni,ij,nj->n", columns.conj(), B, columns).real
report("smallest and largest Krein norm of the 5000 unit columns",
       f"{krein_norms.min():.3f} and {krein_norms.max():.3f}")
check(krein_norms.min() < -0.5 and krein_norms.max() > 0.5
      and np.all(np.abs(krein_norms) <= 1 + 1e-12),
      "random unit columns have Krein norms of both signs, all between -1 and 1")
```

```
fig, ax = plt.subplots(figsize=(6.0, 4.6))
ax.plot(lengths[:, 0], lengths[:, 1], "o", color="#2a78d6", markersize=4,
        label="300 random states $\\phi$")
line = np.linspace(-1.4, 1.2, 2)
ax.plot(line, 1 - line, color="#52514e", linewidth=1, label="$x + y = 1$")
ax.plot(line, -1 - line, "--", color="#eb6834", linewidth=2,
        label="$x + y = -1$ (canonical rule)")
ax.set_xlim(-1.4, 1.2)
ax.set_ylim(-1.4, 1.2)
ax.set_xlabel("$x = \\|X\\phi\\|^2$")
ax.set_ylabel("$y = \\|X^*\\phi\\|^2$")
ax.set_title("Squared lengths are never negative")
ax.legend(loc="lower left")
save_figure(fig, "positivity",
            "For 300 random states $\\phi$ of the Fock space (blue dots): the "
            "squared length of $X\\phi$ (horizontal axis) and of $X^{\\ast}\\phi$ "
            "(vertical axis), where $X = \\sum_A \\bar{u}_A\\Psi_A$ with the "
            "recorded column $Bu = -u$ and $X^{\\ast}$ is its Hilbert adjoint; both "
            "axes are pure numbers. Every dot lies on the grey line $x + y = 1$ in "
            "the quarter where both are positive, because $\\|X\\phi\\|^2 + "
            "\\|X^{\\ast}\\phi\\|^2 = \\langle\\phi|\\{X, X^{\\ast}\\}|\\phi\\rangle = "
            "1$. The canonical rule "
            "would require $x + y = u^\\dagger B u = -1$ (orange dashed line), "
            "which no state can reach: the canonical conjugate is not the Hilbert "
            "adjoint.")

fig, ax = plt.subplots()
ax.set_axisbelow(True)  # draw the grid lines behind the bars
ax.hist(krein_norms, bins=50, range=(-1, 1), color="#2a78d6", edgecolor="white")
ax.axvline(0.0, color="#52514e", linewidth=1)
ax.set_xlabel("Krein norm $v^\\dagger B v$ of a random column with $v^\\dagger v = 1$")
ax.set_ylabel("number of columns (of 5000)")
ax.set_title("The Krein form is indefinite: both signs occur")
save_figure(fig, "krein_norms_random",
            "Histogram of the Krein norm $v^\\dagger B v$ of 5000 random complex "
            "columns $v$ with 16 entries, each scaled to ordinary length "
            "$v^\\dagger v = 1$ (horizontal axis: the Krein norm, a pure number "
            "between $-1$ and $1$; vertical axis: how many columns fall into each "
            "of the 50 bins). The ordinary length is 1 for every column, but the "
            "Krein norm takes both signs, symmetrically about 0, and is small most "
            "of the time: eight of the 16 directions carry positive and eight "
            "carry negative charge, and a random column mixes them.")
```

Out [6]:

```
RESULT smallest and largest |X phi|^2 + |X^* phi|^2 = 1.000000000000 and 1.000000000000
PASS positivity: |X phi|^2 + |X^* phi|^2 = 1 for every random state
RESULT smallest and largest Krein norm of the 5000 unit columns = -0.777 and 0.704
PASS random unit columns have Krein norms of both signs, all between -1 and 1
```

Out [6]:

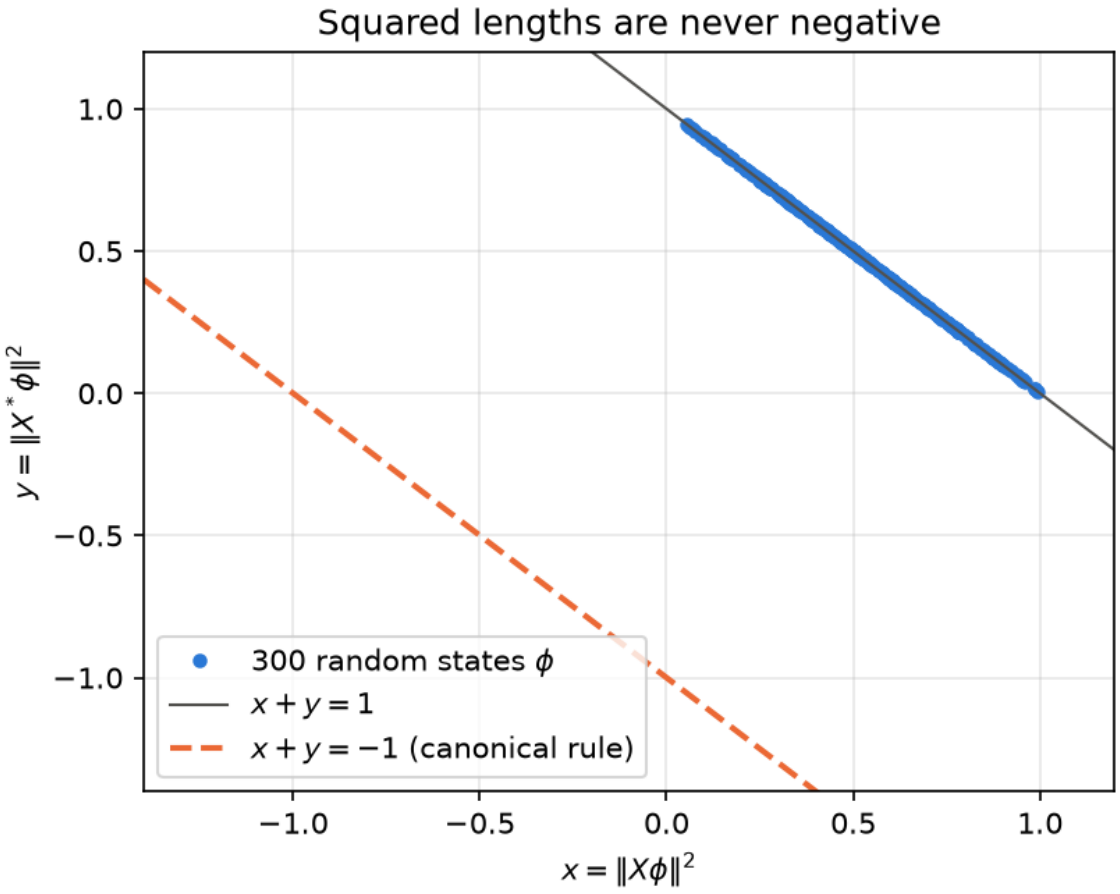

Figure 273: For 300 random states ϕ of the Fock space (blue dots): the squared length of $X\phi$ (horizontal axis) and of $X^*\phi$ (vertical axis), where $X = \sum_A \bar{u}_A \Psi_A$ with the recorded column $Bu = -u$ and X^* is its Hilbert adjoint; both axes are pure numbers. Every dot lies on the grey line $x + y = 1$ in the quarter where both are positive, because $\|X\phi\|^2 + \|X^*\phi\|^2 = \langle\phi|\{X, X^*\}|\phi\rangle = 1$. The canonical rule would require $x + y = u^\dagger B u = -1$ (orange dashed line), which no state can reach: the canonical conjugate is not the Hilbert adjoint. (Notebook 10b, figure 1.)

```
Figure 10b.1 saved as Revision/textbook/figures/10b_1_positivity.png
```

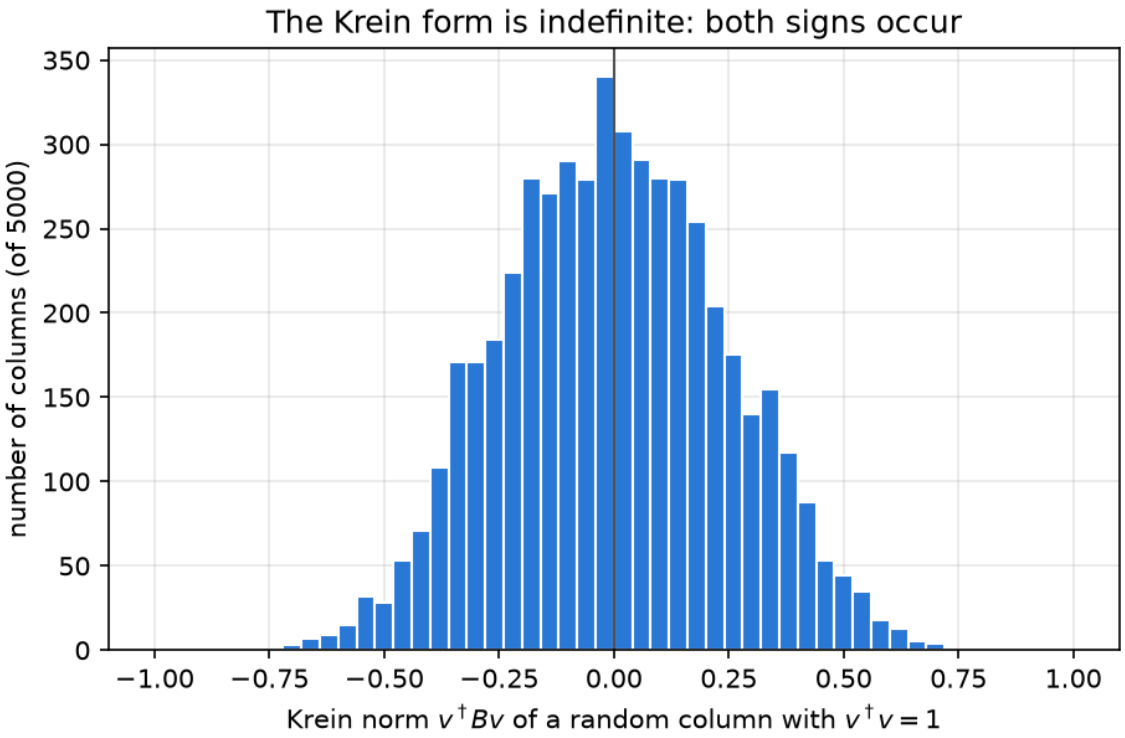

Figure 274: Histogram of the Krein norm $v^\dagger B v$ of 5000 random complex columns v with 16 entries, each scaled to ordinary length $v^\dagger v = 1$ (horizontal axis: the Krein norm, a pure number between -1 and 1; vertical axis: how many columns fall into each of the 50 bins). The ordinary length is 1 for every column, but the Krein norm takes both signs, symmetrically about 0, and is small most of the time: eight of the 16 directions carry positive and eight carry negative charge, and a random column mixes them. (Notebook 10b, figure 2.)

Out [6]:

```
Figure 10b.2 saved as Revision/textbook/figures/10b_2_krein_norms_random.png
```

Text cell:

```
## 9. The Krein-Fock realisation and the expectation-value rule

The Revision record also describes the other way out: keep $\Psi^\dagger$ as the
canonical conjugate and let the modes have Krein norms $\pm1$. The next cell

- builds 16 Krein-orthonormal modes: 8 eigenvectors of $B$ with eigenvalue $+1$
  ($\epsilon = +1$) and 8 with eigenvalue $-1$ ($\epsilon = -1$), each set made
  orthonormal by the Gram-Schmidt method applied to the columns of the projectors
  $\frac12(I \pm B)$;
- Krein-boosts the first pair, exactly as the record does:
  $u_1' = \frac54u_1 + \frac34u_9$, $u_9' = \frac34u_1 + \frac54u_9$ (with
```

```
  $\cosh t = \frac54$ and $\sinh t = \frac34$, and indeed $\frac{25}{16} -
  \frac{9}{16} = 1$);
- checks $U^\dagger BU = E$ and $UEU^\dagger = B$, where the columns of $U$ are the
  modes and $E = \mathrm{diag}(\epsilon_1, \dots, \epsilon_{16})$;
- realises the Krein-Fock operators on the Fock space as $b_n = f_n$ and
  $b_n^\dagger = \epsilon_n f_n^*$, so that $\{b_n, b_n^\dagger\} = \epsilon_n$, and
  the field as $\Psi = \sum_n u_n b_n$, $\Psi^\dagger = \sum_n u_n^* b_n^\dagger$.
  Then $\{\Psi_A, \Psi^\dagger_C\} = \sum_n\epsilon_n(u_n)_A(u_n)_C^* =
  (UEU^\dagger)_{AC} = B_{AC}$: the same canonical rule.
```

In [7]:

```
def orthonormal_columns(P):
    """Orthonormal columns that span the range of P (Gram-Schmidt, done twice)."""
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T


plus_modes = orthonormal_columns((np.eye(16) + B) / 2)  # B u = +u, 8 columns
minus_modes = orthonormal_columns((np.eye(16) - B) / 2)  # B u = -u, 8 columns
U = np.hstack([plus_modes, minus_modes]).astype(complex)  # 16 modes as columns
epsilon = np.array([1.0] * 8 + [-1.0] * 8)  # their Krein norms
cosh_t, sinh_t = 5 / 4, 3 / 4  # a Krein boost of the modes 1 and 9
first, ninth = U[:, 0].copy(), U[:, 8].copy()
U[:, 0] = cosh_t * first + sinh_t * ninth
U[:, 8] = sinh_t * first + cosh_t * ninth
E = np.diag(epsilon)
gram_ok = np.max(np.abs(U.conj().T @ B @ U - E)) < 1e-14  # U^dagger B U = E
complete_ok = np.max(np.abs(U @ E @ U.conj().T - B)) < 1e-14  # U E U^dagger = B
report("squared ordinary length of the boosted mode 1",
       f"{np.linalg.norm(U[:, 0]) ** 2:.6f}")
check(gram_ok and complete_ok,
      "Krein-orthonormal modes: U^dagger B U = E and U E U^dagger = B")


def b(n, state):  # b_n = f_n
    return annihilate(n, state)


def b_dagger(n, state):  # b_n^dagger = epsilon_n f_n^*
    return add((epsilon[n], create(n, state)))


def krein_psi(A, state):  # Psi_A = sum_n (u_n)_A b_n
    return add(*[(U[A - 1, n], b(n, state)) for n in range(16)])
```

```
def krein_psi_dagger(A, state):  # Psi^dagger_A = sum_n conj((u_n)_A) b_n^dagger
    return add(*[(np.conj(U[A - 1, n]), b_dagger(n, state)) for n in range(16)])


worst = 0.0
for A in range(1, 17):
    for C_index in range(1, 17):
        both = add((1, krein_psi(A, krein_psi_dagger(C_index, phi))),
                   (1, krein_psi_dagger(C_index, krein_psi(A, phi))))
        worst = max(worst, largest(add((1, both), (-B[A - 1, C_index - 1], phi))))
report("Krein-Fock: largest violation of {Psi_A, Psi^dagger_C} = B_AC", f"{worst:.1e}")
check(worst < 1e-13, "Krein-Fock realisation: {Psi, Psi^dagger} = B again")
```

Out [7]:

```
RESULT squared ordinary length of the boosted mode 1 = 2.125000
PASS Krein-orthonormal modes: U^dagger B U = E and U E U^dagger = B
RESULT Krein-Fock: largest violation of {Psi_A, Psi^dagger_C} = B_AC = 1.6e-16
PASS Krein-Fock realisation: {Psi, Psi^dagger} = B again
```

Text cell:

```
In the one-mode state $b_n^\dagger|0\rangle$ the Krein norm is
$\langle 0|b_n b_n^\dagger|0\rangle = \epsilon_n$, and the record's rule for the
normalised expectation value of an observable $\Psi^\dagger M\Psi$ is

$$\frac{\langle 0|b_n(\Psi^\dagger M\Psi)b_n^\dagger|0\rangle}{\langle 0|b_n b_n^\dagger
|0\rangle} = \epsilon_n\,u_n^\dagger M u_n .$$

For a mode that is an eigenvector of $B$ ($Bu_n = \epsilon_n u_n$, hence
$u_n^\dagger B = \epsilon_n u_n^\dagger$) this equals $u_n^\dagger BMu_n$; for the
Krein-boosted mode it does not. The next cell computes the left-hand side on the
Fock space (vacuum expectation values of operator products) for 60 random complex
matrices $M$, for the unboosted mode $n = 4$ and the boosted mode $n = 1$, and
compares it with both formulas.
```

In [8]:

```
def expectation(n, M):
    """<0| b_n (Psi^dagger M Psi) b_n^dagger |0> / <0| b_n b_n^dagger |0>."""
    excited = b_dagger(n, VACUUM)  # b_n^dagger |0>
    acted = {}
    for A in range(1, 17):
        psi_part = krein_psi(A, excited)
        for C_index in range(1, 17):
            if M[C_index - 1, A - 1] != 0:
                acted = add((1, acted), (M[C_index - 1, A - 1],
                                         krein_psi_dagger(C_index, psi_part)))
    numerator = inner(VACUUM, b(n, acted))  # <0| b_n ... |0>, a number
```

```
    norm = inner(VACUUM, b(n, excited))  # <0| b_n b_n^dagger |0> = epsilon_n
    return numerator / norm


rule_errors, naive_errors, points = [], [], []
for _ in range(60):
    M = rng.normal(size=(16, 16)) + 1j * rng.normal(size=(16, 16))
    for n in (3, 0):  # mode 4 (unboosted) and mode 1 (boosted), counted from 0
        value = expectation(n, M)
        rule = epsilon[n] * (U[:, n].conj() @ M @ U[:, n])
        naive = U[:, n].conj() @ B @ M @ U[:, n]
        rule_errors.append(abs(value - rule))
        if n == 3:
            naive_errors.append(abs(value - naive))
        else:
            points.append((value.real, rule.real, naive.real))
points = np.array(points)
naive_boosted = np.max(np.abs(points[:, 0] - points[:, 2]))
report("largest |Fock value - epsilon u^dagger M u| (both modes)",
       f"{max(rule_errors):.1e}")
report("boosted mode: largest |Fock value - u^dagger B M u|", f"{naive_boosted:.2f}")
check_record(max(rule_errors) < 1e-12 and max(naive_errors) < 1e-12
             and naive_boosted > 0.1,
             "expectation rule eps u^dagger M u; u^dagger B M u fails if boosted",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "expectation_value_rule")
```

Out [8]:

```
RESULT largest |Fock value - epsilon u^dagger M u| (both modes) = 1.3e-15
RESULT boosted mode: largest |Fock value - u^dagger B M u| = 5.82
PASS expectation rule eps u^dagger M u; u^dagger B M u fails if boosted
     reproduces Revision/theory/reports/python-field-theory.json, check
    expectation_value_rule
```

Text cell:

```
The next cell draws two pictures of what was just computed. The first shows the
plane spanned by the two modes $u_1$ ($B = +1$) and $u_9$ ($B = -1$) before the
boost. A column $a u_1 + b u_9$ has the ordinary squared length $a^2 + b^2$
(constant on circles) but the Krein norm $a^2 - b^2$ (constant on hyperbolas). The
boosted modes $(\frac54, \frac34)$ and $(\frac34, \frac54)$ lie on the hyperbolas of
Krein norm $+1$ and $-1$, outside the unit circle, and they are Krein-orthogonal:
$\frac54\cdot\frac34 - \frac34\cdot\frac54 = 0$. The second shows the expectation
values of the boosted mode for the 60 random matrices: the Fock-space value against
the rule $\epsilon u^\dagger Mu$ (on the diagonal) and against $u^\dagger BMu$
(scattered away from it).
```

In [9]:

```
fig, ax = plt.subplots(figsize=(6.0, 6.0))
angle = np.linspace(0, 2 * np.pi, 400)
```

```
ax.plot(np.cos(angle), np.sin(angle), color="#52514e", linewidth=1.5,
        label="ordinary length 1: $a^2 + b^2 = 1$")
t = np.linspace(-1.6, 1.6, 400)
for side in (1, -1):  # the two branches of each hyperbola
    ax.plot(side * np.cosh(t), np.sinh(t), color="#2a78d6", linewidth=2,
            label="Krein norm $+1$: $a^2 - b^2 = 1$" if side == 1 else None)
    ax.plot(np.sinh(t), side * np.cosh(t), color="#eb6834", linewidth=2,
            label="Krein norm $-1$: $a^2 - b^2 = -1$" if side == 1 else None)
ax.plot([0, 1, 0], [1, 0, 0], "s", color="#52514e", markersize=7)
ax.annotate("$u_1$", (1.0, 0.0), xytext=(1.05, -0.3))
ax.annotate("$u_9$", (0.0, 1.0), xytext=(-0.35, 1.05))
ax.plot([1.25, 0.75], [0.75, 1.25], "o", color="#1baf7a", markersize=9,
        label="boosted modes $(5/4, 3/4)$, $(3/4, 5/4)$")
for x_end, y_end in ((1.25, 0.75), (0.75, 1.25)):
    ax.annotate("", xy=(x_end, y_end), xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": "#1baf7a"})
ax.set_xlim(-2.6, 2.6)
ax.set_ylim(-2.6, 2.6)
ax.set_aspect("equal")
ax.set_xlabel("coefficient $a$ of $u_1$")
ax.set_ylabel("coefficient $b$ of $u_9$")
ax.set_title("Ordinary length versus Krein norm in one plane")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "krein_plane",
            "The plane of the columns $a u_1 + b u_9$ spanned by a mode $u_1$ with "
            "$Bu_1 = u_1$ and a mode $u_9$ with $Bu_9 = -u_9$, both of ordinary "
            "length 1 (horizontal axis $a$, vertical axis $b$, pure numbers). The "
            "grey circle holds the columns of ordinary length 1, $a^2 + b^2 = 1$; "
            "the blue hyperbola those of Krein norm $a^2 - b^2 = +1$, the orange "
            "one those of Krein norm $-1$. The Krein-boosted modes $(5/4, 3/4)$ and "
            "$(3/4, 5/4)$ (green) keep the Krein norms $+1$ and $-1$ but have the "
            "ordinary squared length $34/16$, and they are Krein-orthogonal: "
            "$\\frac{5}{4}\\cdot\\frac{3}{4} - \\frac{3}{4}\\cdot\\frac{5}{4} = 0$.")

fig, ax = plt.subplots(figsize=(6.0, 5.0))
ax.plot(points[:, 1], points[:, 0], "o", color="#1baf7a", markersize=5,
        label="against the rule $\\epsilon\\,u^\\dagger M u$")
ax.plot(points[:, 2], points[:, 0], "x", color="#eb6834", markersize=6,
        label="against $u^\\dagger B M u$")
low, high = points.min() - 1, points.max() + 1
ax.plot([low, high], [low, high], color="#52514e", linewidth=1)
ax.set_xlabel("value of the formula (real part)")
ax.set_ylabel("value computed on the Fock space (real part)")
ax.set_title("The boosted mode: which formula is right?")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "expectation_rule",
            "For 60 random complex $16 \\times 16$ matrices $M$: the normalised "
            "expectation value of $\\Psi^\\dagger M\\Psi$ in the Krein-boosted mode, "
            "computed on the Fock space (vertical axis), against the rule "
            "$\\epsilon u^\\dagger Mu$ of the Revision record (green dots) and "
            "against the formula $u^\\dagger BMu$ (orange crosses); horizontal "
            "axis: the value of the formula; real parts, pure numbers. The green "
            "dots lie exactly on the diagonal; the orange crosses do not, because "
            "the boosted mode is not an eigenvector of $B$.")
```

Out [9]:

```
Figure 10b.3 saved as Revision/textbook/figures/10b_3_krein_plane.png
```

Out [9]:

```
Figure 10b.4 saved as Revision/textbook/figures/10b_4_expectation_rule.png
```

Text cell:

```
## 10. Maps that keep the anticommutator; the conjugation of the quantised field

**Linear maps.** If $\Psi' = M\Psi$ with a constant matrix $M$, then
$\{\Psi'_A, \Psi'^\dagger_C\} = \sum_{D,F} M_{AD}M_{CF}^*\{\Psi_D, \Psi^\dagger_F\} =
(MBM^\dagger)_{AC}$. The Revision pairing record lists $MBM^\dagger = \sigma B$ with a
sign $\sigma$ for 17 maps: the chirality $\Gamma$, the eight gammas
$\gamma^{(x_a)}$ and the eight reflections $P_a = \Gamma\gamma^{(x_a)}$. A map with
$\sigma = -1$ turns the canonical rule into its negative: the chirality image
$\Gamma\Psi$ of the pairing theorem T1 carries the Krein metric $-B$ (statement Q1
of the quantum reading), while the mirror map $\gamma^{(x_0)}$ of theorem T2 keeps
$+B$.

**Conjugation of the quantised field.** Charge conjugation is a MATRIX map. For the
quantised field it has the form $\Psi^c = M\Psi^{\dagger T}$, that is
$\Psi^c_A = \sum_C M_{AC}\Psi^\dagger_C$. Then, line by line,

$$\{\Psi^c_A, \Psi^{c\dagger}_C\} = \sum_{D,F} M_{AD}M_{CF}^*\{\Psi^\dagger_D,
\Psi_F\} = \sum_{D,F} M_{AD}B_{FD}M_{CF}^* = (MB^TM^\dagger)_{AC}.$$

$B$ is Hermitian and purely imaginary, so $B^T = B^* = -B$. For $M = I$ (the map
$\Psi \to \Psi^{\dagger T}$, which is not a symmetry here) the result is $B^T = -B$:
the sign of the canonical rule is reversed. For $M = \Gamma$ the result is
$\Gamma B^T\Gamma^\dagger = -\Gamma B\Gamma = +B$, because $\Gamma B\Gamma = -B$
($\Gamma$ anticommutes with the five gammas in $B$, an odd number). So the
conjugation of the quantised field that preserves $\{\Psi, \Psi^\dagger\} = B\delta$
is $\Psi \to \Gamma\Psi^{\dagger T}$, and since $\Gamma$ anticommutes with every
gamma it maps the field of mass $m$ to the field of mass $-m$. The next cell checks
all of this. It also counts the checks of the lead-check report
‘Revision/lead_checks/reports/charge-conjugation-and-u1.json‘, which ‘check_record‘
has just read into ‘REPORT_CHECKS‘: there are 12, all with the verdict PASS.
```

In [10]:

```
pairing = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
rows = pairing["data"]["Krein_signs_M_B_Mdagger"]
maps = {"Gamma": Gamma}
for a in range(8):
    maps[f"gamma^x{a}"] = gamma[a]
    maps[f"P_x{a}"] = Gamma @ gamma[a]  # the reflection P_a = Gamma gamma^(x_a)
```

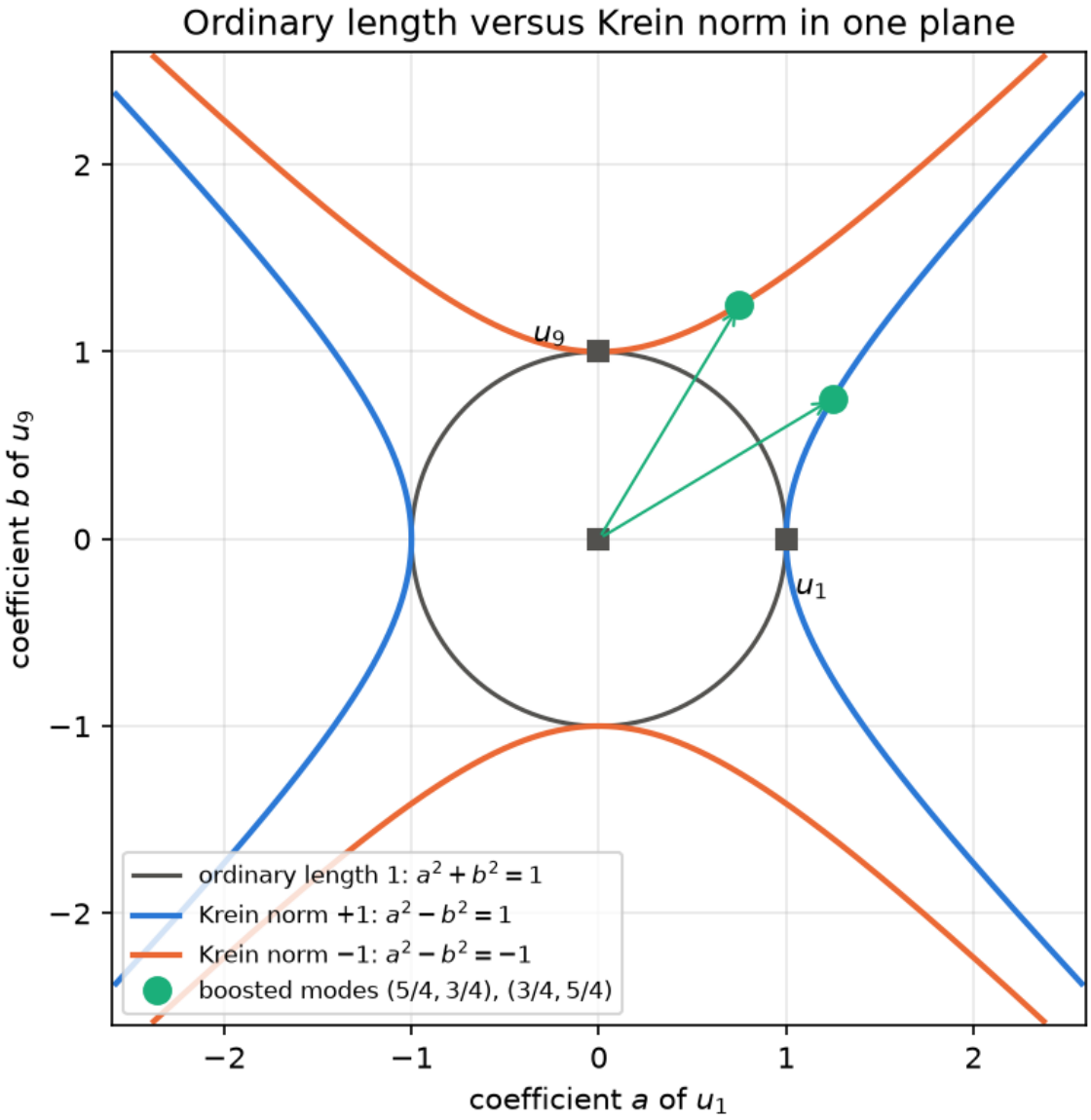

Figure 275: The plane of the columns $au_1 + bu_9$ spanned by a mode u_1 with $Bu_1 = u_1$ and a mode u_9 with $Bu_9 = -u_9$, both of ordinary length 1 (horizontal axis a, vertical axis b, pure numbers). The grey circle holds the columns of ordinary length 1, $a^2 + b^2 = 1$; the blue hyperbola those of Krein norm $a^2 - b^2 = +1$, the orange one those of Krein norm -1. The Krein-boosted modes $(5/4, 3/4)$ and $(3/4, 5/4)$ (green) keep the Krein norms $+1$ and -1 but have the ordinary squared length $34/16$, and they are Krein-orthogonal: $\frac{5}{4} \cdot \frac{3}{4} - \frac{3}{4} \cdot \frac{5}{4} = 0$. (Notebook 10b, figure 3.)

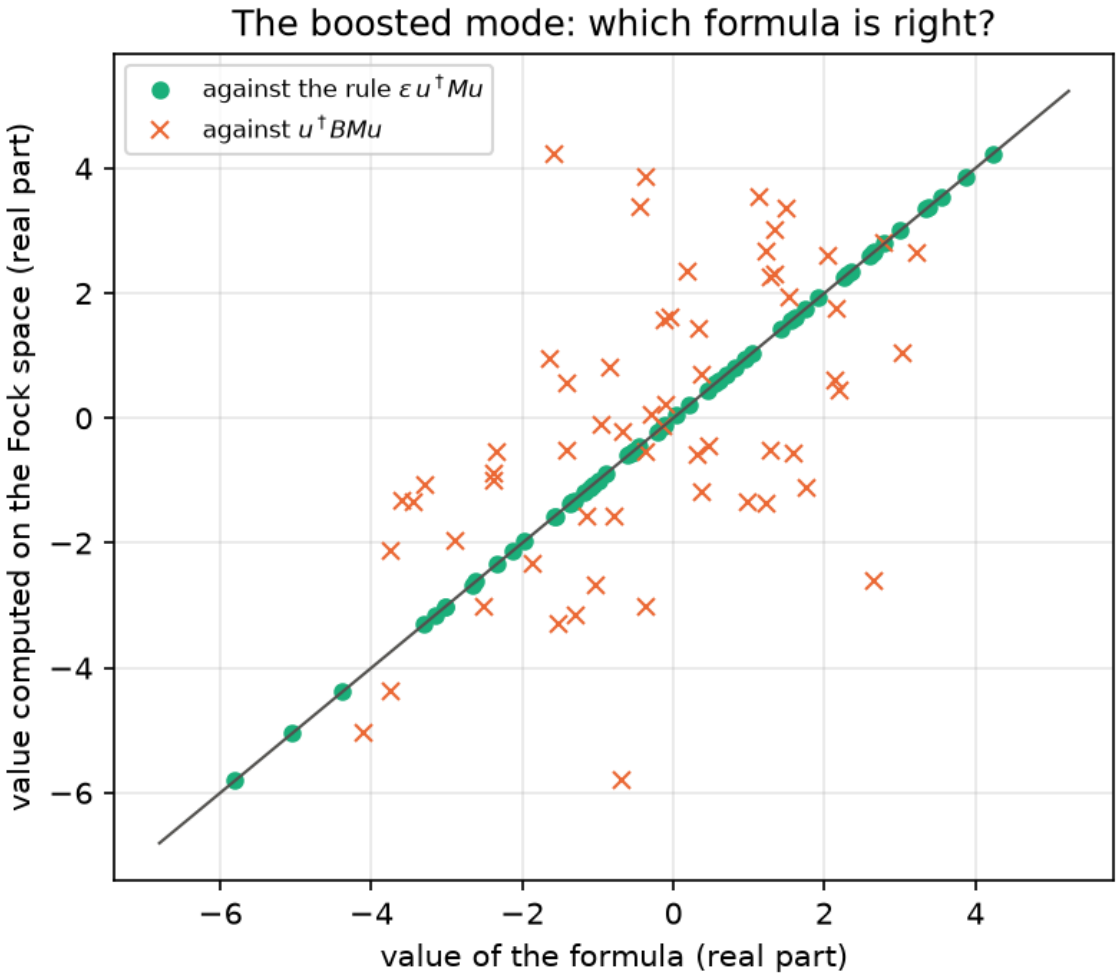

Figure 276: For 60 random complex 16×16 matrices M: the normalised expectation value of $\Psi^\dagger M \Psi$ in the Krein-boosted mode, computed on the Fock space (vertical axis), against the rule $\epsilon u^\dagger M u$ of the Revision record (green dots) and against the formula $u^\dagger BMu$ (orange crosses); horizontal axis: the value of the formula; real parts, pure numbers. The green dots lie exactly on the diagonal; the orange crosses do not, because the boosted mode is not an eigenvector of B. (Notebook 10b, figure 4.)

```
signs, agree = [], True
for row in rows:
    M = maps[row["map"]]
    transformed = M @ B @ M.conj().T  # M B M^dagger
    sign = (1 if np.array_equal(transformed, B)
            else -1 if np.array_equal(transformed, -B) else 0)
    signs.append(sign)
    agree = agree and sign == row["sign"]
names = [row["map"] for row in rows]  # the names of the 17 maps in the record
say("map: sign sigma in M B M^dagger = sigma B")
say(", ".join(f"{name}: {s:+d}" for name, s in zip(names, signs)))
check_record(agree and len(rows) == 17,
             "the Krein signs of all 17 recorded maps are reproduced",
             record="Revision/pairing/pairing-theory.json, data "
                    "Krein_signs_M_B_Mdagger")
check_record(np.array_equal(Gamma @ B @ Gamma.T, -B),
             "the chirality image Gamma Psi carries the Krein metric -B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_krein_metric")
check_record(np.array_equal(gamma[0] @ B @ gamma[0].T, B),
             "the mirror image gamma^(x0) Psi keeps +B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.T2_image_keeps_B")

identity = np.eye(16)
conjugation_identity = identity @ B.T @ identity.conj().T  # M = I
conjugation_gamma = Gamma @ B.T @ Gamma.conj().T  # M = Gamma
check(np.array_equal(B.T, -B), "B^T = -B (B is Hermitian and purely imaginary)")
check_record(np.array_equal(conjugation_gamma, B)
             and np.array_equal(conjugation_identity, -B),
             "Psi -> Gamma Psi^(dagger T) keeps B; with M = I one gets -B",
             record="Revision/lead_checks/reports/charge-conjugation-and-u1.json, "
                    "check quantum_charge_conjugation_unitary_type")
lead_file = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
lead_verdicts = list(REPORT_CHECKS[lead_file].values())  # read by check_record
passed = lead_verdicts.count("PASS")
report("checks of charge-conjugation-and-u1.json with the verdict PASS",
       f"{passed} of {len(lead_verdicts)}")
check(lead_verdicts == ["PASS"] * 12,
      "the lead-check report charge-conjugation-and-u1.json holds 12 checks, all PASS")
mass_reversed = all(np.array_equal(Gamma @ gamma[a] @ Gamma, -gamma[a])
                    for a in range(8))
check(mass_reversed, "Gamma anticommutes with every gamma: the conjugation reverses m")
```

Out [10]:

```
map: sign sigma in M B M^dagger = sigma B
Gamma: -1, gamma^x0: +1, gamma^x1: +1, gamma^x2: +1, gamma^x3: +1, gamma^x4: +1,
    gamma^x5: -1, gamma^x6: -1, gamma^x7: -1, P_x0: -1, P_x1: -1, P_x2: -1, P_x3: -1,
    P_x4: -1, P_x5: +1, P_x6: +1, P_x7: +1
PASS the Krein signs of all 17 recorded maps are reproduced
     reproduces Revision/pairing/pairing-theory.json, data Krein_signs_M_B_Mdagger
PASS the chirality image Gamma Psi carries the Krein metric -B
```

```
     reproduces Revision/pairing/reports/python-pairing.json, check Q.image_krein_metric
PASS the mirror image gamma^(x0) Psi keeps +B
     reproduces Revision/pairing/reports/python-pairing.json, check Q.T2_image_keeps_B
PASS B^T = -B (B is Hermitian and purely imaginary)
PASS Psi -> Gamma Psi^(dagger T) keeps B; with M = I one gets -B
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    quantum_charge_conjugation_unitary_type
RESULT checks of charge-conjugation-and-u1.json with the verdict PASS = 12 of 12
PASS the lead-check report charge-conjugation-and-u1.json holds 12 checks, all PASS
PASS Gamma anticommutes with every gamma: the conjugation reverses m
```

Text cell:

```
The next cell draws the signs as a bar chart: one bar per map, $+1$ (the map keeps
the canonical rule) or $-1$ (it reverses it), and, separately, the two candidate
conjugations of the quantised field.
```

In [11]:

```
labels = [row["map"].replace("gamma^x", "g").replace("P_x", "P") for row in rows]
labels = [lab if lab != "Gamma" else "Gam" for lab in labels]
colours = ["#2a78d6" if s > 0 else "#eb6834" for s in signs]
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0),
                         gridspec_kw={"width_ratios": [4, 1]})
for ax in axes:
    ax.set_axisbelow(True)  # draw the grid lines behind the bars
axes[0].bar(range(len(signs)), signs, color=colours, width=0.7)
axes[0].set_xticks(range(len(signs)), labels)
axes[0].axhline(0, color="#52514e", linewidth=1)
axes[0].set_ylim(-1.4, 1.4)
axes[0].set_yticks([-1, 0, 1])
axes[0].set_ylabel("sign $\\sigma$ in $M B M^\\dagger = \\sigma B$")
axes[0].set_title("linear maps $\\Psi \\to M\\Psi$")
conjugation_signs = [-1, 1]  # M = I gives -B, M = Gamma gives +B
axes[1].bar([0, 1], conjugation_signs, color=["#eb6834", "#2a78d6"], width=0.6)
axes[1].set_xticks([0, 1], ["$M = I$", "$M = \\Gamma$"])
axes[1].axhline(0, color="#52514e", linewidth=1)
axes[1].set_ylim(-1.4, 1.4)
axes[1].set_yticks([-1, 0, 1])
axes[1].set_title("$\\Psi \\to M\\Psi^{\\dagger T}$")
save_figure(fig, "krein_signs",
            "Left: the sign $\\sigma$ (vertical axis) with $MBM^\\dagger = \\sigma B$ "
            "for the 17 matrix maps of the Revision pairing record (horizontal "
            "axis: Gam is the chirality $\\Gamma$, g1 to g8 the gammas "
            "$\\gamma^{(x_0)}$ to $\\gamma^{(x_7)}$, P1 to P0 the reflections "
            "$\\Gamma\\gamma^{(x_a)}$); blue bars keep the canonical rule "
            "$\\{\\Psi, \\Psi^\\dagger\\} = B\\delta$, orange bars reverse it to "
            "$-B$. Right: the two candidate conjugations $\\Psi \\to "
            "M\\Psi^{\\dagger T}$ of the quantised field, with the sign of "
            "$MB^TM^\\dagger$: only $M = \\Gamma$ keeps $+B$, and it reverses the "
            "mass.")
```

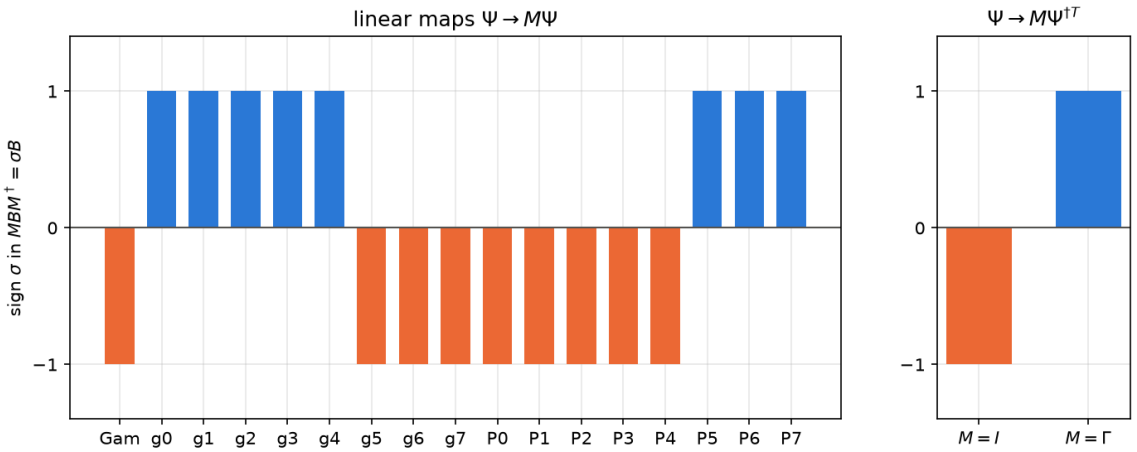

Figure 277: Left: the sign σ (vertical axis) with $MBM^\dagger = \sigma B$ for the 17 matrix maps of the Revision pairing record (horizontal axis: Gam is the chirality Γ, g1 to g8 the gammas $\gamma^{(x_0)}$ to $\gamma^{(x_7)}$, P1 to P0 the reflections $\Gamma\gamma^{(x_a)}$); blue bars keep the canonical rule $\{\Psi, \Psi^\dagger\} = B\delta$, orange bars reverse it to $-B$. Right: the two candidate conjugations $\Psi \to M\Psi^{\dagger T}$ of the quantised field, with the sign of $MB^T M^\dagger$: only $M = \Gamma$ keeps $+B$, and it reverses the mass. (Notebook 10b, figure 5.)

Out [11]:

```
Figure 10b.5 saved as Revision/textbook/figures/10b_5_krein_signs.png
```

Text cell:

```
## 11. The last check

The last cell checks that all five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [12]:

```
for name in ("10b_1_positivity.png", "10b_2_krein_norms_random.png",
             "10b_3_krein_plane.png", "10b_4_expectation_rule.png",
             "10b_5_krein_signs.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [12]:

```
PASS the figure file 10b_1_positivity.png exists
PASS the figure file 10b_2_krein_norms_random.png exists
PASS the figure file 10b_3_krein_plane.png exists
PASS the figure file 10b_4_expectation_rule.png exists
PASS the figure file 10b_5_krein_signs.png exists
ALL 25 CHECKS PASSED (notebook 10b)
```

Text cell:

```
## 12. What this notebook showed

- PROVED (exactly): the time-derivative term of the Lagrangian is
  $\Psi^\dagger K\partial_4\Psi$ with $K = i\cos z\,B$, and the canonical rule
  $\{\Psi, \Psi^\dagger\} = iK^{-1}\delta^7 = B\delta^7/\cos z$.
- CHECKED on an explicit fermionic Fock space (16 modes, 65536 patterns, of which
  only the needed ones are stored): the anticommutation relations, the canonical
  rule with $\Psi^\dagger = \chi B$, and the Heisenberg equation $i\partial_4\Psi =
  h\Psi$ for a plane wave, the classical wave equation of the field.
- PROVED (and checked on the Fock space): because $B$ has the eigenvalue $-1$, the
  canonical conjugate $\Psi^\dagger$ cannot be the Hilbert adjoint of $\Psi$ in a
  space with a positive inner product. The canonical rule defines a Krein space
  with fundamental symmetry $B$; in a positive Hilbert space the canonical
  conjugate is $\Psi^\dagger = \chi B$, with $\chi$ the Hilbert adjoint.
- CHECKED: the Krein-Fock realisation with modes of Krein norm $\pm1$ (including a
  Krein-boosted pair) gives the same rule, and the expectation value in a mode is
  $\epsilon_n u_n^\dagger M u_n$, which equals $u_n^\dagger BMu_n$ only when the mode
  is an eigenvector of $B$.
- PROVED: the chirality image $\Gamma\Psi$ carries $-B$, the mirror image
  $\gamma^{(x_0)}\Psi$ keeps $+B$ (all 17 recorded signs reproduced); the
  conjugation of the quantised field that preserves the canonical rule is the
  MATRIX map $\Psi \to \Gamma\Psi^{\dagger T}$, which reverses the mass; the map
  with $M = I$ would reverse the sign of the rule.
- NOT shown here: a positive state space for the whole field. The next notebook
  builds the positive Fock space of the good sector, one momentum at a time, with
  frozen coefficients.
```

10.19 Line-by-line walk-through of Notebook 10b

The notebook has 12 code cells, In [1] to In [12]. As in Section 10.10, the docstrings of the functions (the text in triple quotes under a `def` line) are left out of the quotations; they are printed in Section 10.18.

In [1], the set-up cell. Its first 248 lines are comments that repeat the run instructions of Section 10.17. The code below the heading THE SET-UP is word for word the code of In [1] of Notebook 10a, explained line by line in Section 10.10, except for one line, `NOTEBOOK_ID = "10b"  # this notebook:  chapter 10, example b`. It defines `REPO`, `OUTPUT_ROOT`, `repository_file`, `output_file`, `say`, `save_figure`, `check`, `report` and `all_checks_passed`, and prints the single line `Set-up of notebook 10b complete:  repository folder found, helpers defined.`

In [2], the time-derivative kernel and the canonical rule.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols
```

The same imports as In [2] of Notebook 10a: three modules of Python for printing into a buffer, numpy for numbers and sympy for exact algebra.

```
REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"
```

`REPORT_CHECKS` and `record_says_pass` are those of In [2] of Notebook 10a (Section 10.10): for a record name `<report file>, check <check name>` the function reads the report once and answers True only if a check of that name exists there with the verdict PASS; for a data entry of a file that the notebook reads itself it answers True.

```
def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output
```

`check_record` is the function of In [2] of Notebook 10a (Section 10.10): it stops the notebook with an error if the cited record check is missing or not PASS; otherwise it runs `check(condition, name, record=record)`, collects the PASS line and the line naming the reproduced Revision record in a text buffer, and prints both in one piece, so that the stored output is the same in every run.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])  # the chirality matrix
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
```

The record `Revision/algebra/gammas.json` is read and the eight gammas are stored as whole-number arrays, `gamma[a]` being $\gamma^{(x_a)}$. `C` is $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, `Gamma` the chirality Γ, the product of all eight gammas in the author's order (the parentheses let the product run over two lines), and `B` is $B = -iC\gamma^{(x_4)}$.

```
check(np.array_equal(C @ gamma[4], 1j * B), "C gamma^(x4) = i B")
check(np.array_equal(B @ B, np.eye(16)), "B B = I, so the inverse of B is B")
```

The two facts used in Section 10.12: $C\gamma^{(x_4)} = iB$ and $BB = I_{16}$. Both comparisons are exact (whole numbers times $\pm i$), and two PASS lines are printed.

```
c = sp.Symbol("c", positive=True)  # c stands for cos z, positive on the patch
B_exact = sp.Matrix(16, 16, lambda i, j: sp.I * int(round(B[i, j].imag)))
K = sp.I * c * B_exact  # the kernel of the time-derivative term
rule = (sp.I * K.inv()).applyfunc(sp.simplify)  # the anticommutator matrix i K^-1
```

`c` is a sympy letter for $\cos z$, declared positive (on the patch $0 < z < \pi/2$, $\cos z > 0$). `sp.Matrix(16, 16, lambda i, j: ...)` builds a 16×16 exact matrix entry by entry: entry (i, j) is i times the whole number nearest to the imaginary part of the numpy entry B_{ij} (`round` and `int` turn ± 1.0 into ± 1), so `B_exact` is B with exact entries. `K` is the kernel $K = i \cos z\, B$ of Section 10.12, `K.inv()` its exact inverse, and `rule` is iK^{-1}, every entry simplified.

```
check_record(rule == B_exact / c,
             "exact: i K^(-1) = B / cos z with K = i cos z B",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "first_order_form_and_anticommutator")
```

The check confirms $iK^{-1} = B/\cos z$ exactly, the canonical rule of Section 10.12, and reproduces the Wolfram record check `first_order_form_and_anticommutator`. The cell prints three PASS lines.

In [3], a fermionic Fock space.

```
def sign_below(n, p):
    occupied_below = bin(n & ((1 << p) - 1)).count("1")  # count the 1-digits below p
    return -1 if occupied_below % 2 else 1
```

`sign_below(n, p)` is the sign $(-1)^s$ of Section 10.11, where s is the number of occupied modes below p in the pattern n. The operator written with two less-than signs shifts the binary digits of 1 to the left by p places, which gives 2^p; minus 1 gives the number whose p lowest binary digits are all 1. The operator `&` (binary *and*) keeps a binary digit only where both numbers have a 1, so `n & ...` keeps the digits of n below p. `bin` writes a whole number in binary as a text (for example `bin(9)` is the text `0b1001`), and `.count("1")` counts the digits 1 in it. `occupied_below % 2` is the remainder after division by 2: 1 for an odd count (sign -1), 0 for an even one (sign $+1$; a remainder 0 counts as false in the `if`).

```
def annihilate(p, state):
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:  # mode p is occupied in the pattern n
            new = n ^ (1 << p)  # the same pattern with mode p emptied
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result
```

`annihilate(p, state)` is f_p. A state is a dictionary `{pattern: amplitude}`, and the loop takes its pairs one by one. The test in the `if` line shifts n to the right by p binary places (the operator of

two greater-than signs) and keeps the last digit (`& 1`): it is 1 exactly when mode p is occupied. Then `^` (binary *exclusive or*) with 2^p flips digit p from 1 to 0: `new` is the pattern with mode p emptied. The amplitude times the sign is added to whatever `result` already holds for `new` (`result.get(new, 0)` is 0 when `new` is not yet there). Patterns in which mode p is empty contribute nothing.

```
def create(p, state):
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:  # mode p is empty in the pattern n
            new = n | (1 << p)  # the same pattern with mode p filled
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result
```

`create(p, state)` is f_p^*, the Hilbert adjoint of f_p: it acts only where mode p is empty, and `|` (binary *or*) with 2^p sets digit p to 1. The sign is the same $(-1)^s$.

```
def add(*terms):
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result
```

`add` forms a combination of states. The star in `*terms` lets the function take any number of arguments, each a pair `(coefficient, state)`; the result is the sum of the coefficients times the states, pattern by pattern. For example `add((1, a), (-1, b))` is the state $a - b$.

```
def inner(left, right):
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    return max((abs(a) for a in state.values()), default=0.0)
```

`inner(left, right)` is the positive inner product $\langle \text{left}|\text{right}\rangle = \sum_n \text{left}_n^* \, \text{right}_n$ of Section 10.11; `np.conj` is the complex conjugate, and patterns missing from `right` count as amplitude 0. `largest(state)` is the largest size of an amplitude; `default=0.0` returns 0 for the empty dictionary, the zero state. A check of the form "`largest(a - b)` is tiny" says that the states a and b agree.

```
VACUUM = {0: 1.0}
rng = np.random.default_rng(12345)  # random numbers with a fixed seed


def random_state(patterns=6):
    state = {}
    for n in rng.integers(0, 2 ** 16, size=patterns):
        state[int(n)] = complex(rng.normal(), rng.normal())
    length = np.sqrt(inner(state, state).real)
    return {n: a / length for n, a in state.items()}
```

`VACUUM` is the pattern 0 (every mode empty) with amplitude 1. `rng` is a generator of random numbers started with the fixed *seed* 12345, so that every run draws the same numbers and the notebook prints the same output every time. `random_state` picks a few patterns at random (`rng.integers(0, 2 ** 16, size=patterns)` draws whole numbers from 0 up to $2^{16} - 1$), gives each a complex amplitude whose real and imaginary parts are drawn from the normal distribution (`rng.normal()`), and divides by the length, so that the state is normalised.

```
phi = random_state()
worst = 0.0
for p in range(16):
    for q in range(16):
        # {f_p, f_q^*} phi = f_p f_q^* phi + f_q^* f_p phi must be phi (p = q) or 0
        both = add((1, annihilate(p, create(q, phi))),
                   (1, create(q, annihilate(p, phi))))
        expected = phi if p == q else {}
        worst = max(worst, largest(add((1, both), (-1, expected))))
        # {f_p, f_q} phi must be 0
        pair = add((1, annihilate(p, annihilate(q, phi))),
                   (1, annihilate(q, annihilate(p, phi))))
        worst = max(worst, largest(pair))
```

For a random state ϕ the loops test every pair of modes (p, q), $16 \times 16 = 256$ pairs. `both` is $\{f_p, f_q^*\}\phi = f_p f_q^* \phi + f_q^* f_p \phi$ (an operator applied to the result of another is a product of operators), which must be ϕ for $p = q$ and 0 otherwise, as the comment line says; `pair` is $\{f_p, f_q\}\phi$, which must be 0. `worst` keeps the largest deviation seen in the $2 \times 256 = 512$ relations.

```
report("largest violation of the 512 anticommutation relations", f"{worst:.1e}")
check(worst < 1e-14, "the Fock operators obey {f_p, f_q^*} = delta_pq, {f_p, f_q} = 0")
```

The RESULT line shows `0.0e+00`: the relations hold exactly, because the amplitudes are only multiplied by ± 1 and added. The check prints its PASS line.

In [4], the canonical field on the Fock space.

```
def psi(A, state):
    return annihilate(A - 1, state)


def psi_dagger(A, state):
    return add(*[(B[C - 1, A - 1], create(C - 1, state))
                 for C in range(1, 17) if B[C - 1, A - 1] != 0])
```

These are the operators of the positive realisation of Section 10.14: $\Psi_A = f_{A-1}$ (component A uses mode $A - 1$, because the computer counts from 0), and the canonical conjugate $\Psi_A^\dagger = \sum_C \chi_C B_{CA}$ with $\chi_C = f_{C-1}^*$. The list comprehension builds the pairs (coefficient B_{CA}, state f_{C-1}^* applied to the state) for the components C with $B_{CA} \neq 0$ (only one per column, because B has one nonzero entry in every column), and the star in `add(*[...])` passes the list as separate arguments.

```
worst = 0.0
for A in range(1, 17):
```

```
    for C_index in range(1, 17):
        both = add((1, psi(A, psi_dagger(C_index, phi))),
                   (1, psi_dagger(C_index, psi(A, phi))))
        worst = max(worst, largest(add((1, both), (-B[A - 1, C_index - 1], phi))))
report("largest violation of {Psi_A, Psi^dagger_C} = B_AC", f"{worst:.1e}")
check_record(worst < 1e-14, "on the Fock space {Psi_A, Psi^dagger_C} = B_AC (256 pairs)",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "canonical_anticommutator_B")
```

For all 256 pairs (A, C) the loop computes $\{\Psi_A, \Psi_C^\dagger\}\phi$ and subtracts $B_{AC}\phi$ (the name `C_index` avoids overwriting the matrix `C`). The largest deviation is printed as `0.0e+00`, and the check reproduces the record check `canonical_anticommutator_B`: the canonical rule holds on a positive Fock space once $\Psi^\dagger$ is defined as χB.

```
m_value = 1.3  # any mass and momenta (here with momenta along the extra times too)
k = {0: 1.1, 1: 0.4, 2: -0.7, 3: 0.2, 5: 0.9, 6: -0.3, 7: 0.5}
h_prime = m_value * C - 1j * sum(k_a * (C @ gamma[a]) for a, k_a in k.items())
h_mode = -1j * m_value * gamma[4] - sum(k_a * (gamma[4] @ gamma[a])
                                        for a, k_a in k.items())
check(np.max(np.abs(B @ h_prime - h_mode)) < 1e-14,
      "B (m C - i sum_a k_a C gamma^a) equals the mode Hamiltonian h")
```

A mass of 1.3 and momenta along all seven slice directions, including the extra times (the identity holds for all of them). `h_prime` is the Hamiltonian-density matrix $h' = mC - i\sum_a k_a C\gamma^{(x_a)}$ of Section 10.12 and `h_mode` the mode Hamiltonian $h = -im\gamma^{(x_4)} - \gamma^{(x_4)}\sum_a k_a\gamma^{(x_a)}$ of Section 10.3. The check confirms $Bh' = h$, derived line by line in Section 10.12.

```
def hamiltonian(state):
    M = B @ h_prime
    return add(*[(M[c_, d_], create(c_, annihilate(d_, state)))
                 for c_ in range(16) for d_ in range(16) if abs(M[c_, d_]) > 0])
```

`hamiltonian(state)` applies the energy operator $H = \Psi^\dagger h'\Psi = \sum_{A,D}\chi_C B_{CA}h'_{AD}\Psi_D = \sum_{C,D}(Bh')_{CD}\chi_C\Psi_D$ to a state: for each pair of modes (c, d) with a nonzero entry of $M = Bh'$ it takes $f_c^* f_d$ applied to the state (first empty mode d, then fill mode c) with the coefficient M_{cd}. The underscores in `c_` and `d_` only distinguish these names from others.

```
worst = 0.0
for c_ in range(16):
    commutator = add((1, hamiltonian(annihilate(c_, phi))),
                     (-1, annihilate(c_, hamiltonian(phi))))  # [H, Psi_c] phi
    expected = add(*[(-h_mode[c_, d_], annihilate(d_, phi)) for d_ in range(16)])
    worst = max(worst, largest(add((1, commutator), (-1, expected))))
report("largest violation of [H, Psi_c] = -(h Psi)_c", f"{worst:.1e}")
check_record(worst < 1e-12,
             "Heisenberg: i d4 Psi = h Psi, the classical wave equation",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "Heisenberg_equation_reproduces_field_equation")
```

For each component the loop computes $[H, \Psi_c]\phi = H\Psi_c\phi - \Psi_c H\phi$ and compares it with $-(h\Psi)_c\phi = -\sum_d h_{cd}\Psi_d\phi$, the result $[H, \Psi_c] = -(Ah'\Psi)_c$ of Section 10.12 with $A = B$ and $Bh' = h$. The largest deviation is printed as `1.6e-17`, rounding only. By the Heisenberg equation this is $i\,\partial_4\Psi = h\Psi$, the classical wave equation of the field, and the check reproduces the Wolfram record check `Heisenberg_equation_reproduces_field_equation`. The cell prints three PASS lines in all.

In [5], Theorem 10.2 on the recorded column.

```
u = np.zeros(16, dtype=complex)
u[6] = -1j / np.sqrt(2)  # u_7 (the computer counts from 0)
u[15] = 1 / np.sqrt(2)  # u_16
krein_norm_u = (u.conj() @ B @ u).real
report("u^dagger u and u^dagger B u", f"{(u.conj() @ u).real:.12f} and "
       f"{krein_norm_u:.12f}")
```

`np.zeros(16, dtype=complex)` is a column of 16 complex zeros; entries 7 and 16 (positions 6 and 15) are set to $-i/\sqrt{2}$ and $1/\sqrt{2}$, the column of the proof of Theorem 10.2. The RESULT line prints its ordinary squared length $u^\dagger u = 1.000000000000$ and its Krein norm $u^\dagger B u = -1.000000000000$.

```
def X(state):  # X = sum_A conj(u_A) Psi_A
    return add(*[(np.conj(u[A - 1]), psi(A, state)) for A in range(1, 17)])


def X_hilbert(state):  # X^* = sum_A u_A chi_A (the Hilbert adjoint of X)
    return add(*[(u[A - 1], create(A - 1, state)) for A in range(1, 17)])


def X_canonical(state):  # X^dagger = sum_A u_A Psi^dagger_A (canonical conjugate)
    return add(*[(u[A - 1], psi_dagger(A, state)) for A in range(1, 17)])
```

The three operators of Section 10.14: $X = \sum_A u_A^* \Psi_A$, its Hilbert adjoint $X^* = \sum_A u_A \chi_A$ (with $\chi_A = f^*_{A-1}$, the function `create`), and its canonical conjugate $X^\dagger = \sum_A u_A \Psi^\dagger_A$, as the comments say.

```
hilbert = add((1, X(X_hilbert(phi))), (1, X_hilbert(X(phi))))
canonical = add((1, X(X_canonical(phi))), (1, X_canonical(X(phi))))
hilbert_ok = largest(add((1, hilbert), (-1, phi))) < 1e-14  # {X, X^*} = +1
canonical_ok = largest(add((1, canonical), (1, phi))) < 1e-14  # {X, X^dagger} = -1
check_record(np.max(np.abs(B @ u + u)) < 1e-15 and abs(krein_norm_u + 1) < 1e-15
             and hilbert_ok and canonical_ok,
             "B u = -u: {X, X^*} = +1 but the canonical {X, X^dagger} = -1",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "no_positive_inner_product")
```

`hilbert` is $\{X, X^*\}\phi$ and `canonical` is $\{X, X^\dagger\}\phi$. The check requires $Bu = -u$, $u^\dagger B u = -1$, $\{X, X^*\}\phi = +\phi$ and $\{X, X^\dagger\}\phi = -\phi$, and reproduces the record check `no_positive_inner_product`: on a positive space the canonical conjugate is $-X^*$, not X^*, exactly as derived in Section 10.14.

In [6], positivity made visible.

```
lengths = []
for _ in range(300):
    state = random_state(patterns=4)
    x_part, x_star_part = X(state), X_hilbert(state)
    lengths.append((inner(x_part, x_part).real, inner(x_star_part, x_star_part).real))
lengths = np.array(lengths)
sums = lengths.sum(axis=1)
```

For 300 random normalised states (each a combination of 4 random patterns) the loop stores the pair of squared lengths $(\|X\phi\|^2, \|X^*\phi\|^2)$. `np.array` turns the list of pairs into a table with 300 rows and 2 columns, and `.sum(axis=1)` adds the two numbers of each row.

```
report("smallest and largest |X phi|^2 + |X^* phi|^2",
       f"{sums.min():.12f} and {sums.max():.12f}")
check(np.all(lengths >= 0) and np.max(np.abs(sums - 1)) < 1e-12,
      "positivity: |X phi|^2 + |X^* phi|^2 = 1 for every random state")
```

By Section 10.11, $\|X\phi\|^2 + \|X^*\phi\|^2 = \langle\phi|\{X, X^*\}\phi\rangle = \langle\phi|\phi\rangle = 1$. The printed smallest and largest sums are both `1.000000000000`, and the check requires every squared length to be ≥ 0 and every sum to be 1.

```
columns = rng.normal(size=(5000, 16)) + 1j * rng.normal(size=(5000, 16))
columns = columns / np.linalg.norm(columns, axis=1, keepdims=True)  # unit length
krein_norms = np.einsum("ni,ij,nj->n", columns.conj(), B, columns).real
```

5000 random complex columns with 16 entries (one per row of the table `columns`) are drawn and each is divided by its length (`np.linalg.norm(..., axis=1, keepdims=True)` computes the length of every row and keeps it as a column, so that the division acts row by row). `np.einsum("ni,ij,nj->n", ...)` computes, for each row n, the sum $\sum_{i,j} v_i^* B_{ij} v_j = v^\dagger B v$: the letters name the indices, and the indices that do not appear after the arrow are summed over.

```
report("smallest and largest Krein norm of the 5000 unit columns",
       f"{krein_norms.min():.3f} and {krein_norms.max():.3f}")
check(krein_norms.min() < -0.5 and krein_norms.max() > 0.5
      and np.all(np.abs(krein_norms) <= 1 + 1e-12),
      "random unit columns have Krein norms of both signs, all between -1 and 1")
```

The printed range is from `-0.777` to `0.704`. A Krein norm of a unit column can never lie outside $[-1, 1]$, because the eigenvalues of B are ± 1; the check requires values below -0.5 and above 0.5 (both signs) and none outside that interval.

```
fig, ax = plt.subplots(figsize=(6.0, 4.6))
ax.plot(lengths[:, 0], lengths[:, 1], "o", color="#2a78d6", markersize=4,
        label="300 random states $\\phi$")
line = np.linspace(-1.4, 1.2, 2)
ax.plot(line, 1 - line, color="#52514e", linewidth=1, label="$x + y = 1$")
ax.plot(line, -1 - line, "--", color="#eb6834", linewidth=2,
        label="$x + y = -1$ (canonical rule)")
```

The first picture: each random state is a blue dot at $x = \|X\phi\|^2$, $y = \|X^*\phi\|^2$. `line` holds the two end points -1.4 and 1.2 of a horizontal range, and the two `plot` calls draw the straight lines $y = 1 - x$ (grey) and $y = -1 - x$ (orange, dashed): the line on which the dots lie, and the line on which they would have to lie if the canonical conjugate were the Hilbert adjoint.

```
ax.set_xlim(-1.4, 1.2)
ax.set_ylim(-1.4, 1.2)
ax.set_xlabel("$x = \\|X\\phi\\|^2$")
ax.set_ylabel("$y = \\|X^*\\phi\\|^2$")
ax.set_title("Squared lengths are never negative")
ax.legend(loc="lower left")
save_figure(fig, "positivity",
...)
```

The ranges of both axes are fixed so that the negative quarter, where the dashed line runs, is visible. Figure 1 of Notebook 10b shows all 300 dots on the grey line, inside the quarter where both squared lengths are positive, and the orange dashed line far away in the region of negative values that no state can reach: this is Theorem 10.2 in one picture.

```
fig, ax = plt.subplots()
ax.set_axisbelow(True)  # draw the grid lines behind the bars
ax.hist(krein_norms, bins=50, range=(-1, 1), color="#2a78d6", edgecolor="white")
ax.axvline(0.0, color="#52514e", linewidth=1)
ax.set_xlabel("Krein norm $v^\\dagger B v$ of a random column with $v^\\dagger v = 1$")
ax.set_ylabel("number of columns (of 5000)")
ax.set_title("The Krein form is indefinite: both signs occur")
save_figure(fig, "krein_norms_random",
...)
```

The second picture is a **histogram**: `ax.hist` divides the interval from -1 to 1 into 50 equal bins and draws one bar per bin, as tall as the number of Krein norms that fall into it. Figure 2 of Notebook 10b shows a bell-shaped distribution centred at 0 and symmetric: every random unit column has the ordinary length 1, but its Krein norm is positive about as often as negative, and usually small, because a random column mixes the eight directions of positive and the eight of negative charge.

In [7], the Krein-Fock realisation.

```
def orthonormal_columns(P):
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T
```

This is the Gram-Schmidt function `orthonormal_basis` of In [10] of Notebook 10a under another name (Section 10.10): it returns orthonormal columns that span the range of the matrix P, removing from each column of P its parts along the columns found so far (twice, to remove rounding errors) and keeping what is left, divided by its length, when anything is left.

```
plus_modes = orthonormal_columns((np.eye(16) + B) / 2)  # B u = +u, 8 columns
minus_modes = orthonormal_columns((np.eye(16) - B) / 2)  # B u = -u, 8 columns
U = np.hstack([plus_modes, minus_modes]).astype(complex)  # 16 modes as columns
epsilon = np.array([1.0] * 8 + [-1.0] * 8)  # their Krein norms
```

$\frac{1}{2}(I_{16} \pm B)$ are the projectors onto the eigenspaces of B for $+1$ and -1 (by $BB = I_{16}$, the argument of Section 10.5). Their orthonormal columns are 8 modes with $Bu = u$ and 8 with $Bu = -u$; `U` holds the 16 modes as its columns, and `epsilon` their Krein norms $+1$ (eight times) and -1 (eight times).

```
cosh_t, sinh_t = 5 / 4, 3 / 4  # a Krein boost of the modes 1 and 9
first, ninth = U[:, 0].copy(), U[:, 8].copy()
U[:, 0] = cosh_t * first + sinh_t * ninth
U[:, 8] = sinh_t * first + cosh_t * ninth
E = np.diag(epsilon)
```

The Krein boost of Section 10.14 with $\cosh t = \frac{5}{4}$, $\sinh t = \frac{3}{4}$ replaces mode 1 by $\frac{5}{4}u_1 + \frac{3}{4}u_9$ and mode 9 by $\frac{3}{4}u_1 + \frac{5}{4}u_9$. `.copy()` keeps the old columns, so that the second line still uses the old first column. `E` is the diagonal matrix $E = \mathrm{diag}(\epsilon_1, \ldots, \epsilon_{16})$.

```
gram_ok = np.max(np.abs(U.conj().T @ B @ U - E)) < 1e-14  # U^dagger B U = E
complete_ok = np.max(np.abs(U @ E @ U.conj().T - B)) < 1e-14  # U E U^dagger = B
report("squared ordinary length of the boosted mode 1",
       f"{np.linalg.norm(U[:, 0]) ** 2:.6f}")
check(gram_ok and complete_ok,
      "Krein-orthonormal modes: U^dagger B U = E and U E U^dagger = B")
```

The check confirms the two matrix identities of Section 10.14, $U^\dagger B U = E$ (the boosted modes are still Krein-orthonormal) and $UEU^\dagger = B$. The RESULT line prints the squared ordinary length of the boosted mode, `2.125000`, which is $\frac{34}{16}$.

```
def b(n, state):  # b_n = f_n
    return annihilate(n, state)


def b_dagger(n, state):  # b_n^dagger = epsilon_n f_n^*
    return add((epsilon[n], create(n, state)))
```

On the positive Fock space the Krein-Fock operators are realised as $b_n = f_n$ and $b_n^\dagger = \epsilon_n f_n^*$, so that $\{b_n, b_n^\dagger\} = \epsilon_n \{f_n, f_n^*\} = \epsilon_n$, the rule of Section 10.14.

```
def krein_psi(A, state):  # Psi_A = sum_n (u_n)_A b_n
    return add(*[(U[A - 1, n], b(n, state)) for n in range(16)])
```

```
def krein_psi_dagger(A, state):  # Psi^dagger_A = sum_n conj((u_n)_A) b_n^dagger
    return add(*[(np.conj(U[A - 1, n]), b_dagger(n, state)) for n in range(16)])
```

The field $\Psi = \sum_n u_n b_n$ and its conjugate $\Psi^\dagger = \sum_n u_n^\dagger b_n^\dagger$, component by component; `U[A - 1, n]` is entry A of the mode u_{n+1}.

```
worst = 0.0
for A in range(1, 17):
    for C_index in range(1, 17):
        both = add((1, krein_psi(A, krein_psi_dagger(C_index, phi))),
                   (1, krein_psi_dagger(C_index, krein_psi(A, phi))))
        worst = max(worst, largest(add((1, both), (-B[A - 1, C_index - 1], phi))))
report("Krein-Fock: largest violation of {Psi_A, Psi^dagger_C} = B_AC", f"{worst:.1e}")
check(worst < 1e-13, "Krein-Fock realisation: {Psi, Psi^dagger} = B again")
```

As in In [4], all 256 anticommutators are compared with B_{AC}. The largest deviation is printed as `1.6e-16`: the Krein-Fock realisation gives the same canonical rule. The cell prints two PASS lines.

In [8], the expectation-value rule.

```
def expectation(n, M):
    excited = b_dagger(n, VACUUM)  # b_n^dagger |0>
    acted = {}
    for A in range(1, 17):
        psi_part = krein_psi(A, excited)
        for C_index in range(1, 17):
            if M[C_index - 1, A - 1] != 0:
                acted = add((1, acted), (M[C_index - 1, A - 1],
                                         krein_psi_dagger(C_index, psi_part)))
    numerator = inner(VACUUM, b(n, acted))  # <0| b_n ... |0>, a number
    norm = inner(VACUUM, b(n, excited))  # <0| b_n b_n^dagger |0> = epsilon_n
    return numerator / norm
```

`expectation(n, M)` computes the left side of the normalised rule of Section 10.14. `excited` is the state $b_n^\dagger|0\rangle$. The double loop applies $\Psi^\dagger M \Psi = \sum_{C,A} M_{CA} \Psi_C^\dagger \Psi_A$ to it: for each A it computes Ψ_A applied to the excited state once (`psi_part`), and adds $M_{CA}\Psi_C^\dagger$ applied to that. Then b_n is applied and the result is projected on the vacuum: `numerator` is $\langle 0|b_n(\Psi^\dagger M \Psi)b_n^\dagger|0\rangle$ and `norm` is $\langle 0|b_n b_n^\dagger|0\rangle = \epsilon_n$, as the comments say. The function returns their quotient.

```
rule_errors, naive_errors, points = [], [], []
for _ in range(60):
    M = rng.normal(size=(16, 16)) + 1j * rng.normal(size=(16, 16))
    for n in (3, 0):  # mode 4 (unboosted) and mode 1 (boosted), counted from 0
        value = expectation(n, M)
        rule = epsilon[n] * (U[:, n].conj() @ M @ U[:, n])
        naive = U[:, n].conj() @ B @ M @ U[:, n]
        rule_errors.append(abs(value - rule))
        if n == 3:
```

```
            naive_errors.append(abs(value - naive))
        else:
            points.append((value.real, rule.real, naive.real))
points = np.array(points)
```

For 60 random complex matrices M and two modes, mode 4 (not boosted, an eigenvector of B) and mode 1 (boosted), the loop compares the Fock-space value with the rule $\epsilon_n u_n^\dagger M u_n$ and with the form $u_n^\dagger BMu_n$. The deviations from the rule are collected for both modes; the deviations from $u^\dagger BMu$ only for mode 4, where it must also hold; for the boosted mode the three real parts are stored for the figure.

```
naive_boosted = np.max(np.abs(points[:, 0] - points[:, 2]))
report("largest |Fock value - epsilon u^dagger M u| (both modes)",
       f"{max(rule_errors):.1e}")
report("boosted mode: largest |Fock value - u^dagger B M u|", f"{naive_boosted:.2f}")
check_record(max(rule_errors) < 1e-12 and max(naive_errors) < 1e-12
             and naive_boosted > 0.1,
             "expectation rule eps u^dagger M u; u^dagger B M u fails if boosted",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "expectation_value_rule")
```

The printed lines show the deviation from the rule, `1.3e-15` (rounding only), and the deviation of the form $u^\dagger BMu$ for the boosted mode, `5.82`: large. The check requires both and reproduces the record check `expectation_value_rule`.

In [9], two pictures of the Krein-Fock realisation.

```
fig, ax = plt.subplots(figsize=(6.0, 6.0))
angle = np.linspace(0, 2 * np.pi, 400)
ax.plot(np.cos(angle), np.sin(angle), color="#52514e", linewidth=1.5,
        label="ordinary length 1: $a^2 + b^2 = 1$")
```

In the plane of the columns $au_1 + bu_9$ (with $Bu_1 = u_1$, $Bu_9 = -u_9$, both of length 1 and orthogonal) the ordinary squared length is $a^2 + b^2$ and the Krein norm is $a^2 - b^2$. The points $(\cos\theta, \sin\theta)$ for 400 angles from 0 to 2π draw the grey unit circle, the columns of ordinary length 1.

```
t = np.linspace(-1.6, 1.6, 400)
for side in (1, -1):  # the two branches of each hyperbola
    ax.plot(side * np.cosh(t), np.sinh(t), color="#2a78d6", linewidth=2,
            label="Krein norm $+1$: $a^2 - b^2 = 1$" if side == 1 else None)
    ax.plot(np.sinh(t), side * np.cosh(t), color="#eb6834", linewidth=2,
            label="Krein norm $-1$: $a^2 - b^2 = -1$" if side == 1 else None)
```

Since $\cosh^2 t - \sinh^2 t = 1$, the points $(\pm\cosh t, \sinh t)$ lie on the **hyperbola** $a^2 - b^2 = 1$ (blue, the columns of Krein norm $+1$) and the points $(\sinh t, \pm\cosh t)$ on $a^2 - b^2 = -1$ (orange). Each hyperbola has two branches, drawn by the two passes of the loop; only the first pass labels the curves (`None` gives no legend entry).

```
ax.plot([0, 1, 0], [1, 0, 0], "s", color="#52514e", markersize=7)
ax.annotate("$u_1$", (1.0, 0.0), xytext=(1.05, -0.3))
ax.annotate("$u_9$", (0.0, 1.0), xytext=(-0.35, 1.05))
ax.plot([1.25, 0.75], [0.75, 1.25], "o", color="#1baf7a", markersize=9,
        label="boosted modes $(5/4, 3/4)$, $(3/4, 5/4)$")
for x_end, y_end in ((1.25, 0.75), (0.75, 1.25)):
    ax.annotate("", xy=(x_end, y_end), xytext=(0, 0),
                arrowprops={"arrowstyle": "->", "color": "#1baf7a"})
```

Grey squares mark the origin and the two unboosted modes, $u_1 = (1, 0)$ and $u_9 = (0, 1)$, which `annotate` labels. Green dots mark the boosted modes $(\frac{5}{4}, \frac{3}{4})$ and $(\frac{3}{4}, \frac{5}{4})$, and an annotation without text but with `arrowprops` draws an arrow from the origin to each.

```
ax.set_xlim(-2.6, 2.6)
ax.set_ylim(-2.6, 2.6)
ax.set_aspect("equal")
ax.set_xlabel("coefficient $a$ of $u_1$")
ax.set_ylabel("coefficient $b$ of $u_9$")
ax.set_title("Ordinary length versus Krein norm in one plane")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "krein_plane",
...)
```

`set_aspect("equal")` gives both axes the same scale, so that the circle looks round. Figure 3 of Notebook 10b shows the boosted modes outside the unit circle (their ordinary length is larger than 1) but exactly on the hyperbolas of Krein norm $+1$ and -1: a Krein boost keeps the Krein norms and changes the ordinary lengths, as a Lorentz boost keeps $t^2 - x^2$ and changes t and x.

```
fig, ax = plt.subplots(figsize=(6.0, 5.0))
ax.plot(points[:, 1], points[:, 0], "o", color="#1baf7a", markersize=5,
        label="against the rule $\\epsilon\\,u^\\dagger M u$")
ax.plot(points[:, 2], points[:, 0], "x", color="#eb6834", markersize=6,
        label="against $u^\\dagger B M u$")
low, high = points.min() - 1, points.max() + 1
ax.plot([low, high], [low, high], color="#52514e", linewidth=1)
ax.set_xlabel("value of the formula (real part)")
ax.set_ylabel("value computed on the Fock space (real part)")
ax.set_title("The boosted mode: which formula is right?")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "expectation_rule",
...)
```

The second picture plots, for the boosted mode and the 60 random matrices, the Fock-space value (vertical) against the rule (green dots) and against $u^\dagger BMu$ (orange crosses), with the grey diagonal from a little below the smallest to a little above the largest value. Figure 4 of Notebook 10b shows the green dots exactly on the diagonal and the orange crosses scattered around it: the rule of the record is right, the form $u^\dagger BMu$ is right only for modes that are eigenvectors of B.

In [10], maps that keep the canonical rule, and the conjugation.

```
pairing = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
rows = pairing["data"]["Krein_signs_M_B_Mdagger"]
maps = {"Gamma": Gamma}
for a in range(8):
    maps[f"gamma^x{a}"] = gamma[a]
    maps[f"P_x{a}"] = Gamma @ gamma[a]  # the reflection P_a = Gamma gamma^(x_a)
```

The pairing record is read, and `rows` is its list of 17 entries, each holding the name of a map and the recorded sign. The dictionary `maps` pairs the same names with the matrices: the chirality, the eight gammas and the eight reflections $P_a = \Gamma\gamma^{(x_a)}$.

```
signs, agree = [], True
for row in rows:
    M = maps[row["map"]]
    transformed = M @ B @ M.conj().T  # M B M^dagger
    sign = (1 if np.array_equal(transformed, B)
            else -1 if np.array_equal(transformed, -B) else 0)
    signs.append(sign)
    agree = agree and sign == row["sign"]
names = [row["map"] for row in rows]  # the names of the 17 maps in the record
```

For each recorded map the loop computes $MBM^\dagger$ exactly and reads off its sign: $+1$ if it equals B, -1 if it equals $-B$, and 0 otherwise (which never happens). `agree` stays true while every computed sign equals the recorded one.

```
say("map: sign sigma in M B M^dagger = sigma B")
say(", ".join(f"{name}: {s:+d}" for name, s in zip(names, signs)))
check_record(agree and len(rows) == 17,
             "the Krein signs of all 17 recorded maps are reproduced",
             record="Revision/pairing/pairing-theory.json, data "
                    "Krein_signs_M_B_Mdagger")
```

The format code `+d` prints a whole number with its sign. The printed list is the table of Section 10.15: `Gamma: -1`, the gammas of x_0 to x_3 and x_7 with $+1$, those of x_5 to x_7 with -1, the reflections P_1 to P_4 and P_0 with -1 and P_5 to P_7 with $+1$. The check reproduces the record's data.

```
check_record(np.array_equal(Gamma @ B @ Gamma.T, -B),
             "the chirality image Gamma Psi carries the Krein metric -B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_krein_metric")
check_record(np.array_equal(gamma[0] @ B @ gamma[0].T, B),
             "the mirror image gamma^(x0) Psi keeps +B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.T2_image_keeps_B")
```

The two maps of the pairing theorems separately: $\Gamma B\Gamma^T = -B$ and $\gamma^{(x_0)}B(\gamma^{(x_0)})^T = +B$ (for real matrices the conjugate transpose is the transpose). They reproduce the sympy record checks `Q.image_krein_metric` and `Q.T2_image_keeps_B`.

```
identity = np.eye(16)
conjugation_identity = identity @ B.T @ identity.conj().T  # M = I
conjugation_gamma = Gamma @ B.T @ Gamma.conj().T  # M = Gamma
check(np.array_equal(B.T, -B), "B^T = -B (B is Hermitian and purely imaginary)")
check_record(np.array_equal(conjugation_gamma, B)
             and np.array_equal(conjugation_identity, -B),
             "Psi -> Gamma Psi^(dagger T) keeps B; with M = I one gets -B",
             record="Revision/lead_checks/reports/charge-conjugation-and-u1.json, "
                    "check quantum_charge_conjugation_unitary_type")
```

The conjugations of the quantised field, $\Psi \to M\Psi^{\dagger T}$, change the canonical matrix into $MB^TM^\dagger$ (Section 10.15). The cell checks $B^T = -B$, then $\Gamma B^T \Gamma^\dagger = +B$ and $I_{16} B^T I_{16} = -B$, and reproduces the lead check `quantum_charge_conjugation_unitary_type`.

```
lead_file = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
lead_verdicts = list(REPORT_CHECKS[lead_file].values())  # read by check_record
passed = lead_verdicts.count("PASS")
report("checks of charge-conjugation-and-u1.json with the verdict PASS",
       f"{passed} of {len(lead_verdicts)}")
check(lead_verdicts == ["PASS"] * 12,
      "the lead-check report charge-conjugation-and-u1.json holds 12 checks, all PASS")
```

The previous `check_record` has just read the lead-check report into the dictionary `REPORT_CHECKS` of In [2]. `REPORT_CHECKS[lead_file].values()` are the verdicts of all its checks, and `list(...)` makes a list of them; `lead_verdicts.count("PASS")` counts the entries equal to `PASS`. The RESULT line prints `12 of 12`, and the check confirms that the report holds exactly 12 checks, all with the verdict PASS (`["PASS"] * 12` is the list of twelve such entries): the number of checks that the table of records in Section 10.1 gives for this report.

```
mass_reversed = all(np.array_equal(Gamma @ gamma[a] @ Gamma, -gamma[a])
                    for a in range(8))
check(mass_reversed, "Gamma anticommutes with every gamma: the conjugation reverses m")
```

$\Gamma\gamma^{(x_a)}\Gamma = -\gamma^{(x_a)}$ for all eight gammas, which is the reason why $\Gamma\Psi^*$ obeys the equation of mass $-m$ (Section 10.15). The cell prints seven PASS lines.

In [11], the signs as a bar chart.

```
labels = [row["map"].replace("gamma^x", "g").replace("P_x", "P") for row in rows]
labels = [lab if lab != "Gamma" else "Gam" for lab in labels]
colours = ["#2a78d6" if s > 0 else "#eb6834" for s in signs]
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0),
                         gridspec_kw={"width_ratios": [4, 1]})
for ax in axes:
    ax.set_axisbelow(True)  # draw the grid lines behind the bars
```

Short labels are made from the recorded names (`gamma^x1` becomes `g1`, `P_x1` becomes `P1`, and `Gamma` becomes `Gam`), and each bar gets blue for the sign $+1$ and orange for -1. The figure has two panels; `width_ratios` makes the first four times as wide as the second.

```
axes[0].bar(range(len(signs)), signs, color=colours, width=0.7)
axes[0].set_xticks(range(len(signs)), labels)
axes[0].axhline(0, color="#52514e", linewidth=1)
axes[0].set_ylim(-1.4, 1.4)
axes[0].set_yticks([-1, 0, 1])
axes[0].set_ylabel("sign $\\sigma$ in $M B M^\\dagger = \\sigma B$")
axes[0].set_title("linear maps $\\Psi \\to M\\Psi$")
```

The left panel draws one bar of height $+1$ or -1 per map, labelled under the axis.

```
conjugation_signs = [-1, 1]  # M = I gives -B, M = Gamma gives +B
axes[1].bar([0, 1], conjugation_signs, color=["#eb6834", "#2a78d6"], width=0.6)
axes[1].set_xticks([0, 1], ["$M = I$", "$M = \\Gamma$"])
axes[1].axhline(0, color="#52514e", linewidth=1)
axes[1].set_ylim(-1.4, 1.4)
axes[1].set_yticks([-1, 0, 1])
axes[1].set_title("$\\Psi \\to M\\Psi^{\\dagger T}$")
save_figure(fig, "krein_signs",
...)
```

The right panel draws the two conjugations, with the signs just checked in In [10]. Figure 5 of Notebook 10b shows five orange bars among the gammas and reflections (Γ, $\gamma^{(x_5)}$ to $\gamma^{(x_7)}$ and five reflections) and, on the right, the orange bar of $M = I$ next to the blue bar of $M = \Gamma$: only the conjugation $\Psi \to \Gamma\Psi^{\dagger T}$, which reverses the mass, keeps the canonical rule.

In [12], the last check.

```
for name in ("10b_1_positivity.png", "10b_2_krein_norms_random.png",
             "10b_3_krein_plane.png", "10b_4_expectation_rule.png",
             "10b_5_krein_signs.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The loop checks that each of the five figure files exists, and the last line prints ALL 25 CHECKS PASSED (notebook 10b): the five figure checks and the 20 checks of the cells before (3 in In [2], 1 in In [3], 3 in In [4], 1 in In [5], 2 in In [6], 2 in In [7], 1 in In [8] and 7 in In [10]).

10.20 The good sector: a positive Fock space for one momentum

Theorem 10.2 forbids a positive inner product with $\Psi^\dagger$ read as the Hilbert adjoint, and Theorem 10.1 says that even the one-particle waves of real frequency mix the two signs of charge four and four. Yet the Revision record constructs, in the **good sector** (no momentum along the extra times, $k_5 = k_6 = k_7 = 0$), a POSITIVE quantum state space for each single momentum with frozen coefficients (flat-frame plane waves, in which the deflation of the extra times does not enter), with particles and antiparticles of positive energy. This section derives it line by line; the scope is stated at its end.

The waves of one good-sector momentum. In the good sector the mode Hamiltonian is

$$h = m\beta + \sum_{a=1,2,3,8} k_a \alpha^a, \qquad \beta = -i\gamma^{(x_4)}, \qquad \alpha^a = -\gamma^{(x_4)}\gamma^{(x_a)}.$$

All five matrices are Hermitian (Section 10.4), so h is Hermitian. By Section 10.4, $hh = E^2 I_{16}$ with $E = \sqrt{m^2 + k_1^2 + k_2^2 + k_3^2 + k_0^2} > 0$ and $\operatorname{tr} h = 0$, so h has the eigenvalue $+E$ eight times and $-E$ eight times. Choose orthonormal eigenvectors $u_1, \ldots, u_8$ with $hu_s = Eu_s$ and $v_1, \ldots, v_8$ with $hv_s = -Ev_s$ (for example by the Gram-Schmidt method applied to the columns of the projectors $\frac{1}{2}(I_{16} \pm h/E)$ of Section 10.5). A u and a v are orthogonal:

$$E\, v^\dagger u = v^\dagger h u = (h^\dagger v)^\dagger u = (hv)^\dagger u = -E\, v^\dagger u,$$

by $hu = Eu$, the rule $(Xv)^\dagger = v^\dagger X^\dagger$, $h^\dagger = h$ and $hv = -Ev$ (E is real); so $2E\, v^\dagger u = 0$ and $v^\dagger u = 0$. The 16×16 matrix $W = (u_1, \ldots, u_8, v_1, \ldots, v_8)$ therefore has orthonormal columns, $W^\dagger W = I_{16}$. Then W is invertible with $W^{-1} = W^\dagger$, and so also $WW^\dagger = I_{16}$, which written out is the **completeness** relation $\sum_s u_s u_s^\dagger + \sum_s v_s v_s^\dagger = I_{16}$.

The field on a positive Fock space. Take 16 fermion modes on a positive Fock space, called $b_1, \ldots, b_8$ (particles) and $d_1, \ldots, d_8$ (antiparticles), with $\{b_s, b_{s'}^*\} = \{d_s, d_{s'}^*\} = \delta_{ss'}$ and all other anticommutators zero, and put

$$\Psi = \sum_s \left(u_s\, b_s + v_s\, d_s^*\right), \qquad \chi = \sum_s \left(u_s^\dagger\, b_s^* + v_s^\dagger\, d_s\right), \qquad \Psi^\dagger = \chi B.$$

χ (a row of 16 operators) is the Hilbert adjoint of Ψ, and $\Psi^\dagger = \chi B$ is the positive realisation of Section 10.14. Line by line:

$$\{\Psi_A, \chi_C\} = \sum_s (u_s)_A (u_s)_C^* \{b_s, b_s^*\} + \sum_s (v_s)_A (v_s)_C^* \{d_s^*, d_s\}$$

$$= \sum_s (u_s)_A (u_s)_C^* + \sum_s (v_s)_A (v_s)_C^* = (WW^\dagger)_{AC} = \delta_{AC}.$$

The first step keeps only the anticommutators of an operator with its own adjoint (all others vanish); the second uses $\{b_s, b_s^*\} = \{d_s^*, d_s\} = 1$; the third recognises the product $WW^\dagger$; the fourth is completeness. Then, exactly as in Section 10.14, $\{\Psi_A, \Psi_C^\dagger\} = \sum_D \{\Psi_A, \chi_D\} B_{DC} = B_{AC}$: the canonical rule holds.

The energy, line by line. By Section 10.12 the energy of the momentum is $H = \Psi^\dagger h' \Psi = \chi B h' \Psi = \chi h \Psi$. Since $h\Psi = \sum_s (Eu_s b_s - Ev_s d_s^*)$,

$$\chi h \Psi = \sum_{s,s'} \left(u_{s'}^\dagger b_{s'}^* + v_{s'}^\dagger d_{s'}\right)\left(Eu_s b_s - Ev_s d_s^*\right) = \sum_s E\, b_s^* b_s - \sum_s E\, d_s d_s^*.$$

The second step multiplies out and uses the orthonormality: $u_{s'}^\dagger u_s = v_{s'}^\dagger v_s = \delta_{ss'}$ and $u_{s'}^\dagger v_s = v_{s'}^\dagger u_s = 0$ (the numbers stand in front of the operators). Now $d_s d_s^* = 1 - d_s^* d_s$ by the anticommutator, for each of the 8 values of s:

$$H = \sum_s E\left(b_s^* b_s + d_s^* d_s\right) - 8E.$$

The **vacuum** $|0\rangle$ is the state with $b_s|0\rangle = d_s|0\rangle = 0$ for all s. Its energy is $-8E$: in the old picture of Dirac, every one of the 8 negative-energy levels $-E$ is filled, the filled **Dirac sea**. **Normal ordering** is the prescription that drops this constant: the normal-ordered energy is the energy minus that of the vacuum,

$$:H: = \sum_s E\left(b_s^* b_s + d_s^* d_s\right) \geq 0.$$

A **particle** $b_s^*|0\rangle$ and an **antiparticle** $d_s^*|0\rangle$ (a hole in the filled sea) both have the energy $+E > 0$.

The charge. $Q = \Psi^\dagger B \Psi = \chi B B \Psi = \chi\Psi$, and the same steps with h replaced by I_{16} give

$$\chi\Psi = \sum_s \left(b_s^* b_s + d_s d_s^*\right) = \sum_s \left(b_s^* b_s - d_s^* d_s\right) + 8.$$

After normal ordering a particle has the charge $+1$ and an antiparticle -1.

Counting the states of one momentum. A pattern with N_b occupied particle modes and N_d occupied antiparticle modes is an eigenstate of both operators: its normal-ordered energy is $E(N_b + N_d)$ and its normal-ordered charge is $N_b - N_d$ (each $b_s^* b_s$ and $d_s^* d_s$ counts whether its mode is occupied). The number of ways to choose j occupied modes out of 8 is the **binomial coefficient** $\binom{8}{j} = \frac{8!}{j!\,(8-j)!}$, where $n! = 1 \cdot 2 \cdots n$ and $0! = 1$; for example $\binom{8}{0} = 1$, $\binom{8}{1} = 8$ and $\binom{8}{4} = \frac{40320}{24 \cdot 24} = 70$. So $\binom{8}{N_b}\binom{8}{N_d}$ patterns have the given N_b and N_d, and the total is $\sum_{N_b}\binom{8}{N_b}\sum_{N_d}\binom{8}{N_d} = 2^8 \cdot 2^8 = 65536$. Only the vacuum ($N_b = N_d = 0$) has the energy 0; $8 + 8 = 16$ states have one quantum; the largest group, $\binom{8}{4}^2 = 4900$ states, has 8 quanta and the charge 0. No state has a negative normal-ordered energy.

statement	**status**	**where it is verified**
good sector, one momentum with frozen coefficients: positive Fock space, $\Psi^\dagger = \chi B$, $\{\Psi, \Psi^\dagger\} = B$; vacuum $-8E$; every quantum $+E$; charges ± 1	PROVED (derivation above) and checked for $m = 2$, $k = (k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$ (vacuum -40) and for $m = 3$, $k_1 = 4$, $E = 5$	`python-field-theory.json`, check `good_sector_positive_fock_realisation`; `wolfram-field-theory.json`, check `Fock_space_good_sector_example`
normal ordering (dropping the vacuum value)	ASSUMED	a prescription, as in Dirac's theory

statement	status	where it is verified
scope: single good-sector momenta, in flat 4+4 space or in the frozen-coefficient model of Section 10.3 (ASSUMED: it leaves out the hidden-direction terms, including the connection term $3H\gamma^{(x_0)}$)	OPEN beyond it	a positive-norm Hilbert space for the whole field in the deflating background, the extra-time sector and the operator theory for $\lambda \neq 0$ beyond the finite model of Section 10.21 are not constructed; in the author's metric the curved good-sector mode operator is Hermitian only up to a boundary term at $z = \pi/2$ (Section 10.27; Revision theory document, section 11)

10.21 The expectation-value rule and the energy-momentum tensor operator

The rule in the positive realisation. Every classical bilinear $\Psi^\dagger M \Psi$ becomes the operator $\chi BM\Psi$. Write $N = BM$ and insert the expansion:

$$\chi N\Psi = \sum_{s,s'} \Big[(u_{s'}^\dagger N u_s)\, b_{s'}^* b_s + (u_{s'}^\dagger N v_s)\, b_{s'}^* d_s^* + (v_{s'}^\dagger N u_s)\, d_{s'} b_s + (v_{s'}^\dagger N v_s)\, d_{s'} d_s^* \Big].$$

In the particle state $b_1^*|0\rangle$ the second and third kinds of terms change the number of quanta by two and have expectation value 0; the first kind gives $u_1^\dagger N u_1$; the fourth gives $\sum_s v_s^\dagger N v_s$, which is also its vacuum value and is removed by normal ordering. So the normal-ordered expectation value is $u_1^\dagger BMu_1$. In the antiparticle state $d_1^*|0\rangle$ the first kind gives 0, and the fourth gives $\sum_{s\neq 1} v_s^\dagger N v_s$ (the term $s = 1$ vanishes, because $d_1^* d_1^*|0\rangle = 0$); minus the vacuum value this is $-v_1^\dagger N v_1$. Hence the **expectation-value rule** of the Revision record:

$$\langle :\Psi^\dagger M \Psi: \rangle = u^\dagger BMu \text{ (particle)}, \qquad \langle :\Psi^\dagger M \Psi: \rangle = -v^\dagger BMv \text{ (antiparticle)}.$$

Two examples: for $M = B$ (the charge) the rule gives $u^\dagger BBu = u^\dagger u = 1$ and $-v^\dagger v = -1$; for $M = h' = Bh$ (the energy) it gives $u^\dagger BBhu = u^\dagger hu = E$ and $-v^\dagger hv = -(-E) = E$.

The energy-momentum tensor operator. The energy-momentum tensor of Chapter 9 becomes an operator by replacing the field by the quantised field and normal ordering with respect to the good-sector modes (Revision record `Revision/docs/DIRAC16COMPLEX_FIELD_THEORY.md`, section 12):

$$\hat{T}^\nu{}_\mu(x) = :T^\nu{}_\mu\big[\hat{\Psi}, \hat{\bar{\Psi}}\big]:, \qquad \hat{\bar{\Psi}} = \hat{\Psi}^\dagger C, \qquad \hat{\Psi}^\dagger = \hat{\chi} B.$$

With $\hat{S} = \hat{\Psi}^\dagger C \hat{\Psi}$ and the kinetic terms $\hat{K}_\mu = \frac{1}{2f_\mu}(\hat{\bar{\Psi}}\gamma^{(\mu)}\partial_\mu\hat{\Psi} - \partial_\mu\hat{\bar{\Psi}}\gamma^{(\mu)}\hat{\Psi})$, the energy density and the pressures are

$$\hat{\rho} = -\hat{T}^{x_4}{}_{x_4} = :\Big(- \sum_{\mu \neq x_4} \hat{K}_\mu + m\hat{S} + U(\hat{S})\Big):, \qquad \hat{p}_\mu = \hat{T}^{\mu}{}_{\mu} = :\Big(\sum_{\nu \neq \mu} \hat{K}_\nu - m\hat{S} - U(\hat{S})\Big): \quad (\mu \neq x_4),$$

with the kinetic part $:(-\sum_{\mu \neq x_4} \hat{K}_\mu):$ and the potential part $:(m\hat{S} + U(\hat{S})):$ of the energy density. For $\lambda = 0$ every component is a normal-ordered bilinear, and its one-particle expectation values follow the rule just derived. For $\lambda \neq 0$ the potential term is quartic in the field, and the expectation value of $:U(S):$ is not U of the expectation value of $:S:$ in general.

The case $\lambda \neq 0$ in a finite model. The operator form of the on-shell identity $\sum_\mu \langle :K_\mu: \rangle = \langle :(m + U')S: \rangle$, and with it $\rho = mS + U$ and $p = SU' - U$, is verified by the Revision record in one finite model only (`Revision/theory/fock_quartic/`, one Python checker with sympy and an exact engine for the anticommutation rules, no Wolfram counterpart; 21 of 21 checks pass). The model is the positive Fock space of Section 10.20 ($\Psi^\dagger = \chi B$) for one good-sector plane-wave mode set with frozen coefficients: flat frame, volume 1, 16 modes and 2^{16} states, at rest ($k = 0$, symbolic m) and for $m = 3$, $k = (4, 0, 0, 0)$. There, for solutions of the interacting operator field equation, $:\sum_\mu K_\mu: = :(m + U'(S))S:$ holds as an exact operator identity on every state if and only if the potential $U = \frac{\lambda}{2}:SS:$ and the energy-momentum tensor are both Wick (normal) ordered, and at rest $\hat{\rho} = :mS + U:$ and $\hat{p} = :SU' - U:$ hold in the same sense (checks `trace_identity_operator_identity_wick` and `homogeneous_rho_p_operator_identities_wick`). Every other tested combination (four orderings of the potential, two definitions of the left-hand side, two right-hand sides; 30 combinations on the two mode sets) violates the identity by a nonzero one-body operator, sometimes plus a constant, that is linear in λ (check `trace_identity_fails_for_every_other_combination`), and it fails already in expectation values: at rest on every state except the vacuum of the mode set (check `expectation_classes_M0`), and for $k = (4, 0, 0, 0)$ in a state with a single particle or a single antiparticle (check `expectation_values_M1`). An occupation state with n quanta at rest has $\rho = mn + \frac{\lambda}{2}n(n-1)$ and $p = \frac{\lambda}{2}n(n-1)$, so $\langle :U(S): \rangle = \frac{\lambda}{2}n(n-1)$ differs from $U(\langle :S: \rangle) = \frac{\lambda}{2}n^2$ (check `homogeneous_expectation_values_are_not_U_of_expectation`). For the field on a whole slice (many momenta), for the curved x_0 dependence with its boundary term at $z = \pi/2$ and for the extra-time sector the operator form stays OPEN, and symmetry and conservation of the quartic operator are not verified. Numerical expectation values of $\hat{T}$ for many-fermion states of the field on a whole slice are computed only in the Kohn-Sham approximation (Chapters 14 and 15); the finite model gives none for a physical many-fermion state.

statement	**status**	**where it is verified**
expectation-value rule $u^\dagger BMu$ (particle), $-v^\dagger BMv$ (antiparticle)	PROVED (above) and checked for a generic matrix and five recorded matrices	`python-field-theory.json`, check `good_sector_positive_fock_realisation`; `wolfram-field-theory.json`, check `Fock_space_good_sector_example`

statement	status	where it is verified
$\hat{T} = :T[\hat{\Psi}, \hat{\Psi}^\dagger C]:$; normal ordering removes the vacuum value $-8E$ per momentum	definition; the vacuum value PROVED (Section 10.20) and checked in the exact examples	Revision theory document, section 12; `python-field-theory.json`, check `good_sector_positive_fock_realisation`; `wolfram-field-theory.json`, check `Fock_space_good_sector_example`
operator form of the on-shell identity for $\lambda \neq 0$ (at rest also $\hat{\rho} = :mS + U:$, $\hat{p} = :SU' - U:$)	PROVED (exact, sympy and an exact engine for the anticommutation rules) in the finite model of one good-sector mode set with frozen coefficients only, where, of the tested orderings, it holds if and only if U and $\hat{T}$ are Wick ordered; OPEN for the field on a whole slice, the curved x_0 dependence and the extra-time sector; symmetry and conservation of the quartic operator not verified	`fock-quartic.json`, checks `trace_identity_operator_identity_wick`, `trace_identity_fails_for_every_other_combination` and `homogeneous_rho_p_operator_identities_wick`

10.22 The commuting field dirac16complex00: an energy unbounded below

The second field of the theory, dirac16complex00 (written Φ), has 16 COMMUTING complex components. The author defines it as a classical ("semi-classical") field, the analogue of Dirac's wave function of 1928, and the Revision record does not quantise it. Without anticommutators there is no Pauli principle, no filled sea and no normal ordering, and its classical energy is not positive.

The energy density of one plane wave, line by line. Take flat space, $U = 0$ and a positive-frequency plane wave $\Phi = c\,u\,e^{i(k\cdot x - Ex_4)}$ of the good sector, with $hu = Eu$, $E > 0$, a complex number c and $k \cdot x = \sum_{a\neq 4} k_a x_a$. The energy density is $\rho = -T^{x_4}{}_{x_4} = -\sum_{\mu\neq x_4} K_\mu + mS$ (Chapter 9), with $K_a = \frac{1}{2}(\bar{\Phi}\gamma^{(x_a)}\partial_a\Phi - \partial_a\bar{\Phi}\gamma^{(x_a)}\Phi)$, $S = \bar{\Phi}\Phi$ and $\bar{\Phi} = \Phi^\dagger C$.

- $\partial_a\Phi = ik_a\Phi$, and $\partial_a\bar{\Phi} = (\partial_a\Phi)^\dagger C = -ik_a\bar{\Phi}$, because the conjugate of ik_a is $-ik_a$ (the derivative of the exponential, and the conjugate transpose of a number times a column).
- So $K_a = \frac{1}{2}(ik_a + ik_a)\bar{\Phi}\gamma^{(x_a)}\Phi = ik_a|c|^2\,u^\dagger C\gamma^{(x_a)}u$, because in $\bar{\Phi}\gamma^{(x_a)}\Phi$ the exponentials $e^{-i(\ldots)}$ and $e^{+i(\ldots)}$ cancel and $c^*c = |c|^2$; in the same way $S = |c|^2 u^\dagger Cu$.
- Therefore $\rho = |c|^2\,u^\dagger\big(mC - i\sum_a k_a C\gamma^{(x_a)}\big)u = |c|^2\,u^\dagger h'u$, with the matrix h' of Section 10.12.
- From $Bh' = h$ and $BB = I_{16}$ follows $h' = Bh$ (multiply by B from the left), and $hu = Eu$ gives $u^\dagger h'u = u^\dagger Bhu = E\,u^\dagger Bu$:

$$\rho = E\,|c|^2\,u^\dagger Bu.$$

The charge density is $\Phi^\dagger B\Phi = |c|^2 u^\dagger B u$. In the good sector B commutes with h, and by Theorem 10.1 the eigenspace of $+E$ contains four orthonormal columns with $Bu = u$ and four with $Bu = -u$. The first four have $\rho = +E|c|^2$, the other four $\rho = -E|c|^2$, although all eight have the positive frequency E. Multiplying the wave by a large number c makes its energy density as negative as one likes: **the classical energy of dirac16complex00 is unbounded below**, already for $U = 0$ in the good sector, and its charge density $\Phi^\dagger B\Phi$ is an indefinite form. For its charge, as for that of dirac16complex (Section 10.2), only the local conservation law is proved; the total over a slice is constant only if no charge flows through the boundary, which at $z = \pi/2$ is an ASSUMED no-flux condition that the record does not impose (`wolfram-field-theory.json`, check `charge_density_is_Krein_form_C`). Dirac's wave function of 1928 has the positive density $\psi^\dagger\psi$; dirac16complex00 does not. For the recorded momentum $m = 2$, $k = (k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$ and $|c| = 1$ the energy densities are $+5$ and -5 and the charge densities $+1$ and -1 (`python-scope.json`, check `commuting_field_energy_unbounded_below`). Quantising the fermion field with anticommutators and normal ordering makes the good-sector energy positive; for the classical commuting field there is no such mechanism, and no quantum statement is made for dirac16complex00 in this book.

10.23 Example: Notebook 10c builds the positive Fock space of one momentum

Notebook 10c builds the good-sector waves of the recorded momentum $m = 2$, $k = (k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$, the positive Fock space with the operators b_s and d_s, and the field Ψ with $\Psi^\dagger = \chi B$ (Section 10.20); it checks the canonical rule, computes the vacuum energy -40, the normal-ordered energies $+5$ and charges ± 1 of all 16 one-quantum states, checks the expectation-value rule of Section 10.21 for 100 random matrices and for the second recorded momentum $m = 3$, $k_1 = 4$, counts all 65536 states of one momentum, and computes the energy densities ± 5 of the commuting field (Section 10.22). It draws five figures and ends with the line ALL 14 CHECKS PASSED (notebook 10c).

10.24 How to run Notebook 10c

Step 1. What this notebook does and what it needs.

Notebook 10c (The good sector: a positive Fock space for one momentum) is the file `Revision/textbook/notebooks/10c_good_sector_fock.ipynb` of the repository Dirac_claude. For one momentum without extra-time components (the good sector) it computes the orthonormal eigenvectors of the Hermitian mode Hamiltonian, expands the field in particle and antiparticle operators on a positive fermionic Fock space with the canonical conjugate realised as chi B, and checks the canonical anticommutator, the vacuum energy of the filled Dirac sea (minus 8 E), the normal-ordered energies (plus E for particles and for antiparticles), the charges (plus 1 and minus 1) and the expectation-value rule, for the two exact examples of the Revision record; it counts the states of all 65536 occupation patterns by energy and charge, and contrasts the quantised fermion with the classical commuting field dirac16complex00, whose energy is unbounded below; five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10c_good_sector_fock.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10c_good_sector_fock.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 10c_good_sector_fock.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10c.captions.json`
- `Revision/textbook/figures/10c_1_expectation_rule.png`
- `Revision/textbook/figures/10c_2_dirac_sea.png`
- `Revision/textbook/figures/10c_3_fock_spectrum.png`
- `Revision/textbook/figures/10c_4_commuting_energy.png`
- `Revision/textbook/figures/10c_5_energy_density_spread.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10c_5_energy_density_spread.png exists
ALL 14 CHECKS PASSED (notebook 10c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10c_good_sector_fock.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10c_good_sector_fock.ipynb
python -m nbconvert --execute --inplace 10c_good_sector_fock.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.25 Notebook 10c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10c_good_sector_fock.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The previous notebooks of this chapter showed that the canonical rule
```

$\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$ (at one point) forces an indefinite inner product when $\Psi^\dagger$ is read as the Hilbert adjoint. The Revision record shows that in the *good sector* (waves that do not depend on the extra times x_5, x_6, x_7) a POSITIVE quantum state space exists for each single momentum with frozen coefficients (flat-frame plane waves, in which the deflation of the extra times does not enter). This notebook builds it, for one momentum at a time, and checks every recorded number:

- the mode Hamiltonian h of a good-sector momentum is Hermitian, with eigenvalues $+E$ and $-E$ (eight each), $E = \sqrt{m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2}$;
- the field is expanded in particle operators b_s and antiparticle operators d_s on a positive Fock space, and the canonical conjugate is realised as $\Psi^\dagger = \chi B$ with χ the Hilbert adjoint; then $\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$ holds;
- the vacuum (the *filled Dirac sea*) has energy $-8E$; after *normal ordering* a particle and an antiparticle both have energy $+E$, and charges $+1$ and -1;
- the expectation-value rule $\langle{:}\Psi^\dagger M\Psi{:}\rangle = u^\dagger BMu$ for a particle and $-v^\dagger BMv$ for an antiparticle;
- all $2^{16} = 65536$ occupation patterns of one momentum, counted by energy and charge;
- the contrast with the classical COMMUTING field dirac16complex00, which has no Fock space and no normal ordering: its positive-frequency waves have energy densities of both signs, so its energy is unbounded below.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 10c (The good sector: a positive Fock space for one momentum) is the file ‘Revision/textbook/notebooks/10c_good_sector_fock.ipynb‘ of the repository Dirac_claude. For one momentum without extra-time components (the good sector) it computes the orthonormal eigenvectors of the Hermitian mode Hamiltonian, expands the field in particle and antiparticle operators on a positive fermionic Fock space with the canonical conjugate realised as chi B, and checks the canonical anticommutator, the vacuum energy of the filled Dirac sea (minus 8 E), the normal-ordered energies (plus E for particles and for antiparticles), the charges (plus 1 and minus 1) and the expectation-value rule, for the two exact examples of the Revision record; it counts the states of all 65536 occupation patterns by energy and charge, and contrasts the quantised fermion with the classical commuting field dirac16complex00, whose energy is unbounded below; five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any

terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned

versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10c_good_sector_fock.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10c_good_sector_fock.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10c_good_sector_fock.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10c.captions.json‘
- ‘Revision/textbook/figures/10c_1_expectation_rule.png‘
- ‘Revision/textbook/figures/10c_2_dirac_sea.png‘
- ‘Revision/textbook/figures/10c_3_fock_spectrum.png‘
- ‘Revision/textbook/figures/10c_4_commuting_energy.png‘
- ‘Revision/textbook/figures/10c_5_energy_density_spread.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10c_5_energy_density_spread.png exists
ALL 14 CHECKS PASSED (notebook 10c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10c_good_sector_fock.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 10c_good_sector_fock.ipynb
  python -m nbconvert --execute --inplace 10c_good_sector_fock.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.

- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 10c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 10c (The good sector: a positive Fock space for one momentum) is the file
# Revision/textbook/notebooks/10c_good_sector_fock.ipynb of the repository Dirac_claude.
# For one momentum without extra-time components (the good sector) it computes the
# orthonormal eigenvectors of the Hermitian mode Hamiltonian, expands the field in
# particle and antiparticle operators on a positive fermionic Fock space with the
# canonical conjugate realised as chi B, and checks the canonical anticommutator, the
# vacuum energy of the filled Dirac sea (minus 8 E), the normal-ordered energies (plus E
# for particles and for antiparticles), the charges (plus 1 and minus 1) and the
# expectation-value rule, for the two exact examples of the Revision record; it counts
# the states of all 65536 occupation patterns by energy and charge, and contrasts the
# quantised fermion with the classical commuting field dirac16complex00, whose energy is
# unbounded below; five teaching figures. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
```

```
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
```

```
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
```

```
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10c_good_sector_fock.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10c_good_sector_fock.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10c_good_sector_fock.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10c.captions.json
# - Revision/textbook/figures/10c_1_expectation_rule.png
# - Revision/textbook/figures/10c_2_dirac_sea.png
# - Revision/textbook/figures/10c_3_fock_spectrum.png
# - Revision/textbook/figures/10c_4_commuting_energy.png
# - Revision/textbook/figures/10c_5_energy_density_spread.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
```

```
#
#     PASS the figure file 10c_5_energy_density_spread.png exists
#     ALL 14 CHECKS PASSED (notebook 10c)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10c_good_sector_fock.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10c_good_sector_fock.ipynb
#     python -m nbconvert --execute --inplace 10c_good_sector_fock.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
```

```
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10c"  # this notebook: chapter 10, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
```

```
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Good sector**: no momentum along the extra times, $k_5 = k_6 = k_7 = 0$.
- **Mode Hamiltonian** $h$: the $16 \times 16$ matrix with $i\,du/dx_4 = hu$ for a
  plane wave $u\,e^{ik\cdot x}$ (built in the first notebook of this chapter); its
  eigenvalues are the frequencies, read as energies.
- **Orthonormal**: columns $u_s$ with $u_s^\dagger u_{s'} = 1$ for $s = s'$ and 0
  otherwise. **Completeness**: 16 orthonormal columns $w_1, \dots, w_{16}$ satisfy
  $\sum_n w_n w_n^\dagger = I_{16}$.
- **Particle, antiparticle**: a quantum of positive energy created by $b_s^*$, and
  a quantum created by $d_s^*$, which is a hole in the filled negative-energy levels.
- **Dirac sea**: the picture in which every negative-energy level is filled in the
  vacuum. Its energy, $-E$ for each of the 8 negative levels, is $-8E$.
- **Normal ordering**: writing every product with the annihilators to the right, by
  definition dropping the constant that the reordering produces (here $-8E$ for the
  energy). The normal-ordered value of an operator $X$ in a state is its value minus
  the vacuum value.
- **Charge**: the U(1) charge $Q = \Psi^\dagger B\Psi$ of the field (its density, an
  indefinite form). Its local conservation law is proved; the total over a slice is
  constant only if no charge flows through the boundary, which at $z = \pi/2$ is an
  ASSUMED no-flux condition that the Revision record does not impose.
- **Expectation value**: $\langle\phi|X|\phi\rangle$ in a normalised state $\phi$.
- **Fock space, pattern, vacuum, $f_p$, $f_p^*$**: as in the previous notebook of
  this chapter: a basis state is a pattern of occupations of 16 fermion modes,
  stored as a whole number whose binary digit $p$ is the occupation of mode $p$.
- **dirac16complex00**: the second field of the theory, with 16 COMMUTING complex
  components; it is a classical field (it is not quantised).
- **Energy density**: $\rho = -T^{x_4}{}_{x_4}$, the time-time component of the
  energy-momentum tensor with the sign convention of the Revision record.
```

Text cell:

```
## 4. The physical and mathematical situation

**One good-sector momentum.** In the good sector $h = m\beta + \sum_{a}k_a\alpha^a$
with $\beta = -i\gamma^{(x_4)}$ and $\alpha^a = -\gamma^{(x_4)}\gamma^{(x_a)}$ for
$a = 1, 2, 3, 8$; all five matrices are Hermitian, so $h$ is Hermitian and has real
eigenvalues; $h^2 = E^2 I_{16}$ and $\mathrm{tr}\,h = 0$ give eight eigenvalues $+E$
and eight $-E$. Choose orthonormal eigenvectors $u_1, \dots, u_8$ ($hu_s = Eu_s$) and
$v_1, \dots, v_8$ ($hv_s = -Ev_s$). Eigenvectors of a Hermitian matrix for different
eigenvalues are orthogonal, so the 16 columns are orthonormal, and the matrix $W$
with these columns obeys $WW^\dagger = I_{16}$ (completeness).

**The field.** On a positive Fock space with operators $b_s, d_s$ ($\{b_s, b_{s'}^*\}
= \{d_s, d_{s'}^*\} = \delta_{ss'}$, all other anticommutators zero) put

$$\Psi = \sum_s (u_s b_s + v_s d_s^*),\qquad \chi = \sum_s (u_s^\dagger b_s^* +
v_s^\dagger d_s),\qquad \Psi^\dagger = \chi B .$$

Here $\chi$ is the Hilbert adjoint of $\Psi$. Line by line:
$\{\Psi_A, \chi_C\} = \sum_s (u_s)_A(u_s)_C^* + \sum_s (v_s)_A(v_s)_C^* =
(WW^\dagger)_{AC} = \delta_{AC}$, and therefore $\{\Psi_A, \Psi^\dagger_C\} = \sum_D
\{\Psi_A, \chi_D\}B_{DC} = B_{AC}$: the canonical rule.

**The energy.** The Hamiltonian of the momentum is $\Psi^\dagger h'\Psi$ with
$h' = mC - i\sum_a k_aC\gamma^{(x_a)}$, and $Bh' = h$ (previous notebook), so it
equals $\chi h\Psi$. Insert the expansion; $hu_s = Eu_s$, $hv_s = -Ev_s$ and the
orthonormality leave

$$\chi h\Psi = \sum_s E\,b_s^*b_s - \sum_s E\,d_sd_s^* = \sum_s E\,(b_s^*b_s +
d_s^*d_s) - 8E,$$

where the last step used $d_sd_s^* = 1 - d_s^*d_s$ for each of the 8 values of $s$.
The vacuum ($b_s|0\rangle = d_s|0\rangle = 0$) has energy $-8E$; after normal
ordering every particle and every antiparticle has energy $+E > 0$.

**The charge.** $Q = \Psi^\dagger B\Psi = \chi BB\Psi = \chi\Psi = \sum_s(b_s^*b_s +
d_sd_s^*) = \sum_s(b_s^*b_s - d_s^*d_s) + 8$: after normal ordering particles have
charge $+1$ and antiparticles $-1$.

**The commuting field.** For dirac16complex00 there is no Fock space and no
normal ordering. For a plane wave $\Phi = c\,u\,e^{i(k\cdot x - Ex_4)}$ with
$hu = Eu$ the energy density is $\rho = E\,|c|^2\,u^\dagger Bu$ (derived in
Section 10), and $u^\dagger Bu$ takes both signs on the eigenspace of $+E$: the
classical energy is unbounded below.
```

Text cell:

```
## 5. The gamma matrices, $B$ and the mode Hamiltonian

The next cell reads the gammas, builds $C$ and $B$, defines the mode Hamiltonian
$h$ of a good-sector momentum and the helper ‘check_record‘ (the PASS line and the
line of the reproduced Revision record are printed in one piece, so that the stored
output is the same in every run), and the colours of the figures.
Before it checks anything, ‘check_record‘ asks the helper ‘record_says_pass‘ whether
```

the cited record still says PASS: for a record name of the form
‘<report file>, check <check name>‘ it opens that report (each file once, kept in
the dictionary ‘REPORT_CHECKS‘) and stops the notebook with an error unless a check
of that name exists there with the verdict PASS; a record name without ‘, check ‘
names a data entry that the notebook reads and compares itself.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)


def mode_hamiltonian(m, k):
    """h = -i m gamma^(x4) - gamma^(x4) sum_a k_a gamma^(x_a); k = {a: k_a}."""
    h = -1j * m * gamma[4]
    for a, k_a in k.items():
        h = h - k_a * (gamma[4] @ gamma[a])
    return h
```

```
def orthonormal_columns(P):
    """Orthonormal columns that span the range of P (Gram-Schmidt, done twice)."""
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T


say("gammas, C, B and the mode Hamiltonian are ready")
```

Out [2]:

```
gammas, C, B and the mode Hamiltonian are ready
```

Text cell:

```
## 6. One good-sector momentum: $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$

The sympy verifier of the Revision record uses the mass $m = 2$ and the momenta
$k_1 = 1$, $k_2 = 2$, $k_3 = 0$, $k_0 = 4$, so $E = \sqrt{4 + 1 + 4 + 0 + 16} = 5$.
The next cell builds $h$, checks that it is Hermitian, that $h^2 = 25 I_{16}$ and
that it commutes with $B$, and computes the eigenvectors as the orthonormal columns
of the projectors $\frac12(I \pm h/E)$ (the same procedure as in the first notebook
of this chapter). It then checks orthonormality and completeness, $W^\dagger W =
WW^\dagger = I_{16}$, where $W = (u_1, \dots, u_8, v_1, \dots, v_8)$.
```

In [3]:

```
def good_sector_modes(m, k):
    """E, and the matrix W whose columns are u_1..u_8 (energy +E), v_1..v_8 (-E)."""
    h = mode_hamiltonian(m, k)
    E = np.sqrt(m ** 2 + sum(k_a ** 2 for k_a in k.values()))  # no extra times
    U_plus = orthonormal_columns((np.eye(16) + h / E) / 2)
    V_minus = orthonormal_columns((np.eye(16) - h / E) / 2)
    return h, E, np.hstack([U_plus, V_minus])


h, E, W = good_sector_modes(2, {0: 4, 1: 1, 2: 2})
report("E for m = 2, k = (4, 1, 2, 0)", f"{E:.12f}")
hermitian = np.max(np.abs(h - h.conj().T)) < 1e-15
square_ok = np.max(np.abs(h @ h - 25 * np.eye(16))) < 1e-13
commutes = np.max(np.abs(B @ h - h @ B)) < 1e-15
eigen_ok = (np.max(np.abs(h @ W[:, :8] - E * W[:, :8])) < 1e-13
```

```
            and np.max(np.abs(h @ W[:, 8:] + E * W[:, 8:])) < 1e-13)
complete = (np.max(np.abs(W.conj().T @ W - np.eye(16))) < 1e-13
            and np.max(np.abs(W @ W.conj().T - np.eye(16))) < 1e-13)
say(f"h Hermitian: {hermitian}, h^2 = 25 I: {square_ok}, B h = h B: {commutes}")
say(f"8 + 8 eigenvectors: {eigen_ok}; orthonormal and complete: {complete}")
check(hermitian and square_ok and commutes and eigen_ok and complete
      and W.shape == (16, 16),
      "good sector: h Hermitian, h^2 = E^2, 8 + 8 orthonormal complete eigenvectors")
```

Out [3]:

```
RESULT E for m = 2, k = (4, 1, 2, 0) = 5.000000000000
h Hermitian: True, h^2 = 25 I: True, B h = h B: True
8 + 8 eigenvectors: True; orthonormal and complete: True
PASS good sector: h Hermitian, h^2 = E^2, 8 + 8 orthonormal complete eigenvectors
```

Text cell:

```
## 7. The positive Fock space and the field operators

The next cell copies the fermionic Fock space of the previous notebook (states are
dictionaries `{pattern: amplitude}`; `annihilate(p, state)` is $f_p$ and
`create(p, state)` is $f_p^*$, with the sign that makes different modes
anticommute). The 16 modes are used as follows: modes $p = 0, \dots, 7$ are the
particles, $b_{s} = f_{s-1}$; modes $p = 8, \dots, 15$ are the antiparticles,
$d_s = f_{s+7}$. The field $\Psi = \sum_s (u_sb_s + v_sd_s^*)$ is then
$\Psi_A = \sum_p W_{Ap}F_p$ with $F_p = f_p$ for $p < 8$ and $F_p = f_p^*$ for
$p \geq 8$, and its Hilbert adjoint is $\chi_A = \sum_p W_{Ap}^*F_p^*$.

Every operator of the form $\chi N\Psi = \sum_{A,C}\chi_A N_{AC}\Psi_C$ becomes
$\sum_{p,q} (W^\dagger NW)_{pq}F_p^*F_q$; the function `bilinear(N, state)` applies
it to a state. The cell checks $\{\Psi_A, \chi_C\} = \delta_{AC}$ and
$\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$ on a random state.
```

In [4]:

```
def sign_below(n, p):
    """(-1) to the power of the number of occupied modes below mode p in pattern n."""
    return -1 if bin(n & ((1 << p) - 1)).count("1") % 2 else 1


def annihilate(p, state):  # f_p
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:
            new = n ^ (1 << p)
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):  # f_p^*
```

```
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:
            new = n | (1 << p)
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def add(*terms):
    """The state sum of coefficient * state over the pairs (coefficient, state)."""
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result


def inner(left, right):
    """The positive inner product <left|right>."""
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    return max((abs(a) for a in state.values()), default=0.0)


def F(p, state):  # F_p: b (p < 8) or d^* (p >= 8)
    return annihilate(p, state) if p < 8 else create(p, state)


def F_star(p, state):  # the Hilbert adjoint of F_p
    return create(p, state) if p < 8 else annihilate(p, state)


def bilinear(N, state, modes):
    """chi N Psi = sum_(p,q) (W^dagger N W)_pq F_p^* F_q applied to a state."""
    M = modes.conj().T @ N @ modes
    terms = []
    for q in range(16):
        lowered = F(q, state)
        if lowered:
            terms += [(M[p, q], F_star(p, lowered)) for p in range(16)
                      if abs(M[p, q]) > 1e-15]
    return add(*terms)


def psi(A, state, modes):  # Psi_A = sum_p W_Ap F_p
    return add(*[(modes[A - 1, p], F(p, state)) for p in range(16)])


def chi(A, state, modes):  # chi_A = sum_p conj(W_Ap) F_p^*
    return add(*[(np.conj(modes[A - 1, p]), F_star(p, state)) for p in range(16)])
```

```
def psi_dagger(A, state, modes):  # the canonical conjugate: sum_C chi_C B_CA
    return add(*[(B[C_ - 1, A - 1], chi(C_, state, modes)) for C_ in range(1, 17)
                 if B[C_ - 1, A - 1] != 0])


VACUUM = {0: 1.0}
rng = np.random.default_rng(12345)
phi = {int(n): complex(rng.normal(), rng.normal())
       for n in rng.integers(0, 2 ** 16, size=5)}  # a random state
worst_delta, worst_B = 0.0, 0.0
for A in range(1, 17):
    for C_ in range(1, 17):
        with_chi = add((1, psi(A, chi(C_, phi, W), W)), (1, chi(C_, psi(A, phi, W), W)))
        expected = phi if A == C_ else {}
        worst_delta = max(worst_delta, largest(add((1, with_chi), (-1, expected))))
        with_dagger = add((1, psi(A, psi_dagger(C_, phi, W), W)),
                          (1, psi_dagger(C_, psi(A, phi, W), W)))
        worst_B = max(worst_B, largest(add((1, with_dagger), (-B[A - 1, C_ - 1], phi))))
report("largest violation of {Psi_A, chi_C} = delta_AC", f"{worst_delta:.1e}")
report("largest violation of {Psi_A, Psi^dagger_C} = B_AC", f"{worst_B:.1e}")
check(worst_delta < 1e-12 and worst_B < 1e-12,
      "positive Fock space: {Psi, chi} = I and {Psi, Psi^dagger} = B")
```

Out [4]:

```
RESULT largest violation of {Psi_A, chi_C} = delta_AC = 6.3e-16
RESULT largest violation of {Psi_A, Psi^dagger_C} = B_AC = 6.3e-16
PASS positive Fock space: {Psi, chi} = I and {Psi, Psi^dagger} = B
```

Text cell:

```
## 8. Vacuum energy, normal ordering, energies and charges

The next cell computes, on the Fock space:

- the vacuum energy $\langle 0|\chi h\Psi|0\rangle$, which must be $-8E = -40$;
- for each of the 16 one-quantum states $b_s^*|0\rangle$ and $d_s^*|0\rangle$ the
  normal-ordered energy (its energy minus the vacuum energy), which must be $+5$
  for all 16;
- the normal-ordered charge $\chi\Psi$ (the vacuum charge is 8), which must be $+1$
  for the 8 particles and $-1$ for the 8 antiparticles.
```

In [5]:

```
def one_quantum_values(N, modes):
    """Normal-ordered values of chi N Psi in the 16 one-quantum states, and the
    vacuum value."""
    vacuum_value = inner(VACUUM, bilinear(N, VACUUM, modes)).real
    values = []
    for p in range(16):  # p < 8: b^*|0>, p >= 8: d^*|0>
        state = create(p, VACUUM)
```

```
        values.append(inner(state, bilinear(N, state, modes)).real - vacuum_value)
    return np.array(values), vacuum_value


energies, vacuum_energy = one_quantum_values(h, W)
charges, vacuum_charge = one_quantum_values(np.eye(16), W)
report("vacuum energy (the filled Dirac sea)", f"{vacuum_energy:.10f}")
report("vacuum charge before normal ordering", f"{vacuum_charge:.10f}")
say(f"normal-ordered energies of the 16 quanta: {np.round(energies, 10).tolist()}")
say(f"normal-ordered charges of the 16 quanta:  {np.round(charges, 10).tolist()}")
expected_charges = np.array([1] * 8 + [-1] * 8)  # 8 particles, 8 antiparticles
check_record(abs(vacuum_energy + 40) < 1e-12 and np.max(np.abs(energies - 5)) < 1e-12
             and np.max(np.abs(charges - expected_charges)) < 1e-12,
             "m = 2, k = (4, 1, 2, 0): vacuum -40, every quantum +5, charges +1, -1",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_positive_fock_realisation")
```

Out [5]:

```
RESULT vacuum energy (the filled Dirac sea) = -40.0000000000
RESULT vacuum charge before normal ordering = 8.0000000000
normal-ordered energies of the 16 quanta: [5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0,
    5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0]
normal-ordered charges of the 16 quanta:  [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, -1.0,
    -1.0, -1.0, -1.0, -1.0, -1.0, -1.0, -1.0]
PASS m = 2, k = (4, 1, 2, 0): vacuum -40, every quantum +5, charges +1, -1
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_positive_fock_realisation
```

Text cell:

```
## 9. The expectation-value rule, and the second recorded example

In the positive realisation every classical bilinear $\Psi^\dagger M\Psi$ becomes
$\chi BM\Psi$. For the particle state $b_s^*|0\rangle$ only the term
$(u_s^\dagger BMu_s)\,b_s^*b_s$ contributes after normal ordering, so the
normal-ordered expectation value is $u_s^\dagger BMu_s$; for the antiparticle state
$d_s^*|0\rangle$ the term $(v_s^\dagger BMv_s)\,d_sd_s^* = (v_s^\dagger BMv_s)(1 -
d_s^*d_s)$ gives $-v_s^\dagger BMv_s$. The next cell checks both for 100 random
complex matrices $M$ and draws the result.
```

In [6]:

```
particle_points, antiparticle_points = [], []
for _ in range(100):
    M = rng.normal(size=(16, 16)) + 1j * rng.normal(size=(16, 16))
    values, _ = one_quantum_values(B @ M, W)  # real parts of <:chi B M Psi:>
    particle_points.append((values[0], (W[:, 0].conj() @ B @ M @ W[:, 0]).real))
    antiparticle_points.append((values[8], -(W[:, 8].conj() @ B @ M @ W[:, 8]).real))
particle_points = np.array(particle_points)
antiparticle_points = np.array(antiparticle_points)
```

```
worst = max(np.max(np.abs(particle_points[:, 0] - particle_points[:, 1])),
            np.max(np.abs(antiparticle_points[:, 0] - antiparticle_points[:, 1])))
report("largest deviation from the rule (100 random M)", f"{worst:.1e}")
check(worst < 1e-12,
      "rule: u^dagger B M u for a particle, -v^dagger B M v for an antiparticle")

fig, ax = plt.subplots(figsize=(6.0, 5.0))
ax.plot(particle_points[:, 1], particle_points[:, 0], "o", color=BLUE, markersize=5,
        label="particle: against $u^\\dagger B M u$")
ax.plot(antiparticle_points[:, 1], antiparticle_points[:, 0], "s", color=ORANGE,
        markersize=5, fillstyle="none",
        label="antiparticle: against $-v^\\dagger B M v$")
low = min(particle_points.min(), antiparticle_points.min()) - 1
high = max(particle_points.max(), antiparticle_points.max()) + 1
ax.plot([low, high], [low, high], color=GREY, linewidth=1)
ax.set_xlabel("value of the formula (real part)")
ax.set_ylabel("normal-ordered value on the Fock space (real part)")
ax.set_title("The expectation-value rule in the good sector")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "expectation_rule",
            "For 100 random complex $16 \\times 16$ matrices $M$: the normal-ordered "
            "expectation value of the bilinear $\\Psi^\\dagger M\\Psi$ computed on "
            "the positive Fock space of the good-sector momentum $m = 2$, $k = (1, "
            "2, 0, 4)$ (vertical axis), against the formula $u^\\dagger BMu$ in a "
            "particle state (blue dots) and $-v^\\dagger BMv$ in an antiparticle "
            "state (orange squares); horizontal axis: the formula; real parts, "
            "pure numbers. All points lie on the diagonal: the rule of the "
            "Revision record holds.")
```

Out [6]:

```
RESULT largest deviation from the rule (100 random M) = 1.8e-15
PASS rule: u^dagger B M u for a particle, -v^dagger B M v for an antiparticle
```

Out [6]:

```
Figure 10c.1 saved as Revision/textbook/figures/10c_1_expectation_rule.png
```

Text cell:

```
The WolframScript verifier of the Revision record checks the same construction on a
second momentum, $m = 3$ and $k_1 = 4$ (so $E = \sqrt{9 + 16} = 5$ again), for the
matrices $M = C$, $-iC\gamma^{(x_4)}$, $-iC\gamma^{(x_1)}$, $C\gamma^{(x_2)}
\gamma^{(x_3)}$ and a dense matrix of whole numbers. The next cell repeats it (the
dense matrix here is our own: entry $(A, C)$ is the remainder of $3A + 5C$ divided
by 7, minus 3).
```

In [7]:

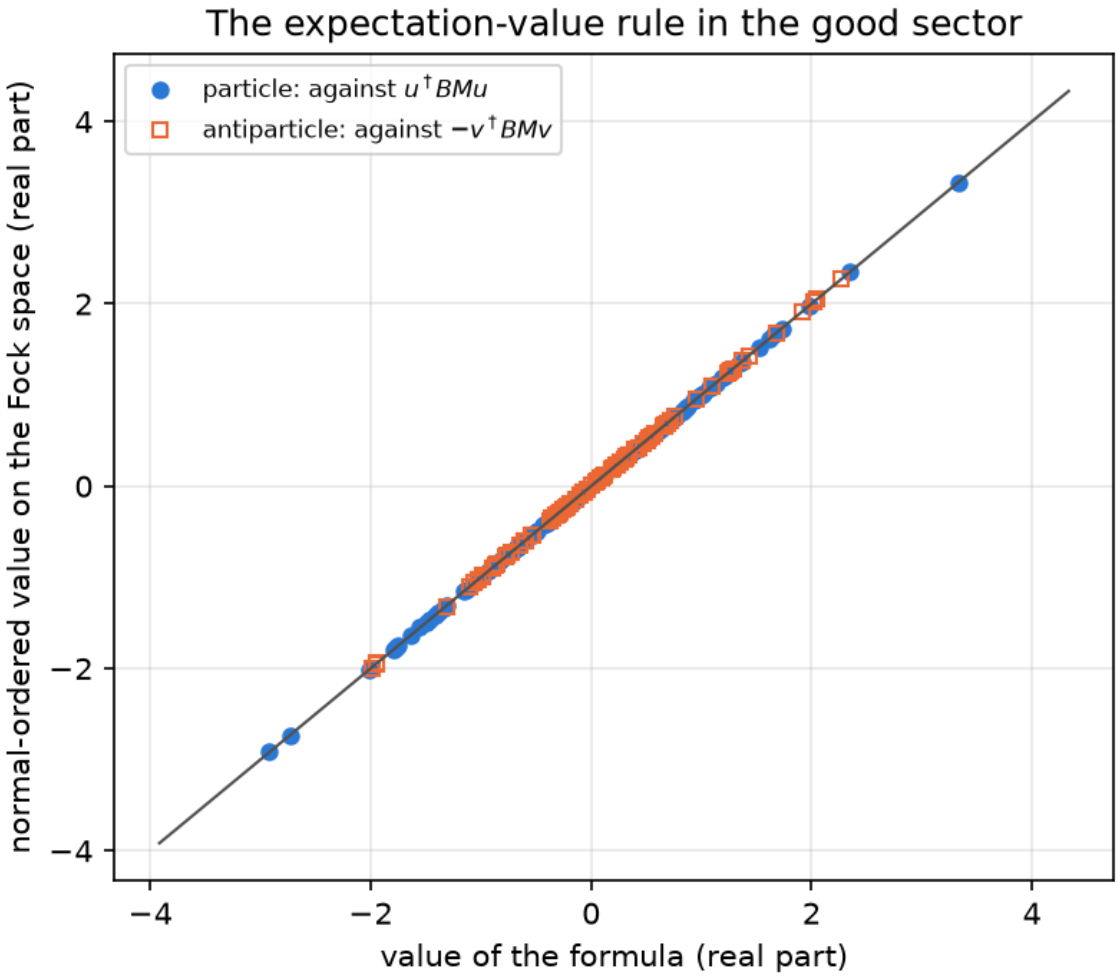

Figure 278: For 100 random complex 16×16 matrices M: the normal-ordered expectation value of the bilinear $\Psi^\dagger M \Psi$ computed on the positive Fock space of the good-sector momentum $m = 2$, $k = (1, 2, 0, 4)$ (vertical axis), against the formula $u^\dagger BMu$ in a particle state (blue dots) and $-v^\dagger BMv$ in an antiparticle state (orange squares); horizontal axis: the formula; real parts, pure numbers. All points lie on the diagonal: the rule of the Revision record holds. (Notebook 10c, figure 1.)

```
h2, E2, W2 = good_sector_modes(3, {1: 4})
energies2, vacuum2 = one_quantum_values(h2, W2)
charges2, _ = one_quantum_values(np.eye(16), W2)
dense = np.array([[(3 * A + 5 * C_) % 7 - 3 for C_ in range(1, 17)]
                  for A in range(1, 17)])
tests = {"C": C, "-i C gamma4": -1j * C @ gamma[4], "-i C gamma1": -1j * C @ gamma[1],
         "C gamma2 gamma3": C @ gamma[2] @ gamma[3], "dense": dense}
rule_ok = True
for label, M in tests.items():
    values, _ = one_quantum_values(B @ M, W2)
    expected_particle = (W2[:, 0].conj() @ B @ M @ W2[:, 0]).real
    expected_anti = -(W2[:, 8].conj() @ B @ M @ W2[:, 8]).real
    rule_ok = rule_ok and abs(values[0] - expected_particle) < 1e-12 \
        and abs(values[8] - expected_anti) < 1e-12
    say(f"M = {label:16s} particle {values[0]:+.6f}, antiparticle {values[8]:+.6f}")
report("E and the vacuum energy for m = 3, k1 = 4", f"{E2:.6f} and {vacuum2:.6f}")
check_record(abs(vacuum2 + 8 * E2) < 1e-12 and np.max(np.abs(energies2 - E2)) < 1e-12
             and np.max(np.abs(charges2 - np.array([1] * 8 + [-1] * 8))) < 1e-12
             and rule_ok,
             "m = 3, k1 = 4: vacuum -8E, energies +E, charges +-1, the rule holds",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "Fock_space_good_sector_example")
```

Out [7]:

```
M = C                particle +0.600000, antiparticle +0.600000
M = -i C gamma4      particle +1.000000, antiparticle -1.000000
M = -i C gamma1      particle +0.800000, antiparticle +0.800000
M = C gamma2 gamma3  particle +0.000000, antiparticle +0.000000
M = dense            particle -0.480000, antiparticle -1.920000
RESULT E and the vacuum energy for m = 3, k1 = 4 = 5.000000 and -40.000000
PASS m = 3, k1 = 4: vacuum -8E, energies +E, charges +-1, the rule holds
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Fock_space_good_sector_example
```

Text cell:

```
## 10. All 65536 states of one momentum, and the picture of the Dirac sea

A pattern $n$ with $N_b$ occupied particle modes and $N_d$ occupied antiparticle
modes is an eigenstate of the energy and of the charge: by Section 4 its
normal-ordered energy is $E(N_b + N_d)$ and its normal-ordered charge is
$N_b - N_d$. The next cell checks this on 200 random patterns with the Fock-space
operators, then counts all 65536 patterns by energy and charge (the number of
patterns with given $N_b$ and $N_d$ is $\binom{8}{N_b}\binom{8}{N_d}$, where
$\binom{8}{j}$ counts the ways to choose $j$ of 8 modes), and draws two figures: the
classical energies $\pm E$ of the waves against the momentum (with the filled sea),
and the counts of the quantum states.
```

In [8]:

```
from math import comb  # comb(8, j): the number of ways to choose j of 8 things

eigen_ok = True
for n in rng.integers(0, 2 ** 16, size=200):
    n = int(n)
    n_b = bin(n & 0xFF).count("1")  # occupied particle modes (digits 0 to 7)
    n_d = bin(n >> 8).count("1")  # occupied antiparticle modes (digits 8 to 15)
    state = {n: 1.0}
    energy_state = bilinear(h, state, W)
    charge_state = bilinear(np.eye(16), state, W)
    eigen_ok = eigen_ok and largest(add(
        (1, energy_state), (-(E * (n_b + n_d) - 8 * E), state))) < 1e-12
    eigen_ok = eigen_ok and largest(add(
        (1, charge_state), (-(n_b - n_d + 8), state))) < 1e-12
check(eigen_ok, "every pattern has energy E (N_b + N_d) - 8E and charge N_b - N_d + 8")

counts = np.zeros((17, 17), dtype=int)  # rows: N_b + N_d, columns: N_b - N_d + 8
for n in range(2 ** 16):
    n_b, n_d = bin(n & 0xFF).count("1"), bin(n >> 8).count("1")
    counts[n_b + n_d, n_b - n_d + 8] += 1
formula = np.zeros((17, 17), dtype=int)
for n_b in range(9):
    for n_d in range(9):
        formula[n_b + n_d, n_b - n_d + 8] += comb(8, n_b) * comb(8, n_d)
report("number of patterns", int(counts.sum()))
report("patterns with energy 0 (the vacuum only)", int(counts[0].sum()))
report("patterns with one quantum (energy 5)", int(counts[1].sum()))
check(np.array_equal(counts, formula) and counts.sum() == 2 ** 16,
      "the 65536 patterns are counted by binomial(8, N_b) binomial(8, N_d)")
```

Out [8]:

```
PASS every pattern has energy E (N_b + N_d) - 8E and charge N_b - N_d + 8
RESULT number of patterns = 65536
RESULT patterns with energy 0 (the vacuum only) = 1
RESULT patterns with one quantum (energy 5) = 16
PASS the 65536 patterns are counted by binomial(8, N_b) binomial(8, N_d)
```

Text cell:

```
The next cell draws the two figures announced above.
```

In [9]:

```
k_values = np.linspace(0.0, 5.0, 201)  # the size of the momentum
branch = np.sqrt(2.0 ** 2 + k_values ** 2)  # E for m = 2
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
ax = axes[0]
ax.plot(k_values, branch, color=BLUE, linewidth=2, label="$+E$: particle levels")
ax.plot(k_values, -branch, color=ORANGE, linewidth=2,
        label="$-E$: levels filled in the vacuum")
```

```
ax.fill_between(k_values, -branch, -6.5, color=ORANGE, alpha=0.2,
                label="the filled Dirac sea")
ax.set_xlabel("size of the momentum $|k|$ (with $m = 2$)")
ax.set_ylabel("eigenvalue of $h$ (classical wave energy)")
ax.set_ylim(-6.5, 6.5)
ax.set_title("classical energies of the waves")
ax.legend(loc="center right", fontsize=8)
ax = axes[1]
ax.plot(k_values, branch, color=BLUE, linewidth=2, label="particle $b^*|0\\rangle$")
ax.plot(k_values, branch, "--", color=ORANGE, linewidth=2,
        label="antiparticle $d^*|0\\rangle$ (same curve)")
ax.axhline(0.0, color=GREY, linewidth=1)
ax.set_xlabel("size of the momentum $|k|$ (with $m = 2$)")
ax.set_ylabel("normal-ordered energy of one quantum")
ax.set_ylim(-6.5, 6.5)
ax.set_title("quantum: both quanta have energy $+E$")
ax.legend(loc="lower right", fontsize=8)
save_figure(fig, "dirac_sea",
            "Left: the eigenvalues $\\pm E = \\pm\\sqrt{m^2 + |k|^2}$ of the "
            "good-sector mode Hamiltonian for $m = 2$ against the size of the "
            "momentum $|k|$ (horizontal axis, same units as $m$); in the vacuum every "
            "negative level is filled (shaded: the Dirac sea), with energy $-8E$ "
            "per momentum. Right: after normal ordering, a particle (blue) and an "
            "antiparticle (orange dashed, on top of the blue curve) both have the "
            "positive energy $+E$; vertical axes in the units of $m$. The good "
            "sector has a positive energy for every quantum.")

from matplotlib.colors import LinearSegmentedColormap  # colour scales

fig, ax = plt.subplots(figsize=(7.0, 5.0))
shown = np.where(counts > 0, np.log10(np.maximum(counts, 1)), np.nan)  # NaN: empty
light_to_dark = LinearSegmentedColormap.from_list(
    "light_to_dark_blue", ["#cde2fb", "#2a78d6", "#0d366b"])  # 1 state is visible
image = ax.imshow(shown, origin="lower", cmap=light_to_dark, aspect="auto",
                  extent=(-8.5, 8.5, -0.5, 16.5))
ax.annotate("vacuum", (0, 0), xytext=(1.5, 0.3), fontsize=8,
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.annotate("4900 states", (0, 8), xytext=(3.0, 8.6), fontsize=8,
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.set_xlabel("normal-ordered charge $N_b - N_d$")
ax.set_ylabel("number of quanta $N_b + N_d$ (energy $= 5(N_b + N_d)$)")
ax.set_title("The 65536 states of one momentum, $E = 5$")
ax.grid(False)
fig.colorbar(image, ax=ax, label="$\\log_{10}$ of the number of states")
save_figure(fig, "fock_spectrum",
            "All $2^{16} = 65536$ quantum states of one good-sector momentum ($m = "
            "2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$), sorted by their "
            "normal-ordered charge "
            "$N_b - N_d$ (horizontal axis) and their number of quanta $N_b + N_d$ "
            "(vertical axis; the energy is $5(N_b + N_d)$). The colour is the "
            "base-10 logarithm of the number of states in each square (lightest "
            "blue: one state); white squares are empty. The vacuum is the single "
            "state at the bottom; "
            "there are 16 states with one quantum, and the largest number, 4900, "
```

```
"has 8 quanta and charge 0. No state has negative energy.")
```

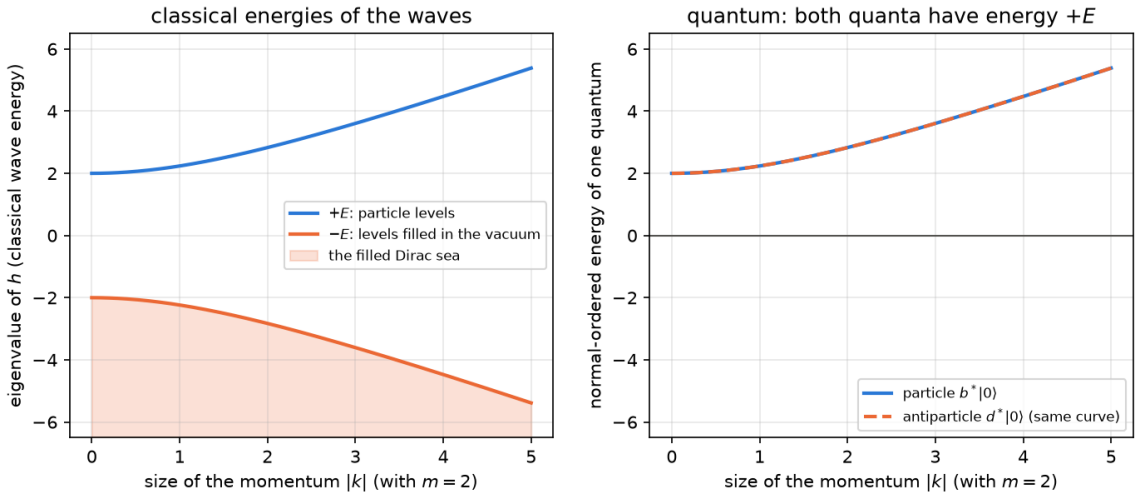

Figure 279: Left: the eigenvalues $\pm E = \pm\sqrt{m^2 + |k|^2}$ of the good-sector mode Hamiltonian for $m = 2$ against the size of the momentum $|k|$ (horizontal axis, same units as m); in the vacuum every negative level is filled (shaded: the Dirac sea), with energy $-8E$ per momentum. Right: after normal ordering, a particle (blue) and an antiparticle (orange dashed, on top of the blue curve) both have the positive energy $+E$; vertical axes in the units of m. The good sector has a positive energy for every quantum. (Notebook 10c, figure 2.)

Out [9]:

```
Figure 10c.2 saved as Revision/textbook/figures/10c_2_dirac_sea.png
```

Out [9]:

```
Figure 10c.3 saved as Revision/textbook/figures/10c_3_fock_spectrum.png
```

Text cell:

```
## 11. The contrast: the classical commuting field dirac16complex00

The field dirac16complex00 has 16 COMMUTING components and is not quantised. Its
energy density, with the sign convention of the Revision record, is
$\rho = -T^{x_4}{}_{x_4} = -\sum_{\mu \neq x_4}K_\mu + mS$ for $U = 0$, with
$K_\mu = \frac12(\bar\Phi\gamma^{(\mu)}\partial_\mu\Phi - \partial_\mu\bar\Phi
\gamma^{(\mu)}\Phi)$ in flat space and $S = \bar\Phi\Phi$, $\bar\Phi = \Phi^\dagger C$.
For the plane wave $\Phi = c\,u\,e^{i(k\cdot x - Ex_4)}$, line by line:

- $\partial_a\Phi = ik_a\Phi$ and $\partial_a\bar\Phi = -ik_a\bar\Phi$ (the
  conjugate of $e^{ik_ax_a}$ is $e^{-ik_ax_a}$);
- so $K_a = \frac12(ik_a + ik_a)\bar\Phi\gamma^{(x_a)}\Phi = ik_a|c|^2u^\dagger
  C\gamma^{(x_a)}u$, and $S = |c|^2u^\dagger Cu$;
```

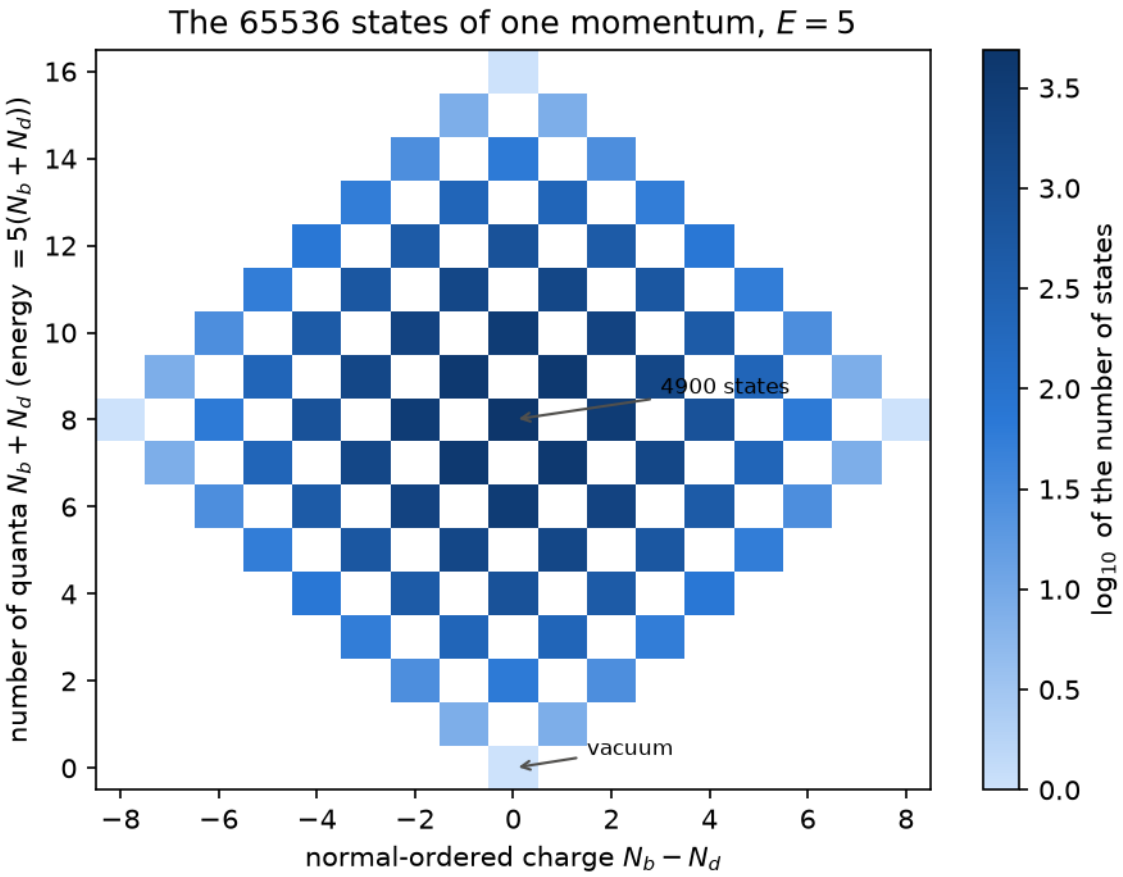

Figure 280: All $2^{16} = 65536$ quantum states of one good-sector momentum ($m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$), sorted by their normal-ordered charge $N_b - N_d$ (horizontal axis) and their number of quanta $N_b + N_d$ (vertical axis; the energy is $5(N_b + N_d)$). The colour is the base-10 logarithm of the number of states in each square (lightest blue: one state); white squares are empty. The vacuum is the single state at the bottom; there are 16 states with one quantum, and the largest number, 4900, has 8 quanta and charge 0. No state has negative energy. (Notebook 10c, figure 3.)

```
- $\rho = |c|^2u^\dagger(mC - i\sum_a k_aC\gamma^{(x_a)})u = |c|^2u^\dagger h'u$;
- since $Bh' = h$ and $B^2 = I$, $h' = Bh$, and $hu = Eu$ gives $\rho =
  E\,|c|^2\,u^\dagger Bu$.

The charge density is $\Phi^\dagger B\Phi = |c|^2u^\dagger Bu$. In the good sector
$B$ commutes with $h$, so the eigenspace of $+E$ contains 4 columns with $Bu = u$
and 4 with $Bu = -u$ (the inertia $(4, 4)$ of the first notebook). The next cell
finds them (orthonormal columns of the product of the projectors
$\frac12(I + h/E)$ and $\frac12(I \pm B)$), computes $\rho$ directly from the
formula for $K_\mu$ and $S$ (not from the shortcut), and checks the record: for
$|c| = 1$ the energy densities are $+5$ and $-5$ and the charge densities $+1$ and
$-1$.
```

In [10]:

```
energy_projector = (np.eye(16) + h / E) / 2  # onto the eigenspace of +E
plus_part = orthonormal_columns(energy_projector @ (np.eye(16) + B) / 2)
minus_part = orthonormal_columns(energy_projector @ (np.eye(16) - B) / 2)
k_sample = {0: 4, 1: 1, 2: 2}  # the momenta of the sample, m = 2


def energy_density(u, m=2.0, k=k_sample):
    """rho = -sum_(a != 4) K_a + m S for Phi = u exp(i(k.x - E x4)), |c| = 1."""
    Phibar = u.conj() @ C  # the row Phi^dagger C at x = 0
    S = (Phibar @ u).real
    K = {a: (0.5 * (Phibar @ gamma[a] @ (1j * k_a * u))
             - 0.5 * ((-1j * k_a) * Phibar) @ gamma[a] @ u) for a, k_a in k.items()}
    return (-sum(K.values()) + m * S).real


densities_plus = [energy_density(plus_part[:, j]) for j in range(plus_part.shape[1])]
densities_minus = [energy_density(minus_part[:, j])
                   for j in range(minus_part.shape[1])]
charge_plus = [(u.conj() @ B @ u).real for u in plus_part.T]
charge_minus = [(u.conj() @ B @ u).real for u in minus_part.T]
say(f"dimensions: {plus_part.shape[1]} with B u = +u, {minus_part.shape[1]} with "
    f"B u = -u")
say(f"energy densities: {np.round(densities_plus, 10).tolist()} and "
    f"{np.round(densities_minus, 10).tolist()}")
say(f"charge densities: {np.round(charge_plus, 10).tolist()} and "
    f"{np.round(charge_minus, 10).tolist()}")
check_record(plus_part.shape[1] == 4 and minus_part.shape[1] == 4
             and np.max(np.abs(np.array(densities_plus) - 5)) < 1e-12
             and np.max(np.abs(np.array(densities_minus) + 5)) < 1e-12
             and np.max(np.abs(np.array(charge_plus) - 1)) < 1e-12
             and np.max(np.abs(np.array(charge_minus) + 1)) < 1e-12,
             "commuting field: positive-frequency waves with energy +5 and -5",
             record="Revision/theory/reports/python-scope.json, check "
                    "commuting_field_energy_unbounded_below")
```

Out [10]:

```
dimensions: 4 with B u = +u, 4 with B u = -u
energy densities: [5.0, 5.0, 5.0, 5.0] and [-5.0, -5.0, -5.0, -5.0]
charge densities: [1.0, 1.0, 1.0, 1.0] and [-1.0, -1.0, -1.0, -1.0]
PASS commuting field: positive-frequency waves with energy +5 and -5
     reproduces Revision/theory/reports/python-scope.json, check
    commuting_field_energy_unbounded_below
```

Text cell:

```
The next cell compares the two fields in a bar chart, and then shows the whole range
of classical energy densities: for 4000 random unit columns $u$ in the eigenspace of
$+E$ (random combinations of its 8 orthonormal columns) the energy density
$\rho = E\,u^\dagger Bu$ of the commuting wave $\Phi = u\,e^{i(k\cdot x - Ex_4)}$.
Since $u^\dagger Bu$ lies between $-1$ and $1$, $\rho$ lies between $-5$ and $5$, and
multiplying $\Phi$ by a number $c$ multiplies $\rho$ by $|c|^2$: the energy of the
classical commuting field is unbounded below.
```

In [11]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0), sharey=True)
for ax in axes:
    ax.set_axisbelow(True)  # draw the grid lines behind the bars
classical = densities_plus + densities_minus
axes[0].bar(range(1, 9), classical,
            color=[BLUE] * len(densities_plus) + [ORANGE] * len(densities_minus))
axes[0].axhline(0.0, color=GREY, linewidth=1)
axes[0].set_xticks(range(1, 9))
axes[0].set_xlabel("positive-frequency wave (4 with $Bu = u$, 4 with $Bu = -u$)")
axes[0].set_ylabel("energy (units of $m$)")
axes[0].set_title("classical commuting field: $\\rho = \\pm 5$")
axes[1].bar(range(1, 17), energies, color=[BLUE] * 8 + [GREEN] * 8)
axes[1].axhline(0.0, color=GREY, linewidth=1)
axes[1].set_xticks(range(1, 17, 3))
axes[1].set_xlabel("quantum (1 to 8 particles, 9 to 16 antiparticles)")
axes[1].set_title("quantised fermion: every quantum $+5$")
save_figure(fig, "commuting_energy",
            "The same momentum $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$ "
            "in the two "
            "fields of the theory; vertical axes: energy in units of $m$. Left: "
            "the classical COMMUTING field dirac16complex00, energy density of the 8 "
            "positive-frequency waves $\\Phi = u\\,e^{i(k\\cdot x - 5x_4)}$ of a "
            "basis with $Bu = \\pm u$ ($|c| = 1$): four have $+5$ (blue), four have "
            "$-5$ (orange). Right: the quantised fermion field dirac16complex, "
            "normal-ordered energy of its 16 one-quantum states: $+5$ for each of "
            "the 8 particles (blue) and the 8 antiparticles (green). Quantisation "
            "with anticommutators and normal ordering makes the good-sector energy "
            "positive; the classical commuting field has no such mechanism.")

coefficients = rng.normal(size=(4000, 8)) + 1j * rng.normal(size=(4000, 8))
eigenspace = np.hstack([plus_part, minus_part])  # 8 orthonormal columns
samples = coefficients @ eigenspace.T  # each row: a random column u in the space
samples = samples / np.linalg.norm(samples, axis=1, keepdims=True)
```

```
rho = E * np.einsum("ni,ij,nj->n", samples.conj(), B, samples).real
report("smallest and largest energy density of the 4000 waves",
       f"{rho.min():.3f} and {rho.max():.3f}")
check(rho.min() < -2 and rho.max() > 2 and np.all(np.abs(rho) <= 5 + 1e-12),
      "random positive-frequency commuting waves have energy densities of both signs")
fig, ax = plt.subplots()
ax.set_axisbelow(True)
ax.hist(rho, bins=40, range=(-5, 5), color=BLUE, edgecolor="white")
ax.axvline(0.0, color=GREY, linewidth=1)
ax.set_xlabel("energy density $\\rho = E\\,u^\\dagger B u$ of a commuting wave "
              "($|c| = 1$)")
ax.set_ylabel("number of waves (of 4000)")
ax.set_title("dirac16complex00: positive frequency, energy of both signs")
save_figure(fig, "energy_density_spread",
            "Histogram of the energy density $\\rho = E\\,u^\\dagger Bu$ (horizontal "
            "axis, units of $m$, between $-5$ and $5$) of 4000 random "
            "positive-frequency plane waves $\\Phi = u\\,e^{i(k\\cdot x - 5x_4)}$ of "
            "the classical commuting field dirac16complex00 with $m = 2$, $k = (1, "
            "2, 0, 4)$ and $u^\\dagger u = 1$ (vertical axis: number of waves per "
            "bin). About half of the waves have negative energy density; multiplying a "
            "wave by a large number makes its energy as negative as one likes: the "
            "classical energy is unbounded below.")
```

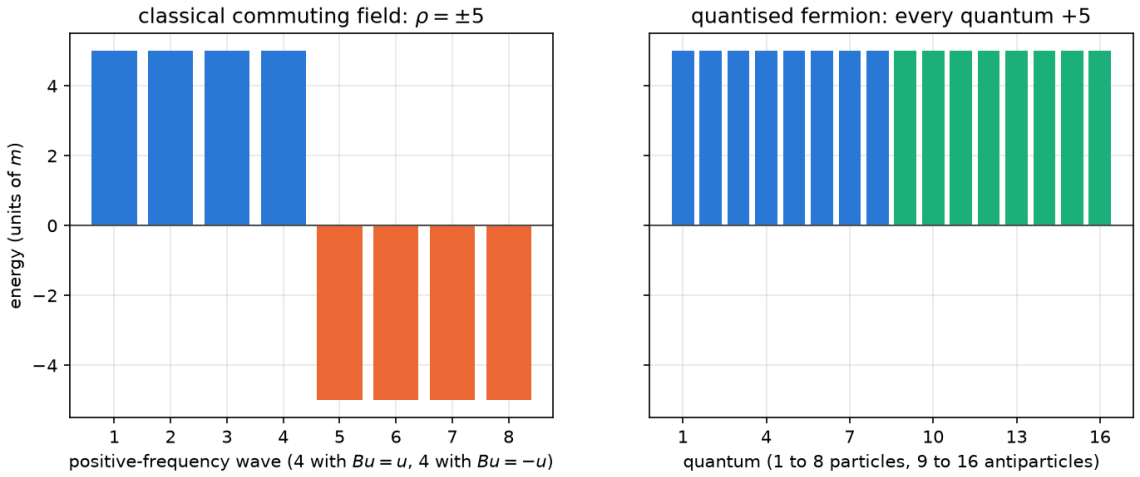

Figure 281: The same momentum $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$ in the two fields of the theory; vertical axes: energy in units of m. Left: the classical COMMUTING field dirac16complex00, energy density of the 8 positive-frequency waves $\Phi = u\, e^{i(k\cdot x - 5x_4)}$ of a basis with $Bu = \pm u$ ($|c| = 1$): four have $+5$ (blue), four have -5 (orange). Right: the quantised fermion field dirac16complex, normal-ordered energy of its 16 one-quantum states: $+5$ for each of the 8 particles (blue) and the 8 antiparticles (green). Quantisation with anticommutators and normal ordering makes the good-sector energy positive; the classical commuting field has no such mechanism. (Notebook 10c, figure 4.)

Out [11]:

```
Figure 10c.4 saved as Revision/textbook/figures/10c_4_commuting_energy.png
```

```
RESULT smallest and largest energy density of the 4000 waves = -4.364 and 4.333
PASS random positive-frequency commuting waves have energy densities of both signs
```

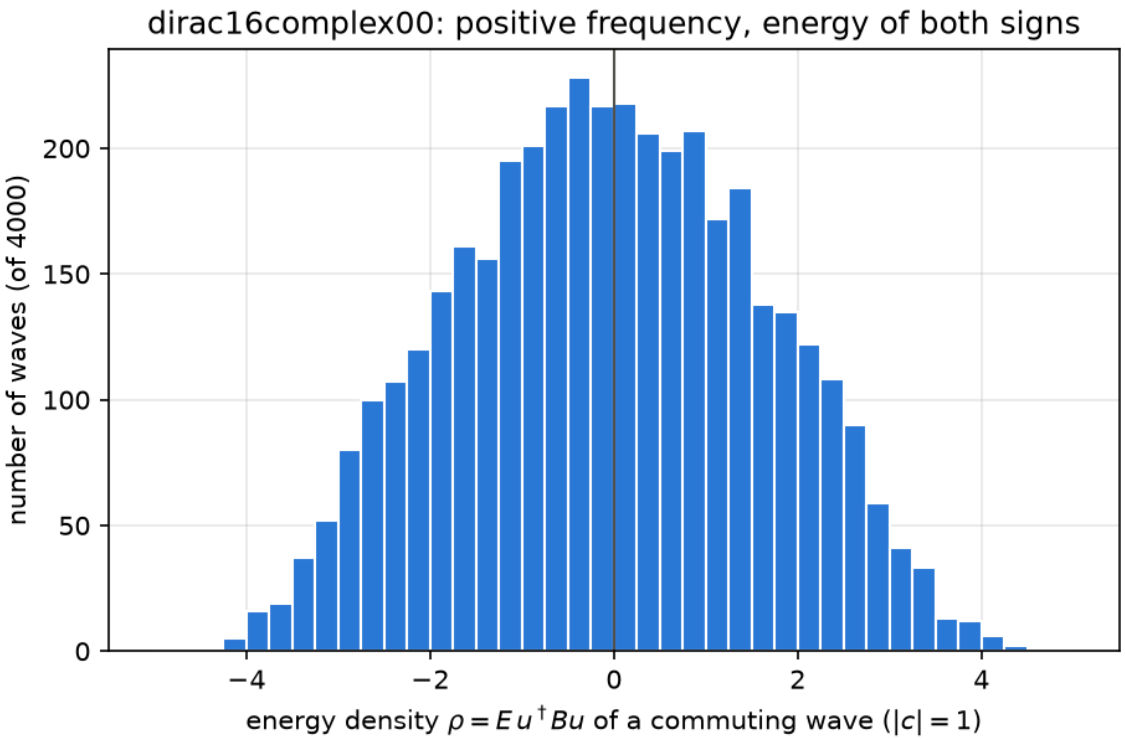

Figure 282: Histogram of the energy density $\rho = E\,u^\dagger B u$ (horizontal axis, units of m, between -5 and 5) of 4000 random positive-frequency plane waves $\Phi = u\,e^{i(k\cdot x - 5x_4)}$ of the classical commuting field dirac16complex00 with $m = 2$, $k = (1, 2, 0, 4)$ and $u^\dagger u = 1$ (vertical axis: number of waves per bin). About half of the waves have negative energy density; multiplying a wave by a large number makes its energy as negative as one likes: the classical energy is unbounded below. (Notebook 10c, figure 5.)

Out [11]:

```
Figure 10c.5 saved as Revision/textbook/figures/10c_5_energy_density_spread.png
```

Text cell:

```
## 12. The last check

The last cell checks that all five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [12]:

```
for name in ("10c_1_expectation_rule.png", "10c_2_dirac_sea.png",
             "10c_3_fock_spectrum.png", "10c_4_commuting_energy.png",
```

```
            "10c_5_energy_density_spread.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [12]:

```
PASS the figure file 10c_1_expectation_rule.png exists
PASS the figure file 10c_2_dirac_sea.png exists
PASS the figure file 10c_3_fock_spectrum.png exists
PASS the figure file 10c_4_commuting_energy.png exists
PASS the figure file 10c_5_energy_density_spread.png exists
ALL 14 CHECKS PASSED (notebook 10c)
```

Text cell:

```
## 13. What this notebook showed

- CONSTRUCTED and CHECKED (for the two exact examples of the Revision record,
  $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ and $m = 3$, $k_1 = 4$, both with $E =
  5$): in the good
  sector the mode Hamiltonian is Hermitian with eigenvalues $\pm E$, the field is
  expanded in particle and antiparticle operators on a POSITIVE Fock space, and with
  $\Psi^\dagger = \chi B$ the canonical rule $\{\Psi, \Psi^\dagger\} = B$ holds.
- The vacuum, the filled Dirac sea, has energy $-8E = -40$ per momentum; after
  normal ordering every particle and every antiparticle has energy $+E$, and their
  charges are $+1$ and $-1$. All 65536 states of one momentum have non-negative
  normal-ordered energy $E(N_b + N_d)$.
- The expectation-value rule: $u^\dagger BMu$ for a particle, $-v^\dagger BMv$ for an
  antiparticle (checked for 100 random matrices and for the matrices of the
  WolframScript record).
- The classical COMMUTING field dirac16complex00 has positive-frequency waves of
  energy density $+5$ and $-5$ for the same momentum: its classical energy is
  unbounded below already for $U = 0$ in the good sector (reproduced), and its
  charge is indefinite. The quantum statements of this chapter concern only the
  fermion field dirac16complex: the author defines dirac16complex00 as a classical
  ("semi-classical") field, and the Revision record does not quantise it.
- SCOPE: the positive Fock space is built for single good-sector momenta with frozen
  coefficients (flat space). A positive-norm Hilbert space for the whole field, the
  extra-time sector with its growing waves, and the interacting operator theory
  ($\lambda \neq 0$) beyond a finite model of one good-sector mode set (in which the
  energy-momentum tensor operator obeys the on-shell identity with Wick ordering
  only) are not constructed; the next notebook shows what happens in the curved
  good sector of the author's metric without a boundary condition at $z = \pi/2$.
```

10.26 Line-by-line walk-through of Notebook 10c

The notebook has 12 code cells, In [1] to In [12]; docstrings are left out of the quotations (they are printed in Section 10.25).

In [1], the set-up cell. Its first 249 lines are comments that repeat the run instructions of Section 10.24. The code is that of In [1] of Notebook 10a (Section 10.10), with the line `NOTEBOOK_ID = "10c" # this notebook: chapter 10, example c`. It prints `Set-up of notebook 10c complete: repository folder found, helpers defined.`

In [2], the gammas, B and the mode Hamiltonian.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output
```

The imports and the functions `record_says_pass` and `check_record` of In [2] of Notebook 10a (Section 10.10): `check_record` stops the notebook if the cited report check is missing or not PASS (the question that `record_says_pass` answers, reading each report file once into `REPORT_CHECKS`), and otherwise prints the PASS line and the line naming the reproduced record in one piece. This notebook needs no sympy.

```
BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
```

The four colours of the figures (as in Notebook 10a), the gammas of the record `Revision/algebra/gammas.json`, C and B.

```
def mode_hamiltonian(m, k):
    h = -1j * m * gamma[4]
    for a, k_a in k.items():
        h = h - k_a * (gamma[4] @ gamma[a])
    return h
```

The mode Hamiltonian $h = -im\gamma^{(x_4)} - \gamma^{(x_4)} \sum_a k_a \gamma^{(x_a)}$ of Section 10.3, exactly as in In [5] of Notebook 10a; `k` is a dictionary of momenta.

```
def orthonormal_columns(P):
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T


say("gammas, C, B and the mode Hamiltonian are ready")
```

The Gram-Schmidt function of In [7] of Notebook 10b (Section 10.19): orthonormal columns that span the range of P. The last line prints the single output line of the cell.

In [3], the waves of one good-sector momentum.

```
def good_sector_modes(m, k):
    h = mode_hamiltonian(m, k)
    E = np.sqrt(m ** 2 + sum(k_a ** 2 for k_a in k.values()))  # no extra times
    U_plus = orthonormal_columns((np.eye(16) + h / E) / 2)
    V_minus = orthonormal_columns((np.eye(16) - h / E) / 2)
    return h, E, np.hstack([U_plus, V_minus])
```

For a good-sector momentum (only the directions 1, 2, 3, 8 in `k`) the function computes h, the energy $E = \sqrt{m^2 + \sum_a k_a^2}$, and orthonormal bases of the ranges of the projectors $\frac{1}{2}(I_{16} \pm h/E)$: the eigenvectors $u_1, \ldots, u_8$ of $+E$ and $v_1, \ldots, v_8$ of $-E$ (Section 10.20). It returns h, E and the matrix W with these 16 columns.

```
h, E, W = good_sector_modes(2, {0: 4, 1: 1, 2: 2})
report("E for m = 2, k = (4, 1, 2, 0)", f"{E:.12f}")
hermitian = np.max(np.abs(h - h.conj().T)) < 1e-15
square_ok = np.max(np.abs(h @ h - 25 * np.eye(16))) < 1e-13
commutes = np.max(np.abs(B @ h - h @ B)) < 1e-15
```

The recorded momentum $m = 2$, $k_1 = 1$, $k_2 = 2$, $k_0 = 4$ gives $E = \sqrt{4 + 1 + 4 + 16} = 5$, printed as `5.000000000000`. The three truth values test that h is Hermitian, that $hh = 25I_{16}$ and that h commutes with B.

```
eigen_ok = (np.max(np.abs(h @ W[:, :8] - E * W[:, :8])) < 1e-13
            and np.max(np.abs(h @ W[:, 8:] + E * W[:, 8:])) < 1e-13)
complete = (np.max(np.abs(W.conj().T @ W - np.eye(16))) < 1e-13
            and np.max(np.abs(W @ W.conj().T - np.eye(16))) < 1e-13)
say(f"h Hermitian: {hermitian}, h^2 = 25 I: {square_ok}, B h = h B: {commutes}")
say(f"8 + 8 eigenvectors: {eigen_ok}; orthonormal and complete: {complete}")
check(hermitian and square_ok and commutes and eigen_ok and complete
      and W.shape == (16, 16),
      "good sector: h Hermitian, h^2 = E^2, 8 + 8 orthonormal complete eigenvectors")
```

`W[:, :8]` are the first eight columns, the u_s, which must obey $hu = Eu$; `W[:, 8:]` are the last eight, the v_s, with $hv = -Ev$. `complete` tests $W^\dagger W = I_{16}$ (orthonormal) and $WW^\dagger = I_{16}$ (complete). The two printed lines show `True` for all five properties, and the check requires them and that W has 16 rows and 16 columns.

In [4], the positive Fock space and the field operators.

```
def sign_below(n, p):
    return -1 if bin(n & ((1 << p) - 1)).count("1") % 2 else 1


def annihilate(p, state):  # f_p
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:
            new = n ^ (1 << p)
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):  # f_p^*
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:
            new = n | (1 << p)
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result
```

The Fock space of In [3] of Notebook 10b, explained line by line in Section 10.19: `sign_below` is the sign $(-1)^s$ (s the number of occupied modes below p, written here in one line), `annihilate` is f_p (it empties mode p where it is occupied) and `create` is f_p^* (it fills mode p where it is empty), both with that sign.

```
def add(*terms):
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result
```

```
def inner(left, right):
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    return max((abs(a) for a in state.values()), default=0.0)
```

The same three helpers as in Notebook 10b: combinations of states, the positive inner product, and the largest size of an amplitude.

```
def F(p, state):  # F_p: b (p < 8) or d^* (p >= 8)
    return annihilate(p, state) if p < 8 else create(p, state)


def F_star(p, state):  # the Hilbert adjoint of F_p
    return create(p, state) if p < 8 else annihilate(p, state)
```

The 16 modes are used as follows: modes 0 to 7 are the particles, $b_s = f_{s-1}$; modes 8 to 15 the antiparticles, $d_s = f_{s+7}$. In the field $\Psi = \sum_s (u_s b_s + v_s d_s^*)$ the particle modes enter with b and the antiparticle modes with d^*, so the field is $\Psi_A = \sum_p W_{Ap} F_p$ with $F_p = f_p$ for $p < 8$ and $F_p = f_p^*$ for $p \geq 8$. `F` is F_p, and `F_star` its Hilbert adjoint.

```
def bilinear(N, state, modes):
    M = modes.conj().T @ N @ modes
    terms = []
    for q in range(16):
        lowered = F(q, state)
        if lowered:
            terms += [(M[p, q], F_star(p, lowered)) for p in range(16)
                      if abs(M[p, q]) > 1e-15]
    return add(*terms)
```

`bilinear(N, state, modes)` applies the operator $\chi N \Psi = \sum_{A,C} \chi_A N_{AC} \Psi_C$ to a state. Inserting $\Psi_C = \sum_q W_{Cq} F_q$ and $\chi_A = \sum_p W_{Ap}^* F_p^*$ gives $\sum_{p,q} (W^\dagger N W)_{pq} F_p^* F_q$, which needs only the 16×16 matrix $\mathtt{M} = W^\dagger N W$. For each q the function applies F_q once (`lowered`); an empty dictionary counts as false, so a zero result is skipped. Then it adds $M_{pq} F_p^*$ applied to it for every p with a nonzero entry; `terms += [...]` appends the new pairs to the list.

```
def psi(A, state, modes):  # Psi_A = sum_p W_Ap F_p
    return add(*[(modes[A - 1, p], F(p, state)) for p in range(16)])


def chi(A, state, modes):  # chi_A = sum_p conj(W_Ap) F_p^*
    return add(*[(np.conj(modes[A - 1, p]), F_star(p, state)) for p in range(16)])


def psi_dagger(A, state, modes):  # the canonical conjugate: sum_C chi_C B_CA
    return add(*[(B[C_ - 1, A - 1], chi(C_, state, modes)) for C_ in range(1, 17)
                 if B[C_ - 1, A - 1] != 0])
```

The field component Ψ_A, its Hilbert adjoint χ_A and the canonical conjugate $\Psi^\dagger_A = \sum_C \chi_C B_{CA}$, as the comments say.

```
VACUUM = {0: 1.0}
rng = np.random.default_rng(12345)
phi = {int(n): complex(rng.normal(), rng.normal())
       for n in rng.integers(0, 2 ** 16, size=5)}  # a random state
```

The vacuum (every mode empty: no particle and, in this realisation, no antiparticle), a random generator with the fixed seed 12345, and a random state of 5 patterns (not normalised; the anticommutator test does not need it).

```
worst_delta, worst_B = 0.0, 0.0
for A in range(1, 17):
    for C_ in range(1, 17):
        with_chi = add((1, psi(A, chi(C_, phi, W), W)), (1, chi(C_, psi(A, phi, W), W)))
        expected = phi if A == C_ else {}
        worst_delta = max(worst_delta, largest(add((1, with_chi), (-1, expected))))
        with_dagger = add((1, psi(A, psi_dagger(C_, phi, W), W)),
                          (1, psi_dagger(C_, psi(A, phi, W), W)))
        worst_B = max(worst_B, largest(add((1, with_dagger), (-B[A - 1, C_ - 1], phi))))
report("largest violation of {Psi_A, chi_C} = delta_AC", f"{worst_delta:.1e}")
report("largest violation of {Psi_A, Psi^dagger_C} = B_AC", f"{worst_B:.1e}")
check(worst_delta < 1e-12 and worst_B < 1e-12,
      "positive Fock space: {Psi, chi} = I and {Psi, Psi^dagger} = B")
```

For all 256 pairs (A, C) the loops compute $\{\Psi_A, \chi_C\}\phi$, which must be $\delta_{AC}\phi$ (completeness, Section 10.20), and $\{\Psi_A, \Psi^\dagger_C\}\phi$, which must be $B_{AC}\phi$. Both largest deviations are printed as `6.3e-16`, rounding only, and the check prints its PASS line: on this positive Fock space the canonical rule holds.

In [5], vacuum energy, normal ordering, energies and charges.

```
def one_quantum_values(N, modes):
    vacuum_value = inner(VACUUM, bilinear(N, VACUUM, modes)).real
    values = []
    for p in range(16):  # p < 8: b^*|0>, p >= 8: d^*|0>
        state = create(p, VACUUM)
        values.append(inner(state, bilinear(N, state, modes)).real - vacuum_value)
    return np.array(values), vacuum_value
```

`one_quantum_values(N, modes)` computes the vacuum value $\langle 0|\chi N\Psi|0\rangle$ and, for each of the 16 one-quantum states (filling mode p of the vacuum gives $b^*_{p+1}|0\rangle$ for $p < 8$ and $d^*_{p-7}|0\rangle$ for $p \geq 8$), the normal-ordered value: the value in that state minus the vacuum value (Section 10.20).

```
energies, vacuum_energy = one_quantum_values(h, W)
charges, vacuum_charge = one_quantum_values(np.eye(16), W)
report("vacuum energy (the filled Dirac sea)", f"{vacuum_energy:.10f}")
report("vacuum charge before normal ordering", f"{vacuum_charge:.10f}")
say(f"normal-ordered energies of the 16 quanta: {np.round(energies, 10).tolist()}")
say(f"normal-ordered charges of the 16 quanta:  {np.round(charges, 10).tolist()}")
```

The energy is $\chi h \Psi$ ($N = h$) and the charge $\chi \Psi$ ($N = I_{16}$). The printed lines show the vacuum energy $-40.0000000000 = -8E$, the vacuum charge 8 before normal ordering, the 16 normal-ordered energies, all 5.0, and the 16 charges, eight times 1.0 and eight times -1.0: exactly the results of Section 10.20.

```
expected_charges = np.array([1] * 8 + [-1] * 8)  # 8 particles, 8 antiparticles
check_record(abs(vacuum_energy + 40) < 1e-12 and np.max(np.abs(energies - 5)) < 1e-12
             and np.max(np.abs(charges - expected_charges)) < 1e-12,
             "m = 2, k = (4, 1, 2, 0): vacuum -40, every quantum +5, charges +1, -1",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_positive_fock_realisation")
```

The check requires these numbers and reproduces the record check `good_sector_positive_fock_realisation`.

In [6], the expectation-value rule.

```
particle_points, antiparticle_points = [], []
for _ in range(100):
    M = rng.normal(size=(16, 16)) + 1j * rng.normal(size=(16, 16))
    values, _ = one_quantum_values(B @ M, W)  # real parts of <:chi B M Psi:>
    particle_points.append((values[0], (W[:, 0].conj() @ B @ M @ W[:, 0]).real))
    antiparticle_points.append((values[8], -(W[:, 8].conj() @ B @ M @ W[:, 8]).real))
particle_points = np.array(particle_points)
antiparticle_points = np.array(antiparticle_points)
```

For 100 random complex matrices M the classical bilinear $\Psi^\dagger M \Psi$ becomes the operator $\chi B M \Psi$, and `one_quantum_values(B @ M, W)` gives its normal-ordered values (real parts) in the 16 one-quantum states. The value in the first particle state ($p = 0$) is paired with the formula $u_1^\dagger B M u_1$, and the value in the first antiparticle state ($p = 8$) with $-v_1^\dagger B M v_1$ (Section 10.21).

```
worst = max(np.max(np.abs(particle_points[:, 0] - particle_points[:, 1])),
            np.max(np.abs(antiparticle_points[:, 0] - antiparticle_points[:, 1])))
report("largest deviation from the rule (100 random M)", f"{worst:.1e}")
check(worst < 1e-12,
      "rule: u^dagger B M u for a particle, -v^dagger B M v for an antiparticle")
```

The largest deviation is printed as `1.8e-15`, rounding only, and the check prints its PASS line.

```
fig, ax = plt.subplots(figsize=(6.0, 5.0))
ax.plot(particle_points[:, 1], particle_points[:, 0], "o", color=BLUE, markersize=5,
        label="particle: against $u^\\dagger B M u$")
ax.plot(antiparticle_points[:, 1], antiparticle_points[:, 0], "s", color=ORANGE,
        markersize=5, fillstyle="none",
        label="antiparticle: against $-v^\\dagger B M v$")
low = min(particle_points.min(), antiparticle_points.min()) - 1
high = max(particle_points.max(), antiparticle_points.max()) + 1
ax.plot([low, high], [low, high], color=GREY, linewidth=1)
ax.set_xlabel("value of the formula (real part)")
```

```
ax.set_ylabel("normal-ordered value on the Fock space (real part)")
ax.set_title("The expectation-value rule in the good sector")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "expectation_rule",
...)
```

The Fock-space values (vertical) are drawn against the formulas (horizontal): blue dots for particles, open orange squares for antiparticles, and the grey diagonal. Figure 1 of Notebook 10c shows all 200 points on the diagonal: the expectation-value rule of the Revision record holds in the good sector.

In [7], the second recorded example.

```
h2, E2, W2 = good_sector_modes(3, {1: 4})
energies2, vacuum2 = one_quantum_values(h2, W2)
charges2, _ = one_quantum_values(np.eye(16), W2)
dense = np.array([[(3 * A + 5 * C_) % 7 - 3 for C_ in range(1, 17)]
                  for A in range(1, 17)])
tests = {"C": C, "-i C gamma4": -1j * C @ gamma[4], "-i C gamma1": -1j * C @ gamma[1],
         "C gamma2 gamma3": C @ gamma[2] @ gamma[3], "dense": dense}
```

The momentum of the Wolfram record, $m = 3$ and $k_1 = 4$ ($E = \sqrt{9+16} = 5$), with its energies and charges. The five test matrices are those named by the record: C, $-iC\gamma^{(x_4)}$, $-iC\gamma^{(x_1)}$, $C\gamma^{(x_2)}\gamma^{(x_3)}$, and a dense matrix of whole numbers, here the notebook's own: entry (A, C) is the remainder of $3A + 5C$ after division by 7 (`%` is the remainder), minus 3.

```
rule_ok = True
for label, M in tests.items():
    values, _ = one_quantum_values(B @ M, W2)
    expected_particle = (W2[:, 0].conj() @ B @ M @ W2[:, 0]).real
    expected_anti = -(W2[:, 8].conj() @ B @ M @ W2[:, 8]).real
    rule_ok = rule_ok and abs(values[0] - expected_particle) < 1e-12 \
        and abs(values[8] - expected_anti) < 1e-12
    say(f"M = {label:16s} particle {values[0]:+.6f}, antiparticle {values[8]:+.6f}")
report("E and the vacuum energy for m = 3, k1 = 4", f"{E2:.6f} and {vacuum2:.6f}")
```

For each test matrix the loop compares the normal-ordered values with the rule (a backslash at the end of a line continues the statement on the next line) and prints one line. The printed values are $+0.6$ for $M = C$ in both states, $+1$ and -1 for $M = -iC\gamma^{(x_4)} = B$ (the charges, as Section 10.21 predicts), 0.8 for $M = -iC\gamma^{(x_1)}$, 0 for $M = C\gamma^{(x_2)}\gamma^{(x_3)}$, and -0.48 and -1.92 for the dense matrix. The RESULT line shows $E = 5$ and the vacuum energy -40.

```
check_record(abs(vacuum2 + 8 * E2) < 1e-12 and np.max(np.abs(energies2 - E2)) < 1e-12
             and np.max(np.abs(charges2 - np.array([1] * 8 + [-1] * 8))) < 1e-12
             and rule_ok,
             "m = 3, k1 = 4: vacuum -8E, energies +E, charges +-1, the rule holds",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "Fock_space_good_sector_example")
```

The check reproduces the Wolfram record check `Fock_space_good_sector_example`: vacuum $-8E$, every quantum $+E$, charges ± 1, and the rule for all five matrices.

In [8], all 65536 states of one momentum.

```
from math import comb  # comb(8, j): the number of ways to choose j of 8 things

eigen_ok = True
for n in rng.integers(0, 2 ** 16, size=200):
    n = int(n)
    n_b = bin(n & 0xFF).count("1")  # occupied particle modes (digits 0 to 7)
    n_d = bin(n >> 8).count("1")  # occupied antiparticle modes (digits 8 to 15)
    state = {n: 1.0}
```

`comb(8, j)` is the binomial coefficient $\binom{8}{j}$. For 200 random patterns: `0xFF` is the number 255 written in base 16, whose eight lowest binary digits are 1, so `n & 0xFF` keeps the digits 0 to 7 of n and their count is N_b; shifting n to the right by 8 binary places keeps the digits 8 to 15, whose count is N_d. `state` is the pattern itself with amplitude 1.

```
    energy_state = bilinear(h, state, W)
    charge_state = bilinear(np.eye(16), state, W)
    eigen_ok = eigen_ok and largest(add(
        (1, energy_state), (-(E * (n_b + n_d) - 8 * E), state))) < 1e-12
    eigen_ok = eigen_ok and largest(add(
        (1, charge_state), (-(n_b - n_d + 8), state))) < 1e-12
check(eigen_ok, "every pattern has energy E (N_b + N_d) - 8E and charge N_b - N_d + 8")
```

The energy and the charge operators are applied to the pattern, and the results are compared with the pattern times $E(N_b + N_d) - 8E$ and times $N_b - N_d + 8$ (the values before normal ordering, Section 10.20). The check confirms that every one of the 200 patterns is an eigenstate with these eigenvalues.

```
counts = np.zeros((17, 17), dtype=int)  # rows: N_b + N_d, columns: N_b - N_d + 8
for n in range(2 ** 16):
    n_b, n_d = bin(n & 0xFF).count("1"), bin(n >> 8).count("1")
    counts[n_b + n_d, n_b - n_d + 8] += 1
formula = np.zeros((17, 17), dtype=int)
for n_b in range(9):
    for n_d in range(9):
        formula[n_b + n_d, n_b - n_d + 8] += comb(8, n_b) * comb(8, n_d)
```

`counts` is a table of whole numbers: row $N_b + N_d$ (0 to 16 quanta) and column $N_b - N_d + 8$ (the charge from -8 to 8, shifted by 8 so that the column numbers start at 0). The first loop goes through all 65536 patterns and adds 1 in the square of each (`+= 1` adds to the stored value). `formula` holds the counts $\binom{8}{N_b}\binom{8}{N_d}$ of Section 10.20.

```
report("number of patterns", int(counts.sum()))
report("patterns with energy 0 (the vacuum only)", int(counts[0].sum()))
report("patterns with one quantum (energy 5)", int(counts[1].sum()))
check(np.array_equal(counts, formula) and counts.sum() == 2 ** 16,
      "the 65536 patterns are counted by binomial(8, N_b) binomial(8, N_d)")
```

The printed numbers are 65536 patterns in all, 1 with the energy 0 (the vacuum) and 16 with one quantum. The check confirms that the direct count equals the formula in every square.

In [9], the Dirac sea and the states of one momentum.

```
k_values = np.linspace(0.0, 5.0, 201)  # the size of the momentum
branch = np.sqrt(2.0 ** 2 + k_values ** 2)  # E for m = 2
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
ax = axes[0]
ax.plot(k_values, branch, color=BLUE, linewidth=2, label="$+E$: particle levels")
ax.plot(k_values, -branch, color=ORANGE, linewidth=2,
        label="$-E$: levels filled in the vacuum")
ax.fill_between(k_values, -branch, -6.5, color=ORANGE, alpha=0.2,
                label="the filled Dirac sea")
ax.set_xlabel("size of the momentum $|k|$ (with $m = 2$)")
ax.set_ylabel("eigenvalue of $h$ (classical wave energy)")
ax.set_ylim(-6.5, 6.5)
ax.set_title("classical energies of the waves")
ax.legend(loc="center right", fontsize=8)
```

The left panel draws the classical wave energies $\pm E = \pm\sqrt{4 + |k|^2}$ for $m = 2$ against the size $|k|$ of the momentum, from 0 to 5. `fill_between` shades, with 20 percent opacity (`alpha=0.2`), the region between the lower curve and the bottom of the picture: the filled sea of negative levels.

```
ax = axes[1]
ax.plot(k_values, branch, color=BLUE, linewidth=2, label="particle $b^*|0\\rangle$")
ax.plot(k_values, branch, "--", color=ORANGE, linewidth=2,
        label="antiparticle $d^*|0\\rangle$ (same curve)")
ax.axhline(0.0, color=GREY, linewidth=1)
ax.set_xlabel("size of the momentum $|k|$ (with $m = 2$)")
ax.set_ylabel("normal-ordered energy of one quantum")
ax.set_ylim(-6.5, 6.5)
ax.set_title("quantum: both quanta have energy $+E$")
ax.legend(loc="lower right", fontsize=8)
save_figure(fig, "dirac_sea",
...)
```

The right panel draws the normal-ordered energies of a particle and of an antiparticle, both $+E$, so the dashed orange curve lies on the blue one. Figure 2 of Notebook 10c shows on the left two classical branches, one of them negative and filled, and on the right only positive energies: quantisation with normal ordering turns the negative classical branch into antiparticles of positive energy.

```
from matplotlib.colors import LinearSegmentedColormap  # colour scales

fig, ax = plt.subplots(figsize=(7.0, 5.0))
shown = np.where(counts > 0, np.log10(np.maximum(counts, 1)), np.nan)  # NaN: empty
light_to_dark = LinearSegmentedColormap.from_list(
    "light_to_dark_blue", ["#cde2fb", "#2a78d6", "#0d366b"])  # 1 state is visible
image = ax.imshow(shown, origin="lower", cmap=light_to_dark, aspect="auto",
                  extent=(-8.5, 8.5, -0.5, 16.5))
```

The second figure is a heat map of the table `counts`. The colour shows the base-10 logarithm of the count (`np.log10`), so that 1 and 4900 states can both be seen; `np.maximum(counts, 1)` avoids the logarithm of 0, and `np.where` puts NaN ("not a number", drawn as white) into the empty squares. The colour scale runs from light blue (one state) to dark blue. `origin="lower"` puts row 0 at the bottom, and `extent` places the squares at the charges $-8, \ldots, 8$ horizontally and the numbers of quanta $0, \ldots, 16$ vertically.

```
ax.annotate("vacuum", (0, 0), xytext=(1.5, 0.3), fontsize=8,
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.annotate("4900 states", (0, 8), xytext=(3.0, 8.6), fontsize=8,
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.set_xlabel("normal-ordered charge $N_b - N_d$")
ax.set_ylabel("number of quanta $N_b + N_d$ (energy $= 5(N_b + N_d)$)")
ax.set_title("The 65536 states of one momentum, $E = 5$")
ax.grid(False)
fig.colorbar(image, ax=ax, label="$\\log_{10}$ of the number of states")
save_figure(fig, "fock_spectrum",
...)
```

Two arrows label the vacuum and the largest group, 4900 states with 8 quanta and charge 0. Figure 3 of Notebook 10c shows a diamond of filled squares: one state at the bottom (the vacuum), 16 in the row above, the largest numbers in the middle, and nothing below the vacuum: no state has negative energy.

In [10], the commuting field dirac16complex00.

```
energy_projector = (np.eye(16) + h / E) / 2  # onto the eigenspace of +E
plus_part = orthonormal_columns(energy_projector @ (np.eye(16) + B) / 2)
minus_part = orthonormal_columns(energy_projector @ (np.eye(16) - B) / 2)
k_sample = {0: 4, 1: 1, 2: 2}  # the momenta of the sample, m = 2
```

In the good sector B commutes with h, so the product of the projector onto the eigenspace of $+E$ with the projector $\frac{1}{2}(I_{16} \pm B)$ projects onto the columns that have both $hu = Eu$ and $Bu = \pm u$. Their orthonormal bases are `plus_part` and `minus_part`.

```
def energy_density(u, m=2.0, k=k_sample):
    Phibar = u.conj() @ C  # the row Phi^dagger C at x = 0
    S = (Phibar @ u).real
    K = {a: (0.5 * (Phibar @ gamma[a] @ (1j * k_a * u))
             - 0.5 * ((-1j * k_a) * Phibar) @ gamma[a] @ u) for a, k_a in k.items()}
    return (-sum(K.values()) + m * S).real
```

`energy_density(u)` computes $\rho = -\sum_a K_a + mS$ for the wave $\Phi = u\, e^{i(k\cdot x - 5x_4)}$ with $|c| = 1$, at the point $x = 0$ and the time 0, directly from the definitions of Section 10.22 and not from the shortcut $\rho = E\, u^\dagger B u$: `Phibar` is the row $\bar\Phi = \Phi^\dagger C$; $\partial_a \Phi = ik_a \Phi$ and $\partial_a \bar\Phi = -ik_a \bar\Phi$ are written as `1j * k_a * u` and `(-1j * k_a) * Phibar`, and `K` holds $K_a = \frac{1}{2}(\bar\Phi \gamma^{(x_a)} \partial_a \Phi - \partial_a \bar\Phi \gamma^{(x_a)} \Phi)$ for the three directions of the momentum.

```
densities_plus = [energy_density(plus_part[:, j]) for j in range(plus_part.shape[1])]
densities_minus = [energy_density(minus_part[:, j])
```

```
                  for j in range(minus_part.shape[1])]
charge_plus = [(u.conj() @ B @ u).real for u in plus_part.T]
charge_minus = [(u.conj() @ B @ u).real for u in minus_part.T]
say(f"dimensions: {plus_part.shape[1]} with B u = +u, {minus_part.shape[1]} with "
    f"B u = -u")
say(f"energy densities: {np.round(densities_plus, 10).tolist()} and "
    f"{np.round(densities_minus, 10).tolist()}")
say(f"charge densities: {np.round(charge_plus, 10).tolist()} and "
    f"{np.round(charge_minus, 10).tolist()}")
```

The energy densities and the charge densities $u^\dagger B u$ of the columns of both bases. The printed lines show 4 columns of each kind, the energy densities 5.0 (four times) and -5.0 (four times), and the charge densities 1.0 and -1.0.

```
check_record(plus_part.shape[1] == 4 and minus_part.shape[1] == 4
             and np.max(np.abs(np.array(densities_plus) - 5)) < 1e-12
             and np.max(np.abs(np.array(densities_minus) + 5)) < 1e-12
             and np.max(np.abs(np.array(charge_plus) - 1)) < 1e-12
             and np.max(np.abs(np.array(charge_minus) + 1)) < 1e-12,
             "commuting field: positive-frequency waves with energy +5 and -5",
             record="Revision/theory/reports/python-scope.json, check "
                    "commuting_field_energy_unbounded_below")
```

The check reproduces the record check `commuting_field_energy_unbounded_below`: waves of the same positive frequency with the energy densities $+5$ and -5.

In [11], the two fields compared.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0), sharey=True)
for ax in axes:
    ax.set_axisbelow(True)  # draw the grid lines behind the bars
classical = densities_plus + densities_minus
axes[0].bar(range(1, 9), classical,
            color=[BLUE] * len(densities_plus) + [ORANGE] * len(densities_minus))
axes[0].axhline(0.0, color=GREY, linewidth=1)
axes[0].set_xticks(range(1, 9))
axes[0].set_xlabel("positive-frequency wave (4 with $Bu = u$, 4 with $Bu = -u$)")
axes[0].set_ylabel("energy (units of $m$)")
axes[0].set_title("classical commuting field: $\\rho = \\pm 5$")
```

Two panels with a common vertical axis (`sharey=True`). The `+` of two Python lists joins them, so `classical` is the list of the eight energy densities; the left panel draws them as bars, blue for $Bu = u$ and orange for $Bu = -u$.

```
axes[1].bar(range(1, 17), energies, color=[BLUE] * 8 + [GREEN] * 8)
axes[1].axhline(0.0, color=GREY, linewidth=1)
axes[1].set_xticks(range(1, 17, 3))
axes[1].set_xlabel("quantum (1 to 8 particles, 9 to 16 antiparticles)")
axes[1].set_title("quantised fermion: every quantum $+5$")
save_figure(fig, "commuting_energy",
...)
```

The right panel draws the normal-ordered energies of the 16 one-quantum states of In [5], blue for particles and green for antiparticles. Figure 4 of Notebook 10c shows four bars of $+5$ and four of -5 on the left, and sixteen bars of $+5$ on the right: the same momentum, a positive energy for every quantum of the quantised fermion field, energies of both signs for the classical commuting field.

```
coefficients = rng.normal(size=(4000, 8)) + 1j * rng.normal(size=(4000, 8))
eigenspace = np.hstack([plus_part, minus_part])  # 8 orthonormal columns
samples = coefficients @ eigenspace.T  # each row: a random column u in the space
samples = samples / np.linalg.norm(samples, axis=1, keepdims=True)
rho = E * np.einsum("ni,ij,nj->n", samples.conj(), B, samples).real
```

4000 random complex combinations of the 8 orthonormal columns of the eigenspace of $+E$: row n of `samples` is $\sum_j c_{nj} e_j$ with the random coefficients c_{nj}, normalised to length 1. For each, `rho` is the energy density $E\,u^\dagger B u$ of Section 10.22 (the sum $u^\dagger B u$ is formed by `np.einsum`, as in In [6] of Notebook 10b).

```
report("smallest and largest energy density of the 4000 waves",
       f"{rho.min():.3f} and {rho.max():.3f}")
check(rho.min() < -2 and rho.max() > 2 and np.all(np.abs(rho) <= 5 + 1e-12),
      "random positive-frequency commuting waves have energy densities of both signs")
```

The printed range is from `-4.364` to `4.333`, inside $[-5, 5]$, and the check requires values below -2 and above 2.

```
fig, ax = plt.subplots()
ax.set_axisbelow(True)
ax.hist(rho, bins=40, range=(-5, 5), color=BLUE, edgecolor="white")
ax.axvline(0.0, color=GREY, linewidth=1)
ax.set_xlabel("energy density $\\rho = E\\,u^\\dagger B u$ of a commuting wave "
              "($|c| = 1$)")
ax.set_ylabel("number of waves (of 4000)")
ax.set_title("dirac16complex00: positive frequency, energy of both signs")
save_figure(fig, "energy_density_spread",
...)
```

A histogram with 40 bins between -5 and 5. Figure 5 of Notebook 10c shows a distribution symmetric about 0: about half of the positive-frequency waves of the commuting field have a negative energy density, and multiplying a wave by a number c multiplies its energy by $|c|^2$, so the classical energy is unbounded below.

In [12], the last check.

```
for name in ("10c_1_expectation_rule.png", "10c_2_dirac_sea.png",
             "10c_3_fock_spectrum.png", "10c_4_commuting_energy.png",
             "10c_5_energy_density_spread.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The five figure files exist, and the last line prints ALL 14 CHECKS PASSED (notebook 10c): the five figure checks and the 9 checks of the cells before (one each in In [3] to In [7], In [10] and In [11], and two in In [8]).

10.27 The curved good sector: symmetric only up to a boundary term

The positive Fock space of Section 10.20 was built for one plane wave with frozen coefficients. In the author's curved metric the waves of the good sector depend on the hidden coordinate x_0, with $z = 6Hx_0$ between the **tip** $z = 0$ and the **patch end** $z = \pi/2$, where $g_{00} = \cot^2 z$ and the volume factor $\sqrt{|g|} = \cos z$ vanish. Is the evolution there still Hermitian, and is the Krein charge still conserved?

The field equation for waves in x_4 and x_0. In the author's metric the field equation of dirac16complex with $U = 0$ is (Chapter 7, record `Revision/theory/field-theory.json`)

$$e^{-a_4}\sin^{-1/6} z\sum_{i=1}^{3}\gamma^{(x_i)}\partial_i\Psi+\gamma^{(x_4)}\partial_4\Psi+e^{a_4}\sin^{-1/6} z\sum_{t=5}^{7}\gamma^{(x_t)}\partial_t\Psi+\tan z\,\gamma^{(x_0)}\partial_0\Psi+3H\gamma^{(x_0)}\Psi=m\Psi.$$

Each derivative is divided by the scale factor of its direction ($1/f_0 = 1/\cot z = \tan z$), and $3H\gamma^{(x_0)}$ is the term $\gamma^\mu\Omega_\mu$ of the canonical spin connection (Chapter 6). For a wave of the good sector that depends only on x_4 and x_0 the first and the third sums vanish:

$$\gamma^{(x_4)}\partial_4\Psi+\tan z\,\gamma^{(x_0)}\partial_0\Psi+3H\gamma^{(x_0)}\Psi=m\Psi.$$

Multiply from the left by $\gamma^{(x_4)}$, use $\gamma^{(x_4)}\gamma^{(x_4)} = -I_{16}$, solve for $\partial_4\Psi$ (as in Section 10.3) and multiply by i:

$$i\,\partial_4\Psi = h\Psi, \qquad h = A + D\,\partial_0,$$

with the matrices

$$A=-im\gamma^{(x_4)}+3iH\gamma^{(x_4)}\gamma^{(x_0)}, \qquad D=\tan z\,M_0, \qquad M_0=i\gamma^{(x_4)}\gamma^{(x_0)}.$$

Four matrix facts. With $(\gamma^{(x_4)})^T = -\gamma^{(x_4)}$, $(\gamma^{(x_0)})^T = \gamma^{(x_0)}$ and $\gamma^{(x_0)}\gamma^{(x_4)} = -\gamma^{(x_4)}\gamma^{(x_0)}$:

(D1) $M_0^\dagger = -i(\gamma^{(x_0)})^T(\gamma^{(x_4)})^T = -i\gamma^{(x_0)}(-\gamma^{(x_4)}) = i\gamma^{(x_0)}\gamma^{(x_4)} = -i\gamma^{(x_4)}\gamma^{(x_0)} = -M_0$: M_0 is anti-Hermitian, and so is D ($\tan z$ is real).

(D2) $\cos z\, D = \cos z \tan z\, M_0 = \sin z\, M_0$.

(D3) $A^\dagger = im(\gamma^{(x_4)})^T - 3iH(\gamma^{(x_0)})^T(\gamma^{(x_4)})^T = -im\gamma^{(x_4)} + 3iH\gamma^{(x_0)}\gamma^{(x_4)} = -im\gamma^{(x_4)} - 3iH\gamma^{(x_4)}\gamma^{(x_0)}$, so $A - A^\dagger = 6iH\gamma^{(x_4)}\gamma^{(x_0)} = 6HM_0$. Since $\partial_0 \sin z = 6H\cos z$ (chain rule, $z = 6Hx_0$), this is $\cos z\,(A - A^\dagger) = (\partial_0 \sin z)\, M_0$.

(D4) B commutes with $\gamma^{(x_4)}$ and $\gamma^{(x_0)}$ (Section 10.2), hence with A, $A^\dagger$, D and M_0.

Symmetric up to a boundary term, line by line. Write u' for $\partial_0 u$, for two wave functions $u(x_0)$ and $v(x_0)$. Then

$$u^\dagger(hv) - (hu)^\dagger v = u^\dagger A v + u^\dagger D v' - u^\dagger A^\dagger v - u'^\dagger D^\dagger v = u^\dagger(A - A^\dagger)v + u^\dagger D v' + u'^\dagger D v.$$

The first step inserts $h = A + D\partial_0$ and uses $(Xw)^\dagger = w^\dagger X^\dagger$; the second collects the A terms and uses $D^\dagger = -D$ (D1). Multiply by $\cos z$ and write $G = \cos z\, D = \sin z\, M_0$ (D2):

$$\cos z\,[u^\dagger(hv) - (hu)^\dagger v] = \cos z\, u^\dagger(A - A^\dagger)v + u^\dagger G v' + u'^\dagger G v = (\partial_0 \sin z)\, u^\dagger M_0 v + \partial_0(u^\dagger G v) - u^\dagger(\partial_0 G)v.$$

The second step uses (D3) for the first term and the product rule $\partial_0(u^\dagger G v) = u'^\dagger G v + u^\dagger G' v + u^\dagger G v'$ for the other two. Since $\partial_0 G = (\partial_0 \sin z) M_0$, the first and the last terms cancel:

$$\cos z\,[u^\dagger(hv) - (hu)^\dagger v] = \partial_0\big(\sin z\; u^\dagger M_0 v\big).$$

The left side, integrated over the hidden direction with the volume $\cos z\, dx_0$, is the difference between $\int \cos z\, u^\dagger(hv)\, dx_0$ and $\int \cos z\, (hu)^\dagger v\, dx_0$; the right side is a **boundary term**: its integral over $0 < z < \pi/2$ is the bracket $\sin z\, u^\dagger M_0 v$ at $z = \pi/2$ minus its value at $z = 0$. At the tip $\sin z = 0$, so for wave functions that stay finite there the bracket vanishes. At the patch end $\sin z = 1$, and the bracket $u^\dagger M_0 v$, the **flux** through $z = \pi/2$, does not vanish in general. Example: $\gamma^{(x_4)}\gamma^{(x_0)}$ is real and symmetric ($(\gamma^{(x_4)}\gamma^{(x_0)})^T = \gamma^{(x_0)}(-\gamma^{(x_4)}) = \gamma^{(x_4)}\gamma^{(x_0)}$) and squares to $-\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_0)} = I_{16}$, so it has the eigenvalue $+1$; for a column with $\gamma^{(x_4)}\gamma^{(x_0)}u = u$ and $u^\dagger u = 2$ the flux is $u^\dagger M_0 u = i\, u^\dagger u = 2i$. The patch end is not infinitely far away: its proper distance from a point z_0 is $\int \sqrt{g_{00}}\, dx_0 = \frac{1}{6H}\int_{z_0}^{\pi/2} \cot z\, dz = -\frac{1}{6H}\ln \sin z_0$, a finite number.

The Krein version. Because B commutes with every matrix in h (D4) and is constant, $B(hu) = h(Bu)$, so $(hu)^\dagger Bv = (Bhu)^\dagger v = (h(Bu))^\dagger v$, while $u^\dagger B(hv) = (Bu)^\dagger(hv)$. The derivation above with u replaced by Bu gives

$$\cos z\,[u^\dagger B(hv) - (hu)^\dagger Bv] = \partial_0\big(\sin z\; u^\dagger B M_0 v\big).$$

The role of the spin-connection term. Without the term $3H\gamma^{(x_0)}$ the matrix A would be $A_0 = -im\gamma^{(x_4)}$ with $A_0 - A_0^\dagger = 0$, and the same steps would leave

$$\cos z\,[u^\dagger(h_0 v) - (h_0 u)^\dagger v] = \partial_0\big(\sin z\, u^\dagger M_0 v\big) - 6H\cos z\, u^\dagger M_0 v\,:$$

a remainder that is not a boundary term. In these variables and with this volume, the spin-connection term is exactly what makes the hidden-direction operator symmetric up to the boundary term. (This role depends on the variables: with $\Psi = \sin^{-1/2} z\, \chi$ the connection term disappears from the equation altogether; `python-scope.json`, check `rescaling_removes_the_connection_term`.)

What follows. The good-sector evolution in the author's metric is Hermitian, and the Krein charge $\int \cos z\, \Psi^\dagger B \Psi\, d^7x$ conserved, only if a **boundary condition** at $z = \pi/2$ removes the flux. The Kohn-Sham record imposes such a condition, the Z2 brane condition, which is ASSUMED (Chapter 14); the quantisation of the Revision record imposes none.

10.28 Waves that do not depend on x_0, and where their Krein charge goes

Finite norm. A wave of the good sector that depends on x_4 only has $\partial_0\Psi = 0$, so h acts on it as the constant matrix A. Its norm is finite: on the patch $0 < z < \pi/2$, that is $0 < x_0 < \pi/(12H)$,

$$\int_0^{\pi/(12H)} \cos(6Hx_0)\,dx_0 = \Big[\frac{\sin(6Hx_0)}{6H}\Big]_0^{\pi/(12H)} = \frac{\sin(\pi/2) - \sin 0}{6H} = \frac{1}{6H},$$

by the antiderivative of the cosine and the chain rule. So these waves are honest members of the good sector.

Growth for $m^2 < 9H^2$. Square A:

$$(-im\gamma^{(x_4)})^2 = -m^2\gamma^{(x_4)}\gamma^{(x_4)} = m^2 I_{16}, \qquad (3iH\gamma^{(x_4)}\gamma^{(x_0)})^2 = -9H^2(\gamma^{(x_4)}\gamma^{(x_0)})^2 = -9H^2 I_{16},$$

and the mixed terms cancel:

$$(-im\gamma^{(x_4)})(3iH\gamma^{(x_4)}\gamma^{(x_0)}) + (3iH\gamma^{(x_4)}\gamma^{(x_0)})(-im\gamma^{(x_4)}) = 3mH\big(\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)} + \gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_4)}\big)$$

$$= 3mH\big(\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)} - \gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_0)}\big) = 0,$$

by multiplying out ($(-i)(3i) = 3$) and by exchanging $\gamma^{(x_0)}\gamma^{(x_4)}$ in the second product. Hence

$$AA = (m^2 - 9H^2)\, I_{16}.$$

For $m^2 < 9H^2$, that is $-3H < m < 3H$, the eigenvalues of A are $\pm i\sqrt{9H^2 - m^2}$ (eight of each, because $\operatorname{tr} A = 0$), and half of these finite-norm waves grow like $e^{\kappa x_4}$ with $\kappa = \sqrt{9H^2 - m^2}$. At $m = H = 1$ the eigenvalues are $\pm\sqrt{-8} = \pm 2\sqrt{2}\,i$ (`python-scope.json`, check `good_sector_x0_independent_modes_without_boundary_condition`). For $m^2 > 9H^2$ they are real. So without a boundary condition the curved good sector of the author's metric contains growing waves whenever $|m| < 3H$.

An exact growing solution. The equation $i\,\partial_4\Psi = A\Psi$ is $\partial_4\Psi = M\Psi$ with $M = -iA = -m\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}$, and $MM = -AA = (9H^2 - m^2)I_{16} = k^2 I_{16}$ with $k = \sqrt{9H^2 - m^2}$. As in Section 10.4 the exponential series collapses (even powers $M^{2j} = k^{2j} I_{16}$, odd powers $M^{2j+1} = k^{2j} M$), and

$$\Psi(x_4) = e^{Mx_4}\chi = \Big(\cosh(kx_4)\, I_{16} + \frac{\sinh(kx_4)}{k}\, M\Big)\chi$$

for any constant column χ, by the power series of cosh and sinh. Check: $\partial_4\Psi = (k\sinh(kx_4)\, I_{16} + \cosh(kx_4)\, M)\chi$, while $M\Psi = (\cosh(kx_4)\, M + \frac{\sinh(kx_4)}{k} MM)\chi = (\cosh(kx_4)\, M + k\sinh(kx_4)\, I_{16})\chi$: equal. This is the member $\alpha = 0$ of the exact family of the Revision theory record (`python-f`

ield-theory.json, check exact_solution_family_x4_x0). For large x_4 both cosh and sinh grow like $\frac{1}{2}e^{kx_4}$, so the ordinary squared length $\Psi^\dagger\Psi$ grows like e^{2kx_4}.

Where the Krein charge goes, line by line. The Krein charge of this wave is $Q(x_4) = \int \cos z\, \Psi^\dagger B\Psi\, dx_0 = \Psi^\dagger B\Psi/(6H)$, because Ψ does not depend on x_0. Its rate of change: from $\partial_4\Psi = -ih\Psi$ and $\partial_4\Psi^\dagger = i(h\Psi)^\dagger$,

$$\partial_4(\Psi^\dagger B\Psi) = i(h\Psi)^\dagger B\Psi - i\Psi^\dagger B(h\Psi) = -i\big[\Psi^\dagger B(h\Psi) - (h\Psi)^\dagger B\Psi\big].$$

Multiply by $\cos z$, integrate over the patch, and use the Krein version of Section 10.27 with $u = v = \Psi$:

$$\frac{dQ}{dx_4} = -i\int \partial_0\big(\sin z\, \Psi^\dagger B M_0\Psi\big)\, dx_0 = -i\,\Psi^\dagger B M_0\Psi\Big|_{z=\pi/2} = \Psi^\dagger B\gamma^{(x_4)}\gamma^{(x_0)}\Psi.$$

The second step integrates the derivative (the bracket vanishes at the tip and is $\Psi^\dagger BM_0\Psi$ at $z = \pi/2$); the third inserts $M_0 = i\gamma^{(x_4)}\gamma^{(x_0)}$ and $-i\cdot i = 1$. The charge is not conserved: it changes by exactly the flux through the patch end. Notebook 10d follows it for $m = H = 1$ ($k = 2\sqrt{2}$) and a column χ with $B\chi = \chi$ and $\chi^\dagger\chi = 1$: $Q(0) = \chi^\dagger B\chi/6 = 1/6$, printed as 0.166667, and $Q(1.5) = 454.007$; the change agrees with the flux added up over time to a relative accuracy of 6.7×10^{-7}.

statement	status	where it is verified
$\cos z\,[u^\dagger(hv) - (hu)^\dagger v] = \partial_0(\sin z\, u^\dagger M_0 v)$; flux $2i$ at $z = \pi/2$ for the recorded column	PROVED	python-scope.json, check good_sector_hermiticity_up_to_the_brane_flux; wolfram-field-theory.json, check good_sector_hermiticity_curved
without $3H\gamma^{(x_0)}$ the remainder $-6H\cos z\, u^\dagger M_0 v$ survives	PROVED	wolfram-field-theory.json, check good_sector_hermiticity_curved
the Krein form is conserved only up to boundary terms	PROVED	wolfram-field-theory.json, check Krein_form_conserved_curved
x_0-independent waves: norm $1/(6H)$, $AA = (m^2 - 9H^2)I_{16}$, $\pm2\sqrt{2}\,i$ at $m = H = 1$	PROVED	python-scope.json, check good_sector_x0_independent_modes_without_boundary_condition
the exact solution $(\cosh kx_4 + \sinh(kx_4)/k\,M)\chi$	PROVED	python-field-theory.json, check exact_solution_family_x4_x0
its Krein charge changes by the flux through $z = \pi/2$	COMPUTED (relative accuracy 6.7×10^{-7})	Notebook 10d, In [8]
a boundary condition at $z = \pi/2$ for the quantised field	OPEN (the Kohn-Sham record ASSUMES the Z2 brane condition)	Revision theory document, section 11

10.29 Example: Notebook 10d studies the curved good sector

Notebook 10d proves the four matrix facts (D1) to (D4) and the boundary-term identities of Section 10.27 exactly with sympy, checks them on two concrete wave functions, computes the flux $2i$ of the recorded column, shows the role of the spin-connection term in a figure, proves $AA = (m^2 - 9H^2)I_{16}$ and the norm $1/(6H)$, follows the eigenvalues of A as the mass grows, and verifies the exact growing solution and the change of its Krein charge by the flux through $z = \pi/2$ (Section 10.28). It draws five figures and ends with the line ALL 16 CHECKS PASSED (notebook 10d).

10.30 How to run Notebook 10d

Step 1. What this notebook does and what it needs.

Notebook 10d (The curved good sector and the boundary at z = pi/2) is the file `Revision/textbook/notebooks/10d_curved_good_sector.ipynb` of the repository Dirac_claude. In the author's metric it writes the good-sector mode operator along the hidden direction, proves exactly with sympy that it is symmetric, and that the Krein form is conserved, only up to a boundary term that vanishes at the tip z = 0 but not at the patch end z = pi/2, shows that without the spin-connection term 3 H a remainder survives, shows that the waves independent of x0 (finite norm) have the frequencies squared m^2 - 9 H^2 and therefore grow when m^2 is below 9 H^2, checks an exact growing solution and that its Krein charge changes by exactly the flux through z = pi/2, reproduces the recorded numbers, and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10d_curved_good_sector.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10d_curved_good_sector.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 10d_curved_good_sector.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10d.captions.json`
- `Revision/textbook/figures/10d_1_volume_and_flux.png`
- `Revision/textbook/figures/10d_2_symmetry_defect.png`
- `Revision/textbook/figures/10d_3_frequency_paths.png`
- `Revision/textbook/figures/10d_4_growth_rate.png`
- `Revision/textbook/figures/10d_5_krein_leak.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10d_5_krein_leak.png exists
ALL 16 CHECKS PASSED (notebook 10d)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10d_curved_good_sector.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10d_curved_good_sector.ipynb
python -m nbconvert --execute --inplace 10d_curved_good_sector.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.31 Notebook 10d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10d_curved_good_sector.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The previous notebooks of this chapter worked with plane waves in flat 4+4 space (or
with the coefficients frozen at one point). This notebook goes back to the author's
curved metric and looks at the good sector (no dependence on the extra times
$x_5, x_6, x_7$) along the hidden direction $x_0$, with $z = 6Hx_0$ running from
the tip $z = 0$ to the patch end $z = \pi/2$. It

- writes the mode operator $h$ of the field equation for waves that depend only on
  $x_4$ and $x_0$: $i\,\partial_4\Psi = h\Psi$;
- proves exactly (sympy) that $h$ is symmetric for the volume $\cos z\,dx_0$ only up
  to a boundary term $\partial_0(\sin z\,u^\dagger M_0v)$, and that the Krein form is
  conserved only up to a similar term; the term vanishes at the tip but NOT at
  $z = \pi/2$;
- shows that the spin-connection term $3H\gamma^{(x_0)}$ of the field equation is
  exactly what makes the hidden-direction operator symmetric up to that boundary
  term (without it a remainder $-6H\cos z\,u^\dagger M_0v$ survives);
- shows that the waves that do not depend on $x_0$ (they have a finite norm) see the
  matrix $A = -im\gamma^{(x_4)} + 3iH\gamma^{(x_4)}\gamma^{(x_0)}$, with $A^2 = (m^2 -
  9H^2)I_{16}$: for $m^2 < 9H^2$ their frequencies are imaginary and they grow;
- checks an exact growing solution, and that its Krein charge changes in time by
  exactly the flux through $z = \pi/2$;
- reproduces the recorded numbers and draws five teaching figures.

The message for the chapter: the positive Fock space of the previous notebook is a
```

```
construction for flat space (frozen coefficients). In the curved good sector the
evolution is Hermitian, and the Krein charge conserved, only if a boundary condition
is imposed at $z = \pi/2$, and the Revision record imposes none.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 10d (The curved good sector and the boundary at z = pi/2) is the file
‘Revision/textbook/notebooks/10d_curved_good_sector.ipynb‘ of the repository
Dirac_claude. In the author’s metric it writes the good-sector mode operator along the
hidden direction, proves exactly with sympy that it is symmetric, and that the Krein form
is conserved, only up to a boundary term that vanishes at the tip z = 0 but not at the
patch end z = pi/2, shows that without the spin-connection term 3 H a remainder survives,
shows that the waves independent of x0 (finite norm) have the frequencies squared m^2 - 9
H^2 and therefore grow when m^2 is below 9 H^2, checks an exact growing solution and that
its Krein charge changes by exactly the flux through z = pi/2, reproduces the recorded
numbers, and draws five teaching figures. It needs a computer with Windows 11, macOS or
Linux, an internet connection for the installation, the program Git, and Python 3.12 or
newer (the notebooks were built with Python 3.14.5) with these packages at exactly these
versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10,
nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook
itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program
that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 10d_curved_good_sector.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical

laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10d_curved_good_sector.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10d_curved_good_sector.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10d.captions.json‘
- ‘Revision/textbook/figures/10d_1_volume_and_flux.png‘
- ‘Revision/textbook/figures/10d_2_symmetry_defect.png‘
- ‘Revision/textbook/figures/10d_3_frequency_paths.png‘
- ‘Revision/textbook/figures/10d_4_growth_rate.png‘
- ‘Revision/textbook/figures/10d_5_krein_leak.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10d_5_krein_leak.png exists
ALL 16 CHECKS PASSED (notebook 10d)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started

without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10d_curved_good_sector.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 10d_curved_good_sector.ipynb
python -m nbconvert --execute --inplace 10d_curved_good_sector.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 10d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 10d (The curved good sector and the boundary at z = pi/2) is the file
# Revision/textbook/notebooks/10d_curved_good_sector.ipynb of the repository
# Dirac_claude. In the author's metric it writes the good-sector mode operator along the
# hidden direction, proves exactly with sympy that it is symmetric, and that the Krein
# form is conserved, only up to a boundary term that vanishes at the tip z = 0 but not at
# the patch end z = pi/2, shows that without the spin-connection term 3 H a remainder
# survives, shows that the waves independent of x0 (finite norm) have the frequencies
# squared m^2 - 9 H^2 and therefore grow when m^2 is below 9 H^2, checks an exact growing
# solution and that its Krein charge changes by exactly the flux through z = pi/2,
# reproduces the recorded numbers, and draws five teaching figures. It needs a computer
# with Windows 11, macOS or Linux, an internet connection for the installation, the
# program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
```

```
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
```

```
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10d_curved_good_sector.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10d_curved_good_sector.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
```

```
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10d_curved_good_sector.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10d.captions.json
# - Revision/textbook/figures/10d_1_volume_and_flux.png
# - Revision/textbook/figures/10d_2_symmetry_defect.png
# - Revision/textbook/figures/10d_3_frequency_paths.png
# - Revision/textbook/figures/10d_4_growth_rate.png
# - Revision/textbook/figures/10d_5_krein_leak.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 10d_5_krein_leak.png exists
#     ALL 16 CHECKS PASSED (notebook 10d)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10d_curved_good_sector.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
```

```
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10d_curved_good_sector.ipynb
#     python -m nbconvert --execute --inplace 10d_curved_good_sector.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10d"  # this notebook: chapter 10, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
```

```
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
```

```
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Hidden direction, $z$, tip, patch end**: $x_0$ is the hidden space direction;
```

  $z = 6Hx_0$ runs over $0 < z < \pi/2$. The *tip* is $z \to 0$, the *patch end* is $z = \pi/2$, where the author's metric component $g_{00} = \cot^2 z$ and the volume factor $\sqrt{|g|} = \cos z$ vanish.
- **Volume element**: $\sqrt{|g|} = \cos z$; an integral over the hidden direction is $\int \cos z\,(\ldots)\,dx_0$.
- **Symmetric operator (formally Hermitian)**: an operator h with $\int\cos z\,u^\dagger(hv)\,dx_0 = \int\cos z\,(hu)^\dagger v\,dx_0$ for all wave functions u, v. A *boundary term* is a part of the integrand that is a derivative, $\partial_0(\ldots)$; its integral is the difference of the bracket at the two ends, so the operator is symmetric when that difference vanishes.
- **Krein form, Krein charge**: $\int\cos z\,u^\dagger Bv\,dx_0$; for $u = v = \Psi$ it is the charge of the wave.
- **Flux**: the value of the bracket at an end; a nonzero flux at $z = \pi/2$ means that something flows through the patch end (in or out).
- **Boundary condition**: a rule imposed on the wave functions at an end (for example that a flux vanishes there); it is an assumption added to the equations.
- **M_0**: the matrix $i\gamma^{(x_4)}\gamma^{(x_0)}$.
- **Exponential of a matrix**: $e^{Mx_4} = \sum_n (Mx_4)^n/n!$; if $M^2 = k^2 I$ the series collapses to $\cosh(kx_4)I + \frac{\sinh(kx_4)}{k}M$ (even and odd powers).

Text cell:

```
## 4. The physical and mathematical situation

**The field equation for waves in $x_4$ and $x_0$ only.** The field equation of
dirac16complex in the author's metric (Revision theory record, $U = 0$) is

$$e^{-a_4}\sin^{-1/6}z\sum_{i=1}^3\gamma^{(x_i)}\partial_i\Psi + \gamma^{(x_4)}
\partial_4\Psi + e^{a_4}\sin^{-1/6}z\sum_{t=5}^7\gamma^{(x_t)}\partial_t\Psi +
\tan z\,\gamma^{(x_0)}\partial_0\Psi + 3H\gamma^{(x_0)}\Psi = m\Psi .$$

For a wave that depends only on $x_4$ and $x_0$ the first and third groups vanish:

$$\gamma^{(x_4)}\partial_4\Psi + \tan z\,\gamma^{(x_0)}\partial_0\Psi +
3H\gamma^{(x_0)}\Psi = m\Psi .$$

Multiply from the left by $\gamma^{(x_4)}$, use $(\gamma^{(x_4)})^2 = -1$ and solve
for $\partial_4\Psi$:

$$\partial_4\Psi = -m\gamma^{(x_4)}\Psi + \tan z\,\gamma^{(x_4)}\gamma^{(x_0)}
\partial_0\Psi + 3H\gamma^{(x_4)}\gamma^{(x_0)}\Psi .$$

Multiply by $i$:

$$i\,\partial_4\Psi = h\Psi,\qquad h = A + D\,\partial_0,\qquad
A = -im\gamma^{(x_4)} + 3iH\gamma^{(x_4)}\gamma^{(x_0)},\qquad D = \tan z\,M_0,
\qquad M_0 = i\gamma^{(x_4)}\gamma^{(x_0)} .$$

**Symmetry up to a boundary term, line by line.** Write $u'$ for $\partial_0u$. Then

$$u^\dagger(hv) - (hu)^\dagger v = u^\dagger(A - A^\dagger)v + u^\dagger Dv' -
u'^\dagger D^\dagger v .$$
```

```
The notebook checks that $M_0$ is anti-Hermitian, so $D^\dagger = -D$ ($\tan z$ is
real), and the last two terms are $u^\dagger Dv' + u'^\dagger Dv =
(u^\dagger Dv)' - u^\dagger D'v$ (product rule). Multiply by $\cos z$. The notebook
checks the two matrix identities $\cos z\,D = \sin z\,M_0$ and
$\cos z\,(A - A^\dagger) = (\sin z)'M_0$; with them (and the product rule
$(\cos z\,D)' = (\cos z)'D + \cos z\,D'$) everything collects into one derivative:

$$\cos z\,\big[u^\dagger(hv) - (hu)^\dagger v\big] = \partial_0\big(\sin z\;
u^\dagger M_0 v\big).$$

Integrated over $0 < z < \pi/2$ this is $[\sin z\,u^\dagger M_0v]$ at $z = \pi/2$
minus its value at $z = 0$. At the tip $\sin z = 0$; at the patch end $\sin z = 1$,
so the flux $u^\dagger M_0v$ there does not vanish in general. The term
$3iH\gamma^{(x_4)}\gamma^{(x_0)}$ of $A$ (the spin-connection term $3H\gamma^{(x_0)}$
of the field equation) supplies exactly $A - A^\dagger = 6HM_0$, which is needed
because $(\sin z)' = 6H\cos z$.

**Waves that do not depend on $x_0$.** For them $\partial_0\Psi = 0$ and $h$ acts as
the constant matrix $A$. Their norm $\int_0^{\pi/(12H)}\cos(6Hx_0)\,dx_0 = 1/(6H)$ is
finite (the patch $0 < z < \pi/2$ is $0 < x_0 < \pi/(12H)$), so they are honest
members of the good sector. The notebook proves $A^2 = (m^2 - 9H^2)I_{16}$: for
$m^2 < 9H^2$, that is $-3H < m < 3H$, the frequencies are imaginary and these waves
grow. (The pictures below take $m \geq 0$; the mass enters only through $m^2$.)
```

Text cell:

```
## 5. The matrices of the hidden direction

The next cell reads the gammas, builds $C$ and $B$ with numpy and as exact sympy
matrices, defines the symbols $m$, $H$ (positive) and $x_0$, the expression
$z = 6Hx_0$, and the matrices $M_0$, $A$ and $D$ of Section 4. It also defines
‘check_record‘ (the PASS line and the line of the reproduced Revision record are
printed in one piece, so that the stored output is the same in every run; before
that it confirms with the helper ‘record_says_pass‘ that the cited report check
still exists with the verdict PASS) and the colours. Then it checks the matrix
facts of Section 4 exactly: $M_0^\dagger = -M_0$, $\cos z\,D = \sin z\,M_0$ and
$\cos z\,(A - A^\dagger) = \partial_0(\sin z)\,M_0$, and in addition that $B$
commutes with $\gamma^{(x_4)}$ and $\gamma^{(x_0)}$. The last fact gives the Krein
version: $B$ then commutes with $A$, $A^\dagger$, $D$ and $M_0$, so the same steps
with $u^\dagger B$ in place of $u^\dagger$ give $\cos z\,[u^\dagger B(hv) -
(hu)^\dagger Bv] = \partial_0(\sin z\,u^\dagger BM_0v)$.
```

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols
```

```
REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
B_exact = -sp.I * g[0] * g[1] * g[2] * g[3] * g[4]

m = sp.Symbol("m", real=True)  # the mass
H = sp.Symbol("H", positive=True)  # the author's constant H > 0
x0 = sp.Symbol("x0", real=True)  # the hidden coordinate
z = 6 * H * x0  # z = 6 H x0
M0 = sp.I * g[4] * g[0]  # M_0 = i gamma^(x4) gamma^(x0)
A = -sp.I * m * g[4] + 3 * sp.I * H * g[4] * g[0]  # the part without derivative
A_no_connection = -sp.I * m * g[4]  # the same without the term 3 H
D = sp.tan(z) * M0  # the coefficient of the derivative d/dx0
zero = sp.zeros(16, 16)
anti_hermitian = M0.H == -M0
identity_D = (sp.cos(z) * D - sp.sin(z) * M0).applyfunc(sp.simplify) == zero
identity_A = (sp.cos(z) * (A - A.H) - sp.diff(sp.sin(z), x0) * M0).applyfunc(
    sp.simplify) == zero
B_commutes = (B_exact * g[4] == g[4] * B_exact) and (B_exact * g[0] == g[0] * B_exact)
say(f"M0 anti-Hermitian: {anti_hermitian}; cos z D = sin z M0: {identity_D}; "
    f"cos z (A - A^dagger) = d0(sin z) M0: {identity_A}; B commutes with "
    f"gamma^(x4) and gamma^(x0): {B_commutes}")
check(anti_hermitian and identity_D and identity_A and B_commutes,
      "exact: the four matrix identities of the hidden direction")
```

Out [2]:

```
M0 anti-Hermitian: True; cos z D = sin z M0: True; cos z (A - A^dagger) = d0(sin z) M0:
    True; B commutes with gamma^(x4) and gamma^(x0): True
PASS exact: the four matrix identities of the hidden direction
```

Text cell:

```
## 6. Symmetric up to a boundary term, checked on wave functions

The matrix identities prove the statement of Section 4 for all wave functions. The
next cell also checks it directly on two concrete wave functions $u(x_0)$ and
$v(x_0)$: each of their 16 entries is a combination $a + b\sin z + c\cos z$ with
small whole numbers $a, b, c$ (real and imaginary parts chosen at random with a
fixed seed). It computes, exactly,

- the defect $\cos z\,[u^\dagger(hv) - (hu)^\dagger v] - \partial_0(\sin z\,u^\dagger
  M_0v)$, which must be 0;
- the same with the operator WITHOUT the spin-connection term, which must equal
  $-6H\cos z\,u^\dagger M_0v$ instead;
- the Krein version $\cos z\,[u^\dagger B(hv) - (hu)^\dagger Bv] - \partial_0(\sin z
  \,u^\dagger BM_0v)$, which must be 0.

To let sympy recognise the zeros, $x_0$ is replaced by $z/(6H)$ and the
trigonometric functions are expanded before simplifying.
```

In [3]:

```
rng = np.random.default_rng(12345)


def test_function():
    """16 entries a + b sin z + c cos z with small random whole numbers."""
    def number():
        return int(rng.integers(-3, 4)) + sp.I * int(rng.integers(-3, 4))
    return sp.Matrix([number() + number() * sp.sin(z) + number() * sp.cos(z)
                      for _ in range(16)])


def apply_h(matrix_A, w):
    """h w = A w + D dw/dx0 for a column w of functions of x0."""
    return matrix_A * w + D * w.diff(x0)


def is_zero(expression):
    """Exact zero test: write x0 = z / (6 H), expand the trigonometric functions."""
    zz = sp.Symbol("zz")
    rewritten = sp.expand_trig(sp.expand(expression.subs(x0, zz / (6 * H))))
    return sp.simplify(rewritten) == 0


u, v = test_function(), test_function()
hv, hu = apply_h(A, v), apply_h(A, u)
defect = (sp.cos(z) * ((u.H * hv)[0] - (hu.H * v)[0])
```

```
          - sp.diff(sp.sin(z) * (u.H * M0 * v)[0], x0))
hv0, hu0 = apply_h(A_no_connection, v), apply_h(A_no_connection, u)
defect_without = (sp.cos(z) * ((u.H * hv0)[0] - (hu0.H * v)[0])
                  - sp.diff(sp.sin(z) * (u.H * M0 * v)[0], x0))
remainder_ok = is_zero(defect_without + 6 * H * sp.cos(z) * (u.H * M0 * v)[0])
krein_defect = (sp.cos(z) * ((u.H * B_exact * hv)[0] - (hu.H * B_exact * v)[0])
                - sp.diff(sp.sin(z) * (u.H * B_exact * M0 * v)[0], x0))
check_record(is_zero(defect),
             "exact: cos z (u^dagger h v - (h u)^dagger v) = d0(sin z u^dagger M0 v)",
             record="Revision/theory/reports/python-scope.json, check "
                    "good_sector_hermiticity_up_to_the_brane_flux")
check_record(remainder_ok,
             "exact: without the term 3 H the remainder -6 H cos z u^dagger M0 v stays",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "good_sector_hermiticity_curved")
check_record(is_zero(krein_defect),
             "exact: the Krein form is conserved up to d0(sin z u^dagger B M0 v)",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "Krein_form_conserved_curved")
```

Out [3]:

```
PASS exact: cos z (u^dagger h v - (h u)^dagger v) = d0(sin z u^dagger M0 v)
     reproduces Revision/theory/reports/python-scope.json, check
    good_sector_hermiticity_up_to_the_brane_flux
PASS exact: without the term 3 H the remainder -6 H cos z u^dagger M0 v stays
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    good_sector_hermiticity_curved
PASS exact: the Krein form is conserved up to d0(sin z u^dagger B M0 v)
     reproduces Revision/theory/reports/wolfram-field-theory.json, check
    Krein_form_conserved_curved
```

Text cell:

```
## 7. The flux at the two ends

The bracket $\sin z\,u^\dagger M_0v$ vanishes at the tip ($\sin 0 = 0$) for wave
functions that stay finite there, but at the patch end $z = \pi/2$ it is
$u^\dagger M_0v$. The Revision record gives an example: the matrix
$\gamma^{(x_4)}\gamma^{(x_0)}$ is real, symmetric and squares to $I$, so it has the
eigenvalue $+1$; for a column with $\gamma^{(x_4)}\gamma^{(x_0)}u = u$ and
$u^\dagger u = 2$ the flux is $u^\dagger M_0u = i\,u^\dagger u = 2i$, not 0. The next
cell builds such a column ($u = e_A + \gamma^{(x_4)}\gamma^{(x_0)}e_A$ for the first
unit column $e_A$) and draws the volume factor $\cos z$ and the flux factor
$\sin z$ along the hidden direction.
```

In [4]:

```
G40 = gamma[4] @ gamma[0]  # gamma^(x4) gamma^(x0), real
symmetric_square = np.array_equal(G40, G40.T) and np.array_equal(G40 @ G40,
                                                                 np.eye(16))
```

```
e_first = np.eye(16)[:, 0]
u_flux = e_first + G40 @ e_first  # an eigenvector of G40 with eigenvalue +1
M0_numbers = 1j * G40
flux_value = u_flux.conj() @ M0_numbers @ u_flux
report("u^dagger u for the column u = e_1 + gamma4 gamma0 e_1", f"{u_flux @ u_flux:.1f}")
report("flux u^dagger M0 u at z = pi/2",
       f"{flux_value.imag:.1f} i (real part {abs(flux_value.real):.1f})")
check_record(symmetric_square and np.allclose(G40 @ u_flux, u_flux)
             and abs(flux_value - 2j) < 1e-14,
             "the flux at z = pi/2 is u^dagger M0 u = 2 i for gamma4 gamma0 u = u",
             record="Revision/theory/reports/python-scope.json, check "
                    "good_sector_hermiticity_up_to_the_brane_flux")

z_values = np.linspace(0.0, np.pi / 2, 400)
fig, ax = plt.subplots()
ax.plot(z_values, np.cos(z_values), color=BLUE, linewidth=2,
        label="volume factor $\\sqrt{|g|} = \\cos z$")
ax.plot(z_values, np.sin(z_values), color=ORANGE, linewidth=2,
        label="flux factor $\\sin z$ of the bracket")
ax.plot([0.0], [0.0], "o", color=ORANGE, markersize=8)
ax.plot([np.pi / 2], [1.0], "o", color=ORANGE, markersize=8)
ax.annotate("tip: the flux vanishes", (0.0, 0.0), xytext=(0.15, 0.25),
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.annotate("patch end: the flux remains", (np.pi / 2, 1.0), xytext=(0.55, 1.08),
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.set_xlabel("$z = 6 H x_0$ (from the tip $z = 0$ to the patch end $z = \\pi/2$)")
ax.set_ylabel("value")
ax.set_ylim(-0.05, 1.25)
ax.set_title("The hidden direction: volume and boundary flux")
ax.legend(loc="center left")
save_figure(fig, "volume_and_flux",
            "Along the hidden direction, $z = 6Hx_0$ from the tip $z = 0$ to the "
            "patch end $z = \\pi/2$ (horizontal axis): the volume factor $\\sqrt{|g|}"
            " = \\cos z$ of the author's metric (blue) and the factor $\\sin z$ of "
            "the boundary bracket $\\sin z\\,u^\\dagger M_0 v$ (orange); vertical "
            "axis: pure numbers. The volume vanishes at the patch end, but the "
            "bracket there is $u^\\dagger M_0 v$, for example $2i$ for the recorded "
            "column: the mode operator is symmetric, and the Krein charge "
            "conserved, only if a boundary condition removes this flux at $z = "
            "\\pi/2$.")
```

Out [4]:

```
RESULT u^dagger u for the column u = e_1 + gamma4 gamma0 e_1 = 2.0
RESULT flux u^dagger M0 u at z = pi/2 = 2.0 i (real part 0.0)
PASS the flux at z = pi/2 is u^dagger M0 u = 2 i for gamma4 gamma0 u = u
     reproduces Revision/theory/reports/python-scope.json, check
    good_sector_hermiticity_up_to_the_brane_flux
```

Out [4]:

```
Figure 10d.1 saved as Revision/textbook/figures/10d_1_volume_and_flux.png
```

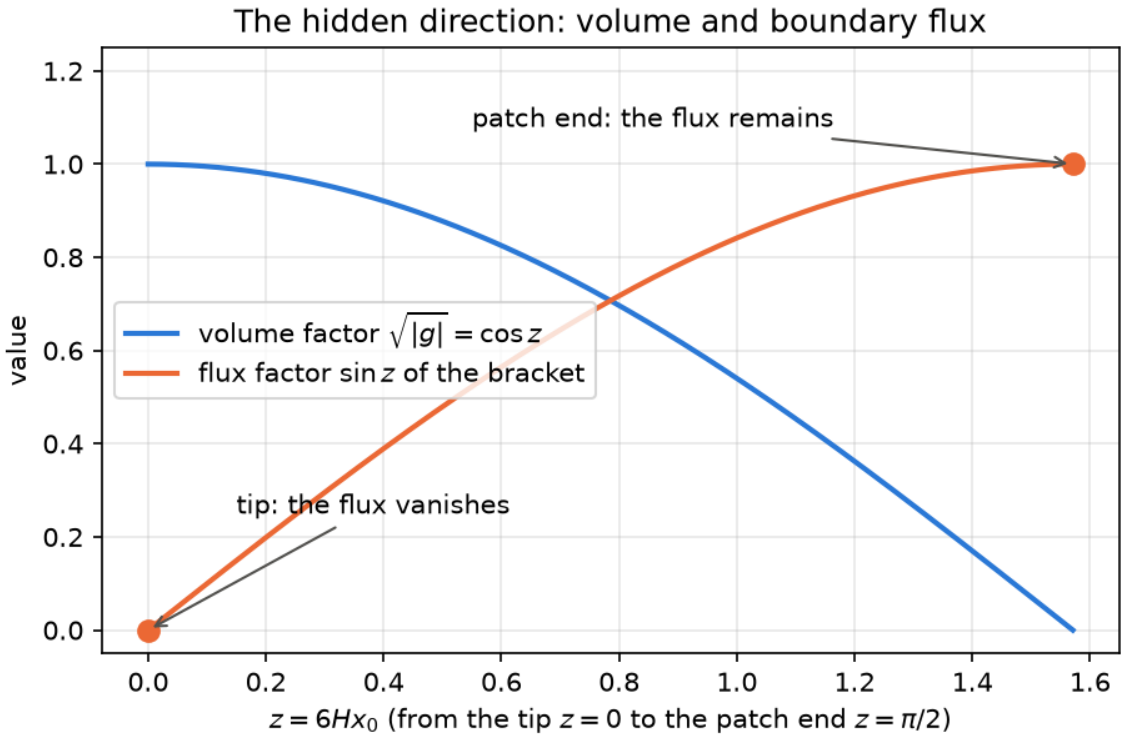

Figure 283: Along the hidden direction, $z = 6Hx_0$ from the tip $z = 0$ to the patch end $z = \pi/2$ (horizontal axis): the volume factor $\sqrt{|g|} = \cos z$ of the author's metric (blue) and the factor $\sin z$ of the boundary bracket $\sin z\, u^\dagger M_0 v$ (orange); vertical axis: pure numbers. The volume vanishes at the patch end, but the bracket there is $u^\dagger M_0 v$, for example $2i$ for the recorded column: the mode operator is symmetric, and the Krein charge conserved, only if a boundary condition removes this flux at $z = \pi/2$. (Notebook 10d, figure 1.)

Text cell:

```
The next cell makes the role of the spin-connection term visible for one pair of
wave functions, here $u = v$ (the test function $u$ of Section 6) at $m = H = 1$:
it evaluates along $0 < z < \pi/2$ the left-hand side $\cos z\,[u^\dagger(hu) -
(hu)^\dagger u]$, the right-hand side $\partial_0(\sin z\,u^\dagger M_0u)$, and the
left-hand side computed WITHOUT the term $3H$. For $u = v$ all three are purely
imaginary (a number minus its complex conjugate), so their imaginary parts are
plotted. The exact expressions are turned into fast numerical functions with
‘sp.lambdify‘.
```

In [5]:

```
numbers = {m: 1, H: 1}  # m = H = 1, so z = 6 x0
hu_full, hu_bare = apply_h(A, u), apply_h(A_no_connection, u)
left = sp.cos(z) * ((u.H * hu_full)[0] - (hu_full.H * u)[0])
right = sp.diff(sp.sin(z) * (u.H * M0 * u)[0], x0)
left_bare = sp.cos(z) * ((u.H * hu_bare)[0] - (hu_bare.H * u)[0])
x0_values = np.linspace(0.002, np.pi / 12 - 0.002, 300)  # inside 0 < z < pi/2
curves = [np.imag(sp.lambdify(x0, e.subs(numbers), "numpy")(x0_values))
          for e in (left, right, left_bare)]
difference = np.max(np.abs(curves[0] - curves[1]))
report("largest |left - right| on the grid", f"{difference:.1e}")
check(difference < 1e-9, "numerical: the two sides agree along the hidden direction")

fig, ax = plt.subplots()
ax.plot(6 * x0_values, curves[0], color=BLUE, linewidth=3,
        label="left side, with the term $3H$")
ax.plot(6 * x0_values, curves[1], "--", color=GREEN, linewidth=2,
        label="right side $\\partial_0(\\sin z\\,u^\\dagger M_0 u)$")
ax.plot(6 * x0_values, curves[2], color=ORANGE, linewidth=2,
        label="left side WITHOUT the term $3H$")
ax.set_xlabel("$z = 6 H x_0$ ($m = H = 1$)")
ax.set_ylabel("imaginary part")
ax.set_title("The spin-connection term makes the defect a pure boundary term")
ax.legend(fontsize=8)
save_figure(fig, "symmetry_defect",
            "For one concrete wave function $u(x_0)$ (16 entries of the form $a + "
            "b\\sin z + c\\cos z$) and $m = H = 1$: the imaginary parts of the "
            "symmetry defect $\\cos z\\,(u^\\dagger(hu) - (hu)^\\dagger u)$ of the "
            "curved good-sector mode operator (blue), of the derivative "
            "$\\partial_0(\\sin z\\,u^\\dagger M_0u)$ (green dashed, on top of the "
            "blue curve), and of the defect of the operator without the "
            "spin-connection term $3H$ (orange), against $z$ (horizontal axis). "
            "With the term the defect is exactly a derivative, whose integral "
            "is the flux at the ends; without it a remainder $-6H\\cos z\\,"
            "u^\\dagger M_0u$ survives.")
```

Out [5]:

```
RESULT largest |left - right| on the grid = 5.7e-13
PASS numerical: the two sides agree along the hidden direction
```

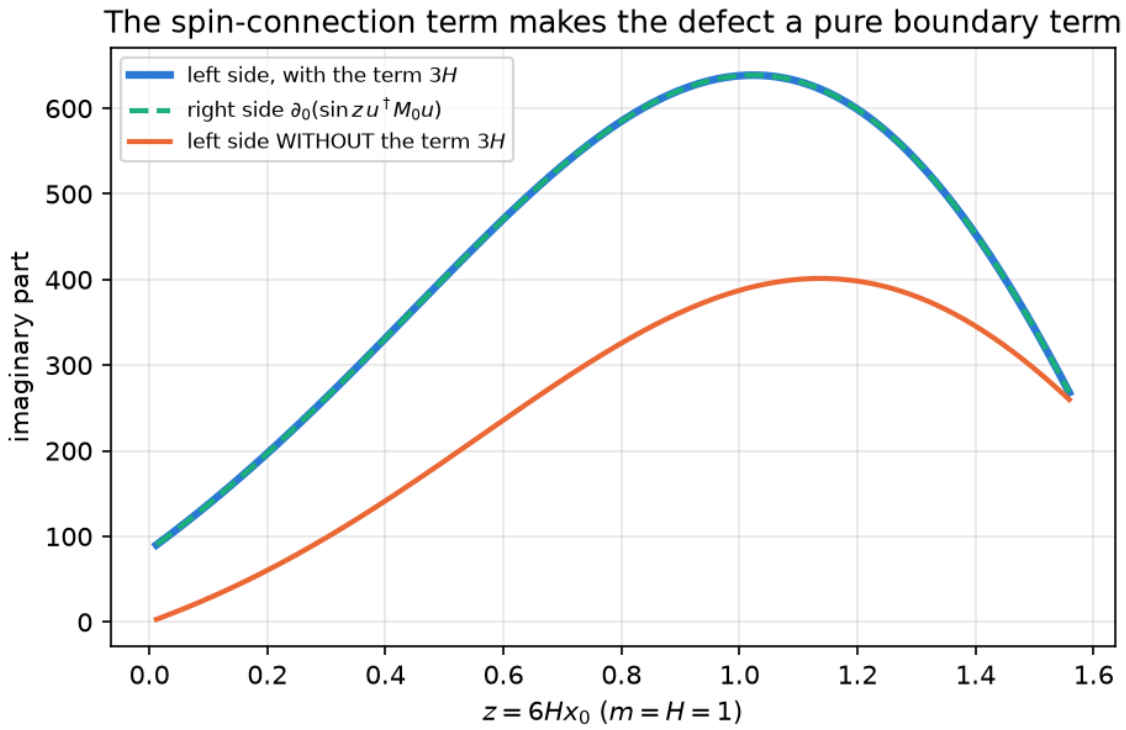

Figure 284: For one concrete wave function $u(x_0)$ (16 entries of the form $a + b\sin z + c\cos z$) and $m = H = 1$: the imaginary parts of the symmetry defect $\cos z\,(u^\dagger(hu) - (hu)^\dagger u)$ of the curved good-sector mode operator (blue), of the derivative $\partial_0(\sin z\, u^\dagger M_0 u)$ (green dashed, on top of the blue curve), and of the defect of the operator without the spin-connection term $3H$ (orange), against z (horizontal axis). With the term the defect is exactly a derivative, whose integral is the flux at the ends; without it a remainder $-6H\cos z\, u^\dagger M_0 u$ survives. (Notebook 10d, figure 2.)

Out [5]:

```
Figure 10d.2 saved as Revision/textbook/figures/10d_2_symmetry_defect.png
```

Text cell:

```
## 8. Waves that do not depend on $x_0$: growth for $m^2 < 9H^2$

For a wave that depends on $x_4$ only, $h$ acts as the constant matrix $A$. The next
cell checks exactly: the norm integral $\int_0^{\pi/(12H)}\cos(6Hx_0)\,dx_0 =
1/(6H)$ is finite; $A$ is not Hermitian; $A^2 = (m^2 - 9H^2)I_{16}$ (the same
cancellation of mixed terms as for the mode Hamiltonian of flat space, because
$\gamma^{(x_4)}$ and $\gamma^{(x_4)}\gamma^{(x_0)}$ anticommute); and at $m = H = 1$
the eigenvalues are $\pm 2\sqrt2\,i$, eight of each (the trace of $A$ is 0).
```

In [6]:

```
norm_integral = sp.integrate(sp.cos(6 * H * x0), (x0, 0, sp.pi / (12 * H)))
say(f"integral of cos(6 H x0) over the patch: {norm_integral}")
A_square_ok = (A * A - (m ** 2 - 9 * H ** 2) * sp.eye(16)).applyfunc(
    sp.expand) == zero
A_not_hermitian = (A - A.H).applyfunc(sp.expand) != zero
G40_numbers = gamma[4] @ gamma[0]  # gamma^(x4) gamma^(x0) as numbers


def A_numbers_of(mass, H_value=1.0):
    """The matrix A as numbers: -i m gamma^(x4) + 3 i H gamma^(x4) gamma^(x0)."""
    return -1j * mass * gamma[4] + 3j * H_value * G40_numbers


A_numbers = A_numbers_of(1.0)
eigenvalues = np.linalg.eigvals(A_numbers)
upper = int(np.sum(np.abs(eigenvalues - 2j * np.sqrt(2)) < 1e-9))
lower = int(np.sum(np.abs(eigenvalues + 2j * np.sqrt(2)) < 1e-9))
report("eigenvalues 2 sqrt(2) i and -2 sqrt(2) i at m = H = 1, how many",
       f"{upper} and {lower}")
check_record(norm_integral == 1 / (6 * H) and A_square_ok and A_not_hermitian
             and (upper, lower) == (8, 8),
             "x0-independent waves: A^2 = (m^2 - 9 H^2) I, at m = H = 1: +-2 sqrt(2) i",
             record="Revision/theory/reports/python-scope.json, check "
                    "good_sector_x0_independent_modes_without_boundary_condition")
```

Out [6]:

```
integral of cos(6 H x0) over the patch: 1/(6*H)
RESULT eigenvalues 2 sqrt(2) i and -2 sqrt(2) i at m = H = 1, how many = 8 and 8
PASS x0-independent waves: A^2 = (m^2 - 9 H^2) I, at m = H = 1: +-2 sqrt(2) i
     reproduces Revision/theory/reports/python-scope.json, check
    good_sector_x0_independent_modes_without_boundary_condition
```

Text cell:

```
The next cell draws two figures. The first follows the 16 eigenvalues of $A$ (with
$H = 1$) in the complex plane as $m$ grows from 0 to 5: they are $\pm\sqrt{m^2 -
9H^2}$, on the imaginary axis for $m < 3H$ (growth) and on the real axis for
$m > 3H$ (oscillation). The second shows the growth rate
$\kappa = \sqrt{9H^2 - m^2}$ (zero for $m \geq 3H$) against $m/H$, with the recorded
point $m = H = 1$, $\kappa = 2\sqrt2$.
```

In [7]:

```
mass_values = np.linspace(0.0, 5.0, 101)
paths = []
for mass in mass_values:
    paths.append(np.linalg.eigvals(A_numbers_of(mass)))
paths = np.array(paths)
worst = np.max(np.abs(np.sort(np.abs(paths), axis=1)
                      - np.sqrt(np.abs(mass_values ** 2 - 9))[:, None]))
report("largest deviation of |eigenvalue| from sqrt|m^2 - 9|", f"{worst:.1e}")
check(worst < 1e-6, "the eigenvalues of A follow +-sqrt(m^2 - 9 H^2)")

fig, ax = plt.subplots(figsize=(6.0, 5.0))
growing = mass_values < 3.0
for column in range(16):
    ax.plot(paths[growing, column].real, paths[growing, column].imag, ".",
            color=ORANGE, markersize=4)
    ax.plot(paths[~growing, column].real, paths[~growing, column].imag, ".",
            color=BLUE, markersize=4)
ax.plot([], [], ".", color=ORANGE, label="$m < 3H$: imaginary, the waves grow")
ax.plot([], [], ".", color=BLUE, label="$m > 3H$: real, the waves oscillate")
ax.plot([0, 0], [2 * np.sqrt(2), -2 * np.sqrt(2)], "o", color=GREEN, markersize=9,
        fillstyle="none", label="recorded: $m = H = 1$, $\\pm 2\\sqrt{2}\\,i$")
ax.set_xlabel("real part of the eigenvalue of $A$ (units of $H$)")
ax.set_ylabel("imaginary part (units of $H$)")
ax.set_aspect("equal")
ax.set_title("Frequencies of the $x_0$-independent waves, $m$ from 0 to 5")
ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1.0), fontsize=8)  # outside
save_figure(fig, "frequency_paths",
            "The 16 eigenvalues of the matrix $A = -im\\gamma^{(x_4)} + "
            "3iH\\gamma^{(x_4)}\\gamma^{(x_0)}$, which governs the good-sector waves "
            "that do not depend on $x_0$, in the complex plane (horizontal axis: "
            "real part, vertical axis: imaginary part, units of $H$) for $H = 1$ "
            "and the mass $m$ from 0 to 5. For $m < 3H$ (orange) they are "
            "$\\pm i\\sqrt{9H^2 - m^2}$ on the imaginary axis: these finite-norm "
            "waves grow. For $m > 3H$ (blue) they are real. Green circles: the "
            "recorded values $\\pm 2\\sqrt{2}\\,i$ at $m = H = 1$.")

rate = np.sqrt(np.maximum(9.0 - mass_values ** 2, 0.0))
fig, ax = plt.subplots()
ax.plot(mass_values, rate, color=ORANGE, linewidth=2,
        label="growth rate $\\kappa = \\sqrt{9H^2 - m^2}$")
ax.plot([1.0], [2 * np.sqrt(2)], "o", color=GREEN, markersize=9,
```

```
        label="recorded: $m = H = 1$, $\\kappa = 2\\sqrt{2}$")
ax.axvline(3.0, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("mass $m$ in units of $H$")
ax.set_ylabel("growth rate $\\kappa$ (units of $H$)")
ax.set_title("Without a boundary condition: growth for $m < 3H$")
ax.legend()
save_figure(fig, "growth_rate",
            "The growth rate $\\kappa = \\sqrt{9H^2 - m^2}$ (vertical axis, units of "
            "$H$) of the good-sector waves that do not depend on $x_0$ in the "
            "author's metric, against the mass $m$ (horizontal axis, units of $H$). "
            "It is positive for $m < 3H$ (left of the dotted line) and zero for "
            "$m \\geq 3H$; the green dot is the recorded value $2\\sqrt{2}$ at "
            "$m = H = 1$. These waves have a finite norm, so without a boundary "
            "condition at $z = \\pi/2$ the curved good sector contains growing "
            "waves.")
```

Out [7]:

```
RESULT largest deviation of |eigenvalue| from sqrt|m^2 - 9| = 4.6e-08
PASS the eigenvalues of A follow +-sqrt(m^2 - 9 H^2)
```

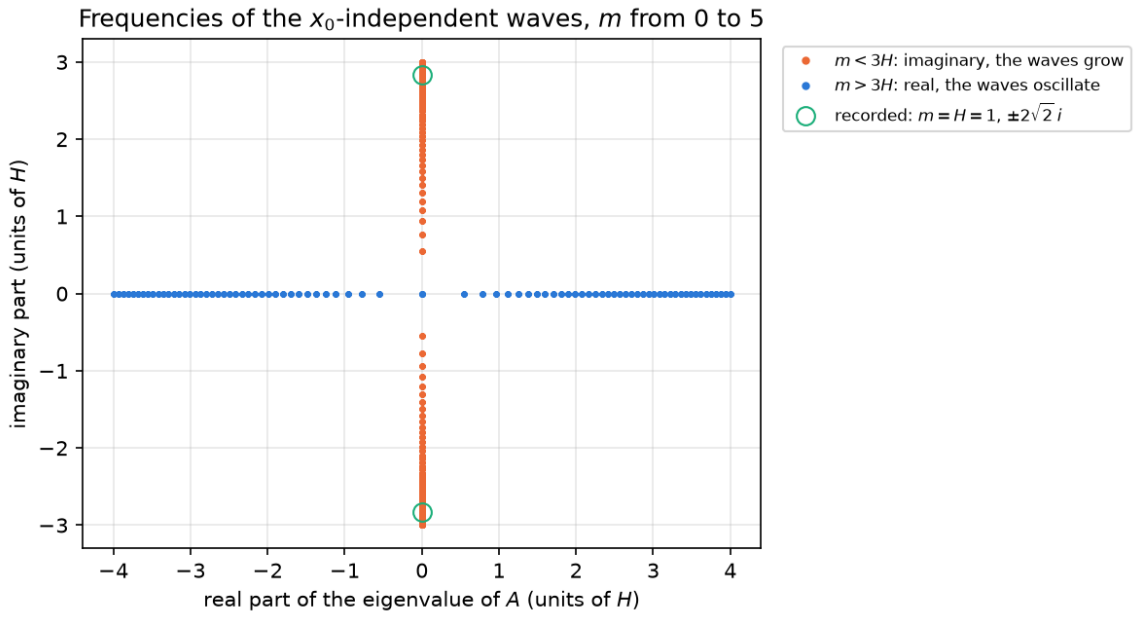

Figure 285: The 16 eigenvalues of the matrix $A = -im\gamma^{(x_4)} + 3iH\gamma^{(x_4)}\gamma^{(x_0)}$, which governs the good-sector waves that do not depend on x_0, in the complex plane (horizontal axis: real part, vertical axis: imaginary part, units of H) for $H = 1$ and the mass m from 0 to 5. For $m < 3H$ (orange) they are $\pm i\sqrt{9H^2 - m^2}$ on the imaginary axis: these finite-norm waves grow. For $m > 3H$ (blue) they are real. Green circles: the recorded values $\pm 2\sqrt{2}\,i$ at $m = H = 1$. (Notebook 10d, figure 3.)

Out [7]:

```
Figure 10d.3 saved as Revision/textbook/figures/10d_3_frequency_paths.png
```

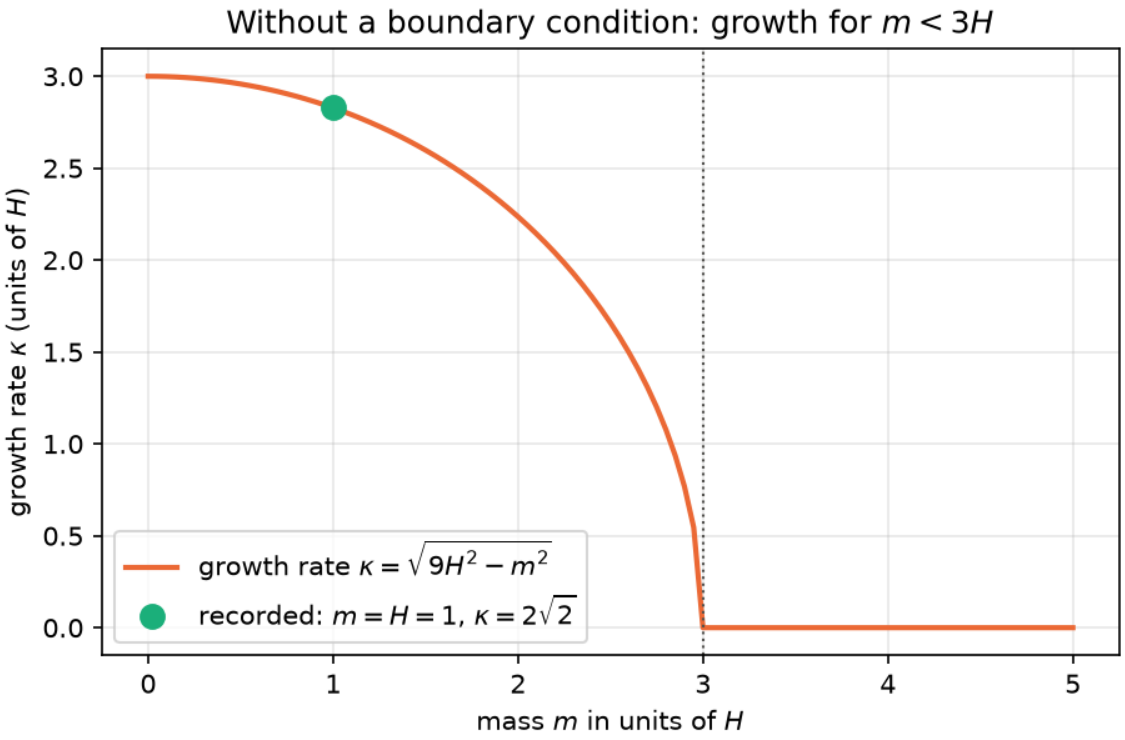

Figure 286: The growth rate $\kappa = \sqrt{9H^2 - m^2}$ (vertical axis, units of H) of the good-sector waves that do not depend on x_0 in the author's metric, against the mass m (horizontal axis, units of H). It is positive for $m < 3H$ (left of the dotted line) and zero for $m \geq 3H$; the green dot is the recorded value $2\sqrt{2}$ at $m = H = 1$. These waves have a finite norm, so without a boundary condition at $z = \pi/2$ the curved good sector contains growing waves. (Notebook 10d, figure 4.)

Out [7]:

```
Figure 10d.4 saved as Revision/textbook/figures/10d_4_growth_rate.png
```

Text cell:

```
## 9. An exact growing solution, and where its Krein charge goes

For a wave of $x_4$ alone the equation $i\partial_4\Psi = A\Psi$ reads
$\partial_4\Psi = M\Psi$ with $M = -iA = -m\gamma^{(x_4)} + 3H\gamma^{(x_4)}
\gamma^{(x_0)}$, and $M^2 = (9H^2 - m^2)I = k^2I$. Its solution is $\Psi(x_4) =
e^{Mx_4}\chi = \big(\cosh(kx_4)I + \frac{\sinh(kx_4)}{k}M\big)\chi$ for any constant
column $\chi$: the case $\alpha = 0$ of the exact family of the Revision theory
record. The next cell checks exactly that it solves the full field equation of
Section 4 (with $\partial_0\Psi = 0$).

Then it follows, for $m = H = 1$ ($k = 2\sqrt2$) and a column $\chi$ with
$B\chi = \chi$, the ordinary squared length $\Psi^\dagger\Psi$ and the Krein charge
$Q(x_4) = \int\cos z\,\Psi^\dagger B\Psi\,dx_0 = \Psi^\dagger B\Psi/(6H)$. By
Section 6 (with $u = v = \Psi$ and $\partial_0\Psi = 0$) the charge changes at the
rate of the flux through the patch end:
$dQ/dx_4 = -i\,[\sin z\,\Psi^\dagger BM_0\Psi]_{z = \pi/2} =
\Psi^\dagger B\gamma^{(x_4)}\gamma^{(x_0)}\Psi$. The cell checks that $Q(x_4) - Q(0)$
equals the flux added up over time (trapezoidal sums on a fine grid).
```

In [8]:

```
x4, k = sp.symbols("x4 k", positive=True)
M_exact = -m * g[4] + 3 * H * g[4] * g[0]
chi = sp.Matrix(sp.symbols("q1:17"))  # any constant column
Psi = (sp.cosh(k * x4) * sp.eye(16) + sp.sinh(k * x4) / k * M_exact) * chi
field_equation = g[4] * Psi.diff(x4) + 3 * H * g[0] * Psi - m * Psi  # d0 Psi = 0
# multiply by k, then replace k^2 by 9 H^2 - m^2 (the definition of k)
residual = field_equation.applyfunc(
    lambda e: sp.expand(sp.expand(k * e).subs(k ** 2, 9 * H ** 2 - m ** 2)))
square_ok = (M_exact * M_exact - (9 * H ** 2 - m ** 2) * sp.eye(16)).applyfunc(
    sp.expand) == zero
check_record(square_ok and residual == sp.zeros(16, 1),
             "exact: Psi = (cosh kx4 + sinh(kx4)/k M) chi solves the field equation",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "exact_solution_family_x4_x0")

M_numbers = -1.0 * gamma[4] + 3.0 * G40  # m = H = 1
k_number = np.sqrt(8.0)  # k = sqrt(9 - 1) = 2 sqrt(2)
plus_columns = (np.eye(16) + B) / 2  # columns with B chi = chi (after scaling)
chi_number = plus_columns[:, 0] / np.linalg.norm(plus_columns[:, 0])
x4_values = np.linspace(0.0, 1.5, 3001)
lengths, charges, fluxes = [], [], []
for t in x4_values:
    Psi_t = (np.cosh(k_number * t) * np.eye(16)
             + np.sinh(k_number * t) / k_number * M_numbers) @ chi_number
```

```
    lengths.append((Psi_t.conj() @ Psi_t).real)
    charges.append((Psi_t.conj() @ B @ Psi_t).real / 6.0)  # 6 H = 6
    fluxes.append((Psi_t.conj() @ B @ G40 @ Psi_t).real)
lengths, charges, fluxes = map(np.array, (lengths, charges, fluxes))
step = x4_values[1] - x4_values[0]
added_flux = np.concatenate([[0.0], np.cumsum((fluxes[1:] + fluxes[:-1]) / 2) * step])
relative = np.max(np.abs(charges - charges[0] - added_flux)) / np.max(np.abs(charges))
report("Krein charge Q at x4 = 0 and at x4 = 1.5", f"{charges[0]:.6f} and "
       f"{charges[-1]:.3f}")
report("largest relative mismatch of Q(x4) - Q(0) and the added flux",
       f"{relative:.1e}")
check(relative < 1e-5, "the Krein charge changes by the flux through z = pi/2")
growth = np.polyfit(x4_values[1500:], np.log(lengths[1500:]), 1)[0]
report("late slope of ln(Psi^dagger Psi)", f"{growth:.4f} (2 k = {2 * k_number:.4f})")
check(abs(growth - 2 * k_number) < 0.05, "the squared length grows like exp(2 k x4)")
```

Out [8]:

```
PASS exact: Psi = (cosh kx4 + sinh(kx4)/k M) chi solves the field equation
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_solution_family_x4_x0
RESULT Krein charge Q at x4 = 0 and at x4 = 1.5 = 0.166667 and 454.007
RESULT largest relative mismatch of Q(x4) - Q(0) and the added flux = 6.7e-07
PASS the Krein charge changes by the flux through z = pi/2
RESULT late slope of ln(Psi^dagger Psi) = 5.6600 (2 k = 5.6569)
PASS the squared length grows like exp(2 k x4)
```

Text cell:

```
The next cell draws the result: the logarithm of the ordinary squared length
(left) and the Krein charge together with the flux added up over time (right).
```

In [9]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
axes[0].plot(x4_values, np.log(lengths), color=BLUE, linewidth=2,
             label="$\\ln(\\Psi^\\dagger\\Psi)$")
axes[0].plot(x4_values, 2 * k_number * x4_values + np.log(lengths[-1])
             - 2 * k_number * x4_values[-1], "--", color=GREY, linewidth=1,
             label="slope $2k = 4\\sqrt{2}$")
axes[0].set_xlabel("time $x_4$ (units of $1/H$)")
axes[0].set_ylabel("$\\ln$ of the squared length $\\Psi^\\dagger\\Psi$")
axes[0].set_title("the wave grows")
axes[0].legend()
axes[1].plot(x4_values, charges, color=ORANGE, linewidth=3,
             label="Krein charge $Q(x_4)$")
axes[1].plot(x4_values, charges[0] + added_flux, "--", color=GREEN, linewidth=2,
             label="$Q(0)$ + flux through $z = \\pi/2$")
axes[1].set_xlabel("time $x_4$ (units of $1/H$)")
axes[1].set_ylabel("Krein charge (units of $1/H$)")
axes[1].set_title("not conserved: it changes by the flux")
```

```
axes[1].legend()
save_figure(fig, "krein_leak",
            "The exact good-sector solution $\\Psi(x_4) = (\\cosh kx_4 + \\sinh(kx_4)"
            "/k\\,M)\\chi$, independent of $x_0$, for $m = H = 1$ ($k = 2\\sqrt{2}$) "
            "and a column $\\chi$ with $B\\chi = \\chi$; horizontal axes: the time "
            "$x_4$ in units of $1/H$. Left: the logarithm of its squared length "
            "grows with the slope $2k$ (grey dashed line). Right: its Krein charge "
            "$Q = \\int\\cos z\\,\\Psi^\\dagger B\\Psi\\,dx_0$ (orange) is not "
            "constant; it equals its starting value plus the flux through the "
            "patch end $z = \\pi/2$ added up over time (green dashed). Without a "
            "boundary condition at $z = \\pi/2$ the Krein charge is not conserved: "
            "it changes by exactly what passes through the patch end.")
```

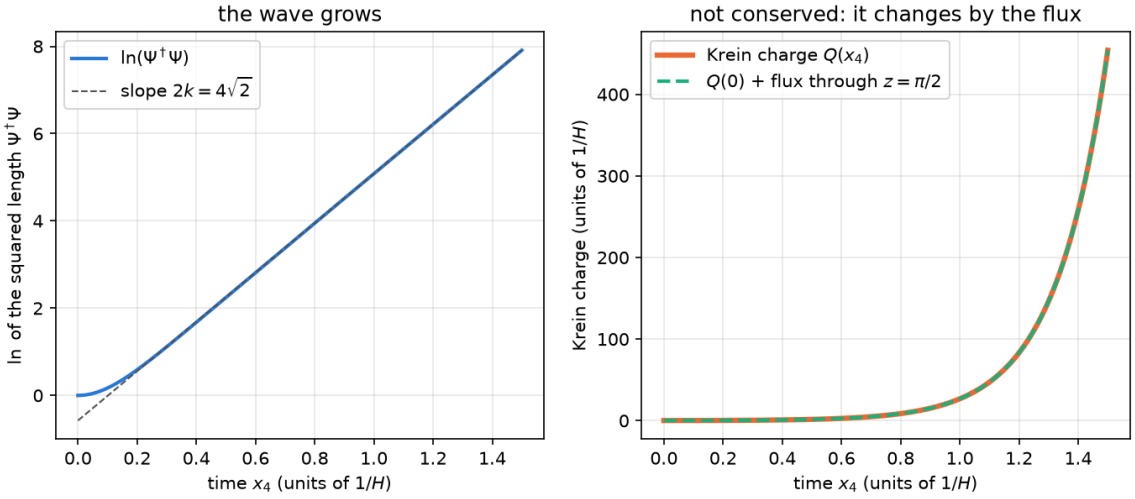

Figure 287: The exact good-sector solution $\Psi(x_4) = (\cosh kx_4 + \sinh(kx_4)/k\,M)\chi$, independent of x_0, for $m = H = 1$ ($k = 2\sqrt{2}$) and a column χ with $B\chi = \chi$; horizontal axes: the time x_4 in units of $1/H$. Left: the logarithm of its squared length grows with the slope $2k$ (grey dashed line). Right: its Krein charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, dx_0$ (orange) is not constant; it equals its starting value plus the flux through the patch end $z = \pi/2$ added up over time (green dashed). Without a boundary condition at $z = \pi/2$ the Krein charge is not conserved: it changes by exactly what passes through the patch end. (Notebook 10d, figure 5.)

Out [9]:

```
Figure 10d.5 saved as Revision/textbook/figures/10d_5_krein_leak.png
```

Text cell:

```
## 10. The last check

The last cell checks that all five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [10]:

```
for name in ("10d_1_volume_and_flux.png", "10d_2_symmetry_defect.png",
             "10d_3_frequency_paths.png", "10d_4_growth_rate.png",
             "10d_5_krein_leak.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [10]:

```
PASS the figure file 10d_1_volume_and_flux.png exists
PASS the figure file 10d_2_symmetry_defect.png exists
PASS the figure file 10d_3_frequency_paths.png exists
PASS the figure file 10d_4_growth_rate.png exists
PASS the figure file 10d_5_krein_leak.png exists
ALL 16 CHECKS PASSED (notebook 10d)
```

Text cell:

```
## 11. What this notebook showed

- PROVED (exactly): in the author's metric the good-sector mode operator along the
  hidden direction, $h = -im\gamma^{(x_4)} + 3iH\gamma^{(x_4)}\gamma^{(x_0)} +
  i\tan z\,\gamma^{(x_4)}\gamma^{(x_0)}\partial_0$, satisfies $\cos z\,[u^\dagger(hv)
  - (hu)^\dagger v] = \partial_0(\sin z\,u^\dagger M_0v)$: it is symmetric for the
  volume $\cos z\,dx_0$ only up to a boundary term, which vanishes at the tip but
  not at the patch end $z = \pi/2$ (flux $2i$ for the recorded column). The Krein
  form is conserved up to the analogous term.
- PROVED: the spin-connection term $3H\gamma^{(x_0)}$ is what makes the defect a pure
  boundary term in these variables and this volume; without it the remainder
  $-6H\cos z\,u^\dagger M_0v$ survives. (In other variables, $\Psi = \sin^{-1/2}z\,
  \chi$ with the volume $dy$, no such term is needed: this role of $3H$ depends on
  the variables, as the Revision record states.)
- PROVED: the good-sector waves that do not depend on $x_0$ have a finite norm and
  the frequencies $\pm\sqrt{m^2 - 9H^2}$; for $m^2 < 9H^2$ they grow (at
  $m = H = 1$ the eigenvalues are $\pm 2\sqrt2\,i$). The exact solution of the
  record with $\alpha = 0$ is such a wave.
- COMPUTED: for that solution the Krein charge is not conserved; it changes by
  exactly the flux through $z = \pi/2$.
- CONSEQUENCE (the scope of the quantisation in the Revision record): Hermiticity of
  the curved good-sector evolution and conservation of the Krein charge need a
  boundary condition at $z = \pi/2$ (for example the ASSUMED brane condition of the
  Kohn-Sham record); the Revision quantisation imposes none. The positive Fock space
  of the previous notebook is built for single good-sector momenta with frozen
  coefficients; a positive Hilbert space for the whole field is not constructed.
```

10.32 Line-by-line walk-through of Notebook 10d

The notebook has 10 code cells, In [1] to In [10]; docstrings are left out of the quotations (they are printed in Section 10.31).

In [1], the set-up cell. Its first 248 lines are comments that repeat the run instructions of Section 10.30. The code is that of In [1] of Notebook 10a (Section 10.10), with the line `NOTEBOOK_ID = "10d" # this notebook: chapter 10, example d`. It prints `Set-up of notebook 10d complete: repository folder found, helpers defined.`

In [2], the matrices of the hidden direction.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output
```

The imports and the helpers `REPORT_CHECKS`, `record_says_pass` and `check_record` of In [2] of Notebook 10a (Section 10.10): a cited report check must still exist with the verdict PASS, and the PASS line and the line naming the record are printed in one piece.

```
BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
B_exact = -sp.I * g[0] * g[1] * g[2] * g[3] * g[4]
```

The colours; the gammas as numpy arrays of decimal numbers (`dtype=float`, because they will be combined with decimal numbers such as the mass 1.0), C and B; and the same gammas and B as exact sympy matrices.

```
m = sp.Symbol("m", real=True)  # the mass
H = sp.Symbol("H", positive=True)  # the author's constant H > 0
x0 = sp.Symbol("x0", real=True)  # the hidden coordinate
z = 6 * H * x0  # z = 6 H x0
M0 = sp.I * g[4] * g[0]  # M_0 = i gamma^(x4) gamma^(x0)
A = -sp.I * m * g[4] + 3 * sp.I * H * g[4] * g[0]  # the part without derivative
A_no_connection = -sp.I * m * g[4]  # the same without the term 3 H
D = sp.tan(z) * M0  # the coefficient of the derivative d/dx0
zero = sp.zeros(16, 16)
```

The letters m (any real number), H (positive) and x_0, and the expression $z = 6Hx_0$. Then the matrices of Section 10.27: $M_0 = i\gamma^{(x_4)}\gamma^{(x_0)}$, $A = -im\gamma^{(x_4)} + 3iH\gamma^{(x_4)}\gamma^{(x_0)}$, the matrix $A_0 = -im\gamma^{(x_4)}$ without the spin-connection term, and $D = \tan z\, M_0$; `zero` is the 16×16 zero matrix.

```
anti_hermitian = M0.H == -M0
identity_D = (sp.cos(z) * D - sp.sin(z) * M0).applyfunc(sp.simplify) == zero
identity_A = (sp.cos(z) * (A - A.H) - sp.diff(sp.sin(z), x0) * M0).applyfunc(
    sp.simplify) == zero
B_commutes = (B_exact * g[4] == g[4] * B_exact) and (B_exact * g[0] == g[0] * B_exact)
```

The four matrix facts: (D1) $M_0^\dagger = -M_0$ (`.H` is the conjugate transpose); (D2) $\cos z\, D - \sin z\, M_0 = 0$, every entry simplified; (D3) $\cos z\,(A - A^\dagger) - (\partial_0 \sin z) M_0 = 0$, where `sp.diff(sp.sin(z), x0)` is the derivative $6H\cos z$; (D4) B commutes with $\gamma^{(x_4)}$ and with $\gamma^{(x_0)}$.

```
say(f"M0 anti-Hermitian: {anti_hermitian}; cos z D = sin z M0: {identity_D}; "
    f"cos z (A - A^dagger) = d0(sin z) M0: {identity_A}; B commutes with "
    f"gamma^(x4) and gamma^(x0): {B_commutes}")
check(anti_hermitian and identity_D and identity_A and B_commutes,
      "exact: the four matrix identities of the hidden direction")
```

The printed line shows `True` four times, and the check prints its PASS line.

In [3], symmetric up to a boundary term, on wave functions.

```
rng = np.random.default_rng(12345)


def test_function():
    def number():
        return int(rng.integers(-3, 4)) + sp.I * int(rng.integers(-3, 4))
    return sp.Matrix([number() + number() * sp.sin(z) + number() * sp.cos(z)
                      for _ in range(16)])
```

A random generator with the fixed seed 12345. `test_function` builds a column of 16 exact functions of x_0, each of the form $a + b\sin z + c\cos z$. The inner function `number` returns a complex whole number whose real and imaginary parts are drawn from -3 to 3 (`rng.integers(-3, 4)` excludes the upper end 4).

```
def apply_h(matrix_A, w):
    return matrix_A * w + D * w.diff(x0)
```

```
def is_zero(expression):
    zz = sp.Symbol("zz")
    rewritten = sp.expand_trig(sp.expand(expression.subs(x0, zz / (6 * H))))
    return sp.simplify(rewritten) == 0
```

`apply_h(matrix_A, w)` applies the operator $h = A + D\partial_0$ (or h_0 when the matrix A_0 is given) to a column w of functions; `w.diff(x0)` differentiates every entry. `is_zero` decides exactly whether an expression vanishes: it writes $x_0 = zz/(6H)$ with a new letter zz, so that z becomes zz and H drops out of the trigonometric functions, multiplies out, expands the trigonometric functions of sums (`sp.expand_trig`), and simplifies.

```
u, v = test_function(), test_function()
hv, hu = apply_h(A, v), apply_h(A, u)
defect = (sp.cos(z) * ((u.H * hv)[0] - (hu.H * v)[0])
          - sp.diff(sp.sin(z) * (u.H * M0 * v)[0], x0))
```

Two test functions u and v, and hv, hu. `u.H * hv` is the row $u^\dagger$ times the column hv, a 1×1 matrix, and `[0]` takes its single entry. `defect` is $\cos z\,[u^\dagger(hv) - (hu)^\dagger v] - \partial_0(\sin z\, u^\dagger M_0 v)$, which must vanish (Section 10.27).

```
hv0, hu0 = apply_h(A_no_connection, v), apply_h(A_no_connection, u)
defect_without = (sp.cos(z) * ((u.H * hv0)[0] - (hu0.H * v)[0])
                  - sp.diff(sp.sin(z) * (u.H * M0 * v)[0], x0))
remainder_ok = is_zero(defect_without + 6 * H * sp.cos(z) * (u.H * M0 * v)[0])
krein_defect = (sp.cos(z) * ((u.H * B_exact * hv)[0] - (hu.H * B_exact * v)[0])
                - sp.diff(sp.sin(z) * (u.H * B_exact * M0 * v)[0], x0))
```

The variable `defect_without` is the same defect for the operator without the spin-connection term. By Section 10.27 it must equal $-6H\cos z\, u^\dagger M_0 v$, so `remainder_ok` tests that it plus $6H\cos z\, u^\dagger M_0 v$ vanishes. The variable `krein_defect` is the Krein version, with $u^\dagger B$ in place of $u^\dagger$.

```
check_record(is_zero(defect),
             "exact: cos z (u^dagger h v - (h u)^dagger v) = d0(sin z u^dagger M0 v)",
             record="Revision/theory/reports/python-scope.json, check "
                    "good_sector_hermiticity_up_to_the_brane_flux")
check_record(remainder_ok,
             "exact: without the term 3 H the remainder -6 H cos z u^dagger M0 v stays",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "good_sector_hermiticity_curved")
check_record(is_zero(krein_defect),
             "exact: the Krein form is conserved up to d0(sin z u^dagger B M0 v)",
             record="Revision/theory/reports/wolfram-field-theory.json, check "
                    "Krein_form_conserved_curved")
```

Three exact checks, reproducing the record checks `good_sector_hermiticity_up_to_the_brane_flux`, `good_sector_hermiticity_curved` and `Krein_form_conserved_curved`.

In [4], the flux at the two ends.

```
G40 = gamma[4] @ gamma[0]  # gamma^(x4) gamma^(x0), real
symmetric_square = np.array_equal(G40, G40.T) and np.array_equal(G40 @ G40,
                                                                 np.eye(16))
e_first = np.eye(16)[:, 0]
u_flux = e_first + G40 @ e_first  # an eigenvector of G40 with eigenvalue +1
M0_numbers = 1j * G40
flux_value = u_flux.conj() @ M0_numbers @ u_flux
```

`G48` is $\gamma^{(x_4)}\gamma^{(x_0)}$; `symmetric_square` tests that it is symmetric and squares to I_{16} (Section 10.27). `e_first` is the first unit column e_1 (the first column of the identity matrix), and `u_flux` $= e_1 + \gamma^{(x_4)}\gamma^{(x_0)}e_1$ is an eigenvector with eigenvalue $+1$: applying $\gamma^{(x_4)}\gamma^{(x_0)}$ gives $\gamma^{(x_4)}\gamma^{(x_0)}e_1 + e_1$, the same column, because the matrix squares to I_{16}. `flux_value` is $u^\dagger M_0 u$.

```
report("u^dagger u for the column u = e_1 + gamma4 gamma0 e_1", f"{u_flux @ u_flux:.1f}")
report("flux u^dagger M0 u at z = pi/2",
       f"{flux_value.imag:.1f} i (real part {abs(flux_value.real):.1f})")
check_record(symmetric_square and np.allclose(G40 @ u_flux, u_flux)
             and abs(flux_value - 2j) < 1e-14,
             "the flux at z = pi/2 is u^dagger M0 u = 2 i for gamma4 gamma0 u = u",
             record="Revision/theory/reports/python-scope.json, check "
                    "good_sector_hermiticity_up_to_the_brane_flux")
```

The printed lines show $u^\dagger u = 2.0$ and the flux $2.0\,i$ with real part 0.0 (the size of the real part is printed, so that no minus sign of a rounding zero appears). The check reproduces the record's example.

```
z_values = np.linspace(0.0, np.pi / 2, 400)
fig, ax = plt.subplots()
ax.plot(z_values, np.cos(z_values), color=BLUE, linewidth=2,
        label="volume factor $\\sqrt{|g|} = \\cos z$")
ax.plot(z_values, np.sin(z_values), color=ORANGE, linewidth=2,
        label="flux factor $\\sin z$ of the bracket")
ax.plot([0.0], [0.0], "o", color=ORANGE, markersize=8)
ax.plot([np.pi / 2], [1.0], "o", color=ORANGE, markersize=8)
```

400 values of z from 0 to $\pi/2$; the blue curve is the volume factor $\cos z$, the orange curve the factor $\sin z$ of the boundary bracket, with dots at its two ends.

```
ax.annotate("tip: the flux vanishes", (0.0, 0.0), xytext=(0.15, 0.25),
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.annotate("patch end: the flux remains", (np.pi / 2, 1.0), xytext=(0.55, 1.08),
            arrowprops={"arrowstyle": "->", "color": GREY})
ax.set_xlabel("$z = 6 H x_0$ (from the tip $z = 0$ to the patch end $z = \\pi/2$)")
ax.set_ylabel("value")
ax.set_ylim(-0.05, 1.25)
ax.set_title("The hidden direction: volume and boundary flux")
ax.legend(loc="center left")
save_figure(fig, "volume_and_flux",
...)
```

Two arrows explain the two dots. Figure 1 of Notebook 10d shows the volume falling from 1 to 0 and the bracket factor rising from 0 to 1: the volume vanishes at the patch end, but the boundary bracket does not, so a flux can pass through $z = \pi/2$.

In [5], the role of the spin-connection term in a picture.

```
numbers = {m: 1, H: 1}  # m = H = 1, so z = 6 x0
hu_full, hu_bare = apply_h(A, u), apply_h(A_no_connection, u)
left = sp.cos(z) * ((u.H * hu_full)[0] - (hu_full.H * u)[0])
right = sp.diff(sp.sin(z) * (u.H * M0 * u)[0], x0)
left_bare = sp.cos(z) * ((u.H * hu_bare)[0] - (hu_bare.H * u)[0])
```

For $u = v$ and $m = H = 1$ the cell forms the three exact expressions: the defect with the term $3H$ (`left`), the derivative $\partial_0(\sin z\, u^\dagger M_0 u)$ (`right`) and the defect without the term (`left_bare`).

```
x0_values = np.linspace(0.002, np.pi / 12 - 0.002, 300)  # inside 0 < z < pi/2
curves = [np.imag(sp.lambdify(x0, e.subs(numbers), "numpy")(x0_values))
          for e in (left, right, left_bare)]
difference = np.max(np.abs(curves[0] - curves[1]))
report("largest |left - right| on the grid", f"{difference:.1e}")
check(difference < 1e-9, "numerical: the two sides agree along the hidden direction")
```

For $H = 1$ the patch is $0 < x_0 < \pi/12$; the 300 points stay a little inside it. `e.subs(numbers)` puts $m = H = 1$ into an expression, and `sp.lambdify(x0, ..., "numpy")` turns it into a fast numpy function of x_0, which is evaluated at all points at once. For $u = v$ each expression is a number minus its complex conjugate, purely imaginary, so `np.imag` keeps the imaginary parts. The largest difference between the left and the right side is printed as `5.7e-13`, rounding only.

```
fig, ax = plt.subplots()
ax.plot(6 * x0_values, curves[0], color=BLUE, linewidth=3,
        label="left side, with the term $3H$")
ax.plot(6 * x0_values, curves[1], "--", color=GREEN, linewidth=2,
        label="right side $\\partial_0(\\sin z\\,u^\\dagger M_0 u)$")
ax.plot(6 * x0_values, curves[2], color=ORANGE, linewidth=2,
        label="left side WITHOUT the term $3H$")
ax.set_xlabel("$z = 6 H x_0$ ($m = H = 1$)")
ax.set_ylabel("imaginary part")
ax.set_title("The spin-connection term makes the defect a pure boundary term")
ax.legend(fontsize=8)
save_figure(fig, "symmetry_defect",
...)
```

The three curves against $z = 6x_0$. Figure 2 of Notebook 10d shows the green dashed curve exactly on the thick blue one, and the orange curve away from both: with the spin-connection term the defect is a derivative, whose integral is the flux at the ends; without it the remainder $-6H\cos z\, u^\dagger M_0 u$ of Section 10.27 separates the curves.

In [6], the waves that do not depend on x_0.

```
norm_integral = sp.integrate(sp.cos(6 * H * x0), (x0, 0, sp.pi / (12 * H)))
say(f"integral of cos(6 H x0) over the patch: {norm_integral}")
A_square_ok = (A * A - (m ** 2 - 9 * H ** 2) * sp.eye(16)).applyfunc(
    sp.expand) == zero
A_not_hermitian = (A - A.H).applyfunc(sp.expand) != zero
G40_numbers = gamma[4] @ gamma[0]  # gamma^(x4) gamma^(x0) as numbers
```

`sp.integrate(f, (x0, a, b))` computes the definite integral exactly; the printed value is `1/(6*H)`, the norm of Section 10.28. The two truth values test $AA = (m^2 - 9H^2)I_{16}$ and that A is not Hermitian ($A - A^\dagger$ is not the zero matrix). `G48_numbers` is $\gamma^{(x_4)}\gamma^{(x_0)}$ again, for the numerical part.

```
def A_numbers_of(mass, H_value=1.0):
    return -1j * mass * gamma[4] + 3j * H_value * G40_numbers


A_numbers = A_numbers_of(1.0)
eigenvalues = np.linalg.eigvals(A_numbers)
upper = int(np.sum(np.abs(eigenvalues - 2j * np.sqrt(2)) < 1e-9))
lower = int(np.sum(np.abs(eigenvalues + 2j * np.sqrt(2)) < 1e-9))
report("eigenvalues 2 sqrt(2) i and -2 sqrt(2) i at m = H = 1, how many",
       f"{upper} and {lower}")
```

`A_numbers_of(mass)` is the matrix A as numbers for a given mass ($H = 1$ unless another value is given). At $m = H = 1$ the 16 eigenvalues are counted: how many lie within 10^{-9} of $2\sqrt{2}\,i$ and how many of $-2\sqrt{2}\,i$. The printed counts are 8 and 8.

```
check_record(norm_integral == 1 / (6 * H) and A_square_ok and A_not_hermitian
             and (upper, lower) == (8, 8),
             "x0-independent waves: A^2 = (m^2 - 9 H^2) I, at m = H = 1: +-2 sqrt(2) i",
             record="Revision/theory/reports/python-scope.json, check "
                    "good_sector_x0_independent_modes_without_boundary_condition")
```

The check reproduces the record check `good_sector_x0_independent_modes_without_boundary_condition`.

In [7], the frequencies against the mass.

```
mass_values = np.linspace(0.0, 5.0, 101)
paths = []
for mass in mass_values:
    paths.append(np.linalg.eigvals(A_numbers_of(mass)))
paths = np.array(paths)
worst = np.max(np.abs(np.sort(np.abs(paths), axis=1)
                      - np.sqrt(np.abs(mass_values ** 2 - 9))[:, None]))
report("largest deviation of |eigenvalue| from sqrt|m^2 - 9|", f"{worst:.1e}")
check(worst < 1e-6, "the eigenvalues of A follow +-sqrt(m^2 - 9 H^2)")
```

For 101 masses from 0 to 5 (step 0.05) the 16 eigenvalues of A are stored as one row of the table `paths`. Every eigenvalue must have the size $\sqrt{|m^2 - 9|}$ ($H = 1$): `np.abs(paths)` are the sizes,

sorted in each row, and `[:, None]` turns the 101 predicted values into a column, so that each is subtracted from its own row. The largest deviation is printed as `4.6e-08`: at $m = 3$ exactly the matrix A has $AA = 0$ and cannot be diagonalised, and there the numerical eigenvalues are accurate only to about 10^{-8}; the tolerance 10^{-6} allows for it.

```
fig, ax = plt.subplots(figsize=(6.0, 5.0))
growing = mass_values < 3.0
for column in range(16):
    ax.plot(paths[growing, column].real, paths[growing, column].imag, ".",
            color=ORANGE, markersize=4)
    ax.plot(paths[~growing, column].real, paths[~growing, column].imag, ".",
            color=BLUE, markersize=4)
ax.plot([], [], ".", color=ORANGE, label="$m < 3H$: imaginary, the waves grow")
ax.plot([], [], ".", color=BLUE, label="$m > 3H$: real, the waves oscillate")
ax.plot([0, 0], [2 * np.sqrt(2), -2 * np.sqrt(2)], "o", color=GREEN, markersize=9,
        fillstyle="none", label="recorded: $m = H = 1$, $\\pm 2\\sqrt{2}\\,i$")
```

`growing` is a list of truth values, true for the masses below 3; `paths[growing, column]` picks the eigenvalue number `column` at those masses, and `~growing` is the opposite list. Each eigenvalue is drawn as a dot in the complex plane (real part horizontal, imaginary part vertical), orange for $m < 3$ and blue for $m \geq 3$; the empty plots only create legend entries; green circles mark the recorded values $\pm 2\sqrt{2}\,i$.

```
ax.set_xlabel("real part of the eigenvalue of $A$ (units of $H$)")
ax.set_ylabel("imaginary part (units of $H$)")
ax.set_aspect("equal")
ax.set_title("Frequencies of the $x_0$-independent waves, $m$ from 0 to 5")
ax.legend(loc="upper left", bbox_to_anchor=(1.02, 1.0), fontsize=8)  # outside
save_figure(fig, "frequency_paths",
...)
```

Equal scales on both axes; `bbox_to_anchor` places the legend outside the drawing, to its right. Figure 3 of Notebook 10d shows the eigenvalues coming down the imaginary axis from $\pm 3i$ (at $m = 0$) to 0 (at $m = 3$) and then moving out along the real axis: growth for $m < 3H$, oscillation for $m > 3H$.

```
rate = np.sqrt(np.maximum(9.0 - mass_values ** 2, 0.0))
fig, ax = plt.subplots()
ax.plot(mass_values, rate, color=ORANGE, linewidth=2,
        label="growth rate $\\kappa = \\sqrt{9H^2 - m^2}$")
ax.plot([1.0], [2 * np.sqrt(2)], "o", color=GREEN, markersize=9,
        label="recorded: $m = H = 1$, $\\kappa = 2\\sqrt{2}$")
ax.axvline(3.0, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("mass $m$ in units of $H$")
ax.set_ylabel("growth rate $\\kappa$ (units of $H$)")
ax.set_title("Without a boundary condition: growth for $m < 3H$")
ax.legend()
save_figure(fig, "growth_rate",
...)
```

The growth rate $\kappa = \sqrt{9 - m^2}$ for $m < 3$ and 0 beyond (`np.maximum(..., 0.0)` replaces the negative values under the root by 0), with the recorded point $\kappa = 2\sqrt{2} \approx 2.83$ at $m = 1$. Figure 4 of Notebook 10d shows the curve falling from 3 at $m = 0$ to 0 at $m = 3$ (a quarter of the circle $\kappa^2 + m^2 = 9$, drawn with unequal scales on the two axes): without a boundary condition at $z = \pi/2$, every mass below $3H$ gives growing waves of finite norm.

In [8], the exact growing solution and its Krein charge.

```
x4, k = sp.symbols("x4 k", positive=True)
M_exact = -m * g[4] + 3 * H * g[4] * g[0]
chi = sp.Matrix(sp.symbols("q1:17"))  # any constant column
Psi = (sp.cosh(k * x4) * sp.eye(16) + sp.sinh(k * x4) / k * M_exact) * chi
field_equation = g[4] * Psi.diff(x4) + 3 * H * g[0] * Psi - m * Psi  # d0 Psi = 0
```

`sp.symbols("x4 k", positive=True)` makes two positive letters at once, and `sp.symbols("q1:17")` the 16 letters $q_1, \ldots, q_{16}$, the entries of an arbitrary constant column χ. `M_exact` is $M = -m\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}$ and `Psi` the solution of Section 10.28. `field_equation` is the left side minus the right side of the field equation of Section 10.27 for a wave with $\partial_0 \Psi = 0$, which must vanish.

```
# multiply by k, then replace k^2 by 9 H^2 - m^2 (the definition of k)
residual = field_equation.applyfunc(
    lambda e: sp.expand(sp.expand(k * e).subs(k ** 2, 9 * H ** 2 - m ** 2)))
square_ok = (M_exact * M_exact - (9 * H ** 2 - m ** 2) * sp.eye(16)).applyfunc(
    sp.expand) == zero
check_record(square_ok and residual == sp.zeros(16, 1),
             "exact: Psi = (cosh kx4 + sinh(kx4)/k M) chi solves the field equation",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "exact_solution_family_x4_x0")
```

The expression contains $1/k$ (from $\sinh(kx_4)/k$); as the comment says, every entry is multiplied by k to clear it, multiplied out, and every k^2 is replaced by $9H^2 - m^2$, the definition of k; then every entry must be 0. `square_ok` checks $MM = (9H^2 - m^2)I_{16}$. The check reproduces the record check `exact_solution_family_x4_x0`.

```
M_numbers = -1.0 * gamma[4] + 3.0 * G40  # m = H = 1
k_number = np.sqrt(8.0)  # k = sqrt(9 - 1) = 2 sqrt(2)
plus_columns = (np.eye(16) + B) / 2  # columns with B chi = chi (after scaling)
chi_number = plus_columns[:, 0] / np.linalg.norm(plus_columns[:, 0])
x4_values = np.linspace(0.0, 1.5, 3001)
lengths, charges, fluxes = [], [], []
```

For $m = H = 1$: M as numbers and $k = \sqrt{9 - 1} = 2\sqrt{2}$. The columns of the projector $\frac{1}{2}(I_{16} + B)$ lie in the eigenspace $B\chi = \chi$; its first column, divided by its length, is the starting column χ with $\chi^\dagger \chi = 1$. 3001 times from 0 to 1.5 (step 0.0005).

```
for t in x4_values:
    Psi_t = (np.cosh(k_number * t) * np.eye(16)
```

```
            + np.sinh(k_number * t) / k_number * M_numbers) @ chi_number
    lengths.append((Psi_t.conj() @ Psi_t).real)
    charges.append((Psi_t.conj() @ B @ Psi_t).real / 6.0)  # 6 H = 6
    fluxes.append((Psi_t.conj() @ B @ G40 @ Psi_t).real)
lengths, charges, fluxes = map(np.array, (lengths, charges, fluxes))
```

At each time the loop computes $\Psi(x_4)$, its ordinary squared length $\Psi^\dagger\Psi$, its Krein charge $Q = \Psi^\dagger B\Psi/(6H)$ and the flux $\Psi^\dagger B\gamma^{(x_4)}\gamma^{(x_0)}\Psi$, the rate dQ/dx_4 of Section 10.28. `map(np.array, ...)` turns each of the three lists into an array.

```
step = x4_values[1] - x4_values[0]
added_flux = np.concatenate([[0.0], np.cumsum((fluxes[1:] + fluxes[:-1]) / 2) * step])
relative = np.max(np.abs(charges - charges[0] - added_flux)) / np.max(np.abs(charges))
report("Krein charge Q at x4 = 0 and at x4 = 1.5", f"{charges[0]:.6f} and "
       f"{charges[-1]:.3f}")
report("largest relative mismatch of Q(x4) - Q(0) and the added flux",
       f"{relative:.1e}")
check(relative < 1e-5, "the Krein charge changes by the flux through z = pi/2")
```

The flux is added up over time by the **trapezoidal rule**: on each step the area under the curve is the step times the mean of the two end values, `(fluxes[1:] + fluxes[:-1]) / 2` (`fluxes[1:]` drops the first value and `fluxes[:-1]` the last); `np.cumsum` forms the running sums, and a 0 is put in front for the time 0. The mismatch between $Q(x_4) - Q(0)$ and the added flux, divided by the largest charge, is printed as `6.7e-07`; the RESULT lines show $Q(0) = 0.166667 = 1/6$ and $Q(1.5) = 454.007$. The check confirms that the charge changes by the flux through the patch end.

```
growth = np.polyfit(x4_values[1500:], np.log(lengths[1500:]), 1)[0]
report("late slope of ln(Psi^dagger Psi)", f"{growth:.4f} (2 k = {2 * k_number:.4f})")
check(abs(growth - 2 * k_number) < 0.05, "the squared length grows like exp(2 k x4)")
```

`np.polyfit(x, y, 1)` fits the straight line $y = ax + b$ through the points by least squares and returns (a, b); `[0]` takes the slope. The fit uses the second half of the times (from index 1500, $x_4 = 0.75$, on) and the logarithm of the squared length $\Psi^\dagger\Psi$ (the list `lengths` holds these squared lengths). The printed slope is 5.6600, close to $2k = 5.6569$ (the squared length contains cosh and sinh, not a pure exponential, so the slope only approaches $2k$), and the check allows a difference of 0.05. The cell prints three PASS lines.

In [9], the growing wave and its charge in a picture.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
axes[0].plot(x4_values, np.log(lengths), color=BLUE, linewidth=2,
             label="$\\ln(\\Psi^\\dagger\\Psi)$")
axes[0].plot(x4_values, 2 * k_number * x4_values + np.log(lengths[-1])
             - 2 * k_number * x4_values[-1], "--", color=GREY, linewidth=1,
             label="slope $2k = 4\\sqrt{2}$")
axes[0].set_xlabel("time $x_4$ (units of $1/H$)")
axes[0].set_ylabel("$\\ln$ of the squared length $\\Psi^\\dagger\\Psi$")
axes[0].set_title("the wave grows")
axes[0].legend()
```

The left panel draws $\ln(\Psi^\dagger\Psi)$ against x_4 and a grey dashed straight line of slope $2k = 4\sqrt{2}$ through the last point.

```
axes[1].plot(x4_values, charges, color=ORANGE, linewidth=3,
             label="Krein charge $Q(x_4)$")
axes[1].plot(x4_values, charges[0] + added_flux, "--", color=GREEN, linewidth=2,
             label="$Q(0)$ + flux through $z = \\pi/2$")
axes[1].set_xlabel("time $x_4$ (units of $1/H$)")
axes[1].set_ylabel("Krein charge (units of $1/H$)")
axes[1].set_title("not conserved: it changes by the flux")
axes[1].legend()
save_figure(fig, "krein_leak",
...)
```

The right panel draws the Krein charge and its starting value plus the added flux. Figure 5 of Notebook 10d shows the logarithm of the squared length rising along a straight line, and the charge rising from 1/6 to several hundred, with the green dashed curve exactly on the orange one: without a boundary condition at $z = \pi/2$ the Krein charge of the curved good sector is not conserved; it changes by exactly what passes through the patch end.

In [10], the last check.

```
for name in ("10d_1_volume_and_flux.png", "10d_2_symmetry_defect.png",
             "10d_3_frequency_paths.png", "10d_4_growth_rate.png",
             "10d_5_krein_leak.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The five figure files exist, and the last line prints ALL 16 CHECKS PASSED (notebook 10d): the five figure checks and the 11 checks of the cells before (1 in In [2], 3 in In [3], 1 each in In [4] to In [7], and 3 in In [8]).

10.33 No invariant charge density is positive

The charge density $\Psi^\dagger B\Psi$ takes both signs. Could a cleverer choice of the charge density avoid this? A charge density must not depend on how the frame of the eight directions is turned, so it must be built from a form that is unchanged by the spin transformations. This section shows that every such density is indefinite.

Spin transformations and invariant forms. The 28 **generators** $S^{ab} = \frac{1}{4}[\gamma^{(x_a)}, \gamma^{(x_b)}] = \frac{1}{2}\gamma^{(x_a)}\gamma^{(x_b)}$ ($a < b$; the second form uses $\gamma^{(x_b)}\gamma^{(x_a)} = -\gamma^{(x_a)}\gamma^{(x_b)}$) generate the spin transformations $R = \exp(\theta S^{ab})$, which turn the frame, and their products form Spin(4,4) (Chapter 5). A **bilinear form** $\Psi^T G\Phi$ with a 16×16 matrix G is **invariant** if it does not change when both columns are transformed by the same R: $R^T GR = G$. For $R = I_{16} + \theta S$ with a small number θ,

$$R^T GR = G + \theta\,(S^T G + GS) + \theta^2 S^T GS,$$

by multiplying out; the form is unchanged to first order in θ exactly when

$$S^T G + GS = 0 \quad \text{for all 28 generators.}$$

The gammas are real, so $S^\dagger = S^T$, and the same condition makes the Hermitian form $\Psi^\dagger G \Psi$ invariant.

All invariant forms. Chapter 5 showed two facts. First, $C\gamma^{(x_a)}C = -(\gamma^{(x_a)})^T$ (the charge-conjugation property of $C = \mathcal{C}_+$; `wolfram-algebra.json`, check `C_gamma_C_inverse`). Hence

$$(S^{ab})^T = \tfrac{1}{2}(\gamma^{(x_b)})^T(\gamma^{(x_a)})^T = \tfrac{1}{2}(C\gamma^{(x_b)}C)(C\gamma^{(x_a)}C) = \tfrac{1}{2}C\gamma^{(x_b)}\gamma^{(x_a)}C = -CS^{ab}C,$$

by the transpose of a product, the property of C (the two minus signs cancel), $CC = I_{16}$, and $\gamma^{(x_b)}\gamma^{(x_a)} = -\gamma^{(x_a)}\gamma^{(x_b)}$. Insert it into the condition and multiply from the left by C:

$$S^T G + GS = 0 \iff -CSCG + GS = 0 \iff -SCG + CGS = 0 \iff (CG)S = S(CG).$$

So G is invariant exactly when CG commutes with all 28 generators. Second, the matrices that commute with all generators of Spin(4,4) form a two-dimensional family, spanned by the chiral projectors $P_- = \frac{1}{2}(I_{16} - \Gamma)$ and $P_+ = \frac{1}{2}(I_{16} + \Gamma)$ (the commutant of Spin(4,4) has dimension 2; `python-algebra.json`, check `spin_commutant_dimension_2`; `wolfram-algebra.json`, check `Spin44_commutant_dim_2_chiral_projectors`). Hence $CG = r_- P_- + r_+ P_+$, and, multiplying by C,

$$G = r_-\, CP_- + r_+\, CP_+.$$

Every invariant form is a combination of CP_- and CP_+; the choice $r_- = r_+ = 1$ gives $G = C$ and the Spin(4,4) scalar $\bar{\Psi}\Psi = \Psi^\dagger C \Psi$ (`wolfram-algebra.json`, check `S_preserves_C`).

Every invariant charge density is indefinite. A charge density is the time component of a current, so it is built from an invariant form and the time gamma: its matrix is the Hermitian part $X = \frac{1}{2}(Y + Y^\dagger)$ of $Y = cG\gamma^{(x_4)}$ with a complex number c (the Hermitian part makes the density $\Psi^\dagger X \Psi$ real). Now C is a product of four gammas, so it commutes with Γ (moving Γ through four gammas costs $(-1)^4 = +1$), and $P_\mp$ commute with Γ; so G commutes with Γ, while $\gamma^{(x_4)}$ anticommutes with it. With $\Gamma\Gamma = I_{16}$:

$$\Gamma Y \Gamma = cG\,\Gamma\gamma^{(x_4)}\Gamma = -cG\gamma^{(x_4)} = -Y, \qquad \Gamma X \Gamma = \tfrac{1}{2}\big(\Gamma Y\Gamma + (\Gamma Y\Gamma)^\dagger\big) = -X,$$

where the second identity uses that Γ is Hermitian, so $\Gamma Y^\dagger \Gamma = (\Gamma Y \Gamma)^\dagger$. Multiply $\Gamma X \Gamma = -X$ from the left by Γ: $X\Gamma = -\Gamma X$. If $Xv = \lambda v$, then

$$X(\Gamma v) = -\Gamma X v = -\lambda\, \Gamma v :$$

Γ carries every eigenvector of X with eigenvalue λ to one with eigenvalue $-\lambda$, and back. So the eigenvalues of X come in pairs $\lambda, -\lambda$ of equal multiplicity, and the density $\Psi^\dagger X \Psi$ takes both signs

unless $X = 0$. The canonical choice $c = -i$, $G = C$ gives $Y = -iC\gamma^{(x_4)} = B$, which is already Hermitian, so $X = B$ with eigenvalues ± 1, eight each. **In signature (4,4) no invariant charge density is positive**: the indefinite charge of dirac16complex is not an accident of the choice B. (In ordinary 3+1 dimensions the time gamma also anticommutes with the chirality γ_5, so that fact alone is not the difference. The difference is the invariant form. In 3+1 dimensions it is $G = \gamma^0$, a single gamma, which anticommutes with γ_5; so the density matrix $G\gamma^0 = \gamma^0\gamma^0 = I_4$ commutes with γ_5, the argument above gives no pairing, and the density $\psi^\dagger\psi$ is positive. In (4,4) the invariant forms $r_- CP_- + r_+ CP_+$ are built from C, a product of four gammas, and commute with Γ, while $\gamma^{(x_4)}$ anticommutes with it; this is what produces the pairing $\lambda \to -\lambda$.)

10.34 The symmetries that keep the canonical rule

A transformation $\Psi \to R\Psi$ keeps the canonical anticommutator exactly when $RBR^\dagger = B$ (Section 10.15). This is equivalent to $R^\dagger BR = B$: multiplying $RBR^\dagger = B$ from the left by R^{-1} and from the right by $(R^\dagger)^{-1}$ gives $B = R^{-1}B(R^\dagger)^{-1}$; the inverse of both sides, with $B^{-1} = B$, is $B = R^\dagger BR$. We call such an R **Krein-unitary**. An R with $R^\dagger R = I_{16}$ keeps the ordinary length and is **unitary**.

The conditions on a generator. For $R = I_{16} + \theta S$ to first order, $R^\dagger R = I_{16} + \theta(S^\dagger + S)$ and $R^\dagger BR = B + \theta(S^\dagger B + BS)$, so the conditions are

$$\text{unitary: } S^\dagger = -S, \qquad \text{Krein-unitary: } S^\dagger B + BS = 0.$$

The first order suffices for all θ: if $S^\dagger B = -BS$, then $(S^\dagger)^n B = B(-S)^n$ by moving B through one factor at a time, so $\exp(\theta S^\dagger)B = B\exp(-\theta S)$ (term by term in the power series) and $R^\dagger BR = B\exp(-\theta S)\exp(\theta S) = B$.

Which generators are anti-Hermitian. For $a \neq b$, with $(\gamma^{(x_a)})^\dagger = \eta_{aa}\gamma^{(x_a)}$ (Section 10.15),

$$(\gamma^{(x_a)}\gamma^{(x_b)})^\dagger = (\gamma^{(x_b)})^\dagger(\gamma^{(x_a)})^\dagger = \eta_{aa}\eta_{bb}\,\gamma^{(x_b)}\gamma^{(x_a)} = -\eta_{aa}\eta_{bb}\,\gamma^{(x_a)}\gamma^{(x_b)}.$$

So S^{ab} is anti-Hermitian when $\eta_{aa}\eta_{bb} = +1$ (both directions space-like or both time-like: a **rotation**), and Hermitian when $\eta_{aa}\eta_{bb} = -1$ (one space-like and one time-like: a **boost**). The space-like directions are x_0, x_1, x_2, x_3 and the time-like ones x_4, x_5, x_6, x_7, so there are $6 + 6 = 12$ rotations (pairs within each group of four, $\binom{4}{2} = 6$) and $4 \times 4 = 16$ boosts.

Which generators commute with B. $\gamma^{(x_a)}B = s_a B\gamma^{(x_a)}$ with $s_a = +1$ for $a = 1, 2, 3, 4, 8$ and -1 for $a = 5, 6, 7$ (Section 10.2), so $\gamma^{(x_a)}\gamma^{(x_b)}B = s_a s_b\, B\gamma^{(x_a)}\gamma^{(x_b)}$: S^{ab} commutes with B when $s_a s_b = +1$, that is both in $\{1, 2, 3, 4, 8\}$ ($\binom{5}{2} = 10$ pairs) or both in $\{5, 6, 7\}$ (3 pairs), 13 in all, and anticommutes otherwise ($5 \times 3 = 15$ pairs).

The Krein condition. For an anti-Hermitian S the condition $S^\dagger B + BS = 0$ reads $BS - SB = 0$: S must commute with B. For a Hermitian S it reads $SB + BS = 0$: S must anticommute with B. Counting:

- rotations that commute with B: the 6 among x_0, x_1, x_2, x_3 and the 3 among x_5, x_6, x_7; the 3 rotations $S^{(x_4 x_t)}$, $t = 5, 6, 7$, anticommute and fail;
- boosts that anticommute with B: one direction among x_0, x_1, x_2, x_3 and one among x_5, x_6, x_7, $4 \times 3 = 12$; the 4 boosts $S^{(x_a x_4)}$, $a = 1, 2, 3, 8$, commute and fail.

So exactly $9+12=21$ generators are Krein-unitary: precisely the $\binom{7}{2}=21$ pairs that do not contain x_4. They generate **Spin(4,3)**, the spin group of the seven slice directions (four space-like, three time-like). For the 7 generators with x_4 written first, $S^{(x_4x_b)}=\frac{1}{2}\gamma^{(x_4)}\gamma^{(x_b)}$, the condition fails by $S^\dagger B+BS=2BS=-iC\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_b)}=iC\gamma^{(x_b)}$ (in both cases the left side equals $2BS$, because either $S^\dagger=S$ and $SB=BS$, or $S^\dagger=-S$ and $SB=-BS$; then $\gamma^{(x_4)}\gamma^{(x_4)}=-I_{16}$). Of the 21, the 9 rotations are also unitary; the 12 boosts mixing a space direction with an extra time keep the canonical rule but change the ordinary squared length (and so the length). The canonical structure singles out the time x_4, as $\psi^\dagger\psi$ does in ordinary Dirac theory: a transformation that turns x_4 into another direction also changes the slices $x_4=\text{const}$ on which the field is quantised.

Finite rotations and boosts. For $a\neq b$, $(\gamma^{(x_a)}\gamma^{(x_b)})^2=-\gamma^{(x_a)}\gamma^{(x_a)}\gamma^{(x_b)}\gamma^{(x_b)}=-\eta_{aa}\eta_{bb}I_{16}$, so $(S^{ab})^2=-\frac{1}{4}\eta_{aa}\eta_{bb}I_{16}$. As in Section 10.4 the exponential series collapses:

$$\exp(\theta S^{ab})=\cos\tfrac{\theta}{2}\,I_{16}+2\sin\tfrac{\theta}{2}\,S^{ab}\ \text{(rotation)},\qquad \exp(\theta S^{ab})=\cosh\tfrac{\theta}{2}\,I_{16}+2\sinh\tfrac{\theta}{2}\,S^{ab}\ \text{(boost)}.$$

(For a rotation write $S=\frac{1}{2}J$ with $JJ=-I_{16}$; the even powers of θS give the cosine series of $\theta/2$ and the odd ones J times the sine series. For a boost $JJ=+I_{16}$ and the hyperbolic functions appear.) Example: a column u with $Bu=u$ and $u^\dagger u=1$, boosted by $S^{(x_1x_5)}$ (Hermitian, Krein-unitary). Since R is Hermitian, $R^\dagger R=RR=\exp(2\theta S)=\cosh\theta\,I_{16}+2\sinh\theta\,S$, and $u^\dagger Su=0$ (because S anticommutes with B: $u^\dagger Su=u^\dagger SBu=-u^\dagger BSu=-(Bu)^\dagger Su=-u^\dagger Su$); so the ordinary squared length $(Ru)^\dagger(Ru)=u^\dagger R^\dagger Ru$ becomes $\cosh\theta$, while the Krein norm stays 1.

statement	**status**	**where it is verified**
the invariant bilinear forms are $r_-CP_-+r_+CP_+$	PROVED (from the commutant of Spin(4,4)); COMPUTED in Notebook 10e (two zero eigenvalues, separation from about 7 down to 1.3×10^{-15})	`python-algebra.json`, checks `spin_commutant_dimension_2` and `S_preserves_C_and_commutes_with_Gamma`; `wolfram-algebra.json`, check `S_preserves_C`
every invariant charge density satisfies $\Gamma X\Gamma=-X$ and is indefinite	PROVED (above) and checked for 40 random densities	Notebook 10e, In [5], check `40 random invariant densities: Gamma X Gamma = -X, 8 positive, 8 negative`; `wolfram-algebra.json`, check `B_signature_8_8` for $X=B$
exactly the 21 generators without x_4 are Krein-unitary; defect $iC\gamma^{(x_b)}$ for $S^{(x_4x_b)}$	PROVED	`wolfram-algebra.json`, check `S_preserves_B_only_off_x4`
the counts 12 (anti-Hermitian), 13 (commuting with B), 9 (both)	PROVED (above) and checked	Notebook 10e, In [6], check `counts: 12 anti-Hermitian, 13 commute with B, 9 both unitary and Krein`

10.35 Example: Notebook 10e computes the invariant forms and the generators

Notebook 10e builds the 28 generators, solves the 28×256 linear equations $S^T G + GS = 0$ numerically and checks the two solutions $CP_{\mp}$ exactly (Section 10.33), checks for 40 random invariant densities that every one has eight positive and eight negative eigenvalues, sorts the 28 generators into unitary, commuting with B and Krein-unitary (Section 10.34), and follows a column under a rotation and two boosts. It draws five figures and ends with the line ALL 15 CHECKS PASSED (notebook 10e).

10.36 How to run Notebook 10e

Step 1. What this notebook does and what it needs.

Notebook 10e (Invariant forms and Krein-unitary symmetries) is the file `Revision/textbook/notebooks/10e_invariant_forms.ipynb` of the repository Dirac_claude. It finds every bilinear form of the 16 components that is invariant under the 28 generators of Spin(4,4) (a two-dimensional family, spanned by C times the two chiral projectors), shows that every charge density built from such a form and the time direction x4 has as many negative as positive eigenvalues, so that no choice makes the charge positive; it then sorts the 28 generators by whether they keep the canonical anticommutator (Krein-unitary: the 21 that do not involve x4) and whether they are unitary (9 do both), follows finite rotations and boosts, reproduces the recorded checks, and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10e_invariant_forms.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10e_invariant_forms.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 10e_invariant_forms.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10e.captions.json`
- `Revision/textbook/figures/10e_1_invariant_forms.png`
- `Revision/textbook/figures/10e_2_singular_values.png`
- `Revision/textbook/figures/10e_3_density_eigenvalues.png`
- `Revision/textbook/figures/10e_4_generator_types.png`
- `Revision/textbook/figures/10e_5_finite_transformations.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10e_5_finite_transformations.png exists
ALL 15 CHECKS PASSED (notebook 10e)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10e_invariant_forms.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10e_invariant_forms.ipynb
python -m nbconvert --execute --inplace 10e_invariant_forms.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.37 Notebook 10e: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10e_invariant_forms.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The canonical anticommutator of dirac16complex contains the matrix $B =
-iC\gamma^{(x_4)}$, which has eight positive and eight negative eigenvalues: the
charge $\Psi^\dagger B\Psi$ is indefinite. Could a cleverer choice of the charge
density avoid this? This notebook answers no, and then asks which symmetry
transformations of the spinor keep the canonical anticommutator. It

- finds EVERY matrix $G$ for which the bilinear form $\Psi^TG\Phi$ is unchanged by
  the 28 generators $S^{ab} = \frac14[\gamma^{(x_a)}, \gamma^{(x_b)}]$ of Spin(4,4):
  the solutions form a two-dimensional family, spanned by $CP_-$ and $CP_+$, where
  $P_\mp = \frac12(I \mp \Gamma)$ are the projectors onto the two chiral halves;
- shows that every charge density made from such a form and the time direction,
  the Hermitian part of $cG\gamma^{(x_4)}$, satisfies $\Gamma X\Gamma = -X$ and
  therefore has as many negative as positive eigenvalues (unless it is zero);
- sorts the 28 generators: which are anti-Hermitian (their exponentials are
  unitary), which commute with $B$, and which are *Krein-unitary* (their
  exponentials keep $B$): exactly the 21 generators that do not involve $x_4$, the
  generators of Spin(4,3);
- follows finite rotations and boosts and draws what happens to the ordinary
  squared length and to the Krein norm of a column;
- reproduces the recorded checks and draws five teaching figures.
```

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 10e (Invariant forms and Krein-unitary symmetries) is the file ‘Revision/textbook/notebooks/10e_invariant_forms.ipynb‘ of the repository Dirac_claude. It finds every bilinear form of the 16 components that is invariant under the 28 generators of Spin(4,4) (a two-dimensional family, spanned by C times the two chiral projectors), shows that every charge density built from such a form and the time direction x4 has as many negative as positive eigenvalues, so that no choice makes the charge positive; it then sorts the 28 generators by whether they keep the canonical anticommutator (Krein-unitary: the 21 that do not involve x4) and whether they are unitary (9 do both), follows finite rotations and boosts, reproduces the recorded checks, and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 10e_invariant_forms.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 10 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.

**Step 6. Or run the notebook without a browser (headless).**
```

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10e_invariant_forms.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10e_invariant_forms.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10e.captions.json‘
- ‘Revision/textbook/figures/10e_1_invariant_forms.png‘
- ‘Revision/textbook/figures/10e_2_singular_values.png‘
- ‘Revision/textbook/figures/10e_3_density_eigenvalues.png‘
- ‘Revision/textbook/figures/10e_4_generator_types.png‘
- ‘Revision/textbook/figures/10e_5_finite_transformations.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10e_5_finite_transformations.png exists
ALL 15 CHECKS PASSED (notebook 10e)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10e_invariant_forms.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 10e_invariant_forms.ipynb
      python -m nbconvert --execute --inplace 10e_invariant_forms.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 10e (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
```

```
# Notebook 10e (Invariant forms and Krein-unitary symmetries) is the file
# Revision/textbook/notebooks/10e_invariant_forms.ipynb of the repository Dirac_claude.
# It finds every bilinear form of the 16 components that is invariant under the 28
# generators of Spin(4,4) (a two-dimensional family, spanned by C times the two chiral
# projectors), shows that every charge density built from such a form and the time
# direction x4 has as many negative as positive eigenvalues, so that no choice makes the
# charge positive; it then sorts the 28 generators by whether they keep the canonical
# anticommutator (Krein-unitary: the 21 that do not involve x4) and whether they are
# unitary (9 do both), follows finite rotations and boosts, reproduces the recorded
# checks, and draws five teaching figures. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
```

```
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10e_invariant_forms.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10e_invariant_forms.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10e_invariant_forms.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
```

```
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10e.captions.json
# - Revision/textbook/figures/10e_1_invariant_forms.png
# - Revision/textbook/figures/10e_2_singular_values.png
# - Revision/textbook/figures/10e_3_density_eigenvalues.png
# - Revision/textbook/figures/10e_4_generator_types.png
# - Revision/textbook/figures/10e_5_finite_transformations.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 10e_5_finite_transformations.png exists
#     ALL 15 CHECKS PASSED (notebook 10e)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10e_invariant_forms.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10e_invariant_forms.ipynb
```

```
#     python -m nbconvert --execute --inplace 10e_invariant_forms.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10e"  # this notebook: chapter 10, example e


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
```

```
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
```

```
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10e complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Generator, Spin(4,4)**: the 28 matrices $S^{ab}$ ($a < b$) generate the spin
  transformations $R = \exp(\theta S^{ab})$, which turn the frame of the eight
  directions; their products form Spin(4,4). Spin(4,3) is the part generated by the
  21 $S^{ab}$ with $a, b \neq x_4$ (the transformations that leave the time $x_4$
  alone).
- **Bilinear form, invariant**: a form $\Psi^TG\Phi$ with a $16 \times 16$ matrix $G$.
  It is *invariant* if it does not change when both columns are transformed by the
  same spin transformation; to first order in $\theta$ this is the condition
  $(S^{ab})^TG + GS^{ab} = 0$ for all 28 generators.
```

- **Hermitian form, charge density**: a form $\Psi^\dagger X\Psi$ with a Hermitian matrix X; it is real for every Ψ. A charge density is the time component of a current.
- **Chiral projectors**: $P_\mp = \frac12(I \mp \Gamma)$ with the chirality $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$; P_- keeps components 1 to 8, P_+ components 9 to 16.
- **Singular values**: for a matrix M the square roots of the eigenvalues of M^TM. The number of zero singular values of M is the number of independent solutions of $Mx = 0$.
- **Unitary, Krein-unitary**: R is unitary if $R^\dagger R = I$ (it keeps the ordinary length), Krein-unitary if $R^\dagger BR = B$ (it keeps the Krein form and the canonical anticommutator). For $R = \exp(\theta S)$ the first-order conditions are $S^\dagger = -S$ and $S^\dagger B + BS = 0$.
- **Rotation, boost**: $R = \exp(\theta S^{ab})$ is a rotation when a and b are both space-like or both time-like ($S^2 = -\frac14 I$, cosines and sines), and a boost when one is space-like and one time-like ($S^2 = +\frac14 I$, hyperbolic cosines and sines).

Text cell:

```
## 4. The physical and mathematical situation
```

Why invariant forms are the right candidates. Under a spin transformation $\Psi \to R\Psi$ the form $\Psi^TG\Psi$ becomes $\Psi^TR^TGR\Psi$. For $R = I + \theta S$ with small θ, $R^TGR = G + \theta(S^TG + GS) + \theta^2S^TGS$; the form is unchanged to first order exactly when $S^TG + GS = 0$. The gammas, and hence the S^{ab}, are real, so $S^\dagger = S^T$, and the same condition makes the Hermitian form $\Psi^\dagger G\Psi$ invariant. The notebook solves these 28×256 linear equations for the 256 entries of G.

Why every such density is indefinite. The solutions will be $G = r_-CP_- + r_+CP_+$. Both C and $P_\mp$ commute with Γ (C is a product of four gammas, an even number), so G commutes with Γ; $\gamma^{(x_4)}$ anticommutes with Γ. Hence $\Gamma(cG\gamma^{(x_4)})\Gamma = cG\Gamma\gamma^{(x_4)}\Gamma = -cG\gamma^{(x_4)}$, and the same for its Hermitian part X: $\Gamma X\Gamma = -X$. If $Xv = \lambda v$, then $X(\Gamma v) = -\Gamma Xv = -\lambda\,\Gamma v$: every eigenvalue λ comes with $-\lambda$, with the same multiplicity. So the density $\Psi^\dagger X\Psi$ is indefinite unless $X = 0$. The canonical choice is $B = -iC\gamma^{(x_4)}$ ($r_- = r_+ = 1$, $c = -i$).

Which transformations keep B. B is $-i$ times the product of the five gammas $\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}\gamma^{(x_4)}$. A gamma of x_1, x_2, x_3, x_4, x_0 anticommutes with four of the five factors and commutes with itself, so it commutes with B; a gamma of x_5, x_6, x_7 anticommutes with all five and so anticommutes with B. For an anti-Hermitian generator the Krein condition $S^\dagger B + BS = 0$ means $[B, S] = 0$; for a Hermitian one it means $\{B, S\} = 0$. The notebook counts the generators of each kind.

Text cell:

```
## 5. The gammas, the generators and the chiral projectors
```

The next cell reads the gammas, builds C, Γ, B, the projectors $P_\mp$,
the 28 generators S^{ab} ($a < b$) and the helper ‘check_record‘ (the PASS line
and the line of the reproduced Revision record are printed in one piece, so that
the stored output is the same in every run). It checks the recorded relations of
B with the gammas.
Before it checks anything, ‘check_record‘ asks the helper ‘record_says_pass‘ whether
the cited record still says PASS: for a record name of the form
‘<report file>, check <check name>‘ it opens that report (each file once, kept in
the dictionary ‘REPORT_CHECKS‘) and stops the notebook with an error unless a check
of that name exists there with the verdict PASS; a record name without ‘, check ‘
names a data entry that the notebook reads and compares itself.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import itertools  # all pairs (a, b) with a < b
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
from matplotlib.colors import LinearSegmentedColormap, ListedColormap  # colours


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
```

```
eta = {a: fixture["eta"][a] for a in range(8)}
I16 = np.eye(16)
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])
B = -1j * (C @ gamma[4])
P_minus, P_plus = (I16 - Gamma) / 2, (I16 + Gamma) / 2  # the chiral projectors
pairs = list(itertools.combinations(range(8), 2))  # (0, 1), (0, 2), ..., (6, 7)
S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}
report("number of generators S^ab", len(S))
commuting = [a for a in range(8) if np.array_equal(B @ gamma[a], gamma[a] @ B)]
anticommuting = [a for a in range(8) if np.array_equal(B @ gamma[a], -gamma[a] @ B)]
say(f"B commutes with gamma^(x_a) for a = {commuting}, anticommutes for a = "
    f"{anticommuting}")
check_record(commuting == [0, 1, 2, 3, 4] and anticommuting == [5, 6, 7],
             "B commutes with the gammas of x0, x1, x2, x3, x4, anticommutes with x5-x7",
             record="Revision/algebra/reports/python-algebra.json, check "
                    "B_gamma_relations")
```

Out [2]:

```
RESULT number of generators S^ab = 28
B commutes with gamma^(x_a) for a = [0, 1, 2, 3, 4], anticommutes for a = [5, 6, 7]
PASS B commutes with the gammas of x0, x1, x2, x3, x4, anticommutes with x5-x7
     reproduces Revision/algebra/reports/python-algebra.json, check B_gamma_relations
```

Text cell:

```
## 6. Every invariant bilinear form

The unknown $G$ has 256 entries; write them as one long column $\vec G$ (row after
row). The matrix equation $S^TG + GS = 0$ is linear in $\vec G$: with the rule
"the row-by-row column of $AXD$ is $(A \otimes D^T)\vec X$" ($\otimes$ is the
Kronecker product, `np.kron`), it reads $(S^T \otimes I + I \otimes S^T)\vec G = 0$.
Stacking the 28 generators gives a matrix $M$ with $28 \cdot 256 = 7168$ rows and
256 columns, and the invariant forms are the solutions of $M\vec G = 0$.

The number of independent solutions is the number of zero singular values of $M$,
that is the number of zero eigenvalues of the $256 \times 256$ matrix $M^TM$. The
next cell computes them. Because $M$ is made of the numbers $0, \pm\frac14,
\pm\frac12$, a nonzero eigenvalue of $M^TM$ cannot be tiny; the cell prints the
smallest nonzero one and the largest "zero" one, to show that the separation is
enormous, so rounding cannot change the count. Then it checks EXACTLY (the entries
of $C$, $P_\mp$ and $S^{ab}$ are multiples of $\frac12$ and $\frac14$, which the
computer stores without rounding) that $CP_-$ and $CP_+$ solve every equation and
are independent, so they span the whole solution space.
```

In [3]:

```
rows = [np.kron(s.T, I16) + np.kron(I16, s.T) for s in S.values()]
M = np.vstack(rows)  # 7168 rows, 256 columns
```

```
values = np.linalg.eigvalsh(M.T @ M)  # 256 eigenvalues, sorted, all >= 0
zero_count = int(np.sum(values < 1e-9))
report("shape of M", M.shape)
report("eigenvalues of M^T M below 1e-9 (independent invariant forms)", zero_count)
report("largest of them and smallest nonzero eigenvalue",
       f"{values[zero_count - 1]:.1e} and {values[zero_count]:.4f}")

forms = {"C P_minus": C @ P_minus, "C P_plus": C @ P_plus}
exact_ok = all(np.array_equal(s.T @ G + G @ s, np.zeros((16, 16)))
               for G in forms.values() for s in S.values())
independent = np.linalg.matrix_rank(np.vstack([G.reshape(1, -1)
                                               for G in forms.values()])) == 2
check(zero_count == 2 and exact_ok and independent,
      "the invariant bilinear forms are exactly the combinations of C P- and C P+")
check_record(all(np.array_equal(s.T @ C + C @ s, np.zeros((16, 16)))
                 for s in S.values()),
             "C itself (= C P- + C P+) is invariant: Psibar Psi is a Spin(4,4) scalar",
             record="Revision/algebra/reports/wolfram-algebra.json, check "
                    "S_preserves_C")
check_record(all(np.array_equal(Gamma @ s, s @ Gamma) for s in S.values()),
             "every generator commutes with Gamma: the chiral halves are invariant",
             record="Revision/algebra/reports/python-algebra.json, check "
                    "S_preserves_C_and_commutes_with_Gamma")
```

Out [3]:

```
RESULT shape of M = (7168, 256)
RESULT eigenvalues of M^T M below 1e-9 (independent invariant forms) = 2
RESULT largest of them and smallest nonzero eigenvalue = 1.3e-15 and 7.0000
PASS the invariant bilinear forms are exactly the combinations of C P- and C P+
PASS C itself (= C P- + C P+) is invariant: Psibar Psi is a Spin(4,4) scalar
     reproduces Revision/algebra/reports/wolfram-algebra.json, check S_preserves_C
PASS every generator commutes with Gamma: the chiral halves are invariant
     reproduces Revision/algebra/reports/python-algebra.json, check
    S_preserves_C_and_commutes_with_Gamma
```

Text cell:

```
The next cell draws the two invariant forms as heat maps and the 256 eigenvalues of
$M^TM$ on a logarithmic scale, where the two zeros stand far below all the others.
```

In [4]:

```
titles = {"C P_minus": "$CP_-$ (components 1 to 8)",
          "C P_plus": "$CP_+$ (components 9 to 16)"}
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.0))
for ax, (label, G) in zip(axes, forms.items()):
    image = ax.imshow(G, cmap=DIVERGING, vmin=-1, vmax=1)
    ticks = list(range(0, 16, 3))
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
```

```
    ax.grid(False)
    ax.set_title(titles[label])
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "invariant_forms",
            "The two independent Spin(4,4)-invariant bilinear forms of the 16 "
            "components as heat maps: $CP_-$ (left) and $CP_+$ (right), where $C$ "
            "is the author's sigma16 and $P_\\mp$ project onto the chiral halves "
            "(rows and columns 1 to 8, and 9 to 16). Horizontal axis: column "
            "number, vertical axis: row number; red $+1$, blue $-1$, light grey 0. "
            "Every invariant form is a combination of these two; $C = CP_- + CP_+$ "
            "gives the scalar $\\bar\\Psi\\Psi$.")

fig, ax = plt.subplots()
ax.semilogy(range(1, 257), np.maximum(values, 1e-18), "o", color=BLUE, markersize=3)
ax.axhline(1e-9, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("number of the eigenvalue (sorted)")
ax.set_ylabel("eigenvalue of $M^TM$ (log scale)")
ax.set_title("Two zero eigenvalues: two invariant forms")
save_figure(fig, "singular_values",
            "The 256 eigenvalues of $M^TM$ (vertical axis, logarithmic scale, pure "
            "numbers; values below $10^{-18}$ drawn at $10^{-18}$), sorted from the "
            "smallest (horizontal axis: their number). $M$ collects the equations "
            "$(S^{ab})^TG + GS^{ab} = 0$ for the 28 generators. Exactly two "
            "eigenvalues lie at the rounding level, far below the dotted line at "
            "$10^{-9}$, and all others are at least about 7: the invariant forms "
            "make a two-dimensional family.")
```

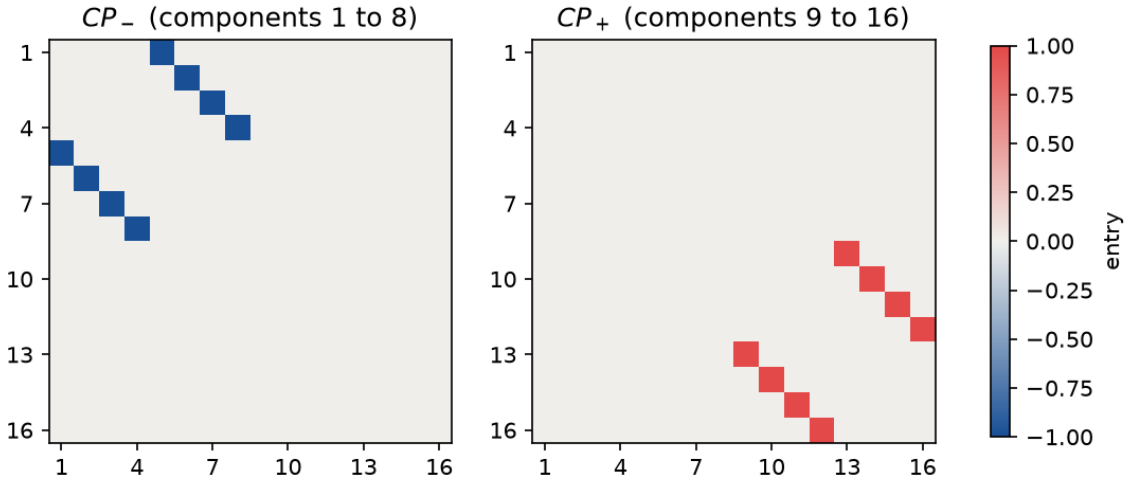

Figure 288: The two independent Spin(4,4)-invariant bilinear forms of the 16 components as heat maps: CP_- (left) and CP_+ (right), where C is the author's sigma16 and $P_\mp$ project onto the chiral halves (rows and columns 1 to 8, and 9 to 16). Horizontal axis: column number, vertical axis: row number; red $+1$, blue -1, light grey 0. Every invariant form is a combination of these two; $C = CP_- + CP_+$ gives the scalar $\bar\Psi\Psi$. (Notebook 10e, figure 1.)

Out [4]:

```
Figure 10e.1 saved as Revision/textbook/figures/10e_1_invariant_forms.png
```

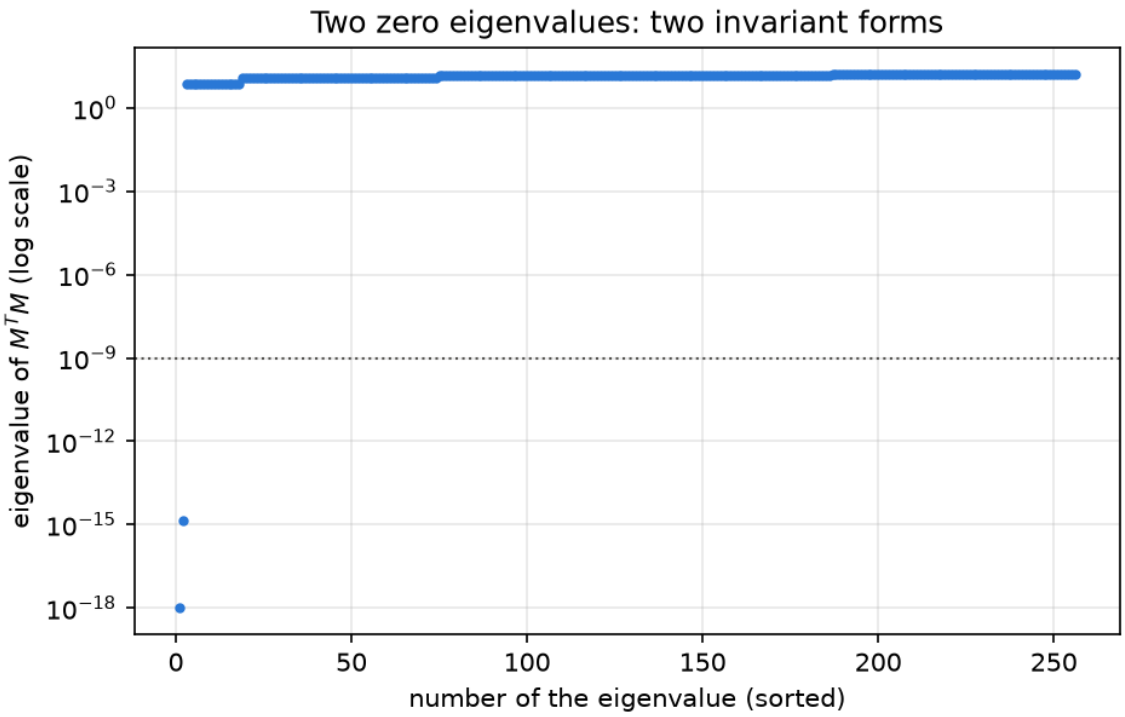

Figure 289: The 256 eigenvalues of $M^T M$ (vertical axis, logarithmic scale, pure numbers; values below 10^{-18} drawn at 10^{-18}), sorted from the smallest (horizontal axis: their number). M collects the equations $(S^{ab})^T G + G S^{ab} = 0$ for the 28 generators. Exactly two eigenvalues lie at the rounding level, far below the dotted line at 10^{-9}, and all others are at least about 7: the invariant forms make a two-dimensional family. (Notebook 10e, figure 2.)

Out [4]:

```
Figure 10e.2 saved as Revision/textbook/figures/10e_2_singular_values.png
```

Text cell:

```
## 7. Every invariant charge density is indefinite

The next cell takes 40 random invariant forms $G = r_-CP_- + r_+CP_+$ (real $r_\mp$,
because $CP_\mp$ is real and symmetric, so $G$ is Hermitian exactly for real
coefficients) and random complex numbers $c$, builds the Hermitian part $X =
\frac12(cG\gamma^{(x_4)} + (cG\gamma^{(x_4)})^\dagger)$, and checks $\Gamma X\Gamma =
-X$ and that $X$ has eight positive and eight negative eigenvalues (every sample has
$X \neq 0$). It also checks that the canonical choice $c = -i$, $r_\mp = 1$ gives
$X = B$ with eigenvalues $\pm1$, eight each.
```

In [5]:

```
rng = np.random.default_rng(12345)
spectra, all_ok = [], True
```

```
for _ in range(40):
    r_minus, r_plus = rng.normal(size=2)
    c = complex(rng.normal(), rng.normal())
    G = r_minus * C @ P_minus + r_plus * C @ P_plus
    Y = c * G @ gamma[4]
    X = (Y + Y.conj().T) / 2  # the Hermitian part
    eigenvalues = np.linalg.eigvalsh(X)
    spectra.append(eigenvalues)
    positive = int(np.sum(eigenvalues > 1e-12))  # how many eigenvalues are > 0
    negative = int(np.sum(eigenvalues < -1e-12))  # how many are < 0
    all_ok = (all_ok and np.allclose(Gamma @ X @ Gamma, -X)
              and positive == 8 and negative == 8)
G_canonical = C @ P_minus + C @ P_plus  # = C
Y = -1j * G_canonical @ gamma[4]
X_canonical = (Y + Y.conj().T) / 2
canonical_ok = np.allclose(X_canonical, B)
canonical_values = np.linalg.eigvalsh(X_canonical)
check(all_ok,
      "40 random invariant densities: Gamma X Gamma = -X, 8 positive, 8 negative")
check_record(canonical_ok and np.allclose(canonical_values, [-1] * 8 + [1] * 8),
             "the canonical density is B, with eigenvalues -1 and +1, eight each",
             record="Revision/algebra/reports/wolfram-algebra.json, check "
                    "B_signature_8_8")

fig, ax = plt.subplots()
for index, eigenvalues in enumerate(spectra, 1):
    ax.plot([index] * 16, eigenvalues, "_", color=BLUE if index % 2 else GREEN,
            markersize=10, markeredgewidth=2)
ax.plot([42] * 16, canonical_values, "_", color=ORANGE, markersize=12,
        markeredgewidth=3)
ax.annotate("$B$", (42, 1.0), xytext=(40.5, 1.6), color=ORANGE)
ax.axhline(0.0, color=GREY, linewidth=1)
ax.set_xlabel("random invariant density (1 to 40), and the canonical $B$ (42)")
ax.set_ylabel("eigenvalues of the density matrix $X$")
ax.set_title("Every invariant charge density is indefinite")
save_figure(fig, "density_eigenvalues",
            "The 16 eigenvalues (vertical axis, pure numbers) of 40 random "
            "Spin(4,4)-invariant charge densities $X$, the Hermitian part of "
            "$cG\\gamma^{(x_4)}$ with $G = r_-CP_- + r_+CP_+$ (horizontal axis: the "
            "sample number; blue and green alternate), and of the canonical "
            "density $B$ (orange, at 42). Every density has eight positive and "
            "eight negative eigenvalues, mirror images in the grey line: no "
            "invariant choice makes the charge positive, because the time gamma "
            "$\\gamma^{(x_4)}$ is odd and every invariant form even under the "
            "chirality $\\Gamma$.")
```

Out [5]:

```
PASS 40 random invariant densities: Gamma X Gamma = -X, 8 positive, 8 negative
PASS the canonical density is B, with eigenvalues -1 and +1, eight each
     reproduces Revision/algebra/reports/wolfram-algebra.json, check B_signature_8_8
```

Out [5]:

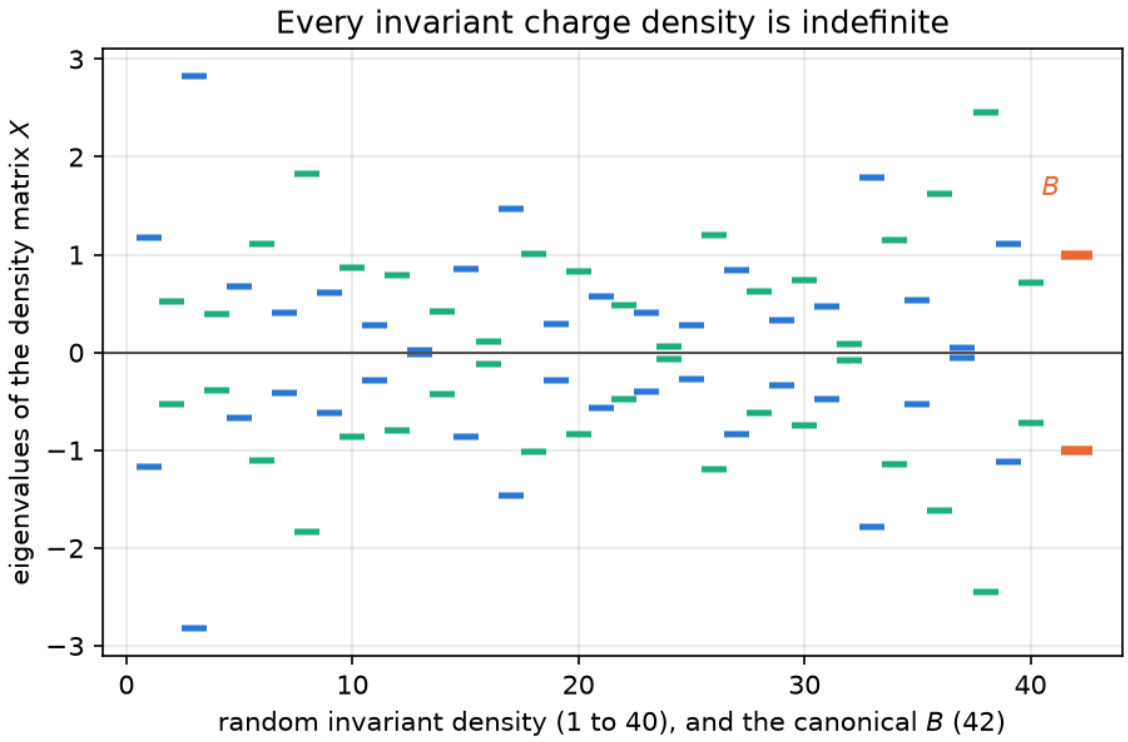

Figure 290: The 16 eigenvalues (vertical axis, pure numbers) of 40 random Spin(4,4)-invariant charge densities X, the Hermitian part of $cG\gamma^{(x_4)}$ with $G = r_-CP_- + r_+CP_+$ (horizontal axis: the sample number; blue and green alternate), and of the canonical density B (orange, at 42). Every density has eight positive and eight negative eigenvalues, mirror images in the grey line: no invariant choice makes the charge positive, because the time gamma $\gamma^{(x_4)}$ is odd and every invariant form even under the chirality Γ. (Notebook 10e, figure 3.)

```
Figure 10e.3 saved as Revision/textbook/figures/10e_3_density_eigenvalues.png
```

Text cell:

```
## 8. Which generators keep the canonical anticommutator?

The next cell sorts the 28 generators. For each it records whether it is
anti-Hermitian ($S^\dagger = -S$; its exponential is then unitary), whether it
commutes with $B$, and whether it satisfies the Krein condition $S^\dagger B + BS
= 0$ (its exponential then keeps $B$ and the canonical anticommutator). The Revision
record states that the Krein condition holds exactly for the 21 generators with
$a, b \neq x_4$ and that for the 7 generators $S^{(x_4b)}$ (with $x_4$ written first)
the left-hand side equals $iC\gamma^{(x_b)}$.
```

In [6]:

```
kinds = {}
for (a, b), s in S.items():
    anti_hermitian = np.array_equal(s.conj().T, -s)
    commutes = np.allclose(B @ s, s @ B)
    krein = np.allclose(s.conj().T @ B + B @ s, 0)
    kinds[(a, b)] = (anti_hermitian, commutes, krein)
counts = {"anti-Hermitian (unitary)": sum(k[0] for k in kinds.values()),
          "commute with B": sum(k[1] for k in kinds.values()),
          "Krein-unitary": sum(k[2] for k in kinds.values()),
          "both unitary and Krein-unitary": sum(k[0] and k[2] for k in kinds.values())}
for label, number in counts.items():
    report(label, number)
krein_pairs = sorted(pair for pair, k in kinds.items() if k[2])
off_x4 = sorted(pair for pair in pairs if 4 not in pair)
x4_ok = True
for (a, b), s in S.items():
    if 4 in (a, b):
        other = b if a == 4 else a
        s_x4_first = s if a == 4 else -s  # S^(x4 b) = -S^(b x4)
        left = s_x4_first.conj().T @ B + B @ s_x4_first
        x4_ok = x4_ok and np.allclose(left, 1j * C @ gamma[other])
check_record(krein_pairs == off_x4 and len(krein_pairs) == 21 and x4_ok,
             "Krein-unitary: exactly the 21 generators without x4 (Spin(4,3))",
             record="Revision/algebra/reports/wolfram-algebra.json, check "
                    "S_preserves_B_only_off_x4")
check(counts["anti-Hermitian (unitary)"] == 12 and counts["commute with B"] == 13
      and counts["both unitary and Krein-unitary"] == 9,
      "counts: 12 anti-Hermitian, 13 commute with B, 9 both unitary and Krein")
```

Out [6]:

```
RESULT anti-Hermitian (unitary) = 12
RESULT commute with B = 13
RESULT Krein-unitary = 21
RESULT both unitary and Krein-unitary = 9
```

```
PASS Krein-unitary: exactly the 21 generators without x4 (Spin(4,3))
     reproduces Revision/algebra/reports/wolfram-algebra.json, check
    S_preserves_B_only_off_x4
PASS counts: 12 anti-Hermitian, 13 commute with B, 9 both unitary and Krein
```

Text cell:

```
The next cell draws the 28 generators as the squares above the diagonal of an
$8 \times 8$ grid (row $a$, column $b$, $a < b$), coloured by their kind: blue for
the 9 that are unitary AND Krein-unitary (rotations among $x_0, x_1, x_2, x_3$ and
among $x_5, x_6, x_7$), green for the 12 that are Krein-unitary but not unitary
(boosts mixing $x_0, x_1, x_2, x_3$ with $x_5, x_6, x_7$), orange for the 7 that
involve $x_4$ and break the canonical anticommutator.
```

In [7]:

```
grid = np.full((8, 8), np.nan)  # NaN: no generator (diagonal and below)
for (a, b), (anti_hermitian, _, krein) in kinds.items():
    grid[a - 1, b - 1] = 0 if (anti_hermitian and krein) else (1 if krein else 2)
colours = ListedColormap([BLUE, GREEN, ORANGE])
fig, ax = plt.subplots(figsize=(6.0, 5.4))
ax.imshow(grid, cmap=colours, vmin=-0.5, vmax=2.5)
names = [f"$x_{a}$" for a in range(8)]
ax.set_xticks(range(8), names)
ax.set_yticks(range(8), names)
ax.grid(False)
for (a, b), (anti_hermitian, _, krein) in kinds.items():
    mark = "rot" if anti_hermitian else "boost"  # a rotation or a boost
    ax.text(b - 1, a - 1, mark, ha="center", va="center", color="white", fontsize=7)
ax.plot([], [], "s", color=BLUE, label="unitary and Krein-unitary (9)")
ax.plot([], [], "s", color=GREEN, label="Krein-unitary only (12)")
ax.plot([], [], "s", color=ORANGE, label="breaks the anticommutator (7)")
ax.legend(loc="lower left", fontsize=8)
ax.set_xlabel("second direction $b$")
ax.set_ylabel("first direction $a$")
ax.set_title("The 28 generators $S^{ab}$ and the canonical anticommutator")
save_figure(fig, "generator_types",
            "The 28 generators $S^{ab}$ of Spin(4,4), one square for each pair of "
            "directions $a < b$ (vertical axis: $a$, horizontal axis: $b$); rot "
            "marks a rotation (anti-Hermitian generator), boost a boost (Hermitian "
            "generator). Blue: unitary and Krein-unitary (9); green: "
            "Krein-unitary but not unitary (12); orange: the 7 generators that "
            "involve the time $x_4$ and do not keep $B$. The 21 blue and green "
            "squares, every pair without $x_4$, generate Spin(4,3), the "
            "transformations that keep the canonical anticommutator.")
```

Out [7]:

```
Figure 10e.4 saved as Revision/textbook/figures/10e_4_generator_types.png
```

Text cell:

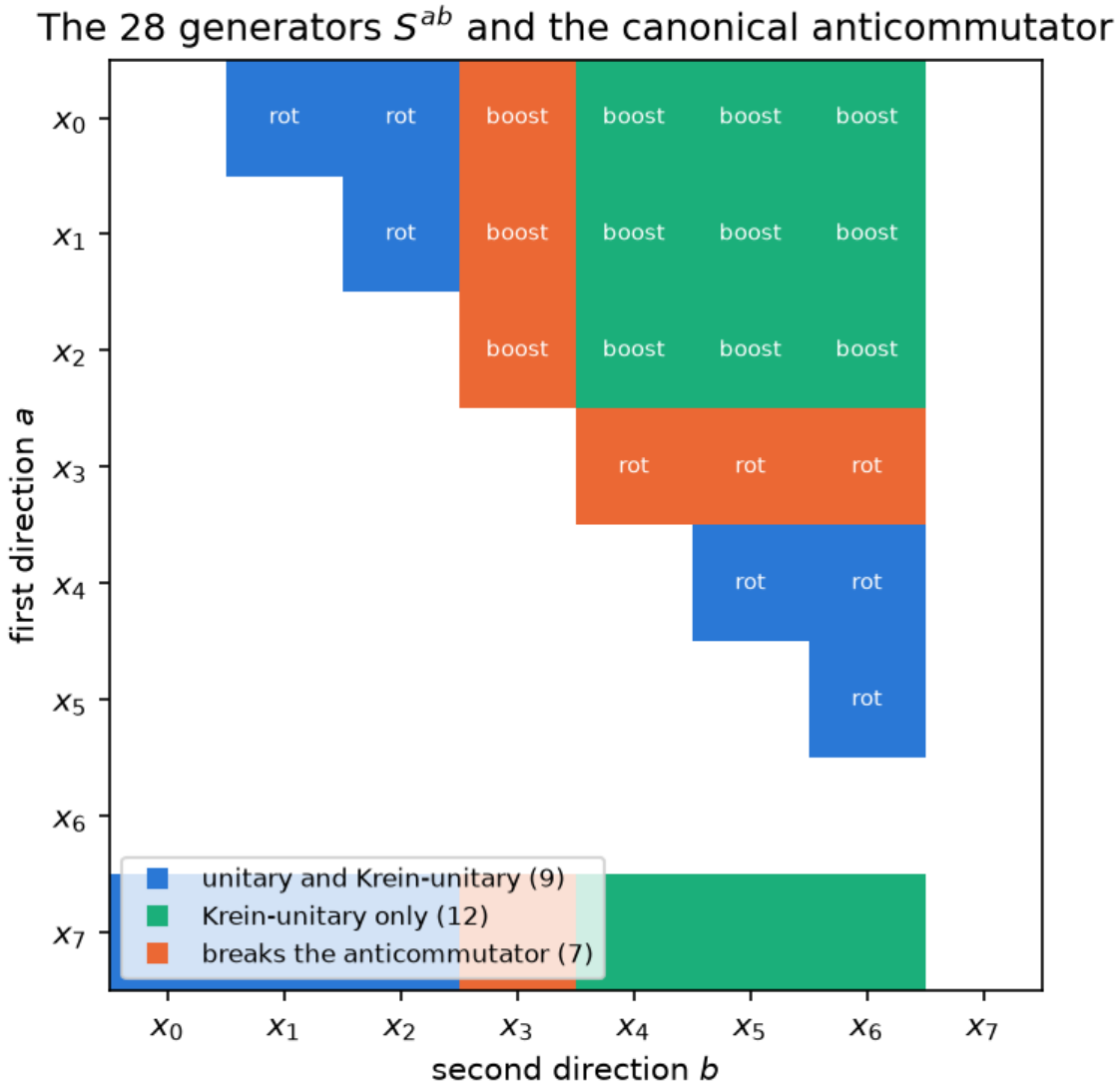

Figure 291: The 28 generators S^{ab} of Spin(4,4), one square for each pair of directions $a < b$ (vertical axis: a, horizontal axis: b); rot marks a rotation (anti-Hermitian generator), boost a boost (Hermitian generator). Blue: unitary and Krein-unitary (9); green: Krein-unitary but not unitary (12); orange: the 7 generators that involve the time x_4 and do not keep B. The 21 blue and green squares, every pair without x_4, generate Spin(4,3), the transformations that keep the canonical anticommutator. (Notebook 10e, figure 4.)

```
## 9. Finite rotations and boosts

Since $(\gamma^{(x_a)}\gamma^{(x_b)})^2 = -\eta_{aa}\eta_{bb}I$ for $a \neq b$, the
generator obeys $(S^{ab})^2 = -\frac14\eta_{aa}\eta_{bb}I$, and the exponential series
collapses: for $\eta_{aa}\eta_{bb} = +1$ (a rotation) $\exp(\theta S) =
\cos\frac\theta2\,I + 2\sin\frac\theta2\,S$, and for $\eta_{aa}\eta_{bb} = -1$ (a
boost) $\exp(\theta S) = \cosh\frac\theta2\,I + 2\sinh\frac\theta2\,S$. The next
cell checks the formula against a numerical matrix exponential (the sum of the
series up to the 30th power) and then follows a column $u$ with $u^\dagger u = 1$ and
$u^\dagger Bu = 1$ under three transformations: the rotation $S^{(x_1x_2)}$, the
boost $S^{(x_1x_5)}$ (Krein-unitary) and the boost $S^{(x_1x_4)}$ (not
Krein-unitary). It plots the squared length $(Ru)^\dagger(Ru)$ and the Krein norm
$(Ru)^\dagger B(Ru)$ against $\theta$.
```

In [8]:

```
def spin_transformation(a, b, theta):
    """exp(theta S^ab) from the closed formula."""
    s = S[(a, b)]
    if eta[a] * eta[b] == 1:  # a rotation
        return np.cos(theta / 2) * I16 + 2 * np.sin(theta / 2) * s
    return np.cosh(theta / 2) * I16 + 2 * np.sinh(theta / 2) * s  # a boost


def series_exponential(matrix, terms=30):
    """exp(matrix) as the sum of matrix^n / n! for n = 0, ..., terms."""
    result, power = np.eye(16), np.eye(16)
    for n in range(1, terms + 1):
        power = power @ matrix / n
        result = result + power
    return result


worst = max(np.max(np.abs(spin_transformation(a, b, 0.7)
                          - series_exponential(0.7 * S[(a, b)]))) for a, b in pairs)
report("largest difference between the formula and the series", f"{worst:.1e}")
check(worst < 1e-12, "the closed formula for exp(theta S^ab) is right for all 28")

u = ((I16 + B) / 2)[:, 0]  # a column with B u = u
u = u / np.linalg.norm(u)
thetas = np.linspace(-3.0, 3.0, 241)
curves = {}
for pair in ((1, 2), (1, 5), (1, 4)):
    lengths, kreins = [], []
    for theta in thetas:
        v = spin_transformation(*pair, theta) @ u
        lengths.append((v.conj() @ v).real)
        kreins.append((v.conj() @ B @ v).real)
    curves[pair] = (np.array(lengths), np.array(kreins))
kept = {pair: np.max(np.abs(curves[pair][1] - 1)) < 1e-12 for pair in curves}
say(f"Krein norm kept: rotation x1x2 {kept[(1, 2)]}, boost x1x5 {kept[(1, 5)]}, "
    f"boost x1x4 {kept[(1, 4)]}")
check(kept[(1, 2)] and kept[(1, 5)] and not kept[(1, 4)]
```

```
      and np.max(np.abs(curves[(1, 2)][0] - 1)) < 1e-12,
      "finite: the rotation keeps both norms, the x1x5 boost only the Krein norm")
```

Out [8]:

```
RESULT largest difference between the formula and the series = 6.7e-16
PASS the closed formula for exp(theta S^ab) is right for all 28
Krein norm kept: rotation x1x2 True, boost x1x5 True, boost x1x4 False
PASS finite: the rotation keeps both norms, the x1x5 boost only the Krein norm
```

Text cell:

```
The next cell draws the two norms for the three transformations.
```

In [9]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharex=True)
styles = {(1, 2): (BLUE, "-", "rotation $S^{(x_1x_2)}$"),
          (1, 5): (GREEN, "--", "boost $S^{(x_1x_5)}$"),
          (1, 4): (ORANGE, ":", "boost $S^{(x_1x_4)}$")}
for pair, (colour, style, label) in styles.items():
    axes[0].plot(thetas, curves[pair][0], style, color=colour, linewidth=2,
                 label=label)
    axes[1].plot(thetas, curves[pair][1], style, color=colour, linewidth=2,
                 label=label)
axes[0].set_ylabel("squared length $(Ru)^\\dagger(Ru)$")
axes[1].set_ylabel("Krein norm $(Ru)^\\dagger B (Ru)$")
for ax in axes:
    ax.set_xlabel("angle or rapidity $\\theta$")
    ax.legend(fontsize=8)
axes[0].set_title("unitary: only the rotation keeps the length")
axes[1].set_title("Krein-unitary: the rotation and the $x_1x_5$ boost")
save_figure(fig, "finite_transformations",
            "A column $u$ with $u^\\dagger u = 1$ and $u^\\dagger Bu = 1$ transformed "
            "by $R = \\exp(\\theta S^{ab})$ for the angle or rapidity $\\theta$ from "
            "$-3$ to $3$ (horizontal axes): the rotation $S^{(x_1x_2)}$ (blue), the "
            "boost $S^{(x_1x_5)}$ between a space direction and an extra time (green "
            "dashed) and the boost $S^{(x_1x_4)}$ that involves the time $x_4$ "
            "(orange dotted). Left: the squared length, kept only by the rotation. "
            "Right: the Krein norm, kept by the rotation and by the $x_1x_5$ boost, "
            "but not by the boost that involves $x_4$. Vertical axes: pure numbers.")
```

Out [9]:

```
Figure 10e.5 saved as Revision/textbook/figures/10e_5_finite_transformations.png
```

Text cell:

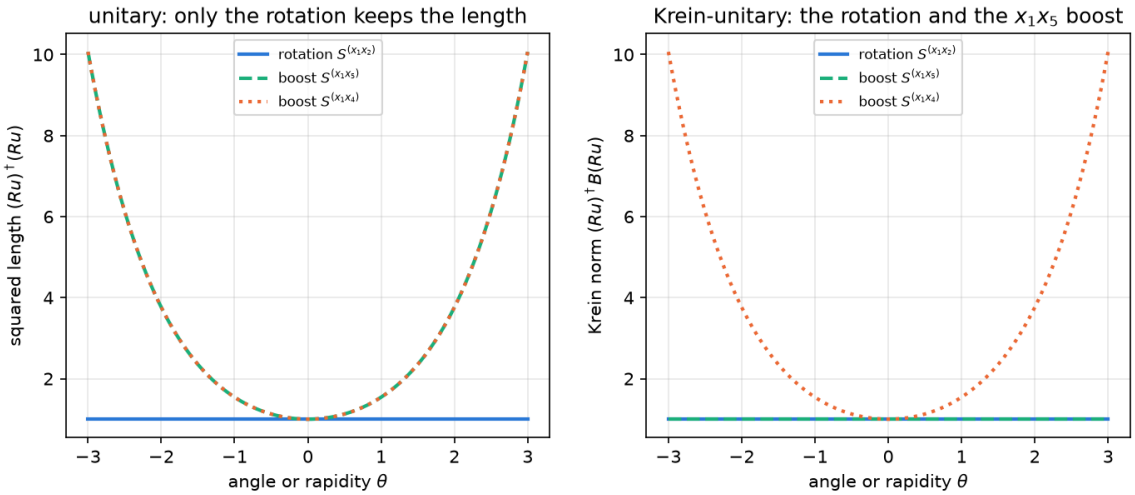

Figure 292: A column u with $u^\dagger u = 1$ and $u^\dagger B u = 1$ transformed by $R = \exp(\theta S^{ab})$ for the angle or rapidity θ from -3 to 3 (horizontal axes): the rotation $S^{(x_1x_2)}$ (blue), the boost $S^{(x_1x_5)}$ between a space direction and an extra time (green dashed) and the boost $S^{(x_1x_4)}$ that involves the time x_4 (orange dotted). Left: the squared length, kept only by the rotation. Right: the Krein norm, kept by the rotation and by the x_1x_5 boost, but not by the boost that involves x_4. Vertical axes: pure numbers. (Notebook 10e, figure 5.)

```
## 10. The last check

The last cell checks that all five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [10]:

```
for name in ("10e_1_invariant_forms.png", "10e_2_singular_values.png",
             "10e_3_density_eigenvalues.png", "10e_4_generator_types.png",
             "10e_5_finite_transformations.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [10]:

```
PASS the figure file 10e_1_invariant_forms.png exists
PASS the figure file 10e_2_singular_values.png exists
PASS the figure file 10e_3_density_eigenvalues.png exists
PASS the figure file 10e_4_generator_types.png exists
PASS the figure file 10e_5_finite_transformations.png exists
ALL 15 CHECKS PASSED (notebook 10e)
```

Text cell:

```
## 11. What this notebook showed

- COMPUTED (with a separation of many orders of magnitude between zero and nonzero
  eigenvalues) and CHECKED EXACTLY: the bilinear forms invariant under all 28
  generators of Spin(4,4) form a two-dimensional family, spanned by $CP_-$ and
  $CP_+$; $C$ itself gives the scalar $\bar\Psi\Psi$.
- PROVED (Section 4) and CHECKED for 40 random choices: every charge density built
  from an invariant form and the time gamma, the Hermitian part of
  $cG\gamma^{(x_4)}$, satisfies $\Gamma X\Gamma = -X$ and has eight positive and
  eight negative eigenvalues. The indefinite (Krein) charge of dirac16complex is not
  an accident of the choice $B$: in signature (4,4) no invariant density is
  positive.
- REPRODUCED: exactly the 21 generators that do not involve $x_4$ are
  Krein-unitary (they generate Spin(4,3), the transformations that keep the
  canonical anticommutator), and for the 7 generators $S^{(x_4b)}$ the defect is
  $iC\gamma^{(x_b)}$; of the 21, the 9 rotations among $x_0, x_1, x_2, x_3$ and among
  $x_5, x_6, x_7$ are also unitary. COMPUTED: the counts 12 (anti-Hermitian) and 13
  (commuting with $B$).
- The canonical structure singles out the time $x_4$, as $\Psi^\dagger\Psi$ does in
  ordinary Dirac theory; transformations that turn $x_4$ into another direction
  (here the 7 generators with $x_4$) do not keep $B$, and the slices $x_4 = $ const
  on which the theory is quantised are then changed as well.
```

10.38 Line-by-line walk-through of Notebook 10e

The notebook has 10 code cells, In [1] to In [10]; docstrings are left out of the quotations (they are printed in Section 10.37).

In [1], the set-up cell. Its first 246 lines are comments that repeat the run instructions of Section 10.36. The code is that of In [1] of Notebook 10a (Section 10.10), with the line `NOTEBOOK_ID = "10e" # this notebook: chapter 10, example e`. It prints `Set-up of notebook 10e complete: repository folder found, helpers defined.`

In [2], the gammas, the generators and the chiral projectors.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import itertools  # all pairs (a, b) with a < b
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
from matplotlib.colors import LinearSegmentedColormap, ListedColormap  # colours


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
```

```
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output
```

The imports of In [2] of Notebook 10a, and in addition `itertools`, a module of Python that lists combinations, and two kinds of colour scales of matplotlib. `REPORT_CHECKS`, `record_says_pass` and `check_record` are the helpers of Section 10.10 (a cited report check must still exist with the verdict PASS; the two printed lines come in one piece).

```
BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
eta = {a: fixture["eta"][a] for a in range(8)}
I16 = np.eye(16)
```

The colours and the blue-grey-red colour scale of Notebook 10a; the gammas of the record as decimal numbers, the signs η_{aa} of the frame metric, and the identity I_{16}.

```
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])
B = -1j * (C @ gamma[4])
P_minus, P_plus = (I16 - Gamma) / 2, (I16 + Gamma) / 2  # the chiral projectors
```

C, the chirality Γ, B, and the projectors $P_{\mp} = \frac{1}{2}(I_{16} \mp \Gamma)$; since $\Gamma = \mathrm{diag}(-I_8, I_8)$, P_- keeps the components 1 to 8 and P_+ the components 9 to 16.

```
pairs = list(itertools.combinations(range(8), 2))  # (0, 1), (0, 2), ..., (6, 7)
S = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}
report("number of generators S^ab", len(S))
```

`itertools.combinations(range(8), 2)` lists every pair (a, b) with $a < b$ from 0 to 7, $\binom{8}{2} = 28$ pairs. The dictionary `S` stores $S^{ab} = \frac{1}{4}(\gamma^{(x_a)}\gamma^{(x_b)} - \gamma^{(x_b)}\gamma^{(x_a)})$ for each pair; the RESULT line prints 28.

```
commuting = [a for a in range(8) if np.array_equal(B @ gamma[a], gamma[a] @ B)]
anticommuting = [a for a in range(8) if np.array_equal(B @ gamma[a], -gamma[a] @ B)]
```

```
say(f"B commutes with gamma^(x_a) for a = {commuting}, anticommutes for a = "
    f"{anticommuting}")
check_record(commuting == [0, 1, 2, 3, 4] and anticommuting == [5, 6, 7],
             "B commutes with the gammas of x0, x1, x2, x3, x4, anticommutes with x5-x7",
             record="Revision/algebra/reports/python-algebra.json, check "
                    "B_gamma_relations")
```

The lists of the directions whose gamma commutes or anticommutes with B. The printed line shows `[0, 1, 2, 3, 4]` and `[5, 6, 7]`, as Section 10.2 derived, and the check reproduces the record check `B_gamma_relations`.

In [3], every invariant bilinear form.

```
rows = [np.kron(s.T, I16) + np.kron(I16, s.T) for s in S.values()]
M = np.vstack(rows)  # 7168 rows, 256 columns
values = np.linalg.eigvalsh(M.T @ M)  # 256 eigenvalues, sorted, all >= 0
zero_count = int(np.sum(values < 1e-9))
```

The unknown G has 256 entries; write them, row after row, as one long column $\vec{G}$. The rule "the row-by-row column of XGY is $(X \otimes Y^T)\vec{G}$", where $\otimes$ is the **Kronecker product** (`np.kron`: the matrix made of the blocks $X_{ij}Y^T$), turns $S^TG = S^TGI_{16}$ into $(S^T \otimes I_{16})\vec{G}$ and $GS = I_{16}GS$ into $(I_{16} \otimes S^T)\vec{G}$. So each generator gives a 256×256 block of equations, and `np.vstack` stacks the 28 blocks into the matrix M with $28 \times 256 = 7168$ rows. The invariant forms are the solutions of $M\vec{G} = 0$. Their number equals the number of zero eigenvalues of the 256×256 matrix M^TM (if $M^TM\vec{G} = 0$ then $\vec{G}^TM^TM\vec{G} = \|M\vec{G}\|^2 = 0$, so $M\vec{G} = 0$). `np.linalg.eigvalsh` computes its 256 eigenvalues, sorted, and `zero_count` counts those below 10^{-9}.

```
report("shape of M", M.shape)
report("eigenvalues of M^T M below 1e-9 (independent invariant forms)", zero_count)
report("largest of them and smallest nonzero eigenvalue",
       f"{values[zero_count - 1]:.1e} and {values[zero_count]:.4f}")
```

The printed lines show the shape `(7168, 256)`, two zero eigenvalues, the larger of them 1.3×10^{-15} (rounding) and the smallest nonzero eigenvalue 7.0000: a separation of about fifteen orders of magnitude, so rounding cannot change the count.

```
forms = {"C P_minus": C @ P_minus, "C P_plus": C @ P_plus}
exact_ok = all(np.array_equal(s.T @ G + G @ s, np.zeros((16, 16)))
               for G in forms.values() for s in S.values())
independent = np.linalg.matrix_rank(np.vstack([G.reshape(1, -1)
                                               for G in forms.values()])) == 2
check(zero_count == 2 and exact_ok and independent,
      "the invariant bilinear forms are exactly the combinations of C P- and C P+")
```

The two candidates CP_- and CP_+ of Section 10.33. Their entries, and those of the S^{ab}, are multiples of $\frac{1}{2}$ and $\frac{1}{4}$, which the computer stores without rounding, so `np.array_equal` checks $S^TG + GS = 0$ exactly for both forms and all 28 generators. `G.reshape(1, -1)` writes a matrix as one row of 256 numbers; the two rows stacked have rank 2, so the two forms are independent. With exactly two zero eigenvalues they span all solutions.

```
check_record(all(np.array_equal(s.T @ C + C @ s, np.zeros((16, 16)))
                 for s in S.values()),
             "C itself (= C P- + C P+) is invariant: Psibar Psi is a Spin(4,4) scalar",
             record="Revision/algebra/reports/wolfram-algebra.json, check "
                    "S_preserves_C")
check_record(all(np.array_equal(Gamma @ s, s @ Gamma) for s in S.values()),
             "every generator commutes with Gamma: the chiral halves are invariant",
             record="Revision/algebra/reports/python-algebra.json, check "
                    "S_preserves_C_and_commutes_with_Gamma")
```

Two facts of the Revision algebra record: C itself is invariant, and every generator commutes with Γ (a product of two gammas passes Γ with the sign $(-1)^2 = +1$). The cell prints three PASS lines.

In [4], pictures of the invariant forms and of the eigenvalues.

```
titles = {"C P_minus": "$CP_-$ (components 1 to 8)",
          "C P_plus": "$CP_+$ (components 9 to 16)"}
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.0))
for ax, (label, G) in zip(axes, forms.items()):
    image = ax.imshow(G, cmap=DIVERGING, vmin=-1, vmax=1)
    ticks = list(range(0, 16, 3))
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.grid(False)
    ax.set_title(titles[label])
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "invariant_forms",
...)
```

The two forms as heat maps, with the steps of the function `draw_matrix` of Notebook 10a written out (rows and columns labelled from 1 at every third one, no grid). Figure 1 of Notebook 10e shows CP_- filled only in the upper left 8×8 block and CP_+ only in the lower right one, each block with one red or blue square per row and column; together they make C.

```
fig, ax = plt.subplots()
ax.semilogy(range(1, 257), np.maximum(values, 1e-18), "o", color=BLUE, markersize=3)
ax.axhline(1e-9, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("number of the eigenvalue (sorted)")
ax.set_ylabel("eigenvalue of $M^TM$ (log scale)")
ax.set_title("Two zero eigenvalues: two invariant forms")
save_figure(fig, "singular_values",
...)
```

`ax.semilogy` draws with a logarithmic vertical axis; values below 10^{-18} (the zeros, which rounding may even make slightly negative) are drawn at 10^{-18}. The dotted line marks the threshold 10^{-9}. Figure 2 of Notebook 10e shows two points far down at the rounding level and all 254 others at about 7 or higher: two invariant forms.

In [5], every invariant charge density is indefinite.

```
rng = np.random.default_rng(12345)
spectra, all_ok = [], True
for _ in range(40):
    r_minus, r_plus = rng.normal(size=2)
    c = complex(rng.normal(), rng.normal())
    G = r_minus * C @ P_minus + r_plus * C @ P_plus
    Y = c * G @ gamma[4]
    X = (Y + Y.conj().T) / 2  # the Hermitian part
```

For 40 random choices: two real coefficients $r_{\mp}$ (the two values returned by `rng.normal(size=2)` are given two names at once), a random complex number c, the invariant form $G = r_{-}CP_{-} + r_{+}CP_{+}$ (real coefficients, so that G is Hermitian), $Y = cG\gamma^{(x_4)}$ and its Hermitian part X (Section 10.33).

```
    eigenvalues = np.linalg.eigvalsh(X)
    spectra.append(eigenvalues)
    positive = int(np.sum(eigenvalues > 1e-12))  # how many eigenvalues are > 0
    negative = int(np.sum(eigenvalues < -1e-12))  # how many are < 0
    all_ok = (all_ok and np.allclose(Gamma @ X @ Gamma, -X)
              and positive == 8 and negative == 8)
```

The 16 eigenvalues of X are stored for the figure and counted by sign; `all_ok` stays true while every sample has $\Gamma X \Gamma = -X$ (`np.allclose` allows rounding) and eight eigenvalues of each sign.

```
G_canonical = C @ P_minus + C @ P_plus  # = C
Y = -1j * G_canonical @ gamma[4]
X_canonical = (Y + Y.conj().T) / 2
canonical_ok = np.allclose(X_canonical, B)
canonical_values = np.linalg.eigvalsh(X_canonical)
check(all_ok,
      "40 random invariant densities: Gamma X Gamma = -X, 8 positive, 8 negative")
check_record(canonical_ok and np.allclose(canonical_values, [-1] * 8 + [1] * 8),
             "the canonical density is B, with eigenvalues -1 and +1, eight each",
             record="Revision/algebra/reports/wolfram-algebra.json, check "
                    "B_signature_8_8")
```

The canonical choice $r_{\mp} = 1$, $c = -i$ gives $X = B$ with the sorted eigenvalues eight times -1 and eight times $+1$. Two PASS lines; the second reproduces the record check `B_signature_8_8`.

```
fig, ax = plt.subplots()
for index, eigenvalues in enumerate(spectra, 1):
    ax.plot([index] * 16, eigenvalues, "_", color=BLUE if index % 2 else GREEN,
            markersize=10, markeredgewidth=2)
ax.plot([42] * 16, canonical_values, "_", color=ORANGE, markersize=12,
        markeredgewidth=3)
ax.annotate("$B$", (42, 1.0), xytext=(40.5, 1.6), color=ORANGE)
ax.axhline(0.0, color=GREY, linewidth=1)
ax.set_xlabel("random invariant density (1 to 40), and the canonical $B$ (42)")
ax.set_ylabel("eigenvalues of the density matrix $X$")
ax.set_title("Every invariant charge density is indefinite")
save_figure(fig, "density_eigenvalues",
...)
```

`enumerate(spectra, 1)` numbers the samples from 1. Each sample is drawn as 16 short horizontal marks (the marker `"_"`) at its eigenvalues, above its number; the colours alternate between blue and green, and B is drawn in orange at position 42. Figure 3 of Notebook 10e shows for every sample marks above and below the grey zero line in mirror-image positions: eigenvalues in pairs $\pm\lambda$, eight of each sign, as Section 10.33 proved.

In [6], which generators keep the canonical rule.

```
kinds = {}
for (a, b), s in S.items():
    anti_hermitian = np.array_equal(s.conj().T, -s)
    commutes = np.allclose(B @ s, s @ B)
    krein = np.allclose(s.conj().T @ B + B @ s, 0)
    kinds[(a, b)] = (anti_hermitian, commutes, krein)
```

For each generator three truth values are stored: anti-Hermitian ($S^\dagger = -S$), commuting with B, and the Krein condition $S^\dagger B + BS = 0$ (Section 10.34).

```
counts = {"anti-Hermitian (unitary)": sum(k[0] for k in kinds.values()),
          "commute with B": sum(k[1] for k in kinds.values()),
          "Krein-unitary": sum(k[2] for k in kinds.values()),
          "both unitary and Krein-unitary": sum(k[0] and k[2] for k in kinds.values())}
for label, number in counts.items():
    report(label, number)
```

Summing truth values counts the true ones (`True` counts as 1). The RESULT lines print 12 anti-Hermitian generators, 13 that commute with B, 21 Krein-unitary ones and 9 that are both: the counts derived in Section 10.34.

```
krein_pairs = sorted(pair for pair, k in kinds.items() if k[2])
off_x4 = sorted(pair for pair in pairs if 4 not in pair)
x4_ok = True
for (a, b), s in S.items():
    if 4 in (a, b):
        other = b if a == 4 else a
        s_x4_first = s if a == 4 else -s  # S^(x4 b) = -S^(b x4)
        left = s_x4_first.conj().T @ B + B @ s_x4_first
        x4_ok = x4_ok and np.allclose(left, 1j * C @ gamma[other])
```

`krein_pairs` is the sorted list of the Krein-unitary pairs, `off_x4` the sorted list of the pairs without the direction 4. For each of the 7 pairs that contain 4, the generator is written with x_4 first ($S^{ab} = -S^{ba}$, so for the pairs $(1,4)$, $(2,4)$, $(3,4)$ the sign is reversed), and the defect $S^\dagger B + BS$ is compared with $iC\gamma^{(x_b)}$, b being the other direction.

```
check_record(krein_pairs == off_x4 and len(krein_pairs) == 21 and x4_ok,
             "Krein-unitary: exactly the 21 generators without x4 (Spin(4,3))",
             record="Revision/algebra/reports/wolfram-algebra.json, check "
                    "S_preserves_B_only_off_x4")
check(counts["anti-Hermitian (unitary)"] == 12 and counts["commute with B"] == 13
      and counts["both unitary and Krein-unitary"] == 9,
      "counts: 12 anti-Hermitian, 13 commute with B, 9 both unitary and Krein")
```

The first check reproduces the record check `S_preserves_B_only_off_x4`: the Krein-unitary generators are exactly the 21 without x_4, and the 7 with x_4 have the defect $iC\gamma^{(x_b)}$. The second confirms the counts. Two PASS lines.

In [7], the 28 generators as a picture.

```
grid = np.full((8, 8), np.nan)  # NaN: no generator (diagonal and below)
for (a, b), (anti_hermitian, _, krein) in kinds.items():
    grid[a - 1, b - 1] = 0 if (anti_hermitian and krein) else (1 if krein else 2)
colours = ListedColormap([BLUE, GREEN, ORANGE])
fig, ax = plt.subplots(figsize=(6.0, 5.4))
ax.imshow(grid, cmap=colours, vmin=-0.5, vmax=2.5)
```

An 8×8 table, empty (NaN, drawn white) on and below the diagonal; above it, the square of row a and column b gets 0 for a generator that is unitary and Krein-unitary, 1 for Krein-unitary only, 2 otherwise. `ListedColormap` is a colour scale of three fixed colours; with the range from -0.5 to 2.5 the values 0, 1 and 2 fall into its three thirds: blue, green and orange.

```
names = [f"$x_{a}$" for a in range(8)]
ax.set_xticks(range(8), names)
ax.set_yticks(range(8), names)
ax.grid(False)
for (a, b), (anti_hermitian, _, krein) in kinds.items():
    mark = "rot" if anti_hermitian else "boost"  # a rotation or a boost
    ax.text(b - 1, a - 1, mark, ha="center", va="center", color="white", fontsize=7)
```

The rows and columns are labelled x_0 to x_7, and each square gets the word `rot` (an anti-Hermitian generator, a rotation) or `boost` (a Hermitian one), written in white at its centre.

```
ax.plot([], [], "s", color=BLUE, label="unitary and Krein-unitary (9)")
ax.plot([], [], "s", color=GREEN, label="Krein-unitary only (12)")
ax.plot([], [], "s", color=ORANGE, label="breaks the anticommutator (7)")
ax.legend(loc="lower left", fontsize=8)
ax.set_xlabel("second direction $b$")
ax.set_ylabel("first direction $a$")
ax.set_title("The 28 generators $S^{ab}$ and the canonical anticommutator")
save_figure(fig, "generator_types",
...)
```

Empty plots with square markers create the three legend entries, placed in the empty lower left corner. Figure 4 of Notebook 10e shows the column and the row of x_4 orange (the 7 generators that break the canonical rule), blue rotations among x_1, x_2, x_3 and x_0 and among x_5, x_6, x_7, and green boosts between those two groups.

In [8], finite rotations and boosts.

```
def spin_transformation(a, b, theta):
    s = S[(a, b)]
    if eta[a] * eta[b] == 1:  # a rotation
```

```
        return np.cos(theta / 2) * I16 + 2 * np.sin(theta / 2) * s
    return np.cosh(theta / 2) * I16 + 2 * np.sinh(theta / 2) * s  # a boost


def series_exponential(matrix, terms=30):
    result, power = np.eye(16), np.eye(16)
    for n in range(1, terms + 1):
        power = power @ matrix / n
        result = result + power
    return result
```

`spin_transformation` is the closed formula of Section 10.34; `series_exponential` sums the power series $\sum_{n=0}^{30} X^n/n!$ directly (each pass multiplies the previous term by X/n, so `power` is $X^n/n!$).

```
worst = max(np.max(np.abs(spin_transformation(a, b, 0.7)
                          - series_exponential(0.7 * S[(a, b)]))) for a, b in pairs)
report("largest difference between the formula and the series", f"{worst:.1e}")
check(worst < 1e-12, "the closed formula for exp(theta S^ab) is right for all 28")
```

For $\theta = 0.7$ and all 28 generators the two agree to 6.7×10^{-16}, printed as `6.7e-16`.

```
u = ((I16 + B) / 2)[:, 0]  # a column with B u = u
u = u / np.linalg.norm(u)
thetas = np.linspace(-3.0, 3.0, 241)
curves = {}
for pair in ((1, 2), (1, 5), (1, 4)):
    lengths, kreins = [], []
    for theta in thetas:
        v = spin_transformation(*pair, theta) @ u
        lengths.append((v.conj() @ v).real)
        kreins.append((v.conj() @ B @ v).real)
    curves[pair] = (np.array(lengths), np.array(kreins))
```

A column u with $Bu = u$ and length 1 (so its Krein norm is 1) is transformed by the rotation $S^{(x_1x_2)}$, the boost $S^{(x_1x_5)}$ and the boost $S^{(x_1x_4)}$ for 241 values of θ from -3 to 3, and its ordinary squared length and its Krein norm are stored. `spin_transformation(*pair, theta)` passes the two numbers of the pair as the first two arguments.

```
kept = {pair: np.max(np.abs(curves[pair][1] - 1)) < 1e-12 for pair in curves}
say(f"Krein norm kept: rotation x1x2 {kept[(1, 2)]}, boost x1x5 {kept[(1, 5)]}, "
    f"boost x1x4 {kept[(1, 4)]}")
check(kept[(1, 2)] and kept[(1, 5)] and not kept[(1, 4)]
      and np.max(np.abs(curves[(1, 2)][0] - 1)) < 1e-12,
      "finite: the rotation keeps both norms, the x1x5 boost only the Krein norm")
```

The printed line shows that the Krein norm is kept by the rotation and the x_1x_5 boost (`True`) but not by the x_1x_4 boost (`False`); the check also requires the rotation to keep the ordinary length. Two PASS lines in the cell.

In [9], the two norms in a picture.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharex=True)
styles = {(1, 2): (BLUE, "-", "rotation $S^{(x_1x_2)}$"),
          (1, 5): (GREEN, "--", "boost $S^{(x_1x_5)}$"),
          (1, 4): (ORANGE, ":", "boost $S^{(x_1x_4)}$")}
for pair, (colour, style, label) in styles.items():
    axes[0].plot(thetas, curves[pair][0], style, color=colour, linewidth=2,
                 label=label)
    axes[1].plot(thetas, curves[pair][1], style, color=colour, linewidth=2,
                 label=label)
```

Each transformation gets a colour, a line style (solid, dashed, dotted) and a label; the left panel draws the ordinary squared lengths $(Ru)^\dagger(Ru)$, the right panel the Krein norms.

```
axes[0].set_ylabel("squared length $(Ru)^\\dagger(Ru)$")
axes[1].set_ylabel("Krein norm $(Ru)^\\dagger B (Ru)$")
for ax in axes:
    ax.set_xlabel("angle or rapidity $\\theta$")
    ax.legend(fontsize=8)
axes[0].set_title("unitary: only the rotation keeps the length")
axes[1].set_title("Krein-unitary: the rotation and the $x_1x_5$ boost")
save_figure(fig, "finite_transformations",
...)
```

Figure 5 of Notebook 10e shows on the left the rotation flat at 1 while both boosts rise ($\cosh\theta$ for the x_1x_5 boost, Section 10.34), and on the right the rotation and the x_1x_5 boost flat at 1 while the x_1x_4 boost moves away: a boost between a space direction and an extra time keeps the canonical rule, a boost that involves the time x_4 does not.

In [10], the last check.

```
for name in ("10e_1_invariant_forms.png", "10e_2_singular_values.png",
             "10e_3_density_eigenvalues.png", "10e_4_generator_types.png",
             "10e_5_finite_transformations.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The five figure files exist, and the last line prints ALL 15 CHECKS PASSED (notebook 10e): the five figure checks and the 10 checks of the cells before (1 in In [2], 3 in In [3], 2 each in In [5], In [6] and In [8]).

10.39 The Krein structure along the deflating history

Sections 10.3 to 10.6 studied waves with momenta that do not change. In the author's metric they DO change: the three extra times x_5, x_6, x_7 deflate exponentially (scale factor $e^{-a_4}\sin^{1/6} z$, with a_4 increasing), and 3-space inflates (scale factor $e^{a_4}\sin^{1/6} z$). This section follows the one-particle structure of the chapter along the deflating history that the Kohn-Sham record uses.

The history and its status. The Kohn-Sham record takes the linear history

$$a_4 = AHx_4, \qquad A = H = m = 1$$

(record `Revision/kohn_sham/results/parameters.json`, entries `physics.historyA`, `physics.H`, `physics.m`; its slices are $a_{4,0} = 0, 0.5, 1, 1.5, 2$). This history is a PRESCRIBED BACKGROUND: it is not solved from the field equations for a_4 with this field as the source, and the Kohn-Sham states violate the source conditions that the linear member requires (`Revision/field_equations_a4/reports/ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`; Chapter 17). Every statement of this section is made under this ASSUMPTION.

The local-frame model. At each instant x_4 and at one hidden position, the coefficients of the wave equation are evaluated there and the two hidden-direction terms of Section 10.27 are left out; this is the *local-frame* (WKB-type) statement of the Revision record that "every extra-time mode eventually enters the growing regime" (`python-field-theory.json`, check `extra_time_modes_grow`). The model is an ASSUMPTION; the extra times are never treated as static: their scale factor changes at every instant. The Kohn-Sham record measures the hidden direction by $y = \ln(\sin z)/(6H)$, so that

$$\sin^{1/6} z = e^{\ln(\sin z)/6} = e^{Hy},$$

the patch end $z = \pi/2$ is $y = 0$, and $y < 0$ lies towards the tip.

The wave equation of one instant, line by line. Take a wave $\Psi = u(x_4)\, e^{i(q_1 x_1 + q_5 x_5)}$ with the **coordinate momenta** q_1 (along 3-space) and q_5 (along an extra time). In the field equation of Section 10.27 the derivative ∂_1 becomes iq_1 and ∂_5 becomes iq_5, each divided by its scale factor:

$$ik_1\gamma^{(x_1)}u + \gamma^{(x_4)}\frac{du}{dx_4} + ik_5\gamma^{(x_5)}u = mu,$$

with the frame momenta

$$k_1 = \frac{q_1}{e^{a_4}\sin^{1/6} z} = q_1 e^{-AHx_4 - Hy}, \qquad k_5 = \frac{q_5}{e^{-a_4}\sin^{1/6} z} = q_5 e^{AHx_4 - Hy}.$$

The **frame momentum** k_1 SHRINKS as 3-space inflates; k_5 GROWS as the extra times deflate. The steps of Section 10.3 (multiply by $\gamma^{(x_4)}$, solve for du/dx_4, multiply by i) give

$$i\frac{du}{dx_4} = h(x_4)\, u, \qquad h(x_4) = -im\gamma^{(x_4)} - k_1(x_4)\,\gamma^{(x_4)}\gamma^{(x_1)} - k_5(x_4)\,\gamma^{(x_4)}\gamma^{(x_5)}.$$

At every instant this is a mode Hamiltonian of Section 10.3 with real frame momenta, so by Section 10.4

$$h(x_4)^2 = w(x_4)^2 I_{16}, \qquad w(x_4)^2 = m^2 + k_1^2 - k_5^2 = m^2 + q_1^2 e^{-2AHx_4 - 2Hy} - q_5^2 e^{2AHx_4 - 2Hy}.$$

For $A > 0$ the second term falls and the third grows, so w^2 falls all the time, and if $q_5 \neq 0$ it becomes negative after a finite time: every wave with an extra-time momentum eventually grows.

The onset time, line by line. Write $X = e^{2AHx_4} > 0$ and $c = e^{-2Hy} > 0$. Then $e^{-2AHx_4 - 2Hy} = c/X$ and $e^{2AHx_4 - 2Hy} = cX$, and the condition $w^2 = 0$ reads

$$m^2 + q_1^2\,\frac{c}{X} - q_5^2\,cX = 0.$$

Multiply by $-X$ (which is not 0):

$$q_5^2 c\,X^2 - m^2 X - q_1^2 c = 0.$$

This is a quadratic equation $aX^2 + bX + d = 0$ with $a = q_5^2 c$, $b = -m^2$, $d = -q_1^2 c$; its roots are $X = (-b \pm \sqrt{b^2 - 4ad})/(2a)$, here $(m^2 \pm \sqrt{m^4 + 4q_1^2 q_5^2 c^2})/(2q_5^2 c)$. The square root is at least m^2, so the root with the minus sign is negative (or 0 when $q_1 = 0$), while $X = e^{2AHx_4}$ is positive. The only possible root is

$$X^* = \frac{m^2 + \sqrt{m^4 + 4q_1^2 q_5^2 c^2}}{2q_5^2 c}, \qquad x_4^* = \frac{\ln X^*}{2AH},$$

the **onset time**, by taking the logarithm of $X^* = e^{2AHx_4^*}$. Before x_4^* the frequency is real (the wave oscillates); after it, it is imaginary (the wave grows). For $q_1 = 0$ and $y = 0$ ($c = 1$): $X^* = 2m^2/(2q_5^2) = m^2/q_5^2$ and $x_4^* = \ln(m/q_5)/(AH)$: the onset comes when the growing frame momentum $k_5 = q_5 e^{AHx_4}$ reaches the mass. With $A = H = m = 1$ and $q_5 = 0.05$ this is $\ln 20 \approx 2.996$; Notebook 10f computes the onset of the wave $q_1 = 0.5$, $q_5 = 0.05$, $y = 0$ as 2.996044. The record's exact sample "$m = 1$, $k_5 = 2$: eigenvalues $\pm i\sqrt{3}$" is an instant of this history: the wave $q_1 = 0$, $q_5 = 0.05$ has $k_5 = 0.05\,e^{x_4} = 2$ at $x_4 = \ln 40$.

How far from Hermitian, line by line. By Section 10.4, $-i\gamma^{(x_4)}$ and $\gamma^{(x_4)}\gamma^{(x_1)}$ are Hermitian and commute with B, while $\gamma^{(x_4)}\gamma^{(x_5)}$ is anti-Hermitian and anticommutes with B. So $h = h_H + h_A$ with the anti-Hermitian part

$$h_A = -k_5\,\gamma^{(x_4)}\gamma^{(x_5)},$$

$$h_A^\dagger h_A = -h_A h_A = -k_5^2(\gamma^{(x_4)}\gamma^{(x_5)})^2 = -k_5^2\big(-\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_5)}\gamma^{(x_5)}\big) = k_5^2 I_{16}.$$

The steps: $h_A^\dagger = -h_A$; the square of the product; exchanging the middle factors costs a sign; $\gamma^{(x_4)}\gamma^{(x_4)} = \gamma^{(x_5)}\gamma^{(x_5)} = -I_{16}$. The **size** of a matrix (the largest factor by which it stretches the length of a column) is therefore exactly k_5 for h_A: $\|h_A u\|^2 = u^\dagger h_A^\dagger h_A u = k_5^2\,u^\dagger u$. In the same way $Bh - hB = Bh_A - h_A B = 2Bh_A$ (h_H commutes with B, h_A anticommutes) has the size $2k_5$, because B keeps lengths ($B^\dagger B = BB = I_{16}$). Both sizes grow like e^{a_4}: the deflation drives every wave with extra-time momentum ever further from a Hermitian evolution that keeps the two eigenspaces of B apart. Both vanish at every time in the good sector, $q_5 = 0$.

Krein inertia at every instant. At each instant Theorem 10.1 applies: while $w^2 > 0$ each eigenspace of $h(x_4)$ has the Krein inertia $(4, 4)$; once $w^2 < 0$ each is Krein-neutral. The wave crosses from the first kind to the second at the onset time. For the band of waves with $q_1 = 0.5$ and $10^{-3} \le q_5 \le 1$ the onset times run from 6.9078 (for $q_5 = 10^{-3}$) down to 0.0941 (for $q_5 = 1$): as time goes on, smaller and smaller extra-time momenta cross into the growing, Krein-neutral regime, and only the good sector keeps real frequencies for ever.

The good sector along the history. For $q_5 = 0$ the mode Hamiltonian is Hermitian and commutes with B at every time, with the energies

$$\pm E(x_4), \qquad E(x_4) = \sqrt{m^2 + k_1^2} = \sqrt{m^2 + q_1^2 e^{-2AHx_4 - 2Hy}},$$

which fall towards m as 3-space inflates. At each instant the positive Fock space of Section 10.20 can be built with the coefficients of that instant (an **instantaneous** Fock space): the filled sea has the value $-8E(x_4)$, and every particle and antiparticle the energy $+E(x_4) > 0$. For example, for $q_1 = 1$ and $y = 0$, $E = \sqrt{2} = 1.414214$ at $a_{4,0} = 0$ and $E = \sqrt{1 + e^{-4}} = 1.009116$ at $a_{4,0} = 2$. Whether the instantaneous vacuum of one time contains quanta of a later time (the evolution of the quantum state through the changing background) is not computed: OPEN.

The universes of masses $+m$ and $-m$ along the history. At every instant $\Gamma h_m(x_4)\Gamma = h_{-m}(x_4)$ (Section 10.6; Γ anticommutes with $\gamma^{(x_4)}$ and commutes with $\gamma^{(x_4)}\gamma^{(x_1)}$ and $\gamma^{(x_4)}\gamma^{(x_5)}$), and w^2 contains the mass only as m^2: the two universes have the same frequencies, the same onset times and the same Krein inertia at every instant.

statement	status	where it is verified
the history $a_4 = AHx_4$, $A = H = m = 1$	ASSUMED (PRESCRIBED BACKGROUND)	`parameters.json`; `ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`
the scale factors $e^{\pm a_4}\sin^{1/6} z$, 1, $\cot z$	from the record	`Revision/theory/field-theory.json`, formula `vielbein_diagonal`
local-frame model (coefficients of one instant and position, no hidden-direction terms)	ASSUMED	`python-field-theory.json`, check `extra_time_modes_grow` (a local-frame statement)
$h(x_4)^2 = w(x_4)^2 I_{16}$; $\|h_A\| = k_5$, $\|Bh - hB\| = 2k_5$; good sector Hermitian at all times	PROVED (exact, every instant)	Notebook 10f, In [4]; `python-field-theory.json`, check `good_sector_spectrum_and_B_sectors`
every wave with $q_5 \neq 0$ reaches the onset $x_4^* = \ln X^*/(2AH)$	PROVED (above)	Notebook 10f, In [6], check `the exact onset time agrees with bisection; q1 = 0, y = 0: ln(m / q5)` (agreement with bisection 8.9×10^{-16})
onset 2.996044 for $q_1 = 0.5$, $q_5 = 0.05$; inertia $(4, 4)$ before, neutral after; onset map	COMPUTED	Notebook 10f, In [6], In [9], In [10]; `python-pairing.json`, checks `Q.one_particle_Krein_inertia_proof` and `Q.one_particle_complex_frequency_Krein_neutral`
instantaneous good-sector Fock space: sea $-8E(x_4)$, quanta $+E(x_4)$	COMPUTED at the five slices	Notebook 10f, In [11]

statement	status	where it is verified
evolution of the quantum state through the background; a positive state space for the extra-time sector	OPEN	not computed in the Revision record

10.40 Example: Notebook 10f follows the Krein structure along the deflating history

Notebook 10f reads the history and the scale factors from the Revision records, proves the identities of Section 10.39 exactly for every instant, compares the sizes of h_A and $Bh - hB$ with k_5 and $2k_5$, computes the onset time and checks it against a numerical root search for 48 waves, places the record's sample $k_5 = 2$ at $x_4 = \ln 40$, follows the eigenvalues and the Krein inertia of one wave through its onset, maps which waves still oscillate at which time, builds the instantaneous good-sector Fock space at the five slices of the Kohn-Sham record, and checks the masses $+m$ and $-m$ at every time. It draws five figures and ends with the line ALL 19 CHECKS PASSED (notebook 10f).

10.41 How to run Notebook 10f

Step 1. What this notebook does and what it needs.

Notebook 10f (The Krein structure along the deflating history) is the file `Revision/textbook/notebooks/10f_krein_along_history.ipynb` of the repository Dirac_claude. Along the prescribed deflating history a4 = A H x4 of the Revision Kohn-Sham record (A = H = m = 1) it takes the scale factors of the author's metric from the Revision theory record, turns the coordinate momenta of a wave along x1 and along the extra time x5 into frame momenta (the extra-time one grows like e to the a4 because the extra times deflate), proves exactly that the local mode Hamiltonian of every instant squares to w(x4)^2 times the identity and that its departure from Hermiticity and its mixing of the two sectors of B grow like e to the a4, computes the exact time at which a wave turns from oscillation to growth, follows the eigenvalues and the Krein inertia of the eigenspaces through that time ((4, 4) before, Krein-neutral after), maps which waves are still of real frequency at each time, shows that the good sector keeps a Hermitian mode Hamiltonian, the Krein inertia (4, 4) and a positive instantaneous energy at every time, checks that the masses +m and -m have the same onset, and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs

the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is

changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10f_krein_along_history.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10f_krein_along_history.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 10f_krein_along_history.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10f.captions.json`

- `Revision/textbook/figures/10f_1_hermiticity_defect.png`
- `Revision/textbook/figures/10f_2_eigenvalues_along_history.png`
- `Revision/textbook/figures/10f_3_inertia_along_history.png`
- `Revision/textbook/figures/10f_4_onset_map.png`
- `Revision/textbook/figures/10f_5_good_sector_energies.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10f_5_good_sector_energies.png exists
ALL 19 CHECKS PASSED (notebook 10f)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10f_krein_along_history.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10f_krein_along_history.ipynb
python -m nbconvert --execute --inplace 10f_krein_along_history.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.42 Notebook 10f: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10f_krein_along_history.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The earlier notebooks of this chapter studied one wave of dirac16complex with
momenta that do not change. In the author's metric they DO change: the three extra
times $x_5, x_6, x_7$ deflate exponentially (their scale factor is
$e^{-a_4}\sin^{1/6}z$, with $a_4$ increasing), and 3-space inflates (scale factor
```

$e^{a_4}\sin^{1/6}z$). A wave with a fixed coordinate momentum along an extra time therefore has a frame momentum that GROWS like e^{a_4}. This notebook follows the quantum structure of the chapter along the deflating history $a_4 = AHx_4$ of the Revision Kohn-Sham record. It

- reads the scale factors from the Revision theory record and turns coordinate momenta into frame momenta;
- proves exactly that the mode Hamiltonian $h(x_4)$ of every instant satisfies $h(x_4)^2 = w(x_4)^2I_{16}$, and that its departure from Hermiticity and its mixing of the two eigenspaces of B both grow like e^{a_4};
- computes the exact time at which a wave with extra-time momentum turns from oscillation into growth, and shows that EVERY such wave reaches it;
- follows the eigenvalues and the Krein inertia of the eigenspaces through that time: inertia $(4, 4)$ before, Krein-neutral after;
- maps, for a whole range of extra-time momenta, which waves still have a real frequency at each time;
- shows that the good sector (no extra-time momentum) keeps a Hermitian mode Hamiltonian, the inertia $(4, 4)$ and a positive energy for every quantum at every time, with an energy that falls towards m as 3-space inflates;
- checks that the masses $+m$ and $-m$ have the same onset of growth;
- draws five teaching figures.

Status of the inputs: the history $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND (it is not solved from the field equations for a_4), and every statement here is made in the *local-frame model*: at each instant the coefficients of the wave equation are evaluated at that instant and at one hidden position, as the Revision record does for its statement that every extra-time mode eventually grows.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 10f (The Krein structure along the deflating history) is the file ‘Revision/textbook/notebooks/10f_krein_along_history.ipynb‘ of the repository Dirac_claude. Along the prescribed deflating history a4 = A H x4 of the Revision Kohn-Sham record (A = H = m = 1) it takes the scale factors of the author’s metric from the Revision theory record, turns the coordinate momenta of a wave along x1 and along the extra time x5 into frame momenta (the extra-time one grows like e to the a4 because the extra times deflate), proves exactly that the local mode Hamiltonian of every instant squares to w(x4)^2 times the identity and that its departure from Hermiticity and its mixing of the two sectors of B grow like e to the a4, computes the exact time at which a wave turns from oscillation to growth, follows the eigenvalues and the Krein inertia of the eigenspaces through that time ((4, 4) before, Krein-neutral after), maps which waves are still of real frequency at each time, shows that the good sector keeps a Hermitian mode Hamiltonian, the Krein inertia (4, 4) and a positive instantaneous energy at every time, checks that the masses +m and -m have the same onset, and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy

and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already

active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10f_krein_along_history.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10f_krein_along_history.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10f_krein_along_history.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10f.captions.json‘
- ‘Revision/textbook/figures/10f_1_hermiticity_defect.png‘
- ‘Revision/textbook/figures/10f_2_eigenvalues_along_history.png‘
- ‘Revision/textbook/figures/10f_3_inertia_along_history.png‘
- ‘Revision/textbook/figures/10f_4_onset_map.png‘
- ‘Revision/textbook/figures/10f_5_good_sector_energies.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10f_5_good_sector_energies.png exists
ALL 19 CHECKS PASSED (notebook 10f)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10f_krein_along_history.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
    python -m jupyterlab 10f_krein_along_history.ipynb
    python -m nbconvert --execute --inplace 10f_krein_along_history.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
  the Revision record: the data it starts from and the report files whose checks it
  reproduces (each cited check must still exist there with the verdict PASS). It must be
  opened inside the folder Revision/textbook/notebooks of a complete copy of the
  repository (a notebook copied alone to another folder cannot find them). Clone the
  repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the
  Revision record no longer holds the check that the notebook names, or holds it with
  another verdict: the copy of the repository is incomplete or was changed. Clone the
  repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 10f (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 10f (The Krein structure along the deflating history) is the file
# Revision/textbook/notebooks/10f_krein_along_history.ipynb of the repository
# Dirac_claude. Along the prescribed deflating history a4 = A H x4 of the Revision
# Kohn-Sham record (A = H = m = 1) it takes the scale factors of the author’s metric from
# the Revision theory record, turns the coordinate momenta of a wave along x1 and along
# the extra time x5 into frame momenta (the extra-time one grows like e to the a4 because
# the extra times deflate), proves exactly that the local mode Hamiltonian of every
# instant squares to w(x4)^2 times the identity and that its departure from Hermiticity
# and its mixing of the two sectors of B grow like e to the a4, computes the exact time
# at which a wave turns from oscillation to growth, follows the eigenvalues and the Krein
# inertia of the eigenspaces through that time ((4, 4) before, Krein-neutral after), maps
# which waves are still of real frequency at each time, shows that the good sector keeps
# a Hermitian mode Hamiltonian, the Krein inertia (4, 4) and a positive instantaneous
# energy at every time, checks that the masses +m and -m have the same onset, and draws
# five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
```

```
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
# shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
```

```
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10f_krein_along_history.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10f_krein_along_history.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10f_krein_along_history.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10f.captions.json
# - Revision/textbook/figures/10f_1_hermiticity_defect.png
# - Revision/textbook/figures/10f_2_eigenvalues_along_history.png
# - Revision/textbook/figures/10f_3_inertia_along_history.png
# - Revision/textbook/figures/10f_4_onset_map.png
# - Revision/textbook/figures/10f_5_good_sector_energies.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
```

```
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 10f_5_good_sector_energies.png exists
#     ALL 19 CHECKS PASSED (notebook 10f)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10f_krein_along_history.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10f_krein_along_history.ipynb
#     python -m nbconvert --execute --inplace 10f_krein_along_history.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
```

```
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10f"  # this notebook: chapter 10, example f


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative
```

```
def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
```

```
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10f complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Scale factor** $f_a$: the factor by which the author's metric stretches the
  coordinate $x_a$; a coordinate distance $dx_a$ has the proper length $f_a\,dx_a$.
  For 3-space $f = e^{a_4}\sin^{1/6}z$, for the extra times $f = e^{-a_4}
  \sin^{1/6}z$.
- **Deflating history**: the time dependence $a_4 = AHx_4$ with $A = 1$; as $x_4$
  grows, 3-space inflates and the extra times deflate, both exponentially.
- **Coordinate momentum** $q_a$ and **frame momentum** $k_a$: a wave $e^{iq_ax_a}$
  has the coordinate momentum $q_a$; measured with proper lengths its momentum is
  $k_a = q_a/f_a$, the frame momentum.
- **Hidden position** $y$: the Kohn-Sham record measures the hidden direction by
  $y = \ln(\sin z)/(6H)$, so that $\sin^{1/6}z = e^{Hy}$; the patch end
  $z = \pi/2$ is $y = 0$, and $y < 0$ lies towards the tip.
- **Local-frame model**: at the instant $x_4$ and the position $y$ the wave equation
  is written with the coefficients of that instant and that position; the
  hidden-direction terms (studied in the notebook on the curved good sector) are
  left out.
- **Mode Hamiltonian** $h$: the $16 \times 16$ matrix with $i\,du/dx_4 = hu$; its
  eigenvalues $\pm w$ are the frequencies.
- **Onset time** $x_4^\ast$: the time at which $w^2$ changes from positive (the wave
  oscillates) to negative (the wave grows).
```

```
- **Hermitian part, anti-Hermitian part**: every matrix is $h = h_H + h_A$ with
  $h_H = \frac12(h + h^\dagger)$ Hermitian and $h_A = \frac12(h - h^\dagger)$
  anti-Hermitian ($h_A^\dagger = -h_A$).
- **Size of a matrix (spectral norm)**: the largest factor by which the matrix
  stretches the length of a column; in code `np.linalg.norm(M, 2)`.
- **Krein inertia, Krein-neutral**: as in the first notebook of this chapter, the
  numbers of positive, negative and zero eigenvalues of the Krein form $u^\dagger Bv$
  on a subspace; neutral means that the form vanishes on it.
- **Good sector**: waves without extra-time momentum, $q_5 = q_6 = q_7 = 0$.
- **Instantaneous**: computed with the coefficients of one instant, as if they did
  not change.
```

Text cell:

```
## 4. The physical and mathematical situation

**The wave equation of one instant, line by line.** The field equation of
dirac16complex in the author's metric (Revision theory record, $U = 0$) is

$$e^{-a_4}\sin^{-1/6}z\sum_{i=1}^{3}\gamma^{(x_i)}\partial_i\Psi + \gamma^{(x_4)}
\partial_4\Psi + e^{a_4}\sin^{-1/6}z\sum_{t=5}^{7}\gamma^{(x_t)}\partial_t\Psi +
\tan z\,\gamma^{(x_0)}\partial_0\Psi + 3H\gamma^{(x_0)}\Psi = m\Psi .$$

Each derivative $\partial_a$ is divided by the scale factor $f_a$ of its direction.
In the local-frame model take a wave $\Psi = u(x_4)\,e^{i(q_1x_1 + q_5x_5)}$ at one
hidden position and leave out the two hidden-direction terms. Then $\partial_1$
becomes $iq_1$ and $\partial_5$ becomes $iq_5$:

$$i k_1\gamma^{(x_1)}u + \gamma^{(x_4)}\frac{du}{dx_4} + ik_5\gamma^{(x_5)}u = mu,
\qquad k_1 = q_1e^{-a_4}\sin^{-1/6}z,\qquad k_5 = q_5e^{a_4}\sin^{-1/6}z .$$

Multiply from the left by $\gamma^{(x_4)}$, use $(\gamma^{(x_4)})^2 = -1$, solve for
$du/dx_4$ and multiply by $i$ (exactly the steps of the first notebook of this
chapter):

$$i\frac{du}{dx_4} = h(x_4)\,u,\qquad h(x_4) = -im\gamma^{(x_4)} -
k_1(x_4)\,\gamma^{(x_4)}\gamma^{(x_1)} - k_5(x_4)\,\gamma^{(x_4)}\gamma^{(x_5)} .$$

With $\sin^{1/6}z = e^{Hy}$ and $a_4 = AHx_4$ the frame momenta are
$k_1 = q_1e^{-AHx_4 - Hy}$, which SHRINKS (3-space inflates), and
$k_5 = q_5e^{AHx_4 - Hy}$, which GROWS (the extra times deflate).

**The squared frequency.** By the identity $h^2 = w^2I_{16}$ of the first notebook
of this chapter, at every instant

$$w(x_4)^2 = m^2 + k_1^2 - k_5^2 = m^2 + q_1^2e^{-2AHx_4 - 2Hy} -
q_5^2e^{2AHx_4 - 2Hy} .$$

For $A > 0$ the second term falls and the third grows in size, so $w^2$ falls all
the time; if $q_5 \neq 0$ it becomes negative after a finite time.

**The onset time, line by line.** Write $X = e^{2AHx_4} > 0$ and $c = e^{-2Hy}$.
The condition $w^2 = 0$ reads $m^2 + q_1^2c/X - q_5^2cX = 0$. Multiply by $X$:
```

```
$$q_5^2c\,X^2 - m^2X - q_1^2c = 0 .$$

This is a quadratic equation for $X$; its only positive root is

$$X^\ast = \frac{m^2 + \sqrt{m^4 + 4q_1^2q_5^2c^2}}{2q_5^2c},\qquad
x_4^\ast = \frac{\ln X^\ast}{2AH} .$$

(The other root has the minus sign in front of the square root; it is negative, or
zero when $q_1 = 0$, because the square root is at least $m^2$, while
$X = e^{2AHx_4}$ is always positive.) For $q_1 = 0$ and $y = 0$ this is
$X^\ast = m^2/q_5^2$, so $x_4^\ast = \ln(m/q_5)/(AH)$: the onset comes when the
growing frame momentum $k_5$ reaches the mass.

**Hermitian and anti-Hermitian parts.** The matrices $-i\gamma^{(x_4)}$ and
$\gamma^{(x_4)}\gamma^{(x_1)}$ are Hermitian and commute with $B$, while
$\gamma^{(x_4)}\gamma^{(x_5)}$ is anti-Hermitian and anticommutes with $B$ (first
notebook of this chapter). Hence $h_A = -k_5\gamma^{(x_4)}\gamma^{(x_5)}$, and since
$(\gamma^{(x_4)}\gamma^{(x_5)})^2 = -(\gamma^{(x_4)})^2(\gamma^{(x_5)})^2 = -I$,

$$h_A^\dagger h_A = -h_A^2 = k_5^2 I_{16} .$$

So $h_A$ stretches every column by exactly the factor $k_5$: the size of the
anti-Hermitian part is $k_5 = q_5e^{AHx_4 - Hy}$. In the same way
$Bh - hB = 2Bh_A$ has the size $2k_5$ ($B$ keeps lengths, because $B^\dagger B =
B^2 = I$). Both grow exponentially with the deflation; both vanish at every time in
the good sector, $q_5 = 0$.

**Krein inertia at each instant.** At each instant $h(x_4)$ is the mode Hamiltonian
of the first notebook of this chapter with real frame momenta, so the Revision
proof applies instant by instant: while $w^2 > 0$ each eigenspace has Krein inertia
$(4, 4)$; once $w^2 < 0$ each eigenspace is Krein-neutral.
```

Text cell:

```
## 5. The deflating history of the Revision record

The next cell reads the parameters of the Revision Kohn-Sham record: the constant
$H$, the mass $m$, the history constant $A$ and the five slices $a_{4,0}$ at which
that record computes its instantaneous Kohn-Sham states. It prints the first
sentence of the record's own statement of the status of the history and checks the
values $A = H = m = 1$. It also defines ‘check_record‘ (the PASS line and the line
naming the reproduced Revision record are printed in one piece, because Jupyter
sends printed text to the screen in pieces whose boundaries depend on timing) and
the colours of the figures.
Before it checks anything, ‘check_record‘ asks the helper ‘record_says_pass‘ whether
the cited record still says PASS: for a record name of the form
‘<report file>, check <check name>‘ it opens that report (each file once, kept in
the dictionary ‘REPORT_CHECKS‘) and stops the notebook with an error unless a check
of that name exists there with the verdict PASS; a record name without ‘, check ‘
names a data entry that the notebook reads and compares itself.
```

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
H = parameters["physics"]["H"]  # the author's constant H
mass = parameters["physics"]["m"]  # the mass m
A = parameters["physics"]["historyA"]  # the history a4 = A H x4
slices = parameters["physics"]["slicesA4"]  # the values a4,0 of the record
say("status of the history: "
    + parameters["conventions"]["history"].split(": ")[0] + ".")
say(f"slices a4,0 of the Kohn-Sham record: {slices}")
check_record(A == 1.0 and H == 1.0 and mass == 1.0,
             "the deflating history a4 = A H x4 with A = 1, H = 1, m = 1",
             record="Revision/kohn_sham/results/parameters.json, physics.historyA, "
                    "physics.H, physics.m")


def a4_of(x4):
    """The prescribed deflating history a4 = A H x4."""
    return A * H * x4
```

Out [2]:

```
status of the history: PRESCRIBED BACKGROUND.
slices a4,0 of the Kohn-Sham record: [0.0, 0.5, 1.0, 1.5, 2.0]
PASS the deflating history a4 = A H x4 with A = 1, H = 1, m = 1
     reproduces Revision/kohn_sham/results/parameters.json, physics.historyA, physics.H,
    physics.m
```

Text cell:

```
## 6. Frame momenta from the scale factors of the record

The Revision theory record ‘Revision/theory/field-theory.json‘ stores the scale
factors $f_a$ of the eight directions (formula key ‘vielbein_diagonal‘) in the
notation of the Wolfram Language, for example ‘E^a4[x4]*Sin[6*H*x0]^(1/6)‘. The
next cell translates this text into sympy (‘E^a4[x4]‘ becomes ‘exp(a4)‘,
‘Sin[6*H*x0]‘ becomes ‘sin(z)‘, ‘^‘ becomes ‘**‘, curly brackets become square
brackets), and checks EXACTLY that it equals $e^{a_4}\sin^{1/6}z$ for $x_1, x_2,
x_3$, 1 for $x_4$, $e^{-a_4}\sin^{1/6}z$ for the extra times and $\cot z$ for
$x_0$. Then it defines the frame momenta $k_1 = q_1/f_1$ and $k_5 = q_5/f_5$ at
the time $x_4$ and the hidden position $y$ (with $\sin^{1/6}z = e^{Hy}$).
```

In [3]:

```
theory = json.loads(repository_file("Revision/theory/field-theory.json").read_text(
    encoding="utf-8"))
text = next(f["wl"] for f in theory["formulas"] if f["key"] == "vielbein_diagonal")
a4_symbol, z_symbol = sp.symbols("a4 z", real=True)
translated = (text.replace("E^a4[x4]", "exp(a4)").replace("Sin[6*H*x0]", "sin(z)")
              .replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
              .replace("{", "[").replace("}", "]"))
f_record = sp.sympify(translated, locals={"a4": a4_symbol, "z": z_symbol})
sixth_root = sp.sin(z_symbol) ** sp.Rational(1, 6)  # sin(z)^(1/6)
f_expected = ([sp.cot(z_symbol)] + [sp.exp(a4_symbol) * sixth_root] * 3
              + [sp.Integer(1)] + [sp.exp(-a4_symbol) * sixth_root] * 3)  # x0, ..., x7
same = all(sp.simplify(r - e) == 0 for r, e in zip(f_record, f_expected))
say(f"scale factor of x1: {f_record[1]}")
say(f"scale factor of x5: {f_record[5]}")
check_record(len(f_record) == 8 and same,
             "the record's scale factors: e^a4 sin^(1/6) z (3-space), e^-a4 ... (x5-x7)",
             record="Revision/theory/field-theory.json, formula vielbein_diagonal")


def frame_momenta(x4, q1, q5, y):
    """(k_1, k_5) of the wave exp(i (q1 x1 + q5 x5)) at time x4 and position y."""
    root = np.exp(H * y)  # sin(z)^(1/6) = e^(H y)
    k1 = q1 / (np.exp(a4_of(x4)) * root)  # q1 / f_1: shrinks, 3-space inflates
    k5 = q5 / (np.exp(-a4_of(x4)) * root)  # q5 / f_5: grows, extra times deflate
    return k1, k5
```

Out [3]:

```
scale factor of x1: exp(a4)*sin(z)**(1/6)
scale factor of x5: exp(-a4)*sin(z)**(1/6)
```

```
PASS the record's scale factors: e^a4 sin^(1/6) z (3-space), e^-a4 ... (x5-x7)
     reproduces Revision/theory/field-theory.json, formula vielbein_diagonal
```

Text cell:

```
## 7. The mode Hamiltonian of each instant, exactly

The next cell reads the gammas, builds $B$ and the chirality $\Gamma$, and defines
the mode Hamiltonian `mode_hamiltonian(x4, q1, q5, y, m)` of Section 4 as a numpy
matrix. Then it repeats the algebra of Section 4 EXACTLY with sympy, with symbols
for $m$, $q_1$, $q_5$, $H$, $y$ and the time $x_4$ (so the statements hold at every
instant and every position):

- $h(x_4)^2 = (m^2 + q_1^2e^{-2a_4 - 2Hy} - q_5^2e^{2a_4 - 2Hy})I_{16}$;
- the anti-Hermitian part is $h_A = -k_5\gamma^{(x_4)}\gamma^{(x_5)}$ and
  $h_A^\dagger h_A = k_5^2I_{16}$;
- $Bh - hB = 2Bh_A$;
- in the good sector ($q_5 = 0$) $h$ is Hermitian and commutes with $B$ at every
  time.
```

In [4]:

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])  # the chirality matrix


def mode_hamiltonian(x4, q1, q5, y, m=mass):
    """h(x4) = -i m g4 - k1 g4 g1 - k5 g4 g5 with the frame momenta of time x4."""
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return (-1j * m * gamma[4] - k1 * (gamma[4] @ gamma[1])
            - k5 * (gamma[4] @ gamma[5]))


def w_squared(x4, q1, q5, y, m=mass):
    """w^2 = m^2 + k1^2 - k5^2 at the time x4."""
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return m ** 2 + k1 ** 2 - k5 ** 2


g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
B_exact = -sp.I * g[0] * g[1] * g[2] * g[3] * g[4]
m_s, q1_s, q5_s, y_s = sp.symbols("m q1 q5 y", real=True)
x4_s, H_s = sp.symbols("x4 H", positive=True)
a4_s = A * H_s * x4_s  # the history, with the record's A = 1
k1_s = q1_s * sp.exp(-a4_s - H_s * y_s)  # frame momentum along x1
k5_s = q5_s * sp.exp(a4_s - H_s * y_s)  # frame momentum along x5
h_s = -sp.I * m_s * g[4] - k1_s * g[4] * g[1] - k5_s * g[4] * g[5]
```

```
zero = sp.zeros(16, 16)
w2_s = m_s ** 2 + k1_s ** 2 - k5_s ** 2
square_ok = (h_s * h_s - w2_s * sp.eye(16)).applyfunc(sp.expand) == zero
h_A = ((h_s - h_s.H) / 2).applyfunc(sp.expand)  # the anti-Hermitian part
part_ok = (h_A + k5_s * g[4] * g[5]).applyfunc(sp.expand) == zero
size_ok = (h_A.H * h_A - k5_s ** 2 * sp.eye(16)).applyfunc(sp.expand) == zero
mixing_ok = (B_exact * h_s - h_s * B_exact - 2 * B_exact * h_A).applyfunc(
    sp.expand) == zero
good = h_s.subs(q5_s, 0)  # the good sector
good_ok = ((good - good.H).applyfunc(sp.expand) == zero
           and (B_exact * good - good * B_exact).applyfunc(sp.expand) == zero)
check(square_ok, "exact: h(x4)^2 = (m^2 + k1^2 - k5^2) I at every instant")
check(part_ok and size_ok and mixing_ok,
      "exact: h_A = -k5 g4 g5, h_A^dagger h_A = k5^2 I, B h - h B = 2 B h_A")
check_record(good_ok,
             "exact: good sector, h(x4) Hermitian and commuting with B at all times",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")
```

Out [4]:

```
PASS exact: h(x4)^2 = (m^2 + k1^2 - k5^2) I at every instant
PASS exact: h_A = -k5 g4 g5, h_A^dagger h_A = k5^2 I, B h - h B = 2 B h_A
PASS exact: good sector, h(x4) Hermitian and commuting with B at all times
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_spectrum_and_B_sectors
```

Text cell:

```
## 8. Departure from Hermiticity and mixing of the two sectors of $B$

The next cell measures, with numbers, the size of the anti-Hermitian part $h_A$ and
of the commutator $Bh - hB$ (‘np.linalg.norm(M, 2)‘, the largest stretching factor)
for three extra-time momenta $q_5 = 0.01$, $0.05$, $0.2$ (with $q_1 = 0.5$, $y = 0$)
at the times $x_4 = 0, 0.5, \dots, 6$, compares them with the formulas $k_5 =
q_5e^{a_4}$ and $2k_5$ of Section 4, and draws them on a logarithmic scale, where an
exponential is a straight line. In the good sector both sizes are 0 at every time.
```

In [5]:

```
times = np.linspace(0.0, 6.0, 13)  # 0, 0.5, ..., 6
curve_times = np.linspace(0.0, 6.0, 241)
measured, worst = {}, 0.0
for q5 in (0.01, 0.05, 0.2):
    sizes = []
    for x4 in times:
        h = mode_hamiltonian(x4, 0.5, q5, 0.0)
        h_anti = (h - h.conj().T) / 2
        sizes.append((np.linalg.norm(h_anti, 2), np.linalg.norm(B @ h - h @ B, 2)))
        k5 = frame_momenta(x4, 0.5, q5, 0.0)[1]
        worst = max(worst, abs(sizes[-1][0] - k5) / k5,
```

```
                    abs(sizes[-1][1] - 2 * k5) / (2 * k5))
    measured[q5] = np.array(sizes)
good_sizes = [np.linalg.norm(mode_hamiltonian(x4, 0.5, 0.0, 0.0)
                             - mode_hamiltonian(x4, 0.5, 0.0, 0.0).conj().T, 2)
              for x4 in times]
report("largest relative difference from k5 and 2 k5", f"{worst:.1e}")
check(worst < 1e-12 and max(good_sizes) == 0.0,
      "sizes of h_A and of B h - h B are k5 = q5 e^a4 and 2 k5; zero if q5 = 0")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
for q5, colour in ((0.01, BLUE), (0.05, ORANGE), (0.2, GREEN)):
    k5_curve = frame_momenta(curve_times, 0.5, q5, 0.0)[1]
    axes[0].plot(curve_times, k5_curve, color=colour, linewidth=2,
                 label=f"$q_5 = {q5}$")
    axes[0].plot(times, measured[q5][:, 0], "o", color=colour, markersize=5)
    axes[1].plot(curve_times, 2 * k5_curve, color=colour, linewidth=2,
                 label=f"$q_5 = {q5}$")
    axes[1].plot(times, measured[q5][:, 1], "o", color=colour, markersize=5)
for ax in axes:
    ax.set_yscale("log")
    ax.axhline(mass, color=GREY, linestyle=":", linewidth=1)
    ax.set_xlabel("time $x_4$ (units of $1/m$)")
    ax.legend(loc="lower right", fontsize=8)
axes[0].set_ylabel("size (units of $m$)")
axes[0].set_title("anti-Hermitian part $h_A$: size $k_5 = q_5e^{a_4}$")
axes[1].set_title("mixing of the $B$ sectors: size of $Bh - hB$")
save_figure(fig, "hermiticity_defect",
            "Along the deflating history $a_4 = x_4$ ($A = H = m = 1$, hidden "
            "position $y = 0$), for a wave with the coordinate momenta $q_1 = 0.5$ "
            "and $q_5 = 0.01$, $0.05$, $0.2$ (blue, orange, green): left, the size "
            "of the anti-Hermitian part of the mode Hamiltonian; right, the size of "
            "the commutator $Bh - hB$, which mixes the two eigenspaces of $B$. "
            "Horizontal axes: the time $x_4$ in units of $1/m$; vertical axes: the "
            "size in units of $m$, logarithmic. Dots: measured matrices; lines: "
            "the formulas $k_5 = q_5e^{a_4}$ and $2k_5$; dotted grey line: the "
            "mass. The straight lines show exponential growth: the deflation of "
            "the extra times drives every wave with extra-time momentum ever "
            "further from a Hermitian, sector-preserving evolution. In the good "
            "sector, $q_5 = 0$, both sizes are 0 at every time.")
```

Out [5]:

```
RESULT largest relative difference from k5 and 2 k5 = 0.0e+00
PASS sizes of h_A and of B h - h B are k5 = q5 e^a4 and 2 k5; zero if q5 = 0
```

Out [5]:

```
Figure 10f.1 saved as Revision/textbook/figures/10f_1_hermiticity_defect.png
```

Text cell:

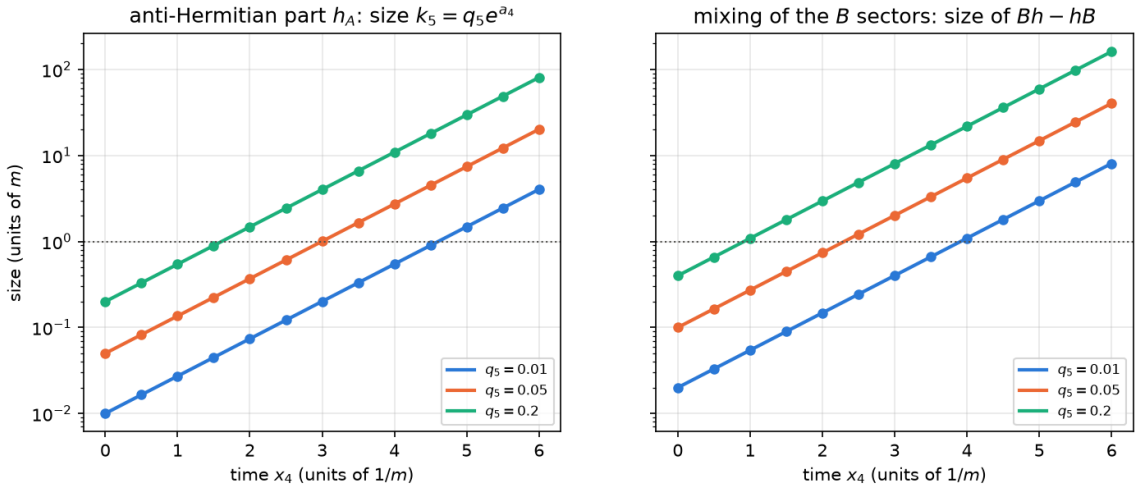

Figure 293: Along the deflating history $a_4 = x_4$ ($A = H = m = 1$, hidden position $y = 0$), for a wave with the coordinate momenta $q_1 = 0.5$ and $q_5 = 0.01$, 0.05, 0.2 (blue, orange, green): left, the size of the anti-Hermitian part of the mode Hamiltonian; right, the size of the commutator $Bh - hB$, which mixes the two eigenspaces of B. Horizontal axes: the time x_4 in units of $1/m$; vertical axes: the size in units of m, logarithmic. Dots: measured matrices; lines: the formulas $k_5 = q_5 e^{a_4}$ and $2k_5$; dotted grey line: the mass. The straight lines show exponential growth: the deflation of the extra times drives every wave with extra-time momentum ever further from a Hermitian, sector-preserving evolution. In the good sector, $q_5 = 0$, both sizes are 0 at every time. (Notebook 10f, figure 1.)

```
## 9. One wave through the onset: eigenvalues and Krein inertia

The next cell defines the exact onset time of Section 4 and checks it against a
numerical root search for 48 waves (four values of $q_1$, four of $q_5$, three hidden
positions $y$). The search is *bisection*: start with an interval of times in which
$w^2$ changes sign and halve it 200 times, always keeping the half in which the sign
changes. It also checks the short form $x_4^\ast = \ln(m/q_5)$ for $q_1 = 0$,
$y = 0$.
```

In [6]:

```
def onset_time(q1, q5, y):
    """The exact time at which w^2 = 0 (the positive root of the quadratic)."""
    c = np.exp(-2 * H * y)
    X = (mass ** 2 + np.sqrt(mass ** 4 + 4 * q1 ** 2 * q5 ** 2 * c ** 2)) / (
        2 * q5 ** 2 * c)
    return np.log(X) / (2 * A * H)


def bisection(q1, q5, y, low=-40.0, high=40.0):
    """The time at which w^2 changes sign, by halving [low, high] 200 times."""
    for _ in range(200):
        middle = (low + high) / 2
        if w_squared(middle, q1, q5, y) > 0:  # still oscillating: the onset is later
            low = middle
        else:
```

```
            high = middle
    return (low + high) / 2


worst = max(abs(onset_time(q1, q5, y) - bisection(q1, q5, y))
            for q1 in (0.0, 0.5, 1.0, 3.0) for q5 in (0.01, 0.05, 0.1, 0.4)
            for y in (0.0, -1.0, -3.0))
report("largest difference, exact onset time minus bisection (48 waves)",
       f"{worst:.1e}")
check(worst < 1e-12 and abs(onset_time(0.0, 0.05, 0.0) - np.log(1 / 0.05)) < 1e-14,
      "the exact onset time agrees with bisection; q1 = 0, y = 0: ln(m / q5)")
onset = onset_time(0.5, 0.05, 0.0)  # the wave followed below
report("onset time of the wave q1 = 0.5, q5 = 0.05, y = 0", f"{onset:.6f}")
```

Out [6]:

```
RESULT largest difference, exact onset time minus bisection (48 waves) = 8.9e-16
PASS the exact onset time agrees with bisection; q1 = 0, y = 0: ln(m / q5)
RESULT onset time of the wave q1 = 0.5, q5 = 0.05, y = 0 = 2.996044
```

Text cell:

```
The Revision theory record names the exact sample $m = 1$ with the frame momentum
$k_5 = 2$ (and no other momentum): the eigenvalues are $\pm i\sqrt3$, eight of each.
Along the history this sample is an INSTANT: a wave with $q_1 = 0$ and $q_5 = 0.05$
at $y = 0$ has $k_5 = 0.05\,e^{x_4} = 2$ at $x_4 = \ln 40$. The next cell checks
that the mode Hamiltonian of that instant has exactly the recorded eigenvalues, and
that the same wave at $x_4 = 0$ (where $k_5 = 0.05$) still oscillates.
```

In [7]:

```
instant = np.log(40.0)  # the time at which k5 = 0.05 e^x4 = 2
k5_instant = frame_momenta(instant, 0.0, 0.05, 0.0)[1]
eigenvalues = np.linalg.eigvals(mode_hamiltonian(instant, 0.0, 0.05, 0.0))
plus = int(np.sum(np.abs(eigenvalues - 1j * np.sqrt(3)) < 1e-9))
minus = int(np.sum(np.abs(eigenvalues + 1j * np.sqrt(3)) < 1e-9))
early = np.linalg.eigvals(mode_hamiltonian(0.0, 0.0, 0.05, 0.0))
report("frame momentum k5 at x4 = ln 40", f"{k5_instant:.12f}")
report("eigenvalues +i sqrt(3) and -i sqrt(3), how many", f"{plus} and {minus}")
largest_imaginary = np.max(np.abs(early.imag))  # 0 up to rounding: oscillation
report("largest imaginary part of an eigenvalue at x4 = 0", f"{largest_imaginary:.1e}")
check_record(abs(k5_instant - 2) < 1e-12 and (plus, minus) == (8, 8)
             and largest_imaginary < 1e-12,
             "the recorded sample m = 1, k5 = 2 (+-i sqrt 3) is reached at x4 = ln 40",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "extra_time_modes_grow")
```

Out [7]:

```
RESULT frame momentum k5 at x4 = ln 40 = 2.000000000000
RESULT eigenvalues +i sqrt(3) and -i sqrt(3), how many = 8 and 8
RESULT largest imaginary part of an eigenvalue at x4 = 0 = 1.8e-16
PASS the recorded sample m = 1, k5 = 2 (+-i sqrt 3) is reached at x4 = ln 40
     reproduces Revision/theory/reports/python-field-theory.json, check
    extra_time_modes_grow
```

Text cell:

```
The next cell follows the wave $q_1 = 0.5$, $q_5 = 0.05$ at $y = 0$ from $x_4 = 0$ to
$x_4 = 5$: it computes the 16 eigenvalues of $h(x_4)$ at 251 times, compares them
with $\pm w(x_4)$ (eight each) away from the onset (where the matrix cannot be
diagonalised and numerical eigenvalues are inaccurate), and draws their real and
imaginary parts.
```

In [8]:

```
scan_times = np.linspace(0.0, 5.0, 251)
real_parts, imaginary_parts, worst = [], [], 0.0
for x4 in scan_times:
    eigenvalues = np.linalg.eigvals(mode_hamiltonian(x4, 0.5, 0.05, 0.0))
    real_parts.append(np.sort(eigenvalues.real))
    imaginary_parts.append(np.sort(eigenvalues.imag))
    w2 = w_squared(x4, 0.5, 0.05, 0.0)
    if abs(w2) > 0.05:  # away from the onset
        w = np.sqrt(complex(w2))  # real, or i kappa after the onset
        predicted = np.array([w] * 8 + [-w] * 8)
        worst = max(worst,
                    np.max(np.abs(np.sort(eigenvalues.real) - np.sort(predicted.real))),
                    np.max(np.abs(np.sort(eigenvalues.imag) - np.sort(predicted.imag))))
report("largest difference from +-w(x4) away from the onset", f"{worst:.1e}")
check(worst < 1e-10, "at every instant the eigenvalues of h(x4) are +-w(x4), 8 each")

fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0), sharex=True)
axes[0].plot(scan_times, np.array(real_parts), color=BLUE, linewidth=2)
axes[1].plot(scan_times, np.array(imaginary_parts), color=ORANGE, linewidth=2)
for ax, what in ((axes[0], "real part"), (axes[1], "imaginary part")):
    ax.axvline(onset, color=GREY, linestyle=":", linewidth=1)
    ax.set_xlabel("time $x_4$ (units of $1/m$)")
    ax.set_ylabel(f"{what} of the eigenvalues (units of $m$)")
axes[0].set_title("oscillation before the onset")
axes[1].set_title("growth after the onset")
save_figure(fig, "eigenvalues_along_history",
            "The 16 eigenvalues of the mode Hamiltonian $h(x_4)$ of a wave with the "
            "coordinate momenta $q_1 = 0.5$, $q_5 = 0.05$ at $y = 0$ along the "
            "deflating history $a_4 = x_4$ ($A = H = m = 1$): real parts (left, "
            "blue) and imaginary parts (right, orange), each curve carrying eight "
            "eigenvalues; horizontal axes: the time $x_4$ in units of $1/m$, "
            "vertical axes in units of $m$. Before the onset time $x_4^\\ast = "
            "2.996$ (dotted line) the eigenvalues are the real frequencies "
            "$\\pm w(x_4)$, which shrink as the extra-time frame momentum grows; at "
            "the onset they meet at 0 and then move onto the imaginary axis as "
            "$\\pm i\\kappa(x_4)$, and the growth rate $\\kappa$ keeps increasing.")
```

Out [8]:

```
RESULT largest difference from +-w(x4) away from the onset = 8.0e-15
PASS at every instant the eigenvalues of h(x4) are +-w(x4), 8 each
```

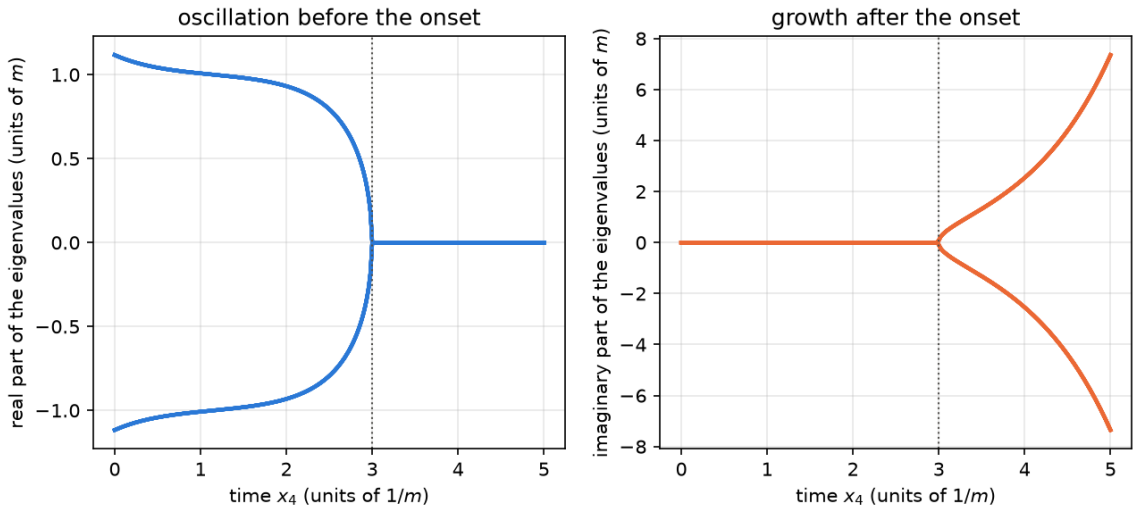

Figure 294: The 16 eigenvalues of the mode Hamiltonian $h(x_4)$ of a wave with the coordinate momenta $q_1 = 0.5$, $q_5 = 0.05$ at $y = 0$ along the deflating history $a_4 = x_4$ ($A = H = m = 1$): real parts (left, blue) and imaginary parts (right, orange), each curve carrying eight eigenvalues; horizontal axes: the time x_4 in units of $1/m$, vertical axes in units of m. Before the onset time $x_4^* = 2.996$ (dotted line) the eigenvalues are the real frequencies $\pm w(x_4)$, which shrink as the extra-time frame momentum grows; at the onset they meet at 0 and then move onto the imaginary axis as $\pm i\kappa(x_4)$, and the growth rate κ keeps increasing. (Notebook 10f, figure 2.)

Out [8]:

```
Figure 10f.2 saved as Revision/textbook/figures/10f_2_eigenvalues_along_history.png
```

Text cell:

```
The next cell computes the Krein inertia of the eigenspace of $+w$ (for $w^2 < 0$:
of $+i\kappa$) at the same 251 times, with the projector and Gram-matrix method of
the first notebook of this chapter, skipping the times closer than 0.03 to the
onset. The Revision proof says: $(4, 4, 0)$ at every time before the onset,
$(0, 0, 8)$ (neutral) at every time after it. The cell checks this and draws it.
```

In [9]:

```
def orthonormal_basis(P):
    """Orthonormal columns that span the range of P (Gram-Schmidt, done twice)."""
    basis = []
    for column in P.T:
```

```
        v = column.astype(complex)
        for _ in range(2):  # the second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T


def eigenspace(h, w2, sign):
    """Orthonormal basis of the eigenspace of sign * w of h (w = sqrt(w2))."""
    w = np.sqrt(complex(w2))
    return orthonormal_basis((np.eye(16) + sign * h / w) / 2)


def krein_inertia(V):
    """(positive, negative, zero) eigenvalues of the Gram matrix V^dagger B V."""
    values = np.linalg.eigvalsh(V.conj().T @ B @ V)
    return (int(np.sum(values > 1e-8)), int(np.sum(values < -1e-8)),
            int(np.sum(np.abs(values) <= 1e-8)))


kept = [x4 for x4 in scan_times if abs(x4 - onset) > 0.03]
inertias = [krein_inertia(eigenspace(mode_hamiltonian(x4, 0.5, 0.05, 0.0),
                                     w_squared(x4, 0.5, 0.05, 0.0), +1))
            for x4 in kept]
before = {c for x4, c in zip(kept, inertias) if x4 < onset}
after = {c for x4, c in zip(kept, inertias) if x4 > onset}
say(f"inertia before the onset: {sorted(before)}; after the onset: {sorted(after)}")
check_record(before == {(4, 4, 0)},
             "every instant before the onset: inertia (4, 4) (real frequency)",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_Krein_inertia_proof")
check_record(after == {(0, 0, 8)},
             "every instant after the onset: the eigenspace is Krein-neutral",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_complex_frequency_Krein_neutral")

counts = np.array(inertias)
fig, ax = plt.subplots()
width = 0.0205  # a little wider than the step 0.02 between the times (no gaps)
ax.bar(kept, counts[:, 0], width=width, color=BLUE,
       label="positive charge $u^\\dagger B u > 0$")
ax.bar(kept, counts[:, 1], width=width, bottom=counts[:, 0], color=ORANGE,
       label="negative charge $u^\\dagger B u < 0$")
ax.bar(kept, counts[:, 2], width=width, bottom=counts[:, 0] + counts[:, 1],
       color=GREEN, label="neutral")
ax.axvline(onset, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("directions in the $+w$ eigenspace")
ax.set_yticks(range(0, 9))
ax.set_ylim(0.0, 10.5)  # room for the legend above the bars
ax.set_title("Krein inertia of one wave along the deflating history")
```

```
ax.legend(loc="upper center", ncol=3, fontsize=8)
save_figure(fig, "inertia_along_history",
            "The Krein inertia of the eigenspace of $+w$ of the mode Hamiltonian "
            "of the wave $q_1 = 0.5$, $q_5 = 0.05$ ($y = 0$) at each time $x_4$ of "
            "the deflating history $a_4 = x_4$ (horizontal axis, units of $1/m$). "
            "Each thin bar splits the eight directions of the eigenspace (vertical "
            "axis) into positive charge (blue), negative charge (orange) and "
            "neutral (green). Before the onset time $x_4^\\ast = 2.996$ (dotted "
            "line) the split is 4 and 4 at every instant; after it the whole "
            "eigenspace is neutral. The deflation of the extra times moves the wave "
            "from the first kind to the second at a definite time.")
```

Out [9]:

```
inertia before the onset: [(4, 4, 0)]; after the onset: [(0, 0, 8)]
PASS every instant before the onset: inertia (4, 4) (real frequency)
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_Krein_inertia_proof
PASS every instant after the onset: the eigenspace is Krein-neutral
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_complex_frequency_Krein_neutral
```

Out [9]:

```
Figure 10f.3 saved as Revision/textbook/figures/10f_3_inertia_along_history.png
```

Text cell:

```
## 10. Every extra-time wave becomes Krein-neutral: the onset map

For $q_1 = 0.5$ and $y = 0$, the next cell computes the sign of $w^2$ on a grid of
241 times from 0 to 6 and 121 extra-time momenta $q_5$ from $10^{-3}$ to 1 (equally
spaced in $\log_{10} q_5$), checks that the sign agrees everywhere with the exact
onset time ($w^2 > 0$ exactly when $x_4 < x_4^\ast(q_5)$), computes the Krein
inertia directly at the grid points of every fourth momentum and every 60th time
(skipping those closer than 0.05 to the onset), and draws the map with the
onset curve and the five slices $a_{4,0}$ of the Kohn-Sham record. For every
$q_5 > 0$ the onset time is finite: no wave with extra-time momentum keeps a real
frequency for ever.
```

In [10]:

```
map_times = np.linspace(0.0, 6.0, 241)
exponents = np.linspace(-3.0, 0.0, 121)  # log10 of q5
q5_values = 10.0 ** exponents
real_frequency = np.array([[w_squared(x4, 0.5, q5, 0.0) > 0 for x4 in map_times]
                           for q5 in q5_values])  # rows: q5, columns: x4
onsets = np.array([onset_time(0.5, q5, 0.0) for q5 in q5_values])
agree = np.array_equal(real_frequency, map_times[None, :] < onsets[:, None])
```

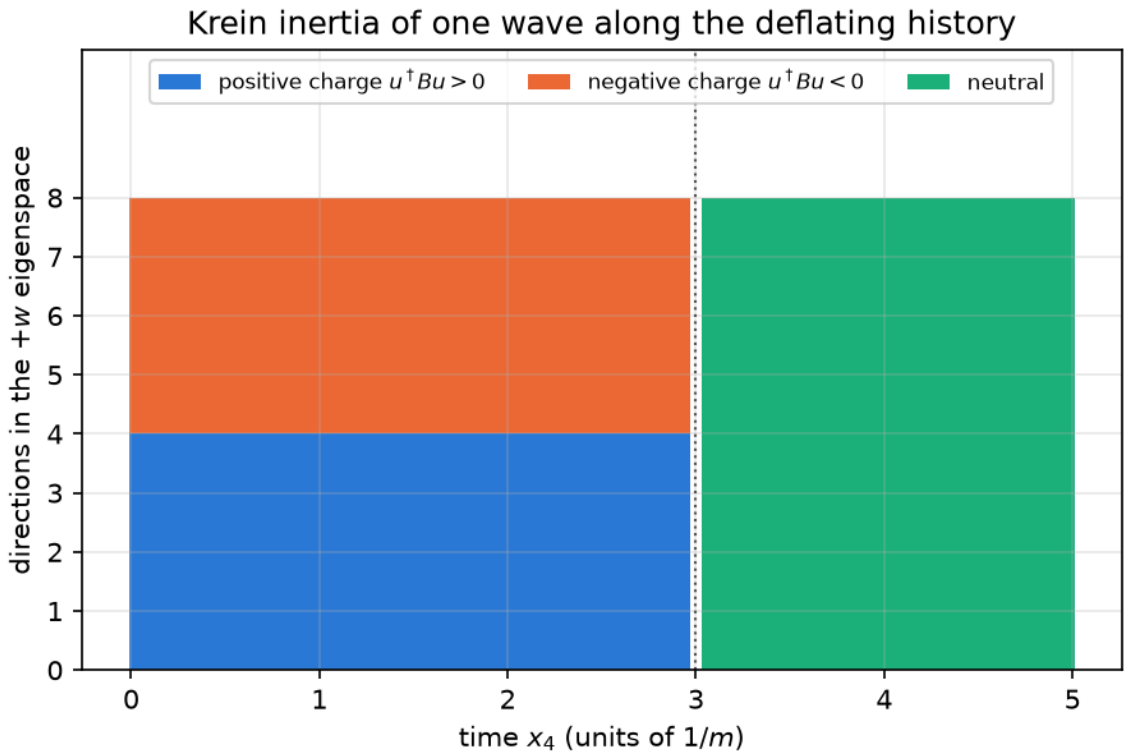

Figure 295: The Krein inertia of the eigenspace of $+w$ of the mode Hamiltonian of the wave $q_1 = 0.5$, $q_5 = 0.05$ ($y = 0$) at each time x_4 of the deflating history $a_4 = x_4$ (horizontal axis, units of $1/m$). Each thin bar splits the eight directions of the eigenspace (vertical axis) into positive charge (blue), negative charge (orange) and neutral (green). Before the onset time $x_4^* = 2.996$ (dotted line) the split is 4 and 4 at every instant; after it the whole eigenspace is neutral. The deflation of the extra times moves the wave from the first kind to the second at a definite time. (Notebook 10f, figure 3.)

```
sampled, sample_ok = 0, True
for i in range(0, 121, 4):
    for j in range(0, 241, 60):
        x4, q5 = map_times[j], q5_values[i]
        if abs(x4 - onsets[i]) < 0.05:
            continue  # too close to the onset
        w2 = w_squared(x4, 0.5, q5, 0.0)
        found = krein_inertia(eigenspace(mode_hamiltonian(x4, 0.5, q5, 0.0), w2, +1))
        sample_ok = sample_ok and found == ((4, 4, 0) if w2 > 0 else (0, 0, 8))
        sampled += 1
report("grid points sampled for the inertia", sampled)
report("onset time for q5 = 0.001 and for q5 = 1",
       f"{onsets[0]:.4f} and {onsets[-1]:.4f}")
check(agree and sample_ok and np.all(np.isfinite(onsets)),
      "the map agrees with the exact onset; (4, 4) before it, neutral after it")

from matplotlib.colors import ListedColormap  # a colour scale with two colours

fig, ax = plt.subplots(figsize=(7.0, 4.8))
ax.pcolormesh(map_times, exponents, real_frequency.astype(float),
              cmap=ListedColormap(["#f6c9b5", "#bcd6f5"]), vmin=0, vmax=1,
              shading="nearest")
inside = onsets <= map_times[-1]  # the part of the onset curve inside the map
ax.plot(onsets[inside], exponents[inside], color=GREY, linewidth=2,
        label="onset time $x_4^\\ast$")
ax.set_xlim(map_times[0], map_times[-1])
for a4_slice in slices:
    ax.axvline(a4_slice / (A * H), color=GREEN, linestyle="--", linewidth=1)
ax.plot([], [], "--", color=GREEN, label="slices $a_{4,0}$ of the Kohn-Sham record")
ax.text(0.3, -1.9, "real frequency:\ninertia (4, 4)", color=BLUE, fontsize=9)
ax.text(4.4, -0.6, "imaginary frequency:\nKrein-neutral", color=ORANGE, fontsize=9)
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("$\\log_{10}$ of the extra-time momentum $q_5$")
ax.set_title("Which waves still oscillate? ($q_1 = 0.5$, $y = 0$)")
ax.legend(loc="lower left", fontsize=8)  # the lower left corner is all blue
ax.grid(False)
save_figure(fig, "onset_map",
            "For waves with the coordinate momenta $q_1 = 0.5$ and $q_5$ from "
            "$10^{-3}$ to 1 (vertical axis: $\\log_{10}q_5$) at the hidden position "
            "$y = 0$, along the deflating history $a_4 = x_4$ (horizontal axis: the "
            "time $x_4$ in units of $1/m$): light blue where the frequency is real "
            "(the eigenspaces have Krein inertia (4, 4)), light orange where it is "
            "imaginary (the eigenspaces are Krein-neutral and the wave grows). The "
            "grey curve is the exact onset time; the green dashed lines are the "
            "times of the five slices $a_{4,0} = 0$, 0.5, 1, 1.5, 2 of the Kohn-Sham "
            "record. As time goes on, smaller and smaller extra-time momenta cross "
            "into the orange region: the band of oscillating extra-time waves "
            "shrinks exponentially, and only the good sector $q_5 = 0$ stays blue "
            "for ever.")
```

Out [10]:

```
RESULT grid points sampled for the inertia = 153
```

```
RESULT onset time for q5 = 0.001 and for q5 = 1 = 6.9078 and 0.0941
PASS the map agrees with the exact onset; (4, 4) before it, neutral after it
```

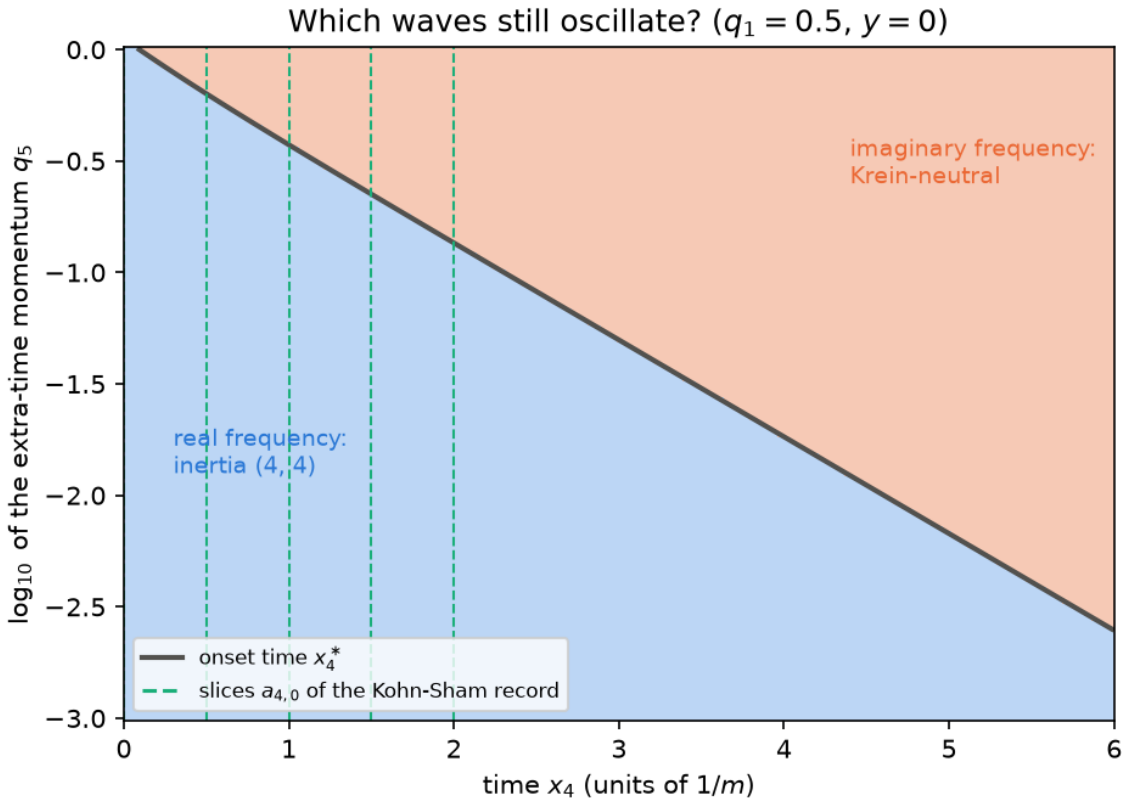

Figure 296: For waves with the coordinate momenta $q_1 = 0.5$ and q_5 from 10^{-3} to 1 (vertical axis: $\log_{10} q_5$) at the hidden position $y = 0$, along the deflating history $a_4 = x_4$ (horizontal axis: the time x_4 in units of $1/m$): light blue where the frequency is real (the eigenspaces have Krein inertia (4, 4)), light orange where it is imaginary (the eigenspaces are Krein-neutral and the wave grows). The grey curve is the exact onset time; the green dashed lines are the times of the five slices $a_{4,0} = 0$, 0.5, 1, 1.5, 2 of the Kohn-Sham record. As time goes on, smaller and smaller extra-time momenta cross into the orange region: the band of oscillating extra-time waves shrinks exponentially, and only the good sector $q_5 = 0$ stays blue for ever. (Notebook 10f, figure 4.)

Out [10]:

```
Figure 10f.4 saved as Revision/textbook/figures/10f_4_onset_map.png
```

Text cell:

```
## 11. The good sector along the history

In the good sector ($q_5 = 0$) the mode Hamiltonian of every instant is Hermitian
and commutes with $B$ (Section 7), with the energies $\pm E(x_4)$,
```

```
$$E(x_4) = \sqrt{m^2 + k_1^2} = \sqrt{m^2 + q_1^2e^{-2AHx_4 - 2Hy}},$$

which falls towards $m$ as 3-space inflates and its frame momentum shrinks. At each
instant the positive Fock space of the previous notebooks can be built with the
coefficients of that instant (an *instantaneous* Fock space): the filled sea has the
value $\sum_s v_s^\dagger hv_s = -8E(x_4)$, and a particle and an antiparticle have
the normal-ordered energies $u_s^\dagger hu_s = +E(x_4)$ and $-v_s^\dagger hv_s =
+E(x_4)$. The next cell checks, at the five slices of the Kohn-Sham record and for
four momenta $q_1 = 0.5, 1, 2, 4$ ($y = 0$): $h$ Hermitian, $Bh = hB$, eight
eigenvalues $+E$ and eight $-E$, the Krein inertia $(4, 4)$ of both eigenspaces, the
sea value $-8E$ and the quantum energies $+E$. Then it draws $E(x_4)$ and the sea
value.
```

In [11]:

```
slice_ok, table = True, []
for q1 in (0.5, 1.0, 2.0, 4.0):
    for a4_slice in slices:
        x4 = a4_slice / (A * H)  # the time of the slice
        h = mode_hamiltonian(x4, q1, 0.0, 0.0)
        E = np.sqrt(w_squared(x4, q1, 0.0, 0.0))
        U_plus = eigenspace(h, E ** 2, +1)  # the 8 particle columns u_s
        V_minus = eigenspace(h, E ** 2, -1)  # the 8 antiparticle columns v_s
        sea = sum((v.conj() @ h @ v).real for v in V_minus.T)
        particle = [(u.conj() @ h @ u).real for u in U_plus.T]
        antiparticle = [-(v.conj() @ h @ v).real for v in V_minus.T]
        slice_ok = (slice_ok and np.allclose(h, h.conj().T) and np.allclose(B @ h, h @ B)
                    and U_plus.shape[1] == 8 and V_minus.shape[1] == 8
                    and krein_inertia(U_plus) == (4, 4, 0)
                    and krein_inertia(V_minus) == (4, 4, 0)
                    and abs(sea + 8 * E) < 1e-12
                    and np.allclose(particle, E) and np.allclose(antiparticle, E))
        table.append((q1, a4_slice, E, sea))
say("q1    a4,0    E(x4)      sea value -8E")
for q1, a4_slice, E, sea in table:
    if q1 in (1.0, 4.0):  # print two of the four momenta
        say(f"{q1:3.1f}   {a4_slice:3.1f}   {E:9.6f}   {sea:11.6f}")
check_record(slice_ok,
             "good sector at all 5 slices: Hermitian, [B, h] = 0, +-E, (4, 4), sea -8E",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for q1, colour in ((0.5, BLUE), (1.0, ORANGE), (2.0, GREEN), (4.0, GREY)):
    E_curve = np.sqrt(w_squared(curve_times, q1, 0.0, 0.0))
    axes[0].plot(curve_times, E_curve, color=colour, linewidth=2, label=f"$q_1 = {q1}$")
    axes[1].plot(curve_times, -8 * E_curve, color=colour, linewidth=2,
                 label=f"$q_1 = {q1}$")
    dots = [(a4_slice, E, sea) for q, a4_slice, E, sea in table if q == q1]
    axes[0].plot([d[0] for d in dots], [d[1] for d in dots], "o", color=colour)
    axes[1].plot([d[0] for d in dots], [d[2] for d in dots], "o", color=colour)
axes[0].axhline(mass, color=GREY, linestyle=":", linewidth=1)
axes[1].axhline(-8 * mass, color=GREY, linestyle=":", linewidth=1)
```

```
axes[0].set_ylabel("energy $E(x_4)$ of one quantum (units of $m$)")
axes[1].set_ylabel("value $-8E(x_4)$ of the filled sea (units of $m$)")
axes[0].set_title("particles and antiparticles: $+E(x_4) > 0$")
axes[1].set_title("the instantaneous vacuum")
for ax in axes:
    ax.set_xlabel("time $x_4$ (units of $1/m$)")
    ax.legend(fontsize=8)
save_figure(fig, "good_sector_energies",
            "The good sector along the deflating history $a_4 = x_4$ ($A = H = m = "
            "1$, $y = 0$) for the coordinate momenta $q_1 = 0.5$, 1, 2, 4 (blue, "
            "orange, green, grey); horizontal axes: the time $x_4$ in units of "
            "$1/m$. Left: the normal-ordered energy $E(x_4) = \\sqrt{m^2 + "
            "q_1^2e^{-2x_4}}$ of one particle or one antiparticle of the "
            "instantaneous Fock space, falling towards the mass $m$ (dotted line) "
            "as 3-space inflates; right: the value $-8E(x_4)$ of the filled sea, "
            "rising towards $-8m$ (dotted line). Vertical axes in units of $m$. "
            "Dots: computed from the eigenvectors "
            "at the five slices of the Kohn-Sham record. In the good sector every "
            "quantum has a positive energy at every time, and the Krein inertia "
            "stays (4, 4).")
```

Out [11]:

```
q1    a4,0    E(x4)      sea value -8E
1.0   0.0    1.414214    -11.313708
1.0   0.5    1.169564     -9.356510
1.0   1.0    1.065521     -8.524169
1.0   1.5    1.024591     -8.196729
1.0   2.0    1.009116     -8.072930
4.0   0.0    4.123106    -32.984845
4.0   0.5    2.624132    -20.993060
4.0   1.0    1.779147    -14.233177
4.0   1.5    1.340371    -10.722964
4.0   2.0    1.137124     -9.096989
PASS good sector at all 5 slices: Hermitian, [B, h] = 0, +-E, (4, 4), sea -8E
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_spectrum_and_B_sectors
```

Out [11]:

```
Figure 10f.5 saved as Revision/textbook/figures/10f_5_good_sector_energies.png
```

Text cell:

```
## 12. The universes of masses $+m$ and $-m$ along the history

At every instant the mode Hamiltonian of the history is a mode Hamiltonian of the
first notebook of this chapter, so the identity $\Gamma h_m\Gamma = h_{-m}$ of the
Revision pairing record holds at every time ($\Gamma$ anticommutes with
$\gamma^{(x_4)}$ and commutes with $\gamma^{(x_4)}\gamma^{(x_1)}$ and
$\gamma^{(x_4)}\gamma^{(x_5)}$). The mass enters $w^2$ only as $m^2$, so the two
```

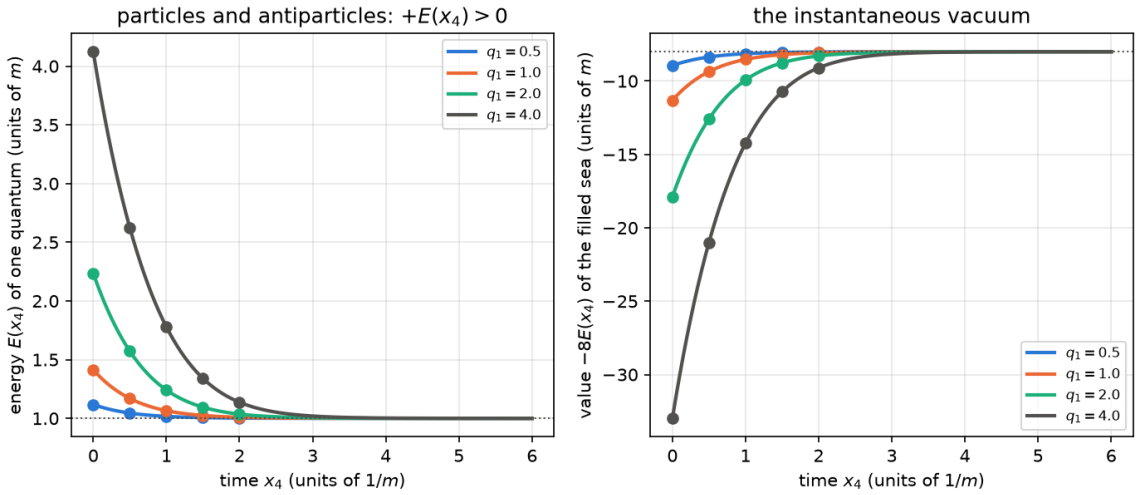

Figure 297: The good sector along the deflating history $a_4 = x_4$ ($A = H = m = 1$, $y = 0$) for the coordinate momenta $q_1 = 0.5$, 1, 2, 4 (blue, orange, green, grey); horizontal axes: the time x_4 in units of $1/m$. Left: the normal-ordered energy $E(x_4) = \sqrt{m^2 + q_1^2 e^{-2x_4}}$ of one particle or one antiparticle of the instantaneous Fock space, falling towards the mass m (dotted line) as 3-space inflates; right: the value $-8E(x_4)$ of the filled sea, rising towards $-8m$ (dotted line). Vertical axes in units of m. Dots: computed from the eigenvectors at the five slices of the Kohn-Sham record. In the good sector every quantum has a positive energy at every time, and the Krein inertia stays (4, 4). (Notebook 10f, figure 5.)

```
universes have the same frequencies, the same onset time and the same Krein inertia
at every instant. The next cell checks the identity at 25 times for the wave
$q_1 = 0.5$, $q_5 = 0.05$, and that the eigenvalues for $+m$ and $-m$ agree.
```

In [12]:

```
pair_ok, worst = True, 0.0
for x4 in np.linspace(0.0, 6.0, 25):
    h_plus = mode_hamiltonian(x4, 0.5, 0.05, 0.0, m=mass)
    h_minus = mode_hamiltonian(x4, 0.5, 0.05, 0.0, m=-mass)
    pair_ok = pair_ok and np.allclose(Gamma @ h_plus @ Gamma, h_minus, atol=1e-14)
    values_plus = np.linalg.eigvals(h_plus)
    values_minus = np.linalg.eigvals(h_minus)
    # compare the sorted real parts and the sorted imaginary parts separately
    worst = max(worst,
                np.max(np.abs(np.sort(values_plus.real) - np.sort(values_minus.real))),
                np.max(np.abs(np.sort(values_plus.imag) - np.sort(values_minus.imag))))
report("largest difference of the eigenvalues for +m and -m", f"{worst:.1e}")
check_record(pair_ok and worst < 1e-9,
             "Gamma h_m(x4) Gamma = h_-m(x4) at every time: same spectra and onset",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_maps")
```

Out [12]:

```
RESULT largest difference of the eigenvalues for +m and -m = 7.1e-15
PASS Gamma h_m(x4) Gamma = h_-m(x4) at every time: same spectra and onset
     reproduces Revision/pairing/reports/python-pairing.json, check Q.one_particle_maps
```

Text cell:

```
## 13. The last check

The last cell checks that all five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [13]:

```
for name in ("10f_1_hermiticity_defect.png", "10f_2_eigenvalues_along_history.png",
             "10f_3_inertia_along_history.png", "10f_4_onset_map.png",
             "10f_5_good_sector_energies.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [13]:

```
PASS the figure file 10f_1_hermiticity_defect.png exists
PASS the figure file 10f_2_eigenvalues_along_history.png exists
PASS the figure file 10f_3_inertia_along_history.png exists
PASS the figure file 10f_4_onset_map.png exists
PASS the figure file 10f_5_good_sector_energies.png exists
ALL 19 CHECKS PASSED (notebook 10f)
```

Text cell:

```
## 14. What this notebook showed

- REPRODUCED from the Revision records: the deflating history $a_4 = AHx_4$ with
  $A = H = m = 1$ and its status, and the scale factors of the author's metric; a
  wave $e^{i(q_1x_1 + q_5x_5)}$ has the frame momenta $k_1 = q_1e^{-a_4}
  \sin^{-1/6}z$ (shrinking: 3-space inflates) and $k_5 = q_5e^{a_4}\sin^{-1/6}z$
  (growing: the extra times deflate).
- PROVED (exactly, at every instant of the local-frame model): $h(x_4)^2 =
  w(x_4)^2I_{16}$ with $w^2 = m^2 + k_1^2 - k_5^2$; the anti-Hermitian part of
  $h(x_4)$ has the size $k_5$ and the mixing of the two eigenspaces of $B$ the size
  $2k_5$, both growing like $e^{a_4}$; in the good sector $h(x_4)$ is Hermitian and
  commutes with $B$ at every time.
- PROVED (exactly): every wave with $q_5 \neq 0$ reaches the onset time
  $x_4^\ast = \ln X^\ast/(2AH)$ and grows afterwards. COMPUTED: the onset of the
  wave $q_1 = 0.5$, $q_5 = 0.05$ is $x_4^\ast = 2.996$; the recorded sample $m = 1$,
  $k_5 = 2$ (eigenvalues $\pm i\sqrt3$) is the instant $x_4 = \ln 40$ of the wave
  $q_5 = 0.05$; the Krein inertia is $(4, 4)$ at every instant before the onset and
  the eigenspaces are Krein-neutral at every instant after it; the band of
```

```
  oscillating extra-time waves shrinks exponentially, and only the good sector
  keeps real frequencies for ever.
- COMPUTED (at the five slices of the Kohn-Sham record): in the good sector the
  instantaneous Fock space has the sea value $-8E(x_4)$ and quanta of energy
  $+E(x_4) > 0$, with $E$ falling towards $m$.
- PROVED (at every instant): the masses $+m$ and $-m$ have identical one-particle
  spectra, onset times and Krein inertia.
- ASSUMED: the history $a_4 = AHx_4$ is a prescribed background, not a solution of
  the field equations for $a_4$ with this field as the source; the local-frame model
  evaluates the coefficients at one instant and one hidden position and leaves out
  the hidden-direction terms.
- OPEN (not computed here): the evolution of the quantum state through the changing
  background (whether the instantaneous vacuum of one time contains quanta of a
  later time); a positive state space for the extra-time sector, whose waves all
  become Krein-neutral.
```

10.43 Line-by-line walk-through of Notebook 10f

The notebook has 13 code cells, In [1] to In [13]; docstrings are left out of the quotations (they are printed in Section 10.42).

In [1], the set-up cell. Its first 252 lines are comments that repeat the run instructions of Section 10.41. The code is that of In [1] of Notebook 10a (Section 10.10), with the line `NOTEBOOK_ID = "10f" # this notebook: chapter 10, example f`. It prints `Set-up of notebook 10f complete: repository folder found, helpers defined.`

In [2], the deflating history of the Revision record.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
```

```
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
```

The imports, the helpers `REPORT_CHECKS`, `record_says_pass` and `check_record` of Section 10.10 (a cited report check must still exist with the verdict PASS; the two printed lines come in one piece), and the four colours.

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
H = parameters["physics"]["H"]  # the author's constant H
mass = parameters["physics"]["m"]  # the mass m
A = parameters["physics"]["historyA"]  # the history a4 = A H x4
slices = parameters["physics"]["slicesA4"]  # the values a4,0 of the record
```

The parameters of the Kohn-Sham record are read: the constant H, the mass m (called `mass` here, because the letter `m` is kept free), the history constant A and the list of the five slices $a_{4,0}$.

```
say("status of the history: "
    + parameters["conventions"]["history"].split(": ")[0] + ".")
say(f"slices a4,0 of the Kohn-Sham record: {slices}")
check_record(A == 1.0 and H == 1.0 and mass == 1.0,
             "the deflating history a4 = A H x4 with A = 1, H = 1, m = 1",
             record="Revision/kohn_sham/results/parameters.json, physics.historyA, "
                    "physics.H, physics.m")
```

The record states the status of the history in a sentence that begins with `PRESCRIBED BACKGROUND:`; `.split(":  ")` cuts the text at the first colon and blank (and at any later one), and `[0]` keeps the first piece. The printed lines are `status of the history:  PRESCRIBED BACKGROUND.` and the slices `[0.0, 0.5, 1.0, 1.5, 2.0]`; the check confirms $A = H = m = 1$ and reproduces the three entries of the record.

```
def a4_of(x4):
    return A * H * x4
```

`a4_of(x4)` is the prescribed history $a_4 = AHx_4$.

In [3], frame momenta from the scale factors of the record.

```
theory = json.loads(repository_file("Revision/theory/field-theory.json").read_text(
    encoding="utf-8"))
text = next(f["wl"] for f in theory["formulas"] if f["key"] == "vielbein_diagonal")
a4_symbol, z_symbol = sp.symbols("a4 z", real=True)
```

The theory record holds a list of formulas, each a dictionary with a key and the formula as text in the Wolfram Language (entry `wl`). `next(...)` returns the first item of the generator expression in the parentheses: the text of the formula whose key is `vielbein_diagonal`, the eight scale factors. `a4_symbol` and `z_symbol` are two real sympy letters.

```
translated = (text.replace("E^a4[x4]", "exp(a4)").replace("Sin[6*H*x0]", "sin(z)")
              .replace("Cot[6*H*x0]", "cot(z)").replace("^", "**")
              .replace("{", "[").replace("}", "]"))
f_record = sp.sympify(translated, locals={"a4": a4_symbol, "z": z_symbol})
```

The Wolfram text, for example `E^a4[x4]*Sin[6*H*x0]^(1/6)`, is translated step by step into sympy notation: e^{a_4}, $\sin z$ and $\cot z$ are written with the letters a_4 and z, `^` becomes `**` and the curly brackets of a Wolfram list become square brackets. `sp.sympify` reads the text as a list of eight exact expressions; `locals` tells it which letters to use.

```
sixth_root = sp.sin(z_symbol) ** sp.Rational(1, 6)  # sin(z)^(1/6)
f_expected = ([sp.cot(z_symbol)] + [sp.exp(a4_symbol) * sixth_root] * 3
              + [sp.Integer(1)] + [sp.exp(-a4_symbol) * sixth_root] * 3)  # x0, ..., x7
same = all(sp.simplify(r - e) == 0 for r, e in zip(f_record, f_expected))
say(f"scale factor of x1: {f_record[1]}")
say(f"scale factor of x5: {f_record[5]}")
check_record(len(f_record) == 8 and same,
             "the record's scale factors: e^a4 sin^(1/6) z (3-space), e^-a4 ... (x5-x7)",
             record="Revision/theory/field-theory.json, formula vielbein_diagonal")
```

`sp.Rational(1, 6)` is the exact fraction $\frac{1}{6}$. The expected list is built by repeating and joining lists: three times $e^{a_4}\sin^{1/6} z$, then 1, three times $e^{-a_4}\sin^{1/6} z$, then $\cot z$. The comparison simplifies each difference to 0. The printed lines show the scale factors of x_1, `exp(a4)*sin(z)**(1/6)`, and of x_5, `exp(-a4)*sin(z)**(1/6)`: the extra times deflate exponentially as a_4 grows. The check reproduces the record's formula.

```
def frame_momenta(x4, q1, q5, y):
    root = np.exp(H * y)  # sin(z)^(1/6) = e^(H y)
    k1 = q1 / (np.exp(a4_of(x4)) * root)  # q1 / f_1: shrinks, 3-space inflates
    k5 = q5 / (np.exp(-a4_of(x4)) * root)  # q5 / f_5: grows, extra times deflate
    return k1, k5
```

The frame momenta of Section 10.39: $k_1 = q_1/(e^{a_4}e^{Hy})$ and $k_5 = q_5/(e^{-a_4}e^{Hy})$, with $\sin^{1/6} z = e^{Hy}$.

In [4], the mode Hamiltonian of each instant, exactly.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])  # the chirality matrix
```

The gammas of the record as decimal numbers, C, B and the chirality Γ.

```
def mode_hamiltonian(x4, q1, q5, y, m=mass):
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return (-1j * m * gamma[4] - k1 * (gamma[4] @ gamma[1])
            - k5 * (gamma[4] @ gamma[5]))


def w_squared(x4, q1, q5, y, m=mass):
    k1, k5 = frame_momenta(x4, q1, q5, y)
    return m ** 2 + k1 ** 2 - k5 ** 2
```

The mode Hamiltonian $h(x_4)$ of Section 10.39 and its squared frequency $w^2 = m^2 + k_1^2 - k_5^2$. The argument `m=mass` has a **default value**: when the call does not name `m`, the mass of the record is used.

```
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
B_exact = -sp.I * g[0] * g[1] * g[2] * g[3] * g[4]
m_s, q1_s, q5_s, y_s = sp.symbols("m q1 q5 y", real=True)
x4_s, H_s = sp.symbols("x4 H", positive=True)
a4_s = A * H_s * x4_s  # the history, with the record's A = 1
k1_s = q1_s * sp.exp(-a4_s - H_s * y_s)  # frame momentum along x1
k5_s = q5_s * sp.exp(a4_s - H_s * y_s)  # frame momentum along x5
h_s = -sp.I * m_s * g[4] - k1_s * g[4] * g[1] - k5_s * g[4] * g[5]
zero = sp.zeros(16, 16)
```

The same objects exactly: letters for m, q_1, q_5, y (real) and x_4, H (positive), the history, the frame momenta $k_1 = q_1 e^{-a_4 - Hy}$ and $k_5 = q_5 e^{a_4 - Hy}$, and the exact $h(x_4)$. A statement about these letters holds at every instant and every position.

```
w2_s = m_s ** 2 + k1_s ** 2 - k5_s ** 2
square_ok = (h_s * h_s - w2_s * sp.eye(16)).applyfunc(sp.expand) == zero
h_A = ((h_s - h_s.H) / 2).applyfunc(sp.expand)  # the anti-Hermitian part
part_ok = (h_A + k5_s * g[4] * g[5]).applyfunc(sp.expand) == zero
size_ok = (h_A.H * h_A - k5_s ** 2 * sp.eye(16)).applyfunc(sp.expand) == zero
mixing_ok = (B_exact * h_s - h_s * B_exact - 2 * B_exact * h_A).applyfunc(
    sp.expand) == zero
```

The identities of Section 10.39: $h^2 = w^2 I_{16}$; the anti-Hermitian part $h_A = \frac{1}{2}(h - h^\dagger)$ equals $-k_5 \gamma^{(x_4)} \gamma^{(x_5)}$; $h_A^\dagger h_A = k_5^2 I_{16}$; and $Bh - hB = 2Bh_A$.

```
good = h_s.subs(q5_s, 0)  # the good sector
good_ok = ((good - good.H).applyfunc(sp.expand) == zero
           and (B_exact * good - good * B_exact).applyfunc(sp.expand) == zero)
check(square_ok, "exact: h(x4)^2 = (m^2 + k1^2 - k5^2) I at every instant")
check(part_ok and size_ok and mixing_ok,
      "exact: h_A = -k5 g4 g5, h_A^dagger h_A = k5^2 I, B h - h B = 2 B h_A")
check_record(good_ok,
             "exact: good sector, h(x4) Hermitian and commuting with B at all times",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")
```

With $q_5 = 0$ the matrix must be Hermitian and commute with B at all times. Three PASS lines; the third reproduces the record check `good_sector_spectrum_and_B_sectors`.

In [5], the departure from Hermiticity and the mixing of the two sectors of B.

```
times = np.linspace(0.0, 6.0, 13)  # 0, 0.5, ..., 6
curve_times = np.linspace(0.0, 6.0, 241)
measured, worst = {}, 0.0
for q5 in (0.01, 0.05, 0.2):
    sizes = []
    for x4 in times:
        h = mode_hamiltonian(x4, 0.5, q5, 0.0)
        h_anti = (h - h.conj().T) / 2
        sizes.append((np.linalg.norm(h_anti, 2), np.linalg.norm(B @ h - h @ B, 2)))
```

For three extra-time momenta $q_5 = 0.01, 0.05, 0.2$ (with $q_1 = 0.5$ and $y = 0$) and 13 times from 0 to 6, the cell measures the size of the anti-Hermitian part and of the commutator $Bh - hB$; `np.linalg.norm(M, 2)` is the size of a matrix (its largest stretching factor).

```
        k5 = frame_momenta(x4, 0.5, q5, 0.0)[1]
        worst = max(worst, abs(sizes[-1][0] - k5) / k5,
                    abs(sizes[-1][1] - 2 * k5) / (2 * k5))
    measured[q5] = np.array(sizes)
good_sizes = [np.linalg.norm(mode_hamiltonian(x4, 0.5, 0.0, 0.0)
                             - mode_hamiltonian(x4, 0.5, 0.0, 0.0).conj().T, 2)
              for x4 in times]
report("largest relative difference from k5 and 2 k5", f"{worst:.1e}")
check(worst < 1e-12 and max(good_sizes) == 0.0,
      "sizes of h_A and of B h - h B are k5 = q5 e^a4 and 2 k5; zero if q5 = 0")
```

The measured sizes are compared with k_5 and $2k_5$ (`sizes[-1]` is the pair just appended; `[1]` of the frame momenta is k_5), and in the good sector the size of $h - h^\dagger$ must be exactly 0 at every time. The largest relative difference is printed as `0.0e+00`.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
for q5, colour in ((0.01, BLUE), (0.05, ORANGE), (0.2, GREEN)):
    k5_curve = frame_momenta(curve_times, 0.5, q5, 0.0)[1]
    axes[0].plot(curve_times, k5_curve, color=colour, linewidth=2,
                 label=f"$q_5 = {q5}$")
    axes[0].plot(times, measured[q5][:, 0], "o", color=colour, markersize=5)
    axes[1].plot(curve_times, 2 * k5_curve, color=colour, linewidth=2,
                 label=f"$q_5 = {q5}$")
    axes[1].plot(times, measured[q5][:, 1], "o", color=colour, markersize=5)
```

For each q_5 the formulas k_5 and $2k_5$ are drawn as lines (`frame_momenta` works on the whole array of 241 times at once) and the measured sizes as dots.

```
for ax in axes:
    ax.set_yscale("log")
```

```
    ax.axhline(mass, color=GREY, linestyle=":", linewidth=1)
    ax.set_xlabel("time $x_4$ (units of $1/m$)")
    ax.legend(loc="lower right", fontsize=8)
axes[0].set_ylabel("size (units of $m$)")
axes[0].set_title("anti-Hermitian part $h_A$: size $k_5 = q_5e^{a_4}$")
axes[1].set_title("mixing of the $B$ sectors: size of $Bh - hB$")
save_figure(fig, "hermiticity_defect",
...)
```

Logarithmic vertical axes (`set_yscale("log")`), on which an exponential is a straight line, and a dotted line at the mass. Figure 1 of Notebook 10f shows three parallel straight lines in each panel with the dots on them: the departure from Hermiticity and the mixing of the two sectors of B grow like e^{a_4}, crossing the level of the mass sooner for larger q_5.

In [6], the exact onset time.

```
def onset_time(q1, q5, y):
    c = np.exp(-2 * H * y)
    X = (mass ** 2 + np.sqrt(mass ** 4 + 4 * q1 ** 2 * q5 ** 2 * c ** 2)) / (
        2 * q5 ** 2 * c)
    return np.log(X) / (2 * A * H)
```

The formula $x_4^* = \ln X^*/(2AH)$ of Section 10.39, with $c = e^{-2Hy}$.

```
def bisection(q1, q5, y, low=-40.0, high=40.0):
    for _ in range(200):
        middle = (low + high) / 2
        if w_squared(middle, q1, q5, y) > 0:  # still oscillating: the onset is later
            low = middle
        else:
            high = middle
    return (low + high) / 2
```

An independent numerical root search, **bisection**: the interval from -40 to 40 contains the onset; at its midpoint, if $w^2 > 0$ the wave still oscillates and the onset lies in the right half, otherwise in the left half; halving 200 times shrinks the interval far below the rounding level, and its midpoint is returned.

```
worst = max(abs(onset_time(q1, q5, y) - bisection(q1, q5, y))
            for q1 in (0.0, 0.5, 1.0, 3.0) for q5 in (0.01, 0.05, 0.1, 0.4)
            for y in (0.0, -1.0, -3.0))
report("largest difference, exact onset time minus bisection (48 waves)",
       f"{worst:.1e}")
check(worst < 1e-12 and abs(onset_time(0.0, 0.05, 0.0) - np.log(1 / 0.05)) < 1e-14,
      "the exact onset time agrees with bisection; q1 = 0, y = 0: ln(m / q5)")
onset = onset_time(0.5, 0.05, 0.0)  # the wave followed below
report("onset time of the wave q1 = 0.5, q5 = 0.05, y = 0", f"{onset:.6f}")
```

For $4 \times 4 \times 3 = 48$ waves the formula and the bisection agree to `8.9e-16`; the check also confirms the short form $\ln(m/q_5) = \ln 20$ for $q_1 = 0$, $y = 0$. The onset time of the wave followed below is printed as `2.996044`.

In [7], the recorded sample as an instant of the history.

```
instant = np.log(40.0)  # the time at which k5 = 0.05 e^x4 = 2
k5_instant = frame_momenta(instant, 0.0, 0.05, 0.0)[1]
eigenvalues = np.linalg.eigvals(mode_hamiltonian(instant, 0.0, 0.05, 0.0))
plus = int(np.sum(np.abs(eigenvalues - 1j * np.sqrt(3)) < 1e-9))
minus = int(np.sum(np.abs(eigenvalues + 1j * np.sqrt(3)) < 1e-9))
early = np.linalg.eigvals(mode_hamiltonian(0.0, 0.0, 0.05, 0.0))
```

At $x_4 = \ln 40$ the wave $q_1 = 0$, $q_5 = 0.05$, $y = 0$ has $k_5 = 0.05 \cdot 40 = 2$. The 16 eigenvalues of that instant are counted near $+i\sqrt{3}$ and near $-i\sqrt{3}$; `early` are the eigenvalues of the same wave at $x_4 = 0$, where $k_5 = 0.05$.

```
report("frame momentum k5 at x4 = ln 40", f"{k5_instant:.12f}")
report("eigenvalues +i sqrt(3) and -i sqrt(3), how many", f"{plus} and {minus}")
largest_imaginary = np.max(np.abs(early.imag))  # 0 up to rounding: oscillation
report("largest imaginary part of an eigenvalue at x4 = 0", f"{largest_imaginary:.1e}")
check_record(abs(k5_instant - 2) < 1e-12 and (plus, minus) == (8, 8)
             and largest_imaginary < 1e-12,
             "the recorded sample m = 1, k5 = 2 (+-i sqrt 3) is reached at x4 = ln 40",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "extra_time_modes_grow")
```

The printed lines show $k_5 = 2.000000000000$, eight eigenvalues near each of $\pm i\sqrt{3}$, and at $x_4 = 0$ a largest imaginary part of `1.8e-16` (zero up to rounding: the wave still oscillates). The check reproduces the record check `extra_time_modes_grow` and places its sample on the deflating history.

In [8], the eigenvalues of one wave through the onset.

```
scan_times = np.linspace(0.0, 5.0, 251)
real_parts, imaginary_parts, worst = [], [], 0.0
for x4 in scan_times:
    eigenvalues = np.linalg.eigvals(mode_hamiltonian(x4, 0.5, 0.05, 0.0))
    real_parts.append(np.sort(eigenvalues.real))
    imaginary_parts.append(np.sort(eigenvalues.imag))
    w2 = w_squared(x4, 0.5, 0.05, 0.0)
    if abs(w2) > 0.05:  # away from the onset
        w = np.sqrt(complex(w2))  # real, or i kappa after the onset
        predicted = np.array([w] * 8 + [-w] * 8)
        worst = max(worst,
                    np.max(np.abs(np.sort(eigenvalues.real) - np.sort(predicted.real))),
                    np.max(np.abs(np.sort(eigenvalues.imag) - np.sort(predicted.imag))))
report("largest difference from +-w(x4) away from the onset", f"{worst:.1e}")
check(worst < 1e-10, "at every instant the eigenvalues of h(x4) are +-w(x4), 8 each")
```

The loop of In [9] of Notebook 10a (Section 10.10), now along the history: for the wave $q_1 = 0.5$, $q_5 = 0.05$ at 251 times from 0 to 5 it computes the 16 eigenvalues of $h(x_4)$, stores their sorted real and imaginary parts, and, away from the onset (where the matrix cannot be diagonalised), compares them with $\pm w(x_4)$, eight each. The largest difference is printed as `8.0e-15`.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0), sharex=True)
axes[0].plot(scan_times, np.array(real_parts), color=BLUE, linewidth=2)
axes[1].plot(scan_times, np.array(imaginary_parts), color=ORANGE, linewidth=2)
for ax, what in ((axes[0], "real part"), (axes[1], "imaginary part")):
    ax.axvline(onset, color=GREY, linestyle=":", linewidth=1)
    ax.set_xlabel("time $x_4$ (units of $1/m$)")
    ax.set_ylabel(f"{what} of the eigenvalues (units of $m$)")
axes[0].set_title("oscillation before the onset")
axes[1].set_title("growth after the onset")
save_figure(fig, "eigenvalues_along_history",
...)
```

The real parts (left) and imaginary parts (right) against the time, with a dotted line at the onset. Figure 2 of Notebook 10f shows the real frequencies $\pm w(x_4)$ closing in to 0 at $x_4^* = 2.996$ and the imaginary parts opening up after it, growing further as the extra-time frame momentum keeps growing: the picture of Figure 3 of Notebook 10a, now produced by the deflation itself.

In [9], the Krein inertia of one wave along the history.

```
def orthonormal_basis(P):
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):  # the second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T
```

The Gram-Schmidt function of In [10] of Notebook 10a (Section 10.10).

```
def eigenspace(h, w2, sign):
    w = np.sqrt(complex(w2))
    return orthonormal_basis((np.eye(16) + sign * h / w) / 2)


def krein_inertia(V):
    values = np.linalg.eigvalsh(V.conj().T @ B @ V)
    return (int(np.sum(values > 1e-8)), int(np.sum(values < -1e-8)),
            int(np.sum(np.abs(values) <= 1e-8)))
```

`eigenspace(h, w2, sign)` returns an orthonormal basis of the eigenspace of $\pm w$ (or $\pm i\kappa$) of a given matrix h, as the range of the projector $\frac{1}{2}(I_{16} \pm h/w)$; `krein_inertia(V)` counts the positive, negative and zero eigenvalues of the Gram matrix $V^\dagger B V$ (Section 10.5), as in Notebook 10a.

```
kept = [x4 for x4 in scan_times if abs(x4 - onset) > 0.03]
inertias = [krein_inertia(eigenspace(mode_hamiltonian(x4, 0.5, 0.05, 0.0),
                                     w_squared(x4, 0.5, 0.05, 0.0), +1))
```

```
            for x4 in kept]
before = {c for x4, c in zip(kept, inertias) if x4 < onset}
after = {c for x4, c in zip(kept, inertias) if x4 > onset}
say(f"inertia before the onset: {sorted(before)}; after the onset: {sorted(after)}")
```

The inertia of the $+w$ eigenspace at every time farther than 0.03 from the onset. `before` and `after` are the **sets** of the different inertias found before and after the onset (a set, written with braces, keeps each value once). The printed line shows `[(4, 4, 0)]` before and `[(0, 0, 8)]` after.

```
check_record(before == {(4, 4, 0)},
             "every instant before the onset: inertia (4, 4) (real frequency)",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_Krein_inertia_proof")
check_record(after == {(0, 0, 8)},
             "every instant after the onset: the eigenspace is Krein-neutral",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_complex_frequency_Krein_neutral")
```

Two PASS lines, reproducing the record checks of Theorem 10.1 instant by instant.

```
counts = np.array(inertias)
fig, ax = plt.subplots()
width = 0.0205  # a little wider than the step 0.02 between the times (no gaps)
ax.bar(kept, counts[:, 0], width=width, color=BLUE,
       label="positive charge $u^\\dagger B u > 0$")
ax.bar(kept, counts[:, 1], width=width, bottom=counts[:, 0], color=ORANGE,
       label="negative charge $u^\\dagger B u < 0$")
ax.bar(kept, counts[:, 2], width=width, bottom=counts[:, 0] + counts[:, 1],
       color=GREEN, label="neutral")
```

Stacked bars, as in In [14] of Notebook 10a: at each time a bar of height 8, split into its positive (blue), negative (orange) and neutral (green) directions.

```
ax.axvline(onset, color=GREY, linestyle=":", linewidth=1)
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("directions in the $+w$ eigenspace")
ax.set_yticks(range(0, 9))
ax.set_ylim(0.0, 10.5)  # room for the legend above the bars
ax.set_title("Krein inertia of one wave along the deflating history")
ax.legend(loc="upper center", ncol=3, fontsize=8)
save_figure(fig, "inertia_along_history",
...)
```

Figure 3 of Notebook 10f shows half-blue, half-orange bars up to the onset time 2.996 and green bars after it: the deflation of the extra times moves the wave from the kind with inertia $(4, 4)$ to the Krein-neutral kind at a definite time.

In [10], the onset map.

```
map_times = np.linspace(0.0, 6.0, 241)
exponents = np.linspace(-3.0, 0.0, 121)  # log10 of q5
q5_values = 10.0 ** exponents
real_frequency = np.array([[w_squared(x4, 0.5, q5, 0.0) > 0 for x4 in map_times]
                           for q5 in q5_values])  # rows: q5, columns: x4
onsets = np.array([onset_time(0.5, q5, 0.0) for q5 in q5_values])
agree = np.array_equal(real_frequency, map_times[None, :] < onsets[:, None])
```

A grid of 241 times from 0 to 6 and 121 momenta q_5 from 10^{-3} to 1, equally spaced in $\log_{10} q_5$ (`10.0 ** exponents`). `real_frequency` is a table of truth values, true where $w^2 > 0$. `onsets` holds the exact onset of each momentum, and `agree` compares the table with "the time lies before the onset": `map_times[None, :]` is a row and `onsets[:, None]` a column, and comparing them gives the whole table at once.

```
sampled, sample_ok = 0, True
for i in range(0, 121, 4):
    for j in range(0, 241, 60):
        x4, q5 = map_times[j], q5_values[i]
        if abs(x4 - onsets[i]) < 0.05:
            continue  # too close to the onset
        w2 = w_squared(x4, 0.5, q5, 0.0)
        found = krein_inertia(eigenspace(mode_hamiltonian(x4, 0.5, q5, 0.0), w2, +1))
        sample_ok = sample_ok and found == ((4, 4, 0) if w2 > 0 else (0, 0, 8))
        sampled += 1
```

At every fourth momentum and every 60th time (`range(0, 121, 4)` counts in steps of 4) the Krein inertia is computed directly and compared with the prediction: $(4, 4, 0)$ for a real frequency, $(0, 0, 8)$ for an imaginary one; points closer than 0.05 to the onset are skipped (`continue` jumps to the next pass of the loop).

```
report("grid points sampled for the inertia", sampled)
report("onset time for q5 = 0.001 and for q5 = 1",
       f"{onsets[0]:.4f} and {onsets[-1]:.4f}")
check(agree and sample_ok and np.all(np.isfinite(onsets)),
      "the map agrees with the exact onset; (4, 4) before it, neutral after it")
```

153 grid points are sampled; the onset times run from `6.9078` ($q_5 = 0.001$) to `0.0941` ($q_5 = 1$). The check requires the agreement, the sampled inertias, and a finite onset for every momentum (`np.isfinite`): no wave with extra-time momentum keeps a real frequency for ever.

```
from matplotlib.colors import ListedColormap  # a colour scale with two colours

fig, ax = plt.subplots(figsize=(7.0, 4.8))
ax.pcolormesh(map_times, exponents, real_frequency.astype(float),
              cmap=ListedColormap(["#f6c9b5", "#bcd6f5"]), vmin=0, vmax=1,
              shading="nearest")
inside = onsets <= map_times[-1]  # the part of the onset curve inside the map
ax.plot(onsets[inside], exponents[inside], color=GREY, linewidth=2,
        label="onset time $x_4^\\ast$")
ax.set_xlim(map_times[0], map_times[-1])
```

`ax.pcolormesh` colours every grid cell by the table (`astype(float)` turns the truth values into 0 and 1): light orange for 0 (imaginary frequency) and light blue for 1 (real frequency); `shading="nearest"` centres each cell on its grid point. The grey curve is the exact onset time, drawn where it lies inside the map.

```
for a4_slice in slices:
    ax.axvline(a4_slice / (A * H), color=GREEN, linestyle="--", linewidth=1)
ax.plot([], [], "--", color=GREEN, label="slices $a_{4,0}$ of the Kohn-Sham record")
ax.text(0.3, -1.9, "real frequency:\ninertia (4, 4)", color=BLUE, fontsize=9)
ax.text(4.4, -0.6, "imaginary frequency:\nKrein-neutral", color=ORANGE, fontsize=9)
ax.set_xlabel("time $x_4$ (units of $1/m$)")
ax.set_ylabel("$\\log_{10}$ of the extra-time momentum $q_5$")
ax.set_title("Which waves still oscillate? ($q_1 = 0.5$, $y = 0$)")
ax.legend(loc="lower left", fontsize=8)  # the lower left corner is all blue
ax.grid(False)
save_figure(fig, "onset_map",
...)
```

Green dashed lines mark the times $x_4 = a_{4,0}/(AH)$ of the five slices of the Kohn-Sham record; the two texts (with a line break written as a backslash and n) name the two regions. Figure 4 of Notebook 10f shows the blue region shrinking towards small q_5 as time goes on, bounded by the grey onset curve, which is nearly a straight line on this logarithmic scale (for small q_5, $x_4^* \approx \ln(1/q_5)$): the band of oscillating extra-time waves shrinks exponentially, and only the good sector $q_5 = 0$ stays blue for ever.

In [11], the good sector along the history.

```
slice_ok, table = True, []
for q1 in (0.5, 1.0, 2.0, 4.0):
    for a4_slice in slices:
        x4 = a4_slice / (A * H)  # the time of the slice
        h = mode_hamiltonian(x4, q1, 0.0, 0.0)
        E = np.sqrt(w_squared(x4, q1, 0.0, 0.0))
        U_plus = eigenspace(h, E ** 2, +1)  # the 8 particle columns u_s
        V_minus = eigenspace(h, E ** 2, -1)  # the 8 antiparticle columns v_s
```

For four momenta $q_1 = 0.5, 1, 2, 4$ ($q_5 = 0$, $y = 0$) and the five slices, the cell builds the instantaneous good-sector waves: the energy $E(x_4) = \sqrt{w^2}$ and orthonormal bases of the eigenspaces of $+E$ (the particle columns) and $-E$ (the antiparticle columns).

```
        sea = sum((v.conj() @ h @ v).real for v in V_minus.T)
        particle = [(u.conj() @ h @ u).real for u in U_plus.T]
        antiparticle = [-(v.conj() @ h @ v).real for v in V_minus.T]
```

The value of the filled sea, $\sum_s v_s^\dagger h v_s$ (each term is $-E$), and the normal-ordered energies of a particle, $u_s^\dagger h u_s$, and of an antiparticle, $-v_s^\dagger h v_s$ (Section 10.20).

```
        slice_ok = (slice_ok and np.allclose(h, h.conj().T) and np.allclose(B @ h, h @ B)
                    and U_plus.shape[1] == 8 and V_minus.shape[1] == 8
```

```
                    and krein_inertia(U_plus) == (4, 4, 0)
                    and krein_inertia(V_minus) == (4, 4, 0)
                    and abs(sea + 8 * E) < 1e-12
                    and np.allclose(particle, E) and np.allclose(antiparticle, E))
        table.append((q1, a4_slice, E, sea))
```

At each slice: h Hermitian, $Bh = hB$, eight columns in each eigenspace, Krein inertia $(4, 4)$ on both, the sea value $-8E$, and every quantum $+E$. A row of the table is kept for the printout and the figure.

```
say("q1    a4,0    E(x4)      sea value -8E")
for q1, a4_slice, E, sea in table:
    if q1 in (1.0, 4.0):  # print two of the four momenta
        say(f"{q1:3.1f}   {a4_slice:3.1f}   {E:9.6f}   {sea:11.6f}")
check_record(slice_ok,
             "good sector at all 5 slices: Hermitian, [B, h] = 0, +-E, (4, 4), sea -8E",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_spectrum_and_B_sectors")
```

The printed table shows, for $q_1 = 1$, the energies 1.414214, 1.169564, 1.065521, 1.024591, 1.009116 at $a_{4,0} = 0, 0.5, 1, 1.5, 2$ (that is $\sqrt{1 + e^{-2a_{4,0}}}$), with the sea values -8 times these, from -11.313708 to -8.072930; and for $q_1 = 4$ the energies from 4.123106 down to 1.137124. The check reproduces the record check `good_sector_spectrum_and_B_sectors` at every slice.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for q1, colour in ((0.5, BLUE), (1.0, ORANGE), (2.0, GREEN), (4.0, GREY)):
    E_curve = np.sqrt(w_squared(curve_times, q1, 0.0, 0.0))
    axes[0].plot(curve_times, E_curve, color=colour, linewidth=2, label=f"$q_1 = {q1}$")
    axes[1].plot(curve_times, -8 * E_curve, color=colour, linewidth=2,
                 label=f"$q_1 = {q1}$")
    dots = [(a4_slice, E, sea) for q, a4_slice, E, sea in table if q == q1]
    axes[0].plot([d[0] for d in dots], [d[1] for d in dots], "o", color=colour)
    axes[1].plot([d[0] for d in dots], [d[2] for d in dots], "o", color=colour)
```

For each momentum the curves $E(x_4)$ (left) and $-8E(x_4)$ (right) are drawn from the formula, and the values computed from the eigenvectors at the slices as dots (the time of a slice equals $a_{4,0}$, because $AH = 1$).

```
axes[0].axhline(mass, color=GREY, linestyle=":", linewidth=1)
axes[1].axhline(-8 * mass, color=GREY, linestyle=":", linewidth=1)
axes[0].set_ylabel("energy $E(x_4)$ of one quantum (units of $m$)")
axes[1].set_ylabel("value $-8E(x_4)$ of the filled sea (units of $m$)")
axes[0].set_title("particles and antiparticles: $+E(x_4) > 0$")
axes[1].set_title("the instantaneous vacuum")
for ax in axes:
    ax.set_xlabel("time $x_4$ (units of $1/m$)")
    ax.legend(fontsize=8)
save_figure(fig, "good_sector_energies",
...)
```

Dotted lines at the limits m and $-8m$. Figure 5 of Notebook 10f shows the four energy curves falling towards the mass as 3-space inflates (larger q_1 fall from higher), with the dots on them, and the sea values rising towards $-8m$: in the good sector every quantum has a positive energy at every time, and the inertia stays $(4, 4)$.

In [12], the universes of masses $+m$ and $-m$ along the history.

```
pair_ok, worst = True, 0.0
for x4 in np.linspace(0.0, 6.0, 25):
    h_plus = mode_hamiltonian(x4, 0.5, 0.05, 0.0, m=mass)
    h_minus = mode_hamiltonian(x4, 0.5, 0.05, 0.0, m=-mass)
    pair_ok = pair_ok and np.allclose(Gamma @ h_plus @ Gamma, h_minus, atol=1e-14)
    values_plus = np.linalg.eigvals(h_plus)
    values_minus = np.linalg.eigvals(h_minus)
```

At 25 times from 0 to 6, for the wave $q_1 = 0.5$, $q_5 = 0.05$, the cell builds $h_m(x_4)$ and $h_{-m}(x_4)$ (the default mass is overridden with `m=-mass`), checks $\Gamma h_m \Gamma = h_{-m}$ to within 10^{-14} (`atol` is the allowed absolute difference), and computes both spectra.

```
    # compare the sorted real parts and the sorted imaginary parts separately
    worst = max(worst,
                np.max(np.abs(np.sort(values_plus.real) - np.sort(values_minus.real))),
                np.max(np.abs(np.sort(values_plus.imag) - np.sort(values_minus.imag))))
report("largest difference of the eigenvalues for +m and -m", f"{worst:.1e}")
check_record(pair_ok and worst < 1e-9,
             "Gamma h_m(x4) Gamma = h_-m(x4) at every time: same spectra and onset",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.one_particle_maps")
```

The sorted real parts and imaginary parts are compared separately, as the comment says (the times include the onset region, where the eigenvalues are complex). The largest difference is printed as `7.1e-15`, and the check reproduces the record check `Q.one_particle_maps` along the whole history.

In [13], the last check.

```
for name in ("10f_1_hermiticity_defect.png", "10f_2_eigenvalues_along_history.png",
             "10f_3_inertia_along_history.png", "10f_4_onset_map.png",
             "10f_5_good_sector_energies.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The five figure files exist, and the last line prints ALL 19 CHECKS PASSED (notebook 10f): the five figure checks and the 14 checks of the cells before (1 each in In [2], In [3], In [5] to In [8], In [10], In [11] and In [12], 3 in In [4] and 2 in In [9]).

10.44 The quantum reading of the pairing

The pairing theorem T1 of the Revision record (proved in Chapter 18) says that the chirality matrix maps every solution Ψ of the field with mass m and coupling λ to a solution $\Gamma\Psi$ of the field with

$-m$ and $-\lambda$, with the energy-momentum tensor and the current reversed: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$, $T \to -T$, $J \to -J$. Theorem T2 combines Γ with a reflection; in the author's field it is the mirror $\gamma^{(x_0)}$ across the ASSUMED Z2 brane, and it pairs (m, λ) with $(-m, \lambda)$ at equal energy. What do these maps mean once dirac16complex is QUANTISED? The Revision pairing record answers with five statements, the **quantum reading Q** (record `Revision/pairing/pairing-theory.json`, theorem entry `Q`; Revision proof document, section 6). This section derives them, in flat 4+4 space at one point with $U = 0$ (so $\lambda = 0$; the record states them for general λ).

The hypotheses. (HQ1) dirac16complex is canonically quantised with x_4 as the evolution time; the x_4-derivative kernel of its Lagrangian is $N = B$ and the canonical anticommutator is $\{\Psi, \Psi^\dagger\} = N^{-1} = B$ at one point (Section 10.12). (HQ2) dirac16complex00 is a classical field and has no quantum reading.

Two sign rules. Γ anticommutes with every gamma, so moving it through a product of n gammas costs $(-1)^n$; with $\Gamma\Gamma = I_{16}$:

$$\Gamma C \Gamma = C \ (n = 4), \qquad \Gamma C \gamma^{(x_a)} \Gamma = -C\gamma^{(x_a)} \ (n = 5), \qquad \Gamma B \Gamma = -B \ (n = 5).$$

The Lagrangian identity, line by line. In flat space with $U = 0$, $\mathcal{L}_m[\Psi] = \frac{1}{2}\sum_a (\Psi^\dagger C\gamma^{(x_a)}\partial_a\Psi - \partial_a\Psi^\dagger C\gamma^{(x_a)}\Psi) - m\Psi^\dagger C\Psi$. Insert $\Psi = \Gamma\chi$ and $\Psi^\dagger = \chi^\dagger\Gamma$ (Γ is real and symmetric):

$$\mathcal{L}_m[\Gamma\chi] = \tfrac{1}{2}\sum_a \left(\chi^\dagger\Gamma C\gamma^{(x_a)}\Gamma\,\partial_a\chi - \partial_a\chi^\dagger\,\Gamma C\gamma^{(x_a)}\Gamma\chi\right) - m\,\chi^\dagger\Gamma C\Gamma\chi.$$

The two sign rules turn this into

$$\mathcal{L}_m[\Gamma\chi] = -\tfrac{1}{2}\sum_a \left(\chi^\dagger C\gamma^{(x_a)}\partial_a\chi - \partial_a\chi^\dagger C\gamma^{(x_a)}\chi\right) - m\,\chi^\dagger C\chi.$$

Take the sign -1 out of both terms; the mass term becomes $+m\chi^\dagger C\chi = -(-m)\chi^\dagger C\chi$, so the bracket is the Lagrangian with the mass $-m$:

$$\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi].$$

(Q1) The chirality image carries the Krein metric $-B$. By Section 10.15, $\{\Gamma\Psi, (\Gamma\Psi)^\dagger\} = \Gamma B\Gamma^\dagger = -B$. The image's own Lagrangian says the same: in $\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi]$ the time-derivative terms of the variable χ have the kernel $\Gamma B\Gamma = -B$, and canonical quantisation of χ gives $\{\chi, \chi^\dagger\} = (-B)^{-1} = -B$. The two agree.

(Q2) The image is the same quantum system. For one plane wave the energy is $H_m[\Psi] = \Psi^\dagger h'_m\Psi$ with $h'_m = mC - i\sum_a k_a C\gamma^{(x_a)}$ (Section 10.12), and the charge is $Q[\Psi] = \Psi^\dagger B\Psi$. By the sign rules,

$$\Gamma h'_m\Gamma = mC + i\sum_a k_a C\gamma^{(x_a)} = -\Big((-m)C - i\sum_a k_a C\gamma^{(x_a)}\Big) = -h'_{-m}.$$

Written in the image variable $\chi = \Gamma\Psi$ (so $\Psi = \Gamma\chi$),

$$H_m[\Psi] = \chi^\dagger \Gamma h'_m \Gamma \chi = -\chi^\dagger h'_{-m} \chi = -H_{-m}[\Gamma\Psi], \qquad Q[\Psi] = \chi^\dagger \Gamma B \Gamma \chi = -Q[\Gamma\Psi].$$

The energy and the charge of the field are MINUS the energy and the charge that the mass $-m$ theory assigns to the image: $T \to -T$ and $J \to -J$ of T1 are identities between operators of ONE quantum system. The image is not a second universe with negative energy; it is the first universe written in other variables. Its motion agrees. In its own variable $\chi = \Gamma\Psi$ the image has the metric $-B$ (Q1) and, by the display above ($H_m[\Psi] = -\chi^\dagger h'_{-m}\chi$), the generator density $-h'_{-m}$; its one-particle generator is the product of the two, as in Section 10.12, so it evolves by

$$(-B)(-h'_{-m}) = B h'_{-m} = h_{-m},$$

the wave equation of mass $-m$. Written back in the variable $\Psi = \Gamma\chi$, the same motion is $\Gamma h_{-m}\Gamma = (\Gamma B\Gamma)(\Gamma h'_{-m}\Gamma) = (-B)(-h'_m) = B h'_m = h_m$ (insert $\Gamma\Gamma = I_{16}$ between the factors; $\Gamma h'_{-m}\Gamma = -h'_m$ is the identity $\Gamma h'_m \Gamma = -h'_{-m}$ with m replaced by $-m$): the dynamics of Ψ itself. The Revision record states the same identity in the other direction, for the image of a field of mass $-m$: that image carries the metric $-B$ and the generator density $-h'_m$ and evolves by $(-B)(-h'_m) = B h'_m = h_m$, the dynamics of mass m (`python-pairing.json`, check `Q.image_generators_same_dynamics`). In terms of the image field, $i\,\partial_4(\Gamma\Psi) = \Gamma h_m \Psi = (\Gamma h_m \Gamma)(\Gamma\Psi) = h_{-m}\Gamma\Psi$: the image obeys the wave equation of mass $-m$ and is moved by the SAME energy operator $H_m[\Psi]$.

(Q3) An independent universe of mass $-m$, and no cancellation. An independently quantised universe of mass $-m$ is a NEW field Ψ_2 with its own Lagrangian $\mathcal{L}_{-m}[\Psi_2]$. The mass does not enter the time-derivative kernel, so its kernel is $+B$ and $\{\Psi_2, \Psi_2^\dagger\} = +B$; being independent of the first universe Ψ_1, it anticommutes with it: $\{\Psi_{2A}, \Psi_{1C}^\dagger\} = 0$ and $\{\Psi_{2A}, \Psi_{1C}\} = 0$. The image $\Gamma\Psi_1$ fails both requirements: it carries $-B$, not $+B$, and $\{(\Gamma\Psi_1)_A, \Psi_{1C}^\dagger\} = \sum_D \Gamma_{AD} B_{DC} = (\Gamma B)_{AC}$, a matrix of rank 16 (it is invertible), not 0. So no identification $\Psi_2 = \Gamma\Psi_1$ exists. The same holds for the other images built from Ψ_1: the mirror image $\gamma^{(x_0)}\Psi_1$ has the right rule $+B$ but $\{\gamma^{(x_0)}\Psi_1, \Psi_1^\dagger\} = \gamma^{(x_0)} B \neq 0$, and the conjugate field $\Psi^c = \Gamma\Psi_1^{\dagger T}$ has $+B$ (Section 10.15) but $\{\Psi^c_A, \Psi_{1C}\} = \sum_D \Gamma_{AD} B_{CD} = (\Gamma B^T)_{AC} = -(\Gamma B)_{AC} \neq 0$. A second universe must live on a larger state space. There the total energy is $H_1 + H_2$, the generators ADD ($P_{\text{total}} = P_1 \otimes 1 + 1 \otimes P_2$ on the product space), and the T1 identity relates operators of universe 1 only; it gives no relation $P_2 = -P_1$. The one-particle generator of the two universes is the block matrix $\mathrm{diag}(B h'_m, B h'_{-m})$. At zero momentum $B h'_{\pm m} = \pm m BC$, and

$$BC = -iC\gamma^{(x_4)}C = -iCC\gamma^{(x_4)} = -i\gamma^{(x_4)}, \qquad (BC)^2 = -\gamma^{(x_4)}\gamma^{(x_4)} = I_{16}, \qquad \mathrm{tr}(BC) = 0,$$

by (B1) of Section 10.2 and $CC = I_{16}$; so BC has the eigenvalues $+1$ and -1, eight each, and the block generator has the eigenvalues $+m$ and $-m$, sixteen each: none is 0 for $m \neq 0$. With a momentum they are $\pm\sqrt{m^2 + k^2}$, sixteen each. After normal ordering each universe has quanta of energy $+E > 0$ (Section 10.20), so every state of the two universes has a total energy of at least 0, and only the vacuum has 0. A particle in universe 1 (charge $+1$) and an antiparticle in universe 2 (charge -1) have the total charge 0 and the total energy $2E$, NOT 0.

(Q4) Identical one-particle spectra. $\Gamma h_m \Gamma = h_{-m}$ and $\gamma^{(x_0)} h_m(k) \gamma^{(x_0)} = h_{-m}(R_0 k)$, so the one-particle spectra of the universes of masses $+m$ and $-m$ are identical, not opposite (Section 10.6), and along the deflating history at every instant (Section 10.39).

(Q5) The mirror image keeps $+B$. $\gamma^{(x_0)}B(\gamma^{(x_0)})^\dagger = +B$ (Section 10.15): the mirror universe of T2, with $(-m, \lambda)$, can be quantised as an ordinary independent copy with equal energies.

statement	status	where it is verified
(HQ1) kernel $N = B$, $\{\Psi, \Psi^\dagger\} = B\delta^7/\sqrt{\lvert g\rvert}$	PROVED	`wolfram-pairing.json`, checks `Q_symplectic_kernel_grassmann` and `Q_symplectic_kernel_commuting`; `python-pairing.json`, check `Q.canonical_anticommutator`
(Q1) the image carries $-B$, also from its own Lagrangian	PROVED	`wolfram-pairing.json`, check `Q_Krein_metric_of_images`; `python-pairing.json`, checks `Q.image_krein_metric` and `Q.image_own_quantisation`
(Q2) the image's generators are those of Ψ; $T \to -T$, $J \to -J$ inside one system	PROVED	`wolfram-pairing.json`, checks `Q_generators_of_the_image_grassmann` and `Q_generators_of_the_image_commuting`; `python-pairing.json`, check `Q.image_generators_same_dynamics`
(Q3) no identification with an independent universe; generators add; block eigenvalues $\pm m$, 16 each	PROVED	`wolfram-pairing.json`, check `Q_no_identification_of_independent_universes`; `python-pairing.json`, check `Q.no_cancellation_independent_universes`
(Q4) identical one-particle spectra; inertia $(4, 4)$ at real frequencies	PROVED	Sections 10.5 and 10.6; `wolfram-pairing.json`, checks `Q_one_particle_maps` and `Q_one_particle_Krein_inertia_real_frequencies`; `python-pairing.json`, checks `Q.one_particle_maps` and `Q.one_particle_Krein_inertia_proof`
(Q5) the mirror image keeps $+B$	PROVED	`python-pairing.json`, check `Q.T2_image_keeps_B`
the two engines agree on Q	PROVED	`python-pairing.json`, check `compare.theory.theorem_Q` (10 sympy checks confirm the 12 Wolfram verifications)

The statements Q1 to Q5 are statements about the canonical anticommutator; they neither use nor establish a positive state space for either universe (the positive Fock spaces of this chapter are built for single good-sector momenta with frozen coefficients).

10.45 What the quantum reading does not establish

The author asked to prove that universes of masses $+m$ and $-m$ are created in pairs. The honest answer, as far as this chapter reaches, is the following.

What is proved. The quantum reading Q1 to Q5 of Section 10.44, under the hypotheses HQ1 and HQ2, and the identical one-particle spectra of the universes of masses $+m$ and $-m$ (Sections 10.6 and 10.39). Like the classical theorems T1 and T2 (Chapter 18) they are exact MAPS BETWEEN SOLUTIONS and statements about operators; they say what the quantum theory does with a solution, its image and a second universe.

What is NOT established (the Revision record lists the same points, written independently by the two verification engines: `python-pairing.json`, check `compare.theory.not_established`):

1. No creation process. Nothing in these equations produces a universe, a pair of universes or a change of the number of universes; no transition from "no universe" to "two universes", no initial state, no vacuum decay (the transition of a state of lowest energy into another state of still lower energy) and no tunnelling process (a quantum transition through a barrier that the classical motion cannot cross) is derived.

2. No rate, probability or amplitude. No transition amplitude, probability, rate or Bogoliubov coefficient for creating universes of masses $+m$ and $-m$ is computed or implied (when the background changes in time, a **Bogoliubov coefficient** measures how much of a wave of negative frequency at a later time is contained in a wave of positive frequency at an earlier time; its square counts the quanta that the change creates). No wave function of the universe (in quantum cosmology, a quantum state of the geometry of the whole universe, from which probabilities for universes are computed) is part of these statements.

3. No dynamical necessity. No equation and no conservation law forces the partner to exist: a single universe of mass $+m$ is an equally valid solution without its partner.

4. No cancellation between two universes. The vanishing of the total energy-momentum and charge of a T1 pair holds for classical bilinears and, as Q2 shows, as an operator identity within ONE quantum system. For two independently quantised universes the generators ADD; a pair of total charge 0 has the positive energy $2E$ (Section 10.44).

5. No positive quantum theory of the whole field. The canonical rule forces a Krein space (Theorem 10.2); in flat 4+4 space every real-frequency eigenspace has Krein inertia $(4, 4)$; a positive Fock space is constructed and checked only for single good-sector momenta with frozen coefficients; the extra-time sector grows and becomes Krein-neutral along the deflating history (Section 10.39); the curved good sector needs a boundary condition at $z = \pi/2$ that the quantisation record does not impose (Section 10.27).

6. The gravitational field is a fixed background, the same for both members; the back-reaction through the field equations for a_4 is not part of these statements, and the history $a_4 = AHx_4$ used in Section 10.39 is a PRESCRIBED BACKGROUND.

7. dirac16complex00 is a classical field; no quantum statement is made for it.

The hypothesis. That the big bang creates universes in pairs of masses $+m$ and $-m$ is the author's HYPOTHESIS. This chapter neither supports nor refutes it: it contains no creation process. Chapter 20 states the hypothesis, what the equations prove, and what they do not, for both fields; Chapter 21 treats matter and antimatter, where the conjugation of the quantised field $\Psi \to \Gamma\Psi^{\dagger T}$ of Section 10.15 (which reverses the mass) appears again, and states precisely that the theory as built does not solve the matter-antimatter problem.

10.46 Example: Notebook 10g checks the quantum reading of the pairing

Notebook 10g proves the Lagrangian identity $\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi]$ for symbolic field components and computes the three time-derivative kernels B, $-B$ and $+B$ (Q1, Q3); checks on a Fock space that the image carries $-B$ and the mirror image $+B$ (Q1, Q5); checks $H_m[\Psi] = -H_{-m}[\Gamma\Psi]$ and $Q[\Psi] = -Q[\Gamma\Psi]$ as operator identities on 150 random states and that the image obeys the wave equation of mass $-m$ under the same energy operator (Q2); tests three candidates for an independent universe (Q3); builds two independently quantised universes on 32 fermion modes, computes their block generator, and counts all 2^{32} states of one good-sector momentum in both universes (Q3). It draws five figures and ends with the line ALL 30 CHECKS PASSED (notebook 10g).

10.47 How to run Notebook 10g

Step 1. What this notebook does and what it needs.

Notebook 10g (The quantum reading of the pairing: one system or two universes) is the file `Revision /textbook/notebooks/10g_quantum_pair_reading.ipynb` of the repository Dirac_claude. It checks, exactly with sympy and on explicit fermionic Fock spaces, the quantum reading Q of the Revision pairing record: the Lagrangian identity L_m of Gamma chi equals minus L_-m of chi, the time-derivative kernels of the three Lagrangians involved (the field, its chirality image, an independent universe of mass -m), the canonical anticommutators they force (+B, -B, +B), that the chirality image Gamma Psi carries the Krein metric -B and the mirror image keeps +B, that the energy and the charge of the field are minus those that the mass -m theory assigns to its image (one quantum system, T to -T and J to -J as operator identities), that two independently quantised universes of masses +m and -m have the anticommutator +B each, anticommute with each other and cannot be identified with the image, and that their generators add without cancelling (block eigenvalues +m and -m, 16 each; one good-sector momentum: every state of the product Fock space has a non-negative total energy, a pair of zero total charge has energy 2E); five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs

the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is

changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10g_quantum_pair_reading.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10g_quantum_pair_reading.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 10g_quantum_pair_reading.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/10g.captions.json`

- `Revision/textbook/figures/10g_1_krein_metrics.png`
- `Revision/textbook/figures/10g_2_one_system.png`
- `Revision/textbook/figures/10g_3_anticommutator_blocks.png`
- `Revision/textbook/figures/10g_4_block_generator.png`
- `Revision/textbook/figures/10g_5_two_universe_states.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10g_5_two_universe_states.png exists
ALL 30 CHECKS PASSED (notebook 10g)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/10g_quantum_pair_reading.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 10g_quantum_pair_reading.ipynb
python -m nbconvert --execute --inplace 10g_quantum_pair_reading.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

10.48 Notebook 10g: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/10g_quantum_pair_reading.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The pairing theorem T1 of the Revision record says that the chirality matrix
$\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$ turns every solution
$\Psi$ of the field of mass $m$ (and coupling $\lambda$) into a solution
$\Gamma\Psi$ of the field of mass $-m$ (and coupling $-\lambda$), with the
```

```
energy-momentum tensor and the current reversed: $T \to -T$, $J \to -J$. What
does this mean once dirac16complex is QUANTISED? The Revision pairing record
answers with five statements, the *quantum reading* Q. This notebook checks the
four of them that concern the canonical anticommutator, exactly with sympy and on
explicit fermionic Fock spaces (the fifth, about one-particle spectra, is the
subject of the first notebook of this chapter). It

- proves the Lagrangian identity $\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi]$
  for symbolic field components, and computes the time-derivative kernels of three
  Lagrangians: that of the field $\Psi$, that of its chirality image $\chi =
  \Gamma\Psi$ (the same system written in new variables) and that of an
  independent universe of mass $-m$; they force the canonical anticommutators
  $+B$, $-B$ and $+B$;
- checks on a Fock space that the image $\Gamma\Psi$ carries the Krein metric $-B$,
  while the mirror image $\gamma^{(x_0)}\Psi$ of theorem T2 keeps $+B$;
- checks that the energy and the charge of the field are MINUS the energy and the
  charge that the mass $-m$ theory assigns to the image: $T \to -T$ and $J \to -J$
  hold as identities between operators of ONE quantum system;
- builds two independently quantised universes of masses $+m$ and $-m$ on one Fock
  space and checks that each has the anticommutator $+B$, that they anticommute
  with each other, and that the image $\Gamma\Psi$ cannot be one of them;
- shows that their energies ADD and never cancel: the block generator has the
  eigenvalues $+m$ and $-m$ (16 each, none zero), and for one good-sector momentum
  every state of the two universes has a total energy of at least 0, while a pair
  of total charge 0 has the energy $2E$, not 0;
- reproduces the corresponding checks of the Revision record and draws five
  teaching figures.

Nothing here describes the CREATION of a universe or of a pair: the statements are
about how the canonical rule treats a solution, its image and a second universe.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 10g (The quantum reading of the pairing: one system or two universes) is the
file ‘Revision/textbook/notebooks/10g_quantum_pair_reading.ipynb‘ of the repository
Dirac_claude. It checks, exactly with sympy and on explicit fermionic Fock spaces, the
quantum reading Q of the Revision pairing record: the Lagrangian identity L_m of Gamma
chi equals minus L_-m of chi, the time-derivative kernels of the three Lagrangians
involved (the field, its chirality image, an independent universe of mass -m), the
canonical anticommutators they force (+B, -B, +B), that the chirality image Gamma Psi
carries the Krein metric -B and the mirror image keeps +B, that the energy and the charge
of the field are minus those that the mass -m theory assigns to its image (one quantum
system, T to -T and J to -J as operator identities), that two independently quantised
universes of masses +m and -m have the anticommutator +B each, anticommute with each
other and cannot be identified with the image, and that their generators add without
cancelling (block eigenvalues +m and -m, 16 each; one good-sector momentum: every state
of the product Fock space has a non-negative total energy, a pair of zero total charge
has energy 2E); five teaching figures. It needs a computer with Windows 11, macOS or
Linux, an internet connection for the installation, the program Git, and Python 3.12 or
```

newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 10g_quantum_pair_reading.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 10g_quantum_pair_reading.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 10g_quantum_pair_reading.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/10g.captions.json‘
- ‘Revision/textbook/figures/10g_1_krein_metrics.png‘
- ‘Revision/textbook/figures/10g_2_one_system.png‘
- ‘Revision/textbook/figures/10g_3_anticommutator_blocks.png‘
- ‘Revision/textbook/figures/10g_4_block_generator.png‘
- ‘Revision/textbook/figures/10g_5_two_universe_states.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 10g_5_two_universe_states.png exists
ALL 30 CHECKS PASSED (notebook 10g)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/10g_quantum_pair_reading.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for

programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 10g_quantum_pair_reading.ipynb
python -m nbconvert --execute --inplace 10g_quantum_pair_reading.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for a file below the folder Revision: the notebook reads files of the Revision record: the data it starts from and the report files whose checks it reproduces (each cited check must still exist there with the verdict PASS). It must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.
- "AssertionError: the cited record check is missing or not PASS": a report file of the Revision record no longer holds the check that the notebook names, or holds it with another verdict: the copy of the repository is incomplete or was changed. Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 10g (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 10g (The quantum reading of the pairing: one system or two universes) is the
# file Revision/textbook/notebooks/10g_quantum_pair_reading.ipynb of the repository
# Dirac_claude. It checks, exactly with sympy and on explicit fermionic Fock spaces, the
# quantum reading Q of the Revision pairing record: the Lagrangian identity L_m of Gamma
# chi equals minus L_-m of chi, the time-derivative kernels of the three Lagrangians
# involved (the field, its chirality image, an independent universe of mass -m), the
# canonical anticommutators they force (+B, -B, +B), that the chirality image Gamma Psi
# carries the Krein metric -B and the mirror image keeps +B, that the energy and the
# charge of the field are minus those that the mass -m theory assigns to its image (one
# quantum system, T to -T and J to -J as operator identities), that two independently
# quantised universes of masses +m and -m have the anticommutator +B each, anticommute
# with each other and cannot be identified with the image, and that their generators add
# without cancelling (block eigenvalues +m and -m, 16 each; one good-sector momentum:
# every state of the product Fock space has a non-negative total energy, a pair of zero
```

```
# total charge has energy 2E); five teaching figures. It needs a computer with Windows
# 11, macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
```

```
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
```

```
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 10g_quantum_pair_reading.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 10g_quantum_pair_reading.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 10g_quantum_pair_reading.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/10g.captions.json
# - Revision/textbook/figures/10g_1_krein_metrics.png
# - Revision/textbook/figures/10g_2_one_system.png
# - Revision/textbook/figures/10g_3_anticommutator_blocks.png
# - Revision/textbook/figures/10g_4_block_generator.png
# - Revision/textbook/figures/10g_5_two_universe_states.png
```

```
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 10g_5_two_universe_states.png exists
#     ALL 30 CHECKS PASSED (notebook 10g)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/10g_quantum_pair_reading.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 10g_quantum_pair_reading.ipynb
#     python -m nbconvert --execute --inplace 10g_quantum_pair_reading.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
```

```
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for a file below the folder Revision: the notebook reads files of
#   the Revision record: the data it starts from and the report files whose checks it
#   reproduces (each cited check must still exist there with the verdict PASS). It must
#   be opened inside the folder Revision/textbook/notebooks of a complete copy of the
#   repository (a notebook copied alone to another folder cannot find them). Clone the
#   repository again and open the notebook there.
# - "AssertionError: the cited record check is missing or not PASS": a report file of the
#   Revision record no longer holds the check that the notebook names, or holds it with
#   another verdict: the copy of the repository is incomplete or was changed. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "10g"  # this notebook: chapter 10, example g


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 10g complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Chirality image**: for a field $\Psi$, the field $\chi = \Gamma\Psi$. Because
  $\Gamma^2 = I_{16}$, also $\Psi = \Gamma\chi$: the image carries exactly the
  same information, written in other variables.
- **Mirror image**: the field $\gamma^{(x_0)}\Psi$ (with the hidden direction
  reflected) of the pairing theorem T2.
- **Lagrangian $\mathcal{L}_m[\Psi]$**: the Lagrangian density of dirac16complex
  with mass $m$, evaluated on the field $\Psi$. Here (flat 4+4 space, $U = 0$) it is
  $\frac12\sum_a(\Psi^\dagger C\gamma^{(x_a)}\partial_a\Psi -
  \partial_a\Psi^\dagger C\gamma^{(x_a)}\Psi) - m\Psi^\dagger C\Psi$.
- **Time-derivative kernel** $N$: the matrix in the terms of a Lagrangian that
  contain $\partial_4$, written as $\frac{i}{2}(\Psi^\dagger N\partial_4\Psi -
  \partial_4\Psi^\dagger N\Psi)$. Canonical quantisation turns it into the
  anticommutator $\{\Psi_A, \Psi^\dagger_C\} = (N^{-1})_{AC}$ (at one point; derived
  in the second notebook of this chapter).
- **Anticommutator matrix**: for two lists of operators $X_1, \dots, X_n$ and
  $Y_1, \dots, Y_n$ whose anticommutators are numbers, the matrix with entries
  $\{X_i, Y_j\}$.
```

```
- **Independent universes**: two quantum fields $\Psi_1, \Psi_2$ on one state
  space with $\{\Psi_{1A}, \Psi_{2C}^\dagger\} = 0$ and $\{\Psi_{1A}, \Psi_{2C}\} = 0$:
  nothing done to one changes the anticommutation rules of the other.
- **Product Fock space**: the Fock space of the modes of both universes together,
  here $16 + 16 = 32$ fermion modes ($2^{32}$ occupation patterns).
- **Generator**: the operator (or, for one particle, the matrix) that moves a
  system in time; the energy is the generator of time translations.
- **Rank** of a matrix: the number of independent columns; a matrix of rank 16 out
  of 16 has no column that is a combination of the others, in particular it is not 0.
- **Normal-ordered energy**: the energy of a state minus the energy of the vacuum
  (the filled sea), as in the third notebook of this chapter.
- **Status words**: PROVED (exact, for all values), CHECKED (exact or to rounding,
  for the values shown), REPRODUCED (agrees with a Revision record), NOT
  ESTABLISHED (not shown by these equations).
```

Text cell:

```
## 4. The physical and mathematical situation

Everything is at one point of flat 4+4 space (the same equations hold in the
frozen-coefficient model, an ASSUMPTION that leaves out the hidden-direction terms
of the author's field equation), $U = 0$, and with the conventions of the earlier
notebooks of this chapter: $C =
\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, $B = -iC\gamma^{(x_4)}$,
$\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)} = \mathrm{diag}(-I_8,
I_8)$, real, symmetric, $\Gamma^2 = I_{16}$.

**Two sign rules.** $\Gamma$ anticommutes with every gamma. Moving $\Gamma$ through
a product of $n$ gammas therefore costs the sign $(-1)^n$:

$$\Gamma C\Gamma = C\ (n = 4),\qquad \Gamma C\gamma^{(x_a)}\Gamma = -C\gamma^{(x_a)}\
(n = 5),\qquad \Gamma B\Gamma = -B\ (n = 5).$$

The mirror matrix $\gamma^{(x_0)}$ commutes with itself and anticommutes with the
four other gammas in $B = -i\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}
\gamma^{(x_4)}$, so $\gamma^{(x_0)}B\gamma^{(x_0)} = (-1)^4B = +B$.

**The Lagrangian identity, line by line.** Insert $\Psi = \Gamma\chi$ (and
$\Psi^\dagger = \chi^\dagger\Gamma$, because $\Gamma$ is real and symmetric) into
$\mathcal{L}_m$:

$$\mathcal{L}_m[\Gamma\chi] = \tfrac12\sum_a(\chi^\dagger\Gamma C\gamma^{(x_a)}
\Gamma\partial_a\chi - \partial_a\chi^\dagger\Gamma C\gamma^{(x_a)}\Gamma\chi) -
m\chi^\dagger\Gamma C\Gamma\chi .$$

Use the two sign rules:

$$\mathcal{L}_m[\Gamma\chi] = -\tfrac12\sum_a(\chi^\dagger C\gamma^{(x_a)}
\partial_a\chi - \partial_a\chi^\dagger C\gamma^{(x_a)}\chi) - m\chi^\dagger C\chi .$$

Take the sign $-1$ out of both terms; the mass term becomes $+m\chi^\dagger C\chi =
-(-m)\chi^\dagger C\chi$, so the bracket is the Lagrangian with mass $-m$:
```

$$\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi] .$$

Three kernels. In $\mathcal{L}_m[\Psi]$ the ∂_4 terms are $\frac12(\Psi^\dagger C\gamma^{(x_4)}\partial_4\Psi - \ldots)$, and $C\gamma^{(x_4)} = iB$, so $N = B$ and $\{\Psi, \Psi^\dagger\} = B^{-1} = B$. The SAME system written in the variable χ has the Lagrangian $\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi]$, whose kernel is $\Gamma B\Gamma = -B$: canonical quantisation of the image gives $\{\chi, \chi^\dagger\} = -B$, which agrees with the map, $\{\Gamma\Psi, (\Gamma\Psi)^\dagger\} = \Gamma B\Gamma = -B$. An INDEPENDENT universe of mass $-m$ has its own Lagrangian $\mathcal{L}_{-m}[\Psi_2]$ with kernel $+B$ (the mass does not enter the kernel), hence $\{\Psi_2, \Psi_2^\dagger\} = +B$. Since $B \neq -B$, the image of universe 1 cannot be such a universe.

Energy and charge. For one plane wave with momenta k_a ($a \neq 4$) the energy of the field is $H = \Psi^\dagger h'_m\Psi$ with $h'_m = mC - i\sum_a k_aC \gamma^{(x_a)}$ (second notebook of this chapter), and its charge is $Q = \Psi^\dagger B\Psi$. Written in the image variable:

$$H = \chi^\dagger\Gamma h'_m\Gamma\chi = \chi^\dagger(mC + i\sum_a k_aC \gamma^{(x_a)})\chi = -\chi^\dagger h'_{-m}\chi,\qquad Q = \chi^\dagger\Gamma B \Gamma\chi = -\chi^\dagger B\chi .$$

So the energy and the charge of the field are MINUS the energy and the charge that the mass $-m$ theory assigns to the image: this is $T \to -T$, $J \to -J$ of T1 as an identity between operators of one system. The image is not a second universe with negative energy; it is the first universe described in other variables.

Two independent universes. On the product Fock space put Ψ_1 (mass m, anticommutator $+B$) and Ψ_2 (mass $-m$, anticommutator $+B$) with $\{\Psi_1, \Psi_2^\dagger\} = 0$. The total energy is $H_1 + H_2$, and its one-particle generator is the block matrix $\mathrm{diag}(Bh'_m, Bh'_{-m})$. At zero momentum $Bh'_{\pm m} = \pm mBC$; since $(BC)^2 = -C\gamma^{(x_4)}CC \gamma^{(x_4)}C = I_{16}$ and $\mathrm{tr}(BC) = 0$, the matrix BC has the eigenvalues $+1$ and -1, eight each, and the block generator has the eigenvalues $+m$ and $-m$, sixteen each: none is 0 for $m \neq 0$. After normal ordering each universe has quanta of energy $+E > 0$ (third notebook of this chapter), so the total energy of every state is at least 0, and equals 0 only in the vacuum. A particle in universe 1 (charge $+1$) and an antiparticle in universe 2 (charge -1) have the total charge 0 but the total energy $2E$.

Text cell:

5. The matrices and the statements of the record

The next cell reads the author's gamma matrices, builds C, Γ and B (exact whole numbers and the imaginary unit '1j'), checks the sign rules of Section 4, and reads the theorem entry Q of the Revision pairing record 'Revision/pairing/pairing-theory.json': its hypothesis and its five statements. Finally it reads the check 'compare.theory.theorem_Q' of the sympy pairing report 'Revision/pairing/reports/python-pairing.json', in which the two verification engines are compared: it says that the 12 Wolfram verifications of theorem Q are confirmed by 10 sympy checks, and names them; the cell confirms these numbers and

that each of the 10 named checks exists with the verdict PASS.

It also defines ‘check_record(condition, name, record)‘: it does what
‘check(condition, name, record=record)‘ of the set-up cell does, but prints the PASS
line and the line naming the reproduced Revision record in one piece (Jupyter
sends printed text to the screen in pieces whose boundaries depend on timing;
printing them in one piece keeps the stored output the same in every run).
Before it checks anything, ‘check_record‘ asks the helper ‘record_says_pass‘ whether
the cited record still says PASS: for a record name of the form
‘<report file>, check <check name>‘ it opens that report (each file once, kept in
the dictionary ‘REPORT_CHECKS‘) and stops the notebook with an error unless a check
of that name exists there with the verdict PASS; a record name without ‘, check ‘
names a data entry that the notebook reads and compares itself.

In [2]:

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout
from math import comb  # comb(8, j): the number of ways to choose j of 8 things

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols
from matplotlib.colors import LinearSegmentedColormap  # colour scales


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    """True if the cited report check exists today with the verdict PASS."""
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    """check(condition, name, record=record), printed in one piece."""
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output


BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])  # -1 blue, 0 grey, +1 red
```

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
# gamma[a] is the matrix gamma^(x_a) of the coordinate x_a, a = 1, ..., 8.
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])  # the chirality matrix
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
I16 = np.eye(16)
diagonal = np.array([-1] * 8 + [1] * 8)  # the expected diagonal of Gamma
sign_rules = (np.array_equal(Gamma, np.diag(diagonal))
              and np.array_equal(Gamma @ C @ Gamma, C)
              and all(np.array_equal(Gamma @ C @ gamma[a] @ Gamma, -C @ gamma[a])
                      for a in range(8)))
check(sign_rules, "Gamma = diag(-I8, I8), Gamma C Gamma = C, Gamma C g^a Gamma = -C g^a")

pairing = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
theorem_Q = next(t for t in pairing["theorems"] if t["id"] == "Q")
say(f"record entry Q: {theorem_Q['name']}; {len(theorem_Q['hypotheses'])} "
    f"hypothesis, {len(theorem_Q['statement'])} statements")
for number, statement in enumerate(theorem_Q["statement"], 1):
    say(f"Q{number}: {statement[:70]} ...")  # the first 70 characters of each
check(len(theorem_Q["statement"]) == 5, "the record states five quantum statements")

compare_file = "Revision/pairing/reports/python-pairing.json"
compare_checks = json.loads(repository_file(compare_file).read_text(
    encoding="utf-8"))["checks"]
detail = next(c["detail"] for c in compare_checks
              if c["name"] == "compare.theory.theorem_Q")
confirming = detail.split("all PASS: ")[1].split(", ")  # the sympy checks named
say("compare.theory.theorem_Q: " + detail.split(" is independently")[0])
say(f"confirmed by {len(confirming)} sympy checks: {confirming[0]}, ...")
check_record("5 statements, 12 Wolfram verifications" in detail
             and "confirmed by 10 sympy checks" in detail and len(confirming) == 10
             and all(record_says_pass(f"{compare_file}, check {name}")
                     for name in confirming),
             "theorem Q: 10 sympy checks, all PASS, confirm 12 Wolfram verifications",
             record=f"{compare_file}, check compare.theory.theorem_Q")
```

Out [2]:

```
PASS Gamma = diag(-I8, I8), Gamma C Gamma = C, Gamma C g^a Gamma = -C g^a
record entry Q: quantum-level reading; 1 hypothesis, 5 statements
Q1: the chirality image chi = Gamma Psi carries the Krein metric -B: {chi, ...
Q2: the image's own generators (energy, momentum, charge from its own Lagr ...
Q3: an independently quantised (-m,-lambda) universe has the anticommutato ...
Q4: the one-particle Hamiltonians satisfy Gamma h_m Gamma = h_(-m) (flat s ...
Q5: the T2 image gamma^(x0) Psi keeps the anticommutator +B: the mirror (- ...
PASS the record states five quantum statements
compare.theory.theorem_Q: Wolfram theorem Q (quantum-level reading; 5 statements, 12
    Wolfram verifications)
```

```
confirmed by 10 sympy checks: Q.canonical_anticommutator, ...
PASS theorem Q: 10 sympy checks, all PASS, confirm 12 Wolfram verifications
     reproduces Revision/pairing/reports/python-pairing.json, check
    compare.theory.theorem_Q
```

Text cell:

```
The next cell computes the Krein metrics carried by the two images,
$\Gamma B\Gamma^\dagger$ and $\gamma^{(x_0)}B(\gamma^{(x_0)})^\dagger$, compares
them with $-B$ and $+B$, and draws the three matrices $B$, $\Gamma B\Gamma^\dagger$
and $\gamma^{(x_0)}B(\gamma^{(x_0)})^\dagger$ as heat maps. All three are purely
imaginary, so their imaginary parts are drawn (blue $-1$, light grey 0, red $+1$).
```

In [3]:

```
image_metric = Gamma @ B @ Gamma.conj().T  # Gamma B Gamma^dagger
mirror_metric = gamma[0] @ B @ gamma[0].conj().T  # gamma0 B gamma0^dagger
signature = np.round(np.linalg.eigvalsh(image_metric)).astype(int)
report("eigenvalues -1 and +1 of Gamma B Gamma^dagger",
       f"{int(np.sum(signature == -1))} and {int(np.sum(signature == 1))}")
check_record(np.array_equal(image_metric, -B)
             and sorted(signature.tolist()) == [-1] * 8 + [1] * 8,
             "the chirality image Gamma Psi carries the Krein metric -B, (8,8)",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_krein_metric")
check_record(np.array_equal(mirror_metric, B),
             "the mirror image gamma^(x0) Psi keeps the Krein metric +B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.T2_image_keeps_B")
check_record(np.array_equal(image_metric, -B) and np.array_equal(mirror_metric, B),
             "Krein signs: Gamma gives -1, gamma^(x0) gives +1",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_Krein_metric_of_images")


def draw_matrix(ax, matrix, title):
    """Draw a real matrix as a heat map with rows and columns numbered from 1."""
    image = ax.imshow(matrix, cmap=DIVERGING, vmin=-1.0, vmax=1.0)
    size = matrix.shape[0]
    step = 3 if size <= 16 else 4  # label every third (or fourth) row and column
    ticks = list(range(0, size, step))
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.grid(False)  # no grid lines on top of the squares
    ax.set_title(title)
    return image


fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2))
draw_matrix(axes[0], B.imag, "field $\\Psi$: $B$")
draw_matrix(axes[1], image_metric.imag,
            "image $\\Gamma\\Psi$: $\\Gamma B\\Gamma^\\dagger = -B$")
```

```
image = draw_matrix(axes[2], mirror_metric.imag,
                    "mirror $\\gamma^{(x_0)}\\Psi$: $+B$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="imaginary part of the entry")
save_figure(fig, "krein_metrics",
            "The Krein metrics, that is the matrices of the canonical anticommutator, "
            "carried by the field $\\Psi$ (left: $B = -iC\\gamma^{(x_4)}$), by its "
            "chirality image $\\Gamma\\Psi$ (middle: $\\Gamma B\\Gamma^\\dagger$) "
            "and by its mirror image $\\gamma^{(x_0)}\\Psi$ (right: $\\gamma^{(x_0)}"
            "B\\gamma^{(x_0)\\dagger}$), drawn as heat maps of their imaginary parts "
            "(the real parts are zero). Horizontal axis: column number, vertical "
            "axis: row number; red is $+1$, blue is $-1$, light grey is 0. The "
            "middle picture is the left one with every colour exchanged: the "
            "chirality image carries $-B$. The right picture equals the left one: "
            "the mirror image keeps $+B$.")
```

Out [3]:

```
RESULT eigenvalues -1 and +1 of Gamma B Gamma^dagger = 8 and 8
PASS the chirality image Gamma Psi carries the Krein metric -B, (8,8)
     reproduces Revision/pairing/reports/python-pairing.json, check Q.image_krein_metric
PASS the mirror image gamma^(x0) Psi keeps the Krein metric +B
     reproduces Revision/pairing/reports/python-pairing.json, check Q.T2_image_keeps_B
PASS Krein signs: Gamma gives -1, gamma^(x0) gives +1
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_Krein_metric_of_images
```

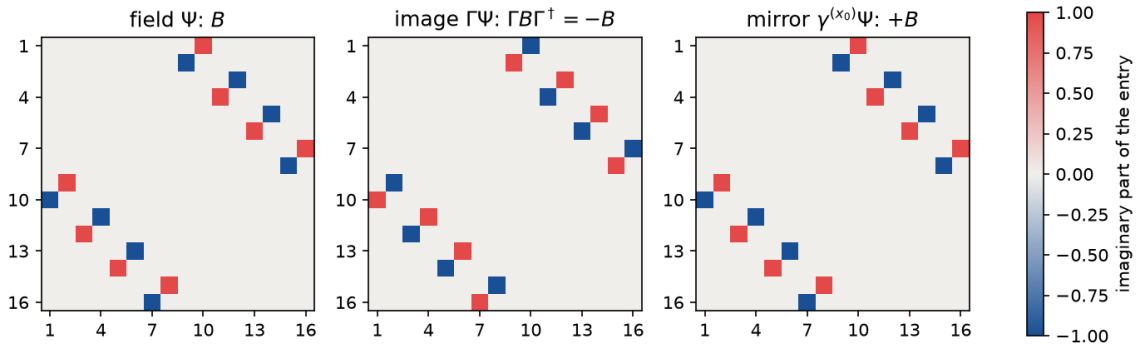

Figure 298: The Krein metrics, that is the matrices of the canonical anticommutator, carried by the field Ψ (left: $B = -iC\gamma^{(x_4)}$), by its chirality image $\Gamma\Psi$ (middle: $\Gamma B\Gamma^\dagger$) and by its mirror image $\gamma^{(x_0)}\Psi$ (right: $\gamma^{(x_0)}B\gamma^{(x_0)\dagger}$), drawn as heat maps of their imaginary parts (the real parts are zero). Horizontal axis: column number, vertical axis: row number; red is $+1$, blue is -1, light grey is 0. The middle picture is the left one with every colour exchanged: the chirality image carries $-B$. The right picture equals the left one: the mirror image keeps $+B$. (Notebook 10g, figure 1.)

Out [3]:

```
Figure 10g.1 saved as Revision/textbook/figures/10g_1_krein_metrics.png
```

Text cell:

6. The Lagrangian identity and the three kernels, exactly

The next cell writes the Lagrangian $\mathcal{L}_m$ of Section 3 with SYMBOLS for the 16 components of a field and their derivatives along the eight directions, and for the complex conjugates of all of them (sympy treats a symbol and its conjugate as independent letters, which is exactly how the Lagrangian uses them). The function ‘lagrangian(mass, M)‘ evaluates $\mathcal{L}_{\mathrm{mass}}$ on the field $M\chi$ for a constant matrix M, where χ is the column of symbols. Then the cell

- checks $\mathcal{L}_m[\Gamma\chi] + \mathcal{L}_{-m}[\chi] = 0$ for every value of the 16 components and their 128 derivatives (T1 for flat space, the Lagrangian level);
- reads off the kernel N of each Lagrangian: the coefficient of $\chi_A^*\,\partial_4\chi_C$ is $\frac{i}{2}N_{AC}$, so N_{AC} is $-2i$ times that coefficient;
- checks $N = B$ for $\mathcal{L}_m[\chi]$, $N = -B$ for the image Lagrangian $\mathcal{L}_m[\Gamma\chi]$, $N = +B$ for an independent universe $\mathcal{L}_{-m}[\chi]$, and the anticommutators $N^{-1} = B, -B, B$.

In [4]:

```
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
C_exact = g[0] * g[1] * g[2] * g[3]
Gamma_exact = sp.diag(*[int(x) for x in diagonal])  # diag(-I8, I8), exact
B_exact = -sp.I * C_exact * g[4]
m = sp.Symbol("m", real=True)  # the mass: any real number
field = sp.Matrix(sp.symbols("c1:17"))  # chi_A, A = 1, ..., 16
field_conj = sp.Matrix(sp.symbols("cc1:17"))  # their complex conjugates chi_A^*
# d[a] and d_conj[a]: the derivatives of chi and of chi^* along x_a
d = {a: sp.Matrix(sp.symbols(f"d{a}c1:17")) for a in range(8)}
d_conj = {a: sp.Matrix(sp.symbols(f"d{a}cc1:17")) for a in range(8)}


def lagrangian(mass, M):
    """L_mass evaluated on the field M chi (M a constant real matrix)."""
    psi, psi_conj = M * field, M * field_conj  # M is real: (M chi)^* = M chi^*
    total = sp.Integer(0)
    for a in range(8):
        kinetic = C_exact * g[a]  # the matrix C gamma^(x_a)
        dpsi, dpsi_conj = M * d[a], M * d_conj[a]
        total += sp.Rational(1, 2) * ((psi_conj.T * kinetic * dpsi)[0]
                                      - (dpsi_conj.T * kinetic * psi)[0])
    total -= mass * (psi_conj.T * C_exact * psi)[0]
    return sp.expand(total)


def kernel(expression):
    """N_AC = -2 i times the coefficient of chi_A^* d4 chi_C."""
    return sp.Matrix(16, 16, lambda A, C_: -2 * sp.I * expression.coeff(
        field_conj[A] * d[4][C_]))


L_field = lagrangian(m, sp.eye(16))  # L_m[chi]
```

```
L_image = lagrangian(m, Gamma_exact)  # L_m[Gamma chi]: the same system, new variables
L_independent = lagrangian(-m, sp.eye(16))  # L_-m[chi]: an independent universe
say(f"number of terms of L_m[chi]: {len(L_field.args)}")
identity_ok = sp.expand(L_image + L_independent) == 0
check(identity_ok, "exact: L_m[Gamma chi] = -L_-m[chi] for all components and "
                   "derivatives")
N_field, N_image, N_independent = map(kernel, (L_field, L_image, L_independent))
kernels_ok = (N_field == B_exact and N_image == -B_exact
              and N_independent == B_exact)
say(f"kernels equal to B: field {N_field == B_exact}, image {N_image == B_exact}, "
    f"independent universe {N_independent == B_exact}; image kernel = -B: "
    f"{N_image == -B_exact}")
check_record(kernels_ok,
             "kernels: N = B (field), N = -B (image), N = +B (independent universe)",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_symplectic_kernel_grassmann")
anticommutators = [N.inv() for N in (N_field, N_image, N_independent)]
check_record(anticommutators[0] == B_exact and anticommutators[1] == -B_exact
             and anticommutators[2] == B_exact,
             "canonical rules N^(-1): {Psi,Psi^dag} = B, {chi,chi^dag} = -B, +B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_own_quantisation")
```

Out [4]:

```
number of terms of L_m[chi]: 272
PASS exact: L_m[Gamma chi] = -L_-m[chi] for all components and derivatives
kernels equal to B: field True, image False, independent universe True; image kernel =
    -B: True
PASS kernels: N = B (field), N = -B (image), N = +B (independent universe)
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_symplectic_kernel_grassmann
PASS canonical rules N^(-1): {Psi,Psi^dag} = B, {chi,chi^dag} = -B, +B
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.image_own_quantisation
```

Text cell:

```
## 7. The field and its two images on a Fock space

The next cell builds the fermionic Fock space of the second notebook of this
chapter, here for any number of modes: a state is a dictionary `{pattern:
amplitude}`, a pattern is a whole number whose binary digit $p$ is the occupation of
mode $p$, `annihilate(p, state)` is $f_p$ and `create(p, state)` is $f_p^*$ (with
the sign $(-1)^{\text{(occupied modes below } p)}$ that makes different modes
anticommute). The field of one universe uses the 16 modes starting at the number
`offset`: $\Psi_A = f_{A - 1 + \mathrm{offset}}$, with the canonical conjugate
$\Psi^\dagger_A = \sum_C f^*_{C - 1 + \mathrm{offset}}B_{CA}$ (the positive
realisation, so that $\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$).

For a matrix $M$ the image field is $(M\Psi)_A = \sum_D M_{AD}\Psi_D$ and its
conjugate $((M\Psi)^\dagger)_A = \sum_D \Psi^\dagger_D M^*_{AD}$. The function
```

‘anticommutator_matrix(X, Y, state)‘ returns the matrix with entries
$\langle\phi|\{X_i, Y_j\}|\phi\rangle$ for a normalised state ϕ; when the
anticommutators are numbers (as here), these entries ARE those numbers. The cell
computes it for the image $\Gamma\Psi$ and for the mirror image
$\gamma^{(x_0)}\Psi$.

In [5]:

```
def sign_below(n, p):
    """(-1) to the power of the number of occupied modes below mode p in pattern n."""
    return -1 if bin(n & ((1 << p) - 1)).count("1") % 2 else 1


def annihilate(p, state):
    """f_p applied to a state {pattern: amplitude}."""
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:  # mode p is occupied in the pattern n
            new = n ^ (1 << p)  # the same pattern with mode p emptied
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):
    """f_p^* (the Hilbert adjoint of f_p) applied to a state."""
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:  # mode p is empty in the pattern n
            new = n | (1 << p)  # the same pattern with mode p filled
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def add(*terms):
    """The state sum of coefficient * state over the pairs (coefficient, state)."""
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result


def inner(left, right):
    """The positive inner product <left|right>."""
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    """The largest size of an amplitude of the state (0 for the zero state)."""
    return max((abs(a) for a in state.values()), default=0.0)


rng = np.random.default_rng(12345)  # random numbers with a fixed seed
```

```
def random_state(patterns, modes):
    """A normalised superposition of a few random patterns of the given modes."""
    state = {}
    for n in rng.integers(0, 2 ** modes, size=patterns):
        state[int(n)] = complex(rng.normal(), rng.normal())
    length = np.sqrt(inner(state, state).real)
    return {n: a / length for n, a in state.items()}


def field_op(offset):
    """The operators Psi_A (A = 1..16) of the universe whose modes start at offset."""
    return [lambda s, A=A: annihilate(A - 1 + offset, s) for A in range(1, 17)]


def conjugate_op(offset):
    """Psi^dagger_A = sum_C f^*_(C-1+offset) B_CA (the canonical conjugate)."""
    return [lambda s, A=A: add(*[(B[C_ - 1, A - 1], create(C_ - 1 + offset, s))
                                 for C_ in range(1, 17) if B[C_ - 1, A - 1] != 0])
            for A in range(1, 17)]


def mapped(M, ops):
    """The operators (M X)_A = sum_D M_AD X_D for a list X of 16 operators."""
    return [lambda s, A=A: add(*[(M[A - 1, D - 1], ops[D - 1](s))
                                 for D in range(1, 17) if M[A - 1, D - 1] != 0])
            for A in range(1, 17)]


def mapped_conjugate(M, conj_ops):
    """((M Psi)^dagger)_A = sum_D Psi^dagger_D conj(M_AD)."""
    return mapped(M.conj(), conj_ops)


def anticommutator_matrix(X, Y, state):
    """The matrix <state| {X_i, Y_j} |state> for two lists of operators."""
    result = np.zeros((len(X), len(Y)), dtype=complex)
    for i, x in enumerate(X):
        for j, y in enumerate(Y):
            both = add((1, x(y(state))), (1, y(x(state))))
            result[i, j] = inner(state, both)
    return result


Psi, Psi_dagger = field_op(0), conjugate_op(0)  # one universe: modes 0, ..., 15
phi = random_state(6, 16)
own_rule = anticommutator_matrix(Psi, Psi_dagger, phi)
image_rule = anticommutator_matrix(mapped(Gamma, Psi),
                                   mapped_conjugate(Gamma, Psi_dagger), phi)
mirror_rule = anticommutator_matrix(mapped(gamma[0], Psi),
                                    mapped_conjugate(gamma[0], Psi_dagger), phi)
errors = [np.max(np.abs(own_rule - B)), np.max(np.abs(image_rule + B)),
          np.max(np.abs(mirror_rule - B))]
```

```
report("largest deviations from B, -B and +B", ", ".join(f"{e:.1e}" for e in errors))
check(max(errors) < 1e-12,
      "Fock space: {Psi, Psi^dag} = B, image -B, mirror +B (256 entries each)")
```

Out [5]:

```
RESULT largest deviations from B, -B and +B = 2.2e-16, 2.2e-16, 2.2e-16
PASS Fock space: {Psi, Psi^dag} = B, image -B, mirror +B (256 entries each)
```

Text cell:

```
## 8. One quantum system: $T \to -T$ and $J \to -J$ as operator identities

The next cell takes a mass and a plane wave with momenta along all seven slice
directions (also along the extra times) and builds, on the Fock space, four
operators of the form $X^\dagger NX = \sum_{A,C}X^\dagger_AN_{AC}X_C$:

- the energy of the field, $H_m[\Psi] = \Psi^\dagger h'_m\Psi$, and the energy that
  the mass $-m$ theory assigns to the image, $H_{-m}[\Gamma\Psi] =
  (\Gamma\Psi)^\dagger h'_{-m}(\Gamma\Psi)$;
- the charge of the field, $Q[\Psi] = \Psi^\dagger B\Psi$, and the charge the
  theory assigns to the image, $Q[\Gamma\Psi] = (\Gamma\Psi)^\dagger B(\Gamma\Psi)$.

It checks $H_m[\Psi] = -H_{-m}[\Gamma\Psi]$ and $Q[\Psi] = -Q[\Gamma\Psi]$ by
applying both sides to 150 random states, and records the expectation values for
the figure. Then it checks the quantum equation of motion of the image: with the
energy $H = H_m[\Psi]$ of the system, $[H, (\Gamma\Psi)_c] = -(h_{-m}\Gamma\Psi)_c$
with the mode Hamiltonian $h_{-m} = Bh'_{-m}$, so $i\,\partial_4(\Gamma\Psi) =
h_{-m}\Gamma\Psi$: the image obeys the wave equation of mass $-m$, moved by the
SAME energy operator.
```

In [6]:

```
m_value = 1.3  # any mass and momenta (here with momenta along the extra times too)
k = {0: 1.1, 1: 0.4, 2: -0.7, 3: 0.2, 5: 0.9, 6: -0.3, 7: 0.5}


def h_prime(mass):
    """The energy matrix h'_mass = mass C - i sum_a k_a C gamma^(x_a) of the wave."""
    return mass * C - 1j * sum(k_a * (C @ gamma[a]) for a, k_a in k.items())


def bilinear(X_dagger, N, X, state):
    """sum_(A,C) X^dagger_A N_AC X_C applied to a state."""
    terms = []
    for C_ in range(16):
        lowered = X[C_](state)
        if lowered:
            terms += [(N[A, C_], X_dagger[A](lowered)) for A in range(16)
                      if N[A, C_] != 0]
    return add(*terms)
```

```
image_field = mapped(Gamma, Psi)  # Gamma Psi
image_conjugate = mapped_conjugate(Gamma, Psi_dagger)  # (Gamma Psi)^dagger
worst_H, worst_Q, values = 0.0, 0.0, []
for _ in range(150):
    state = random_state(4, 16)
    H_field = bilinear(Psi_dagger, h_prime(m_value), Psi, state)
    H_image = bilinear(image_conjugate, h_prime(-m_value), image_field, state)
    Q_field = bilinear(Psi_dagger, B, Psi, state)
    Q_image = bilinear(image_conjugate, B, image_field, state)
    worst_H = max(worst_H, largest(add((1, H_field), (1, H_image))))
    worst_Q = max(worst_Q, largest(add((1, Q_field), (1, Q_image))))
    values.append([inner(state, X).real for X in (H_field, H_image, Q_field,
                                                  Q_image)])
values = np.array(values)  # columns: <H_m[Psi]>, <H_-m[Gamma Psi]>, <Q>, <Q image>
report("largest |(H_m[Psi] + H_-m[Gamma Psi]) phi| over 150 states", f"{worst_H:.1e}")
report("largest |(Q[Psi] + Q[Gamma Psi]) phi| over 150 states", f"{worst_Q:.1e}")
check_record(worst_H < 1e-12 and worst_Q < 1e-12,
             "operators: H_m[Psi] = -H_-m[Gamma Psi] and Q[Psi] = -Q[Gamma Psi]",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_generators_of_the_image_grassmann")

h_mode_minus = B @ h_prime(-m_value)  # the mode Hamiltonian of mass -m
worst = 0.0
for c_ in range(16):
    H_after = bilinear(Psi_dagger, h_prime(m_value), Psi, image_field[c_](phi))
    H_before = image_field[c_](bilinear(Psi_dagger, h_prime(m_value), Psi, phi))
    commutator = add((1, H_after), (-1, H_before))  # [H, (Gamma Psi)_c] phi
    expected = add(*[(-h_mode_minus[c_, d_], image_field[d_](phi))
                     for d_ in range(16)])
    worst = max(worst, largest(add((1, commutator), (-1, expected))))
report("largest violation of [H, (Gamma Psi)_c] = -(h_-m Gamma Psi)_c", f"{worst:.1e}")
check(worst < 1e-12, "the image obeys i d4 (Gamma Psi) = h_-m Gamma Psi under the "
                     "same H")
```

Out [6]:

```
RESULT largest |(H_m[Psi] + H_-m[Gamma Psi]) phi| over 150 states = 0.0e+00
RESULT largest |(Q[Psi] + Q[Gamma Psi]) phi| over 150 states = 0.0e+00
PASS operators: H_m[Psi] = -H_-m[Gamma Psi] and Q[Psi] = -Q[Gamma Psi]
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_generators_of_the_image_grassmann
RESULT largest violation of [H, (Gamma Psi)_c] = -(h_-m Gamma Psi)_c = 1.6e-17
PASS the image obeys i d4 (Gamma Psi) = h_-m Gamma Psi under the same H
```

Text cell:

```
The Revision record states the same fact for the one-particle generators, exactly
and for all momenta: a field of mass $-m$ moves with the generator $Bh'_{-m}$; its
image under $\Gamma$ carries the metric $-B$ and the energy matrix $-h'_m$, and the
two signs compensate, $(-B)(-h'_m) = \Gamma(Bh'_{-m})\Gamma = Bh'_m$. The next cell
```

```
checks this with sympy for symbolic $m$ and $k_0, \dots, k_7$, together with
$\Gamma h'_m\Gamma = -h'_{-m}$, and then draws the expectation values of the
previous cell.
```

In [7]:

```
k_symbol = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}


def h_prime_exact(mass):
    """The exact energy matrix mass C - i sum_a k_a C gamma^(x_a)."""
    result = mass * C_exact
    for a, k_a in k_symbol.items():
        result = result - sp.I * k_a * C_exact * g[a]
    return result


zero = sp.zeros(16, 16)
own_data = (-B_exact) * (-h_prime_exact(m))  # the image: metric -B, energy -h'_m
by_map = Gamma_exact * (B_exact * h_prime_exact(-m)) * Gamma_exact
same = ((own_data - by_map).applyfunc(sp.expand) == zero
        and (own_data - B_exact * h_prime_exact(m)).applyfunc(sp.expand) == zero)
energy_rule = (Gamma_exact * h_prime_exact(m) * Gamma_exact
               + h_prime_exact(-m)).applyfunc(sp.expand) == zero
check_record(same and energy_rule,
             "exact: (-B)(-h'_m) = Gamma (B h'_-m) Gamma = B h'_m; "
             "Gamma h'_m Gamma = -h'_-m",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_generators_same_dynamics")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.4))
panels = ((axes[0], 0, 1, BLUE, "energy",
           "$\\langle H_m[\\Psi]\\rangle$", "$\\langle H_{-m}[\\Gamma\\Psi]\\rangle$"),
          (axes[1], 2, 3, ORANGE, "charge",
           "$\\langle Q[\\Psi]\\rangle$", "$\\langle Q[\\Gamma\\Psi]\\rangle$"))
for ax, x_col, y_col, colour, what, x_label, y_label in panels:
    ax.plot(values[:, x_col], values[:, y_col], "o", color=colour, markersize=4,
            label="150 random states")
    low, high = values[:, x_col].min() - 0.5, values[:, x_col].max() + 0.5
    ax.plot([low, high], [-low, -high], color=GREY, linewidth=1,
            label="the line $y = -x$")
    ax.set_xlabel(f"{what} of the field, {x_label}")
    ax.set_ylabel(f"{what} given to the image, {y_label}")
    ax.legend(fontsize=8)
axes[0].set_title("$T \\to -T$ inside one system")
axes[1].set_title("$J \\to -J$ inside one system")
save_figure(fig, "one_system",
            "For 150 random states of the Fock space of one field: left, the "
            "expectation value of the energy $H_m(\\Psi) = \\Psi^\\dagger h'_m\\Psi$ "
            "of the field (horizontal axis) against the expectation value of the "
            "energy $H_{-m}(\\Gamma\\Psi)$ that the mass $-m$ theory assigns to the "
            "chirality image (vertical axis), for $m = 1.3$ and a plane wave with "
            "momenta along all seven slice directions; right, the same for the "
```

```
        "charges $Q = \\Psi^\\dagger B\\Psi$ and $Q(\\Gamma\\Psi)$. Axes in the "
        "units of $m$ (left) and pure numbers (right). Every point lies on the "
        "grey line $y = -x$: the reversal of the energy and of the charge "
        "under T1 is an identity between operators of ONE quantum system, not "
        "a statement about a second universe.")
```

Out [7]:

```
PASS exact: (-B)(-h'_m) = Gamma (B h'_-m) Gamma = B h'_m; Gamma h'_m Gamma = -h'_-m
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.image_generators_same_dynamics
```

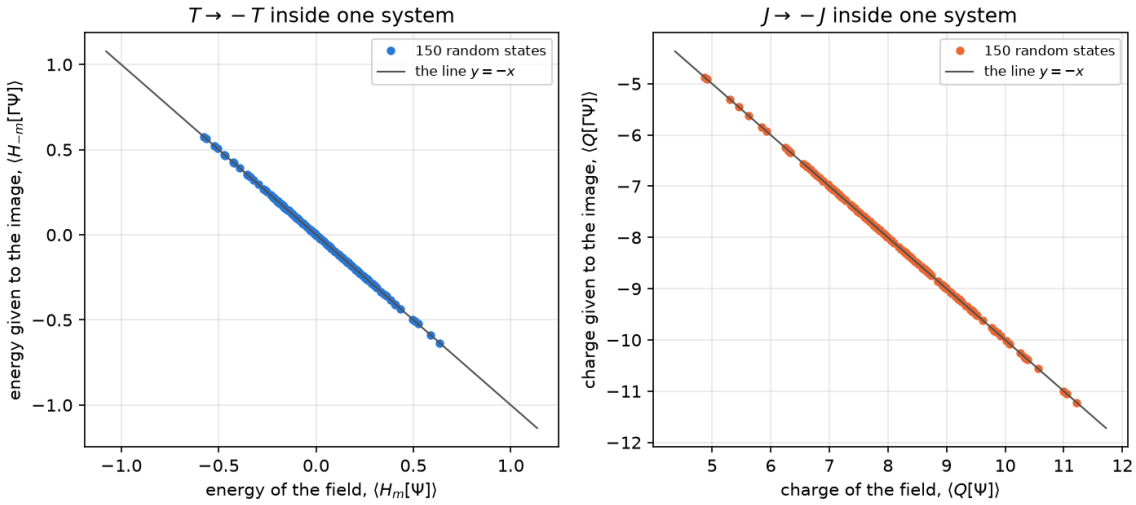

Figure 299: For 150 random states of the Fock space of one field: left, the expectation value of the energy $H_m(\Psi) = \Psi^\dagger h'_m \Psi$ of the field (horizontal axis) against the expectation value of the energy $H_{-m}(\Gamma\Psi)$ that the mass $-m$ theory assigns to the chirality image (vertical axis), for $m = 1.3$ and a plane wave with momenta along all seven slice directions; right, the same for the charges $Q = \Psi^\dagger B \Psi$ and $Q(\Gamma\Psi)$. Axes in the units of m (left) and pure numbers (right). Every point lies on the grey line $y = -x$: the reversal of the energy and of the charge under T1 is an identity between operators of ONE quantum system, not a statement about a second universe. (Notebook 10g, figure 2.)

Out [7]:

```
Figure 10g.2 saved as Revision/textbook/figures/10g_2_one_system.png
```

Text cell:

```
## 9. Which images could be an independent universe?

An independently quantised universe of mass $-m$ would be a field $\Psi_2$ with
$\{\Psi_2, \Psi_2^\dagger\} = +B$ (its own Lagrangian, Section 6) that
anticommutes with the first field: $\{\Psi_{2A}, \Psi_C^\dagger\} = 0$ and
$\{\Psi_{2A}, \Psi_C\} = 0$. The next cell tests three candidates built from the
```

```
first field $\Psi$:

- the chirality image $\Gamma\Psi$ (T1);
- the mirror image $\gamma^{(x_0)}\Psi$ (T2, here without the reflection of the
  hidden coordinate, which does not change the anticommutators at one point);
- the conjugate field $\Psi^c = \Gamma\Psi^{\dagger T}$, that is
  $\Psi^c_A = \sum_D\Gamma_{AD}\Psi^\dagger_D$, the conjugation of the quantised
  field that keeps the canonical rule (it reverses the mass; second notebook of
  this chapter).

For each it computes, on the Fock space, its own anticommutator matrix and the two
cross matrices with the first field, and their ranks. Independence would need both
cross matrices to be 0.
```

In [8]:

```
conjugate_field = mapped(Gamma, Psi_dagger)  # Psi^c_A = sum_D Gamma_AD Psi^dag_D
# Its canonical conjugate: (Psi^c)^dagger_A = sum_D conj(Gamma_AD) Psi_D, by the
# rule (Psi^dagger)^dagger = Psi of the canonical relations (Gamma is real).
conjugate_field_dagger = mapped(Gamma, Psi)
candidates = {
    "Gamma Psi": (image_field, image_conjugate),
    "gamma0 Psi": (mapped(gamma[0], Psi), mapped_conjugate(gamma[0], Psi_dagger)),
    "Psi^c = Gamma Psi^dag T": (conjugate_field, conjugate_field_dagger),
}
phi = random_state(6, 16)
results = {}
for label, (X, X_dagger) in candidates.items():
    metric = anticommutator_matrix(X, X_dagger, phi)  # {X_A, X_C^dagger}
    cross_1 = anticommutator_matrix(X, Psi_dagger, phi)  # {X_A, Psi^dagger_C}
    cross_2 = anticommutator_matrix(X, Psi, phi)  # {X_A, Psi_C}
    sign = (1 if np.allclose(metric, B) else -1 if np.allclose(metric, -B) else 0)
    ranks = (np.linalg.matrix_rank(cross_1), np.linalg.matrix_rank(cross_2))
    results[label] = (sign, ranks, cross_1, cross_2)
    say(f"{label:24s} metric {sign:+d} B; rank of {{X, Psi^dag}} = {ranks[0]:2d}, "
        f"rank of {{X, Psi}} = {ranks[1]:2d}")
chirality = results["Gamma Psi"]
conjugate = results["Psi^c = Gamma Psi^dag T"]
check_record(chirality[0] == -1 and chirality[1] == (16, 0)
             and np.allclose(chirality[2], Gamma @ B),
             "Gamma Psi: metric -B and {Gamma Psi, Psi^dag} = Gamma B of rank 16",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_no_identification_of_independent_universes")
check_record(conjugate[0] == 1 and conjugate[1] == (0, 16)
             and np.allclose(conjugate[3], -Gamma @ B),
             "Psi^c keeps +B (Gamma B^T Gamma^dag = B) but {Psi^c, Psi} = -Gamma B",
             record="Revision/lead_checks/reports/charge-conjugation-and-u1.json, "
                    "check quantum_charge_conjugation_unitary_type")
check(results["gamma0 Psi"][0] == 1 and results["gamma0 Psi"][1] == (16, 0),
      "gamma0 Psi keeps +B; as an operator of the same field it is not independent")
```

Out [8]:

```
Gamma Psi                metric -1 B; rank of {X, Psi^dag} = 16, rank of {X, Psi} =  0
gamma0 Psi               metric +1 B; rank of {X, Psi^dag} = 16, rank of {X, Psi} =  0
Psi^c = Gamma Psi^dag T  metric +1 B; rank of {X, Psi^dag} =  0, rank of {X, Psi} = 16
PASS Gamma Psi: metric -B and {Gamma Psi, Psi^dag} = Gamma B of rank 16
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_no_identification_of_independent_universes
PASS Psi^c keeps +B (Gamma B^T Gamma^dag = B) but {Psi^c, Psi} = -Gamma B
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    quantum_charge_conjugation_unitary_type
PASS gamma0 Psi keeps +B; as an operator of the same field it is not independent
```

Text cell:

```
The result in words. The chirality image has the wrong canonical rule ($-B$
instead of $+B$) AND does not anticommute with the first field (rank 16): it is the
first universe in other variables. The mirror image and the conjugate field have
the right rule $+B$, but they too are built from the operators of the first field
and do not anticommute with it. The rule $+B$ of the mirror image matches the rule
of an independently quantised mirror universe, so such a universe can be quantised
as an ordinary copy (statement Q5 of the record); for the chirality image this is
impossible (statement Q3). A second universe must therefore be a NEW field on a
larger state space, which the next section builds.
```

Text cell:

```
## 10. Two independently quantised universes

The next cell puts universe 1 (mass $+m$) on the modes 0 to 15 and universe 2 (mass
$-m$) on the modes 16 to 31 of one Fock space, each with its own canonical
conjugate. It computes the $32 \times 32$ anticommutator matrix of the combined
list $(\Psi_1, \Psi_2)$ with $(\Psi_1^\dagger, \Psi_2^\dagger)$, which must be
$\mathrm{diag}(B, B)$, and, for comparison, that of the list $(\Psi_1,
\Gamma\Psi_1)$ with its conjugates, which must be the block matrix with $B$,
$B\Gamma$, $\Gamma B$ and $-B$. Then it draws both.
```

In [9]:

```
Psi_1, Psi_1_dagger = field_op(0), conjugate_op(0)  # universe 1: modes 0..15
Psi_2, Psi_2_dagger = field_op(16), conjugate_op(16)  # universe 2: modes 16..31
phi_32 = random_state(6, 32)
two_universes = anticommutator_matrix(Psi_1 + Psi_2, Psi_1_dagger + Psi_2_dagger,
                                      phi_32)
image_pair = anticommutator_matrix(
    Psi_1 + mapped(Gamma, Psi_1),
    Psi_1_dagger + mapped_conjugate(Gamma, Psi_1_dagger), phi_32)
zero_16 = np.zeros((16, 16))
expected_two = np.block([[B, zero_16], [zero_16, B]])
expected_image = np.block([[B, B @ Gamma], [Gamma @ B, -B]])
cross_zero = np.max(np.abs(anticommutator_matrix(Psi_1, Psi_2, phi_32)))
report("largest |{Psi_1, Psi_2}| (must be 0)", f"{cross_zero:.1e}")
check(np.allclose(two_universes, expected_two) and cross_zero < 1e-12
```

```
      and np.allclose(image_pair, expected_image),
      "two universes: diag(B, B); field and image: blocks B, B Gamma, Gamma B, -B")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 5.0))
draw_matrix(axes[0], two_universes.imag,
            "independent: $(\\Psi_1, \\Psi_2)$")
image = draw_matrix(axes[1], image_pair.imag,
                    "one field and its image: $(\\Psi_1, \\Gamma\\Psi_1)$")
for ax in axes:
    ax.axhline(15.5, color=GREY, linewidth=1)  # the border between the two halves
    ax.axvline(15.5, color=GREY, linewidth=1)
fig.colorbar(image, ax=list(axes), shrink=0.75, label="imaginary part of the entry")
save_figure(fig, "anticommutator_blocks",
            "The $32 \\times 32$ anticommutator matrices computed on the Fock space "
            "of 32 fermion modes (imaginary parts; the real parts are zero; red "
            "$+1$, blue $-1$, light grey 0; horizontal axis: column, vertical axis: "
            "row; grey lines separate the two halves). Left: two independently "
            "quantised universes $\\Psi_1$ (mass $+m$) and $\\Psi_2$ (mass $-m$): "
            "the diagonal blocks are both $+B$ and the off-diagonal blocks are "
            "empty, the universes anticommute. Right: a field and its chirality "
            "image $\\Gamma\\Psi_1$: the lower right block is $-B$ and the "
            "off-diagonal blocks $B\\Gamma$ and $\\Gamma B$ are full, so the image "
            "is not a second universe but the same one.")
```

Out [9]:

```
RESULT largest |{Psi_1, Psi_2}| (must be 0) = 0.0e+00
PASS two universes: diag(B, B); field and image: blocks B, B Gamma, Gamma B, -B
```

Out [9]:

```
Figure 10g.3 saved as Revision/textbook/figures/10g_3_anticommutator_blocks.png
```

Text cell:

```
## 11. The generators add: no cancellation

The one-particle generator of the two independent universes is the block matrix
$\mathrm{diag}(Bh'_m, Bh'_{-m})$. The next cell computes its exact eigenvalues
with sympy at zero momentum (the record's case: $+m$ and $-m$, 16 times each), and
then, with numbers, its eigenvalues for $m$ from $-3$ to $3$ at zero momentum and
at the good-sector momentum $k_1 = 1.5$. In the second case they are
$\pm\sqrt{m^2 + k_1^2}$ (16 each); in no case is an eigenvalue 0 for $m \neq 0$, so
the total generator is not zero: the two universes do not cancel each other.
```

In [10]:

```
zero_momentum = {k_a: 0 for k_a in k_symbol.values()}
block = sp.diag((B_exact * h_prime_exact(m)).subs(zero_momentum),
```

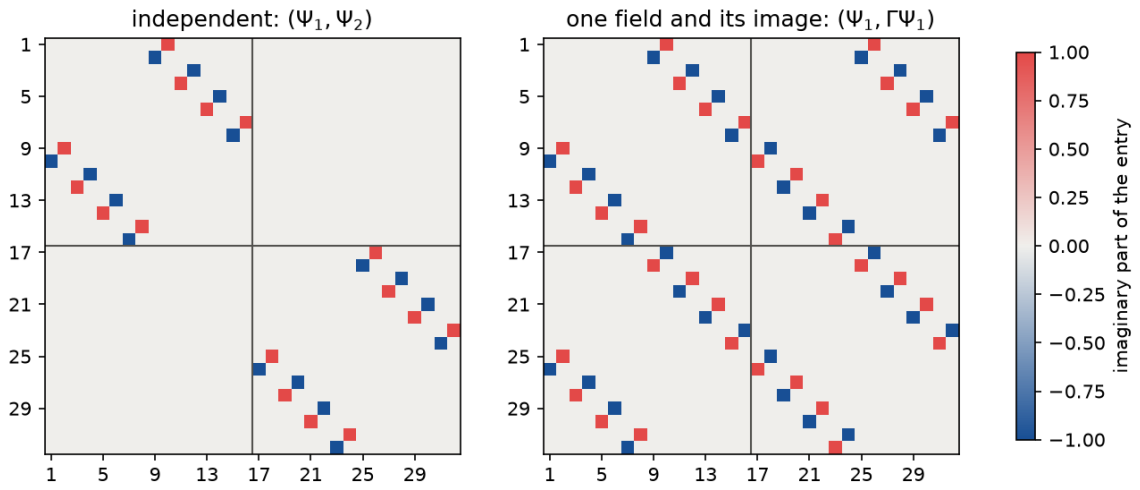

Figure 300: The 32×32 anticommutator matrices computed on the Fock space of 32 fermion modes (imaginary parts; the real parts are zero; red $+1$, blue -1, light grey 0; horizontal axis: column, vertical axis: row; grey lines separate the two halves). Left: two independently quantised universes Ψ_1 (mass $+m$) and Ψ_2 (mass $-m$): the diagonal blocks are both $+B$ and the off-diagonal blocks are empty, the universes anticommute. Right: a field and its chirality image $\Gamma\Psi_1$: the lower right block is $-B$ and the off-diagonal blocks $B\Gamma$ and ΓB are full, so the image is not a second universe but the same one. (Notebook 10g, figure 3.)

```
                (B_exact * h_prime_exact(-m)).subs(zero_momentum))
block_eigenvalues = block.eigenvals()  # {eigenvalue: how often}
say(f"exact eigenvalues of diag(B h'_m, B h'_-m) at k = 0: +m "
    f"{block_eigenvalues.get(m, 0)} times, -m {block_eigenvalues.get(-m, 0)} times, "
    f"{len(block_eigenvalues)} different values")
check_record(block_eigenvalues == {m: 16, -m: 16},
             "block generator at k = 0: eigenvalues +m and -m, 16 each, none zero",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.no_cancellation_independent_universes")


def block_numbers(mass, k1):
    """The 32 eigenvalues of diag(B h'_mass, B h'_-mass) with momentum k1 along x1."""
    def energy(mu):  # the energy matrix of mass mu and momentum k1
        return mu * C - 1j * k1 * (C @ gamma[1])
    top, bottom = B @ energy(mass), B @ energy(-mass)
    values = np.concatenate([np.linalg.eigvals(top), np.linalg.eigvals(bottom)])
    return np.sort(values.real)


masses = np.linspace(-3.0, 3.0, 121)
spectra_0 = np.array([block_numbers(x, 0.0) for x in masses])
spectra_k = np.array([block_numbers(x, 1.5) for x in masses])
formula_k = np.sqrt(masses ** 2 + 1.5 ** 2)
error_k = max(np.max(np.abs(spectra_k[:, 16:] - formula_k[:, None])),
              np.max(np.abs(spectra_k[:, :16] + formula_k[:, None])))
report("largest deviation from +-sqrt(m^2 + k1^2), 16 each", f"{error_k:.1e}")
```

```
check(error_k < 1e-10 and np.min(np.abs(spectra_k)) >= 1.5 - 1e-10,
      "with k1 = 1.5 the 32 eigenvalues are +-sqrt(m^2 + k1^2): never 0")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
axes[0].plot(masses, spectra_0[:, [0, 31]], color=BLUE, linewidth=2)
axes[0].set_title("zero momentum: $+m$ and $-m$, 16 each")
axes[1].plot(masses, spectra_k[:, [0, 31]], color=GREEN, linewidth=2)
axes[1].set_title("momentum $k_1 = 1.5$: $\\pm\\sqrt{m^2 + k_1^2}$, 16 each")
for ax in axes:
    ax.axhline(0.0, color=GREY, linewidth=1)
    ax.set_xlabel("mass $m$ of universe 1 (universe 2 has $-m$)")
axes[0].set_ylabel("eigenvalues of diag$(Bh'_m, Bh'_{-m})$")
save_figure(fig, "block_generator",
            "The 32 eigenvalues (vertical axes, units of the momentum and mass) of "
            "the one-particle generator $\\mathrm{diag}(Bh'_m, Bh'_{-m})$ of two "
            "independently quantised universes of masses $+m$ and $-m$, against "
            "$m$ from $-3$ to $3$ (horizontal axes); each line carries 16 "
            "eigenvalues. Left: zero momentum, eigenvalues $+m$ and $-m$, which "
            "vanish only at $m = 0$. Right: the momentum $k_1 = 1.5$, eigenvalues "
            "$\\pm\\sqrt{m^2 + k_1^2}$, never 0. The generators of the two universes "
            "add; nothing in them cancels.")
```

Out [10]:

```
exact eigenvalues of diag(B h'_m, B h'_-m) at k = 0: +m 16 times, -m 16 times, 2
    different values
PASS block generator at k = 0: eigenvalues +m and -m, 16 each, none zero
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.no_cancellation_independent_universes
RESULT largest deviation from +-sqrt(m^2 + k1^2), 16 each = 4.9e-15
PASS with k1 = 1.5 the 32 eigenvalues are +-sqrt(m^2 + k1^2): never 0
```

Out [10]:

```
Figure 10g.4 saved as Revision/textbook/figures/10g_4_block_generator.png
```

Text cell:

```
## 12. One good-sector momentum in both universes

The last computation repeats, for the two universes together, the positive Fock
construction of the third notebook of this chapter, for the recorded good-sector
momentum $m = 2$, $(k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ along $(x_0, x_1, x_2, x_3)$,
$E = 5$.
Universe 1 uses the orthonormal eigenvectors $u_s$ ($+E$) and $v_s$ ($-E$) of $h_m$;
universe 2 uses $\Gamma u_s$ and $\Gamma v_s$, which are eigenvectors of $h_{-m} =
\Gamma h_m\Gamma$ with the same eigenvalues. In each universe modes 0 to 7 of its
half are particles and modes 8 to 15 antiparticles (created by $d^*$, so
$\Psi = \sum_s(u_sb_s + v_sd_s^*)$ and $\Psi^\dagger = \chi B$, with $\chi$ the
Hilbert adjoint). The cell checks the canonical rule in both universes, the vacuum
energy $-8E$ of each, that every pattern is an energy and charge eigenstate with
```

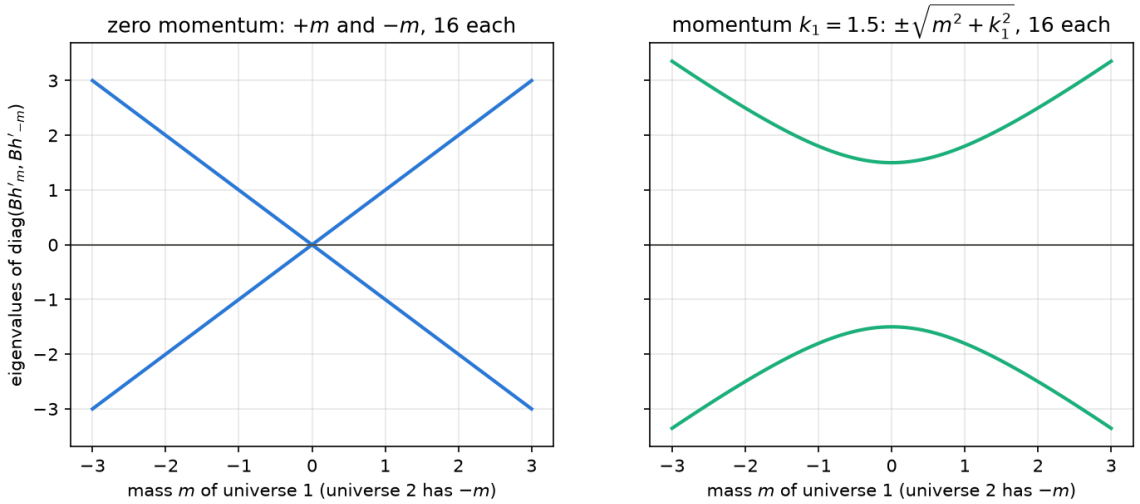

Figure 301: The 32 eigenvalues (vertical axes, units of the momentum and mass) of the one-particle generator $\mathrm{diag}(Bh'_m, Bh'_{-m})$ of two independently quantised universes of masses $+m$ and $-m$, against m from -3 to 3 (horizontal axes); each line carries 16 eigenvalues. Left: zero momentum, eigenvalues $+m$ and $-m$, which vanish only at $m = 0$. Right: the momentum $k_1 = 1.5$, eigenvalues $\pm\sqrt{m^2 + k_1^2}$, never 0. The generators of the two universes add; nothing in them cancels. (Notebook 10g, figure 4.)

```
the normal-ordered total energy $E \times$(number of quanta) and the total charge
(particles minus antiparticles), on 200 random patterns, and the pair state
"particle in universe 1, antiparticle in universe 2". Energy and charge are applied
in the mode form $\chi N\Psi = \sum_{p,q}(W^\dagger NW)_{pq}F_p^*F_q$, where $F_p$
is $b$ or $d^*$ (the same operator as $\sum_{A,C}\chi_AN_{AC}\Psi_C$, but much
faster to apply, because $W^\dagger hW$ is diagonal).
```

In [11]:

```
def orthonormal_columns(P):
    """Orthonormal columns that span the range of P (Gram-Schmidt, done twice)."""
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):  # the second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T


def mode_hamiltonian(mass, momenta):
    """h = -i mass gamma^(x4) - gamma^(x4) sum_a k_a gamma^(x_a)."""
    h = -1j * mass * gamma[4]
    for a, k_a in momenta.items():
```

```
        h = h - k_a * (gamma[4] @ gamma[a])
    return h


sample = {0: 4, 1: 1, 2: 2}  # the recorded good-sector momentum, with m = 2
h_plus, h_minus = mode_hamiltonian(2, sample), mode_hamiltonian(-2, sample)
E = 5.0  # sqrt(4 + 1 + 4 + 16)
W1 = np.hstack([orthonormal_columns((I16 + h_plus / E) / 2),
                orthonormal_columns((I16 - h_plus / E) / 2)])  # u_1..u_8, v_1..v_8
W2 = Gamma @ W1  # the eigenvectors of h_-m = Gamma h_m Gamma
check(np.allclose(h_minus @ W2[:, :8], E * W2[:, :8])
      and np.allclose(h_minus @ W2[:, 8:], -E * W2[:, 8:])
      and np.allclose(W2.conj().T @ W2, I16),
      "Gamma u_s and Gamma v_s are orthonormal eigenvectors of h_-m, energies +-E")


def universe(W, offset):
    """Psi, chi (Hilbert adjoint), Psi^dagger = chi B and the mode operators of
    one universe whose 16 modes start at offset."""
    def F(p, s):  # b (particle modes p < 8) or d^* (antiparticle modes p >= 8)
        return annihilate(p + offset, s) if p < 8 else create(p + offset, s)

    def F_star(p, s):  # the Hilbert adjoint of F_p
        return create(p + offset, s) if p < 8 else annihilate(p + offset, s)

    psi = [lambda s, A=A: add(*[(W[A, p], F(p, s)) for p in range(16)])
           for A in range(16)]  # Psi_A = sum_p W_Ap F_p
    chi = [lambda s, A=A: add(*[(np.conj(W[A, p]), F_star(p, s)) for p in range(16)])
           for A in range(16)]  # chi_A = sum_p conj(W_Ap) F_p^*
    psi_dagger = mapped(B.T, chi)  # Psi^dagger_A = sum_C chi_C B_CA
    return psi, chi, psi_dagger, F, F_star


def mode_bilinear(W, F, F_star, N, state):
    """chi N Psi = sum_(p,q) (W^dagger N W)_pq F_p^* F_q applied to a state (the
    same operator as sum_(A,C) chi_A N_AC Psi_C, written with the mode operators)."""
    M = W.conj().T @ N @ W
    terms = []
    for q in range(16):
        lowered = F(q, state)
        if lowered:
            terms += [(M[p, q], F_star(p, lowered)) for p in range(16)
                      if abs(M[p, q]) > 1e-12]
    return add(*terms)


psi_1, chi_1, psi_1_dagger, F_1, F_1_star = universe(W1, 0)
psi_2, chi_2, psi_2_dagger, F_2, F_2_star = universe(W2, 16)
phi_32 = random_state(4, 32)
rule_1 = anticommutator_matrix(psi_1, psi_1_dagger, phi_32)
rule_2 = anticommutator_matrix(psi_2, psi_2_dagger, phi_32)
rule_12 = anticommutator_matrix(psi_1, psi_2_dagger, phi_32)
check(np.allclose(rule_1, B) and np.allclose(rule_2, B) and np.allclose(rule_12, 0),
      "positive realisation: {Psi_1, Psi_1^dag} = {Psi_2, Psi_2^dag} = B, cross 0")
```

```
def total_energy(state):
    """(H_1 + H_2) state, with H = chi h Psi in each universe."""
    return add((1, mode_bilinear(W1, F_1, F_1_star, h_plus, state)),
               (1, mode_bilinear(W2, F_2, F_2_star, h_minus, state)))


def total_charge(state):
    """(Q_1 + Q_2) state, with Q = Psi^dagger B Psi = chi Psi in each universe."""
    return add((1, mode_bilinear(W1, F_1, F_1_star, I16, state)),
               (1, mode_bilinear(W2, F_2, F_2_star, I16, state)))


VACUUM = {0: 1.0}
vacuum_energy = inner(VACUUM, total_energy(VACUUM)).real
vacuum_charge = inner(VACUUM, total_charge(VACUUM)).real
report("vacuum energy and charge of the two universes",
       f"{vacuum_energy:.10f} and {vacuum_charge:.10f}")
check_record(abs(vacuum_energy + 16 * E) < 1e-9 and abs(vacuum_charge - 16) < 1e-9,
             "each universe has the sea energy -8E = -40 (total -80) and charge 8",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_positive_fock_realisation")
```

Out [11]:

```
PASS Gamma u_s and Gamma v_s are orthonormal eigenvectors of h_-m, energies +-E
PASS positive realisation: {Psi_1, Psi_1^dag} = {Psi_2, Psi_2^dag} = B, cross 0
RESULT vacuum energy and charge of the two universes = -80.0000000000 and 16.0000000000
PASS each universe has the sea energy -8E = -40 (total -80) and charge 8
     reproduces Revision/theory/reports/python-field-theory.json, check
    good_sector_positive_fock_realisation
```

Text cell:

```
The next cell checks the eigenvalue statement on 200 random patterns, computes the
pair state $b^*_{1}d^*_{2}|0\rangle$ (a particle in universe 1 and an antiparticle
in universe 2), counts all $2^{32}$ patterns by the number of quanta and the total
charge with the binomial numbers $\binom{8}{j}$ (four groups of 8 modes: particles
and antiparticles of each universe), and draws the counts.
```

In [12]:

```
def quanta_and_charge(n):
    """(number of quanta, total charge) of the pattern n of the 32 modes."""
    groups = [bin((n >> shift) & 0xFF).count("1") for shift in (0, 8, 16, 24)]
    b_1, d_1, b_2, d_2 = groups  # particles and antiparticles of each universe
    return b_1 + d_1 + b_2 + d_2, b_1 - d_1 + b_2 - d_2


eigen_ok = True
```

```
for n in rng.integers(0, 2 ** 32, size=200):
    n = int(n)
    quanta, charge = quanta_and_charge(n)
    state = {n: 1.0}
    eigen_ok = eigen_ok and largest(add(
        (1, total_energy(state)), (-(E * quanta + vacuum_energy), state))) < 1e-9
    eigen_ok = eigen_ok and largest(add(
        (1, total_charge(state)), (-(charge + vacuum_charge), state))) < 1e-9
check(eigen_ok, "200 random patterns: energy E (quanta) - 16E, charge (b - d) + 16")

pair = create(24, create(0, VACUUM))  # b_1^* (mode 0), then d_2^* (mode 24)
pair_energy = inner(pair, total_energy(pair)).real - vacuum_energy
pair_charge = inner(pair, total_charge(pair)).real - vacuum_charge
report("pair state: normal-ordered total energy and total charge",
       f"{pair_energy:.10f} and {pair_charge:.10f}")
check(abs(pair_energy - 2 * E) < 1e-9 and abs(pair_charge) < 1e-9,
      "a particle in universe 1 and an antiparticle in universe 2: charge 0, "
      "energy 2E")

counts = np.zeros((33, 33), dtype=np.int64)  # rows: quanta, columns: charge + 16
for b_1 in range(9):
    for d_1 in range(9):
        for b_2 in range(9):
            for d_2 in range(9):
                number = comb(8, b_1) * comb(8, d_1) * comb(8, b_2) * comb(8, d_2)
                counts[b_1 + d_1 + b_2 + d_2, b_1 - d_1 + b_2 - d_2 + 16] += number
zero_energy_states = int(counts[0].sum())
zero_charge_lowest = min(q for q in range(33) if q > 0 and counts[q, 16] > 0)
report("number of patterns", int(counts.sum()))
report("patterns of zero normal-ordered energy", zero_energy_states)
report("fewest quanta of a non-vacuum state of total charge 0", zero_charge_lowest)
check(counts.sum() == 2 ** 32 and zero_energy_states == 1 and zero_charge_lowest == 2,
      "only the vacuum has energy 0; charge 0 needs at least 2 quanta (energy 2E)")

fig, ax = plt.subplots(figsize=(7.5, 5.6))
shown = np.where(counts > 0, np.log10(np.maximum(counts, 1)), np.nan)  # NaN: empty
light_to_dark = LinearSegmentedColormap.from_list(
    "light_to_dark_blue", ["#cde2fb", "#2a78d6", "#0d366b"])
image = ax.imshow(shown, origin="lower", cmap=light_to_dark, aspect="auto",
                  extent=(-16.5, 16.5, -0.5, 32.5))
ax.annotate("vacuum: the only state of energy 0", (0, 0), xytext=(2.0, 1.5),
            fontsize=8, arrowprops={"arrowstyle": "->", "color": GREY})
ax.annotate("pair of total charge 0: energy $2E$", (0, 2), xytext=(3.0, 5.5),
            fontsize=8, arrowprops={"arrowstyle": "->", "color": GREY})
ax.set_xlabel("total charge (particles minus antiparticles, both universes)")
ax.set_ylabel("number of quanta (total energy $= 5 \\times$ quanta)")
ax.set_title("Two universes, one momentum: $2^{32}$ states")
ax.grid(False)
fig.colorbar(image, ax=ax, label="$\\log_{10}$ of the number of states")
save_figure(fig, "two_universe_states",
            "All $2^{32}$ quantum states of two independently quantised universes "
            "of masses $+m$ and $-m$ for one good-sector momentum ($m = 2$, $k = (1, "
            "2, 0, 4)$, $E = 5$ in both), sorted by the total charge (horizontal "
            "axis) and the number of quanta (vertical axis; the normal-ordered total "
```

```
            "energy is $5$ times the number of quanta). The colour is the base-10 "
            "logarithm of the number of states in each square; white squares are "
            "empty. The energies of the two universes add: only the vacuum has "
            "total energy 0, every other state has a positive energy, and a pair of "
            "total charge 0 (a particle in one universe, an antiparticle in the "
            "other) has the energy $2E$, not 0.")
```

Out [12]:

```
PASS 200 random patterns: energy E (quanta) - 16E, charge (b - d) + 16
RESULT pair state: normal-ordered total energy and total charge = 10.0000000000 and
    0.0000000000
PASS a particle in universe 1 and an antiparticle in universe 2: charge 0, energy 2E
RESULT number of patterns = 4294967296
RESULT patterns of zero normal-ordered energy = 1
RESULT fewest quanta of a non-vacuum state of total charge 0 = 2
PASS only the vacuum has energy 0; charge 0 needs at least 2 quanta (energy 2E)
```

Out [12]:

```
Figure 10g.5 saved as Revision/textbook/figures/10g_5_two_universe_states.png
```

Text cell:

```
## 13. The last check

The last cell checks that all five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [13]:

```
for name in ("10g_1_krein_metrics.png", "10g_2_one_system.png",
             "10g_3_anticommutator_blocks.png", "10g_4_block_generator.png",
             "10g_5_two_universe_states.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

Out [13]:

```
PASS the figure file 10g_1_krein_metrics.png exists
PASS the figure file 10g_2_one_system.png exists
PASS the figure file 10g_3_anticommutator_blocks.png exists
PASS the figure file 10g_4_block_generator.png exists
PASS the figure file 10g_5_two_universe_states.png exists
ALL 30 CHECKS PASSED (notebook 10g)
```

Text cell:

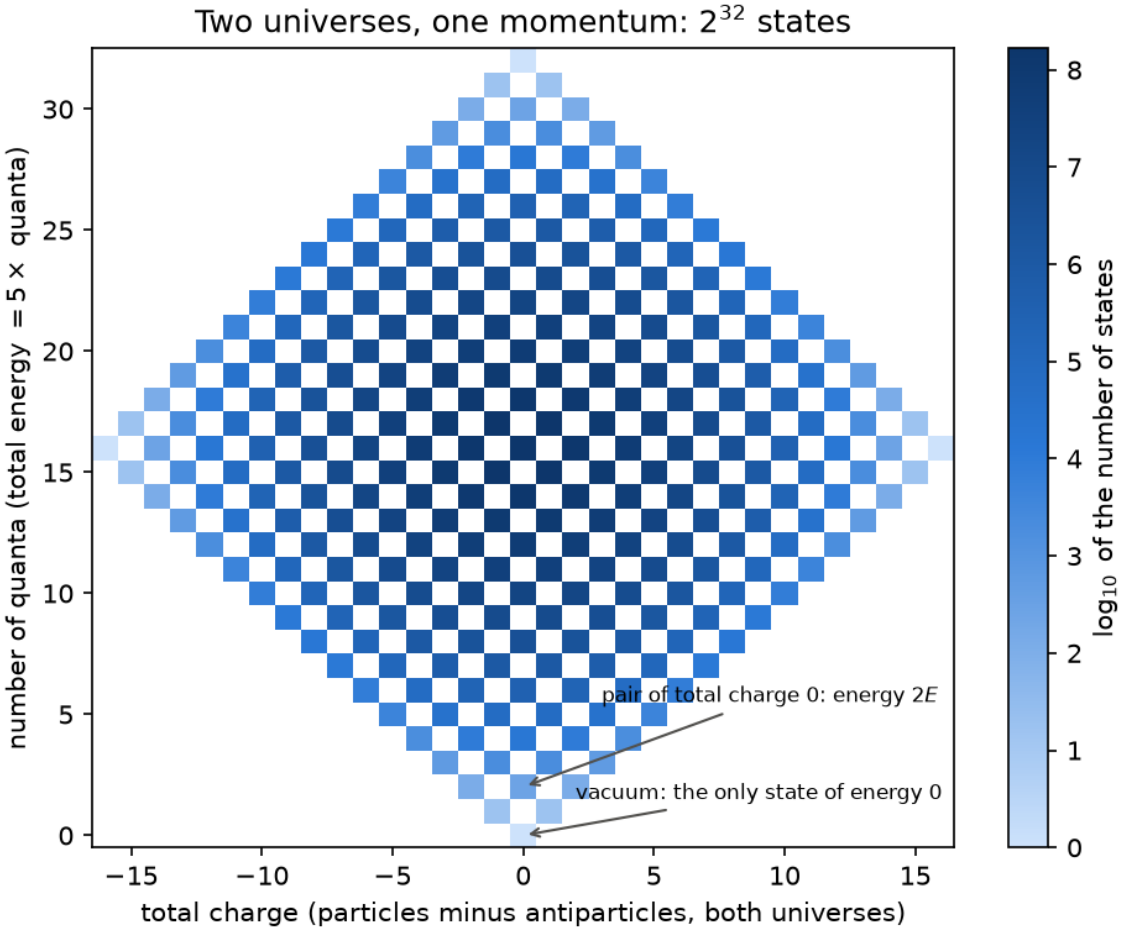

Figure 302: All 2^{32} quantum states of two independently quantised universes of masses $+m$ and $-m$ for one good-sector momentum ($m = 2$, $k = (1, 2, 0, 4)$, $E = 5$ in both), sorted by the total charge (horizontal axis) and the number of quanta (vertical axis; the normal-ordered total energy is 5 times the number of quanta). The colour is the base-10 logarithm of the number of states in each square; white squares are empty. The energies of the two universes add: only the vacuum has total energy 0, every other state has a positive energy, and a pair of total charge 0 (a particle in one universe, an antiparticle in the other) has the energy $2E$, not 0. (Notebook 10g, figure 5.)

```
## 14. What this notebook showed

- PROVED (exactly, for all field values, flat 4+4 space, $U = 0$):
  $\mathcal{L}_m[\Gamma\chi] = -\mathcal{L}_{-m}[\chi]$; the time-derivative kernels
  are $N = B$ for the field, $N = -B$ for its chirality image (the same system in
  the variable $\chi = \Gamma\Psi$) and $N = +B$ for an independent universe of
  mass $-m$, so the canonical rules are $+B$, $-B$ and $+B$ (REPRODUCED: the
  Revision checks of the image metric and of its own quantisation).
- CHECKED on a Fock space: the chirality image carries $-B$, the mirror image keeps
  $+B$; $H_m[\Psi] = -H_{-m}[\Gamma\Psi]$ and $Q[\Psi] = -Q[\Gamma\Psi]$ as
  operators, and the image obeys the wave equation of mass $-m$ under the same
  energy operator. PROVED (exactly): $(-B)(-h'_m) = \Gamma(Bh'_{-m})\Gamma =
  Bh'_m$. So $T \to -T$ and $J \to -J$ of T1 are identities inside ONE quantum
  system: the field and its image are the same universe, and the image is not a
  universe of negative energy.
- CHECKED: an independent second universe must anticommute with the first; the
  chirality image, the mirror image and the conjugate field $\Gamma\Psi^{\dagger T}$
  all fail this (cross anticommutators of rank 16). The chirality image also has
  the wrong rule $-B$, so it cannot be identified with an independently quantised
  mass $-m$ universe (REPRODUCED); a mirror universe has the rule $+B$ of an
  ordinary independent copy.
- CHECKED (32 modes): two independently quantised universes of masses $+m$ and $-m$
  have the rules $+B$ each and anticommute. PROVED (exactly, zero momentum): their
  block generator has the eigenvalues $+m$ and $-m$, 16 each (REPRODUCED);
  COMPUTED: $\pm\sqrt{m^2 + k_1^2}$ with momentum. For one good-sector momentum
  the normal-ordered total energy of every state is $E$ times its number of
  quanta: only the vacuum has energy 0, and a pair of total charge 0 has energy
  $2E$. The energies of two universes ADD; there is no cancellation.
- NOT ESTABLISHED by these computations: that any universe, or any pair of
  universes, is CREATED; any rate, amplitude or mechanism of creation; a positive
  state space for the whole field (the positive Fock spaces here are built for
  single good-sector momenta with frozen coefficients).
```

10.49 Line-by-line walk-through of Notebook 10g

The notebook has 13 code cells, In [1] to In [13]; docstrings are left out of the quotations (they are printed in Section 10.48).

In [1], the set-up cell. Its first 252 lines are comments that repeat the run instructions of Section 10.47. The code is that of In [1] of Notebook 10a (Section 10.10), with the line `NOTEBOOK_ID = "10g" # this notebook: chapter 10, example g`. It prints `Set-up of notebook 10g complete: repository folder found, helpers defined.`

In [2], the matrices and the statements of the record.

```
import contextlib  # lets a block of code print into a text buffer
import io  # the text buffer
import sys  # the screen output, sys.stdout
from math import comb  # comb(8, j): the number of ways to choose j of 8 things

import numpy as np  # numbers, arrays and matrices
import sympy as sp  # exact algebra with symbols
```

```
from matplotlib.colors import LinearSegmentedColormap  # colour scales


REPORT_CHECKS = {}  # report file -> {check name: verdict}; each file is read once


def record_says_pass(record):
    path, separator, check_name = record.partition(", check ")
    if not separator:  # a data entry of a record file that the notebook reads itself
        return True
    if path not in REPORT_CHECKS:
        checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
        REPORT_CHECKS[path] = {c["name"]: c["verdict"].upper() for c in checks}
    return REPORT_CHECKS[path].get(check_name) == "PASS"


def check_record(condition, name, record):
    if not record_says_pass(record):  # the cited check must exist and say PASS
        raise AssertionError("the cited record check is missing or not PASS: " + record)
    buffer = io.StringIO()
    with contextlib.redirect_stdout(buffer):  # what check prints goes to buffer
        check(condition, name, record=record)
    sys.stdout.write(buffer.getvalue())  # one single piece of output
```

The imports of the earlier notebooks (with the binomial coefficient `comb` of Notebook 10c and the colour scales of Notebook 10a) and the helpers `REPORT_CHECKS`, `record_says_pass` and `check_record` of Section 10.10 (a cited report check must still exist with the verdict PASS; the two printed lines come in one piece).

```
BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])  # -1 blue, 0 grey, +1 red
```

The colours and the blue-grey-red colour scale of Notebook 10a.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json").read_text(
    encoding="utf-8"))
# gamma[a] is the matrix gamma^(x_a) of the coordinate x_a, a = 1, ..., 8.
gamma = {a: np.array(fixture["gamma"][a], dtype=int) for a in range(8)}
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the author's sigma16
Gamma = (gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5]
         @ gamma[6] @ gamma[7])  # the chirality matrix
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4)
I16 = np.eye(16)
```

The gammas of the record as whole numbers, C, Γ, B and the identity.

```
diagonal = np.array([-1] * 8 + [1] * 8)  # the expected diagonal of Gamma
sign_rules = (np.array_equal(Gamma, np.diag(diagonal))
              and np.array_equal(Gamma @ C @ Gamma, C)
```

```
            and all(np.array_equal(Gamma @ C @ gamma[a] @ Gamma, -C @ gamma[a])
                    for a in range(8)))
check(sign_rules, "Gamma = diag(-I8, I8), Gamma C Gamma = C, Gamma C g^a Gamma = -C g^a")
```

The check confirms $\Gamma = \mathrm{diag}(-I_8, I_8)$ (`np.diag` makes a diagonal matrix from a list) and the two sign rules of Section 10.44.

```
pairing = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
theorem_Q = next(t for t in pairing["theorems"] if t["id"] == "Q")
say(f"record entry Q: {theorem_Q['name']}; {len(theorem_Q['hypotheses'])} "
    f"hypothesis, {len(theorem_Q['statement'])} statements")
for number, statement in enumerate(theorem_Q["statement"], 1):
    say(f"Q{number}: {statement[:70]} ...")  # the first 70 characters of each
check(len(theorem_Q["statement"]) == 5, "the record states five quantum statements")
```

The pairing record holds a list of theorems; `next(...)` finds the one whose identifier is `Q`. The cell prints its name, `quantum-level reading`, the number of its hypotheses (1) and of its statements (5), and the first 70 characters of each statement (`statement[:70]`): Q1 about the Krein metric $-B$ of the image, Q2 about the image's own generators, Q3 about an independently quantised universe, Q4 about the one-particle Hamiltonians, Q5 about the mirror image. The check confirms the five statements.

```
compare_file = "Revision/pairing/reports/python-pairing.json"
compare_checks = json.loads(repository_file(compare_file).read_text(
    encoding="utf-8"))["checks"]
detail = next(c["detail"] for c in compare_checks
              if c["name"] == "compare.theory.theorem_Q")
confirming = detail.split("all PASS: ")[1].split(", ")  # the sympy checks named
say("compare.theory.theorem_Q: " + detail.split(" is independently")[0])
say(f"confirmed by {len(confirming)} sympy checks: {confirming[0]}, ...")
check_record("5 statements, 12 Wolfram verifications" in detail
             and "confirmed by 10 sympy checks" in detail and len(confirming) == 10
             and all(record_says_pass(f"{compare_file}, check {name}")
                     for name in confirming),
             "theorem Q: 10 sympy checks, all PASS, confirm 12 Wolfram verifications",
             record=f"{compare_file}, check compare.theory.theorem_Q")
```

The sympy pairing report `python-pairing.json` holds, besides its own checks, checks that compare its results with those of the Wolfram report; `compare.theory.theorem_Q` is the one for theorem Q. The cell reads the report and takes the detail text of that check (`next(...)` returns the first matching entry). The text ends with `all PASS:` and the names of the confirming sympy checks separated by commas: `detail.split("all PASS: ")[1]` is the part after `all PASS:` , and `.split(", ")` cuts it into the list `confirming` of the names. The two `say` lines print the first part of the detail text, `Wolfram theorem Q (quantum-level reading; 5 statements, 12 Wolfram verifications)`, and the number 10 with the first name, `Q.canonical_anticommutator`. The check confirms that the text states 5 statements, 12 Wolfram verifications and 10 sympy checks, that it names exactly 10 checks, and, with `record_says_pass` of In [2], that each of the 10 exists in the report with the verdict PASS; `check_record` itself also confirms the verdict of `compare.theory.t`

heorem_Q. These are the numbers that the table of Section 10.44 quotes. Three PASS lines in the cell.

In [3], the Krein metrics of the two images.

```
image_metric = Gamma @ B @ Gamma.conj().T  # Gamma B Gamma^dagger
mirror_metric = gamma[0] @ B @ gamma[0].conj().T  # gamma0 B gamma0^dagger
signature = np.round(np.linalg.eigvalsh(image_metric)).astype(int)
report("eigenvalues -1 and +1 of Gamma B Gamma^dagger",
       f"{int(np.sum(signature == -1))} and {int(np.sum(signature == 1))}")
```

The anticommutator matrices of the two images (Section 10.15): $\Gamma B \Gamma^\dagger$ and $\gamma^{(x_0)} B (\gamma^{(x_0)})^\dagger$. The eigenvalues of the first are rounded to whole numbers (`astype(int)`) and counted: 8 and 8, so $-B$ also has the signature (8,8).

```
check_record(np.array_equal(image_metric, -B)
             and sorted(signature.tolist()) == [-1] * 8 + [1] * 8,
             "the chirality image Gamma Psi carries the Krein metric -B, (8,8)",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_krein_metric")
check_record(np.array_equal(mirror_metric, B),
             "the mirror image gamma^(x0) Psi keeps the Krein metric +B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.T2_image_keeps_B")
check_record(np.array_equal(image_metric, -B) and np.array_equal(mirror_metric, B),
             "Krein signs: Gamma gives -1, gamma^(x0) gives +1",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_Krein_metric_of_images")
```

Three checks: Q1 and Q5 of the sympy record and the corresponding Wolfram check.

```
def draw_matrix(ax, matrix, title):
    image = ax.imshow(matrix, cmap=DIVERGING, vmin=-1.0, vmax=1.0)
    size = matrix.shape[0]
    step = 3 if size <= 16 else 4  # label every third (or fourth) row and column
    ticks = list(range(0, size, step))
    ax.set_xticks(ticks, [str(t + 1) for t in ticks])
    ax.set_yticks(ticks, [str(t + 1) for t in ticks])
    ax.grid(False)  # no grid lines on top of the squares
    ax.set_title(title)
    return image
```

The heat-map function of In [4] of Notebook 10a (Section 10.10), with one change: for matrices larger than 16×16 (the 32×32 matrices of In [9]) every fourth row and column is labelled.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2))
draw_matrix(axes[0], B.imag, "field $\\Psi$: $B$")
draw_matrix(axes[1], image_metric.imag,
            "image $\\Gamma\\Psi$: $\\Gamma B\\Gamma^\\dagger = -B$")
image = draw_matrix(axes[2], mirror_metric.imag,
```

```
                    "mirror $\\gamma^{(x_0)}\\Psi$: $+B$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="imaginary part of the entry")
save_figure(fig, "krein_metrics",
...)
```

The three matrices are purely imaginary; their imaginary parts are drawn. Figure 1 of Notebook 10g shows the middle picture as the left one with every red and blue square exchanged ($-B$), and the right picture equal to the left one ($+B$).

In [4], the Lagrangian identity and the three kernels, exactly.

```
g = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # exact gammas
C_exact = g[0] * g[1] * g[2] * g[3]
Gamma_exact = sp.diag(*[int(x) for x in diagonal])  # diag(-I8, I8), exact
B_exact = -sp.I * C_exact * g[4]
m = sp.Symbol("m", real=True)  # the mass: any real number
```

Exact gammas, C, Γ (`sp.diag` with the 16 whole numbers of the diagonal) and B, and the letter m.

```
field = sp.Matrix(sp.symbols("c1:17"))  # chi_A, A = 1, ..., 16
field_conj = sp.Matrix(sp.symbols("cc1:17"))  # their complex conjugates chi_A^*
# d[a] and d_conj[a]: the derivatives of chi and of chi^* along x_a
d = {a: sp.Matrix(sp.symbols(f"d{a}c1:17")) for a in range(8)}
d_conj = {a: sp.Matrix(sp.symbols(f"d{a}cc1:17")) for a in range(8)}
```

Letters for the 16 components χ_A (`c1` to `c16`), for their complex conjugates (`cc1` to `cc16`), and for the derivatives of both along each of the eight directions (`d1c1` to `d8c16` and `d1cc1` to `d8cc16`): $16 \times 2 + 128 \times 2 = 288$ independent letters. sympy treats a component and its conjugate as independent letters, which is how a Lagrangian uses them.

```
def lagrangian(mass, M):
    psi, psi_conj = M * field, M * field_conj  # M is real: (M chi)^* = M chi^*
    total = sp.Integer(0)
    for a in range(8):
        kinetic = C_exact * g[a]  # the matrix C gamma^(x_a)
        dpsi, dpsi_conj = M * d[a], M * d_conj[a]
        total += sp.Rational(1, 2) * ((psi_conj.T * kinetic * dpsi)[0]
                                      - (dpsi_conj.T * kinetic * psi)[0])
    total -= mass * (psi_conj.T * C_exact * psi)[0]
    return sp.expand(total)
```

`lagrangian(mass, M)` evaluates $\mathcal{L}_{\text{mass}}$ of Section 10.44 on the field $M\chi$ for a constant real matrix M: the column of conjugates of $M\chi$ is $M\chi^*$, and the row $(M\chi)^\dagger$ is its transpose, written `psi_conj .T`. For each direction the kinetic term $\frac{1}{2}((M\chi)^\dagger C\gamma^{(x_a)}\partial_a(M\chi) - \partial_a(M\chi)^\dagger C\gamma^{(x_a)}M\chi)$ is added (`+=`), then the mass term is subtracted (`-=`), and the result is multiplied out.

```
def kernel(expression):
    return sp.Matrix(16, 16, lambda A, C_: -2 * sp.I * expression.coeff(
        field_conj[A] * d[4][C_]))
```

The kernel N is read off the terms $\frac{i}{2} N_{AC} \chi^*_A \partial_4 \chi_C$: `expression.coeff(...)` returns the coefficient of the product $\chi^*_A \, \partial_4 \chi_C$ in the expanded Lagrangian, and N_{AC} is $-2i$ times it (because $-2i \cdot \frac{i}{2} = 1$).

```
L_field = lagrangian(m, sp.eye(16))  # L_m[chi]
L_image = lagrangian(m, Gamma_exact)  # L_m[Gamma chi]: the same system, new variables
L_independent = lagrangian(-m, sp.eye(16))  # L_-m[chi]: an independent universe
say(f"number of terms of L_m[chi]: {len(L_field.args)}")
identity_ok = sp.expand(L_image + L_independent) == 0
check(identity_ok, "exact: L_m[Gamma chi] = -L_-m[chi] for all components and "
                   "derivatives")
```

The three Lagrangians of Section 10.44: of the field, of its chirality image (the same system in the variable χ), and of an independent universe of mass $-m$. The expanded Lagrangian of the field has 272 terms (`.args` lists the terms of a sum). The check proves $\mathcal{L}_m[\Gamma\chi] + \mathcal{L}_{-m}[\chi] = 0$ for all values of the 288 letters.

```
N_field, N_image, N_independent = map(kernel, (L_field, L_image, L_independent))
kernels_ok = (N_field == B_exact and N_image == -B_exact
              and N_independent == B_exact)
say(f"kernels equal to B: field {N_field == B_exact}, image {N_image == B_exact}, "
    f"independent universe {N_independent == B_exact}; image kernel = -B: "
    f"{N_image == -B_exact}")
check_record(kernels_ok,
             "kernels: N = B (field), N = -B (image), N = +B (independent universe)",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_symplectic_kernel_grassmann")
```

`map(kernel, ...)` applies `kernel` to each of the three Lagrangians. The printed line shows the kernels equal to B for the field and the independent universe (`True`), not for the image (`False`), whose kernel is $-B$ (`True`). The check reproduces the Wolfram record check `Q_symplectic_kernel_grassmann`.

```
anticommutators = [N.inv() for N in (N_field, N_image, N_independent)]
check_record(anticommutators[0] == B_exact and anticommutators[1] == -B_exact
             and anticommutators[2] == B_exact,
             "canonical rules N^(-1): {Psi,Psi^dag} = B, {chi,chi^dag} = -B, +B",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_own_quantisation")
```

The canonical rules are the inverses of the kernels (Section 10.12): B, $-B$ and $+B$. Three PASS lines in the cell.

In [5], the field and its two images on a Fock space.

```
def sign_below(n, p):
    return -1 if bin(n & ((1 << p) - 1)).count("1") % 2 else 1


def annihilate(p, state):
```

```
    result = {}
    for n, amplitude in state.items():
        if n >> p & 1:  # mode p is occupied in the pattern n
            new = n ^ (1 << p)  # the same pattern with mode p emptied
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result


def create(p, state):
    result = {}
    for n, amplitude in state.items():
        if not n >> p & 1:  # mode p is empty in the pattern n
            new = n | (1 << p)  # the same pattern with mode p filled
            result[new] = result.get(new, 0) + sign_below(n, p) * amplitude
    return result
```

The Fock space of In [3] of Notebook 10b (Section 10.19): the sign $(-1)^s$, f_p and f_p^*. Nothing in these functions limits the number of modes, so they work for the 32 modes used below.

```
def add(*terms):
    result = {}
    for coefficient, state in terms:
        for n, amplitude in state.items():
            result[n] = result.get(n, 0) + coefficient * amplitude
    return result


def inner(left, right):
    return sum(np.conj(a) * right.get(n, 0) for n, a in left.items())


def largest(state):
    return max((abs(a) for a in state.values()), default=0.0)
```

Combinations of states, the positive inner product and the largest amplitude, as in Notebook 10b.

```
rng = np.random.default_rng(12345)  # random numbers with a fixed seed


def random_state(patterns, modes):
    state = {}
    for n in rng.integers(0, 2 ** modes, size=patterns):
        state[int(n)] = complex(rng.normal(), rng.normal())
    length = np.sqrt(inner(state, state).real)
    return {n: a / length for n, a in state.items()}
```

A normalised random state of a few patterns, as in Notebook 10b, now for any number of modes.

```
def field_op(offset):
    return [lambda s, A=A: annihilate(A - 1 + offset, s) for A in range(1, 17)]
```

```
def conjugate_op(offset):
    return [lambda s, A=A: add(*[(B[C_ - 1, A - 1], create(C_ - 1 + offset, s))
                                 for C_ in range(1, 17) if B[C_ - 1, A - 1] != 0])
            for A in range(1, 17)]
```

`field_op(offset)` returns the 16 operators $\Psi_A = f_{A-1+\text{offset}}$ of one universe whose modes start at `offset`, as a list of functions; `conjugate_op(offset)` the canonical conjugates $\Psi_A^\dagger = \sum_C f^*_{C-1+\text{offset}} B_{CA}$ (the positive realisation of Section 10.14). Each `lambda s, A=A: ...` is a small function of a state `s`; the default value `A=A` stores the current value of `A` in each function (without it all 16 functions would use the last value of the loop).

```
def mapped(M, ops):
    return [lambda s, A=A: add(*[(M[A - 1, D - 1], ops[D - 1](s))
                                 for D in range(1, 17) if M[A - 1, D - 1] != 0])
            for A in range(1, 17)]


def mapped_conjugate(M, conj_ops):
    return mapped(M.conj(), conj_ops)
```

`mapped(M, ops)` returns the 16 operators $(MX)_A = \sum_D M_{AD} X_D$ of an image; `mapped_conjugate(M, conj_ops)` its conjugate $((M\Psi)^\dagger)_A = \sum_D \Psi_D^\dagger M^*_{AD}$, which is `mapped` with the conjugate matrix applied to the conjugate operators.

```
def anticommutator_matrix(X, Y, state):
    result = np.zeros((len(X), len(Y)), dtype=complex)
    for i, x in enumerate(X):
        for j, y in enumerate(Y):
            both = add((1, x(y(state))), (1, y(x(state))))
            result[i, j] = inner(state, both)
    return result
```

For two lists of operators the matrix of the numbers $\langle\phi|\{X_i, Y_j\}\phi\rangle$; for a normalised state ϕ and anticommutators that are numbers, these entries ARE the anticommutators.

```
Psi, Psi_dagger = field_op(0), conjugate_op(0)  # one universe: modes 0, ..., 15
phi = random_state(6, 16)
own_rule = anticommutator_matrix(Psi, Psi_dagger, phi)
image_rule = anticommutator_matrix(mapped(Gamma, Psi),
                                   mapped_conjugate(Gamma, Psi_dagger), phi)
mirror_rule = anticommutator_matrix(mapped(gamma[0], Psi),
                                    mapped_conjugate(gamma[0], Psi_dagger), phi)
errors = [np.max(np.abs(own_rule - B)), np.max(np.abs(image_rule + B)),
          np.max(np.abs(mirror_rule - B))]
report("largest deviations from B, -B and +B", ", ".join(f"{e:.1e}" for e in errors))
check(max(errors) < 1e-12,
      "Fock space: {Psi, Psi^dag} = B, image -B, mirror +B (256 entries each)")
```

One universe on the modes 0 to 15 and a random normalised state. The three 16×16 anticommutator matrices of the field, of its chirality image and of its mirror image are compared with B, $-B$ and $+B$; the deviations are printed as `2.2e-16` each, rounding only.

In [6], one quantum system: $T \to -T$ and $J \to -J$ as operator identities.

```
m_value = 1.3  # any mass and momenta (here with momenta along the extra times too)
k = {0: 1.1, 1: 0.4, 2: -0.7, 3: 0.2, 5: 0.9, 6: -0.3, 7: 0.5}


def h_prime(mass):
    return mass * C - 1j * sum(k_a * (C @ gamma[a]) for a, k_a in k.items())
```

The mass and the momenta of In [4] of Notebook 10b, and the energy matrix $h'_{\text{mass}} = \text{mass}\, C - i \sum_a k_a C \gamma^{(x_a)}$ of Section 10.12.

```
def bilinear(X_dagger, N, X, state):
    terms = []
    for C_ in range(16):
        lowered = X[C_](state)
        if lowered:
            terms += [(N[A, C_], X_dagger[A](lowered)) for A in range(16)
                      if N[A, C_] != 0]
    return add(*terms)
```

`bilinear(X_dagger, N, X, state)` applies $\sum_{A,C} X^\dagger_A N_{AC} X_C$ to a state: for each C it applies X_C once and then every $X^\dagger_A$ with $N_{AC} \neq 0$.

```
image_field = mapped(Gamma, Psi)  # Gamma Psi
image_conjugate = mapped_conjugate(Gamma, Psi_dagger)  # (Gamma Psi)^dagger
worst_H, worst_Q, values = 0.0, 0.0, []
for _ in range(150):
    state = random_state(4, 16)
    H_field = bilinear(Psi_dagger, h_prime(m_value), Psi, state)
    H_image = bilinear(image_conjugate, h_prime(-m_value), image_field, state)
    Q_field = bilinear(Psi_dagger, B, Psi, state)
    Q_image = bilinear(image_conjugate, B, image_field, state)
```

For 150 random states the four operators of Section 10.44 are applied: the energy of the field $H_m[\Psi] = \Psi^\dagger h'_m \Psi$, the energy that the mass $-m$ theory assigns to the image $H_{-m}[\Gamma\Psi]$, the charge $Q[\Psi] = \Psi^\dagger B \Psi$ and the charge assigned to the image $Q[\Gamma\Psi]$.

```
    worst_H = max(worst_H, largest(add((1, H_field), (1, H_image))))
    worst_Q = max(worst_Q, largest(add((1, Q_field), (1, Q_image))))
    values.append([inner(state, X).real for X in (H_field, H_image, Q_field,
                                                  Q_image)])
values = np.array(values)  # columns: <H_m[Psi]>, <H_-m[Gamma Psi]>, <Q>, <Q image>
report("largest |(H_m[Psi] + H_-m[Gamma Psi]) phi| over 150 states", f"{worst_H:.1e}")
report("largest |(Q[Psi] + Q[Gamma Psi]) phi| over 150 states", f"{worst_Q:.1e}")
```

```
check_record(worst_H < 1e-12 and worst_Q < 1e-12,
             "operators: H_m[Psi] = -H_-m[Gamma Psi] and Q[Psi] = -Q[Gamma Psi]",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_generators_of_the_image_grassmann")
```

The sums $(H_m[\Psi] + H_{-m}[\Gamma\Psi])\phi$ and $(Q[\Psi] + Q[\Gamma\Psi])\phi$ must vanish; the largest amplitudes are printed as `0.0e+00`. The expectation values are kept for the figure. The check reproduces the Wolfram record check `Q_generators_of_the_image_grassmann`: Q2 as an operator identity.

```
h_mode_minus = B @ h_prime(-m_value)  # the mode Hamiltonian of mass -m
worst = 0.0
for c_ in range(16):
    H_after = bilinear(Psi_dagger, h_prime(m_value), Psi, image_field[c_](phi))
    H_before = image_field[c_](bilinear(Psi_dagger, h_prime(m_value), Psi, phi))
    commutator = add((1, H_after), (-1, H_before))  # [H, (Gamma Psi)_c] phi
    expected = add(*[(-h_mode_minus[c_, d_], image_field[d_](phi))
                     for d_ in range(16)])
    worst = max(worst, largest(add((1, commutator), (-1, expected))))
report("largest violation of [H, (Gamma Psi)_c] = -(h_-m Gamma Psi)_c", f"{worst:.1e}")
check(worst < 1e-12, "the image obeys i d4 (Gamma Psi) = h_-m Gamma Psi under the "
                     "same H")
```

With the energy $H = H_m[\Psi]$ of the system, the commutator $[H, (\Gamma\Psi)_c]$ must equal $-(h_{-m}\Gamma\Psi)_c$ with $h_{-m} = Bh'_{-m}$; by the Heisenberg equation (Section 10.12) this is $i\,\partial_4(\Gamma\Psi) = h_{-m}\Gamma\Psi$. The largest violation is printed as `1.6e-17`. Two PASS lines in the cell.

In [7], the same exactly, and a picture.

```
k_symbol = {a: sp.Symbol(f"k{a}", real=True) for a in (0, 1, 2, 3, 5, 6, 7)}


def h_prime_exact(mass):
    result = mass * C_exact
    for a, k_a in k_symbol.items():
        result = result - sp.I * k_a * C_exact * g[a]
    return result
```

Real letters for the seven momenta, and the exact energy matrix h'_{mass}.

```
zero = sp.zeros(16, 16)
own_data = (-B_exact) * (-h_prime_exact(m))  # the image: metric -B, energy -h'_m
by_map = Gamma_exact * (B_exact * h_prime_exact(-m)) * Gamma_exact
same = ((own_data - by_map).applyfunc(sp.expand) == zero
        and (own_data - B_exact * h_prime_exact(m)).applyfunc(sp.expand) == zero)
energy_rule = (Gamma_exact * h_prime_exact(m) * Gamma_exact
               + h_prime_exact(-m)).applyfunc(sp.expand) == zero
check_record(same and energy_rule,
             "exact: (-B)(-h'_m) = Gamma (B h'_-m) Gamma = B h'_m; "
             "Gamma h'_m Gamma = -h'_-m",
             record="Revision/pairing/reports/python-pairing.json, check "
                    "Q.image_generators_same_dynamics")
```

The identities of Q2, exactly for all masses and momenta, in the direction in which the record states them: `own_data` is the generator of the image of a field of mass $-m$, which carries the metric $-B$ and the energy matrix $-h'_m$ (the comment of the line says so), and `by_map` is the generator Bh'_{-m} of that field carried over by Γ. The check confirms $(-B)(-h'_m) = \Gamma(Bh'_{-m})\Gamma = Bh'_m$, and $\Gamma h'_m \Gamma = -h'_{-m}$. Since m is a symbol, replacing m by $-m$ gives the direction of Section 10.44: the image of the field of mass m evolves by $(-B)(-h'_{-m}) = Bh'_{-m} = h_{-m}$. The check reproduces the sympy record check `Q.image_generators_same_dynamics`.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.4))
panels = ((axes[0], 0, 1, BLUE, "energy",
           "$\\langle H_m[\\Psi]\\rangle$", "$\\langle H_{-m}[\\Gamma\\Psi]\\rangle$"),
          (axes[1], 2, 3, ORANGE, "charge",
           "$\\langle Q[\\Psi]\\rangle$", "$\\langle Q[\\Gamma\\Psi]\\rangle$"))
```

Two panels; for each, the tuple names the drawing area, the two columns of `values` to plot, a colour, a word and the two axis labels.

```
for ax, x_col, y_col, colour, what, x_label, y_label in panels:
    ax.plot(values[:, x_col], values[:, y_col], "o", color=colour, markersize=4,
            label="150 random states")
    low, high = values[:, x_col].min() - 0.5, values[:, x_col].max() + 0.5
    ax.plot([low, high], [-low, -high], color=GREY, linewidth=1,
            label="the line $y = -x$")
    ax.set_xlabel(f"{what} of the field, {x_label}")
    ax.set_ylabel(f"{what} given to the image, {y_label}")
    ax.legend(fontsize=8)
axes[0].set_title("$T \\to -T$ inside one system")
axes[1].set_title("$J \\to -J$ inside one system")
save_figure(fig, "one_system",
...)
```

Each state is a dot at (value of the field, value given to the image), and the grey line is $y = -x$. Figure 2 of Notebook 10g shows all 150 dots exactly on the line $y = -x$ in both panels: the reversal of the energy and of the charge under T1 is an identity between operators of ONE quantum system, not a statement about a second universe.

In [8], which images could be an independent universe?

```
conjugate_field = mapped(Gamma, Psi_dagger)  # Psi^c_A = sum_D Gamma_AD Psi^dag_D
# Its canonical conjugate: (Psi^c)^dagger_A = sum_D conj(Gamma_AD) Psi_D, by the
# rule (Psi^dagger)^dagger = Psi of the canonical relations (Gamma is real).
conjugate_field_dagger = mapped(Gamma, Psi)
```

The conjugate field $\Psi^c = \Gamma\Psi^{\dagger T}$ of Section 10.15 and, as the comment lines say, its canonical conjugate $\sum_D \Gamma_{AD}\Psi_D$ (Γ is real, and the conjugate of $\Psi^\dagger$ is Ψ).

```
candidates = {
    "Gamma Psi": (image_field, image_conjugate),
    "gamma0 Psi": (mapped(gamma[0], Psi), mapped_conjugate(gamma[0], Psi_dagger)),
```

```
    "Psi^c = Gamma Psi^dag T": (conjugate_field, conjugate_field_dagger),
}
phi = random_state(6, 16)
results = {}
```

The three candidates of Section 10.44, each with its operators and their conjugates, and a new random state.

```
for label, (X, X_dagger) in candidates.items():
    metric = anticommutator_matrix(X, X_dagger, phi)  # {X_A, X_C^dagger}
    cross_1 = anticommutator_matrix(X, Psi_dagger, phi)  # {X_A, Psi^dagger_C}
    cross_2 = anticommutator_matrix(X, Psi, phi)  # {X_A, Psi_C}
    sign = (1 if np.allclose(metric, B) else -1 if np.allclose(metric, -B) else 0)
    ranks = (np.linalg.matrix_rank(cross_1), np.linalg.matrix_rank(cross_2))
    results[label] = (sign, ranks, cross_1, cross_2)
    say(f"{label:24s} metric {sign:+d} B; rank of {{X, Psi^dag}} = {ranks[0]:2d}, "
        f"rank of {{X, Psi}} = {ranks[1]:2d}")
```

For each candidate: its own anticommutator matrix (its sign, $+1$ for B and -1 for $-B$), and the two cross matrices with the first field, with their ranks. In an f-string a doubled brace prints a single brace. The printed lines show: $\Gamma\Psi$ with the metric $-B$ and the ranks 16 and 0; $\gamma^{(x_0)}\Psi$ with $+B$ and the ranks 16 and 0; Ψ^c with $+B$ and the ranks 0 and 16. Independence would need both ranks to be 0; every candidate has a cross matrix of rank 16.

```
chirality = results["Gamma Psi"]
conjugate = results["Psi^c = Gamma Psi^dag T"]
check_record(chirality[0] == -1 and chirality[1] == (16, 0)
             and np.allclose(chirality[2], Gamma @ B),
             "Gamma Psi: metric -B and {Gamma Psi, Psi^dag} = Gamma B of rank 16",
             record="Revision/pairing/reports/wolfram-pairing.json, check "
                    "Q_no_identification_of_independent_universes")
check_record(conjugate[0] == 1 and conjugate[1] == (0, 16)
             and np.allclose(conjugate[3], -Gamma @ B),
             "Psi^c keeps +B (Gamma B^T Gamma^dag = B) but {Psi^c, Psi} = -Gamma B",
             record="Revision/lead_checks/reports/charge-conjugation-and-u1.json, "
                    "check quantum_charge_conjugation_unitary_type")
check(results["gamma0 Psi"][0] == 1 and results["gamma0 Psi"][1] == (16, 0),
      "gamma0 Psi keeps +B; as an operator of the same field it is not independent")
```

Three checks: the chirality image has $-B$ and $\{\Gamma\Psi, \Psi^\dagger\} = \Gamma B$ (Q3, reproducing `Q_no_identification_of_independent_universes`); the conjugate field keeps $+B$ but $\{\Psi^c, \Psi\} = -\Gamma B$ (reproducing `quantum_charge_conjugation_unitary_type`); the mirror image keeps $+B$ but is built from the same field.

In [9], two independently quantised universes.

```
Psi_1, Psi_1_dagger = field_op(0), conjugate_op(0)  # universe 1: modes 0..15
Psi_2, Psi_2_dagger = field_op(16), conjugate_op(16)  # universe 2: modes 16..31
phi_32 = random_state(6, 32)
two_universes = anticommutator_matrix(Psi_1 + Psi_2, Psi_1_dagger + Psi_2_dagger,
```

```
                                    phi_32)
image_pair = anticommutator_matrix(
    Psi_1 + mapped(Gamma, Psi_1),
    Psi_1_dagger + mapped_conjugate(Gamma, Psi_1_dagger), phi_32)
```

Universe 1 lives on the modes 0 to 15 and universe 2 on the modes 16 to 31 of one Fock space of 32 modes, each with its own canonical conjugate; `phi_32` is a random state of 32 modes. The `+` of two lists of operators joins them into a list of 32. `two_universes` is the 32×32 anticommutator matrix of (Ψ_1, Ψ_2) with their conjugates; `image_pair` the same for a field and its chirality image.

```
zero_16 = np.zeros((16, 16))
expected_two = np.block([[B, zero_16], [zero_16, B]])
expected_image = np.block([[B, B @ Gamma], [Gamma @ B, -B]])
cross_zero = np.max(np.abs(anticommutator_matrix(Psi_1, Psi_2, phi_32)))
report("largest |{Psi_1, Psi_2}| (must be 0)", f"{cross_zero:.1e}")
check(np.allclose(two_universes, expected_two) and cross_zero < 1e-12
      and np.allclose(image_pair, expected_image),
      "two universes: diag(B, B); field and image: blocks B, B Gamma, Gamma B, -B")
```

`np.block` assembles a matrix from blocks. The expected matrices are $\mathrm{diag}(B, B)$ for two universes, and the blocks B, $B\Gamma$, ΓB, $-B$ for a field and its image (Section 10.44: $\{\Psi, (\Gamma\Psi)^\dagger\} = B\Gamma$ because $(\Gamma\Psi)^\dagger = \Psi^\dagger\Gamma$). The largest $\{\Psi_1, \Psi_2\}$ is printed as `0.0e+00`: the two universes anticommute.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 5.0))
draw_matrix(axes[0], two_universes.imag,
            "independent: $(\\Psi_1, \\Psi_2)$")
image = draw_matrix(axes[1], image_pair.imag,
                    "one field and its image: $(\\Psi_1, \\Gamma\\Psi_1)$")
for ax in axes:
    ax.axhline(15.5, color=GREY, linewidth=1)  # the border between the two halves
    ax.axvline(15.5, color=GREY, linewidth=1)
fig.colorbar(image, ax=list(axes), shrink=0.75, label="imaginary part of the entry")
save_figure(fig, "anticommutator_blocks",
...)
```

The two 32×32 matrices as heat maps, with grey lines between the halves. Figure 3 of Notebook 10g shows on the left two copies of B on the diagonal and empty off-diagonal blocks (two independent universes), and on the right a lower right block $-B$ and full off-diagonal blocks (a field and its image are one system).

In [10], the generators add.

```
zero_momentum = {k_a: 0 for k_a in k_symbol.values()}
block = sp.diag((B_exact * h_prime_exact(m)).subs(zero_momentum),
                (B_exact * h_prime_exact(-m)).subs(zero_momentum))
block_eigenvalues = block.eigenvals()  # {eigenvalue: how often}
say(f"exact eigenvalues of diag(B h'_m, B h'_-m) at k = 0: +m "
    f"{block_eigenvalues.get(m, 0)} times, -m {block_eigenvalues.get(-m, 0)} times, "
    f"{len(block_eigenvalues)} different values")
check_record(block_eigenvalues == {m: 16, -m: 16},
```

```
        "block generator at k = 0: eigenvalues +m and -m, 16 each, none zero",
        record="Revision/pairing/reports/python-pairing.json, check "
               "Q.no_cancellation_independent_universes")
```

At zero momentum (every momentum letter replaced by 0) the function `sp.diag` assembles the exact block generator $\mathrm{diag}(Bh'_m, Bh'_{-m})$ from its two 16×16 blocks, and the method `eigenvals` returns its exact eigenvalues, each with its multiplicity. The printed line shows $+m$ 16 times and $-m$ 16 times (2 different values), as Section 10.44 derived. The check reproduces the record check `Q.no_cancellation_independent_universes` of the sympy pairing report.

```
def block_numbers(mass, k1):
    def energy(mu):  # the energy matrix of mass mu and momentum k1
        return mu * C - 1j * k1 * (C @ gamma[1])
    top, bottom = B @ energy(mass), B @ energy(-mass)
    values = np.concatenate([np.linalg.eigvals(top), np.linalg.eigvals(bottom)])
    return np.sort(values.real)
```

The 32 eigenvalues of the block generator as numbers, for a mass and a momentum k_1 along x_1: the inner function `energy` is h'_μ, and the eigenvalues of the two blocks are joined and sorted.

```
masses = np.linspace(-3.0, 3.0, 121)
spectra_0 = np.array([block_numbers(x, 0.0) for x in masses])
spectra_k = np.array([block_numbers(x, 1.5) for x in masses])
formula_k = np.sqrt(masses ** 2 + 1.5 ** 2)
error_k = max(np.max(np.abs(spectra_k[:, 16:] - formula_k[:, None])),
              np.max(np.abs(spectra_k[:, :16] + formula_k[:, None])))
report("largest deviation from +-sqrt(m^2 + k1^2), 16 each", f"{error_k:.1e}")
check(error_k < 1e-10 and np.min(np.abs(spectra_k)) >= 1.5 - 1e-10,
      "with k1 = 1.5 the 32 eigenvalues are +-sqrt(m^2 + k1^2): never 0")
```

For 121 masses from -3 to 3, at zero momentum and at $k_1 = 1.5$: with the momentum the 16 largest eigenvalues must be $+\sqrt{m^2 + k_1^2}$ and the 16 smallest $-\sqrt{m^2 + k_1^2}$; the largest deviation is printed as `4.9e-15`, and no eigenvalue is smaller in size than $k_1 = 1.5$.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), sharey=True)
axes[0].plot(masses, spectra_0[:, [0, 31]], color=BLUE, linewidth=2)
axes[0].set_title("zero momentum: $+m$ and $-m$, 16 each")
axes[1].plot(masses, spectra_k[:, [0, 31]], color=GREEN, linewidth=2)
axes[1].set_title("momentum $k_1 = 1.5$: $\\pm\\sqrt{m^2 + k_1^2}$, 16 each")
for ax in axes:
    ax.axhline(0.0, color=GREY, linewidth=1)
    ax.set_xlabel("mass $m$ of universe 1 (universe 2 has $-m$)")
axes[0].set_ylabel("eigenvalues of diag$(Bh'_m, Bh'_{-m})$")
save_figure(fig, "block_generator",
...)
```

The smallest and the largest eigenvalue (columns 0 and 31; each carries 16) against the mass. Figure 4 of Notebook 10g shows on the left the two straight lines $\pm m$, crossing only at $m = 0$, and on the

right the two branches $\pm\sqrt{m^2 + 2.25}$, which never reach 0: the generators of the two universes add, and nothing in them cancels.

In [11], one good-sector momentum in both universes.

```
def orthonormal_columns(P):
    basis = []
    for column in P.T:
        v = column.astype(complex)
        for _ in range(2):  # the second pass removes rounding errors
            for e in basis:
                v = v - (e.conj() @ v) * e
        length = np.sqrt((v.conj() @ v).real)
        if length > 1e-8:
            basis.append(v / length)
    return np.array(basis).T


def mode_hamiltonian(mass, momenta):
    h = -1j * mass * gamma[4]
    for a, k_a in momenta.items():
        h = h - k_a * (gamma[4] @ gamma[a])
    return h
```

The Gram-Schmidt function (Section 10.19) and the mode Hamiltonian of Section 10.3, as in the earlier notebooks.

```
sample = {0: 4, 1: 1, 2: 2}  # the recorded good-sector momentum, with m = 2
h_plus, h_minus = mode_hamiltonian(2, sample), mode_hamiltonian(-2, sample)
E = 5.0  # sqrt(4 + 1 + 4 + 16)
W1 = np.hstack([orthonormal_columns((I16 + h_plus / E) / 2),
                orthonormal_columns((I16 - h_plus / E) / 2)])  # u_1..u_8, v_1..v_8
W2 = Gamma @ W1  # the eigenvectors of h_-m = Gamma h_m Gamma
check(np.allclose(h_minus @ W2[:, :8], E * W2[:, :8])
      and np.allclose(h_minus @ W2[:, 8:], -E * W2[:, 8:])
      and np.allclose(W2.conj().T @ W2, I16),
      "Gamma u_s and Gamma v_s are orthonormal eigenvectors of h_-m, energies +-E")
```

The recorded good-sector momentum $m = 2$, $k = (k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$, $E = 5$. Universe 1 uses the orthonormal eigenvectors u_s and v_s of h_m (Section 10.20); universe 2 uses Γu_s and Γv_s, which are eigenvectors of $h_{-m} = \Gamma h_m \Gamma$ with the same eigenvalues ($h_{-m}\Gamma u = \Gamma h_m u = E\,\Gamma u$) and orthonormal ($\Gamma$ keeps lengths). The check confirms both.

```
def universe(W, offset):
    def F(p, s):  # b (particle modes p < 8) or d^* (antiparticle modes p >= 8)
        return annihilate(p + offset, s) if p < 8 else create(p + offset, s)

    def F_star(p, s):  # the Hilbert adjoint of F_p
        return create(p + offset, s) if p < 8 else annihilate(p + offset, s)
```

`universe(W, offset)` builds the positive realisation of one universe whose 16 modes start at `offset`. The inner functions `F` and `F_star` are those of In [4] of Notebook 10c (Section 10.26): F_p is b for the particle modes and d^* for the antiparticle modes, and `F_star` its Hilbert adjoint.

```
    psi = [lambda s, A=A: add(*[(W[A, p], F(p, s)) for p in range(16)])
           for A in range(16)]  # Psi_A = sum_p W_Ap F_p
    chi = [lambda s, A=A: add(*[(np.conj(W[A, p]), F_star(p, s)) for p in range(16)])
           for A in range(16)]  # chi_A = sum_p conj(W_Ap) F_p^*
    psi_dagger = mapped(B.T, chi)  # Psi^dagger_A = sum_C chi_C B_CA
    return psi, chi, psi_dagger, F, F_star
```

The field $\Psi_A = \sum_p W_{Ap} F_p$, its Hilbert adjoint χ_A and the canonical conjugate $\Psi^\dagger_A = \sum_C \chi_C B_{CA}$, written as `mapped(B.T, chi)` because $\sum_C (B^T)_{AC} \chi_C = \sum_C \chi_C B_{CA}$. (Here the components are counted from 0, so `W[A, p]` has no `- 1`.) The function returns the three lists and the two mode functions.

```
def mode_bilinear(W, F, F_star, N, state):
    M = W.conj().T @ N @ W
    terms = []
    for q in range(16):
        lowered = F(q, state)
        if lowered:
            terms += [(M[p, q], F_star(p, lowered)) for p in range(16)
                      if abs(M[p, q]) > 1e-12]
    return add(*terms)
```

`mode_bilinear` applies $\chi N \Psi = \sum_{p,q} (W^\dagger N W)_{pq} F^*_p F_q$, the function `bilinear` of In [4] of Notebook 10c; it is much faster than the sum over the components because $W^\dagger h W$ is diagonal.

```
psi_1, chi_1, psi_1_dagger, F_1, F_1_star = universe(W1, 0)
psi_2, chi_2, psi_2_dagger, F_2, F_2_star = universe(W2, 16)
phi_32 = random_state(4, 32)
rule_1 = anticommutator_matrix(psi_1, psi_1_dagger, phi_32)
rule_2 = anticommutator_matrix(psi_2, psi_2_dagger, phi_32)
rule_12 = anticommutator_matrix(psi_1, psi_2_dagger, phi_32)
check(np.allclose(rule_1, B) and np.allclose(rule_2, B) and np.allclose(rule_12, 0),
      "positive realisation: {Psi_1, Psi_1^dag} = {Psi_2, Psi_2^dag} = B, cross 0")
```

The two universes on the modes 0 to 15 and 16 to 31. The check confirms the canonical rule B in each and the cross anticommutator 0.

```
def total_energy(state):
    return add((1, mode_bilinear(W1, F_1, F_1_star, h_plus, state)),
               (1, mode_bilinear(W2, F_2, F_2_star, h_minus, state)))


def total_charge(state):
    return add((1, mode_bilinear(W1, F_1, F_1_star, I16, state)),
               (1, mode_bilinear(W2, F_2, F_2_star, I16, state)))
```

The total energy $H_1 + H_2$ (with $H = \chi h \Psi$ in each universe, Section 10.20) and the total charge $Q_1 + Q_2$ (with $Q = \chi \Psi$).

```
VACUUM = {0: 1.0}
vacuum_energy = inner(VACUUM, total_energy(VACUUM)).real
vacuum_charge = inner(VACUUM, total_charge(VACUUM)).real
report("vacuum energy and charge of the two universes",
       f"{vacuum_energy:.10f} and {vacuum_charge:.10f}")
check_record(abs(vacuum_energy + 16 * E) < 1e-9 and abs(vacuum_charge - 16) < 1e-9,
             "each universe has the sea energy -8E = -40 (total -80) and charge 8",
             record="Revision/theory/reports/python-field-theory.json, check "
                    "good_sector_positive_fock_realisation")
```

The vacuum of both universes has the energy $-80 = 2 \times (-40)$ and the charge $16 = 2 \times 8$ before normal ordering; the check reproduces the record check `good_sector_positive_fock_realisation` for each universe. Three PASS lines in the cell.

In [12], the states of the two universes.

```
def quanta_and_charge(n):
    groups = [bin((n >> shift) & 0xFF).count("1") for shift in (0, 8, 16, 24)]
    b_1, d_1, b_2, d_2 = groups  # particles and antiparticles of each universe
    return b_1 + d_1 + b_2 + d_2, b_1 - d_1 + b_2 - d_2
```

The 32 binary digits of a pattern fall into four groups of 8: the particles and the antiparticles of universe 1, then those of universe 2. For each group the pattern is shifted to the right by 0, 8, 16 or 24 places, the eight lowest digits are kept (`& 0xFF`) and their ones are counted. The function returns the number of quanta and the total charge (particles minus antiparticles).

```
eigen_ok = True
for n in rng.integers(0, 2 ** 32, size=200):
    n = int(n)
    quanta, charge = quanta_and_charge(n)
    state = {n: 1.0}
    eigen_ok = eigen_ok and largest(add(
        (1, total_energy(state)), (-(E * quanta + vacuum_energy), state))) < 1e-9
    eigen_ok = eigen_ok and largest(add(
        (1, total_charge(state)), (-(charge + vacuum_charge), state))) < 1e-9
check(eigen_ok, "200 random patterns: energy E (quanta) - 16E, charge (b - d) + 16")
```

For 200 random patterns the total energy and charge are applied, and the results compared with the pattern times $E \times (\text{quanta}) - 16E$ and times $(\text{charge}) + 16$: every pattern is an eigenstate, with the normal-ordered total energy E times its number of quanta.

```
pair = create(24, create(0, VACUUM))  # b_1^* (mode 0), then d_2^* (mode 24)
pair_energy = inner(pair, total_energy(pair)).real - vacuum_energy
pair_charge = inner(pair, total_charge(pair)).real - vacuum_charge
report("pair state: normal-ordered total energy and total charge",
       f"{pair_energy:.10f} and {pair_charge:.10f}")
```

```
check(abs(pair_energy - 2 * E) < 1e-9 and abs(pair_charge) < 1e-9,
      "a particle in universe 1 and an antiparticle in universe 2: charge 0, "
      "energy 2E")
```

The pair state: a particle in universe 1 (mode 0) and an antiparticle in universe 2 (mode 24, the first antiparticle mode of universe 2). Its normal-ordered total energy is printed as $10.0000000000 = 2E$ and its total charge as 0: the energies of the two universes add.

```
counts = np.zeros((33, 33), dtype=np.int64)  # rows: quanta, columns: charge + 16
for b_1 in range(9):
    for d_1 in range(9):
        for b_2 in range(9):
            for d_2 in range(9):
                number = comb(8, b_1) * comb(8, d_1) * comb(8, b_2) * comb(8, d_2)
                counts[b_1 + d_1 + b_2 + d_2, b_1 - d_1 + b_2 - d_2 + 16] += number
```

All $2^{32} = 4294967296$ patterns are counted by formula, not one by one: for given numbers b_1, d_1, b_2, d_2 of occupied modes in the four groups there are $\binom{8}{b_1}\binom{8}{d_1}\binom{8}{b_2}\binom{8}{d_2}$ patterns, added in the square (number of quanta, charge +16). `np.int64` holds whole numbers up to about 9×10^{18}.

```
zero_energy_states = int(counts[0].sum())
zero_charge_lowest = min(q for q in range(33) if q > 0 and counts[q, 16] > 0)
report("number of patterns", int(counts.sum()))
report("patterns of zero normal-ordered energy", zero_energy_states)
report("fewest quanta of a non-vacuum state of total charge 0", zero_charge_lowest)
check(counts.sum() == 2 ** 32 and zero_energy_states == 1 and zero_charge_lowest == 2,
      "only the vacuum has energy 0; charge 0 needs at least 2 quanta (energy 2E)")
```

The printed numbers: 4294967296 patterns, 1 of zero energy (the vacuum), and 2 as the smallest positive number of quanta of a state of total charge 0 (column 16): a state of charge 0 other than the vacuum has at least the energy $2E$. Three PASS lines in the cell.

```
fig, ax = plt.subplots(figsize=(7.5, 5.6))
shown = np.where(counts > 0, np.log10(np.maximum(counts, 1)), np.nan)  # NaN: empty
light_to_dark = LinearSegmentedColormap.from_list(
    "light_to_dark_blue", ["#cde2fb", "#2a78d6", "#0d366b"])
image = ax.imshow(shown, origin="lower", cmap=light_to_dark, aspect="auto",
                  extent=(-16.5, 16.5, -0.5, 32.5))
ax.annotate("vacuum: the only state of energy 0", (0, 0), xytext=(2.0, 1.5),
            fontsize=8, arrowprops={"arrowstyle": "->", "color": GREY})
ax.annotate("pair of total charge 0: energy $2E$", (0, 2), xytext=(3.0, 5.5),
            fontsize=8, arrowprops={"arrowstyle": "->", "color": GREY})
ax.set_xlabel("total charge (particles minus antiparticles, both universes)")
ax.set_ylabel("number of quanta (total energy $= 5 \\times$ quanta)")
ax.set_title("Two universes, one momentum: $2^{32}$ states")
ax.grid(False)
fig.colorbar(image, ax=ax, label="$\\log_{10}$ of the number of states")
save_figure(fig, "two_universe_states",
...)
```

The heat map of In [9] of Notebook 10c, now for two universes: the logarithm of the counts, white for empty squares, with arrows at the vacuum and at the pairs of total charge 0. Figure 5 of Notebook 10g shows a diamond of states from charge -16 to 16 and up to 32 quanta, with a single state at energy 0 and nothing below it: two independently quantised universes of masses $+m$ and $-m$ have energies that add; nothing cancels.

In [13], the last check.

```
for name in ("10g_1_krein_metrics.png", "10g_2_one_system.png",
             "10g_3_anticommutator_blocks.png", "10g_4_block_generator.png",
             "10g_5_two_universe_states.png"):
    check(output_file(f"{FIGURE_FOLDER}/{name}").is_file(),
          f"the figure file {name} exists")
all_checks_passed()
```

The five figure files exist, and the last line prints ALL 30 CHECKS PASSED (notebook 10g): the five figure checks and the 25 checks of the cells before (3 in In [2], 3 each in In [3], In [4], In [8], In [11] and In [12], 1 each in In [5], In [7] and In [9], and 2 each in In [6] and In [10]).

10.50 What we proved, what we computed, what we assumed

Proved. Exactly, for all values of the letters involved, each with the Revision record that verifies it and the notebook that reproduces it:

statement	where it is verified	notebook
$B = -iC\gamma^{(x_4)}$ is purely imaginary and Hermitian, $BB = I_{16}$, $\mathrm{tr}\, B = 0$, signature (8,8); it commutes with the gammas of x_1, x_2, x_3, x_4, x_0 and anticommutes with those of x_5, x_6, x_7	`python-field-theory.json`, check `B_properties`; `wolfram-algebra.json`, check `B_signature_8_8`; `python-algebra.json`, check `B_gamma_relations`	10a, 10e
the local conservation law $\partial_\mu(\cos z\, J^\mu) = 0$ on shell; the charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, d^7x$ is an indefinite (Krein) form (it is constant in time only when no charge flows through the boundary of the slice, which at $z = \pi/2$ is an ASSUMED no-flux condition that the record does not impose)	`charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity`; `wolfram-field-theory.json`, check `charge_density_is_Krein_form_G`	none

statement	where it is verified	notebook
one plane wave: $hh = w^2 I_{16}$, $Bh = h^\dagger B$ (the Krein form is conserved, also by growing waves); h Hermitian and $[B, h] = 0$ exactly in the good sector	`wolfram-pairing.json`, check `Q_one_particle_flat_dispersion`; `python-field-theory.json`, checks `mode_hamiltonian_B_selfadjoint_dispersion` and `good_sector_spectrum_and_B_sectors`; `wolfram-field-theory.json`, checks `mode_hamiltonian_good_sector` and `mode_hamiltonian_Krein_selfadjoint`	10a
waves with $w^2 < 0$ grow, with rates that have no upper bound	`python-field-theory.json`, check `extra_time_modes_grow`; `python-scope.json`, check `extra_time_growth_rates_unbounded`	10a, 10f
Theorem 10.1: Krein inertia (4,4) of every real-frequency eigenspace; Krein-neutral eigenspaces at imaginary and zero frequency	`wolfram-pairing.json`, checks `Q_one_particle_Krein_inertia_real_frequencies` and `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`; `python-pairing.json`, checks `Q.one_particle_Krein_inertia_proof` and `Q.one_particle_complex_frequency_Krein_neutral`	10a, 10f
$\Gamma h_m \Gamma = h_{-m}$ and $\gamma^{(x_0)} h_m(k) \gamma^{(x_0)} = h_{-m}(R_0 k)$: identical one-particle spectra for $+m$ and $-m$	`python-pairing.json` and `wolfram-pairing.json`, check `Q.one_particle_maps` / `Q_one_particle_maps`	10a, 10f
the canonical rule $\{\Psi_A(x), \Psi^\dagger_C(y)\} = B_{AC}\, \delta^7(x-y)/\cos z$; the Heisenberg equation is the field equation	`python-field-theory.json`, check `canonical_anticommutator_B`; `wolfram-field-theory.json`, checks `first_order_form_and_anticommutator` and `Heisenberg_equation_reproduces_field_equation`	10b

statement	where it is verified	notebook
Theorem 10.2: no positive inner product with $\Psi^\dagger$ the Hilbert adjoint; the state space is a Krein space with fundamental symmetry B	`python-field-theory.json` and `wolfram-field-theory.json`, check `no_positive_inner_product`	10b
the expectation-value rule $\epsilon_n u_n^\dagger M u_n$ (Krein-Fock), $u^\dagger B M u$ and $-v^\dagger B M v$ (positive realisation)	`python-field-theory.json`, checks `expectation_value_rule` and `good_sector_positive_fock_realisation`	10b, 10c
the 17 signs of $MBM^\dagger = \sigma B$; the conjugation $\Psi \to \Gamma \Psi^{\dagger T}$ keeps B and reverses the mass, $M = I_{16}$ gives $-B$	`pairing-theory.json`, data `Krein_signs_M_B_Mdagger`; `charge-conjugation-and-u1.json`, check `quantum_charge_conjugation_unitary_type`	10b, 10g
good sector, one momentum with frozen coefficients: positive Fock space with $\Psi^\dagger = \chi B$, vacuum $-8E$, quanta $+E$, charges ± 1	`python-field-theory.json`, check `good_sector_positive_fock_realisation`; `wolfram-field-theory.json`, check `Fock_space_good_sector_example`	10c, 10f, 10g
the classical commuting field dirac16complex00 has an energy unbounded below and an indefinite charge	`python-scope.json`, check `commuting_field_energy_unbounded_below`	10c
$\lambda \neq 0$, in the finite Fock model of one good-sector mode set with frozen coefficients (16 modes; symbolic λ; at rest with symbolic m, and $m = 3$, $k = (4,0,0,0)$): $:\sum_\mu K_\mu: = :(m + U'(S))S:$ and, at rest, $\hat{\rho} = :mS + U:$, $\hat{p} = :SU' - U:$ are exact operator identities for solutions of the interacting operator field equation if and only if U and $\hat{T}$ are Wick ordered	`fock-quartic.json`, checks `trace_identity_operator_identity_wick`, `trace_identity_fails_for_every_other_combination` and `homogeneous_rho_p_operator_identities_wick`	none

statement	where it is verified	notebook
curved good sector: symmetric, and the Krein form conserved, only up to the boundary term $\partial_0(\sin z\, u^\dagger M_0 v)$; flux $2i$; role of $3H\gamma^{(x_0)}$; x_0-independent waves grow for $m^2 < 9H^2$; the exact solution	`python-scope.json`, checks `good_sector_hermiticity_up_to_the_brane_flux` and `good_sector_x0_independent_modes_without_boundary_condition`; `wolfram-field-theory.json`, checks `good_sector_hermiticity_curved` and `Krein_form_conserved_curved`; `python-field-theory.json`, check `exact_solution_family_x4_x0`	10d
the invariant forms are $r_- CP_- + r_+ CP_+$; every invariant charge density is indefinite; exactly the 21 generators without x_4 keep B (Spin(4,3))	`python-algebra.json`, checks `spin_commutant_dimension_2` and `S_preserves_C_and_commutes_with_Gamma`; `wolfram-algebra.json`, checks `S_preserves_C` and `S_preserves_B_only_off_x4`	10e
along the deflating history (local-frame model): $h(x_4)^2 = w(x_4)^2 I_{16}$, $\|h_A\| = k_5$, $\|Bh - hB\| = 2k_5$; every wave with extra-time momentum reaches the onset time $\ln X^*/(2AH)$; the good sector stays Hermitian	this chapter, Section 10.39 (the book's own derivation, exact in sympy)	10f

statement	where it is verified	notebook
the quantum reading Q1 to Q5 of the pairing	`wolfram-pairing.json`, checks `Q_Krein_metric_of_images`, `Q_symplectic_kernel_grassmann`, `Q_generators_of_the_image_grassmann`, `Q_no_identification_of_independent_universes`; `python-pairing.json`, checks `Q.image_krein_metric`, `Q.image_own_quantisation`, `Q.image_generators_same_dynamics`, `Q.no_cancellation_independent_universes`, `Q.T2_image_keeps_B`, `compare.theorem_Q`	10g

Computed. Numerical results of the notebooks, with their accuracy:

- Notebook 10a: the 16 eigenvalues of h agree with $\pm w$ to 3.6×10^{-15}; in the scan across the threshold the inertia is $(4, 4, 0)$ at all 110 real-frequency points and $(0, 0, 8)$ at all 137 imaginary-frequency points, with the smallest nonzero Gram eigenvalue 0.2225 below the threshold.
- Notebook 10b: the canonical rule holds on the Fock space exactly (0.0) and the Heisenberg equation to 1.6×10^{-17}; the Krein norms of 5000 random unit columns lie between -0.777 and 0.704; for the boosted mode the rule $\epsilon u^\dagger M u$ holds to 1.3×10^{-15}, while $u^\dagger B M u$ misses by up to 5.82.
- Notebook 10c: vacuum -40, every quantum $+5$, charges ± 1 (to 10^{-12}), for $m = 2$, $k = (k_0, k_1, k_2, k_3) = (4, 1, 2, 0)$ and for $m = 3$, $k_1 = 4$; the 65536 states of one momentum, one of them of energy 0; the commuting field: densities ± 5, and from -4.364 to 4.333 for 4000 random waves.
- Notebook 10d: the Krein charge of the exact growing solution rises from 1/6 to 454.007 between $x_4 = 0$ and 1.5 and equals the flux added up over time to a relative 6.7×10^{-7}; the late growth slope 5.6600 approaches $2k = 5.6569$.
- Notebook 10e: exactly two zero eigenvalues of $M^T M$ (at most 1.3×10^{-15}) below the smallest nonzero one, 7.0.
- Notebook 10f: the onset time 2.996044 of the wave $q_1 = 0.5$, $q_5 = 0.05$ (agreement with bisection 8.9×10^{-16} for 48 waves); onset times from 6.9078 to 0.0941 for $10^{-3} \le q_5 \le 1$; the good-sector energies at the five slices (for $q_1 = 1$ from 1.414214 down to 1.009116).

- Notebook 10g: the pair of total charge 0 in two independent universes has the normal-ordered energy $10 = 2E$; of the $2^{32} = 4294967296$ states of one momentum in both universes exactly one (the vacuum) has energy 0.

Assumed.

- Anticommutators (not commutators) for dirac16complex: part of its definition as a fermion field; no spin-statistics theorem for signature (4,4) is proved.
- Normal ordering: a prescription that drops the energy $-8E$ of the filled sea per momentum.
- Flat 4+4 space, or the frozen-coefficient model of Section 10.3 (the coefficients of one point and one instant, without the two hidden-direction terms of the author's field equation, among them the spin-connection term $3H\gamma^{(x_0)}$; keeping that term breaks $Bh = h^\dagger B$), for the one-particle statements and the Fock spaces; the good sector itself is a restriction imposed by hand (no mechanism that removes the extra-time waves is derived).
- The deflating history $a_4 = AHx_4$ ($A = H = m = 1$) is a PRESCRIBED BACKGROUND, and the local-frame model of Section 10.39 evaluates the coefficients at one instant and one hidden position (`parameters.json`; `ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`).
- The Z2 brane condition at $z = \pi/2$ is ASSUMED by the Kohn-Sham record (Chapter 14); the quantisation of this chapter imposes no boundary condition there. The total charge Q is constant in time only under such an ASSUMED no-flux condition at $z = \pi/2$; what is proved is its local conservation law (Section 10.2).

Hypothesis. That the big bang creates universes in pairs of masses $+m$ and $-m$ is the author's HYPOTHESIS; no equation of this chapter describes a creation, in pairs or otherwise (Section 10.45).

Open.

- A positive Hilbert space for the whole quantised field (all momenta, the curved metric), and the quantisation of the extra-time sector, whose waves grow and become Krein-neutral along the deflating history.
- A boundary condition at the patch end $z = \pi/2$ that makes the curved good-sector evolution Hermitian and conserves the Krein charge.
- The interacting theory $\lambda \neq 0$, in particular the operator form of the on-shell identity of the energy-momentum tensor beyond the finite model of one good-sector mode set with frozen coefficients (where it holds with Wick ordering only, Section 10.21): for the field on a whole slice, with the curved x_0 dependence and in the extra-time sector; symmetry and conservation of the quartic operator are not verified.
- The evolution of the quantum state through the changing background (whether the instantaneous vacuum of one time contains quanta of a later time).
- Any creation process, rate or amplitude for universes, which these equations do not contain.

10.51 Exercises

Exercise 10.1. Use the table of B in Section 10.2. (a) Compute Be_1 and Be_{10}, where e_j is the unit column with 1 in place j. (b) Find the two columns $u = ae_1 + be_{10}$ of length 1 that are eigenvectors of B, with their eigenvalues. (c) Compute the Krein norms of e_1 and of the two eigenvectors.

Exercise 10.2. For each wave compute w^2, say whether it oscillates, grows linearly or grows exponentially, and give the growth rate where there is one: (a) $m = 2$, $k_1 = 1$, $k_6 = 2$; (b) $m = 1$, $k_0 = 1$, $k_5 = 1$, $k_7 = 1$; (c) $m = 1$, $k_5 = 3$. In case (a), what is the Krein inertia of the eigenspaces?

Exercise 10.3. A two-dimensional model of a Krein space: $B = \mathrm{diag}(1, -1)$ and $h = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$. (a) Show $h^\dagger B = Bh$ and $hh = -I_2$. (b) Solve $i\,du/dt = hu$ with $u(0) = (1, 0)$. (c) Compute $u^\dagger u$ and $u^\dagger Bu$ as functions of t.

Exercise 10.4. A toy Dirac sea: one positive level (operator b, energy E) and one negative level (operator d^*, energy $-E$), with $\{b, b^*\} = \{d, d^*\} = 1$ and all other anticommutators 0, have $H = E\,b^*b - E\,dd^*$ and $Q = b^*b + dd^*$. (a) Find the vacuum energy and charge before normal ordering. (b) Write $:H:$ and $:Q:$. (c) Give the normal-ordered energy and charge of $b^*|0\rangle$, $d^*|0\rangle$ and $b^*d^*|0\rangle$.

Exercise 10.5. For one good-sector momentum (8 particle and 8 antiparticle modes, energy E per quantum), count the states (a) with 2 quanta, (b) with 2 quanta and charge 0, (c) with 8 quanta and charge 0, (d) in all.

Exercise 10.6. Using only $(\gamma^{(x_a)})^\dagger = \eta_{aa}\gamma^{(x_a)}$, $\gamma^{(x_a)}\gamma^{(x_a)} = \eta_{aa} I_{16}$, the commutation of B with the gammas (Section 10.2) and $\Gamma B \Gamma = -B$, compute the sign σ in $MBM^\dagger = \sigma B$ for $M = \gamma^{(x_5)}$, $M = \gamma^{(x_4)}$ and $M = P_5 = \Gamma\gamma^{(x_5)}$.

Exercise 10.7. (a) Show that $S^{(x_1x_5)} = \frac{1}{2}\gamma^{(x_1)}\gamma^{(x_5)}$ is Hermitian and anticommutes with B, so that it is Krein-unitary but not unitary. (b) Show that $S^{(x_4x_1)} = \frac{1}{2}\gamma^{(x_4)}\gamma^{(x_1)}$ is Hermitian and commutes with B, and compute its Krein defect $S^\dagger B + BS$.

Exercise 10.8. On the deflating history with $A = H = m = 1$ and $y = 0$: (a) find the onset time of the waves with $q_1 = 0$ and $q_5 = 0.05$, and $q_5 = 0.001$; (b) at what time does the wave $q_1 = 0$, $q_5 = 0.05$ reach the frame momentum $k_5 = 2$? (c) Compute X^* and the onset time for $q_1 = 0.5$, $q_5 = 0.05$, and explain why it is later than in (a).

Exercise 10.9. For the good-sector waves that do not depend on x_0 (Section 10.28), with $H = 1$: find the eigenvalues of A and the growth rate for $m = 2$ and for $m = 4$.

10.52 Answers to the exercises

Answer 10.1. (a) $(Be_1)_r = B_{r1}$, the entries of column 1: by the table only row 10 has its nonzero entry in column 1, with the value $-i$, so $Be_1 = -ie_{10}$. Row 1 has its entry $+i$ in column 10, so $Be_{10} = ie_1$. (b) For $u = ae_1 + be_{10}$, $Bu = aBe_1 + bBe_{10} = ib\,e_1 - ia\,e_{10}$, by (a). $Bu = \lambda u$ means $ib = \lambda a$ and $-ia = \lambda b$. The first gives $b = -i\lambda a$ (multiply by $-i$); insert it into the second: $-ia = \lambda(-i\lambda a) = -i\lambda^2 a$, so $\lambda^2 = 1$. For $\lambda = +1$: $b = -ia$, and with length 1, $u_+ = (e_1 - ie_{10})/\sqrt{2}$. For $\lambda = -1$: $b = ia$, $u_- = (e_1 + ie_{10})/\sqrt{2}$. Check: $Bu_- = (i \cdot i\,e_1 - ie_{10})/\sqrt{2} = (-e_1 - ie_{10})/\sqrt{2} = -u_-$. (c) $e_1^\dagger Be_1 = B_{11} = 0$: the unit column e_1 has length 1 but Krein norm 0. $u_\pm^\dagger Bu_\pm = \pm u_\pm^\dagger u_\pm = \pm 1$.

Answer 10.2. $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$ (Section 10.4). (a) $w^2 = 4+1-4 = 1$: the frequencies ± 1 are real and the wave oscillates, although h is not Hermitian ($k_6 \neq 0$). By Theorem 10.1 both eigenspaces have the Krein inertia $(4,4)$. (b) $w^2 = 1+1-1-1 = 0$ with $h \neq 0$: $hh = 0$, $e^{-ihx_4} = I_{16} - ihx_4$, and the wave grows linearly unless $hu(0) = 0$; the range of h is Krein-neutral. (c) $w^2 = 1 - 9 = -8$: $w = \pm i\sqrt{8} = \pm 2\sqrt{2}\, i$, half of the waves grow like $e^{2\sqrt{2}\, x_4}$ (rate $2\sqrt{2} \approx 2.828$), and both eigenspaces are Krein-neutral.

Answer 10.3. (a) $h^\dagger = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (transpose; the entries are real). $h^\dagger B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (the second column of $h^\dagger$ is multiplied by -1) and $Bh = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (the second row of h is multiplied by -1): equal. $hh = \begin{pmatrix} 0\cdot 0 + 1\cdot(-1) & 0\cdot 1 + 1\cdot 0 \\ (-1)\cdot 0 + 0\cdot(-1) & (-1)\cdot 1 + 0\cdot 0 \end{pmatrix} = -I_2$, so $w^2 = -1$. (b) As in Section 10.4, $e^{-iht} = \cos(wt) I_2 - i\frac{\sin(wt)}{w} h$ with $w = i$; $\cos(it) = \cosh t$ and $\sin(it)/i = \sinh t$, so $e^{-iht} = \cosh t\, I_2 - i \sinh t\, h$. With $hu(0) = (0, -1)$ (the first column of h): $u(t) = (\cosh t, i \sinh t)$. Check: $i\, du/dt = i(\sinh t, i\cosh t) = (i\sinh t, -\cosh t)$ and $hu = (i\sinh t, -\cosh t)$. (c) $u^\dagger u = \cosh^2 t + \sinh^2 t = \cosh 2t$, which grows; $u^\dagger B u = \cosh^2 t - \sinh^2 t = 1$, constant: the Krein norm is conserved by a growing wave, as Section 10.4 proved for dirac16complex.

Answer 10.4. (a) The vacuum has $b|0\rangle = d|0\rangle = 0$. With $dd^* = 1 - d^*d$: $H = E\, b^*b + E\, d^*d - E$ and $Q = b^*b - d^*d + 1$, so the vacuum has the energy $-E$ (the filled negative level) and the charge 1. (b) $:H: = E(b^*b + d^*d)$ and $:Q: = b^*b - d^*d$. (c) $b^*|0\rangle$: energy E, charge $+1$; $d^*|0\rangle$: energy E, charge -1; $b^*d^*|0\rangle$: energy $2E$, charge 0. A state of total charge 0 other than the vacuum has a positive energy, as for the two universes of Section 10.44.

Answer 10.5. A state is fixed by the set of occupied modes; with N_b particles and N_d antiparticles it has $N_b + N_d$ quanta and the charge $N_b - N_d$ (Section 10.20). (a) Any 2 of the 16 modes: $\binom{16}{2} = \frac{16\cdot 15}{2} = 120$; split as $\binom{8}{2} = 28$ (two particles, charge 2), $8 \cdot 8 = 64$ (one of each, charge 0) and 28 (two antiparticles, charge -2). (b) 64. (c) $N_b = N_d = 4$: $\binom{8}{4}\binom{8}{4} = 70 \cdot 70 = 4900$, the largest group of Figure 3 of Notebook 10c. (d) $2^{16} = 65536$.

Answer 10.6. $M = \gamma^{(x_5)}$: it is time-like, so $(\gamma^{(x_5)})^\dagger = -\gamma^{(x_5)}$ and $\gamma^{(x_5)}\gamma^{(x_5)} = -I_{16}$, and it anticommutes with B: $\gamma^{(x_5)} B (\gamma^{(x_5)})^\dagger = -\gamma^{(x_5)} B \gamma^{(x_5)} = B\gamma^{(x_5)}\gamma^{(x_5)} = -B$, $\sigma = -1$. $M = \gamma^{(x_4)}$: time-like and commuting with B: $\gamma^{(x_4)} B (\gamma^{(x_4)})^\dagger = -\gamma^{(x_4)} B \gamma^{(x_4)} = -B\gamma^{(x_4)}\gamma^{(x_4)} = B$, $\sigma = +1$. $M = P_5$: $P_5 B P_5^\dagger = \Gamma\gamma^{(x_5)} B (\gamma^{(x_5)})^\dagger \Gamma^\dagger = \Gamma(-B)\Gamma = -\Gamma B \Gamma = B$, $\sigma = +1$. These are the signs of the record (Section 10.15).

Answer 10.7. (a) $(\gamma^{(x_1)}\gamma^{(x_5)})^\dagger = (\gamma^{(x_5)})^\dagger (\gamma^{(x_1)})^\dagger = (-\gamma^{(x_5)})\gamma^{(x_1)} = \gamma^{(x_1)}\gamma^{(x_5)}$, so $S = S^{(x_1 x_5)}$ is Hermitian, $S^\dagger = S$ (and not anti-Hermitian, so $\exp(\theta S)$ is not unitary). $\gamma^{(x_1)}$ commutes and $\gamma^{(x_5)}$ anticommutes with B, so $SB = -BS$. The Krein condition: $S^\dagger B + BS = SB + BS = -BS + BS = 0$. (b) $(\gamma^{(x_4)}\gamma^{(x_1)})^\dagger = \gamma^{(x_1)}(-\gamma^{(x_4)}) = \gamma^{(x_4)}\gamma^{(x_1)}$: Hermitian; both gammas commute with B, so $SB = BS$. Then $S^\dagger B + BS = 2BS = -iC\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(x_1)} = iC\gamma^{(x_1)} \neq 0$, the defect of the record check `S_preserves_B_only_off_x4`.

Answer 10.8. (a) For $q_1 = 0$, $y = 0$: $x_4^* = \ln(m/q_5)/(AH) = \ln(1/q_5)$; $\ln 20 = 2.995732$ for $q_5 = 0.05$, and $\ln 1000 = 6.907755$ for $q_5 = 0.001$. (b) $k_5 = 0.05\, e^{x_4} = 2$ gives $e^{x_4} = 40$, $x_4 = \ln 40 = 3.688879$, where the eigenvalues are $\pm i\sqrt{3}$ (Notebook 10f, In [7]). (c) With $c = 1$: $X^* = (1 + \sqrt{1 + 4 \cdot 0.25 \cdot 0.0025})/(2 \cdot 0.0025) = (1 + \sqrt{1.0025})/0.005 = 400.249844$, and $x_4^* = \frac{1}{2}\ln 400.249844 = 2.996044$, the value of Notebook 10f, In [6]. It is later than $\ln 20$ because the

3-space momentum adds $k_1^2 > 0$ to w^2, so the growing k_5^2 needs a little longer to overcome $m^2 + k_1^2$.

Answer 10.9. $AA = (m^2 - 9H^2)I_{16}$ and $\operatorname{tr} A = 0$ (Section 10.28). For $m = 2$: $AA = (4-9)I_{16} = -5I_{16}$, eigenvalues $\pm i\sqrt{5}$ (eight each), growth rate $\sqrt{5} \approx 2.236$: these finite-norm waves grow. For $m = 4$: $AA = 7I_{16}$, eigenvalues $\pm\sqrt{7} \approx \pm 2.646$, real: the waves oscillate (Figure 4 of Notebook 10d: zero growth rate for $m \geq 3H$).

11 GKD and the Lovelock tensors

Einstein's equations of gravity set one curvature tensor, the Einstein tensor, equal to the energy and momentum of matter. In eight dimensions there is room for more: two further tensors, built from products of two and of three curvature tensors, have the same good properties as Einstein's. Together with Einstein's they are the three **Lovelock tensors** of the formula (4.38) that the author quotes in his notebook. Every term of these tensors carries a weight $+1$, -1 or 0 given by the **generalized Kronecker delta**, which the author defined as a determinant and which the Revision program computes with a fast rule called **GKD**. This chapter proves that the rule is right, counts and tests it, builds the three Lovelock tensors of the author's metric twice (with the Revision Rust program and with independent Python code), proves their classical identities, and studies their components along the history in which the three extra times deflate exponentially. Chapter 12 then turns these components into the field equations for a_4.

11.1 What this chapter does, and why

In Chapter 3 we computed the curvature of the author's metric: its Christoffel symbols, its Riemann tensor $R^{ab}{}_{cd}$, its Ricci tensor and its Einstein tensor $G^h{}_j$. Einstein's field equations, $G^\mu{}_\nu + \Lambda\delta^\mu{}_\nu = \kappa T^\mu{}_\nu$, use only the Einstein tensor. Why that tensor? Because its **divergence** vanishes for every metric (the contracted Bianchi identity, Section 3.25), as the conservation of energy and momentum demands of the right-hand side. In 1971 David Lovelock found every tensor with this property that is built from the metric and its first and second derivatives (D. Lovelock, J. Math. Phys. 12, 498 (1971); we quote his theorem and do not prove it). In four dimensions only the Einstein tensor (and the constant term Λ) exists. In eight dimensions there are exactly three: the Einstein tensor and two companions. The author's notebook quotes them in the form of equation (4.38) of the book of Lovelock and Rund, and the Revision computed them for the author's metric (folder `Revision/gkd_lovelock`).

The chapter has three parts, each with one notebook.

- **The generalized Kronecker delta** (Sections 11.3 to 11.9). We restate the author's definition, a determinant of a matrix of zeros and ones, prove that it equals the sign of a permutation or zero, and show that this rule (GKD) is exactly what the Revision's Rust program computes. We count how often each value occurs, prove that a delta with nine labels in eight dimensions is always zero, compare the cost of the rule with the cost of the determinant, and connect the delta with the Levi-Civita tensors of the curved metric (the author's second route). Notebook 11a checks all of this in Python and runs the Rust self-test.
- **The three Lovelock tensors** (Sections 11.10 to 11.19). We recall the curvature of the author's metric, define the Lovelock tensors $P_{(1)}, P_{(2)}, P_{(3)}$ and the Lovelock scalars $L_{(1)}, L_{(2)}, L_{(3)}$,

explain how the enormous sums are organised so that a computer can do them exactly, and prove by hand, line by line, that $P_{(1)} = -4G$, that $P_{(2)} = -8$ times the Gauss-Bonnet tensor, and the trace identity. Notebook 11b runs the Rust program `lovelock_gkd`, checks that it reproduces the Revision records byte for byte, and computes everything a second time in Python.

- **The components along the deflating history** (Sections 11.20 to 11.27). We write out the components of the normalised tensors $E_{(k)} = -P_{(k)}/2^{k+1}$, prove their structure (which components are equal, which contain a_4'', which do not care about the sign of a_4'), derive the two conservation identities that they obey, and draw them along the exponentially deflating history $a_4 = AHx_4$ and along an illustrative test history. Notebook 11c does this with exact algebra.

Everything in this chapter is geometry of the given metric: no physical assumption about matter enters, and nothing here concerns pairs of universes or matter and antimatter. Every formula is PROVED (by a derivation written out here, or by exact computation in the Revision records and the notebooks) or COMPUTED (a numerical test), and each statement names the record file and check that confirm it.

11.2 The words and symbols of this chapter

Each word is defined in plain terms here; the later sections make the definitions precise.

- **Coordinates** $x_0, \ldots, x_7$: the author's names of the eight directions. x_1, x_2, x_3: ordinary 3-space (they inflate). x_4: the time. x_5, x_6, x_7: the three **extra times**, time-like directions that **deflate exponentially** when a_4 grows. x_0: the hidden space direction, through the angle $z = 6Hx_0$ with $0 < z < \pi/2$. Programs number the coordinates $0, 1, \ldots, 7$, so the number 3 stands for x_4 and the number 7 for x_0.

- **Label**: the name of a coordinate. **Index list**: an ordered list of labels, such as (x_2, x_1, x_4); its **length** p is the number of entries. A **pair** of index lists is a **lower** list $(l_1, \ldots, l_p)$ and an **upper** list $(u_1, \ldots, u_p)$ of the same length.

- **Kronecker delta** $\delta(a, b) = \delta^b_a$: the number 1 if the labels a and b are equal, 0 otherwise.

- **Permutation**, **inversion**, **sign**: a permutation is a re-ordering of p objects, written as the list $(\sigma(1), \ldots, \sigma(p))$; an inversion is a pair of places $i < j$ with $\sigma(i) > \sigma(j)$; the sign is $(-1)^{\text{number of inversions}}$, $+1$ for an **even** and -1 for an **odd** permutation (Section 1.19).

- **Determinant** (Leibniz formula): $\det M = \sum_\pi \operatorname{sign}(\pi)\, M_{1\pi(1)} \cdots M_{p\pi(p)}$, a sum of $p!$ products, one for every permutation π (Section 1.20). **Minor** $M^{(rc)}$: the matrix left when row r and column c of M are removed.

- **Generalized Kronecker delta**: $\delta^{u_1 \ldots u_p}_{l_1 \ldots l_p} = \det[\delta(l_i, u_j)]$, the author's `kδ[lower, upper]`. **GKD**: the Rust function of the Revision program that computes it by the rule proved in Section 11.3.

- **Pigeonhole principle**: if more than n objects are put into n boxes, some box holds two of them.

- **Levi-Civita symbol** $[i_1 \dots i_n]$: the sign of the list $(i_1, \dots, i_n)$ if it is a re-ordering of all n labels, and 0 otherwise (Section 1.44).
- **Metric** g_{ab}, **inverse metric** g^{ab}: the author's metric is diagonal, so $g^{aa} = 1/g_{aa}$ and all other entries are zero.
- **Christoffel symbol** $\Gamma^a{}_{bc}$, **Riemann tensor** $R^a{}_{bcd}$ and $R^{ab}{}_{cd} = g^{bb} R^a{}_{bcd}$, **plane curvature** $K(a,b) = R^{ab}{}_{ab}$ (no sum), **Ricci tensor** $R^h{}_j = \sum_c R^{hc}{}_{jc}$, **Ricci scalar** $R = \sum_h R^h{}_h$, **Einstein tensor** $G^h{}_j = R^h{}_j - \frac{1}{2}\delta^h_j R$: Chapter 3 (Sections 3.15 and 3.17) defines them.
- **Summed label**: in this chapter a label that appears once up and once down inside one term is summed over its eight values, unless "no sum" is said; the free labels h and j are never summed.
- **Lovelock tensor** $P_{(k)}{}^h{}_j$ of **order** k and **Lovelock scalar** $L_{(k)}$: the sums of Section 11.11. **Normalised Lovelock tensor** $E_{(k)} = -P_{(k)}/2^{k+1}$; $E_{(1)}$ is the Einstein tensor. **Gauss-Bonnet tensor**: the classical name of the order-2 Lovelock tensor. **Density** $A_{(k)}{}^{lh} = \sqrt{|\det g|}\, g^{ll} P_{(k)}{}^h{}_l$: the form in which formula (4.38) is written.
- **Euler density**: in a space of even dimension $n = 2m$, the Lovelock scalar of order m (in eight dimensions $L_{(4)}$). For a closed space (finite, without an edge) with a positive-definite metric its integral is a constant times the **Euler characteristic**, a whole number fixed by the shape of the space that does not change when the metric is deformed (Section 11.11; a classical theorem, quoted: ASSUMED).
- **Component**: one number of a tensor, such as $E^{x_1}{}_{x_1}$; a **diagonal** component has $h = j$.
- **Leaf**, **pruning** (skipping): a complete combination of k curvature entries in the computer's search; pruning means leaving out combinations known to give zero.
- **Divergence** $\nabla_h P^h{}_j$: the curved-space form of "change plus outflow"; zero divergence means **conservation**.
- **History** $a_4(x_4)$: the metric function. **Linear history**: $a_4 = AHx_4$ with a constant **slope** A, so $a_4' = AH$ and $a_4'' = 0$; for $A > 0$ the extra times shrink like e^{-AHx_4}. **Illustration**: a choice made only to show how a formula behaves; it is not a solution of any equation.
- **Exact** computation: with whole numbers and fractions, no rounding. **Byte for byte**: two files equal in every byte.
- **Status labels**: PROVED (exact, by a derivation or by exact computation, with the record file and check), COMPUTED (numerical, with its accuracy), ASSUMED, HYPOTHESIS, OPEN.
- **The record files of this chapter.** Where a record file is named without its folder, it lies in the folder `Revision/gkd_lovelock/results`: `lovelock-report.json` (the report of the Rust program, with its 19 checks and its counters), `curvature.json` (the metric, the Christoffel symbols, the Riemann, Ricci and Einstein tensors), `lovelock-tensors.json` (every component of the Lovelock tensors and the scalars), `gkd-selftest.json` (the GKD self-test), `python-lovelock-report.json` (the Revision's independent sympy verification) and `wolfram-gkd-report.json` (the Revision's independent Wolfram verification). The record of the field equations of a_4 is `Revision/field_equations_a4/a4-equations.json`, with its sympy report `Revision/field_equations_a4/reports/python-a4-report.json`.

11.3 The author's generalized Kronecker delta and the rule GKD

The definition. The author defined the generalized delta in an input cell of his Mathematica notebook. The Revision record quotes the cell verbatim in `Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md` (the cell stored as In[87], the author's In[54]); in Mathematica's plain-text spelling it reads

```
k\[Delta][lower_, upper_] /; Length[lower] == Length[upper] :=
    Det[Outer[delta, lower, upper]]
```

For a lower list $(l_1, \ldots, l_p)$ and an upper list $(u_1, \ldots, u_p)$ of the same length (the condition after `/;`), `Outer[delta, lower, upper]` is the $p \times p$ matrix M with the entry $M_{ij} = \delta(l_i, u_j)$ in row i and column j: one row for each lower label, one column for each upper label, and a 1 wherever the two labels are equal. `Det` takes its determinant. So

$$\delta^{u_1 \ldots u_p}_{l_1 \ldots l_p} = \det\left[\delta(l_i, u_j)\right]_{i,j=1,\ldots,p}.$$

For two lists of different lengths the definition does not apply, and Mathematica leaves the expression unevaluated (record `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, check `definition_unequal_lengths_stay_unevaluated`). For $p = 1$ the matrix is the single number $\delta(l_1, u_1)$, so the generalized delta of one label is the ordinary Kronecker delta. For $p = 2$:

$$\delta^{u_1 u_2}_{l_1 l_2} = \det \begin{pmatrix} \delta(l_1, u_1) & \delta(l_1, u_2) \\ \delta(l_2, u_1) & \delta(l_2, u_2) \end{pmatrix}$$

(the definition with $p = 2$),

$$= \delta(l_1, u_1)\, \delta(l_2, u_2) - \delta(l_1, u_2)\, \delta(l_2, u_1)$$

(the 2×2 determinant is $ad - bc$, Section 1.20). For example $\delta^{x_1 x_2}_{x_1 x_2} = 1 \cdot 1 - 0 \cdot 0 = 1$ and $\delta^{x_2 x_1}_{x_1 x_2} = 0 \cdot 0 - 1 \cdot 1 = -1$.

Theorem (the rule GKD; PROVED). Let $M_{ij} = \delta(l_i, u_j)$. Then $\det M = 0$ if a label appears twice in the lower list, or twice in the upper list, or if some lower label does not appear in the upper list. Otherwise $\det M = \mathrm{sign}(\sigma)$, where $\sigma(i)$ is the place of l_i in the upper list.

Proof, case by case.

(a) Two equal lower labels, $l_i = l_k$ with $i \neq k$. For every column j,

$$M_{ij} = \delta(l_i, u_j) = \delta(l_k, u_j) = M_{kj}$$

(the definition of M; then $l_i = l_k$; then the definition again). So rows i and k are equal, and a matrix with two equal rows has determinant 0 (rule 4 of Section 1.20: exchanging the two rows changes nothing, yet flips the sign, so $\det M = -\det M$ and $\det M = 0$).

(b) Two equal upper labels, $u_j = u_k$ with $j \neq k$. By the same steps columns j and k are equal, so the transpose M^T has two equal rows and $\det M = \det M^T = 0$ (rules 2 and 4 of Section 1.20).

(c) A lower label l_i that equals no upper label. Then $M_{ij} = 0$ for every j: row i is all zeros. Every product of the Leibniz formula contains exactly one entry of row i, so every product is 0, and $\det M = 0$.

(d) Otherwise: the p lower labels are different, the p upper labels are different, and every lower label appears among the upper labels. Then l_i equals exactly one upper label, the one at the place $\sigma(i)$. Different lower labels sit at different places of the upper list, so σ sends the places $1, \ldots, p$ to the places $1, \ldots, p$ without repetition: σ is a permutation. Row i of M has its only 1 in column $\sigma(i)$, so M is the permutation matrix of σ. In the Leibniz formula the product of a permutation π is $M_{1\pi(1)} \cdots M_{p\pi(p)}$; it is 1 when $\pi(i) = \sigma(i)$ for every i and 0 otherwise (one factor is then an entry off the 1s). Only the term $\pi = \sigma$ survives:

$$\det M = \operatorname{sign}(\sigma) \cdot 1 \cdots 1 = \operatorname{sign}(\sigma).$$

End of proof. □

The same theorem is proved in Section 1.42; it is repeated here because the whole chapter rests on it. It is also written, as the opening comment, in the Rust file `Revision/gkd_lovelock/code/src/gkd.rs`.

The rule as a recipe. The Rust function `GKD(lower, upper)` follows the proof step by step, without building a matrix:

- It walks through the upper list and notes the place of every label. A label seen a second time means two equal columns: it returns 0 (case b).
- It walks through the lower list and looks up the place of every label. A label not found means a row of zeros: it returns 0 (case c). A place found a second time means two equal lower labels: it returns 0 (case a). The places form the list σ.
- It counts the inversions of σ and returns $+1$ for an even count and -1 for an odd count (case d).

Counting the inversions of a list of p entries compares each of the $p(p-1)/2$ pairs of places once. The Leibniz formula instead adds $p!$ products. For $p = 7$, the length that the third Lovelock tensor needs, this is 21 comparisons against 5,040 products.

Examples (PROVED). Take the lower list (x_1, x_2, x_3, x_4).

- Upper list (x_2, x_1, x_4, x_3): the places of x_1, x_2, x_3, x_4 in it are $\sigma = (2, 1, 4, 3)$ (counting from 1). The inversions are the pairs of entries $(2, 1)$ and $(4, 3)$: two, an even number, so the value is $+1$.
- Upper list (x_2, x_3, x_4, x_1): $\sigma = (4, 1, 2, 3)$; the inversions are $(4, 1)$, $(4, 2)$, $(4, 3)$: three, odd, value -1.
- Lower list (x_1, x_2, x_3, x_3) with upper list (x_1, x_2, x_3, x_4): x_3 repeats in the lower list, case (a), value 0.

These are the three matrices of Figure 11a.1 (Notebook 11a, In [3]). The Revision record lists five examples with the labels 1 to 4, lower list first: `kdelta[{1,2},{1,2}] = 1`, `kdelta[{1,2},{2,1}] = -1`, `kdelta[{1,1},{1,1}] = 0`, `kdelta[{1,2,3},{2,3,1}] = 1`, `kdelta[{1,2,3},{1,2,4}] = 0` (`Revision/gkd_lovelock/results/python-lovelock-report.json`, check `gkd_examples`). Any whole numbers can serve as labels, because only the equality of two labels matters. Notebook 11a, In [2] and In [4], reproduces all five with both the determinant and the rule.

11.4 Counting the values; nine labels in eight dimensions; the cost of the determinant

The number of nonzero values (PROVED). Fix the length $p \le 8$. By the theorem, a pair has a nonzero value exactly when the lower list holds p different labels and the upper list holds the same p labels in some order. The lower list can be chosen in

$$8 \cdot 7 \cdots (8-p+1) = \frac{8!}{(8-p)!}$$

ways (8 choices for the first label, 7 for the second, which must differ from the first, and so on; the numbers of choices multiply). For each such lower list the upper list can be any of its $p!$ orders. So the number of pairs with a nonzero value is

$$N_{\neq 0}(p) = \frac{8!}{(8-p)!}\, p!$$

out of $8^p \cdot 8^p = 8^{2p}$ pairs (8 choices for each of the $2p$ entries).

Half are $+1$, half are -1 (PROVED, for $p \ge 2$). Exchange the first two labels of the upper list. In σ this exchanges the values 1 and 2 (the places of these two labels). The two entries of σ that hold the values 1 and 2 change their relative order, so the pair they form becomes an inversion if it was none, and stops being one if it was one. Every other pair of entries is compared as before: a pair that contains neither of the two entries is unchanged, and an entry with a value of 3 or more is larger than both 1 and 2, so its comparison with either of them gives the same answer. Hence the number of inversions changes by exactly one, and the exchange turns every even order into an odd one. Doing it twice gives back the original order, so the exchange pairs the even orders with the odd orders one to one: there are as many of each. For $p = 1$ the only order is the identity, which is even. (Section 1.19 proves the same for every exchange of two entries.)

The table (PROVED by the formula; COMPUTED by counting). With $8!/(8-p)!$ equal to 8, 56, 336, 1680 for $p = 1, 2, 3, 4$:

length p	**all pairs** 8^{2p}	**value** $+1$	**value** -1	**value 0**
1	64	8	0	56
2	4096	56	56	3984
3	262144	1008	1008	260128
4	16777216	20160	20160	16736896

For example $p = 3$: $336 \cdot 3! = 336 \cdot 6 = 2016$ nonzero values, half of them, 1008, of each sign, and $262144 - 2016 = 260128$ zeros. The counts for $p = 1, 2, 3$ are those of the Revision's

Wolfram verification, which called the author's verbatim definition on every one of these 266,304 pairs (`Revision/gkd_lovelock/results/wolfram-gkd-report.json`, field `measurements`, entry `gkdComparison`, and checks `gkd_equals_kdelta_exhaustive_length_1` to `_length_3`); Notebook 11a counts them again (In [6]) and compares them with the formula and with the record (In [7]). The row $p = 4$ comes from the formula; the Rust self-test of the record compared all 16,777,216 pairs of length 4 (`Revision/gkd_lovelock/results/gkd-selftest.json`), and Notebook 11a runs it again (In [14]). Figure 11a.4 draws the table.

Nine labels in eight dimensions (PROVED). A list of 9 labels taken from only 8 must repeat a label, by the pigeonhole principle. So case (a) applies, and every generalized delta with $p \geq 9$ is zero. The Lovelock tensor of order k (Section 11.11) carries a delta of length $p = 2k + 1$: $p = 3, 5, 7$ for $k = 1, 2, 3$, and $p = 9$ for $k = 4$. Therefore the Lovelock tensor of order 4, and every higher one, vanishes identically in eight dimensions, and the Lovelock sum stops at $k = 3$ (record `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `gkd_nine_indices_in_eight_dimensions_vanish`; record `Revision/gkd_lovelock/results/lovelock-report.json`, check `k4_tensor_vanishes`; Notebook 11a, In [10]).

The cost of the determinant (PROVED by counting). The Leibniz formula of a $p \times p$ determinant adds $p!$ products; the rule GKD makes $p(p-1)/2$ comparisons of places. And the number 8^{2p} of all pairs grows so fast that only the lengths up to 4 can be tested exhaustively:

length p	**products** $p!$	**comparisons** $p(p-1)/2$	**all pairs** 8^{2p}
1	1	0	64
2	2	1	4096
3	6	3	2.6×10^5
4	24	6	1.7×10^7
5	120	10	1.1×10^9
6	720	15	6.9×10^{10}
7	5040	21	4.4×10^{12}
8	40320	28	2.8×10^{14}
9	362880	36	1.8×10^{16}

(Notebook 11a, In [12], prints this table and draws it as Figure 11a.5.) The work of one literal determinant grows by the factor p from one length to the next, because $p! = p \cdot (p-1)!$. Notebook 11a also measures both methods on the same 200 pairs of length 7 and checks that the rule is more than ten times faster (In [12]). It does not print the measured times: they differ from computer to computer and from run to run, and the printed output of every notebook of this book must be the same on every run.

Random tests and the birthday problem (PROVED; COMPUTED). For the lengths 5 to 9 the Rust self-test of the record compares GKD with the literal determinant on 200,000 pseudo-random pairs of each length (Section 1.49 explains the generator `xorshift64*` and its seeds). Half of the samples are **rearranged** pairs: p random labels form the upper list and the lower list is the same list in a random order. The other half are two **independent** random lists. A rearranged pair has a nonzero value exactly when its p random labels are all different. Each label is one of 8 with equal chance, so this happens with the probability

$$r_p = \frac{8}{8}\cdot\frac{7}{8}\cdots\frac{8-p+1}{8} = \frac{8!}{(8-p)!\,8^p}$$

(the first label may be anything; the second must avoid the first, 7 of 8 values; the third must avoid the first two, 6 of 8; and so on; independent chances multiply, a rule of probability ASSUMED here, Section 1.50). This is the **birthday problem** with 8 possible birthdays. An independent pair is nonzero with the small probability $q_p = N_{\neq 0}(p)/8^{2p}$. With N samples of each kind the expected number of nonzero values is $N(r_p + q_p)$, and the usual random scatter of such a count, its **standard deviation**, is $\sqrt{Nr_p(1-r_p) + Nq_p(1-q_p)}$ (ASSUMED from probability theory). For $p = 5$ and $N = 100000$:

$$r_5 = \frac{8\cdot 7\cdot 6\cdot 5\cdot 4}{8^5} = \frac{6720}{32768} = 0.20508, \qquad q_5 = \frac{6720\cdot 120}{8^{10}} = \frac{806400}{1073741824} = 0.00075$$

(the formula; then $8^5 = 32768$ and $8^{10} = 1073741824$), so the expected count is $100000\cdot(0.20508 + 0.00075) = 20583$ (rounded), with a standard deviation of about 128. The record's count is 20538 (`gkd-selftest.json`), 0.35 standard deviations below the expectation. Notebook 11a, In [15], makes this comparison for every length 5 to 9 of the record, and In [11] for its own 2,000 Python samples of each length 4 to 9; all counts lie within 2 standard deviations (Figure 11a.6). For $p = 9$ the chance is $r_9 = 0$, because the factor $(8 - 9 + 1)/8 = 0$ appears: the pigeonhole principle once more, and both tests find no nonzero value. A random test is evidence, not a proof; the proof that GKD equals the determinant is the theorem of Section 11.3. The tests show that the two programs implement the theorem without a slip.

11.5 The author's second route: two Levi-Civita tensors of the curved metric

The author's notebook builds the generalized delta also in a second way, from two **Levi-Civita tensors**, in the cell In[32] (record `Revision/gkd_lovelock/results/notebook-input-cells.txt`). Section 1.44 proved the identity behind it for the Levi-Civita **symbol** $[i_1 \ldots i_8]$ (the sign of the list, or 0):

$$\sum_{c_2,\ldots,c_8} [a\, c_2 \ldots c_8]\,[b\, c_2 \ldots c_8] = 7!\;\delta^b_a.$$

The author uses the Levi-Civita **tensor** of a curved metric g, whose second factor carries upper labels raised with the inverse metric. Section 1.44 used the fact that raising all labels multiplies the product by the sign of $\det g$, and promised it here. We prove it now.

A determinant identity (PROVED). For every $n \times n$ matrix N and every list $(i_1, \ldots, i_n)$,

$$\sum_{m_1,\ldots,m_n} N_{i_1 m_1}\cdots N_{i_n m_n}\,[m_1 \ldots m_n] = \det N\;[i_1 \ldots i_n].$$

Proof. (i) For the list $(i_1, \ldots, i_n) = (1, \ldots, n)$: the symbol $[m_1 \ldots m_n]$ is 0 unless $(m_1, \ldots, m_n)$ is a permutation π, and then it is $\mathrm{sign}(\pi)$; so the left side is $\sum_\pi \mathrm{sign}(\pi) N_{1\pi(1)}\cdots N_{n\pi(n)}$, the Leibniz formula, which is $\det N = \det N\,[1 \ldots n]$. (ii) Exchanging two entries of the list $(i_1, \ldots, i_n)$ exchanges

two rows of the products on the left, which flips the sign of the left side (rule 3 of Section 1.20, applied to the matrix with the rows $i_1, \dots, i_n$), and it flips the sign of $[i_1 \dots i_n]$ on the right (Section 1.19). Every re-ordering of $(1, \dots, n)$ is reached by such exchanges, so both sides agree for every re-ordering. (iii) If the list repeats a label, the matrix with the rows $i_1, \dots, i_n$ has two equal rows, so the left side is 0 (rule 4), and so is the right side. □

Raising all labels (PROVED). The Levi-Civita tensor of g is $\varepsilon_{i_1 \dots i_8} = \sqrt{|\det g|}\,[i_1 \dots i_8]$; raising all its labels gives

$$\varepsilon^{i_1 \dots i_8} = \sum_{m_1, \dots, m_8} g^{i_1 m_1} \cdots g^{i_8 m_8} \sqrt{|\det g|}\,[m_1 \dots m_8]$$

(one inverse metric for each label),

$$= \sqrt{|\det g|}\ \det(g^{-1})\,[i_1 \dots i_8] = \frac{\sqrt{|\det g|}}{\det g}\,[i_1 \dots i_8]$$

(the identity just proved with $N = g^{-1}$; then $\det(g^{-1}) = 1/\det g$, rule 1 of Section 1.20 applied to $g\,g^{-1} = I$). For the author's diagonal metric no general rule is needed: only $m_r = i_r$ survives in each sum, and the product $g^{i_1 i_1} \cdots g^{i_8 i_8}$ over a re-ordering of all eight labels is the product of all eight diagonal entries $1/g_{aa}$, which is $1/\det g$ (the determinant of a diagonal matrix is the product of its diagonal, Section 1.20). Multiplying the two tensors,

$$\varepsilon_{a_1 \dots a_8}\,\varepsilon^{b_1 \dots b_8} = \sqrt{|\det g|} \cdot \frac{\sqrt{|\det g|}}{\det g}\,[a_1 \dots a_8][b_1 \dots b_8] = \frac{|\det g|}{\det g}\,[a_1 \dots a_8][b_1 \dots b_8]$$

(the two definitions; $\sqrt{u}\sqrt{u} = u$), and $|\det g|/\det g$ is the sign of $\det g$. For the author's metric $\det g = \cos^2 z > 0$ for every a_4 (with the second function b_4 of the metric of 2026-10-14, $\det g = b_4^3 \cos^2 z$, still positive) (record `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `sqrt_abs_det_g`; `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, check `sqrt_det_g_is_cos`; Section 1.21), so the sign is $+1$: the product of the two curved Levi-Civita tensors is exactly the product of the two symbols, and the author's second route gives, after the contraction over seven labels and the division by 7!, the ordinary Kronecker delta δ^b_a with no extra sign. In a metric with an odd number of time-like directions, such as the four-dimensional spacetime of everyday physics, the sign would be -1. Notebook 01c checks the author's second route with the flat metric of signature (4,4) (Section 1.44).

11.6 Example: the generalized Kronecker delta in Python and in Rust

Notebook 11a puts Sections 11.3 and 11.4 to work. It writes the author's definition literally in Python, as an exact determinant computed by sympy; writes the rule GKD as a Python function; reproduces the record's five examples; draws three 4×4 matrices and all six arrangements of three labels; compares the rule with the determinant for all 266,304 pairs of lengths 1 to 3 and for 12,000 pseudo-random pairs of lengths 4 to 9; counts the values and compares the counts with the formula and with the Wolfram record; shows that every delta of nine labels vanishes; compares the cost of the two methods; builds the Revision Rust program `lovelock_gkd` with cargo and reads the record `Revision/gkd_lovelock/results/gkd-selftest.json` of its GKD self-test, checking every

row; compares the self-test's counts with the birthday formula; and checks once more that every check and number it quotes from the Revision records is there. It needs Rust. The Rust self-test itself takes 7 to 13 minutes, because it computes 200,000 literal determinants of 9×9 matrices, each a sum of 362,880 products; that is longer than the limit of 7 minutes that this book sets for one computation, so the notebook prints the command that repeats it instead of running it (In [13]). The whole notebook took 8.2 seconds when it was built on 2026-10-15 (the provenance file `Revision/textbook/notebooks/11a_kronecker_delta_gkd.PROVENANCE.md`). It prints 24 PASS lines, draws six figures and ends with the line ALL 24 CHECKS PASSED (notebook 11a).

11.7 How to run Notebook 11a

Step 1. What this notebook does and what it needs.

Notebook 11a (The generalized Kronecker delta GKD in Python and in Rust) is the file `Revision /textbook/notebooks/11a_kronecker_delta_gkd.ipynb` of the repository Dirac_claude. It writes the author's generalized Kronecker delta literally as the determinant of a matrix of zeros and ones, proves and checks that it equals the sign of a permutation or zero (the rule of the Rust function GKD), compares the two for all 266,304 pairs of index lists of length 1, 2 and 3 over the eight coordinate labels and for 12,000 random pairs of length 4 to 9, counts how often the values +1, -1 and 0 occur and derives these counts by a formula, shows why the delta of nine indices in eight dimensions is always zero, and builds the Revision Rust program lovelock_gkd with cargo inside the notebook when the program file is missing (a few seconds, up to a minute on a slow computer; about a second when it is up to date), reads the record of its GKD self-test (16,777,216 pairs of length 4 compared exhaustively, 200,000 random pairs of each length 5 to 9) and checks every row of it; the self-test itself takes 7 to 13 minutes, longer than the limit of 7 minutes set for one computation, so the notebook does not run it and prints the command that repeats it; it checks again that every check and number it quotes from the Revision records is there, with the verdict PASS; it draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program lovelock_gkd, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/gkd_lovelock/code/target/release/lovelock_gkd` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 11a_kronecker_delta_gkd.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows

a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 11a_kronecker_delta_gkd.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 11a_kronecker_delta_gkd.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/11a.captions.json`
- `Revision/textbook/figures/11a_1_outer_delta_matrices.png`
- `Revision/textbook/figures/11a_2_six_permutations.png`
- `Revision/textbook/figures/11a_3_gkd_map_length_two.png`
- `Revision/textbook/figures/11a_4_value_counts.png`
- `Revision/textbook/figures/11a_5_cost_of_the_determinant.png`
- `Revision/textbook/figures/11a_6_all_labels_different.png`

It changes no other file of the repository except the Rust build folder `target` next to each `Cargo.toml` it builds; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files of the notebook exist
ALL 24 CHECKS PASSED (notebook 11a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/11a_kronecker_delta_gkd.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 11a_kronecker_delta_gkd.ipynb
python -m nbconvert --execute --inplace 11a_kronecker_delta_gkd.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- AssertionError: check failed: the record of the Rust program lovelock_gkd gkd-selftest has the verdict SUCCESS: The committed record differs from the one this notebook was written for. Run git status in the repository folder to see whether the record was changed, and restore it with the command below.

```
git checkout -- Revision/gkd_lovelock/results/gkd-selftest.json
```

11.8 Notebook 11a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/11a_kronecker_delta_gkd.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Lovelock tensors of chapter 11 are long sums of products of the curvature of the
author's metric. Every term of these sums carries a weight that is the value of the
**generalized Kronecker delta**, a number $+1$, $-1$ or $0$ that depends on two lists
of coordinate labels. The author defined it, in his Mathematica notebook, as the
determinant of a matrix of zeros and ones. The Revision Rust program `lovelock_gkd`
computes it with a faster rule, the function **GKD**. This notebook

- writes the author's definition literally in Python (a determinant computed exactly
  by sympy) and evaluates it on examples;
- proves that the determinant is always the sign of a permutation or zero, writes
  this rule (GKD) in Python, and checks that the two agree on all 266,304 pairs of
  index lists of length 1, 2 and 3 over the eight labels $x_0, \dots, x_7$, and on
  12,000 pseudo-random pairs of length 4 to 9;
- counts how often the values $+1$, $-1$ and $0$ occur, derives the counts with a
  formula, and reproduces the counts recorded by the Revision's Wolfram check;
- shows that a delta with nine or more indices is always zero in eight dimensions
  (the pigeonhole principle), which is why the Lovelock sum stops at order 3;
- measures that the rule is much faster than the determinant and draws the cost of
  both;
- builds the Revision Rust program `lovelock_gkd` with cargo and reads the record
  `Revision/gkd_lovelock/results/gkd-selftest.json` of its GKD self-test, which
  compares GKD with the literal determinant on 16,777,216 pairs of length 4 and on
  200,000 random pairs of each length 5 to 9; the self-test itself takes 7 to 13
  minutes, longer than the limit of 7 minutes for one computation of this book, so
  the notebook prints the command that repeats it instead of running it;
```

- reads the Revision records once more and checks that every check and number it
  quotes from them is there, every quoted check with the verdict PASS.

It draws six figures and takes about a minute.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 11a (The generalized Kronecker delta GKD in Python and in Rust) is the file
‘Revision/textbook/notebooks/11a_kronecker_delta_gkd.ipynb‘ of the repository
Dirac_claude. It writes the author’s generalized Kronecker delta literally as the
determinant of a matrix of zeros and ones, proves and checks that it equals the sign of a
permutation or zero (the rule of the Rust function GKD), compares the two for all 266,304
pairs of index lists of length 1, 2 and 3 over the eight coordinate labels and for 12,000
random pairs of length 4 to 9, counts how often the values +1, -1 and 0 occur and derives
these counts by a formula, shows why the delta of nine indices in eight dimensions is
always zero, and builds the Revision Rust program lovelock_gkd with cargo inside the
notebook when the program file is missing (a few seconds, up to a minute on a slow
computer; about a second when it is up to date), reads the record of its GKD self-test
(16,777,216 pairs of length 4 compared exhaustively, 200,000 random pairs of each length
5 to 9) and checks every row of it; the self-test itself takes 7 to 13 minutes, longer
than the limit of 7 minutes set for one computation, so the notebook does not run it and
prints the command that repeats it; it checks again that every check and number it quotes
from the Revision records is there, with the verdict PASS; it draws six teaching figures.
It needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust
program lovelock_gkd, which is part of the repository and is built on your computer.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):
```

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
```

```
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘

again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/gkd_lovelock/code/target/release/lovelock_gkd‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 11a_kronecker_delta_gkd.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 11a_kronecker_delta_gkd.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 11a_kronecker_delta_gkd.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/11a.captions.json‘
- ‘Revision/textbook/figures/11a_1_outer_delta_matrices.png‘
- ‘Revision/textbook/figures/11a_2_six_permutations.png‘
- ‘Revision/textbook/figures/11a_3_gkd_map_length_two.png‘
- ‘Revision/textbook/figures/11a_4_value_counts.png‘
- ‘Revision/textbook/figures/11a_5_cost_of_the_determinant.png‘
- ‘Revision/textbook/figures/11a_6_all_labels_different.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS all six figure files of the notebook exist
    ALL 24 CHECKS PASSED (notebook 11a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from

  a copy outside the repository. Open the file
  ‘Revision/textbook/notebooks/11a_kronecker_delta_gkd.ipynb‘ inside the folder
  Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 11a_kronecker_delta_gkd.ipynb
      python -m nbconvert --execute --inplace 11a_kronecker_delta_gkd.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open
  a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this
  terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++
  Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
  again.
- AssertionError: check failed: the record of the Rust program lovelock_gkd gkd-selftest
  has the verdict SUCCESS: The committed record differs from the one this notebook was
  written for. Run git status in the repository folder to see whether the record was
  changed, and restore it with the command below.

      git checkout -- Revision/gkd_lovelock/results/gkd-selftest.json

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 11a (the complete instructions)
#
```

```
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 11a (The generalized Kronecker delta GKD in Python and in Rust) is the file
# Revision/textbook/notebooks/11a_kronecker_delta_gkd.ipynb of the repository
# Dirac_claude. It writes the author's generalized Kronecker delta literally as the
# determinant of a matrix of zeros and ones, proves and checks that it equals the sign of
# a permutation or zero (the rule of the Rust function GKD), compares the two for all
# 266,304 pairs of index lists of length 1, 2 and 3 over the eight coordinate labels and
# for 12,000 random pairs of length 4 to 9, counts how often the values +1, -1 and 0
# occur and derives these counts by a formula, shows why the delta of nine indices in
# eight dimensions is always zero, and builds the Revision Rust program lovelock_gkd with
# cargo inside the notebook when the program file is missing (a few seconds, up to a
# minute on a slow computer; about a second when it is up to date), reads the record of
# its GKD self-test (16,777,216 pairs of length 4 compared exhaustively, 200,000 random
# pairs of each length 5 to 9) and checks every row of it; the self-test itself takes 7
# to 13 minutes, longer than the limit of 7 minutes set for one computation, so the
# notebook does not run it and prints the command that repeats it; it checks again that
# every check and number it quotes from the Revision records is there, with the verdict
# PASS; it draws six teaching figures. It needs a computer with Windows 11, macOS or
# Linux, an internet connection for the installation, the program Git, and Python 3.12 or
# newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It also needs Rust (the program cargo,
# version 1.91.1 or newer), because it runs the Rust program lovelock_gkd, which is part
# of the repository and is built on your computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
```

```
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
#
# Windows, macOS and Linux:
#     cargo --version
#
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
#     cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
#
```

```
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
# Revision/gkd_lovelock/code/target/release/lovelock_gkd (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 11a_kronecker_delta_gkd.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 1 minute
# on a typical laptop. Scroll to the end and compare the last printed lines with the
# lines in the step "What the notebook writes and what you must see" below. To keep the
# results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the
# menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 11a_kronecker_delta_gkd.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 11a_kronecker_delta_gkd.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/11a.captions.json
# - Revision/textbook/figures/11a_1_outer_delta_matrices.png
# - Revision/textbook/figures/11a_2_six_permutations.png
# - Revision/textbook/figures/11a_3_gkd_map_length_two.png
# - Revision/textbook/figures/11a_4_value_counts.png
# - Revision/textbook/figures/11a_5_cost_of_the_determinant.png
# - Revision/textbook/figures/11a_6_all_labels_different.png
#
```

```
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds; running it headless or saving it in JupyterLab also rewrites
# the notebook file itself. It does not use the internet while it runs. The files it
# writes are the same files that are stored in the repository (on another computer a
# figure may differ in a few bytes, which is harmless). To get the stored versions back,
# run this command in the repository folder (it also undoes every change you made
# yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files of the notebook exist
#     ALL 24 CHECKS PASSED (notebook 11a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/11a_kronecker_delta_gkd.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 11a_kronecker_delta_gkd.ipynb
#     python -m nbconvert --execute --inplace 11a_kronecker_delta_gkd.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
```

```
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - AssertionError: check failed: the record of the Rust program lovelock_gkd
#   gkd-selftest has the verdict SUCCESS: The committed record differs from the one this
#   notebook was written for. Run git status in the repository folder to see whether the
#   record was changed, and restore it with the command below.
#     git checkout -- Revision/gkd_lovelock/results/gkd-selftest.json
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "11a"  # this notebook: chapter 11, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
```

```
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 11a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinate label**: the name of one of the eight coordinates $x_0, \dots, x_7$ of
  the author's metric ($x_1, x_2, x_3$ ordinary space, $x_4$ the time, $x_5, x_6,
  x_7$ the three extra times, which deflate exponentially, $x_0$ the hidden
  direction). Programs number them
  $0, 1, \dots, 7$: the number 0 stands for $x_1$, the number 7 for $x_0$.
- **Index list**: a list of labels such as $(x_2, x_1, x_4)$; its **length** $p$ is
  the number of entries. Two lists of the same length are a **pair**: the
  **lower** list $(l_1, \dots, l_p)$ and the **upper** list $(u_1, \dots, u_p)$.
- **Kronecker delta** $\delta(a, b)$: the number 1 if the labels $a$ and $b$ are
  equal and 0 if they are different.
- **Matrix, entry, row, column**: a matrix is a rectangle of numbers; the number in
  row $i$ and column $j$ is the entry $M_{ij}$. A **0/1 matrix** has only the
  entries 0 and 1.
- **Outer[delta, lower, upper]**: the author's Mathematica name for the $p \times p$
  matrix with the entries $M_{ij} = \delta(l_i, u_j)$: row $i$ belongs to the lower
  label $l_i$, column $j$ to the upper label $u_j$.
- **Permutation**: a rearrangement of $p$ objects; we describe it by the list
  $(\sigma(1), \dots, \sigma(p))$ that says where object $i$ goes. There are
  $p! = 1 \cdot 2 \cdots p$ permutations of $p$ objects ($p!$ is read
  "$p$ factorial").
- **Inversion**: a pair of places $i < j$ with $\sigma(i) > \sigma(j)$, that is, two
  objects that the permutation puts in the opposite order.
- **Sign of a permutation**: $\mathrm{sign}(\sigma) = (-1)^{N}$, where $N$ is the
  number of inversions: $+1$ for an **even** permutation (even $N$), $-1$ for an
  **odd** one. A **transposition** exchanges two objects.
- **Determinant** (Leibniz formula): for a $p \times p$ matrix $M$,
  $\det M = \sum_{\pi} \mathrm{sign}(\pi)\, M_{1\pi(1)} M_{2\pi(2)} \cdots
  M_{p\pi(p)}$, a sum of $p!$ products, one for every permutation $\pi$.
- **Generalized Kronecker delta**: the number
  $\delta^{u_1 \dots u_p}_{l_1 \dots l_p} = \det[\delta(l_i, u_j)]$; the author
  writes it k$\delta$[lower, upper].
- **GKD**: the name of the Rust function of the Revision program that computes the
  generalized Kronecker delta by the permutation rule proved in this notebook.
- **Exhaustive test**: a test of every possible case. **Pseudo-random numbers**:
  numbers made by a fixed recipe that look random; the recipe starts from a number
  called the **seed**, and the same seed gives the same numbers on every computer.
- **Pigeonhole principle**: if more than $n$ objects are put into $n$ boxes, some
  box holds two objects.
- **Rust, cargo, program**: Rust is a programming language; cargo builds a Rust
  **crate** (a folder with a file `Cargo.toml` and source files) into a program file
  that the computer runs.
- **Revision record**: a file committed under `Revision/` that holds a result of the
  Revision computations. **Byte for byte**: two files are equal in every byte, so
  they are the same file.
```

Text cell:

4. The physical and mathematical situation

Why the delta is needed. In eight dimensions Lovelock's equation (4.38), which the author's notebook quotes, adds to Einstein's tensor two more tensors built from products of two and of three curvature tensors. Written out, the tensor of order k is a sum over $2k$ upper and $2k$ lower coordinate labels plus one free pair, and every term is weighted by a generalized Kronecker delta with $p = 2k + 1$ upper and $2k + 1$ lower labels. For the 64 components of the tensor of order $k = 3$ the program ‘lovelock_gkd‘ evaluates the delta 495,360 times (record ‘Revision/gkd_lovelock/results/lovelock-report.json‘, counter ‘gkdCalls‘). So the delta must be right and fast.

The author's definition. The Revision record ‘Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md‘ quotes it from the input cell In[54] of the author's notebook (with δ written as delta here):

```
    kdelta[lower_, upper_] /; Length[lower] == Length[upper] :=
        Det[Outer[delta, lower, upper]]
```

In words: for two lists of the same length p, build the $p \times p$ matrix $M_{ij} = \delta(l_i, u_j)$ and take its determinant. For lists of different lengths the definition gives nothing (Mathematica leaves the expression unevaluated). For $p = 1$ the matrix is the single number $\delta(l_1, u_1)$, so the generalized delta is the ordinary Kronecker delta. For $p = 2$,

$$\delta^{u_1 u_2}_{l_1 l_2} = \delta(l_1, u_1)\,\delta(l_2, u_2)
  - \delta(l_1, u_2)\,\delta(l_2, u_1),$$

by the formula $\det = M_{11} M_{22} - M_{12} M_{21}$ of a 2×2 determinant.

Theorem (the rule GKD). Let $M_{ij} = \delta(l_i, u_j)$. Then $\det M = 0$ when a label appears twice in the lower list, or twice in the upper list, or when some lower label is missing from the upper list; otherwise $\det M = \mathrm{sign}(\sigma)$, where $\sigma(i)$ is the place of l_i in the upper list.

Proof, case by case.

(a) Two equal lower labels, $l_i = l_k$ with $i \ne k$. Then row i and row k of M are equal, because $M_{ij} = \delta(l_i, u_j) = \delta(l_k, u_j) = M_{kj}$ for every column j. Exchanging two rows of a matrix changes the sign of its determinant; exchanging two EQUAL rows does not change the matrix. So $\det M = -\det M$, which gives $2 \det M = 0$ and $\det M = 0$.

(b) Two equal upper labels, $u_j = u_k$. Then columns j and k are equal, and the same argument for columns (a matrix and its transpose have the same determinant) gives $\det M = 0$.

(c) Some lower label l_i equals no upper label. Then row i is all zeros. Every product in the Leibniz formula contains exactly one entry of row i, so every product is zero, and $\det M = 0$.

(d) Otherwise the p lower labels are different, the p upper labels are different, and every lower label appears in the upper list. Then l_i equals

exactly one upper label, $u_{\sigma(i)}$. Different lower labels sit at different places, so σ sends the places $1, \dots, p$ to the places $1, \dots, p$ without repetition: σ is a permutation. Row i of M has a single 1, in column $\sigma(i)$. In the Leibniz formula the product for a permutation π is $M_{1\pi(1)} \cdots M_{p\pi(p)}$; it is 1 if $\pi(i) = \sigma(i)$ for every i and 0 otherwise. So only the term $\pi = \sigma$ survives, and $\det M = \mathrm{sign}(\sigma) \cdot 1 = \mathrm{sign}(\sigma)$. End of proof.

The rule needs only about $p^2/2$ comparisons of two numbers, while the Leibniz formula adds $p!$ products. For $p = 7$ that is 21 comparisons against 5,040 products.

How many pairs give $+1$, -1 and 0. A lower list of p DIFFERENT labels out of 8 can be chosen in $8 \cdot 7 \cdots (8 - p + 1) = 8!/(8 - p)!$ ways (8 choices for the first label, 7 for the second, and so on). The delta is nonzero exactly when the upper list holds the same p labels in some order: $p!$ orders. Hence the number of pairs with a nonzero delta is

$$N_{\ne 0}(p) = \frac{8!}{(8 - p)!}\, p!,$$

out of $8^p \cdot 8^p = 8^{2p}$ pairs. For $p \ge 2$ exactly half of the $p!$ orders are even. To see this, exchange the first two labels of the upper list. In the list σ this exchanges the values 1 and 2 (the places of these two labels). The two entries of σ that hold the values 1 and 2 change their relative order, so the pair they form becomes an inversion if it was none, and stops being one if it was one. Every other pair of entries is compared as before: a pair that contains neither of the two entries is unchanged, and an entry with a value of 3 or more is larger than both 1 and 2, so its comparison with either of them gives the same answer. Hence the number of inversions changes by exactly one: the exchange turns every even order into an odd one and back, and so pairs the even orders one to one with the odd orders. For $p = 1$ the only order is the identity, which is even. So $p = 1$ gives 8 values $+1$; $p = 2$ gives $56 \cdot 2 = 112$ nonzero values, 56 of each sign; $p = 3$ gives $336 \cdot 6 = 2016$, 1008 of each sign.

Nine labels in eight dimensions. A list of 9 labels taken from 8 must repeat a label (pigeonhole principle), so by case (a) every delta with $p \ge 9$ is zero. The Lovelock tensor of order k needs $p = 2k + 1$: $p = 3, 5, 7$ for $k = 1, 2, 3$ and $p = 9$ for $k = 4$. That is why the Lovelock sum in eight dimensions stops at $k = 3$.

Status. The theorem is PROVED above (exact). The comparisons below are COMPUTED checks of the proof and of the two programs.

Text cell:

5. The literal definition: a determinant of zeros and ones

The next cell imports the packages and defines two functions. ‘outer_delta(lower, upper)‘ builds the matrix Outer[delta, lower, upper] as a list of rows. ‘kdelta(lower, upper)‘ is the author’s definition: it refuses two lists of different lengths and returns the determinant of that matrix, computed EXACTLY by sympy (‘sp.Matrix(...).det()‘ gives a whole number, not a decimal approximation). Because many different pairs of lists give the same 0/1 matrix, ‘kdelta‘ remembers every

determinant it has computed in the dictionary ‘DETERMINANTS‘ and computes each matrix only once.

Then it evaluates the five examples of the Revision record ‘python-lovelock-report.json‘ (check ‘gkd_examples‘), whose lists use the labels 1, 2, 3, 4 (any whole numbers can serve as labels: only equality of two labels matters).

In [2]:

```
import itertools  # loops over all lists of labels, and over all permutations
import math  # factorials n! and the numbers n!/(n-p)!
import re  # regular expressions: reads numbers out of the texts of the records
import time  # a stopwatch (used inside one check; no time is ever printed)

import numpy as np  # arrays of numbers and pseudo-random numbers
import sympy as sp  # exact algebra: sympy computes determinants exactly

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # the labels 0..7 by name


def outer_delta(lower, upper):
    """The matrix Outer[delta, lower, upper] as a list of rows: the entry in row i
    and column j is 1 if lower[i] equals upper[j], else 0."""
    return [[1 if a == b else 0 for b in upper] for a in lower]


DETERMINANTS = {}  # memory: a matrix (a tuple of rows) -> its determinant


def kdelta(lower, upper):
    """The author's generalized Kronecker delta, literally: the exact determinant
    of Outer[delta, lower, upper].  Two lists of different lengths are refused."""
    if len(lower) != len(upper):
        raise ValueError("kdelta needs two index lists of the same length")
    matrix = tuple(tuple(row) for row in outer_delta(lower, upper))
    if matrix not in DETERMINANTS:  # each different matrix is computed only once
        DETERMINANTS[matrix] = int(sp.Matrix(matrix).det())
    return DETERMINANTS[matrix]


EXAMPLES = [  # (lower list, upper list, value stated in the Revision record)
    ((1, 2), (1, 2), 1),
    ((1, 2), (2, 1), -1),
    ((1, 1), (1, 1), 0),
    ((1, 2, 3), (2, 3, 1), 1),
    ((1, 2, 3), (1, 2, 4), 0),
]
for lower, upper, stated in EXAMPLES:
    say(f"kdelta[{list(lower)}, {list(upper)}] = {kdelta(lower, upper)}   "
        f"(the record states {stated})")
check(all(kdelta(lower, upper) == stated for lower, upper, stated in EXAMPLES),
      "the five examples give 1, -1, 0, 1, 0",
```

```
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check gkd_examples")
try:  # lists of different lengths must be refused, as in the author's definition
    kdelta((1,), (1, 2))
    refused = False
except ValueError:
    refused = True
check(refused, "kdelta refuses two lists of different lengths")
```

Out [2]:

```
kdelta[[1, 2], [1, 2]] = 1   (the record states 1)
kdelta[[1, 2], [2, 1]] = -1   (the record states -1)
kdelta[[1, 1], [1, 1]] = 0   (the record states 0)
kdelta[[1, 2, 3], [2, 3, 1]] = 1   (the record states 1)
kdelta[[1, 2, 3], [1, 2, 4]] = 0   (the record states 0)
PASS the five examples give 1, -1, 0, 1, 0
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    gkd_examples
PASS kdelta refuses two lists of different lengths
```

Text cell:

```
The next cell draws the matrix Outer[delta, lower, upper] for three pairs of lists
of length 4, one for each kind of value: the upper list $(x_2, x_1, x_4, x_3)$ is the
lower list with two exchanges (an even permutation, value $+1$); the upper list
$(x_2, x_3, x_4, x_1)$ moves every label one place (a cycle of four, which is three
exchanges: odd, value $-1$); and the lower list $(x_1, x_2, x_3, x_3)$ repeats a
label (two equal rows, value $0$). Each entry is written into its square; dark
squares are the ones. ‘ax.imshow‘ draws a matrix as a picture of coloured squares,
and ‘ax.text(column, row, ...)‘ writes a number at the centre of a square.
```

In [3]:

```
SHOWN = [  # (lower list, upper list) with labels 0..7 standing for x0..x7
    ((0, 1, 2, 3), (1, 0, 3, 2)),  # x2 x1 x4 x3: two exchanges, even
    ((0, 1, 2, 3), (1, 2, 3, 0)),  # x2 x3 x4 x1: a cycle of four, odd
    ((0, 1, 2, 2), (0, 1, 2, 3)),  # x3 twice in the lower list
]
fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.9))
for ax, (lower, upper) in zip(axes, SHOWN):
    matrix = np.array(outer_delta(lower, upper))
    ax.imshow(matrix, cmap="Blues", vmin=0.0, vmax=1.5)  # 0 white, 1 blue
    for i in range(4):  # i: the row
        for j in range(4):  # j: the column
            ax.text(j, i, str(matrix[i, j]), ha="center", va="center")
    ax.set_xticks(range(4), [NAMES[label] for label in upper])  # column labels
    ax.set_yticks(range(4), [NAMES[label] for label in lower])  # row labels
    ax.set_xlabel("upper list (one column per label)")
    ax.set_ylabel("lower list (one row per label)")
    value = kdelta(lower, upper)
```

```
    ax.set_title(f"determinant = {value:+d}" if value else "determinant = 0")
    ax.grid(False)  # no grid lines across the squares
fig.tight_layout()  # no overlapping labels
save_figure(fig, "outer_delta_matrices",
            "The matrix Outer of delta for three pairs of index lists of length 4; "
            "row i belongs to the i-th lower label, column j to the j-th upper "
            "label, and the entry is 1 where the two labels are equal. Left: the "
            "upper list is the lower list with two exchanges, the matrix is a "
            "permutation matrix of an even permutation and its determinant is +1. "
            "Middle: a cycle of four labels, an odd permutation, determinant -1. "
            "Right: the label $x_3$ appears twice in the lower list, so two rows are "
            "equal and the column of $x_4$ is empty: determinant 0.")
check([kdelta(lower, upper) for lower, upper in SHOWN] == [1, -1, 0],
      "the three drawn matrices have the determinants +1, -1 and 0")
```

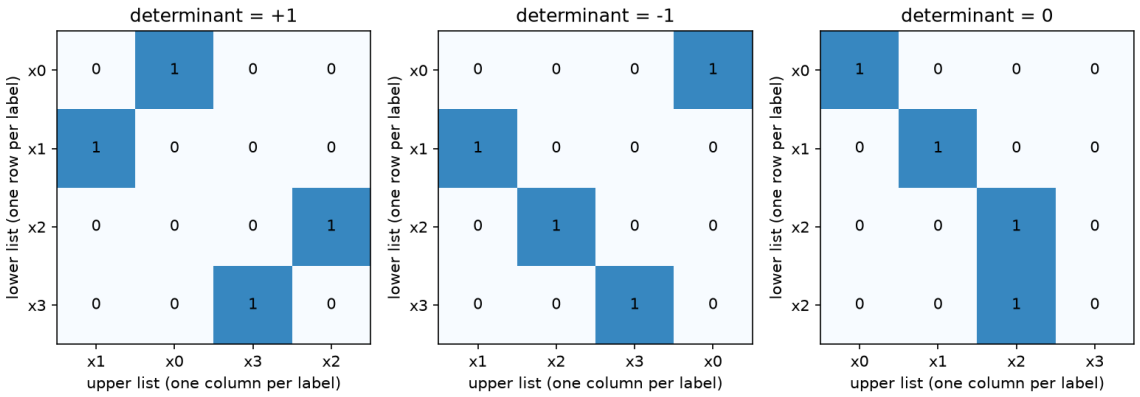

Figure 303: The matrix Outer of delta for three pairs of index lists of length 4; row i belongs to the i-th lower label, column j to the j-th upper label, and the entry is 1 where the two labels are equal. Left: the upper list is the lower list with two exchanges, the matrix is a permutation matrix of an even permutation and its determinant is +1. Middle: a cycle of four labels, an odd permutation, determinant -1. Right: the label x_3 appears twice in the lower list, so two rows are equal and the column of x_4 is empty: determinant 0. (Notebook 11a, figure 1.)

Out [3]:

```
Figure 11a.1 saved as Revision/textbook/figures/11a_1_outer_delta_matrices.png
PASS the three drawn matrices have the determinants +1, -1 and 0
```

Text cell:

```
## 6. The rule behind GKD: the sign of a permutation

The next cell writes the rule of the theorem as the Python function ‘gkd‘. It
follows the Rust function GKD of ‘Revision/gkd_lovelock/code/src/gkd.rs‘ step by
step:

1. It walks through the upper list and records in the dictionary ‘position‘ the
   place of every label; a label seen twice means two equal columns: value 0
```

```
   (case b).
2. It walks through the lower list and looks up the place of every label; a label
   that is not in the upper list means a row of zeros: value 0 (case c). The places
   form the list ‘sigma‘.
3. If a place occurs twice in ‘sigma‘, a label occurs twice in the lower list: value
   0 (case a).
4. Otherwise ‘sigma‘ is a permutation. The cell counts its inversions (pairs of
   places $i < j$ with ‘sigma[i] > sigma[j]‘) and returns $+1$ for an even count and
   $-1$ for an odd count (case d).

The cell then checks that ‘gkd‘ and ‘kdelta‘ agree on the five examples.
```

In [4]:

```
def inversions(sigma):
    """The number of pairs of places i < j with sigma[i] > sigma[j]."""
    return sum(1 for i in range(len(sigma)) for j in range(i + 1, len(sigma))
               if sigma[i] > sigma[j])


def gkd(lower, upper):
    """The rule GKD (the Rust function GKD, written in Python): the sign of the
    permutation that carries the lower list into the upper list, or 0."""
    if len(lower) != len(upper):
        raise ValueError("gkd needs two index lists of the same length")
    position = {}  # label -> its place in the upper list
    for place, label in enumerate(upper):
        if label in position:
            return 0  # a label twice in the upper list: equal columns (case b)
        position[label] = place
    sigma = []  # sigma[i]: the place in the upper list of the i-th lower label
    for label in lower:
        if label not in position:
            return 0  # a lower label missing above: a row of zeros (case c)
        sigma.append(position[label])
    if len(set(sigma)) < len(sigma):
        return 0  # a label twice in the lower list: equal rows (case a)
    return 1 if inversions(sigma) % 2 == 0 else -1  # even +1, odd -1 (case d)


for lower, upper, stated in EXAMPLES:
    say(f"gkd({list(lower)}, {list(upper)}) = {gkd(lower, upper)}")
check(all(gkd(lower, upper) == kdelta(lower, upper) for lower, upper, _ in EXAMPLES),
      "gkd and kdelta agree on the five examples")
```

Out [4]:

```
gkd([1, 2], [1, 2]) = 1
gkd([1, 2], [2, 1]) = -1
gkd([1, 1], [1, 1]) = 0
gkd([1, 2, 3], [2, 3, 1]) = 1
gkd([1, 2, 3], [1, 2, 4]) = 0
PASS gkd and kdelta agree on the five examples
```

Text cell:

```
The next cell draws all $3! = 6$ arrangements of the three labels $x_1, x_2, x_3$:
for each one the matrix Outer[delta, lower, upper] with the lower list
$(x_1, x_2, x_3)$, the permutation $\sigma$ (the places of $x_1, x_2, x_3$ in the
upper list, counted from 0 as Python counts), its number of inversions and its sign.
Three arrangements are even and three are odd, as the counting argument of section 4
says. The check confirms that the sign from the inversions equals the exact
determinant for all six.
```

In [5]:

```
lower = (0, 1, 2)  # x1, x2, x3
fig, axes = plt.subplots(2, 3, figsize=(9.0, 6.4))
signs = []
for ax, upper in zip(axes.flat, itertools.permutations(lower)):
    matrix = np.array(outer_delta(lower, upper))
    sigma = [upper.index(label) for label in lower]  # places of x1, x2, x3 above
    sign = 1 if inversions(sigma) % 2 == 0 else -1
    signs.append((sign, kdelta(lower, upper)))
    ax.imshow(matrix, cmap="Greens" if sign > 0 else "Oranges", vmin=0.0, vmax=1.4)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, str(matrix[i, j]), ha="center", va="center")
    ax.set_xticks(range(3), [NAMES[label] for label in upper])
    ax.set_yticks(range(3), [NAMES[label] for label in lower])
    count = inversions(sigma)  # the number of inversions of sigma
    plural = "" if count == 1 else "s"  # 1 inversion, 2 inversions
    ax.set_title(f"sigma = {tuple(sigma)}, {count} inversion{plural}, "
                 f"sign {sign:+d}", fontsize=9)
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "six_permutations",
            "All six arrangements of the labels $x_1, x_2, x_3$. Each panel shows "
            "the matrix Outer of delta with the lower list $(x_1, x_2, x_3)$ as rows "
            "and the arranged upper list as columns; the title gives the permutation "
            "$\\sigma$ (the place of each lower label in the upper list, counted from "
            "0), its number of inversions and its sign. Green panels are even "
            "permutations (sign +1), orange ones odd (sign -1): three of each, and "
            "each sign equals the determinant of the matrix.")
check(all(sign == det for sign, det in signs) and
      sorted(sign for sign, _ in signs) == [-1, -1, -1, 1, 1, 1],
      "the six arrangements of three labels: three even, three odd, sign = determinant")
```

Out [5]:

```
Figure 11a.2 saved as Revision/textbook/figures/11a_2_six_permutations.png
PASS the six arrangements of three labels: three even, three odd, sign = determinant
```

Text cell:

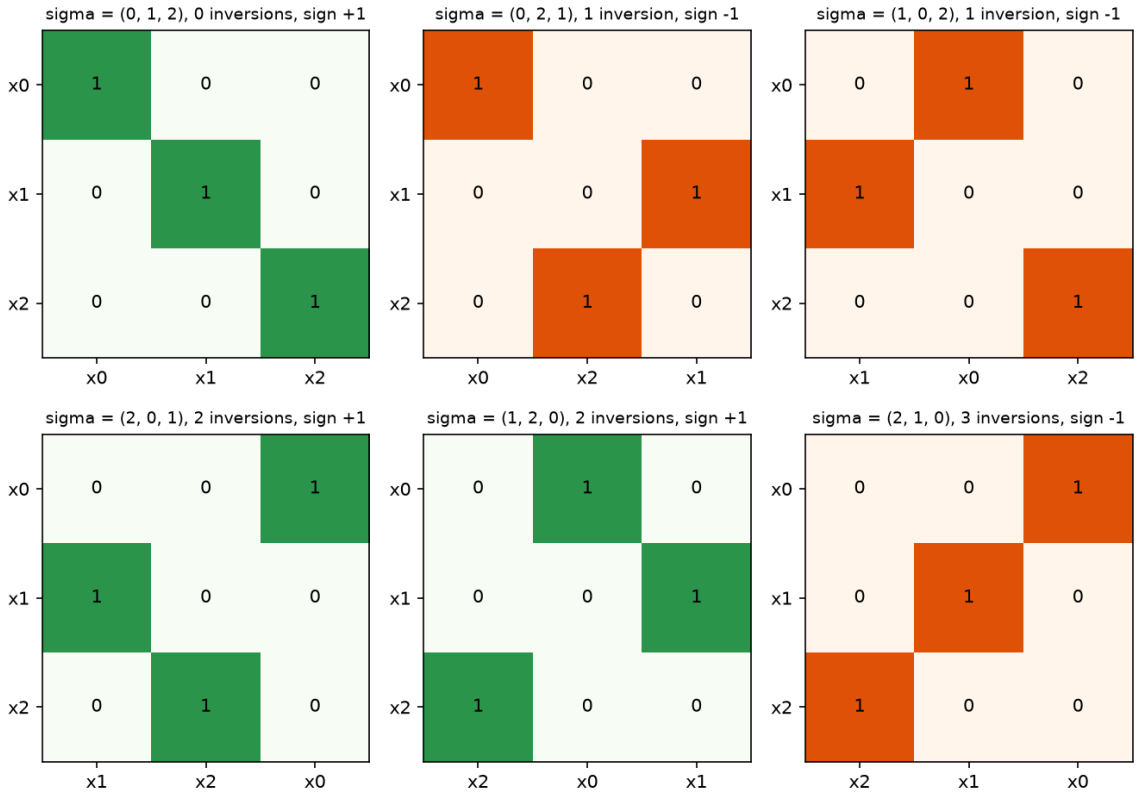

Figure 304: All six arrangements of the labels x_1, x_2, x_3. Each panel shows the matrix Outer of delta with the lower list (x_1, x_2, x_3) as rows and the arranged upper list as columns; the title gives the permutation σ (the place of each lower label in the upper list, counted from 0), its number of inversions and its sign. Green panels are even permutations (sign +1), orange ones odd (sign -1): three of each, and each sign equals the determinant of the matrix. (Notebook 11a, figure 2.)

7. Every pair of lists of length 1, 2 and 3

The next cell compares ‘gkd‘ with the literal determinant ‘kdelta‘ for EVERY pair of index lists of length $p = 1, 2, 3$ over the eight labels: 8^{2p} pairs for each p, that is $64 + 4096 + 262144 = 266304$ pairs. ‘itertools.product(range(8), repeat=p)‘ produces all 8^p lists of length p with entries $0, \dots, 7$. The cell counts how often each value occurs and prints, for each p, the number of pairs and the counts of $+1$, -1 and 0. It takes a few seconds. At the end it reads the Revision’s Wolfram verification ‘wolfram-gkd-report.json‘, whose checks ‘gkd_equals_kdelta_exhaustive_length_1‘ to ‘_3‘ compared GKD with the literal determinant on the same 266,304 pairs, and requires that they found no difference.

In [6]:

```
COUNTS = {}  # p -> {+1: count, -1: count, 0: count}
mismatches = 0  # pairs where gkd and the determinant differ
for p in (1, 2, 3):
    counts = {1: 0, -1: 0, 0: 0}
    for lower in itertools.product(range(8), repeat=p):  # all 8^p lower lists
        for upper in itertools.product(range(8), repeat=p):  # all 8^p upper lists
            value = gkd(lower, upper)
            if value != kdelta(lower, upper):
                mismatches += 1
            counts[value] += 1
    COUNTS[p] = counts
    say(f"p = {p}: {8 ** (2 * p):6d} pairs; value +1: {counts[1]:4d} times, "
        f"-1: {counts[-1]:4d} times, 0: {counts[0]:6d} times")
compared = sum(8 ** (2 * p) for p in (1, 2, 3))
report("pairs compared (all pairs of lengths 1, 2 and 3)", compared)
report("pairs where GKD and the determinant differ", mismatches)
report("different 0/1 matrices whose determinant sympy computed", len(DETERMINANTS))
wolfram_record = json.loads(repository_file(
    "Revision/gkd_lovelock/results/wolfram-gkd-report.json").read_text(
        encoding="utf-8"))
entries = [c for c in wolfram_record["checks"]
           if c["name"].startswith("gkd_equals_kdelta_exhaustive_length_")]
recorded_pairs = sum(int(re.search(r"all (\d+) pairs", e["detail"]).group(1))
                     for e in entries)  # the numbers of pairs the record names
check(mismatches == 0 and compared == 266304 and len(entries) == 3
      and all(e["verdict"] == "PASS" and "(0 mismatches;" in e["detail"]
              for e in entries) and recorded_pairs == 266304,
      "GKD equals the literal determinant for all 266304 pairs of lengths 1 to 3",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks "
             "gkd_equals_kdelta_exhaustive_length_1 to _3")
```

Out [6]:

```
p = 1:     64 pairs; value +1:    8 times, -1:    0 times, 0:     56 times
p = 2:   4096 pairs; value +1:   56 times, -1:   56 times, 0:   3984 times
p = 3: 262144 pairs; value +1: 1008 times, -1: 1008 times, 0: 260128 times
RESULT pairs compared (all pairs of lengths 1, 2 and 3) = 266304
RESULT pairs where GKD and the determinant differ = 0
```

```
RESULT different 0/1 matrices whose determinant sympy computed = 145
PASS GKD equals the literal determinant for all 266304 pairs of lengths 1 to 3
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks
    gkd_equals_kdelta_exhaustive_length_1 to _3
```

Text cell:

```
The next cell compares these counts with two things: the formula of section 4,
$N_{\ne 0}(p) = 8!/(8 - p)! \cdot p!$ with half of the nonzero values $+1$ and half
$-1$ for $p \ge 2$ (all $+1$ for $p = 1$), and the counts that the Revision's Wolfram
verification recorded in ‘wolfram-gkd-report.json‘ (its entry ‘measurements‘,
‘gkdComparison‘, fields ‘plusOne‘, ‘minusOne‘, ‘zero‘). ‘math.perm(8, p)‘ is
$8!/(8 - p)!$ and ‘math.factorial(p)‘ is $p!$.
```

In [7]:

```
def nonzero_pairs(p, n=8):
    """Pairs of lists of length p over n labels whose delta is not 0:
    n!/(n-p)! lower lists of different labels, times p! orders above."""
    return math.perm(n, p) * math.factorial(p)


def formula_counts(p):
    """{+1, -1, 0} -> count, from the formula of section 4."""
    nonzero = nonzero_pairs(p)
    plus = nonzero if p == 1 else nonzero // 2  # p = 1: only the identity
    return {1: plus, -1: nonzero - plus, 0: 8 ** (2 * p) - nonzero}


wolfram = json.loads(repository_file(
    "Revision/gkd_lovelock/results/wolfram-gkd-report.json").read_text(
        encoding="utf-8"))
for p in (1, 2, 3):
    recorded = [m for m in wolfram["measurements"]["gkdComparison"] if m["p"] == p][0]
    plus, minus, zero = recorded["plusOne"], recorded["minusOne"], recorded["zero"]
    say(f"p = {p}: formula {formula_counts(p)}, Wolfram record +1: {plus}, "
        f"-1: {minus}, 0: {zero}")
check(all(COUNTS[p] == formula_counts(p) for p in (1, 2, 3)),
      "the counts of +1, -1 and 0 equal the formula for p = 1, 2, 3")
check(all(COUNTS[p] == {1: m["plusOne"], -1: m["minusOne"], 0: m["zero"]}
          for p in (1, 2, 3)
          for m in wolfram["measurements"]["gkdComparison"] if m["p"] == p),
      "the counts equal those of the Wolfram verification",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, measurements "
             "gkdComparison, p = 1, 2, 3")
```

Out [7]:

```
p = 1: formula {1: 8, -1: 0, 0: 56}, Wolfram record +1: 8, -1: 0, 0: 56
p = 2: formula {1: 56, -1: 56, 0: 3984}, Wolfram record +1: 56, -1: 56, 0: 3984
p = 3: formula {1: 1008, -1: 1008, 0: 260128}, Wolfram record +1: 1008, -1: 1008, 0:
```

```
    260128
PASS the counts of +1, -1 and 0 equal the formula for p = 1, 2, 3
PASS the counts equal those of the Wolfram verification
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, measurements
    gkdComparison, p = 1, 2, 3
```

Text cell:

```
The next cell draws the value of GKD for all 4096 pairs of length 2 as one picture
of $64 \times 64$ squares. Row number $8 l_1 + l_2$ belongs to the lower list
$(l_1, l_2)$ and column number $8 u_1 + u_2$ to the upper list $(u_1, u_2)$ (so the
rows and columns run through $(x_0, x_0), (x_0, x_1), \dots, (x_7, x_7)$). A red
square is $+1$, a blue square $-1$, white is $0$. The red squares lie on the
diagonal (upper list = lower list), except at the 8 lists with $l_1 = l_2$; each
blue square is the mirror of a red one (the two labels exchanged).
```

In [8]:

```
pairs_of_labels = list(itertools.product(range(8), repeat=2))  # 64 lists (l1, l2)
grid = np.array([[gkd(lower, upper) for upper in pairs_of_labels]
                 for lower in pairs_of_labels])  # 64 x 64 values
fig, ax = plt.subplots(figsize=(6.6, 6.0))
picture = ax.imshow(grid, cmap="bwr", vmin=-1, vmax=1)  # blue -1, red +1
ticks = range(0, 64, 8)  # one tick at the start of each block of 8
ax.set_xticks(ticks, [f"({NAMES[t // 8]},x1)" for t in ticks], rotation=90)
ax.set_yticks(ticks, [f"({NAMES[t // 8]},x1)" for t in ticks])
ax.set_xlabel("upper list $(u_1, u_2)$, numbered $8 u_1 + u_2$")
ax.set_ylabel("lower list $(l_1, l_2)$, numbered $8 l_1 + l_2$")
ax.set_title("GKD for all 4096 pairs of index lists of length 2")
ax.grid(False)
fig.colorbar(picture, ax=ax, ticks=[-1, 0, 1], shrink=0.8, label="value of GKD")
save_figure(fig, "gkd_map_length_two",
            "The value of the generalized Kronecker delta for all 4096 pairs of "
            "index lists of length 2 over the labels $x_0, \\dots, x_7$; row "
            "$8 l_1 + l_2$ is the lower list $(l_1, l_2)$ and column $8 u_1 + u_2$ "
            "the upper list, counted from 0. Red squares are +1, blue squares -1, "
            "white squares 0. There are 56 red squares on the diagonal, where the "
            "upper list equals the lower list (the 8 lists with two equal labels "
            "are white), and 56 blue squares at the mirrored places, where the two "
            "labels are exchanged; the other 3984 values are 0.")
check(int((grid == 1).sum()) == 56 and int((grid == -1).sum()) == 56
      and np.array_equal(grid, grid.T),
      "length 2: 56 values +1, 56 values -1, and the picture is symmetric")
```

Out [8]:

```
Figure 11a.3 saved as Revision/textbook/figures/11a_3_gkd_map_length_two.png
PASS length 2: 56 values +1, 56 values -1, and the picture is symmetric
```

Text cell:

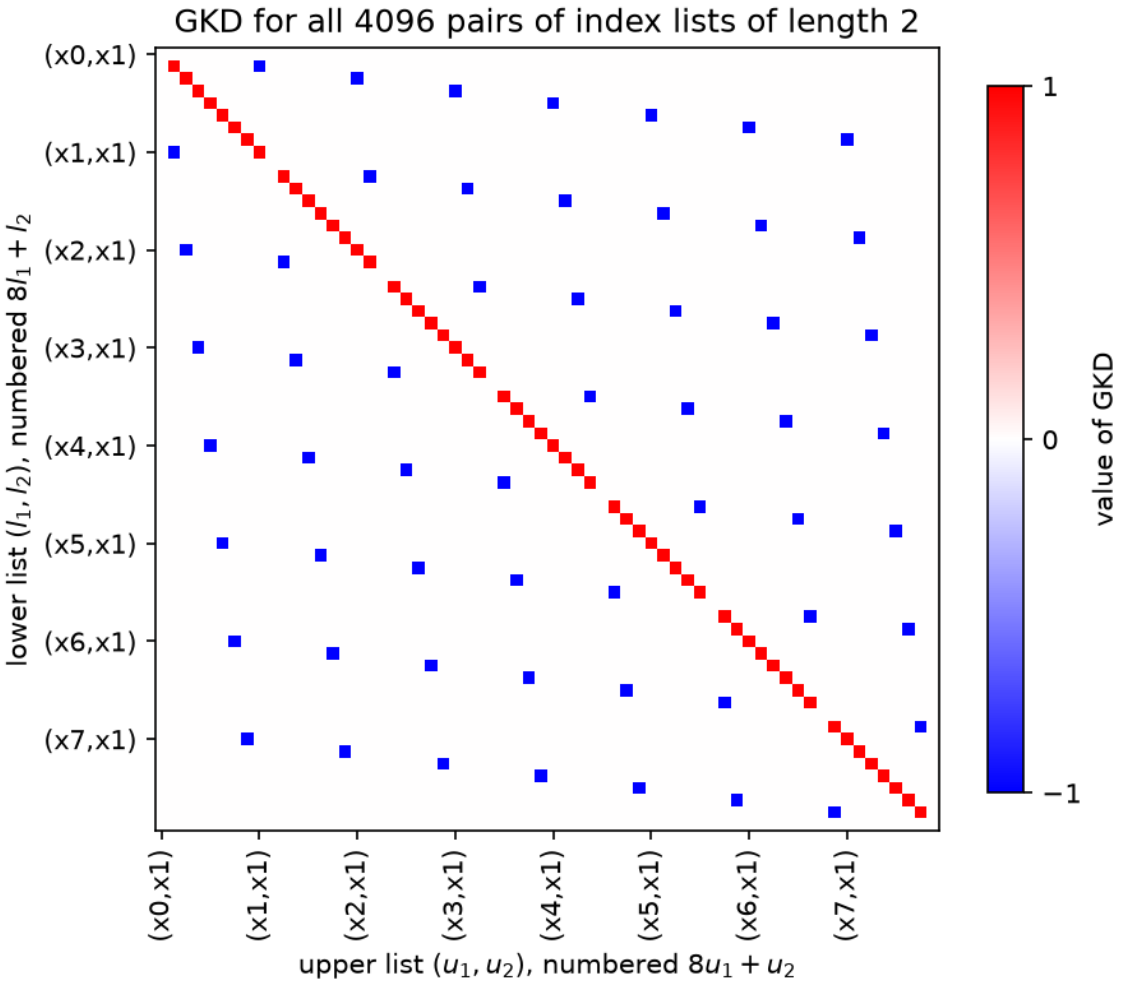

Figure 305: The value of the generalized Kronecker delta for all 4096 pairs of index lists of length 2 over the labels $x_0, \ldots, x_7$; row $8l_1 + l_2$ is the lower list (l_1, l_2) and column $8u_1 + u_2$ the upper list, counted from 0. Red squares are +1, blue squares -1, white squares 0. There are 56 red squares on the diagonal, where the upper list equals the lower list (the 8 lists with two equal labels are white), and 56 blue squares at the mirrored places, where the two labels are exchanged; the other 3984 values are 0. (Notebook 11a, figure 3.)

```
The next cell draws the three counts for each length $p = 1, 2, 3, 4$ as bars on a
logarithmic vertical axis (each step of the axis is a factor of 10, so that 8 and
16 million fit into one picture). For $p = 1, 2, 3$ the bars are the counts measured
above; for $p = 4$ they come from the formula (the Rust self-test of section 10
compares all 16,777,216 pairs of length 4 but prints only the number of
disagreements). A count of zero cannot be drawn on a logarithmic axis; it is marked
"none".
```

In [9]:

```
fig, ax = plt.subplots(figsize=(7.4, 4.4))
width = 0.26  # the width of one bar
for shift, value, colour, name in ((-width, 1, "tab:red", "value +1"),
                                   (0.0, -1, "tab:blue", "value -1"),
                                   (width, 0, "0.6", "value 0")):
    heights = [COUNTS[p][value] if p < 4 else formula_counts(4)[value]
               for p in (1, 2, 3, 4)]
    bars = ax.bar([p + shift for p in (1, 2, 3, 4)], heights, width, color=colour,
                  label=name, hatch=None)
    bars[3].set_hatch("//")  # p = 4: from the formula
    for p, height in zip((1, 2, 3, 4), heights):
        ax.text(p + shift, max(height, 1.2) * 1.15, f"{height:,}" if height else
                "none", ha="center", va="bottom", fontsize=7, rotation=90)
ax.set_yscale("log")  # a logarithmic vertical axis
ax.set_ylim(1.0, 2e9)
ax.set_xticks([1, 2, 3, 4], ["p = 1", "p = 2", "p = 3", "p = 4 (formula)"])
ax.set_xlabel("length $p$ of the two index lists")
ax.set_ylabel("number of pairs (logarithmic scale)")
ax.set_title("How often GKD is +1, -1 and 0 among all $8^{2p}$ pairs")
ax.legend(loc="upper left")
save_figure(fig, "value_counts",
            "The number of pairs of index lists of length $p$ over the eight labels "
            "whose generalized Kronecker delta is +1 (red), -1 (blue) and 0 (grey), "
            "for $p = 1$ to $4$, on a logarithmic vertical axis; the hatched bars of "
            "$p = 4$ come from the formula $N = 8!/(8 - p)! \\cdot p!$ for the "
            "nonzero values, the others are counted by the notebook. The value 0 "
            "dominates more and more: for $p = 4$ only 40,320 of the 16,777,216 "
            "pairs are nonzero, half of them +1 and half -1.")
check(formula_counts(4) == {1: 20160, -1: 20160, 0: 16736896},
      "length 4: 20160 values +1, 20160 values -1 and 16736896 zeros (formula)")
```

Out [9]:

```
Figure 11a.4 saved as Revision/textbook/figures/11a_4_value_counts.png
PASS length 4: 20160 values +1, 20160 values -1 and 16736896 zeros (formula)
```

Text cell:

```
## 8. Longer lists: random pairs and the pigeonhole principle

For $p = 4$ to $9$ there are too many pairs to test them all in Python ($8^{18}$ for
```

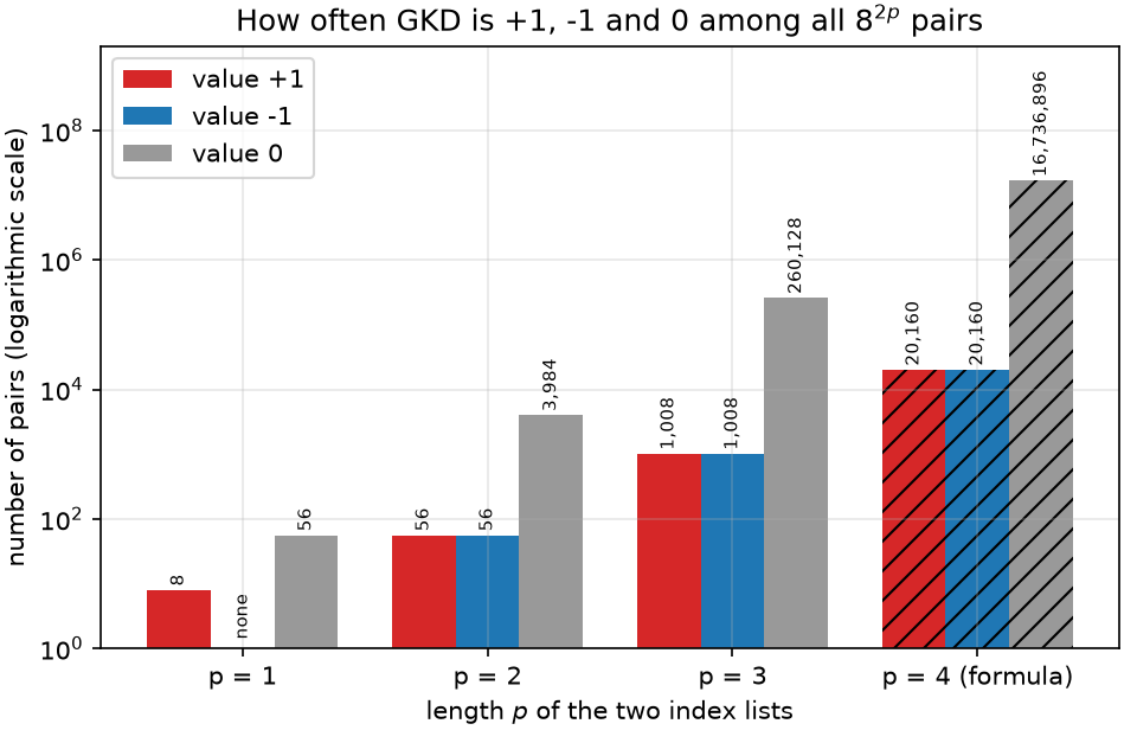

Figure 306: The number of pairs of index lists of length p over the eight labels whose generalized Kronecker delta is +1 (red), -1 (blue) and 0 (grey), for $p = 1$ to 4, on a logarithmic vertical axis; the hatched bars of $p = 4$ come from the formula $N = 8!/(8-p)! \cdot p!$ for the nonzero values, the others are counted by the notebook. The value 0 dominates more and more: for $p = 4$ only 40,320 of the 16,777,216 pairs are nonzero, half of them +1 and half -1. (Notebook 11a, figure 4.)

$p = 9$). The next cell tests 2,000 pseudo-random pairs of each length. The generator ‘np.random.default_rng(12345)‘ makes the same numbers on every computer. For an even sample number ‘s‘ the lower list is the upper list rearranged in random order (‘rng.permutation(p)‘ is a random arrangement of the places), so the delta is nonzero whenever the upper list has p different labels; for an odd ‘s‘ the lower list is a second, independent random list, and the delta is almost always zero. The cell counts the disagreements between ‘gkd‘ and ‘kdelta‘ and the number of nonzero values. Because a list of 9 labels from 8 must repeat a label, every value of length 9 must be zero. The cell takes a few seconds: sympy computes the determinants of about ten thousand new matrices of sizes 4×4 to 9×9 (the cell prints how many different matrices ‘kdelta‘ has met so far).

In [10]:

```
rng = np.random.default_rng(12345)  # pseudo-random numbers with a fixed seed
SAMPLES = 2000  # pairs per length p
python_nonzero = {}  # p -> number of nonzero values among the samples
random_mismatches = 0
for p in range(4, 10):
    nonzero = 0
    for s in range(SAMPLES):
        upper = [int(v) for v in rng.integers(0, 8, size=p)]  # p labels from 0..7
        if s % 2 == 0:  # even s: the lower list is the upper list rearranged
            lower = [upper[i] for i in rng.permutation(p)]
        else:  # odd s: an independent random lower list
            lower = [int(v) for v in rng.integers(0, 8, size=p)]
        value = gkd(lower, upper)
        nonzero += value != 0  # True counts as 1, False as 0
        random_mismatches += value != kdelta(lower, upper)
    python_nonzero[p] = nonzero
    say(f"p = {p}: {SAMPLES} random pairs, {nonzero:4d} nonzero values")
report("random pairs compared (lengths 4 to 9)", 6 * SAMPLES)
report("random pairs where GKD and the determinant differ", random_mismatches)
report("different 0/1 matrices whose determinant sympy computed (so far)",
       len(DETERMINANTS))
check(random_mismatches == 0,
      "GKD equals the literal determinant for 12000 random pairs of lengths 4 to 9")
check(python_nonzero[9] == 0,
      "every delta with 9 indices in 8 dimensions is 0 (pigeonhole)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check gkd_nine_indices_in_eight_dimensions_vanish")
```

Out [10]:

```
p = 4: 2000 random pairs,  394 nonzero values
p = 5: 2000 random pairs,  186 nonzero values
p = 6: 2000 random pairs,   82 nonzero values
p = 7: 2000 random pairs,   24 nonzero values
p = 8: 2000 random pairs,    3 nonzero values
p = 9: 2000 random pairs,    0 nonzero values
RESULT random pairs compared (lengths 4 to 9) = 12000
RESULT random pairs where GKD and the determinant differ = 0
```

```
RESULT different 0/1 matrices whose determinant sympy computed (so far) = 10097
PASS GKD equals the literal determinant for 12000 random pairs of lengths 4 to 9
PASS every delta with 9 indices in 8 dimensions is 0 (pigeonhole)
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    gkd_nine_indices_in_eight_dimensions_vanish
```

Text cell:

```
How many nonzero values should we expect? In an even sample the delta is nonzero
exactly when the $p$ random labels of the upper list are all different. Each label
is one of 8 with equal chance, so the chance that $p$ labels are all different is

$$r_p = \frac{8}{8} \cdot \frac{7}{8} \cdots \frac{8 - p + 1}{8}
      = \frac{8!}{(8 - p)!\, 8^p}$$

(the second label must avoid the first, the third the first two, and so on; this is
the "birthday problem" with 8 possible birthdays). In an odd sample the chance is
$q_p = N_{\ne 0}(p)/8^{2p}$, tiny for $p \ge 4$. With 1,000 samples of each kind the
expected number of nonzero values is $1000\,(r_p + q_p)$, and the usual random
scatter of such a count (its standard deviation) is
$\sqrt{1000\, r_p (1 - r_p) + 1000\, q_p (1 - q_p)}$. The next cell checks that every
count lies within 5 standard deviations of its expectation (a count further away
would happen by chance less than once in a million times).
```

In [11]:

```
def all_different(p, n=8):
    """The chance r_p that p random labels out of n are all different."""
    return math.perm(n, p) / n ** p if p <= n else 0.0


def expected_nonzero(p, half):
    """Expected number of nonzero deltas and its standard deviation, for `half`
    rearranged pairs and `half` independent pairs of length p."""
    r = all_different(p)
    q = nonzero_pairs(p) / 8 ** (2 * p) if p <= 8 else 0.0
    mean = half * (r + q)
    spread = math.sqrt(half * r * (1 - r) + half * q * (1 - q))
    return mean, spread


within = True
for p in range(4, 10):
    mean, spread = expected_nonzero(p, SAMPLES // 2)
    distance = abs(python_nonzero[p] - mean) / spread if spread > 0 else 0.0
    say(f"p = {p}: counted {python_nonzero[p]:4d}, expected {mean:7.1f} "
        f"+- {spread:5.1f}  ({distance:.2f} standard deviations away)")
    within = within and (distance < 5.0 if spread > 0 else python_nonzero[p] == 0)
check(within, "the Python counts of nonzero values agree with the birthday formula")
```

Out [11]:

```
p = 4: counted  394, expected   412.6 +-  15.6  (1.19 standard deviations away)
p = 5: counted  186, expected   205.8 +-  12.8  (1.55 standard deviations away)
p = 6: counted   82, expected    77.1 +-   8.4  (0.58 standard deviations away)
p = 7: counted   24, expected    19.3 +-   4.3  (1.09 standard deviations away)
p = 8: counted    3, expected     2.4 +-   1.6  (0.38 standard deviations away)
p = 9: counted    0, expected     0.0 +-   0.0  (0.00 standard deviations away)
PASS the Python counts of nonzero values agree with the birthday formula
```

Text cell:

```
## 9. Why the rule is needed: the cost of the determinant

The Leibniz formula adds $p!$ products of $p$ entries; the rule GKD makes
$p(p - 1)/2$ comparisons of places. The next cell prints both numbers for
$p = 1, \dots, 9$, together with the number $8^{2p}$ of all pairs of lists of length
$p$, and draws them. Then it MEASURES the difference: it times 200 evaluations of
‘gkd‘ and 200 exact determinants of sympy (without the memory of ‘kdelta‘) on the
same pairs of lists of length 7 and checks that the rule is more than ten times
faster. The measured times differ from computer to computer and from run to run, so
they are not printed (the printed output of a notebook of this book is the same on
every run); only the result of the comparison is.
```

In [12]:

```
lengths = list(range(1, 10))
leibniz_terms = [math.factorial(p) for p in lengths]  # products in the formula
comparisons = [p * (p - 1) // 2 for p in lengths]  # comparisons of the rule
all_pairs = [8 ** (2 * p) for p in lengths]  # all pairs of lists of length p
for p, terms, compare, total in zip(lengths, leibniz_terms, comparisons, all_pairs):
    say(f"p = {p}: Leibniz products {terms:6d}, comparisons of GKD {compare:2d}, "
        f"pairs of lists {total:.3e}")

fig, ax = plt.subplots(figsize=(7.4, 4.4))
ax.semilogy(lengths, leibniz_terms, "o-", label="products in the Leibniz formula, $p!$")
ax.semilogy(lengths, [max(c, 0.5) for c in comparisons], "s-",
            label="comparisons made by GKD, $p(p-1)/2$")
ax.semilogy(lengths, all_pairs, "^:", label="all pairs of index lists, $8^{2p}$")
ax.axvline(4.5, color="0.5", lw=0.8)  # where exhaustive testing ends
ax.text(4.65, 3e15, "right of this line:\nrandom samples only", fontsize=8,
        va="top")
ax.set_xlabel("length $p$ of the index lists")
ax.set_ylabel("count (logarithmic scale)")
ax.set_title("The work of the literal determinant and of the rule GKD")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "cost_of_the_determinant",
            "Work per evaluation of the generalized Kronecker delta for index lists "
            "of length $p$: the Leibniz formula of the determinant adds $p!$ "
            "products (circles), the rule GKD makes $p(p-1)/2$ comparisons "
            "(squares; drawn at 0.5 for $p = 1$, where it makes none); the "
            "triangles count all $8^{2p}$ pairs of index lists over eight labels. "
            "The vertical axis is logarithmic. For $p = 9$ the formula needs "
            "362,880 products against 36 comparisons, and the number of pairs "
```

```
            "grows so fast that only $p \\le 4$ can be tested exhaustively.")

timing_rng = np.random.default_rng(7)
timing_pairs = []
for _ in range(200):
    upper = [int(v) for v in timing_rng.permutation(8)[:7]]  # 7 different labels
    lower = [upper[i] for i in timing_rng.permutation(7)]  # the same, rearranged
    timing_pairs.append((lower, upper))
start = time.perf_counter()
rule_values = [gkd(lower, upper) for lower, upper in timing_pairs]
rule_seconds = time.perf_counter() - start
start = time.perf_counter()
determinant_values = [int(sp.Matrix(outer_delta(lower, upper)).det())
                      for lower, upper in timing_pairs]  # no memory: all computed
determinant_seconds = time.perf_counter() - start
check(rule_values == determinant_values,
      "200 pairs of length 7: the rule and sympy's determinant agree")
check(determinant_seconds > 10 * rule_seconds,
      "the rule GKD is more than ten times faster than the determinant (p = 7)")
```

Out [12]:

```
p = 1: Leibniz products      1, comparisons of GKD  0, pairs of lists 6.400e+01
p = 2: Leibniz products      2, comparisons of GKD  1, pairs of lists 4.096e+03
p = 3: Leibniz products      6, comparisons of GKD  3, pairs of lists 2.621e+05
p = 4: Leibniz products     24, comparisons of GKD  6, pairs of lists 1.678e+07
p = 5: Leibniz products    120, comparisons of GKD 10, pairs of lists 1.074e+09
p = 6: Leibniz products    720, comparisons of GKD 15, pairs of lists 6.872e+10
p = 7: Leibniz products   5040, comparisons of GKD 21, pairs of lists 4.398e+12
p = 8: Leibniz products  40320, comparisons of GKD 28, pairs of lists 2.815e+14
p = 9: Leibniz products 362880, comparisons of GKD 36, pairs of lists 1.801e+16
```

Out [12]:

```
Figure 11a.5 saved as Revision/textbook/figures/11a_5_cost_of_the_determinant.png
PASS 200 pairs of length 7: the rule and sympy's determinant agree
PASS the rule GKD is more than ten times faster than the determinant (p = 7)
```

Text cell:

```
## 10. The Rust program: the self-test of GKD

The Revision computes the Lovelock tensors with the Rust program `lovelock_gkd`
(crate `Revision/gkd_lovelock/code`). Its function GKD is the rule of section 6, and
its function `kdelta_det` is the literal Leibniz determinant of
Outer[delta, lower, upper] (it runs through all $p!$ permutations with Heap's
method, which changes one permutation into the next by one exchange). Its command
`gkd-selftest` compares the two:

- for $p = 1, 2, 3, 4$ (the option `--exhaustive-max 4`), EVERY pair of lists of
  length $p$ over the 8 labels: $64 + 4096 + 262144 + 16777216$ pairs;
```

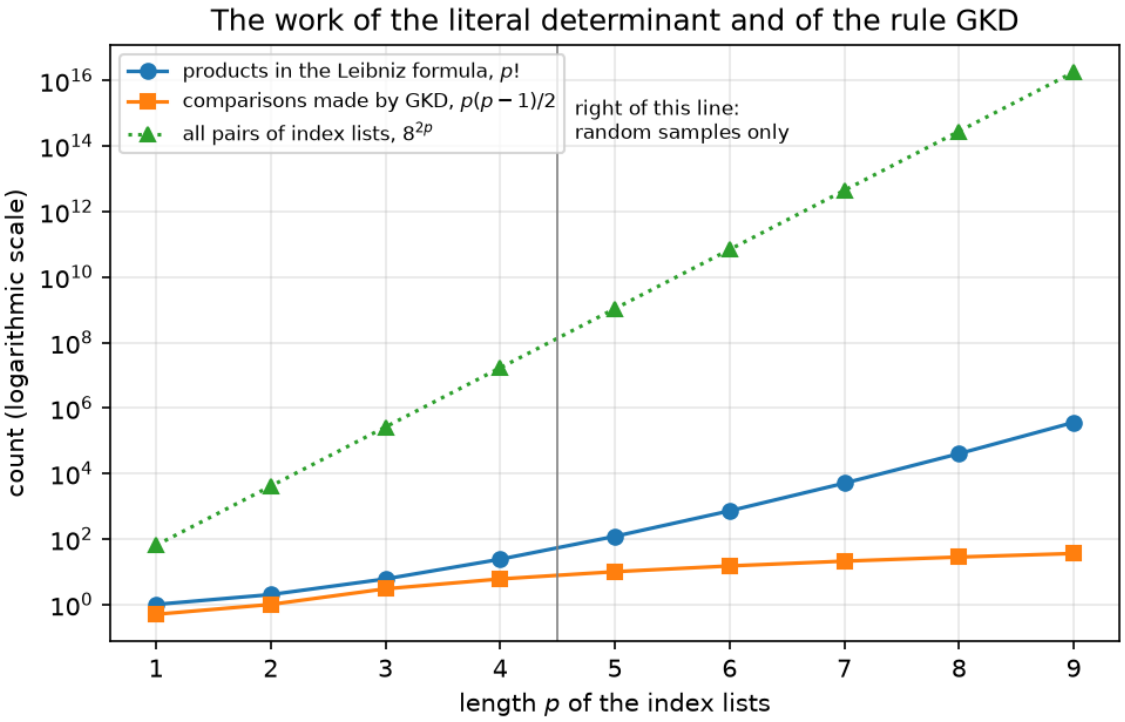

Figure 307: Work per evaluation of the generalized Kronecker delta for index lists of length p: the Leibniz formula of the determinant adds $p!$ products (circles), the rule GKD makes $p(p-1)/2$ comparisons (squares; drawn at 0.5 for $p = 1$, where it makes none); the triangles count all 8^{2p} pairs of index lists over eight labels. The vertical axis is logarithmic. For $p = 9$ the formula needs 362,880 products against 36 comparisons, and the number of pairs grows so fast that only $p \leq 4$ can be tested exhaustively. (Notebook 11a, figure 5.)

```
- for $p = 5, \dots, 9$, 200,000 pseudo-random pairs each from its own fixed
  generator (xorshift64*), half of them rearrangements (Fisher-Yates shuffle) and
  half independent lists, as in section 8.

It writes the result to the file ‘gkd-selftest.json‘ in the folder given by
‘--output‘, without any time in it, so the file is the same on every run. The
committed Revision record ‘Revision/gkd_lovelock/results/gkd-selftest.json‘ was
written by the same command. The run takes 7 to 13 minutes, most of it in the
200,000 literal determinants of $9 \times 9$ matrices: longer than the limit of 7
minutes that this book sets for one computation. So this notebook builds the
program but does not run the self-test; it reads the record.

The next cell builds the program with cargo, through the helper ‘rust_program‘ of the
set-up cell, which runs ‘cargo build --release‘ for the crate and returns the path of
the program file (‘lovelock_gkd.exe‘ on Windows). When the program file is missing,
cargo compiles it: on the computer that built the book this took 4 seconds; a slow
computer may need up to a minute. When the program is up to date, cargo only checks
that and needs about a second. The crate has no dependencies, so cargo downloads
nothing.
```

In [13]:

```
program = rust_program("Revision/gkd_lovelock/code/Cargo.toml", "lovelock_gkd")
COMMAND = ["lovelock_gkd", "gkd-selftest", "--exhaustive-max", "4", "--output", "FOLDER"]
say("To repeat the self-test (7 to 13 minutes), run in the folder of the program: "
    + " ".join(COMMAND))
```

Out [13]:

```
Rust program lovelock_gkd is built and ready.
To repeat the self-test (7 to 13 minutes), run in the folder of the program: lovelock_gkd
    gkd-selftest --exhaustive-max 4 --output FOLDER
```

Text cell:

```
The next cell reads the record ‘Revision/gkd_lovelock/results/gkd-selftest.json‘ of
the run of ‘lovelock_gkd gkd-selftest --exhaustive-max 4 --output FOLDER‘ (the cell
before prints this command). Its entry ‘results‘ holds one row per length: the mode
(‘exhaustive‘ or ‘random‘), the number of pairs, for the random lengths the number
of nonzero values, and the number of disagreements between GKD and the literal
determinant. The cell prints the rows as a table and checks the verdict SUCCESS of
the record, that the rows cover the lengths 1 to 9 with no disagreement and that
the length 4 was tested exhaustively on all 16,777,216 pairs, and that the program
file that repeats the run is built. To repeat the run yourself, run the printed
command in a terminal, in the folder ‘Revision/gkd_lovelock/code/target/release‘,
with a folder of your choice in place of FOLDER: it writes ‘gkd-selftest.json‘ into
that folder, and the file must equal the record byte for byte.
```

In [14]:

```
SELFTEST = "Revision/gkd_lovelock/results/gkd-selftest.json"
written = repository_file(SELFTEST).read_bytes()  # the record of the Rust run
selftest_record = json.loads(written.decode("utf-8"))
say("The record of the Rust self-test (as a table):")
rust_rows = {}  # p -> (mode, pairs, nonzero or None, mismatches)
for row in selftest_record["results"]:
    rust_rows[row["p"]] = (row["mode"], row["pairs"], row.get("nonzero"),
                           row["mismatches"])
for p, (mode, pairs, nonzero, bad) in sorted(rust_rows.items()):
    extra = "" if nonzero is None else f", {nonzero:6d} nonzero"
    say(f"  p = {p}: {mode:10s} {pairs:9d} pairs{extra}; disagreements: {bad}")
check(selftest_record["program"] == "lovelock_gkd gkd-selftest"
      and selftest_record["verdict"] == "SUCCESS",
      "the record of the Rust program lovelock_gkd gkd-selftest has the verdict "
      "SUCCESS", record=SELFTEST)
check(sorted(rust_rows) == list(range(1, 10))
      and all(row[3] == 0 for row in rust_rows.values())
      and rust_rows[4][:2] == ("exhaustive", 16777216),
      "Rust: GKD equals the literal determinant for every tested pair, p = 1 to 9",
      record=SELFTEST)
check(program.is_file(), "the program lovelock_gkd that repeats the self-test is built")
```

Out [14]:

```
The record of the Rust self-test (as a table):
  p = 1: exhaustive        64 pairs; disagreements: 0
  p = 2: exhaustive      4096 pairs; disagreements: 0
  p = 3: exhaustive    262144 pairs; disagreements: 0
  p = 4: exhaustive  16777216 pairs; disagreements: 0
  p = 5: random        200000 pairs,  20538 nonzero; disagreements: 0
  p = 6: random        200000 pairs,   7812 nonzero; disagreements: 0
  p = 7: random        200000 pairs,   1949 nonzero; disagreements: 0
  p = 8: random        200000 pairs,    244 nonzero; disagreements: 0
  p = 9: random        200000 pairs,      0 nonzero; disagreements: 0
PASS the record of the Rust program lovelock_gkd gkd-selftest has the verdict SUCCESS
     reproduces Revision/gkd_lovelock/results/gkd-selftest.json
PASS Rust: GKD equals the literal determinant for every tested pair, p = 1 to 9
     reproduces Revision/gkd_lovelock/results/gkd-selftest.json
PASS the program lovelock_gkd that repeats the self-test is built
```

Text cell:

```
The record just read is now read as JSON (a text format of names and numbers),
and its numbers are compared with the birthday formula of section 8. In the
program's random samples one half are rearrangements and one half independent lists,
100,000 each, so the expected number of nonzero values of length $p$ is
$100000\,(r_p + q_p)$, with the standard deviation of section 8. The next cell checks
that every count lies within 5 standard deviations of its expectation, and that the
count of length 9 is exactly 0. Then it draws the chance $r_p$ that $p$ random
labels out of 8 are all different, with the measured fractions: the number of
nonzero values found among ALL the samples of one length, divided by the number of
rearranged samples (1,000 in the Python test of section 8, 100,000 in the Rust
```

test). This fraction should be close to $r_p + q_p$; the independent pairs add the
small chance q_p, which is at most 0.0025 (for $p = 4$, $q_4 = 40320/8^8$) and
invisible on the logarithmic axis.

In [15]:

```
selftest = json.loads(written.decode("utf-8"))
rust_nonzero = {row["p"]: row["nonzero"] for row in selftest["results"]
                if row["mode"] == "random"}
agree = True
for p, nonzero in sorted(rust_nonzero.items()):
    mean, spread = expected_nonzero(p, 100000)
    distance = abs(nonzero - mean) / spread if spread > 0 else 0.0
    say(f"Rust p = {p}: counted {nonzero:6d}, expected {mean:8.1f} +- {spread:6.1f}"
        f"  ({distance:.2f} standard deviations away)")
    agree = agree and (distance < 5.0 if spread > 0 else nonzero == 0)
check(selftest["verdict"] == "SUCCESS" and agree and rust_nonzero[9] == 0,
      "the Rust counts of nonzero values agree with the birthday formula")

lengths_1_to_8 = np.arange(1, 9)  # r_p is drawn where it is not zero
fig, ax = plt.subplots(figsize=(7.4, 4.4))
ax.semilogy(lengths_1_to_8, [all_different(p) for p in lengths_1_to_8], "k-",
            label="chance that $p$ labels out of 8 are all different, $r_p$")
python_points = [p for p in python_nonzero if python_nonzero[p] > 0]
ax.semilogy(python_points, [python_nonzero[p] / (SAMPLES // 2)
                            for p in python_points], "o",
            label="Python: nonzero values / 1000 rearranged pairs")
rust_points = [p for p in rust_nonzero if rust_nonzero[p] > 0]
ax.semilogy(rust_points, [rust_nonzero[p] / 100000 for p in rust_points], "s",
            fillstyle="none", markersize=10,
            label="Rust: nonzero values / 100000 rearranged pairs")
ax.plot([9], [1e-6], "v", color="black", markersize=9)  # marks the zeros of p = 9
ax.text(9.0, 2.2e-6, "p = 9: none\n(0 of 1000,\n0 of 100000)", fontsize=8,
        ha="center", va="bottom")
ax.axvline(8.5, color="0.5", lw=0.8)
ax.text(8.6, 0.3, "more than 8\nlabels: always\na repetition", fontsize=8, va="top")
ax.set_xlim(0.5, 10.0)
ax.set_ylim(5e-7, 2.0)
ax.set_xlabel("length $p$ of the index lists")
ax.set_ylabel("fraction (logarithmic scale)")
ax.set_title("Nonzero deltas among rearranged random pairs")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "all_labels_different",
            "The chance $r_p = 8!/((8 - p)!\\, 8^p)$ that $p$ random labels out of "
            "eight are all different (line), which is the chance that a rearranged "
            "random pair of index lists has a nonzero generalized Kronecker delta, "
            "with the measured fractions: the nonzero values found among all the "
            "samples of one length divided by the number of rearranged samples, "
            "for the Python samples of this notebook (dots) and the Rust self-test "
            "(open squares); the independent samples add at most 0.0025. The "
            "vertical axis is logarithmic. The measured points follow the formula. "
            "For $p = 9$ no list can have nine "
            "different labels, so the chance and both measured fractions are 0, "
```

```
        "which a logarithmic axis cannot show; the black triangle at the "
        "bottom edge marks them.")
```

Out [15]:

```
Rust p = 5: counted  20538, expected  20582.9 +-  128.0  (0.35 standard deviations away)
Rust p = 6: counted   7812, expected   7711.6 +-   84.4  (1.19 standard deviations away)
Rust p = 7: counted   1949, expected   1927.2 +-   43.5  (0.50 standard deviations away)
Rust p = 8: counted    244, expected    240.9 +-   15.5  (0.20 standard deviations away)
Rust p = 9: counted      0, expected      0.0 +-    0.0  (0.00 standard deviations away)
PASS the Rust counts of nonzero values agree with the birthday formula
```

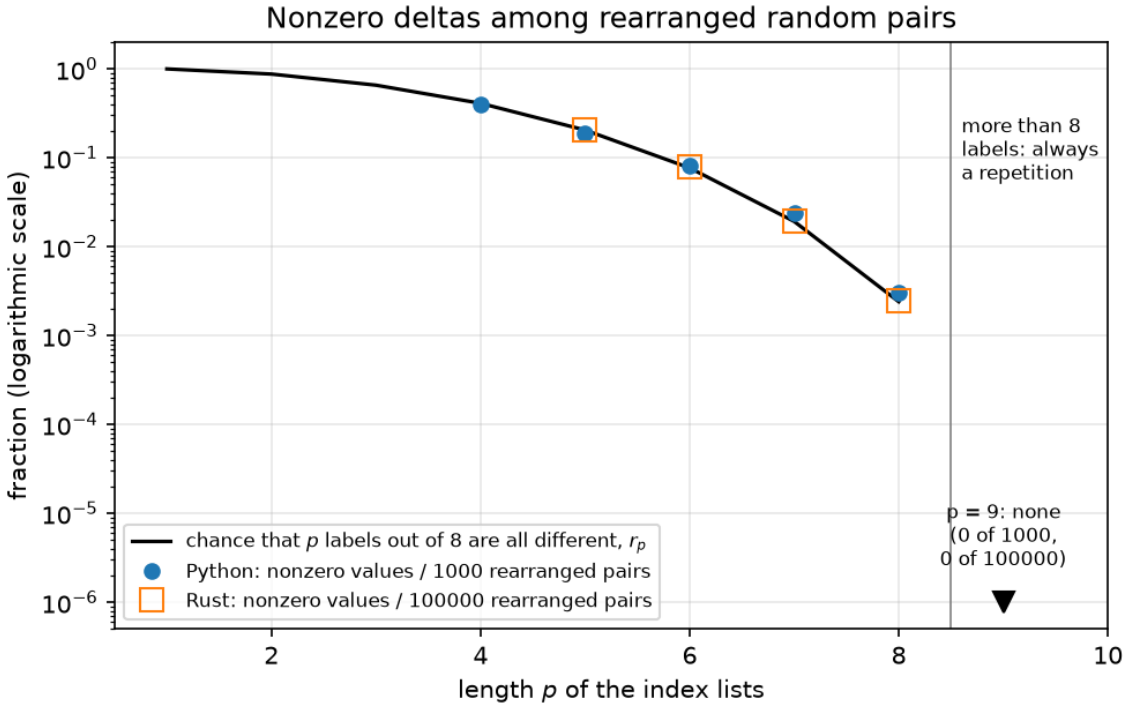

Figure 308: The chance $r_p = 8!/((8-p)!\,8^p)$ that p random labels out of eight are all different (line), which is the chance that a rearranged random pair of index lists has a nonzero generalized Kronecker delta, with the measured fractions: the nonzero values found among all the samples of one length divided by the number of rearranged samples, for the Python samples of this notebook (dots) and the Rust self-test (open squares); the independent samples add at most 0.0025. The vertical axis is logarithmic. The measured points follow the formula. For $p = 9$ no list can have nine different labels, so the chance and both measured fractions are 0, which a logarithmic axis cannot show; the black triangle at the bottom edge marks them. (Notebook 11a, figure 6.)

Out [15]:

```
Figure 11a.6 saved as Revision/textbook/figures/11a_6_all_labels_different.png
```

Text cell:

11. The Revision records quoted in this notebook

This notebook quotes check names and numbers from five Revision records. Two of them were already compared in full: the self-test record ‘gkd-selftest.json‘ (byte for byte, section 10) and the value counts of ‘wolfram-gkd-report.json‘ (section 7). The next cell reads the records once more and checks the rest of what the text quotes, so that a later change of a record cannot pass unnoticed:

- every check quoted from the sympy verification ‘python-lovelock-report.json‘ and from the Wolfram verification ‘wolfram-gkd-report.json‘ is present in the record with the verdict PASS;
- the record ‘PROVENANCE_OF_THE_COMPUTATION.md‘ contains the author’s definition as quoted in section 4 (the part after the name kδ, which is written with a Greek letter there);
- the counter ‘gkdCalls‘ of the order-3 sum in ‘lovelock-report.json‘ is 495,360, the number quoted in section 4.

The function ‘verdicts(path)‘ returns, for one report, a dictionary from the name of each check to its verdict. The reports store their checks in one of two forms: a list of entries, each with a ‘name‘ and a ‘verdict‘, or a dictionary from the name to an entry whose field ‘passed‘ is true or false; the function reads both.

In [16]:

```
def verdicts(path):
    """{check name: "PASS" or "FAIL"} of the Revision report at ‘path‘."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    checks = data["checks"]
    if isinstance(checks, dict):  # name -> {"passed": true or false, ...}
        return {name: "PASS" if entry["passed"] else "FAIL"
                for name, entry in checks.items()}
    return {entry["name"]: entry["verdict"] for entry in checks}  # a list of entries


RECORDS = "Revision/gkd_lovelock/results"  # the folder of the Revision records
QUOTED = {  # report -> the checks that this notebook quotes from it
    "python-lovelock-report.json": [
        "gkd_examples", "gkd_literal_equals_cofactor_expansion",
        "gkd_nine_indices_in_eight_dimensions_vanish"],
    "wolfram-gkd-report.json": [
        "gkd_equals_kdelta_exhaustive_length_1",
        "gkd_equals_kdelta_exhaustive_length_2",
        "gkd_equals_kdelta_exhaustive_length_3"],
}
for report_name, names in QUOTED.items():
    found = verdicts(f"{RECORDS}/{report_name}")
    for name in names:
        say(f"{report_name}: {name} {found.get(name, 'MISSING')}")
    check(all(found.get(name) == "PASS" for name in names),
          f"the {len(names)} checks quoted from {report_name} are there and PASS",
          record=f"{RECORDS}/{report_name}, checks " + ", ".join(names))
provenance = repository_file(
    f"{RECORDS}/PROVENANCE_OF_THE_COMPUTATION.md").read_text(encoding="utf-8")
```

```
definition = ("[lower_, upper_] /; Length[lower] == Length[upper] := "
              "Det[Outer[delta, lower, upper]]")  # after the name of the delta
check(definition in provenance,
      "the record quotes the author's definition as section 4 of this notebook "
      "does",
      record=f"{RECORDS}/PROVENANCE_OF_THE_COMPUTATION.md")
order_three = [row for row in json.loads(repository_file(
    f"{RECORDS}/lovelock-report.json").read_text(encoding="utf-8"))["counters"]
    if row["k"] == 3][0]
report("GKD calls of the order-3 Lovelock sum (record)", order_three["gkdCalls"])
check(order_three["gkdCalls"] == 495360,
      "the order-3 Lovelock sum calls GKD 495360 times, as section 4 of this "
      "notebook says",
      record=f"{RECORDS}/lovelock-report.json, counters, k = 3, gkdCalls")
```

Out [16]:

```
python-lovelock-report.json: gkd_examples PASS
python-lovelock-report.json: gkd_literal_equals_cofactor_expansion PASS
python-lovelock-report.json: gkd_nine_indices_in_eight_dimensions_vanish PASS
PASS the 3 checks quoted from python-lovelock-report.json are there and PASS
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, checks
    gkd_examples, gkd_literal_equals_cofactor_expansion,
    gkd_nine_indices_in_eight_dimensions_vanish
wolfram-gkd-report.json: gkd_equals_kdelta_exhaustive_length_1 PASS
wolfram-gkd-report.json: gkd_equals_kdelta_exhaustive_length_2 PASS
wolfram-gkd-report.json: gkd_equals_kdelta_exhaustive_length_3 PASS
PASS the 3 checks quoted from wolfram-gkd-report.json are there and PASS
     reproduces Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks
    gkd_equals_kdelta_exhaustive_length_1, gkd_equals_kdelta_exhaustive_length_2,
    gkd_equals_kdelta_exhaustive_length_3
PASS the record quotes the author's definition as section 4 of this notebook does
     reproduces Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md
RESULT GKD calls of the order-3 Lovelock sum (record) = 495360
PASS the order-3 Lovelock sum calls GKD 495360 times, as section 4 of this notebook says
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, counters, k = 3,
    gkdCalls
```

Text cell:

```
The last cell checks that all six figure files exist in the folder
‘Revision/textbook/figures‘ and prints the number of checks that passed.
```

In [17]:

```
figure_files = [f"11a_{k}_{name}.png" for k, name in enumerate(
    ["outer_delta_matrices", "six_permutations", "gkd_map_length_two",
     "value_counts", "cost_of_the_determinant", "all_labels_different"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_files),
      "all six figure files of the notebook exist")
all_checks_passed()
```

Out [17]:

```
PASS all six figure files of the notebook exist
ALL 24 CHECKS PASSED (notebook 11a)
```

Text cell:

```
## 12. What this notebook showed

- The author's generalized Kronecker delta, the determinant of the 0/1 matrix
  Outer[delta, lower, upper], is $+1$, $-1$ or $0$: it is zero when a label repeats
  in either list or a lower label is missing above, and otherwise it is the sign of
  the permutation that carries the lower list into the upper list (PROVED in
  section 4).
- The Python rule `gkd` and the literal determinant agree on all 266,304 pairs of
  lengths 1 to 3 and on 12,000 random pairs of lengths 4 to 9 (COMPUTED). The counts
  of the values (8; 56 and 56; 1008 and 1008 nonzero values for $p = 1, 2, 3$) follow
  the formula $8!/(8 - p)! \cdot p!$ and equal those of the Revision's Wolfram
  verification.
- Every delta with 9 or more indices vanishes in eight dimensions, so the Lovelock
  tensor of order 4 is zero and Lovelock's sum stops at order 3.
- The rule costs $p(p-1)/2$ comparisons instead of $p!$ products and is measured to
  be more than ten times faster already for $p = 7$.
- The Revision Rust program `lovelock_gkd`, built here with cargo, compared GKD with
  the literal determinant on 16,777,216 pairs of length 4 and 1,000,000 random
  pairs of lengths 5 to 9 without a single disagreement, and wrote the Revision
  record `gkd-selftest.json` again, byte for byte.
- Every check and number that the notebook quotes from the Revision records (the
  sympy and Wolfram verifications, the record of the author's definition and the
  counter of the order-3 Lovelock sum) is present in them, and every quoted check
  has the verdict PASS.
```

11.9 Line-by-line walk-through of Notebook 11a

The notebook has 17 code cells, In [1] to In [17]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted, then explained. A line that starts with `#`, and the part of a line after `#`, is a **comment**, which Python skips; it is there for the reader. Inside a function the text in triple quotes below the `def` line is its **docstring**, a description that Python stores but does not run; to keep the quotations short, docstrings are left out of them where the explanation repeats them, and a long figure caption passed to `save_figure` is shortened to `...` (the full caption is printed under the figure in Section 11.8). Because Notebook 11a is the first notebook of this chapter, its set-up cell is explained here in full; the walk-throughs of Notebooks 11b and 11c refer back to this explanation.

In [1], the set-up cell. Its first part, down to the heading THE SET-UP between two lines of `=` signs, consists of comment lines that repeat the complete run instructions of Section 11.7, so that the notebook file carries its own instructions. The code below the heading computes no physics; it is the same in every notebook of the book except for the line that names the notebook and, in notebooks without Rust, the two lines `import shutil` and `import subprocess` and the helper `rust_program`, which such notebooks lack (all three are explained below, the helper at the end).

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module**, a part of Python or of an installed package, so that the code can use its functions. `json`, `os`, `textwrap` and `pathlib` come with Python. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs
```

These load the plotting package matplotlib, its drawing functions under the short name `plt`, the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell, and the two modules that a notebook with Rust needs: `shutil` can look for a program on the computer, and `subprocess` can start another program and collect what it prints.

```
NOTEBOOK_ID = "11a"  # this notebook: chapter 11, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"11a"`; the figures and the captions file are named after it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs each time it is called. `Path.cwd()` is the folder in which Jupyter runs the notebook (the folder of the notebook file), and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it, and `[here, *here.parents]` is the list that starts with `here` and continues with them (the star unpacks one list into another). The `for` loop visits these folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains the file `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If no folder does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
REPO = find_repository_root()
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

(The comment lines above each of these two lines are left out.) The first line calls the function and names the result `REPO`. It is never printed, because it differs from computer to computer and the printed output of a notebook must not. The second line chooses where files are written: `os.environ` holds the **environment variables** of the running program (named texts it receives from the computer), and `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and `str(REPO)` otherwise. When you run the notebook the variable is not set, so files go into the repository; the book's checking tool sets it to a scratch folder, so that a check run changes nothing in the repository.

```
def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small helpers (docstrings left out). `repository_file` gives the full path of a repository file, for READING a Revision record. `output_file` gives the full path at which to WRITE a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the file's folder and any missing folder above it, and does nothing if they exist.

```
def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of the book: `str(text)` turns any value into text, and `textwrap.fill` breaks it at blanks and starts every line after the first with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns matplotlib to its built-in settings, so the figures are the same on every computer. `plt.rcParams.update({...})` then sets the size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid (`grid.alpha` 0.3 means 30 per cent opaque). The braces make a **dictionary**, a collection of pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: a name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/11a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The two comment lines explain the last line, which writes an empty JSON dictionary and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored as bytes and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

(Docstring and three comment lines left out.) `setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far: the figures are numbered 1, 2, 3, and a cell run twice keeps its number. The file name joins the notebook id, the number and the name, for example `11a_1_outer_delta_matrices.png`. `fig.savefig` writes a PNG picture file with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time. The caption is stored, the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys), `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list**, an ordered collection in square brackets. `check` is behind every check of the book: if `condition` is false, `raise AssertionError(...)` stops the notebook with an error that names the check (an `if` is used instead of the statement `assert`, because Python started with its optimising option skips every `assert`); otherwise the name is appended to `PASSED` and the line PASS name is printed, followed by a second line naming the Revision record when `record` is given. `record=None` makes that argument optional.

```
def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

`report` prints a key number as a line RESULT label = value, with the unit when one is given (`a if c else b` is `a` when `c` is true and `b` otherwise; an empty string counts as false). `all_checks_passed` prints the last line of the notebook with the number of checks that passed; `len` is the length of a list.

```
def rust_program(manifest, binary):
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
```

The helper that builds a Rust program (docstring left out). `shutil.which("cargo")` searches the folders where the computer looks for programs and returns `None` (Python's "nothing") when cargo is not installed; the notebook then stops with a message that says how to install it. `crate` is the folder of the crate, the folder that holds the file `Cargo.toml` named by `manifest`.

```
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
```

`subprocess.run([...])` starts the program cargo with the words of the list as its command line, waits until it ends and returns a record of the run, here named `completed`. The command is the build command of the run instructions: build the crate in the release (optimised) mode and put the result into the folder `target` next to `Cargo.toml`. `capture_output=True` collects what cargo prints instead of showing it; `text=True` with `encoding="utf-8"` turns the collected bytes into text, and `errors="replace"` replaces any byte that is not valid text by a placeholder instead of stopping.

```
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")
```

Every program ends with a **return code**, 0 for success. If cargo failed, the last 3,000 characters of its error output (`stderr[-3000:]`, the slice from 3,000 characters before the end to the end) are

printed and the notebook stops. Otherwise the loop looks for the program file without and with the ending `.exe` (Windows adds it), prints that the program is ready and returns its path. If neither file exists, the notebook stops with a message.

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

The last line prints the cell's only output, Set-up of notebook 11a complete: repository folder found, helpers defined. (Two strings written next to each other inside the parentheses are joined into one.)

In [2], the literal definition.

```
import itertools  # loops over all lists of labels, and over all permutations
import math  # factorials n! and the numbers n!/(n-p)!
import re  # regular expressions: reads numbers out of the texts of the records
import time  # a stopwatch (used inside one check; no time is ever printed)

import numpy as np  # arrays of numbers and pseudo-random numbers
import sympy as sp  # exact algebra: sympy computes determinants exactly
```

Six more modules: `itertools` produces all lists of labels and all permutations; `math` has `math.factorial(p)` $= p!$ and `math.perm(n, p)` $= n!/(n-p)!$; `re` reads numbers out of text with **regular expressions** (patterns of characters); `time` provides a stopwatch; numpy (short name `np`) handles arrays of numbers and makes pseudo-random numbers; sympy (short name `sp`) does exact algebra.

```
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # the labels 0..7 by name
```

The list of the eight coordinate names; `NAMES[0]` is `"x1"` and `NAMES[7]` is `"x0"`, because Python counts places from 0.

```
def outer_delta(lower, upper):
    return [[1 if a == b else 0 for b in upper] for a in lower]
```

The matrix `Outer[delta, lower, upper]` as a list of rows. The inner part `[1 if a == b else 0 for b in upper]` is a **list comprehension**: for each upper label `b` it writes 1 if `b` equals the lower label `a` and 0 otherwise; that is one row. The outer comprehension makes one such row for each lower label `a`. So the entry in row i and column j is $\delta(l_i, u_j)$, exactly the author's matrix.

```
DETERMINANTS = {}  # memory: a matrix (a tuple of rows) -> its determinant


def kdelta(lower, upper):
    if len(lower) != len(upper):
        raise ValueError("kdelta needs two index lists of the same length")
    matrix = tuple(tuple(row) for row in outer_delta(lower, upper))
    if matrix not in DETERMINANTS:  # each different matrix is computed only once
        DETERMINANTS[matrix] = int(sp.Matrix(matrix).det())
    return DETERMINANTS[matrix]
```

The author's definition, literally (docstring left out). Two lists of different lengths are refused with a `ValueError`, as Mathematica leaves the author's expression unevaluated. The matrix is turned into a **tuple** of tuples (a tuple is a list that cannot be changed; only such values can be keys of a dictionary). If this matrix has not been met before, sympy computes its determinant exactly (`sp.Matrix(matrix).det()`), `int` turns the result into a Python whole number, and it is stored in the dictionary `DETERMINANTS`. Many different pairs of lists give the same 0/1 matrix, so this memory saves most of the work.

```
EXAMPLES = [  # (lower list, upper list, value stated in the Revision record)
    ((1, 2), (1, 2), 1),
    ((1, 2), (2, 1), -1),
    ((1, 1), (1, 1), 0),
    ((1, 2, 3), (2, 3, 1), 1),
    ((1, 2, 3), (1, 2, 4), 0),
]
```

The five examples of the record check `gkd_examples` of `Revision/gkd_lovelock/results/python-lovelock-report.json`, each as a lower list, an upper list and the stated value.

```
for lower, upper, stated in EXAMPLES:
    say(f"kdelta[{list(lower)}, {list(upper)}] = {kdelta(lower, upper)}   "
        f"(the record states {stated})")
check(all(kdelta(lower, upper) == stated for lower, upper, stated in EXAMPLES),
      "the five examples give 1, -1, 0, 1, 0",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check gkd_examples")
```

The loop takes the three parts of each example apart (`for lower, upper, stated in ...`) and prints the computed value next to the stated one; these are the first five output lines. `all(...)` is true when every comparison in the parentheses is true; the check prints PASS the five examples give 1, -1, 0, 1, 0 and the line naming the record. Look at the fourth example: the place of the lower label 1 in the upper list $(2, 3, 1)$ is 3, that of 2 is 1, that of 3 is 2; $\sigma = (3, 1, 2)$ has the two inversions $(3, 1)$ and $(3, 2)$, so the value is $+1$, as printed.

```
try:  # lists of different lengths must be refused, as in the author's definition
    kdelta((1,), (1, 2))
    refused = False
except ValueError:
    refused = True
check(refused, "kdelta refuses two lists of different lengths")
```

`try ... except ValueError` runs the line `kdelta((1,), (1, 2))` and catches the `ValueError` it should raise. (`(1,)` is a tuple with the single entry 1.) If the error comes, `refused` is set to `True`; if it did not come, the line after the call would set it to `False` and the check would fail. Output: PASS kdelta refuses two lists of different lengths. Two checks so far.

In [3], three matrices drawn (Figure 11a.1).

```
SHOWN = [  # (lower list, upper list) with labels 0..7 standing for x0..x7
    ((0, 1, 2, 3), (1, 0, 3, 2)),  # x2 x1 x4 x3: two exchanges, even
    ((0, 1, 2, 3), (1, 2, 3, 0)),  # x2 x3 x4 x1: a cycle of four, odd
    ((0, 1, 2, 2), (0, 1, 2, 3)),  # x3 twice in the lower list
]
fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.9))
```

The three pairs of the examples of Section 11.3, written with the numbers 0 to 7 for x_0 to x_7. `plt.subplots(1, 3, ...)` makes a figure `fig` with one row of three panels, the **axes** (drawing areas) `axes[0]`, `axes[1]`, `axes[2]`, 10.5 by 3.9 inches in all.

```
for ax, (lower, upper) in zip(axes, SHOWN):
    matrix = np.array(outer_delta(lower, upper))
    ax.imshow(matrix, cmap="Blues", vmin=0.0, vmax=1.5)  # 0 white, 1 blue
```

`zip(axes, SHOWN)` pairs the first panel with the first pair of lists, and so on. For each, the matrix is built and turned into a numpy array; `ax.imshow` draws it as a picture of coloured squares, one per entry, with the colour map `Blues` running from white at `vmin` $= 0$ to dark blue at `vmax` $= 1.5$ (so that a 1 is a medium blue and the numbers stay readable).

```
    for i in range(4):  # i: the row
        for j in range(4):  # j: the column
            ax.text(j, i, str(matrix[i, j]), ha="center", va="center")
```

`range(4)` is $0, 1, 2, 3$. The two loops write each entry as text at the centre of its square; `ax.text(x, y, ...)` takes the horizontal position first, which is the column `j`, then the row `i`.

```
    ax.set_xticks(range(4), [NAMES[label] for label in upper])  # column labels
    ax.set_yticks(range(4), [NAMES[label] for label in lower])  # row labels
    ax.set_xlabel("upper list (one column per label)")
    ax.set_ylabel("lower list (one row per label)")
```

The columns are marked with the names of the upper labels and the rows with the names of the lower labels; then the two axes get their titles.

```
    value = kdelta(lower, upper)
    ax.set_title(f"determinant = {value:+d}" if value else "determinant = 0")
    ax.grid(False)  # no grid lines across the squares
fig.tight_layout()  # no overlapping labels
```

The exact determinant becomes the panel title; the format `:+d` writes a whole number with its sign, $+1$ or -1, and a value 0 (which counts as false) gives the title determinant $= 0$. The grid of the set-up cell is switched off for these pictures, and `fig.tight_layout()` moves the panels so that no labels overlap.

```
save_figure(fig, "outer_delta_matrices",
            "The matrix Outer of delta for three pairs of index lists of length 4; "
            ...)
check([kdelta(lower, upper) for lower, upper in SHOWN] == [1, -1, 0],
      "the three drawn matrices have the determinants +1, -1 and 0")
```

The figure is saved as `11a_1_outer_delta_matrices.png` with its caption (the output line Figure 11a.1 saved as ...), and the check confirms the three values $+1$, -1, 0 of Section 11.3 (third PASS line).

What Figure 11a.1 shows. Three 4×4 matrices of zeros and ones; rows are the lower labels, columns the upper labels, and blue squares are the ones. In the left and middle panels every row and every column has exactly one 1: these are permutation matrices, with determinant $+1$ (two exchanges) and -1 (three inversions). In the right panel the two rows of x_3 are equal and the column of x_4 is empty, so the determinant is 0, by case (a) and case (c) of the theorem at once.

In [4], the rule GKD in Python.

```
def inversions(sigma):
    return sum(1 for i in range(len(sigma)) for j in range(i + 1, len(sigma))
               if sigma[i] > sigma[j])
```

The number of inversions of a list `sigma` (docstring left out): for every pair of places `i < j` (the inner range starts at `i + 1`) it counts 1 when the entries stand in the wrong order, and `sum` adds the ones.

```
def gkd(lower, upper):
    if len(lower) != len(upper):
        raise ValueError("gkd needs two index lists of the same length")
    position = {}  # label -> its place in the upper list
    for place, label in enumerate(upper):
        if label in position:
            return 0  # a label twice in the upper list: equal columns (case b)
        position[label] = place
```

The rule of Section 11.3, step by step (docstring left out). `enumerate(upper)` gives each upper label together with its place 0, 1, 2, ... A label already in the dictionary `position` has been seen before: two equal columns, value 0. Otherwise its place is stored.

```
    sigma = []  # sigma[i]: the place in the upper list of the i-th lower label
    for label in lower:
        if label not in position:
            return 0  # a lower label missing above: a row of zeros (case c)
        sigma.append(position[label])
    if len(set(sigma)) < len(sigma):
        return 0  # a label twice in the lower list: equal rows (case a)
    return 1 if inversions(sigma) % 2 == 0 else -1  # even +1, odd -1 (case d)
```

For each lower label its place in the upper list is looked up; a label that is not there gives a row of zeros, value 0. A **set** keeps only different entries, so `len(set(sigma)) < len(sigma)` is true when a place occurs twice, which happens exactly when a label occurs twice in the lower list: value 0. Otherwise `sigma` is a permutation; `% 2` is the remainder after division by 2, so an even number of inversions gives $+1$ and an odd number -1.

```
for lower, upper, stated in EXAMPLES:
    say(f"gkd({list(lower)}, {list(upper)}) = {gkd(lower, upper)}")
```

```
check(all(gkd(lower, upper) == kdelta(lower, upper) for lower, upper, _ in EXAMPLES),
      "gkd and kdelta agree on the five examples")
```

The rule is applied to the five examples, which prints the five values 1, −1, 0, 1, 0, and the check confirms that it agrees with the literal determinant on all of them (the name `_` marks the stated value, which is not needed here). Fourth PASS line.

In [5], the six arrangements of three labels (Figure 11a.2).

```
lower = (0, 1, 2)  # x1, x2, x3
fig, axes = plt.subplots(2, 3, figsize=(9.0, 6.4))
signs = []
for ax, upper in zip(axes.flat, itertools.permutations(lower)):
    matrix = np.array(outer_delta(lower, upper))
    sigma = [upper.index(label) for label in lower]  # places of x1, x2, x3 above
    sign = 1 if inversions(sigma) % 2 == 0 else -1
    signs.append((sign, kdelta(lower, upper)))
```

A figure with two rows of three panels; `axes.flat` runs through the six panels one by one. `itertools.permutations(lower)` produces the $3! = 6$ arrangements of $(0, 1, 2)$. For each arrangement used as the upper list, `upper.index(label)` is the place of a label in it, so `sigma` lists the places of x_1, x_2, x_3; its sign comes from the inversions, and the pair (sign, exact determinant) is stored in `signs`.

```
    ax.imshow(matrix, cmap="Greens" if sign > 0 else "Oranges", vmin=0.0, vmax=1.4)
    for i in range(3):
        for j in range(3):
            ax.text(j, i, str(matrix[i, j]), ha="center", va="center")
    ax.set_xticks(range(3), [NAMES[label] for label in upper])
    ax.set_yticks(range(3), [NAMES[label] for label in lower])
```

Each matrix is drawn in green for an even and in orange for an odd arrangement, with its entries written in, its columns marked with the arranged upper labels and its rows with x_1, x_2, x_3.

```
    count = inversions(sigma)  # the number of inversions of sigma
    plural = "" if count == 1 else "s"  # 1 inversion, 2 inversions
    ax.set_title(f"sigma = {tuple(sigma)}, {count} inversion{plural}, "
                 f"sign {sign:+d}", fontsize=9)
    ax.grid(False)
fig.tight_layout()
```

The title of each panel gives σ, its number of inversions (with the right plural) and its sign.

```
save_figure(fig, "six_permutations",
            "All six arrangements of the labels $x_1, x_2, x_3$. Each panel shows "
            ...)
check(all(sign == det for sign, det in signs) and
      sorted(sign for sign, _ in signs) == [-1, -1, -1, 1, 1, 1],
      "the six arrangements of three labels: three even, three odd, sign = determinant")
```

The figure is saved, and the check confirms two things: every sign from the inversions equals the exact determinant, and the sorted list of the six signs is three times -1 and three times $+1$, half even and half odd as Section 11.4 proved. Fifth PASS line.

What Figure 11a.2 shows. Six 3×3 permutation matrices. The identity and the two cyclic arrangements, (x_2, x_3, x_1) and (x_3, x_1, x_2), are green: 0 or 2 inversions, sign $+1$. The three arrangements that exchange two labels are orange: 1 or 3 inversions, sign -1. Each title's sign is the determinant of the panel's matrix.

In [6], every pair of lengths 1, 2 and 3.

```
COUNTS = {}  # p -> {+1: count, -1: count, 0: count}
mismatches = 0  # pairs where gkd and the determinant differ
for p in (1, 2, 3):
    counts = {1: 0, -1: 0, 0: 0}
    for lower in itertools.product(range(8), repeat=p):  # all 8^p lower lists
        for upper in itertools.product(range(8), repeat=p):  # all 8^p upper lists
            value = gkd(lower, upper)
            if value != kdelta(lower, upper):
                mismatches += 1
            counts[value] += 1
    COUNTS[p] = counts
```

`itertools.product(range(8), repeat=p)` produces all 8^p lists of length p with entries 0 to 7. For each length the two nested loops visit all $8^p \cdot 8^p$ pairs, compute the rule, compare it with the literal determinant (counting any disagreement in `mismatches`; `+= 1` adds one) and count how often each value occurs. The counts of each length are stored in `COUNTS`.

```
    say(f"p = {p}: {8 ** (2 * p):6d} pairs; value +1: {counts[1]:4d} times, "
        f"-1: {counts[-1]:4d} times, 0: {counts[0]:6d} times")
```

Inside the loop over `p`, one line per length; `**` is a power, and `:6d` writes a whole number right-aligned in 6 places. Output: 64, 4096 and 262144 pairs, with the counts 8, 0, 56; 56, 56, 3984; and 1008, 1008, 260128, exactly the table of Section 11.4.

```
compared = sum(8 ** (2 * p) for p in (1, 2, 3))
report("pairs compared (all pairs of lengths 1, 2 and 3)", compared)
report("pairs where GKD and the determinant differ", mismatches)
report("different 0/1 matrices whose determinant sympy computed", len(DETERMINANTS))
```

Three RESULT lines: $64 + 4096 + 262144 = 266304$ pairs compared, 0 disagreements, and 145 different 0/1 matrices for which sympy had to compute a determinant; all other pairs reused a stored one.

```
wolfram_record = json.loads(repository_file(
    "Revision/gkd_lovelock/results/wolfram-gkd-report.json").read_text(
        encoding="utf-8"))
entries = [c for c in wolfram_record["checks"]
           if c["name"].startswith("gkd_equals_kdelta_exhaustive_length_")]
```

The Revision's Wolfram report is read: `read_text` reads the file as text and `json.loads` turns the JSON text into Python lists and dictionaries. Its checks form a list of entries; the comprehension keeps the entries whose names begin with `gkd_equals_kdelta_exhaustive_length_` (`startswith` tests how a text begins). There are three of them, for the lengths 1, 2 and 3; in each, the Revision's Wolfram verification compared the values of the Rust function GKD (written out by a small Rust program that calls it) with the author's literal determinant, evaluated by Wolfram, on every pair of lists of that length.

```
recorded_pairs = sum(int(re.search(r"all (\d+) pairs", e["detail"]).group(1))
                     for e in entries)  # the numbers of pairs the record names
```

Each entry's `detail` text names its number of pairs, as in "all 4096 pairs of index lists of length 2". The regular expression `all (\d+) pairs` finds this phrase (`\d+` is one or more digits, and the brackets mark the part to keep), `.group(1)` is the kept text and `int` turns it into a whole number; `sum` adds the three numbers: $64 + 4096 + 262144$.

```
check(mismatches == 0 and compared == 266304 and len(entries) == 3
      and all(e["verdict"] == "PASS" and "(0 mismatches;" in e["detail"]
              for e in entries) and recorded_pairs == 266304,
      "GKD equals the literal determinant for all 266304 pairs of lengths 1 to 3",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, checks "
             "gkd_equals_kdelta_exhaustive_length_1 to _3")
```

The check requires no disagreement among the notebook's own 266,304 pairs, and that the record's three checks are there, have the verdict PASS, report 0 mismatches and together speak of the same 266,304 pairs (`in` tests whether one text occurs inside another). Sixth PASS line. (The Revision's sympy report `python-lovelock-report.json` also has a check on these 266,304 pairs, `gkd_literal_equals_cofactor_expansion`, but it compares something else: the literal determinant with a second, independent way of computing a determinant, the cofactor expansion (Laplace's rule, proved in Section 11.13: the determinant is the sum of the entries of one row, each times the smaller determinant left when its row and column are removed, with alternating signs). It confirms the determinant side, not GKD, and is therefore not the record of this check; In [16] only confirms that it is there and passed.)

In [7], the counts against the formula and the Wolfram record.

```
def nonzero_pairs(p, n=8):
    return math.perm(n, p) * math.factorial(p)


def formula_counts(p):
    nonzero = nonzero_pairs(p)
    plus = nonzero if p == 1 else nonzero // 2  # p = 1: only the identity
    return {1: plus, -1: nonzero - plus, 0: 8 ** (2 * p) - nonzero}
```

`nonzero_pairs` is the formula $N_{\neq 0}(p) = 8!/(8-p)! \cdot p!$ of Section 11.4 (`n=8` gives the argument `n` the default value 8). `formula_counts` splits it: for $p = 1$ every nonzero value is $+1$, otherwise half of them (`//` is whole-number division); the zeros are the rest of the 8^{2p} pairs.

```
wolfram = json.loads(repository_file(
    "Revision/gkd_lovelock/results/wolfram-gkd-report.json").read_text(
        encoding="utf-8"))
for p in (1, 2, 3):
    recorded = [m for m in wolfram["measurements"]["gkdComparison"] if m["p"] == p][0]
    plus, minus, zero = recorded["plusOne"], recorded["minusOne"], recorded["zero"]
    say(f"p = {p}: formula {formula_counts(p)}, Wolfram record +1: {plus}, "
        f"-1: {minus}, 0: {zero}")
```

The Revision's Wolfram report is read; for each length the entry of its table `gkdComparison` with that `p` is found, its three counts are taken out, and the formula and the record are printed side by side (three output lines, the third broken by `say`).

```
check(all(COUNTS[p] == formula_counts(p) for p in (1, 2, 3)),
      "the counts of +1, -1 and 0 equal the formula for p = 1, 2, 3")
check(all(COUNTS[p] == {1: m["plusOne"], -1: m["minusOne"], 0: m["zero"]}
          for p in (1, 2, 3)
          for m in wolfram["measurements"]["gkdComparison"] if m["p"] == p),
      "the counts equal those of the Wolfram verification",
      record="Revision/gkd_lovelock/results/wolfram-gkd-report.json, measurements "
             "gkdComparison, p = 1, 2, 3")
```

Two checks: the measured counts equal the formula, and they equal the record's counts (two dictionaries are equal when they have the same keys with the same values). PASS lines seven and eight.

In [8], the map of all pairs of length 2 (Figure 11a.3).

```
pairs_of_labels = list(itertools.product(range(8), repeat=2))  # 64 lists (l1, l2)
grid = np.array([[gkd(lower, upper) for upper in pairs_of_labels]
                 for lower in pairs_of_labels])  # 64 x 64 values
```

The 64 lists of length 2 in the order $(0,0),(0,1),\ldots,(7,7)$, so the list (l_1, l_2) has the number $8l_1 + l_2$. `grid` holds the value of GKD for every pair: row = lower list, column = upper list.

```
fig, ax = plt.subplots(figsize=(6.6, 6.0))
picture = ax.imshow(grid, cmap="bwr", vmin=-1, vmax=1)  # blue -1, red +1
ticks = range(0, 64, 8)  # one tick at the start of each block of 8
ax.set_xticks(ticks, [f"({NAMES[t // 8]},x1)" for t in ticks], rotation=90)
ax.set_yticks(ticks, [f"({NAMES[t // 8]},x1)" for t in ticks])
```

The values are drawn with the colour map `bwr` (blue, white, red) from -1 (blue) through 0 (white) to $+1$ (red). `range(0, 64, 8)` is $0, 8, 16, \ldots, 56$, the first list of each block of eight; `t // 8` is the first label of that block, so the marks read (x_1, x_1), (x_2, x_1), and so on (rotated by 90 degrees on the horizontal axis).

```
ax.set_xlabel("upper list $(u_1, u_2)$, numbered $8 u_1 + u_2$")
ax.set_ylabel("lower list $(l_1, l_2)$, numbered $8 l_1 + l_2$")
ax.set_title("GKD for all 4096 pairs of index lists of length 2")
ax.grid(False)
fig.colorbar(picture, ax=ax, ticks=[-1, 0, 1], shrink=0.8, label="value of GKD")
```

Axis titles (matplotlib draws text between dollar signs as mathematics), the title, no grid, and a colour bar beside the picture that shows which colour means which value.

```
save_figure(fig, "gkd_map_length_two",
            "The value of the generalized Kronecker delta for all 4096 pairs of "
            ...)
check(int((grid == 1).sum()) == 56 and int((grid == -1).sum()) == 56
      and np.array_equal(grid, grid.T),
      "length 2: 56 values +1, 56 values -1, and the picture is symmetric")
```

After saving, the check counts the entries equal to 1 and to -1 (`grid == 1` is an array of true and false, and `.sum()` counts the trues) and tests that the picture equals its mirror image across the diagonal (`grid.T` is the transpose). The symmetry holds because the pair (lower, upper) and the pair (upper, lower) have transposed matrices and therefore the same determinant. Ninth PASS line.

What Figure 11a.3 shows. A 64×64 picture, almost all white. Red squares lie on the diagonal (upper list equal to the lower list, $+1$) except at the 8 lists with two equal labels, which are white. Blue squares lie at the mirrored places, where the upper list is the lower one reversed (-1). There are 56 of each; the remaining 3984 values are 0.

In [9], the value counts (Figure 11a.4).

```
fig, ax = plt.subplots(figsize=(7.4, 4.4))
width = 0.26  # the width of one bar
for shift, value, colour, name in ((-width, 1, "tab:red", "value +1"),
                                   (0.0, -1, "tab:blue", "value -1"),
                                   (width, 0, "0.6", "value 0")):
```

A bar chart with three bars per length: the loop runs over the three values, each with its sideways shift, its colour (`"0.6"` is a grey) and its name in the legend.

```
    heights = [COUNTS[p][value] if p < 4 else formula_counts(4)[value]
               for p in (1, 2, 3, 4)]
    bars = ax.bar([p + shift for p in (1, 2, 3, 4)], heights, width, color=colour,
                  label=name, hatch=None)
    bars[3].set_hatch("//")  # p = 4: from the formula
```

The heights are the measured counts for $p = 1, 2, 3$ and the formula for $p = 4$. `ax.bar` draws the four bars of this value; the fourth bar (place 3, counted from 0) gets a hatching to mark that it comes from the formula.

```
    for p, height in zip((1, 2, 3, 4), heights):
        ax.text(p + shift, max(height, 1.2) * 1.15, f"{height:,}" if height else
                "none", ha="center", va="bottom", fontsize=7, rotation=90)
```

Each bar gets its count written above it (the format `:,` puts commas between groups of three digits); a count of 0 cannot be drawn on a logarithmic axis, so it is written as "none" just above the bottom.

```
ax.set_yscale("log")  # a logarithmic vertical axis
ax.set_ylim(1.0, 2e9)
ax.set_xticks([1, 2, 3, 4], ["p = 1", "p = 2", "p = 3", "p = 4 (formula)"])
ax.set_xlabel("length $p$ of the two index lists")
ax.set_ylabel("number of pairs (logarithmic scale)")
ax.set_title("How often GKD is +1, -1 and 0 among all $8^{2p}$ pairs")
ax.legend(loc="upper left")
```

On a **logarithmic axis** each equal step is a factor of 10, so the counts 8 and 16,736,896 fit into one picture. The axis runs from 1 to 2×10^9; the remaining lines set the marks, titles and legend.

```
save_figure(fig, "value_counts",
            "The number of pairs of index lists of length $p$ over the eight labels "
            ...)
check(formula_counts(4) == {1: 20160, -1: 20160, 0: 16736896},
      "length 4: 20160 values +1, 20160 values -1 and 16736896 zeros (formula)")
```

The figure is saved and the formula's counts for $p = 4$ are checked: $1680 \cdot 24 = 40320$ nonzero values, half of each sign. Tenth PASS line.

What Figure 11a.4 shows. For each length three bars: red (+1), blue (−1), grey (0), on a logarithmic axis. Red and blue have equal height for $p \geq 2$; for $p = 1$ there is no −1. The grey bar grows much faster than the others: the fraction of nonzero values falls from 8 in 64 at $p = 1$ to 40,320 in 16,777,216 at $p = 4$, about one in 416.

In [10], random pairs of lengths 4 to 9.

```
rng = np.random.default_rng(12345)  # pseudo-random numbers with a fixed seed
SAMPLES = 2000  # pairs per length p
python_nonzero = {}  # p -> number of nonzero values among the samples
random_mismatches = 0
```

`np.random.default_rng(12345)` makes a generator of pseudo-random numbers that starts from the seed 12345, so every run on every computer draws the same numbers. 2,000 pairs are drawn for each length.

```
for p in range(4, 10):
    nonzero = 0
    for s in range(SAMPLES):
        upper = [int(v) for v in rng.integers(0, 8, size=p)]  # p labels from 0..7
        if s % 2 == 0:  # even s: the lower list is the upper list rearranged
            lower = [upper[i] for i in rng.permutation(p)]
        else:  # odd s: an independent random lower list
            lower = [int(v) for v in rng.integers(0, 8, size=p)]
```

`range(4, 10)` is 4 to 9. `rng.integers(0, 8, size=p)` draws p whole numbers from 0 to 7 (the upper bound 8 is not included). For an even sample number `s`, `rng.permutation(p)` is a random arrangement of the places $0, \ldots, p-1$, and the lower list takes the upper labels in that order: a rearranged pair. For an odd `s` the lower list is drawn independently.

```
        value = gkd(lower, upper)
        nonzero += value != 0  # True counts as 1, False as 0
        random_mismatches += value != kdelta(lower, upper)
    python_nonzero[p] = nonzero
    say(f"p = {p}: {SAMPLES} random pairs, {nonzero:4d} nonzero values")
```

For every sample the rule is computed, the nonzero values are counted, and every disagreement with the literal determinant is counted (a true comparison adds 1, a false one 0). One line per length is printed: 394, 186, 82, 24, 3 and 0 nonzero values for $p = 4$ to 9.

```
report("random pairs compared (lengths 4 to 9)", 6 * SAMPLES)
report("random pairs where GKD and the determinant differ", random_mismatches)
report("different 0/1 matrices whose determinant sympy computed (so far)",
       len(DETERMINANTS))
check(random_mismatches == 0,
      "GKD equals the literal determinant for 12000 random pairs of lengths 4 to 9")
check(python_nonzero[9] == 0,
      "every delta with 9 indices in 8 dimensions is 0 (pigeonhole)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check gkd_nine_indices_in_eight_dimensions_vanish")
```

RESULT lines: 12,000 pairs compared, 0 disagreements, and 10,097 different matrices met so far (this is why the cell takes a few seconds: sympy computes almost ten thousand new determinants of sizes 4 to 9). The two checks: no disagreement, and no nonzero value of length 9, as the pigeonhole principle demands (PASS lines eleven and twelve).

In [11], the counts against the birthday formula.

```
def all_different(p, n=8):
    return math.perm(n, p) / n ** p if p <= n else 0.0


def expected_nonzero(p, half):
    r = all_different(p)
    q = nonzero_pairs(p) / 8 ** (2 * p) if p <= 8 else 0.0
    mean = half * (r + q)
    spread = math.sqrt(half * r * (1 - r) + half * q * (1 - q))
    return mean, spread
```

`all_different` is the chance $r_p = 8!/((8-p)!\,8^p)$ of Section 11.4; for more than 8 labels it returns 0. `expected_nonzero` returns the expectation $N(r_p + q_p)$ and the standard deviation $\sqrt{Nr_p(1-r_p) + Nq_p(1-q_p)}$ for `half` $= N$ rearranged and N independent samples (`math.sqrt` is the square root).

```
within = True
for p in range(4, 10):
    mean, spread = expected_nonzero(p, SAMPLES // 2)
    distance = abs(python_nonzero[p] - mean) / spread if spread > 0 else 0.0
    say(f"p = {p}: counted {python_nonzero[p]:4d}, expected {mean:7.1f} "
```

```
        f"+- {spread:5.1f}  ({distance:.2f} standard deviations away)")
    within = within and (distance < 5.0 if spread > 0 else python_nonzero[p] == 0)
check(within, "the Python counts of nonzero values agree with the birthday formula")
```

For each length, with $N = 1000$, the distance of the count from its expectation is measured in standard deviations (`abs` is the size of a number; the format `:7.1f` writes a decimal number with one digit after the point). The output: 394 against 412.6 ± 15.6 (1.19 standard deviations), 186 against 205.8 ± 12.8 (1.55), 82 against 77.1 ± 8.4 (0.58), 24 against 19.3 ± 4.3 (1.09), 3 against 2.4 ± 1.6 (0.38), and 0 against exactly 0 for $p = 9$. The check requires every distance to be below 5 standard deviations (a chance of less than one in a million for each count), and exactly 0 where the standard deviation is 0. Thirteenth PASS line.

In [12], the cost of the determinant (Figure 11a.5).

```
lengths = list(range(1, 10))
leibniz_terms = [math.factorial(p) for p in lengths]  # products in the formula
comparisons = [p * (p - 1) // 2 for p in lengths]  # comparisons of the rule
all_pairs = [8 ** (2 * p) for p in lengths]  # all pairs of lists of length p
for p, terms, compare, total in zip(lengths, leibniz_terms, comparisons, all_pairs):
    say(f"p = {p}: Leibniz products {terms:6d}, comparisons of GKD {compare:2d}, "
        f"pairs of lists {total:.3e}")
```

The three rows of the cost table of Section 11.4 for $p = 1$ to 9, printed one line per length (the format `:.3e` writes a number in the form 1.678×10^7 as `1.678e+07`).

```
fig, ax = plt.subplots(figsize=(7.4, 4.4))
ax.semilogy(lengths, leibniz_terms, "o-", label="products in the Leibniz formula, $p!$")
ax.semilogy(lengths, [max(c, 0.5) for c in comparisons], "s-",
            label="comparisons made by GKD, $p(p-1)/2$")
ax.semilogy(lengths, all_pairs, "^:", label="all pairs of index lists, $8^{2p}$")
```

`ax.semilogy` draws a curve with a logarithmic vertical axis; `"o-"` means circles joined by a line, `"s-"` squares and a line, `"^:"` triangles and a dotted line. The comparison count 0 of $p = 1$ is drawn at 0.5, because 0 has no place on a logarithmic axis.

```
ax.axvline(4.5, color="0.5", lw=0.8)  # where exhaustive testing ends
ax.text(4.65, 3e15, "right of this line:\nrandom samples only", fontsize=8,
        va="top")
ax.set_xlabel("length $p$ of the index lists")
ax.set_ylabel("count (logarithmic scale)")
ax.set_title("The work of the literal determinant and of the rule GKD")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "cost_of_the_determinant",
            "Work per evaluation of the generalized Kronecker delta for index lists "
            ...)
```

A thin vertical line at $p = 4.5$ separates the lengths tested exhaustively from those tested by samples, with a note beside it (`\n` starts a new line of text); then titles, legend, and the saved figure.

```
timing_rng = np.random.default_rng(7)
timing_pairs = []
for _ in range(200):
    upper = [int(v) for v in timing_rng.permutation(8)[:7]]  # 7 different labels
    lower = [upper[i] for i in timing_rng.permutation(7)]  # the same, rearranged
    timing_pairs.append((lower, upper))
```

200 pairs of length 7 for the timing, from a second generator with the seed 7: the first 7 entries (`[:7]`) of a random arrangement of the eight labels are 7 different labels, and the lower list is the same labels in another random order. Every one of these pairs has a nonzero value.

```
start = time.perf_counter()
rule_values = [gkd(lower, upper) for lower, upper in timing_pairs]
rule_seconds = time.perf_counter() - start
start = time.perf_counter()
determinant_values = [int(sp.Matrix(outer_delta(lower, upper)).det())
                      for lower, upper in timing_pairs]  # no memory: all computed
determinant_seconds = time.perf_counter() - start
```

`time.perf_counter()` reads a stopwatch; the difference of two readings is the time between them in seconds. The rule is timed on the 200 pairs, then sympy's exact determinant on the same pairs, computed afresh each time (without the memory of `kdelta`).

```
check(rule_values == determinant_values,
      "200 pairs of length 7: the rule and sympy's determinant agree")
check(determinant_seconds > 10 * rule_seconds,
      "the rule GKD is more than ten times faster than the determinant (p = 7)")
```

Two checks: the 200 values agree, and the determinants took more than ten times as long as the rule. The times themselves are never printed, so the output stays the same on every run. PASS lines fourteen and fifteen.

What Figure 11a.5 shows. Three curves on a logarithmic axis against the length p. The circles ($p!$) rise ever more steeply, to 362,880 at $p = 9$; the squares ($p(p-1)/2$) stay small, 36 at $p = 9$; the triangles (8^{2p}) rise as a straight line, because a power 64^p is a straight line on a logarithmic axis, to 1.8×10^{16}. Right of the line at $p = 4.5$ only samples can be tested.

In [13], the Rust program is built, and the command of its self-test.

```
program = rust_program("Revision/gkd_lovelock/code/Cargo.toml", "lovelock_gkd")
```

The helper of the set-up cell runs cargo for the crate `Revision/gkd_lovelock/code` and returns the path of the program `lovelock_gkd` (with `.exe` on Windows), named `program` here. When the program is up to date this takes about a second; a first build took a few seconds on the computer that built the book (the crate has no dependencies, so nothing is downloaded). Output: Rust program lovelock_gkd is built and ready.

```
COMMAND = ["lovelock_gkd", "gkd-selftest", "--exhaustive-max", "4", "--output", "FOLDER"]
say("To repeat the self-test (7 to 13 minutes), run in the folder of the program: "
    + " ".join(COMMAND))
```

`COMMAND` is the list of the words of the command that runs the self-test: the program, the command `gkd-selftest`, the option that makes the lengths 1 to 4 exhaustive, and the folder for the result file; `" ".join(COMMAND)` joins the words with spaces. The cell does not run the command. The self-test takes 7 to 13 minutes on the computer that built the book, most of it in the 200,000 literal determinants of 9×9 matrices, each a sum of $9! = 362880$ products; that is longer than the limit of 7 minutes that this book sets for one computation. Output: the line with the command

```
lovelock_gkd gkd-selftest --exhaustive-max 4 --output FOLDER
```

To repeat the self-test yourself, run this command in a terminal, in the folder `Revision/gkd_lovelock/code/target/release` (on Windows the program file is `lovelock_gkd.exe`), with a folder of your choice in place of FOLDER: the file `gkd-selftest.json` that it writes there must equal the record byte for byte.

In [14], the record of the Rust self-test.

```
SELFTEST = "Revision/gkd_lovelock/results/gkd-selftest.json"
written = repository_file(SELFTEST).read_bytes()  # the record of the Rust run
selftest_record = json.loads(written.decode("utf-8"))
say("The record of the Rust self-test (as a table):")
```

`read_bytes()` reads the record as its raw bytes (the next cell uses them again), `decode("utf-8")` turns them into text and `json.loads` into a dictionary. Then a heading line is printed.

```
rust_rows = {}  # p -> (mode, pairs, nonzero or None, mismatches)
for row in selftest_record["results"]:
    rust_rows[row["p"]] = (row["mode"], row["pairs"], row.get("nonzero"),
                           row["mismatches"])
```

The entry `results` of the record holds one row per length p: the mode (`exhaustive` or `random`), the number of pairs, for the random lengths the number of nonzero values, and the number of disagreements between GKD and the literal determinant. `row.get("nonzero")` gives the number of nonzero values, and `None` for an exhaustive row, which has no such entry. The dictionary `rust_rows` keeps each row as a tuple under its length.

```
for p, (mode, pairs, nonzero, bad) in sorted(rust_rows.items()):
    extra = "" if nonzero is None else f", {nonzero:6d} nonzero"
    say(f"  p = {p}: {mode:10s} {pairs:9d} pairs{extra}; disagreements: {bad}")
```

The rows are printed in the order of p as a table. The output shows 64, 4096, 262144 and 16777216 pairs compared exhaustively for $p = 1$ to 4, and 200,000 random pairs for each $p = 5$ to 9 with 20538, 7812, 1949, 244 and 0 nonzero values; every disagreement count is 0.

```
check(selftest_record["program"] == "lovelock_gkd gkd-selftest"
      and selftest_record["verdict"] == "SUCCESS",
      "the record of the Rust program lovelock_gkd gkd-selftest has the verdict "
      "SUCCESS", record=SELFTEST)
```

```
check(sorted(rust_rows) == list(range(1, 10))
      and all(row[3] == 0 for row in rust_rows.values())
      and rust_rows[4][:2] == ("exhaustive", 16777216),
      "Rust: GKD equals the literal determinant for every tested pair, p = 1 to 9",
      record=SELFTEST)
check(program.is_file(), "the program lovelock_gkd that repeats the self-test is built")
```

The first check requires the program name of the record and its verdict SUCCESS. The second requires a row for every length 1 to 9, no disagreement in any row (`row[3]` is the fourth entry, the mismatches) and the exhaustive test of all 16,777,216 pairs of length 4 (`[:2]` takes the first two entries of the row). The third requires that the program file that repeats the self-test exists (`is_file()`). PASS lines sixteen to eighteen.

In [15], the self-test against the birthday formula (Figure 11a.6).

```
selftest = json.loads(written.decode("utf-8"))
rust_nonzero = {row["p"]: row["nonzero"] for row in selftest["results"]
                if row["mode"] == "random"}
```

The bytes just compared are decoded as text and read as JSON. A **dictionary comprehension** collects, for every random length, its number of nonzero values.

```
agree = True
for p, nonzero in sorted(rust_nonzero.items()):
    mean, spread = expected_nonzero(p, 100000)
    distance = abs(nonzero - mean) / spread if spread > 0 else 0.0
    say(f"Rust p = {p}: counted {nonzero:6d}, expected {mean:8.1f} +- {spread:6.1f}"
        f"  ({distance:.2f} standard deviations away)")
    agree = agree and (distance < 5.0 if spread > 0 else nonzero == 0)
check(selftest["verdict"] == "SUCCESS" and agree and rust_nonzero[9] == 0,
      "the Rust counts of nonzero values agree with the birthday formula")
```

The same comparison as in In [11], now with $N = 100000$ samples of each kind: 20538 against 20582.9 ± 128.0 (0.35 standard deviations), 7812 against 7711.6 ± 84.4 (1.19), 1949 against 1927.2 ± 43.5 (0.50), 244 against 240.9 ± 15.5 (0.20), and 0 for $p = 9$. The check also requires the verdict SUCCESS of the file. Nineteenth PASS line.

```
lengths_1_to_8 = np.arange(1, 9)  # r_p is drawn where it is not zero
fig, ax = plt.subplots(figsize=(7.4, 4.4))
ax.semilogy(lengths_1_to_8, [all_different(p) for p in lengths_1_to_8], "k-",
            label="chance that $p$ labels out of 8 are all different, $r_p$")
```

`np.arange(1, 9)` is the array $1, \ldots, 8$; the black line (`"k-"`) is r_p on a logarithmic axis.

```
python_points = [p for p in python_nonzero if python_nonzero[p] > 0]
ax.semilogy(python_points, [python_nonzero[p] / (SAMPLES // 2)
                            for p in python_points], "o",
            label="Python: nonzero values / 1000 rearranged pairs")
```

```
rust_points = [p for p in rust_nonzero if rust_nonzero[p] > 0]
ax.semilogy(rust_points, [rust_nonzero[p] / 100000 for p in rust_points], "s",
            fillstyle="none", markersize=10,
            label="Rust: nonzero values / 100000 rearranged pairs")
```

The measured fractions: the nonzero values of all the samples of one length divided by the number of rearranged samples, dots for Python and large open squares for Rust. Lengths with no nonzero value are left out, because 0 has no place on a logarithmic axis.

```
ax.plot([9], [1e-6], "v", color="black", markersize=9)  # marks the zeros of p = 9
ax.text(9.0, 2.2e-6, "p = 9: none\n(0 of 1000,\n0 of 100000)", fontsize=8,
        ha="center", va="bottom")
ax.axvline(8.5, color="0.5", lw=0.8)
ax.text(8.6, 0.3, "more than 8\nlabels: always\na repetition", fontsize=8, va="top")
```

A black triangle at the bottom edge and a note mark the zeros of $p = 9$; a vertical line at 8.5 with a note marks where every list must repeat a label.

```
ax.set_xlim(0.5, 10.0)
ax.set_ylim(5e-7, 2.0)
ax.set_xlabel("length $p$ of the index lists")
ax.set_ylabel("fraction (logarithmic scale)")
ax.set_title("Nonzero deltas among rearranged random pairs")
ax.legend(loc="lower left", fontsize=8)
save_figure(fig, "all_labels_different",
            "The chance $r_p = 8!/((8 - p)!\\, 8^p)$ that $p$ random labels out of "
            ...)
```

The ranges of the two axes, titles, legend and the saved figure (in the caption string a doubled backslash stands for one backslash).

What Figure 11a.6 shows. The chance r_p falls from 1 at $p = 1$ to 0.0024 at $p = 8$; the dots and squares lie on the line, within their random scatter. The independent samples add at most $q_4 = 40320/8^8 \approx 0.0024$, too little to see. For $p = 9$ both tests found no nonzero value, as the pigeonhole principle demands.

In [16], the quoted records once more.

```
def verdicts(path):
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    checks = data["checks"]
    if isinstance(checks, dict):  # name -> {"passed": true or false, ...}
        return {name: "PASS" if entry["passed"] else "FAIL"
                for name, entry in checks.items()}
    return {entry["name"]: entry["verdict"] for entry in checks}  # a list of entries
```

A helper (docstring left out) that returns, for one report, a dictionary from the name of each check to PASS or FAIL. The Revision reports store their checks in one of two forms, and `isinstance(checks, dict)` tells which: a dictionary of entries with a field `passed`, or a list of entries with a `name` and a `verdict`.

```
RECORDS = "Revision/gkd_lovelock/results"  # the folder of the Revision records
QUOTED = {  # report -> the checks that this notebook quotes from it
    "python-lovelock-report.json": [
        "gkd_examples", "gkd_literal_equals_cofactor_expansion",
        "gkd_nine_indices_in_eight_dimensions_vanish"],
    "wolfram-gkd-report.json": [
        "gkd_equals_kdelta_exhaustive_length_1",
        "gkd_equals_kdelta_exhaustive_length_2",
        "gkd_equals_kdelta_exhaustive_length_3"],
}
```

The checks that the notebook's text quotes, by report.

```
for report_name, names in QUOTED.items():
    found = verdicts(f"{RECORDS}/{report_name}")
    for name in names:
        say(f"{report_name}: {name} {found.get(name, 'MISSING')}")
    check(all(found.get(name) == "PASS" for name in names),
          f"the {len(names)} checks quoted from {report_name} are there and PASS",
          record=f"{RECORDS}/{report_name}, checks " + ", ".join(names))
```

For each report every quoted check is printed with its verdict (`get(name, 'MISSING')` gives MISSING for a name that is not there), and the check requires all of them to be present with PASS. `", ".join(names)` joins the names with commas. Two PASS lines, twenty and twenty-one.

```
provenance = repository_file(
    f"{RECORDS}/PROVENANCE_OF_THE_COMPUTATION.md").read_text(encoding="utf-8")
definition = ("[lower_, upper_] /; Length[lower] == Length[upper] := "
              "Det[Outer[delta, lower, upper]]")  # after the name of the delta
check(definition in provenance,
      "the record quotes the author's definition as section 4 of this notebook "
      "does",
      record=f"{RECORDS}/PROVENANCE_OF_THE_COMPUTATION.md")
```

The record of the author's definition is read, and the check requires the definition, from the bracket after its name on, to occur in it exactly as the notebook quotes it in its own section 4 (the name itself is written with a Greek letter in the record). Two strings written one after the other inside brackets are joined into one by Python. Twenty-second PASS line.

```
order_three = [row for row in json.loads(repository_file(
    f"{RECORDS}/lovelock-report.json").read_text(encoding="utf-8"))["counters"]
    if row["k"] == 3][0]
report("GKD calls of the order-3 Lovelock sum (record)", order_three["gkdCalls"])
check(order_three["gkdCalls"] == 495360,
      "the order-3 Lovelock sum calls GKD 495360 times, as section 4 of this "
      "notebook says",
      record=f"{RECORDS}/lovelock-report.json, counters, k = 3, gkdCalls")
```

The Rust report's counters are read and the row of order 3 is taken; its number of GKD calls, 495,360, is printed and checked (the number quoted in the notebook's section 4 and explained in Section 11.12). Twenty-third PASS line.

In [17], the end.

```
figure_files = [f"11a_{k}_{name}.png" for k, name in enumerate(
    ["outer_delta_matrices", "six_permutations", "gkd_map_length_two",
     "value_counts", "cost_of_the_determinant", "all_labels_different"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_files),
      "all six figure files of the notebook exist")
all_checks_passed()
```

`enumerate(..., 1)` numbers the six figure names from 1, which gives the six file names; the check requires all six files to exist in the figures folder (PASS line twenty-four), and `all_checks_passed()` prints the last line, ALL 24 CHECKS PASSED (notebook 11a). The 24 checks are: 2 in In [2], 1 each in In [3], In [4], In [5], In [6], 2 in In [7], 1 each in In [8] and In [9], 2 in In [10], 1 in In [11], 2 in In [12], 3 in In [14], 1 in In [15], 4 in In [16] and 1 in In [17].

11.10 The curvature that the Lovelock sums need

The Lovelock tensors are built from the Riemann tensor. Chapter 3 computed it for the author's metric by hand; here we collect what the sums need, in the convention of every Revision record (the convention of Misner, Thorne and Wheeler, Section 3.17):

$$\Gamma^{a}{}_{bc} = \tfrac{1}{2}\, g^{aa}\big(\partial_b g_{ac} + \partial_c g_{ab} - \partial_a g_{bc}\big), \qquad R^{a}{}_{bcd} = \partial_c \Gamma^{a}{}_{db} - \partial_d \Gamma^{a}{}_{cb} + \sum_e \big(\Gamma^{a}{}_{ce}\Gamma^{e}{}_{db} - \Gamma^{a}{}_{de}\Gamma^{e}{}_{cb}\big),$$

and $R^{ab}{}_{cd} = g^{bb} R^{a}{}_{bcd}$ (no sum; for a diagonal metric the general $\sum_e g^{be} R^{a}{}_{ecd}$ keeps only $e = b$). The form $R^{ab}{}_{cd}$, with two labels up and two down, is the one the Lovelock sums use. It changes sign when the two upper labels or the two lower labels are exchanged:

$$R^{ab}{}_{cd} = -R^{ba}{}_{cd} = -R^{ab}{}_{dc}$$

(the Rust report `lovelock-report.json`, check `riemann_antisymmetry`; Section 3.17 proves it). In particular $R^{aa}{}_{cd} = 0$: a component with two equal upper labels or two equal lower labels vanishes.

The plane curvatures. For two different directions a and b, $K(a,b) = R^{ab}{}_{ab}$ (no sum) is the curvature of their coordinate plane. Section 3.24 computed all 28 planes by hand. With i standing for a 3-space direction (x_1, x_2, x_3), j for an extra time (x_5, x_6, x_7) and k for either:

plane	**how many**	$K(a,b)$	**at** $a_4' = 2H$, $a_4'' = 0$
two 3-space directions (i, i')	3	$a_4'^2 - H^2$	$3H^2$
two extra times (j, j')	3	$a_4'^2 - H^2$	$3H^2$
a 3-space direction and an extra time (i, j)	9	$-(a_4'^2 + H^2)$	$-5H^2$
a 3-space direction and the time (i, x_4)	3	$a_4'^2 + a_4''$	$4H^2$

plane	**how many**	$K(a,b)$	**at** $a_4' = 2H$**,** $a_4'' = 0$
an extra time and the time (j, x_4)	3	$a_4'^2 - a_4''$	$4H^2$
a transverse direction and the hidden one (k, x_0)	6	$-H^2$	$-H^2$
the time and the hidden direction (x_4, x_0)	1	0	0

Figure 11b.2 draws these 28 numbers as an 8×8 table at $a_4' = 2H$, once with $a_4'' = 0$ and once with $a_4'' = H^2$ (Notebook 11b, In [9]). Each plane with $K \neq 0$ gives four nonzero components, $R^{ab}{}_{ab} = R^{ba}{}_{ba} = K(a,b)$ and $R^{ab}{}_{ba} = R^{ba}{}_{ab} = -K(a,b)$: 27 planes, 108 components.

The components that mix the time and the hidden direction (PROVED). The other nonzero components connect the plane of x_4 and x_k with the plane of x_0 and x_k. We derive the first kind once more, line by line, writing the scale-factor exponent of the transverse direction k as σ_k: the metric entry is $g_{kk} = \epsilon_k\, e^{2\sigma_k a_4} \sin^{1/3} z$ with $\sigma_k = +1$, $\epsilon_k = +1$ for 3-space (inflating, space-like) and $\sigma_k = -1$, $\epsilon_k = -1$ for an extra time (deflating, time-like). The formula for $R^a{}_{bcd}$ with $a = x_4$, $b = k$, $c = x_0$, $d = k$ reads

$$R^4{}_{k0k} = \partial_0 \Gamma^4{}_{kk} - \partial_k \Gamma^4{}_{0k} + \sum_e \Gamma^4{}_{0e}\Gamma^e{}_{kk} - \sum_e \Gamma^4{}_{ke}\Gamma^e{}_{0k}$$

(the definition with these four labels). Nothing depends on x_k, so $\partial_k(\dots) = 0$; every symbol $\Gamma^4{}_{0e}$ is zero (the only nonzero symbols with the upper label x_4 are $\Gamma^4{}_{kk}$, Section 3.23); and in the last sum only $e = k$ survives:

$$R^4{}_{k0k} = \partial_0 \Gamma^4{}_{kk} - \Gamma^4{}_{kk}\,\Gamma^k{}_{0k}.$$

The two symbols are, by the Christoffel formula with $g^{44} = -1$,

$$\Gamma^4{}_{kk} = -\tfrac{1}{2}\, g^{44}\, \partial_4 g_{kk} = \tfrac{1}{2}\, \epsilon_k\, 2\sigma_k a_4'\, e^{2\sigma_k a_4} \sin^{1/3} z = \epsilon_k \sigma_k\, a_4'\, e^{2\sigma_k a_4} \sin^{1/3} z,$$

$$\Gamma^k{}_{0k} = \tfrac{1}{2}\, g^{kk}\, \partial_0 g_{kk} = H \cot z$$

(the chain rule: $\partial_4 e^{2\sigma_k a_4} = 2\sigma_k a_4' e^{2\sigma_k a_4}$; and $\partial_0 \ln \sin^{1/3} z = \tfrac{1}{3} \cot z \cdot 6H = 2H \cot z$, half of which is $H \cot z$). Since $\partial_0 \sin^{1/3} z = 2H \cot z \, \sin^{1/3} z$,

$$R^4{}_{k0k} = \epsilon_k \sigma_k\, a_4'\, e^{2\sigma_k a_4} \sin^{1/3} z\, (2H \cot z - H \cot z) = \epsilon_k \sigma_k\, H a_4' \cot z\, e^{2\sigma_k a_4} \sin^{1/3} z$$

(the derivative of the product; then the common factor taken out). Raising the second label multiplies by $g^{kk} = 1/(\epsilon_k e^{2\sigma_k a_4} \sin^{1/3} z)$:

$$R^{4k}{}_{0k} = \sigma_k\, H a_4' \cot z$$

($\epsilon_k/\epsilon_k = 1$ and the exponentials cancel). So the sign is set by σ_k alone, that is by whether the direction inflates or deflates, not by whether it is space-like or time-like: $R^{4i}{}_{0i} = +Ha_4' \cot z$ for the three 3-space directions and $R^{4j}{}_{0j} = -Ha_4' \cot z$ for the three extra times, as Section 3.24 found. The same steps with $a = x_0$, $c = x_4$ give $R^{0i}{}_{4i} = -Ha_4' \tan z$ and $R^{0j}{}_{4j} = +Ha_4' \tan z$ (Section 3.24). With the antisymmetries each of the six transverse k gives eight such components: 48. Together $108 + 48 = 156$ nonzero components of the 4096 (record check `riemann_antisymmetry`, "156 nonzero entries"; Notebook 11b, In [7], recomputes them; Figure 11b.3 draws all 4096 at one point).

What this means for the Ricci tensor. The off-diagonal Ricci component $R^{x_4}{}_{x_0} = \sum_c R^{x_4 c}{}_{x_0 c}$ is the sum of the seven terms $c = x_1, \ldots, x_7$ (the term $c = x_0$ vanishes by antisymmetry):

$$R^{x_4}{}_{x_0} = 3 \cdot (+Ha_4' \cot z) + 0 + 3 \cdot (-Ha_4' \cot z) = 0$$

(three inflating directions, the time itself, three deflating directions). On the background $b_4 = 1$ the cancellation is exact for every a_4, because there are as many inflating as deflating directions (Notebook 11b, In [11], Figure 11b.4). Section 11.21 shows that the same cancellation leaves all three Lovelock tensors without an x_4-x_0 component.

With the second function b_4 (PROVED by exact computation). The Revision records compute the curvature for the author's metric of 2026-10-14, whose extra-time components carry the second function $b_4(x_4)$: $g_{jj} = -e^{-2a_4} b_4 \sin^{1/3} z$. The table and the components above are its background $b_4 = 1$ (and every constant b_4, which is a rescaling of the extra times). With a changing b_4 the same 156 components are nonzero, but those with an extra time contain b_4'/b_4 and b_4''/b_4 as well, and the three extra-time terms of $R^{x_4}{}_{x_0}$ no longer cancel the three space terms:

$$R^{x_4}{}_{x_0} = \frac{3}{2} H \cot z \frac{b_4'}{b_4}, \qquad R = 6a_4'^2 - 42H^2 + 3\frac{b_4''}{b_4} - 3a_4'\frac{b_4'}{b_4}$$

(record `curvature.json`, entries `einsteinMixed` and `ricciScalar`; Notebook 11b, In [8] and In [11], recomputes both with sympy). So with a changing b_4 the time and the hidden direction mix; Chapter 12 shows what this does to the field equations.

Three facts used below (PROVED, on the background $b_4 = 1$).

- Every nonzero $R^{ab}{}_{cd}$ is one of $a_4'^2 - H^2$, $-(a_4'^2 + H^2)$, $a_4'^2 \pm a_4''$, $-H^2$, $\pm Ha_4' \cot z$, $\pm Ha_4' \tan z$: a polynomial in H, a_4', a_4'', $\cot z$ and $\tan z = 1/\cot z$, free of e^{a_4} and of $\sin^{1/3} z$ (record check `mixed_riemann_free_of_sin_third`; `Revision/gkd_lovelock/results/python-lovelock-report.json`, check `mixed_riemann_free_of_warp_and_exponential`; Notebook 11b, In [7]).
- **Weight 2.** Count H and a_4' as 1 and a_4'' as 2 (each is one inverse length per count: $z = 6Hx_0$ is a pure number, so H is an inverse length; a_4 is a pure number, so $a_4' = da_4/dx_4$ is an inverse length and a_4'' an inverse length squared), and $\cot z$ as 0. Then every one of the components just listed has the weight 2: curvature has the dimension $1/\text{length}^2$.
- **The Ricci tensor, the Ricci scalar and the Einstein tensor** (Section 3.25): $R^i{}_i = a_4'' - 6H^2$, $R^4{}_4 = 6a_4'^2$, $R^j{}_j = -a_4'' - 6H^2$, $R^0{}_0 = -6H^2$, every off-diagonal entry zero; $R = 6a_4'^2 - 42H^2$; and

$$G^{i}{}_{i} = -3a_4'^2 + a_4'' + 15H^2, \quad G^{4}{}_{4} = 3a_4'^2 + 21H^2, \quad G^{j}{}_{j} = -3a_4'^2 - a_4'' + 15H^2, \quad G^{0}{}_{0} = 15H^2 - 3a_4'^2$$

(record `curvature.json`, its entries for the Ricci tensor, the Ricci scalar and the Einstein tensor; Notebook 11b, In [8], reproduces the Ricci scalar).

11.11 Lovelock's equation (4.38): the three Lovelock tensors

The formula. The author's notebook contains, as the cell stored in the file as In[101] (his In[68]), an image of equation (4.38) of the book of Lovelock and Rund; the Revision rendered it to `Revision/gkd_lovelock/results/notebook-in68-image.png` and read the formula from that picture (record `Revision/gkd_lovelock/results/PROVENANCE_OF_THE_COMPUTATION.md`). For a space of $n = 2m$ dimensions it reads, as programmed in the record (`Revision/gkd_lovelock/code/src/lovelock.rs`),

$$A^{lh} = \sqrt{|\det g|} \sum_{k=1}^{m-1} \alpha_{(k)}\, g^{jl}\, \delta^{h\, h_1 \ldots h_{2k}}_{j\, j_1 \ldots j_{2k}}\, R^{j_1 j_2}{}_{h_1 h_2} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}} + \lambda \sqrt{|\det g|}\, g^{lh},$$

with constants $\alpha_{(k)}$ and λ, and a sum over every label that appears once up and once down (j and the $4k$ labels $h_1, \ldots, h_{2k}, j_1, \ldots, j_{2k}$). The building blocks are, for each **order** k, the **Lovelock tensor**

$$P_{(k)}{}^{h}{}_{j} = \sum \delta^{h\, h_1 \ldots h_{2k}}_{j\, j_1 \ldots j_{2k}}\, R^{j_1 j_2}{}_{h_1 h_2}\, R^{j_3 j_4}{}_{h_3 h_4} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}}$$

(the sum over the $4k$ labels $h_1, \ldots, h_{2k}, j_1, \ldots, j_{2k}$; h and j are free), the **Lovelock scalar**, the same sum without the free pair,

$$L_{(k)} = \sum \delta^{h_1 \ldots h_{2k}}_{j_1 \ldots j_{2k}}\, R^{j_1 j_2}{}_{h_1 h_2} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}},$$

and the **density** $A_{(k)}{}^{lh} = \sqrt{|\det g|}\, g^{ll}\, P_{(k)}{}^{h}{}_{l}$ (no sum; diagonal metric). Read the generalized delta carefully: its upper list $(h, h_1, \ldots, h_{2k})$ collects the LOWER labels of the curvature factors, and its lower list $(j, j_1, \ldots, j_{2k})$ their UPPER labels. In the language of GKD,

$$P_{(k)}{}^{h}{}_{j} = \sum \mathrm{GKD}([j, j_1, \ldots, j_{2k}],\, [h, h_1, \ldots, h_{2k}]) \prod_{i=1}^{k} R^{j_{2i-1} j_{2i}}{}_{h_{2i-1} h_{2i}},$$

the lower list first, as in the author's `kδ[lower, upper]`. The matrix of the exchanged lists, with the entries $\delta(u_i, l_j)$, is the transpose of M, and a matrix and its transpose have the same determinant (rule 2 of Section 1.20); so exchanging the two lists does not change the value. The record of the field equations writes them in the other order (`Revision/field_equations_a4/a4-equations.json`, key `conventions`, entry `lovelock`).

Which orders exist (PROVED). The delta of order k has $2k + 1$ labels in each list. In eight dimensions ($m = 4$) the sum runs over $k = 1, 2, 3$, and the order $k = 4$, with nine labels, vanishes

identically by the pigeonhole principle (Section 11.4; record check `k4_tensor_vanishes`). So there are exactly three Lovelock tensors in eight dimensions. The scalar $L_{(4)}$, with eight labels per list, does not vanish: it is the **Euler density** of eight dimensions, and only its tensor $P_{(4)}$ is zero. The name comes from a classical theorem that this book quotes and does not prove (ASSUMED): in a space of even dimension $n = 2m$ the Lovelock scalar of order m is called the Euler density, because for a **closed** space (finite and without an edge, like the surface of a ball) with a positive-definite metric (every squared length positive, unlike the author's metric) its integral over the whole space, with the volume factor $\sqrt{|\det g|}$, is a constant times the **Euler characteristic**: a whole number that depends only on the shape of the space (2 for the surface of a ball, 0 for the surface of a ring) and does not change when the metric is deformed (the Chern-Gauss-Bonnet theorem). A quantity whose integral does not change when the metric changes gives no field equation; this agrees with $P_{(4)} = 0$, which the pigeonhole principle proved above for every metric. For the author's metric the program prints it (Notebook 11b, In [3]):

$$L_{(4)} = -663552\, H^2 a_4'^6 - 442368\, H^4 a_4'^4 - 663552\, H^6 a_4'^2.$$

The usual normalisation. Books on gravity use the normalised tensors

$$E_{(k)}{}^h{}_j = -\frac{1}{2^{k+1}}\, P_{(k)}{}^h{}_j,$$

because then $E_{(1)}$ is exactly Einstein's tensor G (Section 11.14). The field equations of Chapter 12 are $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda \delta^\mu_\nu = \kappa T^\mu{}_\nu$, with $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$ for Einstein's gravity (record `Revision/field_equations_a4/a4-equations.json`, key `conventions`, entry `fieldEquations`). Any other normalisation is absorbed into the constants α_k.

What Lovelock proved (quoted, ASSUMED in general). For every metric each $P_{(k)}$ has zero covariant divergence, $\sum_h \nabla_h P_{(k)}{}^h{}_j = 0$, and is symmetric once its upper label is lowered, $g_{hh} P_{(k)}{}^h{}_j = g_{jj} P_{(k)}{}^j{}_h$ (for a diagonal metric); and every tensor with these properties that is built from the metric and its first and second derivatives is a combination of the $P_{(k)}$ and of δ^h_j. We do not prove this theorem. For the author's metric both properties are PROVED by exact computation in the record: checks `k1_divergence_free` to `k3_divergence_free` and `k1_symmetric` to `k3_symmetric` of `Revision/gkd_lovelock/results/lovelock-report.json` and of `Revision/gkd_lovelock/results/python-lovelock-report.json` (the symmetry makes $A_{(k)}{}^{lh} = A_{(k)}{}^{hl}$). Section 11.22 writes out what zero divergence means for the components.

Weight $2k$ (PROVED). The generalized delta is a pure number, and each of the k curvature factors has the weight 2 (Section 11.10). So every term of $P_{(k)}$ has the weight $2k$: the tensor of order k has the dimension $1/\text{length}^{2k}$. The three orders can be added in one equation only with constants α_k of different dimensions, for example $\alpha_k = w_k / H^{2k-2}$ with pure numbers w_k; this is the role of H as "a fundamental inverse length" in the author's notebook (Chapter 12 uses the combinations $\alpha_2 H^2$ and $\alpha_3 H^4$).

11.12 Organising the sum: only nonzero curvature, no repeated label

How large the literal sum is. For one component (h, j) the sum over the $4k$ labels $h_1, \ldots, h_{2k}, j_1, \ldots, j_{2k}$, each with 8 values, has 8^{4k} terms; for all 64 components $64 \cdot 8^{4k}$. That

is 2.6×10^5 terms for $k = 1$, 1.1×10^9 for $k = 2$, 4.4×10^{12} for $k = 3$ and 1.8×10^{16} for $k = 4$ (Notebook 11b, In [5]). Each term is a product of exact polynomials, so the literal sum of order 3 is out of reach. Three exact facts make it small.

Step 1: only nonzero curvature factors. A term with a vanishing curvature factor vanishes. So each factor need only run through the 156 nonzero entries $R^{ab}{}_{cd}$ of Section 11.10, and a term is an ordered choice of k entries: $64 \cdot 156^k$ terms for all components, 1.0×10^4, 1.6×10^6, 2.4×10^8 and 3.8×10^{10} for $k = 1$ to 4.

Step 2: no repeated label. If two chosen entries share an upper label, or an entry shares an upper label with the free label j, the delta's lower list repeats a label and the delta is 0 (case a of Section 11.3); the same for the lower labels and the free h (case b). The program therefore builds the choices one entry at a time and keeps an entry only if it repeats no label already used. To test this quickly it stores a set of labels as a **bit mask**, the whole number $\sum 2^{\text{label}}$: the set $\{x_1, x_3\}$ (numbers 0 and 2) is $2^0 + 2^2 = 5$. Two sets share a label exactly when the bitwise AND of their masks is not 0 (the AND keeps a bit where both numbers have a 1, Section 1.49), and the union of two sets is their bitwise OR (a bit is 1 where at least one of the two numbers has a 1). A complete choice of k entries is called a **leaf**. The counts of leaves for all 64 components are 5,616, 176,640, 1,128,960 and 0 for $k = 1, 2, 3, 4$ (record `Revision/gkd_lovelock/results/lovelock-report.json`, field `counters`, entry `leaves`). For $k = 4$ there is no leaf: three entries already use seven different labels in the lower list (j and six more), and a fourth entry would bring two more, nine different labels out of eight.

Step 3: equal label sets. At a leaf the lower list holds $2k + 1$ different labels and so does the upper list. If the two sets of labels differ, some lower label is missing above and the delta is 0 (case c). Only leaves whose two masks are equal are passed to GKD, which then returns $+1$ or -1 (case d: both lists hold the same different labels). The numbers of GKD calls are 696, 32,640, 495,360 and 0, and every one of them gives a nonzero value (record entries `gkdCalls` and `gkdNonzero`, equal for every k). For the scalars $L_{(k)}$ the calls are 108, 7,200, 213,120 and, for the nonzero Euler density $L_{(4)}$, 1,105,920 (entry `scalarGkdCalls`).

Step 4: exact products, collected. Every term that remains is weighted by an explicit call of GKD, multiplied out exactly and added. The program represents every quantity as an exact **Laurent polynomial** (a sum of monomials that may contain negative powers, such as $\cot^{-1} z$) in H, a_4', a_4'' (and the higher derivatives), e^{a_4}, $\sin^{1/3} z$ and $\cot z$, with coefficients that are exact fractions of whole numbers (the files `poly.rs` and `rational.rs` of the crate; every operation that would overflow stops the program instead of rounding). So the result is exact: no rounding happens anywhere.

How the skipping is tested. The steps 1 to 3 rest on the theorem of Section 11.3, and they are tested in four independent ways, none of which skips anything:

- the program itself evaluates the literal, unpruned sum over all $64 \cdot 8^4 = 262144$ index lists of order 1 and all $64 \cdot 8^8 = 1073741824$ index lists of order 2, with GKD weights, at one numerical point, and finds the exact tensors up to rounding errors of 1.40×10^{-15} and 4.41×10^{-14} (relative; checks `k1_brute_force_numeric` and `k2_brute_force_numeric` of `lovelock-report.json`);
- the Revision's sympy checker evaluates the unpruned literal sums of orders 1 and 2 with the author's determinant on every ordered product of nonzero entries (10,140 and 1,581,840

calls) and finds the same tensors exactly (`python-lovelock-report.json`, checks `k1_unpruned_literal_sum_agrees` and `k2_unpruned_literal_sum_agrees`);

- the Revision's Wolfram check calls the author's verbatim definition on every ordered product for $k = 1$ (9,984 calls) and $k = 2$ (1,557,504 calls) and on every leaf for $k = 3$ (1,128,960 calls), and finds the Rust tensors exactly (`wolfram-gkd-report.json`, checks `k1_P_equals_rust_all_64_components` to `k3_P_equals_rust_all_64_components`);
- Notebook 11b repeats the literal sums of orders 1 and 2 with its own GKD (In [16]).

Figure 11b.1 draws the four counts of each order side by side.

11.13 Laplace's rule and the expansion along the column of h

The classical identities of the Lovelock tensors all come from one tool: the expansion of a determinant along a row or a column. Chapter 1 used it only for 3×3 matrices; we prove it in general.

Laplace's rule (PROVED). For a $p \times p$ matrix M and any row r,

$$\det M = \sum_{c=1}^{p} (-1)^{r+c} M_{rc} \det M^{(rc)},$$

where $M^{(rc)}$ is the minor (row r and column c removed); and the same holds along any column c, with the sum over r.

Proof. Every product of the Leibniz formula contains exactly one entry of row r. Group the products by the column c of that entry:

$$\det M = \sum_{c} M_{rc} C_{rc}, \qquad C_{rc} = \sum_{\pi:\ \pi(r)=c} \operatorname{sign}(\pi) \prod_{i \neq r} M_{i\pi(i)}$$

(the Leibniz formula, with the factor of row r taken out of each product). C_{rc}, the **cofactor**, contains no entry of row r. Now let M_* be M with row r replaced by the row that has 1 in column c and 0 elsewhere. Its cofactors are those of M (they contain no entry of row r), and only the column c of its row r is not zero, so

$$\det M_* = 1 \cdot C_{rc} = C_{rc}$$

(the grouping above, applied to M_*). We compute $\det M_*$ a second way. Move row r to the top by $r - 1$ exchanges with the row above it, and column c to the left by $c - 1$ exchanges with the column before it. Each exchange flips the sign of the determinant (rule 3 of Section 1.20 for rows, and for columns rule 3 applied to the transpose, rule 2), so the new matrix M' has

$$\det M' = (-1)^{(r-1)+(c-1)} \det M_* = (-1)^{r+c} \det M_*$$

(and $(-1)^{-2} = 1$). Its first row is $(1, 0, \ldots, 0)$, and removing its first row and column leaves $M^{(rc)}$, because the other rows and columns keep their order. In the Leibniz formula of M' only the

permutations with $\pi(1)=1$ have a nonzero factor in row 1. Such a π has the smallest value 1 at the first place, so the first place forms no inversion with any later place, and its sign is the sign of its restriction to the places $2,\dots,p$, which runs through all permutations of these places. Hence $\det M' = 1\cdot\det M^{(rc)}$, and

$$C_{rc} = \det M_* = (-1)^{r+c}\det M' = (-1)^{r+c}\det M^{(rc)}$$

(the last two equations together, multiplied by $(-1)^{r+c}$, whose square is 1). Inserting this into the grouping gives the rule along row r. Along a column, apply the rule to the transpose (rule 2). □

The expansion of every Lovelock tensor (PROVED). Write the delta of $P_{(k)}$ as the determinant of the $(2k+1)\times(2k+1)$ matrix M whose rows belong to the lower labels $j, j_1, \dots, j_{2k}$ in this order and whose columns belong to the upper labels $h, h_1, \dots, h_{2k}$. Its first column holds $\delta(j,h), \delta(j_1,h), \dots, \delta(j_{2k},h)$. Expand along this column:

$$\delta^{h\,h_1\dots h_{2k}}_{j\,j_1\dots j_{2k}} = \delta^h_j\,\delta^{h_1\dots h_{2k}}_{j_1\dots j_{2k}} + \sum_{s=1}^{2k}(-1)^{(s+1)+1}\,\delta^h_{j_s}\,\delta^{h_1\dots h_{2k}}_{j\,j_1\dots j_{s-1}j_{s+1}\dots j_{2k}}$$

(Laplace's rule along column 1: the entry in row 1 has the sign $(-1)^{1+1}=+1$ and the minor with the rows $j_1,\dots,j_{2k}$; the entry $\delta(j_s,h)$ stands in row $s+1$, and its minor keeps the rows $j, j_1, \dots, j_{s-1}, j_{s+1}, \dots, j_{2k}$). In the minor, move the row of j down past the $s-1$ rows $j_1,\dots,j_{s-1}$:

$$\delta^{h_1\dots h_{2k}}_{j\,j_1\dots j_{s-1}j_{s+1}\dots j_{2k}} = (-1)^{s-1}\,\delta^{h_1\dots h_{2k}}_{j_1\dots j_{s-1}\,j\,j_{s+1}\dots j_{2k}}$$

($s-1$ exchanges of neighbouring rows, each a factor -1 by rule 3; now j stands at the place s, where j_s stood). The two signs combine to $(-1)^{s+2}(-1)^{s-1}=(-1)^{2s+1}=-1$, so

$$\delta^{h\,h_1\dots h_{2k}}_{j\,j_1\dots j_{2k}} = \delta^h_j\,\delta^{h_1\dots h_{2k}}_{j_1\dots j_{2k}} - \sum_{s=1}^{2k}\delta^h_{j_s}\,\delta^{h_1\dots h_{2k}}_{j_1\dots(j\text{ at place }s)\dots j_{2k}}.$$

Multiply by the product of the curvature factors and sum over the $4k$ labels. The first term gives $\delta^h_j L_{(k)}$ (the definition of the scalar). In the term s, the factor $\delta^h_{j_s}$ keeps only $j_s=h$ in the sum over j_s, so h takes the place of j_s in its curvature factor:

$$P_{(k)}{}^h{}_j = \delta^h_j L_{(k)} - \sum_{s=1}^{2k} T_s,$$

$$T_s = \sum \delta^{h_1\dots h_{2k}}_{j_1\dots(j\text{ at place }s)\dots j_{2k}}\; R^{j_1j_2}{}_{h_1h_2}\cdots R^{j_{2k-1}j_{2k}}{}_{h_{2k-1}h_{2k}} \quad (\text{with } h \text{ in place of } j_s).$$

The $2k$ terms are equal. (i) Let s be even, the second upper place of its factor $R^{j_{s-1}h}{}_{h_{s-1}h_s}$. Exchange the rows of j_{s-1} and j in the delta (the places $s-1$ and s), a factor -1 (rule 3), and write $R^{j_{s-1}h}{}_{h_{s-1}h_s} = -R^{hj_{s-1}}{}_{h_{s-1}h_s}$, another factor -1 (antisymmetry). The term is unchanged, and now j stands at the place $s-1$ and h first in its factor: $T_s = T_{s-1}$. (ii) Let $s=2m-1$ be odd with $m>1$, the first place of the m-th factor. Exchange the m-th factor with the first factor: the product of the curvature factors is unchanged (numbers commute). In the delta this exchanges

the pair of rows at the places $2m-1, 2m$ with the pair at the places 1, 2, which is two exchanges of rows, a factor $(-1)^2 = +1$, and the pair of columns h_{2m-1}, h_{2m} with h_1, h_2, again $+1$. After renaming the summed labels the term is T_1. So all $2k$ terms equal T_1, and

$$P_{(k)}{}^{h}{}_{j} = \delta^{h}_{j}\, L_{(k)} - 2k\, Y_{(k)}{}^{h}{}_{j}, \qquad Y_{(k)}{}^{h}{}_{j} = \sum \delta^{h_1 \ldots h_{2k}}_{j\, j_2 \ldots j_{2k}}\, R^{h j_2}{}_{h_1 h_2}\, R^{j_3 j_4}{}_{h_3 h_4} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}}.$$

Notebook 11b computes $Y_{(k)}$ with its own GKD sum and checks this expansion exactly for all 64 components and $k = 1, 2, 3$ (In [21]). The Revision's sympy checker derived the same expansion, along the row of h in its convention (`Revision/gkd_lovelock/verification/check_lovelock_gkd.py`, module docstring).

11.14 Order 1 is Einstein's tensor; the trace identity

The pieces of order 1 (PROVED). For $k = 1$ the delta in $Y_{(1)}$ has two labels, and by the formula of Section 11.3

$$Y_{(1)}{}^{h}{}_{j} = \sum_{h_1, h_2, j_2} (\delta^{h_1}_{j}\delta^{h_2}_{j_2} - \delta^{h_2}_{j}\delta^{h_1}_{j_2})\, R^{h j_2}{}_{h_1 h_2}$$

(the 2×2 delta with the lower list (j, j_2) and the upper list (h_1, h_2)),

$$= \sum_{j_2} (R^{h j_2}{}_{j j_2} - R^{h j_2}{}_{j_2 j}) = 2 \sum_{j_2} R^{h j_2}{}_{j j_2} = 2R^{h}{}_{j}$$

(each Kronecker delta keeps one value of a summed label; then the antisymmetry $R^{h j_2}{}_{j_2 j} = -R^{h j_2}{}_{j j_2}$; then the definition of the Ricci tensor). In the same way

$$L_{(1)} = \sum (\delta^{h_1}_{j_1}\delta^{h_2}_{j_2} - \delta^{h_2}_{j_1}\delta^{h_1}_{j_2}) R^{j_1 j_2}{}_{h_1 h_2} = \sum_{h_1, h_2} (R^{h_1 h_2}{}_{h_1 h_2} - R^{h_2 h_1}{}_{h_1 h_2}) = 2 \sum_{h_1, h_2} R^{h_1 h_2}{}_{h_1 h_2} = 2R$$

(the same steps; $\sum_{a,b} R^{ab}{}_{ab} = \sum_a R^{a}{}_{a} = R$).

$P_{(1)} = -4G$ and $E_{(1)} = G$ (PROVED). The expansion of Section 11.13 with $k = 1$ gives

$$P_{(1)}{}^{h}{}_{j} = \delta^{h}_{j} \cdot 2R - 2 \cdot 2R^{h}{}_{j} = -4(R^{h}{}_{j} - \tfrac{1}{2}\delta^{h}_{j} R) = -4\, G^{h}{}_{j}$$

(insert $L_{(1)} = 2R$ and $Y_{(1)} = 2R^{h}{}_{j}$; take out -4; the definition of G), and therefore $E_{(1)} = -P_{(1)}/2^2 = G$. This holds for every metric. For the author's metric the record checks it in all 64 components (`lovelock-report.json` and `python-lovelock-report.json`, check `k1_equals_minus_4_einstein`; `wolfram-gkd-report.json`, check `k1_equals_minus_4_einstein`), and so does Notebook 11b (In [18]). With the Einstein tensor of Section 11.10 the components of $P_{(1)}$ are -4 times those of G: for example $P_{(1)}{}^{x_1}{}_{x_1} = 12a_4'^2 - 4a_4'' - 60H^2$, as the Wolfram record lists (`wolfram-gkd-report.json`, field `measurements`, entry `wolframComponents`). And $L_{(1)} = 2R = 12a_4'^2 - 84H^2$ (record `Revision/gkd_lovelock/results/lovelock-tensors.json`, key `L1`; Notebook 11b, In [22]).

The trace identity (PROVED). Set $j = h$ in the expansion of Section 11.13 and sum over the eight values of h:

$$\sum_h P_{(k)}{}^h{}_h = \Big(\sum_h \delta^h_h\Big) L_{(k)} - 2k \sum_h Y_{(k)}{}^h{}_h = 8\, L_{(k)} - 2k\, L_{(k)} = (8 - 2k)\, L_{(k)}$$

($\delta^h_h = 1$ for each of the eight values; in $\sum_h Y_{(k)}{}^h{}_h$ the label h is summed exactly like j_1 in the definition of the scalar, so renaming it j_1 gives $L_{(k)}$). The factors are 6, 4 and 2 for $k = 1, 2, 3$, the factors of Section 1.44, found there by contracting the delta directly. The record checks the identity exactly (`lovelock-report.json`, checks `k1_trace_identity` to `k3_trace_identity`; `wolfram-gkd-report.json`, checks `k1_trace_equals_6_L1`, `k2_trace_equals_4_L2`, `k3_trace_equals_2_L3`), and so does Notebook 11b (In [22]). For $k = 1$: the trace of $P_{(1)} = -4G$ is $-4(R - 4R) = 12R = 6 \cdot 2R$, as it must be.

11.15 Order 2 is the Gauss-Bonnet tensor; order 3 is the cubic Lovelock density

The piece $Y_{(2)}$, line by line (PROVED). Write $X^h{}_j = Y_{(2)}{}^h{}_j = \sum \delta^{h_1 h_2 h_3 h_4}_{j\, j_2 j_3 j_4}\, R^{h j_2}{}_{h_1 h_2}\, R^{j_3 j_4}{}_{h_3 h_4}$. The delta is a 4×4 determinant with the rows j, j_2, j_3, j_4 and the columns h_1, h_2, h_3, h_4. Expand it along its first row, the row of j, whose entries are $\delta(j, h_c) = \delta^{h_c}_j$ with the signs $+, -, +, -$ (Laplace's rule, $(-1)^{1+c}$). This gives four terms; in each, the factor $\delta^{h_c}_j$ puts j in place of h_c in the curvature factors. We use the result of order 1 in the form

$$\sum \delta^{a\, c_1 c_2}_{b\, d_1 d_2} R^{d_1 d_2}{}_{c_1 c_2} = P_{(1)}{}^a{}_b = 2R\, \delta^a_b - 4R^a{}_b$$

(Section 11.14, with the labels renamed), and write the summed labels as $b = j_2$, $c = j_3$, $d = j_4$.

Term 1 ($c = 1$, sign $+$): the minor has the rows j_2, j_3, j_4 and the columns h_2, h_3, h_4:

$$\sum R^{h j_2}{}_{j h_2}\, \delta^{h_2 h_3 h_4}_{j_2 j_3 j_4} R^{j_3 j_4}{}_{h_3 h_4} = \sum_{j_2, h_2} R^{h j_2}{}_{j h_2} (2R\, \delta^{h_2}_{j_2} - 4R^{h_2}{}_{j_2}) = 2R\, R^h{}_j - 4 \sum_{b,x} R^{hb}{}_{jx} R^x{}_b$$

(the order-1 result with $a = h_2$, $b = j_2$; then $\delta^{h_2}_{j_2}$ sets $h_2 = j_2$ and $\sum_{j_2} R^{h j_2}{}_{j j_2} = R^h{}_j$; the summed labels renamed $b = j_2$, $x = h_2$).

Term 2 ($c = 2$, sign $-$): the minor has the columns h_1, h_3, h_4, and h_2 becomes j:

$$-\sum R^{h j_2}{}_{h_1 j} (2R\, \delta^{h_1}_{j_2} - 4R^{h_1}{}_{j_2}) = -2R \sum_{j_2} R^{h j_2}{}_{j_2 j} + 4 \sum R^{h j_2}{}_{h_1 j} R^{h_1}{}_{j_2} = 2R\, R^h{}_j - 4 \sum_{b,x} R^{hb}{}_{jx} R^x{}_b$$

(the same steps; then $R^{h j_2}{}_{j_2 j} = -R^{h j_2}{}_{j j_2}$ and $R^{h j_2}{}_{h_1 j} = -R^{h j_2}{}_{j h_1}$ by antisymmetry). Terms 1 and 2 together: $4R\, R^h{}_j - 8 \sum R^{hb}{}_{jx} R^x{}_b$.

Term 3 ($c = 3$, sign $+$): h_3 becomes j, and the minor $\delta^{h_1 h_2 h_4}_{j_2 j_3 j_4}$ has the rows j_2, j_3, j_4 and the columns h_1, h_2, h_4. Expand it along its third column, the column of h_4, with the signs $+, -, +$:

$$\delta^{h_1 h_2 h_4}_{j_2 j_3 j_4} = \delta^{h_4}_{j_2}\, \delta^{h_1 h_2}_{j_3 j_4} - \delta^{h_4}_{j_3}\, \delta^{h_1 h_2}_{j_2 j_4} + \delta^{h_4}_{j_4}\, \delta^{h_1 h_2}_{j_2 j_3}$$

(Laplace's rule along column 3; the sign of row r is $(-1)^{r+3}$). Multiply by $R^{hj_2}{}_{h_1h_2}R^{j_3j_4}{}_{jh_4}$ and sum, using $\sum_{h_1,h_2}\delta^{h_1h_2}_{uv}R^{hj_2}{}_{h_1h_2} = R^{hj_2}{}_{uv} - R^{hj_2}{}_{vu} = 2R^{hj_2}{}_{uv}$:

$$2\sum R^{hj_2}{}_{j_3j_4}R^{j_3j_4}{}_{jj_2} - 2\sum R^{hj_2}{}_{j_2j_4}R^{j_3j_4}{}_{jj_3} + 2\sum R^{hj_2}{}_{j_2j_3}R^{j_3j_4}{}_{jj_4}$$

(each δ^{h_4} puts the matching lower label in place of h_4). In the second sum $\sum_{j_2} R^{hj_2}{}_{j_2j_4} = -R^h{}_{j_4}$ and $\sum_{j_3} R^{j_3j_4}{}_{jj_3} = -R^{j_4}{}_j$; in the third $\sum_{j_2} R^{hj_2}{}_{j_2j_3} = -R^h{}_{j_3}$ and $\sum_{j_4} R^{j_3j_4}{}_{jj_4} = R^{j_3}{}_j$ (antisymmetry, then the definition of the Ricci tensor). So term 3 is

$$2\sum R^{hb}{}_{cd}R^{cd}{}_{jb} - 2\sum_c R^h{}_cR^c{}_j - 2\sum_c R^h{}_cR^c{}_j = 2\sum R^{hb}{}_{cd}R^{cd}{}_{jb} - 4\sum_c R^h{}_cR^c{}_j.$$

Term 4 ($c = 4$, sign $-$): h_4 becomes j, the factor is $R^{j_3j_4}{}_{h_3j} = -R^{j_3j_4}{}_{jh_3}$, and the two minus signs give $+\sum \delta^{h_1h_2h_3}_{j_2j_3j_4}R^{hj_2}{}_{h_1h_2}R^{j_3j_4}{}_{jh_3}$, which is term 3 with h_4 renamed h_3. So term 4 equals term 3. Adding the four terms:

$$X^h{}_j = 4R\,R^h{}_j - 8\sum R^{hb}{}_{jx}R^x{}_b - 8\sum_c R^h{}_cR^c{}_j + 4\sum R^{hb}{}_{cd}R^{cd}{}_{jb}.$$

$L_{(2)}$ is four times the Gauss-Bonnet scalar (PROVED). By the trace argument of Section 11.14, $L_{(2)} = \sum_h X^h{}_h$:

$$\begin{aligned} L_{(2)} &= 4R^2 - 8\sum R^b{}_xR^x{}_b - 8\sum R^h{}_cR^c{}_h + 4\sum R^{ab}{}_{cd}R^{cd}{}_{ab} \\ &= 4\Big(R^2 - 4\sum_{a,b} R^a{}_bR^b{}_a + \sum R^{ab}{}_{cd}R^{cd}{}_{ab}\Big) = 4\,\mathrm{GB} \end{aligned}$$

($\sum_h R^{hb}{}_{hx} = R^b{}_x$; the two Ricci squares are the same sum; then 4 taken out). $\mathrm{GB} = R^2 - 4R^a{}_bR^b{}_a + R^{ab}{}_{cd}R^{cd}{}_{ab}$ is the classical **Gauss-Bonnet scalar**.

$P_{(2)}$ is -8 times the Gauss-Bonnet tensor (PROVED). The expansion of Section 11.13 with $k = 2$:

$$P_{(2)}{}^h{}_j = \delta^h_j\,4\,\mathrm{GB} - 4X^h{}_j = -16R\,R^h{}_j + 32\sum R^{hb}{}_{jx}R^x{}_b + 32\sum_c R^h{}_cR^c{}_j - 16\sum R^{hb}{}_{cd}R^{cd}{}_{jb} + 4\,\delta^h_j\,\mathrm{GB}$$

(insert $L_{(2)}$ and X; multiply out). The classical Gauss-Bonnet tensor of Lanczos, which uses no generalized delta, is (in the mixed form of the record)

$$2\mathcal{H}^h{}_j = 4\Big(R\,R^h{}_j - 2\sum_a R^h{}_aR^a{}_j - 2\sum_{a,b} R^{ha}{}_{jb}R^b{}_a + \sum_{a,b,c} R^{ha}{}_{bc}R^{bc}{}_{ja}\Big) - \delta^h_j\,\mathrm{GB}.$$

Multiplying by -4 gives $-8\mathcal{H}^h{}_j = -16R\,R^h{}_j + 32\sum R^h{}_aR^a{}_j + 32\sum R^{ha}{}_{jb}R^b{}_a - 16\sum R^{ha}{}_{bc}R^{bc}{}_{ja} + 4\delta^h_j\,\mathrm{GB}$, which is $P_{(2)}{}^h{}_j$ term by term after renaming the summed labels ($b \to a$, $x \to b$ in the second term; $b \to a$, $c \to b$, $d \to c$ in the fourth). So $P_{(2)} = -8\mathcal{H}$ and $E_{(2)} = -P_{(2)}/8 = \mathcal{H}$ for every metric: the second Lovelock tensor is the Gauss-Bonnet tensor. For the author's metric the record checks it

in all 64 components (`python-lovelock-report.json`, checks `k2_equals_minus_8_gauss_bonnet` and `L2_equals_4_gauss_bonnet`), and so does Notebook 11b (In [19]). The record also found the factors -4 and -8 without any algebra, from literal sums on random curvature tensors in four and five dimensions (checks `normalisation_P1_derived_minus_4` and `normalisation_P2_derived_minus_8`). With $L_{(2)} = -96a_4'^4 - 2112a_4'^2H^2 + 3360H^4$ (record key `L2`), the Gauss-Bonnet scalar of the author's metric is $\mathrm{GB} = L_{(2)}/4 = -24a_4'^4 - 528a_4'^2H^2 + 840H^4$.

Order 3 (PROVED for the author's metric by exact computation; the general formula quoted). The third scalar is the **cubic Lovelock density**,

$$L_{(3)} = 8\big(2T_1 + 8T_2 + 24T_3 + 3T_4 + 24T_5 + 16T_6 - 12T_7 + T_8\big),$$

with the eight cubic invariants $T_1 = R^{ab}{}_{cd}R^{cd}{}_{ef}R^{ef}{}_{ab}$, $T_2 = R^{ab}{}_{cd}R^{ce}{}_{bf}R^{df}{}_{ae}$, $T_3 = R^{ab}{}_{cd}R^{cd}{}_{be}R^{e}{}_{a}$, $T_4 = R\,R^{ab}{}_{cd}R^{cd}{}_{ab}$, $T_5 = R^{ab}{}_{cd}R^{c}{}_{a}R^{d}{}_{b}$, $T_6 = R^{a}{}_{b}R^{b}{}_{c}R^{c}{}_{a}$, $T_7 = R\,R^{a}{}_{b}R^{b}{}_{a}$, $T_8 = R^3$ (every label summed). We do not derive these coefficients by hand: the record determined them from literal sums on nine random curvature tensors in six dimensions (a system of nine equations for eight unknowns with a unique solution; `python-lovelock-report.json`, check `normalisation_L3_cubic_derived`), and checked the identity exactly for the author's metric (check `L3_equals_8_cubic_lovelock_density`); Notebook 11b checks it again (In [20]). For the author's metric $L_{(3)} = 1152a_4'^6 + 31104a_4'^4H^2 + 100224a_4'^2H^4 - 40320H^6$ (record key `L3`).

11.16 Example: the three Lovelock tensors computed twice

Notebook 11b computes the three Lovelock tensors of the author's metric in two independent ways and checks Sections 11.10 to 11.15. It builds and runs the Revision Rust program `lovelock_gkd`, which computes everything exactly from the metric alone (the author's metric of 2026-10-14, with its second function $b_4(x_4)$), runs its 19 checks and writes four files; the notebook compares three of the four files with the committed Revision records byte for byte, and the fourth, the report `lovelock-report.json`, apart from its two floating-point numbers (the rounding errors measured by its two brute-force checks, whose last digits may depend on the computer). Then it recomputes the Riemann tensor with sympy, for a general b_4, and compares all 156 nonzero components with the record; from there on it works on the background $b_4 = 1$; draws the plane curvatures, the whole Riemann tensor and the mixed terms, which cancel at $b_4 = 1$ and leave $\frac{3}{2}H\cot z\, b_4'/b_4$ with b_4; recomputes the three tensors and scalars with its own Python GKD sum and its own exact arithmetic, reproduces every counter of the Rust sums and all 3×64 components; repeats the sums of orders 1 and 2 literally; checks the expansion of Section 11.13, the identities $P_{(1)} = -4G$, $P_{(2)} = -8\mathcal{H}$, the cubic density, the trace identities and $P_{(4)} = 0$; draws the components of $E_{(1)}, E_{(2)}, E_{(3)}$ at one moment of a deflating history; and checks that every check it quotes from the records is there. It needs Rust and takes about a minute (45.7 seconds when it was built on 2026-10-15; provenance file `Revision/textbook/notebooks/11b_lovelock_tensors.PROVENANCE.md`). It prints 38 PASS lines, draws five figures and ends with the line ALL 38 CHECKS PASSED (notebook 11b).

11.17 How to run Notebook 11b

Step 1. What this notebook does and what it needs.

Notebook 11b (The three Lovelock tensors of the author's metric, computed with GKD) is the file `Revision/textbook/notebooks/11b_lovelock_tensors.ipynb` of the repository Dirac_claude. It builds the Revision Rust program lovelock_gkd with cargo inside the notebook when the program file is missing (a few seconds, up to a minute on a slow computer; about a second when it is up to date) and runs it; the program computes exactly the curvature of the author's metric and the three Lovelock tensors of Lovelock's equation (4.38) in eight dimensions with GKD, and the notebook checks that its four output files are byte for byte the committed Revision records (the report apart from two floating-point rounding errors, which must stay below 1e-10). Then it recomputes the Riemann tensor with sympy and the three Lovelock tensors and scalars with its own Python GKD sum, compares every component exactly with the records, reproduces the counters of the Rust sums, repeats the sums of orders 1 and 2 literally without skipping any term, derives and checks the expansion of every Lovelock tensor along the column of its free upper label, and checks that the first Lovelock tensor is -4 times the Einstein tensor, the second -8 times the Gauss-Bonnet tensor, the third scalar 8 times the cubic Lovelock density, that the traces obey the trace identity and that the fourth tensor vanishes; it checks again that every check it quotes from the Revision records is there with the verdict PASS; it draws five teaching figures, among them the components of the three tensors. The Rust program writes its four output files (curvature.json, lovelock-tensors.json, lovelock-components.md and lovelock-report.json) into the folder `Revision/gkd_lovelock/code/target/textbook_11b`, inside the Rust build folder, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program lovelock_gkd, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and

accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/gkd_lovelock/code/target/release/lovelock_gkd` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 11b_lovelock_tensors.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 11b_lovelock_tensors.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 11b_lovelock_tensors.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/11b.captions.json`
- `Revision/textbook/figures/11b_1_work_of_the_sums.png`
- `Revision/textbook/figures/11b_2_curvature_of_planes.png`
- `Revision/textbook/figures/11b_3_riemann_matrix.png`
- `Revision/textbook/figures/11b_4_mixing_terms_cancel.png`
- `Revision/textbook/figures/11b_5_lovelock_component_tables.png`

It changes no other file of the repository except the Rust build folder `target` next to each `Cargo.toml` it builds (inside it the notebook keeps the raw output of its program runs in `Revision/gkd_lovelock/code/target/textbook_11b`, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files of the notebook exist
ALL 38 CHECKS PASSED (notebook 11b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/11b_lovelock_tensors.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 11b_lovelock_tensors.ipynb
python -m nbconvert --execute --inplace 11b_lovelock_tensors.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- AssertionError: check failed: the program wrote curvature.json equal to the Revision record byte for byte (or the same message for one of the other three files): The file that the program wrote differs from the committed record. Run git status in the repository folder to see whether the records or the Rust source were changed, and restore both with the command below.

```
git checkout -- Revision/gkd_lovelock
```

11.18 Notebook 11b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/11b_lovelock_tensors.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Einstein's field equations equate the Einstein tensor $G$, built from the curvature
of space-time, to the energy and momentum of matter. In eight dimensions Lovelock's
theorem allows two more tensors of the same kind, built from products of two and of
three curvature tensors. Together with Einstein's they are the three **Lovelock
tensors** $P_{(1)}, P_{(2)}, P_{(3)}$ of Lovelock's equation (4.38), which the
author's notebook quotes. This notebook computes them for the author's metric, in
two independent ways, and checks them. It

- builds the Revision Rust program `lovelock_gkd` with cargo and runs it: from the
  metric alone it computes exactly the Christoffel symbols, the Riemann tensor and
  the three Lovelock tensors (with GKD, the generalized Kronecker delta), checks 19
  identities, and writes four files; the notebook checks that the four files are,
  byte for byte, the committed Revision records;
- draws how much work the sums are, and how the program reduces it;
- recomputes the Riemann tensor with sympy and compares all 156 nonzero components
  exactly with the record; draws the curvature of every coordinate plane and the
  whole Riemann tensor, and shows why the mixed $x_4$-$x_0$ terms cancel;
- recomputes the three Lovelock tensors and the three Lovelock scalars in Python
  with its own GKD sum and its own exact polynomial arithmetic, reproduces the
  counters of the Rust sums, and compares all $3 \times 64$ components exactly;
- repeats the sums of orders 1 and 2 LITERALLY, calling GKD on every ordered
  product of curvature entries without skipping anything, and finds the same
  tensors: the skipping changes nothing;
- derives, line by line, the expansion of every Lovelock tensor along the column
  of its free upper label, and checks it exactly for $k = 1, 2, 3$;
- checks three classical identities: $P_{(1)} = -4\,G$ (Einstein),
  $P_{(2)} = -8\,\mathcal{H}$ with the Gauss-Bonnet tensor $\mathcal{H}$, and
  $L_{(3)} = 8 \times$ the cubic Lovelock density; the trace identities; and
  $P_{(4)} = 0$;
- draws the components of the three normalised Lovelock tensors at one moment of
  a deflating history;
- reads the Revision verification reports once more and checks that every check
  it quotes from them is there with the verdict PASS.

It draws five figures and takes about one and a half minutes.
```

Text cell:

```
## 2. How to run this notebook
```

Step 1. What this notebook does and what it needs.

Notebook 11b (The three Lovelock tensors of the author's metric, computed with GKD) is the file ‘Revision/textbook/notebooks/11b_lovelock_tensors.ipynb‘ of the repository Dirac_claude. It builds the Revision Rust program lovelock_gkd with cargo inside the notebook when the program file is missing (a few seconds, up to a minute on a slow computer; about a second when it is up to date) and runs it; the program computes exactly the curvature of the author's metric and the three Lovelock tensors of Lovelock's equation (4.38) in eight dimensions with GKD, and the notebook checks that its four output files are byte for byte the committed Revision records (the report apart from two floating-point rounding errors, which must stay below 1e-10). Then it recomputes the Riemann tensor with sympy and the three Lovelock tensors and scalars with its own Python GKD sum, compares every component exactly with the records, reproduces the counters of the Rust sums, repeats the sums of orders 1 and 2 literally without skipping any term, derives and checks the expansion of every Lovelock tensor along the column of its free upper label, and checks that the first Lovelock tensor is -4 times the Einstein tensor, the second -8 times the Gauss-Bonnet tensor, the third scalar 8 times the cubic Lovelock density, that the traces obey the trace identity and that the fourth tensor vanishes; it checks again that every check it quotes from the Revision records is there with the verdict PASS; it draws five teaching figures, among them the components of the three tensors. The Rust program writes its four output files (curvature.json, lovelock-tensors.json, lovelock-components.md and lovelock-report.json) into the folder ‘Revision/gkd_lovelock/code/target/textbook_11b‘, inside the Rust build folder, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program lovelock_gkd, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘ again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
    curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
    cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
    cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/gkd_lovelock/code/target/release/lovelock_gkd‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 11b_lovelock_tensors.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 11b_lovelock_tensors.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 11b_lovelock_tensors.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/11b.captions.json‘
- ‘Revision/textbook/figures/11b_1_work_of_the_sums.png‘
- ‘Revision/textbook/figures/11b_2_curvature_of_planes.png‘
- ‘Revision/textbook/figures/11b_3_riemann_matrix.png‘
- ‘Revision/textbook/figures/11b_4_mixing_terms_cancel.png‘
- ‘Revision/textbook/figures/11b_5_lovelock_component_tables.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds (inside it the notebook keeps the raw output of its program runs in ‘Revision/gkd_lovelock/code/target/textbook_11b‘, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files of the notebook exist
ALL 38 CHECKS PASSED (notebook 11b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/11b_lovelock_tensors.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was

  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 11b_lovelock_tensors.ipynb
python -m nbconvert --execute --inplace 11b_lovelock_tensors.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- AssertionError: check failed: the program wrote curvature.json equal to the Revision record byte for byte (or the same message for one of the other three files): The file that the program wrote differs from the committed record. Run git status in the repository folder to see whether the records or the Rust source were changed, and restore both with the command below.

```
git checkout -- Revision/gkd_lovelock
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 11b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 11b (The three Lovelock tensors of the author’s metric, computed with GKD) is
```

```
# the file Revision/textbook/notebooks/11b_lovelock_tensors.ipynb of the repository
# Dirac_claude. It builds the Revision Rust program lovelock_gkd with cargo inside the
# notebook when the program file is missing (a few seconds, up to a minute on a slow
# computer; about a second when it is up to date) and runs it; the program computes
# exactly the curvature of the author's metric and the three Lovelock tensors of
# Lovelock's equation (4.38) in eight dimensions with GKD, and the notebook checks that
# its four output files are byte for byte the committed Revision records (the report
# apart from two floating-point rounding errors, which must stay below 1e-10). Then it
# recomputes the Riemann tensor with sympy and the three Lovelock tensors and scalars
# with its own Python GKD sum, compares every component exactly with the records,
# reproduces the counters of the Rust sums, repeats the sums of orders 1 and 2 literally
# without skipping any term, derives and checks the expansion of every Lovelock tensor
# along the column of its free upper label, and checks that the first Lovelock tensor is
# -4 times the Einstein tensor, the second -8 times the Gauss-Bonnet tensor, the third
# scalar 8 times the cubic Lovelock density, that the traces obey the trace identity and
# that the fourth tensor vanishes; it checks again that every check it quotes from the
# Revision records is there with the verdict PASS; it draws five teaching figures, among
# them the components of the three tensors. The Rust program writes its four output files
# (curvature.json, lovelock-tensors.json, lovelock-components.md and
# lovelock-report.json) into the folder Revision/gkd_lovelock/code/target/textbook_11b,
# inside the Rust build folder, which git ignores. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It also needs Rust (the program
# cargo, version 1.91.1 or newer), because it runs the Rust program lovelock_gkd, which
# is part of the repository and is built on your computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
```

```
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
```

```
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
#
# Windows, macOS and Linux:
#     cargo --version
#
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
```

```
#     cargo build --release --manifest-path Revision/gkd_lovelock/code/Cargo.toml
#
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
# Revision/gkd_lovelock/code/target/release/lovelock_gkd (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 11b_lovelock_tensors.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 11b_lovelock_tensors.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 11b_lovelock_tensors.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/11b.captions.json
# - Revision/textbook/figures/11b_1_work_of_the_sums.png
# - Revision/textbook/figures/11b_2_curvature_of_planes.png
# - Revision/textbook/figures/11b_3_riemann_matrix.png
# - Revision/textbook/figures/11b_4_mixing_terms_cancel.png
# - Revision/textbook/figures/11b_5_lovelock_component_tables.png
```

```
#
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds (inside it the notebook keeps the raw output of its program
# runs in Revision/gkd_lovelock/code/target/textbook_11b, which git ignores); running it
# headless or saving it in JupyterLab also rewrites the notebook file itself. It does not
# use the internet while it runs. The files it writes are the same files that are stored
# in the repository (on another computer a figure may differ in a few bytes, which is
# harmless). To get the stored versions back, run this command in the repository folder
# (it also undoes every change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all five figure files of the notebook exist
#     ALL 38 CHECKS PASSED (notebook 11b)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/11b_lovelock_tensors.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 11b_lovelock_tensors.ipynb
#     python -m nbconvert --execute --inplace 11b_lovelock_tensors.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
```

```
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - AssertionError: check failed: the program wrote curvature.json equal to the Revision
#   record byte for byte (or the same message for one of the other three files): The file
#   that the program wrote differs from the committed record. Run git status in the
#   repository folder to see whether the records or the Rust source were changed, and
#   restore both with the command below.
#     git checkout -- Revision/gkd_lovelock
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "11b"  # this notebook: chapter 11, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
```

```
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
```

```
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
```

```
    "defined.")
```

Out [1]:

```
Set-up of notebook 11b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$ of the author's metric: $x_1, x_2, x_3$ ordinary
  space, $x_4$ the time, $x_5, x_6, x_7$ the three extra times, $x_0$ the hidden
  direction. The programs number them $0, \dots, 7$.
- **Metric** $g_{ab}$: the table of numbers that gives lengths and times; here it is
  diagonal (only $g_{11}, \dots, g_{00}$ are not zero). **Inverse metric**
  $g^{aa} = 1/g_{aa}$ for a diagonal metric.
- **Index up, index down**: a tensor component carries upper and lower coordinate
  labels, such as $R^{ab}{}_{cd}$; a label is raised with the inverse metric.
- **Christoffel symbol** $\Gamma^a{}_{bc}$: the combination of first derivatives of
  the metric that says how the coordinate directions turn from point to point.
- **Riemann tensor** $R^a{}_{bcd}$: the combination of Christoffel symbols and their
  derivatives that measures curvature. We use mostly $R^{ab}{}_{cd} =
  g^{bb} R^a{}_{bcd}$, which changes sign when $a, b$ or $c, d$ are exchanged.
- **Curvature of a plane** $K(a, b) = R^{ab}{}_{ab}$ (no sum): the curvature of the
  surface spanned by the directions $x_a$ and $x_b$.
- **Ricci tensor** $R^h{}_j = \sum_a R^{ha}{}_{ja}$, **Ricci scalar**
  $R = \sum_h R^h{}_h$, **Einstein tensor** $G^h{}_j = R^h{}_j - \tfrac12
  \delta^h{}_j R$.
- **Generalized Kronecker delta, GKD**: the number $+1$, $-1$ or $0$ that weights
  every term of the Lovelock sums (built and tested in Notebook 11a).
- **Lovelock tensor** $P_{(k)}{}^h{}_j$ of order $k$, **Lovelock scalar**
  $L_{(k)}$: defined in section 4. **Gauss-Bonnet tensor**: the classical name of
  the order-2 Lovelock tensor.
- **Exact**: computed with whole numbers and fractions, without rounding.
  **Polynomial, monomial**: a polynomial is a sum of monomials, each a number times
  powers of symbols such as $H^2 a_4'^2$; a **Laurent polynomial** may also contain
  negative powers, such as $\cot^{-1}$.
- **Brute force**, **literal sum**: a computation that tries every case without any
  shortcut. **Skipping** (pruning): leaving out terms that are known to be zero.
- **Expansion of a determinant along a column** (Laplace's rule): the determinant
  equals the sum, over the entries of one column, of the entry times
  $(-1)^{\text{row} + \text{column}}$ times the determinant of the smaller matrix
  that is left when the entry's row and column are removed (its **minor**).
  Exchanging two neighbouring rows of a matrix changes the sign of its determinant.
- **Revision record**: a file committed under 'Revision/' that holds a result.
  **Byte for byte**: two files equal in every byte.
```

Text cell:

```
## 4. The physical and mathematical situation
```

The metric. The author's primordial metric of 2026-10-14 is diagonal,

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4} s,\ e^{2a_4} s,\ e^{2a_4} s,\ -1,\
  -e^{-2a_4} b_4 s,\ -e^{-2a_4} b_4 s,\ -e^{-2a_4} b_4 s\big),\qquad
  s = \sin^{1/3} z,\quad z = 6 H x_0,$$

in the order $x_0, \dots, x_7$, with a constant $H > 0$ and two functions $a_4(x_4)$ and $b_4(x_4) > 0$ of the time; $0 < z < \pi/2$. Lengths along ordinary space carry the scale factor $e^{a_4} \sin^{1/6} z$ (it INFLATES as a_4 grows), lengths along the three extra times the factor $e^{-a_4} b_4^{1/2} \sin^{1/6} z$ (they DEFLATE exponentially), and $\sqrt{|\det g|} = b_4^{3/2}\cos z$. The background of the Revision records is $b_4 = 1$. Primes are derivatives with respect to the time: $a_4' = da_4/dx_4$, $a_4'' = d^2a_4/dx_4^2$. Below, A_1 stands for a_4', A_2 for a_4'', B_0, B_1, B_2 for b_4, b_4', b_4'', and C for $\cot z$.

Curvature (the convention of Misner, Thorne and Wheeler, used by the Revision):

$$\Gamma^a{}_{bc} = \tfrac12 g^{aa}\big(\partial_b g_{ac} + \partial_c g_{ab}
  - \partial_a g_{bc}\big),$$

$$R^a{}_{bcd} = \partial_c \Gamma^a{}_{db} - \partial_d \Gamma^a{}_{cb}
  + \sum_e \big(\Gamma^a{}_{ce} \Gamma^e{}_{db} - \Gamma^a{}_{de} \Gamma^e{}_{cb}\big),
  \qquad R^{ab}{}_{cd} = g^{bb} R^a{}_{bcd}.$$

(For a diagonal metric the sum $g^{ad}(\dots)$ of the general formula keeps only $d = a$.)

Lovelock's equation (4.38) gives, for n dimensions with $n = 2m$ even, the tensors

$$P_{(k)}{}^h{}_j = \sum \delta^{h\, h_1 \dots h_{2k}}_{j\, j_1 \dots j_{2k}}\,
  R^{j_1 j_2}{}_{h_1 h_2} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}},$$

where the sum runs over all $4k$ labels $h_1, \dots, h_{2k}, j_1, \dots, j_{2k}$ and δ is the generalized Kronecker delta with the upper list $(h, h_1, \dots, h_{2k})$ and the lower list $(j, j_1, \dots, j_{2k})$. The orders are $k = 1, \dots, m - 1$; for $n = 8$, $m = 4$ and $k = 1, 2, 3$. The **Lovelock scalar** $L_{(k)}$ is the same sum without the free pair h, j. The equation itself (with constants α_k and λ) is written with the densities $A_{(k)}{}^{lh} = \sqrt{|\det g|}\, g^{ll} P_{(k)}{}^h{}_l$; the usual normalisation is $E_{(k)} = -P_{(k)}/2^{k+1}$, so that $E_{(1)}$ is Einstein's tensor G. The field equations of a_4 built from them are the subject of chapter 12.

Why the sum must be organised. Written literally, each component is a sum over 8^{4k} lists of labels: $8^{12} \approx 6.9 \times 10^{10}$ terms per component for $k = 3$. Two facts make it small. First, a term with a vanishing curvature factor vanishes, so only the 156 nonzero $R^{ab}{}_{cd}$ need to be combined. Second, a term in which a label repeats in the upper or in the lower list has GKD $= 0$ (Notebook 11a), so such combinations are skipped as soon as they appear. Every term that remains is weighted by an explicit call of GKD. The Rust program and the Python code of section 8 both follow this plan, written independently; the program also checks it against the literal, unpruned sum for $k = 1$ and $k = 2$ at a numerical point ("brute force").

Status. The tensors are exact polynomials, computed by two independent programs and verified again by the Revision's sympy and Wolfram checks (records ‘python-lovelock-report.json‘, 49 checks, and ‘wolfram-gkd-report.json‘, 29 checks): PROVED by exact computation. No physical assumption enters this notebook: it is the geometry of the given metric.

Text cell:

5. The Rust program ‘lovelock_gkd‘

The next cell builds the program with cargo, through the helper ‘rust_program‘ of the set-up cell (it runs ‘cargo build --release‘ for the crate ‘Revision/gkd_lovelock/code‘ and returns the path of the program file). When the program file is missing, cargo compiles it: 4 seconds on the computer that built the book, up to a minute on a slow computer. When the program is up to date, cargo only checks that, in about a second. The crate has no dependencies, so cargo downloads nothing.

In [2]:

```
program = rust_program("Revision/gkd_lovelock/code/Cargo.toml", "lovelock_gkd")
```

Out [2]:

```
Rust program lovelock_gkd is built and ready.
```

Text cell:

The next cell runs ‘lovelock_gkd lovelock --output FOLDER --brute-force-k2‘. FOLDER is ‘Revision/gkd_lovelock/code/target/textbook_11b‘, inside the crate's build folder ‘target‘, which git ignores; so the run adds no file to the repository. The program computes everything exactly from the metric, runs its 19 checks (the option ‘--brute-force-k2‘ adds the literal unpruned sum of order 2 over 1,073,741,824 index lists at one numerical point) and writes four files: ‘curvature.json‘, ‘lovelock-tensors.json‘, ‘lovelock-components.md‘ and ‘lovelock-report.json‘. It takes about 15 seconds.

The program prints one line per check (‘PASS - name: detail‘ or ‘FAIL - ...‘), one line per order k with its counters and run time, and an information line with the eighth-order scalar $L_{(4)}$. The cell prints the names of the checks with their verdicts, and $L_{(4)}$ in a readable form (the program writes a_4' as ‘Derivative[1][a4][x4]‘, Mathematica's notation); it leaves out the run times, which differ from run to run. ‘re.match‘ and ‘re.search‘ read parts of a line with a pattern (a regular expression).

In [3]:

```
import re  # regular expressions: patterns that read parts of a line of text

RESULTS = "Revision/gkd_lovelock/results"  # the folder of the Revision records
OUT_FOLDER = REPO / "Revision" / "gkd_lovelock" / "code" / "target" / "textbook_11b"
completed = subprocess.run(
    [str(program), "lovelock", "--output", str(OUT_FOLDER), "--brute-force-k2"],
    capture_output=True, text=True, encoding="utf-8", errors="replace")
rust_checks = []  # (name, verdict) of every check the program printed
for line in completed.stdout.splitlines():
    found = re.match(r"(PASS|FAIL) - (\w+): ", line)
    if found:
        rust_checks.append((found.group(2), found.group(1)))
    if line.startswith("info - L_(4)"):
        l4_text = line.split("): ", 1)[1]  # the formula after the explanation
say("The checks of the Rust program (name, verdict):")
for number, (name, verdict) in enumerate(rust_checks, 1):
    say(f"  {number:2d} {name:34s} {verdict}")
readable = (l4_text.replace("Derivative[1][a4][x4]", "a4'").replace("*", " ")
            .replace("(", "").replace(")", ""))
report("eighth-order scalar printed by the program, L(4)", readable)
check(completed.returncode == 0 and "lovelock: SUCCESS" in completed.stdout,
      "the Rust program lovelock_gkd lovelock ended with SUCCESS")
check(len(rust_checks) == 19 and all(v == "PASS" for _, v in rust_checks),
      "all 19 checks of the Rust program passed",
      record=f"{RESULTS}/lovelock-report.json, checkCount 19, failedCheckCount 0")
```

Out [3]:

```
The checks of the Rust program (name, verdict):
   1 riemann_antisymmetry               PASS
   2 riemann_first_bianchi              PASS
   3 mixed_riemann_free_of_sin_third    PASS
   4 k1_equals_minus_4_einstein         PASS
   5 k1_trace_identity                  PASS
   6 k1_divergence_free                 PASS
   7 k1_symmetric                       PASS
   8 k1_free_of_sin_third               PASS
   9 k2_trace_identity                  PASS
  10 k2_divergence_free                 PASS
  11 k2_symmetric                       PASS
  12 k2_free_of_sin_third               PASS
  13 k3_trace_identity                  PASS
  14 k3_divergence_free                 PASS
  15 k3_symmetric                       PASS
  16 k3_free_of_sin_third               PASS
  17 k4_tensor_vanishes                 PASS
  18 k1_brute_force_numeric             PASS
  19 k2_brute_force_numeric             PASS
RESULT eighth-order scalar printed by the program, L(4) = -13824 H^2 a4'
    Derivative[2][a4][x4] b4[x4]^-3 Derivative[1][b4][x4]^3 + 10368 H^2 a4'^2 b4[x4]^-4
    Derivative[1][b4][x4]^4 - 20736 H^2 a4'^2 b4[x4]^-3 Derivative[1][b4][x4]^2
    Derivative[2][b4][x4] + 82944 H^2 a4'^2 Derivative[2][a4][x4] b4[x4]^-2
    Derivative[1][b4][x4]^2 + 69120 H^2 a4'^3 b4[x4]^-3 Derivative[1][b4][x4]^3 + 55296
```

```
    H^2 a4'^3 b4[x4]^-2 Derivative[1][b4][x4] Derivative[2][b4][x4] - 110592 H^2 a4'^3
    Derivative[2][a4][x4] b4[x4]^-1 Derivative[1][b4][x4] - 511488 H^2 a4'^4 b4[x4]^-2
    Derivative[1][b4][x4]^2 - 27648 H^2 a4'^4 b4[x4]^-1 Derivative[2][b4][x4] + 995328
    H^2 a4'^5 b4[x4]^-1 Derivative[1][b4][x4] - 663552 H^2 a4'^6 - 3456 H^4 b4[x4]^-4
    Derivative[1][b4][x4]^4 + 6912 H^4 b4[x4]^-3 Derivative[1][b4][x4]^2
    Derivative[2][b4][x4] + 27648 H^4 Derivative[2][a4][x4] b4[x4]^-2
    Derivative[1][b4][x4]^2 - 69120 H^4 a4' b4[x4]^-3 Derivative[1][b4][x4]^3 + 55296 H^4
    a4' b4[x4]^-2 Derivative[1][b4][x4] Derivative[2][b4][x4] - 110592 H^4 a4'
    Derivative[2][a4][x4] b4[x4]^-1 Derivative[1][b4][x4] - 27648 H^4 a4'^2 b4[x4]^-2
    Derivative[1][b4][x4]^2 - 55296 H^4 a4'^2 b4[x4]^-1 Derivative[2][b4][x4] + 442368
    H^4 a4'^3 b4[x4]^-1 Derivative[1][b4][x4] - 442368 H^4 a4'^4 + 96768 H^6 b4[x4]^-2
    Derivative[1][b4][x4]^2 - 138240 H^6 b4[x4]^-1 Derivative[2][b4][x4] + 331776 H^6 a4'
    b4[x4]^-1 Derivative[1][b4][x4] - 663552 H^6 a4'^2
PASS the Rust program lovelock_gkd lovelock ended with SUCCESS
PASS all 19 checks of the Rust program passed
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, checkCount 19,
    failedCheckCount 0
```

Text cell:

```
The next cell compares the four files that the program has just written with the
committed Revision records in ‘Revision/gkd_lovelock/results‘, byte for byte
(‘read_bytes()‘ reads a file as a sequence of bytes). Equal bytes mean that the
program, built on this computer, has reproduced every symbol of the Revision’s
results.

The first three files hold only exact whole numbers and fractions, so they must be
equal byte for byte on every computer. The fourth, ‘lovelock-report.json‘, holds
two decimal numbers: the largest relative deviation between the exact $P_{(1)}$,
$P_{(2)}$ and their literal brute-force sums, evaluated in floating point at one
numerical point (on the build computer $1.40 \times 10^{-15}$ and
$4.41 \times 10^{-14}$, rounding errors of the decimal arithmetic). The last digits
of such rounding errors can depend on the mathematics library of the operating
system (the functions $\sin$ and $\cos$). So the cell compares this file byte for
byte after replacing these two numbers by the letter X in both texts
(‘re.sub‘), and checks separately that both numbers are below the program’s own
limit $10^{-10}$. On the computer that built the book the file is equal byte for
byte without any replacement.
```

In [4]:

```
DEVIATION = re.compile(r"(max relative deviation from the exact P_\(\d\) = )"
                       r"([0-9.]+e[-+]?\d+)")  # the two floating-point numbers
for file_name in ("curvature.json", "lovelock-tensors.json",
                  "lovelock-components.md"):
    written = (OUT_FOLDER / file_name).read_bytes()
    stored = repository_file(f"{RESULTS}/{file_name}").read_bytes()
    report(f"size of {file_name}", len(written), "bytes")
    check(written == stored,
          f"the program wrote {file_name} equal to the Revision record byte for byte",
          record=f"{RESULTS}/{file_name} (the whole file)")
written = (OUT_FOLDER / "lovelock-report.json").read_text(encoding="utf-8")
```

```
stored = repository_file(f"{RESULTS}/lovelock-report.json").read_text(
    encoding="utf-8")
deviations = [float(number) for _, number in DEVIATION.findall(written)]
masked_written = DEVIATION.sub(r"\1X", written)  # the two numbers replaced by X
masked_stored = DEVIATION.sub(r"\1X", stored)
report("size of lovelock-report.json", len(written.encode("utf-8")), "bytes")
check(masked_written == masked_stored and len(deviations) == 2
      and all(value < 1e-10 for value in deviations),
      "the program wrote lovelock-report.json equal to the Revision record byte for "
      "byte, apart from its two brute-force deviations, both below 1e-10",
      record=f"{RESULTS}/lovelock-report.json (the whole file; checks "
             "k1_brute_force_numeric and k2_brute_force_numeric)")
```

Out [4]:

```
RESULT size of curvature.json = 40878 bytes
PASS the program wrote curvature.json equal to the Revision record byte for byte
     reproduces Revision/gkd_lovelock/results/curvature.json (the whole file)
RESULT size of lovelock-tensors.json = 163905 bytes
PASS the program wrote lovelock-tensors.json equal to the Revision record byte for byte
     reproduces Revision/gkd_lovelock/results/lovelock-tensors.json (the whole file)
RESULT size of lovelock-components.md = 50784 bytes
PASS the program wrote lovelock-components.md equal to the Revision record byte for byte
     reproduces Revision/gkd_lovelock/results/lovelock-components.md (the whole file)
RESULT size of lovelock-report.json = 3231 bytes
PASS the program wrote lovelock-report.json equal to the Revision record byte for byte,
    apart from its two brute-force deviations, both below 1e-10
     reproduces Revision/gkd_lovelock/results/lovelock-report.json (the whole file;
    checks k1_brute_force_numeric and k2_brute_force_numeric)
```

Text cell:

```
## 6. How much work the sums are

The report ‘lovelock-report.json‘ (just reproduced) lists, for each order $k$, the
**leaves** (the combinations of $k$ nonzero curvature entries that survive the
skipping of repeated labels, for all 64 pairs $(h, j)$), the **GKD calls** (the
leaves whose upper and lower label sets are equal, so that GKD may be nonzero; GKD
is then called) and the number of nonzero GKD values. The next cell reads them and
compares them with two counts without any shortcut: the number of ordered
combinations of $k$ of the 156 nonzero entries for the 64 pairs, $64 \cdot 156^k$,
and the literal number of index lists, $64 \cdot 8^{4k}$. For $k = 4$ nothing
survives: a combination of four entries needs nine different labels in each list,
and there are only eight.
```

In [5]:

```
import numpy as np  # arrays of numbers

lovelock_report = json.loads((OUT_FOLDER / "lovelock-report.json").read_text(
    encoding="utf-8"))
```

```
COUNTERS = {row["k"]: row for row in lovelock_report["counters"]}
say("order k   literal lists   ordered products    leaves   GKD calls   nonzero GKD")
for k in (1, 2, 3, 4):
    leaves, calls = COUNTERS[k]["leaves"], COUNTERS[k]["gkdCalls"]
    nonzero = COUNTERS[k]["gkdNonzero"]
    say(f"      {k}   {64 * 8 ** (4 * k):13.2e}   {64 * 156 ** k:16.2e}   "
        f"{leaves:7d}   {calls:9d}   {nonzero:11d}")

fig, ax = plt.subplots(figsize=(7.6, 4.5))
orders = np.array([1, 2, 3, 4])
series = [("literal index lists, $64 \\cdot 8^{4k}$", [64 * 8 ** (4 * k)
                                                       for k in orders], "0.7"),
          ("ordered products of nonzero entries, $64 \\cdot 156^k$",
           [64 * 156 ** k for k in orders], "tab:purple"),
          ("leaves after skipping repeated labels",
           [COUNTERS[k]["leaves"] for k in orders], "tab:blue"),
          ("GKD calls", [COUNTERS[k]["gkdCalls"] for k in orders], "tab:orange")]
width = 0.2
for number, (name, heights, colour) in enumerate(series):
    place = orders + (number - 1.5) * width
    ax.bar(place, [h if h > 0 else np.nan for h in heights], width, color=colour,
           label=name)
    for x, h in zip(place, heights):
        if h == 0:
            ax.text(x, 1.5, "0", ha="center", fontsize=8)
ax.set_yscale("log")
ax.set_ylim(1.0, 1e18)
ax.set_xlim(0.5, 4.5)
ax.set_xticks(orders, [f"k = {k}" for k in orders])
ax.set_xlabel("order $k$ of the Lovelock tensor")
ax.set_ylabel("number of terms (logarithmic scale)")
ax.set_title("The work of the Lovelock sums, and how it is reduced")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "work_of_the_sums",
            "The number of terms of the Lovelock sums for the 64 components of "
            "$P_{(k)}$, order $k = 1$ to $4$, on a logarithmic vertical axis: all "
            "index lists of the literal formula (grey), the ordered products of $k$ "
            "of the 156 nonzero curvature entries (purple), the combinations left "
            "after skipping those with a repeated label (blue), and the ones for "
            "which GKD is called (orange). For $k = 3$ the literal sum has about "
            "$4.4 \\times 10^{12}$ terms, of which only 495,360 are evaluated; for "
            "$k = 4$ nothing is left, because nine labels out of eight must repeat.")
check([COUNTERS[k]["leaves"] for k in (1, 2, 3, 4)] == [5616, 176640, 1128960, 0]
      and [COUNTERS[k]["gkdCalls"] for k in (1, 2, 3, 4)] == [696, 32640, 495360, 0],
      "the counters of the Rust sums are 5616, 176640, 1128960, 0 leaves and 696, "
      "32640, 495360, 0 GKD calls")
```

Out [5]:

```
order k   literal lists   ordered products    leaves   GKD calls   nonzero GKD
      1        2.62e+05           9.98e+03      5616         696           696
      2        1.07e+09           1.56e+06    176640       32640         32640
      3        4.40e+12           2.43e+08   1128960      495360        495360
```

```
4          1.80e+16            3.79e+10          0              0              0
```

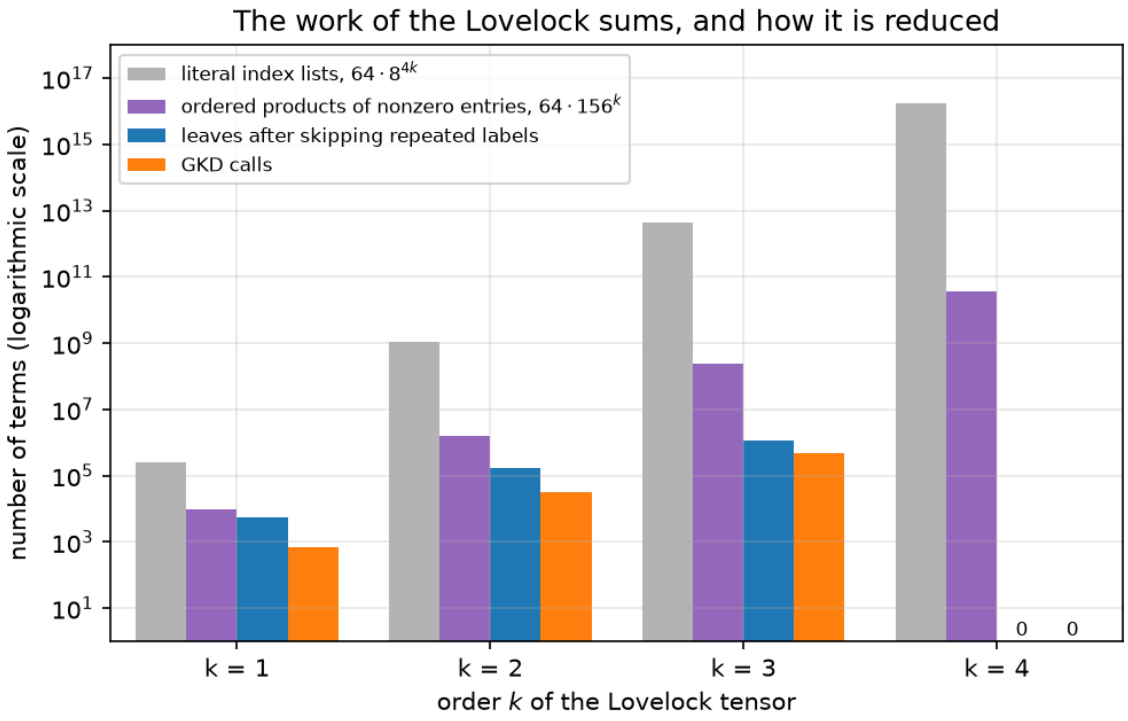

Figure 309: The number of terms of the Lovelock sums for the 64 components of $P_{(k)}$, order $k = 1$ to 4, on a logarithmic vertical axis: all index lists of the literal formula (grey), the ordered products of k of the 156 nonzero curvature entries (purple), the combinations left after skipping those with a repeated label (blue), and the ones for which GKD is called (orange). For $k = 3$ the literal sum has about 4.4×10^{12} terms, of which only 495,360 are evaluated; for $k = 4$ nothing is left, because nine labels out of eight must repeat. (Notebook 11b, figure 1.)

Out [5]:

```
Figure 11b.1 saved as Revision/textbook/figures/11b_1_work_of_the_sums.png
PASS the counters of the Rust sums are 5616, 176640, 1128960, 0 leaves and 696, 32640,
    495360, 0 GKD calls
```

Text cell:

```
## 7. The Riemann tensor, recomputed with sympy

The next two cells compute the curvature of the metric of section 4 with sympy, from
the formulas of section 4, without using anything of the Rust program. The first one
defines the metric and its Christoffel symbols. ‘sp.Function("a4")(x4)‘ and
‘sp.Function("b4")(x4)‘ are unknown functions of the time; ‘sp.diff‘
differentiates exactly. Because the metric is
diagonal, the formula $\Gamma^a{}_{bc} = \tfrac12 g^{aa}(\partial_b g_{ac} +
\partial_c g_{ab} - \partial_a g_{bc})$ has at most three terms, each present only
when two of the labels $a, b, c$ are equal; the cell adds exactly those and keeps the
symbols that are not zero in the dictionary ‘christoffel‘.
```

In [6]:

```
import sympy as sp  # exact algebra and calculus

H = sp.Symbol("H", positive=True)  # the author's constant H > 0
X = sp.symbols("x0:8", real=True)  # the coordinates x0, ..., x7
a4 = sp.Function("a4")(X[4])  # the unknown function a4(x4)
b4 = sp.Function("b4")(X[4])  # the unknown second function b4(x4) > 0
z = 6 * H * X[0]  # z = 6 H x0
warp = sp.sin(z) ** sp.Rational(1, 3)  # sin(z)^(1/3)
METRIC = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * warp] * 3 + [sp.Integer(-1)]
          + [-sp.exp(-2 * a4) * b4 * warp] * 3)  # g_00, g_11, ..., g_77

christoffel = {}  # (a, b, c) -> Gamma^a_bc, only the nonzero ones
for a in range(8):
    for b in range(8):
        for c in range(8):
            value = 0
            if a == c:
                value += sp.diff(METRIC[a], X[b])  # d_b g_ac
            if a == b:
                value += sp.diff(METRIC[a], X[c])  # d_c g_ab
            if b == c:
                value -= sp.diff(METRIC[b], X[a])  # - d_a g_bc
            if value != 0:
                christoffel[a, b, c] = value / (2 * METRIC[a])  # times g^aa / 2


def gamma(a, b, c):
    """Gamma^a_bc (0 when it is not in the dictionary)."""
    return christoffel.get((a, b, c), 0)


christoffel_b_le_c = [key for key in christoffel if key[1] <= key[2]]
report("nonzero Christoffel symbols Gamma^a_bc with b <= c", len(christoffel_b_le_c))
check(all(christoffel.get((a, c, b)) == value
          for (a, b, c), value in christoffel.items()),
      "Gamma^a_bc = Gamma^a_cb (symmetric in the lower labels)")
```

Out [6]:

```
RESULT nonzero Christoffel symbols Gamma^a_bc with b <= c = 25
PASS Gamma^a_bc = Gamma^a_cb (symmetric in the lower labels)
```

Text cell:

```
The next cell computes the Riemann tensor from the formula of section 4 and raises
its second label, $R^{ab}{}_{cd} = R^a{}_{bcd}/g_{bb}$. For every component it
replaces $a_4'$, $a_4''$, $b_4'$, $b_4''$ and $b_4$ by the symbols `A1`, `A2`,
`B1`, `B2` and `B0`, simplifies (`sp.simplify`), and writes $\cot z$ as the symbol
`C` (with $\tan z = 1/C$ and $\cos z = C \sin z$). If a component still contained
```

```
$x_0$, $\sin z$ or $e^{a_4}$, the check at the end would fail: the warp factor
$\sin^{1/3} z$ and the exponentials cancel in every $R^{ab}{}_{cd}$. The cell takes
about 15 seconds, almost all of it in ‘sp.simplify‘.
```

In [7]:

```
A1, A2, C = sp.symbols("A1 A2 C")  # a4', a4'' and cot(z)
B0, B1, B2 = sp.symbols("B0 B1 B2")  # b4, b4' and b4''
SYMBOLS = {sp.Derivative(a4, (X[4], 2)): A2, sp.Derivative(a4, X[4]): A1,
           sp.Derivative(b4, (X[4], 2)): B2, sp.Derivative(b4, X[4]): B1}
RIEMANN = {}  # (a, b, c, d) -> R^ab_cd in the symbols H, A1, A2, C
for a in range(8):
    for b in range(8):
        for c in range(8):
            for d in range(8):
                value = sp.diff(gamma(a, d, b), X[c]) - sp.diff(gamma(a, c, b), X[d])
                for e in range(8):
                    value += (gamma(a, c, e) * gamma(e, d, b)
                              - gamma(a, d, e) * gamma(e, c, b))
                if value == 0:
                    continue
                value = sp.simplify((value / METRIC[b]).subs(SYMBOLS)
                                    .subs(b4, B0))  # R^ab_cd
                value = sp.expand(value.subs({sp.tan(z): 1 / C, sp.cot(z): C,
                                              sp.cos(z): C * sp.sin(z)}))
                if value != 0:
                    RIEMANN[a, b, c, d] = value
report("nonzero components R^ab_cd", len(RIEMANN))
check(all(v.free_symbols <= {H, A1, A2, B0, B1, B2, C} for v in RIEMANN.values()),
      "every R^ab_cd is a rational expression in H, a4', a4'', b4, b4', b4'' and "
      "cot z, free of sin(z)^(1/3) and e^a4",
      record=f"{RESULTS}/lovelock-report.json, check mixed_riemann_free_of_sin_third")
```

Out [7]:

```
RESULT nonzero components R^ab_cd = 156
PASS every R^ab_cd is a rational expression in H, a4', a4'', b4, b4', b4'' and cot z,
    free of sin(z)^(1/3) and e^a4
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    mixed_riemann_free_of_sin_third
```

Text cell:

```
The next cell reads the 156 components that the Rust program wrote into
‘curvature.json‘ (as Mathematica text such as
‘H*Derivative[1][a4][x4]*Cot[6*H*x0]^(-1)‘), turns each into a sympy expression by
replacing the Mathematica names with ‘A1‘, ‘A2‘, ‘B0‘, ‘B1‘, ‘B2‘, ‘C‘ and ‘^‘ with
‘**‘, and checks that the two computations give the same nonzero components with
EXACTLY the same values (‘sp.simplify(mine - theirs) == 0‘), for a general $b_4$.
It also checks the antisymmetry
$R^{ab}{}_{cd} = -R^{ba}{}_{cd} = -R^{ab}{}_{dc}$, the number of Christoffel symbols
and the Ricci scalar $R = \sum_{a,b} R^{ab}{}_{ab}$.
```

In [8]:

```
curvature = json.loads((OUT_FOLDER / "curvature.json").read_text(encoding="utf-8"))
NAMES = curvature["coordinates"]  # ["x0", ..., "x7"]


def from_mathematica(text):
    """A component written by the Rust program, as a sympy expression."""
    text = re.sub(r"Derivative\[(\d)\]\[a4\]\[x4\]", r"A\1", text)  # a4' -> A1
    text = re.sub(r"Derivative\[(\d)\]\[b4\]\[x4\]", r"B\1", text)  # b4' -> B1
    text = text.replace("b4[x4]", "B0")  # b4 -> B0
    text = text.replace("Cot[6*H*x0]", "C").replace("^", "**")
    return sp.sympify(text, locals={"H": H, "A1": A1, "A2": A2, "C": C, "B0": B0,
                                    "B1": B1, "B2": B2})


rust_riemann = {}
for item in curvature["riemannMixedNonzero"]:
    a, b = (NAMES.index(name) for name in item["up"])
    c, d = (NAMES.index(name) for name in item["down"])
    rust_riemann[a, b, c, d] = from_mathematica(item["value"])
same_values = set(rust_riemann) == set(RIEMANN) and all(
    sp.simplify(RIEMANN[key] - rust_riemann[key]) == 0 for key in RIEMANN)
check(same_values, "sympy and Rust give the same 156 nonzero R^ab_cd exactly, "
      "with b4",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check rust_riemann_agrees")
antisymmetric = all(sp.expand(v + RIEMANN.get((b, a, c, d), 0)) == 0 and
                    sp.expand(v + RIEMANN.get((a, b, d, c), 0)) == 0
                    for (a, b, c, d), v in RIEMANN.items())
check(antisymmetric, "R^ab_cd = -R^ba_cd = -R^ab_dc for all components",
      record=f"{RESULTS}/lovelock-report.json, check riemann_antisymmetry")
check(len(christoffel_b_le_c) == len(curvature["christoffelNonzero_b_le_c"]) == 25,
      "25 nonzero Christoffel symbols with b <= c, as in the record")
ricci_scalar = sp.expand(sum(RIEMANN.get((a, b, a, b), 0) for a in range(8)
                             for b in range(8)))
report("Ricci scalar R", ricci_scalar)
check(sp.simplify(ricci_scalar - from_mathematica(curvature["ricciScalar"])) == 0
      and sp.expand(ricci_scalar.subs({B0: 1, B1: 0, B2: 0}))
      == 6 * A1 ** 2 - 42 * H ** 2,
      "the Ricci scalar is R = 6 a4'^2 - 42 H^2 + 3 b4''/b4 - 3 a4' b4'/b4, as in the "
      "record (6 a4'^2 - 42 H^2 at b4 = 1)",
      record=f"{RESULTS}/curvature.json, ricciScalar")
# From here on: the background b4 = 1 (b4' = b4'' = 0) of the Revision records.
RIEMANN_B4 = RIEMANN  # the components with a general b4
BACKGROUND = {B0: 1, B1: 0, B2: 0}
RIEMANN = {key: sp.expand(value.subs(BACKGROUND)) for key, value in RIEMANN_B4.items()}
RIEMANN = {key: value for key, value in RIEMANN.items() if value != 0}
check(len(RIEMANN) == 156,
      "on the background b4 = 1 the same 156 components are nonzero")
```

Out [8]:

```
PASS sympy and Rust give the same 156 nonzero R^ab_cd exactly, with b4
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    rust_riemann_agrees
PASS R^ab_cd = -R^ba_cd = -R^ab_dc for all components
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    riemann_antisymmetry
PASS 25 nonzero Christoffel symbols with b <= c, as in the record
RESULT Ricci scalar R = 6*A1**2 - 3*A1*B1/B0 - 42*H**2 + 3*B2/B0
PASS the Ricci scalar is R = 6 a4'^2 - 42 H^2 + 3 b4''/b4 - 3 a4' b4'/b4, as in the
    record (6 a4'^2 - 42 H^2 at b4 = 1)
     reproduces Revision/gkd_lovelock/results/curvature.json, ricciScalar
PASS on the background b4 = 1 the same 156 components are nonzero
```

Text cell:

```
From here on the notebook works on the background $b_4 = 1$ of the Revision records
($b_4' = b_4'' = 0$): ‘RIEMANN‘ now holds the 156 components there, and
‘RIEMANN_B4‘ keeps those with a general $b_4$.

The next cell draws the curvature of every coordinate plane, $K(a, b) =
R^{ab}{}_{ab}$, as two $8 \times 8$ tables of coloured squares, in units of $H^2$
(that is, with $H = 1$). The left table is a moment of the exponentially deflating
history $a_4 = A H x_4$ with $A = 2$, so $a_4' = 2H$ and $a_4'' = 0$; the right
table has the same $a_4'$ and $a_4'' = H^2$. The function ‘value_at‘ puts numbers in
place of the symbols (‘expr.subs(...)‘), and ‘float‘ turns the exact result into a
decimal number. The table is symmetric, $K(a, b) = K(b, a)$. Reading it: inside
ordinary space and inside the extra times $K = a_4'^2 - H^2$; between them
$K = -(a_4'^2 + H^2)$; with the time $x_4$, $K = a_4'^2 + a_4''$ for ordinary space
and $a_4'^2 - a_4''$ for the extra times; with the hidden direction $x_0$,
$K = -H^2$; the time and the hidden direction together are flat.
```

In [9]:

```
def value_at(expr, h=1.0, a1=2.0, a2=0.0, cot=1.0):
    """The number expr takes at H = h, a4' = a1, a4'' = a2, cot z = cot."""
    return float(sp.sympify(expr).subs({H: h, A1: a1, A2: a2, C: cot}))


fig, axes = plt.subplots(1, 2, figsize=(11.0, 5.0))
planes = {}
for ax, a2 in zip(axes, (0.0, 1.0)):
    table = np.array([[value_at(RIEMANN.get((a, b, a, b), 0), a2=a2)
                       for b in range(8)] for a in range(8)])
    planes[a2] = table
    picture = ax.imshow(table, cmap="RdBu_r", vmin=-6, vmax=6)
    for a in range(8):
        for b in range(8):
            if a != b:
                ax.text(b, a, f"{table[a, b]:g}", ha="center", va="center",
                        fontsize=8)
    ax.set_xticks(range(8), NAMES)
    ax.set_yticks(range(8), NAMES)
```

```
    ax.set_title(f"$a_4' = 2H$, $a_4'' = {a2:g}\\,H^2$")
    ax.grid(False)
fig.colorbar(picture, ax=list(axes), shrink=0.8, label="$K(a, b)$ in units of $H^2$")
save_figure(fig, "curvature_of_planes",
            "The curvature $K(a, b) = R^{ab}_{\\ ab}$ of the plane of the coordinate "
            "directions $x_a$ and $x_b$, in units of $H^2$, for $a_4^{\\prime} = 2H$ "
            "and $a_4^{\\prime\\prime} = 0$ (left, a moment of the exponentially "
            "deflating history $a_4 = 2 H x_4$) and $a_4^{\\prime\\prime} = H^2$ "
            "(right); red is positive, blue negative, the diagonal is left empty. "
            "Planes inside ordinary space or inside the extra times have "
            "$K = a_4^{\\prime 2} - H^2 = 3$, planes between them "
            "$K = -(a_4^{\\prime 2} + H^2) = -5$, planes with $x_0$ have $K = -1$, "
            "and the time $x_4$ couples to space with $a_4^{\\prime 2} + "
            "a_4^{\\prime\\prime}$ and to the extra times with $a_4^{\\prime 2} - "
            "a_4^{\\prime\\prime}$: only these entries change on the right.")
check(planes[0.0][1, 2] == 3.0 and planes[0.0][1, 5] == -5.0
      and planes[0.0][1, 0] == -1.0 and planes[1.0][1, 4] == 5.0
      and planes[1.0][5, 4] == 3.0 and planes[0.0][4, 0] == 0.0,
      "plane curvatures at a4' = 2H: 3, -5, -1; with x4: 4 + a4'' and 4 - a4''; "
      "x4 with x0: 0")
```

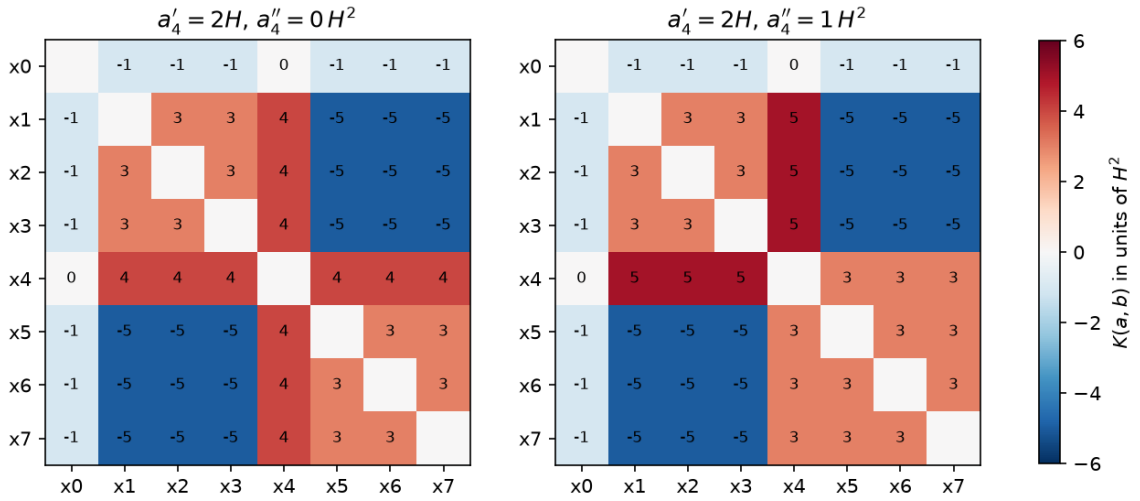

Figure 310: The curvature $K(a,b) = R^{ab}_{\ ab}$ of the plane of the coordinate directions x_a and x_b, in units of H^2, for $a_4' = 2H$ and $a_4'' = 0$ (left, a moment of the exponentially deflating history $a_4 = 2Hx_4$) and $a_4'' = H^2$ (right); red is positive, blue negative, the diagonal is left empty. Planes inside ordinary space or inside the extra times have $K = a_4'^2 - H^2 = 3$, planes between them $K = -(a_4'^2 + H^2) = -5$, planes with x_0 have $K = -1$, and the time x_4 couples to space with $a_4'^2 + a_4''$ and to the extra times with $a_4'^2 - a_4''$: only these entries change on the right. (Notebook 11b, figure 2.)

Out [9]:

```
Figure 11b.2 saved as Revision/textbook/figures/11b_2_curvature_of_planes.png
PASS plane curvatures at a4' = 2H: 3, -5, -1; with x4: 4 + a4'' and 4 - a4''; x4 with x0:
    0
```

Text cell:

```
The next cell draws the WHOLE tensor $R^{ab}{}_{cd}$ at one point as a
$64 \times 64$ table: row number $8a + b$ belongs to the upper pair $(x_a, x_b)$ and
column number $8c + d$ to the lower pair $(x_c, x_d)$. The point is $H = 1$,
$a_4' = 2$, $a_4'' = 1$ and $z = \pi/3$, where $\cot z = 1/\sqrt{3}$. Thin lines
separate the blocks of the first labels $a$ and $c$. Most squares are white: only 156
of the 4096 components are not zero. The squares far from the two diagonals are the
terms that MIX the time $x_4$ and the hidden direction $x_0$, such as
$R^{x_1 x_4}{}_{x_1 x_0} = H a_4' \cot z$.
```

In [10]:

```
point = {"h": 1.0, "a1": 2.0, "a2": 1.0, "cot": 1.0 / np.sqrt(3.0)}
whole = np.zeros((64, 64))
for (a, b, c, d), value in RIEMANN.items():
    whole[8 * a + b, 8 * c + d] = value_at(value, **point)
fig, ax = plt.subplots(figsize=(6.8, 6.2))
largest = np.abs(whole).max()
picture = ax.imshow(whole, cmap="RdBu_r", vmin=-largest, vmax=largest)
for edge in range(8, 64, 8):  # lines between the blocks
    ax.axhline(edge - 0.5, color="0.8", lw=0.5)
    ax.axvline(edge - 0.5, color="0.8", lw=0.5)
ticks = list(range(3, 64, 8))  # the middle of each block
ax.set_xticks(ticks, [f"c = {name}" for name in NAMES], rotation=90, fontsize=8)
ax.set_yticks(ticks, [f"a = {name}" for name in NAMES], fontsize=8)
ax.set_xlabel("lower pair $(x_c, x_d)$, column $8c + d$")
ax.set_ylabel("upper pair $(x_a, x_b)$, row $8a + b$")
ax.set_title("All components $R^{ab}{}_{cd}$ at one point")
ax.grid(False)
fig.colorbar(picture, ax=ax, shrink=0.8, label="value (units of $H^2$)")
save_figure(fig, "riemann_matrix",
            "All 4096 components of the Riemann tensor $R^{ab}_{\\ \\ cd}$ of the "
            "author's metric at the point $H = 1$, $a_4^{\\prime} = 2$, "
            "$a_4^{\\prime\\prime} = 1$, $z = \\pi/3$, arranged as a 64 by 64 table: "
            "row $8a + b$ is the upper pair of labels, column $8c + d$ the lower "
            "pair, counted from 0; red is positive, blue negative, white zero. Only "
            "156 components are not zero. The two diagonals hold the curvatures of "
            "the coordinate planes and their sign-reversed copies; the scattered "
            "squares between them mix the time $x_4$ with the hidden direction "
            "$x_0$.")
check(int(np.count_nonzero(whole)) == 156,
      "156 of the 4096 components are not zero at this point")
```

Out [10]:

```
Figure 11b.3 saved as Revision/textbook/figures/11b_3_riemann_matrix.png
PASS 156 of the 4096 components are not zero at this point
```

Text cell:

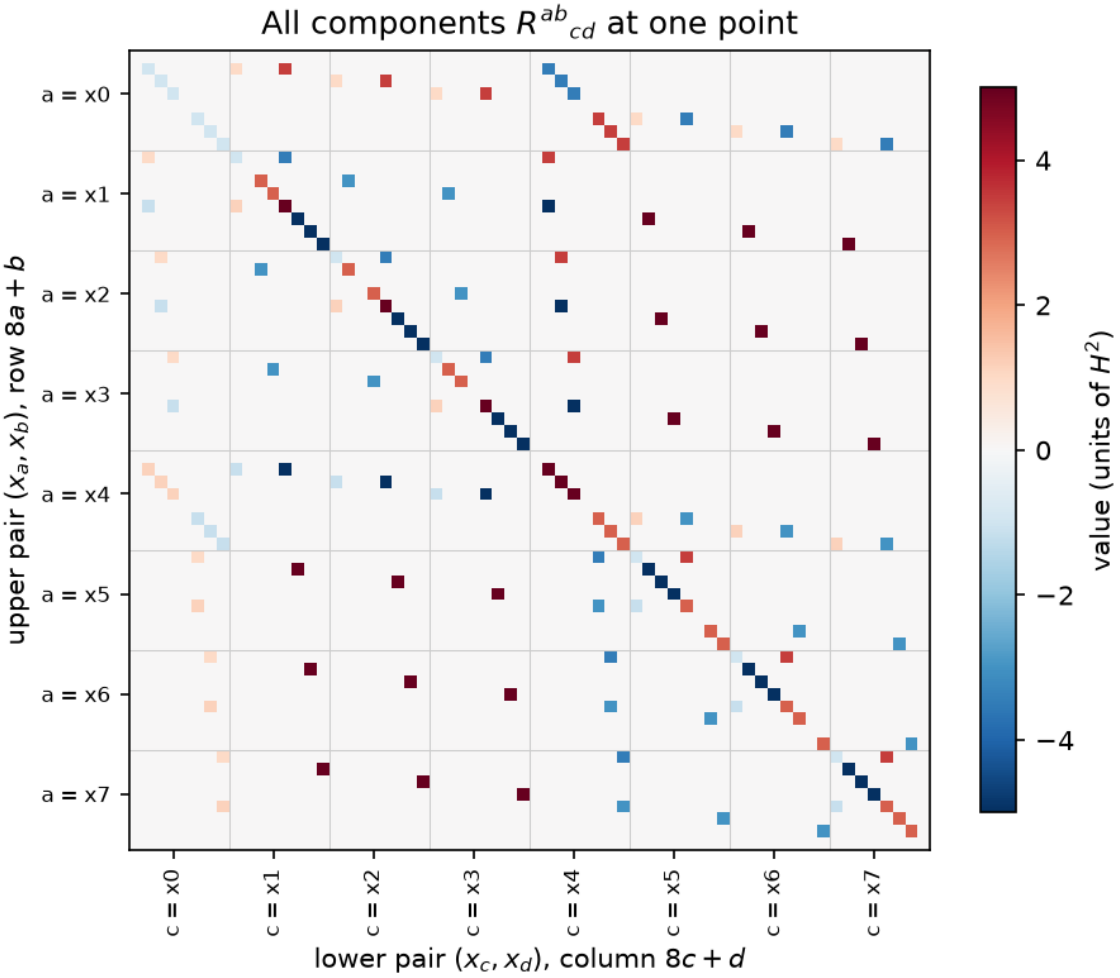

Figure 311: All 4096 components of the Riemann tensor $R^{ab}{}_{cd}$ of the author's metric at the point $H = 1$, $a_4' = 2$, $a_4'' = 1$, $z = \pi/3$, arranged as a 64 by 64 table: row $8a + b$ is the upper pair of labels, column $8c + d$ the lower pair, counted from 0; red is positive, blue negative, white zero. Only 156 components are not zero. The two diagonals hold the curvatures of the coordinate planes and their sign-reversed copies; the scattered squares between them mix the time x_4 with the hidden direction x_0. (Notebook 11b, figure 3.)

```
Why do the mixed $x_4$-$x_0$ terms not enter Einstein's tensor? The Ricci component
$R^{x_4}{}_{x_0} = \sum_c R^{x_4 c}{}_{x_0 c}$ adds one term for each direction $c$.
The next cell lists these seven terms: each of the three space directions gives
$+H a_4' \cot z$, each of the three extra times $-H a_4' \cot z$, and the hidden
direction itself gives 0. The sum vanishes EXACTLY, for every $a_4$, because there
are as many inflating as deflating directions, on the background $b_4 = 1$. With
a changing $b_4$ the three extra-time terms carry an additional piece with $b_4'$,
and the cell checks that the sum is then $R^{x_4}{}_{x_0} = \tfrac32 H\cot z\,
b_4'/b_4$, the mixed component of the Einstein tensor in the record (`einsteinMixed`
of `curvature.json`): with $b_4' \neq 0$ the time and the hidden direction DO mix.
The cell draws the seven terms at the point $H = 1$, $a_4' = 1$, $z = \pi/4$
($\cot z = 1$), $b_4 = 1$.
```

In [11]:

```
mixing = [sp.expand(RIEMANN.get((4, c, 0, c), 0)) for c in range(1, 8)]  # x1..x7
for c, term in enumerate(mixing, 1):
    say(f"R^(x4 {NAMES[c]})_(x0 {NAMES[c]}) = {term}")
total = sp.expand(sum(mixing))
say(f"sum over c = {total} (b4 = 1)")
total_b4 = sp.simplify(sum(RIEMANN_B4.get((4, c, 0, c), 0) for c in range(1, 8)))
say(f"with a general b4: sum over c = {total_b4}")
fig, ax = plt.subplots(figsize=(7.0, 4.0))
heights = [value_at(term, h=1.0, a1=1.0, cot=1.0) for term in mixing]
colours = ["tab:red"] * 3 + ["0.6"] + ["tab:blue"] * 3
ax.bar(NAMES[1:], heights, color=colours)
ax.text(3, 0.05, "0", ha="center", va="bottom")  # the term of c = x4 is zero
ax.axhline(0.0, color="black", lw=0.8)
ax.set_xlabel("the direction $c$ of the term $R^{x_4 c}{}_{x_0 c}$")
ax.set_ylabel("value (units of $H^2$)")
ax.set_title(f"Seven terms of $R^{{x_4}}{{}}_{{x_0}}$; their sum is {total}")
save_figure(fig, "mixing_terms_cancel",
            "The seven terms $R^{x_4 c}_{\\ \\ x_0 c}$, $c = x_1$ to $x_7$, whose sum "
            "is the Ricci component $R^{x_4}_{\\ x_0}$, at $H = 1$, "
            "$a_4^{\\prime} = 1$, $\\cot z = 1$, in units of $H^2$: the three space "
            "directions (red) give $+H a_4^{\\prime} \\cot z$ each, the three extra "
            "times (blue) $-H a_4^{\\prime} \\cot z$ each, and the time direction "
            "itself (grey) gives 0. Three inflating and three deflating directions "
            "cancel exactly, so Einstein's tensor and, as section 8 of this "
            "notebook shows, all three Lovelock tensors have no $x_4$-$x_0$ "
            "component.")
check(total == 0 and sp.expand(mixing[0] - H * A1 * C) == 0,
      "the seven terms of R^x4_x0 are +H a4' cot z (three times), -H a4' cot z (three "
      "times) and 0, and cancel exactly at b4 = 1")
check(sp.simplify(total_b4 - sp.Rational(3, 2) * H * C * B1 / B0) == 0
      and sp.simplify(total_b4 - from_mathematica(
          curvature["einsteinMixed"]["x4,x0"]["mathematica"])) == 0,
      "with b4: R^x4_x0 = (3/2) H cot z b4'/b4, the record's G^x4_x0",
      record=f"{RESULTS}/curvature.json, einsteinMixed x4,x0")
```

Out [11]:

```
R^(x4 x1)_(x0 x1) = A1*C*H
R^(x4 x2)_(x0 x2) = A1*C*H
R^(x4 x3)_(x0 x3) = A1*C*H
R^(x4 x4)_(x0 x4) = 0
R^(x4 x5)_(x0 x5) = -A1*C*H
R^(x4 x6)_(x0 x6) = -A1*C*H
R^(x4 x7)_(x0 x7) = -A1*C*H
sum over c = 0 (b4 = 1)
with a general b4: sum over c = 3*B1*C*H/(2*B0)
```

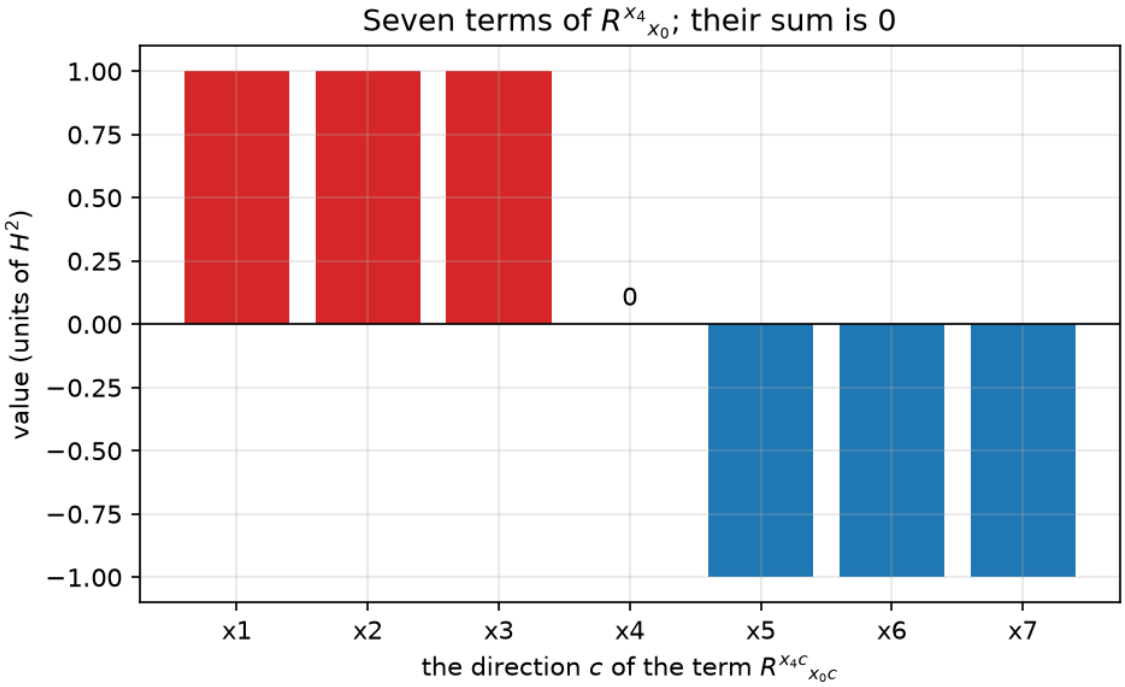

Figure 312: The seven terms $R^{x_4 c}{}_{x_0 c}$, $c = x_1$ to x_7, whose sum is the Ricci component $R^{x_4}{}_{x_0}$, at $H = 1$, $a_4' = 1$, $\cot z = 1$, in units of H^2: the three space directions (red) give $+Ha_4' \cot z$ each, the three extra times (blue) $-Ha_4' \cot z$ each, and the time direction itself (grey) gives 0. Three inflating and three deflating directions cancel exactly, so Einstein's tensor and, as section 8 of this notebook shows, all three Lovelock tensors have no x_4-x_0 component. (Notebook 11b, figure 4.)

Out [11]:

```
Figure 11b.4 saved as Revision/textbook/figures/11b_4_mixing_terms_cancel.png
PASS the seven terms of R^x4_x0 are +H a4' cot z (three times), -H a4' cot z (three
    times) and 0, and cancel exactly at b4 = 1
PASS with b4: R^x4_x0 = (3/2) H cot z b4'/b4, the record's G^x4_x0
     reproduces Revision/gkd_lovelock/results/curvature.json, einsteinMixed x4,x0
```

Text cell:

```
## 8. The three Lovelock tensors with a Python GKD sum
```

The next cell prepares the exact arithmetic, on the background $b_4 = 1$. A polynomial in H, A_1, A_2, C is
stored as a Python dictionary that maps the exponents $(e_H, e_{A_1}, e_{A_2}, e_C)$ of a monomial to its coefficient, a ‘Fraction‘ (an exact fraction of whole numbers); e_C may be negative. ‘poly_mul‘ multiplies two polynomials (every monomial of the first by every monomial of the second, adding the exponents), ‘poly_add‘ adds a multiple of one polynomial to another, dropping monomials whose coefficient becomes 0. ‘to_poly‘ turns a sympy expression into this form (‘sp.Poly‘ lists the monomials of ‘expr * C**8‘; the factor C^8 makes every power of C positive, and 8 is then subtracted again). The 156 nonzero entries $R^{ab}{}_{cd}$ of the sympy computation of section 7, sorted by their labels, become the list ‘ENTRIES‘.

The cell also defines ‘gkd‘, the rule of Notebook 11a: the sign of the permutation that carries the lower list into the upper list, or 0. In a term of the sum the UPPER labels a, b of an entry $R^{ab}{}_{cd}$ go into the LOWER list of the delta and its lower labels c, d into the upper list (look at the formula of section 4). The cell stores both pairs of every entry as **bit masks**: a set of labels is written as the whole number $\sum 2^{\text{label}}$, so that two sets share a label exactly when the bitwise AND (‘&‘) of their numbers is not 0, and the union of two sets is the bitwise OR (‘|‘). ‘np.array‘ holds the 156 masks, so that one comparison tests all 156 entries at once.

In [12]:

```
from fractions import Fraction  # exact fractions of whole numbers

ONE = {(0, 0, 0, 0): Fraction(1)}  # the polynomial 1


def poly_mul(p, q):
    """The product of the polynomials p and q."""
    out = {}
    for m1, c1 in p.items():
        for m2, c2 in q.items():
            m = (m1[0] + m2[0], m1[1] + m2[1], m1[2] + m2[2], m1[3] + m2[3])
            out[m] = out.get(m, 0) + c1 * c2
    return {m: c for m, c in out.items() if c != 0}


def poly_add(p, q, factor=1):
    """p + factor * q."""
    out = dict(p)
    for m, c in q.items():
        out[m] = out.get(m, 0) + factor * c
    return {m: c for m, c in out.items() if c != 0}


def to_poly(expr):
    """A sympy expression in H, A1, A2, C (C may have negative powers) as a dict."""
    poly = sp.Poly(sp.expand(expr * C ** 8), H, A1, A2, C)
    return {(m[0], m[1], m[2], m[3] - 8): Fraction(int(v.p), int(v.q))
            for m, v in zip(poly.monoms(), poly.coeffs())}
```

```
def gkd(lower, upper):
    """GKD (Notebook 11a): the sign of the permutation from lower to upper, or 0."""
    position = {}
    for place, label in enumerate(upper):
        if label in position:
            return 0
        position[label] = place
    sigma = [position.get(label) for label in lower]
    if None in sigma or len(set(sigma)) < len(sigma):
        return 0
    inversions = sum(1 for i in range(len(sigma)) for j in range(i + 1, len(sigma))
                     if sigma[i] > sigma[j])
    return 1 if inversions % 2 == 0 else -1


ENTRIES = [(key, to_poly(RIEMANN[key])) for key in sorted(RIEMANN)]  # 156 entries
LOWER_MASK = np.array([(1 << a) | (1 << b) for (a, b, c, d), _ in ENTRIES])
UPPER_MASK = np.array([(1 << c) | (1 << d) for (a, b, c, d), _ in ENTRIES])
report("nonzero curvature entries in the sums", len(ENTRIES))
check(gkd([0, 1, 2], [1, 2, 0]) == 1 and gkd([0, 1], [1, 0]) == -1,
      "gkd: a cycle of three labels is even (+1), an exchange of two odd (-1)",
      record=f"{RESULTS}/lovelock-report.json, gkdSelfCheck")
```

Out [12]:

```
RESULT nonzero curvature entries in the sums = 156
PASS gkd: a cycle of three labels is even (+1), an exchange of two odd (-1)
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, gkdSelfCheck
```

Text cell:

```
The next cell defines ‘lovelock_sum(k, free_pair)‘, the sum of section 4. It works
with a list ‘stack‘ of partial combinations: each holds the chosen entries (their
numbers in ‘ENTRIES‘) and the masks of the labels used so far in the lower and in
the upper list. A partial combination with fewer than $k$ entries is extended by
every entry that shares no label with it, in the lower list or in the upper list
(‘np.nonzero‘ finds them among all 156 at once); this is the skipping of repeated
labels. A complete combination (a **leaf**) is kept only if its lower and upper
label sets are equal (otherwise some lower label is missing above and GKD is 0);
then GKD is called on the lower list $(j, j_1, \dots, j_{2k})$ and the upper list
$(h, h_1, \dots, h_{2k})$, and its value is added to the weight of the combination.
The weights are collected per SET of entries (‘tuple(sorted(...))‘), because the
product of the entries does not depend on their order; at the end every set’s
product is multiplied out once. With ‘free_pair=True‘ the sum is done for each of
the 64 pairs $(h, j)$ and gives the tensor $P_{(k)}{}^h{}_j$; with ‘free_pair=False‘
it gives the scalar $L_{(k)}$. The function also counts leaves and GKD calls.
```

In [13]:

```
def lovelock_sum(k, free_pair):
    """P_(k) as {(h, j): polynomial} (free_pair True) or L_(k) as {None: polynomial},
```

```
    and the numbers of leaves and of GKD calls."""
    pairs = [(h, j) for h in range(8) for j in range(8)] if free_pair else [None]
    weights, leaves, calls = {}, 0, 0
    for pair in pairs:
        start_lower = 1 << pair[1] if pair else 0  # j is the first lower label
        start_upper = 1 << pair[0] if pair else 0  # h is the first upper label
        stack = [((), start_lower, start_upper)]
        while stack:
            chosen, lower_mask, upper_mask = stack.pop()
            if len(chosen) == k:  # a leaf
                leaves += 1
                if lower_mask != upper_mask:
                    continue  # a lower label is missing above: GKD = 0
                lower = ([pair[1]] if pair else []) + [
                    x for i in chosen for x in ENTRIES[i][0][:2]]  # j, j1, ..., j2k
                upper = ([pair[0]] if pair else []) + [
                    x for i in chosen for x in ENTRIES[i][0][2:]]  # h, h1, ..., h2k
                sign = gkd(lower, upper)
                calls += 1
                if sign != 0:
                    bucket = weights.setdefault(pair, {})
                    key = tuple(sorted(chosen))  # the set of entries
                    bucket[key] = bucket.get(key, 0) + sign
                continue
            free = np.nonzero(((LOWER_MASK & lower_mask) == 0)
                              & ((UPPER_MASK & upper_mask) == 0))[0]
            for i in free:  # every entry that repeats no label
                stack.append((chosen + (int(i),), lower_mask | int(LOWER_MASK[i]),
                              upper_mask | int(UPPER_MASK[i])))
    products = {}  # set of entries -> the product of their polynomials
    result = {}
    for pair, bucket in weights.items():
        total = {}
        for key, weight in bucket.items():
            if weight == 0:
                continue
            if key not in products:
                product = ONE
                for i in key:
                    product = poly_mul(product, ENTRIES[i][1])
                products[key] = product
            total = poly_add(total, products[key], weight)
        if total:
            result[pair] = total
    return result, leaves, calls
```

Text cell:

```
The next cell runs the sums for $k = 1, 2, 3$, the tensors and the scalars, and
compares the counters with those of the Rust program (‘lovelock-report.json‘:
‘leaves‘, ‘gkdCalls‘, ‘gkdNonzero‘ and ‘scalarGkdCalls‘). The numbers of leaves and
of GKD calls depend only on which entries are not zero, so they are the same for a
general $b_4$ and for $b_4 = 1$. The Rust program, which keeps $b_4$ general, finds
```

```
10 nonzero components of each tensor: the 8 diagonal ones and the two mixed
components $x_4$-$x_0$ and $x_0$-$x_4$, which contain $b_4'$; on the background
$b_4 = 1$ the Python sums find the 8 diagonal ones. It takes about 10 seconds,
almost all of it for $k = 3$.
```

In [14]:

```
P, L, MY_COUNTERS = {}, {}, {}
for k in (1, 2, 3):
    P[k], leaves, calls = lovelock_sum(k, free_pair=True)
    scalar, scalar_leaves, scalar_calls = lovelock_sum(k, free_pair=False)
    L[k] = scalar.get(None, {})
    MY_COUNTERS[k] = (leaves, calls, scalar_calls)
    say(f"k = {k}: leaves {leaves:7d}, GKD calls {calls:6d}, scalar GKD calls "
        f"{scalar_calls:6d}, nonzero components {len(P[k])}")
same_counters = all(
    MY_COUNTERS[k] == (COUNTERS[k]["leaves"], COUNTERS[k]["gkdCalls"],
                       COUNTERS[k]["scalarGkdCalls"])
    and COUNTERS[k]["gkdNonzero"] == COUNTERS[k]["gkdCalls"]
    and len(P[k]) == 8 and COUNTERS[k]["nonzeroComponents"] == 10
    for k in (1, 2, 3))
check(same_counters, "the Python sums reproduce the counters of the Rust sums; "
      "nonzero components: 8 at b4 = 1, 10 (with x4-x0, x0-x4) for a general b4",
      record=f"{RESULTS}/lovelock-report.json, counters k = 1, 2, 3")
```

Out [14]:

```
k = 1: leaves    5616, GKD calls    696, scalar GKD calls    108, nonzero components 8
k = 2: leaves  176640, GKD calls  32640, scalar GKD calls   7200, nonzero components 8
k = 3: leaves 1128960, GKD calls 495360, scalar GKD calls 213120, nonzero components 8
PASS the Python sums reproduce the counters of the Rust sums; nonzero components: 8 at b4
    = 1, 10 (with x4-x0, x0-x4) for a general b4
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, counters k = 1, 2, 3
```

Text cell:

```
The next cell compares every one of the $3 \times 64$ components with the Rust
record `lovelock-tensors.json`, which stores each component, for a general $b_4$, as
an exact list of monomials `[numerator, denominator, [e_H, e_a4', e_a4'', e_a4''',
e_a4'''', e_E, e_S, e_C, e_Q, e_b4', e_b4'', e_b4''', e_b4''']]` (E is $e^{a_4}$,
S is $\sin^{1/3} z$, C is $\cot z$, Q is $b_4^{1/2}$). The record is evaluated on
the background $b_4 = 1$: a monomial with a derivative of $b_4$ is dropped, and
$Q = 1$ (monomials that then agree are added). The comparison is exact
(dictionaries of fractions must be equal). The components of the third and fourth
derivatives of $a_4$, of $E$ and of $S$ must all be 0. Then it compares the three
scalars with the record's `L1`, `L2`, `L3`, and prints the components that are not
zero: only the eight diagonal ones $h = j$, for every $k$. In particular there is no
$x_4$-$x_0$ component.
```

In [15]:

```
tensors = json.loads((OUT_FOLDER / "lovelock-tensors.json").read_text(
    encoding="utf-8"))


def from_monomials(monomials):
    """A component of lovelock-tensors.json at b4 = 1 as a polynomial dict (only H,
    a4', a4'' and cot z may then occur)."""
    out = {}
    for numerator, denominator, e in monomials:
        if any(e[9:13]):  # a derivative of b4: zero on the background b4 = 1
            continue
        if e[3] or e[4] or e[5] or e[6]:  # 3rd, 4th derivative, e^a4 or sin^(1/3)
            raise ValueError("unexpected symbol in a Lovelock component")
        key = (e[0], e[1], e[2], e[7])  # e[8], the power of b4^(1/2), drops out
        out[key] = out.get(key, 0) + Fraction(numerator, denominator)
    return {key: value for key, value in out.items() if value != 0}


agree = {}
for k in (1, 2, 3):
    block = tensors[f"P{k}_mixed_up_h_down_j"]
    agree[k] = all(from_monomials(block[f"{NAMES[h]},{NAMES[j]}"]["monomials"])
                   == P[k].get((h, j), {}) for h in range(8) for j in range(8))
    nonzero = " ".join(sorted(f"{NAMES[h]},{NAMES[j]}" for h, j in P[k]))
    say(f"k = {k}: nonzero components (h, j): {nonzero}")
check(all(agree.values()),
      "all 3 x 64 components of P(1), P(2), P(3) equal the Rust components exactly",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "rust_k1_mixed_components_agree, rust_k2_mixed_components_agree and "
             "rust_k3_mixed_components_agree")
scalars_agree = all(to_poly(from_mathematica(tensors[f"L{k}"]).subs(BACKGROUND))
                    == L[k] for k in (1, 2, 3))
check(scalars_agree, "the scalars L(1), L(2), L(3) equal the Rust scalars exactly "
      "(at b4 = 1)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "rust_L1_agrees, rust_L2_agrees and rust_L3_agrees")
check(all(set(P[k]) == {(h, h) for h in range(8)} for k in (1, 2, 3)),
      "only the 8 diagonal components are nonzero, for k = 1, 2, 3")
```

Out [15]:

```
k = 1: nonzero components (h, j): x0,x0 x1,x1 x2,x2 x3,x3 x4,x4 x5,x5 x6,x6 x7,x7
k = 2: nonzero components (h, j): x0,x0 x1,x1 x2,x2 x3,x3 x4,x4 x5,x5 x6,x6 x7,x7
k = 3: nonzero components (h, j): x0,x0 x1,x1 x2,x2 x3,x3 x4,x4 x5,x5 x6,x6 x7,x7
PASS all 3 x 64 components of P(1), P(2), P(3) equal the Rust components exactly
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, checks
    rust_k1_mixed_components_agree, rust_k2_mixed_components_agree and
    rust_k3_mixed_components_agree
PASS the scalars L(1), L(2), L(3) equal the Rust scalars exactly (at b4 = 1)
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, checks
    rust_L1_agrees, rust_L2_agrees and rust_L3_agrees
PASS only the 8 diagonal components are nonzero, for k = 1, 2, 3
```

Text cell:

```
**The skipping changes nothing: the literal sums of orders 1 and 2.** The sums
above leave out every combination in which a label repeats, because GKD is 0 there
(Notebook 11a). The next cell does NOT rely on this. For $k = 1$ and $k = 2$ it
runs through EVERY ordered list of $k$ of the 156 nonzero entries
(‘itertools.product(range(156), repeat=k)‘ gives all $156^k$ of them), builds the
lower list $(j_1, \dots, j_{2k})$ and the upper list $(h_1, \dots, h_{2k})$ of the
delta, and calls GKD for the scalar and for each of the 64 pairs $(h, j)$, with no
test before the call: $65 \cdot 156 = 10140$ calls for $k = 1$ and
$65 \cdot 156^2 = 1581840$ calls for $k = 2$, the numbers recorded by the Revision’s
sympy checker for the same literal sums. The weights are collected per set of
entries as before, the products are multiplied out, and the results must equal the
tensors and scalars of the skipping sums EXACTLY. The cell takes about 15 seconds.
```

In [16]:

```
import itertools  # all ordered lists of k entries


def literal_sum(k):
    """P_(k) and L_(k) from EVERY ordered list of k nonzero entries, GKD called on
    every index list without any test; also the number of GKD calls."""
    pairs = [None] + [(h, j) for h in range(8) for j in range(8)]
    weights, calls = {}, 0
    for chosen in itertools.product(range(len(ENTRIES)), repeat=k):
        lower = [x for i in chosen for x in ENTRIES[i][0][:2]]  # j1, ..., j2k
        upper = [x for i in chosen for x in ENTRIES[i][0][2:]]  # h1, ..., h2k
        key = tuple(sorted(chosen))  # the set of entries of this product
        for pair in pairs:
            if pair is None:  # the scalar: no free pair
                sign = gkd(lower, upper)
            else:  # the tensor: j first in the lower list, h first in the upper
                sign = gkd([pair[1]] + lower, [pair[0]] + upper)
            calls += 1
            if sign != 0:
                bucket = weights.setdefault(pair, {})
                bucket[key] = bucket.get(key, 0) + sign
    result = {}
    for pair, bucket in weights.items():
        total = {}
        for key, weight in bucket.items():
            product = ONE
            for i in key:
                product = poly_mul(product, ENTRIES[i][1])
            total = poly_add(total, product, weight)
        if total:
            result[pair] = total
    return result, calls


record_checks = {c["name"]: c for c in json.loads(repository_file(
```

```
    "Revision/gkd_lovelock/results/python-lovelock-report.json").read_text(
        encoding="utf-8"))["checks"]}
for k in (1, 2):
    literal, calls = literal_sum(k)
    recorded = record_checks[f"k{k}_unpruned_literal_sum_agrees"]
    recorded_calls = int(re.search(r"(\d+) kdelta calls", recorded["detail"]).group(1))
    say(f"k = {k}: {calls} GKD calls (the record: {recorded_calls})")
    same = (literal.pop(None, {}) == L[k]
            and all(literal.get(hj, {}) == P[k].get(hj, {})
                    for hj in set(literal) | set(P[k])))
    check(same and calls == recorded_calls == 65 * 156 ** k
          and recorded["verdict"] == "PASS",
          f"the literal sum of order {k}, without skipping, gives the same P({k}) "
          f"and L({k})",
          record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
                 f"k{k}_unpruned_literal_sum_agrees")
```

Out [16]:

```
k = 1: 10140 GKD calls (the record: 10140)
PASS the literal sum of order 1, without skipping, gives the same P(1) and L(1)
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    k1_unpruned_literal_sum_agrees
k = 2: 1581840 GKD calls (the record: 1581840)
PASS the literal sum of order 2, without skipping, gives the same P(2) and L(2)
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    k2_unpruned_literal_sum_agrees
```

Text cell:

```
**Order 4 is zero.** A term of $P_{(4)}$ needs four curvature entries, that is nine
labels in each list. The next cell looks at the leaves of order 3: every one of them
already uses seven DIFFERENT labels in its lower list ($j$ and six more), so a
fourth entry, which brings two new labels, would need nine different labels out of
eight. No leaf of order 4 exists, which is the Rust counter ‘leaves = 0‘ for
$k = 4$. (‘bin(mask).count("1")‘ counts the labels in a mask.) The scalar $L_{(4)}$,
with eight labels per list, is NOT zero (the program printed it in section 5); it is
the Euler density of eight dimensions, and its tensor $P_{(4)}$ vanishes
identically.
```

In [17]:

```
seven_labels = True
for h in range(8):
    for j in range(8):
        stack = [((), 1 << j, 1 << h)]
        while stack:
            chosen, lower_mask, upper_mask = stack.pop()
            if len(chosen) == 3:  # a leaf of order 3
                seven_labels = seven_labels and bin(lower_mask).count("1") == 7
                continue
```

```
            free = np.nonzero(((LOWER_MASK & lower_mask) == 0)
                              & ((UPPER_MASK & upper_mask) == 0))[0]
            for i in free:
                stack.append((chosen + (int(i),), lower_mask | int(LOWER_MASK[i]),
                              upper_mask | int(UPPER_MASK[i])))
check(seven_labels and COUNTERS[4]["leaves"] == 0,
      "every leaf of order 3 uses 7 labels, so P(4) has no term: P(4) = 0",
      record=f"{RESULTS}/lovelock-report.json, check k4_tensor_vanishes")
```

Out [17]:

```
PASS every leaf of order 3 uses 7 labels, so P(4) has no term: P(4) = 0
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    k4_tensor_vanishes
```

Text cell:

```
## 9. Three classical identities, and the traces

**Order 1 is Einstein's tensor, derived line by line.** Expand the $3 \times 3$
determinant $\delta^{h h_1 h_2}_{j j_1 j_2}$ along its column of $h$ (rows belong to
the lower labels $j, j_1, j_2$, columns to the upper labels $h, h_1, h_2$):

$$\delta^{h h_1 h_2}_{j j_1 j_2} = \delta^h_j\, \delta^{h_1 h_2}_{j_1 j_2}
  - \delta^h_{j_1}\, \delta^{h_1 h_2}_{j j_2} + \delta^h_{j_2}\,
  \delta^{h_1 h_2}_{j j_1}.$$

(Each term is an entry of the column times the $2 \times 2$ determinant left when
its row and the column are removed, with the signs $+, -, +$.) Multiply by
$R^{j_1 j_2}{}_{h_1 h_2}$ and sum. Since $\delta^{h_1 h_2}_{j_1 j_2} =
\delta^{h_1}_{j_1}\delta^{h_2}_{j_2} - \delta^{h_2}_{j_1}\delta^{h_1}_{j_2}$, the
first term gives $\delta^h_j (R^{h_1 h_2}{}_{h_1 h_2} - R^{h_2 h_1}{}_{h_1 h_2})
= 2R\,\delta^h_j$ (antisymmetry, and $\sum R^{ab}{}_{ab} = R$). The second gives
$-(R^{h j_2}{}_{j j_2} - R^{h j_2}{}_{j_2 j}) = -2 R^h{}_j$. The third gives
$R^{j_1 h}{}_{j j_1} - R^{j_1 h}{}_{j_1 j} = 2 R^{j_1 h}{}_{j j_1} = -2 R^h{}_j$.
Together

$$P_{(1)}{}^h{}_j = 2R\,\delta^h_j - 4 R^h{}_j = -4\big(R^h{}_j - \tfrac12 R\,
  \delta^h_j\big) = -4\, G^h{}_j,$$

and in the same way $L_{(1)} = \delta^{h_1 h_2}_{j_1 j_2} R^{j_1 j_2}{}_{h_1 h_2} =
2R$.

**Order 2 is the Gauss-Bonnet tensor.** The same expansion, done twice, gives
$P_{(2)} = -8\,\mathcal{H}$ and $L_{(2)} = 4\,\mathrm{GB}$, with the Gauss-Bonnet
scalar $\mathrm{GB} = R^2 - 4 R^a{}_b R^b{}_a + R^{ab}{}_{cd} R^{cd}{}_{ab}$ and
Lanczos's tensor
$2\mathcal{H}^h{}_j = 4\big(R R^h{}_j - 2 R^h{}_a R^a{}_j - 2 R^{ha}{}_{jb} R^b{}_a
+ R^{ha}{}_{bc} R^{bc}{}_{ja}\big) - \delta^h_j\, \mathrm{GB}$ (the derivation is
written out in the Revision checker `check_lovelock_gkd.py`, which also confirmed the
factors $-4$ and $-8$ on random curvature tensors).
```

```
**Order 3 is the cubic Lovelock density.** $L_{(3)} = 8\,(2T_1 + 8T_2 + 24T_3 + 3T_4
+ 24T_5 + 16T_6 - 12T_7 + T_8)$ with the eight cubic invariants
$T_1 = R^{ab}{}_{cd}R^{cd}{}_{ef}R^{ef}{}_{ab}$,
$T_2 = R^{ab}{}_{cd}R^{ce}{}_{bf}R^{df}{}_{ae}$,
$T_3 = R^{ab}{}_{cd}R^{cd}{}_{be}R^e{}_a$,
$T_4 = R\,R^{ab}{}_{cd}R^{cd}{}_{ab}$,
$T_5 = R^{ab}{}_{cd}R^c{}_aR^d{}_b$,
$T_6 = R^a{}_bR^b{}_cR^c{}_a$, $T_7 = R\,R^a{}_bR^b{}_a$, $T_8 = R^3$ (the
coefficients were derived by the Revision checker from 9 random curvature tensors).

The next three cells compute, from ‘ENTRIES‘ and with the exact polynomial
arithmetic, the Ricci tensor, the Ricci scalar and Einstein’s tensor; then the
Gauss-Bonnet scalar and tensor; then the eight cubic invariants; and each checks its
identity for all 64 components. The first cell checks order 1. ‘RM‘ is the
dictionary of the entries, ‘get(key, {})‘ gives the empty polynomial (zero) for a
missing key, ‘poly_sum‘ adds a list of polynomials, and ‘delta[h, j]‘ is the
polynomial 1 for $h = j$ and 0 otherwise.
```

In [18]:

```
RM = dict(ENTRIES)  # (a, b, c, d) -> R^ab_cd as a polynomial dict


def poly_sum(polynomials):
    """The sum of a list of polynomials."""
    total = {}
    for p in polynomials:
        total = poly_add(total, p)
    return total


RIC = {(h, j): poly_sum(RM.get((h, a, j, a), {}) for a in range(8))
       for h in range(8) for j in range(8)}  # R^h_j
R_SCALAR = poly_sum(RIC[h, h] for h in range(8))  # R
delta = {(h, j): (ONE if h == j else {}) for h in range(8) for j in range(8)}
EINSTEIN = {hj: poly_add(RIC[hj], poly_mul(delta[hj], R_SCALAR), Fraction(-1, 2))
            for hj in RIC}  # G = Ric - R/2
check(all(P[1].get(hj, {}) == poly_mul({(0, 0, 0, 0): Fraction(-4)}, EINSTEIN[hj])
          for hj in EINSTEIN) and L[1] == poly_add({}, R_SCALAR, 2),
      "P(1) = -4 G for all 64 components and L(1) = 2 R",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "k1_equals_minus_4_einstein and L1_equals_2R")
```

Out [18]:

```
PASS P(1) = -4 G for all 64 components and L(1) = 2 R
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, checks
    k1_equals_minus_4_einstein and L1_equals_2R
```

Text cell:

```
The next cell checks order 2. It forms the two squares $R^a{}_b R^b{}_a$ and
$R^{ab}{}_{cd} R^{cd}{}_{ab}$, the Gauss-Bonnet scalar
$\mathrm{GB} = R^2 - 4 R^a{}_b R^b{}_a + R^{ab}{}_{cd} R^{cd}{}_{ab}$, and, for each
of the 64 pairs $(h, j)$, the four sums of Lanczos's tensor $2\mathcal{H}^h{}_j$
written above (`t` collects them before the factor 4 and the term
$-\delta^h_j\,\mathrm{GB}$ are applied).
```

In [19]:

```
ric_square = poly_sum(poly_mul(RIC[a, b], RIC[b, a]) for a in range(8)
                      for b in range(8))  # R^a_b R^b_a
riem_square = poly_sum(poly_mul(v, RM[(c, d, a, b)]) for (a, b, c, d), v in
                       RM.items() if (c, d, a, b) in RM)  # R^ab_cd R^cd_ab
GB = poly_add(poly_add(poly_mul(R_SCALAR, R_SCALAR), ric_square, -4), riem_square)
two_h = {}  # 2 H^h_j
for h in range(8):
    for j in range(8):
        t = poly_mul(R_SCALAR, RIC[h, j])
        t = poly_add(t, poly_sum(poly_mul(RIC[h, a], RIC[a, j]) for a in range(8)),
                     -2)
        t = poly_add(t, poly_sum(poly_mul(RM.get((h, a, j, b), {}), RIC[b, a])
                                 for a in range(8) for b in range(8)), -2)
        t = poly_add(t, poly_sum(poly_mul(v, RM.get((b, c, j, a), {}))
                                 for (hh, a, b, c), v in RM.items() if hh == h))
        two_h[h, j] = poly_add(poly_mul({(0, 0, 0, 0): Fraction(4)}, t),
                               poly_mul(delta[h, j], GB), -1)
check(all(P[2].get(hj, {}) == poly_mul({(0, 0, 0, 0): Fraction(-4)}, two_h[hj])
          for hj in two_h) and L[2] == poly_add({}, GB, 4),
      "P(2) = -8 times the Gauss-Bonnet tensor for all 64 components and L(2) = 4 GB",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "k2_equals_minus_8_gauss_bonnet and L2_equals_4_gauss_bonnet")
```

Out [19]:

```
PASS P(2) = -8 times the Gauss-Bonnet tensor for all 64 components and L(2) = 4 GB
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, checks
    k2_equals_minus_8_gauss_bonnet and L2_equals_4_gauss_bonnet
```

Text cell:

```
The next cell computes the eight cubic invariants, in the same mixed form as the
Revision checker (sums over the nonzero entries), and checks the cubic identity.
```

In [20]:

```
T = [{} for _ in range(8)]  # T1 ... T8
for (a, b, c, d), v in RM.items():
    for (c2, d2, e, f), w in RM.items():
        if (c2, d2) == (c, d) and (e, f, a, b) in RM:  # R^ab_cd R^cd_ef R^ef_ab
```

```
            T[0] = poly_add(T[0], poly_mul(poly_mul(v, w), RM[(e, f, a, b)]))
    for e in range(8):
        for f in range(8):
            if (c, e, b, f) in RM and (d, f, a, e) in RM:  # R^ab_cd R^ce_bf R^df_ae
                T[1] = poly_add(T[1], poly_mul(poly_mul(v, RM[(c, e, b, f)]),
                                               RM[(d, f, a, e)]))
        if (c, d, b, e) in RM:  # R^ab_cd R^cd_be R^e_a
            T[2] = poly_add(T[2], poly_mul(poly_mul(v, RM[(c, d, b, e)]), RIC[e, a]))
    T[4] = poly_add(T[4], poly_mul(poly_mul(v, RIC[c, a]), RIC[d, b]))  # R R^c_a R^d_b
T[3] = poly_mul(R_SCALAR, riem_square)  # R R^ab_cd R^cd_ab
T[5] = poly_sum(poly_mul(poly_mul(RIC[a, b], RIC[b, c]), RIC[c, a])
                for a in range(8) for b in range(8) for c in range(8))
T[6] = poly_mul(R_SCALAR, ric_square)  # R R^a_b R^b_a
T[7] = poly_mul(poly_mul(R_SCALAR, R_SCALAR), R_SCALAR)  # R^3
density = {}
for coefficient, t in zip([2, 8, 24, 3, 24, 16, -12, 1], T):
    density = poly_add(density, t, coefficient)
check(L[3] == poly_add({}, density, 8),
      "L(3) = 8 (2 T1 + 8 T2 + 24 T3 + 3 T4 + 24 T5 + 16 T6 - 12 T7 + T8)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "L3_equals_8_cubic_lovelock_density")
```

Out [20]:

```
PASS L(3) = 8 (2 T1 + 8 T2 + 24 T3 + 3 T4 + 24 T5 + 16 T6 - 12 T7 + T8)
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, check
    L3_equals_8_cubic_lovelock_density
```

Text cell:

```
**The expansion along the column of $h$, for every order (derived line by line).**
The delta of $P_{(k)}$ is the determinant of a $(2k + 1) \times (2k + 1)$ matrix
whose rows belong to the lower labels $j, j_1, \dots, j_{2k}$ (in this order) and
whose columns belong to the upper labels $h, h_1, \dots, h_{2k}$. Expand it along
its first column, the column of $h$ (Laplace's rule, section 3).

1. The first entry of the column is $\delta^h_j$, in row 1 and column 1: its sign
   is $(-1)^{1+1} = +1$, and its minor is $\delta^{h_1 \dots h_{2k}}_{j_1 \dots
   j_{2k}}$. Multiplied by the curvature factors and summed, it gives
   $\delta^h_j\, L_{(k)}$, by the definition of the scalar.
2. The entry in row $r + 1$ ($r = 1, \dots, 2k$) is $\delta^h_{j_r}$, with the sign
   $(-1)^{(r+1)+1} = (-1)^r$. Its minor has the rows $j, j_1, \dots, j_{r-1},
   j_{r+1}, \dots, j_{2k}$. Moving the row of $j$ down past the $r - 1$ rows
   $j_1, \dots, j_{r-1}$ is $r - 1$ exchanges of neighbouring rows, so the minor is
   $(-1)^{r-1}$ times the delta whose lower list is $j_1, \dots, j_{2k}$ with $j$
   in place $r$. The two signs give $(-1)^r (-1)^{r-1} = -1$.
3. The factor $\delta^h_{j_r}$ is 1 only for $j_r = h$, so the sum over $j_r$
   puts $h$ into the curvature factor that held $j_r$. Term $r$ is therefore
   $-\delta^{h_1 \dots h_{2k}}_{j_1 \dots j \dots j_{2k}} R \cdots R$, with $j$ in
   place $r$ of the delta and $h$ in place $r$ of the upper labels of the factors.
4. The $2k$ terms are equal. If $r$ is the second place of its factor
   $R^{j_{r-1} j_r}{}_{h_{r-1} h_r}$, exchange the rows $r - 1$ and $r$ of the
```

```
   delta (sign $-1$) and write $R^{j_{r-1} h}{}_{\dots} = -R^{h j_{r-1}}{}_{\dots}$
   (sign $-1$): the term is unchanged and $h$ stands first. If the factor is not
   the first one, exchange it with the first factor: the product of the factors is
   unchanged, and in the delta two pairs of rows and the two matching pairs of
   columns change places; each such exchange of two pairs is two exchanges of two
   rows (or columns), sign $(-1)^2 = +1$ for the rows and $+1$ for the columns.
   After renaming the summed labels every term is the term $r = 1$.
5. Hence, with $Y_{(k)}{}^h{}_j = \sum \delta^{h_1 \dots h_{2k}}_{j\, j_2 \dots
   j_{2k}}\, R^{h j_2}{}_{h_1 h_2} R^{j_3 j_4}{}_{h_3 h_4} \cdots
   R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}}$,

$$P_{(k)}{}^h{}_j = \delta^h_j\, L_{(k)} - 2k\, Y_{(k)}{}^h{}_j .$$

For $k = 1$: $Y_{(1)}{}^h{}_j = \delta^{h_1 h_2}_{j j_2} R^{h j_2}{}_{h_1 h_2} =
R^{h j_2}{}_{j j_2} - R^{h j_2}{}_{j_2 j} = 2 R^h{}_j$, and $P_{(1)} = 2R\,\delta
- 4 R^h{}_j = -4G$, the result above. The next cell computes $Y_{(k)}$ for
$k = 1, 2, 3$ with GKD (the sum of section 8, with the first entry required to
carry the upper label $h$) and checks the expansion for all 64 components exactly.
```

In [21]:

```
def expansion_sum(k):
    """Y_(k)^h_j as {(h, j): polynomial}: like lovelock_sum, but the first entry
    R^{h j2}_{h1 h2} must have the upper label h, and the delta has the lower list
    (j, j2, ..., j2k) and the upper list (h1, ..., h2k)."""
    result = {}
    for h in range(8):
        first = [i for i, (key, _) in enumerate(ENTRIES) if key[0] == h]
        for j in range(8):
            bucket = {}
            stack = []
            for i in first:  # the first factor R^{h j2}_{h1 h2}
                a, b, c, d = ENTRIES[i][0]
                if b != j:  # j2 = j would repeat j in the lower list: GKD = 0
                    stack.append(((i,), (1 << j) | (1 << b), (1 << c) | (1 << d)))
            while stack:
                chosen, lower_mask, upper_mask = stack.pop()
                if len(chosen) == k:  # a leaf
                    if lower_mask != upper_mask:
                        continue  # a lower label is missing above: GKD = 0
                    lower = [j, ENTRIES[chosen[0]][0][1]] + [
                        x for i in chosen[1:] for x in ENTRIES[i][0][:2]]
                    upper = [x for i in chosen for x in ENTRIES[i][0][2:]]
                    sign = gkd(lower, upper)
                    if sign != 0:
                        key = tuple(sorted(chosen))
                        bucket[key] = bucket.get(key, 0) + sign
                    continue
                free = np.nonzero(((LOWER_MASK & lower_mask) == 0)
                                  & ((UPPER_MASK & upper_mask) == 0))[0]
                for i in free:
                    stack.append((chosen + (int(i),), lower_mask | int(LOWER_MASK[i]),
                                  upper_mask | int(UPPER_MASK[i])))
```

```
            total = {}
            for key, weight in bucket.items():
                product = ONE
                for i in key:
                    product = poly_mul(product, ENTRIES[i][1])
                total = poly_add(total, product, weight)
            result[h, j] = total
    return result


Y = {k: expansion_sum(k) for k in (1, 2, 3)}
expansion_holds = all(
    P[k].get((h, j), {})
    == poly_add(poly_mul(delta[h, j], L[k]), Y[k][h, j], -2 * k)
    for k in (1, 2, 3) for h in range(8) for j in range(8))
check(expansion_holds and all(Y[1][hj] == poly_add({}, RIC[hj], 2) for hj in RIC),
      "P(k) = delta L(k) - 2k Y(k) for all 64 components, k = 1, 2, 3; Y(1) = 2 Ric")
```

Out [21]:

```
PASS P(k) = delta L(k) - 2k Y(k) for all 64 components, k = 1, 2, 3; Y(1) = 2 Ric
```

Text cell:

```
**The trace identity.** Set $j = h$ in the expansion and sum over $h$. The first
term gives $\sum_h \delta^h_h\, L_{(k)} = 8\, L_{(k)}$ (eight values of $h$). In
$\sum_h Y_{(k)}{}^h{}_h$ the label $h$ is summed like $j_1$ in the definition of
the scalar; renaming it $j_1$ gives exactly $L_{(k)}$. So
$\sum_h P_{(k)}{}^h{}_h = 8 L_{(k)} - 2k\, L_{(k)} = (8 - 2k)\, L_{(k)}$. The next
cell checks it for $k = 1, 2, 3$, and prints the three scalars.
```

In [22]:

```
def show(p):
    """A polynomial dict as text, with a4' and a4'' written A1 and A2."""
    return str(sp.expand(sum(v * H ** m[0] * A1 ** m[1] * A2 ** m[2] * C ** m[3]
                             for m, v in p.items())))


for k in (1, 2, 3):
    trace = poly_sum(P[k].get((h, h), {}) for h in range(8))
    report(f"Lovelock scalar L({k})", show(L[k]))
    check(trace == poly_add({}, L[k], 8 - 2 * k),
          f"the trace of P({k}) is (8 - {2 * k}) L({k})",
          record=f"{RESULTS}/lovelock-report.json, check k{k}_trace_identity")
```

Out [22]:

```
RESULT Lovelock scalar L(1) = 12*A1**2 - 84*H**2
PASS the trace of P(1) is (8 - 2) L(1)
```

```
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    k1_trace_identity
RESULT Lovelock scalar L(2) = -96*A1**4 - 2112*A1**2*H**2 + 3360*H**4
PASS the trace of P(2) is (8 - 4) L(2)
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    k2_trace_identity
RESULT Lovelock scalar L(3) = 1152*A1**6 + 31104*A1**4*H**2 + 100224*A1**2*H**4 -
    40320*H**6
PASS the trace of P(3) is (8 - 6) L(3)
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, check
    k3_trace_identity
```

Text cell:

```
## 10. The components of the three tensors at one moment

The field equations of $a_4$ (chapter 12) use the normalised tensors
$E_{(k)} = -P_{(k)}/2^{k+1}$ of section 4. The next cell draws all 64 components
of $E_{(1)}$ (Einstein's tensor $G$), $E_{(2)}$ and $E_{(3)}$ as three
$8 \times 8$ tables, at the moment $a_4' = 2H$, $a_4'' = H^2$ of a deflating
history (the same moment as the right table of the plane curvatures), in units of
$H^2$, $H^4$ and $H^6$ (that is, with $H = 1$). Each square shows the number of row
$h$ and column $j$; red is positive, blue negative, white zero, and each table has
its own colour scale (its largest value is the darkest colour). The tables show what
sections 7 and 8 proved: only the diagonal is filled, the $x_4$-$x_0$ squares are
zero, the three space entries are equal, the three extra-time entries are equal,
and the hidden entry $x_0$ lies halfway between the space and the extra-time entry.
```

In [23]:

```
def poly_value(p, h, a1, a2, cot=1):
    """The exact value (a Fraction) of the polynomial dict p at H = h, a4' = a1,
    a4'' = a2 and cot z = cot (whole numbers or fractions)."""
    total = Fraction(0)
    for (e_h, e_1, e_2, e_c), coefficient in p.items():
        total += (coefficient * Fraction(h) ** e_h * Fraction(a1) ** e_1
                  * Fraction(a2) ** e_2 * Fraction(cot) ** e_c)
    return total


fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.6))
tables = {}
for ax, k in zip(axes, (1, 2, 3)):
    scale = Fraction(-1, 2 ** (k + 1))  # E_(k) = -P_(k) / 2^(k+1)
    # every component at a4' = 2H, a4'' = H^2 with H = 1, exactly, then as a decimal
    table = np.array([[float(scale * poly_value(P[k].get((h, j), {}), 1, 2, 1))
                       for j in range(8)] for h in range(8)])
    tables[k] = table
    largest = np.abs(table).max()
    ax.imshow(table, cmap="RdBu_r", vmin=-largest, vmax=largest)
    for h in range(8):
        for j in range(8):
```

```
            if h != j:
                colour = "0.6"  # grey zeros off the diagonal
            elif abs(table[h, j]) > 0.6 * largest:
                colour = "white"  # white on the darkest squares, to stay readable
            else:
                colour = "black"
            ax.text(j, h, f"{table[h, j]:g}", ha="center", va="center",
                    fontsize=6.5 if h == j else 6, color=colour)
    ax.set_xticks(range(8), NAMES)
    ax.set_yticks(range(8), NAMES)
    ax.set_xlabel("lower label $j$")
    ax.set_ylabel("upper label $h$")
    ax.set_title(f"$E_{{({k})}}{{}}^h{{}}_j$ in units of $H^{{{2 * k}}}$")
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "lovelock_component_tables",
            "All 64 components $E_{(k)}{}^h{}_j$ of the normalised Lovelock tensors "
            "$E_{(1)} = G$ (left), $E_{(2)}$ (middle) and $E_{(3)}$ (right) of the "
            "author's metric at the moment $a_4^{\\prime} = 2H$, "
            "$a_4^{\\prime\\prime} = H^2$ of a deflating history, in units of $H^2$, "
            "$H^4$ and $H^6$; row $h$, column $j$; red positive, blue negative, white "
            "zero, each table on its own colour scale. Only the diagonal is filled, "
            "so the $x_4$-$x_0$ components vanish; the three space entries are equal, "
            "the three extra-time entries are equal, and the hidden entry is their "
            "mean, for example $(4 + 2)/2 = 3$ for Einstein's tensor.")
diagonal_ok = all(
    np.count_nonzero(tables[k] - np.diag(np.diag(tables[k]))) == 0
    and tables[k][1, 1] == tables[k][2, 2] == tables[k][3, 3]
    and tables[k][5, 5] == tables[k][6, 6] == tables[k][7, 7]
    and tables[k][0, 0] == (tables[k][1, 1] + tables[k][5, 5]) / 2
    for k in (1, 2, 3))
say("diagonals (x0 ... x7): " + "; ".join(
    f"E({k}): " + " ".join(f"{v:g}" for v in np.diag(tables[k])) for k in (1, 2, 3)))
check(diagonal_ok and list(np.diag(tables[1])) == [3, 4, 4, 4, 33, 2, 2, 2],
      "at a4' = 2H, a4'' = H^2: only diagonal entries, space and extra-time entries "
      "equal, hidden entry their mean; G = diag(3, 4, 4, 4, 33, 2, 2, 2) H^2")
```

Out [23]:

```
Figure 11b.5 saved as Revision/textbook/figures/11b_5_lovelock_component_tables.png
diagonals (x0 ... x7): E(1): 3 4 4 4 33 2 2 2; E(2): 684 548 548 548 -1476 820 820 820;
    E(3): -22392 -14544 -14544 -14544 40248 -30240 -30240 -30240
PASS at a4' = 2H, a4'' = H^2: only diagonal entries, space and extra-time entries equal,
    hidden entry their mean; G = diag(3, 4, 4, 4, 33, 2, 2, 2) H^2
```

Text cell:

```
## 11. The Revision records quoted in this notebook

This notebook quotes check names and numbers from the Revision records. The four
result files of the Rust program were compared in full in section 5 (byte for byte),
and its counters in sections 6 and 8. The next cell reads the two verification
```

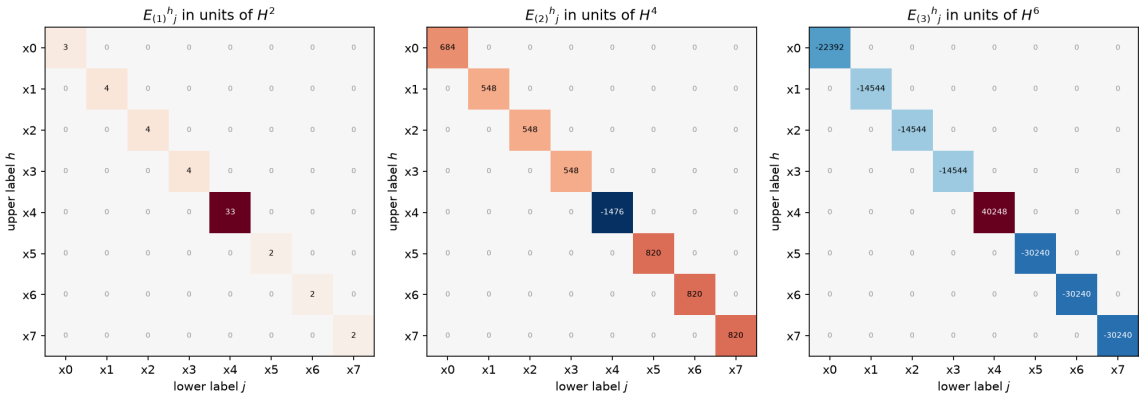

Figure 313: All 64 components $E_{(k)}{}^{h}{}_{j}$ of the normalised Lovelock tensors $E_{(1)} = G$ (left), $E_{(2)}$ (middle) and $E_{(3)}$ (right) of the author's metric at the moment $a_4' = 2H$, $a_4'' = H^2$ of a deflating history, in units of H^2, H^4 and H^6; row h, column j; red positive, blue negative, white zero, each table on its own colour scale. Only the diagonal is filled, so the x_4-x_0 components vanish; the three space entries are equal, the three extra-time entries are equal, and the hidden entry is their mean, for example $(4 + 2)/2 = 3$ for Einstein's tensor. (Notebook 11b, figure 5.)

```
reports once more and checks the rest of what the text quotes, so that a later
change of a record cannot pass unnoticed:

- every check quoted from the sympy verification ‘python-lovelock-report.json‘ and
  from the Rust report ‘lovelock-report.json‘ is present with the verdict PASS;
- the sympy verification has 49 checks and the Wolfram verification
  ‘wolfram-gkd-report.json‘ has 29 checks, none of them failed, as section 4 says.

The function ‘verdicts(path)‘ returns, for one report, a dictionary from the name of
each check to its verdict. The reports store their checks in one of two forms: a
list of entries, each with a ‘name‘ and a ‘verdict‘, or a dictionary from the name to
an entry whose field ‘passed‘ is true or false; the function reads both.
```

In [24]:

```
def verdicts(path):
    """{check name: "PASS" or "FAIL"} of the Revision report at ‘path‘."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    checks = data["checks"]
    if isinstance(checks, dict):  # name -> {"passed": true or false, ...}
        return {name: "PASS" if entry["passed"] else "FAIL"
                for name, entry in checks.items()}
    return {entry["name"]: entry["verdict"] for entry in checks}  # a list of entries


QUOTED = {  # report -> the checks that this notebook quotes from it
    "python-lovelock-report.json": [
        "rust_riemann_agrees", "rust_k1_mixed_components_agree",
        "rust_k2_mixed_components_agree", "rust_k3_mixed_components_agree",
        "rust_L1_agrees", "rust_L2_agrees", "rust_L3_agrees",
        "k1_unpruned_literal_sum_agrees", "k2_unpruned_literal_sum_agrees",
```

```
        "k1_equals_minus_4_einstein", "L1_equals_2R",
        "k2_equals_minus_8_gauss_bonnet", "L2_equals_4_gauss_bonnet",
        "L3_equals_8_cubic_lovelock_density", "normalisation_P1_derived_minus_4",
        "normalisation_P2_derived_minus_8", "normalisation_L3_cubic_derived"],
    "lovelock-report.json": [
        "riemann_antisymmetry", "mixed_riemann_free_of_sin_third",
        "k1_trace_identity", "k2_trace_identity", "k3_trace_identity",
        "k4_tensor_vanishes", "k1_brute_force_numeric", "k2_brute_force_numeric"],
}
for report_name, names in QUOTED.items():
    found = verdicts(f"{RESULTS}/{report_name}")
    missing = [name for name in names if found.get(name) != "PASS"]
    say(f"{report_name}: {len(names)} quoted checks; not there or not PASS: "
        f"{missing if missing else 'none'}")
    check(not missing,
          f"the {len(names)} checks quoted from {report_name} are there and PASS",
          record=f"{RESULTS}/{report_name}, checks " + ", ".join(names))
totals = {}
for report_name in ("python-lovelock-report.json", "wolfram-gkd-report.json"):
    data = json.loads(repository_file(f"{RESULTS}/{report_name}").read_text(
        encoding="utf-8"))
    totals[report_name] = (data["checkCount"], data["failedCheckCount"],
                           data["verdict"])
    say(f"{report_name}: {data['checkCount']} checks, {data['failedCheckCount']} "
        f"failed, verdict {data['verdict']}")
check(totals == {"python-lovelock-report.json": (49, 0, "SUCCESS"),
                 "wolfram-gkd-report.json": (29, 0, "SUCCESS")},
      "the sympy verification has 49 checks and the Wolfram verification 29, none "
      "failed",
      record=f"{RESULTS}/python-lovelock-report.json and wolfram-gkd-report.json, "
             "checkCount and failedCheckCount")
```

Out [24]:

```
python-lovelock-report.json: 17 quoted checks; not there or not PASS: none
PASS the 17 checks quoted from python-lovelock-report.json are there and PASS
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json, checks
    rust_riemann_agrees, rust_k1_mixed_components_agree, rust_k2_mixed_components_agree,
    rust_k3_mixed_components_agree, rust_L1_agrees, rust_L2_agrees, rust_L3_agrees,
    k1_unpruned_literal_sum_agrees, k2_unpruned_literal_sum_agrees,
    k1_equals_minus_4_einstein, L1_equals_2R, k2_equals_minus_8_gauss_bonnet,
    L2_equals_4_gauss_bonnet, L3_equals_8_cubic_lovelock_density,
    normalisation_P1_derived_minus_4, normalisation_P2_derived_minus_8,
    normalisation_L3_cubic_derived
lovelock-report.json: 8 quoted checks; not there or not PASS: none
PASS the 8 checks quoted from lovelock-report.json are there and PASS
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, checks
    riemann_antisymmetry, mixed_riemann_free_of_sin_third, k1_trace_identity,
    k2_trace_identity, k3_trace_identity, k4_tensor_vanishes, k1_brute_force_numeric,
    k2_brute_force_numeric
python-lovelock-report.json: 49 checks, 0 failed, verdict SUCCESS
wolfram-gkd-report.json: 29 checks, 0 failed, verdict SUCCESS
PASS the sympy verification has 49 checks and the Wolfram verification 29, none failed
```

reproduces Revision/gkd_lovelock/results/python-lovelock-report.json and wolfram-gkd-report.json, checkCount and failedCheckCount

Text cell:

```
The last cell checks that all five figure files exist and prints the number of
checks that passed.
```

In [25]:

```
figure_files = [f"11b_{k}_{name}.png" for k, name in enumerate(
    ["work_of_the_sums", "curvature_of_planes", "riemann_matrix",
     "mixing_terms_cancel", "lovelock_component_tables"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_files),
      "all five figure files of the notebook exist")
all_checks_passed()
```

Out [25]:

```
PASS all five figure files of the notebook exist
ALL 38 CHECKS PASSED (notebook 11b)
```

Text cell:

```
## 12. What this notebook showed

- The Revision Rust program ‘lovelock_gkd‘, built here, computed the curvature and
  the three Lovelock tensors of the author’s metric, passed its 19 checks (including
  the literal unpruned sums of orders 1 and 2), and wrote its four result files
  again, byte for byte equal to the Revision records.
- The Riemann tensor, recomputed independently with sympy, has the same 156 nonzero
  components; they depend only on $H$, $a_4’$, $a_4’’$ and $\cot z$. Planes inside
  ordinary space or inside the extra times have the curvature $a_4’^2 - H^2$, planes
  between them $-(a_4’^2 + H^2)$, planes with the hidden direction $-H^2$.
- The mixed $x_4$-$x_0$ curvature terms of the three inflating and the three
  deflating directions cancel exactly.
- Pruning (only nonzero curvature entries, no repeated labels) reduces the sum of
  order 3 from about $4.4 \times 10^{12}$ terms to 495,360 GKD calls; a Python
  implementation of the same plan, written independently, reproduces every counter
  of the Rust program and all $3 \times 64$ components and the three scalars exactly.
  The literal sums of orders 1 and 2, with GKD called on every one of 10,140 and
  1,581,840 index lists and nothing skipped, give the same tensors: the skipping
  changes nothing.
- Every Lovelock tensor expands along the column of its free upper label as
  $P_{(k)} = \delta\, L_{(k)} - 2k\, Y_{(k)}$ (derived line by line and checked
  exactly for $k = 1, 2, 3$); for $k = 1$ this gives Einstein’s tensor, and for
  every $k$ the trace identity $\sum_h P_{(k)}{}^h{}_h = (8 - 2k) L_{(k)}$.
- Only the eight diagonal components of each Lovelock tensor are nonzero; at a
  moment of a deflating history the three space entries are equal, the three
```

```
  extra-time entries are equal and the hidden entry is their mean. Order 1 is
  $-4$ times Einstein's tensor, order 2 is $-8$ times the Gauss-Bonnet tensor, the
  third scalar is 8 times the cubic Lovelock density, and $P_{(4)} = 0$ (PROVED by
  exact computation).
- Every check that the notebook quotes from the Revision verifications is present in
  them with the verdict PASS, and the sympy and Wolfram verifications have 49 and 29
  checks, none of which failed.
```

11.19 Line-by-line walk-through of Notebook 11b

The notebook has 25 code cells, In [1] to In [25]. As in Section 11.9, every line or small group of lines is quoted and explained; docstrings are left out where the explanation repeats them, and long captions are shortened to `...` (they are printed in full under the figures in Section 11.18).

In [1], the set-up cell. It is word for word the set-up cell of Notebook 11a, explained line by line in Section 11.9 under In [1], with two differences: its comment lines are the run instructions of Notebook 11b (Section 11.17), and its line `NOTEBOOK_ID = "11b" # this notebook: chapter 11, example b` names this notebook, so the figures are saved as `11b_<k>_<name>.png` and their captions in `11b.captions.json`. It contains the helper `rust_program`. It prints Set-up of notebook 11b complete: repository folder found, helpers defined.

In [2], the Rust program is built.

```
program = rust_program("Revision/gkd_lovelock/code/Cargo.toml", "lovelock_gkd")
```

The same line as In [13] of Notebook 11a: cargo builds the crate (or confirms in about a second that it is up to date), and `program` is the path of `lovelock_gkd`. Output: Rust program lovelock_gkd is built and ready.

In [3], the program computes everything. The command it runs is

```
lovelock_gkd lovelock --output FOLDER --brute-force-k2
```

with the option that adds the literal unpruned sum of order 2 at a numerical point (Section 11.12).

```
import re  # regular expressions: patterns that read parts of a line of text

RESULTS = "Revision/gkd_lovelock/results"  # the folder of the Revision records
OUT_FOLDER = REPO / "Revision" / "gkd_lovelock" / "code" / "target" / "textbook_11b"
completed = subprocess.run(
    [str(program), "lovelock", "--output", str(OUT_FOLDER), "--brute-force-k2"],
    capture_output=True, text=True, encoding="utf-8", errors="replace")
```

The module `re` reads parts of the printed lines. `RESULTS` is the folder of the Revision records, and `OUT_FOLDER` a folder inside the crate's build folder `target`, which git ignores, so the run adds nothing to the repository. `subprocess.run` starts the program and collects what it prints (Section 11.9, In [1] and In [14]). The program computes, exactly and from the metric alone, the Christoffel symbols, the Riemann tensor and the Lovelock tensors of orders 1 to 4, runs its 19 checks and writes four files; this takes about 15 seconds.

```
rust_checks = []  # (name, verdict) of every check the program printed
for line in completed.stdout.splitlines():
    found = re.match(r"(PASS|FAIL) - (\w+): ", line)
    if found:
        rust_checks.append((found.group(2), found.group(1)))
    if line.startswith("info - L_(4)"):
        l4_text = line.split("): ", 1)[1]  # the formula after the explanation
```

Each printed line is examined. `re.match` tests whether the line BEGINS with the pattern: PASS or FAIL (`(PASS|FAIL)` means one or the other), a dash, and a name made of letters, digits and underscores (`\w+`), followed by a colon. `found.group(2)` is the name and `found.group(1)` the verdict; the pair is stored. The program also prints an information line with the scalar $L_{(4)}$; `line.split("): ", 1)` cuts that line once at the first `):` and `[1]` keeps the part after it, the formula.

```
say("The checks of the Rust program (name, verdict):")
for number, (name, verdict) in enumerate(rust_checks, 1):
    say(f"  {number:2d} {name:34s} {verdict}")
```

The 19 checks are printed as a numbered table (the format `:34s` pads a text to 34 places). They are: the antisymmetry and the first Bianchi identity of the Riemann tensor, the freedom of every $R^{ab}{}_{cd}$ from $\sin^{1/3} z$, the identity $P_{(1)} = -4G$; for each order $k = 1, 2, 3$ the trace identity, zero divergence, symmetry and freedom from $\sin^{1/3} z$; the vanishing of $P_{(4)}$; and the two brute-force sums. All 19 print PASS.

```
readable = (l4_text.replace("Derivative[1][a4][x4]", "a4'").replace("*", " ")
            .replace("(", "").replace(")", ""))
report("eighth-order scalar printed by the program, L(4)", readable)
```

The program writes a_4' in Mathematica's notation `Derivative[1][a4][x4]`; the chain of `replace` calls turns it into `a4'`, multiplication stars into blanks, and removes parentheses. The RESULT line prints $L_{(4)} = -663552H^2a_4'^6 - 442368H^4a_4'^4 - 663552H^6a_4'^2$, the nonzero Euler density of Section 11.11.

```
check(completed.returncode == 0 and "lovelock: SUCCESS" in completed.stdout,
      "the Rust program lovelock_gkd lovelock ended with SUCCESS")
check(len(rust_checks) == 19 and all(v == "PASS" for _, v in rust_checks),
      "all 19 checks of the Rust program passed",
      record=f"{RESULTS}/lovelock-report.json, checkCount 19, failedCheckCount 0")
```

Two checks: the program ended with the return code 0 and the verdict SUCCESS, and it printed exactly 19 checks, all PASS, as the record states. PASS lines one and two.

In [4], the four files against the records.

```
DEVIATION = re.compile(r"(max relative deviation from the exact P_\(\d\) = )"
                       r"([0-9.]+e[-+]?\d+)")  # the two floating-point numbers
```

`re.compile` prepares a pattern for repeated use. It finds the text "max relative deviation from the exact P_(1) = " (or P_(2); `\d` is one digit, `\(` and `\)` literal parentheses) as the first group, and the decimal number that follows as the second group: digits and points, the letter e, an optional sign and digits, for example `1.40e-15`.

```
for file_name in ("curvature.json", "lovelock-tensors.json",
                  "lovelock-components.md"):
    written = (OUT_FOLDER / file_name).read_bytes()
    stored = repository_file(f"{RESULTS}/{file_name}").read_bytes()
    report(f"size of {file_name}", len(written), "bytes")
    check(written == stored,
          f"the program wrote {file_name} equal to the Revision record byte for byte",
          record=f"{RESULTS}/{file_name} (the whole file)")
```

The three files that hold only exact numbers are compared with the committed records byte for byte, and their sizes are printed: 28,162, 55,056 and 19,604 bytes. PASS lines three to five.

```
written = (OUT_FOLDER / "lovelock-report.json").read_text(encoding="utf-8")
stored = repository_file(f"{RESULTS}/lovelock-report.json").read_text(
    encoding="utf-8")
deviations = [float(number) for _, number in DEVIATION.findall(written)]
masked_written = DEVIATION.sub(r"\1X", written)  # the two numbers replaced by X
masked_stored = DEVIATION.sub(r"\1X", stored)
```

The report holds two decimal numbers, the rounding errors of the two brute-force checks, whose last digits may depend on the mathematics library of the operating system. `DEVIATION.findall(written)` finds both, and `float` turns the second group of each into a decimal number. `DEVIATION.sub(r"\1X", ...)` replaces every match by its first group followed by the letter X, so the two numbers disappear from both texts.

```
report("size of lovelock-report.json", len(written.encode("utf-8")), "bytes")
check(masked_written == masked_stored and len(deviations) == 2
      and all(value < 1e-10 for value in deviations),
      "the program wrote lovelock-report.json equal to the Revision record byte for "
      "byte, apart from its two brute-force deviations, both below 1e-10",
      record=f"{RESULTS}/lovelock-report.json (the whole file; checks "
             "k1_brute_force_numeric and k2_brute_force_numeric)")
```

The size is printed (3,154 bytes). The check requires the masked texts to be equal, exactly two deviations, and both below the program's own limit 10^{-10}. (On the computer that built the book the file was equal byte for byte even without the masking.) Sixth PASS line.

In [5], the work of the sums (Figure 11b.1).

```
import numpy as np  # arrays of numbers

lovelock_report = json.loads((OUT_FOLDER / "lovelock-report.json").read_text(
    encoding="utf-8"))
COUNTERS = {row["k"]: row for row in lovelock_report["counters"]}
```

The report just compared is read as JSON; its field `counters` is a list of four rows, one per order, and the dictionary comprehension stores each row under its order k.

```
say("order k   literal lists   ordered products    leaves   GKD calls   nonzero GKD")
for k in (1, 2, 3, 4):
    leaves, calls = COUNTERS[k]["leaves"], COUNTERS[k]["gkdCalls"]
    nonzero = COUNTERS[k]["gkdNonzero"]
    say(f"      {k}   {64 * 8 ** (4 * k):13.2e}   {64 * 156 ** k:16.2e}   "
        f"{leaves:7d}   {calls:9d}   {nonzero:11d}")
```

A table of Section 11.12: for each order the literal number $64 \cdot 8^{4k}$ of index lists, the number $64 \cdot 156^k$ of ordered products of nonzero entries, and the record's leaves, GKD calls and nonzero GKD values. The output shows 5616, 176640, 1128960, 0 leaves and 696, 32640, 495360, 0 calls, each call nonzero.

```
fig, ax = plt.subplots(figsize=(7.6, 4.5))
orders = np.array([1, 2, 3, 4])
series = [("literal index lists, $64 \\cdot 8^{4k}$", [64 * 8 ** (4 * k)
                                                       for k in orders], "0.7"),
          ("ordered products of nonzero entries, $64 \\cdot 156^k$",
           [64 * 156 ** k for k in orders], "tab:purple"),
          ("leaves after skipping repeated labels",
           [COUNTERS[k]["leaves"] for k in orders], "tab:blue"),
          ("GKD calls", [COUNTERS[k]["gkdCalls"] for k in orders], "tab:orange")]
```

The four series of bars, each a name for the legend, four heights and a colour.

```
width = 0.2
for number, (name, heights, colour) in enumerate(series):
    place = orders + (number - 1.5) * width
    ax.bar(place, [h if h > 0 else np.nan for h in heights], width, color=colour,
           label=name)
    for x, h in zip(place, heights):
        if h == 0:
            ax.text(x, 1.5, "0", ha="center", fontsize=8)
```

Each series is shifted sideways by its own amount (adding a number to the numpy array `orders` adds it to every entry). A height 0 is drawn as `np.nan` ("not a number", which matplotlib leaves out), because 0 has no place on a logarithmic axis, and a small "0" is written instead.

```
ax.set_yscale("log")
ax.set_ylim(1.0, 1e18)
ax.set_xlim(0.5, 4.5)
ax.set_xticks(orders, [f"k = {k}" for k in orders])
ax.set_xlabel("order $k$ of the Lovelock tensor")
ax.set_ylabel("number of terms (logarithmic scale)")
ax.set_title("The work of the Lovelock sums, and how it is reduced")
ax.legend(loc="upper left", fontsize=8)
save_figure(fig, "work_of_the_sums",
            "The number of terms of the Lovelock sums for the 64 components of "
            ...)
```

A logarithmic vertical axis from 1 to 10^{18}, the marks, titles, legend, and the saved figure.

```
check([COUNTERS[k]["leaves"] for k in (1, 2, 3, 4)] == [5616, 176640, 1128960, 0]
      and [COUNTERS[k]["gkdCalls"] for k in (1, 2, 3, 4)] == [696, 32640, 495360, 0],
      "the counters of the Rust sums are 5616, 176640, 1128960, 0 leaves and 696, "
      "32640, 495360, 0 GKD calls")
```

The check fixes the counters quoted in the text. Seventh PASS line.

What Figure 11b.1 shows. For each order four bars on a logarithmic axis: grey (all literal index lists), purple (ordered products of nonzero curvature entries), blue (leaves without a repeated label) and orange (GKD calls). For $k = 3$ the grey bar reaches 4.4×10^{12} while the orange one stays at 495,360, about one term in nine million. For $k = 4$ the blue and orange bars are missing (marked 0): no leaf exists, because nine labels out of eight must repeat.

In [6], the metric and its Christoffel symbols with sympy.

```
import sympy as sp  # exact algebra and calculus

H = sp.Symbol("H", positive=True)  # the author's constant H > 0
X = sp.symbols("x0:8", real=True)  # the coordinates x0, ..., x7
a4 = sp.Function("a4")(X[4])  # the unknown function a4(x4)
b4 = sp.Function("b4")(X[4])  # the unknown second function b4(x4) > 0
z = 6 * H * X[0]  # z = 6 H x0
warp = sp.sin(z) ** sp.Rational(1, 3)  # sin(z)^(1/3)
```

`sp.Symbol` makes a symbol that sympy computes with exactly; `positive=True` tells sympy that $H > 0$. `sp.symbols("x0:8")` makes the eight symbols `x0` to `x7` (the range 0 to 7), stored in the tuple `X`, so `X[k]` is x_k: `X[4]` is x_4 and `X[0]` is x_0. `sp.Function("a4")(X[4])` and `sp.Function("b4")(X[4])` are unknown functions of x_4, which sympy can differentiate: the history a_4 and the second function b_4 of the author's metric of 2026-10-14. `sp.Rational(1, 3)` is the exact fraction $1/3$, so `warp` is $\sin^{1/3} z$.

```
METRIC = ([sp.cot(z) ** 2] + [sp.exp(2 * a4) * warp] * 3 + [sp.Integer(-1)]
          + [-sp.exp(-2 * a4) * b4 * warp] * 3)  # g_00, g_11, ..., g_77
```

The diagonal of the author's metric as a list of eight entries: a list multiplied by 3 repeats it three times, and `+` joins lists. So `METRIC[0]` is $\cot^2 z$, `METRIC[1]` to `METRIC[3]` are $e^{2a_4} \sin^{1/3} z$, `METRIC[4]` is -1, and `METRIC[5]` to `METRIC[7]` are $-e^{-2a_4} b_4 \sin^{1/3} z$.

```
christoffel = {}  # (a, b, c) -> Gamma^a_bc, only the nonzero ones
for a in range(8):
    for b in range(8):
        for c in range(8):
            value = 0
            if a == c:
                value += sp.diff(METRIC[a], X[b])  # d_b g_ac
            if a == b:
                value += sp.diff(METRIC[a], X[c])  # d_c g_ab
```

```
            if b == c:
                value -= sp.diff(METRIC[b], X[a])  # - d_a g_bc
            if value != 0:
                christoffel[a, b, c] = value / (2 * METRIC[a])  # times g^aa / 2
```

The Christoffel formula of Section 11.10 for a diagonal metric: $\partial_b g_{ac}$ is nonzero only for $c = a$, $\partial_c g_{ab}$ only for $b = a$, and $\partial_a g_{bc}$ only for $c = b$. The three `if` lines add exactly these terms (`sp.diff(f, x)` is the exact derivative of `f` with respect to `x`). A nonzero result is divided by $2g_{aa}$, which is the factor $\frac{1}{2}g^{aa}$, and stored under the key `(a, b, c)`.

```
def gamma(a, b, c):
    return christoffel.get((a, b, c), 0)


christoffel_b_le_c = [key for key in christoffel if key[1] <= key[2]]
report("nonzero Christoffel symbols Gamma^a_bc with b <= c", len(christoffel_b_le_c))
check(all(christoffel.get((a, c, b)) == value
          for (a, b, c), value in christoffel.items()),
      "Gamma^a_bc = Gamma^a_cb (symmetric in the lower labels)")
```

`gamma(a, b, c)` returns a symbol, or 0 when it is not stored. The list `christoffel_b_le_c` keeps the symbols with $b \le c$; there are 25 (RESULT line), the 25 of the record (Section 3.23). The check confirms the symmetry $\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$. Eighth PASS line.

In [7], the Riemann tensor with sympy.

```
A1, A2, C = sp.symbols("A1 A2 C")  # a4', a4'' and cot(z)
B0, B1, B2 = sp.symbols("B0 B1 B2")  # b4, b4' and b4''
SYMBOLS = {sp.Derivative(a4, (X[4], 2)): A2, sp.Derivative(a4, X[4]): A1,
           sp.Derivative(b4, (X[4], 2)): B2, sp.Derivative(b4, X[4]): B1}
RIEMANN = {}  # (a, b, c, d) -> R^ab_cd in the symbols H, A1, A2, C
```

Three plain symbols stand for a_4', a_4'' and $\cot z$, and three more, `B0`, `B1` and `B2`, for b_4, b_4' and b_4''. The dictionary `SYMBOLS` says which derivative becomes which symbol (each second derivative is listed before the first, so that it is replaced before the first derivative inside it); b_4 itself becomes `B0` afterwards. `RIEMANN` will hold the nonzero components.

```
for a in range(8):
    for b in range(8):
        for c in range(8):
            for d in range(8):
                value = sp.diff(gamma(a, d, b), X[c]) - sp.diff(gamma(a, c, b), X[d])
                for e in range(8):
                    value += (gamma(a, c, e) * gamma(e, d, b)
                              - gamma(a, d, e) * gamma(e, c, b))
                if value == 0:
                    continue
```

The formula of Section 11.10 for $R^a{}_{bcd}$, for all $8^4 = 4096$ combinations of labels: two derivative terms and the sum over e of the two products. `continue` skips to the next combination when the result is exactly zero.

```
                value = sp.simplify((value / METRIC[b]).subs(SYMBOLS)
                                    .subs(b4, B0))  # R^ab_cd
                value = sp.expand(value.subs({sp.tan(z): 1 / C, sp.cot(z): C,
                                              sp.cos(z): C * sp.sin(z)}))
                if value != 0:
                    RIEMANN[a, b, c, d] = value
report("nonzero components R^ab_cd", len(RIEMANN))
```

Dividing by g_{bb} raises the second label: $R^{ab}{}_{cd} = R^{a}{}_{bcd}/g_{bb}$. `.subs(SYMBOLS)` puts `A1`, `A2`, `B1` and `B2` in place of the derivatives, `.subs(b4, B0)` puts `B0` in place of b_4, and `sp.simplify` simplifies (this takes most of the cell's fifteen seconds). Then $\tan z$, $\cot z$ and $\cos z$ are written with the symbol `C` ($\tan z = 1/C$, $\cos z = C \sin z$), and `sp.expand` multiplies out. The nonzero results are stored: 156 of them (RESULT line), as Section 11.10 counted.

```
check(all(v.free_symbols <= {H, A1, A2, B0, B1, B2, C} for v in RIEMANN.values()),
      "every R^ab_cd is a rational expression in H, a4', a4'', b4, b4', b4'' and "
      "cot z, free of sin(z)^(1/3) and e^a4",
      record=f"{RESULTS}/lovelock-report.json, check mixed_riemann_free_of_sin_third")
```

`v.free_symbols` is the set of symbols in a component; `<=` between two sets means "is contained in". If any component still contained x_0 (through $\sin z$) or x_4 (through e^{a_4}), the check would fail. Ninth PASS line.

In [8], sympy against the Rust record.

```
curvature = json.loads((OUT_FOLDER / "curvature.json").read_text(encoding="utf-8"))
NAMES = curvature["coordinates"]  # ["x0", ..., "x7"]


def from_mathematica(text):
    text = re.sub(r"Derivative\[(\d)\]\[a4\]\[x4\]", r"A\1", text)  # a4' -> A1
    text = re.sub(r"Derivative\[(\d)\]\[b4\]\[x4\]", r"B\1", text)  # b4' -> B1
    text = text.replace("b4[x4]", "B0")  # b4 -> B0
    text = text.replace("Cot[6*H*x0]", "C").replace("^", "**")
    return sp.sympify(text, locals={"H": H, "A1": A1, "A2": A2, "C": C, "B0": B0,
                                    "B1": B1, "B2": B2})
```

The program's file `curvature.json` is read; its list of coordinate names becomes `NAMES`. `from_mathematica` turns a component written in Mathematica's notation, for example `H*Derivative[1][a4][x4]*Cot[6*H*x0]^(-1)`, into a sympy expression: `re.sub` replaces `Derivative[1][a4][x4]` by `A1` and `Derivative[2][a4][x4]` by `A2` (the group `(\d)` captures the digit and `\1` puts it back), and in the same way the derivatives of b_4 by `B1` and `B2`; `b4[x4]` becomes `B0`, $\cot z$ becomes `C`, Mathematica's power sign `^` becomes Python's `**`, and `sp.sympify` reads the text with the seven names bound to our symbols.

```
rust_riemann = {}
for item in curvature["riemannMixedNonzero"]:
    a, b = (NAMES.index(name) for name in item["up"])
    c, d = (NAMES.index(name) for name in item["down"])
```

```
    rust_riemann[a, b, c, d] = from_mathematica(item["value"])
same_values = set(rust_riemann) == set(RIEMANN) and all(
    sp.simplify(RIEMANN[key] - rust_riemann[key]) == 0 for key in RIEMANN)
check(same_values, "sympy and Rust give the same 156 nonzero R^ab_cd exactly, "
      "with b4",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "check rust_riemann_agrees")
```

Each of the record's nonzero components carries its upper pair, its lower pair and its value; `NAMES.index(name)` turns a name into its number. The check requires the same set of label combinations (a set of the keys of a dictionary) and an exactly vanishing difference for every component. Tenth PASS line.

```
antisymmetric = all(sp.expand(v + RIEMANN.get((b, a, c, d), 0)) == 0 and
                    sp.expand(v + RIEMANN.get((a, b, d, c), 0)) == 0
                    for (a, b, c, d), v in RIEMANN.items())
check(antisymmetric, "R^ab_cd = -R^ba_cd = -R^ab_dc for all components",
      record=f"{RESULTS}/lovelock-report.json, check riemann_antisymmetry")
check(len(christoffel_b_le_c) == len(curvature["christoffelNonzero_b_le_c"]) == 25,
      "25 nonzero Christoffel symbols with b <= c, as in the record")
```

The antisymmetry of Section 11.10: each component plus the component with the upper pair exchanged, and plus the one with the lower pair exchanged, must vanish. The second check compares the number of Christoffel symbols with the record's list (`a == b == 25` tests both equalities). PASS lines eleven and twelve.

```
ricci_scalar = sp.expand(sum(RIEMANN.get((a, b, a, b), 0) for a in range(8)
                             for b in range(8)))
report("Ricci scalar R", ricci_scalar)
check(sp.simplify(ricci_scalar - from_mathematica(curvature["ricciScalar"])) == 0
      and sp.expand(ricci_scalar.subs({B0: 1, B1: 0, B2: 0}))
      == 6 * A1 ** 2 - 42 * H ** 2,
      "the Ricci scalar is R = 6 a4'^2 - 42 H^2 + 3 b4''/b4 - 3 a4' b4'/b4, as in the "
      "record (6 a4'^2 - 42 H^2 at b4 = 1)",
      record=f"{RESULTS}/curvature.json, ricciScalar")
# From here on: the background b4 = 1 (b4' = b4'' = 0) of the Revision records.
RIEMANN_B4 = RIEMANN  # the components with a general b4
BACKGROUND = {B0: 1, B1: 0, B2: 0}
RIEMANN = {key: sp.expand(value.subs(BACKGROUND)) for key, value in RIEMANN_B4.items()}
RIEMANN = {key: value for key, value in RIEMANN.items() if value != 0}
check(len(RIEMANN) == 156,
      "on the background b4 = 1 the same 156 components are nonzero")
```

$R = \sum_{a,b} R^{ab}{}_{ab}$, twice the sum of the 28 plane curvatures. The RESULT line prints `6*A1**2 - 3*A1*B1/B0 - 42*H**2 + 3*B2/B0`, and the check compares it with the record and requires $6a_4'^2 - 42H^2$ at $b_4 = 1$. Then the notebook turns to the background $b_4 = 1$ of the records: `RIEMANN_B4` keeps the components with a general b_4, `BACKGROUND` is the substitution $b_4 = 1$, $b_4' = b_4'' = 0$, and `RIEMANN` now holds the components there, dropping those that become 0 (none does: the second check requires the same 156). PASS lines thirteen and fourteen.

In [9], the plane curvatures (Figure 11b.2).

```
def value_at(expr, h=1.0, a1=2.0, a2=0.0, cot=1.0):
    return float(sp.sympify(expr).subs({H: h, A1: a1, A2: a2, C: cot}))
```

`value_at` puts numbers in place of the four symbols (by default $H = 1$, $a_4' = 2$, $a_4'' = 0$, $\cot z = 1$) and returns a decimal number; `sp.sympify` also accepts the plain 0 of a missing component.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 5.0))
planes = {}
for ax, a2 in zip(axes, (0.0, 1.0)):
    table = np.array([[value_at(RIEMANN.get((a, b, a, b), 0), a2=a2)
                       for b in range(8)] for a in range(8)])
    planes[a2] = table
    picture = ax.imshow(table, cmap="RdBu_r", vmin=-6, vmax=6)
```

Two panels, for $a_4'' = 0$ and $a_4'' = H^2$. Each `table` is the 8×8 array of the plane curvatures $K(a, b) = R^{ab}{}_{ab}$ (zero on the diagonal); it is kept in `planes` for the check and drawn with the colour map `RdBu_r` (red positive, blue negative) from -6 to 6.

```
    for a in range(8):
        for b in range(8):
            if a != b:
                ax.text(b, a, f"{table[a, b]:g}", ha="center", va="center",
                        fontsize=8)
    ax.set_xticks(range(8), NAMES)
    ax.set_yticks(range(8), NAMES)
    ax.set_title(f"$a_4' = 2H$, $a_4'' = {a2:g}\\,H^2$")
    ax.grid(False)
fig.colorbar(picture, ax=list(axes), shrink=0.8, label="$K(a, b)$ in units of $H^2$")
```

The values off the diagonal are written into the squares (the format `:g` writes a number without needless zeros), the rows and columns get the coordinate names, each panel its title, and one colour bar serves both panels.

```
save_figure(fig, "curvature_of_planes",
            "The curvature $K(a, b) = R^{ab}_{\\ ab}$ of the plane of the coordinate "
            ...)
check(planes[0.0][1, 2] == 3.0 and planes[0.0][1, 5] == -5.0
      and planes[0.0][1, 0] == -1.0 and planes[1.0][1, 4] == 5.0
      and planes[1.0][5, 4] == 3.0 and planes[0.0][4, 0] == 0.0,
      "plane curvatures at a4' = 2H: 3, -5, -1; with x4: 4 + a4'' and 4 - a4''; "
      "x4 with x0: 0")
```

After saving, the check reads six entries: $K(x_1, x_2) = 4 - 1 = 3$, $K(x_1, x_5) = -(4 + 1) = -5$, $K(x_1, x_0) = -1$, $K(x_1, x_4) = 4 + 1 = 5$ and $K(x_5, x_4) = 4 - 1 = 3$ at $a_4'' = H^2$, and $K(x_4, x_0) = 0$ (in units of H^2), the table of Section 11.10. Fifteenth PASS line.

What Figure 11b.2 shows. Two 8×8 tables of the plane curvatures in units of H^2. Red blocks of 3 inside 3-space and inside the extra times, blue blocks of -5 between them, light blue -1 for every plane with x_0, and 0 for the plane of x_4 and x_0. The row and column of x_4 hold 4 on the left; on the

right, with $a_4'' = H^2$, they hold 5 towards 3-space and 3 towards the extra times: the acceleration of a_4 enters 3-space and the extra times with opposite signs.

In [10], the whole Riemann tensor (Figure 11b.3).

```
point = {"h": 1.0, "a1": 2.0, "a2": 1.0, "cot": 1.0 / np.sqrt(3.0)}
whole = np.zeros((64, 64))
for (a, b, c, d), value in RIEMANN.items():
    whole[8 * a + b, 8 * c + d] = value_at(value, **point)
```

The point $H = 1$, $a_4' = 2$, $a_4'' = 1$, $z = \pi/3$, where $\cot z = 1/\sqrt{3}$. `np.zeros((64, 64))` is a 64×64 array of zeros; every nonzero component is put into row $8a + b$ (the upper pair) and column $8c + d$ (the lower pair). `**point` passes the four entries of the dictionary as the named arguments of `value_at`.

```
fig, ax = plt.subplots(figsize=(6.8, 6.2))
largest = np.abs(whole).max()
picture = ax.imshow(whole, cmap="RdBu_r", vmin=-largest, vmax=largest)
for edge in range(8, 64, 8):  # lines between the blocks
    ax.axhline(edge - 0.5, color="0.8", lw=0.5)
    ax.axvline(edge - 0.5, color="0.8", lw=0.5)
```

The colour scale runs from minus to plus the largest size, so that 0 is white. Thin grey lines separate the blocks of eight rows and columns that share their first label.

```
ticks = list(range(3, 64, 8))  # the middle of each block
ax.set_xticks(ticks, [f"c = {name}" for name in NAMES], rotation=90, fontsize=8)
ax.set_yticks(ticks, [f"a = {name}" for name in NAMES], fontsize=8)
ax.set_xlabel("lower pair $(x_c, x_d)$, column $8c + d$")
ax.set_ylabel("upper pair $(x_a, x_b)$, row $8a + b$")
ax.set_title("All components $R^{ab}{}_{cd}$ at one point")
ax.grid(False)
fig.colorbar(picture, ax=ax, shrink=0.8, label="value (units of $H^2$)")
save_figure(fig, "riemann_matrix",
            "All 4096 components of the Riemann tensor $R^{ab}_{\\ \\ cd}$ of the "
            ...)
check(int(np.count_nonzero(whole)) == 156,
      "156 of the 4096 components are not zero at this point")
```

Marks at the middle of each block, axis titles, title, colour bar and the saved figure; the check counts the nonzero entries of the array: 156 (`np.count_nonzero`). Sixteenth PASS line.

What Figure 11b.3 shows. A 64×64 table, mostly white. Along the main diagonal ($a = c$, $b = d$) lie the plane curvatures; along the second diagonal pattern ($a = d$, $b = c$) their sign-reversed copies. The scattered squares away from these lines are the 48 components that mix x_4 and x_0, such as ${R^{x_1 x_4}}_{x_1 x_0} = H a_4' \cot z$.

In [11], the mixed terms cancel (Figure 11b.4).

```
mixing = [sp.expand(RIEMANN.get((4, c, 0, c), 0)) for c in range(1, 8)]  # x1..x7
for c, term in enumerate(mixing, 1):
```

```
    say(f"R^(x4 {NAMES[c]})_(x0 {NAMES[c]}) = {term}")
total = sp.expand(sum(mixing))
say(f"sum over c = {total} (b4 = 1)")
total_b4 = sp.simplify(sum(RIEMANN_B4.get((4, c, 0, c), 0) for c in range(1, 8)))
say(f"with a general b4: sum over c = {total_b4}")
```

The seven terms $R^{x_4 c}{}_{x_0 c}$ of the Ricci component $R^{x_4}{}_{x_0}$, for $c = x_1$ to x_7 (numbers 4 and 0 are x_4 and x_0; `enumerate(mixing, 1)` counts the terms from 1, so that `NAMES[c]` names the direction). The output: `A1*C*H` three times, 0 for $c = x_4$, `-A1*C*H` three times, and the sum 0 at $b_4 = 1$, as derived in Section 11.10. `total_b4` is the same sum with a general b_4 (from `RIEMANN_B4`): `3*B1*C*H /(2*B0)`, that is $\frac{3}{2} H \cot z \, b_4'/b_4$; the second check compares it with this formula and with the record's mixed component of the Einstein tensor.

```
fig, ax = plt.subplots(figsize=(7.0, 4.0))
heights = [value_at(term, h=1.0, a1=1.0, cot=1.0) for term in mixing]
colours = ["tab:red"] * 3 + ["0.6"] + ["tab:blue"] * 3
ax.bar(NAMES[1:], heights, color=colours)
ax.text(3, 0.05, "0", ha="center", va="bottom")  # the term of c = x4 is zero
ax.axhline(0.0, color="black", lw=0.8)
```

The seven terms at $H = 1$, $a_4' = 1$, $\cot z = 1$ as bars, red for 3-space, grey for x_4, blue for the extra times; `NAMES[:7]` are the first seven names. A small "0" marks the bar of height zero, and a black line marks the value 0.

```
ax.set_xlabel("the direction $c$ of the term $R^{x_4 c}{}_{x_0 c}$")
ax.set_ylabel("value (units of $H^2$)")
ax.set_title(f"Seven terms of $R^{{x_4}}{{}}_{{x_0}}$; their sum is {total}")
save_figure(fig, "mixing_terms_cancel",
            "The seven terms $R^{x_4 c}_{\\ \\ x_0 c}$, $c = x_1$ to $x_7$, whose sum "
            "is the Ricci component $R^{x_4}_{\\ x_0}$, at $H = 1$, "
            ...)
check(total == 0 and sp.expand(mixing[0] - H * A1 * C) == 0,
      "the seven terms of R^x4_x0 are +H a4' cot z (three times), -H a4' cot z (three "
      "times) and 0, and cancel exactly at b4 = 1")
check(sp.simplify(total_b4 - sp.Rational(3, 2) * H * C * B1 / B0) == 0
      and sp.simplify(total_b4 - from_mathematica(
          curvature["einsteinMixed"]["x4,x0"]["mathematica"])) == 0,
      "with b4: R^x4_x0 = (3/2) H cot z b4'/b4, the record's G^x4_x0",
      record=f"{RESULTS}/curvature.json, einsteinMixed x4,x0")
```

In an f-string a doubled brace `{{` prints one brace, so the title reads $R^{x_4}{}_{x_0}$ followed by the sum. The check requires the sum to be exactly 0 and the first term to be $H a_4' \cot z$. PASS lines seventeen and eighteen.

What Figure 11b.4 shows. Three red bars of height $+1$ (x_1, x_2, x_3), a grey bar of height 0 (x_4) and three blue bars of height -1 (x_5, x_6, x_7), in units of H^2: the three inflating and the three deflating directions cancel exactly.

In [12], exact polynomial arithmetic and GKD.

```
from fractions import Fraction  # exact fractions of whole numbers

ONE = {(0, 0, 0, 0): Fraction(1)}  # the polynomial 1
```

From here on the notebook computes with its own exact polynomials, independent of sympy's: a polynomial in H, A_1, A_2, C is a dictionary from the exponents $(e_H, e_{A_1}, e_{A_2}, e_C)$ of each monomial to its coefficient, a `Fraction` (an exact fraction of whole numbers); e_C may be negative. `ONE` is the polynomial 1.

```
def poly_mul(p, q):
    out = {}
    for m1, c1 in p.items():
        for m2, c2 in q.items():
            m = (m1[0] + m2[0], m1[1] + m2[1], m1[2] + m2[2], m1[3] + m2[3])
            out[m] = out.get(m, 0) + c1 * c2
    return {m: c for m, c in out.items() if c != 0}
```

The product of two polynomials (docstring left out): every monomial of the first times every monomial of the second, which adds the exponents and multiplies the coefficients; equal monomials are collected, and monomials whose coefficient became 0 are dropped.

```
def poly_add(p, q, factor=1):
    out = dict(p)
    for m, c in q.items():
        out[m] = out.get(m, 0) + factor * c
    return {m: c for m, c in out.items() if c != 0}
```

`poly_add(p, q, factor)` is $p + \text{factor} \cdot q$: a copy of `p` (`dict(p)`), to which each monomial of `q` times the factor is added; zeros are dropped.

```
def to_poly(expr):
    poly = sp.Poly(sp.expand(expr * C ** 8), H, A1, A2, C)
    return {(m[0], m[1], m[2], m[3] - 8): Fraction(int(v.p), int(v.q))
            for m, v in zip(poly.monoms(), poly.coeffs())}
```

`to_poly` turns a sympy expression into this form. `sp.Poly` needs non-negative powers, so the expression is first multiplied by C^8 (no component has a power of C below -8); `poly.monoms()` lists the exponents and `poly.coeffs()` the coefficients of the monomials; 8 is subtracted from the exponent of C again, and each sympy fraction `v` becomes a `Fraction` of its numerator `v.p` and denominator `v.q`.

```
def gkd(lower, upper):
    position = {}
    for place, label in enumerate(upper):
        if label in position:
            return 0
        position[label] = place
    sigma = [position.get(label) for label in lower]
```

```
    if None in sigma or len(set(sigma)) < len(sigma):
        return 0
    inversions = sum(1 for i in range(len(sigma)) for j in range(i + 1, len(sigma))
                     if sigma[i] > sigma[j])
    return 1 if inversions % 2 == 0 else -1
```

The rule GKD of Notebook 11a (In [4]), written more compactly: `position.get(label)` gives `None` for a lower label that is missing above, and the test `None in sigma` then returns 0 (case c); a repeated place returns 0 (case a); otherwise the sign of the inversions.

```
ENTRIES = [(key, to_poly(RIEMANN[key])) for key in sorted(RIEMANN)]  # 156 entries
LOWER_MASK = np.array([(1 << a) | (1 << b) for (a, b, c, d), _ in ENTRIES])
UPPER_MASK = np.array([(1 << c) | (1 << d) for (a, b, c, d), _ in ENTRIES])
```

The 156 nonzero entries, sorted by their labels, each with its polynomial. For each entry $R^{ab}{}_{cd}$ the bit masks of Section 11.12 are stored: its upper labels a, b go into the delta's LOWER list and its lower labels c, d into the delta's UPPER list. The operator written with two less-than signs shifts the number 1 to the left by the given number of places, so it is 2^a; the vertical bar is the bitwise OR, so the mask of $\{a, b\}$ is $2^a + 2^b$. `np.array` holds the 156 masks, so that one comparison can test all of them at once.

```
report("nonzero curvature entries in the sums", len(ENTRIES))
check(gkd([0, 1, 2], [1, 2, 0]) == 1 and gkd([0, 1], [1, 0]) == -1,
      "gkd: a cycle of three labels is even (+1), an exchange of two odd (-1)",
      record=f"{RESULTS}/lovelock-report.json, gkdSelfCheck")
```

RESULT: 156 entries. The check repeats the two values that the Rust program records for itself in the field `gkdSelfCheck` of its report: $+1$ for the cycle $(0, 1, 2) \to (1, 2, 0)$ and -1 for one exchange. Nineteenth PASS line.

In [13], the Lovelock sum. This cell only defines a function; it prints nothing.

```
def lovelock_sum(k, free_pair):
    pairs = [(h, j) for h in range(8) for j in range(8)] if free_pair else [None]
    weights, leaves, calls = {}, 0, 0
```

`lovelock_sum(k, free_pair)` computes $P_{(k)}$ for all 64 pairs (h, j) when `free_pair` is true, and $L_{(k)}$ (one sum, marked `None`) otherwise. `weights` collects, per pair, the summed GKD weight of each set of entries; `leaves` and `calls` count.

```
    for pair in pairs:
        start_lower = 1 << pair[1] if pair else 0  # j is the first lower label
        start_upper = 1 << pair[0] if pair else 0  # h is the first upper label
        stack = [((), start_lower, start_upper)]
```

For a tensor component the delta's lower list starts with j and its upper list with h, so the masks start with these labels; for the scalar they start empty. The **stack** is a list of partial combinations still to be extended, each a tuple of the chosen entry numbers with the two masks; it starts with the empty combination.

```
        while stack:
            chosen, lower_mask, upper_mask = stack.pop()
            if len(chosen) == k:  # a leaf
                leaves += 1
                if lower_mask != upper_mask:
                    continue  # a lower label is missing above: GKD = 0
```

`while stack:` repeats as long as the stack is not empty; `stack.pop()` takes the last partial combination off it. A combination of k entries is a leaf and is counted. If its two masks differ, some lower label is missing above, GKD would be 0 (case c), and the leaf is dropped (step 3 of Section 11.12).

```
                lower = ([pair[1]] if pair else []) + [
                    x for i in chosen for x in ENTRIES[i][0][:2]]  # j, j1, ..., j2k
                upper = ([pair[0]] if pair else []) + [
                    x for i in chosen for x in ENTRIES[i][0][2:]]  # h, h1, ..., h2k
                sign = gkd(lower, upper)
                calls += 1
```

The two lists of the delta: j followed by the upper labels of the chosen entries in order (`ENTRIES[i][0][:2]` is the pair a, b of entry `i`), and h followed by their lower labels (`[2:]` is c, d). GKD is called, and the call is counted.

```
                if sign != 0:
                    bucket = weights.setdefault(pair, {})
                    key = tuple(sorted(chosen))  # the set of entries
                    bucket[key] = bucket.get(key, 0) + sign
                continue
```

A nonzero value is added to the weight of the SET of chosen entries (`sorted` makes the key independent of the order), because the product of the entries does not depend on their order. `continue` goes back to the top of the `while` loop.

```
            free = np.nonzero(((LOWER_MASK & lower_mask) == 0)
                              & ((UPPER_MASK & upper_mask) == 0))[0]
            for i in free:  # every entry that repeats no label
                stack.append((chosen + (int(i),), lower_mask | int(LOWER_MASK[i]),
                              upper_mask | int(UPPER_MASK[i])))
```

A combination with fewer than k entries is extended (step 2 of Section 11.12). `LOWER_MASK & lower_mask` is computed for all 156 entries at once; `== 0` marks the entries that share no label with the lower list so far, and the same for the upper list; `&` between the two true/false arrays keeps the entries that pass both tests, and `np.nonzero(...)[0]` lists their numbers. Each such entry is appended to the combination, and the masks grow by its labels (bitwise OR).

```
    products = {}  # set of entries -> the product of their polynomials
    result = {}
    for pair, bucket in weights.items():
```

```
        total = {}
        for key, weight in bucket.items():
            if weight == 0:
                continue
            if key not in products:
                product = ONE
                for i in key:
                    product = poly_mul(product, ENTRIES[i][1])
                products[key] = product
            total = poly_add(total, products[key], weight)
        if total:
            result[pair] = total
    return result, leaves, calls
```

After the search, every set of entries with a nonzero total weight is multiplied out once (and remembered in `products`, because the same set appears for many pairs), multiplied by its weight and added to the component (step 4). Components that are not zero are returned in `result`, together with the two counters.

In [14], the sums of orders 1 to 3.

```
P, L, MY_COUNTERS = {}, {}, {}
for k in (1, 2, 3):
    P[k], leaves, calls = lovelock_sum(k, free_pair=True)
    scalar, scalar_leaves, scalar_calls = lovelock_sum(k, free_pair=False)
    L[k] = scalar.get(None, {})
    MY_COUNTERS[k] = (leaves, calls, scalar_calls)
    say(f"k = {k}: leaves {leaves:7d}, GKD calls {calls:6d}, scalar GKD calls "
        f"{scalar_calls:6d}, nonzero components {len(P[k])}")
```

For each order the tensor and the scalar are computed (about ten seconds, mostly for $k = 3$), and the counters are stored and printed: 5616, 176640, 1128960 leaves; 696, 32640, 495360 GKD calls; 108, 7200, 213120 scalar calls; and 8 nonzero components for every order.

```
same_counters = all(
    MY_COUNTERS[k] == (COUNTERS[k]["leaves"], COUNTERS[k]["gkdCalls"],
                       COUNTERS[k]["scalarGkdCalls"])
    and COUNTERS[k]["gkdNonzero"] == COUNTERS[k]["gkdCalls"]
    and len(P[k]) == 8 and COUNTERS[k]["nonzeroComponents"] == 10
    for k in (1, 2, 3))
check(same_counters, "the Python sums reproduce the counters of the Rust sums; "
      "nonzero components: 8 at b4 = 1, 10 (with x4-x0, x0-x4) for a general b4",
      record=f"{RESULTS}/lovelock-report.json, counters k = 1, 2, 3")
```

The Python search, written independently, finds exactly the counters of the Rust search, and every GKD call of the record is nonzero. The numbers of nonzero components differ in the expected way: 8 on the background $b_4 = 1$ in Python, 10 in the record of the Rust program, which keeps b_4 general and also finds the two mixed components x_4-x_0 and x_0-x_4. Twentieth PASS line.

In [15], every component against the record.

```
tensors = json.loads((OUT_FOLDER / "lovelock-tensors.json").read_text(
    encoding="utf-8"))


def from_monomials(monomials):
    out = {}
    for numerator, denominator, e in monomials:
        if any(e[9:13]):  # a derivative of b4: zero on the background b4 = 1
            continue
        if e[3] or e[4] or e[5] or e[6]:  # 3rd, 4th derivative, e^a4 or sin^(1/3)
            raise ValueError("unexpected symbol in a Lovelock component")
        key = (e[0], e[1], e[2], e[7])  # e[8], the power of b4^(1/2), drops out
        out[key] = out.get(key, 0) + Fraction(numerator, denominator)
    return {key: value for key, value in out.items() if value != 0}
```

The program's file lists every component as exact monomials, each a numerator, a denominator and thirteen exponents (of H, a_4', a_4'', a_4''', a_4'''', e^{a_4}, $\sin^{1/3} z$, $\cot z$, $Q = b_4^{1/2}$, b_4', b_4'', b_4''', b_4''''). `from_monomials` rebuilds a component on the background $b_4 = 1$ as a polynomial dictionary: a monomial with a derivative of b_4 (`any(e[9:13])`) is left out, and the power of Q drops out because $Q = 1$, so that monomials which then agree are added (`out.get(key, 0) + ...`); if any of the exponents 3 to 6 were not 0 it would stop, so the components contain only H, a_4', a_4'' and $\cot z$.

```
agree = {}
for k in (1, 2, 3):
    block = tensors[f"P{k}_mixed_up_h_down_j"]
    agree[k] = all(from_monomials(block[f"{NAMES[h]},{NAMES[j]}"]["monomials"])
                   == P[k].get((h, j), {}) for h in range(8) for j in range(8))
    nonzero = " ".join(sorted(f"{NAMES[h]},{NAMES[j]}" for h, j in P[k]))
    say(f"k = {k}: nonzero components (h, j): {nonzero}")
```

For each order the record's 64 components, stored under keys such as `"x1,x1"`, are compared exactly with the Python components (a missing Python component is the empty polynomial, zero). The nonzero components are printed: for every order only the eight diagonal ones, x_0x_0 to x_7x_7.

```
check(all(agree.values()),
      "all 3 x 64 components of P(1), P(2), P(3) equal the Rust components exactly",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "rust_k1_mixed_components_agree, rust_k2_mixed_components_agree and "
             "rust_k3_mixed_components_agree")
scalars_agree = all(to_poly(from_mathematica(tensors[f"L{k}"]).subs(BACKGROUND))
                    == L[k] for k in (1, 2, 3))
check(scalars_agree, "the scalars L(1), L(2), L(3) equal the Rust scalars exactly "
      "(at b4 = 1)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "rust_L1_agrees, rust_L2_agrees and rust_L3_agrees")
check(all(set(P[k]) == {(h, h) for h in range(8)} for k in (1, 2, 3)),
      "only the 8 diagonal components are nonzero, for k = 1, 2, 3")
```

Three checks: all 3×64 components agree exactly; the three scalars of the record (in Mathematica text, read with `from_mathematica`) equal the Python scalars; and the nonzero components are

exactly the eight diagonal ones (a set comprehension in braces). PASS lines twenty-one to twenty-three. In particular no Lovelock tensor has an x_4-x_0 component at $b_4 = 1$ (the record's mixed components contain b'_4 and vanish there).

In [16], the literal sums of orders 1 and 2.

```
import itertools  # all ordered lists of k entries


def literal_sum(k):
    pairs = [None] + [(h, j) for h in range(8) for j in range(8)]
    weights, calls = {}, 0
    for chosen in itertools.product(range(len(ENTRIES)), repeat=k):
        lower = [x for i in chosen for x in ENTRIES[i][0][:2]]  # j1, ..., j2k
        upper = [x for i in chosen for x in ENTRIES[i][0][2:]]  # h1, ..., h2k
        key = tuple(sorted(chosen))  # the set of entries of this product
```

`literal_sum` does NOT skip anything: `itertools.product(range(156), repeat=k)` runs through all 156^k ordered lists of k entries, and for each the two lists of the delta are built.

```
        for pair in pairs:
            if pair is None:  # the scalar: no free pair
                sign = gkd(lower, upper)
            else:  # the tensor: j first in the lower list, h first in the upper
                sign = gkd([pair[1]] + lower, [pair[0]] + upper)
            calls += 1
            if sign != 0:
                bucket = weights.setdefault(pair, {})
                bucket[key] = bucket.get(key, 0) + sign
```

For the scalar and for each of the 64 pairs, GKD is called on the complete lists, with no test before the call: $65 \cdot 156^k$ calls. Nonzero values are added to the weights as before.

```
    result = {}
    for pair, bucket in weights.items():
        total = {}
        for key, weight in bucket.items():
            product = ONE
            for i in key:
                product = poly_mul(product, ENTRIES[i][1])
            total = poly_add(total, product, weight)
        if total:
            result[pair] = total
    return result, calls
```

The weighted products are multiplied out and added, as in In [13], and the components are returned with the number of calls.

```
record_checks = {c["name"]: c for c in json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json").read_text(
        encoding="utf-8"))["checks"]}
```

The checks of the Revision's sympy report are read into a dictionary by name.

```
for k in (1, 2):
    literal, calls = literal_sum(k)
    recorded = record_checks[f"k{k}_unpruned_literal_sum_agrees"]
    recorded_calls = int(re.search(r"(\d+) kdelta calls", recorded["detail"]).group(1))
    say(f"k = {k}: {calls} GKD calls (the record: {recorded_calls})")
```

For $k = 1$ and 2 the literal sum is computed (about 15 seconds in all), and the number of calls stated in the record's check text (the digits before "kdelta calls") is read. Output: 10140 and 1581840 calls, both equal to the record.

```
    same = (literal.pop(None, {}) == L[k]
            and all(literal.get(hj, {}) == P[k].get(hj, {})
                    for hj in set(literal) | set(P[k])))
    check(same and calls == recorded_calls == 65 * 156 ** k
          and recorded["verdict"] == "PASS",
          f"the literal sum of order {k}, without skipping, gives the same P({k}) "
          f"and L({k})",
          record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
                 f"k{k}_unpruned_literal_sum_agrees")
```

`literal.pop(None, {})` takes the scalar out of the result; it must equal $L_{(k)}$, and every component, over the union (`|`) of the components of both results, must equal $P_{(k)}$. The check also requires the number of calls to be $65 \cdot 156^k$ and the record's verdict PASS. PASS lines twenty-four and twenty-five: the skipping changes nothing.

In [17], order 4 has no term.

```
seven_labels = True
for h in range(8):
    for j in range(8):
        stack = [((), 1 << j, 1 << h)]
        while stack:
            chosen, lower_mask, upper_mask = stack.pop()
            if len(chosen) == 3:  # a leaf of order 3
                seven_labels = seven_labels and bin(lower_mask).count("1") == 7
                continue
```

The search of In [13] is repeated for order 3 without calling GKD. At every leaf `bin(mask)` writes the mask in binary and `.count("1")` counts its ones, the number of labels in the lower list; it must be 7 (j and the six upper labels of three entries).

```
            free = np.nonzero(((LOWER_MASK & lower_mask) == 0)
                              & ((UPPER_MASK & upper_mask) == 0))[0]
            for i in free:
                stack.append((chosen + (int(i),), lower_mask | int(LOWER_MASK[i]),
                              upper_mask | int(UPPER_MASK[i])))
check(seven_labels and COUNTERS[4]["leaves"] == 0,
      "every leaf of order 3 uses 7 labels, so P(4) has no term: P(4) = 0",
      record=f"{RESULTS}/lovelock-report.json, check k4_tensor_vanishes")
```

The extension step as before. Since every leaf of order 3 already uses seven different lower labels, a fourth entry would need nine different labels out of eight; together with the record's counter of 0 leaves for $k = 4$ the check confirms $P_{(4)} = 0$. Twenty-sixth PASS line.

In [18], order 1 is Einstein's tensor.

```
RM = dict(ENTRIES)  # (a, b, c, d) -> R^ab_cd as a polynomial dict


def poly_sum(polynomials):
    total = {}
    for p in polynomials:
        total = poly_add(total, p)
    return total
```

`RM` is the dictionary of the 156 entries; `poly_sum` adds a list of polynomials (docstring left out).

```
RIC = {(h, j): poly_sum(RM.get((h, a, j, a), {}) for a in range(8))
       for h in range(8) for j in range(8)}  # R^h_j
R_SCALAR = poly_sum(RIC[h, h] for h in range(8))  # R
delta = {(h, j): (ONE if h == j else {}) for h in range(8) for j in range(8)}
EINSTEIN = {hj: poly_add(RIC[hj], poly_mul(delta[hj], R_SCALAR), Fraction(-1, 2))
            for hj in RIC}  # G = Ric - R/2
```

The Ricci tensor $R^h{}_j = \sum_a R^{ha}{}_{ja}$ for all 64 pairs, the Ricci scalar, the Kronecker delta as polynomials (1 or empty), and the Einstein tensor $G = \mathrm{Ric} - \frac{1}{2}\delta R$, all with the exact arithmetic of In [12].

```
check(all(P[1].get(hj, {}) == poly_mul({(0, 0, 0, 0): Fraction(-4)}, EINSTEIN[hj])
          for hj in EINSTEIN) and L[1] == poly_add({}, R_SCALAR, 2),
      "P(1) = -4 G for all 64 components and L(1) = 2 R",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "k1_equals_minus_4_einstein and L1_equals_2R")
```

The check of Section 11.14: every component of $P_{(1)}$ equals -4 times the Einstein component (the constant polynomial -4 times G), and $L_{(1)} = 2R$. Twenty-seventh PASS line.

In [19], order 2 is the Gauss-Bonnet tensor.

```
ric_square = poly_sum(poly_mul(RIC[a, b], RIC[b, a]) for a in range(8)
                      for b in range(8))  # R^a_b R^b_a
riem_square = poly_sum(poly_mul(v, RM[(c, d, a, b)]) for (a, b, c, d), v in
                       RM.items() if (c, d, a, b) in RM)  # R^ab_cd R^cd_ab
GB = poly_add(poly_add(poly_mul(R_SCALAR, R_SCALAR), ric_square, -4), riem_square)
```

The two squares $\sum R^a{}_b R^b{}_a$ and $\sum R^{ab}{}_{cd} R^{cd}{}_{ab}$ (only entries whose partner with the pairs exchanged is nonzero contribute), and the Gauss-Bonnet scalar $\mathrm{GB} = R^2 - 4R^a{}_b R^b{}_a + R^{ab}{}_{cd} R^{cd}{}_{ab}$ of Section 11.15.

```
two_h = {}  # 2 H^h_j
for h in range(8):
    for j in range(8):
        t = poly_mul(R_SCALAR, RIC[h, j])
        t = poly_add(t, poly_sum(poly_mul(RIC[h, a], RIC[a, j]) for a in range(8)),
                     -2)
        t = poly_add(t, poly_sum(poly_mul(RM.get((h, a, j, b), {}), RIC[b, a])
                                 for a in range(8) for b in range(8)), -2)
        t = poly_add(t, poly_sum(poly_mul(v, RM.get((b, c, j, a), {}))
                                 for (hh, a, b, c), v in RM.items() if hh == h))
        two_h[h, j] = poly_add(poly_mul({(0, 0, 0, 0): Fraction(4)}, t),
                               poly_mul(delta[h, j], GB), -1)
```

For each pair (h, j) the four sums of Lanczos's tensor (Section 11.15) are collected in `t`: $R\,R^h{}_j$, then $-2\sum_a R^h{}_a R^a{}_j$, then $-2\sum_{a,b} R^{ha}{}_{jb} R^b{}_a$, then $\sum R^{ha}{}_{bc} R^{bc}{}_{ja}$ over the entries with first label h (the name `hh` is the entry's first label). Finally $2\mathcal{H}^h{}_j = 4t - \delta^h_j\,\mathrm{GB}$.

```
check(all(P[2].get(hj, {}) == poly_mul({(0, 0, 0, 0): Fraction(-4)}, two_h[hj])
          for hj in two_h) and L[2] == poly_add({}, GB, 4),
      "P(2) = -8 times the Gauss-Bonnet tensor for all 64 components and L(2) = 4 GB",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, checks "
             "k2_equals_minus_8_gauss_bonnet and L2_equals_4_gauss_bonnet")
```

$-4 \cdot 2\mathcal{H} = -8\mathcal{H}$ must equal $P_{(2)}$ in all 64 components, and $L_{(2)} = 4\,\mathrm{GB}$: the result derived by hand in Section 11.15. Twenty-eighth PASS line.

In [20], the cubic density.

```
T = [{} for _ in range(8)]  # T1 ... T8
for (a, b, c, d), v in RM.items():
    for (c2, d2, e, f), w in RM.items():
        if (c2, d2) == (c, d) and (e, f, a, b) in RM:  # R^ab_cd R^cd_ef R^ef_ab
            T[0] = poly_add(T[0], poly_mul(poly_mul(v, w), RM[(e, f, a, b)]))
```

Eight empty polynomials for the eight invariants (`T[0]` is T_1). For every entry $v = R^{ab}{}_{cd}$ and every entry w whose upper pair is (c, d), the product with $R^{ef}{}_{ab}$ is added to T_1, when that entry is nonzero.

```
    for e in range(8):
        for f in range(8):
            if (c, e, b, f) in RM and (d, f, a, e) in RM:  # R^ab_cd R^ce_bf R^df_ae
                T[1] = poly_add(T[1], poly_mul(poly_mul(v, RM[(c, e, b, f)]),
                                               RM[(d, f, a, e)]))
        if (c, d, b, e) in RM:  # R^ab_cd R^cd_be R^e_a
            T[2] = poly_add(T[2], poly_mul(poly_mul(v, RM[(c, d, b, e)]), RIC[e, a]))
    T[4] = poly_add(T[4], poly_mul(poly_mul(v, RIC[c, a]), RIC[d, b]))  # R R^c_a R^d_b
```

Still inside the loop over $v = R^{ab}{}_{cd}$: $T_2 = R^{ab}{}_{cd} R^{ce}{}_{bf} R^{df}{}_{ae}$ over e, f; $T_3 = R^{ab}{}_{cd} R^{cd}{}_{be} R^e{}_a$ over e; and $T_5 = R^{ab}{}_{cd} R^c{}_a R^d{}_b$ (the comment's first R stands for the entry v).

```
T[3] = poly_mul(R_SCALAR, riem_square)  # R R^ab_cd R^cd_ab
T[5] = poly_sum(poly_mul(poly_mul(RIC[a, b], RIC[b, c]), RIC[c, a])
                for a in range(8) for b in range(8) for c in range(8))
T[6] = poly_mul(R_SCALAR, ric_square)  # R R^a_b R^b_a
T[7] = poly_mul(poly_mul(R_SCALAR, R_SCALAR), R_SCALAR)  # R^3
```

The remaining four invariants: $T_4 = R\,R^{ab}{}_{cd}R^{cd}{}_{ab}$, $T_6 = R^{a}{}_{b}R^{b}{}_{c}R^{c}{}_{a}$, $T_7 = R\,R^{a}{}_{b}R^{b}{}_{a}$, $T_8 = R^3$.

```
density = {}
for coefficient, t in zip([2, 8, 24, 3, 24, 16, -12, 1], T):
    density = poly_add(density, t, coefficient)
check(L[3] == poly_add({}, density, 8),
      "L(3) = 8 (2 T1 + 8 T2 + 24 T3 + 3 T4 + 24 T5 + 16 T6 - 12 T7 + T8)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, check "
             "L3_equals_8_cubic_lovelock_density")
```

The cubic density with the record's coefficients, and the check $L_{(3)} = 8\times$ density. Twenty-ninth PASS line.

In [21], the expansion along the column of h.

```
def expansion_sum(k):
    result = {}
    for h in range(8):
        first = [i for i, (key, _) in enumerate(ENTRIES) if key[0] == h]
```

`expansion_sum(k)` computes $Y_{(k)}$ of Section 11.13 (docstring left out). For each h, `first` lists the entries whose first upper label is h: these are the possible first factors $R^{hj_2}{}_{h_1h_2}$.

```
        for j in range(8):
            bucket = {}
            stack = []
            for i in first:  # the first factor R^{h j2}_{h1 h2}
                a, b, c, d = ENTRIES[i][0]
                if b != j:  # j2 = j would repeat j in the lower list: GKD = 0
                    stack.append(((i,), (1 << j) | (1 << b), (1 << c) | (1 << d)))
```

For each j the search starts with every possible first factor whose second upper label j_2 differs from j (otherwise the delta's lower list $(j, j_2, \dots)$ would repeat a label). The lower mask holds j and j_2; the upper mask the two lower labels h_1, h_2 of the factor. The label h itself is not in the delta of $Y_{(k)}$.

```
            while stack:
                chosen, lower_mask, upper_mask = stack.pop()
                if len(chosen) == k:  # a leaf
                    if lower_mask != upper_mask:
                        continue  # a lower label is missing above: GKD = 0
                    lower = [j, ENTRIES[chosen[0]][0][1]] + [
                        x for i in chosen[1:] for x in ENTRIES[i][0][:2]]
                    upper = [x for i in chosen for x in ENTRIES[i][0][2:]]
```

```
                    sign = gkd(lower, upper)
                    if sign != 0:
                        key = tuple(sorted(chosen))
                        bucket[key] = bucket.get(key, 0) + sign
                    continue
```

The same search as in In [13], with the lists of $Y_{(k)}$: the lower list is j, then j_2 (the second upper label of the first factor), then the upper labels of the other factors (`chosen[1:]` leaves out the first); the upper list is the lower labels of all factors.

```
                free = np.nonzero(((LOWER_MASK & lower_mask) == 0)
                                  & ((UPPER_MASK & upper_mask) == 0))[0]
                for i in free:
                    stack.append((chosen + (int(i),), lower_mask | int(LOWER_MASK[i]),
                                  upper_mask | int(UPPER_MASK[i])))
            total = {}
            for key, weight in bucket.items():
                product = ONE
                for i in key:
                    product = poly_mul(product, ENTRIES[i][1])
                total = poly_add(total, product, weight)
            result[h, j] = total
    return result
```

The extension step and the multiplication of the weighted products, as before; the component (h, j) of $Y_{(k)}$ is stored.

```
Y = {k: expansion_sum(k) for k in (1, 2, 3)}
expansion_holds = all(
    P[k].get((h, j), {})
    == poly_add(poly_mul(delta[h, j], L[k]), Y[k][h, j], -2 * k)
    for k in (1, 2, 3) for h in range(8) for j in range(8))
check(expansion_holds and all(Y[1][hj] == poly_add({}, RIC[hj], 2) for hj in RIC),
      "P(k) = delta L(k) - 2k Y(k) for all 64 components, k = 1, 2, 3; Y(1) = 2 Ric")
```

$Y_{(k)}$ for the three orders, and the check of the expansion $P_{(k)} = \delta L_{(k)} - 2kY_{(k)}$ in all 3×64 components, together with $Y_{(1)} = 2\,\mathrm{Ric}$ of Section 11.14. Thirtieth PASS line.

In [22], the trace identity.

```
def show(p):
    return str(sp.expand(sum(v * H ** m[0] * A1 ** m[1] * A2 ** m[2] * C ** m[3]
                             for m, v in p.items())))
```

`show` turns a polynomial dictionary back into sympy text, for printing (docstring left out).

```
for k in (1, 2, 3):
    trace = poly_sum(P[k].get((h, h), {}) for h in range(8))
    report(f"Lovelock scalar L({k})", show(L[k]))
```

```
    check(trace == poly_add({}, L[k], 8 - 2 * k),
          f"the trace of P({k}) is (8 - {2 * k}) L({k})",
          record=f"{RESULTS}/lovelock-report.json, check k{k}_trace_identity")
```

For each order the trace $\sum_h {P_{(k)}}^h{}_h$ is formed, the scalar is printed, and the trace identity of Section 11.14 is checked. The RESULT lines give $L_{(1)} = 12a_4'^2 - 84H^2$, $L_{(2)} = -96a_4'^4 - 2112a_4'^2H^2 + 3360H^4$ and $L_{(3)} = 1152a_4'^6 + 31104a_4'^4H^2 + 100224a_4'^2H^4 - 40320H^6$; no a_4'' and no $\cot z$ appear. PASS lines thirty-one to thirty-three.

In [23], the components at one moment (Figure 11b.5).

```
def poly_value(p, h, a1, a2, cot=1):
    total = Fraction(0)
    for (e_h, e_1, e_2, e_c), coefficient in p.items():
        total += (coefficient * Fraction(h) ** e_h * Fraction(a1) ** e_1
                  * Fraction(a2) ** e_2 * Fraction(cot) ** e_c)
    return total
```

`poly_value` evaluates a polynomial dictionary exactly, with fractions, at given values of H, a_4', a_4'' and $\cot z$ (docstring left out).

```
fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.6))
tables = {}
for ax, k in zip(axes, (1, 2, 3)):
    scale = Fraction(-1, 2 ** (k + 1))  # E_(k) = -P_(k) / 2^(k+1)
    # every component at a4' = 2H, a4'' = H^2 with H = 1, exactly, then as a decimal
    table = np.array([[float(scale * poly_value(P[k].get((h, j), {}), 1, 2, 1))
                       for j in range(8)] for h in range(8)])
    tables[k] = table
    largest = np.abs(table).max()
    ax.imshow(table, cmap="RdBu_r", vmin=-largest, vmax=largest)
```

Three panels, one per order. Each component of $E_{(k)} = -P_{(k)}/2^{k+1}$ is evaluated exactly at $H = 1$, $a_4' = 2$, $a_4'' = 1$ (the moment $a_4' = 2H$, $a_4'' = H^2$; the components contain no $\cot z$), turned into a decimal number, and the 8×8 table is drawn with its own colour scale.

```
    for h in range(8):
        for j in range(8):
            if h != j:
                colour = "0.6"  # grey zeros off the diagonal
            elif abs(table[h, j]) > 0.6 * largest:
                colour = "white"  # white on the darkest squares, to stay readable
            else:
                colour = "black"
            ax.text(j, h, f"{table[h, j]:g}", ha="center", va="center",
                    fontsize=6.5 if h == j else 6, color=colour)
```

Every value is written into its square: grey for the zeros off the diagonal, white on the darkest squares and black elsewhere.

```
    ax.set_xticks(range(8), NAMES)
    ax.set_yticks(range(8), NAMES)
    ax.set_xlabel("lower label $j$")
    ax.set_ylabel("upper label $h$")
    ax.set_title(f"$E_{{({k})}}{{}}^h{{}}_j$ in units of $H^{{{2 * k}}}$")
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "lovelock_component_tables",
            "All 64 components $E_{(k)}{}^h{}_j$ of the normalised Lovelock tensors "
            ...)
```

Marks, axis titles and the panel titles (the doubled braces print single braces, so the title reads ${E_{(k)}}^h{}_j$ in units of H^{2k}), then the saved figure.

```
diagonal_ok = all(
    np.count_nonzero(tables[k] - np.diag(np.diag(tables[k]))) == 0
    and tables[k][1, 1] == tables[k][2, 2] == tables[k][3, 3]
    and tables[k][5, 5] == tables[k][6, 6] == tables[k][7, 7]
    and tables[k][0, 0] == (tables[k][1, 1] + tables[k][5, 5]) / 2
    for k in (1, 2, 3))
```

`np.diag(table)` takes the diagonal of a table, and `np.diag` of that diagonal builds a table with only this diagonal; their difference is the part off the diagonal, which must have no nonzero entry. Further, the three space entries must be equal, the three extra-time entries equal, and the hidden entry their mean.

```
say("diagonals (x0 ... x7): " + "; ".join(
    f"E({k}): " + " ".join(f"{v:g}" for v in np.diag(tables[k])) for k in (1, 2, 3)))
check(diagonal_ok and list(np.diag(tables[1])) == [3, 4, 4, 4, 33, 2, 2, 2],
      "at a4' = 2H, a4'' = H^2: only diagonal entries, space and extra-time entries "
      "equal, hidden entry their mean; G = diag(3, 4, 4, 4, 33, 2, 2, 2) H^2")
```

The three diagonals are printed in the order $x_0, \ldots, x_7$: $E_{(1)}$: 3, 4, 4, 4, 33, 2, 2, 2; $E_{(2)}$: 684, 548, 548, 548, −1476, 820, 820, 820; $E_{(3)}$: −22392, −14544 (three times), 40248, −30240 (three times) (Section 11.20 recomputes every one of these entries by hand). With the Einstein tensor of Section 11.10 and $a_4' = 2H$, $a_4'' = H^2$: ${G^{x_1}}_{x_1} = -12 + 1 + 15 = 4$, ${G^{x_4}}_{x_4} = 12 + 21 = 33$, ${G^{x_5}}_{x_5} = -12 - 1 + 15 = 2$, ${G^{x_0}}_{x_0} = 15 - 12 = 3$, in units of H^2, which the check confirms. Thirty-fourth PASS line.

What Figure 11b.5 shows. Three 8×8 tables in units of H^2, H^4 and H^6, each filled only on its diagonal. In each table the first three diagonal squares are equal (3-space), the squares 5 to 7 are equal (extra times), and the last square (hidden direction) lies exactly halfway between them: $(4 + 2)/2 = 3$, $(548 + 820)/2 = 684$, $(-14544 - 30240)/2 = -22392$. The time square x_4 stands apart: in $E_{(1)}$ it is much larger than the others (33 against 2 to 4, all positive); in $E_{(2)}$ and $E_{(3)}$ it has the opposite sign to all the others (−1476 against positive entries, +40248 against negative ones). Section 11.21 proves these patterns for every history.

In [24], the quoted records once more.

```
def verdicts(path):
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    checks = data["checks"]
    if isinstance(checks, dict):  # name -> {"passed": true or false, ...}
        return {name: "PASS" if entry["passed"] else "FAIL"
                for name, entry in checks.items()}
    return {entry["name"]: entry["verdict"] for entry in checks}  # a list of entries
```

The helper of Notebook 11a, In [16]: for one report, a dictionary from the name of each check to PASS or FAIL, for both forms in which the reports store their checks.

```
QUOTED = {  # report -> the checks that this notebook quotes from it
    "python-lovelock-report.json": [
        "rust_riemann_agrees", "rust_k1_mixed_components_agree",
        "rust_k2_mixed_components_agree", "rust_k3_mixed_components_agree",
        "rust_L1_agrees", "rust_L2_agrees", "rust_L3_agrees",
        "k1_unpruned_literal_sum_agrees", "k2_unpruned_literal_sum_agrees",
        "k1_equals_minus_4_einstein", "L1_equals_2R",
        "k2_equals_minus_8_gauss_bonnet", "L2_equals_4_gauss_bonnet",
        "L3_equals_8_cubic_lovelock_density", "normalisation_P1_derived_minus_4",
        "normalisation_P2_derived_minus_8", "normalisation_L3_cubic_derived"],
    "lovelock-report.json": [
        "riemann_antisymmetry", "mixed_riemann_free_of_sin_third",
        "k1_trace_identity", "k2_trace_identity", "k3_trace_identity",
        "k4_tensor_vanishes", "k1_brute_force_numeric", "k2_brute_force_numeric"],
}
```

The 17 checks that the notebook's text quotes from the sympy report and the 8 that it quotes from the Rust report.

```
for report_name, names in QUOTED.items():
    found = verdicts(f"{RESULTS}/{report_name}")
    missing = [name for name in names if found.get(name) != "PASS"]
    say(f"{report_name}: {len(names)} quoted checks; not there or not PASS: "
        f"{missing if missing else 'none'}")
    check(not missing,
          f"the {len(names)} checks quoted from {report_name} are there and PASS",
          record=f"{RESULTS}/{report_name}, checks " + ", ".join(names))
```

For each report the list `missing` collects every quoted check that is absent or not PASS; it is printed ("none") and must be empty (`not missing` is true for an empty list). PASS lines thirty-five and thirty-six.

```
totals = {}
for report_name in ("python-lovelock-report.json", "wolfram-gkd-report.json"):
    data = json.loads(repository_file(f"{RESULTS}/{report_name}").read_text(
        encoding="utf-8"))
    totals[report_name] = (data["checkCount"], data["failedCheckCount"],
                           data["verdict"])
    say(f"{report_name}: {data['checkCount']} checks, {data['failedCheckCount']} "
        f"failed, verdict {data['verdict']}")
```

The numbers of checks of the two verification reports are read and printed: 49 checks, 0 failed, verdict SUCCESS for the sympy report, and 29, 0, SUCCESS for the Wolfram report.

```
check(totals == {"python-lovelock-report.json": (49, 0, "SUCCESS"),
                 "wolfram-gkd-report.json": (29, 0, "SUCCESS")},
      "the sympy verification has 49 checks and the Wolfram verification 29, none "
      "failed",
      record=f"{RESULTS}/python-lovelock-report.json and wolfram-gkd-report.json, "
             "checkCount and failedCheckCount")
```

The counts quoted in the notebook's text are fixed by this check, so a later change of either report would be noticed. Thirty-seventh PASS line.

In [25], the end.

```
figure_files = [f"11b_{k}_{name}.png" for k, name in enumerate(
    ["work_of_the_sums", "curvature_of_planes", "riemann_matrix",
     "mixing_terms_cancel", "lovelock_component_tables"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_files),
      "all five figure files of the notebook exist")
all_checks_passed()
```

As in Notebook 11a, In [17]: the five figure files must exist (thirty-eighth PASS line), and the last line is ALL 38 CHECKS PASSED (notebook 11b). The 38 checks are: 2 in In [3], 4 in In [4], 1 each in In [5], In [6] and In [7], 5 in In [8], 1 each in In [9] and In [10], 2 in In [11], 1 in In [12], 1 in In [14], 3 in In [15], 2 in In [16], 1 each in In [17] to In [21], 3 in In [22], 1 in In [23], 3 in In [24] and 1 in In [25].

11.20 The components of the three tensors for the author's metric

The result (PROVED by exact computation). The Rust program of the record computes all 3×64 components of $P_{(1)}, P_{(2)}, P_{(3)}$ exactly and writes them into the record `lovelock-tensors.json`. The Revision's sympy and Wolfram verifications recompute every one of them independently and find the same (checks `rust_k1_mixed_components_agree` to `rust_k3_mixed_components_agree` of the sympy report, and `k1_P_equals_rust_all_64_components` to `k3_P_equals_rust_all_64_components` of the Wolfram report), and so does Notebook 11b (In [15]). Only the eight diagonal components are nonzero, for every order, on the background $b_4 = 1$; for the metric with a changing b_4 the record also has the mixed components $P^{x_4}{}_{x_0}$ and $P^{x_0}{}_{x_4}$, which contain b_4' (Section 11.21). With $E_{(k)} = -P_{(k)}/2^{k+1}$ and the abbreviations a_4', a_4'' for the derivatives with respect to the time x_4, the independent components are

$$
\begin{aligned}
{E_{(1)}}^{x_1}{}_{x_1} &= -3a_4'^2 + a_4'' + 15H^2, & {E_{(1)}}^{x_4}{}_{x_4} &= 3a_4'^2 + 21H^2,\\
{E_{(1)}}^{x_5}{}_{x_5} &= -3a_4'^2 - a_4'' + 15H^2, & {E_{(1)}}^{x_0}{}_{x_0} &= -3a_4'^2 + 15H^2,
\end{aligned}
$$

$$
\begin{aligned}
{E_{(2)}}^{x_1}{}_{x_1} &= 12a_4'^4 - 24a_4'^2a_4'' + 168a_4'^2H^2 - 40a_4''H^2 - 180H^4,\\
{E_{(2)}}^{x_4}{}_{x_4} &= -36a_4'^4 - 120a_4'^2H^2 - 420H^4,\\
{E_{(2)}}^{x_5}{}_{x_5} &= 12a_4'^4 + 24a_4'^2a_4'' + 168a_4'^2H^2 + 40a_4''H^2 - 180H^4,\\
{E_{(2)}}^{x_0}{}_{x_0} &= 12a_4'^4 + 168a_4'^2H^2 - 180H^4,
\end{aligned}
$$

$$
\begin{aligned}
{E_{(3)}}^{x_1}{}_{x_1} &= -72a_4'^6 + 360a_4'^4 a_4'' - 648a_4'^4 H^2 + 432a_4'^2 a_4'' H^2 - 1944a_4'^2 H^4 + 360a_4'' H^4 + 360H^6,\\
{E_{(3)}}^{x_4}{}_{x_4} &= 360a_4'^6 + 648a_4'^4 H^2 + 1080a_4'^2 H^4 + 2520H^6,\\
{E_{(3)}}^{x_5}{}_{x_5} &= -72a_4'^6 - 360a_4'^4 a_4'' - 648a_4'^4 H^2 - 432a_4'^2 a_4'' H^2 - 1944a_4'^2 H^4 - 360a_4'' H^4 + 360H^6,\\
{E_{(3)}}^{x_0}{}_{x_0} &= -72a_4'^6 - 648a_4'^4 H^2 - 1944a_4'^2 H^4 + 360H^6,
\end{aligned}
$$

with ${E^{x_2}}_{x_2} = {E^{x_3}}_{x_3} = {E^{x_1}}_{x_1}$ and ${E^{x_6}}_{x_6} = {E^{x_7}}_{x_7} = {E^{x_5}}_{x_5}$ for every order. These are exactly the components that the record of the field equations of a_4 lists (`Revision/field_equations_a4/a4-equations.json`, key `lovelockTensors`, compared in its sympy report `Revision/field_equations_a4/reports/python-a4-report.json`, check `json_lovelock_components`); Notebook 11c reads them from the Rust record and compares them with this list (In [2] and In [3]). The components contain only H, a_4' and a_4'': no x_0 (no $\cot z$ and no $\sin^{1/3} z$), no e^{a_4} and no higher derivative. The order-1 components are those of the Einstein tensor of Section 11.10, as Section 11.14 proved they must be.

Every component checked by hand. Take the moment $a_4' = 2H$, $a_4'' = H^2$ of a history in which the extra times deflate and the deflation speeds up. Then $a_4'^2 = 4H^2$, $a_4'^4 = 16H^4$ and $a_4'^6 = 64H^6$ (products of two, four and six factors $2H$). We insert these values into the twelve formulas above and add the whole numbers; because the components x_2, x_3 equal the component x_1 and the components x_6, x_7 equal the component x_5 (stated just above), it suffices to do the components x_1, x_4, x_5 and x_0 of each order. Order 1:

$$
\begin{aligned}
{E_{(1)}}^{x_1}{}_{x_1} &= -3\cdot 4H^2 + H^2 + 15H^2 = (-12+1+15)H^2 = 4H^2,\\
{E_{(1)}}^{x_4}{}_{x_4} &= 3\cdot 4H^2 + 21H^2 = (12+21)H^2 = 33H^2,\\
{E_{(1)}}^{x_5}{}_{x_5} &= -3\cdot 4H^2 - H^2 + 15H^2 = (-12-1+15)H^2 = 2H^2,\\
{E_{(1)}}^{x_0}{}_{x_0} &= -3\cdot 4H^2 + 15H^2 = (-12+15)H^2 = 3H^2.
\end{aligned}
$$

In each line the first step inserts $a_4'^2 = 4H^2$ and $a_4'' = H^2$, the second multiplies out ($3\cdot 4 = 12$) and takes the common factor H^2 out of the sum, and the last adds the whole numbers. Order 2: the four products that occur become $a_4'^4 = 16H^4$, $a_4'^2 a_4'' = 4H^2\cdot H^2 = 4H^4$, $a_4'^2 H^2 = 4H^4$ and $a_4'' H^2 = H^4$ (powers of H multiply by adding their exponents), and H^4 stays H^4. In ${E_{(2)}}^{x_1}{}_{x_1}$, written out factor by factor:

$$
12\cdot 16H^4 - 24\cdot 4H^2\cdot H^2 + 168\cdot 4H^2\cdot H^2 - 40H^2\cdot H^2 - 180H^4 = (192-96+672-40-180)H^4 = 548H^4.
$$

The other three components of order 2 in the same steps (insert, multiply the coefficient of each term by 16, 4, 4 or 1 according to its product, the term in H^4 keeping its coefficient, add):

$$
\begin{aligned}
{E_{(2)}}^{x_4}{}_{x_4} &= (-36\cdot 16 - 120\cdot 4 - 420)H^4 = (-576-480-420)H^4 = -1476H^4,\\
{E_{(2)}}^{x_5}{}_{x_5} &= (12\cdot 16 + 24\cdot 4 + 168\cdot 4 + 40 - 180)H^4 = (192+96+672+40-180)H^4 = 820H^4,\\
{E_{(2)}}^{x_0}{}_{x_0} &= (12\cdot 16 + 168\cdot 4 - 180)H^4 = (192+672-180)H^4 = 684H^4.
\end{aligned}
$$

Order 3: the six products that occur become $a_4'^6 = 64H^6$, $a_4'^4 a_4'' = 16H^4 \cdot H^2 = 16H^6$, $a_4'^4 H^2 = 16H^6$, $a_4'^2 a_4'' H^2 = 4H^2 \cdot H^2 \cdot H^2 = 4H^6$, $a_4'^2 H^4 = 4H^6$ and $a_4'' H^4 = H^6$, so the coefficient of each term is multiplied by 64, 16, 16, 4, 4 or 1 according to its product, and the term in H^6 keeps its coefficient:

$$\begin{aligned}
{E_{(3)}}^{x_1}{}_{x_1} &= (-72 \cdot 64 + 360 \cdot 16 - 648 \cdot 16 + 432 \cdot 4 - 1944 \cdot 4 + 360 + 360)H^6 \\
&= (-4608 + 5760 - 10368 + 1728 - 7776 + 360 + 360)H^6 = -14544H^6, \\
{E_{(3)}}^{x_4}{}_{x_4} &= (360 \cdot 64 + 648 \cdot 16 + 1080 \cdot 4 + 2520)H^6 \\
&= (23040 + 10368 + 4320 + 2520)H^6 = 40248H^6, \\
{E_{(3)}}^{x_5}{}_{x_5} &= (-72 \cdot 64 - 360 \cdot 16 - 648 \cdot 16 - 432 \cdot 4 - 1944 \cdot 4 - 360 + 360)H^6 \\
&= (-4608 - 5760 - 10368 - 1728 - 7776 - 360 + 360)H^6 = -30240H^6, \\
{E_{(3)}}^{x_0}{}_{x_0} &= (-72 \cdot 64 - 648 \cdot 16 - 1944 \cdot 4 + 360)H^6 \\
&= (-4608 - 10368 - 7776 + 360)H^6 = -22392H^6.
\end{aligned}$$

For each component the first line inserts the products, the second line multiplies out each coefficient, and its last step adds the whole numbers. With ${E^{x_2}}_{x_2} = {E^{x_3}}_{x_3} = {E^{x_1}}_{x_1}$ and ${E^{x_6}}_{x_6} = {E^{x_7}}_{x_7} = {E^{x_5}}_{x_5}$ the diagonals at this moment are, in the order $x_0, \ldots, x_7$,

$$\begin{aligned}
E_{(1)} &= (3, 4, 4, 4, 33, 2, 2, 2)H^2, \\
E_{(2)} &= (684, 548, 548, 548, -1476, 820, 820, 820)H^4, \\
E_{(3)} &= (-22392, -14544, -14544, -14544, 40248, -30240, -30240, -30240)H^6,
\end{aligned}$$

exactly the numbers that Notebook 11b prints (In [23], Figure 11b.5).

11.21 Their structure: equal directions, constraint, evolution factor, mean, sign and weight

The components have a clear structure, which Chapter 12 uses at every step. We prove each part by hand from the curvature table of Section 11.10 and from the rule GKD; Notebook 11c confirms every part with exact algebra (In [2] for part (a), In [4] for parts (b) to (h)).

Three tools. (i) **Relabelling.** If a bijection τ of the eight labels is applied to every label of both lists of a generalized delta, its matrix does not change, because $\delta(\tau l_i, \tau u_j) = \delta(l_i, u_j)$ (τ maps equal labels to equal labels and different labels to different labels); so $\delta^{\tau U}_{\tau L} = \delta^U_L$. And because τ runs through all labels exactly once, a sum over all values of the summed labels is unchanged when every summed label is replaced by its image. (ii) **The label 4 and the acceleration.** Every curvature entry that contains a_4'' is a plane entry of a plane (k, x_4), so it carries the label x_4 in its upper pair and in its lower pair (Section 11.10). (iii) **The labels 4 and 0 in the mixed entries.** Every mixed entry, such as ${R^{x_4 k}}_{x_0 k}$ or ${R^{x_0 k}}_{x_4 k}$, has the label x_4 in one of its pairs and x_0 in the other; a plane entry has the same two labels in both pairs.

(a) Only diagonal components (PROVED). In a term of ${P_{(k)}}^h{}_j$ the delta's lower list is j followed by the upper pairs of the factors, and its upper list is h followed by their lower pairs. A plane entry adds the same two labels to both lists. A mixed entry of the first kind (x_4 up, x_0 down) adds x_4 to the lower list and x_0 to the upper list; one of the second kind does the opposite. For a nonzero term the two lists must hold the same labels (case c). If $h \neq j$, the labels j and h must

therefore be exactly x_4 and x_0 in some order, balanced by one more mixed entry of one kind than of the other. So every off-diagonal component other than ${P^{x_4}}_{x_0}$ and ${P^{x_0}}_{x_4}$ vanishes identically. Now take ${P^{x_4}}_{x_0}$: the label x_0 stands in the lower list (as j) and must appear once in the upper list, and x_4 stands in the upper list (as h) and must appear once in the lower list; so each term contains exactly one mixed entry, ${R^{x_4 k}}_{x_0 k}$ or one of its antisymmetric variants, and every other factor avoids both x_4 and x_0: it is a plane curvature among the six transverse directions, $a_4'^2 - H^2$ or $-(a_4'^2 + H^2)$. Apply the relabelling τ that exchanges $x_1 \leftrightarrow x_5$, $x_2 \leftrightarrow x_6$, $x_3 \leftrightarrow x_7$ and keeps x_4 and x_0. It leaves every factor among the transverse directions unchanged (the planes (i, i') and (j, j') have the same curvature, and the planes (i, j) go to planes (j, i)), it leaves the delta unchanged (tool i), and it changes the sign of the mixed entry, because ${R^{x_4 i}}_{x_0 i} = +H a_4' \cot z$ and ${R^{x_4 j}}_{x_0 j} = -H a_4' \cot z$ (Section 11.10). Summing over all relabelled terms is the same sum, so ${P^{x_4}}_{x_0} = -{P^{x_4}}_{x_0}$, hence ${P^{x_4}}_{x_0} = 0$; the same for ${P^{x_0}}_{x_4}$ with the second kind of mixed entry, ${R^{x_0 i}}_{x_4 i} = -H a_4' \tan z$ against ${R^{x_0 j}}_{x_4 j} = +H a_4' \tan z$. This is the cancellation of three inflating against three deflating directions of Section 11.10, now for every order. It holds on the background $b_4 = 1$, where the extra-time entries are the space entries with the sign of the mixed ones reversed; a changing b_4 changes only the extra-time entries, the cancellation is incomplete, and for the metric with b_4 the components ${P^{x_4}}_{x_0}$ and ${P^{x_0}}_{x_4}$ are not zero: they are proportional to b_4' (record `lovelock-tensors.json`; Notebook 11c, In [2]; for Einstein's tensor ${G^{x_4}}_{x_0} = \frac{3}{2} H \cot z \, b_4'/b_4$, Section 11.10). The record finds the same eight nonzero components for every order (`wolfram-gkd-report.json`, field `measurements`, entries `k1NonzeroComponents` to `k3NonzeroComponents`).

(b) The three space directions are alike, and so are the three extra times (PROVED). The relabelling that exchanges x_1 and x_2 is a change of coordinates that leaves the author's metric unchanged ($g_{11} = g_{22}$), so it maps every curvature entry to the entry with the exchanged labels, of the same value. By tool (i) it maps the sum ${P^{x_1}}_{x_1}$ term by term onto ${P^{x_2}}_{x_2}$: the two are equal. The same holds for x_3, and for the extra times x_5, x_6, x_7.

(c) a_4'' appears at most once in each term (PROVED). By tool (ii) an entry with a_4'' puts the label x_4 into both lists of the delta. Two such entries in one term would repeat x_4 in both lists, and GKD would be 0. So every component has the form (polynomial in H, a_4')+$a_4''\cdot$(polynomial in H, a_4').

(d) The time component contains no a_4'' (PROVED). In ${P^{x_4}}_{x_4}$ the label x_4 already stands in both lists (as h and j), so no factor may carry x_4: no entry with a_4'' (tool ii) and no mixed entry (tool iii). The factors are plane curvatures among the directions $x_0, x_1, x_2, x_3, x_5, x_6, x_7$, that is $a_4'^2 - H^2$, $-(a_4'^2 + H^2)$ and $-H^2$. So ${E^{x_4}}_{x_4}$ is a polynomial in $a_4'^2$ and H^2 alone, as the list of Section 11.20 shows. In the field equations this component is the **constraint**, a condition on a_4', not an equation for its change.

(e) Space against extra times: the evolution factor (PROVED). Apply the relabelling τ of part (a) to ${P^{x_1}}_{x_1}$: it becomes ${P^{x_5}}_{x_5}$, term by term, with every factor replaced by its image. The image of a plane entry is the same entry with a_4'' replaced by $-a_4''$ (the table of Section 11.10: (i, x_4) with $a_4'^2 + a_4''$ goes to (j, x_4) with $a_4'^2 - a_4''$, and every other kind of plane goes to a plane with the same curvature). The image of a mixed entry has the opposite sign; but mixed entries occur in pairs in every nonzero term of a diagonal component other than ${P^{x_4}}_{x_4}$ and ${P^{x_0}}_{x_0}$ (the label x_0 must appear in both lists or in neither, so a mixed entry of one kind needs one of the other kind), and two sign changes cancel. Hence

$$E^{x_5}{}_{x_5}(H, a_4', a_4'') = E^{x_1}{}_{x_1}(H, a_4', -a_4'').$$

With part (c) write $E^{x_1}{}_{x_1} = \mathcal{A}_k + a_4''\mathcal{B}_k$, where $\mathcal{A}_k$ and $\mathcal{B}_k$ are polynomials in H and a_4'. Then $E^{x_5}{}_{x_5} = \mathcal{A}_k - a_4''\mathcal{B}_k$ (the identity just proved), and the difference is

$$E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5} = 2\mathcal{B}_k\, a_4'' = a_4''\, F_k(a_4'), \qquad F_k = 2\mathcal{B}_k.$$

This difference is the only combination in which a_4'' survives; in the field equations it is the **evolution equation**, which decides how a_4 changes (Chapter 12). From the components of Section 11.20,

$$F_1 = 2, \qquad F_2 = -16(3a_4'^2 + 5H^2), \qquad F_3 = 144(5a_4'^4 + 6a_4'^2H^2 + 5H^4)$$

(for example $E_{(2)}{}^{x_1}{}_{x_1} - E_{(2)}{}^{x_5}{}_{x_5} = -48a_4'^2a_4'' - 80a_4''H^2 = -16a_4''(3a_4'^2 + 5H^2)$: the terms without a_4'' cancel). The record lists $F = \alpha_1F_1 + \alpha_2F_2 + \alpha_3F_3$ (`a4-equations.json`, key `generalSource`, entry `evolution_F`; `python-a4-report.json`, check `evolution_factorises`); Notebook 11c, In [4], computes the F_k and compares them with the record.

(f) The hidden component (PROVED). The same relabelling maps $P^{x_0}{}_{x_0}$ to itself (τ keeps x_0), with a_4'' replaced by $-a_4''$; in its terms no factor carries x_0, so there are no mixed entries. So $E^{x_0}{}_{x_0}$ is unchanged when a_4'' changes sign, and by part (c) it is linear in a_4'': it contains no a_4''. Section 11.22 proves more: $E^{x_0}{}_{x_0} = \mathcal{A}_k$, the mean of the space and the extra-time components.

(g) Even in a_4': deflation is a choice of sign (PROVED). Reverse the time: use the coordinate $y_4 = -x_4$ instead of x_4. Since $dx_4^2 = dy_4^2$, the metric in the new coordinates has exactly the author's form, with the function $b(y_4) = a_4(-y_4)$ in place of a_4. Its Lovelock components are therefore given by the same polynomials, with $b'(y_4) = -a_4'(x_4)$ and $b''(y_4) = a_4''(x_4)$ (the chain rule: each derivative brings a factor $dx_4/dy_4 = -1$). On the other hand a diagonal component of a tensor with one upper and one lower label does not change under this reflection: the component x_4x_4 is multiplied by $(-1)(-1) = 1$, and the others are not touched (the tensor rule of Section 3.14; $P_{(k)}$ is a tensor, built from the Riemann tensor and the generalized delta, whose entries are the same in every system of coordinates). So every diagonal component satisfies $E(H, -a_4', a_4'') = E(H, a_4', a_4'')$ at every point and for every history. Any two numbers u, v are the values $a_4'(0) = u$, $a_4''(0) = v$ of the history $a_4 = ux_4 + vx_4^2/2$, so the identity holds for all values: **every component is an even function of** a_4'. The list of Section 11.20 shows it: a_4' appears only in even powers. The meaning: replacing a_4 by $-a_4$ along a linear history, $A \to -A$, turns deflating extra times and inflating 3-space into inflating extra times and deflating 3-space, and the Lovelock tensors do not notice. The gravitational side of the field equations does not prefer deflation; deflation of the extra times is a choice of sign (Chapter 12 draws the consequences). Notebook 11c checks the evenness exactly (In [4]) and draws it (In [6], Figure 11c.1).

(h) Weight $2k$ (PROVED in Section 11.11). Every term of $E_{(k)}$ has the weight $2k$ when H and a_4' count 1 and a_4'' counts 2: $E_{(1)}$ is quadratic, $E_{(2)}$ quartic, $E_{(3)}$ of sixth degree in this sense. Notebook 11c checks it by replacing H, a_4', a_4'' by uH, ua_4', u^2a_4'' and finding the factor u^{2k} (In [4]).

One more factorisation used in Chapter 12. The difference of the time and the hidden component is

$$E^{x_4}{}_{x_4} - E^{x_0}{}_{x_0} = 6(a_4'^2 + H^2)V_k, \qquad V_1 = 1, \quad V_2 = -8(a_4'^2 + 5H^2), \quad V_3 = 72(a_4'^4 + 2a_4'^2H^2 + 5H^4).$$

For $k = 2$, line by line:

$$E_{(2)}{}^{x_4}{}_{x_4} - E_{(2)}{}^{x_0}{}_{x_0} = -36a_4'^4 - 120a_4'^2H^2 - 420H^4 - 12a_4'^4 - 168a_4'^2H^2 + 180H^4$$
$$= -48a_4'^4 - 288a_4'^2H^2 - 240H^4$$

(the two components of Section 11.20; collect equal powers),

$$= -48(a_4'^4 + 6a_4'^2H^2 + 5H^4) = -48(a_4'^2 + H^2)(a_4'^2 + 5H^2) = 6(a_4'^2 + H^2)\cdot(-8)(a_4'^2 + 5H^2)$$

(take out -48; the quadratic $u^2 + 6u + 5 = (u+1)(u+5)$ in $u = a_4'^2/H^2$; $-48 = 6\cdot(-8)$). Notebook 11c checks all three (In [5]); the record uses the factor for the linear history (`python-a4-report.json`, check `linear_member_vacuum_factor`).

11.22 The two conservation identities

Every Lovelock tensor has zero covariant divergence (Lovelock's theorem; for the author's metric PROVED by exact computation: `Revision/gkd_lovelock/results/lovelock-report.json`, checks `k1_divergence_free` to `k3_divergence_free`). For the author's metric this one statement becomes two short identities between the components. Section 3.25 derived them for the Einstein tensor; here we derive them for every order, line by line.

The divergence of a diagonal tensor. For a tensor $P^h{}_j$ with one upper and one lower label, the covariant divergence is

$$\nabla_h P^h{}_j = \sum_h \partial_h P^h{}_j + \sum_{h,m} \Gamma^h{}_{hm} P^m{}_j - \sum_{h,m} \Gamma^m{}_{hj} P^h{}_m$$

(one $+\Gamma$ for the upper label, one $-\Gamma$ for the lower label, then the upper label contracted with the derivative; Section 3.15). If P is diagonal, the first sum keeps $h = j$, the second keeps $m = j$, and the third keeps $m = h$:

$$\nabla_h P^h{}_j = \partial_j P^j{}_j + \sum_h \Gamma^h{}_{hj}(P^j{}_j - P^h{}_h) \qquad \text{(no sum over } j\text{)}$$

(the second and third sums combined, since both now run over h).

The time direction, $j = x_4$. The symbols $\Gamma^h{}_{hx_4}$ are a_4' for the three space directions, $-a_4'$ for the three extra times, and 0 for x_4 and x_0 (Section 3.23). So

$$\nabla_h P^h{}_{x_4} = \partial_4 P^{x_4}{}_{x_4} + 3a_4'(P^{x_4}{}_{x_4} - P^{x_1}{}_{x_1}) - 3a_4'(P^{x_4}{}_{x_4} - P^{x_5}{}_{x_5}) = \partial_4 P^{x_4}{}_{x_4} - 3a_4'(P^{x_1}{}_{x_1} - P^{x_5}{}_{x_5})$$

(the three space components are equal, and so are the three extra-time components, Section 11.21 (b); the terms $3a_4'P^{x_4}{}_{x_4}$ cancel). Setting the divergence to zero and multiplying by $-1/2^{k+1}$:

$$\frac{d}{dx_4} E^{x_4}{}_{x_4} = 3a_4'(E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5}). \qquad \text{(I)}$$

The hidden direction, $j = x_0$. Nothing depends on x_0, so $\partial_0 P^{x_0}{}_{x_0} = 0$. The symbols $\Gamma^h{}_{hx_0}$ are $H\cot z$ for the six transverse directions and 0 for x_4; the term $h = x_0$ has the factor $P^{x_0}{}_{x_0} - P^{x_0}{}_{x_0} = 0$. So

$$\nabla_h P^h{}_{x_0} = 3H\cot z\big(P^{x_0}{}_{x_0} - P^{x_1}{}_{x_1}\big) + 3H\cot z\big(P^{x_0}{}_{x_0} - P^{x_5}{}_{x_5}\big) = 3H\cot z\big(2P^{x_0}{}_{x_0} - P^{x_1}{}_{x_1} - P^{x_5}{}_{x_5}\big).$$

Since $H\cot z > 0$ on the patch $0 < z < \pi/2$, zero divergence gives

$$E^{x_0}{}_{x_0} = \tfrac{1}{2}\big(E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5}\big). \qquad \text{(II)}$$

The transverse directions. For j one of $x_1, x_2, x_3, x_5, x_6, x_7$: nothing depends on x_j, and every $\Gamma^h{}_{hj}$ is zero (Section 3.23), so the divergence vanishes without any condition.

What the identities say about the components (PROVED). With Section 11.21 (e), identity (II) gives $E^{x_0}{}_{x_0} = \frac{1}{2}\big((\mathcal{A}_k + a_4''\mathcal{B}_k) + (\mathcal{A}_k - a_4''\mathcal{B}_k)\big) = \mathcal{A}_k$: the hidden component is the space component without its a_4'' part. For $k = 1$: $\frac{1}{2}\big((-3a_4'^2 + a_4'' + 15H^2) + (-3a_4'^2 - a_4'' + 15H^2)\big) = -3a_4'^2 + 15H^2$. Identity (I) gives, since $E^{x_4}{}_{x_4}$ depends on x_4 only through a_4' (Section 11.21 (d)) and by the chain rule $\frac{d}{dx_4}E^{x_4}{}_{x_4} = \frac{\partial E^{x_4}{}_{x_4}}{\partial a_4'}\,a_4''$,

$$\frac{\partial E^{x_4}{}_{x_4}}{\partial a_4'}\,a_4'' = 3a_4'\,a_4''\,F_k \qquad \Longrightarrow \qquad \frac{\partial E^{x_4}{}_{x_4}}{\partial a_4'} = 3a_4'\,F_k(a_4')$$

(identity (I) with Section 11.21 (e); both sides are polynomials that agree for every a_4'', so the factors of a_4'' agree). For $k = 2$: $\partial(-36a_4'^4 - 120a_4'^2H^2 - 420H^4)/\partial a_4' = -144a_4'^3 - 240a_4'H^2 = 3a_4' \cdot (-48a_4'^2 - 80H^2) = 3a_4'F_2$. The derivative of the constraint is $3a_4'$ times the evolution factor: this is the **constraint propagation** of the record (`a4-equations.json`, key `generalSource`, entry `constraint_propagation`; `python-a4-report.json`, check `bianchi_x4`). Notebook 11c checks (I) exactly for $k = 1, 2, 3$ (In [5]), and (II) both exactly (In [4]) and numerically along a test history (In [9]); In [10] checks (I) numerically as well, along the same history.

What they mean in the field equations (a preview of Chapter 12). In $\sum_k \alpha_k E_{(k)} + \Lambda\delta = \kappa T$, with $T = \mathrm{diag}(p_0, p_3, p_3, p_3, -\rho, p_t, p_t, p_t)$ (in the order $x_0, \ldots, x_7$), identity (I) becomes the conservation of energy $\rho' = -3a_4'(p_3 - p_t)$ (energy flows between 3-space and the extra times when their pressures differ), and identity (II) becomes the condition $p_3 + p_t = 2p_0$ on the pressures (record `a4-equations.json`, key `generalSource`, entries `conservation_reduced` and `algebraic_condition`; Notebook 11c, In [11], checks that the record states both).

11.23 Along the deflating history; an illustrative test history

The linear history (PROVED). Along $a_4 = AHx_4$ we have $a_4' = AH$ and $a_4'' = 0$; the extra times shrink like e^{-AHx_4} (exponential deflation for $A > 0$; the author's history is $A = 1$). With $a_4'' = 0$ the space, extra-time and hidden components coincide (Section 11.21 (e) and identity (II)), and each order has only two different components. In units of H^{2k}:

order	time component $E^{x_4}{}_{x_4}$	the seven others
$k=1$	$3A^2+21$	$15-3A^2$
$k=2$	$-36A^4-120A^2-420$	$12A^4+168A^2-180$
$k=3$	$360A^6+648A^4+1080A^2+2520$	$-72A^6-648A^4-1944A^2+360$

(the components of Section 11.20 with $a_4'=AH$, $a_4''=0$). Every entry is even in A: the history A and its mirror $-A$ give the same tensors (Section 11.21 (g)). Figure 11c.1 draws the six curves for $-3\le A\le 3$ (Notebook 11c, In [6]).

The Lovelock scalars along the linear history. From the record (keys `L1` to `L3` of `lovelock-tensors.json`), in units of H^{2k}:

$$L_{(1)}=12A^2-84,\qquad L_{(2)}=-96A^4-2112A^2+3360,$$
$$L_{(3)}=1152A^6+31104A^4+100224A^2-40320.$$

At the mirror point $A=0$ they are -84, 3360 and -40320. Each has exactly one positive zero. For $k=1$: $12A^2=84$, so $A^2=7$ and $A=\sqrt{7}\approx 2.646$ (divide by 12; take the positive root). For $k=2$, write $u=A^2$:

$$-96u^2-2112u+3360=0 \quad\Longleftrightarrow\quad u^2+22u-35=0$$

(divide by -96: $2112/96=22$ and $3360/96=35$),

$$u=-11+\sqrt{121+35}=-11+\sqrt{156}=2\sqrt{39}-11\approx 1.49000$$

(the formula for the roots of a quadratic, $u=-11\pm\sqrt{11^2+35}$, with the positive root, because $u=A^2\ge 0$; $\sqrt{156}=2\sqrt{39}$), so $A=\sqrt{2\sqrt{39}-11}\approx 1.2207$. For $k=3$, $u^3+27u^2+87u-35=0$ (divide by 1152) has one positive root, which Notebook 11c computes numerically, $A\approx 0.6010$ (In [7], COMPUTED with 15 digits). Figure 11c.2 draws the three scalars with their zeros.

An illustrative test history (ILLUSTRATION, not a solution of any equation). On the linear history $a_4''=0$, so it cannot show where a_4'' enters. Notebook 11c therefore also uses a test history in which the extra times deflate at all times and the deflation speeds up:

$$a_4(x_4)=Hx_4+\tfrac{1}{2}\ln\cosh(Hx_4).$$

Its derivatives, line by line:

$$a_4'=H+\tfrac{1}{2}\cdot\frac{\sinh(Hx_4)}{\cosh(Hx_4)}\cdot H=H\bigl(1+\tfrac{1}{2}\tanh(Hx_4)\bigr)$$

(the derivative of $\ln u$ is u'/u, the derivative of $\cosh t$ is $\sinh t$, and the chain rule gives the factor H; $\sinh/\cosh=\tanh$), and

$$a_4'' = \tfrac{1}{2}H \cdot \frac{H}{\cosh^2(Hx_4)} = \frac{H^2}{2}\operatorname{sech}^2(Hx_4)$$

(the derivative of $\tanh t$ is $1/\cosh^2 t = \operatorname{sech}^2 t$, and the chain rule again). Since $-1 < \tanh < 1$, a_4' lies between $H/2$ and $3H/2$: it is always positive, so the extra times always deflate, slowly in the far past ($a_4' \to H/2$, the linear history with $A = 1/2$) and three times faster in the far future ($a_4' \to 3H/2$, $A = 3/2$). The acceleration a_4'' is a bump of height $H^2/2$ at $x_4 = 0$ ($\operatorname{sech} 0 = 1$). Figure 11c.3 draws the history (Notebook 11c, In [8]). Along it, the components behave as Section 11.21 says (Figure 11c.4, In [9]): the time component depends on a_4' only and moves from one constant to another, for $k = 1$ from $3/4 + 21 = 21.75H^2$ to $27/4 + 21 = 27.75H^2$; the space and extra-time components separate while $a_4'' \neq 0$, by $a_4''F_k$; and the hidden component runs exactly halfway between them. Finally Notebook 11c checks identity (I) along this history with a finite-difference derivative, $f'(t) \approx (f(t+\epsilon) - f(t-\epsilon))/(2\epsilon)$ with $\epsilon = 0.01$, whose error is of the size $\epsilon^2/6$ times the third derivative of the time component; the largest relative deviations are 3.8×10^{-5}, 4.0×10^{-5} and 4.5×10^{-5} for $k = 1, 2, 3$ (COMPUTED, In [10], Figure 11c.5), the size of this error, while the exact identity holds without any error (In [5]).

What the deflation does and does not decide (honesty). The Lovelock tensors are the gravitational side of the field equations. They are the same for A and $-A$: whether the extra times deflate is not decided by them but by the source and the initial data (Chapter 12). This symmetry $A \to -A$ is a property of the metric's geometry; it is NOT the pairing of universes of masses $+m$ and $-m$ of Chapters 18 to 20, and nothing in this chapter says anything about the creation of universes or about matter and antimatter.

11.24 Example: the components and identities along the deflating history

Notebook 11c reads the exact components of the three Lovelock tensors from the Revision record `lovelock-tensors.json`, for the author's metric with a general second function b_4, forms the normalised tensors $E_{(k)}$ and checks that with b_4 they are the components of the Revision's sympy verification (including the two mixed components x_4-x_0 and x_0-x_4, proportional to b_4') and that on the background $b_4 = 1$ they are the components recorded for the field equations of a_4 (`a4-equations.json`); proves with sympy the structure of Section 11.21 (space and extra-time components alike, the constraint free of a_4'', the evolution factors F_k, the hidden component as the mean, evenness in a_4', weight $2k$) and the identities of Section 11.22; draws the components and the scalars along the linear history as functions of A, with the zeros of the scalars; and uses the illustrative test history to show where a_4'' enters and to check identity (I) numerically. It quotes the reports of the field equations of a_4 only by the names of their checks, never by their numbers of checks, so that it stays valid when these reports gain checks. It needs no Rust, takes a few seconds (4.7 seconds when it was built on 2026-10-15; provenance file `Revision/textbook/notebooks/11c_lovelock_components.PROVENANCE.md`), prints 20 PASS lines, draws five figures and ends with the line ALL 20 CHECKS PASSED (notebook 11c).

11.25 How to run Notebook 11c

Step 1. What this notebook does and what it needs.

Notebook 11c (The Lovelock tensors along the deflating history: components and identities) is the file `Revision/textbook/notebooks/11c_lovelock_components.ipynb` of the repository Dirac_claude. It reads the exact components of the three Lovelock tensors of the author's metric from the Revision record, writes the normalised tensors E(1) (the Einstein tensor), E(2) and E(3) as polynomials in H and in the first and second time derivatives of a4, reproduces the components recorded for the field equations of a4, proves with sympy their structure (equal components within ordinary space and within the extra times, a time component without second derivatives, a space minus extra-time difference proportional to the second derivative, a hidden component equal to the mean of the two, evenness in the first derivative, the order of each term) and the two conservation identities, and plots every component and the three Lovelock scalars along the exponentially deflating linear history a4 = A H x4 and along an illustrative test history; it checks again that every check and statement it quotes from the Revision records is there, every check with the verdict PASS; it draws five figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 11c_lovelock_components.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 11c_lovelock_components.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 11c_lovelock_components.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/11c.captions.json`
- `Revision/textbook/figures/11c_1_components_linear_history.png`
- `Revision/textbook/figures/11c_2_lovelock_scalars.png`
- `Revision/textbook/figures/11c_3_test_history.png`
- `Revision/textbook/figures/11c_4_components_test_history.png`
- `Revision/textbook/figures/11c_5_conservation_identity.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files of the notebook exist
ALL 20 CHECKS PASSED (notebook 11c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/11c_lovelock_components.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 11c_lovelock_components.ipynb
python -m nbconvert --execute --inplace 11c_lovelock_components.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

11.26 Notebook 11c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/11c_lovelock_components.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the

cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Notebook 11b computed the three Lovelock tensors $P_{(1)}, P_{(2)}, P_{(3)}$ of the
author's metric and found that only their eight diagonal components are not zero.
This notebook studies those components: what they look like, which structure they
have, and how they behave as the extra times deflate. It

- reads the exact components from the Revision record `lovelock-tensors.json` and
  forms the normalised tensors $E_{(k)} = -P_{(k)}/2^{k+1}$ ($E_{(1)}$ is Einstein's
  tensor $G$), and checks that they are the components recorded for the field
  equations of $a_4$ in `Revision/field_equations_a4/a4-equations.json`;
- proves with sympy their structure: the three space components are equal, the three
  extra-time components are equal, the time component contains no $a_4''$, the
  difference of the space and the extra-time component is $a_4''$ times a
  polynomial, the hidden component is the mean of the two, every component is
  unchanged when $a_4'$ changes sign, and every term of order $k$ is $2k$ powers of
  an inverse length;
- derives and checks the two conservation identities (the Bianchi identities) that
  every Lovelock tensor obeys in this metric;
- draws the components and the Lovelock scalars along the exponentially deflating
  history $a_4 = A H x_4$, as functions of $A$; and along an illustrative test
  history in which the deflation speeds up, to show where $a_4''$ enters, with a
  numerical check of the conservation identity along it;
- reads the Revision records once more and checks that every check and statement
  it quotes from them is there, every quoted check with the verdict PASS.

It draws five figures and takes about 15 seconds.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 11c (The Lovelock tensors along the deflating history: components and
identities) is the file `Revision/textbook/notebooks/11c_lovelock_components.ipynb` of
the repository Dirac_claude. It reads the exact components of the three Lovelock tensors
of the author's metric from the Revision record, writes the normalised tensors E(1) (the
Einstein tensor), E(2) and E(3) as polynomials in H and in the first and second time
derivatives of a4, reproduces the components recorded for the field equations of a4,
proves with sympy their structure (equal components within ordinary space and within the
extra times, a time component without second derivatives, a space minus extra-time
difference proportional to the second derivative, a hidden component equal to the mean of
the two, evenness in the first derivative, the order of each term) and the two
conservation identities, and plots every component and the three Lovelock scalars along
the exponentially deflating linear history a4 = A H x4 and along an illustrative test
history; it checks again that every check and statement it quotes from the Revision
records is there, every check with the verdict PASS; it draws five figures. It needs a
```

computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
```

```
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

**Step 4. In every new terminal: activate the environment and go into the repository

folder.**

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 11c_lovelock_components.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 11c_lovelock_components.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 11c_lovelock_components.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/11c.captions.json‘
- ‘Revision/textbook/figures/11c_1_components_linear_history.png‘
- ‘Revision/textbook/figures/11c_2_lovelock_scalars.png‘
- ‘Revision/textbook/figures/11c_3_test_history.png‘
- ‘Revision/textbook/figures/11c_4_components_test_history.png‘
- ‘Revision/textbook/figures/11c_5_conservation_identity.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files of the notebook exist
ALL 20 CHECKS PASSED (notebook 11c)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/11c_lovelock_components.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs

  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 11c_lovelock_components.ipynb
      python -m nbconvert --execute --inplace 11c_lovelock_components.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 11c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 11c (The Lovelock tensors along the deflating history: components and
# identities) is the file Revision/textbook/notebooks/11c_lovelock_components.ipynb of
# the repository Dirac_claude. It reads the exact components of the three Lovelock
# tensors of the author’s metric from the Revision record, writes the normalised tensors
# E(1) (the Einstein tensor), E(2) and E(3) as polynomials in H and in the first and
# second time derivatives of a4, reproduces the components recorded for the field
# equations of a4, proves with sympy their structure (equal components within ordinary
# space and within the extra times, a time component without second derivatives, a space
# minus extra-time difference proportional to the second derivative, a hidden component
# equal to the mean of the two, evenness in the first derivative, the order of each term)
# and the two conservation identities, and plots every component and the three Lovelock
# scalars along the exponentially deflating linear history a4 = A H x4 and along an
# illustrative test history; it checks again that every check and statement it quotes
# from the Revision records is there, every check with the verdict PASS; it draws five
# figures. It needs a computer with Windows 11, macOS or Linux, an internet connection
# for the installation, the program Git, and Python 3.12 or newer (the notebooks were
# built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6,
# sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
# nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports
# numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and
# runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
```

```
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
```

```
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
```

```
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 11c_lovelock_components.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 11c_lovelock_components.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 11c_lovelock_components.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/11c.captions.json
# - Revision/textbook/figures/11c_1_components_linear_history.png
# - Revision/textbook/figures/11c_2_lovelock_scalars.png
# - Revision/textbook/figures/11c_3_test_history.png
# - Revision/textbook/figures/11c_4_components_test_history.png
# - Revision/textbook/figures/11c_5_conservation_identity.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
```

```
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all five figure files of the notebook exist
#     ALL 20 CHECKS PASSED (notebook 11c)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/11c_lovelock_components.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 11c_lovelock_components.ipynb
#     python -m nbconvert --execute --inplace 11c_lovelock_components.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
```

```
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "11c"  # this notebook: chapter 11, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
```

```
                    "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))
```

```
def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 11c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$: $x_1, x_2, x_3$ ordinary space, $x_4$ the time,
  $x_5, x_6, x_7$ the three extra times, $x_0$ the hidden direction.
- **History** $a_4(x_4)$: the function of the time in the author's metric. The
  scale factor of ordinary space is $e^{a_4}$, that of the extra times $e^{-a_4}$
  (times the same factor $\sin^{1/6} z$ of the hidden direction). If $a_4$ grows,
  space **inflates** and the extra times **deflate**.
- **Linear history** $a_4 = A H x_4$ with a constant number $A$: then
  $a_4' = AH$ and $a_4'' = 0$, and the extra times shrink like $e^{-AHx_4}$:
  exponential deflation for $A > 0$.
- $a_4'$, $a_4''$: the first and second derivatives of $a_4$ with respect to the time
  $x_4$. In code they are the symbols `A1` and `A2`.
- **Component** $E^h{}_j$: the number in row $h$ and column $j$ of a tensor;
  $E^{x_1}{}_{x_1}$ is written $E^{x1}_{x1}$ in printed text.
- **Lovelock tensor** $P_{(k)}$, **normalised Lovelock tensor**
  $E_{(k)} = -P_{(k)}/2^{k+1}$, **Lovelock scalar** $L_{(k)}$: built from the
  curvature with the generalized Kronecker delta (Notebooks 11a and 11b).
- **Covariant divergence** $\nabla_h E^h{}_j$: the derivative of a tensor that takes
  the curving of the coordinates into account (through the Christoffel symbols).
  **Bianchi identity**: the statement that it is zero for every Lovelock tensor.
- **Even function**: $f(-u) = f(u)$. **Polynomial in** $u$: a sum of numbers times
  powers $u^n$.
- **Finite difference**: the approximation $f'(t) \approx (f(t + \epsilon) -
  f(t - \epsilon))/(2\epsilon)$ of a derivative.
- **Illustration**: a choice made only to show how a formula behaves; it is not a
  result and not a solution of any equation.
```

Text cell:

```
## 4. The physical and mathematical situation

**The tensors.** For the author's metric (Notebook 11b) the Lovelock tensors
$P_{(k)}{}^h{}_j$, $k = 1, 2, 3$, are diagonal, contain only $H$, $a_4'$ and
$a_4''$, and do not depend on $x_0$. The field equations of $a_4$ (chapter 12) use
the normalised tensors
```

```
$$E_{(k)}{}^h{}_j = -\frac{1}{2^{k+1}}\, P_{(k)}{}^h{}_j ,\qquad
  \sum_{k=1}^{3} \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda\,\delta^\mu_\nu
  = \kappa\, T^\mu{}_\nu ,$$

with $E_{(1)} = G$, Einstein's tensor. The independent components are
$E^{x1}_{x1}$ (equal to $E^{x2}_{x2}$ and $E^{x3}_{x3}$: ordinary space),
$E^{x4}_{x4}$ (the time), $E^{x5}_{x5}$ (equal to $E^{x6}_{x6}$ and $E^{x7}_{x7}$:
the extra times) and $E^{x0}_{x0}$ (the hidden direction).

**The two conservation identities, derived line by line.** Every Lovelock tensor has
zero covariant divergence (Lovelock's theorem; the Revision checked it exactly). For
a diagonal tensor that depends only on $x_4$ the divergence is

$$\nabla_h P^h{}_j = \partial_h P^h{}_j + \sum_{h,m} \Gamma^h{}_{hm} P^m{}_j
  - \sum_{h,m} \Gamma^m{}_{hj} P^h{}_m .$$

Take $j = x_4$. The first term is $\partial_4 P^4{}_4$ (only $h = x_4$ survives,
because $P^h{}_4 = 0$ for $h \ne x_4$). In the second term only $m = x_4$ survives:
$\sum_h \Gamma^h{}_{h4} P^4{}_4$. In the third, $P^h{}_m$ is diagonal, so $m = h$:
$\sum_h \Gamma^h{}_{h4} P^h{}_h$. Hence

$$\nabla_h P^h{}_4 = \partial_4 P^4{}_4 + \sum_h \Gamma^h{}_{h4}
  \big(P^4{}_4 - P^h{}_h\big).$$

The Christoffel symbols of the metric (Notebook 11b) are $\Gamma^h{}_{h4} = a_4'$ for
the three space directions, $-a_4'$ for the three extra times, and 0 for $x_4$ and
$x_0$. So the sum is $3a_4'(P^4{}_4 - P^1{}_1) - 3a_4'(P^4{}_4 - P^5{}_5) =
-3a_4'(P^1{}_1 - P^5{}_5)$, and the identity reads

$$\frac{d}{dx_4} E^{x4}{}_{x4} = 3\, a_4'\, \big(E^{x1}{}_{x1} - E^{x5}{}_{x5}\big).
  \qquad (\text{I})$$

Take $j = x_0$. Nothing depends on $x_0$, and the same steps give
$\sum_h \Gamma^h{}_{h8}(P^8{}_8 - P^h{}_h)$ with $\Gamma^h{}_{h8} = H \cot z$ for the
three space and the three extra-time directions, 0 for $x_4$, and the term $h = x_0$
is zero because $P^8{}_8 - P^8{}_8 = 0$. So $3H\cot z\,(2P^8{}_8 - P^1{}_1 -
P^5{}_5) = 0$, that is

$$E^{x0}{}_{x0} = \tfrac12 \big(E^{x1}{}_{x1} + E^{x5}{}_{x5}\big).
  \qquad (\text{II})$$

The divergences along $x_1, x_2, x_3, x_5, x_6, x_7$ vanish because nothing depends
on these coordinates and the tensors are diagonal. In the field equations (I) becomes
the conservation of energy, $\rho' = -3a_4'(p_3 - p_t)$, and (II) the condition
$p_3 + p_t = 2p_0$ on the pressures (chapter 12), on the background $b_4 = 1$
of the author's metric of 2026-10-14 (with a changing second function $b_4(x_4)$ the
tensors also have the mixed components $x_4$-$x_0$ and $x_0$-$x_4$, proportional
to $b_4'$; section 5).

**Status.** The components are PROVED by exact computation (Revision records); the
structure and the identities are PROVED here with sympy; the test history of
section 9 is an ILLUSTRATION, not a solution of the field equations.
```

Text cell:

5. The components, from the Revision record

The next cell reads ‘lovelock-tensors.json‘, which lists every component of $P_{(k)}$, for the author’s metric with a general second function $b_4(x_4)$, as exact monomials ‘[numerator, denominator, [e_H, e_a4’, e_a4’’, e_a4’’’, e_a4’’’’, e_E, e_S, e_C, e_Q, e_b4’, e_b4’’, e_b4’’’, e_b4’’’’]]‘ (the exponents of H, of the four derivatives of a_4, of e^{a_4}, of $\sin^{1/3} z$, of $\cot z$, of $Q = b_4^{1/2}$ and of the four derivatives of b_4). It rebuilds each component as a sympy expression in ‘H‘, ‘A1‘ ($a_4’$), ‘A2‘ ($a_4’’$), ‘c‘ ($\cot z$) and ‘B0‘, ‘B1‘, ‘B2‘ (b_4, $b_4’$, $b_4’’$); if any other exponent were not zero the cell would stop, so the components are free of e^{a_4}, of $\sin^{1/3} z$ and of higher derivatives. With b_4 exactly two components off the diagonal are not zero, x_4-x_0 and x_0-x_4, and both contain $b_4’$. From here on the notebook works on the background $b_4 = 1$ ($b_4’ = b_4’’ = 0$) of the Revision records: ‘P‘ holds the components there, ‘P_B4‘ those with a general b_4. It forms $E_{(k)} = -P_{(k)}/2^{k+1}$ and prints the four independent components of each order at $b_4 = 1$.

In [2]:

```
import re  # regular expressions: rewrites the record's names

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra

H, A1, A2 = sp.symbols("H A1 A2")  # H, a4' and a4''
B0, B1, B2, c = sp.symbols("B0 B1 B2 c")  # b4, b4', b4'' and cot z
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
tensors = json.loads(repository_file(
    "Revision/gkd_lovelock/results/lovelock-tensors.json").read_text(encoding="utf-8"))


def from_monomials(monomials):
    """A component of lovelock-tensors.json as a sympy expression in H, A1, A2, c,
    B0, B1 and B2."""
    total = sp.Integer(0)
    for numerator, denominator, e in monomials:
        if e[3] or e[4] or e[5] or e[6] or e[11] or e[12]:
            raise ValueError("the component contains a3, a4 derivatives, e^a4, "
                             "sin^(1/3) z or higher derivatives of b4")
        monomial = (H ** e[0] * A1 ** e[1] * A2 ** e[2] * c ** e[7]
                    * B0 ** sp.Rational(e[8], 2) * B1 ** e[9] * B2 ** e[10])
        total += sp.Rational(numerator, denominator) * monomial
    return total


P_B4 = {k: {(h, j): from_monomials(tensors[f"P{k}_mixed_up_h_down_j"][f"{a},{b}"]
                                   ["monomials"])
            for h, a in enumerate(NAMES) for j, b in enumerate(NAMES)}
        for k in (1, 2, 3)}
BACKGROUND = {B0: 1, B1: 0, B2: 0}  # the background b4 = 1 of the records
P = {k: {hj: sp.expand(value.subs(BACKGROUND)) for hj, value in P_B4[k].items()}
     for k in (1, 2, 3)}
```

```
mixed_b4 = all({hj for hj, value in P_B4[k].items() if hj[0] != hj[1] and value != 0}
               == {(4, 0), (0, 4)} and P_B4[k][4, 0].has(B1) and P_B4[k][0, 4].has(B1)
               for k in (1, 2, 3))
check(mixed_b4 and all(not P[k][hj].has(c) for k in (1, 2, 3) for hj in P[k]),
      "with b4 the only nonzero off-diagonal components are x4-x0 and x0-x4, both "
      "with b4'; at b4 = 1 no component contains cot z")
E = {k: {hj: sp.expand(-value / 2 ** (k + 1)) for hj, value in P[k].items()}
     for k in (1, 2, 3)}
for k in (1, 2, 3):
    for h in (1, 4, 5, 0):  # x1, x4, x5, x0
        say(f"E({k})^{NAMES[h]}_{NAMES[h]} = {E[k][h, h]}")
check(all(E[k][h, j] == 0 for k in (1, 2, 3) for h in range(8) for j in range(8)
          if h != j),
      "at b4 = 1 every off-diagonal component of E(1), E(2), E(3) is zero, x4-x0 "
      "included")
```

Out [2]:

```
PASS with b4 the only nonzero off-diagonal components are x4-x0 and x0-x4, both with b4';
    at b4 = 1 no component contains cot z
E(1)^x1_x1 = -3*A1**2 + A2 + 15*H**2
E(1)^x4_x4 = 3*A1**2 + 21*H**2
E(1)^x5_x5 = -3*A1**2 - A2 + 15*H**2
E(1)^x0_x0 = -3*A1**2 + 15*H**2
E(2)^x1_x1 = 12*A1**4 - 24*A1**2*A2 + 168*A1**2*H**2 - 40*A2*H**2 - 180*H**4
E(2)^x4_x4 = -36*A1**4 - 120*A1**2*H**2 - 420*H**4
E(2)^x5_x5 = 12*A1**4 + 24*A1**2*A2 + 168*A1**2*H**2 + 40*A2*H**2 - 180*H**4
E(2)^x0_x0 = 12*A1**4 + 168*A1**2*H**2 - 180*H**4
E(3)^x1_x1 = -72*A1**6 + 360*A1**4*A2 - 648*A1**4*H**2 + 432*A1**2*A2*H**2 -
    1944*A1**2*H**4 + 360*A2*H**4 + 360*H**6
E(3)^x4_x4 = 360*A1**6 + 648*A1**4*H**2 + 1080*A1**2*H**4 + 2520*H**6
E(3)^x5_x5 = -72*A1**6 - 360*A1**4*A2 - 648*A1**4*H**2 - 432*A1**2*A2*H**2 -
    1944*A1**2*H**4 - 360*A2*H**4 + 360*H**6
E(3)^x0_x0 = -72*A1**6 - 648*A1**4*H**2 - 1944*A1**2*H**4 + 360*H**6
PASS at b4 = 1 every off-diagonal component of E(1), E(2), E(3) is zero, x4-x0 included
```

Text cell:

```
The next cell compares these components with two other Revision records. The record
‘a4-equations.json‘ of the field equations of $a_4$ (written by the Revision’s
Wolfram derivation and checked by its sympy checker) lists $E_{(1)}, E_{(2)},
E_{(3)}$ in the entry ‘lovelockTensors‘, in Wolfram’s input notation with ‘ad1‘ for
$a_4’$ and ‘ad2‘ for $a_4’’$. The record ‘python-lovelock-report.json‘ lists the
nonzero components of $P_{(k)}$ in its entry ‘independentResults‘. Both texts are
turned into sympy expressions with ‘sp.sympify‘ (‘^‘ is first replaced by ‘**‘).
```

In [3]:

```
a4_record = json.loads(repository_file(
    "Revision/field_equations_a4/a4-equations.json").read_text(encoding="utf-8"))
WOLFRAM_NAMES = {"ad1": A1, "ad2": A2, "H": H}
```

```
def from_text(text):
    """A formula of the records as a sympy expression in H, A1, A2."""
    return sp.sympify(text.replace("^", "**"), locals=WOLFRAM_NAMES)


same = True
for k in (1, 2, 3):
    listed = a4_record["lovelockTensors"][f"E{k}"]
    for key, (h, j) in (("x1x1", (1, 1)), ("x4x4", (4, 4)), ("x5x5", (5, 5)),
                        ("x0x0", (0, 0)), ("x4x0", (4, 0)), ("x0x4", (0, 4))):
        same = same and sp.expand(from_text(listed[key]["input"]) - E[k][h, j]) == 0
check(same, "E(1), E(2), E(3) equal the components recorded for the a4 equations",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "json_lovelock_components")
checker = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json").read_text(
        encoding="utf-8"))
independent = checker["independentResults"]
same = all(
    sp.simplify(sp.sympify(text, locals={"A1": A1, "A2": A2, "H": H, "B0": B0,
                                         "B1": B1, "B2": B2, "c": c})
                - P_B4[k][NAMES.index(key.split(",")[0]),
                          NAMES.index(key.split(",")[1])]) == 0
    for k in (1, 2, 3) for key, text in independent[f"P{k}_mixed_nonzero"].items())
check(same and all(len(independent[f"P{k}_mixed_nonzero"]) == 10 for k in (1, 2, 3)),
      "P(1), P(2), P(3) equal the independent sympy results with b4 (10 nonzero "
      "components each)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "independentResults")
```

Out [3]:

```
PASS E(1), E(2), E(3) equal the components recorded for the a4 equations
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_lovelock_components
PASS P(1), P(2), P(3) equal the independent sympy results with b4 (10 nonzero components
    each)
     reproduces Revision/gkd_lovelock/results/python-lovelock-report.json,
    independentResults
```

Text cell:

```
## 6. The structure of the components

The next cell proves, for $k = 1, 2, 3$, with exact sympy algebra:

1. $E^{x1}_{x1} = E^{x2}_{x2} = E^{x3}_{x3}$ and $E^{x5}_{x5} = E^{x6}_{x6} =
   E^{x7}_{x7}$: the three space directions are alike, and so are the three extra
   times.
2. $E^{x4}_{x4}$ and $E^{x0}_{x0}$ contain no $a_4''$ (‘expr.has(A2)‘ is False): the
```

```
   time component of the field equations is a condition on $a_4'$ only (a
   constraint), not an equation for its change.
3. $E^{x1}_{x1} - E^{x5}_{x5} = a_4''\, F_k(a_4')$ with a polynomial $F_k$ that
   contains no $a_4''$ (`sp.cancel` divides exactly, and the quotient must be free
   of `A2`): this difference is the only place where $a_4''$ appears, so it is the
   equation that decides how $a_4$ changes (the evolution equation of chapter 12).
   The record `a4-equations.json` lists
   $F = \alpha_1 F_1 + \alpha_2 F_2 + \alpha_3 F_3$ in its entry `evolution_F`; the
   cell compares the coefficient of each $\alpha_k$.
4. $E^{x0}_{x0} = \tfrac12 (E^{x1}_{x1} + E^{x5}_{x5})$: identity (II).
5. Every component is even in $a_4'$: replacing $a_4'$ by $-a_4'$ changes nothing.
   Inflating ordinary space with deflating extra times ($a_4' > 0$) and the reverse
   ($a_4' < 0$) give the same components: deflation is a choice of sign that these
   tensors do not select.
6. Every term of $E_{(k)}$ has the **weight** $2k$, where $H$ and $a_4'$ count 1 and
   $a_4''$ counts 2 (each is one inverse length per count): order $k$ has the
   dimension $1/\text{length}^{2k}$. So the three orders can be added in one field
   equation only with constants $\alpha_k$ of different dimensions, for example
   $\alpha_k = w_k / H^{2k - 2}$ with pure numbers $w_k$.
```

In [4]:

```
u = sp.Symbol("u")  # an auxiliary number used to test the weights
F = {}  # k -> F_k(a4')
alphas = sp.symbols("alpha1 alpha2 alpha3")  # the constants of the a4 equations
recorded_F = sp.sympify(
    a4_record["generalSource"]["evolution_F"]["input"].replace("^", "**"),
    locals={"ad1": A1, "H": H, "alpha1": alphas[0], "alpha2": alphas[1],
            "alpha3": alphas[2]})
structure = {name: True for name in ("alike", "constraint", "evolution", "mean",
                                     "even", "weight")}
for k in (1, 2, 3):
    e = E[k]
    structure["alike"] &= (e[1, 1] == e[2, 2] == e[3, 3]
                           and e[5, 5] == e[6, 6] == e[7, 7])
    structure["constraint"] &= not e[4, 4].has(A2) and not e[0, 0].has(A2)
    F[k] = sp.factor(sp.cancel((e[1, 1] - e[5, 5]) / A2))
    structure["evolution"] &= (not F[k].has(A2)
                               and sp.expand(F[k] - recorded_F.coeff(alphas[k - 1]))
                               == 0)
    structure["mean"] &= sp.expand(e[0, 0] - (e[1, 1] + e[5, 5]) / 2) == 0
    structure["even"] &= all(sp.expand(e[h, h].subs(A1, -A1) - e[h, h]) == 0
                             for h in range(8))
    weighted = {h: sp.expand(e[h, h].subs({H: u * H, A1: u * A1, A2: u ** 2 * A2}))
                for h in range(8)}
    structure["weight"] &= all(sp.expand(weighted[h] - u ** (2 * k) * e[h, h]) == 0
                               for h in range(8))
    report(f"F_{k}(a4') = (E({k})^x1_x1 - E({k})^x5_x5) / a4''", F[k])
check(structure["alike"], "space components equal, extra-time components equal")
check(structure["constraint"], "the x4 and x0 components contain no a4''")
check(structure["evolution"],
      "E^x1_x1 - E^x5_x5 = a4'' F_k(a4') with F_1 = 2, F_2 = -16 (3 a4'^2 + 5 H^2), "
      "F_3 = 144 (5 a4'^4 + 6 a4'^2 H^2 + 5 H^4)",
```

```
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "evolution_factorises (a4-equations.json, evolution_F)")
check(structure["mean"], "E^x0_x0 is the mean of E^x1_x1 and E^x5_x5 (identity II)")
check(structure["even"] and structure["weight"],
      "every component is even in a4' and of weight 2k (dimension 1/length^(2k))")
```

Out [4]:

```
RESULT F_1(a4') = (E(1)^x1_x1 - E(1)^x5_x5) / a4'' = 2
RESULT F_2(a4') = (E(2)^x1_x1 - E(2)^x5_x5) / a4'' = -16*(3*A1**2 + 5*H**2)
RESULT F_3(a4') = (E(3)^x1_x1 - E(3)^x5_x5) / a4'' = 144*(5*A1**4 + 6*A1**2*H**2 +
    5*H**4)
PASS space components equal, extra-time components equal
PASS the x4 and x0 components contain no a4''
PASS E^x1_x1 - E^x5_x5 = a4'' F_k(a4') with F_1 = 2, F_2 = -16 (3 a4'^2 + 5 H^2), F_3 =
    144 (5 a4'^4 + 6 a4'^2 H^2 + 5 H^4)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    evolution_factorises (a4-equations.json, evolution_F)
PASS E^x0_x0 is the mean of E^x1_x1 and E^x5_x5 (identity II)
PASS every component is even in a4' and of weight 2k (dimension 1/length^(2k))
```

Text cell:

```
## 7. The conservation identity along the time

Identity (I) of section 4 says $\frac{d}{dx_4} E^{x4}_{x4} = 3a_4'(E^{x1}_{x1} -
E^{x5}_{x5})$. The time component depends on $x_4$ only through $a_4'$, so by the
chain rule $\frac{d}{dx_4} E^{x4}_{x4} = \frac{\partial E^{x4}_{x4}}{\partial a_4'}
\, a_4''$. The next cell checks the identity exactly for $k = 1, 2, 3$ with
`sp.diff(..., A1) * A2`. With section 6 it says
$\partial E^{x4}_{x4}/\partial a_4' = 3 a_4' F_k(a_4')$: the derivative of the
constraint is $3a_4'$ times the evolution factor, the "constraint propagation" of
the Revision record. The cell also checks the factorisation
$E^{x4}_{x4} - E^{x0}_{x0} = 6(a_4'^2 + H^2)\, V_k(a_4')$, with the polynomials $V_1 =
1$, $V_2 = -8(a_4'^2 + 5H^2)$, $V_3 = 72(a_4'^4 + 2a_4'^2 H^2 + 5H^4)$, which the
Revision record of the field equations uses for the linear history
(check `linear_member_vacuum_factor`).
```

In [5]:

```
V = {1: sp.Integer(1), 2: -8 * (A1 ** 2 + 5 * H ** 2),
     3: 72 * (A1 ** 4 + 2 * A1 ** 2 * H ** 2 + 5 * H ** 4)}
identity_I = all(
    sp.expand(sp.diff(E[k][4, 4], A1) * A2 - 3 * A1 * (E[k][1, 1] - E[k][5, 5])) == 0
    for k in (1, 2, 3))
check(identity_I, "d/dx4 E^x4_x4 = 3 a4' (E^x1_x1 - E^x5_x5) for k = 1, 2, 3 "
                  "(identity I)",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "bianchi_x4; Revision/gkd_lovelock/results/lovelock-report.json, checks "
             "k1_divergence_free, k2_divergence_free and k3_divergence_free")
factor_ok = all(sp.expand(E[k][4, 4] - E[k][0, 0] - 6 * (A1 ** 2 + H ** 2) * V[k]) == 0
```

```
                for k in (1, 2, 3))
check(factor_ok, "E^x4_x4 - E^x0_x0 = 6 (a4'^2 + H^2) V_k for k = 1, 2, 3",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "linear_member_vacuum_factor")
```

Out [5]:

```
PASS d/dx4 E^x4_x4 = 3 a4' (E^x1_x1 - E^x5_x5) for k = 1, 2, 3 (identity I)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    bianchi_x4; Revision/gkd_lovelock/results/lovelock-report.json, checks
    k1_divergence_free, k2_divergence_free and k3_divergence_free
PASS E^x4_x4 - E^x0_x0 = 6 (a4'^2 + H^2) V_k for k = 1, 2, 3
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    linear_member_vacuum_factor
```

Text cell:

```
## 8. Along the exponentially deflating history $a_4 = A H x_4$

On the linear history $a_4' = AH$ and $a_4'' = 0$. Then every component is $H^{2k}$
times a polynomial in $A$, and the plots below show $E_{(k)}{}^h{}_h / H^{2k}$
against $A$ from $-3$ to $3$ ($A > 0$: the extra times deflate exponentially, the
author's case; $A < 0$: they would inflate; $A = 0$, where $a_4$ is constant, is the
mirror point between the two). Because $a_4'' = 0$, the space, extra-time and
hidden components coincide (section 6, items 3 and 4), so each panel has two curves:
the time component $E^{x4}_{x4}$ and the common component of the seven other
directions. `sp.lambdify` turns a sympy expression into a numpy function that can
be evaluated on an array of values of $A$. The check confirms that the two curves
are mirror-symmetric about $A = 0$.
```

In [6]:

```
A = np.linspace(-3.0, 3.0, 601)  # values of A
LINEAR = {H: 1, A2: 0}  # H = 1 (units of H) and a4'' = 0; then a4' = A


def on_linear_history(expr):
    """expr / H^(2k) on the linear history, as numbers for every A."""
    function = sp.lambdify(A1, expr.subs(LINEAR), "numpy")
    return np.broadcast_to(function(A), A.shape).astype(float)


fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
symmetric = True
for ax, k in zip(axes, (1, 2, 3)):
    time_part = on_linear_history(E[k][4, 4])
    other_part = on_linear_history(E[k][1, 1])
    symmetric &= np.allclose(time_part, time_part[::-1]) and np.allclose(
        other_part, other_part[::-1])
    ax.plot(A, time_part, label="$E^{x_4}{}_{x_4}$ (time)")
    ax.plot(A, other_part, "--", label="$E^{x_1}{}_{x_1} = E^{x_5}{}_{x_5} = "
```

```
                                        "E^{x_0}{}_{x_0}$")
    ax.axvline(0.0, color="0.5", lw=0.8)
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("$A = a_4^{\\prime}/H$")
    ax.set_ylabel(f"$E_{{({k})}}$ in units of $H^{{{2 * k}}}$")
    ax.set_title(f"order k = {k}")
    ax.legend(fontsize=7, loc="best")
fig.tight_layout()
save_figure(fig, "components_linear_history",
            "The normalised Lovelock tensors $E_{(1)}$ (Einstein), $E_{(2)}$ and "
            "$E_{(3)}$ of the author's metric along the linear history "
            "$a_4 = A H x_4$, in units of $H^2$, $H^4$ and $H^6$, against "
            "$A = a_4^{\\prime}/H$: the time component (solid) and the common "
            "component of the space, extra-time and hidden directions (dashed), "
            "which coincide because $a_4^{\\prime\\prime} = 0$. Every curve is "
            "symmetric about $A = 0$: exponential deflation of the extra times "
            "($A > 0$) and their inflation ($A < 0$) give the same tensors. For "
            "$k = 1$ the curves are $3A^2 + 21$ and $15 - 3A^2$.")
check(symmetric, "on the linear history every curve is symmetric under A -> -A")
```

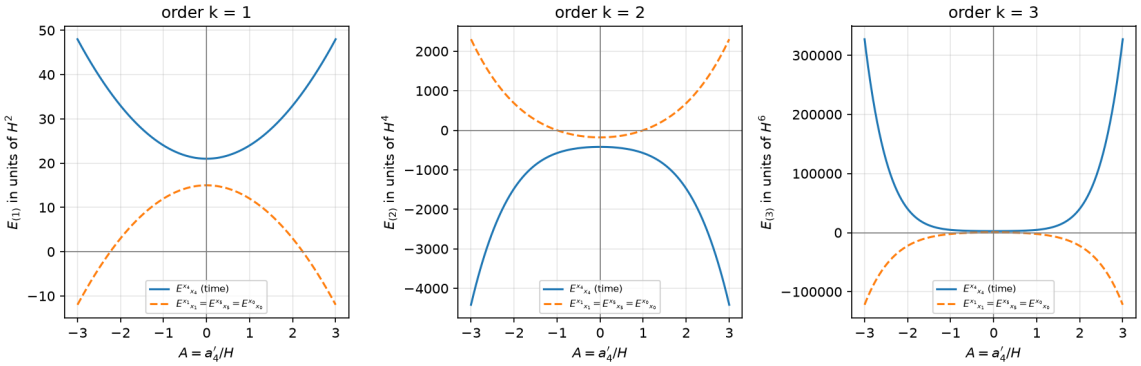

Figure 314: The normalised Lovelock tensors $E_{(1)}$ (Einstein), $E_{(2)}$ and $E_{(3)}$ of the author's metric along the linear history $a_4 = AHx_4$, in units of H^2, H^4 and H^6, against $A = a_4'/H$: the time component (solid) and the common component of the space, extra-time and hidden directions (dashed), which coincide because $a_4'' = 0$. Every curve is symmetric about $A = 0$: exponential deflation of the extra times ($A > 0$) and their inflation ($A < 0$) give the same tensors. For $k = 1$ the curves are $3A^2 + 21$ and $15 - 3A^2$. (Notebook 11c, figure 1.)

Out [6]:

```
Figure 11c.1 saved as Revision/textbook/figures/11c_1_components_linear_history.png
PASS on the linear history every curve is symmetric under A -> -A
```

Text cell:

```
The next cell draws the three Lovelock scalars $L_{(k)}$ (read from the record) on
the linear history, in units of $H^{2k}$, and computes where they are zero. On the
linear history $L_{(k)}/H^{2k}$ is a polynomial in $A$; `sp.Poly(...).nroots()`
computes all its roots numerically (15 digits), and the cell keeps the real positive
```

```
ones (the negative ones are their mirror images). For $k = 1$,
$L_{(1)} = 2R = 12a_4'^2 - 84H^2$ is zero exactly at $A = \sqrt{7} \approx 2.646$.
The scalar of order 3 grows so fast ($A^6$) that its dip near $A = 0$ cannot be
seen at the scale of the whole panel, so the cell adds a small magnified panel
(‘ax.inset_axes‘, a panel inside a panel, placed by its left edge, bottom edge,
width and height as fractions of the big panel) for $-1 \le A \le 1$.
```

In [7]:

```
L_TEXT = {k: re.sub(r"Derivative\[(\d)\]\[b4\]\[x4\]", r"B\1",
                    re.sub(r"Derivative\[(\d)\]\[a4\]\[x4\]", r"A\1",
                           tensors[f"L{k}"])).replace("b4[x4]", "B0")
          for k in (1, 2, 3)}
L = {k: sp.expand(sp.sympify(L_TEXT[k].replace("^", "**"),
                             locals={"A1": A1, "A2": A2, "H": H, "B0": B0, "B1": B1,
                                     "B2": B2}).subs(BACKGROUND))
     for k in (1, 2, 3)}  # the scalars at b4 = 1
fig, axes = plt.subplots(1, 3, figsize=(12.0, 3.8))
zeros = {}
for ax, k in zip(axes, (1, 2, 3)):
    values = on_linear_history(L[k])
    roots = sp.Poly(L[k].subs(H, 1), A1).nroots(n=15)  # every root, numerically
    zeros[k] = [float(r) for r in roots if r.is_real and r > 0]
    listed = ", ".join(f"{r:.4f}" for r in zeros[k])  # the zeros, 4 decimals
    say(f"L({k}) = {L[k]}")
    report(f"positive zero of L({k}) on the linear history (and minus it), A", listed)
    ax.plot(A, values, color="tab:green")
    for r in zeros[k]:
        ax.plot([r, -r], [0.0, 0.0], "o", color="black", markersize=4)
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.axvline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("$A = a_4^{\\prime}/H$")
    ax.set_ylabel(f"$L_{{({k})}}$ in units of $H^{{{2 * k}}}$")
    ax.set_title(f"Lovelock scalar of order {k}")
    if k == 3:  # a magnified view of the middle, where the zeros of L(3) lie
        zoom = ax.inset_axes([0.30, 0.42, 0.42, 0.42])  # [left, bottom, width, height]
        near = np.abs(A) <= 1.0  # the values of A between -1 and 1
        zoom.plot(A[near], values[near], color="tab:green")
        zoom.axhline(0.0, color="0.5", lw=0.8)
        for r in zeros[k]:
            zoom.plot([r, -r], [0.0, 0.0], "o", color="black", markersize=3)
        zoom.tick_params(labelsize=6)
        zoom.set_title("magnified: $-1 \\leq A \\leq 1$", fontsize=7)
fig.tight_layout()
save_figure(fig, "lovelock_scalars",
            "The Lovelock scalars $L_{(1)} = 2R = 12 a_4^{\\prime 2} - 84 H^2$, "
            "$L_{(2)}$ and $L_{(3)}$ of the author’s metric along the linear history "
            "$a_4 = A H x_4$, in units of $H^2$, $H^4$ and $H^6$, against "
            "$A = a_4^{\\prime}/H$; the small panel inside the right one magnifies "
            "the range $-1 \\leq A \\leq 1$. Each scalar is an even polynomial in "
            "$A$ with one positive zero, marked by a black dot together with its "
            f"mirror image: $A = {zeros[1][0]:.3f}$ (that is $\\sqrt{{7}}$), "
            f"${zeros[2][0]:.3f}$ and ${zeros[3][0]:.3f}$. At the mirror point "
```

```
          "$A = 0$ the three values are $-84$, $3360$ and $-40320$.")
check(all(len(zeros[k]) == 1 for k in (1, 2, 3))
      and abs(zeros[1][0] - float(sp.sqrt(7))) < 1e-12
      and L[1].subs({A1: 0, H: 1}) == -84 and L[2].subs({A1: 0, H: 1}) == 3360
      and L[3].subs({A1: 0, H: 1}) == -40320,
      "each scalar has one positive zero, L(1) at A = sqrt(7); at A = 0 the scalars "
      "are -84, 3360, -40320")
```

Out [7]:

```
L(1) = 12*A1**2 - 84*H**2
RESULT positive zero of L(1) on the linear history (and minus it), A = 2.6458
L(2) = -96*A1**4 - 2112*A1**2*H**2 + 3360*H**4
RESULT positive zero of L(2) on the linear history (and minus it), A = 1.2207
L(3) = 1152*A1**6 + 31104*A1**4*H**2 + 100224*A1**2*H**4 - 40320*H**6
RESULT positive zero of L(3) on the linear history (and minus it), A = 0.6010
```

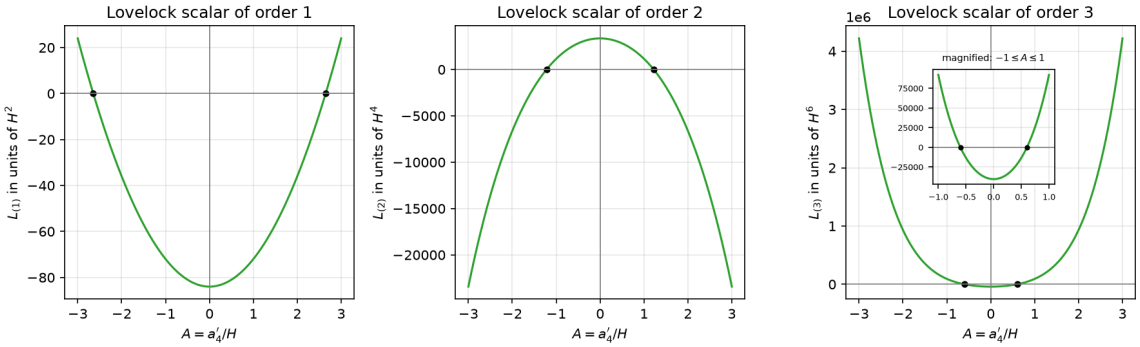

Figure 315: The Lovelock scalars $L_{(1)} = 2R = 12a_4'^2 - 84H^2$, $L_{(2)}$ and $L_{(3)}$ of the author's metric along the linear history $a_4 = AHx_4$, in units of H^2, H^4 and H^6, against $A = a_4'/H$; the small panel inside the right one magnifies the range $-1 \le A \le 1$. Each scalar is an even polynomial in A with one positive zero, marked by a black dot together with its mirror image: $A = 2.646$ (that is $\sqrt{7}$), 1.221 and 0.601. At the mirror point $A = 0$ the three values are -84, 3360 and -40320. (Notebook 11c, figure 2.)

Out [7]:

```
Figure 11c.2 saved as Revision/textbook/figures/11c_2_lovelock_scalars.png
PASS each scalar has one positive zero, L(1) at A = sqrt(7); at A = 0 the scalars are
    -84, 3360, -40320
```

Text cell:

```
## 9. An illustrative test history in which the deflation speeds up

On the linear history $a_4'' = 0$, so it cannot show where $a_4''$ enters. The next
cells therefore use a test history in which the extra times deflate exponentially
at ALL times, slowly in the far past and three times faster in the far future:
```

```
$$a_4(x_4) = H x_4 + \tfrac12 \ln\cosh(H x_4),\qquad
  a_4' = H\big(1 + \tfrac12 \tanh(H x_4)\big),\qquad
  a_4'' = \tfrac{H^2}{2}\,\mathrm{sech}^2(H x_4).$$

(The derivative of $\ln\cosh t$ is $\tanh t$, and that of $\tanh t$ is
$\mathrm{sech}^2 t = 1/\cosh^2 t$.) Since $-1 < \tanh < 1$, $a_4'$ lies between
$H/2$ and $3H/2$ and is always positive: the extra times always deflate. For
$x_4 \to -\infty$, $a_4' \to H/2$ (the linear history with $A = 1/2$); for
$x_4 \to +\infty$, $a_4' \to 3H/2$ ($A = 3/2$); $a_4''$ is largest, $H^2/2$, at
$x_4 = 0$, where the deflation speeds up. This history is an ILLUSTRATION chosen
only to show how the components depend on $a_4'$ and $a_4''$; it is not a solution
of the field equations, and no source is attached to it. The next cell checks the
two derivatives with sympy and draws the history.
```

In [8]:

```
t = sp.Symbol("t")  # t = H x4 (the time in units of 1/H); H = 1 below
history = t + sp.log(sp.cosh(t)) / 2
first = sp.diff(history, t)  # a4' (with H = 1)
second = sp.diff(history, t, 2)  # a4''
check(sp.simplify(first - (1 + sp.tanh(t) / 2)) == 0
      and sp.simplify(second - sp.sech(t) ** 2 / 2) == 0,
      "the test history has a4' = 1 + tanh/2 > 0 and a4'' = sech^2/2 (H = 1)")
times = np.linspace(-6.0, 6.0, 1201)  # H x4 from -6 to 6
a4_values = sp.lambdify(t, history, "numpy")(times)
a1_values = sp.lambdify(t, first, "numpy")(times)
a2_values = sp.lambdify(t, second, "numpy")(times)
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0))
axes[0].semilogy(times, np.exp(a4_values), label="space, $e^{a_4}$ (inflates)")
axes[0].semilogy(times, np.exp(-a4_values), "--",
                 label="extra times, $e^{-a_4}$ (deflate)")
axes[0].set_xlabel("time $H x_4$")
axes[0].set_ylabel("scale factor (logarithmic scale)")
axes[0].set_title("scale factors of the test history")
axes[0].legend(fontsize=8)
axes[1].plot(times, a1_values, label="$a_4^{\\prime}/H$")
axes[1].plot(times, a2_values, "--", label="$a_4^{\\prime\\prime}/H^2$")
axes[1].set_xlabel("time $H x_4$")
axes[1].set_ylabel("value")
axes[1].set_title("its first and second derivatives")
axes[1].legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "test_history",
            "The illustrative test history $a_4 = H x_4 + \\frac{1}{2} \\ln\\cosh H "
            "x_4$, not a solution of any field equation, against the time $H x_4$. "
            "Left: the scale factor $e^{a_4}$ of ordinary space (solid) grows and the "
            "scale factor $e^{-a_4}$ of the extra times (dashed) shrinks at all "
            "times, both on a logarithmic axis, where exponential change is a "
            "straight line: the slope steepens from $1/2$ to $3/2$ around $x_4 = 0$. "
            "Right: $a_4^{\\prime}/H$ rises from 1/2 to 3/2 and "
            "$a_4^{\\prime\\prime}/H^2$ is a bump of height 1/2 at $x_4 = 0$.")
```

Out [8]:

```
PASS the test history has a4' = 1 + tanh/2 > 0 and a4'' = sech^2/2 (H = 1)
```

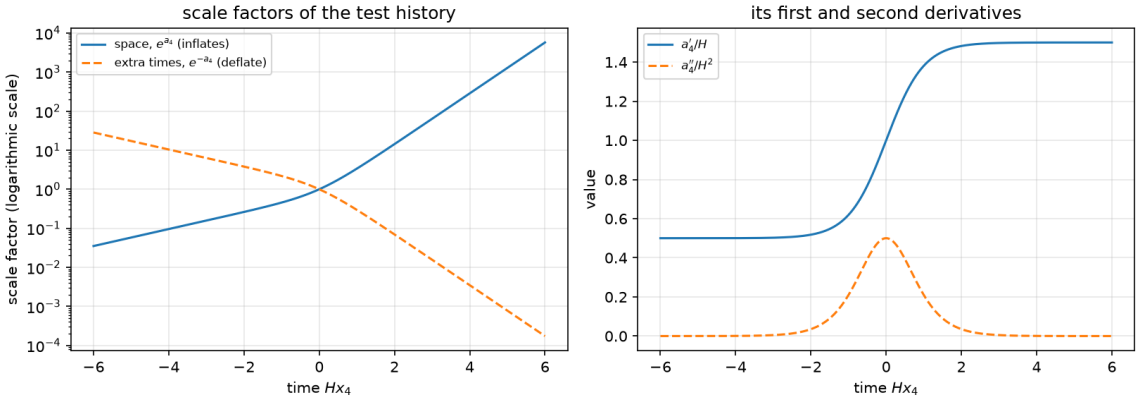

Figure 316: The illustrative test history $a_4 = Hx_4 + \frac{1}{2}\ln\cosh Hx_4$, not a solution of any field equation, against the time Hx_4. Left: the scale factor e^{a_4} of ordinary space (solid) grows and the scale factor e^{-a_4} of the extra times (dashed) shrinks at all times, both on a logarithmic axis, where exponential change is a straight line: the slope steepens from $1/2$ to $3/2$ around $x_4 = 0$. Right: a_4'/H rises from $1/2$ to $3/2$ and a_4''/H^2 is a bump of height $1/2$ at $x_4 = 0$. (Notebook 11c, figure 3.)

Out [8]:

```
Figure 11c.3 saved as Revision/textbook/figures/11c_3_test_history.png
```

Text cell:

```
The next cell evaluates the four independent components of $E_{(1)}, E_{(2)},
E_{(3)}$ along the test history (with $H = 1$, so in units of $H^{2k}$) and draws
them. Where $a_4''$ is not zero, the space component $E^{x1}_{x1}$ and the
extra-time component $E^{x5}_{x5}$ separate (their difference is
$a_4'' F_k(a_4')$), and the hidden component $E^{x0}_{x0}$ runs exactly in the
middle; far in the past and far in the future, where $a_4'' \to 0$, the three
coincide again. The check confirms the middle position at all 1201 times.
```

In [9]:

```
def along_history(expr):
    """expr (H = 1) at every time of the test history."""
    function = sp.lambdify((A1, A2), expr.subs(H, 1), "numpy")
    return np.broadcast_to(function(a1_values, a2_values), times.shape).astype(float)


fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
middle = True
curves = {}
```

```
for ax, k in zip(axes, (1, 2, 3)):
    curves[k] = {h: along_history(E[k][h, h]) for h in (1, 4, 5, 0)}
    middle &= np.allclose(curves[k][0], (curves[k][1] + curves[k][5]) / 2,
                          rtol=1e-12, atol=1e-9)
    for h, style in ((4, "-"), (1, "-"), (5, "--"), (0, ":")):
        ax.plot(times, curves[k][h], style,
                label=f"$E^{{x_{h}}}_{{}}_{{x_{h}}}$")
    ax.set_xlabel("time $H x_4$")
    ax.set_ylabel(f"$E_{{({k})}}$ in units of $H^{{{2 * k}}}$")
    ax.set_title(f"order k = {k}")
    ax.legend(fontsize=7)
fig.tight_layout()
save_figure(fig, "components_test_history",
            "The four independent components of $E_{(1)}$, $E_{(2)}$ and $E_{(3)}$ "
            "along the illustrative test history, in units of $H^2$, $H^4$, $H^6$, "
            "against the time $H x_4$: the time component $E^{x_4}_{\\ x_4}$, the "
            "space component $E^{x_1}_{\\ x_1}$, the extra-time component "
            "$E^{x_5}_{\\ x_5}$ (dashed) and the hidden component "
            "$E^{x_0}_{\\ x_0}$ (dotted). Space and extra-time components separate "
            "where $a_4^{\\prime\\prime} \\neq 0$, by $a_4^{\\prime\\prime} "
            "F_k(a_4^{\\prime})$, and the hidden one lies exactly halfway between "
            "them; the time component depends on $a_4^{\\prime}$ only.")
check(middle, "along the test history E^x0_x0 is the mean of E^x1_x1 and E^x5_x5 "
              "at all 1201 times")
```

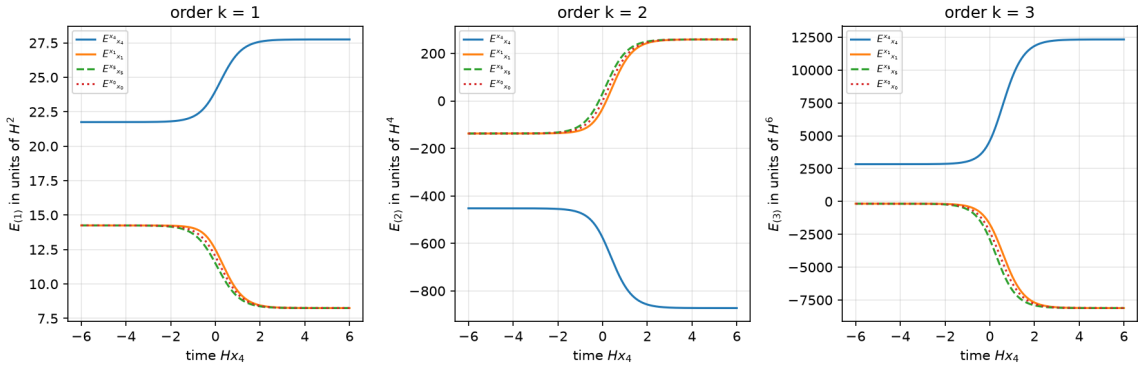

Figure 317: The four independent components of $E_{(1)}$, $E_{(2)}$ and $E_{(3)}$ along the illustrative test history, in units of H^2, H^4, H^6, against the time Hx_4: the time component $E^{x_4}_{\ x_4}$, the space component $E^{x_1}_{\ x_1}$, the extra-time component $E^{x_5}_{\ x_5}$ (dashed) and the hidden component $E^{x_0}_{\ x_0}$ (dotted). Space and extra-time components separate where $a_4'' \neq 0$, by $a_4'' F_k(a_4')$, and the hidden one lies exactly halfway between them; the time component depends on a_4' only. (Notebook 11c, figure 4.)

Out [9]:

```
Figure 11c.4 saved as Revision/textbook/figures/11c_4_components_test_history.png
PASS along the test history E^x0_x0 is the mean of E^x1_x1 and E^x5_x5 at all 1201 times
```

Text cell:

The last computation checks identity (I) NUMERICALLY along the test history, as an independent confirmation of the exact proof of section 7: the derivative of the time component, computed as a finite difference of the 1201 values (‘np.gradient‘ uses $(f(t + \epsilon) - f(t - \epsilon))/(2\epsilon)$ inside the interval, with $\epsilon = 0.01$), must agree with $3a_4'(E^{x1}_{x1} - E^{x5}_{x5})$ up to the error of the finite difference, which is of the order of ϵ^2 times the third derivative. The cell prints the largest deviation for each order, relative to the largest value of the curve, and draws both sides.

In [10]:

```
step = times[1] - times[0]  # epsilon = 0.01
fig, axes = plt.subplots(1, 3, figsize=(12.0, 3.8))
worst = {}
for ax, k in zip(axes, (1, 2, 3)):
    left = np.gradient(curves[k][4], step)  # d/dx4 of the time component
    right = 3 * a1_values * (curves[k][1] - curves[k][5])
    inner = slice(1, -1)  # the two end points use a one-sided difference
    worst[k] = float(np.max(np.abs(left[inner] - right[inner]))
                     / np.max(np.abs(right)))
    report(f"order {k}: largest relative deviation of the finite difference",
           f"{worst[k]:.1e}")
    ax.plot(times, right, color="tab:blue", lw=3, alpha=0.5,
            label="$3 a_4^{\\prime} (E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5})$")
    ax.plot(times, left, "k:", label="$d E^{x_4}{}_{x_4} / d x_4$ (finite difference)")
    ax.set_xlabel("time $H x_4$")
    ax.set_ylabel(f"units of $H^{{{2 * k + 1}}}$")
    ax.set_title(f"identity (I), order k = {k}")
    ax.legend(fontsize=7)
fig.tight_layout()
largest = max(worst.values())  # the largest relative deviation of the three orders
power = int(np.floor(np.log10(largest)))  # its power of ten
deviation_text = f"${largest / 10 ** power:.1f} \\times 10^{{{power}}}$"
save_figure(fig, "conservation_identity",
            "Identity (I), the conservation identity along the time, checked "
            "numerically along the illustrative test history for $k = 1, 2, 3$: the "
            "finite-difference derivative of the time component $E^{x_4}_{\\ x_4}$ "
            "(dotted black) lies on $3 a_4^{\\prime} (E^{x_1}_{\\ x_1} - "
            "E^{x_5}_{\\ x_5})$ (thick blue), in units of $H^{2k+1}$, against the "
            "time $H x_4$. Both are zero where $a_4^{\\prime\\prime} = 0$ and peak "
            "where the deflation speeds up; the largest relative deviation is "
            f"about {deviation_text}, the size of the finite-difference error.")
check(all(value < 1e-4 for value in worst.values()),
      "the finite-difference derivative agrees with identity (I) to 1e-4")
```

Out [10]:

```
RESULT order 1: largest relative deviation of the finite difference = 3.8e-05
RESULT order 2: largest relative deviation of the finite difference = 4.0e-05
RESULT order 3: largest relative deviation of the finite difference = 4.5e-05
```

Out [10]:

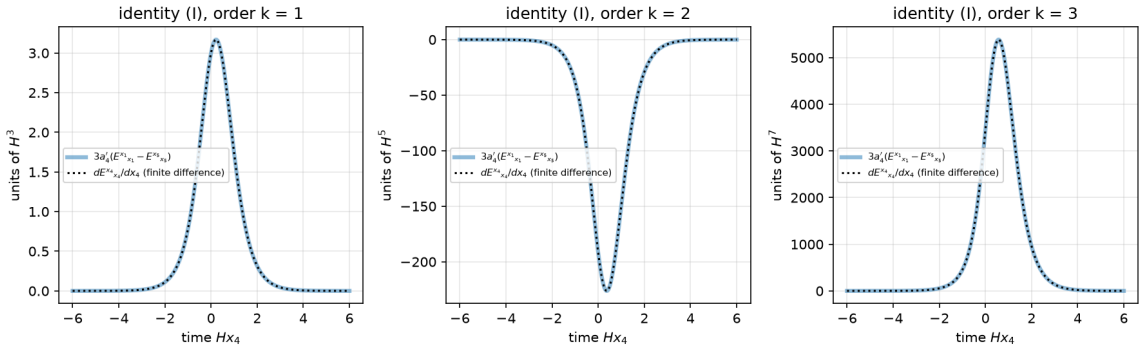

Figure 318: Identity (I), the conservation identity along the time, checked numerically along the illustrative test history for $k = 1, 2, 3$: the finite-difference derivative of the time component $E^{x_4}_{x_4}$ (dotted black) lies on $3a_4'(E^{x_1}_{x_1} - E^{x_5}_{x_5})$ (thick blue), in units of H^{2k+1}, against the time Hx_4. Both are zero where $a_4'' = 0$ and peak where the deflation speeds up; the largest relative deviation is about 4.5×10^{-5}, the size of the finite-difference error. (Notebook 11c, figure 5.)

```
Figure 11c.5 saved as Revision/textbook/figures/11c_5_conservation_identity.png
PASS the finite-difference derivative agrees with identity (I) to 1e-4
```

Text cell:

```
## 10. The Revision records quoted in this notebook

This notebook quotes check names and statements from the Revision records. The
components themselves were compared with ‘lovelock-tensors.json‘,
‘python-lovelock-report.json‘ and ‘a4-equations.json‘ in section 5. The next cell
reads the records once more and checks the rest of what the text quotes, so that a
later change of a record cannot pass unnoticed:

- every check quoted from the report ‘python-a4-report.json‘ of the field equations
  of $a_4$ and from the Rust report ‘lovelock-report.json‘ is present with the
  verdict PASS;
- the record ‘a4-equations.json‘ states the two consequences of the conservation
  identities quoted in section 4 (its entries ‘conservation_reduced‘ and
  ‘algebraic_condition‘) and the constraint propagation quoted in section 7 (its
  entry ‘constraint_propagation‘).

The function ‘verdicts(path)‘ returns, for one report, a dictionary from the name of
each check to its verdict. The reports store their checks in one of two forms: a
list of entries, each with a ‘name‘ and a ‘verdict‘, or a dictionary from the name to
an entry whose field ‘passed‘ is true or false; the function reads both.
```

In [11]:

```
def verdicts(path):
    """{check name: "PASS" or "FAIL"} of the Revision report at ‘path‘."""
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    checks = data["checks"]
```

```
    if isinstance(checks, dict):  # name -> {"passed": true or false, ...}
        return {name: "PASS" if entry["passed"] else "FAIL"
                for name, entry in checks.items()}
    return {entry["name"]: entry["verdict"] for entry in checks}  # a list of entries


QUOTED = {  # report -> the checks that this notebook quotes from it
    "Revision/field_equations_a4/reports/python-a4-report.json": [
        "json_lovelock_components", "evolution_factorises", "bianchi_x4",
        "linear_member_vacuum_factor"],
    "Revision/gkd_lovelock/results/lovelock-report.json": [
        "k1_divergence_free", "k2_divergence_free", "k3_divergence_free"],
}
for path, names in QUOTED.items():
    found = verdicts(path)
    for name in names:
        say(f"{path.split('/')[-1]}: {name} {found.get(name, 'MISSING')}")
    check(all(found.get(name) == "PASS" for name in names),
          f"the {len(names)} checks quoted from {path.split('/')[-1]} are there and "
          "PASS", record=f"{path}, checks " + ", ".join(names))
general = a4_record["generalSource"]  # a4-equations.json, read in section 5
stated = (general["algebraic_condition"]["input"] == "p3 + pt == 2*p0"
          and "rho' = -3 a4' (p3 - p_t)" in general["conservation_reduced"]
          and "p3 + p_t = 2 p0" in general["conservation_reduced"]
          and "d/dx4 (constraint) = 3 a4' (evolution)"
          in general["constraint_propagation"])
check(stated, "a4-equations.json states rho' = -3 a4' (p3 - p_t), p3 + p_t = 2 p0 and "
              "the constraint propagation, as quoted",
      record="Revision/field_equations_a4/a4-equations.json, generalSource: "
             "conservation_reduced, algebraic_condition, constraint_propagation")
```

Out [11]:

```
python-a4-report.json: json_lovelock_components PASS
python-a4-report.json: evolution_factorises PASS
python-a4-report.json: bianchi_x4 PASS
python-a4-report.json: linear_member_vacuum_factor PASS
PASS the 4 checks quoted from python-a4-report.json are there and PASS
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, checks
    json_lovelock_components, evolution_factorises, bianchi_x4,
    linear_member_vacuum_factor
lovelock-report.json: k1_divergence_free PASS
lovelock-report.json: k2_divergence_free PASS
lovelock-report.json: k3_divergence_free PASS
PASS the 3 checks quoted from lovelock-report.json are there and PASS
     reproduces Revision/gkd_lovelock/results/lovelock-report.json, checks
    k1_divergence_free, k2_divergence_free, k3_divergence_free
PASS a4-equations.json states rho' = -3 a4' (p3 - p_t), p3 + p_t = 2 p0 and the
    constraint propagation, as quoted
     reproduces Revision/field_equations_a4/a4-equations.json, generalSource:
    conservation_reduced, algebraic_condition, constraint_propagation
```

Text cell:

```
The last cell checks that all five figure files exist and prints the number of
checks that passed.
```

In [12]:

```
figure_files = [f"11c_{k}_{name}.png" for k, name in enumerate(
    ["components_linear_history", "lovelock_scalars", "test_history",
     "components_test_history", "conservation_identity"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_files),
      "all five figure files of the notebook exist")
all_checks_passed()
```

Out [12]:

```
PASS all five figure files of the notebook exist
ALL 20 CHECKS PASSED (notebook 11c)
```

Text cell:

```
## 11. What this notebook showed

- The normalised Lovelock tensors of the author's metric are diagonal polynomials in
  $H$, $a_4'$ and $a_4''$, for example Einstein's tensor
  $E^{x1}_{x1} = -3a_4'^2 + a_4'' + 15H^2$, $E^{x4}_{x4} = 3a_4'^2 + 21H^2$,
  $E^{x5}_{x5} = -3a_4'^2 - a_4'' + 15H^2$, $E^{x0}_{x0} = -3a_4'^2 + 15H^2$; all
  three orders equal the components recorded for the field equations of $a_4$.
- Their structure (PROVED): space components equal, extra-time components equal;
  time and hidden components free of $a_4''$; the space minus extra-time difference
  is $a_4''$ times $F_1 = 2$, $F_2 = -16(3a_4'^2 + 5H^2)$, $F_3 = 144(5a_4'^4 +
  6a_4'^2H^2 + 5H^4)$; the hidden component is the mean of the space and extra-time
  components; everything is even in $a_4'$, so deflation of the extra times is not
  selected by these tensors; order $k$ has the dimension $1/\text{length}^{2k}$.
- The two conservation identities, $\frac{d}{dx_4}E^{x4}_{x4} = 3a_4'(E^{x1}_{x1} -
  E^{x5}_{x5})$ and $E^{x0}_{x0} = \frac12(E^{x1}_{x1} + E^{x5}_{x5})$, hold exactly
  for every order and were confirmed numerically along a test history.
- On the exponentially deflating linear history $a_4 = AHx_4$ only two different
  components remain (time, and all the others), even functions of $A$; the test
  history (an illustration only) shows that $a_4''$ separates the space and
  extra-time components while the hidden one stays halfway between them.
- Every check and statement that the notebook quotes from the Revision records is
  present in them, every quoted check with the verdict PASS.
```

11.27 Line-by-line walk-through of Notebook 11c

The notebook has 12 code cells, In [1] to In [12]. As in Sections 11.9 and 11.19, every line or small group of lines is quoted and explained; docstrings are left out where the explanation repeats them, and long captions are shortened to ... (they are printed in full under the figures in Section 11.26).

In [1], the set-up cell. It is the set-up cell of Notebook 11a, explained line by line in Section 11.9 under In [1], with three differences: its comment lines are the run instructions of Notebook

11c (Section 11.25); its line `NOTEBOOK_ID = "11c" # this notebook: chapter 11, example c` names this notebook; and, because this notebook runs no Rust program, the two lines `import shutil` and `import subprocess` and the helper `rust_program` are absent. It prints Set-up of notebook 11c complete: repository folder found, helpers defined.

In [2], the components from the Revision record.

```
import re  # regular expressions: rewrites the record's names

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra

H, A1, A2 = sp.symbols("H A1 A2")  # H, a4' and a4''
B0, B1, B2, c = sp.symbols("B0 B1 B2 c")  # b4, b4', b4'' and cot z
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
tensors = json.loads(repository_file(
    "Revision/gkd_lovelock/results/lovelock-tensors.json").read_text(encoding="utf-8"))
```

`re` (regular expressions, used in In [7] to rewrite the names of the record), numpy and sympy are loaded; three sympy symbols stand for H, a_4' and a_4'', and four more, `B0`, `B1`, `B2` and `c`, for b_4, b_4', b_4'' and $\cot z$; `NAMES` lists the coordinate names; and the Rust record of the Lovelock tensors is read as JSON (Section 11.9 explains `repository_file`, `read_text` and `json.loads`).

```
def from_monomials(monomials):
    total = sp.Integer(0)
    for numerator, denominator, e in monomials:
        if e[3] or e[4] or e[5] or e[6] or e[11] or e[12]:
            raise ValueError("the component contains a3, a4 derivatives, e^a4, "
                             "sin^(1/3) z or higher derivatives of b4")
        monomial = (H ** e[0] * A1 ** e[1] * A2 ** e[2] * c ** e[7]
                    * B0 ** sp.Rational(e[8], 2) * B1 ** e[9] * B2 ** e[10])
        total += sp.Rational(numerator, denominator) * monomial
    return total
```

A component of the record is a list of monomials, each a numerator, a denominator and thirteen exponents (of H, a_4', a_4'', a_4''', a_4'''', e^{a_4}, $\sin^{1/3} z$, $\cot z$, $Q = b_4^{1/2}$, b_4', b_4'', b_4''', b_4''''). The function (docstring left out) rebuilds the component as a sympy expression, for a general b_4. If one of the exponents of a_4''', a_4'''', e^{a_4}, $\sin^{1/3} z$, b_4''' or b_4'''' were not 0, the notebook would stop. Since it does not stop, every component is free of e^{a_4}, of $\sin^{1/3} z$ and of higher derivatives (Section 11.20). Each monomial is the exact fraction (`sp.Rational`) times $H^{e_0} a_4'^{e_1} a_4''^{e_2} \cot^{e_7} z\, b_4^{e_8/2} b_4'^{e_9} b_4''^{e_{10}}$.

```
P_B4 = {k: {(h, j): from_monomials(tensors[f"P{k}_mixed_up_h_down_j"][f"{a},{b}"]
                                   ["monomials"])
            for h, a in enumerate(NAMES) for j, b in enumerate(NAMES)}
        for k in (1, 2, 3)}
BACKGROUND = {B0: 1, B1: 0, B2: 0}  # the background b4 = 1 of the records
P = {k: {hj: sp.expand(value.subs(BACKGROUND)) for hj, value in P_B4[k].items()}
     for k in (1, 2, 3)}
mixed_b4 = all({hj for hj, value in P_B4[k].items() if hj[0] != hj[1] and value != 0}
               == {(4, 0), (0, 4)} and P_B4[k][4, 0].has(B1) and P_B4[k][0, 4].has(B1)
```

```
                for k in (1, 2, 3))
check(mixed_b4 and all(not P[k][hj].has(c) for k in (1, 2, 3) for hj in P[k]),
      "with b4 the only nonzero off-diagonal components are x4-x0 and x0-x4, both "
      "with b4'; at b4 = 1 no component contains cot z")
E = {k: {hj: sp.expand(-value / 2 ** (k + 1)) for hj, value in P[k].items()}
     for k in (1, 2, 3)}
```

`P_B4[k][h, j]` is the component ${P_{(k)}}^h{}_j$ with a general b_4, built for all 64 pairs (`enumerate(NAMES)` gives each number with its name, used for the record's keys such as `"x1,x1"`). `BACKGROUND` is the substitution $b_4 = 1$, $b_4' = b_4'' = 0$, and `P[k][h, j]` the component there. The first check requires that with b_4 exactly the two mixed components x_4-x_0 and x_0-x_4 are nonzero off the diagonal, both containing b_4', and that at $b_4 = 1$ no component contains $\cot z$. First PASS line. `E[k]` holds the normalised components $E_{(k)} = -P_{(k)}/2^{k+1}$ at $b_4 = 1$, multiplied out.

```
for k in (1, 2, 3):
    for h in (1, 4, 5, 0):  # x1, x4, x5, x0
        say(f"E({k})^{NAMES[h]}_{NAMES[h]} = {E[k][h, h]}")
check(all(E[k][h, j] == 0 for k in (1, 2, 3) for h in range(8) for j in range(8)
          if h != j),
      "at b4 = 1 every off-diagonal component of E(1), E(2), E(3) is zero, x4-x0 "
      "included")
```

The four independent components of each order are printed: twelve lines that are exactly the list of Section 11.20 (sympy writes `A1**2` for $a_4'^2$). The check confirms that at $b_4 = 1$ every off-diagonal component is zero (Section 11.21 (a)). Second PASS line.

In [3], against the other records.

```
a4_record = json.loads(repository_file(
    "Revision/field_equations_a4/a4-equations.json").read_text(encoding="utf-8"))
WOLFRAM_NAMES = {"ad1": A1, "ad2": A2, "H": H}


def from_text(text):
    return sp.sympify(text.replace("^", "**"), locals=WOLFRAM_NAMES)
```

The record of the field equations of a_4 is read. It writes formulas in Wolfram's input notation, with `ad1` for a_4', `ad2` for a_4'' and `^` for powers; `from_text` (docstring left out) replaces `^` by `**` and reads the text with these names bound to the notebook's symbols.

```
same = True
for k in (1, 2, 3):
    listed = a4_record["lovelockTensors"][f"E{k}"]
    for key, (h, j) in (("x1x1", (1, 1)), ("x4x4", (4, 4)), ("x5x5", (5, 5)),
                        ("x0x0", (0, 0)), ("x4x0", (4, 0)), ("x0x4", (0, 4))):
        same = same and sp.expand(from_text(listed[key]["input"]) - E[k][h, j]) == 0
check(same, "E(1), E(2), E(3) equal the components recorded for the a4 equations",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "json_lovelock_components")
```

For each order the record's entry `lovelockTensors` lists six components, the four independent diagonal ones and the two x_4-x_0 ones (which are 0). Each must equal the notebook's component exactly. Third PASS line.

```
checker = json.loads(repository_file(
    "Revision/gkd_lovelock/results/python-lovelock-report.json").read_text(
        encoding="utf-8"))
independent = checker["independentResults"]
same = all(
    sp.simplify(sp.sympify(text, locals={"A1": A1, "A2": A2, "H": H, "B0": B0,
                                         "B1": B1, "B2": B2, "c": c})
                - P_B4[k][NAMES.index(key.split(",")[0]),
                          NAMES.index(key.split(",")[1])]) == 0
    for k in (1, 2, 3) for key, text in independent[f"P{k}_mixed_nonzero"].items())
check(same and all(len(independent[f"P{k}_mixed_nonzero"]) == 10 for k in (1, 2, 3)),
      "P(1), P(2), P(3) equal the independent sympy results with b4 (10 nonzero "
      "components each)",
      record="Revision/gkd_lovelock/results/python-lovelock-report.json, "
             "independentResults")
```

The Revision's sympy verification stores its own, independently computed nonzero components in the entry `independentResults`, under keys such as `"x1,x1"`; `key.split(",")` cuts a key into the two names. These are computed with a general b_4, in the symbols `B0`, `B1`, `B2` and `c`. Every one must equal the notebook's $P_{(k)}$ with b_4 exactly (`sp.simplify` of the difference must be 0), and there must be exactly 10 for each order: the eight diagonal ones and the two mixed ones. Fourth PASS line.

In [4], the structure.

```
u = sp.Symbol("u")  # an auxiliary number used to test the weights
F = {}  # k -> F_k(a4')
alphas = sp.symbols("alpha1 alpha2 alpha3")  # the constants of the a4 equations
recorded_F = sp.sympify(
    a4_record["generalSource"]["evolution_F"]["input"].replace("^", "**"),
    locals={"ad1": A1, "H": H, "alpha1": alphas[0], "alpha2": alphas[1],
            "alpha3": alphas[2]})
```

The symbol `u` will scale the components; `F` will hold the evolution factors; the three Lovelock constants $\alpha_1, \alpha_2, \alpha_3$ are symbols; and the record's evolution factor $F = \alpha_1 F_1 + \alpha_2 F_2 + \alpha_3 F_3$ (key `generalSource`, entry `evolution_F`) is read.

```
structure = {name: True for name in ("alike", "constraint", "evolution", "mean",
                                     "even", "weight")}
```

A dictionary of six true/false flags, one for each property of Section 11.21; each starts true and is combined below with the result for every order.

```
for k in (1, 2, 3):
    e = E[k]
```

```
    structure["alike"] &= (e[1, 1] == e[2, 2] == e[3, 3]
                           and e[5, 5] == e[6, 6] == e[7, 7])
    structure["constraint"] &= not e[4, 4].has(A2) and not e[0, 0].has(A2)
```

`&=` combines a flag with a new result by AND: it stays true only if both are true. "alike": the three space components are equal and the three extra-time components are equal (Section 11.21 (b)). "constraint": the time and the hidden component do not contain a_4'' (`expr.has(A2)` is true when the symbol occurs; parts (d) and (f)).

```
    F[k] = sp.factor(sp.cancel((e[1, 1] - e[5, 5]) / A2))
    structure["evolution"] &= (not F[k].has(A2)
                               and sp.expand(F[k] - recorded_F.coeff(alphas[k - 1]))
                               == 0)
```

The space minus the extra-time component, divided by a_4'': `sp.cancel` divides exactly and `sp.facto r` writes the result as a product. It must be free of a_4'' (part (e)) and equal to the coefficient of α_k in the record's F (`.coeff(alphas[k - 1])`; Python counts the list `alphas` from 0).

```
    structure["mean"] &= sp.expand(e[0, 0] - (e[1, 1] + e[5, 5]) / 2) == 0
    structure["even"] &= all(sp.expand(e[h, h].subs(A1, -A1) - e[h, h]) == 0
                             for h in range(8))
```

"mean": identity (II) of Section 11.22. "even": replacing a_4' by $-a_4'$ (`.subs(A1, -A1)`) changes no diagonal component (part (g)).

```
    weighted = {h: sp.expand(e[h, h].subs({H: u * H, A1: u * A1, A2: u ** 2 * A2}))
                for h in range(8)}
    structure["weight"] &= all(sp.expand(weighted[h] - u ** (2 * k) * e[h, h]) == 0
                               for h in range(8))
    report(f"F_{k}(a4') = (E({k})^x1_x1 - E({k})^x5_x5) / a4''", F[k])
```

"weight": replacing H, a_4', a_4'' by uH, ua_4', u^2a_4'' must multiply each component by u^{2k}, which is true exactly when every term has the weight $2k$ (part (h)). The RESULT lines print $F_1 = 2$, $F_2 = -16(3a_4'^2 + 5H^2)$ and $F_3 = 144(5a_4'^4 + 6a_4'^2H^2 + 5H^4)$.

```
check(structure["alike"], "space components equal, extra-time components equal")
check(structure["constraint"], "the x4 and x0 components contain no a4''")
check(structure["evolution"],
      "E^x1_x1 - E^x5_x5 = a4'' F_k(a4') with F_1 = 2, F_2 = -16 (3 a4'^2 + 5 H^2), "
      "F_3 = 144 (5 a4'^4 + 6 a4'^2 H^2 + 5 H^4)",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "evolution_factorises (a4-equations.json, evolution_F)")
check(structure["mean"], "E^x0_x0 is the mean of E^x1_x1 and E^x5_x5 (identity II)")
check(structure["even"] and structure["weight"],
      "every component is even in a4' and of weight 2k (dimension 1/length^(2k))")
```

Five checks, one per flag (evenness and weight together). PASS lines five to nine.

In [5], identity (I) and the factor V_k.

```
V = {1: sp.Integer(1), 2: -8 * (A1 ** 2 + 5 * H ** 2),
     3: 72 * (A1 ** 4 + 2 * A1 ** 2 * H ** 2 + 5 * H ** 4)}
identity_I = all(
    sp.expand(sp.diff(E[k][4, 4], A1) * A2 - 3 * A1 * (E[k][1, 1] - E[k][5, 5])) == 0
    for k in (1, 2, 3))
```

`V` holds the three factors V_k of Section 11.21. Identity (I): the time component depends on x_4 only through a_4', so its derivative is $\partial E^{x_4}{}_{x_4}/\partial a_4'$ times a_4'' (the chain rule; `sp.diff(..., A1)` differentiates with respect to the symbol a_4'), and this must equal $3a_4'(E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5})$ for every order.

```
check(identity_I, "d/dx4 E^x4_x4 = 3 a4' (E^x1_x1 - E^x5_x5) for k = 1, 2, 3 "
                  "(identity I)",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "bianchi_x4; Revision/gkd_lovelock/results/lovelock-report.json, checks "
             "k1_divergence_free, k2_divergence_free and k3_divergence_free")
factor_ok = all(sp.expand(E[k][4, 4] - E[k][0, 0] - 6 * (A1 ** 2 + H ** 2) * V[k]) == 0
                for k in (1, 2, 3))
check(factor_ok, "E^x4_x4 - E^x0_x0 = 6 (a4'^2 + H^2) V_k for k = 1, 2, 3",
      record="Revision/field_equations_a4/reports/python-a4-report.json, check "
             "linear_member_vacuum_factor")
```

The two checks: identity (I) holds exactly, and the factorisation $E^{x_4}{}_{x_4} - E^{x_0}{}_{x_0} = 6(a_4'^2 + H^2)V_k$ holds for $k = 1, 2, 3$. PASS lines ten and eleven.

In [6], the linear history (Figure 11c.1).

```
A = np.linspace(-3.0, 3.0, 601)  # values of A
LINEAR = {H: 1, A2: 0}  # H = 1 (units of H) and a4'' = 0; then a4' = A


def on_linear_history(expr):
    function = sp.lambdify(A1, expr.subs(LINEAR), "numpy")
    return np.broadcast_to(function(A), A.shape).astype(float)
```

`np.linspace(-3.0, 3.0, 601)` is the array of 601 equally spaced values from -3 to 3. On the linear history, with $H = 1$, $a_4' = A$ and $a_4'' = 0$. `on_linear_history` (docstring left out) substitutes these values, and `sp.lambdify` turns the remaining expression in `A1` into a numpy function that is evaluated on the whole array at once. `np.broadcast_to(..., A.shape)` makes a constant result (an expression without A) into an array of the same length, and `.astype(float)` into decimal numbers.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
symmetric = True
for ax, k in zip(axes, (1, 2, 3)):
    time_part = on_linear_history(E[k][4, 4])
    other_part = on_linear_history(E[k][1, 1])
    symmetric &= np.allclose(time_part, time_part[::-1]) and np.allclose(
        other_part, other_part[::-1])
```

One panel per order. `time_part` is ${E^{x_4}}_{x_4}$ and `other_part` the common value of the seven other components along the history. `[::-1]` reverses an array, which maps the value at A to the value at $-A$ (the grid is symmetric about 0), and `np.allclose` tests that two arrays agree up to rounding: both curves must be mirror-symmetric.

```
    ax.plot(A, time_part, label="$E^{x_4}{}_{x_4}$ (time)")
    ax.plot(A, other_part, "--", label="$E^{x_1}{}_{x_1} = E^{x_5}{}_{x_5} = "
                                       "E^{x_0}{}_{x_0}$")
    ax.axvline(0.0, color="0.5", lw=0.8)
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("$A = a_4^{\\prime}/H$")
    ax.set_ylabel(f"$E_{{({k})}}$ in units of $H^{{{2 * k}}}$")
    ax.set_title(f"order k = {k}")
    ax.legend(fontsize=7, loc="best")
```

The two curves (solid and dashed), grey lines at $A = 0$ and at the value 0, axis titles, title and legend (`loc="best"` lets matplotlib choose a free corner).

```
fig.tight_layout()
save_figure(fig, "components_linear_history",
            "The normalised Lovelock tensors $E_{(1)}$ (Einstein), $E_{(2)}$ and "
            ...)
check(symmetric, "on the linear history every curve is symmetric under A -> -A")
```

The figure is saved and the symmetry checked. Twelfth PASS line.

What Figure 11c.1 shows. Three panels, one per order, against A from -3 to 3 in units of H^{2k}. For $k = 1$ the time component $3A^2 + 21$ is a parabola opening upward from 21, and the other component $15 - 3A^2$ one opening downward from 15, crossing 0 at $A = \pm\sqrt{5}$. For $k = 2$ the curves open the other way, and the dashed one crosses 0 at $A = \pm 1$ ($12 + 168 - 180 = 0$). For $k = 3$ both grow like A^6. Every curve is a mirror image of itself about $A = 0$: deflating ($A > 0$) and inflating ($A < 0$) extra times give the same tensors.

In [7], the Lovelock scalars (Figure 11c.2).

```
L_TEXT = {k: re.sub(r"Derivative\[(\d)\]\[b4\]\[x4\]", r"B\1",
                    re.sub(r"Derivative\[(\d)\]\[a4\]\[x4\]", r"A\1",
                           tensors[f"L{k}"])).replace("b4[x4]", "B0")
          for k in (1, 2, 3)}
L = {k: sp.expand(sp.sympify(L_TEXT[k].replace("^", "**"),
                             locals={"A1": A1, "A2": A2, "H": H, "B0": B0, "B1": B1,
                                     "B2": B2}).subs(BACKGROUND))
     for k in (1, 2, 3)}  # the scalars at b4 = 1
fig, axes = plt.subplots(1, 3, figsize=(12.0, 3.8))
zeros = {}
```

The three scalars of the record (keys `L1` to `L3`, in Mathematica text, with b_4) are read as sympy expressions: `re.sub` writes the derivatives of a_4 as `A1`, `A2` and those of b_4 as `B1`, `B2`, `b4[x4]` becomes `B0`, and `.subs(BACKGROUND)` puts $b_4 = 1$. Three panels and an empty dictionary for the zeros.

```
for ax, k in zip(axes, (1, 2, 3)):
    values = on_linear_history(L[k])
    roots = sp.Poly(L[k].subs(H, 1), A1).nroots(n=15)  # every root, numerically
    zeros[k] = [float(r) for r in roots if r.is_real and r > 0]
    listed = ", ".join(f"{r:.4f}" for r in zeros[k])  # the zeros, 4 decimals
    say(f"L({k}) = {L[k]}")
    report(f"positive zero of L({k}) on the linear history (and minus it), A", listed)
```

For each order the scalar is evaluated along the history. `sp.Poly(..., A1).nroots(n=15)` computes all roots of the polynomial in A (with $H = 1$) numerically to 15 digits, including complex ones; the comprehension keeps the real positive roots. The output lines print each scalar and its positive zero: $A = 2.6458$, 1.2207 and 0.6010 (Section 11.23 derives the first two exactly: $\sqrt{7}$ and $\sqrt{2\sqrt{39} - 11}$).

```
    ax.plot(A, values, color="tab:green")
    for r in zeros[k]:
        ax.plot([r, -r], [0.0, 0.0], "o", color="black", markersize=4)
    ax.axhline(0.0, color="0.5", lw=0.8)
    ax.axvline(0.0, color="0.5", lw=0.8)
    ax.set_xlabel("$A = a_4^{\\prime}/H$")
    ax.set_ylabel(f"$L_{{({k})}}$ in units of $H^{{{2 * k}}}$")
    ax.set_title(f"Lovelock scalar of order {k}")
```

The green curve, black dots at the zero and at its mirror image, grey lines, axis titles and title.

```
    if k == 3:  # a magnified view of the middle, where the zeros of L(3) lie
        zoom = ax.inset_axes([0.30, 0.42, 0.42, 0.42])  # [left, bottom, width, height]
        near = np.abs(A) <= 1.0  # the values of A between -1 and 1
        zoom.plot(A[near], values[near], color="tab:green")
        zoom.axhline(0.0, color="0.5", lw=0.8)
        for r in zeros[k]:
            zoom.plot([r, -r], [0.0, 0.0], "o", color="black", markersize=3)
        zoom.tick_params(labelsize=6)
        zoom.set_title("magnified: $-1 \\leq A \\leq 1$", fontsize=7)
```

For the third order the scale of the whole panel (millions) hides the dip near $A = 0$, so a small panel inside the big one (`ax.inset_axes`, placed by its left edge, bottom edge, width and height as fractions of the big panel) shows the part $-1 \leq A \leq 1$. `near` is a true/false array, and `A[near]` keeps the values where it is true.

```
fig.tight_layout()
save_figure(fig, "lovelock_scalars",
            "The Lovelock scalars $L_{(1)} = 2R = 12 a_4^{\\prime 2} - 84 H^2$, "
            ...)
check(all(len(zeros[k]) == 1 for k in (1, 2, 3))
      and abs(zeros[1][0] - float(sp.sqrt(7))) < 1e-12
      and L[1].subs({A1: 0, H: 1}) == -84 and L[2].subs({A1: 0, H: 1}) == 3360
      and L[3].subs({A1: 0, H: 1}) == -40320,
      "each scalar has one positive zero, L(1) at A = sqrt(7); at A = 0 the scalars "
      "are -84, 3360, -40320")
```

The caption written into the figure includes the computed zeros (the f-strings of the caption insert them, rounded to three decimals). The check: exactly one positive zero for each order, the first at $\sqrt{7}$ to 12 digits, and the values -84, 3360, -40320 at $A = 0$. Thirteenth PASS line.

What Figure 11c.2 shows. $L_{(1)} = 12A^2 - 84$ is a parabola with its lowest point -84 at $A = 0$ and zeros at $\pm\sqrt{7}$. $L_{(2)}$ opens downward from its highest point 3360 and crosses 0 at ± 1.221. $L_{(3)}$ has a shallow dip to -40320 at $A = 0$, visible only in the magnified inset, crosses 0 at ± 0.601 and then grows to about 4×10^6 at $A = \pm 3$.

In [8], the test history (Figure 11c.3).

```
t = sp.Symbol("t")  # t = H x4 (the time in units of 1/H); H = 1 below
history = t + sp.log(sp.cosh(t)) / 2
first = sp.diff(history, t)  # a4' (with H = 1)
second = sp.diff(history, t, 2)  # a4''
check(sp.simplify(first - (1 + sp.tanh(t) / 2)) == 0
      and sp.simplify(second - sp.sech(t) ** 2 / 2) == 0,
      "the test history has a4' = 1 + tanh/2 > 0 and a4'' = sech^2/2 (H = 1)")
```

The illustrative history of Section 11.23 with $H = 1$, in the variable $t = Hx_4$. sympy differentiates it once and twice (`sp.diff(history, t, 2)`), and the check confirms the derivatives derived by hand: $1 + \frac{1}{2}\tanh t$ and $\frac{1}{2}\mathrm{sech}^2 t$. Fourteenth PASS line.

```
times = np.linspace(-6.0, 6.0, 1201)  # H x4 from -6 to 6
a4_values = sp.lambdify(t, history, "numpy")(times)
a1_values = sp.lambdify(t, first, "numpy")(times)
a2_values = sp.lambdify(t, second, "numpy")(times)
```

1201 times from -6 to 6, 0.01 apart, and the values of a_4, a_4' and a_4'' at all of them.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0))
axes[0].semilogy(times, np.exp(a4_values), label="space, $e^{a_4}$ (inflates)")
axes[0].semilogy(times, np.exp(-a4_values), "--",
                 label="extra times, $e^{-a_4}$ (deflate)")
axes[0].set_xlabel("time $H x_4$")
axes[0].set_ylabel("scale factor (logarithmic scale)")
axes[0].set_title("scale factors of the test history")
axes[0].legend(fontsize=8)
```

The left panel: the scale factors e^{a_4} of 3-space and e^{-a_4} of the extra times (without the common factor $\sin^{1/6} z$) on a logarithmic axis, where exponential growth or decay is a straight line.

```
axes[1].plot(times, a1_values, label="$a_4^{\\prime}/H$")
axes[1].plot(times, a2_values, "--", label="$a_4^{\\prime\\prime}/H^2$")
axes[1].set_xlabel("time $H x_4$")
axes[1].set_ylabel("value")
axes[1].set_title("its first and second derivatives")
axes[1].legend(fontsize=8)
fig.tight_layout()
```

```
save_figure(fig, "test_history",
            "The illustrative test history $a_4 = H x_4 + \\frac{1}{2} \\ln\\cosh H "
            ...)
```

The right panel: a_4'/H and a_4''/H^2; then the figure is saved.

What Figure 11c.3 shows. Left: the solid line (e^{a_4}) rises and the dashed line (e^{-a_4}) falls at all times; both are straight on the logarithmic axis far from $x_4 = 0$, with the slope $\pm 1/2$ in the past and $\pm 3/2$ in the future, and they bend around $x_4 = 0$. Right: a_4'/H rises like a step from $1/2$ to $3/2$, and a_4''/H^2 is a bump of height $1/2$ at $x_4 = 0$. The history is an illustration, not a solution of any field equation.

In [9], the components along the test history (Figure 11c.4).

```
def along_history(expr):
    function = sp.lambdify((A1, A2), expr.subs(H, 1), "numpy")
    return np.broadcast_to(function(a1_values, a2_values), times.shape).astype(float)
```

`along_history` (docstring left out) evaluates an expression in a_4' and a_4'' (with $H = 1$) at all 1201 times at once.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
middle = True
curves = {}
for ax, k in zip(axes, (1, 2, 3)):
    curves[k] = {h: along_history(E[k][h, h]) for h in (1, 4, 5, 0)}
    middle &= np.allclose(curves[k][0], (curves[k][1] + curves[k][5]) / 2,
                          rtol=1e-12, atol=1e-9)
```

For each order the four independent components are evaluated along the history and kept in `curves` (keys 0, 3, 4, 7 for x_1, x_4, x_5, x_0). The hidden component must equal the mean of the space and extra-time components at every time, up to rounding (`rtol` and `atol` are the allowed relative and absolute differences).

```
    for h, style in ((4, "-"), (1, "-"), (5, "--"), (0, ":")):
        ax.plot(times, curves[k][h], style,
                label=f"$E^{{x_{h}}}{{}}_{{x_{h}}}$")
    ax.set_xlabel("time $H x_4$")
    ax.set_ylabel(f"$E_{{({k})}}$ in units of $H^{{{2 * k}}}$")
    ax.set_title(f"order k = {k}")
    ax.legend(fontsize=7)
```

The four curves: time and space solid, extra times dashed, hidden dotted; the legend labels are built from the number `h + 1` of each coordinate.

```
fig.tight_layout()
save_figure(fig, "components_test_history",
            "The four independent components of $E_{(1)}$, $E_{(2)}$ and $E_{(3)}$ "
            ...)
check(middle, "along the test history E^x0_x0 is the mean of E^x1_x1 and E^x5_x5 "
              "at all 1201 times")
```

The figure is saved, and the check confirms identity (II) at all 1201 times. Fifteenth PASS line.

What Figure 11c.4 shows. In each panel the time component (blue) steps from one constant to another, for $k = 1$ from 21.75 to 27.75, because it depends on a_4' alone. The space (orange) and extra-time (dashed green) components step the other way and separate only near $x_4 = 0$, where $a_4'' \neq 0$: for $k = 1$ and $k = 3$ the space component lies above ($F_1, F_3 > 0$), for $k = 2$ below ($F_2 < 0$). The dotted hidden component runs exactly between them.

In [10], identity (I) numerically (Figure 11c.5).

```
step = times[1] - times[0]  # epsilon = 0.01
fig, axes = plt.subplots(1, 3, figsize=(12.0, 3.8))
worst = {}
for ax, k in zip(axes, (1, 2, 3)):
    left = np.gradient(curves[k][4], step)  # d/dx4 of the time component
    right = 3 * a1_values * (curves[k][1] - curves[k][5])
```

`step` is the spacing of the times, 0.01. `np.gradient(values, step)` estimates the derivative at every time by the central difference $(f(t+\epsilon) - f(t-\epsilon))/(2\epsilon)$ (at the two end points by a one-sided difference). `left` is the estimated derivative of the time component, `right` the right side of identity (I).

```
    inner = slice(1, -1)  # the two end points use a one-sided difference
    worst[k] = float(np.max(np.abs(left[inner] - right[inner]))
                     / np.max(np.abs(right)))
    report(f"order {k}: largest relative deviation of the finite difference",
           f"{worst[k]:.1e}")
```

`slice(1, -1)` leaves out the first and the last time, where the less accurate one-sided difference is used. The largest difference between the two sides, divided by the largest size of the right side, is the relative deviation; the RESULT lines print 3.8×10^{-5}, 4.0×10^{-5} and 4.5×10^{-5} (the format `:.1e`).

```
    ax.plot(times, right, color="tab:blue", lw=3, alpha=0.5,
            label="$3 a_4^{\\prime} (E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5})$")
    ax.plot(times, left, "k:", label="$d E^{x_4}{}_{x_4} / d x_4$ (finite difference)")
    ax.set_xlabel("time $H x_4$")
    ax.set_ylabel(f"units of $H^{{{2 * k + 1}}}$")
    ax.set_title(f"identity (I), order k = {k}")
    ax.legend(fontsize=7)
fig.tight_layout()
```

Both sides are drawn: the right side as a thick, half-transparent blue line (`lw=3`, `alpha=0.5`) and the finite difference as a dotted black line on top of it. One derivative with respect to x_4 adds one power of H, so the unit is H^{2k+1}.

```
largest = max(worst.values())  # the largest relative deviation of the three orders
power = int(np.floor(np.log10(largest)))  # its power of ten
deviation_text = f"${largest / 10 ** power:.1f} \\times 10^{{{power}}}$"
```

```
save_figure(fig, "conservation_identity",
            "Identity (I), the conservation identity along the time, checked "
            ...)
check(all(value < 1e-4 for value in worst.values()),
      "the finite-difference derivative agrees with identity (I) to 1e-4")
```

The largest deviation of the three orders is written into the caption in the form 4.5×10^{-5}: `np.log10` is the logarithm to the base 10, and `np.floor` rounds it down, which gives the power of ten. The check requires every relative deviation to be below 10^{-4}. Sixteenth PASS line.

What Figure 11c.5 shows. In each panel the dotted black line lies on the thick blue line: a bump at $x_4 = 0$, zero far away, where $a_4'' = 0$ and the time component is constant. The sign of the bump follows F_k (positive for $k = 1, 3$, negative for $k = 2$), times the positive $3a_4'a_4''$.

In [11], the quoted records once more.

```
def verdicts(path):
    data = json.loads(repository_file(path).read_text(encoding="utf-8"))
    checks = data["checks"]
    if isinstance(checks, dict):  # name -> {"passed": true or false, ...}
        return {name: "PASS" if entry["passed"] else "FAIL"
                for name, entry in checks.items()}
    return {entry["name"]: entry["verdict"] for entry in checks}  # a list of entries
```

The helper of Notebook 11a, In [16] (Section 11.9): for one report, a dictionary from the name of each check to PASS or FAIL.

```
QUOTED = {  # report -> the checks that this notebook quotes from it
    "Revision/field_equations_a4/reports/python-a4-report.json": [
        "json_lovelock_components", "evolution_factorises", "bianchi_x4",
        "linear_member_vacuum_factor"],
    "Revision/gkd_lovelock/results/lovelock-report.json": [
        "k1_divergence_free", "k2_divergence_free", "k3_divergence_free"],
}
```

The checks quoted in the notebook, by report (here with the full paths, because the two reports lie in different folders).

```
for path, names in QUOTED.items():
    found = verdicts(path)
    for name in names:
        say(f"{path.split('/')[-1]}: {name} {found.get(name, 'MISSING')}")
    check(all(found.get(name) == "PASS" for name in names),
          f"the {len(names)} checks quoted from {path.split('/')[-1]} are there and "
          "PASS", record=f"{path}, checks " + ", ".join(names))
```

Each quoted check is printed with its verdict (`path.split('/')[-1]` is the file name, the last part of the path) and must be present with PASS. The notebook asks only for these names, not for the number of checks of the report, so it stays valid when the report gains checks. PASS lines seventeen and eighteen.

```
general = a4_record["generalSource"]  # a4-equations.json, read in section 5
stated = (general["algebraic_condition"]["input"] == "p3 + pt == 2*p0"
          and "rho' = -3 a4' (p3 - p_t)" in general["conservation_reduced"]
          and "p3 + p_t = 2 p0" in general["conservation_reduced"]
          and "d/dx4 (constraint) = 3 a4' (evolution)"
          in general["constraint_propagation"])
check(stated, "a4-equations.json states rho' = -3 a4' (p3 - p_t), p3 + p_t = 2 p0 and "
              "the constraint propagation, as quoted",
      record="Revision/field_equations_a4/a4-equations.json, generalSource: "
             "conservation_reduced, algebraic_condition, constraint_propagation")
```

The record of the field equations must state the three consequences quoted in Section 11.22: the algebraic condition $p_3 + p_t = 2p_0$ (in Wolfram's notation), the reduced conservation law $\rho' = -3a_4'(p_3 - p_t)$, and the constraint propagation. Nineteenth PASS line.

In [12], the end.

```
figure_files = [f"11c_{k}_{name}.png" for k, name in enumerate(
    ["components_linear_history", "lovelock_scalars", "test_history",
     "components_test_history", "conservation_identity"], 1)]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_files),
      "all five figure files of the notebook exist")
all_checks_passed()
```

The five figure files must exist (twentieth PASS line), and the last line is ALL 20 CHECKS PASSED (notebook 11c). The 20 checks are: 2 in In [2], 2 in In [3], 5 in In [4], 2 in In [5], 1 each in In [6] to In [10], 3 in In [11] and 1 in In [12].

11.28 What we proved, what we computed, what we assumed

PROVED in this chapter by derivations written out line by line (each confirmed by a check of a notebook and, where named, of a Revision record):

- **The generalized Kronecker delta** (Sections 11.3 and 11.4): the author's determinant $\det[\delta(l_i, u_j)]$ is 0 when a label repeats in either list or a lower label is missing above, and otherwise the sign of the permutation that carries the lower list into the upper list; so the rule GKD of the Revision program is exactly the author's definition (Notebook 11a, In [2] to In [6]; records `Revision/gkd_lovelock/results/wolfram-gkd-report.json`, checks `gkd_equals_kdelta_exhaustive_length_1` to `_length_3`, which compare the Rust function GKD with the literal determinant, and `gkd-selftest.json`; the literal determinant itself is checked in `python-lovelock-report.json`, checks `gkd_examples` and `gkd_literal_equa ls_cofactor_expansion`, the second of which compares it with an independent cofactor expansion). The number of nonzero values $8!/(8-p)! \cdot p!$, half of them $+1$ and half -1 for $p \geq 2$, with the counts 8; 56 and 56; 1008 and 1008; 20160 and 20160 for $p = 1$ to 4 (In [6], In [7], In [9]; record `wolfram-gkd-report.json`, entry `gkdComparison`). Every delta with nine or more labels vanishes in eight dimensions (pigeonhole), so $P_{(4)} = 0$ and Lovelock's sum stops at order 3 (In [10]; record `lovelock-report.json`, check `k4_tensor_vanishes`). The rule needs $p(p-1)/2$ comparisons instead of the $p!$ products of the determinant. The birthday

probability $r_p = 8!/((8-p)!\,8^p)$ of a nonzero rearranged random pair (with the assumed rules of probability).

- **The author's second route** (Section 11.5): a determinant identity for the Levi-Civita symbol; raising all labels of the Levi-Civita tensor of a metric gives the factor $\sqrt{|\det g|}/\det g$, so the product of two Levi-Civita tensors is the sign of $\det g$ times the product of two symbols; for the author's metric $\det g = \cos^2 z > 0$ (record `python-lovelock-report.json`, check `sqrt_abs_det_g`), the sign is $+1$, and the author's second route gives the generalized delta without an extra sign.

- **The curvature that the sums use** (Section 11.10): the mixed components $R^{x_4 k}{}_{x_0 k} = \sigma_k H a_4' \cot z$ carry the sign $\sigma_k = \pm 1$ of inflation or deflation, not of the space-like or time-like character, and $R^{x_4}{}_{x_0} = 0$ because three inflating and three deflating directions cancel, on the background $b_4 = 1$ (with a changing b_4, $R^{x_4}{}_{x_0} = \frac{3}{2} H \cot z\, b_4'/b_4$, record `curvature.json`); every nonzero $R^{ab}{}_{cd}$ has the weight 2.

- **Laplace's rule** for determinants of every size, and **the expansion of every Lovelock tensor** along the column of its free upper label, $P_{(k)} = \delta\, L_{(k)} - 2k\, Y_{(k)}$ (Section 11.13; Notebook 11b, In [21]).

- **For every metric** (Sections 11.14 and 11.15): $L_{(1)} = 2R$, $P_{(1)} = -4G$, so $E_{(1)} = G$; the trace identity $\sum_h P_{(k)}{}^h{}_h = (8-2k) L_{(k)}$; $L_{(2)} = 4\,\mathrm{GB}$ and $P_{(2)} = -8\mathcal{H}$ with Lanczos's Gauss-Bonnet tensor $\mathcal{H}$, so $E_{(2)} = \mathcal{H}$ (Notebook 11b, In [18], In [19], In [22]; records `lovelock-report.json`, checks `k1_equals_minus_4_einstein`, `k1_trace_identity` to `k3_trace_identity`, and `python-lovelock-report.json`, checks `L1_equals_2R`, `k2_equals_minus_8_gauss_bonnet`, `L2_equals_4_gauss_bonnet`).

- **The structure of the components for the author's metric** (Section 11.21), for every history $a_4(x_4)$ and every order: only the diagonal is nonzero (in particular no x_4-x_0 component, by the relabelling that exchanges 3-space and the extra times), on the background $b_4 = 1$; with a changing b_4 the two mixed components are proportional to b_4'; the three space components are equal, and so are the three extra-time components; a_4'' enters at most linearly; the time component is a polynomial in $a_4'^2$ and H^2 alone (the constraint); the extra-time component is the space component with a_4'' replaced by $-a_4''$, so their difference is $a_4'' F_k(a_4')$ with $F_1 = 2$, $F_2 = -16(3a_4'^2 + 5H^2)$, $F_3 = 144(5a_4'^4 + 6a_4'^2 H^2 + 5H^4)$ (the evolution factor; record `Revision /field_equations_a4/reports/python-a4-report.json`, check `evolution_factorises`); the hidden component contains no a_4''; every component is even in a_4' (by reversing the time), so the tensors do not distinguish deflating from inflating extra times; order k has the weight $2k$; and $E^{x_4}{}_{x_4} - E^{x_0}{}_{x_0} = 6(a_4'^2 + H^2) V_k$ (record check `linear_member_vacuum_factor`). Notebook 11c, In [2] to In [5], confirms each statement with exact algebra.

- **The two conservation identities** (Section 11.22), from zero divergence: $\frac{d}{dx_4} E^{x_4}{}_{x_4} = 3a_4'(E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5})$ (I) and $E^{x_0}{}_{x_0} = \frac{1}{2}(E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5})$ (II), and the constraint propagation $\partial E^{x_4}{}_{x_4}/\partial a_4' = 3a_4' F_k$ (record `python-a4-report.json`, check `bianchi_x4`; Notebook 11c, In [4] and In [5]).

- **Along the linear history** (Section 11.23): only two different components per order, even in the slope A; the scalars $L_{(1)}$ and $L_{(2)}$ vanish exactly at $A = \sqrt{7}$ and $A = \sqrt{2\sqrt{39} - 11}$; the derivatives of the illustrative test history.

PROVED by exact computation (no rounding; by the Revision's programs, recomputed by the notebooks):

- GKD equals the author's determinant for every one of the 266,304 pairs of lengths 1 to 3 (Notebook 11a, In [6]; the Wolfram report, checks `gkd_equals_kdelta_exhaustive_length_1` to `_length_3`, and the Rust self-test record `gkd-selftest.json`, whose exhaustive part also covers these lengths; on the same pairs the sympy report's check `gkd_literal_equals_cofactor_expansion` confirms the determinant side: the literal determinant equals an independent cofactor expansion) and for every one of the 16,777,216 pairs of length 4 (the Rust self-test of the record `gkd-selftest.json`, read and checked by Notebook 11a, In [14]; the run itself takes 7 to 13 minutes).
- The 25 nonzero Christoffel symbols, the 156 nonzero components $R^{ab}{}_{cd}$ and the Ricci scalar $R = 6a_4'^2 - 42H^2$ (Notebook 11b, In [6] to In [8]; records `curvature.json` and `python-lovelock-report.json`, check `rust_riemann_agrees`).
- All 3×64 components of $P_{(1)}, P_{(2)}, P_{(3)}$ and the scalars $L_{(1)}, L_{(2)}, L_{(3)}$, computed by the Rust program and again by the notebook's own GKD sum (Notebook 11b, In [3], In [4], In [14], In [15]; record `lovelock-tensors.json`, written again byte for byte; `python-lovelock-report.json`, checks `rust_k1_mixed_components_agree` to `rust_k3_mixed_components_agree` and `rust_L1_agrees` to `rust_L3_agrees`); the counters of the sums, 696, 32,640 and 495,360 GKD calls (In [14]; record `lovelock-report.json`, field `counters`); the literal sums of orders 1 and 2 without any skipping, with 10,140 and 1,581,840 GKD calls (In [16]; checks `k1_unpruned_literal_sum_agrees` and `k2_unpruned_literal_sum_agrees`); the cubic identity $L_{(3)} = 8(2T_1 + 8T_2 + \cdots + T_8)$ for the author's metric (In [20]; check `L3_equals_8_cubic_lovelock_density`); zero divergence and symmetry of all three tensors (record `lovelock-report.json`, checks `k1_divergence_free` to `k3_symmetric`); the 19 checks of the Rust program, the 49 of the sympy verification and the 29 of the Wolfram verification, none failed (In [3] and In [24]).
- The components $E_{(1)}, E_{(2)}, E_{(3)}$ of Section 11.20 equal those recorded for the field equations of a_4 (Notebook 11c, In [3]; `Revision/field_equations_a4/reports/python-a4-report.json`, check `json_lovelock_components`).

COMPUTED (numerical, with the measured accuracy):

- The Rust program's literal unpruned sums at a numerical point agree with the exact tensors to the relative rounding errors 1.40×10^{-15} ($k = 1$) and 4.41×10^{-14} ($k = 2$) (record `lovelock-report.json`, checks `k1_brute_force_numeric` and `k2_brute_force_numeric`; Notebook 11b, In [4], requires both below 10^{-10}).
- GKD equals the determinant on 12,000 pseudo-random pairs of lengths 4 to 9 (Notebook 11a, In [10]) and on 200,000 pairs of each length 5 to 9 (record `gkd-selftest.json`); the numbers of nonzero values lie within 1.6 standard deviations of the birthday expectation (In [11] and In [15]).
- The rule GKD is more than ten times faster than sympy's determinant for 200 pairs of length 7 (In [12]; the times themselves are not printed).
- The positive zero $A \approx 0.6010$ of $L_{(3)}$ on the linear history (Notebook 11c, In [7], 15 digits).

- Identity (I) along the illustrative test history by finite differences, to the relative deviations 3.8×10^{-5}, 4.0×10^{-5}, 4.5×10^{-5} for $k = 1, 2, 3$, the size of the finite-difference error (Notebook 11c, In [10]).

ASSUMED (used, not proved here):

- The author's metric, as given in the Revision record, and the curvature convention of Misner, Thorne and Wheeler (Chapter 3).
- Lovelock's theorem in general: for every metric the tensors $P_{(k)}$ are divergence-free and symmetric, and they are the only such tensors built from the metric and its first and second derivatives (D. Lovelock, J. Math. Phys. 12, 498 (1971)). For the author's metric divergence and symmetry are not assumed but PROVED by exact computation.
- The Chern-Gauss-Bonnet theorem (a classical result), quoted in Section 11.11 only to explain the name Euler density of $L_{(4)}$; nothing in this chapter depends on it, and $P_{(4)} = 0$ is proved there by the pigeonhole principle.
- The general formula of the cubic Lovelock density (a classical result): the record derived its coefficients from literal sums on random curvature tensors, and checked the identity exactly for the author's metric; we do not derive it by hand.
- The tensor rule for a change of coordinates (Section 3.14), used for the time reversal and the relabelling; the rules of determinants of Section 1.20 (rule 1 for matrices larger than 2×2 is a standard theorem); that two polynomials that agree for all values of their variables have the same coefficients; the rules of probability used for the random tests (independent chances multiply; the expectation and the standard deviation of a count).

HYPOTHESIS. None: this chapter is geometry of the given metric.

OPEN. Whether nature has the couplings α_2 and α_3 of the Gauss-Bonnet and the cubic tensor, and which sign of a_4' is realised, are not decided by the tensors computed here; Chapter 12 studies the field equations with the couplings as free parameters and shows that the sign of the deflation is set by the source and the initial data, not by gravity. The symmetry $A \to -A$ of this chapter is NOT the pairing of universes of masses $+m$ and $-m$ (Chapters 18 to 20); nothing in this chapter concerns the creation of universes or matter and antimatter.

11.29 Exercises

Exercise 1. Use the rule of Section 11.3 to compute (a) the delta with the lower list (x_1, x_2, x_3, x_4) and the upper list (x_3, x_1, x_4, x_2); (b) the delta with the lower list (x_5, x_1, x_2) and the upper list (x_1, x_2, x_5), and check (b) with the Leibniz formula; (c) the delta with the lower list (x_1, x_2, x_0) and the upper list (x_1, x_2, x_3).

Answer. (a) The places of x_1, x_2, x_3, x_4 in the upper list are 2, 4, 1, 3, so $\sigma = (2, 4, 1, 3)$. Its inversions are the pairs of entries $(2, 1)$, $(4, 1)$ and $(4, 3)$: three, an odd number, so the value is -1. (b) The places of x_5, x_1, x_2 in the upper list (x_1, x_2, x_5) are 3, 1, 2: $\sigma = (3, 1, 2)$ with the two inversions $(3, 1)$ and $(3, 2)$, so the value is $+1$. The matrix has the rows $(0, 0, 1)$ (x_5 is the third

upper label), $(1,0,0)$ and $(0,1,0)$; in the Leibniz formula only the permutation $\pi = (3,1,2)$ picks three ones, and its sign is $+1$, so $\det = +1$. (c) The lower label x_0 is not among the upper labels: row 3 is all zeros, case (c), and the value is 0.

Exercise 2. (a) How many of the 8^{10} pairs of index lists of length 5 have the value $+1$, how many -1, how many 0? (b) For the Rust self-test of length 6 (100,000 rearranged and 100,000 independent pairs), compute the expected number of nonzero values and its standard deviation, and compare with the record's count 7812.

Answer. (a) By the formula of Section 11.4,

$$N_{\neq 0}(5) = 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 5! = 6720 \cdot 120 = 806400,$$

half of each sign: 403,200 values $+1$ and 403,200 values -1. The zeros are the rest:

$$8^{10} - 806400 = 1073741824 - 806400 = 1072935424.$$

(b) The two chances are

$$r_6 = \frac{8!}{2! \cdot 8^6} = \frac{20160}{262144} = 0.0769043, \qquad q_6 = \frac{N_{\neq 0}(6)}{8^{12}} = \frac{20160 \cdot 720}{68719476736} = 0.000211.$$

The expectation is $100000 \cdot (0.0769043 + 0.000211) = 7711.6$, and the standard deviation is

$$\begin{aligned} &\sqrt{100000 \cdot 0.0769043 \cdot 0.9230957 + 100000 \cdot 0.000211 \cdot 0.999789} \\ &\quad = \sqrt{7099.0 + 21.1} = \sqrt{7120.1} = 84.4. \end{aligned}$$

The record's 7812 lies $(7812 - 7711.6)/84.4 = 1.19$ standard deviations above the expectation, as Notebook 11a prints (In [15]).

Exercise 3. Compute the determinant of the matrix with the rows $(2,1,0)$, $(0,1,3)$, $(1,0,1)$ by Laplace's rule along its second column, and compare with the six-term formula, which gives 5.

Answer. The second column holds $M_{12} = 1$, $M_{22} = 1$, $M_{32} = 0$, with the signs $(-1)^{1+2} = -1$, $(-1)^{2+2} = +1$, $(-1)^{3+2} = -1$. The minor $M^{(12)}$ (row 1 and column 2 removed) has the rows $(0,3)$ and $(1,1)$, determinant $0 \cdot 1 - 3 \cdot 1 = -3$; the minor $M^{(22)}$ has the rows $(2,0)$ and $(1,1)$, determinant 2. So $\det M = -1 \cdot (-3) + 1 \cdot 2 - 0 = 3 + 2 = 5$, the same value.

Exercise 4. In a space of n dimensions, which orders k of the Lovelock tensors $P_{(k)}$ can be nonzero, for $n = 4, 5, 6, 8, 10$? Which scalars $L_{(k)}$ can be nonzero in eight dimensions?

Answer. The delta of $P_{(k)}$ has $2k + 1$ labels in each list, so $P_{(k)}$ vanishes when $2k + 1 > n$ (pigeonhole, Section 11.4). Hence: $n = 4$: $k = 1$ only (Einstein); $n = 5$ and $n = 6$: $k = 1, 2$; $n = 8$: $k = 1, 2, 3$; $n = 10$: $k = 1, 2, 3, 4$. For even $n = 2m$ the highest order is $m - 1$, as in the formula of Section 11.11. The scalar $L_{(k)}$ has $2k$ labels per list, so in eight dimensions $L_{(1)}$ to $L_{(4)}$ can be nonzero: $L_{(4)}$ is the Euler density, whose tensor $P_{(4)}$ vanishes; for the author's metric it is $-663552H^2a_4'^6 - 442368H^4a_4'^4 - 663552H^6a_4'^2$ (Notebook 11b, In [3]).

Exercise 5. On the author's history $a_4 = Hx_4$ ($A = 1$): (a) write the Einstein tensor and $P_{(1)}$; (b) check the trace identity of order 1 with $L_{(1)}$.

Answer. (a) With $a_4' = H$ and $a_4'' = 0$ the components of Section 11.20 give $G^{x_1}{}_{x_1} = -3 + 15 = 12$, $G^{x_4}{}_{x_4} = 3 + 21 = 24$, $G^{x_5}{}_{x_5} = 12$, $G^{x_0}{}_{x_0} = 15 - 3 = 12$ (in units of H^2), so

$$G = \mathrm{diag}(12, 12, 12, 24, 12, 12, 12, 12)\, H^2,$$
$$P_{(1)} = -4G = \mathrm{diag}(-48, -48, -48, -96, -48, -48, -48, -48)\, H^2.$$

(b) The trace is $\sum_h P_{(1)}{}^h{}_h = -4(7 \cdot 12 + 24)H^2 = -432H^2$. With $L_{(1)} = 12a_4'^2 - 84H^2 = -72H^2$, the identity gives $(8-2)L_{(1)} = 6 \cdot (-72)H^2 = -432H^2$: the same.

Exercise 6. On the same history $A = 1$: compute the four independent components of $E_{(2)}$, then $P_{(2)}$, $L_{(2)}$, and check the trace identity of order 2. What is special about the Gauss-Bonnet tensor on this history?

Answer. With $a_4' = H$, $a_4'' = 0$ (units of H^4): $E_{(2)}{}^{x_1}{}_{x_1} = 12 + 168 - 180 = 0$; $E_{(2)}{}^{x_4}{}_{x_4} = -36 - 120 - 420 = -576$; $E_{(2)}{}^{x_5}{}_{x_5} = 12 + 168 - 180 = 0$; $E_{(2)}{}^{x_0}{}_{x_0} = 12 + 168 - 180 = 0$. So $P_{(2)} = -8E_{(2)} = \mathrm{diag}(0, 0, 0, 4608, 0, 0, 0, 0)H^4$ and its trace is $4608H^4$. $L_{(2)} = -96 - 2112 + 3360 = 1152$ (units of H^4), and $(8-4)L_{(2)} = 4608$: the identity holds. On the author's history the Gauss-Bonnet tensor $E_{(2)}$ has only its time component; all seven other components vanish, because $12A^4 + 168A^2 - 180 = 12(A^2 - 1)(A^2 + 15)$ is zero at $A = 1$ (multiply out: $12(A^4 + 14A^2 - 15)$).

Exercise 7. Check identity (I) of Section 11.22 for $k = 3$ by hand, from the components of Section 11.20.

Answer. The time component $E_{(3)}{}^{x_4}{}_{x_4} = 360a_4'^6 + 648a_4'^4H^2 + 1080a_4'^2H^4 + 2520H^6$ depends on x_4 only through a_4', so its derivative is $\frac{d}{dx_4}E^{x_4}{}_{x_4} = (2160a_4'^5 + 2592a_4'^3H^2 + 2160a_4'H^4)\, a_4''$ (the chain rule; $6 \cdot 360 = 2160$, $4 \cdot 648 = 2592$, $2 \cdot 1080 = 2160$). The difference of the space and the extra-time component is $E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5} = 720a_4'^4a_4'' + 864a_4'^2a_4''H^2 + 720a_4''H^4$ (every term without a_4'' cancels, every term with a_4'' doubles). Then $3a_4'(E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5}) = (2160a_4'^5 + 2592a_4'^3H^2 + 2160a_4'H^4)a_4''$, which is the same. So (I) holds, and $F_3 = 720a_4'^4 + 864a_4'^2H^2 + 720H^4 = 144(5a_4'^4 + 6a_4'^2H^2 + 5H^4)$.

Exercise 8. Check identity (II) for $k = 3$, and explain without any computation why $E^{x_0}{}_{x_0}$ contains no $\cot z$ for every order.

Answer. $\frac{1}{2}(E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5})$: the terms with a_4'' have opposite signs in the two components and cancel; the others are equal, so the mean is $-72a_4'^6 - 648a_4'^4H^2 - 1944a_4'^2H^4 + 360H^6$, which is $E_{(3)}{}^{x_0}{}_{x_0}$. Without computation: in $P^{x_0}{}_{x_0}$ the label x_0 already stands in both lists of the delta, so no curvature factor may carry x_0 (case a or b); every entry with $\cot z$ or $\tan z$ is a mixed entry and carries x_0 (Section 11.21, tool iii); so none occurs, and the factors are plane curvatures among $x_1, \ldots, x_7$, which contain no z.

Exercise 9. Show that $L_{(3)}$ has exactly one positive zero on the linear history, and locate it between two decimals.

Answer. With $u = A^2$ and $H = 1$, $L_{(3)} = 1152u^3 + 31104u^2 + 100224u - 40320 = 1152\, f(u)$ with $f(u) = u^3 + 27u^2 + 87u - 35$ (divide by 1152: $31104/1152 = 27$, $100224/1152 = 87$, $40320/1152 = 35$). For $u > 0$ the derivative $f'(u) = 3u^2 + 54u + 87$ is positive, so f increases; $f(0) = -35 < 0$ and $f(1) = 1 + 27 + 87 - 35 = 80 > 0$, so f has exactly one zero for $u > 0$, between 0 and 1. Further, $f(0.3612) = 0.04712 + 3.52257 + 31.42440 - 35 = -0.0059$ and $f(0.3613) = 0.04716 + 3.52452 + 31.43310 - 35 = +0.0048$, so the zero lies between $u = 0.3612$ and 0.3613, and $A = \sqrt{u}$ between $\sqrt{0.3612} \approx 0.60100$ and $\sqrt{0.3613} \approx 0.60108$, in agreement with $A = 0.6010$ of Notebook 11c (In [7]).

Exercise 10. (a) In two dimensions with the metric $g = \mathrm{diag}(1, -1)$ (one space-like, one time-like direction), compute $\varepsilon_{12}\varepsilon^{12}$ with the rule of Section 11.5. (b) The same with $g = \mathrm{diag}(-1, -1)$. (c) What decides the sign, and what is it for the author's metric?

Answer. (a) $|\det g| = 1$, so $\varepsilon_{12} = [12] = 1$, and $\varepsilon^{12} = g^{11}g^{22}\varepsilon_{12} = 1 \cdot (-1) \cdot 1 = -1$; the product is -1, the sign of $\det g = -1$. (b) $\varepsilon^{12} = (-1)(-1) \cdot 1 = +1$, the product is $+1$, the sign of $\det g = +1$. (c) The sign of $\det g$, which for a diagonal metric is (-1) to the number of negative entries: the number of time-like directions. The author's metric has four, $\det g = \cos^2 z > 0$, and the sign is $+1$.

Exercise 11. For the illustrative test history of Section 11.23, compute a_4' and a_4'' at $x_4 = 0$, and the four components of $E_{(1)}$ there. Check identity (II).

Answer. $a_4'(0) = H(1 + \frac{1}{2}\tanh 0) = H$ and $a_4''(0) = \frac{H^2}{2}\mathrm{sech}^2 0 = \frac{H^2}{2}$. Then (units of H^2): $E^{x_1}{}_{x_1} = -3+\frac{1}{2}+15 = 12.5$, $E^{x_4}{}_{x_4} = 3+21 = 24$, $E^{x_5}{}_{x_5} = -3-\frac{1}{2}+15 = 11.5$, $E^{x_0}{}_{x_0} = -3+15 = 12$. Identity (II): $\frac{1}{2}(12.5 + 11.5) = 12 = E^{x_0}{}_{x_0}$. The space and extra-time components differ by $a_4''F_1 = \frac{1}{2} \cdot 2 = 1$, as in Figure 11c.4 at $x_4 = 0$.

12 The field equations for a4: Einstein and Einstein-Lovelock

The author's metric contains exactly one unknown function, $a_4(x_4)$. This chapter derives, step by step, the equations that gravity imposes on that function: first by hand, as far as the algebra stays short, and then completely with four notebooks. We will find that the equations split into a constraint, an evolution equation, a hidden-direction equation and conditions on the matter; that no empty universe of this form exists; that the exponentially deflating history needs a source of a very unusual kind; that the equations do not care whether the extra times deflate or inflate; and that a condensate of the commuting field dirac16complex00 can be such a source exactly.

12.1 What this chapter does, and why

In Chapter 3 we met the author's metric: a rule that turns small steps $dx_0, \ldots, dx_7$ into squared lengths. Three of its directions, x_1, x_2, x_3, are ordinary 3-space; x_4 is the time; x_5, x_6, x_7 are the three **extra times**, which behave like time; x_0 is the **hidden** space direction. Lengths along 3-space are multiplied by e^{a_4} and lengths along the extra times by e^{-a_4}. When a_4 grows with the time x_4, 3-space inflates and the extra times **deflate exponentially**. That is the author's picture of the primordial universe.

A metric is not free. Einstein's great idea is that matter tells space how to curve: the curvature of the metric, collected into a tensor, equals a constant times the energy and momentum of the matter. The equations that say this are called the **field equations of gravity**. For the author's metric they become equations for the single function $a_4(x_4)$, and they answer questions such as:

- Can the author's metric exist with no matter at all? (No: Section 12.11.)
- If a_4 is given, what matter must be present? (The **required source**: Sections 12.8 to 12.12.)
- If the matter is given, how does a_4 evolve? (The evolution equation: Sections 12.9 and 12.21.)
- Do the equations prefer extra times that deflate? (No; deflation is a choice of sign: Section 12.12.)

- Can a field of this theory be the required source exactly, with the extra times deflating? (Yes, in three exact examples of the commuting field dirac16complex00, computed by this book: Section 12.26.)

We use the most general gravity theory that keeps the field equations of second order in eight dimensions, **Einstein-Lovelock gravity**, and its special case, Einstein gravity. Every formula comes from the Revision record (the folders `Revision/field_equations_a4`, `Revision/gkd_lovelock` and `Revision/lead_checks` of the repository) or from the book's own notebooks, which reproduce the record wherever the two overlap and say so in a PASS line. The four notebooks of the chapter are: Notebook 12a, which derives all the equations from the metric with exact computer algebra; Notebook 12b, which computes the source that the linear history $a_4 = AHx_4 + a_0$ requires; Notebook 12c, which integrates the evolution equation for prescribed sources; and Notebook 12d, which builds exact solutions of the coupled equations of gravity and of the field dirac16complex00, using the author's eight real 16 by 16 gamma matrices.

12.2 The words and symbols of this chapter

Each word below is defined in plain terms; the later sections make every definition precise with formulas.

- **Coordinates** $x_0, \ldots, x_7$: the eight numbers that name a point, in the author's order. x_1, x_2, x_3: ordinary 3-space. x_4: the time. x_5, x_6, x_7: the three extra times (time-like, they deflate exponentially). x_0: the hidden direction. In the Python code the positions 0 to 7 of a list stand for x_0 to x_7; so position 3 is the time x_4 and position 7 the hidden x_0.
- **Hidden angle** $z = 6Hx_0$: the metric depends on x_0 only through z, which runs from 0 to $\pi/2$ (the **patch**). $H > 0$ is a constant of the author's metric with the unit of an inverse length.
- **Prime**: $a_4' = da_4/dx_4$ and $a_4'' = d^2a_4/dx_4^2$. The code writes them as the symbols `ad1` and `ad2`.
- **Metric** $g_{\mu\nu}$: the table of numbers that turns coordinate steps into squared lengths, $ds^2 = \sum_{\mu,\nu} g_{\mu\nu}\, dx_\mu dx_\nu$. The author's metric is **diagonal**: only $g_{11}, g_{22}, \ldots, g_{00}$ are not zero. Its inverse $g^{\mu\nu}$ is then diagonal too, with $g^{\mu\mu} = 1/g_{\mu\mu}$.
- **Signature**: the signs of the diagonal entries, here $(+,+,+,+,-,-,-,-)$: four space-like and four time-like directions, written (4,4).
- **Scale factor**: the number by which lengths along a direction are multiplied; for a diagonal metric it is $\sqrt{|g_{\mu\mu}|}$.
- **Index up, index down**: a tensor is a table of numbers labelled by indices; an upper index and a lower index transform in opposite ways. $T^\mu{}_\nu$ has one of each and is called **mixed**. The diagonal metric moves an index up or down by multiplying by $g^{\mu\mu}$ or $g_{\mu\mu}$.
- **Sum convention**: in this chapter a sum is always written out with $\sum$ or said in words, except in the definitions of Section 12.7 and in the formulas quoted from the record, where an index that appears once up and once down is summed over its eight values; "no sum" marks the exceptions.

- **Christoffel symbol** $\Gamma^a{}_{bc}$: a combination of first derivatives of the metric that says how the coordinate directions turn from point to point.
- **Riemann tensor** $R^{ab}{}_{mn}$: the curvature, built from the Christoffel symbols and their derivatives. **Pair curvature** $K_{ab} = R^{ab}{}_{ab}$ (no sum): the curvature of the plane spanned by the directions a and b (a name used in this chapter).
- **Ricci tensor** $R^h{}_j$, **Ricci scalar** R, **Einstein tensor** $G^h{}_j$: sums of Riemann components defined in Section 12.6.
- **Generalized Kronecker delta** (GKD): a sign, $+1$, -1 or 0, attached to two lists of indices (Section 12.7). **Lovelock tensors** $E_{(1)}, E_{(2)}, E_{(3)}$: the three curvature tensors built with the GKD; $E_{(1)}$ is the Einstein tensor and $E_{(2)}$ the **Gauss-Bonnet** tensor.
- **Couplings**: $\alpha_1, \alpha_2, \alpha_3$ weigh the three Lovelock tensors; Λ is the **cosmological constant**; $\kappa > 0$ measures the strength of gravity. **Einstein gravity**: $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$. **Einstein-Gauss-Bonnet gravity**: $\alpha_1 = 1$, $\alpha_3 = 0$.
- **Energy-momentum tensor** $T^\mu{}_\nu$, also called the **source**: the energy and momentum of the matter. Its parts are the **energy density** $\rho = -T^{x_4}{}_{x_4}$, the **pressures** p_3 (3-space), p_t (extra times), p_0 (hidden direction), and the **mixed components** $q_{40} = T^{x_4}{}_{x_0}$ and $q_{04} = T^{x_0}{}_{x_4}$.
- **Anisotropic stress**: the difference $p_3 - p_t$ between the 3-space pressure and the extra-time pressure.
- **Equation of state** $w = p/\rho$: pressure divided by energy density.
- **Field equations**: the equations of gravity, $\sum_k \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda \delta^\mu_\nu = \kappa T^\mu{}_\nu$ (Section 12.8). Their four independent parts are the **constraint** (time, x_4), the 3-space equation, the extra-time equation and the **hidden equation** (x_0). The **evolution equation** is the 3-space equation minus the extra-time equation; the **algebraic condition** is $p_3 + p_t = 2p_0$.
- **Divergence** $\nabla_\mu T^\mu{}_\nu$: the curved-space form of "the change of a quantity in time plus its outflow"; zero divergence means **conservation**. The **Bianchi identity** says that the left-hand side of the field equations has zero divergence for every metric.
- **Vacuum**: no matter, $T^\mu{}_\nu = 0$.
- **Null vector**: a direction of zero length, such as the frame direction $x_4 + x_0$ (time-like plus space-like). The **null energy condition** along it is $\rho + p_0 \geq 0$; ordinary matter satisfies it. Matter with $\rho > 0$ and $w < -1$ violates it and is called **phantom**.
- **Linear member**: the history $a_4 = AHx_4 + a_0$ with a constant **slope** A. For $A > 0$ the extra times deflate as e^{-AHx_4}. The author's metric leaves $a_4(x_4)$ free, and the author requires only that the extra times deflate ($A > 0$); the **canonical history** $A = 1$ is the choice of the Revision record (ASSUMED, a prescribed background; Section 12.12).
- **Prescribed source**: a source chosen by hand to see what the equations do. It is ASSUMED, not derived from a field.
- **Condensate**: a solution of a field equation that depends on the time x_4 only.
- **Status labels**: PROVED (exact, with the verifier file and check name), COMPUTED (numerical, with its measured accuracy), ASSUMED, HYPOTHESIS, OPEN.

12.3 The author's metric, its determinant and the volume of seven directions

The author's metric is diagonal. With the abbreviation $s = \sin^{1/3} z$ and $z = 6Hx_0$ its eight diagonal entries are

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4}s,\ e^{2a_4}s,\ e^{2a_4}s,\ -1,\ -e^{-2a_4}s,\ -e^{-2a_4}s,\ -e^{-2a_4}s\big),$$

in the order $x_0, \ldots, x_7$, on the background $b_4 = 1$ of the author's metric of 2026-10-14 (record check `metric_is_SPEC_section_1` of `Revision/field_equations_a4/reports/wolfram-a4-report.json`, which states the metric with b_4; Section 12.12 says what a changing b_4 changes). The scale factors are $\sqrt{|g_{11}|} = e^{a_4}\sin^{1/6} z$ for 3-space, $\sqrt{|g_{44}|} = 1$ for the time, $\sqrt{|g_{55}|} = e^{-a_4}\sin^{1/6} z$ for the extra times and $\sqrt{g_{00}} = \cot z$ for the hidden direction. When a_4 grows, the first grows and the third shrinks.

The **determinant** of the metric measures volume. We compute it line by line; after each line we name the rule that produced it.

$$\det g = g_{11}\, g_{22}\, g_{33}\, g_{44}\, g_{55}\, g_{66}\, g_{77}\, g_{00}$$

(the determinant of a diagonal matrix is the product of its diagonal entries)

$$= \big(e^{2a_4}s\big)^3 \cdot (-1) \cdot \big(-e^{-2a_4}s\big)^3 \cdot \cot^2 z$$

(the entries inserted; three equal factors written as a cube)

$$= (-1)\,(-1)^3\, e^{6a_4}\, e^{-6a_4}\, s^6\, \cot^2 z$$

(the power of a product is the product of the powers, $(xy)^3 = x^3y^3$, and $(e^u)^3 = e^{3u}$)

$$= s^6 \cot^2 z$$

($(-1)(-1)^3 = (-1)^4 = 1$ and $e^{6a_4}e^{-6a_4} = e^0 = 1$)

$$= \sin^2 z\, \cot^2 z = \cos^2 z$$

($s^6 = (\sin^{1/3} z)^6 = \sin^2 z$, and $\cot z = \cos z/\sin z$). On the patch $0 < z < \pi/2$ the cosine is positive, so

$$\sqrt{|\det g|} = \cos z.$$

This is PROVED (record `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `sqrt_abs_det_g_is_cos_z`; reproduced in Notebook 12a, In [4]). The function a_4 has dropped out. The reason is visible in the fourth line: the volume factor e^{3a_4} of the three inflating 3-space directions and the factor e^{-3a_4} of the three deflating extra times multiply to 1. The volume of the seven directions other than the time stays constant: what 3-space gains, the extra times lose.

12.4 The Christoffel symbols of the author's metric

Definition. For any metric the Christoffel symbols are

$$\Gamma^a{}_{bc} = \frac{1}{2}\sum_{d=1}^{8} g^{ad}(\partial_b g_{dc} + \partial_c g_{db} - \partial_d g_{bc}),$$

where ∂_b is the partial derivative with respect to the coordinate x_b. They are symmetric in the two lower indices, $\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$, because exchanging b and c exchanges the first two terms in the bracket.

The formula for a diagonal metric. We simplify the definition for our metric, line by line.

$$\Gamma^a{}_{bc} = \frac{1}{2g_{aa}}(\partial_b g_{ac} + \partial_c g_{ab} - \partial_a g_{bc})$$

(the inverse metric is diagonal, so in the sum over d only $d = a$ survives, and $g^{aa} = 1/g_{aa}$; no sum over a)

$$\Gamma^a{}_{bc} = \frac{1}{2g_{aa}}(\delta_{ac}\,\partial_b g_{aa} + \delta_{ab}\,\partial_c g_{aa} - \delta_{bc}\,\partial_a g_{bb})$$

(an entry g_{ac} of a diagonal matrix is g_{aa} when $c = a$ and 0 otherwise; the Kronecker delta δ_{ac} is 1 when $a = c$ and 0 otherwise). This is the formula that Notebook 12a programs in In [5].

The derivatives of the metric. The metric depends on x_4 (through a_4) and on x_0 (through z) only; every other partial derivative is zero. By the chain rule $\partial_0 = (dz/dx_0)\,\partial_z = 6H\,\partial_z$. Then:

- $\partial_4 g_{11} = 2a_4'\,e^{2a_4}s = 2a_4'\,g_{11}$ (chain rule: the derivative of $e^{2a_4(x_4)}$ is $2a_4'e^{2a_4}$); the same for g_{22}, g_{33}.
- $\partial_4 g_{55} = -2a_4'\,g_{55}$ (the exponent is $-2a_4$); the same for g_{66}, g_{77}.
- $\partial_0 g_{11} = 6H\,e^{2a_4}\,\frac{1}{3}\sin^{-2/3} z\,\cos z = 2H\cot z\,g_{11}$ (power rule for $\sin^{1/3} z$, then $\sin^{-2/3} z\cos z = \sin^{1/3} z\cdot\cos z/\sin z$); likewise $\partial_0 g_{55} = 2H\cot z\,g_{55}$.
- $\partial_0 g_{00} = 6H\cdot 2\cot z\cdot(-1/\sin^2 z) = -12H\cot z/\sin^2 z$ (chain rule, with $d\cot z/dz = -1/\sin^2 z$).
- $g_{44} = -1$ is constant.

The symbols. Inserting these into the diagonal formula:

- $\Gamma^{x_1}{}_{x_1x_4} = \partial_4 g_{11}/(2g_{11}) = a_4'$ (the term with δ_{ac}, $a = c = x_1$, $b = x_4$).
- $\Gamma^{x_5}{}_{x_5x_4} = \partial_4 g_{55}/(2g_{55}) = -a_4'$: the same symbol with the opposite sign, because the extra times deflate where 3-space inflates.
- $\Gamma^{x_1}{}_{x_1x_0} = \Gamma^{x_5}{}_{x_5x_0} = 2H\cot z\,g_{11}/(2g_{11}) = H\cot z$ (sympy prints it as `H/tan(z)`).
- $\Gamma^{x_4}{}_{x_1x_1} = -\partial_4 g_{11}/(2g_{44}) = -2a_4'g_{11}/(-2) = a_4'\,e^{2a_4}\sin^{1/3} z$ (the term with δ_{bc}, $b = c = x_1$, $a = x_4$).

- $\Gamma^{x_4}{}_{x_5x_5} = -\partial_4 g_{55}/(2g_{44}) = -a_4'\, g_{55} = a_4'\, e^{-2a_4} \sin^{1/3} z$.
- $\Gamma^{x_0}{}_{x_1x_1} = -\partial_0 g_{11}/(2g_{00}) = -H \tan z \;\; g_{11} = -H e^{2a_4} \sin^{4/3} z/\cos z$ (using $\cot z/\cot^2 z = \tan z = \sin z/\cos z$).
- $\Gamma^{x_0}{}_{x_5x_5} = -H \tan z \; g_{55} = H e^{-2a_4} \sin^{4/3} z/\cos z$.
- $\Gamma^{x_0}{}_{x_0x_0} = \partial_0 g_{00}/(2g_{00}) = -6H/(\sin z \cos z) = -12H/\sin 2z$ (using $\sin 2z = 2\sin z \cos z$).

The same holds with x_1 replaced by x_2 or x_3 and x_5 by x_6 or x_7. **Counting**: for each of the six directions $i \in \{x_1, x_2, x_3, x_5, x_6, x_7\}$ there are $\Gamma^i{}_{ix_4}$, $\Gamma^i{}_{x_4i}$, $\Gamma^i{}_{ix_0}$, $\Gamma^i{}_{x_0i}$, $\Gamma^{x_4}{}_{ii}$ and $\Gamma^{x_0}{}_{ii}$: $6 \times 6 = 36$ symbols, plus $\Gamma^{x_0}{}_{x_0x_0}$: 37 nonzero symbols of the 512, of which 25 are different. Every other symbol is zero; for example $\Gamma^e{}_{x_4x_4} = 0$ for every e, because g_{44} is constant and no other entry has an index pair (x_4, x_4). This is PROVED by Notebook 12a (In [5], checks "37 nonzero Christoffel symbols (25 distinct, 12 of them twice)" and the symmetry check; In [6] prints the 25 distinct symbols; the record computes the same symbols in `Revision/field_equations_a4/wolfram/FieldEquationsA4.wl`).

12.5 The curvature computed by hand

Definition (the convention of Misner, Thorne and Wheeler, MTW).

$$R^a{}_{bmn} = \partial_m \Gamma^a{}_{bn} - \partial_n \Gamma^a{}_{bm} + \sum_e \Gamma^a{}_{me}\Gamma^e{}_{bn} - \sum_e \Gamma^a{}_{ne}\Gamma^e{}_{bm}, \qquad R^{ab}{}_{mn} = g^{bb}\, R^a{}_{bmn} \;\; \text{(no sum)}.$$

The tensor $R^{ab}{}_{mn}$ changes sign when a and b are exchanged, and when m and n are exchanged (record check `riemann_pair_antisymmetry`). So it is zero when $a = b$ or $m = n$, and the **pair curvature** $K_{ab} = R^{ab}{}_{ab}$ (no sum) is symmetric, $K_{ba} = R^{ba}{}_{ba} = (-1)(-1)R^{ab}{}_{ab} = K_{ab}$. The eight directions form $8 \cdot 7/2 = 28$ pairs. We compute four pair curvatures by hand.

The pair (3-space direction, time). Take $a = x_1$, $b = x_4$, $m = x_1$, $n = x_4$:

$$R^{x_1}{}_{x_4x_1x_4} = \partial_1 \Gamma^{x_1}{}_{x_4x_4} - \partial_4 \Gamma^{x_1}{}_{x_4x_1} + \sum_e \Gamma^{x_1}{}_{x_1e}\Gamma^e{}_{x_4x_4} - \sum_e \Gamma^{x_1}{}_{x_4e}\Gamma^e{}_{x_4x_1}$$

(the definition with these indices)

$$= 0 - a_4'' + 0 - a_4' \cdot a_4'$$

(nothing depends on x_1; $\Gamma^{x_1}{}_{x_4x_1} = a_4'$ has the derivative a_4''; every $\Gamma^e{}_{x_4x_4}$ is zero; in the last sum only $e = x_1$ survives, with $\Gamma^{x_1}{}_{x_4x_1} = a_4'$ twice)

$$R^{x_1x_4}{}_{x_1x_4} = g^{44}\, R^{x_1}{}_{x_4x_1x_4} = (-1)\big(-a_4'' - (a_4')^2\big) = (a_4')^2 + a_4''$$

(raise the index x_4 with $g^{44} = 1/g_{44} = -1$). The same steps with $\Gamma^{x_5}{}_{x_4x_5} = -a_4'$ give for an extra time and the time $K_{x_5x_4} = (a_4')^2 - a_4''$: the second derivative enters with the opposite sign.

The pair (3-space direction, hidden direction). With $a = m = x_1$ and $b = n = x_0$:

$$R^{x_1}{}_{x_0x_1x_0} = \partial_1\Gamma^{x_1}{}_{x_0x_0} - \partial_0\Gamma^{x_1}{}_{x_0x_1} + \sum_e \Gamma^{x_1}{}_{x_1e}\Gamma^{e}{}_{x_0x_0} - \sum_e \Gamma^{x_1}{}_{x_0e}\Gamma^{e}{}_{x_0x_1}$$

(the definition)

$$= 0 - 6H\frac{d}{dz}(H\cot z) + \Gamma^{x_1}{}_{x_1x_0}\Gamma^{x_0}{}_{x_0x_0} - \Gamma^{x_1}{}_{x_0x_1}\Gamma^{x_1}{}_{x_0x_1}$$

($\Gamma^{x_1}{}_{x_0x_0} = 0$; $\partial_0 = 6H\,d/dz$; of the $\Gamma^{e}{}_{x_0x_0}$ only $e = x_0$ is nonzero, and $\Gamma^{x_1}{}_{x_0e}$ is nonzero only for $e = x_1$)

$$= \frac{6H^2}{\sin^2 z} + H\cot z\cdot\Big(-\frac{6H}{\sin z\cos z}\Big) - H^2\cot^2 z$$

($d\cot z/dz = -1/\sin^2 z$, and the symbols of Section 12.4 inserted)

$$= \frac{6H^2}{\sin^2 z} - \frac{6H^2}{\sin^2 z} - H^2\cot^2 z = -H^2\cot^2 z$$

($\cot z/(\sin z\cos z) = 1/\sin^2 z$, because $\cot z = \cos z/\sin z$)

$$K_{x_1x_0} = g^{00}\,R^{x_1}{}_{x_0x_1x_0} = \frac{1}{\cot^2 z}(-H^2\cot^2 z) = -H^2$$

(raise x_0 with $g^{00} = 1/\cot^2 z$). The angle z has disappeared: the curvature of this plane is the same constant $-H^2$ everywhere.

Two 3-space directions. With $a = m = x_1$, $b = n = x_2$ the two derivative terms vanish ($\Gamma^{x_1}{}_{x_2x_2} = \Gamma^{x_1}{}_{x_2x_1} = 0$), and so does the last sum (every $\Gamma^{x_1}{}_{x_2e}$ is zero):

$$R^{x_1}{}_{x_2x_1x_2} = \Gamma^{x_1}{}_{x_1x_4}\Gamma^{x_4}{}_{x_2x_2} + \Gamma^{x_1}{}_{x_1x_0}\Gamma^{x_0}{}_{x_2x_2}$$

(the two values of e for which $\Gamma^{x_1}{}_{x_1e}$ is not zero)

$$= a_4'\cdot a_4'\,g_{22} + H\cot z\cdot(-H\tan z\,g_{22}) = \big((a_4')^2 - H^2\big)\,g_{22}$$

(the symbols of Section 12.4, and $\cot z\tan z = 1$), so $K_{x_1x_2} = g^{22}R^{x_1}{}_{x_2x_1x_2} = (a_4')^2 - H^2$.

A 3-space direction and an extra time. With $b = n = x_5$ the same two terms give $\Gamma^{x_1}{}_{x_1x_4}\Gamma^{x_4}{}_{x_5x_5} + \Gamma^{x_1}{}_{x_1x_0}\Gamma^{x_0}{}_{x_5x_5} = (a_4')^2e^{-2a_4}s + H^2e^{-2a_4}s$, and raising with $g^{55} = -e^{2a_4}/s$ gives $K_{x_1x_5} = -\big((a_4')^2 + H^2\big)$.

All 28 pairs. The computation of Notebook 12a (In [9]) confirms these values and gives the rest; the 28 pair curvatures fall into seven groups:

the pair	**number of pairs**	**pair curvature K**
two 3-space directions	3	$(a_4')^2 - H^2$

the pair	number of pairs	pair curvature K
two extra times	3	$(a_4')^2 - H^2$
a 3-space direction and an extra time	9	$-(a_4')^2 - H^2$
a 3-space direction and the time	3	$(a_4')^2 + a_4''$
an extra time and the time	3	$(a_4')^2 - a_4''$
any of the six above directions and x_0	6	$-H^2$
the time and x_0	1	0

Besides these 27 nonzero diagonal components there are 12 nonzero **off-diagonal** ones. They connect a pair that contains the time with the pair in which the time is replaced by x_0; for example

$$R^{x_1x_4}{}_{x_1x_0} = R^{x_4x_5}{}_{x_5x_0} = Ha_4' \cot z, \qquad R^{x_1x_0}{}_{x_1x_4} = R^{x_5x_0}{}_{x_4x_5} = -\frac{Ha_4'}{\cot z}.$$

Together: 39 nonzero components with $a < b$ and $m < n$, and $4 \times 39 = 156$ of the 4096 components $R^{ab}{}_{mn}$ (each one appears with its four orderings). Three facts follow: no component contains the warp factor $\sin^{1/3} z$ or e^{a_4}; every component is a polynomial in H, a_4', a_4'' and $\cot z$, divided at most by $\cot z$; and the antisymmetries hold. All three are PROVED. The sympy report of the record, `Revision/field_equations_a4/reports/python-a4-report.json`, has the checks `mixed_riemann_free_of_warp_and_a4`, `riemann_entries_laurent` and `riemann_pair_antisymmetry`; the Wolfram report has `mixed_riemann_free_of_warp_and_a4` (whose detail includes the Laurent form in $\cot z$) and `riemann_pair_antisymmetry`; and Notebook 12a reproduces them in In [8].

12.6 The Ricci tensor, the Ricci scalar and the Einstein tensor by hand

Definitions.

$$R^h{}_j = \sum_a R^{ah}{}_{aj}, \qquad R = \sum_h R^h{}_h, \qquad G^h{}_j = R^h{}_j - \tfrac{1}{2}\,\delta^h_j\, R.$$

A rule for the diagonal. For $h = j$ the definition gives $R^h{}_h = \sum_a R^{ah}{}_{ah} = \sum_{a \neq h} K_{ah}$: the sum of the pair curvatures of the seven pairs that contain h. Summing over h counts every pair twice (once from each of its two directions), so $R = 2 \sum_{\text{pairs}} K$. Therefore

$$G^h{}_h = \sum_{\text{pairs containing } h} K - \sum_{\text{all 28 pairs}} K = - \sum_{\text{pairs not containing } h} K,$$

(the definition of G with the two sums inserted; the pairs that contain h cancel). With the table of Section 12.5 everything follows by counting.

The Ricci scalar. Adding the seven groups, with their numbers of pairs:

$$\sum_{\text{pairs}} K = 3\big((a_4')^2 - H^2\big) + 3\big((a_4')^2 - H^2\big) - 9\big((a_4')^2 + H^2\big) + 3\big((a_4')^2 + a_4''\big) + 3\big((a_4')^2 - a_4''\big) - 6H^2 + 0$$

(the table, row by row)

$$= (3+3-9+3+3)(a_4')^2 + (-3-3-9-6)H^2 + (3-3)a_4'' = 3(a_4')^2 - 21H^2$$

(collect the terms with $(a_4')^2$, with H^2 and with a_4''), so $R = 6(a_4')^2 - 42H^2$, the RESULT line of Notebook 12a, In [11].

The time component. The pairs that do not contain x_4 are all rows of the table except the three rows with the time:

$$G^{x_4}{}_{x_4} = -\Big[3\big((a_4')^2 - H^2\big) + 3\big((a_4')^2 - H^2\big) - 9\big((a_4')^2 + H^2\big) - 6H^2\Big]$$

(the rule)

$$= -\big[\,-3(a_4')^2 - 21H^2\big] = 3(a_4')^2 + 21H^2$$

$(3+3-9 = -3$ and $-3-3-9-6 = -21)$.

The hidden component. The pairs without x_0 are the first five rows: $(3+3-9+3+3)(a_4')^2 + (-3-3-9)H^2 + (3-3)a_4'' = 3(a_4')^2 - 15H^2$, so $G^{x_0}{}_{x_0} = 15H^2 - 3(a_4')^2$.

The 3-space component. The pairs without x_1: one pair of two 3-space directions (x_2x_3), three of two extra times, six of a 3-space direction (x_2 or x_3) and an extra time, two of a 3-space direction and the time, three of an extra time and the time, five with x_0 (the six minus x_1x_0), and the time with x_0:

$$\sum = (1+3-6+2+3)(a_4')^2 + (-1-3-6-5)H^2 + (2-3)a_4'' = 3(a_4')^2 - 15H^2 - a_4'',$$

so $G^{x_1}{}_{x_1} = a_4'' - 3(a_4')^2 + 15H^2$.

The extra-time component. It uses the pairs without x_5: three pairs of two 3-space directions, one of two extra times (x_6x_7), six of a 3-space direction and an extra time (x_6 or x_7), three of a 3-space direction and the time, two of an extra time (x_6 or x_7) and the time, five with x_0 (the six minus x_5x_0), and the time with x_0:

$$\sum = (3+1-6+3+2)(a_4')^2 + (-3-1-6-5)H^2 + (3-2)a_4'' = 3(a_4')^2 - 15H^2 + a_4''$$

(the table, row by row, with the pair x_4x_0 contributing 0; collect the terms with $(a_4')^2$, with H^2 and with a_4''), so $G^{x_5}{}_{x_5} = -a_4'' - 3(a_4')^2 + 15H^2$.

These four components are PROVED (record `Revision/field_equations_a4/reports/python-a4-report.json` and the Wolfram report, check `einstein_components`; reproduced in Notebook 12a, In [12]).

Why the mixed component $G^{x_4}{}_{x_0}$ vanishes. For $h = x_4$, $j = x_0$ the sum $R^{x_4}{}_{x_0} = \sum_a R^{a x_4}{}_{a x_0}$ picks up the off-diagonal Riemann components. For a 3-space direction, $R^{x_1 x_4}{}_{x_1 x_0} = H a_4' \cot z$. For an extra time, $R^{x_5 x_4}{}_{x_5 x_0} = -R^{x_4 x_5}{}_{x_5 x_0} = -H a_4' \cot z$ (exchange of the upper pair). So

$$G^{x_4}{}_{x_0} = R^{x_4}{}_{x_0} = 3 H a_4' \cot z - 3 H a_4' \cot z = 0.$$

The three inflating 3-space directions and the three deflating extra times contribute equal and opposite amounts. The same happens for $G^{x_0}{}_{x_4}$, and every other off-diagonal component has no nonzero term at all. So the Einstein tensor is diagonal and does not depend on x_0 (PROVED: record `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, checks `einstein_off_diagonal_zero`, `einstein_isotropy`, `einstein_x0_independent`; reproduced in Notebook 12a, In [12]).

12.7 The Lovelock tensors: the Einstein tensor and its two companions

The generalized Kronecker delta. For two lists of n indices, $\delta^{h_1 \ldots h_n}_{j_1 \ldots j_n}$ is the determinant of the n by n matrix whose entry in row r and column c is the ordinary Kronecker delta $\delta^{h_r}_{j_c}$. Two equal upper indices give two equal rows and the determinant 0. Otherwise it is $+1$ if the lower list is an even rearrangement of the upper list (an even number of exchanges turns one into the other), -1 if it is an odd one, and 0 if the two lists do not hold the same indices (Chapter 1 and Chapter 11 prove this).

The Lovelock tensors (formula (4.38) of the book by Lovelock and Rund, as programmed in the record, with a sum over every repeated index):

$$P_{(k)}{}^{h}{}_{j} = \delta^{h\, h_1 \ldots h_{2k}}_{j\, j_1 \ldots j_{2k}}\; R^{j_1 j_2}{}_{h_1 h_2} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}}, \qquad E_{(k)}{}^{h}{}_{j} = -\frac{P_{(k)}{}^{h}{}_{j}}{2^{k+1}}.$$

Only $k = 1, 2, 3$. For $k = 4$ the delta has $2k + 1 = 9$ upper indices taking only 8 values; two must be equal, so $P_{(4)} = 0$ (PROVED: Wolfram record, check `P4_vanishes_pigeonhole`; Notebook 12a, In [14]). In eight dimensions there are exactly three Lovelock tensors.

$E_{(1)}$ is the Einstein tensor. For $h = j$ the delta $\delta^{h\, h_1 h_2}_{h\, j_1 j_2}$ is not zero only when $h_1 \neq h_2$, both differ from h, and $\{j_1, j_2\} = \{h_1, h_2\}$. So only diagonal Riemann components enter:

$$P_{(1)}{}^{h}{}_{h} = \sum_{h_1 \neq h_2;\ h_1, h_2 \neq h} \left(\delta^{h\, h_1 h_2}_{h\, h_1 h_2} R^{h_1 h_2}{}_{h_1 h_2} + \delta^{h\, h_1 h_2}_{h\, h_2 h_1} R^{h_2 h_1}{}_{h_1 h_2} \right)$$

(the two possible orders of the lower indices)

$$= \sum_{h_1 \neq h_2;\ h_1, h_2 \neq h} \left(K_{h_1 h_2} + (-1)(-K_{h_1 h_2}) \right) = 4 \sum_{\text{pairs not containing } h} K$$

(one exchange gives the delta -1 and the Riemann component $-K$; each unordered pair appears twice as an ordered one). Hence $E_{(1)}{}^{h}{}_{h} = -P_{(1)}{}^{h}{}_{h}/4 = -\sum_{\text{pairs not containing } h} K = G^{h}{}_{h}$, the rule of Section 12.6. The record checks it in all 64 components (PROVED: Wolfram check `P1_direct_equals_minus_4_Einstein`, sympy check `P1_equals_minus_4_Einstein`; Notebook 12a, In [16]).

$E_{(2)}$ **and** $E_{(3)}$**.** For $k = 2, 3$ products of two or three Riemann components appear. Now the off-diagonal components also contribute: in a product such as $R^{x_1x_4}{}_{x_1x_0}\, R^{x_5x_0}{}_{x_4x_5}$ the factors $\cot z$ and $1/\cot z$ cancel. There are 690 contributing index lists for $k = 2$ and 4410 for $k = 3$ (Notebook 12a, In [15]); this bookkeeping is a job for the computer. The record's result, computed with the GKD in Rust (`Revision/gkd_lovelock/results/lovelock-tensors.json`), recomputed in Wolfram and in sympy, and reproduced by Notebook 12a in all 64 components of each tensor, is (record `Revision/field_equations_a4/a4-equations.json`, key `lovelockTensors`):

$$
\begin{aligned}
E_{(1)}{}^{x_1}{}_{x_1} &= -3(a_4')^2 + a_4'' + 15H^2, \qquad E_{(1)}{}^{x_4}{}_{x_4} = 3(a_4')^2 + 21H^2,\\
E_{(1)}{}^{x_5}{}_{x_5} &= -3(a_4')^2 - a_4'' + 15H^2, \qquad E_{(1)}{}^{x_0}{}_{x_0} = 15H^2 - 3(a_4')^2,\\
E_{(2)}{}^{x_1}{}_{x_1} &= 12(a_4')^4 - 24(a_4')^2 a_4'' + 168(a_4')^2H^2 - 40a_4''H^2 - 180H^4,\\
E_{(2)}{}^{x_4}{}_{x_4} &= -36(a_4')^4 - 120(a_4')^2H^2 - 420H^4,\\
E_{(2)}{}^{x_5}{}_{x_5} &= 12(a_4')^4 + 24(a_4')^2 a_4'' + 168(a_4')^2H^2 + 40a_4''H^2 - 180H^4,\\
E_{(2)}{}^{x_0}{}_{x_0} &= 12(a_4')^4 + 168(a_4')^2H^2 - 180H^4,
\end{aligned}
$$

$$
\begin{aligned}
E_{(3)}{}^{x_1}{}_{x_1} &= -72(a_4')^6 + 360(a_4')^4 a_4'' - 648(a_4')^4H^2 + 432(a_4')^2 a_4''H^2\\
&\quad - 1944(a_4')^2H^4 + 360a_4''H^4 + 360H^6,\\
E_{(3)}{}^{x_4}{}_{x_4} &= 360(a_4')^6 + 648(a_4')^4H^2 + 1080(a_4')^2H^4 + 2520H^6,\\
E_{(3)}{}^{x_5}{}_{x_5} &= -72(a_4')^6 - 360(a_4')^4 a_4'' - 648(a_4')^4H^2 - 432(a_4')^2 a_4''H^2\\
&\quad - 1944(a_4')^2H^4 - 360a_4''H^4 + 360H^6,\\
E_{(3)}{}^{x_0}{}_{x_0} &= -72(a_4')^6 - 648(a_4')^4H^2 - 1944(a_4')^2H^4 + 360H^6,
\end{aligned}
$$

with x_2, x_3 equal to x_1, x_6, x_7 equal to x_5, and every off-diagonal component zero. All of this is PROVED: the components computed from the curvature equal the Rust GKD record monomial by monomial (checks `P1_equals_gkd_branch_monomials` to `P3_equals_gkd_branch_monomials` of the sympy report and `P1_direct_equals_gkd_branch_monomials` to `P3_direct_equals_gkd_branch_monomials` of the Wolfram report); each $E_{(k)}$ is diagonal, free of x_0 and has zero divergence (checks `P1_structure` to `P3_structure` and `E1_divergence_free` to `E3_divergence_free` in both reports); and $E_{(2)}$ equals the classical Gauss-Bonnet tensor of Lanczos, which uses no GKD (Notebook 12a, In [18], for every a_4; for the linear member also record `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, checks `gauss_bonnet_rho_alpha2` and `gauss_bonnet_p_alpha2`). That every divergence-free tensor built from the metric and its first and second derivatives is a combination of these tensors is Lovelock's theorem (D. Lovelock, J. Math. Phys. 12, 498 (1971)), which this book quotes and does not prove.

12.8 The source and the independent field equations

The field equations of Einstein-Lovelock gravity are

$$
\sum_{k=1}^{3} \alpha_k\, E_{(k)}{}^{\mu}{}_{\nu} + \Lambda\, \delta^{\mu}_{\nu} = \kappa\, T^{\mu}{}_{\nu},
$$

64 equations, one for each pair (μ, ν). The left-hand side is built from the metric; the right-hand side is the source. We allow the most general source that the symmetry of the problem suggests:

$$T^{\mu}{}_{\nu} = \mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0) + q_{40}\ (\mu = x_4, \nu = x_0) + q_{04}\ (\mu = x_0, \nu = x_4).$$

The minus sign in $T^{x_4}{}_{x_4} = -\rho$ comes from $g^{44} = -1$: the energy density is the component T_{44} with both indices down, and raising the first index multiplies it by g^{44} (the convention of the record; the pressures are $p_\mu = T^{\mu}{}_{\mu}$, no sum).

Reduction. Three facts of Section 12.7 reduce the 64 equations.

1. Every off-diagonal left-hand side is zero. So the off-diagonal equations read $0 = \kappa T^{\mu}{}_{\nu}$: the source must have $q_{40} = q_{04} = 0$ and no other mixed component.
2. The left-hand sides of x_1, x_2, x_3 are equal, and so are those of x_5, x_6, x_7. So the source must have one pressure for 3-space and one for the extra times, as assumed.
3. No left-hand side depends on x_0. So no part of the source may depend on x_0.

What remains are four equations. With the abbreviations $e_{hh} = \sum_k \alpha_k E_{(k)}{}^{h}{}_{h}$ (no Λ; Notebook 12a calls them `e11`, `e44`, `e55`, `e00`):

$$\begin{aligned}
&\text{constraint } (x_4): && e_{44} + \Lambda = -\kappa\rho,\\
&\text{3-space } (x_1 = x_2 = x_3): && e_{11} + \Lambda = \kappa p_3,\\
&\text{extra times } (x_5 = x_6 = x_7): && e_{55} + \Lambda = \kappa p_t,\\
&\text{hidden } (x_0): && e_{00} + \Lambda = \kappa p_0.
\end{aligned}$$

This is PROVED (Wolfram record, check `independent_components`; sympy record, check `other_components_vanish`; reproduced in Notebook 12a, In [21]).

12.9 The constraint, the evolution equation and the algebraic condition

Einstein gravity first. With $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$ the four abbreviations are the Einstein components of Section 12.6:

$$e_{44} = 3(a_4')^2 + 21H^2, \quad e_{11} = a_4'' - 3(a_4')^2 + 15H^2, \quad e_{55} = -a_4'' - 3(a_4')^2 + 15H^2, \quad e_{00} = 15H^2 - 3(a_4')^2.$$

The time and hidden components contain a_4' but no a_4'': they are first-order equations. Subtracting the extra-time equation from the 3-space equation removes Λ:

$$e_{11} - e_{55} = \kappa(p_3 - p_t)$$

(3-space equation minus extra-time equation)

$$(a_4'' - 3(a_4')^2 + 15H^2) - (-a_4'' - 3(a_4')^2 + 15H^2) = \kappa(p_3 - p_t)$$

(the components inserted)

$$2a_4'' = \kappa(p_3 - p_t)$$

(the terms $-3(a_4')^2$ and $15H^2$ cancel). This is the **evolution equation**: the second derivative of a_4 is driven by the anisotropic stress, nothing else. Adding the two equations instead:

$$e_{11} + e_{55} = 30H^2 - 6(a_4')^2 = 2\,e_{00}$$

(the a_4'' cancel; $30H^2 - 6(a_4')^2 = 2(15H^2 - 3(a_4')^2)$), so the 3-space equation plus the extra-time equation minus twice the hidden equation reads $0 = \kappa(p_3 + p_t - 2p_0)$: the **algebraic condition** $p_3 + p_t = 2p_0$. It is not an equation for a_4 at all; it is a condition that every source must meet.

Einstein-Lovelock gravity. The same two combinations work for every coupling. From the components of Section 12.7, the Gauss-Bonnet part of $e_{11} - e_{55}$ is

$$E_{(2)}{}^{x_1}{}_{x_1} - E_{(2)}{}^{x_5}{}_{x_5} = -48(a_4')^2 a_4'' - 80H^2 a_4'' = a_4''(-48(a_4')^2 - 80H^2)$$

(the terms without a_4'' are equal in both and cancel; the others double), and the third-order part is $a_4''(720(a_4')^4 + 864(a_4')^2H^2 + 720H^4)$. So the evolution equation is

$$a_4''\,F(a_4') = \kappa(p_3 - p_t), \qquad F = 2\alpha_1 - \alpha_2(48(a_4')^2 + 80H^2) + \alpha_3(720(a_4')^4 + 864(a_4')^2H^2 + 720H^4),$$

and $e_{11}+e_{55} = 2e_{00}$ holds identically for each k (check for $k = 2$: $2\cdot12(a_4')^4+2\cdot168(a_4')^2H^2-2\cdot180H^4$ on both sides). All of this is PROVED; the checks are listed here:

statement	**sympy record**	**Wolfram record**
$e_{11} - e_{55} = a_4''F(a_4')$	`evolution_factorises`	`evolution_factorises_a4pp_times_F`
F is not identically zero	`evolution_F_not_identically_zero`	`evolution_F_not_identically_zero`
$e_{11} + e_{55} = 2e_{00}$	`algebraic_identity`	`algebraic_identity_x1_plus_x5_minus_2x0`
no a_4'' in e_{44} and e_{00}	`constraint_and_x0_first_order`	`x0_component_contains_no_a4pp`

Notebook 12a reproduces them in In [22] and In [23]. F vanishes for every a_4' and H only when all three couplings are zero. But for Einstein-Gauss-Bonnet gravity with $\alpha_2 > 0$, $F = 2 - 80\alpha_2H^2 - 48\alpha_2(a_4')^2$ is zero at $(a_4')^2 = (1 - 40\alpha_2H^2)/(24\alpha_2)$; there the evolution equation cannot be solved for a_4'' (figure 4 of Notebook 12a; Section 12.21).

What each equation does. Given a_4' at one time, the constraint fixes the energy density ρ at that time, and the hidden equation fixes p_0. Given the anisotropic stress, the evolution equation

fixes a_4'' and so the future of a_4. The algebraic condition then fixes p_3 and p_t from p_0 and the stress: $p_3 = p_0 + \frac{1}{2}(p_3 - p_t)$, $p_t = p_0 - \frac{1}{2}(p_3 - p_t)$.

12.10 The Bianchi identity and the conservation law

The identity. Each Lovelock tensor has zero divergence for every metric (PROVED for the author's metric with an arbitrary a_4: record checks `E1_divergence_free`, `E2_divergence_free`, `E3_divergence_free` in both reports; Notebook 12a, In [19]). Since $\Lambda\delta^\mu_\nu$ has zero divergence too, the field equations can hold only if the source has zero divergence.

The divergence of the general source. We compute it for the source of Section 12.8, whose parts $\rho, p_3, p_t, p_0, q_{40}, q_{04}$ may now depend on x_4 and x_0, from the definition of the covariant divergence (Section 9.9):

$$\nabla_\mu T^\mu{}_\nu = \sum_\mu \partial_\mu T^\mu{}_\nu + \sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda}\, T^\lambda{}_\nu - \sum_{\mu,\lambda} \Gamma^\lambda{}_{\mu\nu}\, T^\mu{}_\lambda .$$

Two sums of Christoffel symbols are needed first. From the list of Section 12.4:

$$\sum_\mu \Gamma^\mu{}_{\mu x_4} = 3a_4' + 3(-a_4') + 0 + 0 = 0$$

(the three 3-space directions give $\Gamma^{x_1}{}_{x_1 x_4} = a_4'$ each, the three extra times $\Gamma^{x_5}{}_{x_5 x_4} = -a_4'$ each, and $\Gamma^{x_4}{}_{x_4 x_4} = \Gamma^{x_0}{}_{x_0 x_4} = 0$), and

$$\sum_\mu \Gamma^\mu{}_{\mu x_0} = 6H\cot z - \frac{6H}{\sin z\cos z} = 6H\,\frac{\cos^2 z - 1}{\sin z\cos z} = -\frac{6H\sin z}{\cos z} = -6H\tan z$$

(the six directions $x_1, x_2, x_3, x_5, x_6, x_7$ give $H\cot z$ each, $\Gamma^{x_0}{}_{x_0 x_0} = -6H/(\sin z\cos z)$ and $\Gamma^{x_4}{}_{x_4 x_0} = 0$; then the common denominator $\sin z\cos z$, with $\cot z = \cos^2 z/(\sin z\cos z)$, and $\cos^2 z - 1 = -\sin^2 z$). This is $\partial_0 \ln\cos z = 6H\cdot(-\sin z/\cos z)$, the logarithmic derivative of $\sqrt{|\det g|} = \cos z$ (Section 12.3), as the lemma of Section 9.9 says.

The component $\nu = x_4$. The column $T^\lambda{}_{x_4}$ has only the entries $T^{x_4}{}_{x_4} = -\rho$ and $T^{x_0}{}_{x_4} = q_{04}$, and the entries $T^\mu{}_\lambda$ that are not zero are the eight diagonal ones and the two mixed ones $T^{x_4}{}_{x_0} = q_{40}$, $T^{x_0}{}_{x_4} = q_{04}$. So the three sums of the definition become

$$\nabla_\mu T^\mu{}_{x_4} = \partial_4 T^{x_4}{}_{x_4} + \partial_0 T^{x_0}{}_{x_4} + \Big(\sum_\mu \Gamma^\mu{}_{\mu x_4}\Big) T^{x_4}{}_{x_4} + \Big(\sum_\mu \Gamma^\mu{}_{\mu x_0}\Big) T^{x_0}{}_{x_4}$$
$$- \sum_\mu \Gamma^\mu{}_{\mu x_4} T^\mu{}_\mu - \Gamma^{x_0}{}_{x_4 x_4} T^{x_4}{}_{x_0} - \Gamma^{x_4}{}_{x_0 x_4} T^{x_0}{}_{x_4}$$

(the definition with $\nu = x_4$, keeping only the nonzero entries of T; in the next-to-last sum the diagonal entry $T^\mu{}_\mu$ has $\lambda = \mu$)

$$= -\partial_4\rho + \partial_0 q_{04} + 0 - 6H\tan z\, q_{04} - (3a_4'\, p_3 - 3a_4'\, p_t) - 0 - 0$$

(the entries inserted; the two sums of symbols above; on the diagonal $\Gamma^{x_1}{}_{x_1x_4} = a_4'$ multiplies p_3 three times and $\Gamma^{x_5}{}_{x_5x_4} = -a_4'$ multiplies p_t three times, while $\Gamma^{x_4}{}_{x_4x_4} = \Gamma^{x_0}{}_{x_0x_4} = 0$; and $\Gamma^{x_0}{}_{x_4x_4} = \Gamma^{x_4}{}_{x_0x_4} = 0$ by Section 12.4)

$$= -\partial_4\rho - 3a_4'(p_3 - p_t) + \partial_0 q_{04} - 6H\tan z\, q_{04}$$

(the terms reordered and $3a_4'$ taken out of the bracket).

The component $\nu = x_0$. In the same way, with the column $T^{x_4}{}_{x_0} = q_{40}$, $T^{x_0}{}_{x_0} = p_0$:

$$\nabla_\mu T^\mu{}_{x_0} = \partial_4 T^{x_4}{}_{x_0} + \partial_0 T^{x_0}{}_{x_0} + \Big(\sum_\mu \Gamma^\mu{}_{\mu x_4}\Big) T^{x_4}{}_{x_0} + \Big(\sum_\mu \Gamma^\mu{}_{\mu x_0}\Big) T^{x_0}{}_{x_0} - \sum_\mu \Gamma^\mu{}_{\mu x_0} T^\mu{}_\mu - \Gamma^{x_0}{}_{x_4x_0} T^{x_4}{}_{x_0} - \Gamma^{x_4}{}_{x_0x_0} T^{x_0}{}_{x_4}$$

(the definition with $\nu = x_0$, keeping only the nonzero entries of T)

$$= \partial_4 q_{40} + \partial_0 p_0 + 0 - 6H\tan z\, p_0 - \Big(3H\cot z\, p_3 + 3H\cot z\, p_t - \frac{6H}{\sin z\cos z}\, p_0\Big) - 0 - 0$$

(the entries inserted; on the diagonal the six directions give $H\cot z$ times their pressure, $\Gamma^{x_4}{}_{x_4x_0} = 0$, and $\Gamma^{x_0}{}_{x_0x_0} = -6H/(\sin z\cos z)$ multiplies p_0; $\Gamma^{x_0}{}_{x_4x_0} = 0$ because g_{00} does not depend on x_4, and $\Gamma^{x_4}{}_{x_0x_0} = -\partial_4 g_{00}/(2g_{44}) = 0$ for the same reason)

$$= \partial_0 p_0 + \Big(\frac{6H}{\sin z\cos z} - \frac{6H\sin z}{\cos z}\Big) p_0 - 3H\cot z\,(p_3 + p_t) + \partial_4 q_{40}$$

(the two terms with p_0 collected, $\tan z = \sin z/\cos z$)

$$= \partial_0 p_0 + 6H\cot z\, p_0 - 3H\cot z\,(p_3 + p_t) + \partial_4 q_{40}$$
$$= \partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t) + \partial_4 q_{40}$$

($\frac{6H}{\sin z\cos z} - \frac{6H\sin^2 z}{\sin z\cos z} = 6H\,\frac{1-\sin^2 z}{\sin z\cos z} = 6H\,\frac{\cos z}{\sin z}$; then $3H\cot z$ taken out).

The six other components. For $\nu = x_1$ every term is zero: the column $T^\lambda{}_{x_1}$ has only $T^{x_1}{}_{x_1} = p_3$, which does not depend on x_1; $\sum_\mu \Gamma^\mu{}_{\mu x_1} = 0$, because no entry of the metric depends on x_1; and every symbol $\Gamma^\lambda{}_{\mu x_1}$ that multiplies a nonzero entry $T^\mu{}_\lambda$ is zero ($\Gamma^{x_1}{}_{x_1x_1} = 0$, and none of the nonzero symbols of Section 12.4 has exactly one index equal to x_1). The same holds for x_2, x_3, x_5, x_6, x_7.

These are the record's formulas (PROVED: record `Revision/lead_checks/reports/emt-divergence-and-spin-connection.json`, checks `divergence_x4_component`, `divergence_x0_component` and `divergence_other_components_zero`; Notebook 12a, In [25], which reproduces the first two of them and the sympy record's check `conservation_components`):

$$\nabla_\mu T^\mu{}_{x_4} = -\partial_4\rho - 3a_4'(p_3 - p_t) + \partial_0 q_{04} - 6H\tan z\, q_{04},$$
$$\nabla_\mu T^\mu{}_{x_0} = \partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t) + \partial_4 q_{40},$$

and zero for the six other components. With the conditions of Section 12.8 (no x_0 dependence, $q_{40} = q_{04} = 0$) the first says $\rho' = -3a_4'(p_3 - p_t)$ and the second repeats the algebraic condition.

The constraint propagates. In Einstein gravity we can see directly why the constraint stays true along a solution:

$$-\kappa\rho = 3(a_4')^2 + 21H^2 + \Lambda$$

(the constraint)

$$-\kappa\rho' = 6a_4'a_4''$$

(differentiate with respect to x_4: chain rule for $(a_4')^2$; H and Λ are constants)

$$-\kappa\rho' = 3a_4' \cdot 2a_4'' = 3a_4'\,\kappa(p_3 - p_t)$$

(the evolution equation $2a_4'' = \kappa(p_3 - p_t)$ inserted)

$$\rho' = -3a_4'(p_3 - p_t)$$

(divided by $-\kappa$). So the constraint, differentiated, is $3a_4'$ times the evolution equation, exactly when the source obeys the conservation law. The record proves the same for every coupling, $\frac{d}{dx_4}e_{44} = 3a_4'(e_{11} - e_{55})$ (checks `constraint_propagation_bianchi` and `bianchi_x4`; Notebook 12a, In [23]). For the Gauss-Bonnet part, for example, $\frac{d}{dx_4}(-36(a_4')^4 - 120(a_4')^2H^2) = -144(a_4')^3a_4'' - 240a_4'a_4''H^2 = 3a_4' \cdot a_4''(-48(a_4')^2 - 80H^2)$.

What the conservation law means. The law $\rho' = -3a_4'(p_3 - p_t)$ is the first law of thermodynamics, $dE = -p\,dV$, for the two kinds of directions. The total volume of the seven directions other than the time is constant (Section 12.3), so the energy density can change only through the work of the pressures. The volume of 3-space grows like e^{3a_4}, so its logarithm grows at the rate $3a_4'$, and the pressure p_3 contributes $-3a_4'p_3$ per unit volume and time. The volume of the extra times shrinks like e^{-3a_4}, at the rate $-3a_4'$, and the pressure p_t contributes $+3a_4'p_t$. The total is $-3a_4'(p_3 - p_t)$: energy flows between 3-space and the extra times, and the energy density stays constant when the two pressures are equal. (This reading is an interpretation of the PROVED identity.)

12.11 Einstein gravity: no empty universe, and the null energy condition along x0

In Einstein gravity the constraint and the hidden equation read

$$3(a_4')^2 + 21H^2 + \Lambda = -\kappa\rho, \qquad -3(a_4')^2 + 15H^2 + \Lambda = \kappa p_0.$$

The null energy condition fails. Subtracting the first from the second:

$$(-3(a_4')^2 + 15H^2 + \Lambda) - (3(a_4')^2 + 21H^2 + \Lambda) = \kappa p_0 + \kappa\rho$$

(second equation minus first)

$$\kappa(\rho + p_0) = -6(a_4')^2 - 6H^2 = -6\big((a_4')^2 + H^2\big)$$

(Λ cancels; $15 - 21 = -6$). For $H > 0$ the right-hand side is negative for every a_4 and every Λ, so $\rho + p_0 < 0$ (with $\kappa > 0$). In the frame of the vielbein (the directions of unit length of Chapter 6) the energy-momentum tensor has $T_{\hat{4}\hat{4}} = \rho$ and $T_{\hat{8}\hat{8}} = p_0$, and the vector k with frame components $k^{\hat{4}} = k^{\hat{8}} = 1$ (all others 0) has zero length, $\eta(k,k) = -1 + 1 = 0$: a **null vector**. Then $T(k,k) = T_{\hat{4}\hat{4}} + 2T_{\hat{4}\hat{8}} + T_{\hat{8}\hat{8}} = \rho + 0 + p_0$. Every source of the author's metric in Einstein gravity violates the null energy condition along this null direction (PROVED: sympy and Wolfram records, check `einstein_null_energy_x0`; Notebook 12a, In [26]).

No vacuum. A vacuum needs $\rho = p_0 = 0$, so $6\big((a_4')^2 + H^2\big) = 0$, which no real a_4' satisfies when $H > 0$. If one allows complex numbers, the vacuum equations have exactly the two solutions $a_4' = \pm iH$, $\Lambda = -18H^2$ (from $(a_4')^2 = -H^2$ and $\Lambda = 3(a_4')^2 - 15H^2$), which are not real histories (PROVED: Wolfram record, check `einstein_no_vacuum_solution`; lead's record `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, check `no_vacuum_for_H_positive`; Notebook 12a, In [26] and figure 5). The author's metric is never empty space: it always needs matter, and that matter is of an unusual kind.

Every coupling. For Einstein-Lovelock gravity the same combination factorises: $\kappa(\rho + p_0) = -6\big((a_4')^2 + H^2\big)\,V$, with $V = \alpha_1 - 8\alpha_2\big((a_4')^2 + 5H^2\big) + \alpha_3\big(72(a_4')^4 + 144(a_4')^2H^2 + 360H^4\big)$ (PROVED by Notebook 12a, In [29], from the record's components). The null energy condition can hold only where $V \le 0$, and a vacuum needs $V = 0$. With the Einstein term at its usual weight, $\alpha_1 = 1$, this needs Lovelock couplings beyond Einstein's. The bracket $8\big((a_4')^2 + 5H^2\big)$ that multiplies $-\alpha_2$ and the bracket $72(a_4')^4 + 144(a_4')^2H^2 + 360H^4$ that multiplies α_3 are both positive, so $V \le 0$ needs a positive Gauss-Bonnet coupling α_2 or a negative third-order coupling α_3, large enough. For example, with $\alpha_3 = 0$:

$$V \le 0 \iff 1 \le 8\alpha_2\big((a_4')^2 + 5H^2\big) \iff \alpha_2 \ge \frac{1}{8\big((a_4')^2 + 5H^2\big)}$$

(the definition of V with $\alpha_1 = 1$ and $\alpha_3 = 0$; then division by the positive bracket). With $\alpha_2 = 0$, at a moment where $a_4' = 0$: $V = 1 + 360\alpha_3H^4 \le 0$ exactly when $\alpha_3H^4 \le -1/360$; for instance $\alpha_3H^4 = -1/300$ gives $V = 1 - 360/300 = -0.2$.

12.12 The linear member: the extra times deflate by a choice of sign

The history. The simplest history is $a_4 = AHx_4 + a_0$ with constants A and a_0. Then $a_4' = AH$ and $a_4'' = 0$, the 3-space scale factor grows like e^{AHx_4} and the extra-time scale factor shrinks like e^{-AHx_4}: for $A > 0$ the extra times deflate exponentially, at the rate AH. The author's metric does not fix the slope: the author requires only that the extra times deflate, $A > 0$. The Revision record takes $A = 1$ as its canonical history (ASSUMED, a prescribed background: the record `Revision/kohn_sham/results/parameters.json` writes "a4 = A H x4 (A = 1 canonical)" and calls it a PRESCRIBED BACKGROUND); this chapter calls $A = 1$ the **canonical history**.

The source it requires. Line by line:

- evolution equation with $a_4'' = 0$: $0 = \kappa(p_3 - p_t)$, so $p_3 = p_t$;
- algebraic condition: $p_3 + p_t = 2p_0$ with $p_3 = p_t$ gives $p_0 = p_3$; so all three pressures are equal, $p_3 = p_t = p_0 =: p$;
- constraint and hidden equation with $a_4' = AH$ give constant ρ and p.

In Einstein gravity:

$$\kappa\rho = -(21 + 3A^2)H^2 - \Lambda, \qquad \kappa p = (15 - 3A^2)H^2 + \Lambda, \qquad \kappa(\rho + p) = -6(1 + A^2)H^2.$$

For all three couplings the record's formulas are (PROVED: record checks `linear_member_equal_pressures` and `json_linear_member`; Notebook 12a, In [28], and Notebook 12b, In [3] to In [5]):

$$\begin{aligned}\kappa\rho &= \alpha_1(-3A^2 - 21)H^2 + \alpha_2(36A^4 + 120A^2 + 420)H^4 \\ &\quad + \alpha_3(-360A^6 - 648A^4 - 1080A^2 - 2520)H^6 - \Lambda, \\ \kappa p &= \alpha_1(15 - 3A^2)H^2 + \alpha_2(12A^4 + 168A^2 - 180)H^4 \\ &\quad + \alpha_3(-72A^6 - 648A^4 - 1944A^2 + 360)H^6 + \Lambda.\end{aligned}$$

Deflation is a choice of sign. The slope enters only as A^2, A^4 and A^6. Replacing A by $-A$ changes nothing: the equations accept extra times that deflate ($A > 0$) and extra times that inflate ($A < 0$) on exactly the same footing, and $A = 0$ (a static metric) as well. That the author's extra times deflate is a choice of sign, an initial condition; it is not a consequence of the field equations (PROVED: Notebook 12a, In [28], and Notebook 12b, In [4]; record key `theoremLinear` of `Revision/field_equations_a4/a4-equations.json`). This symmetry relates two histories of the metric of one universe. It is not the pairing of universes of masses $+m$ and $-m$, which is the subject of Chapters 18 to 20, and it says nothing about how a universe could be created.

The canonical history in numbers. For $A = 1$ and $\Lambda = 0$ (units $H = \kappa = 1$): $\kappa\rho = -24$, $\kappa p = 12$, $w = p/\rho = -1/2$ (Notebook 12b, In [5]). The energy density is negative. A positive energy density needs $-(21 + 3A^2)H^2 - \Lambda > 0$, that is $\Lambda < -(21 + 3A^2)H^2$, and then:

$$\kappa(\rho + p) = -6(1 + A^2)H^2 < 0 \;\Rightarrow\; p < -\rho \;\Rightarrow\; \frac{p}{\rho} < -1$$

(the sum of the two formulas; then divide by $\rho > 0$, which keeps the direction of the inequality). A positive energy density comes only with $w < -1$: the source is phantom (PROVED: Notebook 12b, In [8], checks for $\Lambda = -30H^2$, $-40H^2$, $-60H^2$ and the identity $w + 1 = (\rho + p)/\rho$).

The Gauss-Bonnet vacuum. With $\alpha_1 = 1$, $\alpha_3 = 0$ the record factorises $\kappa(\rho+p) = -6(A^2+1)H^2\,V$ with $V = 1 - 8\alpha_2 H^2(A^2 + 5)$ (record checks `linear_member_vacuum_factor`, `einstein_gauss_bonnet_vacuum_linear`). A vacuum needs $V = 0$, that is

$$A^2 = \frac{1 - 40\alpha_2 H^2}{8\alpha_2 H^2},$$

a real slope only for $0 < \alpha_2 H^2 \le 1/40$, and then the Λ that makes $\rho = 0$ (Notebook 12b, In [10] and In [11], figures 5 and 6). At the edge $\alpha_2 H^2 = 1/40$ the vacuum is static, $A = 0$, with $\Lambda = -21H^2 + 420H^2/40 = -10.5H^2$. With the third-order coupling the vacuum condition is, for each A, a straight line in the plane of $\alpha_2 H^2$ and $\alpha_3 H^4$ (Notebook 12b, In [12], figure 7). Whether nature has such couplings is not known; the Revision record makes no claim about their values.

The metric with b_4: what changes in the field equations. On 2026-10-14 the author gave the extra-time entries of the metric a second function $b_4(x_4) > 0$: $g_{x_5x_5} = g_{x_6x_6} = g_{x_7x_7} = -e^{-2a_4} b_4 \sin^{1/3} z$ (Chapter 3). Every derivation of Sections 12.3 to 12.12, and the four notebooks of this chapter, are made on the background $b_4 = 1$; the keys `lovelockTensors` to `fields` of `Revision/field_equations_a4/a4-equations.json` hold this case. The record's key `b4` holds the same equations for a general b_4 (the Wolfram verifier `Revision/field_equations_a4/wolfram/verify_field_equations_a4.wls`, whose report `wolfram-a4-report.json` treats the metric with b_4, and the sympy twin `Revision/field_equations_a4/python/check_b4_block.py` with the report `python-b4-block.json`, 11 of 11; `Revision/metric_b4/METRIC_CHANGE_PROVENANCE.md`). This section states what changes; it quotes the record, with the symbols $u = b_4'/b_4$.

The Einstein tensor gets two mixed components. For example (key `b4/lovelockTensors/E1`)

$$G^{x_4}{}_{x_0} = \frac{3}{2} H \cot z \, \frac{b_4'}{b_4}, \qquad G^{x_0}{}_{x_4} = -\frac{3}{2} H \tan z \, \frac{b_4'}{b_4},$$

and the diagonal components gain terms in b_4'/b_4 and b_4''/b_4; at $b_4 = 1$ they are those of Section 12.6. So "every off-diagonal left-hand side is zero" (Section 12.8) holds only for a constant b_4.

A changing b_4 needs a mixed source. The mixed equations read $\kappa q_{40} = \cot z \, (b_4'/b_4) \, Q$ and $\kappa q_{04} = -\tan z \, (b_4'/b_4) \, Q$ with a polynomial Q in H, a_4' and u (key `b4/generalSource/offDiagonalFactor_Q`); for Einstein gravity $Q = \frac{3}{2} H$. A source with $q_{40} = q_{04} = 0$, in particular every source that does not depend on x_0 and has no $T^{x_4}{}_{x_0}$, therefore forces $b_4' Q = 0$, and for Einstein gravity $b_4' = 0$ (key `b4/generalSource/offDiagonalStatement`).

The algebraic condition gains a b_4 term. For Einstein gravity $\kappa \, (p_3 + p_t - 2p_0) = \frac{1}{4}(b_4'/b_4)^2 + \frac{b_4''}{2b_4}$ (key `b4/generalSource/identity_einstein`); with $b_4' = b_4'' = 0$ it is the condition $p_3 + p_t = 2p_0$ of Section 12.9.

The conservation law gains the ordinary term of the extra times. The x_4 component becomes $\partial_4 \rho = -3a_4'(p_3 - p_t) - \frac{3}{2}\frac{b_4'}{b_4}(p_t + \rho) + \dots$ (the dots are the terms with q_{04}; key `b4/generalSource/conservation_x4`; Section 9.10).

Two second-order equations and a constraint. The $x_1 - x_5$ and x_0 components are linear in (a_4'', b_4'') with a nonsingular Jacobian, so they determine a_4'' and b_4'' from the source; the x_4 component is the constraint (key `b4/generalSource/secondOrderEinstein`).

The vacuum. With the author's weights ($w_1 = 1$, $w_2 = 1/32$, $w_3 = 1/1152$, $\Lambda = 10$) every vacuum solution has a constant b_4 and $a_4 = \pm H x_4 +$ const (`Revision/metric_b4/results/vacuum-b4.json`, 6 of 6 checks); the background $b_4 = 1$ of this chapter is of this kind.

The condensate. For a changing b_4 the gravitational term is $3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ (Chapter 6); the condensate of Section 12.26 becomes $\Phi = b_4^{-3/4} \Phi_1$ with a solution Φ_1 of the case $b_4 = 1$, and $b_4^{3/2} S$ is constant (key `b4/fields/condensate`).

Status: these statements are PROVED in the record (`python-b4-block.json` and `wolfram-a4-report.json`); this book quotes them and does not derive them.

12.13 Example: the field equations derived by computer algebra

The first notebook repeats everything of Sections 12.3 to 12.12 with exact computer algebra, starting from nothing but the metric, and compares every result with the Revision record. It computes the 37 Christoffel symbols, the 156 nonzero Riemann components, the Einstein tensor and the three Lovelock tensors (with its own GKD), checks them against the Rust program of the GKD record in all 64 components and against the classical Gauss-Bonnet formula, writes and reduces the field equations, checks the Bianchi identity and the conservation law, and specialises to Einstein gravity and to the linear member. It runs in about 20 seconds, prints 59 PASS lines and draws five figures. It needs no Rust.

12.14 How to run Notebook 12a

Step 1. What this notebook does and what it needs.

Notebook 12a (The field equations for a4 derived with sympy) is the file `Revision/textbook/notebooks/12a_a4_field_equations.ipynb` of the repository Dirac_claude. It derives with sympy, from the metric of the author, the Christoffel symbols, the Riemann, Ricci and Einstein tensors and the three Lovelock tensors (with the generalized Kronecker delta), writes the Einstein-Lovelock field equations for a4, reduces them to the constraint, the evolution equation, the hidden equation and the conditions on the source, checks the Bianchi identity, the conservation law of the source, the Einstein case (no vacuum, the null energy condition along x0) and the linear member a4 = A H x4, compares every result with the Revision records, and draws five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
```

```
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 12a_a4_field_equations.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12a_a4_field_equations.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 12a_a4_field_equations.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/12a.captions.json`
- `Revision/textbook/figures/12a_1_riemann_matrix.png`
- `Revision/textbook/figures/12a_2_einstein_components.png`
- `Revision/textbook/figures/12a_3_lovelock_heat_maps.png`
- `Revision/textbook/figures/12a_4_evolution_factor.png`
- `Revision/textbook/figures/12a_5_no_vacuum_gap.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files exist
ALL 59 CHECKS PASSED (notebook 12a)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/12a_a4_field_equations.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 12a_a4_field_equations.ipynb
python -m nbconvert --execute --inplace 12a_a4_field_equations.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cell of section 10, which computes the three Lovelock tensors, runs for a minute or more: it adds about five thousand products of curvature components in pure Python, which takes a few seconds on a fast computer and can take a few minutes on a slow one. The cell is not stuck: wait until the PASS lines appear below it.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

12.15 Notebook 12a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/12a_a4_field_equations.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook derives, step by step and with exact algebra (no rounding anywhere),
the field equations of gravity for the one unknown function $a_4(x_4)$ of the
author's metric. Starting from nothing but the metric, it

- writes the metric of the author in the computer-algebra package sympy and checks
  its signature and its determinant;
- computes the Christoffel symbols, the Riemann curvature tensor, the Ricci tensor,
  the Ricci scalar and the Einstein tensor;
- computes the three Lovelock tensors $E_{(1)}, E_{(2)}, E_{(3)}$ with the
  generalized Kronecker delta (the first one is the Einstein tensor, the second one
  the Gauss-Bonnet tensor), and checks the second one against the classical
  Gauss-Bonnet formula;
- writes the Einstein-Lovelock field equations with a general source and reduces
  them to a constraint, an evolution equation, a hidden-direction equation and the
```

  conditions every source must satisfy;
- checks the Bianchi identity and the conservation law of the source;
- specialises to Einstein gravity: no vacuum solution exists for $H > 0$, and every
  source violates the null energy condition along x_0;
- specialises to the linear member $a_4 = A H x_4 + a_0$ (exponentially deflating
  extra times for $A > 0$) and shows that the equations do not see the sign of A;
- compares every result with the Revision records and draws five teaching plots.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 12a (The field equations for a4 derived with sympy) is the file
‘Revision/textbook/notebooks/12a_a4_field_equations.ipynb‘ of the repository
Dirac_claude. It derives with sympy, from the metric of the author, the Christoffel
symbols, the Riemann, Ricci and Einstein tensors and the three Lovelock tensors (with the
generalized Kronecker delta), writes the Einstein-Lovelock field equations for a4,
reduces them to the constraint, the evolution equation, the hidden equation and the
conditions on the source, checks the Bianchi identity, the conservation law of the
source, the Einstein case (no vacuum, the null energy condition along x0) and the linear
member a4 = A H x4, compares every result with the Revision records, and draws five
teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy,
sympy and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:
```

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in

this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 12a_a4_field_equations.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the

address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12a_a4_field_equations.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 12a_a4_field_equations.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/12a.captions.json‘
- ‘Revision/textbook/figures/12a_1_riemann_matrix.png‘
- ‘Revision/textbook/figures/12a_2_einstein_components.png‘
- ‘Revision/textbook/figures/12a_3_lovelock_heat_maps.png‘
- ‘Revision/textbook/figures/12a_4_evolution_factor.png‘
- ‘Revision/textbook/figures/12a_5_no_vacuum_gap.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files exist
ALL 59 CHECKS PASSED (notebook 12a)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/12a_a4_field_equations.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 12a_a4_field_equations.ipynb
      python -m nbconvert --execute --inplace 12a_a4_field_equations.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cell of section 10, which computes the three Lovelock tensors, runs for a minute or more: it adds about five thousand products of curvature components in pure Python, which takes a few seconds on a fast computer and can take a few minutes on a slow one. The cell is not stuck: wait until the PASS lines appear below it.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a

check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 12a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 12a (The field equations for a4 derived with sympy) is the file
# Revision/textbook/notebooks/12a_a4_field_equations.ipynb of the repository
# Dirac_claude. It derives with sympy, from the metric of the author, the Christoffel
# symbols, the Riemann, Ricci and Einstein tensors and the three Lovelock tensors (with
# the generalized Kronecker delta), writes the Einstein-Lovelock field equations for a4,
# reduces them to the constraint, the evolution equation, the hidden equation and the
# conditions on the source, checks the Bianchi identity, the conservation law of the
# source, the Einstein case (no vacuum, the null energy condition along x0) and the
# linear member a4 = A H x4, compares every result with the Revision records, and draws
# five teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
# shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
```

```
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 12a_a4_field_equations.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
```

```
#     jupyter nbconvert --to notebook --execute --inplace 12a_a4_field_equations.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 12a_a4_field_equations.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/12a.captions.json
# - Revision/textbook/figures/12a_1_riemann_matrix.png
# - Revision/textbook/figures/12a_2_einstein_components.png
# - Revision/textbook/figures/12a_3_lovelock_heat_maps.png
# - Revision/textbook/figures/12a_4_evolution_factor.png
# - Revision/textbook/figures/12a_5_no_vacuum_gap.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all five figure files exist
#     ALL 59 CHECKS PASSED (notebook 12a)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/12a_a4_field_equations.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
```

```
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 12a_a4_field_equations.ipynb
#     python -m nbconvert --execute --inplace 12a_a4_field_equations.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - The cell of section 10, which computes the three Lovelock tensors, runs for a minute
#   or more: it adds about five thousand products of curvature components in pure Python,
#   which takes a few seconds on a fast computer and can take a few minutes on a slow
#   one. The cell is not stuck: wait until the PASS lines appear below it.
# - "FileNotFoundError" naming a file below the folder Revision: the notebook reads the
#   Revision records of the repository. Run it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone,
#   not on a copy of the notebook file alone.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "12a"  # this notebook: chapter 12, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
```

```
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
```

```
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 12a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook
```

- **Coordinates** $x_0, \dots, x_7$ (the author's names): x_1, x_2, x_3 are ordinary 3-space; x_4 is the time; x_5, x_6, x_7 are the three **extra times** (time-like directions); x_0 is the **hidden** space direction. In the code the positions 0 to 7 of a list stand for x_0 to x_7.
- **Metric** $g_{\mu\nu}$: the table of numbers that turns small coordinate steps into squared lengths. Here it is **diagonal**: only $g_{11}, \dots, g_{00}$ are not zero.
- **Scale factor**: the factor by which a length along a direction grows. 3-space has $e^{a_4}\sin^{1/6} z$, the extra times have $e^{-a_4}\sin^{1/6} z$.
- **Deflating**: shrinking. When a_4 grows, e^{-a_4} shrinks: the extra times deflate exponentially while 3-space inflates.
- **Christoffel symbol** $\Gamma^a{}_{bc}$: the combination of first derivatives of the metric that tells how the coordinate directions turn from point to point.
- **Riemann tensor** $R^{ab}{}_{cd}$: the curvature; built from the Christoffel symbols and their derivatives. **Ricci tensor** $R^h{}_j = R^{ah}{}_{aj}$ (sum over a), **Ricci scalar** $R = R^h{}_h$, **Einstein tensor** $G^h{}_j = R^h{}_j - \tfrac12 \delta^h_j R$.
- **Generalized Kronecker delta** (GKD) $\delta^{h_1 \dots h_n}_{j_1 \dots j_n}$: $+1$ when the lower list is an even rearrangement of the upper list, -1 when it is an odd rearrangement, 0 otherwise (also when an index repeats).
- **Lovelock tensor** $E_{(k)}$: a tensor built from k factors of the Riemann tensor with the GKD; $E_{(1)}$ is the Einstein tensor, $E_{(2)}$ the Gauss-Bonnet tensor. In 8 dimensions only $k = 1, 2, 3$ exist.
- **Source**, **energy-momentum tensor** $T^\mu{}_\nu$: the matter that bends the metric. Its parts: the energy density $\rho = -T^{x_4}{}_{x_4}$, the pressures p_3 (3-space), p_t (extra times), p_0 (hidden direction), and the mixed components $q_{40} = T^{x_4}{}_{x_0}$, $q_{04} = T^{x_0}{}_{x_4}$.
- **Couplings**: $\alpha_1, \alpha_2, \alpha_3$ (the weights of the three Lovelock tensors), Λ (the cosmological constant), κ (the strength of gravity). **Einstein gravity** is $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$.
- **Constraint**: an equation with no second derivative a_4''. **Evolution equation**: the equation that contains a_4''.
- **Bianchi identity**: the left-hand side of the field equations has zero divergence for every metric; **conservation**: the source must then have zero divergence too.
- **Vacuum**: no source, $T = 0$. **Null energy condition** along x_0: $\rho + p_0 \ge 0$.
- **sympy symbol**: a letter that stands for any number; ‘ad1‘ and ‘ad2‘ stand for $a_4' = da_4/dx_4$ and $a_4'' = d^2a_4/dx_4^2$.
- **PASS line**: a printed line that says a check passed; "reproduces" names the Revision record (report file and check name) that found the same result.

Text cell:

```
## 4. The physical and mathematical situation

The author's metric (the primordial gravitational field) is diagonal:

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4}s,\ e^{2a_4}s,\ e^{2a_4}s,\ -1,\
-e^{-2a_4}s,\ -e^{-2a_4}s,\ -e^{-2a_4}s\big),\qquad
s = \sin^{1/3} z,\quad z = 6 H x_0 \in (0, \pi/2),$$

in the order $x_0, \dots, x_7$, with $a_4 = a_4(x_4)$ and a constant $H > 0$. This is
```

```
the metric of 2026-10-14 on its background $b_4 = 1$: the second function
$b_4(x_4)$ multiplies the three extra-time components (section 10 says how the
notebook reads the Rust record, which keeps $b_4$ general). Its signs are
$(+,+,+,+,-,-,-,-)$: four space-like directions ($x_0, x_1, x_2, x_3$) and four
time-like ones ($x_4, \dots, x_7$), the signature (4,4).

The field equations of Einstein-Lovelock gravity are

$$\sum_{k=1}^{3} \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda \delta^\mu_\nu
= \kappa T^\mu{}_\nu,\qquad E_{(k)}{}^h{}_j = -\frac{P_{(k)}{}^h{}_j}{2^{k+1}},$$

$$P_{(k)}{}^h{}_j = \delta^{h h_1 \dots h_{2k}}_{j j_1 \dots j_{2k}}
R^{j_1 j_2}{}_{h_1 h_2} \cdots R^{j_{2k-1} j_{2k}}{}_{h_{2k-1} h_{2k}}$$

(a sum over every repeated index), with the source

$$T^\mu{}_\nu = \mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0)
+ q_{40}\ (\mu = x_4, \nu = x_0) + q_{04}\ (\mu = x_0, \nu = x_4).$$

Every statement this notebook checks is an exact algebraic identity (status
PROVED): sympy computes with symbols, not with rounded numbers. The plots
evaluate the exact formulas at chosen numbers, in units in which $H = 1$.
```

Text cell:

```
## 5. The Revision records this notebook reproduces

The Revision record contains two independent derivations of these equations (one
in the Wolfram Language, one in sympy) and an independent check by the lead. Each
writes a report file: a list of checks with a name and a verdict. The next cell
defines a helper that looks up a check by its name, and the helper ‘reproduces‘:
a check that passes only when this notebook’s own result holds AND the record
lists the named check with the verdict PASS. ‘reproduces‘ collects its two printed
lines (PASS and reproduces) in a text buffer and prints them with one call, so
that they always stay together in the output. Finally the cell prints, for each of
the four reports, how many of its checks passed, and checks that every one did:
a record with a failed check stops the notebook here. The numbers of checks are
printed, not fixed in advance, because a record may gain checks when it is
extended.
```

In [2]:

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory

PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
LEAD = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"  # lead check
EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the equations
LOVELOCK = "Revision/gkd_lovelock/results/lovelock-tensors.json"  # Rust GKD output
```

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict ("PASS" or "FAIL") of the check name in a report, None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS.  Its
    printed lines (PASS and reproduces) are collected in a text buffer and printed
    by one print call, so that they always stay together in the cell's output."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name, record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # all lines at once


for report_file in (PY, WL, LEAD, EMT):
    verdicts = [entry["verdict"] for entry in read_json(report_file)["checks"]]
    passed = verdicts.count("PASS")  # how many checks of the record passed
    say(f"{report_file}: {passed} of {len(verdicts)} checks PASS")
    check(len(verdicts) > 0 and passed == len(verdicts),
          f"every check of the record {Path(report_file).name} passed")
```

Out [2]:

```
Revision/field_equations_a4/reports/python-a4-report.json: 63 of 63 checks PASS
PASS every check of the record python-a4-report.json passed
Revision/field_equations_a4/reports/wolfram-a4-report.json: 64 of 64 checks PASS
PASS every check of the record wolfram-a4-report.json passed
Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json: 15 of 15 checks PASS
PASS every check of the record einstein-gauss-bonnet-a4.json passed
Revision/lead_checks/reports/emt-divergence-and-spin-connection.json: 10 of 10 checks
    PASS
PASS every check of the record emt-divergence-and-spin-connection.json passed
```

Text cell:

```
## 6. The metric in sympy

The next cell makes the symbols: $H$ (positive), $z = 6Hx_0$ (positive; we work on
the patch $0 < z < \pi/2$), the time $x_4$ and the unknown function $a_4(x_4)$. Then
it writes the eight diagonal entries of the metric as a Python list ‘g‘ (position 0
is $g_{11}$, position 7 is $g_{00}$) and prints them. sympy writes a power as ‘**‘
and the function $a_4(x_4)$ as ‘a4(x4)‘.
```

In [3]:

```
import itertools  # loops over all combinations of indices

import numpy as np  # arrays of numbers, used for the plots
import sympy as sp  # exact algebra with symbols

H = sp.symbols("H", positive=True)  # the constant H of the metric, H > 0
z = sp.symbols("z", positive=True)  # the hidden angle z = 6 H x0, 0 < z < pi/2
x4 = sp.symbols("x4", real=True)  # the time x4
a4 = sp.Function("a4")(x4)  # the unknown function a4(x4)
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
N = 8  # the number of dimensions
warp = sp.sin(z) ** sp.Rational(1, 3)  # the warp factor sin(z)^(1/3)
g = ([sp.cot(z) ** 2]  # g00 = cot(z)^2: the hidden direction
     + [sp.exp(2 * a4) * warp] * 3  # g11 = g22 = g33: 3-space, factor e^(2 a4)
     + [sp.Integer(-1)]  # g44 = -1: the time x4
     + [-sp.exp(-2 * a4) * warp] * 3)  # g55 = g66 = g77: extra times, e^(-2 a4)
for name, entry in zip(NAMES, g):
    say(f"g[{name},{name}] = {entry}")
```

Out [3]:

```
g[x0,x0] = cot(z)**2
g[x1,x1] = exp(2*a4(x4))*sin(z)**(1/3)
g[x2,x2] = exp(2*a4(x4))*sin(z)**(1/3)
g[x3,x3] = exp(2*a4(x4))*sin(z)**(1/3)
g[x4,x4] = -1
g[x5,x5] = -exp(-2*a4(x4))*sin(z)**(1/3)
g[x6,x6] = -exp(-2*a4(x4))*sin(z)**(1/3)
g[x7,x7] = -exp(-2*a4(x4))*sin(z)**(1/3)
```

Text cell:

```
The next cell checks two facts. (1) The signs of the diagonal entries, at any
point (here $a_4 = 0$, $z = \pi/4$), are $(+,+,+,+,-,-,-,-)$. (2) The determinant
of a diagonal matrix is the product of its diagonal entries:
$\det g = (e^{2a_4}s)^3 \cdot (-1) \cdot (-e^{-2a_4}s)^3 \cot^2 z
= s^6 \cot^2 z = \sin^2 z \cot^2 z = \cos^2 z$, because $e^{6a_4} e^{-6a_4} = 1$ and
$(-1)(-1)^3 = 1$. So $\sqrt{|\det g|} = \cos z$ (positive on the patch) does not
depend on $a_4$: the volume factor $e^{3a_4}$ of the inflating 3-space and
$e^{-3a_4}$ of the deflating extra times cancel.
```

In [4]:

```
point = {a4: 0, z: sp.pi / 4}  # any point of the patch gives the same signs
signs = [int(sp.sign(entry.subs(point))) for entry in g]
say(f"signs of the diagonal entries: {signs}")
check(signs == [1, 1, 1, 1, -1, -1, -1, -1],
      "the signature is (4,4): eta = diag(+,+,+,+,-,-,-,-) in the order x0..x7")
```

```
det_g = sp.Mul(*g)  # the product of the eight diagonal entries
say(f"det g = {sp.simplify(det_g)}")
reproduces(sp.simplify(det_g - sp.cos(z) ** 2) == 0,
           "det g = cos(z)^2, so sqrt|det g| = cos z, independent of a4",
           WL, "sqrt_abs_det_g_is_cos_z")
```

Out [4]:

```
signs of the diagonal entries: [1, 1, 1, 1, -1, -1, -1, -1]
PASS the signature is (4,4): eta = diag(+,+,+,+,-,-,-,-) in the order x0..x7
det g = cos(z)**2
PASS det g = cos(z)^2, so sqrt|det g| = cos z, independent of a4
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    sqrt_abs_det_g_is_cos_z
```

Text cell:

```
## 7. Derivatives and the Christoffel symbols

The metric depends on $x_4$ and on $x_0$ only, and on $x_0$ only through
$z = 6Hx_0$. By the chain rule $\partial/\partial x_0 = (dz/dx_0)\,\partial/\partial z
= 6H\,\partial/\partial z$. The function ‘d(expr, mu)‘ below takes the partial
derivative with respect to the coordinate at position ‘mu‘.

For a diagonal metric the Christoffel symbols
$\Gamma^a{}_{bc} = \tfrac12 g^{ad}(\partial_b g_{dc} + \partial_c g_{db}
- \partial_d g_{bc})$ reduce (only $d = a$ survives, and $g^{aa} = 1/g_{aa}$) to

$$\Gamma^a{}_{bc} = \frac{1}{2g_{aa}}\big(\delta_{ac}\,\partial_b g_{aa}
+ \delta_{ab}\,\partial_c g_{aa} - \delta_{bc}\,\partial_a g_{bb}\big)$$

(no sum over $a$ or $b$). The cell computes all $8^3 = 512$ of them, counts those
that are not zero (37: the 25 distinct ones printed by the cell after it, 12 of
which appear twice because $\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$ with $b \ne c$) and
checks that symmetry.
```

In [5]:

```
def d(expr, mu):
    """The partial derivative of expr with respect to the coordinate at position
    mu (0..7 for x0..x7); d/dx0 = 6 H d/dz by the chain rule."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7


Gamma = [[[sp.Integer(0)] * N for _ in range(N)] for _ in range(N)]  # 512 zeros
for a, b, c in itertools.product(range(N), repeat=3):  # every index triple
    value = 0
```

```
    if a == c:
        value += d(g[a], b)
    if a == b:
        value += d(g[a], c)
    if b == c:
        value -= d(g[b], a)
    Gamma[a][b][c] = value / (2 * g[a])  # Gamma^a_bc of a diagonal metric
nonzero = [(a, b, c) for a, b, c in itertools.product(range(N), repeat=3)
           if Gamma[a][b][c] != 0]
say(f"{len(nonzero)} of the 512 Christoffel symbols are not zero")
check(len(nonzero) == 37,
      "37 nonzero Christoffel symbols (25 distinct, 12 of them twice)")
check(all(Gamma[a][b][c] == Gamma[a][c][b] for a, b, c in nonzero),
      "Gamma^a_bc = Gamma^a_cb (symmetric in the two lower indices)")
```

Out [5]:

```
37 of the 512 Christoffel symbols are not zero
PASS 37 nonzero Christoffel symbols (25 distinct, 12 of them twice)
PASS Gamma^a_bc = Gamma^a_cb (symmetric in the two lower indices)
```

Text cell:

```
The next cell prints the Christoffel symbols that are not zero, each pair
$\Gamma^a{}_{bc} = \Gamma^a{}_{cb}$ once. The helper ‘readable‘ writes the
derivatives $a_4'$ and $a_4''$ as the symbols ‘ad1‘ and ‘ad2‘ and simplifies.
For example $\Gamma^{x_1}{}_{x_1 x_4} = \tfrac12 g^{11}\partial_4 g_{11}
= \tfrac12 \cdot 2a_4' = a_4'$ and $\Gamma^{x_5}{}_{x_4 x_5} = -a_4'$: the sign
difference between the inflating and the deflating directions.
```

In [6]:

```
# The first, second and third derivatives of a4 as plain symbols:
ad1, ad2, ad3 = sp.symbols("ad1 ad2 ad3", real=True)


def derivatives_as_symbols(expr):
    """Replace the derivatives of a4 by the symbols ad1, ad2, ad3."""
    expr = expr.subs(sp.Derivative(a4, (x4, 3)), ad3)  # highest derivative first
    expr = expr.subs(sp.Derivative(a4, (x4, 2)), ad2)
    return expr.subs(sp.Derivative(a4, x4), ad1)


def readable(expr):
    """expr with ad1, ad2 for the derivatives of a4, simplified for printing."""
    return sp.simplify(derivatives_as_symbols(expr))


for a, b, c in nonzero:
    if b <= c:  # print each symmetric pair once
        say(f"Gamma^{NAMES[a]}_{NAMES[b]}{NAMES[c]} = {readable(Gamma[a][b][c])}")
```

Out [6]:

```
Gamma^x0_x0x0 = -12*H/sin(2*z)
Gamma^x0_x1x1 = -H*exp(2*a4(x4))*sin(z)**(4/3)/cos(z)
Gamma^x0_x2x2 = -H*exp(2*a4(x4))*sin(z)**(4/3)/cos(z)
Gamma^x0_x3x3 = -H*exp(2*a4(x4))*sin(z)**(4/3)/cos(z)
Gamma^x0_x5x5 = H*exp(-2*a4(x4))*sin(z)**(4/3)/cos(z)
Gamma^x0_x6x6 = H*exp(-2*a4(x4))*sin(z)**(4/3)/cos(z)
Gamma^x0_x7x7 = H*exp(-2*a4(x4))*sin(z)**(4/3)/cos(z)
Gamma^x1_x0x1 = H/tan(z)
Gamma^x1_x1x4 = ad1
Gamma^x2_x0x2 = H/tan(z)
Gamma^x2_x2x4 = ad1
Gamma^x3_x0x3 = H/tan(z)
Gamma^x3_x3x4 = ad1
Gamma^x4_x1x1 = ad1*exp(2*a4(x4))*sin(z)**(1/3)
Gamma^x4_x2x2 = ad1*exp(2*a4(x4))*sin(z)**(1/3)
Gamma^x4_x3x3 = ad1*exp(2*a4(x4))*sin(z)**(1/3)
Gamma^x4_x5x5 = ad1*exp(-2*a4(x4))*sin(z)**(1/3)
Gamma^x4_x6x6 = ad1*exp(-2*a4(x4))*sin(z)**(1/3)
Gamma^x4_x7x7 = ad1*exp(-2*a4(x4))*sin(z)**(1/3)
Gamma^x5_x0x5 = H/tan(z)
Gamma^x5_x4x5 = -ad1
Gamma^x6_x0x6 = H/tan(z)
Gamma^x6_x4x6 = -ad1
Gamma^x7_x0x7 = H/tan(z)
Gamma^x7_x4x7 = -ad1
```

Text cell:

```
## 8. The Riemann tensor

The Riemann tensor in the sign convention of Misner, Thorne and Wheeler (MTW) is

$$R^a{}_{bmn} = \partial_m \Gamma^a{}_{bn} - \partial_n \Gamma^a{}_{bm}
+ \Gamma^a{}_{me}\Gamma^e{}_{bn} - \Gamma^a{}_{ne}\Gamma^e{}_{bm}$$

(sum over $e$), and with the second index raised,
$R^{ab}{}_{mn} = g^{bb} R^a{}_{bmn}$. It is antisymmetric in $(a, b)$ and in
$(m, n)$, so only $a \ne b$, $m \ne n$ can be nonzero.

To bring every component to one canonical form, the helper ‘clean‘ writes every
function of $z$ through $c = \cot z > 0$: $\sin z = 1/\sqrt{1 + c^2}$,
$\cos z = c/\sqrt{1 + c^2}$, $\tan z = 1/c$ (from $\cot z = \cos z/\sin z$ and
$\sin^2 z + \cos^2 z = 1$), first expanding $\sin 2z = 2\sin z\cos z$. In the code
$c$ is called ‘cz‘.
```

In [7]:

```
cz = sp.symbols("cz", positive=True)  # cz = cot z > 0 on the patch
TRIG = {sp.sin(z): 1 / sp.sqrt(1 + cz ** 2), sp.cos(z): cz / sp.sqrt(1 + cz ** 2),
```

```
        sp.tan(z): 1 / cz, sp.cot(z): cz}  # every function of z through cot z


def clean(expr):
    """expr in canonical form: derivatives of a4 as ad1, ad2, ad3; functions of z
    through cz; powers combined; common factors cancelled; expanded."""
    expr = sp.expand_trig(derivatives_as_symbols(expr)).subs(TRIG)
    return sp.expand(sp.cancel(sp.powsimp(sp.expand(expr), force=True)))


R = {}  # R[(a, b, m, n)] = R^{ab}_{mn}; only the components that are not zero
for a, b, m, n in itertools.product(range(N), repeat=4):
    if a == b or m == n:
        continue  # antisymmetry: zero when the two indices of a pair are equal
    value = (d(Gamma[a][b][n], m) - d(Gamma[a][b][m], n)
             + sum(Gamma[a][m][e] * Gamma[e][b][n] - Gamma[a][n][e] * Gamma[e][b][m]
                   for e in range(N)))  # R^a_bmn in the MTW convention
    if value != 0:
        value = clean(value / g[b])  # raise the index b: multiply by g^bb
        if value != 0:
            R[(a, b, m, n)] = value
ordered = {key: value for key, value in R.items()
           if key[0] < key[1] and key[2] < key[3]}  # a < b and m < n
report("nonzero components R^ab_mn of the 4096", len(R))
report("nonzero components with a < b and m < n", len(ordered))
```

Out [7]:

```
RESULT nonzero components R^ab_mn of the 4096 = 156
RESULT nonzero components with a < b and m < n = 39
```

Text cell:

```
The next cell checks four facts that the Revision record found. (1) There are 156
nonzero components, 39 of them with $a < b$, $m < n$. (2) Not one contains the warp
factor or $e^{a_4}$: the curvature depends only on $H$, $a_4'$, $a_4''$ and
$\cot z$. (3) Every component is a polynomial in $H, a_4', a_4'', \cot z$ divided
at most by $\cot z$ (a *Laurent polynomial* in $\cot z$). (4) The pair
antisymmetry $R^{ab}{}_{mn} = -R^{ba}{}_{mn} = -R^{ab}{}_{nm}$.
```

In [8]:

```
reproduces(len(R) == 156, "156 nonzero components R^ab_mn of the 4096",
           PY, "mixed_riemann_free_of_warp_and_a4")
reproduces(len(ordered) == 39, "39 nonzero components with a < b and m < n",
           WL, "mixed_riemann_free_of_warp_and_a4")
free = all(not v.has(a4) and not v.has(z) and not v.has(sp.exp) for v in R.values())
reproduces(free, "R^ab_mn is free of the warp factor and of e^(a4)",
           PY, "mixed_riemann_free_of_warp_and_a4")
laurent = all(sp.expand(v * cz).is_polynomial(H, ad1, ad2, cz) for v in R.values())
reproduces(laurent, "every R^ab_mn is a Laurent polynomial in cot z",
```

```
          PY, "riemann_entries_laurent")
antisymmetric = all(R.get((b, a, m, n)) == -v and R.get((a, b, n, m)) == -v
                    for (a, b, m, n), v in R.items())
reproduces(antisymmetric, "R^ab_mn = -R^ba_mn = -R^ab_nm",
          PY, "riemann_pair_antisymmetry")
```

Out [8]:

```
PASS 156 nonzero components R^ab_mn of the 4096
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    mixed_riemann_free_of_warp_and_a4
PASS 39 nonzero components with a < b and m < n
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    mixed_riemann_free_of_warp_and_a4
PASS R^ab_mn is free of the warp factor and of e^(a4)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    mixed_riemann_free_of_warp_and_a4
PASS every R^ab_mn is a Laurent polynomial in cot z
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    riemann_entries_laurent
PASS R^ab_mn = -R^ba_mn = -R^ab_nm
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    riemann_pair_antisymmetry
```

Text cell:

```
The next cell prints the 39 components with $a < b$, $m < n$, grouped by their
value; each member is written ‘x1x2|x1x2‘ for $R^{x_1x_2}{}_{x_1x_2}$. Read the
groups as follows: a pair of two 3-space directions, and a pair of two extra
times, has curvature $(a_4')^2 - H^2$; a 3-space direction with an extra time has
$-(a_4')^2 - H^2$; a pair with the hidden direction has $-H^2$; a 3-space
direction with the time has $(a_4')^2 + a_4''$ and an extra time with the time
$(a_4')^2 - a_4''$. The two remaining groups (12 components, written with ‘cz‘
for $\cot z$) connect a pair that contains the time $x_4$ with the pair in which
$x_4$ is replaced by $x_0$; they are proportional to $H a_4'$, so they vanish when
$a_4$ is constant.
```

In [9]:

```
def pair_name(a, b):
    return NAMES[a] + NAMES[b]


groups = {}  # printed value -> the components that have it
for (a, b, m, n), value in ordered.items():
    label = f"{pair_name(a, b)}|{pair_name(m, n)}"
    groups.setdefault(str(value), []).append(label)
for value, members in groups.items():
    say(f"{value} ({len(members)}): " + " ".join(members))
```

Out [9]:

```
-H**2 (6): x0x1|x0x1 x0x2|x0x2 x0x3|x0x3 x0x5|x0x5 x0x6|x0x6 x0x7|x0x7
H*ad1/cz (6): x0x1|x1x4 x0x2|x2x4 x0x3|x3x4 x0x5|x4x5 x0x6|x4x6 x0x7|x4x7
-H**2 + ad1**2 (6): x1x2|x1x2 x1x3|x1x3 x2x3|x2x3 x5x6|x5x6 x5x7|x5x7 x6x7|x6x7
-H*ad1*cz (6): x1x4|x0x1 x2x4|x0x2 x3x4|x0x3 x4x5|x0x5 x4x6|x0x6 x4x7|x0x7
ad1**2 + ad2 (3): x1x4|x1x4 x2x4|x2x4 x3x4|x3x4
-H**2 - ad1**2 (9): x1x5|x1x5 x1x6|x1x6 x1x7|x1x7 x2x5|x2x5 x2x6|x2x6 x2x7|x2x7 x3x5|x3x5
    x3x6|x3x6 x3x7|x3x7
ad1**2 - ad2 (3): x4x5|x4x5 x4x6|x4x6 x4x7|x4x7
```

Text cell:

```
The next cell draws the 39 components (and their zeros) as a heat map: a 28 by 28
table whose rows are the pairs $(a, b)$ with $a < b$ and whose columns are the
pairs $(m, n)$ with $m < n$, evaluated at the sample point $H = 1$,
$a_4' = 0.6$, $a_4'' = 0.3$, $z = \pi/3$ (so $\cot z = 1/\sqrt3$). Red is
positive, blue negative, white zero.
```

In [10]:

```
PAIRS28 = [(a, b) for a in range(N) for b in range(a + 1, N)]  # the 28 pairs
SAMPLE = {H: 1, ad1: sp.Rational(3, 5), ad2: sp.Rational(3, 10),
          cz: 1 / sp.sqrt(3)}  # H = 1, a4' = 0.6, a4'' = 0.3, z = pi/3
table = np.zeros((28, 28))
for i, (a, b) in enumerate(PAIRS28):
    for j, (m, n) in enumerate(PAIRS28):
        if (a, b, m, n) in R:
            table[i, j] = float(R[(a, b, m, n)].subs(SAMPLE))
limit = np.abs(table).max()  # the same scale for both signs
labels = [pair_name(a, b) for a, b in PAIRS28]
fig, ax = plt.subplots(figsize=(7.5, 6.6))
image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
ax.set_xticks(range(28))
ax.set_xticklabels(labels, rotation=90, fontsize=6)
ax.set_yticks(range(28))
ax.set_yticklabels(labels, fontsize=6)
ax.set_xlabel("lower index pair $(m, n)$")
ax.set_ylabel("upper index pair $(a, b)$")
ax.grid(False)
ax.set_title("Riemann tensor $R^{ab}{}_{mn}$ at $H=1$, $a_4'=0.6$, "
             "$a_4''=0.3$, $z=\\pi/3$")
fig.colorbar(image, ax=ax, shrink=0.8, label="value (units of $H^2$)")
save_figure(fig, "riemann_matrix",
            "The Riemann tensor $R^{ab}{}_{mn}$ of the author's metric as a 28 by "
            "28 table: rows are the index pairs $(a, b)$ with $a < b$, columns the "
            "pairs $(m, n)$ with $m < n$, colour the value in units of $H^2$ at "
            "$H = 1$, $a_4' = 0.6$, $a_4^{\\prime\\prime} = 0.3$, $z = \\pi/3$ (red "
            "positive, blue negative, white zero). Almost all of the curvature sits "
            "on the "
            "diagonal, one number per pair of directions; the only entries off the "
            "diagonal couple a pair that contains the time $x_4$ to the pair in "
            "which $x_4$ is replaced by $x_0$ (for example $x_1x_4$ and $x_1x_0$), "
            "and they are proportional to $H a_4'$.")
```

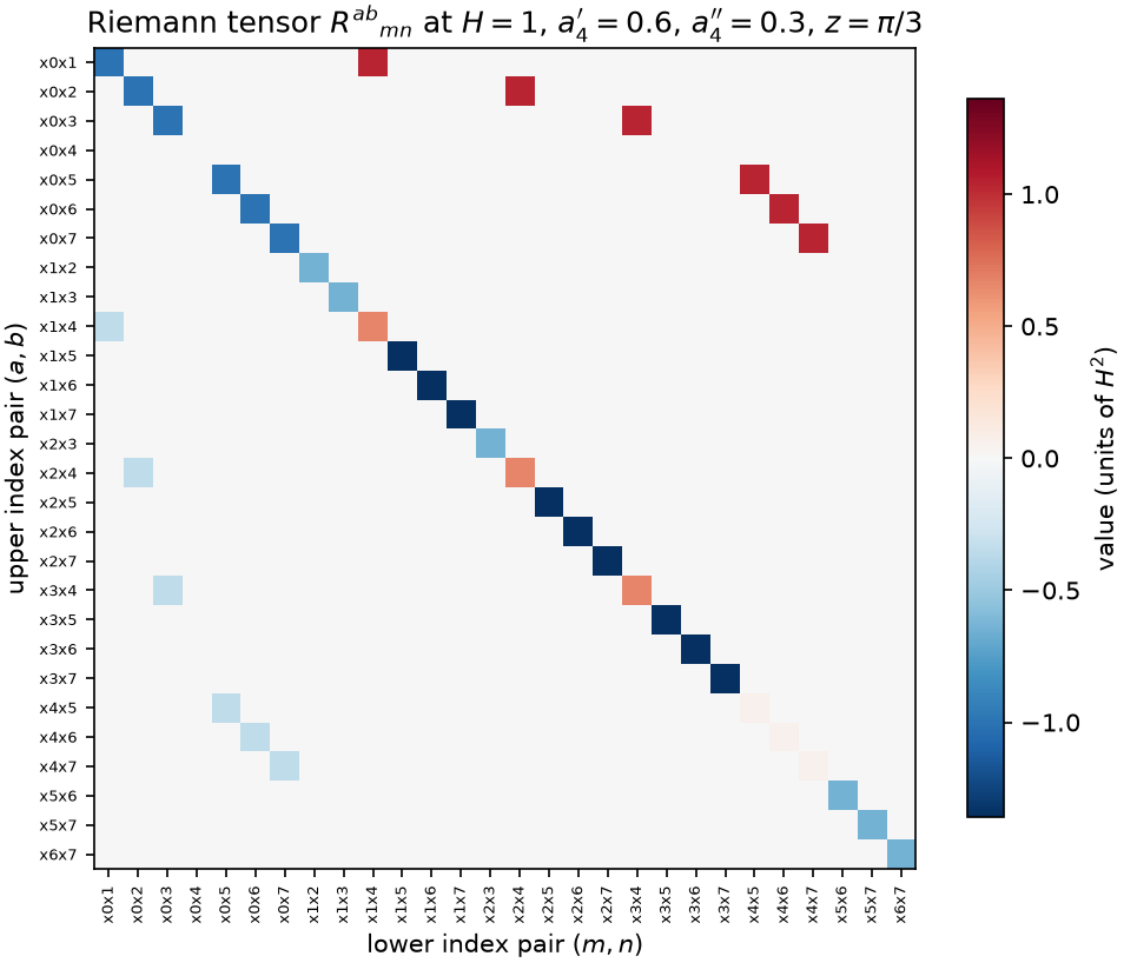

Figure 319: The Riemann tensor $R^{ab}{}_{mn}$ of the author's metric as a 28 by 28 table: rows are the index pairs (a, b) with $a < b$, columns the pairs (m, n) with $m < n$, colour the value in units of H^2 at $H = 1$, $a_4' = 0.6$, $a_4'' = 0.3$, $z = \pi/3$ (red positive, blue negative, white zero). Almost all of the curvature sits on the diagonal, one number per pair of directions; the only entries off the diagonal couple a pair that contains the time x_4 to the pair in which x_4 is replaced by x_0 (for example x_1x_4 and x_1x_0), and they are proportional to Ha_4'. (Notebook 12a, figure 1.)

Out [10]:

```
Figure 12a.1 saved as Revision/textbook/figures/12a_1_riemann_matrix.png
```

Text cell:

```
## 9. The Ricci tensor, the Ricci scalar and the Einstein tensor

The Ricci tensor with one index up is $R^h{}_j = \sum_a R^{ah}{}_{aj}$, the Ricci
scalar $R = \sum_h R^h{}_h$, and the Einstein tensor
$G^h{}_j = R^h{}_j - \tfrac12\delta^h_j R$. The next cell computes all 64
components of each and prints $R$ and the diagonal of $G$.
```

In [11]:

```
Ricci = [[sp.expand(sum(R.get((a, h, a, j), 0) for a in range(N)))
          for j in range(N)] for h in range(N)]  # R^h_j
ricci_scalar = sp.expand(sum(Ricci[h][h] for h in range(N)))
G = [[sp.expand(Ricci[h][j] - (ricci_scalar / 2 if h == j else 0))
      for j in range(N)] for h in range(N)]  # G^h_j
report("Ricci scalar R", ricci_scalar)
for h in range(N):
    say(f"G^{NAMES[h]}_{NAMES[h]} = {G[h][h]}")
```

Out [11]:

```
RESULT Ricci scalar R = -42*H**2 + 6*ad1**2
G^x0_x0 = 15*H**2 - 3*ad1**2
G^x1_x1 = 15*H**2 - 3*ad1**2 + ad2
G^x2_x2 = 15*H**2 - 3*ad1**2 + ad2
G^x3_x3 = 15*H**2 - 3*ad1**2 + ad2
G^x4_x4 = 21*H**2 + 3*ad1**2
G^x5_x5 = 15*H**2 - 3*ad1**2 - ad2
G^x6_x6 = 15*H**2 - 3*ad1**2 - ad2
G^x7_x7 = 15*H**2 - 3*ad1**2 - ad2
```

Text cell:

```
The next cell checks the structure of $G$ and compares it with the records: every
off-diagonal component is zero (in particular $G^{x_4}{}_{x_0} = 0$); the three
3-space components are equal, and so are the three extra-time components; nothing
depends on $x_0$ (no $\cot z$ is left); and the four distinct components are

$$G^{x_4}{}_{x_4} = 3(a_4')^2 + 21H^2,\quad
G^{x_1}{}_{x_1} = a_4'' - 3(a_4')^2 + 15H^2,$$
$$G^{x_5}{}_{x_5} = -a_4'' - 3(a_4')^2 + 15H^2,\quad
G^{x_0}{}_{x_0} = -3(a_4')^2 + 15H^2.$$
```

In [12]:

```
reproduces(all(G[h][j] == 0 for h in range(N) for j in range(N) if h != j),
           "G^h_j is diagonal (in particular G^x4_x0 = G^x0_x4 = 0)",
           LEAD, "einstein_off_diagonal_zero")
reproduces(G[1][1] == G[2][2] == G[3][3] and G[5][5] == G[6][6] == G[7][7],
           "G^x1_x1 = G^x2_x2 = G^x3_x3 and G^x5_x5 = G^x6_x6 = G^x7_x7",
           LEAD, "einstein_isotropy")
reproduces(not any(G[h][j].has(cz) for h in range(N) for j in range(N)),
           "every component of G is independent of x0",
           LEAD, "einstein_x0_independent")
expected = {4: 3 * ad1 ** 2 + 21 * H ** 2, 1: ad2 - 3 * ad1 ** 2 + 15 * H ** 2,
            5: -ad2 - 3 * ad1 ** 2 + 15 * H ** 2, 0: -3 * ad1 ** 2 + 15 * H ** 2}
reproduces(all(sp.expand(G[i][i] - v) == 0 for i, v in expected.items()),
           "the four Einstein components agree with the record",
           PY, "einstein_components")
```

Out [12]:

```
PASS G^h_j is diagonal (in particular G^x4_x0 = G^x0_x4 = 0)
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_off_diagonal_zero
PASS G^x1_x1 = G^x2_x2 = G^x3_x3 and G^x5_x5 = G^x6_x6 = G^x7_x7
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_isotropy
PASS every component of G is independent of x0
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_x0_independent
PASS the four Einstein components agree with the record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_components
```

Text cell:

```
The next cell draws the four distinct components of the Einstein tensor as
functions of $a_4'/H$ from $-3$ to $3$, in units of $H^2$. The 3-space and the
extra-time components are drawn twice: with $a_4'' = 0$ (they coincide with the
hidden one) and with $a_4'' = H^2$ (dashed: they split by $\pm a_4''$). This is
the origin of the evolution equation of section 11: only the difference of the
3-space and the extra-time components contains $a_4''$.
```

In [13]:

```
u = np.linspace(-3.0, 3.0, 301)  # a4'/H
fig, ax = plt.subplots()
ax.plot(u, 3 * u ** 2 + 21, label="$G^{x_4}{}_{x_4}$ (constraint)")
ax.plot(u, 15 - 3 * u ** 2, color="black",
        label="$G^{x_0}{}_{x_0}$ $=$ $G^{x_1}{}_{x_1}$ $=$ $G^{x_5}{}_{x_5}$ "
              "at $a_4''=0$")
ax.plot(u, 15 - 3 * u ** 2 + 1, "--", label="$G^{x_1}{}_{x_1}$ at $a_4''=H^2$")
ax.plot(u, 15 - 3 * u ** 2 - 1, "--", label="$G^{x_5}{}_{x_5}$ at $a_4''=H^2$")
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("component of $G^\\mu{}_\\nu$ (units of $H^2$)")
```

```
ax.set_title("The Einstein tensor of the author's metric")
ax.legend(fontsize=8)
save_figure(fig, "einstein_components",
            "The four distinct components of the Einstein tensor of the author's "
            "metric as functions of $a_4'/H$, in units of $H^2$: the time component "
            "$G^{x_4}{}_{x_4} = 3(a_4')^2 + 21H^2$ (top curve), the hidden "
            "component $G^{x_0}{}_{x_0} = 15H^2 - 3(a_4')^2$ (black), and the "
            "3-space and extra-time components, which equal the hidden one when "
            "the second derivative $a_4^{\\prime\\prime}$ is zero and split by "
            "$+a_4^{\\prime\\prime}$ and $-a_4^{\\prime\\prime}$ (dashed, drawn for "
            "$a_4^{\\prime\\prime} = H^2$). The time component is never smaller than "
            "$21H^2$, and only the difference of the 3-space and extra-time "
            "components feels $a_4^{\\prime\\prime}$.")
```

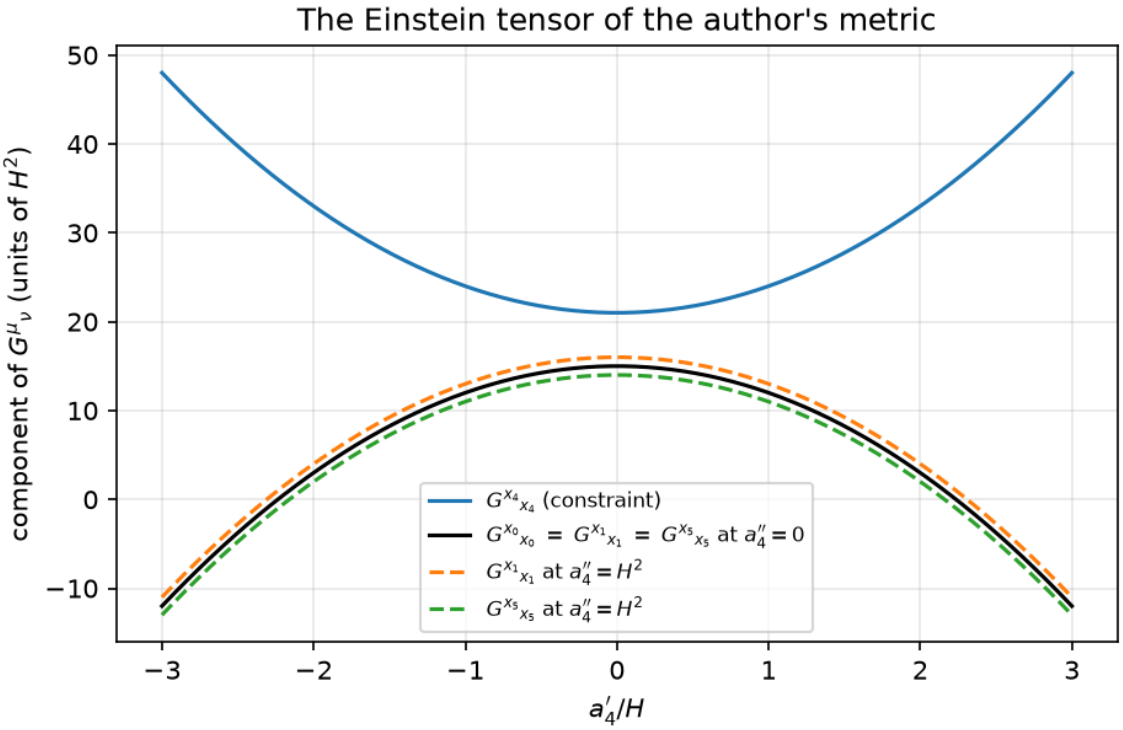

Figure 320: The four distinct components of the Einstein tensor of the author's metric as functions of a_4'/H, in units of H^2: the time component $G^{x_4}{}_{x_4} = 3(a_4')^2 + 21H^2$ (top curve), the hidden component $G^{x_0}{}_{x_0} = 15H^2 - 3(a_4')^2$ (black), and the 3-space and extra-time components, which equal the hidden one when the second derivative a_4'' is zero and split by $+a_4''$ and $-a_4''$ (dashed, drawn for $a_4'' = H^2$). The time component is never smaller than $21H^2$, and only the difference of the 3-space and extra-time components feels a_4''. (Notebook 12a, figure 2.)

Out [13]:

```
Figure 12a.2 saved as Revision/textbook/figures/12a_2_einstein_components.png
```

Text cell:

```
## 10. The three Lovelock tensors with the generalized Kronecker delta

The GKD $\delta^{h_1 \dots h_n}_{j_1 \dots j_n}$ is the determinant of the
$n \times n$ matrix of ordinary Kronecker deltas $\delta^{h_r}_{j_s}$. That
determinant is $0$ when two upper (or two lower) indices are equal (two equal
rows), and otherwise the sign of the permutation that turns the upper list into
the lower list (or $0$ when the two lists hold different indices). The next cell
implements it by counting *inversions*: pairs of positions that appear in the
wrong order. It prints three small examples and checks that 9 indices in 8
dimensions always give 0 (two of them must be equal): so there is no fourth
Lovelock tensor in 8 dimensions.
```

In [14]:

```
def gkd(upper, lower):
    """The generalized Kronecker delta: +1 (-1) if lower is an even (odd)
    rearrangement of upper without a repeated index, otherwise 0."""
    if len(set(upper)) != len(upper) or sorted(upper) != sorted(lower):
        return 0
    position = [upper.index(index) for index in lower]  # where each one sits
    inversions = sum(1 for i in range(len(position))
                     for j in range(i + 1, len(position))
                     if position[i] > position[j])
    return -1 if inversions % 2 else 1


say(f"gkd([1, 2], [1, 2]) = {gkd([1, 2], [1, 2])}, gkd([1, 2], [2, 1]) = "
    f"{gkd([1, 2], [2, 1])}, gkd([1, 1], [1, 1]) = {gkd([1, 1], [1, 1])}")
nine = list(range(8)) + [0]  # nine indices from eight values: one repeats
reproduces(gkd(nine, nine) == 0 and gkd(list(range(8)), list(range(8))) == 1,
           "a GKD of 9 indices in 8 dimensions is 0: P_(4) = 0",
           WL, "P4_vanishes_pigeonhole")
```

Out [14]:

```
gkd([1, 2], [1, 2]) = 1, gkd([1, 2], [2, 1]) = -1, gkd([1, 1], [1, 1]) = 0
PASS a GKD of 9 indices in 8 dimensions is 0: P_(4) = 0
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    P4_vanishes_pigeonhole
```

Text cell:

```
In $P_{(k)}{}^h{}_j$ every curvature factor $R^{j_1 j_2}{}_{h_1 h_2}$ appears with
its index pairs in both orders. Exchanging $j_1, j_2$ changes the sign of $R$ and
of the GKD (one more inversion), so both orders give the same term; the same holds
for $h_1, h_2$. Hence the full sum is $4^k$ times the sum over the 39 components
with $j_1 < j_2$ and $h_1 < h_2$. The next cell does this sum for $k = 1, 2, 3$,
together with the Lovelock scalar $L_{(k)}$ (the same sum without $h$ and $j$).
To multiply fast, each component, a Laurent polynomial in $\cot z$, is stored as
a sympy polynomial in $H, a_4', a_4'', \cot z$ and $t = \tan z = 1/\cot z$.
```

In [15]:

```
tz = sp.symbols("tz", positive=True)  # tz = tan z = 1/cot z
GENERATORS = (H, ad1, ad2, cz, tz)
# xreplace replaces exactly the expression 1/cz by tz and leaves cz itself alone:
FACTORS = [(key, sp.Poly(value.xreplace({1 / cz: tz}), *GENERATORS))
           for key, value in ordered.items()]  # the 39 components as polynomials


def lovelock(k):
    """P_(k)^h_j (8 x 8 list of expressions), L_(k) and the number of terms."""
    zero = sp.Poly(0, *GENERATORS)
    P = [[zero] * N for _ in range(N)]
    L = zero
    terms = 0
    for factors in itertools.product(FACTORS, repeat=k):
        lower = [i for key, _ in factors for i in key[0:2]]  # upper indices of R
        upper = [i for key, _ in factors for i in key[2:4]]  # lower indices of R
        if len(set(lower)) < 2 * k or len(set(upper)) < 2 * k:
            continue  # a repeated index: the GKD is zero
        terms += 1
        product = sp.Poly(4 ** k, *GENERATORS)  # the factor 4^k
        for _, poly in factors:
            product = product * poly
        L = L + gkd(upper, lower) * product
        for h, j in itertools.product(range(N), repeat=2):
            sign = gkd([h] + upper, [j] + lower)
            if sign:
                P[h][j] = P[h][j] + sign * product
    back = {tz: 1 / cz}  # tan z back to 1/cot z
    P = [[sp.expand(P[h][j].as_expr().subs(back)) for j in range(N)]
         for h in range(N)]
    return P, sp.expand(L.as_expr().subs(back)), terms


P, Lscalar, E = {}, {}, {}
for k in (1, 2, 3):
    P[k], Lscalar[k], terms = lovelock(k)
    E[k] = [[sp.expand(-P[k][h][j] / 2 ** (k + 1)) for j in range(N)]
            for h in range(N)]  # E_(k) = -P_(k)/2^(k+1)
    say(f"k = {k}: {terms} lists of index pairs contribute; L_({k}) = {Lscalar[k]}")
```

Out [15]:

```
k = 1: 39 lists of index pairs contribute; L_(1) = -84*H**2 + 12*ad1**2
k = 2: 690 lists of index pairs contribute; L_(2) = 3360*H**4 - 2112*H**2*ad1**2 -
    96*ad1**4
k = 3: 4410 lists of index pairs contribute; L_(3) = -40320*H**6 + 100224*H**4*ad1**2 +
    31104*H**2*ad1**4 + 1152*ad1**6
```

Text cell:

```
The next cell checks what the Revision record found about these tensors:
$E_{(1)} = G$ in all 64 components ($P_{(1)} = -4G$); each $E_{(k)}$ is diagonal,
free of $x_0$, equal in the three 3-space directions and in the three extra
times; the trace identity $\sum_h P_{(k)}{}^h{}_h = (8 - 2k) L_{(k)}$; and the
scalars $L_{(k)}$ agree with the Rust program of the GKD record.
```

In [16]:

```
reproduces(all(sp.expand(E[1][h][j] - G[h][j]) == 0
               for h in range(N) for j in range(N)),
           "E_(1) = G in all 64 components (P_(1) = -4 G)",
           PY, "P1_equals_minus_4_Einstein")
lovelock_json = read_json(LOVELOCK)
for k in (1, 2, 3):
    Ek = E[k]
    structure = (all(Ek[h][j] == 0 for h in range(N) for j in range(N) if h != j)
                 and Ek[1][1] == Ek[2][2] == Ek[3][3]
                 and Ek[5][5] == Ek[6][6] == Ek[7][7]
                 and not any(Ek[h][h].has(cz) for h in range(N)))
    reproduces(structure, f"E_({k}) diagonal, free of x0, isotropic in each group",
               PY, f"P{k}_structure")
    trace = sp.expand(sum(P[k][h][h] for h in range(N)) - (8 - 2 * k) * Lscalar[k])
    reproduces(trace == 0, f"trace identity sum_h P_({k})^h_h = (8 - {2 * k}) L_({k})",
               PY, f"P{k}_trace_identity")
    text = (lovelock_json[f"L{k}"].replace("Derivative[1][a4][x4]", "ad1")
            .replace("Derivative[2][a4][x4]", "ad2"))
    for b4_text, value in (("Derivative[2][b4][x4]", "0"),
                           ("Derivative[1][b4][x4]", "0"), ("b4[x4]", "1")):
        text = text.replace(b4_text, value)  # the record at b4 = 1
    rust = sp.sympify(text.replace("^", "**"), locals={"H": H, "ad1": ad1, "ad2": ad2})
    reproduces(sp.expand(rust - Lscalar[k]) == 0,
               f"L_({k}) equals the Rust GKD program", PY, f"L{k}_equals_gkd_branch")
```

Out [16]:

```
PASS E_(1) = G in all 64 components (P_(1) = -4 G)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P1_equals_minus_4_Einstein
PASS E_(1) diagonal, free of x0, isotropic in each group
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P1_structure
PASS trace identity sum_h P_(1)^h_h = (8 - 2) L_(1)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P1_trace_identity
PASS L_(1) equals the Rust GKD program
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    L1_equals_gkd_branch
PASS E_(2) diagonal, free of x0, isotropic in each group
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P2_structure
PASS trace identity sum_h P_(2)^h_h = (8 - 4) L_(2)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
```

```
    P2_trace_identity
PASS L_(2) equals the Rust GKD program
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    L2_equals_gkd_branch
PASS E_(3) diagonal, free of x0, isotropic in each group
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P3_structure
PASS trace identity sum_h P_(3)^h_h = (8 - 6) L_(3)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P3_trace_identity
PASS L_(3) equals the Rust GKD program
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    L3_equals_gkd_branch
```

Text cell:

```
The Rust program ‘lovelock_gkd‘ of the GKD record stored every component of
$P_{(k)}{}^h{}_j$ as a list of monomials: a fraction and the 13 exponents of $H$,
$a_4’, a_4’’, a_4’’’, a_4’’’’$, $e^{a_4}$, $\sin^{1/3} z$, $\cot z$, $b_4^{1/2}$ and
$b_4’, b_4’’, b_4’’’, b_4’’’’$. The program computes the author’s metric with the
second function $b_4(x_4)$ of 2026-10-14; this notebook works at $b_4 = 1$, so
‘from_monomials‘ keeps only the monomials without a derivative of $b_4$ (the others
vanish at $b_4 = 1$) and sets $b_4^{1/2} = 1$. The same is done with the scalars
$L_{(k)}$ in the cell before (‘b4[x4]‘ becomes 1, its derivatives 0). The next
cell rebuilds all $3 \times 64$ components from those lists and compares them with
ours; it also compares the four distinct components of each $E_{(k)}$ with the
Wolfram record ‘a4-equations.json‘.
```

In [17]:

```
symbols_of_monomial = (H, ad1, ad2, ad3, sp.Symbol("ad4"), sp.Symbol("Ee"),
                       sp.Symbol("Ss"), cz)  # the order of the exponents


def from_monomials(monomials):
    """A component of the Rust record as a sympy expression."""
    total = sp.Integer(0)
    for numerator, denominator, exponents in monomials:
        if any(exponents[9:13]):  # a derivative of b4: zero at b4 = 1
            continue
        term = sp.Rational(numerator, denominator)  # b4^(1/2) = 1 (exponent 8)
        for symbol, power in zip(symbols_of_monomial, exponents[:8]):
            term *= symbol ** power
        total += term
    return total


for k in (1, 2, 3):
    block = lovelock_json[f"P{k}_mixed_up_h_down_j"]
    same = all(sp.expand(from_monomials(block[f"{NAMES[h]},{NAMES[j]}"]["monomials"])
                         - P[k][h][j]) == 0 for h in range(N) for j in range(N))
    reproduces(same, f"all 64 components of P_({k}) equal the Rust GKD record",
```

```
               PY, f"P{k}_equals_gkd_branch_monomials")
equations_json = read_json(EQUATIONS)
LOCALS = {"ad1": ad1, "ad2": ad2, "H": H}


def parse(text, names=None):
    """A Wolfram InputForm text of the record as a sympy expression."""
    return sp.sympify(text.replace("^", "**"), locals=names or LOCALS)


POSITIONS = {"x1x1": (1, 1), "x4x4": (4, 4), "x5x5": (5, 5), "x0x0": (0, 0),
             "x4x0": (4, 0), "x0x4": (0, 4)}
same = all(sp.expand(parse(equations_json["lovelockTensors"][f"E{k}"][key]["input"])
                     - E[k][h][j]) == 0
           for k in (1, 2, 3) for key, (h, j) in POSITIONS.items())
reproduces(same, "E_(1), E_(2), E_(3) equal the Wolfram record a4-equations.json",
           PY, "json_lovelock_components")
```

Out [17]:

```
PASS all 64 components of P_(1) equal the Rust GKD record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P1_equals_gkd_branch_monomials
PASS all 64 components of P_(2) equal the Rust GKD record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P2_equals_gkd_branch_monomials
PASS all 64 components of P_(3) equal the Rust GKD record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    P3_equals_gkd_branch_monomials
PASS E_(1), E_(2), E_(3) equal the Wolfram record a4-equations.json
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_lovelock_components
```

Text cell:

```
**An independent route to $E_{(2)}$.** The Gauss-Bonnet tensor has a classical
formula (Lanczos), written here with one index up:

$$H^\mu{}_\nu = 2\big(R\,R^\mu{}_\nu - 2R^\mu{}_a R^a{}_\nu
- 2R^{\mu a}{}_{\nu b} R^b{}_a + R^{\mu a}{}_{bc} R^{bc}{}_{\nu a}\big)
- \tfrac12\delta^\mu_\nu\big(R^2 - 4R^a{}_b R^b{}_a
+ R^{ab}{}_{cd} R^{cd}{}_{ab}\big).$$

It uses no GKD. The next cell computes it from our Riemann tensor for a general
$a_4$ and checks that it equals $E_{(2)}$ in all 64 components. For the linear
member ($a_4' = AH$, $a_4'' = 0$) it then reproduces the lead's numbers
$H^{x_4}{}_{x_4} = -H^4(36A^4 + 120A^2 + 420)$ and
$H^{x_1}{}_{x_1} = 12H^4(A^4 + 14A^2 - 15)$.
```

In [18]:

```
def contract(mu, nu):
    """The four products in the bracket of the Lanczos formula."""
    term_a = ricci_scalar * Ricci[mu][nu]
    term_b = sum(Ricci[mu][a] * Ricci[a][nu] for a in range(N))
    term_c = sum(R.get((mu, a, nu, b), 0) * Ricci[b][a]
                 for a in range(N) for b in range(N))
    term_d = sum(R.get((mu, a, b, c), 0) * R.get((b, c, nu, a), 0)
                 for a in range(N) for b in range(N) for c in range(N))
    return 2 * (term_a - 2 * term_b - 2 * term_c + term_d)


gauss_bonnet_scalar = (ricci_scalar ** 2
                       - 4 * sum(Ricci[a][b] * Ricci[b][a]
                                 for a in range(N) for b in range(N))
                       + sum(v * R.get((m, n, a, b), 0)
                             for (a, b, m, n), v in R.items()))
lanczos = [[sp.expand(contract(mu, nu)
                      - (gauss_bonnet_scalar / 2 if mu == nu else 0))
            for nu in range(N)] for mu in range(N)]
check(all(sp.expand(lanczos[h][j] - E[2][h][j]) == 0
          for h in range(N) for j in range(N)),
      "E_(2) (GKD) equals the classical Gauss-Bonnet tensor for every a4")
A = sp.symbols("A", real=True)  # the slope of the linear member, a4' = A H
LINEAR = {ad1: A * H, ad2: 0}
reproduces(sp.expand(lanczos[4][4].subs(LINEAR)
                     + H ** 4 * (36 * A ** 4 + 120 * A ** 2 + 420)) == 0,
           "linear member: Gauss-Bonnet H^x4_x4 = -H^4 (36 A^4 + 120 A^2 + 420)",
           LEAD, "gauss_bonnet_rho_alpha2")
reproduces(sp.expand(lanczos[1][1].subs(LINEAR)
                     - 12 * H ** 4 * (A ** 4 + 14 * A ** 2 - 15)) == 0,
           "linear member: Gauss-Bonnet H^x1_x1 = 12 H^4 (A^4 + 14 A^2 - 15)",
           LEAD, "gauss_bonnet_p_alpha2")
```

Out [18]:

```
PASS E_(2) (GKD) equals the classical Gauss-Bonnet tensor for every a4
PASS linear member: Gauss-Bonnet H^x4_x4 = -H^4 (36 A^4 + 120 A^2 + 420)
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    gauss_bonnet_rho_alpha2
PASS linear member: Gauss-Bonnet H^x1_x1 = 12 H^4 (A^4 + 14 A^2 - 15)
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    gauss_bonnet_p_alpha2
```

Text cell:

```
**The Bianchi identity.** Lovelock's theorem says that each $E_{(k)}$ has zero
divergence for every metric. The next cell checks this directly: it computes
$\nabla_\mu E^\mu{}_\nu = \partial_\mu E^\mu{}_\nu + \Gamma^\mu{}_{\mu l}
E^l{}_\nu - \Gamma^l{}_{\mu\nu} E^\mu{}_l$ (sums over $\mu$ and $l$) for
$\nu = x_0, \dots, x_7$, with $a_4'$ and $a_4''$ written back as derivatives of
the function $a_4(x_4)$ so that $\partial_4$ can act on them.
```

In [19]:

```
def divergence(T):
    """nabla_mu T^mu_nu of a mixed tensor T (8 x 8 list), for nu = x0..x7."""
    return [clean(sum(d(T[m][n], m) for m in range(N))
                  + sum(Gamma[m][m][l] * T[l][n] for m in range(N) for l in range(N))
                  - sum(Gamma[l][m][n] * T[m][l] for m in range(N) for l in range(N)))
            for n in range(N)]


AS_FUNCTIONS = {ad2: sp.Derivative(a4, (x4, 2)), ad1: sp.Derivative(a4, x4)}
for k in (1, 2, 3):
    Ek_functions = [[E[k][h][j].subs(AS_FUNCTIONS) for j in range(N)]
                    for h in range(N)]
    reproduces(all(value == 0 for value in divergence(Ek_functions)),
               f"nabla_mu E_({k})^mu_nu = 0 for nu = x0..x7",
               PY, f"E{k}_divergence_free")
```

Out [19]:

```
PASS nabla_mu E_(1)^mu_nu = 0 for nu = x0..x7
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    E1_divergence_free
PASS nabla_mu E_(2)^mu_nu = 0 for nu = x0..x7
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    E2_divergence_free
PASS nabla_mu E_(3)^mu_nu = 0 for nu = x0..x7
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    E3_divergence_free
```

Text cell:

```
The next cell draws the diagonal of $E_{(1)}, E_{(2)}, E_{(3)}$ at the sample point
$H = 1$, $a_4' = 0.6$, $a_4'' = 0.3$ as three 8 by 8 heat maps, with the value
written in each diagonal cell. Each panel has its own colour scale. All three
tensors have the same pattern: diagonal, one value for $x_1 = x_2 = x_3$, one for
$x_4$, one for $x_5 = x_6 = x_7$, one for $x_0$.
```

In [20]:

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.4))
for ax, k in zip(axes, (1, 2, 3)):
    values = np.array([[float(E[k][h][j].subs(SAMPLE)) for j in range(N)]
                       for h in range(N)])
    scale = np.abs(values).max()
    ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
    for h in range(N):  # write each diagonal value into its cell
        value = values[h, h]
        text = f"{value:.0f}" if abs(value) >= 100 else f"{value:.1f}"
        ink = "white" if abs(value) > 0.6 * scale else "black"  # readable on dark
```

```
        ax.text(h, h, text, ha="center", va="center", fontsize=6.5, color=ink)
    ax.set_xticks(range(N))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(N))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)
    ax.set_title(f"$E_{{({k})}}{{}}^h{{}}_j$")
fig.suptitle("The three Lovelock tensors at $H=1$, $a_4'=0.6$, $a_4''=0.3$ "
             "(row $h$, column $j$)")
save_figure(fig, "lovelock_heat_maps",
            "The three Lovelock tensors $E_{(1)}$ (the Einstein tensor), $E_{(2)}$ "
            "(the Gauss-Bonnet tensor) and $E_{(3)}$ of the author's metric as 8 by "
            "8 tables, row $h$ and column $j$ running over $x_0$ to $x_7$, evaluated "
            "at $H = 1$, $a_4' = 0.6$, $a_4^{\\prime\\prime} = 0.3$; the colour is "
            "the value (each panel with its own scale, red positive, blue negative, "
            "white zero) and "
            "the number in each diagonal cell is the value (one decimal below 100, "
            "a whole number above). All off-diagonal entries are zero, and each "
            "tensor has only four different diagonal values: one for 3-space, one "
            "for the time, one for the extra times and one for the hidden "
            "direction.")
```

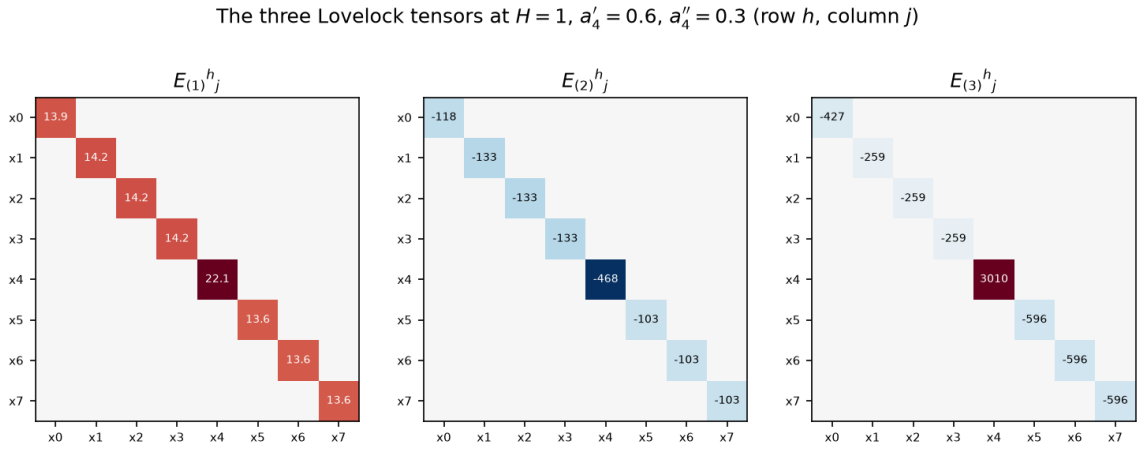

Figure 321: The three Lovelock tensors $E_{(1)}$ (the Einstein tensor), $E_{(2)}$ (the Gauss-Bonnet tensor) and $E_{(3)}$ of the author's metric as 8 by 8 tables, row h and column j running over x_0 to x_7, evaluated at $H = 1$, $a_4' = 0.6$, $a_4'' = 0.3$; the colour is the value (each panel with its own scale, red positive, blue negative, white zero) and the number in each diagonal cell is the value (one decimal below 100, a whole number above). All off-diagonal entries are zero, and each tensor has only four different diagonal values: one for 3-space, one for the time, one for the extra times and one for the hidden direction. (Notebook 12a, figure 3.)

Out [20]:

```
Figure 12a.3 saved as Revision/textbook/figures/12a_3_lovelock_heat_maps.png
```

Text cell:

```
## 11. The field equations and their independent components

The next cell writes the left-hand side
$\sum_k \alpha_k E_{(k)}{}^h{}_j + \Lambda\delta^h_j$ and the source $T^h{}_j$ and
checks that every off-diagonal left-hand side is zero (so the field equations
there read $0 = \kappa T^h{}_j$: the source must have $q_{40} = q_{04} = 0$ and no
other mixed component), and that the diagonal reduces to four independent
components. It names the four: ‘e44‘ (time), ‘e11‘ (3-space), ‘e55‘ (extra times),
‘e00‘ (hidden), each without the $\Lambda$.
```

In [21]:

```
alpha1, alpha2, alpha3 = sp.symbols("alpha1 alpha2 alpha3", real=True)
Lam, kappa = sp.symbols("Lam kappa", real=True)  # Lambda and kappa
rho, p3, pt, p0, q40, q04 = sp.symbols("rho p3 pt p0 q40 q04", real=True)
ALPHA = {1: alpha1, 2: alpha2, 3: alpha3}
LHS = [[sp.expand(sum(ALPHA[k] * E[k][h][j] for k in (1, 2, 3))
                  + (Lam if h == j else 0)) for j in range(N)] for h in range(N)]
T = [[sp.Integer(0)] * N for _ in range(N)]
for i, value in enumerate([p0, p3, p3, p3, -rho, pt, pt, pt]):
    T[i][i] = value  # the diagonal of the source, in the order x0..x7
T[4][0], T[0][4] = q40, q04  # the two mixed components x4-x0 and x0-x4
off_diagonal_zero = all(LHS[h][j] == 0 for h in range(N) for j in range(N)
                        if h != j)
groups_equal = (LHS[1][1] == LHS[2][2] == LHS[3][3]
                and LHS[5][5] == LHS[6][6] == LHS[7][7])
reproduces(off_diagonal_zero and groups_equal,
           "every off-diagonal left-hand side is 0; four independent components",
           WL, "independent_components")
e11, e44, e55, e00 = (sp.expand(LHS[i][i] - Lam) for i in (1, 4, 5, 0))
say(f"constraint (x4): {e44} + Lam = -kappa*rho")
say(f"hidden (x0): {e00} + Lam = kappa*p0")
```

Out [21]:

```
PASS every off-diagonal left-hand side is 0; four independent components
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    independent_components
constraint (x4): 2520*H**6*alpha3 + 1080*H**4*ad1**2*alpha3 - 420*H**4*alpha2 +
    648*H**2*ad1**4*alpha3 - 120*H**2*ad1**2*alpha2 + 21*H**2*alpha1 + 360*ad1**6*alpha3
    - 36*ad1**4*alpha2 + 3*ad1**2*alpha1 + Lam = -kappa*rho
hidden (x0): 360*H**6*alpha3 - 1944*H**4*ad1**2*alpha3 - 180*H**4*alpha2 -
    648*H**2*ad1**4*alpha3 + 168*H**2*ad1**2*alpha2 + 15*H**2*alpha1 - 72*ad1**6*alpha3 +
    12*ad1**4*alpha2 - 3*ad1**2*alpha1 + Lam = kappa*p0
```

Text cell:

```
**The evolution equation.** The 3-space equation minus the extra-time equation
removes $\Lambda$: $e_{11} - e_{55} = \kappa(p_3 - p_t)$. The next cell shows that
$e_{11} - e_{55}$ is $a_4''$ times a polynomial $F(a_4')$, that $F$ vanishes for
every $a_4'$ and $H$ only when $\alpha_1 = \alpha_2 = \alpha_3 = 0$, and that the
```

```
3-space plus the extra-time equation is exactly twice the hidden equation
(so the source must satisfy $p_3 + p_t = 2p_0$). It also checks that the time
and hidden components contain no $a_4''$.
```

In [22]:

```
F = sp.expand(sp.cancel((e11 - e55) / ad2))  # (e11 - e55)/a4''
report("F(a4')", F)
reproduces(not F.has(ad2) and sp.expand(ad2 * F - (e11 - e55)) == 0,
           "evolution: e11 - e55 = a4'' F(a4') = kappa (p3 - pt)",
           PY, "evolution_factorises")
coefficients = sp.Poly(F, ad1, H).coeffs()  # every coefficient must vanish
solutions = sp.solve(coefficients, [alpha1, alpha2, alpha3], dict=True)
reproduces(solutions == [{alpha1: 0, alpha2: 0, alpha3: 0}],
           "F vanishes identically only for alpha1 = alpha2 = alpha3 = 0",
           PY, "evolution_F_not_identically_zero")
reproduces(sp.expand(e11 + e55 - 2 * e00) == 0,
           "e11 + e55 = 2 e00 identically: the source needs p3 + pt = 2 p0",
           PY, "algebraic_identity")
reproduces(not e44.has(ad2) and not e00.has(ad2),
           "the x4 (constraint) and x0 components contain a4' but not a4''",
           PY, "constraint_and_x0_first_order")
```

Out [22]:

```
RESULT F(a4') = 720*H**4*alpha3 + 864*H**2*ad1**2*alpha3 - 80*H**2*alpha2 +
    720*ad1**4*alpha3 - 48*ad1**2*alpha2 + 2*alpha1
PASS evolution: e11 - e55 = a4'' F(a4') = kappa (p3 - pt)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    evolution_factorises
PASS F vanishes identically only for alpha1 = alpha2 = alpha3 = 0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    evolution_F_not_identically_zero
PASS e11 + e55 = 2 e00 identically: the source needs p3 + pt = 2 p0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    algebraic_identity
PASS the x4 (constraint) and x0 components contain a4' but not a4''
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    constraint_and_x0_first_order
```

Text cell:

```
**The constraint and the evolution equation are linked (Bianchi identity).**
$e_{44}$ depends on $x_4$ only through $a_4'$, so by the chain rule
$\tfrac{d}{dx_4} e_{44} = \tfrac{\partial e_{44}}{\partial a_4'}\,a_4''$. The next
cell checks $\tfrac{d}{dx_4} e_{44} = 3a_4'(e_{11} - e_{55})$: the derivative of
the constraint is $3a_4'$ times the evolution equation. Hence, when the source
obeys $\rho' = -3a_4'(p_3 - p_t)$ (section 12), the constraint stays true along
a solution of the evolution equation. The cell also compares the four component
equations, $F$ and the algebraic condition with the Wolfram record.
```

In [23]:

```
reproduces(sp.expand(sp.diff(e44, ad1) * ad2 - 3 * ad1 * (e11 - e55)) == 0,
           "d/dx4 e44 = 3 a4' (e11 - e55) (constraint propagation)",
           PY, "bianchi_x4")
general = equations_json["generalSource"]
NAMES_RECORD = {"ad1": ad1, "ad2": ad2, "H": H, "alpha1": alpha1, "alpha2": alpha2,
                "alpha3": alpha3, "Lam": Lam, "kappa": kappa, "rho": rho, "p3": p3,
                "pt": pt, "p0": p0, "AA": A}


def parse_equation(text):
    """An equation "left == right" of the record as left - right."""
    left, right = text.split(" == ")
    return sp.expand(parse(left, NAMES_RECORD) - parse(right, NAMES_RECORD))


mine = {"constraint_x4": e44 + Lam + kappa * rho,
        "space_x1_eq_x2_eq_x3": e11 + Lam - kappa * p3,
        "extraTime_x5_eq_x6_eq_x7": e55 + Lam - kappa * pt,
        "hidden_x0": e00 + Lam - kappa * p0,
        "evolution_x1_minus_x5": ad2 * F - kappa * (p3 - pt),
        "algebraic_condition": p3 + pt - 2 * p0}
same = all(sp.expand(parse_equation(general[key]["input"]) - value) == 0
           for key, value in mine.items())
same = same and sp.expand(parse(general["evolution_F"]["input"], NAMES_RECORD)
                          - F) == 0
reproduces(same, "the six equations and F equal the record a4-equations.json",
           PY, "json_general_source_equations")
```

Out [23]:

```
PASS d/dx4 e44 = 3 a4' (e11 - e55) (constraint propagation)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    bianchi_x4
PASS the six equations and F equal the record a4-equations.json
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_general_source_equations
```

Text cell:

```
The next cell draws $F(a_4')/2$ for Einstein gravity ($F = 2$: the evolution
equation is simply $2a_4'' = \kappa(p_3 - p_t)$) and for several Lovelock
couplings, in units $H = 1$. Where $F = 0$ the evolution equation cannot be solved
for $a_4''$: for Einstein-Gauss-Bonnet gravity with $\alpha_2 > 0$ this happens at
$(a_4')^2 = (1 - 40\alpha_2H^2)/(24\alpha_2)$, which follows from
$F = 2 - 80\alpha_2H^2 - 48\alpha_2(a_4')^2 = 0$.
```

In [24]:

```
F_numeric = sp.lambdify((ad1, alpha1, alpha2, alpha3), F.subs(H, 1), "numpy")
u = np.linspace(-3.0, 3.0, 601)  # a4'/H
cases = [(1, 0.0, 0.0, "Einstein: $\\alpha_1=1$"),
         (1, 0.005, 0.0, "$\\alpha_2H^2=0.005$"),
         (1, 0.01, 0.0, "$\\alpha_2H^2=0.01$"),
         (1, 0.02, 0.0, "$\\alpha_2H^2=0.02$"),
         (1, 0.01, 0.0005, "$\\alpha_2H^2=0.01$, $\\alpha_3H^4=0.0005$")]
fig, ax = plt.subplots()
for a1, a2, a3, label in cases:
    values = F_numeric(u, a1, a2, a3) * np.ones_like(u) / 2
    ax.plot(u, values, label=label)
for a2 in (0.005, 0.01, 0.02):  # the zeros of F for Gauss-Bonnet gravity
    root = np.sqrt((1 - 40 * a2) / (24 * a2))
    ax.plot([-root, root], [0, 0], "o", color="black", markersize=4)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("$F(a_4')/2$")
ax.set_title("The factor $F$ of the evolution equation $a_4''F = \\kappa(p_3-p_t)$")
ax.legend(fontsize=8)
save_figure(fig, "evolution_factor",
            "The factor $F(a_4')$ of the evolution equation "
            "$a_4^{\\prime\\prime} F(a_4') = \\kappa(p_3 - p_t)$, divided by 2, as a "
            "function of $a_4'/H$ (units $H = 1$) for Einstein gravity ($F = 2$, flat "
            "line), for "
            "Einstein-Gauss-Bonnet gravity with $\\alpha_2 H^2 = 0.005$, $0.01$, "
            "$0.02$, and with a third-order coupling $\\alpha_3 H^4 = 0.0005$ added. "
            "Black dots mark the zeros of the Gauss-Bonnet curves, at "
            "$(a_4')^2 = (1 - 40\\alpha_2H^2)/(24\\alpha_2)$: there the evolution "
            "equation cannot be solved for $a_4^{\\prime\\prime}$. The third-order "
            "term lifts the curve again at large $a_4'$.")
```

Out [24]:

```
Figure 12a.4 saved as Revision/textbook/figures/12a_4_evolution_factor.png
```

Text cell:

```
## 12. The conservation law of the source

The Bianchi identity makes the divergence of the left-hand side zero, so the field
equations can hold only if the source has zero divergence too. The next cell
computes $\nabla_\mu T^\mu{}_\nu$ for a general source whose seven parts
$\rho, p_3, p_t, p_0, q_{40}, q_{04}$ are arbitrary functions of $x_4$ and $x_0$,
and compares it with the record:

$$\nabla_\mu T^\mu{}_{x_4} = -\partial_4\rho - 3a_4'(p_3 - p_t)
+ \partial_0 q_{04} - 6H\tan z\, q_{04},$$
$$\nabla_\mu T^\mu{}_{x_0} = \partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t)
+ \partial_4 q_{40},$$

and zero for the six other $\nu$. With an $x_0$-independent source and
$q_{40} = q_{04} = 0$ this is $\rho' = -3a_4'(p_3 - p_t)$ and
```

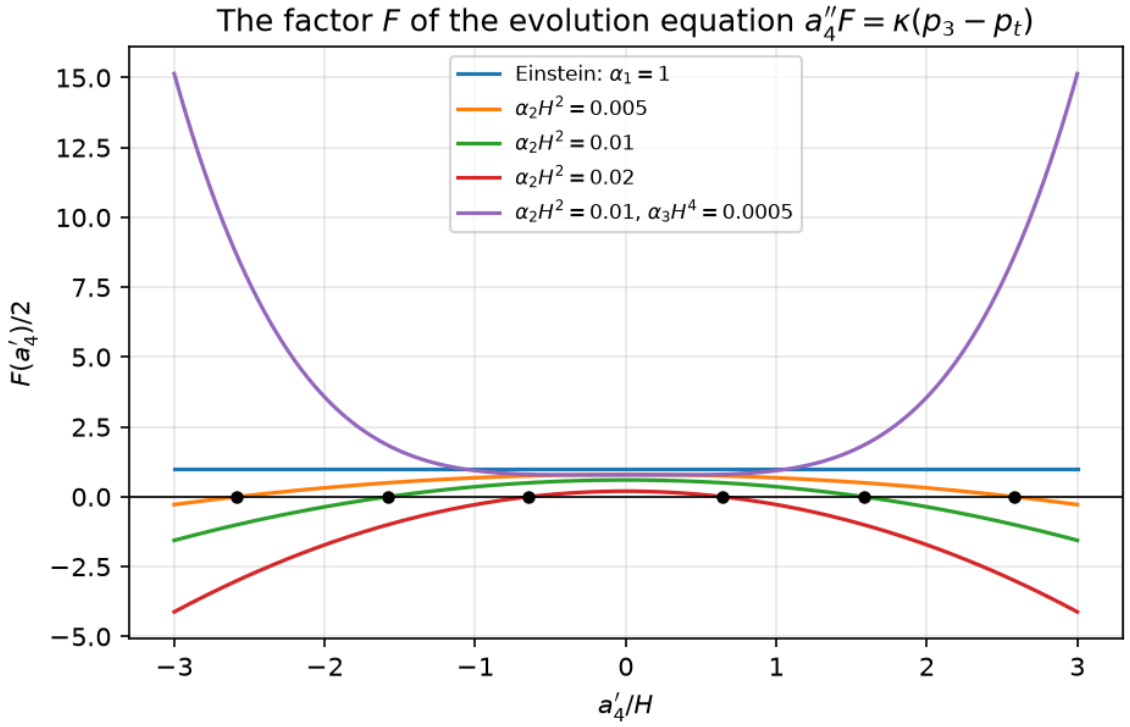

Figure 322: The factor $F(a_4')$ of the evolution equation $a_4'' F(a_4') = \kappa(p_3 - p_t)$, divided by 2, as a function of a_4'/H (units $H = 1$) for Einstein gravity ($F = 2$, flat line), for Einstein-Gauss-Bonnet gravity with $\alpha_2 H^2 = 0.005$, 0.01, 0.02, and with a third-order coupling $\alpha_3 H^4 = 0.0005$ added. Black dots mark the zeros of the Gauss-Bonnet curves, at $(a_4')^2 = (1 - 40\alpha_2 H^2)/(24\alpha_2)$: there the evolution equation cannot be solved for a_4''. The third-order term lifts the curve again at large a_4'. (Notebook 12a, figure 4.)

```
$p_3 + p_t = 2p_0$: the energy of the source changes only when the 3-space and the
extra-time pressures differ, and the hidden equation repeats the algebraic
condition.
```

In [25]:

```
source = {name: sp.Function(name)(x4, z)
          for name in ("rho", "p3", "pt", "p0", "q40", "q04")}  # f(x4, z)
Tgeneral = [[sp.Integer(0)] * N for _ in range(N)]
for i, name in enumerate(["p0", "p3", "p3", "p3", "rho", "pt", "pt", "pt"]):
    Tgeneral[i][i] = source[name]
Tgeneral[4][4] = -source["rho"]  # T^x4_x4 = -rho
Tgeneral[4][0], Tgeneral[0][4] = source["q40"], source["q04"]
divT = divergence(Tgeneral)
a4p = sp.Derivative(a4, x4)  # a4' as a derivative, for the comparison
expect_x4 = (-d(source["rho"], 4) - 3 * a4p * (source["p3"] - source["pt"])
             + d(source["q04"], 0) - 6 * H * sp.tan(z) * source["q04"])
expect_x0 = (d(source["p0"], 0) + d(source["q40"], 4)
             + 3 * H * sp.cot(z) * (2 * source["p0"] - source["p3"] - source["pt"]))
reproduces(clean(divT[4] - expect_x4) == 0, "nabla_mu T^mu_x4 as in the record",
           EMT, "divergence_x4_component")
reproduces(clean(divT[0] - expect_x0) == 0, "nabla_mu T^mu_x0 as in the record",
           EMT, "divergence_x0_component")
reproduces(all(divT[n] == 0 for n in (1, 2, 3, 5, 6, 7)),
           "nabla_mu T^mu_nu = 0 identically for the six other nu",
           PY, "conservation_components")
```

Out [25]:

```
PASS nabla_mu T^mu_x4 as in the record
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check divergence_x4_component
PASS nabla_mu T^mu_x0 as in the record
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check divergence_x0_component
PASS nabla_mu T^mu_nu = 0 identically for the six other nu
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    conservation_components
```

Text cell:

```
## 13. Einstein gravity: no vacuum, and the null energy condition along $x_0$

In Einstein gravity ($\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$) the equations read

$$3(a_4')^2 + 21H^2 + \Lambda = -\kappa\rho,\qquad 2a_4'' = \kappa(p_3 - p_t),
\qquad -3(a_4')^2 + 15H^2 + \Lambda = \kappa p_0.$$

Adding the first and the third: $-\kappa\rho + \kappa p_0 = 36H^2 + 2\Lambda$.
Subtracting the first from the third: $\kappa(\rho + p_0) = -6(a_4')^2 - 6H^2$,
which is negative for every $a_4$ and every $\Lambda$: the source always violates
```

```
the null energy condition along the null direction $x_4 + x_0$ (for $\kappa > 0$).
A vacuum ($T = 0$) would need $6(a_4')^2 + 6H^2 = 0$, impossible for real $a_4'$
and $H > 0$. The next cell checks both and lets sympy solve the vacuum equations:
the only solutions have the imaginary slope $a_4' = \pm iH$.
```

In [26]:

```
EINSTEIN = {alpha1: 1, alpha2: 0, alpha3: 0}
null_x0 = sp.expand((e00 - e44).subs(EINSTEIN))  # kappa (rho + p0)
report("Einstein: kappa (rho + p0)", null_x0)
reproduces(sp.expand(null_x0 + 6 * ad1 ** 2 + 6 * H ** 2) == 0,
           "Einstein: kappa (rho + p0) = -6 (a4'^2 + H^2) < 0",
           PY, "einstein_null_energy_x0")
vacuum = [LHS[i][i].subs(EINSTEIN) for i in (1, 4, 5, 0)]  # T = 0
real_solutions = sp.solve(vacuum, [Lam, ad1, ad2], dict=True)  # ad1, ad2 real
say(f"real solutions of the vacuum equations: {real_solutions}")
w1, w2 = sp.symbols("w1 w2")  # a4' and a4'' again, now allowed to be complex
as_complex = [value.subs({ad1: w1, ad2: w2}) for value in vacuum]
complex_solutions = sorted(sp.solve(as_complex, [Lam, w1, w2], dict=True), key=str)
say(f"solutions when a4' = w1 and a4'' = w2 may be complex: {complex_solutions}")
imaginary = all(not solution[w1].is_real for solution in complex_solutions)
reproduces(real_solutions == [] and len(complex_solutions) == 2 and imaginary,
           "Einstein: no real vacuum solution for H > 0 and any Lambda",
           LEAD, "no_vacuum_for_H_positive")
einstein_record = equations_json["einstein"]
mine_einstein = {"constraint_x4": e44.subs(EINSTEIN) + Lam + kappa * rho,
                 "hidden_x0": e00.subs(EINSTEIN) + Lam - kappa * p0,
                 "evolution": 2 * ad2 - kappa * (p3 - pt)}
same = all(sp.expand(parse_equation(einstein_record[key]["input"]) - value) == 0
           for key, value in mine_einstein.items())
reproduces(same, "the Einstein equations equal the record", PY, "json_einstein")
```

Out [26]:

```
RESULT Einstein: kappa (rho + p0) = -6*H**2 - 6*ad1**2
PASS Einstein: kappa (rho + p0) = -6 (a4'^2 + H^2) < 0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_null_energy_x0
real solutions of the vacuum equations: []
solutions when a4' = w1 and a4'' = w2 may be complex: [{Lam: -18*H**2, w1: -I*H, w2: 0},
    {Lam: -18*H**2, w1: I*H, w2: 0}]
PASS Einstein: no real vacuum solution for H > 0 and any Lambda
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    no_vacuum_for_H_positive
PASS the Einstein equations equal the record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_einstein
```

Text cell:

```
The next cell draws the reason why there is no vacuum. With $T = 0$ the
constraint asks for $\Lambda = -(3(a_4')^2 + 21H^2)$ and the hidden equation for
```

```
$\Lambda = 3(a_4')^2 - 15H^2$. As functions of $a_4'$ the two curves never meet:
the first is never above $-21H^2$, the second never below $-15H^2$. The vertical
distance between them is $6(a_4')^2 + 6H^2 = -\kappa(\rho + p_0)$.
```

In [27]:

```
u = np.linspace(-3.0, 3.0, 301)  # a4'/H
lam_constraint = -(3 * u ** 2 + 21)  # Lambda/H^2 that the x4 equation needs
lam_hidden = 3 * u ** 2 - 15  # Lambda/H^2 that the x0 equation needs
fig, ax = plt.subplots()
ax.plot(u, lam_constraint, label="$\\Lambda$ needed by the constraint ($x_4$)")
ax.plot(u, lam_hidden, label="$\\Lambda$ needed by the hidden equation ($x_0$)")
ax.fill_between(u, lam_constraint, lam_hidden, color="grey", alpha=0.25,
                label="gap $6(a_4')^2+6H^2=-\\kappa(\\rho+p_0)$")
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("$\\Lambda/H^2$")
ax.set_title("Einstein gravity: the vacuum equations have no common solution")
ax.legend(fontsize=8, loc="upper center")  # the empty area above the curves
save_figure(fig, "no_vacuum_gap",
            "Einstein gravity without a source: the value of $\\Lambda/H^2$ that the "
            "time equation (constraint) requires, $-(3(a_4')^2 + 21H^2)$, and the "
            "value that the hidden $x_0$ equation requires, $3(a_4')^2 - 15H^2$, as "
            "functions of $a_4'/H$. The two curves never meet, so there is no vacuum "
            "solution for $H > 0$ and any $\\Lambda$; the grey gap between them, "
            "$6(a_4')^2 + 6H^2$, equals $-\\kappa(\\rho + p_0)$, so every source of the "
            "metric violates the null energy condition along $x_0$.")
```

Out [27]:

```
Figure 12a.5 saved as Revision/textbook/figures/12a_5_no_vacuum_gap.png
```

Text cell:

```
## 14. The linear member $a_4 = A H x_4 + a_0$

For $a_4 = AHx_4 + a_0$ we have $a_4' = AH$ and $a_4'' = 0$: 3-space grows like
$e^{AHx_4}$ and the extra times shrink like $e^{-AHx_4}$ (exponential deflation
for $A > 0$). The next cell checks that then $p_3 = p_t = p_0 =: p$ (the three
pressures are equal) and that $\rho$ and $p$ are constants; it prints $\kappa\rho$
and $\kappa p$ and checks that they contain $A$ only through $A^2$: the equations
do not distinguish deflating extra times ($A > 0$) from inflating ones
($A < 0$). Deflation is a choice of sign, an initial condition, not a consequence
of the equations.
```

In [28]:

```
reproduces(sp.expand((e11 - e00).subs(LINEAR)) == 0
           and sp.expand((e55 - e00).subs(LINEAR)) == 0,
           "linear member: p3 = pt = p0 = p (any alpha_k)",
```

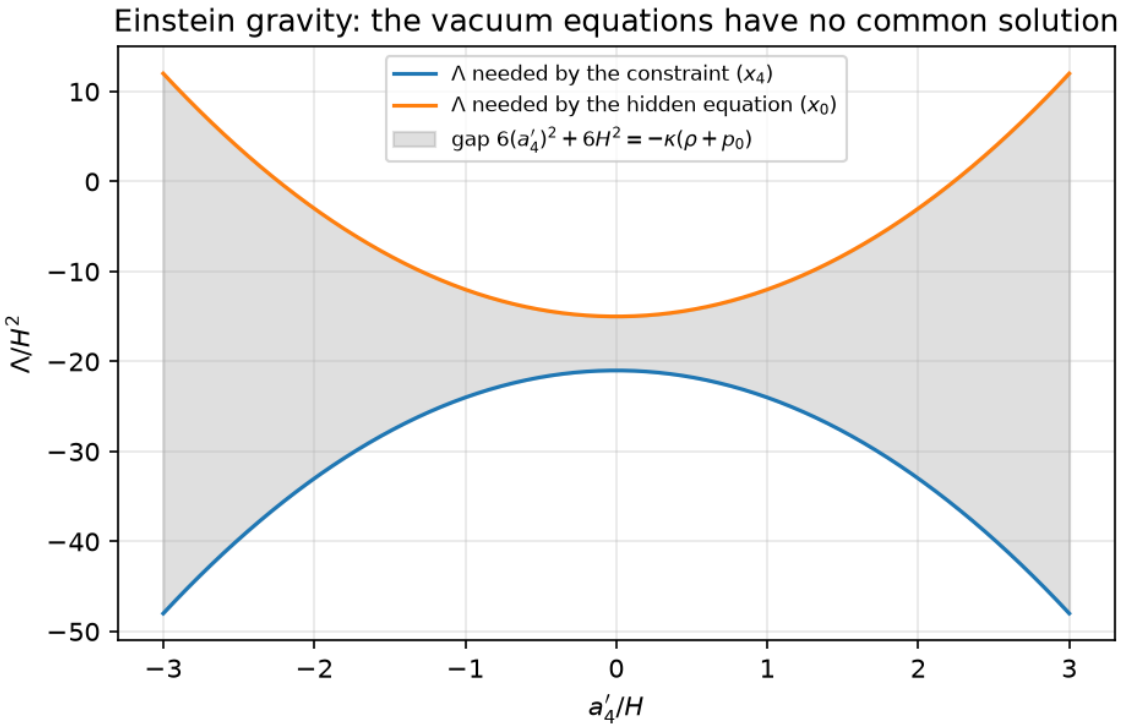

Figure 323: Einstein gravity without a source: the value of Λ/H^2 that the time equation (constraint) requires, $-(3(a_4')^2 + 21H^2)$, and the value that the hidden x_0 equation requires, $3(a_4')^2 - 15H^2$, as functions of a_4'/H. The two curves never meet, so there is no vacuum solution for $H > 0$ and any Λ; the grey gap between them, $6(a_4')^2 + 6H^2$, equals $-\kappa(\rho + p_0)$, so every source of the metric violates the null energy condition along x_0. (Notebook 12a, figure 5.)

```
          PY, "linear_member_equal_pressures")
rho_linear = sp.expand(-(e44 + Lam).subs(LINEAR) / kappa)  # from the constraint
p_linear = sp.expand((e00 + Lam).subs(LINEAR) / kappa)  # from the hidden equation
say(f"kappa rho = {sp.expand(kappa * rho_linear)}")
say(f"kappa p = {sp.expand(kappa * p_linear)}")
check(not rho_linear.has(x4) and not p_linear.has(x4),
      "linear member: rho and p are constants")
check(sp.expand(rho_linear.subs(A, -A) - rho_linear) == 0
      and sp.expand(p_linear.subs(A, -A) - p_linear) == 0,
      "rho and p are even in A: A -> -A is a symmetry (deflation is a choice)")
```

Out [28]:

```
PASS linear member: p3 = pt = p0 = p (any alpha_k)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    linear_member_equal_pressures
kappa rho = -360*A**6*H**6*alpha3 - 648*A**4*H**6*alpha3 + 36*A**4*H**4*alpha2 -
    1080*A**2*H**6*alpha3 + 120*A**2*H**4*alpha2 - 3*A**2*H**2*alpha1 - 2520*H**6*alpha3
    + 420*H**4*alpha2 - 21*H**2*alpha1 - Lam
kappa p = -72*A**6*H**6*alpha3 - 648*A**4*H**6*alpha3 + 12*A**4*H**4*alpha2 -
    1944*A**2*H**6*alpha3 + 168*A**2*H**4*alpha2 - 3*A**2*H**2*alpha1 + 360*H**6*alpha3 -
    180*H**4*alpha2 + 15*H**2*alpha1 + Lam
PASS linear member: rho and p are constants
PASS rho and p are even in A: A -> -A is a symmetry (deflation is a choice)
```

Text cell:

```
**The vacuum of the linear member.** A vacuum needs $\rho = p = 0$, so in
particular $e_{44} = e_{00}$. The next cell factors
$e_{44} - e_{00} = 6(A^2 + 1)H^2\,V$ with the polynomial

$$V = \alpha_1 - 8\alpha_2H^2(A^2 + 5) + \alpha_3 H^4(72A^4 + 144A^2 + 360).$$

In Einstein gravity $V = 1$: no vacuum. In Einstein-Gauss-Bonnet gravity
($\alpha_1 = 1$, $\alpha_3 = 0$) $V = 0$ gives
$A^2 = (1 - 40\alpha_2H^2)/(8\alpha_2H^2)$, a real $A$ only for
$0 < \alpha_2H^2 \le 1/40$. Because $e_{44}$ and $e_{00}$ contain no $a_4''$, the
same factorisation holds for EVERY $a_4$ with $AH$ replaced by $a_4'$:
$\kappa(\rho + p_0) = -6((a_4')^2 + H^2)\,V$. The cell also compares $\rho$, $p$
and $V$ with the record.
```

In [29]:

```
V = sp.expand(sp.cancel((e44 - e00).subs(LINEAR) / (6 * (A ** 2 + 1) * H ** 2)))
report("vacuum factor V", V)
reproduces(V.is_polynomial(A) and sp.expand(6 * (A ** 2 + 1) * H ** 2 * V
                                            - (e44 - e00).subs(LINEAR)) == 0,
           "e44 - e00 = 6 (A^2 + 1) H^2 V with a polynomial V",
           PY, "linear_member_vacuum_factor")
V_gauss_bonnet = sp.expand(V.subs({alpha1: 1, alpha3: 0}))
reproduces(sp.expand(V_gauss_bonnet - (1 - 8 * alpha2 * H ** 2 * (A ** 2 + 5))) == 0,
```

```
          "Gauss-Bonnet: V = 1 - 8 alpha2 H^2 (A^2 + 5)",
          PY, "einstein_gauss_bonnet_vacuum_linear")
V_any = V.subs(A, ad1 / H)  # the same polynomial with A H -> a4'
check(sp.expand((e00 - e44) + 6 * (ad1 ** 2 + H ** 2) * V_any) == 0,
      "for every a4: kappa (rho + p0) = -6 (a4'^2 + H^2) V(a4')")
linear_record = equations_json["linearMember"]
same = (sp.expand(parse(linear_record["rho"]["input"], NAMES_RECORD) - rho_linear)
        == 0
        and sp.expand(parse(linear_record["p"]["input"], NAMES_RECORD) - p_linear)
        == 0
        and sp.expand(parse(linear_record["vacuumFactor"]["input"], NAMES_RECORD)
                      - V) == 0)
reproduces(same, "rho, p and V of the linear member equal the record",
          PY, "json_linear_member")
```

Out [29]:

```
RESULT vacuum factor V = 72*A**4*H**4*alpha3 + 144*A**2*H**4*alpha3 - 8*A**2*H**2*alpha2
    + 360*H**4*alpha3 - 40*H**2*alpha2 + alpha1
PASS e44 - e00 = 6 (A^2 + 1) H^2 V with a polynomial V
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    linear_member_vacuum_factor
PASS Gauss-Bonnet: V = 1 - 8 alpha2 H^2 (A^2 + 5)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_gauss_bonnet_vacuum_linear
PASS for every a4: kappa (rho + p0) = -6 (a4'^2 + H^2) V(a4')
PASS rho, p and V of the linear member equal the record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_linear_member
```

Text cell:

```
## 15. The last check

The last cell checks that the five figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [30]:

```
figure_names = ["12a_1_riemann_matrix.png", "12a_2_einstein_components.png",
                "12a_3_lovelock_heat_maps.png", "12a_4_evolution_factor.png",
                "12a_5_no_vacuum_gap.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all five figure files exist")
all_checks_passed()
```

Out [30]:

```
PASS all five figure files exist
ALL 59 CHECKS PASSED (notebook 12a)
```

Text cell:

```
## 16. What this notebook showed

- From the metric alone, sympy finds 37 nonzero Christoffel symbols and 156 nonzero
  curvature components $R^{ab}{}_{mn}$ (39 with $a < b$, $m < n$), none of which
  contains the warp factor or $e^{a_4}$ (PROVED, as in the Revision record).
- The Einstein tensor is diagonal with four distinct components, and the three
  Lovelock tensors built with the generalized Kronecker delta agree, in all 64
  components, with the Rust program and the Wolfram record; $E_{(2)}$ equals the
  classical Gauss-Bonnet tensor; each $E_{(k)}$ has zero divergence (PROVED).
- The field equations reduce to a constraint ($x_4$), an evolution equation
  $a_4''F(a_4') = \kappa(p_3 - p_t)$, a hidden equation ($x_0$), and conditions on
  the source: $p_3 + p_t = 2p_0$, no dependence on $x_0$, no mixed components
  (PROVED).
- The derivative of the constraint is $3a_4'$ times the evolution equation, and
  the source must obey $\rho' = -3a_4'(p_3 - p_t)$ (PROVED).
- In Einstein gravity there is no vacuum for $H > 0$, and every source has
  $\kappa(\rho + p_0) = -6((a_4')^2 + H^2) < 0$ (PROVED).
- For the linear member $a_4 = AHx_4 + a_0$ the three pressures are equal and
  constant, and the equations contain $A$ only as $A^2$: the exponential deflation
  of the extra times ($A > 0$) is a choice of sign, not selected by the equations
  (PROVED).
```

12.16 Line-by-line walk-through of Notebook 12a

The notebook has 30 code cells, In [1] to In [30]. This section explains every line of every one of them, in order. A line or a group of lines is quoted, then explained. Where a cell repeats a pattern already explained, we say so and explain only what is new.

In [1], the set-up cell. Every line that starts with `#` is a **comment**, which Python skips. The first comment lines of the cell are the complete run instructions of Section 12.14, so that the notebook file carries its own instructions. The code starts after the line that announces THE SET-UP (the same in every notebook of this book; it computes no physics).

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module**, a part of Python or of a package, so that the code can use it. `json`, `os`, `textwrap` and `pathlib` come with Python. `from pathlib import Path` takes the one name `Path` out of `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell.

```
NOTEBOOK_ID = "12a"  # this notebook: chapter 12, example a
```

A **variable** is a name for a value; this line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"12a"`. The figure files are named after it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called. (In the notebook the function also has a **docstring**, the text in triple quotes under the `def` line, which says what it does; it is left out here.) `Path.cwd()` is the folder in which Jupyter runs the notebook and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with them (the star unpacks a list into another list). The `for` loop visits these folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If none does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
REPO = find_repository_root()
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The first line calls the function and names its result `REPO`; it is never printed, because it differs from computer to computer while the printed output of a notebook must not. The second line chooses where files are written: `os.environ` holds the **environment variables** of the program (named texts it receives from the computer), and `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and `str(REPO)` otherwise. When you run the notebook it is not set, so files go into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions (docstrings left out). `repository_file` gives the full path of a repository file, for reading a Revision record. `output_file` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates its folder and any missing folder above it, and does nothing if it exists.

```
def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters (the width of a page of the book); `textwrap.fill` breaks the text at blanks and starts every line after the first with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings, so that the figures are the same on every computer. `plt.rcParams.update({...})` sets the size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid (`grid.alpha` 0.3 means 30 per cent opaque). The braces make a **dictionary**: pairs `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: a name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/12a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty JSON dictionary and a line end into the captions file (the comment lines above it in the notebook say so); `encoding="utf-8"` fixes how letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

(Shown without its docstring and its three comment lines.) `setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of

figures so far: the figures are numbered 1, 2, 3, and a cell run twice keeps its number. The file name is the notebook id, the number and the name, for example `12a_1_riemann_matrix.png`. `fig.savefig` writes a PNG file with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time. The caption is stored, the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys), `display(Image(...))` shows the saved picture, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection in square brackets). `check` is the function behind every check: if `condition` is false (`False`), `raise AssertionError(...)` stops the notebook with an error naming the check (an `if` is used instead of the statement `assert`, because Python started with the option that optimises skips every `assert`); otherwise the name is appended to `PASSED` and the line PASS name is printed, followed by a second line naming the Revision record when `record` is given. `record=None` makes that argument optional.

```
def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds the unit when one is given. `all_checks_passed` prints the last line of the notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one output line of In [1].

In [2], the Revision records.

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory
```

Two modules of Python: `contextlib` provides `redirect_stdout`, which sends everything that `print` writes into another place, and `io` provides `StringIO`, a text buffer held in memory.

```
PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
LEAD = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"  # lead check
EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the equations
LOVELOCK = "Revision/gkd_lovelock/results/lovelock-tensors.json"  # Rust GKD output
```

Six short names for the six record files the notebook reads: the sympy report and the Wolfram report of the field equations, the lead's two independent reports, the equations themselves (written by the Wolfram verifier) and the Lovelock tensors written by the Rust program `lovelock_gkd`.

```
def read_json(relative):
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None
```

`read_json` reads a file of the repository as text and turns it into Python dictionaries and lists with `json.loads`. Every report holds a list under the key `"checks"`; each entry is a dictionary with a `"name"` and a `"verdict"`. `record_verdict` walks through that list and returns the verdict of the check with the given name, or `None` if there is no such check.

```
def reproduces(condition, name, report_file, record_name):
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name, record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # all lines at once
```

`reproduces` is a check that passes only when two things are true: the notebook's own result (`condition`) and the record's verdict PASS for the named check (`found`). `with contextlib.redirect_stdout(lines):` makes every `print` inside the indented block write into the buffer `lines` instead of the screen; after the block, `print(lines.getvalue(), end="")` prints the collected PASS and reproduces lines with one call (`end=""` adds no extra line end), so the two lines always stay together.

```
for report_file in (PY, WL, LEAD, EMT):
    verdicts = [entry["verdict"] for entry in read_json(report_file)["checks"]]
    passed = verdicts.count("PASS")  # how many checks of the record passed
    say(f"{report_file}: {passed} of {len(verdicts)} checks PASS")
    check(len(verdicts) > 0 and passed == len(verdicts),
          f"every check of the record {Path(report_file).name} passed")
```

For each of the four reports, a **list comprehension** collects the verdicts of all its checks, `.count("PASS")` counts the PASS verdicts, and `say` prints this count together with the number of checks

of the report. `check` then requires that the report has at least one check and that every one of them passed; `Path(report_file).name` is the last part of the path, the file name, which names the report in the PASS line. So Out [2] shows, for each of the four reports by name, how many checks it has and that all of them passed, followed by a PASS line. This text does not repeat the counts: a report may gain checks when its record is extended, and Out [2] always shows the counts of the reports the notebook was run with.

In [3], the metric.

```
import itertools  # loops over all combinations of indices

import numpy as np  # arrays of numbers, used for the plots
import sympy as sp  # exact algebra with symbols
```

`itertools` (part of Python) produces combinations of indices; `numpy` (as `np`) computes with arrays of floating-point numbers; `sympy` (as `sp`) computes exactly with symbols.

```
H = sp.symbols("H", positive=True)  # the constant H of the metric, H > 0
z = sp.symbols("z", positive=True)  # the hidden angle z = 6 H x0, 0 < z < pi/2
x4 = sp.symbols("x4", real=True)  # the time x4
a4 = sp.Function("a4")(x4)  # the unknown function a4(x4)
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
N = 8  # the number of dimensions
warp = sp.sin(z) ** sp.Rational(1, 3)  # the warp factor sin(z)^(1/3)
```

`sp.symbols` makes a symbol, a letter that stands for any number; `positive=True` and `real=True` tell sympy what kind of number, which lets it simplify safely (for example $\sqrt{H^2} = H$ for $H > 0$). `sp.Function("a4")(x4)` is an unknown function of x_4: sympy can differentiate it without knowing it. `NAMES` lists the coordinates in the author's order; `N = 8`. `sp.Rational(1, 3)` is the exact fraction 1/3 (the floating-point number 0.333... would introduce rounding), and `**` means "to the power".

```
g = ([sp.cot(z) ** 2]  # g00 = cot(z)^2: the hidden direction
     + [sp.exp(2 * a4) * warp] * 3  # g11 = g22 = g33: 3-space, factor e^(2 a4)
     + [sp.Integer(-1)]  # g44 = -1: the time x4
     + [-sp.exp(-2 * a4) * warp] * 3)  # g55 = g66 = g77: extra times, e^(-2 a4)
for name, entry in zip(NAMES, g):
    say(f"g[{name},{name}] = {entry}")
```

A list times 3 repeats it three times, and `+` joins lists; so `g` is the list of the eight diagonal entries of Section 12.3, in the order $x_0, \ldots, x_7$. `zip(NAMES, g)` pairs each name with its entry, and the loop prints the eight lines of Out [3].

In [4], signature and determinant.

```
point = {a4: 0, z: sp.pi / 4}  # any point of the patch gives the same signs
signs = [int(sp.sign(entry.subs(point))) for entry in g]
say(f"signs of the diagonal entries: {signs}")
check(signs == [1, 1, 1, 1, -1, -1, -1, -1],
      "the signature is (4,4): eta = diag(+,+,+,+,-,-,-,-) in the order x0..x7")
```

`entry.subs(point)` replaces a_4 by 0 and z by $\pi/4$ in an entry; `sp.sign` gives $+1$ or -1 and `int` makes it a Python integer. The check compares the list of signs with the signature (4,4).

```
det_g = sp.Mul(*g)  # the product of the eight diagonal entries
say(f"det g = {sp.simplify(det_g)}")
reproduces(sp.simplify(det_g - sp.cos(z) ** 2) == 0,
           "det g = cos(z)^2, so sqrt|det g| = cos z, independent of a4",
           WL, "sqrt_abs_det_g_is_cos_z")
```

`sp.Mul(*g)` multiplies all entries of the list (the star passes the eight entries as eight separate arguments). `sp.simplify` reduces the product to `cos(z)**2`, the result of Section 12.3, and `reproduces` checks that $\det g - \cos^2 z$ simplifies to 0 and that the Wolfram record has the same check.

In [5], the Christoffel symbols.

```
def d(expr, mu):
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7
```

The function `d` is the partial derivative with respect to the coordinate at position `mu` (docstring left out): for position 3 (x_4) it is `sp.diff(expr, x4)`; for position 7 (x_0) it is $6H\,\partial/\partial z$ (the chain rule of Section 12.4); for every other position it is zero, because nothing depends on those coordinates.

```
Gamma = [[[sp.Integer(0)] * N for _ in range(N)] for _ in range(N)]  # 512 zeros
for a, b, c in itertools.product(range(N), repeat=3):  # every index triple
    value = 0
    if a == c:
        value += d(g[a], b)
    if a == b:
        value += d(g[a], c)
    if b == c:
        value -= d(g[b], a)
    Gamma[a][b][c] = value / (2 * g[a])  # Gamma^a_bc of a diagonal metric
```

The first line makes a nested list of $8 \times 8 \times 8 = 512$ exact zeros, `Gamma[a][b][c]` (the underscore `_` is a loop variable whose value is not used). `itertools.product(range(N), repeat=3)` produces all 512 triples (a, b, c). The body is the diagonal formula of Section 12.4: the three `if` lines add $\delta_{ac}\partial_b g_{aa}$, add $\delta_{ab}\partial_c g_{aa}$ and subtract $\delta_{bc}\partial_a g_{bb}$ (`+=` adds to a variable, `-=` subtracts), and the last line divides by $2g_{aa}$.

```
nonzero = [(a, b, c) for a, b, c in itertools.product(range(N), repeat=3)
           if Gamma[a][b][c] != 0]
say(f"{len(nonzero)} of the 512 Christoffel symbols are not zero")
check(len(nonzero) == 37,
      "37 nonzero Christoffel symbols (25 distinct, 12 of them twice)")
check(all(Gamma[a][b][c] == Gamma[a][c][b] for a, b, c in nonzero),
      "Gamma^a_bc = Gamma^a_cb (symmetric in the two lower indices)")
```

The list comprehension keeps the triples whose symbol is not zero (`!=` means "is not equal to"); `len` counts them: 37, as counted in Section 12.4. `all(...)` is true when the condition holds for every triple of the list; the second check is the symmetry in the two lower indices.

In [6], the symbols printed.

```
ad1, ad2, ad3 = sp.symbols("ad1 ad2 ad3", real=True)


def derivatives_as_symbols(expr):
    expr = expr.subs(sp.Derivative(a4, (x4, 3)), ad3)  # highest derivative first
    expr = expr.subs(sp.Derivative(a4, (x4, 2)), ad2)
    return expr.subs(sp.Derivative(a4, x4), ad1)


def readable(expr):
    return sp.simplify(derivatives_as_symbols(expr))
```

Three plain symbols stand for a_4', a_4'' and a_4'''. `derivatives_as_symbols` replaces the derivatives of the function a_4 by them, the highest derivative first (a second derivative contains the first derivative inside it, so replacing the first derivative first could break it). `readable` also simplifies, for printing.

```
for a, b, c in nonzero:
    if b <= c:  # print each symmetric pair once
        say(f"Gamma^{NAMES[a]}_{NAMES[b]}{NAMES[c]} = {readable(Gamma[a][b][c])}")
```

The loop prints each of the 25 distinct symbols once (`b <= c` skips the second member of each symmetric pair). Out [6] shows exactly the symbols derived by hand in Section 12.4, for example `Gamma^x5_x4x5 = -ad1` and `Gamma^x0_x0x0 = -12*H/sin(2*z)`.

In [7], the Riemann tensor.

```
cz = sp.symbols("cz", positive=True)  # cz = cot z > 0 on the patch
TRIG = {sp.sin(z): 1 / sp.sqrt(1 + cz ** 2), sp.cos(z): cz / sp.sqrt(1 + cz ** 2),
        sp.tan(z): 1 / cz, sp.cot(z): cz}  # every function of z through cot z
```

A positive symbol `cz` stands for $\cot z$, and the dictionary `TRIG` says how to write each function of z through it: from $\cot z = \cos z / \sin z$ and $\sin^2 z + \cos^2 z = 1$ follow $\sin z = 1/\sqrt{1+\cot^2 z}$ and $\cos z = \cot z/\sqrt{1+\cot^2 z}$ on the patch.

```
def clean(expr):
    expr = sp.expand_trig(derivatives_as_symbols(expr)).subs(TRIG)
    return sp.expand(sp.cancel(sp.powsimp(sp.expand(expr), force=True)))
```

`clean` brings an expression to one canonical form (docstring left out): derivatives become `ad1`, `ad2`, `ad3`; `sp.expand_trig` writes $\sin 2z$ as $2\sin z\cos z$; `.subs(TRIG)` writes every function of z through `cz`; `sp.expand` multiplies out; `sp.powsimp(..., force=True)` combines powers of the same base (so that $e^{2a_4}e^{-2a_4}$ becomes 1 and powers of $1+\cot^2 z$ combine); `sp.cancel` cancels common factors of numerator and denominator; the last `sp.expand` multiplies out again. Two expressions that are equal then look equal.

```
R = {}  # R[(a, b, m, n)] = R^{ab}_{mn}; only the components that are not zero
for a, b, m, n in itertools.product(range(N), repeat=4):
    if a == b or m == n:
        continue  # antisymmetry: zero when the two indices of a pair are equal
    value = (d(Gamma[a][b][n], m) - d(Gamma[a][b][m], n)
             + sum(Gamma[a][m][e] * Gamma[e][b][n] - Gamma[a][n][e] * Gamma[e][b][m]
                   for e in range(N)))  # R^a_bmn in the MTW convention
    if value != 0:
        value = clean(value / g[b])  # raise the index b: multiply by g^bb
        if value != 0:
            R[(a, b, m, n)] = value
```

`R` starts as an empty dictionary; its keys will be index lists (a, b, m, n). The loop runs over all $8^4 = 4096$ index lists; `continue` skips those with $a = b$ or $m = n$, which are zero by antisymmetry. The expression for `value` is the definition of Section 12.5 term by term: two derivatives and the sum over e of the two products (`sum(... for e in range(N))` adds the expression for $e = 0, \ldots, 7$). If the result is not zero, dividing by g_{bb} raises the index b (multiplication by $g^{bb} = 1/g_{bb}$), `clean` simplifies, and a component that is still not zero is stored.

```
ordered = {key: value for key, value in R.items()
           if key[0] < key[1] and key[2] < key[3]}  # a < b and m < n
report("nonzero components R^ab_mn of the 4096", len(R))
report("nonzero components with a < b and m < n", len(ordered))
```

A **dictionary comprehension** keeps the components with $a < b$ and $m < n$ (`key[0]` is the first index of the key). The two RESULT lines report 156 and 39 (Section 12.5).

In [8], four facts about the curvature.

```
reproduces(len(R) == 156, "156 nonzero components R^ab_mn of the 4096",
           PY, "mixed_riemann_free_of_warp_and_a4")
reproduces(len(ordered) == 39, "39 nonzero components with a < b and m < n",
           WL, "mixed_riemann_free_of_warp_and_a4")
```

The two counts, compared with the sympy and the Wolfram record (both records state the counts in the detail of the named check).

```
free = all(not v.has(a4) and not v.has(z) and not v.has(sp.exp) for v in R.values())
reproduces(free, "R^ab_mn is free of the warp factor and of e^(a4)",
           PY, "mixed_riemann_free_of_warp_and_a4")
```

`v.has(x)` is true when the expression `v` contains `x` anywhere. The condition says that no component contains the function a_4, the angle z (so no warp factor) or an exponential.

```
laurent = all(sp.expand(v * cz).is_polynomial(H, ad1, ad2, cz) for v in R.values())
reproduces(laurent, "every R^ab_mn is a Laurent polynomial in cot z",
           PY, "riemann_entries_laurent")
```

Multiplying a component by $\cot z$ must give a polynomial in H, a_4', a_4'' and $\cot z$; then the component itself is a polynomial divided at most by $\cot z$, a **Laurent polynomial** in $\cot z$.

```
antisymmetric = all(R.get((b, a, m, n)) == -v and R.get((a, b, n, m)) == -v
                    for (a, b, m, n), v in R.items())
reproduces(antisymmetric, "R^ab_mn = -R^ba_mn = -R^ab_nm",
           PY, "riemann_pair_antisymmetry")
```

`R.get(key)` returns the stored value or `None` if the key is absent. For every stored component the exchanged components must be stored with the opposite value.

In [9], the 39 components grouped.

```
def pair_name(a, b):
    return NAMES[a] + NAMES[b]


groups = {}  # printed value -> the components that have it
for (a, b, m, n), value in ordered.items():
    label = f"{pair_name(a, b)}|{pair_name(m, n)}"
    groups.setdefault(str(value), []).append(label)
for value, members in groups.items():
    say(f"{value} ({len(members)}): " + " ".join(members))
```

`pair_name` joins two coordinate names, for example `x1x4`. The first loop files each component under its printed value: `groups.setdefault(str(value), [])` returns the list stored for this value, creating an empty one the first time, and `.append(label)` adds the label `x1x4|x1x4`. The second loop prints each value, the number of its members and the members joined by blanks. Out [9] is the table of Section 12.5: seven groups, of which two are the off-diagonal ones with `cz`.

In [10], figure 1: the Riemann tensor as a heat map.

```
PAIRS28 = [(a, b) for a in range(N) for b in range(a + 1, N)]  # the 28 pairs
SAMPLE = {H: 1, ad1: sp.Rational(3, 5), ad2: sp.Rational(3, 10),
          cz: 1 / sp.sqrt(3)}  # H = 1, a4' = 0.6, a4'' = 0.3, z = pi/3
table = np.zeros((28, 28))
for i, (a, b) in enumerate(PAIRS28):
    for j, (m, n) in enumerate(PAIRS28):
        if (a, b, m, n) in R:
            table[i, j] = float(R[(a, b, m, n)].subs(SAMPLE))
```

`PAIRS28` lists the 28 pairs $a < b$. `SAMPLE` is a sample point at which the exact components are evaluated: $H = 1$, $a_4' = 0.6$, $a_4'' = 0.3$ and $\cot z = 1/\sqrt{3}$, that is $z = \pi/3$. `np.zeros((28, 28))` is a 28 by 28 table of zeros. `enumerate` gives each pair together with its position `i` (or `j`); for every stored component the table entry becomes its value at the sample point, converted to a floating-point number by `float`.

```
limit = np.abs(table).max()  # the same scale for both signs
labels = [pair_name(a, b) for a, b in PAIRS28]
fig, ax = plt.subplots(figsize=(7.5, 6.6))
image = ax.imshow(table, cmap="RdBu_r", vmin=-limit, vmax=limit)
```

`limit` is the largest absolute value in the table. `plt.subplots` makes a figure and one pair of axes, 7.5 by 6.6 inches. `ax.imshow` draws the table as coloured squares with the colour map `RdBu_r` (blue for negative, white for zero, red for positive); `vmin=-limit, vmax=limit` make the scale symmetric, so that white means exactly zero.

```
ax.set_xticks(range(28))
ax.set_xticklabels(labels, rotation=90, fontsize=6)
ax.set_yticks(range(28))
ax.set_yticklabels(labels, fontsize=6)
ax.set_xlabel("lower index pair $(m, n)$")
ax.set_ylabel("upper index pair $(a, b)$")
ax.grid(False)
ax.set_title("Riemann tensor $R^{ab}{}_{mn}$ at $H=1$, $a_4'=0.6$, "
             "$a_4''=0.3$, $z=\\pi/3$")
fig.colorbar(image, ax=ax, shrink=0.8, label="value (units of $H^2$)")
```

These lines put a tick at each of the 28 positions on both axes and label it with the pair name (turned by 90 degrees on the horizontal axis), name the axes, switch off the grid (it would cover the squares), set the title (text between dollar signs is typeset as mathematics; a doubled backslash in a Python string is one backslash) and add a colour bar that translates colours into values.

```
save_figure(fig, "riemann_matrix",
            "The Riemann tensor $R^{ab}{}_{mn}$ of the author's metric as a 28 by "
            ...)
```

`save_figure` saves the figure as `12a_1_riemann_matrix.png` with its caption (printed in full in the notebook text; Python joins strings written next to each other into one). **What the figure shows.** Almost all the colour sits on the diagonal: the 27 pair curvatures of Section 12.5 (the diagonal square of x_4x_0 is white, because that pair curvature is zero). The few coloured squares off the diagonal are the 12 components that couple, for example, x_1x_4 with x_1x_0; they are proportional to Ha_4' and would vanish for a constant a_4.

In [11], Ricci, Ricci scalar and Einstein tensor.

```
Ricci = [[sp.expand(sum(R.get((a, h, a, j), 0) for a in range(N)))
          for j in range(N)] for h in range(N)]  # R^h_j
ricci_scalar = sp.expand(sum(Ricci[h][h] for h in range(N)))
G = [[sp.expand(Ricci[h][j] - (ricci_scalar / 2 if h == j else 0))
      for j in range(N)] for h in range(N)]  # G^h_j
```

The three definitions of Section 12.6. `R.get((a, h, a, j), 0)` returns 0 for a component that is not stored. `Ricci[h][j]` is the sum over a; `ricci_scalar` the sum of the diagonal; `G` subtracts half the Ricci scalar on the diagonal only (`x if condition else y` chooses between two values).

```
report("Ricci scalar R", ricci_scalar)
for h in range(N):
    say(f"G^{NAMES[h]}_{NAMES[h]} = {G[h][h]}")
```

The RESULT line shows $R = 6(a_4')^2 - 42H^2$ and the loop prints the eight diagonal components: exactly those derived by hand in Section 12.6.

In [12], the Einstein tensor compared with the records.

```
reproduces(all(G[h][j] == 0 for h in range(N) for j in range(N) if h != j),
           "G^h_j is diagonal (in particular G^x4_x0 = G^x0_x4 = 0)",
           LEAD, "einstein_off_diagonal_zero")
reproduces(G[1][1] == G[2][2] == G[3][3] and G[5][5] == G[6][6] == G[7][7],
           "G^x1_x1 = G^x2_x2 = G^x3_x3 and G^x5_x5 = G^x6_x6 = G^x7_x7",
           LEAD, "einstein_isotropy")
reproduces(not any(G[h][j].has(cz) for h in range(N) for j in range(N)),
           "every component of G is independent of x0",
           LEAD, "einstein_x0_independent")
```

Three structural checks against the lead's record: every off-diagonal component is zero (the cancellation of Section 12.6), the components are equal within 3-space and within the extra times (Python allows the chain `x == y == z`), and no component contains $\cot z$ (`any` is true when at least one element is true; `not any` when none is).

```
expected = {4: 3 * ad1 ** 2 + 21 * H ** 2, 1: ad2 - 3 * ad1 ** 2 + 15 * H ** 2,
            5: -ad2 - 3 * ad1 ** 2 + 15 * H ** 2, 0: -3 * ad1 ** 2 + 15 * H ** 2}
reproduces(all(sp.expand(G[i][i] - v) == 0 for i, v in expected.items()),
           "the four Einstein components agree with the record",
           PY, "einstein_components")
```

The dictionary `expected` holds the four distinct components, keyed by their positions (3 for x_4, 0 for x_1, 4 for x_5, 7 for x_0); the check requires each difference to expand to zero.

In [13], figure 2: the Einstein components.

```
u = np.linspace(-3.0, 3.0, 301)  # a4'/H
fig, ax = plt.subplots()
ax.plot(u, 3 * u ** 2 + 21, label="$G^{x_4}{}_{x_4}$ (constraint)")
ax.plot(u, 15 - 3 * u ** 2, color="black",
        label="$G^{x_0}{}_{x_0}$ $=$ $G^{x_1}{}_{x_1}$ $=$ $G^{x_5}{}_{x_5}$ "
              "at $a_4''=0$")
ax.plot(u, 15 - 3 * u ** 2 + 1, "--", label="$G^{x_1}{}_{x_1}$ at $a_4''=H^2$")
ax.plot(u, 15 - 3 * u ** 2 - 1, "--", label="$G^{x_5}{}_{x_5}$ at $a_4''=H^2$")
```

`np.linspace(-3.0, 3.0, 301)` makes 301 equally spaced values of a_4'/H from -3 to 3 (step 0.02). In units $H = 1$ the four formulas of Section 12.6 are evaluated on the whole array at once and drawn with `ax.plot(x, y, ...)`; the string of two hyphens makes a dashed line, and `label=` names the curve in the legend.

```
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("component of $G^\\mu{}_\\nu$ (units of $H^2$)")
ax.set_title("The Einstein tensor of the author's metric")
ax.legend(fontsize=8)
save_figure(fig, "einstein_components",
            ...)
```

Axis labels, title, legend, and `save_figure` with the caption printed in the notebook text. **What the figure shows.** The time component is a parabola that opens upwards and never falls below $21H^2$; the hidden component opens downwards. The two dashed curves lie a distance H^2 above and below the black one: only the difference of the 3-space and the extra-time components feels a_4'', which is why the evolution equation of Section 12.9 is their difference.

In [14], the generalized Kronecker delta.

```
def gkd(upper, lower):
    if len(set(upper)) != len(upper) or sorted(upper) != sorted(lower):
        return 0
    position = [upper.index(index) for index in lower]  # where each one sits
    inversions = sum(1 for i in range(len(position))
                     for j in range(i + 1, len(position))
                     if position[i] > position[j])
    return -1 if inversions % 2 else 1
```

`set(upper)` keeps each index once, so `len(set(upper)) != len(upper)` detects a repeated index; `sorted(upper) != sorted(lower)` detects two lists that do not hold the same indices; in both cases the delta is 0. Otherwise `position` lists where each lower index sits in the upper list (`upper.index(x)` is the position of `x`): this is the rearrangement. `inversions` counts the pairs of positions $i < j$ that appear in the wrong order. An even number of inversions means an even rearrangement (sign $+1$), an odd number an odd one (-1); `inversions % 2` is the remainder after division by 2.

```
say(f"gkd([1, 2], [1, 2]) = {gkd([1, 2], [1, 2])}, gkd([1, 2], [2, 1]) = "
    f"{gkd([1, 2], [2, 1])}, gkd([1, 1], [1, 1]) = {gkd([1, 1], [1, 1])}")
nine = list(range(8)) + [0]  # nine indices from eight values: one repeats
reproduces(gkd(nine, nine) == 0 and gkd(list(range(8)), list(range(8))) == 1,
           "a GKD of 9 indices in 8 dimensions is 0: P_(4) = 0",
           WL, "P4_vanishes_pigeonhole")
```

Three small examples are printed: 1, -1 and 0. Then a list of nine indices made of the eight values 0 to 7 and a repeated 0: its delta is 0, while the delta of the eight different indices with themselves is 1. This is the pigeonhole argument of Section 12.7.

In [15], the three Lovelock tensors.

```
tz = sp.symbols("tz", positive=True)  # tz = tan z = 1/cot z
GENERATORS = (H, ad1, ad2, cz, tz)
# xreplace replaces exactly the expression 1/cz by tz and leaves cz itself alone:
FACTORS = [(key, sp.Poly(value.xreplace({1 / cz: tz}), *GENERATORS))
           for key, value in ordered.items()]  # the 39 components as polynomials
```

To multiply thousands of products quickly, each of the 39 components is stored as a sympy **polynomial** (`sp.Poly`) in the five generators H, a_4', a_4'', $\cot z$ and a new symbol $t = \tan z$, listed in `GENERATORS`. The comment line says what the next line does: `value.xreplace({1 / cz: tz})` replaces exactly the expression $1/\cot z$ by t and leaves every $\cot z$ itself unchanged, so that no division is left and the component is a polynomial. `FACTORS` is the list of pairs (index list, polynomial).

```
def lovelock(k):
    zero = sp.Poly(0, *GENERATORS)
    P = [[zero] * N for _ in range(N)]
    L = zero
    terms = 0
```

The function `lovelock(k)` (docstring left out) starts with the zero polynomial, an 8 by 8 table `P` of zeros, a zero scalar `L` and a counter `terms`.

```
    for factors in itertools.product(FACTORS, repeat=k):
        lower = [i for key, _ in factors for i in key[0:2]]  # upper indices of R
        upper = [i for key, _ in factors for i in key[2:4]]  # lower indices of R
        if len(set(lower)) < 2 * k or len(set(upper)) < 2 * k:
            continue  # a repeated index: the GKD is zero
        terms += 1
```

`itertools.product(FACTORS, repeat=k)` produces every list of k components (39^k lists). For each, `lower` collects the two upper indices of each Riemann factor (`key[0:2]` is the first two entries of the key): in the formula of Section 12.7 these are the lower indices $j_1 \dots j_{2k}$ of the delta. `upper` collects the lower indices of the factors, the upper indices $h_1 \dots h_{2k}$ of the delta. A list with a repeated index is skipped, because its delta is zero; the others are counted.

```
        product = sp.Poly(4 ** k, *GENERATORS)  # the factor 4^k
        for _, poly in factors:
            product = product * poly
        L = L + gkd(upper, lower) * product
        for h, j in itertools.product(range(N), repeat=2):
            sign = gkd([h] + upper, [j] + lower)
            if sign:
                P[h][j] = P[h][j] + sign * product
```

The product of the k factors is multiplied by 4^k: each factor stands for its four orderings ($b < a$ or $n < m$ as well), which give the same contribution (the text cell before the cell explains why). The product, with the sign of the delta, is added to the scalar `L` and, for every pair (h, j) with a nonzero delta `gkd([h] + upper, [j] + lower)`, to `P[h][j]` (`if sign:` is true for $+1$ and -1, false for 0).

```
    back = {tz: 1 / cz}  # tan z back to 1/cot z
    P = [[sp.expand(P[h][j].as_expr().subs(back)) for j in range(N)]
         for h in range(N)]
    return P, sp.expand(L.as_expr().subs(back)), terms
```

At the end `.as_expr()` turns each polynomial back into an ordinary expression, t is replaced by $1/\cot z$ again, and the function returns the table, the scalar and the count.

```
P, Lscalar, E = {}, {}, {}
for k in (1, 2, 3):
    P[k], Lscalar[k], terms = lovelock(k)
    E[k] = [[sp.expand(-P[k][h][j] / 2 ** (k + 1)) for j in range(N)]
            for h in range(N)]  # E_(k) = -P_(k)/2^(k+1)
    say(f"k = {k}: {terms} lists of index pairs contribute; L_({k}) = {Lscalar[k]}")
```

For $k = 1, 2, 3$ the cell computes $P_{(k)}$, $L_{(k)}$ and the number of contributing lists, forms $E_{(k)} = -P_{(k)}/2^{k+1}$ and prints a line. Out [15] shows 39, 690 and 4410 contributing lists and the three Lovelock scalars; for example $L_{(1)} = 12(a_4')^2 - 84H^2$, which is twice the Ricci scalar.

In [16], the Lovelock tensors compared with the records.

```
reproduces(all(sp.expand(E[1][h][j] - G[h][j]) == 0
               for h in range(N) for j in range(N)),
           "E_(1) = G in all 64 components (P_(1) = -4 G)",
           PY, "P1_equals_minus_4_Einstein")
lovelock_json = read_json(LOVELOCK)
```

The first check is the statement $E_{(1)} = G$ of Section 12.7 in all 64 components. The next line reads the Rust record.

```
for k in (1, 2, 3):
    Ek = E[k]
    structure = (all(Ek[h][j] == 0 for h in range(N) for j in range(N) if h != j)
                 and Ek[1][1] == Ek[2][2] == Ek[3][3]
                 and Ek[5][5] == Ek[6][6] == Ek[7][7]
                 and not any(Ek[h][h].has(cz) for h in range(N)))
    reproduces(structure, f"E_({k}) diagonal, free of x0, isotropic in each group",
               PY, f"P{k}_structure")
```

For each k: the tensor is diagonal, equal within 3-space and within the extra times, and free of $\cot z$; compared with the record checks `P1_structure`, `P2_structure`, `P3_structure`.

```
    trace = sp.expand(sum(P[k][h][h] for h in range(N)) - (8 - 2 * k) * Lscalar[k])
    reproduces(trace == 0, f"trace identity sum_h P_({k})^h_h = (8 - {2 * k}) L_({k})",
               PY, f"P{k}_trace_identity")
```

The trace identity $\sum_h {P_{(k)}}^h{}_h = (8 - 2k)L_{(k)}$: summing the delta with $h = j$ over the $8 - 2k$ values of h that are not among the other $2k$ indices gives $8 - 2k$ times the scalar.

```
    text = (lovelock_json[f"L{k}"].replace("Derivative[1][a4][x4]", "ad1")
            .replace("Derivative[2][a4][x4]", "ad2"))
    for b4_text, value in (("Derivative[2][b4][x4]", "0"),
                           ("Derivative[1][b4][x4]", "0"), ("b4[x4]", "1")):
        text = text.replace(b4_text, value)  # the record at b4 = 1
    rust = sp.sympify(text.replace("^", "**"), locals={"H": H, "ad1": ad1, "ad2": ad2})
    reproduces(sp.expand(rust - Lscalar[k]) == 0,
               f"L_({k}) equals the Rust GKD program", PY, f"L{k}_equals_gkd_branch")
```

The Rust record stores each scalar as a text in the Wolfram Language; `.replace` turns the Wolfram name of the derivative into `ad1` and the power sign `^` into `**`, and `sp.sympify` turns the text into a sympy expression (`locals` says which names are our symbols). The scalars must agree. Out [16] shows the ten PASS lines.

In [17], all 192 components compared with the Rust and Wolfram records.

```
symbols_of_monomial = (H, ad1, ad2, ad3, sp.Symbol("ad4"), sp.Symbol("Ee"),
                       sp.Symbol("Ss"), cz)  # the order of the exponents


def from_monomials(monomials):
    total = sp.Integer(0)
    for numerator, denominator, exponents in monomials:
        if any(exponents[9:13]):  # a derivative of b4: zero at b4 = 1
            continue
        term = sp.Rational(numerator, denominator)  # b4^(1/2) = 1 (exponent 8)
        for symbol, power in zip(symbols_of_monomial, exponents[:8]):
            term *= symbol ** power
        total += term
    return total
```

The Rust program stores each component as a list of **monomials**: a fraction (numerator, denominator) and the eight exponents of H, a_4', a_4'', a_4''', a_4'''', e^{a_4}, $\sin^{1/3} z$ and $\cot z$, in this order. `from_monomials` rebuilds the expression: each term is the fraction times every symbol to its power (`*=` multiplies a variable), and the terms are added.

```
for k in (1, 2, 3):
    block = lovelock_json[f"P{k}_mixed_up_h_down_j"]
    same = all(sp.expand(from_monomials(block[f"{NAMES[h]},{NAMES[j]}"]["monomials"])
                         - P[k][h][j]) == 0 for h in range(N) for j in range(N))
    reproduces(same, f"all 64 components of P_({k}) equal the Rust GKD record",
               PY, f"P{k}_equals_gkd_branch_monomials")
```

For each k the record's block is a dictionary keyed by `"x1,x1"` and so on; every one of the 64 rebuilt components must equal the notebook's. A component of the record that contained e^{a_4}, the warp or a third derivative would differ, so the check also confirms that none does.

```
equations_json = read_json(EQUATIONS)
LOCALS = {"ad1": ad1, "ad2": ad2, "H": H}


def parse(text, names=None):
    return sp.sympify(text.replace("^", "**"), locals=names or LOCALS)
```

The Wolfram record is read; `parse` turns one of its texts into a sympy expression (`names or LOCALS` uses the dictionary `names` when one is given and `LOCALS` otherwise; docstring left out).

```
POSITIONS = {"x1x1": (1, 1), "x4x4": (4, 4), "x5x5": (5, 5), "x0x0": (0, 0),
             "x4x0": (4, 0), "x0x4": (0, 4)}
same = all(sp.expand(parse(equations_json["lovelockTensors"][f"E{k}"][key]["input"])
                     - E[k][h][j]) == 0
           for k in (1, 2, 3) for key, (h, j) in POSITIONS.items())
reproduces(same, "E_(1), E_(2), E_(3) equal the Wolfram record a4-equations.json",
           PY, "json_lovelock_components")
```

The record stores six components of each tensor (the four distinct diagonal ones and the two mixed ones x_4x_0, x_0x_4), each with an `"input"` text. All 18 must equal the notebook's.

In [18], an independent route to the Gauss-Bonnet tensor.

```
def contract(mu, nu):
    term_a = ricci_scalar * Ricci[mu][nu]
    term_b = sum(Ricci[mu][a] * Ricci[a][nu] for a in range(N))
    term_c = sum(R.get((mu, a, nu, b), 0) * Ricci[b][a]
                 for a in range(N) for b in range(N))
    term_d = sum(R.get((mu, a, b, c), 0) * R.get((b, c, nu, a), 0)
                 for a in range(N) for b in range(N) for c in range(N))
    return 2 * (term_a - 2 * term_b - 2 * term_c + term_d)
```

The bracket of the Lanczos formula printed in the text cell before the cell, term by term: $R\,R^{\mu}{}_{\nu}$, $\sum_a R^{\mu}{}_a R^a{}_{\nu}$, $\sum_{a,b} R^{\mu a}{}_{\nu b} R^b{}_a$ and $\sum_{a,b,c} R^{\mu a}{}_{bc} R^{bc}{}_{\nu a}$, combined with the factors 2, -4, -4 and 2 (docstring left out).

```
gauss_bonnet_scalar = (ricci_scalar ** 2
                       - 4 * sum(Ricci[a][b] * Ricci[b][a]
                                 for a in range(N) for b in range(N))
                       + sum(v * R.get((m, n, a, b), 0)
                             for (a, b, m, n), v in R.items()))
lanczos = [[sp.expand(contract(mu, nu)
                      - (gauss_bonnet_scalar / 2 if mu == nu else 0))
            for nu in range(N)] for mu in range(N)]
check(all(sp.expand(lanczos[h][j] - E[2][h][j]) == 0
          for h in range(N) for j in range(N)),
      "E_(2) (GKD) equals the classical Gauss-Bonnet tensor for every a4")
```

The Gauss-Bonnet scalar $R^2 - 4R^a{}_b R^b{}_a + R^{ab}{}_{cd} R^{cd}{}_{ab}$ (the last sum multiplies each stored component by the one with the two pairs exchanged); the Lanczos tensor subtracts half of it on the diagonal. The check compares all 64 components with $E_{(2)}$ from the GKD: two entirely different routes give the same tensor for every a_4.

```
A = sp.symbols("A", real=True)  # the slope of the linear member, a4' = A H
LINEAR = {ad1: A * H, ad2: 0}
reproduces(sp.expand(lanczos[4][4].subs(LINEAR)
                     + H ** 4 * (36 * A ** 4 + 120 * A ** 2 + 420)) == 0,
           "linear member: Gauss-Bonnet H^x4_x4 = -H^4 (36 A^4 + 120 A^2 + 420)",
           LEAD, "gauss_bonnet_rho_alpha2")
reproduces(sp.expand(lanczos[1][1].subs(LINEAR)
                     - 12 * H ** 4 * (A ** 4 + 14 * A ** 2 - 15)) == 0,
           "linear member: Gauss-Bonnet H^x1_x1 = 12 H^4 (A^4 + 14 A^2 - 15)",
           LEAD, "gauss_bonnet_p_alpha2")
```

For the linear member ($a_4' = AH$, $a_4'' = 0$) the time and 3-space components are compared with the numbers of the lead's independent computation.

In [19], the Bianchi identity.

```
def divergence(T):
    return [clean(sum(d(T[m][n], m) for m in range(N))
                  + sum(Gamma[m][m][l] * T[l][n] for m in range(N) for l in range(N))
                  - sum(Gamma[l][m][n] * T[m][l] for m in range(N) for l in range(N)))
            for n in range(N)]
```

The covariant divergence of a mixed tensor, $\nabla_\mu T^\mu{}_\nu = \sum_\mu \partial_\mu T^\mu{}_\nu + \sum_{\mu,l} \Gamma^\mu{}_{\mu l} T^l{}_\nu - \sum_{\mu,l} \Gamma^l{}_{\mu\nu} T^\mu{}_l$, for each ν; the result is a list of eight cleaned expressions (docstring left out).

```
AS_FUNCTIONS = {ad2: sp.Derivative(a4, (x4, 2)), ad1: sp.Derivative(a4, x4)}
for k in (1, 2, 3):
    Ek_functions = [[E[k][h][j].subs(AS_FUNCTIONS) for j in range(N)]
                    for h in range(N)]
    reproduces(all(value == 0 for value in divergence(Ek_functions)),
               f"nabla_mu E_({k})^mu_nu = 0 for nu = x0..x7",
               PY, f"E{k}_divergence_free")
```

The symbols `ad1`, `ad2` are plain letters, which `sp.diff` would treat as constants; so they are written back as derivatives of the function $a_4(x_4)$ before the divergence is taken. For each k all eight components of the divergence must vanish: the Bianchi identity of Section 12.10.

In [20], figure 3: the three Lovelock tensors as heat maps.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.4))
for ax, k in zip(axes, (1, 2, 3)):
    values = np.array([[float(E[k][h][j].subs(SAMPLE)) for j in range(N)]
                       for h in range(N)])
    scale = np.abs(values).max()
    ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
```

`plt.subplots(1, 3, ...)` makes one row of three axes. For each tensor the 64 components are evaluated at the sample point of In [10] into an 8 by 8 array and drawn with its own symmetric colour scale.

```
    for h in range(N):  # write each diagonal value into its cell
        value = values[h, h]
        text = f"{value:.0f}" if abs(value) >= 100 else f"{value:.1f}"
        ink = "white" if abs(value) > 0.6 * scale else "black"  # readable on dark
        ax.text(h, h, text, ha="center", va="center", fontsize=6.5, color=ink)
```

Each diagonal value is written into its cell: as a whole number (`:.0f`) when it is at least 100 and with one decimal (`:.1f`) otherwise, in white on dark cells and black on light ones; `ha` and `va` centre the text.

```
    ax.set_xticks(range(N))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(N))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)
```

```
    ax.set_title(f"$E_{{({k})}}{{}}^h{{}}_j$")
fig.suptitle("The three Lovelock tensors at $H=1$, $a_4'=0.6$, $a_4''=0.3$ "
             "(row $h$, column $j$)")
save_figure(fig, "lovelock_heat_maps",
            ...)
```

Ticks labelled x_0 to x_7, no grid, a title per panel (in an f-string a doubled brace prints one brace, so the title is the mathematics ${E_{(k)}}^h{}_j$), a title for the whole figure, and `save_figure`. **What the figure shows.** All three tensors are diagonal with only four different values, one each for 3-space, the time, the extra times and the hidden direction: the structure that reduces the 64 field equations to four.

In [21], the field equations.

```
alpha1, alpha2, alpha3 = sp.symbols("alpha1 alpha2 alpha3", real=True)
Lam, kappa = sp.symbols("Lam kappa", real=True)  # Lambda and kappa
rho, p3, pt, p0, q40, q04 = sp.symbols("rho p3 pt p0 q40 q04", real=True)
ALPHA = {1: alpha1, 2: alpha2, 3: alpha3}
```

Symbols for the couplings, Λ (named `Lam`, because `lambda` is a word of Python), κ and the six parts of the source; `ALPHA` finds the coupling of each k.

```
LHS = [[sp.expand(sum(ALPHA[k] * E[k][h][j] for k in (1, 2, 3))
                  + (Lam if h == j else 0)) for j in range(N)] for h in range(N)]
T = [[sp.Integer(0)] * N for _ in range(N)]
for i, value in enumerate([p0, p3, p3, p3, -rho, pt, pt, pt]):
    T[i][i] = value  # the diagonal of the source, in the order x0..x7
T[4][0], T[0][4] = q40, q04  # the two mixed components x4-x0 and x0-x4
```

`LHS` is the left-hand side $\sum_k \alpha_k {E_{(k)}}^h{}_j + \Lambda\delta^h_j$ of Section 12.8. `T` is the source: zeros, then the diagonal from the list, then the two mixed components (two variables can be set in one line).

```
off_diagonal_zero = all(LHS[h][j] == 0 for h in range(N) for j in range(N)
                        if h != j)
groups_equal = (LHS[1][1] == LHS[2][2] == LHS[3][3]
                and LHS[5][5] == LHS[6][6] == LHS[7][7])
reproduces(off_diagonal_zero and groups_equal,
           "every off-diagonal left-hand side is 0; four independent components",
           WL, "independent_components")
```

The two facts of the reduction: every off-diagonal left-hand side is zero, and the left-hand sides are equal within the two groups.

```
e11, e44, e55, e00 = (sp.expand(LHS[i][i] - Lam) for i in (1, 4, 5, 0))
say(f"constraint (x4): {e44} + Lam = -kappa*rho")
say(f"hidden (x0): {e00} + Lam = kappa*p0")
```

The four abbreviations $e_{11}, e_{44}, e_{55}, e_{00}$ (the diagonal without Λ; a parenthesised generator expression yields the four values, which are unpacked into four names). The constraint and the hidden equation

are printed for all three couplings; set $\alpha_2 = \alpha_3 = 0$, $\alpha_1 = 1$ in Out [21] and you read off $3(a_4')^2 + 21H^2$ and $15H^2 - 3(a_4')^2$.

In [22], the evolution equation.

```
F = sp.expand(sp.cancel((e11 - e55) / ad2))  # (e11 - e55)/a4''
report("F(a4')", F)
reproduces(not F.has(ad2) and sp.expand(ad2 * F - (e11 - e55)) == 0,
           "evolution: e11 - e55 = a4'' F(a4') = kappa (p3 - pt)",
           PY, "evolution_factorises")
```

$(e_{11} - e_{55})/a_4''$ is simplified by `sp.cancel`; the result F contains no a_4'', and $a_4'' F$ gives back $e_{11} - e_{55}$ exactly. Out [22] prints F, the polynomial of Section 12.9.

```
coefficients = sp.Poly(F, ad1, H).coeffs()  # every coefficient must vanish
solutions = sp.solve(coefficients, [alpha1, alpha2, alpha3], dict=True)
reproduces(solutions == [{alpha1: 0, alpha2: 0, alpha3: 0}],
           "F vanishes identically only for alpha1 = alpha2 = alpha3 = 0",
           PY, "evolution_F_not_identically_zero")
```

Read as a polynomial in a_4' and H, F has coefficients that depend on the couplings (`.coeffs()` lists them). F is zero for every a_4' and H only if every coefficient is zero; `sp.solve` solves these equations for the couplings and finds only $\alpha_1 = \alpha_2 = \alpha_3 = 0$.

```
reproduces(sp.expand(e11 + e55 - 2 * e00) == 0,
           "e11 + e55 = 2 e00 identically: the source needs p3 + pt = 2 p0",
           PY, "algebraic_identity")
reproduces(not e44.has(ad2) and not e00.has(ad2),
           "the x4 (constraint) and x0 components contain a4' but not a4''",
           PY, "constraint_and_x0_first_order")
```

The algebraic identity of Section 12.9, and the first-order character of the constraint and of the hidden equation.

In [23], constraint propagation and the record's equations.

```
reproduces(sp.expand(sp.diff(e44, ad1) * ad2 - 3 * ad1 * (e11 - e55)) == 0,
           "d/dx4 e44 = 3 a4' (e11 - e55) (constraint propagation)",
           PY, "bianchi_x4")
```

e_{44} depends on x_4 only through a_4', so by the chain rule its derivative with respect to x_4 is $(\partial e_{44}/\partial a_4')\, a_4''$, written `sp.diff(e44, ad1) * ad2`. It must equal $3a_4'(e_{11} - e_{55})$: the identity of Section 12.10.

```
general = equations_json["generalSource"]
NAMES_RECORD = {"ad1": ad1, "ad2": ad2, "H": H, "alpha1": alpha1, "alpha2": alpha2,
                "alpha3": alpha3, "Lam": Lam, "kappa": kappa, "rho": rho, "p3": p3,
                "pt": pt, "p0": p0, "AA": A}
```

```
def parse_equation(text):
    left, right = text.split(" == ")
    return sp.expand(parse(left, NAMES_RECORD) - parse(right, NAMES_RECORD))
```

The record's general equations are read; `NAMES_RECORD` maps every name used in the record to our symbols (the record writes the slope as `AA`). `parse_equation` splits a text `left == right` at the equality sign and returns left minus right (docstring left out).

```
mine = {"constraint_x4": e44 + Lam + kappa * rho,
        "space_x1_eq_x2_eq_x3": e11 + Lam - kappa * p3,
        "extraTime_x5_eq_x6_eq_x7": e55 + Lam - kappa * pt,
        "hidden_x0": e00 + Lam - kappa * p0,
        "evolution_x1_minus_x5": ad2 * F - kappa * (p3 - pt),
        "algebraic_condition": p3 + pt - 2 * p0}
same = all(sp.expand(parse_equation(general[key]["input"]) - value) == 0
           for key, value in mine.items())
same = same and sp.expand(parse(general["evolution_F"]["input"], NAMES_RECORD)
                          - F) == 0
reproduces(same, "the six equations and F equal the record a4-equations.json",
           PY, "json_general_source_equations")
```

The notebook's own six equations, each written as left minus right under the record's key, are compared with the record's, and so is F.

In [24], figure 4: the factor F.

```
F_numeric = sp.lambdify((ad1, alpha1, alpha2, alpha3), F.subs(H, 1), "numpy")
u = np.linspace(-3.0, 3.0, 601)  # a4'/H
cases = [(1, 0.0, 0.0, "Einstein: $\\alpha_1=1$"),
         (1, 0.005, 0.0, "$\\alpha_2H^2=0.005$"),
         (1, 0.01, 0.0, "$\\alpha_2H^2=0.01$"),
         (1, 0.02, 0.0, "$\\alpha_2H^2=0.02$"),
         (1, 0.01, 0.0005, "$\\alpha_2H^2=0.01$, $\\alpha_3H^4=0.0005$")]
```

`sp.lambdify` turns the exact formula (with $H = 1$) into a fast numpy function of a_4' and the three couplings. `u` holds 601 values of a_4'/H. `cases` lists five sets of couplings with their legend labels.

```
fig, ax = plt.subplots()
for a1, a2, a3, label in cases:
    values = F_numeric(u, a1, a2, a3) * np.ones_like(u) / 2
    ax.plot(u, values, label=label)
for a2 in (0.005, 0.01, 0.02):  # the zeros of F for Gauss-Bonnet gravity
    root = np.sqrt((1 - 40 * a2) / (24 * a2))
    ax.plot([-root, root], [0, 0], "o", color="black", markersize=4)
ax.axhline(0.0, color="black", linewidth=0.8)
```

For each case $F/2$ is computed on the whole array and drawn. In Einstein gravity $F = 2$ does not depend on a_4', but the function still returns an array of 601 values, all equal to 2: the formula that

`sp.lambdify` turned into Python contains terms such as `720*ad1**4*alpha3`, and with `alpha3` equal to `0.0` such a term is the array `u**4` times zero. Multiplying by `np.ones_like(u)` (an array of ones of the same length) is therefore only a safeguard: a formula with no a_4' left in it (for example after a simplification) would make the function return the single number 2, and the factor would turn that into an array of the right length as well. The second loop marks the two zeros $\pm\sqrt{(1-40\alpha_2)/(24\alpha_2)}$ of each Gauss-Bonnet curve (Section 12.9) with black dots; `ax.axhli ne` draws the horizontal line $F = 0$.

```
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("$F(a_4')/2$")
ax.set_title("The factor $F$ of the evolution equation $a_4''F = \\kappa(p_3-p_t)$")
ax.legend(fontsize=8)
save_figure(fig, "evolution_factor",
            ...)
```

Labels, title, legend and `save_figure`. **What the figure shows.** In Einstein gravity $F/2 = 1$ everywhere. With a positive Gauss-Bonnet coupling F falls like a downward parabola and crosses zero at the black dots, sooner for larger α_2; beyond the dots the evolution equation changes sign, and at the dots it cannot be solved for a_4'' (Notebook 12c, section 12 of the notebook, integrates into such a point). The third-order coupling adds a positive $(a_4')^4$ term that lifts the curve again.

In [25], the conservation law of a general source.

```
source = {name: sp.Function(name)(x4, z)
          for name in ("rho", "p3", "pt", "p0", "q40", "q04")}  # f(x4, z)
Tgeneral = [[sp.Integer(0)] * N for _ in range(N)]
for i, name in enumerate(["p0", "p3", "p3", "p3", "rho", "pt", "pt", "pt"]):
    Tgeneral[i][i] = source[name]
Tgeneral[4][4] = -source["rho"]  # T^x4_x4 = -rho
Tgeneral[4][0], Tgeneral[0][4] = source["q40"], source["q04"]
divT = divergence(Tgeneral)
```

Now the six parts of the source are unknown functions of x_4 and z (so of x_0), made with `sp.Funct ion(name)(x4, z)`. The tensor is filled as in In [21] (the time entry is first set to ρ by the loop and then corrected to $-\rho$), and its divergence is computed with the function of In [19].

```
a4p = sp.Derivative(a4, x4)  # a4' as a derivative, for the comparison
expect_x4 = (-d(source["rho"], 4) - 3 * a4p * (source["p3"] - source["pt"])
             + d(source["q04"], 0) - 6 * H * sp.tan(z) * source["q04"])
expect_x0 = (d(source["p0"], 0) + d(source["q40"], 4)
             + 3 * H * sp.cot(z) * (2 * source["p0"] - source["p3"] - source["pt"]))
```

The two expected components of Section 12.10, written with the function `d` (so that $\partial_0 = 6H\,\partial_z$ is used consistently) and with a_4' as a derivative.

```
reproduces(clean(divT[4] - expect_x4) == 0, "nabla_mu T^mu_x4 as in the record",
           EMT, "divergence_x4_component")
reproduces(clean(divT[0] - expect_x0) == 0, "nabla_mu T^mu_x0 as in the record",
```

```
           EMT, "divergence_x0_component")
reproduces(all(divT[n] == 0 for n in (1, 2, 3, 5, 6, 7)),
           "nabla_mu T^mu_nu = 0 identically for the six other nu",
           PY, "conservation_components")
```

The x_4 and x_0 components equal the expected ones, and the six others vanish.

In [26], Einstein gravity: no vacuum and the null energy condition.

```
EINSTEIN = {alpha1: 1, alpha2: 0, alpha3: 0}
null_x0 = sp.expand((e00 - e44).subs(EINSTEIN))  # kappa (rho + p0)
report("Einstein: kappa (rho + p0)", null_x0)
reproduces(sp.expand(null_x0 + 6 * ad1 ** 2 + 6 * H ** 2) == 0,
           "Einstein: kappa (rho + p0) = -6 (a4'^2 + H^2) < 0",
           PY, "einstein_null_energy_x0")
```

From the constraint $\kappa\rho = -(e_{44} + \Lambda)$ and the hidden equation $\kappa p_0 = e_{00} + \Lambda$ follows $\kappa(\rho + p_0) = e_{00} - e_{44}$; with the Einstein couplings this is $-6H^2 - 6(a_4')^2$, the result of Section 12.11.

```
vacuum = [LHS[i][i].subs(EINSTEIN) for i in (1, 4, 5, 0)]  # T = 0
real_solutions = sp.solve(vacuum, [Lam, ad1, ad2], dict=True)  # ad1, ad2 real
say(f"real solutions of the vacuum equations: {real_solutions}")
```

With $T = 0$ the four left-hand sides must vanish. `sp.solve` looks for Λ, a_4', a_4''; because `ad1` and `ad2` were declared real, it returns only real solutions, and there are none (`[]`).

```
w1, w2 = sp.symbols("w1 w2")  # a4' and a4'' again, now allowed to be complex
as_complex = [value.subs({ad1: w1, ad2: w2}) for value in vacuum]
complex_solutions = sorted(sp.solve(as_complex, [Lam, w1, w2], dict=True), key=str)
say(f"solutions when a4' = w1 and a4'' = w2 may be complex: {complex_solutions}")
imaginary = all(not solution[w1].is_real for solution in complex_solutions)
reproduces(real_solutions == [] and len(complex_solutions) == 2 and imaginary,
           "Einstein: no real vacuum solution for H > 0 and any Lambda",
           LEAD, "no_vacuum_for_H_positive")
```

To see what the equations would want, a_4' and a_4'' are replaced by symbols without the property real. Now sympy finds exactly two solutions, $\Lambda = -18H^2$, $a_4' = \pm iH$, $a_4'' = 0$ (sorted by their printed text, so that the order is the same in every run). Both slopes are imaginary (`.is_real` is false), as derived in Section 12.11.

```
einstein_record = equations_json["einstein"]
mine_einstein = {"constraint_x4": e44.subs(EINSTEIN) + Lam + kappa * rho,
                 "hidden_x0": e00.subs(EINSTEIN) + Lam - kappa * p0,
                 "evolution": 2 * ad2 - kappa * (p3 - pt)}
same = all(sp.expand(parse_equation(einstein_record[key]["input"]) - value) == 0
           for key, value in mine_einstein.items())
reproduces(same, "the Einstein equations equal the record", PY, "json_einstein")
```

The Einstein constraint, hidden equation and evolution equation are compared with the record's.

In [27], figure 5: why there is no vacuum.

```
u = np.linspace(-3.0, 3.0, 301)  # a4'/H
lam_constraint = -(3 * u ** 2 + 21)  # Lambda/H^2 that the x4 equation needs
lam_hidden = 3 * u ** 2 - 15  # Lambda/H^2 that the x0 equation needs
fig, ax = plt.subplots()
ax.plot(u, lam_constraint, label="$\\Lambda$ needed by the constraint ($x_4$)")
ax.plot(u, lam_hidden, label="$\\Lambda$ needed by the hidden equation ($x_0$)")
ax.fill_between(u, lam_constraint, lam_hidden, color="grey", alpha=0.25,
                label="gap $6(a_4')^2+6H^2=-\\kappa(\\rho+p_0)$")
```

With $\rho = 0$ the constraint needs $\Lambda = -(3(a_4')^2 + 21H^2)$, and with $p_0 = 0$ the hidden equation needs $\Lambda = 3(a_4')^2 - 15H^2$. Both are drawn against a_4'/H; `ax.fill_between` shades the region between them in grey (25 per cent opaque).

```
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("$\\Lambda/H^2$")
ax.set_title("Einstein gravity: the vacuum equations have no common solution")
ax.legend(fontsize=8, loc="upper center")  # the empty area above the curves
save_figure(fig, "no_vacuum_gap",
            ...)
```

Labels, title, the legend placed in the empty upper middle, and `save_figure`. **What the figure shows.** The two curves never meet: the lower one is never above $-21H^2$, the upper one never below $-15H^2$. Their vertical distance, $6(a_4')^2 + 6H^2$, is at its smallest, $6H^2$, at $a_4' = 0$; it is exactly $-\kappa(\rho + p_0)$, the violation of the null energy condition.

In [28], the linear member.

```
reproduces(sp.expand((e11 - e00).subs(LINEAR)) == 0
           and sp.expand((e55 - e00).subs(LINEAR)) == 0,
           "linear member: p3 = pt = p0 = p (any alpha_k)",
           PY, "linear_member_equal_pressures")
```

With $a_4' = AH$, $a_4'' = 0$ (the dictionary `LINEAR` of In [18]) the 3-space, extra-time and hidden left-hand sides are equal, so the three pressures are equal, for every coupling.

```
rho_linear = sp.expand(-(e44 + Lam).subs(LINEAR) / kappa)  # from the constraint
p_linear = sp.expand((e00 + Lam).subs(LINEAR) / kappa)  # from the hidden equation
say(f"kappa rho = {sp.expand(kappa * rho_linear)}")
say(f"kappa p = {sp.expand(kappa * p_linear)}")
```

The required ρ and p follow from the constraint and the hidden equation; $\kappa\rho$ and κp are printed: the general formulas of Section 12.12.

```
check(not rho_linear.has(x4) and not p_linear.has(x4),
      "linear member: rho and p are constants")
```

```
check(sp.expand(rho_linear.subs(A, -A) - rho_linear) == 0
      and sp.expand(p_linear.subs(A, -A) - p_linear) == 0,
      "rho and p are even in A: A -> -A is a symmetry (deflation is a choice)")
```

Neither contains x_4, so both are constants; and replacing A by $-A$ changes neither: deflation is a choice of sign.

In [29], the vacuum factor.

```
V = sp.expand(sp.cancel((e44 - e00).subs(LINEAR) / (6 * (A ** 2 + 1) * H ** 2)))
report("vacuum factor V", V)
reproduces(V.is_polynomial(A) and sp.expand(6 * (A ** 2 + 1) * H ** 2 * V
                                            - (e44 - e00).subs(LINEAR)) == 0,
           "e44 - e00 = 6 (A^2 + 1) H^2 V with a polynomial V",
           PY, "linear_member_vacuum_factor")
```

$e_{44} - e_{00} = -\kappa(\rho + p)$ is divided by $6(A^2+1)H^2$; the quotient V is a polynomial in A (no remainder), printed in the RESULT line.

```
V_gauss_bonnet = sp.expand(V.subs({alpha1: 1, alpha3: 0}))
reproduces(sp.expand(V_gauss_bonnet - (1 - 8 * alpha2 * H ** 2 * (A ** 2 + 5))) == 0,
           "Gauss-Bonnet: V = 1 - 8 alpha2 H^2 (A^2 + 5)",
           PY, "einstein_gauss_bonnet_vacuum_linear")
V_any = V.subs(A, ad1 / H)  # the same polynomial with A H -> a4'
check(sp.expand((e00 - e44) + 6 * (ad1 ** 2 + H ** 2) * V_any) == 0,
      "for every a4: kappa (rho + p0) = -6 (a4'^2 + H^2) V(a4')")
```

For Einstein-Gauss-Bonnet gravity $V = 1 - 8\alpha_2 H^2(A^2 + 5)$. Because e_{44} and e_{00} contain no a_4'', the same factorisation holds for every history once AH is replaced by a_4': the last check (Section 12.11).

```
linear_record = equations_json["linearMember"]
same = (sp.expand(parse(linear_record["rho"]["input"], NAMES_RECORD) - rho_linear)
        == 0
        and sp.expand(parse(linear_record["p"]["input"], NAMES_RECORD) - p_linear)
        == 0
        and sp.expand(parse(linear_record["vacuumFactor"]["input"], NAMES_RECORD)
                      - V) == 0)
reproduces(same, "rho, p and V of the linear member equal the record",
           PY, "json_linear_member")
```

ρ, p and V are compared with the record's linear-member formulas.

In [30], the last check.

```
figure_names = ["12a_1_riemann_matrix.png", "12a_2_einstein_components.png",
                "12a_3_lovelock_heat_maps.png", "12a_4_evolution_factor.png",
                "12a_5_no_vacuum_gap.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all five figure files exist")
all_checks_passed()
```

The five figure files must exist, and the last line prints ALL 59 CHECKS PASSED (notebook 12a): 4 checks in In [2], 2 in In [4], 2 in In [5], 5 in In [8], 4 in In [12], 1 in In [14], 10 in In [16], 4 in In [17], 3 in In [18], 3 in In [19], 1 in In [21], 4 in In [22], 2 in In [23], 3 in In [25], 3 in In [26], 3 in In [28], 4 in In [29] and 1 in In [30].

12.17 Example: the source that the linear member requires

The second notebook takes the field equations from the record and asks, for the linear member $a_4 = AHx_4 + a_0$, what source they require, as a function of the slope A, of the cosmological constant and of the Lovelock couplings. It draws the signs of ρ and p, shows that a positive energy density in Einstein gravity comes only with $w < -1$, finds the Gauss-Bonnet and third-order vacua, and writes the two conditions that a homogeneous condensate of dirac16complex00 must meet to be the source. It runs in about 15 seconds, prints 24 PASS lines and draws eight figures.

12.18 How to run Notebook 12b

Step 1. What this notebook does and what it needs.

Notebook 12b (The source that the linear member a4 = A H x4 requires) is the file `Revision /textbook/notebooks/12b_required_source.ipynb` of the repository Dirac_claude. It reads the Lovelock components of the a4 field equations from the Revision record, derives the energy density and the pressure that the exponentially deflating linear member a4 = A H x4 + a0 requires for every slope A, every cosmological constant and every Lovelock coupling, checks the symmetry A to minus A, the null energy condition, the Einstein-Gauss-Bonnet and third-order vacua and the conditions on a homogeneous condensate of dirac16complex00, compares every result with the Revision records, and draws eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 12b_required_source.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12b_required_source.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 12b_required_source.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/12b.captions.json`
- `Revision/textbook/figures/12b_1_einstein_source.png`
- `Revision/textbook/figures/12b_2_source_regions.png`
- `Revision/textbook/figures/12b_3_phantom_w.png`
- `Revision/textbook/figures/12b_4_gauss_bonnet_source.png`
- `Revision/textbook/figures/12b_5_null_energy_map.png`
- `Revision/textbook/figures/12b_6_gauss_bonnet_vacuum.png`
- `Revision/textbook/figures/12b_7_third_order_vacua.png`

- `Revision/textbook/figures/12b_8_condensate_slope.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all eight figure files exist
ALL 24 CHECKS PASSED (notebook 12b)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/12b_required_source.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 12b_required_source.ipynb
python -m nbconvert --execute --inplace 12b_required_source.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a4-equations.json: the notebook reads the Revision record of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

12.19 Notebook 12b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/12b_required_source.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The author's metric contains one unknown function, $a_4(x_4)$. Its simplest
choice is the **linear member** $a_4 = AHx_4 + a_0$: 3-space grows like
$e^{AHx_4}$ and the three extra times shrink like $e^{-AHx_4}$ (exponential
deflation for $A > 0$). The field equations then say exactly what source (energy
density $\rho$ and pressure $p$) must be present. This notebook

- reads the Lovelock tensors of the field equations from the Revision record and
  derives from them the required $\kappa\rho$ and $\kappa p$ for every slope $A$,
  cosmological constant $\Lambda$ and Lovelock couplings $\alpha_1, \alpha_2,
  \alpha_3$, and compares them with the record;
- shows that the result does not change when $A$ is replaced by $-A$ (deflation is
  a choice of sign);
- maps, in Einstein gravity, where $\rho$ and $p$ are positive or negative, and
  shows that a positive energy density always comes with $w = p/\rho < -1$;
- shows what the Gauss-Bonnet coupling $\alpha_2$ and the third-order coupling
  $\alpha_3$ change, where the null energy condition holds, and where a vacuum
  (no source at all) exists;
- writes the conditions that a homogeneous condensate of the commuting field
  dirac16complex00 must meet to be the source;
- draws eight teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 12b (The source that the linear member a4 = A H x4 requires) is the file
‘Revision/textbook/notebooks/12b_required_source.ipynb‘ of the repository Dirac_claude.
It reads the Lovelock components of the a4 field equations from the Revision record,
derives the energy density and the pressure that the exponentially deflating linear
member a4 = A H x4 + a0 requires for every slope A, every cosmological constant and every
Lovelock coupling, checks the symmetry A to minus A, the null energy condition, the
Einstein-Gauss-Bonnet and third-order vacua and the conditions on a homogeneous
condensate of dirac16complex00, compares every result with the Revision records, and
draws eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
notebooks were built with Python 3.14.5) with these packages at exactly these versions:
numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv

Then close the terminal and open a NEW one (a terminal opened before the installation
does not know the new programs), and check the version of Python; it must print 3.12 or a
higher number:

Windows (PowerShell):
```

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

Linux (bash):

    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 12b_required_source.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 15 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.

**Step 6. Or run the notebook without a browser (headless).**
```

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12b_required_source.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 12b_required_source.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/12b.captions.json‘
- ‘Revision/textbook/figures/12b_1_einstein_source.png‘
- ‘Revision/textbook/figures/12b_2_source_regions.png‘
- ‘Revision/textbook/figures/12b_3_phantom_w.png‘
- ‘Revision/textbook/figures/12b_4_gauss_bonnet_source.png‘
- ‘Revision/textbook/figures/12b_5_null_energy_map.png‘
- ‘Revision/textbook/figures/12b_6_gauss_bonnet_vacuum.png‘
- ‘Revision/textbook/figures/12b_7_third_order_vacua.png‘
- ‘Revision/textbook/figures/12b_8_condensate_slope.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all eight figure files exist
ALL 24 CHECKS PASSED (notebook 12b)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start

JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/12b_required_source.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 12b_required_source.ipynb
python -m nbconvert --execute --inplace 12b_required_source.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a4-equations.json: the notebook reads the Revision record of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 12b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 12b (The source that the linear member a4 = A H x4 requires) is the file
# Revision/textbook/notebooks/12b_required_source.ipynb of the repository Dirac_claude.
# It reads the Lovelock components of the a4 field equations from the Revision record,
```

```
# derives the energy density and the pressure that the exponentially deflating linear
# member a4 = A H x4 + a0 requires for every slope A, every cosmological constant and
# every Lovelock coupling, checks the symmetry A to minus A, the null energy condition,
# the Einstein-Gauss-Bonnet and third-order vacua and the conditions on a homogeneous
# condensate of dirac16complex00, compares every result with the Revision records, and
# draws eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
```

```
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
```

```
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 12b_required_source.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 12b_required_source.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 12b_required_source.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/12b.captions.json
```

```
# - Revision/textbook/figures/12b_1_einstein_source.png
# - Revision/textbook/figures/12b_2_source_regions.png
# - Revision/textbook/figures/12b_3_phantom_w.png
# - Revision/textbook/figures/12b_4_gauss_bonnet_source.png
# - Revision/textbook/figures/12b_5_null_energy_map.png
# - Revision/textbook/figures/12b_6_gauss_bonnet_vacuum.png
# - Revision/textbook/figures/12b_7_third_order_vacua.png
# - Revision/textbook/figures/12b_8_condensate_slope.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all eight figure files exist
#     ALL 24 CHECKS PASSED (notebook 12b)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/12b_required_source.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 12b_required_source.ipynb
#     python -m nbconvert --execute --inplace 12b_required_source.ipynb
```

```
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a4-equations.json: the notebook reads the Revision record
#   of the repository. Run it inside the folder Revision/textbook/notebooks of a complete
#   copy of the repository made with git clone, not on a copy of the notebook file alone.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "12b"  # this notebook: chapter 12, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 12b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Linear member**: the choice $a_4 = AHx_4 + a_0$ with constant **slope** $A$, so
  that $a_4' = AH$ and $a_4'' = 0$. $A > 0$: 3-space inflates and the extra times
  $x_5, x_6, x_7$ deflate exponentially; $A < 0$: the opposite; $A = 0$: static.
- **Source**: the matter whose energy-momentum tensor stands on the right-hand side
  of the field equations: energy density $\rho$, pressures $p_3$ (3-space), $p_t$
  (extra times), $p_0$ (hidden direction $x_0$).
- **Required source**: the $\rho$ and $p$ that the field equations demand for a
  given $a_4$; we read them off the equations instead of assuming a kind of matter.
- **Couplings**: $\alpha_1$ (Einstein), $\alpha_2$ (Gauss-Bonnet), $\alpha_3$ (third
  order); **Einstein gravity** is $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$;
  $\Lambda$ is the cosmological constant, $\kappa > 0$ the strength of gravity.
- **Units**: the plots use $H = 1$ and $\kappa = 1$, so $\kappa\rho$, $\kappa p$
  and $\Lambda$ are in units of $H^2$; $\alpha_2 H^2$ and $\alpha_3 H^4$ are pure
  numbers.
- **Equation of state** $w = p/\rho$.
- **Null energy condition** (NEC) along $x_0$: $\rho + p_0 \ge 0$. Ordinary matter
  satisfies it; a source with $w < -1$ and $\rho > 0$ (called *phantom*) violates
```

```
  it.
- **Vacuum**: $\rho = p = 0$, no source at all.
- **Condensate**: a configuration of the field dirac16complex00 that depends only on
  the time $x_4$; $S = \bar\Phi\Phi$ is its scalar density, $m$ the mass, and
  $U = \tfrac{\lambda}{2}S^2$ the self-interaction with strength $\lambda$.
- **Status labels**: PROVED (an exact identity, checked here and in the record),
  ASSUMED (a choice made, not derived), OPEN (not established).
```

Text cell:

```
## 4. The physical and mathematical situation

For a source that does not depend on $x_0$ and has no mixed components, the
Einstein-Lovelock field equations of the author's metric reduce to (the Revision
record `a4-equations.json`):

$$\sum_k \alpha_k E_{(k)}{}^{x_4}{}_{x_4} + \Lambda = -\kappa\rho,\qquad
\sum_k \alpha_k E_{(k)}{}^{x_0}{}_{x_0} + \Lambda = \kappa p_0,$$
$$\sum_k \alpha_k E_{(k)}{}^{x_1}{}_{x_1} + \Lambda = \kappa p_3,\qquad
\sum_k \alpha_k E_{(k)}{}^{x_5}{}_{x_5} + \Lambda = \kappa p_t.$$

For the linear member every component depends only on $A$, $H$ and the couplings,
so the required $\rho$ and $p$ are constants. The equations allow ANY $a_4$; what
they fix is the source that this $a_4$ needs. Whether a real field supplies that
source is a separate question (section 9 of this notebook states what is known for
one case).
```

Text cell:

```
## 5. The Revision records this notebook reproduces

The next cell defines the helpers that read the Revision records: `read_json`
reads a JSON file of the repository, `record_verdict` finds a check by its name in
a report, and `reproduces` is a check that passes only when this notebook's own
result holds AND the record lists the named check with the verdict PASS; it
collects its two printed lines (PASS and reproduces) in a text buffer and prints
them with one call, so that they always stay together in the output.
```

In [2]:

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory

PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
LEAD = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"  # lead check
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the equations


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
```

```
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict of the check name in a report ("PASS"), None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS.  Its
    printed lines (PASS and reproduces) are collected in a text buffer and printed
    by one print call, so that they always stay together in the cell's output."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name, record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # all lines at once


record = read_json(EQUATIONS)
title = record["title"]  # the title stored in the record
say(f"read {EQUATIONS}: {title}")
```

Out [2]:

```
read Revision/field_equations_a4/a4-equations.json: Einstein-Lovelock field equations for
    a4[x4] and b4[x4] in the primordial metric (Revision/SPEC.md section 5); the keys
    lovelockTensors .. fields are the restriction b4 = 1 (the metric before 2026-10-14),
    the key b4 is the general case
```

Text cell:

```
## 6. The required source of the linear member

The record stores each Lovelock component as a text in the Wolfram Language
(‘ad1‘ stands for $a_4’$, ‘ad2‘ for $a_4’’$, ‘^‘ for a power). The next cell turns
the four distinct components of $E_{(1)}, E_{(2)}, E_{(3)}$ into sympy expressions,
forms the four left-hand sides, and substitutes the linear member
$a_4’ = AH$, $a_4’’ = 0$. Then it reads off

$$\kappa\rho = -\Big(\sum_k \alpha_k E_{(k)}{}^{x_4}{}_{x_4} + \Lambda\Big),\qquad
\kappa p_0 = \sum_k \alpha_k E_{(k)}{}^{x_0}{}_{x_0} + \Lambda,$$

and the same for $p_3$ and $p_t$.
```

In [3]:

```
import numpy as np  # arrays of numbers for the plots
import sympy as sp  # exact algebra with symbols
```

```
H = sp.symbols("H", positive=True)  # the constant H of the metric
A = sp.symbols("A", real=True)  # the slope of the linear member
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
alpha1, alpha2, alpha3 = sp.symbols("alpha1 alpha2 alpha3", real=True)
Lam, kappa = sp.symbols("Lam kappa", real=True)  # Lambda and kappa
NAMES = {"ad1": ad1, "ad2": ad2, "H": H, "AA": A, "alpha1": alpha1,
         "alpha2": alpha2, "alpha3": alpha3, "Lam": Lam, "kappa": kappa}


def parse(text):
    """A Wolfram InputForm text of the record as a sympy expression."""
    return sp.sympify(text.replace("^", "**"), locals=NAMES)


ALPHA = {1: alpha1, 2: alpha2, 3: alpha3}
lovelock = record["lovelockTensors"]
side = {key: sum(ALPHA[k] * parse(lovelock[f"E{k}"][key]["input"]) for k in (1, 2, 3))
        for key in ("x1x1", "x4x4", "x5x5", "x0x0")}  # sum_k alpha_k E_(k)
LINEAR = {ad1: A * H, ad2: 0}  # a4' = A H, a4'' = 0
kappa_rho = sp.expand(-(side["x4x4"] + Lam).subs(LINEAR))
kappa_p3 = sp.expand((side["x1x1"] + Lam).subs(LINEAR))
kappa_pt = sp.expand((side["x5x5"] + Lam).subs(LINEAR))
kappa_p0 = sp.expand((side["x0x0"] + Lam).subs(LINEAR))
say(f"kappa rho = {kappa_rho}")
say(f"kappa p0 = {kappa_p0}")
```

Out [3]:

```
kappa rho = -360*A**6*H**6*alpha3 - 648*A**4*H**6*alpha3 + 36*A**4*H**4*alpha2 -
    1080*A**2*H**6*alpha3 + 120*A**2*H**4*alpha2 - 3*A**2*H**2*alpha1 - 2520*H**6*alpha3
    + 420*H**4*alpha2 - 21*H**2*alpha1 - Lam
kappa p0 = -72*A**6*H**6*alpha3 - 648*A**4*H**6*alpha3 + 12*A**4*H**4*alpha2 -
    1944*A**2*H**6*alpha3 + 168*A**2*H**4*alpha2 - 3*A**2*H**2*alpha1 + 360*H**6*alpha3 -
    180*H**4*alpha2 + 15*H**2*alpha1 + Lam
```

Text cell:

```
The next cell checks four things. (1) The three pressures are equal,
$p_3 = p_t = p_0 =: p$ (for $a_4'' = 0$ the components $E^{x_1}{}_{x_1}$,
$E^{x_5}{}_{x_5}$, $E^{x_0}{}_{x_0}$ coincide). (2) $\kappa\rho$ and $\kappa p$ equal
the linear-member formulas stored in the record. (3) Both contain $A$ only through
$A^2$: replacing $A$ by $-A$ changes nothing, so the equations accept deflating
extra times ($A > 0$) and inflating ones ($A < 0$) on the same footing. (4) The
Einstein part ($\alpha_1$) of $\kappa\rho$ is $-3H^2(A^2 + 7)$, as the lead's
independent check found.
```

In [4]:

```
reproduces(sp.expand(kappa_p3 - kappa_p0) == 0 and sp.expand(kappa_pt - kappa_p0) == 0,
           "linear member: p3 = pt = p0 = p", PY, "linear_member_equal_pressures")
```

```
kappa_p = kappa_p0  # the common pressure, times kappa
linear_record = record["linearMember"]
same = (sp.expand(kappa * parse(linear_record["rho"]["input"]) - kappa_rho) == 0
        and sp.expand(kappa * parse(linear_record["p"]["input"]) - kappa_p) == 0)
reproduces(same, "kappa rho and kappa p equal the record", PY, "json_linear_member")
check(sp.expand(kappa_rho.subs(A, -A) - kappa_rho) == 0
      and sp.expand(kappa_p.subs(A, -A) - kappa_p) == 0,
      "A -> -A leaves rho and p unchanged: deflation is a choice of sign")
reproduces(sp.expand(sp.diff(kappa_rho, alpha1) + 3 * H ** 2 * (A ** 2 + 7)) == 0,
           "the alpha1 part of kappa rho is -3 H^2 (A^2 + 7)",
           LEAD, "einstein_linear_rho_alpha1")
```

Out [4]:

```
PASS linear member: p3 = pt = p0 = p
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    linear_member_equal_pressures
PASS kappa rho and kappa p equal the record
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_linear_member
PASS A -> -A leaves rho and p unchanged: deflation is a choice of sign
PASS the alpha1 part of kappa rho is -3 H^2 (A^2 + 7)
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_linear_rho_alpha1
```

Text cell:

```
## 7. Einstein gravity

With $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$:

$$\kappa\rho = -(21 + 3A^2)H^2 - \Lambda,\qquad \kappa p = (15 - 3A^2)H^2 + \Lambda,
\qquad \kappa(\rho + p) = -6(1 + A^2)H^2.$$

The last line is the sum of the first two; it does not contain $\Lambda$ and is
negative for every $A$. The next cell checks these formulas against the record and
prints the numbers for $\Lambda = 0$ and $A = 1$ (units $H = \kappa = 1$):
$\kappa\rho = -24$, $\kappa p = 12$, $w = p/\rho = -1/2$.
```

In [5]:

```
EINSTEIN = {alpha1: 1, alpha2: 0, alpha3: 0}
rho_e = sp.expand(kappa_rho.subs(EINSTEIN))  # kappa rho in Einstein gravity
p_e = sp.expand(kappa_p.subs(EINSTEIN))  # kappa p in Einstein gravity
report("Einstein: kappa rho", rho_e)
report("Einstein: kappa p", p_e)
same = (sp.expand(kappa * parse(linear_record["rhoEinstein"]["input"]) - rho_e) == 0
        and sp.expand(kappa * parse(linear_record["pEinstein"]["input"]) - p_e) == 0
        and sp.expand(kappa * parse(linear_record["rhoPlusPEinstein"]["input"])
                      - (rho_e + p_e)) == 0)
reproduces(same and sp.expand(rho_e + p_e + 6 * (1 + A ** 2) * H ** 2) == 0,
```

```
           "Einstein: kappa (rho + p) = -6 (1 + A^2) H^2 < 0", PY,
           "json_linear_member")
UNITS = {H: 1, Lam: 0, A: 1}
rho_1, p_1 = rho_e.subs(UNITS), p_e.subs(UNITS)
report("Einstein, Lambda = 0, A = 1: kappa rho", rho_1, "H^2")
report("Einstein, Lambda = 0, A = 1: kappa p", p_1, "H^2")
report("Einstein, Lambda = 0, A = 1: w = p/rho", sp.Rational(p_1, rho_1))
check(rho_1 == -24 and p_1 == 12, "A = 1, Lambda = 0: kappa rho = -24, kappa p = 12")
```

Out [5]:

```
RESULT Einstein: kappa rho = -3*A**2*H**2 - 21*H**2 - Lam
RESULT Einstein: kappa p = -3*A**2*H**2 + 15*H**2 + Lam
PASS Einstein: kappa (rho + p) = -6 (1 + A^2) H^2 < 0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    json_linear_member
RESULT Einstein, Lambda = 0, A = 1: kappa rho = -24 H^2
RESULT Einstein, Lambda = 0, A = 1: kappa p = 12 H^2
RESULT Einstein, Lambda = 0, A = 1: w = p/rho = -1/2
PASS A = 1, Lambda = 0: kappa rho = -24, kappa p = 12
```

Text cell:

```
The next cell draws $\kappa\rho$ and $\kappa p$ of Einstein gravity with
$\Lambda = 0$ as functions of the slope $A$ from $-3$ to $3$. The right half
($A > 0$, deflating extra times) is the mirror image of the left half ($A < 0$,
inflating extra times). With $\Lambda = 0$ the energy density is negative for every
$A$: it is never above $-21H^2/\kappa$.
```

In [6]:

```
rho_f = sp.lambdify((A, Lam), rho_e.subs(H, 1), "numpy")  # numbers from formulas
p_f = sp.lambdify((A, Lam), p_e.subs(H, 1), "numpy")
a_values = np.linspace(-3.0, 3.0, 301)
fig, ax = plt.subplots()
ax.axvspan(0, 3, color="tab:green", alpha=0.08)  # the deflating half, A > 0
ax.plot(a_values, rho_f(a_values, 0.0), label="$\\kappa\\rho/H^2$")
ax.plot(a_values, p_f(a_values, 0.0), label="$\\kappa p/H^2$")
ax.plot(a_values, rho_f(a_values, 0.0) + p_f(a_values, 0.0), ":", color="black",
        label="$\\kappa(\\rho+p)/H^2=-6(1+A^2)$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.text(1.5, 15, "$A>0$: extra times deflate", ha="center", fontsize=9)
ax.text(-1.5, 15, "$A<0$: extra times inflate", ha="center", fontsize=9)
ax.set_xlabel("slope $A$ of $a_4 = AHx_4 + a_0$")
ax.set_ylabel("required source (units of $H^2/\\kappa$)")
ax.set_title("Einstein gravity, $\\Lambda = 0$: the source of the linear member")
ax.legend(fontsize=8, loc="lower center")
save_figure(fig, "einstein_source",
            "The source that the linear member $a_4 = AHx_4 + a_0$ requires in "
            "Einstein gravity with $\\Lambda = 0$, as a function of the slope $A$: "
            "$\\kappa\\rho = -(21 + 3A^2)H^2$, $\\kappa p = (15 - 3A^2)H^2$ and their "
```

```
"sum $-6(1 + A^2)H^2$ (dotted), in units of $H^2$. The shaded right half "
"is the deflating branch $A > 0$; every curve is symmetric under "
"$A \\to -A$, so the equations do not prefer deflation. The energy "
"density is negative for every $A$.")
```

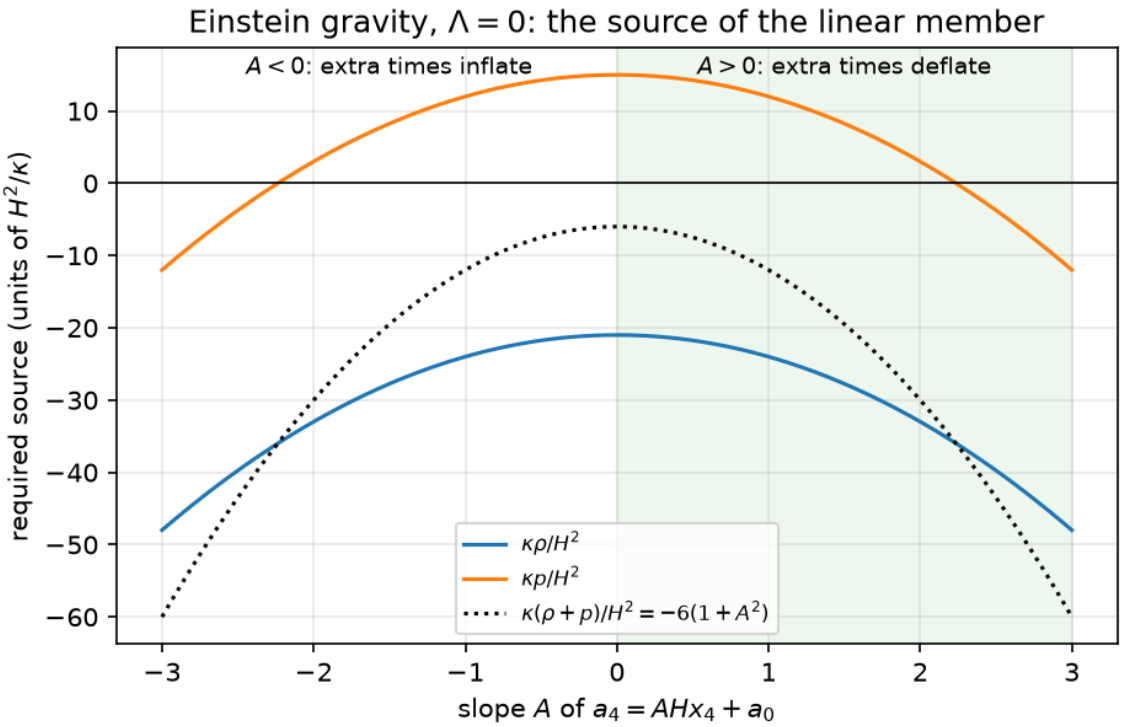

Figure 324: The source that the linear member $a_4 = AHx_4 + a_0$ requires in Einstein gravity with $\Lambda = 0$, as a function of the slope A: $\kappa\rho = -(21 + 3A^2)H^2$, $\kappa p = (15 - 3A^2)H^2$ and their sum $-6(1 + A^2)H^2$ (dotted), in units of H^2. The shaded right half is the deflating branch $A > 0$; every curve is symmetric under $A \to -A$, so the equations do not prefer deflation. The energy density is negative for every A. (Notebook 12b, figure 1.)

Out [6]:

```
Figure 12b.1 saved as Revision/textbook/figures/12b_1_einstein_source.png
```

Text cell:

```
**Where can the energy density be positive?** A cosmological constant shifts
$\kappa\rho$ by $-\Lambda$ and $\kappa p$ by $+\Lambda$. The energy density is
positive when $\Lambda < -(21 + 3A^2)H^2$, and the pressure is positive when
$\Lambda > (3A^2 - 15)H^2$. The two boundary curves never meet (the lower one is
never above $-21H^2$, the upper one never below $-15H^2$), so $\rho$ and $p$ are
never both positive and never both zero. The next cell colours the plane of $A$
and $\Lambda/H^2$ by the signs of $\rho$ and $p$.
```

In [7]:

```
a_grid, lam_grid = np.meshgrid(np.linspace(-3, 3, 241), np.linspace(-60, 30, 241))
rho_grid = rho_f(a_grid, lam_grid)
p_grid = p_f(a_grid, lam_grid)
region = np.where(rho_grid > 0, 0, np.where(p_grid < 0, 1, 2))  # three regions
fig, ax = plt.subplots()
colours = matplotlib.colors.ListedColormap(["#f4a582", "#d1e5f0", "#92c5de"])
ax.pcolormesh(a_grid, lam_grid, region, cmap=colours, shading="auto")
ax.plot(a_values, -(21 + 3 * a_values ** 2), color="black",
        label="$\\rho = 0$: $\\Lambda = -(21+3A^2)H^2$")
ax.plot(a_values, 3 * a_values ** 2 - 15, "--", color="black",
        label="$p = 0$: $\\Lambda = (3A^2-15)H^2$")
ax.text(0, -50, "$\\rho > 0$, $p < 0$: $w < -1$", ha="center")
ax.text(0, -18, "$\\rho < 0$, $p < 0$", ha="center")
ax.text(0, 5, "$\\rho < 0$, $p > 0$", ha="center")
ax.set_xlim(-3, 3)
ax.set_ylim(-60, 30)
ax.set_xlabel("slope $A$")
ax.set_ylabel("$\\Lambda/H^2$")
ax.set_title("Einstein gravity: the signs of the required $\\rho$ and $p$")
ax.legend(fontsize=8, loc="upper center")
save_figure(fig, "source_regions",
            "Einstein gravity: the plane of the slope $A$ (horizontal) and the "
            "cosmological constant $\\Lambda/H^2$ (vertical), coloured by the signs "
            "of the energy density $\\rho$ and the pressure $p$ that the linear "
            "member requires. Below the solid curve $\\Lambda = -(21+3A^2)H^2$ the "
            "energy density is positive and the pressure negative, with $w < -1$; "
            "above the dashed curve $\\Lambda = (3A^2-15)H^2$ the pressure is "
            "positive and the energy density negative; in between both are "
            "negative. No point has both positive, and no point is a vacuum.")
```

Out [7]:

```
Figure 12b.2 saved as Revision/textbook/figures/12b_2_source_regions.png
```

Text cell:

```
**A positive energy density forces $w < -1$.** Because
$\kappa(\rho + p) = -6(1 + A^2)H^2 < 0$, we have $p < -\rho$; dividing by
$\rho > 0$ gives $w = p/\rho < -1$. A source with $\rho > 0$ and $w < -1$ violates
the null energy condition (it is *phantom*). The next cell checks the algebra and
draws $w$ for three values of $\Lambda$, only where $\rho > 0$.
```

In [8]:

```
w_e = sp.simplify(p_e / rho_e)  # w = p/rho in Einstein gravity
say(f"Einstein: w = p/rho = {w_e}")
check(sp.simplify(w_e + 1 - (rho_e + p_e) / rho_e) == 0,
      "w + 1 = (rho + p)/rho, so rho > 0 and rho + p < 0 give w < -1")
fig, ax = plt.subplots()
for lam in (-30.0, -40.0, -60.0):
    rho_values = rho_f(a_values, lam)
```

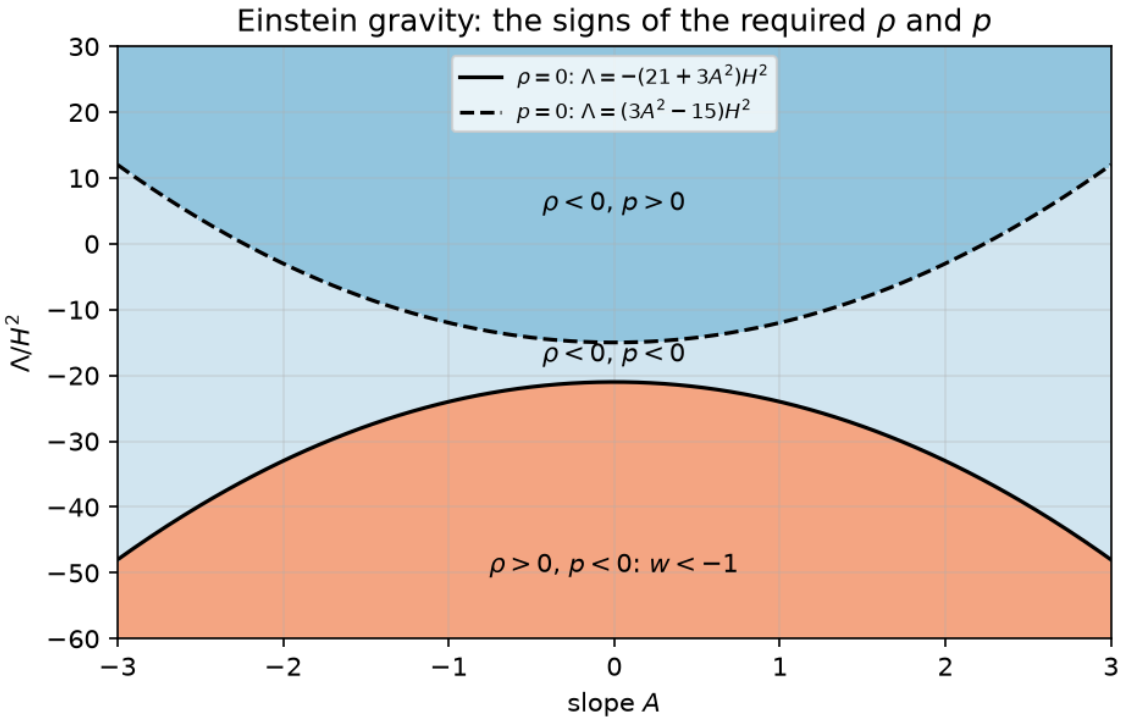

Figure 325: Einstein gravity: the plane of the slope A (horizontal) and the cosmological constant Λ/H^2 (vertical), coloured by the signs of the energy density ρ and the pressure p that the linear member requires. Below the solid curve $\Lambda = -(21 + 3A^2)H^2$ the energy density is positive and the pressure negative, with $w < -1$; above the dashed curve $\Lambda = (3A^2 - 15)H^2$ the pressure is positive and the energy density negative; in between both are negative. No point has both positive, and no point is a vacuum. (Notebook 12b, figure 2.)

```
    keep = rho_values > 1.0  # where the energy density is clearly positive
    w_values = np.where(keep, p_f(a_values, lam) / np.where(keep, rho_values, 1.0),
                        np.nan)  # nan: not drawn
    ax.plot(a_values, w_values, label=f"$\\Lambda = {lam:.0f}\\,H^2$")
    check(np.nanmax(w_values) < -1.0, f"Lambda = {lam:.0f} H^2: w < -1 where rho > 0")
ax.axhline(-1.0, color="black", linewidth=0.8)
ax.set_ylim(-6, 0)
ax.set_xlabel("slope $A$")
ax.set_ylabel("$w = p/\\rho$")
ax.set_title("Einstein gravity: where $\\rho > 0$ the source has $w < -1$")
ax.legend(fontsize=8)
save_figure(fig, "phantom_w",
            "Einstein gravity: the equation of state $w = p/\\rho$ that the linear "
            "member requires, drawn only where the energy density is positive "
            "($\\kappa\\rho > H^2$), for $\\Lambda = -3OH^2$, $-40H^2$ and $-60H^2$, "
            "as a function of the slope $A$. Every curve lies below the line "
            "$w = -1$: a positive energy density comes with $w < -1$, because "
            "$\\kappa(\\rho + p) = -6(1 + A^2)H^2$ is negative for every $A$.")
```

Out [8]:

```
Einstein: w = p/rho = (3*A**2*H**2 - 15*H**2 - Lam)/(3*A**2*H**2 + 21*H**2 + Lam)
PASS w + 1 = (rho + p)/rho, so rho > 0 and rho + p < 0 give w < -1
PASS Lambda = -30 H^2: w < -1 where rho > 0
PASS Lambda = -40 H^2: w < -1 where rho > 0
PASS Lambda = -60 H^2: w < -1 where rho > 0
```

Out [8]:

```
Figure 12b.3 saved as Revision/textbook/figures/12b_3_phantom_w.png
```

Text cell:

```
## 8. The Gauss-Bonnet and the third-order couplings

The coupling $\alpha_2$ multiplies the Gauss-Bonnet tensor $E_{(2)}$. The next
cell extracts the $\alpha_2$ parts of $\kappa\rho$ and $\kappa p$ (the derivatives
with respect to $\alpha_2$) and compares them with the lead's independent
computation from the classical Gauss-Bonnet formula:
$H^4(36A^4 + 120A^2 + 420)$ and $12H^4(A^4 + 14A^2 - 15)$. Then it draws
$\kappa\rho$ and $\kappa p$ for Einstein-Gauss-Bonnet gravity
($\alpha_1 = 1$, $\alpha_3 = 0$, $\Lambda = 0$) for four values of $\alpha_2H^2$,
and checks two facts that the plot shows: at $A = 0$ the energy density is
$\kappa\rho = -21H^2 + 420\alpha_2H^4$ (positive only for $\alpha_2H^2 > 1/20$),
and at $A = \pm 1$ the $\alpha_2$ part of the pressure,
$12H^4(1 + 14 - 15) = 0$, vanishes, so $\kappa p = (15 - 3)H^2 = 12H^2$ for every
$\alpha_2$.
```

In [9]:

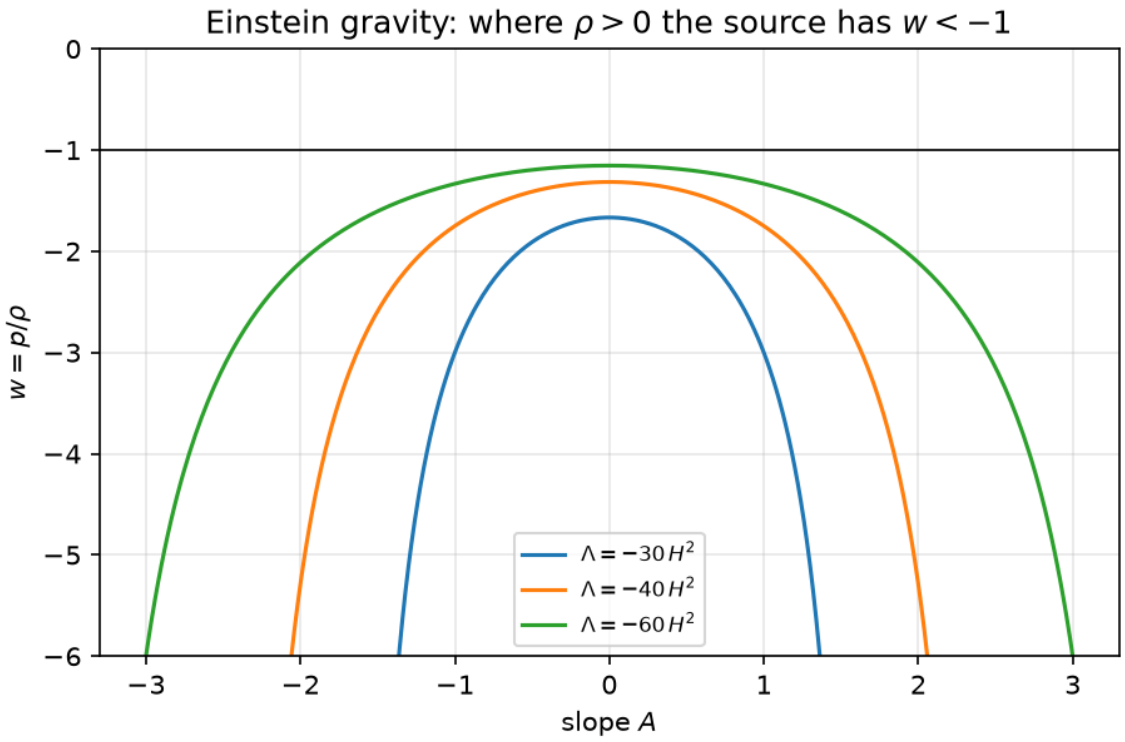

Figure 326: Einstein gravity: the equation of state $w = p/\rho$ that the linear member requires, drawn only where the energy density is positive ($\kappa\rho > H^2$), for $\Lambda = -30H^2$, $-40H^2$ and $-60H^2$, as a function of the slope A. Every curve lies below the line $w = -1$: a positive energy density comes with $w < -1$, because $\kappa(\rho + p) = -6(1 + A^2)H^2$ is negative for every A. (Notebook 12b, figure 3.)

```
reproduces(sp.expand(sp.diff(kappa_rho, alpha2)
                     - H ** 4 * (36 * A ** 4 + 120 * A ** 2 + 420)) == 0,
           "the alpha2 part of kappa rho is H^4 (36 A^4 + 120 A^2 + 420)",
           LEAD, "gauss_bonnet_rho_alpha2")
reproduces(sp.expand(sp.diff(kappa_p, alpha2)
                     - 12 * H ** 4 * (A ** 4 + 14 * A ** 2 - 15)) == 0,
           "the alpha2 part of kappa p is 12 H^4 (A^4 + 14 A^2 - 15)",
           LEAD, "gauss_bonnet_p_alpha2")
symbols_numeric = (A, Lam, alpha1, alpha2, alpha3)
rho_l = sp.lambdify(symbols_numeric, kappa_rho.subs(H, 1), "numpy")  # any coupling
p_l = sp.lambdify(symbols_numeric, kappa_p.subs(H, 1), "numpy")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
for a2 in (0.0, 0.01, 0.025, 0.04):
    label = f"$\\alpha_2H^2 = {a2}$"
    left.plot(a_values, rho_l(a_values, 0, 1, a2, 0), label=label)
    right.plot(a_values, p_l(a_values, 0, 1, a2, 0), label=label)
for ax, name in ((left, "\\kappa\\rho/H^2"), (right, "\\kappa p/H^2")):
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xlabel("slope $A$")
    ax.set_ylabel(f"${name}$")
    ax.legend(fontsize=8)
left.set_title("energy density, $\\Lambda = 0$")
right.set_title("pressure, $\\Lambda = 0$")
save_figure(fig, "gauss_bonnet_source",
            "Einstein-Gauss-Bonnet gravity ($\\alpha_1 = 1$, $\\alpha_3 = 0$, "
            "$\\Lambda = 0$): the energy density $\\kappa\\rho$ (left) and the pressure "
            "$\\kappa p$ (right) that the linear member requires, in units of $H^2$, "
            "as functions of the slope $A$, for $\\alpha_2 H^2 = 0$ (Einstein), "
            "$0.01$, $0.025$ and $0.04$. The Gauss-Bonnet term adds "
            "$\\alpha_2 H^4(36A^4 + 120A^2 + 420)$ to $\\kappa\\rho$: for "
            "$\\alpha_2 H^2 = 0.025$ and $0.04$ the energy density is positive at "
            "large $|A|$ without any cosmological constant, while at $A = 0$ it is "
            "$(420\\alpha_2H^2 - 21)H^2$, positive only for $\\alpha_2 H^2 > 1/20$. "
            "All pressure curves meet at $A = \\pm 1$, where $\\kappa p = 12H^2$ for "
            "every $\\alpha_2$; every curve is symmetric under $A \\to -A$.")
rho_gb_0 = sp.expand(kappa_rho.subs({alpha1: 1, alpha3: 0, Lam: 0, A: 0}))
check(sp.expand(rho_gb_0 - (420 * alpha2 * H ** 4 - 21 * H ** 2)) == 0,
      "Gauss-Bonnet, Lambda = 0, A = 0: kappa rho = (420 alpha2 H^2 - 21) H^2")
p_gb_1 = [sp.expand(kappa_p.subs({alpha1: 1, alpha3: 0, Lam: 0, A: s}))
          for s in (1, -1)]
check(p_gb_1 == [12 * H ** 2, 12 * H ** 2],
      "Gauss-Bonnet, Lambda = 0, A = +1 or -1: kappa p = 12 H^2 for every alpha2")
```

Out [9]:

```
PASS the alpha2 part of kappa rho is H^4 (36 A^4 + 120 A^2 + 420)
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    gauss_bonnet_rho_alpha2
PASS the alpha2 part of kappa p is 12 H^4 (A^4 + 14 A^2 - 15)
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    gauss_bonnet_p_alpha2
```

Out [9]:

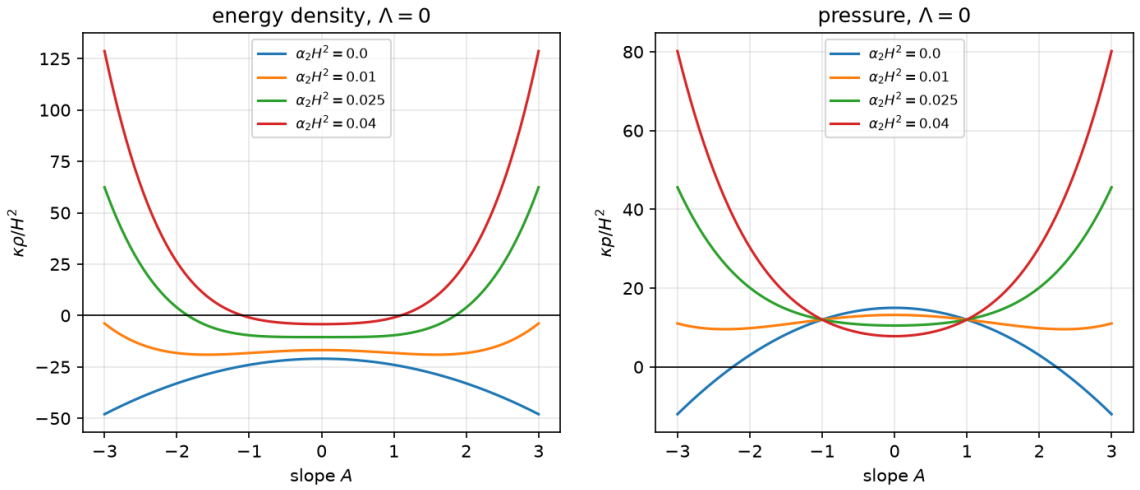

Figure 327: Einstein-Gauss-Bonnet gravity ($\alpha_1 = 1$, $\alpha_3 = 0$, $\Lambda = 0$): the energy density $\kappa\rho$ (left) and the pressure κp (right) that the linear member requires, in units of H^2, as functions of the slope A, for $\alpha_2 H^2 = 0$ (Einstein), 0.01, 0.025 and 0.04. The Gauss-Bonnet term adds $\alpha_2 H^4(36A^4 + 120A^2 + 420)$ to $\kappa\rho$: for $\alpha_2 H^2 = 0.025$ and 0.04 the energy density is positive at large $|A|$ without any cosmological constant, while at $A = 0$ it is $(420\alpha_2 H^2 - 21)H^2$, positive only for $\alpha_2 H^2 > 1/20$. All pressure curves meet at $A = \pm 1$, where $\kappa p = 12H^2$ for every α_2; every curve is symmetric under $A \to -A$. (Notebook 12b, figure 4.)

```
Figure 12b.4 saved as Revision/textbook/figures/12b_4_gauss_bonnet_source.png
PASS Gauss-Bonnet, Lambda = 0, A = 0: kappa rho = (420 alpha2 H^2 - 21) H^2
PASS Gauss-Bonnet, Lambda = 0, A = +1 or -1: kappa p = 12 H^2 for every alpha2
```

Text cell:

```
**The null energy condition and the vacuum.** For every coupling the record
factorises

$$\kappa(\rho + p) = -6(A^2 + 1)H^2\,V,\qquad
V = \alpha_1 - 8\alpha_2H^2(A^2 + 5) + \alpha_3H^4(72A^4 + 144A^2 + 360).$$

So the null energy condition $\rho + p \ge 0$ holds exactly where $V \le 0$, and a
vacuum ($\rho = p = 0$) needs $V = 0$ (and then $\Lambda$ chosen so that
$\rho = 0$). The next cell checks the factorisation against the record and draws
the sign of $V$ in the plane of $A$ and $\alpha_2H^2$ (with $\alpha_1 = 1$,
$\alpha_3 = 0$). The boundary $V = 0$ is the curve
$A^2 = (1 - 40\alpha_2H^2)/(8\alpha_2H^2)$, which has real points only for
$0 < \alpha_2H^2 \le 1/40$.
```

In [10]:

```
V = parse(linear_record["vacuumFactor"]["input"])  # the record vacuum factor
reproduces(sp.expand(kappa_rho + kappa_p + 6 * (A ** 2 + 1) * H ** 2 * V) == 0,
           "kappa (rho + p) = -6 (A^2 + 1) H^2 V for every coupling",
```

```
          PY, "linear_member_vacuum_factor")
V_gb = sp.expand(V.subs({alpha1: 1, alpha3: 0}))
reproduces(sp.expand(V_gb - (1 - 8 * alpha2 * H ** 2 * (A ** 2 + 5))) == 0,
          "Gauss-Bonnet: V = 1 - 8 alpha2 H^2 (A^2 + 5)",
          PY, "einstein_gauss_bonnet_vacuum_linear")
a_grid, g_grid = np.meshgrid(np.linspace(-3, 3, 241), np.linspace(0.0, 0.06, 241))
V_grid = 1 - 8 * g_grid * (a_grid ** 2 + 5)  # V with H = 1
fig, ax = plt.subplots()
ax.pcolormesh(a_grid, g_grid, np.sign(V_grid), cmap="coolwarm", shading="auto",
              vmin=-2, vmax=2)
g_curve = np.linspace(0.0025, 1 / 40, 200)  # alpha2 H^2 on the vacuum curve
a_curve = np.sqrt((1 - 40 * g_curve) / (8 * g_curve))
ax.plot(a_curve, g_curve, color="black", label="vacuum $V = 0$, $A > 0$ (deflating)")
ax.plot(-a_curve, g_curve, "--", color="black", label="vacuum $V = 0$, $A < 0$")
ax.text(0, 0.006, "$V > 0$: NEC violated", ha="center")
ax.text(0, 0.045, "$V < 0$: NEC holds", ha="center")
ax.set_xlim(-3, 3)
ax.set_xlabel("slope $A$")
ax.set_ylabel("$\\alpha_2 H^2$")
ax.set_title("Einstein-Gauss-Bonnet: the sign of $V$ and the vacuum curve")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "null_energy_map",
            "Einstein-Gauss-Bonnet gravity ($\\alpha_1 = 1$, $\\alpha_3 = 0$): the "
            "sign of the factor $V = 1 - 8\\alpha_2H^2(A^2 + 5)$ in the plane of the "
            "slope $A$ (horizontal) and $\\alpha_2 H^2$ (vertical). Where $V > 0$ "
            "(lower region) the required source violates the null energy condition, "
            "$\\rho + p < 0$; where $V < 0$ it satisfies it. On the black curves "
            "$V = 0$, $A^2 = (1 - 40\\alpha_2H^2)/(8\\alpha_2H^2)$, a vacuum is "
            "possible; the solid branch has $A > 0$ (deflating extra times), the "
            "dashed one is its mirror image $A < 0$. Above $\\alpha_2H^2 = 1/40$ "
            "there is no vacuum.")
```

Out [10]:

```
PASS kappa (rho + p) = -6 (A^2 + 1) H^2 V for every coupling
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    linear_member_vacuum_factor
PASS Gauss-Bonnet: V = 1 - 8 alpha2 H^2 (A^2 + 5)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_gauss_bonnet_vacuum_linear
```

Out [10]:

```
Figure 12b.5 saved as Revision/textbook/figures/12b_5_null_energy_map.png
```

Text cell:

```
**The Einstein-Gauss-Bonnet vacuum.** On the curve $V = 0$ we still need $\rho = 0$,
which fixes $\Lambda = -\sum_k \alpha_k E_{(k)}{}^{x_4}{}_{x_4}$ at $a_4' = AH$;
then $p = 0$ follows from $\rho + p \propto V = 0$. The next cell substitutes
$A^2 = (1 - 40\alpha_2H^2)/(8\alpha_2H^2)$ and this $\Lambda$ and checks that
```

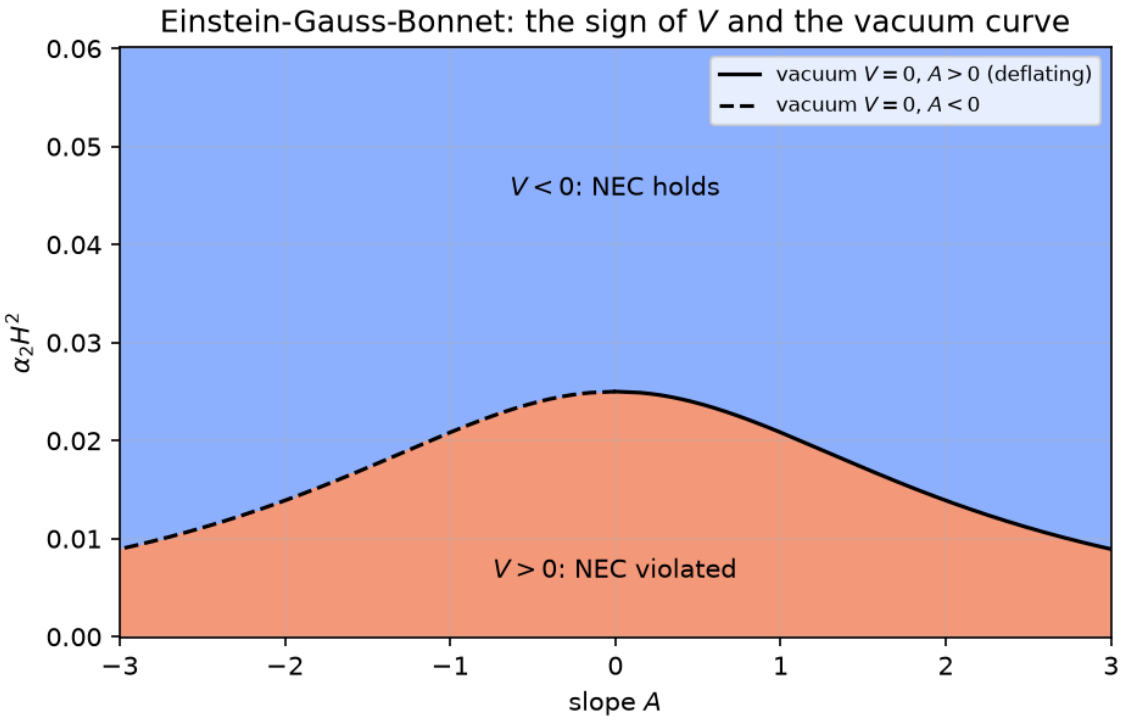

Figure 328: Einstein-Gauss-Bonnet gravity ($\alpha_1 = 1$, $\alpha_3 = 0$): the sign of the factor $V = 1 - 8\alpha_2 H^2(A^2 + 5)$ in the plane of the slope A (horizontal) and $\alpha_2 H^2$ (vertical). Where $V > 0$ (lower region) the required source violates the null energy condition, $\rho + p < 0$; where $V < 0$ it satisfies it. On the black curves $V = 0$, $A^2 = (1 - 40\alpha_2 H^2)/(8\alpha_2 H^2)$, a vacuum is possible; the solid branch has $A > 0$ (deflating extra times), the dashed one is its mirror image $A < 0$. Above $\alpha_2 H^2 = 1/40$ there is no vacuum. (Notebook 12b, figure 5.)

```
$\rho = p = 0$ for every $\alpha_2$; then it draws the slope $A$ and $\Lambda/H^2$
of the vacuum as functions of $\alpha_2H^2$. At $\alpha_2H^2 = 1/40$ the vacuum
is static ($A = 0$) with $\Lambda = -21H^2 + 420H^2/40 = -10.5H^2$.
```

In [11]:

```
A2_vacuum = (1 - 40 * alpha2 * H ** 2) / (8 * alpha2 * H ** 2)  # A^2 on V = 0
GB = {alpha1: 1, alpha3: 0}
rho_gb = sp.expand(kappa_rho.subs(GB))
Lam_vacuum = sp.solve(rho_gb, Lam)[0]  # the Lambda that makes rho = 0
Lam_on_curve = sp.simplify(Lam_vacuum.subs(A ** 2, A2_vacuum))
say(f"Lambda of the vacuum: {Lam_on_curve}")
p_on_curve = sp.simplify((kappa_p.subs(GB).subs(Lam, Lam_vacuum)).subs(A ** 2,
                                                                       A2_vacuum))
check(p_on_curve == 0,
      "Gauss-Bonnet vacuum: with V = 0 and Lambda from rho = 0, also p = 0")
at_edge = Lam_on_curve.subs({alpha2: sp.Rational(1, 40), H: 1})
report("vacuum at alpha2 H^2 = 1/40: Lambda/H^2", at_edge)
check(at_edge == sp.Rational(-21, 2), "at alpha2 H^2 = 1/40: A = 0, Lambda = -10.5 H^2")
lam_vac_f = sp.lambdify(alpha2, Lam_on_curve.subs(H, 1), "numpy")
g_curve = np.linspace(0.004, 1 / 40, 200)
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(g_curve, np.sqrt((1 - 40 * g_curve) / (8 * g_curve)), label="$A > 0$")
left.plot(g_curve, -np.sqrt((1 - 40 * g_curve) / (8 * g_curve)), "--",
          label="$A < 0$")
left.set_xlabel("$\\alpha_2 H^2$")
left.set_ylabel("slope $A$ of the vacuum")
left.legend(fontsize=8)
right.plot(g_curve, lam_vac_f(g_curve))
right.set_xlabel("$\\alpha_2 H^2$")
right.set_ylabel("$\\Lambda/H^2$ of the vacuum")
fig.suptitle("Einstein-Gauss-Bonnet gravity: the vacuum linear members")
save_figure(fig, "gauss_bonnet_vacuum",
            "The vacuum solutions of Einstein-Gauss-Bonnet gravity ($\\alpha_1 = 1$, "
            "$\\alpha_3 = 0$) among the linear members, as functions of "
            "$\\alpha_2 H^2$ from $0.004$ to $1/40$: left, the slope $A$ (two "
            "branches $\\pm A$ with $A^2 = (1 - 40\\alpha_2H^2)/(8\\alpha_2H^2)$); "
            "right, the cosmological constant $\\Lambda/H^2$ that makes the energy "
            "density zero. The slope shrinks to $0$ at $\\alpha_2H^2 = 1/40$, where "
            "$\\Lambda = -10.5H^2$; for smaller $\\alpha_2$ the vacuum deflates (or "
            "inflates) the extra times faster.")
```

Out [11]:

```
Lambda of the vacuum: 720*H**4*alpha2 - 36*H**2 + 3/(16*alpha2)
PASS Gauss-Bonnet vacuum: with V = 0 and Lambda from rho = 0, also p = 0
RESULT vacuum at alpha2 H^2 = 1/40: Lambda/H^2 = -21/2
PASS at alpha2 H^2 = 1/40: A = 0, Lambda = -10.5 H^2
```

Out [11]:

```
Figure 12b.6 saved as Revision/textbook/figures/12b_6_gauss_bonnet_vacuum.png
```

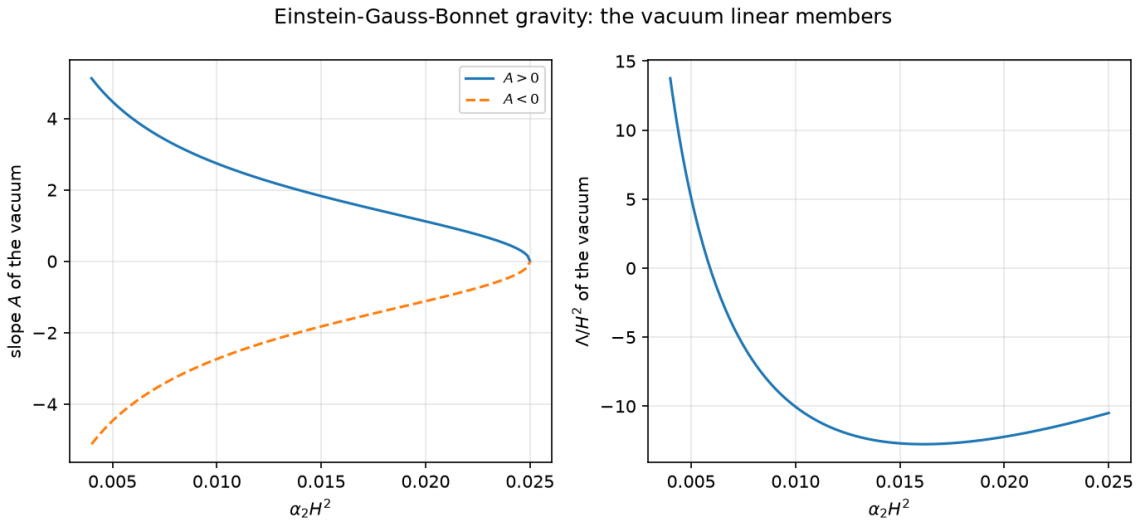

Figure 329: The vacuum solutions of Einstein-Gauss-Bonnet gravity ($\alpha_1 = 1$, $\alpha_3 = 0$) among the linear members, as functions of $\alpha_2 H^2$ from 0.004 to 1/40: left, the slope A (two branches $\pm A$ with $A^2 = (1 - 40\alpha_2 H^2)/(8\alpha_2 H^2)$); right, the cosmological constant Λ/H^2 that makes the energy density zero. The slope shrinks to 0 at $\alpha_2 H^2 = 1/40$, where $\Lambda = -10.5 H^2$; for smaller α_2 the vacuum deflates (or inflates) the extra times faster. (Notebook 12b, figure 6.)

Text cell:

```
**Third order.** With $\alpha_3 \ne 0$ the vacuum condition $V = 0$ is, for a fixed
slope $A$, one straight line in the plane of $\alpha_2H^2$ and $\alpha_3H^4$:

$$\alpha_3H^4 = \frac{8\alpha_2H^2(A^2 + 5) - 1}{72A^4 + 144A^2 + 360}
\qquad(\alpha_1 = 1).$$

The next cell draws these lines for $A = 0, 1, 2, 3$ and checks one point:
for $A = 1$ and $\alpha_2H^2 = 1/48$ the line gives $\alpha_3 = 0$ and indeed
$V = 1 - 8 \cdot \tfrac{1}{48} \cdot 6 = 0$.
```

In [12]:

```
V_one = sp.expand(V.subs({alpha1: 1, H: 1}))  # V with alpha1 = 1, H = 1
alpha3_line = sp.solve(V_one, alpha3)[0]  # alpha3 H^4 on the vacuum line
say(f"vacuum line: alpha3 H^4 = {sp.factor(alpha3_line)}")
check(alpha3_line.subs({A: 1, alpha2: sp.Rational(1, 48)}) == 0,
      "A = 1, alpha2 H^2 = 1/48: the vacuum line passes through alpha3 = 0")
line_f = sp.lambdify((A, alpha2), alpha3_line, "numpy")
g_values = np.linspace(0.0, 0.08, 201)
fig, ax = plt.subplots()
for slope in (0, 1, 2, 3):
    label = "$A = 0$" if slope == 0 else f"$A = \\pm{slope}$"
    ax.plot(g_values, line_f(slope, g_values), label=label)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("$\\alpha_2 H^2$")
```

```
ax.set_ylabel("$\\alpha_3 H^4$")
ax.set_title("Third-order Lovelock gravity: the vacuum lines $V = 0$")
ax.legend(fontsize=8)
save_figure(fig, "third_order_vacua",
            "Lovelock gravity of third order ($\\alpha_1 = 1$): for each slope $A$ "
            "of the linear member the couplings $(\\alpha_2 H^2, \\alpha_3 H^4)$ that "
            "allow a vacuum lie on one straight line, $\\alpha_3 H^4 = "
            "(8\\alpha_2H^2(A^2 + 5) - 1)/(72A^4 + 144A^2 + 360)$, drawn for "
            "$A = 0, 1, 2, 3$ (each line serves $+A$ and $-A$). Where a line crosses "
            "$\\alpha_3 = 0$ it meets the Gauss-Bonnet vacuum; a positive $\\alpha_3$ "
            "allows vacua at larger $\\alpha_2$.")
```

Out [12]:

```
vacuum line: alpha3 H^4 = (8*A**2*alpha2 + 40*alpha2 - 1)/(72*(A**4 + 2*A**2 + 5))
PASS A = 1, alpha2 H^2 = 1/48: the vacuum line passes through alpha3 = 0
```

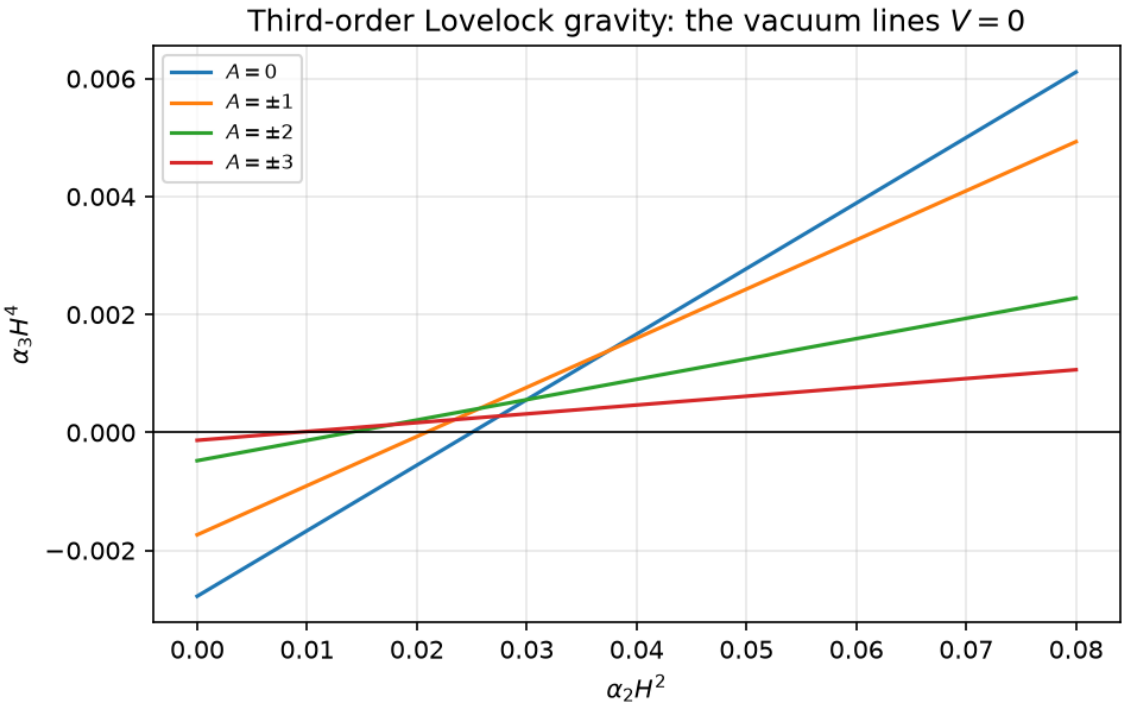

Figure 330: Lovelock gravity of third order ($\alpha_1 = 1$): for each slope A of the linear member the couplings $(\alpha_2 H^2, \alpha_3 H^4)$ that allow a vacuum lie on one straight line, $\alpha_3 H^4 = (8\alpha_2 H^2(A^2 + 5) - 1)/(72A^4 + 144A^2 + 360)$, drawn for $A = 0, 1, 2, 3$ (each line serves $+A$ and $-A$). Where a line crosses $\alpha_3 = 0$ it meets the Gauss-Bonnet vacuum; a positive α_3 allows vacua at larger α_2. (Notebook 12b, figure 7.)

Out [12]:

```
Figure 12b.7 saved as Revision/textbook/figures/12b_7_third_order_vacua.png
```

Text cell:

```
## 9. A homogeneous condensate of dirac16complex00 as the source

The record proves (checks ‘condensate_S_constant‘,
‘condensate_kinetic_tensor_diagonal‘) that a condensate $\Phi(x_4)$ of the commuting
field has constant $S = \bar\Phi\Phi$, energy density $\rho = mS + U(S)$ and equal
pressures $p = SU’(S) - U(S)$ (sign convention $\sigma_T = +1$ of the record), and
that, when its off-diagonal conditions hold (see below), it allows only the
linear member. With $U = \tfrac{\lambda}{2}S^2$ we have
$U’ = \lambda S$, so $\rho = mS + \tfrac{\lambda}{2}S^2$ and $p = \tfrac{\lambda}{2}S^2$.
Setting them equal to the Einstein values and taking the difference and the sum:

$$\kappa mS = -(36H^2 + 2\Lambda),\qquad 6(A^2 + 1)H^2 = -\kappa S(m + \lambda S).$$

The next cell checks this equivalence with sympy, then solves for $A^2$ in units
$H = \kappa = 1$: $S = -(36 + 2\Lambda)/m$ and
$A^2 = 5 + \Lambda/3 - \lambda S^2/6$. For the two worked examples it also prints
the density $S$ and the effective mass $M = m + \lambda S$ that the condensate
must have.
```

In [13]:

```
m, lam, S = sp.symbols("m lam S", real=True)  # mass, coupling lambda, density S
rho_condensate = m * S + lam * S ** 2 / 2  # rho = m S + U, U = (lambda/2) S^2
p_condensate = lam * S ** 2 / 2  # p = S U’ - U
condition_rho = sp.expand(kappa * rho_condensate - rho_e)  # = 0
condition_p = sp.expand(kappa * p_condensate - p_e)  # = 0
first = sp.expand(kappa * m * S + 36 * H ** 2 + 2 * Lam)  # kappa m S = -(36 H^2 + 2 L)
second = sp.expand(6 * (A ** 2 + 1) * H ** 2 + kappa * S * (m + lam * S))
reproduces(sp.expand(condition_rho - condition_p - first) == 0
           and sp.expand(condition_rho + condition_p - second) == 0,
           "condensate, Einstein: kappa m S = -(36 H^2 + 2 Lambda) and "
           "6 (A^2 + 1) H^2 = -kappa S (m + lambda S)",
           PY, "condensate_einstein_quadratic_U")
UNIT = {H: 1, kappa: 1}
S_solution = sp.solve(first.subs(UNIT), S)[0]
A2_solution = sp.expand(sp.solve(second.subs(UNIT), A ** 2)[0].subs(S, S_solution))
say(f"S = {S_solution},  A^2 = {A2_solution}")
example_1 = A2_solution.subs({m: 5, lam: 0, Lam: 0})
example_2 = A2_solution.subs({m: -15, lam: sp.Rational(25, 6), Lam: 0})
report("m = 5, lambda = 0, Lambda = 0: A^2", example_1)
report("m = -15, lambda = 25/6, Lambda = 0: A^2", example_2)
check(example_1 == 5 and example_2 == 1, "the two worked examples: A^2 = 5 and A^2 = 1")
effective = []  # the effective mass M = m + lambda S of each example
for mass, coupling in ((5, 0), (-15, sp.Rational(25, 6))):
    S_value = S_solution.subs({m: mass, Lam: 0})  # the density the example needs
    effective.append(mass + coupling * S_value)
    report(f"m = {mass}, lambda = {coupling}: S and M = m + lambda S",
           f"{S_value}, {effective[-1]}")
check(effective == [5, -5], "the examples need M = 5 (S < 0) and M = -5 (S > 0)")
```

Out [13]:

```
PASS condensate, Einstein: kappa m S = -(36 H^2 + 2 Lambda) and 6 (A^2 + 1) H^2 = -kappa
    S (m + lambda S)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    condensate_einstein_quadratic_U
S = 2*(-Lam - 18)/m,  A^2 = -2*Lam**2*lam/(3*m**2) - 24*Lam*lam/m**2 + Lam/3 -
    216*lam/m**2 + 5
RESULT m = 5, lambda = 0, Lambda = 0: A^2 = 5
RESULT m = -15, lambda = 25/6, Lambda = 0: A^2 = 1
PASS the two worked examples: A^2 = 5 and A^2 = 1
RESULT m = 5, lambda = 0: S and M = m + lambda S = -36/5, 5
RESULT m = -15, lambda = 25/6: S and M = m + lambda S = 12/5, -5
PASS the examples need M = 5 (S < 0) and M = -5 (S > 0)
```

Text cell:

```
The record also lists every off-diagonal kinetic component of the condensate: each
is a multiple of one of 15 three-gamma bilinears
$\bar\Phi\gamma^a\gamma^b\gamma^c\Phi$, and the off-diagonal field equations
$0 = \kappa T^\mu{}_\nu$ require those bilinears to vanish. The next cell counts
the entries (42 ordered pairs $(\mu, \nu)$) and the distinct bilinears (15).

Then it draws $A^2$ as a function of $\lambda$ for $m = 5$ and $m = -15$
($\Lambda = 0$, $H = \kappa = 1$): $A^2 = 5 - 216\lambda/m^2$. A real slope needs
$A^2 \ge 0$, that is $\lambda \le 5m^2/216$. The two worked examples are marked.
```

In [14]:

```
condensate = record["fields"]["dirac16complex00"]
entries = condensate["offDiagonalKinetic"]
bilinears = sorted({term["bilinear"] for entry in entries for term in entry["terms"]})
report("off-diagonal kinetic components of the condensate", len(entries))
report("distinct three-gamma bilinears", len(bilinears))
reproduces(len(entries) == 42 and len(bilinears) == 15,
           "42 off-diagonal components, multiples of 15 three-gamma bilinears",
           WL, "condensate_offdiagonal_are_three_gamma_bilinears")
a2_f = sp.lambdify((m, lam), A2_solution.subs(Lam, 0), "numpy")
fig, ax = plt.subplots()
for mass, upper in ((5, 1.0), (-15, 6.0)):
    lam_values = np.linspace(0.0, upper, 200)
    ax.plot(lam_values, a2_f(mass, lam_values), label=f"$m = {mass}$")
ax.plot([0], [5], "o", color="black")
ax.plot([25 / 6], [1], "s", color="black")
arrow = {"arrowstyle": "->"}  # a thin arrow from the text to the point
ax.annotate("$m=5$, $\\lambda=0$: $A^2=5$", (0, 5), (0.5, 5.8), fontsize=8,
            arrowprops=arrow)
ax.annotate("$m=-15$, $\\lambda=25/6$: $A^2=1$", (25 / 6, 1), (3.6, 2.6),
            fontsize=8, arrowprops=arrow)
ax.set_ylim(-4.2, 6.6)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("self-coupling $\\lambda$ (units $H = \\kappa = 1$)")
ax.set_ylabel("$A^2$ required by the condensate")
ax.set_title("A dirac16complex00 condensate as the source, Einstein, $\\Lambda=0$")
```

```
ax.legend(fontsize=8)
save_figure(fig, "condensate_slope",
            "Einstein gravity with $\\Lambda = 0$ and a homogeneous dirac16complex00 "
            "condensate with $U = \\lambda S^2/2$ as the source: the square $A^2$ of "
            "the slope of the linear member that the field equations then require, "
            "$A^2 = 5 - 216\\lambda/m^2$, as a function of the self-coupling "
            "$\\lambda$ for the masses $m = 5$ and $m = -15$ (units "
            "$H = \\kappa = 1$). A real slope needs $A^2 \\ge 0$. The circle and the "
            "square mark the worked examples $A^2 = 5$ and $A^2 = 1$; both signs "
            "$\\pm A$ are allowed.")
```

Out [14]:

```
RESULT off-diagonal kinetic components of the condensate = 42
RESULT distinct three-gamma bilinears = 15
PASS 42 off-diagonal components, multiples of 15 three-gamma bilinears
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_offdiagonal_are_three_gamma_bilinears
```

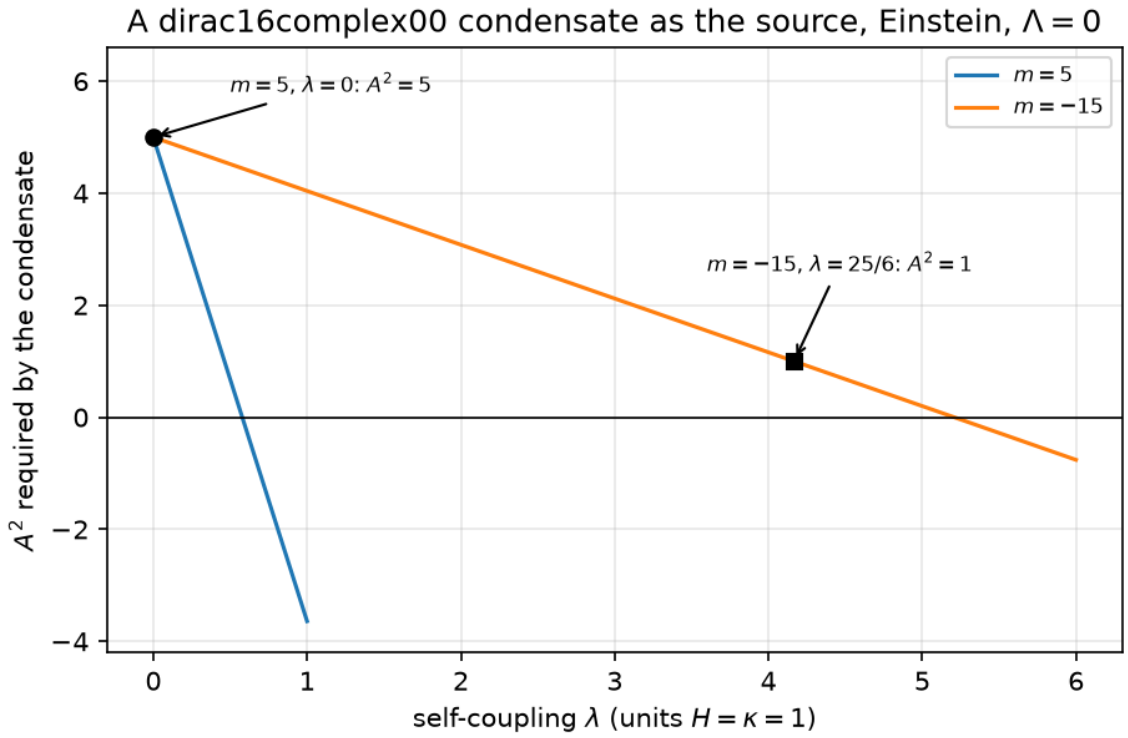

Figure 331: Einstein gravity with $\Lambda = 0$ and a homogeneous dirac16complex00 condensate with $U = \lambda S^2/2$ as the source: the square A^2 of the slope of the linear member that the field equations then require, $A^2 = 5 - 216\lambda/m^2$, as a function of the self-coupling λ for the masses $m = 5$ and $m = -15$ (units $H = \kappa = 1$). A real slope needs $A^2 \geq 0$. The circle and the square mark the worked examples $A^2 = 5$ and $A^2 = 1$; both signs $\pm A$ are allowed. (Notebook 12b, figure 8.)

Out [14]:

```
Figure 12b.8 saved as Revision/textbook/figures/12b_8_condensate_slope.png
```

Text cell:

```
**What is established here, and where it is completed.** The two conditions above
are PROVED consequences of the field equations for a condensate source. The
Revision record has exact condensates with all 15 bilinears zero, but it contains
no check that combines one of them with these Einstein conditions. Notebook 12d of
this chapter does that combination: it builds condensates with all 15 bilinears
zero and exactly the values $(M, S) = (5, -36/5)$ and $(-5, 12/5)$ of the two worked
examples, and checks every field equation exactly (a computation of the textbook,
not a Revision record). With $\Lambda = 0$ the required energy density is
$\kappa\rho = -(21 + 3A^2)H^2 < 0$: such a source has negative energy.
```

Text cell:

```
## 10. The last check

The last cell checks that the eight figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [15]:

```
figure_names = ["einstein_source", "source_regions", "phantom_w",
                "gauss_bonnet_source", "null_energy_map", "gauss_bonnet_vacuum",
                "third_order_vacua", "condensate_slope"]
paths = [output_file(f"{FIGURE_FOLDER}/12b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths), "all eight figure files exist")
all_checks_passed()
```

Out [15]:

```
PASS all eight figure files exist
ALL 24 CHECKS PASSED (notebook 12b)
```

Text cell:

```
## 11. What this notebook showed

- The linear member $a_4 = AHx_4 + a_0$ needs a source with equal pressures
  $p_3 = p_t = p_0 = p$ and constant $\rho$ and $p$, given by the record formulas
  (PROVED).
- $\rho$ and $p$ contain $A$ only as $A^2$: the field equations accept deflating
  extra times ($A > 0$) and inflating ones ($A < 0$) equally; deflation is a choice
  of sign, an initial condition (PROVED).
- Einstein gravity: $\kappa\rho = -(21 + 3A^2)H^2 - \Lambda$,
  $\kappa p = (15 - 3A^2)H^2 + \Lambda$, $\kappa(\rho + p) = -6(1 + A^2)H^2 < 0$;
```

```
  $\rho > 0$ is possible only with $\Lambda < -(21 + 3A^2)H^2$ and then $w < -1$
  (PROVED).
- Gauss-Bonnet and third-order couplings: $\kappa(\rho + p) = -6(A^2 + 1)H^2V$; the
  null energy condition holds where $V \le 0$; vacua exist on $V = 0$, for
  Einstein-Gauss-Bonnet gravity only when $0 < \alpha_2 H^2 \le 1/40$ (PROVED).
- A homogeneous dirac16complex00 condensate as the source needs
  $\kappa mS = -(36H^2 + 2\Lambda)$ and $6(A^2 + 1)H^2 = -\kappa S(m + \lambda S)$
  (PROVED); the two worked examples need $(M, S) = (5, -36/5)$ and $(-5, 12/5)$,
  and Notebook 12d builds condensates that meet them exactly.
```

12.20 Line-by-line walk-through of Notebook 12b

The notebook has 15 code cells, In [1] to In [15].

In [1], the set-up cell. It is the set-up cell of Notebook 12a, word for word, except for two things: the comment lines at the top hold the run instructions of Notebook 12b (Section 12.18), and the line `NOTEBOOK_ID = "12b"` names this notebook. Every line is explained in Section 12.16 under In [1]; the cell prints its one line, Set-up of notebook 12b complete.

In [2], the Revision records. The first lines are those of In [2] of Notebook 12a: the imports of `contextlib` and `io`, the short names `PY`, `WL`, `LEAD` and `EQUATIONS` of four record files, and the three helpers `read_json`, `record_verdict` and `reproduces` (Section 12.16, In [2]). New are the last three lines:

```
record = read_json(EQUATIONS)
title = record["title"]  # the title stored in the record
say(f"read {EQUATIONS}: {title}")
```

The record of the field equations is read once into the dictionary `record`, which the later cells use; its title is printed to show which file was read.

In [3], the required source for every coupling.

```
import numpy as np  # arrays of numbers for the plots
import sympy as sp  # exact algebra with symbols

H = sp.symbols("H", positive=True)  # the constant H of the metric
A = sp.symbols("A", real=True)  # the slope of the linear member
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
alpha1, alpha2, alpha3 = sp.symbols("alpha1 alpha2 alpha3", real=True)
Lam, kappa = sp.symbols("Lam kappa", real=True)  # Lambda and kappa
NAMES = {"ad1": ad1, "ad2": ad2, "H": H, "AA": A, "alpha1": alpha1,
         "alpha2": alpha2, "alpha3": alpha3, "Lam": Lam, "kappa": kappa}
```

The imports and symbols as in Notebook 12a; `NAMES` tells the parser which names of the record mean which symbols (the record calls the slope `AA`).

```
def parse(text):
    return sp.sympify(text.replace("^", "**"), locals=NAMES)
```

parse turns a formula of the record, written in the Wolfram Language, into a sympy expression: the power sign ^ becomes ** and sp.sympify reads the text (docstring left out).

```
ALPHA = {1: alpha1, 2: alpha2, 3: alpha3}
lovelock = record["lovelockTensors"]
side = {key: sum(ALPHA[k] * parse(lovelock[f"E{k}"][key]["input"]) for k in (1, 2, 3))
        for key in ("x1x1", "x4x4", "x5x5", "x0x0")}  # sum_k alpha_k E_(k)
```

For each of the four distinct diagonal components the dictionary comprehension forms $\sum_k \alpha_k E_{(k)}$ from the record's three tensors: side["x4x4"] is e_{44} of Section 12.8, and so on.

```
LINEAR = {ad1: A * H, ad2: 0}  # a4' = A H, a4'' = 0
kappa_rho = sp.expand(-(side["x4x4"] + Lam).subs(LINEAR))
kappa_p3 = sp.expand((side["x1x1"] + Lam).subs(LINEAR))
kappa_pt = sp.expand((side["x5x5"] + Lam).subs(LINEAR))
kappa_p0 = sp.expand((side["x0x0"] + Lam).subs(LINEAR))
say(f"kappa rho = {kappa_rho}")
say(f"kappa p0 = {kappa_p0}")
```

The linear member is substituted, and the four field equations of Section 12.8 are solved for the source: $\kappa\rho = -(e_{44} + \Lambda)$, $\kappa p_3 = e_{11} + \Lambda$, and so on. Out [3] prints $\kappa\rho$ and κp_0: the general formulas of Section 12.12.

In [4], equal pressures and the symmetry A to minus A.

```
reproduces(sp.expand(kappa_p3 - kappa_p0) == 0 and sp.expand(kappa_pt - kappa_p0) == 0,
           "linear member: p3 = pt = p0 = p", PY, "linear_member_equal_pressures")
kappa_p = kappa_p0  # the common pressure, times kappa
```

The three pressures agree for every coupling; the common value is named kappa_p.

```
linear_record = record["linearMember"]
same = (sp.expand(kappa * parse(linear_record["rho"]["input"]) - kappa_rho) == 0
        and sp.expand(kappa * parse(linear_record["p"]["input"]) - kappa_p) == 0)
reproduces(same, "kappa rho and kappa p equal the record", PY, "json_linear_member")
```

The record stores ρ and p themselves (divided by κ); multiplied by kappa they must equal the notebook's $\kappa\rho$ and κp.

```
check(sp.expand(kappa_rho.subs(A, -A) - kappa_rho) == 0
      and sp.expand(kappa_p.subs(A, -A) - kappa_p) == 0,
      "A -> -A leaves rho and p unchanged: deflation is a choice of sign")
reproduces(sp.expand(sp.diff(kappa_rho, alpha1) + 3 * H ** 2 * (A ** 2 + 7)) == 0,
           "the alpha1 part of kappa rho is -3 H^2 (A^2 + 7)",
           LEAD, "einstein_linear_rho_alpha1")
```

Replacing A by $-A$ changes nothing (Section 12.12). The part of $\kappa\rho$ that multiplies α_1 is its derivative with respect to α_1 (the formula is linear in the couplings); it must be $-3H^2(A^2 + 7)$, as the lead's independent computation found.

In [5], Einstein gravity.

```
EINSTEIN = {alpha1: 1, alpha2: 0, alpha3: 0}
rho_e = sp.expand(kappa_rho.subs(EINSTEIN))  # kappa rho in Einstein gravity
p_e = sp.expand(kappa_p.subs(EINSTEIN))  # kappa p in Einstein gravity
report("Einstein: kappa rho", rho_e)
report("Einstein: kappa p", p_e)
```

The Einstein couplings are substituted and $\kappa\rho = -3A^2H^2 - 21H^2 - \Lambda$ and $\kappa p = -3A^2H^2 + 15H^2 + \Lambda$ are printed.

```
same = (sp.expand(kappa * parse(linear_record["rhoEinstein"]["input"]) - rho_e) == 0
        and sp.expand(kappa * parse(linear_record["pEinstein"]["input"]) - p_e) == 0
        and sp.expand(kappa * parse(linear_record["rhoPlusPEinstein"]["input"])
                      - (rho_e + p_e)) == 0)
reproduces(same and sp.expand(rho_e + p_e + 6 * (1 + A ** 2) * H ** 2) == 0,
           "Einstein: kappa (rho + p) = -6 (1 + A^2) H^2 < 0", PY,
           "json_linear_member")
```

The three Einstein formulas of the record (ρ, p, $\rho + p$) are compared, and the sum $\kappa(\rho + p) = -6(1 + A^2)H^2$ is checked.

```
UNITS = {H: 1, Lam: 0, A: 1}
rho_1, p_1 = rho_e.subs(UNITS), p_e.subs(UNITS)
report("Einstein, Lambda = 0, A = 1: kappa rho", rho_1, "H^2")
report("Einstein, Lambda = 0, A = 1: kappa p", p_1, "H^2")
report("Einstein, Lambda = 0, A = 1: w = p/rho", sp.Rational(p_1, rho_1))
check(rho_1 == -24 and p_1 == 12, "A = 1, Lambda = 0: kappa rho = -24, kappa p = 12")
```

The canonical history $A = 1$ without cosmological constant, in units $H = 1$: $\kappa\rho = -24$, $\kappa p = 12$, and the exact fraction `sp.Rational(p_1, rho_1)` $= -1/2$.

In [6], figure 1: the Einstein source against A.

```
rho_f = sp.lambdify((A, Lam), rho_e.subs(H, 1), "numpy")  # numbers from formulas
p_f = sp.lambdify((A, Lam), p_e.subs(H, 1), "numpy")
a_values = np.linspace(-3.0, 3.0, 301)
```

`sp.lambdify` makes numpy functions of A and Λ from the exact formulas with $H = 1$; `a_values` holds 301 slopes from -3 to 3.

```
fig, ax = plt.subplots()
ax.axvspan(0, 3, color="tab:green", alpha=0.08)  # the deflating half, A > 0
ax.plot(a_values, rho_f(a_values, 0.0), label="$\\kappa\\rho/H^2$")
ax.plot(a_values, p_f(a_values, 0.0), label="$\\kappa p/H^2$")
ax.plot(a_values, rho_f(a_values, 0.0) + p_f(a_values, 0.0), ":", color="black",
        label="$\\kappa(\\rho+p)/H^2=-6(1+A^2)$")
ax.axhline(0.0, color="black", linewidth=0.8)
```

`ax.axvspan(0, 3, ...)` shades the vertical strip $0 < A < 3$ in a faint green: the deflating half. The three curves are $\kappa\rho$, κp and their sum (dotted, `":"`) at $\Lambda = 0$; `axhline` draws the zero line.

```
ax.text(1.5, 15, "$A>0$: extra times deflate", ha="center", fontsize=9)
ax.text(-1.5, 15, "$A<0$: extra times inflate", ha="center", fontsize=9)
ax.set_xlabel("slope $A$ of $a_4 = AHx_4 + a_0$")
ax.set_ylabel("required source (units of $H^2/\\kappa$)")
ax.set_title("Einstein gravity, $\\Lambda = 0$: the source of the linear member")
ax.legend(fontsize=8, loc="lower center")
save_figure(fig, "einstein_source",
            ...)
```

`ax.text(x, y, text, ...)` writes a text at the point (x, y) of the plot; the two texts name the halves. Labels, title, legend and `save_figure` as before. **What the figure shows.** Every curve is symmetric about $A = 0$: the deflating half is the mirror image of the inflating half. The energy density is a downward parabola that never rises above $-21H^2/\kappa$; the pressure is positive for $|A| < \sqrt{5}$; their sum lies below zero everywhere.

In [7], figure 2: the signs of rho and p.

```
a_grid, lam_grid = np.meshgrid(np.linspace(-3, 3, 241), np.linspace(-60, 30, 241))
rho_grid = rho_f(a_grid, lam_grid)
p_grid = p_f(a_grid, lam_grid)
region = np.where(rho_grid > 0, 0, np.where(p_grid < 0, 1, 2))  # three regions
```

`np.meshgrid` makes two 241 by 241 arrays that together hold every point of a grid in the plane of A (from -3 to 3) and Λ/H^2 (from -60 to 30); the functions of In [6] evaluate $\kappa\rho$ and κp at all of them at once. `np.where(condition, x, y)` takes `x` where the condition holds and `y` elsewhere; the nested use labels each point 0 ($\rho > 0$), 1 ($\rho \le 0$, $p < 0$) or 2 ($\rho \le 0$, $p \ge 0$).

```
fig, ax = plt.subplots()
colours = matplotlib.colors.ListedColormap(["#f4a582", "#d1e5f0", "#92c5de"])
ax.pcolormesh(a_grid, lam_grid, region, cmap=colours, shading="auto")
ax.plot(a_values, -(21 + 3 * a_values ** 2), color="black",
        label="$\\rho = 0$: $\\Lambda = -(21+3A^2)H^2$")
ax.plot(a_values, 3 * a_values ** 2 - 15, "--", color="black",
        label="$p = 0$: $\\Lambda = (3A^2-15)H^2$")
```

`ListedColormap` makes a colour map of three colours (written as hexadecimal colour codes), one per region, and `ax.pcolormesh` colours the grid with it. The solid curve is the border $\rho = 0$, the dashed one the border $p = 0$ (Section 12.12).

```
ax.text(0, -50, "$\\rho > 0$, $p < 0$: $w < -1$", ha="center")
ax.text(0, -18, "$\\rho < 0$, $p < 0$", ha="center")
ax.text(0, 5, "$\\rho < 0$, $p > 0$", ha="center")
ax.set_xlim(-3, 3)
ax.set_ylim(-60, 30)
ax.set_xlabel("slope $A$")
ax.set_ylabel("$\\Lambda/H^2$")
ax.set_title("Einstein gravity: the signs of the required $\\rho$ and $p$")
ax.legend(fontsize=8, loc="upper center")
save_figure(fig, "source_regions",
            ...)
```

Three texts name the regions; `set_xlim` and `set_ylim` fix the ranges of the axes. **What the figure shows.** Below the solid curve the energy density is positive, but there the pressure is negative and $w < -1$. Above the dashed curve the pressure is positive and the energy density negative. In the band between both are negative. No colour stands for "both positive", and since the two curves never touch, no point is a vacuum.

In [8], figure 3: positive energy forces w below minus 1.

```
w_e = sp.simplify(p_e / rho_e)  # w = p/rho in Einstein gravity
say(f"Einstein: w = p/rho = {w_e}")
check(sp.simplify(w_e + 1 - (rho_e + p_e) / rho_e) == 0,
      "w + 1 = (rho + p)/rho, so rho > 0 and rho + p < 0 give w < -1")
```

The equation of state $w = p/\rho$ is printed, and the identity $w + 1 = (\rho + p)/\rho$ (add $\rho/\rho = 1$ to p/ρ) is checked; with $\rho > 0$ and $\rho + p < 0$ it gives $w + 1 < 0$.

```
fig, ax = plt.subplots()
for lam in (-30.0, -40.0, -60.0):
    rho_values = rho_f(a_values, lam)
    keep = rho_values > 1.0  # where the energy density is clearly positive
    w_values = np.where(keep, p_f(a_values, lam) / np.where(keep, rho_values, 1.0),
                        np.nan)  # nan: not drawn
    ax.plot(a_values, w_values, label=f"$\\Lambda = {lam:.0f}\\,H^2$")
    check(np.nanmax(w_values) < -1.0, f"Lambda = {lam:.0f} H^2: w < -1 where rho > 0")
```

For three cosmological constants the energy density is computed on the 301 slopes; `keep` is an array of true and false values that marks where $\kappa\rho > 1$ (clearly positive). There $w = p/\rho$ is computed; elsewhere `np.nan` (not a number) is stored, which matplotlib does not draw. The inner `np.where(keep, rho_values, 1.0)` avoids a division by a value near zero outside the kept points. `np.nanmax` is the largest value that is not `nan`; it must be below -1.

```
ax.axhline(-1.0, color="black", linewidth=0.8)
ax.set_ylim(-6, 0)
ax.set_xlabel("slope $A$")
ax.set_ylabel("$w = p/\\rho$")
ax.set_title("Einstein gravity: where $\\rho > 0$ the source has $w < -1$")
ax.legend(fontsize=8)
save_figure(fig, "phantom_w",
            ...)
```

The line $w = -1$, the range of the vertical axis, labels, legend and `save_figure`. **What the figure shows.** Each curve is drawn only for slopes small enough that $\kappa\rho > H^2$ (for $\Lambda = -30H^2$ the energy density is positive for $|A| < \sqrt{3}$, from $21 + 3A^2 < 30$), and each lies entirely below $w = -1$: a positive energy density of the author's metric is always phantom.

In [9], figure 4: the Gauss-Bonnet coupling.

```
reproduces(sp.expand(sp.diff(kappa_rho, alpha2)
                     - H ** 4 * (36 * A ** 4 + 120 * A ** 2 + 420)) == 0,
```

```
           "the alpha2 part of kappa rho is H^4 (36 A^4 + 120 A^2 + 420)",
           LEAD, "gauss_bonnet_rho_alpha2")
reproduces(sp.expand(sp.diff(kappa_p, alpha2)
                     - 12 * H ** 4 * (A ** 4 + 14 * A ** 2 - 15)) == 0,
           "the alpha2 part of kappa p is 12 H^4 (A^4 + 14 A^2 - 15)",
           LEAD, "gauss_bonnet_p_alpha2")
```

The parts of $\kappa\rho$ and κp that multiply α_2 are compared with the lead's classical Gauss-Bonnet computation.

```
symbols_numeric = (A, Lam, alpha1, alpha2, alpha3)
rho_l = sp.lambdify(symbols_numeric, kappa_rho.subs(H, 1), "numpy")  # any coupling
p_l = sp.lambdify(symbols_numeric, kappa_p.subs(H, 1), "numpy")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
for a2 in (0.0, 0.01, 0.025, 0.04):
    label = f"$\\alpha_2H^2 = {a2}$"
    left.plot(a_values, rho_l(a_values, 0, 1, a2, 0), label=label)
    right.plot(a_values, p_l(a_values, 0, 1, a2, 0), label=label)
```

Numpy functions of the slope, Λ and the three couplings. A figure with two panels side by side (`left` and `right`); for four values of $\alpha_2 H^2$, with $\Lambda = 0$, $\alpha_1 = 1$, $\alpha_3 = 0$, the energy density is drawn on the left and the pressure on the right.

```
for ax, name in ((left, "\\kappa\\rho/H^2"), (right, "\\kappa p/H^2")):
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xlabel("slope $A$")
    ax.set_ylabel(f"${name}$")
    ax.legend(fontsize=8)
left.set_title("energy density, $\\Lambda = 0$")
right.set_title("pressure, $\\Lambda = 0$")
save_figure(fig, "gauss_bonnet_source",
            ...)
```

The same zero line, labels and legend for both panels, then titles and `save_figure`.

```
rho_gb_0 = sp.expand(kappa_rho.subs({alpha1: 1, alpha3: 0, Lam: 0, A: 0}))
check(sp.expand(rho_gb_0 - (420 * alpha2 * H ** 4 - 21 * H ** 2)) == 0,
      "Gauss-Bonnet, Lambda = 0, A = 0: kappa rho = (420 alpha2 H^2 - 21) H^2")
p_gb_1 = [sp.expand(kappa_p.subs({alpha1: 1, alpha3: 0, Lam: 0, A: s}))
          for s in (1, -1)]
check(p_gb_1 == [12 * H ** 2, 12 * H ** 2],
      "Gauss-Bonnet, Lambda = 0, A = +1 or -1: kappa p = 12 H^2 for every alpha2")
```

Two facts that the figure shows, checked exactly: at $A = 0$ the energy density is $(420\alpha_2 H^2 - 21)H^2$, and at $A = \pm 1$ the pressure is $12H^2$ whatever α_2 is, because the Gauss-Bonnet part $12H^4(1+14-15)$ vanishes there. (These two PASS lines appear after the figure in Out [9], because they are printed after `save_figure`.) **What the figure shows.** The Gauss-Bonnet term adds $\alpha_2 H^4(36A^4 + 120A^2 + 420) > 0$ to $\kappa\rho$: for $\alpha_2 H^2 = 0.025$ and 0.04 the energy density becomes positive at large $|A|$ without any cosmological constant. All pressure curves cross at $A = \pm 1$.

In [10], figure 5: the null energy condition and the vacuum curve.

```
V = parse(linear_record["vacuumFactor"]["input"])  # the record vacuum factor
reproduces(sp.expand(kappa_rho + kappa_p + 6 * (A ** 2 + 1) * H ** 2 * V) == 0,
           "kappa (rho + p) = -6 (A^2 + 1) H^2 V for every coupling",
           PY, "linear_member_vacuum_factor")
V_gb = sp.expand(V.subs({alpha1: 1, alpha3: 0}))
reproduces(sp.expand(V_gb - (1 - 8 * alpha2 * H ** 2 * (A ** 2 + 5))) == 0,
           "Gauss-Bonnet: V = 1 - 8 alpha2 H^2 (A^2 + 5)",
           PY, "einstein_gauss_bonnet_vacuum_linear")
```

The record's vacuum factor V is read and the factorisation $\kappa(\rho + p) = -6(A^2 + 1)H^2V$ is checked, then its Gauss-Bonnet form.

```
a_grid, g_grid = np.meshgrid(np.linspace(-3, 3, 241), np.linspace(0.0, 0.06, 241))
V_grid = 1 - 8 * g_grid * (a_grid ** 2 + 5)  # V with H = 1
fig, ax = plt.subplots()
ax.pcolormesh(a_grid, g_grid, np.sign(V_grid), cmap="coolwarm", shading="auto",
              vmin=-2, vmax=2)
```

A grid in the plane of A and $\alpha_2 H^2$ (from 0 to 0.06); V on the grid; `np.sign` is $+1$, 0 or -1, and the colour map `coolwarm` with the range -2 to 2 shows the two signs in muted blue and red.

```
g_curve = np.linspace(0.0025, 1 / 40, 200)  # alpha2 H^2 on the vacuum curve
a_curve = np.sqrt((1 - 40 * g_curve) / (8 * g_curve))
ax.plot(a_curve, g_curve, color="black", label="vacuum $V = 0$, $A > 0$ (deflating)")
ax.plot(-a_curve, g_curve, "--", color="black", label="vacuum $V = 0$, $A < 0$")
```

The vacuum curve $A^2 = (1 - 40\alpha_2 H^2)/(8\alpha_2 H^2)$, drawn for $\alpha_2 H^2$ from 0.0025 to 1/40, with its deflating branch $A > 0$ (solid) and the mirror branch (dashed).

```
ax.text(0, 0.006, "$V > 0$: NEC violated", ha="center")
ax.text(0, 0.045, "$V < 0$: NEC holds", ha="center")
ax.set_xlim(-3, 3)
ax.set_xlabel("slope $A$")
ax.set_ylabel("$\\alpha_2 H^2$")
ax.set_title("Einstein-Gauss-Bonnet: the sign of $V$ and the vacuum curve")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "null_energy_map",
            ...)
```

Texts naming the two regions (NEC is the null energy condition), labels and `save_figure`. **What the figure shows.** Below the curve $V > 0$ and the required source violates the null energy condition, as in Einstein gravity; above it a sufficiently large Gauss-Bonnet coupling allows a source that satisfies it. On the curve itself a vacuum is possible; the curve reaches $A = 0$ at $\alpha_2 H^2 = 1/40$.

In [11], figure 6: the Gauss-Bonnet vacua.

```
A2_vacuum = (1 - 40 * alpha2 * H ** 2) / (8 * alpha2 * H ** 2)  # A^2 on V = 0
GB = {alpha1: 1, alpha3: 0}
rho_gb = sp.expand(kappa_rho.subs(GB))
```

```
Lam_vacuum = sp.solve(rho_gb, Lam)[0]  # the Lambda that makes rho = 0
Lam_on_curve = sp.simplify(Lam_vacuum.subs(A ** 2, A2_vacuum))
say(f"Lambda of the vacuum: {Lam_on_curve}")
```

A^2 on the vacuum curve; $\kappa\rho$ of Einstein-Gauss-Bonnet gravity; `sp.solve(rho_gb, Lam)` solves $\kappa\rho = 0$ for Λ and returns a list with one solution, of which `[0]` takes the first. Then A^2 is replaced by its value on the curve (sympy also replaces $A^4 = (A^2)^2$) and the result is simplified: $\Lambda = 720H^4\alpha_2 - 36H^2 + 3/(16\alpha_2)$, printed in Out [11].

```
p_on_curve = sp.simplify((kappa_p.subs(GB).subs(Lam, Lam_vacuum)).subs(A ** 2,
                                                                       A2_vacuum))
check(p_on_curve == 0,
      "Gauss-Bonnet vacuum: with V = 0 and Lambda from rho = 0, also p = 0")
at_edge = Lam_on_curve.subs({alpha2: sp.Rational(1, 40), H: 1})
report("vacuum at alpha2 H^2 = 1/40: Lambda/H^2", at_edge)
check(at_edge == sp.Rational(-21, 2), "at alpha2 H^2 = 1/40: A = 0, Lambda = -10.5 H^2")
```

With this Λ and on the curve the pressure vanishes as well: a true vacuum. At the edge $\alpha_2 H^2 = 1/40$ the cosmological constant is exactly $-21/2$ (in units H^2).

```
lam_vac_f = sp.lambdify(alpha2, Lam_on_curve.subs(H, 1), "numpy")
g_curve = np.linspace(0.004, 1 / 40, 200)
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(g_curve, np.sqrt((1 - 40 * g_curve) / (8 * g_curve)), label="$A > 0$")
left.plot(g_curve, -np.sqrt((1 - 40 * g_curve) / (8 * g_curve)), "--",
          label="$A < 0$")
left.set_xlabel("$\\alpha_2 H^2$")
left.set_ylabel("slope $A$ of the vacuum")
left.legend(fontsize=8)
right.plot(g_curve, lam_vac_f(g_curve))
right.set_xlabel("$\\alpha_2 H^2$")
right.set_ylabel("$\\Lambda/H^2$ of the vacuum")
fig.suptitle("Einstein-Gauss-Bonnet gravity: the vacuum linear members")
save_figure(fig, "gauss_bonnet_vacuum",
            ...)
```

The two branches $\pm A$ of the vacuum slope on the left and the vacuum's cosmological constant on the right, for $\alpha_2 H^2$ from 0.004 to 1/40. **What the figure shows.** The smaller the Gauss-Bonnet coupling, the faster the vacuum deflates (or inflates) the extra times; at 1/40 the vacuum is static with $\Lambda = -10.5H^2$.

In [12], figure 7: the third-order vacuum lines.

```
V_one = sp.expand(V.subs({alpha1: 1, H: 1}))  # V with alpha1 = 1, H = 1
alpha3_line = sp.solve(V_one, alpha3)[0]  # alpha3 H^4 on the vacuum line
say(f"vacuum line: alpha3 H^4 = {sp.factor(alpha3_line)}")
check(alpha3_line.subs({A: 1, alpha2: sp.Rational(1, 48)}) == 0,
      "A = 1, alpha2 H^2 = 1/48: the vacuum line passes through alpha3 = 0")
line_f = sp.lambdify((A, alpha2), alpha3_line, "numpy")
```

$V = 0$ is solved for α_3 (with $\alpha_1 = H = 1$): for fixed A the solution is linear in α_2, a straight line, printed in factored form by `sp.factor`. One point is checked: $A = 1$, $\alpha_2 H^2 = 1/48$ gives $\alpha_3 = 0$, because $V = 1 - 8 \cdot \frac{1}{48} \cdot 6 = 0$.

```
g_values = np.linspace(0.0, 0.08, 201)
fig, ax = plt.subplots()
for slope in (0, 1, 2, 3):
    label = "$A = 0$" if slope == 0 else f"$A = \\pm{slope}$"
    ax.plot(g_values, line_f(slope, g_values), label=label)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("$\\alpha_2 H^2$")
ax.set_ylabel("$\\alpha_3 H^4$")
ax.set_title("Third-order Lovelock gravity: the vacuum lines $V = 0$")
ax.legend(fontsize=8)
save_figure(fig, "third_order_vacua",
            ...)
```

The lines for $A = 0, 1, 2, 3$ (each serves $\pm A$). **What the figure shows.** Each line crosses the axis $\alpha_3 = 0$ at the Gauss-Bonnet vacuum of that slope; with a positive α_3 a vacuum exists at larger α_2.

In [13], a condensate of dirac16complex00 as the source.

```
m, lam, S = sp.symbols("m lam S", real=True)  # mass, coupling lambda, density S
rho_condensate = m * S + lam * S ** 2 / 2  # rho = m S + U, U = (lambda/2) S^2
p_condensate = lam * S ** 2 / 2  # p = S U' - U
```

The mass m, the self-coupling λ (named `lam`) and the density S of a condensate; its energy density $mS + U$ and pressure $SU' - U$ with $U = \frac{\lambda}{2}S^2$, $U' = \lambda S$, so $SU' - U = \lambda S^2 - \frac{\lambda}{2}S^2 = \frac{\lambda}{2}S^2$ (the record's formulas, Section 12.26).

```
condition_rho = sp.expand(kappa * rho_condensate - rho_e)  # = 0
condition_p = sp.expand(kappa * p_condensate - p_e)  # = 0
first = sp.expand(kappa * m * S + 36 * H ** 2 + 2 * Lam)  # kappa m S = -(36 H^2 + 2 L)
second = sp.expand(6 * (A ** 2 + 1) * H ** 2 + kappa * S * (m + lam * S))
reproduces(sp.expand(condition_rho - condition_p - first) == 0
           and sp.expand(condition_rho + condition_p - second) == 0,
           "condensate, Einstein: kappa m S = -(36 H^2 + 2 Lambda) and "
           "6 (A^2 + 1) H^2 = -kappa S (m + lambda S)",
           PY, "condensate_einstein_quadratic_U")
```

The two Einstein equations for this source are `condition_rho = 0` and `condition_p = 0`. Their difference is the first condition and their sum the second; the derivation is written out in Section 12.26.

```
UNIT = {H: 1, kappa: 1}
S_solution = sp.solve(first.subs(UNIT), S)[0]
A2_solution = sp.expand(sp.solve(second.subs(UNIT), A ** 2)[0].subs(S, S_solution))
say(f"S = {S_solution},  A^2 = {A2_solution}")
```

In units $H = \kappa = 1$ the first condition gives $S = -2(\Lambda + 18)/m$, and the second, solved for A^2 with this S, gives A^2 as a function of m, λ and Λ (printed in Out [13]); for $\Lambda = 0$ it is $5 - 216\lambda/m^2$.

```
example_1 = A2_solution.subs({m: 5, lam: 0, Lam: 0})
example_2 = A2_solution.subs({m: -15, lam: sp.Rational(25, 6), Lam: 0})
report("m = 5, lambda = 0, Lambda = 0: A^2", example_1)
report("m = -15, lambda = 25/6, Lambda = 0: A^2", example_2)
check(example_1 == 5 and example_2 == 1, "the two worked examples: A^2 = 5 and A^2 = 1")
```

Two worked examples: $m = 5$, $\lambda = 0$ gives $A^2 = 5$; $m = -15$, $\lambda = 25/6$ gives $A^2 = 5 - 216 \cdot \frac{25}{6}/225 = 5 - 4 = 1$, the slope of the canonical history.

```
effective = []  # the effective mass M = m + lambda S of each example
for mass, coupling in ((5, 0), (-15, sp.Rational(25, 6))):
    S_value = S_solution.subs({m: mass, Lam: 0})  # the density the example needs
    effective.append(mass + coupling * S_value)
    report(f"m = {mass}, lambda = {coupling}: S and M = m + lambda S",
           f"{S_value}, {effective[-1]}")
check(effective == [5, -5], "the examples need M = 5 (S < 0) and M = -5 (S > 0)")
```

For each example the density it needs ($S = -36/5$ and $12/5$) and the effective mass $M = m + \lambda S$ (5 and -5) are printed; `effective[-1]` is the last entry of the list.

In [14], figure 8: the off-diagonal conditions and the slope of a condensate.

```
condensate = record["fields"]["dirac16complex00"]
entries = condensate["offDiagonalKinetic"]
bilinears = sorted({term["bilinear"] for entry in entries for term in entry["terms"]})
report("off-diagonal kinetic components of the condensate", len(entries))
report("distinct three-gamma bilinears", len(bilinears))
reproduces(len(entries) == 42 and len(bilinears) == 15,
           "42 off-diagonal components, multiples of 15 three-gamma bilinears",
           WL, "condensate_offdiagonal_are_three_gamma_bilinears")
```

The record lists every nonzero off-diagonal kinetic component of a condensate, each with the bilinear it multiplies. A **set comprehension** (braces) keeps each bilinear once, `sorted` puts them in order: 42 components, 15 distinct three-gamma bilinears.

```
a2_f = sp.lambdify((m, lam), A2_solution.subs(Lam, 0), "numpy")
fig, ax = plt.subplots()
for mass, upper in ((5, 1.0), (-15, 6.0)):
    lam_values = np.linspace(0.0, upper, 200)
    ax.plot(lam_values, a2_f(mass, lam_values), label=f"$m = {mass}$")
ax.plot([0], [5], "o", color="black")
ax.plot([25 / 6], [1], "s", color="black")
```

A^2 as a numpy function of m and λ at $\Lambda = 0$, drawn against λ for $m = 5$ (up to $\lambda = 1$) and $m = -15$ (up to 6); a circle and a square mark the two worked examples.

```
arrow = {"arrowstyle": "->"}  # a thin arrow from the text to the point
ax.annotate("$m=5$, $\\lambda=0$: $A^2=5$", (0, 5), (0.5, 5.8), fontsize=8,
            arrowprops=arrow)
ax.annotate("$m=-15$, $\\lambda=25/6$: $A^2=1$", (25 / 6, 1), (3.6, 2.6),
            fontsize=8, arrowprops=arrow)
ax.set_ylim(-4.2, 6.6)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("self-coupling $\\lambda$ (units $H = \\kappa = 1$)")
ax.set_ylabel("$A^2$ required by the condensate")
ax.set_title("A dirac16complex00 condensate as the source, Einstein, $\\Lambda=0$")
ax.legend(fontsize=8)
save_figure(fig, "condensate_slope",
            ...)
```

`ax.annotate(text, point, text_position, ...)` writes a text with an arrow to a point. **What the figure shows.** Both lines fall linearly, $A^2 = 5 - 216\lambda/m^2$, the line of the light mass $m = 5$ much faster; a real slope exists only while $A^2 \geq 0$, that is $\lambda \leq 5m^2/216$.

In [15], the last check.

```
figure_names = ["einstein_source", "source_regions", "phantom_w",
                "gauss_bonnet_source", "null_energy_map", "gauss_bonnet_vacuum",
                "third_order_vacua", "condensate_slope"]
paths = [output_file(f"{FIGURE_FOLDER}/12b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths), "all eight figure files exist")
all_checks_passed()
```

`enumerate(figure_names, 1)` numbers the names from 1, which gives the eight file names; all must exist. The last line is ALL 24 CHECKS PASSED (notebook 12b): 4 checks in In [4], 2 in In [5], 4 in In [8], 4 in In [9], 2 in In [10], 2 in In [11], 1 in In [12], 3 in In [13], 1 in In [14] and 1 in In [15].

12.21 Solving the evolution equation for a prescribed stress

The idea. If the anisotropic stress $\Delta = p_3 - p_t$ is known as a function of time (or of the state), the evolution equation $a_4'' F(a_4') = \kappa\Delta$ is an **ordinary differential equation** for $a_4(x_4)$, and with the starting values $a_4(0)$ and $a_4'(0)$ it has one solution, which a computer can follow step by step. We prescribe the stress by hand: it is ASSUMED, chosen to see what the equations do. The Revision record constructs no state of either field that produces these stresses, and Section 12.22 recalls why the Kohn-Sham states of the repository are not admissible sources at all.

A first-order system. Write $v = a_4'$. Then

$$\frac{d}{dx_4} a_4 = v, \qquad \frac{d}{dx_4} v = \frac{\kappa\Delta}{F(v)}, \qquad \frac{d}{dx_4}(\kappa\rho) = -3v\,\kappa\Delta.$$

The first line is the definition of v, the second the evolution equation divided by F, the third the conservation law of Section 12.10. The third is not needed for a_4; Notebook 12c integrates it alongside, so that the energy density obtained from conservation can be compared with the one the constraint gives: a numerical test of the Bianchi identity.

The Runge-Kutta method of fourth order (RK4), from Chapter 2: for $dy/dx = f(x,y)$ one step of size h computes $k_1 = f(x,y)$, $k_2 = f(x+h/2, y+hk_1/2)$, $k_3 = f(x+h/2, y+hk_2/2)$, $k_4 = f(x+h, y+hk_3)$ and $y_{\text{new}} = y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$. Its error after a fixed time shrinks like h^4: halving the step divides it by about 16.

No stress. With $\Delta = 0$: $a_4'' = 0$, so $a_4' = AH$ is constant and $a_4 = AHx_4 + a_0$: the linear member.

A pulse of stress (Einstein gravity). We start on the canonical history ($a_4(0) = 0$, $a_4'(0) = H$; units $H = 1$) and prescribe $\kappa\Delta(x_4) = \kappa\Delta_0\, e^{-((x_4 - x_c)/w)^2}$ with centre $x_c = 3$, width $w = 0.5$ and height $\kappa\Delta_0 = 2/(w\sqrt{\pi})$. With $F = 2$:

$$a_4'(x_4) = 1 + \int_0^{x_4} a_4''(s)\, ds = 1 + \frac{\kappa\Delta_0}{2} \int_0^{x_4} e^{-((s-x_c)/w)^2}\, ds$$

(the fundamental theorem of calculus, then $a_4'' = \kappa\Delta/2$)

$$= 1 + \frac{\kappa\Delta_0\, w}{2} \int_{-x_c/w}^{(x_4 - x_c)/w} e^{-t^2}\, dt$$

(substitution $t = (s - x_c)/w$, $ds = w\, dt$; the limits $s = 0$ and $s = x_4$ become $t = -x_c/w$ and $t = (x_4 - x_c)/w$)

$$= 1 + \frac{\kappa\Delta_0\, w\sqrt{\pi}}{4} \Big(\operatorname{erf} \frac{x_4 - x_c}{w} + \operatorname{erf} \frac{x_c}{w} \Big)$$

(the **error function** $\operatorname{erf}(u) = \frac{2}{\sqrt{\pi}} \int_0^u e^{-t^2} dt$ gives $\int_{u_0}^{u_1} e^{-t^2} dt = \frac{\sqrt{\pi}}{2}(\operatorname{erf} u_1 - \operatorname{erf} u_0)$, and erf is odd, $\operatorname{erf}(-u) = -\operatorname{erf} u$)

$$= 1 + \tfrac{1}{2} \Big(\operatorname{erf} \frac{x_4 - x_c}{w} + \operatorname{erf} \frac{x_c}{w} \Big)$$

($\kappa\Delta_0\, w\sqrt{\pi}/4 = 2/4 = 1/2$). At $x_4 = 0$ the bracket is zero and $a_4' = 1$. Long after the pulse $\operatorname{erf}((x_4 - x_c)/w) \to 1$, and $\operatorname{erf}(x_c/w) = \operatorname{erf}(6)$ differs from 1 by less than 10^{-15}, so $a_4' \to 2$: the pulse moves the history from the linear member $A = 1$ to $A = 2$, and the extra times deflate at every time, only faster afterwards. Integrating once more with $\int \operatorname{erf}(t)\, dt = t \operatorname{erf}(t) + e^{-t^2}/\sqrt{\pi}$ gives a_4 exactly (Notebook 12c, In [7]).

A relaxing stress. Now the stress depends on the state: $\kappa\Delta = -2\eta\,(a_4' - H)$ with a constant $\eta > 0$, starting on the faster member $a_4'(0) = 2H$ (units $H = 1$). Then

$$a_4'' = \frac{\kappa\Delta}{2} = -\eta\,(a_4' - 1)$$

(Einstein, $F = 2$). The excess $u = a_4' - 1$ obeys $u' = a_4'' = -\eta u$ with $u(0) = 1$, whose solution is $u = e^{-\eta x_4}$ (the derivative of $e^{-\eta x_4}$ is $-\eta e^{-\eta x_4}$). So

$$a_4' = 1 + e^{-\eta x_4}, \qquad a_4 = x_4 + \frac{1 - e^{-\eta x_4}}{\eta}$$

(integrate from 0, with $a_4(0) = 0$). The rate relaxes from $2H$ to H; the extra times keep deflating, first at the rate $2H$, later at the rate H. The conservation law gives $\kappa\rho' = -3a_4'\kappa\Delta = 6\eta\, a_4'(a_4' - 1) > 0$: the energy density rises. With $\Lambda = 0$ (the value Notebook 12c uses) the constraint gives $\kappa\rho = -3(a_4')^2 - 21H^2$, so $\kappa\rho$ rises from $-33H^2$ (at the rate $2H$, the value of the linear member $A = 2$) towards $-24H^2$ (the value of $A = 1$).

A constant stress in Einstein-Gauss-Bonnet gravity: the equation breaks down. With $\alpha_1 = 1$, $\alpha_3 = 0$ and $H = 1$, $F(v) = 2 - 80\alpha_2 - 48\alpha_2 v^2$. Define $G(v) = (2 - 80\alpha_2)v - 16\alpha_2 v^3$; its derivative is $G'(v) = 2 - 80\alpha_2 - 48\alpha_2 v^2 = F(v)$. Then, along a solution with a constant stress $\kappa\Delta$,

$$\frac{d}{dx_4} G(a_4'(x_4)) = G'(a_4')\, a_4'' = F(a_4')\, a_4'' = \kappa\Delta$$

(chain rule, then the evolution equation), and integrating from 0:

$$G(a_4'(x_4)) = G(a_4'(0)) + \kappa\Delta\, x_4.$$

The right-hand side grows without limit, but G has a largest value at the **critical rate** $v_c = \sqrt{(2 - 80\alpha_2)/(48\alpha_2)}$, where $G' = F = 0$. So the rate reaches v_c at the finite time

$$x_4^\star = \frac{G(v_c) - G(a_4'(0))}{\kappa\Delta},$$

and there $a_4'' = \kappa\Delta/F$ becomes infinite: beyond $x_4^\star$ no solution with a smooth a_4' exists. For $\kappa\Delta = 0.5$ and $a_4'(0) = 1$: $\alpha_2 = 0.005$ gives $v_c = 2.5820$ and $x_4^\star = 2.4682$; $\alpha_2 = 0.01$ gives $v_c = 1.5811$ and $x_4^\star = 0.4498$ (Notebook 12c, In [13]; Exercise 8 repeats the second by hand). In Einstein gravity ($F = 2$) the same stress simply makes the rate grow linearly, $a_4' = 1 + 0.25x_4$, for ever.

12.22 Example: four prescribed histories integrated with RK4

The third notebook programs RK4, tests it, and integrates the evolution equation for the four prescribed stresses of Section 12.21. It compares every numerical history with its exact solution, measures the order of convergence of RK4, compares the energy density from the conservation law with the one from the constraint, and finds the Gauss-Bonnet breakdown at the predicted time. Along every history with the deflating sign the extra times deflate exponentially at every time. At the end it reads the record's verdict on the Kohn-Sham states and recomputes the record's numbers. The record `Revision/field_equations_a4/reports/ks-source-conditions.json` (every one of its checks PASS; Notebook 12c, In [15]) finds that every recorded Kohn-Sham state with a nonzero energy-momentum tensor depends on x_0 and violates $p_3 + p_t = 2p_0$, even after integration over x_0 (check `ks_integrals_violate_algebraic_condition`). Notebook 12c, In [16], recomputes the record's numbers from the Kohn-Sham table `Revision/kohn_sham/results/ground/emt-integrals.csv` and asserts that they equal the record: of the 75 recorded states, 70 have a nonzero energy-momentum tensor; for these the ratio $(\int p_3 + \int p_t)/(2\int p_0)$, which equals 1 for every admissible source, is nowhere closer to 1 than 0.414328, and for $N = 136$ particles and $\lambda = 0$ it is 0.339767, 0.25969 and 0.239714 at the slices $a_{4,0} = 0, 1, 2$ (COMPUTED from the record's table, equal to the record). So the Kohn-Sham history $a_4 = AHx_4$ of Chapters 14 and 15 is a

PRESCRIBED background, not a solution of these equations with the Kohn-Sham source (Chapter 17). The notebook runs in about 15 seconds, prints 30 PASS lines and draws six figures.

12.23 How to run Notebook 12c

Step 1. What this notebook does and what it needs.

Notebook 12c (Integrating the evolution equation of a4 for prescribed sources) is the file `Revision /textbook/notebooks/12c_a4_evolution.ipynb` of the repository Dirac_claude. It integrates the evolution equation of the author's metric, in which the second derivative of a4 times the factor F equals kappa (p3 - p_t), with the fourth-order Runge-Kutta method for prescribed anisotropic stresses p3 - p_t (zero, a pulse that raises the deflation rate, a stress that relaxes it, a constant stress in Einstein-Gauss-Bonnet gravity), with the extra times deflating along every history, compares every history a4(x4) with its exact solution, measures the convergence order, computes the energy density and the pressures that each history requires, checks numerically that the constraint and the conservation law hold along the solutions, and draws six teaching plots of a4, its rate and the scale factors of 3-space and of the deflating extra times. At the end it reads the record on the Kohn-Sham states and recomputes its numbers from the table of their integrated energy-momentum tensors. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 12c_a4_evolution.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12c_a4_evolution.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 12c_a4_evolution.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/12c.captions.json`
- `Revision/textbook/figures/12c_1_linear_member.png`
- `Revision/textbook/figures/12c_2_stress_pulse.png`
- `Revision/textbook/figures/12c_3_required_source.png`
- `Revision/textbook/figures/12c_4_damped_deflation.png`
- `Revision/textbook/figures/12c_5_rk4_convergence.png`
- `Revision/textbook/figures/12c_6_gauss_bonnet_breakdown.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 30 CHECKS PASSED (notebook 12c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/12c_a4_evolution.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 12c_a4_evolution.ipynb
python -m nbconvert --execute --inplace 12c_a4_evolution.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a4-equations.json: the notebook reads the Revision record of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.
- "RuntimeWarning: overflow" or "invalid value" below a cell after you changed a number: a changed coupling or stress drove a4' past the point where F vanishes; the notebook as distributed stops every integration before that point. Undo the change or lower the stress.

12.24 Notebook 12c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/12c_a4_evolution.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The field equations for $a_4(x_4)$ contain one equation with the second derivative
$a_4''$, the **evolution equation**

$$a_4''\,F(a_4') = \kappa\,(p_3 - p_t),$$

where $p_3 - p_t$ is the difference between the pressure along 3-space and the
pressure along the extra times (the **anisotropic stress**). If $p_3 - p_t$ is
given as a function of the time $x_4$, this is an ordinary differential equation
for $a_4$, and we can solve it step by step on the computer. This notebook

- takes $F$ and the other Lovelock components from the Revision record;
- writes a fourth-order Runge-Kutta solver (RK4) and tests it;
- integrates the equation for four **prescribed** stresses: zero (the linear
  member), a short pulse that raises the deflation rate of the extra times from
  $a_4' = H$ to $a_4' = 2H$, a stress that relaxes the rate from $2H$ back to $H$,
  and a constant stress in Einstein-Gauss-Bonnet gravity, where the equation
  breaks down at a finite time; along every one of these histories the extra
  times deflate exponentially at all times ($a_4' > 0$);
- compares each numerical history with its exact solution and measures the
  convergence order of RK4;
- computes the energy density and the pressures that each history requires, and
  checks that the constraint and the conservation law hold along the solution;
- draws six plots of $a_4$, its rate $a_4'$, and the scale factors $e^{a_4}$ of
  3-space and $e^{-a_4}$ of the deflating extra times.
```

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 12c (Integrating the evolution equation of a4 for prescribed sources) is the file ‘Revision/textbook/notebooks/12c_a4_evolution.ipynb‘ of the repository Dirac_claude. It integrates the evolution equation of the author’s metric, in which the second derivative of a4 times the factor F equals kappa (p3 - p_t), with the fourth-order Runge-Kutta method for prescribed anisotropic stresses p3 - p_t (zero, a pulse that raises the deflation rate, a stress that relaxes it, a constant stress in Einstein-Gauss-Bonnet gravity), with the extra times deflating along every history, compares every history a4(x4) with its exact solution, measures the convergence order, computes the energy density and the pressures that each history requires, checks numerically that the constraint and the conservation law hold along the solutions, and draws six teaching plots of a4, its rate and the scale factors of 3-space and of the deflating extra times. At the end it reads the record on the Kohn-Sham states and recomputes its numbers from the table of their integrated energy-momentum tensors. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a

higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
```

```
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 12c_a4_evolution.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut

Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12c_a4_evolution.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 12c_a4_evolution.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/12c.captions.json‘
- ‘Revision/textbook/figures/12c_1_linear_member.png‘
- ‘Revision/textbook/figures/12c_2_stress_pulse.png‘
- ‘Revision/textbook/figures/12c_3_required_source.png‘
- ‘Revision/textbook/figures/12c_4_damped_deflation.png‘
- ‘Revision/textbook/figures/12c_5_rk4_convergence.png‘
- ‘Revision/textbook/figures/12c_6_gauss_bonnet_breakdown.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 30 CHECKS PASSED (notebook 12c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel

```
  called python3; with the environment active, run the following command and start
  JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from
  a copy outside the repository. Open the file
  ‘Revision/textbook/notebooks/12c_a4_evolution.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
  ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
  "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
  install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
  active; do Step 4. If the environment is active and the message stays, the programs
  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 12c_a4_evolution.ipynb
      python -m nbconvert --execute --inplace 12c_a4_evolution.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a4-equations.json: the notebook reads the Revision record of
  the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy
  of the repository made with git clone, not on a copy of the notebook file alone.
- "RuntimeWarning: overflow" or "invalid value" below a cell after you changed a number:
  a changed coupling or stress drove a4’ past the point where F vanishes; the notebook as
  distributed stops every integration before that point. Undo the change or lower the
  stress.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 12c (the complete instructions)
#
```

```
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 12c (Integrating the evolution equation of a4 for prescribed sources) is the
# file Revision/textbook/notebooks/12c_a4_evolution.ipynb of the repository Dirac_claude.
# It integrates the evolution equation of the author's metric, in which the second
# derivative of a4 times the factor F equals kappa (p3 - p_t), with the fourth-order
# Runge-Kutta method for prescribed anisotropic stresses p3 - p_t (zero, a pulse that
# raises the deflation rate, a stress that relaxes it, a constant stress in
# Einstein-Gauss-Bonnet gravity), with the extra times deflating along every history,
# compares every history a4(x4) with its exact solution, measures the convergence order,
# computes the energy density and the pressures that each history requires, checks
# numerically that the constraint and the conservation law hold along the solutions, and
# draws six teaching plots of a4, its rate and the scale factors of 3-space and of the
# deflating extra times. At the end it reads the record on the Kohn-Sham states and
# recomputes its numbers from the table of their integrated energy-momentum tensors. It
# needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
```

```
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
```

```
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 12c_a4_evolution.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 12c_a4_evolution.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
```

```
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 12c_a4_evolution.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/12c.captions.json
# - Revision/textbook/figures/12c_1_linear_member.png
# - Revision/textbook/figures/12c_2_stress_pulse.png
# - Revision/textbook/figures/12c_3_required_source.png
# - Revision/textbook/figures/12c_4_damped_deflation.png
# - Revision/textbook/figures/12c_5_rk4_convergence.png
# - Revision/textbook/figures/12c_6_gauss_bonnet_breakdown.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 30 CHECKS PASSED (notebook 12c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/12c_a4_evolution.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
```

```
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 12c_a4_evolution.ipynb
#     python -m nbconvert --execute --inplace 12c_a4_evolution.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a4-equations.json: the notebook reads the Revision record
#   of the repository. Run it inside the folder Revision/textbook/notebooks of a complete
#   copy of the repository made with git clone, not on a copy of the notebook file alone.
# - "RuntimeWarning: overflow" or "invalid value" below a cell after you changed a
#   number: a changed coupling or stress drove a4' past the point where F vanishes; the
#   notebook as distributed stops every integration before that point. Undo the change or
#   lower the stress.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "12c"  # this notebook: chapter 12, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
```

```
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
```

```
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 12c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Ordinary differential equation (ODE)**: an equation for an unknown function of
  one variable (here $a_4(x_4)$) that contains its derivatives.
- **Initial value problem**: an ODE together with the starting values, here
  $a_4(0) = 0$ and the starting rate $a_4'(0)$.
- **First-order system**: the second-order equation written as two first-order
```

```
  ones, $\frac{d}{dx_4}a_4 = v$ and $\frac{d}{dx_4}v = \kappa(p_3 - p_t)/F(v)$,
  with $v = a_4'$.
- **RK4** (Runge-Kutta of fourth order): a rule that advances the solution by one
  step $h$ using four evaluations of the right-hand side; its error shrinks like
  $h^4$.
- **Convergence order**: the power $q$ in "error $\approx$ constant $\times h^q$";
  halving $h$ divides the error by $2^q$.
- **Anisotropic stress** $\Delta = p_3 - p_t$: the difference of the pressures along
  3-space and along the extra times.
- **Prescribed source**: a source that we choose by hand to see what the equations
  do (status ASSUMED); it is not derived from any field of the theory.
- **Scale factor**: the factor by which lengths along a direction grow:
  $e^{a_4}$ for 3-space and $e^{-a_4}$ for the extra times $x_5, x_6, x_7$ (times
  the common factor $\sin^{1/6} z$, which does not change with $x_4$).
- **Constraint**: the time component of the field equations, which fixes the
  energy density $\rho$ from $a_4'$; **conservation**: $\rho' = -3a_4'(p_3 - p_t)$.
- **Deflation rate**: the rate $a_4'$. While $a_4' > 0$ the scale factor
  $e^{-a_4}$ of the extra times shrinks exponentially, at the momentary rate $a_4'$.
- **Canonical history**: the author's metric leaves the function $a_4(x_4)$ free;
  the author requires only that the extra times deflate ($a_4' > 0$). The linear
  member with the constant rate $a_4' = AH$ and $A = 1$ is the canonical history
  of the Revision record (ASSUMED, a prescribed choice); this notebook also uses
  the faster member $A = 2$.
- **Breakdown**: a point where $F(a_4') = 0$, so that the evolution equation can no
  longer be solved for $a_4''$.
- **Units**: $H = 1$ and $\kappa = 1$: the time $x_4$ is measured in units of
  $1/H$, $a_4'$ in units of $H$, $\kappa\rho$ and the pressures in units of $H^2$.
```

Text cell:

```
## 4. The physical and mathematical situation

For a source that does not depend on $x_0$ and has no mixed components, the
field equations of the author's metric are (Revision record `a4-equations.json`)

$$\kappa\rho = -\Big(\sum_k\alpha_k E_{(k)}{}^{x_4}{}_{x_4}(a_4') + \Lambda\Big),
\qquad \kappa p_0 = \sum_k\alpha_k E_{(k)}{}^{x_0}{}_{x_0}(a_4') + \Lambda,$$
$$a_4''\,F(a_4') = \kappa(p_3 - p_t),\qquad p_3 + p_t = 2p_0.$$

We choose (prescribe) the anisotropic stress $\Delta(x_4) = p_3 - p_t$ and the
starting values $a_4(0)$, $a_4'(0)$. The evolution equation then determines
$a_4(x_4)$; the first equation gives the energy density, the second the hidden
pressure, and $p_3 = p_0 + \Delta/2$, $p_t = p_0 - \Delta/2$. The record proves
that the derivative of the constraint is $3a_4'$ times the evolution equation; so
along a solution the energy density must obey $\rho' = -3a_4'\Delta$, which this
notebook checks numerically.

**Honesty.** Every stress used here is a mathematical test input (ASSUMED). The
Revision record constructs no state of either field that produces it, and it shows
that the Kohn-Sham states of the repository are not admissible sources (section 13
of this notebook reads that record). The histories below show what the equations
do, not what the universe did. In all of them the three extra times $x_5, x_6,
x_7$ deflate exponentially at every time (the rate $a_4'$ stays positive); the
```

```
only exception is the history with $A = -1$ in section 7, the mirror image that the
equations allow, drawn only to show that symmetry.
```

Text cell:

```
## 5. The Revision records and the Lovelock components

The next cell defines the helpers that read the Revision records (as in every
notebook of this chapter; ‘reproduces‘ prints its PASS and reproduces lines with
one call, through a text buffer, so that they stay together) and turns three
entries of ‘a4-equations.json‘ into Python functions of the rate $v = a_4’$ and
the couplings, with $H = 1$:
‘rho_side(v, a1, a2, a3)‘ $= \sum_k\alpha_kE_{(k)}{}^{x_4}{}_{x_4}$,
‘p0_side(v, a1, a2, a3)‘ $= \sum_k\alpha_kE_{(k)}{}^{x_0}{}_{x_0}$ and
‘F_of(v, a1, a2, a3)‘ $= F$. ‘sp.lambdify‘ turns a sympy formula into a fast
numpy function.
```

In [2]:

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory
import math  # the error function erf, for an exact solution

import matplotlib.ticker  # control of the tick labels of an axis
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra, used here only to read the record formulas

PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the equations
KS = "Revision/field_equations_a4/reports/ks-source-conditions.json"


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict of the check name in a report ("PASS"), None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS.  Its
    printed lines (PASS and reproduces) are collected in a text buffer and printed
    by one print call, so that they always stay together in the cell’s output."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
```

```
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name, record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # all lines at once


record = read_json(EQUATIONS)
H, ad1, ad2 = sp.symbols("H ad1 ad2", real=True)
a1, a2, a3 = sp.symbols("alpha1 alpha2 alpha3", real=True)
LOCALS = {"H": H, "ad1": ad1, "ad2": ad2, "alpha1": a1, "alpha2": a2, "alpha3": a3}


def from_record(text):
    """A Wolfram InputForm text of the record as a sympy expression with H = 1."""
    return sp.sympify(text.replace("^", "**"), locals=LOCALS).subs(H, 1)


lovelock = record["lovelockTensors"]
weights = {1: a1, 2: a2, 3: a3}


def lovelock_sum(component):
    """sum_k alpha_k E_(k) of one component ("x4x4" or "x0x0") of the record."""
    return sum(weights[k] * from_record(lovelock[f"E{k}"][component]["input"])
               for k in (1, 2, 3))


e44 = lovelock_sum("x4x4")  # the time component (constraint)
e00 = lovelock_sum("x0x0")  # the hidden component
F_expr = from_record(record["generalSource"]["evolution_F"]["input"])
rho_side = sp.lambdify((ad1, a1, a2, a3), e44, "numpy")
p0_side = sp.lambdify((ad1, a1, a2, a3), e00, "numpy")
F_of = sp.lambdify((ad1, a1, a2, a3), F_expr, "numpy")
say(f"F(a4') with H = 1: {F_expr}")
EINSTEIN = (1.0, 0.0, 0.0)  # alpha1, alpha2, alpha3 of Einstein gravity
```

Out [2]:

```
F(a4') with H = 1: 720*ad1**4*alpha3 - 48*ad1**2*alpha2 + 864*ad1**2*alpha3 + 2*alpha1 -
    80*alpha2 + 720*alpha3
```

Text cell:

```
The next cell checks that for Einstein gravity these functions are what the record
says: $F = 2$ (the evolution equation is $2a_4'' = \kappa(p_3 - p_t)$),
$E^{x_4}{}_{x_4} = 3(a_4')^2 + 21$ and $E^{x_0}{}_{x_0} = 15 - 3(a_4')^2$ (with
$H = 1$), at five rates $a_4'$.
```

In [3]:

```
rates = np.array([-2.0, -0.5, 0.0, 1.0, 3.0])  # a few values of a4'
ok = (np.allclose(F_of(rates, *EINSTEIN), 2.0)
```

```
      and np.allclose(rho_side(rates, *EINSTEIN), 3 * rates ** 2 + 21)
      and np.allclose(p0_side(rates, *EINSTEIN), 15 - 3 * rates ** 2))
reproduces(ok, "Einstein: F = 2, E44 = 3 a4'^2 + 21, E88 = 15 - 3 a4'^2 (H = 1)",
           PY, "einstein_components")
```

Out [3]:

```
PASS Einstein: F = 2, E44 = 3 a4'^2 + 21, E88 = 15 - 3 a4'^2 (H = 1)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_components
```

Text cell:

```
## 6. The Runge-Kutta method of fourth order

For a system $\frac{dy}{dx} = f(x, y)$ (here $y$ is a list of numbers) one RK4 step
of size $h$ from $(x, y)$ computes

$$k_1 = f(x, y),\quad k_2 = f(x + \tfrac h2, y + \tfrac h2 k_1),\quad
k_3 = f(x + \tfrac h2, y + \tfrac h2 k_2),\quad k_4 = f(x + h, y + hk_3),$$
$$y_{\text{new}} = y + \tfrac h6(k_1 + 2k_2 + 2k_3 + k_4).$$

The function ‘rk4‘ below repeats this step. Its optional argument ‘stop‘ is a
function of $y$; when it returns True the integration ends (used in section 12 to
stop before $F$ reaches zero). The test integrates $y' = -y$, $y(0) = 1$, from 0
to 1 with 20 steps; the exact value is $y(1) = e^{-1}$, and the RK4 error must be
tiny (below $10^{-7}$).
```

In [4]:

```
def rk4(rhs, y0, h, steps, stop=None):
    """Solve dy/dx = rhs(x, y), y(0) = y0, with at most steps RK4 steps of size h.
    Return the arrays x (one value per step) and y (one row per step)."""
    xs = [0.0]
    ys = [np.array(y0, dtype=float)]
    for n in range(steps):
        x, y = xs[-1], ys[-1]
        k1 = rhs(x, y)
        k2 = rhs(x + h / 2, y + h / 2 * k1)
        k3 = rhs(x + h / 2, y + h / 2 * k2)
        k4 = rhs(x + h, y + h * k3)
        xs.append((n + 1) * h)  # (n + 1) h avoids adding up rounding errors
        ys.append(y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4))
        if stop is not None and stop(ys[-1]):
            break
    return np.array(xs), np.array(ys)


x_test, y_test = rk4(lambda x, y: -y, [1.0], 0.05, 20)
error_test = abs(y_test[-1, 0] - math.exp(-1.0))
say(f"y' = -y: RK4 with h = 0.05 gives y(1) = {y_test[-1, 0]:.10f}; "
```

```
    f"exact {math.exp(-1.0):.10f}")
check(error_test < 1e-7, "RK4 solves y' = -y to better than 1e-7 with 20 steps")
```

Out [4]:

```
y' = -y: RK4 with h = 0.05 gives y(1) = 0.3678794611; exact 0.3678794412
PASS RK4 solves y' = -y to better than 1e-7 with 20 steps
```

Text cell:

```
## 7. No anisotropic stress: the linear member

The state is $y = (a_4, a_4', \kappa\rho)$. The right-hand side is

$$\frac{d}{dx_4}a_4 = a_4',\qquad \frac{d}{dx_4}a_4' = \frac{\kappa\Delta(x_4)}
{F(a_4')},\qquad \frac{d}{dx_4}(\kappa\rho) = -3a_4'\,\kappa\Delta(x_4).$$

The third line is the conservation law; we integrate it alongside so that we can
later compare the energy density it gives with the one the constraint gives. The
helper `history` builds the right-hand side for a stress function and the
couplings, starts with $\kappa\rho$ from the constraint, and integrates.

With $\Delta = 0$ the equation is $a_4'' = 0$, so $a_4 = a_4'(0)\,x_4$: the linear
member $a_4 = AHx_4$ with $A = a_4'(0)/H$. The next cell integrates it for the
deflating rates $A = 1$ and $A = 2$ and, to show the symmetry $A \to -A$ of the
equations, for the mirror image $A = -1$ (inflating extra times; a departure
from the author's model, drawn only for the comparison), up to $x_4 = 3$, and
compares with the exact straight lines.
```

In [5]:

```
def history(stress, rate0, h, steps, couplings=EINSTEIN, Lam=0.0, stop=None):
    """Integrate (a4, a4', kappa rho) for the prescribed stress kappa Delta(x4, y),
    starting at a4 = 0, a4' = rate0 and kappa rho from the constraint."""
    def rhs(x, y):
        push = stress(x, y)  # kappa (p3 - pt) at this time
        return np.array([y[1], push / F_of(y[1], *couplings), -3.0 * y[1] * push])

    rho0 = -(rho_side(rate0, *couplings) + Lam)  # kappa rho at x4 = 0
    return rk4(rhs, [0.0, rate0, rho0], h, steps, stop)


def no_stress(x, y):
    return 0.0  # Delta = 0: an isotropic source


linear = {}  # slope A -> (x4, solution)
for slope in (2.0, 1.0, -1.0):  # deflating A = 2, 1 and the mirror image A = -1
    linear[slope] = history(no_stress, slope, 0.01, 300)
    x4, y = linear[slope]
    error = np.max(np.abs(y[:, 0] - slope * x4))  # compare with a4 = A x4
```

```
    check(error < 1e-12, f"Delta = 0, A = {slope:g}: a4 = A H x4 exactly")
x4, y = linear[1.0]
report("A = 1, Lambda = 0: kappa rho along the history", f"{y[-1, 2]:.6f}", "H^2")
product = np.exp(3 * y[:, 0]) * np.exp(-3 * y[:, 0])  # e^(3 a4) e^(-3 a4)
check(np.max(np.abs(product - 1.0)) < 1e-12,
      "the 7-volume factor e^(3 a4) e^(-3 a4) stays 1")
```

Out [5]:

```
PASS Delta = 0, A = 2: a4 = A H x4 exactly
PASS Delta = 0, A = 1: a4 = A H x4 exactly
PASS Delta = 0, A = -1: a4 = A H x4 exactly
RESULT A = 1, Lambda = 0: kappa rho along the history = -24.000000 H^2
PASS the 7-volume factor e^(3 a4) e^(-3 a4) stays 1
```

Text cell:

```
The next cell draws the three linear histories and, for $A = 1$, the scale factor
$e^{a_4}$ of 3-space and $e^{-a_4}$ of the extra times on a logarithmic vertical
axis (on which an exponential is a straight line). The product of their cubes,
$e^{3a_4}e^{-3a_4} = 1$, is the only way $a_4$ enters the volume of the seven
directions $x_1, x_2, x_3, x_5, x_6, x_7, x_0$ (the 7-volume; the hidden
direction's factor does not depend on $a_4$), and it stays constant: 3-space gains
exactly what the extra times lose.
```

In [6]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
for slope, style, words in ((2.0, "-", "deflating"), (1.0, "-.", "deflating"),
                            (-1.0, "--", "mirror image")):
    x4, y = linear[slope]
    left.plot(x4, y[:, 0], style, label=f"$A = {slope:g}$ ({words})")
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("$a_4$")
left.set_title("$\\Delta = 0$: $a_4 = AHx_4$")
left.legend(fontsize=8)
x4, y = linear[1.0]
right.semilogy(x4, np.exp(y[:, 0]), label="3-space $e^{a_4}$ (inflates)")
right.semilogy(x4, np.exp(-y[:, 0]), "--", label="extra times $e^{-a_4}$ (deflate)")
right.semilogy(x4, np.exp(3 * y[:, 0]) * np.exp(-3 * y[:, 0]), ":", color="black",
               label="$e^{3a_4}e^{-3a_4} = 1$")
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("scale factor")
right.set_title("$A = 1$: the scale factors")
right.legend(fontsize=8)
save_figure(fig, "linear_member",
            "Without anisotropic stress ($p_3 = p_t$) the evolution equation gives "
            "$a_4^{\\prime\\prime} = 0$. Left: the numerical histories $a_4(x_4)$ "
            "for the deflating rates $A = 2$ (solid) and $A = 1$ (dash-dotted) and "
            "for the mirror image $A = -1$ (dashed; inflating extra times, which "
            "the equations allow equally), straight lines "
```

```
"$a_4 = AHx_4$; horizontal axis the time $x_4$ in units of $1/H$. Right: "
"for $A = 1$ the scale factor $e^{a_4}$ of 3-space (inflating) and "
"$e^{-a_4}$ of the extra times (deflating exponentially) on a "
"logarithmic axis, and the constant product $e^{3a_4}e^{-3a_4} = 1$: "
"what 3-space gains, the extra times lose.")
```

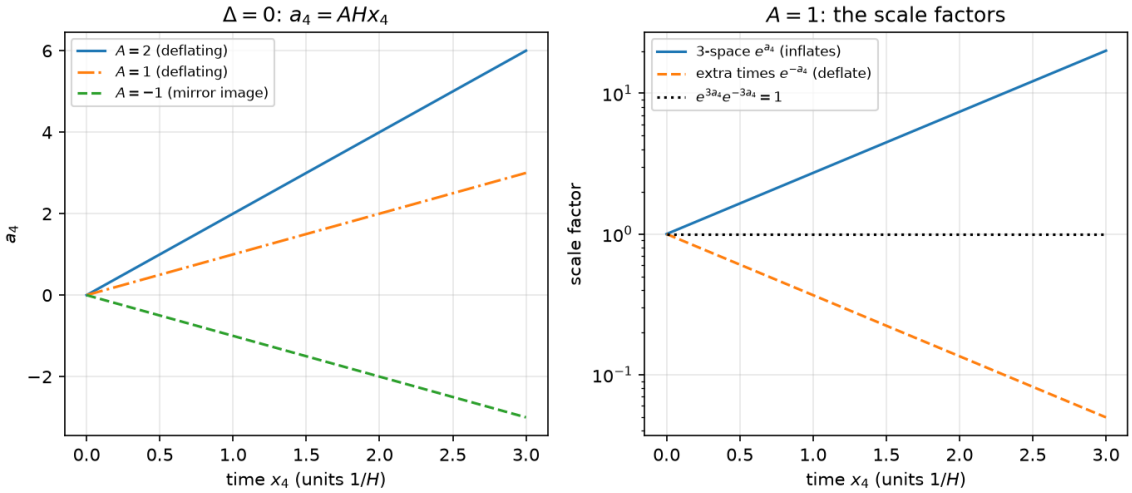

Figure 332: Without anisotropic stress ($p_3 = p_t$) the evolution equation gives $a_4'' = 0$. Left: the numerical histories $a_4(x_4)$ for the deflating rates $A = 2$ (solid) and $A = 1$ (dash-dotted) and for the mirror image $A = -1$ (dashed; inflating extra times, which the equations allow equally), straight lines $a_4 = AHx_4$; horizontal axis the time x_4 in units of $1/H$. Right: for $A = 1$ the scale factor e^{a_4} of 3-space (inflating) and e^{-a_4} of the extra times (deflating exponentially) on a logarithmic axis, and the constant product $e^{3a_4}e^{-3a_4} = 1$: what 3-space gains, the extra times lose. (Notebook 12c, figure 1.)

Out [6]:

```
Figure 12c.1 saved as Revision/textbook/figures/12c_1_linear_member.png
```

Text cell:

```
## 8. A pulse of anisotropic stress raises the deflation rate

Now we start on the canonical history, the deflating linear member with $A = 1$
($a_4(0) = 0$, $a_4'(0) = H$), and prescribe a short pulse of stress around the
time $x_c = 3$ with width $w = 0.5$:

$$\kappa\Delta(x_4) = \kappa\Delta_0\,e^{-((x_4 - x_c)/w)^2},\qquad
\kappa\Delta_0 = \frac{2}{w\sqrt\pi}.$$

In Einstein gravity $a_4'' = \kappa\Delta/2$. Integrating once, with
$\int e^{-t^2}dt = \tfrac{\sqrt\pi}{2}\operatorname{erf}(t)$ (erf is the *error
function*; it rises from $-1$ to $1$), and with $H = 1$:
```

```
$$a_4'(x_4) = 1 + \frac{\kappa\Delta_0 w\sqrt\pi}{4}\Big(\operatorname{erf}
\frac{x_4 - x_c}{w} + \operatorname{erf}\frac{x_c}{w}\Big)
= 1 + \frac12\Big(\operatorname{erf}\frac{x_4 - x_c}{w}
+ \operatorname{erf}\frac{x_c}{w}\Big),$$

which rises from 1 to almost exactly 2: after the pulse the history is the
deflating linear member with $A = 2$; the extra times deflate at every time, only
faster after the pulse. Integrating again, with
$\int\operatorname{erf}(t)\,dt = t\operatorname{erf}(t) + e^{-t^2}/\sqrt\pi$, gives
$a_4(x_4)$ exactly. The next cell integrates numerically up to $x_4 = 8$ with
$h = 0.01$ and compares both.
```

In [7]:

```
X_C, WIDTH = 3.0, 0.5  # the centre and the width of the pulse
PUSH = 2.0 / (WIDTH * math.sqrt(math.pi))  # kappa Delta_0
RATE_START = 1.0  # a4'(0)/H: the canonical deflating history A = 1


def pulse(x, y):
    return PUSH * math.exp(-((x - X_C) / WIDTH) ** 2)  # kappa Delta(x4)


def exact_pulse(x):
    """The exact a4 and a4' of the pulse history (Einstein, a4(0) = 0,
    a4'(0) = 1)."""
    scale = PUSH * WIDTH * math.sqrt(math.pi) / 4  # = 1/2
    t, t0 = (x - X_C) / WIDTH, -X_C / WIDTH  # the argument now and at x4 = 0

    def antiderivative(s):  # an antiderivative of erf
        return s * math.erf(s) + math.exp(-s * s) / math.sqrt(math.pi)

    rate = RATE_START + scale * (math.erf(t) + math.erf(X_C / WIDTH))
    a4 = RATE_START * x + scale * (WIDTH * (antiderivative(t) - antiderivative(t0))
                                   + x * math.erf(X_C / WIDTH))
    return a4, rate


x4_pulse, y_pulse = history(pulse, RATE_START, 0.01, 800)
exact = np.array([exact_pulse(x) for x in x4_pulse])  # columns: a4, a4'
error_a4 = np.max(np.abs(y_pulse[:, 0] - exact[:, 0]))
error_rate = np.max(np.abs(y_pulse[:, 1] - exact[:, 1]))
say(f"largest error: a4 {error_a4:.0e}, a4' {error_rate:.0e}")
check(error_a4 < 1e-8 and error_rate < 1e-8,
      "pulse: RK4 (h = 0.01) agrees with the exact solution to 1e-8")
check(np.min(y_pulse[:, 1]) >= 1.0 - 1e-12,
      "the rate a4' never falls below H: the extra times deflate at every time")
report("rate a4'/H after the pulse (x4 = 8)", f"{y_pulse[-1, 1]:.10f}")
check(abs(y_pulse[-1, 1] - 2.0) < 1e-8,
      "after the pulse a4' = 2 H: the deflating linear member A = 2")
```

Out [7]:

```
largest error: a4 9e-11, a4' 1e-10
PASS pulse: RK4 (h = 0.01) agrees with the exact solution to 1e-8
PASS the rate a4' never falls below H: the extra times deflate at every time
RESULT rate a4'/H after the pulse (x4 = 8) = 2.0000000000
PASS after the pulse a4' = 2 H: the deflating linear member A = 2
```

Text cell:

```
The next cell draws the pulse history in four panels: the prescribed stress; the
rate $a_4'$ (numerical line and exact points); $a_4$ itself; and the two scale
factors on a logarithmic axis.
```

In [8]:

```
fig, axes = plt.subplots(2, 2, figsize=(10.0, 7.0))
stress_values = np.array([pulse(x, None) for x in x4_pulse])
axes[0, 0].plot(x4_pulse, stress_values)
axes[0, 0].set_ylabel("$\\kappa(p_3 - p_t)$ (units $H^2$)")
axes[0, 0].set_title("the prescribed stress pulse")
every = slice(0, None, 40)  # every 40th point for the exact markers
axes[0, 1].plot(x4_pulse, y_pulse[:, 1], label="RK4")
axes[0, 1].plot(x4_pulse[every], exact[every, 1], "o", markersize=3, label="exact")
axes[0, 1].set_ylabel("$a_4'/H$")
axes[0, 1].set_title("the rate $a_4'$")
axes[0, 1].legend(fontsize=8)
axes[1, 0].plot(x4_pulse, y_pulse[:, 0], label="RK4")
axes[1, 0].plot(x4_pulse[every], exact[every, 0], "o", markersize=3, label="exact")
axes[1, 0].set_ylabel("$a_4$")
axes[1, 0].set_title("$a_4$")
axes[1, 0].legend(fontsize=8)
axes[1, 1].semilogy(x4_pulse, np.exp(y_pulse[:, 0]), label="3-space $e^{a_4}$")
axes[1, 1].semilogy(x4_pulse, np.exp(-y_pulse[:, 0]), "--",
                    label="extra times $e^{-a_4}$")
axes[1, 1].set_ylabel("scale factor")
axes[1, 1].set_title("the scale factors")
axes[1, 1].legend(fontsize=8)
for ax in axes[1]:
    ax.set_xlabel("time $x_4$ (units $1/H$)")
fig.tight_layout()
save_figure(fig, "stress_pulse",
            "Einstein gravity, starting on the deflating linear member $A = 1$ "
            "($a_4 = 0$, $a_4' = H$): a prescribed pulse of anisotropic stress "
            "$\\kappa(p_3 - p_t)$ centred at $x_4 = 3$ (top left, units $H^2$) raises "
            "the rate $a_4'$ from $H$ to $2H$ (top right); $a_4$ grows linearly with "
            "slope 1 before the pulse and with slope 2 after it (bottom left): the "
            "history moves from the linear member $A = 1$ to $A = 2$. Bottom right: "
            "on a logarithmic axis the scale factor $e^{a_4}$ of 3-space inflates and "
            "$e^{-a_4}$ of the extra times deflates exponentially at every time, "
            "twice as fast after the pulse. Lines: RK4 with step $h = 0.01$; dots: "
            "the exact solution with the error function. Horizontal axes: the time "
            "$x_4$ in units of $1/H$.")
```

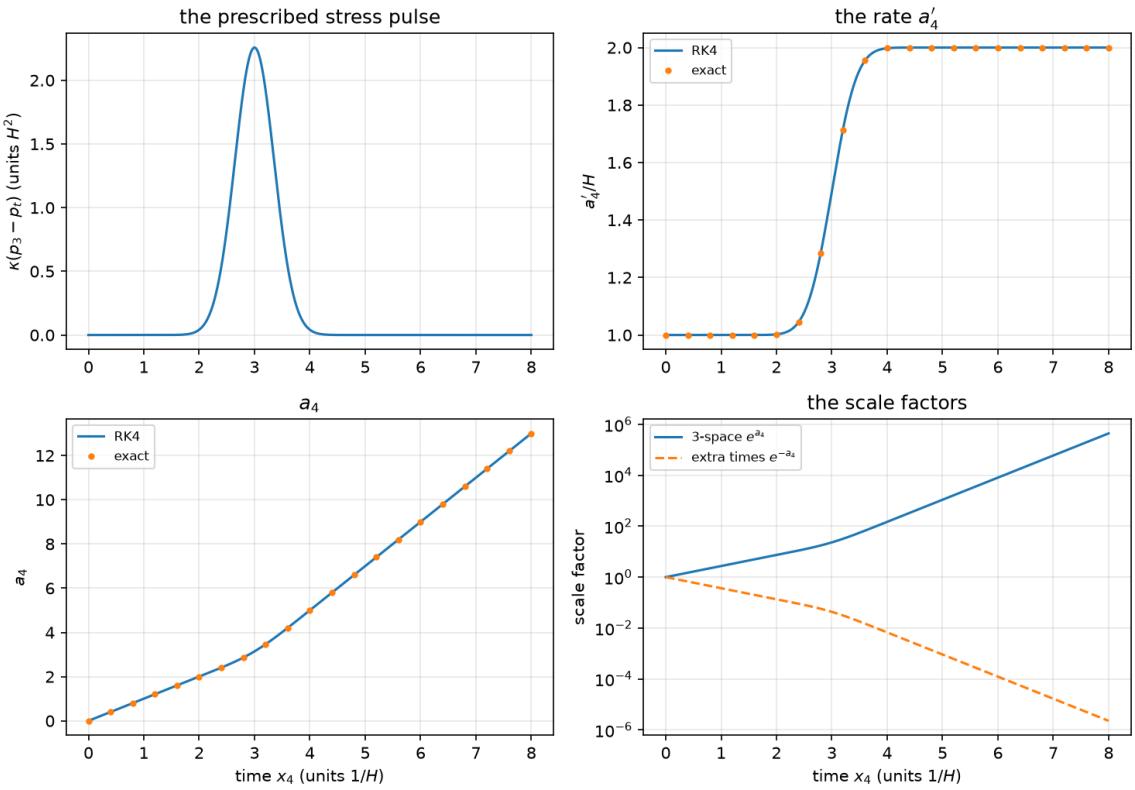

Figure 333: Einstein gravity, starting on the deflating linear member $A = 1$ ($a_4 = 0$, $a_4' = H$): a prescribed pulse of anisotropic stress $\kappa(p_3 - p_t)$ centred at $x_4 = 3$ (top left, units H^2) raises the rate a_4' from H to $2H$ (top right); a_4 grows linearly with slope 1 before the pulse and with slope 2 after it (bottom left): the history moves from the linear member $A = 1$ to $A = 2$. Bottom right: on a logarithmic axis the scale factor e^{a_4} of 3-space inflates and e^{-a_4} of the extra times deflates exponentially at every time, twice as fast after the pulse. Lines: RK4 with step $h = 0.01$; dots: the exact solution with the error function. Horizontal axes: the time x_4 in units of $1/H$. (Notebook 12c, figure 2.)

Out [8]:

```
Figure 12c.2 saved as Revision/textbook/figures/12c_2_stress_pulse.png
```

Text cell:

```
## 9. The source the pulse history requires, and the constraint

Along the pulse history the field equations fix the whole source (with
$\Lambda = 0$):

$$\kappa\rho = -(3(a_4')^2 + 21),\quad \kappa p_0 = 15 - 3(a_4')^2,\quad
\kappa p_3 = \kappa p_0 + \tfrac12\kappa\Delta,\quad
\kappa p_t = \kappa p_0 - \tfrac12\kappa\Delta.$$

The third component of our state, $\kappa\rho$ integrated from the conservation law
$\rho' = -3a_4'\Delta$, must agree with the $\kappa\rho$ that the constraint gives
at every time: this is the numerical form of the record's proof that the
derivative of the constraint is $3a_4'$ times the evolution equation. The next
cell defines a small helper that writes the measured disagreement as a power of
ten for the caption, checks the agreement, checks that the source starts on the
values of the linear member $A = 1$ ($\kappa\rho = -(3 + 21) = -24$,
$\kappa p = 15 - 3 = 12$) and ends on those of $A = 2$
($\kappa\rho = -(12 + 21) = -33$, $\kappa p = 15 - 12 = 3$), and draws the four
source components and the size of the disagreement.
```

In [9]:

```
def as_power_of_ten(value):
    """A positive number rounded to one digit, written for a caption:
    2e-11 becomes $2 \\times 10^{-11}$."""
    mantissa, exponent = f"{value:.0e}".split("e")
    return f"${mantissa} \\times 10^{{{int(exponent)}}}$"


rate = y_pulse[:, 1]
rho_constraint = -(rho_side(rate, *EINSTEIN) + 0.0)  # kappa rho, Lambda = 0
p0_values = p0_side(rate, *EINSTEIN) + 0.0  # kappa p0
p3_values = p0_values + stress_values / 2  # kappa p3
pt_values = p0_values - stress_values / 2  # kappa pt
mismatch = np.abs(y_pulse[:, 2] - rho_constraint)  # conservation versus constraint
say(f"largest difference of the two energy densities: {mismatch.max():.0e}")
reproduces(mismatch.max() < 1e-8,
           "the energy density from conservation equals the constraint along x4",
           WL, "constraint_propagation_bianchi")
check(abs(rho_constraint[0] + 24) < 1e-6 and abs(p0_values[0] - 12) < 1e-6,
      "before the pulse: kappa rho = -24, kappa p = 12 (the linear member A = 1)")
check(abs(rho_constraint[-1] + 33) < 1e-6 and abs(p0_values[-1] - 3) < 1e-6,
      "after the pulse: kappa rho = -33, kappa p = 3 (the linear member A = 2)")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(x4_pulse, rho_constraint, label="$\\kappa\\rho$")
```

```
left.plot(x4_pulse, p3_values, label="$\\kappa p_3$")
left.plot(x4_pulse, pt_values, "--", label="$\\kappa p_t$")
left.plot(x4_pulse, p0_values, ":", color="black", label="$\\kappa p_0$")
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("required source (units $H^2$)")
left.set_title("the source of the pulse history")
left.legend(fontsize=8)
right.semilogy(x4_pulse, np.maximum(mismatch, 1e-16))  # exact zeros at 1e-16
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("$|\\kappa\\rho$ conserved $-$ $\\kappa\\rho$ constraint$|$")
right.set_title("conservation agrees with the constraint")
save_figure(fig, "required_source",
            "Left: the source that the pulse history requires in Einstein gravity "
            "with $\\Lambda = 0$, in units of $H^2$: the energy density $\\kappa\\rho$, "
            "the pressures $\\kappa p_3$ (3-space), $\\kappa p_t$ (extra times, "
            "dashed) and $\\kappa p_0$ (hidden direction, dotted). During the pulse "
            "$p_3$ and $p_t$ split by the prescribed stress; before it the source "
            "is that of the linear member $A = 1$ ($\\kappa\\rho = -24$, "
            "$\\kappa p = 12$), after it that of the linear member $A = 2$ "
            "($\\kappa\\rho = -33$, $\\kappa p = 3$). Right: on a logarithmic axis, "
            "the difference between "
            "the energy density integrated from the conservation law and the one "
            "given by the constraint (an exact zero is drawn at $10^{-16}$). The two "
            f"agree to about {as_power_of_ten(mismatch.max())}, the accuracy of the "
            "RK4 integration with the step $h = 0.01$.")
```

Out [9]:

```
largest difference of the two energy densities: 2e-11
PASS the energy density from conservation equals the constraint along x4
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    constraint_propagation_bianchi
PASS before the pulse: kappa rho = -24, kappa p = 12 (the linear member A = 1)
PASS after the pulse: kappa rho = -33, kappa p = 3 (the linear member A = 2)
```

Out [9]:

```
Figure 12c.3 saved as Revision/textbook/figures/12c_3_required_source.png
```

Text cell:

```
## 10. A stress that relaxes the deflation rate

A stress can also depend on the state. We start on the faster deflating member
$A = 2$ ($a_4'(0) = 2H$) and prescribe
$\kappa\Delta = -2\eta\,(a_4' - H)$ with a constant $\eta > 0$: a resistance
proportional to the excess of the rate over $H$, like friction. In Einstein
gravity $a_4'' = \kappa\Delta/2 = -\eta(a_4' - H)$; the excess $u = a_4' - H$
obeys $u' = -\eta u$, so $u = He^{-\eta x_4}$ and, with $H = 1$,

$$a_4' = 1 + e^{-\eta x_4},\qquad
```

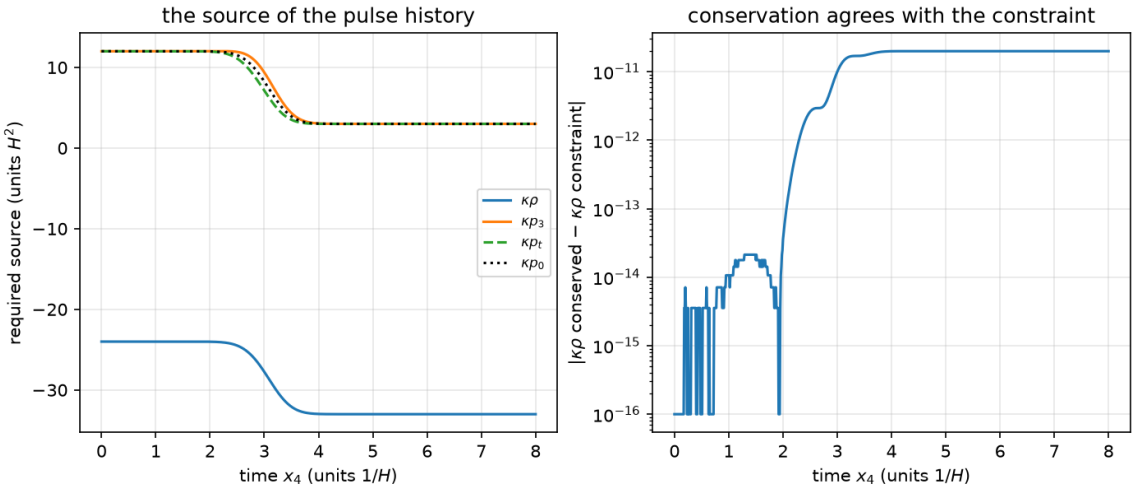

Figure 334: Left: the source that the pulse history requires in Einstein gravity with $\Lambda = 0$, in units of H^2: the energy density $\kappa\rho$, the pressures κp_3 (3-space), κp_t (extra times, dashed) and κp_0 (hidden direction, dotted). During the pulse p_3 and p_t split by the prescribed stress; before it the source is that of the linear member $A = 1$ ($\kappa\rho = -24$, $\kappa p = 12$), after it that of the linear member $A = 2$ ($\kappa\rho = -33$, $\kappa p = 3$). Right: on a logarithmic axis, the difference between the energy density integrated from the conservation law and the one given by the constraint (an exact zero is drawn at 10^{-16}). The two agree to about 2×10^{-11}, the accuracy of the RK4 integration with the step $h = 0.01$. (Notebook 12c, figure 3.)

```
a_4 = x_4 + \frac{1}{\eta}\big(1 - e^{-\eta x_4}\big).$$

The deflation rate relaxes from $2H$ to $H$: the history moves smoothly from the
linear member $A = 2$ to $A = 1$, and the extra-time scale factor $e^{-a_4}$
shrinks exponentially at every time, first at the rate $2H$, later at the rate
$H$. The next cell integrates this for $\eta = 0.25, 0.5, 1$ up to $x_4 = 10$,
compares with the exact solution, and checks the conservation law, which here
reads $\kappa\rho' = -3a_4'\kappa\Delta = 6\eta\,a_4'(a_4' - 1) > 0$: the energy
density rises from $-33$ (the value of $A = 2$) towards $-24$ (the value of
$A = 1$), in units of $H^2/\kappa$.
```

In [10]:

```
damped = {}  # eta -> (x4, solution)
for eta in (0.25, 0.5, 1.0):
    def friction(x, y, eta=eta):
        return -2.0 * eta * (y[1] - 1.0)  # kappa Delta = -2 eta (a4' - H)

    x4, y = history(friction, 2.0, 0.01, 1000)  # start on A = 2
    damped[eta] = (x4, y)
    exact_a4 = x4 + (1.0 / eta) * (1.0 - np.exp(-eta * x4))
    error = np.max(np.abs(y[:, 0] - exact_a4))
    mismatch = np.max(np.abs(y[:, 2] + rho_side(y[:, 1], *EINSTEIN)))
    check(error < 1e-9 and mismatch < 1e-8,
          f"eta = {eta}: a4 = x4 + (1 - e^(-eta x4))/eta, conservation holds")
    check(np.min(y[:, 1]) > 1.0,
```

```
          f"eta = {eta}: the rate a4' stays above H (the extra times deflate)")
    report(f"eta = {eta}: rate a4'/H at x4 = 10", f"{y[-1, 1]:.6f}")
```

Out [10]:

```
PASS eta = 0.25: a4 = x4 + (1 - e^(-eta x4))/eta, conservation holds
PASS eta = 0.25: the rate a4' stays above H (the extra times deflate)
RESULT eta = 0.25: rate a4'/H at x4 = 10 = 1.082085
PASS eta = 0.5: a4 = x4 + (1 - e^(-eta x4))/eta, conservation holds
PASS eta = 0.5: the rate a4' stays above H (the extra times deflate)
RESULT eta = 0.5: rate a4'/H at x4 = 10 = 1.006738
PASS eta = 1.0: a4 = x4 + (1 - e^(-eta x4))/eta, conservation holds
PASS eta = 1.0: the rate a4' stays above H (the extra times deflate)
RESULT eta = 1.0: rate a4'/H at x4 = 10 = 1.000045
```

Text cell:

```
The next cell draws the rate $a_4'$ and the extra-time scale factor $e^{-a_4}$ for
the three values of $\eta$, together with the two linear members $A = 2$ and
$A = 1$ (dotted) between which the histories move.
```

In [11]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
x4 = damped[0.25][0]
for slope, style, power in ((2.0, ":", "-2x_4"), (1.0, "-.", "-x_4")):
    left.plot(x4, slope * np.ones_like(x4), style, color="black",
              label=f"linear member $A = {slope:g}$")  # the two linear members
    right.semilogy(x4, np.exp(-slope * x4), style, color="black",
                   label=f"$A = {slope:g}$: $e^{{{power}}}$")
for eta, (x4, y) in damped.items():
    left.plot(x4, y[:, 1], label=f"$\\eta = {eta}$")
    right.semilogy(x4, np.exp(-y[:, 0]), label=f"$\\eta = {eta}$")
left.set_ylim(0.0, 2.3)
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("$a_4'/H$")
left.set_title("the deflation rate relaxes from $2H$ to $H$")
left.legend(fontsize=8)
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("extra-time scale factor $e^{-a_4}$")
right.set_title("the extra times keep deflating")
right.legend(fontsize=8)
save_figure(fig, "damped_deflation",
            "Einstein gravity with the prescribed relaxing stress "
            "$\\kappa(p_3 - p_t) = -2\\eta(a_4' - H)$, starting on the deflating "
            "linear member $A = 2$: left, the rate $a_4' = H(1 + e^{-\\eta x_4})$ for "
            "$\\eta = 0.25$, $0.5$ and $1$ (units $H$), which relaxes from $2H$ to "
            "$H$, with the linear members $A = 2$ and $A = 1$ in black; right, the "
            "scale factor $e^{-a_4}$ of the extra times on a logarithmic axis, which "
            "falls exponentially at every time: first as fast as for $A = 2$ "
            "(slope $-2$), later as for $A = 1$ (slope $-1$). The extra times never "
            "stop deflating. Horizontal axes: the time $x_4$ in units of $1/H$.")
```

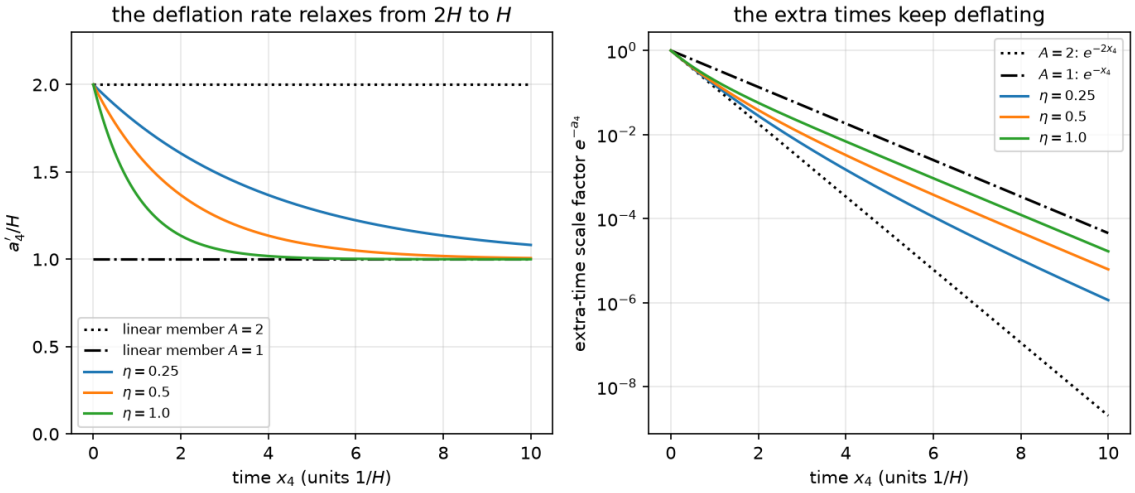

Figure 335: Einstein gravity with the prescribed relaxing stress $\kappa(p_3 - p_t) = -2\eta(a_4' - H)$, starting on the deflating linear member $A = 2$: left, the rate $a_4' = H(1 + e^{-\eta x_4})$ for $\eta = 0.25$, 0.5 and 1 (units H), which relaxes from $2H$ to H, with the linear members $A = 2$ and $A = 1$ in black; right, the scale factor e^{-a_4} of the extra times on a logarithmic axis, which falls exponentially at every time: first as fast as for $A = 2$ (slope -2), later as for $A = 1$ (slope -1). The extra times never stop deflating. Horizontal axes: the time x_4 in units of $1/H$. (Notebook 12c, figure 4.)

Out [11]:

```
Figure 12c.4 saved as Revision/textbook/figures/12c_4_damped_deflation.png
```

Text cell:

```
## 11. How fast does RK4 converge?

For the relaxing history the right-hand side depends on the solution itself, so
it is a genuine differential equation and a good test of the method. (For the
pulse the stress depends on $x_4$ alone; RK4 then reduces to Simpson's rule for an
integrand that vanishes smoothly at both ends, and such integrals converge much
faster than $h^4$, which would hide the order.) The next cell integrates the
relaxing history with $\eta = 1$ up to $x_4 = 8$ with the step sizes
$h = 0.4, 0.2, 0.1, 0.05, 0.025$ and measures the error of $a_4(8)$ against the
exact value $8 + (1 - e^{-8})$. If the error behaves like $Ch^4$, halving $h$
divides it by $2^4 = 16$, and the measured order $\log_2(e_h / e_{h/2})$ is close
to 4. The table prints each step, its error and the order; the plot shows the
errors on logarithmic axes, where $Ch^4$ is a straight line of slope 4.
```

In [12]:

```
def relaxing_unit(x, y):
    return -2.0 * (y[1] - 1.0)  # kappa Delta = -2 eta (a4' - H) with eta = 1
```

```
exact_end = 8.0 + (1.0 - math.exp(-8.0))  # the exact a4(8) for eta = 1
steps_list = [20, 40, 80, 160, 320]  # h = 8/steps = 0.4, 0.2, 0.1, 0.05, 0.025
sizes, errors = [], []
for steps in steps_list:
    x4, y = history(relaxing_unit, 2.0, 8.0 / steps, steps)
    sizes.append(8.0 / steps)
    errors.append(abs(y[-1, 0] - exact_end))
orders = [math.log2(errors[i] / errors[i + 1]) for i in range(len(errors) - 1)]
for i, (size, error) in enumerate(zip(sizes, errors)):
    order = f"{orders[i - 1]:.2f}" if i > 0 else "-"
    say(f"h = {size:<6} error of a4(8) = {error:.2e}   measured order {order}")
check(all(3.7 < q < 4.3 for q in orders),
      "RK4 converges with order 4 (measured orders between 3.7 and 4.3)")
fig, ax = plt.subplots()
ax.loglog(sizes, errors, "o-", label="error of $a_4(8)$")
reference = errors[-1] * (np.array(sizes) / sizes[-1]) ** 4
ax.loglog(sizes, reference, "--", color="black", label="slope 4: $C h^4$")
ax.set_xticks(sizes)  # one tick at each step size used
ax.set_xticklabels([f"{size:g}" for size in sizes])
ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())  # no extra labels
ax.set_xlabel("step size $h$ (units $1/H$)")
ax.set_ylabel("absolute error")
ax.set_title("RK4 on the relaxing history: fourth-order convergence")
ax.legend(fontsize=8)
save_figure(fig, "rk4_convergence",
            "The error of the RK4 value of $a_4$ at $x_4 = 8$ for the relaxing "
            "history with $\\eta = 1$ (exact value $8 + 1 - e^{-8}$), for the step "
            "sizes $h = 0.4$, $0.2$, $0.1$, $0.05$ and $0.025$ (units $1/H$), on "
            "logarithmic axes. The points follow the dashed reference line $Ch^4$ "
            "of slope 4: halving the step divides the error by about 16, the "
            "fourth-order convergence of the Runge-Kutta method.")
```

Out [12]:

```
h = 0.4    error of a4(8) = 8.01e-07   measured order -
h = 0.2    error of a4(8) = 4.23e-08   measured order 4.24
h = 0.1    error of a4(8) = 2.43e-09   measured order 4.12
h = 0.05   error of a4(8) = 1.46e-10   measured order 4.06
h = 0.025  error of a4(8) = 8.92e-12   measured order 4.03
PASS RK4 converges with order 4 (measured orders between 3.7 and 4.3)
```

Out [12]:

```
Figure 12c.5 saved as Revision/textbook/figures/12c_5_rk4_convergence.png
```

Text cell:

```
## 12. Einstein-Gauss-Bonnet gravity: the evolution equation breaks down

With the Gauss-Bonnet coupling ($\alpha_1 = 1$, $\alpha_3 = 0$, $H = 1$),
$F(a_4') = 2 - 80\alpha_2 - 48\alpha_2(a_4')^2$ decreases as $|a_4'|$ grows and
```

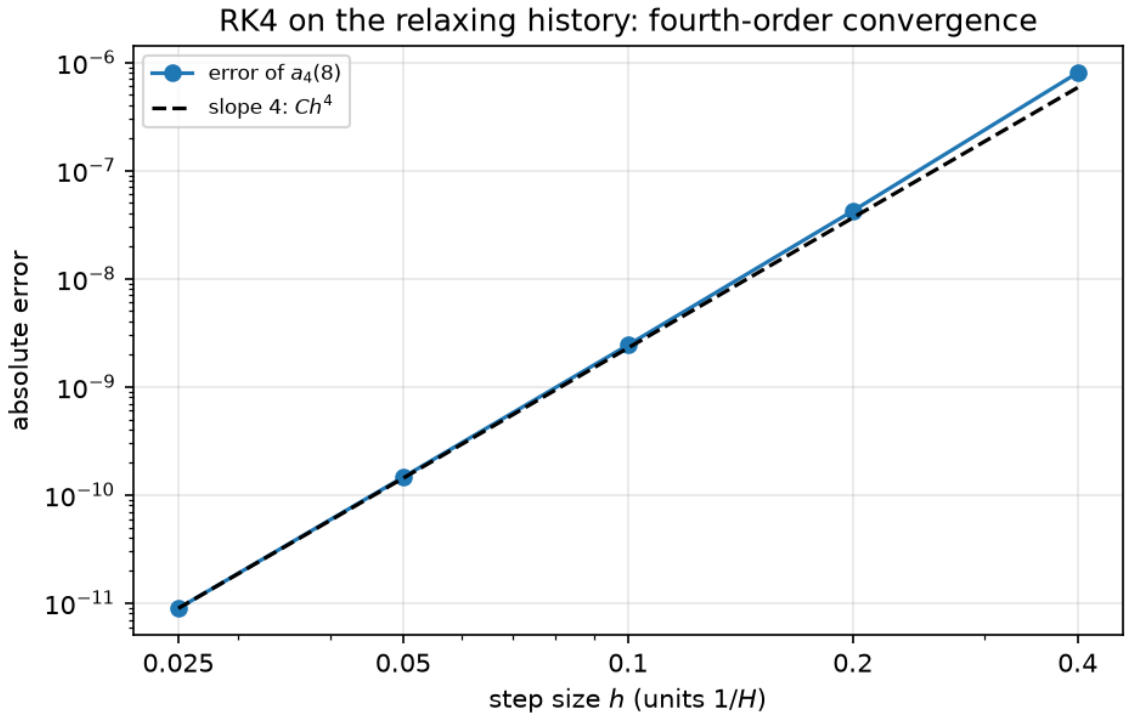

Figure 336: The error of the RK4 value of a_4 at $x_4 = 8$ for the relaxing history with $\eta = 1$ (exact value $8 + 1 - e^{-8}$), for the step sizes $h = 0.4$, 0.2, 0.1, 0.05 and 0.025 (units $1/H$), on logarithmic axes. The points follow the dashed reference line Ch^4 of slope 4: halving the step divides the error by about 16, the fourth-order convergence of the Runge-Kutta method. (Notebook 12c, figure 5.)

```
vanishes at $a_4'_c = \sqrt{(2 - 80\alpha_2)/(48\alpha_2)}$. For a constant stress
$\kappa\Delta$ the equation $F(a_4')\,a_4'' = \kappa\Delta$ can be integrated
once: with $G(v) = (2 - 80\alpha_2)v - 16\alpha_2v^3$ (so that $dG/dv = F$),

$$G(a_4'(x_4)) = G(a_4'(0)) + \kappa\Delta\,x_4 .$$

$G$ has its maximum at $a_4'_c$, so the rate reaches $a_4'_c$ at the finite time
$x_4^{\star} = (G(a_4'_c) - G(a_4'(0)))/(\kappa\Delta)$, with $a_4'' = \kappa\Delta/F$
growing without bound; beyond it the equation has no solution with a smooth
$a_4'$. The next cell starts on the canonical deflating member ($a_4'(0) = H$),
integrates with $\kappa\Delta = 0.5$ and $\alpha_2 = 0.005, 0.01$ (step
$h = 0.001$), stops when $F < 0.05$, and checks the integrated relation and the
breakdown time.
```

In [13]:

```
STRESS, RATE0 = 0.5, 1.0  # kappa Delta and the starting rate a4'(0) = H (A = 1)


def constant_stress(x, y):
    return STRESS


breakdown = {}  # alpha2 -> (x4, solution, predicted breakdown time)
for alpha2 in (0.005, 0.01):
    couplings = (1.0, alpha2, 0.0)

    def G(v, alpha2=alpha2):
        return (2 - 80 * alpha2) * v - 16 * alpha2 * v ** 3  # dG/dv = F

    critical = math.sqrt((2 - 80 * alpha2) / (48 * alpha2))  # F(critical) = 0
    x_star = (G(critical) - G(RATE0)) / STRESS  # the breakdown time

    def near_zero(y, couplings=couplings):
        return F_of(y[1], *couplings) < 0.05  # stop before F reaches zero

    x4, y = history(constant_stress, RATE0, 0.001, 20000, couplings,
                    stop=near_zero)
    breakdown[alpha2] = (x4, y, x_star)
    residual = np.max(np.abs(G(y[:, 1]) - G(RATE0) - STRESS * x4))
    check(residual < 1e-6, f"alpha2 = {alpha2}: G(a4') = G(a4'(0)) + kappa Delta x4")
    check(abs(x4[-1] - x_star) < 0.01,
          f"alpha2 = {alpha2}: F reaches 0 near the predicted time x4*")
    report(f"alpha2 = {alpha2}: critical rate, breakdown time x4*",
           f"{critical:.4f} H, {x_star:.4f}/H")
```

Out [13]:

```
PASS alpha2 = 0.005: G(a4') = G(a4'(0)) + kappa Delta x4
PASS alpha2 = 0.005: F reaches 0 near the predicted time x4*
RESULT alpha2 = 0.005: critical rate, breakdown time x4* = 2.5820 H, 2.4682/H
PASS alpha2 = 0.01: G(a4') = G(a4'(0)) + kappa Delta x4
```

```
PASS alpha2 = 0.01: F reaches 0 near the predicted time x4*
RESULT alpha2 = 0.01: critical rate, breakdown time x4* = 1.5811 H, 0.4498/H
```

Text cell:

```
The next cell compares the two Gauss-Bonnet histories with Einstein gravity, where
$F = 2$ and the rate simply grows linearly, $a_4' = 1 + 0.25x_4$, and draws
$F(a_4'(x_4))$ along each history.
```

In [14]:

```
x_einstein, y_einstein = history(constant_stress, RATE0, 0.01, 500)
check(np.max(np.abs(y_einstein[:, 1] - (RATE0 + STRESS * x_einstein / 2))) < 1e-12,
      "Einstein: a4' = 1 + 0.25 x4 for the constant stress")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(x_einstein, y_einstein[:, 1], color="black", label="Einstein")
right.plot(x_einstein, F_of(y_einstein[:, 1], *EINSTEIN) * np.ones_like(x_einstein),
           color="black", label="Einstein")
for alpha2, (x4, y, x_star) in breakdown.items():
    line = left.plot(x4, y[:, 1], label=f"$\\alpha_2H^2 = {alpha2}$")[0]
    left.axvline(x_star, linestyle="--", color=line.get_color(), linewidth=0.8)
    right.plot(x4, F_of(y[:, 1], 1.0, alpha2, 0.0), color=line.get_color(),
               label=f"$\\alpha_2H^2 = {alpha2}$")
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("$a_4'/H$")
left.set_title("the rate under a constant stress")
left.legend(fontsize=8)
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("$F(a_4')$")
right.set_title("$F$ along the history")
right.legend(fontsize=8)
save_figure(fig, "gauss_bonnet_breakdown",
            "A constant prescribed stress $\\kappa(p_3 - p_t) = 0.5H^2$ starting on "
            "the deflating linear member $A = 1$ ($a_4' = H$). Left: the rate $a_4'$ "
            "in Einstein gravity grows "
            "linearly (black), while in Einstein-Gauss-Bonnet gravity with "
            "$\\alpha_2H^2 = 0.005$ and $0.01$ it bends upward and reaches the "
            "critical rate at the predicted finite time $x_4^{\\star}$ (dashed vertical "
            "lines), where its slope becomes infinite. Right: the factor $F(a_4')$ "
            "of the evolution equation along each history; the integration stops "
            "when $F$ falls below $0.05$, because at $F = 0$ the evolution equation "
            "breaks down. Horizontal axes: the time $x_4$ in units of $1/H$.")
```

Out [14]:

```
PASS Einstein: a4' = 1 + 0.25 x4 for the constant stress
```

Out [14]:

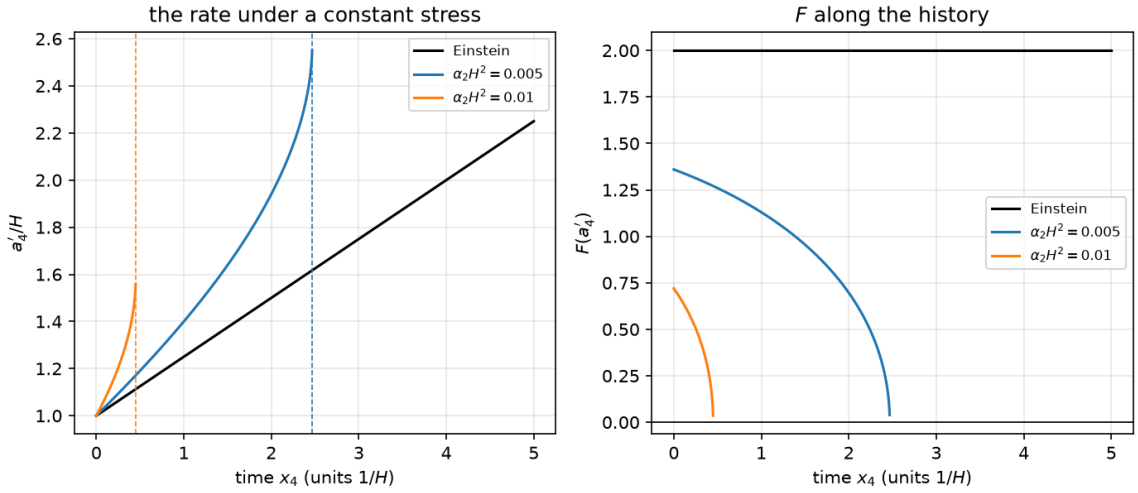

Figure 337: A constant prescribed stress $\kappa(p_3 - p_t) = 0.5H^2$ starting on the deflating linear member $A = 1$ ($a_4' = H$). Left: the rate a_4' in Einstein gravity grows linearly (black), while in Einstein-Gauss-Bonnet gravity with $\alpha_2 H^2 = 0.005$ and 0.01 it bends upward and reaches the critical rate at the predicted finite time $x_4^\star$ (dashed vertical lines), where its slope becomes infinite. Right: the factor $F(a_4')$ of the evolution equation along each history; the integration stops when F falls below 0.05, because at $F = 0$ the evolution equation breaks down. Horizontal axes: the time x_4 in units of $1/H$. (Notebook 12c, figure 6.)

```
Figure 12c.6 saved as Revision/textbook/figures/12c_6_gauss_bonnet_breakdown.png
```

Text cell:

```
## 13. Which sources are real? The record on the Kohn-Sham states

The stresses above were chosen by hand. The Revision record asks the opposite
question for the only many-particle states it has computed, the Kohn-Sham states
of dirac16complex: can they be the source of the author's metric? The next cell
reads the record's answer, prints how many of its checks passed and checks that
every one did. The answer is no: every nonzero Kohn-Sham state depends on $x_0$
and violates $p_3 + p_t = 2p_0$, so the Kohn-Sham history $a_4 = AHx_4$ is a
prescribed background, not a solution of these equations with that source.
```

In [15]:

```
ks = read_json(KS)
say("record: " + ks["conclusion"])
detail = {entry["name"]: entry["detail"] for entry in ks["checks"]}
verdicts = [entry["verdict"] for entry in ks["checks"]]
passed = verdicts.count("PASS")  # how many checks of the record passed
say(f"{KS}: {passed} of {len(verdicts)} checks PASS")
check(len(verdicts) > 0 and passed == len(verdicts) == ks["summary"]["checks"]
      and "ks_history_is_a_prescribed_background" in detail,
      "record: no Kohn-Sham state is an admissible source (every check PASS)")
```

Out [15]:

```
record: No Kohn-Sham state recorded in Revision/kohn_sham is an admissible source of the
    author's metric: C1 and C2 fail for every nonzero state (C2 also after integration
    over x0), and the Kohn-Sham history uses the linear member as a prescribed
    background, without back-reaction.  With the metric of 2026-10-14 (b4[x4]) C1 alone
    excludes every recorded state for every b4 (check ks_verdict_holds_for_every_b4); C2
    is the b4 = 1 form.
Revision/field_equations_a4/reports/ks-source-conditions.json: 6 of 6 checks PASS
PASS record: no Kohn-Sham state is an admissible source (every check PASS)
```

Text cell:

```
## 14. The record's numbers, recomputed from the Kohn-Sham table

The record does not only say no; it gives numbers. We recompute them here from
the table `emt-integrals.csv` of the Kohn-Sham record, which holds one row per
recorded ground state: its name (for example `N136_lam0_a10`: $N = 136$
particles, coupling $\lambda = 0$, slice $a_{4,0} = 1$) and the integrals over
the hidden direction, $2\,\mathrm{Vol}_7\int e^{6Hy}\,T\,dy$, of the energy
density and of the three pressures (columns `int_rho`, `int_p3`, `int_p_t`,
`int_p0`). If the source obeyed $p_3 + p_t = 2p_0$ at every point, it would also
obey it after the integration, and the ratio
$$r = \frac{\int p_3 + \int p_t}{2\int p_0}$$
would be exactly 1. The next cell

- counts the states, and those whose four integrals are not all zero (a state
  with a zero energy-momentum tensor is no source at all);
- computes $r$ for every other state, finds the one closest to 1, and prints $r$
  for the history $N = 136$, $\lambda = 0$ at the slices $a_{4,0} = 0, 1, 2$;
- compares every number with the text of the record, which prints them with six
  significant digits (the format `.6g`), so that a change of either is caught.
```

In [16]:

```
import csv  # reads tables of comma-separated values
import re  # regular expressions: find a pattern in a text

TABLE = "Revision/kohn_sham/results/ground/emt-integrals.csv"
with repository_file(TABLE).open(encoding="utf-8", newline="") as handle:
    table = {row["id"]: row for row in csv.DictReader(handle)}
INTEGRALS = ("int_rho", "int_p3", "int_p_t", "int_p0")  # 2 Vol_7 int e^(6Hy) T dy
zero = sorted(name for name, row in table.items()
              if all(float(row[column]) == 0.0 for column in INTEGRALS))
nonzero = [name for name in table if name not in zero]
report("Kohn-Sham states in the table", len(table))
report("states with a nonzero energy-momentum tensor", len(nonzero))
say("states with a zero energy-momentum tensor: " + ", ".join(zero))

counted = re.search(r"(\d+) ground-state profiles .*?\((\d+) with a nonzero",
                    detail["ks_profiles_depend_on_x0"])
```

```
reproduces(counted is not None
           and counted.groups() == (str(len(table)), str(len(nonzero))),
           f"{len(nonzero)} of the {len(table)} states have a nonzero tensor",
           KS, "ks_profiles_depend_on_x0")
listed = detail["ks_zero_source_states_listed"].rsplit(": ", 1)[-1].split(", ")
reproduces(sorted(listed) == zero,
           "the states with a zero tensor are the ones the record lists",
           KS, "ks_zero_source_states_listed")

ratio = {}  # r = (int p3 + int p_t)/(2 int p0) of every nonzero state
for name in nonzero:
    row = table[name]
    if float(row["int_p0"]) != 0.0:
        ratio[name] = ((float(row["int_p3"]) + float(row["int_p_t"]))
                       / (2 * float(row["int_p0"])))
closest = min(ratio, key=lambda name: abs(ratio[name] - 1.0))
report(f"closest to 1: r of {closest}", f"{ratio[closest]:.6g}")
HISTORY = ["N136_lam0_a00", "N136_lam0_a10", "N136_lam0_a20"]  # N = 136, lambda = 0
for name in HISTORY:
    slice_a4 = float(table[name]["a4"])  # the slice a4,0 of this state
    report(f"N = 136, lambda = 0, a4,0 = {slice_a4:g}: r", f"{ratio[name]:.6g}")

text = detail["ks_integrals_violate_algebraic_condition"]
tolerance = float(ks["tolerance"].split()[-1])  # "relative 1e-06" gives 1e-06
best = re.search(r"closest to 1: (\S+) at (\w+)", text)
reproduces(len(ratio) == len(nonzero) and abs(ratio[closest] - 1.0) > tolerance
           and best is not None
           and best.groups() == (f"{ratio[closest]:.6g}", closest),
           "r differs from 1 for every nonzero state; the closest as recorded",
           KS, "ks_integrals_violate_algebraic_condition")
found = [re.search(name + r": (\S+?)[,\s]", text) for name in HISTORY]
reproduces(all(item is not None and item.group(1) == f"{ratio[name]:.6g}"
               for item, name in zip(found, HISTORY)),
           "r of N = 136, lambda = 0 at a4,0 = 0, 1, 2 equals the record",
           KS, "ks_integrals_violate_algebraic_condition")
```

Out [16]:

```
RESULT Kohn-Sham states in the table = 75
RESULT states with a nonzero energy-momentum tensor = 70
states with a zero energy-momentum tensor: N8_lam0_a00, N8_lam0_a05, N8_lam0_a10,
    N8_lam0_a15, N8_lam0_a20
PASS 70 of the 75 states have a nonzero tensor
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_profiles_depend_on_x0
PASS the states with a zero tensor are the ones the record lists
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_zero_source_states_listed
RESULT closest to 1: r of N688_lamm2_a00 = 0.414328
RESULT N = 136, lambda = 0, a4,0 = 0: r = 0.339767
RESULT N = 136, lambda = 0, a4,0 = 1: r = 0.25969
RESULT N = 136, lambda = 0, a4,0 = 2: r = 0.239714
PASS r differs from 1 for every nonzero state; the closest as recorded
```

```
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_integrals_violate_algebraic_condition
PASS r of N = 136, lambda = 0 at a4,0 = 0, 1, 2 equals the record
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_integrals_violate_algebraic_condition
```

Text cell:

```
## 15. The last check

The last cell checks that the six figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [17]:

```
figure_names = ["linear_member", "stress_pulse", "required_source",
                "damped_deflation", "rk4_convergence", "gauss_bonnet_breakdown"]
paths = [output_file(f"{FIGURE_FOLDER}/12c_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths), "all six figure files exist")
all_checks_passed()
```

Out [17]:

```
PASS all six figure files exist
ALL 30 CHECKS PASSED (notebook 12c)
```

Text cell:

```
## 16. What this notebook showed

- With a prescribed anisotropic stress $p_3 - p_t$ the evolution equation
  $a_4''F(a_4') = \kappa(p_3 - p_t)$ is an ordinary differential equation for
  $a_4(x_4)$, which RK4 solves with fourth-order accuracy (COMPUTED, compared with
  exact solutions).
- No stress gives the linear member $a_4 = AHx_4$: 3-space inflates as
  $e^{AHx_4}$ and the extra times deflate as $e^{-AHx_4}$, with a constant
  7-volume (PROVED, and reproduced numerically).
- A pulse of stress can raise the deflation rate from $H$ to $2H$, and a relaxing
  stress can bring it back to $H$; along both histories the extra times deflate
  exponentially at every time (COMPUTED for ASSUMED stresses).
- The energy density that the conservation law gives equals the one that the
  constraint gives along every history, as the record's Bianchi identity requires
  (COMPUTED).
- In Einstein-Gauss-Bonnet gravity a constant stress drives $a_4'$ to the zero of
  $F$ in a finite time, where the evolution equation breaks down (PROVED by the
  integrated relation, COMPUTED numerically).
- None of these stresses is known to come from a field of the theory; the record
  shows that the Kohn-Sham states of the repository are not admissible sources
```

```
  (record ks-source-conditions.json). The notebook recomputes the record's
  numbers from the Kohn-Sham table: the integrated ratio
  $(\int p_3 + \int p_t)/(2\int p_0)$, which must be 1 for an admissible source,
  differs from 1 for every recorded state with a nonzero energy-momentum tensor
  (COMPUTED from the record's table; equal to the record).
```

12.25 Line-by-line walk-through of Notebook 12c

The notebook has 17 code cells, In [1] to In [17].

In [1], the set-up cell. As in Notebook 12a (Section 12.16, In [1]), except that the comments hold the run instructions of Notebook 12c (Section 12.23) and `NOTEBOOK_ID = "12c"`.

In [2], the records and the Lovelock components as functions.

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory
import math  # the error function erf, for an exact solution

import matplotlib.ticker  # control of the tick labels of an axis
import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra, used here only to read the record formulas
```

Besides the modules met before: `math`, Python's module of mathematical functions (it has the error function `math.erf`), and `matplotlib.ticker`, which controls the labels of axis ticks.

```
PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the equations
KS = "Revision/field_equations_a4/reports/ks-source-conditions.json"
```

Short names of four record files; `KS` is the record on the Kohn-Sham sources. The three helpers `read_json`, `record_verdict` and `reproduces` that follow are those of Notebook 12a, In [2] (Section 12.16).

```
record = read_json(EQUATIONS)
H, ad1, ad2 = sp.symbols("H ad1 ad2", real=True)
a1, a2, a3 = sp.symbols("alpha1 alpha2 alpha3", real=True)
LOCALS = {"H": H, "ad1": ad1, "ad2": ad2, "alpha1": a1, "alpha2": a2, "alpha3": a3}


def from_record(text):
    return sp.sympify(text.replace("^", "**"), locals=LOCALS).subs(H, 1)
```

The record is read; the symbols for H, a_4', a_4'' and the couplings are made (in this notebook the Python names of the couplings are `a1`, `a2`, `a3`, so that the name `alpha2` stays free for a plain number later); `from_record` parses a formula of the record and sets $H = 1$ (docstring left out).

```
lovelock = record["lovelockTensors"]
weights = {1: a1, 2: a2, 3: a3}


def lovelock_sum(component):
    return sum(weights[k] * from_record(lovelock[f"E{k}"][component]["input"])
               for k in (1, 2, 3))


e44 = lovelock_sum("x4x4")  # the time component (constraint)
e00 = lovelock_sum("x0x0")  # the hidden component
F_expr = from_record(record["generalSource"]["evolution_F"]["input"])
```

`lovelock_sum` forms $\sum_k \alpha_k E_{(k)}$ for one component of the record. It gives e_{44} and e_{00} of Section 12.8; F is read directly from the record.

```
rho_side = sp.lambdify((ad1, a1, a2, a3), e44, "numpy")
p0_side = sp.lambdify((ad1, a1, a2, a3), e00, "numpy")
F_of = sp.lambdify((ad1, a1, a2, a3), F_expr, "numpy")
say(f"F(a4') with H = 1: {F_expr}")
EINSTEIN = (1.0, 0.0, 0.0)  # alpha1, alpha2, alpha3 of Einstein gravity
```

Three fast numpy functions of the rate $v = a_4'$ and the couplings: `rho_side` is e_{44} (so $\kappa\rho = -(e_{44}+\Lambda)$), `p0_side` is e_{00} (so $\kappa p_0 = e_{00} + \Lambda$) and `F_of` is F. F is printed. `EINSTEIN` is a **tuple** (a fixed list in round brackets) of the Einstein couplings; `F_of(v, *EINSTEIN)` passes its three numbers as three arguments.

In [3], a test of the functions.

```
rates = np.array([-2.0, -0.5, 0.0, 1.0, 3.0])  # a few values of a4'
ok = (np.allclose(F_of(rates, *EINSTEIN), 2.0)
      and np.allclose(rho_side(rates, *EINSTEIN), 3 * rates ** 2 + 21)
      and np.allclose(p0_side(rates, *EINSTEIN), 15 - 3 * rates ** 2))
reproduces(ok, "Einstein: F = 2, E44 = 3 a4'^2 + 21, E88 = 15 - 3 a4'^2 (H = 1)",
           PY, "einstein_components")
```

At five rates the three functions must give the Einstein values of Section 12.9 (`np.allclose` compares arrays up to rounding).

In [4], the RK4 solver and its test.

```
def rk4(rhs, y0, h, steps, stop=None):
    xs = [0.0]
    ys = [np.array(y0, dtype=float)]
    for n in range(steps):
        x, y = xs[-1], ys[-1]
        k1 = rhs(x, y)
        k2 = rhs(x + h / 2, y + h / 2 * k1)
        k3 = rhs(x + h / 2, y + h / 2 * k2)
        k4 = rhs(x + h, y + h * k3)
```

```
        xs.append((n + 1) * h)  # (n + 1) h avoids adding up rounding errors
        ys.append(y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4))
        if stop is not None and stop(ys[-1]):
            break
    return np.array(xs), np.array(ys)
```

`rk4` solves $dy/dx =$ `rhs(x, y)` from $x = 0$ with the starting value `y0`, by at most `steps` steps of size `h` (docstring left out). The lists `xs` and `ys` hold the points reached; `[-1]` is the last one. The four lines `k1` to `k4` and the update are the RK4 formulas of Section 12.21, applied to the whole array `y` at once. The new x is computed as $(n + 1)h$ rather than by adding h again and again, which would accumulate rounding errors. If a function `stop` is given and returns true for the new point, `break` ends the loop early. The function returns the points as arrays: `ys[n, 0]` is the first component at step n.

```
x_test, y_test = rk4(lambda x, y: -y, [1.0], 0.05, 20)
error_test = abs(y_test[-1, 0] - math.exp(-1.0))
say(f"y' = -y: RK4 with h = 0.05 gives y(1) = {y_test[-1, 0]:.10f}; "
    f"exact {math.exp(-1.0):.10f}")
check(error_test < 1e-7, "RK4 solves y' = -y to better than 1e-7 with 20 steps")
```

A test with a known answer: $y' = -y$, $y(0) = 1$, whose solution is e^{-x}. `lambda x, y:  -y` is a one-line function without a name. Twenty steps of 0.05 reach $x = 1$; the result 0.3678794611 differs from $e^{-1} = 0.3678794412$ by 2×10^{-8}.

In [5], no stress: the linear member.

```
def history(stress, rate0, h, steps, couplings=EINSTEIN, Lam=0.0, stop=None):
    def rhs(x, y):
        push = stress(x, y)  # kappa (p3 - pt) at this time
        return np.array([y[1], push / F_of(y[1], *couplings), -3.0 * y[1] * push])

    rho0 = -(rho_side(rate0, *couplings) + Lam)  # kappa rho at x4 = 0
    return rk4(rhs, [0.0, rate0, rho0], h, steps, stop)
```

`history` integrates the first-order system of Section 12.21 for a given stress function (docstring left out). Inside it, `rhs` is the right-hand side for the state $y = (a_4, v, \kappa\rho)$: $dy_0/dx = v$, $dy_1/dx = \kappa\Delta/F(v)$, $dy_2/dx = -3v\,\kappa\Delta$. The starting energy density is taken from the constraint, $\kappa\rho = -(e_{44} + \Lambda)$ at the starting rate. The defaults are Einstein gravity and $\Lambda = 0$.

```
def no_stress(x, y):
    return 0.0  # Delta = 0: an isotropic source


linear = {}  # slope A -> (x4, solution)
for slope in (2.0, 1.0, -1.0):  # deflating A = 2, 1 and the mirror image A = -1
    linear[slope] = history(no_stress, slope, 0.01, 300)
    x4, y = linear[slope]
    error = np.max(np.abs(y[:, 0] - slope * x4))  # compare with a4 = A x4
    check(error < 1e-12, f"Delta = 0, A = {slope:g}: a4 = A H x4 exactly")
```

With zero stress, three histories of 300 steps of 0.01 (up to $x_4 = 3$) are integrated, starting at the rates $A = 2$, 1 and -1; `y[:, 0]` is the column of a_4 values, which must equal Ax_4 (the format `:g` prints 2.0 as 2).

```
x4, y = linear[1.0]
report("A = 1, Lambda = 0: kappa rho along the history", f"{y[-1, 2]:.6f}", "H^2")
product = np.exp(3 * y[:, 0]) * np.exp(-3 * y[:, 0])  # e^(3 a4) e^(-3 a4)
check(np.max(np.abs(product - 1.0)) < 1e-12,
      "the 7-volume factor e^(3 a4) e^(-3 a4) stays 1")
```

For $A = 1$ the energy density at the end is -24, the value of Section 12.12, and the 7-volume factor stays 1.

In [6], figure 1: the linear histories and the scale factors.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
for slope, style, words in ((2.0, "-", "deflating"), (1.0, "-.", "deflating"),
                            (-1.0, "--", "mirror image")):
    x4, y = linear[slope]
    left.plot(x4, y[:, 0], style, label=f"$A = {slope:g}$ ({words})")
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("$a_4$")
left.set_title("$\\Delta = 0$: $a_4 = AHx_4$")
left.legend(fontsize=8)
```

Two panels. The left one draws the three histories with three line styles (solid, dash-dotted, dashed).

```
x4, y = linear[1.0]
right.semilogy(x4, np.exp(y[:, 0]), label="3-space $e^{a_4}$ (inflates)")
right.semilogy(x4, np.exp(-y[:, 0]), "--", label="extra times $e^{-a_4}$ (deflate)")
right.semilogy(x4, np.exp(3 * y[:, 0]) * np.exp(-3 * y[:, 0]), ":", color="black",
               label="$e^{3a_4}e^{-3a_4} = 1$")
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("scale factor")
right.set_title("$A = 1$: the scale factors")
right.legend(fontsize=8)
save_figure(fig, "linear_member",
            ...)
```

`semilogy` draws with a logarithmic vertical axis, on which an exponential is a straight line. **What the figure shows.** On the left three straight lines of slopes 2, 1 and -1. On the right e^{a_4} rises and e^{-a_4} falls along straight lines of equal and opposite slope, and the product of their cubes is the flat line 1.

In [7], the pulse.

```
X_C, WIDTH = 3.0, 0.5  # the centre and the width of the pulse
PUSH = 2.0 / (WIDTH * math.sqrt(math.pi))  # kappa Delta_0
RATE_START = 1.0  # a4'(0)/H: the canonical deflating history A = 1
```

```
def pulse(x, y):
    return PUSH * math.exp(-((x - X_C) / WIDTH) ** 2)  # kappa Delta(x4)
```

The constants of the pulse of Section 12.21 and the stress function itself (it ignores the state `y`).

```
def exact_pulse(x):
    scale = PUSH * WIDTH * math.sqrt(math.pi) / 4  # = 1/2
    t, t0 = (x - X_C) / WIDTH, -X_C / WIDTH  # the argument now and at x4 = 0

    def antiderivative(s):  # an antiderivative of erf
        return s * math.erf(s) + math.exp(-s * s) / math.sqrt(math.pi)

    rate = RATE_START + scale * (math.erf(t) + math.erf(X_C / WIDTH))
    a4 = RATE_START * x + scale * (WIDTH * (antiderivative(t) - antiderivative(t0))
                                   + x * math.erf(X_C / WIDTH))
    return a4, rate
```

The exact solution (docstring left out): `rate` is the formula for a_4' derived in Section 12.21; `a4` is its integral from 0, in which $\int_0^x \operatorname{erf}((s - x_c)/w)\, ds = w\,(\text{antiderivative}(t) - \text{antiderivative}(t_0))$ by the same substitution as before.

```
x4_pulse, y_pulse = history(pulse, RATE_START, 0.01, 800)
exact = np.array([exact_pulse(x) for x in x4_pulse])  # columns: a4, a4'
error_a4 = np.max(np.abs(y_pulse[:, 0] - exact[:, 0]))
error_rate = np.max(np.abs(y_pulse[:, 1] - exact[:, 1]))
say(f"largest error: a4 {error_a4:.0e}, a4' {error_rate:.0e}")
check(error_a4 < 1e-8 and error_rate < 1e-8,
      "pulse: RK4 (h = 0.01) agrees with the exact solution to 1e-8")
```

800 steps of 0.01 up to $x_4 = 8$; the exact values at the same points; the largest differences are 9×10^{-11} for a_4 and 10^{-10} for a_4' (COMPUTED).

```
check(np.min(y_pulse[:, 1]) >= 1.0 - 1e-12,
      "the rate a4' never falls below H: the extra times deflate at every time")
report("rate a4'/H after the pulse (x4 = 8)", f"{y_pulse[-1, 1]:.10f}")
check(abs(y_pulse[-1, 1] - 2.0) < 1e-8,
      "after the pulse a4' = 2 H: the deflating linear member A = 2")
```

The rate never falls below 1 (the stress is never negative), and at the end it is 2.0000000000.

In [8], figure 2: the pulse history in four panels.

```
fig, axes = plt.subplots(2, 2, figsize=(10.0, 7.0))
stress_values = np.array([pulse(x, None) for x in x4_pulse])
axes[0, 0].plot(x4_pulse, stress_values)
axes[0, 0].set_ylabel("$\\kappa(p_3 - p_t)$ (units $H^2$)")
axes[0, 0].set_title("the prescribed stress pulse")
```

A 2 by 2 grid of panels; `axes[0, 0]` is the top left one. The stress is evaluated at every time (`None` stands for the unused state) and drawn.

```
every = slice(0, None, 40)  # every 40th point for the exact markers
axes[0, 1].plot(x4_pulse, y_pulse[:, 1], label="RK4")
axes[0, 1].plot(x4_pulse[every], exact[every, 1], "o", markersize=3, label="exact")
axes[0, 1].set_ylabel("$a_4'/H$")
axes[0, 1].set_title("the rate $a_4'$")
axes[0, 1].legend(fontsize=8)
axes[1, 0].plot(x4_pulse, y_pulse[:, 0], label="RK4")
axes[1, 0].plot(x4_pulse[every], exact[every, 0], "o", markersize=3, label="exact")
axes[1, 0].set_ylabel("$a_4$")
axes[1, 0].set_title("$a_4$")
axes[1, 0].legend(fontsize=8)
```

`slice(0, None, 40)` selects every 40th entry of an array. The top right panel draws the RK4 rate as a line and the exact rate as dots; the bottom left one does the same for a_4.

```
axes[1, 1].semilogy(x4_pulse, np.exp(y_pulse[:, 0]), label="3-space $e^{a_4}$")
axes[1, 1].semilogy(x4_pulse, np.exp(-y_pulse[:, 0]), "--",
                    label="extra times $e^{-a_4}$")
axes[1, 1].set_ylabel("scale factor")
axes[1, 1].set_title("the scale factors")
axes[1, 1].legend(fontsize=8)
for ax in axes[1]:
    ax.set_xlabel("time $x_4$ (units $1/H$)")
fig.tight_layout()
save_figure(fig, "stress_pulse",
            ...)
```

The bottom right panel draws both scale factors on a logarithmic axis; `axes[1]` is the bottom row, whose panels get the label of the time axis; `fig.tight_layout()` spaces the panels so that labels do not overlap. **What the figure shows.** The rate climbs from 1 to 2 while the pulse acts and stays there; a_4 bends from slope 1 to slope 2; on the logarithmic axis e^{-a_4} falls along a straight line that becomes twice as steep after the pulse: the extra times deflate at every time.

In [9], figure 3: the required source and the test of conservation.

```
def as_power_of_ten(value):
    mantissa, exponent = f"{value:.0e}".split("e")
    return f"${mantissa} \\times 10^{{{int(exponent)}}}$"
```

A small helper for the caption (docstring left out): the format `.0e` writes a number with one digit, for example `2e-11`; splitting at the letter e gives the digit and the exponent, which are put into mathematics as 2×10^{-11} (a tripled brace in an f-string prints one brace around the value).

```
rate = y_pulse[:, 1]
rho_constraint = -(rho_side(rate, *EINSTEIN) + 0.0)  # kappa rho, Lambda = 0
p0_values = p0_side(rate, *EINSTEIN) + 0.0  # kappa p0
```

```
p3_values = p0_values + stress_values / 2  # kappa p3
pt_values = p0_values - stress_values / 2  # kappa pt
mismatch = np.abs(y_pulse[:, 2] - rho_constraint)  # conservation versus constraint
say(f"largest difference of the two energy densities: {mismatch.max():.0e}")
```

Along the pulse history the four parts of the source follow from the field equations (Section 12.9; the `+ 0.0` is $\Lambda = 0$ written out): $\kappa\rho$ from the constraint, κp_0 from the hidden equation, and p_3, p_t from p_0 and the stress. `mismatch` is the difference between the energy density carried along by the conservation law (the third column of the state) and the constraint's: at most 2×10^{-11}.

```
reproduces(mismatch.max() < 1e-8,
           "the energy density from conservation equals the constraint along x4",
           WL, "constraint_propagation_bianchi")
check(abs(rho_constraint[0] + 24) < 1e-6 and abs(p0_values[0] - 12) < 1e-6,
      "before the pulse: kappa rho = -24, kappa p = 12 (the linear member A = 1)")
check(abs(rho_constraint[-1] + 33) < 1e-6 and abs(p0_values[-1] - 3) < 1e-6,
      "after the pulse: kappa rho = -33, kappa p = 3 (the linear member A = 2)")
```

The agreement is the numerical form of the record's constraint propagation (Section 12.10). The source starts at the values of $A = 1$ and ends at those of $A = 2$: $-(3 \cdot 4 + 21) = -33$ and $15 - 12 = 3$.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(x4_pulse, rho_constraint, label="$\\kappa\\rho$")
left.plot(x4_pulse, p3_values, label="$\\kappa p_3$")
left.plot(x4_pulse, pt_values, "--", label="$\\kappa p_t$")
left.plot(x4_pulse, p0_values, ":", color="black", label="$\\kappa p_0$")
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("required source (units $H^2$)")
left.set_title("the source of the pulse history")
left.legend(fontsize=8)
right.semilogy(x4_pulse, np.maximum(mismatch, 1e-16))  # exact zeros at 1e-16
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("$|\\kappa\\rho$ conserved $-$ $\\kappa\\rho$ constraint$|$")
right.set_title("conservation agrees with the constraint")
save_figure(fig, "required_source",
            ...)
```

The four source curves on the left; the mismatch on a logarithmic axis on the right, where a value of exactly zero (whose logarithm does not exist) is drawn at 10^{-16} by `np.maximum`. The caption, passed to `save_figure`, contains the measured mismatch written by `as_power_of_ten`. **What the figure shows.** Outside the pulse all three pressures coincide; during it p_3 and p_t split symmetrically about p_0, and ρ falls from -24 to -33. The mismatch stays at the level of the RK4 error.

In [10], the relaxing stress.

```
damped = {}  # eta -> (x4, solution)
for eta in (0.25, 0.5, 1.0):
    def friction(x, y, eta=eta):
        return -2.0 * eta * (y[1] - 1.0)  # kappa Delta = -2 eta (a4' - H)
```

```
    x4, y = history(friction, 2.0, 0.01, 1000)  # start on A = 2
    damped[eta] = (x4, y)
```

For three values of η the stress $\kappa\Delta = -2\eta(a_4' - 1)$ depends on the state. The argument `eta=eta` stores the current value of η in the function; without it all three functions would use the last value of the loop. Each history has 1000 steps of 0.01, up to $x_4 = 10$.

```
    exact_a4 = x4 + (1.0 / eta) * (1.0 - np.exp(-eta * x4))
    error = np.max(np.abs(y[:, 0] - exact_a4))
    mismatch = np.max(np.abs(y[:, 2] + rho_side(y[:, 1], *EINSTEIN)))
    check(error < 1e-9 and mismatch < 1e-8,
          f"eta = {eta}: a4 = x4 + (1 - e^(-eta x4))/eta, conservation holds")
    check(np.min(y[:, 1]) > 1.0,
          f"eta = {eta}: the rate a4' stays above H (the extra times deflate)")
    report(f"eta = {eta}: rate a4'/H at x4 = 10", f"{y[-1, 1]:.6f}")
```

The exact a_4 of Section 12.21 is compared, and conservation against the constraint as in In [9]. The rate stays above 1 and at $x_4 = 10$ equals $1 + e^{-10\eta}$: 1.082085, 1.006738 and 1.000045.

In [11], figure 4: the relaxing histories.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
x4 = damped[0.25][0]
for slope, style, power in ((2.0, ":", "-2x_4"), (1.0, "-.", "-x_4")):
    left.plot(x4, slope * np.ones_like(x4), style, color="black",
              label=f"linear member $A = {slope:g}$")  # the two linear members
    right.semilogy(x4, np.exp(-slope * x4), style, color="black",
                   label=f"$A = {slope:g}$: $e^{{{power}}}$")
for eta, (x4, y) in damped.items():
    left.plot(x4, y[:, 1], label=f"$\\eta = {eta}$")
    right.semilogy(x4, np.exp(-y[:, 0]), label=f"$\\eta = {eta}$")
```

The two linear members $A = 2$ and $A = 1$ in black (constant rates on the left; e^{-2x_4} and e^{-x_4} on the right, the exponent written into the label by the f-string), then the three relaxing histories: their rates on the left and their extra-time scale factors on the right.

```
left.set_ylim(0.0, 2.3)
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("$a_4'/H$")
left.set_title("the deflation rate relaxes from $2H$ to $H$")
left.legend(fontsize=8)
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("extra-time scale factor $e^{-a_4}$")
right.set_title("the extra times keep deflating")
right.legend(fontsize=8)
save_figure(fig, "damped_deflation",
            ...)
```

Labels and `save_figure`. **What the figure shows.** Each rate falls from 2 towards 1, faster for larger η. On the logarithmic axis each scale factor starts along the steeper black line and ends parallel to the flatter one: the deflation slows down but never stops.

In [12], figure 5: the convergence of RK4.

```
def relaxing_unit(x, y):
    return -2.0 * (y[1] - 1.0)  # kappa Delta = -2 eta (a4' - H) with eta = 1


exact_end = 8.0 + (1.0 - math.exp(-8.0))  # the exact a4(8) for eta = 1
steps_list = [20, 40, 80, 160, 320]  # h = 8/steps = 0.4, 0.2, 0.1, 0.05, 0.025
sizes, errors = [], []
for steps in steps_list:
    x4, y = history(relaxing_unit, 2.0, 8.0 / steps, steps)
    sizes.append(8.0 / steps)
    errors.append(abs(y[-1, 0] - exact_end))
```

The relaxing history with $\eta = 1$ is integrated to $x_4 = 8$ five times, with 20, 40, 80, 160 and 320 steps, and the error of $a_4(8)$ is stored for each step size.

```
orders = [math.log2(errors[i] / errors[i + 1]) for i in range(len(errors) - 1)]
for i, (size, error) in enumerate(zip(sizes, errors)):
    order = f"{orders[i - 1]:.2f}" if i > 0 else "-"
    say(f"h = {size:<6} error of a4(8) = {error:.2e}   measured order {order}")
check(all(3.7 < q < 4.3 for q in orders),
      "RK4 converges with order 4 (measured orders between 3.7 and 4.3)")
```

If the error is Ch^q, then halving h divides it by 2^q, and $q = \log_2(e_h/e_{h/2})$ (`math.log2` is the logarithm to base 2). The table prints each step (`:<6` pads it to six characters), its error with two decimals in powers of ten, and the measured order: 4.24, 4.12, 4.06, 4.03 (COMPUTED). The check requires every order to lie between 3.7 and 4.3.

```
fig, ax = plt.subplots()
ax.loglog(sizes, errors, "o-", label="error of $a_4(8)$")
reference = errors[-1] * (np.array(sizes) / sizes[-1]) ** 4
ax.loglog(sizes, reference, "--", color="black", label="slope 4: $C h^4$")
ax.set_xticks(sizes)  # one tick at each step size used
ax.set_xticklabels([f"{size:g}" for size in sizes])
ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())  # no extra labels
ax.set_xlabel("step size $h$ (units $1/H$)")
ax.set_ylabel("absolute error")
ax.set_title("RK4 on the relaxing history: fourth-order convergence")
ax.legend(fontsize=8)
save_figure(fig, "rk4_convergence",
            ...)
```

`loglog` uses logarithmic scales on both axes, where Ch^4 is a straight line of slope 4. The reference line passes through the last measured point. The ticks of the horizontal axis are put at the five step sizes; `NullFormatter` removes the labels of the small extra ticks that a logarithmic axis would add.
What the figure shows. The five points lie on the reference line: the method is of fourth order.

In [13], the Gauss-Bonnet breakdown.

```
STRESS, RATE0 = 0.5, 1.0  # kappa Delta and the starting rate a4'(0) = H (A = 1)


def constant_stress(x, y):
    return STRESS
```

A constant stress $\kappa\Delta = 0.5$, starting on the canonical history.

```
breakdown = {}  # alpha2 -> (x4, solution, predicted breakdown time)
for alpha2 in (0.005, 0.01):
    couplings = (1.0, alpha2, 0.0)

    def G(v, alpha2=alpha2):
        return (2 - 80 * alpha2) * v - 16 * alpha2 * v ** 3  # dG/dv = F

    critical = math.sqrt((2 - 80 * alpha2) / (48 * alpha2))  # F(critical) = 0
    x_star = (G(critical) - G(RATE0)) / STRESS  # the breakdown time
```

For two Gauss-Bonnet couplings: the function G of Section 12.21, the critical rate where $F = 0$ and the predicted breakdown time $x_4^\star$.

```
    def near_zero(y, couplings=couplings):
        return F_of(y[1], *couplings) < 0.05  # stop before F reaches zero

    x4, y = history(constant_stress, RATE0, 0.001, 20000, couplings,
                    stop=near_zero)
    breakdown[alpha2] = (x4, y, x_star)
```

`near_zero` tells `rk4` to stop when F falls below 0.05, before the division by F explodes. The integration uses the small step 0.001, because a_4'' grows steeply near the breakdown.

```
    residual = np.max(np.abs(G(y[:, 1]) - G(RATE0) - STRESS * x4))
    check(residual < 1e-6, f"alpha2 = {alpha2}: G(a4') = G(a4'(0)) + kappa Delta x4")
    check(abs(x4[-1] - x_star) < 0.01,
          f"alpha2 = {alpha2}: F reaches 0 near the predicted time x4*")
    report(f"alpha2 = {alpha2}: critical rate, breakdown time x4*",
           f"{critical:.4f} H, {x_star:.4f}/H")
```

Along the numerical history the integrated relation $G(a_4') = G(a_4'(0)) + \kappa\Delta x_4$ must hold, and the integration must stop within 0.01 of $x_4^\star$. The RESULT lines give 2.5820 and 2.4682 for $\alpha_2 = 0.005$, and 1.5811 and 0.4498 for $\alpha_2 = 0.01$.

In [14], figure 6: the breakdown compared with Einstein gravity.

```
x_einstein, y_einstein = history(constant_stress, RATE0, 0.01, 500)
check(np.max(np.abs(y_einstein[:, 1] - (RATE0 + STRESS * x_einstein / 2))) < 1e-12,
      "Einstein: a4' = 1 + 0.25 x4 for the constant stress")
```

In Einstein gravity ($F = 2$) the same stress gives $a_4' = 1 + 0.25x_4$ up to $x_4 = 5$, exactly.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(x_einstein, y_einstein[:, 1], color="black", label="Einstein")
right.plot(x_einstein, F_of(y_einstein[:, 1], *EINSTEIN) * np.ones_like(x_einstein),
           color="black", label="Einstein")
for alpha2, (x4, y, x_star) in breakdown.items():
    line = left.plot(x4, y[:, 1], label=f"$\\alpha_2H^2 = {alpha2}$")[0]
    left.axvline(x_star, linestyle="--", color=line.get_color(), linewidth=0.8)
    right.plot(x4, F_of(y[:, 1], 1.0, alpha2, 0.0), color=line.get_color(),
               label=f"$\\alpha_2H^2 = {alpha2}$")
```

The Einstein rate and $F = 2$ in black. `F_of(y_einstein[:, 1], *EINSTEIN)` is already an array with one value 2 per time, because the formula of F still contains the rate in terms multiplied by the couplings $\alpha_2 = \alpha_3 = 0$; multiplying by `np.ones_like(x_einstein)`, an array of ones of the same length, is only a safeguard, as in Notebook 12a, In [24] (Section 12.16): a formula with no rate left in it would give the single number 2. For each Gauss-Bonnet history the rate is drawn; `plot` returns a list of drawn lines, and `[0]` takes the line, whose colour `get_color()` is reused for a dashed vertical line at $x_4^\star$ (`axvline`) and for the curve of F on the right.

```
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("$a_4'/H$")
left.set_title("the rate under a constant stress")
left.legend(fontsize=8)
right.axhline(0.0, color="black", linewidth=0.8)
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("$F(a_4')$")
right.set_title("$F$ along the history")
right.legend(fontsize=8)
save_figure(fig, "gauss_bonnet_breakdown",
            ...)
```

What the figure shows. In Einstein gravity the rate rises along a straight line for ever. With the Gauss-Bonnet coupling it bends upwards ever more steeply and reaches the dashed line, the predicted $x_4^\star$, where F (right) drops to zero: the evolution equation breaks down at a finite time.

In [15], the record on the Kohn-Sham states.

```
ks = read_json(KS)
say("record: " + ks["conclusion"])
detail = {entry["name"]: entry["detail"] for entry in ks["checks"]}
verdicts = [entry["verdict"] for entry in ks["checks"]]
passed = verdicts.count("PASS")  # how many checks of the record passed
say(f"{KS}: {passed} of {len(verdicts)} checks PASS")
check(len(verdicts) > 0 and passed == len(verdicts) == ks["summary"]["checks"]
      and "ks_history_is_a_prescribed_background" in detail,
      "record: no Kohn-Sham state is an admissible source (every check PASS)")
```

The record is read and its conclusion printed. `detail` is made by a **dictionary comprehension**: a dictionary that maps the name of each check of the record to its detail text, which the next cell reads. The verdicts are counted as in Notebook 12a, In [2], and the count is printed with the name of the report. The check requires that the report has checks, that all of them passed, that the

report's own summary counts the same number (a chained comparison `a == b == c` is true when $a = b$ and $b = c$), and that the check stating the prescribed background is present (`in` tests whether a name is a key of the dictionary).

In [16], the record's numbers recomputed from the Kohn-Sham table.

```
import csv  # reads tables of comma-separated values
import re  # regular expressions: find a pattern in a text

TABLE = "Revision/kohn_sham/results/ground/emt-integrals.csv"
with repository_file(TABLE).open(encoding="utf-8", newline="") as handle:
    table = {row["id"]: row for row in csv.DictReader(handle)}
INTEGRALS = ("int_rho", "int_p3", "int_p_t", "int_p0")  # 2 Vol_7 int e^(6Hy) T dy
```

`csv` and `re` come with Python. The `with` block opens the table of the Kohn-Sham record and closes it again at the end of the block. `csv.DictReader` reads the table line by line: its first line holds the names of the columns, and every further line becomes a dictionary from column name to text. `table` maps the name of each state (column `id`) to its row. `INTEGRALS` names the four columns of the integrated energy density and pressures.

```
zero = sorted(name for name, row in table.items()
              if all(float(row[column]) == 0.0 for column in INTEGRALS))
nonzero = [name for name in table if name not in zero]
report("Kohn-Sham states in the table", len(table))
report("states with a nonzero energy-momentum tensor", len(nonzero))
say("states with a zero energy-momentum tensor: " + ", ".join(zero))
```

`zero` lists, in alphabetical order, the states whose four integrals are all exactly zero (`float` turns a text such as `0.000000000000000e0` into the number 0; `all` is true when every one of its values is true); `nonzero` holds the other states. Out [16] reports 75 states, 70 of them with a nonzero energy-momentum tensor, and names the five others: the five slices of the state with $N = 8$ particles and $\lambda = 0$.

```
counted = re.search(r"(\d+) ground-state profiles .*?\((\d+) with a nonzero",
                    detail["ks_profiles_depend_on_x0"])
reproduces(counted is not None
           and counted.groups() == (str(len(table)), str(len(nonzero))),
           f"{len(nonzero)} of the {len(table)} states have a nonzero tensor",
           KS, "ks_profiles_depend_on_x0")
listed = detail["ks_zero_source_states_listed"].rsplit(": ", 1)[-1].split(", ")
reproduces(sorted(listed) == zero,
           "the states with a zero tensor are the ones the record lists",
           KS, "ks_zero_source_states_listed")
```

The record writes its two counts into the detail text of its check `ks_profiles_depend_on_x0`, in the words "75 ground-state profiles ... (70 with a nonzero energy-momentum tensor)". `re.search` looks for a **pattern** in that text: `\d+` matches one or more digits, round brackets around a part of the pattern **capture** what that part matches, `.*?` matches any text up to the next part of the pattern, and `\(` matches a round bracket itself. `counted.groups()` returns the two captured numbers as

texts, which must equal our two counts. The detail text of the check `ks_zero_source_states_listed` ends, after its last colon, with the list of the states whose tensor is zero: `rsplit(":  ", 1)[-1]` takes the text after the last colon and space, and `split(", ")` cuts it at the commas. Both comparisons are made with `reproduces`, so the record's checks must also have the verdict PASS.

```
ratio = {}  # r = (int p3 + int p_t)/(2 int p0) of every nonzero state
for name in nonzero:
    row = table[name]
    if float(row["int_p0"]) != 0.0:
        ratio[name] = ((float(row["int_p3"]) + float(row["int_p_t"]))
                       / (2 * float(row["int_p0"])))
closest = min(ratio, key=lambda name: abs(ratio[name] - 1.0))
report(f"closest to 1: r of {closest}", f"{ratio[closest]:.6g}")
HISTORY = ["N136_lam0_a00", "N136_lam0_a10", "N136_lam0_a20"]  # N = 136, lambda = 0
for name in HISTORY:
    slice_a4 = float(table[name]["a4"])  # the slice a4,0 of this state
    report(f"N = 136, lambda = 0, a4,0 = {slice_a4:g}: r", f"{ratio[name]:.6g}")
```

For every nonzero state with $\int p_0 \neq 0$ the ratio $r = (\int p_3 + \int p_t)/(2 \int p_0)$ is computed. `min(ratio, key=...)` returns the name whose ratio has the smallest distance $|r - 1|$ from 1 (`lambda name: ...` is a small function written in one line). The format `.6g` prints six significant digits, as the record does, and `:g` prints the slice $a_{4,0}$ (column `a4`) without trailing zeros. Out [16] shows the state closest to 1, N688_lamm2_a00, with $r = 0.414328$, and for the history $N = 136$, $\lambda = 0$ the ratios 0.339767, 0.25969 and 0.239714 at $a_{4,0} = 0, 1, 2$: far from the required 1 at every slice.

```
text = detail["ks_integrals_violate_algebraic_condition"]
tolerance = float(ks["tolerance"].split()[-1])  # "relative 1e-06" gives 1e-06
best = re.search(r"closest to 1: (\S+) at (\w+)", text)
reproduces(len(ratio) == len(nonzero) and abs(ratio[closest] - 1.0) > tolerance
           and best is not None
           and best.groups() == (f"{ratio[closest]:.6g}", closest),
           "r differs from 1 for every nonzero state; the closest as recorded",
           KS, "ks_integrals_violate_algebraic_condition")
found = [re.search(name + r": (\S+?)[,\s]", text) for name in HISTORY]
reproduces(all(item is not None and item.group(1) == f"{ratio[name]:.6g}"
               for item, name in zip(found, HISTORY)),
           "r of N = 136, lambda = 0 at a4,0 = 0, 1, 2 equals the record",
           KS, "ks_integrals_violate_algebraic_condition")
```

The record's tolerance is read from its text "relative 1e-06" (`split()` cuts the text at the spaces, `[-1]` takes the last part). The first check requires a ratio for every nonzero state, that even the closest one differs from 1 by more than the tolerance, and that the record names the same closest state with the same six digits (`\S+` matches characters that are not spaces, `\w+` letters, digits and underscores). The second check finds in the record's text, for each state of the history, the number after its name and a colon (`\S+?` takes as few characters as possible, up to the comma or space that `[,\s]` matches) and requires it to be exactly our number printed with six digits; `zip` walks through the two lists side by side. A change of the Kohn-Sham record, or of its checker, that moved any of these numbers would stop the notebook here.

In [17], the last check.

```
figure_names = ["linear_member", "stress_pulse", "required_source",
                "damped_deflation", "rk4_convergence", "gauss_bonnet_breakdown"]
paths = [output_file(f"{FIGURE_FOLDER}/12c_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths), "all six figure files exist")
all_checks_passed()
```

The same pattern as In [15] of Notebook 12b (Section 12.20): `figure_names` lists the short names of the six figures in the order in which they were drawn, `enumerate(figure_names, 1)` numbers them from 1, which gives the six file names `12c_1_linear_member.png` to `12c_6_gauss_bonnet_breakdown.png`, and `output_file` gives the path where each was written. `path.is_file()` is true when the file exists, and `all` requires this of every one of the six. `all_checks_passed()` prints the last line, ALL 30 CHECKS PASSED (notebook 12c): 1 check in In [3], 1 in In [4], 4 in In [5], 3 in In [7], 3 in In [9], 6 in In [10], 1 in In [12], 4 in In [13], 1 in In [14], 1 in In [15], 4 in In [16] and 1 in In [17].

12.26 A condensate of dirac16complex00 as an exact source

The previous examples prescribed the source. Can a field of this theory be the source, exactly? The Revision record proves three facts about the commuting field dirac16complex00 (Chapter 7), and leaves one question open. This section derives what the last notebook needs.

The field and its equation. dirac16complex00 is a column Φ of 16 complex numbers at every point, which transform as a Pin(4,4) spinor. The gamma matrices are the author's eight real 16 by 16 matrices $\gamma^{(x_0)}, \ldots, \gamma^{(x_7)}$ of the record `Revision/algebra/gammas.json` (built from the author's formulas in Chapter 4), which satisfy the Clifford relation $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}$ with $\eta = \mathrm{diag}(+,+,+,+,-,-,-,-)$ (record checks `authorT16_clifford`; Notebook 12d, In [3]). The adjoint is $\bar{\Phi} = \Phi^\dagger C$ with the real symmetric matrix $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, $C^2 = 1$, and $C\gamma^{(a)}$ antisymmetric for every a (record check `authorT16_C_properties`). The density $S = \bar{\Phi}\Phi$ is a real number. With the potential $U = \frac{\lambda}{2}S^2$ the field equation of the record is

$$\gamma^\mu D_\mu \Phi = (m + \lambda S)\,\Phi = M\,\Phi, \qquad D_\mu = \partial_\mu + \Omega_\mu,$$

where $M = m + \lambda S$ is the **effective mass**, $\gamma^\mu = \gamma^{(\mu)}/\sqrt{|g_{\mu\mu}|}$ are the gammas of the curved metric and Ω_μ is the spin connection of Chapter 6. For the author's metric, with the author's gammas, the spin connection enters the equation only through one term (PROVED: sympy record, check `authorT16_gravity_term`; lead's record, check `gamma_Omega_equals_3H_gamma0`; Notebook 12d, In [5]):

$$\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}.$$

The condensate equation. For a condensate, $\Phi = \Phi(x_4)$, only the derivative along x_4 survives, and $\gamma^{x_4} = \gamma^{(x_4)}$ because $g_{44} = -1$. Line by line:

$$\gamma^{(x_4)}\Phi' + 3H\gamma^{(x_0)}\Phi = M\Phi$$

(the field equation for $\Phi(x_4)$, with the spin-connection term inserted)

$$\gamma^{(x_4)}\Phi' = \big(M - 3H\gamma^{(x_0)}\big)\Phi$$

(subtract $3H\gamma^{(x_0)}\Phi$ from both sides)

$$\Phi' = -\gamma^{(x_4)}\big(M - 3H\gamma^{(x_0)}\big)\Phi =: \mathcal{A}\,\Phi$$

(multiply from the left by $-\gamma^{(x_4)}$, since $(-\gamma^{(x_4)})\gamma^{(x_4)} = -\eta^{44} = 1$). The 16 by 16 matrix $\mathcal{A}$ contains neither a_4 nor z: the condensate does not feel the deflation (record check `condensate_equation_x0_consistent`).

Its square. Line by line:

$$\mathcal{A}^2 = \gamma^{(x_4)}\big(M - 3H\gamma^{(x_0)}\big)\gamma^{(x_4)}\big(M - 3H\gamma^{(x_0)}\big)$$

(the two minus signs multiply to plus)

$$= \gamma^{(x_4)}\gamma^{(x_4)}\big(M + 3H\gamma^{(x_0)}\big)\big(M - 3H\gamma^{(x_0)}\big)$$

(move the middle $\gamma^{(x_4)}$ to the left: $\gamma^{(x_0)}\gamma^{(x_4)} = -\gamma^{(x_4)}\gamma^{(x_0)}$ by the Clifford relation, so $(M - 3H\gamma^{(x_0)})\gamma^{(x_4)} = \gamma^{(x_4)}(M + 3H\gamma^{(x_0)})$)

$$= (-1)\big(M^2 - 3HM\gamma^{(x_0)} + 3HM\gamma^{(x_0)} - 9H^2(\gamma^{(x_0)})^2\big) = -\big(M^2 - 9H^2\big)$$

($(\gamma^{(x_4)})^2 = -1$, $(\gamma^{(x_0)})^2 = +1$; the middle terms cancel). So for $M^2 > 9H^2$, with the frequency $w = \sqrt{M^2 - 9H^2}$, the condensate oscillates. If $\mathcal{A}v = \mu v$ for a nonzero vector v, then $\mu^2 v = \mathcal{A}^2 v = -w^2 v$, so every eigenvalue of $\mathcal{A}$ is $+iw$ or $-iw$. Both occur, each 8 times: $\mathcal{A}$ is real, so its characteristic polynomial $\det(\mu - \mathcal{A})$ has real coefficients, and the non-real root iw comes together with its complex conjugate $-iw$, with the same multiplicity; the 16 roots therefore split as $8 + 8$ (Exercise 10 gives a second proof with the trace; Notebook 12d, In [7], counts eight and eight numerically). Every vector u is a sum of eigenvectors of the two kinds, $u = u_- + u_+$ with $u_- = \frac{1}{2}(u + \frac{i}{w}\mathcal{A}u)$ and $u_+ = \frac{1}{2}(u - \frac{i}{w}\mathcal{A}u)$: indeed $\mathcal{A}u_- = \frac{1}{2}(\mathcal{A}u + \frac{i}{w}\mathcal{A}^2 u) = \frac{1}{2}(\mathcal{A}u - iwu) = -iw\,u_-$ (with $\mathcal{A}^2 = -w^2$), and in the same way $\mathcal{A}u_+ = +iw\,u_+$. An eigenvector Φ_0 with $\mathcal{A}\Phi_0 = -iw\Phi_0$ gives the solution $\Phi(x_4) = e^{-iwx_4}\Phi_0$ (its derivative is $-iw\Phi = \mathcal{A}\Phi$). For $M = \pm 5H$, $w = 4H$. The hidden-direction term moves the threshold of oscillation from $|M| = 0$ to $|M| = 3H$ (Notebook 12d, In [6] and figure 1).

The density is constant. Line by line:

$$S' = (\Phi')^\dagger C\Phi + \Phi^\dagger C\Phi' = \Phi^\dagger(\mathcal{A}^T C + C\mathcal{A})\Phi$$

(product rule; $(\mathcal{A}\Phi)^\dagger = \Phi^\dagger\mathcal{A}^\dagger$, and $\mathcal{A}^\dagger = \mathcal{A}^T$ because $\mathcal{A}$ is real). Now $\mathcal{A} = -M\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}$, and $C\gamma^{(a)}$ antisymmetric means $(\gamma^{(a)})^T C = -C\gamma^{(a)}$ (transpose $(C\gamma^{(a)})^T = (\gamma^{(a)})^T C^T$ and $C^T = C$). Hence $(\gamma^{(x_4)})^T C + C\gamma^{(x_4)} = 0$, and

$$(\gamma^{(x_4)}\gamma^{(x_0)})^T C = (\gamma^{(x_0)})^T(\gamma^{(x_4)})^T C = -(\gamma^{(x_0)})^T C\gamma^{(x_4)} = C\gamma^{(x_0)}\gamma^{(x_4)} = -C\gamma^{(x_4)}\gamma^{(x_0)}$$

(transpose of a product reverses the order; the rule above twice; the Clifford relation), so $\mathcal{A}^T C + C\mathcal{A} = 0$ and $S' = 0$ (record check `authorT16_condensate_S_constant`).

Its energy-momentum tensor. The record writes the source as $T^{\mu}{}_{\nu} = \sigma_T\big(-K^{(\mu}{}_{\nu)} + \delta^{\mu}_{\nu} L\big)$ with the kinetic tensor $K^{\mu}{}_{\nu} = \frac{1}{2}\big(\bar{\Phi}\gamma^{\mu} D_{\nu}\Phi - (D_{\nu}\bar{\Phi})\gamma^{\mu}\Phi\big)$, its symmetrised form $K^{(\mu}{}_{\nu)}$, and the on-shell Lagrangian $L = \sum_{\mu} K^{\mu}{}_{\mu} - mS - U$. The overall sign $\sigma_T = +1$ is the record's declared convention and is not fixed by a Revision check: it is ASSUMED. For a condensate the record proves that the diagonal of K is $K^{x_4}{}_{x_4} = MS$ and zero in the seven other directions, and that $K^{x_4}{}_{x_0} = 0$ (check `authorT16_condensate_kinetic_diagonal`). Then, line by line:

$$L = MS - mS - \tfrac{\lambda}{2}S^2 = (m + \lambda S)S - mS - \tfrac{\lambda}{2}S^2 = \tfrac{\lambda}{2}S^2$$

(the definition, $M = m + \lambda S$, and $\lambda S^2 - \frac{\lambda}{2}S^2 = \frac{\lambda}{2}S^2$), and

$$\rho = -T^{x_4}{}_{x_4} = K^{x_4}{}_{x_4} - L = MS - \tfrac{\lambda}{2}S^2 = mS + \tfrac{\lambda}{2}S^2, \qquad p_3 = p_t = p_0 = L = \tfrac{\lambda}{2}S^2.$$

So the condensate supplies exactly what the linear member needs: equal pressures and constant ρ and p. But the 42 nonzero off-diagonal components of K are multiples of 15 **three-gamma bilinears** $\bar{\Phi}\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$: the six with $\{a,b,c\} = \{i, x_4, x_0\}$ (i a 3-space direction or an extra time) and the nine with $\{i, j, x_4\}$ (i in 3-space, j an extra time). The off-diagonal field equations $0 = \kappa T^{\mu}{}_{\nu}$ need all 15 to vanish (Wolfram record, check `condensate_offdiagonal_are_three_gamma_bilinears`). The record shows that condensates with all 15 zero exist (checks `condensate_diagonal_witness_exact` and `authorT16_condensate_witness`), and it proves that such a condensate allows only the linear member (key `theoremLinear` of `Revision/field_equations_a4/a4-equations.json`), under exact hypotheses. *Hypotheses*: Φ is a homogeneous condensate as above (classical, commuting) that solves its field equation and satisfies the off-diagonal conditions (all 15 bilinears zero); a_4 is twice continuously differentiable; the couplings are not all zero, $(\alpha_1, \alpha_2, \alpha_3) \neq (0,0,0)$. *Conclusion*: $a_4 = AHx_4 + a_0$ with real constants A and a_0. *Proof*: the condensate has $p_3 = p_t$, so the evolution equation gives $a_4'' F(a_4') = 0$ at every time. Suppose $a_4'' \neq 0$ at some time. Since a_4'' is continuous, $a_4'' \neq 0$ on a whole interval around it, so $F(a_4') = 0$ there: a_4' takes only values among the roots of F. F is a polynomial in a_4' that is not identically zero when the couplings are not all zero (record check `evolution_F_not_identically_zero`), so it has finitely many roots. A continuous function on an interval that takes only finitely many values is constant there (between two different values it would have to pass through all the values in between). So a_4' is constant on the interval and $a_4'' = 0$ there, a contradiction. Hence $a_4'' = 0$ at every time, $a_4' = AH$ is constant and $a_4 = AHx_4 + a_0$.

The two Einstein conditions. For the linear member the Einstein equations with this source are the time equation and seven equal equations (Section 12.12):

$$(21 + 3A^2)H^2 + \Lambda = -\kappa\rho, \qquad (15 - 3A^2)H^2 + \Lambda = \kappa p.$$

Subtracting the second from the first:

$$(21 + 3A^2 - 15 + 3A^2)H^2 = -\kappa(\rho + p) \;\Rightarrow\; \kappa MS = -6(A^2 + 1)H^2$$

(Λ cancels; $\rho + p = mS + \lambda S^2 = (m + \lambda S)S = MS$). Adding them:

$$(21+3A^2+15-3A^2)H^2+2\Lambda=-\kappa(\rho-p)\ \Rightarrow\ \kappa mS=-(36H^2+2\Lambda)$$

($\rho-p=mS$). These are the record's two conditions (PROVED: check `condensate_einstein_quadratic_U` in both reports; Notebook 12b, In [13], and Notebook 12d, In [13]). The first says that a real slope needs $\kappa MS\le -6H^2$: the density and the effective mass must have opposite signs. Given A, Λ and M they fix $S=-6(A^2+1)H^2/(\kappa M)$, then $m=-(36H^2+2\Lambda)/(\kappa S)$, then $\lambda=(M-m)/S$.

What the record leaves open. The record's document `Revision/docs/DIRAC16COMPLEX00_FIELD_THEORY.md` says: "This record contains no check that combines a witness with the Einstein conditions." Notebook 12d makes that combination. It needs condensates with all 15 bilinears zero AND a density of the required sign and size. It builds them from two eigenvectors v_1, v_2 of $\mathcal{A}$ with the same frequency (the record's recipe; Section 12.30, In [8], explains how they are found) as the family $\Phi_0(t)=v_1+t\,c\,v_2$ with a real parameter t and $c=\overline{v_1^\dagger C v_2}$ (the bar is the complex conjugate). Its density, with $\sigma=v_1^\dagger C v_2$ and the notebook's finding $v_1^\dagger C v_1=v_2^\dagger C v_2=0$:

$$S(t)=v_1^\dagger C v_1+t\,c\,v_1^\dagger C v_2+t\,\bar c\,v_2^\dagger C v_1+t^2|c|^2\,v_2^\dagger C v_2=t\,\bar\sigma\sigma+t\,\sigma\bar\sigma=2t\,|\sigma|^2$$

(expand the product; $c=\bar\sigma$; $v_2^\dagger C v_1=\bar\sigma$ because C is real and symmetric). So every real density, positive or negative, occurs. With the record's $H=\kappa=1$ and $\sigma=-128/25-96i/25$ at $M=-5$: $|\sigma|^2=(128^2+96^2)/625=1024/25$ and $S(t)=2048t/25$ (Notebook 12d, In [9]).

Three exact solutions (units $H=\kappa=1$):

example	A	Λ	M	S	m	λ	(ρ,p)
1	1	0	-5	$12/5$	-15	$25/6$	$(-24,12)$
2	$\sqrt{5}$	0	5	$-36/5$	5	0	$(-36,0)$
3	1	-30	-5	$12/5$	10	$-25/4$	$(6,-18)$

The equations of state $w=p/\rho$ are $-1/2$, 0 and -3. Example 1 by hand: $S=-6(1+1)/(-5)=12/5$; $m=-36/S=-36\cdot 5/12=-15$; $\lambda=(M-m)/S=(-5+15)\cdot 5/12=25/6$; $\rho=mS+\frac{\lambda}{2}S^2=-36+\frac{25}{12}\cdot\frac{144}{25}=-36+12=-24$; $p=12$. Examples 1 and 3 have the canonical history $a_4=Hx_4$: the extra times deflate as e^{-Hx_4}. Notebook 12d checks, exactly and for every time and every value of the hidden coordinate, all 16 components of the field equation and all 64 components of the Einstein equations, for $+A$ and for $-A$ (In [15]).

Status. These three solutions are PROVED for the stated numbers by exact computer algebra, under ASSUMED inputs: Einstein gravity, the sign convention $\sigma_T=+1$, the classical commuting field dirac16complex00 (not the quantised dirac16complex), and the chosen values of Λ, m and λ. They are a computation of this book, not a Revision record. The equations do not select the sign of A: the same condensate solves them with $-A$. With $\Lambda=0$ the energy density is negative; a positive one (example 3) needs $\Lambda<-(21+3A^2)H^2$ and then has $w<-1$. Not shown: whether such a condensate is stable, how it could arise, anything about the quantised field, and anything about the creation of universes.

12.27 Example: three exact solutions of the coupled equations

The last notebook rebuilds, with exact rational and complex-rational arithmetic and the author's eight real gamma matrices, the spin-connection term, the condensate equation and its solution, builds the family of condensates with all 15 three-gamma bilinears zero, computes their complete energy-momentum tensor in the author's metric with an arbitrary $a_4(x_4)$, solves the two Einstein conditions and checks the three exact solutions of Section 12.26 component by component. It runs in about 25 seconds, prints 40 PASS lines and draws six figures.

12.28 How to run Notebook 12d

Step 1. What this notebook does and what it needs.

Notebook 12d (An exact solution: a condensate of dirac16complex00 as the source of the deflating member) is the file `Revision/textbook/notebooks/12d_condensate_solution.ipynb` of the repository Dirac_claude. It builds, with exact arithmetic and the author's gamma matrices, homogeneous condensates of the commuting field dirac16complex00 whose 15 three-gamma bilinears vanish, computes their complete energy-momentum tensor in the author's metric, chooses the mass, the self-coupling and the cosmological constant so that the Einstein equations hold, and checks every component of the field equation of the condensate and of the Einstein equations for the exponentially deflating linear member a4 = H x4 (and a4 = sqrt(5) H x4), at every time x4 and every value of the hidden coordinate. It reproduces the Revision records where they overlap and draws six teaching plots. The combination of a condensate with the Einstein conditions is a computation of this notebook; the Revision record states that it does not contain one. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two

commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 12d_condensate_solution.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 25 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12d_condensate_solution.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 12d_condensate_solution.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/12d.captions.json`
- `Revision/textbook/figures/12d_1_condensate_frequency.png`
- `Revision/textbook/figures/12d_2_three_gamma_bilinears.png`
- `Revision/textbook/figures/12d_3_allowed_sources.png`
- `Revision/textbook/figures/12d_4_tensor_equality.png`
- `Revision/textbook/figures/12d_5_solution_in_time.png`
- `Revision/textbook/figures/12d_6_family_in_lambda.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 40 CHECKS PASSED (notebook 12d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/12d_condensate_solution.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 12d_condensate_solution.ipynb
python -m nbconvert --execute --inplace 12d_condensate_solution.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

12.29 Notebook 12d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/12d_condensate_solution.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The other notebooks of this chapter derive the field equations for $a_4(x_4)$ and
the source that a given $a_4$ requires. Here we ask whether a field of the theory
can BE that source, exactly, with the extra times deflating exponentially. The
Revision record proves three things about a homogeneous condensate of the commuting
field dirac16complex00 (a solution $\Phi(x_4)$ that depends on the time only):

1. it can be a source of the author's metric only for the linear member
   $a_4 = AHx_4 + a_0$, and then only if 15 numbers built from it, the three-gamma
   bilinears, are zero;
2. in Einstein gravity it must then meet two conditions that tie its mass $m$,
   its self-coupling $\lambda$ and its density $S$ to $A$ and $\Lambda$;
3. condensates with all 15 bilinears zero exist (exact examples).

The record states that it contains no check that combines 3 with 2. This notebook
does that combination, with exact arithmetic (rational and complex-rational
numbers, never rounded) and the author's gamma matrices. It

- rebuilds the spin-connection term $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ and
  the condensate equation, and solves it for every starting spinor;
- builds a one-parameter family of condensates with all 15 bilinears zero and a
  density $S$ of either sign;
- computes their complete energy-momentum tensor (all 64 components) in the
  author's metric with an arbitrary $a_4(x_4)$;
- solves the two Einstein conditions and finds three exact solutions of the
  coupled equations, two of them with the extra times deflating as $e^{-Hx_4}$;
- checks all 16 components of the field equation of the condensate and all 64
  components of the Einstein equations, at every time $x_4$ and every value of
  the hidden coordinate;
- states exactly what this shows and what it does not, and draws six plots.
```

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 12d (An exact solution: a condensate of dirac16complex00 as the source of the deflating member) is the file ‘Revision/textbook/notebooks/12d_condensate_solution.ipynb‘ of the repository Dirac_claude. It builds, with exact arithmetic and the author’s gamma matrices, homogeneous condensates of the commuting field dirac16complex00 whose 15 three-gamma bilinears vanish, computes their complete energy-momentum tensor in the author’s metric, chooses the mass, the self-coupling and the cosmological constant so that the Einstein equations hold, and checks every component of the field equation of the condensate and of the Einstein equations for the exponentially deflating linear member a4 = H x4 (and a4 = sqrt(5) H x4), at every time x4 and every value of the hidden coordinate. It reproduces the Revision records where they overlap and draws six teaching plots. The combination of a condensate with the Einstein conditions is a computation of this notebook; the Revision record states that it does not contain one. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
```

```
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 12d_condensate_solution.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 25 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 12d_condensate_solution.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 12d_condensate_solution.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/12d.captions.json‘
- ‘Revision/textbook/figures/12d_1_condensate_frequency.png‘
- ‘Revision/textbook/figures/12d_2_three_gamma_bilinears.png‘
- ‘Revision/textbook/figures/12d_3_allowed_sources.png‘
- ‘Revision/textbook/figures/12d_4_tensor_equality.png‘
- ‘Revision/textbook/figures/12d_5_solution_in_time.png‘
- ‘Revision/textbook/figures/12d_6_family_in_lambda.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files exist
ALL 40 CHECKS PASSED (notebook 12d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
    python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/12d_condensate_solution.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
    python -m jupyterlab 12d_condensate_solution.ipynb
    python -m nbconvert --execute --inplace 12d_condensate_solution.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 12d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 12d (An exact solution: a condensate of dirac16complex00 as the source of the
# deflating member) is the file Revision/textbook/notebooks/12d_condensate_solution.ipynb
```

```
# of the repository Dirac_claude. It builds, with exact arithmetic and the author's gamma
# matrices, homogeneous condensates of the commuting field dirac16complex00 whose 15
# three-gamma bilinears vanish, computes their complete energy-momentum tensor in the
# author's metric, chooses the mass, the self-coupling and the cosmological constant so
# that the Einstein equations hold, and checks every component of the field equation of
# the condensate and of the Einstein equations for the exponentially deflating linear
# member a4 = H x4 (and a4 = sqrt(5) H x4), at every time x4 and every value of the
# hidden coordinate. It reproduces the Revision records where they overlap and draws six
# teaching plots. The combination of a condensate with the Einstein conditions is a
# computation of this notebook; the Revision record states that it does not contain one.
# It needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
```

```
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 12d_condensate_solution.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 25
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 12d_condensate_solution.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 12d_condensate_solution.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
```

```
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/12d.captions.json
# - Revision/textbook/figures/12d_1_condensate_frequency.png
# - Revision/textbook/figures/12d_2_three_gamma_bilinears.png
# - Revision/textbook/figures/12d_3_allowed_sources.png
# - Revision/textbook/figures/12d_4_tensor_equality.png
# - Revision/textbook/figures/12d_5_solution_in_time.png
# - Revision/textbook/figures/12d_6_family_in_lambda.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files exist
#     ALL 40 CHECKS PASSED (notebook 12d)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/12d_condensate_solution.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
```

```
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 12d_condensate_solution.ipynb
#     python -m nbconvert --execute --inplace 12d_condensate_solution.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file below the folder Revision: the notebook reads the
#   Revision records of the repository. Run it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone,
#   not on a copy of the notebook file alone.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "12d"  # this notebook: chapter 12, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
```

```
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
```

```
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 12d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$ (the author's names): $x_1, x_2, x_3$ ordinary
  3-space, $x_4$ the time, $x_5, x_6, x_7$ the three extra times (which deflate
  exponentially), $x_0$ the hidden direction with $z = 6Hx_0$ in $(0, \pi/2)$. In the
  code the positions 0 to 7 of a list stand for $x_0$ to $x_7$.
- **Gamma matrices** $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$: the author's eight real
  16 by 16 matrices with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} =
  2\eta^{ab}$, $\eta = \mathrm{diag}(+,+,+,+,-,-,-,-)$ (the Clifford relation).
- **Spinor** $\Phi$: a column of 16 complex numbers (here: functions of $x_4$).
  dirac16complex00 is a field of such spinors whose components are ordinary
  (commuting) complex numbers.
- **Adjoint** $\bar\Phi = \Phi^\dagger C$ with $C = \gamma^{(x_0)}\gamma^{(x_1)}
  \gamma^{(x_2)}\gamma^{(x_3)}$; $\Phi^\dagger$ is the row of complex conjugates.
```

```
- **Density** $S = \bar\Phi\Phi$, a real number. **Bilinear**: a number
  $\bar\Phi X \Phi$ with a 16 by 16 matrix $X$; a **three-gamma bilinear** has
  $X = \gamma^{(a)}\gamma^{(b)}\gamma^{(c)}$ with three different directions.
- **Mass** $m$, **self-coupling** $\lambda$, **potential** $U(S) = \tfrac\lambda2 S^2$
  with $U'(S) = \lambda S$; **effective mass** $M = m + \lambda S$.
- **Spin connection** $\Omega_\mu$: the 16 by 16 matrices that make the derivative
  of a spinor covariant, $D_\mu\Phi = \partial_\mu\Phi + \Omega_\mu\Phi$.
- **Condensate**: a solution that depends on the time $x_4$ only.
- **Frequency** $w$: in $e^{-iwx_4}$, the number of radians per unit of time.
- **Energy-momentum tensor** $T^\mu{}_\nu$: the energy density
  $\rho = -T^{x_4}{}_{x_4}$, the pressures $p_\mu = T^\mu{}_\mu$ (no sum), and the
  mixed components; **sign convention** $\sigma_T = +1$ of the Revision record.
- **Einstein equations** with cosmological constant $\Lambda$ and coupling
  $\kappa$: $G^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu$.
- **Linear member** $a_4 = AHx_4 + a_0$: for $A > 0$ the extra times deflate as
  $e^{-AHx_4}$ while 3-space inflates as $e^{AHx_4}$.
- **Witness**: an explicit example that proves that something exists.
- **Units**: $H = 1$ and $\kappa = 1$ in the examples ($\kappa$ and the size of
  $\Phi$ enter the equations only through $\kappa S$).
```

Text cell:

```
## 4. The physical and mathematical situation

The coupled equations are, for the field (the Euler-Lagrange equation of the
Revision record, with $\gamma^\mu = e_a{}^\mu\gamma^{(a)}$ the gammas of the curved
metric),

$$\gamma^\mu D_\mu\Phi = (m + U'(S))\,\Phi,$$

and for the metric, in Einstein gravity,

$$G^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu,\qquad
T^\mu{}_\nu = \sigma_T\big(-K^{(\mu}{}_{\nu)} + \delta^\mu_\nu L\big),$$

$$K^\mu{}_\nu = \tfrac12\big(\bar\Phi\gamma^\mu D_\nu\Phi
- (D_\nu\bar\Phi)\gamma^\mu\Phi\big),\qquad
L = \sum_\mu K^\mu{}_\mu - mS - U(S),$$

where $K^{(\mu}{}_{\nu)}$ is the symmetrised kinetic tensor and $L$ the Lagrangian
on shell (the record's formulas; $\sigma_T = +1$). For the linear member the
Einstein tensor is diagonal with $G^{x_4}{}_{x_4} = (21 + 3A^2)H^2$ and all seven
other diagonal entries $(15 - 3A^2)H^2$.

**Status.** Every check below is an exact identity verified by computer algebra
(PROVED for the stated numbers). ASSUMED: Einstein gravity, the sign convention
$\sigma_T = +1$, the classical commuting field dirac16complex00 (not the quantised
dirac16complex), and the values of $\Lambda$, $m$, $\lambda$ chosen below. The
combination of a condensate with the Einstein conditions is new in this notebook;
it is not a Revision record.
```

Text cell:

5. The Revision records this notebook reproduces

The next cell defines the helpers that read the Revision records: ‘read_json‘
reads a JSON file of the repository, ‘record_verdict‘ finds a check by its name in
a report, and ‘reproduces‘ is a check that passes only when this notebook’s own
result holds AND the record lists the named check with the verdict PASS (it
collects its two printed lines in a text buffer and prints them with one call, so
that they always stay together in the output). It then
prints the sentence of the record’s document that this notebook answers.

In [2]:

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory

GAMMAS = "Revision/algebra/gammas.json"  # the author's gamma matrices
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
DOC = "Revision/docs/DIRAC16COMPLEX00_FIELD_THEORY.md"  # the record's document


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict of the check name in a report ("PASS"), None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS.  Its
    printed lines (PASS and reproduces) are collected in a text buffer and printed
    by one print call, so that they always stay together in the cell's output."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name, record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # all lines at once


OPEN_SENTENCE = ("This record contains no check that combines a witness with the "
                 "Einstein conditions")
document = repository_file(DOC).read_text(encoding="utf-8")
say(f"{DOC} says: \"{OPEN_SENTENCE}.\"")
check(OPEN_SENTENCE in document, "the record leaves this combination open")
```

Out [2]:

```
Revision/docs/DIRAC16COMPLEX00_FIELD_THEORY.md says: "This record contains no check that
    combines a witness with the Einstein conditions."
PASS the record leaves this combination open
```

Text cell:

```
## 6. The author's gamma matrices and the matrix $C$

The next cell reads the eight gamma matrices from the Revision record
`gammas.json` (each entry an integer or a fraction written as text, which
`sp.Rational` turns into an exact number), checks the Clifford relation for all 64
pairs, builds $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$,
compares it with the $C$ stored in the record, and checks the three properties the
adjoint needs: $C$ is real and symmetric, $C^2 = 1$, and every $C\gamma^{(a)}$ is
antisymmetric.
```

In [3]:

```
import itertools  # loops over all combinations of indices

import numpy as np  # floating-point arrays, used only for the plots
import sympy as sp  # exact algebra

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the frame metric eta, x0..x7
I16, Z16 = sp.eye(16), sp.zeros(16, 16)  # the 16 by 16 unit and zero matrices
fixture = read_json(GAMMAS)
gamma = [sp.Matrix([[sp.Rational(x) for x in row] for row in matrix])
         for matrix in fixture["gamma"]]  # gamma[a] = gamma^(xa)
clifford = all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
               == 2 * ETA[a] * (1 if a == b else 0) * I16
               for a in range(8) for b in range(8))
reproduces(clifford, "the author's gammas satisfy the Clifford relation",
           PY, "authorT16_clifford")
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g^(x0) g^(x1) g^(x2) g^(x3)
stored_C = sp.Matrix([[sp.Rational(x) for x in row] for row in fixture["C"]])
properties = (C == stored_C and C == C.T and C * C == I16
              and all((C * g).T == -(C * g) for g in gamma))
reproduces(properties, "C is the record's C, real symmetric, C^2 = 1, C gamma "
           "antisymmetric", PY, "authorT16_C_properties")
```

Out [3]:

```
PASS the author's gammas satisfy the Clifford relation
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_clifford
PASS C is the record's C, real symmetric, C^2 = 1, C gamma antisymmetric
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_C_properties
```

Text cell:

```
## 7. The spin connection and the term $3H\gamma^{(x_0)}$

The vielbein of the author's diagonal metric is diagonal: $e^a{}_\mu = E_a
\delta^a_\mu$ with $E_a = \sqrt{|g_{aa}|}$, that is $E = (e^{a_4}s, e^{a_4}s,
e^{a_4}s, 1, e^{-a_4}s, e^{-a_4}s, e^{-a_4}s, \cot z)$ with $s = \sin^{1/6} z$.
The canonical spin connection is
$\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu e_b{}^\nu + \Gamma^\nu{}_{\mu\lambda}
e_b{}^\lambda)$ with the inverse vielbein $e_b{}^\nu = \delta_b^\nu/E_b$. For
$a = b$ the two terms cancel ($E_a\partial_\mu(1/E_a) = -\partial_\mu\ln E_a$ and
$\Gamma^a{}_{\mu a} = \partial_\mu\ln E_a$); for $a \ne b$ only the second survives:

$$\omega_{\mu\,ab} = \eta_{aa}\,\omega_\mu{}^a{}_b
= \eta_{aa}\,\frac{E_a}{E_b}\,\Gamma^a{}_{\mu b}\qquad(a \ne b),$$

and $\Omega_\mu = \tfrac12\sum_{a,b}\omega_{\mu\,ab}S^{ab}$ with
$S^{ab} = \tfrac14[\gamma^{(a)}, \gamma^{(b)}]$. The next cell computes the
Christoffel symbols of the diagonal metric (with $\partial/\partial x_0 =
6H\,\partial/\partial z$), the connection and the eight matrices $\Omega_\mu$ for an
arbitrary function $a_4(x_4)$, writing every function of $z$ through
$\cot z$ (the helper `clean`, as in Notebook 12a).
```

In [4]:

```
H = sp.symbols("H", positive=True)  # the constant H of the metric
z = sp.symbols("z", positive=True)  # z = 6 H x0 in (0, pi/2)
x4 = sp.symbols("x4", real=True)  # the time
a4 = sp.Function("a4")(x4)  # an arbitrary function a4(x4)
cz = sp.symbols("cz", positive=True)  # cot z > 0 on the patch
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
s6 = sp.sin(z) ** sp.Rational(1, 6)  # the warp sin(z)^(1/6) of a length
E = [sp.cot(z)] + [sp.exp(a4) * s6] * 3 + [sp.Integer(1)] + [sp.exp(-a4) * s6] * 3
g = [ETA[a] * E[a] ** 2 for a in range(8)]  # g_aa = eta_aa E_a^2
TRIG = {sp.sin(z): 1 / sp.sqrt(1 + cz ** 2), sp.cos(z): cz / sp.sqrt(1 + cz ** 2),
        sp.tan(z): 1 / cz, sp.cot(z): cz}  # every function of z through cot z


def clean(expr):
    """expr with a4', a4'' as ad1, ad2, functions of z through cz, simplified."""
    expr = expr.subs(sp.Derivative(a4, (x4, 2)), ad2).subs(sp.Derivative(a4, x4), ad1)
    expr = sp.expand_trig(expr).subs(TRIG)
    return sp.expand(sp.cancel(sp.powsimp(sp.expand(expr), force=True)))


def d(expr, mu):
    """The partial derivative with respect to the coordinate at position mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # d/dx0 = (dz/dx0) d/dz = 6 H d/dz
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7
```

```
Gamma = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for a, b, c in itertools.product(range(8), repeat=3):  # diagonal-metric formula
    value = 0
    if a == c:
        value += d(g[a], b)
    if a == b:
        value += d(g[a], c)
    if b == c:
        value -= d(g[b], a)
    Gamma[a][b][c] = value / (2 * g[a])
S_ab = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
        for a in range(8)]
omega = [[[clean(ETA[a] * E[a] * Gamma[a][mu][b] / E[b]) if a != b else 0
           for b in range(8)] for a in range(8)] for mu in range(8)]
antisymmetric = all(sp.expand(omega[mu][a][b] + omega[mu][b][a]) == 0
                    for mu in range(8) for a in range(8) for b in range(8))
check(antisymmetric, "omega_mu ab = -omega_mu ba (canonical spin connection)")
Omega = [sum((omega[mu][a][b] * S_ab[a][b] / 2 for a in range(8) for b in range(8)
              if omega[mu][a][b] != 0), Z16) for mu in range(8)]
for mu in range(8):
    terms = sum(1 for a in range(8) for b in range(8) if omega[mu][a][b] != 0)
    say(f"Omega_{NAMES[mu]}: {terms} nonzero entries omega_mu ab")
```

Out [4]:

```
PASS omega_mu ab = -omega_mu ba (canonical spin connection)
Omega_x0: 0 nonzero entries omega_mu ab
Omega_x1: 4 nonzero entries omega_mu ab
Omega_x2: 4 nonzero entries omega_mu ab
Omega_x3: 4 nonzero entries omega_mu ab
Omega_x4: 0 nonzero entries omega_mu ab
Omega_x5: 4 nonzero entries omega_mu ab
Omega_x6: 4 nonzero entries omega_mu ab
Omega_x7: 4 nonzero entries omega_mu ab
```

Text cell:

```
The next cell forms the curved-space gammas $\gamma^\mu = \gamma^{(\mu)}/E_\mu$
(no sum) and checks three facts that the Revision record proved with the author's
gammas: $\Omega_{x_4} = \Omega_{x_0} = 0$; $\gamma^\mu\Omega_\mu +
\Omega_\mu\gamma^\mu = 0$ for each $\mu$ separately; and the sum
$\sum_\mu\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$, which contains neither $a_4$
nor $z$: the time-direction terms of the three inflating 3-space directions and
of the three deflating extra times cancel.
```

In [5]:

```
gamma_curved = [gamma[mu] / E[mu] for mu in range(8)]  # gamma^mu = gamma^(mu)/E_mu
reproduces(Omega[4] == Z16 and Omega[0] == Z16, "Omega_x4 = Omega_x0 = 0",
```

```
          PY, "authorT16_Omega_x0_x4_zero")
anticommute = all((gamma_curved[mu] * Omega[mu] + Omega[mu] * gamma_curved[mu])
                  .applyfunc(clean) == Z16 for mu in range(8))
reproduces(anticommute, "{gamma^mu, Omega_mu} = 0 for each mu (no sum)",
          PY, "authorT16_anticommutator_no_sum")
gravity_term = sum((gamma_curved[mu] * Omega[mu] for mu in range(8)), Z16)
gravity_term = gravity_term.applyfunc(clean)
reproduces(gravity_term == 3 * H * gamma[0], "gamma^mu Omega_mu = 3 H gamma^(x0)",
          PY, "authorT16_gravity_term")
reproduces(gravity_term == 3 * H * gamma[0], "the same, the lead's independent check",
          EMT, "gamma_Omega_equals_3H_gamma0")
```

Out [5]:

```
PASS Omega_x4 = Omega_x0 = 0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_Omega_x0_x4_zero
PASS {gamma^mu, Omega_mu} = 0 for each mu (no sum)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_anticommutator_no_sum
PASS gamma^mu Omega_mu = 3 H gamma^(x0)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_gravity_term
PASS the same, the lead's independent check
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check gamma_Omega_equals_3H_gamma0
```

Text cell:

```
## 8. The condensate equation and its solution for every starting spinor

For $\Phi = \Phi(x_4)$ only the $x_4$ derivative survives, and $\gamma^{x_4} =
\gamma^{(x_4)}$ (because $E_4 = 1$). With the term of section 7 the field equation
becomes $\gamma^{(x_4)}\Phi' + 3H\gamma^{(x_0)}\Phi = M\Phi$, $M = m + U'(S)$.
Multiplying by $\gamma^{(x_4)}$ and using $(\gamma^{(x_4)})^2 = -1$:

$$\Phi' = \mathcal{A}\,\Phi,\qquad \mathcal{A} = -\gamma^{(x_4)}\big(M -
3H\gamma^{(x_0)}\big).$$

The matrix $\mathcal{A}$ contains neither $a_4$ nor $z$: the condensate does not
feel the deflation at all. Squaring, with $(\gamma^{(x_4)})^2 = -1$,
$(\gamma^{(x_0)})^2 = 1$ and $\gamma^{(x_4)}\gamma^{(x_0)} =
-\gamma^{(x_0)}\gamma^{(x_4)}$, the mixed terms cancel and

$$\mathcal{A}^2 = -(M^2 - 9H^2)\cdot 1 .$$

So, for $M^2 > 9H^2$ and $w = \sqrt{M^2 - 9H^2}$, every solution is
$\Phi(x_4) = \cos(wx_4)\,\Phi(0) + \tfrac{\sin(wx_4)}{w}\,\mathcal{A}\Phi(0)$
(differentiate twice: $\Phi'' = -w^2\Phi$, and $\Phi'(0) = \mathcal{A}\Phi(0)$).
The next cell checks the reduction, the square, and $C\mathcal{A} +
\mathcal{A}^TC = 0$, which makes $S$ constant: $S' = \Phi^\dagger(\mathcal{A}^T C
+ C\mathcal{A})\Phi = 0$ (the matrices are real, $M$ is real).
```

In [6]:

```
MM = sp.symbols("M", real=True)  # the effective mass M = m + lambda S


def condensate_matrix(M_value, H_value):
    """The matrix A of Phi' = A Phi for given M and H."""
    return -gamma[4] * (M_value * I16 - 3 * H_value * gamma[0])


A_sym = condensate_matrix(MM, H)
reduced = (gamma[4] * A_sym + gravity_term - MM * I16).applyfunc(sp.expand)
reproduces(reduced == Z16 and not A_sym.has(z),
           "gamma^(x4) A + 3 H gamma^(x0) = M: the equation is Phi' = A Phi",
           WL, "condensate_equation_x0_consistent")
check((A_sym * A_sym + (MM ** 2 - 9 * H ** 2) * I16).applyfunc(sp.expand) == Z16,
      "A^2 = -(M^2 - 9 H^2): every condensate oscillates with w = sqrt(M^2 - 9 H^2)")
reproduces((C * A_sym + A_sym.T * C).applyfunc(sp.expand) == Z16,
           "C A + A^T C = 0: S is constant along x4", PY,
           "authorT16_condensate_S_constant")
```

Out [6]:

```
PASS gamma^(x4) A + 3 H gamma^(x0) = M: the equation is Phi' = A Phi
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_equation_x0_consistent
PASS A^2 = -(M^2 - 9 H^2): every condensate oscillates with w = sqrt(M^2 - 9 H^2)
PASS C A + A^T C = 0: S is constant along x4
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_S_constant
```

Text cell:

```
The next cell draws the frequency $w = \sqrt{M^2 - 9H^2}$ against $M/H$ and, as
dots, the absolute values of the imaginary parts of the 16 eigenvalues of
$\mathcal{A}$ computed numerically at a few values of $M$. It also counts the
eigenvalues with a positive imaginary part: they are $\pm iw$, each eight times. For
$|M| < 3H$ the eigenvalues are real, $\pm\sqrt{9H^2 - M^2}$: the condensate grows
or decays exponentially instead of oscillating (shaded band). The examples of this
notebook use $M = \pm 5H$, where $w = 4H$.
```

In [7]:

```
m_axis = np.linspace(-8.0, 8.0, 801)  # M/H
fig, ax = plt.subplots()
ax.plot(m_axis[m_axis <= -3.0], np.sqrt(m_axis[m_axis <= -3.0] ** 2 - 9.0),
        color="tab:blue", label="$w = \\sqrt{M^2 - 9H^2}$")
ax.plot(m_axis[m_axis >= 3.0], np.sqrt(m_axis[m_axis >= 3.0] ** 2 - 9.0),
        color="tab:blue")
ax.axvspan(-3.0, 3.0, color="grey", alpha=0.2, label="$|M| < 3H$: no oscillation")
```

```
worst = 0.0  # the largest deviation of a numerical eigenvalue from +-i w
upper = []  # for each mass: how many eigenvalues have a positive imaginary part
for M_value in (-7.0, -5.0, -4.0, 4.0, 5.0, 7.0):
    matrix = np.array(condensate_matrix(M_value, 1).tolist(), dtype=float)
    eigenvalues = np.linalg.eigvals(matrix)
    w_value = np.sqrt(M_value ** 2 - 9.0)
    worst = max(worst, np.max(np.abs(np.abs(eigenvalues.imag) - w_value)),
                np.max(np.abs(eigenvalues.real)))
    upper.append(int(np.sum(eigenvalues.imag > 0)))  # the ones near +i w
    ax.plot([M_value] * 16, np.abs(eigenvalues.imag), "o", color="black",
            markersize=4, zorder=3)  # zorder 3: drawn on top of the red squares
ax.plot([], [], "o", color="black", markersize=4,
        label="$|\\mathrm{Im}|$ of the eigenvalues of $\\mathcal{A}$")
ax.plot([-5, 5], [4, 4], "s", color="tab:red", markersize=7,
        label="$M = \\pm5H$: $w = 4H$")
ax.set_xlabel("effective mass $M/H$")
ax.set_ylabel("frequency $w/H$")
ax.set_title("The frequency of a homogeneous condensate")
ax.legend(fontsize=8, loc="upper center")
say(f"largest deviation of a numerical eigenvalue from +-i w: {worst:.0e}")
say(f"eigenvalues near +i w, for each of the six masses: {upper}")
check(worst < 1e-9 and upper == [8] * 6,
      "the 16 eigenvalues of A are +i w and -i w, eight of each (six masses)")
save_figure(fig, "condensate_frequency",
            "The frequency $w$ of a homogeneous condensate of dirac16complex00 in "
            "the author's metric as a function of its effective mass $M = m + "
            "\\lambda S$, both in units of $H$: the curve $w = \\sqrt{M^2 - 9H^2}$ "
            "from $\\mathcal{A}^2 = -(M^2 - 9H^2)$, and as black dots the absolute "
            "imaginary parts of the 16 eigenvalues of the 16 by 16 matrix "
            "$\\mathcal{A}$, computed numerically at six masses (each dot stands "
            "for all 16 eigenvalues of its mass, eight equal to $+iw$ and eight "
            "equal to $-iw$). In the grey band $|M| < 3H$ the eigenvalues are real and "
            "the condensate grows or decays instead of oscillating. The red squares "
            "mark $M = \\pm 5H$, $w = 4H$, the masses used in this notebook. The "
            "hidden-direction term $3H\\gamma^{(x_0)}$ of the field equation shifts "
            "the threshold from $0$ to $3H$.")
```

Out [7]:

```
largest deviation of a numerical eigenvalue from +-i w: 1e-14
eigenvalues near +i w, for each of the six masses: [8, 8, 8, 8, 8, 8]
PASS the 16 eigenvalues of A are +i w and -i w, eight of each (six masses)
```

Out [7]:

```
Figure 12d.1 saved as Revision/textbook/figures/12d_1_condensate_frequency.png
```

Text cell:

```
## 9. Condensates whose 15 three-gamma bilinears vanish
```

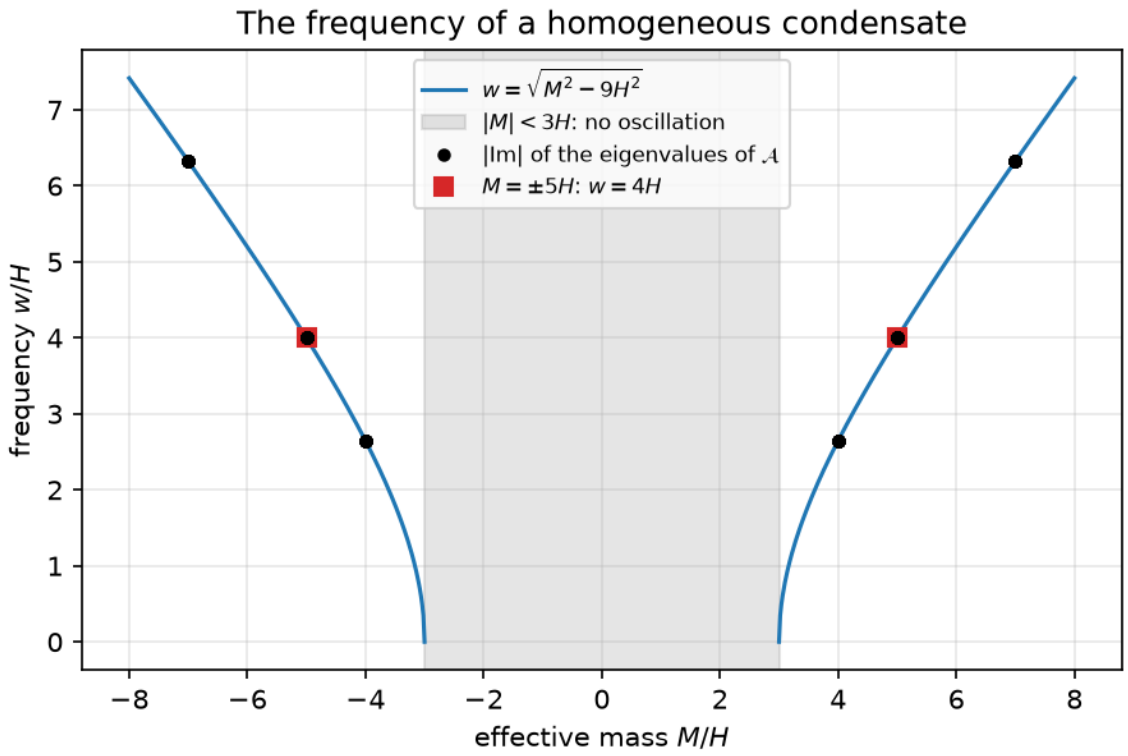

Figure 338: The frequency w of a homogeneous condensate of dirac16complex00 in the author's metric as a function of its effective mass $M = m+\lambda S$, both in units of H: the curve $w = \sqrt{M^2 - 9H^2}$ from $\mathcal{A}^2 = -(M^2 - 9H^2)$, and as black dots the absolute imaginary parts of the 16 eigenvalues of the 16 by 16 matrix $\mathcal{A}$, computed numerically at six masses (each dot stands for all 16 eigenvalues of its mass, eight equal to $+iw$ and eight equal to $-iw$). In the grey band $|M| < 3H$ the eigenvalues are real and the condensate grows or decays instead of oscillating. The red squares mark $M = \pm 5H$, $w = 4H$, the masses used in this notebook. The hidden-direction term $3H\gamma^{(x_0)}$ of the field equation shifts the threshold from 0 to $3H$. (Notebook 12d, figure 1.)

The Revision record shows that the off-diagonal parts of the kinetic tensor of a condensate are multiples of 15 three-gamma bilinears $B_{abc} = \bar\Phi\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$: the six with $\{a, b, c\} = \{i, x_4, x_0\}$ (i a 3-space direction or an extra time) and the nine with $\{i, j, x_4\}$ (i in 3-space, j an extra time). The off-diagonal Einstein equations $0 = \kappa T^\mu{}_\nu$ need all 15 to vanish. The record's recipe for such a condensate, at $M = -5$, $H = 1$, $w = 4$: take the spinor v_1 with $\mathcal{A}v_1 = -iwv_1$ and $\gamma^{(x_1)}\gamma^{(x_5)} = \gamma^{(x_2)}\gamma^{(x_6)} = \gamma^{(x_3)}\gamma^{(x_7)} = -1$ on it, the spinor v_2 with the same frequency and the value $+1$, and $c = \overline{v_1^\dagger C v_2}$ (the bar is the complex conjugate). Each is found as the null space (the solutions of a set of linear equations) of a stacked matrix. We extend the recipe to the family

$$\Phi_0(t) = v_1 + t\,c\,v_2\qquad(t \text{ any real number}),$$

and the condensate is $\Phi(x_4) = e^{-iwx_4}\Phi_0(t)$. The next cell builds v_1, v_2 and c, checks that each null space is one-dimensional, that $\Phi_0(t)$ has the frequency w, and that the 15 bilinears vanish for EVERY real t (sympy keeps t as a symbol).

In [8]:

```
t = sp.symbols("t", real=True)  # the free real parameter of the family
TRIPLES = list(itertools.combinations(range(8), 3))  # all 56 sets {a, b, c}
NEEDED = sorted({tuple(sorted((i, 4, 0))) for i in (1, 2, 3, 5, 6, 7)}
                | {tuple(sorted((i, j, 4))) for i in (1, 2, 3) for j in (5, 6, 7)})


def witness_parts(M_value, H_value):
    """v1, v2, c and the frequency w of the record's recipe."""
    w_value = sp.sqrt(M_value ** 2 - 9 * H_value ** 2)
    A_value = condensate_matrix(M_value, H_value)
    spaces = []
    for sign in (-1, 1):  # the eigenvalue of g^(x1)g^(x5) = g^(x2)g^(x6) = ...
        stack = sp.Matrix.vstack(A_value + sp.I * w_value * I16,
                                 gamma[1] * gamma[5] - sign * I16,
                                 gamma[2] * gamma[6] - sign * I16,
                                 gamma[3] * gamma[7] - sign * I16)
        spaces.append(stack.nullspace())  # all solutions of stack * v = 0
    v1, v2 = spaces[0][0], spaces[1][0]
    c = sp.conjugate(sp.expand((v1.H * C * v2)[0]))  # c = conj(v1^dagger C v2)
    return spaces, v1, v2, c, w_value, A_value


def bilinear(phi, matrix):
    """The bilinear phibar matrix phi = phi^dagger C matrix phi."""
    return sp.expand((phi.H * C * matrix * phi)[0])


def three_gamma(triple):
    a, b, c = triple
    return gamma[a] * gamma[b] * gamma[c]
```

```
spaces, v1, v2, c, w, A_witness = witness_parts(-5, 1)
phi_t = (v1 + t * c * v2).applyfunc(sp.expand)  # the family Phi_0(t)
check([len(space) for space in spaces] == [1, 1],
      "the two null spaces are one-dimensional (v1 and v2 are unique)")
report("frequency w at M = -5 H", w, "H")
check((A_witness * phi_t + sp.I * w * phi_t).applyfunc(sp.expand)
      == sp.zeros(16, 1), "A Phi_0(t) = -i w Phi_0(t) for every t")
needed_values = [bilinear(phi_t, three_gamma(triple)) for triple in NEEDED]
check(len(NEEDED) == 15 and all(value == 0 for value in needed_values),
      "the 15 three-gamma bilinears vanish for every real t")
```

Out [8]:

```
PASS the two null spaces are one-dimensional (v1 and v2 are unique)
RESULT frequency w at M = -5 H = 4 H
PASS A Phi_0(t) = -i w Phi_0(t) for every t
PASS the 15 three-gamma bilinears vanish for every real t
```

Text cell:

```
**The density of the family.** $S(t) = \Phi_0^\dagger C\Phi_0$ has four parts:
$v_1^\dagger Cv_1$, $t^2|c|^2v_2^\dagger Cv_2$ and the two cross terms
$t\,c\,v_1^\dagger Cv_2 + t\,\bar c\,v_2^\dagger Cv_1 = 2t\,|v_1^\dagger Cv_2|^2$
(because $\bar c = v_1^\dagger Cv_2$ and $v_2^\dagger Cv_1$ is its complex
conjugate, $C$ being real symmetric). The next cell finds the first two zero, so
$S(t) = 2t\,|v_1^\dagger Cv_2|^2$: the family contains condensates of every real
density, positive for $t > 0$ and negative for $t < 0$. It also repeats the
construction at $M = +5$, where the Revision record ran it with the author's
gammas, and checks the record's statement there ($t = 1$: the 15 bilinears vanish,
$S$ is real and not zero).
```

In [9]:

```
S_t = bilinear(phi_t, I16)  # S(t) = Phi_0(t)^dagger C Phi_0(t)
cross = sp.expand((v1.H * C * v2)[0])  # v1^dagger C v2
report("v1^dagger C v2", cross)
report("S(t)", S_t)
check(bilinear(v1, I16) == 0 and bilinear(v2, I16) == 0
      and sp.expand(S_t - 2 * t * cross * sp.conjugate(cross)) == 0,
      "S(t) = 2 t |v1^dagger C v2|^2: every real density occurs")
_, u1, u2, c_plus, w_plus, A_plus = witness_parts(5, 1)  # the record's M = +5
phi_plus = (u1 + c_plus * u2).applyfunc(sp.expand)  # t = 1
S_plus = bilinear(phi_plus, I16)
ok_plus = ((A_plus * phi_plus + sp.I * w_plus * phi_plus).applyfunc(sp.expand)
           == sp.zeros(16, 1)
           and all(bilinear(phi_plus, three_gamma(tr)) == 0 for tr in NEEDED)
           and S_plus != 0 and sp.im(S_plus) == 0)
reproduces(ok_plus, "M = +5, H = 1: a witness with S real, not 0, 15 bilinears 0",
           PY, "authorT16_condensate_witness")
```

Out [9]:

```
RESULT v1^dagger C v2 = -128/25 - 96*I/25
RESULT S(t) = 2048*t/25
PASS S(t) = 2 t |v1^dagger C v2|^2: every real density occurs
PASS M = +5, H = 1: a witness with S real, not 0, 15 bilinears 0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_witness
```

Text cell:

```
The next cell computes all 56 three-gamma bilinears of $\Phi_0(1)$ (at
$M = -5$), checks that each is a real number, and draws them: the 15 that the
field equations need to vanish in red (all zero), the others in blue. Most of the
others vanish as well; the eight that do not are built from three directions
among $x_1, x_2, x_3, x_5, x_6, x_7$ only, and no field equation asks them to
vanish.
```

In [10]:

```
phi_one = phi_t.subs(t, 1)  # Phi_0(1)
all_values = [bilinear(phi_one, three_gamma(triple)) for triple in TRIPLES]
check(all(sp.im(value) == 0 for value in all_values),
      "all 56 three-gamma bilinears of Phi_0(1) are real numbers")
nonzero = [triple for triple, value in zip(TRIPLES, all_values) if value != 0]
say("nonzero bilinears: " + ", ".join(
    "".join(NAMES[i][1] for i in triple) for triple in nonzero))
check(all(4 not in triple and 0 not in triple for triple in nonzero),
      "every nonzero one avoids x4 and x0")
labels = ["".join(NAMES[i][1] for i in triple) for triple in TRIPLES]
numbers = np.array([float(value) for value in all_values])
is_needed = np.array([triple in NEEDED for triple in TRIPLES])
fig, ax = plt.subplots(figsize=(11.0, 4.2))
positions = np.arange(len(TRIPLES))
ax.bar(positions[~is_needed], numbers[~is_needed], color="tab:blue",
       label="other bilinears")
ax.plot(positions[is_needed], numbers[is_needed], "x", color="tab:red",
        markersize=8, label="the 15 the field equations need to vanish")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(positions)
ax.set_xticklabels(labels, rotation=90, fontsize=6.5)
ax.set_xlabel("directions $abc$ of $\\bar\\Phi\\gamma^{(a)}\\gamma^{(b)}"
              "\\gamma^{(c)}\\Phi$ (digits: $x_0$ = 1, ..., $x_7$ = 8)")
ax.set_ylabel("value of the bilinear")
ax.set_title("The 56 three-gamma bilinears of the condensate $\\Phi_0(1)$, "
             "$M = -5H$")
ax.legend(fontsize=8)
save_figure(fig, "three_gamma_bilinears",
            "The 56 three-gamma bilinears $\\bar\\Phi\\gamma^{(a)}\\gamma^{(b)}"
            "\\gamma^{(c)}\\Phi$ of the condensate $\\Phi_0(1) = v_1 + c\\,v_2$ at "
            "$M = -5H$, $H = 1$, one for each set of three directions $a < b < c$ "
```

```
"(horizontal axis, labelled by the digits of $x_0$ to $x_7$); vertical "
"axis the exact value (a real number). Red crosses: the 15 bilinears "
"whose vanishing the off-diagonal Einstein equations require (they "
"contain $x_4$ together with $x_0$ or with a 3-space direction and an "
"extra time); all are exactly zero. Blue bars: the other 41, of which "
"only eight are not zero, all built from 3-space directions and extra "
"times.")
```

Out [10]:

```
PASS all 56 three-gamma bilinears of Phi_0(1) are real numbers
nonzero bilinears: 123, 127, 136, 167, 235, 257, 356, 567
PASS every nonzero one avoids x4 and x0
```

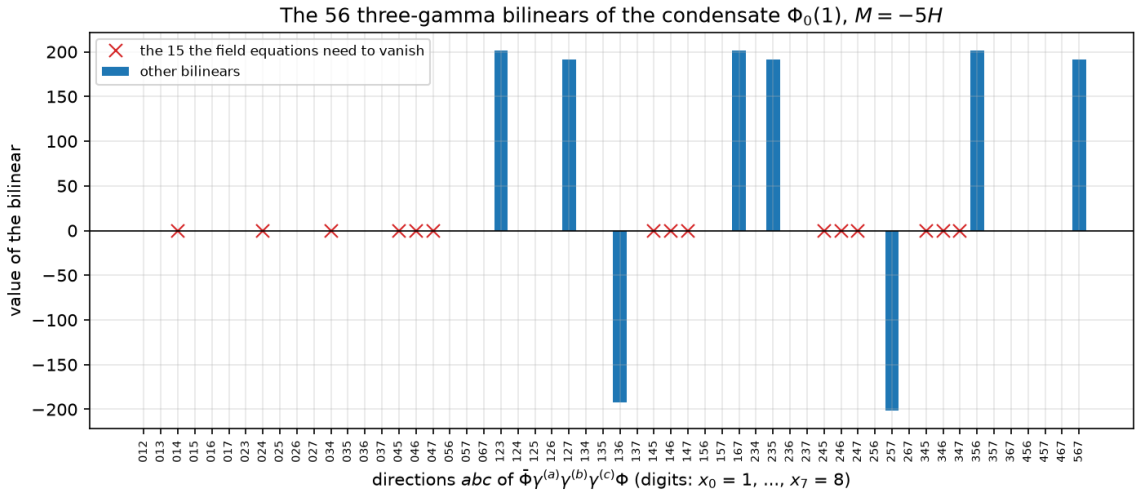

Figure 339: The 56 three-gamma bilinears $\bar{\Phi}\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$ of the condensate $\Phi_0(1) = v_1 + c\,v_2$ at $M = -5H$, $H = 1$, one for each set of three directions $a < b < c$ (horizontal axis, labelled by the digits of x_0 to x_7); vertical axis the exact value (a real number). Red crosses: the 15 bilinears whose vanishing the off-diagonal Einstein equations require (they contain x_4 together with x_0 or with a 3-space direction and an extra time); all are exactly zero. Blue bars: the other 41, of which only eight are not zero, all built from 3-space directions and extra times. (Notebook 12d, figure 2.)

Out [10]:

```
Figure 12d.2 saved as Revision/textbook/figures/12d_2_three_gamma_bilinears.png
```

Text cell:

```
## 10. The complete energy-momentum tensor of the condensate

The next cell computes the kinetic tensor $K^\mu{}_\nu$ for every pair
$(\mu, \nu)$ as a bilinear $\Phi_0^\dagger N^\mu{}_\nu\Phi_0$, with the matrix
$N^\mu{}_\nu = \tfrac12(C\gamma^\mu\mathcal{D}_\nu - \bar{\mathcal{D}}_\nu
```

```
\gamma^\mu)$, where $\mathcal{D}_\nu = \mathcal{A}\delta_{\nu 4} + \Omega_\nu$ acts
on $\Phi$ ($\Phi' = \mathcal{A}\Phi$) and $\bar{\mathcal{D}}_\nu =
\mathcal{A}^TC\delta_{\nu 4} - C\Omega_\nu$ on $\bar\Phi$ (the phase $e^{-iwx_4}$
cancels in every bilinear). It works for the whole family (symbol $t$) and for an
ARBITRARY $a_4(x_4)$ and every $z$, at $H = 1$. Then it symmetrises,
$K^{(\mu}{}_{\nu)} = \tfrac12\big(K^\mu{}_\nu + (g_{\nu\nu}/g_{\mu\mu})
K^\nu{}_\mu\big)$ (lower the upper index, symmetrise, raise it again), and checks
the record's statements: every off-diagonal component vanishes, for every $a_4$,
$a_4'$ and $z$; on the diagonal only $K^{x_4}{}_{x_4} = MS$ survives.
```

In [11]:

```
UNIT_H = {H: 1}  # the examples use H = 1
Omega_1 = [matrix.subs(UNIT_H) for matrix in Omega]
gamma_1 = [matrix.subs(UNIT_H) for matrix in gamma_curved]
g_1 = [entry.subs(UNIT_H) for entry in g]


def D_phi(nu):
    """The matrix that gives D_nu Phi from Phi."""
    return (A_witness if nu == 4 else Z16) + Omega_1[nu]


def D_phibar(nu):
    """The matrix that gives D_nu Phibar from Phi^dagger (row times matrix)."""
    return (A_witness.T * C if nu == 4 else Z16) - C * Omega_1[nu]


K = [[(phi_t.H * ((C * gamma_1[mu] * D_phi(nu) - D_phibar(nu) * gamma_1[mu]) / 2)
       * phi_t)[0] for nu in range(8)] for mu in range(8)]
K_sym = [[clean((K[mu][nu] + (g_1[nu] / g_1[mu]) * K[nu][mu]) / 2)
          for nu in range(8)] for mu in range(8)]
off_diagonal = all(K_sym[mu][nu] == 0 for mu in range(8) for nu in range(8)
                   if mu != nu)
reproduces(off_diagonal, "every off-diagonal K^(mu nu) vanishes, for every a4, a4', z",
           WL, "condensate_diagonal_witness_exact")
diagonal = [K_sym[mu][mu] for mu in range(8)]
say(f"diagonal of K: {diagonal}")
reproduces(all(diagonal[mu] == 0 for mu in range(8) if mu != 4)
           and sp.expand(diagonal[4] - (-5) * S_t) == 0,
           "K^x4_x4 = M S and every other diagonal entry is 0",
           PY, "authorT16_condensate_kinetic_diagonal")
```

Out [11]:

```
PASS every off-diagonal K^(mu nu) vanishes, for every a4, a4', z
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_diagonal_witness_exact
diagonal of K: [0, 0, 0, 0, -2048*t/5, 0, 0, 0]
PASS K^x4_x4 = M S and every other diagonal entry is 0
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_kinetic_diagonal
```

Text cell:

```
With $\sigma_T = +1$ the energy-momentum tensor is $T^\mu{}_\nu = -K^{(\mu}{}_{\nu)} +
\delta^\mu_\nu L$, $L = \sum_\mu K^\mu{}_\mu - mS - U(S) = MS - mS - U$. On shell
$M = m + \lambda S$, so $L = \lambda S^2 - \tfrac\lambda2 S^2 = \tfrac\lambda2 S^2$,
and therefore

$$\rho = -T^{x_4}{}_{x_4} = MS - L = mS + \tfrac\lambda2S^2,\qquad
p_3 = p_t = p_0 = L = \tfrac\lambda2 S^2,$$

with all mixed components zero. The next cell builds this tensor with symbols $m$
and $\lambda$ (and $M = -5 = m + \lambda S$) and checks these formulas.
```

In [12]:

```
m, lam = sp.symbols("m lambda", real=True)  # the mass and the self-coupling


def source_tensor(K_matrix, S_value, m_value, lam_value):
    """T^mu_nu = -K^(mu nu) + delta L with L = sum K^mu_mu - m S - U(S)."""
    L_value = (sum(K_matrix[mu][mu] for mu in range(8)) - m_value * S_value
               - lam_value * S_value ** 2 / 2)
    return [[sp.expand(-K_matrix[mu][nu] + (L_value if mu == nu else 0))
             for nu in range(8)] for mu in range(8)]


on_shell = {m: -5 - lam * S_t}  # M = m + lambda S = -5
T_family = source_tensor(K_sym, S_t, m, lam)
rho_family = sp.expand(-T_family[4][4].subs(on_shell))
pressures = [sp.expand(T_family[mu][mu].subs(on_shell)) for mu in (1, 5, 0)]
check(sp.expand(rho_family - ((-5 - lam * S_t) * S_t + lam * S_t ** 2 / 2)) == 0
      and all(sp.expand(p - lam * S_t ** 2 / 2) == 0 for p in pressures),
      "rho = m S + (lambda/2) S^2 and p3 = pt = p0 = (lambda/2) S^2 on shell")
```

Out [12]:

```
PASS rho = m S + (lambda/2) S^2 and p3 = pt = p0 = (lambda/2) S^2 on shell
```

Text cell:

```
## 11. The two Einstein conditions and their solution

For the linear member the Einstein equations with this source are the time
equation $(21 + 3A^2)H^2 + \Lambda = -\kappa\rho$ and the seven equal equations
$(15 - 3A^2)H^2 + \Lambda = \kappa p$ (the mixed ones read $0 = 0$, section 10).
Their difference and their sum give, with $\rho + p = (m + \lambda S)S = MS$ and
$\rho - p = mS$:

$$\kappa MS = -6(A^2 + 1)H^2,\qquad \kappa mS = -(36H^2 + 2\Lambda).$$
```

```
These are the record's two conditions. The first fixes the density: a real slope
$A$ needs $\kappa MS \le -6H^2$, so $S$ and $M$ must have opposite signs (possible
in our family, which has both signs of $S$). The second then fixes $m$, and
$\lambda = (M - m)/S$. The next cell reads the Einstein components from the record
'a4-equations.json', checks this equivalence with sympy, and solves for $S$, $m$
and $\lambda$.
```

In [13]:

```
A, Lam, kappa = sp.symbols("A Lambda kappa", real=True)
record = read_json(EQUATIONS)
E1 = record["lovelockTensors"]["E1"]  # the Einstein tensor G = E_(1)
LINEAR = {"ad1": A * H, "ad2": 0, "H": H}  # a4' = A H, a4'' = 0


def from_record(key):
    """A component of G from the record, for the linear member."""
    return sp.sympify(E1[key]["input"].replace("^", "**"), locals=LINEAR)


G_time, G_space = from_record("x4x4"), from_record("x1x1")
G_extra, G_hidden = from_record("x5x5"), from_record("x0x0")
check(sp.expand(G_space - G_hidden) == 0 and sp.expand(G_extra - G_hidden) == 0,
      "linear member: G^x1_x1 = G^x5_x5 = G^x0_x0 (the record's components)")
S_sym, rho_sym, p_sym = sp.symbols("S rho p", real=True)
time_equation = G_time + Lam + kappa * rho_sym  # = 0
space_equation = G_hidden + Lam - kappa * p_sym  # = 0
condensate = {rho_sym: m * S_sym + lam * S_sym ** 2 / 2, p_sym: lam * S_sym ** 2 / 2}
first = sp.expand(kappa * (m + lam * S_sym) * S_sym + 6 * (A ** 2 + 1) * H ** 2)
second = sp.expand(kappa * m * S_sym + 36 * H ** 2 + 2 * Lam)
reproduces(sp.expand((time_equation - space_equation).subs(condensate) - first) == 0
           and sp.expand((time_equation + space_equation).subs(condensate)
                         - second) == 0,
           "Einstein: kappa M S = -6 (A^2 + 1) H^2 and kappa m S = -(36 H^2 + 2 Lambda)",
           PY, "condensate_einstein_quadratic_U")
M_value = sp.symbols("M_value", real=True)
S_needed = sp.solve(kappa * M_value * S_sym + 6 * (A ** 2 + 1) * H ** 2, S_sym)[0]
m_needed = sp.solve(second, m)[0]
say(f"S = {S_needed},  m = {m_needed},  lambda = (M - m)/S")
```

Out [13]:

```
PASS linear member: G^x1_x1 = G^x5_x5 = G^x0_x0 (the record's components)
PASS Einstein: kappa M S = -6 (A^2 + 1) H^2 and kappa m S = -(36 H^2 + 2 Lambda)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    condensate_einstein_quadratic_U
S = 6*H**2*(-A**2 - 1)/(M_value*kappa),  m = 2*(-18*H**2 - Lambda)/(S*kappa),  lambda =
    (M - m)/S
```

Text cell:

```
The next cell draws the plane of $M/H$ (horizontal) and $\kappa S$ (vertical). A
condensate can be the source of a linear member only below the hyperbola
$\kappa MS = -6H^2$ in the second quarter or above it in the fourth (outside the
grey region), where $A^2 = -\kappa MS/(6H^2) - 1 \ge 0$; each hyperbola
$\kappa MS = -6(A^2 + 1)H^2$ belongs to one slope $\pm A$. Our family oscillates
only for $|M| > 3H$ (outside the hatched band). The three examples of the next
section are marked.
```

In [14]:

```
m_left = np.linspace(-10.0, -0.01, 1000)  # M/H < 0
m_right = np.linspace(0.01, 10.0, 1000)  # M/H > 0
fig, ax = plt.subplots(figsize=(7.0, 5.2))
# grey: kappa M S > -6, where A^2 = -kappa M S/6 - 1 would be negative
ax.fill_between(m_left, -12.0, np.minimum(-6.0 / m_left, 12.0), color="grey",
                alpha=0.25, linewidth=0.0)
ax.fill_between(m_right, np.maximum(-6.0 / m_right, -12.0), 12.0, color="grey",
                alpha=0.25, linewidth=0.0, label="no real slope $A$")
ax.axvspan(-3.0, 3.0, facecolor="none", edgecolor="black", hatch="//",
           linewidth=0.0, label="$|M| < 3H$: no oscillating condensate")
for slope_squared, style in ((0, ":"), (1, "-"), (4, "--"), (5, "-.")):
    level = 6.0 * (slope_squared + 1)
    label = f"$A^2 = {slope_squared}$"
    ax.plot(m_left, -level / m_left, style, color="tab:blue", label=label)
    ax.plot(m_right, -level / m_right, style, color="tab:blue")
ax.plot([-5], [12 / 5], "o", color="tab:red", markersize=8,
        label="examples 1 and 3: $M = -5H$, $\\kappa S = 12/5$")
ax.plot([5], [-36 / 5], "s", color="tab:green", markersize=8,
        label="example 2: $M = 5H$, $\\kappa S = -36/5$")
ax.set_xlim(-10, 10)
ax.set_ylim(-12, 12)
ax.set_xlabel("effective mass $M/H$")
ax.set_ylabel("$\\kappa S$ (units $H = 1$)")
ax.set_title("Which condensates can drive a linear member (Einstein gravity)")
ax.legend(fontsize=7.5, loc="lower left")
save_figure(fig, "allowed_sources",
            "The plane of the effective mass $M/H$ (horizontal) and $\\kappa S$, the "
            "gravitational coupling times the density of the condensate (vertical, "
            "units $H = 1$). In Einstein gravity a homogeneous condensate is the "
            "source of the linear member $a_4 = AHx_4$ only if "
            "$\\kappa MS = -6(A^2 + 1)H^2$: on the blue hyperbolas, drawn for "
            "$A^2 = 0, 1, 4, 5$ (each serves $+A$ and $-A$); in the grey region no "
            "real slope exists. Inside the hatched band $|M| < 3H$ a condensate does "
            "not oscillate. The red circle (examples 1 and 3, $A = 1$) and the green "
            "square (example 2, $A^2 = 5$) are the exact solutions of this notebook; "
            "$S$ and $M$ have opposite signs, as the condition requires.")
```

Out [14]:

```
Figure 12d.3 saved as Revision/textbook/figures/12d_3_allowed_sources.png
```

Text cell:

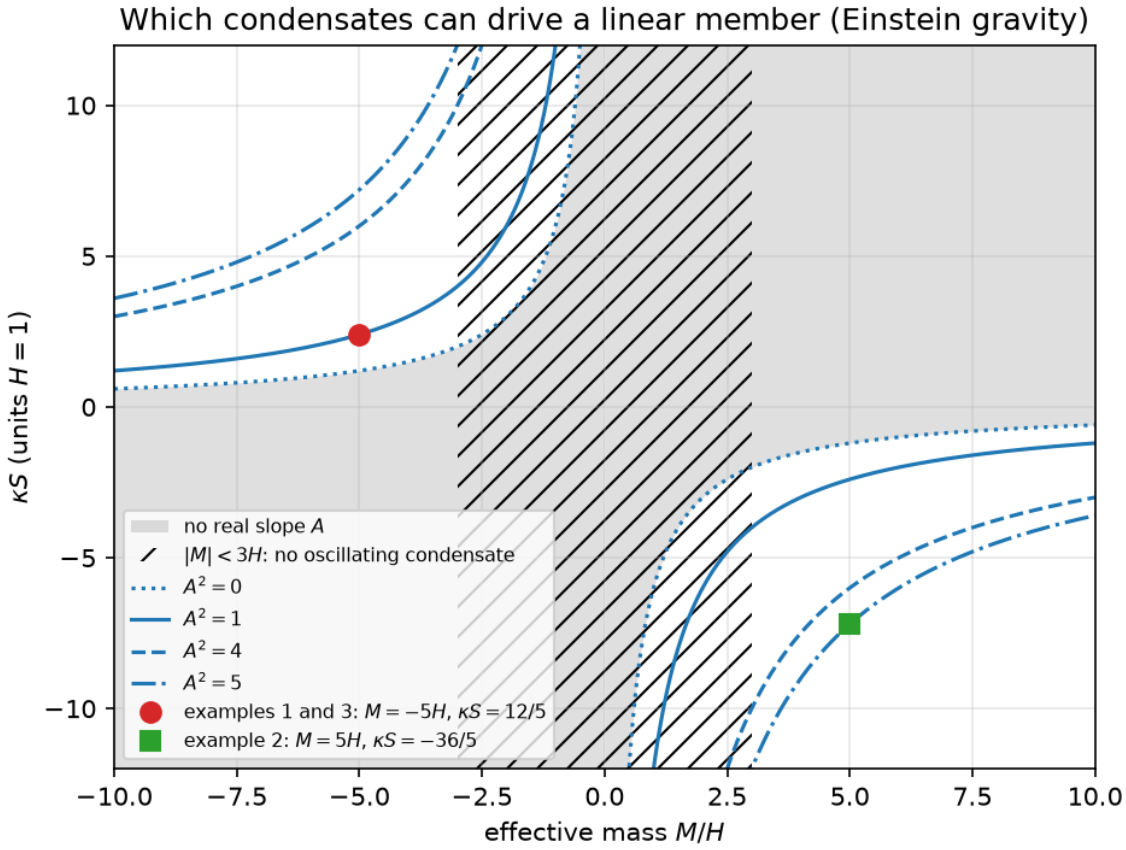

Figure 340: The plane of the effective mass M/H (horizontal) and κS, the gravitational coupling times the density of the condensate (vertical, units $H = 1$). In Einstein gravity a homogeneous condensate is the source of the linear member $a_4 = AHx_4$ only if $\kappa MS = -6(A^2 + 1)H^2$: on the blue hyperbolas, drawn for $A^2 = 0, 1, 4, 5$ (each serves $+A$ and $-A$); in the grey region no real slope exists. Inside the hatched band $|M| < 3H$ a condensate does not oscillate. The red circle (examples 1 and 3, $A = 1$) and the green square (example 2, $A^2 = 5$) are the exact solutions of this notebook; S and M have opposite signs, as the condition requires. (Notebook 12d, figure 3.)

```
## 12. Three exact solutions of the coupled equations

With $H = \kappa = 1$ and the slopes $A = 1$ (the canonical deflating history of
the Revision record) and $A = \sqrt5$:

| example | $A$ | $\Lambda$ | $M$ | $S$ | $m$ | $\lambda$ | $\rho$ | $p$ |
| --- | --- | --- | --- | --- | --- | --- | --- | --- |
| 1 | $1$ | $0$ | $-5$ | $12/5$ | $-15$ | $25/6$ | $-24$ | $12$ |
| 2 | $\sqrt5$ | $0$ | $5$ | $-36/5$ | $5$ | $0$ | $-36$ | $0$ |
| 3 | $1$ | $-30$ | $-5$ | $12/5$ | $10$ | $-25/4$ | $6$ | $-18$ |

(from $S = -6(A^2 + 1)/M$, $m = -(36 + 2\Lambda)/S$, $\lambda = (M - m)/S$,
$\rho = mS + \lambda S^2/2$, $p = \lambda S^2/2$). Examples 1 and 2 are the two
worked examples of Notebook 12b (there $S$ and $A^2$ were found from $m$,
$\lambda$ and $\Lambda$); example 3 has a positive energy density. For each, the
next cell computes the parameter $t$ of the family from $S(t)$, builds
$\Phi_0(t)$ (at $M = +5$ for example 2), and checks EXACTLY:

1. the 16 components of the field equation $\gamma^\mu D_\mu\Phi = (m +
   \lambda S)\Phi$ for $\Phi = e^{-iwx_4}\Phi_0$, with the full $\sum_\mu
   \gamma^\mu\Omega_\mu$ of section 7, an arbitrary $a_4(x_4)$ and every $z$;
2. all 64 components $G^\mu{}_\nu + \Lambda\delta^\mu_\nu - \kappa T^\mu{}_\nu = 0$
   of the Einstein equations for the linear member with this $A$, at every
   $x_4$ and $z$ ($T$ from section 10, $G$ from the record);
3. the same 64 components with $-A$: the equations contain $A$ only as $A^2$.
```

In [15]:

```
EXAMPLES = [  # name, A, Lambda, M (with H = kappa = 1)
    ("example 1", sp.Integer(1), sp.Integer(0), -5),
    ("example 2", sp.sqrt(5), sp.Integer(0), 5),
    ("example 3", sp.Integer(1), sp.Integer(-30), -5),
]
UNITS = {H: 1, kappa: 1}
solutions = {}  # name -> the numbers of the solution
for name, slope, cosmological, M_example in EXAMPLES:
    S_value = sp.nsimplify(S_needed.subs(UNITS).subs({M_value: M_example,
                                                      A: slope}))
    m_value = m_needed.subs(UNITS).subs({S_sym: S_value, Lam: cosmological})
    lam_value = (M_example - m_value) / S_value
    spaces_e, v1_e, v2_e, c_e, w_e, A_e = witness_parts(M_example, 1)
    S_of_t = bilinear((v1_e + t * c_e * v2_e).applyfunc(sp.expand), I16)
    t_value = sp.solve(S_of_t - S_value, t)[0]
    phi0 = (v1_e + t_value * c_e * v2_e).applyfunc(sp.expand)
    phi = phi0 * sp.exp(-sp.I * w_e * x4)  # the condensate Phi(x4)
    # 1. the field equation, with the full spin-connection sum of section 7:
    residual = (gamma[4] * sp.diff(phi, x4) + gravity_term.subs(UNIT_H) * phi
                - (m_value + lam_value * S_value) * phi).applyfunc(sp.simplify)
    field_ok = residual == sp.zeros(16, 1) and bilinear(phi0, I16) == S_value
    # 2. the Einstein equations, all 64 components, for +A and for -A:
    K_e = K_sym if M_example == -5 else None
    if K_e is None:  # example 2 has its own condensate matrix A (M = +5)
        K_raw = [[(phi0.H * ((C * gamma_1[mu] * ((A_e if nu == 4 else Z16)
```

```
                                            + Omega_1[nu])
                            - ((A_e.T * C if nu == 4 else Z16) - C * Omega_1[nu])
                            * gamma_1[mu]) / 2) * phi0)[0]
                for nu in range(8)] for mu in range(8)]
        K_e = [[clean((K_raw[mu][nu] + (g_1[nu] / g_1[mu]) * K_raw[nu][mu]) / 2)
                for nu in range(8)] for mu in range(8)]
    else:
        K_e = [[entry.subs(t, t_value) for entry in row] for row in K_e]
    T_e = source_tensor(K_e, S_value, m_value, lam_value)
    einstein_ok = {}
    for sign in (1, -1):
        G_diag = [G_hidden] + [G_space] * 3 + [G_time] + [G_extra] * 3
        values = {A: sign * slope, H: 1}
        einstein_ok[sign] = all(
            sp.simplify((G_diag[mu].subs(values) + cosmological if mu == nu else 0)
                        - T_e[mu][nu]) == 0
            for mu in range(8) for nu in range(8))
    rho_e, p_e = -T_e[4][4], T_e[1][1]
    solutions[name] = {"A": slope, "Lambda": cosmological, "M": M_example,
                       "S": S_value, "m": m_value, "lambda": lam_value, "t": t_value,
                       "rho": rho_e, "p": p_e, "phi0": phi0, "w": w_e, "T": T_e}
    say(f"{name}: A = {slope}, Lambda = {cosmological}, M = {M_example}, "
        f"S = {S_value}, t = {t_value}, m = {m_value}, lambda = {lam_value}, "
        f"rho = {rho_e}, p = {p_e}")
    check(field_ok, f"{name}: the 16 components of the field equation hold exactly")
    check(einstein_ok[1], f"{name}: all 64 Einstein components hold exactly")
    check(einstein_ok[-1], f"{name}: they hold as well with -A (A enters as A^2)")
```

Out [15]:

```
example 1: A = 1, Lambda = 0, M = -5, S = 12/5, t = 15/512, m = -15, lambda = 25/6, rho =
    -24, p = 12
PASS example 1: the 16 components of the field equation hold exactly
PASS example 1: all 64 Einstein components hold exactly
PASS example 1: they hold as well with -A (A enters as A^2)
example 2: A = sqrt(5), Lambda = 0, M = 5, S = -36/5, t = -45/512, m = 5, lambda = 0, rho
    = -36, p = 0
PASS example 2: the 16 components of the field equation hold exactly
PASS example 2: all 64 Einstein components hold exactly
PASS example 2: they hold as well with -A (A enters as A^2)
example 3: A = 1, Lambda = -30, M = -5, S = 12/5, t = 15/512, m = 10, lambda = -25/4, rho
    = 6, p = -18
PASS example 3: the 16 components of the field equation hold exactly
PASS example 3: all 64 Einstein components hold exactly
PASS example 3: they hold as well with -A (A enters as A^2)
```

Text cell:

```
The next cell compares the numbers with the table above and with Notebook 12b's
formula $A^2 = 5 + \Lambda/3 - \lambda S^2/6$ (Einstein gravity, $H = \kappa = 1$),
and reports the equation of state $w = p/\rho$ of each source.
```

In [16]:

```
table = {"example 1": (sp.Rational(12, 5), -15, sp.Rational(25, 6), -24, 12),
         "example 2": (sp.Rational(-36, 5), 5, 0, -36, 0),
         "example 3": (sp.Rational(12, 5), 10, sp.Rational(-25, 4), 6, -18)}
for name, (S_v, m_v, lam_v, rho_v, p_v) in table.items():
    sol = solutions[name]
    same = (sol["S"] == S_v and sol["m"] == m_v and sol["lambda"] == lam_v
            and sol["rho"] == rho_v and sol["p"] == p_v)
    formula = 5 + sol["Lambda"] / 3 - sol["lambda"] * sol["S"] ** 2 / 6
    check(same and sp.simplify(formula - sol["A"] ** 2) == 0,
          f"{name}: the numbers of the table and A^2 = 5 + Lambda/3 - lambda S^2/6")
    report(f"{name}: w = p/rho", sol["p"] / sol["rho"])
```

Out [16]:

```
PASS example 1: the numbers of the table and A^2 = 5 + Lambda/3 - lambda S^2/6
RESULT example 1: w = p/rho = -1/2
PASS example 2: the numbers of the table and A^2 = 5 + Lambda/3 - lambda S^2/6
RESULT example 2: w = p/rho = 0
PASS example 3: the numbers of the table and A^2 = 5 + Lambda/3 - lambda S^2/6
RESULT example 3: w = p/rho = -3
```

Text cell:

```
The next cell draws, for example 3, the left-hand side $G^\mu{}_\nu +
\Lambda\delta^\mu_\nu$ and the right-hand side $\kappa T^\mu{}_\nu$ of the Einstein
equations as two 8 by 8 tables, and their difference (zero everywhere).
```

In [17]:

```
sol = solutions["example 3"]
G_diag = [G_hidden] + [G_space] * 3 + [G_time] + [G_extra] * 3
left_side = np.array([[float(G_diag[mu].subs({A: 1, H: 1}) + sol["Lambda"])
                       if mu == nu else 0.0 for nu in range(8)] for mu in range(8)])
right_side = np.array([[float(sol["T"][mu][nu]) for nu in range(8)]
                       for mu in range(8)])
check(np.max(np.abs(left_side - right_side)) == 0.0,
      "example 3: the two 8 by 8 tables are equal entry by entry")
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.4))
scale = np.abs(right_side).max()
panels = ((left_side, "$G^\\mu{}_\\nu + \\Lambda\\delta^\\mu_\\nu$"),
          (right_side, "$\\kappa T^\\mu{}_\\nu$ of the condensate"),
          (left_side - right_side, "difference"))
for ax, (values, title) in zip(axes, panels):
    ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
    for mu in range(8):
        value = values[mu, mu]
        ink = "white" if abs(value) > 0.6 * scale else "black"  # readable on dark
        ax.text(mu, mu, f"{value:.0f}", ha="center", va="center", fontsize=7,
```

```
                color=ink)
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)
    ax.set_title(title, fontsize=9)
fig.suptitle("Example 3 ($A = 1$, $\\Lambda = -30H^2$): the Einstein equations hold "
             "component by component")
save_figure(fig, "tensor_equality",
            "Example 3 (the deflating member $A = 1$, $\\Lambda = -30H^2$, units "
            "$H = \\kappa = 1$): left, the left-hand side $G^\\mu{}_\\nu + "
            "\\Lambda\\delta^\\mu_\\nu$ of the Einstein equations; middle, the "
            "right-hand side $\\kappa T^\\mu{}_\\nu$ computed from the condensate "
            "$\\Phi = e^{-4ix_4}\\Phi_0$; right, their difference. Rows $\\mu$ and "
            "columns $\\nu$ run over $x_0$ to $x_7$; the colour is the value (red "
            "positive, blue negative, white zero) and the numbers are the diagonal "
            "entries: $-6$ (minus the energy density, $x_4$) and $-18$ (the "
            "pressure, every other direction). All 64 entries agree exactly; every "
            "mixed entry is zero.")
```

Out [17]:

```
PASS example 3: the two 8 by 8 tables are equal entry by entry
```

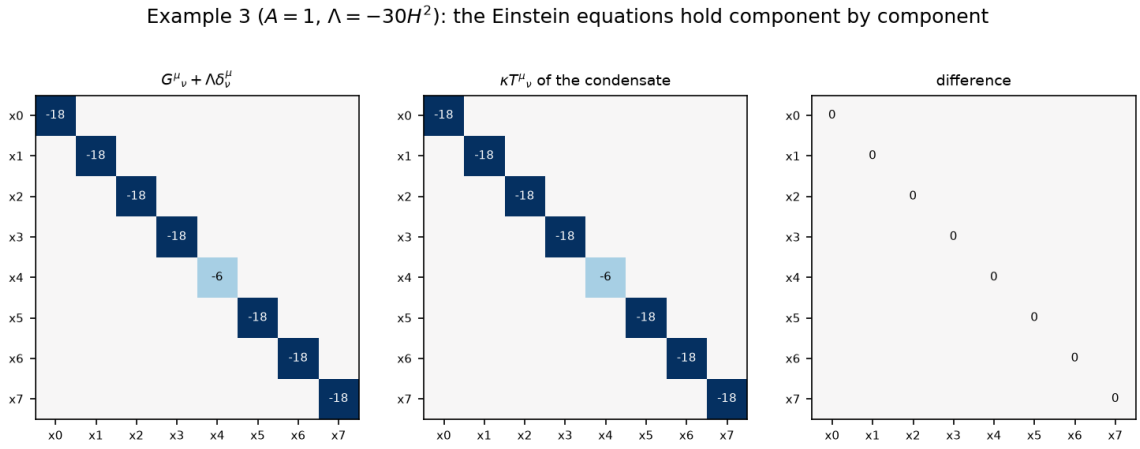

Figure 341: Example 3 (the deflating member $A = 1$, $\Lambda = -30H^2$, units $H = \kappa = 1$): left, the left-hand side $G^\mu{}_\nu + \Lambda\delta^\mu_\nu$ of the Einstein equations; middle, the right-hand side $\kappa T^\mu{}_\nu$ computed from the condensate $\Phi = e^{-4ix_4}\Phi_0$; right, their difference. Rows μ and columns ν run over x_0 to x_7; the colour is the value (red positive, blue negative, white zero) and the numbers are the diagonal entries: -6 (minus the energy density, x_4) and -18 (the pressure, every other direction). All 64 entries agree exactly; every mixed entry is zero. (Notebook 12d, figure 4.)

Out [17]:

```
Figure 12d.4 saved as Revision/textbook/figures/12d_4_tensor_equality.png
```

Text cell:

```
## 13. The solution in time

The next cell draws example 1 as functions of the time $x_4$: on the left the real
parts of three components of $\Phi(x_4) = e^{-4ix_4}\Phi_0$ (they oscillate with
the period $2\pi/4$) and the density $S = 12/5$, which stays constant; on the
right, on a logarithmic axis, the scale factor $e^{a_4} = e^{x_4}$ of 3-space and
$e^{-a_4} = e^{-x_4}$ of the extra times, which deflate exponentially, together
with those of example 2 ($a_4 = \sqrt5\,x_4$).
```

In [18]:

```
sol = solutions["example 1"]
phi_numbers = np.array([complex(entry) for entry in sol["phi0"]])
biggest = np.argsort(-np.abs(phi_numbers), kind="stable")[:3]  # three components
times = np.linspace(0.0, 3.0, 601)  # x4 in units 1/H
density = []
for x_value in times:  # S from the numbers, at every time (it must stay 12/5)
    phi_x = phi_numbers * np.exp(-4j * x_value)
    density.append((phi_x.conj() @ np.array(C.tolist(), dtype=float) @ phi_x).real)
density = np.array(density)
check(np.max(np.abs(density - 12 / 5)) < 1e-12,
      "example 1: S(x4) = 12/5 at every time (numerically)")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
for index in sorted(biggest):
    left.plot(times, (phi_numbers[index] * np.exp(-4j * times)).real,
              label=f"Re $\\Phi_{{{index + 1}}}(x_4)$")
left.plot(times, density, color="black", linewidth=2.0,
          label="$S = \\bar\\Phi\\Phi = 12/5$")
left.set_ylim(-1.35, 4.0)  # room for the legend above the curves
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("value")
left.set_title("example 1: the condensate oscillates, $S$ stays constant")
left.legend(fontsize=8, loc="upper center", ncol=2)
for name, style in (("example 1", "-"), ("example 2", "--")):
    slope = float(solutions[name]["A"])
    right.semilogy(times, np.exp(slope * times), style, color="tab:orange",
                   label=f"3-space $e^{{a_4}}$, {name}")
    right.semilogy(times, np.exp(-slope * times), style, color="tab:blue",
                   label=f"extra times $e^{{-a_4}}$, {name}")
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("scale factor")
right.set_title("3-space inflates, the extra times deflate")
right.legend(fontsize=8)
save_figure(fig, "solution_in_time",
            "The exact solution of example 1 as a function of the time $x_4$ (units "
            "$1/H$). Left: the real parts of the three largest components of the "
            "condensate $\\Phi(x_4) = e^{-4ix_4}\\Phi_0$, which oscillate with the "
            "period $2\\pi/4 \\approx 1.57$, and its density $S = \\bar\\Phi\\Phi = "
            "12/5$ (black), constant at every time. Right, on a logarithmic axis: "
            "the scale factor $e^{a_4}$ of 3-space (orange) and $e^{-a_4}$ of the "
```

```
"three extra times (blue), for example 1 ($a_4 = Hx_4$, solid) and "
"example 2 ($a_4 = \\sqrt5\\,Hx_4$, dashed): the extra times deflate "
"exponentially while 3-space inflates, driven by the condensate.")
```

Out [18]:

```
PASS example 1: S(x4) = 12/5 at every time (numerically)
```

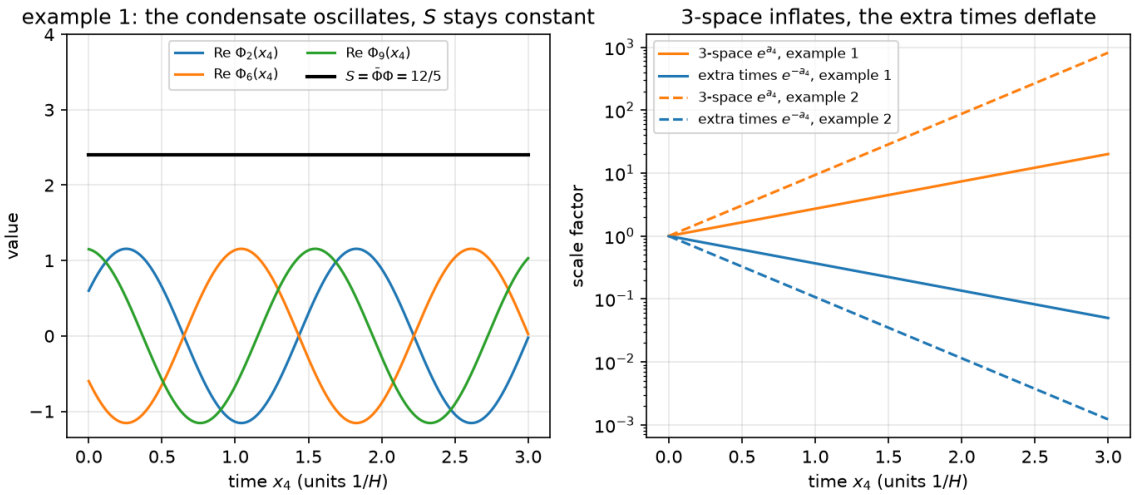

Figure 342: The exact solution of example 1 as a function of the time x_4 (units $1/H$). Left: the real parts of the three largest components of the condensate $\Phi(x_4) = e^{-4ix_4}\Phi_0$, which oscillate with the period $2\pi/4 \approx 1.57$, and its density $S = \bar{\Phi}\Phi = 12/5$ (black), constant at every time. Right, on a logarithmic axis: the scale factor e^{a_4} of 3-space (orange) and e^{-a_4} of the three extra times (blue), for example 1 ($a_4 = Hx_4$, solid) and example 2 ($a_4 = \sqrt{5}\,Hx_4$, dashed): the extra times deflate exponentially while 3-space inflates, driven by the condensate. (Notebook 12d, figure 5.)

Out [18]:

```
Figure 12d.5 saved as Revision/textbook/figures/12d_5_solution_in_time.png
```

Text cell:

```
## 14. The whole family for the deflating member $A = 1$

For $A = 1$, $M = -5$ and $S = 12/5$ (the condensate of examples 1 and 3) every
cosmological constant gives a solution: $m = -(36 + 2\Lambda)/S$,
$\lambda = (M - m)/S$, $\kappa\rho = -24 - \Lambda$, $\kappa p = 12 + \Lambda$. The
next cell checks this family against the exact solutions and draws $m$, $\lambda$,
$\kappa\rho$ and $\kappa p$ as functions of $\Lambda$. The energy density is
positive only for $\Lambda < -24H^2$, and then $p < -\rho$: $w < -1$, because
$\kappa(\rho + p) = \kappa MS = -12H^2$ for every $\Lambda$ (the violation of the
null energy condition that Notebook 12a proved for every source of this metric).
```

In [19]:

```
lam_axis = sp.symbols("Lambda_axis", real=True)
S_one = sp.Rational(12, 5)
m_family = -(36 + 2 * lam_axis) / S_one
lam_family = (-5 - m_family) / S_one
rho_family_line = m_family * S_one + lam_family * S_one ** 2 / 2
p_family_line = lam_family * S_one ** 2 / 2
check(sp.expand(rho_family_line - (-24 - lam_axis)) == 0
      and sp.expand(p_family_line - (12 + lam_axis)) == 0
      and m_family.subs(lam_axis, 0) == solutions["example 1"]["m"]
      and m_family.subs(lam_axis, -30) == solutions["example 3"]["m"],
      "A = 1 family: kappa rho = -24 - Lambda, kappa p = 12 + Lambda")
axis = np.linspace(-45.0, 15.0, 301)
functions = [sp.lambdify(lam_axis, f, "numpy") for f in
             (m_family, lam_family, rho_family_line, p_family_line)]
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(axis, functions[0](axis), label="mass $m/H$")
left.plot(axis, functions[1](axis), "--", label="self-coupling $\\lambda$")
right.plot(axis, functions[2](axis), label="$\\kappa\\rho/H^2$")
right.plot(axis, functions[3](axis), "--", label="$\\kappa p/H^2$")
for ax in (left, right):
    ax.axvspan(-45.0, -24.0, color="tab:green", alpha=0.1)
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xlabel("cosmological constant $\\Lambda/H^2$")
for name, marker in (("example 1", "o"), ("example 3", "s")):
    sol = solutions[name]
    x_value = float(sol["Lambda"])
    left.plot([x_value] * 2, [float(sol["m"]), float(sol["lambda"])], marker,
              color="black")
    right.plot([x_value] * 2, [float(sol["rho"]), float(sol["p"])], marker,
               color="black", label=name)
left.set_title("the condensate's parameters ($A = 1$, $M = -5H$)")
right.set_title("the source it provides")
left.legend(fontsize=8)
right.legend(fontsize=8)
save_figure(fig, "family_in_lambda",
            "The family of exact solutions with the deflating member $a_4 = Hx_4$ "
            "and the condensate of examples 1 and 3 ($M = -5H$, $\\kappa S = 12/5$), "
            "as functions of the cosmological constant $\\Lambda/H^2$ (units "
            "$H = \\kappa = 1$). Left: the mass $m = -(36 + 2\\Lambda)/S$ and the "
            "self-coupling $\\lambda = (M - m)/S$ the condensate must have. Right: "
            "its energy density $\\kappa\\rho = -24 - \\Lambda$ and pressure "
            "$\\kappa p = 12 + \\Lambda$. Black circles: example 1 ($\\Lambda = 0$); "
            "black squares: example 3 ($\\Lambda = -30$). In the green region "
            "$\\Lambda < -24H^2$ the energy density is positive, and there "
            "$p < -\\rho$ ($w < -1$), since $\\kappa(\\rho + p) = -12H^2$ for every "
            "$\\Lambda$.")
```

Out [19]:

```
PASS A = 1 family: kappa rho = -24 - Lambda, kappa p = 12 + Lambda
```

Out [19]:

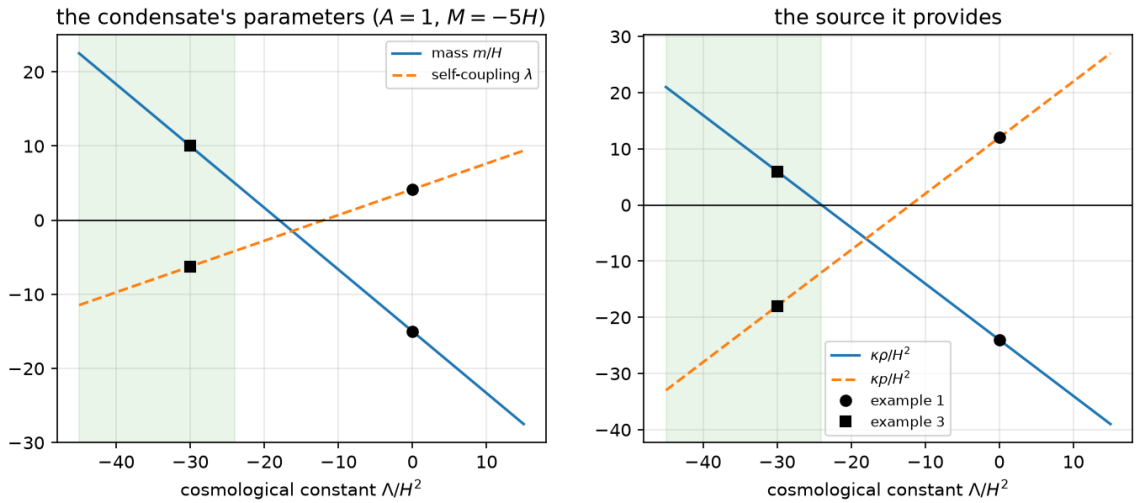

Figure 343: The family of exact solutions with the deflating member $a_4 = Hx_4$ and the condensate of examples 1 and 3 ($M = -5H$, $\kappa S = 12/5$), as functions of the cosmological constant Λ/H^2 (units $H = \kappa = 1$). Left: the mass $m = -(36 + 2\Lambda)/S$ and the self-coupling $\lambda = (M - m)/S$ the condensate must have. Right: its energy density $\kappa\rho = -24 - \Lambda$ and pressure $\kappa p = 12 + \Lambda$. Black circles: example 1 ($\Lambda = 0$); black squares: example 3 ($\Lambda = -30$). In the green region $\Lambda < -24H^2$ the energy density is positive, and there $p < -\rho$ ($w < -1$), since $\kappa(\rho + p) = -12H^2$ for every Λ. (Notebook 12d, figure 6.)

```
Figure 12d.6 saved as Revision/textbook/figures/12d_6_family_in_lambda.png
```

Text cell:

```
## 15. What this shows, and what it does not

- **PROVED here (exact):** for each of the three examples the author's metric with
  the linear member ($a_4 = Hx_4$ for examples 1 and 3: the extra times deflate as
  $e^{-Hx_4}$; $a_4 = \sqrt5\,Hx_4$ for example 2) and the condensate
  $\Phi = e^{-iwx_4}\Phi_0$ solve the field equation of dirac16complex00 (16
  components) and the Einstein equations (64 components) at every time and every
  value of the hidden coordinate. So the coupled equations of the theory have exact
  solutions in which a field of the theory drives the exponential deflation of the
  extra times. This answers, for these examples, the combination that the Revision
  record leaves open; it is a computation of this notebook, not a Revision record.
- **ASSUMED:** Einstein gravity ($\alpha_2 = \alpha_3 = 0$); the record's sign
  convention $\sigma_T = +1$ of the energy-momentum tensor (with the opposite sign
  every $\rho$ and $p$ changes sign and the conditions change accordingly); the
  classical commuting field dirac16complex00, not the quantised dirac16complex; the
  chosen values of $\Lambda$, $m$, $\lambda$.
- **The sign of $A$ is not selected:** the same condensate solves the equations
  with $-A$ (inflating extra times); the deflation is the author's choice of sign.
- **Energy:** with $\Lambda = 0$ the energy density is negative (examples 1 and 2);
  a positive energy density needs $\Lambda < -(21 + 3A^2)H^2$ and then has
  $w < -1$ (example 3: $w = -3$).
- **Not shown:** whether such a condensate is stable, how it could arise, any
```

```
  statement about the quantised field, and nothing about the creation of
  universes.
```

Text cell:

```
## 16. The last check

The last cell checks that the six figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [20]:

```
figure_names = ["condensate_frequency", "three_gamma_bilinears", "allowed_sources",
                "tensor_equality", "solution_in_time", "family_in_lambda"]
paths = [output_file(f"{FIGURE_FOLDER}/12d_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths), "all six figure files exist")
all_checks_passed()
```

Out [20]:

```
PASS all six figure files exist
ALL 40 CHECKS PASSED (notebook 12d)
```

Text cell:

```
## 17. What this notebook showed

- With the author's gammas, the spin connection of the author's metric gives
  $\sum_\mu\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$, and a homogeneous condensate
  obeys $\Phi' = \mathcal{A}\Phi$ with $\mathcal{A}^2 = -(M^2 - 9H^2)$: it
  oscillates with $w = \sqrt{M^2 - 9H^2}$ and keeps $S$ constant, whatever $a_4$
  does (PROVED, reproducing the Revision record).
- The family $\Phi_0(t) = v_1 + t\,c\,v_2$ has all 15 three-gamma bilinears zero for
  every real $t$ and the density $S(t) = 2t|v_1^\dagger Cv_2|^2$ of either sign
  (PROVED); its energy-momentum tensor is diagonal, $\rho = mS + \lambda S^2/2$,
  $p_3 = p_t = p_0 = \lambda S^2/2$, for every $a_4$ (PROVED).
- Three exact solutions of the coupled Einstein and dirac16complex00 equations,
  two with the extra times deflating as $e^{-Hx_4}$: all 16 + 64 components hold
  exactly (PROVED for the stated numbers, under the ASSUMED convention
  $\sigma_T = +1$ and Einstein gravity).
- The equations do not select the sign of $A$; a positive energy density requires
  $\Lambda < -(21 + 3A^2)H^2$ and $w < -1$ (PROVED).
```

12.30 Line-by-line walk-through of Notebook 12d

The notebook has 20 code cells, In [1] to In [20].

In [1], the set-up cell. As in Notebook 12a (Section 12.16, In [1]), except that the comments hold the run instructions of Notebook 12d (Section 12.28) and `NOTEBOOK_ID = "12d"`.

In [2], the records and the open question. After the imports of `contextlib` and `io` the cell names six record files:

```
GAMMAS = "Revision/algebra/gammas.json"  # the author's gamma matrices
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
PY = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy record
WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram record
EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
DOC = "Revision/docs/DIRAC16COMPLEX00_FIELD_THEORY.md"  # the record's document
```

These are the gamma matrices, the equations of gravity, the three reports and the record's document on the field dirac16complex00. Then follow the three helpers of Notebook 12a, In [2], which Section 12.16 explains.

```
OPEN_SENTENCE = ("This record contains no check that combines a witness with the "
                 "Einstein conditions")
document = repository_file(DOC).read_text(encoding="utf-8")
say(f"{DOC} says: \"{OPEN_SENTENCE}.\"")
check(OPEN_SENTENCE in document, "the record leaves this combination open")
```

The sentence of the record's document that states the open question is stored (two strings in brackets are joined into one), the document is read as text, the sentence is printed (`\"` is a quotation mark inside a string), and the check requires the document to contain it. If someone changes the document, this check fails and the notebook must be revised.

In [3], the author's gamma matrices and C.

```
import itertools  # loops over all combinations of indices

import numpy as np  # floating-point arrays, used only for the plots
import sympy as sp  # exact algebra

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the frame metric eta, x0..x7
I16, Z16 = sp.eye(16), sp.zeros(16, 16)  # the 16 by 16 unit and zero matrices
```

The imports; the coordinate names; the diagonal of the frame metric η; the exact 16 by 16 unit matrix `sp.eye(16)` and zero matrix.

```
fixture = read_json(GAMMAS)
gamma = [sp.Matrix([[sp.Rational(x) for x in row] for row in matrix])
         for matrix in fixture["gamma"]]  # gamma[a] = gamma^(xa)
```

The record stores the eight gammas as lists of 16 rows of 16 entries, each written as text; `sp.Rational(x)` turns each text into an exact number and `sp.Matrix` makes a matrix. `gamma[0]` is $\gamma^{(x_0)}$, ..., `gamma[7]` is $\gamma^{(x_7)}$: the author's eight real 16 by 16 matrices.

```
clifford = all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
               == 2 * ETA[a] * (1 if a == b else 0) * I16
               for a in range(8) for b in range(8))
reproduces(clifford, "the author's gammas satisfy the Clifford relation",
           PY, "authorT16_clifford")
```

For all 64 pairs: $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}$ (for sympy matrices `*` is the matrix product) must equal $2\eta^{ab}$ times the unit matrix, that is $2\eta^{aa}$ on the diagonal $a = b$ and 0 otherwise.

```
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g^(x0) g^(x1) g^(x2) g^(x3)
stored_C = sp.Matrix([[sp.Rational(x) for x in row] for row in fixture["C"]])
properties = (C == stored_C and C == C.T and C * C == I16
              and all((C * g).T == -(C * g) for g in gamma))
reproduces(properties, "C is the record's C, real symmetric, C^2 = 1, C gamma "
           "antisymmetric", PY, "authorT16_C_properties")
```

C is the product of the four space-like gammas; it must equal the C stored in the record, be symmetric (`C.T` is the transpose), square to 1 and make every $C\gamma^{(a)}$ antisymmetric (Section 12.26).

In [4], the spin connection.

```
H = sp.symbols("H", positive=True)  # the constant H of the metric
z = sp.symbols("z", positive=True)  # z = 6 H x0 in (0, pi/2)
x4 = sp.symbols("x4", real=True)  # the time
a4 = sp.Function("a4")(x4)  # an arbitrary function a4(x4)
cz = sp.symbols("cz", positive=True)  # cot z > 0 on the patch
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
s6 = sp.sin(z) ** sp.Rational(1, 6)  # the warp sin(z)^(1/6) of a length
E = [sp.cot(z)] + [sp.exp(a4) * s6] * 3 + [sp.Integer(1)] + [sp.exp(-a4) * s6] * 3
g = [ETA[a] * E[a] ** 2 for a in range(8)]  # g_aa = eta_aa E_a^2
```

Symbols as in Notebook 12a. `E` lists the eight scale factors $E_a = \sqrt{|g_{aa}|}$ (the diagonal **vielbein** of Chapter 6), and the metric is rebuilt from them as $g_{aa} = \eta_{aa}E_a^2$; a_4 is an arbitrary function, so every result holds for every history.

```
TRIG = {sp.sin(z): 1 / sp.sqrt(1 + cz ** 2), sp.cos(z): cz / sp.sqrt(1 + cz ** 2),
        sp.tan(z): 1 / cz, sp.cot(z): cz}  # every function of z through cot z


def clean(expr):
    expr = expr.subs(sp.Derivative(a4, (x4, 2)), ad2).subs(sp.Derivative(a4, x4), ad1)
    expr = sp.expand_trig(expr).subs(TRIG)
    return sp.expand(sp.cancel(sp.powsimp(sp.expand(expr), force=True)))


def d(expr, mu):
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # d/dx0 = (dz/dx0) d/dz = 6 H d/dz
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7
```

The same tools as in Notebook 12a, In [5] and In [7] (docstrings left out): `clean` writes derivatives as `ad1`, `ad2` (the second derivative first) and every function of z through $\cot z$; `d` is the partial derivative.

```
Gamma = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
for a, b, c in itertools.product(range(8), repeat=3):  # diagonal-metric formula
    value = 0
    if a == c:
        value += d(g[a], b)
    if a == b:
        value += d(g[a], c)
    if b == c:
        value -= d(g[b], a)
    Gamma[a][b][c] = value / (2 * g[a])
S_ab = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
        for a in range(8)]
```

The Christoffel symbols exactly as in Notebook 12a, In [5]; then the 64 matrices $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$, the generators of the spinor transformations (Chapter 5).

```
omega = [[[clean(ETA[a] * E[a] * Gamma[a][mu][b] / E[b]) if a != b else 0
           for b in range(8)] for a in range(8)] for mu in range(8)]
antisymmetric = all(sp.expand(omega[mu][a][b] + omega[mu][b][a]) == 0
                    for mu in range(8) for a in range(8) for b in range(8))
check(antisymmetric, "omega_mu ab = -omega_mu ba (canonical spin connection)")
```

The canonical spin connection of the diagonal vielbein, $\omega_{\mu\,ab} = \eta_{aa}(E_a/E_b)\,\Gamma^a{}_{\mu b}$ for $a \neq b$ and 0 for $a = b$ (the text cell before the cell derives it from the definition). The check confirms that it is antisymmetric in a and b, as a connection of this kind must be.

```
Omega = [sum((omega[mu][a][b] * S_ab[a][b] / 2 for a in range(8) for b in range(8)
              if omega[mu][a][b] != 0), Z16) for mu in range(8)]
for mu in range(8):
    terms = sum(1 for a in range(8) for b in range(8) if omega[mu][a][b] != 0)
    say(f"Omega_{NAMES[mu]}: {terms} nonzero entries omega_mu ab")
```

$\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu\,ab}S^{ab}$ for each μ; `sum(..., Z16)` starts the sum at the zero matrix (a sum of matrices must start from a matrix). The loop prints how many $\omega_{\mu\,ab}$ are not zero: four for each 3-space direction and extra time (the pairs with x_4 and with x_0, in both orders), none for x_4 and x_0.

In [5], the term 3H gamma^(x0).

```
gamma_curved = [gamma[mu] / E[mu] for mu in range(8)]  # gamma^mu = gamma^(mu)/E_mu
reproduces(Omega[4] == Z16 and Omega[0] == Z16, "Omega_x4 = Omega_x0 = 0",
           PY, "authorT16_Omega_x0_x4_zero")
```

The curved gammas $\gamma^\mu = \gamma^{(\mu)}/E_\mu$; the spin connection along the time and along x_0 is zero.

```
anticommute = all((gamma_curved[mu] * Omega[mu] + Omega[mu] * gamma_curved[mu])
                  .applyfunc(clean) == Z16 for mu in range(8))
reproduces(anticommute, "{gamma^mu, Omega_mu} = 0 for each mu (no sum)",
           PY, "authorT16_anticommutator_no_sum")
```

For each μ separately $\gamma^\mu\Omega_\mu + \Omega_\mu\gamma^\mu = 0$; `.applyfunc(clean)` applies `clean` to every entry of the matrix. (A line that starts with a dot continues the expression of the line above, inside the brackets.)

```
gravity_term = sum((gamma_curved[mu] * Omega[mu] for mu in range(8)), Z16)
gravity_term = gravity_term.applyfunc(clean)
reproduces(gravity_term == 3 * H * gamma[0], "gamma^mu Omega_mu = 3 H gamma^(x0)",
           PY, "authorT16_gravity_term")
reproduces(gravity_term == 3 * H * gamma[0], "the same, the lead's independent check",
           EMT, "gamma_Omega_equals_3H_gamma0")
```

The sum $\sum_\mu \gamma^\mu\Omega_\mu$ equals $3H\gamma^{(x_0)}$: no a_4 and no z remain, because the time-direction terms of the three inflating and the three deflating directions cancel. Two records confirm it.

In [6], the condensate equation.

```
MM = sp.symbols("M", real=True)  # the effective mass M = m + lambda S


def condensate_matrix(M_value, H_value):
    return -gamma[4] * (M_value * I16 - 3 * H_value * gamma[0])


A_sym = condensate_matrix(MM, H)
```

`condensate_matrix` builds $\mathcal{A} = -\gamma^{(x_4)}(M - 3H\gamma^{(x_0)})$ of Section 12.26 for given M and H (docstring left out); `A_sym` is the matrix with the symbols M and H.

```
reduced = (gamma[4] * A_sym + gravity_term - MM * I16).applyfunc(sp.expand)
reproduces(reduced == Z16 and not A_sym.has(z),
           "gamma^(x4) A + 3 H gamma^(x0) = M: the equation is Phi' = A Phi",
           WL, "condensate_equation_x0_consistent")
check((A_sym * A_sym + (MM ** 2 - 9 * H ** 2) * I16).applyfunc(sp.expand) == Z16,
      "A^2 = -(M^2 - 9 H^2): every condensate oscillates with w = sqrt(M^2 - 9 H^2)")
reproduces((C * A_sym + A_sym.T * C).applyfunc(sp.expand) == Z16,
           "C A + A^T C = 0: S is constant along x4", PY,
           "authorT16_condensate_S_constant")
```

Three facts derived in Section 12.26: inserting $\Phi' = \mathcal{A}\Phi$ into the field equation gives $\gamma^{(x_4)}\mathcal{A} + 3H\gamma^{(x_0)} = M$ (and $\mathcal{A}$ is free of z); $\mathcal{A}^2 = -(M^2 - 9H^2)$; and $C\mathcal{A} + \mathcal{A}^T C = 0$, so S is constant.

In [7], figure 1: the frequency of a condensate.

```
m_axis = np.linspace(-8.0, 8.0, 801)  # M/H
fig, ax = plt.subplots()
```

```
ax.plot(m_axis[m_axis <= -3.0], np.sqrt(m_axis[m_axis <= -3.0] ** 2 - 9.0),
        color="tab:blue", label="$w = \\sqrt{M^2 - 9H^2}$")
ax.plot(m_axis[m_axis >= 3.0], np.sqrt(m_axis[m_axis >= 3.0] ** 2 - 9.0),
        color="tab:blue")
ax.axvspan(-3.0, 3.0, color="grey", alpha=0.2, label="$|M| < 3H$: no oscillation")
```

801 masses from -8 to 8 (units $H = 1$); `m_axis[m_axis <= -3.0]` keeps the entries not above -3 (an array of true and false values used as an index), and the two branches of $w = \sqrt{M^2 - 9}$ are drawn; the grey band marks $|M| < 3$.

```
worst = 0.0  # the largest deviation of a numerical eigenvalue from +-i w
upper = []  # for each mass: how many eigenvalues have a positive imaginary part
for M_value in (-7.0, -5.0, -4.0, 4.0, 5.0, 7.0):
    matrix = np.array(condensate_matrix(M_value, 1).tolist(), dtype=float)
    eigenvalues = np.linalg.eigvals(matrix)
    w_value = np.sqrt(M_value ** 2 - 9.0)
    worst = max(worst, np.max(np.abs(np.abs(eigenvalues.imag) - w_value)),
                np.max(np.abs(eigenvalues.real)))
    upper.append(int(np.sum(eigenvalues.imag > 0)))  # the ones near +i w
    ax.plot([M_value] * 16, np.abs(eigenvalues.imag), "o", color="black",
            markersize=4, zorder=3)  # zorder 3: drawn on top of the red squares
```

For six masses the exact matrix is converted to a floating-point array and its 16 eigenvalues are computed numerically with `np.linalg.eigvals`. `worst` records the largest distance of an imaginary part (in absolute value) from w and the largest real part. `eigenvalues.imag > 0` is an array of true and false values, one for each eigenvalue; `np.sum` counts the true ones (true counts as 1), and `upper.append` stores this count, the number of eigenvalues near $+iw$, for each mass. The absolute imaginary parts are drawn as black dots at the mass (`[M_value] * 16` repeats the mass 16 times; `zorder=3` draws them above the other marks).

```
ax.plot([], [], "o", color="black", markersize=4,
        label="$|\\mathrm{Im}|$ of the eigenvalues of $\\mathcal{A}$")
ax.plot([-5, 5], [4, 4], "s", color="tab:red", markersize=7,
        label="$M = \\pm5H$: $w = 4H$")
ax.set_xlabel("effective mass $M/H$")
ax.set_ylabel("frequency $w/H$")
ax.set_title("The frequency of a homogeneous condensate")
ax.legend(fontsize=8, loc="upper center")
say(f"largest deviation of a numerical eigenvalue from +-i w: {worst:.0e}")
say(f"eigenvalues near +i w, for each of the six masses: {upper}")
check(worst < 1e-9 and upper == [8] * 6,
      "the 16 eigenvalues of A are +i w and -i w, eight of each (six masses)")
save_figure(fig, "condensate_frequency",
            ...)
```

An empty plot with a label gives the dots one legend entry; red squares mark $M = \pm5$, $w = 4$. The largest deviation, 10^{-14}, and the six counts, all 8, are printed; the check requires a deviation below 10^{-9} and the count 8 at every mass (`[8] * 6` is the list of six eights), so eight eigenvalues are near $+iw$ and the other eight near $-iw$. **What the figure shows.** All dots lie on the curve $w = \sqrt{M^2 - 9H^2}$; each dot stands for all 16 eigenvalues of its mass, because the absolute value of

the imaginary part does not distinguish $+iw$ from $-iw$. Inside the grey band there is no oscillation, so the examples use $M = \pm 5H$.

In [8], the family of condensates with the 15 bilinears zero.

```
t = sp.symbols("t", real=True)  # the free real parameter of the family
TRIPLES = list(itertools.combinations(range(8), 3))  # all 56 sets {a, b, c}
NEEDED = sorted({tuple(sorted((i, 4, 0))) for i in (1, 2, 3, 5, 6, 7)}
                | {tuple(sorted((i, j, 4))) for i in (1, 2, 3) for j in (5, 6, 7)})
```

The real parameter t. `itertools.combinations(range(8), 3)` lists the 56 sets of three different positions in increasing order. `NEEDED` is the set (braces) of the 15 triples whose bilinears the field equations need to vanish: $\{i, x_4, x_0\}$ for the six directions i, joined (`|`) with $\{i, j, x_4\}$ for i in 3-space and j an extra time; each triple is sorted, and the whole is sorted into a list.

```
def witness_parts(M_value, H_value):
    w_value = sp.sqrt(M_value ** 2 - 9 * H_value ** 2)
    A_value = condensate_matrix(M_value, H_value)
    spaces = []
    for sign in (-1, 1):  # the eigenvalue of g^(x1)g^(x5) = g^(x2)g^(x6) = ...
        stack = sp.Matrix.vstack(A_value + sp.I * w_value * I16,
                                 gamma[1] * gamma[5] - sign * I16,
                                 gamma[2] * gamma[6] - sign * I16,
                                 gamma[3] * gamma[7] - sign * I16)
        spaces.append(stack.nullspace())  # all solutions of stack * v = 0
    v1, v2 = spaces[0][0], spaces[1][0]
    c = sp.conjugate(sp.expand((v1.H * C * v2)[0]))  # c = conj(v1^dagger C v2)
    return spaces, v1, v2, c, w_value, A_value
```

The record's recipe (docstring left out). `sp.Matrix.vstack` stacks four 16 by 16 matrices into one 64 by 16 matrix; a column v with `stack * v = 0` satisfies four equations at once: $\mathcal{A}v = -iwv$ (`sp.I` is i), and $\gamma^{(x_1)}\gamma^{(x_5)}v = \gamma^{(x_2)}\gamma^{(x_6)}v = \gamma^{(x_3)}\gamma^{(x_7)}v = \pm v$ (each of these products squares to 1, so its eigenvalues are ± 1). `.nullspace()` returns a list of columns that span all solutions, computed exactly. v_1 is the first solution with the sign -1, v_2 with $+1$. `v1.H` is the conjugate transpose $v_1^\dagger$, and `[0]` takes the single entry of the 1 by 1 product; c is its complex conjugate.

```
def bilinear(phi, matrix):
    return sp.expand((phi.H * C * matrix * phi)[0])


def three_gamma(triple):
    a, b, c = triple
    return gamma[a] * gamma[b] * gamma[c]
```

`bilinear` computes $\bar{\Phi}X\Phi = \Phi^\dagger C X \Phi$ for a matrix X; `three_gamma` builds $\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}$ for a triple (docstrings left out).

```
spaces, v1, v2, c, w, A_witness = witness_parts(-5, 1)
phi_t = (v1 + t * c * v2).applyfunc(sp.expand)  # the family Phi_0(t)
```

```
check([len(space) for space in spaces] == [1, 1],
      "the two null spaces are one-dimensional (v1 and v2 are unique)")
report("frequency w at M = -5 H", w, "H")
check((A_witness * phi_t + sp.I * w * phi_t).applyfunc(sp.expand)
      == sp.zeros(16, 1), "A Phi_0(t) = -i w Phi_0(t) for every t")
needed_values = [bilinear(phi_t, three_gamma(triple)) for triple in NEEDED]
check(len(NEEDED) == 15 and all(value == 0 for value in needed_values),
      "the 15 three-gamma bilinears vanish for every real t")
```

The recipe at $M = -5$, $H = 1$, and the family $\Phi_0(t) = v_1 + t\,c\,v_2$ with t kept as a symbol. Each solution space is one-dimensional, so v_1 and v_2 are unique up to a factor; the frequency is $w = 4$; every member of the family is an eigenvector of $\mathcal{A}$ with the eigenvalue $-iw$; and the 15 needed bilinears are zero as expressions in t, so for every real t.

In [9], the density of the family.

```
S_t = bilinear(phi_t, I16)  # S(t) = Phi_0(t)^dagger C Phi_0(t)
cross = sp.expand((v1.H * C * v2)[0])  # v1^dagger C v2
report("v1^dagger C v2", cross)
report("S(t)", S_t)
check(bilinear(v1, I16) == 0 and bilinear(v2, I16) == 0
      and sp.expand(S_t - 2 * t * cross * sp.conjugate(cross)) == 0,
      "S(t) = 2 t |v1^dagger C v2|^2: every real density occurs")
```

The density of the family, the cross term $\sigma = v_1^\dagger C v_2 = -128/25 - 96i/25$, and $S(t) = 2048t/25$; the check confirms $v_1^\dagger C v_1 = v_2^\dagger C v_2 = 0$ and $S(t) = 2t|\sigma|^2$ (derived in Section 12.26).

```
_, u1, u2, c_plus, w_plus, A_plus = witness_parts(5, 1)  # the record's M = +5
phi_plus = (u1 + c_plus * u2).applyfunc(sp.expand)  # t = 1
S_plus = bilinear(phi_plus, I16)
ok_plus = ((A_plus * phi_plus + sp.I * w_plus * phi_plus).applyfunc(sp.expand)
           == sp.zeros(16, 1)
           and all(bilinear(phi_plus, three_gamma(tr)) == 0 for tr in NEEDED)
           and S_plus != 0 and sp.im(S_plus) == 0)
reproduces(ok_plus, "M = +5, H = 1: a witness with S real, not 0, 15 bilinears 0",
           PY, "authorT16_condensate_witness")
```

The recipe repeated at $M = +5$, where the record ran it with the author's gammas, at $t = 1$ (the record's witness; `_` receives the unused first value): it is an eigenvector, its 15 bilinears vanish and its density is a real number different from 0 (`sp.im` is the imaginary part).

In [10], figure 2: all 56 three-gamma bilinears.

```
phi_one = phi_t.subs(t, 1)  # Phi_0(1)
all_values = [bilinear(phi_one, three_gamma(triple)) for triple in TRIPLES]
check(all(sp.im(value) == 0 for value in all_values),
      "all 56 three-gamma bilinears of Phi_0(1) are real numbers")
nonzero = [triple for triple, value in zip(TRIPLES, all_values) if value != 0]
say("nonzero bilinears: " + ", ".join(
    "".join(NAMES[i][1] for i in triple) for triple in nonzero))
check(all(4 not in triple and 0 not in triple for triple in nonzero),
      "every nonzero one avoids x4 and x0")
```

All 56 bilinears of the member $t = 1$ are computed and are real. The nonzero ones are printed as digit strings (`NAMES[i][1]` is the second character of `"x1"`, the digit): 123, 127, 136, 167, 235, 257, 356, 567, all built from 3-space directions and extra times (positions 3 and 7, x_4 and x_0, never occur).

```
labels = ["".join(NAMES[i][1] for i in triple) for triple in TRIPLES]
numbers = np.array([float(value) for value in all_values])
is_needed = np.array([triple in NEEDED for triple in TRIPLES])
fig, ax = plt.subplots(figsize=(11.0, 4.2))
positions = np.arange(len(TRIPLES))
ax.bar(positions[~is_needed], numbers[~is_needed], color="tab:blue",
       label="other bilinears")
ax.plot(positions[is_needed], numbers[is_needed], "x", color="tab:red",
        markersize=8, label="the 15 the field equations need to vanish")
```

Labels, values and a true/false array marking the needed triples. `np.arange(56)` is 0 to 55; `~is_needed` is the opposite array. The other 41 bilinears are drawn as blue bars (`ax.bar`), the 15 needed ones as red crosses.

```
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(positions)
ax.set_xticklabels(labels, rotation=90, fontsize=6.5)
ax.set_xlabel("directions $abc$ of $\\bar\\Phi\\gamma^{(a)}\\gamma^{(b)}"
              "\\gamma^{(c)}\\Phi$ (digits: $x_0$ = 1, ..., $x_7$ = 8)")
ax.set_ylabel("value of the bilinear")
ax.set_title("The 56 three-gamma bilinears of the condensate $\\Phi_0(1)$, "
             "$M = -5H$")
ax.legend(fontsize=8)
save_figure(fig, "three_gamma_bilinears",
            ...)
```

Axis, tick labels turned by 90 degrees, titles and `save_figure`. **What the figure shows.** Every red cross sits on zero; only eight blue bars rise, at triples without x_4 and x_0, which no field equation constrains.

In [11], the complete kinetic tensor.

```
UNIT_H = {H: 1}  # the examples use H = 1
Omega_1 = [matrix.subs(UNIT_H) for matrix in Omega]
gamma_1 = [matrix.subs(UNIT_H) for matrix in gamma_curved]
g_1 = [entry.subs(UNIT_H) for entry in g]
```

The spin connection, the curved gammas and the metric with $H = 1$; a_4 and z stay arbitrary.

```
def D_phi(nu):
    return (A_witness if nu == 4 else Z16) + Omega_1[nu]


def D_phibar(nu):
    return (A_witness.T * C if nu == 4 else Z16) - C * Omega_1[nu]
```

The matrices that give the covariant derivatives (docstrings left out): $D_\nu\Phi = \partial_\nu\Phi + \Omega_\nu\Phi$, with $\partial_4\Phi = \mathcal{A}\Phi$ and no other derivative; and $D_\nu\bar\Phi = \partial_\nu\bar\Phi - \bar\Phi\Omega_\nu = \Phi^\dagger(\mathcal{A}^T C\,\delta_{\nu 4} - C\Omega_\nu)$, because $\partial_4\bar\Phi = (\mathcal{A}\Phi)^\dagger C = \Phi^\dagger\mathcal{A}^T C$.

```
K = [[(phi_t.H * ((C * gamma_1[mu] * D_phi(nu) - D_phibar(nu) * gamma_1[mu]) / 2)
       * phi_t)[0] for nu in range(8)] for mu in range(8)]
K_sym = [[clean((K[mu][nu] + (g_1[nu] / g_1[mu]) * K[nu][mu]) / 2)
          for nu in range(8)] for mu in range(8)]
```

All 64 components $K^\mu{}_\nu = \frac{1}{2}(\bar\Phi\gamma^\mu D_\nu\Phi - (D_\nu\bar\Phi)\gamma^\mu\Phi)$ as bilinears of $\Phi_0(t)$ (the phase e^{-iwx_4} cancels between $\Phi^\dagger$ and Φ). The symmetrised tensor: lower the upper index ($K_{\mu\nu} = g_{\mu\mu}K^\mu{}_\nu$), average with the exchanged component, raise again; this gives $\frac{1}{2}(K^\mu{}_\nu + (g_{\nu\nu}/g_{\mu\mu})K^\nu{}_\mu)$.

```
off_diagonal = all(K_sym[mu][nu] == 0 for mu in range(8) for nu in range(8)
                   if mu != nu)
reproduces(off_diagonal, "every off-diagonal K^(mu nu) vanishes, for every a4, a4', z",
           WL, "condensate_diagonal_witness_exact")
diagonal = [K_sym[mu][mu] for mu in range(8)]
say(f"diagonal of K: {diagonal}")
reproduces(all(diagonal[mu] == 0 for mu in range(8) if mu != 4)
           and sp.expand(diagonal[4] - (-5) * S_t) == 0,
           "K^x4_x4 = M S and every other diagonal entry is 0",
           PY, "authorT16_condensate_kinetic_diagonal")
```

Every off-diagonal component is zero for the whole family, every a_4 and every z; the diagonal is $[0,0,0,-2048t/5,0,0,0,0]$, that is $K^{x_4}{}_{x_4} = MS = -5\cdot 2048t/25$.

In [12], the energy-momentum tensor of the family.

```
m, lam = sp.symbols("m lambda", real=True)  # the mass and the self-coupling


def source_tensor(K_matrix, S_value, m_value, lam_value):
    L_value = (sum(K_matrix[mu][mu] for mu in range(8)) - m_value * S_value
               - lam_value * S_value ** 2 / 2)
    return [[sp.expand(-K_matrix[mu][nu] + (L_value if mu == nu else 0))
             for nu in range(8)] for mu in range(8)]
```

`source_tensor` builds $T^\mu{}_\nu = -K^{(\mu}{}_{\nu)} + \delta^\mu_\nu L$ with $L = \sum_\mu K^\mu{}_\mu - mS - \frac{\lambda}{2}S^2$ ($\sigma_T = +1$; docstring left out).

```
on_shell = {m: -5 - lam * S_t}  # M = m + lambda S = -5
T_family = source_tensor(K_sym, S_t, m, lam)
rho_family = sp.expand(-T_family[4][4].subs(on_shell))
pressures = [sp.expand(T_family[mu][mu].subs(on_shell)) for mu in (1, 5, 0)]
check(sp.expand(rho_family - ((-5 - lam * S_t) * S_t + lam * S_t ** 2 / 2)) == 0
      and all(sp.expand(p - lam * S_t ** 2 / 2) == 0 for p in pressures),
      "rho = m S + (lambda/2) S^2 and p3 = pt = p0 = (lambda/2) S^2 on shell")
```

On shell the mass is fixed by $M = m + \lambda S = -5$. The check confirms $\rho = mS + \frac{\lambda}{2}S^2$ and $p_3 = p_t = p_0 = \frac{\lambda}{2}S^2$ (Section 12.26).

In [13], the two Einstein conditions.

```
A, Lam, kappa = sp.symbols("A Lambda kappa", real=True)
record = read_json(EQUATIONS)
E1 = record["lovelockTensors"]["E1"]  # the Einstein tensor G = E_(1)
LINEAR = {"ad1": A * H, "ad2": 0, "H": H}  # a4' = A H, a4'' = 0


def from_record(key):
    return sp.sympify(E1[key]["input"].replace("^", "**"), locals=LINEAR)
```

Symbols for the slope, Λ and κ; the Einstein tensor of the record. `LINEAR` is used as the dictionary of names for the parser, so that reading a formula already replaces `ad1` by AH and `ad2` by 0 (docstring left out).

```
G_time, G_space = from_record("x4x4"), from_record("x1x1")
G_extra, G_hidden = from_record("x5x5"), from_record("x0x0")
check(sp.expand(G_space - G_hidden) == 0 and sp.expand(G_extra - G_hidden) == 0,
      "linear member: G^x1_x1 = G^x5_x5 = G^x0_x0 (the record's components)")
```

The four Einstein components for the linear member; the seven non-time ones are equal.

```
S_sym, rho_sym, p_sym = sp.symbols("S rho p", real=True)
time_equation = G_time + Lam + kappa * rho_sym  # = 0
space_equation = G_hidden + Lam - kappa * p_sym  # = 0
condensate = {rho_sym: m * S_sym + lam * S_sym ** 2 / 2, p_sym: lam * S_sym ** 2 / 2}
first = sp.expand(kappa * (m + lam * S_sym) * S_sym + 6 * (A ** 2 + 1) * H ** 2)
second = sp.expand(kappa * m * S_sym + 36 * H ** 2 + 2 * Lam)
reproduces(sp.expand((time_equation - space_equation).subs(condensate) - first) == 0
           and sp.expand((time_equation + space_equation).subs(condensate)
                         - second) == 0,
           "Einstein: kappa M S = -6 (A^2 + 1) H^2 and kappa m S = -(36 H^2 + 2 Lambda)",
           PY, "condensate_einstein_quadratic_U")
```

The time equation and the common equation of the other directions, written as expressions that must vanish; with the condensate's ρ and p, their difference and sum are the two conditions of Section 12.26.

```
M_value = sp.symbols("M_value", real=True)
S_needed = sp.solve(kappa * M_value * S_sym + 6 * (A ** 2 + 1) * H ** 2, S_sym)[0]
m_needed = sp.solve(second, m)[0]
say(f"S = {S_needed},  m = {m_needed},  lambda = (M - m)/S")
```

The first condition solved for S and the second for m; λ then follows as $(M - m)/S$.

In [14], figure 3: which condensates can drive a linear member.

```
m_left = np.linspace(-10.0, -0.01, 1000)  # M/H < 0
m_right = np.linspace(0.01, 10.0, 1000)  # M/H > 0
fig, ax = plt.subplots(figsize=(7.0, 5.2))
# grey: kappa M S > -6, where A^2 = -kappa M S/6 - 1 would be negative
ax.fill_between(m_left, -12.0, np.minimum(-6.0 / m_left, 12.0), color="grey",
                alpha=0.25, linewidth=0.0)
ax.fill_between(m_right, np.maximum(-6.0 / m_right, -12.0), 12.0, color="grey",
                alpha=0.25, linewidth=0.0, label="no real slope $A$")
```

Masses on either side of zero (the hyperbolas are not defined at $M = 0$); `figsize=(7.0, 5.2)` makes the figure 7 inches wide and 5.2 high. The comment line states the rule for the grey region: the first condition of Section 12.26, $\kappa MS = -6(A^2 + 1)$ (units $H = 1$), solved for the slope gives $A^2 = -\kappa MS/6 - 1$, and this is negative, so that no real A exists, exactly when $\kappa MS > -6$. A real slope therefore needs $\kappa MS \le -6$. For $M < 0$ this means $\kappa S \ge -6/M$, so the region below the curve $\kappa S = -6/M$ is shaded grey; for $M > 0$ it means $\kappa S \le -6/M$, so the region above is shaded. `np.minimum` and `np.maximum` keep the curves inside the plotted range ± 12.

```
ax.axvspan(-3.0, 3.0, facecolor="none", edgecolor="black", hatch="//",
           linewidth=0.0, label="$|M| < 3H$: no oscillating condensate")
for slope_squared, style in ((0, ":"), (1, "-"), (4, "--"), (5, "-.")):
    level = 6.0 * (slope_squared + 1)
    label = f"$A^2 = {slope_squared}$"
    ax.plot(m_left, -level / m_left, style, color="tab:blue", label=label)
    ax.plot(m_right, -level / m_right, style, color="tab:blue")
```

The hatched band $|M| < 3$, where no condensate oscillates. Then the hyperbolas $\kappa MS = -6(A^2+1)$ for $A^2 = 0, 1, 4, 5$, on both sides.

```
ax.plot([-5], [12 / 5], "o", color="tab:red", markersize=8,
        label="examples 1 and 3: $M = -5H$, $\\kappa S = 12/5$")
ax.plot([5], [-36 / 5], "s", color="tab:green", markersize=8,
        label="example 2: $M = 5H$, $\\kappa S = -36/5$")
ax.set_xlim(-10, 10)
ax.set_ylim(-12, 12)
ax.set_xlabel("effective mass $M/H$")
ax.set_ylabel("$\\kappa S$ (units $H = 1$)")
ax.set_title("Which condensates can drive a linear member (Einstein gravity)")
ax.legend(fontsize=7.5, loc="lower left")
save_figure(fig, "allowed_sources",
            ...)
```

The three examples are marked. **What the figure shows.** Allowed condensates lie in the upper left and lower right quarters, where S and M have opposite signs and outside the grey; the red circle lies on the hyperbola $A^2 = 1$ and the green square on $A^2 = 5$.

In [15], the three exact solutions.

```
EXAMPLES = [  # name, A, Lambda, M (with H = kappa = 1)
    ("example 1", sp.Integer(1), sp.Integer(0), -5),
    ("example 2", sp.sqrt(5), sp.Integer(0), 5),
```

```
    ("example 3", sp.Integer(1), sp.Integer(-30), -5),
]
UNITS = {H: 1, kappa: 1}
solutions = {}  # name -> the numbers of the solution
```

The three examples of Section 12.26 by their slope, cosmological constant and effective mass; $\sqrt{5}$ is kept exact as `sp.sqrt(5)`.

```
for name, slope, cosmological, M_example in EXAMPLES:
    S_value = sp.nsimplify(S_needed.subs(UNITS).subs({M_value: M_example,
                                                      A: slope}))
    m_value = m_needed.subs(UNITS).subs({S_sym: S_value, Lam: cosmological})
    lam_value = (M_example - m_value) / S_value
```

For each example: the density from the first condition (`sp.nsimplify` writes the result in its simplest exact form), the mass from the second, the self-coupling from $\lambda = (M - m)/S$.

```
    spaces_e, v1_e, v2_e, c_e, w_e, A_e = witness_parts(M_example, 1)
    S_of_t = bilinear((v1_e + t * c_e * v2_e).applyfunc(sp.expand), I16)
    t_value = sp.solve(S_of_t - S_value, t)[0]
    phi0 = (v1_e + t_value * c_e * v2_e).applyfunc(sp.expand)
    phi = phi0 * sp.exp(-sp.I * w_e * x4)  # the condensate Phi(x4)
```

The family at this mass; the member whose density is the required one ($S(t)$ is linear in t, so `sp.solve` gives one t: $15/512$ for examples 1 and 3, $-45/512$ for example 2); the condensate $\Phi(x_4) = e^{-iwx_4}\Phi_0$.

```
    # 1. the field equation, with the full spin-connection sum of section 7:
    residual = (gamma[4] * sp.diff(phi, x4) + gravity_term.subs(UNIT_H) * phi
                - (m_value + lam_value * S_value) * phi).applyfunc(sp.simplify)
    field_ok = residual == sp.zeros(16, 1) and bilinear(phi0, I16) == S_value
```

The 16 components of the field equation $\gamma^\mu D_\mu \Phi - (m + \lambda S)\Phi$: the only derivative is along x_4, where $\gamma^{x_4} = \gamma^{(x_4)}$, plus the spin-connection term of In [5] at $H = 1$. Every component must simplify to zero, at every time, and the density must be the required one. (The comment refers to section 7 of the notebook.)

```
    # 2. the Einstein equations, all 64 components, for +A and for -A:
    K_e = K_sym if M_example == -5 else None
    if K_e is None:  # example 2 has its own condensate matrix A (M = +5)
        K_raw = [[(phi0.H * ((C * gamma_1[mu] * ((A_e if nu == 4 else Z16)
                                                 + Omega_1[nu])
                             - ((A_e.T * C if nu == 4 else Z16) - C * Omega_1[nu])
                             * gamma_1[mu]) / 2) * phi0)[0]
                  for nu in range(8)] for mu in range(8)]
        K_e = [[clean((K_raw[mu][nu] + (g_1[nu] / g_1[mu]) * K_raw[nu][mu]) / 2)
                for nu in range(8)] for mu in range(8)]
    else:
        K_e = [[entry.subs(t, t_value) for entry in row] for row in K_e]
    T_e = source_tensor(K_e, S_value, m_value, lam_value)
```

For $M = -5$ the kinetic tensor of In [11] is reused with the found t. For example 2 ($M = +5$) it is computed afresh exactly as in In [11], with the matrix $\mathcal{A}$ of $M = +5$. Then the energy-momentum tensor (with $\kappa = 1$ it is κT).

```
    einstein_ok = {}
    for sign in (1, -1):
        G_diag = [G_hidden] + [G_space] * 3 + [G_time] + [G_extra] * 3
        values = {A: sign * slope, H: 1}
        einstein_ok[sign] = all(
            sp.simplify((G_diag[mu].subs(values) + cosmological if mu == nu else 0)
                        - T_e[mu][nu]) == 0
            for mu in range(8) for nu in range(8))
```

For the slope $+A$ and for $-A$ all 64 components of $G^{\mu}{}_{\nu} + \Lambda\delta^{\mu}_{\nu} - \kappa T^{\mu}{}_{\nu}$ must simplify to zero (off the diagonal the left-hand side is 0, so T must vanish there). Since T contains no x_4 and no z, this holds at every time and every value of the hidden coordinate.

```
    rho_e, p_e = -T_e[4][4], T_e[1][1]
    solutions[name] = {"A": slope, "Lambda": cosmological, "M": M_example,
                       "S": S_value, "m": m_value, "lambda": lam_value, "t": t_value,
                       "rho": rho_e, "p": p_e, "phi0": phi0, "w": w_e, "T": T_e}
    say(f"{name}: A = {slope}, Lambda = {cosmological}, M = {M_example}, "
        f"S = {S_value}, t = {t_value}, m = {m_value}, lambda = {lam_value}, "
        f"rho = {rho_e}, p = {p_e}")
    check(field_ok, f"{name}: the 16 components of the field equation hold exactly")
    check(einstein_ok[1], f"{name}: all 64 Einstein components hold exactly")
    check(einstein_ok[-1], f"{name}: they hold as well with -A (A enters as A^2)")
```

The energy density and pressure are read off, all numbers are stored for the later cells, printed in one line, and the three checks of the example are made. Out [15] shows the table of Section 12.26 and nine PASS lines.

In [16], the numbers compared with the table.

```
table = {"example 1": (sp.Rational(12, 5), -15, sp.Rational(25, 6), -24, 12),
         "example 2": (sp.Rational(-36, 5), 5, 0, -36, 0),
         "example 3": (sp.Rational(12, 5), 10, sp.Rational(-25, 4), 6, -18)}
for name, (S_v, m_v, lam_v, rho_v, p_v) in table.items():
    sol = solutions[name]
    same = (sol["S"] == S_v and sol["m"] == m_v and sol["lambda"] == lam_v
            and sol["rho"] == rho_v and sol["p"] == p_v)
    formula = 5 + sol["Lambda"] / 3 - sol["lambda"] * sol["S"] ** 2 / 6
    check(same and sp.simplify(formula - sol["A"] ** 2) == 0,
          f"{name}: the numbers of the table and A^2 = 5 + Lambda/3 - lambda S^2/6")
    report(f"{name}: w = p/rho", sol["p"] / sol["rho"])
```

The computed numbers must equal the table of Section 12.26, and A^2 must equal Notebook 12b's formula $5 + \Lambda/3 - \lambda S^2/6$ (Exercise 7 derives it). The equations of state are $-1/2$, 0 and -3.

In [17], figure 4: both sides of the Einstein equations for example 3.

```
sol = solutions["example 3"]
G_diag = [G_hidden] + [G_space] * 3 + [G_time] + [G_extra] * 3
left_side = np.array([[float(G_diag[mu].subs({A: 1, H: 1}) + sol["Lambda"])
                       if mu == nu else 0.0 for nu in range(8)] for mu in range(8)])
right_side = np.array([[float(sol["T"][mu][nu]) for nu in range(8)]
                       for mu in range(8)])
check(np.max(np.abs(left_side - right_side)) == 0.0,
      "example 3: the two 8 by 8 tables are equal entry by entry")
```

The two sides as 8 by 8 arrays of numbers; they agree exactly (the difference is 0.0, not merely small, because both come from exact numbers).

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.4))
scale = np.abs(right_side).max()
panels = ((left_side, "$G^\\mu{}_\\nu + \\Lambda\\delta^\\mu_\\nu$"),
          (right_side, "$\\kappa T^\\mu{}_\\nu$ of the condensate"),
          (left_side - right_side, "difference"))
for ax, (values, title) in zip(axes, panels):
    ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
    for mu in range(8):
        value = values[mu, mu]
        ink = "white" if abs(value) > 0.6 * scale else "black"  # readable on dark
        ax.text(mu, mu, f"{value:.0f}", ha="center", va="center", fontsize=7,
                color=ink)
```

Three heat maps with one common scale (left side, right side, difference), drawn as in Notebook 12a, In [20], with the diagonal values written in.

```
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)
    ax.set_title(title, fontsize=9)
fig.suptitle("Example 3 ($A = 1$, $\\Lambda = -3OH^2$): the Einstein equations hold "
             "component by component")
save_figure(fig, "tensor_equality",
            ...)
```

What the figure shows. The two tables are identical: -6 at x_4 (minus the energy density 6) and -18 at the seven other diagonal places, white everywhere else; the difference is white throughout.

In [18], figure 5: the solution in time.

```
sol = solutions["example 1"]
phi_numbers = np.array([complex(entry) for entry in sol["phi0"]])
biggest = np.argsort(-np.abs(phi_numbers), kind="stable")[:3]  # three components
times = np.linspace(0.0, 3.0, 601)  # x4 in units 1/H
```

Φ_0 of example 1 as 16 complex floating-point numbers. `np.argsort` lists the positions in the order of the sorted values; sorting $-|\Phi_0|$ puts the largest first, `kind="stable"` keeps equal values in their

original order (so that every computer chooses the same three), and `[:3]` keeps the first three positions.

```
density = []
for x_value in times:  # S from the numbers, at every time (it must stay 12/5)
    phi_x = phi_numbers * np.exp(-4j * x_value)
    density.append((phi_x.conj() @ np.array(C.tolist(), dtype=float) @ phi_x).real)
density = np.array(density)
check(np.max(np.abs(density - 12 / 5)) < 1e-12,
      "example 1: S(x4) = 12/5 at every time (numerically)")
```

At 601 times the condensate $\Phi = e^{-4ix_4}\Phi_0$ is formed (`4j` is $4i$ in Python), and $S = \Phi^\dagger C \Phi$ is computed with `@`, the matrix product of numpy, and `.conj()`, the complex conjugate. It stays $12/5$ up to rounding.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
for index in sorted(biggest):
    left.plot(times, (phi_numbers[index] * np.exp(-4j * times)).real,
              label=f"Re $\\Phi_{{{index + 1}}}(x_4)$")
left.plot(times, density, color="black", linewidth=2.0,
          label="$S = \\bar\\Phi\\Phi = 12/5$")
left.set_ylim(-1.35, 4.0)  # room for the legend above the curves
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("value")
left.set_title("example 1: the condensate oscillates, $S$ stays constant")
left.legend(fontsize=8, loc="upper center", ncol=2)
```

The real parts of the three largest components (numbered from 1 in the label) and the constant density; `ncol=2` arranges the legend in two columns.

```
for name, style in (("example 1", "-"), ("example 2", "--")):
    slope = float(solutions[name]["A"])
    right.semilogy(times, np.exp(slope * times), style, color="tab:orange",
                   label=f"3-space $e^{{a_4}}$, {name}")
    right.semilogy(times, np.exp(-slope * times), style, color="tab:blue",
                   label=f"extra times $e^{{-a_4}}$, {name}")
right.set_xlabel("time $x_4$ (units $1/H$)")
right.set_ylabel("scale factor")
right.set_title("3-space inflates, the extra times deflate")
right.legend(fontsize=8)
save_figure(fig, "solution_in_time",
            ...)
```

The scale factors of examples 1 and 2 on a logarithmic axis. **What the figure shows.** The components oscillate with the period $2\pi/4 \approx 1.57$ while S stays flat; the extra-time scale factor falls along straight lines of slope -1 (example 1) and $-\sqrt{5}$ (example 2): exponential deflation, driven by the condensate.

In [19], figure 6: the family in the cosmological constant.

```
lam_axis = sp.symbols("Lambda_axis", real=True)
S_one = sp.Rational(12, 5)
m_family = -(36 + 2 * lam_axis) / S_one
lam_family = (-5 - m_family) / S_one
rho_family_line = m_family * S_one + lam_family * S_one ** 2 / 2
p_family_line = lam_family * S_one ** 2 / 2
check(sp.expand(rho_family_line - (-24 - lam_axis)) == 0
      and sp.expand(p_family_line - (12 + lam_axis)) == 0
      and m_family.subs(lam_axis, 0) == solutions["example 1"]["m"]
      and m_family.subs(lam_axis, -30) == solutions["example 3"]["m"],
      "A = 1 family: kappa rho = -24 - Lambda, kappa p = 12 + Lambda")
```

For $A = 1$, $M = -5$, $S = 12/5$ every Λ gives a solution with $m = -(36+2\Lambda)/S$ and $\lambda = (M-m)/S$; then $\kappa\rho = -24 - \Lambda$ and $\kappa p = 12 + \Lambda$, and examples 1 and 3 are members.

```
axis = np.linspace(-45.0, 15.0, 301)
functions = [sp.lambdify(lam_axis, f, "numpy") for f in
             (m_family, lam_family, rho_family_line, p_family_line)]
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.2))
left.plot(axis, functions[0](axis), label="mass $m/H$")
left.plot(axis, functions[1](axis), "--", label="self-coupling $\\lambda$")
right.plot(axis, functions[2](axis), label="$\\kappa\\rho/H^2$")
right.plot(axis, functions[3](axis), "--", label="$\\kappa p/H^2$")
```

The four formulas as numpy functions of Λ, drawn from -45 to 15: parameters on the left, the source on the right.

```
for ax in (left, right):
    ax.axvspan(-45.0, -24.0, color="tab:green", alpha=0.1)
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xlabel("cosmological constant $\\Lambda/H^2$")
for name, marker in (("example 1", "o"), ("example 3", "s")):
    sol = solutions[name]
    x_value = float(sol["Lambda"])
    left.plot([x_value] * 2, [float(sol["m"]), float(sol["lambda"])], marker,
              color="black")
    right.plot([x_value] * 2, [float(sol["rho"]), float(sol["p"])], marker,
               color="black", label=name)
left.set_title("the condensate's parameters ($A = 1$, $M = -5H$)")
right.set_title("the source it provides")
left.legend(fontsize=8)
right.legend(fontsize=8)
save_figure(fig, "family_in_lambda",
            ...)
```

The green band $\Lambda < -24$ in both panels, the zero lines, and black markers for the two examples. **What the figure shows.** Along the family $\rho + p = -12$ never changes; the energy density becomes positive only in the green band, and there $p < -\rho$: $w < -1$.

In [20], the last check.

```
figure_names = ["condensate_frequency", "three_gamma_bilinears", "allowed_sources",
                "tensor_equality", "solution_in_time", "family_in_lambda"]
paths = [output_file(f"{FIGURE_FOLDER}/12d_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths), "all six figure files exist")
all_checks_passed()
```

As in In [15] of Notebook 12b (Section 12.20): `figure_names` lists the short names of the six figures in the order in which they were drawn, `enumerate(figure_names, 1)` numbers them from 1, which gives the six file names `12d_1_condensate_frequency.png` to `12d_6_family_in_lambda.png`, and `output_file` gives the path where each was written. `path.is_file()` is true when the file exists, and `all` requires this of every one of the six. `all_checks_passed()` prints the last line, ALL 40 CHECKS PASSED (notebook 12d): 1 check in In [2], 2 in In [3], 1 in In [4], 4 in In [5], 3 in In [6], 1 in In [7], 3 in In [8], 2 in In [9], 2 in In [10], 2 in In [11], 1 in In [12], 2 in In [13], 9 in In [15], 3 in In [16], 1 in In [17], 1 in In [18], 1 in In [19] and 1 in In [20].

12.31 What we proved, what we computed, what we assumed

PROVED (exact). The table names, for each statement, the checks of the Revision record that verify it and the notebook cell of this chapter that reproduces them. "Sympy report" and "Wolfram report" are the two reports of the field equations in the folder `Revision/field_equations_a4/reports`; "lead's reports" are `einstein-gauss-bonnet-a4.json` and `emt-divergence-and-spin-connection.json` in `Revision/lead_checks/reports`.

statement	where it is verified
$\sqrt{\lvert\det g\rvert} = \cos z$, independent of a_4: the 7-volume is constant	Wolfram report, `sqrt_abs_det_g_is_cos_z`; Notebook 12a, In [4]
37 nonzero Christoffel symbols; 156 nonzero Riemann components, free of the warp and of e^{a_4}, polynomials in $\cot z$ and $1/\cot z$	Notebook 12a, In [5]; sympy report, `mixed_riemann_free_of_warp_and_a4`, `riemann_entries_laurent`; In [8]
the Einstein tensor is diagonal, isotropic in 3-space and in the extra times, independent of x_0, with the four components of Section 12.6	lead's report, `einstein_off_diagonal_zero`, `einstein_isotropy`, `einstein_x0_independent`; sympy report, `einstein_components`; In [12]
the three Lovelock tensors from the GKD equal the Rust record in all 64 components; $E_{(1)} = G$; each is divergence-free; $P_{(4)} = 0$	sympy report, `P1_equals_gkd_branch_monomials` to `P3_equals_gkd_branch_monomials`, `P1_equals_minus_4_Einstein`, `E1_divergence_free` to `E3_divergence_free`; Wolfram report, `P4_vanishes_pigeonhole`; In [14] to In [19]
$E_{(2)}$ equals the classical Gauss-Bonnet tensor	Notebook 12a, In [18] (every a_4); lead's report, `gauss_bonnet_rho_alpha2`, `gauss_bonnet_p_alpha2` (linear member)
the field equations reduce to the constraint, the evolution equation $a_4'' F(a_4') = \kappa(p_3 - p_t)$, the hidden equation and the conditions $p_3 + p_t = 2p_0$, no x_0 dependence, no mixed components	Wolfram report, `independent_components`; sympy report, `evolution_factorises`, `evolution_F_not_identically_zero`, `algebraic_identity`; In [21] and In [22]

statement	**where it is verified**
the constraint propagates; the source obeys $\rho' = -3a_4'(p_3 - p_t)$	sympy report, `bianchi_x4`, `conservation_components`; Wolfram report, `constraint_propagation_bianchi`; lead's report, `divergence_x4_component`, `divergence_x0_component`; In [23] and In [25]
Einstein gravity: no vacuum for $H > 0$; every source has $\kappa(\rho + p_0) = -6((a_4')^2 + H^2) < 0$	sympy report, `einstein_null_energy_x0`; Wolfram report, `einstein_no_vacuum_solution`; lead's report, `no_vacuum_for_H_positive`; In [26]
the linear member needs $p_3 = p_t = p_0$ and constant ρ, p; A enters only as A^2: the deflation of the extra times is a choice of sign	sympy report, `linear_member_equal_pressures`, `json_linear_member`; Notebook 12a, In [28]; Notebook 12b, In [4]
Einstein gravity: a positive energy density of the linear member needs $\Lambda < -(21 + 3A^2)H^2$ and has $w < -1$	Notebook 12b, In [5] and In [8]
Einstein-Gauss-Bonnet gravity: among the linear members, vacua exist exactly for $0 < \alpha_2 H^2 \leq 1/40$	sympy report, `linear_member_vacuum_factor`, `einstein_gauss_bonnet_vacuum_linear`; Notebook 12b, In [10] and In [11]
with the author's eight real 16 by 16 gammas: $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$; a condensate obeys $\Phi' = \mathcal{A}\Phi$, $\mathcal{A}^2 = -(M^2 - 9H^2)$, with constant S, $\rho = mS + \frac{\lambda}{2}S^2$, $p_3 = p_t = p_0 = \frac{\lambda}{2}S^2$	sympy report, `authorT16_gravity_term`, `authorT16_condensate_S_constant`, `authorT16_condensate_kinetic_diagonal`; lead's report, `gamma_Omega_equals_3H_gamma0`; Notebook 12d, In [5] to In [12]
in Einstein gravity a condensate is a source of the linear member only if $\kappa M S = -6(A^2 + 1)H^2$ and $\kappa m S = -(36H^2 + 2\Lambda)$	sympy and Wolfram reports, `condensate_einstein_quadratic_U`; Notebook 12b, In [13]; Notebook 12d, In [13]
three exact solutions of the coupled equations of Einstein gravity and dirac16complex00, two with the extra times deflating as e^{-Hx_4}: all 16 field-equation components and all 64 Einstein components hold, for $+A$ and $-A$	Notebook 12d, In [15]: a computation of this book, not a Revision record, under the assumptions listed below

COMPUTED (numerical, with measured accuracy; Notebook 12c): RK4 integrations of the evolution equation for prescribed stresses, agreeing with the exact solutions to 9×10^{-11} (pulse, step 0.01) and to better than 10^{-9}, the bound of the check (relaxing stress); measured convergence orders 4.24, 4.12, 4.06, 4.03; the energy density from conservation agrees with the constraint to 2×10^{-11}; the Gauss-Bonnet breakdown is reached at the predicted times $x_4^\star = 2.4682/H$ ($\alpha_2 H^2 = 0.005$) and $0.4498/H$ ($\alpha_2 H^2 = 0.01$) within 0.01; the eigenvalues of $\mathcal{A}$ agree with $\pm iw$ to 10^{-14}, eight of each sign (Notebook 12d, In [7]); the ratio $(\int p_3 + \int p_t)/(2\int p_0)$ of the 70 recorded Kohn-Sham states with a nonzero energy-momentum tensor, recomputed from the record's table `Revision/kohn_sham/results/ground/emt-integrals.csv` and equal to the record, is nowhere closer to the required 1 than 0.414328 (Notebook 12c, In [16]).

ASSUMED: the stresses of Notebook 12c (test inputs; no field of the theory is known to produce

them); Einstein gravity and the sign convention $\sigma_T = +1$ of the energy-momentum tensor in Notebook 12d; the classical commuting field dirac16complex00 (not the quantised dirac16complex) as the source there; the chosen values of Λ, m, λ and of the Lovelock couplings; Lovelock's theorem is quoted from the literature, not proved.

HYPOTHESIS: none is used in this chapter.

OPEN: whether the condensates of Notebook 12d are stable and how they could arise; whether any state of the quantised field dirac16complex is an admissible source (the recorded Kohn-Sham states are not: `Revision/field_equations_a4/reports/ks-source-conditions.json`, every one of its checks PASS, its numbers recomputed in Notebook 12c, In [15] and In [16]); the values of α_2, α_3, Λ and κ in nature. Nothing in this chapter concerns the creation of universes, in pairs or otherwise, or the asymmetry between matter and antimatter: the symmetry $A \to -A$ relates two histories of one metric and is not the pairing of universes of masses $+m$ and $-m$, whose exact statement and limits are the subject of Chapters 18 to 21.

12.32 Exercises

Exercise 1. Compute ${\Gamma^{x_5}}_{x_5x_0}$ and ${\Gamma^{x_0}}_{x_5x_5}$ by hand from the diagonal formula of Section 12.4.

Answer. $g_{55} = -e^{-2a_4}\sin^{1/3} z$, and $\partial_0 g_{55} = 6H\cdot(-e^{-2a_4})\frac{1}{3}\sin^{-2/3} z\cos z = 2H\cot z\ g_{55}$. First symbol ($a = c = x_5$, $b = x_0$, the term with δ_{ac}): ${\Gamma^{x_5}}_{x_5x_0} = \partial_0 g_{55}/(2g_{55}) = H\cot z$. Second ($b = c = x_5$, $a = x_0$, the term with δ_{bc}): ${\Gamma^{x_0}}_{x_5x_5} = -\partial_0 g_{55}/(2g_{00}) = -2H\cot z\, g_{55}/(2\cot^2 z) = -H\tan z\ g_{55} = He^{-2a_4}\sin^{1/3} z\,\tan z = He^{-2a_4}\sin^{4/3} z/\cos z$. Both agree with Out [6] of Notebook 12a.

Exercise 2. Compute the pair curvature $K_{x_5x_6}$ of two extra times by hand.

Answer. With $a = m = x_5$, $b = n = x_6$ the derivative terms vanish (${\Gamma^{x_5}}_{x_6x_6} = {\Gamma^{x_5}}_{x_6x_5} = 0$, because g_{55} does not depend on x_6), and so does the last sum. The remaining sum has $e = x_4$ and $e = x_0$: ${R^{x_5}}_{x_6x_5x_6} = {\Gamma^{x_5}}_{x_5x_4}{\Gamma^{x_4}}_{x_6x_6} + {\Gamma^{x_5}}_{x_5x_0}{\Gamma^{x_0}}_{x_6x_6} = (-a_4')(-a_4' g_{66}) + H\cot z\,(-H\tan z\, g_{66}) = ((a_4')^2 - H^2)g_{66}$ (Section 12.4: ${\Gamma^{x_4}}_{x_6x_6} = -a_4' g_{66}$ and ${\Gamma^{x_0}}_{x_6x_6} = -H\tan z\, g_{66}$). Raising with $g^{66} = 1/g_{66}$: $K_{x_5x_6} = (a_4')^2 - H^2$, the same as for two 3-space directions (the table of Section 12.5).

Exercise 3. Use the rule ${G^h}_h = -\sum_{\text{pairs not containing } h} K$ to compute all diagonal components of the Einstein tensor for the canonical history $a_4 = Hx_4$, and from them the required $\kappa\rho$, κp and w for $\Lambda = 0$.

Answer. With $a_4' = H$, $a_4'' = 0$ the pair curvatures are: two 3-space directions or two extra times 0; 3-space with extra time $-2H^2$; 3-space or extra time with the time H^2; with x_0 $-H^2$; time with x_0 0. Pairs without x_4: $9\cdot(-2H^2) + 6\cdot(-H^2) = -24H^2$, so ${G^{x_4}}_{x_4} = 24H^2$. Pairs without x_0: $9\cdot(-2H^2) + 6\cdot H^2 = -12H^2$, so ${G^{x_0}}_{x_0} = 12H^2$. Pairs without x_1: six mixed pairs ($-12H^2$), five time pairs ($+5H^2$), five with x_0 ($-5H^2$): $-12H^2$, so ${G^{x_1}}_{x_1} = 12H^2$, and the same for an extra time. Then $\kappa\rho = -{G^{x_4}}_{x_4} = -24H^2$, $\kappa p = 12H^2$, $w = 12/(-24) = -1/2$, as in Notebook 12b, In [5]. Check with the formulas: $3H^2 + 21H^2 = 24H^2$ and $15H^2 - 3H^2 = 12H^2$.

Exercise 4. In Einstein gravity, for the canonical history ($A = 1$), which cosmological constant makes the required energy density zero? What are then the pressure and $\rho + p$? Is this a vacuum?

Answer. $\kappa\rho = -24H^2 - \Lambda = 0$ gives $\Lambda = -24H^2$. Then $\kappa p = 12H^2 + \Lambda = -12H^2$ and $\kappa(\rho + p) = -12H^2$, as the formula $-6(1 + A^2)H^2$ says. The pressure is not zero, so this is not a vacuum: a

source with zero energy density and negative pressure is still needed.

Exercise 5. In Einstein-Gauss-Bonnet gravity, find the coupling $\alpha_2 H^2$ for which the canonical history $A = 1$ is a vacuum, and the cosmological constant it needs. Check that the pressure vanishes too.

Answer. $V = 1 - 8\alpha_2 H^2(A^2+5) = 1 - 48\alpha_2 H^2 = 0$ gives $\alpha_2 H^2 = 1/48$ (the point checked in Notebook 12b, In [12]). With $\alpha_1 = 1$, $\alpha_3 = 0$: $\kappa\rho = -(21+3)H^2 + \alpha_2 H^4(36+120+420) - \Lambda = -24H^2 + 576H^2/48 - \Lambda = -12H^2 - \Lambda$, which is zero for $\Lambda = -12H^2$. The pressure: $\kappa p = (15-3)H^2 + 12\alpha_2 H^4(1+14-15) + \Lambda = 12H^2 + 0 - 12H^2 = 0$. (The formula of Notebook 12b, $\Lambda = 720H^4\alpha_2 - 36H^2 + 3/(16\alpha_2)$, gives $15H^2 - 36H^2 + 9H^2 = -12H^2$ as well.)

Exercise 6. Show, for Einstein gravity, that if the constraint holds at every time and the source obeys the conservation law $\rho' = -3a_4'(p_3 - p_t)$, then the evolution equation holds wherever $a_4' \neq 0$.

Answer. Differentiate the constraint $-\kappa\rho = 3(a_4')^2 + 21H^2 + \Lambda$: $-\kappa\rho' = 6a_4'a_4''$. Insert the conservation law on the left: $3a_4'\kappa(p_3 - p_t) = 6a_4'a_4''$. Where $a_4' \neq 0$ divide by $3a_4'$: $\kappa(p_3 - p_t) = 2a_4''$, the evolution equation. (Where $a_4' = 0$ the division is not allowed, and the evolution equation is an independent statement.)

Exercise 7. A condensate with $U = \frac{\lambda}{2}S^2$ is the source of the linear member in Einstein gravity. Derive Notebook 12b's formula $A^2 = 5 + \Lambda/3 - \lambda S^2/6$ (units $H = \kappa = 1$) and check it on example 3 of Section 12.26.

Answer. The two conditions read $MS = -6(A^2+1)$ and $mS = -(36 + 2\Lambda)$. Since $M = m + \lambda S$: $MS = mS + \lambda S^2 = -(36+2\Lambda) + \lambda S^2$. So $-6(A^2+1) = -36 - 2\Lambda + \lambda S^2$, hence $A^2 + 1 = 6 + \Lambda/3 - \lambda S^2/6$ and $A^2 = 5 + \Lambda/3 - \lambda S^2/6$. Example 3: $\Lambda = -30$, $\lambda = -25/4$, $S = 12/5$: $5 - 10 + \frac{25}{4}\cdot\frac{144}{25}\cdot\frac{1}{6} = 5 - 10 + 6 = 1 = A^2$.

Exercise 8. In Einstein-Gauss-Bonnet gravity with $\alpha_2 H^2 = 0.01$, a constant stress $\kappa\Delta = 0.5H^2$ acts from $a_4'(0) = H$. Compute the critical rate and the breakdown time (units $H = 1$).

Answer. $F(v) = 2 - 0.8 - 0.48v^2 = 1.2 - 0.48v^2$ vanishes at $v_c = \sqrt{1.2/0.48} = \sqrt{2.5} \approx 1.5811$. $G(v) = 1.2v - 0.16v^3$: $G(v_c) = 1.2\cdot 1.58114 - 0.16\cdot 3.95285 \approx 1.89737 - 0.63246 = 1.26491$ (using $v_c^3 = 2.5\,v_c$), and $G(1) = 1.2 - 0.16 = 1.04$. So $x_4^\star = (1.26491 - 1.04)/0.5 \approx 0.4498$, the value of Notebook 12c, In [13].

Exercise 9. For the relaxing stress of Section 12.21, by how much does the extra-time scale factor e^{-a_4} end up smaller than on the history $a_4 = x_4$, for $\eta = 0.5$?

Answer. $a_4 - x_4 = (1 - e^{-\eta x_4})/\eta \to 1/\eta = 2$ for large x_4. So $e^{-a_4} = e^{-x_4}e^{-(a_4 - x_4)} \to e^{-x_4}e^{-2}$: the extra times end up smaller by the factor $e^{-2} \approx 0.135$ than on the canonical history, because they deflated faster at first.

Exercise 10. Show that $\mathcal{A} = -\gamma^{(x_4)}(M - 3H\gamma^{(x_0)})$ has the eigenvalues $\pm i\sqrt{M^2 - 9H^2}$ when $M^2 > 9H^2$ and $\pm\sqrt{9H^2 - M^2}$ when $M^2 < 9H^2$. What is the period of the condensate for $M = -5H$?

Answer. If $\mathcal{A}v = \mu v$ for a nonzero v, then $\mathcal{A}^2 v = \mu^2 v$; with $\mathcal{A}^2 = -(M^2 - 9H^2)$ (Section 12.26) this gives $\mu^2 = 9H^2 - M^2$. For $M^2 > 9H^2$ the right-hand side is negative and μ is one of $\pm i\sqrt{M^2 - 9H^2}$; for $M^2 < 9H^2$ it is positive and μ is one of $\pm\sqrt{9H^2 - M^2}$, real: solutions grow or decay. This shows only that no other value occurs; that both signs occur, each 8 times, follows from the **trace** (the sum of the diagonal entries), which equals the sum of the 16 eigenvalues, each counted as often as it occurs.

Write $\mathcal{A} = -M\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}$. Then $\operatorname{tr}\gamma^{(x_4)} = \operatorname{tr}\big(\gamma^{(x_0)}\gamma^{(x_0)}\gamma^{(x_4)}\big) = \operatorname{tr}\big(\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)}\big) = -\operatorname{tr}\big(\gamma^{(x_0)}\gamma^{(x_0)}\gamma^{(x_4)}\big) = -\operatorname{tr}\gamma^{(x_4)}$, so $\operatorname{tr}\gamma^{(x_4)} = 0$ (first $(\gamma^{(x_0)})^2 = 1$; then $\operatorname{tr}(XY) = \operatorname{tr}(YX)$ with $X = \gamma^{(x_0)}$, which moves the first factor to the end; then the Clifford relation $\gamma^{(x_4)}\gamma^{(x_0)} = -\gamma^{(x_0)}\gamma^{(x_4)}$; then $(\gamma^{(x_0)})^2 = 1$ again). In the same way $\operatorname{tr}\big(\gamma^{(x_4)}\gamma^{(x_0)}\big) = \operatorname{tr}\big(\gamma^{(x_0)}\gamma^{(x_4)}\big) = -\operatorname{tr}\big(\gamma^{(x_4)}\gamma^{(x_0)}\big) = 0$. So $\operatorname{tr}\mathcal{A} = 0$. If n_+ eigenvalues equal $+\mu_0$ and n_- equal $-\mu_0$, with $\mu_0 \neq 0$ (here $M^2 \neq 9H^2$), then $n_+ + n_- = 16$ and $\operatorname{tr}\mathcal{A} = (n_+ - n_-)\mu_0 = 0$, hence $n_+ = n_- = 8$. For $M^2 > 9H^2$ this also follows from $\mathcal{A}$ being real (Section 12.26). For $M = -5H$: $w = \sqrt{25-9}\,H = 4H$, and e^{-iwx_4} repeats after $x_4 = 2\pi/w = \pi/(2H) \approx 1.57/H$.

13 Many-body quantum mechanics and DFT from zero

This chapter builds, from zero, the tool with which Chapters 14 to 16 compute the states of many quanta of the field dirac16complex in the author's primordial field: **density functional theory**, DFT for short, in the form of Kohn and Sham. Every idea is first developed for ordinary particles that move slowly (non-relativistic particles), because there every step can be seen and checked by hand. The chapter has five worked examples, Notebooks 13a to 13e, and every number in its text is computed by one of them.

13.1 What this chapter is for

The problem. Chapter 14 asks for the lowest-energy state (the **ground state**) and the first excited state of a fixed number N of dirac16complex quanta that interact with each other inside the author's primordial gravitational field. A quantum state of one particle in ordinary three-dimensional space is a complex-valued function of three variables (its **wave function**, defined in Section 13.2). If we store such a function on a grid with 10 points along each coordinate, we need 10^3 complex numbers. The state of two particles is a function of 6 variables, 10^6 numbers; the state of ten particles a function of 30 variables, 10^{30} numbers, more than every computer on Earth can hold together. Every added particle multiplies the work by 10^3. This growth is called the **exponential wall** of the many-body problem.

The way around it. DFT works with the **density** n of the particles instead of their wave function: the expected number of particles per unit volume at each point of space. The density is a function of three variables, whatever N is. Sections 13.8 to 13.10 prove that the density alone fixes the ground-state energy, and they show how Kohn and Sham turned this fact into equations that a computer can solve: one-particle equations in an effective potential, solved again and again until the potential and the density agree (**self-consistency**). The price is one quantity, the **exchange-correlation energy**, that is not known exactly and must be approximated. We will see exactly where the approximation enters and what it is.

Plan. Sections 13.2 to 13.7 are the quantum mechanics of many identical fermions: one particle, many particles, Slater determinants, creation and annihilation operators with Wick's theorem, the energy of a determinant with its direct and exchange parts, the Hartree-Fock equations, and an exactly solvable model of two electrons on two sites. Sections 13.8 to 13.10 are density functional theory proper: the Hohenberg-Kohn theorem, functional derivatives and the Kohn-Sham equations. Notebook 13c (Sections 13.11 to 13.14) checks all of this with numbers. Sections 13.15 and 13.16 compute the exchange energy of a uniform gas, show that for a contact interaction it is exactly

local, and carry the result over to the 16-component field dirac16complex, reproducing the exchange formulas of the Revision record (Notebook 13b, Sections 13.17 to 13.20). Section 13.21 explains how the self-consistency loop is solved and why it can fail (Notebook 13d, Sections 13.22 to 13.25). Sections 13.26 and 13.27 extend everything to a finite temperature (Mermin's theorem) and to excited states (the Kohn-Sham gap, Janak's theorem and the Delta-SCF method), checked in Notebook 13e (Sections 13.28 to 13.31). Section 13.32 puts all the pieces together in one complete Kohn-Sham calculation, eight fermions in a trap, solved in Notebook 13a (Sections 13.33 to 13.36). Section 13.37 says what of this chapter is used for dirac16complex, Section 13.38 lists what was proved, computed and assumed, and Section 13.39 has exercises with complete answers.

The order of the notebooks. The notebooks are lettered in the order in which they were planned, and the chapter presents them in the order in which their ideas are needed: 13c, 13b, 13d, 13e, and last 13a, the complete calculation, which uses every idea of the others.

Units. In this chapter Planck's constant divided by 2π and the particle mass are set to 1 ($\hbar = m = 1$), so the kinetic-energy operator of one particle on a line is $-\frac{1}{2}\,d^2/dx^2$. Each notebook states its other units (the hopping t of the two-site model, the trap frequency of Notebook 13a). A temperature is measured as an energy (Boltzmann's constant is 1).

Symbols with a local meaning. Several letters mean something else in this chapter than in the rest of the book. (i) x is the position of a particle, or a point of a grid, or (in Section 13.21) the excess of charge on one site; it is never one of the author's spacetime coordinates $x_0, \ldots, x_7$, which appear only in Sections 13.16 and 13.37 and are named there. (ii) Ψ is the wave function of N particles, except in Sections 13.16 and 13.37, where it is the dirac16complex field, as those sections say. (iii) ρ is a density matrix (Section 13.4) or a density operator (Section 13.26), not an energy density. (iv) t is the hopping between two sites, T the temperature and T_s a kinetic energy. (v) S is a one-body quantity $\sum V_{pq} a_p^\dagger a_q$ in Section 13.4, the symmetric two-site state in Section 13.7, the scalar density of dirac16complex in Sections 13.16 to 13.20 and 13.37, and the entropy in Sections 13.26 to 13.31. (vi) U is the repulsion of two electrons on one site in the two-site model, U_{kp} the entries of a unitary matrix, and $U(S)$ the interaction potential of dirac16complex in Section 13.16. (vii) g counts the internal labels of a particle and g_c is the strength of a contact interaction; neither is the metric. (viii) H and $\hat{H}$ are Hamiltonians (energy operators); the author's constant H of the metric appears only in Section 13.37.

13.2 One quantum particle: states, operators and the variational principle

States on a grid. The state of one particle on a line is a complex-valued function $\phi(x)$, its **wave function**. Its squared modulus $|\phi(x)|^2 = \phi^*(x)\,\phi(x)$ (the star means complex conjugate) is the probability per unit length of finding the particle at x, so a physical state is **normalised**: $\int |\phi|^2\,dx = 1$. On a computer the line is replaced by M equally spaced points x_k, $k = 1, \ldots, M$, with the spacing h, and a state becomes a column of M numbers $(\phi_1, \ldots, \phi_M)$; integrals become sums, $\int f\,dx \approx \sum_k f_k\,h$. Two states are compared by their **inner product**

$$\langle \chi | \phi \rangle = \int \chi^*(x)\,\phi(x)\,dx \approx \sum_k \chi_k^*\,\phi_k\,h.$$

It is linear in ϕ, it gives the complex conjugate when the two states are exchanged, $\langle \phi|\chi\rangle = \langle \chi|\phi\rangle^*$, and $\langle \phi|\phi\rangle \geq 0$. Two states with $\langle \chi|\phi\rangle = 0$ are **orthogonal**; a set of normalised, pairwise orthogonal

states is **orthonormal**.

Operators. A measurable quantity is represented by a linear **operator** A: a rule that turns a state into a state, with $A(c_1\phi + c_2\chi) = c_1 A\phi + c_2 A\chi$ for numbers c_1, c_2. On a grid an operator is an $M \times M$ matrix. It is **Hermitian** when $\langle\chi|A\phi\rangle = \langle A\chi|\phi\rangle$ for all states; for a matrix this means that it equals its conjugate transpose, $A^\dagger = A$ (for a real matrix: it is symmetric). The **expectation value** $\langle A\rangle = \langle\phi|A\phi\rangle$ in a normalised state is the average of many measurements. The energy operator of one particle in an external potential $v(x)$ is the **Hamiltonian**

$$h = -\frac{1}{2}\,\frac{d^2}{dx^2} + v(x).$$

On the grid the second derivative is replaced by the **difference formula** $(\phi_{k+1} - 2\phi_k + \phi_{k-1})/h^2$, which follows from the Taylor expansions $\phi_{k\pm1} = \phi_k \pm h\,\phi_k' + \frac{1}{2}h^2\phi_k'' \pm \frac{1}{6}h^3\phi_k''' + O(h^4)$: adding them, the odd powers cancel, and the error is of order h^2. So $-\frac{1}{2}\,d^2/dx^2$ becomes the symmetric matrix with $1/h^2$ on the diagonal and $-1/(2h^2)$ just above and below it. A state with $h\phi = \epsilon\,\phi$ is an **eigenstate** (a stationary state) with the energy ϵ, an **eigenvalue** of h.

Two facts about Hermitian operators, line by line. Let $A\phi = a\,\phi$ with $\phi \neq 0$. Then

$$a\,\langle\phi|\phi\rangle = \langle\phi|A\phi\rangle = \langle A\phi|\phi\rangle = a^*\,\langle\phi|\phi\rangle.$$

The first equality puts $A\phi = a\phi$ into the second slot (linearity); the second is the definition of Hermitian; the third puts $A\phi = a\phi$ into the first slot, where a number comes out complex conjugated. Since $\langle\phi|\phi\rangle > 0$, we may divide by it: $a = a^*$, **the eigenvalues are real**. Next let $A\phi = a\phi$ and $A\chi = b\chi$ with $a \neq b$ (both real by the first fact). Then

$$b\,\langle\phi|\chi\rangle = \langle\phi|A\chi\rangle = \langle A\phi|\chi\rangle = a\,\langle\phi|\chi\rangle,$$

by the same three rules, so $(b-a)\langle\phi|\chi\rangle = 0$, and because $b - a \neq 0$, $\langle\phi|\chi\rangle = 0$: **eigenstates with different eigenvalues are orthogonal**. For a Hermitian matrix one can always choose an orthonormal basis of eigenvectors (the spectral theorem of linear algebra); numpy's function `eigh` returns exactly such a basis, with the eigenvalues in increasing order.

Worked example: two sites. Two grid points ("sites" L and R) and the hopping matrix

$$h = \begin{pmatrix} 0 & -t \\ -t & 0 \end{pmatrix}, \qquad t > 0.$$

The eigenvalues solve $\det(h - \epsilon) = \epsilon^2 - t^2 = 0$, so $\epsilon = \pm t$. For $\epsilon = -t$ the first row of $(h + t)\phi = 0$ reads $t\phi_L - t\phi_R = 0$, so $\phi_L = \phi_R$, and normalised $\phi = (1, 1)/\sqrt{2}$ (the **bonding** orbital, lower energy); for $\epsilon = +t$ we get $(1, -1)/\sqrt{2}$ (antibonding). They are orthogonal: $(1 \cdot 1 + 1 \cdot (-1))/2 = 0$.

The variational principle, line by line. Let $\phi_0, \phi_1, \ldots$ be an orthonormal basis of eigenvectors of a Hermitian H with eigenvalues $E_0 \le E_1 \le \ldots$, and let $\phi = \sum_n c_n\phi_n$ be any normalised state. Then

$$\langle\phi|H\phi\rangle = \sum_{n,m} c_m^* c_n\,\langle\phi_m|H\phi_n\rangle = \sum_{n,m} c_m^* c_n E_n\,\delta_{mn} = \sum_n |c_n|^2 E_n.$$

The first step expands both slots (linearity in the second slot, conjugation in the first); the second uses $H\phi_n = E_n\phi_n$ and orthonormality, $\langle\phi_m|\phi_n\rangle = \delta_{mn}$ (1 if $m = n$, else 0); the third keeps only $m = n$. Since every $E_n \geq E_0$ and $\sum_n |c_n|^2 = \langle\phi|\phi\rangle = 1$,

$$\sum_n |c_n|^2 E_n \;\geq\; E_0 \sum_n |c_n|^2 = E_0,$$

with equality exactly when only the c_n with $E_n = E_0$ are nonzero. So **the ground-state energy is the smallest expectation value of the energy over all normalised states**, and the states that reach it are the ground states. Every method of this chapter is an application of this inequality.

Internal labels. Real particles carry a label besides their position: an electron has a spin with two values, up and down. A particle with g labels has a wave function $\phi(x, \sigma)$ with $\sigma = 1, \ldots, g$. The quanta of dirac16complex in the sector of Chapter 14 have $g = 8$ states for each momentum (Section 13.16).

13.3 Many identical fermions and Slater determinants

The many-particle wave function. N particles are described by one complex function of all their positions and labels, $\Psi(x_1\sigma_1, \ldots, x_N\sigma_N)$. Its squared modulus is the probability density of finding particle 1 at x_1 with label σ_1, particle 2 at x_2 with σ_2, and so on. (In this paragraph the letters $x_1, \ldots, x_N$ are the positions of particles $1, \ldots, N$, not spacetime coordinates.)

Identical particles and the Pauli principle. No experiment can tell two electrons apart, so $|\Psi|^2$ must not change when two particles are exchanged. Nature realises this in two ways: for **bosons** Ψ itself is unchanged, for **fermions** it changes sign:

$$\Psi(\ldots, x_i\sigma_i, \ldots, x_j\sigma_j, \ldots) = -\,\Psi(\ldots, x_j\sigma_j, \ldots, x_i\sigma_i, \ldots) \qquad \text{(fermions)}.$$

Electrons, protons and neutrons are fermions, and so are the quanta of dirac16complex, whose components are anticommuting (Grassmann) numbers (Chapter 7). That a field describes fermions is an ASSUMED property of the field: it is built into dirac16complex by the choice of Grassmann components. Put $x_i\sigma_i = x_j\sigma_j$ in the rule above: then $\Psi = -\Psi$ at such points, so $\Psi = 0$ there. **Two identical fermions are never found at the same place with the same label** (the Pauli principle).

The many-body Hamiltonian. For N particles in an external potential v with a pair interaction w,

$$\hat{H} = \sum_{i=1}^{N} \Big[-\frac{1}{2}\frac{\partial^2}{\partial x_i^2} + v(x_i) \Big] + \sum_{i<j} w(x_i, x_j) = \hat{T} + \hat{V} + \hat{W}.$$

Only $\hat{V}$ distinguishes one system from another: all atoms with N electrons share the same kinetic energy $\hat{T}$ and interaction $\hat{W}$. This observation is the seed of DFT.

The density. The density is the expected number of particles per unit length at x,

$$n(x) = N \sum_{\sigma} \int |\Psi(x\sigma, x_2\sigma_2, \ldots, x_N\sigma_N)|^2 \, dx_2 \cdots dx_N$$

(the sum over σ also includes the sums over $\sigma_2, \ldots, \sigma_N$). The factor N appears because any of the N identical particles may be the one at x, and antisymmetry makes all N choices give the same integral; so $\int n \, dx = N$. The external energy needs only the density: $\langle \Psi | \hat{V} \Psi \rangle = \int v(x) \, n(x) \, dx$, because each of the N terms $v(x_i)$ gives the same integral.

Slater determinants for two particles. Take two orthonormal one-particle states ϕ_a, ϕ_b (**orbitals**; we write r for the position and label of one particle together). The product $\phi_a(r_1)\phi_b(r_2)$ is not antisymmetric, but

$$\Phi(r_1, r_2) = \frac{1}{\sqrt{2}} \big[\phi_a(r_1)\phi_b(r_2) - \phi_b(r_1)\phi_a(r_2) \big] = \frac{1}{\sqrt{2}} \det \begin{pmatrix} \phi_a(r_1) & \phi_b(r_1) \\ \phi_a(r_2) & \phi_b(r_2) \end{pmatrix}$$

is: exchanging r_1 and r_2 exchanges the two rows of the determinant, which changes its sign. If $\phi_a = \phi_b$ the two columns are equal and $\Phi = 0$: two fermions cannot occupy the same orbital. The norm follows line by line:

$$\begin{aligned} \iint |\Phi|^2 &= \tfrac{1}{2} \iint \big[|\phi_a(r_1)|^2 |\phi_b(r_2)|^2 + |\phi_b(r_1)|^2 |\phi_a(r_2)|^2 \big] - \tfrac{1}{2} \iint 2 \operatorname{Re}\big[\phi_a^*(r_1)\phi_b(r_1) \, \phi_b^*(r_2)\phi_a(r_2) \big] \\ &= \tfrac{1}{2} \, (1 \cdot 1 + 1 \cdot 1) - \operatorname{Re}\big[\langle \phi_a | \phi_b \rangle \langle \phi_b | \phi_a \rangle \big] = 1 - 0 = 1. \end{aligned}$$

The first line multiplies out $|u - v|^2 = |u|^2 + |v|^2 - 2 \operatorname{Re}(u^* v)$; the second integrates each factor separately (the integral of a product of a function of r_1 and a function of r_2 is the product of the integrals), uses the normalisation of ϕ_a and ϕ_b, and uses their orthogonality, $\langle \phi_a | \phi_b \rangle = 0$.

N particles, line by line. For N orthonormal orbitals $\phi_1, \ldots, \phi_N$ the **Slater determinant** is $\Phi = \det[\phi_a(r_i)]/\sqrt{N!}$, the determinant of the $N \times N$ table whose row i holds the values of all orbitals at the position (and label) r_i of particle i: the entry in row i and column a is $\phi_a(r_i)$. The Leibniz formula of Section 1.20 writes it as a sum over the $N!$ permutations σ of $1, \ldots, N$, each term taking from row i the entry in column $\sigma(i)$:

$$\Phi(r_1, \ldots, r_N) = \frac{1}{\sqrt{N!}} \sum_{\sigma} \operatorname{sign}(\sigma) \, \phi_{\sigma(1)}(r_1) \, \phi_{\sigma(2)}(r_2) \cdots \phi_{\sigma(N)}(r_N).$$

For $N = 2$ the two permutations give back the formula above. Three properties follow from the rules of determinants of Section 1.20. (i) **Antisymmetry**: exchanging the particles i and j exchanges the rows i and j of the table, which changes the sign of the determinant (rule 3). (ii) **Pauli**: if two orbitals are equal, $\phi_a = \phi_b$ with $a \neq b$, the columns a and b of the table are equal; the transposed table then has two equal rows, so its determinant is 0 (rule 4), and the determinant of the table is the same number (rule 2): $\Phi = 0$. (iii) **The norm.** Here and below $\int dr$ means the integral over the position together with the sum over the label, and $\int d^N r$ the same for all N particles. Writing $|\Phi|^2 = \Phi^* \Phi$ with one Leibniz sum over σ for Φ^* and another over τ for Φ:

$$\int |\Phi|^2 \, d^N r = \frac{1}{N!} \sum_\sigma \sum_\tau \operatorname{sign}(\sigma) \operatorname{sign}(\tau) \int \prod_{i=1}^{N} \phi^*_{\sigma(i)}(r_i) \, \phi_{\tau(i)}(r_i) \, d^N r$$
$$= \frac{1}{N!} \sum_\sigma \sum_\tau \operatorname{sign}(\sigma) \operatorname{sign}(\tau) \prod_{i=1}^{N} \langle \phi_{\sigma(i)} | \phi_{\tau(i)} \rangle$$
$$= \frac{1}{N!} \sum_\sigma \sum_\tau \operatorname{sign}(\sigma) \operatorname{sign}(\tau) \prod_{i=1}^{N} \delta_{\sigma(i)\tau(i)}$$
$$= \frac{1}{N!} \sum_\sigma \operatorname{sign}(\sigma)^2 = \frac{N!}{N!} = 1.$$

The first line multiplies the two sums term by term (the complex conjugate of a product is the product of the conjugates, and the signs are real) and collects the two factors that belong to particle i; the second integrates each particle's variable separately (the integral of a product of functions of different variables is the product of the integrals), and each factor is an inner product of two orbitals; the third uses orthonormality; the fourth notes that the product of the Kronecker deltas is 1 exactly when $\tau(i) = \sigma(i)$ for every i, that is $\tau = \sigma$, and 0 otherwise, so only the $N!$ terms with $\tau = \sigma$ survive, each with $\operatorname{sign}(\sigma)^2 = 1$. (iv) **The density.** By the definition of the density, with the variable of particle 1 set to r and the other $N-1$ variables integrated, the same steps give

$$n(r) = N \int |\Phi(r, r_2, \ldots, r_N)|^2 \, dr_2 \cdots dr_N = \frac{N}{N!} \sum_\sigma \sum_\tau \operatorname{sign}(\sigma) \operatorname{sign}(\tau) \, \phi^*_{\sigma(1)}(r) \, \phi_{\tau(1)}(r) \prod_{i=2}^{N} \delta_{\sigma(i)\tau(i)}$$
$$= \frac{1}{(N-1)!} \sum_\sigma |\phi_{\sigma(1)}(r)|^2 = \frac{1}{(N-1)!} \sum_{a=1}^{N} (N-1)! \, |\phi_a(r)|^2 = \sum_{a=1}^{N} |\phi_a(r)|^2.$$

The first line inserts the two Leibniz sums and integrates over $r_2, \ldots, r_N$ as in the norm; the factor of particle 1 is not integrated. In the second line, the deltas for $i = 2, \ldots, N$ force $\tau(i) = \sigma(i)$ for these places, and then also $\tau(1) = \sigma(1)$, because a permutation uses every number once and only one number is left for place 1; so again only $\tau = \sigma$ survives, with $\operatorname{sign}(\sigma)^2 = 1$, and $N/N! = 1/(N-1)!$. Next, the permutations with $\sigma(1) = a$ are the $(N-1)!$ orderings of the other $N-1$ numbers on the places $2, \ldots, N$, so each orbital a appears $(N-1)!$ times; the factorials cancel. So **the density of a Slater determinant is the sum of the orbital densities** (here $n(r)$ is the density at the position and label r; summed over the label it is the density $n(x)$ defined above, so $\int n \, dr = N$ by the norm of each orbital). For two particles the same computation reads $n(r) = 2 \int |\Phi(r, r_2)|^2 \, dr_2 = |\phi_a(r)|^2 \cdot 1 + |\phi_b(r)|^2 \cdot 1 - 2 \operatorname{Re}[\phi^*_a(r) \phi_b(r) \langle \phi_b | \phi_a \rangle] = |\phi_a(r)|^2 + |\phi_b(r)|^2$ (multiply out as above, integrate over r_2 only, use orthonormality). A Slater determinant is the state of N **independent** fermions; a general antisymmetric state is a sum of many determinants. Kohn-Sham theory (Section 13.10) rests on the fact that a single determinant can nevertheless carry the exact density.

Worked example (Notebook 13c, In [2]). Let space be three points $r = 0, 1, 2$ (no label), with the orthonormal orbitals $\phi_0 = (1, 0, 0)$ and $\phi_1 = (0, 1, 1)/\sqrt{2}$. Then $\Phi(0, 1) = [\phi_0(0)\phi_1(1) - \phi_1(0)\phi_0(1)]/\sqrt{2} = [1 \cdot \frac{1}{\sqrt{2}} - 0]/\sqrt{2} = \frac{1}{2}$, and $\Phi(1, 2) = [0 \cdot \frac{1}{\sqrt{2}} - \frac{1}{\sqrt{2}} \cdot 0]/\sqrt{2} = 0$. The nine values are $\pm\frac{1}{2}$ (four of them) and 0 (five of them, among them the three diagonal values), so $\sum |\Phi|^2 = 4 \cdot \frac{1}{4} = 1$, and the density is $n(r) = 2 \sum_{r_2} |\Phi(r, r_2)|^2 = (1, \frac{1}{2}, \frac{1}{2}) = \phi_0^2 + \phi_1^2$. Figure 13c.1 draws this table and a determinant of two fermions in a box.

13.4 Creation and annihilation operators and Wick's theorem

Determinants are long to write. **Second quantisation** labels a determinant only by which orbitals it uses. It is the same formalism that Chapter 10 uses for the quantised dirac16complex field, here in its simplest form.

Occupation numbers. Fix an orthonormal basis of M orbitals $\phi_0, \dots, \phi_{M-1}$. A determinant built from some of them is fixed, up to its sign, by the **occupation numbers** $n_p \in \{0, 1\}$, $n_p = 1$ when ϕ_p is used. We write it $|n_0 n_1 \cdots n_{M-1}\rangle$ and fix the sign by listing the occupied orbitals in increasing order of p as the columns of the determinant. The state without any particle is the **vacuum** $|0\rangle$. With $M = 4$ orbitals there are $2^4 = 16$ such states; numbering them $s = 0, \dots, 15$, the occupation n_p is the binary digit (bit) p of s, so for example $s = 5 = 1 + 4$ occupies the orbitals 0 and 2.

The operators. The **creation operator** $a_p^\dagger$ adds a particle in orbital p, the **annihilation operator** a_p removes one:

$$
\begin{aligned}
a_p^\dagger \,|\cdots n_p \cdots\rangle &= (-1)^{\nu_p}\,(1 - n_p)\,|\cdots n_p{+}1 \cdots\rangle, \\
a_p \,|\cdots n_p \cdots\rangle &= (-1)^{\nu_p}\,n_p\,|\cdots n_p{-}1 \cdots\rangle, \qquad \nu_p = \sum_{q<p} n_q .
\end{aligned}
$$

The factor $1 - n_p$ makes $a_p^\dagger$ give zero on an occupied orbital (Pauli), the factor n_p makes a_p give zero on an empty one, and the sign $(-1)^{\nu_p}$ is the sign that a determinant picks up when the column of orbital p is moved past the ν_p occupied columns in front of it (each exchange of two columns changes the sign). On the 16 states of four orbitals each a_p is a 16×16 matrix of zeros and ± 1, and $a_p^\dagger$ is its transpose (Figure 13c.2 draws two of them).

The anticommutation relations. With the **anticommutator** $\{A, B\} = AB + BA$,

$$
\{a_p, a_q^\dagger\} = \delta_{pq}, \qquad \{a_p, a_q\} = 0, \qquad \{a_p^\dagger, a_q^\dagger\} = 0.
$$

Proof for $p = q$, on a state with $n_p = 0$: $a_p^\dagger$ creates the particle with the sign $(-1)^{\nu_p}$, and a_p removes it again with the same sign (the orbitals before p are unchanged), so $a_p a_p^\dagger$ gives the state back with the sign $(-1)^{2\nu_p} = +1$; and $a_p^\dagger a_p$ gives 0 because a_p acts on an empty orbital. The sum is 1. On a state with $n_p = 1$ the two roles are exchanged, and the sum is again 1. For $p < q$: whichever of the two operators acts on orbital p changes n_p, which changes ν_q by one and so flips the sign factor of the operator acting on q; the two orders therefore give the same state with opposite signs, and their sum vanishes. In particular $a_p^\dagger a_p^\dagger = 0$: the Pauli principle once more. The **number operator** $\hat{n}_p = a_p^\dagger a_p$ gives n_p, and $\hat{N} = \sum_p \hat{n}_p$ counts the particles of a state.

Operators of the many-body theory. A one-body operator, one copy $A(i)$ of a one-particle operator A for each particle, becomes $\hat{A} = \sum_{p,q} A_{pq}\, a_p^\dagger a_q$ with $A_{pq} = \langle \phi_p | A \phi_q \rangle$: applied to a determinant, $\sum_i A(i)$ replaces one occupied orbital ϕ_q at a time by $A\phi_q = \sum_p \phi_p A_{pq}$, and "remove q, put p, with the weight A_{pq}" is what $A_{pq}\, a_p^\dagger a_q$ does, sign included. The pair interaction is

$$
\hat{W} = \frac{1}{2} \sum_{p,q,r,s} w_{pqrs}\, a_p^\dagger a_q^\dagger a_s a_r, \qquad w_{pqrs} = \iint \phi_p^*(r)\, \phi_q^*(r')\, w(r, r')\, \phi_r(r)\, \phi_s(r')\, dr\, dr',
$$

where $\int dr$ includes the sum over the label. The operator removes an occupied pair (r, s) and puts

(p,q) in its place; the factor $\frac{1}{2}$ compensates for counting each pair twice, and the reversed order $a_s a_r$ makes the direct term come with a plus sign.

Expectation values in a determinant. Let $|\Phi\rangle$ occupy the set O of basis orbitals, and write $[p \in O]$ for 1 if p is occupied and 0 otherwise. Then

$$\langle\Phi|a_p^\dagger a_q|\Phi\rangle = \delta_{pq}\,[p \in O].$$

Reason: a_q removes q, and $a_p^\dagger$ must put back the same orbital to return to $|\Phi\rangle$, because states with different occupations are orthogonal; for $p = q$ the operator is $\hat{n}_p$, which gives $[p \in O]$. For four operators the pair removed must be the pair put back, $\{p,q\} = \{r,s\}$ with $p \neq q$:

$$\langle\Phi|a_p^\dagger a_q^\dagger a_s a_r|\Phi\rangle = [p \in O]\,[q \in O]\,(\delta_{pr}\delta_{qs} - \delta_{ps}\delta_{qr}).$$

If $p = r$ and $q = s$ the operator is $a_p^\dagger a_q^\dagger a_q a_p = a_p^\dagger \hat{n}_q a_p = \hat{n}_q \hat{n}_p$ (for $q \neq p$, $\hat{n}_q$ commutes with a_p and $a_p^\dagger$, because moving a_p past the pair $a_q^\dagger a_q$ costs two sign changes), which gives $[p \in O][q \in O]$; if $p = s$ and $q = r$, one exchange $a_q a_p = -a_p a_q$ gives the minus sign. Both formulas are summarised by the **one-body density matrix** $\rho = \sum_{a\in O} |\phi_a\rangle\langle\phi_a|$, whose matrix elements are $\rho_{qp} = \langle a_p^\dagger a_q\rangle$:

$$\langle a_p^\dagger a_q\rangle = \rho_{qp}, \qquad \langle a_p^\dagger a_q^\dagger a_s a_r\rangle = \rho_{rp}\,\rho_{sq} - \rho_{sp}\,\rho_{rq}.$$

This is **Wick's theorem** for a determinant: every expectation value is a sum of products of the density matrix. For a determinant ρ is a projector: $\rho^2 = \rho$ (applying "project on the occupied orbitals" twice is the same as once), and $\operatorname{Tr}\rho = N$.

Every basis. If new orbitals are made from the old ones by a unitary matrix U ($U^\dagger U = 1$), $\phi'_p = \sum_k \phi_k U_{kp}$, then a determinant is linear in each of its columns, so the creation operator of ϕ'_p is $a'^\dagger_p = \sum_k U_{kp}\, a_k^\dagger$, and its adjoint is $a'_p = \sum_k U^*_{kp}\, a_k$. Expectation values are linear in the operator, so

$$\langle a'^\dagger_p a'_q\rangle = \sum_{k,l} U_{kp}\, U^*_{lq}\, \langle a_k^\dagger a_l\rangle = \sum_{k,l} U^*_{lq}\, \rho_{lk}\, U_{kp} = (U^\dagger \rho\, U)_{qp},$$

which is the matrix element of the same ρ between the new orbitals. Call this matrix $\rho' = U^\dagger \rho\, U$, so that $\langle a'^\dagger_p a'_q\rangle = \rho'_{qp}$. The four-operator formula transforms in the same way; line by line:

$$\begin{aligned}
\langle a'^\dagger_p a'^\dagger_q a'_s a'_r\rangle &= \sum_{k,l,m,n} U_{kp}\, U_{lq}\, U^*_{ms}\, U^*_{nr}\, \langle a_k^\dagger a_l^\dagger a_m a_n\rangle \\
&= \sum_{k,l,m,n} U_{kp}\, U_{lq}\, U^*_{ms}\, U^*_{nr}\, (\rho_{nk}\,\rho_{ml} - \rho_{mk}\,\rho_{nl}) \\
&= \Big(\sum_{n,k} U^*_{nr}\,\rho_{nk}\,U_{kp}\Big)\Big(\sum_{m,l} U^*_{ms}\,\rho_{ml}\,U_{lq}\Big) \\
&\qquad - \Big(\sum_{m,k} U^*_{ms}\,\rho_{mk}\,U_{kp}\Big)\Big(\sum_{n,l} U^*_{nr}\,\rho_{nl}\,U_{lq}\Big) \\
&= \rho'_{rp}\,\rho'_{sq} - \rho'_{sp}\,\rho'_{rq}.
\end{aligned}$$

The first line inserts the four expansions $a'^\dagger_p = \sum_k U_{kp}\, a^\dagger_k$, $a'^\dagger_q = \sum_l U_{lq}\, a^\dagger_l$, $a'_s = \sum_m U^*_{ms}\, a_m$, $a'_r = \sum_n U^*_{nr}\, a_n$, multiplies out, and uses the linearity of expectation values; the second is the four-operator formula above with p, q, s, r renamed k, l, m, n; the third (written on two rows) splits each of the two products into two independent sums (each sum collects the factors that carry its two summation letters); the fourth recognises each sum as an element of the matrix product $U^\dagger \rho\, U$, for example $\sum_{n,k} U^*_{nr}\, \rho_{nk}\, U_{kp} = (U^\dagger \rho\, U)_{rp}$ because $(U^\dagger)_{rn} = U^*_{nr}$. The result is Wick's formula again, now with the density matrix ρ' of the new basis. So Wick's theorem holds in every orthonormal basis.

A thermal ensemble. Wick's theorem also holds for a **thermal ensemble** of non-interacting fermions, the statistical mixture of states that describes them at a temperature $T > 0$ (Section 13.26), with occupations f_a between 0 and 1 in place of 0 and 1. Section 13.26 shows two facts about it: in the basis of the orbitals ϕ_a of the non-interacting Hamiltonian, the ensemble is a mixture of the occupation-number states $|n_0 n_1 \cdots\rangle$ (they are the eigenstates of $\hat{H} - \mu\hat{N}$, so the Gibbs state, a function of this operator, gives each of them a probability, its Gibbs weight); and in this mixture orbital a is occupied with the Fermi-Dirac probability f_a, independently of the other orbitals. An expectation value in a mixture is the average, with these probabilities, of the expectation values in its states (Section 13.26), and each state is a determinant of basis orbitals. So the two determinant formulas above give, in this basis, line by line:

$$\begin{aligned}
\langle a^\dagger_p a_q\rangle &= \delta_{pq}\,\langle \hat{n}_p\rangle = \delta_{pq}\, f_p, \\
\langle a^\dagger_p a^\dagger_q a_s a_r\rangle &= \langle \hat{n}_p\, \hat{n}_q\rangle\, (\delta_{pr}\delta_{qs} - \delta_{ps}\delta_{qr}) \\
&= f_p\, f_q\, (\delta_{pr}\delta_{qs} - \delta_{ps}\delta_{qr}) \\
&= \rho_{rp}\,\rho_{sq} - \rho_{sp}\,\rho_{rq}, \qquad \rho = \mathrm{diag}(f_0, f_1, \dots).
\end{aligned}$$

The first line averages $\delta_{pq}[p \in O]$; the average of $[p \in O]$ is the probability that orbital p is occupied, f_p. So $\rho_{qp} = \delta_{pq} f_p$: the density matrix is diagonal, with the occupations on its diagonal. The second line averages $[p \in O]\,[q \in O]$, the value of $\hat{n}_p \hat{n}_q$ in each occupation-number state; this average is the probability that both p and q are occupied. The third line uses the independence: for $p \neq q$ that probability is the product $f_p f_q$. (For $p = q$ the bracket $\delta_{pr}\delta_{ps} - \delta_{ps}\delta_{pr}$ is 0, so this case needs nothing.) Independence is needed here because the four-operator formula is a product of two density-matrix elements, and the average of a product of two occupations is in general not the product of their averages; for independent occupations it is. The fourth line inserts $\rho_{rp} = \delta_{pr} f_p$, $\rho_{sq} = \delta_{qs} f_q$, $\rho_{sp} = \delta_{ps} f_p$ and $\rho_{rq} = \delta_{qr} f_q$: the two products are $f_p f_q\, \delta_{pr}\delta_{qs}$ and $f_p f_q\, \delta_{ps}\delta_{qr}$. The basis change above then carries Wick's theorem for the thermal ensemble to every orthonormal basis, with $\rho' = U^\dagger\, \mathrm{diag}(f_0, f_1, \dots)\, U$. Notebook 13c checks all $16 + 256$ two- and four-operator values in a randomly rotated basis, for a determinant and for a thermal ensemble (In [6] and In [7]).

The normal-ordered square of a one-body quantity. The interaction of the dirac16complex Kohn-Sham model (Section 13.16) is the square of a one-body quantity $S = \sum_{p,q} V_{pq}\, a^\dagger_p a_q$ with a Hermitian matrix V (the **vertex**). Its **normal-ordered square** puts all creation operators to the left: $:S^2: = \sum V_{pq} V_{rs}\, a^\dagger_p a^\dagger_r a_s a_q$. Line by line:

$$\begin{aligned}\langle{:}S^2{:}\rangle &= \sum_{p,q,r,s} V_{pq}V_{rs}\,\langle a_p^\dagger a_r^\dagger a_s a_q\rangle \\ &= \sum_{p,q,r,s} V_{pq}V_{rs}\,(\rho_{qp}\,\rho_{sr} - \rho_{sp}\,\rho_{qr}) \\ &= \Big(\sum_{p,q} V_{pq}\rho_{qp}\Big)\Big(\sum_{r,s} V_{rs}\rho_{sr}\Big) - \sum_{p,q,r,s} V_{pq}\,\rho_{qr}\,V_{rs}\,\rho_{sp} \\ &= (\mathrm{Tr}\,V\rho)^2 - \mathrm{Tr}(V\rho\,V\rho).\end{aligned}$$

The first line uses the linearity of expectation values; the second is Wick's four-operator formula with the indices renamed (its q, s, r are here r, s, q); the third splits the first product into two independent sums and reorders the factors of the second; the fourth recognises the sums as traces of matrix products ($\mathrm{Tr}\,AB = \sum_{p,q} A_{pq}B_{qp}$). The first term is a **Hartree** (direct) term, the second an **exchange** term. PROVED here, and recorded in the Revision record as the check hf_wick_contraction of `Revision/kohn_sham/reports/ks-theory-python.json` (brute force on a two-mode state; Notebook 13b, In [21], checks that its verdict is PASS); Notebook 13c (In [8]) checks it on four orbitals with a random Hermitian vertex.

13.5 The energy of a determinant: direct and exchange terms

The formula. The Hamiltonian in second quantisation is $\hat{H} = \sum h_{pq}\,a_p^\dagger a_q + \frac{1}{2}\sum w_{pqrs}\,a_p^\dagger a_q^\dagger a_s a_r$ with $h = -\frac{1}{2}\,d^2/dx^2 + v$. For the determinant of the basis orbitals in O, line by line:

$$\begin{aligned}\langle\Phi|\hat{H}|\Phi\rangle &= \sum_{p,q} h_{pq}\,\delta_{pq}[p\in O] + \tfrac{1}{2}\sum_{p,q,r,s} w_{pqrs}\,[p\in O][q\in O]\,(\delta_{pr}\delta_{qs} - \delta_{ps}\delta_{qr}) \\ &= \sum_{a\in O} h_{aa} + \tfrac{1}{2}\sum_{a,b\in O}(w_{abab} - w_{abba}).\end{aligned}$$

The first line inserts the two expectation values of Section 13.4; the second carries out the sums over the Kronecker deltas (δ_{pr} sets $r = p$, and so on) and renames p, q as a, b. Notebook 13c (In [10]) checks this formula for random one-body and two-body integrals.

Direct and exchange in space. Writing out the integrals and using $\sum_{a\in O}\phi_a(r)\phi_a^*(r') = \rho(r, r')$ (the density matrix in space, whose diagonal is the density n):

$$\begin{aligned}E_H &= \tfrac{1}{2}\sum_{a,b} w_{abab} = \tfrac{1}{2}\iint n(r)\,w(r,r')\,n(r')\,dr\,dr', \\ E_x &= -\tfrac{1}{2}\sum_{a,b} w_{abba} = -\tfrac{1}{2}\iint |\rho(r,r')|^2\,w(r,r')\,dr\,dr'.\end{aligned}$$

E_H is the **Hartree energy**: the classical energy of the cloud n with itself. E_x is the **exchange energy** (the Fock term); it has no classical analogue, it comes from the antisymmetry of Φ, and for a repulsive w it lowers the energy. Three remarks. (1) **No self-interaction.** In E_H the terms $a = b$ describe a particle repelling its own cloud, which is unphysical; in E_x the terms $a = b$ are the same numbers with a minus sign, so they cancel. (2) **Exchange acts only between equal labels.** If every orbital has a definite label, $\rho(x\sigma, x'\sigma') = 0$ for $\sigma \neq \sigma'$, so for a label-independent

w the exchange term couples only particles with the same label. (3) **Double counting.** Adding orbital energies counts every pair twice (Section 13.6).

The contact interaction. Let the particles interact only when they are at the same point, with the strength g_c and independently of their labels: $w(x\sigma, x'\sigma') = g_c\,\delta(x - x')$, where δ is Dirac's delta function ($\int f(x')\,\delta(x - x')\,dx' = f(x)$). For orbitals of definite label, with the label densities $n_\sigma(x) = \rho(x\sigma, x\sigma)$, line by line:

$$\begin{aligned} E_x &= -\tfrac{1}{2}\sum_{\sigma,\sigma'}\iint |\rho(x\sigma, x'\sigma')|^2\, g_c\,\delta(x - x')\,dx\,dx' \\ &= -\tfrac{g_c}{2}\sum_{\sigma,\sigma'}\int |\rho(x\sigma, x\sigma')|^2\,dx \\ &= -\tfrac{g_c}{2}\int \sum_{\sigma} n_\sigma(x)^2\,dx. \end{aligned}$$

The first line writes the space-and-label integral as a sum over both labels and integrals over both positions; the second does the x' integral with the delta function; the third keeps only $\sigma' = \sigma$ (the other terms vanish for orbitals of definite label) and uses $\rho(x\sigma, x\sigma) = n_\sigma(x)$. In the same way $E_H = \frac{g_c}{2}\int n^2\,dx$. Two facts follow. **First, the exchange energy of a contact interaction is exactly local**: an integral over one point of a function of the densities at that point, although the general exchange integral involves two points. **Second**, if all g labels are equally occupied, $n_\sigma = n/g$, then $\sum_\sigma n_\sigma^2 = g\,(n/g)^2 = n^2/g$, and

$$E_x = -\frac{1}{g}\,E_H \qquad \text{(contact interaction, } g \text{ equally occupied labels).}$$

For $g = 2$: $E_H + E_x = \frac{g_c}{2}\int [(n_\uparrow + n_\downarrow)^2 - n_\uparrow^2 - n_\downarrow^2]dx = g_c\int n_\uparrow n_\downarrow\,dx$. Two fermions with the same label never meet (Pauli), so only opposite labels feel a contact force, and exchange removes exactly the unphysical same-label part of E_H. For a single label ($g = 1$) the contact interaction cancels completely. PROVED here; Notebook 13b checks it for the uniform gas, for a non-uniform determinant and with label-mixing orbitals (In [6], In [12], In [13]).

13.6 The Hartree and Hartree-Fock equations

The **Hartree-Fock approximation** takes the best single determinant: it minimises $\langle\Phi|\hat{H}|\Phi\rangle$ over all choices of N orthonormal orbitals. By the variational principle (Section 13.2) its energy E_{HF} is an upper bound on the true ground-state energy E_0.

Minimising with side conditions: Lagrange multipliers. To minimise a function $f(y_1, \ldots, y_m)$ of m real variables under a side condition $c(y) = c_0$, look at a small step dy that keeps the condition: it changes c by $dc = \sum_i (\partial c/\partial y_i)\,dy_i = 0$. At a minimum the change $df = \sum_i (\partial f/\partial y_i)\,dy_i$ must vanish for every such step (otherwise the step or the opposite step would lower f). Write the gradient of f as $\partial f/\partial y_i = \lambda\,\partial c/\partial y_i + r_i$, where λ is chosen so that the rest r is perpendicular to the gradient of c, $\sum_i r_i\,\partial c/\partial y_i = 0$. Then the step $dy_i = \epsilon\,r_i$ keeps the condition, and

$$0 = df = \epsilon\sum_i (\lambda\,\partial c/\partial y_i + r_i)\,r_i = \epsilon\sum_i r_i^2,$$

where the first equality is the condition for a minimum, the second inserts the split gradient, and the third uses the perpendicularity of r. So every $r_i = 0$: **at a constrained minimum,** $\partial f/\partial y_i = \lambda\,\partial c/\partial y_i$ **for every** i. The number λ is a **Lagrange multiplier**; with several conditions one takes one multiplier for each. (That a first-order step can be completed to a step that keeps the condition exactly is the implicit function theorem of analysis, used here without proof.) Differentiating the minimum value $f^*(c_0)$ by the chain rule shows that $df^*/dc_0 = \lambda$: the multiplier is the rate at which the constrained minimum changes with the condition.

The Hartree-Fock equations, line by line. The unknowns are the N occupied orbitals $\phi_1, \dots, \phi_N$; in this paragraph every sum over b and c runs over them, and $\int dr$ includes the sum over the label. The energy to be minimised is (Section 13.5)

$$E = \sum_c h_{cc} + \tfrac{1}{2}\sum_{c,b} (w_{cbcb} - w_{cbbc}), \qquad h_{cc} = \int \phi_c^*(r)\,(h\phi_c)(r)\,dr,$$

with, by the definition of w_{pqrs} in Section 13.4,

$$w_{cbcb} = \iint \phi_c^*(r)\,\phi_b^*(r')\,w(r,r')\,\phi_c(r)\,\phi_b(r')\,dr\,dr',$$
$$w_{cbbc} = \iint \phi_c^*(r)\,\phi_b^*(r')\,w(r,r')\,\phi_b(r)\,\phi_c(r')\,dr\,dr'.$$

The side conditions and their multipliers. The conditions are $\langle\phi_c|\phi_b\rangle = \delta_{cb}$. Since $\langle\phi_b|\phi_c\rangle = \langle\phi_c|\phi_b\rangle^*$, the independent real conditions are $\langle\phi_c|\phi_c\rangle = 1$ and, for $c < b$, $\operatorname{Re} X = 0$ and $\operatorname{Im} X = 0$ with $X = \langle\phi_c|\phi_b\rangle$. Give each a real multiplier, λ_c and α, β. With $\operatorname{Re} X = (X + X^*)/2$ and $\operatorname{Im} X = (X - X^*)/(2i)$ one has $\alpha \operatorname{Re} X + \beta \operatorname{Im} X = \Lambda_{bc} X + \Lambda_{bc}^* X^*$ with $\Lambda_{bc} = (\alpha - i\beta)/2$; so, putting $\Lambda_{cc} = \lambda_c$ and $\Lambda_{cb} = \Lambda_{bc}^*$, the multipliers form a **Hermitian** matrix Λ, and the function to be made stationary is

$$\mathcal{L} = E - \sum_{c,b} \Lambda_{bc}\big(\langle\phi_c|\phi_b\rangle - \delta_{cb}\big).$$

$\mathcal{L}$ is real for any orbitals, orthonormal or not. In E, h_{cc} is real because h is Hermitian; $w_{cbcb} = \iint |\phi_c(r)|^2\,w(r,r')\,|\phi_b(r')|^2\,dr\,dr'$ is real because the interaction w is real; and w_{cbbc} equals its own complex conjugate after the integration variables are renamed ($r \leftrightarrow r'$), because w is real and symmetric, $w(r',r) = w(r,r')$, as every interaction of this chapter. The complex conjugate of the multiplier sum is $\sum_{c,b} \Lambda_{bc}^* \langle\phi_b|\phi_c\rangle = \sum_{c,b} \Lambda_{cb} \langle\phi_b|\phi_c\rangle$, the same sum with the names b and c exchanged. By the Lagrange rule above, at a constrained minimum $\partial\mathcal{L}/\partial y_i = 0$ for every real variable y_i (here: the real and imaginary parts of the orbital values; on a grid these are finitely many numbers), so the first-order change of $\mathcal{L}$ vanishes for **every** small change of the orbitals.

Changing one orbital. Change ϕ_a by $\varepsilon\zeta$, where ζ is any function and ε a small real number; then ϕ_a^* changes by $\varepsilon\zeta^*$. Each term of $\mathcal{L}$ is a sum of products in which ϕ_a and ϕ_a^* appear as factors, so to first order in ε the change of $\mathcal{L}$ is $\varepsilon(A + B)$ with $A = \int \zeta^*(r)\,G_a(r)\,dr$, where G_a collects what multiplies the change of ϕ_a^*, and $B = \int \zeta(r)\,K_a(r)\,dr$, where K_a collects what multiplies the change of ϕ_a. Since $\mathcal{L}$ is real for every ζ, $A + B$ is real; with $i\zeta$ in place of ζ, A becomes $-iA$ and B becomes iB, so $i(B - A)$ is real as well. Writing $A = a_1 + ia_2$ and $B = b_1 + ib_2$, the first statement says $a_2 + b_2 = 0$ and the second $b_1 - a_1 = 0$: $B = A^*$, and the change is $\varepsilon(A + A^*) = 2\varepsilon \operatorname{Re} \int \zeta^* G_a\,dr$. It

must vanish for every ζ; for $\zeta = G_a$ it is $2\varepsilon \int |G_a|^2\, dr$, which vanishes only if $G_a(r) = 0$ at every point. So **the minimum requires $G_a = 0$, and G_a is found by changing ϕ_a^* alone** while ϕ_a and all other orbitals are kept fixed. (This is the precise meaning of "varying ϕ^* as if it were independent of ϕ". Section 13.9 calls $G_a(r)$ the **functional derivative** of $\mathcal{L}$ with respect to $\phi_a^*(r)$; here it was found directly as the coefficient of the first-order change.)

The four parts of G_a. Replace ϕ_a^* by $\phi_a^* + \varepsilon\zeta^*$ and keep everything else. The one-body part:

$$\delta\Big(\sum_c h_{cc}\Big) = \varepsilon \int \zeta^*(r)\,(h\phi_a)(r)\,dr,$$

because only the term $c = a$ contains ϕ_a^*, as its first factor. The direct part:

$$\begin{aligned}\delta\Big(\tfrac{1}{2}\sum_{c,b} w_{cbcb}\Big) &= \tfrac{\varepsilon}{2}\sum_b \iint \zeta^*(r)\,\phi_b^*(r')\,w(r,r')\,\phi_a(r)\,\phi_b(r')\,dr\,dr' \\ &\quad + \tfrac{\varepsilon}{2}\sum_c \iint \phi_c^*(r)\,\zeta^*(r')\,w(r,r')\,\phi_c(r)\,\phi_a(r')\,dr\,dr' \\ &= \tfrac{\varepsilon}{2}\int \zeta^*(r)\,v_H(r)\,\phi_a(r)\,dr + \tfrac{\varepsilon}{2}\int \zeta^*(r)\,v_H(r)\,\phi_a(r)\,dr = \varepsilon\int \zeta^*(r)\,v_H(r)\,\phi_a(r)\,dr.\end{aligned}$$

The first line collects the terms with $c = a$, where ϕ_a^* is the first factor (variable r), and the second those with $b = a$, where it is the second factor (variable r'); the term $c = b = a$ contains ϕ_a^* twice and contributes to both lines, one change each, as the product rule says. In the third line, the first part uses $\sum_b \phi_b^*(r')\phi_b(r') = n(r')$ and the **Hartree potential** $v_H(r) = \int w(r,r')\,n(r')\,dr'$; the second part becomes the same after renaming the two integration variables ($r \leftrightarrow r'$) and using $w(r',r) = w(r,r')$. The two halves add up: the factor $\frac{1}{2}$ is cancelled. The exchange part, in the same way:

$$\begin{aligned}\delta\Big(-\tfrac{1}{2}\sum_{c,b} w_{cbbc}\Big) &= -\tfrac{\varepsilon}{2}\sum_b \iint \zeta^*(r)\,\phi_b^*(r')\,w(r,r')\,\phi_b(r)\,\phi_a(r')\,dr\,dr' \\ &\quad - \tfrac{\varepsilon}{2}\sum_c \iint \phi_c^*(r)\,\zeta^*(r')\,w(r,r')\,\phi_a(r)\,\phi_c(r')\,dr\,dr' \\ &= -\varepsilon\int \zeta^*(r)\Big[\int \rho(r,r')\,w(r,r')\,\phi_a(r')\,dr'\Big]dr.\end{aligned}$$

The first two lines are again the terms $c = a$ and $b = a$; in the first, $\sum_b \phi_b(r)\phi_b^*(r') = \rho(r,r')$ is the density matrix in space (Section 13.5); the second becomes the same after renaming $r \leftrightarrow r'$ and using the symmetry of w, and the two halves add up. The multiplier part:

$$\delta\Big(-\sum_{c,b}\Lambda_{bc}\,\langle\phi_c|\phi_b\rangle\Big) = -\varepsilon\sum_b \Lambda_{ba}\int \zeta^*(r)\,\phi_b(r)\,dr,$$

because $\langle\phi_c|\phi_b\rangle = \int \phi_c^*\phi_b\,dr$ contains ϕ_a^* only for $c = a$, and the numbers δ_{cb} do not change. Adding the four parts, $G_a(r) = (h\phi_a)(r) + v_H(r)\,\phi_a(r) - \int \rho(r,r')\,w(r,r')\,\phi_a(r')\,dr' - \sum_b \Lambda_{ba}\,\phi_b(r)$, and $G_a = 0$ is

$$\hat{F}\phi_a = \sum_b \Lambda_{ba}\,\phi_b, \qquad (\hat{F}\phi)(r) = h\phi(r) + v_H(r)\,\phi(r) - \int \rho(r,r')\,w(r,r')\,\phi(r')\,dr',$$

with the Hartree potential $v_H(r) = \int w(r,r')\,n(r')\,dr'$ found above. $\hat{F}$ is the **Fock operator**. It depends on the occupied orbitals only through ρ, which does not change when the occupied orbitals are mixed among themselves by a unitary matrix; such a mixing can make the Hermitian matrix Λ diagonal, which gives the **canonical Hartree-Fock equations** $\hat{F}\phi_a = \epsilon_a\phi_a$. They look like one-particle equations, but $\hat{F}$ contains the unknown orbitals: they are **nonlinear** and are solved by iteration, starting from a guess and repeating until nothing changes any more (**self-consistency**, Section 13.21). For the ground state one occupies the N orbitals with the lowest ϵ_a (the **aufbau** rule, German for "building up"). The older **Hartree approximation** keeps v_H and drops the exchange integral, and with it the cancellation of the self-interaction.

Double counting. Summing the orbital energies over the occupied orbitals,

$$\sum_{a\in O}\epsilon_a = \sum_a \langle\phi_a|\hat{F}\phi_a\rangle = \sum_a h_{aa} + \sum_{a,b}(w_{abab} - w_{abba}) = \sum_a h_{aa} + 2E_H + 2E_x,$$

where the first step uses $\hat{F}\phi_a = \epsilon_a\phi_a$ and normalisation, the second inserts the definition of $\hat{F}$, and the third the definitions of E_H and E_x of Section 13.5. Comparing with the energy of the determinant,

$$E_{HF} = \sum_{a\in O}\epsilon_a - E_H - E_x.$$

Koopmans' theorem. Removing the particle of orbital a while the other orbitals are kept unchanged lowers the energy by $h_{aa} + \sum_b(w_{abab} - w_{abba}) = \langle\phi_a|\hat{F}\phi_a\rangle = \epsilon_a$: the orbital energy is the energy of that particle in the unrelaxed determinant. The Kohn-Sham analogue, Janak's theorem, is proved in Section 13.27.

13.7 Two electrons on two sites: exact and Hartree-Fock

The smallest system in which the interaction matters: two sites L and R with one orbital each, the hopping t of Section 13.2 between them, a repulsion $U > 0$ when both electrons sit on the same site, and two electrons with opposite labels. In second quantisation, with the four orbitals L up, R up, L down, R down (numbers 0, 1, 2, 3) and site energies $-\Delta/2$ on L and $+\Delta/2$ on R,

$$\hat{H} = -t\sum_\sigma (a^\dagger_{L\sigma}a_{R\sigma} + a^\dagger_{R\sigma}a_{L\sigma}) + U(\hat{n}_{L\uparrow}\hat{n}_{L\downarrow} + \hat{n}_{R\uparrow}\hat{n}_{R\downarrow}) - \tfrac{\Delta}{2}\hat{n}_L + \tfrac{\Delta}{2}\hat{n}_R,$$

with $\hat{n}_L = \hat{n}_{L\uparrow} + \hat{n}_{L\downarrow}$. In this section $\Delta = 0$.

Exact solution. Two electrons with opposite labels can be both on L (state LL), both on R (RR), or one on each, in the symmetric combination S of "up on L, down on R" and "up on R, down on L"; the antisymmetric combination has energy 0 and plays no role. Hopping one electron turns LL into either of the two mixed states, which gives the matrix element $-\sqrt{2}\,t$ between LL and S (two

terms $-t$, and the normalisation $1/\sqrt{2}$ of S), and likewise between RR and S; the repulsion gives U on LL and RR and 0 on S. In the basis (LL, RR, S)

$$\hat{H} = \begin{pmatrix} U & 0 & -\sqrt{2}\,t \\ 0 & U & -\sqrt{2}\,t \\ -\sqrt{2}\,t & -\sqrt{2}\,t & 0 \end{pmatrix}.$$

The combination $(LL - RR)/\sqrt{2}$ is decoupled, with the energy U. The combination $D = (LL + RR)/\sqrt{2}$ couples to S with the element $\frac{1}{\sqrt{2}}(-\sqrt{2}\,t - \sqrt{2}\,t) = -2t$, which leaves the 2×2 matrix $\begin{pmatrix} U & -2t \\ -2t & 0 \end{pmatrix}$. Its eigenvalues solve $\epsilon(\epsilon - U) - 4t^2 = 0$, so

$$E_0 = \tfrac{1}{2}(U - \sqrt{U^2 + 16t^2}),$$

the lower root (it is negative, because $\sqrt{U^2 + 16t^2} > U$, so it is below the energies 0 and U of the other states). For $t = 1$, $U = 2$: $E_0 = \frac{1}{2}(2 - \sqrt{20}) = -1.236068$ (Notebook 13c, In [11]).

The energy of every determinant. A determinant for two electrons of opposite labels uses one up orbital and one down orbital. Take the up orbital with the values $(\cos\alpha, \sin\alpha)$ on (L, R) and the down orbital $(\cos\beta, \sin\beta)$, $0 \le \alpha, \beta \le \pi/2$ (real positive values give the lowest hopping energy). The hopping energy of an orbital (c_L, c_R) is $-2t\,c_L c_R$, which is $-2t\cos\alpha\sin\alpha = -t\sin 2\alpha$ for the up orbital. The repulsion is a contact interaction on each site, so by Section 13.5 its Hartree plus exchange energy is U times the product of the up and down densities on each site. Writing $c_\alpha = \cos 2\alpha$ and $s_\alpha = \sin 2\alpha$ (and the same for β), line by line:

$$\begin{aligned} E(\alpha,\beta) &= -t\,(s_\alpha + s_\beta) + U(\cos^2\alpha\,\cos^2\beta + \sin^2\alpha\,\sin^2\beta) \\ &= -t\,(s_\alpha + s_\beta) + \tfrac{U}{4}\big[(1 + c_\alpha)(1 + c_\beta) + (1 - c_\alpha)(1 - c_\beta)\big] \\ &= -t\,(s_\alpha + s_\beta) + \tfrac{U}{2}(1 + c_\alpha c_\beta). \end{aligned}$$

The first line adds the two hopping energies and $U(n_{L\uparrow}n_{L\downarrow} + n_{R\uparrow}n_{R\downarrow})$; the second uses $\cos^2\alpha = \frac{1}{2}(1 + \cos 2\alpha)$ and $\sin^2\alpha = \frac{1}{2}(1 - \cos 2\alpha)$; the third multiplies out (the terms $\pm c_\alpha$, $\pm c_\beta$ cancel). Notebook 13c (In [12]) checks this formula against the expectation value of the Hamiltonian matrix.

The best determinant. Restricted Hartree-Fock puts both electrons into the bonding orbital, $\alpha = \beta = \pi/4$: $s = 1$, $c = 0$, so $E_{RHF} = -2t + U/2$. Is there a better determinant? Since $(c_\alpha + c_\beta)^2 \ge 0$, we have $c_\alpha c_\beta \ge -\frac{1}{2}(c_\alpha^2 + c_\beta^2)$, and with $1 - c^2 = s^2$:

$$E \;\ge\; -t\,(s_\alpha + s_\beta) + \tfrac{U}{2}\Big(1 - \tfrac{c_\alpha^2 + c_\beta^2}{2}\Big) = \Big(-t\,s_\alpha + \tfrac{U}{4}s_\alpha^2\Big) + \Big(-t\,s_\beta + \tfrac{U}{4}s_\beta^2\Big).$$

Each bracket is a parabola in one number $s \in [0,1]$; its derivative $-t + \frac{U}{2}s$ vanishes at $s = 2t/U$. For $U \le 2t$ this lies at or beyond 1, so the smallest value on $[0,1]$ is at $s = 1$, $-t + U/4$, and the minimum over all determinants is $E_{HF} = -2t + U/2$, the restricted one. For $U > 2t$ the smallest value is at $s = 2t/U$: $-t \cdot \frac{2t}{U} + \frac{U}{4} \cdot \frac{4t^2}{U^2} = -\frac{t^2}{U}$, so $E_{HF} = -2t^2/U$. Equality holds in the inequality for $\beta = \pi/2 - \alpha$ ($c_\beta = -c_\alpha$, $s_\beta = s_\alpha$): the **unrestricted** determinant puts the up electron mostly on one site and the down electron mostly on the other. It breaks the left-right symmetry of each

label but keeps the total density $(1,1)$. Figure 13c.6 shows the energy landscape $E(\alpha,\beta)$ at $U=4t$, with the restricted point as a saddle.

Correlation. For $t=1$, $U=2$: $E_{HF}=-1$, and $E_c = E_0 - E_{HF} = -0.236068$ is the **correlation energy**, the energy that no single determinant can capture (it is never positive, by the variational principle). Its meaning is visible in the **double occupancy**, the probability that both electrons sit on the same site: $\frac{1}{2}$ in the restricted determinant for every U, but only 0.276393 in the exact ground state at $U=2t$ (Notebook 13c, In [11]). The exact electrons avoid each other and keep each label shared equally between the sites; a single determinant cannot do both. Yet the density is the same, $(1,1)$, in the exact state and in the Hartree-Fock state. A theory that works with the density alone must therefore contain somewhere the information that turns the density $(1,1)$ into -1.236068 rather than -1. In DFT that information is the exchange-correlation energy.

13.8 The density decides everything: the Hohenberg-Kohn theorem

We now leave wave functions behind. Fix the particle number N and the interaction $\hat{W}$, and regard the external potential v as the only thing that distinguishes one system from another.

Theorem (Hohenberg and Kohn, 1964). Let v and v' be two external potentials whose Hamiltonians $\hat{H}=\hat{T}+\hat{W}+\hat{V}$ and $\hat{H}'=\hat{T}+\hat{W}+\hat{V}'$ have non-degenerate ground states Ψ and Ψ' (non-degenerate: no other state has the same lowest energy). If $v-v'$ is not a constant, the ground-state densities n and n' differ. So the ground-state density determines the potential up to a constant, hence the Hamiltonian, hence every property of the system.

Proof, line by line. First, $\Psi \neq \Psi'$. If they were equal, subtracting the two Schroedinger equations $\hat{H}\Psi = E_0\Psi$ and $\hat{H}'\Psi = E_0'\Psi$ would give $(\hat{V}-\hat{V}')\Psi = (E_0-E_0')\Psi$, that is $\sum_i [v(x_i)-v'(x_i)] = E_0-E_0'$ wherever $\Psi \neq 0$, which forces $v-v'$ to be a constant (this step ASSUMES that Ψ does not vanish on a whole region, which holds for the potentials of physics). Now suppose, to reach a contradiction, that $n=n'$. Because Ψ is the only ground state of $\hat{H}$ and $\Psi' \neq \Psi$, the variational principle holds with a strict inequality:

$$E_0 < \langle \Psi' | \hat{H} \Psi' \rangle = \langle \Psi' | \hat{H}' \Psi' \rangle + \langle \Psi' | (\hat{V}-\hat{V}')\Psi' \rangle = E_0' + \int (v-v')\, n'\, dx.$$

The first step is the strict variational principle; the second writes $\hat{H} = \hat{H}' + (\hat{V}-\hat{V}')$; the third uses $\hat{H}'\Psi' = E_0'\Psi'$ and the rule that an external energy needs only the density (Section 13.3). Exchanging the roles of the two systems gives in the same way

$$E_0' < E_0 + \int (v'-v)\, n\, dx.$$

Adding the two inequalities and using $n=n'$, the two integrals cancel and we get $E_0 + E_0' < E_0 + E_0'$, which is false. Hence $n \neq n'$. $\square$

The universal functional (constrained search). For every density n that comes from some antisymmetric N-particle wave function define

$$F[n] = \min_{\Psi \to n} \langle \Psi | \hat{T} + \hat{W} | \Psi \rangle,$$

the smallest kinetic-plus-interaction energy among all wave functions with the density n (the arrow means "has the density"). F is **universal**: it does not depend on v. (We ASSUME that the minimum is attained.) **The variational principle for the density** then reads

$$E_0 = \min_n \Big(F[n] + \int v\, n\, dx \Big).$$

Proof: the variational principle over wave functions is $E_0 = \min_\Psi \langle \Psi | \hat{T} + \hat{W} + \hat{V} | \Psi \rangle$. Organise the minimisation in two steps, first over all Ψ with a given density n, then over n. In the first step $\langle \Psi | \hat{V} | \Psi \rangle = \int v\, n\, dx$ is the same for all these Ψ, so the inner minimum is $F[n] + \int v\, n\, dx$; the outer minimum over n is then the minimum over all Ψ, which is E_0. $\square$

These statements say that the energy is a functional of the density alone and that minimising it over densities gives the exact ground-state energy and density. They do not say what $F[n]$ is: finding good approximations to it is the work of DFT. On two sites the theorem can be seen at work: Notebook 13c (In [16], Figure 13c.7) shows that the exact density n_L on the left site rises strictly with the site-energy difference Δ, so each density belongs to exactly one Δ.

13.9 Functionals and functional derivatives

A **functional** assigns a number to a whole function: $F[n]$ takes the function $n(x)$ and returns one number. Examples: $\int n\, dx$, $\int n^2\, dx$, the Hartree energy $E_H[n]$.

Definition. Change n by a small amount $\epsilon\, \eta(x)$, where η is any fixed function and ϵ a small number. If

$$F[n + \epsilon\eta] = F[n] + \epsilon \int \frac{\delta F}{\delta n(x)}\, \eta(x)\, dx + O(\epsilon^2)$$

for every η, the function $\delta F / \delta n(x)$ is the **functional derivative** of F at n. On a grid with spacing h, F is an ordinary function of the numbers n_k, and $\delta F / \delta n(x_k) = (\partial F / \partial n_k)/h$: the functional derivative is the gradient, divided by the length that belongs to one point.

Examples, line by line. (i) For $A[n] = \int n^2\, dx$: $A[n + \epsilon\eta] = \int (n^2 + 2\epsilon\, n\eta + \epsilon^2 \eta^2)\, dx$ (multiply out the square), so the part of first order in ϵ is $\epsilon \int 2n\, \eta\, dx$ and $\delta A / \delta n = 2n$. (ii) For $G[n] = \int f(n(x))\, dx$ with a differentiable function f: Taylor's formula $f(n + \epsilon\eta) = f(n) + \epsilon f'(n)\, \eta + O(\epsilon^2)$ gives $\delta G / \delta n = f'(n)$. (iii) For $E_H = \frac{1}{2} \int\int n(x)\, w(x, x')\, n(x')\, dx\, dx'$, the first-order change is $\frac{\epsilon}{2} \int\int [\eta(x)\, w\, n(x') + n(x)\, w\, \eta(x')]$; renaming $x \leftrightarrow x'$ in the second term and using $w(x, x') = w(x', x)$ makes the two terms equal, so $\delta E_H / \delta n(x) = \int w(x, x')\, n(x')\, dx' = v_H(x)$, the Hartree potential. (iv) $\delta \int v\, n\, dx / \delta n = v$. Notebook 13a (In [15]) tests the definition directly on a grid: it changes the density by $\pm\epsilon\eta$ and compares the difference quotient with $\int (\delta F / \delta n)\, \eta\, dx$.

Minimising with a fixed particle number. To minimise $E[n]$ over densities with $\int n\, dx = N$, take a Lagrange multiplier μ (Section 13.6) and require $\delta E / \delta n(x) = \mu$ at every point where $n > 0$. The multiplier is the **chemical potential**: $\mu = dE_0 / dN$, the change of the lowest energy per added particle (where this derivative exists), by the meaning of a multiplier shown in Section 13.6.

13.10 The Kohn-Sham equations

The universal functional $F[n]$ contains the kinetic energy, the hardest part to write as an explicit formula of n (Notebook 13a shows the crude attempt, the Thomas-Fermi approximation). Kohn and Sham (1965) avoided the problem: compute the kinetic energy **exactly**, but for a fictitious system of non-interacting particles that has the same density.

The non-interacting reference system. For non-interacting particles the constrained search defines

$$T_s[n] = \min_{\Phi \to n} \langle \Phi | \hat{T} | \Phi \rangle,$$

the smallest kinetic energy of a non-interacting state with the density n; its minimiser is a single Slater determinant of orbitals ϕ_a with $n = \sum_a |\phi_a|^2$. The Kohn-Sham scheme ASSUMES that the density of interest is the ground-state density of some non-interacting system in some potential v_s (**non-interacting v-representability**). This is an assumption, not a theorem; on two sites it holds (Notebook 13c, In [16], finds the potential exactly).

Splitting the functional. Write

$$F[n] = T_s[n] + E_H[n] + E_{xc}[n],$$

which **defines** the **exchange-correlation energy** $E_{xc} = F - T_s - E_H$. It collects everything the easy pieces miss: the exchange energy of Section 13.5, the correlation energy of Section 13.7, and the difference between the true kinetic energy and T_s. Nothing has been approximated yet.

The equations. Minimise $E[n] = T_s[n] + \int v\, n\, dx + E_H[n] + E_{xc}[n]$ by varying the orbitals of the determinant, with Lagrange multipliers ϵ_a for their normalisation. Since $n = \sum_a |\phi_a|^2$, a change of ϕ_a^* at the point x changes n at x by $\phi_a(x)$ times that change; so every density functional $G[n]$ contributes $(\delta G/\delta n)\, \phi_a$ (chain rule), and the kinetic term contributes $-\frac{1}{2}\, \phi_a''$. Setting the derivative to zero gives the **Kohn-Sham equations**

$$\Big[-\frac{1}{2}\, \frac{d^2}{dx^2} + v_s(x) \Big] \phi_a = \epsilon_a\, \phi_a, \qquad n(x) = \sum_{a \in O} |\phi_a(x)|^2,$$

$$v_s = v + v_H + v_{xc}, \qquad v_{xc}(x) = \frac{\delta E_{xc}}{\delta n(x)},$$

with the N lowest orbitals occupied. The **Kohn-Sham potential** v_s is local: a multiplication by a function, simpler than the Fock operator. If E_{xc} were known exactly, these one-particle equations would give the exact ground-state density and energy of the interacting system.

Total energy, line by line. Multiply the equation of orbital a by ϕ_a^*, integrate and sum over the occupied orbitals:

$$\sum_{a \in O} \epsilon_a = \sum_a \int \phi_a^* \Big(-\frac{1}{2} \phi_a'' \Big) dx + \int v_s\, n\, dx = T_s + \int v_s\, n\, dx.$$

The first step uses normalisation, $\int |\phi_a|^2 = 1$, on the right and $\sum_a |\phi_a|^2 = n$ in the potential term; the second is the definition of T_s for the minimising determinant. Solving for T_s and inserting it into $E = T_s + \int v\, n + E_H + E_{xc}$, with $v_s = v + v_H + v_{xc}$ and $\int v_H\, n\, dx = 2E_H$:

$$E_0 = \sum_{a \in O} \epsilon_a - E_H[n] + E_{xc}[n] - \int v_{xc}\, n\, dx,$$

the Kohn-Sham analogue of the double-counting formula of Section 13.6.

Self-consistency. v_s depends on n, n on the orbitals, and the orbitals on v_s. The equations are solved by the loop: (1) guess a density; (2) build v_s; (3) solve the one-particle equations and occupy the N lowest orbitals; (4) form the new density; (5) if it equals the old one within a tolerance, stop; otherwise mix old and new (Section 13.21) and go back to (2).

What the orbitals mean. The Kohn-Sham orbitals and levels belong to the fictitious non-interacting system; only the density and the total energy are guaranteed to be physical. The levels are nevertheless useful: each is the derivative of the energy with respect to the occupation of its own orbital (Janak's theorem, Section 13.27).

The exact Kohn-Sham potential on two sites. Give the two sites the energies $\mp\Delta_s/2$ and put two non-interacting electrons into the lowest orbital of $\begin{pmatrix} -\Delta_s/2 & -t \\ -t & \Delta_s/2 \end{pmatrix}$. Section 13.21 derives that the lowest eigenvector of $\begin{pmatrix} a & -t \\ -t & b \end{pmatrix}$ has $|c_L|^2 = \frac{1}{2}(1 + (b-a)/\sqrt{(b-a)^2 + 4t^2})$; here $b - a = \Delta_s$, so with $x = n_L - 1 = 2|c_L|^2 - 1$, line by line:

$$\begin{aligned} x &= \frac{\Delta_s}{\sqrt{\Delta_s^2 + 4t^2}} \\ x^2\,(\Delta_s^2 + 4t^2) &= \Delta_s^2 \\ \Delta_s^2\,(1 - x^2) &= 4t^2 x^2 \\ \Delta_s &= \frac{2t\,x}{\sqrt{1 - x^2}}. \end{aligned}$$

The second line squares the first and multiplies by the denominator; the third collects the terms with Δ_s^2; the fourth takes the square root, with the sign of x (which is the sign of Δ_s by the first line). So for every exact density n_L between 0 and 2 there is exactly one Kohn-Sham site-energy difference Δ_s, and $\Delta_s - \Delta$ is the exact Hartree-exchange-correlation potential difference between the sites. Notebook 13c (In [16] to In [18], Figure 13c.8) computes it and splits it into a Hartree-exchange part and a correlation part.

13.11 Example: determinants, operators and the two-site model

Notebook 13c turns Sections 13.3 to 13.10 into numbers that can be checked: the three-point determinant of Section 13.3, the operator matrices and Wick's theorem of Section 13.4 (with the identity for $:S^2:$ that the Revision Kohn-Sham theory uses), the energy formula of Section 13.5, the exact and Hartree-Fock solutions of the two-site model of Section 13.7, and the Hohenberg-Kohn

map and the exact Kohn-Sham potential of Sections 13.8 and 13.10. Everything in it is exact finite-dimensional mathematics: it is PROVED by the derivations above and checked by the notebook to rounding. It ends with the line ALL 33 CHECKS PASSED (notebook 13c) and draws eight figures.

13.12 How to run Notebook 13c

Step 1. What this notebook does and what it needs.

Notebook 13c (Slater determinants, second quantization and the two-site model) is the file `Revision/textbook/notebooks/13c_two_site_model.ipynb` of the repository Dirac_claude. It evaluates Slater determinants of two fermions (on three points and in a box), builds the creation and annihilation operators of four orbitals as 16 x 16 matrices and checks their anticommutation relations, Wick's theorem for a determinant and for a thermal ensemble, the normal-ordered square of a one-body vertex and the energy formula of a determinant; it then solves the two-site model with two electrons exactly and in restricted and unrestricted Hartree-Fock (energies, correlation energy, double occupancy, the Hartree-Fock energy landscape), and shows the Hohenberg-Kohn map from the site-energy difference to the density and its exact Kohn-Sham inversion, with eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13c_two_site_model.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13c_two_site_model.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 13c_two_site_model.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/13c.captions.json`
- `Revision/textbook/figures/13c_1_slater_determinants.png`
- `Revision/textbook/figures/13c_2_operator_matrices.png`
- `Revision/textbook/figures/13c_3_density_matrices.png`
- `Revision/textbook/figures/13c_4_two_site_energies.png`
- `Revision/textbook/figures/13c_5_double_occupancy.png`
- `Revision/textbook/figures/13c_6_hf_landscape.png`
- `Revision/textbook/figures/13c_7_density_map.png`
- `Revision/textbook/figures/13c_8_ks_inversion.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13c_8_ks_inversion.png exists
ALL 33 CHECKS PASSED (notebook 13c)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/13c_two_site_model.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 13c_two_site_model.ipynb
python -m nbconvert --execute --inplace 13c_two_site_model.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

13.13 Notebook 13c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/13c_two_site_model.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook turns the first half of many-body quantum mechanics into numbers that
can be checked. It

- evaluates the Slater determinant of two fermions on three points (every value,
  the normalisation and the density) and draws a two-fermion determinant in a box;
- builds the creation and annihilation operators of four orbitals as $16 \times 16$
  matrices and checks the anticommutation relations;
- checks Wick's theorem (every expectation value of a determinant is made of its
  density matrix) for a determinant and for a thermal ensemble, and the formula for
  the normal-ordered square of a one-body vertex that the Revision Kohn-Sham theory
  uses;
- checks the energy formula of a determinant (direct minus exchange terms);
- solves the model of two electrons on two sites exactly, and in restricted and
  unrestricted Hartree-Fock, and compares energies, correlation energy and double
  occupancy for all strengths of the repulsion;
- shows the Hohenberg-Kohn theorem at work on two sites (the density determines the
  site-energy difference) and finds the exact Kohn-Sham potential by inverting the
  density;
- draws eight teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 13c (Slater determinants, second quantization and the two-site model) is the
file `Revision/textbook/notebooks/13c_two_site_model.ipynb` of the repository
Dirac_claude. It evaluates Slater determinants of two fermions (on three points and in a
box), builds the creation and annihilation operators of four orbitals as 16 x 16 matrices
and checks their anticommutation relations, Wick's theorem for a determinant and for a
thermal ensemble, the normal-ordered square of a one-body vertex and the energy formula
of a determinant; it then solves the two-site model with two electrons exactly and in
```

restricted and unrestricted Hartree-Fock (energies, correlation energy, double occupancy, the Hartree-Fock energy landscape), and shows the Hohenberg-Kohn map from the site-energy difference to the density and its exact Kohn-Sham inversion, with eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 13c_two_site_model.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 13c_two_site_model.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp]

Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 13c_two_site_model.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/13c.captions.json‘
- ‘Revision/textbook/figures/13c_1_slater_determinants.png‘
- ‘Revision/textbook/figures/13c_2_operator_matrices.png‘
- ‘Revision/textbook/figures/13c_3_density_matrices.png‘
- ‘Revision/textbook/figures/13c_4_two_site_energies.png‘
- ‘Revision/textbook/figures/13c_5_double_occupancy.png‘
- ‘Revision/textbook/figures/13c_6_hf_landscape.png‘
- ‘Revision/textbook/figures/13c_7_density_map.png‘
- ‘Revision/textbook/figures/13c_8_ks_inversion.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13c_8_ks_inversion.png exists
ALL 33 CHECKS PASSED (notebook 13c)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/13c_two_site_model.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use

  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 13c_two_site_model.ipynb
python -m nbconvert --execute --inplace 13c_two_site_model.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 13c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 13c (Slater determinants, second quantization and the two-site model) is the
# file Revision/textbook/notebooks/13c_two_site_model.ipynb of the repository
# Dirac_claude. It evaluates Slater determinants of two fermions (on three points and in
# a box), builds the creation and annihilation operators of four orbitals as 16 x 16
# matrices and checks their anticommutation relations, Wick's theorem for a determinant
# and for a thermal ensemble, the normal-ordered square of a one-body vertex and the
# energy formula of a determinant; it then solves the two-site model with two electrons
# exactly and in restricted and unrestricted Hartree-Fock (energies, correlation energy,
# double occupancy, the Hartree-Fock energy landscape), and shows the Hohenberg-Kohn map
# from the site-energy difference to the density and its exact Kohn-Sham inversion, with
# eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy and matplotlib; the other packages run Jupyter, the program that shows
# and runs notebooks. It does not need Rust.
```

```
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
```

```
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
```

```
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 13c_two_site_model.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 13c_two_site_model.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 13c_two_site_model.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/13c.captions.json
# - Revision/textbook/figures/13c_1_slater_determinants.png
# - Revision/textbook/figures/13c_2_operator_matrices.png
# - Revision/textbook/figures/13c_3_density_matrices.png
# - Revision/textbook/figures/13c_4_two_site_energies.png
# - Revision/textbook/figures/13c_5_double_occupancy.png
# - Revision/textbook/figures/13c_6_hf_landscape.png
# - Revision/textbook/figures/13c_7_density_map.png
# - Revision/textbook/figures/13c_8_ks_inversion.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
```

```
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 13c_8_ks_inversion.png exists
#     ALL 33 CHECKS PASSED (notebook 13c)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/13c_two_site_model.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 13c_two_site_model.ipynb
#     python -m nbconvert --execute --inplace 13c_two_site_model.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
```

```
# ================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "13c"  # this notebook: chapter 13, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
```

```
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 13c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Orbital**: a one-particle state; here either a function on a few points or one
  of a finite list of basis states.
- **Slater determinant**: the antisymmetric state of $N$ fermions built from $N$
  orthonormal orbitals, $\Phi = \det[\phi_a(x_i)]/\sqrt{N!}$.
- **Antisymmetric**: changing sign when two particles are exchanged.
- **Occupation-number state** $|n_0 n_1 n_2 n_3\rangle$: the determinant that uses
  the orbitals $p$ with $n_p = 1$; there are $2^4 = 16$ of them for four orbitals.
- **Creation operator** $a_p^\dagger$, **annihilation operator** $a_p$: add or
  remove a fermion in orbital $p$, with the sign $(-1)^{\nu_p}$, where $\nu_p$ is the
  number of occupied orbitals before $p$; on the 16 occupation-number states they
  are $16 \times 16$ matrices.
- **Anticommutator** $\{A, B\} = AB + BA$.
- **Density matrix** $\rho_{qp} = \langle a_p^\dagger a_q\rangle$; for a determinant
  its eigenvalues are 0 and 1, for a thermal ensemble the occupation probabilities.
- **Wick's theorem**: $\langle a_p^\dagger a_q^\dagger a_s a_r\rangle =
  \rho_{rp}\rho_{sq} - \rho_{sp}\rho_{rq}$ for determinants and thermal ensembles of
  non-interacting fermions.
- **Two-site model**: two places L and R, one orbital each, two labels (up, down);
  hopping $t$ between the sites, repulsion $U$ when both electrons sit on the same
  site.
- **Hartree-Fock (HF)**: the best single determinant; **restricted** (both labels in
  the same orbital) or **unrestricted** (each label its own orbital).
- **Correlation energy**: the exact ground-state energy minus the Hartree-Fock
  energy (never positive).
- **Double occupancy**: the probability that both electrons sit on the same site.
- **Hohenberg-Kohn theorem**: the ground-state density determines the external
  potential (up to a constant); **Kohn-Sham inversion**: finding the potential of
  NON-interacting electrons that gives the same density.
```

Text cell:

```
## 4. The physical and mathematical situation

**Determinants.** For fermion 1 at the position $r_1$ and fermion 2 at $r_2$, in
the orthonormal orbitals $\phi_a$, $\phi_b$,
$\Phi(r_1, r_2) = [\phi_a(r_1)\phi_b(r_2) - \phi_b(r_1)\phi_a(r_2)]/\sqrt2$. It is
antisymmetric, vanishes when $r_1 = r_2$, is normalised, and its density is
$|\phi_a|^2 + |\phi_b|^2$. (The letters $r_1$, $r_2$ name the positions of the two
particles; they are not coordinates of spacetime.)

**Operators.** With four orbitals the occupation-number states are the 16 numbers
$0, \dots, 15$ written in binary: bit $p$ of the number is $n_p$. The matrix of
$a_p$ maps a state with $n_p = 1$ to the state with $n_p = 0$, times
$(-1)^{\nu_p}$; $a_p^\dagger$ is its transpose. The anticommutation relations
$\{a_p, a_q^\dagger\} = \delta_{pq}$, $\{a_p, a_q\} = 0$ follow.

**The two-site model.** The four orbitals are L up, R up, L down, R down (numbers 0,
1, 2, 3). With site energies $-\Delta/2$ (L) and $+\Delta/2$ (R),

$$\hat H = -t\sum_{\sigma}(a_{L\sigma}^\dagger a_{R\sigma} + a_{R\sigma}^\dagger
a_{L\sigma}) + U(\hat n_{L\uparrow}\hat n_{L\downarrow} + \hat n_{R\uparrow}\hat
n_{R\downarrow}) - \tfrac{\Delta}{2}\hat n_L + \tfrac{\Delta}{2}\hat n_R .$$

For $\Delta = 0$ the exact ground-state energy of two electrons with opposite labels
is $E_0 = \tfrac12(U - \sqrt{U^2 + 16t^2})$.

**Hartree-Fock.** A determinant with the up orbital $(\cos\alpha, \sin\alpha)$ and
the down orbital $(\cos\beta, \sin\beta)$ on (L, R) has the energy
$E(\alpha, \beta) = -t(\sin 2\alpha + \sin 2\beta) + \tfrac{U}{2}(1 + \cos 2\alpha
\cos 2\beta)$ (hopping energy of each orbital, plus $U$ times the double
occupancy). Restricted HF ($\alpha = \beta = \pi/4$) gives $-2t + U/2$; the
minimum over all $\alpha, \beta$ is $-2t + U/2$ for $U \le 2t$ and $-2t^2/U$ for
$U > 2t$ (unrestricted).

**Units.** All energies are in units of the hopping $t$ ($t = 1$).

**Status.** Exact finite-dimensional mathematics (PROVED: every statement follows
from the definitions; this notebook checks each one to rounding). No Revision number
is reproduced; the Wick identity for the normal-ordered square is the one of the
Revision check hf_wick_contraction.
```

Text cell:

```
## 5. A Slater determinant on three points

Space is three points $r = 0, 1, 2$; the orbitals are $\phi_0 = (1, 0, 0)$ and
$\phi_1 = (0, 1, 1)/\sqrt2$. The next cell builds the $3 \times 3$ table of
$\Phi(r_1, r_2)$ (row $r_1$, column $r_2$), checks its values, its antisymmetry, its
normalisation $\sum |\Phi|^2 = 1$ and its density
$n(r) = 2\sum_{r_2}|\Phi(r, r_2)|^2 = (1, \tfrac12, \tfrac12)$.
```

In [2]:

```
import itertools  # loops over all combinations of indices

import numpy as np  # arrays, matrices and linear algebra

phi0 = np.array([1.0, 0.0, 0.0])
phi1 = np.array([0.0, 1.0, 1.0]) / np.sqrt(2.0)
# Phi[r1, r2] = (phi0(r1) phi1(r2) - phi1(r1) phi0(r2)) / sqrt(2)
Phi = (np.outer(phi0, phi1) - np.outer(phi1, phi0)) / np.sqrt(2.0)
for r1 in range(3):
    say("Phi(%d, r2) for r2 = 0, 1, 2: %s" % (r1, np.array2string(
        Phi[r1], precision=6, floatmode="fixed", suppress_small=True)))
density_3 = 2.0 * np.sum(Phi ** 2, axis=1)  # n(r) = N sum_r2 |Phi(r, r2)|^2
check(abs(Phi[0, 1] - 0.5) < 1e-15 and abs(Phi[1, 2]) < 1e-15,
      "Phi(0,1) = 1/2 and Phi(1,2) = 0")
check(np.allclose(Phi, -Phi.T, atol=1e-15) and np.allclose(np.diag(Phi), 0.0),
      "Phi is antisymmetric and zero on the diagonal (Pauli)")
check(abs(np.sum(Phi ** 2) - 1.0) < 1e-15
      and np.allclose(density_3, [1.0, 0.5, 0.5], atol=1e-15)
      and np.allclose(density_3, phi0 ** 2 + phi1 ** 2, atol=1e-15),
      "Phi is normalised and its density is (1, 1/2, 1/2) = phi0^2 + phi1^2")
```

Out [2]:

```
Phi(0, r2) for r2 = 0, 1, 2: [0.000000 0.500000 0.500000]
Phi(1, r2) for r2 = 0, 1, 2: [-0.500000  0.000000  0.000000]
Phi(2, r2) for r2 = 0, 1, 2: [-0.500000  0.000000  0.000000]
PASS Phi(0,1) = 1/2 and Phi(1,2) = 0
PASS Phi is antisymmetric and zero on the diagonal (Pauli)
PASS Phi is normalised and its density is (1, 1/2, 1/2) = phi0^2 + phi1^2
```

Text cell:

```
The next cell draws this table and, next to it, the determinant of the two lowest
orbitals $\sqrt2\sin(\pi x)$ and $\sqrt2\sin(2\pi x)$ of a particle in the box
$0 < x < 1$ as a heat map over the positions $(r_1, r_2)$ of the two fermions.
```

In [3]:

```
x_box = np.linspace(0.0, 1.0, 101)
chi1 = np.sqrt(2.0) * np.sin(np.pi * x_box)
chi2 = np.sqrt(2.0) * np.sin(2.0 * np.pi * x_box)
Phi_box = (np.outer(chi1, chi2) - np.outer(chi2, chi1)) / np.sqrt(2.0)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
image = left.imshow(Phi, origin="lower", cmap="coolwarm", vmin=-0.6, vmax=0.6)
for r1, r2 in itertools.product(range(3), repeat=2):  # write every value in
    left.text(r2, r1, f"{Phi[r1, r2]:+.2f}", ha="center", va="center")
left.set_xticks([0, 1, 2])
left.set_yticks([0, 1, 2])
left.set_xlabel("position $r_2$ of fermion 2")
left.set_ylabel("position $r_1$ of fermion 1")
left.set_title("three points")
```

```
image = right.imshow(Phi_box, origin="lower", extent=(0, 1, 0, 1),
                     cmap="coolwarm")
right.plot([0, 1], [0, 1], "k--", lw=0.8)
right.set_xlabel("position $r_2$ of fermion 2")
right.set_ylabel("position $r_1$ of fermion 1")
right.set_title("two fermions in a box")
fig.colorbar(image, ax=right, shrink=0.85)
save_figure(fig, "slater_determinants",
            "Two Slater determinants of two fermions as heat maps (red positive, "
            "blue negative): left, $\\Phi(r_1, r_2)$ on three points for the "
            "orbitals $(1,0,0)$ and $(0,1,1)/\\sqrt2$ with its values written in; "
            "right, the determinant of the two lowest orbitals of a box "
            "$0 < x < 1$, against the positions $r_1$ (vertical) and $r_2$ "
            "(horizontal) of the two fermions. Both change sign when $r_1$ and "
            "$r_2$ are exchanged (mirror in the dashed diagonal) and vanish on the "
            "diagonal: two identical fermions are never at the same place.")
check(np.allclose(Phi_box, -Phi_box.T, atol=1e-14), "the box determinant is "
      "antisymmetric")
```

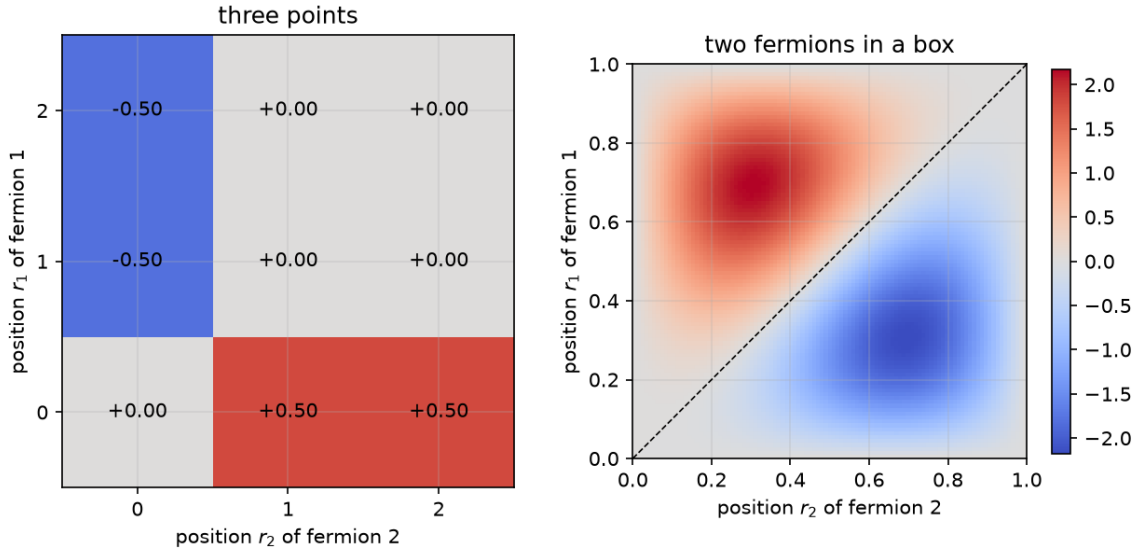

Figure 344: Two Slater determinants of two fermions as heat maps (red positive, blue negative): left, $\Phi(r_1, r_2)$ on three points for the orbitals $(1, 0, 0)$ and $(0, 1, 1)/\sqrt{2}$ with its values written in; right, the determinant of the two lowest orbitals of a box $0 < x < 1$, against the positions r_1 (vertical) and r_2 (horizontal) of the two fermions. Both change sign when r_1 and r_2 are exchanged (mirror in the dashed diagonal) and vanish on the diagonal: two identical fermions are never at the same place. (Notebook 13c, figure 1.)

Out [3]:

```
Figure 13c.1 saved as Revision/textbook/figures/13c_1_slater_determinants.png
PASS the box determinant is antisymmetric
```

Text cell:

```
## 6. Creation and annihilation operators as matrices

The next cell builds the matrices $a_0, \dots, a_3$ on the 16 occupation-number
states. The state number $s$ has $n_p = $ bit $p$ of $s$ (in Python `(s >> p) & 1`);
removing the fermion of orbital $p$ gives the state number `s ^ (1 << p)` (bit $p$
switched off), with the sign $(-1)^{\nu_p}$, $\nu_p = n_0 + \dots + n_{p-1}$. Then
it checks all anticommutation relations.
```

In [4]:

```
M = 4  # orbitals
DIM = 2 ** M  # occupation-number states


def bit(state, p):
    """The occupation n_p (0 or 1) of orbital p in the state number state."""
    return (state >> p) & 1


a = []  # a[p] is the 16 x 16 matrix of the annihilation operator a_p
for p in range(M):
    matrix = np.zeros((DIM, DIM))
    for state in range(DIM):
        if bit(state, p) == 1:
            nu = sum(bit(state, q) for q in range(p))  # occupied before p
            matrix[state ^ (1 << p), state] = (-1) ** nu
    a.append(matrix)
a_dag = [matrix.T for matrix in a]  # creation operators: the transposes
identity = np.eye(DIM)
worst = 0.0
for p, q in itertools.product(range(M), repeat=2):
    worst = max(worst,
                np.abs(a[p] @ a_dag[q] + a_dag[q] @ a[p] - (p == q) * identity).max(),
                np.abs(a[p] @ a[q] + a[q] @ a[p]).max(),
                np.abs(a_dag[p] @ a_dag[q] + a_dag[q] @ a_dag[p]).max())
check(worst == 0.0, "{a_p, a_q^dag} = delta_pq and {a_p, a_q} = 0 exactly")
number = sum(a_dag[p] @ a[p] for p in range(M))  # the particle-number operator
check(np.array_equal(np.diag(number), [bin(s).count("1") for s in range(DIM)]),
      "the number operator counts the occupied orbitals of every state")
```

Out [4]:

```
PASS {a_p, a_q^dag} = delta_pq and {a_p, a_q} = 0 exactly
PASS the number operator counts the occupied orbitals of every state
```

Text cell:

```
The next cell draws two of these matrices. Every column of $a_1^\dagger$ has at most
one nonzero entry ($\pm 1$): the operator moves a state to exactly one other state,
and the sign records how many occupied orbitals it had to pass.
```

In [5]:

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, p in zip(axes, (1, 3)):
    image = ax.imshow(a_dag[p], cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(f"creation operator $a_{p}^\\dagger$")
    ax.set_xlabel("state number before (column)")
    ax.set_ylabel("state number after (row)")
fig.colorbar(image, ax=axes, shrink=0.8)
save_figure(fig, "operator_matrices",
            "The creation operators $a_1^\\dagger$ (left) and $a_3^\\dagger$ "
            "(right) of four orbitals as $16 \\times 16$ matrices on the "
            "occupation-number states $0, \\dots, 15$ (bit $p$ of the state number "
            "is $n_p$); red $+1$, blue $-1$, grey 0. A column has one entry when "
            "orbital $p$ is empty in that state and none when it is occupied "
            "(Pauli); the sign is $(-1)^{\\nu_p}$, the parity of the number of "
            "occupied orbitals before $p$.")
```

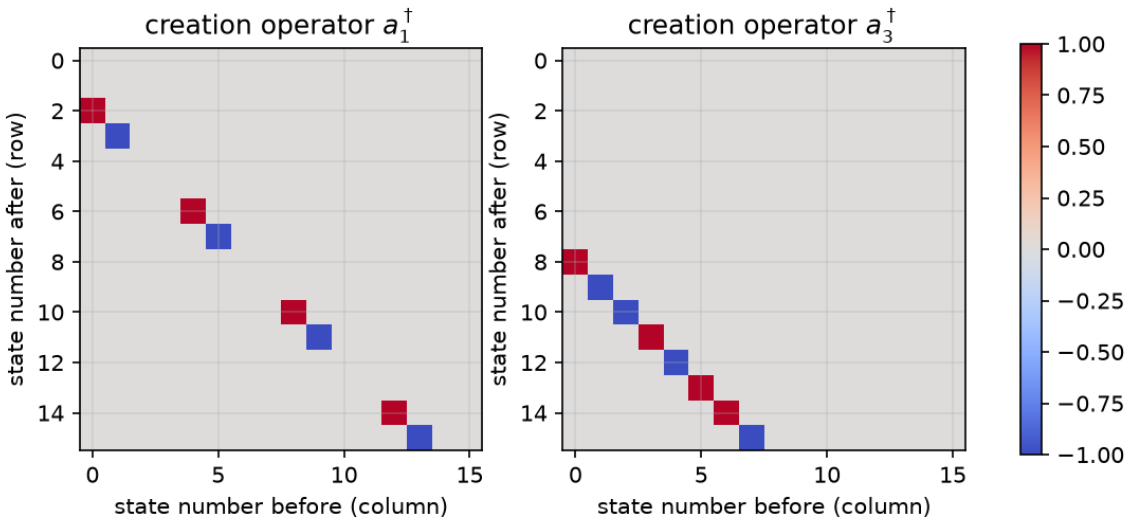

Figure 345: The creation operators $a_1^\dagger$ (left) and $a_3^\dagger$ (right) of four orbitals as 16×16 matrices on the occupation-number states $0, \dots, 15$ (bit p of the state number is n_p); red $+1$, blue -1, grey 0. A column has one entry when orbital p is empty in that state and none when it is occupied (Pauli); the sign is $(-1)^{\nu_p}$, the parity of the number of occupied orbitals before p. (Notebook 13c, figure 2.)

Out [5]:

```
Figure 13c.2 saved as Revision/textbook/figures/13c_2_operator_matrices.png
```

Text cell:

```
## 7. Wick's theorem for a determinant and for a thermal ensemble

The next cell mixes the four orbitals with a random unitary $4 \times 4$ matrix $U$
```

(from the QR factorisation of a random complex matrix with a fixed seed), builds the new creation operators $a_p'^\dagger = \sum_k U_{kp}a_k^\dagger$, and the determinant $|\Phi\rangle = a_0'^\dagger a_1'^\dagger|0\rangle$ that occupies the new orbitals 0 and 1. Its density matrix is $\rho = \sum_{a = 0, 1} U_{:,a}U_{:,a}^\dagger$. It then checks $\langle a_p^\dagger a_q\rangle = \rho_{qp}$ and all 256 four-operator expectation values against Wick's formula.

In [6]:

```
rng = np.random.default_rng(12345)  # fixed seed: the same numbers every run
U_mix = np.linalg.qr(rng.normal(size=(M, M)) + 1j * rng.normal(size=(M, M)))[0]
a_dag_new = [sum(U_mix[k, p] * a_dag[k] for k in range(M)) for p in range(M)]
vacuum = np.zeros(DIM)
vacuum[0] = 1.0  # state number 0: no fermion at all
Phi_state = a_dag_new[0] @ a_dag_new[1] @ vacuum
rho_det = sum(np.outer(U_mix[:, k], U_mix[:, k].conj()) for k in (0, 1))


def wick_errors(expect, rho):
    """Largest errors of <a+_p a_q> = rho_qp and of the four-operator formula."""
    two = max(abs(expect(a_dag[p] @ a[q]) - rho[q, p])
              for p, q in itertools.product(range(M), repeat=2))
    four = max(abs(expect(a_dag[p] @ a_dag[q] @ a[s] @ a[r])
                   - (rho[r, p] * rho[s, q] - rho[s, p] * rho[r, q]))
               for p, q, r, s in itertools.product(range(M), repeat=4))
    return two, four


def in_determinant(operator):
    """<Phi| operator |Phi> for the determinant Phi_state."""
    return np.vdot(Phi_state, operator @ Phi_state)


check(abs(np.vdot(Phi_state, Phi_state) - 1.0) < 1e-14, "the determinant is "
      "normalised")
two, four = wick_errors(in_determinant, rho_det)  # the largest errors (rounding)
check(two < 1e-14 and four < 1e-14,
      "Wick's theorem holds for the determinant (all 16 + 256 values, to 1e-14)")
check(np.allclose(rho_det @ rho_det, rho_det, atol=1e-14)
      and abs(np.trace(rho_det).real - 2.0) < 1e-14,
      "the density matrix of a determinant obeys rho^2 = rho, Tr rho = 2")
```

Out [6]:

```
PASS the determinant is normalised
PASS Wick's theorem holds for the determinant (all 16 + 256 values, to 1e-14)
PASS the density matrix of a determinant obeys rho^2 = rho, Tr rho = 2
```

Text cell:

```
For a **thermal ensemble** of non-interacting fermions with the orbital energies
$\epsilon_p$ in the mixed orbitals, the density operator on the 16 states is
```

```
$e^{-(\hat H_0 - \mu\hat N)/T}/Z$ with $\hat H_0 = \sum_p \epsilon_p
a_p'^\dagger a_p'$. The next cell builds it (the exponential of a Hermitian matrix
through its eigenvalues), and checks Wick's theorem with
$\rho = \sum_p f_p U_{:,p}U_{:,p}^\dagger$ and the Fermi-Dirac occupations
$f_p = 1/(e^{(\epsilon_p - \mu)/T} + 1)$.
```

In [7]:

```
energies = np.array([-1.0, -0.3, 0.4, 1.2])  # orbital energies (units of t)
MU, TEMPERATURE = 0.0, 0.5
H0 = sum(energies[p] * a_dag_new[p] @ a_dag_new[p].conj().T for p in range(M))
K = H0 - MU * number  # H0 - mu N, a Hermitian 16 x 16 matrix
k_values, k_vectors = np.linalg.eigh(K)
weights = np.exp(-(k_values - k_values.min()) / TEMPERATURE)  # shifted: no overflow
gibbs = (k_vectors * (weights / weights.sum())) @ k_vectors.conj().T
f = 1.0 / (np.exp((energies - MU) / TEMPERATURE) + 1.0)  # Fermi-Dirac
rho_thermal = sum(f[p] * np.outer(U_mix[:, p], U_mix[:, p].conj()) for p in range(M))
two, four = wick_errors(lambda operator: np.trace(gibbs @ operator), rho_thermal)
check(two < 1e-13 and four < 1e-13,
      "Wick's theorem holds for the thermal ensemble of non-interacting fermions "
      "(to 1e-13)")
```

Out [7]:

```
PASS Wick's theorem holds for the thermal ensemble of non-interacting fermions (to 1e-13)
```

Text cell:

```
The interaction of the dirac16complex Kohn-Sham model is the square of a one-body
quantity, $S = \sum_{pq}V_{pq}a_p^\dagger a_q$ (there with the vertex $V = C$ on 16
spinor components). Its normal-ordered square
$:S^2: = \sum V_{pq}V_{rs}\,a_p^\dagger a_r^\dagger a_s a_q$ has, by Wick's theorem,
$\langle :S^2: \rangle = (\mathrm{Tr}\,V\rho)^2 - \mathrm{Tr}(V\rho V\rho)$: a Hartree
part and an exchange part. The Revision record verified this identity on a two-mode
state; the next cell checks it on the four orbitals for a random Hermitian vertex,
in the determinant and in the thermal ensemble.
```

In [8]:

```
raw = rng.normal(size=(M, M)) + 1j * rng.normal(size=(M, M))
V = (raw + raw.conj().T) / 2.0  # a random Hermitian vertex
S_square = sum(V[p, q] * V[r, s] * a_dag[p] @ a_dag[r] @ a[s] @ a[q]
               for p, q, r, s in itertools.product(range(M), repeat=4))
for label, expect, rho in (("determinant", in_determinant, rho_det),
                           ("thermal", lambda o: np.trace(gibbs @ o), rho_thermal)):
    lhs = expect(S_square)
    rhs = np.trace(V @ rho) ** 2 - np.trace(V @ rho @ V @ rho)
    say(f"{label}: <:S^2:> = {lhs.real:.10f}, Hartree - exchange = {rhs.real:.10f}")
    check(abs(lhs - rhs) < 1e-12,
          f"<:S^2:> = (Tr V rho)^2 - Tr(V rho V rho) in the {label} state",
          record="Revision/kohn_sham/reports/ks-theory-python.json, check "
                 "hf_wick_contraction")
```

Out [8]:

```
determinant: <:S^2:> = -1.0717850309, Hartree - exchange = -1.0717850309
PASS <:S^2:> = (Tr V rho)^2 - Tr(V rho V rho) in the determinant state
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    hf_wick_contraction
thermal: <:S^2:> = -1.9027816067, Hartree - exchange = -1.9027816067
PASS <:S^2:> = (Tr V rho)^2 - Tr(V rho V rho) in the thermal state
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    hf_wick_contraction
```

Text cell:

```
The next cell draws the density matrix of the determinant (its absolute values) and
the eigenvalues of both density matrices: a determinant has the occupations 1, 1, 0,
0; the thermal ensemble has the Fermi-Dirac occupations $f_p$.
```

In [9]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
image = left.imshow(np.abs(rho_det), cmap="viridis", vmin=0.0)
left.set_title("$|\\rho_{qp}|$ of the determinant")
left.set_xticks(range(M))
left.set_yticks(range(M))
left.set_xlabel("$p$")
left.set_ylabel("$q$")
fig.colorbar(image, ax=left, shrink=0.85)
occupations_det = np.sort(np.linalg.eigvalsh(rho_det))[::-1]
occupations_th = np.sort(np.linalg.eigvalsh(rho_thermal))[::-1]
positions = np.arange(M)
right.bar(positions - 0.2, occupations_det, width=0.4, label="determinant")
right.bar(positions + 0.2, occupations_th, width=0.4, label="thermal, $T = 0.5$")
right.set_xticks(positions)
right.set_xlabel("eigenvalue number")
right.set_ylabel("occupation (eigenvalue of $\\rho$)")
right.legend()
save_figure(fig, "density_matrices",
            "Left: the absolute values of the density matrix "
            "$\\rho_{qp} = \\langle a_p^\\dagger a_q \\rangle$ of a determinant of "
            "two fermions in four randomly mixed orbitals (heat map; $p$ and $q$ "
            "label the four basis orbitals). Right: the eigenvalues of the density "
            "matrices of this determinant (exactly 1, 1, 0, 0) and of a thermal "
            "ensemble of non-interacting fermions at $T = 0.5$ (the Fermi-Dirac "
            "occupations, between 0 and 1); in both cases Wick's theorem holds.")
check(np.allclose(occupations_det, [1, 1, 0, 0], atol=1e-14)
      and np.allclose(occupations_th, np.sort(f)[::-1], atol=1e-14),
      "the occupations are 1, 1, 0, 0 and the Fermi-Dirac numbers")
```

Out [9]:

```
Figure 13c.3 saved as Revision/textbook/figures/13c_3_density_matrices.png
PASS the occupations are 1, 1, 0, 0 and the Fermi-Dirac numbers
```

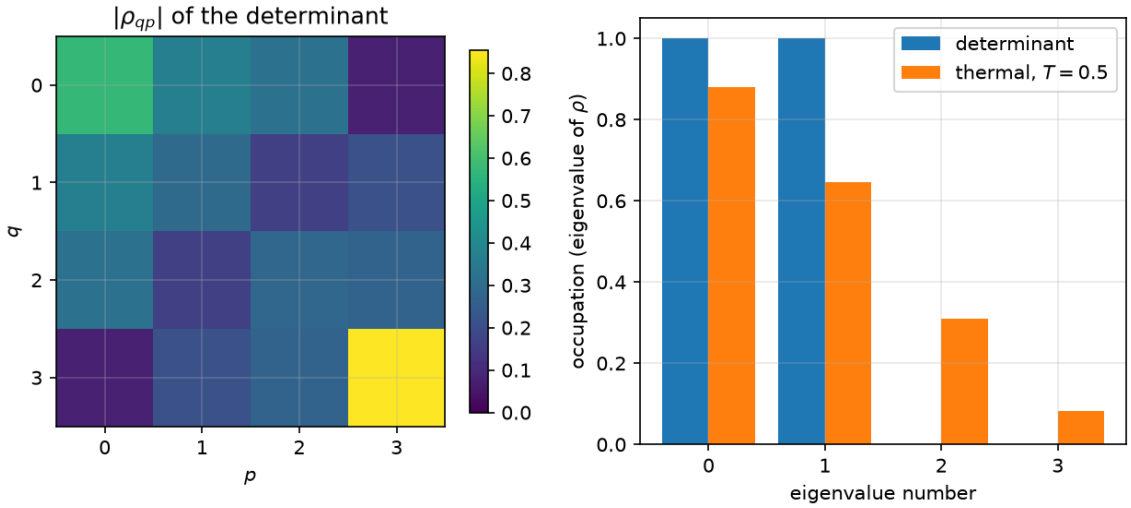

Figure 346: Left: the absolute values of the density matrix $\rho_{qp} = \langle a_p^\dagger a_q \rangle$ of a determinant of two fermions in four randomly mixed orbitals (heat map; p and q label the four basis orbitals). Right: the eigenvalues of the density matrices of this determinant (exactly 1, 1, 0, 0) and of a thermal ensemble of non-interacting fermions at $T = 0.5$ (the Fermi-Dirac occupations, between 0 and 1); in both cases Wick's theorem holds. (Notebook 13c, figure 3.)

Text cell:

```
## 8. The energy of a determinant: direct minus exchange

With a one-body matrix $h_{pq}$ and two-body integrals $w_{pqrs}$ the Hamiltonian is
$\hat H = \sum h_{pq}a_p^\dagger a_q + \tfrac12\sum w_{pqrs}a_p^\dagger a_q^\dagger
a_s a_r$. For the determinant of the basis orbitals 0 and 1 its energy must be
$\sum_{a} h_{aa} + \tfrac12\sum_{a,b}(w_{abab} - w_{abba})$ over $a, b \in \{0,
1\}$. The next cell makes the integrals from four orthonormal orbitals on a grid of
six points and a random symmetric interaction $W(x, x')$ (so that they have the
symmetries of a real interaction) and compares.
```

In [10]:

```
chi = np.linalg.qr(rng.normal(size=(6, M)))[0]  # four orthonormal orbitals on 6 points
W_raw = rng.normal(size=(6, 6))
W_pair = (W_raw + W_raw.T) / 2.0  # a symmetric interaction W(x, x')
h_raw = rng.normal(size=(M, M))
h_one = (h_raw + h_raw.T) / 2.0  # a symmetric one-body matrix
# w[p,q,r,s] = sum_{x,x'} chi_p(x) chi_q(x') W(x,x') chi_r(x) chi_s(x')
w = np.einsum("xp,yq,xy,xr,ys->pqrs", chi, chi, W_pair, chi, chi)
H_many = sum(h_one[p, q] * a_dag[p] @ a[q] for p, q in itertools.product(range(M),
                                                                         repeat=2))
H_many = H_many + 0.5 * sum(w[p, q, r, s] * a_dag[p] @ a_dag[q] @ a[s] @ a[r]
                            for p, q, r, s in itertools.product(range(M), repeat=4))
basis_det = a_dag[0] @ a_dag[1] @ vacuum  # occupies the basis orbitals 0 and 1
direct = basis_det @ H_many @ basis_det
formula = sum(h_one[k, k] for k in (0, 1)) + 0.5 * sum(
```

```
    w[k, l, k, l] - w[k, l, l, k] for k in (0, 1) for l in (0, 1))
say(f"<Phi|H|Phi> = {direct:.10f};  sum h_aa + (1/2) sum (w_abab - w_abba) = "
    f"{formula:.10f}")
check(abs(direct - formula) < 1e-12,
      "the energy of a determinant is one-body + direct - exchange")
```

Out [10]:

```
<Phi|H|Phi> = -1.5322375649;  sum h_aa + (1/2) sum (w_abab - w_abba) = -1.5322375649
PASS the energy of a determinant is one-body + direct - exchange
```

Text cell:

```
## 9. The two-site model: exact solution

The next cell builds the Hamiltonian of section 4 from the operator matrices (orbital
0 = L up, 1 = R up, 2 = L down, 3 = R down), keeps the four states with one up and
one down electron, and diagonalises it. It checks the closed form of $E_0$ and
computes the double occupancy $\langle \hat n_{L\uparrow}\hat n_{L\downarrow} +
\hat n_{R\uparrow}\hat n_{R\downarrow}\rangle$ of the ground state.
```

In [11]:

```
n_op = [a_dag[p] @ a[p] for p in range(M)]  # occupation operators
DOUBLE = n_op[0] @ n_op[2] + n_op[1] @ n_op[3]  # both electrons on one site
# The states with exactly one up (orbital 0 or 1) and one down (2 or 3) electron:
SECTOR = [s for s in range(DIM) if bit(s, 0) + bit(s, 1) == 1
          and bit(s, 2) + bit(s, 3) == 1]


def two_site(t, U, delta=0.0):
    """Ground-state energy, double occupancy and n_L of the two-site model."""
    H = -t * (a_dag[0] @ a[1] + a_dag[1] @ a[0] + a_dag[2] @ a[3] + a_dag[3] @ a[2])
    H = H + U * DOUBLE - 0.5 * delta * (n_op[0] + n_op[2]) \
        + 0.5 * delta * (n_op[1] + n_op[3])
    block = H[np.ix_(SECTOR, SECTOR)]  # the 4 x 4 block of the sector
    values, vectors = np.linalg.eigh(block)
    ground = vectors[:, 0]
    occupancy = ground @ DOUBLE[np.ix_(SECTOR, SECTOR)] @ ground
    n_left = ground @ (n_op[0] + n_op[2])[np.ix_(SECTOR, SECTOR)] @ ground
    return values[0], occupancy, n_left


say(f"the sector holds the states {SECTOR}")
for U in (2.0, 4.0):
    E0, occupancy, n_left = two_site(1.0, U)
    report(f"U = {U:.0f}: exact E_0", f"{E0:.6f}")
    report(f"U = {U:.0f}: exact double occupancy", f"{occupancy:.6f}")
    check(abs(E0 - 0.5 * (U - np.sqrt(U ** 2 + 16.0))) < 1e-12
          and abs(n_left - 1.0) < 1e-12,
          f"U = {U:.0f}: E_0 = (U - sqrt(U^2 + 16 t^2))/2 and n_L = 1")
```

Out [11]:

```
the sector holds the states [5, 6, 9, 10]
RESULT U = 2: exact E_0 = -1.236068
RESULT U = 2: exact double occupancy = 0.276393
PASS U = 2: E_0 = (U - sqrt(U^2 + 16 t^2))/2 and n_L = 1
RESULT U = 4: exact E_0 = -0.828427
RESULT U = 4: exact double occupancy = 0.146447
PASS U = 4: E_0 = (U - sqrt(U^2 + 16 t^2))/2 and n_L = 1
```

Text cell:

```
## 10. Hartree-Fock: restricted and unrestricted

The next cell first checks the energy formula $E(\alpha, \beta)$ of section 4
against the Fock-space expectation value of the determinant
$(\cos\alpha\,a_{L\uparrow}^\dagger + \sin\alpha\,a_{R\uparrow}^\dagger)
(\cos\beta\,a_{L\downarrow}^\dagger + \sin\beta\,a_{R\downarrow}^\dagger)|0\rangle$
at a few angles, then minimises $E(\alpha, \beta)$ on a fine grid of angles for many
values of $U$. With $\beta = \pi/2 - \alpha$ and $s = \sin 2\alpha$ the energy is
$-2ts + \tfrac{U}{2}s^2$ (a parabola in $s \in [0, 1]$), whose minimum gives the
closed forms checked below.
```

In [12]:

```
def hf_energy(alpha, beta, U, t=1.0):
    """E(alpha, beta) of the determinant with up (cos a, sin a), down (cos b, sin b)."""
    return (-t * (np.sin(2 * alpha) + np.sin(2 * beta))
            + 0.5 * U * (1.0 + np.cos(2 * alpha) * np.cos(2 * beta)))


H_test = -(a_dag[0] @ a[1] + a_dag[1] @ a[0] + a_dag[2] @ a[3] + a_dag[3] @ a[2]) \
    + 3.0 * DOUBLE  # t = 1, U = 3
worst = 0.0
for alpha, beta in ((0.3, 1.1), (0.7, 0.2), (np.pi / 4, np.pi / 4)):
    det = (np.cos(alpha) * a_dag[0] + np.sin(alpha) * a_dag[1]) @ (
        np.cos(beta) * a_dag[2] + np.sin(beta) * a_dag[3]) @ vacuum
    worst = max(worst, abs(det @ H_test @ det - hf_energy(alpha, beta, 3.0)))
check(worst < 1e-12, "the Hartree-Fock energy formula equals <Phi|H|Phi>")
angles = np.linspace(0.0, np.pi / 2, 1001)
A, B = np.meshgrid(angles, angles, indexing="ij")
U_values = np.linspace(0.0, 8.0, 33)
E_exact = np.array([two_site(1.0, U)[0] for U in U_values])
D_exact = np.array([two_site(1.0, U)[1] for U in U_values])
E_rhf = -2.0 + 0.5 * U_values  # restricted: alpha = beta = pi/4
E_uhf = np.array([hf_energy(A, B, U).min() for U in U_values])  # grid minimum
U_safe = np.where(U_values > 2.0, U_values, 1.0)  # avoids dividing by U = 0
E_closed = np.where(U_values <= 2.0, -2.0 + 0.5 * U_values, -2.0 / U_safe)
check(np.max(np.abs(E_uhf - E_closed)) < 1e-5,
      "the minimum over all determinants is -2t + U/2 (U <= 2t) and -2t^2/U")
check(np.all(E_exact <= E_uhf + 1e-12) and np.all(E_uhf <= E_rhf + 1e-12),
```

```
      "variational order: exact <= unrestricted HF <= restricted HF")
E_c = E_exact - E_closed  # correlation energy
report("U = 2: correlation energy E_0 - E_HF", f"{E_c[8]:.6f}")
fine_U = np.linspace(2.0, 8.0, 60001)  # closed forms on a fine grid of U > 2t
fine_c = 0.5 * (fine_U - np.sqrt(fine_U ** 2 + 16.0)) + 2.0 / fine_U
report("U of the largest correlation energy (in size)",
       f"{fine_U[np.argmin(fine_c)]:.3f}")
```

Out [12]:

```
PASS the Hartree-Fock energy formula equals <Phi|H|Phi>
PASS the minimum over all determinants is -2t + U/2 (U <= 2t) and -2t^2/U
PASS variational order: exact <= unrestricted HF <= restricted HF
RESULT U = 2: correlation energy E_0 - E_HF = -0.236068
RESULT U of the largest correlation energy (in size) = 3.335
```

Text cell:

```
The next cell draws the three energies and the correlation energy against $U/t$.
```

In [13]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(U_values, E_rhf, "--", label="restricted HF $-2t + U/2$")
left.plot(U_values, E_closed, "-.", label="unrestricted HF")
left.plot(U_values, E_exact, color="black", lw=2.0, label="exact $E_0$")
left.axvline(2.0, color="gray", lw=0.8)
left.set_xlabel("repulsion $U/t$")
left.set_ylabel("ground-state energy ($t$)")
left.legend(fontsize=8)
right.plot(U_values, E_c, "o-", ms=3, color="black")
right.set_xlabel("repulsion $U/t$")
right.set_ylabel("correlation energy $E_0 - E_{HF}$ ($t$)")
fig.subplots_adjust(wspace=0.32)  # room between the panels for the axis label
fig.suptitle("Two electrons on two sites: exact versus Hartree-Fock")
save_figure(fig, "two_site_energies",
            "Left: the ground-state energy of two electrons on two sites against "
            "the repulsion $U/t$: exact (black), restricted Hartree-Fock "
            "$-2t + U/2$ (dashed) and the best determinant, unrestricted for "
            "$U > 2t$ (dash-dotted; the grey line marks $U = 2t$); energies in "
            "units of the hopping $t$. Right: the correlation energy, exact minus "
            "best Hartree-Fock energy, which no single determinant can capture; "
            "its size grows from zero, is largest at about "
            f"$U = {fine_U[np.argmin(fine_c)]:.1f}t$ (beyond the "
            "point $U = 2t$ where restricted and unrestricted Hartree-Fock "
            "separate) and then falls slowly.")
```

Out [13]:

```
Figure 13c.4 saved as Revision/textbook/figures/13c_4_two_site_energies.png
```

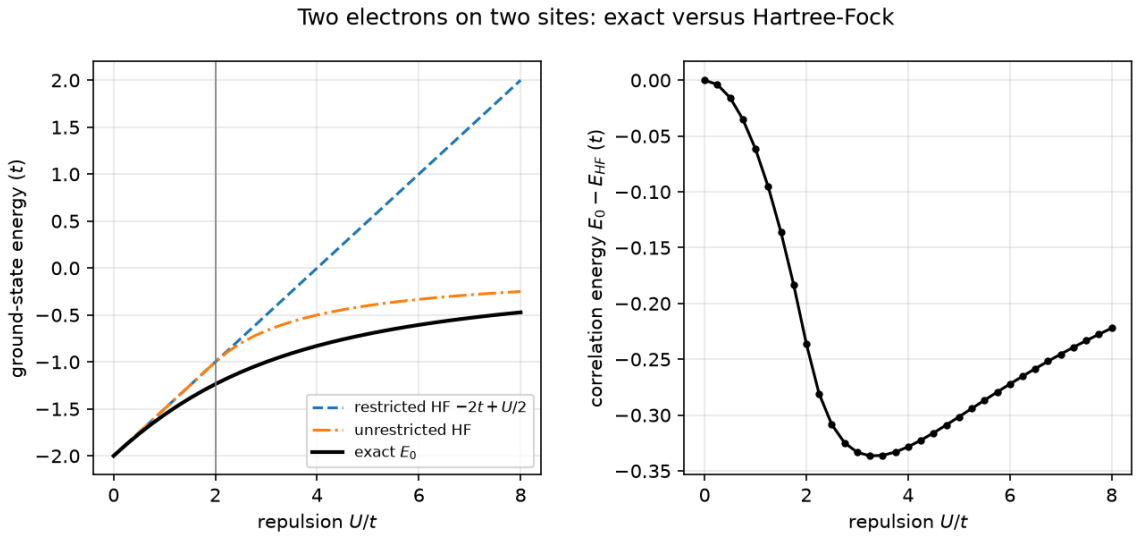

Figure 347: Left: the ground-state energy of two electrons on two sites against the repulsion U/t: exact (black), restricted Hartree-Fock $-2t + U/2$ (dashed) and the best determinant, unrestricted for $U > 2t$ (dash-dotted; the grey line marks $U = 2t$); energies in units of the hopping t. Right: the correlation energy, exact minus best Hartree-Fock energy, which no single determinant can capture; its size grows from zero, is largest at about $U = 3.3t$ (beyond the point $U = 2t$ where restricted and unrestricted Hartree-Fock separate) and then falls slowly. (Notebook 13c, figure 4.)

Text cell:

```
The next cell draws the probability that both electrons sit on the same site. The
restricted determinant always gives $1/2$; the exact state avoids double occupancy
more and more as $U$ grows; the unrestricted determinant gives
$\tfrac12(1 + \cos 2\alpha\cos 2\beta) = 2t^2/U^2$ for $U > 2t$, by breaking the
left-right symmetry of each label.
```

In [14]:

```
D_uhf = np.where(U_values <= 2.0, 0.5, 2.0 / U_safe ** 2)
fig, ax = plt.subplots()
ax.plot(U_values, 0.5 * np.ones_like(U_values), "--", label="restricted HF: 1/2")
ax.plot(U_values, D_uhf, "-.", label="unrestricted HF: $2t^2/U^2$ for $U > 2t$")
ax.plot(U_values, D_exact, color="black", lw=2.0, label="exact")
ax.set_xlabel("repulsion $U/t$")
ax.set_ylabel("double occupancy")
ax.set_title("How often both electrons sit on the same site")
ax.legend()
save_figure(fig, "double_occupancy",
            "The probability that both electrons sit on the same site (double "
            "occupancy, a pure number) against $U/t$: exact ground state (black), "
            "restricted Hartree-Fock (always 1/2, dashed) and unrestricted "
            "Hartree-Fock (dash-dotted). The exact electrons avoid each other while "
            "keeping each label shared equally between the sites; a single "
            "determinant can lower the double occupancy only by breaking that "
```

```
            "symmetry, and then overshoots.")
check(abs(D_exact[8] - 0.276393) < 1e-6 and abs(D_exact[16] - 0.146447) < 1e-6,
      "exact double occupancy 0.276393 at U = 2t and 0.146447 at U = 4t")
check(np.all(np.diff(D_exact) < 0), "the exact double occupancy falls as U grows")
```

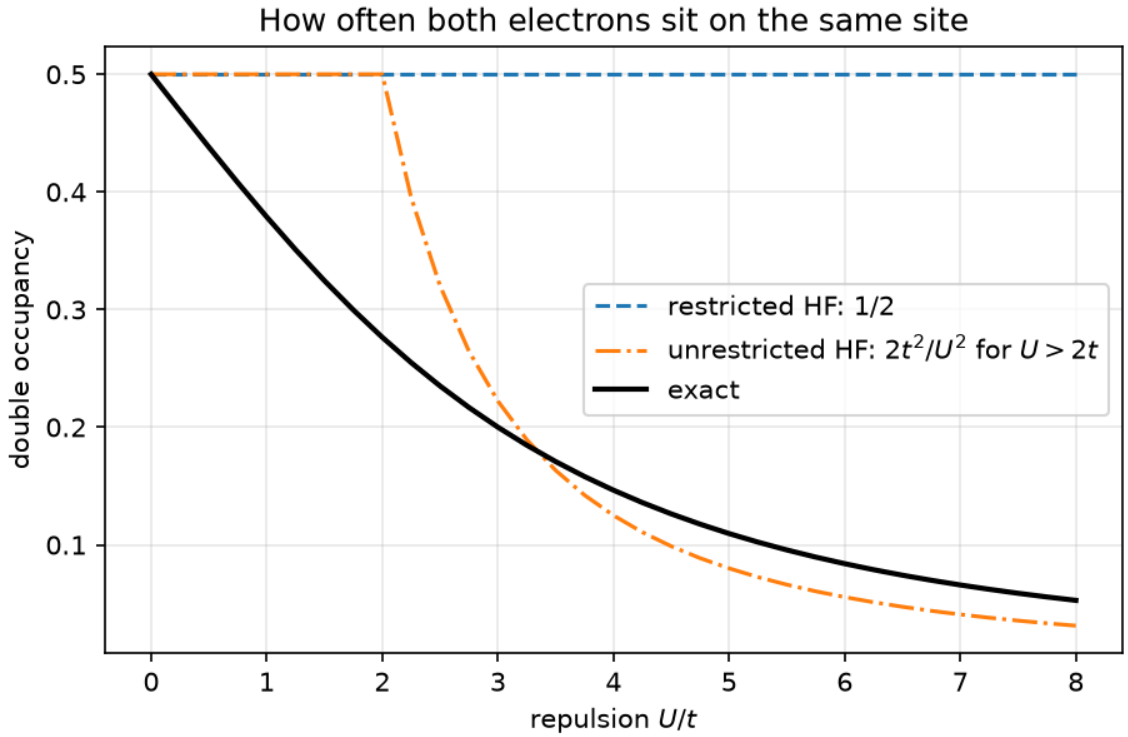

Figure 348: The probability that both electrons sit on the same site (double occupancy, a pure number) against U/t: exact ground state (black), restricted Hartree-Fock (always 1/2, dashed) and unrestricted Hartree-Fock (dash-dotted). The exact electrons avoid each other while keeping each label shared equally between the sites; a single determinant can lower the double occupancy only by breaking that symmetry, and then overshoots. (Notebook 13c, figure 5.)

Out [14]:

```
Figure 13c.5 saved as Revision/textbook/figures/13c_5_double_occupancy.png
PASS exact double occupancy 0.276393 at U = 2t and 0.146447 at U = 4t
PASS the exact double occupancy falls as U grows
```

Text cell:

```
The next cell draws the Hartree-Fock energy $E(\alpha, \beta)$ at $U = 4t$ as a
contour map. The restricted point $\alpha = \beta = \pi/4$ is a saddle; the two
lowest points (unrestricted solutions) lie on the line $\beta = \pi/2 - \alpha$.
```

In [15]:

```
E_map = hf_energy(A, B, 4.0)
index = np.unravel_index(np.argmin(E_map), E_map.shape)
fig, ax = plt.subplots(figsize=(6.0, 5.0))
contours = ax.contourf(A, B, E_map, levels=30, cmap="viridis")
fig.colorbar(contours, ax=ax, label="$E(\\alpha, \\beta)$ ($t$)")
ax.plot([np.pi / 4], [np.pi / 4], "wx", ms=10, label="restricted (saddle)")
ax.plot([angles[index[0]], angles[index[1]]], [angles[index[1]], angles[index[0]]],
        "r*", ms=12, label="unrestricted minima")
ax.set_xlabel("$\\alpha$ (up orbital)")
ax.set_ylabel("$\\beta$ (down orbital)")
ax.set_title("Hartree-Fock energy landscape at $U = 4t$")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "hf_landscape",
            "The energy $E(\\alpha, \\beta)$ of the determinant with the up orbital "
            "$(\\cos\\alpha, \\sin\\alpha)$ and the down orbital "
            "$(\\cos\\beta, \\sin\\beta)$ on the sites (L, R), for $U = 4t$, as a "
            "contour map over the two angles (radians; colors in units of $t$). "
            "The restricted determinant $\\alpha = \\beta = \\pi/4$ (white cross) is "
            "a saddle; the two minima (red stars), mirror images of each other, "
            "put the up electron mostly on one site and the down electron mostly "
            "on the other, with energy $-2t^2/U = -0.5t$.")
check(abs(E_map.min() + 0.5) < 1e-5 and abs(hf_energy(np.pi / 4, np.pi / 4, 4.0)
                                            - 0.0) < 1e-12,
      "at U = 4t: minimum -0.5 t, restricted value 0")
```

Out [15]:

```
Figure 13c.6 saved as Revision/textbook/figures/13c_6_hf_landscape.png
PASS at U = 4t: minimum -0.5 t, restricted value 0
```

Text cell:

```
## 11. The Hohenberg-Kohn map and the exact Kohn-Sham potential on two sites

Give the sites the energies $\mp\Delta/2$. For every $\Delta$ the exact ground state
has a density $n_L$ on the left site ($n_R = 2 - n_L$). The Hohenberg-Kohn theorem
says that different $\Delta$ give different $n_L$: the map $\Delta \to n_L$ is
one-to-one, so $n_L$ determines $\Delta$. The next cell computes $n_L(\Delta)$ for
$U = 0, 2, 4$ and checks that it strictly increases.

Kohn and Sham ask: which site-energy difference $\Delta_s$ makes NON-interacting
electrons have the same $n_L$? For two non-interacting electrons in the lowest
orbital of $\begin{pmatrix}-\Delta_s/2 & -t\\ -t & \Delta_s/2\end{pmatrix}$,
$n_L - 1 = \Delta_s/\sqrt{\Delta_s^2 + 4t^2}$, which inverts to
$\Delta_s = 2t\,(n_L - 1)/\sqrt{1 - (n_L - 1)^2}$. The difference $\Delta_s -
\Delta$ is the exact Hartree-exchange-correlation potential difference between the
sites.
```

In [16]:

```
deltas = np.linspace(-4.0, 4.0, 81)
density_maps = {U: np.array([two_site(1.0, U, d)[2] for d in deltas])
```

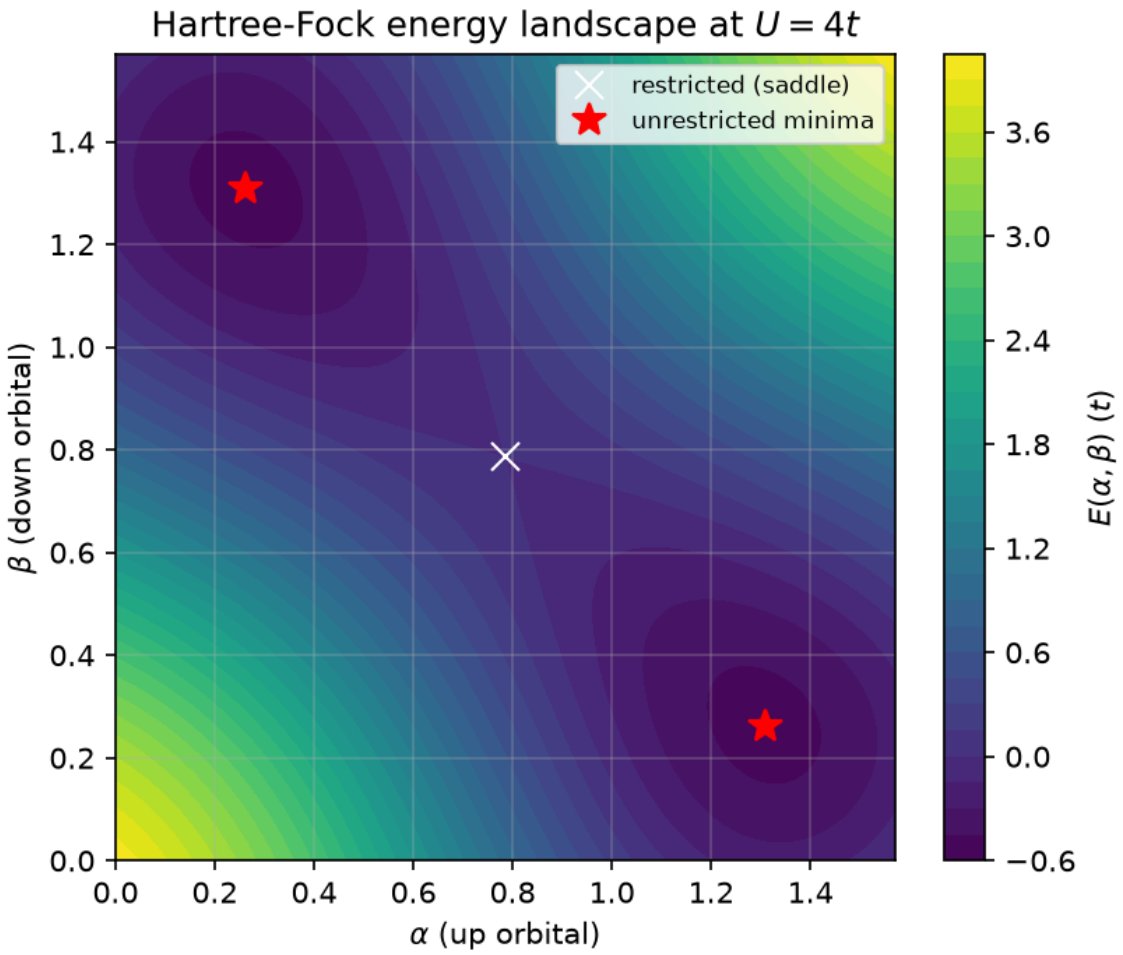

Figure 349: The energy $E(\alpha, \beta)$ of the determinant with the up orbital $(\cos\alpha, \sin\alpha)$ and the down orbital $(\cos\beta, \sin\beta)$ on the sites (L, R), for $U = 4t$, as a contour map over the two angles (radians; colors in units of t). The restricted determinant $\alpha = \beta = \pi/4$ (white cross) is a saddle; the two minima (red stars), mirror images of each other, put the up electron mostly on one site and the down electron mostly on the other, with energy $-2t^2/U = -0.5t$. (Notebook 13c, figure 6.)

```
                for U in (0.0, 2.0, 4.0)}
for U, n_left in density_maps.items():
    check(np.all(np.diff(n_left) > 0),
          f"U = {U:.0f}: n_L increases strictly with Delta (one-to-one map)")


def kohn_sham_delta(n_left, t=1.0):
    """The site-energy difference of non-interacting electrons with density n_L."""
    excess = n_left - 1.0
    return 2.0 * t * excess / np.sqrt(1.0 - excess ** 2)


def free_density(delta_s, t=1.0):
    """n_L of two non-interacting electrons with the site energies -+delta_s/2."""
    values, vectors = np.linalg.eigh(np.array([[-delta_s / 2, -t],
                                               [-t, delta_s / 2]]))
    return 2.0 * vectors[0, 0] ** 2


delta_s = {U: kohn_sham_delta(n) for U, n in density_maps.items()}
rebuilt = max(abs(free_density(ds) - n) for U in density_maps
              for ds, n in zip(delta_s[U], density_maps[U]))
check(rebuilt < 1e-12, "the Kohn-Sham site energies reproduce the exact densities")
check(np.max(np.abs(delta_s[0.0] - deltas)) < 1e-12,
      "without interaction the Kohn-Sham potential is the true one")
```

Out [16]:

```
PASS U = 0: n_L increases strictly with Delta (one-to-one map)
PASS U = 2: n_L increases strictly with Delta (one-to-one map)
PASS U = 4: n_L increases strictly with Delta (one-to-one map)
PASS the Kohn-Sham site energies reproduce the exact densities
PASS without interaction the Kohn-Sham potential is the true one
```

Text cell:

```
The next cell draws the density map $n_L(\Delta)$ for the three repulsions.
```

In [17]:

```
fig, ax = plt.subplots()
for U, style in ((0.0, ":"), (2.0, "--"), (4.0, "-")):
    ax.plot(deltas, density_maps[U], style, label=f"$U = {U:.0f}t$")
ax.axhline(1.0, color="gray", lw=0.8)
ax.set_xlabel("site-energy difference $\\Delta$ ($t$)")
ax.set_ylabel("exact density on the left site $n_L$")
ax.set_title("Hohenberg-Kohn on two sites: the potential determines the density")
ax.legend()
save_figure(fig, "density_map",
            "The exact ground-state density $n_L$ on the left site of the two-site "
            "model against the site-energy difference $\\Delta$ (units of $t$), "
```

```
"for $U = 0$, $2t$ and $4t$. Every curve rises strictly, so each "
"density belongs to exactly one $\\Delta$ (the Hohenberg-Kohn theorem "
"on two sites); the repulsion flattens the curves, because it opposes "
"piling both electrons on one site.")
```

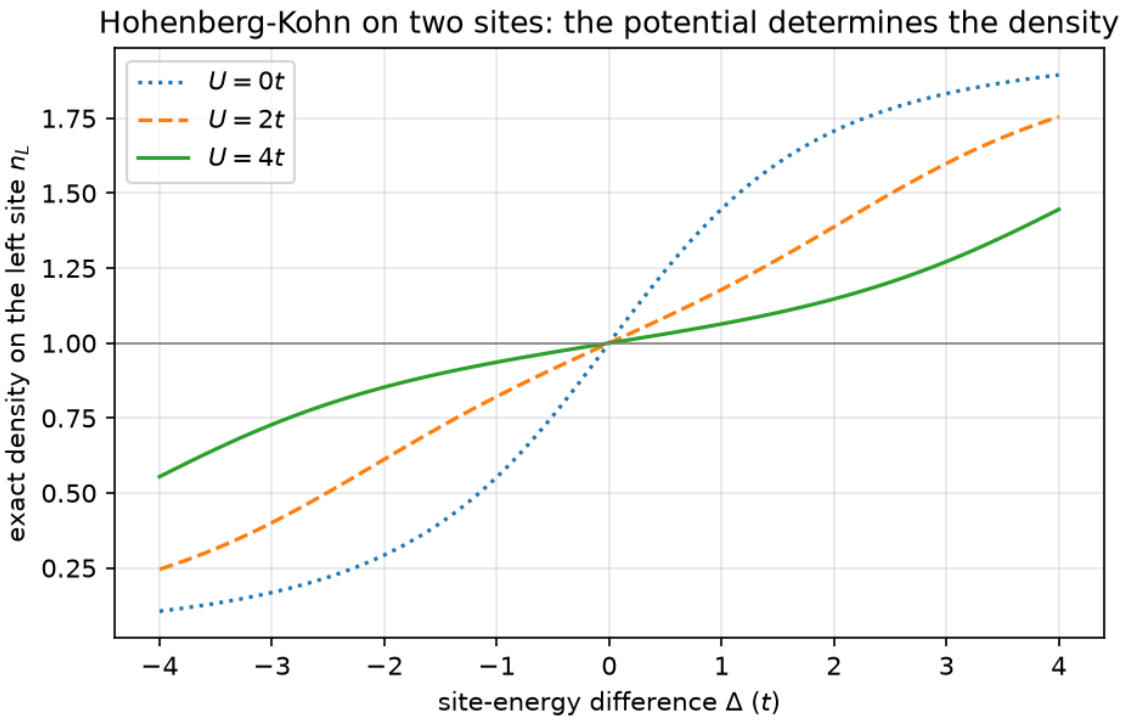

Figure 350: The exact ground-state density n_L on the left site of the two-site model against the site-energy difference Δ (units of t), for $U = 0$, $2t$ and $4t$. Every curve rises strictly, so each density belongs to exactly one Δ (the Hohenberg-Kohn theorem on two sites); the repulsion flattens the curves, because it opposes piling both electrons on one site. (Notebook 13c, figure 7.)

Out [17]:

```
Figure 13c.7 saved as Revision/textbook/figures/13c_7_density_map.png
```

Text cell:

```
For comparison the next cell also computes the potential of the mean-field
(Hartree plus exchange, restricted) approximation, in which each electron feels $U$
times the density of the other label on each site: its site-energy difference is
$\Delta - U(n_L - 1)$, with $n_L$ solved self-consistently (by bisection on the
equation $n_L - 1 = G(n_L - 1)$ with $G(x) = (\Delta - Ux)/\sqrt{(\Delta - Ux)^2 +
4t^2}$; the left side minus the right side increases with $x$, so the root is
unique).

Then it splits the screening $\Delta_s - \Delta$ of the exact Kohn-Sham potential,
AT THE EXACT DENSITY $n_L$, into two parts: the Hartree-exchange part
```

```
$\Delta_{Hx} = -U(n_L - 1)$ (the mean-field formula evaluated with the exact
density) and the rest, the correlation part
$\Delta_c = \Delta_s - \Delta - \Delta_{Hx}$. It checks, for $U = 2t$ and $4t$,
that $\Delta_c$ has the sign opposite to $\Delta$ for every $\Delta \ne 0$
(correlation screens further, in the same direction as Hartree and exchange), and
that the self-consistent mean field screens less than the exact Kohn-Sham
potential. Finally it draws the curves.
```

In [18]:

```
def mean_field_excess(delta, U, t=1.0):
    """The self-consistent x = n_L - 1 of restricted Hartree + exchange."""
    low, high = -1.0, 1.0
    for _ in range(80):  # bisection on x - G(x), which increases with x
        middle = 0.5 * (low + high)
        shifted = delta - U * middle
        if middle - shifted / np.sqrt(shifted ** 2 + 4 * t * t) > 0:
            high = middle
        else:
            low = middle
    return 0.5 * (low + high)


mf_delta_s = np.array([d - 4.0 * mean_field_excess(d, 4.0) for d in deltas])
nonzero = np.abs(deltas) > 1e-9  # every Delta except 0
hx_part = {U: -U * (density_maps[U] - 1.0) for U in (2.0, 4.0)}  # at the exact n_L
c_part = {U: delta_s[U] - deltas - hx_part[U] for U in (2.0, 4.0)}  # the rest
report("U = 4t, Delta = 2t: Hartree-exchange part", f"{hx_part[4.0][60]:.6f}")
report("U = 4t, Delta = 2t: correlation part", f"{c_part[4.0][60]:.6f}")
for U in (2.0, 4.0):
    check(np.all(c_part[U][nonzero] * deltas[nonzero] < 0.0),
          f"U = {U:.0f}t: the correlation part has the sign opposite to Delta")
check(np.all(np.abs(mf_delta_s[nonzero]) > np.abs(delta_s[4.0][nonzero])),
      "U = 4t: the mean field screens less than the exact Kohn-Sham potential")
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4))
left.plot(deltas, deltas, ":", color="gray", label="$\\Delta_s = \\Delta$ ($U = 0$)")
left.plot(deltas, delta_s[2.0], "--", label="exact Kohn-Sham, $U = 2t$")
left.plot(deltas, delta_s[4.0], color="black", lw=2.0,
          label="exact Kohn-Sham, $U = 4t$")
left.plot(deltas, mf_delta_s, "-.", label="mean field, $U = 4t$")
left.set_xlabel("true site-energy difference $\\Delta$ ($t$)")
left.set_ylabel("Kohn-Sham site-energy difference $\\Delta_s$ ($t$)")
left.set_title("The exact Kohn-Sham potential")
left.legend(fontsize=8)
right.plot(deltas, delta_s[4.0] - deltas, color="black", lw=2.0,
           label="total screening $\\Delta_s - \\Delta$")
right.plot(deltas, hx_part[4.0], "--", label="Hartree-exchange $\\Delta_{Hx}$")
right.plot(deltas, c_part[4.0], color="tab:red", label="correlation $\\Delta_c$")
right.axhline(0.0, color="gray", lw=0.8)
right.set_xlabel("true site-energy difference $\\Delta$ ($t$)")
right.set_ylabel("parts of $\\Delta_s - \\Delta$ ($t$)")
right.set_title("Its parts at the exact density, $U = 4t$")
right.legend(fontsize=8)
```

```
save_figure(fig, "ks_inversion",
            "Left: the site-energy difference $\\Delta_s$ with which non-interacting "
            "electrons reproduce the exact density of the two-site model, against "
            "the true difference $\\Delta$ (both in units of $t$), for $U = 2t$ "
            "(dashed) and $U = 4t$ (black), with the line $\\Delta_s = \\Delta$ "
            "(dotted) and the self-consistent mean field for $U = 4t$ "
            "(dash-dotted). The interaction screens the potential; the mean field, "
            "which leaves out correlation, screens less. Right: for $U = 4t$, the "
            "screening $\\Delta_s - \\Delta$ (black) split at the exact density "
            "into the Hartree-exchange part $-U(n_L - 1)$ (dashed) and the "
            "correlation part (red); both have the sign opposite to $\\Delta$.")
check(np.all(np.abs(delta_s[4.0][nonzero]) < np.abs(deltas[nonzero])),
      "the repulsion screens the potential: |Delta_s| < |Delta|")
```

Out [18]:

```
RESULT U = 4t, Delta = 2t: Hartree-exchange part = -0.588239
RESULT U = 4t, Delta = 2t: correlation part = -1.114408
PASS U = 2t: the correlation part has the sign opposite to Delta
PASS U = 4t: the correlation part has the sign opposite to Delta
PASS U = 4t: the mean field screens less than the exact Kohn-Sham potential
```

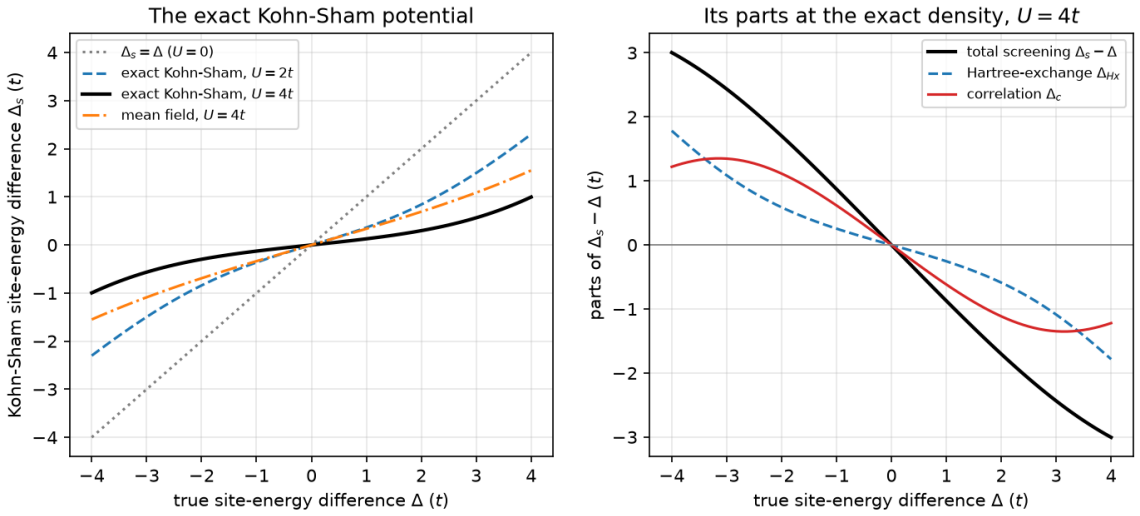

Figure 351: Left: the site-energy difference Δ_s with which non-interacting electrons reproduce the exact density of the two-site model, against the true difference Δ (both in units of t), for $U = 2t$ (dashed) and $U = 4t$ (black), with the line $\Delta_s = \Delta$ (dotted) and the self-consistent mean field for $U = 4t$ (dash-dotted). The interaction screens the potential; the mean field, which leaves out correlation, screens less. Right: for $U = 4t$, the screening $\Delta_s - \Delta$ (black) split at the exact density into the Hartree-exchange part $-U(n_L - 1)$ (dashed) and the correlation part (red); both have the sign opposite to Δ. (Notebook 13c, figure 8.)

Out [18]:

```
Figure 13c.8 saved as Revision/textbook/figures/13c_8_ks_inversion.png
PASS the repulsion screens the potential: |Delta_s| < |Delta|
```

Text cell:

```
## 12. The last check

The last cell checks that all eight figure files exist and prints the number of
checks that passed.
```

In [19]:

```
figure_names = ["slater_determinants", "operator_matrices", "density_matrices",
                "two_site_energies", "double_occupancy", "hf_landscape",
                "density_map", "ks_inversion"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13c_{k}_{name}.png").is_file()]
check(missing == [], "all eight figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13c_8_ks_inversion.png").is_file(),
      "the figure file 13c_8_ks_inversion.png exists")
all_checks_passed()
```

Out [19]:

```
PASS all eight figure files exist
PASS the figure file 13c_8_ks_inversion.png exists
ALL 33 CHECKS PASSED (notebook 13c)
```

Text cell:

```
## 13. What this notebook showed

- A Slater determinant is antisymmetric, vanishes when two fermions meet, is
  normalised, and its density is the sum of its orbital densities.
- Creation and annihilation operators are ordinary matrices on the occupation-number
  states; their anticommutation relations hold exactly.
- Wick's theorem holds for determinants and for thermal ensembles of non-interacting
  fermions; in particular $\langle :S^2:\rangle = (\mathrm{Tr}\,V\rho)^2 -
  \mathrm{Tr}(V\rho V\rho)$, the Hartree-minus-exchange identity that the Revision
  Kohn-Sham theory uses with the vertex $C$.
- The energy of a determinant is the one-body energy plus the direct minus the
  exchange terms.
- For two electrons on two sites: $E_0 = \tfrac12(U - \sqrt{U^2 + 16t^2})$, the best
  determinant gives $-2t + U/2$ ($U \le 2t$) or $-2t^2/U$ ($U > 2t$, unrestricted,
  symmetry broken), and the correlation energy is what no determinant captures. At
  $U = 2t$: $E_0 = -1.236068\,t$, $E_{HF} = -t$, double occupancy 0.276393 versus
  1/2.
- The density determines the site-energy difference (Hohenberg-Kohn on two sites),
  and the exact Kohn-Sham potential exists and screens the true one. At the exact
  density its screening is the Hartree-exchange part $-U(n_L - 1)$ plus a
  correlation part of the same sign; the self-consistent mean field, which leaves
  out correlation, screens less.
```

13.14 Line-by-line walk-through of Notebook 13c

The notebook has nineteen code cells, In [1] to In [19]. This section explains every line of every one of them. Each code cell is preceded in the notebook by a text cell that says what it does; the numbers it prints are in Section 13.13.

In [1], the set-up cell. Apart from its comment lines and the notebook's name, it is the same in every notebook of the book; the notebooks 13b, 13d, 13e and 13a of this chapter have exactly this code with their own name, and their walk-throughs refer back to this paragraph. Its first part repeats the complete run instructions of Section 13.12 as **comment lines**: every line that starts with `#` is skipped by Python, and the lines are there so that the notebook file carries its own instructions. The code starts after the comment line THE SET-UP, which stands between two lines of `=` signs; the comment lines inside the code (which say, for example, why `REPO` is never printed) are left out below, because the text explains the same things.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "13c"  # this notebook: chapter 13, example c
```

`import` loads a **module** (a part of Python or of a package) so that the code can use it; the text after `#` on each line says what it is for. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system. `import matplotlib.pyplot as plt` loads the drawing functions under the short name `plt`. The last line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"13c"`; the figure files are named after it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

(The docstring, the text in triple quotes under the `def` line that says what the function does, is left out here.) `def` defines a **function**, a named piece of code that runs when it is called. `Path.cwd()` is the folder in which the notebook runs and `.resolve()` writes it as a complete address. The list `[here, *here.parents]` holds this folder, its parent folder, the parent of that, and so on up to the top of the disk; the `for` loop takes them one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back; if none does, `raise` stops the notebook with an error message that says what to do.

```
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`REPO` is the repository folder; it is never printed, because it differs from computer to computer while the printed output of a notebook must be the same everywhere. `os.environ.get(name, default)` reads an **environment variable** (a named text that a program receives from the computer) or returns the default; when you run the notebook the variable is not set, so `OUTPUT_ROOT` is the repository, and the book's checking tool sets it to a scratch folder so that a check never changes the repository; the three comment lines before it (they start with `#` and are not executed) say the same. `repository_file` gives the full path of a repository file for reading (a Revision record); `output_file` gives the full path at which to write a file, after creating its folder (`mkdir` with `parents=True` makes missing parent folders too, and `exist_ok=True` makes it do nothing if the folder exists). `say` prints a text in lines of at most 89 characters (the width of a page of the book); continuation lines start with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

`matplotlib.rcdefaults()` returns to the built-in plotting settings, so that the figures are the same on every computer, and `plt.rcParams.update` sets the figure size (7.0 by 4.2 inches), the letter size (10 points) and a faint grid. The braces `{...}` make a **dictionary**: pairs of a key and a value. A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/13c.captions.json"`. The last line, after two comment lines that describe it, writes an empty dictionary `{}` into the captions file; `newline="\n"` stores the same line end on every operating system.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` (shown without its docstring; its three comment lines explain the options of `savefig`) numbers the figures 1, 2, 3, ... in the order in which they are saved (`setdefault` returns the number already stored for this name, or stores one more than the count so far), saves the figure as a PNG file with 150 dots per inch, cut to its content (`bbox_inches="tight"`) and without the program's name in the file (so that two runs write the same bytes), closes it, records its caption in the captions file (`json.dumps` turns the dictionary into JSON text), shows the saved picture below the cell, and prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`PASSED` is an empty **list** (an ordered collection, in square brackets). `check` is the function behind every check: if the statement `condition` is false (`False`), `raise AssertionError(...)` stops the

notebook with an error that names the check; if it is true, the name is appended to `PASSED` and the line PASS name is printed, followed by a line naming the Revision record and its check when the argument `record` is given. (Python's own `assert` statement is not used, because Python started with the option that optimises the code skips it.) `report` prints a key number as a line that starts with RESULT. `all_checks_passed` prints the last line of the notebook with the number of checks that passed. The last statement prints the one output line of In [1].

In [2], the determinant on three points.

```
import itertools  # loops over all combinations of indices

import numpy as np  # arrays, matrices and linear algebra

phi0 = np.array([1.0, 0.0, 0.0])
phi1 = np.array([0.0, 1.0, 1.0]) / np.sqrt(2.0)
# Phi[r1, r2] = (phi0(r1) phi1(r2) - phi1(r1) phi0(r2)) / sqrt(2)
Phi = (np.outer(phi0, phi1) - np.outer(phi1, phi0)) / np.sqrt(2.0)
```

`itertools` is a module of Python that makes all combinations of indices; `numpy`, called `np`, holds arrays and matrices. `np.array([...])` makes an **array** (a list of numbers that supports arithmetic), so `phi0` and `phi1` are the two orbitals of Section 13.3; dividing an array by $\sqrt{2}$ divides every entry. `np.outer(u, w)` is the 3×3 table with the entries $u_{r_1} w_{r_2}$; so `Phi` is the table of $\Phi(r_1, r_2)$, row r_1 and column r_2, exactly the formula in the comment line.

```
for r1 in range(3):
    say("Phi(%d, r2) for r2 = 0, 1, 2: %s" % (r1, np.array2string(
        Phi[r1], precision=6, floatmode="fixed", suppress_small=True)))
density_3 = 2.0 * np.sum(Phi ** 2, axis=1)  # n(r) = N sum_r2 |Phi(r, r2)|^2
```

The loop prints the three rows of the table. `"...  %d ...  %s" % (r1, text)` puts the number `r1` in place of `%d` and the text in place of `%s`; `np.array2string` writes a row as text with 6 digits after the point (`precision=6`, and `floatmode="fixed"` always prints all six), and `suppress_small=True` prints numbers that are very close to zero as zeros instead of in powers-of-ten notation. `Phi ** 2` squares every entry, and `np.sum(..., axis=1)` adds along each row (over r_2); times $N = 2$ this is the density $n(r)$ of Section 13.3.

```
check(abs(Phi[0, 1] - 0.5) < 1e-15 and abs(Phi[1, 2]) < 1e-15,
      "Phi(0,1) = 1/2 and Phi(1,2) = 0")
check(np.allclose(Phi, -Phi.T, atol=1e-15) and np.allclose(np.diag(Phi), 0.0),
      "Phi is antisymmetric and zero on the diagonal (Pauli)")
check(abs(np.sum(Phi ** 2) - 1.0) < 1e-15
      and np.allclose(density_3, [1.0, 0.5, 0.5], atol=1e-15)
      and np.allclose(density_3, phi0 ** 2 + phi1 ** 2, atol=1e-15),
      "Phi is normalised and its density is (1, 1/2, 1/2) = phi0^2 + phi1^2")
```

Three checks. `Phi[0, 1]` is the entry in row 0 and column 1, and `1e-15` means 10^{-15}: the first check compares the two values worked out by hand in Section 13.3. `Phi.T` is the transposed table (rows and columns exchanged), so `Phi` equal to `-Phi.T` is antisymmetry; `np.allclose(a, b, atol=...)` is true when every entry of `a` differs from the entry of `b` by less than the given amount; `np.diag(`

`Phi)` is the diagonal. The third check adds all nine squares (the normalisation) and compares the density with $(1, \frac{1}{2}, \frac{1}{2})$ and with $\phi_0^2 + \phi_1^2$.

In [3], two determinants as pictures.

```
x_box = np.linspace(0.0, 1.0, 101)
chi1 = np.sqrt(2.0) * np.sin(np.pi * x_box)
chi2 = np.sqrt(2.0) * np.sin(2.0 * np.pi * x_box)
Phi_box = (np.outer(chi1, chi2) - np.outer(chi2, chi1)) / np.sqrt(2.0)
```

`np.linspace(0.0, 1.0, 101)` is 101 equally spaced positions from 0 to 1. `chi1` and `chi2` are the two lowest orbitals of a particle in the box $0 < x < 1$ ($\sqrt{2}\sin(\pi x)$ and $\sqrt{2}\sin(2\pi x)$, normalised and orthogonal), and `Phi_box` their determinant on the 101×101 grid of positions of the two fermions.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
image = left.imshow(Phi, origin="lower", cmap="coolwarm", vmin=-0.6, vmax=0.6)
for r1, r2 in itertools.product(range(3), repeat=2):  # write every value in
    left.text(r2, r1, f"{Phi[r1, r2]:+.2f}", ha="center", va="center")
```

`plt.subplots(1, 2, ...)` makes a figure with two pairs of axes side by side, named `left` and `right`. `imshow` draws a table as a **heat map**: each entry is a coloured square, here red for positive and blue for negative (`cmap="coolwarm"`, the colour scale from -0.6 to 0.6), with row 0 at the bottom (`origin="lower"`). `itertools.product(range(3), repeat=2)` gives all nine pairs (r_1, r_2), and `left.text` writes each value, with its sign and two decimals (`:+.2f`), in the middle of its square.

```
left.set_xticks([0, 1, 2])
left.set_yticks([0, 1, 2])
left.set_xlabel("position $r_2$ of fermion 2")
left.set_ylabel("position $r_1$ of fermion 1")
left.set_title("three points")
```

`set_xticks` and `set_yticks` put tick marks only at the three points 0, 1, 2; `set_xlabel`, `set_ylabel` and `set_title` write the axis labels and the title of the left panel. Text between two dollar signs is drawn by matplotlib as a formula, so the text r_2 between dollar signs in the label appears as r_2.

```
image = right.imshow(Phi_box, origin="lower", extent=(0, 1, 0, 1),
                     cmap="coolwarm")
right.plot([0, 1], [0, 1], "k--", lw=0.8)
right.set_xlabel("position $r_2$ of fermion 2")
right.set_ylabel("position $r_1$ of fermion 1")
right.set_title("two fermions in a box")
fig.colorbar(image, ax=right, shrink=0.85)
save_figure(fig, "slater_determinants",
```

The right panel draws the 101×101 table `Phi_box` as a heat map; `extent=(0, 1, 0, 1)` stretches it over the square $0 \le r_1, r_2 \le 1$, so that the axes show positions instead of table indices, and without `vmin` and `vmax` the colour scale runs over the table's own range. The line with `right.pl`

`ot([0, 1], [0, 1], ...)` draws the straight line from $(0, 0)$ to $(1, 1)$, the diagonal $r_1 = r_2$: in its style string the letter k means black and the two hyphens that follow it a dashed line, and `lw=0.8` is the line width in points. `fig.colorbar(image, ax=right, shrink=0.85)` adds the colour scale of the right panel, 85 per cent as tall as the panel. The call `save_figure(fig, "slater_determi nants",` saves Figure 13c.1; the lines that follow it in the cell are the text of the caption, which the book prints under the figure. The last lines check that the box determinant is antisymmetric as well:

```
check(np.allclose(Phi_box, -Phi_box.T, atol=1e-14), "the box determinant is "
      "antisymmetric")
```

(Python joins two strings written next to each other into one.)

What Figure 13c.1 shows. On the left, the nine values of the three-point determinant: $+0.50$ in row $r_1 = 0$ at $r_2 = 1, 2$, -0.50 in column $r_2 = 0$ at $r_1 = 1, 2$, and 0 everywhere else, in particular on the diagonal. Reflecting the table in its diagonal changes every sign: that is antisymmetry. On the right, the box determinant is positive (red, up to about $+2.2$) where fermion 1 is to the right of fermion 2 ($r_1 > r_2$, above the dashed diagonal), negative (blue) in the mirror-image region, and exactly zero along the diagonal: two fermions with the same label are never found at the same place.

In [4], creation and annihilation operators as matrices.

```
M = 4  # orbitals
DIM = 2 ** M  # occupation-number states


def bit(state, p):
    """The occupation n_p (0 or 1) of orbital p in the state number state."""
    return (state >> p) & 1
```

In Python `**` means "to the power", so `DIM` is $2^4 = 16$. The function `bit` returns the occupation n_p of orbital p in the state with the number `state`: the operator written with two greater-than signs shifts the binary digits of the number `state` by `p` places to the right, and `& 1` keeps the last binary digit. For example, for the state 5 (binary 0101) and $p = 2$ the shift gives binary 01, whose last digit is 1: orbital 2 is occupied.

```
a = []  # a[p] is the 16 x 16 matrix of the annihilation operator a_p
for p in range(M):
    matrix = np.zeros((DIM, DIM))
    for state in range(DIM):
        if bit(state, p) == 1:
            nu = sum(bit(state, q) for q in range(p))  # occupied before p
            matrix[state ^ (1 << p), state] = (-1) ** nu
    a.append(matrix)
a_dag = [matrix.T for matrix in a]  # creation operators: the transposes
```

For each orbital p the cell starts from a 16×16 matrix of zeros and goes through the 16 states. Where orbital p is occupied, it counts the occupied orbitals before p (ν_p of Section 13.4). The row index is `state ^ (...)`, where the parentheses hold the number 1 shifted p binary places to the left (the operator written with two less-than signs), which is 2^p; the operator `^` is the "exclusive or"

of the binary digits, so the row index is `state` with bit p switched off: the number of the state after the removal. The entry in that row and in the column of the old state is $(-1)^{\nu_p}$: the matrix of a_p maps the old state to the new one with the sign of Section 13.4. `a.append` adds the matrix to the list. The creation operators are the transposes (`.T`), because the matrices are real.

```
identity = np.eye(DIM)
worst = 0.0
for p, q in itertools.product(range(M), repeat=2):
    worst = max(worst,
                np.abs(a[p] @ a_dag[q] + a_dag[q] @ a[p] - (p == q) * identity).max(),
                np.abs(a[p] @ a[q] + a[q] @ a[p]).max(),
                np.abs(a_dag[p] @ a_dag[q] + a_dag[q] @ a_dag[p]).max())
check(worst == 0.0, "{a_p, a_q^dag} = delta_pq and {a_p, a_q} = 0 exactly")
```

`np.eye(DIM)` is the 16×16 unit matrix and `@` is the matrix product. For all 16 pairs (p, q) the loop computes the three anticommutators of Section 13.4 minus their required values (`(p == q)` is 1 when $p = q$ and 0 otherwise) and keeps the largest entry of any of them in `worst`. The check requires it to be exactly 0: the matrices hold only 0 and ± 1, so there is no rounding at all.

```
number = sum(a_dag[p] @ a[p] for p in range(M))  # the particle-number operator
check(np.array_equal(np.diag(number), [bin(s).count("1") for s in range(DIM)]),
      "the number operator counts the occupied orbitals of every state")
```

`number` is $\hat{N} = \sum_p a_p^\dagger a_p$. Its diagonal must hold, for every state s, the number of ones in the binary form of s (`bin(s)` writes s in binary, `.count("1")` counts its ones).

In [5], the operator matrices as heat maps.

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, p in zip(axes, (1, 3)):
    image = ax.imshow(a_dag[p], cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(f"creation operator $a_{p}^\\dagger$")
    ax.set_xlabel("state number before (column)")
    ax.set_ylabel("state number after (row)")
fig.colorbar(image, ax=axes, shrink=0.8)
save_figure(fig, "operator_matrices",
```

`plt.subplots(1, 2, ...)` makes two panels, kept together in `axes`. `zip(axes, (1, 3))` pairs the first panel with orbital 1 and the second with orbital 3, and the loop draws `a_dag[p]`, the 16×16 matrix of $a_p^\dagger$, as a heat map with the colour scale fixed from -1 (blue) through 0 (grey) to $+1$ (red). Without `origin="lower"`, `imshow` puts row 0 at the top, as a matrix is printed. In the title the f-string puts the value of `p` in place of `{p}`, and the doubled backslash is one backslash in the text, so matplotlib draws $a_1^\dagger$ and $a_3^\dagger$. The axis labels say that a column is the state before the operator acts and a row the state after it. `fig.colorbar(image, ax=axes, shrink=0.8)` draws one colour scale for both panels. `save_figure` saves Figure 13c.2 (the lines after it are the caption).

What Figure 13c.2 shows. Every column holds at most one coloured square, so each operator sends a state to exactly one other state or to zero. In the left panel ($a_1^\dagger$) the squares sit in the columns 0, 1, 4, 5, 8, 9, 12, 13, the states in which orbital 1 is empty, and in the rows two higher

(bit 1 switched on, which adds $2^1 = 2$ to the state number); the columns 0, 4, 8, 12 (orbital 0 empty) carry $+1$ and the columns 1, 5, 9, 13 (orbital 0 occupied) carry -1, the sign $(-1)^{\nu_1}$. In the right panel ($a_3^\dagger$) the columns 0 to 7 go to the rows 8 to 15, with the sign $+1$ or -1 according to whether an even or an odd number of the orbitals 0, 1, 2 is occupied. The columns 8 to 15 are empty: there orbital 3 is already occupied, and the Pauli principle gives zero.

In [6], Wick's theorem for a determinant.

```
rng = np.random.default_rng(12345)  # fixed seed: the same numbers every run
U_mix = np.linalg.qr(rng.normal(size=(M, M)) + 1j * rng.normal(size=(M, M)))[0]
a_dag_new = [sum(U_mix[k, p] * a_dag[k] for k in range(M)) for p in range(M)]
```

`np.random.default_rng(12345)` makes a generator of random numbers with a fixed starting value (**seed**), so every run draws the same numbers. `rng.normal(size=(M, M))` is a 4×4 table of random numbers, `1j` is the imaginary unit i, so the argument is a random complex matrix; `np.linalg.qr(...)[0]` is the first factor of its **QR factorisation**, a unitary matrix U (its columns are orthonormal). `a_dag_new[p]` is $a_p'^\dagger = \sum_k U_{kp}\, a_k^\dagger$, the creation operator of the new orbital p (Section 13.4).

```
vacuum = np.zeros(DIM)
vacuum[0] = 1.0  # state number 0: no fermion at all
Phi_state = a_dag_new[0] @ a_dag_new[1] @ vacuum
rho_det = sum(np.outer(U_mix[:, k], U_mix[:, k].conj()) for k in (0, 1))
```

The vacuum is the column with 1 in place 0 (the state number 0 has no fermion). `Phi_state` is $a_0'^\dagger a_1'^\dagger |0\rangle$, the determinant of the new orbitals 0 and 1, and `rho_det` is its density matrix $\rho = \sum_{a=0,1} U_{:,a} U_{:,a}^\dagger$ (`U_mix[:, k]` is column k, `.conj()` the complex conjugate, and `np.outer` the column times the conjugated row).

```
def wick_errors(expect, rho):
    """Largest errors of <a+_p a_q> = rho_qp and of the four-operator formula."""
    two = max(abs(expect(a_dag[p] @ a[q]) - rho[q, p])
              for p, q in itertools.product(range(M), repeat=2))
    four = max(abs(expect(a_dag[p] @ a_dag[q] @ a[s] @ a[r])
                   - (rho[r, p] * rho[s, q] - rho[s, p] * rho[r, q]))
               for p, q, r, s in itertools.product(range(M), repeat=4))
    return two, four


def in_determinant(operator):
    """<Phi| operator |Phi> for the determinant Phi_state."""
    return np.vdot(Phi_state, operator @ Phi_state)
```

`wick_errors` takes a rule `expect` that turns an operator into its expectation value, and a density matrix `rho`; it returns the largest error of the two-operator formula over the 16 pairs (p, q) and of the four-operator formula over the $4^4 = 256$ combinations (p, q, r, s), both written exactly as in Section 13.4. `in_determinant` is such a rule for the determinant: `np.vdot(u, w)` is $\sum_k u_k^* w_k$, the inner product.

```
check(abs(np.vdot(Phi_state, Phi_state) - 1.0) < 1e-14, "the determinant is "
      "normalised")
two, four = wick_errors(in_determinant, rho_det)  # the largest errors (rounding)
check(two < 1e-14 and four < 1e-14,
      "Wick's theorem holds for the determinant (all 16 + 256 values, to 1e-14)")
check(np.allclose(rho_det @ rho_det, rho_det, atol=1e-14)
      and abs(np.trace(rho_det).real - 2.0) < 1e-14,
      "the density matrix of a determinant obeys rho^2 = rho, Tr rho = 2")
```

Three checks: the determinant has norm 1; all 272 Wick values agree to 10^{-14} (only rounding remains); and $\rho^2 = \rho$ with trace 2 (`np.trace` is the sum of the diagonal, `.real` its real part).

In [7], Wick's theorem for a thermal ensemble.

```
energies = np.array([-1.0, -0.3, 0.4, 1.2])  # orbital energies (units of t)
MU, TEMPERATURE = 0.0, 0.5
H0 = sum(energies[p] * a_dag_new[p] @ a_dag_new[p].conj().T for p in range(M))
K = H0 - MU * number  # H0 - mu N, a Hermitian 16 x 16 matrix
```

Four orbital energies, the chemical potential $\mu = 0$ and the temperature $T = 0.5$. `a_dag_new[p].conj().T` is the conjugate transpose of $a'^{\dagger}_p$, which is a'_p; so `H0` is $\hat{H}_0 = \sum_p \epsilon_p \, a'^{\dagger}_p a'_p$, non-interacting fermions in the mixed orbitals, and `K` is $\hat{H}_0 - \mu\hat{N}$.

```
k_values, k_vectors = np.linalg.eigh(K)
weights = np.exp(-(k_values - k_values.min()) / TEMPERATURE)  # shifted: no overflow
gibbs = (k_vectors * (weights / weights.sum())) @ k_vectors.conj().T
f = 1.0 / (np.exp((energies - MU) / TEMPERATURE) + 1.0)  # Fermi-Dirac
rho_thermal = sum(f[p] * np.outer(U_mix[:, p], U_mix[:, p].conj()) for p in range(M))
```

`np.linalg.eigh` returns the eigenvalues and the orthonormal eigenvectors (as columns) of the Hermitian matrix `K`. The Gibbs weights $e^{-k_i/T}$ are computed after subtracting the smallest eigenvalue, which multiplies every weight by the same number and so cancels after dividing by their sum, but keeps the exponentials from becoming too large for the computer. `k_vectors * (weights / weights.sum())` multiplies each eigenvector column by its normalised weight, and the product with the conjugate transpose gives the density operator $e^{-(\hat{H}_0 - \mu\hat{N})/T}/Z = \sum_i p_i \, u_i u_i^{\dagger}$ (a function of a matrix defined through its eigenvalues, Section 13.26). `f` are the Fermi-Dirac occupations $1/(e^{(\epsilon_p - \mu)/T} + 1)$ and `rho_thermal` the density matrix $\sum_p f_p \, U_{:,p} U^{\dagger}_{:,p}$.

```
two, four = wick_errors(lambda operator: np.trace(gibbs @ operator), rho_thermal)
check(two < 1e-13 and four < 1e-13,
      "Wick's theorem holds for the thermal ensemble of non-interacting fermions "
      "(to 1e-13)")
```

`lambda operator: ...` is a function without a name; here it is the rule "expectation value in the ensemble", $\mathrm{Tr}(\hat{\rho}\, A)$. The check requires all 272 Wick values to agree to 10^{-13}.

In [8], the normal-ordered square of a vertex.

```
raw = rng.normal(size=(M, M)) + 1j * rng.normal(size=(M, M))
V = (raw + raw.conj().T) / 2.0  # a random Hermitian vertex
S_square = sum(V[p, q] * V[r, s] * a_dag[p] @ a_dag[r] @ a[s] @ a[q]
               for p, q, r, s in itertools.product(range(M), repeat=4))
```

A random complex matrix plus its conjugate transpose, halved, is Hermitian: that is the vertex V. `S_square` is the 16×16 matrix of $:S^2: = \sum V_{pq} V_{rs}\, a_p^\dagger a_r^\dagger a_s a_q$, summed over all 256 index combinations.

```
for label, expect, rho in (("determinant", in_determinant, rho_det),
                           ("thermal", lambda o: np.trace(gibbs @ o), rho_thermal)):
    lhs = expect(S_square)
    rhs = np.trace(V @ rho) ** 2 - np.trace(V @ rho @ V @ rho)
    say(f"{label}: <:S^2:> = {lhs.real:.10f}, Hartree - exchange = {rhs.real:.10f}")
    check(abs(lhs - rhs) < 1e-12,
          f"<:S^2:> = (Tr V rho)^2 - Tr(V rho V rho) in the {label} state",
          record="Revision/kohn_sham/reports/ks-theory-python.json, check "
                 "hf_wick_contraction")
```

For the determinant and for the thermal ensemble, `lhs` is the expectation value of $:S^2:$ computed from the matrices and `rhs` the formula $(\mathrm{Tr}\, V\rho)^2 - \mathrm{Tr}(V\rho V\rho)$ of Section 13.4. The printed values agree to all ten printed decimals, -1.0717850309 and -1.9027816067, and each check names the Revision check hf_wick_contraction whose identity it reproduces.

In [9], the density matrices.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
image = left.imshow(np.abs(rho_det), cmap="viridis", vmin=0.0)
left.set_title("$|\\rho_{qp}|$ of the determinant")
left.set_xticks(range(M))
left.set_yticks(range(M))
left.set_xlabel("$p$")
left.set_ylabel("$q$")
fig.colorbar(image, ax=left, shrink=0.85)
```

The left panel is a heat map of `np.abs(rho_det)`, the sizes $|\rho_{qp}|$ of the 16 complex entries of the determinant's density matrix, on the colour scale viridis (dark blue for small, yellow for large values) starting at 0 (`vmin=0.0`); the column is p and the row is q. The tick marks are put at the four orbital numbers (`range(M)` is $0, 1, 2, 3$), and the colour scale is added beside the panel.

```
occupations_det = np.sort(np.linalg.eigvalsh(rho_det))[::-1]
occupations_th = np.sort(np.linalg.eigvalsh(rho_thermal))[::-1]
positions = np.arange(M)
right.bar(positions - 0.2, occupations_det, width=0.4, label="determinant")
right.bar(positions + 0.2, occupations_th, width=0.4, label="thermal, $T = 0.5$")
right.set_xticks(positions)
right.set_xlabel("eigenvalue number")
right.set_ylabel("occupation (eigenvalue of $\\rho$)")
right.legend()
save_figure(fig, "density_matrices",
```

`np.linalg.eigvalsh` computes the eigenvalues of a Hermitian matrix (real numbers, in increasing order); `np.sort(...)[::-1]` sorts them and reverses the order (the slice `[::-1]` steps backwards through an array), so the largest comes first. `positions` is $0, 1, 2, 3$. `right.bar(x, heights, width=0.4, ...)` draws a bar of the given height at each position x; shifting the two sets of bars by ∓ 0.2 puts them side by side. The `label` of each set appears in the legend (`right.legend()`). `save_figure` saves Figure 13c.3, and the check

```
check(np.allclose(occupations_det, [1, 1, 0, 0], atol=1e-14)
      and np.allclose(occupations_th, np.sort(f)[::-1], atol=1e-14),
      "the occupations are 1, 1, 0, 0 and the Fermi-Dirac numbers")
```

requires the eigenvalues to be exactly $1, 1, 0, 0$ for the determinant and the Fermi-Dirac numbers f_p for the ensemble.

What Figure 13c.3 shows. On the left, all 16 entries of the determinant's density matrix are nonzero (between about 0.08 and 0.85): in the randomly mixed basis the two occupied orbitals are spread over all four basis orbitals, so ρ does not look like a projector. Its eigenvalues on the right are nevertheless exactly 1, 1, 0, 0 (blue bars): the determinant occupies two orbitals completely and the others not at all. The thermal ensemble (orange bars) has the four Fermi-Dirac numbers, about 0.88, 0.65, 0.31 and 0.08 for the levels -1, -0.3, 0.4, 1.2 at $T = 0.5$: every level is partly occupied, the lower ones more.

In [10], the energy of a determinant.

```
chi = np.linalg.qr(rng.normal(size=(6, M)))[0]  # four orthonormal orbitals on 6 points
W_raw = rng.normal(size=(6, 6))
W_pair = (W_raw + W_raw.T) / 2.0  # a symmetric interaction W(x, x')
h_raw = rng.normal(size=(M, M))
h_one = (h_raw + h_raw.T) / 2.0  # a symmetric one-body matrix
# w[p,q,r,s] = sum_{x,x'} chi_p(x) chi_q(x') W(x,x') chi_r(x) chi_s(x')
w = np.einsum("xp,yq,xy,xr,ys->pqrs", chi, chi, W_pair, chi, chi)
```

The QR factorisation of a random 6×4 table gives four orthonormal real orbitals on six points. A random table plus its transpose, halved, is symmetric: a pair interaction $W(x, x')$ with the symmetry of a real interaction, and a one-body matrix h_{pq}. `np.einsum` computes a sum of products written as a pattern: here, for every (p, q, r, s), the sum over the points x and y of $\chi_p(x)\chi_q(y)W(x, y)\chi_r(x)\chi_s(y)$, which is the two-body integral w_{pqrs} of Section 13.4 on the six points.

```
H_many = sum(h_one[p, q] * a_dag[p] @ a[q] for p, q in itertools.product(range(M),
                                                                        repeat=2))
H_many = H_many + 0.5 * sum(w[p, q, r, s] * a_dag[p] @ a_dag[q] @ a[s] @ a[r]
                            for p, q, r, s in itertools.product(range(M), repeat=4))
basis_det = a_dag[0] @ a_dag[1] @ vacuum  # occupies the basis orbitals 0 and 1
direct = basis_det @ H_many @ basis_det
formula = sum(h_one[k, k] for k in (0, 1)) + 0.5 * sum(
    w[k, l, k, l] - w[k, l, l, k] for k in (0, 1) for l in (0, 1))
```

`H_many` is the 16×16 matrix of $\hat{H} = \sum h_{pq} a_p^\dagger a_q + \frac{1}{2} \sum w_{pqrs} a_p^\dagger a_q^\dagger a_s a_r$, built in two statements (the one-body part, then the two-body part added to it). `basis_det` is the column of the state

$a_0^\dagger a_1^\dagger|0\rangle$; since all matrices here are real, `basis_det @ H_many @ basis_det` (row times matrix times column) is the expectation value `direct` of $\hat{H}$ in this determinant of the basis orbitals 0 and 1. `formula` is $\sum_a h_{aa} + \frac{1}{2}\sum_{a,b}(w_{abab} - w_{abba})$ of Section 13.5, with the sums over $a, b \in \{0, 1\}$ written as generator expressions.

```
say(f"<Phi|H|Phi> = {direct:.10f};  sum h_aa + (1/2) sum (w_abab - w_abba) = "
    f"{formula:.10f}")
check(abs(direct - formula) < 1e-12,
      "the energy of a determinant is one-body + direct - exchange")
```

The cell prints both numbers with ten decimals, -1.5322375649 and -1.5322375649, and checks that they agree to 10^{-12}.

In [11], the exact two-site model.

```
n_op = [a_dag[p] @ a[p] for p in range(M)]  # occupation operators
DOUBLE = n_op[0] @ n_op[2] + n_op[1] @ n_op[3]  # both electrons on one site
# The states with exactly one up (orbital 0 or 1) and one down (2 or 3) electron:
SECTOR = [s for s in range(DIM) if bit(s, 0) + bit(s, 1) == 1
          and bit(s, 2) + bit(s, 3) == 1]
```

With the orbitals numbered L up (0), R up (1), L down (2), R down (3), `DOUBLE` is $\hat{n}_{L\uparrow}\hat{n}_{L\downarrow} + \hat{n}_{R\uparrow}\hat{n}_{R\downarrow}$. The **list comprehension** `[s for s in range(DIM) if ...]` collects the state numbers with exactly one up and one down electron; it prints as $[5, 6, 9, 10]$: in binary 0101 (L up and L down: both on L), 0110 (R up, L down), 1001 (L up, R down) and 1010 (both on R).

```
def two_site(t, U, delta=0.0):
    """Ground-state energy, double occupancy and n_L of the two-site model."""
    H = -t * (a_dag[0] @ a[1] + a_dag[1] @ a[0] + a_dag[2] @ a[3] + a_dag[3] @ a[2])
    H = H + U * DOUBLE - 0.5 * delta * (n_op[0] + n_op[2]) \
        + 0.5 * delta * (n_op[1] + n_op[3])
    block = H[np.ix_(SECTOR, SECTOR)]  # the 4 x 4 block of the sector
    values, vectors = np.linalg.eigh(block)
    ground = vectors[:, 0]
    occupancy = ground @ DOUBLE[np.ix_(SECTOR, SECTOR)] @ ground
    n_left = ground @ (n_op[0] + n_op[2])[np.ix_(SECTOR, SECTOR)] @ ground
    return values[0], occupancy, n_left
```

The function builds the Hamiltonian of Section 13.7 term by term (the backslash at the end of a line continues the statement on the next line): hopping of each label between L and R, the repulsion U on doubly occupied sites, and the site energies $-\Delta/2$ on L and $+\Delta/2$ on R. `np.ix_(SECTOR, SECTOR)` selects the rows and columns of the four states of the sector, so `block` is a 4×4 matrix. Its lowest eigenvalue `values[0]` is the ground-state energy and the first column of `vectors` the ground state; the expectation values of `DOUBLE` and of $\hat{n}_L$ in it are the double occupancy and the density on L. The function returns the three numbers.

```
say(f"the sector holds the states {SECTOR}")
for U in (2.0, 4.0):
```

```
    E0, occupancy, n_left = two_site(1.0, U)
    report(f"U = {U:.0f}: exact E_0", f"{E0:.6f}")
    report(f"U = {U:.0f}: exact double occupancy", f"{occupancy:.6f}")
    check(abs(E0 - 0.5 * (U - np.sqrt(U ** 2 + 16.0))) < 1e-12
          and abs(n_left - 1.0) < 1e-12,
          f"U = {U:.0f}: E_0 = (U - sqrt(U^2 + 16 t^2))/2 and n_L = 1")
```

For $t = 1$ and $U = 2$ and 4 the cell prints the exact energy and double occupancy and checks the closed form $E_0 = \frac{1}{2}(U - \sqrt{U^2 + 16t^2})$ of Section 13.7 and the symmetric density $n_L = 1$. The results are $E_0 = -1.236068$ with double occupancy 0.276393 at $U = 2$, and -0.828427 with 0.146447 at $U = 4$.

In [12], Hartree-Fock, restricted and unrestricted.

```
def hf_energy(alpha, beta, U, t=1.0):
    """E(alpha, beta) of the determinant with up (cos a, sin a), down (cos b, sin b)."""
    return (-t * (np.sin(2 * alpha) + np.sin(2 * beta))
            + 0.5 * U * (1.0 + np.cos(2 * alpha) * np.cos(2 * beta)))
```

The formula $E(\alpha, \beta) = -t(\sin 2\alpha + \sin 2\beta) + \frac{U}{2}(1 + \cos 2\alpha \cos 2\beta)$ of Section 13.7.

```
H_test = -(a_dag[0] @ a[1] + a_dag[1] @ a[0] + a_dag[2] @ a[3] + a_dag[3] @ a[2]) \
    + 3.0 * DOUBLE  # t = 1, U = 3
worst = 0.0
for alpha, beta in ((0.3, 1.1), (0.7, 0.2), (np.pi / 4, np.pi / 4)):
    det = (np.cos(alpha) * a_dag[0] + np.sin(alpha) * a_dag[1]) @ (
        np.cos(beta) * a_dag[2] + np.sin(beta) * a_dag[3]) @ vacuum
    worst = max(worst, abs(det @ H_test @ det - hf_energy(alpha, beta, 3.0)))
check(worst < 1e-12, "the Hartree-Fock energy formula equals <Phi|H|Phi>")
```

For $t = 1$, $U = 3$ and three pairs of angles the cell builds the determinant $(\cos\alpha\, a^\dagger_{L\uparrow} + \sin\alpha\, a^\dagger_{R\uparrow})(\cos\beta\, a^\dagger_{L\downarrow} + \sin\beta\, a^\dagger_{R\downarrow})|0\rangle$ from the operator matrices and checks that its energy equals the formula.

```
angles = np.linspace(0.0, np.pi / 2, 1001)
A, B = np.meshgrid(angles, angles, indexing="ij")
U_values = np.linspace(0.0, 8.0, 33)
E_exact = np.array([two_site(1.0, U)[0] for U in U_values])
D_exact = np.array([two_site(1.0, U)[1] for U in U_values])
E_rhf = -2.0 + 0.5 * U_values  # restricted: alpha = beta = pi/4
E_uhf = np.array([hf_energy(A, B, U).min() for U in U_values])  # grid minimum
```

1001 angles from 0 to $\pi/2$; `np.meshgrid` makes the two 1001×1001 tables `A` and `B` of all pairs (α, β). For 33 repulsions $U = 0, 0.25, \ldots, 8$ the cell computes the exact energy and double occupancy, the restricted energy $-2t + U/2$, and the smallest value of $E(\alpha, \beta)$ over the whole grid of angles (`.min()`).

```
U_safe = np.where(U_values > 2.0, U_values, 1.0)  # avoids dividing by U = 0
E_closed = np.where(U_values <= 2.0, -2.0 + 0.5 * U_values, -2.0 / U_safe)
```

```
check(np.max(np.abs(E_uhf - E_closed)) < 1e-5,
      "the minimum over all determinants is -2t + U/2 (U <= 2t) and -2t^2/U")
check(np.all(E_exact <= E_uhf + 1e-12) and np.all(E_uhf <= E_rhf + 1e-12),
      "variational order: exact <= unrestricted HF <= restricted HF")
```

`np.where(condition, a, b)` takes `a` where the condition holds and `b` elsewhere. `E_closed` is the closed form of Section 13.7, $-2t + U/2$ for $U \le 2t$ and $-2t^2/U$ beyond; `U_safe` replaces the values $U \le 2$ (where the second formula is not used) by 1, so that no division by zero occurs. The first check compares the grid minimum with the closed form (the grid of angles allows an error below 10^{-5}); the second checks the variational order exact $\le$ unrestricted $\le$ restricted at every U (`np.all` is true when every entry is true).

```
E_c = E_exact - E_closed  # correlation energy
report("U = 2: correlation energy E_0 - E_HF", f"{E_c[8]:.6f}")
fine_U = np.linspace(2.0, 8.0, 60001)  # closed forms on a fine grid of U > 2t
fine_c = 0.5 * (fine_U - np.sqrt(fine_U ** 2 + 16.0)) + 2.0 / fine_U
report("U of the largest correlation energy (in size)",
       f"{fine_U[np.argmin(fine_c)]:.3f}")
```

The correlation energy at the entry 8, $U = 8 \cdot 0.25 = 2$, is -0.236068. On a fine grid of U beyond $2t$ the cell evaluates $E_0 - E_{HF} = \frac{1}{2}(U - \sqrt{U^2 + 16}) + 2/U$, and `np.argmin` finds the place of its most negative value: the correlation energy is largest in size at $U = 3.335\,t$.

In [13], the energies.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(U_values, E_rhf, "--", label="restricted HF $-2t + U/2$")
left.plot(U_values, E_closed, "-.", label="unrestricted HF")
left.plot(U_values, E_exact, color="black", lw=2.0, label="exact $E_0$")
left.axvline(2.0, color="gray", lw=0.8)
left.set_xlabel("repulsion $U/t$")
left.set_ylabel("ground-state energy ($t$)")
left.legend(fontsize=8)
```

`left.plot(x, y, style, label=...)` draws the points (x, y) joined by a line: the restricted energy $-2t + U/2$ dashed (the style string of two hyphens), the best determinant's energy in closed form dash-dotted (`"-."`), and the exact energy as a thick black line (`lw=2.0`). `axvline(2.0, ...)` draws a thin grey vertical line at $U = 2t$, where restricted and unrestricted Hartree-Fock separate. The axis labels give the units: U is divided by t, and the energies are in units of t. `legend(fontsize=8)` lists the three labels in small letters.

```
right.plot(U_values, E_c, "o-", ms=3, color="black")
right.set_xlabel("repulsion $U/t$")
right.set_ylabel("correlation energy $E_0 - E_{HF}$ ($t$)")
fig.subplots_adjust(wspace=0.32)  # room between the panels for the axis label
fig.suptitle("Two electrons on two sites: exact versus Hartree-Fock")
save_figure(fig, "two_site_energies",
```

The right panel draws the correlation energy at the 33 values of U as dots joined by a line (`"o-"`; `ms=3` is the size of the dots in points). `fig.subplots_adjust(wspace=0.32)` widens the gap between

the panels so that the label of the right vertical axis does not touch the left panel, and `fig.suptitle` writes one title above both panels. `save_figure` saves Figure 13c.4. Its caption is not a fixed text: one of its lines is an f-string with the expression `fine_U[np.argmin(fine_c)]:.1f` in braces, which writes the computed place of the largest correlation energy, rounded to one decimal (3.3), into the caption.

What Figure 13c.4 shows. On the left, all three energies start at $-2t$ for $U = 0$ (both electrons in the bonding orbital, no repulsion). The restricted energy rises along the straight line $-2t + U/2$; the best determinant follows it up to $U = 2t$ and then bends away towards 0 as $-2t^2/U$; the exact energy lies below both at every $U > 0$ and also approaches 0 for large U. On the right, the correlation energy is zero at $U = 0$, falls to its most negative value, about $-0.337\,t$, near $U = 3.3t$, and then rises slowly towards zero: at large U the electrons sit on different sites anyway, and the unrestricted determinant describes that well.

In [14], the double occupancy.

```
D_uhf = np.where(U_values <= 2.0, 0.5, 2.0 / U_safe ** 2)
```

The unrestricted double occupancy is $\frac{1}{2}(1 + c_\alpha c_\beta)$ with $c_\beta = -c_\alpha$ and $s_\alpha = 2t/U$, that is $\frac{1}{2}(1 - c_\alpha^2) = \frac{1}{2}s_\alpha^2 = 2t^2/U^2$ for $U > 2t$, and $\frac{1}{2}$ below (the formula $E(\alpha, \beta)$ of Section 13.7 is U times this number plus the hopping energy).

```
fig, ax = plt.subplots()
ax.plot(U_values, 0.5 * np.ones_like(U_values), "--", label="restricted HF: 1/2")
ax.plot(U_values, D_uhf, "-.", label="unrestricted HF: $2t^2/U^2$ for $U > 2t$")
ax.plot(U_values, D_exact, color="black", lw=2.0, label="exact")
ax.set_xlabel("repulsion $U/t$")
ax.set_ylabel("double occupancy")
ax.set_title("How often both electrons sit on the same site")
ax.legend()
save_figure(fig, "double_occupancy",
```

One panel with three curves against U/t: the restricted value $\frac{1}{2}$ (`np.ones_like(U_values)` is an array of ones as long as `U_values`, so `0.5 * np.ones_like(U_values)` is the constant $\frac{1}{2}$ at every U) dashed, the unrestricted value dash-dotted, and the exact double occupancy as a thick black line; then the axis labels, the title, the legend, and `save_figure` for Figure 13c.5. The cell then checks

```
check(abs(D_exact[8] - 0.276393) < 1e-6 and abs(D_exact[16] - 0.146447) < 1e-6,
      "exact double occupancy 0.276393 at U = 2t and 0.146447 at U = 4t")
check(np.all(np.diff(D_exact) < 0), "the exact double occupancy falls as U grows")
```

the two worked values (entries 8 and 16 are $U = 2$ and $U = 4$) and that the exact double occupancy falls at every step of U (`np.diff` gives the differences of neighbouring entries).

What Figure 13c.5 shows. All three curves start at $\frac{1}{2}$ for $U = 0$. The exact double occupancy falls smoothly from the start: the exact electrons begin to avoid each other at any repulsion, while each label stays shared equally between the sites. The restricted determinant stays at $\frac{1}{2}$ for every U. The unrestricted one stays at $\frac{1}{2}$ up to $U = 2t$, then drops steeply as $2t^2/U^2$, crosses the exact curve near $U = 3.3t$ and lies below it beyond: by breaking the left-right symmetry it overshoots.

In [15], the Hartree-Fock energy landscape.

```
E_map = hf_energy(A, B, 4.0)
index = np.unravel_index(np.argmin(E_map), E_map.shape)
```

`E_map` is $E(\alpha, \beta)$ at $U = 4t$ on the whole grid of angles; `np.argmin` finds the position of its smallest value in the table read as one long list, and `np.unravel_index` turns that position back into a row and a column, the pair `index`.

```
fig, ax = plt.subplots(figsize=(6.0, 5.0))
contours = ax.contourf(A, B, E_map, levels=30, cmap="viridis")
fig.colorbar(contours, ax=ax, label="$E(\\alpha, \\beta)$ ($t$)")
ax.plot([np.pi / 4], [np.pi / 4], "wx", ms=10, label="restricted (saddle)")
ax.plot([angles[index[0]], angles[index[1]]], [angles[index[1]], angles[index[0]]],
        "r*", ms=12, label="unrestricted minima")
```

`ax.contourf(A, B, E_map, levels=30, ...)` colours the plane of the angles (α, β) in 30 bands of equal energy (a **contour map**: each band joins the points with nearly the same value), and the colour scale is labelled with the energy and its unit t. The style `"wx"` draws a white cross (w white, x cross) at the restricted point $(\pi/4, \pi/4)$, and `"r*"` red stars at the grid minimum $(\alpha, \beta) =$ `(angles[index[0]], angles[index[1]])` and at its mirror image with the two angles exchanged (the two lists hold the horizontal and the vertical coordinates of both points); `ms` sets the marker sizes.

```
ax.set_xlabel("$\\alpha$ (up orbital)")
ax.set_ylabel("$\\beta$ (down orbital)")
ax.set_title("Hartree-Fock energy landscape at $U = 4t$")
ax.legend(loc="upper right", fontsize=8)
save_figure(fig, "hf_landscape",
```

Axis labels, title, the legend in the upper right corner, and `save_figure` for Figure 13c.6. Then

```
check(abs(E_map.min() + 0.5) < 1e-5 and abs(hf_energy(np.pi / 4, np.pi / 4, 4.0)
                                            - 0.0) < 1e-12,
      "at U = 4t: minimum -0.5 t, restricted value 0")
```

requires the smallest value on the grid to be $-2t^2/U = -0.5$ (to 10^{-5}, the accuracy of the grid of angles) and the restricted value to be $-2 + 4/2 = 0$.

What Figure 13c.6 shows. The landscape is lowest (dark) in two basins, around $(\alpha, \beta) \approx (0.26, 1.31)$ and its mirror image $(1.31, 0.26)$, where the energy is $-0.5t$; there $\beta \approx \pi/2 - \alpha$: the up electron sits mostly on one site and the down electron mostly on the other. The restricted point at $(0.785, 0.785)$ lies between the basins on a pass: moving along the line $\beta = \pi/2 - \alpha$ lowers the energy, moving along $\beta = \alpha$ raises it, so it is a saddle with the energy 0. The highest energies (yellow) are in the corners $(0, 0)$ and $(\pi/2, \pi/2)$, where both electrons sit on the same site.

In [16], the Hohenberg-Kohn map and the exact Kohn-Sham potential.

```
deltas = np.linspace(-4.0, 4.0, 81)
density_maps = {U: np.array([two_site(1.0, U, d)[2] for d in deltas])
```

```
                for U in (0.0, 2.0, 4.0)}
for U, n_left in density_maps.items():
    check(np.all(np.diff(n_left) > 0),
          f"U = {U:.0f}: n_L increases strictly with Delta (one-to-one map)")
```

81 site-energy differences Δ from -4 to 4 in steps of 0.1. The **dictionary comprehension** `{U: ... for U in ...}` stores, for $U = 0, 2, 4$, the exact density n_L of every Δ (the third number returned by `two_site`). The three checks require each map to rise strictly: different Δ give different densities (Section 13.8).

```
def kohn_sham_delta(n_left, t=1.0):
    """The site-energy difference of non-interacting electrons with density n_L."""
    excess = n_left - 1.0
    return 2.0 * t * excess / np.sqrt(1.0 - excess ** 2)


def free_density(delta_s, t=1.0):
    """n_L of two non-interacting electrons with the site energies -+delta_s/2."""
    values, vectors = np.linalg.eigh(np.array([[-delta_s / 2, -t],
                                               [-t, delta_s / 2]]))
    return 2.0 * vectors[0, 0] ** 2
```

`kohn_sham_delta` is the inversion formula $\Delta_s = 2t\,x/\sqrt{1-x^2}$, $x = n_L - 1$, of Section 13.10. `free_density` solves the non-interacting problem directly: the lowest eigenvector of the 2×2 matrix, whose L component squared, times two electrons, is n_L.

```
delta_s = {U: kohn_sham_delta(n) for U, n in density_maps.items()}
rebuilt = max(abs(free_density(ds) - n) for U in density_maps
              for ds, n in zip(delta_s[U], density_maps[U]))
check(rebuilt < 1e-12, "the Kohn-Sham site energies reproduce the exact densities")
check(np.max(np.abs(delta_s[0.0] - deltas)) < 1e-12,
      "without interaction the Kohn-Sham potential is the true one")
```

For every U and every exact density the cell computes Δ_s, puts it back into the non-interacting problem and checks that the exact density comes out (to 10^{-12}); for $U = 0$ the Kohn-Sham potential must be the true one, $\Delta_s = \Delta$.

In [17], the density map.

```
fig, ax = plt.subplots()
for U, style in ((0.0, ":"), (2.0, "--"), (4.0, "-")):
    ax.plot(deltas, density_maps[U], style, label=f"$U = {U:.0f}t$")
ax.axhline(1.0, color="gray", lw=0.8)
ax.set_xlabel("site-energy difference $\\Delta$ ($t$)")
ax.set_ylabel("exact density on the left site $n_L$")
ax.set_title("Hohenberg-Kohn on two sites: the potential determines the density")
ax.legend()
save_figure(fig, "density_map",
```

The loop draws $n_L(\Delta)$ for $U = 0$ (dotted, `":"`), $2t$ (dashed) and $4t$ (solid); the label writes U without decimals (`:.0f`). `axhline(1.0, ...)` draws a thin grey horizontal line at $n_L = 1$, the density of the symmetric sites. Then the axis labels with the unit t of Δ, the title, the legend, and `save_figure` for Figure 13c.7.

What Figure 13c.7 shows. All three curves pass through $n_L = 1$ at $\Delta = 0$ and rise strictly from left to right, so no horizontal line meets a curve twice: each density belongs to exactly one Δ, which is the Hohenberg-Kohn statement on two sites. Without repulsion the curve is steep, from about 0.11 to 1.89 over $-4 \le \Delta \le 4$ (it is $1 + \Delta/\sqrt{\Delta^2 + 4}$); with $U = 4t$ it only reaches about 0.55 and 1.45, because the repulsion opposes putting both electrons on the lower site.

In [18], the mean field and the parts of the exact Kohn-Sham potential.

```
def mean_field_excess(delta, U, t=1.0):
    """The self-consistent x = n_L - 1 of restricted Hartree + exchange."""
    low, high = -1.0, 1.0
    for _ in range(80):  # bisection on x - G(x), which increases with x
        middle = 0.5 * (low + high)
        shifted = delta - U * middle
        if middle - shifted / np.sqrt(shifted ** 2 + 4 * t * t) > 0:
            high = middle
        else:
            low = middle
    return 0.5 * (low + high)
```

In the restricted mean field each electron feels U times the density of the other label on each site, which shifts the site-energy difference to $\Delta - U(n_L - 1)$; the self-consistent excess $x = n_L - 1$ solves $x = G(x) = (\Delta - Ux)/\sqrt{(\Delta - Ux)^2 + 4t^2}$ (derived in Section 13.21). The function finds the root by **bisection**: it starts with the interval $[-1, 1]$, which contains the root, and 80 times halves it, keeping the half in which $x - G(x)$ changes sign (this difference increases with x, because G decreases, so there is exactly one root). `for _ in range(80)` repeats 80 times; the name `_` says that the counter is not used.

```
mf_delta_s = np.array([d - 4.0 * mean_field_excess(d, 4.0) for d in deltas])
nonzero = np.abs(deltas) > 1e-9  # every Delta except 0
hx_part = {U: -U * (density_maps[U] - 1.0) for U in (2.0, 4.0)}  # at the exact n_L
c_part = {U: delta_s[U] - deltas - hx_part[U] for U in (2.0, 4.0)}  # the rest
report("U = 4t, Delta = 2t: Hartree-exchange part", f"{hx_part[4.0][60]:.6f}")
report("U = 4t, Delta = 2t: correlation part", f"{c_part[4.0][60]:.6f}")
```

`mf_delta_s` is the site-energy difference of the self-consistent mean field at $U = 4t$. `nonzero` is a list of true and false values that is false only at $\Delta = 0$. For $U = 2t$ and $4t$, the Hartree-exchange part of the screening, evaluated at the EXACT density, is $\Delta_{Hx} = -U(n_L - 1)$, and the correlation part is the rest, $\Delta_c = \Delta_s - \Delta - \Delta_{Hx}$. At $U = 4t$, $\Delta = 2t$ (entry 60) they are -0.588239 and -1.114408: correlation does more than half of the screening there.

```
for U in (2.0, 4.0):
    check(np.all(c_part[U][nonzero] * deltas[nonzero] < 0.0),
          f"U = {U:.0f}t: the correlation part has the sign opposite to Delta")
check(np.all(np.abs(mf_delta_s[nonzero]) > np.abs(delta_s[4.0][nonzero])),
      "U = 4t: the mean field screens less than the exact Kohn-Sham potential")
```

`c_part[U][nonzero]` keeps the entries with $\Delta \neq 0$. The checks require the correlation part to have the sign opposite to Δ (it screens further, like the Hartree-exchange part) and the mean-field potential to be larger in size than the exact Kohn-Sham one (it screens less).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(11.0, 4.4))
left.plot(deltas, deltas, ":", color="gray", label="$\\Delta_s = \\Delta$ ($U = 0$)")
left.plot(deltas, delta_s[2.0], "--", label="exact Kohn-Sham, $U = 2t$")
left.plot(deltas, delta_s[4.0], color="black", lw=2.0,
          label="exact Kohn-Sham, $U = 4t$")
left.plot(deltas, mf_delta_s, "-.", label="mean field, $U = 4t$")
left.set_xlabel("true site-energy difference $\\Delta$ ($t$)")
left.set_ylabel("Kohn-Sham site-energy difference $\\Delta_s$ ($t$)")
left.set_title("The exact Kohn-Sham potential")
left.legend(fontsize=8)
```

The left panel draws, against the true Δ: the grey dotted line $\Delta_s = \Delta$ (the answer without interaction), the exact Kohn-Sham Δ_s for $U = 2t$ (dashed) and $U = 4t$ (thick black), and the self-consistent mean field for $U = 4t$ (dash-dotted), with axis labels, title and legend.

```
right.plot(deltas, delta_s[4.0] - deltas, color="black", lw=2.0,
           label="total screening $\\Delta_s - \\Delta$")
right.plot(deltas, hx_part[4.0], "--", label="Hartree-exchange $\\Delta_{Hx}$")
right.plot(deltas, c_part[4.0], color="tab:red", label="correlation $\\Delta_c$")
right.axhline(0.0, color="gray", lw=0.8)
right.set_xlabel("true site-energy difference $\\Delta$ ($t$)")
right.set_ylabel("parts of $\\Delta_s - \\Delta$ ($t$)")
right.set_title("Its parts at the exact density, $U = 4t$")
right.legend(fontsize=8)
save_figure(fig, "ks_inversion",
```

The right panel draws, for $U = 4t$, the total screening $\Delta_s - \Delta$ (thick black), the Hartree-exchange part (dashed) and the correlation part (red; `"tab:red"` is the red of matplotlib's standard colours), with a grey zero line, axis labels, title and legend. `save_figure` saves Figure 13c.8, and the last check of the cell

```
check(np.all(np.abs(delta_s[4.0][nonzero]) < np.abs(deltas[nonzero])),
      "the repulsion screens the potential: |Delta_s| < |Delta|")
```

requires $|\Delta_s| < |\Delta|$ for every $\Delta \neq 0$ at $U = 4t$: the repulsion screens the potential.

What Figure 13c.8 shows. On the left, every interacting curve is flatter than the dotted line $\Delta_s = \Delta$: the non-interacting electrons need a smaller site-energy difference than the true one to reproduce the exact density, because the repulsion, which the Kohn-Sham electrons do not have, already pushes against piling up on the lower site. At $\Delta = 4t$ the exact Δ_s is about $2.3t$ for $U = 2t$ and about $1.0t$ for $U = 4t$; the mean field for $U = 4t$ gives about $1.55t$, between the two: it screens less than the exact potential. On the right, at $U = 4t$, both parts of the screening have the sign opposite to Δ; the correlation part is the larger one for $|\Delta|$ up to about $3.4t$ (at $\Delta = 2t$: -1.114 against -0.588, the numbers printed by the cell), and the Hartree-exchange part beyond.

In [19], the last check.

```
figure_names = ["slater_determinants", "operator_matrices", "density_matrices",
                "two_site_energies", "double_occupancy", "hf_landscape",
                "density_map", "ks_inversion"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13c_{k}_{name}.png").is_file()]
check(missing == [], "all eight figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13c_8_ks_inversion.png").is_file(),
      "the figure file 13c_8_ks_inversion.png exists")
all_checks_passed()
```

`enumerate(figure_names, 1)` pairs each name with its number $k = 1, 2, \ldots$; the list comprehension collects the names whose file `13c_k_name.png` does not exist, and the first check requires that list to be empty. The second check names the last figure file, and `all_checks_passed()` prints ALL 33 CHECKS PASSED (notebook 13c): three checks in In [2], one in In [3], two in In [4], three in In [6], one in In [7], two in In [8], one each in In [9] and In [10], two in In [11], three in In [12], two in In [14], one in In [15], five in In [16], four in In [18] and two in In [19].

13.15 The uniform gas, the local density approximation and exact local exchange

To use the Kohn-Sham equations we need an approximation for $E_{xc}[n]$. The oldest one borrows it from the only many-body system whose properties are known accurately: the **uniform gas**, infinitely many particles spread with the same density everywhere. This section computes, in three dimensions, its density matrix, its kinetic energy and its exchange energy, and shows why the exchange of a contact interaction is special.

Spherical coordinates. The integrals below are done in spherical coordinates: a point $\mathbf{r} = (x, y, z)$ is given by its distance r from the origin, the angle θ from the z axis and the angle ϕ around it, $x = r\sin\theta\cos\phi$, $y = r\sin\theta\sin\phi$, $z = r\cos\theta$. A small box with the edges dr, $r\,d\theta$ and $r\sin\theta\,d\phi$ has the volume $d^3r = r^2\sin\theta\,dr\,d\theta\,d\phi$. With $u = \cos\theta$ (so $du = -\sin\theta\,d\theta$, and θ from 0 to π is u from 1 to -1),

$$\int f\,d^3r = \int_0^\infty r^2\,dr \int_{-1}^{1} du \int_0^{2\pi} d\phi\; f,$$

and for a function of r alone this is $4\pi\int_0^\infty r^2 f\,dr$. The same holds for integrals over a wave vector $\mathbf{k}$, with $k = |\mathbf{k}|$ in place of r, and the axis may point in any direction.

Plane waves in a box. Put the gas in a cube of side ℓ and volume $V = \ell^3$ whose opposite faces are glued together (a **periodic box**). The orbitals of a free particle are the **plane waves** $e^{i\mathbf{k}\cdot\mathbf{r}}/\sqrt{V}$ with the kinetic energy $\frac{1}{2}k^2$, and periodicity allows only $\mathbf{k} = (2\pi/\ell)\,\mathbf{m}$ with three integers $\mathbf{m} = (m_x, m_y, m_z)$. Each allowed $\mathbf{k}$ occupies a little cube of volume $(2\pi/\ell)^3 = (2\pi)^3/V$, so for a large box a sum over the allowed vectors becomes an integral, $\sum_{\mathbf{k}} \to V\int d^3k/(2\pi)^3$.

The Fermi sphere. The non-interacting ground state fills all $\mathbf{k}$ with $|\mathbf{k}| < k_F$ (the **Fermi wave number**), once for each of the g labels. Counting, line by line:

$$N_\sigma = V\,\frac{\frac{4}{3}\pi k_F^3}{(2\pi)^3}, \qquad n_\sigma = \frac{N_\sigma}{V} = \frac{k_F^3}{6\pi^2}, \qquad n = g\,n_\sigma = \frac{g\,k_F^3}{6\pi^2}.$$

The first is the number of occupied plane waves of one label (the volume of the sphere divided by the volume per $\mathbf{k}$); the second divides by V and simplifies $\frac{4}{3}\pi/(8\pi^3) = 1/(6\pi^2)$; the third adds the g labels. The kinetic energy per volume is $g\int_{k<k_F} \frac{d^3k}{(2\pi)^3}\frac{k^2}{2} = \frac{g}{2\pi^2}\cdot\frac{k_F^5}{10}$ (spherical coordinates: $4\pi\int_0^{k_F} k^2\cdot\frac{k^2}{2}\,dk/(8\pi^3)$). For $g=2$, eliminating $k_F = (3\pi^2 n)^{1/3}$,

$$e_{kin}(n) = C_F\, n^{5/3}, \qquad C_F = \tfrac{3}{10}\,(3\pi^2)^{2/3} = 2.871234 \quad \text{(Notebook 13b, In [15])}.$$

The **Thomas-Fermi** model (1927) took the kinetic energy of every system to be $\int e_{kin}(n(\mathbf{r}))\,d^3r$, as if each small piece of it were a piece of uniform gas. It is too crude; the Kohn-Sham scheme keeps the kinetic energy exact through T_s and approximates only E_{xc}. Notebook 13a (Section 13.32) compares a Thomas-Fermi density with a Kohn-Sham density.

The density matrix of one label, line by line. For one label the density matrix of the filled sphere is $\rho_\sigma(\mathbf{r},\mathbf{r}') = \sum_{|\mathbf{k}|<k_F} \frac{e^{i\mathbf{k}\cdot\mathbf{r}}}{\sqrt{V}}\frac{e^{-i\mathbf{k}\cdot\mathbf{r}'}}{\sqrt{V}}$, a function of the separation $\mathbf{R} = \mathbf{r}-\mathbf{r}'$ only. For a large box, with $R = |\mathbf{R}|$ and spherical coordinates around $\mathbf{R}$ (so $\mathbf{k}\cdot\mathbf{R} = kRu$):

$$\begin{aligned}
\rho_\sigma(R) &= \int_{k<k_F} \frac{d^3k}{(2\pi)^3}\, e^{i\mathbf{k}\cdot\mathbf{R}} = \frac{1}{(2\pi)^3}\int_0^{k_F} k^2\,dk\int_{-1}^{1} du\int_0^{2\pi} d\phi\; e^{ikRu} \\
&= \frac{2\pi}{(2\pi)^3}\int_0^{k_F} k^2\,\frac{e^{ikR}-e^{-ikR}}{ikR}\,dk = \frac{1}{2\pi^2 R}\int_0^{k_F} k\,\sin(kR)\,dk \\
&= \frac{1}{2\pi^2 R}\Big[\frac{\sin(kR)}{R^2} - \frac{k\cos(kR)}{R}\Big]_0^{k_F} = \frac{\sin(k_FR) - k_FR\cos(k_FR)}{2\pi^2R^3}.
\end{aligned}$$

The first step replaces the sum by the integral; the second writes it in spherical coordinates; the third does the ϕ integral (2π) and the u integral (the antiderivative of e^{ikRu} in u is $e^{ikRu}/(ikR)$); the fourth uses $e^{i\alpha}-e^{-i\alpha} = 2i\sin\alpha$ and simplifies; the fifth integrates by parts ($\int k\sin(kR)\,dk = -k\cos(kR)/R + \int\cos(kR)/R\,dk$); the sixth inserts the limits and collects. Dividing by $n_\sigma = k_F^3/(6\pi^2)$ and writing $s = k_FR$:

$$\frac{\rho_\sigma(R)}{n_\sigma} = F(k_FR), \qquad F(s) = \frac{3\,(\sin s - s\cos s)}{s^3}.$$

Near $s=0$ the Taylor series $\sin s = s - s^3/6 + s^5/120 - s^7/5040$ and $s\cos s = s - s^3/2 + s^5/24 - s^7/720$ give $\sin s - s\cos s = s^3/3 - s^5/30 + s^7/840$, so $F(s) = 1 - s^2/10 + s^4/280 + \ldots$, and $F(0) = 1$: at zero separation the density matrix is the density. Notebook 13b (In [3], Figure 13b.1) compares F with the sums over the plane waves of finite boxes: along the x axis only m_x matters, the sine parts of m_x and $-m_x$ cancel, and $\rho_\sigma(R)/n_\sigma = \frac{1}{N_\sigma}\sum_{m_x} c(m_x)\cos(2\pi m_xR/\ell)$, where $c(m_x)$ counts the occupied lattice points with this m_x; the difference from F shrinks from 0.037 to 0.0008 as the box grows from 251 to 137059 plane waves.

The exchange hole. By Wick's theorem (Section 13.4) the probability density of finding one fermion at $\mathbf{r}$ and another at $\mathbf{r}'$ is $n(\mathbf{r})n(\mathbf{r}') - \sum_\sigma |\rho_\sigma(\mathbf{r},\mathbf{r}')|^2$ (the second term is the exchange term of the four-operator formula; only equal labels contribute). With g equally occupied labels, $n_\sigma = n/g$, and divided by n^2:

$$g_{pair}(R) = 1 - \frac{g\,n_\sigma^2 F(k_FR)^2}{n^2} = 1 - \frac{1}{g}\,F(k_FR)^2.$$

At $R = 0$ it is $1 - 1/g$: a second fermion with the same label is never found at the same point, one with another label as often as without the Pauli principle. This dip is the **exchange hole** (Figure 13b.2); its size is about $1/k_F$.

Contact interaction in the uniform gas. With $w = g_c\,\delta(\mathbf{r} - \mathbf{r}')$, Section 13.5 gives per volume $e_H = \frac{g_c}{2} n^2$ and $e_x = -\frac{g_c}{2}\sum_\sigma n_\sigma^2 = -\frac{g_c}{2g} n^2$, so

$$e_H + e_x = \frac{g_c}{2}\, n^2 \Big(1 - \frac{1}{g}\Big) = \frac{g_c}{2}\, n^2\, g_{pair}(0)\,:$$

the interaction counts only the pairs that can meet, those of different labels (Figure 13b.3).

A finite range shrinking to a contact, line by line. Replace the contact by a Gaussian of range a with the same total strength, $w_a(R) = g_c\,(2\pi a^2)^{-3/2} e^{-R^2/(2a^2)}$, whose integral over all space is g_c. Its Hartree energy is still $\frac{g_c}{2} n^2$ (for a uniform density only the integral of w enters), but its exchange energy per volume is $e_x(a) = -\frac{1}{2}\sum_\sigma n_\sigma^2 \int w_a(R)\, F(k_F R)^2\, d^3R$ (Section 13.5 with $\rho_\sigma = n_\sigma F$). Hence

$$\frac{e_x(a)}{e_x(0)} = \frac{1}{g_c}\int w_a(R)\, F(k_F R)^2\, d^3R = \frac{1}{g_c}\int_0^\infty 4\pi R^2\, w_a(R)\, F(k_F R)^2\, dR.$$

For a small range only small R matter, where $F(s)^2 = (1 - s^2/10 + \dots)^2 = 1 - s^2/5 + \dots$:

$$\frac{e_x(a)}{e_x(0)} \approx \frac{1}{g_c}\int w_a \Big(1 - \frac{k_F^2 R^2}{5}\Big) d^3R = 1 - \frac{k_F^2}{5 g_c}\int w_a\, R^2\, d^3R = 1 - \frac{k_F^2}{5}\cdot 3a^2 = 1 - \frac{3}{5}\,(k_F a)^2.$$

The first step inserts the expansion; the second uses $\int w_a\, d^3R = g_c$; the third uses $\int w_a R^2\, d^3R = 3a^2 g_c$ (the Gaussian has the variance a^2 along each of the three axes, and $R^2 = x^2 + y^2 + z^2$). So the exchange energy of a finite-range interaction approaches the contact value when the range is much smaller than the size $1/k_F$ of the exchange hole (Notebook 13b, In [8], Figure 13b.4).

For comparison: the Coulomb interaction. For electrons ($g = 2$, $w = 1/R$ in atomic units) the same formula gives

$$e_x = -\tfrac{1}{2}\cdot 2\, n_\sigma^2 \int_0^\infty 4\pi R^2\, \frac{F(k_F R)^2}{R}\, dR = -\frac{4\pi n_\sigma^2}{k_F^2}\int_0^\infty s\, F(s)^2\, ds,$$

by substituting $s = k_F R$. The integral is 9/4: COMPUTED by Notebook 13b (In [10]) to 10^{-7} with Simpson's rule up to $s = 4000$ plus the tail beyond (for large s, $F \approx -3\cos s/s^2$, so the integrand is about $9\cos^2 s/s^3$, whose average $\cos^2 = \frac{1}{2}$ gives the tail $9/(4\cdot 4000^2)$); it can also be evaluated exactly by Fourier transforms, which this book does not need. With $n_\sigma = n/2$ and $k_F = (3\pi^2 n)^{1/3}$ this is **Dirac's exchange energy** (1930),

$$e_x(n) = -\frac{3}{4}\Big(\frac{3}{\pi}\Big)^{1/3} n^{4/3} = -0.738559\, n^{4/3} \quad \text{(Notebook 13b, In [10])}.$$

It is a function of the local density because the gas is uniform; the contact exchange $-\frac{g_c}{4} n^2$ ($g = 2$) is a function of the local density because the interaction has no range (Figure 13b.5 compares the two per particle: slopes 1 and 1/3 on logarithmic axes).

Simpson's rule. Several notebooks of this chapter integrate with **Simpson's rule**. On three equally spaced points $-h, 0, h$ it integrates the parabola $a+bx+cx^2$ through the three values exactly: $\int_{-h}^{h}(a+bx+cx^2)\,dx = 2ah + \frac{2}{3}ch^3$; the values give $f(0) = a$ and $f(h) + f(-h) = 2a + 2ch^2$, so $ch^2 = \frac{1}{2}(f(h)+f(-h))-a$, and inserting, $\int = 2ah+\frac{h}{3}(f(h)+f(-h))-\frac{2}{3}ah = \frac{h}{3}[f(-h)+4f(0)+f(h)]$. Adding such panels over an even number of intervals gives $\frac{h}{3}[f_0+4f_1+2f_2+4f_3+\cdots+4f_{K-1}+f_K]$: the end values once, the odd ones four times, the inner even ones twice. (It is even exact for cubic polynomials, because the term x^3 integrates to zero on a symmetric panel.)

The local density approximation. A real density varies in space. The **local density approximation** (LDA) treats each small volume as a piece of uniform gas with the local density:

$$E_{xc}^{LDA}[n] = \int e_{xc}(n(\mathbf{r}))\,d^3r, \qquad v_{xc}^{LDA}(\mathbf{r}) = \left.\frac{de_{xc}}{dn}\right|_{n(\mathbf{r})}$$

(example (ii) of Section 13.9). For electrons its exchange part is Dirac's formula, $v_x = -(3/\pi)^{1/3}n^{1/3}$; its correlation part is taken from numerical simulations of the uniform gas, which we quote as a fact of the literature and do not use in this book. The LDA is exact for a uniform density and an approximation for every other one.

For a contact interaction exchange needs no approximation. Put $w = g_c\,\delta(\mathbf{r}-\mathbf{r}')$ (independent of the labels) into the exchange integral of Section 13.5 for ANY determinant or non-interacting ensemble:

$$E_x = -\frac{g_c}{2}\int \sum_{\sigma,\sigma'} \left|\rho(\mathbf{r}\sigma,\mathbf{r}\sigma')\right|^2 d^3r.$$

This is exactly local: it needs only the $g \times g$ matrix $\rho(\mathbf{r}\sigma,\mathbf{r}\sigma')$ at each point. For orbitals of definite label the off-diagonal entries vanish and $E_x = -\frac{g_c}{2}\int\sum_\sigma n_\sigma^2\,d^3r$; evaluated with all the label densities it is exact, evaluated with the total density alone as $-\frac{g_c}{2g}\int n^2$ it is exact only when all labels are equally occupied. Notebook 13b checks this for a non-uniform determinant in a box (In [12]: the two-point Fock form and the local form both give -19.0000000000) and for orbitals that mix the labels (In [13]: the local form needs the whole 2×2 matrix; its diagonal alone misses 1.24 of the 8.74). An "exchange-only" local functional of this kind is the Hartree-Fock energy written as a density functional; what it leaves out is correlation.

Exchange favours unequal labels, line by line. In this exchange-only picture a gas with two labels can lower its interaction energy by occupying the labels unequally, at the price of kinetic energy. With the **polarisation** $\zeta = (n_\uparrow - n_\downarrow)/n$, so $n_\uparrow = \frac{n}{2}(1+\zeta)$ and $n_\downarrow = \frac{n}{2}(1-\zeta)$: one label alone ($g=1$) has the kinetic energy $\frac{3}{10}(6\pi^2)^{2/3}n_\sigma^{5/3}$ per volume (the formula above with $g=1$), and the contact interaction gives $g_c\,n_\uparrow n_\downarrow$ (Section 13.5). Adding,

$$e(\zeta) = C_F\,n^{5/3}\,\frac{(1+\zeta)^{5/3}+(1-\zeta)^{5/3}}{2} + \frac{g_c}{4}\,n^2\,(1-\zeta^2),$$

where the kinetic part was rewritten with $\frac{3}{10}(6\pi^2)^{2/3}(\frac{n}{2})^{5/3} = \frac{1}{2}C_F n^{5/3}$ (because $6^{2/3}\,2^{-5/3} = 3^{2/3}\,2^{2/3}\,2^{-5/3} = 3^{2/3}/2$) and $n_\uparrow n_\downarrow = \frac{n^2}{4}(1-\zeta^2)$. Differentiating twice at $\zeta = 0$: $\frac{d^2}{d\zeta^2}(1\pm\zeta)^{5/3} = \frac{5}{3}\cdot\frac{2}{3}(1\pm\zeta)^{-1/3} = \frac{10}{9}$ at $\zeta = 0$, so

$$e''(0) = \frac{10}{9}\,C_F\,n^{5/3} - \frac{g_c}{2}\,n^2,$$

which is negative (the unpolarised gas is unstable) exactly when $g_c\,n^{1/3} > \gamma_c = \frac{20}{9}C_F = \frac{2}{3}(3\pi^2)^{2/3} = 6.380520$. This is a property of the exchange-only functional; correlation, left out here, weakens it (Notebook 13b, In [15], Figure 13b.7).

13.16 The exchange of the 16-component field dirac16complex

This section carries Section 13.15 over to the field of this book. In this section and in Section 13.37, Ψ is the dirac16complex field and $x_0, \ldots, x_7$ are the author's coordinates: x_1, x_2, x_3 ordinary space, x_4 the time, x_5, x_6, x_7 the three extra times, which deflate exponentially, and x_0 the hidden direction.

The matrices. The author's gamma matrices $\gamma^{(x0)}, \ldots, \gamma^{(x7)}$ are eight REAL 16×16 matrices (entries 0 and ± 1) with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}\,1$, $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the order $x_0, \ldots, x_7$ (Chapter 4 builds them; `Revision/algebra/reports/wolfram-algebra.json`, checks reality and Clifford_relation, both PASS, as Notebook 13b checks in In [21]). So each space-like gamma squares to $+1$, $\gamma^{(x4)}$ squares to -1, and two different gammas anticommute. From them,

$$C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}, \qquad B = -i\,C\,\gamma^{(x4)}.$$

C is real and symmetric (check C_real_symmetric of the same report, PASS); B is the indefinite (Krein) form of Chapter 10, with the number density $n = \langle \Psi^\dagger B \Psi \rangle$. Four facts, line by line. (a) $C^2 = 1$: in $C^2 = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}\gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$ move the second $\gamma^{(x0)}$ to the left past three gammas (sign $(-1)^3$) and use $(\gamma^{(x0)})^2 = 1$; then move the second $\gamma^{(x1)}$ past two (sign $+1$), then the second $\gamma^{(x2)}$ past one (sign -1); the signs multiply to $(-1)(+1)(-1) = +1$ and every square is $+1$. (b) $\gamma^{(x4)}$ commutes with C: moving it through the four factors of C costs four sign changes, $(-1)^4 = 1$. (c) $B^2 = 1$: $B^2 = (-i)^2\,C\gamma^{(x4)}C\gamma^{(x4)} = -\,C\,C\,\gamma^{(x4)}\gamma^{(x4)} = -(+1)(-1) = 1$, by (b), (a) and $(\gamma^{(x4)})^2 = -1$. (d) $CBC = -i\,C\,C\,\gamma^{(x4)}C = -i\,\gamma^{(x4)}C = -i\,C\gamma^{(x4)} = B$ by (a) and (b); and $\mathrm{Tr}(BC) = -i\,\mathrm{Tr}(C\gamma^{(x4)}C) = -i\,\mathrm{Tr}(\gamma^{(x4)}C^2) = -i\,\mathrm{Tr}\,\gamma^{(x4)} = 0$, using that a trace does not change when the first factor is moved to the end, and that the trace of a single gamma vanishes ($\mathrm{Tr}\,\gamma^{(x4)} = \mathrm{Tr}(\gamma^{(x4)}\gamma^{(x1)}\gamma^{(x1)}) = -\mathrm{Tr}(\gamma^{(x1)}\gamma^{(x4)}\gamma^{(x1)}) = -\mathrm{Tr}(\gamma^{(x4)}\gamma^{(x1)}\gamma^{(x1)}) = -\mathrm{Tr}\,\gamma^{(x4)}$). Finally $\mathrm{Tr}\,1 = 16$.

The interaction and Wick's theorem. The Kohn-Sham model of Chapter 14 has the contact interaction $U(S) = \frac{\lambda}{2}S^2$ with the scalar density $S = \bar\Psi\Psi = \Psi^\dagger C\,\Psi$ (normal ordered, $\lambda > 0$ repulsive). Its expectation values follow the rule $\langle \Psi^\dagger M \Psi \rangle = \mathrm{Tr}(M\rho)$ with a local 16×16 one-body matrix ρ (Chapter 10), and the identity of Section 13.4 with the vertex $V = C$ gives, per unit volume,

$$e_H = \frac{\lambda}{2}\,(\mathrm{Tr}\,C\rho)^2, \qquad e_x = -\frac{\lambda}{2}\,\mathrm{Tr}(C\rho\,C\rho).$$

(For g ordinary labels the vertex is the unit matrix and $\rho = (n/g)\,1$, which gives back $e_x = -\frac{g_c}{2}\,g\,(n/g)^2 = -\frac{g_c}{2g}n^2$.)

The uniform good-sector gas, line by line. For the uniform gas of the good sector (no extra-time momentum, Chapter 14), with any occupation that is symmetric under $\mathbf{p} \to -\mathbf{p}$ and at any temperature, the Revision record finds $\rho = (nB + SC)/16$ (`Revision/kohn_sham/ks-theory.json`, exchange.uniformGas; Chapter 14 derives it). First the densities come back:

$$\mathrm{Tr}(B\rho) = \frac{n\,\mathrm{Tr}\,B^2 + S\,\mathrm{Tr}\,BC}{16} = \frac{16n + 0}{16} = n, \qquad \mathrm{Tr}(C\rho) = \frac{n\,\mathrm{Tr}\,CB + S\,\mathrm{Tr}\,C^2}{16} = S,$$

by linearity of the trace, $B^2 = C^2 = 1$ and $\mathrm{Tr}(BC) = \mathrm{Tr}(CB) = 0$. Then the exchange trace:

$$\begin{aligned}
\mathrm{Tr}(C\rho\,C\rho) &= \frac{1}{256}\,\mathrm{Tr}\big[C(nB + SC)\,C(nB + SC)\big] \\
&= \frac{1}{256}\big[n^2\,\mathrm{Tr}(CBCB) + nS\,\mathrm{Tr}(CBCC) + Sn\,\mathrm{Tr}(CCCB) + S^2\,\mathrm{Tr}(CCCC)\big] \\
&= \frac{1}{256}\big[n^2\,\mathrm{Tr}(B^2) + nS\,\mathrm{Tr}(CB) + Sn\,\mathrm{Tr}(CB) + S^2\,\mathrm{Tr}\,1\big] \\
&= \frac{16\,n^2 + 16\,S^2}{256} = \frac{n^2 + S^2}{16}.
\end{aligned}$$

The first line inserts ρ; the second multiplies out (four terms); the third uses $CBC = B$ in the first term and $C^2 = 1$ in the others; the fourth uses $\mathrm{Tr}\,B^2 = \mathrm{Tr}\,1 = 16$ and $\mathrm{Tr}(CB) = 0$. Hence

$$e_H = \frac{\lambda}{2}\,S^2, \qquad e_x = -\frac{\lambda}{32}\,(n^2 + S^2).$$

The Kohn-Sham potentials. The interaction energy per volume is $e_{int} = e_H + e_x = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2$. As in example (ii) of Section 13.9, the functional derivatives of $\int e_{int}$ are ordinary derivatives of e_{int}: with respect to S it shifts the mass, with respect to n it is a potential,

$$M_{eff} = m + \frac{\partial e_{int}}{\partial S} = m + \frac{15}{16}\,\lambda S, \qquad v_v = \frac{\partial e_{int}}{\partial n} = -\frac{1}{16}\,\lambda n.$$

For one filled 8-fold level at rest, $n = S$, the ratio is $e_x/e_H = -\frac{\lambda}{32}\cdot 2S^2/\frac{\lambda}{2}S^2 = -\frac{1}{8}$: the rule $E_x = -E_H/g$ of Section 13.5 with $g = 8$, the number of good-sector states per momentum.

Status. PROVED in the Revision record by two independent programs, and used unchanged by the Revision Kohn-Sham solver:

record file	checks or entries	result
`Revision/kohn_sham/reports/ks-theory-python.json`	exchange_uniform_gas, ks_potentials, filled_shell_ratio	PASS
`Revision/kohn_sham/reports/ks-theory-wolfram.json`	exchange_uniform_gas, ks_potentials	PASS
`Revision/kohn_sham/results/parameters.json`	theoryInputs: the coefficients $-1/32$, $15/16$, $-1/16$	used by the solver

Notebook 13b reproduces all of them exactly from the gamma matrices. **What is approximate.** The record's functional is Hartree plus this uniform-gas exchange, with no correlation term. For the Kohn-Sham determinants of Chapter 14, which are not uniform, the exact Fock exchange of a closed shell is still local but differs from the uniform-gas form by $+\frac{\lambda}{32}Q^2$ per volume, where Q is a further density (exactFockSlab in `Revision/kohn_sham/ks-theory.json`, labelled DIAGNOSTIC there; check exchange_slab_exact_fock of `Revision/kohn_sham/reports/ks-theory-python.json`, PASS; the solver reports this difference in the column deltaE_x_exact_fock_diag of `Revision/kohn_sham/results/exx/exact-fock-variant.csv`). Using the uniform-gas exchange and leaving out correlation is therefore the approximation of the dirac16complex Kohn-Sham model, as the omission of correlation is the approximation of the toy models of this chapter. Notebook 13b reads every record entry quoted in this section and checks it (In [17] and In [18] for the formulas and the three sympy checks; In [21] for the algebra checks, the Wolfram checks, exchange_slab_exact_fock in both reports, the DIAGNOSTIC entry with its $+\frac{\lambda}{32}Q^2$, the missing correlation term and the column of the table), so that a change of the record would stop the notebook.

13.17 Example: exchange of a contact interaction

Notebook 13b computes everything of Sections 13.15 and 13.16: the density matrix of one label of the uniform gas from its closed form and from box sums, the exchange hole for 1, 2 and 8 labels, the contact energies and the rule $E_x = -E_H/g$, the approach of a Gaussian interaction to the contact limit, Dirac's Coulomb exchange for comparison, the exactness of the local formula for any determinant (also with label-mixing orbitals), the polarisation instability, and, exactly with fractions, the exchange of the 16-component field from the Revision gamma matrices, reproducing three checks of the Revision record; finally it reads and checks the further record entries that Sections 13.16, 13.37 and 13.38 quote (the independent Wolfram verification, the exact-exchange diagnostic, the temperatures, the filling convention, the prescribed background and theorem T3). It ends with ALL 44 CHECKS PASSED (notebook 13b) and draws nine figures.

13.18 How to run Notebook 13b

Step 1. What this notebook does and what it needs.

Notebook 13b (Exchange of a contact interaction in a uniform gas) is the file `Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb` of the repository Dirac_claude. It computes the one-label density matrix of the uniform Fermi gas (closed form and box sums), its exchange hole, the Hartree and exchange energies of a contact interaction with g labels (the rule E_x = -E_H/g), the approach of a finite-range interaction to the contact limit, Dirac's exchange energy of the Coulomb interaction for comparison, the exactness of the local exchange formula for any Slater determinant, and the spin polarization that exchange favours; finally it reproduces, from the Revision gamma matrices, the exchange energy of the uniform 8-fold gas of dirac16complex recorded in the Revision Kohn-Sham theory (coefficients -1/32, potentials 15/16 and -1/16, filled-shell ratio -1/8), checks the entries of the Revision record that state the status of the dirac16complex Kohn-Sham model (the independent Wolfram verification, the exact-exchange diagnostic, the temperatures, the filling convention, the prescribed background and theorem T3), and draws nine teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these

packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13b_contact_exchange_gas.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13b_contact_exchange_gas.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 13b_contact_exchange_gas.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/13b.captions.json`
- `Revision/textbook/figures/13b_1_density_matrix.png`
- `Revision/textbook/figures/13b_2_exchange_hole.png`
- `Revision/textbook/figures/13b_3_contact_energies.png`
- `Revision/textbook/figures/13b_4_finite_range.png`
- `Revision/textbook/figures/13b_5_contact_vs_coulomb.png`
- `Revision/textbook/figures/13b_6_local_exchange.png`
- `Revision/textbook/figures/13b_7_polarization.png`
- `Revision/textbook/figures/13b_8_dirac_matrices.png`
- `Revision/textbook/figures/13b_9_dirac_exchange.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13b_9_dirac_exchange.png exists
ALL 44 CHECKS PASSED (notebook 13b)
```

and the notebook must show 9 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 13b_contact_exchange_gas.ipynb
python -m nbconvert --execute --inplace 13b_contact_exchange_gas.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `gammas.json` or `ks-theory.json`: the notebook reads Revision records of the repository; it must be opened inside the folder `Revision/textbook/notebooks` of a complete clone of the repository, not as a single downloaded file.

13.19 Notebook 13b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The exchange energy is the part of the energy of fermions that comes from the Pauli
principle alone. In general it is an integral over TWO points, but for a contact
interaction (particles feel each other only when they are at the same point) it
becomes an integral over ONE point: it is exactly local. This notebook shows this
step by step for the uniform gas, the system from which the local density
approximation is built. It

- computes the density matrix of one label of the uniform gas in three dimensions
  from its closed form and from sums over the plane waves of a periodic box;
- draws the "exchange hole": the reduced chance of finding a second fermion close
  to a first one, for $g = 1$, $2$ and $8$ labels;
- computes the Hartree and exchange energies of a contact interaction and checks
  the rule $E_x = -E_H/g$ for equally occupied labels;
- lets a Gaussian interaction of range $a$ shrink to a contact and watches the
  exchange energy approach the contact value;
- computes Dirac's exchange energy of the Coulomb interaction for comparison;
- checks that the local exchange formula is exact for ANY Slater determinant, not
  only for the uniform gas;
- shows that exchange favours a gas in which the labels are unequally occupied
  (polarization) when the contact repulsion is strong;
- reproduces, from the Revision gamma matrices, the exchange energy of the uniform
  gas of the 16-component field dirac16complex that the Revision Kohn-Sham solver
  uses;
- reads and checks the entries of the Revision record that say on what these
  formulas rest and how the Kohn-Sham model of dirac16complex must be read;
- draws nine teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 13b (Exchange of a contact interaction in a uniform gas) is the file
`Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb` of the repository
Dirac_claude. It computes the one-label density matrix of the uniform Fermi gas (closed
form and box sums), its exchange hole, the Hartree and exchange energies of a contact
interaction with g labels (the rule E_x = -E_H/g), the approach of a finite-range
interaction to the contact limit, Dirac's exchange energy of the Coulomb interaction for
comparison, the exactness of the local exchange formula for any Slater determinant, and
the spin polarization that exchange favours; finally it reproduces, from the Revision
gamma matrices, the exchange energy of the uniform 8-fold gas of dirac16complex recorded
in the Revision Kohn-Sham theory (coefficients -1/32, potentials 15/16 and -1/16,
filled-shell ratio -1/8), checks the entries of the Revision record that state the status
of the dirac16complex Kohn-Sham model (the independent Wolfram verification, the
exact-exchange diagnostic, the temperatures, the filling convention, the prescribed
background and theorem T3), and draws nine teaching plots. It needs a computer with
Windows 11, macOS or Linux, an internet connection for the installation, the program Git,
and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
```

nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already

active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13b_contact_exchange_gas.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13b_contact_exchange_gas.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 13b_contact_exchange_gas.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/13b.captions.json‘
- ‘Revision/textbook/figures/13b_1_density_matrix.png‘
- ‘Revision/textbook/figures/13b_2_exchange_hole.png‘
- ‘Revision/textbook/figures/13b_3_contact_energies.png‘
- ‘Revision/textbook/figures/13b_4_finite_range.png‘
- ‘Revision/textbook/figures/13b_5_contact_vs_coulomb.png‘
- ‘Revision/textbook/figures/13b_6_local_exchange.png‘
- ‘Revision/textbook/figures/13b_7_polarization.png‘
- ‘Revision/textbook/figures/13b_8_dirac_matrices.png‘
- ‘Revision/textbook/figures/13b_9_dirac_exchange.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13b_9_dirac_exchange.png exists
ALL 44 CHECKS PASSED (notebook 13b)
```

and the notebook must show 9 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs

  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 13b_contact_exchange_gas.ipynb
      python -m nbconvert --execute --inplace 13b_contact_exchange_gas.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘gammas.json‘ or ‘ks-theory.json‘: the notebook reads Revision
  records of the repository; it must be opened inside the folder
  ‘Revision/textbook/notebooks‘ of a complete clone of the repository, not as a single
  downloaded file.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 13b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 13b (Exchange of a contact interaction in a uniform gas) is the file
# Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb of the repository
# Dirac_claude. It computes the one-label density matrix of the uniform Fermi gas (closed
# form and box sums), its exchange hole, the Hartree and exchange energies of a contact
# interaction with g labels (the rule E_x = -E_H/g), the approach of a finite-range
# interaction to the contact limit, Dirac's exchange energy of the Coulomb interaction
# for comparison, the exactness of the local exchange formula for any Slater determinant,
# and the spin polarization that exchange favours; finally it reproduces, from the
# Revision gamma matrices, the exchange energy of the uniform 8-fold gas of
# dirac16complex recorded in the Revision Kohn-Sham theory (coefficients -1/32,
# potentials 15/16 and -1/16, filled-shell ratio -1/8), checks the entries of the
# Revision record that state the status of the dirac16complex Kohn-Sham model (the
# independent Wolfram verification, the exact-exchange diagnostic, the temperatures, the
# filling convention, the prescribed background and theorem T3), and draws nine teaching
# plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for
# the installation, the program Git, and Python 3.12 or newer (the notebooks were built
# with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy
```

```
# and matplotlib; the other packages run Jupyter, the program that shows and runs
# notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
```

```
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
```

```
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 13b_contact_exchange_gas.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 13b_contact_exchange_gas.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 13b_contact_exchange_gas.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/13b.captions.json
# - Revision/textbook/figures/13b_1_density_matrix.png
# - Revision/textbook/figures/13b_2_exchange_hole.png
# - Revision/textbook/figures/13b_3_contact_energies.png
# - Revision/textbook/figures/13b_4_finite_range.png
# - Revision/textbook/figures/13b_5_contact_vs_coulomb.png
# - Revision/textbook/figures/13b_6_local_exchange.png
# - Revision/textbook/figures/13b_7_polarization.png
# - Revision/textbook/figures/13b_8_dirac_matrices.png
# - Revision/textbook/figures/13b_9_dirac_exchange.png
#
```

```
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 13b_9_dirac_exchange.png exists
#     ALL 44 CHECKS PASSED (notebook 13b)
#
# and the notebook must show 9 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/13b_contact_exchange_gas.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 13b_contact_exchange_gas.ipynb
#     python -m nbconvert --execute --inplace 13b_contact_exchange_gas.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
```

```
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for gammas.json or ks-theory.json: the notebook reads Revision
#   records of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete clone of the repository, not as a single
#   downloaded file.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "13b"  # this notebook: chapter 13, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
```

```
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
```

```
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 13b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Uniform gas**: infinitely many fermions spread with the same density everywhere;
  it is studied in a large periodic box (a box whose opposite faces are glued
  together), whose volume is then made infinite.
- **Plane wave**: the orbital $e^{i\mathbf k\cdot\mathbf r}/\sqrt V$ of a free
  particle in the box of volume $V$, with wave vector $\mathbf k$; its kinetic energy
  is $k^2/2$ (units $\hbar = m = 1$).
- **Fermi sphere, Fermi wave number** $k_F$: the ground state of the free gas fills
  every plane wave with $|\mathbf k| < k_F$, once per label.
- **Label**: an internal property with $g$ values (for electrons the spin, $g = 2$;
  for the dirac16complex gas of the Revision record, 8 states per momentum).
- **Density matrix** $\rho(x, x')$: $\sum_a \phi_a(x)\,\phi_a^*(x')$ over the occupied
  orbitals; its diagonal $\rho(x, x)$ is the density.
- **Exchange energy** $E_x = -\tfrac12\iint |\rho(x, x')|^2\,w(x, x')\,dx\,dx'$ of a
  Slater determinant with the pair interaction $w$; **Hartree energy**
  $E_H = \tfrac12\iint n(x)\,w(x, x')\,n(x')\,dx\,dx'$.
- **Contact interaction**: $w(\mathbf r, \mathbf r') = g_c\,\delta(\mathbf r -
  \mathbf r')$; it acts only when two particles are at the same point.
- **Exchange hole**: the dip in the probability of finding a second fermion near a
  first one, caused by the Pauli principle.
- **Coulomb interaction**: $w = 1/|\mathbf r - \mathbf r'|$, the repulsion of two
  electrons (atomic units).
- **Polarization** $\zeta = (n_{up} - n_{down})/n$: how unequally two labels are
  occupied.
- **Vertex**: the matrix that sits between the field components in an interaction;
```

```
  for dirac16complex the contact interaction is $(\lambda/2)\,S^2$ with
  $S = \bar\Psi\Psi = \Psi^\dagger C\,\Psi$, so the vertex is the $16 \times 16$
  matrix $C$.
- **sympy, Rational**: sympy computes with exact fractions such as $-1/32$, so its
  checks are exact, not rounded.
```

Text cell:

```
## 4. The physical and mathematical situation

**The exchange energy of a determinant.** For a Slater determinant with the density
matrix $\rho(x, x') = \sum_{a\,occupied}\phi_a(x)\phi_a^*(x')$, where $x$ stands for
the position $\mathbf r$ and the label $\sigma$ together (this $x$, and the $x$ of
the line in section 10, is not one of the author's spacetime coordinates
$x_0, \dots, x_7$; those appear only in section 12, as the labels of the gamma
matrices $\gamma^{(x0)}, \dots, \gamma^{(x7)}$), the interaction energy is
$E_H + E_x$ with

$$E_H = \tfrac12\iint n(x)\,w(x,x')\,n(x')\,dx\,dx', \qquad
E_x = -\tfrac12\iint |\rho(x,x')|^2\,w(x,x')\,dx\,dx' .$$

**Contact interaction.** With $w = g_c\,\delta(\mathbf r - \mathbf r')$ (the same for
all labels) the delta function sets $\mathbf r' = \mathbf r$ and only one integral
over space remains:

$$E_x = -\frac{g_c}{2}\int \sum_{\sigma,\sigma'}|\rho(\mathbf r\sigma,
\mathbf r\sigma')|^2\,d^3r .$$

If every orbital has a definite label, $\rho(\mathbf r\sigma, \mathbf r\sigma') = 0$
for $\sigma \ne \sigma'$ and $\rho(\mathbf r\sigma,\mathbf r\sigma) = n_\sigma(\mathbf
r)$, so $E_x = -\frac{g_c}{2}\int\sum_\sigma n_\sigma^2\,d^3r$: exactly local. With all
$g$ labels equally occupied, $n_\sigma = n/g$ and $E_x = -E_H/g$.

**The uniform gas.** In a box of volume $V$ the allowed wave vectors form a cubic
lattice of spacing $2\pi/\ell$ ($V = \ell^3$); filling $|\mathbf k| < k_F$ once per
label gives $n_\sigma = k_F^3/(6\pi^2)$. The density matrix of one label depends
only on the separation $R = |\mathbf r - \mathbf r'|$; in spherical coordinates
around $\mathbf R$,

$$\rho_\sigma(R) = \int_{k<k_F}\frac{d^3k}{(2\pi)^3}e^{i\mathbf k\cdot\mathbf R}
= \frac{1}{2\pi^2 R^3}\,[\sin(k_F R) - k_F R\cos(k_F R)] = n_\sigma\,F(k_F R),$$

$$F(s) = \frac{3\,(\sin s - s\cos s)}{s^3}, \qquad F(0) = 1 .$$

**Status.** The formulas above are exact mathematics of the free Fermi gas (PROVED:
they follow from the definitions). Sections 5 to 11 check them with numbers. Where
a closed form is compared with another exact form (sections 6, 7 and 10, the
slopes of section 9 and the curvature of section 11) the agreement is to rounding.
Where a sum or an integral is evaluated numerically, the result is COMPUTED with a
measured accuracy: the box sums of section 5 approach the closed form as the box
grows (the differences are printed), the finite-range ratio of section 8 follows
the small-range formula to $10^{-5}$ at $k_F a = 0.05$, and Simpson's rule gives
Dirac's integral $9/4$ of section 9 to $10^{-7}$. Section 11 describes the
```

```
exchange-only (Hartree-Fock) energy, not the exact one. Section 12 recomputes
exactly, with fractions, formulas that the Revision record PROVES with exact sympy
checks (exchange_uniform_gas, ks_potentials and filled_shell_ratio of its report
ks-theory-python.json) and independently with Wolfram; section 13 reads further
entries of the record and checks them.
```

Text cell:

```
## 5. The density matrix of one label of the uniform gas

The next cell defines the function $F(s)$. For small $s$ the formula
$3(\sin s - s\cos s)/s^3$ divides two tiny numbers and loses digits, so for
$s < 10^{-3}$ the function uses the first terms of its Taylor series,
$F(s) = 1 - s^2/10 + s^4/280$ (expand $\sin$ and $\cos$ and collect the powers).
The check compares the two forms at $s = 10^{-3}$.
```

In [2]:

```
import numpy as np  # arrays, matrices and linear algebra


def F(s):
    """F(s) = 3 (sin s - s cos s) / s^3, the density matrix of one label over n."""
    s = np.asarray(s, dtype=float)
    result = 1.0 - s ** 2 / 10.0 + s ** 4 / 280.0  # the Taylor series near 0
    big = s >= 1e-3  # where the closed form is accurate
    sb = s[big]
    result[big] = 3.0 * (np.sin(sb) - sb * np.cos(sb)) / sb ** 3
    return result


s_test = 1e-3
closed = 3.0 * (np.sin(s_test) - s_test * np.cos(s_test)) / s_test ** 3
series = 1.0 - s_test ** 2 / 10.0 + s_test ** 4 / 280.0
say(f"at s = 0.001: closed form {closed:.12f}, series {series:.12f}")
check(abs(closed - series) < 1e-8,
      "the closed form and the series agree at s = 0.001 (to about 9 digits)")
```

Out [2]:

```
at s = 0.001: closed form 0.999999899944, series 0.999999900000
PASS the closed form and the series agree at s = 0.001 (to about 9 digits)
```

Text cell:

```
The closed form came from an integral over the continuum of wave vectors. In a real
(finite) box the wave vectors are the lattice points $\mathbf k = (2\pi/\ell)\,
\mathbf m$ with integer $\mathbf m = (m_x, m_y, m_z)$, and the density matrix is the
SUM $\rho_\sigma(\mathbf R) = \frac1V\sum_{|\mathbf m| < m_F} e^{i\mathbf k\cdot
```

```
\mathbf R}$. Along the $x$ axis ($\mathbf R = (R, 0, 0)$) only $m_x$ matters, and the
sine parts cancel between $m_x$ and $-m_x$, so

$$\frac{\rho_\sigma(R)}{n_\sigma} = \frac{1}{N_\sigma}\sum_{m_x} c(m_x)\,
\cos\!\Big(\frac{2\pi m_x R}{\ell}\Big),$$

where $c(m_x)$ counts the lattice points with this $m_x$ inside the sphere and
$N_\sigma = \sum c(m_x)$ is the number of occupied plane waves. The next cell
computes this for spheres of radius $m_F = 4, 8, 16, 32$ lattice steps and compares
it with $F(k_F R)$, where $k_F$ is defined by the count,
$N_\sigma = \frac43\pi\,(k_F \ell/2\pi)^3$. The difference must shrink as the box
holds more particles.
```

In [3]:

```
s_grid = np.linspace(0.0, 12.0, 121)  # s = k_F R from 0 to 12
box_results = {}
for m_F in (4, 8, 16, 32):
    m = np.arange(-m_F, m_F + 1)  # the possible integers m_x, m_y, m_z
    my, mz = np.meshgrid(m, m, indexing="ij")  # all pairs (m_y, m_z)
    q = my ** 2 + mz ** 2
    counts = np.array([np.count_nonzero(q < m_F ** 2 - mx ** 2) for mx in m])
    N_label = int(counts.sum())  # occupied plane waves of one label
    kF_ell = 2.0 * np.pi * (3.0 * N_label / (4.0 * np.pi)) ** (1.0 / 3.0)
    R_over_ell = s_grid / kF_ell  # the separations R / ell for these s
    ratio = np.array([np.sum(counts * np.cos(2.0 * np.pi * m * r))
                      for r in R_over_ell]) / N_label
    box_results[m_F] = (N_label, ratio)
    deviation = np.max(np.abs(ratio - F(s_grid)))
    say(f"m_F = {m_F:2d}: {N_label:6d} plane waves; largest difference from "
        f"F = {deviation:.5f}")
deviations = [np.max(np.abs(box_results[m_F][1] - F(s_grid)))
              for m_F in (4, 8, 16, 32)]
shrinking = all(a > b for a, b in zip(deviations, deviations[1:]))
check(shrinking and deviations[-1] < 2e-3,
      "the box sums approach the closed form F as the box holds more particles")
```

Out [3]:

```
m_F =  4:    251 plane waves; largest difference from F = 0.03729
m_F =  8:   2103 plane waves; largest difference from F = 0.01218
m_F = 16:  17071 plane waves; largest difference from F = 0.00184
m_F = 32: 137059 plane waves; largest difference from F = 0.00083
PASS the box sums approach the closed form F as the box holds more particles
```

Text cell:

```
The next cell draws the closed form and the box sums for the smallest and the
largest sphere.
```

In [4]:

```
fig, ax = plt.subplots()
ax.plot(s_grid, F(s_grid), color="black", lw=2.0,
        label="continuum $F(k_F R) = 3(\\sin s - s\\cos s)/s^3$")
for m_F, marker in ((4, "o"), (32, "x")):
    N_label, ratio = box_results[m_F]
    ax.plot(s_grid[::3], ratio[::3], marker, ms=4,
            label=f"box sum, {N_label} plane waves")
ax.axhline(0.0, color="gray", lw=0.8)
ax.set_xlabel("$s = k_F R$ (separation times Fermi wave number)")
ax.set_ylabel("$\\rho_\\sigma(R) / n_\\sigma$")
ax.set_title("Density matrix of one label of the uniform gas")
ax.legend(fontsize=8)
save_figure(fig, "density_matrix",
            "The density matrix of one label of the uniform Fermi gas divided by "
            "the density of that label, $\\rho_\\sigma(R)/n_\\sigma = F(k_F R)$, "
            "against $s = k_F R$ (pure number): the closed form (black line) and "
            "the sums over the occupied plane waves of a periodic box with "
            f"{box_results[4][0]} and {box_results[32][0]} plane waves (markers). "
            "It is 1 at $R = 0$, falls to zero near "
            "$k_F R = 4.5$ and then oscillates with a shrinking amplitude; the box "
            "sums approach the curve as the box grows.")
```

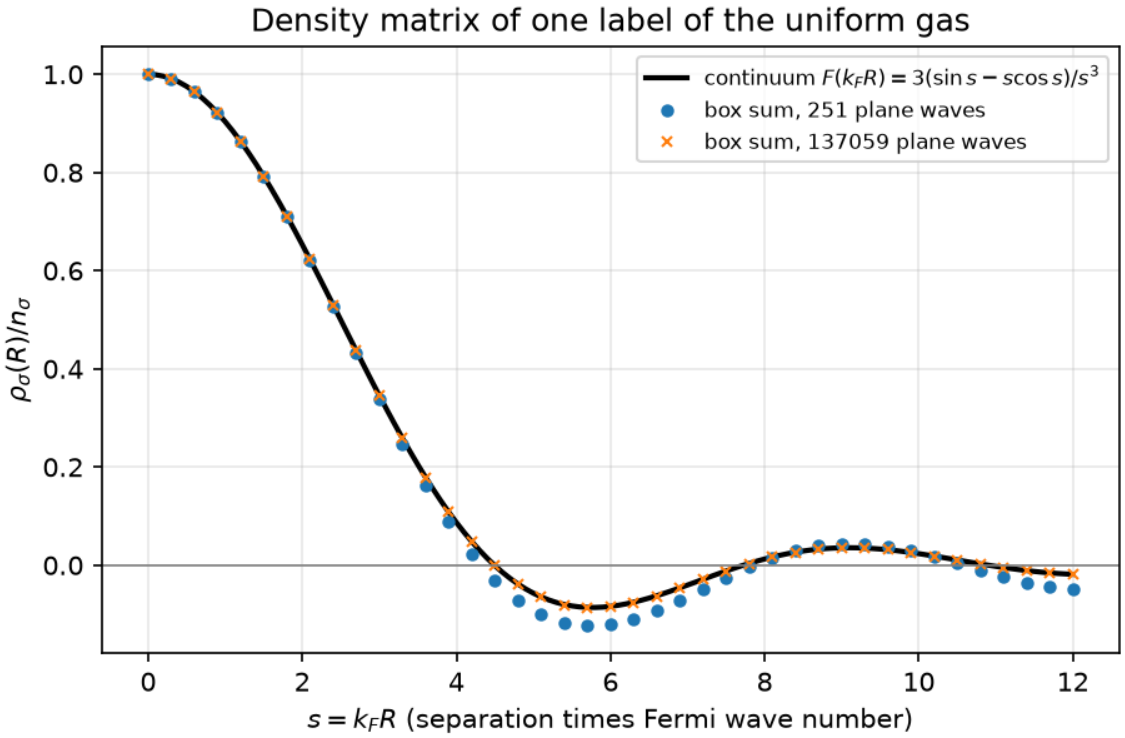

Figure 352: The density matrix of one label of the uniform Fermi gas divided by the density of that label, $\rho_\sigma(R)/n_\sigma = F(k_F R)$, against $s = k_F R$ (pure number): the closed form (black line) and the sums over the occupied plane waves of a periodic box with 251 and 137059 plane waves (markers). It is 1 at $R = 0$, falls to zero near $k_F R = 4.5$ and then oscillates with a shrinking amplitude; the box sums approach the curve as the box grows. (Notebook 13b, figure 1.)

Out [4]:

```
Figure 13b.1 saved as Revision/textbook/figures/13b_1_density_matrix.png
```

Text cell:

```
## 6. The exchange hole

For a determinant, the probability density of finding one fermion at $\mathbf r$ and
another at $\mathbf r'$ is $n(\mathbf r)n(\mathbf r') - \sum_\sigma|\rho_\sigma(
\mathbf r,\mathbf r')|^2$ (Wick's theorem). Divided by $n^2$ it is the **pair
distribution** $g_{pair}(R) = 1 - \frac{1}{g}F(k_F R)^2$ for $g$ equally occupied
labels. At $R = 0$ it is $1 - 1/g$: a second fermion with the SAME label is never
found at the same point, one with another label is found as often as without the
Pauli principle. The next cell computes and draws it for $g = 1, 2, 8$.
```

In [5]:

```
s_fine = np.linspace(0.0, 10.0, 501)
fig, ax = plt.subplots()
for g in (1, 2, 8):
    pair = 1.0 - F(s_fine) ** 2 / g  # the pair distribution
    ax.plot(s_fine, pair, label=f"$g = {g}$ labels: $g_{{pair}}(0) = {1 - 1 / g:.3f}$")
    check(abs(pair[0] - (1.0 - 1.0 / g)) < 1e-15,
          f"the pair distribution at contact is 1 - 1/g for g = {g}")
ax.axhline(1.0, color="gray", ls="--", lw=0.8)
ax.set_xlabel("$k_F R$")
ax.set_ylabel("pair distribution $g_{pair}(R)$")
ax.set_title("The exchange hole of the uniform gas")
ax.legend(fontsize=8)
save_figure(fig, "exchange_hole",
            "The pair distribution $g_{pair}(R) = 1 - F(k_F R)^2/g$ of the uniform "
            "Fermi gas with $g = 1, 2, 8$ equally occupied labels, against "
            "$k_F R$ (pure numbers): the chance of finding a second fermion at the "
            "distance $R$ from a first one, relative to the chance without the "
            "Pauli principle (dashed line 1). The dip near $R = 0$ is the exchange "
            "hole; its depth at contact is $1/g$, the fraction of fermions that "
            "share the first fermion's label.")
```

Out [5]:

```
PASS the pair distribution at contact is 1 - 1/g for g = 1
PASS the pair distribution at contact is 1 - 1/g for g = 2
PASS the pair distribution at contact is 1 - 1/g for g = 8
```

Out [5]:

```
Figure 13b.2 saved as Revision/textbook/figures/13b_2_exchange_hole.png
```

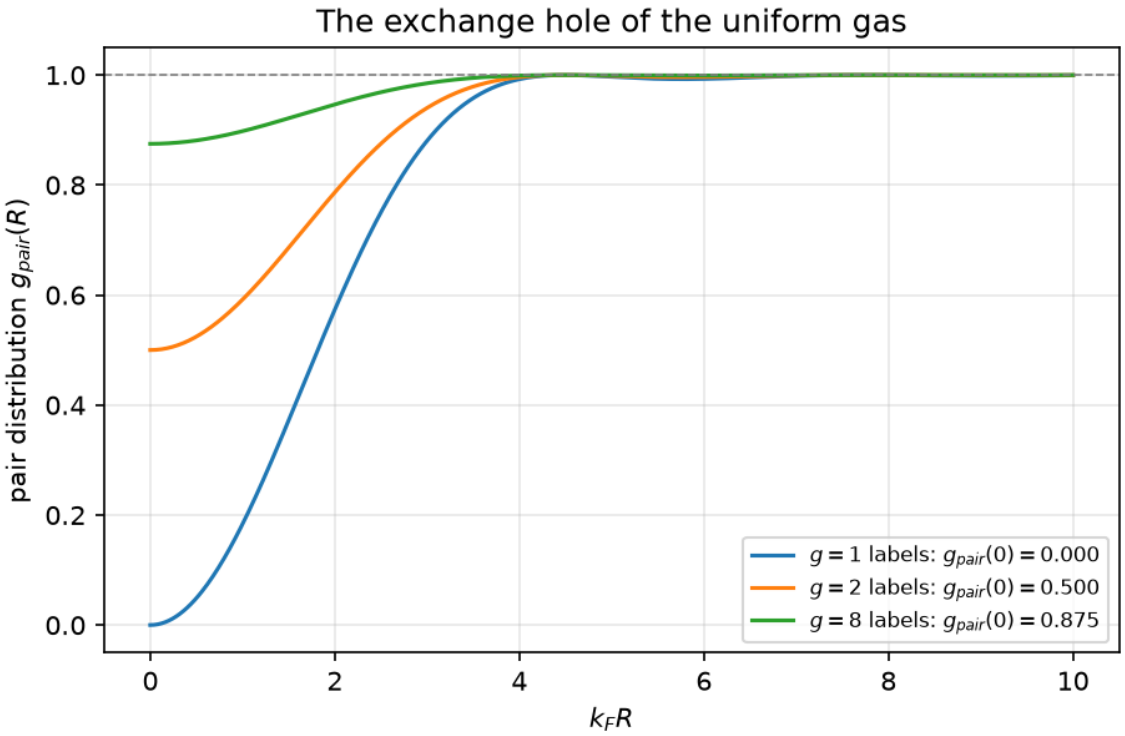

Figure 353: The pair distribution $g_{pair}(R) = 1 - F(k_F R)^2/g$ of the uniform Fermi gas with $g = 1, 2, 8$ equally occupied labels, against $k_F R$ (pure numbers): the chance of finding a second fermion at the distance R from a first one, relative to the chance without the Pauli principle (dashed line 1). The dip near $R = 0$ is the exchange hole; its depth at contact is $1/g$, the fraction of fermions that share the first fermion's label. (Notebook 13b, figure 2.)

Text cell:

```
## 7. Hartree and exchange energy of a contact interaction

For the contact interaction the energies per unit volume of the uniform gas are
$e_H = \frac{g_c}{2}n^2$ and $e_x = -\frac{g_c}{2}\sum_\sigma n_\sigma^2 =
-\frac{g_c}{2g}n^2$, so $e_H + e_x = \frac{g_c}{2}n^2\,(1 - 1/g) =
\frac{g_c}{2}n^2\,g_{pair}(0)$: the interaction energy counts only the pairs that can
meet, those with different labels. The next cell computes them for $g = 2$ and
$g = 8$ ($g_c = 1$) and checks the rule $e_x = -e_H/g$; for $g = 1$ the two cancel
exactly: a single label does not feel a contact interaction at all.
```

In [6]:

```
G_C = 1.0  # the strength of the contact interaction
densities = np.linspace(0.0, 2.0, 41)  # n from 0 to 2 (particles per volume)


def contact_energies(n, g):
    """(e_H, e_x) per volume for g equally occupied labels: n_sigma = n / g."""
    e_hartree = 0.5 * G_C * n ** 2
    e_exchange = -0.5 * G_C * g * (n / g) ** 2  # -(g_c/2) sum over g labels
    return e_hartree, e_exchange


for g in (1, 2, 8):
    e_hartree, e_exchange = contact_energies(densities, g)
    ratio = e_exchange[1:] / e_hartree[1:]  # skip n = 0 (0/0)
    check(np.allclose(ratio, -1.0 / g, rtol=0, atol=1e-15),
          f"e_x = -e_H/g for g = {g}")
e_hartree, e_exchange = contact_energies(densities, 1)
check(np.max(np.abs(e_hartree + e_exchange)) == 0.0,
      "for a single label the contact interaction cancels exactly")
```

Out [6]:

```
PASS e_x = -e_H/g for g = 1
PASS e_x = -e_H/g for g = 2
PASS e_x = -e_H/g for g = 8
PASS for a single label the contact interaction cancels exactly
```

Text cell:

```
The next cell draws the three energy densities for $g = 2$ (electrons) and
$g = 8$ (the number of good-sector states per momentum of the dirac16complex gas).
```

In [7]:

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0), sharey=True)
for ax, g in zip(axes, (2, 8)):
    e_hartree, e_exchange = contact_energies(densities, g)
    ax.plot(densities, e_hartree, label="Hartree $e_H = g_c n^2/2$")
    ax.plot(densities, e_exchange, label=f"exchange $e_x = -e_H/{g}$")
    ax.plot(densities, e_hartree + e_exchange, color="black", lw=2.0,
            label="sum $e_H + e_x$")
    ax.axhline(0.0, color="gray", lw=0.8)
    ax.set_xlabel("density $n$")
    ax.set_title(f"$g = {g}$ labels")
    ax.legend(fontsize=8)
axes[0].set_ylabel("energy per volume (units of $g_c$)")
save_figure(fig, "contact_energies",
            "The Hartree energy $e_H = g_c n^2/2$, the exchange energy "
            "$e_x = -e_H/g$ and their sum per unit volume of the uniform gas with "
            "a contact interaction ($g_c = 1$), against the density $n$, for "
            "$g = 2$ labels (left) and $g = 8$ labels (right). Exchange removes "
            "the fraction $1/g$ of the Hartree energy: the pairs of fermions with "
            "the same label, which never meet.")
```

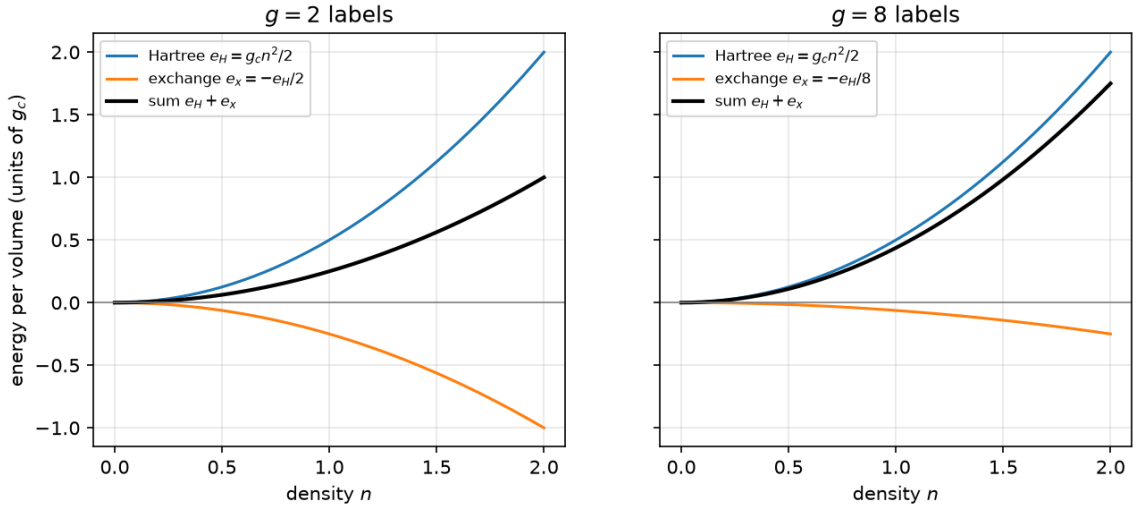

Figure 354: The Hartree energy $e_H = g_c n^2/2$, the exchange energy $e_x = -e_H/g$ and their sum per unit volume of the uniform gas with a contact interaction ($g_c = 1$), against the density n, for $g = 2$ labels (left) and $g = 8$ labels (right). Exchange removes the fraction $1/g$ of the Hartree energy: the pairs of fermions with the same label, which never meet. (Notebook 13b, figure 3.)

Out [7]:

```
Figure 13b.3 saved as Revision/textbook/figures/13b_3_contact_energies.png
```

Text cell:

```
## 8. A finite-range interaction shrinking to a contact
```

```
Replace the contact by a Gaussian of range $a$ and the same total strength,
$w_a(R) = g_c\,(2\pi a^2)^{-3/2}e^{-R^2/(2a^2)}$, whose integral over all space is
$g_c$. Its Hartree energy is still $\frac{g_c}{2}n^2$ (only the integral of $w$
enters for a uniform density), but its exchange energy becomes
$e_x(a) = -\frac12\sum_\sigma n_\sigma^2\int w_a(R)\,F(k_F R)^2\,d^3R$. So
$e_x(a)/e_x(0) = \int_0^\infty 4\pi R^2\,w_a(R)\,F(k_F R)^2\,dR\,/\,g_c$. For small
$a$, $F^2 \approx 1 - s^2/5$ and $\int w_a R^2\,d^3R = 3a^2 g_c$, so
$e_x(a)/e_x(0) \approx 1 - \frac35 (k_F a)^2$. The next cell computes the ratio with
Simpson's rule on a fine grid (in the variable $s = k_F R$, with $a$ measured in
units of $1/k_F$) and checks the small-$a$ formula.
```

In [8]:

```
def simpson(values, step):
    """Simpson's rule for equally spaced values (an odd number of them)."""
    return step / 3.0 * (values[0] + values[-1] + 4.0 * values[1:-1:2].sum()
                         + 2.0 * values[2:-1:2].sum())


s_int = np.linspace(0.0, 80.0, 400001)  # s = k_F R; the Gaussian is tiny beyond
ds = s_int[1] - s_int[0]
F2 = F(s_int) ** 2


def exchange_ratio(kF_a):
    """e_x(a) / e_x(0) for the Gaussian of range a (k_F a given)."""
    norm = (2.0 * np.pi * kF_a ** 2) ** -1.5  # makes the integral of w_a equal g_c
    gauss = norm * np.exp(-s_int ** 2 / (2.0 * kF_a ** 2))
    return simpson(4.0 * np.pi * s_int ** 2 * gauss * F2, ds)


ranges = np.logspace(-2, 1, 31)  # k_F a from 0.01 to 10
ratios = np.array([exchange_ratio(r) for r in ranges])
for r in (0.01, 0.1, 1.0):
    say(f"k_F a = {r:5.2f}: e_x(a)/e_x(0) = {exchange_ratio(r):.6f}, "
        f"small-a formula {1.0 - 0.6 * r ** 2:.6f}")
check(abs(exchange_ratio(0.05) - (1.0 - 0.6 * 0.05 ** 2)) < 1e-5,
      "for small range the ratio is 1 - (3/5)(k_F a)^2")
check(np.all(np.diff(ratios) < 0.0) and ratios[0] > 0.9999,
      "the exchange energy grows towards the contact value as the range shrinks")
```

Out [8]:

```
k_F a =  0.01: e_x(a)/e_x(0) = 0.999940, small-a formula 0.999940
k_F a =  0.10: e_x(a)/e_x(0) = 0.994026, small-a formula 0.994000
k_F a =  1.00: e_x(a)/e_x(0) = 0.588864, small-a formula 0.400000
PASS for small range the ratio is 1 - (3/5)(k_F a)^2
PASS the exchange energy grows towards the contact value as the range shrinks
```

Text cell:

```
The next cell draws the ratio against the range on a logarithmic horizontal axis.
```

In [9]:

```
fig, ax = plt.subplots()
ax.semilogx(ranges, ratios, "o-", ms=3, color="black",
            label="$e_x(a)/e_x(0)$, Gaussian of range $a$")
small = ranges[ranges < 0.6]
ax.semilogx(small, 1.0 - 0.6 * small ** 2, "--",
            label="small-range formula $1 - (3/5)(k_F a)^2$")
ax.set_ylim(0.0, 1.05)
ax.set_xlabel("range times Fermi wave number, $k_F a$")
ax.set_ylabel("exchange energy relative to contact")
ax.set_title("A finite-range interaction shrinking to a contact")
ax.legend(fontsize=8)
save_figure(fig, "finite_range",
            "The exchange energy of a Gaussian interaction of range $a$ (same total "
            "strength $g_c$) divided by the exchange energy of the contact "
            "interaction, against $k_F a$ on a logarithmic axis (pure numbers), "
            "with the small-range formula $1 - (3/5)(k_F a)^2$ (dashed). When the "
            "range is much smaller than the size $1/k_F$ of the exchange hole the "
            "ratio is 1: the exchange of a contact interaction is local and "
            "exact; for a long range most of the exchange is lost.")
```

Out [9]:

```
Figure 13b.4 saved as Revision/textbook/figures/13b_4_finite_range.png
```

Text cell:

```
## 9. For comparison: Dirac's exchange energy of the Coulomb interaction

For $w = 1/R$ and $g = 2$ the same formula gives $e_x = -\frac12 \cdot 2\,
n_\sigma^2\int_0^\infty 4\pi R^2\,\frac{F(k_F R)^2}{R}\,dR = -\frac{4\pi
n_\sigma^2}{k_F^2}\int_0^\infty s\,F(s)^2\,ds$, and the integral is $9/4$, which
gives Dirac's result $e_x = -\frac34(3/\pi)^{1/3}n^{4/3}$ (with $n = 2n_\sigma =
k_F^3/(3\pi^2)$). The integrand falls only like $1/s^3$; the next cell integrates
up to $s = 4000$ and adds the tail beyond, $\int_{4000}^\infty 9\cos^2 s/s^3\,ds
\approx 9/(4\cdot 4000^2)$ (the average of $\cos^2$ is $1/2$).
```

In [10]:

```
S_MAX = 4000.0
s_long = np.linspace(0.0, S_MAX, 4000001)
integral_9_4 = simpson(s_long * F(s_long) ** 2, s_long[1] - s_long[0])
integral_9_4 += 9.0 / (4.0 * S_MAX ** 2)  # the tail beyond S_MAX
report("integral of s F(s)^2 from 0 to infinity", f"{integral_9_4:.8f}")
check(abs(integral_9_4 - 2.25) < 1e-7, "the Coulomb exchange integral is 9/4")
dirac = 0.75 * (3.0 / np.pi) ** (1.0 / 3.0)  # e_x = -dirac n^(4/3)
n_test = 0.3
kF_test = (3.0 * np.pi ** 2 * n_test) ** (1.0 / 3.0)  # g = 2
e_x_coulomb = -4.0 * np.pi * (n_test / 2) ** 2 / kF_test ** 2 * integral_9_4
```

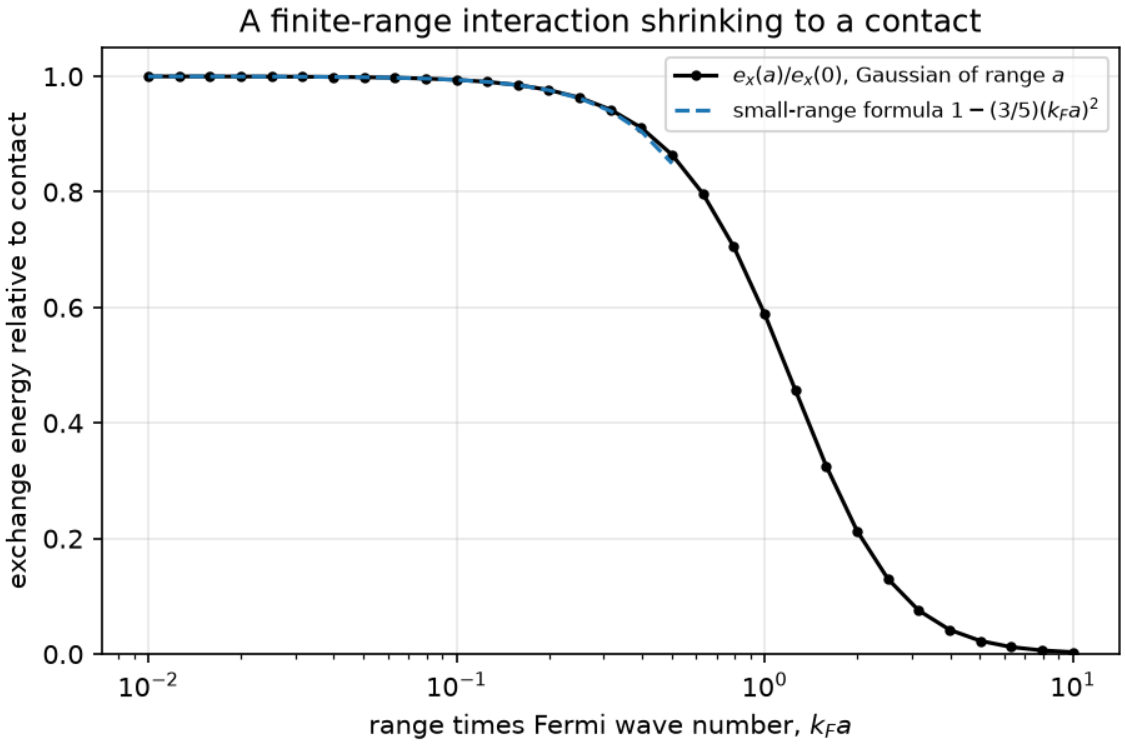

Figure 355: The exchange energy of a Gaussian interaction of range a (same total strength g_c) divided by the exchange energy of the contact interaction, against k_Fa on a logarithmic axis (pure numbers), with the small-range formula $1-(3/5)(k_Fa)^2$ (dashed). When the range is much smaller than the size $1/k_F$ of the exchange hole the ratio is 1: the exchange of a contact interaction is local and exact; for a long range most of the exchange is lost. (Notebook 13b, figure 4.)

```
report("Dirac's constant (3/4)(3/pi)^(1/3)", f"{dirac:.6f}")
check(abs(e_x_coulomb + dirac * n_test ** (4.0 / 3.0)) < 1e-8,
      "the Coulomb exchange of the electron gas is -(3/4)(3/pi)^(1/3) n^(4/3)")
```

Out [10]:

```
RESULT integral of s F(s)^2 from 0 to infinity = 2.25000000
PASS the Coulomb exchange integral is 9/4
RESULT Dirac's constant (3/4)(3/pi)^(1/3) = 0.738559
PASS the Coulomb exchange of the electron gas is -(3/4)(3/pi)^(1/3) n^(4/3)
```

Text cell:

```
The next cell compares the exchange energy PER PARTICLE of the two interactions on
logarithmic axes: $e_x/n = -g_c n/4$ for the contact ($g = 2$) and
$-\frac34(3/\pi)^{1/3}n^{1/3}$ for the Coulomb interaction. On such axes a power
$n^p$ is a straight line of slope $p$; the check fits the slopes.
```

In [11]:

```
n_values = np.logspace(-3, 1, 41)
contact_per_particle = 0.25 * G_C * n_values  # |e_x / n| for the contact, g = 2
coulomb_per_particle = dirac * n_values ** (1.0 / 3.0)  # |e_x / n|, Coulomb
slope_contact = np.polyfit(np.log(n_values), np.log(contact_per_particle), 1)[0]
slope_coulomb = np.polyfit(np.log(n_values), np.log(coulomb_per_particle), 1)[0]
say(f"slopes: contact {slope_contact:.6f}, Coulomb {slope_coulomb:.6f}")
check(abs(slope_contact - 1.0) < 1e-12 and abs(slope_coulomb - 1.0 / 3.0) < 1e-12,
      "exchange per particle grows like n (contact) and n^(1/3) (Coulomb)")
fig, ax = plt.subplots()
ax.loglog(n_values, contact_per_particle, label="contact, $g_c = 1$: $n/4$")
ax.loglog(n_values, coulomb_per_particle, "--",
          label="Coulomb (Dirac): $0.7386\\,n^{1/3}$")
ax.set_xlabel("density $n$")
ax.set_ylabel("$|e_x|/n$, exchange energy per particle")
ax.set_title("Exchange per particle: contact versus Coulomb ($g = 2$)")
ax.legend()
save_figure(fig, "contact_vs_coulomb",
            "The size of the exchange energy per particle of a uniform gas with "
            "two labels, against the density, on logarithmic axes: contact "
            "interaction $g_c n/4$ with $g_c = 1$ (solid) and Coulomb interaction "
            "$(3/4)(3/\\pi)^{1/3}n^{1/3}$ in atomic units (dashed). The slopes 1 and "
            "1/3 show the powers; the contact exchange is a function of the local "
            "density alone because the interaction has no range, the Coulomb one "
            "because the gas is uniform.")
```

Out [11]:

```
slopes: contact 1.000000, Coulomb 0.333333
PASS exchange per particle grows like n (contact) and n^(1/3) (Coulomb)
```

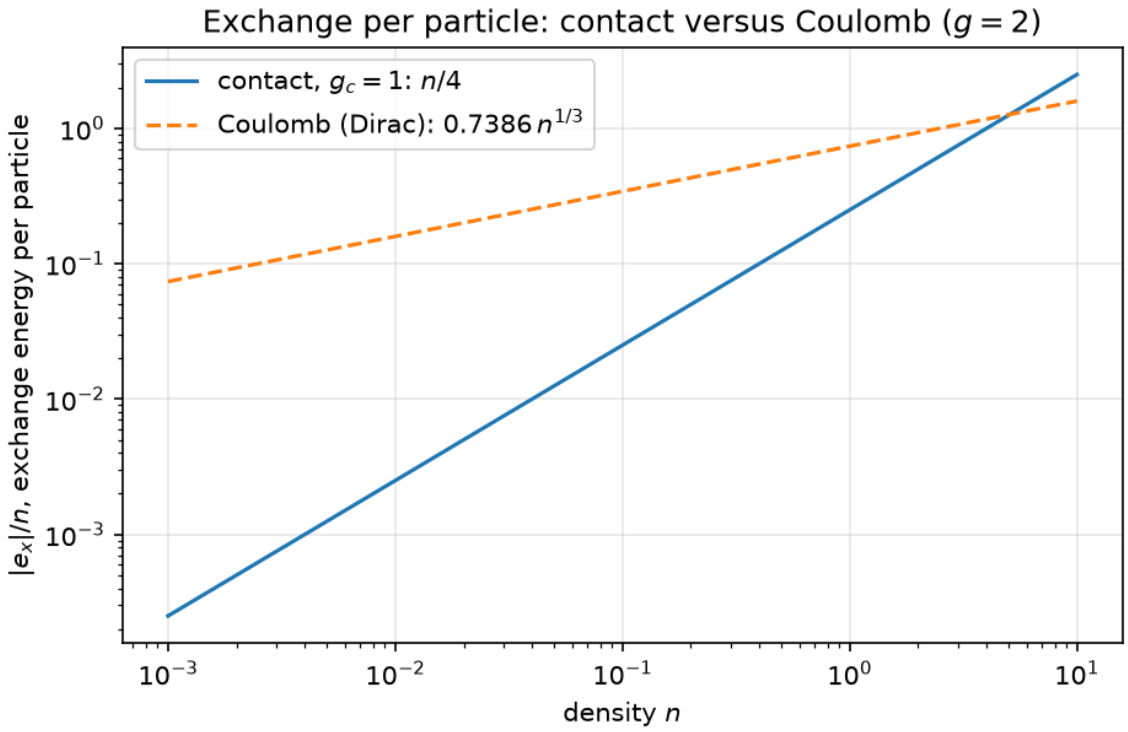

Figure 356: The size of the exchange energy per particle of a uniform gas with two labels, against the density, on logarithmic axes: contact interaction $g_c n/4$ with $g_c = 1$ (solid) and Coulomb interaction $(3/4)(3/\pi)^{1/3} n^{1/3}$ in atomic units (dashed). The slopes 1 and 1/3 show the powers; the contact exchange is a function of the local density alone because the interaction has no range, the Coulomb one because the gas is uniform. (Notebook 13b, figure 5.)

Out [11]:

```
Figure 13b.5 saved as Revision/textbook/figures/13b_5_contact_vs_coulomb.png
```

Text cell:

```
## 10. The local formula is exact for ANY determinant

The uniform gas is special. Is the local contact exchange also exact for a
non-uniform determinant? The next cell takes a line $0 < x < 1$ cut into 99 grid
points (spacing $h = 1/100$) and the orbitals of a particle in this box: five with
label up and three with label down. On the grid the contact is the matrix
$W(x, x') = g_c\,\delta_{xx'}/h$ (one integral times $h$ gives back $g_c$). The
cell computes the exchange energy in the two-point (Fock) form
$-\frac12\sum_{x,x'} h^2\,|\rho_\sigma(x,x')|^2\,W(x,x')$ summed over the labels,
and in the local form $-\frac{g_c}{2}\sum_x h\,(n_{up}^2 + n_{down}^2)$.
```

In [12]:

```
P = 99  # interior grid points of the box 0 < x < 1
h = 1.0 / (P + 1)
x = h * np.arange(1, P + 1)
W = G_C * np.eye(P) / h  # the contact interaction on the grid


def box_orbitals(count):
    """The lowest box orbitals sqrt(2) sin(m pi x), m = 1..count, as columns."""
    return np.array([np.sqrt(2.0) * np.sin(m * np.pi * x)
                     for m in range(1, count + 1)]).T


phi_up, phi_down = box_orbitals(5), box_orbitals(3)
rho_up = phi_up @ phi_up.T  # rho_up(x, x') = sum_a phi_a(x) phi_a(x')
rho_down = phi_down @ phi_down.T
n_up, n_down = np.diag(rho_up), np.diag(rho_down)  # the label densities
fock = -0.5 * h * h * (np.sum(rho_up ** 2 * W) + np.sum(rho_down ** 2 * W))
local = -0.5 * G_C * h * np.sum(n_up ** 2 + n_down ** 2)
report("exchange energy, two-point (Fock) form", f"{fock:.10f}")
report("exchange energy, local form", f"{local:.10f}")
check(abs(h * n_up.sum() - 5.0) < 1e-12 and abs(h * n_down.sum() - 3.0) < 1e-12,
      "the box determinant holds 5 up and 3 down fermions")
check(abs(fock - local) < 1e-12,
      "the contact exchange of a non-uniform determinant is exactly local")
```

Out [12]:

```
RESULT exchange energy, two-point (Fock) form = -19.0000000000
RESULT exchange energy, local form = -19.0000000000
PASS the box determinant holds 5 up and 3 down fermions
PASS the contact exchange of a non-uniform determinant is exactly local
```

Text cell:

```
Orbitals need not have a definite label: an orbital may have an up part AND a down
part (a column of $2 \times 99$ numbers). The next cell builds five such orbitals at
random (orthonormal, from the QR factorisation of a random complex matrix with a
fixed seed, so every run gives the same numbers) and compares three numbers: the
exact exchange from the orbital-pair sum
$-\frac12\sum_{a,b}w_{abba}$, with
$w_{abba} = g_c\sum_x h\,|\sum_\sigma \phi_a^*(x\sigma)\phi_b(x\sigma)|^2$; the local
formula with ALL label pairs, $-\frac{g_c}{2}\sum_x h\sum_{\sigma\sigma'}
|\rho(x\sigma, x\sigma')|^2$; and the local formula with only the label diagonal,
$-\frac{g_c}{2}\sum_x h\sum_\sigma n_\sigma^2$. The first two must agree; the third
misses the label-mixing part.
```

In [13]:

```
rng = np.random.default_rng(12345)  # a fixed seed: the same numbers every run
raw = rng.normal(size=(2 * P, 5)) + 1j * rng.normal(size=(2 * P, 5))
q_matrix = np.linalg.qr(raw)[0]  # five orthonormal columns of length 2P
spinor = (q_matrix / np.sqrt(h)).reshape(P, 2, 5)  # [point, label, orbital]
# pair[x, a, b] = sum over the label of phi_a*(x, label) phi_b(x, label)
pair = np.einsum("xsa,xsb->xab", spinor.conj(), spinor)
exact = -0.5 * G_C * h * np.sum(np.abs(pair) ** 2)
# rho[x, s, t] = sum_a phi_a(x, s) phi_a*(x, t): the local 2 x 2 density matrix
rho_local = np.einsum("xsa,xta->xst", spinor, spinor.conj())
local_all = -0.5 * G_C * h * np.sum(np.abs(rho_local) ** 2)
diagonal = np.einsum("xss->xs", rho_local).real  # n_up(x), n_down(x)
local_diagonal = -0.5 * G_C * h * np.sum(diagonal ** 2)
say(f"exact {exact:.10f}; local, all label pairs {local_all:.10f}; "
    f"local, label diagonal only {local_diagonal:.10f}")
check(abs(exact - local_all) < 1e-10,
      "with label-mixing orbitals the exchange is local in the 2 x 2 matrix rho")
check(abs(exact - local_diagonal) > 0.1,
      "the label diagonal alone misses the label-mixing exchange")
```

Out [13]:

```
exact -8.7398756465; local, all label pairs -8.7398756465; local, label diagonal only
    -7.5021209026
PASS with label-mixing orbitals the exchange is local in the 2 x 2 matrix rho
PASS the label diagonal alone misses the label-mixing exchange
```

Text cell:

```
The next cell draws the two-point function $|\rho_{up}(x, x')|^2$ of the box
determinant as a heat map (left) and, along the line, the local Hartree and exchange
energy densities (right). For a contact interaction only the diagonal $x = x'$ of the
heat map enters the exchange energy.
```

In [14]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
image = left.imshow(rho_up ** 2, origin="lower", extent=(0, 1, 0, 1),
                    cmap="viridis")
left.plot([0, 1], [0, 1], "w--", lw=1.0)  # the diagonal x = x'
left.set_xlabel("$x$")
left.set_ylabel("$x'$")
left.set_title("$|\\rho_{up}(x, x')|^2$, 5 up fermions in a box")
fig.colorbar(image, ax=left, shrink=0.85)
right.plot(x, 0.5 * G_C * (n_up + n_down) ** 2, label="Hartree $g_c n^2/2$")
right.plot(x, -0.5 * G_C * (n_up ** 2 + n_down ** 2),
           label="exchange $-g_c(n_{up}^2 + n_{down}^2)/2$")
right.plot(x, G_C * n_up * n_down, color="black", lw=2.0,
           label="sum $g_c\\,n_{up} n_{down}$")
right.set_ylim(-50.0, 60.0)  # room below the curves for the legend
right.set_xlabel("$x$")
right.set_ylabel("energy per length (units of $g_c$)")
right.legend(fontsize=7, loc="lower center")
save_figure(fig, "local_exchange",
            "Left: the two-point function $|\\rho_{up}(x, x')|^2$ of five fermions "
            "with label up in the box $0 < x < 1$ as a heat map (both axes the "
            "position; bright means large); a contact interaction uses only its "
            "diagonal $x = x'$ (dashed). Right: the local Hartree energy density, "
            "the local exchange energy density and their sum "
            "$g_c\\,n_{up}n_{down}$ along the box (5 up and 3 down fermions, "
            "$g_c = 1$): exchange removes the same-label part of the Hartree "
            "energy point by point.")
check(np.allclose(0.5 * (n_up + n_down) ** 2 - 0.5 * (n_up ** 2 + n_down ** 2),
                  n_up * n_down, atol=1e-12),
      "Hartree plus exchange of a contact is g_c n_up n_down at every point")
```

Out [14]:

```
Figure 13b.6 saved as Revision/textbook/figures/13b_6_local_exchange.png
PASS Hartree plus exchange of a contact is g_c n_up n_down at every point
```

Text cell:

```
## 11. Exchange favours polarization

Exchange lowers the energy of equal-label pairs only. A gas with two labels can
therefore lower its interaction energy by occupying the labels unequally, at the
price of more kinetic energy (the more occupied label needs a bigger Fermi sphere).
With the polarization $\zeta$, $n_{up} = n(1+\zeta)/2$, $n_{down} = n(1-\zeta)/2$,
the energy per volume in this exchange-only (Hartree-Fock) picture is

$$e(\zeta) = C_F n^{5/3}\,\frac{(1+\zeta)^{5/3} + (1-\zeta)^{5/3}}{2}
+ \frac{g_c}{4}n^2(1 - \zeta^2), \qquad C_F = \tfrac{3}{10}(3\pi^2)^{2/3}.$$

The second derivative at $\zeta = 0$ is $\frac{10}{9}C_F n^{5/3} - \frac{g_c}{2}n^2$,
which turns negative (the unpolarized gas becomes unstable) when
$g_c n^{1/3} > \gamma_c = \frac{20}{9}C_F = \frac23(3\pi^2)^{2/3}$. The next cell
checks this threshold with a difference quotient and draws $e(\zeta) - e(0)$ for
```

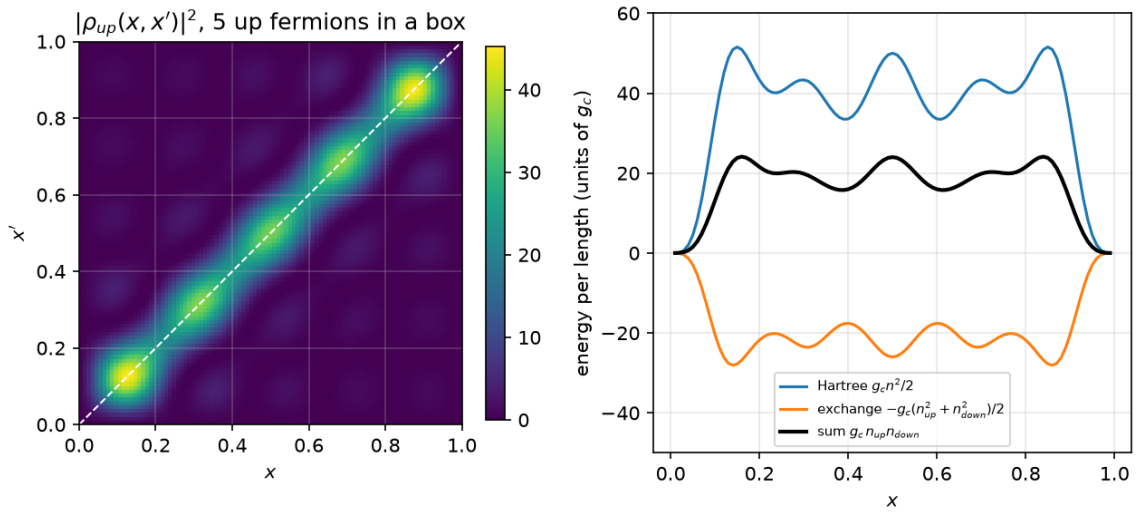

Figure 357: Left: the two-point function $|\rho_{up}(x,x')|^2$ of five fermions with label up in the box $0<x<1$ as a heat map (both axes the position; bright means large); a contact interaction uses only its diagonal $x=x'$ (dashed). Right: the local Hartree energy density, the local exchange energy density and their sum $g_c\, n_{up} n_{down}$ along the box (5 up and 3 down fermions, $g_c=1$): exchange removes the same-label part of the Hartree energy point by point. (Notebook 13b, figure 6.)

```
several couplings $\gamma = g_c n^{1/3}/\gamma_c$ (at $n = 1$). This is a property of
the exchange-only functional; correlation, which is left out here, weakens it.
```

In [15]:

```
C_F = 0.3 * (3.0 * np.pi ** 2) ** (2.0 / 3.0)
gamma_c = (2.0 / 3.0) * (3.0 * np.pi ** 2) ** (2.0 / 3.0)


def energy_polarized(zeta, coupling):
    """e(zeta) at n = 1 for g_c = coupling (exchange-only picture)."""
    kinetic_part = C_F * ((1 + zeta) ** (5 / 3) + (1 - zeta) ** (5 / 3)) / 2
    return kinetic_part + 0.25 * coupling * (1 - zeta ** 2)


report("C_F", f"{C_F:.6f}")
report("threshold gamma_c = g_c n^(1/3)", f"{gamma_c:.6f}")
d = 1e-3
curvature = [(energy_polarized(d, c) - 2 * energy_polarized(0.0, c)
              + energy_polarized(-d, c)) / d ** 2
             for c in (0.99 * gamma_c, 1.01 * gamma_c)]
check(curvature[0] > 0 > curvature[1],
      "the unpolarized gas becomes unstable at g_c n^(1/3) = (2/3)(3 pi^2)^(2/3)")
zetas = np.linspace(-1.0, 1.0, 401)
fig, ax = plt.subplots()
for gamma in (0.5, 1.0, 1.1, 1.5):
    curve = energy_polarized(zetas, gamma * gamma_c) - energy_polarized(0.0,
                                                                        gamma * gamma_c)
```

```
    ax.plot(zetas, curve / C_F, label=f"$\\gamma = {gamma}$")
ax.axhline(0.0, color="gray", lw=0.8)
ax.set_xlabel("polarization $\\zeta = (n_{up} - n_{down})/n$")
ax.set_ylabel("$(e(\\zeta) - e(0)) / (C_F n^{5/3})$")
ax.set_title("Exchange favours unequal labels at strong contact repulsion")
ax.legend()
save_figure(fig, "polarization",
            "The energy per volume of a uniform gas with two labels and a contact "
            "repulsion in the exchange-only (Hartree-Fock) picture, relative to "
            "the unpolarized gas and in units of $C_F n^{5/3}$, against the "
            "polarization $\\zeta$, for four couplings "
            "$\\gamma = g_c n^{1/3}/\\gamma_c$. Below $\\gamma = 1$ the unpolarized "
            "gas ($\\zeta = 0$) has the lowest energy; above it the curve bends "
            "down at $\\zeta = 0$ and polarized states are lower.")
```

Out [15]:

```
RESULT C_F = 2.871234
RESULT threshold gamma_c = g_c n^(1/3) = 6.380520
PASS the unpolarized gas becomes unstable at g_c n^(1/3) = (2/3)(3 pi^2)^(2/3)
```

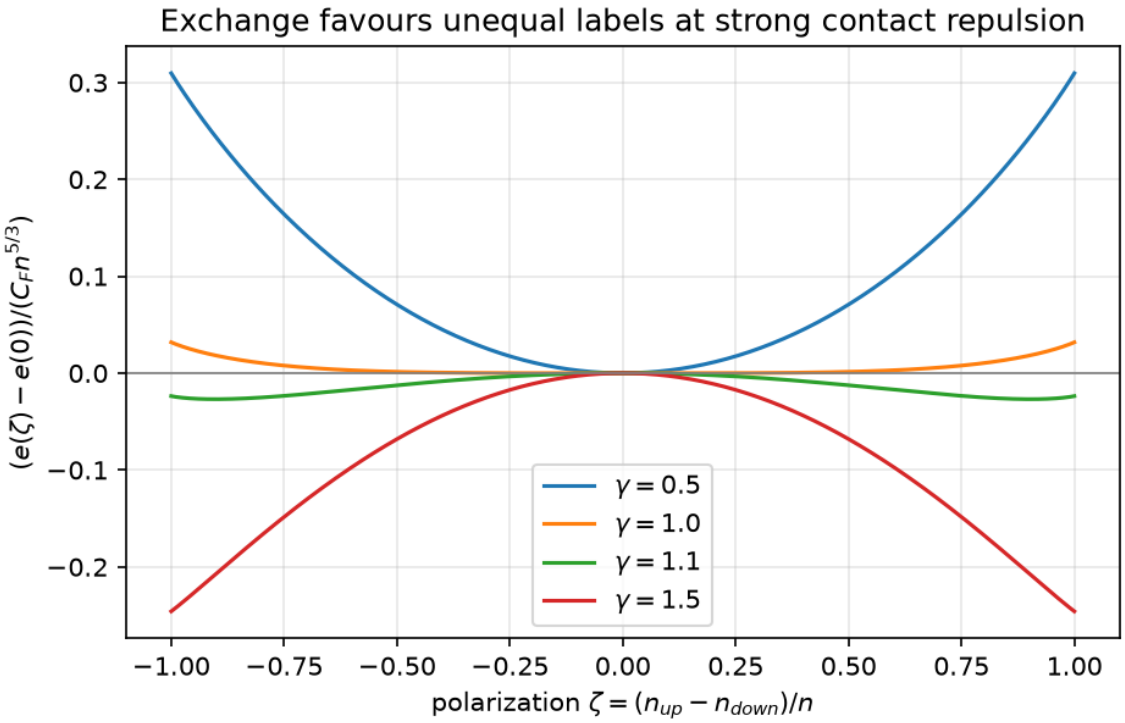

Figure 358: The energy per volume of a uniform gas with two labels and a contact repulsion in the exchange-only (Hartree-Fock) picture, relative to the unpolarized gas and in units of $C_F n^{5/3}$, against the polarization ζ, for four couplings $\gamma = g_c n^{1/3}/\gamma_c$. Below $\gamma = 1$ the unpolarized gas ($\zeta = 0$) has the lowest energy; above it the curve bends down at $\zeta = 0$ and polarized states are lower. (Notebook 13b, figure 7.)

Out [15]:

```
Figure 13b.7 saved as Revision/textbook/figures/13b_7_polarization.png
```

Text cell:

```
## 12. The same exchange for the 16-component field dirac16complex

The Kohn-Sham model of the dirac16complex field in the Revision record has the
contact interaction $U = (\lambda/2)\,S^2$ with the scalar density
$S = \bar\Psi\Psi = \Psi^\dagger C\,\Psi$. Its expectation values follow the rule
$\langle\Psi^\dagger M\Psi\rangle = \mathrm{Tr}(M\rho)$ with a local
$16 \times 16$ one-body matrix $\rho$, and Wick's theorem gives, exactly as for the
labels above,

$$e_H = \frac{\lambda}{2}\,(\mathrm{Tr}\,C\rho)^2, \qquad
e_x = -\frac{\lambda}{2}\,\mathrm{Tr}(C\rho\,C\rho).$$

(For $g$ ordinary labels the vertex is the unit matrix and $\rho = (n/g)\,1$, which
gives back $e_x = -\frac{g_c}{2}\,n^2/g$.) For the uniform good-sector gas the
Revision record finds $\rho = (n B + S C)/16$, with the number density
$n = \mathrm{Tr}(B\rho)$ and $B = -i\,C\gamma^{(x4)}$. The next cell reads the
author's gamma matrices EXACTLY (as fractions) from the Revision record, builds $C$
and $B$, and checks them against the matrices stored in the same record.
```

In [16]:

```
import sympy as sp  # exact algebra with fractions and symbols

gamma_record = json.loads(repository_file("Revision/algebra/gammas.json")
                          .read_text(encoding="utf-8"))


def exact_matrix(rows):
    """A sympy matrix of exact fractions from the record's rows of numbers."""
    return sp.Matrix([[sp.Rational(str(entry)) for entry in row] for row in rows])


gamma = [exact_matrix(rows) for rows in gamma_record["gamma"]]  # x0 .. x7
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g(x0) g(x1) g(x2) g(x3)
B = -sp.I * C * gamma[4]  # B = -i C g(x4)
B_record = (exact_matrix(gamma_record["B"]["re"])
            + sp.I * exact_matrix(gamma_record["B"]["im"]))
check(C == exact_matrix(gamma_record["C"]) and B == B_record,
      "C and B built from the gammas equal the matrices of the record")
check(C * C == sp.eye(16) and B * B == sp.eye(16) and C == C.T,
      "C is symmetric with C^2 = 1, and B^2 = 1")
```

Out [16]:

```
PASS C and B built from the gammas equal the matrices of the record
PASS C is symmetric with C^2 = 1, and B^2 = 1
```

Text cell:

```
The next cell builds $\rho = (nB + SC)/16$ with symbols $n$, $S$, $\lambda$, checks
that it gives back the densities ($\mathrm{Tr}\,B\rho = n$, $\mathrm{Tr}\,C\rho = S$),
computes $e_H$ and $e_x$ exactly, and compares them with the Revision record: the
exchange coefficients $-1/32$ of $n^2$ and of $S^2$ (Revision/kohn_sham/
ks-theory.json and the solver's parameter file) and the checks of the independent
sympy verification (Revision/kohn_sham/reports/ks-theory-python.json).
```

In [17]:

```
n_sym, S_sym, lam = sp.symbols("n S lambda", real=True)
rho = (n_sym * B + S_sym * C) / 16
check(sp.simplify((B * rho).trace() - n_sym) == 0
      and sp.simplify((C * rho).trace() - S_sym) == 0,
      "Tr(B rho) = n and Tr(C rho) = S")
e_H = sp.expand(lam / 2 * (C * rho).trace() ** 2)
e_x = sp.expand(-lam / 2 * (C * rho * C * rho).trace())
say(f"e_H = {e_H}")
say(f"e_x = {e_x}")
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
gas = theory["exchange"]["uniformGas"]
coefficient_n2 = sp.Rational(gas["coefficient_n2"])  # "-1/32" in the record
coefficient_S2 = sp.Rational(gas["coefficient_S2"])
check(e_H == lam * S_sym ** 2 / 2, "the Hartree energy is (lambda/2) S^2")
recorded = lam * (coefficient_n2 * n_sym ** 2 + coefficient_S2 * S_sym ** 2)
check(sp.expand(e_x - recorded) == 0
      and coefficient_n2 == coefficient_S2 == sp.Rational(-1, 32),
      "e_x = -(lambda/32)(n^2 + S^2)",
      record="Revision/kohn_sham/reports/ks-theory-python.json, check "
             "exchange_uniform_gas")
```

Out [17]:

```
PASS Tr(B rho) = n and Tr(C rho) = S
e_H = S**2*lambda/2
e_x = -S**2*lambda/32 - lambda*n**2/32
PASS the Hartree energy is (lambda/2) S^2
PASS e_x = -(lambda/32)(n^2 + S^2)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    exchange_uniform_gas
```

Text cell:

```
The Kohn-Sham potentials are the derivatives of $e_{int} = e_H + e_x$: the
effective mass $M_{eff} = m + \partial e_{int}/\partial S$ and the vector potential
$v_v = \partial e_{int}/\partial n$. One filled 8-fold level at rest has $n = S$; the
ratio $E_x/E_H$ is then $-1/8$, the analogue of $-1/g$ with $g = 8$. The next cell
computes all three exactly and compares them with the record and with the numbers
that the Revision solver uses.
```

In [18]:

```
e_int = e_H + e_x
mass_coefficient = sp.simplify(sp.diff(e_int, S_sym) / (lam * S_sym))  # 15/16
vector_coefficient = sp.simplify(sp.diff(e_int, n_sym) / (lam * n_sym))  # -1/16
filled_ratio = sp.simplify((e_x / e_H).subs(n_sym, S_sym))  # n = S
say(f"M_eff = m + ({mass_coefficient}) lambda S;  v_v = ({vector_coefficient}) "
    f"lambda n;  E_x/E_H at n = S: {filled_ratio}")
potentials = theory["exchange"]["kohnShamPotentials"]
check(mass_coefficient == sp.Rational(potentials["Meff_coefficient_of_lambda_S"])
      == sp.Rational(15, 16)
      and vector_coefficient == sp.Rational(potentials["vv_coefficient_of_lambda_n"])
      == sp.Rational(-1, 16),
      "M_eff = m + (15/16) lambda S and v_v = -(1/16) lambda n",
      record="Revision/kohn_sham/reports/ks-theory-python.json, check ks_potentials")
check(filled_ratio == sp.Rational(-1, 8),
      "one filled 8-fold level at rest: E_x/E_H = -1/8",
      record="Revision/kohn_sham/reports/ks-theory-python.json, check "
             "filled_shell_ratio")
solver = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))["theoryInputs"]
check(solver["exchangeCoefficientN2"] == solver["exchangeCoefficientS2"] == -1 / 32
      and solver["MeffCoefficientOfLambdaS"] == 15 / 16
      and solver["vvCoefficientOfLambdaN"] == -1 / 16,
      "the Revision solver uses exactly these coefficients (parameters.json)")
report_checks = {c["name"]: c["verdict"] for c in json.loads(repository_file(
    "Revision/kohn_sham/reports/ks-theory-python.json").read_text(
    encoding="utf-8"))["checks"]}
check(all(report_checks[name] == "PASS" for name in
          ("exchange_uniform_gas", "ks_potentials", "filled_shell_ratio")),
      "the three Revision checks are recorded as PASS")
```

Out [18]:

```
M_eff = m + (15/16) lambda S;  v_v = (-1/16) lambda n;  E_x/E_H at n = S: -1/8
PASS M_eff = m + (15/16) lambda S and v_v = -(1/16) lambda n
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check ks_potentials
PASS one filled 8-fold level at rest: E_x/E_H = -1/8
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    filled_shell_ratio
PASS the Revision solver uses exactly these coefficients (parameters.json)
PASS the three Revision checks are recorded as PASS
```

Text cell:

```
The next cell draws the two matrices of the vertex and the density rule as heat
maps: $C$ is real (entries $0, \pm 1$) and $B$ is purely imaginary, so its imaginary
part is drawn.
```

In [19]:

```
C_numbers = np.array(C.tolist(), dtype=float)
B_imaginary = np.array([[float(sp.im(entry)) for entry in row] for row in B.tolist()])
check(all(sp.re(entry) == 0 for entry in B), "B is purely imaginary")
check(all(np.count_nonzero(row) == 1 for row in np.vstack([C_numbers, B_imaginary])),
      "every row of C and of B has exactly one nonzero entry")
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, matrix, title in ((axes[0], C_numbers, "$C$ (real)"),
                          (axes[1], B_imaginary, "imaginary part of $B$")):
    image = ax.imshow(matrix, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xlabel("column")
    ax.set_ylabel("row")
fig.colorbar(image, ax=axes, shrink=0.8)
save_figure(fig, "dirac_matrices",
            "Heat maps of the $16 \\times 16$ matrices of the contact interaction "
            "of dirac16complex, built exactly from the author's gamma matrices of "
            "the Revision record: the vertex $C = \\gamma^{(x0)}\\gamma^{(x1)}"
            "\\gamma^{(x2)}\\gamma^{(x3)}$ (left; red $+1$, blue $-1$, grey 0) "
            "and the imaginary part of $B = -i\\,C\\gamma^{(x4)}$ (right); rows and "
            "columns are the 16 spinor components. Each row of either matrix has "
            "exactly one nonzero entry.")
```

Out [19]:

```
PASS B is purely imaginary
PASS every row of C and of B has exactly one nonzero entry
```

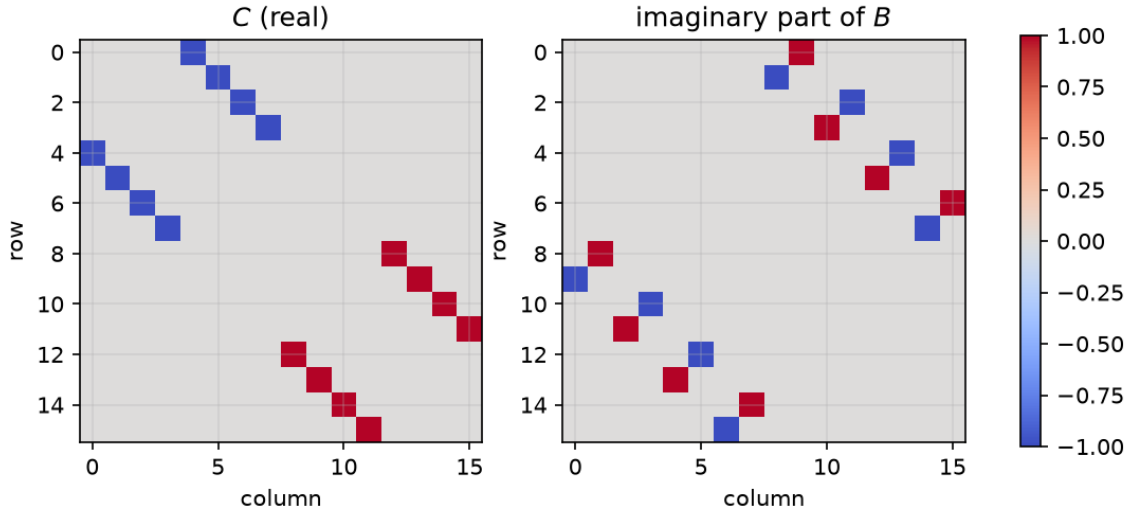

Figure 359: Heat maps of the 16×16 matrices of the contact interaction of dirac16complex, built exactly from the author's gamma matrices of the Revision record: the vertex $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$ (left; red $+1$, blue -1, grey 0) and the imaginary part of $B = -i\,C\gamma^{(x4)}$ (right); rows and columns are the 16 spinor components. Each row of either matrix has exactly one nonzero entry. (Notebook 13b, figure 8.)

Out [19]:

```
Figure 13b.8 saved as Revision/textbook/figures/13b_8_dirac_matrices.png
```

Text cell:

```
Finally, the next cell draws $e_H$, $e_x$ and $e_{int}$ per unit $\lambda n^2$
against the ratio $S/n$ (for a gas at rest $S = n$; moving particles have
$S < n$), and marks the filled level at rest, where $e_x/e_H = -1/8$.
```

In [20]:

```
ratio_values = np.linspace(0.0, 1.0, 101)  # S / n
e_H_curve = 0.5 * ratio_values ** 2  # e_H / (lambda n^2)
e_x_curve = -(1.0 + ratio_values ** 2) / 32.0  # e_x / (lambda n^2)
fig, ax = plt.subplots()
ax.plot(ratio_values, e_H_curve, label="Hartree $e_H = \\lambda S^2/2$")
ax.plot(ratio_values, e_x_curve, label="exchange $e_x = -\\lambda(n^2+S^2)/32$")
ax.plot(ratio_values, e_H_curve + e_x_curve, color="black", lw=2.0,
        label="$e_{int} = e_H + e_x$")
ax.plot([1.0], [-1.0 / 16.0], "o", color="red",
        label="filled level at rest: $e_x/e_H = -1/8$")
ax.axhline(0.0, color="gray", lw=0.8)
ax.set_xlabel("$S/n$ (scalar density over number density)")
ax.set_ylabel("energy per volume / $(\\lambda n^2)$")
ax.set_title("Hartree and exchange of the uniform 8-fold dirac16complex gas")
ax.legend(fontsize=8)
save_figure(fig, "dirac_exchange",
            "The Hartree energy $e_H = \\lambda S^2/2$, the exchange energy "
            "$e_x = -\\lambda(n^2 + S^2)/32$ and their sum per unit volume of the "
            "uniform good-sector gas of dirac16complex, in units of "
            "$\\lambda n^2$, against $S/n$ (pure number), as recorded in the "
            "Revision Kohn-Sham theory and reproduced here from the gamma "
            "matrices. The red point is one filled level at rest ($S = n$), where "
            "$e_x = -e_H/8$, the rule $-1/g$ with $g = 8$.")
check(abs(e_x_curve[-1] / e_H_curve[-1] + 1.0 / 8.0) < 1e-15,
      "the plotted curves give e_x/e_H = -1/8 at S = n")
```

Out [20]:

```
Figure 13b.9 saved as Revision/textbook/figures/13b_9_dirac_exchange.png
PASS the plotted curves give e_x/e_H = -1/8 at S = n
```

Text cell:

```
## 13. What the Revision record says about the dirac16complex model

Section 12 recomputed the exchange formulas of the Revision record from the gamma
matrices. The record also states on what these formulas rest, how good they are,
and how the Kohn-Sham model of dirac16complex that uses them must be read. The
next two cells read these entries and check each of them, so that a change of the
```

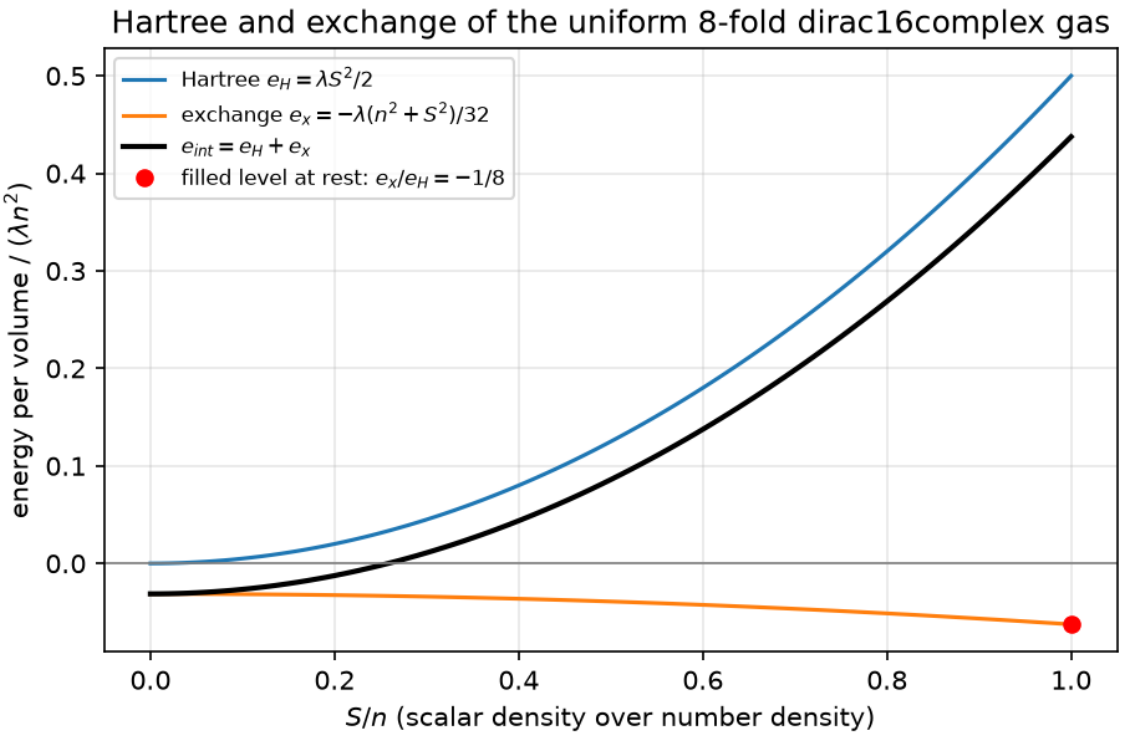

Figure 360: The Hartree energy $e_H = \lambda S^2/2$, the exchange energy $e_x = -\lambda(n^2 + S^2)/32$ and their sum per unit volume of the uniform good-sector gas of dirac16complex, in units of λn^2, against S/n (pure number), as recorded in the Revision Kohn-Sham theory and reproduced here from the gamma matrices. The red point is one filled level at rest ($S = n$), where $e_x = -e_H/8$, the rule $-1/g$ with $g = 8$. (Notebook 13b, figure 9.)

```
record stops this notebook with a failed check. A **report** of the record is a
file that lists checks, each with a name and a verdict (PASS or FAIL). The first
cell checks:

- the algebra report of the Wolfram verification: the checks reality (the gamma
  matrices are real), Clifford_relation
  ($\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}$) and C_real_symmetric are PASS;
- the Wolfram verification of the Kohn-Sham theory, a second program independent
  of the sympy one: the checks exchange_uniform_gas and ks_potentials (the formulas
  of section 12) and exchange_slab_exact_fock are PASS;
- the sympy verification: the checks exchange_slab_exact_fock and
  hf_wick_contraction (the rule of Wick's theorem that section 12 used) are PASS;
- what is approximate: for the non-uniform Kohn-Sham states the exact (Fock)
  exchange is still local but differs from the uniform-gas formula by
  $+\frac{\lambda}{32}Q^2$ per volume, where $Q$ is a further density; the theory
  file labels this entry DIAGNOSTIC, the solver's parameter file states the same
  difference, and the solver reports it in the column deltaE_x_exact_fock_diag of
  its table exact-fock-variant.csv; and the functional has no correlation term.
```

In [21]:

```
def verdicts(path):
    """The verdict of every check of a Revision report, by the name of the check."""
    checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
    return {entry["name"]: entry["verdict"] for entry in checks}


ALGEBRA = "Revision/algebra/reports/wolfram-algebra.json"
KS_WOLFRAM = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
KS_PYTHON = "Revision/kohn_sham/reports/ks-theory-python.json"
algebra = verdicts(ALGEBRA)
check(all(algebra[name] == "PASS"
          for name in ("reality", "Clifford_relation", "C_real_symmetric")),
      f"{ALGEBRA}: reality, Clifford_relation and C_real_symmetric are PASS")
ks_wolfram, ks_python = verdicts(KS_WOLFRAM), verdicts(KS_PYTHON)
check(all(ks_wolfram[name] == "PASS" for name in
          ("exchange_uniform_gas", "ks_potentials", "exchange_slab_exact_fock")),
      f"{KS_WOLFRAM}: exchange_uniform_gas, ks_potentials and "
      "exchange_slab_exact_fock are PASS")
check(ks_python["exchange_slab_exact_fock"] == "PASS"
      and ks_python["hf_wick_contraction"] == "PASS",
      f"{KS_PYTHON}: exchange_slab_exact_fock and hf_wick_contraction are PASS")
slab = theory["exchange"]["exactFockSlab"]  # the exact exchange of the KS states
correlation = theory["exchange"]["kohnShamPotentials"]["correlation"]
solver_file = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                         .read_text(encoding="utf-8"))
conventions = solver_file["conventions"]  # the conventions of the Revision solver
status_word = slab["status"].split(":")[0]  # the word before the first colon
say(f"exactFockSlab status: {status_word}; correlation: {correlation}")
check(slab["status"].startswith("DIAGNOSTIC")
      and "Delta E_x = +(lambda/32) int sqrt|g| Q^2" in slab["status"]
      and "+lambda Q^2/32" in conventions["exactExchange"]
      and correlation == "none (Hartree plus exchange only)",
```

```
      "ks-theory.json: the exact Fock exchange differs by +(lambda/32) Q^2 "
      "(DIAGNOSTIC; also in parameters.json); no correlation term")
table = repository_file("Revision/kohn_sham/results/exx/exact-fock-variant.csv")
columns = table.read_text(encoding="utf-8").splitlines()[0].split(",")
check("deltaE_x_exact_fock_diag" in columns,
      "exact-fock-variant.csv has the column deltaE_x_exact_fock_diag")
```

Out [21]:

```
PASS Revision/algebra/reports/wolfram-algebra.json: reality, Clifford_relation and
    C_real_symmetric are PASS
PASS Revision/kohn_sham/reports/ks-theory-wolfram.json: exchange_uniform_gas,
    ks_potentials and exchange_slab_exact_fock are PASS
PASS Revision/kohn_sham/reports/ks-theory-python.json: exchange_slab_exact_fock and
    hf_wick_contraction are PASS
exactFockSlab status: DIAGNOSTIC; correlation: none (Hartree plus exchange only)
PASS ks-theory.json: the exact Fock exchange differs by +(lambda/32) Q^2 (DIAGNOSTIC;
    also in parameters.json); no correlation term
PASS exact-fock-variant.csv has the column deltaE_x_exact_fock_diag
```

Text cell:

```
The second cell checks how the Kohn-Sham model of dirac16complex must be read:

- the solver's parameter file: the temperatures $T = 0.01$, $0.02$ and $0.05$ (in
  units of the mass $m$), and the first excited state by an ensemble Delta-SCF
  (one particle moved from the highest occupied group of equal levels to the
  lowest empty group, spread evenly over each group);
- which levels the particles fill is a CONVENTION of the theory file whose
  justification is OPEN, and the history $a_4 = AHx_4$ along which the states are
  computed is a PRESCRIBED BACKGROUND (the parameter file says so);
- the report on the source conditions of the $a_4$ field equations: the checks
  ks_history_is_a_prescribed_background, ks_profiles_violate_algebraic_condition
  (the Kohn-Sham states violate the condition $p_3 + p_t = 2p_0$ that the $a_4$
  equations put on their source) and ks_profiles_depend_on_x0 (their profiles
  depend on the hidden coordinate $x_0$) are PASS;
- the pairing theorem T3 (the Kohn-Sham universes of mass $+M$ and $-M$ have equal
  energies and energy-momentum tensors) is checked by two independent programs, a
  Python one and a Wolfram one; ten checks of the theorem that are present in
  both reports (the maps of the block Hamiltonian, of its equation, of the tip condition
  and of the brane parities, the orbital densities, the mean field, the equality
  of the energies and energy-momentum profiles, the exact spectra at zero
  momentum, the matrix $\Gamma$ as the block map, and the mirror copy) are PASS in
  both. T3 is an exact map between two sets of solutions; it does not show that
  any universe is created.
```

In [22]:

```
temperatures = solver_file["physics"]["temperatures"]
say(f"temperatures of the Revision solver: {temperatures}")
check(temperatures == [0.01, 0.02, 0.05]
```

```
      and "ensemble Delta-SCF" in conventions["deltaScf"],
      "parameters.json: T = 0.01, 0.02, 0.05 and the ensemble Delta-SCF (deltaScf)")
filling = theory["thermodynamics"]["fillingConvention"]
check(filling.startswith("CONVENTION") and "its justification is OPEN" in filling
      and conventions["history"].startswith("PRESCRIBED BACKGROUND"),
      "the filling is a CONVENTION whose justification is OPEN (ks-theory.json); "
      "the history is a PRESCRIBED BACKGROUND (parameters.json)")
SOURCE = "Revision/field_equations_a4/reports/ks-source-conditions.json"
source = verdicts(SOURCE)
check(all(source[name] == "PASS" for name in
          ("ks_history_is_a_prescribed_background",
           "ks_profiles_violate_algebraic_condition", "ks_profiles_depend_on_x0")),
      f"{SOURCE}: ks_history_is_a_prescribed_background, "
      "ks_profiles_violate_algebraic_condition and ks_profiles_depend_on_x0 are PASS")
T3_CHECKS = ("block_hamiltonian_map", "ode_map", "tip_condition_map",
             "brane_parities_exchanged", "orbital_densities", "mean_field_map",
             "energies_and_emt_profiles_equal", "exact_k0_spectra",
             "Gamma_is_the_block_map", "z2_mirror_copy_carries_minus_m_plus_lambda")
t3_python = verdicts("Revision/pairing/kohn_sham/reports/python-t3.json")
t3_wolfram = verdicts("Revision/pairing/kohn_sham/reports/wolfram-t3.json")
check(all(t3_python["T3." + name] == "PASS" for name in T3_CHECKS),
      "theorem T3: ten checks of python-t3.json (T3.<name>) are PASS")
check(all(t3_wolfram["T3_" + name] == "PASS" for name in T3_CHECKS),
      "theorem T3: the same ten checks of wolfram-t3.json (T3_<name>) are PASS")
```

Out [22]:

```
temperatures of the Revision solver: [0.01, 0.02, 0.05]
PASS parameters.json: T = 0.01, 0.02, 0.05 and the ensemble Delta-SCF (deltaScf)
PASS the filling is a CONVENTION whose justification is OPEN (ks-theory.json); the
    history is a PRESCRIBED BACKGROUND (parameters.json)
PASS Revision/field_equations_a4/reports/ks-source-conditions.json:
    ks_history_is_a_prescribed_background, ks_profiles_violate_algebraic_condition and
    ks_profiles_depend_on_x0 are PASS
PASS theorem T3: ten checks of python-t3.json (T3.<name>) are PASS
PASS theorem T3: the same ten checks of wolfram-t3.json (T3_<name>) are PASS
```

Text cell:

```
## 14. The last check

The last cell checks that all nine figure files exist and prints the number of
checks that passed.
```

In [23]:

```
figure_names = ["density_matrix", "exchange_hole", "contact_energies",
                "finite_range", "contact_vs_coulomb", "local_exchange",
                "polarization", "dirac_matrices", "dirac_exchange"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13b_{k}_{name}.png").is_file()]
```

```
check(missing == [], "all nine figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13b_9_dirac_exchange.png").is_file(),
      "the figure file 13b_9_dirac_exchange.png exists")
all_checks_passed()
```

Out [23]:

```
PASS all nine figure files exist
PASS the figure file 13b_9_dirac_exchange.png exists
ALL 44 CHECKS PASSED (notebook 13b)
```

Text cell:

```
## 15. What this notebook showed

- The density matrix of one label of the uniform gas is $n_\sigma F(k_F R)$ with
  $F(s) = 3(\sin s - s\cos s)/s^3$; finite boxes approach it as they grow.
- The Pauli principle digs an exchange hole of depth $1/g$ at contact; a contact
  interaction therefore feels only pairs of different labels:
  $E_H + E_x = \frac{g_c}{2}\int n^2(1 - 1/g)$, i.e. $E_x = -E_H/g$, and for a single
  label the interaction cancels completely.
- The exchange energy of a contact interaction is EXACTLY local, for the uniform gas
  and for every Slater determinant (with label-mixing orbitals it is local in the
  local $2\times 2$ density matrix); a finite range spoils this when the range
  becomes comparable with the size $1/k_F$ of the exchange hole. The Coulomb
  interaction gives Dirac's $-\frac34(3/\pi)^{1/3}n^{4/3}$ instead.
- In the exchange-only picture a strong contact repulsion makes the unpolarized gas
  unstable at $g_c n^{1/3} = \frac23(3\pi^2)^{2/3}$.
- For the 16-component field the vertex is the matrix $C$, and the same Wick rule
  gives $e_x = -\frac{\lambda}{32}(n^2 + S^2)$, $M_{eff} = m + \frac{15}{16}\lambda S$,
  $v_v = -\frac{1}{16}\lambda n$ and the filled-shell ratio $-1/8$: exactly the
  numbers of the Revision Kohn-Sham theory and of its solver (REPRODUCED here from
  the gamma matrices).
- The Revision record verifies these formulas with two independent programs and
  states their limits: the exact Fock exchange of the non-uniform Kohn-Sham states
  differs by $+\frac{\lambda}{32}Q^2$ (a DIAGNOSTIC of the record), there is no
  correlation term, the filling of the levels is a CONVENTION, and the history
  $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND; ten checks of theorem T3 are PASS
  in both of its reports (CHECKED here by reading the record).
```

13.20 Line-by-line walk-through of Notebook 13b

The notebook has 21 code cells. **In [1]** is the set-up cell, identical to In [1] of Notebook 13c (explained line by line in Section 13.14) except that its name is `"13b"` and its comment lines hold the instructions of Section 13.18.

In [2], the function F.

```
import numpy as np  # arrays, matrices and linear algebra
```

The cell loads numpy under the short name `np`, as in Notebook 13c.

```
def F(s):
    """F(s) = 3 (sin s - s cos s) / s^3, the density matrix of one label over n."""
    s = np.asarray(s, dtype=float)
    result = 1.0 - s ** 2 / 10.0 + s ** 4 / 280.0  # the Taylor series near 0
    big = s >= 1e-3  # where the closed form is accurate
    sb = s[big]
    result[big] = 3.0 * (np.sin(sb) - sb * np.cos(sb)) / sb ** 3
    return result
```

`np.asarray(s, dtype=float)` turns the argument, a number or an array, into an array of floating-point numbers. For small s the closed form divides two tiny numbers and loses digits, so the function first fills `result` with the Taylor series of Section 13.15 everywhere, then marks the places where $s \geq 10^{-3}$ (`big` is an array of true and false values), takes those values (`s[big]`) and overwrites the result there with the closed form.

```
s_test = 1e-3
closed = 3.0 * (np.sin(s_test) - s_test * np.cos(s_test)) / s_test ** 3
series = 1.0 - s_test ** 2 / 10.0 + s_test ** 4 / 280.0
say(f"at s = 0.001: closed form {closed:.12f}, series {series:.12f}")
check(abs(closed - series) < 1e-8,
      "the closed form and the series agree at s = 0.001 (to about 9 digits)")
```

At the switching point both forms are computed and printed with 12 decimals: 0.999999899944 and 0.999999900000. They differ by $6 \cdot 10^{-11}$: the closed form has already lost several digits to rounding (the exact value is $1 - 10^{-7} + 3.6 \cdot 10^{-15}$), which is why the series is used below this point.

In [3], box sums.

```
s_grid = np.linspace(0.0, 12.0, 121)  # s = k_F R from 0 to 12
box_results = {}
for m_F in (4, 8, 16, 32):
    m = np.arange(-m_F, m_F + 1)  # the possible integers m_x, m_y, m_z
    my, mz = np.meshgrid(m, m, indexing="ij")  # all pairs (m_y, m_z)
    q = my ** 2 + mz ** 2
    counts = np.array([np.count_nonzero(q < m_F ** 2 - mx ** 2) for mx in m])
    N_label = int(counts.sum())  # occupied plane waves of one label
```

For spheres of radius $m_F = 4, 8, 16, 32$ lattice steps: `np.arange(-m_F, m_F + 1)` is the list of integers from $-m_F$ to m_F; `np.meshgrid` makes all pairs (m_y, m_z), and `q` their $m_y^2 + m_z^2$. For each m_x, `np.count_nonzero(q < m_F ** 2 - mx ** 2)` counts the pairs with $m_x^2 + m_y^2 + m_z^2 < m_F^2$: the number $c(m_x)$ of occupied plane waves with this m_x. Their sum is N_σ.

```
    kF_ell = 2.0 * np.pi * (3.0 * N_label / (4.0 * np.pi)) ** (1.0 / 3.0)
    R_over_ell = s_grid / kF_ell  # the separations R / ell for these s
    ratio = np.array([np.sum(counts * np.cos(2.0 * np.pi * m * r))
                      for r in R_over_ell]) / N_label
    box_results[m_F] = (N_label, ratio)
```

```
    deviation = np.max(np.abs(ratio - F(s_grid)))
    say(f"m_F = {m_F:2d}: {N_label:6d} plane waves; largest difference from "
        f"F = {deviation:.5f}")
```

The Fermi wave number is defined by the count, $N_\sigma = \frac{4}{3}\pi(k_F\ell/2\pi)^3$, solved for $k_F\ell$. For each s the separation is $R/\ell = s/(k_F\ell)$, and `ratio` is the box sum $\frac{1}{N_\sigma}\sum_{m_x} c(m_x)\cos(2\pi m_x R/\ell)$ of Section 13.15. The cell stores the result and prints the largest difference from F on the grid of s.

```
deviations = [np.max(np.abs(box_results[m_F][1] - F(s_grid)))
              for m_F in (4, 8, 16, 32)]
shrinking = all(a > b for a, b in zip(deviations, deviations[1:]))
check(shrinking and deviations[-1] < 2e-3,
      "the box sums approach the closed form F as the box holds more particles")
```

`zip(deviations, deviations[1:])` pairs each difference with the next one; `all(a > b ...)` is true when every difference is larger than the next. The check also requires the largest box to be within $2 \cdot 10^{-3}$ of F. The printed differences are 0.03729, 0.01218, 0.00184 and 0.00083 for 251, 2103, 17071 and 137059 plane waves.

In [4], the density matrix as a picture.

```
fig, ax = plt.subplots()
ax.plot(s_grid, F(s_grid), color="black", lw=2.0,
        label="continuum $F(k_F R) = 3(\\sin s - s\\cos s)/s^3$")
```

One panel; the closed form F on the 121 values of s as a thick black line. In the label, `\\sin` reaches matplotlib as `\sin`, which it draws as the function name sin.

```
for m_F, marker in ((4, "o"), (32, "x")):
    N_label, ratio = box_results[m_F]
    ax.plot(s_grid[::3], ratio[::3], marker, ms=4,
            label=f"box sum, {N_label} plane waves")
ax.axhline(0.0, color="gray", lw=0.8)
```

For the smallest box ($m_F = 4$, circles, style `"o"`) and the largest ($m_F = 32$, crosses, `"x"`) the loop takes the stored count and box sum and draws every third value (`[::3]` takes the entries 0, 3, 6, ...), as markers without a connecting line, so that the black curve stays visible; the label gives the number of plane waves. `axhline(0.0, ...)` draws the zero line.

```
ax.set_xlabel("$s = k_F R$ (separation times Fermi wave number)")
ax.set_ylabel("$\\rho_\\sigma(R) / n_\\sigma$")
ax.set_title("Density matrix of one label of the uniform gas")
ax.legend(fontsize=8)
save_figure(fig, "density_matrix",
```

Axis labels (both quantities are pure numbers), title, legend and `save_figure` for Figure 13b.1; one line of its caption is an f-string that writes the two plane-wave counts, 251 and 137059, into the caption.

What Figure 13b.1 shows. The density matrix of one label starts at 1 for $R = 0$ (there it is the density itself), falls to zero near $k_F R = 4.5$ (the first zero of $\sin s - s\cos s$, where $\tan s = s$), swings to a small negative minimum of about -0.086 near $k_F R = 5.76$, and oscillates with a shrinking amplitude. A fermion "remembers" another one of its label only within a distance of a few $1/k_F$. The crosses of the large box lie on the curve; the circles of the small box (251 plane waves) deviate visibly beyond $k_F R \approx 4$, by at most 0.037.

In [5], the exchange hole.

```
s_fine = np.linspace(0.0, 10.0, 501)
fig, ax = plt.subplots()
for g in (1, 2, 8):
    pair = 1.0 - F(s_fine) ** 2 / g  # the pair distribution
    ax.plot(s_fine, pair, label=f"$g = {g}$ labels: $g_{{pair}}(0) = {1 - 1 / g:.3f}$")
    check(abs(pair[0] - (1.0 - 1.0 / g)) < 1e-15,
          f"the pair distribution at contact is 1 - 1/g for g = {g}")
```

For $g = 1, 2, 8$ the cell computes $g_{pair} = 1 - F^2/g$ of Section 13.15 on 501 values of s from 0 to 10, draws it (in an f-string a doubled brace prints one brace, so the label shows g_{pair}, followed by the value $1 - 1/g$ with three decimals), and checks its value at $s = 0$, the first entry `pair[0]`.

```
ax.axhline(1.0, color="gray", ls="--", lw=0.8)
ax.set_xlabel("$k_F R$")
ax.set_ylabel("pair distribution $g_{pair}(R)$")
ax.set_title("The exchange hole of the uniform gas")
ax.legend(fontsize=8)
save_figure(fig, "exchange_hole",
```

A grey dashed horizontal line at 1 (the keyword `ls`, short for line style, set to two hyphens, makes it dashed), the value without the Pauli principle; axis labels, title, legend, and `save_figure` for Figure 13b.2.

What Figure 13b.2 shows. Far from a fermion (beyond about $k_F R = 4$) all three curves are at 1: there the Pauli principle has no effect. Near it they dip to $1 - 1/g$ at contact: to 0 for one label (no second fermion can come close), to $\frac{1}{2}$ for two labels (only the half with the other label can), and to 0.875 for eight labels. The more labels, the shallower the hole, because a smaller fraction $1/g$ of the other fermions shares the first one's label.

In [6], contact energies.

```
G_C = 1.0  # the strength of the contact interaction
densities = np.linspace(0.0, 2.0, 41)  # n from 0 to 2 (particles per volume)


def contact_energies(n, g):
    """(e_H, e_x) per volume for g equally occupied labels: n_sigma = n / g."""
    e_hartree = 0.5 * G_C * n ** 2
    e_exchange = -0.5 * G_C * g * (n / g) ** 2  # -(g_c/2) sum over g labels
    return e_hartree, e_exchange
```

The function returns $e_H = \frac{g_c}{2} n^2$ and $e_x = -\frac{g_c}{2} \sum_\sigma n_\sigma^2$ with the g equal label densities n/g, for 41 densities from 0 to 2.

```
for g in (1, 2, 8):
    e_hartree, e_exchange = contact_energies(densities, g)
    ratio = e_exchange[1:] / e_hartree[1:]  # skip n = 0 (0/0)
    check(np.allclose(ratio, -1.0 / g, rtol=0, atol=1e-15),
          f"e_x = -e_H/g for g = {g}")
e_hartree, e_exchange = contact_energies(densities, 1)
check(np.max(np.abs(e_hartree + e_exchange)) == 0.0,
      "for a single label the contact interaction cancels exactly")
```

`[1:]` leaves out the first entry, $n = 0$, where the ratio would be $0/0$. Three checks of $e_x/e_H = -1/g$, and one that for $g = 1$ the sum is exactly zero.

In [7], the energies as pictures.

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.0), sharey=True)
for ax, g in zip(axes, (2, 8)):
    e_hartree, e_exchange = contact_energies(densities, g)
    ax.plot(densities, e_hartree, label="Hartree $e_H = g_c n^2/2$")
    ax.plot(densities, e_exchange, label=f"exchange $e_x = -e_H/{g}$")
    ax.plot(densities, e_hartree + e_exchange, color="black", lw=2.0,
            label="sum $e_H + e_x$")
```

Two panels with a common vertical axis (`sharey=True`: both use the same scale, so the curves can be compared by eye). For $g = 2$ (left panel) and $g = 8$ (right panel) the loop computes the two energy densities on the 41 densities and draws e_H, e_x and, as a thick black line, their sum.

```
    ax.axhline(0.0, color="gray", lw=0.8)
    ax.set_xlabel("density $n$")
    ax.set_title(f"$g = {g}$ labels")
    ax.legend(fontsize=8)
axes[0].set_ylabel("energy per volume (units of $g_c$)")
save_figure(fig, "contact_energies",
```

Still inside the loop: the zero line, the horizontal label, a title that names g, and a legend for each panel. After the loop, only the left panel (`axes[0]`) gets the vertical label, which the right panel shares; `save_figure` saves Figure 13b.3.

What Figure 13b.3 shows. The Hartree energy $n^2/2$ is the same parabola in both panels (it reaches 2 at $n = 2$). The exchange energy mirrors a part of it below zero: half of it for two labels (-1 at $n = 2$), one eighth for eight labels (-0.25). The sum is therefore half of the Hartree energy for $g = 2$ and seven eighths of it for $g = 8$: the more labels, the larger the fraction of pairs that can meet, and the less exchange removes.

In [8], a finite range.

```
def simpson(values, step):
    """Simpson's rule for equally spaced values (an odd number of them)."""
    return step / 3.0 * (values[0] + values[-1] + 4.0 * values[1:-1:2].sum()
                         + 2.0 * values[2:-1:2].sum())
```

Simpson's rule of Section 13.15: `values[1:-1:2]` are the entries 1, 3, 5, ... (every second one, starting at 1, without the last), weighted 4; `values[2:-1:2]` are the inner even entries 2, 4, ..., weighted 2; the two ends are weighted 1.

```
s_int = np.linspace(0.0, 80.0, 400001)  # s = k_F R; the Gaussian is tiny beyond
ds = s_int[1] - s_int[0]
F2 = F(s_int) ** 2


def exchange_ratio(kF_a):
    """e_x(a) / e_x(0) for the Gaussian of range a (k_F a given)."""
    norm = (2.0 * np.pi * kF_a ** 2) ** -1.5  # makes the integral of w_a equal g_c
    gauss = norm * np.exp(-s_int ** 2 / (2.0 * kF_a ** 2))
    return simpson(4.0 * np.pi * s_int ** 2 * gauss * F2, ds)
```

The integral of Section 13.15 is written in the variable $s = k_F R$, so lengths are measured in units of $1/k_F$ and the range enters only as $k_F a$: the ratio is $\int_0^\infty 4\pi s^2 \, (2\pi (k_F a)^2)^{-3/2} e^{-s^2/(2(k_F a)^2)} F(s)^2 \, ds$, integrated by Simpson's rule on 400001 points from 0 to 80 (spacing 0.0002; beyond 80 the Gaussian is negligible for the ranges used).

```
ranges = np.logspace(-2, 1, 31)  # k_F a from 0.01 to 10
ratios = np.array([exchange_ratio(r) for r in ranges])
for r in (0.01, 0.1, 1.0):
    say(f"k_F a = {r:5.2f}: e_x(a)/e_x(0) = {exchange_ratio(r):.6f}, "
        f"small-a formula {1.0 - 0.6 * r ** 2:.6f}")
check(abs(exchange_ratio(0.05) - (1.0 - 0.6 * 0.05 ** 2)) < 1e-5,
      "for small range the ratio is 1 - (3/5)(k_F a)^2")
check(np.all(np.diff(ratios) < 0.0) and ratios[0] > 0.9999,
      "the exchange energy grows towards the contact value as the range shrinks")
```

`np.logspace(-2, 1, 31)` gives 31 numbers from 10^{-2} to 10^{1}, equally spaced on a logarithmic scale. The printed lines compare the ratio with $1 - \frac{3}{5}(k_F a)^2$: they agree at $k_F a = 0.01$ (0.999940) and nearly at 0.1, and differ at 1 (0.588864 against 0.4), where the expansion no longer holds. The checks: the small-range formula at $k_F a = 0.05$, and the ratio falls with every increase of the range and is above 0.9999 at the smallest one.

In [9], the ratio as a picture.

```
fig, ax = plt.subplots()
ax.semilogx(ranges, ratios, "o-", ms=3, color="black",
            label="$e_x(a)/e_x(0)$, Gaussian of range $a$")
small = ranges[ranges < 0.6]
ax.semilogx(small, 1.0 - 0.6 * small ** 2, "--",
            label="small-range formula $1 - (3/5)(k_F a)^2$")
```

`ax.semilogx` draws like `plot`, but with a logarithmic horizontal axis, on which the 31 ranges from 10^{-2} to 10 are equally spaced. The black dots joined by a line are the computed ratios. `ranges[ranges < 0.6]` keeps only the ranges below 0.6 (an array indexed by an array of true and false values keeps the entries where the value is true), and for them the dashed curve is the small-range formula $1 - \frac{3}{5}(k_F a)^2$; for larger ranges the formula is meaningless (it becomes negative beyond $k_F a = 1.29$).

```
ax.set_ylim(0.0, 1.05)
ax.set_xlabel("range times Fermi wave number, $k_F a$")
ax.set_ylabel("exchange energy relative to contact")
ax.set_title("A finite-range interaction shrinking to a contact")
ax.legend(fontsize=8)
save_figure(fig, "finite_range",
```

`set_ylim(0.0, 1.05)` fixes the vertical range from 0 to 1.05; then the labels, the title, the legend and `save_figure` for Figure 13b.4.

What Figure 13b.4 shows. For ranges up to about $k_F a = 0.1$ the ratio is indistinguishable from 1: the interaction acts as a contact, and its exchange energy is the local contact value. Between 0.1 and 1 the ratio falls, first along the dashed parabola $1 - \frac{3}{5}(k_F a)^2$, then faster (0.589 at $k_F a = 1$), and for $k_F a = 10$ it is nearly 0: when the interaction reaches much farther than the exchange hole (size $1/k_F$), only a small part of it acts inside the hole, and the exchange energy almost disappears relative to the contact value.

In [10], Dirac's exchange.

```
S_MAX = 4000.0
s_long = np.linspace(0.0, S_MAX, 4000001)
integral_9_4 = simpson(s_long * F(s_long) ** 2, s_long[1] - s_long[0])
integral_9_4 += 9.0 / (4.0 * S_MAX ** 2)  # the tail beyond S_MAX
report("integral of s F(s)^2 from 0 to infinity", f"{integral_9_4:.8f}")
check(abs(integral_9_4 - 2.25) < 1e-7, "the Coulomb exchange integral is 9/4")
```

$\int_0^{4000} sF(s)^2\,ds$ by Simpson's rule on 4000001 points (spacing 0.001), plus the tail $9/(4 \cdot 4000^2)$ of Section 13.15 (`+=` adds to the variable). The result prints as 2.25000000 and the check requires it to be within 10^{-7} of 9/4.

```
dirac = 0.75 * (3.0 / np.pi) ** (1.0 / 3.0)  # e_x = -dirac n^(4/3)
n_test = 0.3
kF_test = (3.0 * np.pi ** 2 * n_test) ** (1.0 / 3.0)  # g = 2
e_x_coulomb = -4.0 * np.pi * (n_test / 2) ** 2 / kF_test ** 2 * integral_9_4
report("Dirac's constant (3/4)(3/pi)^(1/3)", f"{dirac:.6f}")
check(abs(e_x_coulomb + dirac * n_test ** (4.0 / 3.0)) < 1e-8,
      "the Coulomb exchange of the electron gas is -(3/4)(3/pi)^(1/3) n^(4/3)")
```

Dirac's constant is 0.738559. At the density $n = 0.3$ the cell evaluates $e_x = -4\pi n_\sigma^2\, I/k_F^2$ with $n_\sigma = n/2$ and the computed integral I, and checks that it equals $-0.738559\, n^{4/3}$.

In [11], exchange per particle.

```
n_values = np.logspace(-3, 1, 41)
contact_per_particle = 0.25 * G_C * n_values  # |e_x / n| for the contact, g = 2
coulomb_per_particle = dirac * n_values ** (1.0 / 3.0)  # |e_x / n|, Coulomb
slope_contact = np.polyfit(np.log(n_values), np.log(contact_per_particle), 1)[0]
slope_coulomb = np.polyfit(np.log(n_values), np.log(coulomb_per_particle), 1)[0]
```

For 41 densities from 10^{-3} to 10, the sizes of the exchange energy per particle: $g_c n/4$ for the contact with two labels ($e_x/n = -\frac{g_c}{2 \cdot 2} n$) and $0.738559\, n^{1/3}$ for the Coulomb gas. `np.polyfit(X, Y, 1)` fits

the straight line $Y = pX + c$ through the points, and `[0]` takes its slope p; with $X = \ln n$ and Y the logarithm of a power n^p, the slope is the power.

```
say(f"slopes: contact {slope_contact:.6f}, Coulomb {slope_coulomb:.6f}")
check(abs(slope_contact - 1.0) < 1e-12 and abs(slope_coulomb - 1.0 / 3.0) < 1e-12,
      "exchange per particle grows like n (contact) and n^(1/3) (Coulomb)")
```

The cell prints the slopes 1.000000 and 0.333333 and checks them to 10^{-12} (the points lie exactly on straight lines, so the fit is exact up to rounding).

```
fig, ax = plt.subplots()
ax.loglog(n_values, contact_per_particle, label="contact, $g_c = 1$: $n/4$")
ax.loglog(n_values, coulomb_per_particle, "--",
          label="Coulomb (Dirac): $0.7386\\,n^{1/3}$")
ax.set_xlabel("density $n$")
ax.set_ylabel("$|e_x|/n$, exchange energy per particle")
ax.set_title("Exchange per particle: contact versus Coulomb ($g = 2$)")
ax.legend()
save_figure(fig, "contact_vs_coulomb",
```

`ax.loglog` draws with both axes logarithmic: the contact curve solid, the Coulomb curve dashed (in the label, `\\,` is a small space in the formula). Labels, title, legend and `save_figure` for Figure 13b.5.

What Figure 13b.5 shows. On logarithmic axes both curves are straight lines: the contact exchange per particle rises with slope 1 (ten times the density gives ten times the exchange per particle), the Coulomb exchange with slope 1/3. The lines cross where $n/4 = 0.7386\, n^{1/3}$, at $n = (2.954)^{3/2} \approx 5.1$: below that density the Coulomb exchange per particle is the larger one, above it the contact exchange.

In [12], exact locality for a non-uniform determinant.

```
P = 99  # interior grid points of the box 0 < x < 1
h = 1.0 / (P + 1)
x = h * np.arange(1, P + 1)
W = G_C * np.eye(P) / h  # the contact interaction on the grid


def box_orbitals(count):
    """The lowest box orbitals sqrt(2) sin(m pi x), m = 1..count, as columns."""
    return np.array([np.sqrt(2.0) * np.sin(m * np.pi * x)
                     for m in range(1, count + 1)]).T
```

A line $0 < x < 1$ with 99 grid points, spacing $h = 1/100$. On the grid the delta function is the unit matrix divided by h (a sum over the grid times h then gives back g_c). `box_orbitals(count)` returns the lowest orbitals of a particle in the box as the columns of a matrix (`.T` transposes the list of rows).

```
phi_up, phi_down = box_orbitals(5), box_orbitals(3)
rho_up = phi_up @ phi_up.T  # rho_up(x, x') = sum_a phi_a(x) phi_a(x')
```

```
rho_down = phi_down @ phi_down.T
n_up, n_down = np.diag(rho_up), np.diag(rho_down)  # the label densities
fock = -0.5 * h * h * (np.sum(rho_up ** 2 * W) + np.sum(rho_down ** 2 * W))
local = -0.5 * G_C * h * np.sum(n_up ** 2 + n_down ** 2)
```

Five fermions with label up and three with label down. The matrix product of the orbital columns with their transpose is the density matrix of each label, $\rho(x,x') = \sum_a \phi_a(x)\phi_a(x')$, and its diagonal the label density. `fock` is the two-point exchange $-\frac{1}{2}\sum_{x,x'} h^2 |\rho_\sigma(x,x')|^2 W(x,x')$ summed over the labels (`rho_up ** 2 * W` multiplies entry by entry); `local` is $-\frac{g_c}{2}\sum_x h\,(n_\uparrow^2 + n_\downarrow^2)$.

```
report("exchange energy, two-point (Fock) form", f"{fock:.10f}")
report("exchange energy, local form", f"{local:.10f}")
check(abs(h * n_up.sum() - 5.0) < 1e-12 and abs(h * n_down.sum() - 3.0) < 1e-12,
      "the box determinant holds 5 up and 3 down fermions")
check(abs(fock - local) < 1e-12,
      "the contact exchange of a non-uniform determinant is exactly local")
```

Both forms print as -19.0000000000 (Exercise 10 derives this number by hand). The checks require the particle numbers $h\sum n_\uparrow = 5$ and $h\sum n_\downarrow = 3$ and the agreement of the two forms to 10^{-12}. (On the grid the two forms are equal term by term, because W is zero off the diagonal; the check confirms that the code implements both formulas correctly.)

In [13], label-mixing orbitals.

```
rng = np.random.default_rng(12345)  # a fixed seed: the same numbers every run
raw = rng.normal(size=(2 * P, 5)) + 1j * rng.normal(size=(2 * P, 5))
q_matrix = np.linalg.qr(raw)[0]  # five orthonormal columns of length 2P
spinor = (q_matrix / np.sqrt(h)).reshape(P, 2, 5)  # [point, label, orbital]
```

Five random orthonormal complex columns of length 2×99 (the QR factorisation of a random matrix with a fixed seed), divided by $\sqrt{h}$ so that $\sum |\phi|^2 h = 1$. `.reshape(P, 2, 5)` reads each column as 99 pairs of numbers: `spinor[x, s, a]` is the value of orbital a at the point x with the label s. Each orbital now has an up part and a down part.

```
# pair[x, a, b] = sum over the label of phi_a*(x, label) phi_b(x, label)
pair = np.einsum("xsa,xsb->xab", spinor.conj(), spinor)
exact = -0.5 * G_C * h * np.sum(np.abs(pair) ** 2)
# rho[x, s, t] = sum_a phi_a(x, s) phi_a*(x, t): the local 2 x 2 density matrix
rho_local = np.einsum("xsa,xta->xst", spinor, spinor.conj())
local_all = -0.5 * G_C * h * np.sum(np.abs(rho_local) ** 2)
diagonal = np.einsum("xss->xs", rho_local).real  # n_up(x), n_down(x)
local_diagonal = -0.5 * G_C * h * np.sum(diagonal ** 2)
```

The two comment lines (they start with `#` and are not executed) state in formulas what the line below each of them computes. `pair[x, a, b]` is $\sum_s \phi_a^*(xs)\,\phi_b(xs)$, and `exact` is the exact exchange $-\frac{1}{2}\sum_{a,b} w_{abba}$ with $w_{abba} = g_c \sum_x h\,|\sum_s \phi_a^*\phi_b|^2$, which is the integral of Section 13.4 for a label-independent contact (put $w = g_c\,\delta$ into w_{abba} and do the delta integral). `rho_local[x, s, t]` is the local 2×2 density matrix $\sum_a \phi_a(xs)\phi_a^*(xt)$, and `local_all` the local formula of Section 13.15 with all label pairs. `np.einsum("xss->xs", ...)` takes the diagonal of each 2×2 matrix (the label densities), and `local_diagonal` the local formula with the diagonal only.

```
say(f"exact {exact:.10f}; local, all label pairs {local_all:.10f}; "
    f"local, label diagonal only {local_diagonal:.10f}")
check(abs(exact - local_all) < 1e-10,
      "with label-mixing orbitals the exchange is local in the 2 x 2 matrix rho")
check(abs(exact - local_diagonal) > 0.1,
      "the label diagonal alone misses the label-mixing exchange")
```

The cell prints -8.7398756465, -8.7398756465 and -7.5021209026 and checks that the first two agree to 10^{-10} and that the third differs from them by more than 0.1 (it misses 1.2377547439, the part of the exchange carried by the off-diagonal entries of the local 2×2 matrix).

In [14], the two-point function and the local energies.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
image = left.imshow(rho_up ** 2, origin="lower", extent=(0, 1, 0, 1),
                    cmap="viridis")
left.plot([0, 1], [0, 1], "w--", lw=1.0)  # the diagonal x = x'
left.set_xlabel("$x$")
left.set_ylabel("$x'$")
left.set_title("$|\\rho_{up}(x, x')|^2$, 5 up fermions in a box")
fig.colorbar(image, ax=left, shrink=0.85)
```

On the left, `rho_up ** 2` (the square of every entry; the entries are real) is $|\rho_\uparrow(x, x')|^2$ of the box determinant, drawn as a heat map over the square $0 < x, x' < 1$; the style string (the letter w, for white, and two hyphens) draws the diagonal $x = x'$ as a white dashed line; axis labels, title and colour scale follow.

```
right.plot(x, 0.5 * G_C * (n_up + n_down) ** 2, label="Hartree $g_c n^2/2$")
right.plot(x, -0.5 * G_C * (n_up ** 2 + n_down ** 2),
           label="exchange $-g_c(n_{up}^2 + n_{down}^2)/2$")
right.plot(x, G_C * n_up * n_down, color="black", lw=2.0,
           label="sum $g_c\\,n_{up} n_{down}$")
```

On the right, along the box: the local Hartree energy density $\frac{g_c}{2}(n_\uparrow + n_\downarrow)^2$, the local exchange energy density $-\frac{g_c}{2}(n_\uparrow^2 + n_\downarrow^2)$, and (thick black) the product $g_c\, n_\uparrow n_\downarrow$, which Section 13.5 showed to be their sum.

```
right.set_ylim(-50.0, 60.0)  # room below the curves for the legend
right.set_xlabel("$x$")
right.set_ylabel("energy per length (units of $g_c$)")
right.legend(fontsize=7, loc="lower center")
save_figure(fig, "local_exchange",
```

`set_ylim(-50.0, 60.0)` fixes the vertical range so that the legend, placed at the bottom centre (`loc="lower center"`), does not cover the curves; labels and `save_figure` for Figure 13b.6. Then

```
check(np.allclose(0.5 * (n_up + n_down) ** 2 - 0.5 * (n_up ** 2 + n_down ** 2),
                  n_up * n_down, atol=1e-12),
      "Hartree plus exchange of a contact is g_c n_up n_down at every point")
```

confirms point by point that the Hartree plus the exchange energy density equals $g_c\, n_\uparrow n_\downarrow$ (here $g_c = 1$).

What Figure 13b.6 shows. On the left, $|\rho_\uparrow(x, x')|^2$ is large only near the diagonal, with five bright spots along it (the density of five fermions has five bumps) and faint ripples away from it: the density matrix of a determinant falls off with the distance $|x - x'|$, just as $F(k_F R)$ does in the uniform gas. A contact interaction samples only the diagonal. On the right, the Hartree energy density (up to about 52) and the exchange energy density (down to about -28) both follow the bumps of the densities, and their sum, $n_\uparrow n_\downarrow$, is everywhere smaller than the Hartree part: exchange removes, point by point, the energy of the same-label pairs.

In [15], polarisation.

```
C_F = 0.3 * (3.0 * np.pi ** 2) ** (2.0 / 3.0)
gamma_c = (2.0 / 3.0) * (3.0 * np.pi ** 2) ** (2.0 / 3.0)


def energy_polarized(zeta, coupling):
    """e(zeta) at n = 1 for g_c = coupling (exchange-only picture)."""
    kinetic_part = C_F * ((1 + zeta) ** (5 / 3) + (1 - zeta) ** (5 / 3)) / 2
    return kinetic_part + 0.25 * coupling * (1 - zeta ** 2)
```

The constants $C_F = \frac{3}{10}(3\pi^2)^{2/3}$ and $\gamma_c = \frac{2}{3}(3\pi^2)^{2/3}$, and the energy $e(\zeta)$ of Section 13.15 at $n = 1$ (so $g_c n^{1/3} = g_c$, the argument `coupling`). The exponent `5 / 3` is written with whole numbers; in Python the division `/` always gives a floating-point number, $1.6666\ldots$.

```
report("C_F", f"{C_F:.6f}")
report("threshold gamma_c = g_c n^(1/3)", f"{gamma_c:.6f}")
```

They print as $C_F = 2.871234$ and $\gamma_c = 6.380520$.

```
d = 1e-3
curvature = [(energy_polarized(d, c) - 2 * energy_polarized(0.0, c)
              + energy_polarized(-d, c)) / d ** 2
             for c in (0.99 * gamma_c, 1.01 * gamma_c)]
check(curvature[0] > 0 > curvature[1],
      "the unpolarized gas becomes unstable at g_c n^(1/3) = (2/3)(3 pi^2)^(2/3)")
```

The second difference $(e(d) - 2e(0) + e(-d))/d^2$ approximates $e''(0)$ (Section 13.2). Just below the threshold ($0.99\gamma_c$) it must be positive, just above ($1.01\gamma_c$) negative; `curvature[0] > 0 > curvature[1]` tests both at once.

```
zetas = np.linspace(-1.0, 1.0, 401)
fig, ax = plt.subplots()
for gamma in (0.5, 1.0, 1.1, 1.5):
    curve = energy_polarized(zetas, gamma * gamma_c) - energy_polarized(0.0,
                                                                        gamma * gamma_c)
    ax.plot(zetas, curve / C_F, label=f"$\\gamma = {gamma}$")
```

401 polarisations from -1 (all fermions down) to 1 (all up). For the couplings $\gamma = 0.5, 1, 1.1, 1.5$ in units of the threshold ($g_c = \gamma\,\gamma_c$), `curve` is $e(\zeta) - e(0)$ (the statement continues on the next line inside the open parenthesis), and the loop draws it divided by C_F, that is in units of $C_F n^{5/3}$ at $n = 1$.

```
ax.axhline(0.0, color="gray", lw=0.8)
ax.set_xlabel("polarization $\\zeta = (n_{up} - n_{down})/n$")
ax.set_ylabel("$(e(\\zeta) - e(0)) / (C_F n^{5/3})$")
ax.set_title("Exchange favours unequal labels at strong contact repulsion")
ax.legend()
save_figure(fig, "polarization",
```

The zero line, axis labels, title, legend and `save_figure` for Figure 13b.7.

What Figure 13b.7 shows. For $\gamma = 0.5$ the curve is a bowl with its lowest point at $\zeta = 0$: equal labels are stable. For $\gamma = 1$ it is extremely flat near $\zeta = 0$ (the second derivative vanishes there) and still rises towards $\zeta = \pm 1$. For $\gamma = 1.1$ the curve bends down at $\zeta = 0$, has shallow minima near $\zeta = \pm 0.9$ and comes back up slightly at $\zeta = \pm 1$. For $\gamma = 1.5$ it falls all the way to $\zeta = \pm 1$: in this exchange-only picture the gas would rather put every fermion into one label.

In [16], the gamma matrices, exactly.

```
import sympy as sp  # exact algebra with fractions and symbols

gamma_record = json.loads(repository_file("Revision/algebra/gammas.json")
                          .read_text(encoding="utf-8"))


def exact_matrix(rows):
    """A sympy matrix of exact fractions from the record's rows of numbers."""
    return sp.Matrix([[sp.Rational(str(entry)) for entry in row] for row in rows])
```

`sympy` computes with exact fractions and symbols. The Revision record `Revision/algebra/gammas.json` is read as text (`read_text`) and turned into Python lists and dictionaries by `json.loads`. `exact_matrix` turns a list of rows of numbers into a sympy matrix of exact fractions: `str(entry)` writes each number as text and `sp.Rational` reads that text as a fraction, so no rounding can occur.

```
gamma = [exact_matrix(rows) for rows in gamma_record["gamma"]]  # x0 .. x7
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g(x0) g(x1) g(x2) g(x3)
B = -sp.I * C * gamma[4]  # B = -i C g(x4)
B_record = (exact_matrix(gamma_record["B"]["re"])
            + sp.I * exact_matrix(gamma_record["B"]["im"]))
check(C == exact_matrix(gamma_record["C"]) and B == B_record,
      "C and B built from the gammas equal the matrices of the record")
check(C * C == sp.eye(16) and B * B == sp.eye(16) and C == C.T,
      "C is symmetric with C^2 = 1, and B^2 = 1")
```

The record lists the eight gammas in the order $x_0, \ldots, x_7$, so `gamma[k]` is $\gamma^{(xk)}$: `gamma[0]` is $\gamma^{(x0)}$ and `gamma[4]` is $\gamma^{(x4)}$; for sympy matrices `*` is the matrix product and `sp.I` is i. The first check

compares C and B with the matrices stored in the same record (B is stored as its real and imaginary parts); the second checks exactly the facts $C^2 = 1$, $B^2 = 1$ and C symmetric (`C.T` is the transpose) of Section 13.16.

In [17], the exchange of the uniform gas, exactly.

```
n_sym, S_sym, lam = sp.symbols("n S lambda", real=True)
rho = (n_sym * B + S_sym * C) / 16
check(sp.simplify((B * rho).trace() - n_sym) == 0
      and sp.simplify((C * rho).trace() - S_sym) == 0,
      "Tr(B rho) = n and Tr(C rho) = S")
e_H = sp.expand(lam / 2 * (C * rho).trace() ** 2)
e_x = sp.expand(-lam / 2 * (C * rho * C * rho).trace())
say(f"e_H = {e_H}")
say(f"e_x = {e_x}")
```

Three real symbols n, S, λ; the matrix $\rho = (nB + SC)/16$; the check that the densities come back ($\mathrm{Tr}\, B\rho = n$, $\mathrm{Tr}\, C\rho = S$; `.trace()` is the trace and `sp.simplify` brings the difference to its simplest form, which must be 0). `e_H` and `e_x` are the Hartree and exchange formulas of Section 13.16, multiplied out by `sp.expand`; they print as `S**2*lambda/2` and `-S**2*lambda/32 - lambda *n**2/32`.

```
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
gas = theory["exchange"]["uniformGas"]
coefficient_n2 = sp.Rational(gas["coefficient_n2"])  # "-1/32" in the record
coefficient_S2 = sp.Rational(gas["coefficient_S2"])
check(e_H == lam * S_sym ** 2 / 2, "the Hartree energy is (lambda/2) S^2")
recorded = lam * (coefficient_n2 * n_sym ** 2 + coefficient_S2 * S_sym ** 2)
check(sp.expand(e_x - recorded) == 0
      and coefficient_n2 == coefficient_S2 == sp.Rational(-1, 32),
      "e_x = -(lambda/32)(n^2 + S^2)",
      record="Revision/kohn_sham/reports/ks-theory-python.json, check "
             "exchange_uniform_gas")
```

The cell reads the Revision Kohn-Sham theory record and its two exchange coefficients, stored as the texts "-1/32", which `sp.Rational` reads as exact fractions. The checks: the Hartree energy is $\frac{\lambda}{2}S^2$, and the exchange energy computed here equals the recorded one, with both coefficients $-1/32$; the PASS line names the Revision check exchange_uniform_gas that it reproduces.

In [18], the potentials and the solver's numbers.

```
e_int = e_H + e_x
mass_coefficient = sp.simplify(sp.diff(e_int, S_sym) / (lam * S_sym))  # 15/16
vector_coefficient = sp.simplify(sp.diff(e_int, n_sym) / (lam * n_sym))  # -1/16
filled_ratio = sp.simplify((e_x / e_H).subs(n_sym, S_sym))  # n = S
```

`sp.diff(e_int, S_sym)` is $\partial e_{int}/\partial S$; divided by λS it is the coefficient $15/16$ of $M_{eff} - m$; in the same way $\partial e_{int}/\partial n$ divided by λn gives $-1/16$. `.subs(n_sym, S_sym)` replaces n by S in e_x/e_H: the ratio of one filled level at rest, $-1/8$.

```
say(f"M_eff = m + ({mass_coefficient}) lambda S;  v_v = ({vector_coefficient}) "
    f"lambda n;  E_x/E_H at n = S: {filled_ratio}")
```

The printed line shows all three exact fractions: M_eff = m + (15/16) lambda S; v_v = (-1/16) lambda n; E_x/E_H at n = S: -1/8.

```
potentials = theory["exchange"]["kohnShamPotentials"]
check(mass_coefficient == sp.Rational(potentials["Meff_coefficient_of_lambda_S"])
      == sp.Rational(15, 16)
      and vector_coefficient == sp.Rational(potentials["vv_coefficient_of_lambda_n"])
      == sp.Rational(-1, 16),
      "M_eff = m + (15/16) lambda S and v_v = -(1/16) lambda n",
      record="Revision/kohn_sham/reports/ks-theory-python.json, check ks_potentials")
check(filled_ratio == sp.Rational(-1, 8),
      "one filled 8-fold level at rest: E_x/E_H = -1/8",
      record="Revision/kohn_sham/reports/ks-theory-python.json, check "
             "filled_shell_ratio")
```

The coefficients are compared with the record's texts "15/16" and "-1/16" (a chain `a == b == c` means `a == b and b == c`), and the ratio with $-1/8$; the PASS lines name the Revision checks ks_potentials and filled_shell_ratio.

```
solver = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))["theoryInputs"]
check(solver["exchangeCoefficientN2"] == solver["exchangeCoefficientS2"] == -1 / 32
      and solver["MeffCoefficientOfLambdaS"] == 15 / 16
      and solver["vvCoefficientOfLambdaN"] == -1 / 16,
      "the Revision solver uses exactly these coefficients (parameters.json)")
```

The Revision solver's parameter file stores the coefficients it uses as floating-point numbers. The fractions $-1/32$, $15/16$ and $-1/16$ have denominators that are powers of 2, so they are stored exactly in binary, and the comparison with `==` is exact.

```
report_checks = {c["name"]: c["verdict"] for c in json.loads(repository_file(
    "Revision/kohn_sham/reports/ks-theory-python.json").read_text(
    encoding="utf-8"))["checks"]}
check(all(report_checks[name] == "PASS" for name in
          ("exchange_uniform_gas", "ks_potentials", "filled_shell_ratio")),
      "the three Revision checks are recorded as PASS")
```

The last check reads the Revision report itself, makes a dictionary from check name to verdict, and requires the three checks reproduced above to be recorded as PASS.

In [19], the matrices as pictures.

```
C_numbers = np.array(C.tolist(), dtype=float)
B_imaginary = np.array([[float(sp.im(entry)) for entry in row] for row in B.tolist()])
check(all(sp.re(entry) == 0 for entry in B), "B is purely imaginary")
check(all(np.count_nonzero(row) == 1 for row in np.vstack([C_numbers, B_imaginary])),
      "every row of C and of B has exactly one nonzero entry")
```

`C.tolist()` gives the rows of the sympy matrix, which numpy turns into floating-point numbers; `sp.im` takes the imaginary part of each entry of B. The checks: every real part of B is 0, and every row of C and of B (stacked into one table by `np.vstack`) has exactly one nonzero entry.

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 4.2))
for ax, matrix, title in ((axes[0], C_numbers, "$C$ (real)"),
                          (axes[1], B_imaginary, "imaginary part of $B$")):
    image = ax.imshow(matrix, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xlabel("column")
    ax.set_ylabel("row")
fig.colorbar(image, ax=axes, shrink=0.8)
save_figure(fig, "dirac_matrices",
```

The loop goes through two triples (panel, matrix, title): C in the left panel and the imaginary part of B in the right one, each drawn as a heat map on the fixed scale from -1 (blue) to $+1$ (red), with row 0 at the top; one colour scale for both, and `save_figure` for Figure 13b.8.

What Figure 13b.8 shows. Each matrix has exactly one coloured square in every row and every column: C and B only exchange and re-sign the 16 components. C pairs the components 0 to 3 with 4 to 7 (entries -1) and 8 to 11 with 12 to 15 (entries $+1$); it is symmetric, as its picture is unchanged by a reflection in the main diagonal. The imaginary part of B pairs the first eight components with the last eight, with signs ± 1; together with the factor i this makes B Hermitian.

In [20], the exchange of the 8-fold gas as a picture.

```
ratio_values = np.linspace(0.0, 1.0, 101)  # S / n
e_H_curve = 0.5 * ratio_values ** 2  # e_H / (lambda n^2)
e_x_curve = -(1.0 + ratio_values ** 2) / 32.0  # e_x / (lambda n^2)
```

Dividing e_H and e_x by λn^2 leaves functions of S/n alone: $\frac{1}{2}(S/n)^2$ and $-(1+(S/n)^2)/32$, on 101 values of S/n from 0 to 1.

```
fig, ax = plt.subplots()
ax.plot(ratio_values, e_H_curve, label="Hartree $e_H = \\lambda S^2/2$")
ax.plot(ratio_values, e_x_curve, label="exchange $e_x = -\\lambda(n^2+S^2)/32$")
ax.plot(ratio_values, e_H_curve + e_x_curve, color="black", lw=2.0,
        label="$e_{int} = e_H + e_x$")
ax.plot([1.0], [-1.0 / 16.0], "o", color="red",
        label="filled level at rest: $e_x/e_H = -1/8$")
```

The two curves and their sum (thick black), and one red dot at $S/n = 1$, $e_x/(\lambda n^2) = -2/32 = -1/16$: the filled level at rest.

```
ax.axhline(0.0, color="gray", lw=0.8)
ax.set_xlabel("$S/n$ (scalar density over number density)")
ax.set_ylabel("energy per volume / $(\\lambda n^2)$")
ax.set_title("Hartree and exchange of the uniform 8-fold dirac16complex gas")
ax.legend(fontsize=8)
save_figure(fig, "dirac_exchange",
```

The zero line, labels, title, legend and `save_figure` for Figure 13b.9 (the lines after it are the caption). Then

```
check(abs(e_x_curve[-1] / e_H_curve[-1] + 1.0 / 8.0) < 1e-15,
      "the plotted curves give e_x/e_H = -1/8 at S = n")
```

checks that the plotted curves give $e_x/e_H = -1/8$ at $S = n$ (the last entries, `[-1]`).

What Figure 13b.9 shows. The Hartree energy grows from 0 at $S = 0$ to $\frac{1}{2}$ at $S = n$. The exchange energy is never zero: it is $-1/32$ even at $S = 0$, because its n^2 term does not depend on S, and $-1/16$ at $S = n$ (the red dot). Their sum is slightly negative for small S/n (below $S/n = \sqrt{1/15} \approx 0.26$, where $\frac{15}{32}S^2 = \frac{1}{32}n^2$) and reaches $7/16$ at $S = n$ (Exercise 9).

In [21], what the record says about the exchange formulas.

```
def verdicts(path):
    """The verdict of every check of a Revision report, by the name of the check."""
    checks = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
    return {entry["name"]: entry["verdict"] for entry in checks}
```

A **report** of the Revision record is a JSON file with a list `"checks"`; each entry has a `"name"` and a `"verdict"` (PASS or FAIL). The function `verdicts(path)` reads the report at the repository path `path` (with `repository_file` and `json.loads`, as in In [16]) and returns a dictionary from each check name to its verdict, built by a dictionary comprehension `{key: value for entry in checks}`.

```
ALGEBRA = "Revision/algebra/reports/wolfram-algebra.json"
KS_WOLFRAM = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
KS_PYTHON = "Revision/kohn_sham/reports/ks-theory-python.json"
algebra = verdicts(ALGEBRA)
check(all(algebra[name] == "PASS"
          for name in ("reality", "Clifford_relation", "C_real_symmetric")),
      f"{ALGEBRA}: reality, Clifford_relation and C_real_symmetric are PASS")
```

The three names in capitals are the paths of three reports: the Wolfram verification of the algebra of the gamma matrices, and the Wolfram and the sympy verifications of the Kohn-Sham theory. The first check requires the algebra checks reality (the gammas are real), Clifford_relation ($\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}$) and C_real_symmetric to be PASS: the facts on which Section 13.16 builds. `all(...)` is true when the condition holds for every name in the tuple; the `f"..."` string puts the path into the name of the check, so the PASS line names the record file.

```
ks_wolfram, ks_python = verdicts(KS_WOLFRAM), verdicts(KS_PYTHON)
check(all(ks_wolfram[name] == "PASS" for name in
          ("exchange_uniform_gas", "ks_potentials", "exchange_slab_exact_fock")),
      f"{KS_WOLFRAM}: exchange_uniform_gas, ks_potentials and "
      "exchange_slab_exact_fock are PASS")
check(ks_python["exchange_slab_exact_fock"] == "PASS"
      and ks_python["hf_wick_contraction"] == "PASS",
      f"{KS_PYTHON}: exchange_slab_exact_fock and hf_wick_contraction are PASS")
```

The first line reads both Kohn-Sham reports (a comma on each side assigns two values at once). The Wolfram program, independent of the sympy one, must have the checks exchange_uniform_gas and ks_potentials (the formulas that In [17] and In [18] recomputed) and exchange_slab_exact_fock (the exact exchange of the non-uniform states, below) as PASS; the sympy program must have exchange_slab_exact_fock and hf_wick_contraction (the rule $\langle{:}S^2{:}\rangle = (\mathrm{Tr}\, V\rho)^2 - \mathrm{Tr}(V\rho V\rho)$ of Section 13.4) as PASS.

```
slab = theory["exchange"]["exactFockSlab"]  # the exact exchange of the KS states
correlation = theory["exchange"]["kohnShamPotentials"]["correlation"]
solver_file = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                         .read_text(encoding="utf-8"))
conventions = solver_file["conventions"]  # the conventions of the Revision solver
status_word = slab["status"].split(":")[0]  # the word before the first colon
say(f"exactFockSlab status: {status_word}; correlation: {correlation}")
```

`theory` is the theory file `Revision/kohn_sham/ks-theory.json`, read in In [17]. `slab` is its entry exactFockSlab, the exact Fock exchange of the Kohn-Sham states, and `correlation` the text of its entry correlation. `solver_file` is the whole parameter file of the Revision solver (In [18] kept only its part theoryInputs), and `conventions` its part conventions. `.split(":")[0]` cuts the status text at its colons and keeps the first piece. The cell prints exactFockSlab status: DIAGNOSTIC; correlation: none (Hartree plus exchange only).

```
check(slab["status"].startswith("DIAGNOSTIC")
      and "Delta E_x = +(lambda/32) int sqrt|g| Q^2" in slab["status"]
      and "+lambda Q^2/32" in conventions["exactExchange"]
      and correlation == "none (Hartree plus exchange only)",
      "ks-theory.json: the exact Fock exchange differs by +(lambda/32) Q^2 "
      "(DIAGNOSTIC; also in parameters.json); no correlation term")
```

`text.startswith(word)` is true when the text begins with the word, and `piece in text` when the piece occurs in it. The check requires four statements of the record that Section 13.16 quotes: the status of the exact exchange is DIAGNOSTIC; the theory file states that the uniform-gas exchange omits $+\frac{\lambda}{32}\int \sqrt{|g|}\, Q^2$; the solver's convention exactExchange states the same difference $+\lambda Q^2/32$; and the functional has no correlation term.

```
table = repository_file("Revision/kohn_sham/results/exx/exact-fock-variant.csv")
columns = table.read_text(encoding="utf-8").splitlines()[0].split(",")
check("deltaE_x_exact_fock_diag" in columns,
      "exact-fock-variant.csv has the column deltaE_x_exact_fock_diag")
```

The solver's table of the exact-exchange variant is a CSV file (comma-separated values: one line per row, the entries separated by commas). `.splitlines()[0]` is its first line, the header, and `.split(",")` cuts it into the column names; the check requires the column deltaE_x_exact_fock_diag, in which the solver reports the difference.

In [22], how the dirac16complex model must be read.

```
temperatures = solver_file["physics"]["temperatures"]
say(f"temperatures of the Revision solver: {temperatures}")
```

```
check(temperatures == [0.01, 0.02, 0.05]
      and "ensemble Delta-SCF" in conventions["deltaScf"],
      "parameters.json: T = 0.01, 0.02, 0.05 and the ensemble Delta-SCF (deltaScf)")
```

The solver's temperatures (in units of the mass m) are printed, temperatures of the Revision solver: [0.01, 0.02, 0.05], and compared with the list that Section 13.37 quotes (two lists are equal, `==`, when they have the same entries in the same order); the convention deltaScf must describe the ensemble Delta-SCF of Section 13.27.

```
filling = theory["thermodynamics"]["fillingConvention"]
check(filling.startswith("CONVENTION") and "its justification is OPEN" in filling
      and conventions["history"].startswith("PRESCRIBED BACKGROUND"),
      "the filling is a CONVENTION whose justification is OPEN (ks-theory.json); "
      "the history is a PRESCRIBED BACKGROUND (parameters.json)")
```

The theory file's fillingConvention must call the filling of the levels a CONVENTION whose justification is OPEN, and the solver's convention history must begin with PRESCRIBED BACKGROUND: the two status words of Section 13.37.

```
SOURCE = "Revision/field_equations_a4/reports/ks-source-conditions.json"
source = verdicts(SOURCE)
check(all(source[name] == "PASS" for name in
          ("ks_history_is_a_prescribed_background",
           "ks_profiles_violate_algebraic_condition", "ks_profiles_depend_on_x0")),
      f"{SOURCE}: ks_history_is_a_prescribed_background, "
      "ks_profiles_violate_algebraic_condition and ks_profiles_depend_on_x0 are PASS")
```

The report on the source conditions of the a_4 field equations must have the three checks named in Section 13.38 as PASS: the history is a prescribed background, the Kohn-Sham profiles violate the algebraic condition $p_3 + p_t = 2p_0$, and they depend on the hidden coordinate x_0.

```
T3_CHECKS = ("block_hamiltonian_map", "ode_map", "tip_condition_map",
             "brane_parities_exchanged", "orbital_densities", "mean_field_map",
             "energies_and_emt_profiles_equal", "exact_k0_spectra",
             "Gamma_is_the_block_map", "z2_mirror_copy_carries_minus_m_plus_lambda")
t3_python = verdicts("Revision/pairing/kohn_sham/reports/python-t3.json")
t3_wolfram = verdicts("Revision/pairing/kohn_sham/reports/wolfram-t3.json")
check(all(t3_python["T3." + name] == "PASS" for name in T3_CHECKS),
      "theorem T3: ten checks of python-t3.json (T3.<name>) are PASS")
check(all(t3_wolfram["T3_" + name] == "PASS" for name in T3_CHECKS),
      "theorem T3: the same ten checks of wolfram-t3.json (T3_<name>) are PASS")
```

`T3_CHECKS` lists ten checks of theorem T3 that are present in both of its reports: the maps of the block Hamiltonian, of its equation (ode_map), of the tip condition and of the brane parities, the orbital densities, the mean field, the equality of the energies and energy-momentum profiles, the exact spectra at zero momentum, the matrix Γ as the block map, and the mirror copy. The Python report names them with the prefix `T3.` and the Wolfram report with `T3_`; `+` joins two strings. Both checks require all ten to be PASS. The cell names these ten checks instead of requiring every check

of the two reports, because further checks may be added to the reports of T3; a renamed or failed check of the theorem itself stops the notebook.

In [23], the last check.

```
figure_names = ["density_matrix", "exchange_hole", "contact_energies",
                "finite_range", "contact_vs_coulomb", "local_exchange",
                "polarization", "dirac_matrices", "dirac_exchange"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13b_{k}_{name}.png").is_file()]
check(missing == [], "all nine figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13b_9_dirac_exchange.png").is_file(),
      "the figure file 13b_9_dirac_exchange.png exists")
all_checks_passed()
```

The same lines as In [19] of Notebook 13c (Section 13.14), with the nine figure names of this notebook. The last line prints ALL 44 CHECKS PASSED (notebook 13b): one check in In [2] and In [3] each, three in In [5], four in In [6], two in In [8], In [10], In [12] and In [13] each, one in In [11], In [14] and In [15] each, two in In [16], three in In [17], four in In [18], two in In [19], one in In [20], five in In [21], five in In [22] and two in In [23].

13.21 Solving the Kohn-Sham equations: iteration and mixing

The loop of Section 13.10 is a map from an input density (or potential) to an output density, $n_{out} = G[n_{in}]$, and the self-consistent solution is a **fixed point** of this map, $G[n] = n$. The simplest loop, "the output becomes the next input" (**plain iteration**), often fails. This section shows why with a model small enough to follow by hand, and how the remedies work.

The model. Two sites L and R with the site energies $-\Delta/2$ and $+\Delta/2$, the hopping t and the on-site repulsion U, and two electrons with opposite labels in the lowest orbital. In the mean field each electron feels U times the density of the OTHER label on its site (the contact rule of Section 13.5: exchange removes the same-label half); with two electrons in the same orbital each label has the density $n_L/2$ on L and $n_R/2$ on R. So the one-electron Hamiltonian and the output density are

$$h[n_L] = \begin{pmatrix} -\frac{\Delta}{2} + \frac{U}{2} n_L & -t \\ -t & \frac{\Delta}{2} + \frac{U}{2} n_R \end{pmatrix}, \qquad n_R = 2 - n_L, \qquad n_L^{out} = 2\,|c_L|^2,$$

where (c_L, c_R) is the normalised lowest eigenvector of $h[n_L]$.

The lowest eigenvector of a real symmetric 2×2 matrix, in four steps. Take $\begin{pmatrix} a & -t \\ -t & b \end{pmatrix}$ with $t > 0$, and write $\delta = b - a$ and $D = \sqrt{\delta^2 + 4t^2}$. (1) The eigenvalues solve $(a-\epsilon)(b-\epsilon) - t^2 = \epsilon^2 - (a+b)\epsilon + ab - t^2 = 0$, so $\epsilon = \frac{1}{2}(a+b) \pm \frac{1}{2}\sqrt{(a+b)^2 - 4ab + 4t^2} = \frac{1}{2}(a+b) \pm \frac{1}{2}D$, because $(a+b)^2 - 4ab = (b-a)^2$; the lowest is $\epsilon = \frac{1}{2}(a+b) - \frac{1}{2}D$. (2) The first row of $(h-\epsilon)c = 0$ reads $(a-\epsilon)c_L - t\,c_R = 0$; since $a - \epsilon = \frac{1}{2}(a-b) + \frac{1}{2}D = \frac{1}{2}(D-\delta)$, the ratio of the components is $r = c_R/c_L = (D-\delta)/(2t)$. (3) From $D^2 - \delta^2 = 4t^2$, that is $4t^2 = (D-\delta)(D+\delta)$, we get $r^2 = (D-\delta)^2/((D-\delta)(D+\delta)) = (D-\delta)/(D+\delta)$. (4) The normalisation $|c_L|^2(1+r^2) = 1$ gives

$$|c_L|^2 = \frac{1}{1+r^2} = \frac{D+\delta}{2D} = \frac{1}{2}\Big(1 + \frac{b-a}{\sqrt{(b-a)^2+4t^2}}\Big).$$

The reduced map. Here, line by line,

$$b - a = \Big(\tfrac{\Delta}{2} + \tfrac{U}{2}(2-n_L)\Big) - \Big(-\tfrac{\Delta}{2} + \tfrac{U}{2}n_L\Big) = \Delta + U(1-n_L),$$
$$n_L^{out} - 1 = 2|c_L|^2 - 1 = \frac{b-a}{\sqrt{(b-a)^2+4t^2}}.$$

The first line subtracts the two diagonal entries and collects; the second inserts the result of step (4). With $x = n_L - 1$ (the excess of charge on L) we have $b - a = \Delta - Ux$, and the whole loop becomes one function of one number:

$$x_{out} = G(x) = \frac{\Delta - Ux}{\sqrt{(\Delta - Ux)^2 + 4t^2}}.$$

Numbers. Take $\Delta = 2$, $t = 1$, $U = 4$ (energies in units of t) and the start $n_L = 2$ (both electrons on the low site). Plain iteration gives (Notebook 13d, In [5])

step	**0**	**1**	**2**	**3**	**4**	**5**
n_L in	2.000000	0.292893	1.923880	0.353344	1.916644	0.359836
n_L out	0.292893	1.923880	0.353344	1.916644	0.359836	1.915809

and continues to jump between about 0.3607 and 1.9157 forever: when the electrons sit on L, the repulsion pushes them to R, and when they sit on R, it pushes them back. This is called **charge sloshing**. **Linear mixing** feeds back only a fraction β of the change, $n_L \leftarrow (1-\beta)\, n_L + \beta\, n_L^{out}$ (so $\beta = 1$ is plain iteration). With $\beta = \frac{1}{2}$ (Notebook 13d, In [6]):

step	**0**	**1**	**2**	**3**	**4**	**5**
n_L in	2.000000	1.146447	1.361898	1.314066	1.331307	1.325494
n_L out	0.292893	1.577350	1.266234	1.348548	1.319682	1.329519

The iteration converges to the self-consistent value $n_L^* = 1.326993$; input and output agree to 10^{-6} from step 13 on.

Why, line by line. Near the fixed point x^* write $x = x^* + e$ with a small error e. To first order (the tangent line), $G(x^* + e) = x^* + G'(x^*)\, e$. One step of linear mixing then gives

$$x^* + e_{new} = (1-\beta)(x^* + e) + \beta\,(x^* + G'(x^*)\, e) \quad \Longrightarrow \quad e_{new} = \big[1 - \beta\,(1 - G'(x^*))\big]\, e,$$

where the left equation is the mixing rule with the tangent line inserted, and the right one subtracts $x^* = (1-\beta)x^* + \beta x^*$ from both sides and collects the terms with e. So every step multiplies the error by the **convergence factor** $1 - \beta(1 - G')$, and the loop converges (for small errors) exactly when this number lies between -1 and 1. The slope follows from the chain rule: with $u = \Delta - Ux$,

$G = u\,(u^2+4t^2)^{-1/2}$ has $dG/du = (u^2+4t^2)^{-1/2} - u^2(u^2+4t^2)^{-3/2} = 4t^2(u^2+4t^2)^{-3/2}$, and $du/dx = -U$, so

$$G'(x) = -\frac{4Ut^2}{\left((\Delta - Ux)^2 + 4t^2\right)^{3/2}}.$$

Since $G' < 0$, the number $1 - G'$ is larger than 1, and the condition $-1 < 1 - \beta(1-G') < 1$ reads $0 < \beta(1-G') < 2$, that is

$$0 < \beta < \beta_{max} = \frac{2}{1 - G'(x^*)}.$$

With the numbers above, $x^* = 0.326993$ and $G'(x^*) = -1.687961$ (found by bisection in Notebook 13d, In [3]), so $\beta_{max} = 0.744058$. Plain iteration ($\beta = 1$) multiplies the error by $1 - 2.687961 = -1.688$: the error grows and changes its sign at every step, which is the sloshing, until the curvature of G (left out by the tangent line) stops the growth on a cycle of period 2. With $\beta = \frac{1}{2}$ the factor is -0.344, and the error shrinks by about a factor 3 per step, as the table shows. The choice $\beta = 1/(1-G') = 0.372029$ makes the factor zero. The stronger the repulsion, the steeper G and the smaller the step the loop may take: for $U = 2$, $G' = -0.688942$ and $\beta_{max} = 1.184174$, so even plain iteration converges, and plain iteration stops converging at the repulsion where $G'(x^*) = -1$, which Notebook 13d (In [12]) finds to be $U = 2.647393$ for $\Delta = 2$, $t = 1$. All these numbers are COMPUTED by Notebook 13d; the linear analysis that explains them is PROVED above for small errors.

Anderson mixing. For densities with many components the best β differs from direction to direction, and it is not known in advance. **Anderson mixing** remembers the last few inputs $w^{(i)}$ and their residuals $R^{(i)} = G[w^{(i)}] - w^{(i)}$, finds numbers c_i with $\sum_i c_i = 1$ that make the combined residual $\sum_i c_i R^{(i)}$ as small as possible, and takes as the next input $\sum_i c_i\,(w^{(i)} + \beta R^{(i)})$. The condition $\sum_i c_i = 1$ is built in by writing $c_i = \theta_i$ for the older passes and $c_{last} = 1 - \sum_i \theta_i$; then

$$\sum_i c_i R^{(i)} = R^{(last)} + \sum_i \theta_i\,(R^{(i)} - R^{(last)}),$$

which is smallest (in the sense of the sum of the squares of its entries) for the **least-squares** solution θ of $\sum_i \theta_i (R^{(i)} - R^{(last)}) \approx -R^{(last)}$, a standard problem of linear algebra that numpy's `lstsq` solves. For one variable and two remembered passes the combined residual can be made exactly zero, and the next input is the point where the straight line through the two last (input, residual) pairs crosses zero: the **secant method**, which estimates the slope of G from the history. Anderson mixing is therefore a cheap substitute for Newton's method, and it needs no tuning of β.

The Revision solver. The Revision Kohn-Sham solver of the dirac16complex field (Chapter 15) uses Anderson mixing with these settings, recorded in `Revision/kohn_sham/results/parameters.json` (numerics): it remembers 6 passes (andersonDepth), uses $\beta = 0.4$ (andersonBeta), and stops when the largest difference between the potentials that come out and those that went in is at most 10^{-11} (scfTolerance), with at most 400 passes. What it mixes is the list of its potentials at every grid point of the hidden direction (the mass shift $M_{eff} - m$, the potential v_v, and a third potential used only by one variant). It solves the same least-squares problem in an equivalent form (with a Lagrange multiplier for $\sum_i c_i = 1$) and adds two safeguards: a tiny number (10^{-12} times the

largest diagonal entry) on the diagonal of its equations, which keeps them solvable, and a restart of the remembered passes whenever the residual grows to more than ten times the best one so far (`Revision/kohn_sham/solver/src/scf.rs`). Notebooks 13d and 13a read these settings from the parameter file and use them in their own small loops.

Level crossings and smearing. With whole-number occupations (aufbau), the output density jumps whenever two levels near the highest occupied one exchange their order during the iteration; if they cross back and forth, the loop never settles, whatever β is. The remedy replaces the whole-number occupations by smooth Fermi-Dirac occupations (Section 13.26); the converged object is then an ensemble with fractional occupations near the highest level, not a single determinant.

13.22 Example: charge sloshing

Notebook 13d computes everything of Section 13.21 for the two-site model: it checks the reduced map against a direct diagonalisation, reproduces the two tables, finds the fixed point, the slope and the largest mixing parameter, compares the error histories of plain iteration, linear mixing and Anderson mixing with the Revision solver's settings (read from the parameter file), maps the long-run behaviour against β and the threshold against U. Its numbers are COMPUTED and exact up to rounding; the model shows the mechanism and is not a physical system of the book. It ends with ALL 17 CHECKS PASSED (notebook 13d) and draws six figures.

13.23 How to run Notebook 13d

Step 1. What this notebook does and what it needs.

Notebook 13d (Charge sloshing and mixing in the two-site model) is the file `Revision/textbook/notebooks/13d_charge_sloshing.ipynb` of the repository Dirac_claude. It runs the self-consistency loop of a two-site mean-field model with two electrons (site energies -1 and +1, hopping 1, repulsion 4) by diagonalising its 2 x 2 Hamiltonian, checks the reduced map x_out = G(x), reproduces the tables of plain iteration (charge sloshing between the sites) and of linear mixing, finds the fixed point and the convergence condition of linear mixing 0 < beta < 2/(1 - G'), compares plain iteration, linear mixing and Anderson mixing (with the settings of the Revision Kohn-Sham solver, read from its parameter file), maps the long-run behaviour against beta and the threshold against the repulsion, and draws six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its

first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook,

skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13d_charge_sloshing.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13d_charge_sloshing.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 13d_charge_sloshing.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/13d.captions.json`
- `Revision/textbook/figures/13d_1_reduced_map.png`
- `Revision/textbook/figures/13d_2_cobwebs.png`
- `Revision/textbook/figures/13d_3_error_histories.png`

- `Revision/textbook/figures/13d_4_convergence_factor.png`
- `Revision/textbook/figures/13d_5_long_run_diagram.png`
- `Revision/textbook/figures/13d_6_threshold_versus_u.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13d_6_threshold_versus_u.png exists
ALL 17 CHECKS PASSED (notebook 13d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/13d_charge_sloshing.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 13d_charge_sloshing.ipynb
python -m nbconvert --execute --inplace 13d_charge_sloshing.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `parameters.json`: the notebook reads the file `Revision/kohn_sham/results/parameters.json` of the repository; it must be opened inside the folder `Revision/textbook/notebooks` of a complete clone of the repository, not as a single downloaded file.

13.24 Notebook 13d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/13d_charge_sloshing.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The self-consistency loop of density functional theory is a map from an input
density to an output density, and a solution is a fixed point of this map. The
simple loop "output becomes the next input" often fails. This notebook shows why,
with a model small enough to understand completely: two electrons on two sites. It

- runs the loop by diagonalising the $2 \times 2$ mean-field Hamiltonian and checks
  that it equals the reduced map $x_{out} = G(x)$ of one number;
- reproduces the tables of plain iteration (the density jumps between the two sites
  forever: charge sloshing) and of linear mixing with $\beta = 1/2$ (it converges);
- finds the fixed point by bisection, the slope $G'$ there, and the condition
  $0 < \beta < 2/(1 - G')$ under which linear mixing converges;
```

```
- compares the error histories of plain iteration, several linear mixings and
  Anderson mixing;
- maps the long-run behaviour of the loop against $\beta$ and the threshold against
  the repulsion $U$;
- draws six teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 13d (Charge sloshing and mixing in the two-site model) is the file
‘Revision/textbook/notebooks/13d_charge_sloshing.ipynb‘ of the repository Dirac_claude.
It runs the self-consistency loop of a two-site mean-field model with two electrons (site
energies -1 and +1, hopping 1, repulsion 4) by diagonalising its 2 x 2 Hamiltonian,
checks the reduced map x_out = G(x), reproduces the tables of plain iteration (charge
sloshing between the sites) and of linear mixing, finds the fixed point and the
convergence condition of linear mixing 0 < beta < 2/(1 - G’), compares plain iteration,
linear mixing and Anderson mixing (with the settings of the Revision Kohn-Sham solver,
read from its parameter file), maps the long-run behaviour against beta and the threshold
against the repulsion, and draws six teaching plots. It needs a computer with Windows 11,
macOS or Linux, an internet connection for the installation, the program Git, and Python
3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at
exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter,
the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):
```

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13d_charge_sloshing.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its

left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13d_charge_sloshing.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 13d_charge_sloshing.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/13d.captions.json‘
- ‘Revision/textbook/figures/13d_1_reduced_map.png‘
- ‘Revision/textbook/figures/13d_2_cobwebs.png‘
- ‘Revision/textbook/figures/13d_3_error_histories.png‘
- ‘Revision/textbook/figures/13d_4_convergence_factor.png‘
- ‘Revision/textbook/figures/13d_5_long_run_diagram.png‘
- ‘Revision/textbook/figures/13d_6_threshold_versus_u.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13d_6_threshold_versus_u.png exists
ALL 17 CHECKS PASSED (notebook 13d)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/13d_charge_sloshing.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 13d_charge_sloshing.ipynb
      python -m nbconvert --execute --inplace 13d_charge_sloshing.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘parameters.json‘: the notebook reads the file ‘Revision/kohn_sham/results/parameters.json‘ of the repository; it must be opened inside the folder ‘Revision/textbook/notebooks‘ of a complete clone of the repository, not as a single downloaded file.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 13d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 13d (Charge sloshing and mixing in the two-site model) is the file
# Revision/textbook/notebooks/13d_charge_sloshing.ipynb of the repository Dirac_claude.
# It runs the self-consistency loop of a two-site mean-field model with two electrons
# (site energies -1 and +1, hopping 1, repulsion 4) by diagonalising its 2 x 2
# Hamiltonian, checks the reduced map x_out = G(x), reproduces the tables of plain
# iteration (charge sloshing between the sites) and of linear mixing, finds the fixed
# point and the convergence condition of linear mixing 0 < beta < 2/(1 - G'), compares
# plain iteration, linear mixing and Anderson mixing (with the settings of the Revision
# Kohn-Sham solver, read from its parameter file), maps the long-run behaviour against
# beta and the threshold against the repulsion, and draws six teaching plots. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
```

```
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
```

```
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 13d_charge_sloshing.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 13d_charge_sloshing.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
```

```
# 13d_charge_sloshing.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/13d.captions.json
# - Revision/textbook/figures/13d_1_reduced_map.png
# - Revision/textbook/figures/13d_2_cobwebs.png
# - Revision/textbook/figures/13d_3_error_histories.png
# - Revision/textbook/figures/13d_4_convergence_factor.png
# - Revision/textbook/figures/13d_5_long_run_diagram.png
# - Revision/textbook/figures/13d_6_threshold_versus_u.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 13d_6_threshold_versus_u.png exists
#     ALL 17 CHECKS PASSED (notebook 13d)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/13d_charge_sloshing.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
```

```
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 13d_charge_sloshing.ipynb
#     python -m nbconvert --execute --inplace 13d_charge_sloshing.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for parameters.json: the notebook reads the file
#   Revision/kohn_sham/results/parameters.json of the repository; it must be opened
#   inside the folder Revision/textbook/notebooks of a complete clone of the repository,
#   not as a single downloaded file.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "13d"  # this notebook: chapter 13, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
```

```
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
```

```
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 13d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Two-site model**: two places L and R with one orbital each; an electron can hop
  between them (hopping $t$); the site energies are $-\Delta/2$ (L) and $+\Delta/2$
  (R).
- **Mean field**: each electron feels the other one only through its average
  density; here an electron on a site feels $U$ times the density of the other label
  on that site, i.e. $U n_L/2$ on L and $U n_R/2$ on R.
- **Density** $n_L$, $n_R = 2 - n_L$: the expected number of electrons on each site.
- **Fixed point** of a map $G$: a number $x^*$ with $G(x^*) = x^*$.
- **Plain iteration**: $x \leftarrow G(x)$; **linear mixing**:
  $x \leftarrow (1 - \beta)\,x + \beta\,G(x)$ with the mixing parameter $\beta$.
```

```
- **Charge sloshing**: the loop moves the charge back and forth between regions
  instead of settling.
- **Error**: the distance $x - x^*$ from the fixed point; **convergence factor**: the
  number by which one step multiplies the error near the fixed point.
- **Cobweb diagram**: a picture of an iteration: go vertically to the curve $G$,
  horizontally to the diagonal, vertically again, and so on.
- **Anderson mixing**: a mixing that uses several earlier steps to estimate the
  slope of $G$ (for one variable it is close to the secant method).
- **Bisection**: finding a root by halving an interval that contains it.
```

Text cell:

```
## 4. The physical and mathematical situation

Two electrons with opposite labels occupy the lowest orbital of the mean-field
Hamiltonian

$$h[n_L] = \begin{pmatrix} -\Delta/2 + U n_L/2 & -t \\ -t & \Delta/2 + U n_R/2
\end{pmatrix}, \qquad n_R = 2 - n_L,$$

so the output density is $n_L^{out} = 2|c_L|^2$, where $(c_L, c_R)$ is the
normalised lowest eigenvector. For a real symmetric matrix
$\begin{pmatrix} a & -t\\ -t & b\end{pmatrix}$ the lowest eigenvector has
$|c_L|^2 = \tfrac12\big(1 + (b - a)/\sqrt{(b - a)^2 + 4t^2}\big)$, and here
$b - a = \Delta + U(1 - n_L)$. With $x = n_L - 1$ (the excess on L) the whole loop is
one function of one number:

$$x_{out} = G(x) = \frac{\Delta - U x}{\sqrt{(\Delta - Ux)^2 + 4t^2}} .$$

Near the fixed point $x^*$, $G(x^* + e) \approx x^* + G'(x^*)\,e$, so linear mixing
multiplies the error by $1 - \beta(1 - G'(x^*))$ at every step, and it converges
exactly when this number lies between $-1$ and $1$, i.e. for
$0 < \beta < 2/(1 - G'(x^*))$. The slope is
$G'(x) = -4Ut^2/((\Delta - Ux)^2 + 4t^2)^{3/2}$.

**Numbers.** $\Delta = 2$, $t = 1$, $U = 4$ (energies in units of $t$), start
$n_L = 2$ (both electrons on the low site). **Status:** exact mathematics of a model
loop (COMPUTED here); the model shows the mechanism, not a physical system of the
book. Its only link to the Revision record is the Anderson-mixing settings, which
section 8 reads from the parameter file of the Revision Kohn-Sham solver.
```

Text cell:

```
## 5. The loop and the reduced map

The next cell defines one pass of the loop by diagonalising $h[n_L]$ with numpy, and
the closed-form map $G$, and checks that they agree for 41 inputs between 0 and 2.
```

In [2]:

```
import numpy as np  # arrays, matrices and linear algebra
```

```
DELTA, T_HOP, U = 2.0, 1.0, 4.0  # site-energy difference, hopping, repulsion


def loop_pass(n_left, delta=DELTA, t=T_HOP, u=U):
    """One pass: the output density n_L of the lowest orbital of h[n_L]."""
    n_right = 2.0 - n_left
    h = np.array([[-delta / 2 + u * n_left / 2, -t],
                  [-t, delta / 2 + u * n_right / 2]])
    values, vectors = np.linalg.eigh(h)  # levels in increasing order
    return 2.0 * vectors[0, 0] ** 2  # two electrons, |c_L|^2 each


def G(x, delta=DELTA, t=T_HOP, u=U):
    """The reduced map x_out = G(x), x = n_L - 1."""
    shift = delta - u * x
    return shift / np.sqrt(shift ** 2 + 4.0 * t * t)


inputs = np.linspace(0.0, 2.0, 41)
worst = max(abs(loop_pass(n) - 1.0 - G(n - 1.0)) for n in inputs)
check(worst < 1e-12, "the 2 x 2 diagonalisation and the reduced map G agree")
```

Out [2]:

```
PASS the 2 x 2 diagonalisation and the reduced map G agree
```

Text cell:

```
The next cell finds the fixed point $x^*$ by bisection on $x - G(x)$, which increases
with $x$ (because $G$ decreases), computes the slope $G'(x^*)$ from the formula and
from a difference quotient, and the largest and the best mixing parameters,
$\beta_{max} = 2/(1 - G')$ and $\beta_{best} = 1/(1 - G')$ (which makes the factor
zero).
```

In [3]:

```
def fixed_point(delta=DELTA, t=T_HOP, u=U):
    """x* with G(x*) = x*, by bisection on [-1, 1] (x - G(x) increases)."""
    low, high = -1.0, 1.0
    for _ in range(80):
        middle = 0.5 * (low + high)
        if middle - G(middle, delta, t, u) > 0.0:
            high = middle
        else:
            low = middle
    return 0.5 * (low + high)


def slope(x, delta=DELTA, t=T_HOP, u=U):
    """G'(x) = -4 U t^2 / ((Delta - U x)^2 + 4 t^2)^(3/2)."""
    return -4.0 * u * t * t / ((delta - u * x) ** 2 + 4.0 * t * t) ** 1.5
```

```
x_star = fixed_point()
g_prime = slope(x_star)
quotient = (G(x_star + 1e-6) - G(x_star - 1e-6)) / 2e-6
beta_max, beta_best = 2.0 / (1.0 - g_prime), 1.0 / (1.0 - g_prime)
report("fixed point n_L* = 1 + x*", f"{1.0 + x_star:.6f}")
report("slope G'(x*)", f"{g_prime:.6f}")
report("largest mixing parameter 2/(1 - G')", f"{beta_max:.6f}")
report("best mixing parameter 1/(1 - G')", f"{beta_best:.6f}")
check(abs(x_star - 0.326993) < 1e-6 and abs(g_prime + 1.687961) < 1e-6
      and abs(beta_max - 0.744058) < 1e-6,
      "x* = 0.326993, G'(x*) = -1.687961, beta_max = 0.744058")
check(abs(quotient - g_prime) < 1e-6, "the slope formula equals a difference quotient")
```

Out [3]:

```
RESULT fixed point n_L* = 1 + x* = 1.326993
RESULT slope G'(x*) = -1.687961
RESULT largest mixing parameter 2/(1 - G') = 0.744058
RESULT best mixing parameter 1/(1 - G') = 0.372029
PASS x* = 0.326993, G'(x*) = -1.687961, beta_max = 0.744058
PASS the slope formula equals a difference quotient
```

Text cell:

```
The next cell draws the output density against the input density, the diagonal
$n^{out} = n^{in}$ (where the fixed point lies) and the fixed point.
```

In [4]:

```
n_in = np.linspace(0.0, 2.0, 401)
fig, ax = plt.subplots(figsize=(6.0, 5.0))
ax.plot(n_in, 1.0 + G(n_in - 1.0), color="black", lw=2.0,
        label="$n_L^{out} = 1 + G(n_L - 1)$")
ax.plot(n_in, n_in, "--", color="gray", label="diagonal $n_L^{out} = n_L$")
ax.plot([1.0 + x_star], [1.0 + x_star], "ro",
        label=f"fixed point $n_L^* = {1.0 + x_star:.6f}$")
ax.set_xlabel("input density on the left site $n_L$")
ax.set_ylabel("output density $n_L^{out}$")
ax.set_title("The self-consistency map of the two-site model")
ax.legend(fontsize=8)
save_figure(fig, "reduced_map",
            "The output density on the left site produced by one pass of the "
            "mean-field loop, against the input density (both in electrons, from 0 "
            "to 2), for $\\Delta = 2$, $t = 1$, $U = 4$; the dashed diagonal marks "
            "equal input and output, and the red point is the self-consistent "
            "solution $n_L^\\ast = 1.326993$. The curve falls steeply through the "
            "diagonal (slope $-1.69$): more charge on the left pushes even more "
            "charge to the right.")
```

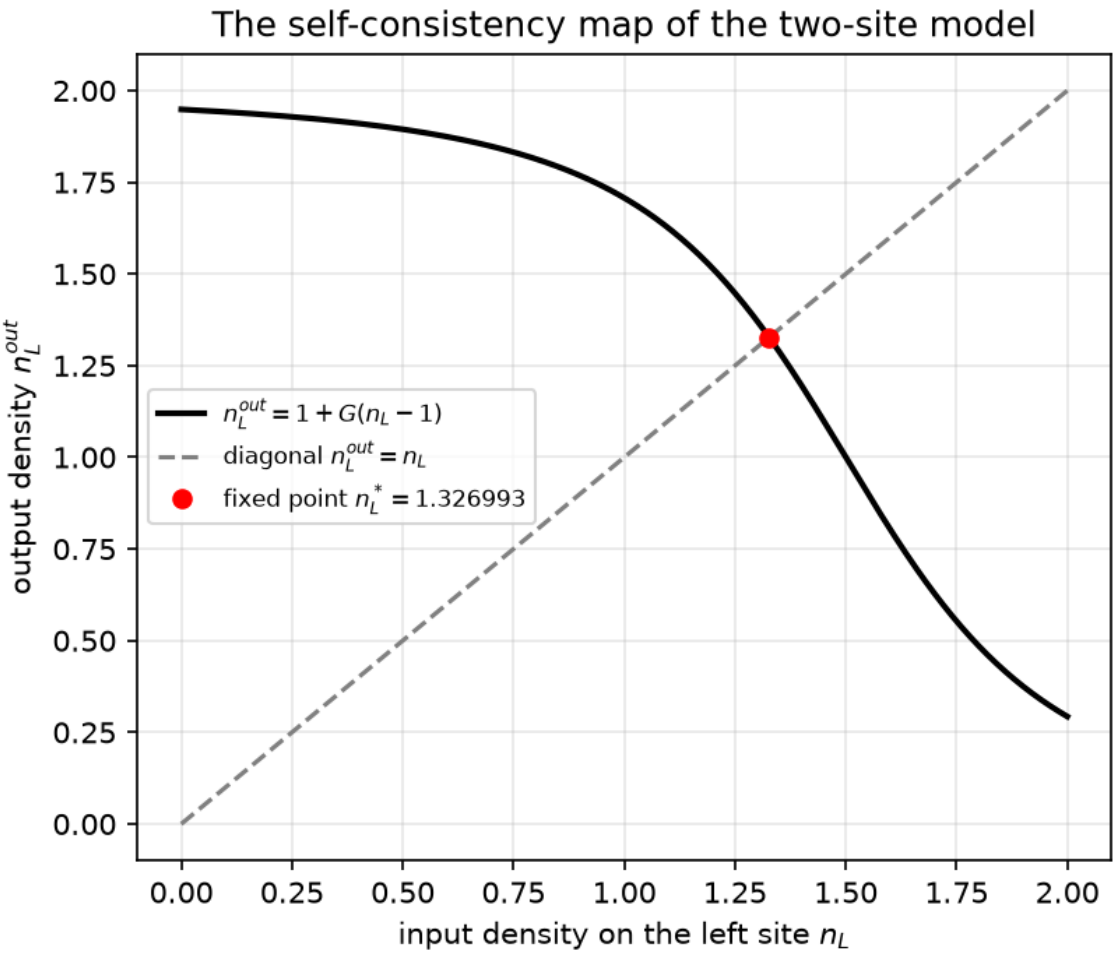

Figure 361: The output density on the left site produced by one pass of the mean-field loop, against the input density (both in electrons, from 0 to 2), for $\Delta = 2$, $t = 1$, $U = 4$; the dashed diagonal marks equal input and output, and the red point is the self-consistent solution $n_L^* = 1.326993$. The curve falls steeply through the diagonal (slope -1.69): more charge on the left pushes even more charge to the right. (Notebook 13d, figure 1.)

Out [4]:

```
Figure 13d.1 saved as Revision/textbook/figures/13d_1_reduced_map.png
```

Text cell:

```
## 6. Plain iteration: charge sloshing

The next cell runs plain iteration ($n_L \leftarrow n_L^{out}$) from $n_L = 2$, prints
the first six passes, and then runs 2000 more passes: the input jumps between two
values forever (a cycle of period 2), never reaching the fixed point.
```

In [5]:

```
def iterate(beta, start=2.0, passes=6):
    """Inputs and outputs of the loop with linear mixing (beta = 1: plain)."""
    n, history = start, []
    for _ in range(passes):
        out = loop_pass(n)
        history.append((n, out))
        n = (1.0 - beta) * n + beta * out
    return history, n


plain, _ = iterate(1.0)
say("step   n_L in     n_L out")
for step, (n, out) in enumerate(plain):
    say(f"{step:4d}   {n:.6f}   {out:.6f}")
expected_in = [2.000000, 0.292893, 1.923880, 0.353344, 1.916644, 0.359836]
check(all(abs(n - e) < 1e-6 for (n, _), e in zip(plain, expected_in)),
      "plain iteration reproduces the table 2.000000, 0.292893, 1.923880, ...")
_, late = iterate(1.0, passes=2000)
cycle = sorted([late, loop_pass(late)])
say(f"after 2000 passes the input alternates between {cycle[0]:.4f} and "
    f"{cycle[1]:.4f}")
check(abs(loop_pass(loop_pass(late)) - late) < 1e-10 and cycle[1] - cycle[0] > 1.5,
      "plain iteration ends in a cycle of period 2 (about 0.3607 and 1.9157)")
```

Out [5]:

```
step   n_L in     n_L out
   0   2.000000   0.292893
   1   0.292893   1.923880
   2   1.923880   0.353344
   3   0.353344   1.916644
   4   1.916644   0.359836
   5   0.359836   1.915809
PASS plain iteration reproduces the table 2.000000, 0.292893, 1.923880, ...
after 2000 passes the input alternates between 0.3607 and 1.9157
PASS plain iteration ends in a cycle of period 2 (about 0.3607 and 1.9157)
```

Text cell:

```
## 7. Linear mixing with beta = 1/2

The next cell prints the first six passes with $\beta = 1/2$ and counts the passes
until the input and the output agree to $10^{-6}$.
```

In [6]:

```
mixed, _ = iterate(0.5)
say("step   n_L in     n_L out")
for step, (n, out) in enumerate(mixed):
    say(f"{step:4d}   {n:.6f}   {out:.6f}")
expected_in = [2.000000, 1.146447, 1.361898, 1.314066, 1.331307, 1.325494]
check(all(abs(n - e) < 1e-6 for (n, _), e in zip(mixed, expected_in)),
      "linear mixing with beta = 1/2 reproduces the table 2.000000, 1.146447, ...")
long_history, _ = iterate(0.5, passes=40)
agree = next(step for step, (n, out) in enumerate(long_history)
             if abs(n - out) < 1e-6)
report("passes until input and output agree to 1e-6 (beta = 1/2)", agree)
check(agree == 13 and abs(long_history[-1][0] - 1.326993) < 1e-6,
      "beta = 1/2 converges to 1.326993; input and output agree after 13 passes")
```

Out [6]:

```
step   n_L in     n_L out
   0   2.000000   0.292893
   1   1.146447   1.577350
   2   1.361898   1.266234
   3   1.314066   1.348548
   4   1.331307   1.319682
   5   1.325494   1.329519
PASS linear mixing with beta = 1/2 reproduces the table 2.000000, 1.146447, ...
RESULT passes until input and output agree to 1e-6 (beta = 1/2) = 13
PASS beta = 1/2 converges to 1.326993; input and output agree after 13 passes
```

Text cell:

```
The next cell draws the two iterations as cobweb diagrams: from an input on the
horizontal axis go up to the map curve (the output), then across to the mixed next
input (for plain iteration: to the diagonal), and repeat.
```

In [7]:

```
def cobweb(ax, beta, passes):
    """Draw the cobweb of the loop with mixing beta on the axes ax."""
    ax.plot(n_in, 1.0 + G(n_in - 1.0), color="black", lw=1.5)
    ax.plot(n_in, n_in, "--", color="gray", lw=0.8)
    history, _ = iterate(beta, passes=passes)
```

```
    for n, out in history:
        n_next = (1.0 - beta) * n + beta * out
        ax.plot([n, n], [n, out], color="tab:red", lw=0.8)  # up to the curve
        ax.plot([n, n_next], [out, n_next], color="tab:blue", lw=0.8)  # across
    ax.plot([1.0 + x_star], [1.0 + x_star], "ko")
    ax.set_xlabel("input $n_L$")
    ax.set_ylabel("output $n_L^{out}$ and next input")


fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.5))
cobweb(left, 1.0, 12)
left.set_title("plain iteration ($\\beta = 1$): sloshing")
cobweb(right, 0.5, 12)
right.set_title("linear mixing ($\\beta = 1/2$): converges")
save_figure(fig, "cobwebs",
            "Cobweb diagrams of the first 12 passes of the two-site loop, starting "
            "at $n_L = 2$: the black curve is the map (output against input), the "
            "dashed line the diagonal, red segments go from an input to its output, "
            "blue segments from the output to the next input. Left, plain "
            "iteration: the path settles on a square around the fixed point (black "
            "dot), a cycle of period 2. Right, linear mixing with $\\beta = 1/2$: "
            "the path spirals into the fixed point.")
```

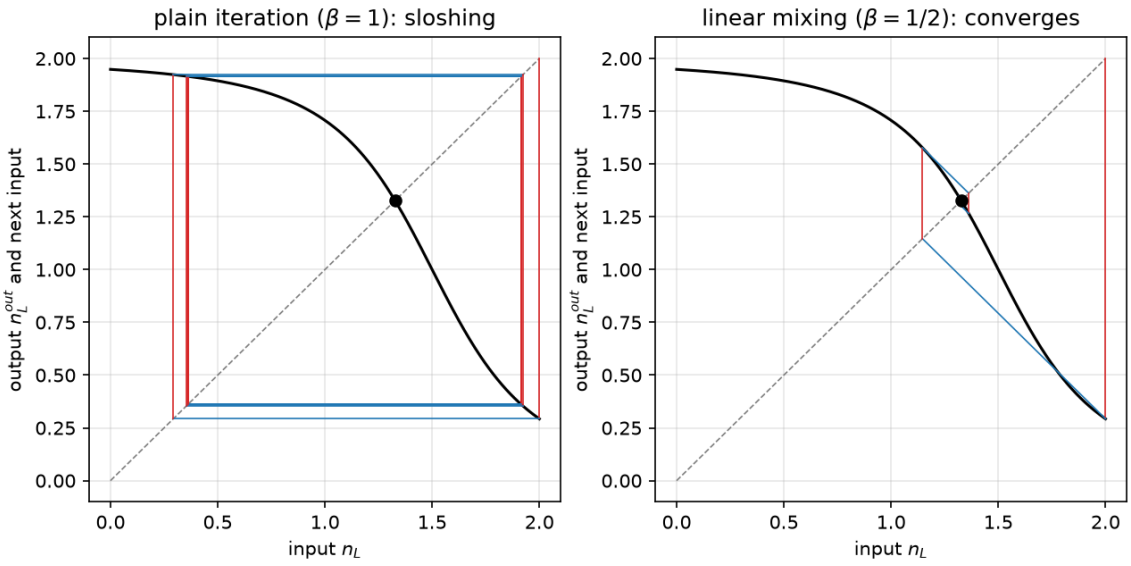

Figure 362: Cobweb diagrams of the first 12 passes of the two-site loop, starting at $n_L = 2$: the black curve is the map (output against input), the dashed line the diagonal, red segments go from an input to its output, blue segments from the output to the next input. Left, plain iteration: the path settles on a square around the fixed point (black dot), a cycle of period 2. Right, linear mixing with $\beta = 1/2$: the path spirals into the fixed point. (Notebook 13d, figure 2.)

Out [7]:

```
Figure 13d.2 saved as Revision/textbook/figures/13d_2_cobwebs.png
```

Text cell:

```
## 8. Error histories and the convergence factor

The next cell runs the loop for $\beta = 1, 0.8, 0.5, 0.2$ and for the best value
$\beta_{best} = 0.372$, and also with Anderson mixing with the settings of the
Revision Kohn-Sham solver of the dirac16complex field, which it reads from the
solver's parameter file (how many earlier passes are remembered, and $\beta$; for
one variable Anderson mixing estimates the slope of $G$ from the last passes, like
the secant method). It records the error $|n_L - n_L^*|$ of every input and checks
the measured ratio of successive errors for $\beta = 1/2$ against the predicted
factor $1 - \beta(1 - G') = -0.344$.
```

In [8]:

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
DEPTH = int(parameters["numerics"]["andersonDepth"])  # passes remembered
BETA_ANDERSON = float(parameters["numerics"]["andersonBeta"])  # mixing parameter
check(DEPTH == 6 and BETA_ANDERSON == 0.4,
      "the Revision solver mixes with depth 6 and beta 0.4 "
      "(Revision/kohn_sham/results/parameters.json, numerics)")


def errors_linear(beta, passes=60):
    """|n_L - n_L*| of the inputs of linear mixing."""
    history, _ = iterate(beta, passes=passes)
    return np.array([abs(n - 1.0 - x_star) for n, _ in history])


def errors_anderson(beta=BETA_ANDERSON, depth=DEPTH, passes=60, tolerance=1e-14):
    """|n_L - n_L*| of the inputs of Anderson mixing (one variable)."""
    n, inputs, residuals, errors = 2.0, [], [], []
    for _ in range(passes):
        errors.append(abs(n - 1.0 - x_star))
        residual = loop_pass(n) - n
        if abs(residual) < tolerance:
            break
        inputs, residuals = (inputs + [n])[-depth:], (residuals + [residual])[-depth:]
        if len(inputs) == 1:
            n = n + beta * residual
            continue
        last = residuals[-1]
        differences = np.array([[r - last for r in residuals[:-1]]])  # 1 row
        theta = np.linalg.lstsq(differences, -np.array([last]), rcond=None)[0]
        c = np.append(theta, 1.0 - theta.sum())
        n = float(sum(ci * (ni + beta * ri)
                      for ci, ni, ri in zip(c, inputs, residuals)))
    return np.array(errors)


histories = {beta: errors_linear(beta) for beta in (1.0, 0.8, 0.5, 0.2)}
```

```
histories["best"] = errors_linear(beta_best)
anderson_errors = errors_anderson()
ratio = histories[0.5][11] / histories[0.5][10]  # the ratio of two error sizes
predicted = 1.0 - 0.5 * (1.0 - g_prime)
report("measured |error ratio| for beta = 1/2", f"{ratio:.6f}")
report("predicted factor 1 - beta (1 - G')", f"{predicted:.6f}")
check(abs(ratio - abs(predicted)) < 1e-4,
      "the error shrinks by the predicted factor |1 - beta(1 - G')| = 0.344")
check(histories[1.0][-1] > 0.5 and histories[0.8][-1] > 0.1,
      "beta = 1 and beta = 0.8 (above 0.744) do not converge")
check(len(anderson_errors) < 15 and anderson_errors[-1] < 1e-12,
      "Anderson mixing converges to 1e-12 in fewer than 15 passes")
```

Out [8]:

```
PASS the Revision solver mixes with depth 6 and beta 0.4
    (Revision/kohn_sham/results/parameters.json, numerics)
RESULT measured |error ratio| for beta = 1/2 = 0.343986
RESULT predicted factor 1 - beta (1 - G') = -0.343980
PASS the error shrinks by the predicted factor |1 - beta(1 - G')| = 0.344
PASS beta = 1 and beta = 0.8 (above 0.744) do not converge
PASS Anderson mixing converges to 1e-12 in fewer than 15 passes
```

Text cell:

```
The next cell draws the error histories on a logarithmic vertical axis.
```

In [9]:

```
fig, ax = plt.subplots()
for key, style in ((1.0, "o-"), (0.8, "s-"), (0.5, "^-"), (0.2, "v-")):
    label = "plain iteration" if key == 1.0 else f"linear, $\\beta = {key}$"
    ax.semilogy(histories[key][:40], style, ms=3, label=label)
best = np.maximum(histories["best"][:12], 1e-16)  # rounding may give exactly 0
ax.semilogy(best, "D-", ms=3, label=f"linear, best $\\beta = {beta_best:.3f}$")
ax.semilogy(anderson_errors, "k*-", ms=6,
            label=f"Anderson, depth {DEPTH}, $\\beta = {BETA_ANDERSON}$")
ax.set_ylim(1e-15, 10.0)
ax.set_xlabel("pass number")
ax.set_ylabel("error $|n_L - n_L^*|$")
ax.set_title("Error histories of the two-site loop")
ax.legend(fontsize=7, loc="upper right", bbox_to_anchor=(1.0, 0.86))
save_figure(fig, "error_histories",
            "The distance $|n_L - n_L^\\ast|$ of the input from the self-consistent "
            "density (logarithmic vertical axis, electrons) against the pass number "
            "for plain iteration and linear mixing with $\\beta = 0.8$ (both stay "
            "far away: sloshing), $\\beta = 0.5$ and $0.2$ (straight lines: the error "
            "shrinks by a fixed factor per pass), the best $\\beta = 0.372$ (very "
            "fast) and Anderson mixing with the Revision solver's settings (fast "
            "without knowing the slope in advance).")
```

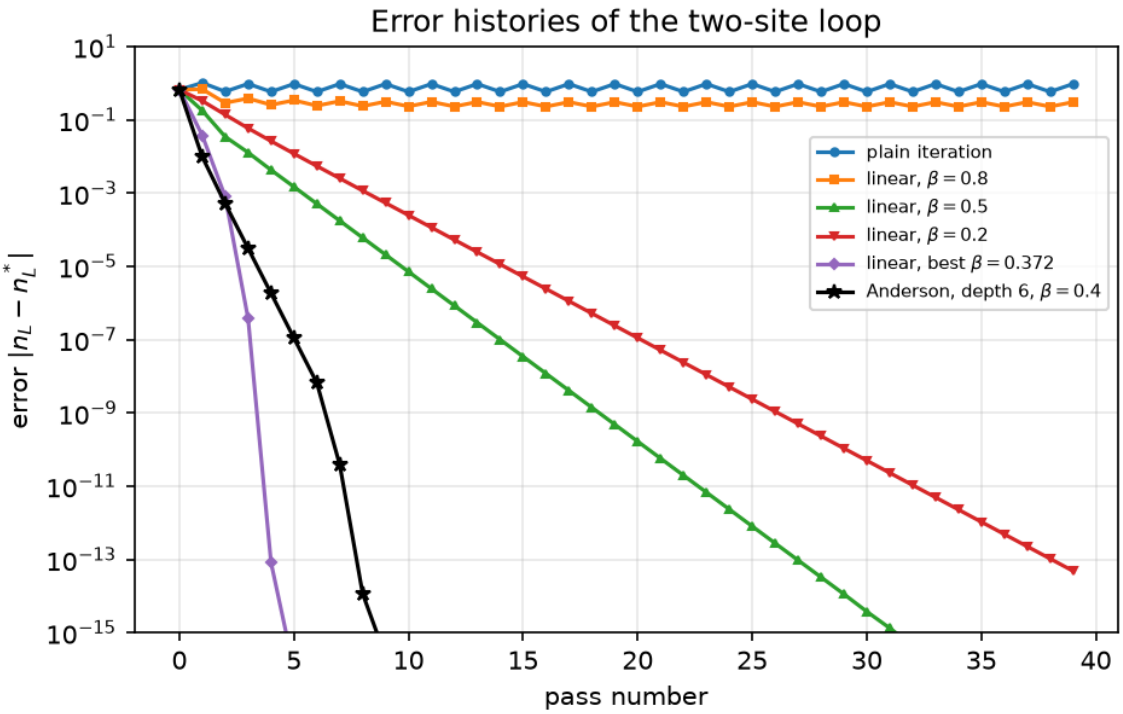

Figure 363: The distance $|n_L - n_L^*|$ of the input from the self-consistent density (logarithmic vertical axis, electrons) against the pass number for plain iteration and linear mixing with $\beta = 0.8$ (both stay far away: sloshing), $\beta = 0.5$ and 0.2 (straight lines: the error shrinks by a fixed factor per pass), the best $\beta = 0.372$ (very fast) and Anderson mixing with the Revision solver's settings (fast without knowing the slope in advance). (Notebook 13d, figure 3.)

Out [9]:

```
Figure 13d.3 saved as Revision/textbook/figures/13d_3_error_histories.png
```

Text cell:

```
The next cell draws the predicted factor $|1 - \beta(1 - G')|$ against $\beta$; the
loop converges where it is below 1.
```

In [10]:

```
betas = np.linspace(0.0, 1.0, 501)
factor = np.abs(1.0 - betas * (1.0 - g_prime))
fig, ax = plt.subplots()
ax.plot(betas, factor, color="black", lw=2.0, label="$|1 - \\beta(1 - G')|$")
ax.axhline(1.0, color="gray", ls="--")
ax.axvspan(0.0, beta_max, alpha=0.15, color="green", label="converges")
ax.axvline(beta_best, color="tab:blue", ls=":",
           label=f"best $\\beta = {beta_best:.3f}$")
ax.set_xlabel("mixing parameter $\\beta$")
ax.set_ylabel("error factor per pass")
ax.set_title(f"Linear mixing converges for $\\beta < {beta_max:.4f}$")
ax.legend()
save_figure(fig, "convergence_factor",
            "The factor $|1 - \\beta(1 - G'(x^\\ast))|$ by which one pass of linear "
            "mixing multiplies the error near the fixed point, against the mixing "
            "parameter $\\beta$ (pure numbers), for $G'(x^\\ast) = -1.688$. The loop "
            "converges where the factor is below 1 (green band, "
            "$0 < \\beta < 0.744$); the factor is zero at the best value "
            "$\\beta = 0.372$, and plain iteration ($\\beta = 1$) has the factor "
            "1.688: the error grows.")
```

Out [10]:

```
Figure 13d.4 saved as Revision/textbook/figures/13d_4_convergence_factor.png
```

Text cell:

```
## 9. The long run against the mixing parameter

The next cell runs the loop with 99 values of $\beta$ from 0.02 to 1 for 600
passes each and keeps the last 30 inputs. Where the loop converges they are all the
same number; where it does not, they alternate between two numbers.
```

In [11]:

```
beta_scan = np.linspace(0.02, 1.0, 99)
tails = []
```

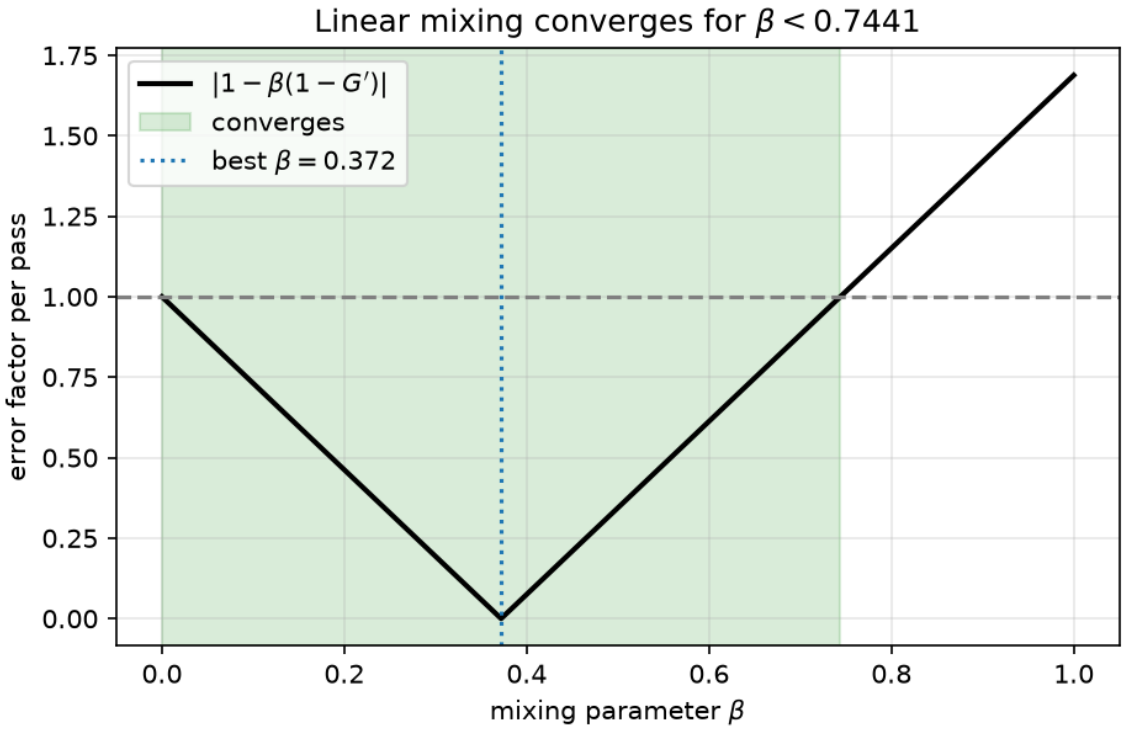

Figure 364: The factor $|1 - \beta(1 - G'(x^*))|$ by which one pass of linear mixing multiplies the error near the fixed point, against the mixing parameter β (pure numbers), for $G'(x^*) = -1.688$. The loop converges where the factor is below 1 (green band, $0 < \beta < 0.744$); the factor is zero at the best value $\beta = 0.372$, and plain iteration ($\beta = 1$) has the factor 1.688: the error grows. (Notebook 13d, figure 4.)

```
for beta in beta_scan:
    history, _ = iterate(beta, passes=600)
    tails.append([n for n, _ in history[-30:]])
tails = np.array(tails)
spread = tails.max(axis=1) - tails.min(axis=1)
check(np.all(spread[beta_scan < 0.72] < 1e-8) and np.all(spread[beta_scan > 0.77]
                                                         > 0.05),
      "the loop converges below beta = 0.744 and oscillates above it")
fig, ax = plt.subplots()
for beta, tail in zip(beta_scan, tails):
    ax.plot(np.full(len(tail), beta), tail, "k.", ms=2)
ax.axvline(beta_max, color="tab:red", ls="--", label=f"$\\beta_{{max}} = "
           f"{beta_max:.4f}$")
ax.set_xlabel("mixing parameter $\\beta$")
ax.set_ylabel("last 30 inputs $n_L$")
ax.set_title("Where the loop ends, against $\\beta$")
ax.legend()
save_figure(fig, "long_run_diagram",
            "The last 30 inputs $n_L$ (electrons) of 600 passes of the two-site loop "
            "for 99 mixing parameters $\\beta$ from 0.02 to 1 (horizontal axis). "
            "Below $\\beta_{max} = 0.744$ (dashed red line) all points coincide at "
            "the self-consistent density 1.327; above it the loop ends in a cycle "
            "of period 2 whose two values move apart as $\\beta$ grows, reaching "
            "0.36 and 1.92 for plain iteration.")
```

Out [11]:

```
PASS the loop converges below beta = 0.744 and oscillates above it
```

Out [11]:

```
Figure 13d.5 saved as Revision/textbook/figures/13d_5_long_run_diagram.png
```

Text cell:

```
## 10. A weaker repulsion, and the threshold against U

With $U = 2$ the slope at the fixed point is milder, and $\beta_{max} > 1$: even
plain iteration converges (slowly, alternating). The next cell checks the numbers
$x^* = 0.468990$, $G' = -0.688942$, $\beta_{max} = 1.184174$, and then computes
$\beta_{max}$ for repulsions from 0.5 to 8, and finds by bisection the repulsion at
which plain iteration stops converging ($\beta_{max} = 1$, i.e. $G'(x^*) = -1$).
```

In [12]:

```
x2 = fixed_point(u=2.0)
g2 = slope(x2, u=2.0)
report("U = 2: n_L*", f"{1.0 + x2:.6f}")
report("U = 2: G'(x*)", f"{g2:.6f}")
```

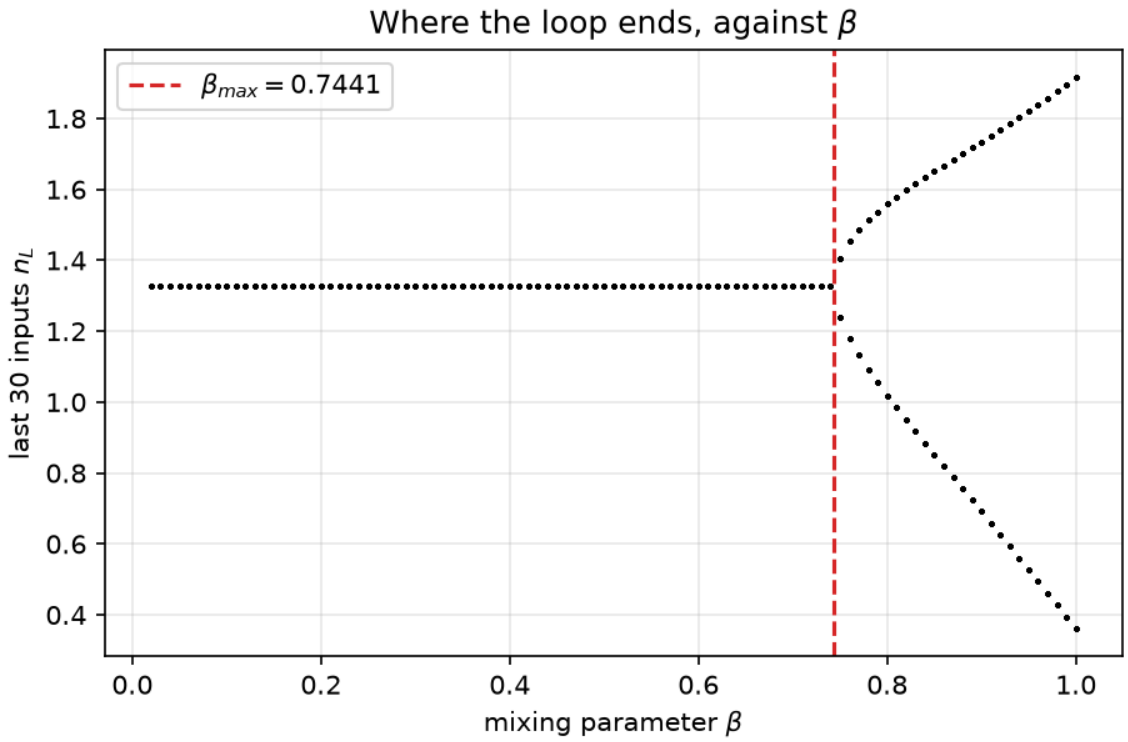

Figure 365: The last 30 inputs n_L (electrons) of 600 passes of the two-site loop for 99 mixing parameters β from 0.02 to 1 (horizontal axis). Below $\beta_{max} = 0.744$ (dashed red line) all points coincide at the self-consistent density 1.327; above it the loop ends in a cycle of period 2 whose two values move apart as β grows, reaching 0.36 and 1.92 for plain iteration. (Notebook 13d, figure 5.)

```
report("U = 2: beta_max", f"{2.0 / (1.0 - g2):.6f}")
check(abs(x2 - 0.468990) < 1e-6 and abs(g2 + 0.688942) < 1e-6
      and abs(2.0 / (1.0 - g2) - 1.184174) < 1e-6,
      "U = 2: x* = 0.468990, G' = -0.688942, beta_max = 1.184174")
n = 2.0
for _ in range(200):  # plain iteration with U = 2
    n = loop_pass(n, u=2.0)
check(abs(n - 1.0 - x2) < 1e-10, "U = 2: plain iteration converges")
U_scan = np.linspace(0.5, 8.0, 76)
beta_limits = np.array([2.0 / (1.0 - slope(fixed_point(u=u), u=u)) for u in U_scan])
low, high = 2.0, 4.0  # beta_max(2) > 1 > beta_max(4)
for _ in range(60):
    middle = 0.5 * (low + high)
    if 2.0 / (1.0 - slope(fixed_point(u=middle), u=middle)) > 1.0:
        low = middle
    else:
        high = middle
U_critical = 0.5 * (low + high)
report("repulsion where plain iteration stops converging", f"{U_critical:.6f}")
check(abs(slope(fixed_point(u=U_critical), u=U_critical) + 1.0) < 1e-9,
      "at this repulsion the slope at the fixed point is exactly -1")
```

Out [12]:

```
RESULT U = 2: n_L* = 1.468990
RESULT U = 2: G'(x*) = -0.688942
RESULT U = 2: beta_max = 1.184174
PASS U = 2: x* = 0.468990, G' = -0.688942, beta_max = 1.184174
PASS U = 2: plain iteration converges
RESULT repulsion where plain iteration stops converging = 2.647393
PASS at this repulsion the slope at the fixed point is exactly -1
```

Text cell:

```
The next cell draws $\beta_{max}$ against $U$.
```

In [13]:

```
fig, ax = plt.subplots()
ax.plot(U_scan, beta_limits, color="black", lw=2.0, label="$\\beta_{max}(U)$")
ax.axhline(1.0, color="gray", ls="--", label="plain iteration, $\\beta = 1$")
ax.axvline(U_critical, color="tab:red", ls=":",
           label=f"$U = {U_critical:.3f}$: plain iteration fails beyond")
ax.set_xlabel("repulsion $U$ (units of $t$)")
ax.set_ylabel("largest convergent mixing parameter")
ax.set_title("Stronger repulsion needs gentler mixing")
ax.legend(fontsize=8)
save_figure(fig, "threshold_versus_u",
            "The largest mixing parameter $\\beta_{max} = 2/(1 - G'(x^\\ast))$ for "
            "which linear mixing converges, against the repulsion $U$ (units of "
            "$t$; $\\Delta = 2$, $t = 1$). Below the dashed line $\\beta = 1$ plain "
```

```
"iteration fails; this happens beyond the repulsion marked by the "
"dotted line. The stronger the feedback of the density on its own "
"potential, the smaller the step the loop may take.")
```

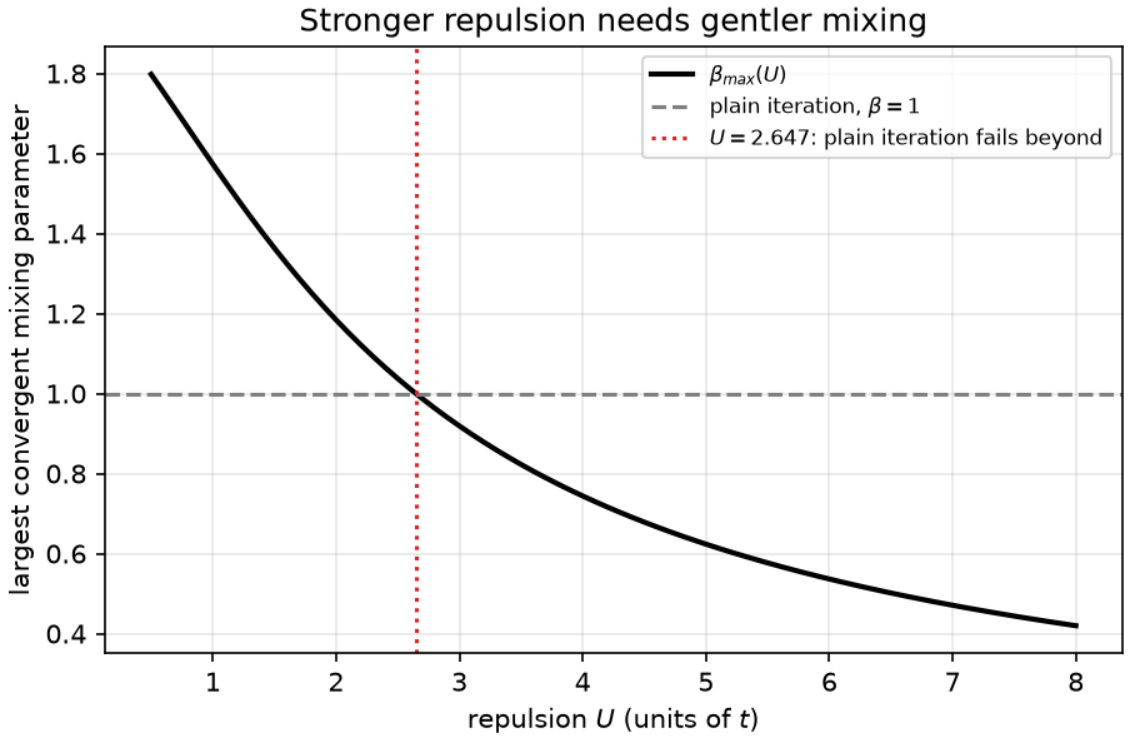

Figure 366: The largest mixing parameter $\beta_{max} = 2/(1 - G'(x^*))$ for which linear mixing converges, against the repulsion U (units of t; $\Delta = 2$, $t = 1$). Below the dashed line $\beta = 1$ plain iteration fails; this happens beyond the repulsion marked by the dotted line. The stronger the feedback of the density on its own potential, the smaller the step the loop may take. (Notebook 13d, figure 6.)

Out [13]:

```
Figure 13d.6 saved as Revision/textbook/figures/13d_6_threshold_versus_u.png
```

Text cell:

```
## 11. The last check

The last cell checks that all six figure files exist and prints the number of
checks that passed.
```

In [14]:

```
figure_names = ["reduced_map", "cobwebs", "error_histories", "convergence_factor",
                "long_run_diagram", "threshold_versus_u"]
missing = [name for k, name in enumerate(figure_names, 1)
```

```
          if not output_file(f"{FIGURE_FOLDER}/13d_{k}_{name}.png").is_file()]
check(missing == [], "all six figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13d_6_threshold_versus_u.png").is_file(),
      "the figure file 13d_6_threshold_versus_u.png exists")
all_checks_passed()
```

Out [14]:

```
PASS all six figure files exist
PASS the figure file 13d_6_threshold_versus_u.png exists
ALL 17 CHECKS PASSED (notebook 13d)
```

Text cell:

```
## 12. What this notebook showed

- The whole self-consistency loop of the two-site mean-field model is one function
  $G(x) = (\Delta - Ux)/\sqrt{(\Delta - Ux)^2 + 4t^2}$ of one number, and its fixed
  point is the self-consistent density $n_L^* = 1.326993$.
- Plain iteration fails: the charge jumps between the sites forever (a cycle of
  period 2 near 0.3607 and 1.9157), because the slope $G'(x^*) = -1.688$ is steeper
  than $-1$.
- Linear mixing converges exactly for $0 < \beta < 2/(1 - G') = 0.744058$; each pass
  multiplies the error by $1 - \beta(1 - G')$ ($-0.344$ for $\beta = 1/2$), and
  $\beta = 1/(1 - G') = 0.372$ is the fastest.
- Anderson mixing, which estimates the slope from earlier passes, converges in a
  few passes without tuning; the Revision Kohn-Sham solver of the dirac16complex
  field uses it with depth 6 and $\beta = 0.4$ (read here from its parameter file).
- The stronger the repulsion, the smaller the step the loop may take; for
  $\Delta = 2$, $t = 1$ plain iteration converges only for $U$ below the printed
  threshold (for example $U = 2$: $\beta_{max} = 1.184$).
```

13.25 Line-by-line walk-through of Notebook 13d

The notebook has fourteen code cells. **In [1]** is the set-up cell, identical to In [1] of Notebook 13c (Section 13.14) except for the name `"13d"` and its comment lines, which hold the instructions of Section 13.23.

In [2], the loop and the reduced map.

```
import numpy as np  # arrays, matrices and linear algebra

DELTA, T_HOP, U = 2.0, 1.0, 4.0  # site-energy difference, hopping, repulsion


def loop_pass(n_left, delta=DELTA, t=T_HOP, u=U):
    """One pass: the output density n_L of the lowest orbital of h[n_L]."""
    n_right = 2.0 - n_left
    h = np.array([[-delta / 2 + u * n_left / 2, -t],
                  [-t, delta / 2 + u * n_right / 2]])
```

```
    values, vectors = np.linalg.eigh(h)  # levels in increasing order
    return 2.0 * vectors[0, 0] ** 2  # two electrons, |c_L|^2 each
```

The cell loads numpy as `np`. The model's numbers $\Delta = 2$, $t = 1$, $U = 4$ are written in one line: Python assigns the three values to the three names in order. `loop_pass` is one pass of the loop: it builds the matrix $h[n_L]$ of Section 13.21, diagonalises it with `eigh` (levels in increasing order, eigenvectors as columns), and returns $2|c_L|^2$, where `vectors[0, 0]` is the L component (row 0) of the lowest eigenvector (column 0). The arguments `delta=DELTA` and so on are **default values**: they are used when the call does not give them, so later cells can call `loop_pass(n, u=2.0)` with another repulsion.

```
def G(x, delta=DELTA, t=T_HOP, u=U):
    """The reduced map x_out = G(x), x = n_L - 1."""
    shift = delta - u * x
    return shift / np.sqrt(shift ** 2 + 4.0 * t * t)


inputs = np.linspace(0.0, 2.0, 41)
worst = max(abs(loop_pass(n) - 1.0 - G(n - 1.0)) for n in inputs)
check(worst < 1e-12, "the 2 x 2 diagonalisation and the reduced map G agree")
```

`G` is the closed form of Section 13.21. For 41 inputs from 0 to 2 the cell compares $n_L^{out} - 1$ from the diagonalisation with $G(n_L - 1)$ and checks that the largest difference is below 10^{-12}: the four-step derivation is confirmed.

In [3], the fixed point and the slope.

```
def fixed_point(delta=DELTA, t=T_HOP, u=U):
    """x* with G(x*) = x*, by bisection on [-1, 1] (x - G(x) increases)."""
    low, high = -1.0, 1.0
    for _ in range(80):
        middle = 0.5 * (low + high)
        if middle - G(middle, delta, t, u) > 0.0:
            high = middle
        else:
            low = middle
    return 0.5 * (low + high)


def slope(x, delta=DELTA, t=T_HOP, u=U):
    """G'(x) = -4 U t^2 / ((Delta - U x)^2 + 4 t^2)^(3/2)."""
    return -4.0 * u * t * t / ((delta - u * x) ** 2 + 4.0 * t * t) ** 1.5
```

`fixed_point` finds x^* by bisection: $x - G(x)$ increases with x (because G decreases), is negative at $x = -1$ and positive at $x = 1$; at each of 80 steps the cell evaluates it at the middle of the interval and keeps the half in which the sign changes. After 80 halvings the interval is far below the rounding of the computer. `slope` is the formula for G' of Section 13.21.

```
x_star = fixed_point()
g_prime = slope(x_star)
```

```
quotient = (G(x_star + 1e-6) - G(x_star - 1e-6)) / 2e-6
beta_max, beta_best = 2.0 / (1.0 - g_prime), 1.0 / (1.0 - g_prime)
```

The fixed point, the slope there, a central difference quotient $(G(x^*+10^{-6})-G(x^*-10^{-6}))/(2\cdot 10^{-6})$ as an independent estimate of the slope, and the largest and the best mixing parameters (two names assigned in one line).

```
report("fixed point n_L* = 1 + x*", f"{1.0 + x_star:.6f}")
report("slope G'(x*)", f"{g_prime:.6f}")
report("largest mixing parameter 2/(1 - G')", f"{beta_max:.6f}")
report("best mixing parameter 1/(1 - G')", f"{beta_best:.6f}")
check(abs(x_star - 0.326993) < 1e-6 and abs(g_prime + 1.687961) < 1e-6
      and abs(beta_max - 0.744058) < 1e-6,
      "x* = 0.326993, G'(x*) = -1.687961, beta_max = 0.744058")
check(abs(quotient - g_prime) < 1e-6, "the slope formula equals a difference quotient")
```

The four `report` lines print $n_L^* = 1.326993$, $G' = -1.687961$, $\beta_{max} = 0.744058$ and $\beta_{best} = 0.372029$; the two checks compare the first three with these values (to 10^{-6}) and the slope formula with the difference quotient (to 10^{-6}; the central quotient has an error of order $(10^{-6})^2$ times the third derivative, plus rounding).

In [4], the map as a picture.

```
n_in = np.linspace(0.0, 2.0, 401)
fig, ax = plt.subplots(figsize=(6.0, 5.0))
ax.plot(n_in, 1.0 + G(n_in - 1.0), color="black", lw=2.0,
        label="$n_L^{out} = 1 + G(n_L - 1)$")
ax.plot(n_in, n_in, "--", color="gray", label="diagonal $n_L^{out} = n_L$")
ax.plot([1.0 + x_star], [1.0 + x_star], "ro",
        label=f"fixed point $n_L^* = {1.0 + x_star:.6f}$")
```

`n_in` holds 401 input densities from 0 to 2. The cell draws the output $1 + G(n_L - 1)$ against the input as a thick black line, the diagonal $n_L^{out} = n_L$ grey and dashed, and the fixed point as a red dot (`"ro"`: r red, o a round marker) at (n_L^*, n_L^*), whose value the legend shows with six decimals.

```
ax.set_xlabel("input density on the left site $n_L$")
ax.set_ylabel("output density $n_L^{out}$")
ax.set_title("The self-consistency map of the two-site model")
ax.legend(fontsize=8)
save_figure(fig, "reduced_map",
```

Labels, title, legend and `save_figure` for Figure 13d.1.

What Figure 13d.1 shows. The map falls from the output 1.95 at the input $n_L = 0$ to the output 0.29 at the input $n_L = 2$: the more charge the input puts on the left site, the more the repulsion pushes the output to the right. Where the curve crosses the diagonal, input and output agree: the self-consistent density 1.326993. The crossing is steep (slope -1.69), which is the cause of all the trouble of Section 13.21.

In [5], plain iteration.

```
def iterate(beta, start=2.0, passes=6):
    """Inputs and outputs of the loop with linear mixing (beta = 1: plain)."""
    n, history = start, []
    for _ in range(passes):
        out = loop_pass(n)
        history.append((n, out))
        n = (1.0 - beta) * n + beta * out
    return history, n
```

`iterate` runs the loop with linear mixing: at each pass it computes the output, stores the pair (input, output) in the list `history` (a pair in parentheses is a **tuple**), and forms the next input $(1-\beta)n + \beta\, n^{out}$. It returns the history and the next input.

```
plain, _ = iterate(1.0)
say("step   n_L in     n_L out")
for step, (n, out) in enumerate(plain):
    say(f"{step:4d}   {n:.6f}   {out:.6f}")
expected_in = [2.000000, 0.292893, 1.923880, 0.353344, 1.916644, 0.359836]
check(all(abs(n - e) < 1e-6 for (n, _), e in zip(plain, expected_in)),
      "plain iteration reproduces the table 2.000000, 0.292893, 1.923880, ...")
```

Six passes of plain iteration ($\beta = 1$); the name `_` receives the second returned value, which is not needed. The loop prints the table of Section 13.21 (`{step:4d}` writes the step number in four places, `{n:.6f}` the density with six decimals), and the check compares the inputs with the six values of that table.

```
_, late = iterate(1.0, passes=2000)
cycle = sorted([late, loop_pass(late)])
say(f"after 2000 passes the input alternates between {cycle[0]:.4f} and "
    f"{cycle[1]:.4f}")
check(abs(loop_pass(loop_pass(late)) - late) < 1e-10 and cycle[1] - cycle[0] > 1.5,
      "plain iteration ends in a cycle of period 2 (about 0.3607 and 1.9157)")
```

After 2000 passes, `late` is the next input; with its output it forms the two values of the cycle, sorted. The check requires that two passes return to the same value (a cycle of period 2, to 10^{-10}) and that the two values are more than 1.5 apart: the charge keeps jumping between the sites.

In [6], linear mixing with $\beta = 1/2$.

```
mixed, _ = iterate(0.5)
say("step   n_L in     n_L out")
for step, (n, out) in enumerate(mixed):
    say(f"{step:4d}   {n:.6f}   {out:.6f}")
expected_in = [2.000000, 1.146447, 1.361898, 1.314066, 1.331307, 1.325494]
check(all(abs(n - e) < 1e-6 for (n, _), e in zip(mixed, expected_in)),
      "linear mixing with beta = 1/2 reproduces the table 2.000000, 1.146447, ...")
long_history, _ = iterate(0.5, passes=40)
agree = next(step for step, (n, out) in enumerate(long_history)
             if abs(n - out) < 1e-6)
report("passes until input and output agree to 1e-6 (beta = 1/2)", agree)
check(agree == 13 and abs(long_history[-1][0] - 1.326993) < 1e-6,
      "beta = 1/2 converges to 1.326993; input and output agree after 13 passes")
```

The same for $\beta = \frac{1}{2}$: the second table of Section 13.21 and its check. Then 40 passes; `next(...)` returns the first step number whose input and output agree to 10^{-6}, which is 13; the last check also requires the last input (`long_history[-1][0]`, the first element of the last pair) to be 1.326993.

In [7], cobweb diagrams.

```
def cobweb(ax, beta, passes):
    """Draw the cobweb of the loop with mixing beta on the axes ax."""
    ax.plot(n_in, 1.0 + G(n_in - 1.0), color="black", lw=1.5)
    ax.plot(n_in, n_in, "--", color="gray", lw=0.8)
    history, _ = iterate(beta, passes=passes)
    for n, out in history:
        n_next = (1.0 - beta) * n + beta * out
        ax.plot([n, n], [n, out], color="tab:red", lw=0.8)  # up to the curve
        ax.plot([n, n_next], [out, n_next], color="tab:blue", lw=0.8)  # across
    ax.plot([1.0 + x_star], [1.0 + x_star], "ko")
    ax.set_xlabel("input $n_L$")
    ax.set_ylabel("output $n_L^{out}$ and next input")
```

A **cobweb diagram** draws an iteration: from the point (input, input) on the diagonal a red segment goes vertically to the curve (the output), and a blue segment goes from there to the point (next input, next input) on the diagonal. The function draws the map and the diagonal, runs the loop, and draws the two segments of every pass (`[n, n]` and `[n, out]` are the horizontal and the vertical coordinates of the two ends of the red segment), the fixed point as a black dot (`"ko"`), and labels the axes.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.5))
cobweb(left, 1.0, 12)
left.set_title("plain iteration ($\\beta = 1$): sloshing")
cobweb(right, 0.5, 12)
right.set_title("linear mixing ($\\beta = 1/2$): converges")
save_figure(fig, "cobwebs",
```

The cell calls `cobweb` for plain iteration in the left panel and for $\beta = \frac{1}{2}$ in the right panel, 12 passes each, gives each panel its title, and saves Figure 13d.2.

What Figure 13d.2 shows. On the left the path starts at $n_L = 2$, goes down to the output 0.29, across to the input 0.29, up to the output 1.92, and so on: after a few passes it runs around the same rectangle with the corners near 0.36 and 1.92, far from the fixed point in the middle; the thick lines are the many passes drawn on top of each other. On the right each blue segment ends on the diagonal at the point halfway between the input and the output, and the path closes in on the fixed point within a few passes, in a spiral, because the error changes its sign at every pass (the factor -0.344).

In [8], error histories and Anderson mixing.

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
DEPTH = int(parameters["numerics"]["andersonDepth"])  # passes remembered
BETA_ANDERSON = float(parameters["numerics"]["andersonBeta"])  # mixing parameter
```

```
check(DEPTH == 6 and BETA_ANDERSON == 0.4,
      "the Revision solver mixes with depth 6 and beta 0.4 "
      "(Revision/kohn_sham/results/parameters.json, numerics)")
```

The cell reads the parameter file of the Revision solver and takes from its part "numerics" the number of remembered passes and β; `int` and `float` make a whole number and a floating-point number of them. The check confirms the values 6 and 0.4.

```
def errors_linear(beta, passes=60):
    """|n_L - n_L*| of the inputs of linear mixing."""
    history, _ = iterate(beta, passes=passes)
    return np.array([abs(n - 1.0 - x_star) for n, _ in history])
```

The distance of every input of 60 passes of linear mixing from the fixed point $n_L^* = 1 + x^*$.

```
def errors_anderson(beta=BETA_ANDERSON, depth=DEPTH, passes=60, tolerance=1e-14):
    """|n_L - n_L*| of the inputs of Anderson mixing (one variable)."""
    n, inputs, residuals, errors = 2.0, [], [], []
    for _ in range(passes):
        errors.append(abs(n - 1.0 - x_star))
        residual = loop_pass(n) - n
        if abs(residual) < tolerance:
            break
        inputs, residuals = (inputs + [n])[-depth:], (residuals + [residual])[-depth:]
        if len(inputs) == 1:
            n = n + beta * residual
            continue
        last = residuals[-1]
        differences = np.array([[r - last for r in residuals[:-1]]])  # 1 row
        theta = np.linalg.lstsq(differences, -np.array([last]), rcond=None)[0]
        c = np.append(theta, 1.0 - theta.sum())
        n = float(sum(ci * (ni + beta * ri)
                      for ci, ni, ri in zip(c, inputs, residuals)))
    return np.array(errors)
```

Anderson mixing of Section 13.21 for one variable. Each pass records the error, computes the residual $R = n^{out} - n$ and stops (`break` leaves the loop) when it is below 10^{-14}. `(inputs + [n])[-depth:]` appends the new input and keeps the last `depth` entries (a negative index counts from the end). In the first pass there is no history, so the step is linear mixing; `continue` goes on to the next pass. Otherwise `differences` is the table with one row of the differences $R^{(i)} - R^{(last)}$, `np.linalg.lstsq` solves the least-squares problem for θ (with one equation and several unknowns it returns the solution with the smallest $\sum \theta_i^2$), `np.append` adds $c_{last} = 1 - \sum \theta_i$, and the next input is $\sum_i c_i (n^{(i)} + \beta R^{(i)})$.

```
histories = {beta: errors_linear(beta) for beta in (1.0, 0.8, 0.5, 0.2)}
histories["best"] = errors_linear(beta_best)
anderson_errors = errors_anderson()
ratio = histories[0.5][11] / histories[0.5][10]  # the ratio of two error sizes
predicted = 1.0 - 0.5 * (1.0 - g_prime)
```

The error histories for $\beta = 1, 0.8, 0.5, 0.2$ and for β_{best}, and that of Anderson mixing. `ratio` divides the error after pass 11 by the error after pass 10 for $\beta = \frac{1}{2}$, and `predicted` is the factor $1 - \beta(1 - G')$ of Section 13.21.

```
report("measured |error ratio| for beta = 1/2", f"{ratio:.6f}")
report("predicted factor 1 - beta (1 - G')", f"{predicted:.6f}")
```

They print as 0.343986 and −0.343980: the size agrees (the sign of the measured ratio is lost because errors are taken as absolute values, and the small difference is the curvature of G).

```
check(abs(ratio - abs(predicted)) < 1e-4,
      "the error shrinks by the predicted factor |1 - beta(1 - G')| = 0.344")
check(histories[1.0][-1] > 0.5 and histories[0.8][-1] > 0.1,
      "beta = 1 and beta = 0.8 (above 0.744) do not converge")
check(len(anderson_errors) < 15 and anderson_errors[-1] < 1e-12,
      "Anderson mixing converges to 1e-12 in fewer than 15 passes")
```

Three checks: the predicted factor; the failure of $\beta = 1$ and $\beta = 0.8$, both above β_{max} (their last errors are still large); and the convergence of Anderson mixing to 10^{-12} in fewer than 15 passes.

In [9], the error histories as a picture.

```
fig, ax = plt.subplots()
for key, style in ((1.0, "o-"), (0.8, "s-"), (0.5, "^-"), (0.2, "v-")):
    label = "plain iteration" if key == 1.0 else f"linear, $\\beta = {key}$"
    ax.semilogy(histories[key][:40], style, ms=3, label=label)
```

`ax.semilogy` draws with a logarithmic vertical axis; given only one list, it uses the pass numbers $0, 1, 2, \ldots$ as horizontal coordinates. For the four linear runs the loop draws the first 40 errors with its own marker (circles, squares, and triangles pointing up and down, each joined by a line); the label is "plain iteration" for $\beta = 1$ and otherwise names β (`a if condition else b` chooses).

```
best = np.maximum(histories["best"][:12], 1e-16)  # rounding may give exactly 0
ax.semilogy(best, "D-", ms=3, label=f"linear, best $\\beta = {beta_best:.3f}$")
ax.semilogy(anderson_errors, "k*-", ms=6,
            label=f"Anderson, depth {DEPTH}, $\\beta = {BETA_ANDERSON}$")
```

The first 12 errors of the best linear mixing, with diamonds (`"D-"`); `np.maximum(..., 1e-16)` replaces an error of exactly 0, which a logarithmic axis cannot show, by 10^{-16}. Then all errors of Anderson mixing as black stars joined by a line (`"k*-"`); the label shows the depth and β read from the Revision parameter file.

```
ax.set_ylim(1e-15, 10.0)
ax.set_xlabel("pass number")
ax.set_ylabel("error $|n_L - n_L^*|$")
ax.set_title("Error histories of the two-site loop")
ax.legend(fontsize=7, loc="upper right", bbox_to_anchor=(1.0, 0.86))
save_figure(fig, "error_histories",
```

`set_ylim(1e-15, 10.0)` fixes the vertical range from 10^{-15} to 10; labels and title; the legend is anchored with its upper right corner at 86 per cent of the height of the panel (`bbox_to_anchor=( 1.0, 0.86)`), a little below the top, so that it does not cover the two upper curves. `save_figure` saves Figure 13d.3.

What Figure 13d.3 shows. Plain iteration and $\beta = 0.8$ stay between about 0.2 and 1 for all 40 passes, zig-zagging: they never converge. The runs with $\beta = 0.5$ and 0.2 are straight falling lines on the logarithmic axis, which means that the error is multiplied by the same factor at every pass (0.344 and about 0.46); the first reaches 10^{-15} after about 31 passes, the second is near 10^{-13} after 40. The best $\beta = 0.372$ (factor zero for small errors) and Anderson mixing fall much faster, reaching rounding level within about 5 and 9 passes; Anderson mixing achieves this without knowing the slope G' in advance.

In [10], the convergence factor.

```
betas = np.linspace(0.0, 1.0, 501)
factor = np.abs(1.0 - betas * (1.0 - g_prime))
```

The size of the factor $|1 - \beta(1 - G')|$ for 501 values of β from 0 to 1.

```
fig, ax = plt.subplots()
ax.plot(betas, factor, color="black", lw=2.0, label="$|1 - \\beta(1 - G')|$")
ax.axhline(1.0, color="gray", ls="--")
ax.axvspan(0.0, beta_max, alpha=0.15, color="green", label="converges")
ax.axvline(beta_best, color="tab:blue", ls=":",
           label=f"best $\\beta = {beta_best:.3f}$")
```

The factor as a thick black line; the grey dashed line at 1; `axvspan(0.0, beta_max, ...)` colours the vertical band $0 < \beta < \beta_{max}$, where the loop converges, in a pale green (`alpha=0.15` makes it 85 per cent transparent); `axvline` draws a dotted vertical line at β_{best}.

```
ax.set_xlabel("mixing parameter $\\beta$")
ax.set_ylabel("error factor per pass")
ax.set_title(f"Linear mixing converges for $\\beta < {beta_max:.4f}$")
ax.legend()
save_figure(fig, "convergence_factor",
```

Labels; the title shows β_{max} with four decimals (0.7441); legend; `save_figure` for Figure 13d.4.

What Figure 13d.4 shows. A V-shaped curve: the factor falls from 1 at $\beta = 0$ (no change at all, so no progress) to 0 at $\beta_{best} = 0.372$, rises again, crosses 1 at $\beta_{max} = 0.744$ (the right edge of the green band) and reaches 1.688 at $\beta = 1$. Every β inside the band converges; the closer to β_{best}, the faster.

In [11], the long run against β.

```
beta_scan = np.linspace(0.02, 1.0, 99)
tails = []
for beta in beta_scan:
```

```
    history, _ = iterate(beta, passes=600)
    tails.append([n for n, _ in history[-30:]])
tails = np.array(tails)
spread = tails.max(axis=1) - tails.min(axis=1)
check(np.all(spread[beta_scan < 0.72] < 1e-8) and np.all(spread[beta_scan > 0.77]
                                                         > 0.05),
      "the loop converges below beta = 0.744 and oscillates above it")
```

For 99 values of β from 0.02 to 1 the cell runs 600 passes and keeps the last 30 inputs (`history[ -30:]`); `tails` becomes a table with one row per β. The spread of each row (largest minus smallest) is tiny where the loop converged and large where it ends in a cycle. The check requires a spread below 10^{-8} for every $\beta < 0.72$ and above 0.05 for every $\beta > 0.77$ (close to β_{max} the convergence or divergence is too slow to decide in 600 passes, so a small gap is left).

```
fig, ax = plt.subplots()
for beta, tail in zip(beta_scan, tails):
    ax.plot(np.full(len(tail), beta), tail, "k.", ms=2)
ax.axvline(beta_max, color="tab:red", ls="--", label=f"$\\beta_{{max}} = "
           f"{beta_max:.4f}$")
```

For every β the loop draws its 30 kept inputs as small black dots (`"k."`) above it: `np.full( len(tail), beta)` is a list of 30 copies of β, the horizontal coordinates. A red dashed vertical line marks β_{max}; its label is made of two f-strings written next to each other (the doubled braces print one brace each, so the label shows $\beta_{max} = 0.7441$).

```
ax.set_xlabel("mixing parameter $\\beta$")
ax.set_ylabel("last 30 inputs $n_L$")
ax.set_title("Where the loop ends, against $\\beta$")
ax.legend()
save_figure(fig, "long_run_diagram",
```

Labels, title, legend and `save_figure` for Figure 13d.5.

What Figure 13d.5 shows. Left of the red line all 30 dots of each β lie on one point at 1.327: the loop has converged. Right of it the dots split into two branches, the two values of the cycle of period 2; just beyond β_{max} they are close to 1.327, and they move apart as β grows, to about 0.36 and 1.92 at $\beta = 1$ (plain iteration). This splitting of one fixed point into a cycle of period 2 is called a **period-doubling bifurcation**.

In [12], a weaker repulsion and the threshold.

```
x2 = fixed_point(u=2.0)
g2 = slope(x2, u=2.0)
```

The fixed point and the slope for $U = 2$ (the keyword `u=2.0` replaces the default repulsion).

```
report("U = 2: n_L*", f"{1.0 + x2:.6f}")
report("U = 2: G'(x*)", f"{g2:.6f}")
```

```
report("U = 2: beta_max", f"{2.0 / (1.0 - g2):.6f}")
check(abs(x2 - 0.468990) < 1e-6 and abs(g2 + 0.688942) < 1e-6
      and abs(2.0 / (1.0 - g2) - 1.184174) < 1e-6,
      "U = 2: x* = 0.468990, G' = -0.688942, beta_max = 1.184174")
```

The three `report` lines print $n_L^* = 1.468990$, $G' = -0.688942$ and $\beta_{max} = 1.184174$, and the check compares them with these values (to 10^{-6}). Then

```
n = 2.0
for _ in range(200):  # plain iteration with U = 2
    n = loop_pass(n, u=2.0)
check(abs(n - 1.0 - x2) < 1e-10, "U = 2: plain iteration converges")
```

runs 200 passes of plain iteration at $U = 2$ and checks that they reach the fixed point to 10^{-10}.

```
U_scan = np.linspace(0.5, 8.0, 76)
beta_limits = np.array([2.0 / (1.0 - slope(fixed_point(u=u), u=u)) for u in U_scan])
low, high = 2.0, 4.0  # beta_max(2) > 1 > beta_max(4)
for _ in range(60):
    middle = 0.5 * (low + high)
    if 2.0 / (1.0 - slope(fixed_point(u=middle), u=middle)) > 1.0:
        low = middle
    else:
        high = middle
U_critical = 0.5 * (low + high)
```

β_{max} for 76 repulsions from 0.5 to 8 (for each one the fixed point is found again). Then a bisection in U between 2 (where $\beta_{max} > 1$) and 4 (where it is below 1) finds the repulsion at which $\beta_{max} = 1$, that is $G'(x^*) = -1$ (if β_{max} at the middle is still above 1, the threshold lies above the middle, so `low` moves up).

```
report("repulsion where plain iteration stops converging", f"{U_critical:.6f}")
check(abs(slope(fixed_point(u=U_critical), u=U_critical) + 1.0) < 1e-9,
      "at this repulsion the slope at the fixed point is exactly -1")
```

The `report` line prints $U = 2.647393$, and the check confirms that the slope at the fixed point is -1 there, to 10^{-9}.

In [13], the threshold as a picture.

```
fig, ax = plt.subplots()
ax.plot(U_scan, beta_limits, color="black", lw=2.0, label="$\\beta_{max}(U)$")
ax.axhline(1.0, color="gray", ls="--", label="plain iteration, $\\beta = 1$")
ax.axvline(U_critical, color="tab:red", ls=":",
           label=f"$U = {U_critical:.3f}$: plain iteration fails beyond")
ax.set_xlabel("repulsion $U$ (units of $t$)")
ax.set_ylabel("largest convergent mixing parameter")
ax.set_title("Stronger repulsion needs gentler mixing")
ax.legend(fontsize=8)
save_figure(fig, "threshold_versus_u",
```

The curve $\beta_{max}(U)$ as a thick black line, the grey dashed line $\beta = 1$ (plain iteration), a red dotted vertical line at the critical repulsion (its label shows $U = 2.647$), labels, title, legend and `save_figure` for Figure 13d.6.

What Figure 13d.6 shows. β_{max} falls steadily from about 1.8 at $U = 0.5$ to about 0.42 at $U = 8$. It crosses the line $\beta = 1$ at $U = 2.647$: for weaker repulsion plain iteration converges (the curve lies above the dashed line), for stronger repulsion it fails, and the mixing step must be smaller the stronger the repulsion.

In [14], the last check.

```
figure_names = ["reduced_map", "cobwebs", "error_histories", "convergence_factor",
                "long_run_diagram", "threshold_versus_u"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13d_{k}_{name}.png").is_file()]
check(missing == [], "all six figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13d_6_threshold_versus_u.png").is_file(),
      "the figure file 13d_6_threshold_versus_u.png exists")
all_checks_passed()
```

The same lines as In [19] of Notebook 13c (Section 13.14), with the six figure names of this notebook. The last line prints ALL 17 CHECKS PASSED (notebook 13d): one check in In [2], two each in In [3], In [5] and In [6], four in In [8], one in In [11], three in In [12] and two in In [14].

13.26 Finite temperature: Mermin's theorem

So far the system was in its ground state. At a temperature $T > 0$ it is in a statistical mixture of states. (Boltzmann's constant is 1, so a temperature is an energy.)

Density operators. A mixture in which the state Ψ_k occurs with the probability $w_k \geq 0$ ($\sum_k w_k = 1$) is described by the **density operator** $\hat{\rho} = \sum_k w_k \, |\Psi_k\rangle\langle\Psi_k|$, where $|\Psi\rangle\langle\Psi|$ is the operator that maps a state χ to $\Psi \, \langle\Psi|\chi\rangle$ (on a grid: the column Ψ times the conjugated row $\Psi^\dagger$). It is Hermitian, $\langle\chi|\hat{\rho}\chi\rangle = \sum_k w_k |\langle\Psi_k|\chi\rangle|^2 \geq 0$ for every state, and $\mathrm{Tr}\,\hat{\rho} = \sum_k w_k = 1$ for normalised Ψ_k; so its eigenvalues p_i lie between 0 and 1 and add up to 1. The expectation value of an observable is $\mathrm{Tr}(\hat{\rho}A)$. A **function of a Hermitian matrix** $A = \sum_i a_i \, u_i u_i^\dagger$ (eigenvalues a_i, orthonormal eigenvectors u_i) is defined through its eigenvalues, $f(A) = \sum_i f(a_i) \, u_i u_i^\dagger$; for $f = \exp$ this agrees with the power series, because $A^m = \sum_i a_i^m u_i u_i^\dagger$. The **entropy** is

$$S = -\mathrm{Tr}(\hat{\rho} \ln \hat{\rho}) = -\sum_i p_i \ln p_i \;\geq 0,$$

with the rule $0 \ln 0 = 0$ (the limit of $p \ln p$ as $p \to 0$); every term is ≥ 0 because $\ln p_i \leq 0$. When the particle number is fixed only on average by a **chemical potential** μ, the equilibrium state at the temperature T minimises the **grand potential**

$$\Omega[\hat{\rho}] = \mathrm{Tr}\big[\hat{\rho}\,(\hat{H} - \mu\hat{N})\big] - T\,S[\hat{\rho}].$$

Theorem (Gibbs principle). In a finite-dimensional state space Ω has exactly one minimiser, the **Gibbs state** $\hat{\rho}_0 = e^{-(\hat{H}-\mu\hat{N})/T}/Z$ with $Z = \mathrm{Tr}\, e^{-(\hat{H}-\mu\hat{N})/T}$, and $\Omega[\hat{\rho}_0] = -T \ln Z$.

Proof, line by line. Taking the logarithm of $\hat\rho_0$ (a function of the Hermitian matrix $\hat H-\mu\hat N$),

$$\ln\hat\rho_0=-\frac{\hat H-\mu\hat N}{T}-\ln Z\quad\Longrightarrow\quad \hat H-\mu\hat N=-T\ln\hat\rho_0-T\ln Z.$$

Inserting this into Ω and using $\operatorname{Tr}\hat\rho=1$:

$$\Omega[\hat\rho]=-T\operatorname{Tr}(\hat\rho\ln\hat\rho_0)-T\ln Z+T\operatorname{Tr}(\hat\rho\ln\hat\rho)=-T\ln Z+T\,D,\qquad D=\operatorname{Tr}\big[\hat\rho\,(\ln\hat\rho-\ln\hat\rho_0)\big].$$

For $\hat\rho=\hat\rho_0$ the number D is 0, so $\Omega[\hat\rho_0]=-T\ln Z$. It remains to show **Klein's inequality** $D\ge 0$ for two density operators, with $D=0$ only when they are equal. Write $\hat\rho=\sum_i p_i|i\rangle\langle i|$ and $\hat\rho_0=\sum_j q_j|j\rangle\langle j|$ with orthonormal eigenvectors (all $q_j>0$). Then

$$\begin{aligned}D&=\sum_i p_i\ln p_i-\sum_{i,j}p_i\,|\langle i|j\rangle|^2\ln q_j=\sum_{i,j}|\langle i|j\rangle|^2\,p_i\,(\ln p_i-\ln q_j)\\&\ge\sum_{i,j}|\langle i|j\rangle|^2\,(p_i-q_j)=\sum_i p_i-\sum_j q_j=1-1=0.\end{aligned}$$

The first step evaluates the two traces in the eigenvectors of $\hat\rho$ (and expands $|i\rangle$ in the eigenvectors of $\hat\rho_0$ for the second); the second inserts $\sum_j|\langle i|j\rangle|^2=1$ (completeness) in the first sum; the inequality uses $a\ln a-a\ln b\ge a-b$ for $a\ge0$, $b>0$, which is $\ln y\le y-1$ with $y=b/a$, multiplied by $-a$ (for $a=0$ it reads $0\ge -b$); the last steps use completeness once more and the traces 1. The inequality $\ln y\le y-1$ holds because $y-1-\ln y$ has the derivative $1-1/y$, negative for $y<1$ and positive for $y>1$, so its smallest value is 0 at $y=1$, and only there. Hence $D=0$ forces $p_i=q_j$ whenever $\langle i|j\rangle\neq0$, which makes $\hat\rho$ and $\hat\rho_0$ act in the same way on every $|j\rangle$: $\hat\rho=\hat\rho_0$. □

Notebook 13e (In [3] and In [4]) builds the Gibbs state of the two-site model, checks $\Omega[\hat\rho_0]=-T\ln Z$, and finds for 2000 random density operators that Ω is always larger, with $\Omega-\Omega[\hat\rho_0]=T\,D$ to 10^{-10} (Figure 13e.1).

Mermin's theorem (1965), line by line. Fix the temperature $T>0$, the chemical potential μ and the interaction $\hat W$, and work, as in the Gibbs principle, in a finite-dimensional state space that holds every particle number (for example fermions on the points of a grid, with $0,1,2,\dots$ particles). For an external potential v write $\hat H_v=\hat T+\hat W+\hat V$, its grand potential $\Omega_v[\hat\rho]=\operatorname{Tr}[\hat\rho\,(\hat H_v-\mu\hat N)]-T\,S[\hat\rho]$, its Gibbs state $\hat\rho_v$, and n_v for the density of $\hat\rho_v$, the **equilibrium density**. The density of a mixture $\hat\rho=\sum_k w_k|\Psi_k\rangle\langle\Psi_k|$ is $n_{\hat\rho}=\sum_k w_k\,n_k$, where n_k is the density of Ψ_k, and

$$\operatorname{Tr}(\hat\rho\,\hat V)=\sum_k w_k\,\langle\Psi_k|\hat V\Psi_k\rangle=\sum_k w_k\int v\,n_k\,dx=\int v\,n_{\hat\rho}\,dx.$$

The first step uses that the trace of $|\Psi\rangle\langle\Psi|\hat V$ (on a grid: the column Ψ times the row $\Psi^\dagger\hat V$) is the number $\langle\Psi|\hat V\Psi\rangle$; the second is the rule of Section 13.3 that an external energy needs only the density, applied to each Ψ_k; the third exchanges the sum and the integral. With $v=1$ the operator $\hat V$ is $\hat N$, so also $\operatorname{Tr}(\hat\rho\,\hat N)=\int n_{\hat\rho}\,dx$.

Theorem (Mermin). If v and v' differ at some point, then $n_v\neq n_{v'}$: at a fixed T and μ the equilibrium density determines the external potential.

Proof. *Step 1: the two Gibbs states differ.* Suppose $\hat\rho_v = \hat\rho_{v'}$. Then their logarithms are equal, and by the first line of the proof of the Gibbs principle, $-(\hat H_v - \mu\hat N)/T - \ln Z = -(\hat H_{v'} - \mu\hat N)/T - \ln Z'$; multiplying by $-T$ and cancelling $\hat T + \hat W - \mu\hat N$ on both sides, $\hat V - \hat V' = T\,(\ln Z' - \ln Z)\cdot 1 = c\cdot 1$, a number c times the unit operator. Applied to the state with no particle, $\hat V - \hat V'$ gives 0 (with no particle there is no potential energy), so $c = 0$. Applied to the state with one particle at the grid point x_k, it gives $v(x_k) - v'(x_k)$ times that state, so $v(x_k) = v'(x_k)$ at every point, against the assumption. Hence $\hat\rho_v \neq \hat\rho_{v'}$. *Step 2: two strict inequalities.* Since $\hat H_v - \mu\hat N = (\hat H_{v'} - \mu\hat N) + (\hat V - \hat V')$ and the entropy does not depend on the potential, the rule above gives, for every density operator, $\Omega_v[\hat\rho] = \Omega_{v'}[\hat\rho] + \int (v - v')\, n_{\hat\rho}\, dx$. Therefore

$$\Omega_v[\hat\rho_v] < \Omega_v[\hat\rho_{v'}] = \Omega_{v'}[\hat\rho_{v'}] + \int (v - v')\, n_{v'}\, dx.$$

The first step is the Gibbs principle: $\hat\rho_v$ is the only minimiser of Ω_v, and $\hat\rho_{v'}$ is a different density operator (step 1), so its grand potential is strictly larger; the second step is the identity just stated, with $\hat\rho = \hat\rho_{v'}$. Exchanging the roles of v and v' gives in the same way

$$\Omega_{v'}[\hat\rho_{v'}] < \Omega_{v'}[\hat\rho_v] = \Omega_v[\hat\rho_v] + \int (v' - v)\, n_v\, dx.$$

Step 3: the contradiction. Suppose $n_v = n_{v'}$. Adding the two inequalities, the two integrals cancel, and $\Omega_v[\hat\rho_v] + \Omega_{v'}[\hat\rho_{v'}] < \Omega_v[\hat\rho_v] + \Omega_{v'}[\hat\rho_{v'}]$, which is false. Hence $n_v \neq n_{v'}$. □

Two differences from the Hohenberg-Kohn theorem: no non-degeneracy and no non-vanishing wave function is needed, because the Gibbs state is always the only minimiser and step 1 uses only the states with zero and one particle; and the potential is determined completely, not only up to a constant, because at a fixed μ a constant added to v changes the equilibrium state (it acts like a change of μ).

The universal functional at a temperature. For every density n of some density operator define, as in the constrained search of Section 13.8,

$$F_T[n] = \min_{\hat\rho\to n} \Big(\mathrm{Tr}[\hat\rho\,(\hat T + \hat W)] - T\,S[\hat\rho]\Big),$$

the smallest value among all density operators with the density n (we ASSUME, as there, that the minimum is attained). It depends on T but not on v or μ. Then

$$\Omega_v[\hat\rho_v] = \min_n \Big(F_T[n] + \int (v - \mu)\, n\, dx\Big).$$

Proof: by the Gibbs principle the left side is the minimum of Ω_v over all density operators. Organise the minimisation in two steps, first over the $\hat\rho$ with a given density n, then over n. For all these $\hat\rho$ the part $\mathrm{Tr}[\hat\rho\,(\hat V - \mu\hat N)] = \int (v - \mu)\, n\, dx$ is the same (the rule above, and its case $v = 1$), so the inner minimum is $F_T[n] + \int (v - \mu)\, n\, dx$; the outer minimum over n is then the minimum over all $\hat\rho$. □ This is the foundation of DFT at a temperature.

Non-interacting fermions: independent orbitals, line by line. For non-interacting fermions with orbital energies ϵ_a, $\hat H - \mu\hat N = \sum_a (\epsilon_a - \mu)\,\hat n_a$, and every occupation-number state $|n_0 n_1 \cdots\rangle$ is an eigenstate with the eigenvalue $\sum_a (\epsilon_a - \mu) n_a$. Its Gibbs weight is

$$\frac{e^{-\sum_a(\epsilon_a-\mu)n_a/T}}{Z} = \prod_a \frac{e^{-(\epsilon_a-\mu)n_a/T}}{1+e^{-(\epsilon_a-\mu)/T}}, \qquad Z = \prod_a \left(1+e^{-(\epsilon_a-\mu)/T}\right),$$

because the exponential of a sum is the product of the exponentials, and the sum over all choices $n_a \in \{0,1\}$ of a product of factors, one per orbital, is the product of the sums of each factor over $n_a = 0, 1$. So the configuration $\{n_a\}$ has the probability $\prod_a p_a(n_a)$ with

$$p_a(1) = \frac{e^{-(\epsilon_a-\mu)/T}}{1+e^{-(\epsilon_a-\mu)/T}} = \frac{1}{e^{(\epsilon_a-\mu)/T}+1} = f_a, \qquad p_a(0) = 1-f_a :$$

each orbital is an independent two-state system, occupied with the **Fermi-Dirac** probability f_a (the second step multiplies numerator and denominator by $e^{(\epsilon_a-\mu)/T}$). Because the logarithm of a product is a sum, the entropy of independent orbitals is the sum of their entropies,

$$S_s = -\sum_a \left[f_a \ln f_a + (1-f_a)\ln(1-f_a)\right].$$

Notebook 13e (In [6]) checks the product form on three orbitals: the eight Gibbs weights equal the eight products to 10^{-15}.

The Mermin-Kohn-Sham functional. The non-interacting reference system at $T > 0$ is an ensemble of orbitals ϕ_a with occupations $f_a \in [0,1]$, density $n = \sum_a f_a|\phi_a|^2$ and entropy S_s. Mermin's Kohn-Sham functional is the free energy

$$F = \sum_a f_a \Big\langle \phi_a \Big| -\frac{1}{2}\frac{d^2}{dx^2}\Big|\phi_a\Big\rangle - T\,S_s + \int v\,n\,dx + E_H[n] + F_{xc}[n],$$

with a temperature-dependent exchange-correlation part F_{xc}. Making F stationary with respect to the orbitals gives the Kohn-Sham equations of Section 13.10 unchanged. Making it stationary with respect to the occupations under the condition $\sum_a f_a = N$ (multiplier μ), line by line:

$$\frac{\partial}{\partial f_a}\Big[F - \mu\Big(\sum_b f_b - N\Big)\Big] = \epsilon_a + T\ln\frac{f_a}{1-f_a} - \mu = 0,$$
$$\ln\frac{f_a}{1-f_a} = -\frac{\epsilon_a-\mu}{T} \quad\Longrightarrow\quad f_a = \frac{1}{e^{(\epsilon_a-\mu)/T}+1}.$$

The first line uses that the energy terms change with f_a by $\langle\phi_a|-\frac{1}{2}d^2/dx^2|\phi_a\rangle + \int(v+v_H+v_{xc})|\phi_a|^2 = \epsilon_a$ (the chain rule through n), and that $\frac{d}{df}[f\ln f + (1-f)\ln(1-f)] = \ln f + 1 - \ln(1-f) - 1 = \ln\frac{f}{1-f}$; the second solves for f_a (take the exponential of both sides and solve $f/(1-f) = e^{-(\epsilon-\mu)/T}$). So the occupations are the Fermi-Dirac numbers. As $T \to 0$, $f_a \to 1$ below μ and $f_a \to 0$ above it: the aufbau rule (Figure 13e.2).

Finding μ. The particle number $\sum_a g_a f_a$ (g_a: the number of states of level a) increases strictly with μ, because each f_a does; so exactly one μ gives $\sum_a g_a f_a = N$, and **bisection** finds it: start with an interval that surely contains μ, look at its middle, keep the half in which the particle number crosses N, and repeat; every step halves the interval (Figure 13e.3). The Revision

Kohn-Sham solver fixes μ by the same condition, written in a form that loses no digits when N is large: it splits the levels at a dividing point and balances the thermally excited particles above it against the holes below it, counting a hole with $f(-x) = 1 - f(x)$ computed without a subtraction, and finds the root of this balance in a logarithmic form (named LogBalance) by Newton steps safeguarded by bisection (`Revision/kohn_sham/results/parameters.json`, conventions merminRoot and numerics merminRoot).

A form of the entropy without $\ln 0$, line by line. With $x = (\epsilon - \mu)/T$, $f = 1/(e^x + 1)$ and $1 - f = e^x/(e^x + 1)$:

$$-\ln f = \ln(1 + e^x), \qquad -\ln(1 - f) = \ln(1 + e^x) - x = \ln(1 + e^{-x}),$$
$$s(x) = -f \ln f - (1 - f)\ln(1 - f) = f \ln(1 + e^x) + (1 - f)\ln(1 + e^{-x}).$$

For $x \geq 0$ write $\ln(1+e^x) = x + \ln(1+e^{-x})$; then $s = f\,x + f\ln(1+e^{-x}) + (1-f)\ln(1+e^{-x}) = \ln(1+e^{-x}) + x\,f(x)$. Since $s(-x) = s(x)$ (exchanging f and $1-f$), for every x: $s = \ln(1+e^{-|x|}) + |x|\,f(|x|)$, a form in which no logarithm of 0 and no overflow can occur. Notebook 13e uses it.

Thermodynamics, line by line. At fixed N the energy is $E = \sum_a g_a f_a \epsilon_a$ and the free energy $F = E - TS_s$, both functions of T through the occupations and μ. (i) $dF/dT = -S_s$ (the **envelope theorem**). F depends on T explicitly (the factor T in $-TS_s$) and through the occupations:

$$\frac{dF}{dT} = \frac{\partial F}{\partial T}\Big|_f + \sum_a \frac{\partial F}{\partial f_a}\,\frac{df_a}{dT} = -S_s + \mu \sum_a g_a \frac{df_a}{dT} = -S_s + \mu\,\frac{dN}{dT} = -S_s.$$

The first step is the chain rule; the second uses the stationarity condition $\partial F/\partial f_a = g_a\mu$ found above (for a level with g_a states); the third recognises the derivative of $N = \sum_a g_a f_a$; the fourth uses that N is fixed. (ii) $C_V = T\,dS_s/dT$: the **heat capacity** $C_V = dE/dT = d(F + TS_s)/dT = -S_s + S_s + T\,dS_s/dT$. (iii) **The variance formula.** With $f = 1/(e^x + 1)$, $df/dx = -f(1-f)$, and $x = (\epsilon - \mu)/T$ changes with T by $dx/dT = -(\epsilon - \mu)/T^2 - \mu'/T$ ($\mu' = d\mu/dT$). So, with $w_a = g_a f_a (1 - f_a)$,

$$g_a \frac{df_a}{dT} = w_a\Big[\frac{\epsilon_a - \mu}{T^2} + \frac{\mu'}{T}\Big], \qquad 0 = \frac{dN}{dT} = \sum_a w_a\Big[\frac{\epsilon_a - \mu}{T^2} + \frac{\mu'}{T}\Big] \;\Rightarrow\; \frac{\mu'}{T} = -\frac{\sum_a w_a(\epsilon_a - \mu)}{T^2 \sum_a w_a},$$
$$C_V = \sum_a \epsilon_a\, g_a \frac{df_a}{dT} = \sum_a (\epsilon_a - \mu)\, g_a \frac{df_a}{dT} = \frac{1}{T^2}\Big[\sum_a w_a(\epsilon_a - \mu)^2 - \frac{\big(\sum_a w_a(\epsilon_a - \mu)\big)^2}{\sum_a w_a}\Big].$$

The first line is the chain rule and the condition of fixed N, solved for μ'; in the second line the second step subtracts $\mu \sum_a g_a\, df_a/dT = \mu\, dN/dT = 0$, and the third inserts the first line and μ'. The bracket is $\sum_a w_a$ times the **weighted variance** of the numbers $\epsilon_a - \mu$ with the weights w_a, which is never negative (it equals $\sum_a w_a(\epsilon_a - \bar\epsilon)^2$ with the weighted mean $\bar\epsilon$, a sum of non-negative terms). Hence $C_V \geq 0$.

Worked example: two levels, one particle. Levels $\epsilon_0 = 0$ and $\epsilon_1 = 1$ (one state each), one particle, $T = \frac{1}{2}$, no interaction. By symmetry $\mu = \frac{1}{2}$: then $f_0 + f_1 = \frac{1}{e^{-1}+1} + \frac{1}{e^{1}+1} = 1$ (the two fractions add to 1, as one sees by multiplying the first by e/e). So $f_0 = 1/(e^{-1} + 1) = 0.731059$, $f_1 = 0.268941$, $E = f_1\epsilon_1 = 0.268941$, $S_s = 1.164406$ (each level contributes 0.582203), and

$F = E - TS_s = 0.268941 - 0.582203 = -0.313262$. Since $\mu = \frac{1}{2}$ at every temperature, $E(T) = 1/(e^{1/(2T)}+1)$ and $C_V = dE/dT = \frac{e^{1/(2T)}}{(e^{1/(2T)}+1)^2}\cdot\frac{1}{2T^2}$, which at $T = \frac{1}{2}$ is $2e/(e+1)^2 = 0.393224$. Notebook 13e (In [8] and In [10]) computes all these numbers and checks $dF/dT = -S_s$ by a difference quotient.

13.27 Excited states: the Kohn-Sham gap, Janak's theorem and Delta-SCF

The ground-state theory says nothing directly about excited states. This section introduces the particle-hole energies, the Kohn-Sham gap, Janak's theorem and the Delta-SCF method. The Revision Kohn-Sham solver reports three of these quantities for dirac16complex, the particle-hole energies, the Kohn-Sham gap and the Delta-SCF excitation energy (the folder `Revision/kohn_sham/results/excited`); Janak's theorem is the exact statement that connects the last two.

Particle-hole excitations and the Kohn-Sham gap. In a non-interacting system, moving one particle from an occupied orbital i (it leaves a **hole**) to an empty orbital a (a **particle**) costs exactly $\epsilon_a - \epsilon_i$, because the levels do not depend on the occupations. The smallest such cost is the **Kohn-Sham gap**

$$\Delta_{KS} = \epsilon_{LUMO} - \epsilon_{HOMO},$$

where HOMO is the highest occupied orbital and LUMO the lowest unoccupied orbital (names taken over from chemistry). In the interacting system the Kohn-Sham levels move when the occupations change, so Δ_{KS} is only a first estimate of the lowest excitation energy.

Janak's theorem. Let the energy $E(\{f\})$ be evaluated with orbitals that are self-consistent for the given occupations. Then

$$\frac{\partial E}{\partial f_a} = \epsilon_a.$$

Proof. $E = \sum_b f_b\langle\phi_b| -\tfrac{1}{2}d^2/dx^2|\phi_b\rangle + E_{pot}[n]$, where E_{pot} collects $\int v\,n$, E_H and E_{xc}, and $n = \sum_b f_b|\phi_b|^2$. E depends on f_a explicitly and through the orbitals. The explicit derivative is $\langle\phi_a| -\tfrac{1}{2}d^2/dx^2|\phi_a\rangle + \int(\delta E_{pot}/\delta n)|\phi_a|^2 = \langle\phi_a|h_s\phi_a\rangle = \epsilon_a$, with the Kohn-Sham Hamiltonian h_s. The change through the orbitals is $\sum_b f_b\big(\langle d\phi_b|h_s\phi_b\rangle + \langle h_s\phi_b|d\phi_b\rangle\big) = \sum_b f_b\,\epsilon_b\,d\langle\phi_b|\phi_b\rangle = 0$, because the Kohn-Sham equations $h_s\phi_b = \epsilon_b\phi_b$ hold and the orbitals stay normalised ($\langle\phi_b|\phi_b\rangle = 1$ does not change). $\square$

Delta-SCF. The **Delta-SCF** method computes an excited state as a second self-consistent solution in which the occupations are fixed by hand: one particle is taken out of the HOMO and put into the LUMO, and the Kohn-Sham loop is run again with these occupations. The excitation energy is the difference of the two energies, $\Delta_{SCF} = E_1 - E_0$. Janak's theorem connects it with the gap: move the particle gradually, $f_{HOMO} = 1-\tau$ and $f_{LUMO} = \tau$; then by the chain rule $dE/d\tau = \partial E/\partial f_{LUMO} - \partial E/\partial f_{HOMO} = \epsilon_{LUMO}(\tau) - \epsilon_{HOMO}(\tau)$, and integrating from $\tau = 0$ to 1,

$$\Delta_{SCF} = \int_0^1 \left[\epsilon_{LUMO}(\tau) - \epsilon_{HOMO}(\tau)\right] d\tau.$$

At $\tau = 0$ the integrand is Δ_{KS}; the difference $\Delta_{SCF} - \Delta_{KS}$ measures how much the levels move when the particle is transferred (the **orbital relaxation**). The midpoint value $\epsilon_{LUMO}(\frac{1}{2}) - \epsilon_{HOMO}(\frac{1}{2})$ (Slater's **transition state**) is a good estimate of the integral when the integrand is nearly a straight line in τ. Janak's theorem and the integral formula are PROVED; that the Delta-SCF state approximates a true excited state of the interacting system is an ASSUMPTION of the method. The Revision solver uses an **ensemble** form: one particle is moved from the highest occupied group of equal levels to the lowest empty group, spread evenly over each group, which keeps the symmetries of its reduction (`Revision/kohn_sham/results/parameters.json`, conventions deltaScf).

Toy example. Take the model energy $E(f_H, f_L) = \epsilon_H^0 f_H + \epsilon_L^0 f_L + \frac{U}{2}(f_H^2 + f_L^2)$ with $\epsilon_H^0 = 0$, $\epsilon_L^0 = 1$, $U = 0.2$ (the last term plays the role of a self-interaction of each orbital). Janak's theorem gives the levels $\epsilon_H = \partial E/\partial f_H = U f_H$ and $\epsilon_L = 1 + U f_L$. In the ground state $(f_H, f_L) = (1, 0)$: $E_0 = 0.1$, $\epsilon_H = 0.2$, $\epsilon_L = 1$, so $\Delta_{KS} = 0.8$. In the Delta-SCF state $(0, 1)$: $E_1 = 1 + 0.1 = 1.1$, so $\Delta_{SCF} = 1.0$. The integral formula reproduces it: $\int_0^1 [1 + U\tau - U(1-\tau)]\, d\tau = \int_0^1 (0.8 + 0.4\tau)\, d\tau = 0.8 + 0.2 = 1$. Here the relaxation raises the excitation energy by $U = 0.2$ above the gap, and the transition state is exact because the integrand is a straight line. (In the trap of Notebook 13a, Section 13.32, the relaxation lowers it instead: its sign depends on the system.)

13.28 Example: Mermin thermodynamics and Delta-SCF

Notebook 13e checks every exact statement of Sections 13.26 and 13.27 with numbers: the Gibbs principle and Klein's inequality on the 16 states of the two-site model with 2000 random density operators, the independence of the orbitals of non-interacting fermions, the Fermi-Dirac function, the bisection for μ (and the rule by which the Revision solver fixes μ, read from its parameter file), the thermodynamics of two levels (the worked numbers 0.731059, 1.164406, −0.313262, 0.393224) and of a ladder of levels, and Janak's theorem with the Delta-SCF integral on the toy model energy. Its statements are PROVED above; its numbers are toy numbers (no Revision number is reproduced). It ends with ALL 24 CHECKS PASSED (notebook 13e) and draws seven figures.

13.29 How to run Notebook 13e

Step 1. What this notebook does and what it needs.

Notebook 13e (Finite temperature (Mermin) and excited states (Delta-SCF)) is the file `Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb` of the repository Dirac_claude. It checks the Gibbs principle and Klein's inequality on the 16 states of the two-site model with 2000 random density operators, shows that the Gibbs state of non-interacting fermions factorises into independent orbitals with Fermi-Dirac occupations, finds the chemical potential by bisection (and reads the rule by which the Revision Kohn-Sham solver fixes it), computes the free energy, energy, entropy and heat capacity of a two-level system (the worked numbers 0.731059, 1.164406, -0.313262, 0.393224) and of a ladder of levels with checks of dF/dT = -S and of the variance formula for the heat capacity, and checks Janak's theorem and the Delta-SCF integral formula on a model energy, with seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the

program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13e_mermin_delta_scf.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13e_mermin_delta_scf.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 13e_mermin_delta_scf.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/13e.captions.json`
- `Revision/textbook/figures/13e_1_gibbs_principle.png`
- `Revision/textbook/figures/13e_2_fermi_dirac.png`
- `Revision/textbook/figures/13e_3_bisection.png`
- `Revision/textbook/figures/13e_4_two_level_thermo.png`
- `Revision/textbook/figures/13e_5_ladder_occupations.png`
- `Revision/textbook/figures/13e_6_ladder_thermo.png`
- `Revision/textbook/figures/13e_7_janak_delta_scf.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13e_7_janak_delta_scf.png exists
ALL 24 CHECKS PASSED (notebook 13e)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 13e_mermin_delta_scf.ipynb
python -m nbconvert --execute --inplace 13e_mermin_delta_scf.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `parameters.json`: the notebook reads the file `Revision/kohn_sham/results/parameters.json` of the repository; it must be opened inside the folder `Revision/textbook/notebooks` of a complete clone of the repository, not as a single downloaded file.

13.30 Notebook 13e: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Density functional theory at a temperature $T > 0$ (Mermin's theorem) and for
excited states (Delta-SCF) rests on a few exact statements. This notebook checks
each of them with numbers. It

- checks the Gibbs principle: among all density operators of the two-site model,
  the Gibbs state $e^{-(\hat H - \mu\hat N)/T}/Z$ has the lowest grand potential
  $\Omega$, and $\Omega - \Omega_{Gibbs} = T\,D$ with Klein's $D \ge 0$ (2000 random
  density operators);
- checks that the Gibbs state of non-interacting fermions is a product of
  independent orbitals, each occupied with the Fermi-Dirac probability;
- draws the Fermi-Dirac function and finds the chemical potential by bisection;
- computes the thermodynamics of a two-level system (the worked example of the
  chapter) and of a ladder of levels: free energy, energy, entropy, heat capacity,
  with the checks $dF/dT = -S$ and $C_V = T\,dS/dT = dE/dT \ge 0$;
- checks Janak's theorem $\partial E/\partial f_a = \epsilon_a$ and the Delta-SCF
  formula $\Delta_{SCF} = \int_0^1 [\epsilon_L(\tau) - \epsilon_H(\tau)]\,d\tau$ on a
  model energy;
- draws seven teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 13e (Finite temperature (Mermin) and excited states (Delta-SCF)) is the file
`Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb` of the repository Dirac_claude.
It checks the Gibbs principle and Klein's inequality on the 16 states of the two-site
model with 2000 random density operators, shows that the Gibbs state of non-interacting
fermions factorises into independent orbitals with Fermi-Dirac occupations, finds the
chemical potential by bisection (and reads the rule by which the Revision Kohn-Sham
solver fixes it), computes the free energy, energy, entropy and heat capacity of a
two-level system (the worked numbers 0.731059, 1.164406, -0.313262, 0.393224) and of a
ladder of levels with checks of dF/dT = -S and of the variance formula for the heat
capacity, and checks Janak's theorem and the Delta-SCF integral formula on a model
energy, with seven teaching plots. It needs a computer with Windows 11, macOS or Linux,
an internet connection for the installation, the program Git, and Python 3.12 or newer
(the notebooks were built with Python 3.14.5) with these packages at exactly these
versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10,
nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook
itself imports numpy and matplotlib; the other packages run Jupyter, the program that
shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
```

default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
```

```
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 13e_mermin_delta_scf.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 13e_mermin_delta_scf.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 13e_mermin_delta_scf.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/13e.captions.json‘
- ‘Revision/textbook/figures/13e_1_gibbs_principle.png‘
- ‘Revision/textbook/figures/13e_2_fermi_dirac.png‘
- ‘Revision/textbook/figures/13e_3_bisection.png‘
- ‘Revision/textbook/figures/13e_4_two_level_thermo.png‘
- ‘Revision/textbook/figures/13e_5_ladder_occupations.png‘
- ‘Revision/textbook/figures/13e_6_ladder_thermo.png‘
- ‘Revision/textbook/figures/13e_7_janak_delta_scf.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it

runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13e_7_janak_delta_scf.png exists
ALL 24 CHECKS PASSED (notebook 13e)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 13e_mermin_delta_scf.ipynb
  python -m nbconvert --execute --inplace 13e_mermin_delta_scf.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues

  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for ‘parameters.json‘: the notebook reads the file
  ‘Revision/kohn_sham/results/parameters.json‘ of the repository; it must be opened
  inside the folder ‘Revision/textbook/notebooks‘ of a complete clone of the repository,
  not as a single downloaded file.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 13e (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 13e (Finite temperature (Mermin) and excited states (Delta-SCF)) is the file
# Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb of the repository Dirac_claude.
# It checks the Gibbs principle and Klein's inequality on the 16 states of the two-site
# model with 2000 random density operators, shows that the Gibbs state of non-interacting
# fermions factorises into independent orbitals with Fermi-Dirac occupations, finds the
# chemical potential by bisection (and reads the rule by which the Revision Kohn-Sham
# solver fixes it), computes the free energy, energy, entropy and heat capacity of a
# two-level system (the worked numbers 0.731059, 1.164406, -0.313262, 0.393224) and of a
# ladder of levels with checks of dF/dT = -S and of the variance formula for the heat
# capacity, and checks Janak's theorem and the Delta-SCF integral formula on a model
# energy, with seven teaching plots. It needs a computer with Windows 11, macOS or Linux,
# an internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
```

```
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 13e_mermin_delta_scf.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
```

```
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 13e_mermin_delta_scf.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 13e_mermin_delta_scf.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/13e.captions.json
# - Revision/textbook/figures/13e_1_gibbs_principle.png
# - Revision/textbook/figures/13e_2_fermi_dirac.png
# - Revision/textbook/figures/13e_3_bisection.png
# - Revision/textbook/figures/13e_4_two_level_thermo.png
# - Revision/textbook/figures/13e_5_ladder_occupations.png
# - Revision/textbook/figures/13e_6_ladder_thermo.png
# - Revision/textbook/figures/13e_7_janak_delta_scf.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 13e_7_janak_delta_scf.png exists
#     ALL 24 CHECKS PASSED (notebook 13e)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
```

```
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/13e_mermin_delta_scf.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 13e_mermin_delta_scf.ipynb
#     python -m nbconvert --execute --inplace 13e_mermin_delta_scf.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for parameters.json: the notebook reads the file
#   Revision/kohn_sham/results/parameters.json of the repository; it must be opened
#   inside the folder Revision/textbook/notebooks of a complete clone of the repository,
#   not as a single downloaded file.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

```
NOTEBOOK_ID = "13e"  # this notebook: chapter 13, example e


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
```

```
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 13e complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Temperature** $T$: measured as an energy (Boltzmann's constant is set to 1).
- **Density operator** $\hat\rho$: a Hermitian matrix with eigenvalues between 0 and
  1 that add up to 1; it describes a mixture of states with these probabilities.
- **Entropy** $S = -\mathrm{Tr}(\hat\rho\ln\hat\rho) = -\sum_i p_i\ln p_i$, where the
  $p_i$ are the eigenvalues of $\hat\rho$ (with $0\ln 0 = 0$).
- **Chemical potential** $\mu$: the energy that fixes the average particle number.
- **Grand potential** $\Omega[\hat\rho] = \mathrm{Tr}[\hat\rho(\hat H - \mu\hat N)]
  - T S$; **Gibbs state** $\hat\rho_0 = e^{-(\hat H - \mu\hat N)/T}/Z$ with
  $Z = \mathrm{Tr}\,e^{-(\hat H - \mu\hat N)/T}$; $\Omega[\hat\rho_0] = -T\ln Z$.
- **Klein's inequality**: $D = \mathrm{Tr}[\hat\rho(\ln\hat\rho -
  \ln\hat\sigma)] \ge 0$ for two density operators, with equality only when they are
  equal.
- **Fermi-Dirac occupation** $f(\epsilon) = 1/(e^{(\epsilon - \mu)/T} + 1)$: the
  probability that an orbital of energy $\epsilon$ is occupied.
- **Free energy** $F = E - TS$; **heat capacity** $C_V = dE/dT$ at a fixed particle
  number.
- **Bisection**: halving an interval that contains the root until it is small.
- **Janak's theorem**: the derivative of the self-consistent energy with respect to
  the occupation $f_a$ of an orbital is its level $\epsilon_a$.
- **Delta-SCF**: the excitation energy as the difference of two self-consistent
  energies, with one particle moved from the highest occupied orbital (H) to the
  lowest empty one (L).
```

Text cell:

```
## 4. The physical and mathematical situation

**Gibbs principle.** For every density operator $\hat\rho$ and the Gibbs state
$\hat\rho_0$: since $\hat H - \mu\hat N = -T\ln\hat\rho_0 - T\ln Z$,

$$\Omega[\hat\rho] = -T\ln Z + T\,\mathrm{Tr}[\hat\rho(\ln\hat\rho -
\ln\hat\rho_0)] = \Omega[\hat\rho_0] + T\,D \ge \Omega[\hat\rho_0] .$$

**Non-interacting fermions.** If $\hat H - \mu\hat N = \sum_a (\epsilon_a - \mu)\hat
n_a$, the weight of the configuration $(n_0, n_1, \dots)$ in the Gibbs state is the
product $\prod_a p_a(n_a)$ with $p_a(1) = f_a$, $p_a(0) = 1 - f_a$: the orbitals are
independent. The entropy is then $S = -\sum_a [f_a\ln f_a + (1 - f_a)\ln(1 - f_a)]$.

**Fixed particle number.** $\sum_a g_a f_a = N$ ($g_a$: the number of states of
level $a$) increases strictly with $\mu$, so exactly one $\mu$ solves it; bisection
finds it. Then $E = \sum g_a f_a\epsilon_a$, $F = E - TS$, and because $F$ is
stationary in the occupations (envelope theorem), $dF/dT = -S$ and
$C_V = dE/dT = T\,dS/dT$. With $w_a = g_a f_a(1 - f_a)$ one finds
```

```
$C_V = \frac{1}{T^2}\big[\sum_a w_a(\epsilon_a - \mu)^2 - (\sum_a w_a(\epsilon_a -
\mu))^2/\sum_a w_a\big] \ge 0$ (a weighted variance).

**Janak and Delta-SCF.** For the model energy
$E(f_H, f_L) = \epsilon_H^0 f_H + \epsilon_L^0 f_L + \tfrac{U}{2}(f_H^2 + f_L^2)$
the levels are $\epsilon_a = \partial E/\partial f_a = \epsilon_a^0 + U f_a$. Moving
a fraction $\tau$ from H to L ($f_H = 1 - \tau$, $f_L = \tau$) gives
$dE/d\tau = \epsilon_L(\tau) - \epsilon_H(\tau)$, so
$\Delta_{SCF} = E(0, 1) - E(1, 0) = \int_0^1[\epsilon_L(\tau) - \epsilon_H(\tau)]
\,d\tau$.

**Status.** Exact finite-dimensional statements (PROVED: they follow from the
definitions; this notebook checks each one to rounding); toy numbers, no Revision
number is reproduced. The Revision Kohn-Sham solver uses the same Mermin
occupations with $\mu$ fixed by $\sum g f = N$; section 8 reads this rule from the
solver's parameter file. That a Delta-SCF state approximates a true excited state
is an ASSUMPTION of the method; Janak's theorem itself is exact.
```

Text cell:

```
## 5. The Gibbs principle on the two-site model

The next cell builds the 16 occupation-number states of four orbitals (L up, R up, L
down, R down), the annihilation operators as $16 \times 16$ matrices (with the sign
$(-1)^{\nu_p}$), the two-site Hamiltonian with hopping $t = 1$ and repulsion
$U = 2$, and the particle-number operator.
```

In [2]:

```
import itertools  # loops over combinations

import numpy as np  # arrays, matrices and linear algebra

M, DIM = 4, 16  # four orbitals, sixteen occupation-number states
a = []  # annihilation operators
for p in range(M):
    matrix = np.zeros((DIM, DIM))
    for state in range(DIM):
        if (state >> p) & 1:  # orbital p is occupied in this state
            nu = sum((state >> q) & 1 for q in range(p))  # occupied before p
            matrix[state ^ (1 << p), state] = (-1) ** nu
    a.append(matrix)
a_dag = [matrix.T for matrix in a]
n_op = [a_dag[p] @ a[p] for p in range(M)]
N_op = sum(n_op)  # the particle-number operator
H = -(a_dag[0] @ a[1] + a_dag[1] @ a[0] + a_dag[2] @ a[3] + a_dag[3] @ a[2]) \
    + 2.0 * (n_op[0] @ n_op[2] + n_op[1] @ n_op[3])  # t = 1, U = 2
check(np.allclose(H, H.T) and np.allclose(H @ N_op, N_op @ H),
      "H is symmetric and conserves the particle number")
```

Out [2]:

```
PASS H is symmetric and conserves the particle number
```

Text cell:

```
The next cell defines the matrix function $f(A) = \sum_i f(a_i)\,u_i u_i^\dagger$
through the eigenvalues $a_i$ and eigenvectors $u_i$ of a Hermitian matrix, the
entropy, and the grand potential. It builds the Gibbs state at $\mu = 1$, $T = 0.5$
and checks $\Omega[\hat\rho_0] = -T\ln Z$.
```

In [3]:

```
MU, TEMPERATURE = 1.0, 0.5
K = H - MU * N_op  # H - mu N


def entropy(rho):
    """S = -sum p ln p over the eigenvalues p of rho (0 ln 0 = 0)."""
    p = np.clip(np.linalg.eigvalsh(rho), 0.0, None)
    p = p[p > 1e-300]
    return float(-np.sum(p * np.log(p)))


def grand_potential(rho):
    """Omega = Tr[rho (H - mu N)] - T S."""
    return float(np.trace(rho @ K).real) - TEMPERATURE * entropy(rho)


k_values, k_vectors = np.linalg.eigh(K)
boltzmann = np.exp(-(k_values - k_values.min()) / TEMPERATURE)  # shifted weights
Z_shifted = boltzmann.sum()
rho_gibbs = (k_vectors * (boltzmann / Z_shifted)) @ k_vectors.T
ln_Z = np.log(Z_shifted) - k_values.min() / TEMPERATURE  # undo the shift
omega_gibbs = grand_potential(rho_gibbs)
report("Omega of the Gibbs state", f"{omega_gibbs:.10f}")
report("-T ln Z", f"{-TEMPERATURE * ln_Z:.10f}")
check(abs(omega_gibbs + TEMPERATURE * ln_Z) < 1e-12, "Omega[Gibbs] = -T ln Z")
```

Out [3]:

```
RESULT Omega of the Gibbs state = -3.4716265446
RESULT -T ln Z = -3.4716265446
PASS Omega[Gibbs] = -T ln Z
```

Text cell:

```
The next cell makes 2000 random density operators: a random complex matrix $A$
gives the density operator $AA^\dagger/\mathrm{Tr}(AA^\dagger)$ (Hermitian, with
non-negative eigenvalues and trace 1); mixing it with the Gibbs state with a random
weight $s$ gives states near and far from the minimum. For each it computes
$\Omega - \Omega_0$ and Klein's $D = \mathrm{Tr}[\hat\rho(\ln\hat\rho -
\ln\hat\rho_0)]$, and checks $\Omega - \Omega_0 = T D \ge 0$.
```

In [4]:

```
ln_rho_gibbs = (k_vectors * (-k_values / TEMPERATURE - ln_Z)) @ k_vectors.T


def matrix_log(rho):
    """ln rho through the eigenvalues (all positive here)."""
    values, vectors = np.linalg.eigh(rho)
    return (vectors * np.log(values)) @ vectors.conj().T


rng = np.random.default_rng(12345)  # fixed seed: the same numbers every run
gaps, kleins = [], []
for _ in range(2000):
    A = rng.normal(size=(DIM, DIM)) + 1j * rng.normal(size=(DIM, DIM))
    random_rho = A @ A.conj().T
    random_rho /= np.trace(random_rho).real
    s = rng.uniform(0.0, 1.0)  # how far from the Gibbs state
    rho = (1.0 - s) * rho_gibbs + s * random_rho
    gaps.append(grand_potential(rho) - omega_gibbs)
    kleins.append(np.trace(rho @ (matrix_log(rho) - ln_rho_gibbs)).real)
gaps, kleins = np.array(gaps), np.array(kleins)
say(f"smallest Omega - Omega_0 among 2000 states: {gaps.min():.3e}")
check(np.all(gaps > 0.0), "every random density operator has Omega > Omega[Gibbs]")
check(np.max(np.abs(gaps - TEMPERATURE * kleins)) < 1e-10,
      "Omega - Omega[Gibbs] = T D with Klein's D (identity)")
```

Out [4]:

```
smallest Omega - Omega_0 among 2000 states: 3.878e-06
PASS every random density operator has Omega > Omega[Gibbs]
PASS Omega - Omega[Gibbs] = T D with Klein's D (identity)
```

Text cell:

```
The next cell draws the distribution of $\Omega - \Omega_0$ of the 2000 states as a
histogram on a logarithmic axis.
```

In [5]:

```
fig, ax = plt.subplots()
bins = np.logspace(np.floor(np.log10(gaps.min())), np.ceil(np.log10(gaps.max())), 40)
ax.hist(gaps, bins=bins, color="tab:blue", edgecolor="black", lw=0.5)
ax.set_xscale("log")
ax.set_xlabel("$\\Omega[\\hat\\rho] - \\Omega[\\hat\\rho_0]$ (units of $t$)")
ax.set_ylabel("number of random states")
ax.set_title("The Gibbs state has the lowest grand potential")
save_figure(fig, "gibbs_principle",
            "Histogram of the grand potential of 2000 random density operators of "
            "the two-site model ($t = 1$, $U = 2$, $\\mu = 1$, $T = 0.5$) minus "
```

```
"that of the Gibbs state, on a logarithmic horizontal axis (units of "
"$t$); the vertical axis counts the states in each bin. All 2000 "
"differences are positive: no state beats the Gibbs state, as the "
"Gibbs principle and Klein's inequality say. States mixed only "
"slightly away from the Gibbs state have the smallest differences.")
```

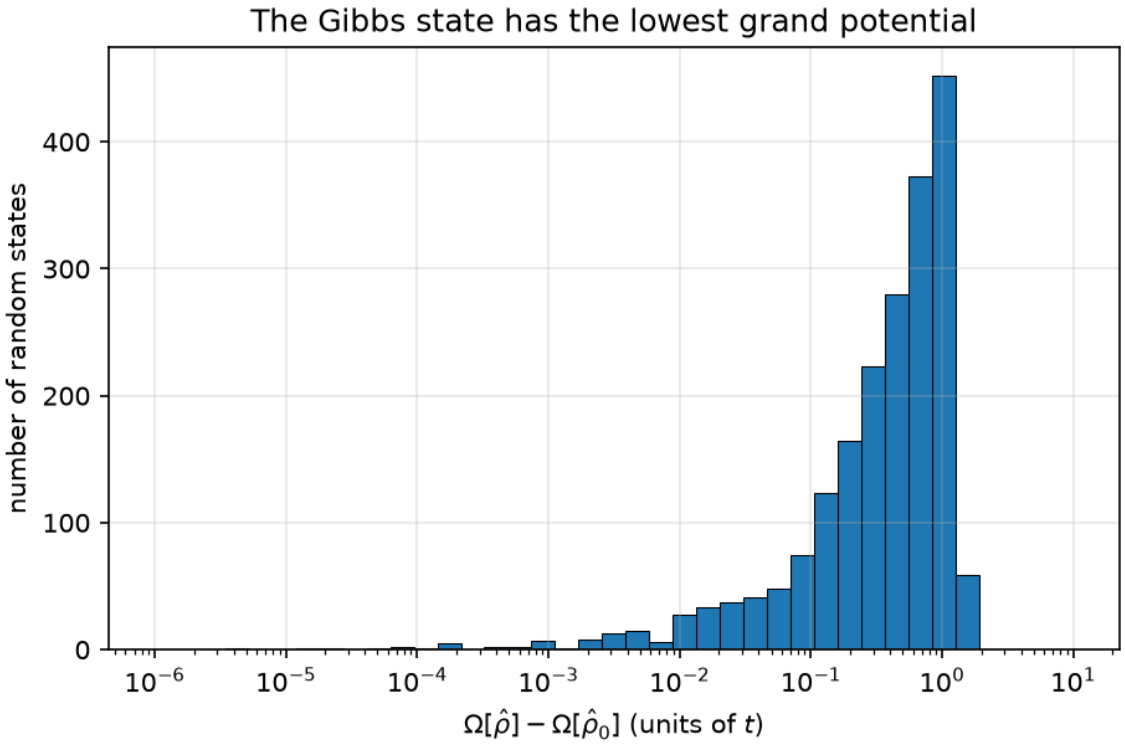

Figure 367: Histogram of the grand potential of 2000 random density operators of the two-site model ($t = 1$, $U = 2$, $\mu = 1$, $T = 0.5$) minus that of the Gibbs state, on a logarithmic horizontal axis (units of t); the vertical axis counts the states in each bin. All 2000 differences are positive: no state beats the Gibbs state, as the Gibbs principle and Klein's inequality say. States mixed only slightly away from the Gibbs state have the smallest differences. (Notebook 13e, figure 1.)

Out [5]:

```
Figure 13e.1 saved as Revision/textbook/figures/13e_1_gibbs_principle.png
```

Text cell:

```
## 6. Non-interacting fermions: independent orbitals

The next cell takes three orbitals with energies $-0.5, 0.2, 1.0$, $\mu = 0.1$,
$T = 0.4$, and compares the Gibbs weight $e^{-\sum_a(\epsilon_a - \mu)n_a/T}/Z$ of
each of the 8 configurations with the product of the single-orbital probabilities
$f_a$ or $1 - f_a$.
```

In [6]:

```
levels3 = np.array([-0.5, 0.2, 1.0])
mu3, T3 = 0.1, 0.4
configurations = list(itertools.product((0, 1), repeat=3))
weights = np.array([np.exp(-np.dot(levels3 - mu3, c) / T3) for c in configurations])
weights /= weights.sum()  # divide by Z
f3 = 1.0 / (np.exp((levels3 - mu3) / T3) + 1.0)
products = np.array([np.prod([f3[k] if c[k] else 1.0 - f3[k] for k in range(3)])
                     for c in configurations])
for c, wgt, prod in zip(configurations, weights, products):
    say(f"occupations {c}: Gibbs weight {wgt:.6f}, product {prod:.6f}")
check(np.max(np.abs(weights - products)) < 1e-15,
      "the Gibbs weights are products of independent Fermi-Dirac probabilities")
```

Out [6]:

```
occupations (0, 0, 0): Gibbs weight 0.092777, product 0.092777
occupations (0, 0, 1): Gibbs weight 0.009779, product 0.009779
occupations (0, 1, 0): Gibbs weight 0.072255, product 0.072255
occupations (0, 1, 1): Gibbs weight 0.007616, product 0.007616
occupations (1, 0, 0): Gibbs weight 0.415797, product 0.415797
occupations (1, 0, 1): Gibbs weight 0.043825, product 0.043825
occupations (1, 1, 0): Gibbs weight 0.323823, product 0.323823
occupations (1, 1, 1): Gibbs weight 0.034131, product 0.034131
PASS the Gibbs weights are products of independent Fermi-Dirac probabilities
```

Text cell:

```
## 7. The Fermi-Dirac function

The next cell defines $f$ in a form that a computer can evaluate for every
argument: $e^{x}$ overflows (exceeds the largest number the computer can store,
about $10^{308}$) for $x > 709$, so the cell uses $f = e^{-\ln(1 + e^{x})}$ with
numpy's `logaddexp(0, x)` $= \ln(e^0 + e^x)$, which never overflows. It draws
$f(\epsilon)$ for four temperatures around $\mu = 0$ and checks $f(\mu) = 1/2$ and
the symmetry $f(\mu + \delta) = 1 - f(\mu - \delta)$ (a hole below $\mu$ is as
likely as a particle above it).
```

In [7]:

```
def fermi(energy, mu, temperature):
    """The Fermi-Dirac occupation 1 / (exp(x) + 1), x = (energy - mu)/T, written
    as exp(-ln(1 + e^x)) so that no overflow can occur."""
    x = (np.asarray(energy, dtype=float) - mu) / temperature
    return np.exp(-np.logaddexp(0.0, x))


eps = np.linspace(-2.0, 2.0, 801)
check(abs(fermi(0.0, 0.0, 0.3) - 0.5) < 1e-15
      and np.allclose(fermi(eps, 0.0, 0.3), 1.0 - fermi(-eps, 0.0, 0.3), atol=1e-15),
      "f(mu) = 1/2 and f(mu + d) = 1 - f(mu - d)")
fig, ax = plt.subplots()
```

```
for temperature in (0.02, 0.1, 0.3, 1.0):
    ax.plot(eps, fermi(eps, 0.0, temperature), label=f"$T = {temperature}$")
ax.axvline(0.0, color="gray", ls="--", lw=0.8)
ax.set_xlabel("orbital energy $\\epsilon - \\mu$")
ax.set_ylabel("occupation $f$")
ax.set_title("The Fermi-Dirac occupation")
ax.legend()
save_figure(fig, "fermi_dirac",
            "The Fermi-Dirac occupation $f = 1/(e^{(\\epsilon - \\mu)/T} + 1)$ "
            "against the orbital energy measured from the chemical potential, for "
            "four temperatures (energies and temperatures in the same units). At "
            "low $T$ it is a step (occupied below $\\mu$, empty above: the aufbau "
            "rule); a higher $T$ smears the step over a width of a few $T$; at "
            "$\\epsilon = \\mu$ it is always 1/2.")
```

Out [7]:

```
PASS f(mu) = 1/2 and f(mu + d) = 1 - f(mu - d)
```

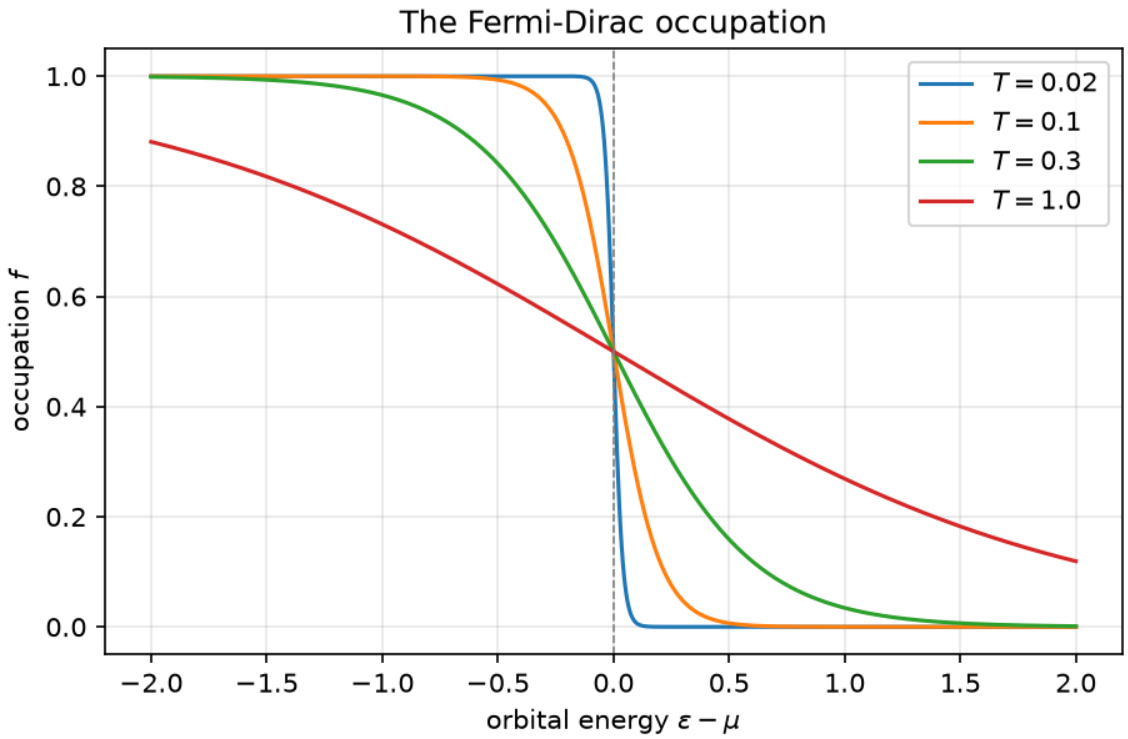

Figure 368: The Fermi-Dirac occupation $f = 1/(e^{(\epsilon-\mu)/T} + 1)$ against the orbital energy measured from the chemical potential, for four temperatures (energies and temperatures in the same units). At low T it is a step (occupied below μ, empty above: the aufbau rule); a higher T smears the step over a width of a few T; at $\epsilon = \mu$ it is always 1/2. (Notebook 13e, figure 2.)

Out [7]:

```
Figure 13e.2 saved as Revision/textbook/figures/13e_2_fermi_dirac.png
```

Text cell:

```
## 8. Two levels, one particle: the chemical potential by bisection

Two levels $\epsilon_0 = 0$, $\epsilon_1 = 1$ (one state each), one particle,
$T = 1/2$. The next cell finds $\mu$ by bisection on $f_0 + f_1 - 1$, recording the
width of the interval at every step, and computes the worked numbers of the
chapter. The bisection starts from an interval that surely contains $\mu$: from the
lowest level minus $50T + 5$ to the highest level plus 5, here $[-30, 6]$. At its
lower end every occupation is below $e^{-50}$, so the levels hold fewer than $N$
particles; at its upper end they hold almost all their states (here nearly 2),
more than $N = 1$. The entropy of one state of energy
$\epsilon$ is $-f\ln f - (1 - f)\ln(1 - f)$; with $x = (\epsilon - \mu)/T$ it equals
$\ln(1 + e^{-|x|}) + |x|\,f(|x|)$ (insert $f = 1/(e^x + 1)$ and simplify), a form
without $\ln 0$ that the cell uses.
```

In [8]:

```
def chemical_potential(levels, degeneracies, N, temperature, history=None):
    """mu with sum g f = N, by bisection (the sum increases with mu)."""
    low, high = levels.min() - 50.0 * temperature - 5.0, levels.max() + 5.0
    for _ in range(100):
        middle = 0.5 * (low + high)
        if np.sum(degeneracies * fermi(levels, middle, temperature)) < N:
            low = middle  # too few particles: mu must rise
        else:
            high = middle
        if history is not None:
            history.append(high - low)
    return 0.5 * (low + high)


def thermodynamics(levels, degeneracies, N, temperature):
    """mu, occupations, E, S, F of non-interacting levels at fixed N."""
    mu = chemical_potential(levels, degeneracies, N, temperature)
    f = fermi(levels, mu, temperature)
    x = np.abs(levels - mu) / temperature
    s_level = np.logaddexp(0.0, -x) + x * fermi(x, 0.0, 1.0)  # entropy of a state
    S = np.sum(degeneracies * s_level)
    E = np.sum(degeneracies * f * levels)
    return mu, f, E, S, E - temperature * S


two_levels, ones = np.array([0.0, 1.0]), np.array([1.0, 1.0])
widths = []
chemical_potential(two_levels, ones, 1.0, 0.5, widths)
mu2, f2, E2, S2, F2 = thermodynamics(two_levels, ones, 1.0, 0.5)
report("mu", f"{mu2:.6f}")
report("f_0, f_1", f"{f2[0]:.6f}, {f2[1]:.6f}")
report("E, S, F", f"{E2:.6f}, {S2:.6f}, {F2:.6f}")
check(abs(mu2 - 0.5) < 1e-12 and abs(f2[0] - 0.731059) < 1e-6
      and abs(f2[1] - 0.268941) < 1e-6,
```

```
      "mu = 1/2, f_0 = 0.731059, f_1 = 0.268941")
check(abs(E2 - 0.268941) < 1e-6 and abs(S2 - 1.164406) < 1e-6
      and abs(F2 + 0.313262) < 1e-6, "E = 0.268941, S = 1.164406, F = -0.313262")
```

Out [8]:

```
RESULT mu = 0.500000
RESULT f_0, f_1 = 0.731059, 0.268941
RESULT E, S, F = 0.268941, 1.164406, -0.313262
PASS mu = 1/2, f_0 = 0.731059, f_1 = 0.268941
PASS E = 0.268941, S = 1.164406, F = -0.313262
```

Text cell:

```
The Revision Kohn-Sham solver of the dirac16complex field fixes its chemical
potential by the same condition $\sum g f = N$. The next cell reads how from the
solver's parameter file. With many particles the direct difference
$\sum g f - N$ loses digits (two large, nearly equal numbers are subtracted), so the
solver splits the levels at a dividing point and balances the thermally excited
particles above it against the holes below it; a hole is counted with $f(-x)$,
which equals $1 - f(x)$ (section 7) but is computed without a subtraction. It
finds the root of this balance, written with logarithms (the form it calls
LogBalance), by Newton steps safeguarded by bisection.
```

In [9]:

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
root_rule = parameters["conventions"]["merminRoot"]  # a sentence of the record
root_form = parameters["numerics"]["merminRoot"]  # the name of the canonical form
say(f"Revision solver: canonical root form {root_form}")
check(root_rule.startswith("mu from sum g f = N")
      and "holes below a split of the levels" in root_rule
      and root_form == "LogBalance",
      "the Revision solver fixes mu by sum g f = N, balancing particles and holes "
      "(Revision/kohn_sham/results/parameters.json, conventions merminRoot)")
```

Out [9]:

```
Revision solver: canonical root form LogBalance
PASS the Revision solver fixes mu by sum g f = N, balancing particles and holes
    (Revision/kohn_sham/results/parameters.json, conventions merminRoot)
```

Text cell:

```
The next cell computes $dF/dT$ and $dE/dT$ by central differences (with $\mu$ solved
again at $T \pm 10^{-4}$), checks $dF/dT = -S$ and $C_V = dE/dT = T\,dS/dT$, and draws
the bisection history.
```

In [10]:

```
def at(temperature, levels=two_levels, degeneracies=ones, N=1.0):
    return thermodynamics(levels, degeneracies, N, temperature)


d = 1e-4
dF_dT = (at(0.5 + d)[4] - at(0.5 - d)[4]) / (2 * d)
dE_dT = (at(0.5 + d)[2] - at(0.5 - d)[2]) / (2 * d)
T_dS_dT = 0.5 * (at(0.5 + d)[3] - at(0.5 - d)[3]) / (2 * d)
report("dF/dT", f"{dF_dT:.6f}")
report("C_V = dE/dT", f"{dE_dT:.6f}")
check(abs(dF_dT + S2) < 1e-7, "dF/dT = -S (envelope theorem)")
check(abs(dE_dT - 0.393224) < 1e-6 and abs(T_dS_dT - dE_dT) < 1e-7,
      "C_V = dE/dT = T dS/dT = 0.393224")
fig, ax = plt.subplots()
ax.semilogy(range(1, 51), widths[:50], "o-", ms=3)
ax.set_xlabel("bisection step")
ax.set_ylabel("width of the interval that contains $\\mu$")
ax.set_title("Bisection halves the interval at every step")
save_figure(fig, "bisection",
            "The width of the interval that contains the chemical potential during "
            "the bisection for two levels and one particle at $T = 1/2$, against "
            "the step number, on a logarithmic vertical axis (energy units). Every "
            "step halves the width (a straight line that falls by "
            "$\\log_{10} 2 = 0.30$ per step), from 18 after the first step to about "
            "$3 \\times 10^{-14}$ after 50 steps, which fixes $\\mu$ to 13 decimal "
            "places.")
check(all(abs(b / a_ - 0.5) < 1e-12 for a_, b in zip(widths[:40], widths[1:41])),
      "every bisection step halves the interval")
```

Out [10]:

```
RESULT dF/dT = -1.164406
RESULT C_V = dE/dT = 0.393224
PASS dF/dT = -S (envelope theorem)
PASS C_V = dE/dT = T dS/dT = 0.393224
```

Out [10]:

```
Figure 13e.3 saved as Revision/textbook/figures/13e_3_bisection.png
PASS every bisection step halves the interval
```

Text cell:

```
## 9. The two-level system at all temperatures

The next cell computes $E$, $S$, $F$ and $C_V = T\,dS/dT$ (by central differences)
from $T = 0.02$ to $T = 3$, checks the limits ($S \to 0$ as $T \to 0$;
$S \to \ln 4$ and $E \to 1/2$ as $T \to \infty$: the four configurations become
equally likely) and draws them.
```

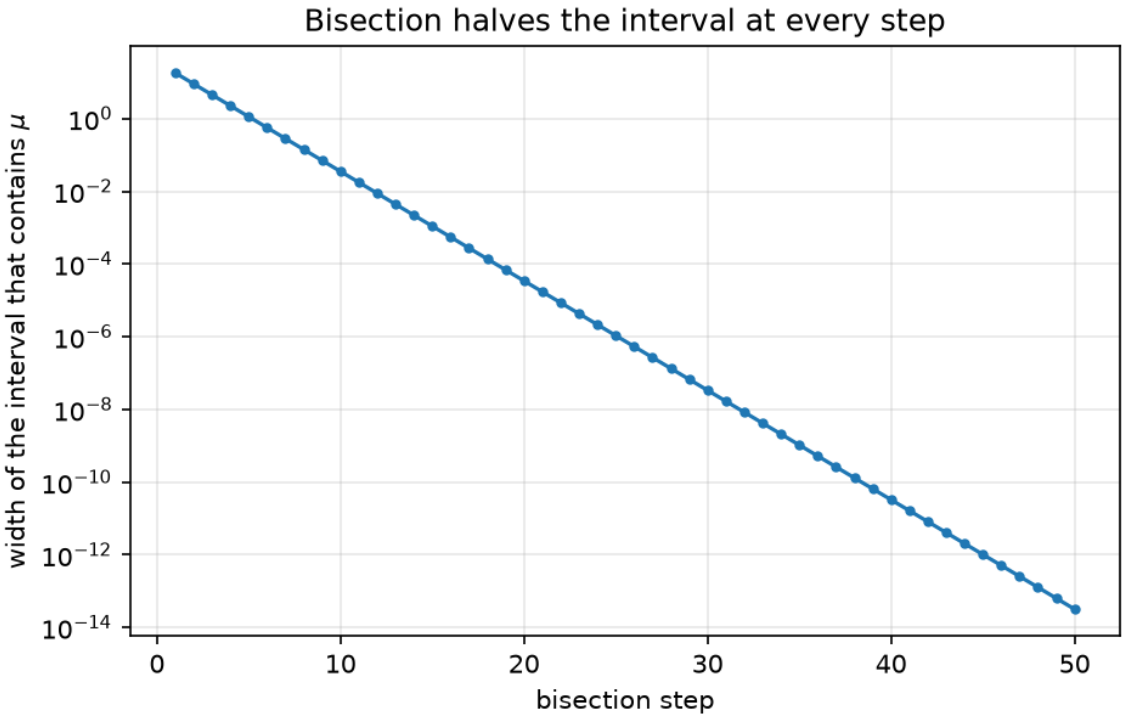

Figure 369: The width of the interval that contains the chemical potential during the bisection for two levels and one particle at $T = 1/2$, against the step number, on a logarithmic vertical axis (energy units). Every step halves the width (a straight line that falls by $\log_{10} 2 = 0.30$ per step), from 18 after the first step to about 3×10^{-14} after 50 steps, which fixes μ to 13 decimal places. (Notebook 13e, figure 3.)

In [11]:

```
temperatures = np.linspace(0.02, 3.0, 150)
table = np.array([at(T)[2:] for T in temperatures])  # columns E, S, F
C_V = np.array([T * (at(T + d)[3] - at(T - d)[3]) / (2 * d) for T in temperatures])
hot = at(1000.0)
check(table[0, 1] < 1e-8 and abs(hot[3] - np.log(4.0)) < 1e-5
      and abs(hot[2] - 0.5) < 1e-3, "S -> 0 for T -> 0; S -> ln 4, E -> 1/2 for T "
      "-> infinity")
check(np.all(C_V >= 0.0), "the heat capacity is never negative")
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(temperatures, table[:, 0], label="energy $E$")
left.plot(temperatures, temperatures * table[:, 1], label="$T S$")
left.plot(temperatures, table[:, 2], color="black", label="free energy $F = E - TS$")
left.set_xlabel("temperature $T$")
left.set_ylabel("energy (level-spacing units)")
left.legend(fontsize=8)
right.plot(temperatures, C_V, color="tab:red")
right.plot([0.5], [0.393224], "ko", label="$T = 1/2$: $C_V = 0.393224$")
right.set_xlabel("temperature $T$")
right.set_ylabel("heat capacity $C_V = dE/dT$")
right.legend(fontsize=8)
fig.suptitle("Two levels (0 and 1), one particle")
save_figure(fig, "two_level_thermo",
            "Thermodynamics of one particle in two levels $\\epsilon_0 = 0$, "
            "$\\epsilon_1 = 1$ against the temperature (level-spacing units). "
            "Left: the energy $E$, the product $TS$ of temperature and entropy, "
            "and the free energy $F = E - TS$, which falls with $T$ at the rate "
            "$-S$. Right: the heat capacity $C_V = dE/dT$, which has a single peak "
            "(it vanishes at low $T$, where the upper level is almost never "
            "occupied, and at "
            "high $T$, where both levels are already equally occupied); the black "
            "point is the worked value at $T = 1/2$.")
```

Out [11]:

```
PASS S -> 0 for T -> 0; S -> ln 4, E -> 1/2 for T -> infinity
PASS the heat capacity is never negative
```

Out [11]:

```
Figure 13e.4 saved as Revision/textbook/figures/13e_4_two_level_thermo.png
```

Text cell:

```
## 10. A ladder of levels

The next cell takes the levels $\epsilon_n = n + 1/2$ ($n = 0, \dots, 59$), two
states each (two labels), with $N = 8$ particles: the trap of Notebook 13a without
interaction. At low $T$ the four lowest levels are full and $\mu$ lies halfway
```

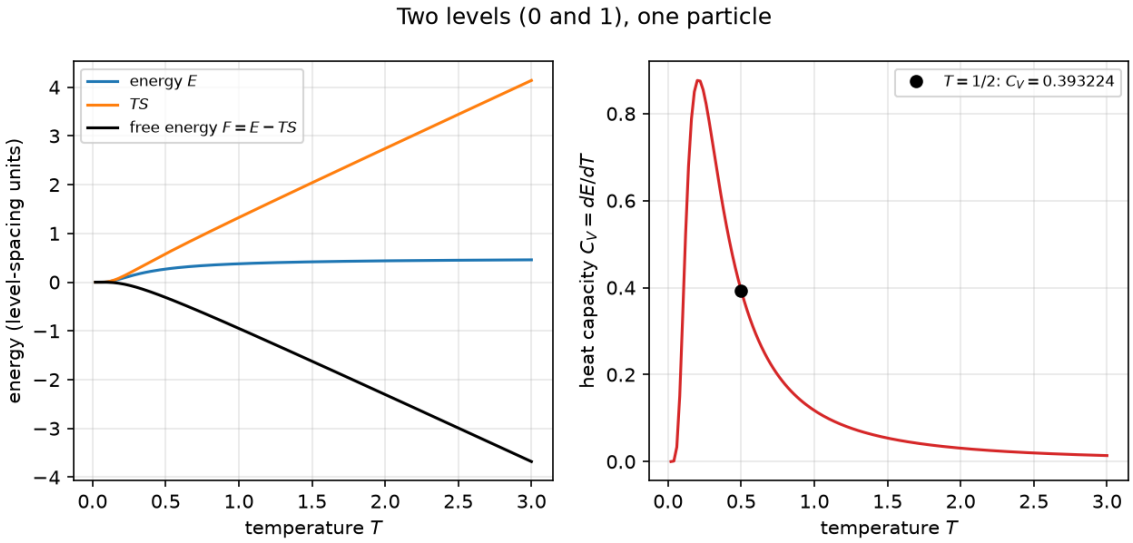

Figure 370: Thermodynamics of one particle in two levels $\epsilon_0 = 0$, $\epsilon_1 = 1$ against the temperature (level-spacing units). Left: the energy E, the product TS of temperature and entropy, and the free energy $F = E - TS$, which falls with T at the rate $-S$. Right: the heat capacity $C_V = dE/dT$, which has a single peak (it vanishes at low T, where the upper level is almost never occupied, and at high T, where both levels are already equally occupied); the black point is the worked value at $T = 1/2$. (Notebook 13e, figure 4.)

between the highest full level 3.5 and the lowest empty level 4.5. The cell computes μ, E, S and C_V (by central differences and by the variance formula of section 4) for 120 temperatures.

In [12]:

```
ladder = np.arange(60) + 0.5
twos = 2.0 * np.ones(60)
N_LADDER = 8.0


def ladder_at(T):
    return thermodynamics(ladder, twos, N_LADDER, T)


def variance_heat_capacity(T):
    """C_V = [sum w x^2 - (sum w x)^2 / sum w] / T^2, x = eps - mu, w = g f (1 - f)."""
    mu, f = ladder_at(T)[:2]
    w = twos * f * (1.0 - f)
    x = ladder - mu
    return (np.sum(w * x * x) - np.sum(w * x) ** 2 / np.sum(w)) / T ** 2


T_ladder = np.linspace(0.05, 3.0, 120)
mus = np.array([ladder_at(T)[0] for T in T_ladder])
C_numeric = np.array([(ladder_at(T + d)[2] - ladder_at(T - d)[2]) / (2 * d)
```

```
                     for T in T_ladder])
C_variance = np.array([variance_heat_capacity(T) for T in T_ladder])
report("mu at T = 0.05", f"{mus[0]:.6f}")
check(abs(mus[0] - 4.0) < 1e-6, "at low T, mu lies halfway between 3.5 and 4.5")
check(np.max(np.abs(C_numeric - C_variance)) < 1e-5 * max(1.0, C_variance.max()),
      "the heat capacity equals the weighted-variance formula")
check(np.all(C_variance >= 0.0), "the variance formula is never negative")
```

Out [12]:

```
RESULT mu at T = 0.05 = 4.000000
PASS at low T, mu lies halfway between 3.5 and 4.5
PASS the heat capacity equals the weighted-variance formula
PASS the variance formula is never negative
```

Text cell:

```
The next cell draws the occupations of the ladder at four temperatures.
```

In [13]:

```
fig, ax = plt.subplots()
for T, marker in ((0.05, "o"), (0.5, "s"), (1.0, "^"), (2.0, "v")):
    mu, f = ladder_at(T)[:2]
    ax.plot(ladder[:12], f[:12], marker + "-", ms=4,
            label=f"$T = {T}$, $\\mu = {mu:.3f}$")
ax.set_xlabel("level $\\epsilon_n = n + 1/2$")
ax.set_ylabel("occupation $f_n$ of each of the two states")
ax.set_title("Eight particles in a ladder of levels")
ax.legend(fontsize=8)
save_figure(fig, "ladder_occupations",
            "The Fermi-Dirac occupation of the levels $\\epsilon_n = n + 1/2$ (two "
            "states each) holding eight particles, at four temperatures (energy "
            "units of the level spacing); the chemical potential of each "
            "temperature is in the legend. At $T = 0.05$ the four lowest levels are "
            "full and the others empty; as $T$ grows, particles move from below "
            "$\\mu$ to above it and $\\mu$ falls.")
```

Out [13]:

```
Figure 13e.5 saved as Revision/textbook/figures/13e_5_ladder_occupations.png
```

Text cell:

```
The next cell draws $\mu$ and $C_V$ of the ladder against the temperature.
```

In [14]:

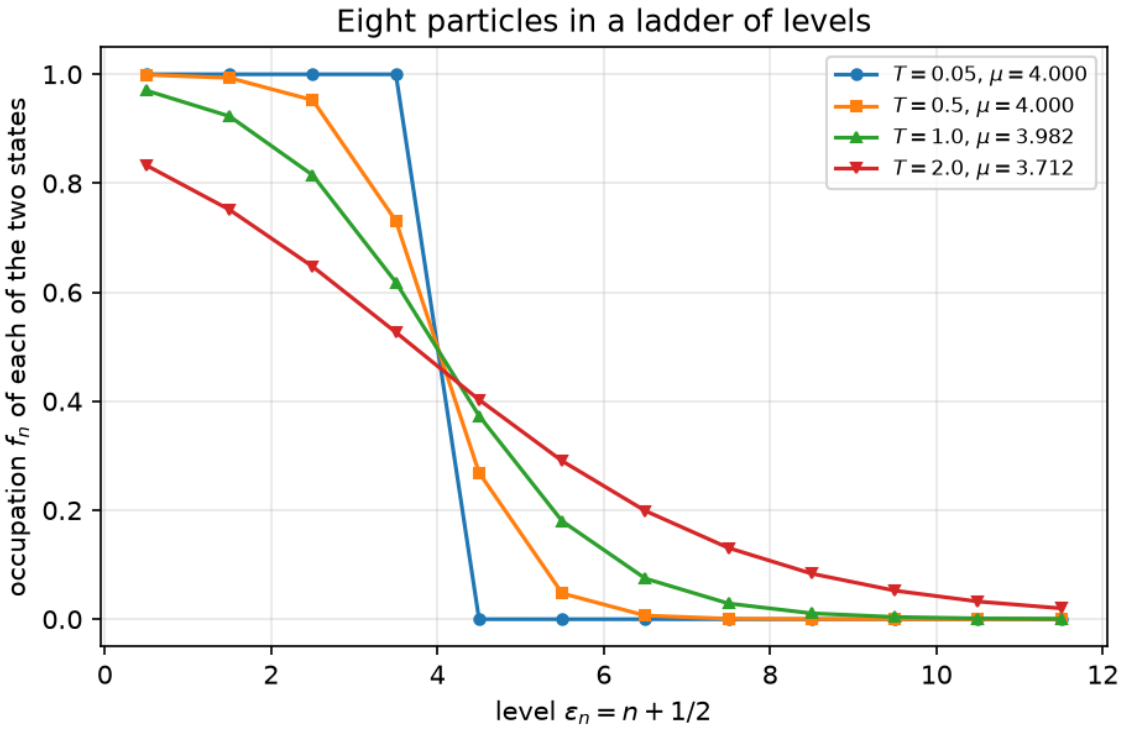

Figure 371: The Fermi-Dirac occupation of the levels $\epsilon_n = n + 1/2$ (two states each) holding eight particles, at four temperatures (energy units of the level spacing); the chemical potential of each temperature is in the legend. At $T = 0.05$ the four lowest levels are full and the others empty; as T grows, particles move from below μ to above it and μ falls. (Notebook 13e, figure 5.)

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(T_ladder, mus, color="black")
left.set_xlabel("temperature $T$")
left.set_ylabel("chemical potential $\\mu$")
right.plot(T_ladder, C_numeric, color="tab:red", label="$dE/dT$ (differences)")
right.plot(T_ladder[::6], C_variance[::6], "ko", ms=3, label="variance formula")
right.set_xlabel("temperature $T$")
right.set_ylabel("heat capacity $C_V$")
right.legend(fontsize=8)
fig.suptitle("Ladder of levels, eight particles")
save_figure(fig, "ladder_thermo",
            "Left: the chemical potential of eight particles in the ladder "
            "$\\epsilon_n = n + 1/2$ (two states per level) against the "
            "temperature; it starts at 4, halfway between the last full and the "
            "first empty level, and falls as $T$ grows. Right: the heat capacity "
            "from the difference quotient of $E(T)$ (line) and from the "
            "weighted-variance formula (points); they agree, and both are never "
            "negative. Temperatures and energies in units of the level spacing.")
```

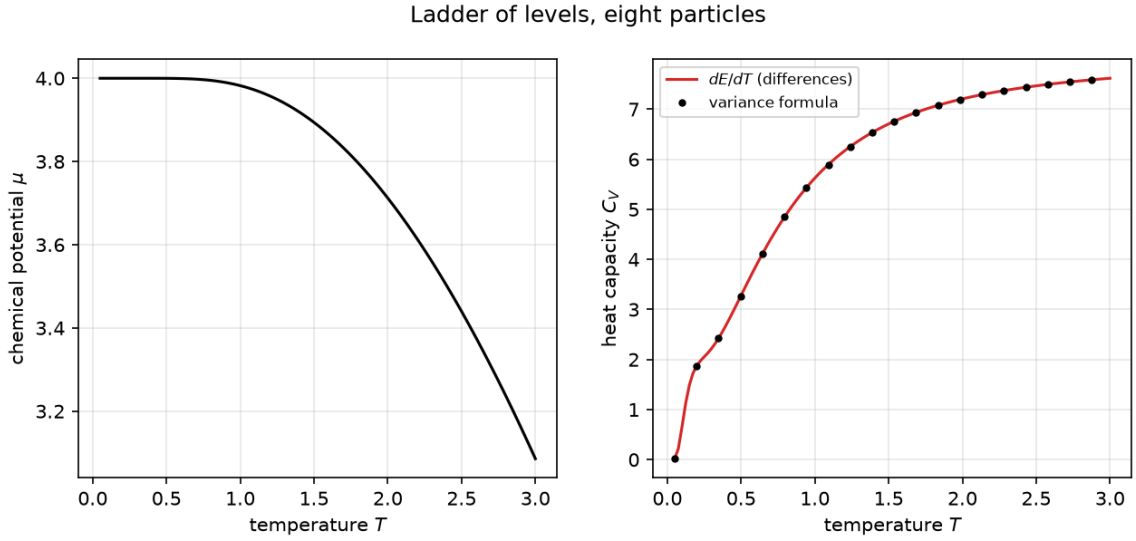

Figure 372: Left: the chemical potential of eight particles in the ladder $\epsilon_n = n + 1/2$ (two states per level) against the temperature; it starts at 4, halfway between the last full and the first empty level, and falls as T grows. Right: the heat capacity from the difference quotient of $E(T)$ (line) and from the weighted-variance formula (points); they agree, and both are never negative. Temperatures and energies in units of the level spacing. (Notebook 13e, figure 6.)

Out [14]:

```
Figure 13e.6 saved as Revision/textbook/figures/13e_6_ladder_thermo.png
```

Text cell:

```
## 11. Janak's theorem and Delta-SCF on a model energy
```

The next cell takes the model energy of section 4 with $\epsilon_H^0 = 0$, $\epsilon_L^0 = 1$, $U = 0.2$, checks Janak's theorem by difference quotients, computes the Kohn-Sham gap $\Delta_{KS} = \epsilon_L - \epsilon_H$ of the ground state $(f_H, f_L) = (1, 0)$, the Delta-SCF energy $E(0, 1) - E(1, 0)$, the integral of the level difference over τ (Simpson's rule) and the transition-state value at $\tau = 1/2$.

In [15]:

```
EPS_H0, EPS_L0, U_MODEL = 0.0, 1.0, 0.2


def model_energy(f_H, f_L):
    return EPS_H0 * f_H + EPS_L0 * f_L + 0.5 * U_MODEL * (f_H ** 2 + f_L ** 2)


def model_levels(f_H, f_L):
    """Janak: eps_a = dE/df_a = eps_a^0 + U f_a."""
    return EPS_H0 + U_MODEL * f_H, EPS_L0 + U_MODEL * f_L


h_step = 1e-6
for f_H, f_L in ((1.0, 0.0), (0.5, 0.5), (0.3, 0.9)):
    dE_dfH = (model_energy(f_H + h_step, f_L) - model_energy(f_H - h_step, f_L)) \
        / (2 * h_step)
    dE_dfL = (model_energy(f_H, f_L + h_step) - model_energy(f_H, f_L - h_step)) \
        / (2 * h_step)
    check(np.allclose((dE_dfH, dE_dfL), model_levels(f_H, f_L), atol=1e-9),
          f"Janak: dE/df_a = eps_a at (f_H, f_L) = ({f_H}, {f_L})")
taus = np.linspace(0.0, 1.0, 101)
integrand = np.array([model_levels(1 - t, t)[1] - model_levels(1 - t, t)[0]
                      for t in taus])
gap_KS = integrand[0]
delta_SCF = model_energy(0.0, 1.0) - model_energy(1.0, 0.0)
step = taus[1] - taus[0]
janak_integral = step / 3 * (integrand[0] + integrand[-1]
                             + 4 * integrand[1:-1:2].sum() + 2 * integrand[2:-1:2].sum())
report("Kohn-Sham gap", f"{gap_KS:.6f}")
report("Delta-SCF", f"{delta_SCF:.6f}")
report("integral of eps_L - eps_H over tau", f"{janak_integral:.6f}")
report("transition state (tau = 1/2)", f"{integrand[50]:.6f}")
check(abs(gap_KS - 0.8) < 1e-12 and abs(delta_SCF - 1.0) < 1e-12
      and abs(janak_integral - delta_SCF) < 1e-12 and abs(integrand[50] - 1.0) < 1e-12,
      "gap 0.8, Delta-SCF 1.0 = the Janak integral = the transition state")
check(abs(delta_SCF - gap_KS - U_MODEL) < 1e-12, "Delta-SCF - gap = U (relaxation)")
```

Out [15]:

```
PASS Janak: dE/df_a = eps_a at (f_H, f_L) = (1.0, 0.0)
PASS Janak: dE/df_a = eps_a at (f_H, f_L) = (0.5, 0.5)
PASS Janak: dE/df_a = eps_a at (f_H, f_L) = (0.3, 0.9)
RESULT Kohn-Sham gap = 0.800000
```

```
RESULT Delta-SCF = 1.000000
RESULT integral of eps_L - eps_H over tau = 1.000000
RESULT transition state (tau = 1/2) = 1.000000
PASS gap 0.8, Delta-SCF 1.0 = the Janak integral = the transition state
PASS Delta-SCF - gap = U (relaxation)
```

Text cell:

```
The next cell draws the energy along the transfer and the level difference whose
area is the Delta-SCF energy.
```

In [16]:

```
energies_tau = np.array([model_energy(1 - t, t) for t in taus])
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(taus, energies_tau - energies_tau[0], color="black")
left.set_xlabel("transferred fraction $\\tau$")
left.set_ylabel("$E(\\tau) - E(0)$")
left.set_title("energy along the transfer")
right.fill_between(taus, 0.0, integrand, alpha=0.25, label="area = Delta-SCF = 1")
right.plot(taus, integrand, color="black", label="$\\epsilon_L - \\epsilon_H$")
right.plot([0.0], [gap_KS], "s", ms=8, label="Kohn-Sham gap 0.8")
right.plot([0.5], [integrand[50]], "^", ms=8, label="transition state 1.0")
right.set_ylim(0.0, 1.3)
right.set_xlabel("transferred fraction $\\tau$")
right.set_ylabel("level difference")
right.legend(fontsize=8, loc="lower right")
fig.suptitle("Janak's theorem and Delta-SCF ($\\epsilon_H^0 = 0$, "
             "$\\epsilon_L^0 = 1$, $U = 0.2$)")
save_figure(fig, "janak_delta_scf",
            "The model energy $E = \\epsilon_H^0 f_H + \\epsilon_L^0 f_L + "
            "(U/2)(f_H^2 + f_L^2)$ when a fraction $\\tau$ of a particle is moved "
            "from H to L (left: the energy gained, rising to 1), and the level "
            "difference $\\epsilon_L - \\epsilon_H = 0.8 + 0.4\\tau$, which is the "
            "slope of the left curve by Janak's theorem (right). The Kohn-Sham gap "
            "0.8 is the value at $\\tau = 0$; the area, the Delta-SCF energy 1.0, "
            "exceeds it by $U = 0.2$ (orbital relaxation), and the midpoint value "
            "(transition state) is exact here because the line is straight.")
```

Out [16]:

```
Figure 13e.7 saved as Revision/textbook/figures/13e_7_janak_delta_scf.png
```

Text cell:

```
## 12. The last check

The last cell checks that all seven figure files exist and prints the number of
checks that passed.
```

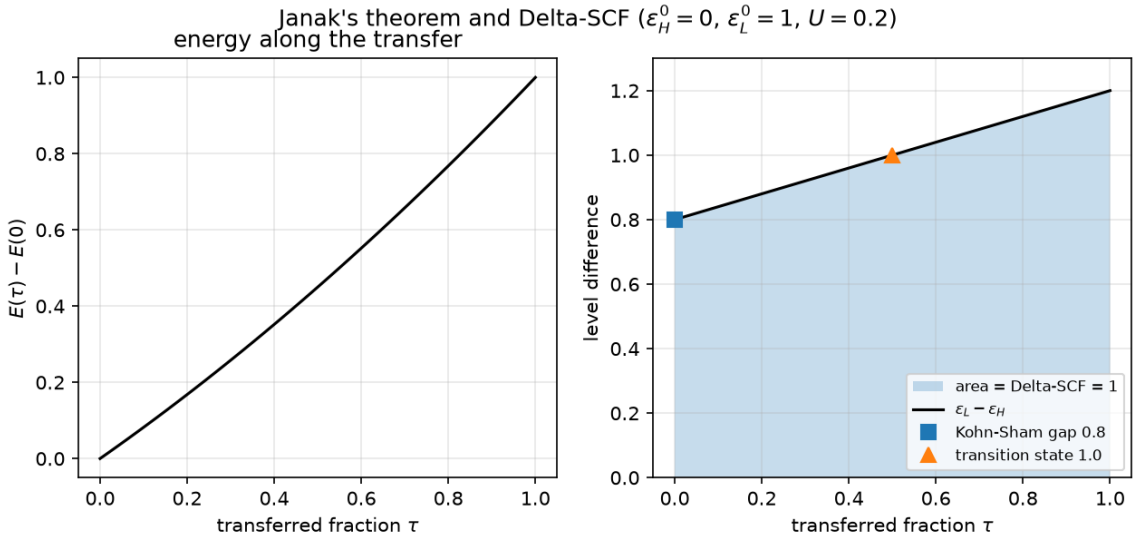

Figure 373: The model energy $E = \epsilon_H^0 f_H + \epsilon_L^0 f_L + (U/2)(f_H^2 + f_L^2)$ when a fraction τ of a particle is moved from H to L (left: the energy gained, rising to 1), and the level difference $\epsilon_L - \epsilon_H = 0.8 + 0.4\tau$, which is the slope of the left curve by Janak's theorem (right). The Kohn-Sham gap 0.8 is the value at $\tau = 0$; the area, the Delta-SCF energy 1.0, exceeds it by $U = 0.2$ (orbital relaxation), and the midpoint value (transition state) is exact here because the line is straight. (Notebook 13e, figure 7.)

In [17]:

```
figure_names = ["gibbs_principle", "fermi_dirac", "bisection", "two_level_thermo",
                "ladder_occupations", "ladder_thermo", "janak_delta_scf"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13e_{k}_{name}.png").is_file()]
check(missing == [], "all seven figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13e_7_janak_delta_scf.png").is_file(),
      "the figure file 13e_7_janak_delta_scf.png exists")
all_checks_passed()
```

Out [17]:

```
PASS all seven figure files exist
PASS the figure file 13e_7_janak_delta_scf.png exists
ALL 24 CHECKS PASSED (notebook 13e)
```

Text cell:

```
## 13. What this notebook showed

- The Gibbs state has the lowest grand potential of all density operators; the
  excess of any other state is $T$ times Klein's $D \ge 0$ (2000 random states).
  This is the variational principle behind Mermin's finite-temperature DFT.
- For non-interacting fermions the Gibbs state is a product of independent orbitals
```

```
  with the Fermi-Dirac occupations; the entropy is the sum of the orbital entropies.
- The chemical potential is found by bisection from $\sum g f = N$; every step halves
  the interval.
- Two levels at $T = 1/2$: $f_0 = 0.731059$, $E = 0.268941$, $S = 1.164406$,
  $F = -0.313262$, $dF/dT = -S$, $C_V = 0.393224$; the heat capacity has one peak.
- For a ladder of levels the heat capacity from $dE/dT$ equals the weighted-variance
  formula, so it is never negative.
- Janak's theorem holds, and the Delta-SCF excitation energy is the integral of the
  level difference over the transferred fraction; in the model it exceeds the
  Kohn-Sham gap by $U$.
```

13.31 Line-by-line walk-through of Notebook 13e

The notebook has seventeen code cells. **In [1]** is the set-up cell, identical to In [1] of Notebook 13c (Section 13.14) except for the name `"13e"` and its comment lines, which hold the instructions of Section 13.29.

In [2], the two-site model in Fock space.

```
import itertools  # loops over combinations

import numpy as np  # arrays, matrices and linear algebra

M, DIM = 4, 16  # four orbitals, sixteen occupation-number states
a = []  # annihilation operators
for p in range(M):
    matrix = np.zeros((DIM, DIM))
    for state in range(DIM):
        if (state >> p) & 1:  # orbital p is occupied in this state
            nu = sum((state >> q) & 1 for q in range(p))  # occupied before p
            matrix[state ^ (1 << p), state] = (-1) ** nu
    a.append(matrix)
a_dag = [matrix.T for matrix in a]
n_op = [a_dag[p] @ a[p] for p in range(M)]
N_op = sum(n_op)  # the particle-number operator
```

The same construction of the 16×16 operator matrices as in In [4] of Notebook 13c (Section 13.14 explains every line), with the bit test written directly instead of through a function `bit`. `n_op` are the occupation operators and `N_op` the number operator $\hat{N}$.

```
H = -(a_dag[0] @ a[1] + a_dag[1] @ a[0] + a_dag[2] @ a[3] + a_dag[3] @ a[2]) \
    + 2.0 * (n_op[0] @ n_op[2] + n_op[1] @ n_op[3])  # t = 1, U = 2
check(np.allclose(H, H.T) and np.allclose(H @ N_op, N_op @ H),
      "H is symmetric and conserves the particle number")
```

The two-site Hamiltonian with $t = 1$, $U = 2$ on all 16 states (all particle numbers from 0 to 4). The check: H is symmetric (real Hermitian) and commutes with $\hat{N}$ ($H\hat{N} = \hat{N}H$), so it never changes the particle number.

In [3], the Gibbs state.

```
MU, TEMPERATURE = 1.0, 0.5
K = H - MU * N_op  # H - mu N


def entropy(rho):
    """S = -sum p ln p over the eigenvalues p of rho (0 ln 0 = 0)."""
    p = np.clip(np.linalg.eigvalsh(rho), 0.0, None)
    p = p[p > 1e-300]
    return float(-np.sum(p * np.log(p)))


def grand_potential(rho):
    """Omega = Tr[rho (H - mu N)] - T S."""
    return float(np.trace(rho @ K).real) - TEMPERATURE * entropy(rho)
```

$\mu = 1$, $T = 0.5$ and $K = \hat{H} - \mu\hat{N}$. `entropy` computes $-\sum_i p_i \ln p_i$ from the eigenvalues of a density operator: `np.clip(..., 0.0, None)` replaces tiny negative rounding errors by 0, and `p[p > 1e-300]` drops the zeros (the rule $0 \ln 0 = 0$). `grand_potential` is $\Omega = \mathrm{Tr}(\hat{\rho}K) - TS$.

```
k_values, k_vectors = np.linalg.eigh(K)
boltzmann = np.exp(-(k_values - k_values.min()) / TEMPERATURE)  # shifted weights
Z_shifted = boltzmann.sum()
rho_gibbs = (k_vectors * (boltzmann / Z_shifted)) @ k_vectors.T
ln_Z = np.log(Z_shifted) - k_values.min() / TEMPERATURE  # undo the shift
omega_gibbs = grand_potential(rho_gibbs)
```

The Gibbs state as a function of K: eigenvalues and eigenvectors of K, the weights $e^{-k_i/T}$ shifted by the smallest eigenvalue to avoid overflow (Section 13.14 explains the shift), and $\hat{\rho}_0 = \sum_i (w_i / \sum w)\, u_i u_i^T$. The true $\ln Z$ undoes the shift: $Z = Z_{shifted}\, e^{-k_{min}/T}$, so $\ln Z = \ln Z_{shifted} - k_{min}/T$. `omega_gibbs` is $\Omega[\hat{\rho}_0]$, computed with the function above.

```
report("Omega of the Gibbs state", f"{omega_gibbs:.10f}")
report("-T ln Z", f"{-TEMPERATURE * ln_Z:.10f}")
check(abs(omega_gibbs + TEMPERATURE * ln_Z) < 1e-12, "Omega[Gibbs] = -T ln Z")
```

The two `report` lines print $\Omega[\hat{\rho}_0]$ and $-T \ln Z$, both -3.4716265446, and the check requires them to agree to 10^{-12}.

In [4], 2000 random density operators.

```
ln_rho_gibbs = (k_vectors * (-k_values / TEMPERATURE - ln_Z)) @ k_vectors.T


def matrix_log(rho):
    """ln rho through the eigenvalues (all positive here)."""
    values, vectors = np.linalg.eigh(rho)
    return (vectors * np.log(values)) @ vectors.conj().T
```

$\ln \hat{\rho}_0 = -K/T - \ln Z$, built from the eigenvectors of K; `matrix_log` is the logarithm of a Hermitian matrix with positive eigenvalues, through its eigenvalues (Section 13.26).

```
rng = np.random.default_rng(12345)  # fixed seed: the same numbers every run
gaps, kleins = [], []
for _ in range(2000):
    A = rng.normal(size=(DIM, DIM)) + 1j * rng.normal(size=(DIM, DIM))
    random_rho = A @ A.conj().T
    random_rho /= np.trace(random_rho).real
    s = rng.uniform(0.0, 1.0)  # how far from the Gibbs state
    rho = (1.0 - s) * rho_gibbs + s * random_rho
    gaps.append(grand_potential(rho) - omega_gibbs)
    kleins.append(np.trace(rho @ (matrix_log(rho) - ln_rho_gibbs)).real)
gaps, kleins = np.array(gaps), np.array(kleins)
```

For a random complex matrix A, the matrix $AA^\dagger$ is Hermitian with non-negative eigenvalues ($\langle\chi|AA^\dagger\chi\rangle = |A^\dagger\chi|^2 \geq 0$); divided by its trace (`/=` divides the variable in place) it is a density operator. Mixed with the Gibbs state with a random weight s between 0 and 1 (`rng.uniform`), it gives states near and far from the minimum. For each, `gaps` stores $\Omega - \Omega_0$ and `kleins` Klein's D.

```
say(f"smallest Omega - Omega_0 among 2000 states: {gaps.min():.3e}")
check(np.all(gaps > 0.0), "every random density operator has Omega > Omega[Gibbs]")
check(np.max(np.abs(gaps - TEMPERATURE * kleins)) < 1e-10,
      "Omega - Omega[Gibbs] = T D with Klein's D (identity)")
```

The smallest excess is $3.878 \cdot 10^{-6}$ (`:.3e` writes a number with three decimals and a power of ten), still positive. The checks: all 2000 excesses are positive, and each equals $T\,D$ to 10^{-10}, the identity of the proof in Section 13.26.

In [5], the histogram.

```
fig, ax = plt.subplots()
bins = np.logspace(np.floor(np.log10(gaps.min())), np.ceil(np.log10(gaps.max())), 40)
ax.hist(gaps, bins=bins, color="tab:blue", edgecolor="black", lw=0.5)
ax.set_xscale("log")
```

`np.log10` is the logarithm to the base 10; `np.floor` and `np.ceil` round down and up to whole numbers, so the two arguments are the exponents of the powers of ten just below the smallest excess and just above the largest (-6 and 1 here). `np.logspace(a, b, 40)` makes 40 numbers from 10^a to 10^b, equally spaced on a logarithmic scale: the edges of the bins (the intervals in which the excesses are counted). `ax.hist` counts the excesses in each bin and draws the counts as blue bars with thin black edges, and `ax.set_xscale("log")` makes the horizontal axis logarithmic.

```
ax.set_xlabel("$\\Omega[\\hat\\rho] - \\Omega[\\hat\\rho_0]$ (units of $t$)")
ax.set_ylabel("number of random states")
ax.set_title("The Gibbs state has the lowest grand potential")
save_figure(fig, "gibbs_principle",
```

Labels (the grand potential is an energy, in units of the hopping t), title, and `save_figure` for Figure 13e.1.

What Figure 13e.1 shows. Every bar lies to the right of zero (on a logarithmic axis zero itself cannot appear, and no excess is negative): all 2000 random states have a larger grand potential

than the Gibbs state. Most excesses are between $0.1t$ and $2t$, from states mixed strongly away from the Gibbs state; a thin tail reaches down to about $4 \cdot 10^{-6}t$, from states with a mixing weight s close to 0. The closer a state is to the Gibbs state, the smaller its excess, but it never becomes negative.

In [6], independent orbitals.

```
levels3 = np.array([-0.5, 0.2, 1.0])
mu3, T3 = 0.1, 0.4
configurations = list(itertools.product((0, 1), repeat=3))
weights = np.array([np.exp(-np.dot(levels3 - mu3, c) / T3) for c in configurations])
weights /= weights.sum()  # divide by Z
f3 = 1.0 / (np.exp((levels3 - mu3) / T3) + 1.0)
products = np.array([np.prod([f3[k] if c[k] else 1.0 - f3[k] for k in range(3)])
                     for c in configurations])
```

Three orbitals with the energies -0.5, 0.2, 1.0 at $\mu = 0.1$, $T = 0.4$. The 8 configurations are all triples of zeros and ones (`itertools.product((0, 1), repeat=3)`). For each, `np.dot(levels3 - mu3, c)` is $\sum_a (\epsilon_a - \mu) n_a$, and its exponential, divided by the sum of all eight (Z), is the Gibbs weight. `products` multiplies, for each configuration, f_a for the occupied and $1 - f_a$ for the empty orbitals (`x if condition else y` chooses; `np.prod` multiplies the three factors).

```
for c, wgt, prod in zip(configurations, weights, products):
    say(f"occupations {c}: Gibbs weight {wgt:.6f}, product {prod:.6f}")
check(np.max(np.abs(weights - products)) < 1e-15,
      "the Gibbs weights are products of independent Fermi-Dirac probabilities")
```

The loop prints the eight pairs (for example 0.415797 and 0.415797 for the configuration (1, 0, 0), the most likely one: the lowest orbital, 0.6 below μ, filled, the others empty), and the check requires every weight to equal its product to 10^{-15}.

In [7], the Fermi-Dirac function.

```
def fermi(energy, mu, temperature):
    """The Fermi-Dirac occupation 1 / (exp(x) + 1), x = (energy - mu)/T, written
    as exp(-ln(1 + e^x)) so that no overflow can occur."""
    x = (np.asarray(energy, dtype=float) - mu) / temperature
    return np.exp(-np.logaddexp(0.0, x))
```

e^x exceeds the largest number the computer can store (about 10^{308}) for $x > 709$. Since $1/(e^x + 1) = e^{-\ln(1+e^x)}$, the function uses numpy's `logaddexp(0, x)` $= \ln(e^0 + e^x)$, which is computed without overflow for every x.

```
eps = np.linspace(-2.0, 2.0, 801)
check(abs(fermi(0.0, 0.0, 0.3) - 0.5) < 1e-15
      and np.allclose(fermi(eps, 0.0, 0.3), 1.0 - fermi(-eps, 0.0, 0.3), atol=1e-15),
      "f(mu) = 1/2 and f(mu + d) = 1 - f(mu - d)")
```

The check: $f(\mu) = \frac{1}{2}$, and $f(\mu + d) = 1 - f(\mu - d)$ for 801 values of d (a hole below μ is as likely as a particle above it; algebraically, $1 - \frac{1}{e^{-x}+1} = \frac{e^{-x}}{e^{-x}+1} = \frac{1}{1+e^x}$).

```
fig, ax = plt.subplots()
for temperature in (0.02, 0.1, 0.3, 1.0):
    ax.plot(eps, fermi(eps, 0.0, temperature), label=f"$T = {temperature}$")
ax.axvline(0.0, color="gray", ls="--", lw=0.8)
ax.set_xlabel("orbital energy $\\epsilon - \\mu$")
ax.set_ylabel("occupation $f$")
ax.set_title("The Fermi-Dirac occupation")
ax.legend()
save_figure(fig, "fermi_dirac",
```

The loop draws f against $\epsilon - \mu$ (with $\mu = 0$) for $T = 0.02$, 0.1, 0.3 and 1; a grey dashed vertical line marks $\epsilon = \mu$; labels, title, legend and `save_figure` for Figure 13e.2.

What Figure 13e.2 shows. At $T = 0.02$ the curve is practically a step: every orbital below μ is occupied and every orbital above it empty, the aufbau rule. As T grows, the step is smeared over a width of a few T: at $T = 1$ an orbital 2 below μ is occupied only with the probability 0.88. All four curves pass through $\frac{1}{2}$ at $\epsilon = \mu$, and each is point-symmetric about that point (the hole-particle symmetry just checked).

In [8], the chemical potential and the two-level system.

```
def chemical_potential(levels, degeneracies, N, temperature, history=None):
    """mu with sum g f = N, by bisection (the sum increases with mu)."""
    low, high = levels.min() - 50.0 * temperature - 5.0, levels.max() + 5.0
    for _ in range(100):
        middle = 0.5 * (low + high)
        if np.sum(degeneracies * fermi(levels, middle, temperature)) < N:
            low = middle  # too few particles: mu must rise
        else:
            high = middle
        if history is not None:
            history.append(high - low)
    return 0.5 * (low + high)
```

Bisection for μ (Section 13.26). The starting interval surely contains μ: at its lower end every occupation is below e^{-50}, so the levels hold fewer than N particles; at its upper end, 5 above the highest level, they hold almost all their states, more than N when N is less than the number of states. 100 halvings follow; if a list `history` is given, the width of the interval after each step is appended to it.

```
def thermodynamics(levels, degeneracies, N, temperature):
    """mu, occupations, E, S, F of non-interacting levels at fixed N."""
    mu = chemical_potential(levels, degeneracies, N, temperature)
    f = fermi(levels, mu, temperature)
    x = np.abs(levels - mu) / temperature
    s_level = np.logaddexp(0.0, -x) + x * fermi(x, 0.0, 1.0)  # entropy of a state
    S = np.sum(degeneracies * s_level)
    E = np.sum(degeneracies * f * levels)
    return mu, f, E, S, E - temperature * S
```

For given levels, degeneracies g_a, particle number and temperature: μ, the occupations, the entropy

of one state in the form $\ln(1+e^{-|x|})+|x|f(|x|)$ of Section 13.26 (`fermi(x, 0.0, 1.0)` is $1/(e^x+1)$), $S=\sum_a g_a s_a$, $E=\sum_a g_a f_a \epsilon_a$ and $F=E-TS$.

```
two_levels, ones = np.array([0.0, 1.0]), np.array([1.0, 1.0])
widths = []
chemical_potential(two_levels, ones, 1.0, 0.5, widths)
mu2, f2, E2, S2, F2 = thermodynamics(two_levels, ones, 1.0, 0.5)
```

The worked example of Section 13.26: levels 0 and 1, one state each, one particle, $T=\frac{1}{2}$. The first call records the widths of the bisection for the figure of In [10] (its result is not stored); the second computes the thermodynamics, and the five returned values are assigned to five names at once.

```
report("mu", f"{mu2:.6f}")
report("f_0, f_1", f"{f2[0]:.6f}, {f2[1]:.6f}")
report("E, S, F", f"{E2:.6f}, {S2:.6f}, {F2:.6f}")
check(abs(mu2 - 0.5) < 1e-12 and abs(f2[0] - 0.731059) < 1e-6
      and abs(f2[1] - 0.268941) < 1e-6,
      "mu = 1/2, f_0 = 0.731059, f_1 = 0.268941")
check(abs(E2 - 0.268941) < 1e-6 and abs(S2 - 1.164406) < 1e-6
      and abs(F2 + 0.313262) < 1e-6, "E = 0.268941, S = 1.164406, F = -0.313262")
```

The `report` lines print $\mu = 0.500000$, the occupations $f_0 = 0.731059$ and $f_1 = 0.268941$, and $E = 0.268941$, $S = 1.164406$ and $F = -0.313262$, and the two checks compare them with the values worked out by hand in Section 13.26 (μ to 10^{-12}, the others to 10^{-6}, the rounding of the six printed decimals).

In [9], the Revision solver's rule for μ.

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
root_rule = parameters["conventions"]["merminRoot"]  # a sentence of the record
root_form = parameters["numerics"]["merminRoot"]  # the name of the canonical form
say(f"Revision solver: canonical root form {root_form}")
check(root_rule.startswith("mu from sum g f = N")
      and "holes below a split of the levels" in root_rule
      and root_form == "LogBalance",
      "the Revision solver fixes mu by sum g f = N, balancing particles and holes "
      "(Revision/kohn_sham/results/parameters.json, conventions merminRoot)")
```

The cell reads two entries of the solver's parameter file: the sentence that describes the rule (`conventions`) and the name of the form it uses (`numerics`). The check requires the sentence to start with "mu from sum g f = N" and to contain the words about particles and holes, and the form to be LogBalance (Section 13.26).

In [10], derivatives with respect to T, and the bisection history.

```
def at(temperature, levels=two_levels, degeneracies=ones, N=1.0):
    return thermodynamics(levels, degeneracies, N, temperature)
```

```
d = 1e-4
dF_dT = (at(0.5 + d)[4] - at(0.5 - d)[4]) / (2 * d)
dE_dT = (at(0.5 + d)[2] - at(0.5 - d)[2]) / (2 * d)
T_dS_dT = 0.5 * (at(0.5 + d)[3] - at(0.5 - d)[3]) / (2 * d)
```

`at(T)` is the two-level thermodynamics at the temperature T; its result is the tuple (μ, f, E, S, F), so `[2]`, `[3]`, `[4]` are E, S, F. Central difference quotients with the step 10^{-4} (with μ solved again at each temperature) give dF/dT, dE/dT and $T\,dS/dT$ at $T = \frac{1}{2}$ (the factor `0.5` in the third line is T).

```
report("dF/dT", f"{dF_dT:.6f}")
report("C_V = dE/dT", f"{dE_dT:.6f}")
check(abs(dF_dT + S2) < 1e-7, "dF/dT = -S (envelope theorem)")
check(abs(dE_dT - 0.393224) < 1e-6 and abs(T_dS_dT - dE_dT) < 1e-7,
      "C_V = dE/dT = T dS/dT = 0.393224")
```

The cell prints $dF/dT = -1.164406$ and $C_V = 0.393224$ and checks $dF/dT = -S$ (to 10^{-7}), $C_V = 0.393224$ (the value $2e/(e+1)^2$ of Section 13.26) and $dE/dT = T\,dS/dT$ (to 10^{-7}).

```
fig, ax = plt.subplots()
ax.semilogy(range(1, 51), widths[:50], "o-", ms=3)
ax.set_xlabel("bisection step")
ax.set_ylabel("width of the interval that contains $\\mu$")
ax.set_title("Bisection halves the interval at every step")
save_figure(fig, "bisection",
```

The first 50 recorded widths against the step numbers 1 to 50 (`range(1, 51)`), on a logarithmic vertical axis; labels, title and `save_figure` for Figure 13e.3. The last check

```
check(all(abs(b / a_ - 0.5) < 1e-12 for a_, b in zip(widths[:40], widths[1:41])),
      "every bisection step halves the interval")
```

requires every width to be half of the previous one (for the first 40 steps; the name `a_` avoids the name `a` of the operator list).

What Figure 13e.3 shows. The starting interval is $[-30, 6]$, 36 wide, so after the first step the width is 18. On the logarithmic axis the points then fall on a straight line, by $\log_{10} 2 = 0.30$ per step, to about $3 \cdot 10^{-14}$ after 50 steps: bisection gains one binary digit of μ at every step, whatever the function, as long as the root is inside the interval.

In [11], two levels at all temperatures.

```
temperatures = np.linspace(0.02, 3.0, 150)
table = np.array([at(T)[2:] for T in temperatures])  # columns E, S, F
C_V = np.array([T * (at(T + d)[3] - at(T - d)[3]) / (2 * d) for T in temperatures])
hot = at(1000.0)
check(table[0, 1] < 1e-8 and abs(hot[3] - np.log(4.0)) < 1e-5
      and abs(hot[2] - 0.5) < 1e-3, "S -> 0 for T -> 0; S -> ln 4, E -> 1/2 for T "
      "-> infinity")
check(np.all(C_V >= 0.0), "the heat capacity is never negative")
```

For 150 temperatures from 0.02 to 3: the table of E, S, F (`[2:]` keeps the last three entries of the tuple) and $C_V = T\,dS/dT$. The limits: at the lowest temperature $S < 10^{-8}$ (the particle sits in level 0); at $T = 1000$, practically infinite, both occupations are $\frac{1}{2}$, so $S = 2\ln 2 = \ln 4$ and $E = \frac{1}{2}$. The second check: $C_V \geq 0$ everywhere.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(temperatures, table[:, 0], label="energy $E$")
left.plot(temperatures, temperatures * table[:, 1], label="$T S$")
left.plot(temperatures, table[:, 2], color="black", label="free energy $F = E - TS$")
left.set_xlabel("temperature $T$")
left.set_ylabel("energy (level-spacing units)")
left.legend(fontsize=8)
```

`table[:, 0]` is the first column of the table (all rows), the energy; `temperatures * table[:, 1]` multiplies each entropy by its temperature; `table[:, 2]` is the free energy, drawn in black. Labels and legend of the left panel.

```
right.plot(temperatures, C_V, color="tab:red")
right.plot([0.5], [0.393224], "ko", label="$T = 1/2$: $C_V = 0.393224$")
right.set_xlabel("temperature $T$")
right.set_ylabel("heat capacity $C_V = dE/dT$")
right.legend(fontsize=8)
fig.suptitle("Two levels (0 and 1), one particle")
save_figure(fig, "two_level_thermo",
```

The right panel draws C_V in red and the worked value at $T = \frac{1}{2}$ as a black dot; then labels, legend, a title above both panels, and `save_figure` for Figure 13e.4.

What Figure 13e.4 shows. On the left, at low temperature E, TS and F are all near 0 (the particle sits in the lower level). As T grows, E rises towards $\frac{1}{2}$ (both levels equally occupied), TS grows almost linearly (towards $T\ln 4$: the entropy S_s approaches $2\ln 2 = \ln 4$, the value for two independent orbitals that are each occupied with the probability $\frac{1}{2}$, Exercise 7), and $F = E - TS$ falls ever more steeply, with the slope $-S$. On the right, the heat capacity is practically 0 below $T = 0.05$, rises to a single peak of about 0.88 near $T = 0.21$, passes through the black dot at $T = \frac{1}{2}$, and decreases slowly towards 0: energy can be absorbed only while the upper level is filling up.

In [12], a ladder of levels.

```
ladder = np.arange(60) + 0.5
twos = 2.0 * np.ones(60)
N_LADDER = 8.0


def ladder_at(T):
    return thermodynamics(ladder, twos, N_LADDER, T)


def variance_heat_capacity(T):
    """C_V = [sum w x^2 - (sum w x)^2 / sum w] / T^2, x = eps - mu, w = g f (1 - f)."""
    mu, f = ladder_at(T)[:2]
```

```
    w = twos * f * (1.0 - f)
    x = ladder - mu
    return (np.sum(w * x * x) - np.sum(w * x) ** 2 / np.sum(w)) / T ** 2
```

The levels $n + \frac{1}{2}$, $n = 0, \ldots, 59$, two states each, eight particles: the trap of Notebook 13a without interaction. `variance_heat_capacity` is the variance formula of Section 13.26.

```
T_ladder = np.linspace(0.05, 3.0, 120)
mus = np.array([ladder_at(T)[0] for T in T_ladder])
C_numeric = np.array([(ladder_at(T + d)[2] - ladder_at(T - d)[2]) / (2 * d)
                      for T in T_ladder])
C_variance = np.array([variance_heat_capacity(T) for T in T_ladder])
```

For 120 temperatures from 0.05 to 3: μ, the heat capacity as a difference quotient of E (the step `d` $= 10^{-4}$ of In [10]), and the variance formula.

```
report("mu at T = 0.05", f"{mus[0]:.6f}")
check(abs(mus[0] - 4.0) < 1e-6, "at low T, mu lies halfway between 3.5 and 4.5")
check(np.max(np.abs(C_numeric - C_variance)) < 1e-5 * max(1.0, C_variance.max()),
      "the heat capacity equals the weighted-variance formula")
check(np.all(C_variance >= 0.0), "the variance formula is never negative")
```

The cell prints $\mu = 4.000000$ at $T = 0.05$ and checks it (at low temperature the four lowest levels are full, and by the symmetry of f about μ the chemical potential lies halfway between the last full level 3.5 and the first empty one 4.5), then that the two heat capacities agree to 10^{-5} times the larger of 1 and the largest value (`max` of two numbers is the larger), and that the variance formula is never negative.

In [13], the occupations of the ladder.

```
fig, ax = plt.subplots()
for T, marker in ((0.05, "o"), (0.5, "s"), (1.0, "^"), (2.0, "v")):
    mu, f = ladder_at(T)[:2]
    ax.plot(ladder[:12], f[:12], marker + "-", ms=4,
            label=f"$T = {T}$, $\\mu = {mu:.3f}$")
ax.set_xlabel("level $\\epsilon_n = n + 1/2$")
ax.set_ylabel("occupation $f_n$ of each of the two states")
ax.set_title("Eight particles in a ladder of levels")
ax.legend(fontsize=8)
save_figure(fig, "ladder_occupations",
```

For $T = 0.05, 0.5, 1, 2$ the loop computes μ and the occupations (`[:2]` keeps the first two returned values) and draws the occupations of the twelve lowest levels against their energies; `marker + "-"` joins a marker letter and a line style into one style string, and the label shows T and μ with three decimals. Labels, title, legend and `save_figure` for Figure 13e.5.

What Figure 13e.5 shows. At $T = 0.05$ the occupation is 1 for the levels 0.5 to 3.5 and 0 above: a perfect step between 3.5 and 4.5. At $T = 0.5$ the levels 3.5 and 4.5 are partly occupied (0.73 and 0.27). At $T = 1$ and 2 the step is smeared over several levels, and particles reach the levels 8.5

and beyond. The chemical potential stays at 4.000 up to $T = 0.5$ and falls to 3.982 at $T = 1$ and 3.712 at $T = 2$: the ladder continues upwards but not downwards (no level below 0.5), so at high temperature the particles spread more to higher levels, and μ must fall to keep the number at 8.

In [14], the chemical potential and the heat capacity of the ladder.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(T_ladder, mus, color="black")
left.set_xlabel("temperature $T$")
left.set_ylabel("chemical potential $\\mu$")
right.plot(T_ladder, C_numeric, color="tab:red", label="$dE/dT$ (differences)")
right.plot(T_ladder[::6], C_variance[::6], "ko", ms=3, label="variance formula")
right.set_xlabel("temperature $T$")
right.set_ylabel("heat capacity $C_V$")
right.legend(fontsize=8)
fig.suptitle("Ladder of levels, eight particles")
save_figure(fig, "ladder_thermo",
```

Left: μ against T. Right: the heat capacity from the difference quotient as a red line and from the variance formula at every sixth temperature (`[::6]`) as black dots, so that both can be seen; labels, legend, a common title and `save_figure` for Figure 13e.6.

What Figure 13e.6 shows. On the left, μ stays at 4 up to about $T = 0.5$ and then falls, to about 3.1 at $T = 3$. On the right, the black dots lie on the red line at every temperature: the difference quotient and the weighted-variance formula agree, and both are positive. The heat capacity is nearly 0 at $T = 0.05$ (a gap of one level spacing separates the full from the empty levels), rises steeply, and approaches 8, one unit per particle, at high temperature, where each particle behaves like a classical oscillator.

In [15], Janak's theorem and Delta-SCF.

```
EPS_H0, EPS_L0, U_MODEL = 0.0, 1.0, 0.2


def model_energy(f_H, f_L):
    return EPS_H0 * f_H + EPS_L0 * f_L + 0.5 * U_MODEL * (f_H ** 2 + f_L ** 2)


def model_levels(f_H, f_L):
    """Janak: eps_a = dE/df_a = eps_a^0 + U f_a."""
    return EPS_H0 + U_MODEL * f_H, EPS_L0 + U_MODEL * f_L
```

The model energy of Section 13.27 and its levels.

```
h_step = 1e-6
for f_H, f_L in ((1.0, 0.0), (0.5, 0.5), (0.3, 0.9)):
    dE_dfH = (model_energy(f_H + h_step, f_L) - model_energy(f_H - h_step, f_L)) \
        / (2 * h_step)
    dE_dfL = (model_energy(f_H, f_L + h_step) - model_energy(f_H, f_L - h_step)) \
        / (2 * h_step)
    check(np.allclose((dE_dfH, dE_dfL), model_levels(f_H, f_L), atol=1e-9),
          f"Janak: dE/df_a = eps_a at (f_H, f_L) = ({f_H}, {f_L})")
```

At three pairs of occupations the derivatives of the energy are computed as central difference quotients and compared with the levels (three checks).

```
taus = np.linspace(0.0, 1.0, 101)
integrand = np.array([model_levels(1 - t, t)[1] - model_levels(1 - t, t)[0]
                      for t in taus])
gap_KS = integrand[0]
delta_SCF = model_energy(0.0, 1.0) - model_energy(1.0, 0.0)
step = taus[1] - taus[0]
janak_integral = step / 3 * (integrand[0] + integrand[-1]
                             + 4 * integrand[1:-1:2].sum() + 2 * integrand[2:-1:2].sum())
```

101 values of the transferred fraction τ; the level difference $\epsilon_L(\tau) - \epsilon_H(\tau)$ at $(f_H, f_L) = (1-\tau, \tau)$ (the function `model_levels` returns the pair (ϵ_H, ϵ_L), so `[1]` is ϵ_L and `[0]` is ϵ_H); the gap (its value at $\tau = 0$); the Delta-SCF energy $E(0,1) - E(1,0)$; and the integral of the level difference by Simpson's rule (Section 13.15) on the 101 values.

```
report("Kohn-Sham gap", f"{gap_KS:.6f}")
report("Delta-SCF", f"{delta_SCF:.6f}")
report("integral of eps_L - eps_H over tau", f"{janak_integral:.6f}")
report("transition state (tau = 1/2)", f"{integrand[50]:.6f}")
check(abs(gap_KS - 0.8) < 1e-12 and abs(delta_SCF - 1.0) < 1e-12
      and abs(janak_integral - delta_SCF) < 1e-12 and abs(integrand[50] - 1.0) < 1e-12,
      "gap 0.8, Delta-SCF 1.0 = the Janak integral = the transition state")
check(abs(delta_SCF - gap_KS - U_MODEL) < 1e-12, "Delta-SCF - gap = U (relaxation)")
```

The `report` lines print 0.800000, 1.000000, 1.000000 and the transition-state value 1.000000 (entry 50 is $\tau = \frac{1}{2}$). The last two checks require these four values (Simpson's rule is exact here, because the integrand is a straight line) and $\Delta_{SCF} - \Delta_{KS} = U$.

In [16], the transfer as a picture.

```
energies_tau = np.array([model_energy(1 - t, t) for t in taus])
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(taus, energies_tau - energies_tau[0], color="black")
left.set_xlabel("transferred fraction $\\tau$")
left.set_ylabel("$E(\\tau) - E(0)$")
left.set_title("energy along the transfer")
```

`energies_tau` is the model energy $E(1-\tau, \tau)$ at every τ; the left panel draws its rise $E(\tau) - E(0)$ in black, with labels and a title.

```
right.fill_between(taus, 0.0, integrand, alpha=0.25, label="area = Delta-SCF = 1")
right.plot(taus, integrand, color="black", label="$\\epsilon_L - \\epsilon_H$")
right.plot([0.0], [gap_KS], "s", ms=8, label="Kohn-Sham gap 0.8")
right.plot([0.5], [integrand[50]], "^", ms=8, label="transition state 1.0")
right.set_ylim(0.0, 1.3)
right.set_xlabel("transferred fraction $\\tau$")
right.set_ylabel("level difference")
right.legend(fontsize=8, loc="lower right")
```

`right.fill_between(taus, 0.0, integrand, ...)` shades the area between the zero line and the level difference (`alpha=0.25`: light), which by Janak's theorem is the Delta-SCF energy; the level difference itself as a black line; the gap as a large square at $\tau = 0$ and the transition state as a large triangle at $\tau = \frac{1}{2}$; the vertical range from 0 to 1.3; labels and a legend in the lower right corner.

```
fig.suptitle("Janak's theorem and Delta-SCF ($\\epsilon_H^0 = 0$, "
             "$\\epsilon_L^0 = 1$, $U = 0.2$)")
save_figure(fig, "janak_delta_scf",
```

A title above both panels (two strings joined into one) with the parameters of the model, and `save_figure` for Figure 13e.7.

What Figure 13e.7 shows. On the left the energy rises from 0 to 1.0 as one whole particle is moved from H to L, slightly curved upwards, because its slope, the level difference, grows with τ. On the right that slope is the straight line $0.8 + 0.4\tau$ from the gap 0.8 (square) to 1.2; the shaded area under it is 1.0, the Delta-SCF energy, and the value at the midpoint (triangle) is also exactly 1.0, because the line is straight. The gap underestimates the excitation energy by $U = 0.2$, the orbital relaxation of this model.

In [17], the last check.

```
figure_names = ["gibbs_principle", "fermi_dirac", "bisection", "two_level_thermo",
                "ladder_occupations", "ladder_thermo", "janak_delta_scf"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13e_{k}_{name}.png").is_file()]
check(missing == [], "all seven figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13e_7_janak_delta_scf.png").is_file(),
      "the figure file 13e_7_janak_delta_scf.png exists")
all_checks_passed()
```

The same lines as In [19] of Notebook 13c (Section 13.14), with the seven figure names of this notebook. The last line prints ALL 24 CHECKS PASSED (notebook 13e): one check each in In [2], In [3], In [6], In [7] and In [9], two each in In [4], In [8] and In [11], three each in In [10] and In [12], five in In [15] and two in In [17].

13.32 A complete Kohn-Sham calculation: eight fermions in a trap

This section puts every piece of the chapter together in one small but complete Kohn-Sham calculation, the example of Notebook 13a. It is a teaching model: its numbers are COMPUTED by the notebook and belong to no physical system of the book. (Its letter x is the position on a line, not a spacetime coordinate.)

The model. $N = 8$ identical fermions on a line, four with the label up and four with the label down, in the **harmonic trap** $v(x) = x^2/2$, with the contact repulsion $w(x, x') = g_c\,\delta(x - x')$ of strength $g_c = 2$. Units: $\hbar = m = \omega = 1$, where ω is the angular frequency of the trap; energies are in units of $\hbar\omega$ and lengths in units of $\sqrt{\hbar/(m\omega)}$. Without interaction a particle in the trap has the levels $\frac{1}{2}, \frac{3}{2}, \frac{5}{2}, \ldots$, and the ground state of the eight fermions fills the four lowest levels with two fermions each: $E = 2(\frac{1}{2} + \frac{3}{2} + \frac{5}{2} + \frac{7}{2}) = 16$.

The energy functional. With the four lowest orbitals $\phi_0, \dots, \phi_3$ each occupied by one up and one down fermion, the density is $n = 2\sum_{a=0}^{3} \phi_a^2$ with $n_\uparrow = n_\downarrow = n/2$, and the energy of the determinant is (Section 13.5)

$$E = T_s + \int v\, n\, dx + E_H + E_x, \qquad E_H = \frac{g_c}{2}\int n^2\, dx, \qquad E_x = -\frac{g_c}{2}\int (n_\uparrow^2 + n_\downarrow^2) dx = -\frac{g_c}{4}\int n^2\, dx,$$

with $T_s = 2\sum_{a=0}^{3} \int \phi_a \left(-\frac{1}{2}\phi_a''\right) dx$. For a contact interaction the exchange of a determinant is exactly this local formula (Section 13.5); the correlation energy is left out. So this is an **exchange-only** Kohn-Sham scheme, which for a contact interaction is the same as the Hartree-Fock approximation: an approximation to the true ground state, whose error is the correlation energy (Section 13.7 showed how large it can be on two sites).

The Kohn-Sham potential, line by line. The functional derivatives are

$$\frac{\delta E_H}{\delta n} = g_c\, n, \qquad \frac{\delta E_x}{\delta n_\uparrow} = -g_c\, n_\uparrow = -\frac{g_c}{2}\, n,$$

by example (i) of Section 13.9 (for E_x applied to the up density, with the down density held fixed). A fermion with label up therefore feels

$$v_s = v + g_c\, n - \frac{g_c}{2}\, n = v + w, \qquad w = \frac{g_c}{2}\, n,$$

the Hartree push of everybody minus the exchange with its own label, and the same holds for label down. w is the **mean-field potential**. A solution of the Kohn-Sham equations is a w for which the orbitals of $v + w$ give back $w_{out} = g_c n/2 = w$; the **residual** of one pass is $r = \max_x |w_{out}(x) - w(x)|$, and the loop stops when $r \le 10^{-11}$, the stopping rule of the Revision solver (Section 13.21).

The energy, two ways, line by line. Multiply the Kohn-Sham equation of orbital a by ϕ_a, integrate and sum with the occupation 2:

$$2\sum_{a=0}^{3} \epsilon_a = T_s + \int v_s\, n\, dx = T_s + \int v\, n\, dx + \frac{g_c}{2}\int n^2\, dx,$$

$$E = T_s + \int v\, n\, dx + \frac{g_c}{4}\int n^2\, dx = 2\sum_{a=0}^{3} \epsilon_a - \frac{g_c}{4}\int n^2\, dx = 2\sum_{a=0}^{3} \epsilon_a - (E_H + E_x).$$

The first line is the computation of Section 13.10 with $v_s = v + g_c n/2$; the second writes $E_H + E_x = \frac{g_c}{4}\int n^2$ and subtracts the first line. This **double-counting formula** gives the same energy as the direct sum of the four parts, which the notebook checks.

Stability of the equal-label solution. So far the up and down densities were forced to be equal. Would the fermions lower their energy by separating the labels, as on two sites at strong repulsion (Section 13.7)? Let each label have its own potential: a fermion with label up feels $v + g_c n - g_c n_\uparrow = v + g_c n_\downarrow$, and one with label down $v + g_c n_\uparrow$, and the energy is $E = T_s + \int v\, n + g_c \int n_\uparrow n_\downarrow$ (Section 13.5). Started from a strongly separated guess, the loop for the two potentials together may return to equal densities; but that alone does not show stability, because the loop can also converge

to a **saddle**: a self-consistent solution from which some change of the densities LOWERS the energy. Only the energy decides: the equal-label solution is a minimum when every small label-separating change raises the energy. Notebook 13a shows both cases: on two sites with $U = 4t$ the loop started from a small separation converges to the equal labels although separating them lowers the energy (a saddle), and in the trap every label-separating trial family raises the energy, quadratically for small separations (a minimum along these families).

The variational principle at work. The Kohn-Sham equations make the energy stationary in the orbitals, and for the ground state the stationary point is a minimum. A one-parameter test: for each number c, take the four lowest orbitals of the trial potential $v + c\,n_{scf}$ (n_{scf} the self-consistent density) and evaluate the same energy formula with them. At $c = g_c/2 = 1$ these are the self-consistent orbitals; the energy must be smallest there, and near the minimum the curve is flat (a small error in the orbitals makes only a second-order error in the energy).

Three simpler pictures. (a) **No interaction**: $w = 0$. (b) **Hartree only**: drop the exchange, $v_s = v + g_c n$; every fermion is then also repelled by its own density (the self-interaction of Section 13.5). (c) **Thomas-Fermi**: the kinetic energy of each small piece of the line is that of a uniform gas with the local density. For a uniform gas on a line with two labels, the plane waves with $|k| < k_F$ are filled, two per wave number, and in a box of length ℓ the wave numbers are spaced by $2\pi/\ell$; line by line:

$$N = 2 \cdot \frac{2k_F}{2\pi/\ell} = \frac{2k_F\,\ell}{\pi} \quad\Longrightarrow\quad n = \frac{2k_F}{\pi},$$
$$\frac{E_{kin}}{\ell} = 2\int_{-k_F}^{k_F} \frac{dk}{2\pi}\,\frac{k^2}{2} = \frac{1}{2\pi}\cdot\frac{2k_F^3}{3} = \frac{k_F^3}{3\pi} = \frac{\pi^2\,n^3}{24}.$$

The first line counts the occupied wave numbers (the interval of length $2k_F$ divided by the spacing, times two labels) and divides by ℓ; the second sums $k^2/2$ over them per unit length (a sum over wave numbers becomes $\ell\int dk/(2\pi)$), integrates ($\int_{-k_F}^{k_F} k^2\,dk = 2k_F^3/3$), and inserts $k_F = \pi n/2$. Minimising $\int[\pi^2 n^3/24 + v\,n + \frac{g_c}{4}n^2]\,dx$ at fixed $\int n\,dx = N$ with the multiplier μ (Section 13.9) gives, where $n > 0$,

$$\frac{\pi^2}{8}\,n^2 + \frac{g_c}{2}\,n = \mu - v(x) \quad\Longrightarrow\quad n = \frac{-b + \sqrt{b^2 + 4a\,(\mu - v)}}{2a}, \qquad a = \frac{\pi^2}{8},\ b = \frac{g_c}{2},$$

the positive root of a quadratic equation at each point, and $n = 0$ where $v > \mu$; μ is found by bisection so that the density holds 8 particles. Thomas-Fermi uses no orbitals, so it cannot show the four bumps of the Kohn-Sham density (its **shell structure**).

The Hellmann-Feynman theorem. How does the energy change with the strength g_c? The Kohn-Sham energy is $E(g_c) = \mathcal{E}[\phi^*(g_c); g_c]$, the functional evaluated at its stationary orbitals ϕ^*. By the chain rule, $dE/dg_c = \partial\mathcal{E}/\partial g_c + \sum(\text{derivative with respect to the orbitals}) \cdot d\phi^*/dg_c$, and the second part vanishes because $\mathcal{E}$ is stationary in the orbitals (with their normalisation kept), exactly as in the envelope theorem of Section 13.26. Only the explicit dependence remains, $E_H + E_x = \frac{g_c}{4}\int n^2$:

$$\frac{dE}{dg_c} = \frac{1}{4}\int n^2\,dx.$$

The virial theorem at $g_c = 0$. For a particle in the harmonic trap the kinetic and the trap energy

of every level are equal. Line by line, for a normalised stationary state ϕ: the stretched states $\phi_\lambda(x) = \sqrt{\lambda}\,\phi(\lambda x)$ are normalised, their kinetic energy is $\lambda^2 T$ and their trap energy V/λ^2 (substitute $y = \lambda x$ in the integrals), so $E(\lambda) = \lambda^2 T + V/\lambda^2$. A stationary state makes the energy stationary under every change of the state, in particular under stretching, so $dE/d\lambda = 2\lambda T - 2V/\lambda^3 = 0$ at $\lambda = 1$, that is $T = V$. At $g_c = 0$ the kinetic and the trap energy of the eight fermions are therefore equal, 8 each.

The first excited state. The Kohn-Sham gap is $\epsilon_4 - \epsilon_3$ of the ground state. The Delta-SCF energy moves one up fermion from orbital 3 (HOMO) to orbital 4 (LUMO) and solves the loop again (Section 13.27). By Janak's theorem it equals the integral of $\epsilon_4(\tau) - \epsilon_3(\tau)$ over the moved fraction τ from 0 to 1, which the notebook computes with Simpson's rule on nine values of τ.

What the notebook finds (COMPUTED, Notebook 13a). The grid (200 points in $-6 < x < 6$) reproduces the levels $n + \frac{1}{2}$ of the trap to 0.013 (In [3]; an error of order h^2, Section 13.2); all numbers below are those of this grid model, exact for it up to the stopping rule of the loop, and they differ from the continuum model by an error of the same size, about 10^{-2} (at $g_c = 0$ the grid energy is 15.990192 instead of 16, In [23]). Plain iteration needs 40 passes, linear mixing with $\beta = 0.7$ needs 23 and with $\beta = 0.3$ needs 73 (In [6]), Anderson mixing with the Revision settings 19 (In [8]); all reach the same potential to 10^{-9} (Figure 13a.2), and plain iteration overshoots: the first density is too narrow, the second too wide (In [10], Figure 13a.3). The Kohn-Sham levels are 2.119598, 3.006214, 3.877952, 4.723945 (occupied) and 5.496606 (LUMO) (In [11]). The energy is $E = 21.851498$ by both formulas, made of $T_s = 6.726297$, $\int v\,n = 9.521279$, $E_H = 11.207843$ and $E_x = -5.603922 = -E_H/2$ (In [14]). From a strongly separated start the two-label loop returns to equal labels (In [16]); the energy test shows that along three label-separating families the equal-label solution is a minimum, while on two sites with $U = 4t$ the loop converges to a saddle (In [17], In [18], Figure 13a.6). In the trial family the lowest energy is at $c = 1.00$ (In [19], Figure 13a.7). The root-mean-square widths of the cloud are 1.413346 (no interaction), 1.542829 (Kohn-Sham) and 1.659857 (Hartree only): the repulsion spreads the cloud, and the self-interaction of Hartree spreads it too much (In [21], Figure 13a.8). $dE/dg_c = 2.801961$ both as a difference quotient and as $\frac{1}{4}\int n^2$ (In [23]). The Kohn-Sham gap is 0.772662, the Delta-SCF energy 0.714594 (equal to the Janak integral), and the transition-state estimate 0.712784 (In [25]): here the orbital relaxation lowers the excitation energy.

13.33 Example: the one-dimensional Kohn-Sham toy

Notebook 13a carries out the calculation of Section 13.32 from the first line to the last: the grid and its check against the exact levels of the trap; the Kohn-Sham map; plain iteration, linear mixing and Anderson mixing with exactly the settings of the Revision Kohn-Sham solver, read from its parameter file (its only link to the Revision record); the converged state with its potentials and density; the energy two ways and the functional-derivative test; the stability of the equal-label solution; the variational principle; the Hartree and Thomas-Fermi comparisons; the switching-on of the interaction with the Hellmann-Feynman theorem; and the first excited state by Delta-SCF with Janak's theorem. It ends with ALL 36 CHECKS PASSED (notebook 13a) and draws ten figures.

13.34 How to run Notebook 13a

Step 1. What this notebook does and what it needs.

Notebook 13a (A one-dimensional Kohn-Sham toy with self-consistency) is the file `Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb` of the repository Dirac_claude. It solves the Kohn-Sham equations of eight fermions with two labels in a one-dimensional harmonic trap with a contact repulsion (Hartree plus the exact local exchange of this interaction, no correlation) on a grid, by plain iteration, by linear mixing and by Anderson mixing with the settings of the Revision Kohn-Sham solver; it checks the particle number, the two energy formulas, that the mean-field potential is the functional derivative of the interaction energy, the variational principle, the stability of the equal-label solution and the Hellmann-Feynman theorem, compares the result with the Hartree and the Thomas-Fermi approximations, computes the first excited state by Delta-SCF and checks Janak's theorem, and draws ten teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 13a_kohn_sham_1d_toy.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 13a_kohn_sham_1d_toy.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 13a_kohn_sham_1d_toy.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/13a.captions.json`
- `Revision/textbook/figures/13a_1_trap_orbitals.png`
- `Revision/textbook/figures/13a_2_scf_convergence.png`
- `Revision/textbook/figures/13a_3_first_iterations.png`
- `Revision/textbook/figures/13a_4_ks_potentials.png`
- `Revision/textbook/figures/13a_5_density_orbitals.png`
- `Revision/textbook/figures/13a_6_label_separation.png`
- `Revision/textbook/figures/13a_7_variational_scan.png`
- `Revision/textbook/figures/13a_8_approximations.png`
- `Revision/textbook/figures/13a_9_coupling_scan.png`
- `Revision/textbook/figures/13a_10_delta_scf.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13a_10_delta_scf.png exists
ALL 36 CHECKS PASSED (notebook 13a)
```

and the notebook must show 10 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 13a_kohn_sham_1d_toy.ipynb
python -m nbconvert --execute --inplace 13a_kohn_sham_1d_toy.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `parameters.json`: the notebook reads the file `Revision/kohn_sham/results/parameters.json` of the repository; it must be opened inside the folder `Revision/textbook/notebooks` of a complete clone of the repository, not as a single downloaded file.

13.35 Notebook 13a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook solves a small but complete Kohn-Sham problem from the first line to the
last, so that every step of density functional theory can be seen at work. Eight
identical fermions, four with the label "up" and four with the label "down", move on a
line in a harmonic trap and repel each other when they meet at the same point. The
notebook

- builds the one-particle Hamiltonian on a grid of 200 points and checks it against
  the exactly known levels of the harmonic trap;
- writes the Kohn-Sham potential of this model (external potential plus Hartree
  potential plus exchange potential) and solves the Kohn-Sham equations
  self-consistently in three ways: plain iteration, linear mixing, and Anderson
  mixing with exactly the settings that the Revision Kohn-Sham solver of the
  dirac16complex field uses (read from its parameter file);
- checks the particle number, computes the total energy in two different ways and
  checks that they agree, and tests with the energy whether the solution with equal
  up and down densities is stable (a converged loop alone cannot tell a minimum from
  a saddle, as two sites show);
- shows the variational principle at work: every other set of orbitals of a family
  gives a higher energy;
- compares the Kohn-Sham density with three simpler pictures: no interaction, the
  Hartree approximation (no exchange) and the Thomas-Fermi approximation (the local
  density approximation for the kinetic energy);
- switches the interaction on step by step and checks the Hellmann-Feynman theorem;
- computes the first excited state by the Delta-SCF method and checks Janak's
  theorem;
- draws ten teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 13a (A one-dimensional Kohn-Sham toy with self-consistency) is the file
‘Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb‘ of the repository Dirac_claude.
It solves the Kohn-Sham equations of eight fermions with two labels in a one-dimensional
harmonic trap with a contact repulsion (Hartree plus the exact local exchange of this
interaction, no correlation) on a grid, by plain iteration, by linear mixing and by
Anderson mixing with the settings of the Revision Kohn-Sham solver; it checks the
```

particle number, the two energy formulas, that the mean-field potential is the functional derivative of the interaction energy, the variational principle, the stability of the equal-label solution and the Hellmann-Feynman theorem, compares the result with the Hartree and the Thomas-Fermi approximations, computes the first excited state by Delta-SCF and checks Janak's theorem, and draws ten teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 13a_kohn_sham_1d_toy.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 13a_kohn_sham_1d_toy.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the

name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 13a_kohn_sham_1d_toy.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/13a.captions.json‘
- ‘Revision/textbook/figures/13a_1_trap_orbitals.png‘
- ‘Revision/textbook/figures/13a_2_scf_convergence.png‘
- ‘Revision/textbook/figures/13a_3_first_iterations.png‘
- ‘Revision/textbook/figures/13a_4_ks_potentials.png‘
- ‘Revision/textbook/figures/13a_5_density_orbitals.png‘
- ‘Revision/textbook/figures/13a_6_label_separation.png‘
- ‘Revision/textbook/figures/13a_7_variational_scan.png‘
- ‘Revision/textbook/figures/13a_8_approximations.png‘
- ‘Revision/textbook/figures/13a_9_coupling_scan.png‘
- ‘Revision/textbook/figures/13a_10_delta_scf.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the figure file 13a_10_delta_scf.png exists
ALL 36 CHECKS PASSED (notebook 13a)
```

and the notebook must show 10 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb‘ inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

    ```
    python -m jupyterlab 13a_kohn_sham_1d_toy.ipynb
    python -m nbconvert --execute --inplace 13a_kohn_sham_1d_toy.ipynb
    ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for `parameters.json`: the notebook reads the file `Revision/kohn_sham/results/parameters.json` of the repository; it must be opened inside the folder `Revision/textbook/notebooks` of a complete clone of the repository, not as a single downloaded file.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers `save_figure` (saves a figure and shows it), `check` (stops with an AssertionError when a check fails, prints a PASS line when it passes), `report` (prints a key number as a RESULT line) and `all_checks_passed` (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 13a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 13a (A one-dimensional Kohn-Sham toy with self-consistency) is the file
# Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb of the repository Dirac_claude.
# It solves the Kohn-Sham equations of eight fermions with two labels in a
# one-dimensional harmonic trap with a contact repulsion (Hartree plus the exact local
# exchange of this interaction, no correlation) on a grid, by plain iteration, by linear
# mixing and by Anderson mixing with the settings of the Revision Kohn-Sham solver; it
# checks the particle number, the two energy formulas, that the mean-field potential is
# the functional derivative of the interaction energy, the variational principle, the
# stability of the equal-label solution and the Hellmann-Feynman theorem, compares the
```

```
# result with the Hartree and the Thomas-Fermi approximations, computes the first excited
# state by Delta-SCF and checks Janak's theorem, and draws ten teaching plots. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
```

```
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
```

```
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 13a_kohn_sham_1d_toy.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 13a_kohn_sham_1d_toy.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 13a_kohn_sham_1d_toy.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/13a.captions.json
# - Revision/textbook/figures/13a_1_trap_orbitals.png
# - Revision/textbook/figures/13a_2_scf_convergence.png
# - Revision/textbook/figures/13a_3_first_iterations.png
# - Revision/textbook/figures/13a_4_ks_potentials.png
```

```
# - Revision/textbook/figures/13a_5_density_orbitals.png
# - Revision/textbook/figures/13a_6_label_separation.png
# - Revision/textbook/figures/13a_7_variational_scan.png
# - Revision/textbook/figures/13a_8_approximations.png
# - Revision/textbook/figures/13a_9_coupling_scan.png
# - Revision/textbook/figures/13a_10_delta_scf.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the figure file 13a_10_delta_scf.png exists
#     ALL 36 CHECKS PASSED (notebook 13a)
#
# and the notebook must show 10 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/13a_kohn_sham_1d_toy.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 13a_kohn_sham_1d_toy.ipynb
#     python -m nbconvert --execute --inplace 13a_kohn_sham_1d_toy.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
```

```
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for parameters.json: the notebook reads the file
#   Revision/kohn_sham/results/parameters.json of the repository; it must be opened
#   inside the folder Revision/textbook/notebooks of a complete clone of the repository,
#   not as a single downloaded file.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "13a"  # this notebook: chapter 13, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 13a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Fermion**: a particle of a kind that obeys the Pauli principle: two identical
  fermions with the same label never occupy the same one-particle state.
- **Label**: an internal property of a particle that takes a few values; here two,
  "up" and "down" (like the spin of an electron). Particles with different labels
  can be told apart; particles with the same label cannot.
- **Orbital**: a one-particle wave function $\phi_a(x)$; its square $\phi_a(x)^2$ is
  the probability per unit length of finding the particle at $x$.
- **Density** $n(x)$: the expected number of particles per unit length at the point
  $x$. Its integral over the line is the particle number $N$.
- **Harmonic trap**: the external potential $v(x) = x^2/2$, a parabola; a particle in
  it has the levels $1/2, 3/2, 5/2, \dots$ (in the units below).
- **Contact interaction**: two particles feel a repulsion of strength $g_c$ only when
  they are at the same point.
- **Kohn-Sham equations**: one-particle Schroedinger equations in an effective
  potential $v_s(x)$ chosen so that their orbitals give the density of the
  interacting system.
- **Hartree potential**: the potential that one particle feels from the average
```

```
  density of all particles; **exchange potential**: the correction that comes from
  the Pauli principle (equal-label particles avoid each other).
- **Mean-field potential** $w(x)$: here the Hartree plus exchange potential, so that
  $v_s = v + w$.
- **Self-consistency**: the potential makes the orbitals, the orbitals make the
  density, the density makes the potential; a solution is self-consistent when the
  potential that comes out equals the potential that went in.
- **Iteration, residual**: one pass around the loop is an iteration; the residual is
  the largest difference between the potential that comes out and the one that went
  in.
- **Mixing**: feeding only part of the change back (linear mixing) or a combination
  of several earlier passes (Anderson mixing) to make the loop converge.
- **Grid, finite differences**: the line is replaced by equally spaced points; the
  second derivative is replaced by the difference formula
  $(f_{k+1} - 2 f_k + f_{k-1})/h^2$.
- **Eigenvalue, eigenvector**: for a matrix $A$, a number $\epsilon$ and a column
  $u$ with $A u = \epsilon u$; numpy's `eigh` finds all of them for a symmetric
  matrix.
- **Thomas-Fermi approximation**: the kinetic energy of each small piece of the
  line is taken from a uniform gas of the same density.
- **HOMO, LUMO**: the highest occupied and the lowest unoccupied orbital.
- **Delta-SCF**: an excited state computed as a second self-consistent solution in
  which one particle is moved by hand from the HOMO to the LUMO.
- **Units**: $\hbar = m = \omega = 1$, where $\omega$ is the angular frequency of the
  trap: energies are measured in units of $\hbar\omega$, lengths in units of
  $\sqrt{\hbar/(m\omega)}$, and $g_c$ in units of $\hbar\omega$ times that length.
```

Text cell:

```
## 4. The physical and mathematical situation

**The model.** $N = 8$ fermions on a line, four with label up and four with label
down, in the trap $v(x) = x^2/2$, with the contact repulsion
$w(x, x') = g_c\,\delta(x - x')$ of strength $g_c = 2$. The exact ground state would
be a wave function of eight variables; Kohn-Sham theory replaces it by orbitals of
one variable. (In this notebook the letter $x$ is the position on the line of the
toy model; it is not one of the author's spacetime coordinates $x_0, \dots, x_7$.)

**The Kohn-Sham equations of this model.** The orbitals obey

$$-\frac{1}{2}\,\frac{d^2\phi_a}{dx^2} + v_s(x)\,\phi_a(x) = \epsilon_a\,\phi_a(x),$$

and the four lowest orbitals are each occupied by one up and one down fermion, so

$$n(x) = 2 \sum_{a=0}^{3} \phi_a(x)^2, \qquad n_{up} = n_{down} = n/2 .$$

The energy is $E = T_s + \int v\,n\,dx + E_H + E_x$ with the kinetic energy of the
orbitals $T_s = 2\sum_{a=0}^{3} \int \phi_a\,(-\tfrac12\,d^2\phi_a/dx^2)\,dx$, the
Hartree energy $E_H = \tfrac{g_c}{2}\int n^2\,dx$ and the exchange energy
$E_x = -\tfrac{g_c}{2}\int (n_{up}^2 + n_{down}^2)\,dx = -\tfrac{g_c}{4}\int n^2\,dx$.
For a contact interaction the exchange energy of a Slater determinant is EXACTLY this
local formula (Notebook 13b checks it); the correlation energy is left out. So this
is an "exchange-only" Kohn-Sham scheme, which for a contact interaction is the same
```

```
as the Hartree-Fock approximation: an approximation to the true ground state.

**The Kohn-Sham potential.** The potential is the derivative of the energy with
respect to the density (the functional derivative): $\delta E_H/\delta n = g_c n$ and
$\delta E_x/\delta n_{up} = -g_c n_{up} = -g_c n/2$, so

$$v_s(x) = v(x) + g_c\,n(x) - \tfrac{g_c}{2}\,n(x) = v(x) + w(x),
\qquad w(x) = \tfrac{g_c}{2}\,n(x).$$

**Self-consistency.** Given a mean-field potential $w$, solve for the orbitals, form
$n$, and compute $w_{out} = g_c n/2$. A solution is a $w$ with $w_{out} = w$. The
residual of one pass is $r = \max_x |w_{out}(x) - w(x)|$; the loop stops when
$r \le 10^{-11}$, the stopping rule of the Revision Kohn-Sham solver.

**The energy, two ways.** Multiply the Kohn-Sham equation by $\phi_a$, integrate
and sum with the occupations 2: $2\sum_a \epsilon_a = T_s + \int v_s\,n\,dx
= T_s + \int v\,n\,dx + \tfrac{g_c}{2}\int n^2\,dx$. Since
$E_H + E_x = \tfrac{g_c}{4}\int n^2\,dx$, this gives the double-counting formula
$E = 2\sum_{a=0}^{3}\epsilon_a - (E_H + E_x)$, which the notebook compares with the
direct sum of the four energies.

**Status.** The model and its numbers are a teaching example (COMPUTED here). Its
only link to the Revision record is the method: the same Anderson mixing and the same
stopping rule as the Revision solver for the dirac16complex field.
```

Text cell:

```
## 5. The grid and the one-particle Hamiltonian

The line is cut to the interval $-6 < x < 6$ (the orbitals of the lowest levels are
negligibly small at its ends) and replaced by $M = 200$ interior points
$x_k = -6 + k h$, $k = 1, \dots, 200$, with spacing $h = 12/201$. An orbital is the
column of its values at these points, and the orbitals vanish at the two end points
$x = \pm 6$ (hard walls). The second derivative becomes the difference formula, so
the kinetic energy $-\tfrac12\,d^2/dx^2$ becomes the matrix `T` with $1/h^2$ on the
diagonal and $-1/(2h^2)$ just above and just below it.
```

In [2]:

```
import numpy as np  # arrays of numbers, matrices and linear algebra

L_HALF = 6.0  # the grid covers -6 < x < 6
M = 200  # the number of interior grid points
h = 2.0 * L_HALF / (M + 1)  # the grid spacing 12/201
x = -L_HALF + h * np.arange(1, M + 1)  # the grid points x_k, k = 1, ..., 200
v = 0.5 * x ** 2  # the harmonic trap v(x) = x^2/2 at every point

# The kinetic-energy matrix: 1/h^2 on the diagonal, -1/(2 h^2) beside it.
T = (np.diag(np.full(M, 1.0 / h ** 2))
     + np.diag(np.full(M - 1, -0.5 / h ** 2), 1)
     + np.diag(np.full(M - 1, -0.5 / h ** 2), -1))
say(f"grid: {M} points, spacing h = {h:.6f}")
check(np.allclose(T, T.T), "the kinetic-energy matrix is symmetric")
```

Out [2]:

```
grid: 200 points, spacing h = 0.059701
PASS the kinetic-energy matrix is symmetric
```

Text cell:

```
The next cell defines the function ‘orbitals(w)‘: it diagonalises the matrix
$T + \mathrm{diag}(v + w)$ (the Kohn-Sham Hamiltonian on the grid) and returns the
levels in increasing order and the orbitals as the columns of a matrix. numpy returns
eigenvectors of length 1 in the sense $\sum_k u_k^2 = 1$; dividing by $\sqrt h$ makes
$\sum_k \phi_k^2\,h = 1$, the grid form of $\int \phi^2\,dx = 1$. An eigenvector is
fixed only up to its sign; the function makes the first clearly nonzero value of each
orbital (counted from the left) positive, so that every run draws the same pictures.
With $w = 0$ the levels must be close to the exact levels $n + 1/2$ of the trap; the
difference formula makes errors of order $h^2$, so the check allows $0.02$.
```

In [3]:

```
def orbitals(w):
    """Levels (increasing) and orbitals (columns, int phi^2 dx = 1) of T + v + w."""
    levels, vectors = np.linalg.eigh(T + np.diag(v + w))
    phi = vectors / np.sqrt(h)  # normalise to sum phi^2 h = 1
    for a in range(M):  # fix the sign of every orbital
        first = np.argmax(np.abs(phi[:, a]) > 1e-3 * np.abs(phi[:, a]).max())
        phi[:, a] *= np.sign(phi[first, a])
    return levels, phi


free_levels, free_phi = orbitals(np.zeros(M))  # no interaction: w = 0
exact_levels = np.arange(8) + 0.5  # 1/2, 3/2, ..., 15/2
for a in range(8):
    say(f"level {a}: grid {free_levels[a]:.6f}   exact {exact_levels[a]:.1f}")
check(np.max(np.abs(free_levels[:8] - exact_levels)) < 0.02,
      "the grid reproduces the levels n + 1/2 of the trap within 0.02")
overlaps = free_phi[:, :8].T @ free_phi[:, :8] * h  # sum_k phi_a phi_b h
check(np.allclose(overlaps, np.eye(8), atol=1e-12),
      "the orbitals are orthonormal on the grid")
```

Out [3]:

```
level 0: grid 0.499889   exact 0.5
level 1: grid 1.499443   exact 1.5
level 2: grid 2.498551   exact 2.5
level 3: grid 3.497213   exact 3.5
level 4: grid 4.495429   exact 4.5
level 5: grid 5.493197   exact 5.5
level 6: grid 6.490519   exact 6.5
level 7: grid 7.487394   exact 7.5
PASS the grid reproduces the levels n + 1/2 of the trap within 0.02
PASS the orbitals are orthonormal on the grid
```

Text cell:

```
The next cell draws the trap, the six lowest levels as horizontal lines and each
orbital drawn around its own level (the orbital times 0.6, shifted up by its level).
The four lowest levels are the ones that the eight fermions occupy (two per level,
one up and one down).
```

In [4]:

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
ax.plot(x, v, color="black", label="trap $v(x) = x^2/2$")
for a in range(6):
    style = "-" if a < 4 else ":"  # occupied: solid; empty: dotted
    ax.axhline(free_levels[a], color="gray", lw=0.6, ls="--")
    ax.plot(x, free_levels[a] + 0.6 * free_phi[:, a], style,
            label=f"orbital {a}" + (" (occupied)" if a < 4 else " (empty)"))
ax.set_xlim(-5.0, 5.0)
ax.set_ylim(0.0, 8.4)  # room above the orbitals for the legend
ax.set_xlabel("position $x$")
ax.set_ylabel("energy (units of $\\hbar\\omega$)")
ax.set_title("Levels and orbitals of one particle in the trap (no interaction)")
ax.legend(fontsize=7, loc="upper center", ncol=3)
save_figure(fig, "trap_orbitals",
            "The harmonic trap $v(x) = x^2/2$ (black parabola), its six lowest "
            "levels $1/2, 3/2, \\dots, 11/2$ (dashed lines) and the orbital of each "
            "level drawn around it (0.6 times the orbital, shifted up by its level), "
            "computed on the grid without interaction; horizontal axis the position "
            "$x$, vertical axis the energy in units of $\\hbar\\omega$. Orbital $a$ "
            "has $a$ zeros; the four lowest (solid) hold the eight fermions, two per "
            "level, the others (dotted) are empty.")
```

Out [4]:

```
Figure 13a.1 saved as Revision/textbook/figures/13a_1_trap_orbitals.png
```

Text cell:

```
## 6. The Kohn-Sham map: from a mean-field potential to a new one

The next cell defines the parameters of the model and three functions:
‘density(phi)‘ makes $n = 2\sum_{a=0}^{3}\phi_a^2$ from the orbitals,
‘ks_map(w)‘ makes one pass of the loop ($w \to$ orbitals $\to n \to w_{out}$), and
‘integral(f)‘ approximates $\int f\,dx$ by the sum $\sum_k f_k\,h$ (the function
vanishes at the walls, so this simple sum is accurate).
```

In [5]:

```
G_C = 2.0  # the strength g_c of the contact repulsion
N_PER_LABEL = 4  # four fermions with label up and four with label down
```

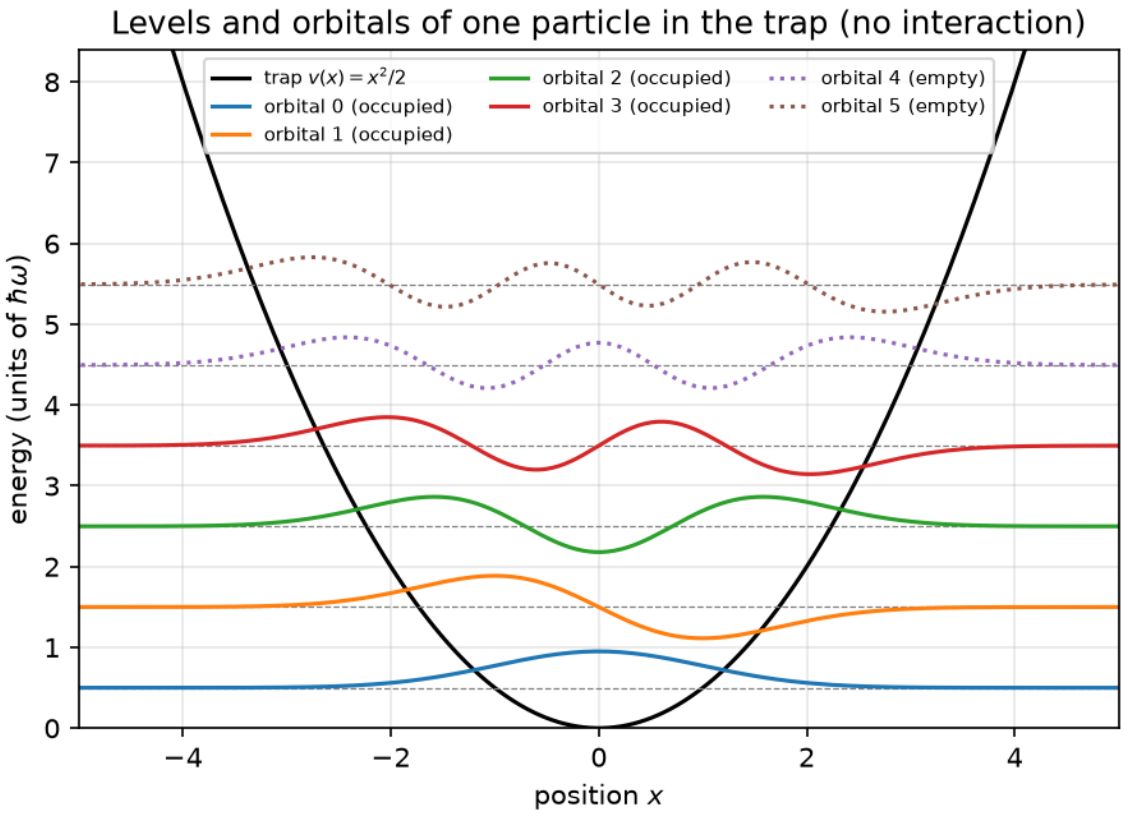

Figure 374: The harmonic trap $v(x) = x^2/2$ (black parabola), its six lowest levels $1/2, 3/2, \ldots, 11/2$ (dashed lines) and the orbital of each level drawn around it (0.6 times the orbital, shifted up by its level), computed on the grid without interaction; horizontal axis the position x, vertical axis the energy in units of $\hbar\omega$. Orbital a has a zeros; the four lowest (solid) hold the eight fermions, two per level, the others (dotted) are empty. (Notebook 13a, figure 1.)

```
N_TOTAL = 2 * N_PER_LABEL


def integral(f):
    """The integral of f over the line, as the sum of f_k h."""
    return float(np.sum(f) * h)


def density(phi):
    """n(x) = 2 (phi_0^2 + phi_1^2 + phi_2^2 + phi_3^2): two fermions per orbital."""
    return 2.0 * np.sum(phi[:, :N_PER_LABEL] ** 2, axis=1)


def ks_map(w):
    """One pass of the loop: the mean-field potential g_c n / 2 made from w."""
    levels, phi = orbitals(w)
    return 0.5 * G_C * density(phi)


w_first = ks_map(np.zeros(M))  # one pass, starting from no interaction
report("largest value of the first mean-field potential", f"{w_first.max():.6f}")
check(abs(integral(2.0 * w_first / G_C) - N_TOTAL) < 1e-10,
      "the density of one pass holds exactly 8 particles")
```

Out [5]:

```
RESULT largest value of the first mean-field potential = 1.887848
PASS the density of one pass holds exactly 8 particles
```

Text cell:

```
## 7. Plain iteration and linear mixing

**Linear mixing** with the mixing parameter $\beta$ replaces $w$ by
$w + \beta\,(w_{out} - w)$; $\beta = 1$ is **plain iteration**
($w \leftarrow w_{out}$). The function `linear_mixing(beta)` starts from $w = 0$,
records the residual of every pass and stops at a residual of at most $10^{-11}$, or
after 150 passes. It returns the last $w$, the list of residuals, and the densities
of the first passes (for a plot below).
```

In [6]:

```
def linear_mixing(beta, tolerance=1e-11, max_passes=150):
    """Linear mixing w <- w + beta (w_out - w), starting from w = 0."""
    w = np.zeros(M)
    residuals, first_densities = [], []
    for _ in range(max_passes):
        w_out = ks_map(w)
        residuals.append(np.max(np.abs(w_out - w)))  # the residual r
        if len(first_densities) < 5:
            first_densities.append(2.0 * w_out / G_C)  # n = 2 w_out / g_c
```

```
        if residuals[-1] <= tolerance:
            break
        w = w + beta * (w_out - w)
    return w, residuals, first_densities


runs = {}
for beta in (1.0, 0.7, 0.3):
    runs[beta] = linear_mixing(beta)
    say(f"beta = {beta}: {len(runs[beta][1])} passes, last residual "
        f"{runs[beta][1][-1]:.1e}")
check(all(run[1][-1] <= 1e-11 for run in runs.values()),
      "plain iteration and linear mixing with beta = 0.7 and 0.3 all converge")
```

Out [6]:

```
beta = 1.0: 40 passes, last residual 9.5e-12
beta = 0.7: 23 passes, last residual 3.8e-12
beta = 0.3: 73 passes, last residual 8.7e-12
PASS plain iteration and linear mixing with beta = 0.7 and 0.3 all converge
```

Text cell:

```
## 8. Anderson mixing with the settings of the Revision solver

**Anderson mixing** remembers the last few inputs $w^{(i)}$ and their residual
vectors $R^{(i)} = w_{out}^{(i)} - w^{(i)}$, finds numbers $c_i$ with
$\sum_i c_i = 1$ that make the combined residual $\sum_i c_i R^{(i)}$ as small as
possible (a least-squares problem), and takes as the next input
$\sum_i c_i\,(w^{(i)} + \beta R^{(i)})$. The Revision Kohn-Sham solver of the
dirac16complex field uses this method; the next cell reads its settings (how many
earlier passes it remembers, its $\beta$ and its tolerance) from the Revision
parameter file and checks them.

What the Revision solver mixes is the list of its potentials at every grid point of
the hidden direction (the mass shift $M_{eff} - m$, the potential $v_v$, and a third
potential used only by one variant), and its residual is, as here, the largest
difference between the potentials that come out and those that went in. It solves
the same least-squares problem in an equivalent form (with a Lagrange multiplier
for the condition that the $c_i$ add up to 1) and adds two safeguards that this
small toy does not need: a tiny number ($10^{-12}$ times the largest diagonal
entry) on the diagonal of its equations, which keeps them solvable, and a restart
of the remembered passes whenever the residual grows to more than ten times the
best one so far.
```

In [7]:

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
numerics = parameters["numerics"]  # the numerical settings of the Revision solver
DEPTH = int(numerics["andersonDepth"])  # how many earlier passes are remembered
```

```
BETA_ANDERSON = float(numerics["andersonBeta"])  # the mixing parameter
TOLERANCE = float(numerics["scfTolerance"])  # the stopping rule
say(f"Revision solver settings: depth {DEPTH}, beta {BETA_ANDERSON}, "
    f"tolerance {TOLERANCE:.0e}")
check(DEPTH == 6 and BETA_ANDERSON == 0.4 and abs(TOLERANCE - 1e-11) < 1e-24,
      "the Revision solver mixes with depth 6, beta 0.4 and stops at 1e-11 "
      "(Revision/kohn_sham/results/parameters.json, numerics)")
```

Out [7]:

```
Revision solver settings: depth 6, beta 0.4, tolerance 1e-11
PASS the Revision solver mixes with depth 6, beta 0.4 and stops at 1e-11
    (Revision/kohn_sham/results/parameters.json, numerics)
```

Text cell:

```
The next cell defines the function ‘anderson(step, w0)‘. It works for any map
‘step‘ (here ‘ks_map‘; below also maps of two potentials at once). With one
remembered pass it is linear mixing; with more, it solves the least-squares problem
with numpy’s ‘lstsq‘: writing $c_i = \theta_i$ for the older passes and
$c_{last} = 1 - \sum_i \theta_i$, the combined residual is
$R^{(last)} + \sum_i \theta_i (R^{(i)} - R^{(last)})$, which is smallest for the
least-squares solution $\theta$ of $\sum_i \theta_i (R^{(i)} - R^{(last)}) \approx
-R^{(last)}$.
```

In [8]:

```
def anderson(step, w0, beta=BETA_ANDERSON, depth=DEPTH, tolerance=TOLERANCE,
             max_passes=400):
    """Anderson mixing for the fixed point step(w) = w, starting from w0."""
    w = np.array(w0, dtype=float)
    inputs, residual_vectors, residuals = [], [], []
    for _ in range(max_passes):
        residual = step(w) - w  # R = w_out - w
        residuals.append(np.max(np.abs(residual)))
        if residuals[-1] <= tolerance:
            return w, residuals
        inputs = (inputs + [w.copy()])[-depth:]  # keep the last ‘depth‘ passes
        residual_vectors = (residual_vectors + [residual])[-depth:]
        if len(inputs) == 1:
            w = w + beta * residual  # the first pass: linear mixing
            continue
        last_r = residual_vectors[-1]
        differences = np.array([r - last_r for r in residual_vectors[:-1]]).T
        theta = np.linalg.lstsq(differences, -last_r, rcond=None)[0]
        c = np.append(theta, 1.0 - theta.sum())  # the c_i; they add up to 1
        w = sum(ci * (wi + beta * ri)
                for ci, wi, ri in zip(c, inputs, residual_vectors))
    raise RuntimeError("Anderson mixing did not converge")
```

```
w_scf, anderson_residuals = anderson(ks_map, np.zeros(M))
say(f"Anderson mixing: {len(anderson_residuals)} passes, last residual "
    f"{anderson_residuals[-1]:.1e}")
differences = [np.max(np.abs(runs[beta][0] - w_scf)) for beta in runs]
check(max(differences) < 1e-9,
      "all four routes reach the same self-consistent potential (within 1e-9)")
check(len(anderson_residuals) < len(runs[1.0][1]),
      "Anderson mixing needs fewer passes than plain iteration")
```

Out [8]:

```
Anderson mixing: 19 passes, last residual 1.8e-12
PASS all four routes reach the same self-consistent potential (within 1e-9)
PASS Anderson mixing needs fewer passes than plain iteration
```

Text cell:

```
The next cell draws the residual of every pass for the four routes, on a logarithmic
vertical axis (each grid line is a factor 10). A straight falling line means that
every pass shrinks the residual by the same factor.
```

In [9]:

```
fig, ax = plt.subplots()
for beta, style in ((1.0, "o-"), (0.7, "s-"), (0.3, "^-")):
    name = "plain iteration" if beta == 1.0 else f"linear mixing, beta = {beta}"
    ax.semilogy(range(1, len(runs[beta][1]) + 1), runs[beta][1], style, ms=3,
                label=f"{name} ({len(runs[beta][1])} passes)")
ax.semilogy(range(1, len(anderson_residuals) + 1), anderson_residuals, "D-",
            ms=3, color="black",
            label=f"Anderson, depth 6, beta 0.4 ({len(anderson_residuals)} passes)")
ax.axhline(TOLERANCE, color="gray", ls="--", lw=0.8)
ax.set_xlabel("pass number")
ax.set_ylabel("residual $\\max_x |w_{out} - w|$")
ax.set_title("Self-consistency: residual versus pass ($g_c = 2$)")
ax.legend(fontsize=8)
save_figure(fig, "scf_convergence",
            "The residual $\\max_x |w_{out}(x) - w(x)|$ of every pass of the "
            "self-consistency loop, logarithmic vertical axis (pure number in "
            "units of $\\hbar\\omega$), horizontal axis the pass number, for plain "
            "iteration, linear mixing with $\\beta = 0.7$ and $0.3$, and Anderson "
            "mixing with the settings of the Revision solver; the dashed line is "
            "the stopping rule $10^{-11}$. Every route reaches the same solution; "
            "the speed depends on $\\beta$, and Anderson mixing is fastest without "
            "any tuning.")
```

Out [9]:

```
Figure 13a.2 saved as Revision/textbook/figures/13a_2_scf_convergence.png
```

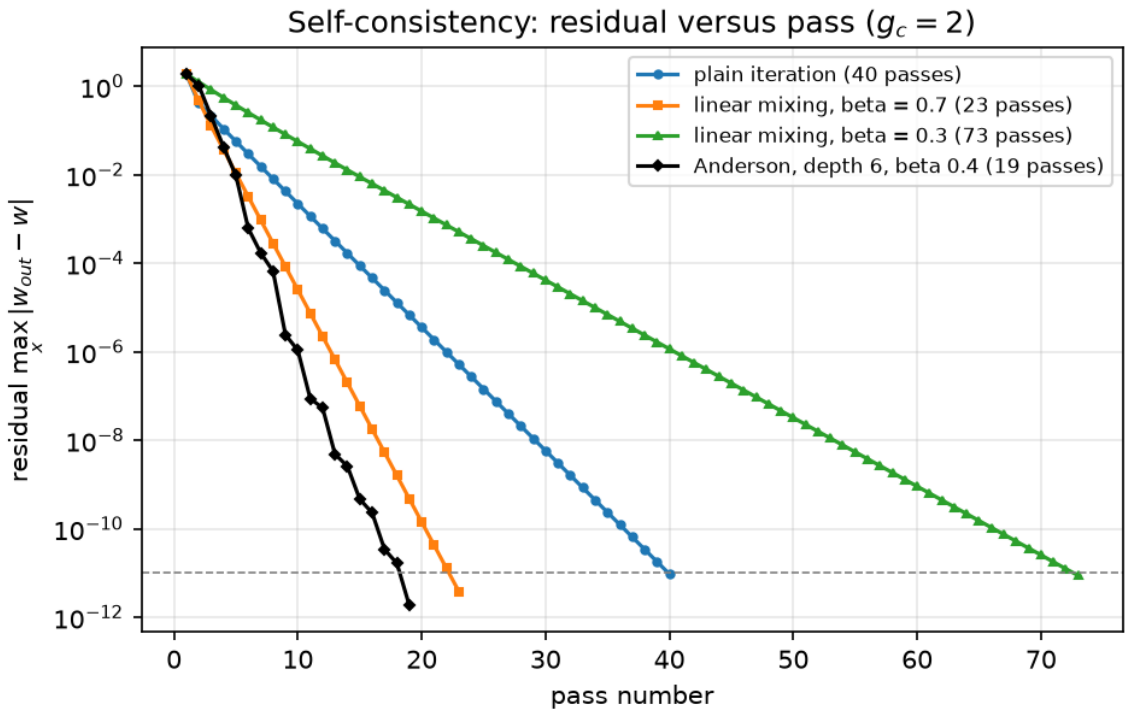

Figure 375: The residual $\max_x |w_{out}(x) - w(x)|$ of every pass of the self-consistency loop, logarithmic vertical axis (pure number in units of $\hbar\omega$), horizontal axis the pass number, for plain iteration, linear mixing with $\beta = 0.7$ and 0.3, and Anderson mixing with the settings of the Revision solver; the dashed line is the stopping rule 10^{-11}. Every route reaches the same solution; the speed depends on β, and Anderson mixing is fastest without any tuning. (Notebook 13a, figure 2.)

Text cell:

```
The next cell shows WHY plain iteration is slow here. It draws the densities that
the first four passes of plain iteration produce, together with the final density.
The first pass starts without interaction, so its density is too narrow; the
repulsion it causes then pushes the next density too far out; and so on: the
density "breathes" in and out around the solution while the swings shrink.
```

In [10]:

```
n_scf = 2.0 * w_scf / G_C  # the self-consistent density n = 2 w / g_c
fig, ax = plt.subplots()
for number, n_pass in enumerate(runs[1.0][2][:4], 1):
    ax.plot(x, n_pass, lw=1.0, label=f"density made by pass {number}")
ax.plot(x, n_scf, color="black", lw=2.0, label="self-consistent density")
ax.set_xlim(-5.0, 5.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("density $n(x)$ (particles per unit length)")
ax.set_title("Plain iteration overshoots: the first passes")
ax.legend(fontsize=8)
save_figure(fig, "first_iterations",
            "The densities made by the first four passes of plain iteration "
            "(thin lines) and the self-consistent density (thick black line); "
            "horizontal axis the position $x$, vertical axis the density in "
            "particles per unit length. Pass 1 starts without repulsion and is too "
            "narrow, pass 2 is too wide, pass 3 too narrow again: the density "
            "swings around the solution, and the swings shrink only slowly.")
check(runs[1.0][2][0].max() > n_scf.max() > runs[1.0][2][1].max(),
      "the first pass is too narrow and the second too wide (overshooting)")
```

Out [10]:

```
Figure 13a.3 saved as Revision/textbook/figures/13a_3_first_iterations.png
PASS the first pass is too narrow and the second too wide (overshooting)
```

Text cell:

```
## 9. The converged Kohn-Sham state

The next cell computes the orbitals of the self-consistent potential, their levels
and the density, and checks the particle number. The highest occupied level (HOMO,
level 3) and the lowest empty one (LUMO, level 4) are printed with the others.
```

In [11]:

```
ks_levels, ks_phi = orbitals(w_scf)
n_ks = density(ks_phi)
for a in range(6):
    status = "occupied by 2" if a < N_PER_LABEL else "empty"
```

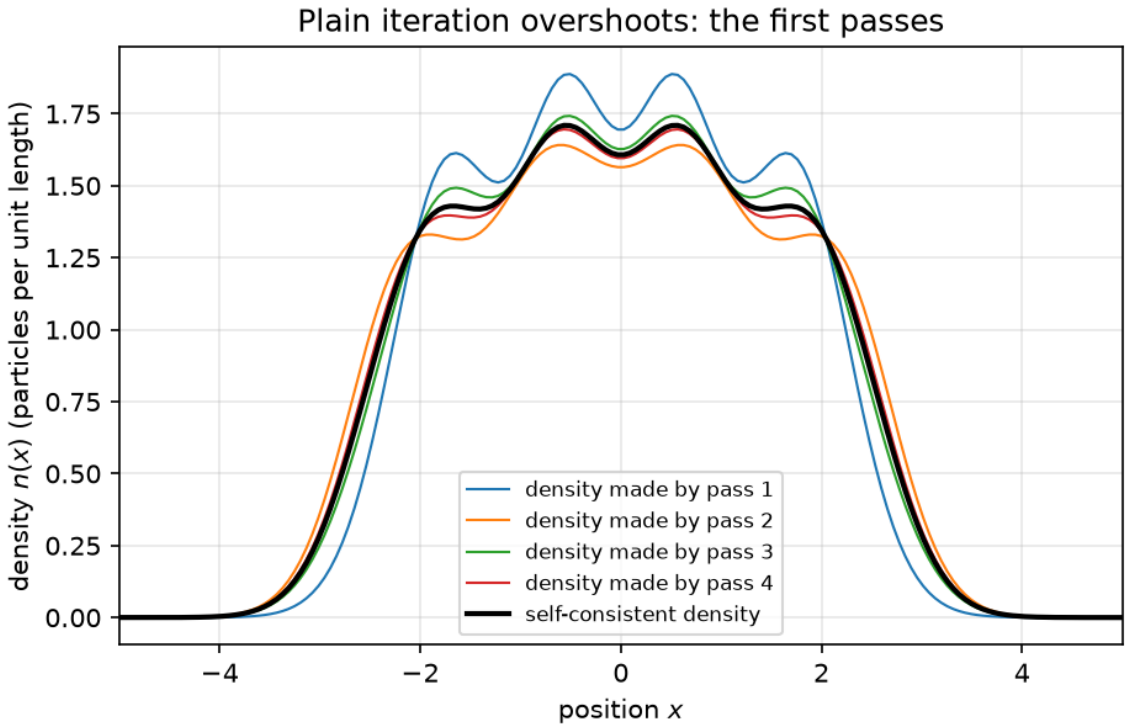

Figure 376: The densities made by the first four passes of plain iteration (thin lines) and the self-consistent density (thick black line); horizontal axis the position x, vertical axis the density in particles per unit length. Pass 1 starts without repulsion and is too narrow, pass 2 is too wide, pass 3 too narrow again: the density swings around the solution, and the swings shrink only slowly. (Notebook 13a, figure 3.)

```
    say(f"Kohn-Sham level {a}: {ks_levels[a]:.6f} ({status})")
report("Kohn-Sham HOMO level", f"{ks_levels[3]:.6f}")
report("Kohn-Sham LUMO level", f"{ks_levels[4]:.6f}")
check(abs(integral(n_ks) - N_TOTAL) < 1e-10, "the density integrates to N = 8")
check(np.max(np.abs(n_ks - n_scf)) < 1e-10,
      "the density of the final orbitals reproduces the input density")
```

Out [11]:

```
Kohn-Sham level 0: 2.119598 (occupied by 2)
Kohn-Sham level 1: 3.006214 (occupied by 2)
Kohn-Sham level 2: 3.877952 (occupied by 2)
Kohn-Sham level 3: 4.723945 (occupied by 2)
Kohn-Sham level 4: 5.496606 (empty)
Kohn-Sham level 5: 6.370592 (empty)
RESULT Kohn-Sham HOMO level = 4.723945
RESULT Kohn-Sham LUMO level = 5.496606
PASS the density integrates to N = 8
PASS the density of the final orbitals reproduces the input density
```

Text cell:

```
The next cell draws the potentials of the converged state: the trap $v$, the Hartree
potential $v_H = g_c n$, the exchange potential $v_x = -g_c n/2$ and their sum
$v_s = v + v_H + v_x$, with the four occupied Kohn-Sham levels.
```

In [12]:

```
v_hartree = G_C * n_ks  # v_H = g_c n
v_exchange = -0.5 * G_C * n_ks  # v_x = -g_c n_up = -g_c n / 2
fig, ax = plt.subplots()
ax.plot(x, v, color="black", label="trap $v$")
ax.plot(x, v_hartree, label="Hartree $v_H = g_c n$")
ax.plot(x, v_exchange, label="exchange $v_x = -g_c n/2$")
ax.plot(x, v + v_hartree + v_exchange, lw=2.0, label="Kohn-Sham $v_s$")
for a in range(N_PER_LABEL):
    ax.axhline(ks_levels[a], color="gray", ls="--", lw=0.6)
ax.set_xlim(-5.0, 5.0)
ax.set_ylim(-3.0, 9.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("potential (units of $\\hbar\\omega$)")
ax.set_title("The potentials of the self-consistent state ($g_c = 2$)")
ax.legend(fontsize=8)
save_figure(fig, "ks_potentials",
            "The potentials of the self-consistent Kohn-Sham state: the trap $v$, "
            "the Hartree potential $v_H = g_c n$, the exchange potential "
            "$v_x = -g_c n/2$ and their sum $v_s$ (thick line), with the four "
            "occupied levels (dashed); horizontal axis the position $x$, vertical "
            "axis the potential in units of $\\hbar\\omega$. The repulsion flattens "
            "the bottom of the trap; exchange removes exactly half of the Hartree "
            "push, because a fermion feels no contact force from fermions with "
```

```
            "its own label.")
check(np.allclose(v + v_hartree + v_exchange, v + w_scf, atol=1e-10),
      "v + v_H + v_x equals the self-consistent v + w")
```

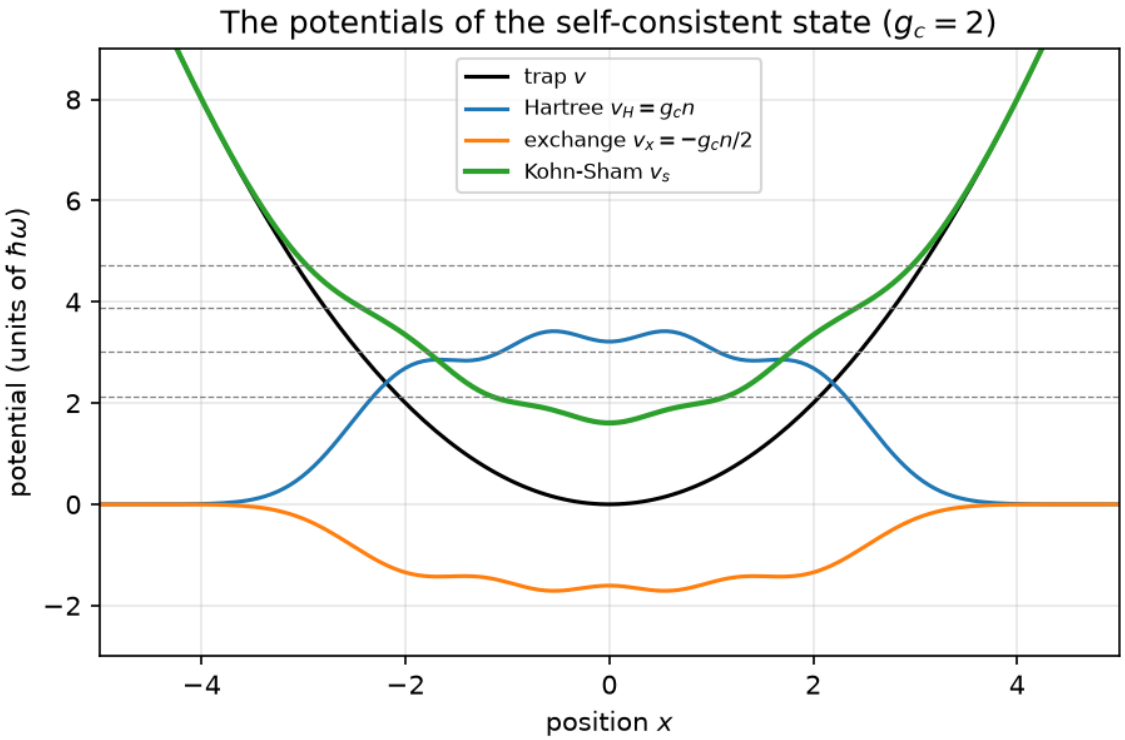

Figure 377: The potentials of the self-consistent Kohn-Sham state: the trap v, the Hartree potential $v_H = g_c n$, the exchange potential $v_x = -g_c n/2$ and their sum v_s (thick line), with the four occupied levels (dashed); horizontal axis the position x, vertical axis the potential in units of $\hbar\omega$. The repulsion flattens the bottom of the trap; exchange removes exactly half of the Hartree push, because a fermion feels no contact force from fermions with its own label. (Notebook 13a, figure 4.)

Out [12]:

```
Figure 13a.4 saved as Revision/textbook/figures/13a_4_ks_potentials.png
PASS v + v_H + v_x equals the self-consistent v + w
```

Text cell:

```
The next cell draws the density as a stack: each occupied orbital contributes
$2\phi_a(x)^2$ (one up and one down fermion), and the four layers add up to $n(x)$.
The four bumps of the density come from the four occupied orbitals (shell
structure).
```

In [13]:

```
fig, ax = plt.subplots()
layers = [2.0 * ks_phi[:, a] ** 2 for a in range(N_PER_LABEL)]
```

```
ax.stackplot(x, layers, labels=[f"$2\\phi_{a}^2$" for a in range(N_PER_LABEL)],
             alpha=0.7)
ax.plot(x, n_ks, color="black", lw=1.5, label="total density $n$")
ax.set_xlim(-5.0, 5.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("density (particles per unit length)")
ax.set_title("The density as the sum of the occupied orbitals")
ax.legend(fontsize=8)
save_figure(fig, "density_orbitals",
            "The self-consistent density $n(x)$ (black line) as a stack of the "
            "contributions $2\\phi_a(x)^2$ of the four occupied Kohn-Sham orbitals "
            "(colored layers, orbital 0 at the bottom); horizontal axis the "
            "position $x$, vertical axis particles per unit length. The area under "
            "each layer is 2 and under the black line 8; the four bumps are the "
            "shell structure of the four occupied orbitals.")
```

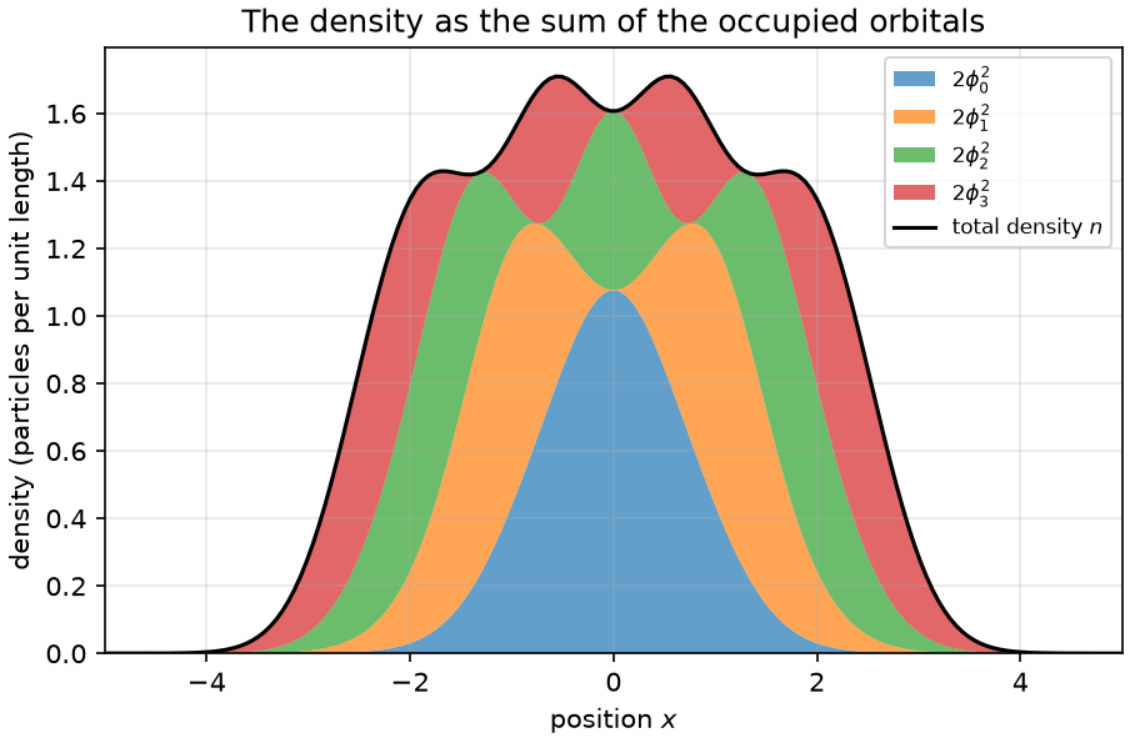

Figure 378: The self-consistent density $n(x)$ (black line) as a stack of the contributions $2\phi_a(x)^2$ of the four occupied Kohn-Sham orbitals (colored layers, orbital 0 at the bottom); horizontal axis the position x, vertical axis particles per unit length. The area under each layer is 2 and under the black line 8; the four bumps are the shell structure of the four occupied orbitals. (Notebook 13a, figure 5.)

Out [13]:

```
Figure 13a.5 saved as Revision/textbook/figures/13a_5_density_orbitals.png
```

Text cell:

```
## 10. The energy, two ways, and the stability of the solution

The next cell computes the four energies $T_s$, $\int v n\,dx$, $E_H$, $E_x$, their
sum $E$, and the double-counting formula $E = 2\sum_a \epsilon_a - (E_H + E_x)$ of
section 4. Both must agree to rounding. It also checks $E_x = -E_H/2$, which holds
exactly when the two labels are equally occupied (the rule $E_x = -E_H/g$ with
$g = 2$ labels).
```

In [14]:

```
def kinetic(phi, occupations):
    """T_s = sum_a f_a int phi_a (-1/2 phi_a'') dx with the occupations f_a."""
    return sum(f * float(phi[:, a] @ (T @ phi[:, a])) * h
               for a, f in enumerate(occupations))


T_s = kinetic(ks_phi, [2.0] * N_PER_LABEL)
E_ext = integral(v * n_ks)
E_H = 0.5 * G_C * integral(n_ks ** 2)
E_x = -0.25 * G_C * integral(n_ks ** 2)
E_total = T_s + E_ext + E_H + E_x
E_double = 2.0 * ks_levels[:N_PER_LABEL].sum() - (E_H + E_x)
for label, value in (("kinetic T_s", T_s), ("external int v n dx", E_ext),
                     ("Hartree E_H", E_H), ("exchange E_x", E_x)):
    say(f"{label:22} = {value:.6f}")
report("total energy E (direct sum)", f"{E_total:.6f}")
report("total energy E (double counting)", f"{E_double:.6f}")
check(abs(E_total - E_double) < 1e-9, "the two energy formulas agree")
check(abs(E_x + 0.5 * E_H) < 1e-12, "E_x = -E_H/2 for two equally occupied labels")
```

Out [14]:

```
kinetic T_s            = 6.726297
external int v n dx    = 9.521279
Hartree E_H            = 11.207843
exchange E_x           = -5.603922
RESULT total energy E (direct sum) = 21.851498
RESULT total energy E (double counting) = 21.851498
PASS the two energy formulas agree
PASS E_x = -E_H/2 for two equally occupied labels
```

Text cell:

```
Section 4 obtained the mean-field potential $w = g_c n/2$ as the **functional
derivative** of the interaction energy $E_H + E_x = \tfrac{g_c}{4}\int n^2\,dx$
with respect to the density. On the grid this has a concrete meaning that the next
cell tests: change the density by a small amount $\epsilon\,\eta(x)$, where $\eta$
is any fixed function (here the off-centre bump $\eta(x) = e^{-(x - 1)^2}$); then
the interaction energy changes, to first order in $\epsilon$, by
$\epsilon\int w\,\eta\,dx$. The cell compares the difference quotient
$(E_{Hx}[n + \epsilon\eta] - E_{Hx}[n - \epsilon\eta])/(2\epsilon)$ with
```

```
$\epsilon = 10^{-4}$ and the integral $\int w\,\eta\,dx$. (Because $E_H + E_x$ is
a square of $n$, the difference quotient is exact here up to rounding.)
```

In [15]:

```
def interaction_energy(n):
    """E_H + E_x = (g_c/4) int n^2 dx for two equally occupied labels."""
    return 0.25 * G_C * integral(n ** 2)


eta = np.exp(-(x - 1.0) ** 2)  # a fixed change of shape, off the centre of the trap
epsilon = 1e-4  # the size of the change
quotient = (interaction_energy(n_ks + epsilon * eta)
            - interaction_energy(n_ks - epsilon * eta)) / (2.0 * epsilon)
w_ks = 0.5 * G_C * n_ks  # the mean-field potential w = g_c n / 2
report("difference quotient of E_H + E_x along eta", f"{quotient:.10f}")
report("integral of w eta dx", f"{integral(w_ks * eta):.10f}")
check(abs(quotient - integral(w_ks * eta)) < 1e-9,
      "w = g_c n/2 is the functional derivative of E_H + E_x")
```

Out [15]:

```
RESULT difference quotient of E_H + E_x along eta = 2.6845567944
RESULT integral of w eta dx = 2.6845567944
PASS w = g_c n/2 is the functional derivative of E_H + E_x
```

Text cell:

```
So far the up and down densities were forced to be equal. Is that solution stable,
or would the fermions lower their energy by separating the labels? The next cell
lets the two labels have their own potentials: a fermion with label up feels the
Hartree potential of everybody minus the exchange with its own label,
$v_{up} = v + g_c n - g_c n_{up} = v + g_c n_{down}$, and the same with up and down
exchanged. It starts from a strongly separated guess (up pushed to the left, down to
the right) and runs Anderson mixing on both potentials together. At $g_c = 2$ the
loop returns to equal densities and to the same energy. This return does NOT yet
show that the equal-label solution is stable: the two cells after it show why, and
make the test that does.
```

In [16]:

```
def label_densities(w_pair, up_occupations, down_occupations):
    """The densities n_up, n_down and the levels for the pair of potentials."""
    up_levels, up_phi = orbitals(w_pair[:M])  # w_up: the first M numbers
    down_levels, down_phi = orbitals(w_pair[M:])  # w_down: the last M numbers
    n_up = (up_phi[:, :len(up_occupations)] ** 2) @ np.array(up_occupations)
    n_down = (down_phi[:, :len(down_occupations)] ** 2) @ np.array(down_occupations)
    return n_up, n_down, up_levels, up_phi, down_levels, down_phi
```

```
def labels_map(w_pair, up_occupations, down_occupations):
    """One pass for two labels: w_up = g_c n_down and w_down = g_c n_up."""
    n_up, n_down = label_densities(w_pair, up_occupations, down_occupations)[:2]
    return np.concatenate([G_C * n_down, G_C * n_up])


def labels_energy(w_pair, up_occupations, down_occupations):
    """E = T_s + int v n + g_c int n_up n_down (E_H + E_x for two labels)."""
    n_up, n_down, _, up_phi, _, down_phi = label_densities(
        w_pair, up_occupations, down_occupations)
    return (kinetic(up_phi, up_occupations) + kinetic(down_phi, down_occupations)
            + integral(v * (n_up + n_down)) + G_C * integral(n_up * n_down))


FILLED = [1.0] * N_PER_LABEL  # the four lowest orbitals of each label occupied
push = 1.0 * np.tanh(x)  # pushes up-fermions to the left, down-fermions to the right
start = np.concatenate([w_scf + push, w_scf - push])
w_pair, pair_residuals = anderson(lambda q: labels_map(q, FILLED, FILLED), start)
n_up, n_down = label_densities(w_pair, FILLED, FILLED)[:2]
say(f"two-label run: {len(pair_residuals)} passes; largest |n_up - n_down| = "
    f"{np.max(np.abs(n_up - n_down)):.1e}")
check(np.max(np.abs(n_up - n_down)) < 1e-8,
      "from a separated start the two-label loop returns to equal densities")
check(abs(labels_energy(w_pair, FILLED, FILLED) - E_total) < 1e-9,
      "the two-label run has the same energy as the equal-label solution")
```

Out [16]:

```
two-label run: 36 passes; largest |n_up - n_down| = 4.9e-12
PASS from a separated start the two-label loop returns to equal densities
PASS the two-label run has the same energy as the equal-label solution
```

Text cell:

```
Why the return of the loop proves nothing about stability: the loop stops wherever
the potentials that come out equal those that went in, that is at EVERY
self-consistent solution. A self-consistent solution makes the energy stationary
(no change to first order); a minimum is stationary, but so is a saddle, where
some changes raise the energy and others lower it. Anderson mixing only looks for
a point where the residual vanishes, and it can land on a saddle. The next cell
shows this on the smallest example: two sites L and R joined by the hopping
$t = 1$, one fermion with label up and one with label down, and the repulsion
$U = 4$ when both sit on the same site. Each label has its own potential
$(w_L, w_R)$ on the two sites; its orbital $(c_L, c_R)$ is the eigenvector of the
lower level of the $2 \times 2$ matrix with $w_L, w_R$ on the diagonal and $-t$
beside it; the energy of the determinant is the hopping energy $-2t\,c_L c_R$ of
each orbital plus $U(n_{L,up}\,n_{L,down} + n_{R,up}\,n_{R,down})$; and one pass of
the loop gives label up the potential $U n_{down}$ and label down $U n_{up}$,
exactly as in the trap. Equal labels, $n = (1/2, 1/2)$ for each label, are
self-consistent, with the energy $-2t + U/2 = 0$; but for $U > 2t$ the lowest
determinant puts the two labels on different sites, with the energy
```

```
$-2t^2/U = -0.5$. The cell runs the SAME function ‘anderson‘ from a slightly
separated start (push 0.5) and from a more strongly separated one (push 1), and
computes the energy along the family of separated potentials
$U/2 \pm \epsilon\,(-1, 1)$.
```

In [17]:

```
T_HOP, U_SITE = 1.0, 4.0  # the hopping t and the on-site repulsion U of two sites
left_right = np.array([-1.0, 1.0])  # lowers the potential on L and raises it on R


def site_orbital(w2):
    """(c_L, c_R): the orbital of the lower level of [[w_L, -t], [-t, w_R]]."""
    return np.linalg.eigh(np.array([[w2[0], -T_HOP], [-T_HOP, w2[1]]]))[1][:, 0]


def sites_map(q):
    """One pass on two sites, q = (w_up on L, R, w_down on L, R)."""
    n_up, n_down = site_orbital(q[:2]) ** 2, site_orbital(q[2:]) ** 2
    return np.concatenate([U_SITE * n_down, U_SITE * n_up])


def sites_energy(q):
    """-2t (c_L c_R of up + c_L c_R of down) + U sum_sites n_up n_down."""
    c_up, c_down = site_orbital(q[:2]), site_orbital(q[2:])
    return (-2.0 * T_HOP * (c_up[0] * c_up[1] + c_down[0] * c_down[1])
            + U_SITE * np.sum(c_up ** 2 * c_down ** 2))


def sites_start(size):
    """The potentials U/2 + size (-1, 1) for label up and U/2 - size (-1, 1)."""
    return np.concatenate([0.5 * U_SITE + size * left_right,
                           0.5 * U_SITE - size * left_right])


site_ends = {}
for site_push in (0.5, 1.0):
    q_end, site_residuals = anderson(sites_map, sites_start(site_push))
    site_ends[site_push] = q_end
    n_up_sites = site_orbital(q_end[:2]) ** 2
    energy = np.round(sites_energy(q_end), 9) + 0.0  # + 0.0 turns -0.0 into 0.0
    say(f"two sites, push {site_push}: {len(site_residuals)} passes, n_up = "
        f"({n_up_sites[0]:.6f}, {n_up_sites[1]:.6f}), energy {energy:.6f}")
site_sizes = np.linspace(0.0, 2.5, 51)  # epsilon = 0, 0.05, ..., 2.5
site_family = np.array([sites_energy(sites_start(size)) for size in site_sizes])
saddle_up = site_orbital(site_ends[0.5][:2]) ** 2
check(np.max(np.abs(saddle_up - 0.5)) < 1e-9
      and abs(sites_energy(site_ends[0.5]) - (-2.0 * T_HOP + 0.5 * U_SITE)) < 1e-9,
      "two sites, U = 4t, push 0.5: the loop converges to equal labels, E = -2t + U/2")
check(abs(sites_energy(site_ends[1.0]) + 2.0 * T_HOP ** 2 / U_SITE) < 1e-9,
      "two sites, push 1: the loop converges to separated labels, E = -2t^2/U")
check(site_family[1] < site_family[0] - 1e-3,
      "two sites: separating the labels lowers the energy, so the equal-label "
```

```
      "solution there is a saddle")
```

Out [17]:

```
two sites, push 0.5: 16 passes, n_up = (0.500000, 0.500000), energy 0.000000
two sites, push 1.0: 18 passes, n_up = (0.933013, 0.066987), energy -0.500000
PASS two sites, U = 4t, push 0.5: the loop converges to equal labels, E = -2t + U/2
PASS two sites, push 1: the loop converges to separated labels, E = -2t^2/U
PASS two sites: separating the labels lowers the energy, so the equal-label solution
    there is a saddle
```

Text cell:

```
So a converged loop can sit on a saddle, and only the ENERGY tells a minimum from
a saddle: at a minimum every small change raises it, at a saddle some change
lowers it. The next cell makes this energy test in the trap. It builds
label-separating trial potentials $w_{up} = w_{scf} + \epsilon\,s(x)$ and
$w_{down} = w_{scf} - \epsilon\,s(x)$ for three shapes $s$: $\tanh x$ (up to the
left, down to the right), $x\,e^{-x^2/4}$ (the same, but only near the centre) and
$e^{-x^2/2}$ (up pushed outwards, down inwards), and the five sizes
$\epsilon = 0.01, 0.05, 0.2, 0.5, 1$. For each it takes the four lowest orbitals of
each label in its trial potential and evaluates the energy of this determinant with
‘labels_energy‘, which needs no self-consistency (for a contact interaction this
formula is the exact energy of the determinant). Every value must lie above the
Kohn-Sham energy; and for small $\epsilon$ the rise must be of second order, so
that five times the size gives about 25 times the rise. The cell then draws the
energies of the trap and of the two sites against $\epsilon$.
```

In [18]:

```
shapes = {"tanh x": np.tanh(x), "x exp(-x^2/4)": x * np.exp(-x ** 2 / 4.0),
          "exp(-x^2/2)": np.exp(-x ** 2 / 2.0)}


def separation_rise(shape, size):
    """E of the determinant made from w_scf +- size * shape, minus E_KS."""
    trial = np.concatenate([w_scf + size * shape, w_scf - size * shape])
    return labels_energy(trial, FILLED, FILLED) - E_total


sizes = [0.01, 0.05, 0.2, 0.5, 1.0]
rises = {name: [separation_rise(shape, size) for size in sizes]
         for name, shape in shapes.items()}
for name, values in rises.items():
    say(f"{name:14} E - E_KS: " + " ".join(f"{value:.2e}" for value in values))
check(all(value > 0.0 for values in rises.values() for value in values),
      "every label-separating trial determinant in the trap has a higher energy "
      "than the Kohn-Sham state")
check(all(24.0 < values[1] / values[0] < 26.0 for values in rises.values()),
      "for small separations the energy rises as epsilon^2: in the trap the "
      "equal-label solution is a minimum along these families, not a saddle")
```

```
fine_sizes = np.linspace(0.0, 1.0, 41)  # epsilon = 0, 0.025, ..., 1
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
for (name, shape), style in zip(shapes.items(), ("-", "--", "-.")):
    left.plot(fine_sizes, [separation_rise(shape, size) for size in fine_sizes],
              style, label=f"shape {name}")
left.set_xlabel("size $\\epsilon$ of the separation")
left.set_ylabel("$E - E_{KS}$ ($\\hbar\\omega$)")
left.set_title("Trap, $g_c = 2$: the energy rises (minimum)")
left.legend(fontsize=8)
right.plot(site_sizes, site_family, color="black", label="energy of the family")
right.plot([0.0], [site_family[0]], "o", ms=8,
           label="loop from push 0.5 (saddle)")
size_end = 0.5 * (site_ends[1.0][1] - site_ends[1.0][0])  # (w_R - w_L)/2 of up
right.plot([size_end], [sites_energy(site_ends[1.0])], "s", ms=8,
           label="loop from push 1 (minimum)")
right.set_xlabel("size $\\epsilon$ of the separation")
right.set_ylabel("$E$ (units of $t$)")
right.set_title("Two sites, $U = 4t$: the energy falls (saddle)")
right.legend(fontsize=8)
save_figure(fig, "label_separation",
            "The energy test of stability. Left: the trap with eight fermions; "
            "the energy of the determinant made from the label-separating trial "
            "potentials $w_{scf} \\pm \\epsilon\\,s(x)$, minus the Kohn-Sham "
            "energy, for three shapes $s$, against the size $\\epsilon$ (pure "
            "number); vertical axis in units of $\\hbar\\omega$. Every curve starts "
            "flat at 0 and rises: along these families the equal-label solution "
            "is a minimum. Right: two sites with $U = 4t$; the energy of the "
            "determinant made from the potentials $U/2 \\pm \\epsilon\\,(-1, 1)$, "
            "in units of $t$, against $\\epsilon$. It falls from the equal-label "
            "solution (circle, energy 0, where the loop from push 0.5 converged: "
            "a saddle) to the separated minimum $-2t^2/U = -0.5t$ (square, where "
            "the loop from push 1 converged). The converged loop could not tell "
            "the two cases apart; the energy does.")
```

Out [18]:

```
tanh x         E - E_KS: 3.50e-05 8.81e-04 1.53e-02 1.24e-01 5.93e-01
x exp(-x^2/4)  E - E_KS: 2.14e-05 5.35e-04 8.67e-03 5.73e-02 2.54e-01
exp(-x^2/2)    E - E_KS: 8.58e-06 2.15e-04 3.43e-03 2.15e-02 8.71e-02
PASS every label-separating trial determinant in the trap has a higher energy than the
    Kohn-Sham state
PASS for small separations the energy rises as epsilon^2: in the trap the equal-label
    solution is a minimum along these families, not a saddle
```

Out [18]:

```
Figure 13a.6 saved as Revision/textbook/figures/13a_6_label_separation.png
```

Text cell:

```
## 11. The variational principle at work
```

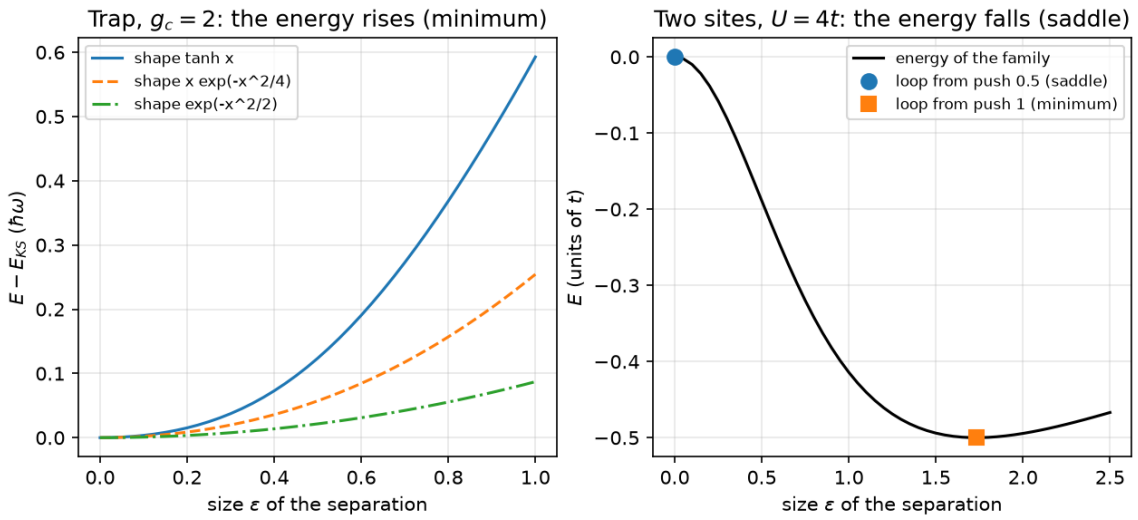

Figure 379: The energy test of stability. Left: the trap with eight fermions; the energy of the determinant made from the label-separating trial potentials $w_{scf} \pm \epsilon\, s(x)$, minus the Kohn-Sham energy, for three shapes s, against the size ϵ (pure number); vertical axis in units of $\hbar\omega$. Every curve starts flat at 0 and rises: along these families the equal-label solution is a minimum. Right: two sites with $U = 4t$; the energy of the determinant made from the potentials $U/2 \pm \epsilon\,(-1, 1)$, in units of t, against ϵ. It falls from the equal-label solution (circle, energy 0, where the loop from push 0.5 converged: a saddle) to the separated minimum $-2t^2/U = -0.5t$ (square, where the loop from push 1 converged). The converged loop could not tell the two cases apart; the energy does. (Notebook 13a, figure 6.)

```
The Kohn-Sham equations were derived by asking that the energy $E$ be stationary
when the orbitals change (with their normalisation kept): a small change of the
self-consistent orbitals changes $E$ only to second order, and for the ground state
the stationary point is a minimum. The next cell tests this on a one-parameter
family of determinants: for each number $c$ it takes the four lowest orbitals of the
potential $v + c\,n_{scf}$, puts two fermions into each, and evaluates the SAME
energy formula $E = T_s + \int v n\,dx + \tfrac{g_c}{4}\int n^2\,dx$ for them. At
$c = g_c/2 = 1$ the orbitals are exactly the self-consistent ones. The energy must be
smallest there, and larger for every other $c$.
```

In [19]:

```
def energy_of_orbitals(phi):
    """E = T_s + int v n dx + (g_c/4) int n^2 dx for 2 fermions in orbitals 0..3."""
    n = density(phi)
    return (kinetic(phi, [2.0] * N_PER_LABEL) + integral(v * n)
            + 0.25 * G_C * integral(n ** 2))


c_values = np.linspace(0.0, 2.0, 41)  # c = 0, 0.05, ..., 2
family_energies = np.array([energy_of_orbitals(orbitals(c * n_scf)[1])
                            for c in c_values])
c_best = c_values[np.argmin(family_energies)]
report("c of the lowest energy in the family", f"{c_best:.2f}")
check(abs(c_best - 0.5 * G_C) < 1e-9,
      "the lowest energy of the family is at c = g_c/2 (the self-consistent one)")
check(np.all(family_energies >= E_total - 1e-10),
      "no member of the family has a lower energy than the Kohn-Sham state")
```

Out [19]:

```
RESULT c of the lowest energy in the family = 1.00
PASS the lowest energy of the family is at c = g_c/2 (the self-consistent one)
PASS no member of the family has a lower energy than the Kohn-Sham state
```

Text cell:

```
The next cell draws the energy of the family against $c$. Near its lowest point the
curve is flat (a parabola): a small error in the orbitals makes only a much smaller
(second-order) error in the energy, a general property of variational methods.
```

In [20]:

```
fig, ax = plt.subplots()
ax.plot(c_values, family_energies - E_total, "o-", ms=3,
        label="$E(c) - E_{KS}$")
ax.axvline(0.5 * G_C, color="gray", ls="--", label="$c = g_c/2$ (self-consistent)")
ax.set_xlabel("$c$ in the trial potential $v + c\\,n_{scf}$")
ax.set_ylabel("energy above the Kohn-Sham energy ($\\hbar\\omega$)")
```

```
ax.set_title("Variational principle: a family of trial determinants")
ax.legend()
save_figure(fig, "variational_scan",
            "The energy $E(c)$ of the determinant built from the four lowest "
            "orbitals of the trial potential $v + c\\,n_{scf}$, minus the "
            "Kohn-Sham energy, for $c$ from 0 to 2; horizontal axis $c$ (the same "
            "units as $g_c$), vertical axis the energy difference in units of "
            "$\\hbar\\omega$. The curve is never negative and touches zero at the "
            "self-consistent value $c = g_c/2 = 1$ (dashed line), where it is flat: "
            "the variational principle.")
```

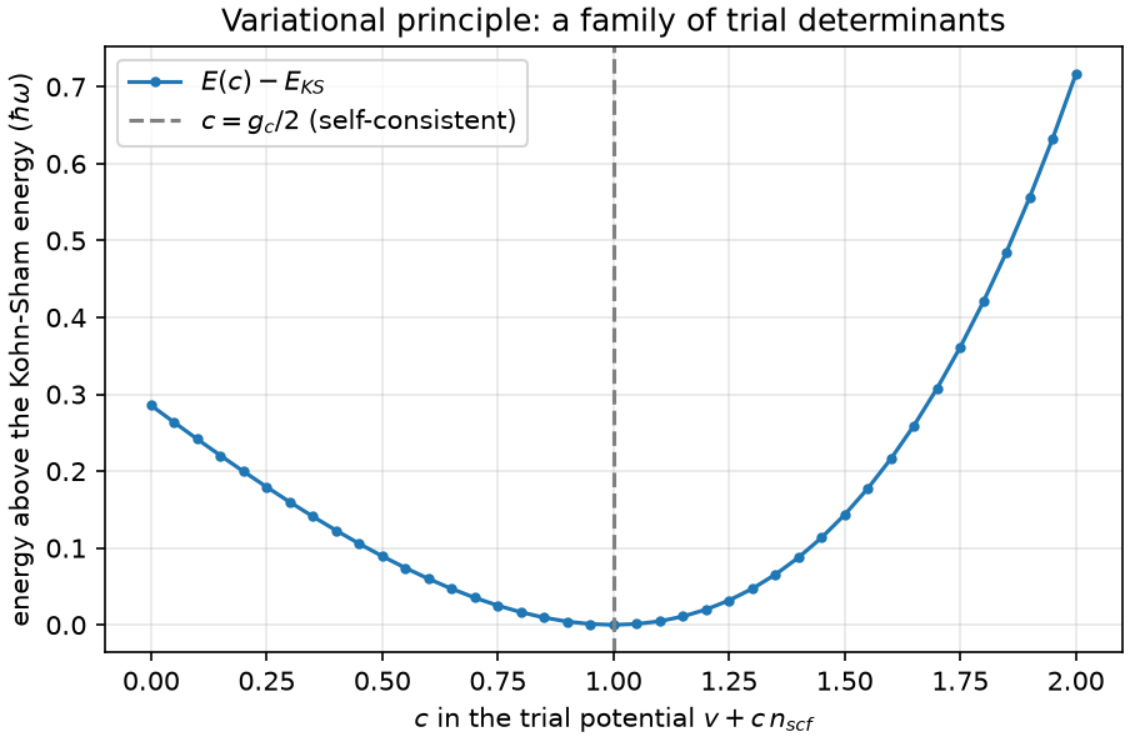

Figure 380: The energy $E(c)$ of the determinant built from the four lowest orbitals of the trial potential $v + c\,n_{scf}$, minus the Kohn-Sham energy, for c from 0 to 2; horizontal axis c (the same units as g_c), vertical axis the energy difference in units of $\hbar\omega$. The curve is never negative and touches zero at the self-consistent value $c = g_c/2 = 1$ (dashed line), where it is flat: the variational principle. (Notebook 13a, figure 7.)

Out [20]:

```
Figure 13a.7 saved as Revision/textbook/figures/13a_7_variational_scan.png
```

Text cell:

```
## 12. Three simpler pictures compared

The next cell computes two other densities for comparison. **Hartree only** drops
```

```
the exchange term, so the potential is $v + g_c n$: every fermion is also repelled
by its own density (self-interaction). The **Thomas-Fermi** density uses the
uniform-gas kinetic energy at the local density instead of the orbitals. For a
uniform gas on a line with two labels, the states with wave numbers
$|k| < k_F$ are filled, two per state, so $n = 2k_F/\pi$, and the kinetic energy
per unit length is $2\int_{-k_F}^{k_F} \frac{dk}{2\pi}\frac{k^2}{2} =
k_F^3/(3\pi) = \pi^2 n^3/24$. Minimising
$\int [\pi^2 n^3/24 + v n + \tfrac{g_c}{4} n^2]\,dx$ at fixed $N$ (Lagrange
multiplier $\mu$) gives $\pi^2 n^2/8 + v + g_c n/2 = \mu$ where $n > 0$, a
quadratic equation for $n$ at each point; $\mu$ is found by bisection so that the
density holds 8 particles.
```

In [21]:

```
def hartree_map(w):
    """One pass of the Hartree approximation: w_out = g_c n (no exchange)."""
    return G_C * density(orbitals(w)[1])


w_hartree, hartree_residuals = anderson(hartree_map, np.zeros(M))
n_hartree = density(orbitals(w_hartree)[1])
n_free = density(free_phi)  # no interaction at all


def thomas_fermi(mu):
    """The Thomas-Fermi density: the positive root of a n^2 + b n = mu - v."""
    a, b = np.pi ** 2 / 8.0, 0.5 * G_C
    room = np.maximum(mu - v, 0.0)  # zero where the trap is higher than mu
    return (-b + np.sqrt(b * b + 4.0 * a * room)) / (2.0 * a)


low, high = 0.0, 50.0  # mu lies between these two numbers
for _ in range(60):  # bisection: halve the interval 60 times
    middle = 0.5 * (low + high)
    if integral(thomas_fermi(middle)) < N_TOTAL:
        low = middle  # too few particles: mu must be larger
    else:
        high = middle
mu_tf = 0.5 * (low + high)
n_tf = thomas_fermi(mu_tf)
report("Thomas-Fermi chemical potential mu", f"{mu_tf:.6f}")
check(abs(integral(n_tf) - N_TOTAL) < 1e-9, "the Thomas-Fermi density holds 8")
check(abs(integral(n_hartree) - N_TOTAL) < 1e-10, "the Hartree density holds 8")
widths = [np.sqrt(integral(x ** 2 * n) / N_TOTAL)
          for n in (n_free, n_ks, n_hartree)]
say("root-mean-square widths: free {:.6f}, Kohn-Sham {:.6f}, Hartree {:.6f}"
    .format(*widths))
check(widths[0] < widths[1] < widths[2],
      "repulsion widens the cloud, and self-interaction (Hartree) widens it more")
```

Out [21]:

```
RESULT Thomas-Fermi chemical potential mu = 5.110729
PASS the Thomas-Fermi density holds 8
PASS the Hartree density holds 8
root-mean-square widths: free 1.413346, Kohn-Sham 1.542829, Hartree 1.659857
PASS repulsion widens the cloud, and self-interaction (Hartree) widens it more
```

Text cell:

```
The next cell draws the four densities together.
```

In [22]:

```
fig, ax = plt.subplots()
ax.plot(x, n_free, ":", label="no interaction")
ax.plot(x, n_hartree, "--", label="Hartree only (self-interaction)")
ax.plot(x, n_ks, lw=2.0, color="black", label="Kohn-Sham (Hartree + exchange)")
ax.plot(x, n_tf, "-.", label="Thomas-Fermi (local kinetic energy)")
ax.set_xlim(-5.0, 5.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("density (particles per unit length)")
ax.set_title("Four pictures of the same eight fermions ($g_c = 2$)")
ax.legend(fontsize=8)
save_figure(fig, "approximations",
            "Four densities of the eight fermions in the trap: without "
            "interaction (dotted), the Hartree approximation that keeps the "
            "self-interaction (dashed), Kohn-Sham with Hartree and exact exchange "
            "(thick black) and Thomas-Fermi with the uniform-gas kinetic energy "
            "(dash-dotted); horizontal axis the position $x$, vertical axis "
            "particles per unit length. The repulsion spreads the cloud; Hartree "
            "overdoes it; Thomas-Fermi follows the Kohn-Sham density on average but "
            "misses its four shell bumps.")
```

Out [22]:

```
Figure 13a.8 saved as Revision/textbook/figures/13a_8_approximations.png
```

Text cell:

```
## 13. Switching the interaction on

The next cell solves the model for nine strengths $g_c = 0, 0.25, \dots, 2$, each
time starting from the solution of the previous strength, and records the energies.
It also checks the **Hellmann-Feynman theorem**: because the energy is stationary in
the orbitals, its derivative with respect to $g_c$ is only the explicit one,
$dE/dg_c = \tfrac14\int n^2\,dx$; the check compares this with the difference
quotient $(E(g_c + \delta) - E(g_c - \delta))/(2\delta)$ at $g_c = 2$,
$\delta = 10^{-3}$.
```

In [23]:

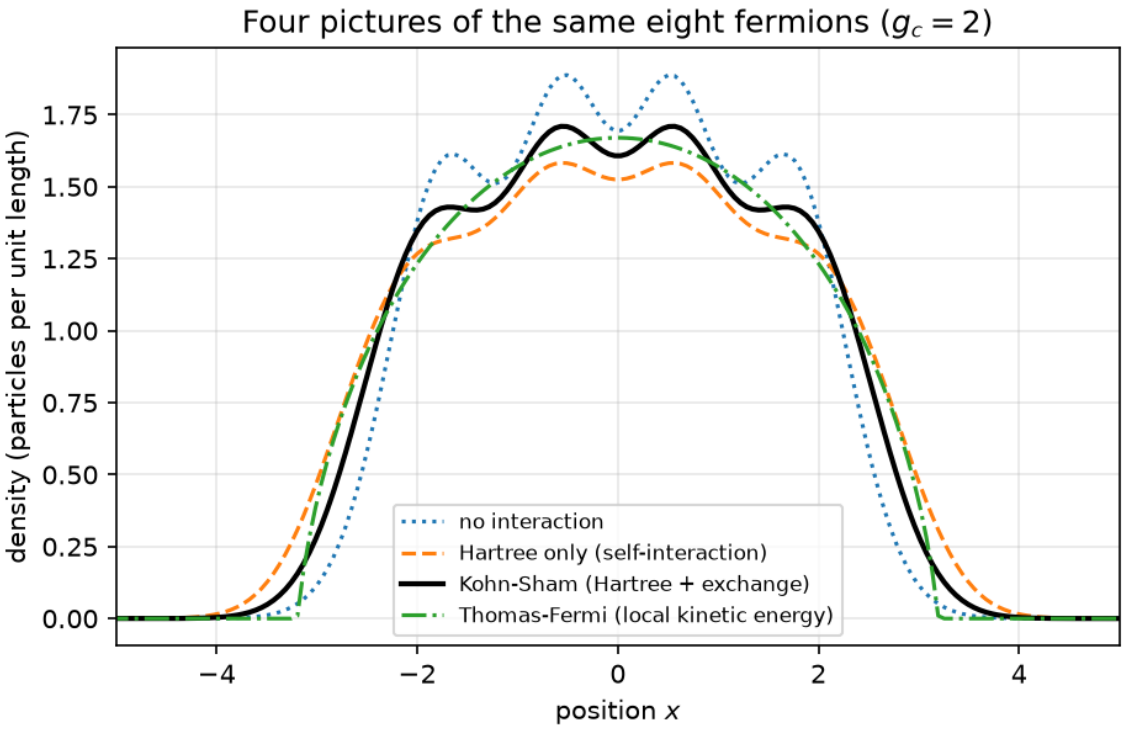

Figure 381: Four densities of the eight fermions in the trap: without interaction (dotted), the Hartree approximation that keeps the self-interaction (dashed), Kohn-Sham with Hartree and exact exchange (thick black) and Thomas-Fermi with the uniform-gas kinetic energy (dash-dotted); horizontal axis the position x, vertical axis particles per unit length. The repulsion spreads the cloud; Hartree overdoes it; Thomas-Fermi follows the Kohn-Sham density on average but misses its four shell bumps. (Notebook 13a, figure 8.)

```
def solve_at(strength, start):
    """The self-consistent w and the three energies (T_s, int v n dx, E_H + E_x)
    at the coupling strength, starting the loop from the potential start."""

    def strength_map(w):  # the Kohn-Sham map with this strength instead of G_C
        return 0.5 * strength * density(orbitals(w)[1])

    w, _ = anderson(strength_map, start)
    phi = orbitals(w)[1]
    n = density(phi)
    parts = (kinetic(phi, [2.0] * N_PER_LABEL), integral(v * n),
             0.25 * strength * integral(n ** 2))
    return w, parts


strengths = np.linspace(0.0, 2.0, 9)  # g_c = 0, 0.25, ..., 2
scan, w_start = [], np.zeros(M)
for strength in strengths:
    w_start, parts = solve_at(strength, w_start)  # start from the last solution
    scan.append(parts)
scan = np.array(scan)  # one row per strength; columns: T_s, int v n, E_H + E_x
say(f"E at g_c = 0: {scan[0].sum():.6f}; at g_c = 2: {scan[-1].sum():.6f}")
check(abs(scan[-1].sum() - E_total) < 1e-9, "the scan ends at the same E as above")
delta = 1e-3
E_plus = sum(solve_at(2.0 + delta, w_scf)[1])  # E at g_c = 2.001
E_minus = sum(solve_at(2.0 - delta, w_scf)[1])  # E at g_c = 1.999
slope = (E_plus - E_minus) / (2.0 * delta)
report("dE/dg_c at g_c = 2 (difference quotient)", f"{slope:.6f}")
report("(1/4) int n^2 dx at g_c = 2 (Hellmann-Feynman)",
       f"{0.25 * integral(n_ks ** 2):.6f}")
check(abs(slope - 0.25 * integral(n_ks ** 2)) < 1e-6,
      "Hellmann-Feynman: dE/dg_c = (1/4) int n^2 dx")
```

Out [23]:

```
E at g_c = 0: 15.990192; at g_c = 2: 21.851498
PASS the scan ends at the same E as above
RESULT dE/dg_c at g_c = 2 (difference quotient) = 2.801961
RESULT (1/4) int n^2 dx at g_c = 2 (Hellmann-Feynman) = 2.801961
PASS Hellmann-Feynman: dE/dg_c = (1/4) int n^2 dx
```

Text cell:

```
The next cell draws the energies of the scan. As $g_c$ grows, the interaction energy
$E_H + E_x$ rises; the cloud spreads, which lowers the kinetic energy (wider orbitals
curve less) and raises the trap energy (the particles sit higher on the parabola).
```

In [24]:

```
fig, ax = plt.subplots()
ax.plot(strengths, scan[:, 0], "o-", ms=4, label="kinetic $T_s$")
```

```
ax.plot(strengths, scan[:, 1], "s-", ms=4, label="trap $\\int v\\,n\\,dx$")
ax.plot(strengths, scan[:, 2], "^-", ms=4, label="interaction $E_H + E_x$")
ax.plot(strengths, scan.sum(axis=1), "D-", ms=4, color="black",
        label="total $E$")
ax.set_xlabel("strength $g_c$ of the contact repulsion")
ax.set_ylabel("energy (units of $\\hbar\\omega$)")
ax.set_title("Switching the repulsion on")
ax.legend(fontsize=8)
save_figure(fig, "coupling_scan",
            "The parts of the Kohn-Sham energy of the eight fermions as the "
            "strength $g_c$ of the contact repulsion grows from 0 to 2: kinetic "
            "energy $T_s$, trap energy, interaction energy $E_H + E_x$ and the "
            "total (black); horizontal axis $g_c$, vertical axis the energy in "
            "units of $\\hbar\\omega$. At $g_c = 0$ the total is "
            "$2(1/2 + 3/2 + 5/2 + 7/2) = 16$ (15.99 on the grid, whose levels lie "
            "slightly below $n + 1/2$) and kinetic and trap energy are equal "
            "(virial theorem); the repulsion spreads the cloud, so the trap energy "
            "rises and the kinetic energy falls.")
check(abs(scan[0].sum() - 16.0) < 0.05 and abs(scan[0, 0] - scan[0, 1]) < 0.05,
      "at g_c = 0: E = 16 and T_s equals the trap energy (virial theorem)")
```

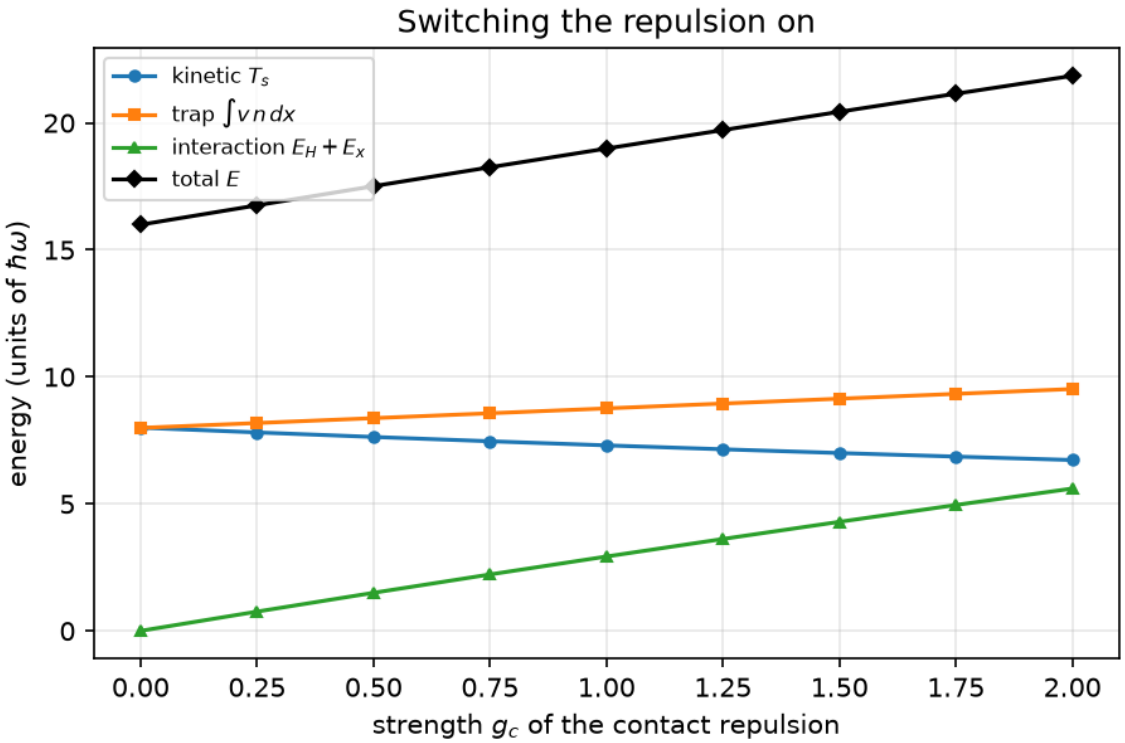

Figure 382: The parts of the Kohn-Sham energy of the eight fermions as the strength g_c of the contact repulsion grows from 0 to 2: kinetic energy T_s, trap energy, interaction energy $E_H + E_x$ and the total (black); horizontal axis g_c, vertical axis the energy in units of $\hbar\omega$. At $g_c = 0$ the total is $2(1/2 + 3/2 + 5/2 + 7/2) = 16$ (15.99 on the grid, whose levels lie slightly below $n + 1/2$) and kinetic and trap energy are equal (virial theorem); the repulsion spreads the cloud, so the trap energy rises and the kinetic energy falls. (Notebook 13a, figure 9.)

Out [24]:

```
Figure 13a.9 saved as Revision/textbook/figures/13a_9_coupling_scan.png
PASS at g_c = 0: E = 16 and T_s equals the trap energy (virial theorem)
```

Text cell:

```
## 14. The first excited state: Kohn-Sham gap and Delta-SCF

The **Kohn-Sham gap** is the difference of the LUMO and HOMO levels of the ground
state. The **Delta-SCF** excitation energy moves one up fermion from orbital 3 (HOMO)
to orbital 4 (LUMO) and solves the loop again with these occupations; it is the
difference of the two total energies. **Janak's theorem** says that the derivative
of the self-consistent energy with respect to an occupation is the level of that
orbital. Moving a fraction $\tau$ of the fermion (occupations $1 - \tau$ in orbital
3 and $\tau$ in orbital 4 of label up) therefore gives
$dE/d\tau = \epsilon_4(\tau) - \epsilon_3(\tau)$, and

$$\Delta_{SCF} = E(1) - E(0) = \int_0^1 [\epsilon_4(\tau) - \epsilon_3(\tau)]\,d\tau .$$

The next cell solves the two-label loop for nine values $\tau = 0, 1/8, \dots, 1$
and integrates the level difference with Simpson's rule (exact for cubic
polynomials, very accurate for smooth curves).
```

In [25]:

```
taus = np.linspace(0.0, 1.0, 9)
gaps, energies, w_pair = [], [], np.concatenate([w_scf, w_scf])
for tau in taus:
    up = [1.0, 1.0, 1.0, 1.0 - tau, tau]  # orbitals 0..4 of label up
    w_pair, _ = anderson(lambda q: labels_map(q, up, FILLED), w_pair)
    up_levels = label_densities(w_pair, up, FILLED)[2]
    gaps.append(up_levels[4] - up_levels[3])  # eps_4(tau) - eps_3(tau)
    energies.append(labels_energy(w_pair, up, FILLED))
gaps, energies = np.array(gaps), np.array(energies)
n_excited = sum(label_densities(w_pair, up, FILLED)[:2])  # tau = 1: n_up + n_down
step = taus[1] - taus[0]
simpson = step / 3.0 * (gaps[0] + gaps[-1] + 4.0 * gaps[1:-1:2].sum()
                        + 2.0 * gaps[2:-1:2].sum())
delta_scf = energies[-1] - energies[0]
report("Kohn-Sham gap (LUMO - HOMO)", f"{gaps[0]:.6f}")
report("Delta-SCF excitation energy E(1) - E(0)", f"{delta_scf:.6f}")
report("Janak integral of eps_4 - eps_3 (Simpson)", f"{simpson:.6f}")
report("transition state eps_4 - eps_3 at tau = 1/2", f"{gaps[4]:.6f}")
check(abs(energies[0] - E_total) < 1e-9, "tau = 0 is the ground state")
check(abs(gaps[0] - (ks_levels[4] - ks_levels[3])) < 1e-9,
      "at tau = 0 the level difference is the Kohn-Sham gap")
check(abs(simpson - delta_scf) < 1e-6,
      "Janak's theorem: the integral of eps_4 - eps_3 equals Delta-SCF")
check(delta_scf < gaps[0], "orbital relaxation lowers the excitation energy here")
```

Out [25]:

```
RESULT Kohn-Sham gap (LUMO - HOMO) = 0.772662
RESULT Delta-SCF excitation energy E(1) - E(0) = 0.714594
RESULT Janak integral of eps_4 - eps_3 (Simpson) = 0.714594
RESULT transition state eps_4 - eps_3 at tau = 1/2 = 0.712784
PASS tau = 0 is the ground state
PASS at tau = 0 the level difference is the Kohn-Sham gap
PASS Janak's theorem: the integral of eps_4 - eps_3 equals Delta-SCF
PASS orbital relaxation lowers the excitation energy here
```

Text cell:

```
The next cell draws the level difference against $\tau$ (its area is the Delta-SCF
energy) and compares the ground-state density with the density of the excited state.
```

In [26]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.fill_between(taus, 0.0, gaps, alpha=0.25, label="area = Delta-SCF")
left.plot(taus, gaps, "o-", color="black", label="$\\epsilon_4 - \\epsilon_3$")
left.plot([0.0], [gaps[0]], "s", ms=8, label=f"Kohn-Sham gap {gaps[0]:.4f}")
left.plot([0.5], [gaps[4]], "^", ms=8, label=f"transition state {gaps[4]:.4f}")
left.axhline(delta_scf, color="gray", ls="--", label=f"Delta-SCF {delta_scf:.4f}")
left.set_ylim(0.0, 1.0)
left.set_xlabel("fraction $\\tau$ of the moved fermion")
left.set_ylabel("level difference ($\\hbar\\omega$)")
left.legend(fontsize=7, loc="lower left")
right.plot(x, n_ks, color="black", label="ground state")
right.plot(x, n_excited, "--", label="excited (Delta-SCF)")
right.set_xlim(-5.0, 5.0)
right.set_xlabel("position $x$")
right.set_ylabel("density (particles per unit length)")
right.legend(fontsize=8)
fig.suptitle("The first excited state by Delta-SCF ($g_c = 2$)")
save_figure(fig, "delta_scf",
            "Left: the level difference $\\epsilon_4(\\tau) - \\epsilon_3(\\tau)$ of "
            "label up when a fraction $\\tau$ of one fermion is moved from orbital 3 "
            "to orbital 4 (horizontal axis $\\tau$, vertical axis energy in units "
            "of $\\hbar\\omega$); its value at $\\tau = 0$ is the Kohn-Sham gap, at "
            "$\\tau = 1/2$ the transition-state estimate, and the shaded area is "
            "the Delta-SCF excitation energy (dashed line), by Janak's theorem. "
            "Right: the ground-state density and the density of the excited state "
            "(horizontal axis $x$); the moved fermion sits in orbital 4, which has "
            "four zeros, so the shell bumps change and a little more density "
            "reaches the outer flanks.")
```

Out [26]:

```
Figure 13a.10 saved as Revision/textbook/figures/13a_10_delta_scf.png
```

Text cell:

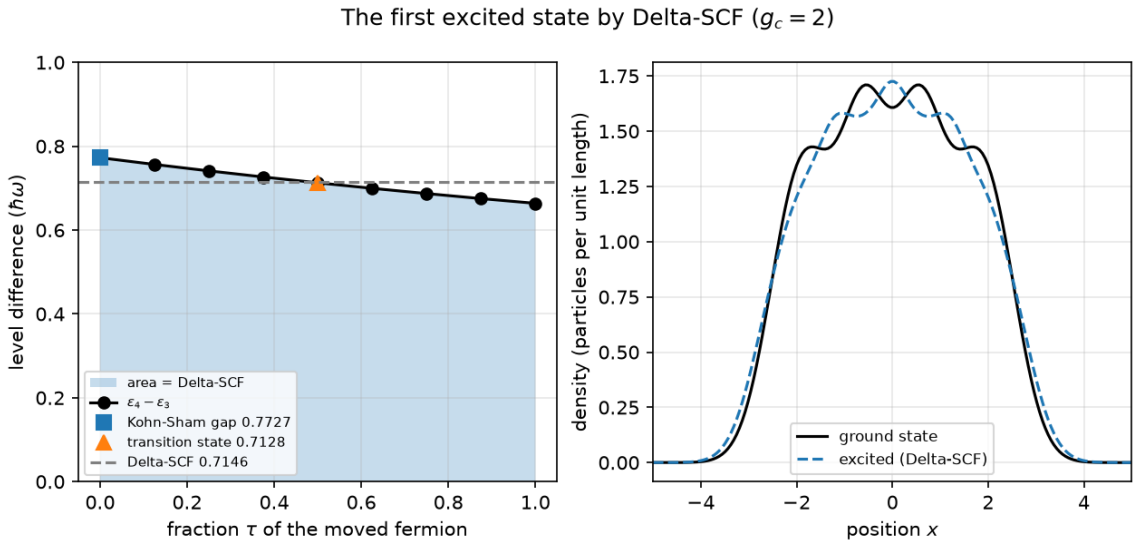

Figure 383: Left: the level difference $\epsilon_4(\tau) - \epsilon_3(\tau)$ of label up when a fraction τ of one fermion is moved from orbital 3 to orbital 4 (horizontal axis τ, vertical axis energy in units of $\hbar\omega$); its value at $\tau = 0$ is the Kohn-Sham gap, at $\tau = 1/2$ the transition-state estimate, and the shaded area is the Delta-SCF excitation energy (dashed line), by Janak's theorem. Right: the ground-state density and the density of the excited state (horizontal axis x); the moved fermion sits in orbital 4, which has four zeros, so the shell bumps change and a little more density reaches the outer flanks. (Notebook 13a, figure 10.)

```
## 15. The last check

The last cell checks that all ten figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [27]:

```
figure_names = ["trap_orbitals", "scf_convergence", "first_iterations",
                "ks_potentials", "density_orbitals", "label_separation",
                "variational_scan", "approximations", "coupling_scan",
                "delta_scf"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13a_{k}_{name}.png").is_file()]
check(missing == [], "all ten figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13a_10_delta_scf.png").is_file(),
      "the figure file 13a_10_delta_scf.png exists")
all_checks_passed()
```

Out [27]:

```
PASS all ten figure files exist
PASS the figure file 13a_10_delta_scf.png exists
ALL 36 CHECKS PASSED (notebook 13a)
```

Text cell:

```
## 16. What this notebook showed

- The Kohn-Sham equations of eight fermions with a contact repulsion reduce to
  one-particle equations in the potential $v_s = v + g_c n/2$: the Hartree push
  $g_c n$ minus the exchange $g_c n/2$, which removes the repulsion between fermions
  with the same label.
- The self-consistency loop converges by plain iteration, by linear mixing and by
  Anderson mixing to the same solution; the speed depends on the mixing parameter,
  and plain iteration overshoots (the density swings in and out). Anderson mixing
  with the settings of the Revision solver (depth 6, $\beta = 0.4$, tolerance
  $10^{-11}$) is the fastest without any tuning.
- The direct energy and the double-counting formula agree; $E_x = -E_H/2$ for two
  equally occupied labels; the mean-field potential $g_c n/2$ is the functional
  derivative of $E_H + E_x$. A converged two-label loop cannot tell a minimum
  from a saddle (on two sites with $U = 4t$ it converges to a saddle); the energy
  test shows that at $g_c = 2$ the equal-label solution in the trap is a minimum
  along three families of label-separating changes.
- Among a family of trial determinants the self-consistent one has the lowest
  energy, and the energy is flat there (variational principle); the derivative of
  the energy with respect to $g_c$ is $\tfrac14\int n^2\,dx$ (Hellmann-Feynman).
- Hartree alone overestimates the spreading (self-interaction); Thomas-Fermi gets
  the average shape but not the shell bumps, because it does not use orbitals.
- The Delta-SCF excitation energy is below the Kohn-Sham gap here, and it equals the
  integral of the level difference over the moved fraction (Janak's theorem).
- What was left out: the correlation energy. All numbers are those of the
  exchange-only (Hartree-Fock) approximation of this teaching model.
```

13.36 Line-by-line walk-through of Notebook 13a

The notebook has 25 code cells. **In [1]** is the set-up cell, identical to In [1] of Notebook 13c (Section 13.14) except for the name `"13a"` and its comment lines, which hold the instructions of Section 13.34.

In [2], the grid and the kinetic-energy matrix.

```
import numpy as np  # arrays of numbers, matrices and linear algebra

L_HALF = 6.0  # the grid covers -6 < x < 6
M = 200  # the number of interior grid points
h = 2.0 * L_HALF / (M + 1)  # the grid spacing 12/201
x = -L_HALF + h * np.arange(1, M + 1)  # the grid points x_k, k = 1, ..., 200
v = 0.5 * x ** 2  # the harmonic trap v(x) = x^2/2 at every point
```

The interval from -6 to 6 is cut into 201 equal steps of length $h = 12/201 = 0.059701$; the 200 inner points are the grid (`np.arange(1, M + 1)` is $1, \ldots, 200$), and the orbitals vanish at the two ends $x = \pm 6$ (hard walls; the occupied orbitals are negligibly small there anyway). `v` is the trap at every grid point.

```
# The kinetic-energy matrix: 1/h^2 on the diagonal, -1/(2 h^2) beside it.
T = (np.diag(np.full(M, 1.0 / h ** 2))
```

```
     + np.diag(np.full(M - 1, -0.5 / h ** 2), 1)
     + np.diag(np.full(M - 1, -0.5 / h ** 2), -1))
say(f"grid: {M} points, spacing h = {h:.6f}")
check(np.allclose(T, T.T), "the kinetic-energy matrix is symmetric")
```

The first line is a comment (it starts with `#` and is not executed) that names the matrix built below. `np.full(M, value)` is a list of M equal values and `np.diag(list, k)` the matrix with this list on the diagonal shifted by k places ($k = 1$ above, $k = -1$ below the main diagonal). So `T` is the matrix of $-\frac{1}{2}\,d^2/dx^2$ of Section 13.2: $1/h^2$ on the diagonal and $-1/(2h^2)$ beside it. The check confirms that it is symmetric, hence Hermitian.

In [3], the levels and orbitals of one particle.

```
def orbitals(w):
    """Levels (increasing) and orbitals (columns, int phi^2 dx = 1) of T + v + w."""
    levels, vectors = np.linalg.eigh(T + np.diag(v + w))
    phi = vectors / np.sqrt(h)  # normalise to sum phi^2 h = 1
    for a in range(M):  # fix the sign of every orbital
        first = np.argmax(np.abs(phi[:, a]) > 1e-3 * np.abs(phi[:, a]).max())
        phi[:, a] *= np.sign(phi[first, a])
    return levels, phi
```

`orbitals(w)` diagonalises the Kohn-Sham Hamiltonian on the grid, the matrix $T + \mathrm{diag}(v + w)$ for a mean-field potential w, and returns its 200 levels in increasing order and the orbitals as the columns of a matrix. numpy returns eigenvectors with $\sum_k u_k^2 = 1$; dividing by $\sqrt{h}$ makes $\sum_k \phi_k^2\, h = 1$, the grid form of $\int \phi^2\, dx = 1$. An eigenvector is fixed only up to its sign, so the loop makes the first clearly nonzero value of each orbital (counted from the left) positive: the comparison gives a list of true and false values, `np.argmax` returns the place of its first true value, and multiplying the column by the sign of the value there (`*=`, `np.sign`) makes that value positive. So every run draws the same pictures.

```
free_levels, free_phi = orbitals(np.zeros(M))  # no interaction: w = 0
exact_levels = np.arange(8) + 0.5  # 1/2, 3/2, ..., 15/2
for a in range(8):
    say(f"level {a}: grid {free_levels[a]:.6f}   exact {exact_levels[a]:.1f}")
check(np.max(np.abs(free_levels[:8] - exact_levels)) < 0.02,
      "the grid reproduces the levels n + 1/2 of the trap within 0.02")
overlaps = free_phi[:, :8].T @ free_phi[:, :8] * h  # sum_k phi_a phi_b h
check(np.allclose(overlaps, np.eye(8), atol=1e-12),
      "the orbitals are orthonormal on the grid")
```

Without interaction the eight lowest grid levels are printed beside the exact levels $n + \frac{1}{2}$: 0.499889 against 0.5, ..., 7.487394 against 7.5. The first check allows 0.02 (the difference formula makes errors of order h^2, growing with the level). `free_phi[:, :8]` keeps the first eight columns; the matrix of their sums $\sum_k \phi_a(x_k)\phi_b(x_k)\, h$ must be the unit matrix: the orbitals are orthonormal on the grid (second check).

In [4], the trap and its orbitals as a picture.

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
ax.plot(x, v, color="black", label="trap $v(x) = x^2/2$")
for a in range(6):
    style = "-" if a < 4 else ":"  # occupied: solid; empty: dotted
    ax.axhline(free_levels[a], color="gray", lw=0.6, ls="--")
    ax.plot(x, free_levels[a] + 0.6 * free_phi[:, a], style,
            label=f"orbital {a}" + (" (occupied)" if a < 4 else " (empty)"))
```

`plt.subplots(figsize=(7.0, 4.6))` makes a figure slightly taller than the standard one. The trap is drawn in black. For each of the six lowest levels the loop chooses a solid line for the four occupied levels and a dotted line for the two empty ones (`style = "-" if a < 4 else ":"`), draws a thin dashed grey horizontal line at the level, and draws the orbital around it: `free_levels[a] + 0.6 * free_phi[:, a]` is 0.6 times the orbital, shifted up by its level, so that each orbital sits on its own energy. The label joins two strings with `+`: the orbital number and either " (occupied)" or " (empty)".

```
ax.set_xlim(-5.0, 5.0)
ax.set_ylim(0.0, 8.4)  # room above the orbitals for the legend
ax.set_xlabel("position $x$")
ax.set_ylabel("energy (units of $\\hbar\\omega$)")
ax.set_title("Levels and orbitals of one particle in the trap (no interaction)")
ax.legend(fontsize=7, loc="upper center", ncol=3)
save_figure(fig, "trap_orbitals",
```

`set_xlim` and `set_ylim` fix the ranges of the axes: positions from -5 to 5 and energies from 0 to 8.4, which leaves room at the top for the legend, written in three columns (`ncol=3`) at the upper centre. Labels, title, and `save_figure` for Figure 13a.1.

What Figure 13a.1 shows. The parabola $x^2/2$ and the six levels $0.5, 1.5, \ldots, 5.5$, equally spaced by 1. Orbital 0 is a single bump, orbital 1 changes sign once, and in general orbital a has a zeros; each orbital is large where the level lies above the parabola (the region a classical particle of that energy could reach) and dies away quickly outside it. The eight fermions fill the four lowest levels (solid), two per level.

In [5], the Kohn-Sham map.

```
G_C = 2.0  # the strength g_c of the contact repulsion
N_PER_LABEL = 4  # four fermions with label up and four with label down
N_TOTAL = 2 * N_PER_LABEL


def integral(f):
    """The integral of f over the line, as the sum of f_k h."""
    return float(np.sum(f) * h)


def density(phi):
    """n(x) = 2 (phi_0^2 + phi_1^2 + phi_2^2 + phi_3^2): two fermions per orbital."""
    return 2.0 * np.sum(phi[:, :N_PER_LABEL] ** 2, axis=1)
```

```
def ks_map(w):
    """One pass of the loop: the mean-field potential g_c n / 2 made from w."""
    levels, phi = orbitals(w)
    return 0.5 * G_C * density(phi)
```

The model's numbers; `integral(f)` approximates $\int f\,dx$ by $\sum_k f_k h$ (accurate here because the functions vanish at the walls); `density(phi)` adds the squares of the four lowest orbitals along each row (`axis=1`) and doubles them; `ks_map(w)` is one pass of the loop of Section 13.32, $w \to$ orbitals $\to n \to w_{out} = g_c n/2$.

```
w_first = ks_map(np.zeros(M))  # one pass, starting from no interaction
report("largest value of the first mean-field potential", f"{w_first.max():.6f}")
check(abs(integral(2.0 * w_first / G_C) - N_TOTAL) < 1e-10,
      "the density of one pass holds exactly 8 particles")
```

One pass from $w = 0$ gives a mean-field potential with the largest value 1.887848; its density $n = 2w/g_c$ must hold 8 particles.

In [6], plain iteration and linear mixing.

```
def linear_mixing(beta, tolerance=1e-11, max_passes=150):
    """Linear mixing w <- w + beta (w_out - w), starting from w = 0."""
    w = np.zeros(M)
    residuals, first_densities = [], []
    for _ in range(max_passes):
        w_out = ks_map(w)
        residuals.append(np.max(np.abs(w_out - w)))  # the residual r
        if len(first_densities) < 5:
            first_densities.append(2.0 * w_out / G_C)  # n = 2 w_out / g_c
        if residuals[-1] <= tolerance:
            break
        w = w + beta * (w_out - w)
    return w, residuals, first_densities
```

Linear mixing of Section 13.21 for the potential: start at $w = 0$; in each pass compute w_{out}, record the residual $\max_x |w_{out} - w|$, keep the densities of the first five passes for the figure of In [10], stop when the residual is at most 10^{-11}, otherwise mix. The function returns the last w, the residuals and the first densities.

```
runs = {}
for beta in (1.0, 0.7, 0.3):
    runs[beta] = linear_mixing(beta)
    say(f"beta = {beta}: {len(runs[beta][1])} passes, last residual "
        f"{runs[beta][1][-1]:.1e}")
check(all(run[1][-1] <= 1e-11 for run in runs.values()),
      "plain iteration and linear mixing with beta = 0.7 and 0.3 all converge")
```

Three runs, stored in the dictionary `runs` under their β: 40 passes for plain iteration, 23 for $\beta = 0.7$, 73 for $\beta = 0.3$. The check requires every last residual to be at most 10^{-11}. (Unlike the two-site model of Section 13.21, plain iteration converges here, slowly.)

In [7], the settings of the Revision solver.

```
parameters = json.loads(repository_file(
    "Revision/kohn_sham/results/parameters.json").read_text(encoding="utf-8"))
numerics = parameters["numerics"]  # the numerical settings of the Revision solver
DEPTH = int(numerics["andersonDepth"])  # how many earlier passes are remembered
BETA_ANDERSON = float(numerics["andersonBeta"])  # the mixing parameter
TOLERANCE = float(numerics["scfTolerance"])  # the stopping rule
```

The cell reads the parameter file of the Revision Kohn-Sham solver and from its part "numerics" the number of remembered passes, β and the stopping rule.

```
say(f"Revision solver settings: depth {DEPTH}, beta {BETA_ANDERSON}, "
    f"tolerance {TOLERANCE:.0e}")
check(DEPTH == 6 and BETA_ANDERSON == 0.4 and abs(TOLERANCE - 1e-11) < 1e-24,
      "the Revision solver mixes with depth 6, beta 0.4 and stops at 1e-11 "
      "(Revision/kohn_sham/results/parameters.json, numerics)")
```

The printed line (`:.0e` writes a number as a power of ten without decimals) and the check confirm 6, 0.4 and 10^{-11} (the file stores the tolerance as the floating-point number nearest to 10^{-11}, so the check allows a difference below 10^{-24}).

In [8], Anderson mixing.

```
def anderson(step, w0, beta=BETA_ANDERSON, depth=DEPTH, tolerance=TOLERANCE,
             max_passes=400):
    """Anderson mixing for the fixed point step(w) = w, starting from w0."""
    w = np.array(w0, dtype=float)
    inputs, residual_vectors, residuals = [], [], []
    for _ in range(max_passes):
        residual = step(w) - w  # R = w_out - w
        residuals.append(np.max(np.abs(residual)))
        if residuals[-1] <= tolerance:
            return w, residuals
        inputs = (inputs + [w.copy()])[-depth:]  # keep the last `depth` passes
        residual_vectors = (residual_vectors + [residual])[-depth:]
        if len(inputs) == 1:
            w = w + beta * residual  # the first pass: linear mixing
            continue
        last_r = residual_vectors[-1]
        differences = np.array([r - last_r for r in residual_vectors[:-1]]).T
        theta = np.linalg.lstsq(differences, -last_r, rcond=None)[0]
        c = np.append(theta, 1.0 - theta.sum())  # the c_i; they add up to 1
        w = sum(ci * (wi + beta * ri)
                for ci, wi, ri in zip(c, inputs, residual_vectors))
    raise RuntimeError("Anderson mixing did not converge")
```

Anderson mixing of Section 13.21 for any map `step` (here `ks_map`; below also maps of two potentials at once and maps with another strength). Each pass computes the residual vector $R = w_{out} - w$ and its largest entry, and returns when that is at most the tolerance. The last `depth` inputs and

residual vectors are kept (`w.copy()` stores a copy, so later changes of `w` do not change the stored input). The first pass is linear mixing. Afterwards `differences` is the matrix whose columns are $R^{(i)} - R^{(last)}$ (200 rows, one per grid point), `np.linalg.lstsq` gives the least-squares θ of $\sum_i \theta_i (R^{(i)} - R^{(last)}) \approx -R^{(last)}$, `c` appends $1 - \sum_i \theta_i$, and the next input is $\sum_i c_i (w^{(i)} + \beta R^{(i)})$. If 400 passes are not enough, the function stops the notebook with an error.

```
w_scf, anderson_residuals = anderson(ks_map, np.zeros(M))
say(f"Anderson mixing: {len(anderson_residuals)} passes, last residual "
    f"{anderson_residuals[-1]:.1e}")
differences = [np.max(np.abs(runs[beta][0] - w_scf)) for beta in runs]
check(max(differences) < 1e-9,
      "all four routes reach the same self-consistent potential (within 1e-9)")
check(len(anderson_residuals) < len(runs[1.0][1]),
      "Anderson mixing needs fewer passes than plain iteration")
```

Anderson mixing from $w = 0$ needs 19 passes (last residual $1.8 \cdot 10^{-12}$). The checks: the three linear-mixing results agree with the Anderson result to 10^{-9} (`for beta in runs` goes through the keys of the dictionary), and Anderson needed fewer passes than plain iteration.

In [9], the residuals as a picture.

```
fig, ax = plt.subplots()
for beta, style in ((1.0, "o-"), (0.7, "s-"), (0.3, "^-")):
    name = "plain iteration" if beta == 1.0 else f"linear mixing, beta = {beta}"
    ax.semilogy(range(1, len(runs[beta][1]) + 1), runs[beta][1], style, ms=3,
                label=f"{name} ({len(runs[beta][1])} passes)")
```

For the three linear runs the loop draws the residual of every pass (`runs[beta][1]`, the list of residuals) against the pass numbers $1, 2, \ldots$ (`range(1, len(...) + 1)`), on a logarithmic vertical axis, with circles, squares and triangles; the label gives the name of the run and its number of passes.

```
ax.semilogy(range(1, len(anderson_residuals) + 1), anderson_residuals, "D-",
            ms=3, color="black",
            label=f"Anderson, depth 6, beta 0.4 ({len(anderson_residuals)} passes)")
ax.axhline(TOLERANCE, color="gray", ls="--", lw=0.8)
ax.set_xlabel("pass number")
ax.set_ylabel("residual $\\max_x |w_{out} - w|$")
ax.set_title("Self-consistency: residual versus pass ($g_c = 2$)")
ax.legend(fontsize=8)
save_figure(fig, "scf_convergence",
```

The residuals of Anderson mixing as black diamonds; a grey dashed horizontal line at the stopping rule 10^{-11}; labels, title, legend, and `save_figure` for Figure 13a.2.

What Figure 13a.2 shows. All four runs start with the residual 1.89 (the first pass from $w = 0$) and end just below the dashed line. The three linear runs are nearly straight lines on the logarithmic axis: each pass multiplies the residual by about the same factor, smallest for $\beta = 0.7$ (23 passes), larger for plain iteration (40) and largest for $\beta = 0.3$ (73). Anderson mixing falls fastest and reaches the line after 19 passes; its line is not straight, because it changes its estimate of the slope as it learns from the remembered passes.

In [10], why plain iteration is slow.

```
n_scf = 2.0 * w_scf / G_C  # the self-consistent density n = 2 w / g_c
fig, ax = plt.subplots()
for number, n_pass in enumerate(runs[1.0][2][:4], 1):
    ax.plot(x, n_pass, lw=1.0, label=f"density made by pass {number}")
ax.plot(x, n_scf, color="black", lw=2.0, label="self-consistent density")
```

The self-consistent density, and the densities made by the first four passes of plain iteration (`runs[1.0][2]` is the list of first densities of the run with $\beta = 1$, and `[:4]` keeps four of them), drawn as thin lines numbered from 1 by `enumerate(..., 1)`, and the self-consistent density as a thick black line.

```
ax.set_xlim(-5.0, 5.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("density $n(x)$ (particles per unit length)")
ax.set_title("Plain iteration overshoots: the first passes")
ax.legend(fontsize=8)
save_figure(fig, "first_iterations",
```

The range of positions, labels, title, legend and `save_figure` for Figure 13a.3. Then the check

```
check(runs[1.0][2][0].max() > n_scf.max() > runs[1.0][2][1].max(),
      "the first pass is too narrow and the second too wide (overshooting)")
```

compares the largest values of the densities: the first density, made without repulsion, is higher (narrower) than the solution, and the second is lower (wider): the density swings around the solution.

What Figure 13a.3 shows. The density of pass 1 (no repulsion yet) is the highest in the middle, up to about 1.89, and the narrowest. Pass 2, made in the strong repulsion of that narrow density, is the lowest in the middle (about 1.64) and spreads furthest out. Passes 3 and 4 lie in between, closer and closer to the thick black solution, from alternating sides. All curves cross near $x = \pm 2$: the charge sloshes between the centre and the flanks, the same mechanism as in the two-site model of Section 13.21, but here the swings shrink, because the slope of the map is below 1 in size.

In [11], the converged state.

```
ks_levels, ks_phi = orbitals(w_scf)
n_ks = density(ks_phi)
for a in range(6):
    status = "occupied by 2" if a < N_PER_LABEL else "empty"
    say(f"Kohn-Sham level {a}: {ks_levels[a]:.6f} ({status})")
report("Kohn-Sham HOMO level", f"{ks_levels[3]:.6f}")
report("Kohn-Sham LUMO level", f"{ks_levels[4]:.6f}")
check(abs(integral(n_ks) - N_TOTAL) < 1e-10, "the density integrates to N = 8")
check(np.max(np.abs(n_ks - n_scf)) < 1e-10,
      "the density of the final orbitals reproduces the input density")
```

The orbitals of the self-consistent potential, their density, the six lowest levels with their occupation, and the HOMO and LUMO levels (4.723945 and 5.496606). The checks: the density holds 8 particles,

and the orbitals of $v + w_{scf}$ give back the density $2w_{scf}/g_c$ that made w_{scf}: self-consistency, to 10^{-10}.

In [12], the potentials.

```
v_hartree = G_C * n_ks  # v_H = g_c n
v_exchange = -0.5 * G_C * n_ks  # v_x = -g_c n_up = -g_c n / 2
```

The Hartree potential $g_c n$ and the exchange potential $-g_c n/2$ of Section 13.32, at the self-consistent density.

```
fig, ax = plt.subplots()
ax.plot(x, v, color="black", label="trap $v$")
ax.plot(x, v_hartree, label="Hartree $v_H = g_c n$")
ax.plot(x, v_exchange, label="exchange $v_x = -g_c n/2$")
ax.plot(x, v + v_hartree + v_exchange, lw=2.0, label="Kohn-Sham $v_s$")
for a in range(N_PER_LABEL):
    ax.axhline(ks_levels[a], color="gray", ls="--", lw=0.6)
```

The trap (black), the two potentials, and their sum $v_s = v + v_H + v_x$ as a thick line; the loop draws the four occupied Kohn-Sham levels as thin dashed grey lines.

```
ax.set_xlim(-5.0, 5.0)
ax.set_ylim(-3.0, 9.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("potential (units of $\\hbar\\omega$)")
ax.set_title("The potentials of the self-consistent state ($g_c = 2$)")
ax.legend(fontsize=8)
save_figure(fig, "ks_potentials",
```

The ranges, labels, title, legend and `save_figure` for Figure 13a.4. After the caption,

```
check(np.allclose(v + v_hartree + v_exchange, v + w_scf, atol=1e-10),
      "v + v_H + v_x equals the self-consistent v + w")
```

checks that $v + v_H + v_x$ equals $v + w_{scf}$ at every point to 10^{-10}.

What Figure 13a.4 shows. The Hartree potential is a broad plateau of about 3 with small bumps (it is twice the density), the exchange potential is the same shape upside down at half the size, and the Kohn-Sham potential v_s (green, thick) is the trap lifted in the middle: its bottom is no longer at 0 but at about 1.6, and it is flattened over $-2 < x < 2$. The four occupied levels (2.12, 3.01, 3.88, 4.72) are pushed up and squeezed together compared with $0.5, 1.5, 2.5, 3.5$ without interaction.

In [13], the density as a stack of orbitals.

```
fig, ax = plt.subplots()
layers = [2.0 * ks_phi[:, a] ** 2 for a in range(N_PER_LABEL)]
ax.stackplot(x, layers, labels=[f"$2\\phi_{a}^2$" for a in range(N_PER_LABEL)],
             alpha=0.7)
```

The four contributions $2\phi_a^2$ of the occupied orbitals; `ax.stackplot` draws them on top of each other as coloured layers (orbital 0 at the bottom; `alpha=0.7` makes them 70 per cent opaque), so that the top edge is the density. The labels are made by a list comprehension, one per orbital. (In a Python string a backslash is written twice, so the label text reaches matplotlib with one backslash, which draws ϕ.)

```
ax.plot(x, n_ks, color="black", lw=1.5, label="total density $n$")
ax.set_xlim(-5.0, 5.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("density (particles per unit length)")
ax.set_title("The density as the sum of the occupied orbitals")
ax.legend(fontsize=8)
save_figure(fig, "density_orbitals",
```

The total density as a black line on top of the layers; range, labels, title, legend and `save_figure` for Figure 13a.5.

What Figure 13a.5 shows. Orbital 0 contributes one central bump (blue), orbital 1 two bumps beside the centre (orange), orbital 2 three and orbital 3 four (each orbital a has a zeros, so ϕ_a^2 has $a+1$ bumps). Stacked, they give the density: about 1.6 at the centre, two peaks of about 1.71 near $x = \pm 0.57$, two shoulders of about 1.43 near $x = \pm 1.7$, and a steep fall to nearly zero by $|x| = 3.8$. These ripples on top of a smooth profile are the shell structure. Each layer holds two fermions and the whole stack eight.

In [14], the energy two ways.

```
def kinetic(phi, occupations):
    """T_s = sum_a f_a int phi_a (-1/2 phi_a'') dx with the occupations f_a."""
    return sum(f * float(phi[:, a] @ (T @ phi[:, a])) * h
               for a, f in enumerate(occupations))
```

The kinetic energy $T_s = \sum_a f_a \int \phi_a(-\frac{1}{2}\phi_a'')\,dx$, on the grid $\sum_a f_a\,(\phi_a \cdot T\phi_a)\,h$, for any list of occupations.

```
T_s = kinetic(ks_phi, [2.0] * N_PER_LABEL)
E_ext = integral(v * n_ks)
E_H = 0.5 * G_C * integral(n_ks ** 2)
E_x = -0.25 * G_C * integral(n_ks ** 2)
E_total = T_s + E_ext + E_H + E_x
E_double = 2.0 * ks_levels[:N_PER_LABEL].sum() - (E_H + E_x)
```

`[2.0] * N_PER_LABEL` is the list $[2, 2, 2, 2]$. The four parts of the energy of Section 13.32, their sum, and the double-counting formula (`ks_levels[:N_PER_LABEL].sum()` adds the four occupied levels).

```
for label, value in (("kinetic T_s", T_s), ("external int v n dx", E_ext),
                     ("Hartree E_H", E_H), ("exchange E_x", E_x)):
    say(f"{label:22} = {value:.6f}")
report("total energy E (direct sum)", f"{E_total:.6f}")
```

```
report("total energy E (double counting)", f"{E_double:.6f}")
check(abs(E_total - E_double) < 1e-9, "the two energy formulas agree")
check(abs(E_x + 0.5 * E_H) < 1e-12, "E_x = -E_H/2 for two equally occupied labels")
```

The loop goes through four pairs (name, value) and prints the parts, 6.726297, 9.521279, 11.207843 and -5.603922 (`{label:22}` pads the name with blanks to 22 characters, so that the equals signs line up); the `report` lines print both totals, 21.851498, and the checks require them to agree to 10^{-9} and $E_x = -E_H/2$ to 10^{-12} (the rule $-1/g$ with $g = 2$).

In [15], the functional derivative on the grid.

```
def interaction_energy(n):
    """E_H + E_x = (g_c/4) int n^2 dx for two equally occupied labels."""
    return 0.25 * G_C * integral(n ** 2)


eta = np.exp(-(x - 1.0) ** 2)  # a fixed change of shape, off the centre of the trap
epsilon = 1e-4  # the size of the change
quotient = (interaction_energy(n_ks + epsilon * eta)
            - interaction_energy(n_ks - epsilon * eta)) / (2.0 * epsilon)
w_ks = 0.5 * G_C * n_ks  # the mean-field potential w = g_c n / 2
```

The definition of Section 13.9 tested directly: change the density by $\pm\epsilon\eta$ with the bump $\eta = e^{-(x-1)^2}$ and $\epsilon = 10^{-4}$, and form the central difference quotient of $E_H + E_x$. By the definition it must equal $\int w\,\eta\,dx$ with $w = \delta(E_H + E_x)/\delta n = g_c n/2$, which is `w_ks`.

```
report("difference quotient of E_H + E_x along eta", f"{quotient:.10f}")
report("integral of w eta dx", f"{integral(w_ks * eta):.10f}")
check(abs(quotient - integral(w_ks * eta)) < 1e-9,
      "w = g_c n/2 is the functional derivative of E_H + E_x")
```

Both print as 2.6845567944, and the check requires agreement to 10^{-9} (because $E_H + E_x$ is a square of n, the central quotient is exact up to rounding: the terms of order ϵ^2 cancel between $+\epsilon$ and $-\epsilon$, and there are no higher terms).

In [16], the stability of equal labels.

```
def label_densities(w_pair, up_occupations, down_occupations):
    """The densities n_up, n_down and the levels for the pair of potentials."""
    up_levels, up_phi = orbitals(w_pair[:M])  # w_up: the first M numbers
    down_levels, down_phi = orbitals(w_pair[M:])  # w_down: the last M numbers
    n_up = (up_phi[:, :len(up_occupations)] ** 2) @ np.array(up_occupations)
    n_down = (down_phi[:, :len(down_occupations)] ** 2) @ np.array(down_occupations)
    return n_up, n_down, up_levels, up_phi, down_levels, down_phi
```

The two potentials are stored in one list of $2M = 400$ numbers: the first 200 for label up, the last 200 for label down (`w_pair[:M]`, `w_pair[M:]`). For each label the function solves the orbitals and forms the density with any occupations: the matrix of squared orbitals times the list of occupations is $\sum_a f_a \phi_a^2$ at every point.

```
def labels_map(w_pair, up_occupations, down_occupations):
    """One pass for two labels: w_up = g_c n_down and w_down = g_c n_up."""
    n_up, n_down = label_densities(w_pair, up_occupations, down_occupations)[:2]
    return np.concatenate([G_C * n_down, G_C * n_up])


def labels_energy(w_pair, up_occupations, down_occupations):
    """E = T_s + int v n + g_c int n_up n_down (E_H + E_x for two labels)."""
    n_up, n_down, _, up_phi, _, down_phi = label_densities(
        w_pair, up_occupations, down_occupations)
    return (kinetic(up_phi, up_occupations) + kinetic(down_phi, down_occupations)
            + integral(v * (n_up + n_down)) + G_C * integral(n_up * n_down))
```

One pass of the two-label loop: the potential of label up is $g_c n_\downarrow$ and that of label down $g_c n_\uparrow$ (Section 13.32), joined into one list by `np.concatenate`. `labels_energy` is $E = T_s + \int v\, n + g_c \int n_\uparrow n_\downarrow$.

```
FILLED = [1.0] * N_PER_LABEL  # the four lowest orbitals of each label occupied
push = 1.0 * np.tanh(x)  # pushes up-fermions to the left, down-fermions to the right
start = np.concatenate([w_scf + push, w_scf - push])
w_pair, pair_residuals = anderson(lambda q: labels_map(q, FILLED, FILLED), start)
n_up, n_down = label_densities(w_pair, FILLED, FILLED)[:2]
```

Each label has its four lowest orbitals occupied once. The start is strongly separated: $\tanh x$ rises from -1 to 1 across the trap, so adding it to the up potential pushes the up fermions to the left, and subtracting it pushes the down fermions to the right. Anderson mixing then runs on both potentials together (the `lambda` fixes the occupations), and the label densities of the result are formed.

```
say(f"two-label run: {len(pair_residuals)} passes; largest |n_up - n_down| = "
    f"{np.max(np.abs(n_up - n_down)):.1e}")
check(np.max(np.abs(n_up - n_down)) < 1e-8,
      "from a separated start the two-label loop returns to equal densities")
check(abs(labels_energy(w_pair, FILLED, FILLED) - E_total) < 1e-9,
      "the two-label run has the same energy as the equal-label solution")
```

The run needs 36 passes, and the largest difference between the label densities is $4.9 \cdot 10^{-12}$; the checks require it to be below 10^{-8} and the energy to equal the equal-label energy 21.851498 to 10^{-9}: from a strongly separated start the loop returns to equal labels. This does NOT yet show that the equal-label solution is stable: a converged loop can sit on a saddle. The next two cells show such a case and then make the test that decides, the energy test.

In [17], a loop that converges to a saddle: two sites.

```
T_HOP, U_SITE = 1.0, 4.0  # the hopping t and the on-site repulsion U of two sites
left_right = np.array([-1.0, 1.0])  # lowers the potential on L and raises it on R
```

The two-site model of Section 13.7 with the hopping $t = 1$ and the repulsion $U = 4$ (so $U = 4t$, stronger than $2t$). `left_right` is the pattern $(-1, +1)$: added to a pair of site potentials (w_L, w_R) it lowers w_L and raises w_R.

```
def site_orbital(w2):
    """(c_L, c_R): the orbital of the lower level of [[w_L, -t], [-t, w_R]]."""
    return np.linalg.eigh(np.array([[w2[0], -T_HOP], [-T_HOP, w2[1]]]))[1][:, 0]
```

For one label with the site potentials `w2` $= (w_L, w_R)$, the one-particle matrix is $\begin{pmatrix} w_L & -t \\ -t & w_R \end{pmatrix}$. `np.linalg.eigh` returns its eigenvalues in increasing order and the eigenvectors as columns; `[1][:, 0]` takes the eigenvector of the LOWER level, the occupied orbital (c_L, c_R) with $c_L^2 + c_R^2 = 1$.

```
def sites_map(q):
    """One pass on two sites, q = (w_up on L, R, w_down on L, R)."""
    n_up, n_down = site_orbital(q[:2]) ** 2, site_orbital(q[2:]) ** 2
    return np.concatenate([U_SITE * n_down, U_SITE * n_up])
```

One pass of the two-label loop on two sites, exactly as in the trap: the four numbers `q` are the potentials of label up on L and R and of label down on L and R; the occupations of the sites are the squares of the orbital components, and the new potential of label up is $Un_\downarrow$ (it feels only the other label), that of label down $Un_\uparrow$.

```
def sites_energy(q):
    """-2t (c_L c_R of up + c_L c_R of down) + U sum_sites n_up n_down."""
    c_up, c_down = site_orbital(q[:2]), site_orbital(q[2:])
    return (-2.0 * T_HOP * (c_up[0] * c_up[1] + c_down[0] * c_down[1])
            + U_SITE * np.sum(c_up ** 2 * c_down ** 2))
```

The energy of the determinant built from the two orbitals: each orbital contributes the hopping energy $-2t\, c_L c_R$, and the repulsion is U times the product of the up and down occupations on each site, summed over the two sites.

```
def sites_start(size):
    """The potentials U/2 + size (-1, 1) for label up and U/2 - size (-1, 1)."""
    return np.concatenate([0.5 * U_SITE + size * left_right,
                           0.5 * U_SITE - size * left_right])
```

A family of label-separating potentials: at `size` $= 0$ both labels have $U/2$ on both sites (the equal-label solution); a positive `size` makes site L cheaper for label up and site R cheaper for label down.

```
site_ends = {}
for site_push in (0.5, 1.0):
    q_end, site_residuals = anderson(sites_map, sites_start(site_push))
    site_ends[site_push] = q_end
    n_up_sites = site_orbital(q_end[:2]) ** 2
    energy = np.round(sites_energy(q_end), 9) + 0.0  # + 0.0 turns -0.0 into 0.0
    say(f"two sites, push {site_push}: {len(site_residuals)} passes, n_up = "
        f"({n_up_sites[0]:.6f}, {n_up_sites[1]:.6f}), energy {energy:.6f}")
```

The SAME Anderson loop as in the trap (`anderson`, In [8]) runs twice, from a small separation (push 0.5) and from a larger one (push 1). For each end it stores the potentials, prints the number of passes, the occupations of label up on L and R and the energy; `np.round(..., 9)` removes rounding noise and `+ 0.0` turns a printed -0.000000 into 0.000000. The output: from push 0.5 the loop needs 16 passes and ends at $n_\uparrow = (0.5, 0.5)$ with the energy 0, the equal-label solution; from push 1 it needs 18 passes and ends at $n_\uparrow = (0.933013, 0.066987)$ with the energy -0.5, the separated solution.

```
site_sizes = np.linspace(0.0, 2.5, 51)  # epsilon = 0, 0.05, ..., 2.5
site_family = np.array([sites_energy(sites_start(size)) for size in site_sizes])
saddle_up = site_orbital(site_ends[0.5][:2]) ** 2
```

The energy along the whole family $\epsilon = 0, 0.05, \ldots, 2.5$ (for the figure and the last check), and the occupations of label up at the end of the push-0.5 run.

```
check(np.max(np.abs(saddle_up - 0.5)) < 1e-9
      and abs(sites_energy(site_ends[0.5]) - (-2.0 * T_HOP + 0.5 * U_SITE)) < 1e-9,
      "two sites, U = 4t, push 0.5: the loop converges to equal labels, E = -2t + U/2")
check(abs(sites_energy(site_ends[1.0]) + 2.0 * T_HOP ** 2 / U_SITE) < 1e-9,
      "two sites, push 1: the loop converges to separated labels, E = -2t^2/U")
check(site_family[1] < site_family[0] - 1e-3,
      "two sites: separating the labels lowers the energy, so the equal-label "
      "solution there is a saddle")
```

Three checks. The first: from push 0.5 the loop converges to equal occupations 1/2 with the equal-label energy $-2t + U/2 = 0$. The second: from push 1 it converges to the separated energy $-2t^2/U = -0.5$ (Section 13.7). The third: already the first step of the family, $\epsilon = 0.05$, has a lower energy than $\epsilon = 0$; so the equal-label solution, although self-consistent and reached by the loop, is a saddle. A converged loop alone cannot tell a minimum from a saddle.

In [18], the energy test in the trap, and Figure 13a.6.

```
shapes = {"tanh x": np.tanh(x), "x exp(-x^2/4)": x * np.exp(-x ** 2 / 4.0),
          "exp(-x^2/2)": np.exp(-x ** 2 / 2.0)}
```

Three shapes $s(x)$ of a label-separating change: $\tanh x$ pushes label up to the left and label down to the right across the whole trap; $x\,e^{-x^2/4}$ does the same but only near the centre; $e^{-x^2/2}$ pushes label up outwards and label down inwards.

```
def separation_rise(shape, size):
    """E of the determinant made from w_scf +- size * shape, minus E_KS."""
    trial = np.concatenate([w_scf + size * shape, w_scf - size * shape])
    return labels_energy(trial, FILLED, FILLED) - E_total
```

The trial potentials are $w_\uparrow = w_{scf} + \epsilon\, s(x)$ and $w_\downarrow = w_{scf} - \epsilon\, s(x)$. `labels_energy` (In [16]) takes the four lowest orbitals of each label in its trial potential and evaluates the energy $E = T_s + \int v\, n + g_c \int n_\uparrow n_\downarrow$ of that determinant; for the contact interaction this formula is the exact energy of the determinant, so no self-consistency is needed. The function returns the energy above the Kohn-Sham energy `E_total`.

```
sizes = [0.01, 0.05, 0.2, 0.5, 1.0]
rises = {name: [separation_rise(shape, size) for size in sizes]
         for name, shape in shapes.items()}
for name, values in rises.items():
    say(f"{name:14} E - E_KS: " + " ".join(f"{value:.2e}" for value in values))
```

The rise for the five sizes $\epsilon = 0.01, 0.05, 0.2, 0.5, 1$ and each shape, printed in powers of ten (`.2e`). The output: for $\tanh x$ the rises are $3.50 \cdot 10^{-5}$, $8.81 \cdot 10^{-4}$, $1.53 \cdot 10^{-2}$, 0.124 and 0.593; for $x\, e^{-x^2/4}$ they are $2.14 \cdot 10^{-5}$ to 0.254; for $e^{-x^2/2}$, $8.58 \cdot 10^{-6}$ to $8.71 \cdot 10^{-2}$. All are positive.

```
check(all(value > 0.0 for values in rises.values() for value in values),
      "every label-separating trial determinant in the trap has a higher energy "
      "than the Kohn-Sham state")
check(all(24.0 < values[1] / values[0] < 26.0 for values in rises.values()),
      "for small separations the energy rises as epsilon^2: in the trap the "
      "equal-label solution is a minimum along these families, not a saddle")
```

Two checks. The first: every one of the fifteen trial determinants lies above the Kohn-Sham energy. The second: going from $\epsilon = 0.01$ to 0.05 (five times larger) multiplies the rise by between 24 and 26, close to $5^2 = 25$; for $\tanh x$, $8.81 \cdot 10^{-4} / 3.50 \cdot 10^{-5} \approx 25.2$. So for small separations the rise is of second order, $E - E_{KS} \approx a\,\epsilon^2$ with $a > 0$: the energy is flat (stationary) at the Kohn-Sham solution and curves upwards. Along these three families the equal-label solution is a minimum, not a saddle. (Three families are a test, not a proof for every possible change; this is a COMPUTED result for this trap at $g_c = 2$.)

```
fine_sizes = np.linspace(0.0, 1.0, 41)  # epsilon = 0, 0.025, ..., 1
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
for (name, shape), style in zip(shapes.items(), ("-", "--", "-.")):
    left.plot(fine_sizes, [separation_rise(shape, size) for size in fine_sizes],
              style, label=f"shape {name}")
```

Figure 13a.6 has two panels side by side (`plt.subplots(1, 2, ...)`). On the left, the rise for each shape on a finer grid of 41 sizes, each shape with its own line style (solid, dashed, dash-dotted).

```
left.set_xlabel("size $\\epsilon$ of the separation")
left.set_ylabel("$E - E_{KS}$ ($\\hbar\\omega$)")
left.set_title("Trap, $g_c = 2$: the energy rises (minimum)")
left.legend(fontsize=8)
```

Axis labels (the size is a pure number; the energy is in units of $\hbar\omega$), the title and the legend of the left panel.

```
right.plot(site_sizes, site_family, color="black", label="energy of the family")
right.plot([0.0], [site_family[0]], "o", ms=8,
           label="loop from push 0.5 (saddle)")
size_end = 0.5 * (site_ends[1.0][1] - site_ends[1.0][0])  # (w_R - w_L)/2 of up
right.plot([size_end], [sites_energy(site_ends[1.0])], "s", ms=8,
           label="loop from push 1 (minimum)")
```

On the right, the two-site family of In [17] as a black line, a circle at $\epsilon = 0$ where the push-0.5 loop ended, and a square where the push-1 loop ended. The square's horizontal position is $(w_R - w_L)/2$ of label up at the end, the size of the separation that the loop found.

```
right.set_xlabel("size $\\epsilon$ of the separation")
right.set_ylabel("$E$ (units of $t$)")
right.set_title("Two sites, $U = 4t$: the energy falls (saddle)")
right.legend(fontsize=8)
save_figure(fig, "label_separation",
            "The energy test of stability. Left: the trap with eight fermions; "
            "the energy of the determinant made from the label-separating trial "
            "potentials $w_{scf} \\pm \\epsilon\\,s(x)$, minus the Kohn-Sham "
            "energy, for three shapes $s$, against the size $\\epsilon$ (pure "
            "number); vertical axis in units of $\\hbar\\omega$. Every curve starts "
            "flat at 0 and rises: along these families the equal-label solution "
            "is a minimum. Right: two sites with $U = 4t$; the energy of the "
            "determinant made from the potentials $U/2 \\pm \\epsilon\\,(-1, 1)$, "
            "in units of $t$, against $\\epsilon$. It falls from the equal-label "
            "solution (circle, energy 0, where the loop from push 0.5 converged: "
            "a saddle) to the separated minimum $-2t^2/U = -0.5t$ (square, where "
            "the loop from push 1 converged). The converged loop could not tell "
            "the two cases apart; the energy does.")
```

Labels, title and legend of the right panel, and `save_figure`, which writes `13a_6_label_separation.png` with this caption, which the book prints below the figure.

What Figure 13a.6 shows. On the left, all three curves start flat at 0 and rise: for $\tanh x$ to about 0.59 at $\epsilon = 1$, for the other two shapes less, because they move fewer fermions. A flat start means that the Kohn-Sham solution is stationary; the upward curve means a minimum along these families. On the right, the two-site energy starts at 0 (the circle, where the loop from push 0.5 converged) and FALLS to the separated minimum $-0.5t$ (the square, where the loop from push 1 converged): the circle is a saddle. Both loops converged; only the energy shows which end is stable.

In [19], the variational principle.

```
def energy_of_orbitals(phi):
    """E = T_s + int v n dx + (g_c/4) int n^2 dx for 2 fermions in orbitals 0..3."""
    n = density(phi)
    return (kinetic(phi, [2.0] * N_PER_LABEL) + integral(v * n)
            + 0.25 * G_C * integral(n ** 2))


c_values = np.linspace(0.0, 2.0, 41)  # c = 0, 0.05, ..., 2
family_energies = np.array([energy_of_orbitals(orbitals(c * n_scf)[1])
                            for c in c_values])
c_best = c_values[np.argmin(family_energies)]
```

The energy formula of Section 13.32 for any set of orbitals; for 41 values of c from 0 to 2 the orbitals of the trial potential $v + c\,n_{scf}$ (`orbitals(c * n_scf)[1]`) and their energy; `c_best` the value with the lowest energy.

```
report("c of the lowest energy in the family", f"{c_best:.2f}")
check(abs(c_best - 0.5 * G_C) < 1e-9,
      "the lowest energy of the family is at c = g_c/2 (the self-consistent one)")
check(np.all(family_energies >= E_total - 1e-10),
      "no member of the family has a lower energy than the Kohn-Sham state")
```

`c_best` prints as 1.00, and the checks require it to equal $g_c/2$ (to 10^{-9}; the grid of c contains the value 1 exactly) and every member of the family to have an energy at least E_{KS} (to 10^{-10}).

In [20], the family as a picture.

```
fig, ax = plt.subplots()
ax.plot(c_values, family_energies - E_total, "o-", ms=3,
        label="$E(c) - E_{KS}$")
ax.axvline(0.5 * G_C, color="gray", ls="--", label="$c = g_c/2$ (self-consistent)")
ax.set_xlabel("$c$ in the trial potential $v + c\\,n_{scf}$")
ax.set_ylabel("energy above the Kohn-Sham energy ($\\hbar\\omega$)")
ax.set_title("Variational principle: a family of trial determinants")
ax.legend()
save_figure(fig, "variational_scan",
```

The energy above the Kohn-Sham energy against c, as dots joined by a line; a grey dashed vertical line at $c = g_c/2$; labels, title, legend and `save_figure` for Figure 13a.7.

What Figure 13a.7 shows. A curve shaped like a parabola that touches zero at $c = 1$ and is flat there: moving c by 0.05 away from 1 raises the energy by only about 0.001, while the ends lie much higher, 0.29 at $c = 0$ (the orbitals of the bare trap) and 0.72 at $c = 2$ (the orbitals of the Hartree-like potential $v + g_c n$). A first-order error in the orbitals gives only a second-order error in the energy.

In [21], Hartree only and Thomas-Fermi.

```
def hartree_map(w):
    """One pass of the Hartree approximation: w_out = g_c n (no exchange)."""
    return G_C * density(orbitals(w)[1])


w_hartree, hartree_residuals = anderson(hartree_map, np.zeros(M))
n_hartree = density(orbitals(w_hartree)[1])
n_free = density(free_phi)  # no interaction at all
```

The Hartree approximation has the mean-field potential $g_c n$ instead of $g_c n/2$; Anderson mixing solves it, and `n_hartree` is its density. `n_free` is the density without interaction.

```
def thomas_fermi(mu):
    """The Thomas-Fermi density: the positive root of a n^2 + b n = mu - v."""
    a, b = np.pi ** 2 / 8.0, 0.5 * G_C
    room = np.maximum(mu - v, 0.0)  # zero where the trap is higher than mu
    return (-b + np.sqrt(b * b + 4.0 * a * room)) / (2.0 * a)
```

The Thomas-Fermi density of Section 13.32: `np.maximum(mu - v, 0.0)` replaces $\mu - v$ by 0 where the trap is higher than μ, which makes the root 0 there.

```
low, high = 0.0, 50.0  # mu lies between these two numbers
for _ in range(60):  # bisection: halve the interval 60 times
    middle = 0.5 * (low + high)
    if integral(thomas_fermi(middle)) < N_TOTAL:
        low = middle  # too few particles: mu must be larger
    else:
        high = middle
mu_tf = 0.5 * (low + high)
n_tf = thomas_fermi(mu_tf)
```

Bisection for μ: the number of particles grows with μ; with $\mu = 0$ there are none and with $\mu = 50$ far more than 8. 60 halvings of the interval of width 50 leave a width of $50/2^{60} \approx 4 \cdot 10^{-17}$, below the rounding of the computer.

```
report("Thomas-Fermi chemical potential mu", f"{mu_tf:.6f}")
check(abs(integral(n_tf) - N_TOTAL) < 1e-9, "the Thomas-Fermi density holds 8")
check(abs(integral(n_hartree) - N_TOTAL) < 1e-10, "the Hartree density holds 8")
```

The cell prints $\mu = 5.110729$ and checks that the Thomas-Fermi and the Hartree densities hold 8 particles.

```
widths = [np.sqrt(integral(x ** 2 * n) / N_TOTAL)
          for n in (n_free, n_ks, n_hartree)]
say("root-mean-square widths: free {:.6f}, Kohn-Sham {:.6f}, Hartree {:.6f}"
    .format(*widths))
check(widths[0] < widths[1] < widths[2],
      "repulsion widens the cloud, and self-interaction (Hartree) widens it more")
```

The **root-mean-square width** $\sqrt{\int x^2 n\, dx / N}$ measures how far the cloud spreads; `.format(*widths)` puts the three numbers into the three braces of the string. The widths 1.413346, 1.542829, 1.659857 must increase in this order (last check).

In [22], four pictures of the same fermions.

```
fig, ax = plt.subplots()
ax.plot(x, n_free, ":", label="no interaction")
ax.plot(x, n_hartree, "--", label="Hartree only (self-interaction)")
ax.plot(x, n_ks, lw=2.0, color="black", label="Kohn-Sham (Hartree + exchange)")
ax.plot(x, n_tf, "-.", label="Thomas-Fermi (local kinetic energy)")
ax.set_xlim(-5.0, 5.0)
ax.set_xlabel("position $x$")
ax.set_ylabel("density (particles per unit length)")
ax.set_title("Four pictures of the same eight fermions ($g_c = 2$)")
ax.legend(fontsize=8)
save_figure(fig, "approximations",
```

The four densities, without interaction (dotted), Hartree only (dashed), Kohn-Sham (thick black) and Thomas-Fermi (dash-dotted); range, labels, title, legend and `save_figure` for Figure 13a.8.

What Figure 13a.8 shows. Without interaction the cloud is the narrowest and the highest (peaks of about 1.89); Kohn-Sham is lower and wider; Hartree only is lower and wider still, because each fermion also pushes against its own density. All three have the bumps of the four occupied orbitals. The Thomas-Fermi density is a smooth dome without bumps, close to the Kohn-Sham density on average, and it ends abruptly at $|x| = \sqrt{2\mu} = 3.20$, where the trap reaches μ (the orbital densities instead die away gradually beyond that point).

In [23], switching the interaction on.

```
def solve_at(strength, start):
    """The self-consistent w and the three energies (T_s, int v n dx, E_H + E_x)
    at the coupling strength, starting the loop from the potential start."""

    def strength_map(w):  # the Kohn-Sham map with this strength instead of G_C
        return 0.5 * strength * density(orbitals(w)[1])

    w, _ = anderson(strength_map, start)
    phi = orbitals(w)[1]
    n = density(phi)
    parts = (kinetic(phi, [2.0] * N_PER_LABEL), integral(v * n),
             0.25 * strength * integral(n ** 2))
    return w, parts
```

`solve_at` solves the model for any strength: the inner function `strength_map` is the Kohn-Sham map with that strength, Anderson mixing solves it from the given start, and the function returns the potential and the three parts T_s, $\int v\,n$ and $E_H + E_x$.

```
strengths = np.linspace(0.0, 2.0, 9)  # g_c = 0, 0.25, ..., 2
scan, w_start = [], np.zeros(M)
for strength in strengths:
    w_start, parts = solve_at(strength, w_start)  # start from the last solution
    scan.append(parts)
scan = np.array(scan)  # one row per strength; columns: T_s, int v n, E_H + E_x
say(f"E at g_c = 0: {scan[0].sum():.6f}; at g_c = 2: {scan[-1].sum():.6f}")
check(abs(scan[-1].sum() - E_total) < 1e-9, "the scan ends at the same E as above")
```

Nine strengths from 0 to 2, each solved starting from the solution of the previous strength (a good start saves passes). The energy is 15.990192 at $g_c = 0$ (the grid's version of 16) and 21.851498 at $g_c = 2$, the same as before (check).

```
delta = 1e-3
E_plus = sum(solve_at(2.0 + delta, w_scf)[1])  # E at g_c = 2.001
E_minus = sum(solve_at(2.0 - delta, w_scf)[1])  # E at g_c = 1.999
slope = (E_plus - E_minus) / (2.0 * delta)
```

The Hellmann-Feynman test: the energies at $g_c = 2.001$ and 1.999 (each solved by Anderson mixing from the solution at $g_c = 2$; `sum(...)` adds the three parts) and their central difference quotient.

```
report("dE/dg_c at g_c = 2 (difference quotient)", f"{slope:.6f}")
report("(1/4) int n^2 dx at g_c = 2 (Hellmann-Feynman)",
```

```
       f"{0.25 * integral(n_ks ** 2):.6f}")
check(abs(slope - 0.25 * integral(n_ks ** 2)) < 1e-6,
      "Hellmann-Feynman: dE/dg_c = (1/4) int n^2 dx")
```

Both the quotient and $\frac{1}{4}\int n^2\,dx$ print as 2.801961, and the check requires agreement to 10^{-6} (the quotient has an error of order $\delta^2 = 10^{-6}$ times the third derivative, plus the effect of the loop's tolerance).

In [24], the energies of the scan.

```
fig, ax = plt.subplots()
ax.plot(strengths, scan[:, 0], "o-", ms=4, label="kinetic $T_s$")
ax.plot(strengths, scan[:, 1], "s-", ms=4, label="trap $\\int v\\,n\\,dx$")
ax.plot(strengths, scan[:, 2], "^-", ms=4, label="interaction $E_H + E_x$")
ax.plot(strengths, scan.sum(axis=1), "D-", ms=4, color="black",
        label="total $E$")
ax.set_xlabel("strength $g_c$ of the contact repulsion")
ax.set_ylabel("energy (units of $\\hbar\\omega$)")
ax.set_title("Switching the repulsion on")
ax.legend(fontsize=8)
save_figure(fig, "coupling_scan",
```

The three columns of the table `scan` against the nine strengths, with different markers, and their sum along each row (`scan.sum(axis=1)`) as the black total; labels, title, legend and `save_figure` for Figure 13a.9. After the caption, the check

```
check(abs(scan[0].sum() - 16.0) < 0.05 and abs(scan[0, 0] - scan[0, 1]) < 0.05,
      "at g_c = 0: E = 16 and T_s equals the trap energy (virial theorem)")
```

requires, at $g_c = 0$ (row 0 of the scan), the total 16 and equal kinetic and trap energies (the virial theorem of Section 13.32), both to 0.05, the accuracy of the grid.

What Figure 13a.9 shows. At $g_c = 0$ the kinetic and the trap energy are both 8 and the total is 16 (the grid gives 15.99). As the repulsion is switched on, the interaction energy grows almost linearly to 5.60; the cloud spreads, so the trap energy rises (to 9.52) and the kinetic energy falls (to 6.73): wider orbitals curve less. The total rises to 21.85 along a curve that bends slightly downwards: by the Hellmann-Feynman theorem its slope is $\frac{1}{4}\int n^2\,dx$, which is 3.07 at $g_c = 0$ (with the density without interaction) and 2.80 at $g_c = 2$, because the spreading cloud has a smaller $\int n^2\,dx$.

In [25], the first excited state.

```
taus = np.linspace(0.0, 1.0, 9)
gaps, energies, w_pair = [], [], np.concatenate([w_scf, w_scf])
for tau in taus:
    up = [1.0, 1.0, 1.0, 1.0 - tau, tau]  # orbitals 0..4 of label up
    w_pair, _ = anderson(lambda q: labels_map(q, up, FILLED), w_pair)
    up_levels = label_densities(w_pair, up, FILLED)[2]
    gaps.append(up_levels[4] - up_levels[3])  # eps_4(tau) - eps_3(tau)
    energies.append(labels_energy(w_pair, up, FILLED))
gaps, energies = np.array(gaps), np.array(energies)
n_excited = sum(label_densities(w_pair, up, FILLED)[:2])  # tau = 1: n_up + n_down
```

Nine values of the moved fraction $\tau = 0, \frac{1}{8}, \ldots, 1$. For each, label up has the occupations $1, 1, 1, 1-\tau, \tau$ in its orbitals 0 to 4 and label down its four lowest orbitals full; the two-label loop is solved by Anderson mixing, starting from the previous solution, and the level difference $\epsilon_4 - \epsilon_3$ of label up and the energy are stored. After the loop `n_excited` is the total density at $\tau = 1$, the Delta-SCF excited state.

```
step = taus[1] - taus[0]
simpson = step / 3.0 * (gaps[0] + gaps[-1] + 4.0 * gaps[1:-1:2].sum()
                        + 2.0 * gaps[2:-1:2].sum())
delta_scf = energies[-1] - energies[0]
```

Simpson's rule (Section 13.15) on the nine level differences (the name `simpson` is here a number, not a function), and the Delta-SCF energy $E(1) - E(0)$.

```
report("Kohn-Sham gap (LUMO - HOMO)", f"{gaps[0]:.6f}")
report("Delta-SCF excitation energy E(1) - E(0)", f"{delta_scf:.6f}")
report("Janak integral of eps_4 - eps_3 (Simpson)", f"{simpson:.6f}")
report("transition state eps_4 - eps_3 at tau = 1/2", f"{gaps[4]:.6f}")
check(abs(energies[0] - E_total) < 1e-9, "tau = 0 is the ground state")
check(abs(gaps[0] - (ks_levels[4] - ks_levels[3])) < 1e-9,
      "at tau = 0 the level difference is the Kohn-Sham gap")
check(abs(simpson - delta_scf) < 1e-6,
      "Janak's theorem: the integral of eps_4 - eps_3 equals Delta-SCF")
check(delta_scf < gaps[0], "orbital relaxation lowers the excitation energy here")
```

The `report` lines print the gap 0.772662, the Delta-SCF energy 0.714594, the Janak integral 0.714594 and the transition state 0.712784 (entry 4 is $\tau = \frac{1}{2}$). The four checks: $\tau = 0$ is the ground state; the level difference at $\tau = 0$ is the Kohn-Sham gap of In [11] ($5.496606 - 4.723945$); the Janak integral equals the Delta-SCF energy to 10^{-6}; and the Delta-SCF energy is below the gap here.

In [26], the excited state as a picture.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.fill_between(taus, 0.0, gaps, alpha=0.25, label="area = Delta-SCF")
left.plot(taus, gaps, "o-", color="black", label="$\\epsilon_4 - \\epsilon_3$")
left.plot([0.0], [gaps[0]], "s", ms=8, label=f"Kohn-Sham gap {gaps[0]:.4f}")
left.plot([0.5], [gaps[4]], "^", ms=8, label=f"transition state {gaps[4]:.4f}")
left.axhline(delta_scf, color="gray", ls="--", label=f"Delta-SCF {delta_scf:.4f}")
```

On the left, `left.fill_between(taus, 0.0, gaps, alpha=0.25, ...)` shades the area between zero and the level difference, which is the Delta-SCF energy by Janak's theorem; the level difference itself as black dots joined by a line; the gap as a large square at $\tau = 0$, the transition state as a large triangle at $\tau = \frac{1}{2}$, and the Delta-SCF energy as a grey dashed horizontal line, each with its value in the legend (four decimals).

```
left.set_ylim(0.0, 1.0)
left.set_xlabel("fraction $\\tau$ of the moved fermion")
left.set_ylabel("level difference ($\\hbar\\omega$)")
left.legend(fontsize=7, loc="lower left")
```

The vertical range from 0 to 1, labels, and the legend in the lower left corner, inside the shaded area.

```
right.plot(x, n_ks, color="black", label="ground state")
right.plot(x, n_excited, "--", label="excited (Delta-SCF)")
right.set_xlim(-5.0, 5.0)
right.set_xlabel("position $x$")
right.set_ylabel("density (particles per unit length)")
right.legend(fontsize=8)
fig.suptitle("The first excited state by Delta-SCF ($g_c = 2$)")
save_figure(fig, "delta_scf",
```

On the right the ground-state density (black) and the density of the Delta-SCF excited state (dashed); range, labels, legend, a common title, and `save_figure` for Figure 13a.10.

What Figure 13a.10 shows. On the left the level difference falls almost linearly from the gap 0.7727 at $\tau = 0$ to about 0.66 at $\tau = 1$: as the up fermion moves into orbital 4, the levels rearrange so that the two orbitals come closer. The shaded area, 0.7146, is the Delta-SCF energy (dashed line); it is smaller than the gap, and the midpoint value 0.7128 (triangle) is close to it, because the curve is nearly straight. On the right the excited density differs from the ground-state density mainly in the middle: the ground state has two peaks beside a dip at $x = 0$, the excited state a peak at $x = 0$ and shoulders beside it (orbital 4 has its largest value at the centre, orbital 3 a zero there), and slightly more density in the outer flanks.

In [27], the last check.

```
figure_names = ["trap_orbitals", "scf_convergence", "first_iterations",
                "ks_potentials", "density_orbitals", "label_separation",
                "variational_scan", "approximations", "coupling_scan",
                "delta_scf"]
missing = [name for k, name in enumerate(figure_names, 1)
           if not output_file(f"{FIGURE_FOLDER}/13a_{k}_{name}.png").is_file()]
check(missing == [], "all ten figure files exist")
check(output_file(f"{FIGURE_FOLDER}/13a_10_delta_scf.png").is_file(),
      "the figure file 13a_10_delta_scf.png exists")
all_checks_passed()
```

The same lines as In [19] of Notebook 13c (Section 13.14), with the ten figure names of this notebook. The last line prints ALL 36 CHECKS PASSED (notebook 13a): one check each in In [2], In [5], In [6], In [7], In [10], In [12], In [15] and In [24], two each in In [3], In [8], In [11], In [14], In [16], In [18], In [19], In [23] and In [27], three each in In [17] and In [21], and four in In [25].

13.37 From the toy models to dirac16complex

This section says which ideas of the chapter the Kohn-Sham model of dirac16complex (Chapters 14 to 16) uses, and what is different there. As in Section 13.16, x_1, x_2, x_3 are the directions of ordinary space, x_4 the time, x_5, x_6, x_7 the three extra times, which deflate exponentially (scale factor $e^{-a_4}\sin^{1/6} z$ while ordinary space inflates with $e^{a_4}\sin^{1/6} z$), and x_0 the hidden direction, $z = 6Hx_0$, with the author's constant $H > 0$.

What carries over.

- **The Kohn-Sham scheme and self-consistency.** The model is a Kohn-Sham fermion gas of dirac16complex quanta in the good sector (no momentum along the extra times), in which the 16-component equation reduces to 2×2 blocks in the hidden coordinate (Chapter 14). Its levels and orbitals are found self-consistently, with Anderson mixing of the potentials at every grid point of the hidden direction (depth 6, $\beta = 0.4$, stopping rule 10^{-11} on the largest residual; Section 13.21 and `Revision/kohn_sham/results/parameters.json`, numerics).

- **The interaction and its exchange.** The contact interaction $\frac{\lambda}{2}S^2$ with the vertex C, treated as Hartree plus the exact local exchange of the uniform 8-fold gas: $e_x = -\frac{\lambda}{32}(n^2 + S^2)$, $M_{eff} = m + \frac{15}{16}\lambda S$, $v_v = -\frac{1}{16}\lambda n$ (Section 13.16; PROVED in `Revision/kohn_sham/reports/ks-theory-python.json`, checks exchange_uniform_gas, ks_potentials, filled_shell_ratio).

- **Mermin's occupations.** At the temperatures $T = 0.01$, 0.02 and 0.05 (in units of the mass m; parameters.json, physics temperatures, checked by Notebook 13b, In [22]) the occupations are Fermi-Dirac numbers with μ fixed by $\sum gf = N$, solved in the LogBalance form (Section 13.26; parameters.json, conventions merminRoot).

- **Excited states.** The Kohn-Sham gap and the particle-hole excitations of Section 13.27, and Delta-SCF in its ensemble form over groups of equal levels (parameters.json, conventions deltaScf; Notebook 13b, In [22]).

- **Densities through the Krein form.** The number density is $n = \mathrm{Tr}(B\rho)$ with the indefinite form $B = -iC\gamma^{(x4)}$ of Chapter 10, and the scalar density is $S = \mathrm{Tr}(C\rho)$.

What is different, and its status.

- **No correlation.** The dirac16complex functional has no correlation term ("correlation: none (Hartree plus exchange only)" in `Revision/kohn_sham/ks-theory.json`), and for the non-uniform Kohn-Sham determinants its uniform-gas exchange differs from the exact local Fock exchange by $+\frac{\lambda}{32}Q^2$ (Section 13.16). These are the approximations of the model.

- **No density-functional theorem is claimed for the field.** The Hohenberg-Kohn and Mermin theorems of Sections 13.8 and 13.26 were proved for particles with a positive inner product and a Hamiltonian bounded from below. For the quantised dirac16complex field with its indefinite Krein form (Chapter 10) the book does not prove such a theorem; the model of Chapter 14 is used as a self-consistent mean-field (exchange-only) model, and whether an exact density functional exists for this field is OPEN.

- **Which levels are filled.** Particles occupy the positive branch of the levels and the brane zero modes; this filling is a CONVENTION of the record, and its justification is OPEN (`Revision/kohn_sham/ks-theory.json`, thermodynamics, fillingConvention; Notebook 13b, In [22]).

- **The background.** The history $a_4 = AHx_4$ along which the instantaneous (adiabatic) Kohn-Sham states are computed is a PRESCRIBED BACKGROUND: the Kohn-Sham states violate the conditions that the a_4 field equations put on their source (`Revision/field_equations_a4/reports/ks-source-conditions.json`; the checks are named in Section 13.38 and checked by Notebook 13b, In [22]). The time-dependent (non-adiabatic) problem is OPEN, and the mirror at the end of the hidden direction (the Z2 brane) is ASSUMED.

The pairs. Chapter 19 uses these Kohn-Sham states for theorem T3: the Kohn-Sham universes of mass $+M$ and $-M$, with the transformed boundary conditions, have equal energies and energy-momentum tensors (PROVED: `Revision/pairing/kohn_sham/reports/python-t3.json` and, independently, `wolfram-t3.json` in the same folder; ten checks of the theorem that are present in both reports, among them the equality of the energies and of the energy-momentum profiles, are PASS in both, as Notebook 13b checks in In [22]). T3 is an exact map between two sets of solutions. It does not prove that any universe is created, in pairs or otherwise: no creation process, rate or amplitude follows from these equations, and nothing in this chapter changes that.

13.38 What we proved, what we computed, what we assumed

PROVED (exact derivations in this chapter, each checked numerically by a notebook; the Revision checks are named where the record holds the statement):

- One particle: the eigenvalues of a Hermitian operator are real and its eigenvectors for different eigenvalues orthogonal; the variational principle (Section 13.2).
- Many fermions: the Pauli principle; Slater determinants of N particles are antisymmetric, normalised, vanish for equal orbitals, and have the density $\sum_a |\phi_a|^2$ (Section 13.3, from the Leibniz formula; Notebook 13c, In [2], for $N = 2$).
- Second quantisation: the anticommutation relations; Wick's theorem for a determinant and a thermal ensemble of non-interacting fermions, in every basis; $\langle :S^2: \rangle = (\mathrm{Tr}\, V\rho)^2 - \mathrm{Tr}(V\rho V\rho)$ (Section 13.4; Revision check hf_wick_contraction of `Revision/kohn_sham/reports/ks-theory-python.json`, its verdict checked by Notebook 13b, In [21]; Notebook 13c, In [4] to In [8]).
- The energy of a determinant, direct minus exchange; the cancellation of the self-interaction; the exact locality of contact exchange and $E_x = -E_H/g$ for equally occupied labels (Sections 13.5 and 13.15; Notebook 13c, In [10]; Notebook 13b, In [6], In [12], In [13]).
- The Lagrange-multiplier rule and the meaning of the multiplier; the Hartree-Fock equations (the variation written out term by term, with the Hermitian multiplier matrix), the double-counting formula and Koopmans' theorem (Section 13.6).
- The two-site model: $E_0 = \frac{1}{2}(U - \sqrt{U^2 + 16t^2})$ and the best determinant ($-2t + U/2$ for $U \le 2t$, $-2t^2/U$ beyond) (Section 13.7; Notebook 13c, In [11], In [12]).
- The Hohenberg-Kohn theorem and the constrained-search variational principle; functional derivatives; the Kohn-Sham equations and their total energy; the exact Kohn-Sham potential of two sites (Sections 13.8 to 13.10; Notebook 13c, In [16]).
- The uniform gas: $n = gk_F^3/(6\pi^2)$, C_F, the density matrix $F(k_F R)$, the exchange hole $1 - F^2/g$, the small-range law $1 - \frac{3}{5}(k_F a)^2$, Dirac's formula given the integral $9/4$, and the polarisation threshold $\frac{2}{3}(3\pi^2)^{2/3}$ (Section 13.15; Notebook 13b).
- The exchange of the 16-component field: $e_x = -\frac{\lambda}{32}(n^2 + S^2)$, $M_{eff} = m + \frac{15}{16}\lambda S$, $v_v = -\frac{1}{16}\lambda n$, the ratio $-1/8$ (Section 13.16; `Revision/kohn_sham/reports/ks-theory-python.json`, checks exchange_uniform_gas, ks_potentials and filled_shell_ratio; independently `ks-theor`

`y-wolfram.json` in the same folder, checks exchange_uniform_gas and ks_potentials; reproduced exactly by Notebook 13b, In [17], In [18], which with In [21] also checks the verdicts of both programs).

- Mixing: the reduced map of the two-site loop, the convergence factor $1-\beta(1-G')$ and the condition $0<\beta<2/(1-G')$ for small errors, the least-squares form of Anderson mixing (Section 13.21).

- Temperature: the Gibbs principle with Klein's inequality; Mermin's theorem and the universal functional at a temperature (both written out in Section 13.26); the independence of the orbitals of non-interacting fermions; the Fermi-Dirac occupations; $dF/dT=-S$, $C_V=T\,dS/dT$, the variance formula and $C_V\geq 0$ (Section 13.26; Notebook 13e).

- Excited states: Janak's theorem and the Delta-SCF integral formula (Section 13.27; Notebooks 13e and 13a).

- The trap model: its Kohn-Sham potential $v+g_c n/2$, the double-counting formula, the one-dimensional Thomas-Fermi equation, the Hellmann-Feynman theorem and the virial theorem (Section 13.32).

COMPUTED (numbers printed by the five executed notebooks of the chapter, which are stored in the folder `Revision/textbook/notebooks`). Each number is given with its uncertainty, which is of one of two kinds. The finite models of Notebooks 13b to 13e are solved exactly up to rounding; for them the brackets give the tolerance to which a check confirms the printed digits, or say how the number was located. The trap of Notebook 13a is solved on a grid of 200 points: its numbers are exact for this grid model up to the stopping rule 10^{-11} of the loop (the brackets give the checks), but they differ from the numbers of the continuum model of Section 13.32 by the error of the grid, about 10^{-2}, which the notebook measures twice: the grid levels of the trap miss the exact levels $n+\frac{1}{2}$ by up to 0.013 (In [3]), and the energy at $g_c=0$ is 15.990192 instead of the exact 16, a difference of 0.0098 (In [23]). As numbers of the continuum model, the six-decimal values of Notebook 13a are therefore good to about two decimals.

- Notebook 13c: the two-site numbers $E_0=-1.236068\,t$ and correlation energy $-0.236068\,t$ at $U=2t$ (the eigenvalue agrees with the closed form to 10^{-12}; In [11], In [12]) and the double occupancy 0.276393 (to 10^{-6}; In [14]); the place of the largest correlation energy, $U=3.335\,t$ (the closed form evaluated on a grid of step 10^{-4} in U and printed to three decimals, so uncertain by $5\cdot 10^{-4}$; no check; In [12]); and the split of the exact Kohn-Sham screening at $U=4t$, $\Delta=2t$ into -0.588239 (Hartree-exchange) and -1.114408 (correlation) (the inverted site energies reproduce the exact densities to 10^{-12}, In [16]; printed to six decimals, In [18]).

- Notebook 13b: box sums within 0.00083 of F for 137059 plane waves (a measured difference; In [3]); $\int_0^\infty sF^2\,ds=9/4$ to 10^{-7} (In [10]); the box exchange -19.0000000000 in two forms (they agree to 10^{-12}, and equal the exact -19 of Exercise 10; In [12]); the label-mixing exchange -8.7398756465 (two forms agree to 10^{-10}; In [13]).

- Notebook 13d: $n_L^*=1.326993$, $G'=-1.687961$, $\beta_{max}=0.744058$ (to 10^{-6}; In [3]), the cycle 0.3607/1.9157 (two passes return to the same value to 10^{-10}; printed to four decimals; In [5]), convergence in 13 passes at $\beta=\frac{1}{2}$ (a count; In [6]), and the threshold $U=2.647393$ (found by 60 halvings of an interval; there the slope is -1 to 10^{-9}; In [12]).

- Notebook 13e: no state among 2000 has a lower grand potential than the Gibbs state (the smallest excess, $3.9 \cdot 10^{-6}$, is a measured value for these random states; In [4]); the two-level numbers 0.731059, 1.164406, -0.313262, 0.393224 (to 10^{-6}; In [8], In [10]); the ladder's $\mu = 4$ at $T = 0.05$ (to 10^{-6}; In [12]).
- Notebook 13a (numbers of the grid model; its distance from the continuum model is about 10^{-2}, as said above): $E = 21.851498$ at $g_c = 2$ (the two formulas agree to 10^{-9}; In [14]), the Kohn-Sham gap 0.772662 (the level difference at $\tau = 0$ agrees with it to 10^{-9}; In [25]), the Delta-SCF energy 0.714594 (the Janak integral agrees to 10^{-6}; In [25]), the widths 1.413346, 1.542829 and 1.659857 (printed to six decimals; only their order is checked; In [21]) and the Hellmann-Feynman slope 2.801961 (the two ways agree to 10^{-6}; In [23]); and the Revision solver's mixing settings, read and checked exactly from `Revision/kohn_sham/results/parameters.json` (In [7]).

ASSUMED:

- that the particles are fermions (for dirac16complex: the choice of Grassmann components, Chapter 7);
- for the Hohenberg-Kohn theorem: a non-degenerate ground state, a wave function that does not vanish on a whole region, and that the constrained minima exist; for the Kohn-Sham scheme: non-interacting v-representability;
- for the Lagrange rule: the implicit function theorem (used without proof); for the Gibbs principle: a finite number of states;
- that a Delta-SCF state approximates a true excited state;
- the approximations: no correlation in the toy models (they are Hartree-Fock models); in the dirac16complex functional, the uniform-gas exchange and no correlation (Sections 13.16 and 13.37);
- for the dirac16complex model (Section 13.37): the Z2 mirror, and the history of a_4 as a prescribed background.

HYPOTHESIS: none is used in this chapter. **OPEN:** an exact density functional for the quantised dirac16complex field, the justification of its filling convention, and its time-dependent (non-adiabatic) problem (Section 13.37).

Where the background status is recorded. That the history $a_4 = AHx_4$ is a prescribed background, not a solution of the a_4 field equations with the Kohn-Sham source, is recorded in the Revision report `Revision/field_equations_a4/reports/ks-source-conditions.json` by its checks ks_history_is_a_prescribed_background, ks_profiles_violate_algebraic_condition and ks_profiles_depend_on_x0, each with the verdict PASS (Notebook 13b, In [22], reads the report and checks the three verdicts).

13.39 Exercises

Exercise 1. In the three-point example of Section 13.3, compute $\Phi(2,1)$, $\Phi(1,0)$ and $\Phi(1,1)$, and show from the definition that the two-particle determinant vanishes when $\phi_a = \phi_b$.

Answer. With $\phi_0 = (1,0,0)$ and $\phi_1 = (0,1,1)/\sqrt{2}$: $\Phi(2,1) = [\phi_0(2)\phi_1(1) - \phi_1(2)\phi_0(1)]/\sqrt{2} = [0 \cdot \frac{1}{\sqrt{2}} - \frac{1}{\sqrt{2}} \cdot 0]/\sqrt{2} = 0$. $\Phi(1,0) = [\phi_0(1)\phi_1(0) - \phi_1(1)\phi_0(0)]/\sqrt{2} = [0 \cdot 0 - \frac{1}{\sqrt{2}} \cdot 1]/\sqrt{2} = -\frac{1}{2}$, as the table of Notebook 13c (In [2]) shows. $\Phi(1,1) = [\phi_0(1)\phi_1(1) - \phi_1(1)\phi_0(1)]/\sqrt{2} = 0$, because the two products are equal. If $\phi_a = \phi_b = \phi$, then $\Phi(r_1, r_2) = [\phi(r_1)\phi(r_2) - \phi(r_1)\phi(r_2)]/\sqrt{2} = 0$ for all arguments.

Exercise 2. Using only the anticommutation relations of Section 13.4, show that $\hat{n}_p^2 = \hat{n}_p$, and conclude that $\hat{n}_p$ has only the eigenvalues 0 and 1.

Answer. $\hat{n}_p^2 = a_p^\dagger a_p a_p^\dagger a_p$. The relation $\{a_p, a_p^\dagger\} = 1$ gives $a_p a_p^\dagger = 1 - a_p^\dagger a_p$, so $\hat{n}_p^2 = a_p^\dagger(1 - a_p^\dagger a_p)a_p = a_p^\dagger a_p - a_p^\dagger a_p^\dagger a_p a_p$. The relation $\{a_p^\dagger, a_p^\dagger\} = 0$ says $2a_p^\dagger a_p^\dagger = 0$, so the last term vanishes and $\hat{n}_p^2 = \hat{n}_p$. If $\hat{n}_p u = \nu u$ with $u \neq 0$, then $\nu^2 u = \hat{n}_p^2 u = \hat{n}_p u = \nu u$, so $\nu^2 = \nu$, that is $\nu = 0$ or $\nu = 1$.

Exercise 3. For the three-point example, write the density matrix $\rho = \phi_0\phi_0^T + \phi_1\phi_1^T$ as a 3×3 matrix and check $\rho^2 = \rho$ and $\operatorname{Tr}\rho = 2$.

Answer. $\phi_0\phi_0^T$ has a single 1 in the top left corner; $\phi_1\phi_1^T$ has the entries $\frac{1}{2}$ in the lower right 2×2 block. So $\rho = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix}$. Squaring, the corner gives $1 \cdot 1 = 1$, and the block $\begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{pmatrix}^2$ has every entry $\frac{1}{4} + \frac{1}{4} = \frac{1}{2}$, so $\rho^2 = \rho$. The trace is $1 + \frac{1}{2} + \frac{1}{2} = 2$, the number of particles, and the diagonal $(1, \frac{1}{2}, \frac{1}{2})$ is the density of Section 13.3.

Exercise 4. Solve the two-site model of Section 13.7 for $t = 1$, $U = 4$: the exact energy, the Hartree-Fock energy, the correlation energy, and the exact double occupancy.

Answer. $E_0 = \frac{1}{2}(4 - \sqrt{16 + 16}) = 2 - 2\sqrt{2} = -0.828427$. Since $U > 2t$, the best determinant is unrestricted, $E_{HF} = -2t^2/U = -0.5$. The correlation energy is $E_0 - E_{HF} = -0.328427$. The ground state lies in the 2×2 problem $\begin{pmatrix} U & -2t \\ -2t & 0 \end{pmatrix}$ on (D, S); its first row gives $(U - E_0)\, d - 2t\, s = 0$, so $s = (U - E_0)\, d/(2t) = (4 + 0.828427)\, d/2 = 2.414214\, d$. The normalisation $d^2(1 + 2.414214^2) = 1$ gives $d^2 = 1/6.828427 = 0.146447$. The double occupancy is the weight of the states LL and RR, which here is $d^2 = 0.146447$, the value of Notebook 13c (In [11]); the restricted determinant would give $\frac{1}{2}$.

Exercise 5. Fermions with two labels and a contact interaction of strength $g_c = 1$ have the label densities $n_\uparrow = 0.3$ and $n_\downarrow = 0.1$ at a point. Compute the Hartree and the exchange energy densities and check that their sum is $g_c\, n_\uparrow n_\downarrow$.

Answer. $n = 0.4$, so $e_H = \frac{g_c}{2} n^2 = 0.5 \cdot 0.16 = 0.08$ and $e_x = -\frac{g_c}{2}(n_\uparrow^2 + n_\downarrow^2) = -0.5 \cdot (0.09 + 0.01) = -0.05$. The sum is $0.03 = 0.3 \cdot 0.1 = n_\uparrow n_\downarrow$. With unequal labels the exchange is not $-e_H/2 = -0.04$: the rule $-1/g$ needs equally occupied labels.

Exercise 6. Repeat the mixing analysis of Section 13.21 for $\Delta = 2$, $t = 1$, $U = 2$: check that $x^* = 0.468990$ is the fixed point, compute $G'(x^*)$ and β_{max}, and decide whether plain iteration converges.

Answer. $\Delta - Ux^* = 2 - 0.937980 = 1.062020$, and $\sqrt{1.062020^2 + 4} = \sqrt{5.127886} = 2.264484$, so $G(x^*) = 1.062020/2.264484 = 0.468990 = x^*$. Then $G'(x^*) = -4 \cdot 2 \cdot 1/5.127886^{3/2} = -8/(5.127886 \cdot 2.264484) = -8/11.612 = -0.688942$, and $\beta_{max} = 2/(1 + 0.688942) = 1.184174$. Since $1 < \beta_{max}$, plain iteration converges: its factor $1 - (1 + 0.688942) = -0.688942$ has a size below 1, so the error

shrinks by about 0.69 per pass while changing its sign (the density still swings, but the swings die out). These are the numbers of Notebook 13d (In [12]).

Exercise 7. For the two-level example of Section 13.26 (levels 0 and 1, one particle), find the limits of f_0, E and S_s as $T \to 0$ and as $T \to \infty$.

Answer. By symmetry $\mu = \frac{1}{2}$ at every T, so $f_0 = 1/(e^{-1/(2T)} + 1)$ and $f_1 = 1 - f_0$. As $T \to 0$, $e^{-1/(2T)} \to 0$: $f_0 \to 1$, $f_1 \to 0$, $E = f_1 \to 0$, and $S_s \to 0$, because $f \ln f$ and $(1-f)\ln(1-f)$ tend to 0 when f tends to 0 or 1. As $T \to \infty$, $e^{-1/(2T)} \to 1$: $f_0, f_1 \to \frac{1}{2}$, $E \to \frac{1}{2}$, and each level contributes $-2 \cdot \frac{1}{2} \ln \frac{1}{2} = \ln 2$, so $S_s \to 2 \ln 2 = \ln 4$. Notebook 13e (In [11]) checks both limits.

Exercise 8. For the model energy $E = \epsilon_H^0 f_H + \epsilon_L^0 f_L + \frac{U}{2}(f_H^2 + f_L^2)$ with any $\epsilon_H^0 < \epsilon_L^0$ and $U < \epsilon_L^0 - \epsilon_H^0$, show that $\Delta_{SCF} - \Delta_{KS} = U$.

Answer. By Janak's theorem the levels are $\epsilon_H = \epsilon_H^0 + U f_H$ and $\epsilon_L = \epsilon_L^0 + U f_L$. In the ground state $(1,0)$ (the lower orbital filled; the condition on U keeps $\epsilon_H < \epsilon_L$ there), $\Delta_{KS} = \epsilon_L^0 - (\epsilon_H^0 + U)$. The energies are $E(1,0) = \epsilon_H^0 + \frac{U}{2}$ and $E(0,1) = \epsilon_L^0 + \frac{U}{2}$, so $\Delta_{SCF} = \epsilon_L^0 - \epsilon_H^0$, and $\Delta_{SCF} - \Delta_{KS} = U$.

Exercise 9. For the uniform 8-fold gas of dirac16complex (Section 13.16), compute e_x/e_H when $S = n/2$, and the interaction energy e_{int} when $S = n$.

Answer. With $S = n/2$: $e_x = -\frac{\lambda}{32}(n^2 + \frac{n^2}{4}) = -\frac{5}{128}\lambda n^2$ and $e_H = \frac{\lambda}{2} \cdot \frac{n^2}{4} = \frac{1}{8}\lambda n^2$, so $e_x/e_H = -\frac{5}{128} \cdot 8 = -\frac{5}{16}$: exchange removes a larger fraction of the Hartree energy than for a filled level at rest ($-\frac{1}{8}$). With $S = n$: $e_{int} = \frac{15}{32}\lambda n^2 - \frac{1}{32}\lambda n^2 = \frac{7}{16}\lambda n^2$, which is $e_H + e_x = \frac{1}{2}\lambda n^2 - \frac{1}{16}\lambda n^2$: in Figure 13b.9 the black curve ends at $7/16 = 0.4375$ at $S/n = 1$.

Exercise 10. Notebook 13b (In [12]) finds the exchange energy -19 for five up and three down fermions in the box $0 < x < 1$ with $g_c = 1$. Derive this number: show that the density $n_N = \sum_{m=1}^{N} 2\sin^2(m\pi x)$ of N fermions with one label has $\int_0^1 n_N^2\,dx = N^2 + N/2$.

Answer. $\int_0^1 n_N^2\,dx = 4\sum_{m,m'} \int_0^1 \sin^2(m\pi x)\sin^2(m'\pi x)\,dx$. With $\sin^2 A = \frac{1}{2}(1 - \cos 2A)$, the product is $\frac{1}{4}[1 - \cos 2A - \cos 2B + \cos 2A \cos 2B]$. Over $0 < x < 1$, $\int \cos(2m\pi x)\,dx = 0$ for $m \geq 1$, and $\int \cos(2m\pi x)\cos(2m'\pi x)\,dx = \frac{1}{2}$ if $m = m'$ and 0 otherwise. So each integral is $\frac{1}{4} + \frac{1}{8}[m = m']$, and $\int n_N^2 = 4[N^2 \cdot \frac{1}{4} + N \cdot \frac{1}{8}] = N^2 + \frac{N}{2}$. Then $E_x = -\frac{g_c}{2}(\int n_\uparrow^2 + \int n_\downarrow^2) = -\frac{1}{2}[(25 + 2.5) + (9 + 1.5)] = -\frac{1}{2} \cdot 38 = -19$.

14 The Kohn-Sham model of dirac16complex in the deflating primordial field

Chapter 13 taught density functional theory from zero: how the hopeless problem of many interacting quanta is replaced by a problem of independent quanta that move in a common, self-made potential, the Kohn-Sham problem. This chapter builds that problem for the quanta of the field dirac16complex in the author's primordial universe, whose three extra times deflate exponentially while ordinary space inflates. Everything is done exactly where it can be done: the change to the hidden coordinate y, the removal of the spin connection, the splitting of the 16 spinor components into eight independent pairs, the boundary conditions, the exact rescaling between the instants of the deflating history, the exact free spectra, and the exact exchange energy of the contact interaction. Four complete notebooks, 14a to 14d, repeat every step with the computer and reproduce the numbers of the Revision record.

14.1 What this chapter does

Why a Kohn-Sham model. The field dirac16complex has 16 complex anticommuting components at every point of the eight-dimensional universe of the author, and its quanta interact through the contact term $U(S) = \frac{\lambda}{2}S^2$ of the Lagrangian (Chapter 7). A state of many such quanta cannot be computed exactly. The Kohn-Sham method of Chapter 13 replaces it by a state of independent quanta, each described by its own wave function, called an **orbital**, which obeys a one-quantum equation with an effective mass M_{eff} and a potential v_v made by all the other quanta. Solving that equation self-consistently is the work of Chapter 15. This chapter prepares it: it turns the one-quantum equation into a form a computer can solve, finds its exact solutions where they exist, and derives the effective mass and potential.

What is derived, in order.

- The hidden coordinate y and the warped form of the author's metric (Section 14.2).
- The Kohn-Sham equation of dirac16complex, the good sector and the stationary-slice ansatz (Section 14.3).
- The spin-connection term $3H\gamma^{(x_0)}$, why the time derivative of a_4 drops out of it, and the factor W^{-3} that removes it (Section 14.4).
- The splitting of the 16 components into eight independent 2×2 blocks of two types (Section 14.5), and the chirality matrix Γ and the rotations of 3-space on these blocks (Section 14.6).
- The exact rescaling identity between the instants of the deflating history (Section 14.7).
- The block equation in real form and its boundary conditions at the brane and at the tip (Section 14.12), the exact free levels at zero 3-momentum (Section 14.13) and the shooting method that finds every level by an integer label (Section 14.14).
- The brane band that grows out of the zero modes when the 3-momentum is switched on, its exact slope, its redshift along the history, its insensitivity to the tip angle at the cutoff $L = 3$ and the slice 0, what the Revision record measured about the position L of the cutoff itself, and the closed shells of the free filling (Sections 14.19 to 14.21).
- The exact local exchange energy of the contact interaction, the Kohn-Sham potentials, the exact Fock exchange of the model's states and the state of the eight zero modes (Sections 14.26 and 14.27).

The four notebooks.

notebook	what it computes	PASS lines	figures
14a	the block reduction and the rescaling identity, with exact algebra	28	5

notebook	what it computes	PASS lines	figures
14b	the free spectra at zero 3-momentum, exact against numerical	20	6
14c	the brane band, its slope, redshift, symmetries and closed shells	15	6
14d	the exact local exchange, the potentials and the zero-mode state	16	4

None of them needs Rust: Notebooks 14b and 14c write the shooting function of the Revision Rust solver (the same RK4 integration and Pruefer angle) in a few lines of numpy, find its roots by plain bisection where the Rust solver uses a safeguarded Newton-bisection, and reproduce the solver's recorded numbers; Chapter 15 runs the Rust solver itself.

The status of every statement. As everywhere in this book, every statement carries one of the five labels of Chapter 0: PROVED, COMPUTED, ASSUMED, HYPOTHESIS, OPEN. In this chapter they are used as follows.

- PROVED: every identity of Sections 14.2 to 14.7, the boundary-condition facts of Section 14.12, the exact spectra of Section 14.13, the counting property of the shooting angle in Section 14.14, the slope formula of Section 14.19 and the exchange formulas of Sections 14.26 and 14.27. The counting property is proved in the text of Section 14.14 and checked numerically by Notebook 14b; each of the others is verified in the Revision record by sympy in `Revision/kohn_sham/reports/ks-theory-python.json` (in its present state 58 of 58 checks passed), most of them also by WolframScript in `Revision/kohn_sham/reports/ks-theory-wolfram.json` (46 of 46 checks passed), and again by the notebooks of this chapter; the check names are given with each statement.
- COMPUTED: the numerical levels, slopes, shifts and closed shells; each comes with its measured error and the record file it reproduces (the Rust solver's report `Revision/kohn_sham/reports/ks-rust-solver.json`, in its present state 42 of 42 checks passed, and its result files).
- ASSUMED: the good sector (no dependence on the extra times); the Z2 mirror brane at $y = 0$; the regular tip condition at the cutoff $y = -L$, $L = 3$ (a choice of the numerical model; Section 14.20 shows that at this cutoff and at the slice $a_{4,0} = 0$, where it was measured, the brane band at nonzero 3-momentum does not feel the tip angle θ, while at the later slices this is not established, and reports what the Revision record measured about the cutoff L itself: the free results with $k \neq 0$ converge as L grows, but the recorded $L = 3$ values are low by up to 1.4% in the Kohn-Sham energy, the nonzero (bulk) $k = 0$ levels approach $\pm m$ only algebraically, and the interaction energy of the brane zero modes depends strongly on L); the deflating history $a_4 = AHx_4$ along which the Kohn-Sham states are computed, which is a **prescribed background**, given and not solved for (Section 14.3); the convention that counts the zero modes at zero 3-momentum as particle levels (Section 14.21).

- OPEN: the justification of that convention (Section 14.21), the effect of the tip angle at the later slices (Section 14.20), and the behaviour of the gas when the history is not slow (the time-dependent problem, Chapter 15 and Chapter 22).
- HYPOTHESIS: none in this chapter.

Notation and units. The author's coordinates are $x_0, \dots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates; x_4 is the time; x_5, x_6, x_7 are the three extra times, which deflate exponentially; x_0 is the hidden space direction. The flat frame metric is $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in this order. The gamma matrix of the direction x_4 is written $\gamma^{(x_4)}$, and so on; these are the author's eight real 16×16 matrices of Chapters 4 and 5, read from the record `Revision/algebra/gammas.json`, with the Clifford relation $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}$. The matrices $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, $B = -iC\gamma^{(x_4)}$ and the chirality $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)} = \mathrm{diag}(-1_8, 1_8)$ are those of Chapter 5. The three **Pauli matrices**, written row by row, are

$$\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Each squares to the 2×2 unit matrix, two different ones anticommute, and their products are $\sigma_1\sigma_2 = i\sigma_3$, $\sigma_2\sigma_3 = i\sigma_1$, $\sigma_3\sigma_1 = i\sigma_2$ (and with the order reversed the sign changes). In the numbers of this chapter the author's constant H is 1 and the mass m of dirac16complex is 1, as in the Revision runs; energies, momenta and temperatures are then in units of m (which equals H), lengths in units of $1/H$.

14.2 The author's field and the hidden coordinate y

The metric. The author's primordial field is the diagonal metric whose entries, in the order $x_0, \dots, x_7$, are

$$\begin{aligned} g = \mathrm{diag}(&e^{2a_4}\sin^{1/3} z,\ e^{2a_4}\sin^{1/3} z,\ e^{2a_4}\sin^{1/3} z,\ -1, \\ &-e^{-2a_4}\sin^{1/3} z,\ -e^{-2a_4}\sin^{1/3} z,\ -e^{-2a_4}\sin^{1/3} z,\ \cot^2 z), \end{aligned}$$

with $z = 6Hx_0$ between 0 and $\pi/2$, the author's constant $H > 0$, and a function $a_4(x_4)$ of the time. A **scale factor** is the number by which a coordinate step must be multiplied to give a length: along x_1 a step dx_1 has the length $e^{a_4}\sin^{1/6} z\, dx_1$ (the square root of the entry). When a_4 grows with the time, the 3-space factor e^{a_4} grows (3-space **inflates**) and the extra-time factor e^{-a_4} shrinks exponentially (the extra times **deflate**). The history used in this book is $a_4 = AHx_4$ with $A = 1$ (Section 14.3).

The hidden coordinate. The entry $g_{00} = \cot^2 z$ makes the hidden direction awkward. We replace x_0 by

$$y = \frac{\ln(\sin z)}{6H}, \qquad z = 6Hx_0.$$

Line by line:

$$\frac{dy}{dx_0} = \frac{1}{6H} \cdot \frac{\cos z}{\sin z} \cdot 6H = \cot z.$$

Rule: the chain rule; the derivative of $\ln u$ is $1/u$ times the derivative of u, the derivative of $\sin z$ is $\cos z$, and $dz/dx_0 = 6H$.

$$dy^2 = \cot^2 z \, dx_0^2 = g_{00} \, dx_0^2.$$

Rule: square the previous line. So y measures length along the hidden direction, and in the coordinate y the metric entry is simply 1.

$$e^{6Hy} = \sin z, \qquad \sin^{1/3} z = e^{2Hy}.$$

Rule: multiply the definition of y by $6H$ and take the exponential of both sides; then take the power $1/3$.

When z runs from 0 to $\pi/2$, $\sin z$ runs from 0 to 1 and y from $-\infty$ to 0. The end $y \to -\infty$ ($z \to 0$) is called the **tip**; the end $y = 0$ ($z = \pi/2$), where the author's coordinate patch ends, is called the **brane**. The **warp factor** is $W = e^{Hy}$; by the last line, $\sin^{1/3} z = W^2$, and the metric takes the **warped form**

$$ds^2 = e^{2Hy} \left[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 + dx_7^2) \right] - dx_4^2 + dy^2.$$

Its scale factors (the entries of the diagonal **vielbein**, the square roots of the absolute values of the metric entries) are

$$h = (e^{a_4}W,\ e^{a_4}W,\ e^{a_4}W,\ 1,\ e^{-a_4}W,\ e^{-a_4}W,\ e^{-a_4}W,\ 1).$$

The volume. The square root of the absolute value of the determinant of a diagonal metric is the product of its scale factors:

$$\sqrt{|g|} = (e^{a_4}W)^3 \cdot 1 \cdot (e^{-a_4}W)^3 \cdot 1 = e^{3a_4} e^{-3a_4} W^6 = W^6 = e^{6Hy}.$$

Rule: multiply the eight entries of h; the powers e^{3a_4} and e^{-3a_4} cancel. In the author's x_0 chart the same product is $\sin z \cdot \cot z = \cos z$; the two differ by the factor $dy/dx_0 = \cot z$, as they must. Neither contains a_4: while 3-space inflates and the extra times deflate, the volume of the seven directions other than the time stays the same. Toward the tip the factor W^6 goes to zero: at $y = -3$ it is e^{-18}. Status: PROVED; checks geometry_hidden_coordinate, geometry_sqrt_det and geometry_warped_form of `Revision/kohn_sham/reports/ks-theory-python.json` (the first two also in the Wolfram report `Revision/kohn_sham/reports/ks-theory-wolfram.json`).

The momentum weight. A plane wave $e^{ik_1 x_1}$ along 3-space has the coordinate momentum k_1; the length of a coordinate step is $e^{a_4}W \, dx_1$, so the momentum per unit length, the one a quantum feels, is $k_1/(e^{a_4}W)$. We write

$$\kappa(y, x_4) = \frac{1}{e^{a_4} W} = e^{-Hy - a_4(x_4)}$$

for this **momentum weight**. It grows toward the tip (a given coordinate momentum costs more energy far from the brane) and, at a fixed y, it decreases as a_4 grows (3-momenta are **redshifted** while 3-space inflates).

14.3 The Kohn-Sham equation, the good sector and the stationary-slice ansatz

The Kohn-Sham equation. The field equation of dirac16complex is $\gamma^\mu D_\mu \Psi = (m + U'(S))\Psi$ (Chapter 7), where $\gamma^\mu = \gamma^{(\mu)}/h_\mu$ are the curved gammas and $D_\mu = \partial_\mu + \Omega_\mu$ is the covariant derivative with the spin connection Ω_μ (Chapter 6). In the Kohn-Sham model every orbital obeys the one-quantum equation

$$\gamma^\mu D_\mu \Psi = \big(M_{\text{eff}}(y) - i\, v_v(y)\, \gamma^{(x_4)}\big)\Psi,$$

where the **effective mass** $M_{\text{eff}} = m + \frac{15}{16}\lambda S$ and the **vector potential** $v_v = -\frac{1}{16}\lambda n$ are made by all the quanta through their scalar density S and number density n. Sections 14.26 and 14.27 derive these two formulas exactly; until then M_{eff} and v_v are any two real functions of y, written M and v for short.

The good sector (ASSUMED). We look only for orbitals that do not depend on the extra times x_5, x_6, x_7. Chapters 8 and 10 explain why: modes with enough momentum along the extra times grow without bound (the problem is ill-posed in the sense of Hadamard), and a quantum space of states with a positive norm has been built only for single good-sector momenta with frozen coefficients (Chapter 10); a positive-norm space of states for the whole field is OPEN. This restriction is an assumption of the Revision theory, not a result.

The box. The three directions of 3-space are taken as a **torus** of coordinate size ℓ: a quantum leaving the box on one side comes back on the other, so the allowed coordinate momenta are $\mathbf{k} = \Delta k\,(n_1, n_2, n_3)$ with whole numbers n_1, n_2, n_3 and $\Delta k = 2\pi/\ell$. The Revision runs use $\Delta k = 0.25$, so $\ell = 8\pi \approx 25.13$. The extra times have the coordinate volume v_t ($v_t = 1$ in the runs).

The ansatz. An **ansatz** is a guessed form of the solution with unknown parts to be determined. We try

$$\Psi = e^{i\mathbf{k}\cdot\mathbf{x}}\, W(y)^{-3}\, \chi(y, x_4),$$

a plane wave along 3-space, the factor $W^{-3} = e^{-3Hy}$, and a column χ of 16 unknown functions of y and of the time. Section 14.4 shows that χ obeys exactly, for every history $a_4(x_4)$, an equation of the form

$$i\,\partial_{x_4}\chi = h(x_4)\,\chi,$$

with a Hermitian matrix operator h that depends on the time only through $a_4(x_4)$. This is the exact evolution of one orbital in the mean field.

The stationary-slice (adiabatic) ansatz. A **slice** is one instant x_4 of the history; $a_{4,0} = a_4(x_4)$ is the value of a_4 there. At a slice we freeze h at its value $h(a_{4,0})$ and look for the stationary orbitals

$$h(a_{4,0})\,\chi = \varepsilon\,\chi, \qquad \Psi = e^{-i\varepsilon x_4}\,e^{i\mathbf{k}\cdot\mathbf{x}}\,W^{-3}\,\chi(y)\,/\sqrt{\ell^3 v_t}, \qquad \int_{-L}^{0}\chi^\dagger\chi\,dy = 1.$$

The number ε is the **level** (the energy of the orbital), and the hidden direction is cut at $y = -L$ ($L = 3$ in the runs). The Kohn-Sham state of the slice fills the lowest particle levels. This is the **adiabatic** picture: if h changes slowly enough along the history, a quantum stays in the level it occupies. The exact evolution conserves the 3-momentum, the block and the brane parity of every orbital (all defined below), so a quantum can only make transitions between levels of the same kind; Chapter 15 measures how slowly h changes. The record states this in `Revision/kohn_sham/ks-theory.json`, section adiabaticity.

The history is a PRESCRIBED BACKGROUND. The Kohn-Sham record works on the backgrounds $b_4 = 1$ of the author's metric (the second function $b_4(x_4)$ of the metric of 2026-10-14 is put equal to 1; Chapter 3). The slices are taken along the history $a_4 = AHx_4$ with $A = 1$, so $a_{4,0} = 0, 0.5, 1, 1.5, 2$ are five instants. This history is prescribed, not solved for: the field equations of a_4 (Chapter 12) allow the linear member $a_4 = AHx_4$ only with a source whose three pressures are equal and whose energy density is constant, and the Kohn-Sham states violate these conditions (their sources depend on x_0, and $p_3 \neq p_t$). The record `Revision/field_equations_a4/reports/ks-source-conditions.json` proves this with its checks ks_profiles_depend_on_x0, ks_profiles_violate_algebraic_condition and ks_history_is_a_prescribed_background (Chapter 17 explains it in full). So the Kohn-Sham gas is a **test field** on a given background: it feels the deflating field but does not act back on it.

14.4 The spin connection, the term 3H γ(x0) and the factor W^(-3)

What we need. The operator $\gamma^\mu D_\mu = \sum_\mu \gamma^\mu\partial_\mu + \sum_\mu \gamma^\mu\Omega_\mu$ contains the spin connection only through the matrix $\sum_\mu \gamma^\mu\Omega_\mu$. Chapter 6 derived the canonical spin connection in general; here we compute this one matrix for the diagonal vielbein h of Section 14.2, with a_4 an arbitrary function of x_4, so that the result holds along every history.

Step 1: the Christoffel symbols of a diagonal metric. For a metric $g = \mathrm{diag}(g_{11}, \ldots, g_{00})$ the Christoffel symbols $\Gamma^n{}_{mk} = \frac{1}{2}g^{nl}(\partial_m g_{lk} + \partial_k g_{lm} - \partial_l g_{mk})$ (sum over l) reduce to

$$\Gamma^n{}_{mk} = \frac{\delta_{mn}\,\partial_k g_{nn} + \delta_{kn}\,\partial_m g_{nn} - \delta_{mk}\,\partial_n g_{mm}}{2g_{nn}} \qquad \text{(no sum)}.$$

Rule: g^{nl} is zero unless $l = n$, where it is $1/g_{nn}$, and g_{lk} is zero unless $l = k$; the symbol δ_{mn} is 1 when $m = n$ and 0 otherwise. So a symbol is nonzero only when two of its three indices are equal.

Step 2: the connection coefficients. The vielbein postulate (Chapter 6) gives, for the diagonal vielbein with entries h_A,

$$\omega_\mu{}^A{}_B = h_A\,\partial_\mu(1/h_B)\,\delta_{AB} + h_A\,\Gamma^A{}_{\mu B}/h_B \qquad \text{(no sum)}.$$

Only $A \neq B$ will matter below, because the generators $S^{AB} = \frac{1}{4}(\gamma^{(A)}\gamma^{(B)} - \gamma^{(B)}\gamma^{(A)})$ vanish for

$A = B$. Take a direction l whose scale factor h_l depends on the coordinates, and $A \neq B$. From Step 1, with $g_{ll} = \eta_{ll} h_l^2$:

$$\Gamma^l{}_{lB} = \frac{\partial_B g_{ll}}{2 g_{ll}} = \frac{\partial_B h_l}{h_l}, \qquad \Gamma^A{}_{ll} = -\frac{\partial_A g_{ll}}{2 g_{AA}} = -\eta_{ll}\eta_{AA}\frac{h_l\,\partial_A h_l}{h_A^2},$$

and every other $\Gamma^A{}_{lB}$ with $A \neq B$ is zero. Rule: Step 1 with two equal indices; $\partial(h^2) = 2h\,\partial h$; and $1/\eta_{AA} = \eta_{AA}$ because $\eta_{AA} = \pm 1$. Inserting into the formula for ω:

$$\omega_l{}^l{}_B = \frac{\partial_B h_l}{h_B}, \qquad \omega_l{}^A{}_l = -\eta_{ll}\eta_{AA}\frac{\partial_A h_l}{h_A}.$$

Step 3: the matrix Ω_l. With $\Omega_\mu = \frac{1}{2}\sum_{A,B}\eta_{AA}\,\omega_\mu{}^A{}_B\,S^{AB}$ (Chapter 6):

$$\Omega_l = \tfrac{1}{2}\sum_{B\neq l}\eta_{ll}\frac{\partial_B h_l}{h_B}S^{lB} + \tfrac{1}{2}\sum_{A\neq l}\eta_{AA}\Big(-\eta_{ll}\eta_{AA}\frac{\partial_A h_l}{h_A}\Big)S^{Al} = \eta_{ll}\sum_{B\neq l}\frac{\partial_B h_l}{h_B}\,S^{lB}.$$

Rule: insert Step 2; in the second sum $\eta_{AA}\eta_{AA} = 1$, rename A as B and use $S^{Bl} = -S^{lB}$, so that the two sums are equal and add up. Since $\gamma^{(l)}\gamma^{(B)} = -\gamma^{(B)}\gamma^{(l)}$ for $B \neq l$, $S^{lB} = \frac{1}{2}\gamma^{(l)}\gamma^{(B)}$.

Step 4: one direction's contribution. Multiply by the curved gamma $\gamma^l = \gamma^{(l)}/h_l$:

$$\frac{\gamma^{(l)}}{h_l}\,\Omega_l = \frac{\eta_{ll}}{2h_l}\sum_{B\neq l}\frac{\partial_B h_l}{h_B}\,\gamma^{(l)}\gamma^{(l)}\gamma^{(B)} = \frac{1}{2}\sum_{B\neq l}\frac{\partial_B \ln h_l}{h_B}\,\gamma^{(B)}.$$

Rule: $\gamma^{(l)}\gamma^{(l)} = \eta_{ll}$ (the Clifford relation), $\eta_{ll}\eta_{ll} = 1$, and $\partial_B h_l/h_l = \partial_B \ln h_l$.

Step 5: the author's field. Here the scale factors depend only on x_4 (through a_4) and on y, and $h_{x_4} = h_y = 1$. For a 3-space direction, $\ln h_l = a_4 + Hy$, so $\partial_{x_4}\ln h_l = a_4'$ (the derivative da_4/dx_4) and $\partial_y \ln h_l = H$; Step 4 gives $\frac{1}{2}a_4'\gamma^{(x_4)} + \frac{1}{2}H\gamma^{(x_0)}$. For an extra time, $\ln h_l = -a_4 + Hy$, which gives $-\frac{1}{2}a_4'\gamma^{(x_4)} + \frac{1}{2}H\gamma^{(x_0)}$. For x_4 and y themselves the scale factor is the constant 1, so $\Omega_{x_4} = \Omega_y = 0$. (In the vielbein language the frame direction of y is that of x_0, so the gamma that goes with y is $\gamma^{(x_0)}$.) Adding the eight contributions:

$$\sum_\mu \gamma^\mu\Omega_\mu = 3\cdot\tfrac{1}{2}a_4'\gamma^{(x_4)} - 3\cdot\tfrac{1}{2}a_4'\gamma^{(x_4)} + 6\cdot\tfrac{1}{2}H\gamma^{(x_0)} = 3H\gamma^{(x_0)}.$$

Rule: three inflating directions, three deflating directions, six warped directions. **The time-direction pieces of the three inflating directions, $+\frac{3}{2}a_4'\gamma^{(x_4)}$, and of the three deflating extra times, $-\frac{3}{2}a_4'\gamma^{(x_4)}$, cancel exactly**, for every history; what survives comes from the warp. Status: PROVED; checks `spin_connection_compatibility` (the curved gammas are covariantly constant, the proof that this is the right connection), `spin_connection_slash_3H`, `spin_connection_time_terms_cancel` and `spin_connection_time_term_value` of `Revision/kohn_sham/reports/ks-theory-python.json` (the first three also in the Wolfram report). Notebook 14a recomputes all of it with sympy and draws the eight contributions (its figure 3). That the surviving term $3H\gamma^{(x_0)}$ belongs to the diagonal vielbein, and can be removed by a change of frame or of the field, is the scope correction of Chapter 8.

Step 6: the factor W^{-3} removes it. Insert the ansatz $\Psi = e^{i\mathbf{k}\cdot\mathbf{x}}W^{-3}\chi(y,x_4)$ into $\gamma^\mu D_\mu\Psi = \sum_\mu \gamma^\mu\partial_\mu\Psi + 3H\gamma^{(x_0)}\Psi$, term by term.

$$\gamma^{(x_0)}\partial_y(e^{-3Hy}\chi) = e^{-3Hy}(\gamma^{(x_0)}\partial_y\chi - 3H\gamma^{(x_0)}\chi).$$

Rule: the product rule and $\partial_y e^{-3Hy} = -3He^{-3Hy}$. The term $-3H\gamma^{(x_0)}\chi$ cancels the $+3H\gamma^{(x_0)}\chi$ of the spin connection.

$$\frac{\gamma^{(x_j)}}{e^{a_4}W}\,\partial_j e^{i\mathbf{k}\cdot\mathbf{x}} = i\kappa k_j\gamma^{(x_j)}\,e^{i\mathbf{k}\cdot\mathbf{x}}, \qquad j=1,2,3.$$

Rule: $\partial_j e^{i\mathbf{k}\cdot\mathbf{x}} = ik_j e^{i\mathbf{k}\cdot\mathbf{x}}$, and $1/(e^{a_4}W) = \kappa$ (Section 14.2). The time term is $\gamma^{(x_4)}\partial_{x_4}\chi$ (the scale factor of x_4 is 1), and the extra times give nothing (good sector). Dividing by the common factor $e^{i\mathbf{k}\cdot\mathbf{x}}e^{-3Hy}$, the Kohn-Sham equation becomes

$$\gamma^{(x_0)}\partial_y\chi + \gamma^{(x_4)}\partial_{x_4}\chi + i\kappa k_j\gamma^{(x_j)}\chi = (M - iv\gamma^{(x_4)})\chi \qquad \text{(sum over } j=1,2,3).$$

The spin connection has disappeared, exactly and for every history. Without the factor W^{-3} the term $3H\gamma^{(x_0)}\chi$ would survive. Status: PROVED; checks ansatz_removes_spin_connection and ansatz_without_W3_term_survives.

Step 7: the Hamiltonian form. Solve the last equation for the time derivative, line by line.

$$\gamma^{(x_4)}\partial_{x_4}\chi = M\chi - iv\gamma^{(x_4)}\chi - \gamma^{(x_0)}\partial_y\chi - i\kappa k_j\gamma^{(x_j)}\chi.$$

Rule: move every term except the time term to the right side.

$$\partial_{x_4}\chi = -M\gamma^{(x_4)}\chi - iv\chi + \gamma^{(x_4)}\gamma^{(x_0)}\partial_y\chi + i\kappa k_j\gamma^{(x_4)}\gamma^{(x_j)}\chi.$$

Rule: multiply from the left by $-\gamma^{(x_4)}$; on the left $-\gamma^{(x_4)}\gamma^{(x_4)} = -\eta_{44} = 1$; on the right $-\gamma^{(x_4)}(-iv\gamma^{(x_4)}) = iv\gamma^{(x_4)}\gamma^{(x_4)} = -iv$.

$$i\partial_{x_4}\chi = h\chi, \qquad h = i\gamma^{(x_4)}\gamma^{(x_0)}\partial_y - \kappa k_j\gamma^{(x_4)}\gamma^{(x_j)} + M(-i\gamma^{(x_4)}) + v.$$

Rule: multiply by i and use $i\cdot i = -1$.

Why h is Hermitian. A matrix X is **Hermitian** when it equals its conjugate transpose, $X^\dagger = X$; an operator is Hermitian (self-adjoint) when $\int \varphi^\dagger(h\chi)\,dy = \int (h\varphi)^\dagger\chi\,dy$ for all allowed columns φ, χ; then its levels are real and orbitals of different levels are orthogonal. Each piece of h qualifies. (i) $\gamma^{(x_4)}\gamma^{(x_0)}$ is real and symmetric: $(\gamma^{(x_4)}\gamma^{(x_0)})^T = (\gamma^{(x_0)})^T(\gamma^{(x_4)})^T = \gamma^{(x_0)}(-\gamma^{(x_4)}) = \gamma^{(x_4)}\gamma^{(x_0)}$, by the symmetry pattern $(\gamma^a)^T = \eta_{aa}\gamma^a$ of Chapter 5 and the anticommutation of two different gammas; so $i\gamma^{(x_4)}\gamma^{(x_0)}$ is anti-Hermitian, ∂_y is anti-Hermitian too (integration by parts, when the boundary terms vanish, Section 14.12), and the product of the two, which commute, is Hermitian. (ii) $\gamma^{(x_4)}\gamma^{(x_j)}$ for a 3-space direction is real and symmetric by the same rule, hence Hermitian. (iii) $(-i\gamma^{(x_4)})^\dagger = i(\gamma^{(x_4)})^T = -i\gamma^{(x_4)}$, and $(-i\gamma^{(x_4)})^2 = -\eta_{44} = 1$. (iv) v is a real number times the unit matrix. Status: PROVED; check hamiltonian_16_hermitian.

The flat measure. The Hilbert norm of the problem is $\int \chi^\dagger \chi \, dy$: since $\sqrt{|g|}\,\Psi^\dagger\Psi = e^{6Hy}e^{-6Hy}\chi^\dagger\chi$, it is the integral of $\sqrt{|g|}\,\Psi^\dagger\Psi$ over the hidden direction, with the plain measure dy. It does not contain a_4.

14.5 Eight blocks of two components

Only $|\mathbf{k}|$ matters. Section 14.6 shows that a rotation of 3-space does not change the levels. So we may turn $\mathbf{k}$ into the direction x_1, $\mathbf{k} = (k, 0, 0)$. Then h contains only the three gammas $\gamma^{(x_0)}$, $\gamma^{(x_1)}$, $\gamma^{(x_4)}$, and it is convenient to name three products:

$$A_0 = \gamma^{(x_0)}, \qquad A_1 = \gamma^{(x_0)}\gamma^{(x_1)}, \qquad A_4 = \gamma^{(x_0)}\gamma^{(x_4)}.$$

Two rules from Chapter 5. Let P be a product of p different gammas. (Rule 1) Moving a gamma γ^c from the left of P to its right gives the sign $(-1)^{p-1}$ if γ^c is one of the factors and $(-1)^p$ if it is not, because passing each different factor costs a sign. (Rule 2) $PP = (-1)^{p(p-1)/2}$ times the product of the η of its factors.

Three labels. Define

$$J = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_4)}, \qquad K_1 = \gamma^{(x_2)}\gamma^{(x_3)}, \qquad K_2 = \gamma^{(x_5)}\gamma^{(x_6)}.$$

By Rule 1, J ($p = 3$) commutes with its own three factors and anticommutes with the five other gammas; K_1 ($p = 2$) anticommutes with $\gamma^{(x_2)}$, $\gamma^{(x_3)}$ and commutes with the six others; K_2 anticommutes with $\gamma^{(x_5)}$, $\gamma^{(x_6)}$ and commutes with the others. A product of gammas commutes with J when it contains an even number of factors that anticommute with J. Therefore all three commute with A_0, A_1, A_4 (built from $\gamma^{(x_0)}, \gamma^{(x_1)}, \gamma^{(x_4)}$), with C (for J the two anticommuting factors $\gamma^{(x_2)}, \gamma^{(x_3)}$ give two signs, which cancel), with $B = -iC\gamma^{(x_4)}$, and with each other. By Rule 2:

$$J^2 = (-1)^3\,\eta_{00}\eta_{11}\eta_{44} = (-1)(1)(1)(-1) = +1,$$

$$K_1^2 = (-1)^1\eta_{22}\eta_{33} = -1, \qquad K_2^2 = (-1)^1\eta_{55}\eta_{66} = -1.$$

So J has the eigenvalues $j = \pm 1$, and K_1, K_2 have the eigenvalues is_2, is_3 with $s_2, s_3 = \pm 1$ (if $Xu = cu$ for a column $u \neq 0$ and $X^2 = 1$, then $u = X^2u = c^2u$, so $c^2 = 1$; with $X^2 = -1$ the same step gives $c^2 = -1$). Status: PROVED; check blocks_commuting_set.

The projectors. A **projector** is a matrix with $P^2 = P$; its eigenvalues are 0 or 1 (from $\lambda^2 = \lambda$), so its trace equals the number of eigenvalues 1, its **rank** (the dimension of the space it projects onto). For each of the eight label triples put

$$P(j, s_2, s_3) = \frac{1 + jJ}{2} \cdot \frac{1 - is_2K_1}{2} \cdot \frac{1 - is_3K_2}{2}.$$

Each factor is $\frac{1}{2}(1 + X)$ with $X^2 = 1$ ($(jJ)^2 = 1$, $(-is_2K_1)^2 = -K_1^2 = 1$), hence a projector; the three factors commute, so their product is a projector; it keeps exactly the columns with $J = j$,

$K_1 = is_2$, $K_2 = is_3$. Multiplied out, P is $\frac{1}{8}$ times the unit matrix plus seven products of different gammas (J, K_1, K_2, JK_1, JK_2, K_1K_2, JK_1K_2, each with a number in front), and every product of one or more different gammas has trace 0 (Rule 5 of Chapter 5). So

$$\operatorname{tr} P(j, s_2, s_3) = \tfrac{1}{8} \operatorname{tr} 1 = \tfrac{16}{8} = 2.$$

The eight projectors have rank 2 and add up to the unit matrix: **the 16 components split into eight blocks of two**. Status: PROVED; check blocks_projectors.

A basis in each block. Inside a block, $A_0 = \gamma^{(x_0)}$ squares to 1 and A_1 anticommutes with it ($A_1 = \gamma^{(x_0)}\gamma^{(x_1)}$ contains $\gamma^{(x_0)}$ once and $p = 2$, so by Rule 1 the sign is -1), so A_0 has the eigenvalue $+1$ once and -1 once in the block. The record chooses

$$v_+ = 8\, P(j, s_2, s_3)\, \tfrac{1}{2}(1 + \gamma^{(x_0)})\, e_c, \qquad v_- = A_1 v_+,$$

where e_c is the first unit column (counting c from 0) that gives a nonzero v_+. Then $A_0 v_+ = v_+$ and $A_0 v_- = A_0 A_1 v_+ = -A_1 A_0 v_+ = -v_-$. The 16 columns v_+, v_- of the eight blocks, in the order $(j, s_2, s_3) = (1,1,1), (1,1,-1), (1,-1,1), (1,-1,-1), (-1,1,1), (-1,1,-1), (-1,-1,1), (-1,-1,-1)$, form the matrix U; every entry of U is 0, ± 1 or $\pm i$, every column has length $\sqrt{8} = 2\sqrt{2}$, and $V = U/(2\sqrt{2})$ is **unitary** ($V^\dagger V = 1$: a change of basis that keeps lengths). The seed columns are $c = 4, 0, 0, 4, 0, 4, 4, 0$. The record stores U entry by entry in `Revision/kohn_sham/ks-theory.jso n` (blockBasis), and the Rust solver uses it. Status: PROVED; check blocks_basis_unitary.

The block forms. In the basis (v_+, v_-) of one block every matrix that commutes with J, K_1, K_2 becomes a 2×2 matrix. Column k of the 2×2 matrix is the image of the k-th basis vector. Line by line:

$$A_0 \to \sigma_3.$$

Rule: $A_0 v_+ = v_+$ and $A_0 v_- = -v_-$.

$$A_1 \to \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -i\sigma_2.$$

Rule: $A_1 v_+ = v_-$ (first column $(0,1)$), and $A_1 v_- = A_1 A_1 v_+ = -v_+$ (second column $(-1,0)$), because $A_1^2 = -1$ by Rule 2.

$$A_4 \to j\sigma_1.$$

Rule: A_4 anticommutes with A_0 (Rule 1), so it has no diagonal entries; and $A_0 A_1 A_4 = \gamma^{(x_0)}\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_0)}\gamma^{(x_4)} = \gamma^{(x_1)}\gamma^{(x_0)}\gamma^{(x_4)} = -J$, which is the number $-j$ in the block; with $\sigma_3(-i\sigma_2) = -i\sigma_3\sigma_2 = -\sigma_1$ the condition $-\sigma_1 A_4 = -j$ gives $A_4 = j\sigma_1$.

$$C = A_1 K_1 \to (-i\sigma_2)(is_2) = s_2\sigma_2, \qquad B = -iJK_1 \to -i\, j\, (is_2) = j s_2.$$

Rule: $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)} = A_1K_1$; and $JK_1 = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_4)}\gamma^{(x_2)}\gamma^{(x_3)} = C\gamma^{(x_4)}$ after moving $\gamma^{(x_4)}$ past two gammas (two signs), so $B = -iC\gamma^{(x_4)} = -iJK_1$. So B is a number in every block, $+1$ or -1: the **Krein sign** js_2 of the block.

$$\gamma^{(x_4)} = A_0A_4 \to ij\sigma_2, \qquad \gamma^{(x_1)} = A_0A_1 \to -\sigma_1, \qquad \gamma^{(x_4)}\gamma^{(x_1)} \to (ij\sigma_2)(-\sigma_1) = -j\sigma_3.$$

Rule: $A_0A_0 = 1$; the Pauli products $\sigma_3\sigma_1 = i\sigma_2$, $\sigma_3\sigma_2 = -i\sigma_1$, $\sigma_2\sigma_1 = -i\sigma_3$.

The record lists eleven such forms (for $\gamma^{(x_0)}$, A_1, A_4, B, C, $BC = -i\gamma^{(x_4)} \to j\sigma_2$, $\gamma^{(x_4)}\gamma^{(x_1)}$, $B\gamma^{(x_0)} \to js_2\sigma_3$, J, K_1, K_2); Notebook 14a checks all of them. Not every matrix is block diagonal: $\gamma^{(x_2)}$, $\gamma^{(x_5)}$ and $\gamma^{(x_0)}\gamma^{(x_2)}$ anticommute with J and connect different blocks, which is why $\mathbf{k}$ was turned along x_1. Status: PROVED; checks blocks_forms and blocks_not_everything_block_diagonal.

The block Hamiltonian. Put the forms into h of Section 14.4 with $\mathbf{k} = (k, 0, 0)$: $i\gamma^{(x_4)}\gamma^{(x_0)} = -iA_4 \to -ij\sigma_1$; $-\kappa k\gamma^{(x_4)}\gamma^{(x_1)} \to j\kappa k\sigma_3$; $M(-i\gamma^{(x_4)}) \to jM\sigma_2$. So in every block of type j

$$h_j = j\left[-i\sigma_1\frac{d}{dy} + M\sigma_2 + \kappa k\,\sigma_3\right] + v.$$

It depends on j but not on s_2, s_3: there are only **two types of block**, four blocks of each type. Status: PROVED; check block_hamiltonian.

The first-order form. The eigenvalue equation $h_j\chi = \varepsilon\chi$ is a pair of first-order differential equations. Line by line:

$$-ij\sigma_1\chi' = (\varepsilon - v)\chi - j(M\sigma_2 + \kappa k\sigma_3)\chi.$$

Rule: move the terms without derivative to the right; χ' is $d\chi/dy$.

$$\chi' = ij(\varepsilon - v)\sigma_1\chi - i\,(M\sigma_1\sigma_2 + \kappa k\,\sigma_1\sigma_3)\chi.$$

Rule: multiply from the left by $ij\sigma_1$; $(ij\sigma_1)(-ij\sigma_1) = j^2\sigma_1^2 = 1$ and $j \cdot j = 1$.

$$\chi' = N\chi, \qquad N = M\sigma_3 - \kappa k\,\sigma_2 + ij(\varepsilon - v)\,\sigma_1.$$

Rule: $\sigma_1\sigma_2 = i\sigma_3$, $\sigma_1\sigma_3 = -i\sigma_2$, so $-i(iM\sigma_3 - i\kappa k\sigma_2) = M\sigma_3 - \kappa k\sigma_2$. Status: PROVED; check block_ode_equivalent.

The two types mirror each other. Write $h_j - v = jD(M,k)$ with $D(M,k) = -i\sigma_1\,d/dy + M\sigma_2 + \kappa k\sigma_3$. Then:

- $h_{-1} - v = -(h_{+1} - v)$. For $v = 0$ the levels of the blocks $j = -1$ are the negatives of those of $j = +1$.
- $\sigma_3 D(M,k)\sigma_3 = i\sigma_1\,d/dy - M\sigma_2 + \kappa k\sigma_3 = -D(M,-k)$, because $\sigma_3\sigma_1\sigma_3 = -\sigma_1$ and $\sigma_3\sigma_2\sigma_3 = -\sigma_2$. Hence $\sigma_3 h_j(k)\sigma_3 = h_{-j}(-k)$: the orbital $\sigma_3\chi$ of $(-j,-k)$ has the same level as the orbital χ of (j,k).

Status: PROVED; check block_type_relation.

14.6 The chirality matrix Γ on the blocks, and the rotations of 3-space

Γ pairs the two types. The chirality Γ is a product of all eight gammas, so it anticommutes with every gamma (Rule 1 with $p = 8$: a factor passes seven others), with every product of an odd number of gammas such as J, and commutes with every even product such as K_1, K_2. If $Jv = jv$, then $J\Gamma v = -\Gamma Jv = -j\,\Gamma v$, while $K_1\Gamma v = \Gamma K_1 v$: so Γ maps the block (j, s_2, s_3) onto the block $(-j, s_2, s_3)$. In the block basis its 2×2 entry between the paired blocks is $s_2\sigma_2$ (computed in Notebook 14a, In [16]; its figure 4 shows where these entries sit). Status: PROVED; check blocks_relation_to_Gamma.

Γ reverses the mass. Line by line:

$$\sigma_2 D(M,k)\sigma_2 = i\sigma_1\frac{d}{dy} + M\sigma_2 - \kappa k\sigma_3 = -D(-M,k).$$

Rule: $\sigma_2\sigma_1\sigma_2 = -\sigma_1$, $\sigma_2\sigma_2\sigma_2 = \sigma_2$, $\sigma_2\sigma_3\sigma_2 = -\sigma_3$.

$$\sigma_2 h_j(M,k)\sigma_2 = j\big(-D(-M,k)\big) + v = h_{-j}(-M,k).$$

Rule: $h_j = jD + v$ and $\sigma_2\sigma_2 = 1$. So if χ solves the block equation of type j with the mass M, then $\sigma_2\chi$ solves the block equation of type $-j$ with the mass $-M$, at the same k, the same potential v and the same level ε. Because $\sigma_2(\chi_1, \chi_2) = (-i\chi_2, i\chi_1)$, the map also exchanges the two brane conditions $\chi_2(0) = 0$ and $\chi_1(0) = 0$ of Section 14.12. Status: PROVED; check block_Gamma_map.

The same fact is visible in 16 components without blocks. Γ anticommutes with every γ^μ and commutes with every Ω_μ (a sum of products of two gammas), so $\gamma^\mu D_\mu(\Gamma\Psi) = -\Gamma\gamma^\mu D_\mu\Psi$. If $\gamma^\mu D_\mu\Psi = (M - iv\gamma^{(x_4)})\Psi$, then

$$\gamma^\mu D_\mu(\Gamma\Psi) = -\Gamma\big(M - iv\gamma^{(x_4)}\big)\Psi = \big(-M - iv\gamma^{(x_4)}\big)\Gamma\Psi,$$

where the last step uses $\Gamma\gamma^{(x_4)} = -\gamma^{(x_4)}\Gamma$. Here M and v are regarded as given functions of y. In the full theory they are made by the quanta themselves, and how the parameters m, λ and the boundary conditions transform is the content of the pairing theorems: T1 for the field equations, which Chapter 18 proves (the chirality MATRIX Γ carries the solutions of the Lagrangian with (m, λ) to those with $(-m, -\lambda)$), and T3 for the Kohn-Sham states, which Chapter 19 proves. On the blocks, σ_2 also exchanges the tip condition $b(-L) = 0$ with $a(-L) = 0$ (Section 14.12). The maps of this theory are matrices. The author's gammas are real, so for a real field plain complex conjugation changes nothing and is not a charge conjugation; the charge-conjugation matrices of the theory are $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$ (Chapter 5). The map used here is the chirality matrix Γ itself, the real matrix map of the theorem T1, which reverses the mass; on the blocks it acts as $s_2\sigma_2$ from the block (j, s_2, s_3) to the block $(-j, s_2, s_3)$.

What this does and does not say (honesty). The map Γ is an exact map between two sets of solutions: every solution with the mass M has a partner with the mass $-M$. It says nothing about whether a partner is ever realised, and nothing about a process that would create universes, in pairs or otherwise. Chapters 18 to 20 give the complete statements of the pairing theorems T1, T2, Q and T3 and the precise list of what they do not establish.

Rotations of 3-space. A rotation by the angle θ in the (x_1, x_2) plane acts on spinors with the matrix $R = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}\, G$, $G = \gamma^{(x_1)}\gamma^{(x_2)}$. Line by line, with $c = \cos\frac{\theta}{2}$, $s = \sin\frac{\theta}{2}$:

$$G^2 = -1, \qquad (c + sG)(c - sG) = c^2 - s^2 G^2 = c^2 + s^2 = 1.$$

Rule: Rule 2 with $p = 2$ and $\eta_{11}\eta_{22} = 1$; so $R^{-1} = c - sG$.

$$\gamma^{(x_1)} G = \gamma^{(x_2)}, \qquad G\gamma^{(x_1)} = -\gamma^{(x_2)}, \qquad R\gamma^{(x_1)} = \gamma^{(x_1)} R^{-1}.$$

Rule: $\gamma^{(x_1)}\gamma^{(x_1)} = 1$ and $\gamma^{(x_2)}\gamma^{(x_1)} = -\gamma^{(x_1)}\gamma^{(x_2)}$; so G anticommutes with $\gamma^{(x_1)}$, which turns $c + sG$ into $c - sG$ when $\gamma^{(x_1)}$ passes it.

$$R\gamma^{(x_1)} R^{-1} = \gamma^{(x_1)}(c - sG)^2 = \gamma^{(x_1)}\big(\cos\theta - \sin\theta\, G\big) = \cos\theta\, \gamma^{(x_1)} - \sin\theta\, \gamma^{(x_2)}.$$

Rule: $(c - sG)^2 = c^2 - s^2 - 2csG$ (with $G^2 = -1$), the double-angle formulas $c^2 - s^2 = \cos\theta$, $2cs = \sin\theta$, and $\gamma^{(x_1)}G = \gamma^{(x_2)}$. So R turns the momentum term $k\gamma^{(x_1)}$ into the same momentum in a rotated direction. And R commutes with $\gamma^{(x_0)}$, $\gamma^{(x_4)}$, B and C (each contains $\gamma^{(x_1)}$ and $\gamma^{(x_2)}$ both or neither, so the signs cancel), so it changes nothing else in h. Hence the levels of the whole 16-component problem depend on $|\mathbf{k}|$ only, and $\mathbf{k} = (k, 0, 0)$ is no loss of generality. Status: PROVED; check rotation_invariance. (The labels j themselves refer to the direction of $\mathbf{k}$: a rotation by π, $R = G$, anticommutes with J and turns the blocks (j, k) into $(-j, -k)$, in agreement with the σ_3 relation of Section 14.5.)

14.7 The exact rescaling identity between the slices

Where the slice enters. Look at the block Hamiltonian h_j of Section 14.5. The slice $a_{4,0}$ appears only in the momentum term, through

$$\kappa k = e^{-Hy - a_{4,0}}\, k = e^{-Hy}\, \big(k\, e^{-a_{4,0}}\big).$$

Rule: $e^{u+w} = e^u e^w$. Line by line, what follows:

- At the slice $a_{4,0}$ the allowed momenta are $\Delta k\, \mathbf{n}$ (Section 14.3). By the line above they act exactly like the momenta $\Delta k\, e^{-a_{4,0}}\, \mathbf{n}$ at the slice 0. So every level and every orbital of the slice $a_{4,0}$ equals a level and orbital of the slice 0 with the lattice spacing $\Delta k\, e^{-a_{4,0}}$.
- The lattice spacing $\Delta k\, e^{-a_{4,0}}$ belongs to the box size $\ell e^{a_{4,0}}$ (because $\Delta k = 2\pi/\ell$).
- Densities carry the factor $P = e^{-6Hy}/(\ell^3 v_t)$ (the orbital is spread over the coordinate volume $\ell^3 v_t$ of 3-space and the extra times, and $e^{-6Hy} = 1/\sqrt{|g|}$ turns a density per unit y into a density per unit proper volume). If the partner problem at the slice 0 has the box $\ell e^{a_{4,0}}$ and the extra-time volume $v_t e^{-3a_{4,0}}$, its factor is $e^{-6Hy}/(\ell^3 e^{3a_{4,0}} v_t e^{-3a_{4,0}}) = P$: the same proper densities.
- The interaction enters only through the proper densities (Section 14.26), so self-consistency, the energy and every profile agree as well.

Hence, exactly (for the free gas, with the interaction, at every temperature):

$$\mathrm{KS}(a_{4,0};\ \Delta k,\ v_t,\ \lambda) = \mathrm{KS}(0;\ \Delta k\, e^{-a_{4,0}},\ v_t\, e^{-3a_{4,0}},\ \lambda).$$

In words: a later slice of the deflating history is the slice 0 with all 3-momenta redshifted by $e^{-a_{4,0}}$ and the same proper box. The proper size of 3-space has grown by $e^{a_{4,0}}$, the proper volume of the extra times has shrunk by $e^{-3a_{4,0}}$, and the proper 7-volume has not changed. Equivalently, keeping the extra-time volume v_t, the partner has the coupling $\lambda e^{3a_{4,0}}$ and every proper density multiplied by $e^{-3a_{4,0}}$ (the coupling enters only as λ times a density). Status: PROVED; check rescaling_identity. The Rust solver solved each partner problem independently and found the same levels to 1.5×10^{-13} and the same energies and profiles to 3.3×10^{-13} (checks rescaling_identity_between_slices and rescaling_identity_energy_profiles of `Revision/kohn_sham/reports/ks-rust-solver.json`; the partner numbers are in `Revision/kohn_sham/results/rescaling/rescaling.csv`). For the slice $a_{4,0} = 0.5$ the partner has $\Delta k\, e^{-0.5} = 0.151632664928$ and $v_t\, e^{-1.5} = 0.223130160148$ (Notebook 14a, Out [18]).

14.8 Example: Notebook 14a, the block reduction and the rescaling identity

Notebook 14a repeats Sections 14.2 to 14.7 with exact computer algebra. It reads the author's gammas from the Revision record, checks the Clifford relation and the matrices C, B, Γ, verifies the hidden coordinate and the determinant, computes the canonical spin connection of the warped metric with a_4 an arbitrary function of the time and shows that $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ with the time-direction pieces cancelling, verifies that W^{-3} removes it, builds the Hamiltonian, the labels J, K_1, K_2, the projectors, the block basis (compared entry by entry with the record) and the block forms, checks the two block types, the matrix Γ on the blocks, the rotations and the rescaling identity, and compares the partner spacings and volumes with all 60 rows of the Rust record. It draws five figures. It runs in about 30 seconds (two cells do exact algebra with 16×16 matrices of symbols and take about 10 seconds each), and its last line is ALL 28 CHECKS PASSED (notebook 14a).

14.9 How to run Notebook 14a

Step 1. What this notebook does and what it needs.

Notebook 14a (The reduction of the Kohn-Sham equation to eight 2x2 blocks and the rescaling identity) is the file `Revision/textbook/notebooks/14a_block_reduction.ipynb` of the repository Dirac_claude. It rebuilds from the author's real 16 x 16 gamma matrices every exact step that turns the 16-component Kohn-Sham equation of dirac16complex in the deflating primordial field into eight independent 2 x 2 equations in the hidden coordinate y: the warped form of the metric, the spin-connection term 3H gamma^(x0) (the time-direction pieces of the three inflating and the three deflating directions cancel), the factor W^(-3) that removes it, the three commuting labels J, K1, K2 and the block basis V, the block forms of every matrix of the equation, the chirality matrix Gamma that pairs the blocks of opposite type and reverses the mass, and the exact rescaling identity between the slices of the deflating history; it checks every step against the Revision records and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were

built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 14a_block_reduction.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 14a_block_reduction.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 14a_block_reduction.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/14a.captions.json`
- `Revision/textbook/figures/14a_1_hidden_coordinate.png`
- `Revision/textbook/figures/14a_2_inflation_deflation.png`
- `Revision/textbook/figures/14a_3_spin_connection_terms.png`
- `Revision/textbook/figures/14a_4_block_structure.png`
- `Revision/textbook/figures/14a_5_rescaling.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files of this notebook exist
ALL 28 CHECKS PASSED (notebook 14a)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/14a_block_reduction.ipynb` inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 14a_block_reduction.ipynb
python -m nbconvert --execute --inplace 14a_block_reduction.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- the cells of sections 7 and 8 seem to hang: they do exact algebra with 16 x 16 matrices of symbols and take about 10 seconds each on a laptop; wait until the PASS lines appear.

14.10 Notebook 14a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/14a_block_reduction.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Kohn-Sham equation of dirac16complex in the author's primordial field is an
equation for 16 complex functions of the eight coordinates $x_0, \dots, x_7$. This
```

```
notebook turns it, step by exact step, into eight independent equations for TWO
functions of ONE coordinate, the hidden coordinate $y$. Every step is computed here
with exact algebra (sympy) from the author's gamma matrices and compared with the
Revision record, which made the same computation twice (in sympy and in
WolframScript). The steps are:

- the hidden coordinate $y$ and the warped form of the author's metric
  (figures 1 and 2);
- the spin connection: the only term that survives in the Dirac operator is
  $3H\gamma^{(x_0)}$; the time-direction pieces of the three inflating and of the
  three deflating directions cancel (figure 3);
- the factor $W^{-3} = e^{-3Hy}$ in the orbital removes this term exactly;
- the equation becomes $i\,\partial_{x_4}\chi = h\,\chi$ with a Hermitian $h$;
- three matrices $J, K_1, K_2$ commute with $h$ and split the 16 components into
  eight blocks of two; the block basis $V$ is built and compared entry by entry
  with the Revision record (figure 4);
- in every block the equation is $h_j = j[-i\sigma_1\,d/dy + M\sigma_2 +
  \kappa k\sigma_3] + v$, and only two types of blocks exist ($j = \pm 1$);
- the chirality MATRIX $\Gamma$ maps a block of type $j$ with mass $M$ onto a
  block of type $-j$ with mass $-M$;
- the slice $a_{4,0}$ of the deflating history enters only through
  $k\,e^{-a_{4,0}}$: the exact rescaling identity (figure 5).

The notebook prints a line starting with PASS for every check (28 in all) and
saves five figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 14a (The reduction of the Kohn-Sham equation to eight 2x2 blocks and the
rescaling identity) is the file ‘Revision/textbook/notebooks/14a_block_reduction.ipynb‘
of the repository Dirac_claude. It rebuilds from the author's real 16 x 16 gamma matrices
every exact step that turns the 16-component Kohn-Sham equation of dirac16complex in the
deflating primordial field into eight independent 2 x 2 equations in the hidden
coordinate y: the warped form of the metric, the spin-connection term 3H gamma^(x0) (the
time-direction pieces of the three inflating and the three deflating directions cancel),
the factor W^(-3) that removes it, the three commuting labels J, K1, K2 and the block
basis V, the block forms of every matrix of the equation, the chirality matrix Gamma that
pairs the blocks of opposite type and reverses the mass, and the exact rescaling identity
between the slices of the deflating history; it checks every step against the Revision
records and draws five teaching figures. It needs a computer with Windows 11, macOS or
Linux, an internet connection for the installation, the program Git, and Python 3.12 or
newer (the notebooks were built with Python 3.14.5) with these packages at exactly these
versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10,
nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook
itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program
that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**
```

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on

the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 14a_block_reduction.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 14a_block_reduction.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 14a_block_reduction.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/14a.captions.json‘
- ‘Revision/textbook/figures/14a_1_hidden_coordinate.png‘
- ‘Revision/textbook/figures/14a_2_inflation_deflation.png‘
- ‘Revision/textbook/figures/14a_3_spin_connection_terms.png‘
- ‘Revision/textbook/figures/14a_4_block_structure.png‘

- ‘Revision/textbook/figures/14a_5_rescaling.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all five figure files of this notebook exist
ALL 28 CHECKS PASSED (notebook 14a)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/14a_block_reduction.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 14a_block_reduction.ipynb
  python -m nbconvert --execute --inplace 14a_block_reduction.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues

normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- the cells of sections 7 and 8 seem to hang: they do exact algebra with 16 x 16 matrices of symbols and take about 10 seconds each on a laptop; wait until the PASS lines appear.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 14a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 14a (The reduction of the Kohn-Sham equation to eight 2x2 blocks and the
# rescaling identity) is the file Revision/textbook/notebooks/14a_block_reduction.ipynb
# of the repository Dirac_claude. It rebuilds from the author's real 16 x 16 gamma
# matrices every exact step that turns the 16-component Kohn-Sham equation of
# dirac16complex in the deflating primordial field into eight independent 2 x 2 equations
# in the hidden coordinate y: the warped form of the metric, the spin-connection term 3H
# gamma^(x0) (the time-direction pieces of the three inflating and the three deflating
# directions cancel), the factor W^(-3) that removes it, the three commuting labels J,
# K1, K2 and the block basis V, the block forms of every matrix of the equation, the
# chirality matrix Gamma that pairs the blocks of opposite type and reverses the mass,
# and the exact rescaling identity between the slices of the deflating history; it checks
# every step against the Revision records and draws five teaching figures. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
```

```
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 14a_block_reduction.ipynb
#
```

```
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 14a_block_reduction.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 14a_block_reduction.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/14a.captions.json
# - Revision/textbook/figures/14a_1_hidden_coordinate.png
# - Revision/textbook/figures/14a_2_inflation_deflation.png
# - Revision/textbook/figures/14a_3_spin_connection_terms.png
# - Revision/textbook/figures/14a_4_block_structure.png
# - Revision/textbook/figures/14a_5_rescaling.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all five figure files of this notebook exist
#     ALL 28 CHECKS PASSED (notebook 14a)
#
# and the notebook must show 5 figures below the cells that draw them.
```

```
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/14a_block_reduction.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 14a_block_reduction.ipynb
#     python -m nbconvert --execute --inplace 14a_block_reduction.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - the cells of sections 7 and 8 seem to hang: they do exact algebra with 16 x 16
#   matrices of symbols and take about 10 seconds each on a laptop; wait until the PASS
#   lines appear.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

```
NOTEBOOK_ID = "14a"  # this notebook: chapter 14, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
```

```
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 14a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates $x_0, \dots, x_7$**: the author's names. $x_1, x_2, x_3$ are
  ordinary 3-space; $x_4$ is the time; $x_5, x_6, x_7$ are the three EXTRA TIMES;
  $x_0$ is the hidden space direction. In Python lists they have the positions
  0 to 7.
- **Metric, scale factor**: the metric says how long a small step $dx_\mu$ is.
  A scale factor multiplies a coordinate step: along $x_1$ a step $dx_1$ has the
  length $e^{a_4} \sin^{1/6} z \, dx_1$.
- **Inflating, deflating**: $a_4(x_4)$ grows with the time $x_4$. Then
  $e^{a_4}$ grows (3-space inflates) and $e^{-a_4}$ shrinks exponentially (the
  three extra times deflate).
- **Slice**: one instant $x_4$ of the history; $a_{4,0} = a_4(x_4)$ is the value
  of $a_4$ there. The history $a_4 = A H x_4$ is a PRESCRIBED BACKGROUND (it is
  given, not solved for, and the Kohn-Sham gas does not act back on it).
- **Hidden coordinate $y$**: $y = \ln(\sin z)/(6H)$ with $z = 6 H x_0$; it measures
  proper length along $x_0$.
- **Warp factor $W$**: $W = e^{Hy}$; the scale factors of the six directions
  $x_1, x_2, x_3, x_5, x_6, x_7$ contain $W$.
- **Gamma matrices $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$**: the author's real
  $16 \times 16$ matrices with $\gamma^a\gamma^b + \gamma^b\gamma^a =
  2\eta^{ab}$ (the Clifford relation); in the code `g1`, ..., `g0`.
- **Spin connection $\Omega_\mu$**: the matrices that the curvature of space adds
  to the derivative of a spinor: $D_\mu = \partial_\mu + \Omega_\mu$.
- **Hermitian matrix**: equal to its conjugate transpose, $X^\dagger = X$; its
  eigenvalues are real. **Unitary**: $V^\dagger V = 1$ (a change of basis that
  keeps lengths).
- **Commute, anticommute**: $XY = YX$, or $XY = -YX$.
- **Projector**: a matrix with $P^2 = P$; its rank is the dimension of the space
  it projects onto.
- **Block**: a $2 \times 2$ piece on the diagonal of a matrix that has no other
  nonzero entries in its rows and columns.
- **Pauli matrices** $\sigma_1 = [[0,1],[1,0]]$, $\sigma_2 = [[0,-i],[i,0]]$,
  $\sigma_3 = [[1,0],[0,-1]]$ (written row by row).
- **Chirality matrix $\Gamma$** $= \gamma^{(x_0)}\gamma^{(x_1)}\cdots
  \gamma^{(x_7)}$: a MATRIX (not a complex conjugation); here it equals
  diag$(-1, \dots, -1, 1, \dots, 1)$.
- **sympy**: computes exactly with symbols and fractions; a check made with
  sympy is an exact identity, not a numerical approximation.
- **Revision record**: the files under `Revision/` that hold the verified
  computations; a PASS line ending with "reproduces ..." names the record file
  and the check that this notebook computed again.
```

Text cell:

```
## 4. The physical and mathematical situation
```

```
The author's metric is diagonal. In the order $x_0, \dots, x_7$ its entries are
$e^{2a_4}\sin^{1/3}z$ (three times), $-1$, $-e^{-2a_4}\sin^{1/3}z$ (three times)
and $\cot^2 z$, with $z = 6Hx_0$ between $0$ and $\pi/2$ and $H > 0$. The signs
$\eta = (+,+,+,+,-,-,-,-)$ say which directions are space-like ($+$) and which
are time-like ($-$).

In the Kohn-Sham model (the mean-field description of many quanta) every quantum
obeys the equation

$$\gamma^\mu D_\mu \Psi = (M_{\rm eff} - i v_v \gamma^{(x_4)})\Psi ,$$

where $M_{\rm eff}(y)$ is the effective mass and $v_v(y)$ the vector potential
made by all the other quanta (their formulas are not needed here). We use the
GOOD SECTOR: $\Psi$ does not depend on the extra times $x_5, x_6, x_7$. This is
an ASSUMPTION of the Revision theory: only without extra-time dependence does the
quantised field have a space of states with a positive norm, and modes with
enough momentum along the extra times grow without bound. We look for orbitals of
the form

$$\Psi = e^{i \mathbf{k}\cdot\mathbf{x}}\, W(y)^{-3}\, \chi(y, x_4),$$

a plane wave with 3-momentum $\mathbf k$ along 3-space, the factor $W^{-3}$, and a
column $\chi$ of 16 functions of $y$ and the time. The notebook shows that
$\chi$ obeys $i\,\partial_{x_4}\chi = h\chi$ and that $h$ splits into eight
$2\times2$ blocks. At one instant (one slice) the stationary orbitals solve
$h\chi = \varepsilon\chi$: the stationary-slice (adiabatic) ansatz of the
Revision theory.

Status of the statements: every identity of this notebook is PROVED (exact, and
verified twice in the Revision record); the good sector is ASSUMED; the history
$a_4 = A H x_4$ is a PRESCRIBED BACKGROUND.
```

Text cell:

```
## 5. The author's gamma matrices

The Revision record `Revision/algebra/gammas.json` stores the eight matrices
$\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$ as lists of rows; `gamma[a]` is
$\gamma^{(x_a)}$. The next cell reads them, turns them into exact sympy
matrices and checks the Clifford relation $\gamma^a\gamma^b + \gamma^b\gamma^a =
2\eta^{ab}\,1$ for all 64 pairs $(a, b)$. It also defines the helper
`record_check`, which reads the verdict of a named check from a Revision report;
every check below that repeats a Revision check also requires that the record
says PASS.
```

In [2]:

```
import numpy as np  # floating-point arrays and matrices
import sympy as sp  # exact algebra with symbols

PY_REPORT = "Revision/kohn_sham/reports/ks-theory-python.json"
WL_REPORT = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
```

```
def record_check(name):
    """True when the check called name is PASS in the sympy record and, if the
    WolframScript record has a check of that name, PASS there too."""
    verdicts = {}
    for report in (PY_REPORT, WL_REPORT):
        data = json.loads(repository_file(report).read_text(encoding="utf-8"))
        found = [c["verdict"] for c in data["checks"] if c["name"] == name]
        verdicts[report] = found[0] if found else "absent"
    return (verdicts[PY_REPORT] == "PASS"
            and verdicts[WL_REPORT] in ("PASS", "absent"))


fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
g = [sp.Matrix(m) for m in fixture["gamma"]]  # g[0] = gamma^(x0) ... g[7] = gamma^(x7)
eta = [int(v) for v in fixture["eta"]]  # +1 space-like, -1 time-like
g0, g1, g2, g3, g4, g5, g6, g7 = g  # gN is gamma^(xN)
I16, Z16 = sp.eye(16), sp.zeros(16)  # the 16 x 16 unit and zero matrices
say(f"eta in the order x0 ... x7: {eta}")
entries = sorted({int(e) for m in g for e in m})  # the different entries
say(f"the entries of the eight gamma matrices are the integers {entries}")
clifford = all(g[a] * g[b] + g[b] * g[a] == (2 * eta[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
check(clifford and eta == [1, 1, 1, 1, -1, -1, -1, -1]
      and record_check("clifford_relation"),
      "the Clifford relation gamma^a gamma^b + gamma^b gamma^a = 2 eta^ab",
      record=f"{PY_REPORT}, check clifford_relation")
```

Out [2]:

```
eta in the order x0 ... x7: [1, 1, 1, 1, -1, -1, -1, -1]
the entries of the eight gamma matrices are the integers [-1, 0, 1]
PASS the Clifford relation gamma^a gamma^b + gamma^b gamma^a = 2 eta^ab
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check clifford_relation
```

Text cell:

```
The next cell builds three products that the theory uses again and again:

- $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, the product of
  the four space-like gammas; the Dirac adjoint is $\bar\Psi = \Psi^\dagger C$.
- $B = -i\,C\,\gamma^{(x_4)}$, the matrix of the positive inner product of the
  good sector ($\langle\Psi^\dagger X \Psi\rangle = u^\dagger B X u$ for one
  quantum in the mode $u$).
- $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$, the chirality
  matrix.

It checks that $C$ is real and symmetric with $C^2 = 1$, that $B$ is Hermitian
with $B^2 = 1$, that $\Gamma$ is diagonal with eight entries $-1$ followed by
eight entries $+1$, and three product rules used later: $BC = -i\gamma^{(x_4)}$,
```

```
$C\gamma^{(x_4)} = iB$ and $CBC = B$. (‘X.H‘ is the conjugate transpose
$X^\dagger$ in sympy and ‘X.T‘ the transpose.)
```

In [3]:

```
C = g0 * g1 * g2 * g3  # the matrix of the Dirac adjoint
B = -sp.I * C * g4  # the matrix of the positive inner product
Gamma = g0 * g1 * g2 * g3 * g4 * g5 * g6 * g7  # the chirality matrix
check(C.T == C and C * C == I16 and B.H == B and B * B == I16
      and Gamma == sp.diag(*([-1] * 8 + [1] * 8)) and record_check("C_B_Gamma"),
      "C real symmetric with C^2 = 1, B Hermitian with B^2 = 1, Gamma = diag(-1, 1)",
      record=f"{PY_REPORT}, check C_B_Gamma")
check(B * C == -sp.I * g4 and C * g4 == sp.I * B and C * B * C == B
      and record_check("BC_and_Cg4"),
      "BC = -i gamma^(x4), C gamma^(x4) = i B and C B C = B",
      record=f"{PY_REPORT}, check BC_and_Cg4")
```

Out [3]:

```
PASS C real symmetric with C^2 = 1, B Hermitian with B^2 = 1, Gamma = diag(-1, 1)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check C_B_Gamma
PASS BC = -i gamma^(x4), C gamma^(x4) = i B and C B C = B
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check BC_and_Cg4
```

Text cell:

```
## 6. The hidden coordinate y and the warped form of the metric

Along $x_0$ the metric has the entry $g_{00} = \cot^2 z$ with $z = 6Hx_0$. We
replace $x_0$ by

$$y = \frac{\ln(\sin z)}{6H}.$$

Line by line:

1. Differentiate with the chain rule: $dy/dx_0 = \frac{1}{6H}\cdot\frac{\cos z}
   {\sin z}\cdot 6H = \cot z$.
2. Square: $dy^2 = \cot^2 z\, dx_0^2 = g_{00}\,dx_0^2$. So $y$ measures proper
   length along the hidden direction, and in the new coordinate the entry is 1.
3. Exponentiate the definition: $e^{6Hy} = \sin z$, hence $\sin^{1/3} z =
   e^{2Hy} = W^2$ with the warp factor $W = e^{Hy}$.
4. Range: $z \to 0$ (the TIP) gives $\sin z \to 0$ and $y \to -\infty$; $z =
   \pi/2$ (the end of the author’s coordinate patch, the BRANE) gives $y = 0$.

The metric becomes the warped form

$$ds^2 = e^{2Hy}\left[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 +
dx_6^2 + dx_7^2)\right] - dx_4^2 + dy^2 .$$

Its scale factors (the vielbein entries) are $h = (e^{a_4}W, e^{a_4}W, e^{a_4}W,
1, e^{-a_4}W, e^{-a_4}W, e^{-a_4}W, 1)$. The square root of the absolute value
```

```
of the determinant is their product: $e^{3a_4}W^3\cdot e^{-3a_4}W^3 = W^6 =
e^{6Hy}$. In the author's $x_0$ chart it is $\sin z \cdot \cot z = \cos z$; the
two differ by the factor $dy/dx_0 = \cot z$ of step 1. Neither depends on $a_4$:
while 3-space inflates and the extra times deflate, the 7-volume stays the same.

The next cell verifies steps 1 to 3 and the determinant with sympy.
```

In [4]:

```
H = sp.symbols("H", positive=True)  # the author's constant H > 0
y_neg = sp.symbols("y", negative=True)  # the hidden coordinate inside the patch
x0, z = sp.symbols("x0 z", positive=True)
z_of_y = sp.asin(sp.exp(6 * H * y_neg))  # z(y): the solution of sin z = e^(6Hy)
dx0_dy = sp.diff(z_of_y / (6 * H), y_neg)  # x0 = z/(6H) differentiated in y
ok_line = sp.simplify(sp.cot(z_of_y) ** 2 * dx0_dy ** 2 - 1) == 0  # g00 dx0^2 = dy^2
ok_warp = sp.simplify(sp.sin(z_of_y) ** sp.Rational(1, 3)
                      - sp.exp(2 * H * y_neg)) == 0  # sin^(1/3) z = e^(2Hy)
ok_dy = sp.simplify(sp.diff(sp.log(sp.sin(6 * H * x0)) / (6 * H), x0)
                    - sp.cot(6 * H * x0)) == 0  # dy/dx0 = cot z
check(ok_line and ok_warp and ok_dy and record_check("geometry_hidden_coordinate"),
      "y = ln(sin z)/(6H): dy = cot z dx0 and sin^(1/3) z = e^(2Hy)",
      record=f"{PY_REPORT}, check geometry_hidden_coordinate")
a0 = sp.symbols("a0", real=True)  # the value of a4 at one slice
third = sp.Rational(1, 3)
metric_x0 = ([sp.exp(2 * a0) * sp.sin(z) ** third] * 3 + [-1]
             + [-sp.exp(-2 * a0) * sp.sin(z) ** third] * 3 + [sp.cot(z) ** 2])
det_x0 = sp.simplify(sp.Abs(sp.prod(metric_x0)))  # |det g| in the x0 chart
say(f"|det g| in the x0 chart of the author = {det_x0}")
check(sp.simplify(det_x0 - sp.cos(z) ** 2) == 0 and record_check("geometry_sqrt_det"),
      "|det g| = cos^2 z in the x0 chart; it does not contain a4",
      record=f"{PY_REPORT}, check geometry_sqrt_det")
```

Out [4]:

```
PASS y = ln(sin z)/(6H): dy = cot z dx0 and sin^(1/3) z = e^(2Hy)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    geometry_hidden_coordinate
|det g| in the x0 chart of the author = cos(z)**2
PASS |det g| = cos^2 z in the x0 chart; it does not contain a4
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check geometry_sqrt_det
```

Text cell:

```
The next cell draws the change of coordinate (with $H = 1$). Left: $y$ as a
function of $z$; at the tip $z \to 0$ the coordinate $y$ falls to $-\infty$, at
the brane $z = \pi/2$ it is 0. Right: three factors along $y$ from $-3$ to $0$
(the Rust solver cuts the hidden direction at $y = -L = -3$): the warp factor
$W = e^{Hy}$, the factor $\sin^{1/3}z = W^2$ of the metric and the volume factor
$\sqrt{|g|} = W^6$ (the proper 7-volume per unit coordinate volume). The
vertical axis is logarithmic, so each exponential is a straight line with slope
1, 2 or 6.
```

In [5]:

```
z_values = np.linspace(1e-3, np.pi / 2, 600)  # z from almost 0 to pi/2
y_of_z = np.log(np.sin(z_values)) / 6.0  # y = ln(sin z)/(6H) with H = 1
y_values = np.linspace(-3.0, 0.0, 301)  # the hidden coordinate from -L to 0

fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(z_values, y_of_z, color="C0")
left.set_xlabel("$z = 6 H x_0$ (radians)")
left.set_ylabel("$y = \\ln(\\sin z)/(6H)$")
left.set_title("the hidden coordinate $y$ ($H = 1$)")
left.annotate("tip: $y \\to -\\infty$", (0.03, -1.0), fontsize=9)
left.annotate("brane: $y = 0$", (1.05, -0.15), fontsize=9)
right.semilogy(y_values, np.exp(y_values), label="warp $W = e^{Hy}$")
right.semilogy(y_values, np.exp(2 * y_values), "--",
               label="$\\sin^{1/3} z = W^2$")
right.semilogy(y_values, np.exp(6 * y_values), ":",
               label="volume factor $\\sqrt{|g|} = W^6$")
right.set_xlabel("$y$ (units of $1/H$)")
right.set_ylabel("factor (logarithmic scale)")
right.set_title("the factors of the warped metric")
right.legend(fontsize=8)
save_figure(fig, "hidden_coordinate",
            "Left: the hidden coordinate $y = \\ln(\\sin z)/(6H)$ against "
            "$z = 6Hx_0$ (radians) for $H = 1$; $y$ tends to minus infinity at the "
            "tip $z \\to 0$ and is $0$ at the brane $z = \\pi/2$. Right: the warp "
            "factor $W = e^{Hy}$ (solid), the metric factor $\\sin^{1/3} z = W^2$ "
            "(dashed) and the volume factor $\\sqrt{|g|} = W^6$ (dotted) against "
            "$y$ from $-3$ to $0$ in units of $1/H$, on a logarithmic vertical "
            "axis; the three straight lines have the slopes 1, 2 and 6, and the "
            "proper 7-volume near the tip is smaller than at the brane by the "
            "factor $e^{-18}$.")
```

Out [5]:

```
Figure 14a.1 saved as Revision/textbook/figures/14a_1_hidden_coordinate.png
```

Text cell:

```
The next cell draws what the deflating history does. Left: along the PRESCRIBED
history $a_4 = A H x_4$ with $A = H = 1$ (so $a_4 = x_4$), the 3-space scale
factor $e^{a_4}$ grows, the extra-time scale factor $e^{-a_4}$ shrinks, and the
product of their cubes, $e^{3a_4}e^{-3a_4} = 1$ (the 7-volume), stays constant.
Right: the MOMENTUM WEIGHT $\kappa(y) = e^{-Hy - a_{4,0}}$ against $y$ at the five
slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$ of the Revision Rust solver. Section 8 shows
that a 3-momentum $k$ enters the equation only as $\kappa k$: the weight grows
toward the tip (a momentum costs more energy far from the brane) and drops by
$e^{-a_{4,0}}$ from slice to slice (the 3-momenta are redshifted).
```

In [6]:

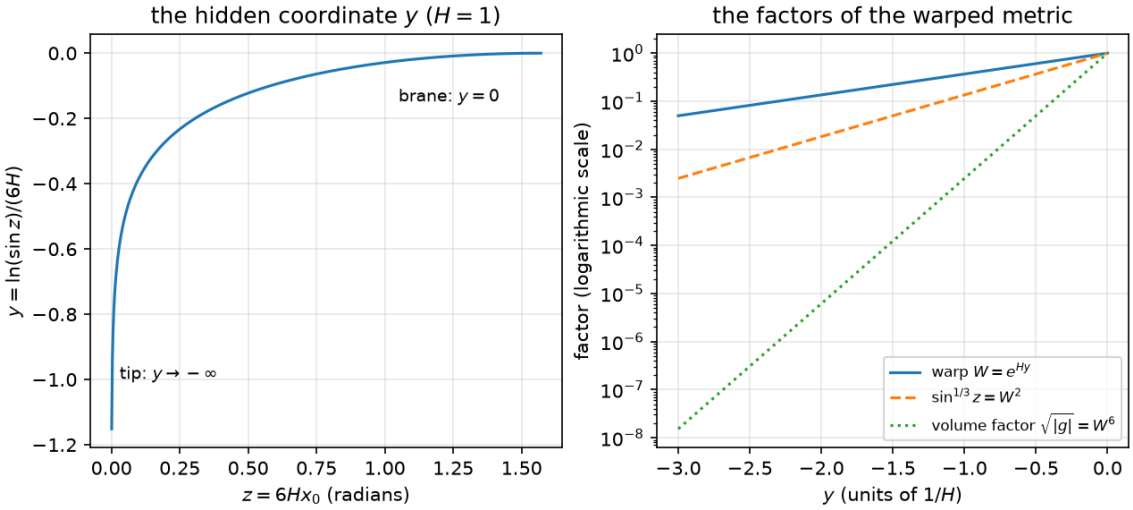

Figure 384: Left: the hidden coordinate $y = \ln(\sin z)/(6H)$ against $z = 6Hx_0$ (radians) for $H = 1$; y tends to minus infinity at the tip $z \to 0$ and is 0 at the brane $z = \pi/2$. Right: the warp factor $W = e^{Hy}$ (solid), the metric factor $\sin^{1/3} z = W^2$ (dashed) and the volume factor $\sqrt{|g|} = W^6$ (dotted) against y from -3 to 0 in units of $1/H$, on a logarithmic vertical axis; the three straight lines have the slopes 1, 2 and 6, and the proper 7-volume near the tip is smaller than at the brane by the factor e^{-18}. (Notebook 14a, figure 1.)

```
a4_values = np.linspace(0.0, 2.0, 201)  # a4 along the history (= x4 for A = H = 1)
slices = [0.0, 0.5, 1.0, 1.5, 2.0]  # the five slices of the Revision solver

fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(a4_values, np.exp(a4_values), label="3-space: $e^{a_4}$ (inflates)")
left.plot(a4_values, np.exp(-a4_values), "--",
          label="extra times: $e^{-a_4}$ (deflates)")
left.plot(a4_values, np.exp(3 * a4_values) * np.exp(-3 * a4_values), ":",
          color="black", label="7-volume: $e^{3a_4} e^{-3a_4} = 1$")
left.set_xlabel("$a_4 = A H x_4$ (with $A = H = 1$: the time $x_4$)")
left.set_ylabel("scale factor at the brane $y = 0$")
left.set_title("inflation and deflation along the history")
left.legend(fontsize=8)
for a in slices:  # one curve per slice
    right.semilogy(y_values, np.exp(-y_values - a), label=f"$a_{{4,0}} = {a}$")
right.set_xlabel("$y$ (units of $1/H$)")
right.set_ylabel("$\\kappa = e^{-Hy - a_{4,0}}$ (logarithmic)")
right.set_title("the momentum weight at five slices")
right.legend(fontsize=8)
save_figure(fig, "inflation_deflation",
            "Left: along the prescribed history $a_4 = A H x_4$ with $A = H = 1$, "
            "the 3-space scale factor $e^{a_4}$ (solid) grows, the scale factor "
            "$e^{-a_4}$ of the three extra times (dashed) deflates exponentially, "
            "and the 7-volume factor $e^{3a_4}e^{-3a_4} = 1$ (dotted) stays "
            "constant; horizontal axis $a_4$, vertical axis the scale factor at "
            "the brane $y = 0$. Right: the momentum weight "
            "$\\kappa = e^{-Hy - a_{4,0}}$ against $y$ (units of $1/H$) at the "
            "five slices $a_{4,0} = 0$ to $2$, logarithmic vertical axis: the "
```

```
"weight grows toward the tip and every later slice lies lower by the "
"factor $e^{-0.5}$, the redshift of the 3-momenta.")
```

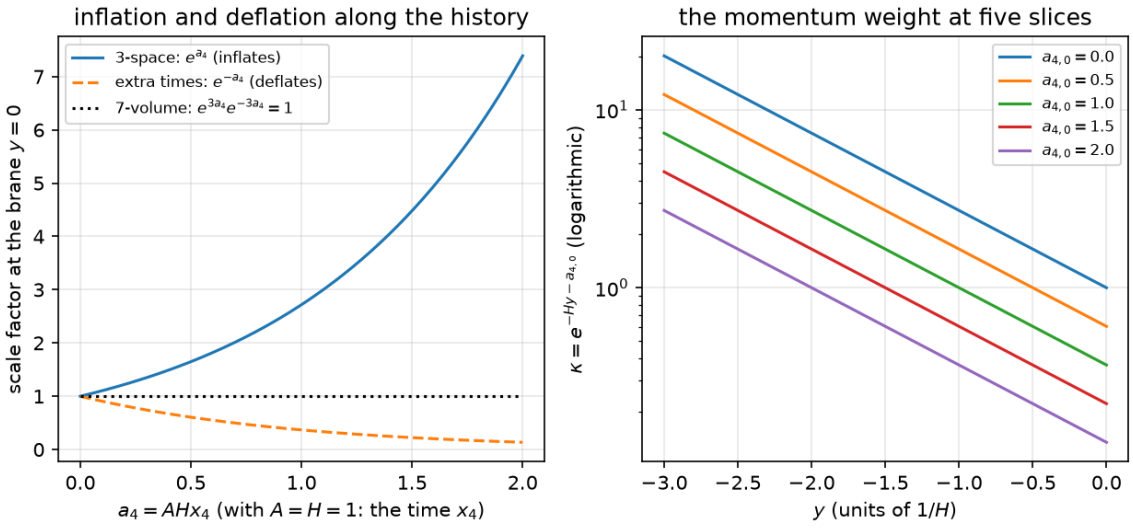

Figure 385: Left: along the prescribed history $a_4 = AHx_4$ with $A = H = 1$, the 3-space scale factor e^{a_4} (solid) grows, the scale factor e^{-a_4} of the three extra times (dashed) deflates exponentially, and the 7-volume factor $e^{3a_4}e^{-3a_4} = 1$ (dotted) stays constant; horizontal axis a_4, vertical axis the scale factor at the brane $y = 0$. Right: the momentum weight $\kappa = e^{-Hy-a_{4,0}}$ against y (units of $1/H$) at the five slices $a_{4,0} = 0$ to 2, logarithmic vertical axis: the weight grows toward the tip and every later slice lies lower by the factor $e^{-0.5}$, the redshift of the 3-momenta. (Notebook 14a, figure 2.)

Out [6]:

```
Figure 14a.2 saved as Revision/textbook/figures/14a_2_inflation_deflation.png
```

Text cell:

```
## 7. The spin connection and the term 3H gamma^(x0)

A spinor needs the covariant derivative $D_\mu\Psi = \partial_\mu\Psi +
\Omega_\mu\Psi$. For the diagonal vielbein $h_a$ of section 6 the canonical spin
connection (the one fixed by the vielbein postulate) is computed in three steps:

1. The Christoffel symbols $\Gamma^n_{mk} = \frac12 g^{nl}(\partial_m g_{lk} +
   \partial_k g_{lm} - \partial_l g_{mk})$. For a diagonal metric only the terms
   with two equal indices survive: $\Gamma^n_{mk} = [\delta_{mn}\partial_k g_{nn}
   + \delta_{kn}\partial_m g_{nn} - \delta_{mk}\partial_n g_{mm}]/(2g_{nn})$.
2. The connection coefficients $\omega_\mu{}^A{}_B = h_A\,\partial_\mu(1/h_B)\,
   \delta^A_B + h_A\,\Gamma^A_{\mu B}/h_B$ (no sum over $A$, $B$).
3. $\Omega_\mu = \frac12\sum_{A,B}\eta_{AA}\,\omega_\mu{}^A{}_B\,S^{AB}$ with
   $S^{AB} = \frac14(\gamma^A\gamma^B - \gamma^B\gamma^A)$.

The next cell does these three steps with $a_4$ an arbitrary function of $x_4$
(so it holds along any history), checks that the curved gammas $\gamma^\mu =
```

```
\gamma^{(\mu)}/h_\mu$ are covariantly constant ($\nabla_\mu\gamma^\nu = 0$: 64
matrix equations, the proof that the connection is the right one), and computes
the only combination that enters the Dirac operator,
$\gamma^\mu\Omega_\mu = \sum_\mu (\gamma^{(\mu)}/h_\mu)\,\Omega_\mu$. It takes
about 10 seconds.
```

In [7]:

```
X = (sp.Symbol("y", real=True),) + sp.symbols("x1:8", real=True)  # y, x1..x7
Y, x4 = X[0], X[4]  # the hidden coordinate and the time
a4 = sp.Function("a4", real=True)(x4)  # a4 is any function of the time x4
a4p = sp.diff(a4, x4)  # its derivative a4'
W = sp.exp(H * Y)  # the warp factor
vb = [sp.Integer(1)] + [sp.exp(a4) * W] * 3 + [sp.Integer(1)] + [sp.exp(-a4) * W] * 3
gdiag = [eta[i] * vb[i] ** 2 for i in range(8)]  # the diagonal metric entries


def christoffel(n, m, k):
    """Gamma^n_mk of the diagonal metric gdiag (step 1)."""
    total = 0
    if m == n:
        total += sp.diff(gdiag[n], X[k])
    if k == n:
        total += sp.diff(gdiag[n], X[m])
    if m == k:
        total -= sp.diff(gdiag[m], X[n])
    return sp.simplify(total / (2 * gdiag[n]))


Chr = [[[christoffel(n, m, k) for k in range(8)] for m in range(8)]
       for n in range(8)]
S = [[(g[a] * g[b] - g[b] * g[a]) / 4 for b in range(8)] for a in range(8)]
Omega = []  # Omega[mu] is the 16 x 16 matrix Omega_mu
for mu in range(8):
    total = sp.zeros(16)
    for A in range(8):
        for Bi in range(8):
            w = (vb[A] * sp.diff(1 / vb[Bi], X[mu]) if A == Bi else 0) \
                + vb[A] * Chr[A][mu][Bi] / vb[Bi]  # omega_mu^A_B (step 2)
            w = sp.simplify(w)
            if w != 0:
                total += sp.Rational(1, 2) * eta[A] * w * S[A][Bi]  # step 3
    Omega.append(sp.simplify(total))
compatible = all(
    (sp.diff(g[n] / vb[n], X[m])
     + sum((Chr[n][m][l] * g[l] / vb[l] for l in range(8)), Z16)
     + Omega[m] * g[n] / vb[n] - g[n] / vb[n] * Omega[m]).applyfunc(sp.simplify)
    .is_zero_matrix for m in range(8) for n in range(8))
check(compatible and record_check("spin_connection_compatibility"),
      "the curved gammas are covariantly constant (64 matrix equations)",
      record=f"{PY_REPORT}, check spin_connection_compatibility")
slash = sp.simplify(sum((g[m] / vb[m] * Omega[m] for m in range(8)), Z16))
check((slash - 3 * H * g0).is_zero_matrix and record_check("spin_connection_slash_3H"),
```

```
      "gamma^mu Omega_mu = 3H gamma^(x0) exactly, for every history a4(x4)",
      record=f"{PY_REPORT}, check spin_connection_slash_3H")
```

Out [7]:

```
PASS the curved gammas are covariantly constant (64 matrix equations)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    spin_connection_compatibility
PASS gamma^mu Omega_mu = 3H gamma^(x0) exactly, for every history a4(x4)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    spin_connection_slash_3H
```

Text cell:

```
Where does $3H\gamma^{(x_0)}$ come from, and where did the time derivative $a_4'$
go? The next cell splits $\gamma^\mu\Omega_\mu$ into the contributions of the
eight directions. Each contribution $(\gamma^{(\mu)}/h_\mu)\Omega_\mu$ turns out
to be a combination $c_4\gamma^{(x_4)} + c_0\gamma^{(x_0)}$. Because
$\gamma^{(x_4)}\gamma^{(x_4)} = -1$, $\gamma^{(x_0)}\gamma^{(x_0)} = +1$ and the
trace of $\gamma^{(x_4)}\gamma^{(x_0)}$ is 0, the coefficients are found with
traces: $c_4 = -\mathrm{Tr}(X\gamma^{(x_4)})/16$ and $c_0 =
\mathrm{Tr}(X\gamma^{(x_0)})/16$. The cell checks that each contribution is
exactly $c_4\gamma^{(x_4)} + c_0\gamma^{(x_0)}$, prints the coefficients, and
checks the two facts of the Revision record: the three inflating directions give
$+\frac32 a_4'\gamma^{(x_4)}$, the three deflating directions give
$-\frac32 a_4'\gamma^{(x_4)}$, so the time-direction pieces cancel; and
$\Omega_{x_4} = \Omega_y = 0$.
```

In [8]:

```
names = ["x0 (y)", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
PRIME = sp.Symbol("da4dx4")  # a name for the derivative da4/dx4 in the printout
c4_list, c0_list = [], []  # the coefficients of gamma^(x4) and gamma^(x0)
split_ok = True
for m in range(8):
    part = sp.simplify(g[m] / vb[m] * Omega[m])  # the contribution of direction m
    c4 = sp.simplify(-(part * g4).trace() / 16)
    c0 = sp.simplify((part * g0).trace() / 16)
    split_ok &= (part - c4 * g4 - c0 * g0).applyfunc(sp.simplify).is_zero_matrix
    c4_list.append(c4)
    c0_list.append(c0)
    shown_c4 = str(c4.subs(a4p, PRIME))  # write da4dx4 for da4/dx4
    say(f"direction {names[m]:7}: c4 = {shown_c4:12} c0 = {c0}")
check(split_ok, "every contribution is c4 gamma^(x4) + c0 gamma^(x0)")
inflating = sp.simplify(sum(c4_list[1:4]))  # x1, x2, x3
deflating = sp.simplify(sum(c4_list[5:8]))  # x5, x6, x7
say("sum of c4 over the inflating directions x1, x2, x3: "
    f"{inflating.subs(a4p, PRIME)}")
say("sum of c4 over the deflating directions x5, x6, x7: "
    f"{deflating.subs(a4p, PRIME)}")
say(f"sum of c0 over all directions: {sp.simplify(sum(c0_list))}")
```

```
check(sp.simplify(inflating - sp.Rational(3, 2) * a4p) == 0
      and sp.simplify(deflating + sp.Rational(3, 2) * a4p) == 0
      and Omega[4].is_zero_matrix and Omega[0].is_zero_matrix
      and record_check("spin_connection_time_terms_cancel")
      and record_check("spin_connection_time_term_value"),
      "the da4/dx4 pieces +3/2 and -3/2 cancel; Omega_x4 = Omega_y = 0",
      record=f"{PY_REPORT}, checks spin_connection_time_terms_cancel and "
             "spin_connection_time_term_value")
```

Out [8]:

```
direction x0 (y) : c4 = 0            c0 = 0
direction x1     : c4 = da4dx4/2     c0 = H/2
direction x2     : c4 = da4dx4/2     c0 = H/2
direction x3     : c4 = da4dx4/2     c0 = H/2
direction x4     : c4 = 0            c0 = 0
direction x5     : c4 = -da4dx4/2    c0 = H/2
direction x6     : c4 = -da4dx4/2    c0 = H/2
direction x7     : c4 = -da4dx4/2    c0 = H/2
PASS every contribution is c4 gamma^(x4) + c0 gamma^(x0)
sum of c4 over the inflating directions x1, x2, x3: 3*da4dx4/2
sum of c4 over the deflating directions x5, x6, x7: -3*da4dx4/2
sum of c0 over all directions: 3*H
PASS the da4/dx4 pieces +3/2 and -3/2 cancel; Omega_x4 = Omega_y = 0
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, checks
    spin_connection_time_terms_cancel and spin_connection_time_term_value
```

Text cell:

```
The next cell draws these coefficients as a bar chart, with the numbers
$a_4' = 1$ and $H = 1$ put in: for each direction one bar for the coefficient of
$\gamma^{(x_4)}$ and one bar for the coefficient of $\gamma^{(x_0)}$.
```

In [9]:

```
c4_numbers = [float(c.subs(a4p, 1).subs(H, 1)) for c in c4_list]  # da4/dx4 = H = 1
c0_numbers = [float(c.subs(a4p, 1).subs(H, 1)) for c in c0_list]
positions = np.arange(8)  # one group of two bars per direction

fig, ax = plt.subplots(figsize=(8.0, 4.0))
ax.bar(positions - 0.2, c4_numbers, width=0.38,
       label="coefficient of $\\gamma^{(x_4)}$ (per unit $da_4/dx_4$)")
ax.bar(positions + 0.2, c0_numbers, width=0.38,
       label="coefficient of $\\gamma^{(x_0)}$ (per unit $H$)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(positions)
ax.set_xticklabels(["$x_0$", "$x_1$", "$x_2$", "$x_3$", "$x_4$", "$x_5$", "$x_6$",
                    "$x_7$"])
ax.set_xlabel("direction $\\mu$ of the term $(\\gamma^{(\\mu)}/h_\\mu)\\,"
              "\\Omega_\\mu$")
ax.set_ylabel("coefficient")
```

```
ax.set_title("$\\gamma^\\mu\\Omega_\\mu = 3H\\gamma^{(x_0)}$: "
             "the $da_4/dx_4$ pieces cancel")
ax.legend(fontsize=8)
save_figure(fig, "spin_connection_terms",
            "The contribution of each direction $\\mu$ to "
            "$\\gamma^\\mu\\Omega_\\mu$, written as "
            "$c_4\\gamma^{(x_4)} + c_0\\gamma^{(x_0)}$: the bars show $c_4$ per "
            "unit $da_4/dx_4$ (left bar of each pair) and $c_0$ per unit $H$ "
            "(right bar). The inflating directions $x_1, x_2, x_3$ give "
            "$+(da_4/dx_4)/2$ each and the deflating extra times $x_5, x_6, x_7$ "
            "give $-(da_4/dx_4)/2$ each, so "
            "the time-direction pieces cancel; each of the six warped directions "
            "gives $H/2$ to the coefficient of $\\gamma^{(x_0)}$, in total $3H$; "
            "the time $x_4$ and the hidden direction give nothing.")
```

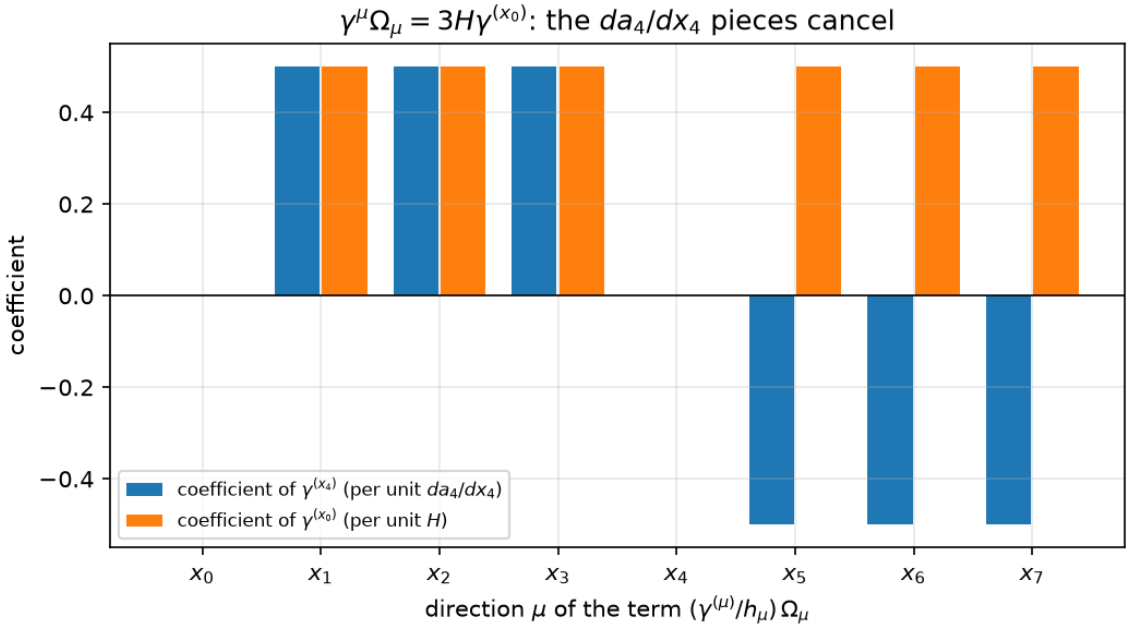

Figure 386: The contribution of each direction μ to $\gamma^\mu\Omega_\mu$, written as $c_4\gamma^{(x_4)} + c_0\gamma^{(x_0)}$: the bars show c_4 per unit da_4/dx_4 (left bar of each pair) and c_0 per unit H (right bar). The inflating directions x_1, x_2, x_3 give $+(da_4/dx_4)/2$ each and the deflating extra times x_5, x_6, x_7 give $-(da_4/dx_4)/2$ each, so the time-direction pieces cancel; each of the six warped directions gives $H/2$ to the coefficient of $\gamma^{(x_0)}$, in total $3H$; the time x_4 and the hidden direction give nothing. (Notebook 14a, figure 3.)

Out [9]:

```
Figure 14a.3 saved as Revision/textbook/figures/14a_3_spin_connection_terms.png
```

Text cell:

```
## 8. The factor W^(-3) removes the spin connection
```

```
Insert $\Psi = e^{i\mathbf k\cdot\mathbf x}W^{-3}\chi(y, x_4)$ into
$\gamma^\mu D_\mu\Psi = \sum_\mu\gamma^\mu\partial_\mu\Psi + 3H\gamma^{(x_0)}\Psi$
(section 7). Term by term:

1. $y$: $\partial_y(W^{-3}\chi) = W^{-3}(\partial_y\chi - 3H\chi)$ because
   $\partial_y e^{-3Hy} = -3He^{-3Hy}$. With $\gamma^y = \gamma^{(x_0)}$ the term
   $-3H\gamma^{(x_0)}\chi$ cancels the $+3H\gamma^{(x_0)}\chi$ of the spin
   connection.
2. 3-space: $\partial_j e^{i\mathbf k\cdot\mathbf x} = ik_j e^{i\mathbf k\cdot
   \mathbf x}$ and $\gamma^{x_j} = \gamma^{(x_j)}/(e^{a_4}W)$, so the term is
   $i\kappa k_j\gamma^{(x_j)}$ with the momentum weight $\kappa = e^{-Hy-a_4}$.
3. Time: $\gamma^{x_4} = \gamma^{(x_4)}$ acts on $\partial_{x_4}\chi$.
4. Extra times: nothing depends on $x_5, x_6, x_7$ (good sector).

So, dividing by the common factor $e^{i\mathbf k\cdot\mathbf x}W^{-3}$:

$$\gamma^\mu D_\mu\Psi \to \gamma^{(x_0)}\partial_y\chi + \gamma^{(x_4)}
\partial_{x_4}\chi + i\kappa\,k_j\gamma^{(x_j)}\chi .$$

The next cell verifies this with sympy for a column of 16 arbitrary functions
$\chi_A(y, x_4)$ and an arbitrary history $a_4(x_4)$, and verifies that WITHOUT
the factor $W^{-3}$ the term $3H\gamma^{(x_0)}\chi$ survives. It takes about 5
seconds.
```

In [10]:

```
k1, k2, k3 = sp.symbols("k1 k2 k3", real=True)  # the 3-momentum
chi = sp.Matrix([sp.Function(f"chi{i}")(Y, x4) for i in range(16)])
phase = sp.exp(sp.I * (k1 * X[1] + k2 * X[2] + k3 * X[3]))  # the plane wave
kappa = sp.exp(-H * Y - a4)  # the momentum weight


def dirac(Psi):
    """gamma^mu D_mu Psi with the spin connection of section 7."""
    result = sp.zeros(16, 1)
    for m in range(8):
        result += g[m] / vb[m] * (sp.diff(Psi, X[m]) + Omega[m] * Psi)
    return result


reduced = (g0 * sp.diff(chi, Y) + g4 * sp.diff(chi, x4)
           + sp.I * kappa * (k1 * g1 + k2 * g2 + k3 * g3) * chi)
with_W3 = sp.simplify(dirac(phase * W ** -3 * chi) / (phase * W ** -3))
without_W3 = sp.simplify(dirac(phase * chi) / phase)
check((with_W3 - reduced).applyfunc(sp.simplify).is_zero_matrix
      and record_check("ansatz_removes_spin_connection"),
      "with W^(-3): the reduced operator g0 d_y + g4 d_x4 + i kappa k.g, exactly",
      record=f"{PY_REPORT}, check ansatz_removes_spin_connection")
check((without_W3 - reduced - 3 * H * g0 * chi).applyfunc(sp.simplify)
      .is_zero_matrix and record_check("ansatz_without_W3_term_survives"),
      "without W^(-3) the term 3H gamma^(x0) chi survives",
      record=f"{PY_REPORT}, check ansatz_without_W3_term_survives")
```

Out [10]:

```
PASS with W^(-3): the reduced operator g0 d_y + g4 d_x4 + i kappa k.g, exactly
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    ansatz_removes_spin_connection
PASS without W^(-3) the term 3H gamma^(x0) chi survives
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    ansatz_without_W3_term_survives
```

Text cell:

```
## 9. The equation i d_x4 chi = h chi and its Hermitian pieces

With the reduced operator of section 8 the Kohn-Sham equation reads

$$\gamma^{(x_0)}\partial_y\chi + \gamma^{(x_4)}\partial_{x_4}\chi +
i\kappa k_j\gamma^{(x_j)}\chi = (M - iv\gamma^{(x_4)})\chi$$

($M$ the effective mass, $v$ the vector potential). Solve for the time
derivative, line by line:

1. Move everything except the time term to the right: $\gamma^{(x_4)}
   \partial_{x_4}\chi = (M - iv\gamma^{(x_4)})\chi - \gamma^{(x_0)}\partial_y\chi
   - i\kappa k_j\gamma^{(x_j)}\chi$.
2. Multiply from the left by $-\gamma^{(x_4)}$; since $\gamma^{(x_4)}
   \gamma^{(x_4)} = -1$, the left side becomes $\partial_{x_4}\chi$.
3. Multiply by $i$: $i\partial_{x_4}\chi = h\chi$ with
   $h = i\gamma^{(x_4)}\gamma^{(x_0)}\partial_y - \kappa k_j\gamma^{(x_4)}
   \gamma^{(x_j)} + M(-i\gamma^{(x_4)}) + v$.

$h$ is Hermitian (so the levels are real and the norm $\int\chi^\dagger\chi\,dy$
is conserved) because: $\gamma^{(x_4)}\gamma^{(x_0)}$ is real and symmetric, so
$i\gamma^{(x_4)}\gamma^{(x_0)}$ is anti-Hermitian, and $\partial_y$ is
anti-Hermitian too (integration by parts when the boundary terms vanish), so their
product is Hermitian; $\gamma^{(x_4)}\gamma^{(x_j)}$ and $-i\gamma^{(x_4)}$ are
Hermitian matrices. The next cell checks the three lines with 16-component
columns of symbols for $\partial_y\chi$ and $\chi$, and the Hermitian pieces.
```

In [11]:

```
M, v = sp.symbols("M v", real=True)  # effective mass and vector potential
chi_s = sp.Matrix(sp.symbols("c0:16"))  # chi at one point (16 symbols)
dchi_s = sp.Matrix(sp.symbols("d0:16"))  # d chi/dy at that point
kap = sp.symbols("kappa", positive=True)
kg = kap * (k1 * g1 + k2 * g2 + k3 * g3)  # kappa k.gamma
# line 1 and 2: d chi/dx4 = -g4 [(M - i v g4) chi - g0 chi' - i kappa k.g chi]
dt_chi = -g4 * ((M * I16 - sp.I * v * g4) * chi_s - g0 * dchi_s - sp.I * kg * chi_s)
# line 3: h chi with the four pieces of h
h_chi = (sp.I * g4 * g0 * dchi_s - kap * g4 * (k1 * g1 + k2 * g2 + k3 * g3) * chi_s
         + M * (-sp.I * g4) * chi_s + v * chi_s)
check((sp.I * dt_chi - h_chi).expand().is_zero_matrix,
```

```
      "i d_x4 chi = h chi with h = i g4 g0 d_y - kappa k_j g4 g_j + M(-i g4) + v")
hermitian = ((g4 * g0).T == g4 * g0 and all((-g4 * gj).H == -g4 * gj
                                            for gj in (g1, g2, g3))
             and (-sp.I * g4).H == -sp.I * g4 and (-sp.I * g4) ** 2 == I16)
check(hermitian and record_check("hamiltonian_16_hermitian"),
      "g4 g0 real symmetric; g4 g_j and -i g4 Hermitian; (-i g4)^2 = 1",
      record=f"{PY_REPORT}, check hamiltonian_16_hermitian")
```

Out [11]:

```
PASS i d_x4 chi = h chi with h = i g4 g0 d_y - kappa k_j g4 g_j + M(-i g4) + v
PASS g4 g0 real symmetric; g4 g_j and -i g4 Hermitian; (-i g4)^2 = 1
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    hamiltonian_16_hermitian
```

Text cell:

```
## 10. Three matrices that commute with everything: J, K1, K2

Rotations of 3-space do not change the levels (section 14), so we may take
$\mathbf k = (k, 0, 0)$ along $x_1$. Then $h$ is built from the three gammas
$\gamma^{(x_0)}$, $\gamma^{(x_1)}$, $\gamma^{(x_4)}$ only; the first-order form
$\chi' = N_{16}\chi$ of the equation (section 12) uses the three products
$A_0 = \gamma^{(x_0)}$, $A_1 = \gamma^{(x_0)}\gamma^{(x_1)}$,
$A_4 = \gamma^{(x_0)}\gamma^{(x_4)}$, and the densities contain $B$ and $C$.

A rule for products of different gammas: moving $\gamma^c$ through a product of
$p$ different gammas gives the sign $(-1)^{p-1}$ if $\gamma^c$ is one of the
factors and $(-1)^p$ if it is not. With it one finds three matrices that commute
with $A_0, A_1, A_4, B, C$ and with each other:

$$J = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_4)},\quad K_1 = \gamma^{(x_2)}
\gamma^{(x_3)},\quad K_2 = \gamma^{(x_5)}\gamma^{(x_6)} .$$

Their squares: $J^2 = (-1)^3(\gamma^{(x_0)})^2(\gamma^{(x_1)})^2(\gamma^{(x_4)})^2
= (-1)(1)(1)(-1) = +1$ (reversing three factors needs three swaps), $K_1^2 =
-(\gamma^{(x_2)})^2(\gamma^{(x_3)})^2 = -1$, $K_2^2 = -(-1)(-1) = -1$. So $J$
has the eigenvalues $j = \pm1$ and $K_1, K_2$ the eigenvalues $is_2, is_3$ with
$s_2, s_3 = \pm1$. The joint eigenspace with the labels $(j, s_2, s_3)$ is the
range of the projector

$$P(j, s_2, s_3) = \frac{1 + jJ}{2}\cdot\frac{1 - is_2K_1}{2}\cdot
\frac{1 - is_3K_2}{2} .$$

The next cell checks the commutation relations and the squares, builds the
eight projectors and checks that each has rank 2, that $P^2 = P$, and that the
eight add up to the unit matrix: the 16 components split into 8 blocks of 2.
```

In [12]:

```
A0, A1, A4 = g0, g0 * g1, g0 * g4  # the matrices of the equation (k along x1)
J, K1, K2 = g0 * g1 * g4, g2 * g3, g5 * g6
```

```
commute = all((P * Q - Q * P).is_zero_matrix for P in (J, K1, K2)
              for Q in (A0, A1, A4, B, C, J, K1, K2))
check(commute and J * J == I16 and K1 * K1 == -I16 and K2 * K2 == -I16
      and record_check("blocks_commuting_set"),
      "J, K1, K2 commute with A0, A1, A4, B, C and each other; J^2 = 1, K^2 = -1",
      record=f"{PY_REPORT}, check blocks_commuting_set")
labels = [(j, s2, s3) for j in (1, -1) for s2 in (1, -1) for s3 in (1, -1)]
projectors = [(I16 + j * J) / 2 * (I16 - sp.I * s2 * K1) / 2
              * (I16 - sp.I * s3 * K2) / 2 for (j, s2, s3) in labels]
check(all(P.rank() == 2 and (P * P - P).is_zero_matrix for P in projectors)
      and sum(projectors, Z16) == I16 and record_check("blocks_projectors"),
      "eight projectors P(j, s2, s3) of rank 2 that add up to 1",
      record=f"{PY_REPORT}, check blocks_projectors")
```

Out [12]:

```
PASS J, K1, K2 commute with A0, A1, A4, B, C and each other; J^2 = 1, K^2 = -1
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    blocks_commuting_set
PASS eight projectors P(j, s2, s3) of rank 2 that add up to 1
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check blocks_projectors
```

Text cell:

```
## 11. The block basis V

In each eigenspace we choose two basis vectors adapted to the equation. Since
$A_0 = \gamma^{(x_0)}$ squares to 1, $\frac12(1 + \gamma^{(x_0)})$ projects on its
eigenvalue $+1$. Take the first unit column $e_c$ (counting $c$ from 0) for which
$v_+ = 8\,P(j, s_2, s_3)\,\frac12(1 + \gamma^{(x_0)})\,e_c$ is not zero; then
$\gamma^{(x_0)}v_+ = v_+$. Put $v_- = A_1v_+$. Because $A_1$ anticommutes with
$A_0$, $\gamma^{(x_0)}v_- = -v_-$. The 16 columns $v_+, v_-$ of the eight blocks,
in the label order $(1,1,1), (1,1,-1), \dots, (-1,-1,-1)$, form the matrix $U$,
and $V = U/(2\sqrt2)$.

The next cell builds $U$ and $V$, prints the seed columns $c$, checks that $V$
is unitary and that every entry of $U$ is $0$, $\pm1$ or $\pm i$, and compares $U$
entry by entry with the basis stored in the Revision record
‘Revision/kohn_sham/ks-theory.json‘ (which the Rust solver uses).
```

In [13]:

```
columns, seeds = [], []
for P in projectors:
    P_plus = P * (I16 + g0) / 2  # project on gamma^(x0) = +1 inside the block
    c = next(c for c in range(16) if not P_plus[:, c].is_zero_matrix)
    seeds.append(c)
    v_plus = 8 * P_plus[:, c]
    columns += [v_plus, A1 * v_plus]  # v+ and v- = A1 v+
U = sp.Matrix.hstack(*columns)  # 16 x 16, entries 0, +-1, +-i
V = U / (2 * sp.sqrt(2))  # the unitary block basis
```

```
say(f"seed columns c of the eight blocks: {seeds}")
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
as_number = {"0": 0, "1": 1, "-1": -1, "I": sp.I, "-I": -sp.I}
U_record = sp.Matrix([[as_number[e] for e in row] for row in
                      theory["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
check(sp.simplify(V.H * V) == I16 and set(U) <= {0, 1, -1, sp.I, -sp.I}
      and seeds == theory["blockBasis"]["seedColumns0Based"]
      and record_check("blocks_basis_unitary"),
      "V = U/(2 sqrt 2) is unitary, U has entries 0, +-1, +-i",
      record=f"{PY_REPORT}, check blocks_basis_unitary")
check(U == U_record,
      "U equals the basis of the Revision record entry by entry",
      record="Revision/kohn_sham/ks-theory.json, blockBasis.unnormalisedColumns2Sqrt2V")
```

Out [13]:

```
seed columns c of the eight blocks: [4, 0, 0, 4, 0, 4, 4, 0]
PASS V = U/(2 sqrt 2) is unitary, U has entries 0, +-1, +-i
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    blocks_basis_unitary
PASS U equals the basis of the Revision record entry by entry
     reproduces Revision/kohn_sham/ks-theory.json, blockBasis.unnormalisedColumns2Sqrt2V
```

Text cell:

```
## 12. Every matrix of the equation in the block basis

The next cell computes $V^\dagger XV$ for the eleven matrices $X$ that occur in
the equation and in the densities, checks that each is block diagonal (zero
outside the eight $2\times2$ diagonal blocks), and checks the form of each
block against the table of the Revision record. In the block with the labels
$(j, s_2, s_3)$ the forms are printed below. A number such as $js_2$ means that
number times the $2\times2$ unit matrix. It also checks that NOT every matrix is
block diagonal: $\gamma^{(x_2)}$, $\gamma^{(x_5)}$ and
$\gamma^{(x_0)}\gamma^{(x_2)}$ anticommute with $J$ and connect different blocks
(that is why $\mathbf k$ was put along $x_1$), while $\gamma^{(x_1)}$ is block
diagonal.
```

In [14]:

```
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices
s2m = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
I2 = sp.eye(2)


def blocks_of(Xm):
    """The 8 diagonal 2 x 2 blocks of V^dagger X V and whether nothing else is."""
    Yb = sp.simplify(V.H * Xm * V)
    diagonal = [Yb[2 * b:2 * b + 2, 2 * b:2 * b + 2] for b in range(8)]
```

```
    rebuilt = sp.zeros(16)
    for b in range(8):
        rebuilt[2 * b:2 * b + 2, 2 * b:2 * b + 2] = diagonal[b]
    return diagonal, rebuilt == Yb


expected = [  # (name, matrix, the block form as a function of (j, s2, s3))
    ("gamma^(x0)", g0, lambda j, a, b: s3, "sigma3"),
    ("gamma^(x0) gamma^(x1)", A1, lambda j, a, b: -sp.I * s2m, "-i sigma2"),
    ("gamma^(x0) gamma^(x4)", A4, lambda j, a, b: j * s1, "j sigma1"),
    ("B", B, lambda j, a, b: j * a * I2, "j s2"),
    ("C", C, lambda j, a, b: a * s2m, "s2 sigma2"),
    ("BC = -i gamma^(x4)", B * C, lambda j, a, b: j * s2m, "j sigma2"),
    ("gamma^(x4) gamma^(x1)", g4 * g1, lambda j, a, b: -j * s3, "-j sigma3"),
    ("B gamma^(x0)", B * g0, lambda j, a, b: j * a * s3, "j s2 sigma3"),
    ("J", J, lambda j, a, b: j * I2, "j"),
    ("K1", K1, lambda j, a, b: sp.I * a * I2, "i s2"),
    ("K2", K2, lambda j, a, b: sp.I * b * I2, "i s3"),
]
all_forms = True
for name, Xm, form, text in expected:
    diagonal, only_diagonal = blocks_of(Xm)
    ok = only_diagonal and all(diagonal[i] == form(*labels[i]) for i in range(8))
    all_forms &= ok
    say(f"{name:24} -> {text:12} in block (j, s2, s3): {ok}")
check(all_forms and record_check("blocks_forms"),
      "all eleven matrices are block diagonal with the forms of the record",
      record=f"{PY_REPORT}, check blocks_forms")
not_diagonal = (not blocks_of(g2)[1] and not blocks_of(g5)[1]
                and not blocks_of(g0 * g2)[1] and blocks_of(g1)[1])
check(not_diagonal and record_check("blocks_not_everything_block_diagonal"),
      "gamma^(x2), gamma^(x5), gamma^(x0)gamma^(x2) connect blocks; gamma^(x1) not",
      record=f"{PY_REPORT}, check blocks_not_everything_block_diagonal")
```

Out [14]:

```
gamma^(x0)               -> sigma3       in block (j, s2, s3): True
gamma^(x0) gamma^(x1)    -> -i sigma2    in block (j, s2, s3): True
gamma^(x0) gamma^(x4)    -> j sigma1     in block (j, s2, s3): True
B                        -> j s2         in block (j, s2, s3): True
C                        -> s2 sigma2    in block (j, s2, s3): True
BC = -i gamma^(x4)       -> j sigma2     in block (j, s2, s3): True
gamma^(x4) gamma^(x1)    -> -j sigma3    in block (j, s2, s3): True
B gamma^(x0)             -> j s2 sigma3  in block (j, s2, s3): True
J                        -> j            in block (j, s2, s3): True
K1                       -> i s2         in block (j, s2, s3): True
K2                       -> i s3         in block (j, s2, s3): True
PASS all eleven matrices are block diagonal with the forms of the record
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check blocks_forms
PASS gamma^(x2), gamma^(x5), gamma^(x0)gamma^(x2) connect blocks; gamma^(x1) not
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    blocks_not_everything_block_diagonal
```

Text cell:

```
The next cell draws the block structure. To have one matrix of the equation
without derivatives, it uses the matrix $N_{16}$ of the first-order form
$\chi' = N_{16}\chi$ of the stationary equation at one point: from section 9 with
$\partial_{x_4}\chi = -i\varepsilon\chi$ and $\mathbf k = (k,0,0)$ one gets
$N_{16} = M\gamma^{(x_0)} + i(\varepsilon - v)\gamma^{(x_0)}\gamma^{(x_4)}
- i\kappa k\,\gamma^{(x_0)}\gamma^{(x_1)}$. The cell takes the sample values
$M = 1$, $\varepsilon = 0.5$, $v = 0$, $\kappa k = 0.7$ and shows the absolute
values of the entries of $N_{16}$ in the original basis (left), of
$V^\dagger N_{16}V$ in the block basis (middle; the thin lines mark the eight
$2\times2$ blocks), and of $V^\dagger\Gamma V$ (right): the chirality matrix is
NOT block diagonal, it connects each block to the block with the opposite $j$
(section 13).
```

In [15]:

```
N16 = (1 * g0 + sp.I * sp.Rational(1, 2) * g0 * g4
       - sp.I * sp.Rational(7, 10) * g0 * g1)  # M = 1, eps = 1/2, v = 0, kappa k = 7/10
pictures = [
    (np.abs(np.array(N16.evalf(), dtype=complex)), "$|N_{16}|$, original basis"),
    (np.abs(np.array(sp.simplify(V.H * N16 * V).evalf(), dtype=complex)),
     "$|V^\\dagger N_{16} V|$, block basis"),
    (np.abs(np.array(sp.simplify(V.H * Gamma * V).evalf(), dtype=complex)),
     "$|V^\\dagger \\Gamma V|$: pairs of blocks"),
]
fig, axes = plt.subplots(1, 3, figsize=(11.5, 3.9))
fig.subplots_adjust(wspace=0.45)  # room between the panels for the colour bars
for ax, (values, title) in zip(axes, pictures):
    image = ax.imshow(values, cmap="Blues", vmin=0.0, interpolation="nearest")
    ax.set_title(title, fontsize=9)
    ax.set_xticks([0, 5, 10, 15])
    ax.set_yticks([0, 5, 10, 15])
    ax.set_xlabel("column")
    ax.set_ylabel("row")
    ax.grid(False)  # no grid lines over the matrix
    fig.colorbar(image, ax=ax, shrink=0.75)
for ax in axes[1:]:  # mark the 2 x 2 blocks in the block basis
    for edge in np.arange(1.5, 15.0, 2.0):
        ax.axhline(edge, color="gray", linewidth=0.4)
        ax.axvline(edge, color="gray", linewidth=0.4)
save_figure(fig, "block_structure",
            "Absolute values of the matrix entries (row and column 0 to 15, "
            "darker is larger) of the first-order matrix $N_{16}$ of the "
            "stationary equation at $M = 1$, $\\varepsilon = 0.5$, $v = 0$, "
            "$\\kappa k = 0.7$: left in the original spinor basis, where the "
            "entries are spread over the matrix; middle in the block basis $V$, "
            "where only the eight $2\\times2$ blocks on the diagonal are nonzero "
            "(thin lines mark the blocks); right the chirality matrix $\\Gamma$ in "
            "the block basis, which connects each block $(j, s_2, s_3)$ with the "
            "block $(-j, s_2, s_3)$ four positions away.")
```

Out [15]:

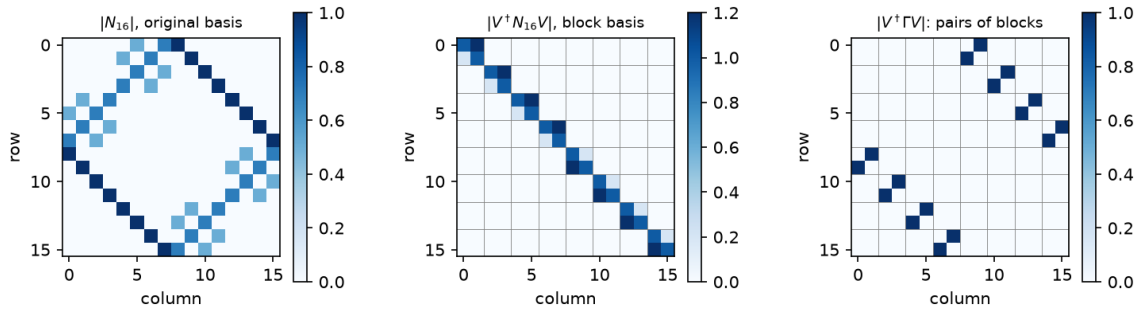

Figure 387: Absolute values of the matrix entries (row and column 0 to 15, darker is larger) of the first-order matrix N_{16} of the stationary equation at $M = 1$, $\varepsilon = 0.5$, $v = 0$, $\kappa k = 0.7$: left in the original spinor basis, where the entries are spread over the matrix; middle in the block basis V, where only the eight 2×2 blocks on the diagonal are nonzero (thin lines mark the blocks); right the chirality matrix Γ in the block basis, which connects each block (j, s_2, s_3) with the block $(-j, s_2, s_3)$ four positions away. (Notebook 14a, figure 4.)

```
Figure 14a.4 saved as Revision/textbook/figures/14a_4_block_structure.png
```

Text cell:

```
## 13. The 2x2 block Hamiltonian, its two types and the matrix Gamma

With the forms of section 12 the Hamiltonian of section 9 becomes, in every
block of type $j$ ($\mathbf k = (k, 0, 0)$):

$$h_j = j\left[-i\sigma_1\frac{d}{dy} + M\sigma_2 + \kappa k\,\sigma_3\right] + v .$$

It depends on $j$ only, not on $s_2, s_3$: there are only TWO types of block,
four blocks of each. The eigenvalue equation $h_j\chi = \varepsilon\chi$ is the
same as the first-order system $\chi' = N\chi$ with
$N = M\sigma_3 - \kappa k\sigma_2 + ij(\varepsilon - v)\sigma_1$. Two exact
relations follow:

- $h_{-1} - v = -(h_{+1} - v)$: the two types have mirrored spectra (for $v = 0$
  the levels of $j = -1$ are the negatives of those of $j = +1$); and
  $\sigma_3 h_j(k)\sigma_3 = h_{-j}(-k)$: the orbital $\sigma_3\chi$ of
  $(-j, -k)$ has the same level as the orbital $\chi$ of $(j, k)$.
- $\sigma_2 h_j(M, k)\sigma_2 = h_{-j}(-M, k)$. This is the block form of the
  chirality MATRIX $\Gamma$: $\Gamma$ maps the block $(j, s_2, s_3)$ onto
  $(-j, s_2, s_3)$ with the $2\times2$ entry $s_2\sigma_2$, so it maps a solution
  with mass $M$ in a block of type $j$ onto a solution with mass $-M$ in a block
  of type $-j$, at the same $k$ and the same level. It also exchanges the two
  brane conditions $\chi_2(0) = 0$ and $\chi_1(0) = 0$. This is the mass-reversing
  matrix map of the pairing theorem T1. (The gammas are real; for a real field
  plain complex conjugation does nothing and is not a charge conjugation. The
  maps of the theory are MATRICES such as $\Gamma$.)

The next cell checks the block Hamiltonian in all eight blocks, the equivalence
with $\chi' = N\chi$, the two type relations and the block form of $\Gamma$.
```

In [16]:

```
ok_h = True
for i, (j, a, b) in enumerate(labels):  # the three pieces of h in block i
    Vb = V[:, 2 * i:2 * i + 2]
    ok_h &= sp.simplify(Vb.H * (sp.I * g4 * g0) * Vb + sp.I * j * s1).is_zero_matrix
    ok_h &= sp.simplify(Vb.H * (-g4 * g1) * Vb - j * s3).is_zero_matrix
    ok_h &= sp.simplify(Vb.H * (-sp.I * g4) * Vb - j * s2m).is_zero_matrix
check(ok_h and record_check("block_hamiltonian"),
      "h_j = j[-i sigma1 d/dy + M sigma2 + kappa k sigma3] + v in all 8 blocks",
      record=f"{PY_REPORT}, check block_hamiltonian")
eps = sp.symbols("epsilon", real=True)  # the level


def h_algebraic(j, M_, k_, kap_):
    """The part of h_j - v without the derivative."""
    return j * (M_ * s2m + kap_ * k_ * s3)


def N_matrix(j, M_, k_, eps_, v_, kap_):
    """N of d chi/dy = N chi."""
    return M_ * s3 - kap_ * k_ * s2m + sp.I * j * (eps_ - v_) * s1


kk = sp.symbols("k", real=True)
# h chi = eps chi  <=>  -i j sigma1 chi' = (eps - v - j(M s2 + kappa k s3)) chi
ok_ode = all(sp.simplify(-sp.I * j * s1 * N_matrix(j, M, kk, eps, v, kap)
                         - ((eps - v) * I2 - h_algebraic(j, M, kk, kap)))
             .is_zero_matrix for j in (1, -1))
check(ok_ode and record_check("block_ode_equivalent"),
      "h_j chi = eps chi is the same as d chi/dy = N chi",
      record=f"{PY_REPORT}, check block_ode_equivalent")
d_term = -sp.I * s1  # the matrix in front of d/dy (times j)
ok_types = ((h_algebraic(1, M, kk, kap) + h_algebraic(-1, M, kk, kap))
            .is_zero_matrix
            and (s3 * h_algebraic(1, M, kk, kap) * s3
                 - h_algebraic(-1, M, -kk, kap)).is_zero_matrix
            and (s3 * d_term * s3 + d_term).is_zero_matrix)
check(ok_types and record_check("block_type_relation"),
      "h_(-1) - v = -(h_(+1) - v) and sigma3 h_j(k) sigma3 = h_(-j)(-k)",
      record=f"{PY_REPORT}, check block_type_relation")
YG = sp.simplify(V.H * Gamma * V)
pairs = [(a, b) for a in range(8) for b in range(8)
         if not YG[2 * a:2 * a + 2, 2 * b:2 * b + 2].is_zero_matrix]
say(f"Gamma connects the blocks (from, to): {pairs}")
ok_pairs = (pairs == [(0, 4), (1, 5), (2, 6), (3, 7), (4, 0), (5, 1), (6, 2), (7, 3)]
            and all(YG[2 * a:2 * a + 2, 2 * b:2 * b + 2] == labels[a][1] * s2m
                    for (a, b) in pairs)
            and (Gamma * J + J * Gamma).is_zero_matrix
            and (Gamma * K1 - K1 * Gamma).is_zero_matrix
            and (Gamma * K2 - K2 * Gamma).is_zero_matrix)
check(ok_pairs and record_check("blocks_relation_to_Gamma"),
      "Gamma maps block (j, s2, s3) onto (-j, s2, s3) with the entry s2 sigma2",
      record=f"{PY_REPORT}, check blocks_relation_to_Gamma")
```

```
ok_map = ((s2m * h_algebraic(1, M, kk, kap) * s2m
           - h_algebraic(-1, -M, kk, kap)).is_zero_matrix
          and (s2m * d_term * s2m + d_term).is_zero_matrix)
check(ok_map and record_check("block_Gamma_map"),
      "sigma2 h_j(M, k) sigma2 = h_(-j)(-M, k): Gamma reverses the mass",
      record=f"{PY_REPORT}, check block_Gamma_map")
```

Out [16]:

```
PASS h_j = j[-i sigma1 d/dy + M sigma2 + kappa k sigma3] + v in all 8 blocks
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check block_hamiltonian
PASS h_j chi = eps chi is the same as d chi/dy = N chi
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    block_ode_equivalent
PASS h_(-1) - v = -(h_(+1) - v) and sigma3 h_j(k) sigma3 = h_(-j)(-k)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    block_type_relation
Gamma connects the blocks (from, to): [(0, 4), (1, 5), (2, 6), (3, 7), (4, 0), (5, 1),
    (6, 2), (7, 3)]
PASS Gamma maps block (j, s2, s3) onto (-j, s2, s3) with the entry s2 sigma2
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    blocks_relation_to_Gamma
PASS sigma2 h_j(M, k) sigma2 = h_(-j)(-M, k): Gamma reverses the mass
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check block_Gamma_map
```

Text cell:

```
## 14. Rotations of 3-space: only |k| matters

A rotation by the angle $\theta$ in the $(x_1, x_2)$ plane acts on spinors with
the matrix $R = \cos(\theta/2) + \sin(\theta/2)\gamma^{(x_1)}\gamma^{(x_2)}$. Its
inverse is $\cos(\theta/2) - \sin(\theta/2)\gamma^{(x_1)}\gamma^{(x_2)}$: with
$G = \gamma^{(x_1)}\gamma^{(x_2)}$, $c = \cos(\theta/2)$, $s = \sin(\theta/2)$
and $G^2 = -1$ one gets $(c + sG)(c - sG) = c^2 - s^2G^2 = c^2 + s^2 = 1$.
The next cell checks this, and that $R$ commutes with $\gamma^{(x_0)}$,
$\gamma^{(x_4)}$, $B$ and $C$ (so it does not change the rest of $h$) and that it
turns $k\gamma^{(x_1)}$ into
$k(\cos\theta\,\gamma^{(x_1)} - \sin\theta\,\gamma^{(x_2)})$, the same momentum
in a rotated direction. So the levels depend on $|\mathbf k|$ only, and
$\mathbf k = (k, 0, 0)$ is no loss of generality.
```

In [17]:

```
theta = sp.symbols("theta", real=True)
R = sp.cos(theta / 2) * I16 + sp.sin(theta / 2) * g1 * g2  # the spinor rotation
R_inverse = sp.cos(theta / 2) * I16 - sp.sin(theta / 2) * g1 * g2  # its inverse
rotated = sp.simplify(R * (kk * g1) * R_inverse)
ok_rotation = (sp.simplify(R * R_inverse) == I16
               and sp.simplify(rotated - kk * (sp.cos(theta) * g1
                                               - sp.sin(theta) * g2)).is_zero_matrix
               and all((R * Xm - Xm * R).is_zero_matrix for Xm in (g0, g4, B, C)))
```

```
check(ok_rotation and record_check("rotation_invariance"),
      "a 3-space rotation commutes with g0, g4, B, C and rotates k: only |k| counts",
      record=f"{PY_REPORT}, check rotation_invariance")
```

Out [17]:

```
PASS a 3-space rotation commutes with g0, g4, B, C and rotates k: only |k| counts
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    rotation_invariance
```

Text cell:

```
## 15. The exact rescaling identity between slices

Look at where the slice $a_{4,0}$ enters the block Hamiltonian: only through
$\kappa k = e^{-Hy}\,(k\,e^{-a_{4,0}})$. Line by line:

1. The 3-space directions form a coordinate torus of size $\ell$; the allowed
   momenta are $\mathbf k = \Delta k\,(n_1, n_2, n_3)$ with integers $n_i$ and
   $\Delta k = 2\pi/\ell$ (the Revision solver uses $\Delta k = 0.25$).
2. At the slice $a_{4,0}$ the lattice $\Delta k\,\mathbf n$ therefore acts exactly
   like the lattice $\Delta k\,e^{-a_{4,0}}\,\mathbf n$ at the slice 0: every
   level and every orbital is the same.
3. Densities carry the factor $P = e^{-6Hy}/(\ell^3 v_t)$ ($v_t$ the coordinate
   volume of the extra times). The slice-0 problem with the spacing
   $\Delta k\,e^{-a}$ has the box size $\ell e^{a}$; choosing its extra-time
   volume $v_t e^{-3a}$ gives $(\ell e^a)^3 v_t e^{-3a} = \ell^3 v_t$: the SAME
   factor $P$, so the same proper densities.

Hence, exactly (for the free gas, with the interaction, at every temperature):
$\mathrm{KS}(a_{4,0}; \Delta k, v_t, \lambda) = \mathrm{KS}(0; \Delta k\,
e^{-a_{4,0}}, v_t e^{-3a_{4,0}}, \lambda)$. Equivalently, with the same box
$v_t$ and the coupling $\lambda e^{3a_{4,0}}$, with every proper density
multiplied by $e^{-3a_{4,0}}$ (the coupling enters only as $\lambda$ times a
density). The next cell verifies the three statements with sympy and then
computes the partner spacings and extra-time volumes of the four later slices,
which the Rust solver used to solve every partner problem independently; they
must equal the columns `partner_dk` and `partner_v_t` of the Revision record
`rescaling.csv`.
```

In [18]:

```
a_sym, y_sym = sp.symbols("a y", real=True)
ell, v_t, lam = sp.symbols("ell v_t lambda", positive=True)


def h_slice(j, k_, a_):
    """The algebraic part of h_j at the slice a4,0 = a_ (kappa k with e^(-a))."""
    return j * (M * s2m + sp.exp(-H * y_sym - a_) * k_ * s3)
```

```
def proper_factor(ell_, v_t_):
    """The factor P = e^(-6Hy)/(ell^3 v_t) of the proper densities."""
    return sp.exp(-6 * H * y_sym) / (ell_ ** 3 * v_t_)


ok_h_slice = all(sp.simplify(h_slice(j, kk, a_sym)
                             - h_slice(j, kk * sp.exp(-a_sym), 0)).is_zero_matrix
                 for j in (1, -1))
ok_box = sp.simplify(proper_factor(ell, v_t) - proper_factor(
    ell * sp.exp(a_sym), v_t * sp.exp(-3 * a_sym))) == 0
ok_lambda = sp.simplify(lam * proper_factor(ell, v_t) - lam * sp.exp(3 * a_sym)
                        * proper_factor(ell * sp.exp(a_sym), v_t)) == 0
check(ok_h_slice and ok_box and ok_lambda and record_check("rescaling_identity"),
      "KS(a4,0; dk, v_t, lambda) = KS(0; dk e^(-a4,0), v_t e^(-3 a4,0), lambda)",
      record=f"{PY_REPORT}, check rescaling_identity")
rows = repository_file("Revision/kohn_sham/results/rescaling/rescaling.csv") \
    .read_text(encoding="utf-8").splitlines()
header = rows[0].split(",")
table = [dict(zip(header, row.split(","))) for row in rows[1:]]
worst = 0.0
for a in slices[1:]:
    dk_partner = 0.25 * np.exp(-a)  # the partner lattice spacing
    vt_partner = 1.0 * np.exp(-3 * a)  # the partner extra-time volume (v_t = 1)
    say(f"slice a4,0 = {a}: partner dk = {dk_partner:.12f}, "
        f"partner v_t = {vt_partner:.12f}")
    for row in table:
        if float(row["a4"]) == a:
            worst = max(worst, abs(float(row["partner_dk"]) / dk_partner - 1),
                        abs(float(row["partner_v_t"]) / vt_partner - 1))
check(len(table) == 60 and worst < 1e-14,
      "the partner spacings and volumes equal those of all 60 rows of the record",
      record="Revision/kohn_sham/results/rescaling/rescaling.csv, columns "
             "partner_dk and partner_v_t")
```

Out [18]:

```
PASS KS(a4,0; dk, v_t, lambda) = KS(0; dk e^(-a4,0), v_t e^(-3 a4,0), lambda)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    rescaling_identity
slice a4,0 = 0.5: partner dk = 0.151632664928, partner v_t = 0.223130160148
slice a4,0 = 1.0: partner dk = 0.091969860293, partner v_t = 0.049787068368
slice a4,0 = 1.5: partner dk = 0.055782540037, partner v_t = 0.011108996538
slice a4,0 = 2.0: partner dk = 0.033833820809, partner v_t = 0.002478752177
PASS the partner spacings and volumes equal those of all 60 rows of the record
     reproduces Revision/kohn_sham/results/rescaling/rescaling.csv, columns partner_dk
    and partner_v_t
```

Text cell:

```
The next cell draws the rescaling. Left: the coordinate momenta of the first
shells of the torus lattice, $|\mathbf k| = \Delta k\sqrt{n_1^2 + n_2^2 + n_3^2}$,
enter the equation as $|\mathbf k|\,e^{-a_{4,0}}$; on a logarithmic axis these are
```

```
parallel straight lines falling with slope $-1$ along the history: all
3-momenta are redshifted by the same factor. Right: the proper volumes at the
brane ($y = 0$) of the 3-space box, $(\ell e^{a_{4,0}})^3$, which grows, and of
the extra-time box, $v_t e^{-3a_{4,0}}$, which deflates; their product, the
proper 7-volume $\ell^3 v_t$, is the same at every slice, which is why the
proper densities are the same in the partner problem.
```

In [19]:

```
ell_value = 2 * np.pi / 0.25  # the coordinate box size of the solver (dk = 0.25)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
for n2 in (1, 2, 3, 4, 5, 6):  # the first shells n1^2 + n2^2 + n3^2
    left.semilogy(a4_values, 0.25 * np.sqrt(n2) * np.exp(-a4_values),
                  label=f"$n^2 = {n2}$")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("$|\\mathbf{k}|\\,e^{-a_{4,0}}$ (units of $H$, logarithmic)")
left.set_title("redshift of the lattice momenta")
left.legend(fontsize=8, ncol=2)
right.semilogy(a4_values, (ell_value * np.exp(a4_values)) ** 3,
               label="3-space box $(\\ell e^{a_{4,0}})^3$")
right.semilogy(a4_values, np.exp(-3 * a4_values), "--",
               label="extra-time box $v_t e^{-3a_{4,0}}$")
right.semilogy(a4_values, ell_value ** 3 * np.exp(3 * a4_values)
               * np.exp(-3 * a4_values), ":", color="black",
               label="7-volume $\\ell^3 v_t$ (constant)")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("proper volume at $y = 0$ (logarithmic)")
right.set_title("the boxes inflate and deflate, the 7-volume stays")
right.legend(fontsize=8)
save_figure(fig, "rescaling",
            "The exact rescaling identity. Left: the momenta of the first six "
            "lattice shells, $|\\mathbf{k}| e^{-a_{4,0}}$ with "
            "$|\\mathbf{k}| = 0.25\\sqrt{n^2}$ (units of $H$), against the slice "
            "$a_{4,0}$ from 0 to 2 on a logarithmic vertical axis: parallel "
            "straight lines, every 3-momentum is redshifted by the same factor "
            "$e^{-a_{4,0}}$. Right: the proper volume of the 3-space box "
            "$(\\ell e^{a_{4,0}})^3$ with $\\ell = 2\\pi/0.25$ (solid, growing), of "
            "the extra-time box $v_t e^{-3a_{4,0}}$ with $v_t = 1$ (dashed, "
            "deflating) and their product $\\ell^3 v_t$ (dotted, constant), all at "
            "the brane $y = 0$; a slice $a_{4,0}$ is therefore the slice 0 with "
            "the lattice spacing $0.25 e^{-a_{4,0}}$ and the same proper 7-volume.")
```

Out [19]:

```
Figure 14a.5 saved as Revision/textbook/figures/14a_5_rescaling.png
```

Text cell:

```
## 16. The last check

The last cell checks that the five figure files exist and prints the number of
checks that passed.
```

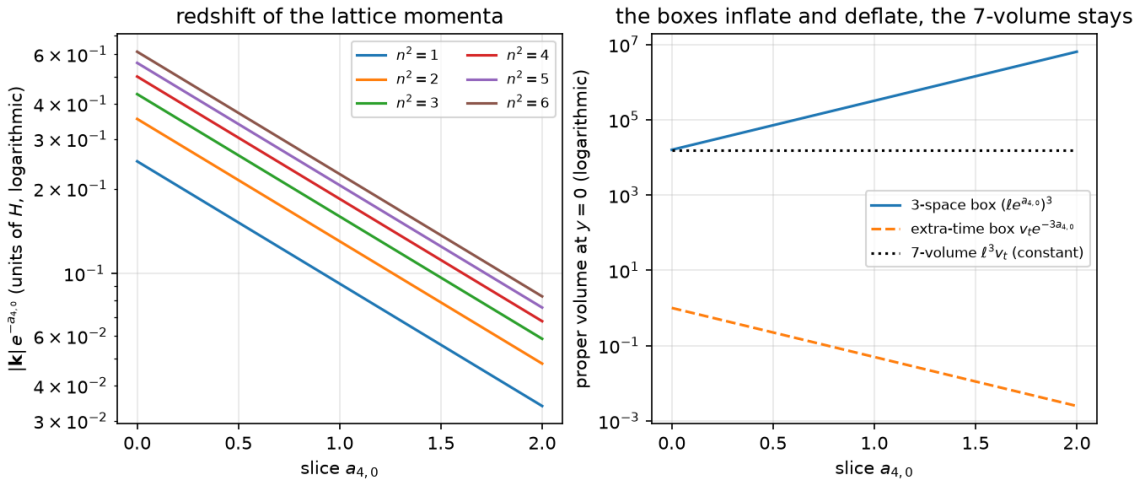

Figure 388: The exact rescaling identity. Left: the momenta of the first six lattice shells, $|\mathbf{k}|e^{-a_{4,0}}$ with $|\mathbf{k}| = 0.25\sqrt{n^2}$ (units of H), against the slice $a_{4,0}$ from 0 to 2 on a logarithmic vertical axis: parallel straight lines, every 3-momentum is redshifted by the same factor $e^{-a_{4,0}}$. Right: the proper volume of the 3-space box $(\ell e^{a_{4,0}})^3$ with $\ell = 2\pi/0.25$ (solid, growing), of the extra-time box $v_t e^{-3a_{4,0}}$ with $v_t = 1$ (dashed, deflating) and their product $\ell^3 v_t$ (dotted, constant), all at the brane $y = 0$; a slice $a_{4,0}$ is therefore the slice 0 with the lattice spacing $0.25e^{-a_{4,0}}$ and the same proper 7-volume. (Notebook 14a, figure 5.)

In [20]:

```
figure_names = ["14a_1_hidden_coordinate.png", "14a_2_inflation_deflation.png",
                "14a_3_spin_connection_terms.png", "14a_4_block_structure.png",
                "14a_5_rescaling.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all five figure files of this notebook exist")
all_checks_passed()
```

Out [20]:

```
PASS all five figure files of this notebook exist
ALL 28 CHECKS PASSED (notebook 14a)
```

Text cell:

```
## 17. What this notebook showed

- The hidden coordinate $y = \ln(\sin z)/(6H)$ turns the author's metric into the
  warped form with $W = e^{Hy}$; the 7-volume factor $W^6$ does not contain
  $a_4$ (PROVED; reproduces the Revision checks).
- In the Dirac operator the spin connection contributes only $3H\gamma^{(x_0)}$,
  for every history $a_4(x_4)$: the time-direction pieces $+\frac32 a_4'$ of the
  three inflating directions and $-\frac32 a_4'$ of the three deflating extra
  times cancel exactly (PROVED).
```

```
- The factor $W^{-3}$ removes this term; the equation becomes
  $i\partial_{x_4}\chi = h\chi$ with a Hermitian $h$ (PROVED).
- $J, K_1, K_2$ split the 16 components into eight blocks of two; the block
  basis equals the one in ‘Revision/kohn_sham/ks-theory.json‘ entry by entry; in
  each block $h_j = j[-i\sigma_1 d/dy + M\sigma_2 + \kappa k\sigma_3] + v$ with
  only two types $j = \pm1$ (PROVED).
- The chirality matrix $\Gamma$ pairs the block types and maps mass $M$ to $-M$:
  a matrix map, the one of the pairing theorem T1 (PROVED here at the level of the
  block equation).
- The slice of the prescribed deflating history enters only as
  $k\,e^{-a_{4,0}}$: a later slice is the slice 0 with redshifted lattice
  momenta and the same proper 7-volume (the exact rescaling identity, PROVED; the
  partner numbers equal those of the Rust solver’s record).
- ASSUMED: the good sector (no dependence on the extra times). PRESCRIBED
  BACKGROUND: the history $a_4 = AHx_4$.
```

14.11 Line-by-line walk-through of Notebook 14a

The notebook has 20 code cells, In [1] to In [20]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted, then explained. Docstrings (the texts in triple quotes under a `def` line, which say what a function does) and some comment lines are left out of the quotations; the complete cells are printed in Section 14.10.

In [1], the set-up cell. Every line that starts with `#` is a **comment**, which Python skips. The first part of the cell, down to the lines of `-` and `=` signs, is the complete run instructions of Section 14.9 again, as comments, so that the notebook file carries its own instructions. The code starts after the heading THE SET-UP; it computes no physics and is the same in every notebook of the book, except for the line that names the notebook. The notebooks 14b, 14c and 14d have exactly this cell with their own name, and their walk-throughs refer back to this paragraph.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of a package) so that the code can use it. `json`, `os`, `textwrap` and `pathlib` come with Python. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or a folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python code), which show a picture file below a cell.

```
NOTEBOOK_ID = "14a"  # this notebook: chapter 14, example a
```

A **variable** is a name for a value; this line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"14a"`. The figure files are named after it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. `Path.cwd()` is the folder in which Jupyter runs the notebook (the folder that holds it), and `.resolve()` writes it as a complete address. `here.parents` lists the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with them (the star unpacks one list into another). The `for` loop visits these folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains the file `Revision /textbook/requirements.txt` (the list of the book's packages) is the repository, and `return` hands it back. If no folder qualifies, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The comment lines say in short what this paragraph explains. The first statement calls the function and names its result `REPO`. It is never printed, because it differs from computer to computer, while the printed output of a notebook must not. The second line chooses where files are written. `os.environ` holds the **environment variables** of the program (named texts that it receives from the computer); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and the default `str(REPO)` (the repository folder as a string) otherwise. When you run the notebook the variable is not set, so the files go into the repository; the book's checking tool sets it to a scratch folder, so that a check never changes the repository.

```
def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions. `repository_file("Revision/...")` gives the full path of a repository file, for reading a Revision record. `output_file("Revision/...")` gives the full path at which to write

a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder that will hold it, together with any missing folder above it, and does nothing if the folder exists.

```
def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of the book; `textwrap.fill` breaks the text at blanks and starts every line after the first with four blanks. This is why some printed lines below continue on an indented second line.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

matplotlib reads personal settings from a file on your computer if you have one; `matplotlib.rcdefaults()` returns to the built-in settings, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets four settings for every figure: the size (7.0 by 4.2 inches), the size of the letters (10 points) and a faint grid of lines behind the curves (`grid.alpha` 0.3 means 30 per cent opaque). The braces make a **dictionary**: pairs of a key and a value, written `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: a name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/14a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty JSON dictionary `{}` and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored as bytes, and `newline="\n"` stores the same line end on Windows, macOS and Linux.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
```

```
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far: the figures are numbered 1, 2, 3, and a cell that is run twice keeps its number. The file name joins the notebook id, the number and the name, for example `14a_1_hidden_coordinate.png`. `fig.savefig` writes a PNG picture with 150 dots per inch, cuts away the empty margin (`bbox_inches="tight"`) and stores no program name (`metadata`), so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would otherwise draw it a second time at the end of the cell. The caption is stored, the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys, one item per line), `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved. The caption given to `save_figure` is the caption that the book prints under the figure.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection, written with square brackets). `check` is the function behind every PASS line: if the condition is false, `raise AssertionError(...)` stops the notebook with a message that names the check; otherwise the name is appended to the list and the line PASS and the name is printed. When the check repeats a number or a verdict of the Revision record, the argument `record` names the record file and its check, and a second line starting with `reproduces` is printed. (An `if` statement is used instead of Python's `assert`, because `python -O` would skip an `assert`.)

```
def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT (the provenance file of the notebook lists these lines). `x if c else y` is the value `x` when `c` is true and `y` otherwise, so the unit is added

only when one is given. `all_checks_passed` prints the last line of the notebook with the number of checks that passed; `len` is the length of a list. The last statement prints the only output of the cell, Out [1].

In [2], the gammas and the Clifford relation.

```
import numpy as np  # floating-point arrays and matrices
import sympy as sp  # exact algebra with symbols
```

numpy computes with **floating-point numbers** (numbers with about 16 significant digits, which the computer rounds after every operation); sympy computes **exactly** with whole numbers, fractions and symbols such as H. Every identity of this notebook is checked with sympy, so a PASS means an exact identity, not an approximation; numpy is used only for the figures.

```
PY_REPORT = "Revision/kohn_sham/reports/ks-theory-python.json"
WL_REPORT = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
```

The two reports of the Revision record of the Kohn-Sham theory: the sympy checker and the WolframScript verifier.

```
def record_check(name):
    verdicts = {}
    for report in (PY_REPORT, WL_REPORT):
        data = json.loads(repository_file(report).read_text(encoding="utf-8"))
        found = [c["verdict"] for c in data["checks"] if c["name"] == name]
        verdicts[report] = found[0] if found else "absent"
    return (verdicts[PY_REPORT] == "PASS"
            and verdicts[WL_REPORT] in ("PASS", "absent"))
```

`record_check(name)` reads both reports (`read_text` reads the file, `json.loads` turns the JSON text into Python lists and dictionaries) and looks for the check called `name` in the list `data["checks"]`. The bracketed expression is a **list comprehension**: it collects `c["verdict"]` for every check `c` whose name matches. If the check is found, its verdict (PASS or FAIL) is kept; otherwise the word absent. The function returns true when the sympy report says PASS and the Wolfram report says PASS or has no check of that name. Every check below that repeats a Revision check uses it, so the notebook passes only if the record itself passed.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
g = [sp.Matrix(m) for m in fixture["gamma"]]  # g[0] = gamma^(x0) ... g[7] = gamma^(x7)
eta = [int(v) for v in fixture["eta"]]  # +1 space-like, -1 time-like
g0, g1, g2, g3, g4, g5, g6, g7 = g  # gN is gamma^(xN)
I16, Z16 = sp.eye(16), sp.zeros(16)  # the 16 x 16 unit and zero matrices
```

The record `Revision/algebra/gammas.json` stores the eight matrices as lists of rows of whole numbers, in the order $x_0, \ldots, x_7$. `sp.Matrix(m)` turns each into an exact sympy matrix; `g[0]` is $\gamma^{(x_0)}$ and `g[7]` is $\gamma^{(x_7)}$ (Python counts from 0, like the author's coordinates). `eta` is the list of the eight signs. The third line gives the eight matrices the names `g0` to `g7` (unpacking a list into names), so `gN` is $\gamma^{(x_N)}$. `sp.eye(16)` is the unit matrix and `sp.zeros(16)` the zero matrix.

```
say(f"eta in the order x0 ... x7: {eta}")
entries = sorted({int(e) for m in g for e in m})  # the different entries
say(f"the entries of the eight gamma matrices are the integers {entries}")
```

The first line prints the signs. The second collects every entry of every matrix into a **set** (braces without colons: a collection in which each value occurs once) and sorts it; the printout shows that the only entries are -1, 0 and 1: the gammas are real.

```
clifford = all(g[a] * g[b] + g[b] * g[a] == (2 * eta[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
check(clifford and eta == [1, 1, 1, 1, -1, -1, -1, -1]
      and record_check("clifford_relation"),
      "the Clifford relation gamma^a gamma^b + gamma^b gamma^a = 2 eta^ab",
      record=f"{PY_REPORT}, check clifford_relation")
```

`range(8)` runs through 0 to 7, so the two `for` clauses visit all 64 pairs (a, b); `all(...)` is true when every comparison is true. For each pair the anticommutator $\gamma^a\gamma^b + \gamma^b\gamma^a$ (the operator `*` multiplies sympy matrices) is compared exactly with $2\eta^{aa}$ times the unit matrix when $a = b$ and with the zero matrix otherwise. The check also requires the signs $\eta = (+,+,+,+,-,-,-,-)$ and the recorded verdict; Out [2] shows its PASS line and the record it reproduces.

In [3], the matrices C, B and Γ.

```
C = g0 * g1 * g2 * g3  # the matrix of the Dirac adjoint
B = -sp.I * C * g4  # the matrix of the positive inner product
Gamma = g0 * g1 * g2 * g3 * g4 * g5 * g6 * g7  # the chirality matrix
```

The three products of Chapter 5; `sp.I` is the imaginary unit i.

```
check(C.T == C and C * C == I16 and B.H == B and B * B == I16
      and Gamma == sp.diag(*([-1] * 8 + [1] * 8)) and record_check("C_B_Gamma"),
      "C real symmetric with C^2 = 1, B Hermitian with B^2 = 1, Gamma = diag(-1, 1)",
      record=f"{PY_REPORT}, check C_B_Gamma")
```

`C.T` is the transpose and `B.H` the conjugate transpose. `[-1] * 8 + [1] * 8` is the list of eight entries -1 followed by eight entries $+1$, and `sp.diag(*...)` the diagonal matrix with these entries (the star hands the list over as separate numbers). So the check says: C is symmetric (it is real because the gammas are) with $C^2 = 1$; B is Hermitian with $B^2 = 1$; $\Gamma = \mathrm{diag}(-1_8, 1_8)$.

```
check(B * C == -sp.I * g4 and C * g4 == sp.I * B and C * B * C == B
      and record_check("BC_and_Cg4"),
      "BC = -i gamma^(x4), C gamma^(x4) = i B and C B C = B",
      record=f"{PY_REPORT}, check BC_and_Cg4")
```

Three product rules used in Section 14.5 and in Notebook 14d. They follow from the definitions: $C\gamma^{(x_4)} = iB$ is $B = -iC\gamma^{(x_4)}$ multiplied by i; $BC = -iC\gamma^{(x_4)}C = -iCC\gamma^{(x_4)} = -i\gamma^{(x_4)}$ because C commutes with $\gamma^{(x_4)}$ (a time-like gamma is not a factor of C, and C has four factors) and $C^2 = 1$; and $CBC = C(-i\gamma^{(x_4)}) = -iC\gamma^{(x_4)} = B$.

In [4], the hidden coordinate and the determinant.

```
H = sp.symbols("H", positive=True)  # the author's constant H > 0
y_neg = sp.symbols("y", negative=True)  # the hidden coordinate inside the patch
x0, z = sp.symbols("x0 z", positive=True)
```

`sp.symbols` makes symbols; the options tell sympy what it may assume: $H > 0$, $y < 0$ (inside the patch), $x_0 > 0$ and $z > 0$. With these assumptions sympy can simplify square roots and inverse sines correctly.

```
z_of_y = sp.asin(sp.exp(6 * H * y_neg))  # z(y): the solution of sin z = e^(6Hy)
dx0_dy = sp.diff(z_of_y / (6 * H), y_neg)  # x0 = z/(6H) differentiated in y
ok_line = sp.simplify(sp.cot(z_of_y) ** 2 * dx0_dy ** 2 - 1) == 0  # g00 dx0^2 = dy^2
```

The first line solves $\sin z = e^{6Hy}$ for z with the inverse sine, `sp.asin`. The second differentiates $x_0 = z/(6H)$ with respect to y (`sp.diff(expression, variable)`). The third checks step 2 of Section 14.2 in the form $\cot^2 z\,(dx_0/dy)^2 = 1$, which is $\cot^2 z\,dx_0^2 = dy^2$: `sp.simplify` brings the expression to its simplest form, and the comparison `== 0` asks whether it is exactly zero.

```
ok_warp = sp.simplify(sp.sin(z_of_y) ** sp.Rational(1, 3)
                      - sp.exp(2 * H * y_neg)) == 0  # sin^(1/3) z = e^(2Hy)
ok_dy = sp.simplify(sp.diff(sp.log(sp.sin(6 * H * x0)) / (6 * H), x0)
                    - sp.cot(6 * H * x0)) == 0  # dy/dx0 = cot z
check(ok_line and ok_warp and ok_dy and record_check("geometry_hidden_coordinate"),
      "y = ln(sin z)/(6H): dy = cot z dx0 and sin^(1/3) z = e^(2Hy)",
      record=f"{PY_REPORT}, check geometry_hidden_coordinate")
```

`sp.Rational(1, 3)` is the exact fraction 1/3 (the Python expression `1/3` would be a rounded floating-point number). The first test is step 3 of Section 14.2, $\sin^{1/3} z = e^{2Hy}$; the second is step 1, $dy/dx_0 = \cot z$, differentiating the definition $y = \ln(\sin 6Hx_0)/(6H)$ directly. The check combines the three tests.

```
a0 = sp.symbols("a0", real=True)  # the value of a4 at one slice
third = sp.Rational(1, 3)
metric_x0 = ([sp.exp(2 * a0) * sp.sin(z) ** third] * 3 + [-1]
             + [-sp.exp(-2 * a0) * sp.sin(z) ** third] * 3 + [sp.cot(z) ** 2])
det_x0 = sp.simplify(sp.Abs(sp.prod(metric_x0)))  # |det g| in the x0 chart
say(f"|det g| in the x0 chart of the author = {det_x0}")
check(sp.simplify(det_x0 - sp.cos(z) ** 2) == 0 and record_check("geometry_sqrt_det"),
      "|det g| = cos^2 z in the x0 chart; it does not contain a4",
      record=f"{PY_REPORT}, check geometry_sqrt_det")
```

`metric_x0` is the list of the eight diagonal entries of the author's metric in his x_0 chart (a list times 3 repeats it three times; `+` joins lists). The determinant of a diagonal matrix is the product of its diagonal entries (`sp.prod`), and `sp.Abs` takes its absolute value. sympy prints `cos(z)**2` (Out [4]; `**` is the power): the cubes of the 3-space and extra-time entries give $e^{6a_0}\sin z$ and $-e^{-6a_0}\sin z$, the factors e^{6a_0} and e^{-6a_0} cancel, the signs $(-1)(-1) = 1$, and $\sin z \cdot \sin z \cdot \cot^2 z = \cos^2 z$. The square root is $\cos z$, as Section 14.2 says.

In [5], figure 1.

```
z_values = np.linspace(1e-3, np.pi / 2, 600)  # z from almost 0 to pi/2
y_of_z = np.log(np.sin(z_values)) / 6.0  # y = ln(sin z)/(6H) with H = 1
y_values = np.linspace(-3.0, 0.0, 301)  # the hidden coordinate from -L to 0
```

`np.linspace(start, stop, count)` makes `count` equally spaced numbers from `start` to `stop`; `1e-3` means 10^{-3} (the value $z = 0$ itself would give $\ln 0$). numpy applies `np.log` and `np.sin` to every number of an array at once. The third line is the hidden coordinate from the cutoff $-L = -3$ of the Revision solver to the brane.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(z_values, y_of_z, color="C0")
left.set_xlabel("$z = 6 H x_0$ (radians)")
left.set_ylabel("$y = \\ln(\\sin z)/(6H)$")
left.set_title("the hidden coordinate $y$ ($H = 1$)")
left.annotate("tip: $y \\to -\\infty$", (0.03, -1.0), fontsize=9)
left.annotate("brane: $y = 0$", (1.05, -0.15), fontsize=9)
```

`plt.subplots(1, 2, ...)` makes a figure with one row of two panels (called **axes** in matplotlib), named `left` and `right`, 10 by 4 inches. `plot(x, y)` draws a curve; `"C0"` is the first colour of matplotlib's standard list. The labels are written in LaTeX between dollar signs, with a doubled backslash because a single backslash starts a special character in a Python string. `annotate(text, (x, y))` writes a text at a point of the panel.

```
right.semilogy(y_values, np.exp(y_values), label="warp $W = e^{Hy}$")
right.semilogy(y_values, np.exp(2 * y_values), "--",
               label="$\\sin^{1/3} z = W^2$")
right.semilogy(y_values, np.exp(6 * y_values), ":",
               label="volume factor $\\sqrt{|g|} = W^6$")
right.set_xlabel("$y$ (units of $1/H$)")
right.set_ylabel("factor (logarithmic scale)")
right.set_title("the factors of the warped metric")
right.legend(fontsize=8)
```

`semilogy` draws with a **logarithmic vertical axis**: equal distances mean equal factors (each tick is ten times the one below). On such an axis an exponential e^{cy} is a straight line of slope c. The three curves are W, W^2 and W^6 with $H = 1$; the third argument of `semilogy`, a short text called a **format string**, sets the line style: two hyphens draw a dashed line and a colon a dotted one (the plain `plot` lines of this book use the same format strings); `legend` shows the `label` texts in a box.

```
save_figure(fig, "hidden_coordinate",
            "Left: the hidden coordinate $y = \\ln(\\sin z)/(6H)$ against "
            ...)
```

The figure is saved as `14a_1_hidden_coordinate.png` with its caption (the long caption, printed under the figure in Section 14.10, is shortened here to `...`). **What figure 14a.1 shows.** Left: y against z; the curve drops steeply to minus infinity as $z \to 0$ (the tip; the plot stops at $z = 10^{-3}$, $y = -1.15$) and approaches 0 with zero slope at the brane $z = \pi/2$ (because $dy/dz = \cot z/(6H)$ vanishes there). Right: the three straight lines with slopes 1, 2 and 6 on the logarithmic axis; all

equal 1 at the brane, and at the cutoff $y = -3$ the volume factor is $e^{-18} \approx 1.5 \times 10^{-8}$. This small proper volume near the tip is why the proper densities of Notebook 14d become large there.

In [6], figure 2.

```
a4_values = np.linspace(0.0, 2.0, 201)  # a4 along the history (= x4 for A = H = 1)
slices = [0.0, 0.5, 1.0, 1.5, 2.0]  # the five slices of the Revision solver
```

The values of a_4 from 0 to 2 (along the history $a_4 = AHx_4$ with $A = H = 1$ these are also the times x_4), and the five slices of the Revision runs.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
left.plot(a4_values, np.exp(a4_values), label="3-space: $e^{a_4}$ (inflates)")
left.plot(a4_values, np.exp(-a4_values), "--",
          label="extra times: $e^{-a_4}$ (deflates)")
left.plot(a4_values, np.exp(3 * a4_values) * np.exp(-3 * a4_values), ":",
          color="black", label="7-volume: $e^{3a_4} e^{-3a_4} = 1$")
left.set_xlabel("$a_4 = A H x_4$ (with $A = H = 1$: the time $x_4$)")
left.set_ylabel("scale factor at the brane $y = 0$")
left.set_title("inflation and deflation along the history")
left.legend(fontsize=8)
```

The left panel draws the 3-space scale factor e^{a_4}, the extra-time scale factor e^{-a_4} and the product of their cubes, all at the brane where $W = 1$.

```
for a in slices:  # one curve per slice
    right.semilogy(y_values, np.exp(-y_values - a), label=f"$a_{{4,0}} = {a}$")
right.set_xlabel("$y$ (units of $1/H$)")
right.set_ylabel("$\\kappa = e^{-Hy - a_{4,0}}$ (logarithmic)")
right.set_title("the momentum weight at five slices")
right.legend(fontsize=8)
save_figure(fig, "inflation_deflation",
            ...)
```

The loop draws the momentum weight $\kappa = e^{-y-a}$ ($H = 1$) for each slice a; in an f-string a doubled brace `{{` prints one brace, so the label reads $a_{4,0} = 0.5$ and so on. **What figure 14a.2 shows.** Left: e^{a_4} grows to $e^2 \approx 7.4$, e^{-a_4} falls to $e^{-2} \approx 0.14$, and the dotted line stays at 1: inflation of 3-space and deflation of the extra times exactly balance in the volume. Right: five parallel straight lines falling from left to right: κ is largest at the tip ($e^3 \approx 20$ at $y = -3$ for the slice 0), and each later slice lies lower by the factor $e^{-0.5}$: the same coordinate momentum costs less energy as 3-space inflates.

In [7], the spin connection. This cell carries out Steps 1 to 3 of Section 14.4 with sympy.

```
X = (sp.Symbol("y", real=True),) + sp.symbols("x1:8", real=True)  # y, x1..x7
Y, x4 = X[0], X[4]  # the hidden coordinate and the time
a4 = sp.Function("a4", real=True)(x4)  # a4 is any function of the time x4
a4p = sp.diff(a4, x4)  # its derivative a4'
```

`sp.symbols("x1:8")` makes the seven symbols $x_1, \ldots, x_7$; a one-element **tuple** (a list in round brackets) with the symbol y put in front of them gives the eight coordinates, with y in the place of x_0 (position 0), so `X[k]` is x_k for $k \geq 1$. The frame factors `vb` are in the same order: 1 for y, $e^{a_4}W$ for x_1, x_2, x_3, 1 for x_4 and $e^{-a_4}W$ for the extra times (the metric on the background $b_4 = 1$; the author's metric of 2026-10-14 has the factor $b_4^{1/2}$ in the extra times, Chapter 3). `sp.Function("a4")(x4)` is an unknown function $a_4(x_4)$, so everything below holds along every history; `a4p` is its derivative a_4'.

```
W = sp.exp(H * Y)  # the warp factor
vb = [sp.Integer(1)] + [sp.exp(a4) * W] * 3 + [sp.Integer(1)] + [sp.exp(-a4) * W] * 3
gdiag = [eta[i] * vb[i] ** 2 for i in range(8)]  # the diagonal metric entries
```

`vb` is the list of the eight scale factors h of Section 14.2 (`sp.Integer(1)` is the exact number 1), and `gdiag` the metric entries $\eta_{ii}h_i^2$, which reproduce the warped form.

```
def christoffel(n, m, k):
    total = 0
    if m == n:
        total += sp.diff(gdiag[n], X[k])
    if k == n:
        total += sp.diff(gdiag[n], X[m])
    if m == k:
        total -= sp.diff(gdiag[m], X[n])
    return sp.simplify(total / (2 * gdiag[n]))
```

Step 1 of Section 14.4, the formula for ${\Gamma^n}_{mk}$ of a diagonal metric: each `if` adds one of the three terms when its Kronecker delta is 1 (`+=` adds to a variable, `-=` subtracts).

```
Chr = [[[christoffel(n, m, k) for k in range(8)] for m in range(8)]
       for n in range(8)]
S = [[(g[a] * g[b] - g[b] * g[a]) / 4 for b in range(8)] for a in range(8)]
```

`Chr[n][m][k]` holds all $8^3 = 512$ Christoffel symbols, and `S[a][b]` the 64 generators $S^{ab} = \frac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$.

```
Omega = []  # Omega[mu] is the 16 x 16 matrix Omega_mu
for mu in range(8):
    total = sp.zeros(16)
    for A in range(8):
        for Bi in range(8):
            w = (vb[A] * sp.diff(1 / vb[Bi], X[mu]) if A == Bi else 0) \
                + vb[A] * Chr[A][mu][Bi] / vb[Bi]  # omega_mu^A_B (step 2)
            w = sp.simplify(w)
            if w != 0:
                total += sp.Rational(1, 2) * eta[A] * w * S[A][Bi]  # step 3
    Omega.append(sp.simplify(total))
```

For each direction μ the double loop over the frame indices A and B (`Bi`, because `B` is the matrix B) computes the connection coefficient ${\omega_\mu}^A{}_B$ of Step 2 (a backslash at the end of a line continues

the statement on the next line) and adds $\frac{1}{2}\eta_{AA}\,\omega_\mu{}^A{}_B\,S^{AB}$ to the matrix Ω_μ (Step 3); terms with a zero coefficient are skipped to save time. `append` puts the finished Ω_μ at the end of the list.

```
compatible = all(
    (sp.diff(g[n] / vb[n], X[m])
     + sum((Chr[n][m][l] * g[l] / vb[l] for l in range(8)), Z16)
     + Omega[m] * g[n] / vb[n] - g[n] / vb[n] * Omega[m]).applyfunc(sp.simplify)
    .is_zero_matrix for m in range(8) for n in range(8))
check(compatible and record_check("spin_connection_compatibility"),
      "the curved gammas are covariantly constant (64 matrix equations)",
      record=f"{PY_REPORT}, check spin_connection_compatibility")
```

The proof that Ω_μ is the right connection: the **curved gammas** $\gamma^\nu = \gamma^{(\nu)}/h_\nu$ must be **covariantly constant**, $\partial_\mu\gamma^\nu + \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda + [\Omega_\mu, \gamma^\nu] = 0$, for all 64 pairs (μ, ν), where $[X, Y] = XY - YX$ is the **commutator**. `sum(..., Z16)` adds sixteen-by-sixteen matrices starting from the zero matrix; `applyfunc(sp.simplify)` simplifies every entry; `.is_zero_matrix` is true when every entry is 0.

```
slash = sp.simplify(sum((g[m] / vb[m] * Omega[m] for m in range(8)), Z16))
check((slash - 3 * H * g0).is_zero_matrix and record_check("spin_connection_slash_3H"),
      "gamma^mu Omega_mu = 3H gamma^(x0) exactly, for every history a4(x4)",
      record=f"{PY_REPORT}, check spin_connection_slash_3H")
```

The sum $\sum_\mu \gamma^\mu\Omega_\mu$ (often written with a slash through Ω, hence the name), compared with $3H\gamma^{(x_0)}$. Out [7] shows the two PASS lines.

In [8], where 3Hγ(x0) comes from.

```
names = ["x0 (y)", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]
PRIME = sp.Symbol("da4dx4")  # a name for the derivative da4/dx4 in the printout
c4_list, c0_list = [], []  # the coefficients of gamma^(x4) and gamma^(x0)
split_ok = True
```

Names for the printout, a symbol that stands for a_4' when it is printed, two empty lists and a flag that starts true.

```
for m in range(8):
    part = sp.simplify(g[m] / vb[m] * Omega[m])  # the contribution of direction m
    c4 = sp.simplify(-(part * g4).trace() / 16)
    c0 = sp.simplify((part * g0).trace() / 16)
    split_ok &= (part - c4 * g4 - c0 * g0).applyfunc(sp.simplify).is_zero_matrix
    c4_list.append(c4)
    c0_list.append(c0)
    shown_c4 = str(c4.subs(a4p, PRIME))  # write da4dx4 for da4/dx4
    say(f"direction {names[m]:7}: c4 = {shown_c4:12} c0 = {c0}")
check(split_ok, "every contribution is c4 gamma^(x4) + c0 gamma^(x0)")
```

For each direction the contribution $X = (\gamma^{(\mu)}/h_\mu)\Omega_\mu$ of Step 4. If $X = c_4\gamma^{(x_4)} + c_0\gamma^{(x_0)}$, then $X\gamma^{(x_4)} = -c_4 + c_0\gamma^{(x_0)}\gamma^{(x_4)}$, and since the **trace** (the sum of the diagonal entries) of the unit

matrix is 16 and that of a product of two different gammas is 0, $\mathrm{tr}(X\gamma^{(x_4)}) = -16c_4$; in the same way $\mathrm{tr}(X\gamma^{(x_0)}) = 16c_0$. These are the second and third lines. The fourth line checks that X is exactly $c_4\gamma^{(x_4)} + c_0\gamma^{(x_0)}$ (`&=` keeps the flag true only if every direction passes). `subs(a4p, PRIME)` replaces the derivative by the printable name, and the format `:7` and `:12` pads the texts to 7 and 12 characters, so that the printout forms columns. Out [8] shows $c_4 = a_4'/2$ and $c_0 = H/2$ for x_1, x_2, x_3; $c_4 = 0$, $c_0 = 0$ for x_4 and for y; $c_4 = -a_4'/2$ and $c_0 = H/2$ for x_5, x_6, x_7: exactly Step 5 of Section 14.4.

```
inflating = sp.simplify(sum(c4_list[1:4]))  # x1, x2, x3
deflating = sp.simplify(sum(c4_list[5:8]))  # x5, x6, x7
say("sum of c4 over the inflating directions x1, x2, x3: "
    f"{inflating.subs(a4p, PRIME)}")
say("sum of c4 over the deflating directions x5, x6, x7: "
    f"{deflating.subs(a4p, PRIME)}")
say(f"sum of c0 over all directions: {sp.simplify(sum(c0_list))}")
```

`c4_list[1:4]` is the part of the list from position 1 up to, but not including, position 4 (the directions x_1, x_2, x_3), and `c4_list[5:8]` the positions 5 to 7 (x_5, x_6, x_7). The printout: $3a_4'/2$, $-3a_4'/2$ and $3H$.

```
check(sp.simplify(inflating - sp.Rational(3, 2) * a4p) == 0
      and sp.simplify(deflating + sp.Rational(3, 2) * a4p) == 0
      and Omega[4].is_zero_matrix and Omega[0].is_zero_matrix
      and record_check("spin_connection_time_terms_cancel")
      and record_check("spin_connection_time_term_value"),
      "the da4/dx4 pieces +3/2 and -3/2 cancel; Omega_x4 = Omega_y = 0",
      record=f"{PY_REPORT}, checks spin_connection_time_terms_cancel and "
             "spin_connection_time_term_value")
```

The two facts of the record: the pieces $+\frac{3}{2}a_4'$ of the inflating and $-\frac{3}{2}a_4'$ of the deflating directions, and $\Omega_{x_4} = \Omega_y = 0$ (positions 3 and 7 of the list).

In [9], figure 3.

```
c4_numbers = [float(c.subs(a4p, 1).subs(H, 1)) for c in c4_list]  # da4/dx4 = H = 1
c0_numbers = [float(c.subs(a4p, 1).subs(H, 1)) for c in c0_list]
positions = np.arange(8)  # one group of two bars per direction
```

The coefficients become ordinary numbers by putting $a_4' = 1$ and $H = 1$ (`float` turns an exact number into a floating-point one); `np.arange(8)` is the array 0, 1, ..., 7, one position per direction.

```
fig, ax = plt.subplots(figsize=(8.0, 4.0))
ax.bar(positions - 0.2, c4_numbers, width=0.38,
       label="coefficient of $\\gamma^{(x_4)}$ (per unit $da_4/dx_4$)")
ax.bar(positions + 0.2, c0_numbers, width=0.38,
       label="coefficient of $\\gamma^{(x_0)}$ (per unit $H$)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(positions)
ax.set_xticklabels(["$x_0$", "$x_1$", "$x_2$", "$x_3$", "$x_4$", "$x_5$", "$x_6$",
                    "$x_7$"])
```

`bar(x, heights, width=...)` draws bars; the two sets are shifted by -0.2 and $+0.2$ so that they stand side by side. `axhline` draws the horizontal line at 0; `set_xticks` and `set_xticklabels` put the names of the directions under the groups.

```
ax.set_xlabel("direction $\\mu$ of the term $(\\gamma^{(\\mu)}/h_\\mu)\\,"
              "\\Omega_\\mu$")
ax.set_ylabel("coefficient")
ax.set_title("$\\gamma^\\mu\\Omega_\\mu = 3H\\gamma^{(x_0)}$: "
             "the $da_4/dx_4$ pieces cancel")
ax.legend(fontsize=8)
save_figure(fig, "spin_connection_terms",
            ...)
```

Labels, title, legend and saving. **What figure 14a.3 shows.** Above x_1, x_2, x_3 two bars of height $+\frac{1}{2}$; above x_5, x_6, x_7 a bar of height $-\frac{1}{2}$ (the coefficient of $\gamma^{(x_4)}$) and one of $+\frac{1}{2}$ (the coefficient of $\gamma^{(x_0)}$); nothing above x_4 and x_0. The six left bars of the $\gamma^{(x_4)}$ coefficient add to zero (inflation against deflation), the six bars of the $\gamma^{(x_0)}$ coefficient add to $3H$.

In [10], the factor W^(-3).

```
k1, k2, k3 = sp.symbols("k1 k2 k3", real=True)  # the 3-momentum
chi = sp.Matrix([sp.Function(f"chi{i}")(Y, x4) for i in range(16)])
phase = sp.exp(sp.I * (k1 * X[1] + k2 * X[2] + k3 * X[3]))  # the plane wave
kappa = sp.exp(-H * Y - a4)  # the momentum weight
```

The three components of $\mathbf{k}$; a column `chi` of 16 unknown functions $\chi_0(y, x_4), \ldots, \chi_{15}(y, x_4)$; the plane wave $e^{i\mathbf{k}\cdot\mathbf{x}}$; and the weight $\kappa = e^{-Hy - a_4(x_4)}$.

```
def dirac(Psi):
    result = sp.zeros(16, 1)
    for m in range(8):
        result += g[m] / vb[m] * (sp.diff(Psi, X[m]) + Omega[m] * Psi)
    return result
```

The Dirac operator $\gamma^\mu D_\mu \Psi = \sum_\mu (\gamma^{(\mu)}/h_\mu)(\partial_\mu \Psi + \Omega_\mu \Psi)$ with the spin connection of In [7]; `sp.zeros(16, 1)` is a zero column.

```
reduced = (g0 * sp.diff(chi, Y) + g4 * sp.diff(chi, x4)
           + sp.I * kappa * (k1 * g1 + k2 * g2 + k3 * g3) * chi)
with_W3 = sp.simplify(dirac(phase * W ** -3 * chi) / (phase * W ** -3))
without_W3 = sp.simplify(dirac(phase * chi) / phase)
```

`reduced` is the operator of Step 6 of Section 14.4 applied to χ. `with_W3` applies the full Dirac operator to $e^{i\mathbf{k}\cdot\mathbf{x}} W^{-3}\chi$ and divides by the common factor; `without_W3` does the same without W^{-3}.

```
check((with_W3 - reduced).applyfunc(sp.simplify).is_zero_matrix
      and record_check("ansatz_removes_spin_connection"),
      "with W^(-3): the reduced operator g0 d_y + g4 d_x4 + i kappa k.g, exactly",
```

```
      record=f"{PY_REPORT}, check ansatz_removes_spin_connection")
check((without_W3 - reduced - 3 * H * g0 * chi).applyfunc(sp.simplify)
      .is_zero_matrix and record_check("ansatz_without_W3_term_survives"),
      "without W^(-3) the term 3H gamma^(x0) chi survives",
      record=f"{PY_REPORT}, check ansatz_without_W3_term_survives")
```

With the factor the result equals the reduced operator exactly; without it, the difference is exactly $3H\gamma^{(x_0)}\chi$. The cell takes about 5 seconds.

In [11], the Hamiltonian and its Hermitian pieces.

```
M, v = sp.symbols("M v", real=True)  # effective mass and vector potential
chi_s = sp.Matrix(sp.symbols("c0:16"))  # chi at one point (16 symbols)
dchi_s = sp.Matrix(sp.symbols("d0:16"))  # d chi/dy at that point
kap = sp.symbols("kappa", positive=True)
kg = kap * (k1 * g1 + k2 * g2 + k3 * g3)  # kappa k.gamma
```

The derivation of Step 7 is pure algebra at one point, so χ and χ' are replaced by two columns of 16 independent symbols, and κ by a positive symbol.

```
# line 1 and 2: d chi/dx4 = -g4 [(M - i v g4) chi - g0 chi' - i kappa k.g chi]
dt_chi = -g4 * ((M * I16 - sp.I * v * g4) * chi_s - g0 * dchi_s - sp.I * kg * chi_s)
# line 3: h chi with the four pieces of h
h_chi = (sp.I * g4 * g0 * dchi_s - kap * g4 * (k1 * g1 + k2 * g2 + k3 * g3) * chi_s
         + M * (-sp.I * g4) * chi_s + v * chi_s)
check((sp.I * dt_chi - h_chi).expand().is_zero_matrix,
      "i d_x4 chi = h chi with h = i g4 g0 d_y - kappa k_j g4 g_j + M(-i g4) + v")
```

The two comment lines name the lines of Step 7 that the statement below each writes out. `dt_chi` is $\partial_{x_4}\chi$ from the first two lines of Step 7 (move the terms, multiply by $-\gamma^{(x_4)}$); `h_chi` is $h\chi$ with the four pieces of the third line. The check confirms $i\,\partial_{x_4}\chi = h\chi$ after multiplying out (`expand`).

```
hermitian = ((g4 * g0).T == g4 * g0 and all((-g4 * gj).H == -g4 * gj
                                            for gj in (g1, g2, g3))
             and (-sp.I * g4).H == -sp.I * g4 and (-sp.I * g4) ** 2 == I16)
check(hermitian and record_check("hamiltonian_16_hermitian"),
      "g4 g0 real symmetric; g4 g_j and -i g4 Hermitian; (-i g4)^2 = 1",
      record=f"{PY_REPORT}, check hamiltonian_16_hermitian")
```

The Hermitian pieces of Section 14.4: $\gamma^{(x_4)}\gamma^{(x_0)}$ symmetric, $-\gamma^{(x_4)}\gamma^{(x_j)}$ Hermitian for the three 3-space directions, $-i\gamma^{(x_4)}$ Hermitian with square 1.

In [12], the labels J, K1, K2 and the projectors.

```
A0, A1, A4 = g0, g0 * g1, g0 * g4  # the matrices of the equation (k along x1)
J, K1, K2 = g0 * g1 * g4, g2 * g3, g5 * g6
commute = all((P * Q - Q * P).is_zero_matrix for P in (J, K1, K2)
              for Q in (A0, A1, A4, B, C, J, K1, K2))
check(commute and J * J == I16 and K1 * K1 == -I16 and K2 * K2 == -I16
      and record_check("blocks_commuting_set"),
      "J, K1, K2 commute with A0, A1, A4, B, C and each other; J^2 = 1, K^2 = -1",
      record=f"{PY_REPORT}, check blocks_commuting_set")
```

The matrices of Section 14.5; `commute` tests the 24 commutators of each of J, K_1, K_2 with each of A_0, A_1, A_4, B, C, J, K_1, K_2; the check adds the three squares.

```
labels = [(j, s2, s3) for j in (1, -1) for s2 in (1, -1) for s3 in (1, -1)]
projectors = [(I16 + j * J) / 2 * (I16 - sp.I * s2 * K1) / 2
              * (I16 - sp.I * s3 * K2) / 2 for (j, s2, s3) in labels]
check(all(P.rank() == 2 and (P * P - P).is_zero_matrix for P in projectors)
      and sum(projectors, Z16) == I16 and record_check("blocks_projectors"),
      "eight projectors P(j, s2, s3) of rank 2 that add up to 1",
      record=f"{PY_REPORT}, check blocks_projectors")
```

`labels` lists the eight triples in the order of the record, $(1,1,1), (1,1,-1), \ldots, (-1,-1,-1)$. `projectors` builds $P(j, s_2, s_3)$ for each. The check: each has `rank()` 2 and $P^2 = P$, and the eight add up to the unit matrix.

In [13], the block basis.

```
columns, seeds = [], []
for P in projectors:
    P_plus = P * (I16 + g0) / 2  # project on gamma^(x0) = +1 inside the block
    c = next(c for c in range(16) if not P_plus[:, c].is_zero_matrix)
    seeds.append(c)
    v_plus = 8 * P_plus[:, c]
    columns += [v_plus, A1 * v_plus]  # v+ and v- = A1 v+
```

For each block: `P_plus` projects onto the block and onto $\gamma^{(x_0)} = +1$. Its columns are the images of the unit columns e_c (`P_plus[:, c]` is column c). `next(...)` returns the first c whose column is not zero, the **seed**. Then $v_+ = 8\,P_+ e_c$, and both v_+ and $v_- = A_1 v_+$ are added to the list (`+=` appends a list to a list).

```
U = sp.Matrix.hstack(*columns)  # 16 x 16, entries 0, +-1, +-i
V = U / (2 * sp.sqrt(2))  # the unitary block basis
say(f"seed columns c of the eight blocks: {seeds}")
```

`hstack` places the 16 columns side by side into the matrix U; $V = U/(2\sqrt{2})$. Out [13] prints the seeds 4, 0, 0, 4, 0, 4, 4, 0.

```
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
as_number = {"0": 0, "1": 1, "-1": -1, "I": sp.I, "-I": -sp.I}
U_record = sp.Matrix([[as_number[e] for e in row] for row in
                      theory["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
```

The record stores U row by row as texts such as `"I"` and `"-1"`; the dictionary `as_number` turns each text into the exact number, and `U_record` is the recorded matrix.

```
check(sp.simplify(V.H * V) == I16 and set(U) <= {0, 1, -1, sp.I, -sp.I}
      and seeds == theory["blockBasis"]["seedColumns0Based"]
```

```
      and record_check("blocks_basis_unitary"),
      "V = U/(2 sqrt 2) is unitary, U has entries 0, +-1, +-i",
      record=f"{PY_REPORT}, check blocks_basis_unitary")
check(U == U_record,
      "U equals the basis of the Revision record entry by entry",
      record="Revision/kohn_sham/ks-theory.json, blockBasis.unnormalisedColumns2Sqrt2V")
```

The first check: $V^\dagger V = 1$, the set of entries of U is contained in (`<=`) the set $\{0, \pm 1, \pm i\}$, and the seeds equal the recorded ones. The second: U equals the recorded matrix in all 256 entries. This is the basis the Rust solver uses.

In [14], the block forms.

```
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices
s2m = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
I2 = sp.eye(2)
```

The Pauli matrices (`s2m` is σ_2; the name `s2` is used for the label s_2) and the 2×2 unit matrix.

```
def blocks_of(Xm):
    Yb = sp.simplify(V.H * Xm * V)
    diagonal = [Yb[2 * b:2 * b + 2, 2 * b:2 * b + 2] for b in range(8)]
    rebuilt = sp.zeros(16)
    for b in range(8):
        rebuilt[2 * b:2 * b + 2, 2 * b:2 * b + 2] = diagonal[b]
    return diagonal, rebuilt == Yb
```

`blocks_of(X)` changes X to the block basis, $V^\dagger X V$, cuts out the eight 2×2 blocks on the diagonal (rows and columns $2b$ and $2b+1$ of block b), rebuilds a matrix from these blocks alone, and returns the blocks and whether the rebuilt matrix equals $V^\dagger X V$, that is, whether X is block diagonal.

```
expected = [  # (name, matrix, the block form as a function of (j, s2, s3))
    ("gamma^(x0)", g0, lambda j, a, b: s3, "sigma3"),
    ("gamma^(x0) gamma^(x1)", A1, lambda j, a, b: -sp.I * s2m, "-i sigma2"),
    ("gamma^(x0) gamma^(x4)", A4, lambda j, a, b: j * s1, "j sigma1"),
    ("B", B, lambda j, a, b: j * a * I2, "j s2"),
    ("C", C, lambda j, a, b: a * s2m, "s2 sigma2"),
    ("BC = -i gamma^(x4)", B * C, lambda j, a, b: j * s2m, "j sigma2"),
    ("gamma^(x4) gamma^(x1)", g4 * g1, lambda j, a, b: -j * s3, "-j sigma3"),
    ("B gamma^(x0)", B * g0, lambda j, a, b: j * a * s3, "j s2 sigma3"),
    ("J", J, lambda j, a, b: j * I2, "j"),
    ("K1", K1, lambda j, a, b: sp.I * a * I2, "i s2"),
    ("K2", K2, lambda j, a, b: sp.I * b * I2, "i s3"),
]
```

A list of eleven entries, one per line: the name, the matrix, its expected block form and the text to print. A `lambda` is a short nameless function: `lambda j, a, b:  j * s1` returns $j\sigma_1$ for the labels $(j, s_2, s_3) = (j, a, b)$; so the name `a` stands for s_2 and `b` for s_3 inside these lines (they are not the

real-form components of Section 14.12). Line by line the expected forms are: $\gamma^{(x_0)} \to \sigma_3$; $A_1 \to -i\sigma_2$; $A_4 \to j\sigma_1$; $B \to js_2$ times the 2×2 unit matrix `I2`; $C \to s_2\sigma_2$; $BC \to j\sigma_2$; $\gamma^{(x_4)}\gamma^{(x_1)} \to -j\sigma_3$; $B\gamma^{(x_0)} \to js_2\sigma_3$; $J \to j$; $K_1 \to is_2$; $K_2 \to is_3$. These are the eleven forms of Section 14.5; the two not derived there follow from them: $BC = (js_2)(s_2\sigma_2) = j\sigma_2$ because $s_2^2 = 1$, and $B\gamma^{(x_0)} = (js_2)\sigma_3$.

```
all_forms = True
for name, Xm, form, text in expected:
    diagonal, only_diagonal = blocks_of(Xm)
    ok = only_diagonal and all(diagonal[i] == form(*labels[i]) for i in range(8))
    all_forms &= ok
    say(f"{name:24} -> {text:12} in block (j, s2, s3): {ok}")
check(all_forms and record_check("blocks_forms"),
      "all eleven matrices are block diagonal with the forms of the record",
      record=f"{PY_REPORT}, check blocks_forms")
```

For each matrix: block diagonal, and in block i equal to the expected form at the labels of that block. Out [14] prints True for all eleven.

```
not_diagonal = (not blocks_of(g2)[1] and not blocks_of(g5)[1]
                and not blocks_of(g0 * g2)[1] and blocks_of(g1)[1])
check(not_diagonal and record_check("blocks_not_everything_block_diagonal"),
      "gamma^(x2), gamma^(x5), gamma^(x0)gamma^(x2) connect blocks; gamma^(x1) not",
      record=f"{PY_REPORT}, check blocks_not_everything_block_diagonal")
```

`blocks_of(X)[1]` is the second returned value, the yes-or-no answer. $\gamma^{(x_2)}$, $\gamma^{(x_5)}$ and $\gamma^{(x_0)}\gamma^{(x_2)}$ are not block diagonal; $\gamma^{(x_1)}$ is.

In [15], figure 4.

```
N16 = (1 * g0 + sp.I * sp.Rational(1, 2) * g0 * g4
       - sp.I * sp.Rational(7, 10) * g0 * g1)  # M = 1, eps = 1/2, v = 0, kappa k = 7/10
```

The first-order matrix of the stationary equation in 16 components, $N_{16} = M\gamma^{(x_0)} + i(\varepsilon - v)\gamma^{(x_0)}\gamma^{(x_4)} - i\kappa k\,\gamma^{(x_0)}\gamma^{(x_1)}$ (from Step 7 of Section 14.4 with $\partial_{x_4}\chi = -i\varepsilon\chi$, multiplied by $\gamma^{(x_0)}$), at sample values. With the block forms it becomes $N = M\sigma_3 - \kappa k\sigma_2 + ij(\varepsilon - v)\sigma_1$ in every block.

```
pictures = [
    (np.abs(np.array(N16.evalf(), dtype=complex)), "$|N_{16}|$, original basis"),
    (np.abs(np.array(sp.simplify(V.H * N16 * V).evalf(), dtype=complex)),
     "$|V^\\dagger N_{16} V|$, block basis"),
    (np.abs(np.array(sp.simplify(V.H * Gamma * V).evalf(), dtype=complex)),
     "$|V^\\dagger \\Gamma V|$: pairs of blocks"),
]
```

Three matrices to draw, each with its title: `evalf()` turns exact entries into decimal numbers, `np.array(..., dtype=complex)` into a numpy array of complex numbers, and `np.abs` takes the absolute value of every entry.

```
fig, axes = plt.subplots(1, 3, figsize=(11.5, 3.9))
fig.subplots_adjust(wspace=0.45)  # room between the panels for the colour bars
for ax, (values, title) in zip(axes, pictures):
    image = ax.imshow(values, cmap="Blues", vmin=0.0, interpolation="nearest")
    ax.set_title(title, fontsize=9)
    ax.set_xticks([0, 5, 10, 15])
    ax.set_yticks([0, 5, 10, 15])
    ax.set_xlabel("column")
    ax.set_ylabel("row")
    ax.grid(False)  # no grid lines over the matrix
    fig.colorbar(image, ax=ax, shrink=0.75)
```

`zip` pairs each panel with its picture. `imshow` draws a matrix as a **heat map**: one small square per entry, coloured by its value (`cmap="Blues"`: white for 0, dark blue for the largest value; `interpolation="nearest"` keeps the squares sharp). `colorbar` adds the scale of colours beside the panel.

```
for ax in axes[1:]:  # mark the 2 x 2 blocks in the block basis
    for edge in np.arange(1.5, 15.0, 2.0):
        ax.axhline(edge, color="gray", linewidth=0.4)
        ax.axvline(edge, color="gray", linewidth=0.4)
save_figure(fig, "block_structure",
            ...)
```

In the second and third panels thin gray lines are drawn between rows and columns 1 and 2, 3 and 4, and so on (`np.arange(1.5, 15.0, 2.0)` is 1.5, 3.5, ..., 13.5), which frame the eight blocks. **What figure 14a.4 shows.** Left: in the original basis the entries of N_{16} are spread over the whole matrix in a diamond pattern. Middle: in the block basis only the eight 2×2 blocks on the diagonal are nonzero. In every block the diagonal entries are $\pm M$ ($|1|$) and the two off-diagonal entries have the sizes $|\kappa k + j\varepsilon| = 1.2$ and $|j\varepsilon - \kappa k| = 0.2$; in the first four blocks ($j = +1$) the large one is above the diagonal, in the last four ($j = -1$) below it: the two block types are visible. Right: $V^{\dagger}\Gamma V$ has entries of size 1 only between the block b and the block $b \pm 4$, the pairs $(j, s_2, s_3) \leftrightarrow (-j, s_2, s_3)$.

In [16], the block Hamiltonian, its two types and Γ.

```
ok_h = True
for i, (j, a, b) in enumerate(labels):  # the three pieces of h in block i
    Vb = V[:, 2 * i:2 * i + 2]
    ok_h &= sp.simplify(Vb.H * (sp.I * g4 * g0) * Vb + sp.I * j * s1).is_zero_matrix
    ok_h &= sp.simplify(Vb.H * (-g4 * g1) * Vb - j * s3).is_zero_matrix
    ok_h &= sp.simplify(Vb.H * (-sp.I * g4) * Vb - j * s2m).is_zero_matrix
check(ok_h and record_check("block_hamiltonian"),
      "h_j = j[-i sigma1 d/dy + M sigma2 + kappa k sigma3] + v in all 8 blocks",
      record=f"{PY_REPORT}, check block_hamiltonian")
```

`enumerate` gives each block its number `i` together with its labels. `Vb` holds the two basis columns of the block. The three lines check the three matrix pieces of h (with **k** along x_1): $i\gamma^{(x_4)}\gamma^{(x_0)} \to -ij\sigma_1$, $-\gamma^{(x_4)}\gamma^{(x_1)} \to j\sigma_3$, $-i\gamma^{(x_4)} \to j\sigma_2$. Together they give h_j.

```
eps = sp.symbols("epsilon", real=True)  # the level
```

```
def h_algebraic(j, M_, k_, kap_):
    return j * (M_ * s2m + kap_ * k_ * s3)


def N_matrix(j, M_, k_, eps_, v_, kap_):
    return M_ * s3 - kap_ * k_ * s2m + sp.I * j * (eps_ - v_) * s1
```

The level ε as a symbol; `h_algebraic` is the part of $h_j - v$ without the derivative, $j(M\sigma_2 + \kappa k\sigma_3)$; `N_matrix` is the matrix N of the first-order form.

```
kk = sp.symbols("k", real=True)
# h chi = eps chi  <=>  -i j sigma1 chi' = (eps - v - j(M s2 + kappa k s3)) chi
ok_ode = all(sp.simplify(-sp.I * j * s1 * N_matrix(j, M, kk, eps, v, kap)
                         - ((eps - v) * I2 - h_algebraic(j, M, kk, kap)))
             .is_zero_matrix for j in (1, -1))
check(ok_ode and record_check("block_ode_equivalent"),
      "h_j chi = eps chi is the same as d chi/dy = N chi",
      record=f"{PY_REPORT}, check block_ode_equivalent")
```

`kk` is the momentum k as a real symbol. The comment line says what the check is about: the equation $h_j\chi = \varepsilon\chi$ reads $-ij\sigma_1\chi' = (\varepsilon - v)\chi - j(M\sigma_2 + \kappa k\sigma_3)\chi$ (the first line of the first-order form in Section 14.5; `<=>` means "is the same as"). If $\chi' = N\chi$, this holds for every χ exactly when $-ij\sigma_1 N = (\varepsilon - v) - j(M\sigma_2 + \kappa k\sigma_3)$, which is checked for both types.

```
d_term = -sp.I * s1  # the matrix in front of d/dy (times j)
ok_types = ((h_algebraic(1, M, kk, kap) + h_algebraic(-1, M, kk, kap))
            .is_zero_matrix
            and (s3 * h_algebraic(1, M, kk, kap) * s3
                 - h_algebraic(-1, M, -kk, kap)).is_zero_matrix
            and (s3 * d_term * s3 + d_term).is_zero_matrix)
check(ok_types and record_check("block_type_relation"),
      "h_(-1) - v = -(h_(+1) - v) and sigma3 h_j(k) sigma3 = h_(-j)(-k)",
      record=f"{PY_REPORT}, check block_type_relation")
```

The two type relations of Section 14.5: the algebraic parts of the two types add to zero; σ_3 turns the algebraic part of $(+1, k)$ into that of $(-1, -k)$; and $\sigma_3(-i\sigma_1)\sigma_3 = i\sigma_1$, so the derivative term changes sign as j does.

```
YG = sp.simplify(V.H * Gamma * V)
pairs = [(a, b) for a in range(8) for b in range(8)
         if not YG[2 * a:2 * a + 2, 2 * b:2 * b + 2].is_zero_matrix]
say(f"Gamma connects the blocks (from, to): {pairs}")
```

Γ in the block basis, and the list of the block pairs (a, b) between which it has a nonzero 2×2 entry. Out [16] prints the eight pairs $(0,4), (1,5), \ldots, (7,3)$.

```
ok_pairs = (pairs == [(0, 4), (1, 5), (2, 6), (3, 7), (4, 0), (5, 1), (6, 2), (7, 3)]
            and all(YG[2 * a:2 * a + 2, 2 * b:2 * b + 2] == labels[a][1] * s2m
```

```
                    for (a, b) in pairs)
            and (Gamma * J + J * Gamma).is_zero_matrix
            and (Gamma * K1 - K1 * Gamma).is_zero_matrix
            and (Gamma * K2 - K2 * Gamma).is_zero_matrix)
check(ok_pairs and record_check("blocks_relation_to_Gamma"),
      "Gamma maps block (j, s2, s3) onto (-j, s2, s3) with the entry s2 sigma2",
      record=f"{PY_REPORT}, check blocks_relation_to_Gamma")
```

Block b and block $b+4$ differ only in j (the order of `labels`); each entry is $s_2\sigma_2$ (`labels[a][1]` is the s_2 of block a); Γ anticommutes with J and commutes with K_1, K_2.

```
ok_map = ((s2m * h_algebraic(1, M, kk, kap) * s2m
           - h_algebraic(-1, -M, kk, kap)).is_zero_matrix
          and (s2m * d_term * s2m + d_term).is_zero_matrix)
check(ok_map and record_check("block_Gamma_map"),
      "sigma2 h_j(M, k) sigma2 = h_(-j)(-M, k): Gamma reverses the mass",
      record=f"{PY_REPORT}, check block_Gamma_map")
```

Section 14.6 line by line: σ_2 turns the algebraic part of $(j=1, M)$ into that of $(j=-1, -M)$ at the same k, and changes the sign of the derivative term.

In [17], rotations.

```
theta = sp.symbols("theta", real=True)
R = sp.cos(theta / 2) * I16 + sp.sin(theta / 2) * g1 * g2  # the spinor rotation
R_inverse = sp.cos(theta / 2) * I16 - sp.sin(theta / 2) * g1 * g2  # its inverse
rotated = sp.simplify(R * (kk * g1) * R_inverse)
```

The spinor rotation of Section 14.6, its inverse, and the rotated momentum term $R(k\gamma^{(x_1)})R^{-1}$.

```
ok_rotation = (sp.simplify(R * R_inverse) == I16
               and sp.simplify(rotated - kk * (sp.cos(theta) * g1
                                               - sp.sin(theta) * g2)).is_zero_matrix
               and all((R * Xm - Xm * R).is_zero_matrix for Xm in (g0, g4, B, C)))
check(ok_rotation and record_check("rotation_invariance"),
      "a 3-space rotation commutes with g0, g4, B, C and rotates k: only |k| counts",
      record=f"{PY_REPORT}, check rotation_invariance")
```

Three facts: $RR^{-1}=1$; $R\,k\gamma^{(x_1)}R^{-1} = k(\cos\theta\,\gamma^{(x_1)} - \sin\theta\,\gamma^{(x_2)})$; R commutes with $\gamma^{(x_0)}$, $\gamma^{(x_4)}$, B, C.

In [18], the rescaling identity.

```
a_sym, y_sym = sp.symbols("a y", real=True)
ell, v_t, lam = sp.symbols("ell v_t lambda", positive=True)


def h_slice(j, k_, a_):
    return j * (M * s2m + sp.exp(-H * y_sym - a_) * k_ * s3)
```

```
def proper_factor(ell_, v_t_):
    return sp.exp(-6 * H * y_sym) / (ell_ ** 3 * v_t_)
```

Symbols for the slice $a_{4,0}$ (`a_sym`), the hidden coordinate, the box size ℓ, the extra-time volume v_t and the coupling λ. `h_slice` is the algebraic part of h_j at a slice, with $\kappa = e^{-Hy-a}$; `proper_factor` is the factor P of the proper densities.

```
ok_h_slice = all(sp.simplify(h_slice(j, kk, a_sym)
                             - h_slice(j, kk * sp.exp(-a_sym), 0)).is_zero_matrix
                 for j in (1, -1))
ok_box = sp.simplify(proper_factor(ell, v_t) - proper_factor(
    ell * sp.exp(a_sym), v_t * sp.exp(-3 * a_sym))) == 0
ok_lambda = sp.simplify(lam * proper_factor(ell, v_t) - lam * sp.exp(3 * a_sym)
                        * proper_factor(ell * sp.exp(a_sym), v_t)) == 0
check(ok_h_slice and ok_box and ok_lambda and record_check("rescaling_identity"),
      "KS(a4,0; dk, v_t, lambda) = KS(0; dk e^(-a4,0), v_t e^(-3 a4,0), lambda)",
      record=f"{PY_REPORT}, check rescaling_identity")
```

The three statements of Section 14.7: the block Hamiltonian at the slice a with momentum k equals the one at the slice 0 with momentum ke^{-a}; the box $(\ell e^{a}, v_t e^{-3a})$ has the same factor P; and the coupling λe^{3a} times the density factor of the box $(\ell e^{a}, v_t)$ equals λ times the original factor.

```
rows = repository_file("Revision/kohn_sham/results/rescaling/rescaling.csv") \
    .read_text(encoding="utf-8").splitlines()
header = rows[0].split(",")
table = [dict(zip(header, row.split(","))) for row in rows[1:]]
```

The Rust solver's record of the partner problems, a **CSV file** (comma-separated values: a table as text, one row per line, the cells separated by commas, the first line the column names). `splitlines` cuts the text into lines and `split(",")` a line into cells; `dict(zip(header, cells))` makes a dictionary from column name to cell for every row.

```
worst = 0.0
for a in slices[1:]:
    dk_partner = 0.25 * np.exp(-a)  # the partner lattice spacing
    vt_partner = 1.0 * np.exp(-3 * a)  # the partner extra-time volume (v_t = 1)
    say(f"slice a4,0 = {a}: partner dk = {dk_partner:.12f}, "
        f"partner v_t = {vt_partner:.12f}")
    for row in table:
        if float(row["a4"]) == a:
            worst = max(worst, abs(float(row["partner_dk"]) / dk_partner - 1),
                        abs(float(row["partner_v_t"]) / vt_partner - 1))
check(len(table) == 60 and worst < 1e-14,
      "the partner spacings and volumes equal those of all 60 rows of the record",
      record="Revision/kohn_sham/results/rescaling/rescaling.csv, columns "
             "partner_dk and partner_v_t")
```

For each later slice (`slices[1:]` leaves out the slice 0) the partner spacing $0.25\,e^{-a}$ and the partner volume e^{-3a} are computed and printed with 12 decimals (the format `:.12f`). For every row of the record at that slice the relative difference is taken, and the largest is kept in `worst`. The record has 60 rows (three particle numbers, five couplings and four later slices) and agrees to better than 10^{-14}. Out [18] shows the partner numbers, for example 0.151632664928 and 0.223130160148 for $a_{4,0} = 0.5$.

In [19], figure 5.

```
ell_value = 2 * np.pi / 0.25  # the coordinate box size of the solver (dk = 0.25)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
for n2 in (1, 2, 3, 4, 5, 6):  # the first shells n1^2 + n2^2 + n3^2
    left.semilogy(a4_values, 0.25 * np.sqrt(n2) * np.exp(-a4_values),
                  label=f"$n^2 = {n2}$")
```

The box size $\ell = 2\pi/0.25$; the left panel draws, for the first six lattice shells ($|\mathbf{k}| = 0.25\sqrt{n^2}$), the redshifted momentum $|\mathbf{k}|e^{-a_{4,0}}$ against the slice.

```
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("$|\\mathbf{k}|\\,e^{-a_{4,0}}$ (units of $H$, logarithmic)")
left.set_title("redshift of the lattice momenta")
left.legend(fontsize=8, ncol=2)
right.semilogy(a4_values, (ell_value * np.exp(a4_values)) ** 3,
               label="3-space box $(\\ell e^{a_{4,0}})^3$")
right.semilogy(a4_values, np.exp(-3 * a4_values), "--",
               label="extra-time box $v_t e^{-3a_{4,0}}$")
right.semilogy(a4_values, ell_value ** 3 * np.exp(3 * a4_values)
               * np.exp(-3 * a4_values), ":", color="black",
               label="7-volume $\\ell^3 v_t$ (constant)")
```

Labels (`ncol=2` puts the legend in two columns), and in the right panel the proper volumes at the brane of the 3-space box, of the extra-time box and their product.

```
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("proper volume at $y = 0$ (logarithmic)")
right.set_title("the boxes inflate and deflate, the 7-volume stays")
right.legend(fontsize=8)
save_figure(fig, "rescaling",
            ...)
```

What figure 14a.5 shows. Left: six parallel straight lines of slope -1 on the logarithmic axis: every lattice momentum is redshifted by the same factor $e^{-a_{4,0}}$, so the ordering of the shells never changes. Right: the 3-space box grows from $(8\pi)^3 \approx 1.6 \times 10^4$ by the factor $e^6 \approx 403$, the extra-time box shrinks from 1 to $e^{-6} \approx 0.0025$, and their product, the dotted line, stays constant. This is the rescaling identity in a picture: a later slice is the slice 0 with a finer momentum lattice and the same proper box.

In [20], the last check.

```
figure_names = ["14a_1_hidden_coordinate.png", "14a_2_inflation_deflation.png",
                "14a_3_spin_connection_terms.png", "14a_4_block_structure.png",
```

```
                "14a_5_rescaling.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all five figure files of this notebook exist")
all_checks_passed()
```

The five figure files must exist where the notebook wrote them; the last line prints ALL 28 CHECKS PASSED (notebook 14a). The 28 checks are: 1 in In [2], 2 in In [3], 2 in In [4], 2 in In [7], 2 in In [8], 2 in In [10], 2 in In [11], 2 in In [12], 2 in In [13], 2 in In [14], 5 in In [16], 1 in In [17], 2 in In [18] and 1 in In [20].

14.12 The block equation in real form and the boundary conditions

The real form. In every block the stationary equation is $\chi' = N\chi$ with $N = M\sigma_3 - \kappa k\,\sigma_2 + ij(\varepsilon - v)\sigma_1$ (Section 14.5). Write the two components of χ as $\chi = (a, ib)$ with two real functions $a(y)$ and $b(y)$, and write $K = \kappa k$ and $\varepsilon' = \varepsilon - v$ for short. Line by line:

$$\sigma_3\chi = (a,\ -ib), \qquad \sigma_2\chi = (b,\ ia), \qquad \sigma_1\chi = (ib,\ a).$$

Rule: multiply each Pauli matrix of Section 14.1 with the column (a, ib); for example the first entry of $\sigma_2\chi$ is $(-i)(ib) = -i^2 b = b$.

$$N\chi = \Big(Ma - Kb - j\varepsilon' b,\ \ i\big[(j\varepsilon' - K)a - Mb\big]\Big).$$

Rule: $N\chi = M\sigma_3\chi - K\sigma_2\chi + ij\varepsilon'\sigma_1\chi$, entry by entry; in the first entry $ij\varepsilon'(ib) = i^2 j\varepsilon' b = -j\varepsilon' b$; the second entry is $-iMb - iKa + ij\varepsilon' a$, which is i times the bracket.

$$a' = Ma - (K + j\varepsilon')\,b, \qquad b' = (j\varepsilon' - K)\,a - Mb.$$

Rule: $\chi' = (a', ib')$; compare the first entries, and the second entries divided by i. Every coefficient is a real number, so a solution that starts with real values a, b stays real: the computer needs only real numbers. Status: PROVED; the record states this form as blockEquation.realForm in `Revision/kohn_sham/ks-theory.json`, and Notebook 14b checks it with sympy (In [2]).

The densities of one orbital. From the three products above and $\chi^\dagger = (a, -ib)$ (the conjugate transpose of the column, with a, b real):

$$\chi^\dagger\chi = a^2 + b^2, \qquad \chi^\dagger\sigma_2\chi = 2ab, \qquad \chi^\dagger\sigma_3\chi = a^2 - b^2, \qquad \chi^\dagger\sigma_1\chi = 0.$$

Rule: multiply the row $(a, -ib)$ with each column; for example $\chi^\dagger\sigma_2\chi = a\cdot b + (-ib)(ia) = ab + ab$, and $\chi^\dagger\sigma_1\chi = a(ib) + (-ib)a = 0$. These four numbers become the number density, the scalar density, the density Q and the current along y of the orbital (Sections 14.26 and 14.27). Status: PROVED (the multiplication above); the record states the same four formulas as blockEquation.realFormDensities in `Revision/kohn_sham/ks-theory.json`.

The current along y is the same at every point. The number $\chi^\dagger\sigma_1\chi$ is the flow along the hidden direction of the U(1) charge of the field (the **current** along y). Of this charge only the local

conservation law is proved; its total is constant only under the ASSUMED no-flux condition at the brane $z = \pi/2$ (Chapter 5, Section 5.6), and the brane parity conditions of the Kohn-Sham record assumed below ($\chi_2(0) = 0$ or $\chi_1(0) = 0$, which its Z2 mirror gives through the parity relation $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$) are such a condition: they make the current vanish at the brane. The gluing of the T2 mirror partner in Chapter 18, which imposes no parity condition (no matching of the field across the brane is derived there, Section 18.28), is not one: the exact solutions of Section 18.21 carry charge through the brane, and gluing does not stop that flow. For every solution of the block equation, complex or real:

$$\frac{d}{dy}(\chi^\dagger\sigma_1\chi) = \chi'^\dagger\sigma_1\chi + \chi^\dagger\sigma_1\chi' = \chi^\dagger(N^\dagger\sigma_1 + \sigma_1 N)\chi.$$

Rule: the product rule; then $\chi' = N\chi$ and $\chi'^\dagger = (N\chi)^\dagger = \chi^\dagger N^\dagger$.

$$N^\dagger = M\sigma_3 - K\sigma_2 - ij\varepsilon'\sigma_1.$$

Rule: the Pauli matrices are Hermitian, M, K, ε' are real, and the complex conjugate of i is $-i$.

$$N^\dagger\sigma_1 + \sigma_1 N = M(\sigma_3\sigma_1 + \sigma_1\sigma_3) - K(\sigma_2\sigma_1 + \sigma_1\sigma_2) - ij\varepsilon'\sigma_1\sigma_1 + ij\varepsilon'\sigma_1\sigma_1 = 0.$$

Rule: two different Pauli matrices anticommute, so the first two brackets are zero, and the last two terms cancel. So the current has the same value at every y. The density $\chi^\dagger\chi$ is not constant: its derivative is $\chi^\dagger(N^\dagger + N)\chi$ with $N^\dagger + N = 2M\sigma_3 - 2K\sigma_2 \neq 0$. Status: PROVED; check bc_current_conserved_along_y of `Revision/kohn_sham/reports/ks-theory-python.json` and of the Wolfram report.

Why the two ends must stop the current. Section 14.4 called h Hermitian "when the boundary terms vanish". Here is the boundary term. For two columns φ and χ of functions of y:

$$\varphi^\dagger h_j\chi - (h_j\varphi)^\dagger\chi = -ij\varphi^\dagger\sigma_1\chi' - ij\varphi'^\dagger\sigma_1\chi = \frac{d}{dy}\big[-ij\,\varphi^\dagger\sigma_1\chi\big].$$

Rule, for the first equality: the part of h_j without derivative, $j(M\sigma_2+K\sigma_3)+v$, is a Hermitian matrix, so it gives $\varphi^\dagger X\chi-(X\varphi)^\dagger\chi = \varphi^\dagger X\chi-\varphi^\dagger X^\dagger\chi = 0$; the derivative part gives $\varphi^\dagger(-ij\sigma_1\chi')-(-ij\sigma_1\varphi')^\dagger\chi$, and $(-ij\sigma_1\varphi')^\dagger = +ij\varphi'^\dagger\sigma_1$. Rule, for the second: the product rule backwards. Integrating from $-L$ to 0:

$$\int_{-L}^{0}\varphi^\dagger h_j\chi\,dy - \int_{-L}^{0}(h_j\varphi)^\dagger\chi\,dy = -ij\big[\varphi^\dagger\sigma_1\chi\big]_{y=-L}^{y=0}.$$

Rule: the integral of a derivative is the difference of its end values. So h_j is self-adjoint (the two integrals are equal, the levels are real and orbitals of different levels are orthogonal, Section 14.4) exactly when the boundary conditions make $\varphi^\dagger\sigma_1\chi$ vanish at both ends: the conditions must **kill the current**. Status: PROVED; check bc_self_adjoint_boundary_term (both reports).

The brane at $y = 0$ (ASSUMED). The author's coordinate patch ends at $y = 0$, where $z = \pi/2$. Something must be said about what lies beyond it. The Revision theory ASSUMES a **Z2 mirror**

(also called an **orbifold**): the patch $y \le 0$ is glued at $y = 0$ to a mirror copy of itself, with the warp $e^{-H|y|}$ on both sides, and a field on the doubled interval obeys $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$. At $y = 0$ this relation says $\Psi(0) = \pm\gamma^{(x_0)}\Psi(0)$, that is $(1 \mp \gamma^{(x_0)})\chi(0) = 0$. In every block $\gamma^{(x_0)}$ is σ_3 (Section 14.5), and $1 - \sigma_3 = \mathrm{diag}(0, 2)$, $1 + \sigma_3 = \mathrm{diag}(2, 0)$. Hence two kinds of orbitals, called the two **parities**:

- EVEN parity (upper sign): $\chi_2(0) = 0$; in real form $b(0) = 0$.
- ODD parity (lower sign): $\chi_1(0) = 0$; in real form $a(0) = 0$.

Both kill the current, because $\chi^\dagger\sigma_1\chi = \chi_1^*\chi_2 + \chi_2^*\chi_1$ (the star is the complex conjugate) is zero when χ_1 or χ_2 is zero. Status: the mirror is ASSUMED; the two conditions follow from it (PROVED; check bc_brane_parity_conditions, both reports).

What the mirror does to the mass. Let χ solve $\chi'(y) = N(y)\chi(y)$ and put $\tilde\chi(y) = \sigma_3\chi(-y)$. Line by line:

$$\tilde\chi'(y) = -\sigma_3\chi'(-y) = -\sigma_3 N(-y)\chi(-y) = -\sigma_3 N(-y)\sigma_3\,\tilde\chi(y).$$

Rule: the chain rule (the derivative of $\chi(-y)$ is $-\chi'(-y)$), then the equation at the point $-y$, then $\chi(-y) = \sigma_3\tilde\chi(y)$ because $\sigma_3\sigma_3 = 1$.

$$-\sigma_3 N\sigma_3 = -M\sigma_3 - K\sigma_2 + ij\varepsilon'\sigma_1.$$

Rule: $\sigma_3\sigma_3\sigma_3 = \sigma_3$, $\sigma_3\sigma_2\sigma_3 = -\sigma_2$, $\sigma_3\sigma_1\sigma_3 = -\sigma_1$. This is the matrix N with M replaced by $-M$. So $\tilde\chi$ solves the block equation with the mass $-M(-y)$, the potential $v(-y)$ and the weight $\kappa(-y)$ (the weight of the mirror copy). The doubled problem is therefore mirror-symmetric only if the mass is **odd** across the brane. The densities say the same: under the map the number density and Q (built from $\chi^\dagger\chi$ and $\chi^\dagger\sigma_3\chi$) are even, while the scalar density S (built from $\chi^\dagger\sigma_2\chi$) and the current change sign, because σ_3 commutes with 1 and σ_3 and anticommutes with σ_2 and σ_1. Since $M_{\text{eff}} = m + \frac{15}{16}\lambda S$ (Section 14.27) and S is odd, the mirror copy carries $(-m, +\lambda)$. Status: PROVED; checks bc_mirror_map_PA and bc_mirror_parities_of_densities (both reports).

What this does and does not say (honesty). The pair $(+m, -m)$ of the patch and its mirror copy is the mirror pair of the pairing theorem T3, which Chapter 19 proves. It is an exact map between solutions of the equations with an ASSUMED boundary construction. It does not say that a mirror copy, a second universe or a pair of universes is created by any process: no creation process, rate or amplitude follows from these equations (Chapters 18 to 20 give the complete statements and the list of what is not proved).

The tip (ASSUMED: a chosen cutoff condition). The hidden coordinate runs to minus infinity at the tip, which a computer cannot reach. The Revision solver stops at $y = -L$ ($L = 3$) and imposes there one member of the family

$$(1 - Q(\theta))\chi(-L) = 0, \qquad Q(\theta) = \cos\theta\,\sigma_3 + \sin\theta\,\sigma_2.$$

Line by line, why every member kills the current. First, $Q^\dagger = Q$ (rule: σ_3, σ_2 are Hermitian, $\cos\theta$ and $\sin\theta$ are real). Second, $Q^2 = \cos^2\theta + \sin^2\theta + \cos\theta\sin\theta(\sigma_3\sigma_2 + \sigma_2\sigma_3) = 1$ (rule: multiply out;

each $\sigma^2 = 1$ and the bracket is zero). Third, $Q\sigma_1 = -\sigma_1 Q$ (rule: σ_1 anticommutes with σ_3 and with σ_2). Now if $Q\chi = \chi$ at the end point, then

$$\chi^\dagger \sigma_1 \chi = \chi^\dagger \sigma_1 Q\chi = -\chi^\dagger Q\sigma_1 \chi = -(Q\chi)^\dagger \sigma_1 \chi = -\chi^\dagger \sigma_1 \chi,$$

so $\chi^\dagger \sigma_1 \chi = 0$ there. Rule: replace χ by $Q\chi$, move Q to the left with the third fact, use $Q^\dagger = Q$, then $Q\chi = \chi$ again. The canonical choice is $\theta = 0$: $(1 - \sigma_3)\chi(-L) = 0$, that is $\chi_2(-L) = 0$, $b(-L) = 0$, the **regular tip**. In real form a member θ is satisfied by the starting point $(a, b) = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2})$ at $y = -L$ (Exercise 14.3), which is where the shooting of Section 14.14 starts. Status: the algebra is PROVED (check bc_tip_family, both reports); the choice of the condition is ASSUMED.

Zero current everywhere. An **orbital** is a solution that satisfies the brane condition and the tip condition. Its current is zero at the two ends and the same at every y, so it is zero everywhere. (For a real solution (a, b) it is zero anyway, by the densities above.)

Why the tip hardly matters when the 3-momentum is not zero. Toward the tip the weight $\kappa = e^{-Hy - a_{4,0}}$ grows without bound, and for large κ the matrix N is dominated by its momentum term, $N \approx -\kappa k\, \sigma_2$. Line by line:

- σ_2 has the eigenvalues $s = +1$ and $s = -1$ (it squares to 1 and its trace is 0), with eigenvectors u_+ and u_-.
- On u_s the equation $\chi' \approx -\kappa k \sigma_2 \chi$ reads $\chi' \approx -s\kappa k\, \chi$.
- Its solution is $\chi \propto \exp(sk\kappa/H)\, u_s$. Rule: the derivative of e^{-Hy} is $-He^{-Hy}$, so the derivative of κ/H is $-\kappa$, and the chain rule gives the derivative of $\exp(sk\kappa/H)$ as $(sk)(-\kappa)\exp(sk\kappa/H)$.
- For $k > 0$ the branch $s = +1$ grows enormously toward the tip and the branch $s = -1$ decays toward the tip (for $k < 0$ the two exchange their roles). The decaying branch, $\sigma_2\chi = -\chi$ for $k > 0$, is the **regular** one.
- An orbital is therefore suppressed at the cutoff, compared with its size at a point y, by about the factor $\exp(-|k|(\kappa(-L) - \kappa(y))/H)$. Rule: the ratio of $\exp(-|k|\kappa/H)$ at the two points.

Whatever condition is imposed at $y = -L$ acts on a part of the orbital that is already this small, so for $k \neq 0$ the levels should change by about this factor at most; this last step is an estimate, not a proof. At the slice $a_{4,0} = 0$, between the brane and the cutoff, $\kappa(-L) - \kappa(0) = e^{HL} - 1 = e^3 - 1 \approx 19.09$, and for the smallest lattice momentum $k = 0.25$ the factor is 8.47×10^{-3} (the column suppression_factor of `Revision/kohn_sham/results/spectrum/tip-angle.csv`). At a later slice the difference is smaller by the factor $e^{-a_{4,0}}$, $\kappa(-L) - \kappa(0) = e^{-a_{4,0}}(e^{HL} - 1)$ (rule: $\kappa = e^{-Hy - a_{4,0}}$ at the two points): along the history the redshifted orbitals reach farther toward the tip, and the position L of the cutoff begins to matter (Section 14.20). At $k = 0$ there is no suppression and the tip condition matters: there the choice $\theta = 0$ selects the brane zero mode of Section 14.13. Status: the two asymptotic branches, the regular branch and the suppression factor of the orbital are PROVED (check bc_tip_asymptotics, both reports); the bound on the change of the levels is an estimate, whose size is COMPUTED in Section 14.20 (check free_tip_angle_insensitivity of `Revision/kohn_sham/reports/ks-rust-solver.json`, record `Revision/kohn_sham/results/spectrum/tip-angle.csv`).

14.13 The exact free levels at zero 3-momentum

Take $k = 0$, $v = 0$ and a constant mass $M > 0$. The real form becomes $a' = Ma - j\varepsilon b$, $b' = j\varepsilon a - Mb$. These two equations can be solved by hand, completely. The solutions are the first test of any numerical method for the model.

Step 1: one equation of second order. Suppose $\varepsilon \neq 0$. Line by line:

$$a = \frac{b' + Mb}{j\varepsilon}.$$

Rule: solve the second equation for a (add Mb and divide by $j\varepsilon \neq 0$).

$$a' = \frac{b'' + Mb'}{j\varepsilon}.$$

Rule: differentiate; M, j, ε are constants.

$$b'' + Mb' = M(b' + Mb) - j^2\varepsilon^2 b.$$

Rule: insert the two lines into the first equation $a' = Ma - j\varepsilon b$ and multiply by $j\varepsilon$.

$$b'' = (M^2 - \varepsilon^2)\, b.$$

Rule: $j^2 = 1$, and Mb' cancels on both sides.

The general solution of this equation depends on the sign of $M^2 - \varepsilon^2$ (check each by differentiating twice): if $\varepsilon^2 > M^2$, write $M^2 - \varepsilon^2 = -p^2$ with $p > 0$, and $b = A\sin(py) + B\cos(py)$; if $\varepsilon^2 < M^2$, write $M^2 - \varepsilon^2 = q^2$ with $q > 0$, and $b = Ae^{qy} + Be^{-qy}$; if $\varepsilon^2 = M^2$, then $b = A + By$. Here A and B are constants.

Step 2: even parity, $b(0) = 0$ and $b(-L) = 0$.

- $\varepsilon^2 > M^2$: $b(0) = B = 0$, so $b = A\sin(py)$; then $b(-L) = -A\sin(pL) = 0$ with $A \neq 0$ requires $pL = n\pi$, $n = 1, 2, 3, \ldots$ Hence the levels $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$.
- $\varepsilon^2 < M^2$, $\varepsilon \neq 0$: $b(0) = A + B = 0$ gives $b = A(e^{qy} - e^{-qy})$, and $b(-L) = A(e^{-qL} - e^{qL}) = 0$ forces $A = 0$ (because $e^{qL} > e^{-qL}$); then $b = 0$ and $a = (b' + Mb)/(j\varepsilon) = 0$: no orbital.
- $\varepsilon^2 = M^2$: $b(0) = A = 0$ and $b(-L) = -BL = 0$, so $b = 0$ and again no orbital.
- $\varepsilon = 0$ (Step 1 does not apply): the equations separate, $a' = Ma$ and $b' = -Mb$, so $a = Ae^{My}$, $b = Be^{-My}$. The brane condition $b(0) = B = 0$ makes $b = 0$ everywhere, which also satisfies $b(-L) = 0$. So $\chi = (Ae^{My}, 0)$ is an orbital with $\varepsilon = 0$, for every L: the **brane zero mode**, largest at the brane.

The constant of the zero mode follows from the normalisation $\int_{-L}^{0} a^2\, dy = 1$:

$$\int_{-L}^{0} A^2 e^{2My}\,dy = A^2\,\frac{1-e^{-2ML}}{2M} = 1, \qquad A = \sqrt{\frac{2M}{1-e^{-2ML}}}.$$

Rule: the integral of e^{2My} is $e^{2My}/(2M)$; evaluate at 0 and $-L$ and subtract; solve for $A > 0$.

Step 3: odd parity, $a(0) = 0$ and $b(-L) = 0$.

- $\varepsilon^2 > M^2$: the solution with $b(-L) = 0$ is $b = A\sin(p(y+L))$ (the general solution written around the point $-L$, where the cosine term must vanish). By Step 1, $a(0) = 0$ means $b'(0) + Mb(0) = 0$, that is $A[p\cos(pL) + M\sin(pL)] = 0$. So the levels are $\varepsilon = \pm\sqrt{M^2+p^2}$ where $p > 0$ solves

$$f(p) = M\sin(pL) + p\cos(pL) = 0, \qquad \text{equivalently} \qquad \tan(pL) = -\frac{p}{M}.$$

Rule: divide by $M\cos(pL)$, which is not zero (if $\cos(pL) = 0$ then $f = \pm M \neq 0$).

- $\varepsilon^2 < M^2$, $\varepsilon \neq 0$: $b = A(e^{q(y+L)} - e^{-q(y+L)})$, and $b'(0) + Mb(0) = A[q(e^{qL} + e^{-qL}) + M(e^{qL} - e^{-qL})] = 0$ forces $A = 0$ (the bracket is positive): no orbital.
- $\varepsilon^2 = M^2$: $b = B(y+L)$ and $b'(0) + Mb(0) = B(1+ML) = 0$ forces $B = 0$: no orbital.
- $\varepsilon = 0$: $a = Ae^{My}$ with $a(0) = A = 0$, and $b = Be^{-My}$ with $b(-L) = Be^{ML} = 0$, so $B = 0$: there is no zero mode of odd parity.

Step 4: the odd roots. Count the roots of $f(p) = M\sin(pL) + p\cos(pL)$ for $p > 0$. At $p = (n+\frac{1}{2})\pi/L$ the sine is $(-1)^n$ and the cosine 0, so $f = M(-1)^n$; at $p = (n+1)\pi/L$ the sine is 0 and the cosine $(-1)^{n+1}$, so $f = p(-1)^{n+1}$. The two values have opposite signs, so f has a root in between (rule: a continuous function that changes sign on an interval is zero somewhere in it). It has exactly one there: on this interval $\tan(pL)$ rises from minus infinity to 0 while $-p/M$ falls, so the two curves meet once; and on the intervals from $n\pi/L$ to $(n+\frac{1}{2})\pi/L$ there is no root, because there $\tan(pL) > 0 > -p/M$. So the roots are $p_0 < p_1 < p_2 < \dots$, one in each interval $((n+\frac{1}{2})\pi/L,\ (n+1)\pi/L)$.

Summary (PROVED). At $k = 0$, $v = 0$, constant $M > 0$ and the canonical tip:

parity	**levels**	**orbital**
even	0 (the brane zero mode)	$(\sqrt{2M/(1-e^{-2ML})}\,e^{My},\ 0)$
even	$\pm\sqrt{M^2+(n\pi/L)^2}$, $n = 1, 2, \dots$	$b = \sin(n\pi y/L)$, $a = (b' + Mb)/(j\varepsilon)$
odd	$\pm\sqrt{M^2+p_n^2}$, $\tan(p_nL) = -p_n/M$	$b = \sin(p_n(y+L))$, $a = (b' + Mb)/(j\varepsilon)$

Apart from the zero mode, every level has $|\varepsilon| > M$: the free spectrum has a gap from $-M$ to M that contains only the zero modes. Every level of one block type is held by its four blocks (s_2, s_3), so it is 4-fold per type and 8-fold in all (for $v = 0$ the levels of the type $j = -1$ are the negatives of

those of $j = +1$, Section 14.5, and the set of levels above is symmetric). Status: PROVED; check bc_exact_k0_spectra (both reports).

The numbers for $M = 1$, $L = 3$ (Notebook 14b, Out [4]): the first odd roots are $p_0 = 0.818548$, $p_1 = 1.744313$, $p_2 = 2.734844$; the lowest odd level is $\sqrt{1 + p_0^2} = 1.292292828069\,m$ and the lowest positive even level is $\sqrt{1 + (\pi/3)^2} = 1.447971930402\,m$. The Rust solver's record `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv` lists 54 such exact levels (labels -3 to 5, both parities, $(m, L) = (1, 3), (1, 2), (2, 3)$) next to its numerical ones (check free_k0_analytic_spectra of `Revision/kohn_sham/reports/ks-rust-solver.json`); Notebook 14b reproduces both columns.

The labels. The Revision solver names the levels by integer **labels**: even parity, label 0 is the zero mode, label $n > 0$ the n-th positive level and $-n$ its negative; odd parity, label $l \geq 0$ is the level $+\sqrt{M^2 + p_l^2}$ and label $l \leq -1$ the level $-\sqrt{M^2 + p_{-l-1}^2}$. Section 14.14 shows that these labels are not arbitrary names: they count half-turns.

14.14 Shooting with a Pruefer angle: every level has a label

Shooting. For nonzero k, or a mass and a potential that depend on y, the block equation cannot be solved by hand, and the computer finds the levels by **shooting** (Chapter 2): start at one end with the condition there, integrate to the other end, and adjust the energy until the condition there holds too. Here the start is the tip, with $(a, b) = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2})$ at $y = -L$, which is $(1, 0)$ for the canonical $\theta = 0$; the real form is integrated from $y = -L$ to $y = 0$ for a trial energy ε; and ε is a level of even parity if $b(0) = 0$, of odd parity if $a(0) = 0$.

The Pruefer angle. Write the point (a, b) of the plane in polar form, $a = r\cos\theta$, $b = r\sin\theta$, with the length $r = \sqrt{a^2 + b^2} > 0$ and the angle θ (the **Pruefer angle**, after the mathematician Pruefer). On a computer the angle of a point is the function atan2(b, a), which returns a value between $-\pi$ and π; following it **continuously** means adding or subtracting 2π whenever that value jumps, so that $\theta(y)$ has no jumps and can count whole turns. Line by line:

$$a' = r'\cos\theta - r\theta'\sin\theta, \qquad b' = r'\sin\theta + r\theta'\cos\theta.$$

Rule: the product rule and the chain rule.

$$ab' - ba' = r^2\theta'(\cos^2\theta + \sin^2\theta) = r^2\theta'.$$

Rule: multiply out; the terms with r' cancel; $\cos^2\theta + \sin^2\theta = 1$.

$$ab' - ba' = j\varepsilon'(a^2 + b^2) - K(a^2 - b^2) - 2Mab.$$

Rule: insert the real form, $a[(j\varepsilon' - K)a - Mb] - b[Ma - (K + j\varepsilon')b]$, and collect the terms.

$$\theta' = j\varepsilon' - K\cos 2\theta - M\sin 2\theta.$$

Rule: divide the last two lines by r^2, with $a^2 - b^2 = r^2(\cos^2\theta - \sin^2\theta) = r^2\cos 2\theta$ and $2ab = 2r^2\sin\theta\cos\theta = r^2\sin 2\theta$ (the double-angle formulas). The equation for θ does not contain r, so the

angle can be followed by itself. Status: PROVED; Notebook 14b checks the last line with sympy (In [6]).

The angle at the brane grows with the energy. Let $a_\varepsilon = \partial a/\partial\varepsilon$ and $b_\varepsilon = \partial b/\partial\varepsilon$ be the rates at which the solution changes when the trial energy changes (the tip values do not depend on ε), and put $w = ab_\varepsilon - ba_\varepsilon$. By the same rule as for θ', with ε in place of y, $\partial\theta/\partial\varepsilon = w/r^2$. Line by line:

$$w' = a'b_\varepsilon + ab'_\varepsilon - b'a_\varepsilon - ba'_\varepsilon.$$

Rule: the product rule.

$$a'_\varepsilon = Ma_\varepsilon - (K + j\varepsilon')b_\varepsilon - jb, \qquad b'_\varepsilon = (j\varepsilon' - K)a_\varepsilon - Mb_\varepsilon + ja.$$

Rule: differentiate the real form with respect to ε; the product rule applied to $j\varepsilon' b$ and $j\varepsilon' a$ gives the extra terms $-jb$ and $+ja$, since $\partial\varepsilon'/\partial\varepsilon = 1$.

$$w' = j(a^2 + b^2) = jr^2.$$

Rule: insert the real form and the last line into the first line; every term that contains M, K or ε' appears twice with opposite signs and cancels, and what is left is $a(ja) - b(-jb)$.

$$\frac{\partial\theta(0)}{\partial\varepsilon} = \frac{w(0)}{r(0)^2} = \frac{j}{r(0)^2}\int_{-L}^{0} r^2\,dy.$$

Rule: $w(-L) = 0$, because the starting point does not depend on ε ($a_\varepsilon = b_\varepsilon = 0$ at $y = -L$); integrate $w' = jr^2$ from $-L$ to 0. So the **shooting function** $\Phi(\varepsilon) = j\,\theta(0)$ has the derivative $\int r^2 dy/r(0)^2 > 0$: it grows strictly with the energy. Status: PROVED (this derivation); Notebook 14b confirms it numerically for $k = 0$ and $k = 0.75$ (In [7]).

Every level has an integer label. $b(0) = 0$ means that $\theta(0)$ is a whole multiple of π; $a(0) = 0$ means that $\theta(0)$ is $\pi/2$ plus a whole multiple of π. With $\Phi = j\theta(0)$:

$$\text{even level: } \Phi(\varepsilon) = l\pi, \qquad \text{odd level: } \Phi(\varepsilon) = \frac{\pi}{2} + l\pi, \qquad l = \ldots, -1, 0, 1, \ldots$$

The integer l is the **label** of the level. Because Φ grows strictly, it passes every target value at most once. It passes every target value at least once, because it runs from $-\infty$ to $+\infty$. Line by line:

$$\theta(0) - \theta(-L) = \int_{-L}^{0} \theta'\,dy = j\varepsilon L - j\int_{-L}^{0} v\,dy - \int_{-L}^{0} \big(K\cos 2\theta + M\sin 2\theta\big)\,dy.$$

Rule: integrate the equation of the Pruefer angle from $-L$ to 0, with $\varepsilon' = \varepsilon - v$ (the integral of the constant $j\varepsilon$ over an interval of length L is $j\varepsilon L$).

$$\big|\Phi(\varepsilon) - \varepsilon L\big| \le |\theta(-L)| + \int_{-L}^{0} |v|\,dy + \int_{-L}^{0} \big(|K| + |M|\big)\,dy.$$

Rule: multiply the last line by j (with $j^2 = 1$ this gives $\Phi = j\theta(0)$ on the left and εL as the first term on the right), move every other term to the right side, and use $|j| = 1$, $|\cos 2\theta| \le 1$, $|\sin 2\theta| \le 1$ and the rule that the size of an integral is at most the integral of the size. The right side does not depend on ε: the start angle $\theta(-L)$ is fixed by the tip condition, and v, K and M are given functions of y. So $\Phi(\varepsilon)$ stays within a fixed distance of the straight line εL: for very negative ε it lies below every target value, for very positive ε above it, and because it has no jumps it passes the target value in between (the intermediate value theorem, Section 2.12). Hence there is exactly one level for every label and parity, no level can be missed and none can be counted twice. Status: PROVED (this derivation; it is not a separate check of the Revision record, and Notebook 14b confirms the labels numerically). At $k = 0$ these are exactly the labels of Section 14.13 (Figure 14b.6 shows the angles).

Finding a level. The notebooks of this chapter find a level by plain **bisection** on $\Phi(\varepsilon) -$ target (Chapter 2): they keep an interval with the target between $\Phi(\text{lo})$ and $\Phi(\text{hi})$ and halve it 72 times, starting from $[-20, 20]$. After 72 halvings the interval has the length $40/2^{72} \approx 8.5 \times 10^{-21}$, far below the spacing of floating-point numbers near 1 (about 2.2×10^{-16}): the result is the level of the integrated equation to the last digit the computer can hold, and the error that remains is the error of the integration. The Revision Rust solver finds the root of the same function Φ with fewer evaluations, by the **safeguarded Newton-bisection** of Section 2.24 (the function find_level of `Revision/kohn_sham/solver/src/shoot.rs`): it widens a bracket around a first guess until $\Phi -$ target changes sign, then takes Newton steps with the derivative $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2$ derived above, takes the midpoint of the bracket instead whenever a Newton step would leave the bracket or the last step has not at least halved the mismatch $|\Phi - \text{target}|$, and stops when the bracket, or its last Newton step, is shorter than the tolerance 10^{-13} (rootTolerance of `Revision/kohn_sham/results/parameters.json`). Both find the root of the same integrated Φ, so the two sets of levels agree to far below 10^{-11}; Notebook 14b measures the largest difference, 7.1×10^{-15} (In [8]).

The integration (RK4). The real form is integrated with the classical fourth-order Runge-Kutta method of Chapter 2, with G steps of size $h = L/G$ (the Revision solver uses $G = 900$ for $L = 3$, so $h = 1/300$). RK4 needs the coefficient $K = \kappa k$ at the nodes and at the midpoints of the steps, that is on a fine grid of $2G + 1$ points. Its error is proportional to h^4: halving the step divides the error of a level by $2^4 = 16$. The Revision record measured exactly this: doubling the step number ($G = 1800$ for $L = 3$, 1200 for $L = 2$) divided the errors of the 39 levels whose error is above 10^{-11} by the median factor 16.00, and the largest errors were 5.050×10^{-8} with the canonical and 3.157×10^{-9} with the refined step (check refined_free_spectra_convergence_order of `Revision/kohn_sham/reports/ks-rust-determinism.json`). Status: COMPUTED; Notebook 14b repeats the measurement (In [10]).

14.15 Example: Notebook 14b, the free spectra exact and numerical

Notebook 14b does Sections 14.12 to 14.14 with the computer. With sympy it checks the real form, the mirror map and the parities of the densities, the brane conditions, the boundary term, the tip family, the constant current, the exact solutions of Section 14.13 and the equation of the Pruefer angle, each against the verdict of the Revision record. With numpy it writes the shooting function of the Revision Rust solver in about forty lines (RK4 on the fine grid, the Pruefer angle followed step by step) with plain bisection on the labels (the Rust solver uses the safeguarded Newton-bisection of Section 14.14 instead), finds the 54 levels of the record and compares them with the exact levels and with the Rust numbers, repeats the fourth-order measurement, and compares the numerical

orbitals with the exact ones. It draws six figures. It needs no Rust. It runs in about 40 seconds (two cells integrate the equation for hundreds of energies at once, 72 times over, and take 10 to 20 seconds each), and its last line is ALL 20 CHECKS PASSED (notebook 14b).

14.16 How to run Notebook 14b

Step 1. What this notebook does and what it needs.

Notebook 14b (The free Kohn-Sham spectrum at zero 3-momentum: exact formulas against numerical shooting) is the file `Revision/textbook/notebooks/14b_free_spectra.ipynb` of the repository Dirac_claude. For the free field (no interaction, constant mass M, zero 3-momentum) it writes the 2 x 2 block equation of dirac16complex in real form, derives its boundary conditions (the ASSUMED Z2 mirror brane at y = 0 and the regular tip at y = -L) and their consequences, solves the equation exactly (the brane zero mode, the even levels plus or minus the square root of M^2 + (n pi/L)^2, the odd levels from tan(pL) = -p/M), solves it again numerically with a fourth-order Runge-Kutta shooting method whose Pruefer angle gives every level an integer label, and compares the two: it reproduces the free spectra, the zero mode and the measured fourth-order convergence of the Revision Rust solver and draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 14b_free_spectra.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 40 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 14b_free_spectra.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 14b_free_spectra.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/14b.captions.json`
- `Revision/textbook/figures/14b_1_odd_condition.png`
- `Revision/textbook/figures/14b_2_shooting_function.png`
- `Revision/textbook/figures/14b_3_spectrum_ladder.png`
- `Revision/textbook/figures/14b_4_rk4_convergence.png`
- `Revision/textbook/figures/14b_5_orbitals.png`
- `Revision/textbook/figures/14b_6_pruefer_angle.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files of this notebook exist
ALL 20 CHECKS PASSED (notebook 14b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/14b_free_spectra.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 14b_free_spectra.ipynb
python -m nbconvert --execute --inplace 14b_free_spectra.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- the cells of sections 9 and 10 run for 10 to 20 seconds each: they integrate the equation for hundreds of energies at once, 72 times over; wait until the PASS lines appear.

14.17 Notebook 14b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/14b_free_spectra.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

In the Kohn-Sham model of dirac16complex in the deflating primordial field every
orbital is described by a pair of functions $\chi = (\chi_1, \chi_2)$ of the
hidden coordinate $y$ between the tip cutoff $y = -L$ and the brane $y = 0$. This
notebook studies the simplest case completely: no interaction ($v = 0$, constant
mass $M$) and zero 3-momentum ($k = 0$). It

- writes the $2\times2$ block equation in real form and checks it;
- derives and checks the boundary conditions: the Z2 mirror brane at $y = 0$
  (ASSUMED) with its two parities, the regular tip at $y = -L$ (a CHOSEN cutoff
  condition), and what they imply (no current through the ends, real levels);
- solves the equation exactly: the brane zero mode $\varepsilon = 0$, the even
  levels $\pm\sqrt{M^2 + (n\pi/L)^2}$ and the odd levels from
  $\tan(pL) = -p/M$ (figure 1);
- solves it again numerically with the shooting function of the Revision Rust
  solver: fourth-order Runge-Kutta (RK4) shooting with a Pruefer angle that
  gives every level an integer label, the levels found by bisection
  (figures 2 and 6);
- compares the two for 54 levels and reproduces the Rust solver's record to
  about $10^{-14}$ (figure 3);
- measures the fourth-order convergence of the method (figure 4) and draws the
  orbitals against the exact ones (figure 5).

It prints a PASS line for every check (20 in all) and saves six figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**
```

Notebook 14b (The free Kohn-Sham spectrum at zero 3-momentum: exact formulas against numerical shooting) is the file ‘Revision/textbook/notebooks/14b_free_spectra.ipynb‘ of the repository Dirac_claude. For the free field (no interaction, constant mass M, zero 3-momentum) it writes the 2 x 2 block equation of dirac16complex in real form, derives its boundary conditions (the ASSUMED Z2 mirror brane at y = 0 and the regular tip at y = -L) and their consequences, solves the equation exactly (the brane zero mode, the even levels plus or minus the square root of M^2 + (n pi/L)^2, the odd levels from tan(pL) = -p/M), solves it again numerically with a fourth-order Runge-Kutta shooting method whose Pruefer angle gives every level an integer label, and compares the two: it reproduces the free spectra, the zero mode and the measured fourth-order convergence of the Revision Rust solver and draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6

**Step 4. In every new terminal: activate the environment and go into the repository
folder.**

Every later step starts in the repository folder with the environment active. Run these
lines whenever you open a new terminal (they do no harm if the environment is already
active):

Windows (PowerShell):

    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"

macOS (zsh) and Linux (bash):

    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude

**Step 5. Run the notebook in JupyterLab.**

In the repository folder, with the environment active, run these two lines (the first one
goes into the folder of the notebooks):

Windows, macOS and Linux:

    cd Revision/textbook/notebooks
    jupyter lab 14b_free_spectra.ipynb

JupyterLab opens in your web browser and shows the notebook (if no browser window opens,
copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the
address bar of your browser). The name at the top right of the notebook must read Python
3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its
left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has
finished when the last code cell shows a number; this takes about 40 seconds on a typical
laptop. Scroll to the end and compare the last printed lines with the lines in the step
"What the notebook writes and what you must see" below. To keep the results, save the
notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut
Down, or press Ctrl+C twice in the terminal.

**Step 6. Or run the notebook without a browser (headless).**

In the repository folder, with the environment active, run:

Windows, macOS and Linux:
```

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 14b_free_spectra.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 14b_free_spectra.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/14b.captions.json‘
- ‘Revision/textbook/figures/14b_1_odd_condition.png‘
- ‘Revision/textbook/figures/14b_2_shooting_function.png‘
- ‘Revision/textbook/figures/14b_3_spectrum_ladder.png‘
- ‘Revision/textbook/figures/14b_4_rk4_convergence.png‘
- ‘Revision/textbook/figures/14b_5_orbitals.png‘
- ‘Revision/textbook/figures/14b_6_pruefer_angle.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS all six figure files of this notebook exist
    ALL 20 CHECKS PASSED (notebook 14b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file

  ‘Revision/textbook/notebooks/14b_free_spectra.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 14b_free_spectra.ipynb
      python -m nbconvert --execute --inplace 14b_free_spectra.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- the cells of sections 9 and 10 run for 10 to 20 seconds each: they integrate the equation for hundreds of energies at once, 72 times over; wait until the PASS lines appear.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 14b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 14b (The free Kohn-Sham spectrum at zero 3-momentum: exact formulas against
# numerical shooting) is the file Revision/textbook/notebooks/14b_free_spectra.ipynb of
# the repository Dirac_claude. For the free field (no interaction, constant mass M, zero
# 3-momentum) it writes the 2 x 2 block equation of dirac16complex in real form, derives
# its boundary conditions (the ASSUMED Z2 mirror brane at y = 0 and the regular tip at y
# = -L) and their consequences, solves the equation exactly (the brane zero mode, the
# even levels plus or minus the square root of M^2 + (n pi/L)^2, the odd levels from
# tan(pL) = -p/M), solves it again numerically with a fourth-order Runge-Kutta shooting
# method whose Pruefer angle gives every level an integer label, and compares the two: it
```

```
# reproduces the free spectra, the zero mode and the measured fourth-order convergence of
# the Revision Rust solver and draws six teaching figures. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
```

```
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
```

```
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 14b_free_spectra.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 40
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 14b_free_spectra.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 14b_free_spectra.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/14b.captions.json
# - Revision/textbook/figures/14b_1_odd_condition.png
# - Revision/textbook/figures/14b_2_shooting_function.png
# - Revision/textbook/figures/14b_3_spectrum_ladder.png
# - Revision/textbook/figures/14b_4_rk4_convergence.png
```

```
# - Revision/textbook/figures/14b_5_orbitals.png
# - Revision/textbook/figures/14b_6_pruefer_angle.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files of this notebook exist
#     ALL 20 CHECKS PASSED (notebook 14b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/14b_free_spectra.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 14b_free_spectra.ipynb
#     python -m nbconvert --execute --inplace 14b_free_spectra.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
```

```
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - the cells of sections 9 and 10 run for 10 to 20 seconds each: they integrate the
#   equation for hundreds of energies at once, 72 times over; wait until the PASS lines
#   appear.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "14b"  # this notebook: chapter 14, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
```

```
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
```

```
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 14b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Hidden coordinate $y$**: proper length along the hidden direction $x_0$;
  the brane is at $y = 0$, the tip cutoff at $y = -L$ (here $L = 3$, $2$).
- **Block equation**: in each of eight $2\times2$ blocks the Kohn-Sham
  equation is $h_j\chi = \varepsilon\chi$ with
  $h_j = j[-i\sigma_1\,d/dy + M\sigma_2 + \kappa k\,\sigma_3] + v$, where
  $j = \pm1$ is the block type, $M$ the (effective) mass, $\kappa k$ the
  weighted 3-momentum and $v$ the vector potential.
- **Level, orbital**: an energy $\varepsilon$ for which a solution exists that
  satisfies both boundary conditions; that solution is the orbital.
- **Real form**: writing $\chi = (a, ib)$ with real $a$, $b$ makes all
  coefficients real.
- **Boundary condition**: a condition on the solution at an end of the interval.
- **Z2 mirror brane**: the ASSUMED picture that the patch $y \le 0$ is glued at
  $y = 0$ to a mirror copy and the field obeys the parity relation
  $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$; the relation (not the gluing alone)
  gives the even parity ($b(0) = 0$) and the odd parity ($a(0) = 0$).
- **Regular tip**: the chosen condition $b(-L) = 0$ at the cutoff.
- **Current**: $\chi^\dagger\sigma_1\chi$, the flow of the U(1) charge along
  $y$; a good boundary condition makes it vanish at the ends.
- **Self-adjoint**: the Hamiltonian satisfies $\int\varphi^\dagger h\chi\,dy =
  \int(h\varphi)^\dagger\chi\,dy$; then the levels are real.
- **Shooting**: start the solution at one end with the condition there,
  integrate to the other end, and adjust $\varepsilon$ until the condition there
```

```
  holds too.
- **RK4**: the classical fourth-order Runge-Kutta method for $u' = f(y, u)$ with
  step $h$: four slopes per step, error per unit length proportional to $h^4$.
- **Pruefer angle** $\theta = \mathrm{atan2}(b, a)$: the angle of the point
  $(a, b)$ in the plane; counting its half-turns labels the levels.
- **Bisection**: halve an interval that contains a root, keep the half where the
  function changes sign, repeat.
- **Simpson's rule**: an integration rule with the weights $1, 4, 2, 4, \dots,
  4, 1$ times (step/3).
```

Text cell:

```
## 4. The physical and mathematical situation

The Revision theory (‘Revision/kohn_sham/ks-theory.json‘) reduces the Kohn-Sham
equation exactly to the block equation $h_j\chi = \varepsilon\chi$ above, with
$\kappa(y) = e^{-Hy - a_{4,0}}$ at the slice $a_{4,0}$ of the deflating history.
In this notebook $k = 0$, so $\kappa$ drops out and the slice does not matter;
$v = 0$ and $M$ is a constant. The equation is then simple enough to solve by
hand, and these exact solutions are the first test of every numerical solver of
the model: the Revision Rust solver checked itself against them (its record
‘Revision/kohn_sham/results/spectrum/free-k0-analytic.csv‘). Here we write the
same numerical method with numpy, so that every step can be read, and we check
that it gives the same numbers.

Units: $H = 1$ and energies in units of the mass $m$ (the canonical runs use
$m = 1$, $L = 3$); the cases $(m, L) = (1, 2)$ and $(2, 3)$ test other values.

Status: the exact solutions are PROVED (and verified in the Revision record);
the numerical levels are COMPUTED (error measured against the exact ones); the
brane condition is ASSUMED; the tip condition is a chosen cutoff condition.
```

Text cell:

```
## 5. The block equation in real form

The eigenvalue equation $h_j\chi = \varepsilon\chi$ is the same as the
first-order system (Revision record, ‘blockEquation.ode‘)

$$\chi' = N\chi,\qquad N = M\sigma_3 - \kappa k\,\sigma_2 + ij(\varepsilon - v)
\sigma_1 .$$

Write $\chi = (a, ib)$ with real functions $a$, $b$ and $K = \kappa k$. Then,
multiplying out the Pauli matrices:

$$a' = Ma - (K + j(\varepsilon - v))\,b,\qquad b' = (j(\varepsilon - v) - K)\,a
- Mb .$$

All coefficients are real, so a real starting point gives a real solution. The
next cell lets sympy multiply $N$ with $(a, ib)$ and checks that the result is
$(a', ib')$ with the two lines above.
```

In [2]:

```
import math  # exp, cos, sin of single numbers

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra with symbols

PY_REPORT = "Revision/kohn_sham/reports/ks-theory-python.json"
WL_REPORT = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
RUST_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"


def record_check(name, reports=(PY_REPORT, WL_REPORT)):
    """True when the check called name is PASS in the first report and, if the
    other reports have a check of that name, PASS there too."""
    verdicts = []
    for report in reports:
        data = json.loads(repository_file(report).read_text(encoding="utf-8"))
        found = [c["verdict"] for c in data["checks"] if c["name"] == name]
        verdicts.append(found[0] if found else "absent")
    return verdicts[0] == "PASS" and all(v in ("PASS", "absent") for v in verdicts)


s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices
s2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
I2 = sp.eye(2)
a, b, M, K, e, v = sp.symbols("a b M K epsilon v", real=True)
ok_all = True
for j in (1, -1):
    N = M * s3 - K * s2 + sp.I * j * (e - v) * s1  # the matrix of chi' = N chi
    derivative = sp.expand(N * sp.Matrix([a, sp.I * b]))  # N times (a, i b)
    a_prime = M * a - (K + j * (e - v)) * b  # the real form
    b_prime = (j * (e - v) - K) * a - M * b
    ok = (sp.expand(derivative[0] - a_prime) == 0
          and sp.expand(derivative[1] - sp.I * b_prime) == 0)
    ok_all &= ok
    say(f"j = {j:+d}: d chi/dy = N chi with chi = (a, i b) gives the real form: "
        f"{ok}")
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
say("record: " + theory["blockEquation"]["realForm"])
check(ok_all, "the real form da/dy = M a - (K + j(eps - v)) b, "
      "db/dy = (j(eps - v) - K) a - M b")
```

Out [2]:

```
j = +1: d chi/dy = N chi with chi = (a, i b) gives the real form: True
j = -1: d chi/dy = N chi with chi = (a, i b) gives the real form: True
record: chi = (a, i b), a, b real: a' = M_eff a - (kappa k + j(eps - v_v)) b, b' = (j(eps
    - v_v) - kappa k) a - M_eff b
PASS the real form da/dy = M a - (K + j(eps - v)) b, db/dy = (j(eps - v) - K) a - M b
```

Text cell:

```
## 6. The boundary conditions

**The brane (ASSUMED).** The Revision theory glues the patch $y \le 0$ at
$y = 0$ to a mirror copy (a Z2 orbifold, with the mirror warp $e^{-H|y|}$) and
asks $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$. At $y = 0$ this says
$(1 \mp \gamma^{(x_0)})\chi(0) = 0$; since $\gamma^{(x_0)} = \sigma_3$ in every
block: EVEN parity $\chi_2(0) = 0$, i.e. $b(0) = 0$; ODD parity $\chi_1(0) = 0$,
i.e. $a(0) = 0$. Line by line, what the mirror map does: if $\chi$ solves
$\chi'(y) = N(y)\chi(y)$, then $\tilde\chi(y) = \sigma_3\chi(-y)$ has
$\tilde\chi'(y) = -\sigma_3N(-y)\sigma_3\tilde\chi(y)$, and with
$\sigma_3\sigma_{1,2}\sigma_3 = -\sigma_{1,2}$ this is the same equation with the
mass $-M(-y)$, the potential $v(-y)$ and the weight $\kappa(-y)$. So the doubled
problem is mirror-symmetric only if the mass is ODD across the brane: the mirror
copy carries $-m$ (with the same $\lambda$), the $(+m, -m)$ mirror pair of the
pairing theorem T3. Under the map, $\chi^\dagger\chi$ and
$\chi^\dagger\sigma_3\chi$ are even, $\chi^\dagger\sigma_2\chi$ and the current
$\chi^\dagger\sigma_1\chi$ odd.

**The tip (chosen).** At the cutoff $y = -L$ the family $(1 - Q(\theta))\chi(-L)
= 0$ with $Q = \cos\theta\,\sigma_3 + \sin\theta\,\sigma_2$ is allowed;
$Q^\dagger = Q$, $Q^2 = 1$ and $Q\sigma_1 + \sigma_1Q = 0$, so every member
kills the current. The canonical choice is $\theta = 0$: $b(-L) = 0$.

**Why these conditions.** Integrating by parts, $\varphi^\dagger h_j\chi -
(h_j\varphi)^\dagger\chi = \frac{d}{dy}[-ij\,\varphi^\dagger\sigma_1\chi]$; if
both ends kill the current, the boundary terms vanish and $h_j$ is
self-adjoint: the levels are real and orbitals of different levels are
orthogonal. Moreover $\chi^\dagger\sigma_1\chi$ is constant in $y$ on every
solution ($N^\dagger\sigma_1 + \sigma_1N = 0$), while $\chi^\dagger\chi$ is not.

The next cell checks all of this with sympy, as the Revision record did.
```

In [3]:

```
y = sp.symbols("y", real=True)
H, a0, k = sp.symbols("H a0 k", real=True)
Mf = sp.Function("M", real=True)(y)  # a mass that depends on y
vf = sp.Function("v", real=True)(y)  # a potential that depends on y
kap = sp.exp(-H * y - a0)  # the momentum weight kappa(y)


def N_of(j, M_, v_, kap_):
    return M_ * s3 - kap_ * k * s2 + sp.I * j * (e - v_) * s1


ok_mirror = True
for j in (1, -1):  # chi~(y) = s3 chi(-y) solves chi~' = -s3 N(-y) s3 chi~
    mirrored = -s3 * N_of(j, Mf.subs(y, -y), vf.subs(y, -y), kap.subs(y, -y)) * s3
    target = N_of(j, -Mf.subs(y, -y), vf.subs(y, -y), kap.subs(y, -y))
    ok_mirror &= sp.simplify(mirrored - target).is_zero_matrix
```

```
check(ok_mirror and record_check("bc_mirror_map_PA"),
      "the mirror map chi(y) -> sigma3 chi(-y) turns M(y) into -M(-y)",
      record=f"{PY_REPORT}, check bc_mirror_map_PA")
even = {name: (s3 * X * s3 - X).is_zero_matrix
        for name, X in {"n": I2, "S": s2, "Q": s3, "current": s1}.items()}
say(f"even under the mirror map: {even}")
check(even == {"n": True, "S": False, "Q": True, "current": False}
      and record_check("bc_mirror_parities_of_densities"),
      "n and Q are even, S and the current odd: the mirror copy carries -m",
      record=f"{PY_REPORT}, check bc_mirror_parities_of_densities")
c1f, c2f = sp.Function("chi1")(y), sp.Function("chi2")(y)
current = sp.expand((sp.Matrix([c1f, c2f]).H * s1 * sp.Matrix([c1f, c2f]))[0])
check(current.subs(c2f, 0) == 0 and current.subs(c1f, 0) == 0
      and record_check("bc_brane_parity_conditions"),
      "both brane parities (chi2(0) = 0 or chi1(0) = 0) kill the current",
      record=f"{PY_REPORT}, check bc_brane_parity_conditions")
phi = sp.Matrix([sp.Function("phi1")(y), sp.Function("phi2")(y)])
chi = sp.Matrix([c1f, c2f])
ok_adjoint = True
for j in (1, -1):
    def h_on(w, j=j):  # h_j applied to the column w
        return j * (-sp.I * s1 * sp.diff(w, y) + Mf * s2 * w + kap * k * s3 * w) \
            + vf * w
    difference = (phi.H * h_on(chi))[0] - (h_on(phi).H * chi)[0]
    boundary = sp.diff(-sp.I * j * (phi.H * s1 * chi)[0], y)
    ok_adjoint &= sp.simplify(sp.expand(difference - boundary)) == 0
check(ok_adjoint and record_check("bc_self_adjoint_boundary_term"),
      "phi^dag h chi - (h phi)^dag chi = d/dy[-i j phi^dag sigma1 chi]",
      record=f"{PY_REPORT}, check bc_self_adjoint_boundary_term")
th = sp.symbols("theta", real=True)
Q = sp.cos(th) * s3 + sp.sin(th) * s2  # the tip family
check(sp.simplify(Q * Q - I2).is_zero_matrix and Q.H == Q
      and sp.simplify(Q * s1 + s1 * Q).is_zero_matrix and record_check("bc_tip_family"),
      "the tip family Q(theta): Q^dag = Q, Q^2 = 1, Q sigma1 = -sigma1 Q",
      record=f"{PY_REPORT}, check bc_tip_family")
kp = sp.symbols("kappa", positive=True)
ok_current = True
for j in (1, -1):
    Nn = M * s3 - kp * k * s2 + sp.I * j * (e - v) * s1
    ok_current &= sp.simplify(Nn.H * s1 + s1 * Nn).is_zero_matrix
    density_changes = not sp.simplify(Nn.H + Nn).is_zero_matrix
check(ok_current and density_changes and record_check("bc_current_conserved_along_y"),
      "the current is constant along y on every solution, the density is not",
      record=f"{PY_REPORT}, check bc_current_conserved_along_y")
```

Out [3]:

```
PASS the mirror map chi(y) -> sigma3 chi(-y) turns M(y) into -M(-y)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check bc_mirror_map_PA
even under the mirror map: {'n': True, 'S': False, 'Q': True, 'current': False}
PASS n and Q are even, S and the current odd: the mirror copy carries -m
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    bc_mirror_parities_of_densities
```

```
PASS both brane parities (chi2(0) = 0 or chi1(0) = 0) kill the current
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    bc_brane_parity_conditions
PASS phi^dag h chi - (h phi)^dag chi = d/dy[-i j phi^dag sigma1 chi]
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    bc_self_adjoint_boundary_term
PASS the tip family Q(theta): Q^dag = Q, Q^2 = 1, Q sigma1 = -sigma1 Q
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check bc_tip_family
PASS the current is constant along y on every solution, the density is not
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    bc_current_conserved_along_y
```

Text cell:

```
## 7. The exact solutions at zero 3-momentum

Take $k = 0$, $v = 0$ and a constant $M > 0$. The real form becomes
$a' = Ma - j\varepsilon b$, $b' = j\varepsilon a - Mb$. Line by line:

1. If $\varepsilon \ne 0$, solve the second equation for $a$:
   $a = (b' + Mb)/(j\varepsilon)$.
2. Differentiate: $a' = (b'' + Mb')/(j\varepsilon)$.
3. Put 1 and 2 into the first equation and multiply by $j\varepsilon$ (with
   $j^2 = 1$): $b'' + Mb' = M(b' + Mb) - \varepsilon^2 b$, that is
   $b'' = (M^2 - \varepsilon^2)\,b$.
4. EVEN parity, $b(0) = 0$ and $b(-L) = 0$: a solution that vanishes at both
   ends needs $M^2 - \varepsilon^2 = -p^2 < 0$, $b = \sin(py)$, and
   $\sin(-pL) = 0$, so $p = n\pi/L$: $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$,
   $n = 1, 2, \dots$
5. EVEN parity and $\varepsilon = 0$: the equations decouple, $a' = Ma$,
   $b' = -Mb$; $b = 0$ satisfies both conditions and $a = e^{My}$: the BRANE
   ZERO MODE, largest at the brane, for every $L$.
6. ODD parity, $a(0) = 0$ and $b(-L) = 0$: $b = \sin(p(y + L))$ and, by 1,
   $a(0) = 0$ means $b'(0) + Mb(0) = 0$, i.e. $p\cos(pL) + M\sin(pL) = 0$, or
   $\tan(pL) = -p/M$; then $\varepsilon = \pm\sqrt{M^2 + p^2}$. In each interval
   $(n + \frac12)\pi/L < p < (n + 1)\pi/L$ there is exactly one root $p_n$.

Every level holds 4 states per block type (the four blocks of the same $j$),
8 in all. The next cell verifies 4 to 6 with sympy and computes the exact levels
for the labels $-3, \dots, 5$ of both parities and the three cases
$(m, L) = (1, 3), (1, 2), (2, 3)$, in the label convention of the Rust solver
(even: label 0 the zero mode, label $n > 0$ the $n$-th positive level, $-n$ its
negative; odd: label $l \ge 0$ the root $p_l$ with the plus sign, label
$l \le -1$ the root $p_{-l-1}$ with the minus sign). The odd roots are found by
bisection in their intervals, as the Rust solver does. The exact levels must
equal the column ‘eps_analytic‘ of the Rust solver’s record.
```

In [4]:

```
Lc = sp.symbols("L", positive=True)
n_int = sp.symbols("n", integer=True, positive=True)
ok_exact = sp.simplify(sp.diff(sp.exp(M * y), y) - M * sp.exp(M * y)) == 0  # step 5
```

```
for j in (1, -1):  # step 4: b = sin(n pi y/L), a = (b' + M b)/(j eps)
    p_n = n_int * sp.pi / Lc
    eps_n = sp.sqrt(M ** 2 + p_n ** 2)
    b_n = sp.sin(p_n * y)
    a_n = (sp.diff(b_n, y) + M * b_n) / (j * eps_n)
    ok_exact &= sp.simplify(sp.diff(a_n, y) - (M * a_n - j * eps_n * b_n)) == 0
    ok_exact &= b_n.subs(y, 0) == 0 and sp.simplify(b_n.subs(y, -Lc)) == 0
p = sp.symbols("p", positive=True)  # step 6 (j = +1)
b_odd = sp.sin(p * (y + Lc))
eps_odd = sp.sqrt(M ** 2 + p ** 2)
a_odd = (sp.diff(b_odd, y) + M * b_odd) / eps_odd
ok_exact &= sp.simplify(sp.diff(a_odd, y) - (M * a_odd - eps_odd * b_odd)) == 0
ok_exact &= sp.simplify(a_odd.subs(y, 0) * eps_odd
                        - (p * sp.cos(p * Lc) + M * sp.sin(p * Lc))) == 0
check(ok_exact and record_check("bc_exact_k0_spectra"),
      "the zero mode, the even levels and the odd condition tan(pL) = -p/M",
      record=f"{PY_REPORT}, check bc_exact_k0_spectra")


def odd_roots(m, L, count):
    """The first count roots p of m sin(pL) + p cos(pL) = 0, by bisection."""
    roots = []
    for n in range(count):
        lo, hi = (n + 0.5) * math.pi / L, (n + 1.0) * math.pi / L
        f = lambda q: m * math.sin(q * L) + q * math.cos(q * L)
        f_lo = f(lo)
        for _ in range(200):
            mid = 0.5 * (lo + hi)
            if mid <= lo or mid >= hi:  # the interval cannot shrink any more
                break
            if (f(mid) > 0.0) == (f_lo > 0.0):
                lo = mid
            else:
                hi = mid
        roots.append(0.5 * (lo + hi))
    return roots


def exact_level(m, L, parity, label):
    """The exact k = 0 level with the given parity and label."""
    if parity == "even":
        if label == 0:
            return 0.0
        return math.copysign(math.sqrt(m * m + (label * math.pi / L) ** 2), label)
    roots = odd_roots(m, L, 6)
    if label >= 0:
        return math.sqrt(m * m + roots[label] ** 2)
    return -math.sqrt(m * m + roots[-label - 1] ** 2)


CASES = [(1.0, 3.0), (1.0, 2.0), (2.0, 3.0)]  # (m, L)
LABELS = list(range(-3, 6))  # the labels -3 ... 5
record_rows = repository_file(
    "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
```

```
).read_text(encoding="utf-8").splitlines()
header = record_rows[0].split(",")
record = [dict(zip(header, row.split(","))) for row in record_rows[1:]]
exact = {}  # (m, L, parity, label) -> exact level
for m_, L_ in CASES:
    for label in LABELS:
        for parity in ("even", "odd"):
            exact[(m_, L_, parity, label)] = exact_level(m_, L_, parity, label)
worst = max(abs(exact[(float(r["m"]), float(r["L"]), r["parity"], int(r["label"]))]
                - float(r["eps_analytic"])) for r in record)
say(f"(m, L) = (1, 3): odd roots p = "
    + ", ".join(f"{q:.6f}" for q in odd_roots(1.0, 3.0, 3)) + ", ...")
lowest_odd = exact[(1.0, 3.0, "odd", 0)]  # the lowest odd level, m = 1, L = 3
lowest_even = exact[(1.0, 3.0, "even", 1)]  # the lowest positive even level
report("lowest odd level for m = 1, L = 3", f"{lowest_odd:.12f}", "m")
report("lowest positive even level for m = 1, L = 3", f"{lowest_even:.12f}", "m")
check(len(record) == 54 and worst < 1e-12,
      "the 54 exact levels equal the column eps_analytic of the record",
      record="Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, eps_analytic")
```

Out [4]:

```
PASS the zero mode, the even levels and the odd condition tan(pL) = -p/M
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    bc_exact_k0_spectra
(m, L) = (1, 3): odd roots p = 0.818548, 1.744313, 2.734844, ...
RESULT lowest odd level for m = 1, L = 3 = 1.292292828069 m
RESULT lowest positive even level for m = 1, L = 3 = 1.447971930402 m
PASS the 54 exact levels equal the column eps_analytic of the record
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, eps_analytic
```

Text cell:

```
The next cell draws the odd condition for $m = 1$, $L = 3$: the function
$f(p) = M\sin(pL) + p\cos(pL)$ for $p$ from 0 to 7, with its roots $p_0, p_1,
\dots$ marked, and the intervals $((n + \frac12)\pi/L, (n+1)\pi/L)$ shaded in
which the bisection looks for them. Each root gives the level
$\sqrt{M^2 + p_n^2}$.
```

In [5]:

```
p_values = np.linspace(0.0, 7.0, 1401)
f_values = 1.0 * np.sin(3.0 * p_values) + p_values * np.cos(3.0 * p_values)
roots_13 = odd_roots(1.0, 3.0, 6)
fig, ax = plt.subplots(figsize=(8.0, 4.0))
for n in range(6):  # the intervals in which the roots lie
    ax.axvspan((n + 0.5) * math.pi / 3.0, (n + 1.0) * math.pi / 3.0, color="C0",
               alpha=0.08)
ax.plot(p_values, f_values, color="C0", label="$f(p) = M\\sin(pL) + p\\cos(pL)$")
ax.plot(roots_13, [0.0] * 6, "o", color="C3", label="roots $p_0, p_1, \\dots$")
ax.axhline(0.0, color="black", linewidth=0.8)
```

```
ax.set_xlabel("$p$ (wave number along $y$, units of $H$)")
ax.set_ylabel("$f(p)$")
ax.set_title("the odd-parity condition $\\tan(pL) = -p/M$ for $M = 1$, $L = 3$")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "odd_condition",
            "The odd-parity level condition for $M = 1$ and $L = 3$: the function "
            "$f(p) = M\\sin(pL) + p\\cos(pL)$ against the wave number $p$ along $y$ "
            "(units of $H$); its zeros (red dots) are the solutions of "
            "$\\tan(pL) = -p/M$, one in each shaded interval from "
            "$(n + 1/2)\\pi/L$ to $(n + 1)\\pi/L$, and each gives the pair of "
            "levels $\\pm\\sqrt{M^2 + p^2}$; the first root $p_0 = 0.8185$ gives "
            "the lowest odd level $1.2923\\,m$.")
```

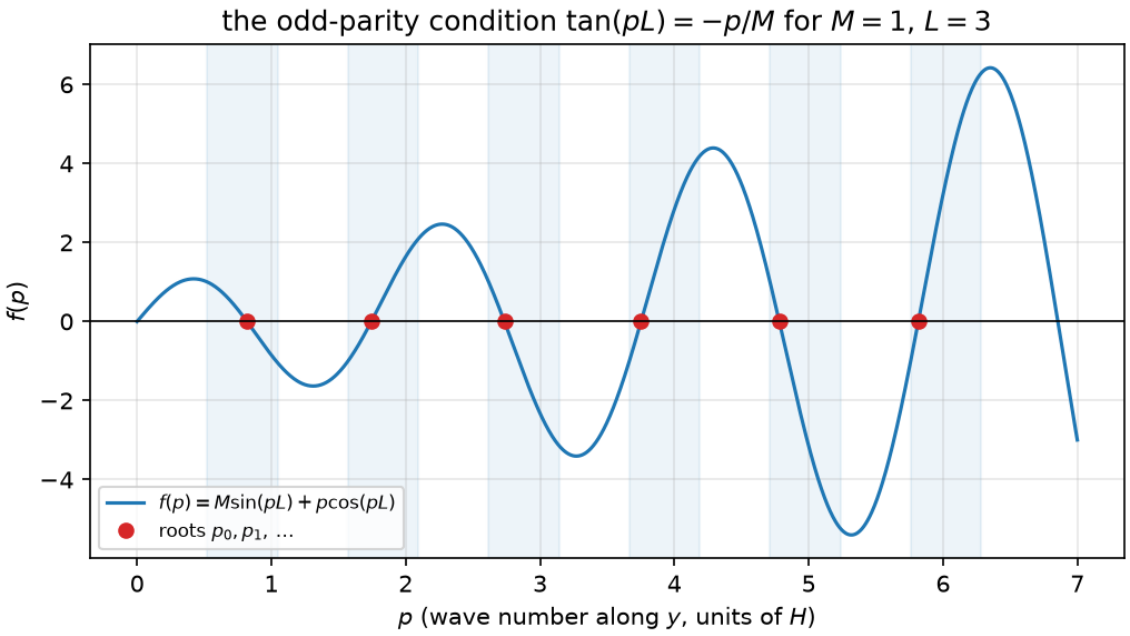

Figure 389: The odd-parity level condition for $M = 1$ and $L = 3$: the function $f(p) = M\sin(pL) + p\cos(pL)$ against the wave number p along y (units of H); its zeros (red dots) are the solutions of $\tan(pL) = -p/M$, one in each shaded interval from $(n + 1/2)\pi/L$ to $(n + 1)\pi/L$, and each gives the pair of levels $\pm\sqrt{M^2 + p^2}$; the first root $p_0 = 0.8185$ gives the lowest odd level $1.2923\,m$. (Notebook 14b, figure 1.)

Out [5]:

```
Figure 14b.1 saved as Revision/textbook/figures/14b_1_odd_condition.png
```

Text cell:

```
## 8. The shooting method and the Pruefer angle

Shooting: start at the tip with the tip condition, $(a, b) = (\cos\frac\theta2,
\sin\frac\theta2)$ (for the canonical $\theta = 0$ this is $(1, 0)$, so
```

```
$b(-L) = 0$), integrate the real form from $y = -L$ to $y = 0$ for a trial
energy $\varepsilon$, and look at the end point. The level is the energy for
which the brane condition holds.

To count, use the Pruefer angle $\theta(y) = \mathrm{atan2}(b, a)$, followed
continuously. Line by line:

1. $a = r\cos\theta$, $b = r\sin\theta$, so $\theta' = (ab' - ba')/r^2$.
2. Insert the real form: $ab' - ba' = j\varepsilon'(a^2 + b^2) - K(a^2 - b^2) -
   2Mab$ with $\varepsilon' = \varepsilon - v$.
3. With $a^2 - b^2 = r^2\cos2\theta$ and $2ab = r^2\sin2\theta$:
   $\theta' = j(\varepsilon - v) - K\cos2\theta - M\sin2\theta$.
4. The derivative of the end angle with respect to the energy is
   $d\theta(0)/d\varepsilon = j\int_{-L}^0 r^2\,dy\,/\,r(0)^2$; so
   $\Phi(\varepsilon) = j\,\theta(0)$ grows strictly with $\varepsilon$.

The brane condition says: even, $b(0) = 0$, i.e. $\Phi = l\pi$; odd, $a(0) = 0$,
i.e. $\Phi = \pi/2 + l\pi$, with an integer label $l$. Because $\Phi$ only grows,
and $\Phi(\varepsilon) - \varepsilon L$ stays bounded (so $\Phi$ runs from
$-\infty$ to $+\infty$), each label has exactly one level, no level can be
missed, and bisection on $\Phi(\varepsilon) - $ target always finds it.

The next cell checks line 3 with sympy and defines the two numerical tools of
the notebook: `shoot`, written exactly as in the Rust solver (classical RK4 with
$G$ steps on $[-L, 0]$, the coefficients taken at the nodes and step midpoints
of a fine grid of $2G + 1$ points, the angle unwrapped after every step), and
`levels` (plain bisection, 72 halvings of the interval $[-20, 20]$, which reaches
the resolution of floating-point numbers; the Rust solver finds the same root
with a safeguarded Newton-bisection that stops at the tolerance $10^{-13}$).
Both work on whole arrays of energies at once (numpy), which makes them fast.
```

In [6]:

```
r_s, t_s = sp.symbols("r t", positive=True)  # polar coordinates of (a, b)
a_p = r_s * sp.cos(t_s)
b_p = r_s * sp.sin(t_s)
j_s = sp.symbols("j")
num = a_p * ((j_s * e - K) * a_p - M * b_p) - b_p * (M * a_p - (K + j_s * e) * b_p)
pruefer = j_s * e - K * sp.cos(2 * t_s) - M * sp.sin(2 * t_s)
check(sp.simplify(num / r_s ** 2 - pruefer) == 0,
      "d theta/dy = j(eps - v) - K cos 2theta - M sin 2theta")


def shoot(eps, k=0.0, j=1.0, M=1.0, L=3.0, G=900, H=1.0, a4=0.0, tip=0.0,
          keep=False):
    """Integrate the real form from the tip to the brane for the energies eps.

    eps, k, j may be arrays (numpy combines them element by element).  Returns
    Phi = j theta(0); with keep=True also theta, a and b at the G + 1 nodes."""
    eps, k, j = np.broadcast_arrays(np.asarray(eps, dtype=float),
                                    np.asarray(k, dtype=float),
                                    np.asarray(j, dtype=float))
    nf = 2 * G + 1  # the fine grid: nodes and step midpoints
```

```
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    e_minus_Hy = np.exp(-H * y_fine)  # e^(-Hy) on the fine grid
    kk = k * np.exp(-a4)  # k e^(-a4,0): kappa k = kk e^(-Hy)
    h = L / G
    a_ = np.full(eps.shape, math.cos(0.5 * tip))  # the tip condition
    b_ = np.full(eps.shape, math.sin(0.5 * tip))
    theta = np.full(eps.shape, 0.5 * tip)
    raw = np.arctan2(b_, a_)
    je = j * eps
    path = [(theta, a_, b_)]
    for i in range(G):
        K0, K1, K2 = (kk * e_minus_Hy[2 * i], kk * e_minus_Hy[2 * i + 1],
                      kk * e_minus_Hy[2 * i + 2])  # K at y, y + h/2, y + h
        p1a, p1b = M * a_ - (K0 + je) * b_, (je - K0) * a_ - M * b_
        a2, b2 = a_ + 0.5 * h * p1a, b_ + 0.5 * h * p1b
        p2a, p2b = M * a2 - (K1 + je) * b2, (je - K1) * a2 - M * b2
        a3, b3 = a_ + 0.5 * h * p2a, b_ + 0.5 * h * p2b
        p3a, p3b = M * a3 - (K1 + je) * b3, (je - K1) * a3 - M * b3
        a4_, b4_ = a_ + h * p3a, b_ + h * p3b
        p4a, p4b = M * a4_ - (K2 + je) * b4_, (je - K2) * a4_ - M * b4_
        a_ = a_ + h / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)
        b_ = b_ + h / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
        new = np.arctan2(b_, a_)
        step = new - raw  # the change of the angle, brought into (-pi, pi]
        step = np.where(step > np.pi, step - 2 * np.pi,
                        np.where(step <= -np.pi, step + 2 * np.pi, step))
        theta = theta + step
        raw = new
        if keep:
            path.append((theta, a_, b_))
    if keep:
        return j * theta, [np.array(q) for q in zip(*path)]
    return j * theta


def target(parity, label):
    """Phi of the level: l pi (even) or pi/2 + l pi (odd)."""
    offset = np.where(np.asarray(parity) == "even", 0.0, 0.5 * np.pi)
    return offset + np.asarray(label, dtype=float) * np.pi


def levels(k, j, parity, label, iterations=72, **options):
    """The levels with the given labels, by bisection on Phi(eps) - target."""
    k, j, parity, label = np.broadcast_arrays(
        np.asarray(k, dtype=float), np.asarray(j, dtype=float),
        np.asarray(parity), np.asarray(label))
    goal = target(parity, label)
    lo, hi = np.full(k.shape, -20.0), np.full(k.shape, 20.0)
    for _ in range(iterations):
        mid = 0.5 * (lo + hi)
        g = shoot(mid, k, j, **options) - goal
        lo = np.where(g <= 0.0, mid, lo)  # g = 0: an exact level
        hi = np.where(g >= 0.0, mid, hi)
    return 0.5 * (lo + hi)
```

```
say("defined: shoot(eps, k, j, M, L, G, ...) and levels(k, j, parity, label, ...)")
```

Out [6]:

```
PASS d theta/dy = j(eps - v) - K cos 2theta - M sin 2theta
defined: shoot(eps, k, j, M, L, G, ...) and levels(k, j, parity, label, ...)
```

Text cell:

```
The next cell computes the shooting function $\Phi(\varepsilon)$ for $m = 1$,
$L = 3$, $k = 0$, $j = +1$ and 801 energies from $-4$ to $4$ at once, for both
parities, checks that it grows strictly (line 4 of the derivation), checks the
same for a nonzero momentum $k = 0.75$ and both block types (the test of the Rust
solver), and draws it with the targets $l\pi$ (even) and $\pi/2 + l\pi$ (odd):
every crossing of a curve with one of its targets is a level.
```

In [7]:

```
trial = np.linspace(-4.0, 4.0, 801)  # 801 trial energies
phi_curve = shoot(trial, 0.0, 1.0)  # Phi for k = 0, j = +1 (both parities)
grows = bool(np.all(np.diff(phi_curve) > 0.0))
grows_k = all(bool(np.all(np.diff(shoot(np.linspace(-4.0, 3.96, 200), 0.75, jj)) > 0))
              for jj in (1.0, -1.0))
check(grows and grows_k,
      "Phi(eps) grows strictly with eps (k = 0 and k = 0.75, both block types)")
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.plot(trial, phi_curve / np.pi, color="C0", label="$\\Phi(\\varepsilon)/\\pi$")
for label in range(-3, 4):  # the targets in units of pi
    ax.axhline(label, color="C1", linewidth=0.7)
    ax.axhline(label + 0.5, color="C2", linewidth=0.7, linestyle="--")
even_levels = [exact[(1.0, 3.0, "even", l)] for l in range(-2, 3)]
odd_levels = [exact[(1.0, 3.0, "odd", l)] for l in range(-2, 2)]
ax.plot(even_levels, range(-2, 3), "o", color="C1",
        label="even levels: $\\Phi = l\\pi$")
ax.plot(odd_levels, [l + 0.5 for l in range(-2, 2)], "s", color="C2",
        label="odd levels: $\\Phi = \\pi/2 + l\\pi$")
ax.set_xlabel("trial energy $\\varepsilon$ (units of $m$)")
ax.set_ylabel("$\\Phi(\\varepsilon) / \\pi$")
ax.set_title("the shooting function for $k = 0$, $j = +1$, $m = 1$, $L = 3$")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "shooting_function",
            "The shooting function $\\Phi(\\varepsilon) = j\\,\\theta(0)$ (the "
            "Pruefer angle at the brane, in units of $\\pi$) against the trial "
            "energy $\\varepsilon$ (units of $m$) for $k = 0$, $j = +1$, $m = 1$, "
            "$L = 3$. The solid horizontal lines are the even targets $l\\pi$, the "
            "dashed ones the odd targets $\\pi/2 + l\\pi$; the curve rises "
            "steadily, so it crosses every target exactly once, and the "
            "crossings are the exact levels (circles even, squares odd), "
            "including the zero mode at $\\varepsilon = 0$, $\\Phi = 0$; between "
```

```
"$-1$ and $1$ the curve rises only a little, because no level lies "
"there except the zero mode.")
```

Out [7]:

```
PASS Phi(eps) grows strictly with eps (k = 0 and k = 0.75, both block types)
```

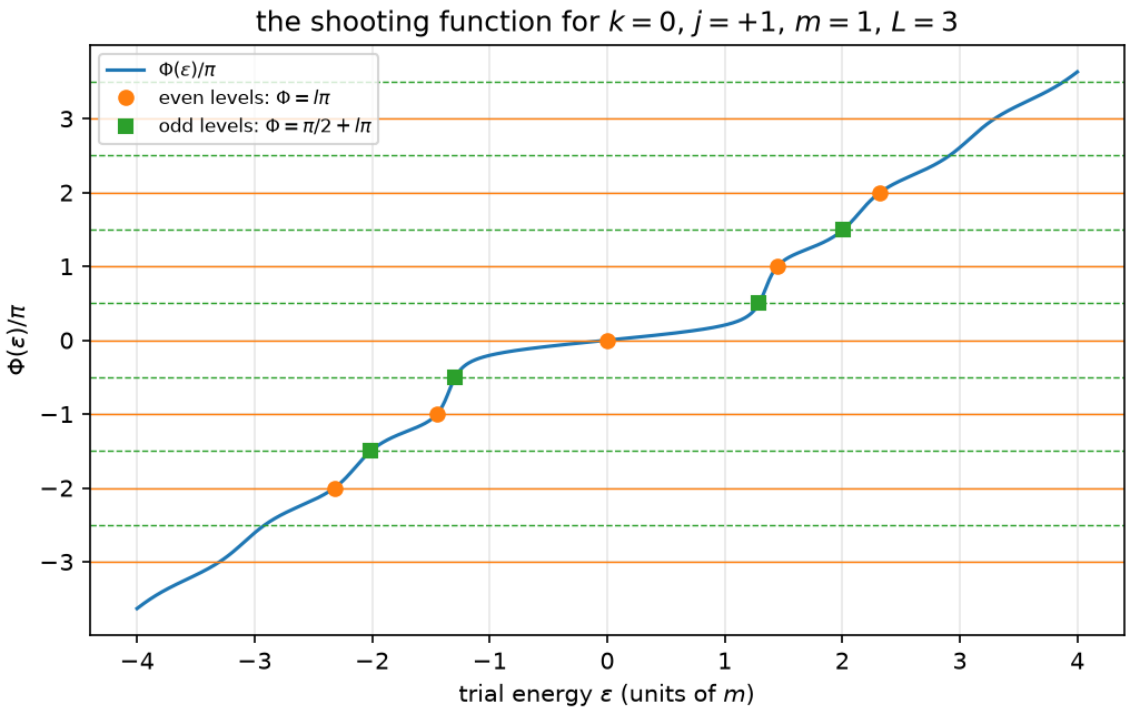

Figure 390: The shooting function $\Phi(\varepsilon) = j\,\theta(0)$ (the Pruefer angle at the brane, in units of π) against the trial energy ε (units of m) for $k = 0$, $j = +1$, $m = 1$, $L = 3$. The solid horizontal lines are the even targets $l\pi$, the dashed ones the odd targets $\pi/2 + l\pi$; the curve rises steadily, so it crosses every target exactly once, and the crossings are the exact levels (circles even, squares odd), including the zero mode at $\varepsilon = 0$, $\Phi = 0$; between -1 and 1 the curve rises only a little, because no level lies there except the zero mode. (Notebook 14b, figure 2.)

Out [7]:

```
Figure 14b.2 saved as Revision/textbook/figures/14b_2_shooting_function.png
```

Text cell:

```
## 9. All 54 levels: numerical against exact

The next cell computes the 54 levels (labels $-3$ to $5$, both parities, three
cases) by shooting and bisection, with the step numbers of the Rust solver:
$G = 900$ for $L = 3$ and $G = 600$ for $L = 2$ (the same step $h = 1/300$). It
```

```
checks three things:

- the numerical levels equal the Rust solver's column `eps_numeric` to
  $10^{-11}$ (both programs integrate the same RK4 discretisation and find the
  root of the same shooting function, this notebook by plain bisection down to
  the last digit, the Rust solver by a safeguarded Newton-bisection that stops
  at its tolerance $10^{-13}$ or sooner; the printed largest difference is of
  order $10^{-14}$);
- the largest numerical errors $|\varepsilon_{\rm num} -
  \varepsilon_{\rm exact}|$, for $|\varepsilon| < 4m$ and above, equal the ones
  the Rust check `free_k0_analytic_spectra` measured (read from its record) and
  lie below its tolerances $5\times10^{-9}$ and $3\times10^{-7}$;
- the zero mode comes out as exactly 0.

It prints the levels of the case $m = 1$, $L = 3$ next to the exact ones, the
largest difference from the Rust levels and the two largest errors.
```

In [8]:

```
numeric = {}  # (m, L, parity, label) -> numerical level at the canonical step
for m_, L_ in CASES:
    pars = np.array([p_ for l_ in LABELS for p_ in ("even", "odd")])
    labs = np.array([l_ for l_ in LABELS for p_ in ("even", "odd")])
    found = levels(0.0, 1.0, pars, labs, M=m_, L=L_, G=round(900 * L_ / 3))
    for p_, l_, x in zip(pars, labs, found):
        numeric[(m_, L_, str(p_), int(l_))] = float(x)
say("m = 1, L = 3:  label  even (numerical)   even (exact)    "
    "odd (numerical)    odd (exact)")
for l_ in LABELS:
    row = [numeric[(1.0, 3.0, "even", l_)], exact[(1.0, 3.0, "even", l_)],
           numeric[(1.0, 3.0, "odd", l_)], exact[(1.0, 3.0, "odd", l_)]]
    say(f"               {l_:+d}   " + "  ".join(f"{x:15.10f}" for x in row))
rust_diff = max(abs(numeric[(float(r["m"]), float(r["L"]), r["parity"],
                             int(r["label"]))] - float(r["eps_numeric"]))
                for r in record)
report("largest difference from the Rust solver's levels", f"{rust_diff:.1e}", "m")
check(rust_diff < 1e-11,
      "the 54 numerical levels equal those of the Rust solver",
      record="Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, eps_numeric")
errors = {key: abs(numeric[key] - exact[key]) for key in numeric}
low = max(err for key, err in errors.items() if abs(exact[key]) < 4.0)
high = max(err for key, err in errors.items() if abs(exact[key]) >= 4.0)
report("largest error for |eps| < 4 m", f"{low:.2e}", "m")
report("largest error for |eps| >= 4 m", f"{high:.2e}", "m")
import re  # regular expressions: patterns that find numbers in a text


def record_detail(name, report_file):
    """The detail text of the check called name in a Revision report."""
    data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
    return [c["detail"] for c in data["checks"] if c["name"] == name][0]
```

```
detail = record_detail("free_k0_analytic_spectra", RUST_REPORT)
rec_low, tol_low, rec_high, tol_high = re.findall(r"\d+(?:\.\d+)?e-\d+", detail)
check(f"{low:.2e}" == f"{float(rec_low):.2e}"
      and f"{high:.2e}" == f"{float(rec_high):.2e}"
      and low < float(tol_low) and high < float(tol_high)
      and record_check("free_k0_analytic_spectra", (RUST_REPORT,)),
      f"largest errors {low:.2e} for |eps| < 4 m and {high:.2e} above, as in the "
      f"record, below its tolerances {tol_low} and {tol_high}",
      record=f"{RUST_REPORT}, check free_k0_analytic_spectra")
check(all(numeric[(m_, L_, "even", 0)] == 0.0 for m_, L_ in CASES),
      "the brane zero mode comes out as exactly eps = 0")
```

Out [8]:

```
m = 1, L = 3:  label  even (numerical)   even (exact)    odd (numerical)    odd (exact)
               -3     -3.2969083098    -3.2969083095    -2.9119358755    -2.9119358754
               -2     -2.3208814802    -2.3208814802    -2.0106285601    -2.0106285600
               -1     -1.4479719304    -1.4479719304    -1.2922928281    -1.2922928281
               +0      0.0000000000     0.0000000000     1.2922928281     1.2922928281
               +1      1.4479719304     1.4479719304     2.0106285601     2.0106285600
               +2      2.3208814802     2.3208814802     2.9119358755     2.9119358754
               +3      3.2969083098     3.2969083095     3.8829900333     3.8829900326
               +4      4.3065024545     4.3065024532     4.8845756333     4.8845756308
               +5      5.3306254627     5.3306254587     5.9016800925     5.9016800858
RESULT largest difference from the Rust solver's levels = 7.1e-15 m
PASS the 54 numerical levels equal those of the Rust solver
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, eps_numeric
RESULT largest error for |eps| < 4 m = 7.23e-10 m
RESULT largest error for |eps| >= 4 m = 5.05e-08 m
PASS largest errors 7.23e-10 for |eps| < 4 m and 5.05e-08 above, as in the record, below
    its tolerances 5e-9 and 3e-7
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_k0_analytic_spectra
PASS the brane zero mode comes out as exactly eps = 0
```

Text cell:

```
The next cell draws the three spectra as level ladders: for each case
$(m, L)$ the exact levels as short horizontal lines and the numerical ones as
markers (even parity on the left of each ladder, odd on the right). The gap
$(-M, M)$ around zero contains only the zero mode; a larger $m$ widens it; a
shorter $L$ spreads the levels (the wave numbers $n\pi/L$ grow).
```

In [9]:

```
fig, ax = plt.subplots(figsize=(8.0, 5.0))
for position, (m_, L_) in enumerate(CASES):
    x0 = 3.0 * position  # the horizontal place of this ladder
    for l_ in LABELS:
        for shift, parity, color, marker in ((0.0, "even", "C0", "o"),
                                             (1.2, "odd", "C1", "s")):
```

```
            level = exact[(m_, L_, parity, l_)]
            ax.plot([x0 + shift - 0.4, x0 + shift + 0.4], [level, level],
                    color=color, linewidth=1.5)
            ax.plot([x0 + shift], [numeric[(m_, L_, parity, l_)]], marker,
                    color=color, markersize=4)
    ax.fill_between([x0 - 0.5, x0 + 1.7], -m_, m_, color="gray", alpha=0.12)
ax.set_xlim(-0.6, 8.4)
ax.set_ylim(-7.0, 9.5)
ax.set_xticks([0.6, 3.6, 6.6])
ax.set_xticklabels(["$m = 1$, $L = 3$", "$m = 1$, $L = 2$", "$m = 2$, $L = 3$"])
ax.plot([], [], "o-", color="C0", label="even parity (line exact, dot numerical)")
ax.plot([], [], "s-", color="C1", label="odd parity (line exact, square numerical)")
ax.set_ylabel("level $\\varepsilon$ (units of $H = 1$)")
ax.set_title("the free $k = 0$ spectra, labels $-3$ to $5$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "spectrum_ladder",
            "The free $k = 0$ spectra for the three cases $(m, L) = (1, 3)$, "
            "$(1, 2)$ and $(2, 3)$, labels $-3$ to $5$: exact levels as short "
            "horizontal lines, numerical RK4 levels as markers on them (even "
            "parity left, odd parity right of each ladder); vertical axis the "
            "level in units of $H = 1$. The shaded band from $-m$ to $m$ contains "
            "only the even zero mode at $0$; a larger mass widens the gap and a "
            "shorter interval $L$ spreads the levels; on this scale the numerical "
            "and the exact levels cannot be told apart.")
```

Out [9]:

```
Figure 14b.3 saved as Revision/textbook/figures/14b_3_spectrum_ladder.png
```

Text cell:

```
## 10. Fourth-order convergence

RK4 has an error proportional to $h^4$: halving the step should divide the error
of every level by $2^4 = 16$. The Rust solver's refined run (the record
`ks-rust-determinism.json`, check `refined_free_spectra_convergence_order`)
doubled the step number ($G = 1800$ for $L = 3$, $1200$ for $L = 2$) and found,
over the 39 levels whose canonical error exceeds $10^{-11}$, the median error
ratio 16.00, with the largest errors $5.050\times10^{-8}$ (canonical) and
$3.157\times10^{-9}$ (refined). The next cell repeats this measurement. It takes
about 15 seconds.
```

In [10]:

```
refined = {}
for m_, L_ in CASES:
    pars = np.array([p_ for l_ in LABELS for p_ in ("even", "odd")])
    labs = np.array([l_ for l_ in LABELS for p_ in ("even", "odd")])
    found = levels(0.0, 1.0, pars, labs, M=m_, L=L_, G=round(1800 * L_ / 3))
    for p_, l_, x in zip(pars, labs, found):
        refined[(m_, L_, str(p_), int(l_))] = float(x)
```

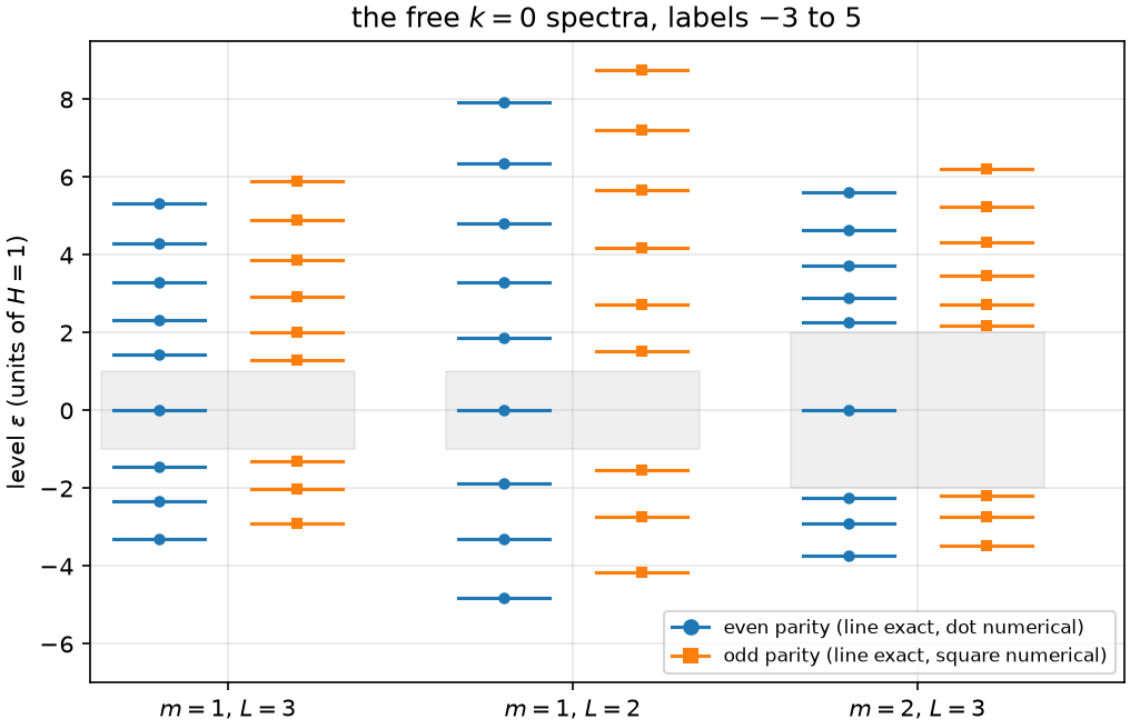

Figure 391: The free $k = 0$ spectra for the three cases $(m, L) = (1, 3)$, $(1, 2)$ and $(2, 3)$, labels -3 to 5: exact levels as short horizontal lines, numerical RK4 levels as markers on them (even parity left, odd parity right of each ladder); vertical axis the level in units of $H = 1$. The shaded band from $-m$ to m contains only the even zero mode at 0; a larger mass widens the gap and a shorter interval L spreads the levels; on this scale the numerical and the exact levels cannot be told apart. (Notebook 14b, figure 3.)

```
refined_errors = {key: abs(refined[key] - exact[key]) for key in refined}
ratios = sorted(errors[key] / refined_errors[key] for key in errors
                if errors[key] > 1e-11 and refined_errors[key] > 0.0)
median = ratios[len(ratios) // 2]
report("number of levels with a canonical error above 1e-11", len(ratios))
report("median error ratio canonical / refined", f"{median:.2f}")
report("largest error, canonical step", f"{max(errors.values()):.3e}", "m")
report("largest error, refined step", f"{max(refined_errors.values()):.3e}", "m")
DETERMINISM = "Revision/kohn_sham/reports/ks-rust-determinism.json"
detail = record_detail("refined_free_spectra_convergence_order", DETERMINISM)
rec_canonical, rec_refined = re.search(r"canonical (\S+), refined (\S+);",
                                       detail).groups()
rec_median, rec_count = re.search(r"ratio (\S+) over (\d+) levels",
                                  detail).groups()
check(len(ratios) == int(rec_count) and f"{median:.2f}" == rec_median
      and f"{max(errors.values()):.3e}" == rec_canonical
      and f"{max(refined_errors.values()):.3e}" == rec_refined
      and record_check("refined_free_spectra_convergence_order", (DETERMINISM,)),
      f"{len(ratios)} levels, median error ratio {median:.2f}, largest errors "
      f"{max(errors.values()):.3e} and {max(refined_errors.values()):.3e}",
      record=f"{DETERMINISM}, check refined_free_spectra_convergence_order")
```

Out [10]:

```
RESULT number of levels with a canonical error above 1e-11 = 39
RESULT median error ratio canonical / refined = 16.00
RESULT largest error, canonical step = 5.050e-08 m
RESULT largest error, refined step = 3.157e-09 m
PASS 39 levels, median error ratio 16.00, largest errors 5.050e-08 and 3.157e-09
     reproduces Revision/kohn_sham/reports/ks-rust-determinism.json, check
    refined_free_spectra_convergence_order
```

Text cell:

```
The next cell draws the convergence for four levels of the case $m = 1$,
$L = 3$ (even labels 1 and 3, odd labels 0 and 3), with the step numbers
$G = 150, 300, 600, 900, 1800$: the error against the step $h = L/G$ on
logarithmic axes, with a line of slope 4 for comparison.
```

In [11]:

```
chosen = [("even", 1), ("even", 3), ("odd", 0), ("odd", 3)]
steps = [150, 300, 600, 900, 1800]
curves = {key: [] for key in chosen}
for G_ in steps:
    found = levels(0.0, 1.0, np.array([p_ for p_, l_ in chosen]),
                   np.array([l_ for p_, l_ in chosen]), M=1.0, L=3.0, G=G_)
    for key, x in zip(chosen, found):
        curves[key].append(abs(float(x) - exact[(1.0, 3.0) + key]))
h_values = np.array([3.0 / G_ for G_ in steps])
fig, ax = plt.subplots(figsize=(7.0, 4.6))
```

```
for (parity, label), errs in curves.items():
    ax.loglog(h_values, errs, "o-", label=f"{parity}, label {label}")
ax.loglog(h_values, 3e-2 * h_values ** 4, ":", color="black",
          label="slope 4: error $\\propto h^4$")
ax.set_xticks(h_values)  # one tick at each step used
ax.set_xticklabels([f"{x:.4f}" for x in h_values], fontsize=8)
ax.xaxis.set_minor_locator(matplotlib.ticker.NullLocator())  # no extra ticks
ax.set_xlabel("RK4 step $h = L/G$ (units of $1/H$)")
ax.set_ylabel("$|\\varepsilon_{\\rm RK4} - \\varepsilon_{\\rm exact}|$ (units of $m$)")
ax.set_title("fourth-order convergence of the shooting method ($m = 1$, $L = 3$)")
ax.legend(fontsize=8)
fourth = all(12.0 < errs[i] / errs[i + 1] < 20.0 for errs in curves.values()
             for i in range(2))  # halving h from 0.02 to 0.005: ratios near 16
check(fourth, "halving the step divides each error by about 16 (between 12 and 20)")
save_figure(fig, "rk4_convergence",
            "The error of four numerical levels ($m = 1$, $L = 3$; even labels 1 "
            "and 3, odd labels 0 and 3) against the RK4 step $h = L/G$ for "
            "$G = 150, 300, 600, 900, 1800$, on logarithmic axes (both in units "
            "of $m = H = 1$). The points fall on lines parallel to the dotted line "
            "of slope 4: halving the step divides the error by 16, the order of "
            "the classical Runge-Kutta method; higher levels have larger errors "
            "because their orbitals oscillate faster.")
```

Out [11]:

```
PASS halving the step divides each error by about 16 (between 12 and 20)
```

Out [11]:

```
Figure 14b.4 saved as Revision/textbook/figures/14b_4_rk4_convergence.png
```

Text cell:

```
## 11. The orbitals

The orbital of a level is the solution $(a, b)$ itself, normalised so that
$\int_{-L}^0(a^2 + b^2)\,dy = 1$. The next cell defines ‘orbital‘, again as in
the Rust solver: it keeps $(a, b)$ at the nodes, fills the step midpoints by
cubic Hermite interpolation (from the values and the derivatives at the two
neighbouring nodes, error of order $h^4$), and normalises with Simpson’s rule on
the fine grid. It then checks the brane zero mode against the exact
$a = \sqrt{2M/(1 - e^{-2ML})}\,e^{My}$, $b = 0$ (the Rust check
‘free_zero_mode_exact‘: $\varepsilon = 0$ exactly, $b = 0$ exactly, $a$ to
$10^{-9}$), and the first even and odd orbitals against the exact ones of
section 7 (normalised the same way and with the sign that makes $a(-L) > 0$).
```

In [12]:

```
def orbital(eps, k=0.0, j=1.0, M=1.0, L=3.0, G=900, H=1.0, a4=0.0, tip=0.0):
    """The normalised orbital (y, a, b) on the fine grid of 2G + 1 points."""
```

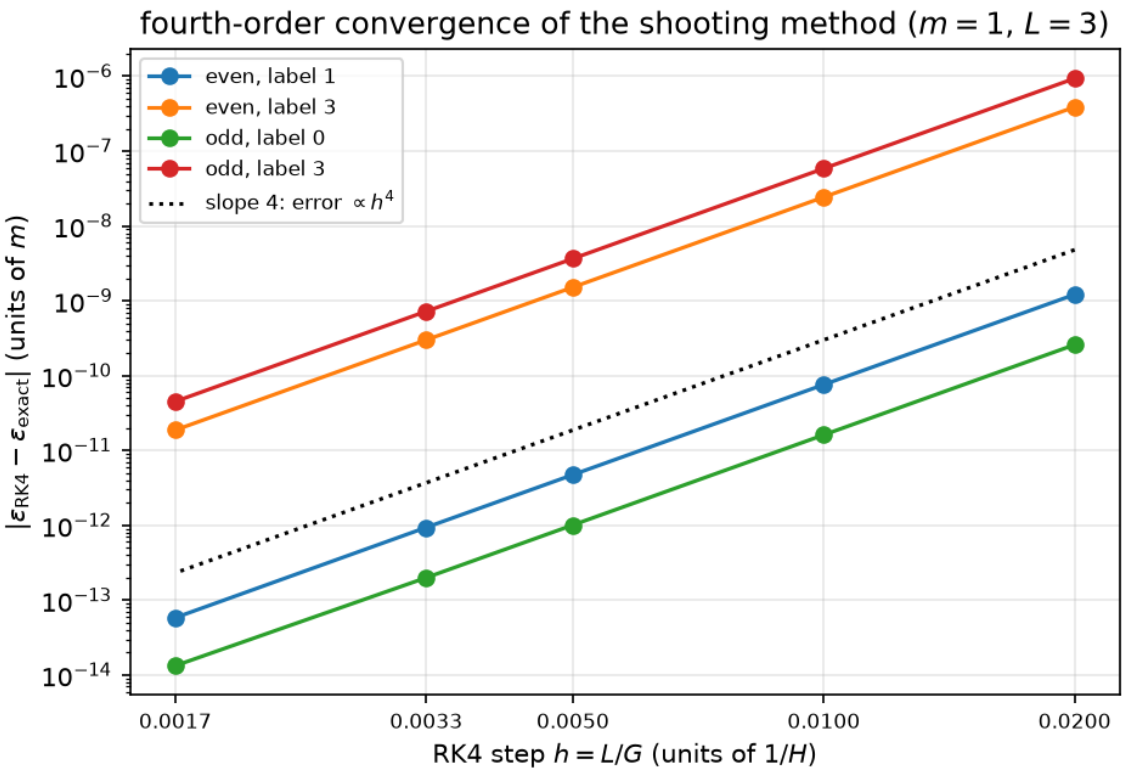

Figure 392: The error of four numerical levels ($m = 1$, $L = 3$; even labels 1 and 3, odd labels 0 and 3) against the RK4 step $h = L/G$ for $G = 150, 300, 600, 900, 1800$, on logarithmic axes (both in units of $m = H = 1$). The points fall on lines parallel to the dotted line of slope 4: halving the step divides the error by 16, the order of the classical Runge-Kutta method; higher levels have larger errors because their orbitals oscillate faster. (Notebook 14b, figure 4.)

```
    _, (theta_n, a_n, b_n) = shoot(eps, k, j, M=M, L=L, G=G, H=H, a4=a4, tip=tip,
                                   keep=True)
    nf, h = 2 * G + 1, L / G
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    K_fine = k * np.exp(-a4) * np.exp(-H * y_fine)
    a_f, b_f = np.zeros(nf), np.zeros(nf)
    a_f[0::2], b_f[0::2] = a_n, b_n  # the node values
    da = M * a_f - (K_fine + j * eps) * b_f  # the derivatives at the nodes
    db = (j * eps - K_fine) * a_f - M * b_f
    for f in range(1, nf, 2):  # cubic Hermite value at each midpoint
        a_f[f] = 0.5 * (a_f[f - 1] + a_f[f + 1]) + h / 8.0 * (da[f - 1] - da[f + 1])
        b_f[f] = 0.5 * (b_f[f - 1] + b_f[f + 1]) + h / 8.0 * (db[f - 1] - db[f + 1])
    weights = np.full(nf, 2.0)  # Simpson: 1, 4, 2, 4, ..., 4, 1 times (h/2)/3
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    weights *= 0.5 * h / 3.0
    norm = math.sqrt(float(np.sum(weights * (a_f ** 2 + b_f ** 2))))
    return y_fine, a_f / norm, b_f / norm, weights


y_f, a_zero, b_zero, simpson = orbital(numeric[(1.0, 3.0, "even", 0)])
a_exact = math.sqrt(2.0 / (1.0 - math.exp(-6.0))) * np.exp(y_f)
deviation = float(np.max(np.abs(a_zero - a_exact)))
check(numeric[(1.0, 3.0, "even", 0)] == 0.0 and np.all(b_zero == 0.0)
      and deviation < 1e-9 and record_check("free_zero_mode_exact", (RUST_REPORT,)),
      "zero mode: eps = 0 and b = 0 exactly, a = sqrt(2M/(1 - e^(-2ML))) e^(My)",
      record=f"{RUST_REPORT}, check free_zero_mode_exact")


def exact_orbital(parity, label, m_=1.0, L_=3.0):
    """The exact orbital of section 7 (j = +1), normalised with Simpson."""
    level = exact[(m_, L_, parity, label)]
    p_ = math.sqrt(level ** 2 - m_ ** 2)
    if parity == "even":
        b_e = np.sin(p_ * y_f)
        a_e = (p_ * np.cos(p_ * y_f) + m_ * b_e) / level
    else:
        b_e = np.sin(p_ * (y_f + L_))
        a_e = (p_ * np.cos(p_ * (y_f + L_)) + m_ * b_e) / level
    sign = 1.0 if a_e[0] > 0 else -1.0  # the sign with a(-L) > 0
    norm = math.sqrt(float(np.sum(simpson * (a_e ** 2 + b_e ** 2))))
    return sign * a_e / norm, sign * b_e / norm


shown = {}
worst_orbital = 0.0
for parity, label in (("even", 1), ("odd", 0)):
    _, a_num, b_num, _ = orbital(numeric[(1.0, 3.0, parity, label)])
    a_ex, b_ex = exact_orbital(parity, label)
    worst_orbital = max(worst_orbital, float(np.max(np.abs(a_num - a_ex))),
                        float(np.max(np.abs(b_num - b_ex))))
    shown[(parity, label)] = (a_num, b_num, a_ex, b_ex)
check(worst_orbital < 1e-8,
      "the even label 1 and odd label 0 orbitals equal the exact ones to 1e-8")
```

Out [12]:

```
PASS zero mode: eps = 0 and b = 0 exactly, a = sqrt(2M/(1 - e^(-2ML))) e^(My)
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_zero_mode_exact
PASS the even label 1 and odd label 0 orbitals equal the exact ones to 1e-8
```

Text cell:

```
The next cell draws the three orbitals of $m = 1$, $L = 3$: the zero mode
(largest at the brane, $b = 0$), the even level 1 and the odd level 0, each
with its two components $a$ (solid) and $b$ (dashed), and the exact curves as
dots on top.
```

In [13]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.8), sharey=True)
panels = [("zero mode, $\\varepsilon = 0$", a_zero, b_zero, a_exact,
           np.zeros_like(a_exact)),
          (f"even label 1, $\\varepsilon = {lowest_even:.4f}$",) + shown[("even", 1)],
          (f"odd label 0, $\\varepsilon = {lowest_odd:.4f}$",) + shown[("odd", 0)]]
for ax, (title, a_num, b_num, a_ex, b_ex) in zip(axes, panels):
    ax.plot(y_f, a_num, color="C0", label="$a$ (numerical)")
    ax.plot(y_f, b_num, "--", color="C1", label="$b$ (numerical)")
    ax.plot(y_f[::60], a_ex[::60], ".", color="black", label="exact")
    ax.plot(y_f[::60], b_ex[::60], ".", color="black")
    ax.set_title(title, fontsize=9)
    ax.set_xlabel("$y$ (units of $1/H$)")
axes[0].set_ylabel("orbital component (normalised)")
axes[0].legend(fontsize=8, loc="upper left")
save_figure(fig, "orbitals",
            "The normalised orbitals $\\chi = (a, ib)$ of three free $k = 0$ "
            "levels for $m = 1$, $L = 3$ against $y$ from the tip $-3$ to the "
            "brane $0$ (units of $1/H$): left the brane zero mode "
            "$a = \\sqrt{2/(1 - e^{-6})}\\,e^{y}$, $b = 0$, concentrated at the "
            "brane; middle the even level 1 with $b(0) = b(-3) = 0$; right the "
            "odd level 0 with $a(0) = 0$ and $b(-3) = 0$. Solid lines $a$, dashed "
            "lines $b$ from the RK4 shooting, black dots the exact solutions.")
```

Out [13]:

```
Figure 14b.5 saved as Revision/textbook/figures/14b_5_orbitals.png
```

Text cell:

```
The next cell draws how the Pruefer angle counts. For the even levels with the
labels 0 to 3 and the odd levels with the labels 0 to 2 ($m = 1$, $L = 3$,
$k = 0$, $j = +1$) it plots $\theta(y)/\pi$ along the hidden coordinate: every
curve starts at 0 at the tip and ends exactly on its target $l$ (even) or
$l + \frac12$ (odd) at the brane. The label is the number of half-turns the
point $(a, b)$ makes on its way.
```

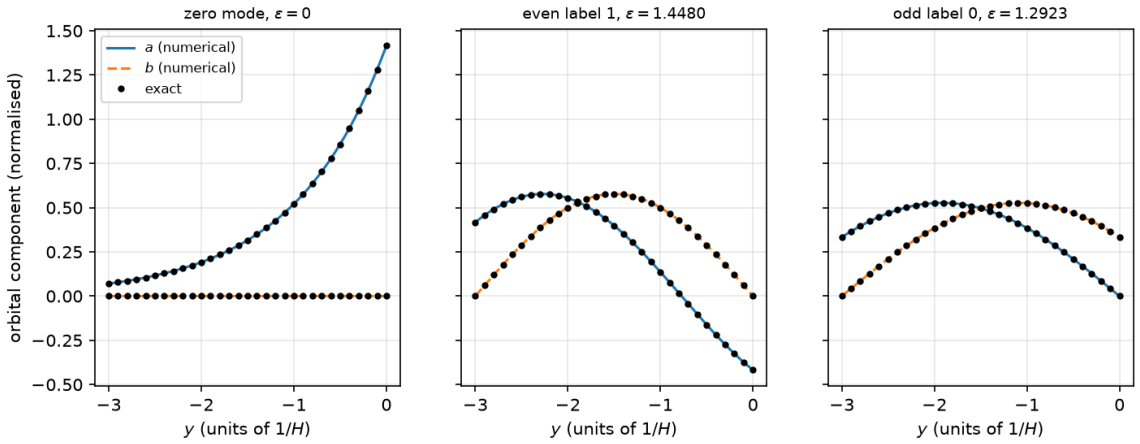

Figure 393: The normalised orbitals $\chi = (a, ib)$ of three free $k = 0$ levels for $m = 1$, $L = 3$ against y from the tip -3 to the brane 0 (units of $1/H$): left the brane zero mode $a = \sqrt{2/(1 - e^{-6})}\, e^{y}$, $b = 0$, concentrated at the brane; middle the even level 1 with $b(0) = b(-3) = 0$; right the odd level 0 with $a(0) = 0$ and $b(-3) = 0$. Solid lines a, dashed lines b from the RK4 shooting, black dots the exact solutions. (Notebook 14b, figure 5.)

In [14]:

```
picks = [("even", l_) for l_ in range(4)] + [("odd", l_) for l_ in range(3)]
energies = np.array([numeric[(1.0, 3.0, p_, l_)] for p_, l_ in picks])
_, (theta_path, _, _) = shoot(energies, 0.0, 1.0, keep=True)
y_nodes = np.linspace(-3.0, 0.0, 901)
end_values = theta_path[-1] / np.pi
goals = np.array([l_ + (0.0 if p_ == "even" else 0.5) for p_, l_ in picks])
check(np.max(np.abs(end_values - goals)) < 1e-9,
      "the Pruefer angle ends on its target l pi or (l + 1/2) pi for every level")
fig, ax = plt.subplots(figsize=(8.0, 4.6))
for column, (p_, l_) in enumerate(picks):
    style = "-" if p_ == "even" else "--"
    ax.plot(y_nodes, theta_path[:, column] / np.pi, style,
            label=f"{p_}, label {l_}")
ax.set_xlabel("$y$ (units of $1/H$; tip at $-3$, brane at $0$)")
ax.set_ylabel("Pruefer angle $\\theta(y)/\\pi$")
ax.set_title("the Pruefer angle counts the half-turns ($m = 1$, $L = 3$, $k = 0$)")
ax.legend(fontsize=8, ncol=2, loc="upper left")
save_figure(fig, "pruefer_angle",
            "The Pruefer angle $\\theta(y) = \\mathrm{atan2}(b, a)$ in units of "
            "$\\pi$ along the hidden coordinate $y$ (units of $1/H$) for the even "
            "levels with labels 0 to 3 (solid) and the odd levels with labels 0 "
            "to 2 (dashed), $m = 1$, $L = 3$, $k = 0$, $j = +1$. Every curve starts "
            "at $0$ at the tip (the condition $b(-3) = 0$) and ends exactly at its "
            "target at the brane, $l$ for even and $l + 1/2$ for odd parity; the "
            "zero mode stays at $0$ all the way.")
```

Out [14]:

```
PASS the Pruefer angle ends on its target l pi or (l + 1/2) pi for every level
```

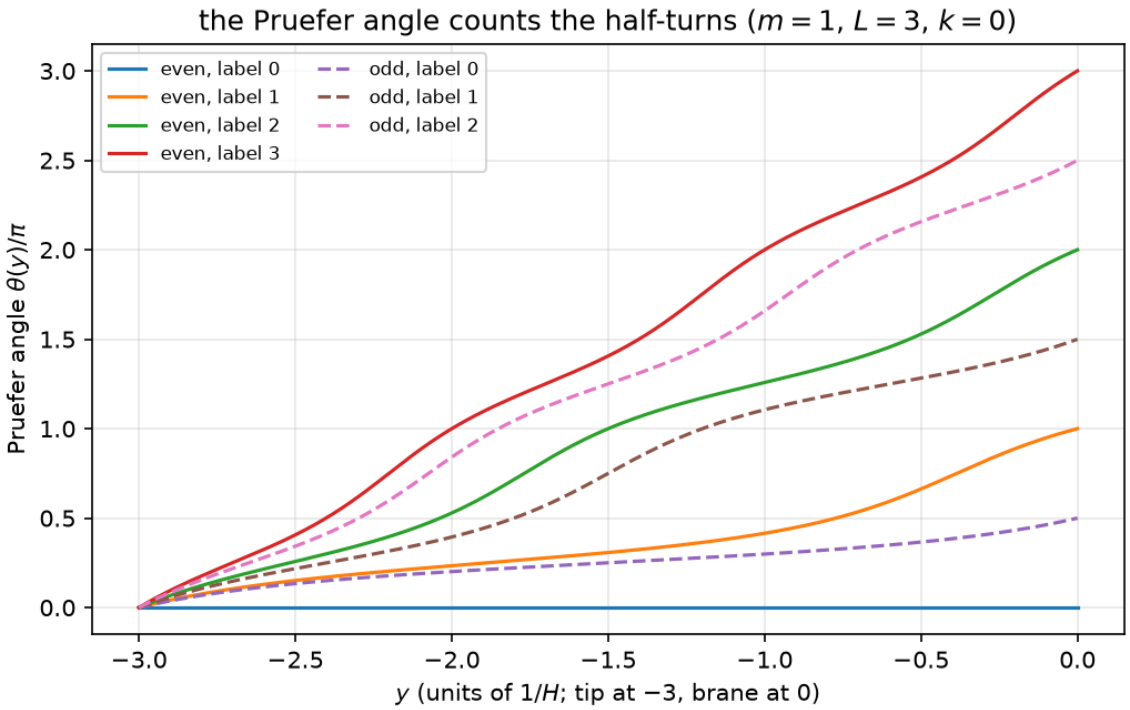

Figure 394: The Pruefer angle $\theta(y) = \mathrm{atan2}(b, a)$ in units of π along the hidden coordinate y (units of $1/H$) for the even levels with labels 0 to 3 (solid) and the odd levels with labels 0 to 2 (dashed), $m = 1$, $L = 3$, $k = 0$, $j = +1$. Every curve starts at 0 at the tip (the condition $b(-3) = 0$) and ends exactly at its target at the brane, l for even and $l + 1/2$ for odd parity; the zero mode stays at 0 all the way. (Notebook 14b, figure 6.)

Out [14]:

```
Figure 14b.6 saved as Revision/textbook/figures/14b_6_pruefer_angle.png
```

Text cell:

```
## 12. The last check

The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [15]:

```
figure_names = ["14b_1_odd_condition.png", "14b_2_shooting_function.png",
                "14b_3_spectrum_ladder.png", "14b_4_rk4_convergence.png",
                "14b_5_orbitals.png", "14b_6_pruefer_angle.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all six figure files of this notebook exist")
all_checks_passed()
```

Out [15]:

```
PASS all six figure files of this notebook exist
ALL 20 CHECKS PASSED (notebook 14b)
```

Text cell:

```
## 13. What this notebook showed

- The block equation of the Kohn-Sham model is real in the variables
  $\chi = (a, ib)$ (PROVED).
- The ASSUMED Z2 mirror brane gives the two parities $b(0) = 0$ and $a(0) = 0$;
  the mirror map turns the mass $M(y)$ into $-M(-y)$, so a mirror-symmetric
  doubled problem has the mass $-m$ in the mirror copy; the chosen regular tip
  gives $b(-L) = 0$; both ends kill the current and make $h_j$ self-adjoint
  (PROVED; reproduces the Revision checks).
- At $k = 0$ without interaction the levels are exactly $0$ (the brane zero mode
  $e^{My}$), $\pm\sqrt{M^2 + (n\pi/L)^2}$ (even) and $\pm\sqrt{M^2 + p^2}$ with
  $\tan(pL) = -p/M$ (odd); for $m = 1$, $L = 3$ the lowest positive levels are
  $1.2922928281\,m$ (odd) and $1.4479719304\,m$ (even) (PROVED).
- RK4 shooting with the Pruefer angle finds every level by its label; the 54
  numerical levels equal the Rust solver's to about $10^{-14}$, their errors are
  within the Rust tolerances, and halving the step divides the error by 16.00
  (median over 39 levels): the method is of fourth order (COMPUTED, error
  measured).
- The numerical orbitals equal the exact ones; the zero mode is exactly
  $\varepsilon = 0$, $b = 0$.
```

14.18 Line-by-line walk-through of Notebook 14b

The notebook has 15 code cells, In [1] to In [15]. As in Section 14.11, a line or a small group of lines is quoted and then explained; docstrings and the long caption texts are left out of the quotations (the captions are printed under the figures in Section 14.17).

In [1], the set-up cell. Its code is, line for line, the code of In [1] of Notebook 14a, which Section 14.11 explains, with one exception: the line

```
NOTEBOOK_ID = "14b"  # this notebook: chapter 14, example b
```

gives this notebook its name, so that its figure files start with 14b and its captions go into the file `Revision/textbook/figures/14b.captions.json`. Its comment lines are the run instructions of Section 14.16. Out [1] is the line Set-up of notebook 14b complete.

In [2], the real form.

```
import math  # exp, cos, sin of single numbers

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra with symbols
```

`math` is the module of Python for single numbers (square roots, π, sine, cosine ...); numpy computes with whole arrays of floating-point numbers at once; sympy computes exactly with symbols.

```
PY_REPORT = "Revision/kohn_sham/reports/ks-theory-python.json"
WL_REPORT = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
RUST_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
```

The names of three Revision reports: the sympy and the WolframScript checks of the theory, and the report of the Rust solver.

```
def record_check(name, reports=(PY_REPORT, WL_REPORT)):
    verdicts = []
    for report in reports:
        data = json.loads(repository_file(report).read_text(encoding="utf-8"))
        found = [c["verdict"] for c in data["checks"] if c["name"] == name]
        verdicts.append(found[0] if found else "absent")
    return verdicts[0] == "PASS" and all(v in ("PASS", "absent") for v in verdicts)
```

The same idea as `record_check` of Notebook 14a, with one improvement: the reports to read are a parameter with a **default value** (`reports=(PY_REPORT, WL_REPORT)` is used when the call gives no second argument). A call can name other reports, for example `record_check("free_zero_mode_exact", (RUST_REPORT,))`; the comma after `RUST_REPORT` makes a **tuple** with one element. The function collects the verdict of the check `name` in every report (or the word absent) and returns true when the first report says PASS and every report says PASS or absent.

```
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices
s2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
I2 = sp.eye(2)
a, b, M, K, e, v = sp.symbols("a b M K epsilon v", real=True)
ok_all = True
```

The three Pauli matrices (in this notebook `s2` is σ_2), the 2×2 unit matrix, six real symbols (a, b, the mass M, $K = \kappa k$, e for ε and v) and a flag.

```
for j in (1, -1):
    N = M * s3 - K * s2 + sp.I * j * (e - v) * s1  # the matrix of chi' = N chi
    derivative = sp.expand(N * sp.Matrix([a, sp.I * b]))  # N times (a, i b)
    a_prime = M * a - (K + j * (e - v)) * b  # the real form
    b_prime = (j * (e - v) - K) * a - M * b
    ok = (sp.expand(derivative[0] - a_prime) == 0
          and sp.expand(derivative[1] - sp.I * b_prime) == 0)
    ok_all &= ok
    say(f"j = {j:+d}: d chi/dy = N chi with chi = (a, i b) gives the real form: "
        f"{ok}")
```

For both block types: the matrix N; the product $N\chi$ for $\chi = (a, ib)$, multiplied out by `sp.expand`; the right sides of the real form of Section 14.12; and the test that the first entry of $N\chi$ is a' and the second is ib'. The format `{j:+d}` prints the whole number j with its sign ($+1$ or -1); two strings written next to each other are joined into one. Out [2] shows True twice.

```
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
say("record: " + theory["blockEquation"]["realForm"])
check(ok_all, "the real form da/dy = M a - (K + j(eps - v)) b, "
      "db/dy = (j(eps - v) - K) a - M b")
```

The Revision theory record is read and its entry blockEquation.realForm is printed, so that the reader can compare it with the derived form (Out [2], third line); then the first PASS line.

In [3], the boundary conditions.

```
y = sp.symbols("y", real=True)
H, a0, k = sp.symbols("H a0 k", real=True)
Mf = sp.Function("M", real=True)(y)  # a mass that depends on y
vf = sp.Function("v", real=True)(y)  # a potential that depends on y
kap = sp.exp(-H * y - a0)  # the momentum weight kappa(y)
```

Symbols for y, H, the slice $a_{4,0}$ (`a0`) and k, and two unknown real functions $M(y)$ and $v(y)$: everything below then holds for every mass profile and every potential, as in the self-consistent problem, not only for constants. `kap` is $\kappa(y)$.

```
def N_of(j, M_, v_, kap_):
    return M_ * s3 - kap_ * k * s2 + sp.I * j * (e - v_) * s1
```

The matrix N as a function of the block type, the mass, the potential and the weight.

```
ok_mirror = True
for j in (1, -1):  # chi~(y) = s3 chi(-y) solves chi~' = -s3 N(-y) s3 chi~
    mirrored = -s3 * N_of(j, Mf.subs(y, -y), vf.subs(y, -y), kap.subs(y, -y)) * s3
    target = N_of(j, -Mf.subs(y, -y), vf.subs(y, -y), kap.subs(y, -y))
    ok_mirror &= sp.simplify(mirrored - target).is_zero_matrix
check(ok_mirror and record_check("bc_mirror_map_PA"),
      "the mirror map chi(y) -> sigma3 chi(-y) turns M(y) into -M(-y)",
      record=f"{PY_REPORT}, check bc_mirror_map_PA")
```

`subs(y, -y)` replaces y by $-y$, so `Mf.subs(y, -y)` is $M(-y)$. `mirrored` is $-\sigma_3 N(-y)\sigma_3$, the matrix of the equation that $\tilde{\chi}(y) = \sigma_3\chi(-y)$ solves (Section 14.12), and `target` is N with the mass $-M(-y)$, the potential $v(-y)$ and the weight $\kappa(-y)$; they must be equal for both block types.

```
even = {name: (s3 * X * s3 - X).is_zero_matrix
        for name, X in {"n": I2, "S": s2, "Q": s3, "current": s1}.items()}
say(f"even under the mirror map: {even}")
check(even == {"n": True, "S": False, "Q": True, "current": False}
      and record_check("bc_mirror_parities_of_densities"),
      "n and Q are even, S and the current odd: the mirror copy carries -m",
      record=f"{PY_REPORT}, check bc_mirror_parities_of_densities")
```

A **dictionary comprehension** (braces with `key: value for ...`): for each density and its 2×2 matrix (the number density with 1, S with σ_2, Q with σ_3, the current with σ_1; `.items()` gives the

pairs of a dictionary) the value is true when $\sigma_3 X \sigma_3 = X$. That is the condition for the density to be even: the mirrored orbital has at y the density $\chi(-y)^\dagger \sigma_3 X \sigma_3 \chi(-y)$, which equals the density of the original orbital at $-y$ exactly when $\sigma_3 X \sigma_3 = X$. Out [3] prints the dictionary.

```
c1f, c2f = sp.Function("chi1")(y), sp.Function("chi2")(y)
current = sp.expand((sp.Matrix([c1f, c2f]).H * s1 * sp.Matrix([c1f, c2f]))[0])
check(current.subs(c2f, 0) == 0 and current.subs(c1f, 0) == 0
      and record_check("bc_brane_parity_conditions"),
      "both brane parities (chi2(0) = 0 or chi1(0) = 0) kill the current",
      record=f"{PY_REPORT}, check bc_brane_parity_conditions")
```

Two unknown complex functions χ_1, χ_2; the current $\chi^\dagger \sigma_1 \chi$ (a 1×1 matrix, whose single entry is `[0]`); putting $\chi_2 = 0$ (even parity) or $\chi_1 = 0$ (odd parity) makes it zero.

```
phi = sp.Matrix([sp.Function("phi1")(y), sp.Function("phi2")(y)])
chi = sp.Matrix([c1f, c2f])
ok_adjoint = True
for j in (1, -1):
    def h_on(w, j=j):  # h_j applied to the column w
        return j * (-sp.I * s1 * sp.diff(w, y) + Mf * s2 * w + kap * k * s3 * w) \
            + vf * w
    difference = (phi.H * h_on(chi))[0] - (h_on(phi).H * chi)[0]
    boundary = sp.diff(-sp.I * j * (phi.H * s1 * chi)[0], y)
    ok_adjoint &= sp.simplify(sp.expand(difference - boundary)) == 0
check(ok_adjoint and record_check("bc_self_adjoint_boundary_term"),
      "phi^dag h chi - (h phi)^dag chi = d/dy[-i j phi^dag sigma1 chi]",
      record=f"{PY_REPORT}, check bc_self_adjoint_boundary_term")
```

Two columns of unknown functions φ and χ. Inside the loop, `h_on(w)` applies $h_j = j[-i\sigma_1 \, d/dy + M\sigma_2 + \kappa k \sigma_3] + v$ to a column; the argument `j=j` stores the current value of `j` in the function (a function defined in a loop otherwise reads the loop variable only when it is called). `difference` is $\varphi^\dagger h_j \chi - (h_j \varphi)^\dagger \chi$, `boundary` is $\frac{d}{dy}[-ij\varphi^\dagger \sigma_1 \chi]$; they must be equal (Section 14.12).

```
th = sp.symbols("theta", real=True)
Q = sp.cos(th) * s3 + sp.sin(th) * s2  # the tip family
check(sp.simplify(Q * Q - I2).is_zero_matrix and Q.H == Q
      and sp.simplify(Q * s1 + s1 * Q).is_zero_matrix and record_check("bc_tip_family"),
      "the tip family Q(theta): Q^dag = Q, Q^2 = 1, Q sigma1 = -sigma1 Q",
      record=f"{PY_REPORT}, check bc_tip_family")
```

The three facts of the tip family: $Q^2 = 1$, $Q^\dagger = Q$, $Q\sigma_1 + \sigma_1 Q = 0$.

```
kp = sp.symbols("kappa", positive=True)
ok_current = True
for j in (1, -1):
    Nn = M * s3 - kp * k * s2 + sp.I * j * (e - v) * s1
    ok_current &= sp.simplify(Nn.H * s1 + s1 * Nn).is_zero_matrix
    density_changes = not sp.simplify(Nn.H + Nn).is_zero_matrix
check(ok_current and density_changes and record_check("bc_current_conserved_along_y"),
      "the current is constant along y on every solution, the density is not",
      record=f"{PY_REPORT}, check bc_current_conserved_along_y")
```

For both types: $N^\dagger\sigma_1 + \sigma_1 N = 0$ (the current is constant) and $N^\dagger + N \neq 0$ (the density is not). The variable `density_changes` is set anew in each pass; the check uses the value of the last pass, $j = -1$, which is the same as for $j = +1$. Out [3] shows six PASS lines.

In [4], the exact solutions and the record.

```
Lc = sp.symbols("L", positive=True)
n_int = sp.symbols("n", integer=True, positive=True)
ok_exact = sp.simplify(sp.diff(sp.exp(M * y), y) - M * sp.exp(M * y)) == 0  # step 5
```

The symbol L (named `Lc` because the plain name `L` is used for numbers below) and a positive whole number n. The first test is the zero mode: the derivative of e^{My} is Me^{My}, so $a = e^{My}$ solves $a' = Ma$ (step 5 of the notebook's section 7 is Step 2 of Section 14.13, case $\varepsilon = 0$).

```
for j in (1, -1):  # step 4: b = sin(n pi y/L), a = (b' + M b)/(j eps)
    p_n = n_int * sp.pi / Lc
    eps_n = sp.sqrt(M ** 2 + p_n ** 2)
    b_n = sp.sin(p_n * y)
    a_n = (sp.diff(b_n, y) + M * b_n) / (j * eps_n)
    ok_exact &= sp.simplify(sp.diff(a_n, y) - (M * a_n - j * eps_n * b_n)) == 0
    ok_exact &= b_n.subs(y, 0) == 0 and sp.simplify(b_n.subs(y, -Lc)) == 0
```

The even levels of Step 2: $b = \sin(n\pi y/L)$, $a = (b' + Mb)/(j\varepsilon_n)$. The second equation of the real form, $b' = j\varepsilon a - Mb$, holds by the construction of a; the loop checks the first one, $a' = Ma - j\varepsilon b$, and the two conditions $b(0) = 0$ and $b(-L) = \sin(-n\pi) = 0$ (sympy knows that the sine of a whole multiple of π is zero, because n was declared a whole number).

```
p = sp.symbols("p", positive=True)  # step 6 (j = +1)
b_odd = sp.sin(p * (y + Lc))
eps_odd = sp.sqrt(M ** 2 + p ** 2)
a_odd = (sp.diff(b_odd, y) + M * b_odd) / eps_odd
ok_exact &= sp.simplify(sp.diff(a_odd, y) - (M * a_odd - eps_odd * b_odd)) == 0
ok_exact &= sp.simplify(a_odd.subs(y, 0) * eps_odd
                        - (p * sp.cos(p * Lc) + M * sp.sin(p * Lc))) == 0
check(ok_exact and record_check("bc_exact_k0_spectra"),
      "the zero mode, the even levels and the odd condition tan(pL) = -p/M",
      record=f"{PY_REPORT}, check bc_exact_k0_spectra")
```

The odd levels of Step 3 for $j = +1$: $b = \sin(p(y + L))$ vanishes at $y = -L$; a is built as before; the first equation is checked; and $\varepsilon\, a(0) = p\cos(pL) + M\sin(pL)$, so $a(0) = 0$ is exactly the condition $f(p) = 0$.

```
def odd_roots(m, L, count):
    roots = []
    for n in range(count):
        lo, hi = (n + 0.5) * math.pi / L, (n + 1.0) * math.pi / L
        f = lambda q: m * math.sin(q * L) + q * math.cos(q * L)
        f_lo = f(lo)
        for _ in range(200):
```

```
            mid = 0.5 * (lo + hi)
            if mid <= lo or mid >= hi:  # the interval cannot shrink any more
                break
            if (f(mid) > 0.0) == (f_lo > 0.0):
                lo = mid
            else:
                hi = mid
        roots.append(0.5 * (lo + hi))
    return roots
```

The first `count` roots of $f(p) = m\sin(pL) + p\cos(pL)$ by **bisection**, one in each interval of Step 4. `f` is the function as a `lambda`; `f_lo` its value at the left end. The inner loop runs at most 200 times (the name `_` is used for a loop counter whose value is not needed); it stops earlier when the midpoint is no longer strictly inside the interval, which happens when the two ends are neighbouring floating-point numbers. If f has the same sign at the midpoint as at the left end, the root lies in the right half and `lo` moves to the midpoint; otherwise `hi` does. (The sign at the left end never changes, because `lo` only moves to points of the same sign.)

```
def exact_level(m, L, parity, label):
    if parity == "even":
        if label == 0:
            return 0.0
        return math.copysign(math.sqrt(m * m + (label * math.pi / L) ** 2), label)
    roots = odd_roots(m, L, 6)
    if label >= 0:
        return math.sqrt(m * m + roots[label] ** 2)
    return -math.sqrt(m * m + roots[-label - 1] ** 2)
```

The exact level of a given parity and label, in the label convention of Section 14.13. `math.copysign(x, label)` is the size of x with the sign of the label.

```
CASES = [(1.0, 3.0), (1.0, 2.0), (2.0, 3.0)]  # (m, L)
LABELS = list(range(-3, 6))  # the labels -3 ... 5
record_rows = repository_file(
    "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
).read_text(encoding="utf-8").splitlines()
header = record_rows[0].split(",")
record = [dict(zip(header, row.split(","))) for row in record_rows[1:]]
```

The three cases (m, L) and the nine labels: with two parities, $3 \times 9 \times 2 = 54$ levels. The record of the Rust solver is a CSV file with the columns m, L, parity, label, eps_numeric, eps_analytic and difference; each row becomes a dictionary from column name to text, as in In [18] of Notebook 14a.

```
exact = {}  # (m, L, parity, label) -> exact level
for m_, L_ in CASES:
    for label in LABELS:
        for parity in ("even", "odd"):
            exact[(m_, L_, parity, label)] = exact_level(m_, L_, parity, label)
worst = max(abs(exact[(float(r["m"]), float(r["L"]), r["parity"], int(r["label"]))]
                - float(r["eps_analytic"])) for r in record)
```

A dictionary whose keys are tuples $(m, L, \text{parity}, \text{label})$ holds the 54 exact levels. `worst` is the largest difference between them and the column eps_analytic of the record (each text is turned into a number with `float` or `int`).

```
say(f"(m, L) = (1, 3): odd roots p = "
    + ", ".join(f"{q:.6f}" for q in odd_roots(1.0, 3.0, 3)) + ", ...")
lowest_odd = exact[(1.0, 3.0, "odd", 0)]  # the lowest odd level, m = 1, L = 3
lowest_even = exact[(1.0, 3.0, "even", 1)]  # the lowest positive even level
report("lowest odd level for m = 1, L = 3", f"{lowest_odd:.12f}", "m")
report("lowest positive even level for m = 1, L = 3", f"{lowest_even:.12f}", "m")
check(len(record) == 54 and worst < 1e-12,
      "the 54 exact levels equal the column eps_analytic of the record",
      record="Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, eps_analytic")
```

`", ".join(...)` joins the three roots, each written with 6 decimals, into one text separated by commas. Then the two RESULT lines, $1.292292828069\,m$ and $1.447971930402\,m$, and the check that the record has 54 rows that agree with the exact levels to better than 10^{-12}. Out [4] shows two PASS lines.

In [5], figure 1.

```
p_values = np.linspace(0.0, 7.0, 1401)
f_values = 1.0 * np.sin(3.0 * p_values) + p_values * np.cos(3.0 * p_values)
roots_13 = odd_roots(1.0, 3.0, 6)
fig, ax = plt.subplots(figsize=(8.0, 4.0))
for n in range(6):  # the intervals in which the roots lie
    ax.axvspan((n + 0.5) * math.pi / 3.0, (n + 1.0) * math.pi / 3.0, color="C0",
               alpha=0.08)
```

1401 values of p from 0 to 7, the function $f(p)$ for $M = 1$, $L = 3$, and its first six roots. `axvspan(x1, x2, ...)` shades the vertical strip between x_1 and x_2; `alpha=0.08` makes it almost transparent. The six strips are the intervals $((n + \frac{1}{2})\pi/3, (n + 1)\pi/3)$.

```
ax.plot(p_values, f_values, color="C0", label="$f(p) = M\\sin(pL) + p\\cos(pL)$")
ax.plot(roots_13, [0.0] * 6, "o", color="C3", label="roots $p_0, p_1, \\dots$")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("$p$ (wave number along $y$, units of $H$)")
ax.set_ylabel("$f(p)$")
ax.set_title("the odd-parity condition $\\tan(pL) = -p/M$ for $M = 1$, $L = 3$")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "odd_condition",
            "The odd-parity level condition for $M = 1$ and $L = 3$: the function "
            ...)
```

The curve, the six roots as red dots on the axis (`[0.0] * 6` is a list of six zeros; the format string `"o"` draws dots without a line), the horizontal axis line, labels, title, legend and saving. **What figure 14b.1 shows.** The function $f(p)$ oscillates with a growing amplitude (the term $p\cos(pL)$ grows with p). It starts at $f(0) = 0$, which is not a level ($p = 0$ gives $b = 0$), and crosses zero once in every shaded strip and nowhere else, at $p_0 = 0.8185$, $p_1 = 1.744$, $p_2 = 2.735$, ... as Step 4 of Section 14.13 proved; each crossing is a pair of odd levels $\pm\sqrt{1 + p_n^2}$.

In [6], the Pruefer angle and the shooting tools.

```
r_s, t_s = sp.symbols("r t", positive=True)  # polar coordinates of (a, b)
a_p = r_s * sp.cos(t_s)
b_p = r_s * sp.sin(t_s)
j_s = sp.symbols("j")
num = a_p * ((j_s * e - K) * a_p - M * b_p) - b_p * (M * a_p - (K + j_s * e) * b_p)
pruefer = j_s * e - K * sp.cos(2 * t_s) - M * sp.sin(2 * t_s)
check(sp.simplify(num / r_s ** 2 - pruefer) == 0,
      "d theta/dy = j(eps - v) - K cos 2theta - M sin 2theta")
```

The point $(a, b) = (r\cos\theta, r\sin\theta)$ with symbols r and θ (`t_s`), a symbol j, the expression $ab' - ba'$ with the real form inserted (`num`, written with $v = 0$, so that e stands for $\varepsilon - v$), and the claimed θ'. The check: $(ab' - ba')/r^2$ equals it (Section 14.14).

```
def shoot(eps, k=0.0, j=1.0, M=1.0, L=3.0, G=900, H=1.0, a4=0.0, tip=0.0,
          keep=False):
    eps, k, j = np.broadcast_arrays(np.asarray(eps, dtype=float),
                                    np.asarray(k, dtype=float),
                                    np.asarray(j, dtype=float))
```

`shoot` integrates the real form from the tip to the brane for many energies at once and returns $\Phi = j\theta(0)$. Its parameters have default values (the canonical $M = 1$, $L = 3$, $G = 900$, $H = 1$, slice 0, tip angle 0), so a call can name only what differs. `np.asarray` turns a single number or a list into a numpy array; `np.broadcast_arrays` brings the arrays to a common shape (a single number is repeated), so that `eps`, `k` and `j` may each be one number or an array of the same length.

```
    nf = 2 * G + 1  # the fine grid: nodes and step midpoints
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    e_minus_Hy = np.exp(-H * y_fine)  # e^(-Hy) on the fine grid
    kk = k * np.exp(-a4)  # k e^(-a4,0): kappa k = kk e^(-Hy)
    h = L / G
```

The fine grid of $2G + 1$ points from $y = -L$ to $y = 0$ (`np.arange(nf)` is $0, 1, \ldots, 2G$, so the first point is $-L$ and the last 0); e^{-Hy} on it; the redshifted momentum $ke^{-a_{4,0}}$, so that $K = \kappa k$ is `kk` times e^{-Hy}; and the step $h = L/G$.

```
    a_ = np.full(eps.shape, math.cos(0.5 * tip))  # the tip condition
    b_ = np.full(eps.shape, math.sin(0.5 * tip))
    theta = np.full(eps.shape, 0.5 * tip)
    raw = np.arctan2(b_, a_)
    je = j * eps
    path = [(theta, a_, b_)]
```

The starting point $(a, b) = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2})$ for every energy (`np.full(shape, value)` is an array filled with one value), the starting angle $\theta(-L) = \theta_{\text{tip}}/2$, `raw`, the angle as `np.arctan2` returns it (between $-\pi$ and π), the products $j\varepsilon$, and a list that will keep the path when it is asked for.

```
    for i in range(G):
        K0, K1, K2 = (kk * e_minus_Hy[2 * i], kk * e_minus_Hy[2 * i + 1],
                      kk * e_minus_Hy[2 * i + 2])  # K at y, y + h/2, y + h
        p1a, p1b = M * a_ - (K0 + je) * b_, (je - K0) * a_ - M * b_
        a2, b2 = a_ + 0.5 * h * p1a, b_ + 0.5 * h * p1b
        p2a, p2b = M * a2 - (K1 + je) * b2, (je - K1) * a2 - M * b2
        a3, b3 = a_ + 0.5 * h * p2a, b_ + 0.5 * h * p2b
        p3a, p3b = M * a3 - (K1 + je) * b3, (je - K1) * a3 - M * b3
        a4_, b4_ = a_ + h * p3a, b_ + h * p3b
        p4a, p4b = M * a4_ - (K2 + je) * b4_, (je - K2) * a4_ - M * b4_
        a_ = a_ + h / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)
        b_ = b_ + h / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
```

One RK4 step per pass, for all energies at once. Step i goes from the node $2i$ of the fine grid to the node $2i + 2$; `K0`, `K1`, `K2` are K at its start, its midpoint and its end. The four slopes of RK4 (Chapter 2): `p1` at the start, `p2` at the midpoint with half a step along `p1`, `p3` at the midpoint with half a step along `p2`, `p4` at the end with a whole step along `p3`; each slope is the right side of the real form, $(Ma - (K + j\varepsilon)b,\ (j\varepsilon - K)a - Mb)$, with $v = 0$. The new point adds $h/6$ times the weighted sum with the weights $1, 2, 2, 1$. (The name `a4_` with a final underscore is the trial point of the fourth slope; `a4` without it is the slice.)

```
        new = np.arctan2(b_, a_)
        step = new - raw  # the change of the angle, brought into (-pi, pi]
        step = np.where(step > np.pi, step - 2 * np.pi,
                        np.where(step <= -np.pi, step + 2 * np.pi, step))
        theta = theta + step
        raw = new
        if keep:
            path.append((theta, a_, b_))
```

Following the angle continuously: the change of the raw angle over one step is brought into the interval from $-\pi$ to π (`np.where(condition, x, y)` takes `x` where the condition holds and `y` elsewhere; a jump of almost 2π of the raw angle is the passage through $\pm\pi$, not a real turn) and added to `theta`. The step is so small that the true change of the angle over one step is much less than π. With `keep=True` the angle and the point after every step are stored.

```
    if keep:
        return j * theta, [np.array(q) for q in zip(*path)]
    return j * theta
```

The result is $\Phi = j\theta(0)$. With `keep=True` also the paths: `zip(*path)` regroups the list of triples (θ, a, b) into three sequences, each turned into an array with one row per node ($G + 1$ rows) and one column per energy.

```
def target(parity, label):
    offset = np.where(np.asarray(parity) == "even", 0.0, 0.5 * np.pi)
    return offset + np.asarray(label, dtype=float) * np.pi
```

The value of Φ at a level: $l\pi$ for even parity and $\pi/2 + l\pi$ for odd parity, for arrays of parities and labels.

```
def levels(k, j, parity, label, iterations=72, **options):
    k, j, parity, label = np.broadcast_arrays(
        np.asarray(k, dtype=float), np.asarray(j, dtype=float),
        np.asarray(parity), np.asarray(label))
    goal = target(parity, label)
    lo, hi = np.full(k.shape, -20.0), np.full(k.shape, 20.0)
    for _ in range(iterations):
        mid = 0.5 * (lo + hi)
        g = shoot(mid, k, j, **options) - goal
        lo = np.where(g <= 0.0, mid, lo)  # g = 0: an exact level
        hi = np.where(g >= 0.0, mid, hi)
    return 0.5 * (lo + hi)
```

The levels with given momenta, block types, parities and labels, all at once, by 72 bisections of $[-20, 20]$ (Section 14.14). `**options` collects any further named arguments (for example `M=2.0` or `G=1800`) and hands them on to `shoot`. In each pass Φ is computed at the midpoints; where it is at or below the target, the level lies above the midpoint and `lo` moves there; where it is at or above, `hi` moves there. If Φ equals the target exactly, both ends move to the midpoint and the level is found exactly; this is what happens for the zero mode (In [8]).

```
say("defined: shoot(eps, k, j, M, L, G, ...) and levels(k, j, parity, label, ...)")
```

Out [6]: the PASS line of the angle equation and this line.

In [7], the shooting function and figure 2.

```
trial = np.linspace(-4.0, 4.0, 801)  # 801 trial energies
phi_curve = shoot(trial, 0.0, 1.0)  # Phi for k = 0, j = +1 (both parities)
grows = bool(np.all(np.diff(phi_curve) > 0.0))
grows_k = all(bool(np.all(np.diff(shoot(np.linspace(-4.0, 3.96, 200), 0.75, jj)) > 0))
              for jj in (1.0, -1.0))
check(grows and grows_k,
      "Phi(eps) grows strictly with eps (k = 0 and k = 0.75, both block types)")
```

$\Phi(\varepsilon)$ at 801 energies from -4 to 4 in one call. `np.diff` gives the differences of neighbouring values; `np.all(... > 0.0)` is true when every difference is positive, that is when Φ grows strictly; `bool` turns numpy's answer into a plain true or false. The same test for $k = 0.75$, 200 energies and both block types. One PASS line: the monotonicity proved in Section 14.14, seen numerically.

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.plot(trial, phi_curve / np.pi, color="C0", label="$\\Phi(\\varepsilon)/\\pi$")
for label in range(-3, 4):  # the targets in units of pi
    ax.axhline(label, color="C1", linewidth=0.7)
    ax.axhline(label + 0.5, color="C2", linewidth=0.7, linestyle="--")
```

The curve Φ/π and the targets: solid lines at the whole numbers (even), dashed lines at the half numbers (odd).

```
even_levels = [exact[(1.0, 3.0, "even", l)] for l in range(-2, 3)]
odd_levels = [exact[(1.0, 3.0, "odd", l)] for l in range(-2, 2)]
ax.plot(even_levels, range(-2, 3), "o", color="C1",
        label="even levels: $\\Phi = l\\pi$")
ax.plot(odd_levels, [l + 0.5 for l in range(-2, 2)], "s", color="C2",
        label="odd levels: $\\Phi = \\pi/2 + l\\pi$")
```

The exact levels of the labels -2 to 2 (even) and -2 to 1 (odd), drawn as circles and squares at the heights of their targets.

```
ax.set_xlabel("trial energy $\\varepsilon$ (units of $m$)")
ax.set_ylabel("$\\Phi(\\varepsilon) / \\pi$")
ax.set_title("the shooting function for $k = 0$, $j = +1$, $m = 1$, $L = 3$")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "shooting_function",
            "The shooting function $\\Phi(\\varepsilon) = j\\,\\theta(0)$ (the "
            ...)
```

What figure 14b.2 shows. The curve rises from about -3.6 to $+3.6$ (in units of π) as the trial energy goes from -4 to 4, never falling. Every circle and square sits exactly on the curve where it crosses its target line: the crossings are the levels, one per label. Between -1 and 1 the curve is almost flat and crosses only the line 0, at $\varepsilon = 0$: the gap $(-M, M)$ contains only the zero mode. Where the levels are dense, the curve is steep.

In [8], all 54 levels.

```
numeric = {}  # (m, L, parity, label) -> numerical level at the canonical step
for m_, L_ in CASES:
    pars = np.array([p_ for l_ in LABELS for p_ in ("even", "odd")])
    labs = np.array([l_ for l_ in LABELS for p_ in ("even", "odd")])
    found = levels(0.0, 1.0, pars, labs, M=m_, L=L_, G=round(900 * L_ / 3))
    for p_, l_, x in zip(pars, labs, found):
        numeric[(m_, L_, str(p_), int(l_))] = float(x)
```

For each case, the 18 parities and labels as two arrays (the double list comprehension runs over the labels and, for each, over the two parities), and the 18 levels found in one call with $k = 0$, $j = +1$, the mass m, the cutoff L and $G = 900L/3$ steps (900 for $L = 3$, 600 for $L = 2$: always the step $h = 1/300$ of the Rust solver). `round` gives the nearest whole number. The levels go into the dictionary `numeric`.

```
say("m = 1, L = 3:  label  even (numerical)   even (exact)    "
    "odd (numerical)    odd (exact)")
for l_ in LABELS:
    row = [numeric[(1.0, 3.0, "even", l_)], exact[(1.0, 3.0, "even", l_)],
           numeric[(1.0, 3.0, "odd", l_)], exact[(1.0, 3.0, "odd", l_)]]
    say(f"               {l_:+d}   " + "  ".join(f"{x:15.10f}" for x in row))
```

A table of the case $m = 1$, $L = 3$: for each label the numerical and exact levels of both parities, each printed in 15 places with 10 decimals (`{x:15.10f}`).

```
rust_diff = max(abs(numeric[(float(r["m"]), float(r["L"]), r["parity"],
                             int(r["label"]))] - float(r["eps_numeric"]))
                for r in record)
report("largest difference from the Rust solver's levels", f"{rust_diff:.1e}", "m")
check(rust_diff < 1e-11,
      "the 54 numerical levels equal those of the Rust solver",
      record="Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, eps_numeric")
```

The largest difference between the notebook's numerical levels and the Rust solver's (column eps_numeric), printed as a RESULT line with one decimal (`.1e`). The two programs integrate the same RK4 discretisation and look for the roots of the same shooting function Φ; they differ only in the root finder (plain bisection to the last digit here, the safeguarded Newton-bisection with the tolerance 10^{-13} in the Rust solver, Section 14.14) and in rounding. The check requires agreement to 10^{-11}; Out [8] shows the difference $7.1 \times 10^{-15}\,m$.

```
errors = {key: abs(numeric[key] - exact[key]) for key in numeric}
low = max(err for key, err in errors.items() if abs(exact[key]) < 4.0)
high = max(err for key, err in errors.items() if abs(exact[key]) >= 4.0)
report("largest error for |eps| < 4 m", f"{low:.2e}", "m")
report("largest error for |eps| >= 4 m", f"{high:.2e}", "m")
```

The errors against the exact levels, and the largest of them below and above $|\varepsilon| = 4m$, printed with two decimals (`.2e`). The Rust check free_k0_analytic_spectra measured the same two numbers; the next lines read them from its record.

```
import re  # regular expressions: patterns that find numbers in a text


def record_detail(name, report_file):
    """The detail text of the check called name in a Revision report."""
    data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
    return [c["detail"] for c in data["checks"] if c["name"] == name][0]
```

`re` is Python's module for **regular expressions**, patterns that describe pieces of text. `record_detail` reads a report as `record_check` does (In [2]), keeps the checks with the given name and returns the detail text of the first, the sentence in which the record states its numbers. The argument is called `report_file` and not `report`, so that inside the function it does not hide the helper `report` of the set-up cell.

```
detail = record_detail("free_k0_analytic_spectra", RUST_REPORT)
rec_low, tol_low, rec_high, tol_high = re.findall(r"\d+(?:\.\d+)?e-\d+", detail)
check(f"{low:.2e}" == f"{float(rec_low):.2e}"
      and f"{high:.2e}" == f"{float(rec_high):.2e}"
      and low < float(tol_low) and high < float(tol_high)
      and record_check("free_k0_analytic_spectra", (RUST_REPORT,)),
      f"largest errors {low:.2e} for |eps| < 4 m and {high:.2e} above, as in the "
      f"record, below its tolerances {tol_low} and {tol_high}",
      record=f"{RUST_REPORT}, check free_k0_analytic_spectra")
```

The detail of the Rust check ends with the words

```
max |difference| 7.23e-10 for |eps| < 4 m (tolerance 5e-9, RK4 error ~ (h eps)^4),
5.05e-8 for 4 m <= |eps| < 7 m (tolerance 3e-7)
```

(one line in the record). The pattern `\d+(?:\.\d+)?e-\d+` describes a number with a negative power of ten: one or more digits (`\d+`), then, if present, a point and more digits (the group `(?:\.\d+)` followed by `?`, which makes it optional), then `e-` and digits; the `r` before the string keeps the backslashes as they are. `re.findall` returns every piece of the text that fits the pattern, in order: the texts `7.23e-10`, `5e-9`, `5.05e-8` and `3e-7`, which are unpacked into the four names (if the record ever stated a different number of such numbers, the unpacking would fail and the notebook would stop). The check requires: the notebook's two largest errors, written with two decimals, equal the record's, written the same way (`float` turns a text into a number, so the record's `5.05e-8` is written `5.05e-08`); both lie below the record's tolerances; and the record's verdict is PASS. Because the numbers are read from the record, the check fails if the record is ever recomputed with different results.

```
check(all(numeric[(m_, L_, "even", 0)] == 0.0 for m_, L_ in CASES),
      "the brane zero mode comes out as exactly eps = 0")
```

The last check: the zero mode is exactly 0 in all three cases. Why exactly: the first midpoint of the bisection is $\varepsilon = 0$; there, with $k = 0$ and the start $b = 0$, every RK4 slope of b is exactly zero, so b stays exactly 0, the angle stays exactly 0, $\Phi-$ target is exactly 0, and both ends of the interval jump to 0. Out [8] shows the table, three RESULT lines and three PASS lines: the largest difference from the Rust levels 7.1×10^{-15}, and the largest errors 7.23×10^{-10} below $4m$ and 5.05×10^{-8} above, the numbers of the record. In the table the numerical and exact levels agree in all 10 printed decimals for the even labels -2 to 2 and the odd labels -1 and 0; the odd labels -2, 1 and 2 differ by one unit in the 10th decimal (for example 2.0106285601 against 2.0106285600 for the odd label 1), and the higher labels differ in the 8th to 10th decimal (for example 5.3306254627 against 5.3306254587 for the even label 5): the error grows with the level, because the orbital oscillates faster.

In [9], figure 3.

```
fig, ax = plt.subplots(figsize=(8.0, 5.0))
for position, (m_, L_) in enumerate(CASES):
    x0 = 3.0 * position  # the horizontal place of this ladder
    for l_ in LABELS:
        for shift, parity, color, marker in ((0.0, "even", "C0", "o"),
                                             (1.2, "odd", "C1", "s")):
            level = exact[(m_, L_, parity, l_)]
            ax.plot([x0 + shift - 0.4, x0 + shift + 0.4], [level, level],
                    color=color, linewidth=1.5)
            ax.plot([x0 + shift], [numeric[(m_, L_, parity, l_)]], marker,
                    color=color, markersize=4)
    ax.fill_between([x0 - 0.5, x0 + 1.7], -m_, m_, color="gray", alpha=0.12)
```

For each case a **level ladder** at the horizontal place $x_0 = 0, 3, 6$: every exact level is a short horizontal line (from $x_0 + \text{shift} - 0.4$ to $x_0 + \text{shift} + 0.4$), even parity at the left and odd parity 1.2 further right, and the numerical level is a marker in its middle. `fill_between(x, y1, y2)` shades the band from $-m$ to m across the ladder.

```
ax.set_xlim(-0.6, 8.4)
ax.set_ylim(-7.0, 9.5)
ax.set_xticks([0.6, 3.6, 6.6])
ax.set_xticklabels(["$m = 1$, $L = 3$", "$m = 1$, $L = 2$", "$m = 2$, $L = 3$"])
ax.plot([], [], "o-", color="C0", label="even parity (line exact, dot numerical)")
ax.plot([], [], "s-", color="C1", label="odd parity (line exact, square numerical)")
ax.set_ylabel("level $\\varepsilon$ (units of $H = 1$)")
ax.set_title("the free $k = 0$ spectra, labels $-3$ to $5$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "spectrum_ladder",
            "The free $k = 0$ spectra for the three cases $(m, L) = (1, 3)$, "
            ...)
```

The ranges of the axes, the names of the three ladders under them, and two empty plots (`ax.plot( [], [], ...)` draws nothing) whose only purpose is to put the two styles into the legend. **What figure 14b.3 shows.** Three ladders. In each, the shaded band from $-m$ to m contains only the even zero mode at 0. For $m = 2$ the band is twice as wide and the lowest levels move out with it; for $L = 2$ the levels are farther apart than for $L = 3$, because the wave numbers $n\pi/L$ are larger. The even and odd levels alternate. On this scale every marker sits exactly on its line.

In [10], fourth-order convergence.

```
refined = {}
for m_, L_ in CASES:
    pars = np.array([p_ for l_ in LABELS for p_ in ("even", "odd")])
    labs = np.array([l_ for l_ in LABELS for p_ in ("even", "odd")])
    found = levels(0.0, 1.0, pars, labs, M=m_, L=L_, G=round(1800 * L_ / 3))
    for p_, l_, x in zip(pars, labs, found):
        refined[(m_, L_, str(p_), int(l_))] = float(x)
refined_errors = {key: abs(refined[key] - exact[key]) for key in refined}
```

The same 54 levels with twice as many steps ($G = 1800$ for $L = 3$, 1200 for $L = 2$, the refined run of the Revision solver) and their errors.

```
ratios = sorted(errors[key] / refined_errors[key] for key in errors
                if errors[key] > 1e-11 and refined_errors[key] > 0.0)
median = ratios[len(ratios) // 2]
```

For every level whose canonical error is above 10^{-11} (smaller errors are dominated by rounding), the ratio of the canonical to the refined error, sorted. The **median** is the middle value of the sorted list: `len(ratios) // 2` is the position of the middle (`//` divides and drops the remainder; with 39 ratios it is the 20th value, position 19 counting from 0).

```
report("number of levels with a canonical error above 1e-11", len(ratios))
report("median error ratio canonical / refined", f"{median:.2f}")
report("largest error, canonical step", f"{max(errors.values()):.3e}", "m")
report("largest error, refined step", f"{max(refined_errors.values()):.3e}", "m")
```

Four RESULT lines: the number of ratios, the median ratio with two decimals, and the largest canonical and refined errors with three decimals and an exponent (`f"{x:.3e}"`).

```
DETERMINISM = "Revision/kohn_sham/reports/ks-rust-determinism.json"
detail = record_detail("refined_free_spectra_convergence_order", DETERMINISM)
rec_canonical, rec_refined = re.search(r"canonical (\S+), refined (\S+);",
                                       detail).groups()
rec_median, rec_count = re.search(r"ratio (\S+) over (\d+) levels",
                                  detail).groups()
```

The detail text of the record's check, read with `record_detail` of In [8], is

```
analytic k = 0 spectra: max error canonical 5.050e-08, refined 3.157e-09; median
error ratio 16.00 over 39 levels (RK4 order 4 predicts 16)
```

(one line in the record). `re.search` finds the first place where a pattern fits, and `.groups()` returns the pieces of text matched by the bracketed parts of the pattern: `\S+` is one or more characters that are not spaces, `\d+` one or more digits. So the first pattern picks out the texts `5.050e-08` and `3.157e-09` (the words "canonical" and "refined", the comma and the semicolon fix where they stand), and the second the texts `16.00` and `39`. If the record's sentence had another form, `re.searc h` would find nothing, return `None`, and `.groups()` would stop the notebook with an error.

```
check(len(ratios) == int(rec_count) and f"{median:.2f}" == rec_median
      and f"{max(errors.values()):.3e}" == rec_canonical
      and f"{max(refined_errors.values()):.3e}" == rec_refined
      and record_check("refined_free_spectra_convergence_order", (DETERMINISM,)),
      f"{len(ratios)} levels, median error ratio {median:.2f}, largest errors "
      f"{max(errors.values()):.3e} and {max(refined_errors.values()):.3e}",
      record=f"{DETERMINISM}, check refined_free_spectra_convergence_order")
```

The check that the notebook's four numbers equal the record's: 39 levels (`int` turns the text into a whole number), the median ratio 16.00 (RK4's $2^4 = 16$), and the largest errors 5.050×10^{-8} and 3.157×10^{-9}. The comparison is made on the printed texts, written in the same form as the record writes them, and the numbers come from the record itself, so a recomputed record with other numbers would make the check fail. The name of the check is built from the notebook's own numbers.

In [11], figure 4.

```
chosen = [("even", 1), ("even", 3), ("odd", 0), ("odd", 3)]
steps = [150, 300, 600, 900, 1800]
curves = {key: [] for key in chosen}
for G_ in steps:
    found = levels(0.0, 1.0, np.array([p_ for p_, l_ in chosen]),
                   np.array([l_ for p_, l_ in chosen]), M=1.0, L=3.0, G=G_)
    for key, x in zip(chosen, found):
        curves[key].append(abs(float(x) - exact[(1.0, 3.0) + key]))
```

Four levels and five step numbers; for each step number the four levels in one call, and their errors appended to four lists. `(1.0, 3.0) + key` joins two tuples into the key $(1, 3, \text{parity}, \text{label})$.

```
h_values = np.array([3.0 / G_ for G_ in steps])
fig, ax = plt.subplots(figsize=(7.0, 4.6))
for (parity, label), errs in curves.items():
    ax.loglog(h_values, errs, "o-", label=f"{parity}, label {label}")
ax.loglog(h_values, 3e-2 * h_values ** 4, ":", color="black",
          label="slope 4: error $\\propto h^4$")
```

The steps $h = 3/G$; `loglog` draws with logarithmic axes in both directions, on which a power law h^4 is a straight line of slope 4; the dotted guide line is $0.03\,h^4$.

```
ax.set_xticks(h_values)  # one tick at each step used
ax.set_xticklabels([f"{x:.4f}" for x in h_values], fontsize=8)
ax.xaxis.set_minor_locator(matplotlib.ticker.NullLocator())  # no extra ticks
ax.set_xlabel("RK4 step $h = L/G$ (units of $1/H$)")
ax.set_ylabel("$|\\varepsilon_{\\rm RK4} - \\varepsilon_{\\rm exact}|$ (units of $m$)")
ax.set_title("fourth-order convergence of the shooting method ($m = 1$, $L = 3$)")
ax.legend(fontsize=8)
```

One tick mark at each step used, labelled with 4 decimals; `NullLocator` removes the small extra ticks that a logarithmic axis draws by default.

```
fourth = all(12.0 < errs[i] / errs[i + 1] < 20.0 for errs in curves.values()
             for i in range(2))  # halving h from 0.02 to 0.005: ratios near 16
check(fourth, "halving the step divides each error by about 16 (between 12 and 20)")
save_figure(fig, "rk4_convergence",
            "The error of four numerical levels ($m = 1$, $L = 3$; even labels 1 "
            ...)
```

For the first three step numbers (150, 300, 600: $h = 0.02$, 0.01, 0.005) each halving must divide each of the four errors by a factor between 12 and 20. **What figure 14b.4 shows.** Four straight lines parallel to the dotted line of slope 4, from errors near 10^{-6} at $h = 0.02$ down to 10^{-13} or 10^{-14} at $h = 0.0017$: the method is of fourth order over the whole range. The lines of the higher levels (labels 3) lie about three powers of ten above those of the lowest levels.

In [12], the orbitals.

```
def orbital(eps, k=0.0, j=1.0, M=1.0, L=3.0, G=900, H=1.0, a4=0.0, tip=0.0):
    _, (theta_n, a_n, b_n) = shoot(eps, k, j, M=M, L=L, G=G, H=H, a4=a4, tip=tip,
                                   keep=True)
    nf, h = 2 * G + 1, L / G
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    K_fine = k * np.exp(-a4) * np.exp(-H * y_fine)
```

The orbital of one level. `shoot(..., keep=True)` returns the angle and the paths; `_` receives Φ, which is not needed, and the paths are unpacked into three arrays (one value per node, since a single energy is given). Then the fine grid and $K = \kappa k$ on it.

```
    a_f, b_f = np.zeros(nf), np.zeros(nf)
    a_f[0::2], b_f[0::2] = a_n, b_n  # the node values
    da = M * a_f - (K_fine + j * eps) * b_f  # the derivatives at the nodes
    db = (j * eps - K_fine) * a_f - M * b_f
```

Two arrays for the fine grid; `a_f[0::2]` is every second entry starting at 0, the nodes, which receive the node values. `da` and `db` are the derivatives a', b' from the real form; only their values at the nodes are used below.

```
    for f in range(1, nf, 2):  # cubic Hermite value at each midpoint
        a_f[f] = 0.5 * (a_f[f - 1] + a_f[f + 1]) + h / 8.0 * (da[f - 1] - da[f + 1])
        b_f[f] = 0.5 * (b_f[f - 1] + b_f[f + 1]) + h / 8.0 * (db[f - 1] - db[f + 1])
```

The midpoints (odd positions 1, 3, 5, ...) get their values by **cubic Hermite interpolation**: the cubic polynomial that has the values f_0, f_1 and the derivatives f_0', f_1' at the two neighbouring nodes, a step h apart, has at the midpoint the value $\frac{1}{2}(f_0 + f_1) + \frac{h}{8}(f_0' - f_1')$ (Exercise 14.4). Its error is of order h^4, like that of RK4.

```
    weights = np.full(nf, 2.0)  # Simpson: 1, 4, 2, 4, ..., 4, 1 times (h/2)/3
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    weights *= 0.5 * h / 3.0
    norm = math.sqrt(float(np.sum(weights * (a_f ** 2 + b_f ** 2))))
    return y_fine, a_f / norm, b_f / norm, weights
```

Simpson's rule (Chapter 2) on the fine grid, whose spacing is $h/2$: the weights $1, 4, 2, 4, \ldots, 4, 1$ times $(h/2)/3$ (`weights[1::2]` are the odd positions, `weights[-1]` the last entry). The integral $\int (a^2 + b^2) dy$ is the weighted sum, and dividing a, b by its square root normalises the orbital. The function returns the grid, the normalised components and the weights.

```
y_f, a_zero, b_zero, simpson = orbital(numeric[(1.0, 3.0, "even", 0)])
a_exact = math.sqrt(2.0 / (1.0 - math.exp(-6.0))) * np.exp(y_f)
deviation = float(np.max(np.abs(a_zero - a_exact)))
check(numeric[(1.0, 3.0, "even", 0)] == 0.0 and np.all(b_zero == 0.0)
      and deviation < 1e-9 and record_check("free_zero_mode_exact", (RUST_REPORT,)),
      "zero mode: eps = 0 and b = 0 exactly, a = sqrt(2M/(1 - e^(-2ML))) e^(My)",
      record=f"{RUST_REPORT}, check free_zero_mode_exact")
```

The zero mode of $m = 1$, $L = 3$ against the exact $a = \sqrt{2/(1 - e^{-6})}\, e^{y}$ (Section 14.13 with $M = 1$, $L = 3$): its level is exactly 0, its b is exactly 0 at every point, and a agrees to 10^{-9}, as in the Rust check free_zero_mode_exact.

```
def exact_orbital(parity, label, m_=1.0, L_=3.0):
    level = exact[(m_, L_, parity, label)]
    p_ = math.sqrt(level ** 2 - m_ ** 2)
    if parity == "even":
        b_e = np.sin(p_ * y_f)
        a_e = (p_ * np.cos(p_ * y_f) + m_ * b_e) / level
    else:
        b_e = np.sin(p_ * (y_f + L_))
        a_e = (p_ * np.cos(p_ * (y_f + L_)) + m_ * b_e) / level
    sign = 1.0 if a_e[0] > 0 else -1.0  # the sign with a(-L) > 0
    norm = math.sqrt(float(np.sum(simpson * (a_e ** 2 + b_e ** 2))))
    return sign * a_e / norm, sign * b_e / norm
```

The exact orbitals of Section 14.13 for $j = +1$: $p = \sqrt{\varepsilon^2 - m^2}$, b the sine, $a = (b' + mb)/\varepsilon$ written out ($b' = p\cos$). An orbital is fixed only up to a constant factor; the numerical one starts with $a(-L) = 1 > 0$, so the exact one is given the sign that makes $a(-L) > 0$ (`a_e[0]` is its value at $y = -L$) and is normalised with the same Simpson weights.

```
shown = {}
worst_orbital = 0.0
for parity, label in (("even", 1), ("odd", 0)):
    _, a_num, b_num, _ = orbital(numeric[(1.0, 3.0, parity, label)])
    a_ex, b_ex = exact_orbital(parity, label)
    worst_orbital = max(worst_orbital, float(np.max(np.abs(a_num - a_ex))),
                        float(np.max(np.abs(b_num - b_ex))))
    shown[(parity, label)] = (a_num, b_num, a_ex, b_ex)
check(worst_orbital < 1e-8,
      "the even label 1 and odd label 0 orbitals equal the exact ones to 1e-8")
```

The numerical and exact orbitals of the even level 1 and the odd level 0, their largest difference over all points (below 10^{-8}), and a dictionary `shown` that keeps them for the figure. Out [12] shows two PASS lines.

In [13], figure 5.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 3.8), sharey=True)
panels = [("zero mode, $\\varepsilon = 0$", a_zero, b_zero, a_exact,
           np.zeros_like(a_exact)),
          (f"even label 1, $\\varepsilon = {lowest_even:.4f}$",) + shown[("even", 1)],
          (f"odd label 0, $\\varepsilon = {lowest_odd:.4f}$",) + shown[("odd", 0)]]
```

Three panels side by side that share the vertical axis (`sharey=True`), and for each a tuple of the title, the numerical a, b and the exact a, b; `(title,) + shown[...]` puts the title in front of the stored four arrays, and `np.zeros_like` is an array of zeros of the same shape (the exact b of the zero mode).

```
for ax, (title, a_num, b_num, a_ex, b_ex) in zip(axes, panels):
    ax.plot(y_f, a_num, color="C0", label="$a$ (numerical)")
    ax.plot(y_f, b_num, "--", color="C1", label="$b$ (numerical)")
    ax.plot(y_f[::60], a_ex[::60], ".", color="black", label="exact")
    ax.plot(y_f[::60], b_ex[::60], ".", color="black")
    ax.set_title(title, fontsize=9)
    ax.set_xlabel("$y$ (units of $1/H$)")
axes[0].set_ylabel("orbital component (normalised)")
axes[0].legend(fontsize=8, loc="upper left")
save_figure(fig, "orbitals",
            "The normalised orbitals $\\chi = (a, ib)$ of three free $k = 0$ "
            ...)
```

In each panel the numerical a (solid) and b (dashed) and every 60th point of the exact ones as black dots (`[::60]` takes every 60th entry). **What figure 14b.5 shows.** Left: the zero mode, a rising like e^y from 0.07 at the tip to 1.42 at the brane, $b = 0$: the orbital lives at the brane. Middle: the

even level 1, b a half sine wave that vanishes at both ends, a falling from $+0.42$ to -0.42. Right: the odd level 0, b vanishing at the tip, a vanishing at the brane. All black dots lie on the curves.

In [14], figure 6.

```
picks = [("even", l_) for l_ in range(4)] + [("odd", l_) for l_ in range(3)]
energies = np.array([numeric[(1.0, 3.0, p_, l_)] for p_, l_ in picks])
_, (theta_path, _, _) = shoot(energies, 0.0, 1.0, keep=True)
y_nodes = np.linspace(-3.0, 0.0, 901)
end_values = theta_path[-1] / np.pi
goals = np.array([l_ + (0.0 if p_ == "even" else 0.5) for p_, l_ in picks])
check(np.max(np.abs(end_values - goals)) < 1e-9,
      "the Pruefer angle ends on its target l pi or (l + 1/2) pi for every level")
```

Seven levels (even labels 0 to 3, odd labels 0 to 2), one call of `shoot` that keeps the angle paths (an array with 901 rows, the nodes, and seven columns), the 901 node positions, the end angles in units of π (`theta_path[-1]` is the last row) and the targets l or $l + \frac{1}{2}$. The check: every path ends on its target to 10^{-9}.

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
for column, (p_, l_) in enumerate(picks):
    style = "-" if p_ == "even" else "--"
    ax.plot(y_nodes, theta_path[:, column] / np.pi, style,
            label=f"{p_}, label {l_}")
ax.set_xlabel("$y$ (units of $1/H$; tip at $-3$, brane at $0$)")
ax.set_ylabel("Pruefer angle $\\theta(y)/\\pi$")
ax.set_title("the Pruefer angle counts the half-turns ($m = 1$, $L = 3$, $k = 0$)")
ax.legend(fontsize=8, ncol=2, loc="upper left")
save_figure(fig, "pruefer_angle",
            "The Pruefer angle $\\theta(y) = \\mathrm{atan2}(b, a)$ in units of "
            ...)
```

`theta_path[:, column]` is one column, the path of one level; even levels are drawn solid, odd ones dashed. **What figure 14b.6 shows.** All curves start at 0 at the tip ($b(-3) = 0$) and climb to their targets at the brane: the even labels 0, 1, 2, 3 end at 0, 1, 2, 3 and the odd labels 0, 1, 2 at 0.5, 1.5, 2.5. The zero mode stays at 0 all the way. A higher level turns faster: the label counts the half-turns of the point (a, b) between the tip and the brane.

In [15], the last check.

```
figure_names = ["14b_1_odd_condition.png", "14b_2_shooting_function.png",
                "14b_3_spectrum_ladder.png", "14b_4_rk4_convergence.png",
                "14b_5_orbitals.png", "14b_6_pruefer_angle.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all six figure files of this notebook exist")
all_checks_passed()
```

The six figure files must exist where the notebook wrote them; the last line prints ALL 20 CHECKS PASSED (notebook 14b). The 20 checks are: 1 in In [2], 6 in In [3], 2 in In [4], 1 in In [6], 1 in In [7], 3 in In [8], 1 in In [10], 1 in In [11], 2 in In [12], 1 in In [14] and 1 in In [15].

14.19 The brane band and its exact slope

What happens when the 3-momentum is switched on. At $k = 0$ the free spectrum has eight zero modes, four in the blocks of type $j = +1$ and four in those of type $j = -1$ (Section 14.13). For $k \neq 0$ the momentum term $j\kappa k\,\sigma_3$ of h_j moves them. In the blocks $j = +1$ the even level of label 0 rises with k: this is the **brane band**, because its orbital stays at the brane. In the blocks $j = -1$ the even level of label 0 falls by exactly the same amount, because $h_{-1} = -h_{+1}$ for $v = 0$ (Section 14.5): its levels are the negatives of those of $j = +1$, with the label l of even parity going to $-l$. How fast does the band rise? Its slope at $k = 0$ can be computed exactly.

The Hellmann-Feynman rule. Let $h(k)$ be a self-adjoint operator that depends on a number k, with boundary conditions that do not depend on k, and let $h(k)\chi = \varepsilon(k)\chi$ with $\int \chi^\dagger \chi \, dy = 1$. Line by line:

$$\varepsilon = \int \chi^\dagger h \chi \, dy.$$

Rule: multiply $h\chi = \varepsilon\chi$ from the left by $\chi^\dagger$, integrate, and use the normalisation.

$$\frac{d\varepsilon}{dk} = \int \frac{\partial \chi^\dagger}{\partial k} h\chi \, dy + \int \chi^\dagger h \frac{\partial \chi}{\partial k} \, dy + \int \chi^\dagger \frac{\partial h}{\partial k} \chi \, dy.$$

Rule: the product rule for the three factors.

$$\int \frac{\partial \chi^\dagger}{\partial k} h\chi \, dy + \int \chi^\dagger h \frac{\partial \chi}{\partial k} \, dy = \varepsilon \int \Big(\frac{\partial \chi^\dagger}{\partial k} \chi + \chi^\dagger \frac{\partial \chi}{\partial k} \Big) dy = \varepsilon \frac{d}{dk} \int \chi^\dagger \chi \, dy = 0.$$

Rule: in the first integral $h\chi = \varepsilon\chi$; in the second h is self-adjoint, so $\int \chi^\dagger h\, \partial_k \chi \, dy = \int (h\chi)^\dagger \partial_k \chi \, dy = \varepsilon \int \chi^\dagger \partial_k \chi \, dy$ (the derivative $\partial_k \chi$ obeys the same boundary conditions, because they do not depend on k); then the product rule backwards, and the normalisation integral is the constant 1. Hence

$$\frac{d\varepsilon}{dk} = \int \chi^\dagger \frac{\partial h}{\partial k} \chi \, dy.$$

In words: the rate of change of a level is the expectation value of the rate of change of the operator.

The slope of the band. Line by line:

$$\frac{\partial h_j}{\partial k} = j\kappa(y)\, \sigma_3.$$

Rule: in $h_j = j[-i\sigma_1 \, d/dy + M\sigma_2 + \kappa k \sigma_3] + v$ only the last term contains k.

$$\frac{d\varepsilon}{dk}\Big|_{k=0} = j \int_{-L}^{0} \kappa(y)\, \chi^\dagger \sigma_3 \chi \, dy = j \int_{-L}^{0} e^{-Hy - a_{4,0}}\, a(y)^2 \, dy.$$

Rule: the Hellmann-Feynman rule at $k = 0$, where the orbital is the zero mode $\chi = (a, 0)$ and $\chi^\dagger \sigma_3 \chi = a^2 - b^2 = a^2$ (Section 14.12).

$$c = \frac{\int_{-L}^{0} e^{-Hy-a_{4,0}} e^{2My}\, dy}{\int_{-L}^{0} e^{2My}\, dy}.$$

Rule: $a^2 = A^2 e^{2My}$ with $A^2 = 1/\int_{-L}^{0} e^{2My} dy$ (the normalisation); we call c the slope of the band of type $j = +1$, so that $d\varepsilon/dk = jc$.

$$\int_{-L}^{0} e^{-Hy} e^{2My}\, dy = \frac{1 - e^{-(2M-H)L}}{2M - H}, \qquad \int_{-L}^{0} e^{2My}\, dy = \frac{1 - e^{-2ML}}{2M}.$$

Rule: $e^{-Hy} e^{2My} = e^{(2M-H)y}$; the integral of e^{sy} is e^{sy}/s, evaluated at 0 and $-L$ (here $2M \neq H$).

$$c = e^{-a_{4,0}}\, \frac{2M}{2M - H}\, \frac{1 - e^{-(2M-H)L}}{1 - e^{-2ML}}.$$

Rule: divide the two integrals and take the constant factor $e^{-a_{4,0}}$ out of the first. For $M = H = 1$:

$$c = e^{-a_{4,0}}\, \frac{2(1 - e^{-L})}{1 - e^{-2L}} = \frac{2e^{-a_{4,0}}}{1 + e^{-L}}.$$

Rule: $1 - e^{-2L} = (1 - e^{-L})(1 + e^{-L})$ (the difference of two squares). For $L = 3$ and $a_{4,0} = 0$: $c = 2/(1+e^{-3}) = 1.9051482536$. Status: PROVED; check brane_band_slope of `Revision/kohn_sham/reports/ks-theory-python.json` and of the Wolfram report, and the value braneBandSlope_M1_H1_L3_a0 = 1.9051482536448664 under checksNumeric in `Revision/kohn_sham/ks-theory.json`.

What the formula says. Three things. (1) The band starts linearly, $\varepsilon \approx ck$, like the energy of a massless particle, although the mass m is not zero: the mass term only keeps the orbital at the brane. The slope is the average of the momentum weight κ over the zero-mode orbital (the integral above), and it is larger than the value $e^{-a_{4,0}}$ of κ at the brane because the orbital reaches somewhat toward the tip, where κ is larger. (2) For a long hidden interval, $L \to \infty$, the slope tends to $2M/(2M - H)$, which is 2 for $M = H = 1$; for $L = 2$, 3, 4 it is 1.761594, 1.905148, 1.964028 (Notebook 14c, Out [5]). (3) Along the deflating history the slope shrinks like $e^{-a_{4,0}}$: at the five slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$ the exact values are 1.905148253645, 1.155530827134, 0.700864874900, 0.425096034942, 0.257833778515 (Notebook 14c, Out [5]).

The band is an odd function of k. The relation $\sigma_3 h_j(k) \sigma_3 = h_{-j}(-k)$ of Section 14.5 says that the level of (j, k) equals the level of $(-j, -k)$ with the same label; and $h_{-1} = -h_{+1}$ says that the level of $(-1, k)$ with even label 0 is minus the level of $(+1, k)$ with even label 0. Together: $\varepsilon_{\text{band}}(-k) = \varepsilon_{-1}(k) = -\varepsilon_{\text{band}}(k)$. So $\varepsilon_{\text{band}}(k)/k = c + dk^2 + \ldots$ contains only even powers of k. The Revision solver used this to measure the slope numerically: with $R(k) = \varepsilon(k)/k$ at $k_1 = 10^{-4}$ and $k_2 = 2 \times 10^{-4}$,

$$\frac{4R(k_1) - R(k_2)}{3} = \frac{4(c + dk_1^2) - (c + 4dk_1^2)}{3} = c.$$

Rule: $k_2 = 2k_1$, so $k_2^2 = 4k_1^2$, and the terms with d cancel (**Richardson extrapolation**, Chapter 2; what remains is of order k^4). The Rust solver found the exact $c\, e^{-a_{4,0}}$ at all five slices to $1.87 \times$

10^{-12} (relative; check free_brane_band_slope of `Revision/kohn_sham/reports/ks-rust-solver.json`; the values are in `Revision/kohn_sham/results/spectrum/brane-band-slope.csv`). Status: COMPUTED; Notebook 14c reproduces the five values.

The band at larger momenta. The record `Revision/kohn_sham/results/spectrum/brane-band.csv` follows the band and three other levels from $k = 0$ to $k = 4$ at the slice 0. At the first lattice shell, $k = \Delta k = 0.25$, the band is at $0.4307336786\,m$ (Notebook 14c, Out [3]), a little below the straight line $ck = 0.476$: the band bends downward. At $k = 4$ it is at $4.992\,m$. The lowest level that is not on the brane band at $k = 0$ is the odd level of label 0 at $1.2922928281\,m$ (Section 14.13), called the **bulk edge**; all other levels start at or above it (Figure 14c.1). Status: COMPUTED.

14.20 The band along the deflating history, and the tip

The redshift. By the exact rescaling identity (Section 14.7) every level at the slice $a_{4,0}$ equals the level at the slice 0 with the redshifted momentum: $\varepsilon(k, a_{4,0}) = \varepsilon(ke^{-a_{4,0}}, 0)$. So the whole band shrinks toward zero along the history: at a later slice the same lattice momentum k sits lower on the band. The Rust solver checked the identity for the three momenta $k = 0.25, 1, 2.5$, the five slices and three sectors, and found the two sides equal to the last bit, with the difference 0 (check free_rescaling_relation_and_band_monotone of the Rust report; the same check found the band strictly increasing in k on $[0, 4]$). Status: PROVED (the identity), COMPUTED (the check).

The gap of the eight zero modes along the history. In the free state $N = 8$ the eight zero modes are filled and the lowest empty particle level is the brane band at the first lattice shell, $k = 0.25$. This level is the **gap** of the state. At the five slices it is

slice $a_{4,0}$	**0**	**0.5**	**1**	**1.5**	**2**
gap of the free $N = 8$ state (units of m)	0.4307336786	0.2726448766	0.1703492513	0.1050152416	0.0641594107

(Notebook 14c, Out [7]; the rows $N = 8$ of `Revision/kohn_sham/results/spectrum/closed-shells.csv`). The gap shrinks almost like $e^{-a_{4,0}}$ (for small momenta the band is nearly $c\,ke^{-a_{4,0}}$): as 3-space inflates, the lattice momenta are redshifted and the empty levels come down toward the filled zero modes. Status: COMPUTED. (Chapter 15 asks whether the gas can follow this change: the adiabaticity of the history.)

The tip angle does not matter for the band (at the cutoff $L = 3$ and the slice 0). Section 14.12 showed that an orbital with $k \neq 0$ is suppressed at the cutoff by about $\exp(-|k|(\kappa(-L) - \kappa(y))/H)$, so the band level should hardly change if the tip condition is changed. The Rust solver computed the band with the tip angles $\theta = 0, 0.5, 1$ (record `Revision/kohn_sham/results/spectrum/tip-angle.csv`): at $k = 0.25$ the level moves by -8.58×10^{-6} and -2.74×10^{-5}, while the suppression factor is 8.47×10^{-3}; at $k = 0.5$ by -2.49×10^{-9} and -8.58×10^{-9} against 7.17×10^{-5}; at $k = 1$ only by rounding errors (below 10^{-15}) against 5.14×10^{-9} (check free_tip_angle_insensitivity of the Rust report: every shift lies below the suppression factor). Notebook 14c computes the shifts for ten momenta from 0.1 to 1 (Out [11]): each step of 0.1 in k makes them 11 to 33 times smaller, faster than the suppression factor itself (which falls by $e^{0.1(e^3-1)} \approx 6.7$ per step), until they reach the rounding level of the computer at $k = 0.9$. Status: COMPUTED. Both the record and the notebook work at the slice $a_{4,0} = 0$. So, at the cutoff $L = 3$ and the slice $a_{4,0} = 0$, where it was

measured (record: $k = 0.25, 0.5, 1$; Notebook 14c: $k = 0.1$ to 1), the choice of the tip angle θ in the ASSUMED tip condition is harmless for the brane band at every lattice momentum $k \neq 0$ (the lattice momenta above 1 were not run, but there the suppression factor lies below 5.14×10^{-9} and the computed shifts are at the rounding level already from $k = 0.9$; the other levels with $k \neq 0$ were not computed with another tip angle, and for them only the estimate of Section 14.12 applies); at $k = 0$, where it selects the zero mode, it is a real assumption of the model.

The tip angle at the later slices (not established). By the rescaling identity above, the lattice momentum k at the slice $a_{4,0}$ acts like the momentum $ke^{-a_{4,0}}$ at the slice 0, and there the suppression factor of Section 14.12 is no longer small: for the first lattice shell, $k = 0.25$, it is $\exp(-0.25e^{-a_{4,0}}(e^3 - 1))$, which is 0.173 at the slice 1 and 0.524 at the slice 2. Rule: $0.25(e^3 - 1) = 4.771$; times $e^{-1} = 0.3679$ this is 1.755, and $e^{-1.755} = 0.173$; times $e^{-2} = 0.1353$ it is 0.646, and $e^{-0.646} = 0.524$. The notebook's own smallest momentum points the same way: by the identity, its shift 1.2×10^{-3} for $\theta = 1$ at $k = 0.1$ and the slice 0 (Out [11]) is the shift at $k = 0.25$ and the slice $\ln 2.5 = 0.92$, more than forty times the shift 2.74×10^{-5} at $k = 0.25$ and the slice 0. Neither the Revision record nor a notebook of this chapter computes the tip angle at the later slices, where the gaps of the table above at the slices 0.5 to 2 and the recorded Kohn-Sham states of those slices live; so at those slices the insensitivity to θ is not established (OPEN).

The position L of the cutoff (measured by the Revision record). The angle is one question; where the cutoff sits is another. The Revision record measures it in `Revision/kohn_sham/tip_convergence/` (its README, "Answer in brief"; the report `Revision/kohn_sham/tip_convergence/tip-convergence.json`, 7 of 7 checks PASS; `Revision/docs/KOHN_SHAM_DEFLATING_FIELD.md`, sections 5, 15.3 and 15.4): $L = 3$ to 6 in steps of 0.5, with the step $h = 1/300$ of the record and every run repeated at $h/2$; a copy of the solver that reads L from the environment reproduces the committed matrix byte for byte at $L = 3$, and the independent reference solver agrees at $L = 3$ and 4. No notebook of this chapter repeats the study; its findings, with the record's numbers:

- The free results with $k \neq 0$ converge as L grows, faster than exponentially, because of the suppression of Section 14.12 (at the cutoff it is $\exp(-|k|e^{L-a_{4,0}}/H)$). But the recorded $L = 3$ values are low by up to 1.4% in the Kohn-Sham energy E_{KS} and 12% in the y-pressure integral $2\mathrm{Vol}_7 \int e^{6Hy} p_0 \, dy$ at $a_{4,0} = 2$ ($N = 136$, $\lambda = 0$: 12.44507 against the extrapolated 12.62207, and 8.47270 against 9.63839): at the late slice the redshifted brane band reaches the tip. The gap of the free $N = 8$ state in the table above is low in the same way: 0.4307336786 against the extrapolated 0.4307456266 at the slice 0 (2.8×10^{-5} relative), but 0.06415941073 against 0.06563459278 at the slice 2 (2.2×10^{-2} relative).

- The nonzero $k = 0$ levels approach $\pm m$ only algebraically, like $(n\pi)^2/(2mL^2)$ (for large L, from the exact levels of Section 14.13, $\sqrt{m^2 + (n\pi/L)^2} - m \approx (n\pi)^2/(2mL^2)$; the record verifies all 84 $k = 0$ levels of its scan against the exact formulas to $1.42 \times 10^{-10}\, m$). The bulk edge $1.2923\, m$ that defines $N = 688$ (Section 14.21) is an $L = 3$ value: $1.0977\, m$ at $L = 6$, and m as $L \to \infty$. From $L = 3.5$ on, the $N = 688$ ground state at $a_{4,0} = 0$ occupies $k = 0$ bulk levels, its Kohn-Sham gap 0.0452 closes, and its energy does not converge up to $L = 6$.

- The interaction energy of the brane zero modes depends strongly on L. Their proper density grows toward the tip like $e^{(6H-2m)|y|}$ (Section 14.27; exact, and verified by the record), so for $N = 8$ the recorded $E_{\mathrm{KS}} = \mp 9.868 \times 10^{-4}$ at $L = 3$ ($\lambda = \pm\lambda_1$, every slice) is about one third of the large-L value $\mp 3.0046 \times 10^{-3}$ (extrapolated from $\lambda = -\lambda_1$ at $a_{4,0} = 2$, the only interacting $N = 8$ state solved up to $L = 6$). The self-consistent iteration fails at some larger

L for 13 of the 19 interacting states studied (a failure of the iteration, not a proof that no self-consistent state exists), and for 7 of them the limit of E_{KS} is not established.

Status: COMPUTED by the Revision record with its two solvers (the L-dependence of the $k = 0$ levels and of the zero-mode density is exact and verified there). So the cutoff $L = 3$ is itself a choice that shapes the recorded numbers: within the percentages above for the free results with $k \neq 0$, and decisively for the nonzero $k = 0$ levels, for $N = 688$ at $a_{4,0} = 0$ and for the interaction energy of the zero modes.

14.21 Particles, degeneracies and closed shells

Lattice shells. On the 3-torus the allowed momenta are $\mathbf{k} = \Delta k\,(n_1, n_2, n_3)$ with whole numbers n_i (Section 14.3). All vectors with the same $n^2 = n_1^2 + n_2^2 + n_3^2$ have the same length $|\mathbf{k}| = \Delta k\sqrt{n^2}$ and form a **shell**; their number is written $r_3(n^2)$. For example $r_3(0) = 1$ (only $(0,0,0)$), $r_3(1) = 6$ (the six vectors $(\pm1,0,0)$, $(0,\pm1,0)$, $(0,0,\pm1)$) and $r_3(2) = 12$ (two entries ±1 and one 0: three places for the 0 times four sign choices). The first values are $r_3 = 1, 6, 12, 8, 6, 24, 24, 12, 30$ for $n^2 = 0, 1, 2, 3, 4, 5, 6, 8, 9$; no vector has $n^2 = 7$, because the squares below 8 are 0, 1 and 4, and no three of them add up to 7.

Degeneracy. The levels depend on $|\mathbf{k}|$ only (Section 14.6), and the four blocks (s_2, s_3) of one type have the same Hamiltonian (Section 14.5). So every level of the block type j on the shell n^2 holds $4r_3(n^2)$ states: four blocks times $r_3(n^2)$ directions. Status: PROVED (the argument above; the record states it as blockEquation.degeneracy in `Revision/kohn_sham/ks-theory.json`).

Particles and the sea (ASSUMED convention; its justification is OPEN). The free spectrum has positive and negative levels. In the quantised theory (Chapter 10) the field is normal ordered: the negative levels form the filled **sea**, which is not counted, and the quanta that are counted, the **particles**, occupy the positive levels. The Revision theory adopts the following rule: in each sector (shell, block type, parity) the particle levels are those whose level in the free problem ($\lambda = 0$) is positive, together with the zero modes at $k = 0$, each followed continuously when the interaction is switched on; the sea contributes nothing to the particle number and the densities. The zero modes need a decision: at $k = 0$ and $v = 0$ the spectrum of h_{+1} is minus that of h_{-1}, so a level at exactly zero is its own mirror image and the sign does not decide whether it is a particle or a sea level. Counting it as a particle level is a stated CONVENTION of the record (ks-theory.json, thermodynamics.fillingConvention), and its justification is OPEN. (The words particle and sea are used here in this technical sense only; nothing in this chapter is a statement about matter and antimatter, which Chapter 21 treats.)

Because Φ grows with ε (Section 14.14), the particle levels of a sector are exactly the labels $l \geq l_{\min}$, where $l_{\min}$ is the first label whose target lies above $\Phi(0)$, the value of the shooting function at $\varepsilon = 0$ (and at $k = 0$, even parity, the zero mode has $\Phi(0) = 0$ exactly on the target of label 0). The Rust solver verified this for every shell $n^2 \leq 30$ and every sector, and found that the level closest to zero, apart from the zero modes, is $0.4307\,m$: the brane band at the first shell (check free_particle_branch_labels of the Rust report). Status: COMPUTED.

Closed shells. The **aufbau** (German for building up) fills the particle states from the lowest level upward. A particle number N is a **closed shell** when the filling ends exactly at the end of a group of degenerate levels (levels equal within 10^{-9}), so that the state is unique. At the slice

0 the first closed shells are built like this (record `Revision/kohn_sham/results/spectrum/closed-shells.csv`, reproduced by Notebook 14c, Out [14]):

N	**the last group filled**	**states added**	**its level (units of m)**
8	the zero modes, both block types	$4 + 4 = 8$	0
32	the brane band on the shell $n^2 = 1$	$4 \times 6 = 24$	0.4307336786
80	the brane band on the shell $n^2 = 2$	$4 \times 12 = 48$	0.5879568528
112	the brane band on the shell $n^2 = 3$	$4 \times 8 = 32$	0.7040995646
136	the brane band on the shell $n^2 = 4$	$4 \times 6 = 24$	0.7996457668
232	the brane band on the shell $n^2 = 5$	$4 \times 24 = 96$	0.8823195405

Only the brane band of the blocks $j = +1$ appears on the shells $n^2 \geq 1$ in this table: the band of the blocks $j = -1$ is negative (sea), and every other level lies higher. The list of closed shells at the slice 0 continues $328, 376, 496, 592, 688, 696, 728, \ldots$ up to the energy $1.6\,m$ (22 closed shells in all). The shell $N = 696$ is the first that fills a level off the brane band: the odd level of label 0 at $k = 0$, at the bulk edge $1.2922928281\,m$.

The particle numbers of the Revision runs. The Revision solver chose its three particle numbers by a rule fixed in advance (record `Revision/kohn_sham/results/parameters.json`, particleNumbers): $N = 8$, the zero modes; N_{large}, the largest closed shell whose last filled level lies below the bulk edge, which is 688; and N_{mid}, the closed shell nearest to $N_{\text{large}}/4 = 172$, which is 136 (the neighbours 136 and 232 are 36 and 60 away). Status: COMPUTED; Notebook 14c repeats the rule and finds 8, 136, 688. The rule is applied at the cutoff $L = 3$: the bulk edge is an $L = 3$ value ($1.0977\,m$ at $L = 6$, and m as $L \to \infty$), so $N = 688$ is itself an $L = 3$ choice (Section 14.20).

At a later slice. At the slice $a_{4,0} = 0.5$ the redshifted band holds many more states below the same energy: 55 closed shells below $1.6\,m$ instead of 22 (Notebook 14c, Out [14]; Figure 14c.6).

14.22 Example: Notebook 14c, the brane band

Notebook 14c does Sections 14.19 to 14.21 with the computer, with the shooting method of Notebook 14b (now also for nonzero 3-momenta and any slice). It computes the brane band and three other levels for k from 0 to 4 and compares 324 levels with the Rust record; it derives the slope formula with sympy, checks the integral of Section 14.19 on the numerical zero mode, measures the slope by Richardson extrapolation at the five slices and for two other cutoffs; it checks the rescaling identity for 45 levels and the gap of the state $N = 8$ at the five slices; it checks the two block-type symmetries; it measures the effect of the tip angle at the cutoff $L = 3$ and the slice 0; and it builds the lattice shells, the particle labels, the closed shells at the slices 0 and 0.5 and the particle numbers of the Revision runs, all against the Rust records. It draws six figures and needs no Rust. It runs in about 90 seconds (the cells of its sections 8 to 11 integrate the equation for hundreds of energies at once, 72 times over, and take 5 to 20 seconds each), and its last line is ALL 15 CHECKS PASSED (notebook 14c).

14.23 How to run Notebook 14c

Step 1. What this notebook does and what it needs.

Notebook 14c (The brane band: free Kohn-Sham levels at nonzero 3-momentum along the deflating history) is the file `Revision/textbook/notebooks/14c_brane_band.ipynb` of the repository Dirac_claude. With the fourth-order Runge-Kutta shooting function of the Revision Rust solver (the levels found by bisection) it computes the free Kohn-Sham levels of dirac16complex at nonzero 3-momentum k: the brane band that grows out of the zero modes, its slope c at k = 0 from the Hellmann-Feynman rule and its exact formula, the redshift of the band along the prescribed deflating history and the exact rescaling identity eps(k, a4) = eps(k e^(-a4), 0), the mirror symmetries of the two block types, the insensitivity of the band to the tip angle at the cutoff L = 3 and the slice 0, the particle branch, the lattice degeneracies 4 r3(n2) and the closed shells of the free aufbau with the particle numbers 8, 136 and 688; it reproduces the Revision Rust solver's records of all of these and draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
```

```
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 14c_brane_band.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a

number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 14c_brane_band.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 14c_brane_band.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/14c.captions.json`
- `Revision/textbook/figures/14c_1_band_structure.png`
- `Revision/textbook/figures/14c_2_band_slope.png`
- `Revision/textbook/figures/14c_3_band_redshift.png`
- `Revision/textbook/figures/14c_4_block_type_mirror.png`
- `Revision/textbook/figures/14c_5_tip_insensitivity.png`
- `Revision/textbook/figures/14c_6_closed_shells.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files of this notebook exist
ALL 15 CHECKS PASSED (notebook 14c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/14c_brane_band.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 14c_brane_band.ipynb
python -m nbconvert --execute --inplace 14c_brane_band.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- the cells of sections 8 to 11 run for 5 to 20 seconds each: they integrate the equation for hundreds of energies at once, 72 times over; wait until the PASS lines appear.

14.24 Notebook 14c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/14c_brane_band.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

At zero 3-momentum the free Kohn-Sham spectrum of dirac16complex contains eight
zero modes ($\varepsilon = 0$) sitting at the brane. This notebook follows what
happens when the 3-momentum $k$ is switched on, at the five slices
$a_{4,0} = 0, 0.5, 1, 1.5, 2$ of the PRESCRIBED deflating history. It

- computes the BRANE BAND (the level that grows out of the zero modes) and three
  other low levels for $k$ from 0 to 4 and reproduces the Rust solver's record
  (figure 1);
- derives the slope $c = d\varepsilon/dk$ at $k = 0$ with the Hellmann-Feynman
  rule, $c = e^{-a_{4,0}}\,\frac{2M}{2M - H}\,\frac{1 - e^{-(2M-H)L}}
  {1 - e^{-2ML}}$, and checks it numerically at every slice (figure 2);
- checks the exact rescaling identity $\varepsilon(k, a_{4,0}) =
  \varepsilon(k\,e^{-a_{4,0}}, 0)$: along the history the band is redshifted
  (figure 3);
- checks the mirror symmetries of the two block types $j = \pm1$ (figure 4);
- shows that, at the cutoff $L = 3$ and the slice $a_{4,0} = 0$, the band does
  not feel the tip angle $\theta$ (figure 5);
- finds the particle branch, the degeneracies $4r_3(n^2)$ of the torus lattice
  and the closed shells of the free aufbau, with the particle numbers
  $N = 8, 136, 688$ of the Revision runs (figure 6).

It prints a PASS line for every check (15 in all) and saves six figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 14c (The brane band: free Kohn-Sham levels at nonzero 3-momentum along the
deflating history) is the file ‘Revision/textbook/notebooks/14c_brane_band.ipynb‘ of the
repository Dirac_claude. With the fourth-order Runge-Kutta shooting function of the
Revision Rust solver (the levels found by bisection) it computes the free Kohn-Sham
levels of dirac16complex at nonzero 3-momentum k: the brane band that grows out of the
zero modes, its slope c at k = 0 from the Hellmann-Feynman rule and its exact formula,
the redshift of the band along the prescribed deflating history and the exact rescaling
identity eps(k, a4) = eps(k e^(-a4), 0), the mirror symmetries of the two block types,
the insensitivity of the band to the tip angle at the cutoff L = 3 and the slice 0, the
particle branch, the lattice degeneracies 4 r3(n2) and the closed shells of the free
```

aufbau with the particle numbers 8, 136 and 688; it reproduces the Revision Rust solver's records of all of these and draws six teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 14c_brane_band.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 14c_brane_band.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 14c_brane_band.ipynb‘ (in the same folder, without running the cells again) to read the

printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/14c.captions.json‘
- ‘Revision/textbook/figures/14c_1_band_structure.png‘
- ‘Revision/textbook/figures/14c_2_band_slope.png‘
- ‘Revision/textbook/figures/14c_3_band_redshift.png‘
- ‘Revision/textbook/figures/14c_4_block_type_mirror.png‘
- ‘Revision/textbook/figures/14c_5_tip_insensitivity.png‘
- ‘Revision/textbook/figures/14c_6_closed_shells.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all six figure files of this notebook exist
ALL 15 CHECKS PASSED (notebook 14c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/14c_brane_band.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

    python -m jupyterlab 14c_brane_band.ipynb
    python -m nbconvert --execute --inplace 14c_brane_band.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- the cells of sections 8 to 11 run for 5 to 20 seconds each: they integrate the equation for hundreds of energies at once, 72 times over; wait until the PASS lines appear.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 14c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 14c (The brane band: free Kohn-Sham levels at nonzero 3-momentum along the
# deflating history) is the file Revision/textbook/notebooks/14c_brane_band.ipynb of the
# repository Dirac_claude. With the fourth-order Runge-Kutta shooting function of the
# Revision Rust solver (the levels found by bisection) it computes the free Kohn-Sham
# levels of dirac16complex at nonzero 3-momentum k: the brane band that grows out of the
# zero modes, its slope c at k = 0 from the Hellmann-Feynman rule and its exact formula,
# the redshift of the band along the prescribed deflating history and the exact rescaling
# identity eps(k, a4) = eps(k e^(-a4), 0), the mirror symmetries of the two block types,
# the insensitivity of the band to the tip angle at the cutoff L = 3 and the slice 0, the
# particle branch, the lattice degeneracies 4 r3(n2) and the closed shells of the free
# aufbau with the particle numbers 8, 136 and 688; it reproduces the Revision Rust
# solver's records of all of these and draws six teaching figures. It needs a computer
# with Windows 11, macOS or Linux, an internet connection for the installation, the
# program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
```

```
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
```

```
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
```

```
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 14c_brane_band.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 14c_brane_band.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 14c_brane_band.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/14c.captions.json
# - Revision/textbook/figures/14c_1_band_structure.png
# - Revision/textbook/figures/14c_2_band_slope.png
# - Revision/textbook/figures/14c_3_band_redshift.png
# - Revision/textbook/figures/14c_4_block_type_mirror.png
# - Revision/textbook/figures/14c_5_tip_insensitivity.png
# - Revision/textbook/figures/14c_6_closed_shells.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
```

```
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all six figure files of this notebook exist
#     ALL 15 CHECKS PASSED (notebook 14c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/14c_brane_band.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 14c_brane_band.ipynb
#     python -m nbconvert --execute --inplace 14c_brane_band.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - the cells of sections 8 to 11 run for 5 to 20 seconds each: they integrate the
#   equation for hundreds of energies at once, 72 times over; wait until the PASS lines
#   appear.
#
```

```
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "14c"  # this notebook: chapter 14, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
```

```
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 14c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Level, orbital, label**: an energy $\varepsilon$ of the block equation
  $h_j\chi = \varepsilon\chi$, its solution, and the integer that counts its
  Pruefer half-turns (even parity: $\Phi = l\pi$, odd: $\Phi = \pi/2 + l\pi$).
- **3-momentum $k$**: the momentum along ordinary 3-space; on the coordinate
  torus of size $\ell$ the allowed vectors are $\Delta k\,(n_1, n_2, n_3)$ with
  integers $n_i$ and $\Delta k = 2\pi/\ell = 0.25$ (in units of $H$).
- **Shell**: all lattice vectors with the same $n^2 = n_1^2 + n_2^2 + n_3^2$;
  $r_3(n^2)$ is their number (for example $r_3(1) = 6$).
- **Brane band**: the even-parity level of label 0 in the blocks $j = +1$, which
  is the zero mode at $k = 0$ and rises with $k$.
- **Slope $c$**: $d\varepsilon/dk$ of the brane band at $k = 0$.
- **Hellmann-Feynman rule**: for a normalised eigenvector, the derivative of
  the level is the expectation value of the derivative of the Hamiltonian:
  $d\varepsilon/dk = \langle\chi|\,\partial h/\partial k\,|\chi\rangle$.
- **Slice, history**: the slice $a_{4,0}$ is the value of $a_4$ at one instant;
  the history $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND (given, not solved for).
- **Redshift**: a 3-momentum $k$ acts like $k\,e^{-a_{4,0}}$ at the slice
  $a_{4,0}$: momenta shrink as 3-space inflates.
- **Tip condition**: the condition $(1 - Q(\theta))\chi(-L) = 0$ at the cutoff
  $y = -L$; $\theta = 0$ ($b(-L) = 0$) is the canonical choice.
- **Particle branch, sea**: by a CONVENTION of the Revision theory, the levels
  that are positive in the free problem (and the $k = 0$ zero modes) are
  particle levels; the negative branch is the filled, normal-ordered sea.
- **Aufbau, closed shell**: fill the $N$ lowest particle states; $N$ is a closed
  shell when the filling ends exactly at the end of a degenerate group of levels.
- **Bulk edge**: the lowest level that is not on the brane band (here a $k = 0$
  level of the odd parity).
- **Richardson extrapolation**: combining two finite-difference estimates so that
  their leading errors cancel.
```

Text cell:

```
## 4. The physical and mathematical situation

In each $2\times2$ block of type $j$ the free Kohn-Sham equation is
$h_j\chi = \varepsilon\chi$ with $h_j = j[-i\sigma_1\,d/dy + M\sigma_2 +
\kappa(y)k\,\sigma_3]$, the momentum weight $\kappa(y) = e^{-Hy - a_{4,0}}$, the
ASSUMED Z2 mirror brane at $y = 0$ (even: $b(0) = 0$, odd: $a(0) = 0$) and the
regular tip $b(-L) = 0$ at the cutoff $y = -L$; in real form $\chi = (a, ib)$:
$a' = Ma - (\kappa k + j\varepsilon)b$, $b' = (j\varepsilon - \kappa k)a - Mb$.

The momentum term $\kappa k\sigma_3$ pushes the orbitals toward the brane
($\kappa$ grows toward the tip) and lifts the zero modes into a band. Every level
holds $4r_3(n^2)$ states for each block type (four blocks times the number of
lattice directions of the shell). The canonical parameters of the Revision
solver are $H = m = 1$, $L = 3$, $\Delta k = 0.25$, the slices $a_{4,0} = 0,
0.5, 1, 1.5, 2$, and RK4 with $G = 900$ steps.

Status: the slope formula and the rescaling identity are PROVED (exact); the
numerical levels are COMPUTED (they reproduce the Rust solver's records); the
brane condition is ASSUMED; the tip condition is a chosen cutoff condition; the
history is a PRESCRIBED BACKGROUND; the counting of the zero modes as particles
is a CONVENTION whose justification is OPEN.
```

Text cell:

```
## 5. The shooting tools

The next cell defines the numerical tools. ‘shoot‘ is the shooting function of
the Revision Rust solver: it integrates the real form from the tip ($(a, b) =
(\cos\frac\theta2, \sin\frac\theta2)$, canonical $\theta = 0$) to the brane with
classical RK4 ($G$ steps; the coefficients at the nodes and step midpoints of a
fine grid of $2G + 1$ points) and follows the Pruefer angle $\mathrm{atan2}(b, a)$
step by step; it returns $\Phi = j\,\theta(0)$, which grows strictly with
$\varepsilon$. ‘levels‘ finds the energy with $\Phi = $ target (even $l\pi$, odd
$\pi/2 + l\pi$) by 72 bisections of $[-20, 20]$ (the Rust solver finds the same
root with a safeguarded Newton-bisection that stops at the tolerance
$10^{-13}$). ‘orbital‘ returns a normalised orbital on the fine grid (cubic
Hermite values at the midpoints, Simpson's rule for the norm). All three accept
arrays (numpy), so that hundreds of levels are found at once. It also reads the
Rust solver's records with a small CSV reader and defines ‘record_check‘, which
reads the verdict of a named check.
```

In [2]:

```
import math  # exp, cos, sin of single numbers

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra with symbols

RUST_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
SPECTRUM = "Revision/kohn_sham/results/spectrum"  # the folder of the records
```

```
def record_check(name, report=RUST_REPORT):
    """The check called name is PASS in the Revision report."""
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    return [c["verdict"] for c in data["checks"] if c["name"] == name] == ["PASS"]


def read_csv(relative):
    """The rows of a CSV record as a list of dictionaries (column -> text)."""
    lines = repository_file(relative).read_text(encoding="utf-8").splitlines()
    header = lines[0].split(",")
    return [dict(zip(header, line.split(","))) for line in lines[1:]]


def shoot(eps, k=0.0, j=1.0, a4=0.0, M=1.0, L=3.0, G=900, H=1.0, tip=0.0,
          keep=False):
    """Phi = j theta(0) for the energies eps (arrays allowed for eps, k, j, a4);
    with keep=True also theta, a, b at the G + 1 nodes."""
    eps, k, j, a4 = np.broadcast_arrays(*(np.asarray(x, dtype=float)
                                          for x in (eps, k, j, a4)))
    nf = 2 * G + 1  # nodes and step midpoints
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    e_minus_Hy = np.exp(-H * y_fine)
    kk = k * np.exp(-a4)  # kappa k = kk e^(-Hy)
    h = L / G
    a_ = np.full(eps.shape, math.cos(0.5 * tip))
    b_ = np.full(eps.shape, math.sin(0.5 * tip))
    theta = np.full(eps.shape, 0.5 * tip)
    raw = np.arctan2(b_, a_)
    je = j * eps
    path = [(theta, a_, b_)]
    for i in range(G):
        K0, K1, K2 = (kk * e_minus_Hy[2 * i], kk * e_minus_Hy[2 * i + 1],
                      kk * e_minus_Hy[2 * i + 2])
        p1a, p1b = M * a_ - (K0 + je) * b_, (je - K0) * a_ - M * b_
        a2, b2 = a_ + 0.5 * h * p1a, b_ + 0.5 * h * p1b
        p2a, p2b = M * a2 - (K1 + je) * b2, (je - K1) * a2 - M * b2
        a3, b3 = a_ + 0.5 * h * p2a, b_ + 0.5 * h * p2b
        p3a, p3b = M * a3 - (K1 + je) * b3, (je - K1) * a3 - M * b3
        a4_, b4_ = a_ + h * p3a, b_ + h * p3b
        p4a, p4b = M * a4_ - (K2 + je) * b4_, (je - K2) * a4_ - M * b4_
        a_ = a_ + h / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)
        b_ = b_ + h / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
        new = np.arctan2(b_, a_)
        step = new - raw  # brought into (-pi, pi]
        step = np.where(step > np.pi, step - 2 * np.pi,
                        np.where(step <= -np.pi, step + 2 * np.pi, step))
        theta = theta + step
        raw = new
        if keep:
            path.append((theta, a_, b_))
    if keep:
        return j * theta, [np.array(q) for q in zip(*path)]
    return j * theta
```

```
def target(parity, label):
    offset = np.where(np.asarray(parity) == "even", 0.0, 0.5 * np.pi)
    return offset + np.asarray(label, dtype=float) * np.pi


def levels(k, j, parity, label, a4=0.0, iterations=72, **options):
    """The levels with these labels (arrays allowed), by bisection."""
    k, j, parity, label, a4 = np.broadcast_arrays(
        np.asarray(k, dtype=float), np.asarray(j, dtype=float), np.asarray(parity),
        np.asarray(label), np.asarray(a4, dtype=float))
    goal = target(parity, label)
    lo, hi = np.full(k.shape, -20.0), np.full(k.shape, 20.0)
    for _ in range(iterations):
        mid = 0.5 * (lo + hi)
        g = shoot(mid, k, j, a4, **options) - goal
        lo = np.where(g <= 0.0, mid, lo)
        hi = np.where(g >= 0.0, mid, hi)
    return 0.5 * (lo + hi)


def orbital(eps, k=0.0, j=1.0, a4=0.0, M=1.0, L=3.0, G=900, H=1.0):
    """The normalised orbital (y, a, b) and the Simpson weights on the fine grid."""
    _, (_, a_n, b_n) = shoot(eps, k, j, a4, M=M, L=L, G=G, H=H, keep=True)
    nf, h = 2 * G + 1, L / G
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    K_fine = k * math.exp(-a4) * np.exp(-H * y_fine)
    a_f, b_f = np.zeros(nf), np.zeros(nf)
    a_f[0::2], b_f[0::2] = a_n, b_n
    da = M * a_f - (K_fine + j * eps) * b_f
    db = (j * eps - K_fine) * a_f - M * b_f
    for f in range(1, nf, 2):  # cubic Hermite values at the midpoints
        a_f[f] = 0.5 * (a_f[f - 1] + a_f[f + 1]) + h / 8.0 * (da[f - 1] - da[f + 1])
        b_f[f] = 0.5 * (b_f[f - 1] + b_f[f + 1]) + h / 8.0 * (db[f - 1] - db[f + 1])
    weights = np.full(nf, 2.0)  # Simpson weights 1, 4, 2, ..., 4, 1 times (h/2)/3
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    weights *= 0.5 * h / 3.0
    norm = math.sqrt(float(np.sum(weights * (a_f ** 2 + b_f ** 2))))
    return y_fine, a_f / norm, b_f / norm, weights


SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]  # the slices of the Revision solver
say("defined: shoot, levels, orbital, read_csv, record_check")
```

Out [2]:

```
defined: shoot, levels, orbital, read_csv, record_check
```

Text cell:

```
## 6. The brane band and three other levels for k from 0 to 4
```

```
The Rust solver's record `brane-band.csv` lists, for $k = 0, 0.05, 0.10, \dots,
4$ at the slice $a_{4,0} = 0$, the brane band (block type $j = +1$, even, label
0), the next even level of $j = +1$ (label 1), the lowest odd level of $j = +1$
(label 0), and the even level of label 1 in the blocks $j = -1$. The next cell
computes these $4\times81$ levels in one call (and, for the figure, the brane
band of the blocks $j = -1$, label 0), and checks that they equal the record to
$10^{-11}$. The $k$ values are made exactly as in the Rust program, by adding
0.05 again and again (so that the last digits agree). It also checks that the
band rises strictly with $k$.
```

In [3]:

```
k_list, kk = [], 0.0
while kk <= 4.0 + 1e-12:  # 0, 0.05, 0.10, ... as the Rust program makes them
    k_list.append(kk)
    kk += 0.05
k_band = np.array(k_list)
sectors = [(1.0, "even", 0), (1.0, "even", 1), (1.0, "odd", 0), (-1.0, "even", 1),
           (-1.0, "even", 0)]  # (j, parity, label); the last one for the figure
K_all = np.concatenate([k_band] * len(sectors))
J_all = np.concatenate([[j_] * len(k_band) for j_, _, _ in sectors])
P_all = np.concatenate([[p_] * len(k_band) for _, p_, _ in sectors])
L_all = np.concatenate([[l_] * len(k_band) for _, _, l_ in sectors])
found = levels(K_all, J_all, P_all, L_all).reshape(len(sectors), len(k_band))
band, bulk_even, odd0, minus_even1, minus_band = found
rows = read_csv(f"{SPECTRUM}/brane-band.csv")
record = np.array([[float(r[c]) for r in rows] for c in
                   ("eps_band_a0", "eps_band_bulk_even_l1", "eps_odd_l0",
                    "eps_jm1_even_l1")])
k_record = np.array([float(r["k"]) for r in rows])
say(f"k = 0.25: brane band {band[5]:.10f}, odd level {odd0[5]:.10f} (units of m)")
check(len(rows) == 81 and np.max(np.abs(k_record - k_band)) < 1e-15
      and np.max(np.abs(found[:4] - record)) < 1e-11,
      "the 324 levels equal the record brane-band.csv",
      record=f"{SPECTRUM}/brane-band.csv")
check(bool(np.all(np.diff(band) > 0.0)) and np.max(np.abs(minus_band + band)) < 1e-12,
      "the band rises strictly with k; the j = -1 band is its mirror image -eps")
```

Out [3]:

```
k = 0.25: brane band 0.4307336786, odd level 1.9195133815 (units of m)
PASS the 324 levels equal the record brane-band.csv
     reproduces Revision/kohn_sham/results/spectrum/brane-band.csv
PASS the band rises strictly with k; the j = -1 band is its mirror image -eps
```

Text cell:

```
The next cell draws the band structure: the five levels against $k$, with the
straight line $c\,k$ of the first-order slope (section 7) and the bulk edge
$1.2923\,m$ (the lowest odd level at $k = 0$, the lowest level not on the band).
```

In [4]:

```
c_theory = 2.0 / (1.0 + math.exp(-3.0))  # the slope for M = H = 1, L = 3, a4,0 = 0
fig, ax = plt.subplots(figsize=(9.5, 4.8))
ax.plot(k_band, band, color="C0", linewidth=2, label="brane band ($j = +1$, even, 0)")
ax.plot(k_band, bulk_even, color="C1", label="$j = +1$, even, label 1")
ax.plot(k_band, odd0, color="C2", label="$j = +1$, odd, label 0")
ax.plot(k_band, minus_even1, color="C4", label="$j = -1$, even, label 1")
ax.plot(k_band, minus_band, "--", color="C0",
        label="$j = -1$, even, label 0 (sea)")
ax.plot(k_band[:13], c_theory * k_band[:13], ":", color="black",
        label="first order: $c\\,k$, $c = 1.9051$")
ax.axhline(odd0[0], color="gray", linewidth=0.8, linestyle="-.")
ax.annotate("bulk edge 1.2923", (2.6, odd0[0] - 0.55), fontsize=8,
            color="gray")  # just below the dash-dotted line
ax.set_xlabel("3-momentum $k$ (units of $H$), slice $a_{4,0} = 0$")
ax.set_ylabel("level $\\varepsilon$ (units of $m$)")
ax.set_title("the free levels at nonzero 3-momentum ($m = 1$, $L = 3$)")
ax.legend(fontsize=7, loc="center left", bbox_to_anchor=(1.01, 0.5))
save_figure(fig, "band_structure",
            "The free Kohn-Sham levels against the 3-momentum $k$ (units of "
            "$H$) at the slice $a_{4,0} = 0$ for $m = 1$, $L = 3$; vertical axis "
            "the level in units of $m$. The brane band (thick) starts at the zero "
            "mode and rises with the slope $c = 1.9051$ (dotted line) and then "
            "more slowly; the other levels start at or above the bulk edge "
            "$1.2923\\,m$ (dash-dotted); the dashed curve is the brane band of "
            "the blocks $j = -1$, the mirror image $-\\varepsilon$, which belongs "
            "to the sea.")
```

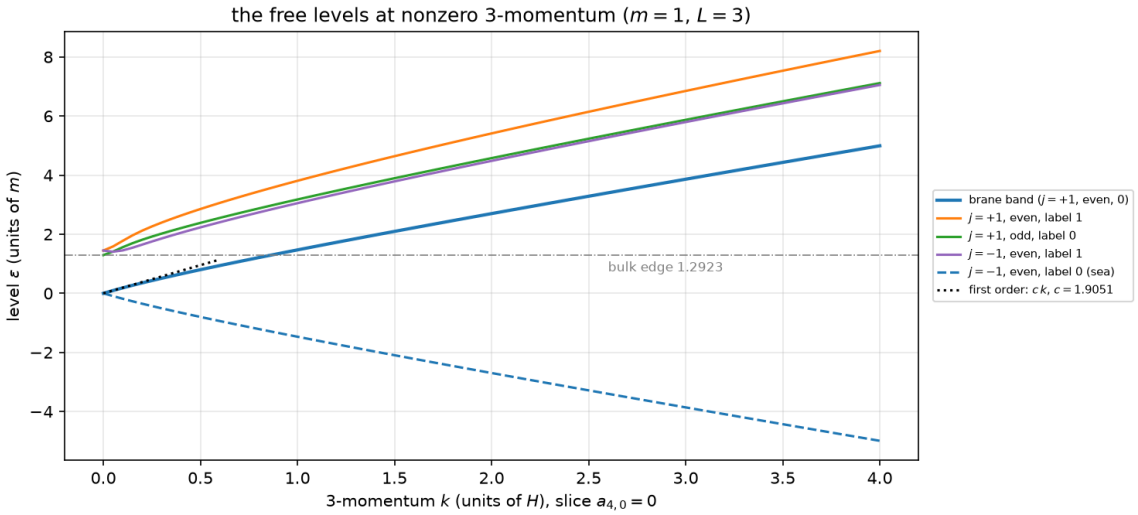

Figure 395: The free Kohn-Sham levels against the 3-momentum k (units of H) at the slice $a_{4,0} = 0$ for $m = 1$, $L = 3$; vertical axis the level in units of m. The brane band (thick) starts at the zero mode and rises with the slope $c = 1.9051$ (dotted line) and then more slowly; the other levels start at or above the bulk edge $1.2923\,m$ (dash-dotted); the dashed curve is the brane band of the blocks $j = -1$, the mirror image $-\varepsilon$, which belongs to the sea. (Notebook 14c, figure 1.)

Out [4]:

```
Figure 14c.1 saved as Revision/textbook/figures/14c_1_band_structure.png
```

Text cell:

```
## 7. The slope of the band at k = 0: the Hellmann-Feynman rule

Line by line:

1. If $h(k)\chi = \varepsilon(k)\chi$ with $\int\chi^\dagger\chi\,dy = 1$ and
   boundary conditions that do not depend on $k$, then $\varepsilon =
   \int\chi^\dagger h\chi\,dy$, and differentiating, the terms with
   $d\chi/dk$ add up to $\varepsilon\,\frac{d}{dk}\int\chi^\dagger\chi\,dy = 0$
   ($h$ is self-adjoint): $d\varepsilon/dk = \int\chi^\dagger(\partial_k h)\chi
   \,dy$.
2. Here $\partial_k h_j = j\kappa(y)\sigma_3$ and, at $k = 0$, $\chi$ is the zero
   mode $(a, 0)$ with $a^2 \propto e^{2My}$, so $\chi^\dagger\sigma_3\chi = a^2$.
3. Hence $d\varepsilon/dk = jc$ with
   $c = \int_{-L}^0 e^{-Hy - a_{4,0}}e^{2My}\,dy\,/\int_{-L}^0 e^{2My}\,dy$.
4. Doing the two integrals: $c = e^{-a_{4,0}}\,\frac{2M}{2M - H}\,
   \frac{1 - e^{-(2M - H)L}}{1 - e^{-2ML}}$; for $M = H = 1$:
   $c = e^{-a_{4,0}}\,\frac{2(1 - e^{-L})}{1 - e^{-2L}} = \frac{2e^{-a_{4,0}}}
   {1 + e^{-L}}$, which is $1.9051482536$ for $L = 3$, $a_{4,0} = 0$.

The next cell does step 4 with sympy, compares the value with the Revision
record (‘checksNumeric.braneBandSlope_M1_H1_L3_a0‘ of ks-theory.json), computes
the integral of step 3 numerically from the zero-mode orbital (Simpson), and
measures the slope numerically at the five slices as the Rust solver did:
levels at $k = 10^{-4}$ and $2\times10^{-4}$, and the Richardson combination
$c = (4\varepsilon_1/10^{-4} - \varepsilon_2/(2\times10^{-4}))/3$, which removes
the $k^2$ error of $\varepsilon/k$. These must equal the record
‘brane-band-slope.csv‘. Finally it measures the slope for $L = 2$ and $L = 4$.
```

In [5]:

```
Ms, Hs, Ls = sp.symbols("M H L", positive=True)
ys, a4s = sp.symbols("y a", real=True)
ratio = (sp.integrate(sp.exp(-Hs * ys - a4s) * sp.exp(2 * Ms * ys), (ys, -Ls, 0))
         / sp.integrate(sp.exp(2 * Ms * ys), (ys, -Ls, 0)))  # step 3
formula = (sp.exp(-a4s) * 2 * Ms / (2 * Ms - Hs) * (1 - sp.exp(-(2 * Ms - Hs) * Ls))
           / (1 - sp.exp(-2 * Ms * Ls)))  # step 4
same = sp.simplify((ratio - formula).subs({Ms: 1, Hs: 1, Ls: 3})) == 0 and \
    sp.simplify((ratio - formula).subs({Ms: 2, Hs: 1, Ls: 3})) == 0
c_exact = float(formula.subs({Ms: 1, Hs: 1, Ls: 3, a4s: 0}))
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
c_record = float(theory["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
report("slope c for M = H = 1, L = 3, a4,0 = 0", f"{c_exact:.13f}", "m/H")
check(same and abs(c_exact - c_record) < 1e-15 and abs(c_exact - c_theory) < 1e-15
      and record_check("brane_band_slope",
```

```
                    "Revision/kohn_sham/reports/ks-theory-python.json"),
      "c = e^(-a4,0) (2M/(2M - H)) (1 - e^(-(2M-H)L))/(1 - e^(-2ML)) = 1.9051482536",
      record="Revision/kohn_sham/ks-theory.json, checksNumeric "
             "braneBandSlope_M1_H1_L3_a0")
y_f, a_zero, b_zero, simpson = orbital(0.0)  # the zero mode at k = 0
c_integral = float(np.sum(simpson * np.exp(-y_f) * a_zero ** 2))  # step 3
check(abs(c_integral - c_exact) < 1e-9,
      "the integral of kappa a^2 over the numerical zero mode gives c")
tiny = np.array([1e-4, 2e-4] * 5)  # two small momenta at each slice
slice_of = np.repeat(SLICES, 2)
small = levels(tiny, 1.0, "even", 0, a4=slice_of).reshape(5, 2)
c_numeric = (4.0 * small[:, 0] / 1e-4 - small[:, 1] / 2e-4) / 3.0
rows = read_csv(f"{SPECTRUM}/brane-band-slope.csv")
c_rows = np.array([float(r["c_numeric"]) for r in rows])
for a_, c_ in zip(SLICES, c_numeric):
    say(f"slice a4,0 = {a_}: c numerical {c_:.12f}, "
        f"c e^(-a4,0) exact {c_exact * math.exp(-a_):.12f}")
check(np.max(np.abs(c_numeric / c_rows - 1.0)) < 1e-11
      and np.max(np.abs(c_numeric / (c_exact * np.exp(-np.array(SLICES))) - 1.0))
      < 1e-9 and record_check("free_brane_band_slope"),
      "the numerical slope equals c e^(-a4,0) at all five slices",
      record=f"{SPECTRUM}/brane-band-slope.csv and {RUST_REPORT}, check "
             "free_brane_band_slope")
c_by_L = {}
for L_ in (2.0, 4.0):  # the slope for two other cutoffs
    pair = levels(np.array([1e-4, 2e-4]), 1.0, "even", 0, L=L_, G=round(300 * L_))
    c_by_L[L_] = (4.0 * pair[0] / 1e-4 - pair[1] / 2e-4) / 3.0
c_by_L[3.0] = float(c_numeric[0])
exact_by_L = {L_: 2.0 / (1.0 + math.exp(-L_)) for L_ in (2.0, 3.0, 4.0)}
say("c for L = 2, 3, 4: " + ", ".join(f"{c_by_L[L_]:.6f}" for L_ in (2.0, 3.0, 4.0))
    + " (exact " + ", ".join(f"{exact_by_L[L_]:.6f}" for L_ in (2.0, 3.0, 4.0)) + ")")
check(all(abs(c_by_L[L_] - exact_by_L[L_]) < 1e-9 for L_ in (2.0, 4.0)),
      "the slope formula holds for L = 2 and L = 4 too")
```

Out [5]:

```
RESULT slope c for M = H = 1, L = 3, a4,0 = 0 = 1.9051482536449 m/H
PASS c = e^(-a4,0) (2M/(2M - H)) (1 - e^(-(2M-H)L))/(1 - e^(-2ML)) = 1.9051482536
     reproduces Revision/kohn_sham/ks-theory.json, checksNumeric
    braneBandSlope_M1_H1_L3_a0
PASS the integral of kappa a^2 over the numerical zero mode gives c
slice a4,0 = 0.0: c numerical 1.905148253642, c e^(-a4,0) exact 1.905148253645
slice a4,0 = 0.5: c numerical 1.155530827132, c e^(-a4,0) exact 1.155530827134
slice a4,0 = 1.0: c numerical 0.700864874898, c e^(-a4,0) exact 0.700864874900
slice a4,0 = 1.5: c numerical 0.425096034942, c e^(-a4,0) exact 0.425096034942
slice a4,0 = 2.0: c numerical 0.257833778514, c e^(-a4,0) exact 0.257833778515
PASS the numerical slope equals c e^(-a4,0) at all five slices
     reproduces Revision/kohn_sham/results/spectrum/brane-band-slope.csv and
    Revision/kohn_sham/reports/ks-rust-solver.json, check free_brane_band_slope
c for L = 2, 3, 4: 1.761594, 1.905148, 1.964028 (exact 1.761594, 1.905148, 1.964028)
PASS the slope formula holds for L = 2 and L = 4 too
```

Text cell:

```
The next cell draws the slope. Left: $c$ at the five slices (numerical points)
on the exact curve $c\,e^{-a_{4,0}}$, logarithmic vertical axis. Right: $c$ as a
function of the cutoff $L$ for $a_{4,0} = 0$, with the numerical values for
$L = 2, 3, 4$; for a long hidden interval it approaches $2M/(2M - H) = 2$.
```

In [6]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
a_grid = np.linspace(0.0, 2.0, 201)
left.semilogy(a_grid, c_exact * np.exp(-a_grid), color="C0",
              label="exact $c\\,e^{-a_{4,0}}$")
left.semilogy(SLICES, c_numeric, "o", color="C3", label="numerical (Richardson)")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("slope $c$ (units of $m/H$, logarithmic)")
left.set_title("the slope along the history")
left.legend(fontsize=8)
L_grid = np.linspace(0.5, 6.0, 221)
right.plot(L_grid, 2.0 / (1.0 + np.exp(-L_grid)), color="C0",
           label="exact $2/(1 + e^{-L})$")
right.plot(list(c_by_L), list(c_by_L.values()), "o", color="C3", label="numerical")
right.axhline(2.0, color="gray", linestyle=":", label="limit $2M/(2M - H) = 2$")
right.set_xlabel("cutoff $L$ (units of $1/H$)")
right.set_ylabel("slope $c$ at $a_{4,0} = 0$")
right.set_title("the slope against the cutoff ($M = H = 1$)")
right.legend(fontsize=8)
save_figure(fig, "band_slope",
            "The slope $c = d\\varepsilon/dk$ of the brane band at $k = 0$ "
            "($M = H = 1$). Left: against the slice $a_{4,0}$ from 0 to 2, "
            "logarithmic vertical axis; the numerical Richardson values (dots) lie "
            "on the exact line $c\\,e^{-a_{4,0}}$ with $c = 1.9051$ for $L = 3$: "
            "the band is redshifted along the history. Right: against the "
            "cutoff $L$ (units of $1/H$) at $a_{4,0} = 0$; exact curve "
            "$2/(1 + e^{-L})$ and numerical values for $L = 2, 3, 4$; for a long "
            "hidden interval the slope tends to $2M/(2M - H) = 2$.")
```

Out [6]:

```
Figure 14c.2 saved as Revision/textbook/figures/14c_2_band_slope.png
```

Text cell:

```
## 8. The deflating history: redshift and the exact rescaling identity

The slice enters the block equation only through $\kappa k = e^{-Hy}(k\,
e^{-a_{4,0}})$, so EXACTLY $\varepsilon(k, a_{4,0}) = \varepsilon(k\,e^{-a_{4,0}},
0)$ for every level. The next cell checks this as the Rust solver did (check
`free_rescaling_relation_and_band_monotone`): for $k = 0.25, 1, 2.5$, the five
slices and three sectors, the level at the slice and the level at slice 0 with
the rescaled momentum agree to $10^{-12}$. It then computes the brane band at
the five slices for the figure, and the gap of the free $N = 8$ state at every
slice: the eight zero modes are filled and the next level is the band at the
```

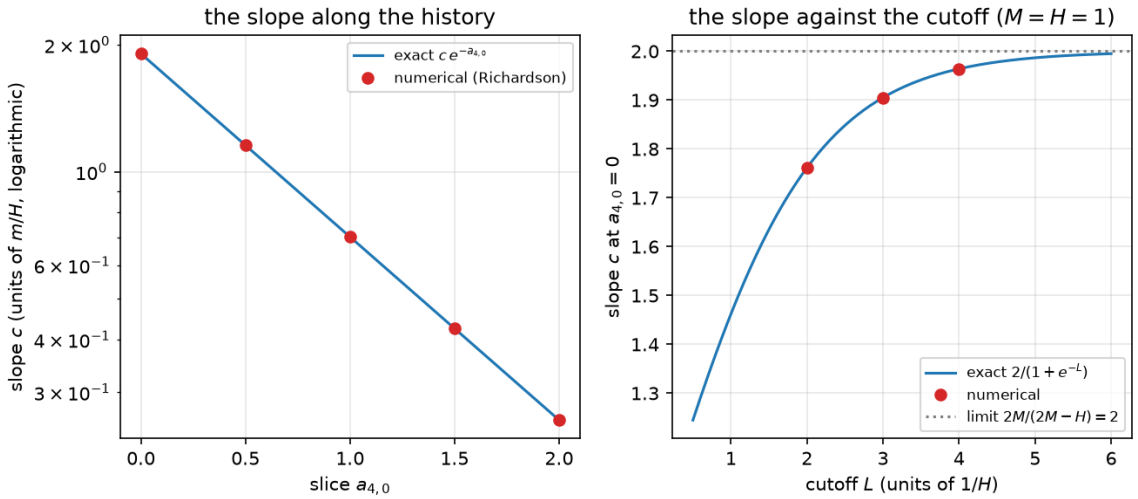

Figure 396: The slope $c = d\varepsilon/dk$ of the brane band at $k = 0$ ($M = H = 1$). Left: against the slice $a_{4,0}$ from 0 to 2, logarithmic vertical axis; the numerical Richardson values (dots) lie on the exact line $c\,e^{-a_{4,0}}$ with $c = 1.9051$ for $L = 3$: the band is redshifted along the history. Right: against the cutoff L (units of $1/H$) at $a_{4,0} = 0$; exact curve $2/(1+e^{-L})$ and numerical values for $L = 2, 3, 4$; for a long hidden interval the slope tends to $2M/(2M-H) = 2$. (Notebook 14c, figure 2.)

```
first lattice shell $k = \Delta k = 0.25$; this gap must equal the record
‘closed-shells.csv‘ (rows $N = 8$). Along the history the gap shrinks like
$e^{-a_{4,0}}$ for small momenta: $0.4307$, ..., $0.0642\,m$.
```

In [7]:

```
test_k = [0.25, 1.0, 2.5]
test_sectors = [(1.0, "even", 0), (1.0, "odd", 0), (-1.0, "even", 1)]
combos = [(a_, k_, s_) for a_ in SLICES for k_ in test_k for s_ in test_sectors]
at_slice = levels(np.array([k_ for a_, k_, s_ in combos]),
                  np.array([s_[0] for a_, k_, s_ in combos]),
                  np.array([s_[1] for a_, k_, s_ in combos]),
                  np.array([s_[2] for a_, k_, s_ in combos]),
                  a4=np.array([a_ for a_, k_, s_ in combos]))
at_zero = levels(np.array([k_ * math.exp(-a_) for a_, k_, s_ in combos]),
                 np.array([s_[0] for a_, k_, s_ in combos]),
                 np.array([s_[1] for a_, k_, s_ in combos]),
                 np.array([s_[2] for a_, k_, s_ in combos]))
check(np.max(np.abs(at_slice - at_zero)) < 1e-12
      and record_check("free_rescaling_relation_and_band_monotone"),
      "eps(k, a4,0) = eps(k e^(-a4,0), 0) for 45 levels at the five slices",
      record=f"{RUST_REPORT}, check free_rescaling_relation_and_band_monotone")
k_fig = np.linspace(0.0, 4.0, 41)
band_slices = levels(np.tile(k_fig, 5), 1.0, "even", 0,
                     a4=np.repeat(SLICES, len(k_fig))).reshape(5, len(k_fig))
gaps = levels(np.full(5, 0.25), 1.0, "even", 0, a4=np.array(SLICES))
shells_record = read_csv(f"{SPECTRUM}/closed-shells.csv")
gap_record = [float(r["gap"]) for r in shells_record if r["N_closed"] == "8"]
for a_, g_ in zip(SLICES, gaps):
```

```
    say(f"slice a4,0 = {a_}: gap of the free N = 8 state {g_:.10f} m")
check(len(gap_record) == 5 and np.max(np.abs(gaps - np.array(gap_record))) < 1e-11,
      "the N = 8 gaps at the five slices equal the record",
      record=f"{SPECTRUM}/closed-shells.csv, rows N_closed = 8")
```

Out [7]:

```
PASS eps(k, a4,0) = eps(k e^(-a4,0), 0) for 45 levels at the five slices
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_rescaling_relation_and_band_monotone
slice a4,0 = 0.0: gap of the free N = 8 state 0.4307336786 m
slice a4,0 = 0.5: gap of the free N = 8 state 0.2726448766 m
slice a4,0 = 1.0: gap of the free N = 8 state 0.1703492513 m
slice a4,0 = 1.5: gap of the free N = 8 state 0.1050152416 m
slice a4,0 = 2.0: gap of the free N = 8 state 0.0641594107 m
PASS the N = 8 gaps at the five slices equal the record
     reproduces Revision/kohn_sham/results/spectrum/closed-shells.csv, rows N_closed = 8
```

Text cell:

```
The next cell draws the redshift. Left: the brane band against $k$ at the five
slices; later slices lie lower. Right: the same curves against the redshifted
momentum $k\,e^{-a_{4,0}}$: they fall onto one curve, the slice-0 band, which is
the rescaling identity.
```

In [8]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), sharey=True)
for index, a_ in enumerate(SLICES):
    left.plot(k_fig, band_slices[index], label=f"$a_{{4,0}} = {a_}$")
    right.plot(k_fig * math.exp(-a_), band_slices[index], "o", markersize=3,
               label=f"$a_{{4,0}} = {a_}$")
right.plot(k_fig, band_slices[0], color="black", linewidth=0.8,
           label="slice 0 band")
left.set_xlabel("3-momentum $k$ (units of $H$)")
left.set_ylabel("brane band $\\varepsilon$ (units of $m$)")
left.set_title("the band at five slices")
left.legend(fontsize=8)
right.set_xlabel("redshifted momentum $k\\,e^{-a_{4,0}}$ (units of $H$)")
right.set_title("all slices on one curve")
right.legend(fontsize=7)
save_figure(fig, "band_redshift",
            "The brane band along the prescribed deflating history. Left: the "
            "band $\\varepsilon(k)$ (units of $m$) against the 3-momentum $k$ "
            "(units of $H$) at the five slices $a_{4,0} = 0$ to $2$; later slices "
            "lie lower. Right: the same levels against the redshifted momentum "
            "$k\\,e^{-a_{4,0}}$; all points fall on the band of the slice 0 "
            "(black line), the exact rescaling identity "
            "$\\varepsilon(k, a_{4,0}) = \\varepsilon(k e^{-a_{4,0}}, 0)$.")
```

Out [8]:

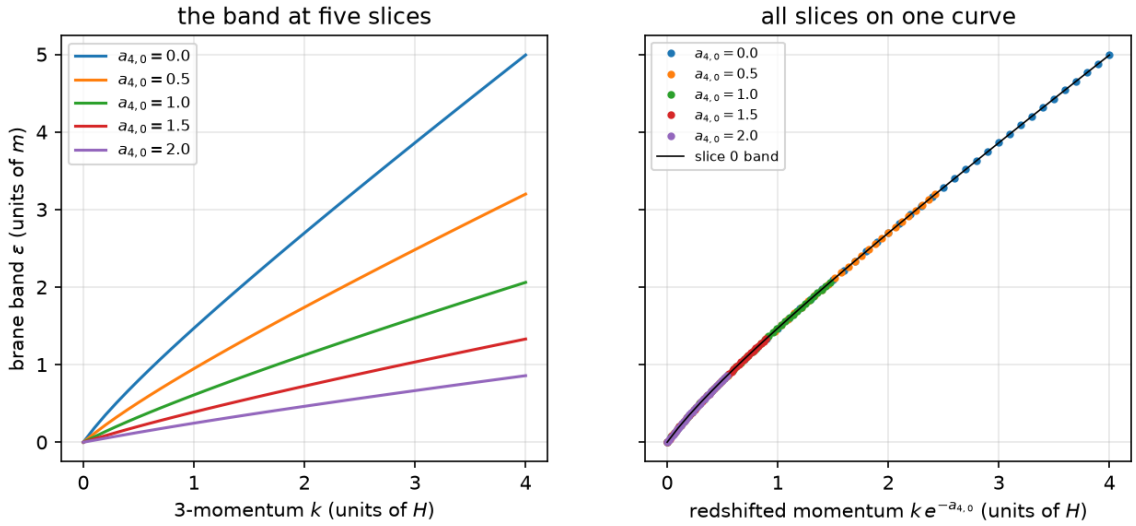

Figure 397: The brane band along the prescribed deflating history. Left: the band $\varepsilon(k)$ (units of m) against the 3-momentum k (units of H) at the five slices $a_{4,0} = 0$ to 2; later slices lie lower. Right: the same levels against the redshifted momentum $k\,e^{-a_{4,0}}$; all points fall on the band of the slice 0 (black line), the exact rescaling identity $\varepsilon(k, a_{4,0}) = \varepsilon(ke^{-a_{4,0}}, 0)$. (Notebook 14c, figure 3.)

```
Figure 14c.3 saved as Revision/textbook/figures/14c_3_band_redshift.png
```

Text cell:

```
## 9. The mirror symmetries of the two block types

Two exact relations of the block Hamiltonian (for $v = 0$): $h_{-1} = -h_{+1}$,
so the levels of $j = -1$ are the negatives of those of $j = +1$, with the
labels $l \to -l$ (even) and $l \to -l - 1$ (odd); and $\sigma_3h_j(k)\sigma_3 =
h_{-j}(-k)$, so the level of $(j = -1, -k)$ with label $l$ equals the level of
$(j = +1, k)$ with the same label. The next cell checks both for the shells
$n^2 = 0, 1, 2, 5, 9$ ($k = 0.25\sqrt{n^2}$) and the labels $-3$ to $3$ of both
parities (the Rust check `free_block_type_symmetries`), and computes the even
levels of both block types for $k$ from 0 to 2 for the figure.
```

In [9]:

```
sym_k = [0.25 * math.sqrt(n2) for n2 in (0, 1, 2, 5, 9)]
sym_l = list(range(-3, 4))
grid = [(k_, l_) for k_ in sym_k for l_ in sym_l]
K_s = np.array([k_ for k_, l_ in grid])
L_s = np.array([l_ for k_, l_ in grid])
plus_even = levels(K_s, 1.0, "even", L_s)
minus_even = levels(K_s, -1.0, "even", -L_s)
plus_odd = levels(K_s, 1.0, "odd", L_s)
minus_odd = levels(K_s, -1.0, "odd", -L_s - 1)
flipped_even = levels(-K_s, -1.0, "even", L_s)
```

```
flipped_odd = levels(-K_s, -1.0, "odd", L_s)
mirror = max(np.max(np.abs(plus_even + minus_even)),
             np.max(np.abs(plus_odd + minus_odd)))
flip = max(np.max(np.abs(flipped_even - plus_even)),
           np.max(np.abs(flipped_odd - plus_odd)))
check(mirror < 1e-12 and flip < 1e-12 and record_check("free_block_type_symmetries"),
      "spec h_(-1) = -spec h_(+1) and eps_(-1)(-k) = eps_(+1)(k), label by label",
      record=f"{RUST_REPORT}, check free_block_type_symmetries")
k_mirror = np.linspace(0.0, 2.0, 41)
fig_labels = list(range(-2, 3))
plus_curves = levels(np.tile(k_mirror, 5), 1.0, "even",
                     np.repeat(fig_labels, len(k_mirror))).reshape(5, -1)
minus_curves = levels(np.tile(k_mirror, 5), -1.0, "even",
                      np.repeat(fig_labels, len(k_mirror))).reshape(5, -1)
say("even levels at k = 2, labels -2 ... 2:")
say("  j = +1: " + ", ".join(f"{x:.4f}" for x in plus_curves[:, -1]))
say("  j = -1: " + ", ".join(f"{x:.4f}" for x in minus_curves[:, -1]))
```

Out [9]:

```
PASS spec h_(-1) = -spec h_(+1) and eps_(-1)(-k) = eps_(+1)(k), label by label
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_block_type_symmetries
even levels at k = 2, labels -2 ... 2:
  j = +1: -6.3742, -4.4814, 2.6976, 5.4109, 7.1695
  j = -1: -7.1695, -5.4109, -2.6976, 4.4814, 6.3742
```

Text cell:

```
The next cell draws the even levels of the two block types against $k$: the
picture of $j = -1$ is the picture of $j = +1$ turned upside down.
```

In [10]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for row, label in enumerate(fig_labels):
    left.plot(k_mirror, plus_curves[row], label=f"label {label}")
    right.plot(k_mirror, minus_curves[row], label=f"label {label}")
for ax, title in ((left, "block type $j = +1$"), (right, "block type $j = -1$")):
    ax.axhline(0.0, color="black", linewidth=0.6)
    ax.set_xlabel("3-momentum $k$ (units of $H$)")
    ax.set_title(title + ", even parity")
    ax.legend(fontsize=7, loc="upper left")
left.set_ylabel("level $\\varepsilon$ (units of $m$)")
save_figure(fig, "block_type_mirror",
            "The even-parity levels with labels $-2$ to $2$ against the "
            "3-momentum $k$ (units of $H$), slice 0, $m = 1$, $L = 3$; vertical "
            "axis the level in units of $m$. Left the blocks $j = +1$, right the "
            "blocks $j = -1$: each picture is the other one reflected in the line "
            "$\\varepsilon = 0$ (label $l$ goes to $-l$), the exact relation "
            "$h_{-1} = -h_{+1}$; the rising curve of the label 0 on the left is the "
            "brane band, the falling one on the right its sea partner.")
```

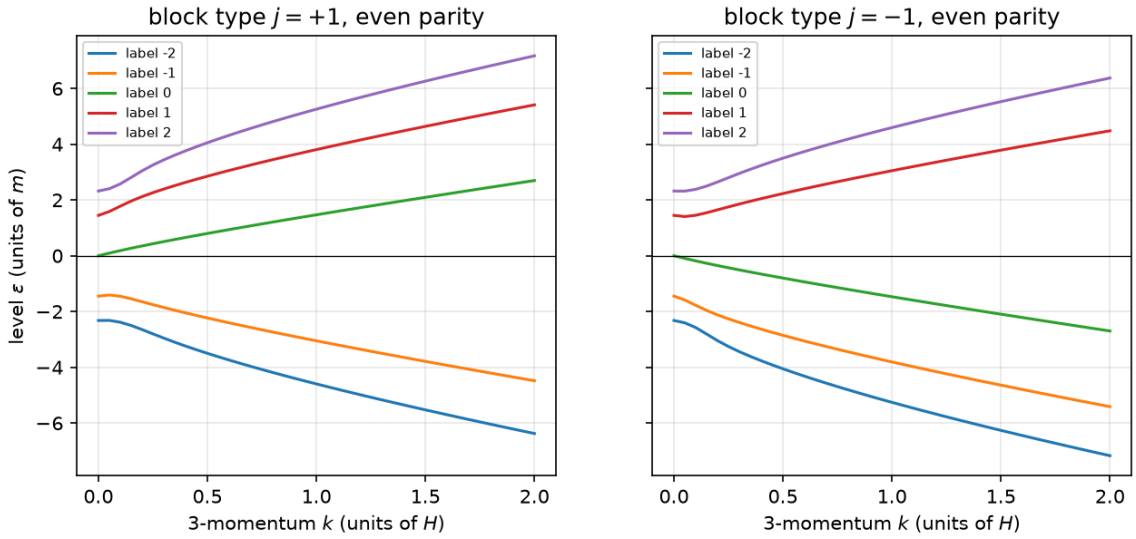

Figure 398: The even-parity levels with labels -2 to 2 against the 3-momentum k (units of H), slice 0, $m = 1$, $L = 3$; vertical axis the level in units of m. Left the blocks $j = +1$, right the blocks $j = -1$: each picture is the other one reflected in the line $\varepsilon = 0$ (label l goes to $-l$), the exact relation $h_{-1} = -h_{+1}$; the rising curve of the label 0 on the left is the brane band, the falling one on the right its sea partner. (Notebook 14c, figure 4.)

Out [10]:

```
Figure 14c.4 saved as Revision/textbook/figures/14c_4_block_type_mirror.png
```

Text cell:

```
## 10. The tip angle does not matter for the band (at the cutoff L = 3, slice 0)

For $k \ne 0$ an orbital is suppressed toward the tip like
$\exp(-k(\kappa(-L) - \kappa(y))/H)$ (the Revision record,
‘boundaryConditions.tip‘): the term $\kappa k\sigma_3$ grows toward the tip and
pushes the orbital away. So the level cannot depend much on the condition
imposed there. The next cell computes the brane band with the tip angles
$\theta = 0, 0.5, 1$ for $k = 0.25, 0.5, 1$ (the record ‘tip-angle.csv‘) and
checks that the shift from $\theta = 0$ is below the suppression factor
$\exp(-k(e^{HL} - 1)/H)$ (the Rust check ‘free_tip_angle_insensitivity‘); then
it computes the shifts for ten momenta from 0.1 to 1 for the figure.

Like the record, this section works at the slice $a_{4,0} = 0$ only (the default
‘a4=0.0‘ of ‘levels‘). At a later slice the lattice momentum $k$ acts like
$k\,e^{-a_{4,0}}$ at the slice 0 (the rescaling identity of section 8), and
there the suppression factor is no longer small: for $k = 0.25$ it is
$\exp(-0.25\,e^{-a_{4,0}}(e^{3} - 1))$, about $0.17$ at $a_{4,0} = 1$ and
$0.52$ at $a_{4,0} = 2$. Neither this notebook nor the Revision record computes
the tip angle at the later slices, so there the insensitivity is not
established.
```

```
This concerns the tip ANGLE at the fixed cutoff $L = 3$. The position $L$ of the
cutoff is a different question, which this notebook does not compute: the
Revision record measures it for $L = 3$ to $6$
(‘Revision/kohn_sham/tip_convergence/‘, report ‘tip-convergence.json‘, 7 of 7
checks PASS). The free results with $k \ne 0$ converge, but the recorded
$L = 3$ values are low by up to 1.4% in $E_{KS}$ and 12% in the $p_0$ integral at
$a_{4,0} = 2$ (the redshifted brane band reaches the tip); the nonzero (bulk)
$k = 0$ levels approach $\pm m$ only algebraically; and the interaction energy
of the brane zero modes depends strongly on $L$.
```

In [11]:

```
tip_rows = read_csv(f"{SPECTRUM}/tip-angle.csv")
tip_k = np.array([0.25, 0.5, 1.0])
tip_levels = {th: levels(tip_k, 1.0, "even", 0, tip=th) for th in (0.0, 0.5, 1.0)}
suppression = np.exp(-tip_k * (math.exp(3.0) - 1.0))  # H = 1, L = 3
worst_tip, below = 0.0, True
for r in tip_rows:
    i_k = list(tip_k).index(float(r["k"]))
    mine = tip_levels[float(r["theta_tip"])][i_k]
    worst_tip = max(worst_tip, abs(mine - float(r["eps"])))
    shift = mine - tip_levels[0.0][i_k]
    below &= abs(shift) <= max(suppression[i_k], 1e-12)
check(len(tip_rows) == 9 and worst_tip < 1e-11 and below
      and record_check("free_tip_angle_insensitivity"),
      "the band levels equal tip-angle.csv; the shifts are below the suppression",
      record=f"{SPECTRUM}/tip-angle.csv and {RUST_REPORT}, check "
             "free_tip_angle_insensitivity")
k_tip = np.linspace(0.1, 1.0, 10)
shifts = {th: np.abs(levels(k_tip, 1.0, "even", 0, tip=th)
                     - levels(k_tip, 1.0, "even", 0, tip=0.0)) for th in (0.5, 1.0)}
def shown(x):
    """A shift as text; below 1e-14 it is rounding noise and differs between
    computers."""
    return f"{x:.1e}" if x >= 1e-14 else "below 1e-14 (rounding)"


for k_, s5, s10 in zip(k_tip, shifts[0.5], shifts[1.0]):
    say(f"k = {k_:.1f}: shift for theta = 0.5: {shown(s5)}, for theta = 1: "
        f"{shown(s10)}")
```

Out [11]:

```
PASS the band levels equal tip-angle.csv; the shifts are below the suppression
     reproduces Revision/kohn_sham/results/spectrum/tip-angle.csv and
    Revision/kohn_sham/reports/ks-rust-solver.json, check free_tip_angle_insensitivity
k = 0.1: shift for theta = 0.5: 4.6e-04, for theta = 1: 1.2e-03
k = 0.2: shift for theta = 0.5: 3.8e-05, for theta = 1: 1.2e-04
k = 0.3: shift for theta = 0.5: 1.8e-06, for theta = 1: 5.9e-06
k = 0.4: shift for theta = 0.5: 7.1e-08, for theta = 1: 2.4e-07
k = 0.5: shift for theta = 0.5: 2.5e-09, for theta = 1: 8.6e-09
k = 0.6: shift for theta = 0.5: 8.1e-11, for theta = 1: 2.8e-10
```

```
k = 0.7: shift for theta = 0.5: 2.6e-12, for theta = 1: 9.0e-12
k = 0.8: shift for theta = 0.5: 7.7e-14, for theta = 1: 2.7e-13
k = 0.9: shift for theta = 0.5: below 1e-14 (rounding), for theta = 1: below 1e-14
    (rounding)
k = 1.0: shift for theta = 0.5: below 1e-14 (rounding), for theta = 1: below 1e-14
    (rounding)
```

Text cell:

```
The next cell draws the shifts against $k$ on a logarithmic axis, together with
the suppression factor. Shifts below about $10^{-15}$ are rounding noise of the
floating-point numbers (their exact values differ from computer to computer); a
shift below $10^{-16}$ (or exactly 0) is drawn at $10^{-16}$, so that it fits on
the logarithmic axis. The printed list above shows the size of the drop: each
step of $0.1$ in $k$ makes the shift 11 to 33 times smaller (the suppression
factor itself only about $e^{0.1(e^3 - 1)} = 6.7$ times), until it reaches the
rounding level at $k = 0.9$.
```

In [12]:

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
k_line = np.linspace(0.1, 1.0, 91)
ax.semilogy(k_line, np.exp(-k_line * (math.exp(3.0) - 1.0)), color="black",
            label="suppression $\\exp(-k(e^{HL} - 1)/H)$")
for th, marker in ((0.5, "o"), (1.0, "s")):
    ax.semilogy(k_tip, np.maximum(shifts[th], 1e-16), marker, markersize=5,
                label=f"$|\\varepsilon(\\theta = {th}) - \\varepsilon(\\theta = 0)|$")
ax.axhline(1e-15, color="gray", linestyle=":", linewidth=0.8)
ax.annotate("rounding level", (0.75, 2e-15), fontsize=8, color="gray")
ax.set_xlabel("3-momentum $k$ (units of $H$)")
ax.set_ylabel("shift of the band level (units of $m$, logarithmic)")
ax.set_title("the band does not feel the tip angle ($L = 3$, slice 0)")
ax.legend(fontsize=8)
save_figure(fig, "tip_insensitivity",
            "The change of the brane-band level when the tip condition "
            "$(1 - Q(\\theta))\\chi(-L) = 0$ is changed from $\\theta = 0$ to "
            "$\\theta = 0.5$ (circles) and $\\theta = 1$ (squares), against the "
            "3-momentum $k$ (units of $H$), for $m = 1$, $L = 3$ at the slice "
            "$a_{4,0} = 0$, logarithmic vertical axis in units of $m$; every "
            "shift lies below the "
            "suppression factor $\\exp(-k(e^{HL} - 1)/H)$ (line), falls by a "
            "factor of 11 to 33 for each step of $0.1$ in $k$, and from "
            "$k = 0.9$ on it is at the rounding level of the computer (dotted "
            "line): at the cutoff $L = 3$ and the slice 0 the band does not feel "
            "the tip angle (at the later slices this is not computed; the "
            "dependence on the cutoff $L$ itself is measured in the Revision "
            "record, Revision/kohn_sham/tip_convergence/).")
```

Out [12]:

```
Figure 14c.5 saved as Revision/textbook/figures/14c_5_tip_insensitivity.png
```

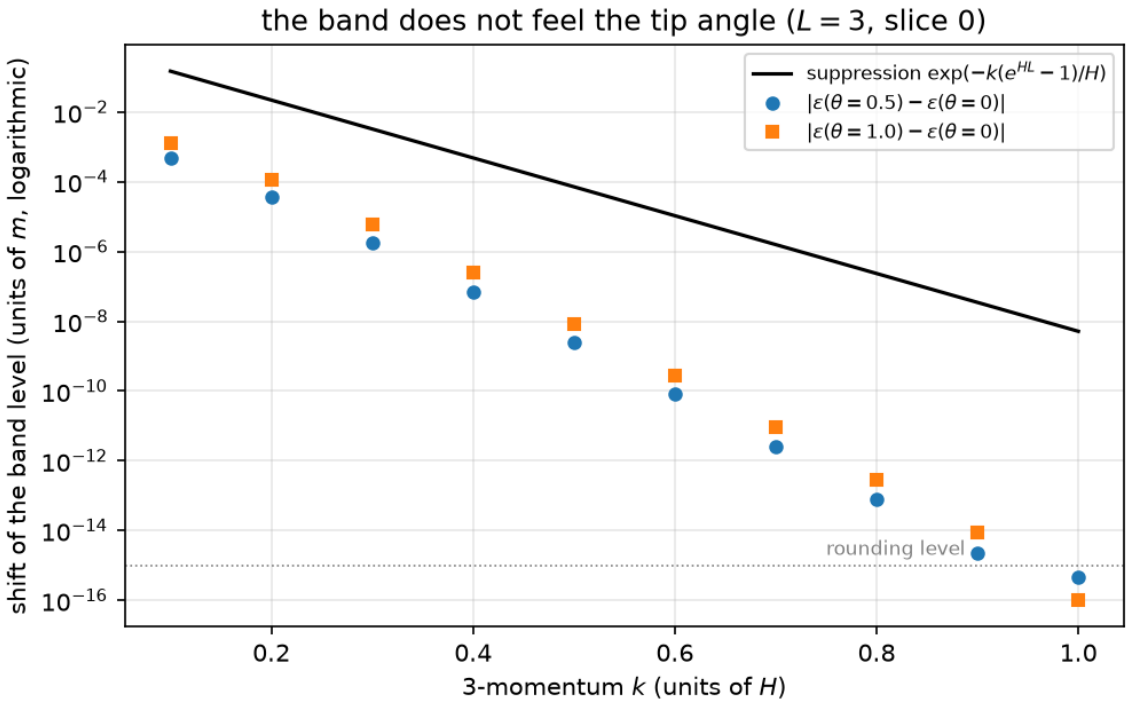

Figure 399: The change of the brane-band level when the tip condition $(1 - Q(\theta))\chi(-L) = 0$ is changed from $\theta = 0$ to $\theta = 0.5$ (circles) and $\theta = 1$ (squares), against the 3-momentum k (units of H), for $m = 1$, $L = 3$ at the slice $a_{4,0} = 0$, logarithmic vertical axis in units of m; every shift lies below the suppression factor $\exp(-k(e^{HL} - 1)/H)$ (line), falls by a factor of 11 to 33 for each step of 0.1 in k, and from $k = 0.9$ on it is at the rounding level of the computer (dotted line): at the cutoff $L = 3$ and the slice 0 the band does not feel the tip angle (at the later slices this is not computed; the dependence on the cutoff L itself is measured in the Revision record, Revision/kohn_sham/tip_convergence/). (Notebook 14c, figure 5.)

Text cell:

```
## 11. Particles, degeneracies and closed shells

**Shells.** The lattice vectors $(n_1, n_2, n_3)$ with the same
$n^2 = n_1^2 + n_2^2 + n_3^2$ have the same $|k| = 0.25\sqrt{n^2}$; $r_3(n^2)$
counts them (the first values, checked against the Rust unit test, are
$r_3 = 1, 6, 12, 8, 6, 24, 24, 12, 30$ for $n^2 = 0, 1, 2, 3, 4, 5, 6, 8, 9$; no
vector has $n^2 = 7$). Each level of a block type then holds $4r_3(n^2)$ states.

**Particles (CONVENTION).** In each sector (shell, $j$, parity) the particle
levels are those with $\varepsilon > 0$ in the free problem, plus the $k = 0$
zero modes; the negative ones form the sea. Because $\Phi$ grows with
$\varepsilon$, the particle levels are exactly the labels $l \ge l_{\min}$, where
$l_{\min}$ is the first label whose target lies above $\Phi(0)$. The Rust check
‘free_particle_branch_labels‘ verified this for all shells $n^2 \le 30$ and
found the level closest to zero (away from the zero modes) at $0.4307\,m$.

**Closed shells.** Filling the particle states from the bottom (the aufbau),
$N$ is a closed shell when it ends exactly at the end of a group of degenerate
levels (equal within $10^{-9}$). The Rust solver listed them up to
$\varepsilon = 1.6\,m$ at every slice (‘closed-shells.csv‘), and chose its
particle numbers by a rule: $N = 8$ (the zero modes), $N_{\rm large}$ = the
largest closed shell below the BULK EDGE (the lowest level not on the brane
band, the smaller of the odd label 0 and the even label 1 at $k = 0$), and
$N_{\rm mid}$ = the closed shell nearest to $N_{\rm large}/4$
(‘parameters.json‘). The next cell computes all of this for the slices 0 and
0.5. It takes about 20 seconds.
```

In [13]:

```
def shells(n2_max):
    """[(n2, r3)] for n2 = 0 ... n2_max with r3(n2) > 0 (counting lattice points)."""
    r = math.isqrt(n2_max) + 1
    count = [0] * (n2_max + 1)
    for x in range(-r, r + 1):
        for y_ in range(-r, r + 1):
            for z_ in range(-r, r + 1):
                s = x * x + y_ * y_ + z_ * z_
                if s <= n2_max:
                    count[s] += 1
    return [(n2, c) for n2, c in enumerate(count) if c > 0]


check(shells(9) == [(0, 1), (1, 6), (2, 12), (3, 8), (4, 6), (5, 24), (6, 24),
                    (8, 12), (9, 30)],
      "r3(n2) = 1, 6, 12, 8, 6, 24, 24, 12, 30 for n2 = 0, 1, 2, 3, 4, 5, 6, 8, 9")
SECTORS = [(1, "even"), (1, "odd"), (-1, "even"), (-1, "odd")]
SIGN = {1: "+1", -1: "-1"}


def lowest_particle_labels(pairs, a4):
```

```
    """l_min of each (n2, r3, j, parity) and Phi(0)."""
    k_ = np.array([0.25 * math.sqrt(n2) for n2, _, _, _ in pairs])
    j_ = np.array([float(j) for _, _, j, _ in pairs])
    par = np.array([p for _, _, _, p in pairs])
    phi0 = shoot(0.0, k_, j_, a4)
    offset = np.where(par == "even", 0.0, 0.5 * np.pi)
    l_min = np.floor((phi0 - offset) / np.pi).astype(int) + 1
    nearest = np.round((phi0 - offset) / np.pi).astype(int)
    zero_mode = (k_ == 0.0) & (np.abs(phi0 - offset - nearest * np.pi) < 1e-12)
    return np.where(zero_mode, nearest, l_min), phi0, offset, k_, j_, par


pairs30 = [(n2, r3, j, p) for n2, r3 in shells(30) for j, p in SECTORS]
l_min, phi0, offset, k30, j30, p30 = lowest_particle_labels(pairs30, 0.0)
e_first = levels(k30, j30, p30, l_min)
e_below = levels(k30, j30, p30, l_min - 1)
zero_k = k30 == 0.0
branch_ok = (np.all(e_below < 0.0)
             and np.all(e_first[zero_k & (p30 == "even")] == 0.0)
             and np.all(e_first[~(zero_k & (p30 == "even"))] > 0.0))
closest = float(min(np.min(e_first[~(zero_k & (p30 == "even"))]),
                    np.min(-e_below)))
report("level closest to zero away from the zero modes (n2 <= 30)",
       f"{closest:.4f}", "m")
check(branch_ok and f"{closest:.4f}" == "0.4307"
      and record_check("free_particle_branch_labels"),
      "particles are the labels l >= l_min in every sector (n2 <= 30)",
      record=f"{RUST_REPORT}, check free_particle_branch_labels")
```

Out [13]:

```
PASS r3(n2) = 1, 6, 12, 8, 6, 24, 24, 12, 30 for n2 = 0, 1, 2, 3, 4, 5, 6, 8, 9
RESULT level closest to zero away from the zero modes (n2 <= 30) = 0.4307 m
PASS particles are the labels l >= l_min in every sector (n2 <= 30)
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_particle_branch_labels
```

Text cell:

```
The next cell builds the closed shells at the slices 0 and 0.5 as the Rust
solver does: for every shell (in the order of $n^2$) and sector it finds the
particle levels $l_{\min}, l_{\min} + 1, \dots$ up to $1.6\,m$; it stops at the
first shell $n^2 > 0$ that has no level below $1.6\,m$; it sorts all levels by
energy (and by name), groups levels that agree within $10^{-9}$, and adds up the
degeneracies $4r_3(n^2)$. Each level is named ‘n2:j:parity:label‘. The table
must equal the record row by row (the numbers $N$, the names of the last group,
the energies to $10^{-11}$). Then it finds the bulk edge and the particle numbers
and compares them with ‘parameters.json‘.
```

In [14]:

```
def closed_shells(a4, n2_max, e_max=1.6):
    """[(N, eps_last, eps_next, names)] of the free aufbau at the slice a4."""
    pairs = [(n2, r3, j, p) for n2, r3 in shells(n2_max) for j, p in SECTORS]
    l_next, _, _, k_, j_, par = lowest_particle_labels(pairs, a4)
    found_levels = []  # (energy, degeneracy, name, n2)
    active = np.arange(len(pairs))
    while len(active) > 0:  # one more label in every sector that is still open
        e_ = levels(k_[active], j_[active], par[active], l_next[active], a4=a4)
        keep = e_ <= e_max
        for index, energy in zip(active[keep], e_[keep]):
            n2, r3, j, p = pairs[index]
            found_levels.append((float(energy), 4.0 * r3,
                                 f"{n2}:{SIGN[j]}:{p}:{l_next[index]}", n2))
        active = active[keep]
        l_next[active] += 1
    with_levels = {n2 for _, _, _, n2 in found_levels}
    stop = next(n2 for n2, _ in shells(n2_max) if n2 > 0 and n2 not in with_levels)
    found_levels = [lev for lev in found_levels if lev[3] < stop]
    found_levels.sort(key=lambda lev: (lev[0], lev[2]))
    table, total, i = [], 0.0, 0
    while i < len(found_levels):  # group the degenerate levels
        e0, group, names = found_levels[i][0], 0.0, []
        while i < len(found_levels) and found_levels[i][0] - e0 <= 1e-9:
            group += found_levels[i][1]
            names.append(found_levels[i][2])
            i += 1
        total += group
        following = found_levels[i][0] if i < len(found_levels) else float("nan")
        table.append((int(total), e0, following, ";".join(names)))
    return table, stop


agree = True
tables = {}
for a_, n2_max in ((0.0, 40), (0.5, 80)):
    table, stop = closed_shells(a_, n2_max)
    tables[a_] = table
    mine_rows = [r for r in shells_record if float(r["a4"]) == a_]
    same = len(table) == len(mine_rows) and all(
        int(r["N_closed"]) == t[0] and r["levels_of_the_last_group"] == t[3]
        and abs(float(r["eps_last_filled"]) - t[1]) < 1e-11
        for r, t in zip(mine_rows, table))
    agree &= same and stop < n2_max
    say(f"slice {a_}: {len(table)} closed shells up to 1.6 m, the first shell "
        f"without a level below 1.6 m is n2 = {stop}; equal to the record: {same}")
check(agree, "the closed shells of the slices 0 and 0.5 equal the record row by row",
      record=f"{SPECTRUM}/closed-shells.csv, rows a4 = 0 and 0.5")
say("slice 0: N = " + ", ".join(str(t[0]) for t in tables[0.0]))
bulk_edge = float(min(levels(0.0, 1.0, "odd", 0), levels(0.0, 1.0, "even", 1)))
below_edge = [t for t in tables[0.0] if t[1] < bulk_edge and t[2] - t[1] > 1e-6]
N_large = below_edge[-1][0]
N_mid = min((t[0] for t in below_edge if t[0] >= 8),
            key=lambda n: abs(n - N_large / 4))
parameters = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
```

```
                        .read_text(encoding="utf-8"))
numbers = parameters["particleNumbers"]
report("bulk edge", f"{bulk_edge:.12f}", "m")
report("particle numbers N", f"8, {N_mid}, {N_large}")
check(abs(bulk_edge - numbers["bulkEdge"]) < 1e-12 and N_large == 688
      and N_mid == 136 and numbers["values"] == [8.0, 136.0, 688.0],
      "the bulk edge 1.2922928281 and the particle numbers 8, 136, 688",
      record="Revision/kohn_sham/results/parameters.json, particleNumbers")
```

Out [14]:

```
slice 0.0: 22 closed shells up to 1.6 m, the first shell without a level below 1.6 m is
    n2 = 20; equal to the record: True
slice 0.5: 55 closed shells up to 1.6 m, the first shell without a level below 1.6 m is
    n2 = 53; equal to the record: True
PASS the closed shells of the slices 0 and 0.5 equal the record row by row
     reproduces Revision/kohn_sham/results/spectrum/closed-shells.csv, rows a4 = 0 and
    0.5
slice 0: N = 8, 32, 80, 112, 136, 232, 328, 376, 496, 592, 688, 696, 728, 824, 1016,
    1040, 1048, 1072, 1264, 1408, 1456, 1552
RESULT bulk edge = 1.292292828069 m
RESULT particle numbers N = 8, 136, 688
PASS the bulk edge 1.2922928281 and the particle numbers 8, 136, 688
     reproduces Revision/kohn_sham/results/parameters.json, particleNumbers
```

Text cell:

```
The next cell draws the aufbau as a staircase: the number $N$ of filled
particle states against the energy of the last filled level, at the slices 0 and
0.5, with the bulk edge and the three particle numbers of the Revision runs
marked. At the later slice the redshifted band holds many more states below the
same energy.
```

In [15]:

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
for a_, style in ((0.0, "-"), (0.5, "--")):
    energies = [t[1] for t in tables[a_]]
    counts = [t[0] for t in tables[a_]]
    ax.step(energies, counts, style, where="post",
            label=f"slice $a_{{4,0}} = {a_}$")
ax.axvline(bulk_edge, color="gray", linestyle="-.", linewidth=0.8)
ax.annotate("bulk edge", (bulk_edge + 0.015, 15), fontsize=8, color="gray")
last_filled = {t[0]: t[1] for t in tables[0.0]}  # N -> its last filled level
for n_ in (8, N_mid, N_large):
    t_ = last_filled[n_]
    ax.plot([t_], [n_], "o", color="C3")
    ax.annotate(f"N = {n_}", (t_ + 0.02, n_ * 0.75), fontsize=8, color="C3")
ax.set_yscale("log")
ax.set_xlabel("energy of the last filled level (units of $m$)")
ax.set_ylabel("closed-shell particle number $N$ (logarithmic)")
```

```
ax.set_title("the free aufbau: closed shells up to $1.6\\,m$")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "closed_shells",
            "The closed shells of the free aufbau: the particle number $N$ "
            "(logarithmic axis) against the energy of the last filled level "
            "(units of $m$), at the slices $a_{4,0} = 0$ (solid) and $0.5$ "
            "(dashed), for $m = 1$, $L = 3$, $\\Delta k = 0.25$. Each step is one "
            "group of degenerate levels; the red dots mark the particle numbers "
            "$N = 8$ (the zero modes), $136$ and $688$ of the Revision runs; $688$ "
            "is the last closed shell below the bulk edge $1.2923\\,m$ "
            "(dash-dotted line); at the later slice the redshifted brane band "
            "holds many more particles below the same energy.")
```

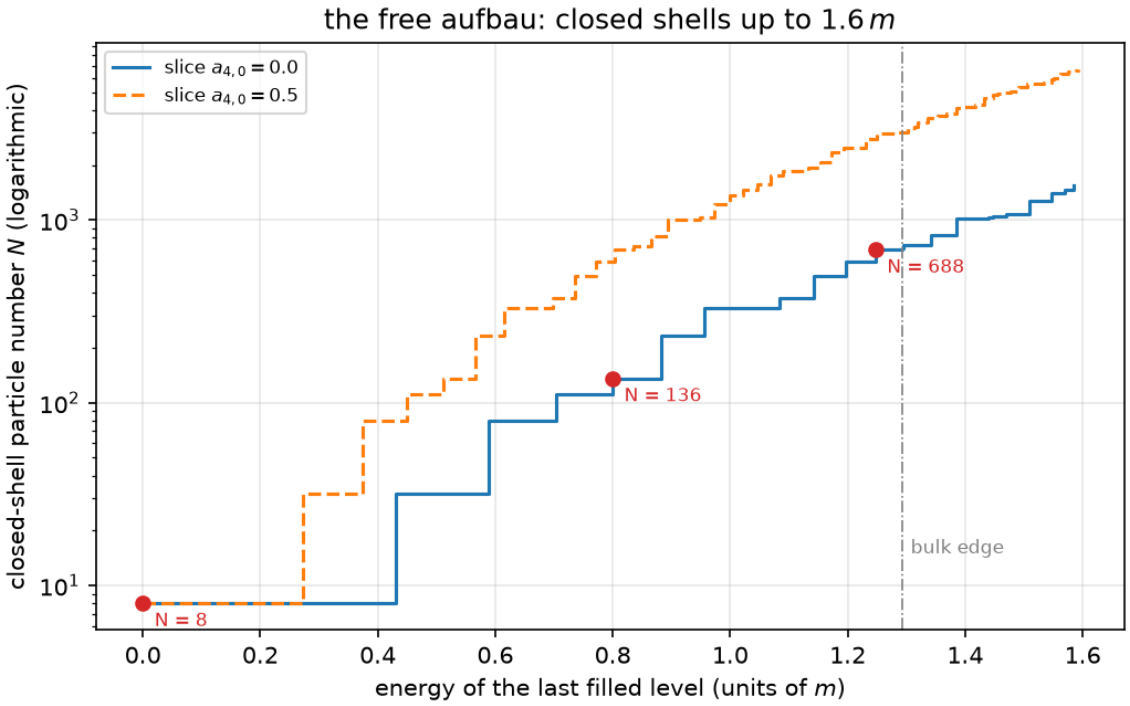

Figure 400: The closed shells of the free aufbau: the particle number N (logarithmic axis) against the energy of the last filled level (units of m), at the slices $a_{4,0} = 0$ (solid) and 0.5 (dashed), for $m = 1$, $L = 3$, $\Delta k = 0.25$. Each step is one group of degenerate levels; the red dots mark the particle numbers $N = 8$ (the zero modes), 136 and 688 of the Revision runs; 688 is the last closed shell below the bulk edge $1.2923\,m$ (dash-dotted line); at the later slice the redshifted brane band holds many more particles below the same energy. (Notebook 14c, figure 6.)

Out [15]:

```
Figure 14c.6 saved as Revision/textbook/figures/14c_6_closed_shells.png
```

Text cell:

```
## 12. The last check
```

```
The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [16]:

```
figure_names = ["14c_1_band_structure.png", "14c_2_band_slope.png",
                "14c_3_band_redshift.png", "14c_4_block_type_mirror.png",
                "14c_5_tip_insensitivity.png", "14c_6_closed_shells.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all six figure files of this notebook exist")
all_checks_passed()
```

Out [16]:

```
PASS all six figure files of this notebook exist
ALL 15 CHECKS PASSED (notebook 14c)
```

Text cell:

```
## 13. What this notebook showed

- The eight zero modes become the brane band when the 3-momentum is switched
  on; its slope at $k = 0$ is $c = e^{-a_{4,0}}\frac{2M}{2M - H}\frac{1 -
  e^{-(2M-H)L}}{1 - e^{-2ML}}$, $1.90514825360$ for $M = H = 1$, $L = 3$ (PROVED
  by the Hellmann-Feynman rule; COMPUTED numerically at all five slices; the
  values reproduce the Rust solver's records).
- Along the PRESCRIBED deflating history every level obeys the exact rescaling
  identity $\varepsilon(k, a_{4,0}) = \varepsilon(k e^{-a_{4,0}}, 0)$: the band
  is redshifted, and the gap of the free $N = 8$ state falls from $0.4307\,m$ at
  $a_{4,0} = 0$ to $0.0642\,m$ at $a_{4,0} = 2$.
- The two block types have mirrored spectra, $\mathrm{spec}\,h_{-1} =
  -\mathrm{spec}\,h_{+1}$, and $\sigma_3$ maps $(j, k)$ to $(-j, -k)$.
- At the cutoff $L = 3$ and the slice 0 the band lives at the brane: changing
  the tip angle shifts it by less than $\exp(-k(e^{HL} - 1)/H)$. At a later
  slice the factor is $\exp(-k\,e^{-a_{4,0}}(e^{HL} - 1)/H)$, which is not small
  for the lowest lattice momenta, and the tip angle is not computed there (by
  this notebook or by the record). Moving the cutoff $L$ itself
  is another matter (not computed here): the Revision record
  'Revision/kohn_sham/tip_convergence/' finds the $L = 3$ values low by up to
  1.4% in $E_{KS}$ at $a_{4,0} = 2$, where the redshifted band reaches the tip.
- With the CONVENTION that the positive free levels and the zero modes are
  particle levels (its justification is OPEN), the free aufbau has the closed
  shells $8, 32, 80, 112, 136, \dots, 688$ below the bulk edge $1.2923\,m$; the
  Revision runs use $N = 8, 136, 688$ (reproduced from the rule).
- ASSUMED: the Z2 mirror brane. CHOSEN: the regular tip at $L = 3$.
```

14.25 Line-by-line walk-through of Notebook 14c

The notebook has 16 code cells, In [1] to In [16]. Docstrings and the long caption texts are again left out of the quotations; the captions are printed under the figures in Section 14.24.

In [1], the set-up cell. The code of In [1] of Notebook 14a (Section 14.11), line for line, except

```
NOTEBOOK_ID = "14c"  # this notebook: chapter 14, example c
```

which names this notebook. Its comment lines are the run instructions of Section 14.23.

In [2], the tools.

```
import math  # exp, cos, sin of single numbers

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra with symbols

RUST_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
SPECTRUM = "Revision/kohn_sham/results/spectrum"  # the folder of the records
```

The three modules of In [2] of Notebook 14b, the name of the Rust solver's report, and the folder of its spectrum records.

```
def record_check(name, report=RUST_REPORT):
    data = json.loads(repository_file(report).read_text(encoding="utf-8"))
    return [c["verdict"] for c in data["checks"] if c["name"] == name] == ["PASS"]
```

A simpler `record_check`: true when the report (by default the Rust report) holds exactly one check of this name and its verdict is PASS (the list of all verdicts of that name must be the one-element list `["PASS"]`).

```
def read_csv(relative):
    lines = repository_file(relative).read_text(encoding="utf-8").splitlines()
    header = lines[0].split(",")
    return [dict(zip(header, line.split(","))) for line in lines[1:]]
```

A small CSV reader: the rows of a record as a list of dictionaries from column name to text, as in Notebooks 14a and 14b.

```
def shoot(eps, k=0.0, j=1.0, a4=0.0, M=1.0, L=3.0, G=900, H=1.0, tip=0.0,
          keep=False):
    eps, k, j, a4 = np.broadcast_arrays(*(np.asarray(x, dtype=float)
                                          for x in (eps, k, j, a4)))
    nf = 2 * G + 1  # nodes and step midpoints
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    e_minus_Hy = np.exp(-H * y_fine)
    kk = k * np.exp(-a4)  # kappa k = kk e^(-Hy)
    h = L / G
    a_ = np.full(eps.shape, math.cos(0.5 * tip))
    b_ = np.full(eps.shape, math.sin(0.5 * tip))
    theta = np.full(eps.shape, 0.5 * tip)
    raw = np.arctan2(b_, a_)
```

```
    je = j * eps
    path = [(theta, a_, b_)]
    for i in range(G):
        K0, K1, K2 = (kk * e_minus_Hy[2 * i], kk * e_minus_Hy[2 * i + 1],
                      kk * e_minus_Hy[2 * i + 2])
        p1a, p1b = M * a_ - (K0 + je) * b_, (je - K0) * a_ - M * b_
        a2, b2 = a_ + 0.5 * h * p1a, b_ + 0.5 * h * p1b
        p2a, p2b = M * a2 - (K1 + je) * b2, (je - K1) * a2 - M * b2
        a3, b3 = a_ + 0.5 * h * p2a, b_ + 0.5 * h * p2b
        p3a, p3b = M * a3 - (K1 + je) * b3, (je - K1) * a3 - M * b3
        a4_, b4_ = a_ + h * p3a, b_ + h * p3b
        p4a, p4b = M * a4_ - (K2 + je) * b4_, (je - K2) * a4_ - M * b4_
        a_ = a_ + h / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)
        b_ = b_ + h / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
        new = np.arctan2(b_, a_)
        step = new - raw  # brought into (-pi, pi]
        step = np.where(step > np.pi, step - 2 * np.pi,
                        np.where(step <= -np.pi, step + 2 * np.pi, step))
        theta = theta + step
        raw = new
        if keep:
            path.append((theta, a_, b_))
    if keep:
        return j * theta, [np.array(q) for q in zip(*path)]
    return j * theta
```

The function `shoot` of Notebook 14b (Section 14.18 explains every line), with one change: the slice `a4` is now the fourth argument and may also be an array, so that levels at different slices are computed in one call. The first two lines therefore bring four inputs to a common shape; `*(...)` hands the four arrays made by the generator over as four separate arguments.

```
def target(parity, label):
    offset = np.where(np.asarray(parity) == "even", 0.0, 0.5 * np.pi)
    return offset + np.asarray(label, dtype=float) * np.pi


def levels(k, j, parity, label, a4=0.0, iterations=72, **options):
    k, j, parity, label, a4 = np.broadcast_arrays(
        np.asarray(k, dtype=float), np.asarray(j, dtype=float), np.asarray(parity),
        np.asarray(label), np.asarray(a4, dtype=float))
    goal = target(parity, label)
    lo, hi = np.full(k.shape, -20.0), np.full(k.shape, 20.0)
    for _ in range(iterations):
        mid = 0.5 * (lo + hi)
        g = shoot(mid, k, j, a4, **options) - goal
        lo = np.where(g <= 0.0, mid, lo)
        hi = np.where(g >= 0.0, mid, hi)
    return 0.5 * (lo + hi)
```

`target` and `levels` as in Notebook 14b, with the slice `a4` as an extra input that may differ from level to level.

```
def orbital(eps, k=0.0, j=1.0, a4=0.0, M=1.0, L=3.0, G=900, H=1.0):
    _, (_, a_n, b_n) = shoot(eps, k, j, a4, M=M, L=L, G=G, H=H, keep=True)
    nf, h = 2 * G + 1, L / G
    y_fine = -L * ((nf - 1 - np.arange(nf)) / (nf - 1))
    K_fine = k * math.exp(-a4) * np.exp(-H * y_fine)
    a_f, b_f = np.zeros(nf), np.zeros(nf)
    a_f[0::2], b_f[0::2] = a_n, b_n
    da = M * a_f - (K_fine + j * eps) * b_f
    db = (j * eps - K_fine) * a_f - M * b_f
    for f in range(1, nf, 2):  # cubic Hermite values at the midpoints
        a_f[f] = 0.5 * (a_f[f - 1] + a_f[f + 1]) + h / 8.0 * (da[f - 1] - da[f + 1])
        b_f[f] = 0.5 * (b_f[f - 1] + b_f[f + 1]) + h / 8.0 * (db[f - 1] - db[f + 1])
    weights = np.full(nf, 2.0)  # Simpson weights 1, 4, 2, ..., 4, 1 times (h/2)/3
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    weights *= 0.5 * h / 3.0
    norm = math.sqrt(float(np.sum(weights * (a_f ** 2 + b_f ** 2))))
    return y_fine, a_f / norm, b_f / norm, weights
```

The normalised orbital of one level, exactly as `orbital` of Notebook 14b (In [12] there): the node values from `shoot`, cubic Hermite values at the midpoints, Simpson's weights and the normalisation. Here the slice is a single number, so `math.exp` suffices.

```
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]  # the slices of the Revision solver
say("defined: shoot, levels, orbital, read_csv, record_check")
```

The five slices of the history and the only output of the cell.

In [3], the band and three other levels.

```
k_list, kk = [], 0.0
while kk <= 4.0 + 1e-12:  # 0, 0.05, 0.10, ... as the Rust program makes them
    k_list.append(kk)
    kk += 0.05
k_band = np.array(k_list)
```

The momenta $0, 0.05, 0.10, \ldots, 4$, made by adding 0.05 again and again, as the Rust program does. A `while` loop repeats as long as its condition holds. The decimal number 0.05 is not exactly representable in binary floating point, so the sums differ in the last digits from $0.05\,n$; making them the same way as the Rust program makes the last digits agree with the record (and the small 10^{-12} in the condition makes sure that the last value, about 4, is included).

```
sectors = [(1.0, "even", 0), (1.0, "even", 1), (1.0, "odd", 0), (-1.0, "even", 1),
           (-1.0, "even", 0)]  # (j, parity, label); the last one for the figure
K_all = np.concatenate([k_band] * len(sectors))
J_all = np.concatenate([[j_] * len(k_band) for j_, _, _ in sectors])
P_all = np.concatenate([[p_] * len(k_band) for _, p_, _ in sectors])
L_all = np.concatenate([[l_] * len(k_band) for _, _, l_ in sectors])
found = levels(K_all, J_all, P_all, L_all).reshape(len(sectors), len(k_band))
band, bulk_even, odd0, minus_even1, minus_band = found
```

Five sectors (block type, parity, label): the brane band, the even label 1 and the odd label 0 of $j = +1$, the even label 1 of $j = -1$ (the four levels of the record), and the even label 0 of $j = -1$ for the figure. `np.concatenate` joins lists end to end, so the four arrays hold all $5 \times 81 = 405$ combinations; one call of `levels` finds all of them, and `reshape` arranges the result as 5 rows of 81. The last line unpacks the five rows into five names.

```
rows = read_csv(f"{SPECTRUM}/brane-band.csv")
record = np.array([[float(r[c]) for r in rows] for c in
                   ("eps_band_a0", "eps_band_bulk_even_l1", "eps_odd_l0",
                    "eps_jm1_even_l1")])
k_record = np.array([float(r["k"]) for r in rows])
say(f"k = 0.25: brane band {band[5]:.10f}, odd level {odd0[5]:.10f} (units of m)")
```

The record's four columns as an array of 4 rows of 81 numbers, its momenta, and a printed sample: position 5 of the list is $k = 0.25$.

```
check(len(rows) == 81 and np.max(np.abs(k_record - k_band)) < 1e-15
      and np.max(np.abs(found[:4] - record)) < 1e-11,
      "the 324 levels equal the record brane-band.csv",
      record=f"{SPECTRUM}/brane-band.csv")
check(bool(np.all(np.diff(band) > 0.0)) and np.max(np.abs(minus_band + band)) < 1e-12,
      "the band rises strictly with k; the j = -1 band is its mirror image -eps")
```

The first check: 81 rows, the same momenta, and the first four rows of levels (`found[:4]`, $4 \times 81 = 324$ levels) equal to the record to 10^{-11}. The second: the band rises strictly, and the band of $j = -1$ is exactly its negative (Section 14.19). Out [3] prints the band at $k = 0.25$, 0.4307336786, and the odd level 1.9195133815, and two PASS lines.

In [4], figure 1.

```
c_theory = 2.0 / (1.0 + math.exp(-3.0))  # the slope for M = H = 1, L = 3, a4,0 = 0
fig, ax = plt.subplots(figsize=(9.5, 4.8))
ax.plot(k_band, band, color="C0", linewidth=2, label="brane band ($j = +1$, even, 0)")
ax.plot(k_band, bulk_even, color="C1", label="$j = +1$, even, label 1")
ax.plot(k_band, odd0, color="C2", label="$j = +1$, odd, label 0")
ax.plot(k_band, minus_even1, color="C4", label="$j = -1$, even, label 1")
ax.plot(k_band, minus_band, "--", color="C0",
        label="$j = -1$, even, label 0 (sea)")
```

The slope $c = 2/(1 + e^{-3})$ of Section 14.19 and the five level curves; the band is drawn thick.

```
ax.plot(k_band[:13], c_theory * k_band[:13], ":", color="black",
        label="first order: $c\\,k$, $c = 1.9051$")
ax.axhline(odd0[0], color="gray", linewidth=0.8, linestyle="-.")
ax.annotate("bulk edge 1.2923", (2.6, odd0[0] - 0.55), fontsize=8,
            color="gray")  # just below the dash-dotted line
```

The straight line ck for the first 13 momenta ($k \leq 0.6$), and a dash-dotted horizontal line at the bulk edge, the odd level at $k = 0$ (`odd0[0]`), with its name written just below it.

```
ax.set_xlabel("3-momentum $k$ (units of $H$), slice $a_{4,0} = 0$")
ax.set_ylabel("level $\\varepsilon$ (units of $m$)")
ax.set_title("the free levels at nonzero 3-momentum ($m = 1$, $L = 3$)")
ax.legend(fontsize=7, loc="center left", bbox_to_anchor=(1.01, 0.5))
save_figure(fig, "band_structure",
            "The free Kohn-Sham levels against the 3-momentum $k$ (units of "
            ...)
```

Labels, title, and a legend placed outside the panel on the right (`bbox_to_anchor=(1.01, 0.5)` anchors its left centre just beyond the right edge). **What figure 14c.1 shows.** The brane band starts at 0, follows the dotted line ck for small k, bends below it and then rises almost straight to about $5\,m$ at $k = 4$. Its mirror image, the dashed band of $j = -1$, falls to about $-5\,m$: it belongs to the sea. The three other levels start at or above the bulk edge $1.2923\,m$ (the even level 1 of $j = -1$ dips slightly below its start before rising) and rise roughly parallel to the band. Nothing else enters the region between the band and the bulk edge.

In [5], the slope.

```
Ms, Hs, Ls = sp.symbols("M H L", positive=True)
ys, a4s = sp.symbols("y a", real=True)
ratio = (sp.integrate(sp.exp(-Hs * ys - a4s) * sp.exp(2 * Ms * ys), (ys, -Ls, 0))
         / sp.integrate(sp.exp(2 * Ms * ys), (ys, -Ls, 0)))  # step 3
formula = (sp.exp(-a4s) * 2 * Ms / (2 * Ms - Hs) * (1 - sp.exp(-(2 * Ms - Hs) * Ls))
           / (1 - sp.exp(-2 * Ms * Ls)))  # step 4
```

Symbols, the ratio of the two integrals of Section 14.19 computed by `sp.integrate(expression, (variable, lower, upper))`, and the closed formula.

```
same = sp.simplify((ratio - formula).subs({Ms: 1, Hs: 1, Ls: 3})) == 0 and \
    sp.simplify((ratio - formula).subs({Ms: 2, Hs: 1, Ls: 3})) == 0
c_exact = float(formula.subs({Ms: 1, Hs: 1, Ls: 3, a4s: 0}))
```

The ratio and the formula agree exactly for $(M, H, L) = (1, 1, 3)$ and $(2, 1, 3)$, for every slice (the symbol a stays free); `subs` with a dictionary replaces several symbols at once. `c_exact` is the number c for $M = H = 1$, $L = 3$, slice 0.

```
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
c_record = float(theory["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
report("slope c for M = H = 1, L = 3, a4,0 = 0", f"{c_exact:.13f}", "m/H")
check(same and abs(c_exact - c_record) < 1e-15 and abs(c_exact - c_theory) < 1e-15
      and record_check("brane_band_slope",
                       "Revision/kohn_sham/reports/ks-theory-python.json"),
      "c = e^(-a4,0) (2M/(2M - H)) (1 - e^(-(2M-H)L))/(1 - e^(-2ML)) = 1.9051482536",
      record="Revision/kohn_sham/ks-theory.json, checksNumeric "
             "braneBandSlope_M1_H1_L3_a0")
```

The value of the record, the RESULT line 1.9051482536449, and the check: the formula, the record's number, the short form $2/(1 + e^{-3})$ of In [4] and the verdict of the sympy report all agree.

```
y_f, a_zero, b_zero, simpson = orbital(0.0)  # the zero mode at k = 0
c_integral = float(np.sum(simpson * np.exp(-y_f) * a_zero ** 2))  # step 3
check(abs(c_integral - c_exact) < 1e-9,
      "the integral of kappa a^2 over the numerical zero mode gives c")
```

The numerical zero mode, and the Hellmann-Feynman integral $\int \kappa\, a^2 dy$ (with $\kappa = e^{-y}$ at the slice 0) by Simpson's rule: it equals c to 10^{-9}.

```
tiny = np.array([1e-4, 2e-4] * 5)  # two small momenta at each slice
slice_of = np.repeat(SLICES, 2)
small = levels(tiny, 1.0, "even", 0, a4=slice_of).reshape(5, 2)
c_numeric = (4.0 * small[:, 0] / 1e-4 - small[:, 1] / 2e-4) / 3.0
```

The two small momenta 10^{-4} and 2×10^{-4}, five times; `np.repeat(SLICES, 2)` repeats every slice twice (0, 0, 0.5, 0.5, ...), so that each slice gets both momenta. The ten band levels in one call, arranged as 5 rows of 2, and the Richardson combination $(4R(k_1) - R(k_2))/3$ of Section 14.19 for each slice.

```
rows = read_csv(f"{SPECTRUM}/brane-band-slope.csv")
c_rows = np.array([float(r["c_numeric"]) for r in rows])
for a_, c_ in zip(SLICES, c_numeric):
    say(f"slice a4,0 = {a_}: c numerical {c_:.12f}, "
        f"c e^(-a4,0) exact {c_exact * math.exp(-a_):.12f}")
```

The Rust solver's numerical slopes, and a printed comparison of the notebook's numerical slopes with the exact $c\, e^{-a_{4,0}}$ at each slice (Out [5]).

```
check(np.max(np.abs(c_numeric / c_rows - 1.0)) < 1e-11
      and np.max(np.abs(c_numeric / (c_exact * np.exp(-np.array(SLICES))) - 1.0))
      < 1e-9 and record_check("free_brane_band_slope"),
      "the numerical slope equals c e^(-a4,0) at all five slices",
      record=f"{SPECTRUM}/brane-band-slope.csv and {RUST_REPORT}, check "
             "free_brane_band_slope")
```

The notebook's slopes equal the Rust slopes to 10^{-11} and the exact ones to 10^{-9} (relative), and the Rust check passed.

```
c_by_L = {}
for L_ in (2.0, 4.0):  # the slope for two other cutoffs
    pair = levels(np.array([1e-4, 2e-4]), 1.0, "even", 0, L=L_, G=round(300 * L_))
    c_by_L[L_] = (4.0 * pair[0] / 1e-4 - pair[1] / 2e-4) / 3.0
c_by_L[3.0] = float(c_numeric[0])
exact_by_L = {L_: 2.0 / (1.0 + math.exp(-L_)) for L_ in (2.0, 3.0, 4.0)}
```

The slope for the cutoffs $L = 2$ and $L = 4$ by the same Richardson combination (with $G = 300L$ steps, the same step $h = 1/300$), the slope for $L = 3$ from the slice 0 above, and the exact $2/(1+e^{-L})$ for the three cutoffs, as a dictionary comprehension.

```
say("c for L = 2, 3, 4: " + ", ".join(f"{c_by_L[L_]:.6f}" for L_ in (2.0, 3.0, 4.0))
    + " (exact " + ", ".join(f"{exact_by_L[L_]:.6f}" for L_ in (2.0, 3.0, 4.0)) + ")")
check(all(abs(c_by_L[L_] - exact_by_L[L_]) < 1e-9 for L_ in (2.0, 4.0)),
      "the slope formula holds for L = 2 and L = 4 too")
```

A printed line with the three numerical and the three exact slopes, 1.761594, 1.905148, 1.964028 (they agree in all six decimals), and the check for the two new cutoffs. Out [5] shows four PASS lines.

In [6], figure 2.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
a_grid = np.linspace(0.0, 2.0, 201)
left.semilogy(a_grid, c_exact * np.exp(-a_grid), color="C0",
              label="exact $c\\,e^{-a_{4,0}}$")
left.semilogy(SLICES, c_numeric, "o", color="C3", label="numerical (Richardson)")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("slope $c$ (units of $m/H$, logarithmic)")
left.set_title("the slope along the history")
left.legend(fontsize=8)
```

Left panel: the exact slope $c\,e^{-a_{4,0}}$ for 201 slices from 0 to 2 on a logarithmic axis, and the five numerical values as red dots.

```
L_grid = np.linspace(0.5, 6.0, 221)
right.plot(L_grid, 2.0 / (1.0 + np.exp(-L_grid)), color="C0",
           label="exact $2/(1 + e^{-L})$")
right.plot(list(c_by_L), list(c_by_L.values()), "o", color="C3", label="numerical")
right.axhline(2.0, color="gray", linestyle=":", label="limit $2M/(2M - H) = 2$")
right.set_xlabel("cutoff $L$ (units of $1/H$)")
right.set_ylabel("slope $c$ at $a_{4,0} = 0$")
right.set_title("the slope against the cutoff ($M = H = 1$)")
right.legend(fontsize=8)
save_figure(fig, "band_slope",
            "The slope $c = d\\varepsilon/dk$ of the brane band at $k = 0$ "
            ...)
```

Right panel: the exact slope $2/(1 + e^{-L})$ for cutoffs from 0.5 to 6, the three numerical values (`list(c_by_L)` is the list of the keys of the dictionary, the cutoffs, and `c_by_L.values()` the slopes), and the limit 2 as a dotted line. **What figure 14c.2 shows.** Left: the five dots lie on a straight line on the logarithmic axis, falling from 1.905 to 0.258: the slope decreases exactly like $e^{-a_{4,0}}$, the redshift. Right: the slope grows with the cutoff and approaches 2 from below; the three dots lie on the curve.

In [7], the rescaling identity and the gap of $N = 8$.

```
test_k = [0.25, 1.0, 2.5]
test_sectors = [(1.0, "even", 0), (1.0, "odd", 0), (-1.0, "even", 1)]
combos = [(a_, k_, s_) for a_ in SLICES for k_ in test_k for s_ in test_sectors]
```

Three momenta and three sectors (the band, the odd label 0, the even label 1 of $j = -1$), and all $5 \times 3 \times 3 = 45$ combinations with the slices.

```
at_slice = levels(np.array([k_ for a_, k_, s_ in combos]),
                  np.array([s_[0] for a_, k_, s_ in combos]),
                  np.array([s_[1] for a_, k_, s_ in combos]),
                  np.array([s_[2] for a_, k_, s_ in combos]),
                  a4=np.array([a_ for a_, k_, s_ in combos]))
at_zero = levels(np.array([k_ * math.exp(-a_) for a_, k_, s_ in combos]),
                 np.array([s_[0] for a_, k_, s_ in combos]),
                 np.array([s_[1] for a_, k_, s_ in combos]),
                 np.array([s_[2] for a_, k_, s_ in combos]))
```

The 45 levels twice: at their slices with the momentum k, and at the slice 0 with the momentum $ke^{-a_{4,0}}$ (`a4` is then the default 0).

```
check(np.max(np.abs(at_slice - at_zero)) < 1e-12
      and record_check("free_rescaling_relation_and_band_monotone"),
      "eps(k, a4,0) = eps(k e^(-a4,0), 0) for 45 levels at the five slices",
      record=f"{RUST_REPORT}, check free_rescaling_relation_and_band_monotone")
```

They agree to 10^{-12}: the rescaling identity of Section 14.7 for the levels.

```
k_fig = np.linspace(0.0, 4.0, 41)
band_slices = levels(np.tile(k_fig, 5), 1.0, "even", 0,
                     a4=np.repeat(SLICES, len(k_fig))).reshape(5, len(k_fig))
gaps = levels(np.full(5, 0.25), 1.0, "even", 0, a4=np.array(SLICES))
```

For the figure, the band at 41 momenta at each of the five slices: `np.tile(k_fig, 5)` repeats the whole list five times, `np.repeat` repeats each slice 41 times, so the pairs match. Then the gaps: the band at $k = 0.25$ at the five slices.

```
shells_record = read_csv(f"{SPECTRUM}/closed-shells.csv")
gap_record = [float(r["gap"]) for r in shells_record if r["N_closed"] == "8"]
for a_, g_ in zip(SLICES, gaps):
    say(f"slice a4,0 = {a_}: gap of the free N = 8 state {g_:.10f} m")
check(len(gap_record) == 5 and np.max(np.abs(gaps - np.array(gap_record))) < 1e-11,
      "the N = 8 gaps at the five slices equal the record",
      record=f"{SPECTRUM}/closed-shells.csv, rows N_closed = 8")
```

The column gap of the five rows with $N = 8$ of the closed-shell record, the five printed gaps (the table of Section 14.20) and the check. Out [7] shows two PASS lines.

In [8], figure 3.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), sharey=True)
for index, a_ in enumerate(SLICES):
    left.plot(k_fig, band_slices[index], label=f"$a_{{4,0}} = {a_}$")
```

```
    right.plot(k_fig * math.exp(-a_), band_slices[index], "o", markersize=3,
               label=f"$a_{{4,0}} = {a_}$")
right.plot(k_fig, band_slices[0], color="black", linewidth=0.8,
           label="slice 0 band")
```

Left: the band against k at each slice. Right: the same levels against the redshifted momentum $ke^{-a_{4,0}}$ as dots, and the band of the slice 0 as a thin black line.

```
left.set_xlabel("3-momentum $k$ (units of $H$)")
left.set_ylabel("brane band $\\varepsilon$ (units of $m$)")
left.set_title("the band at five slices")
left.legend(fontsize=8)
right.set_xlabel("redshifted momentum $k\\,e^{-a_{4,0}}$ (units of $H$)")
right.set_title("all slices on one curve")
right.legend(fontsize=7)
save_figure(fig, "band_redshift",
            "The brane band along the prescribed deflating history. Left: the "
            ...)
```

What figure 14c.3 shows. Left: five curves from the origin, the later slices lower: at $k = 4$ the band is near $5\,m$ at the slice 0 and below $1\,m$ at the slice 2. Right: all dots fall on the black curve; a later slice covers only a shorter piece of it (up to $4e^{-a_{4,0}}$). This is the rescaling identity: nothing but the momentum scale changes along the history.

In [9], the block-type symmetries.

```
sym_k = [0.25 * math.sqrt(n2) for n2 in (0, 1, 2, 5, 9)]
sym_l = list(range(-3, 4))
grid = [(k_, l_) for k_ in sym_k for l_ in sym_l]
K_s = np.array([k_ for k_, l_ in grid])
L_s = np.array([l_ for k_, l_ in grid])
```

The momenta of five shells ($k = 0.25\sqrt{n^2}$ for $n^2 = 0, 1, 2, 5, 9$), the labels -3 to 3, and their 35 combinations as two arrays.

```
plus_even = levels(K_s, 1.0, "even", L_s)
minus_even = levels(K_s, -1.0, "even", -L_s)
plus_odd = levels(K_s, 1.0, "odd", L_s)
minus_odd = levels(K_s, -1.0, "odd", -L_s - 1)
flipped_even = levels(-K_s, -1.0, "even", L_s)
flipped_odd = levels(-K_s, -1.0, "odd", L_s)
```

Six sets of levels: both parities of $j = +1$; those of $j = -1$ with the mirrored labels ($l \to -l$ for even parity and $l \to -l - 1$ for odd parity, the label map of Section 14.13, because the target $\pi/2 + l\pi$ goes to $-(\pi/2 + l\pi) = \pi/2 + (-l - 1)\pi$); and those of $j = -1$ at the reversed momentum $-k$ with the same labels.

```
mirror = max(np.max(np.abs(plus_even + minus_even)),
             np.max(np.abs(plus_odd + minus_odd)))
```

```
flip = max(np.max(np.abs(flipped_even - plus_even)),
           np.max(np.abs(flipped_odd - plus_odd)))
check(mirror < 1e-12 and flip < 1e-12 and record_check("free_block_type_symmetries"),
      "spec h_(-1) = -spec h_(+1) and eps_(-1)(-k) = eps_(+1)(k), label by label",
      record=f"{RUST_REPORT}, check free_block_type_symmetries")
```

The two relations of Section 14.5: the levels of $j = -1$ are minus those of $j = +1$ (`mirror`), and the level of $(-1, -k)$ equals that of $(+1, k)$ (`flip`); both to 10^{-12}, label by label.

```
k_mirror = np.linspace(0.0, 2.0, 41)
fig_labels = list(range(-2, 3))
plus_curves = levels(np.tile(k_mirror, 5), 1.0, "even",
                     np.repeat(fig_labels, len(k_mirror))).reshape(5, -1)
minus_curves = levels(np.tile(k_mirror, 5), -1.0, "even",
                      np.repeat(fig_labels, len(k_mirror))).reshape(5, -1)
say("even levels at k = 2, labels -2 ... 2:")
say("  j = +1: " + ", ".join(f"{x:.4f}" for x in plus_curves[:, -1]))
say("  j = -1: " + ", ".join(f"{x:.4f}" for x in minus_curves[:, -1]))
```

For the figure, the even levels of labels -2 to 2 for 41 momenta from 0 to 2, for both types (`reshape( 5, -1)`: five rows, the length of each row worked out by numpy). The printout of the last column, $k = 2$: for $j = +1$ the levels -6.3742, -4.4814, 2.6976, 5.4109, 7.1695 and for $j = -1$ the same numbers with the opposite signs, in the opposite order (Out [9]).

In [10], figure 4.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), sharey=True)
for row, label in enumerate(fig_labels):
    left.plot(k_mirror, plus_curves[row], label=f"label {label}")
    right.plot(k_mirror, minus_curves[row], label=f"label {label}")
for ax, title in ((left, "block type $j = +1$"), (right, "block type $j = -1$")):
    ax.axhline(0.0, color="black", linewidth=0.6)
    ax.set_xlabel("3-momentum $k$ (units of $H$)")
    ax.set_title(title + ", even parity")
    ax.legend(fontsize=7, loc="upper left")
left.set_ylabel("level $\\varepsilon$ (units of $m$)")
save_figure(fig, "block_type_mirror",
            "The even-parity levels with labels $-2$ to $2$ against the "
            ...)
```

Five curves in each panel, the zero line, labels and titles (a loop over the two panels sets what they share). **What figure 14c.4 shows.** Left ($j = +1$): the label 0 curve starts at 0 and rises (the brane band); labels 1 and 2 start at 1.448 and 2.321 and rise; labels -1 and -2 are their mirror images below zero. Right ($j = -1$): the same picture turned upside down: the label 0 curve falls from 0 (the sea partner of the band).

In [11], the tip angle.

```
tip_rows = read_csv(f"{SPECTRUM}/tip-angle.csv")
tip_k = np.array([0.25, 0.5, 1.0])
```

```
tip_levels = {th: levels(tip_k, 1.0, "even", 0, tip=th) for th in (0.0, 0.5, 1.0)}
suppression = np.exp(-tip_k * (math.exp(3.0) - 1.0))  # H = 1, L = 3
worst_tip, below = 0.0, True
```

The record of the Rust solver (9 rows), the three momenta of the record, the band at these momenta for the three tip angles (a dictionary from the angle to an array of three levels; `tip=th` is handed on to `shoot` through `**options`), the suppression factor $\exp(-k(e^3 - 1))$ at the brane for $H = 1$, $L = 3$ and the slice 0, and two variables for the results.

```
for r in tip_rows:
    i_k = list(tip_k).index(float(r["k"]))
    mine = tip_levels[float(r["theta_tip"])][i_k]
    worst_tip = max(worst_tip, abs(mine - float(r["eps"])))
    shift = mine - tip_levels[0.0][i_k]
    below &= abs(shift) <= max(suppression[i_k], 1e-12)
check(len(tip_rows) == 9 and worst_tip < 1e-11 and below
      and record_check("free_tip_angle_insensitivity"),
      "the band levels equal tip-angle.csv; the shifts are below the suppression",
      record=f"{SPECTRUM}/tip-angle.csv and {RUST_REPORT}, check "
             "free_tip_angle_insensitivity")
```

For every row: the position of its momentum in the list (`index` finds it), the notebook's level for its angle and momentum, the largest difference from the recorded level, and the shift from the angle 0, which must not exceed the suppression factor (or 10^{-12}, the size of rounding, whichever is larger).

```
k_tip = np.linspace(0.1, 1.0, 10)
shifts = {th: np.abs(levels(k_tip, 1.0, "even", 0, tip=th)
                     - levels(k_tip, 1.0, "even", 0, tip=0.0)) for th in (0.5, 1.0)}
def shown(x):
    return f"{x:.1e}" if x >= 1e-14 else "below 1e-14 (rounding)"


for k_, s5, s10 in zip(k_tip, shifts[0.5], shifts[1.0]):
    say(f"k = {k_:.1f}: shift for theta = 0.5: {shown(s5)}, for theta = 1: "
        f"{shown(s10)}")
```

The shifts for ten momenta $0.1, 0.2, \ldots, 1$ and the angles 0.5 and 1. The small function `shown` writes a shift with one decimal and an exponent, or, below 10^{-14}, the words "below 1e-14 (rounding)": such tiny numbers are rounding errors and differ from computer to computer, and the printed output of a notebook must be the same everywhere. Out [11] prints the ten lines of Section 14.20: 4.6×10^{-4} and 1.2×10^{-3} at $k = 0.1$, down to 7.7×10^{-14} and 2.7×10^{-13} at $k = 0.8$, and rounding from $k = 0.9$ on.

In [12], figure 5.

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
k_line = np.linspace(0.1, 1.0, 91)
ax.semilogy(k_line, np.exp(-k_line * (math.exp(3.0) - 1.0)), color="black",
            label="suppression $\\exp(-k(e^{HL} - 1)/H)$")
```

```
for th, marker in ((0.5, "o"), (1.0, "s")):
    ax.semilogy(k_tip, np.maximum(shifts[th], 1e-16), marker, markersize=5,
                label=f"$|\\varepsilon(\\theta = {th}) - \\varepsilon(\\theta = 0)|$")
```

The suppression factor as a line on a logarithmic axis, and the shifts as circles and squares; `np.maximum(x, 1e-16)` replaces any shift below 10^{-16} (or exactly 0, whose logarithm does not exist) by 10^{-16}, so that it can be drawn.

```
ax.axhline(1e-15, color="gray", linestyle=":", linewidth=0.8)
ax.annotate("rounding level", (0.75, 2e-15), fontsize=8, color="gray")
ax.set_xlabel("3-momentum $k$ (units of $H$)")
ax.set_ylabel("shift of the band level (units of $m$, logarithmic)")
ax.set_title("the band does not feel the tip angle ($L = 3$, slice 0)")
ax.legend(fontsize=8)
save_figure(fig, "tip_insensitivity",
            "The change of the brane-band level when the tip condition "
            ...)
```

A dotted line at 10^{-15}, the rounding level, with its name. **What figure 14c.5 shows.** Both rows of markers fall steeply, by a factor 11 to 33 per step of 0.1, much faster than the black suppression line, and stay below it by two to six powers of ten; the squares ($\theta = 1$) lie a little above the circles ($\theta = 0.5$). At $k = 0.9$ and 1 the markers sit at the rounding level, where their exact positions are noise.

In [13], shells and particle labels.

```
def shells(n2_max):
    r = math.isqrt(n2_max) + 1
    count = [0] * (n2_max + 1)
    for x in range(-r, r + 1):
        for y_ in range(-r, r + 1):
            for z_ in range(-r, r + 1):
                s = x * x + y_ * y_ + z_ * z_
                if s <= n2_max:
                    count[s] += 1
    return [(n2, c) for n2, c in enumerate(count) if c > 0]
```

The shells up to $n^2_{\max}$ by counting: `math.isqrt` is the whole-number square root (rounded down), so every integer vector with $n^2 \le n^2_{\max}$ has entries between $-r$ and r; the three loops visit all of them and count each n^2. The result is the list of pairs $(n^2, r_3(n^2))$ with $r_3 > 0$.

```
check(shells(9) == [(0, 1), (1, 6), (2, 12), (3, 8), (4, 6), (5, 24), (6, 24),
                    (8, 12), (9, 30)],
      "r3(n2) = 1, 6, 12, 8, 6, 24, 24, 12, 30 for n2 = 0, 1, 2, 3, 4, 5, 6, 8, 9")
SECTORS = [(1, "even"), (1, "odd"), (-1, "even"), (-1, "odd")]
SIGN = {1: "+1", -1: "-1"}
```

The first values of r_3 (Section 14.21; no shell 7), the four sectors of a shell, and the texts $+1$, -1 used in the names of the levels.

```
def lowest_particle_labels(pairs, a4):
    k_ = np.array([0.25 * math.sqrt(n2) for n2, _, _, _ in pairs])
    j_ = np.array([float(j) for _, _, j, _ in pairs])
    par = np.array([p for _, _, _, p in pairs])
    phi0 = shoot(0.0, k_, j_, a4)
    offset = np.where(par == "even", 0.0, 0.5 * np.pi)
    l_min = np.floor((phi0 - offset) / np.pi).astype(int) + 1
    nearest = np.round((phi0 - offset) / np.pi).astype(int)
    zero_mode = (k_ == 0.0) & (np.abs(phi0 - offset - nearest * np.pi) < 1e-12)
    return np.where(zero_mode, nearest, l_min), phi0, offset, k_, j_, par
```

For a list of sectors ($n^2, r_3, j,$ parity): their momenta, types and parities as arrays, and $\Phi(0)$, the shooting function at $\varepsilon = 0$. Since Φ grows with ε, a label is a particle label when its target lies above $\Phi(0)$; the first such label is $\lfloor(\Phi(0) - \text{offset})/\pi\rfloor + 1$ (`np.floor` rounds down; `astype(int)` makes whole numbers). The exception is a zero mode: at $k = 0$, when $\Phi(0)$ lies exactly on a target (to 10^{-12}; `&` is "and" for arrays), that label itself is the first particle label (the convention of Section 14.21). The function returns these labels and the arrays it made.

```
pairs30 = [(n2, r3, j, p) for n2, r3 in shells(30) for j, p in SECTORS]
l_min, phi0, offset, k30, j30, p30 = lowest_particle_labels(pairs30, 0.0)
e_first = levels(k30, j30, p30, l_min)
e_below = levels(k30, j30, p30, l_min - 1)
zero_k = k30 == 0.0
```

All sectors of the shells up to $n^2 = 30$; their first particle labels at the slice 0; the levels of these labels and of the labels just below; and a marker for the sectors with $k = 0$.

```
branch_ok = (np.all(e_below < 0.0)
             and np.all(e_first[zero_k & (p30 == "even")] == 0.0)
             and np.all(e_first[~(zero_k & (p30 == "even"))] > 0.0))
closest = float(min(np.min(e_first[~(zero_k & (p30 == "even"))]),
                    np.min(-e_below)))
report("level closest to zero away from the zero modes (n2 <= 30)",
       f"{closest:.4f}", "m")
check(branch_ok and f"{closest:.4f}" == "0.4307"
      and record_check("free_particle_branch_labels"),
      "particles are the labels l >= l_min in every sector (n2 <= 30)",
      record=f"{RUST_REPORT}, check free_particle_branch_labels")
```

The particle rule holds when every level just below the first particle label is negative, the first particle level is exactly 0 in the even sectors at $k = 0$ (the zero modes) and positive in all other sectors (`~` is "not" for arrays; an array indexed by a true-false array keeps the entries marked true). `closest` is the level nearest to zero apart from the zero modes, from above or from below: $0.4307\,m$, as in the Rust check. Out [13] shows two PASS lines.

In [14], the closed shells.

```
def closed_shells(a4, n2_max, e_max=1.6):
    pairs = [(n2, r3, j, p) for n2, r3 in shells(n2_max) for j, p in SECTORS]
```

```
    l_next, _, _, k_, j_, par = lowest_particle_labels(pairs, a4)
    found_levels = []  # (energy, degeneracy, name, n2)
    active = np.arange(len(pairs))
```

The free aufbau at one slice, up to the energy $1.6\,m$. For every sector the next label to compute starts at its first particle label; `active` lists the sectors that are still open (all, at the start).

```
    while len(active) > 0:  # one more label in every sector that is still open
        e_ = levels(k_[active], j_[active], par[active], l_next[active], a4=a4)
        keep = e_ <= e_max
        for index, energy in zip(active[keep], e_[keep]):
            n2, r3, j, p = pairs[index]
            found_levels.append((float(energy), 4.0 * r3,
                                 f"{n2}:{SIGN[j]}:{p}:{l_next[index]}", n2))
        active = active[keep]
        l_next[active] += 1
```

In each pass the next level of every open sector is computed at once; the levels at or below $1.6\,m$ are stored with their degeneracy $4r_3(n^2)$ and a name such as `1:+1:even:0` (shell, block type, parity, label); a sector whose next level lies above $1.6\,m$ is closed, because its higher labels lie higher still; the open sectors move on to the next label.

```
    with_levels = {n2 for _, _, _, n2 in found_levels}
    stop = next(n2 for n2, _ in shells(n2_max) if n2 > 0 and n2 not in with_levels)
    found_levels = [lev for lev in found_levels if lev[3] < stop]
    found_levels.sort(key=lambda lev: (lev[0], lev[2]))
```

The set of shells that have a level below $1.6\,m$, the first shell $n^2 > 0$ without one (the band rises with $|\mathbf{k}|$, so no later shell can contribute; this is the rule of the Rust solver), only the levels of the shells before it, sorted by energy and, for equal energies, by name.

```
    table, total, i = [], 0.0, 0
    while i < len(found_levels):  # group the degenerate levels
        e0, group, names = found_levels[i][0], 0.0, []
        while i < len(found_levels) and found_levels[i][0] - e0 <= 1e-9:
            group += found_levels[i][1]
            names.append(found_levels[i][2])
            i += 1
        total += group
        following = found_levels[i][0] if i < len(found_levels) else float("nan")
        table.append((int(total), e0, following, ";".join(names)))
    return table, stop
```

Walking through the sorted levels, the levels within 10^{-9} of the first level of a group are collected into one degenerate group; after each group the running total N is a closed shell, stored with the energy of the group, the energy of the next level (or `nan`, "not a number", at the end) and the names of the group joined by semicolons.

```
agree = True
tables = {}
for a_, n2_max in ((0.0, 40), (0.5, 80)):
    table, stop = closed_shells(a_, n2_max)
    tables[a_] = table
    mine_rows = [r for r in shells_record if float(r["a4"]) == a_]
    same = len(table) == len(mine_rows) and all(
        int(r["N_closed"]) == t[0] and r["levels_of_the_last_group"] == t[3]
        and abs(float(r["eps_last_filled"]) - t[1]) < 1e-11
        for r, t in zip(mine_rows, table))
    agree &= same and stop < n2_max
    say(f"slice {a_}: {len(table)} closed shells up to 1.6 m, the first shell "
        f"without a level below 1.6 m is n2 = {stop}; equal to the record: {same}")
check(agree, "the closed shells of the slices 0 and 0.5 equal the record row by row",
      record=f"{SPECTRUM}/closed-shells.csv, rows a4 = 0 and 0.5")
say("slice 0: N = " + ", ".join(str(t[0]) for t in tables[0.0]))
```

The closed shells at the slices 0 (shells up to $n^2 = 40$) and 0.5 (up to 80; the redshifted band needs more shells), compared row by row with the record: the same numbers N, the same names of the last group, the same energies to 10^{-11}; and the stopping shell must lie inside the range searched. Out [14] prints 22 closed shells at the slice 0 (stopping shell $n^2 = 20$) and 55 at the slice 0.5 (stopping shell 53), and the list of N at the slice 0.

```
bulk_edge = float(min(levels(0.0, 1.0, "odd", 0), levels(0.0, 1.0, "even", 1)))
below_edge = [t for t in tables[0.0] if t[1] < bulk_edge and t[2] - t[1] > 1e-6]
N_large = below_edge[-1][0]
N_mid = min((t[0] for t in below_edge if t[0] >= 8),
            key=lambda n: abs(n - N_large / 4))
```

The bulk edge, the smaller of the odd level 0 and the even level 1 at $k = 0$; the closed shells whose last level lies below it (and are separated from the next level by more than 10^{-6}); N_{large}, the last of them; and N_{mid}, the one nearest to $N_{\mathrm{large}}/4$ (`min` with `key=` picks the element with the smallest value of the key function).

```
parameters = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                        .read_text(encoding="utf-8"))
numbers = parameters["particleNumbers"]
report("bulk edge", f"{bulk_edge:.12f}", "m")
report("particle numbers N", f"8, {N_mid}, {N_large}")
check(abs(bulk_edge - numbers["bulkEdge"]) < 1e-12 and N_large == 688
      and N_mid == 136 and numbers["values"] == [8.0, 136.0, 688.0],
      "the bulk edge 1.2922928281 and the particle numbers 8, 136, 688",
      record="Revision/kohn_sham/results/parameters.json, particleNumbers")
```

The record of the Revision runs, the two RESULT lines (the bulk edge $1.292292828069\,m$ and the numbers 8, 136, 688) and the check against the record. Out [14] shows two PASS lines.

In [15], figure 6.

```
fig, ax = plt.subplots(figsize=(8.0, 4.6))
for a_, style in ((0.0, "-"), (0.5, "--")):
```

```
    energies = [t[1] for t in tables[a_]]
    counts = [t[0] for t in tables[a_]]
    ax.step(energies, counts, style, where="post",
            label=f"slice $a_{{4,0}} = {a_}$")
```

For each slice a **staircase**: `step(x, y, where="post")` draws a horizontal line from each point to the next value of x, then a vertical jump; here x is the energy of the last filled group and y the closed-shell number N.

```
ax.axvline(bulk_edge, color="gray", linestyle="-.", linewidth=0.8)
ax.annotate("bulk edge", (bulk_edge + 0.015, 15), fontsize=8, color="gray")
last_filled = {t[0]: t[1] for t in tables[0.0]}  # N -> its last filled level
for n_ in (8, N_mid, N_large):
    t_ = last_filled[n_]
    ax.plot([t_], [n_], "o", color="C3")
    ax.annotate(f"N = {n_}", (t_ + 0.02, n_ * 0.75), fontsize=8, color="C3")
```

A vertical line at the bulk edge, and the three particle numbers of the Revision runs as red dots at their last filled levels, with their names.

```
ax.set_yscale("log")
ax.set_xlabel("energy of the last filled level (units of $m$)")
ax.set_ylabel("closed-shell particle number $N$ (logarithmic)")
ax.set_title("the free aufbau: closed shells up to $1.6\\,m$")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "closed_shells",
            "The closed shells of the free aufbau: the particle number $N$ "
            ...)
```

A logarithmic vertical axis, labels and title. **What figure 14c.6 shows.** At the slice 0 (solid) the staircase stays at $N = 8$ up to $0.43\,m$ (the gap), then climbs in steps, one per lattice shell of the band, to 688 just below the bulk edge and to 1552 at $1.584\,m$, the last group below $1.6\,m$. At the slice 0.5 (dashed) the first step comes already at $0.27\,m$ and the staircase lies far higher: the redshifted band puts many more states below the same energy.

In [16], the last check.

```
figure_names = ["14c_1_band_structure.png", "14c_2_band_slope.png",
                "14c_3_band_redshift.png", "14c_4_block_type_mirror.png",
                "14c_5_tip_insensitivity.png", "14c_6_closed_shells.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all six figure files of this notebook exist")
all_checks_passed()
```

The six figure files must exist; the last line prints ALL 15 CHECKS PASSED (notebook 14c). The 15 checks are: 2 in In [3], 4 in In [5], 2 in In [7], 1 in In [9], 1 in In [11], 2 in In [13], 2 in In [14] and 1 in In [16].

14.26 The exact local exchange of the contact interaction

What is needed. The quanta of dirac16complex interact through the contact term $U = \frac{\lambda}{2} S^2$ of the Lagrangian, with the scalar density $S = \bar{\Psi}\Psi = \Psi^\dagger C \Psi$, normal ordered in the quantised theory; $\lambda > 0$ is a repulsion, and λ has the dimension mass^{-6} because the field has the dimension 7/2 in eight dimensions (`Revision/kohn_sham/ks-theory.json`, units). The Kohn-Sham model needs the energy of this interaction in a Kohn-Sham state, a Slater determinant of orbitals or a thermal mixture of them (Chapter 13), as a function of the densities. This section derives it exactly. The result is surprising and very useful: the exchange energy, which for most interactions depends on the orbitals in a complicated way, is here an ordinary function of two densities at the same point.

The expectation-value rule (Chapter 10). For one quantum in the mode u (a column of 16 numbers), the expectation value of a density $\Psi^\dagger X \Psi$ is $u^\dagger B X u$, with the matrix B of the indefinite (Krein) inner product. For many independent quanta in the modes u with the occupations f (1 for filled, 0 for empty, or a number between at a temperature),

$$\langle \Psi^\dagger X \Psi \rangle = \mathrm{Tr}(X\rho), \qquad \rho = \sum_{\text{occupied}} f\, u u^\dagger B.$$

Rule: $\mathrm{Tr}(X u u^\dagger B) = \mathrm{Tr}(u^\dagger B X u) = u^\dagger B X u$, because a trace does not change when the first factors are moved to the end, and $u^\dagger B X u$ is a single number. The 16×16 matrix ρ is the **one-body matrix** of the state. The number density and the scalar density are

$$n = \langle \Psi^\dagger B \Psi \rangle = \mathrm{Tr}(B\rho), \qquad S = \langle \bar{\Psi}\Psi \rangle = \langle \Psi^\dagger C \Psi \rangle = \mathrm{Tr}(C\rho).$$

Status: the rule is the expectation-value rule of the canonical quantisation (Chapter 10), stated in the record as exchange.expectationRule of `Revision/kohn_sham/ks-theory.json`; the two density formulas are the check gas_densities of both theory reports.

The plane waves of the good sector. In flat space ($H = 0$, so $W = 1$ and $\kappa = 1$ at the slice 0) a plane wave $e^{-i\varepsilon x_4 + i\mathbf{p}\cdot\mathbf{x}} u$ with the momentum $\mathbf{p} = (p_1, p_2, p_3, p_0)$ along the four space-like directions x_0, x_1, x_2, x_3 (the good sector has no momentum along the extra times) solves $h_{\mathbf{p}} u = \varepsilon u$ with

$$h_{\mathbf{p}} = M(-i\gamma^{(x_4)}) - \sum_a p_a\, \gamma^{(x_4)}\gamma^{(a)} \qquad (a = x_0, x_1, x_2, x_3).$$

Rule: the Hamiltonian h of Section 14.4 with $v = 0$; its derivative term $i\gamma^{(x_4)}\gamma^{(x_0)}\partial_y$ acting on $e^{ip_0 y}$ gives $i\gamma^{(x_4)}\gamma^{(x_0)}(ip_0) = -p_0\gamma^{(x_4)}\gamma^{(x_0)}$. Its square, line by line:

$$\big(-i\gamma^{(x_4)}\big)^2 = -\big(\gamma^{(x_4)}\big)^2 = 1, \qquad \big(\gamma^{(x_4)}\gamma^{(a)}\big)^2 = -\big(\gamma^{(x_4)}\big)^2\big(\gamma^{(a)}\big)^2 = 1.$$

Rule: $(\gamma^{(x_4)})^2 = \eta_{44} = -1$ and $(\gamma^{(a)})^2 = +1$ for a space-like direction; in the second, moving the second $\gamma^{(x_4)}$ to the left past $\gamma^{(a)}$ costs one sign.

$$\big(-i\gamma^{(x_4)}\big)\big(\gamma^{(x_4)}\gamma^{(a)}\big) + \big(\gamma^{(x_4)}\gamma^{(a)}\big)\big(-i\gamma^{(x_4)}\big) = -i\big(\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(a)} - \gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(a)}\big) = 0,$$

Rule: anticommutation of different gammas ($\gamma^{(x_4)}$ is moved to the left past $\gamma^{(a)}$ in the second product, which costs one sign). For two different space-like directions $a \neq b$, line by line:

$$\big(\gamma^{(x_4)}\gamma^{(a)}\big)\big(\gamma^{(x_4)}\gamma^{(b)}\big) = -\gamma^{(x_4)}\gamma^{(x_4)}\gamma^{(a)}\gamma^{(b)} = \gamma^{(a)}\gamma^{(b)}, \qquad \big(\gamma^{(x_4)}\gamma^{(b)}\big)\big(\gamma^{(x_4)}\gamma^{(a)}\big) = \gamma^{(b)}\gamma^{(a)}.$$

Rule: move the second $\gamma^{(x_4)}$ to the left past $\gamma^{(a)}$, which costs one sign, and use $\gamma^{(x_4)}\gamma^{(x_4)} = -1$; the second product is the same with a and b exchanged.

$$\big(\gamma^{(x_4)}\gamma^{(a)}\big)\big(\gamma^{(x_4)}\gamma^{(b)}\big) + \big(\gamma^{(x_4)}\gamma^{(b)}\big)\big(\gamma^{(x_4)}\gamma^{(a)}\big) = \gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 0.$$

Rule: add the two products; different gammas anticommute, $\gamma^{(a)}\gamma^{(b)} = -\gamma^{(b)}\gamma^{(a)}$. Hence

$$h_{\mathbf{p}}^2 = (M^2 + p_1^2 + p_2^2 + p_3^2 + p_0^2)\,1 = E^2\,1, \qquad E = \sqrt{M^2 + p^2}.$$

Rule: multiply out the square of the sum; the squares of the five terms give M^2 and the p_a^2, and the mixed products cancel in pairs. Since $h_{\mathbf{p}}$ is Hermitian (Section 14.4), its eigenvalues are real numbers whose squares are E^2: they are $+E$ and $-E$. Its trace is zero (it is a sum of products of one or two different gammas, Chapter 5), so the two eigenvalues occur equally often: $+E$ eight times and $-E$ eight times. Every momentum carries eight positive-energy modes. The matrix

$$G_{\mathbf{p}} = \tfrac{1}{2}\big(1 + h_{\mathbf{p}}/E\big)$$

is the **projector** on them: $G_{\mathbf{p}}^2 = \frac{1}{4}(1 + 2h_{\mathbf{p}}/E + h_{\mathbf{p}}^2/E^2) = \frac{1}{4}(2 + 2h_{\mathbf{p}}/E) = G_{\mathbf{p}}$, and $\mathrm{Tr}\, G_{\mathbf{p}} = \frac{1}{2}(16 + 0) = 8$. Status: PROVED; check gas_mode_projector (both reports).

Why every symmetric occupation has $\rho = (nB + SC)/16$. The sum of $uu^\dagger$ over the eight positive-energy modes (orthonormal, because $h_{\mathbf{p}}$ is Hermitian) is the projector $G_{\mathbf{p}}$. We assume that the eight modes of one momentum carry the same occupation f, as they do in every closed shell (all eight filled) and in every thermal state (the occupation depends only on the energy, which is the same for the eight). Then these eight modes contribute $fG_{\mathbf{p}}B$ to ρ; we write the case $f = 1$, and a common factor f changes nothing below. Line by line:

$$G_{\mathbf{p}}B = \tfrac{1}{2}B + \frac{M}{2E}\big(-i\gamma^{(x_4)}\big)B - \frac{1}{2E}\sum_a p_a\gamma^{(x_4)}\gamma^{(a)}B.$$

Rule: insert $h_{\mathbf{p}}$ into $G_{\mathbf{p}}$ and multiply by B.

$$\big(-i\gamma^{(x_4)}\big)B = \big(-i\gamma^{(x_4)}\big)\big(-iC\gamma^{(x_4)}\big) = -\gamma^{(x_4)}C\gamma^{(x_4)} = -C\gamma^{(x_4)}\gamma^{(x_4)} = C.$$

Rule: $B = -iC\gamma^{(x_4)}$; $(-i)^2 = -1$; $\gamma^{(x_4)}$ commutes with C (it passes the four factors of C, none of which is $\gamma^{(x_4)}$: four signs); $(\gamma^{(x_4)})^2 = -1$.

$$G_{\mathbf{p}}B = \tfrac{1}{2}\Big(B + \frac{M}{E}\,C\Big) - \frac{1}{2E}\sum_a p_a\gamma^{(x_4)}\gamma^{(a)}B.$$

The last term changes its sign when $\mathbf{p}$ is replaced by $-\mathbf{p}$. If the state occupies $\mathbf{p}$ and $-\mathbf{p}$ equally, which is true for every closed shell and for every thermal state, these terms cancel in pairs, and each pair leaves $B + (M/E)C$ (times the common occupation). Filled negative-energy modes give in the same way $\frac{1}{2}(1 - h_{\mathbf{p}}/E)B$, whose symmetric part is $\frac{1}{2}(B - (M/E)C)$. So the one-body matrix of any such state is a combination of B and C only, $\rho = \alpha B + \beta C$. The two numbers follow from the densities:

$$n = \operatorname{Tr}(B\rho) = \alpha \operatorname{Tr}(B^2) + \beta \operatorname{Tr}(BC) = 16\alpha, \qquad S = \operatorname{Tr}(C\rho) = \alpha \operatorname{Tr}(CB) + \beta \operatorname{Tr}(C^2) = 16\beta.$$

Rule: $B^2 = C^2 = 1$, whose trace is 16, and $\operatorname{Tr}(BC) = \operatorname{Tr}(-i\gamma^{(x_4)}) = 0$ (Chapter 5: $BC = -i\gamma^{(x_4)}$). Hence

$$\rho = \frac{nB + SC}{16},$$

for every occupation that gives the eight modes of each momentum the same occupation and is symmetric under $\mathbf{p} \to -\mathbf{p}$, at every temperature (the record: "any p -> -p symmetric occupation of the 8-fold good-sector gas", `Revision/kohn_sham/ks-theory.json`, exchange.uniformGas). If only some of the eight modes of a momentum were filled, their one-body matrix would in general not be a combination of B and C, and the formula would not apply. Status: PROVED; checks gas_angular_average (both reports), gas_negative_energy_modes (sympy report) and gas_densities (both).

Wick's rule for the contact term. Write $S = \Psi^\dagger C \Psi = \sum_{A,B} \Psi_A^\dagger C_{AB} \Psi_B$, with the components $A, B = 0, \ldots, 15$. For a quasi-free state (a determinant or a thermal state of independent quanta), Wick's theorem (Section 13.4) gives the expectation of a normal-ordered product of two creation and two annihilation operators as the sum over the ways of pairing each $\Psi^\dagger$ with a Ψ, with a minus sign for the crossed pairing because the field anticommutes. With $f_{AB} = \langle \Psi_A^\dagger \Psi_B \rangle$:

$$\langle :S^2: \rangle = \sum_{A,B,C',D} C_{AB} C_{C'D} (f_{AB} f_{C'D} - f_{AD} f_{C'B}).$$

(The index C' carries a prime so that it is not confused with the matrix C.) The first product is the **direct** (Hartree) term, the second the **exchange** (Fock) term. Now the matrix form, line by line:

$$f_{AB} = \rho_{BA}.$$

Rule: $\langle \Psi^\dagger X \Psi \rangle = \sum_{A,B} X_{AB} f_{AB}$ and $\operatorname{Tr}(X\rho) = \sum_{A,B} X_{AB} \rho_{BA}$ must agree for every matrix X.

$$\sum C_{AB} C_{C'D} f_{AB} f_{C'D} = \Big(\sum_{A,B} C_{AB} \rho_{BA} \Big) \Big(\sum_{C',D} C_{C'D} \rho_{DC'} \Big) = (\operatorname{Tr} C\rho)^2.$$

Rule: the sum of a product of two factors with separate indices is the product of the two sums.

$$\sum C_{AB} C_{C'D} f_{AD} f_{C'B} = \sum C_{AB} \rho_{BC'} C_{C'D} \rho_{DA} = \operatorname{Tr}(C\rho C\rho).$$

Rule: $f_{AD} = \rho_{DA}$ and $f_{C'B} = \rho_{BC'}$; reorder the four numbers so that neighbouring indices agree, $A \to B \to C' \to D \to A$: a chain of matrix products that closes on itself is a trace. Hence

$$\langle{:}S^2{:}\rangle = \big(\operatorname{Tr} C\rho\big)^2 - \operatorname{Tr}(C\rho C\rho).$$

Status: PROVED (Section 13.4 and the lines above); the record verified it term by term on an explicit two-mode state (check hf_wick_contraction of the sympy report), and Notebook 14d repeats that on two states (In [6]).

The exact local exchange. Insert $\rho = (nB + SC)/16$. Line by line:

$$C\rho = \frac{nCB + S\,1}{16}.$$

Rule: $C^2 = 1$.

$$C\rho C\rho = \frac{n^2\,CBCB + nS\,CB + nS\,CB + S^2\,1}{256} = \frac{n^2 + 2nS\,CB + S^2}{256}.$$

Rule: multiply out the four products; $CBCB = (CBC)B = BB = 1$ by $CBC = B$ (Chapter 5) and $B^2 = 1$.

$$\operatorname{Tr}(C\rho C\rho) = \frac{16n^2 + 0 + 16S^2}{256} = \frac{n^2 + S^2}{16}.$$

Rule: $\operatorname{Tr} 1 = 16$ and $\operatorname{Tr}(CB) = 0$. The interaction energy per unit (proper) volume, $\frac{\lambda}{2}\langle{:}S^2{:}\rangle$, is therefore $e_H + e_x$ with

$$e_H = \frac{\lambda}{2}S^2, \qquad e_x = -\frac{\lambda}{2}\operatorname{Tr}(C\rho C\rho) = -\frac{\lambda}{32}(n^2 + S^2).$$

The **Hartree** term e_H and the **exchange** term e_x are both functions of the two densities at one point: the exchange of the contact interaction is exactly **local**, at every temperature, for every symmetric occupation. For one filled 8-fold level at rest the scalar and number densities are equal, $n = S$ (Exercise 14.8), and $e_x/e_H = -\frac{1}{32}\cdot 2S^2/\frac{1}{2}S^2 = -\frac{1}{8}$: the exchange removes one eighth of the Hartree energy, the rule $E_x = -E_H/g$ of Section 13.5 with the $g = 8$ modes per momentum. Status: PROVED; checks exchange_uniform_gas (both reports) and filled_shell_ratio (sympy report); the coefficients $-1/32$, $-1/32$ are recorded under exchange.uniformGas in ks-theory.json.

14.27 The Kohn-Sham potentials, the exact Fock exchange of the model's states, and the eight zero modes

The potentials. The interaction energy density is

$$e_{\text{int}}(n, S) = e_H + e_x = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2.$$

Rule: $\frac{1}{2} - \frac{1}{32} = \frac{15}{32}$. As in Chapter 13 (Section 13.9), the Kohn-Sham potential that belongs to a density is the derivative of the interaction energy density with respect to that density (for a local energy the functional derivative is an ordinary derivative). The density S multiplies the mass term of the Lagrangian, so its potential shifts the mass; the number density gives a potential that is added to h as v_v times the unit matrix:

$$M_{\text{eff}} = m + \frac{\partial e_{\text{int}}}{\partial S} = m + \frac{15}{16}\lambda S, \qquad v_v = \frac{\partial e_{\text{int}}}{\partial n} = -\frac{1}{16}\lambda n.$$

Rule: the derivative of S^2 is $2S$, and $2 \cdot \frac{15}{32} = \frac{15}{16}$, $2 \cdot \frac{1}{32} = \frac{1}{16}$. In the covariant equation v_v appears as $-iv_v\gamma^{(x_4)}$ (Section 14.3). These are the M_{eff} and v_v that Sections 14.3 to 14.25 used as given functions. Status: PROVED; check ks_potentials (both reports); the coefficients $15/16$ and $-1/16$ are recorded in ks-theory.json (exchange.kohnShamPotentials) and are the numbers 0.9375 and -0.0625 that the Rust solver reads (`Revision/kohn_sham/results/parameters.json`, theoryInputs).

The Lagrangian on shell. e_{int} is **homogeneous of degree 2**: doubling both densities multiplies it by 4. Line by line:

$$S\frac{\partial e_{\text{int}}}{\partial S} + n\frac{\partial e_{\text{int}}}{\partial n} = \frac{15}{16}\lambda S^2 - \frac{1}{16}\lambda n^2 = 2e_{\text{int}}.$$

Rule: insert the two derivatives and compare with twice e_{int}.

$$\frac{\langle L\rangle}{\sqrt{|g|}} = M_{\text{eff}}S + v_v n - mS - e_{\text{int}} = mS + 2e_{\text{int}} - mS - e_{\text{int}} = e_{\text{int}}.$$

Rule: the Lagrangian of Chapter 7 is a kinetic part minus mS minus the interaction, whose expectation value is e_{int}. On shell (for orbitals that solve the Kohn-Sham equation $\gamma^\mu D_\mu\Psi = (M_{\text{eff}} - iv_v\gamma^{(x_4)})\Psi$) the kinetic part is $\bar\Psi\gamma^\mu D_\mu\Psi = M_{\text{eff}}\bar\Psi\Psi + v_v\Psi^\dagger(-iC\gamma^{(x_4)})\Psi = M_{\text{eff}}S + v_v n$, because $\bar\Psi = \Psi^\dagger C$ and $B = -iC\gamma^{(x_4)}$ (the symmetrised form of the kinetic term has the same value on shell); then the line above gives $M_{\text{eff}}S + v_v n = mS + 2e_{\text{int}}$. This value of the Lagrangian enters the energy-momentum tensor (Chapter 15). Status: PROVED; check ks_onshell_lagrangian (both reports).

What the functional leaves out. The Kohn-Sham functional of the Revision theory is Hartree plus this uniform-gas exchange, with no correlation energy. Its states, however, are not uniform along y. The record computes the EXACT Fock exchange of such a state (exchange.exactFockSlab in ks-theory.json): take one level, closed over a shell ($\mathbf{k}$ and $-\mathbf{k}$ equally filled), with the 2×2 block density $D = \frac{1}{2}(d_0 + d_1\sigma_1 + d_2\sigma_2 + d_3\sigma_3)$ in each of the four blocks of type $j = +1$ and $\sigma_3 D\sigma_3$ in each of the four of type -1 (their degenerate partners, Section 14.5), and average it over the directions of the shell. Two facts make this average simple. First, averaging a matrix over all the rotations of a group gives a matrix that no rotation of the group changes, that is, one that commutes with all of them; and a matrix that already commutes with them is its own average. So the average over all rotations of 3-space is the projection onto the matrices that commute with the rotations, a space of dimension 64. Second, a closed shell contains only a finite set of directions, which the 24 rotations of a cube carry into each other, not all directions. But under the rotations of 3-space the 16×16 matrices split into parts of spin 0 (unchanged by every rotation) and parts of spin 1 (turned like an arrow), and nothing else; and the average of an arrow over the 24 rotations of a cube is zero, just as

its average over all rotations. So for these matrices the average over the cube equals the average over all rotations (both theory reports state this: the 16 x 16 matrices carry only spin 0 and spin 1). The result, with $Q = \langle \Psi^\dagger B \gamma^{(x_0)} \Psi \rangle$ and the y-current Y:

$$n = 8d_0, \qquad S = 8d_2, \qquad Q = 8d_3, \qquad Y = 8d_1, \qquad e_x^{\text{exact}} = -\frac{\lambda}{32}(n^2 + S^2 - Q^2 - Y^2).$$

For orbitals $Y = 0$ (Section 14.12), so the uniform-gas exchange $-\frac{\lambda}{32}(n^2 + S^2)$ omits the term $+\frac{\lambda}{32}Q^2$. The record labels this a DIAGNOSTIC: the canonical functional stays the uniform-gas one, and the Rust solver reports the omitted part and also solves an exact-Fock variant of the model, which adds the potential $w_Q \sigma_3$ with $w_Q = +\frac{\lambda}{16}Q$ to h and $+\frac{\lambda}{32}Q^2$ to e_{int}. Status: PROVED; check exchange_slab_exact_fock (both reports, by two different methods: the sympy report projects onto the 64-dimensional space of matrices that commute with the rotations, the Wolfram report averages over the 48 spinor matrices of the 24 rotations of a cube, each rotation appearing with both signs).

The size of Q. For one orbital $\chi = (a, ib)$ the three densities are proportional to $a^2 + b^2$ (n), $2ab$ (S, up to the sign j) and $a^2 - b^2$ (Q) (Section 14.12). Line by line:

$$(2ab)^2 + (a^2 - b^2)^2 = 4a^2b^2 + a^4 - 2a^2b^2 + b^4 = (a^2 + b^2)^2.$$

Rule: the binomial formulas. So for one orbital $S^2 + Q^2 = n^2$, and for a mixture of orbitals $S^2 + Q^2 \le n^2$ (Exercise 14.9): the possible states fill the disk $(S/n)^2 + (Q/n)^2 \le 1$. The uniform-gas and the exact exchange agree on the line $Q = 0$ and differ most at the top and bottom of the disk, $S = 0$, $Q = \pm n$ (Figure 14d.3).

The state of the eight zero modes ($N = 8$). The simplest Kohn-Sham state of the model fills the eight brane zero modes at $k = 0$ (both block types, even parity). Each orbital is $\chi = (a, 0)$ with $a = \sqrt{2M/(1 - e^{-2ML})}\, e^{My}$ (Section 14.13). The densities, line by line (the record's densities section in ks-theory.json):

- One orbital: $n_o = P(a^2 + b^2) = Pa^2$, $s_o = P\, j\, 2ab = 0$, $q_o = P(a^2 - b^2) = Pa^2$, with $P = e^{-6Hy}/(\ell^3 v_t)$, the factor that makes the densities proper (Section 14.7).
- The state: $n = \sum w\, g\, f\, n_o$, with the degeneracy $g = 4$ per block type, the occupation $f = 1$, and the weight $w = \frac{1}{2}$ of the ASSUMED Z2 doubled system: the orbitals are normalised on the patch, $\int_{-L}^{0} \chi^\dagger \chi \, dy = 1$, but the system consists of the patch and its mirror copy, so the patch holds half of each orbital, and the particle number $N = \sum gf$ counts the doubled system. So $n = \frac{1}{2}(4 + 4)Pa^2 = 4Pa^2$, $S = 0$ and $Q = n$.
- The proper density $n = 4e^{-6Hy}a^2/(\ell^3 v_t) \propto e^{(2M-6H)y}$ is largest at the cutoff $y = -L$ (for $M = H = 1$: e^{-4y}), although the orbital itself sits at the brane: the proper 7-volume e^{6Hy} is tiny near the tip. With $H = m = 1$, $L = 3$, $\ell = 2\pi/0.25$ and $v_t = 1$ its largest value is $n_{\max} = 82.2208638894$ (Notebook 14d, Out [12]; the column n_max of the row N8_lam0_a00 of `Revision/kohn_sham/results/ground/summary.csv`).
- To first order in λ the potentials are $M_{\text{eff}} - m = \frac{15}{16}\lambda S = 0$ and $v_v = -\frac{\lambda}{16}n$. The Revision solver calibrates its couplings by the largest first-order potential per unit λ, $n_{\max}/16 = 5.138803993$; λ_1 and λ_2 are 0.1 and 0.3 divided by it, rounded to 4 significant digits: $\lambda_1 = 0.01946$,

$\lambda_2 = 0.05838$, so that the largest first-order potential is $0.1\,m$ and $0.3\,m$ to four significant digits (after the rounding of λ it is $0.1000011\,m$ and $0.3000034\,m$, Notebook 14d, Out [12]; `Revision/kohn_sham/results/parameters.json`, couplingCalibration; the states $N = 136$ and 688 have their own, smaller couplings, fixed the same way along the whole history). This calibration is an $L = 3$ statement: the largest proper density sits at the cutoff and grows with L like $e^{(6H-2M)L}$, so the same rule applied at a larger cutoff drives λ_1 toward zero, for $N = 8$ like e^{-4L}, from 0.01946 at $L = 3$ to 1.2×10^{-7} at $L = 6$ (`Revision/kohn_sham/tip_convergence/`; `Revision/docs/KOHN_SHAM_DEFLATING_FIELD.md`, section 5); with the couplings held fixed, the interaction energy of the zero modes depends strongly on L (Section 14.20).

- The exact Fock exchange of this state vanishes: with $S = 0$ and $Q = n$, $e_x^{\text{exact}} = -\frac{\lambda}{32}(n^2 - Q^2) = 0$. In the exact-Fock variant the potential on $(a, 0)$ is $v_v + w_Q = \frac{\lambda}{16}(Q - n) = 0$ (because σ_3 acts as $+1$ on $(a, 0)$), so the zero modes stay exact solutions with $\varepsilon = 0$ and the energy of the state is exactly zero. The record's twenty self-consistent exact-Fock states with $N = 8$ (four couplings, five slices) have energies zero to rounding, below 10^{-12} (`Revision/kohn_sham/results/exx/exact-fock-variant.csv`).

Status: the formulas PROVED; the numbers COMPUTED (they reproduce the records named); the weight $w = \frac{1}{2}$ rests on the ASSUMED Z2 mirror.

14.28 Example: Notebook 14d, the exact local exchange

Notebook 14d does Sections 14.26 and 14.27 with exact algebra on the author's gamma matrices: the plane waves of the good sector and their projector, the cancellation that makes $\rho = (nB + SC)/16$, Wick's rule term by term on two explicit states, the exact local exchange and the ratio $-1/8$, the Kohn-Sham potentials and the on-shell Lagrangian (compared with the coefficients that the Rust solver read), and the exact Fock exchange of the closed-shell states by projection onto the 64-dimensional space of matrices that commute with the rotations. With numpy it computes the densities of the eight zero modes, reproduces the largest proper density, the calibration of the couplings and the zero exact-Fock energies of the record. It draws four figures, needs no Rust, runs in about 30 seconds, and its last line is ALL 16 CHECKS PASSED (notebook 14d).

14.29 How to run Notebook 14d

Step 1. What this notebook does and what it needs.

Notebook 14d (The exact local exchange of the contact interaction and the Kohn-Sham potentials) is the file `Revision/textbook/notebooks/14d_local_exchange.ipynb` of the repository Dirac_claude. It derives with exact algebra on the author's gamma matrices the Hartree-Fock energy of the contact interaction U = (lambda/2) S^2 of dirac16complex in Kohn-Sham states: the positive-energy projector of the flat good-sector plane waves, the cancellation between the momenta p and -p that makes the one-body matrix of every symmetric occupation equal to (nB + SC)/16, the bookkeeping of Wick's rule checked term by term, the exact local exchange -(lambda/32)(n^2 + S^2), the filled-shell ratio -1/8, the Kohn-Sham potentials M_eff = m + (15/16) lambda S and v_v = -lambda n/16, and the exact Fock exchange of the closed-shell slab, which differs by +(lambda/32) Q^2; it then computes the densities of the eight brane zero modes and reproduces the Revision calibration of the coupling for N = 8 and the vanishing exact-Fock energy of that state, with four

teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 14d_local_exchange.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 14d_local_exchange.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 14d_local_exchange.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/14d.captions.json`
- `Revision/textbook/figures/14d_1_pair_average.png`
- `Revision/textbook/figures/14d_2_energy_densities.png`
- `Revision/textbook/figures/14d_3_exact_fock_ratio.png`
- `Revision/textbook/figures/14d_4_zero_mode_state.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all four figure files of this notebook exist
ALL 16 CHECKS PASSED (notebook 14d)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/14d_local_exchange.ipynb` inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 14d_local_exchange.ipynb
python -m nbconvert --execute --inplace 14d_local_exchange.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

14.30 Notebook 14d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/14d_local_exchange.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The quanta of dirac16complex interact through the contact term
$U = \frac\lambda2 S^2$ of the Lagrangian, $S = \bar\Psi\Psi$. The Kohn-Sham
model needs the energy of this interaction in a Kohn-Sham state (a Slater
determinant or a thermal mixture of orbitals) as a function of the densities.
This notebook derives it exactly, with the author's gamma matrices:
```

```
- the plane waves of the good sector in flat space, their energies $\pm E$ and
  the projector on the positive ones;
- why every occupation that is symmetric under $\mathbf p \to -\mathbf p$ has
  the one-body matrix $\rho = (nB + SC)/16$ (figure 1);
- the Hartree-Fock energy: Hartree $\frac\lambda2S^2$ plus the EXACT LOCAL
  EXCHANGE $-\frac{\lambda}{32}(n^2 + S^2)$, and the ratio $-1/8$ of a filled
  level at rest (figure 2);
- the Kohn-Sham potentials $M_{\rm eff} = m + \frac{15}{16}\lambda S$ and
  $v_v = -\frac{\lambda}{16}n$;
- the exact Fock exchange of a closed-shell slab state, $-\frac{\lambda}{32}
  (n^2 + S^2 - Q^2)$, which the uniform-gas formula misses by
  $+\frac{\lambda}{32}Q^2$ (figure 3);
- the densities and potentials of the eight brane zero modes (the state
  $N = 8$), which reproduce the Revision calibration of the coupling, and why the
  exact Fock exchange of that state is exactly zero (figure 4).

It prints a PASS line for every check (16 in all) and saves four figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 14d (The exact local exchange of the contact interaction and the Kohn-Sham
potentials) is the file ‘Revision/textbook/notebooks/14d_local_exchange.ipynb‘ of the
repository Dirac_claude. It derives with exact algebra on the author’s gamma matrices the
Hartree-Fock energy of the contact interaction U = (lambda/2) S^2 of dirac16complex in
Kohn-Sham states: the positive-energy projector of the flat good-sector plane waves, the
cancellation between the momenta p and -p that makes the one-body matrix of every
symmetric occupation equal to (nB + SC)/16, the bookkeeping of Wick’s rule checked term
by term, the exact local exchange -(lambda/32)(n^2 + S^2), the filled-shell ratio -1/8,
the Kohn-Sham potentials M_eff = m + (15/16) lambda S and v_v = -lambda n/16, and the
exact Fock exchange of the closed-shell slab, which differs by +(lambda/32) Q^2; it then
computes the densities of the eight brane zero modes and reproduces the Revision
calibration of the coupling for N = 8 and the vanishing exact-Fock energy of that state,
with four teaching figures. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
notebooks were built with Python 3.14.5) with these packages at exactly these versions:
numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
```

default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
```

```
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 14d_local_exchange.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 14d_local_exchange.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 14d_local_exchange.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/14d.captions.json‘
- ‘Revision/textbook/figures/14d_1_pair_average.png‘
- ‘Revision/textbook/figures/14d_2_energy_densities.png‘
- ‘Revision/textbook/figures/14d_3_exact_fock_ratio.png‘
- ‘Revision/textbook/figures/14d_4_zero_mode_state.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every

change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS all four figure files of this notebook exist
ALL 16 CHECKS PASSED (notebook 14d)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/14d_local_exchange.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 14d_local_exchange.ipynb
  python -m nbconvert --execute --inplace 14d_local_exchange.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 14d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 14d (The exact local exchange of the contact interaction and the Kohn-Sham
# potentials) is the file Revision/textbook/notebooks/14d_local_exchange.ipynb of the
# repository Dirac_claude. It derives with exact algebra on the author's gamma matrices
# the Hartree-Fock energy of the contact interaction U = (lambda/2) S^2 of dirac16complex
# in Kohn-Sham states: the positive-energy projector of the flat good-sector plane waves,
# the cancellation between the momenta p and -p that makes the one-body matrix of every
# symmetric occupation equal to (nB + SC)/16, the bookkeeping of Wick's rule checked term
# by term, the exact local exchange -(lambda/32)(n^2 + S^2), the filled-shell ratio -1/8,
# the Kohn-Sham potentials M_eff = m + (15/16) lambda S and v_v = -lambda n/16, and the
# exact Fock exchange of the closed-shell slab, which differs by +(lambda/32) Q^2; it
# then computes the densities of the eight brane zero modes and reproduces the Revision
# calibration of the coupling for N = 8 and the vanishing exact-Fock energy of that
# state, with four teaching figures. It needs a computer with Windows 11, macOS or Linux,
# an internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
```

```
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
```

```
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 14d_local_exchange.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
```

```
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 14d_local_exchange.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 14d_local_exchange.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/14d.captions.json
# - Revision/textbook/figures/14d_1_pair_average.png
# - Revision/textbook/figures/14d_2_energy_densities.png
# - Revision/textbook/figures/14d_3_exact_fock_ratio.png
# - Revision/textbook/figures/14d_4_zero_mode_state.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS all four figure files of this notebook exist
#     ALL 16 CHECKS PASSED (notebook 14d)
#
# and the notebook must show 4 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
```

```
#   Revision/textbook/notebooks/14d_local_exchange.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 14d_local_exchange.ipynb
#     python -m nbconvert --execute --inplace 14d_local_exchange.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "14d"  # this notebook: chapter 14, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
```

```
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
```

```
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 14d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Contact interaction**: the term $U(S) = \frac\lambda2S^2$ of the
  Lagrangian; $\lambda > 0$ is repulsive. $\lambda$ has the dimension
  mass$^{-6}$.
```

```
- **Density**: an expectation value per unit volume. Number density
  $n = \langle\Psi^\dagger B\Psi\rangle$; scalar density
  $S = \langle\bar\Psi\Psi\rangle = \langle\Psi^\dagger C\Psi\rangle$; the
  density $Q = \langle\Psi^\dagger B\gamma^{(x_0)}\Psi\rangle$. PROPER densities
  are per proper 7-volume.
- **Expectation-value rule**: for one quantum in the mode $u$,
  $\langle\Psi^\dagger X\Psi\rangle = u^\dagger BXu$; for many,
  $\langle\Psi^\dagger X\Psi\rangle = \mathrm{Tr}(X\rho)$ with the one-body
  matrix $\rho = \sum_{\rm occupied} f\,uu^\dagger B$ ($f$ the occupation).
- **Trace** $\mathrm{Tr}\,X$: the sum of the diagonal entries; $\mathrm{Tr}(XY)
  = \mathrm{Tr}(YX)$.
- **Normal ordering** $:\;:$: the operators of the free sea are reordered so
  that the vacuum has zero energy; it removes the infinite sea contributions.
- **Quasi-free state, Wick's rule**: a Slater determinant or a thermal state of
  independent quanta; the expectation of four field operators is a sum of
  products of two-operator expectations (a "direct" and an "exchange" term).
- **Hartree term, exchange term**: the direct and the exchange part of the
  interaction energy. **Hartree-Fock**: keeping both, for a quasi-free state.
- **Kohn-Sham potential**: the derivative of the interaction energy density with
  respect to a density; it enters the one-quantum equation.
- **Uniform gas**: quanta in flat space, the same in every place.
- **Closed-shell slab**: the Kohn-Sham states of the model, uniform along 3-space
  but not along the hidden coordinate $y$, with complete lattice shells filled.
- **Commutant**: all matrices that commute with a given set of matrices.
- **Zero modes, $N = 8$**: the eight brane-bound orbitals at zero 3-momentum,
  $\chi = (a, 0)$ with $a \propto e^{My}$; filling them gives the state $N = 8$.
```

Text cell:

```
## 4. The physical and mathematical situation

The Revision theory (‘Revision/kohn_sham/ks-theory.json‘, section ‘exchange‘)
builds the Kohn-Sham functional of dirac16complex from the Hartree-Fock energy of
the uniform good-sector gas, which turns out to be LOCAL (a function of the
densities at one point) at every temperature. The steps, each exact:

1. In flat space a plane wave $e^{-i\varepsilon x_4 + i\mathbf p\cdot\mathbf x}$
   of the good sector solves $h_{\mathbf p}u = \varepsilon u$ with
   $h_{\mathbf p} = M(-i\gamma^{(x_4)}) - p_a\gamma^{(x_4)}\gamma^{(a)}$, the sum
   over the four space-like directions $x_0, x_1, x_2, x_3$.
2. $h_{\mathbf p}^2 = (M^2 + p^2)\,1$: the energies are $\pm E$,
   $E = \sqrt{M^2 + p^2}$, each 8 times.
3. The interaction energy needs $\langle{:}S^2{:}\rangle$, which Wick's rule
   turns into $(\mathrm{Tr}\,C\rho)^2 - \mathrm{Tr}(C\rho C\rho)$.
4. For symmetric occupations $\rho = (nB + SC)/16$, and the second trace is
   $(n^2 + S^2)/16$.

Status: every formula here is PROVED (exact; verified twice in the Revision
record). The use of the uniform-gas exchange for the slab states of the model is
the canonical functional of the Revision theory (an approximation: no
correlation energy); the exact Fock exchange of the slab is a DIAGNOSTIC.
```

Text cell:

```
## 5. The gamma matrices, B and C

The next cell reads the author's gamma matrices from the Revision record
`Revision/algebra/gammas.json` (`g0` is $\gamma^{(x_0)}$, ..., `g7` is
$\gamma^{(x_7)}$), builds $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}
\gamma^{(x_3)}$ and $B = -iC\gamma^{(x_4)}$, and checks the facts used below:
$B$ and $C$ square to 1, $\mathrm{Tr}\,B^2 = \mathrm{Tr}\,C^2 = 16$,
$\mathrm{Tr}\,BC = 0$, $CBC = B$, and $(-i\gamma^{(x_4)})B = C$. It also defines
`record_check`, which reads the verdict of a named check from the two Revision
theory reports.
```

In [2]:

```
import math  # exp, sqrt, log10 of single numbers

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra with symbols

PY_REPORT = "Revision/kohn_sham/reports/ks-theory-python.json"
WL_REPORT = "Revision/kohn_sham/reports/ks-theory-wolfram.json"


def record_check(name):
    """PASS in the sympy record and, if the WolframScript record has a check of
    that name, PASS there too."""
    verdicts = []
    for report_file in (PY_REPORT, WL_REPORT):
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        found = [c["verdict"] for c in data["checks"] if c["name"] == name]
        verdicts.append(found[0] if found else "absent")
    return verdicts[0] == "PASS" and verdicts[1] in ("PASS", "absent")


fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
g = [sp.Matrix(m) for m in fixture["gamma"]]
g0, g1, g2, g3, g4, g5, g6, g7 = g  # gN is gamma^(xN)
I16, Z16 = sp.eye(16), sp.zeros(16)
C = g0 * g1 * g2 * g3
B = -sp.I * C * g4
facts_ok = (B * B == I16 and C * C == I16 and (B * B).trace() == 16
            and (C * C).trace() == 16 and (B * C).trace() == 0 and C * B * C == B
            and (-sp.I * g4) * B == C)
check(facts_ok, "B^2 = C^2 = 1, Tr B^2 = Tr C^2 = 16, Tr BC = 0, CBC = B, (-i g4) B = C")
```

Out [2]:

```
PASS B^2 = C^2 = 1, Tr B^2 = Tr C^2 = 16, Tr BC = 0, CBC = B, (-i g4) B = C
```

Text cell:

```
## 6. The plane waves of the good sector and their projector

The four space-like directions are $x_0, x_1, x_2, x_3$; their gammas square to
$+1$, and $\gamma^{(x_4)}$ squares to $-1$. Line by line:

1. $h_{\mathbf p} = M(-i\gamma^{(x_4)}) - \sum_a p_a\gamma^{(x_4)}\gamma^{(a)}$.
2. Squaring: $(-i\gamma^{(x_4)})^2 = -(\gamma^{(x_4)})^2 = 1$;
   $(\gamma^{(x_4)}\gamma^{(a)})^2 = -(\gamma^{(x_4)})^2(\gamma^{(a)})^2 = 1$;
   all mixed products anticommute and cancel in pairs. So
   $h_{\mathbf p}^2 = (M^2 + p_1^2 + p_2^2 + p_3^2 + p_0^2)\,1 = E^2\,1$.
3. Hence $G_{\mathbf p} = \frac12(1 + h_{\mathbf p}/E)$ satisfies
   $G_{\mathbf p}^2 = G_{\mathbf p}$: it projects on the energy $+E$; its trace
   is 8 because $h_{\mathbf p}$ is traceless: eight positive-energy modes.

The next cell checks the three lines with sympy.
```

In [3]:

```
M = sp.symbols("M", positive=True)  # the mass
p1, p2, p3, p0 = sp.symbols("p1 p2 p3 p0", real=True)  # the momentum
E = sp.symbols("E", positive=True)  # stands for sqrt(M^2 + p^2)
h_p = M * (-sp.I * g4) - (p1 * g4 * g1 + p2 * g4 * g2 + p3 * g4 * g3 + p0 * g4 * g0)
E_squared = M ** 2 + p1 ** 2 + p2 ** 2 + p3 ** 2 + p0 ** 2
square_ok = (h_p * h_p - E_squared * I16).applyfunc(sp.expand).is_zero_matrix
G_p = (I16 + h_p / E) / 2  # the projector on the positive energy
projector_ok = ((G_p * G_p - G_p).subs(E, sp.sqrt(E_squared))
                .applyfunc(sp.simplify).is_zero_matrix
                and G_p.H == G_p and sp.simplify(G_p.trace()) == 8)
check(square_ok and projector_ok and record_check("gas_mode_projector"),
      "h_p^2 = (M^2 + p^2) 1 and G_p = (1 + h_p/E)/2 is a projector of rank 8",
      record=f"{PY_REPORT}, check gas_mode_projector")
```

Out [3]:

```
PASS h_p^2 = (M^2 + p^2) 1 and G_p = (1 + h_p/E)/2 is a projector of rank 8
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    gas_mode_projector
```

Text cell:

```
## 7. Why every symmetric occupation gives rho = (nB + SC)/16

The one-body matrix of the eight positive-energy modes at momentum $\mathbf p$
is $G_{\mathbf p}B$. Split it into a part that does not depend on the direction
of $\mathbf p$ and a part that is linear in $\mathbf p$:

$$G_{\mathbf p}B = \frac12\left(B + \frac ME(-i\gamma^{(x_4)})B\right) -
\frac1{2E}\sum_a p_a\gamma^{(x_4)}\gamma^{(a)}B .$$

The linear part changes sign under $\mathbf p \to -\mathbf p$. So if the
occupation is the same for $\mathbf p$ and $-\mathbf p$ (true for every closed
```

```
shell and for the thermal states), the linear parts cancel in pairs, and with
$(-i\gamma^{(x_4)})B = C$ what is left is $\frac12(B + \frac ME C)$. The same
holds for an average over all directions (the average of each component of a
direction over the sphere is 0). For the negative-energy modes,
$\frac12(1 - h_{\mathbf p}/E)B$, the result is $\frac12(B - \frac ME C)$. Adding
up any such occupation therefore gives a combination of $B$ and $C$ only,
$\rho = \alpha B + \beta C$; and since $\mathrm{Tr}(B\rho) = 16\alpha = n$ and
$\mathrm{Tr}(C\rho) = 16\beta = S$,

$$\rho = \frac{nB + SC}{16}.$$

The next cell checks the pair cancellation, the sphere averages of the three
components (exact integrals), both mode types and the trace formulas.
```

In [4]:

```
p_flip = {p1: -p1, p2: -p2, p3: -p3, p0: -p0}
pair_average = ((G_p * B + (G_p * B).subs(p_flip)) / 2).applyfunc(sp.simplify)
pair_ok = (pair_average - (B + M / E * C) / 2).applyfunc(sp.simplify).is_zero_matrix
th, ph = sp.symbols("vartheta varphi", real=True)  # angles on the sphere
components = [sp.sin(th) * sp.cos(ph), sp.sin(th) * sp.sin(ph), sp.cos(th)]
sphere = [sp.integrate(sp.integrate(c * sp.sin(th), (ph, 0, 2 * sp.pi)), (th, 0, sp.pi))
          / (4 * sp.pi) for c in components]  # the averages of the components
say(f"sphere averages of the three direction components: {sphere}")
check(pair_ok and sphere == [0, 0, 0] and record_check("gas_angular_average"),
      "the average over p and -p (or over all directions) is (B + (M/E) C)/2",
      record=f"{PY_REPORT}, check gas_angular_average")
h_0 = M * (-sp.I * g4)  # the p-independent part of h_p
negative_ok = (((I16 - h_0 / E) / 2) * B - (B - M / E * C) / 2).is_zero_matrix
check(negative_ok and record_check("gas_negative_energy_modes"),
      "negative-energy modes give (B - (M/E) C)/2: only B and C ever appear",
      record=f"{PY_REPORT}, check gas_negative_energy_modes")
n, S, lam = sp.symbols("n S lambda", real=True)
rho = (n * B + S * C) / 16  # the one-body matrix of a symmetric occupation
check(sp.expand((B * rho).trace()) == n and sp.expand((C * rho).trace()) == S
      and record_check("gas_densities"),
      "rho = (nB + SC)/16 has Tr(B rho) = n and Tr(C rho) = S",
      record=f"{PY_REPORT}, check gas_densities")
```

Out [4]:

```
sphere averages of the three direction components: [0, 0, 0]
PASS the average over p and -p (or over all directions) is (B + (M/E) C)/2
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    gas_angular_average
PASS negative-energy modes give (B - (M/E) C)/2: only B and C ever appear
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    gas_negative_energy_modes
PASS rho = (nB + SC)/16 has Tr(B rho) = n and Tr(C rho) = S
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check gas_densities
```

Text cell:

```
The next cell draws the cancellation for $M = 1$, $\mathbf p = (1, 0, 0, 0)$
(so $E = \sqrt2$): the absolute values of the entries of the one-body matrix
$G_{\mathbf p}B$ of one momentum (left), of its average with $-\mathbf p$
(middle), which is exactly $\frac12(B + C/\sqrt2)$, and of the part linear in
$\mathbf p$ (right), which cancels between $\mathbf p$ and $-\mathbf p$.
```

In [5]:

```
values = {M: 1, E: sp.sqrt(2), p1: 1, p2: 0, p3: 0, p0: 0}
one_mode = np.abs(np.array((G_p * B).subs(values).evalf(), dtype=complex))
averaged = np.abs(np.array(pair_average.subs(values).evalf(), dtype=complex))
odd_part = np.abs(np.array((G_p * B - pair_average).subs(values).evalf(),
                           dtype=complex))
fig, axes = plt.subplots(1, 3, figsize=(11.5, 3.9))
fig.subplots_adjust(wspace=0.45)
titles = ["$|G_{\\mathbf{p}}B|$: one momentum",
          "$|(G_{\\mathbf{p}} + G_{-\\mathbf{p}})B/2|$",
          "$|$part linear in $\\mathbf{p}|$"]
for ax, data, title in zip(axes, (one_mode, averaged, odd_part), titles):
    image = ax.imshow(data, cmap="Blues", vmin=0.0, vmax=0.6,
                      interpolation="nearest")
    ax.set_title(title, fontsize=9)
    ax.set_xticks([0, 5, 10, 15])
    ax.set_yticks([0, 5, 10, 15])
    ax.set_xlabel("column")
    ax.set_ylabel("row")
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.75)
save_figure(fig, "pair_average",
            "Absolute values of the entries (rows and columns 0 to 15, darker is "
            "larger) of the one-body matrix of the eight positive-energy plane "
            "waves for $M = 1$ and the momentum $\\mathbf{p} = (1, 0, 0, 0)$, "
            "$E = \\sqrt{2}$: left for one momentum, middle averaged with "
            "$-\\mathbf{p}$, which leaves exactly $(B + C/\\sqrt{2})/2$, a "
            "combination of $B$ and $C$ only; right the part linear in "
            "$\\mathbf{p}$, which cancels between $\\mathbf{p}$ and $-\\mathbf{p}$. "
            "Every occupation that is symmetric under $\\mathbf{p} \\to "
            "-\\mathbf{p}$ therefore has the one-body matrix $(nB + SC)/16$.")
```

Out [5]:

```
Figure 14d.1 saved as Revision/textbook/figures/14d_1_pair_average.png
```

Text cell:

```
## 8. Wick's rule for the contact term

For a quasi-free state with the two-operator expectations
$f_{AB} = \langle\Psi_A^\dagger\Psi_B\rangle = \rho_{BA}$ (the expectation-value
rule), Wick's rule gives for the normal-ordered square of
$S = \Psi^\dagger C\Psi = \sum_{AB}\Psi_A^\dagger C_{AB}\Psi_B$
```

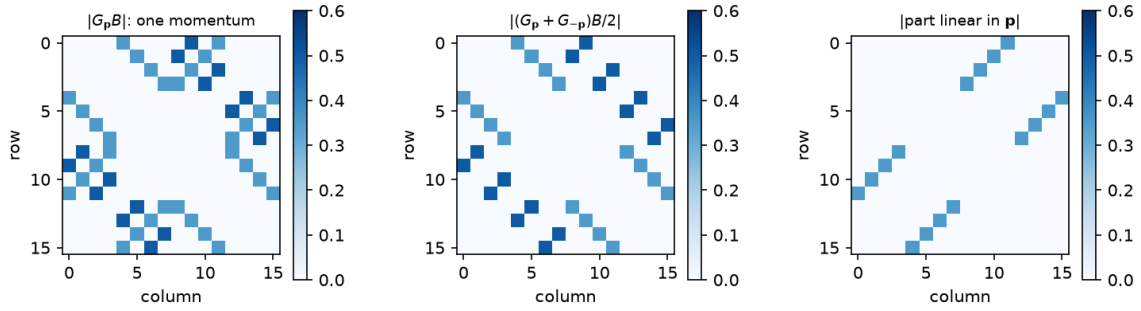

Figure 401: Absolute values of the entries (rows and columns 0 to 15, darker is larger) of the one-body matrix of the eight positive-energy plane waves for $M = 1$ and the momentum $\mathbf{p} = (1, 0, 0, 0)$, $E = \sqrt{2}$: left for one momentum, middle averaged with $-\mathbf{p}$, which leaves exactly $(B + C/\sqrt{2})/2$, a combination of B and C only; right the part linear in $\mathbf{p}$, which cancels between $\mathbf{p}$ and $-\mathbf{p}$. Every occupation that is symmetric under $\mathbf{p} \to -\mathbf{p}$ therefore has the one-body matrix $(nB + SC)/16$. (Notebook 14d, figure 1.)

```
$$\langle{:}S^2{:}\rangle = \sum_{ABCD}C_{AB}C_{CD}\,(f_{AB}f_{CD} -
f_{AD}f_{CB}) .$$

The first product is the DIRECT (Hartree) term, the second, with the minus sign
of the anticommuting fields, the EXCHANGE (Fock) term. In matrix language the
first sum is $(\mathrm{Tr}\,C\rho)^2$ and the second is $\mathrm{Tr}(C\rho C
\rho)$. The next cell checks this bookkeeping term by term, as the Revision
record did, for the explicit two-mode state with the mode vectors $u_1, u_2$ of
the record, and for a second two-mode state $v_1, v_2$ with less symmetry: it
adds up the $16^4$ terms of the sum (only those with nonzero $C$ entries are
kept, 256 of them) and compares with the two traces.
```

In [6]:

```
half, i_half = sp.Rational(1, 2), sp.I / 2
u1 = sp.Matrix([half, i_half, 0, 0, 0, 0, 0, 0, half, 0, 0, 0, 0, 0, -i_half, 0])
u2 = sp.Matrix([0, 0, half, 0, -half, 0, 0, 0, 0, 0, i_half, 0, 0, 0, 0, i_half])
v1 = sp.Matrix([1, 2, 0, -1, sp.I, 0, 3, 1, 0, 1, -2, sp.I, 0, 1, 1, 0]) / 4
v2 = sp.Matrix([0, 1, sp.I, 2, 1, -1, 0, 0, 1, 0, sp.I, 1, 2, 0, -1, 1]) / 4
nonzero = [(A_, B_) for A_ in range(16) for B_ in range(16) if C[A_, B_] != 0]
wick_ok = True
for name, modes in (("u1, u2 (record)", (u1, u2)), ("v1, v2", (v1, v2))):
    rho_2 = (modes[0] * modes[0].H + modes[1] * modes[1].H) * B  # one-body matrix
    f = rho_2.T  # f_AB = rho_BA
    brute = sp.simplify(sum(
        C[A_, B_] * C[C_, D_] * (f[A_, B_] * f[C_, D_] - f[A_, D_] * f[C_, B_])
        for A_, B_ in nonzero for C_, D_ in nonzero))
    direct = sp.simplify((C * rho_2).trace() ** 2)
    exchange = sp.simplify((C * rho_2 * C * rho_2).trace())
    say(f"state {name}: (Tr C rho)^2 = {direct}, Tr(C rho C rho) = {exchange}, "
        f"term-by-term sum = {brute}")
    wick_ok &= sp.simplify(brute - (direct - exchange)) == 0
```

```
check(len(nonzero) == 16 and wick_ok and record_check("hf_wick_contraction"),
      "<:S^2:> = (Tr C rho)^2 - Tr(C rho C rho), checked term by term",
      record=f"{PY_REPORT}, check hf_wick_contraction")
```

Out [6]:

```
state u1, u2 (record): (Tr C rho)^2 = 1/4, Tr(C rho C rho) = 1/4, term-by-term sum = 0
state v1, v2: (Tr C rho)^2 = 25/64, Tr(C rho C rho) = 9/32, term-by-term sum = 7/64
PASS <:S^2:> = (Tr C rho)^2 - Tr(C rho C rho), checked term by term
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    hf_wick_contraction
```

Text cell:

```
## 9. The exact local exchange and the filled-shell ratio

With $\rho = (nB + SC)/16$ (line by line, using $CBC = B$, $C^2 = 1$,
$\mathrm{Tr}\,B^2 = \mathrm{Tr}\,C^2 = 16$, $\mathrm{Tr}\,BC = 0$):

1. $C\rho = (nCB + S)/16$.
2. $(C\rho)^2 = (n^2CBCB + nS\,CB + nS\,CB + S^2)/256 = (n^2 + 2nS\,CB +
   S^2)/256$, because $CBCB = BB = 1$.
3. $\mathrm{Tr}(C\rho C\rho) = (16n^2 + 0 + 16S^2)/256 = (n^2 + S^2)/16$.
4. The energy density is $\frac\lambda2\langle{:}S^2{:}\rangle = e_H + e_x$ with
   the Hartree term $e_H = \frac\lambda2S^2$ and the EXCHANGE
   $e_x = -\frac\lambda2\mathrm{Tr}(C\rho C\rho) = -\frac{\lambda}{32}(n^2 + S^2)$.

This is exact, at every temperature, for every symmetric occupation: the exchange
of the contact interaction is LOCAL. For one filled 8-fold level at rest
($n = S = 8$ per unit volume): $e_x/e_H = -\frac{1}{32}\cdot128/(\frac12\cdot64)
= -\frac18$. The next cell checks 1 to 4 and the ratio.
```

In [7]:

```
e_x = sp.factor(-lam / 2 * (C * rho * C * rho).trace())  # the exchange energy density
say(f"e_x = -(lambda/2) Tr(C rho C rho) = {e_x}")
check(sp.simplify(e_x + lam / 32 * (n ** 2 + S ** 2)) == 0
      and record_check("exchange_uniform_gas"),
      "the exact local exchange e_x = -(lambda/32)(n^2 + S^2)",
      record=f"{PY_REPORT}, check exchange_uniform_gas")
e_H = lam / 2 * S ** 2  # the Hartree energy density
ratio = sp.simplify((e_x / e_H).subs({n: 8, S: 8}))
say(f"one filled level at rest (n = S = 8): e_x/e_H = {ratio}")
check(ratio == sp.Rational(-1, 8) and record_check("filled_shell_ratio"),
      "a filled 8-fold level at rest has e_x/e_H = -1/8",
      record=f"{PY_REPORT}, check filled_shell_ratio")
```

Out [7]:

```
e_x = -(lambda/2) Tr(C rho C rho) = -lambda*(S**2 + n**2)/32
PASS the exact local exchange e_x = -(lambda/32)(n^2 + S^2)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    exchange_uniform_gas
one filled level at rest (n = S = 8): e_x/e_H = -1/8
PASS a filled 8-fold level at rest has e_x/e_H = -1/8
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    filled_shell_ratio
```

Text cell:

```
## 10. The Kohn-Sham potentials

The interaction energy density is $e_{\rm int}(n, S) = e_H + e_x =
\frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2$. The Kohn-Sham potentials are
its derivatives: $M_{\rm eff} = m + \partial e_{\rm int}/\partial S = m +
\frac{15}{16}\lambda S$ (it shifts the mass) and $v_v = \partial e_{\rm int}/
\partial n = -\frac{1}{16}\lambda n$ (it enters $h$ as $v_v$ times the unit
matrix). Because $e_{\rm int}$ is homogeneous of degree 2,
$S\,\partial_S e + n\,\partial_n e = 2e$, so on shell
$\langle L\rangle/\sqrt{|g|} = M_{\rm eff}S + v_vn - mS - e_{\rm int} =
e_{\rm int}$. The next cell checks this and compares the coefficients with the
Revision record (ks-theory.json) and with the numbers the Rust solver read
(parameters.json: 0.9375, -0.0625, -0.03125, -0.03125).
```

In [8]:

```
m = sp.symbols("m", real=True)
e_int = sp.expand(e_H + e_x)
M_eff = m + sp.diff(e_int, S)  # the effective mass
v_v = sp.diff(e_int, n)  # the vector potential
say(f"e_int = {e_int}")
say(f"M_eff = {M_eff},  v_v = {v_v}")
check(sp.simplify(M_eff - m - sp.Rational(15, 16) * lam * S) == 0
      and sp.simplify(v_v + lam * n / 16) == 0 and record_check("ks_potentials"),
      "M_eff = m + (15/16) lambda S and v_v = -lambda n/16",
      record=f"{PY_REPORT}, check ks_potentials")
on_shell = sp.simplify(M_eff * S + v_v * n - m * S - e_int - e_int)
check(on_shell == 0 and record_check("ks_onshell_lagrangian"),
      "on shell <L>/sqrt|g| = M_eff S + v_v n - m S - e_int = e_int",
      record=f"{PY_REPORT}, check ks_onshell_lagrangian")
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
potentials = theory["exchange"]["kohnShamPotentials"]
gas = theory["exchange"]["uniformGas"]
inputs = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))["theoryInputs"]
same = (sp.Rational(potentials["Meff_coefficient_of_lambda_S"]) == sp.Rational(15, 16)
        and sp.Rational(potentials["vv_coefficient_of_lambda_n"]) == sp.Rational(-1, 16)
        and sp.Rational(gas["coefficient_n2"]) == sp.Rational(-1, 32)
        and sp.Rational(gas["coefficient_S2"]) == sp.Rational(-1, 32)
        and inputs["MeffCoefficientOfLambdaS"] == 0.9375
```

```
        and inputs["vvCoefficientOfLambdaN"] == -0.0625
        and inputs["exchangeCoefficientN2"] == -0.03125
        and inputs["exchangeCoefficientS2"] == -0.03125)
check(same and record_check("ks_theory_json_exchange"),
      "the coefficients 15/16, -1/16, -1/32, -1/32 equal the records",
      record="Revision/kohn_sham/ks-theory.json, exchange; "
             "Revision/kohn_sham/results/parameters.json, theoryInputs")
```

Out [8]:

```
e_int = 15*S**2*lambda/32 - lambda*n**2/32
M_eff = 15*S*lambda/16 + m,  v_v = -lambda*n/16
PASS M_eff = m + (15/16) lambda S and v_v = -lambda n/16
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check ks_potentials
PASS on shell <L>/sqrt|g| = M_eff S + v_v n - m S - e_int = e_int
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    ks_onshell_lagrangian
PASS the coefficients 15/16, -1/16, -1/32, -1/32 equal the records
     reproduces Revision/kohn_sham/ks-theory.json, exchange;
    Revision/kohn_sham/results/parameters.json, theoryInputs
```

Text cell:

```
The next cell draws the three energy densities per unit $\lambda n^2$ against
the ratio $S/n$ (between $-1$ and $1$, where the physical states lie): the
Hartree term $\frac12(S/n)^2$, the exchange $-\frac1{32}(1 + (S/n)^2)$ and their
sum $e_{\rm int}$. The exchange is small and negative; at $S = n$ (a filled level
at rest) it is $-1/8$ of the Hartree term; near $S = 0$ only the exchange is
left.
```

In [9]:

```
s_over_n = np.linspace(-1.0, 1.0, 401)
hartree = 0.5 * s_over_n ** 2
exchange_curve = -(1.0 + s_over_n ** 2) / 32.0
fig, ax = plt.subplots(figsize=(7.5, 4.4))
ax.plot(s_over_n, hartree, label="Hartree $e_H/(\\lambda n^2) = (S/n)^2/2$")
ax.plot(s_over_n, exchange_curve, "--",
        label="exchange $e_x/(\\lambda n^2) = -(1 + (S/n)^2)/32$")
ax.plot(s_over_n, hartree + exchange_curve, ":", color="black",
        label="$e_{\\rm int} = e_H + e_x$")
ax.plot([1.0], [-1.0 / 16.0], "o", color="C3")
ax.annotate("$S = n$: $e_x = -e_H/8$", xy=(1.0, -1.0 / 16.0), xytext=(-0.05, 0.3),
            fontsize=8, color="C3", arrowprops={"arrowstyle": "->", "color": "C3"})
ax.axhline(0.0, color="gray", linewidth=0.6)
ax.set_xlabel("ratio of the scalar to the number density $S/n$")
ax.set_ylabel("energy density per $\\lambda n^2$")
ax.set_title("the Hartree-Fock energy of the contact interaction")
ax.legend(fontsize=8, loc="upper center")
save_figure(fig, "energy_densities",
            "The interaction energy densities of the contact term per unit "
```

```
"$\\lambda n^2$ against the ratio $S/n$ of the scalar and the number "
"density: the Hartree term $(S/n)^2/2$ (solid), the exact local "
"exchange $-(1 + (S/n)^2)/32$ (dashed) and their sum $e_{\\rm int}$ "
"(dotted). For $\\lambda > 0$ the exchange lowers the energy a little; "
"at $S = n$ (a filled level at rest, red dot) it is $-1/8$ of the "
"Hartree term, and near $S = 0$ it is all that is left.")
```

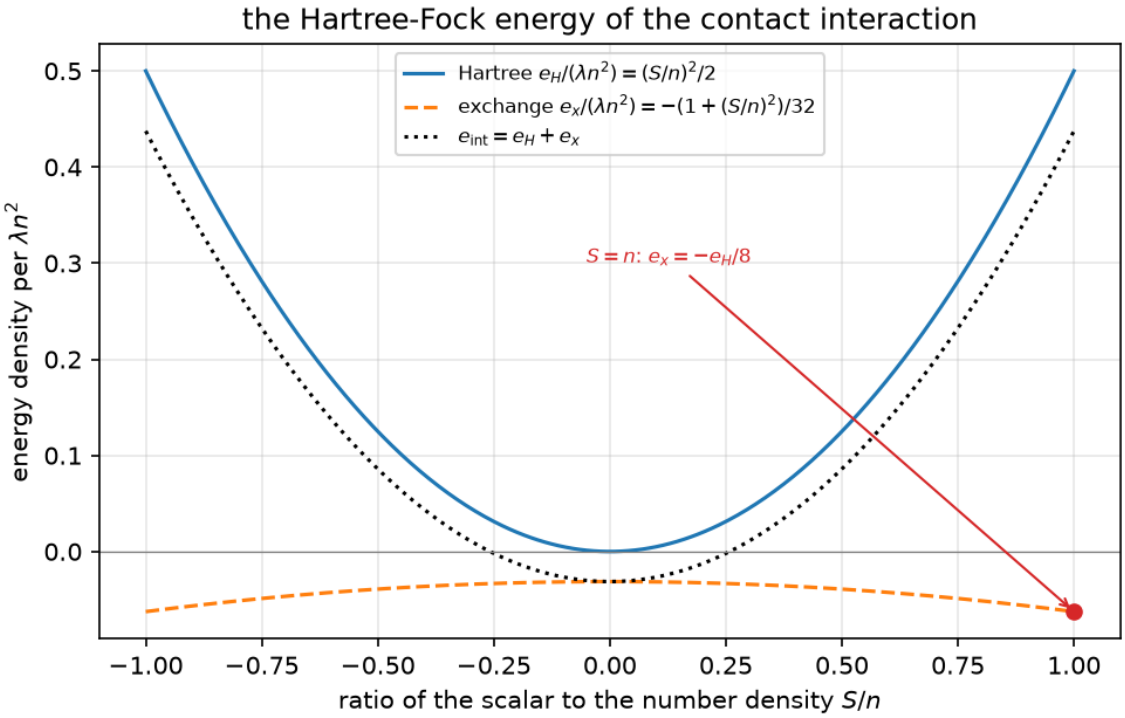

Figure 402: The interaction energy densities of the contact term per unit λn^2 against the ratio S/n of the scalar and the number density: the Hartree term $(S/n)^2/2$ (solid), the exact local exchange $-(1 + (S/n)^2)/32$ (dashed) and their sum $e_{\rm int}$ (dotted). For $\lambda > 0$ the exchange lowers the energy a little; at $S = n$ (a filled level at rest, red dot) it is $-1/8$ of the Hartree term, and near $S = 0$ it is all that is left. (Notebook 14d, figure 2.)

Out [9]:

```
Figure 14d.2 saved as Revision/textbook/figures/14d_2_energy_densities.png
```

Text cell:

```
## 11. The exact Fock exchange of the closed-shell slab

The states of the model are not uniform along $y$, and in each block the
orbital is a $2\times2$ object. The Revision record computes the EXACT Fock
exchange of such a closed-shell state: take one level with the $2\times2$ block
density $D = \frac12(d_0 + d_1\sigma_1 + d_2\sigma_2 + d_3\sigma_3)$ in each of
the four blocks of type $j = +1$ and $\sigma_3D\sigma_3$ in each of the four of
```

```
type $-1$ (the degenerate partners), and average it over a closed shell of
3-space directions; the average is the projection onto the commutant of the
3-space rotations (the 64-dimensional space of matrices that commute with
$\gamma^{(x_1)}\gamma^{(x_2)}$, $\gamma^{(x_1)}\gamma^{(x_3)}$,
$\gamma^{(x_2)}\gamma^{(x_3)}$). The result: $n = 8d_0$, $S = 8d_2$,
$Q = \langle\Psi^\dagger B\gamma^{(x_0)}\Psi\rangle = 8d_3$, the $y$-current
$Y = 8d_1$, and

$$e_x^{\rm exact} = -\frac{\lambda}{32}(n^2 + S^2 - Q^2 - Y^2).$$

For eigen-orbitals $Y = 0$; so the uniform-gas exchange omits $+\frac{\lambda}{32}
Q^2$. The canonical functional of the Revision theory is the uniform-gas one; the
exact Fock exchange is a DIAGNOSTIC that the Rust solver reports and also solves
as a variant. The next cell repeats the computation: it reads the block basis
from the Revision record (and checks that it is unitary), builds the averaged
one-body matrix with sympy's linear solver and checks the four densities and the
exchange.
```

In [10]:

```
as_number = {"0": 0, "1": 1, "-1": -1, "I": sp.I, "-I": -sp.I}
U = sp.Matrix([[as_number[e] for e in row] for row in
               theory["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
V = U / (2 * sp.sqrt(2))  # the block basis of the record
labels = [(j, s2, s3) for j in (1, -1) for s2 in (1, -1) for s3 in (1, -1)]
s1 = sp.Matrix([[0, 1], [1, 0]])
s2m = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
d0, d1, d2, d3 = sp.symbols("d0 d1 d2 d3", real=True)
D = (d0 * sp.eye(2) + d1 * s1 + d2 * s2m + d3 * s3) / 2  # the block density
G_slab = Z16
for b, (j, _, _) in enumerate(labels):
    Vb = V[:, 2 * b:2 * b + 2]
    G_slab = G_slab + Vb * (D if j == 1 else s3 * D * s3) * Vb.H
unknowns = sp.Matrix(16, 16, sp.symbols("x0:256"))  # a general 16 x 16 matrix
equations = []
for rotation in (g1 * g2, g1 * g3, g2 * g3):  # commute with the 3-space rotations
    equations += list(rotation * unknowns - unknowns * rotation)
solution = sp.Matrix(16, 16, list(list(sp.linsolve(equations, list(unknowns)))[0]))
free = sorted(solution.free_symbols, key=lambda s_: int(str(s_)[1:]))
basis = [solution.subs({x: (1 if x == y else 0) for x in free}) for y in free]
gram = sp.Matrix(len(basis), len(basis),
                 lambda i, k: (basis[i].H * basis[k]).trace())
coefficients = gram.LUsolve(sp.Matrix([(b_.H * G_slab).trace() for b_ in basis]))
G_avg = sp.simplify(sum((c_ * b_ for c_, b_ in zip(coefficients, basis)), Z16))
rho_slab = G_avg * B
n_s, S_s = sp.simplify((B * rho_slab).trace()), sp.simplify((C * rho_slab).trace())
Q_s = sp.simplify((B * g0 * rho_slab).trace())
e_x_slab = sp.expand(-lam / 2 * (C * rho_slab * C * rho_slab).trace())
say(f"commutant dimension {len(basis)}; n = {n_s}, S = {S_s}, Q = {Q_s}")
say(f"V unitary: {sp.simplify(V.H * V) == I16}")
check(len(basis) == 64 and n_s == 8 * d0 and S_s == 8 * d2 and Q_s == 8 * d3
      and sp.simplify(e_x_slab + lam / 32 * (n_s ** 2 + S_s ** 2 - Q_s ** 2
```

```
                                          - (8 * d1) ** 2)) == 0
      and record_check("exchange_slab_exact_fock"),
      "exact Fock exchange of the slab: -(lambda/32)(n^2 + S^2 - Q^2 - Y^2)",
      record=f"{PY_REPORT}, check exchange_slab_exact_fock")
```

Out [10]:

```
commutant dimension 64; n = 8*d0, S = 8*d2, Q = 8*d3
V unitary: True
PASS exact Fock exchange of the slab: -(lambda/32)(n^2 + S^2 - Q^2 - Y^2)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    exchange_slab_exact_fock
```

Text cell:

```
For one orbital $\chi = (a, ib)$ the three densities are proportional to
$a^2 + b^2$ ($n$), $2ab$ ($S$, up to the sign $j$) and $a^2 - b^2$ ($Q$), so
$S^2 + Q^2 = n^2$; for a mixture $S^2 + Q^2 \le n^2$: the physical states fill
the disk $(S/n)^2 + (Q/n)^2 \le 1$. The next cell draws, over this disk, the
ratio $e_x^{\rm exact}/e_x^{\rm gas} = (n^2 + S^2 - Q^2)/(n^2 + S^2)$ of the exact
Fock exchange to the uniform-gas exchange ($Y = 0$). On the line $Q = 0$ the two
agree; at the point $S = 0$, $Q = n$, which is the state of the zero modes
(section 12), the exact exchange vanishes.
```

In [11]:

```
grid = np.linspace(-1.0, 1.0, 401)
S_grid, Q_grid = np.meshgrid(grid, grid)  # S/n horizontally, Q/n vertically
inside = S_grid ** 2 + Q_grid ** 2 <= 1.0
ratio_grid = np.where(inside, (1.0 + S_grid ** 2 - Q_grid ** 2) / (1.0 + S_grid ** 2),
                      np.nan)
fig, ax = plt.subplots(figsize=(6.4, 5.2))
filled = ax.contourf(S_grid, Q_grid, ratio_grid, levels=np.linspace(0.0, 1.0, 11),
                     cmap="Blues")
ax.contour(S_grid, Q_grid, ratio_grid, levels=[0.5], colors="gray", linewidths=0.8)
fig.colorbar(filled, ax=ax, label="$e_x^{\\rm exact} / e_x^{\\rm gas}$")
ax.plot([0.0, 0.0], [1.0, -1.0], "o", color="C3")  # S = 0, Q = +n and -n
ax.annotate("zero modes ($N = 8$): ratio 0", (0.05, 0.9), fontsize=8, color="C3")
ax.annotate("$Q = 0$: ratio 1", (-0.95, 0.04), fontsize=8,
            color="white")  # white text on the dark band
ax.set_xlabel("$S/n$")
ax.set_ylabel("$Q/n$")
ax.set_aspect("equal")
ax.grid(False)
ax.set_title("exact Fock exchange / uniform-gas exchange")
save_figure(fig, "exact_fock_ratio",
            "The ratio of the exact Fock exchange of a closed-shell slab state, "
            "$-(\\lambda/32)(n^2 + S^2 - Q^2)$, to the uniform-gas exchange "
            "$-(\\lambda/32)(n^2 + S^2)$, over the disk of possible density "
            "ratios $(S/n)^2 + (Q/n)^2 \\le 1$ (horizontal $S/n$, vertical "
            "$Q/n$, darker is larger, gray line the ratio 1/2). On the line "
```

```
"$Q = 0$ the two exchanges agree; toward the top and bottom of the "
"disk the uniform-gas formula overestimates the exchange, and at "
"$S = 0$, $Q = \\pm n$ (red dots; the top one is the state of the "
"eight zero modes) the exact exchange vanishes.")
```

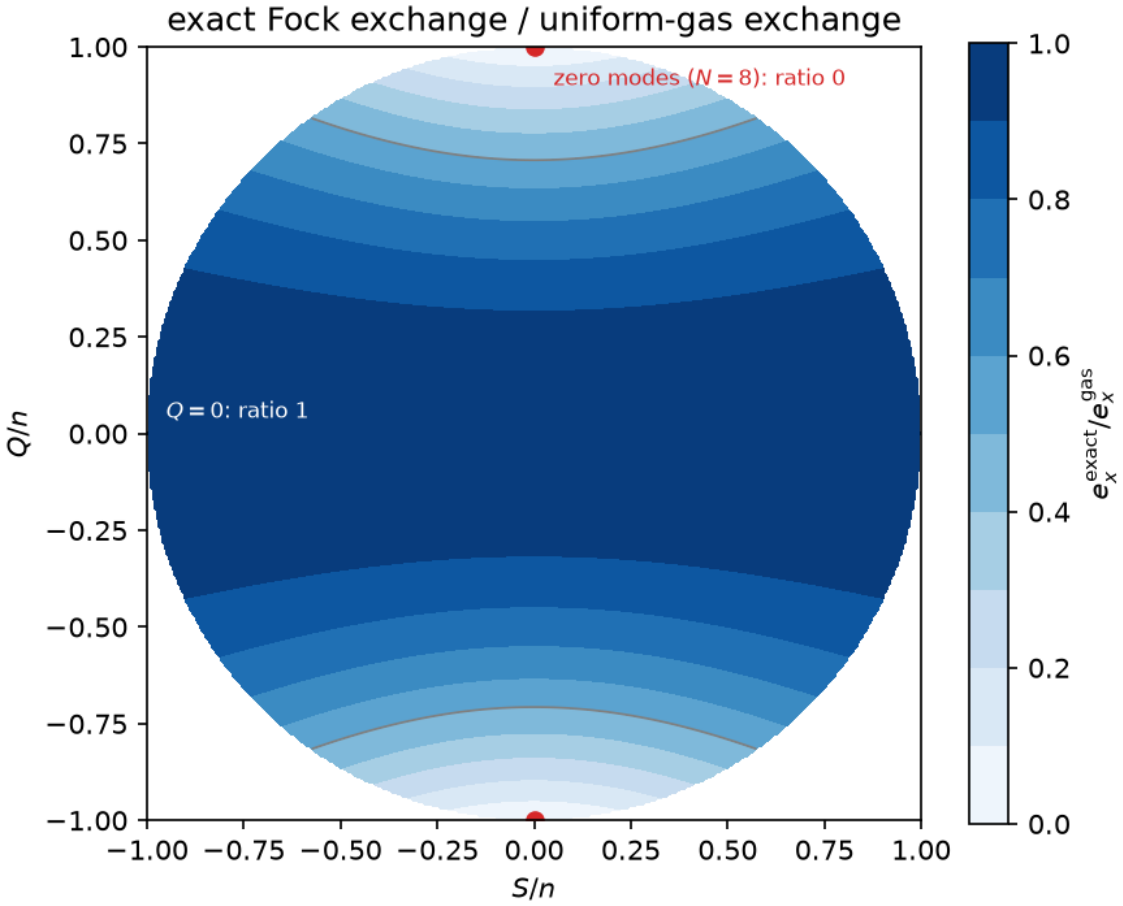

Figure 403: The ratio of the exact Fock exchange of a closed-shell slab state, $-(\lambda/32)(n^2+S^2-Q^2)$, to the uniform-gas exchange $-(\lambda/32)(n^2+S^2)$, over the disk of possible density ratios $(S/n)^2+(Q/n)^2 \leq 1$ (horizontal S/n, vertical Q/n, darker is larger, gray line the ratio 1/2). On the line $Q = 0$ the two exchanges agree; toward the top and bottom of the disk the uniform-gas formula overestimates the exchange, and at $S = 0$, $Q = \pm n$ (red dots; the top one is the state of the eight zero modes) the exact exchange vanishes. (Notebook 14d, figure 3.)

Out [11]:

```
Figure 14d.3 saved as Revision/textbook/figures/14d_3_exact_fock_ratio.png
```

Text cell:

```
## 12. The state of the eight zero modes (N = 8)
```

The simplest Kohn-Sham state of the model fills the eight brane zero modes at $k = 0$ (both block types, even parity). Each orbital is $\chi = (a, 0)$ with $a = \sqrt{2M/(1 - e^{-2ML})}\,e^{My}$ (normalised on the patch). The densities, line by line (Revision record, ‘densities‘):

1. One orbital: $n_o = P\,(a^2 + b^2) = Pa^2$, $s_o = P\,j\,2ab = 0$, $q_o = P\,(a^2 - b^2) = Pa^2$, with $P = e^{-6Hy}/(\ell^3v_t)$ (the factor that makes the densities proper).
2. The state: $n = \sum w\,g\,f\,n_o$ with the weight $w = \frac12$ of the ASSUMED Z2 doubled system, the degeneracy $g = 4$ per block type and $f = 1$: $n = \frac12(4 + 4)Pa^2 = 4Pa^2$; $S = 0$; $Q = n$.
3. The potentials to first order in λ (free orbitals): $M_{\rm eff} - m = \frac{15}{16}\lambda S = 0$, $v_v = -\frac{\lambda}{16}n$.
4. The Revision solver calibrates its couplings by the largest first-order potential per unit λ, max over y of $\max(\frac{15}{16}|S|, |n|/16)$, here $n_{\max}/16$; λ_1 and λ_2 are 0.1 and 0.3 divided by it, rounded to 4 significant digits.
5. Exact Fock exchange: with $S = 0$ and $Q = n$, $e_x^{\rm exact} = -\frac{\lambda}{32}(n^2 - Q^2) = 0$. In the exact-Fock functional the potential is $v_v + w_Q\sigma_3$ with $w_Q = \frac{\lambda}{16}Q$, which on $(a, 0)$ acts as $\frac{\lambda}{16}(Q - n) = 0$: the zero modes stay exact solutions with $\varepsilon = 0$ and the energy of the state is exactly 0.

With $H = m = 1$, $L = 3$, $\ell = 2\pi/0.25$, $v_t = 1$, the next cell computes $n(y)$ and its maximum (at the tip $y = -3$, where $e^{-6Hy}a^2 \propto e^{(2M - 6H)y}$ is largest) and compares with the records: the largest proper density of the free state $N = 8$ (‘ground/summary.csv‘, column ‘n_max‘), the strength per λ, $\lambda_{1,2}$ and the first-order potentials (‘parameters.json‘), and the self-consistent exact-Fock energies of all twenty $N = 8$ states (‘exx/exact-fock-variant.csv‘), which must be zero to rounding.

In [12]:

```
H_val, m_val, L_val = 1.0, 1.0, 3.0
ell = 2.0 * math.pi / 0.25  # the coordinate size of the 3-torus
vol7 = ell ** 3 * 1.0  # the coordinate 7-volume ell^3 v_t
y = np.linspace(-L_val, 0.0, 1801)  # the hidden coordinate
a_sq = 2.0 * m_val / (1.0 - math.exp(-2.0 * m_val * L_val)) * np.exp(2.0 * m_val * y)
P = np.exp(-6.0 * H_val * y) / vol7  # proper-density factor
n_y = 4.0 * P * a_sq  # line 2
n_max = float(np.max(n_y))
summary = repository_file("Revision/kohn_sham/results/ground/summary.csv") \
    .read_text(encoding="utf-8").splitlines()
header = summary[0].split(",")
free8 = dict(zip(header, [r for r in summary if r.startswith("N8_lam0_a00,")][0]
                 .split(",")))
report("largest proper density of the free N = 8 state", f"{n_max:.10f}",
       "per unit proper 7-volume")
check(abs(n_max / float(free8["n_max"]) - 1.0) < 1e-9 and float(y[np.argmax(n_y)])
      == -3.0,
      "n_max = 4 P a^2 at the tip y = -3 equals the record",
      record="Revision/kohn_sham/results/ground/summary.csv, N8_lam0_a00, n_max")
```

```
def round_significant(x, digits):
    """x rounded to digits significant digits (halves away from zero)."""
    exponent = math.floor(math.log10(abs(x))) - (digits - 1)
    scale = 10.0 ** (-exponent)
    return math.copysign(math.floor(abs(x) * scale + 0.5), x) / scale


strength = n_max / 16.0  # line 4: the largest first-order potential per lambda
lambda_1 = round_significant(0.1 / strength, 4)
lambda_2 = round_significant(0.3 / strength, 4)
parameters = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                        .read_text(encoding="utf-8"))
calibration = [c for c in parameters["couplingCalibration"]["values"]
               if c["N"] == 8.0][0]
report("strength per lambda", f"{strength:.9f}")
report("lambda_1, lambda_2", f"{lambda_1}, {lambda_2}")
report("first-order potentials", f"{lambda_1 * strength:.10f}, "
       f"{lambda_2 * strength:.10f}", "m")
check(abs(strength / calibration["strengthPerLambda"] - 1.0) < 1e-9
      and lambda_1 == calibration["lambda1"] and lambda_2 == calibration["lambda2"]
      and abs(lambda_1 * strength - calibration["firstOrderPotential1"]) < 1e-9
      and len(set(calibration["strengthPerLambdaAtSlices"])) == 1,
      "strength 5.1388, lambda_1 = 0.01946, lambda_2 = 0.05838 (the same at all "
      "slices)", record="Revision/kohn_sham/results/parameters.json, "
      "couplingCalibration N = 8")
exx = repository_file("Revision/kohn_sham/results/exx/exact-fock-variant.csv") \
    .read_text(encoding="utf-8").splitlines()
exx_header = exx[0].split(",")
rows8 = [dict(zip(exx_header, r.split(","))) for r in exx[1:] if r.startswith("N8_")]
largest = max(abs(float(r["E_exact_fock_scf"])) for r in rows8)
check(len(rows8) == 20 and largest < 1e-12,
      "the self-consistent exact-Fock energy of all 20 N = 8 states is 0",
      record="Revision/kohn_sham/results/exx/exact-fock-variant.csv, "
             "E_exact_fock_scf")
```

Out [12]:

```
RESULT largest proper density of the free N = 8 state = 82.2208638894 per unit proper
    7-volume
PASS n_max = 4 P a^2 at the tip y = -3 equals the record
     reproduces Revision/kohn_sham/results/ground/summary.csv, N8_lam0_a00, n_max
RESULT strength per lambda = 5.138803993
RESULT lambda_1, lambda_2 = 0.01946, 0.05838
RESULT first-order potentials = 0.1000011257, 0.3000033771 m
PASS strength 5.1388, lambda_1 = 0.01946, lambda_2 = 0.05838 (the same at all slices)
     reproduces Revision/kohn_sham/results/parameters.json, couplingCalibration N = 8
PASS the self-consistent exact-Fock energy of all 20 N = 8 states is 0
     reproduces Revision/kohn_sham/results/exx/exact-fock-variant.csv, E_exact_fock_scf
```

Text cell:

```
The next cell draws the state. Left: the proper density $n(y)$ and the
coordinate density $e^{6Hy}n(y)$ (particles per unit $y$ and per unit coordinate
volume) on a logarithmic axis: the zero modes sit at the brane in the coordinate
density, but because the proper 7-volume $e^{6Hy}$ shrinks toward the tip, the
PROPER density, which the interaction feels, is largest at the tip. Right: the
first-order vector potential $v_v = -\lambda n/16$ of the uniform-gas
functional for $\lambda_1$ and $\lambda_2$ (its largest size is $0.1$ and
$0.3\,m$, by the calibration) and the exact-Fock combination
$v_v + w_Q = \frac{\lambda}{16}(Q - n) = 0$.
```

In [13]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.5, 4.0))
fig.subplots_adjust(wspace=0.32)  # room for the label of the right panel
left.semilogy(y, n_y, label="proper density $n(y) = 4Pa^2$")
left.semilogy(y, np.exp(6.0 * y) * n_y, "--",
              label="coordinate density $e^{6Hy} n(y)$")
left.set_xlabel("$y$ (units of $1/H$; tip at $-3$, brane at $0$)")
left.set_ylabel("density (logarithmic)")
left.set_title("the eight zero modes ($N = 8$, $\\lambda = 0$)")
left.legend(fontsize=8)
right.plot(y, -lambda_1 * n_y / 16.0, label=f"$v_v$, $\\lambda_1 = {lambda_1}$")
right.plot(y, -lambda_2 * n_y / 16.0, label=f"$v_v$, $\\lambda_2 = {lambda_2}$")
right.plot(y, np.zeros_like(y), ":", color="black",
           label="exact Fock: $v_v + w_Q = 0$")
right.set_xlabel("$y$ (units of $1/H$)")
right.set_ylabel("first-order potential (units of $m$)")
right.set_title("the Kohn-Sham potential of the zero modes")
right.legend(fontsize=8, loc="lower right")
save_figure(fig, "zero_mode_state",
            "The state of the eight brane zero modes ($N = 8$, $m = H = 1$, "
            "$L = 3$, $\\Delta k = 0.25$, $v_t = 1$). Left, logarithmic axis: the "
            "proper density $n(y) = 4Pa^2$ (solid) and the coordinate density "
            "$e^{6Hy}n(y)$ (dashed) against $y$ (units of $1/H$); the orbitals "
            "sit at the brane, but the proper density is largest at the tip, "
            "$n_{\\max} = 82.22$ at $y = -3$. Right: the first-order vector "
            "potential $v_v = -\\lambda n/16$ of the uniform-gas functional for "
            "$\\lambda_1 = 0.01946$ and $\\lambda_2 = 0.05838$ (lowest values "
            "$-0.1$ and $-0.3$ in units of $m$, the calibration of the Revision "
            "runs) and the exact-Fock potential, which is zero for this state.")
```

Out [13]:

```
Figure 14d.4 saved as Revision/textbook/figures/14d_4_zero_mode_state.png
```

Text cell:

```
## 13. The last check

The last cell checks that the four figure files exist and prints the number of
checks that passed.
```

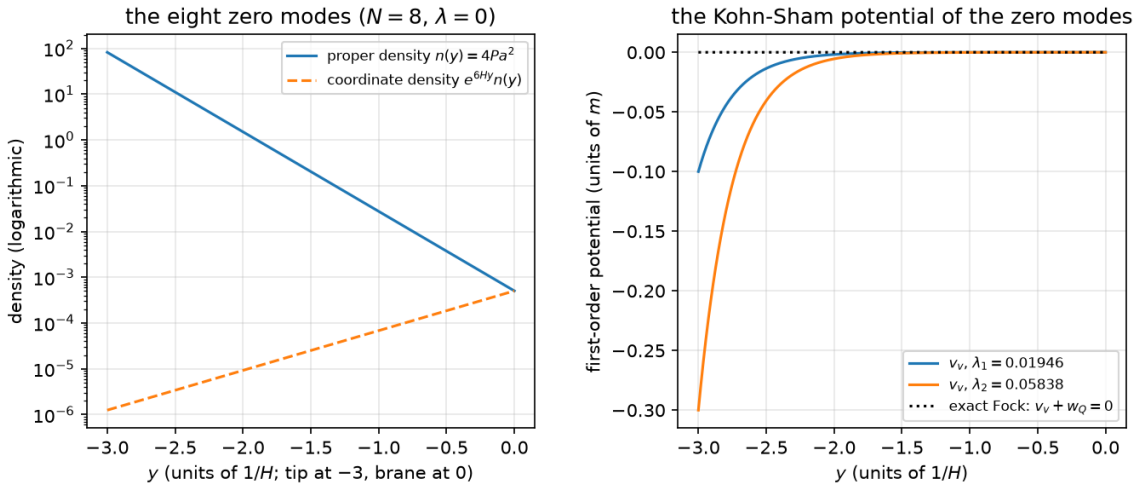

Figure 404: The state of the eight brane zero modes ($N = 8$, $m = H = 1$, $L = 3$, $\Delta k = 0.25$, $v_t = 1$). Left, logarithmic axis: the proper density $n(y) = 4Pa^2$ (solid) and the coordinate density $e^{6Hy}n(y)$ (dashed) against y (units of $1/H$); the orbitals sit at the brane, but the proper density is largest at the tip, $n_{\max} = 82.22$ at $y = -3$. Right: the first-order vector potential $v_v = -\lambda n/16$ of the uniform-gas functional for $\lambda_1 = 0.01946$ and $\lambda_2 = 0.05838$ (lowest values -0.1 and -0.3 in units of m, the calibration of the Revision runs) and the exact-Fock potential, which is zero for this state. (Notebook 14d, figure 4.)

In [14]:

```
figure_names = ["14d_1_pair_average.png", "14d_2_energy_densities.png",
                "14d_3_exact_fock_ratio.png", "14d_4_zero_mode_state.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all four figure files of this notebook exist")
all_checks_passed()
```

Out [14]:

```
PASS all four figure files of this notebook exist
ALL 16 CHECKS PASSED (notebook 14d)
```

Text cell:

```
## 14. What this notebook showed

- The flat good-sector plane waves have the energies $\pm\sqrt{M^2 + p^2}$, each
  8 times; every occupation symmetric under $\mathbf p \to -\mathbf p$ has the
  one-body matrix $\rho = (nB + SC)/16$ (PROVED).
- Wick's rule gives $\langle{:}S^2{:}\rangle = (\mathrm{Tr}\,C\rho)^2 -
  \mathrm{Tr}(C\rho C\rho)$, and the exchange of the contact interaction is
  exactly local: $e_x = -\frac{\lambda}{32}(n^2 + S^2)$ at every temperature;
  a filled level at rest has $e_x/e_H = -1/8$ (PROVED).
- The Kohn-Sham potentials are $M_{\rm eff} = m + \frac{15}{16}\lambda S$ and
```

```
  $v_v = -\frac{\lambda}{16}n$; on shell $\langle L\rangle/\sqrt{|g|} =
  e_{\rm int}$ (PROVED; the coefficients equal those the Rust solver used).
- For the closed-shell slab states the exact Fock exchange is
  $-\frac{\lambda}{32}(n^2 + S^2 - Q^2 - Y^2)$; the canonical (uniform-gas)
  functional omits $+\frac{\lambda}{32}Q^2$ (PROVED; a DIAGNOSTIC of the
  functional; there is no correlation energy in either).
- The eight zero modes have $S = 0$ and $Q = n$; their proper density is largest
  at the tip ($n_{\max} = 82.22$); this fixes the Revision couplings
  $\lambda_1 = 0.01946$, $\lambda_2 = 0.05838$ for $N = 8$; and their exact Fock
  exchange vanishes, so their exact-Fock energy is exactly zero (the record's
  self-consistent values are zero to rounding).
- ASSUMED: the Z2 doubled system (the weight $w = \frac12$) and the good sector.
```

14.31 Line-by-line walk-through of Notebook 14d

The notebook has 14 code cells, In [1] to In [14]. Docstrings and the long caption texts are left out of the quotations; the captions are printed under the figures in Section 14.30.

In [1], the set-up cell. The code of In [1] of Notebook 14a (Section 14.11), line for line, except

```
NOTEBOOK_ID = "14d"  # this notebook: chapter 14, example d
```

which names this notebook. Its comment lines are the run instructions of Section 14.29.

In [2], the gamma matrices, B and C.

```
import math  # exp, sqrt, log10 of single numbers

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra with symbols

PY_REPORT = "Revision/kohn_sham/reports/ks-theory-python.json"
WL_REPORT = "Revision/kohn_sham/reports/ks-theory-wolfram.json"
```

The three modules and the two theory reports.

```
def record_check(name):
    verdicts = []
    for report_file in (PY_REPORT, WL_REPORT):
        data = json.loads(repository_file(report_file).read_text(encoding="utf-8"))
        found = [c["verdict"] for c in data["checks"] if c["name"] == name]
        verdicts.append(found[0] if found else "absent")
    return verdicts[0] == "PASS" and verdicts[1] in ("PASS", "absent")
```

The `record_check` of Notebook 14a: true when the sympy report says PASS and the Wolfram report says PASS or has no check of that name. (The loop variable is called `report_file`, so that it does not hide the helper `report` of the set-up cell.)

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
```

```
g = [sp.Matrix(m) for m in fixture["gamma"]]
g0, g1, g2, g3, g4, g5, g6, g7 = g  # gN is gamma^(xN)
I16, Z16 = sp.eye(16), sp.zeros(16)
C = g0 * g1 * g2 * g3
B = -sp.I * C * g4
```

The author's gammas from the Revision record, exactly as in In [2] and In [3] of Notebook 14a, and the matrices C and B.

```
facts_ok = (B * B == I16 and C * C == I16 and (B * B).trace() == 16
            and (C * C).trace() == 16 and (B * C).trace() == 0 and C * B * C == B
            and (-sp.I * g4) * B == C)
check(facts_ok, "B^2 = C^2 = 1, Tr B^2 = Tr C^2 = 16, Tr BC = 0, CBC = B, (-i g4) B = C")
```

Every matrix fact used in Section 14.26, checked exactly: $B^2 = C^2 = 1$, their traces 16, $\mathrm{Tr}(BC) = 0$, $CBC = B$ and $(-i\gamma^{(x_4)})B = C$. Out [2] is one PASS line.

In [3], the plane waves and their projector.

```
M = sp.symbols("M", positive=True)  # the mass
p1, p2, p3, p0 = sp.symbols("p1 p2 p3 p0", real=True)  # the momentum
E = sp.symbols("E", positive=True)  # stands for sqrt(M^2 + p^2)
h_p = M * (-sp.I * g4) - (p1 * g4 * g1 + p2 * g4 * g2 + p3 * g4 * g3 + p0 * g4 * g0)
E_squared = M ** 2 + p1 ** 2 + p2 ** 2 + p3 ** 2 + p0 ** 2
square_ok = (h_p * h_p - E_squared * I16).applyfunc(sp.expand).is_zero_matrix
```

The mass, the four momentum components, a positive symbol E, the matrix $h_{\mathbf{p}}$ of Section 14.26, and the test $h_{\mathbf{p}}^2 = (M^2 + p^2)\,1$ entry by entry (`applyfunc(sp.expand)` multiplies out every entry).

```
G_p = (I16 + h_p / E) / 2  # the projector on the positive energy
projector_ok = ((G_p * G_p - G_p).subs(E, sp.sqrt(E_squared))
                .applyfunc(sp.simplify).is_zero_matrix
                and G_p.H == G_p and sp.simplify(G_p.trace()) == 8)
check(square_ok and projector_ok and record_check("gas_mode_projector"),
      "h_p^2 = (M^2 + p^2) 1 and G_p = (1 + h_p/E)/2 is a projector of rank 8",
      record=f"{PY_REPORT}, check gas_mode_projector")
```

$G_{\mathbf{p}}$; the test $G_{\mathbf{p}}^2 = G_{\mathbf{p}}$ after replacing E by $\sqrt{M^2 + p^2}$; $G_{\mathbf{p}}$ Hermitian; its trace 8 (for a projector the trace is the rank, Section 14.5). One PASS line.

In [4], the symmetric one-body matrix.

```
p_flip = {p1: -p1, p2: -p2, p3: -p3, p0: -p0}
pair_average = ((G_p * B + (G_p * B).subs(p_flip)) / 2).applyfunc(sp.simplify)
pair_ok = (pair_average - (B + M / E * C) / 2).applyfunc(sp.simplify).is_zero_matrix
```

A dictionary that replaces $\mathbf{p}$ by $-\mathbf{p}$; the average of $G_{\mathbf{p}}B$ and $G_{-\mathbf{p}}B$; and the test that it is $\frac{1}{2}(B + (M/E)C)$ (Section 14.26).

```
th, ph = sp.symbols("vartheta varphi", real=True)  # angles on the sphere
components = [sp.sin(th) * sp.cos(ph), sp.sin(th) * sp.sin(ph), sp.cos(th)]
sphere = [sp.integrate(sp.integrate(c * sp.sin(th), (ph, 0, 2 * sp.pi)), (th, 0, sp.pi))
          / (4 * sp.pi) for c in components]  # the averages of the components
say(f"sphere averages of the three direction components: {sphere}")
check(pair_ok and sphere == [0, 0, 0] and record_check("gas_angular_average"),
      "the average over p and -p (or over all directions) is (B + (M/E) C)/2",
      record=f"{PY_REPORT}, check gas_angular_average")
```

The same for an average over all directions of $\mathbf{p}$: a direction on the sphere has the three components $\sin\vartheta\cos\varphi$, $\sin\vartheta\sin\varphi$ and $\cos\vartheta$, with the two angles ϑ and φ; the average of a function over the sphere is its integral with the area element $\sin\vartheta\, d\varphi\, d\vartheta$ divided by the area 4π. All three components average to 0 (printed in Out [4]), so the part of $G_{\mathbf{p}}B$ linear in $\mathbf{p}$ averages away too.

```
h_0 = M * (-sp.I * g4)  # the p-independent part of h_p
negative_ok = (((I16 - h_0 / E) / 2) * B - (B - M / E * C) / 2).is_zero_matrix
check(negative_ok and record_check("gas_negative_energy_modes"),
      "negative-energy modes give (B - (M/E) C)/2: only B and C ever appear",
      record=f"{PY_REPORT}, check gas_negative_energy_modes")
```

The negative-energy modes: the part of $\frac{1}{2}(1 - h_{\mathbf{p}}/E)B$ that survives the average (the part of $h_{\mathbf{p}}$ without $\mathbf{p}$, `h_0`) is $\frac{1}{2}(B - (M/E)C)$.

```
n, S, lam = sp.symbols("n S lambda", real=True)
rho = (n * B + S * C) / 16  # the one-body matrix of a symmetric occupation
check(sp.expand((B * rho).trace()) == n and sp.expand((C * rho).trace()) == S
      and record_check("gas_densities"),
      "rho = (nB + SC)/16 has Tr(B rho) = n and Tr(C rho) = S",
      record=f"{PY_REPORT}, check gas_densities")
```

Symbols for the two densities and the coupling, the matrix $\rho = (nB + SC)/16$, and the check that it gives back n and S. Out [4] shows three PASS lines.

In [5], figure 1.

```
values = {M: 1, E: sp.sqrt(2), p1: 1, p2: 0, p3: 0, p0: 0}
one_mode = np.abs(np.array((G_p * B).subs(values).evalf(), dtype=complex))
averaged = np.abs(np.array(pair_average.subs(values).evalf(), dtype=complex))
odd_part = np.abs(np.array((G_p * B - pair_average).subs(values).evalf(),
                           dtype=complex))
```

The example $M = 1$, $\mathbf{p} = (1, 0, 0, 0)$, $E = \sqrt{2}$, and the absolute values of the entries of three matrices: $G_{\mathbf{p}}B$, its average with $-\mathbf{p}$, and their difference, the part linear in $\mathbf{p}$.

```
fig, axes = plt.subplots(1, 3, figsize=(11.5, 3.9))
fig.subplots_adjust(wspace=0.45)
titles = ["$|G_{\\mathbf{p}}B|$: one momentum",
          "$|(G_{\\mathbf{p}} + G_{-\\mathbf{p}})B/2|$",
          "$|$part linear in $\\mathbf{p}|$"]
```

```
for ax, data, title in zip(axes, (one_mode, averaged, odd_part), titles):
    image = ax.imshow(data, cmap="Blues", vmin=0.0, vmax=0.6,
                      interpolation="nearest")
    ax.set_title(title, fontsize=9)
    ax.set_xticks([0, 5, 10, 15])
    ax.set_yticks([0, 5, 10, 15])
    ax.set_xlabel("column")
    ax.set_ylabel("row")
    ax.grid(False)
    fig.colorbar(image, ax=ax, shrink=0.75)
save_figure(fig, "pair_average",
            "Absolute values of the entries (rows and columns 0 to 15, darker is "
            ...)
```

Three heat maps as in In [15] of Notebook 14a, with one common colour scale from 0 to 0.6 (`vmax=0.6`), so that the three panels can be compared. **What figure 14d.1 shows.** Left: the matrix of one momentum has entries in many places. Middle: after the average with $-\mathbf{p}$ only the entries of $\frac{1}{2}B$ (size $\frac{1}{2}$, dark) and of $\frac{M}{2E}C = \frac{1}{2\sqrt{2}}C$ (size about 0.35, lighter) remain. Right: the part linear in $\mathbf{p}$, $-\frac{1}{2E}p_1\gamma^{(x_4)}\gamma^{(x_1)}B$, with entries of size $\frac{1}{2\sqrt{2}} \approx 0.35$ in other places; it is exactly what the average with $-\mathbf{p}$ removes.

In [6], Wick's rule term by term.

```
half, i_half = sp.Rational(1, 2), sp.I / 2
u1 = sp.Matrix([half, i_half, 0, 0, 0, 0, 0, 0, half, 0, 0, 0, 0, 0, -i_half, 0])
u2 = sp.Matrix([0, 0, half, 0, -half, 0, 0, 0, 0, 0, i_half, 0, 0, 0, 0, i_half])
v1 = sp.Matrix([1, 2, 0, -1, sp.I, 0, 3, 1, 0, 1, -2, sp.I, 0, 1, 1, 0]) / 4
v2 = sp.Matrix([0, 1, sp.I, 2, 1, -1, 0, 0, 1, 0, sp.I, 1, 2, 0, -1, 1]) / 4
nonzero = [(A_, B_) for A_ in range(16) for B_ in range(16) if C[A_, B_] != 0]
wick_ok = True
```

Two pairs of mode vectors: u_1, u_2, the two-mode state of the Revision check, and v_1, v_2, a second state with less symmetry. `nonzero` lists the 16 positions (A, B) where C has a nonzero entry; only these contribute to the sum.

```
for name, modes in (("u1, u2 (record)", (u1, u2)), ("v1, v2", (v1, v2))):
    rho_2 = (modes[0] * modes[0].H + modes[1] * modes[1].H) * B  # one-body matrix
    f = rho_2.T  # f_AB = rho_BA
    brute = sp.simplify(sum(
        C[A_, B_] * C[C_, D_] * (f[A_, B_] * f[C_, D_] - f[A_, D_] * f[C_, B_])
        for A_, B_ in nonzero for C_, D_ in nonzero))
    direct = sp.simplify((C * rho_2).trace() ** 2)
    exchange = sp.simplify((C * rho_2 * C * rho_2).trace())
    say(f"state {name}: (Tr C rho)^2 = {direct}, Tr(C rho C rho) = {exchange}, "
        f"term-by-term sum = {brute}")
    wick_ok &= sp.simplify(brute - (direct - exchange)) == 0
```

For each state: the one-body matrix $\rho = (u_1u_1^\dagger + u_2u_2^\dagger)B$, the expectations $f_{AB} = \rho_{BA}$ (the transpose), the sum of Wick's rule over the $16 \times 16 = 256$ pairs of nonzero entries of C (`brute`), and the two traces. Out [6] prints, for the record's state, $(\mathrm{Tr}\, C\rho)^2 = 1/4$, $\mathrm{Tr}(C\rho C\rho) = 1/4$ and the

sum 0, and for the second state 25/64, 9/32 and 7/64: in both cases the sum equals the difference of the traces ($25/64 - 18/64 = 7/64$).

```
check(len(nonzero) == 16 and wick_ok and record_check("hf_wick_contraction"),
      "<:S^2:> = (Tr C rho)^2 - Tr(C rho C rho), checked term by term",
      record=f"{PY_REPORT}, check hf_wick_contraction")
```

One PASS line.

In [7], the exchange and the ratio $-1/8$.

```
e_x = sp.factor(-lam / 2 * (C * rho * C * rho).trace())  # the exchange energy density
say(f"e_x = -(lambda/2) Tr(C rho C rho) = {e_x}")
check(sp.simplify(e_x + lam / 32 * (n ** 2 + S ** 2)) == 0
      and record_check("exchange_uniform_gas"),
      "the exact local exchange e_x = -(lambda/32)(n^2 + S^2)",
      record=f"{PY_REPORT}, check exchange_uniform_gas")
```

$e_x = -\frac{\lambda}{2}\mathrm{Tr}(C\rho C\rho)$ with $\rho = (nB + SC)/16$, written by `sp.factor` as a product (Out [7]: `-lambda*(S**2 + n**2)/32`), and the check.

```
e_H = lam / 2 * S ** 2  # the Hartree energy density
ratio = sp.simplify((e_x / e_H).subs({n: 8, S: 8}))
say(f"one filled level at rest (n = S = 8): e_x/e_H = {ratio}")
check(ratio == sp.Rational(-1, 8) and record_check("filled_shell_ratio"),
      "a filled 8-fold level at rest has e_x/e_H = -1/8",
      record=f"{PY_REPORT}, check filled_shell_ratio")
```

The Hartree term and the ratio for one filled level at rest, $n = S = 8$ (Exercise 14.8): $-1/8$. Out [7] shows two PASS lines.

In [8], the Kohn-Sham potentials.

```
m = sp.symbols("m", real=True)
e_int = sp.expand(e_H + e_x)
M_eff = m + sp.diff(e_int, S)  # the effective mass
v_v = sp.diff(e_int, n)  # the vector potential
say(f"e_int = {e_int}")
say(f"M_eff = {M_eff},  v_v = {v_v}")
```

The bare mass m, $e_{\mathrm{int}} = e_H + e_x$, and the two potentials as derivatives (Section 14.27). Out [8] prints $e_{\mathrm{int}} = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2$ and the potentials in sympy's notation.

```
check(sp.simplify(M_eff - m - sp.Rational(15, 16) * lam * S) == 0
      and sp.simplify(v_v + lam * n / 16) == 0 and record_check("ks_potentials"),
      "M_eff = m + (15/16) lambda S and v_v = -lambda n/16",
      record=f"{PY_REPORT}, check ks_potentials")
on_shell = sp.simplify(M_eff * S + v_v * n - m * S - e_int - e_int)
check(on_shell == 0 and record_check("ks_onshell_lagrangian"),
      "on shell <L>/sqrt|g| = M_eff S + v_v n - m S - e_int = e_int",
      record=f"{PY_REPORT}, check ks_onshell_lagrangian")
```

The two potentials, and the on-shell Lagrangian: $M_{\text{eff}}S + v_v n - mS - e_{\text{int}}$ minus e_{int} must be zero.

```
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json")
                    .read_text(encoding="utf-8"))
potentials = theory["exchange"]["kohnShamPotentials"]
gas = theory["exchange"]["uniformGas"]
inputs = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))["theoryInputs"]
```

The parts of the theory record that hold the coefficients, and the coefficients that the Rust solver read (its parameters record).

```
same = (sp.Rational(potentials["Meff_coefficient_of_lambda_S"]) == sp.Rational(15, 16)
        and sp.Rational(potentials["vv_coefficient_of_lambda_n"]) == sp.Rational(-1, 16)
        and sp.Rational(gas["coefficient_n2"]) == sp.Rational(-1, 32)
        and sp.Rational(gas["coefficient_S2"]) == sp.Rational(-1, 32)
        and inputs["MeffCoefficientOfLambdaS"] == 0.9375
        and inputs["vvCoefficientOfLambdaN"] == -0.0625
        and inputs["exchangeCoefficientN2"] == -0.03125
        and inputs["exchangeCoefficientS2"] == -0.03125)
check(same and record_check("ks_theory_json_exchange"),
      "the coefficients 15/16, -1/16, -1/32, -1/32 equal the records",
      record="Revision/kohn_sham/ks-theory.json, exchange; "
             "Revision/kohn_sham/results/parameters.json, theoryInputs")
```

The theory record stores the coefficients as texts such as "15/16", which `sp.Rational` turns into exact fractions; the solver's record stores them as decimal numbers, $0.9375 = 15/16$, $-0.0625 = -1/16$ and $-0.03125 = -1/32$ (all four are exact in binary floating point). The PASS line names both records with the parts that hold the coefficients: the section exchange of the theory record and the section theoryInputs of the solver's parameters record (two strings written next to each other are joined into one). Out [8] shows three PASS lines.

In [9], figure 2.

```
s_over_n = np.linspace(-1.0, 1.0, 401)
hartree = 0.5 * s_over_n ** 2
exchange_curve = -(1.0 + s_over_n ** 2) / 32.0
fig, ax = plt.subplots(figsize=(7.5, 4.4))
ax.plot(s_over_n, hartree, label="Hartree $e_H/(\\lambda n^2) = (S/n)^2/2$")
ax.plot(s_over_n, exchange_curve, "--",
        label="exchange $e_x/(\\lambda n^2) = -(1 + (S/n)^2)/32$")
ax.plot(s_over_n, hartree + exchange_curve, ":", color="black",
        label="$e_{\\rm int} = e_H + e_x$")
```

The ratio S/n from -1 to 1 and the three energy densities divided by λn^2: Hartree $\frac{1}{2}(S/n)^2$, exchange $-\frac{1}{32}(1 + (S/n)^2)$ and their sum.

```
ax.plot([1.0], [-1.0 / 16.0], "o", color="C3")
ax.annotate("$S = n$: $e_x = -e_H/8$", xy=(1.0, -1.0 / 16.0), xytext=(-0.05, 0.3),
```

```
            fontsize=8, color="C3", arrowprops={"arrowstyle": "->", "color": "C3"})
ax.axhline(0.0, color="gray", linewidth=0.6)
ax.set_xlabel("ratio of the scalar to the number density $S/n$")
ax.set_ylabel("energy density per $\\lambda n^2$")
ax.set_title("the Hartree-Fock energy of the contact interaction")
ax.legend(fontsize=8, loc="upper center")
save_figure(fig, "energy_densities",
            "The interaction energy densities of the contact term per unit "
            ...)
```

A red dot at $S/n = 1$ on the exchange curve, $-\frac{1}{16}$, with an arrow from its label (`annotate` with `xy` the point, `xytext` the place of the text and `arrowprops` the style of the arrow), the zero line, labels and saving. **What figure 14d.2 shows.** The Hartree parabola is zero at $S = 0$ and $\frac{1}{2}$ at $S = \pm n$. The exchange curve is small and negative everywhere, between $-\frac{1}{32}$ and $-\frac{1}{16}$. Their sum is slightly negative near $S = 0$, where only the exchange is left, and positive for larger $|S|$. At $S = n$ the exchange $-\frac{1}{16}$ is one eighth of the Hartree term $\frac{1}{2}$.

In [10], the exact Fock exchange of the closed-shell states.

```
as_number = {"0": 0, "1": 1, "-1": -1, "I": sp.I, "-I": -sp.I}
U = sp.Matrix([[as_number[e] for e in row] for row in
               theory["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
V = U / (2 * sp.sqrt(2))  # the block basis of the record
labels = [(j, s2, s3) for j in (1, -1) for s2 in (1, -1) for s3 in (1, -1)]
s1 = sp.Matrix([[0, 1], [1, 0]])
s2m = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
```

The block basis V of the record (as in In [13] of Notebook 14a), the eight block labels in the record's order, and the Pauli matrices (`s2m` is σ_2).

```
d0, d1, d2, d3 = sp.symbols("d0 d1 d2 d3", real=True)
D = (d0 * sp.eye(2) + d1 * s1 + d2 * s2m + d3 * s3) / 2  # the block density
G_slab = Z16
for b, (j, _, _) in enumerate(labels):
    Vb = V[:, 2 * b:2 * b + 2]
    G_slab = G_slab + Vb * (D if j == 1 else s3 * D * s3) * Vb.H
```

A general 2×2 Hermitian block density D with four real numbers $d_0, \ldots, d_3$ (every Hermitian 2×2 matrix is a combination of 1 and the three Pauli matrices), and the 16×16 matrix of one level that has D in the four blocks of type $+1$ and $\sigma_3 D \sigma_3$ in the four of type -1: each block's 2×2 matrix is carried back to 16 components with its two basis columns, $V_b D V_b^\dagger$.

```
unknowns = sp.Matrix(16, 16, sp.symbols("x0:256"))  # a general 16 x 16 matrix
equations = []
for rotation in (g1 * g2, g1 * g3, g2 * g3):  # commute with the 3-space rotations
    equations += list(rotation * unknowns - unknowns * rotation)
solution = sp.Matrix(16, 16, list(list(sp.linsolve(equations, list(unknowns)))[0]))
```

The matrices that commute with the three generators $\gamma^{(x_1)}\gamma^{(x_2)}$, $\gamma^{(x_1)}\gamma^{(x_3)}$, $\gamma^{(x_2)}\gamma^{(x_3)}$ of the rotations of 3-space: a general matrix of 256 unknowns, the 3×256 linear equations "commutator $= 0$", and their general solution by `sp.linsolve`, which returns the set of solutions; `list(...)[0]` takes its single element, a list of 256 expressions in the unknowns that remain free.

```
free = sorted(solution.free_symbols, key=lambda s_: int(str(s_)[1:]))
basis = [solution.subs({x: (1 if x == y else 0) for x in free}) for y in free]
gram = sp.Matrix(len(basis), len(basis),
                 lambda i, k: (basis[i].H * basis[k]).trace())
coefficients = gram.LUsolve(sp.Matrix([(b_.H * G_slab).trace() for b_ in basis]))
G_avg = sp.simplify(sum((c_ * b_ for c_, b_ in zip(coefficients, basis)), Z16))
```

The free unknowns, sorted by their numbers (`str(s_)[1:]` is the name without its first letter); a basis of the solution space, one matrix for each free unknown set to 1 and the others to 0 (64 of them); the **Gram matrix** of the inner products $\mathrm{Tr}(X^\dagger Y)$ of the basis matrices; and the orthogonal projection of G_{slab} onto the space, found by solving the linear equations with the Gram matrix (`LUsolve`). The projection is the average over all directions of the shell.

```
rho_slab = G_avg * B
n_s, S_s = sp.simplify((B * rho_slab).trace()), sp.simplify((C * rho_slab).trace())
Q_s = sp.simplify((B * g0 * rho_slab).trace())
e_x_slab = sp.expand(-lam / 2 * (C * rho_slab * C * rho_slab).trace())
say(f"commutant dimension {len(basis)}; n = {n_s}, S = {S_s}, Q = {Q_s}")
say(f"V unitary: {sp.simplify(V.H * V) == I16}")
```

The averaged one-body matrix, its three densities n, S and $Q = \langle \Psi^\dagger B \gamma^{(x_0)} \Psi \rangle$, and its exchange energy. Out [10] prints the dimension 64, $n = 8d_0$, $S = 8d_2$, $Q = 8d_3$, and that V is unitary.

```
check(len(basis) == 64 and n_s == 8 * d0 and S_s == 8 * d2 and Q_s == 8 * d3
      and sp.simplify(e_x_slab + lam / 32 * (n_s ** 2 + S_s ** 2 - Q_s ** 2
                                             - (8 * d1) ** 2)) == 0
      and record_check("exchange_slab_exact_fock"),
      "exact Fock exchange of the slab: -(lambda/32)(n^2 + S^2 - Q^2 - Y^2)",
      record=f"{PY_REPORT}, check exchange_slab_exact_fock")
```

The check of Section 14.27 with $Y = 8d_1$. This cell does the heaviest algebra of the notebook. One PASS line.

In [11], figure 3.

```
grid = np.linspace(-1.0, 1.0, 401)
S_grid, Q_grid = np.meshgrid(grid, grid)  # S/n horizontally, Q/n vertically
inside = S_grid ** 2 + Q_grid ** 2 <= 1.0
ratio_grid = np.where(inside, (1.0 + S_grid ** 2 - Q_grid ** 2) / (1.0 + S_grid ** 2),
                      np.nan)
```

A square grid of 401×401 points in the plane of S/n and Q/n (`np.meshgrid` makes the two coordinate arrays of the grid), the points inside the unit disk, and there the ratio of the exact to the uniform-gas exchange, $(n^2 + S^2 - Q^2)/(n^2 + S^2)$ with $Y = 0$; outside the disk the value `np.nan` ("not a number"), which is not drawn.

```
fig, ax = plt.subplots(figsize=(6.4, 5.2))
filled = ax.contourf(S_grid, Q_grid, ratio_grid, levels=np.linspace(0.0, 1.0, 11),
                     cmap="Blues")
ax.contour(S_grid, Q_grid, ratio_grid, levels=[0.5], colors="gray", linewidths=0.8)
fig.colorbar(filled, ax=ax, label="$e_x^{\\rm exact} / e_x^{\\rm gas}$")
```

`contourf` fills the regions between the levels $0, 0.1, \ldots, 1$ of the ratio with colours, `contour` draws the line where the ratio is $\frac{1}{2}$, and the colour bar explains the colours.

```
ax.plot([0.0, 0.0], [1.0, -1.0], "o", color="C3")  # S = 0, Q = +n and -n
ax.annotate("zero modes ($N = 8$): ratio 0", (0.05, 0.9), fontsize=8, color="C3")
ax.annotate("$Q = 0$: ratio 1", (-0.95, 0.04), fontsize=8,
            color="white")  # white text on the dark band
ax.set_xlabel("$S/n$")
ax.set_ylabel("$Q/n$")
ax.set_aspect("equal")
ax.grid(False)
ax.set_title("exact Fock exchange / uniform-gas exchange")
save_figure(fig, "exact_fock_ratio",
            "The ratio of the exact Fock exchange of a closed-shell slab state, "
            ...)
```

Two red dots at $S = 0$, $Q = \pm n$, two labels, equal scales on both axes (`set_aspect("equal")`, so that the disk is round), and saving. **What figure 14d.3 shows.** A dark band (ratio near 1) across the middle of the disk, around the line $Q = 0$, where the uniform-gas formula is exact; toward the top and the bottom the colour fades and the ratio falls to 0 at the two red dots; the gray line marks the ratio $\frac{1}{2}$. The state of the eight zero modes sits at the top dot: for it the uniform-gas formula gives a nonzero exchange, while the exact exchange is zero.

In [12], the state of the eight zero modes.

```
H_val, m_val, L_val = 1.0, 1.0, 3.0
ell = 2.0 * math.pi / 0.25  # the coordinate size of the 3-torus
vol7 = ell ** 3 * 1.0  # the coordinate 7-volume ell^3 v_t
y = np.linspace(-L_val, 0.0, 1801)  # the hidden coordinate
a_sq = 2.0 * m_val / (1.0 - math.exp(-2.0 * m_val * L_val)) * np.exp(2.0 * m_val * y)
P = np.exp(-6.0 * H_val * y) / vol7  # proper-density factor
n_y = 4.0 * P * a_sq  # line 2
n_max = float(np.max(n_y))
```

The canonical numbers $H = m = 1$, $L = 3$, $\ell = 2\pi/0.25$, $v_t = 1$; 1801 points of y; a^2 of the normalised zero mode; the factor P; the proper density $n = 4Pa^2$ of Section 14.27; and its largest value.

```
summary = repository_file("Revision/kohn_sham/results/ground/summary.csv") \
    .read_text(encoding="utf-8").splitlines()
header = summary[0].split(",")
free8 = dict(zip(header, [r for r in summary if r.startswith("N8_lam0_a00,")][0]
                 .split(",")))
report("largest proper density of the free N = 8 state", f"{n_max:.10f}",
```

```
       "per unit proper 7-volume")
check(abs(n_max / float(free8["n_max"]) - 1.0) < 1e-9 and float(y[np.argmax(n_y)])
      == -3.0,
      "n_max = 4 P a^2 at the tip y = -3 equals the record",
      record="Revision/kohn_sham/results/ground/summary.csv, N8_lam0_a00, n_max")
```

The summary record of the Rust ground states, its row for $N = 8$, $\lambda = 0$, slice 0 (`startswith` finds the row whose text starts with its name), the RESULT line 82.2208638894, and the check: the notebook's value equals the record's to 10^{-9} (relative), and the maximum lies at $y = -3$ (`np.argma x` gives the position of the largest entry).

```
def round_significant(x, digits):
    exponent = math.floor(math.log10(abs(x))) - (digits - 1)
    scale = 10.0 ** (-exponent)
    return math.copysign(math.floor(abs(x) * scale + 0.5), x) / scale
```

Rounding to a number of significant digits, halves away from zero: for $x = 0.019460\ldots$ and 4 digits, $\log_{10}|x| = -1.71$, rounded down to -2, so the exponent is -5; the number is multiplied by 10^5 to $1946.0\ldots$, rounded to the whole number 1946 (adding $\frac{1}{2}$ and rounding down), and divided back: 0.01946.

```
strength = n_max / 16.0  # line 4: the largest first-order potential per lambda
lambda_1 = round_significant(0.1 / strength, 4)
lambda_2 = round_significant(0.3 / strength, 4)
parameters = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                        .read_text(encoding="utf-8"))
calibration = [c for c in parameters["couplingCalibration"]["values"]
               if c["N"] == 8.0][0]
```

The strength $n_{\max}/16$ (with $S = 0$ only the term $|n|/16$ counts), the two couplings by the record's rule, and the record's calibration entry for $N = 8$.

```
report("strength per lambda", f"{strength:.9f}")
report("lambda_1, lambda_2", f"{lambda_1}, {lambda_2}")
report("first-order potentials", f"{lambda_1 * strength:.10f}, "
       f"{lambda_2 * strength:.10f}", "m")
check(abs(strength / calibration["strengthPerLambda"] - 1.0) < 1e-9
      and lambda_1 == calibration["lambda1"] and lambda_2 == calibration["lambda2"]
      and abs(lambda_1 * strength - calibration["firstOrderPotential1"]) < 1e-9
      and len(set(calibration["strengthPerLambdaAtSlices"])) == 1,
      "strength 5.1388, lambda_1 = 0.01946, lambda_2 = 0.05838 (the same at all "
      "slices)", record="Revision/kohn_sham/results/parameters.json, "
      "couplingCalibration N = 8")
```

Three RESULT lines (5.138803993; 0.01946, 0.05838; the first-order potentials 0.1000011257 and $0.3000033771\,m$, just above 0.1 and 0.3 because of the rounding of λ) and the check against the record; the last condition says that the record found the same strength at all five slices (a set keeps each value once, so its size is 1). That is the rescaling identity again: the zero modes do not depend on the slice.

```
exx = repository_file("Revision/kohn_sham/results/exx/exact-fock-variant.csv") \
    .read_text(encoding="utf-8").splitlines()
exx_header = exx[0].split(",")
rows8 = [dict(zip(exx_header, r.split(","))) for r in exx[1:] if r.startswith("N8_")]
largest = max(abs(float(r["E_exact_fock_scf"])) for r in rows8)
check(len(rows8) == 20 and largest < 1e-12,
      "the self-consistent exact-Fock energy of all 20 N = 8 states is 0",
      record="Revision/kohn_sham/results/exx/exact-fock-variant.csv, "
             "E_exact_fock_scf")
```

The record of the exact-Fock variant, its 20 rows with $N = 8$, and the check that their self-consistent energies are all below 10^{-12} in size: zero up to rounding, as Section 14.27 predicts. Out [12] shows three PASS lines.

In [13], figure 4.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.5, 4.0))
fig.subplots_adjust(wspace=0.32)  # room for the label of the right panel
left.semilogy(y, n_y, label="proper density $n(y) = 4Pa^2$")
left.semilogy(y, np.exp(6.0 * y) * n_y, "--",
              label="coordinate density $e^{6Hy} n(y)$")
left.set_xlabel("$y$ (units of $1/H$; tip at $-3$, brane at $0$)")
left.set_ylabel("density (logarithmic)")
left.set_title("the eight zero modes ($N = 8$, $\\lambda = 0$)")
left.legend(fontsize=8)
```

Left panel, logarithmic: the proper density and the coordinate density $e^{6Hy}n$ (the number of particles per unit y and per unit coordinate volume of the other directions).

```
right.plot(y, -lambda_1 * n_y / 16.0, label=f"$v_v$, $\\lambda_1 = {lambda_1}$")
right.plot(y, -lambda_2 * n_y / 16.0, label=f"$v_v$, $\\lambda_2 = {lambda_2}$")
right.plot(y, np.zeros_like(y), ":", color="black",
           label="exact Fock: $v_v + w_Q = 0$")
right.set_xlabel("$y$ (units of $1/H$)")
right.set_ylabel("first-order potential (units of $m$)")
right.set_title("the Kohn-Sham potential of the zero modes")
right.legend(fontsize=8, loc="lower right")
save_figure(fig, "zero_mode_state",
            "The state of the eight brane zero modes ($N = 8$, $m = H = 1$, "
            ...)
```

Right panel: the first-order potential $v_v = -\lambda n/16$ for the two couplings and the zero exact-Fock potential. **What figure 14d.4 shows.** Left: two straight lines on the logarithmic axis. The coordinate density (dashed) rises like e^{2y} toward the brane, from about 10^{-6} to 5×10^{-4}: the particles sit at the brane. The proper density (solid) falls like e^{-4y} from 82 at the tip to 5×10^{-4} at the brane, where the two meet ($e^{6Hy} = 1$ there): per unit proper volume the gas is densest at the tip, because the proper 7-volume is tiny there. Right: the potential is zero over most of the interval and dips to -0.1 and $-0.3\,m$ only near the tip; the exact-Fock potential is zero everywhere.

In [14], the last check.

```
figure_names = ["14d_1_pair_average.png", "14d_2_energy_densities.png",
                "14d_3_exact_fock_ratio.png", "14d_4_zero_mode_state.png"]
check(all(output_file(f"{FIGURE_FOLDER}/{name}").is_file() for name in figure_names),
      "all four figure files of this notebook exist")
all_checks_passed()
```

The four figure files must exist; the last line prints ALL 16 CHECKS PASSED (notebook 14d). The 16 checks are: 1 in In [2], 1 in In [3], 3 in In [4], 1 in In [6], 2 in In [7], 3 in In [8], 1 in In [10], 3 in In [12] and 1 in In [14].

14.32 What we proved, what we computed, what we assumed

PROVED (exact; the derivation is in this chapter, and the Revision record checks it in `Revision /kohn_sham/reports/ks-theory-python.json`, with the checks marked "both" also in `Revision /kohn_sham/reports/ks-theory-wolfram.json`):

statement (section; notebook)	**record checks**
the hidden coordinate, the warped form, the volume factor e^{6Hy} without a_4 (Section 14.2; Notebook 14a)	`geometry_hidden_coordinate` and `geometry_sqrt_det` (both), `geometry_warped_form`
$\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ along every history; the pieces of the inflating and the deflating directions cancel (Section 14.4; Notebook 14a)	`spin_connection_compatibility`, `spin_connection_slash_3H`, `spin_connection_time_terms_cancel` (both), `spin_connection_time_term_value`
the factor W^{-3} removes the spin connection; the Hamiltonian is Hermitian (Section 14.4; Notebook 14a)	`ansatz_removes_spin_connection`, `ansatz_without_W3_term_survives`, `hamiltonian_16_hermitian` (both)
eight blocks of two components, two block types, the block forms and the block Hamiltonian (Section 14.5; Notebook 14a)	`blocks_commuting_set`, `blocks_basis_unitary`, `blocks_forms`, `block_hamiltonian`, `block_ode_equivalent`, `block_type_relation` (both), `blocks_projectors`, `blocks_not_everything_block_diagonal`
the chirality matrix Γ pairs the block types and reverses the mass (Section 14.6; Notebook 14a)	`blocks_relation_to_Gamma`, `block_Gamma_map` (both)
the levels depend only on the length of the 3-momentum (Section 14.6; Notebook 14a)	`rotation_invariance` (both)
the exact rescaling identity between the slices (Section 14.7; Notebooks 14a and 14c)	`rescaling_identity` (both)
the real form, the boundary term, the mirror map and the parities, the tip family, the constant current, the suppression at the tip (Section 14.12; Notebook 14b)	`bc_mirror_map_PA`, `bc_mirror_parities_of_densities`, `bc_brane_parity_conditions`, `bc_self_adjoint_boundary_term`, `bc_tip_family`, `bc_current_conserved_along_y`, `bc_tip_asymptotics` (all both)
the exact free levels at zero 3-momentum, the zero mode and the gap (Section 14.13; Notebook 14b)	`bc_exact_k0_spectra` (both)

statement (section; notebook)	record checks
the equation of the Pruefer angle; the shooting function grows with the energy; one level per label (Section 14.14; Notebook 14b)	derived in Section 14.14 (not a separate check of the record); Notebook 14b checks the equation of the angle with sympy and the labels numerically
the slope of the brane band; the band is odd in k (Section 14.19; Notebook 14c)	`brane_band_slope` (both)
$h_{\mathbf{p}}^2 = E^2$; $\rho = (nB + SC)/16$; Wick's rule; the exact local exchange; the ratio $-1/8$ (Section 14.26; Notebook 14d)	`gas_mode_projector`, `gas_angular_average`, `gas_densities`, `exchange_uniform_gas` (both), `gas_negative_energy_modes`, `hf_wick_contraction`, `filled_shell_ratio`
the Kohn-Sham potentials; the Lagrangian on shell (Section 14.27; Notebook 14d)	`ks_potentials`, `ks_onshell_lagrangian` (both)
the exact Fock exchange of the closed-shell states and the term $+\frac{\lambda}{32}Q^2$ that the functional omits (Section 14.27; Notebook 14d)	`exchange_slab_exact_fock` (both)

COMPUTED (numerical, with the record that holds the numbers and their measured accuracy):

result (notebook)	record and measured accuracy
the 54 free levels at $k = 0$ by RK4 shooting (Notebook 14b)	`Revision/kohn_sham/results/spectrum/free-k0-analytic.csv`; check `free_k0_analytic_spectra` of `Revision/kohn_sham/reports/ks-rust-solver.json`; measured largest errors against the exact levels 7.23×10^{-10} below $4m$ and 5.05×10^{-8} above (tolerances of the record 5×10^{-9} and 3×10^{-7}); the notebook reproduces both numbers and agrees with the Rust levels to 7.1×10^{-15}
fourth-order convergence (Notebook 14b)	check `refined_free_spectra_convergence_order` of `Revision/kohn_sham/reports/ks-rust-determinism.json`; measured: median error ratio 16.00 over 39 levels, largest errors 5.050×10^{-8} (canonical step) and 3.157×10^{-9} (refined step), reproduced by the notebook
the brane band and three other levels for k from 0 to 4 (Notebook 14c)	`Revision/kohn_sham/results/spectrum/brane-band.csv`; accuracy: reproduced to 10^{-11}
the numerical slope $c\,e^{-\alpha_{4,0}}$ at five slices (Notebook 14c)	`brane-band-slope.csv` (spectrum folder); check `free_brane_band_slope`; accuracy: 1.87×10^{-12} relative
the rescaling identity solved as two separate problems (Notebook 14a, the partner parameters)	`Revision/kohn_sham/results/rescaling/rescaling.csv`; checks `rescaling_identity_between_slices` and `rescaling_identity_energy_profiles`; accuracy: levels to 1.5×10^{-13}, energies and profiles to 3.3×10^{-13}

result (notebook)	record and measured accuracy
the gap of the free $N = 8$ state, 0.4307 to $0.0642\,m$ along the history (Notebook 14c)	`closed-shells.csv` (spectrum folder); accuracy: reproduced to 10^{-11}
at the cutoff $L = 3$ and the slice 0 the band does not feel the tip angle for $k \neq 0$ (Notebook 14c; the later slices are not computed)	`tip-angle.csv` (spectrum folder); check `free_tip_angle_insensitivity`; accuracy: shifts below the suppression factor
the dependence on the cutoff L from 3 to 6 (Section 14.20; the Revision record, not repeated by a notebook of this chapter)	`Revision/kohn_sham/tip_convergence/` (README and `tip-convergence-table.csv`); report `tip-convergence.json`, 7 of 7 checks PASS; the step error of every run at most 5×10^{-9}; the free results with $k \neq 0$ at $L = 3$ low by up to 1.4% (E_{KS}) and 12% (the p_0 integral) at $a_{4,0} = 2$; the interacting $N = 8$ energy at $L = 3$ about one third of its large-L value
the particle labels, the closed shells, the bulk edge and $N = 8, 136, 688$ (Notebook 14c)	check `free_particle_branch_labels`; `closed-shells.csv`; `Revision/kohn_sham/results/parameters.json`; accuracy: exact agreement of the shells; energies to 10^{-11}
the state $N = 8$: $n_{\max} = 82.2208638894$, $\lambda_1 = 0.01946$, $\lambda_2 = 0.05838$, exact-Fock energy 0 (Notebook 14d)	`Revision/kohn_sham/results/ground/summary.csv`; `parameters.json`; `Revision/kohn_sham/results/exx/exact-fock-variant.csv`; accuracy: 10^{-9} relative; energies below 10^{-12}

ASSUMED (starting points that this chapter does not derive):

- The good sector: the orbitals do not depend on the extra times x_5, x_6, x_7 (Section 14.3; Chapters 8 and 10 give the reasons).
- The Z2 mirror brane at $y = 0$, with its two parities; with it the weight $w = \frac{1}{2}$ of the densities and the mirror copy with the mass $-m$ (Section 14.12).
- The regular tip condition at the cutoff $y = -L$, $L = 3$: a choice of the numerical model. At this cutoff and the slice $a_{4,0} = 0$, where it was measured, its angle θ is harmless for the brane band at every lattice momentum $k \neq 0$ (for the other levels with $k \neq 0$ only the estimate of Section 14.12 applies; at the later slices, where the suppression factor of Section 14.12 is no longer small for the lowest lattice momenta, this is not computed), and it is decisive for the zero modes at $k = 0$ (Section 14.20). The cutoff L itself matters, as the Revision record measured for $L = 3$ to 6 (Section 14.20; `Revision/kohn_sham/tip_convergence/`): the free results with $k \neq 0$ converge, but the recorded $L = 3$ values are low by up to 1.4% in E_{KS} and 12% in the p_0 integral at $a_{4,0} = 2$ (the redshifted brane band reaches the tip); the nonzero $k = 0$ levels approach $\pm m$ only algebraically, so the bulk edge and $N = 688$ are $L = 3$ choices; the interaction energy of the brane zero modes depends strongly on L, and the couplings are calibrated at $L = 3$.
- The deflating history $a_4 = AHx_4$ with $A = 1$ is a **prescribed background**: the Kohn-Sham states are computed on it but are not its source in the field equations of a_4 (`Revision/field_e`

quations_a4/reports/ks-source-conditions.json; Chapter 17). The Kohn-Sham gas is a test field.

- The stationary-slice (adiabatic) ansatz: at each slice the instantaneous Kohn-Sham states are computed (Section 14.3); whether the gas can follow the history is measured in Chapter 15.
- The convention that counts the zero modes at $k = 0$ as particle levels (Section 14.21).
- The functional: Hartree plus the uniform-gas exchange, without correlation (an approximation; the exact Fock exchange differs by $+\frac{\lambda}{32}Q^2$ and is reported as a diagnostic, Section 14.27).
- The canonical numbers $H = m = 1$, $L = 3$, $\Delta k = 0.25$, $v_t = 1$ and the five slices.

HYPOTHESIS: none in this chapter.

OPEN: the justification of the zero-mode convention; the effect of the tip angle θ on the levels with $k \neq 0$ at the later slices, which neither the record nor a notebook computes (Section 14.20); the time-dependent (non-adiabatic) evolution of the gas along the history; the action of the gas back on a_4 (the history is not a solution of the coupled equations); the correlation energy of the contact interaction in this geometry; the limit $L \to \infty$ of the cutoff where the Revision record does not establish it (the state $N = 688$ at $a_{4,0} = 0$, and 7 of the 19 interacting states studied, whose self-consistent iteration fails before their energies meet the record's convergence rule; Section 14.20).

Honesty about pairs and charges (rule of Chapter 0). Two places of this chapter touch the pairing of universes. The chirality matrix Γ maps every solution with the mass M onto a solution with the mass $-M$ (Section 14.6), and the ASSUMED mirror brane pairs the patch, with $+m$, with a mirror copy carrying $-m$ (Section 14.12). Both are exact maps between sets of solutions; Chapters 18 and 19 prove the theorems T1 and T3 built on them. Neither says that a universe, or a pair of universes, is created: no creation process, rate or amplitude follows from these equations (Chapter 20 gives the complete list of what is and is not proved). The words particle and sea in Section 14.21 describe the filling of the levels of the normal-ordered quantum field; this chapter makes no statement about matter and antimatter, which Chapter 21 treats, and it does not claim that the theory explains why our universe contains more matter than antimatter.

14.33 Exercises

Exercise 14.1. (a) Show that in the coordinate y the volume factor is $\sqrt{|g|} = e^{6Hy}$, and compute it at $y = -3$ for $H = 1$. (b) Compute the momentum weight κ at the cutoff $y = -3$ and at the brane $y = 0$, for the slices $a_{4,0} = 0$ and 2 ($H = 1$). (c) A density per unit y and per unit coordinate volume is turned into a proper density by dividing it by $\sqrt{|g|}$. Explain with (a) why the proper density of the zero modes is largest at the cutoff, although the orbitals sit at the brane.

Answer 14.1. (a) The scale factors are $e^{a_4}W$ (three times), 1, $e^{-a_4}W$ (three times) and 1, with $W = e^{Hy}$ (Section 14.2). The square root of the absolute value of the determinant of a diagonal metric is the product of the scale factors: $(e^{a_4}W)^3(e^{-a_4}W)^3 = e^{3a_4-3a_4}W^6 = e^{6Hy}$. At $y = -3$: $e^{-18} \approx 1.52 \times 10^{-8}$. (b) $\kappa = e^{-Hy-a_{4,0}}$. At $y = -3$: $e^{3-a_{4,0}}$, which is $e^3 \approx 20.09$ at the slice 0 and $e^1 \approx 2.718$ at the slice 2. At $y = 0$: $e^{-a_{4,0}}$, which is 1 and $e^{-2} \approx 0.135$. (c) The zero-mode orbital gives the coordinate density $\propto a^2 \propto e^{2My}$, largest at the brane. Dividing by e^{6Hy} gives $e^{(2M-6H)y}$, which for $M = H = 1$ is e^{-4y}: it grows toward the tip. The proper 7-volume near the tip is so small (the factor e^{-18} at the cutoff) that the few particles there are packed densely (Figure 14d.4).

Exercise 14.2. Using the two rules of Section 14.5, show that $A_0A_1A_4 = -J$, and deduce from it the block form $A_4 \to j\sigma_1$. Then derive the block form of $\gamma^{(x_4)} = A_0A_4$.

Answer 14.2. $A_0A_1A_4 = \gamma^{(x_0)}\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_0)}\gamma^{(x_4)}$. The first two factors give $(\gamma^{(x_0)})^2 = \eta_{00} = 1$, so the product is $\gamma^{(x_1)}\gamma^{(x_0)}\gamma^{(x_4)}$. Exchanging the first two factors (they anticommute) gives $-\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_4)} = -J$. In a block of type j, J is the number j, so $A_0A_1A_4 = -j$ there. With $A_0 \to \sigma_3$ and $A_1 \to -i\sigma_2$: $\sigma_3(-i\sigma_2) = -i\sigma_3\sigma_2 = -i(-i\sigma_1) = -\sigma_1$, so $-\sigma_1 A_4 = -j$, and multiplying by $-\sigma_1$ from the left ($\sigma_1^2 = 1$): $A_4 = j\sigma_1$. Finally $\gamma^{(x_4)} = \gamma^{(x_0)}\gamma^{(x_0)}\gamma^{(x_4)} = A_0A_4 \to \sigma_3(j\sigma_1) = j\sigma_3\sigma_1 = ij\sigma_2$, as in Section 14.5.

Exercise 14.3. (a) Show that the column $\chi = (\cos\frac{\theta}{2},\ i\sin\frac{\theta}{2})$, that is $(a, b) = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2})$ in real form, satisfies $Q(\theta)\chi = \chi$ for $Q(\theta) = \cos\theta\,\sigma_3 + \sin\theta\,\sigma_2$. (b) Show that the tip condition with $\theta = \pi$ is $a(-L) = 0$, and that the matrix σ_2 of Section 14.6 turns the canonical tip condition $b(-L) = 0$ into it.

Answer 14.3. (a) For $\chi = (a, ib)$ the products of Section 14.12 give $\sigma_3\chi = (a, -ib)$ and $\sigma_2\chi = (b, ia)$, so $Q\chi = (a\cos\theta + b\sin\theta,\ i(a\sin\theta - b\cos\theta))$. With $a = \cos\frac{\theta}{2}$, $b = \sin\frac{\theta}{2}$ the first entry is $\cos\theta\cos\frac{\theta}{2} + \sin\theta\sin\frac{\theta}{2} = \cos(\theta - \frac{\theta}{2}) = \cos\frac{\theta}{2} = a$, and the bracket of the second is $\sin\theta\cos\frac{\theta}{2} - \cos\theta\sin\frac{\theta}{2} = \sin(\theta - \frac{\theta}{2}) = \sin\frac{\theta}{2} = b$ (the subtraction formulas of cosine and sine). So $Q\chi = \chi$. (b) For $\theta = \pi$, $Q = -\sigma_3$ and $(1 - Q)\chi = (1 + \sigma_3)\chi = (2\chi_1, 0) = 0$ means $\chi_1(-L) = 0$, that is $a(-L) = 0$. The map $\chi \to \sigma_2\chi$ sends (a, ib) to (b, ia), so the new components are $\tilde{a} = b$ and $\tilde{b} = a$: the condition $b(-L) = 0$ of the old orbital becomes $\tilde{a}(-L) = 0$, the condition with $\theta = \pi$. (The Rust solver's numerical self-test of the Kohn-Sham-level map, check t3_block_map_solver_selftest of `Revision/kohn_sham/reports/ks-rust-solver.json`, solves the problem with (m, λ) and the tip $\theta = 0$ and the problem with $(-m, \lambda)$ and exactly this transformed tip, $\theta = \pi$, and finds the same levels; the record states that this self-test is not a proof of T3, which Chapter 19 proves.)

Exercise 14.4. Let p be the cubic polynomial with the values f_0, f_1 and the derivatives d_0, d_1 at the two ends of an interval of length h. Show that its value at the midpoint is $\frac{1}{2}(f_0 + f_1) + \frac{h}{8}(d_0 - d_1)$, the formula used by `orbital` in Notebooks 14b and 14c.

Answer 14.4. Measure the position from the midpoint, s, so that the ends are $s = \mp h/2$, and write $p(s) = A + Bs + Cs^2 + Ds^3$; the value at the midpoint is A. Then $p(\mp h/2) = A \mp Bh/2 + Ch^2/4 \mp Dh^3/8$ and $p'(\mp h/2) = B \mp Ch + 3Dh^2/4$. Adding the two values: $f_0 + f_1 = 2A + Ch^2/2$ (the odd terms cancel). Subtracting the two derivatives: $d_0 - d_1 = -2Ch$, so $C = (d_1 - d_0)/(2h)$. Hence $A = \frac{1}{2}(f_0 + f_1) - Ch^2/4 = \frac{1}{2}(f_0 + f_1) - \frac{h}{8}(d_1 - d_0) = \frac{1}{2}(f_0 + f_1) + \frac{h}{8}(d_0 - d_1)$.

Exercise 14.5. For $M = 1$ and $L = 2$ (the second case of Notebook 14b): (a) compute the lowest positive even level; (b) show that the lowest odd root p_0 lies between $\pi/4$ and $\pi/2$ and that no root lies between 0 and $\pi/4$; (c) the record gives the lowest odd level $1.519802561206\,m$; compute p_0 from it and check (b).

Answer 14.5. (a) $\varepsilon = \sqrt{M^2 + (\pi/L)^2} = \sqrt{1 + (\pi/2)^2} = \sqrt{1 + 2.4674} = \sqrt{3.4674} = 1.862096\,m$; the record `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv` lists 1.862095889118587 (column eps_analytic, $m = 1$, $L = 2$, even, label 1). (b) With $f(p) = \sin(2p) + p\cos(2p)$: $f(\pi/4) = \sin(\pi/2) + \frac{\pi}{4}\cos(\pi/2) = 1 > 0$ and $f(\pi/2) = \sin\pi + \frac{\pi}{2}\cos\pi = -\pi/2 < 0$, so a root lies between them (Step 4 of Section 14.13); for $0 < p < \pi/4$ both $\sin(2p)$ and $\cos(2p)$ are positive, so $f(p) > 0$ and there is no root. (c) $p_0 = \sqrt{\varepsilon^2 - M^2} = \sqrt{1.519803^2 - 1} = \sqrt{2.309801 - 1} = \sqrt{1.309801} = 1.14447$, and indeed $\pi/4 = 0.785 < 1.144 < 1.571 = \pi/2$.

Exercise 14.6. (a) Show that for $M = H = 1$ the slope of the brane band is $c = 2e^{-a_{4,0}}/(1 + e^{-L})$.

(b) Evaluate it at the slice 0 for $L = 2$ and $L = 4$ and compare with Notebook 14c, Out [5]. (c) Evaluate it for $L = 3$ at the slice 1. (d) What is the limit for $L \to \infty$, and why is it not 1, the value of κ at the brane at the slice 0?

Answer 14.6. (a) Section 14.19 with $M = H = 1$: $c = e^{-a_{4,0}} \cdot \frac{2}{1} \cdot (1 - e^{-L})/(1 - e^{-2L})$, and $1 - e^{-2L} = (1 - e^{-L})(1 + e^{-L})$, so the factor $1 - e^{-L}$ cancels. (b) $L = 2$: $2/(1 + e^{-2}) = 2/1.135335 = 1.761594$; $L = 4$: $2/(1 + e^{-4}) = 2/1.018316 = 1.964028$; both equal the printed numerical and exact values. (c) $2e^{-1}/(1 + e^{-3}) = 1.905148 \times 0.367879 = 0.700865$, the value of Out [5] for the slice 1 (0.700864874900). (d) For $L \to \infty$, $e^{-L} \to 0$ and $c \to 2$. The slope is the average of $\kappa = e^{-y}$ over the zero mode, whose weight $2e^{2y}$ extends toward the tip; the average of e^{-y} with the weight $2e^{2y}$ over $(-\infty, 0]$ is $\int 2e^{y} dy = 2$, larger than the value 1 at the brane.

Exercise 14.7. Estimate the gap of the free $N = 8$ state at the slices 0 and 1 from the first-order band, $\varepsilon \approx c\, e^{-a_{4,0}} k$ with $k = 0.25$, and compare with the computed gaps of Section 14.20. Why is the estimate too large, and why is it better at the later slice?

Answer 14.7. Slice 0: $1.905148 \times 0.25 = 0.4763\, m$ against the computed $0.4307\, m$, too large by $0.0456\, m$ (about 11 per cent). Slice 1: $0.700865 \times 0.25 = 0.1752\, m$ against $0.1703\, m$, too large by $0.0049\, m$ (about 3 per cent). The band bends below its tangent line ck (Figure 14c.1), so the straight line overestimates it. By the rescaling identity the gap at the slice 1 is the band at the slice 0 at the smaller momentum $0.25e^{-1} = 0.092$, where the band is still close to its tangent: along the history the redshift moves the lattice momenta into the region where the first-order formula is accurate.

Exercise 14.8. For a single momentum at rest, $\mathbf{p} = 0$, show that the projector on the eight positive-energy modes is $G_0 = \frac{1}{2}(1 - i\gamma^{(x_4)})$, that these eight modes, filled, give $\rho = \frac{1}{2}(B + C)$, and deduce $n = S = 8$ and $e_x/e_H = -1/8$.

Answer 14.8. At $\mathbf{p} = 0$, $h_0 = M(-i\gamma^{(x_4)})$ and $E = M$, so $G_0 = \frac{1}{2}(1 + h_0/M) = \frac{1}{2}(1 - i\gamma^{(x_4)})$. The filled modes give $\rho = G_0 B = \frac{1}{2}(B + (-i\gamma^{(x_4)})B) = \frac{1}{2}(B + C)$, by $(-i\gamma^{(x_4)})B = C$ (Section 14.26). Then $n = \mathrm{Tr}(B\rho) = \frac{1}{2}(\mathrm{Tr}\, B^2 + \mathrm{Tr}\, BC) = \frac{1}{2}(16 + 0) = 8$ and $S = \mathrm{Tr}(C\rho) = \frac{1}{2}(\mathrm{Tr}\, CB + \mathrm{Tr}\, C^2) = 8$. With $n = S = 8$: $e_x = -\frac{\lambda}{32}(64 + 64) = -4\lambda$ and $e_H = \frac{\lambda}{2} \cdot 64 = 32\lambda$, so $e_x/e_H = -4/32 = -1/8$, the check filled_shell_ratio and Out [7] of Notebook 14d.

Exercise 14.9. Show that for any state built from orbitals with occupations $f \geq 0$ the densities at a point satisfy $S^2 + Q^2 \leq n^2$, so that the possible states fill the disk of Figure 14d.3. When does the equality hold?

Answer 14.9. For one orbital $\chi = (a, ib)$ at the point y the three contributions are $n_o = P(a^2 + b^2)$, $s_o = Pj\, 2ab$ and $q_o = P(a^2 - b^2)$ with $P > 0$ (Section 14.27), and $(2ab)^2 + (a^2 - b^2)^2 = (a^2 + b^2)^2$ (Section 14.27), so the vector (s_o, q_o) in the plane has the length n_o ($j^2 = 1$). The state has $n = \sum c_o n_o$, $(S, Q) = \sum c_o (s_o, q_o)$ with the non-negative weights $c_o = wgf$. The length of a sum of vectors is at most the sum of their lengths (the triangle inequality), so $\sqrt{S^2 + Q^2} \leq \sum c_o n_o = n$. The equality holds when all the vectors (s_o, q_o) with $c_o > 0$ point in the same direction, for example when only one kind of orbital is filled, as for the eight zero modes, where every vector is $(0, n_o)$ and the state sits at $S = 0$, $Q = n$.

Exercise 14.10. For the eight zero modes with $M = H = 1$, $L = 3$, $\ell = 8\pi$ and $v_t = 1$: (a) compute the largest proper density $n_{\max} = n(-3)$ from $n = 4Pa^2$; (b) compute the couplings λ_1 and λ_2 by the calibration rule of Section 14.27; (c) explain why the exact-Fock energy of this state is zero for every λ, while the uniform-gas exchange is not.

Answer 14.10. (a) $a^2 = \frac{2}{1-e^{-6}}e^{2y}$ and $P = e^{-6y}/(8\pi)^3$, so $n(-3) = 4 \cdot \frac{2}{1-e^{-6}} \cdot e^{-6} \cdot e^{18}/(8\pi)^3 = \frac{8e^{12}}{(1-e^{-6})(8\pi)^3}$. With $e^{12} = 162754.79$, $1 - e^{-6} = 0.997521$ and $(8\pi)^3 = 15875.21$: $n_{\max} = 1302038.3/(0.997521 \times 15875.21) = 82.2209$, the RESULT of Notebook 14d (82.2208638894) and the record. (b) The strength is $n_{\max}/16 = 5.13880$ ($S = 0$, so only $|n|/16$ counts); $0.1/5.13880 = 0.019460$ and $0.3/5.13880 = 0.058380$, rounded to four significant digits $\lambda_1 = 0.01946$, $\lambda_2 = 0.05838$. (c) The state has $S = 0$ and $Q = n$ at every point. The exact Fock exchange $-\frac{\lambda}{32}(n^2 + S^2 - Q^2)$ is therefore zero, and the exact-Fock potential acting on the orbitals $(a, 0)$ is $v_v + w_Q = \frac{\lambda}{16}(Q - n) = 0$: the orbitals are unchanged and their levels stay 0. The Kohn-Sham energy of the record is the sum of the filled levels minus the interaction energy integrated over the doubled patch, $E_{\mathrm{KS}} = \sum gf\varepsilon - 2\ell^3 v_t \int_{-L}^{0} e^{6Hy} e_{\mathrm{int}}\, dy$ (the second term removes the interaction that the levels count twice, as in Chapter 13; `Revision/kohn_sham/ks-theory.json`, thermodynamics.energy). Here both terms vanish: the levels are 0, and $e_{\mathrm{int}}^{\mathrm{exact}} = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda(n^2 - Q^2) = 0$. So the energy is 0. The uniform-gas exchange $-\frac{\lambda}{32}(n^2 + S^2) = -\frac{\lambda}{32}n^2$ is not zero: it misses the $+\frac{\lambda}{32}Q^2 = +\frac{\lambda}{32}n^2$ that cancels it.

15 Solving the Kohn-Sham equations along the deflating history

Chapter 14 turned the Kohn-Sham problem of the field dirac16complex in the author's primordial universe into a problem a computer can solve: pairs of real functions of one variable, the hidden coordinate y, in eight independent blocks, with an effective mass and a potential made by the quanta themselves. This chapter solves that problem. It explains, line by line, the numerical method of the Revision Kohn-Sham solver (a Rust program of the repository), runs the solver over its whole list of states, and reads from the results what happens to the gas of quanta while the three extra times x_5, x_6, x_7 deflate exponentially and ordinary 3-space inflates: its levels, gaps, energies, densities, its energy-momentum tensor, how well it can follow the moving background, and its thermodynamics. Five complete notebooks, 15a to 15e, repeat every step and reproduce the numbers of the Revision record.

15.1 What this chapter does

Why solving is a chapter of its own. The Kohn-Sham equations of Chapter 14 are not solved by writing down a formula. Their orbitals make the potentials and the potentials make the orbitals, so the equations must be solved again and again until the two agree (self-consistency). Each orbital is itself the solution of a boundary-value problem: a pair of differential equations with one condition at each end of the hidden direction, which has solutions only for special energies, the levels. The gas has hundreds of occupied orbitals, and the whole computation must be repeated for three particle numbers, five couplings, five instants of the deflating history and three temperatures. A method is needed that finds every level without missing one, that labels each level so that it can be followed from one iteration, coupling or instant to the next, and that is accurate enough that two independent programs agree to many digits. This chapter teaches that method and what it finds.

What is done, in order.

- The equations the solver must solve, the parameters of the model and the list of states it computes, called the **canonical matrix** (Section 15.2).

- How one level is found: shooting from the tip to the brane, the Pruefer angle that turns the search for levels into counting, and Newton's method inside a bracket (Section 15.3).
- How the levels of the whole gas are organised: shells of 3-momenta, sectors, label sets, the brane band and its redshift along the history (Section 15.4).
- Self-consistency: the loop, Anderson mixing, the energy of the gas, an exact symmetry of the smallest state, and the first excited state (Section 15.5).
- Notebook 15e, which writes the whole method in plain Python and reproduces numbers of the solver (Sections 15.6 to 15.9).
- The Rust solver and its canonical matrix, and what the matrix shows (Sections 15.10 to 15.15, Notebook 15a).
- The energy-momentum tensor of the gas, its conservation along the hidden direction, the energy change along the history, and why the gas cannot be the source of the history (Sections 15.16 to 15.20, Notebook 15b).
- Adiabaticity: whether the gas follows its instantaneous states while the background moves (Sections 15.21 to 15.25, Notebook 15c).
- Thermodynamics: the gas at a temperature, the chemical potential and why it must be computed in a special way (Sections 15.26 to 15.30, Notebook 15d).
- What we proved, what we computed, what we assumed (Section 15.31), and seven exercises with complete worked answers (Section 15.32).

The five notebooks. The run times are the two measured executions of each notebook, the build run and the check run, that section 4.5 of its provenance file `Revision/textbook/notebooks/<name>.PROVENANCE.md` records; they were measured on the computer that built the book, where the solver used 22 threads, while other programs ran at the same time, which explains the spread. A laptop may need two or three times as long, and Notebook 15a, whose solver divides its work over all cores, up to about ten minutes.

notebook	**what it computes**	**Rust, PASS lines, figures, run time**
15e	the solver's method by hand: shooting, Pruefer label, Newton, RK4 order, brane band, self-consistency for $N = 8$	no Rust; 18 PASS lines; 6 figures; 11 to 12 s
15a	the whole canonical matrix with the Rust solver, compared with the record	Rust; 25 PASS lines; 8 figures; 164 to 232 s
15b	the energy-momentum profiles, the conservation law along y, the energy change along the history	Rust; 18 PASS lines; 7 figures; 23 to 29 s

notebook	what it computes	Rust, PASS lines, figures, run time
15c	the adiabaticity measure, Hellmann-Feynman, a Fermi-level crossing	no Rust; 12 PASS lines; 6 figures; 10 to 15 s
15d	thermodynamics: chemical potential, free energy, entropy, heat capacity	Rust; 15 PASS lines; 7 figures; 36 s

The notebooks are placed in the order in which the ideas are needed: 15e first, because it teaches the method that the Rust solver uses in the others.

The status of every statement. Every statement of this chapter carries one of the five labels of Chapter 0.

- PROVED: the exact reduction to blocks, the boundary conditions as conditions on the orbitals, the exact free levels at zero momentum, the slope of the brane band, the rescaling identity between the instants of the history, the formulas of the densities, potentials, energy and energy-momentum tensor, the conservation law along y, the energy-change law along the history, the Hellmann-Feynman relation and the adiabaticity measure. They are verified twice in the Revision record, by sympy in `Revision/kohn_sham/reports/ks-theory-python.json` and by WolframScript in `Revision/kohn_sham/reports/ks-theory-wolfram.json`; the check names are given with each statement. The mathematical facts of the method (the monotone Pruefer phase, the derivative formula of the phase, the Hermite midpoint, the Anderson weights, the double counting of the interaction energy, the symmetry $E(-\lambda) = -E(\lambda)$ of the zero-mode state, the adiabaticity estimate, the heat-capacity formula and the balance form of the chemical potential) are derived line by line in this chapter.

- COMPUTED: every level, energy, density, pressure, gap, adiabaticity number and thermodynamic function. They come from the Rust solver; its own checks are in `Revision/kohn_sham/reports/ks-rust-solver.json` (42 checks, all PASS), its repeat and refined runs in `Revision/kohn_sham/reports/ks-rust-determinism.json` (14 checks, all PASS), its chemical potentials against 40-digit roots in `Revision/kohn_sham/reports/ks-rust-mermin-roots.json` (5 checks, all PASS), and the notebooks of this chapter reproduce them with the measured differences given with each number.

- ASSUMED: the good sector (no dependence on the extra times) and the Z2 mirror brane at $y = 0$. Three further assumptions are of a special kind and carry an extra word. CHOSEN: the regular tip condition at the cutoff $y = -L$ and the value $L = 3$ (the record calls them "chosen"); what the choice of L changes is COMPUTED in the record (`Revision/kohn_sham/tip_convergence/tip-convergence.json`, 7 checks, all PASS): at most 1.4 per cent of the energy of the free gas, except $N = 688$ at the first slice, but much for the interaction (Section 15.2, the paragraph on the cutoff). CONVENTION: which levels count as particles (the positive branch and the zero modes) and that the negative branch, the sea, is never populated, not even thermally; its justification is OPEN. PRESCRIBED BACKGROUND: the history $a_4 = AHx_4$ with $A = 1$; it is given, not solved for, the gas does not act back on it, and Section 15.16 shows why it cannot (the record `Revision/field_equations_a4/reports/ks-source-conditions.json`).

- HYPOTHESIS: none. No statement of this chapter goes beyond the equations and their numerical solutions.
- OPEN: the time-dependent (non-adiabatic) problem, a thermal treatment of the sea, the justification of the filling convention, and the limit of a large cutoff L wherever the record does not establish it.

What this chapter does not touch. No statement of this chapter concerns pairs of universes, their creation or matter and antimatter. One check of the solver's report, `t3_block_map_solver_selftest`, solves a state with the mass m and a state with the mass $-m$ and finds equal energies; the report itself calls it a numerical self-test of the program and NOT a proof of the theorem T3, which belongs to Chapter 19. Nothing computed here says that any universe is created, in pairs or otherwise. Where Section 15.26 speaks of thermal holes in the sea, it means excitations inside one universe, not pairs of universes.

Notation and units. The author's coordinates are $x_0, \dots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates with the scale factor $e^{a_4} \sin^{1/6} z$; x_4 is the time; x_5, x_6, x_7 are the three extra times, which deflate exponentially with the scale factor $e^{-a_4} \sin^{1/6} z$; x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$. The hidden coordinate is $y = \ln(\sin z)/(6H)$ (Chapter 14); it runs from the **tip** $y = -L$ (where the model cuts the space) to the **brane** $y = 0$. A **slice** is one instant of the history, and $a_{4,0}$ is the value of a_4 there. In all numbers $H = 1$ and $m = 1$, so energies, momenta and temperatures are in units of m (which equals H) and lengths in units of $1/H$; Boltzmann's constant is 1, so a temperature is an energy.

15.2 The equations the solver must solve

This section collects, without proofs, the results of Chapter 14 that the solver uses, and adds the parameters of the model. Chapter 14 proves each formula; here we only need to read them.

One orbital. At a slice $a_{4,0}$, an orbital of the gas has a 3-momentum $\mathbf{k}$ (a vector of three numbers), a block type $j = +1$ or $j = -1$, and a pair of real functions $a(y)$, $b(y)$ on the interval $-L \le y \le 0$. Its level ε and the two functions obey the **block equation in real form** (`Revision/kohn_sham/ks-theory.json`, blockEquation.realForm; checks block_hamiltonian and block_ode_equivalent):

$$a' = Ma - \big(\kappa k + j(\varepsilon - v)\big)\, b, \qquad b' = \big(j(\varepsilon - v) - \kappa k\big)\, a - Mb,$$

where the prime means d/dy, $k = |\mathbf{k}|$ is the length of the momentum, $\kappa = e^{-Hy - a_{4,0}}$ is the momentum weight of Chapter 14, $M(y)$ is the **effective mass** and $v(y)$ the **potential**. The 16-component field of the orbital is built from $\chi = (a, ib)$ and the block basis of Chapter 14. The slice enters only through κ.

The boundary conditions. At the tip the solver uses the regular tip condition $b(-L) = 0$ (CHOSEN; check bc_tip_family). At the brane the ASSUMED Z2 mirror allows two kinds of orbitals, called the two **parities**: even orbitals have $b(0) = 0$, odd orbitals have $a(0) = 0$ (check bc_brane_parity_conditions). Both conditions make the current along y vanish, and with them the block Hamiltonian is self-adjoint, so the levels are real (check bc_self_adjoint_boundary_term).

The lattice of 3-momenta. 3-space is a torus of coordinate size ℓ, so the allowed momenta are $\mathbf{k} = \Delta k\,(n_1, n_2, n_3)$ with whole numbers n_1, n_2, n_3 and $\Delta k = 2\pi/\ell = 0.25$. The numbers $n_1^2 + n_2^2 + n_3^2$

are written n_2 (a single symbol, read "n-two", not the second component), and all momenta with the same n_2 form a **shell**; they have the same length $k = \Delta k\sqrt{n_2}$, and their number is written $r_3(n_2)$. For example $r_3(0) = 1$ (only the zero vector), $r_3(1) = 6$ (the six vectors $(\pm1, 0, 0)$, $(0, \pm1, 0)$, $(0, 0, \pm1)$), $r_3(2) = 12$, $r_3(3) = 8$, $r_3(4) = 6$, $r_3(7) = 0$ (no three squares add up to 7). The levels depend only on k (check rotation_invariance), and each level of block type j in the shell n_2 belongs to $4r_3(n_2)$ orbitals: the four blocks of that type times the $r_3(n_2)$ directions (ks-theory.json, blockEquation.degeneracy). This number is the **degeneracy** g of the level. A **sector** is a triple (shell n_2, block type j, parity).

Densities. The **proper** densities (per unit of proper 7-volume) of the number of quanta and of the scalar $S = \bar{\Psi}\Psi$ are sums over the levels (ks-theory.json, densities):

$$n(y) = \sum w\, g\, f\, P\, (a^2 + b^2), \qquad S(y) = \sum w\, g\, f\, P\, j\, 2ab, \qquad P = \frac{e^{-6Hy}}{\mathrm{Vol}_7},$$

where f is the occupation of the level (1 for full, 0 for empty, a number in between at a temperature), $w = \frac{1}{2}$, and $\mathrm{Vol}_7 = \ell^3 v_t$ is the coordinate volume of 3-space times the extra-time volume $v_t = 1$. With $\ell = 2\pi/\Delta k = 8\pi$, line by line:

$$\mathrm{Vol}_7 = (8\pi)^3 \cdot 1 = 512\,\pi^3 = 15875.2137.$$

Rule: $\ell^3 = 8^3\pi^3 = 512\pi^3$, and $\pi^3 = 31.00628$. The record has the same number (`Revision/kohn_sham/results/parameters.json`, physics.Vol7). The factor e^{-6Hy} turns a density per unit y into a density per unit proper volume, because the proper volume element is $\sqrt{|g|}\,dy = e^{6Hy}dy$ (Chapter 14). The weight $w = \frac{1}{2}$ belongs to the **doubled system**: the Z2 mirror makes a copy of the patch, every orbital is normalised on the patch, $\int_{-L}^{0}(a^2 + b^2)\,dy = 1$, and the particle number $N = \sum gf$ counts the quanta of the patch and of its mirror copy together; one patch holds $N/2$.

Potentials and energy. The contact interaction $U = \frac{\lambda}{2}S^2$ with the exact local exchange of Chapters 13 and 14 gives (ks-theory.json, exchange.kohnShamPotentials; check ks_potentials)

$$e_{int} = \lambda\Big(\tfrac{15}{32}S^2 - \tfrac{1}{32}n^2\Big), \qquad M = m + \frac{\partial e_{int}}{\partial S} = m + \tfrac{15}{16}\lambda S, \qquad v = \frac{\partial e_{int}}{\partial n} = -\tfrac{1}{16}\lambda n,$$

and the **Kohn-Sham energy** of the doubled system is

$$E_{KS} = \sum g\, f\, \varepsilon - 2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy} e_{int}\, dy.$$

Section 15.5 derives why the interaction energy is subtracted here. The term $\frac{15}{32}\lambda S^2$ is the Hartree energy $\frac{\lambda}{2}S^2$ plus the scalar part $-\frac{1}{32}\lambda S^2$ of the exchange energy, and $-\frac{1}{32}\lambda n^2$ is the rest of the exchange energy (Chapter 13); the potential v therefore comes from the exchange alone, and the notebooks call it the exchange potential.

Which levels are particles (CONVENTION). The equations have levels of both signs. The convention of the record (ks-theory.json, thermodynamics.fillingConvention) is: particles occupy the **positive branch**, the levels whose value without interaction ($\lambda = 0$) at the same slice is positive, together with the **zero modes** at $k = 0$ (the levels $\varepsilon = 0$ of the free problem); the levels of the

negative branch form the normal-ordered **sea** and are never occupied, not even at a temperature. The justification of this convention is OPEN; Section 15.26 measures where it stops being reasonable.

The particle numbers. The record uses three particle numbers (`Revision/kohn_sham/results/parameters.json`, particleNumbers). $N = 8$ fills exactly the eight zero modes at $k = 0$ (four blocks of each type). The next levels above the zero modes are the **brane band**, the lowest even level of block type $j = +1$ in each shell $n_2 \geq 1$ (Section 15.4); in the free gas at $a_{4,0} = 0$ the band levels increase with n_2, so filling them shell by shell gives **closed shells** (every level either full or empty). Line by line, with $g = 4r_3(n_2)$:

$$8 + 4\big(r_3(1) + r_3(2) + r_3(3) + r_3(4)\big) = 8 + 4\,(6 + 12 + 8 + 6) = 8 + 128 = 136.$$

Rule: the eight zero modes plus four times the numbers of vectors in the shells 1 to 4. Continuing to $n_2 = 11$ with $r_3(5), \ldots, r_3(11) = 24, 24, 0, 12, 30, 24, 24$:

$$8 + 4\,(32 + 24 + 24 + 0 + 12 + 30 + 24 + 24) = 8 + 4 \cdot 170 = 688.$$

Rule: the sum $6 + 12 + 8 + 6 = 32$ of the first four shells plus the seven new shells. $N = 688$ is the largest closed shell whose highest level lies below the **bulk edge** 1.2922928, the lowest level that is not on the brane band (the odd level at $k = 0$; parameters.json, particleNumbers.bulkEdge; a value at the cutoff $L = 3$, see the end of this section); $N = 136$ is the closed shell nearest to $688/4 = 172$ (Exercise 1 checks this). The record calls 688 N_{large} and 136 N_{mid} (parameters.json, particleNumbers.N_large and particleNumbers.N_mid, with the rule in particleNumbers.rule).

The couplings. The coupling λ is a constant of the theory, the same at every slice. For each N the record chooses two values so that the interaction is a moderate perturbation along the whole history (parameters.json, couplingCalibration). In the free ground states of that N at all five slices it takes the largest value over y of $\max\left(\frac{15}{16}|S|, \frac{1}{16}n\right)$, the size of the mean-field potentials per unit λ (called the **strength**), and sets $\lambda_1 = 0.1/\text{strength}$ and $\lambda_2 = 0.3/\text{strength}$, rounded to four significant digits; then the first-order potentials stay below $0.1\,m$ and $0.3\,m$. For $N = 8$ the strength is 5.138804, so $\lambda_1 = 0.1/5.138804 = 0.019460$, rounded 0.01946, and $\lambda_2 = 0.05838$. For $N = 136$ and 688 the strengths are 107.548 and 541.713, so $\lambda_1 = 0.0009298$ and 0.0001846, $\lambda_2 = 0.002789$ and 0.0005538. The strength of $N = 136$ grows from 5.226 at the first slice to 107.548 at the last, that of $N = 688$ from 5.226 to 541.713 (parameters.json, strengthPerLambdaAtSlices): along the history the brane-band orbitals spread toward the tip, where the factor $P = e^{-6Hy}/\mathrm{Vol}_7$ is large. A calibration at the first slice alone would make the late states strongly coupled; this is why the whole history is used. The calibration is made at the cutoff $L = 3$, and the strength grows with L (the end of this section). The sign of λ decides the kind of interaction: $\lambda > 0$ is repulsive, $\lambda < 0$ attractive. The run names use the tags `lam0`, `lamp1`, `lamm1`, `lamp2`, `lamm2` for $\lambda = 0, +\lambda_1, -\lambda_1, +\lambda_2, -\lambda_2$.

The canonical matrix. The solver computes a fixed list of states. **Ground states** ($T = 0$): three particle numbers $N = 8, 136, 688$, five couplings and five slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$, that is $3 \cdot 5 \cdot 5 = 75$ states. **Thermal states**: the same three N, the three couplings $0, \pm\lambda_1$, the five slices and three temperatures $T = 0.01, 0.02, 0.05$, that is $3 \cdot 3 \cdot 5 \cdot 3 = 135$ states. A state is named like `N136_lamp2_a20` ($N = 136$, $+\lambda_2$, $a_{4,0} = 2.0$; the last two digits are ten times the slice), and a thermal state carries in addition `_T10`, `_T20` or `_T50` (a thousand times the temperature).

The numerical parameters (parameters.json, numerics): $G = 900$ steps of the Runge-Kutta method on $[-L, 0]$, a root tolerance of $10^{-13}\,m$ for each level, a self-consistency tolerance of $10^{-11}\,m$

with at most 400 iterations, Anderson mixing with depth 6 and $\beta = 0.4$, the step $\delta = 0.002$ in a_4 for derivatives along the history, and levels closer than $10^{-9}\,m$ count as equal (one degenerate group).

What the cutoff $L = 3$ changes (the choice is CHOSEN, its effect COMPUTED). The regular tip and the value $L = 3$ are choices of the model, and the record measures what they change in the folder `Revision/kohn_sham/tip_convergence/`, summarised in sections 5 and 15 of the document `Revision/docs/KOHN_SHAM_DEFLATING_FIELD.md`. It solves 29 recorded states, the 27 ground states with $N = 8, 136, 688$, $\lambda = 0, \pm\lambda_1$, $a_{4,0} = 0, 1, 2$ and two thermal states of $N = 136$, at $L = 3, 3.5, \ldots, 6$ with the record's step $h = 1/300$, and every run again with the step $h/2$. A copy of the solver that reads L from the environment reproduces the committed matrix byte for byte at $L = 3$, and the independent reference solver agrees at $L = 3$ and $L = 4$ (report `Revision/kohn_sham/tip_convergence/tip-convergence.json`, 7 of 7 checks PASS).

How the study judges the limit $L \to \infty$. The rules were fixed before the comparison and changed once, after the first analysis (sections 3 and 9 of the README of that folder; the change is told after the rule). For each computed number x, for example E_{KS}, the study forms the **successive differences** $d = x(L + 0.5) - x(L)$ over the unbroken run $L = 3, 3.5, 4, \ldots$ of values at which the self-consistent iteration converged (the run ends just before the first L at which the iteration fails, and values at larger L after a failure are not used), and the ratio r of each difference to the one before it. If every difference from some L on is no larger than the numerical noise of its two values, the number has converged within the noise, and its last value is its limit. The noise of a value x_h computed with the step h is $\frac{16}{15}|x_{h/2} - x_h|$, measured with the run at the step $h/2$, plus 10^{-11} times the larger of 1 and $|x|$: the error of the Runge-Kutta rule, which is of fourth order, falls by the factor $2^4 = 16$ when the step is halved, so $x_h - x_{h/2}$ is $\frac{15}{16}$ of the error of x_h. If the last two ratios both lie between 0 and 0.5, the study takes the differences to go on shrinking like a geometric series, each one the fraction r (the last ratio) of the one before; this is the **geometric-tail rule**, and the limit, which the record calls the extrapolated value and this chapter also the large-L value, adds the differences still missing:

$$x_\infty = x(L_{max}) + d_{last}\,(r + r^2 + r^3 + \ldots) = x(L_{max}) + d_{last}\,\frac{r}{1 - r}.$$

Rule: L_{max} is the last L of this run and d_{last} the last difference, so the next differences would be $d_{last}r, d_{last}r^2, \ldots$; the sum $s = r + r^2 + r^3 + \ldots$ obeys $s = r + r\,s$ (take the factor r out of every term after the first), so $s = r/(1 - r)$ for $0 < r < 1$. In every other case, and when this run holds fewer than three values of L (as for $N = 688$, $+\lambda_1$, $a_{4,0} = 0$, which converges at $L = 3$ and fails at $L = 3.5$; its converged values at $L = 4$, 4.5 and 5 come after the failure and are not used), the study takes no limit: the limit is not established. This condition on the last two ratios is the one change of the rules (section 9 of the README). The first analysis accepted a geometric tail whenever the last ratio was below 0.9. That gave $N = 688$, $\lambda = 0$, $a_{4,0} = 0$ an extrapolated value, although its ratios 0.77 to 0.79 are those of a tail that falls like the power L^{-3}, not geometrically. The rule above is the stricter one, and the change only removed extrapolations. This rule follows the dependence on L; it is not the Richardson extrapolation of Chapter 2, which combines two values so that their leading error term cancels. What the study finds:

- The free results with $k \neq 0$ converge faster than exponentially, because an orbital with momentum k is suppressed toward the tip like $\exp\big(-|k|(\kappa(-L) - \kappa(y))/H\big)$. The recorded $L = 3$ values are low by up to 1.4 per cent in E_{KS} and 12 per cent in the integrated pressure along the hidden direction, $2\,\mathrm{Vol}_7 \int e^{6Hy} p_0\,dy$ (the record's column `int_p0`, Section 15.16):

for $N = 136$, $\lambda = 0$, $a_{4,0} = 2$ they are 12.44507 against the extrapolated 12.62207, and 8.47270 against 9.63839.

- The levels at $k = 0$ other than the zero mode approach $\pm m$ only slowly, like $(n\pi)^2/(2mL^2)$ (Section 15.3 derives this for the even levels from their exact formula; the record verifies the exact levels at every L of the study, 84 levels, to $1.42 \times 10^{-10}\, m$). So the bulk edge 1.2922928 that defines $N = 688$ is a value at $L = 3$: it is 1.0977 at $L = 6$ and tends to m as $L \to \infty$. From $L = 3.5$ on, the ground state of $N = 688$ at $a_{4,0} = 0$ occupies levels of the bulk at $k = 0$, its gap 0.0452 closes, and its energy does not converge up to $L = 6$.

- The interacting results depend strongly on L. The proper density of the zero modes grows toward the tip like $e^{(6H-2m)|y|} = e^{4|y|}$ (exact, and verified by the record; Figure 15e.6 shows the potential it makes). For $N = 8$ the record finds $E_{KS} = \mp 9.868 \times 10^{-4}$ at $L = 3$ ($\lambda = \pm\lambda_1$, every slice) against $\mp 3.0046 \times 10^{-3}$ at large L, extrapolated from $\lambda = -\lambda_1$ at $a_{4,0} = 2$, the only interacting $N = 8$ state solved up to $L = 6$ (the $\pm\lambda_1$ partners are negatives of each other to 1.1×10^{-12} relative at every L where both converge; the other interacting $N = 8$ states extrapolate to $\mp 3.007 \times 10^{-3}$ from $L \leq 4.5$, because their self-consistent iteration fails from $L = 5$): the recorded value is about one third of the large-L value. For $N = 136$ at $a_{4,0} = 2$ the interaction shift $E_{KS}(\pm\lambda_1) - E_{KS}(0)$ is about 21 times its value at $L = 3$ at large L for $-\lambda_1$ (extrapolated), and about 28 times at $L = 5.5$ for $+\lambda_1$ (the last L before its iteration fails; its limit is not established).

- The self-consistent iteration fails at some larger L for 13 of the 19 interacting states studied; this is a failure of the iteration, not a proof that no self-consistent state exists. For 7 of them the successive differences of E_{KS} do not meet the geometric-tail rule of the study before the iteration fails, so their limit is not established; for the other 6 the values before the failure still end in a geometric tail.

- The calibration of the couplings is a statement at $L = 3$: the strength is the largest first-order mean-field potential per unit λ, and it grows with L (for $N = 8$ it is the zero-mode potential at the tip, which grows like e^{4L}). The same rule applied at larger L drives λ_1 to zero: at $L = 6$ it gives 1.2×10^{-7} for $N = 8$ (falling like e^{-4L} from 0.01946 at $L = 3$), 1.2×10^{-7} for $N = 136$ and 1.0×10^{-11} for $N = 688$.

Not computed at $L \neq 3$: the couplings $\pm\lambda_2$, the slices 0.5 and 1.5, $T = 0.01$ and the other thermal states, the excited states and the adiabaticity measure; and nothing beyond $L = 6$. Every number printed by the notebooks of this chapter is a value at $L = 3$, and so is every number of the chapter except where this paragraph is cited.

15.3 Finding one level: shooting with the Pruefer angle

The idea of shooting. For a trial energy ε, start at the tip with values that satisfy the tip condition, $(a, b) = (1, 0)$, integrate the two first-order equations of Section 15.2 from $y = -L$ to $y = 0$ with a step rule (the Runge-Kutta rule RK4 of Chapter 2), and look at the end values. If the brane condition of the wanted parity holds there ($b(0) = 0$ or $a(0) = 0$), ε is a level. If not, change ε and shoot again. The start value 1 for a is no restriction: the equations are linear in (a, b), so any other start $(c, 0)$ gives c times the same solution, and the solution is normalised at the end.

Chapter 2 introduced this method for a quantum well. Two difficulties remain: how to find every level without missing one, and how to tell which level is which. The Pruefer angle solves both.

The Pruefer angle. Write the point (a, b) in polar form, $a = r\cos\theta$, $b = r\sin\theta$, with the length $r = \sqrt{a^2 + b^2} > 0$ and the angle θ, followed continuously along y (it may grow beyond 2π). We derive the equation for θ, line by line.

$$ab' - ba' = r\cos\theta\,(r'\sin\theta + r\cos\theta\,\theta') - r\sin\theta\,(r'\cos\theta - r\sin\theta\,\theta') = r^2\theta'.$$

Rule: differentiate $a = r\cos\theta$ and $b = r\sin\theta$ with the product and chain rules; the terms with r' cancel, and $\cos^2\theta + \sin^2\theta = 1$.

$$ab' = \big(j(\varepsilon - v) - \kappa k\big)a^2 - Mab, \qquad ba' = Mab - \big(\kappa k + j(\varepsilon - v)\big)b^2.$$

Rule: multiply the second block equation by a and the first by b.

$$ab' - ba' = j(\varepsilon - v)(a^2 + b^2) - \kappa k\,(a^2 - b^2) - 2Mab.$$

Rule: subtract; the terms with $j(\varepsilon - v)$ collect to $j(\varepsilon - v)(a^2 + b^2)$, the terms with κk to $-\kappa k a^2 + \kappa k b^2$, and the two terms $-Mab$ add.

$$\theta' = j(\varepsilon - v) - \kappa k\cos 2\theta - M\sin 2\theta.$$

Rule: divide by r^2 and use $a^2 + b^2 = r^2$, $a^2 - b^2 = r^2(\cos^2\theta - \sin^2\theta) = r^2\cos 2\theta$ and $2ab = 2r^2\cos\theta\sin\theta = r^2\sin 2\theta$ (the double-angle formulas). The angle obeys an equation of its own, without r. At the tip, $(a, b) = (1, 0)$ gives $\theta(-L) = 0$ for every ε.

The phase function. Define $\Phi(\varepsilon) = j\,\theta(0)$, j times the angle at the brane. The brane conditions become conditions on Φ: $b(0) = 0$ means $\sin\theta(0) = 0$, that is $\theta(0)$ is a whole multiple of π, so $\Phi = l\pi$ with a whole number l (even parity); $a(0) = 0$ means $\cos\theta(0) = 0$, so $\Phi = \frac{\pi}{2} + l\pi$ (odd parity). (Multiplying by $j = \pm 1$ maps the set of these values onto itself.) So every level is a solution of $\Phi(\varepsilon) = t_l$ for one of the **targets** $t_l = l\pi$ (even) or $t_l = \frac{\pi}{2} + l\pi$ (odd).

Φ increases strictly. We compute $u = \partial\theta/\partial\varepsilon$, line by line.

$$u' = j + \big(2\kappa k\sin 2\theta - 2M\cos 2\theta\big)\,u, \qquad u(-L) = 0.$$

Rule: differentiate the angle equation with respect to ε (chain rule: $\partial(\cos 2\theta)/\partial\varepsilon = -2\sin 2\theta\,u$ and $\partial(\sin 2\theta)/\partial\varepsilon = 2\cos 2\theta\,u$); $u(-L) = 0$ because $\theta(-L) = 0$ for every ε.

$$(\ln r)' = \frac{aa' + bb'}{r^2} = M\cos 2\theta - \kappa k\sin 2\theta.$$

Rule: $(\ln r)' = r'/r = (aa' + bb')/r^2$ (differentiate $r^2 = a^2 + b^2$); insert the block equations, where the terms with $j(\varepsilon - v)$ cancel: $aa' + bb' = M(a^2 - b^2) - 2\kappa k\,ab$; then the double-angle formulas.

$$u' = j - 2(\ln r)'\,u.$$

Rule: the bracket of the first line is -2 times the second line.

$$(r^2u)' = 2rr'u + r^2u' = r^2\big(2(\ln r)'u + j - 2(\ln r)'u\big) = j\,r^2.$$

Rule: the product rule, $r' = r\,(\ln r)'$, and the previous line.

$$r(0)^2\,u(0) = j\int_{-L}^{0} r^2\,dy, \qquad \frac{d\Phi}{d\varepsilon} = j\,u(0) = \frac{\int_{-L}^{0} r^2\,dy}{r(0)^2} > 0.$$

Rule: integrate from the tip to the brane, with $u(-L) = 0$; then multiply by j and use $j^2 = 1$. The result is positive because $r^2 = a^2 + b^2 > 0$. So Φ **increases strictly with** ε, for both block types, any potentials and any slice. This formula is also the derivative that Newton's method needs, and the solver computes it at no extra cost while it integrates (its documentation in `Revision/kohn_sham/solver/README.md`, Method).

Φ runs from $-\infty$ to $+\infty$. On the interval the functions $|v|$, κk and $|M|$ are bounded, say by a number C_0 each. Then $j\theta' = (\varepsilon - v) - j\kappa k\cos 2\theta - jM\sin 2\theta$ lies between $\varepsilon - 3C_0$ and $\varepsilon + 3C_0$, and integrating over the length L gives $(\varepsilon - 3C_0)L \le \Phi(\varepsilon) \le (\varepsilon + 3C_0)L$. So $\Phi(\varepsilon) \approx \varepsilon L$ for large $|\varepsilon|$, and it takes every real value.

The oscillation theorem. A continuous function that increases strictly from $-\infty$ to $+\infty$ takes every value exactly once. Hence each target t_l is reached at exactly one energy: **every level has its own whole number l, its Pruefer label, and no level can be missed.** In a sector the levels are ordered by their labels. When the potentials change continuously (from one iteration of the self-consistent loop to the next, when the coupling is switched on, or from one slice to the next), a level moves continuously and keeps its label, because the target does not change. The label is therefore a name that follows a level everywhere. The particle levels of a sector are exactly the labels $l \ge l_{min}$, where l_{min} is the first label whose level without interaction is positive; in the even sectors at $k = 0$, $l_{min} = 0$ is the zero mode, counted as a particle level by the convention of Section 15.2 (the solver checks this split in every sector up to $n_2 = 30$, check free_particle_branch_labels of `Revision/kohn_sham/reports/ks-rust-solver.json`).

The test case: exact levels at $k = 0$. With $k = 0$, $v = 0$ and a constant mass $M > 0$ the levels are known exactly (ks-theory.json, boundaryConditions.exactK0Spectra; check bc_exact_k0_spectra). Line by line:

$$a'' = Ma' - j\varepsilon b' = Ma' - j\varepsilon(j\varepsilon a - Mb) = Ma' - \varepsilon^2 a + j\varepsilon Mb.$$

Rule: differentiate the first block equation (with $k = 0$, $v = 0$) and insert the second; $j^2 = 1$.

$$a'' = Ma' - \varepsilon^2 a + M(Ma - a') = (M^2 - \varepsilon^2)\,a.$$

Rule: the first block equation says $j\varepsilon b = Ma - a'$. The same two steps for b, line by line:

$$b'' = j\varepsilon a' - Mb' = j\varepsilon(Ma - j\varepsilon b) - Mb' = j\varepsilon Ma - \varepsilon^2 b - Mb'.$$

Rule: differentiate the second block equation (with $k = 0$, $v = 0$) and insert the first; $j^2 = 1$.

$$b'' = M(b' + Mb) - \varepsilon^2 b - Mb' = (M^2 - \varepsilon^2)\, b.$$

Rule: the second block equation says $j\varepsilon a = b' + Mb$. For $\varepsilon \neq 0$ this relation gives a from b, so an orbital that is not zero everywhere has a b that is not zero everywhere; the cases $\varepsilon \neq 0$ are sorted by the sign of $M^2 - \varepsilon^2$. For $\varepsilon^2 > M^2$ put $p = \sqrt{\varepsilon^2 - M^2}$; then $b'' = -p^2 b$, and the tip condition $b(-L) = 0$ leaves $b = B \sin\big(p(y + L)\big)$ with $B \neq 0$. Even parity, $b(0) = 0$: $\sin(pL) = 0$, so $p = n\pi/L$ and

$$\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}, \qquad n = 1, 2, \ldots$$

Odd parity, $a(0) = 0$: the second block equation gives $j\varepsilon a = b' + Mb$, so $a(0) = 0$ means $b'(0) + Mb(0) = 0$, that is $Bp\cos(pL) + MB\sin(pL) = 0$, or $\tan(pL) = -p/M$.

Inside the gap, $0 < \varepsilon^2 < M^2$, put $q = \sqrt{M^2 - \varepsilon^2} > 0$; then $b'' = q^2 b$. The functions $\cosh x = (e^x + e^{-x})/2$ and $\sinh x = (e^x - e^{-x})/2$ are each the derivative of the other, so $\cosh\big(q(y + L)\big)$ and $\sinh\big(q(y + L)\big)$ solve $b'' = q^2 b$, every solution is a combination of the two, and the tip condition $b(-L) = 0$ (where $\cosh 0 = 1$, $\sinh 0 = 0$) leaves $b = B \sinh\big(q(y + L)\big)$ with $B \neq 0$. Even parity needs $b(0) = B\sinh(qL) = 0$, impossible because $\sinh x > 0$ for $x > 0$. Odd parity needs $b'(0) + Mb(0) = B\big(q\cosh(qL) + M\sinh(qL)\big) = 0$, impossible because both terms in the bracket are positive. At the edge, $\varepsilon^2 = M^2$, the equation is $b'' = 0$ and the tip condition leaves $b = B(y + L)$; even parity needs $BL = 0$ and odd parity $b'(0) + Mb(0) = B(1 + ML) = 0$, both impossible for $B \neq 0$. So **no level lies in** $0 < |\varepsilon| \leq M$: the only level inside the gap is the zero mode. Finally $\varepsilon = 0$: the equations become $a' = Ma$, $b' = -Mb$, so $b = 0$ (from $b(-L) = 0$) and $a = Ce^{My}$: the **zero mode**, which satisfies the even brane condition. Normalising, $\int_{-L}^{0} C^2 e^{2My} dy = C^2(1 - e^{-2ML})/(2M) = 1$ gives $C = \sqrt{2M/(1 - e^{-2ML})}$. For $M = 1$, $L = 3$: the first even levels are $\pm\sqrt{1 + (\pi/3)^2} = \pm 1.4479719$, and the first odd level is ± 1.2922928, the bulk edge of Section 15.2. Notebook 15e reproduces all of them by shooting (Out [6]).

How these levels depend on the cutoff. Every level other than the zero mode depends on L. For the even levels, line by line:

$$\sqrt{M^2 + (n\pi/L)^2} - M = \frac{(n\pi/L)^2}{\sqrt{M^2 + (n\pi/L)^2} + M} \approx \frac{(n\pi)^2}{2ML^2} \quad (L \text{ large}).$$

Rule: multiply and divide by the sum $\sqrt{M^2 + (n\pi/L)^2} + M$ and use $(A - B)(A + B) = A^2 - B^2$; for large L the denominator tends to $2M$. So the levels at $k = 0$ approach $\pm M$ only like $1/L^2$, slowly; the record finds the same $1/L^2$ approach for the odd levels (the bulk edge is 1.2923 at $L = 3$ and 1.0977 at $L = 6$; Section 15.2, the paragraph on the cutoff). The zero mode stays at $\varepsilon = 0$ for every L; only its normalisation C depends on L.

Newton's method inside a bracket. To solve $\Phi(\varepsilon) = t_l$ the solver uses Newton's method, $\varepsilon \to \varepsilon - (\Phi(\varepsilon) - t_l)/\Phi'(\varepsilon)$, with the derivative from the formula above. Newton's method converges very fast near the root but can jump far away from a poor guess, so it is **safeguarded**: from a guess the solver first walks with doubling steps (up if $\Phi < t_l$, down otherwise) until $\Phi - t_l$ changes sign; the last two points form a **bracket** $[lo, hi]$ that contains the root, because Φ is continuous. Then it takes Newton steps, and replaces a Newton step by **bisection** (the midpoint of the bracket) whenever the Newton step would leave the bracket or the previous step did not halve $|\Phi - t_l|$; after

each step the bracket shrinks to the side where the sign changes. It stops when the bracket is shorter than the tolerance $10^{-13}\,m$ or when a Newton step moved the energy by less than that (Newton's method approaches a root from one side, so the bracket itself need not shrink). The levels are therefore found to about $10^{-13}\,m$ for a given grid.

The grid, RK4 and the order 4. The solver makes $G = 900$ steps of length $h = L/G = 1/300$. One RK4 step needs the coefficients M, v and κ at the start, the middle and the end of the step, so the potentials are stored on the **fine grid** of the $2G + 1 = 1801$ step ends and midpoints, $y_i = -L + i\,h/2$. The error of RK4 falls like h^4 (Chapter 2): halving the step divides it by $2^4 = 16$. The record measured exactly this ratio, 16.00, between its canonical run ($G = 900$) and a refined run ($G = 1800$) (check refined_free_spectra_convergence_order of `Revision/kohn_sham/reports/ks-rust-determinism.json`).

The orbital between the step ends: cubic Hermite interpolation. RK4 gives (a, b) at the step ends only, but the densities are needed on the whole fine grid. Each midpoint value is filled in from the values u_0, u_1 and the derivatives u_0', u_1' at the two ends of a step (the derivatives come from the block equation itself). Take a cubic polynomial and measure from the midpoint, $p(s) = A + Bs + Cs^2 + Ds^3$, $-h/2 \le s \le h/2$, so that the wanted value is $p(0) = A$. Line by line:

$$u_0 + u_1 = p(-h/2) + p(h/2) = 2A + \tfrac{h^2}{2}C.$$

Rule: in the sum of the two end values the odd powers of s cancel and the even powers double.

$$u_1' - u_0' = p'(h/2) - p'(-h/2) = 2Ch.$$

Rule: $p'(s) = B + 2Cs + 3Ds^2$; in the difference only the term $2Cs$ survives.

$$A = \frac{u_0 + u_1}{2} - \frac{h^2}{4}C = \frac{u_0 + u_1}{2} - \frac{h^2}{4}\cdot\frac{u_1' - u_0'}{2h} = \frac{u_0 + u_1}{2} + \frac{h}{8}\,(u_0' - u_1').$$

Rule: solve the first line for A and insert C from the second. The formula is exact for every cubic polynomial, so its error is of the order h^4, like RK4 itself.

Normalisation with Simpson's rule. The integral $\int_{-L}^{0}(a^2 + b^2)\,dy$ is computed with Simpson's rule on the fine grid: the weights are $\frac{h}{6}$ times $1, 4, 2, 4, \ldots, 2, 4, 1$ (Simpson's rule of Chapter 2 with the spacing $h/2$, whose weights are $\frac{h/2}{3}$ times the same numbers). Its error is also of the order h^4. The orbital is then divided by the square root of the integral.

A detail that matters for Newton's method. For the derivative $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2$ the solver integrates r^2 with the simpler **trapezoid rule** on the step ends while it shoots ($\frac{h}{2}(r_i^2 + r_{i+1}^2)$ per step). This derivative only steers Newton's method; an error in it changes how fast the root is found, not where it is.

15.4 From one level to the whole gas

Label sets. For a state of the gas the solver chooses the sectors and labels it will solve, its **label set**. It takes the shells $n_2 = 0, 1, 2, \ldots$ in turn, with all four sectors (two block types, two parities) of each shell, fills the free levels found so far from the lowest upward, and stops adding shells

when the lowest level of a new shell lies more than a margin above the lowest empty level (the LUMO) of that filling (in the canonical runs the margin is $0.25\,m$ plus twice the target size $0.1\,m$ or $0.3\,m$ of the coupling's first-order potential); in the occupied shells and the next one it solves three particle labels per sector, elsewhere the lowest only (`Revision/kohn_sham/solver/src/scf.rs`, the function `specs_t0`). The check ground_window_complete of the solver's report confirms for all 75 ground states that the lowest level outside the label set lies above the LUMO of the converged state, so no level that matters was left out.

The brane band. For $k > 0$ the lowest even level of block type $j = +1$ (label 0) grows out of the zero mode: it is bound to the brane, and its energy grows with k. This is the **brane band**. Its slope at $k = 0$ is known exactly (ks-theory.json, checksNumeric; check brane_band_slope):

$$\frac{d\varepsilon}{dk}\Big|_{k=0} = c\,e^{-a_{4,0}}, \qquad c = \frac{2M}{2M-H}\cdot\frac{1-e^{-(2M-H)L}}{1-e^{-2ML}}.$$

For $M = H = 1$ and $L = 3$, line by line:

$$c = \frac{2}{1}\cdot\frac{1-e^{-3}}{1-e^{-6}} = \frac{2\,(1-e^{-3})}{(1-e^{-3})(1+e^{-3})} = \frac{2}{1+e^{-3}} = 1.9051482536.$$

Rule: $2M - H = 1$; then $1 - e^{-6} = (1 - e^{-3})(1 + e^{-3})$ (the difference of two squares), and the common factor cancels; $e^{-3} = 0.0497871$.

The two block types and the sea. Without interaction the two block types mirror each other: $h_{-1} = -h_{+1}$ as operators with the same boundary conditions (check block_type_relation), so every orbital of $j = +1$ is also an orbital of $j = -1$, with the same k and parity and the opposite level. The band of $j = -1$ therefore lies at $-\varepsilon_{band}(k)$, below zero: it belongs to the sea, and the positive band levels are those of $j = +1$ only, each $4r_3(n_2)$-fold. At $k = 0$ the zero mode ($\varepsilon = 0 = -0$) belongs to both types, so the zero-mode level is $4 + 4 = 8$-fold.

The rescaling identity and the redshift. The slice enters the block equation only through $\kappa k = e^{-Hy}(k\,e^{-a_{4,0}})$. Therefore (ks-theory.json, rescalingIdentity; check rescaling_identity), without interaction,

$$\varepsilon(k;\ a_{4,0}) = \varepsilon(k\,e^{-a_{4,0}};\ 0)$$

for every level: a later slice is the first slice with every 3-momentum **redshifted** by $e^{-a_{4,0}}$. With interaction the same holds for the whole self-consistent state when the extra-time volume is changed too (Chapter 14); the solver solved each partner problem independently and found the same levels to $1.5 \times 10^{-13}\,m$ (check rescaling_identity_between_slices of `Revision/kohn_sham/reports/ks-rust-solver.json`). Three consequences:

- The levels at $k = 0$ (the zero modes and the bulk levels at $k = 0$) do not depend on the slice at all.
- The brane band increases with k (the solver checks this on $0 \le k \le 4$, check free_rescaling_relation_and_band_monotone). A level of the band at a fixed shell therefore decreases strictly along the history, because its redshifted momentum $k\,e^{-a_{4,0}}$ decreases.

- Every energy of the free gas that comes from band levels falls along the history.

The gap of $N = 8$, exactly. For $N = 8$ the highest occupied level is the zero mode ($\varepsilon = 0$) and the lowest empty one is the band level of the first shell, $k = \Delta k = 0.25$. So the Kohn-Sham gap is

$$\Delta_{KS}(a_{4,0}) = \varepsilon_{band}(0.25\, e^{-a_{4,0}};\ 0).$$

The record gives 0.4307337, 0.1703493 and 0.0641594 at $a_{4,0} = 0, 1, 2$ (Notebook 15a, Out [7]). The ratios to the first value are 0.39549 and 0.14895, larger than $e^{-1} = 0.36788$ and $e^{-2} = 0.13534$: the gap shrinks somewhat more slowly than $e^{-a_{4,0}}$. The reason is visible in Figure 15e.4: the band bends below its tangent line $c\,k$, so $\varepsilon_{band}(k)/k$ decreases with k; a redshifted momentum lies where the ratio is larger, closer to c. Exercise 2 makes this precise.

Aufbau and closed shells. At $T = 0$ the solver fills the levels from the lowest upward, with exact groups of equal levels (levels within $10^{-9}\,m$ count as equal); a group that would be filled only in part is flagged as an **open shell**. The particle numbers of Section 15.2 were chosen as closed shells, and the check ground_closed_shell confirms for all 75 ground states that no group is partly filled.

15.5 Self-consistency, the energy, and the first excited state

The loop. The unknowns of the self-consistent problem are the two functions $M(y) - m$ and $v(y)$ on the 1801 points of the fine grid, collected in one long list x of 2×1801 numbers (the solver keeps a third function for a variant with the exact Fock exchange, zero in the canonical functional). One **iteration**: (1) from the input x, solve every level of the label set (Section 15.3); (2) fill them (aufbau at $T = 0$, Fermi-Dirac at $T > 0$); (3) build the densities n and S and from them the output potentials $x_{out} = (\frac{15}{16}\lambda S,\ -\frac{1}{16}\lambda n)$; (4) the **residual** is $r = x_{out} - x$, and its size is its largest entry in absolute value. The loop stops when this size is at most $10^{-11}\,m$. Without interaction $x_{out} = 0 = x$ after one iteration.

Mixing. Taking x_{out} as the next input (plain iteration) can oscillate forever (charge sloshing, Chapter 13). **Linear mixing** takes $x + \beta r$ with $0 < \beta < 1$. **Anderson mixing** remembers the last few inputs $x_1, \ldots, x_p$ and their residuals $r_1, \ldots, r_p$ ($p \le 6$ in the solver), finds numbers $c_1, \ldots, c_p$ with $\sum c_i = 1$ that make the combined residual $\sum c_i r_i$ as small as possible, and takes

$$x_{next} = \sum_i c_i\,(x_i + \beta\, r_i), \qquad \beta = 0.4.$$

The weights, line by line. We minimise $F(c) = |\sum_i c_i r_i|^2 = \sum_{i,k} c_i c_k\, G_{ik}$, with the **Gram matrix** $G_{ik} = r_i \cdot r_k$ (the dot product of two lists, the sum of the products of their entries), under the condition $\sum_i c_i = 1$. With a Lagrange multiplier ν (Chapter 13) we make $F - 2\nu(\sum_i c_i - 1)$ stationary:

$$\frac{\partial}{\partial c_i}\Big[\sum_{k,l} c_k c_l G_{kl} - 2\nu\Big(\sum_k c_k - 1\Big)\Big] = 2\sum_k G_{ik} c_k - 2\nu = 0.$$

Rule: each c_i appears in the double sum once as c_k and once as c_l, and G is symmetric ($r_i \cdot r_k = r_k \cdot r_i$), so the two terms are equal.

$$\begin{pmatrix} G & \mathbf{1} \\ \mathbf{1}^T & 0 \end{pmatrix} \begin{pmatrix} c \\ -\nu \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Rule: divide by 2, move ν to the left as the last unknown (with the sign it carries), and add the condition $\sum_k c_k = 1$ as the last row; $\mathbf{1}$ is the column of p ones. This is a small linear system of $p+1$ equations. The solver adds 10^{-12} times the largest diagonal entry of G to the diagonal (which keeps the system solvable when two residuals are nearly equal), drops the oldest pair when it still cannot be solved, and forgets the whole history when a residual grows to more than ten times the smallest seen so far (`Revision/kohn_sham/solver/src/scf.rs`). Notebook 15e writes the same rule in Python (without the fallback for an unsolvable system, which its runs never need), and Figure 15e.5 shows how much faster it converges than linear mixing. Exercise 4 works the weights out for $p = 2$.

The energy: why the interaction energy is subtracted. The energy of the interacting gas is its kinetic and mass energy plus the interaction energy. Each level ε already contains the potential energy of its orbital in the mean field, so the sum $\sum g f \varepsilon$ counts the interaction twice. Line by line, with $\langle\cdot\rangle$ meaning $2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy}(\cdot)\,dy$ (the amount in the whole doubled box):

$$\sum g f\, \varepsilon = E_{kin} + \big\langle (M-m)S + v\, n \big\rangle.$$

Rule: the level is the expectation value of the block Hamiltonian; the terms of h with $M - m$ and v give, summed with $wgfP$ over the occupied orbitals, the densities S and n times the potentials (the definitions of S and n in Section 15.2); everything else ($-i\sigma_1 d/dy$, $m\sigma_2$, $\kappa k \sigma_3$) is collected in E_{kin}.

$$(M-m)S + v\, n = S\frac{\partial e_{int}}{\partial S} + n\frac{\partial e_{int}}{\partial n} = \lambda\Big(\tfrac{15}{16}S^2 - \tfrac{1}{16}n^2\Big) = 2\, e_{int}.$$

Rule: the potentials are the derivatives of e_{int} (Section 15.2), and e_{int} is a sum of squares, homogeneous of degree 2 (Euler's rule: $S\,\partial_S e + n\,\partial_n e = 2e$ for such a function); compare the coefficients $\frac{15}{16} = 2 \cdot \frac{15}{32}$ and $\frac{1}{16} = 2 \cdot \frac{1}{32}$.

$$E = E_{kin} + \langle e_{int} \rangle = \sum g f \varepsilon - 2\langle e_{int}\rangle + \langle e_{int}\rangle = \sum g f \varepsilon - 2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy} e_{int}\, dy.$$

Rule: insert the first two lines. This is E_{KS} of Section 15.2 (ks-theory.json, thermodynamics.energy; check ks_onshell_lagrangian for the same degree-2 identity). The solver also computes the **variational form**, $\sum g f \varepsilon - \langle (M_{in} - m) S_{out} + v_{in} n_{out} \rangle + \langle e_{int}(S_{out}, n_{out}) \rangle$, with the input potentials and output densities of the last iteration; the two agree to 2.2×10^{-13} relative in all 75 ground states (check ground_energy_two_forms).

An exact symmetry of the zero-mode state. For $N = 8$ the eight occupied orbitals are the zero modes at $k = 0$. We show that changing the sign of λ changes the sign of the energy, $E_{KS}(-\lambda) = -E_{KS}(\lambda)$. Line by line:

- At $k = 0$ the block equations of type j are $a' = Ma - j(\varepsilon - v)b$ and $b' = j(\varepsilon - v)a - Mb$. Rule: Section 15.2 with $k = 0$.

- Put $\tilde{a} = a$, $\tilde{b} = -b$, $\tilde{\varepsilon} = -\varepsilon$, $\tilde{v} = -v$ and keep M and j. Then $\tilde{\varepsilon} - \tilde{v} = -(\varepsilon - v)$, and $\tilde{a}' = M\tilde{a} - j(\tilde{\varepsilon} - \tilde{v})\tilde{b}$, $\tilde{b}' = j(\tilde{\varepsilon} - \tilde{v})\tilde{a} - M\tilde{b}$. Rule: in each equation the two sign changes ($b \to -b$ and $\varepsilon - v \to -(\varepsilon - v)$) cancel.
- So $(a, -b)$ is an orbital of the problem with $(M, -v)$ at the level $-\varepsilon$, with the same boundary conditions ($b = 0$ stays $b = 0$) and the same label (the angle θ becomes $-\theta$, so $\Phi \to -\Phi$, and the zero-mode label 0 stays 0). Rule: the previous line and the definition of the label.
- Its density $a^2 + b^2$ is unchanged and its scalar density $2jab$ changes sign: $n \to n$, $S \to -S$. Rule: the definitions of Section 15.2.
- With $-\lambda$ the potentials made by these densities are $\frac{15}{16}(-\lambda)(-S) = M - m$ and $-\frac{1}{16}(-\lambda)n = -v$: exactly the potentials assumed. So the mapped orbitals form a self-consistent state of $-\lambda$. Rule: the formulas of M and v.
- Its energy: every level changes sign, so $\sum gf\varepsilon$ does; $e_{int} = \lambda(\frac{15}{32}S^2 - \frac{1}{32}n^2)$ changes sign with λ while S^2 and n^2 stay. Hence $E_{KS}(-\lambda) = -E_{KS}(\lambda)$. Rule: the energy formula above.

This is PROVED for the mapped state; that the loop started from the free state finds exactly this state is COMPUTED (Notebook 15e finds $|E(+\lambda) + E(-\lambda)| = 0$ exactly, Out [13], and the record lists $E_{KS} = \mp 9.8684262 \times 10^{-4}$ for $\pm\lambda_1$). These are values at the cutoff $L = 3$; the record's study of larger L finds the partners negatives of each other at every L where both converge, and the energy about three times larger at large L (Section 15.2, the paragraph on the cutoff). The symmetry is special to the zero modes: for larger N the band orbitals have $k \neq 0$, the momentum term spoils the map, and repulsion raises the energy while attraction lowers it.

The first excited state: Delta-SCF. Chapter 13 defined the Kohn-Sham gap $\Delta_{KS} = \varepsilon_{LUMO} - \varepsilon_{HOMO}$ and the Delta-SCF energy $\Delta_{SCF} = E_1 - E_0$, the difference of two self-consistent energies, the second with one particle moved from the highest occupied level to the lowest empty one. The solver uses an **ensemble** form: it removes one particle uniformly from the whole highest occupied group of equal levels and adds it uniformly to the whole lowest empty group, so that every orbital of a group keeps the same occupation and the block and direction symmetry of the reduction is kept (parameters.json, conventions.deltaScf). Without interaction the levels do not depend on the occupations, so $\Delta_{SCF} = \Delta_{KS}$ exactly (Janak's theorem of Chapter 13 with constant levels); the solver checks this in all 15 free states (check excited_delta_scf_free_equals_gap, largest difference $2.0 \times 10^{-11}\, m$). With interaction the orbitals relax and the two differ by the **orbital relaxation** $\Delta_{SCF} - \Delta_{KS}$; for example for `N8_lamp2_a00` the record has $\Delta_{KS} = 0.4312996$ and $\Delta_{SCF} = 0.4312929$ (`Revision/kohn_sham/results/excited/summary.csv`).

15.6 Example: the solver's method by hand (Notebook 15e)

Notebook 15e writes the method of Sections 15.3 to 15.5 in plain Python, exactly as the Rust solver does it (`Revision/kohn_sham/solver/src/shoot.rs` and `scf.rs`), and reproduces numbers of the record: the grid and Simpson's rule; one shooting integration with the Pruefer angle; the phase function and the levels it labels (Figure 15e.1); Newton's method inside a bracket, the exact levels at $k = 0$ and the solver's own levels; the orbitals with Hermite midpoints (Figure 15e.2); the order 4 of RK4 (Figure 15e.3); the brane band, its slope and the rescaling identity (Figure 15e.4); the self-consistent loop with Anderson mixing for $N = 8$, compared with linear mixing (Figure 15e.5); the four couplings and the symmetry $E(-\lambda) = -E(\lambda)$; and the self-consistent potentials (Figure

15e.6). It needs no Rust, runs in less than half a minute, and ends with ALL 18 CHECKS PASSED (notebook 15e).

What to look for: in Figure 15e.1 the steadily rising curve and the places where it crosses the horizontal lines (each crossing is one level, and none can be skipped); in Figure 15e.3 the straight lines of slope 4; in Figure 15e.4 the three band curves that are copies of each other with the momentum axis stretched; in Figure 15e.5 the blue dots, which fall on average about tenfold per iteration, and the black crosses of the Rust solver on top of them.

15.7 How to run Notebook 15e

Step 1. What this notebook does and what it needs.

Notebook 15e (The solver's method by hand: shooting, Pruefer count and Anderson mixing) is the file `Revision/textbook/notebooks/15e_shooting_by_hand.ipynb` of the repository Dirac_claude. It re-implements in plain Python the numerical method of the Rust Kohn-Sham solver (fourth-order Runge-Kutta shooting in the hidden coordinate, the Pruefer angle that labels every level, a safeguarded Newton root, the normalisation of the orbitals, and the self-consistent loop with Anderson mixing), and uses it to reproduce the solver's exact free spectra, its convergence order, the slope of the brane band and the interacting ground states of N = 8 particles from the committed Revision record. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15e_shooting_by_hand.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 25 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15e_shooting_by_hand.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 15e_shooting_by_hand.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/15e.captions.json`
- `Revision/textbook/figures/15e_1_phase_function.png`
- `Revision/textbook/figures/15e_2_orbitals.png`
- `Revision/textbook/figures/15e_3_rk4_convergence.png`
- `Revision/textbook/figures/15e_4_brane_band.png`
- `Revision/textbook/figures/15e_5_scf_mixing.png`
- `Revision/textbook/figures/15e_6_n8_potentials.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 18 CHECKS PASSED (notebook 15e)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/15e_shooting_by_hand.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 15e_shooting_by_hand.ipynb
python -m nbconvert --execute --inplace 15e_shooting_by_hand.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

15.8 Notebook 15e: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/15e_shooting_by_hand.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Rust Kohn-Sham solver of the repository finds the levels and orbitals of the
Kohn-Sham equations of dirac16complex by **shooting**, labels every level with an
integer (the **Pruefer label**), and makes the potentials self-consistent with
**Anderson mixing**. This notebook writes the same method in plain Python, step by
step, and reproduces numbers of the committed Revision record:

- the exact levels at $k = 0$ (formulas) and the solver's numerical levels;
- the order of accuracy of the integration (the error falls 16-fold when the step is
  halved);
- the slope $c = 1.9051482536$ of the brane band and its redshift along the
  deflating history;
- the self-consistent ground states of $N = 8$ particles for the four couplings
  $\pm\lambda_1$, $\pm\lambda_2$: energies, levels and even the residual of each
  iteration of the loop;
- an exact symmetry of these states: $E_{KS}(-\lambda) = -E_{KS}(+\lambda)$.

It draws six figures. It needs no Rust: the Python code is short enough to read in
full, and it is slower than the solver only by a factor of about a hundred.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 15e (The solver's method by hand: shooting, Pruefer count and Anderson mixing)
is the file `Revision/textbook/notebooks/15e_shooting_by_hand.ipynb` of the repository
Dirac_claude. It re-implements in plain Python the numerical method of the Rust Kohn-Sham
solver (fourth-order Runge-Kutta shooting in the hidden coordinate, the Pruefer angle
that labels every level, a safeguarded Newton root, the normalisation of the orbitals,
and the self-consistent loop with Anderson mixing), and uses it to reproduce the solver's
exact free spectra, its convergence order, the slope of the brane band and the
interacting ground states of N = 8 particles from the committed Revision record. It needs
a computer with Windows 11, macOS or Linux, an internet connection for the installation,
the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
```

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home

folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15e_shooting_by_hand.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 25 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15e_shooting_by_hand.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 15e_shooting_by_hand.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/15e.captions.json‘
- ‘Revision/textbook/figures/15e_1_phase_function.png‘

- ‘Revision/textbook/figures/15e_2_orbitals.png‘
- ‘Revision/textbook/figures/15e_3_rk4_convergence.png‘
- ‘Revision/textbook/figures/15e_4_brane_band.png‘
- ‘Revision/textbook/figures/15e_5_scf_mixing.png‘
- ‘Revision/textbook/figures/15e_6_n8_potentials.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 18 CHECKS PASSED (notebook 15e)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/15e_shooting_by_hand.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
      python -m jupyterlab 15e_shooting_by_hand.ipynb
      python -m nbconvert --execute --inplace 15e_shooting_by_hand.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).
```

In [1]:

```
# HOW TO RUN NOTEBOOK 15e (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 15e (The solver's method by hand: shooting, Pruefer count and Anderson mixing)
# is the file Revision/textbook/notebooks/15e_shooting_by_hand.ipynb of the repository
# Dirac_claude. It re-implements in plain Python the numerical method of the Rust
# Kohn-Sham solver (fourth-order Runge-Kutta shooting in the hidden coordinate, the
# Pruefer angle that labels every level, a safeguarded Newton root, the normalisation of
# the orbitals, and the self-consistent loop with Anderson mixing), and uses it to
# reproduce the solver's exact free spectra, its convergence order, the slope of the
# brane band and the interacting ground states of N = 8 particles from the committed
# Revision record. It needs a computer with Windows 11, macOS or Linux, an internet
# connection for the installation, the program Git, and Python 3.12 or newer (the
# notebooks were built with Python 3.14.5) with these packages at exactly these versions:
# numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
# 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
# imports numpy and matplotlib; the other packages run Jupyter, the program that shows
# and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
```

```
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
```

```
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 15e_shooting_by_hand.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
```

```
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 25
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 15e_shooting_by_hand.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 15e_shooting_by_hand.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/15e.captions.json
# - Revision/textbook/figures/15e_1_phase_function.png
# - Revision/textbook/figures/15e_2_orbitals.png
# - Revision/textbook/figures/15e_3_rk4_convergence.png
# - Revision/textbook/figures/15e_4_brane_band.png
# - Revision/textbook/figures/15e_5_scf_mixing.png
# - Revision/textbook/figures/15e_6_n8_potentials.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 18 CHECKS PASSED (notebook 15e)
#
# and the notebook must show 6 figures below the cells that draw them.
#
```

```
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/15e_shooting_by_hand.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 15e_shooting_by_hand.ipynb
#     python -m nbconvert --execute --inplace 15e_shooting_by_hand.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "15e"  # this notebook: chapter 15, example e
```

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 15e complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Orbital**: a one-particle wave function; here a pair of real functions $a(y)$,
  $b(y)$ of the hidden coordinate $y$.
- **Ordinary differential equation (ODE)**: an equation for functions of one
  variable that contains their derivatives. Given the values at one point (the
  **initial values**) it fixes the functions everywhere.
- **RK4** (the classical fourth-order Runge-Kutta method): a recipe that advances
  the solution of an ODE by one **step** $h$ using four evaluations of the
  derivatives; its error falls like $h^4$.
- **Shooting**: guess an energy $\varepsilon$, start the orbital at one end with
  the boundary condition of that end, integrate to the other end, and change
  $\varepsilon$ until the boundary condition there holds too.
- **Pruefer angle**: the angle $\theta$ of the point $(a, b)$ seen from the origin,
  $a = r\cos\theta$, $b = r\sin\theta$, followed continuously (it may grow beyond
  $2\pi$).
- **Phase function** $\Phi(\varepsilon)$: $j$ times the Pruefer angle at the brane.
  It increases strictly with $\varepsilon$.
- **Label** $l$: the whole number that says which crossing of $\Phi$ with the
  targets $l\pi$ (even parity) or $\pi/2 + l\pi$ (odd parity) a level is.
- **Root**: a value where a function is zero. **Newton's method** improves a guess
  $x$ to $x - F(x)/F'(x)$. A **bracket** is an interval known to contain the root;
  **bisection** halves it.
- **Normalisation**: multiplying an orbital by a number so that
  $\int_{-L}^{0} (a^2 + b^2)\,dy = 1$.
- **Simpson's rule**: $\int f\,dy \approx \tfrac{h}{6}[f_0 + 4 f_{1/2} + 2 f_1 +
  4 f_{3/2} + \dots + f_G]$ on the step ends and midpoints of $G$ steps.
- **Cubic Hermite interpolation**: the value in the middle of a step from the values
  and the derivatives at its two ends.
- **Self-consistency, residual, mixing**: the potentials made from the orbitals must
  equal the potentials that made the orbitals. The residual is the largest change
  in one iteration. **Linear mixing** moves a fraction $\beta$ of the way toward the
  new potential; **Anderson mixing** combines the last few iterations so that their
  residuals cancel as far as possible.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's metric has 3-space $x_1, x_2, x_3$ (scale factor $e^{a_4}\sin^{1/6} z$),
the time $x_4$, three extra times $x_5, x_6, x_7$ that deflate exponentially (scale
factor $e^{-a_4}\sin^{1/6} z$) and the hidden direction $x_0$, with $z = 6 H x_0$.
In the hidden coordinate $y = \ln(\sin z)/(6H)$, from the tip $y = -L$ to the brane
$y = 0$, the Kohn-Sham equation of dirac16complex reduces exactly (Revision record
Revision/kohn_sham/ks-theory.json) to eight $2 \times 2$ blocks. With the orbital
written as $\chi = (a, i\,b)$, $a$ and $b$ real, a block of type $j = \pm 1$ at the
3-momentum $k$ is the real system
```

$$a' = M a - (\kappa k + j(\varepsilon - v))\,b, \qquad
b' = (j(\varepsilon - v) - \kappa k)\,a - M b, \qquad \kappa = e^{-Hy - a_{4,0}},$$

with the effective mass $M(y)$ and potential $v(y)$. Boundary conditions: at the tip $b(-L) = 0$ (chosen); at the brane $b(0) = 0$ for even and $a(0) = 0$ for odd parity (the Z2 mirror, ASSUMED).

The Pruefer angle. Write $a = r\cos\theta$, $b = r\sin\theta$. Then $a b' - b a' = r^2 \theta'$. Inserting the system:

$$a b' - b a' = j(\varepsilon - v)(a^2 + b^2) - \kappa k (a^2 - b^2) - 2 M a b,$$

because the terms $-M a b$ (from $a b'$) and $-M a b$ (from $-b a'$) add, the terms with κk give $-\kappa k a^2 + \kappa k b^2$, and the terms with $j(\varepsilon - v)$ give $j(\varepsilon - v)(a^2 + b^2)$. Dividing by r^2 and using $a^2 - b^2 = r^2\cos 2\theta$ and $2ab = r^2\sin 2\theta$:

$$\theta' = j(\varepsilon - v) - \kappa k \cos 2\theta - M \sin 2\theta .$$

At the tip $\theta(-L) = 0$ (because $b = 0$ there). Write $\psi = j\theta$. Multiplying the angle equation by j and using $j^2 = 1$, $\cos(2j\psi) = \cos 2\psi$ and $\sin(2j\psi) = j\sin 2\psi$ gives

$$\psi' = \varepsilon + G(y, \psi), \qquad
G(y, \psi) = -v - j\kappa k \cos 2\psi - M \sin 2\psi ,$$

and G does not contain ε. The **phase function** $\Phi(\varepsilon) = j\,\theta(0) = \psi(0)$ increases strictly with ε, by a comparison of two energies $\varepsilon_1 < \varepsilon_2$: let ψ_1, ψ_2 be their angles (both 0 at the tip) and $D = \psi_2 - \psi_1$. At the tip $D = 0$ and $D' = \varepsilon_2 - \varepsilon_1 > 0$, so $D > 0$ just after the tip. If D came back to 0 at a first point y_1, it would arrive there from positive values, so $D'(y_1) \le 0$; but at y_1 the two angles are equal, the two G terms cancel and $D'(y_1) = \varepsilon_2 - \varepsilon_1 > 0$. This contradiction shows $D(0) > 0$, that is $\Phi(\varepsilon_2) > \Phi(\varepsilon_1)$. (The exact derivative $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2 > 0$, which section 6 uses for Newton's method, says the same.) Φ also runs from $-\infty$ to $+\infty$: if $|v|$, κk and $|M|$ are at most C_0 on the interval, then $|G| \le 3C_0$, and integrating ψ' over the length L gives $(\varepsilon - 3C_0)L \le \Phi(\varepsilon) \le (\varepsilon + 3C_0)L$, so $\Phi(\varepsilon) \approx \varepsilon L$ for large $|\varepsilon|$. Even parity, $b(0) = 0$, means $\theta(0)$ is a whole multiple of π; odd parity, $a(0) = 0$, means $\theta(0) = \pi/2 + $ a multiple of π. Because Φ increases strictly, each target $l\pi$ (even) or $\pi/2 + l\pi$ (odd) is reached at exactly one energy: **every level has its own whole number l, and no level can be missed** (the oscillation theorem).

Exact levels at $k = 0$ with constant $M = m$ and $v = 0$ (Revision record ks-theory.json, boundaryConditions.exactK0Spectra): even parity: $\varepsilon = 0$ with the zero mode $(a, b) = (e^{My}, 0)$, and $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$, $n = 1, 2, \dots$; odd parity: $\varepsilon = \pm\sqrt{M^2 + p^2}$ with $\tan(pL) = -p/M$.

The self-consistent problem for $N = 8$. Eight particles fill the eight zero

```
modes at $k = 0$ (four orbitals for each $j$). Their proper densities are
$n = \sum w\,g\,f\,P\,(a^2 + b^2)$ and $S = \sum w\,g\,f\,P\,j\,2ab$ with
$P = e^{-6Hy}/\mathrm{Vol}_7$, $w = 1/2$, $g = 4$, $f = 1$; the potentials are
$M = m + \tfrac{15}{16}\lambda S$ and $v = -\tfrac{1}{16}\lambda n$; the energy is
$E_{KS} = \sum g f \varepsilon - 2\,\mathrm{Vol}_7 \int e^{6Hy} e_{int}\,dy$ with
$e_{int} = \lambda(\tfrac{15}{32}S^2 - \tfrac{1}{32}n^2)$. Units $H = m = 1$, $L = 3$,
$\Delta k = 0.25$, $\mathrm{Vol}_7 = (2\pi/\Delta k)^3$.

The history $a_4 = A H x_4$ is a PRESCRIBED BACKGROUND; the states are instantaneous
states at a slice $a_{4,0}$, which enters only through $\kappa$.
```

Text cell:

```
## 5. The grid

The solver integrates with $G = 900$ RK4 steps of length $h = L/G$ from $y = -L$ to
$y = 0$. Every RK4 step needs the potentials at its two ends and at its midpoint, so
the potentials live on the **fine grid** of the $2G + 1$ step ends and midpoints.
The next cell builds this grid exactly as the solver does: the coordinates $y_f$,
the factors $e^{-Hy}$ (for $\kappa$) and $e^{6Hy}$ (the proper volume), and the
Simpson weights. The check: Simpson's rule integrates $e^{y}$ from $-3$ to $0$ to
$1 - e^{-3}$ within $10^{-13}$.
```

In [2]:

```
import csv  # reads the tables (CSV files) of the Revision record
import math  # exp, sqrt, atan2, pi for single numbers

import numpy as np  # arrays of numbers

H, MASS, L = 1.0, 1.0, 3.0  # the units H = m = 1 and the tip cutoff L = 3
DK = 0.25  # the spacing of the 3-momenta
VOL7 = (2.0 * math.pi / DK) ** 3  # proper 7-volume per unit e^{6Hy} (v_t = 1)
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order


def make_grid(steps):
    """The solver's grid for `steps` RK4 steps on [-L, 0] (a dictionary)."""
    points = 2 * steps + 1  # step ends and step midpoints
    y = [-L * ((points - 1 - f) / (points - 1)) for f in range(points)]
    half = 0.5 * L / steps  # the distance between two fine points
    weights = [(4.0 if f % 2 == 1 else 2.0) * half / 3.0 for f in range(points)]
    weights[0] = weights[-1] = half / 3.0  # Simpson: 1, 4, 2, 4, ..., 4, 1 (x h/6)
    return {"steps": steps, "h": L / steps, "points": points, "y": y,
            "ew": [math.exp(-H * v) for v in y], "e6": [math.exp(6 * H * v) for v in y],
            "simpson": np.array(weights)}


GRID = make_grid(900)  # the canonical grid of the solver
Y = np.array(GRID["y"])
E6 = np.array(GRID["e6"])
```

```
simpson_test = float(np.sum(GRID["simpson"] * np.exp(Y)))
say(f"fine grid: {GRID['points']} points, step h = {GRID['h']:.6f}")
report("Simpson integral of e^y from -3 to 0", f"{simpson_test:.15f}")
check(abs(simpson_test - (1.0 - math.exp(-3.0))) < 1e-13,
      "Simpson's rule on the fine grid integrates e^y exactly to 1e-13")
```

Out [2]:

```
fine grid: 1801 points, step h = 0.003333
RESULT Simpson integral of e^y from -3 to 0 = 0.950212931632177
PASS Simpson's rule on the fine grid integrates e^y exactly to 1e-13
```

Text cell:

```
## 6. Shooting: one integration from the tip to the brane

The next cell is the heart of the method. `rhs` is the right-hand side $(a', b')$ of
the block equation. `shoot` starts at the tip with $(a, b) = (1, 0)$ (this satisfies
$b(-L) = 0$), makes $G$ RK4 steps to the brane, and follows the Pruefer angle: after
each step it computes the angle of the new point with `atan2` and adds the change of
angle (brought into $(-\pi, \pi]$) to $\theta$, so $\theta$ is followed continuously.
It also adds up $\int r^2\,dy$ with the trapezoid rule on the step ends,
$\tfrac{h}{2}(r_i^2 + r_{i+1}^2)$ per step, exactly as the solver does, for the
derivative $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2$, which Newton's method needs.
When asked (`store=True`) it keeps the values at the step ends. The potentials
`mass` and `pot` are lists of $M$ and $v$ on the fine grid.
```

In [3]:

```
def rhs(m_y, kk, je, a, b):
    """(a', b') of the block equation; kk = kappa k, je = j (eps - v)."""
    return m_y * a - (kk + je) * b, (je - kk) * a - m_y * b


def shoot(eps, k, j, a4, mass, pot, grid=GRID, store=False):
    """Integrate from the tip to the brane.  Returns (Phi, dPhi/deps, node values)."""
    h = grid["h"]
    kk = k * math.exp(-a4)  # k e^{-a4,0}; kappa k = kk e^{-Hy}
    coef = [(mass[f], kk * grid["ew"][f], j * (eps - pot[f]))
            for f in range(grid["points"])]
    a, b = 1.0, 0.0  # the tip condition b(-L) = 0
    theta, raw = 0.0, 0.0  # the followed angle and the last angle from atan2
    area, r2_prev = 0.0, 1.0  # integral of r^2 (trapezoid) and r^2 at the last node
    nodes = [(a, b)] if store else None
    for i in range(grid["steps"]):
        c0, c1, c2 = coef[2 * i], coef[2 * i + 1], coef[2 * i + 2]  # end, mid, end
        k1a, k1b = rhs(*c0, a, b)
        k2a, k2b = rhs(*c1, a + 0.5 * h * k1a, b + 0.5 * h * k1b)
        k3a, k3b = rhs(*c1, a + 0.5 * h * k2a, b + 0.5 * h * k2b)
        k4a, k4b = rhs(*c2, a + h * k3a, b + h * k3b)
        a += h / 6.0 * (k1a + 2.0 * k2a + 2.0 * k3a + k4a)
```

```
        b += h / 6.0 * (k1b + 2.0 * k2b + 2.0 * k3b + k4b)
        new = math.atan2(b, a)
        change = new - raw
        if change > math.pi:  # bring the change of angle into (-pi, pi]
            change -= 2.0 * math.pi
        elif change <= -math.pi:
            change += 2.0 * math.pi
        theta += change
        raw = new
        r2 = a * a + b * b
        area += 0.5 * h * (r2_prev + r2)  # trapezoid rule over this step
        r2_prev = r2
        if store:
            nodes.append((a, b))
    return j * theta, area / r2_prev, nodes


FREE_MASS = [MASS] * GRID["points"]  # M = m everywhere (no interaction)
FREE_POT = [0.0] * GRID["points"]  # v = 0 everywhere
phi_zero, _, _ = shoot(0.0, 0.0, 1, 0.0, FREE_MASS, FREE_POT)
report("Phi(0) for k = 0, j = +1 (the zero mode: b stays 0)", phi_zero)
check(phi_zero == 0.0, "at eps = 0 and k = 0 the Pruefer angle stays exactly 0")
```

Out [3]:

```
RESULT Phi(0) for k = 0, j = +1 (the zero mode: b stays 0) = 0.0
PASS at eps = 0 and k = 0 the Pruefer angle stays exactly 0
```

Text cell:

```
Why $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2$, line by line. Call
$u = \partial\theta/\partial\varepsilon$. (1) Differentiating the angle equation
$\theta' = j(\varepsilon - v) - \kappa k\cos 2\theta - M\sin 2\theta$ with respect
to $\varepsilon$ gives $u' = j + (2\kappa k\sin 2\theta - 2M\cos 2\theta)\,u$ (chain
rule), with $u(-L) = 0$ because $\theta(-L) = 0$ for every $\varepsilon$.
(2) The length $r$ obeys $(\ln r)' = (a a' + b b')/r^2 = M\cos 2\theta -
\kappa k\sin 2\theta$ (insert the system; the terms with $j(\varepsilon - v)$
cancel). (3) So $u' = j - 2(\ln r)'\,u$, and therefore $(r^2 u)' = r^2 u' + 2 r r' u
= j r^2$ (product rule). (4) Integrating from the tip to the brane:
$r(0)^2 u(0) = j\int r^2 dy$. (5) Since $\Phi = j\theta(0)$ and $j^2 = 1$,
$d\Phi/d\varepsilon = j u(0) = \int r^2 dy / r(0)^2 > 0$. This is the strict
increase of $\Phi$ in a formula, and the derivative that Newton's method uses.
```

Text cell:

```
## 7. The phase function and the levels it labels

The next cell evaluates $\Phi(\varepsilon)$ at 401 energies from $-4$ to $4$ for
$k = 0$, $j = +1$, without interaction, and draws $\Phi/\pi$. The levels of even
parity are where $\Phi/\pi$ crosses a whole number, those of odd parity where it
crosses a whole number plus one half. The check confirms that $\Phi$ increases at
every one of the 400 steps.
```

In [4]:

```
energies = np.linspace(-4.0, 4.0, 401)  # 401 trial energies
phase = np.array([shoot(e, 0.0, 1, 0.0, FREE_MASS, FREE_POT)[0] for e in energies])
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(energies, phase / math.pi, color=PALETTE[0], lw=2.0,
        label="$\\Phi(\\varepsilon)/\\pi$, $k = 0$, $j = +1$")
for whole in range(-4, 5):
    ax.axhline(whole, color=PALETTE[1], lw=0.8, alpha=0.7)  # even targets l pi
    ax.axhline(whole + 0.5, color=PALETTE[2], lw=0.8, ls="--", alpha=0.7)  # odd
ax.plot([], [], color=PALETTE[1], lw=0.8, label="even targets $l$")
ax.plot([], [], color=PALETTE[2], lw=0.8, ls="--", label="odd targets $l + 1/2$")
ax.set_xlabel("trial energy $\\varepsilon$ (units of $m$)")
ax.set_ylabel("$\\Phi(\\varepsilon) / \\pi$")
ax.set_title("The phase function counts the levels")
ax.set_ylim(-3.2, 3.2)
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "phase_function",
            "The phase function $\\Phi(\\varepsilon)/\\pi$ (vertical axis) of the "
            "free block $j = +1$ at $k = 0$ against the trial energy $\\varepsilon$ "
            "(horizontal axis, units of $m$). It rises steadily; it crosses a solid "
            "line (a whole number $l$) at each level of even parity and a dashed "
            "line ($l + 1/2$) at each level of odd parity. The flat part around "
            "$\\varepsilon = 0$ is the gap of the free field, with the zero mode at "
            "$\\Phi = 0$.")
check(bool(np.all(np.diff(phase) > 0.0)), "Phi increases strictly at all 400 steps")
```

Out [4]:

```
Figure 15e.1 saved as Revision/textbook/figures/15e_1_phase_function.png
PASS Phi increases strictly at all 400 steps
```

Text cell:

```
## 8. Finding a level: Newton's method inside a bracket

The next cell finds the level with label $l$: the root of $\Phi(\varepsilon) - t_l$
with the target $t_l = l\pi$ (even) or $\pi/2 + l\pi$ (odd). It is the solver's
'find_level', step for step: from a guess it walks with doubling steps (up if
$\Phi < t_l$, down otherwise) until the sign of $\Phi - t_l$ changes (a bracket),
then it takes Newton steps $\varepsilon \to \varepsilon - (\Phi - t_l)/\Phi'$ and
falls back to bisection whenever a Newton step would leave the bracket or the
previous step did not halve the error. It stops when the bracket is shorter than
$10^{-13}$, or when a Newton step moved the energy by at most $10^{-13}$ (Newton's
method approaches the root from one side, so the bracket itself need not shrink).
```

In [5]:

```
def target(parity, label):
    """Phi of the level with this label: l pi (even) or pi/2 + l pi (odd)."""
```

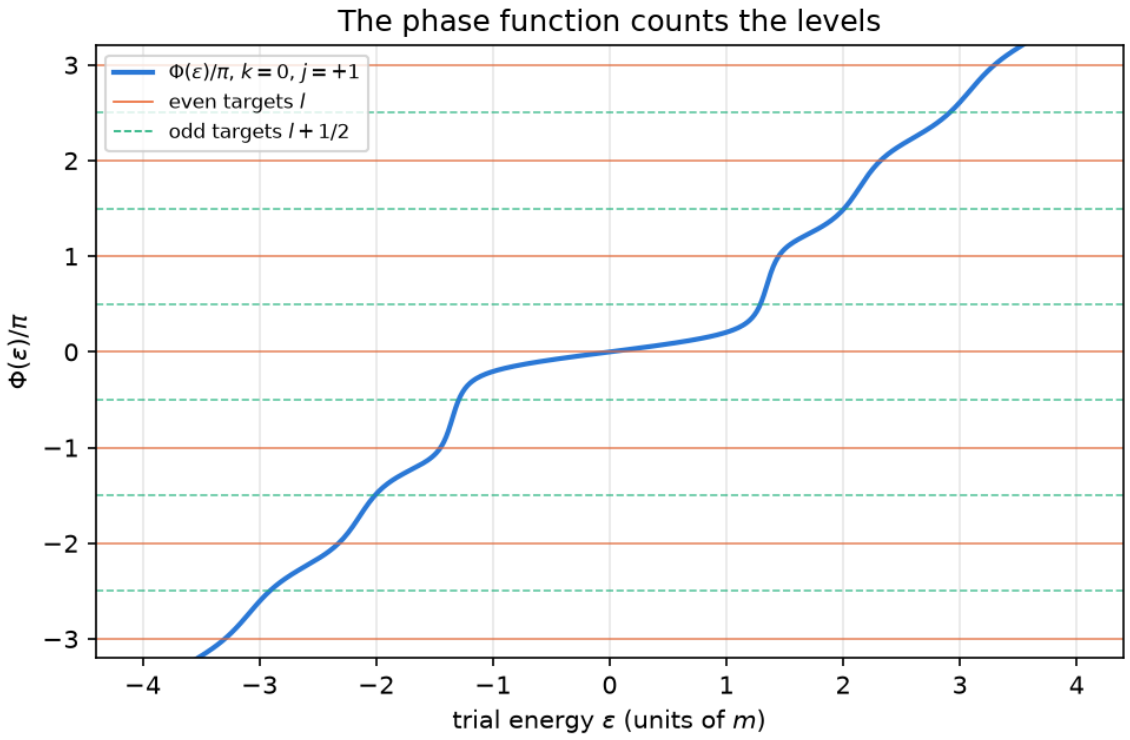

Figure 405: The phase function $\Phi(\varepsilon)/\pi$ (vertical axis) of the free block $j = +1$ at $k = 0$ against the trial energy ε (horizontal axis, units of m). It rises steadily; it crosses a solid line (a whole number l) at each level of even parity and a dashed line ($l + 1/2$) at each level of odd parity. The flat part around $\varepsilon = 0$ is the gap of the free field, with the zero mode at $\Phi = 0$. (Notebook 15e, figure 1.)

```
    return label * math.pi + (0.0 if parity == "even" else 0.5 * math.pi)


def find_level(k, j, parity, label, a4, mass, pot, guess, grid=GRID, tol=1e-13):
    """The energy of the level (k, j, parity, label) in the potentials mass, pot."""
    t = target(parity, label)
    def g(e):  # Phi - target and its derivative at the energy e
        phi, dphi, _ = shoot(e, k, j, a4, mass, pot, grid)
        return phi - t, dphi
    ec = guess
    gc, dc = g(ec)
    if gc == 0.0:
        return ec
    step = min(max(abs(gc / dc) * 1.2, 1e-4), 2.0)  # the first trial step
    if gc < 0.0:  # Phi is too small: walk up until Phi - t >= 0
        lo = ec
        while True:
            e = lo + step
            ge, de = g(e)
            if ge >= 0.0:
                hi = e  # now [lo, hi] brackets the root
                if abs(ge) < abs(gc):
                    ec, gc, dc = e, ge, de  # keep the better end for Newton
                break
            lo, ec, gc, dc = e, e, ge, de
            step *= 2.0  # double the step
    else:  # Phi is too large: walk down until Phi - t <= 0
        hi = ec
        while True:
            e = hi - step
            ge, de = g(e)
            if ge <= 0.0:
                lo = e
                if abs(ge) < abs(gc):
                    ec, gc, dc = e, ge, de
                break
            hi, ec, gc, dc = e, e, ge, de
            step *= 2.0
    if gc == 0.0:
        return ec
    previous = math.inf
    for _ in range(300):
        if hi - lo <= tol:
            break
        new = ec - gc / dc  # the Newton step
        bisect = not lo < new < hi or abs(gc) > 0.5 * previous
        if bisect:
            new = 0.5 * (lo + hi)  # bisection instead
        previous = abs(gc)
        moved = abs(new - ec)  # how far this step moved the energy
        ec = new
        gc, dc = g(ec)
        if gc == 0.0:
            break
```

```
        if gc < 0.0:
            lo = ec
        else:
            hi = ec
        if moved <= tol and not bisect:
            break  # a Newton step that moves less than tol: converged
    return ec
```

Text cell:

```
The next cell computes the exact levels at $k = 0$ (the even ones from the formula,
the odd ones by bisection on $M\sin(pL) + p\cos(pL) = 0$, which is $\tan(pL) = -p/M$
multiplied by $M\cos(pL)$), finds the labels $-3$ to $5$ of both parities by
shooting, and compares. It also compares with the solver's own numerical values in
the record file `spectrum/free-k0-analytic.csv`: the same method on the same grid
must give the same numbers to the root tolerance.
```

In [6]:

```
def odd_momentum(n):
    """The n-th root p of M sin(pL) + p cos(pL) = 0, between (n-1/2) pi/L and n pi/L."""
    lo, hi = (n - 0.5) * math.pi / L, n * math.pi / L
    f = lambda p: MASS * math.sin(p * L) + p * math.cos(p * L)
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        lo, hi = (mid, hi) if f(lo) * f(mid) > 0.0 else (lo, mid)
    return 0.5 * (lo + hi)


def exact_level(parity, label):
    """The exact free level at k = 0 with this label (L = 3, M = 1)."""
    if parity == "even":
        n = abs(label)
        return math.copysign(math.sqrt(MASS ** 2 + (n * math.pi / L) ** 2), label) \
            if n else 0.0
    n = label + 1 if label >= 0 else -label  # labels 0, 1, ... and -1, -2, ...
    return math.copysign(math.sqrt(MASS ** 2 + odd_momentum(n) ** 2), label + 0.5)


RECORD_FREE = "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
recorded = {}
for line in repository_file(RECORD_FREE).read_text(encoding="utf-8").split("\n")[1:]:
    if line:
        m_, l_, parity, label, eps_num, eps_exact, _ = line.split(",")
        if float(m_) == 1.0 and float(l_) == 3.0:
            recorded[(parity, int(label))] = (float(eps_num), float(eps_exact))
worst_exact, worst_record = 0.0, 0.0
say("parity label   shooting           exact              difference")
for parity in ("even", "odd"):
    for label in range(-3, 6):
        exact = exact_level(parity, label)
        found = find_level(0.0, 1, parity, label, 0.0, FREE_MASS, FREE_POT, exact + 0.01)
```

```
        say(f"{parity:5} {label:3d}  {found:17.13f}  {exact:17.13f}"
            f"  {found - exact:10.2e}")
        if abs(exact) < 4.0:
            worst_exact = max(worst_exact, abs(found - exact))
        if (parity, label) in recorded:
            worst_record = max(worst_record, abs(found - recorded[(parity, label)][0]))
report("largest |shooting - exact| for |eps| < 4", f"{worst_exact:.2e}")
report("largest |this notebook - solver record|", f"{worst_record:.2e}")
check(worst_exact < 5e-9,
      "the shooting levels equal the exact levels within 5e-9 for |eps| < 4",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_k0_analytic_spectra")
check(worst_record < 1e-12, "the levels equal the solver's levels within 1e-12",
      record=f"{RECORD_FREE}, column eps_numeric")
```

Out [6]:

```
parity label   shooting           exact              difference
even   -3   -3.2969083097756   -3.2969083094756   -3.00e-10
even   -2   -2.3208814801929   -2.3208814801555   -3.74e-11
even   -1   -1.4479719304030   -1.4479719304020   -9.37e-13
even    0   -0.0000000000000    0.0000000000000   -1.37e-19
even    1    1.4479719304030    1.4479719304020    9.37e-13
even    2    2.3208814801929    2.3208814801555    3.74e-11
even    3    3.2969083097756    3.2969083094756    3.00e-10
even    4    4.3065024545248    4.3065024532345    1.29e-09
even    5    5.3306254626637    5.3306254586872    3.98e-09
odd    -3   -2.9119358754965   -2.9119358753543   -1.42e-10
odd    -2   -2.0106285600592   -2.0106285600459   -1.33e-11
odd    -1   -1.2922928280688   -1.2922928280686   -1.99e-13
odd     0    1.2922928280688    1.2922928280686    2.00e-13
odd     1    2.0106285600592    2.0106285600459    1.33e-11
odd     2    2.9119358754965    2.9119358753543    1.42e-10
odd     3    3.8829900333452    3.8829900326220    7.23e-10
odd     4    4.8845756333015    4.8845756308205    2.48e-09
odd     5    5.9016800924528    5.9016800857684    6.68e-09
RESULT largest |shooting - exact| for |eps| < 4 = 7.23e-10
RESULT largest |this notebook - solver record| = 7.99e-15
PASS the shooting levels equal the exact levels within 5e-9 for |eps| < 4
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_k0_analytic_spectra
PASS the levels equal the solver's levels within 1e-12
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, column
    eps_numeric
```

Text cell:

```
## 9. Orbitals: the values in the middle of the steps, and the normalisation

RK4 gives the orbital at the step ends. The densities (and the potentials) are
needed on the whole fine grid, so the solver fills in each midpoint by **cubic
Hermite interpolation**: with the values $u_0, u_1$ and derivatives $u'_0, u'_1$ at
```

```
the two ends of a step of length $h$, the midpoint value is
$\tfrac12(u_0 + u_1) + \tfrac{h}{8}(u'_0 - u'_1)$ (exact for every cubic polynomial;
the derivatives come from the equation itself). Then the orbital is normalised with
Simpson's rule. The next cell defines this and draws three orbitals: the zero mode,
the first even bulk level at $k = 0$, and the brane-band level at $k = 0.5$ (the
shell $n_2 = 4$). The check compares the zero mode with its exact form
$a = \sqrt{2M/(1 - e^{-2ML})}\,e^{My}$, $b = 0$.
```

In [7]:

```
def orbital(eps, k, j, a4, mass, pot, grid=GRID):
    """The normalised orbital (a, b) on the fine grid (two numpy arrays)."""
    _, _, nodes = shoot(eps, k, j, a4, mass, pot, grid, store=True)
    h = grid["h"]
    kk = k * math.exp(-a4)
    a = np.zeros(grid["points"])
    b = np.zeros(grid["points"])
    a[0::2] = [p[0] for p in nodes]  # the step ends
    b[0::2] = [p[1] for p in nodes]
    f = np.arange(0, grid["points"], 2)  # fine indices of the step ends
    m_y = np.array(mass)[f]
    kap = kk * np.array(grid["ew"])[f]
    je = j * (eps - np.array(pot)[f])
    da = m_y * a[f] - (kap + je) * b[f]  # derivatives from the equation
    db = (je - kap) * a[f] - m_y * b[f]
    a[1::2] = 0.5 * (a[f][:-1] + a[f][1:]) + h / 8.0 * (da[:-1] - da[1:])
    b[1::2] = 0.5 * (b[f][:-1] + b[f][1:]) + h / 8.0 * (db[:-1] - db[1:])
    norm = math.sqrt(float(np.sum(grid["simpson"] * (a * a + b * b))))
    return a / norm, b / norm


zero_a, zero_b = orbital(0.0, 0.0, 1, 0.0, FREE_MASS, FREE_POT)
bulk_eps = find_level(0.0, 1, "even", 1, 0.0, FREE_MASS, FREE_POT, 1.4)
bulk_a, bulk_b = orbital(bulk_eps, 0.0, 1, 0.0, FREE_MASS, FREE_POT)
band_eps = find_level(0.5, 1, "even", 0, 0.0, FREE_MASS, FREE_POT, 0.8)
band_a, band_b = orbital(band_eps, 0.5, 1, 0.0, FREE_MASS, FREE_POT)
fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.6), sharey=True,
                         layout="constrained")
for ax, (a, b, title) in zip(axes, [
        (zero_a, zero_b, "zero mode, $k = 0$, $\\varepsilon = 0$"),
        (bulk_a, bulk_b, f"bulk, $k = 0$, $\\varepsilon = {bulk_eps:.4f}$"),
        (band_a, band_b, f"brane band, $k = 0.5$, $\\varepsilon = {band_eps:.4f}$")]):
    ax.plot(Y, a, color=PALETTE[0], lw=2.0, label="$a(y)$")
    ax.plot(Y, b, color=PALETTE[1], lw=2.0, ls="--", label="$b(y)$")
    ax.set_title(title, fontsize=9)
    ax.set_xlabel("hidden coordinate $y$")
axes[0].set_ylabel("orbital (normalised)")
axes[0].legend()
save_figure(fig, "orbitals",
            "Three normalised orbitals of the free block $j = +1$ (components $a$, "
            "solid, and $b$, dashed; vertical axis, units of $m^{1/2}$) against the "
            "hidden coordinate $y$ (horizontal axis, tip at $-3$, brane at $0$): the "
            "zero mode at $k = 0$, which is $e^{y}$ and sits at the brane; the first "
```

```
            "even bulk level at $k = 0$, which fills the whole interval; and the "
            "brane-band level at $k = 0.5$, again bound to the brane. Each satisfies "
            "$b = 0$ at the tip and at the brane.")
exact_zero = math.sqrt(2.0 * MASS / (1.0 - math.exp(-2.0 * MASS * L))) * np.exp(MASS * Y)
zero_error = max(float(np.max(np.abs(zero_a - exact_zero))),
                 float(np.max(np.abs(zero_b))))
report("largest deviation of the zero mode from its exact form", f"{zero_error:.2e}")
check(zero_error < 1e-11, "the zero mode is (e^{My}, 0), normalised, to 1e-11",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_zero_mode_exact")
```

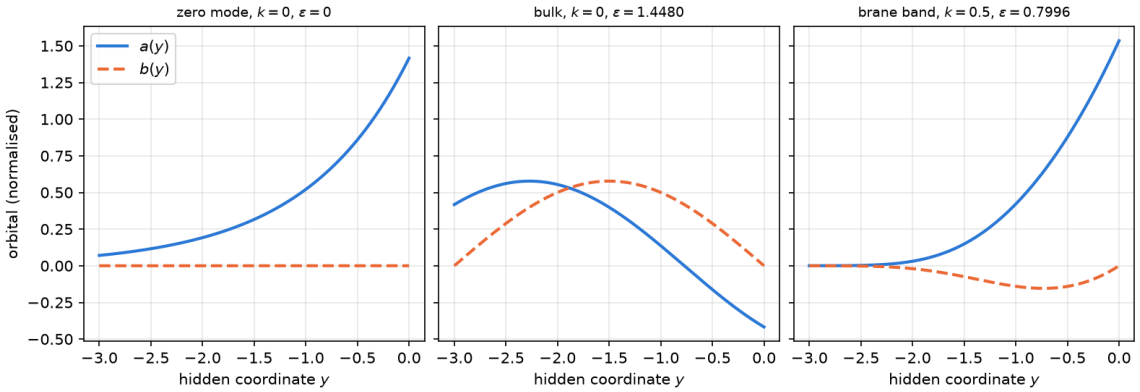

Figure 406: Three normalised orbitals of the free block $j = +1$ (components a, solid, and b, dashed; vertical axis, units of $m^{1/2}$) against the hidden coordinate y (horizontal axis, tip at -3, brane at 0): the zero mode at $k = 0$, which is e^y and sits at the brane; the first even bulk level at $k = 0$, which fills the whole interval; and the brane-band level at $k = 0.5$, again bound to the brane. Each satisfies $b = 0$ at the tip and at the brane. (Notebook 15e, figure 2.)

Out [7]:

```
Figure 15e.2 saved as Revision/textbook/figures/15e_2_orbitals.png
RESULT largest deviation of the zero mode from its exact form = 1.35e-12
PASS the zero mode is (e^{My}, 0), normalised, to 1e-11
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_zero_mode_exact
```

Text cell:

```
## 10. How accurate is RK4? The order 4

The error of RK4 falls like $h^4$: halving the step divides it by $2^4 = 16$. The
next cell finds the even levels with the labels 2 and 3 at $k = 0$ (exact values
$\sqrt{1 + (2\pi/3)^2}$ and $\sqrt{1 + \pi^2}$) on grids of 100, 200, 400 and 800
steps, prints the errors and their ratios, and draws the errors against the step on
logarithmic axes, where $h^4$ is a straight line of slope 4. The solver measured the
ratio 16.00 between its canonical and refined runs (Revision record
ks-rust-determinism.json, check ‘refined_free_spectra_convergence_order‘).
```

In [8]:

```
STEPS = [100, 200, 400, 800]
errors = {2: [], 3: []}
for steps in STEPS:
    grid = make_grid(steps)
    mass = [MASS] * grid["points"]
    pot = [0.0] * grid["points"]
    for label in errors:
        found = find_level(0.0, 1, "even", label, 0.0, mass, pot,
                           exact_level("even", label), grid=grid)
        errors[label].append(abs(found - exact_level("even", label)))
ratios = []
for label, errs in errors.items():
    steps_ratios = [errs[i] / errs[i + 1] for i in range(len(errs) - 1)]
    ratios += steps_ratios
    say(f"label {label}: errors " + ", ".join(f"{e:.2e}" for e in errs)
        + "; ratios " + ", ".join(f"{r:.2f}" for r in steps_ratios))
steps_h = [L / s for s in STEPS]
fig, ax = plt.subplots()
for colour, (label, errs) in zip(PALETTE, errors.items()):
    ax.loglog(steps_h, errs, "o-", color=colour, lw=2.0, ms=7,
              label=f"even level $l = {label}$")
ax.loglog(steps_h, [errors[3][0] * (s / steps_h[0]) ** 4 for s in steps_h], "--",
          color="0.4", lw=1.2, label="slope 4: error $\\propto h^4$")
ax.set_xlabel("step $h$ (units of $1/m$)")
ax.set_ylabel("|shooting level - exact level| (units of $m$)")
ax.set_title("RK4: halving the step divides the error by 16")
ax.legend()
save_figure(fig, "rk4_convergence",
            "Error of two shooting levels (vertical axis, logarithmic, units of $m$) "
            "against the RK4 step $h = 3/G$ for $G = 100$, $200$, $400$, $800$ steps "
            "(horizontal axis, logarithmic): the points lie on lines of slope 4, the "
            "dashed reference, so halving the step divides the error by 16, the "
            "fourth order of the classical Runge-Kutta method.")
middle = sorted(ratios)[len(ratios) // 2]
report("median error ratio for halving the step", f"{middle:.2f}")
check(15.0 < middle < 17.0, "the error ratio is 16 within 1 (fourth order)",
      record="Revision/kohn_sham/reports/ks-rust-determinism.json, check "
             "refined_free_spectra_convergence_order")
```

Out [8]:

```
label 2: errors 2.45e-07, 1.53e-08, 9.59e-10, 5.99e-11; ratios 15.98, 16.00, 16.00
label 3: errors 1.96e-06, 1.23e-07, 7.69e-09, 4.81e-10; ratios 15.96, 15.99, 16.00
```

Out [8]:

```
Figure 15e.3 saved as Revision/textbook/figures/15e_3_rk4_convergence.png
RESULT median error ratio for halving the step = 16.00
PASS the error ratio is 16 within 1 (fourth order)
     reproduces Revision/kohn_sham/reports/ks-rust-determinism.json, check
    refined_free_spectra_convergence_order
```

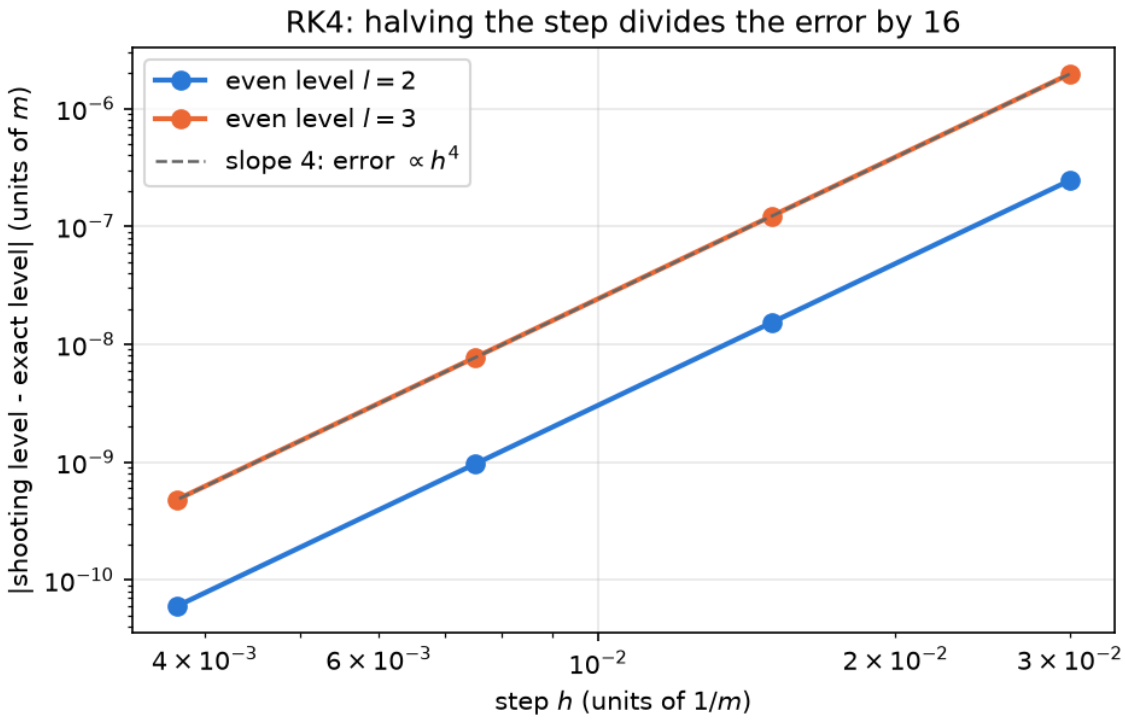

Figure 407: Error of two shooting levels (vertical axis, logarithmic, units of m) against the RK4 step $h = 3/G$ for $G = 100$, 200, 400, 800 steps (horizontal axis, logarithmic): the points lie on lines of slope 4, the dashed reference, so halving the step divides the error by 16, the fourth order of the classical Runge-Kutta method. (Notebook 15e, figure 3.)

Text cell:

```
## 11. The brane band and the deflating history

For $k > 0$ the lowest even level of $j = +1$ (label 0) is the **brane band**: an
orbital bound to the brane whose energy grows with $k$. The record gives its slope
at $k = 0$ in closed form (Revision/kohn_sham/ks-theory.json, checksNumeric):
$c = \frac{2M}{2M - H}\,\frac{1 - e^{-(2M - H)L}}{1 - e^{-2ML}} = 1.90514825360$ for
$M = H = 1$, $L = 3$, times $e^{-a_{4,0}}$ at the slice $a_{4,0}$. The next cell
computes the band at 41 momenta from 0 to 2 at the slices $a_{4,0} = 0$, $1$, $2$,
and measures the slope at $k = 0$ from $s(k) = \varepsilon(k)/k$ at
$k = 10^{-4}$ and $2 \times 10^{-4}$. Without interaction the band is an **odd**
function of $k$: the two exact symmetries of the block Hamiltonian (Revision record
ks-theory.json, blockEquation.typeRelation), $\sigma_3 h_j(k) \sigma_3 = h_{-j}(-k)$
and $h_{-1} = -h_{+1}$ (for $v = 0$), give together that $-\varepsilon$ is a level
of $h_{+1}(-k)$ whenever $\varepsilon$ is a level of $h_{+1}(k)$. The labels go
along: the first map changes the signs of $b$ and $j$, so $\Phi = j\theta(0)$ and
every label stay the same; the second keeps $(a, b)$ and changes the signs of $j$
and $\varepsilon$, so $\Phi \to -\Phi$ and the even label $l$ goes to $-l$
(Revision/kohn_sham/reports/ks-rust-solver.json, check
free_block_type_symmetries). The band, the even label 0, therefore goes to the
even label 0: $\varepsilon(-k) = -\varepsilon(k)$ for the band itself. So
$\varepsilon(k) = c k - d k^3 + \dots$ and $s(k) = c - d k^2 + \dots$: the error of
$s$ grows fourfold when $k$ doubles, and the **Richardson extrapolation**
$\tfrac{1}{3}(4 s(k) - s(2k))$ removes it (the $k^2$ terms cancel:
$4(c - dk^2) - (c - 4dk^2) = 3c$). The cell prints the measured factor. Finally it
checks the **rescaling identity**: the level at the slice $a_{4,0}$ equals the
level at the slice $0$ with the momentum $k e^{-a_{4,0}}$, because $a_{4,0}$ enters
only through $\kappa k$.
```

In [9]:

```
THEORY = json.loads(repository_file("Revision/kohn_sham/ks-theory.json").read_text(
    encoding="utf-8"))  # the theory file of the solver
C_RECORD = float(THEORY["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
c_formula = (2 * MASS / (2 * MASS - H) * (1 - math.exp(-(2 * MASS - H) * L))
             / (1 - math.exp(-2 * MASS * L)))


def band(k, a4):
    """The brane-band level (j = +1, even, label 0) at momentum k and slice a4."""
    return find_level(k, 1, "even", 0, a4, FREE_MASS, FREE_POT, 1.9 * k * math.exp(-a4))


momenta = np.linspace(0.0, 2.0, 41)
fig, ax = plt.subplots()
slopes = []
for shade, a4 in zip(("#86b6ef", "#2a78d6", "#104281"), (0.0, 1.0, 2.0)):
    curve = [band(k, a4) for k in momenta]
    ax.plot(momenta, curve, "o-", color=shade, ms=4, lw=1.8,
            label=f"$a_{{4,0}} = {a4:.0f}$")
    ax.plot(momenta, c_formula * momenta * math.exp(-a4), ":", color=shade, lw=1.2)
```

```
    s1, s2 = band(1e-4, a4) / 1e-4, band(2e-4, a4) / 2e-4  # s(k) and s(2k)
    slopes.append((4.0 * s1 - s2) / 3.0 * math.exp(a4))  # slope times e^{a4,0}
    if a4 == 0.0:
        s4 = band(4e-4, a4) / 4e-4  # s(4k), to measure how the error grows
        growth = (s4 - s2) / (s2 - s1)  # 4 if the error of s is proportional to k^2
ax.plot([], [], ":", color="0.4", label="$c\\,k\\,e^{-a_{4,0}}$")
ax.set_xlabel("3-momentum $k$ (units of $m$)")
ax.set_ylabel("brane-band level $\\varepsilon$ (units of $m$)")
ax.set_title("The brane band redshifts along the history")
ax.legend()
save_figure(fig, "brane_band",
            "The brane-band level $\\varepsilon(k)$ (vertical axis, units of $m$) "
            "against the 3-momentum $k$ (horizontal axis, units of $m$) at the "
            "slices $a_{4,0} = 0$, $1$, $2$ (light to dark blue); the dotted lines "
            "are the tangents $c\\,k\\,e^{-a_{4,0}}$ at $k = 0$ with "
            "$c = 1.9051482536$. Along the deflating history the whole band is "
            "squeezed toward $\\varepsilon = 0$: each curve is the previous one with "
            "the momentum axis stretched by $e = 2.718$.")
rescale = max(abs(band(k, a4) - band(k * math.exp(-a4), 0.0))
              for k in (0.25, 1.0, 2.5) for a4 in (0.5, 1.0, 2.0))
report("growth of the error of eps(k)/k when k doubles", f"{growth:.4f}")
report("slope c times e^{a4,0} at a4,0 = 0, 1, 2",
       ", ".join(f"{s:.10f}" for s in slopes))
report("largest violation of the rescaling identity", f"{rescale:.1e}")
check(abs(c_formula - C_RECORD) < 1e-14 and
      max(abs(s - C_RECORD) for s in slopes) < 1e-9 * C_RECORD,
      "the brane-band slope is c e^{-a4,0} with c = 1.9051482536",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_brane_band_slope")
check(rescale < 1e-12, "eps(k, a4,0) = eps(k e^{-a4,0}, 0) within 1e-12",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_rescaling_relation_and_band_monotone")
```

Out [9]:

```
Figure 15e.4 saved as Revision/textbook/figures/15e_4_brane_band.png
RESULT growth of the error of eps(k)/k when k doubles = 4.0000
RESULT slope c times e^{a4,0} at a4,0 = 0, 1, 2 = 1.9051482536, 1.9051482536,
    1.9051482536
RESULT largest violation of the rescaling identity = 0.0e+00
PASS the brane-band slope is c e^{-a4,0} with c = 1.9051482536
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_brane_band_slope
PASS eps(k, a4,0) = eps(k e^{-a4,0}, 0) within 1e-12
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_rescaling_relation_and_band_monotone
```

Text cell:

```
## 12. The self-consistent loop for N = 8

Now the interaction. The eight particles of the state $N = 8$ fill the zero modes of
```

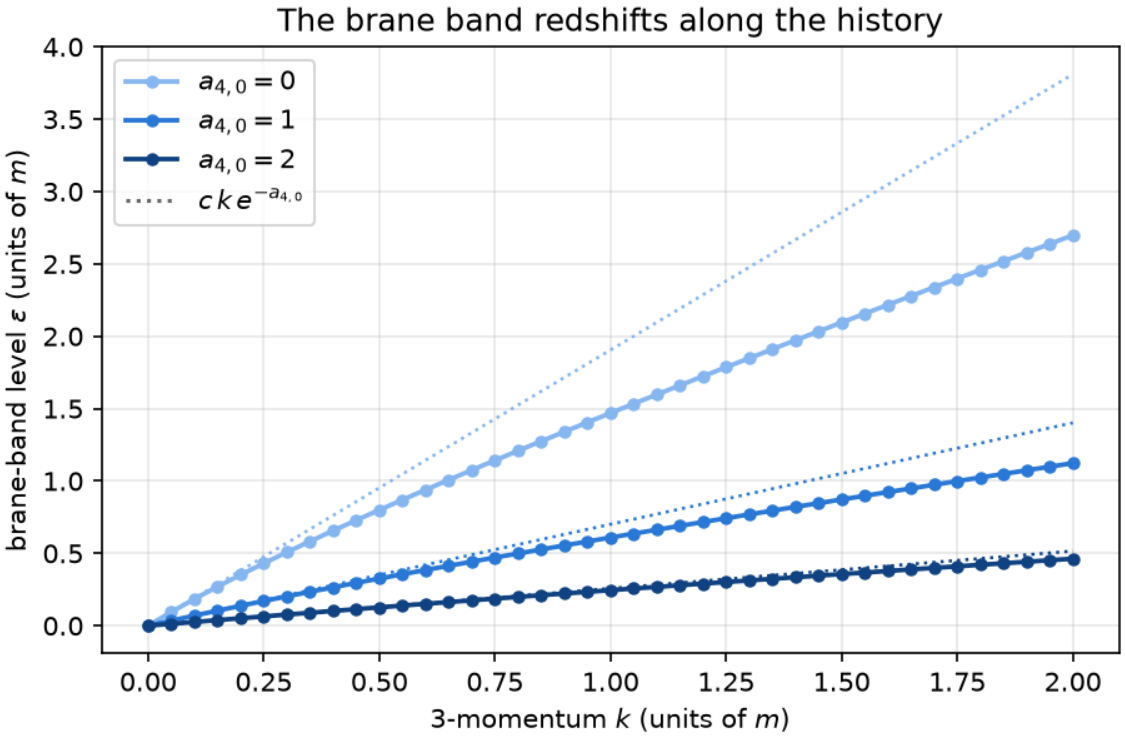

Figure 408: The brane-band level $\varepsilon(k)$ (vertical axis, units of m) against the 3-momentum k (horizontal axis, units of m) at the slices $a_{4,0} = 0$, 1, 2 (light to dark blue); the dotted lines are the tangents $c\,k\,e^{-a_{4,0}}$ at $k = 0$ with $c = 1.9051482536$. Along the deflating history the whole band is squeezed toward $\varepsilon = 0$: each curve is the previous one with the momentum axis stretched by $e = 2.718$. (Notebook 15e, figure 4.)

both block types $j = \pm 1$ at $k = 0$ (four orbitals each). The next cell defines one **iteration**: from the current potentials ($M - m$ and v on the fine grid, one vector `x` of length 2×1801) it finds the two occupied levels (label 0, even parity, $k = 0$, $j = \pm 1$), builds their normalised orbitals, the proper densities n and S, the output potentials, the residual (largest difference between output and input) and the energy E_{KS}. It also defines **Anderson mixing** exactly as the solver does: it keeps the last six inputs x_i and residuals r_i, finds the weights c_i with $\sum c_i = 1$ that make $\sum c_i r_i$ as small as possible (a small linear system), and takes $\sum c_i (x_i + \beta r_i)$ with $\beta = 0.4$ as the next input; its history is cleared when a residual grows tenfold.

In [10]:

```
POINTS = GRID["points"]
P_FACTOR = 1.0 / (E6 * VOL7)  # P = e^{-6Hy}/Vol_7 on the fine grid


def iteration(x, lam, a4, guesses):
    """One Kohn-Sham iteration for N = 8; returns (residual, x_out, E, levels)."""
    mass = (MASS + x[:POINTS]).tolist()
    pot = x[POINTS:].tolist()
    n = np.zeros(POINTS)
    s = np.zeros(POINTS)
    levels = {}
    for j in (1, -1):  # the two block types, each level four-fold (g = 4)
        eps = find_level(0.0, j, "even", 0, a4, mass, pot, guesses.get(j, 0.0))
        a, b = orbital(eps, 0.0, j, a4, mass, pot)
        weight = 0.5 * 4.0 * 1.0  # w g f with w = 1/2 (the Z2 doubling)
        n += weight * P_FACTOR * (a * a + b * b)
        s += weight * P_FACTOR * j * 2.0 * a * b
        levels[j] = eps
    x_out = np.concatenate([lam * (15.0 / 16.0) * s, lam * (-1.0 / 16.0) * n])
    e_int = lam * (15.0 / 32.0 * s * s - 1.0 / 32.0 * n * n)
    energy = sum(4.0 * e for e in levels.values()) \
        - 2.0 * VOL7 * float(np.sum(GRID["simpson"] * E6 * e_int))
    return float(np.max(np.abs(x_out - x))), x_out, energy, levels


class Anderson:
    """Anderson (Pulay) mixing with the solver's depth 6 and beta 0.4."""

    def __init__(self, depth=6, beta=0.4):
        self.depth, self.beta = depth, beta
        self.xs, self.rs, self.best = [], [], math.inf

    def next(self, x, r):
        size = float(np.max(np.abs(r)))
        if size > 10.0 * self.best and self.xs:
            self.xs, self.rs = [], []  # the residual grew tenfold: start again
        self.best = min(self.best, size)
        self.xs = (self.xs + [x.copy()])[-self.depth:]
        self.rs = (self.rs + [r.copy()])[-self.depth:]
```

```
        count = len(self.xs)
        # minimise |sum c_i r_i|^2 with sum c_i = 1 (a Lagrange multiplier):
        system = np.zeros((count + 1, count + 1))
        gram = np.array([[float(np.dot(u, v)) for v in self.rs] for u in self.rs])
        system[:count, :count] = gram + 1e-12 * np.max(np.diag(gram)) * np.eye(count)
        system[:count, count] = 1.0
        system[count, :count] = 1.0
        right = np.zeros(count + 1)
        right[count] = 1.0
        weights = np.linalg.solve(system, right)[:count]
        return sum(c * (xi + self.beta * ri)
                   for c, xi, ri in zip(weights, self.xs, self.rs))


def solve_n8(lam, a4=0.0, mixing="anderson", tol=1e-11, limit=200):
    """The self-consistent N = 8 state: (E, levels, iteration history, x)."""
    x = np.zeros(2 * POINTS)
    mixer = Anderson()
    guesses, history = {}, []
    for it in range(1, limit + 1):
        residual, x_out, energy, levels = iteration(x, lam, a4, guesses)
        guesses = levels
        history.append((it, residual, energy))
        if residual <= tol:
            return energy, levels, history, x
        r = x_out - x
        x = mixer.next(x, r) if mixing == "anderson" else x + 0.4 * r
    raise RuntimeError("no convergence")
```

Text cell:

```
The next cell reads the coupling $\lambda_1$ of $N = 8$ from the record
(‘parameters.json‘), runs the loop with Anderson mixing, prints the residual and the
energy of every iteration, and compares with the record: the energy $E_{KS}$ and the
occupied level (the HOMO) of the state ‘N8_lamp1_a00‘ in ‘ground/summary.csv‘, and
the residuals of the first six iterations recorded in ‘ground/runs.json‘ (from the
seventh iteration on, the residuals are differences of nearly equal numbers and
depend on the last digits of the arithmetic, which Python and Rust do in a
different order, so they are not compared). It checks that both block types give
the same level, as the exact
symmetry $\sigma_3 h_j(k)\sigma_3 = h_{-j}(-k)$ of the block Hamiltonian demands at
$k = 0$.
```

In [11]:

```
params = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))
calib = {int(c["N"]): c for c in params["couplingCalibration"]["values"]}
LAM1, LAM2 = calib[8]["lambda1"], calib[8]["lambda2"]
energy, levels, history, x_final = solve_n8(LAM1)
say("iteration   residual        E_KS")
for it, residual, e in history:
```

```
    say(f"{it:9d}   {residual:9.3e}   {e:.13e}")
SUMMARY = "Revision/kohn_sham/results/ground/summary.csv"
with open(repository_file(SUMMARY), newline="", encoding="utf-8") as handle:
    ground = {row["id"]: row for row in csv.DictReader(handle)}
runs = {run["id"]: run for run in json.loads(repository_file(
    "Revision/kohn_sham/results/ground/runs.json").read_text(encoding="utf-8"))}
recorded_history = runs["N8_lamp1_a00"]["scfHistory_iteration_residual_E"]
early = max(abs(h[1] - r[1]) / r[1] for h, r in zip(history[:6], recorded_history[:6]))
report("E_KS of N = 8 with lambda_1", f"{energy:.15e}")
report("iterations (this notebook / record)",
       f"{len(history)} / {len(recorded_history)}")
report("largest relative difference of the first six residuals", f"{early:.1e}")
check(abs(energy - float(ground["N8_lamp1_a00"]["E_KS"])) < 1e-14
      and abs(levels[1] - float(ground["N8_lamp1_a00"]["HOMO"])) < 1e-13,
      "E_KS and HOMO of N8_lamp1_a00 reproduced", record=f"{SUMMARY}, row N8_lamp1_a00")
check(early < 1e-5, "the first six residuals equal the recorded ones within 1e-5",
      record="Revision/kohn_sham/results/ground/runs.json, N8_lamp1_a00")
check(abs(levels[1] - levels[-1]) < 1e-13, "the levels of j = +1 and j = -1 are equal")
```

Out [11]:

```
iteration   residual        E_KS
        1   1.000e-01   9.9151203205334e-04
        2   6.001e-02   1.9758343924451e-04
        3   9.244e-03   -9.3645832228630e-04
        4   1.827e-04   -9.8185433539593e-04
        5   4.731e-05   -9.8609860452298e-04
        6   4.514e-05   -9.8752614358140e-04
        7   1.045e-06   -9.8685650934276e-04
        8   3.423e-08   -9.8684095228991e-04
        9   9.641e-09   -9.8684215929634e-04
       10   1.136e-09   -9.8684263134797e-04
       11   4.471e-11   -9.8684262120161e-04
       12   2.818e-11   -9.8684262031918e-04
       13   7.547e-13   -9.8684261908764e-04
RESULT E_KS of N = 8 with lambda_1 = -9.868426190876385e-04
RESULT iterations (this notebook / record) = 13 / 13
RESULT largest relative difference of the first six residuals = 3.0e-07
PASS E_KS and HOMO of N8_lamp1_a00 reproduced
     reproduces Revision/kohn_sham/results/ground/summary.csv, row N8_lamp1_a00
PASS the first six residuals equal the recorded ones within 1e-5
     reproduces Revision/kohn_sham/results/ground/runs.json, N8_lamp1_a00
PASS the levels of j = +1 and j = -1 are equal
```

Text cell:

```
## 13. Why Anderson mixing? A comparison with linear mixing

The next cell solves the same state again with **linear mixing** (the next input is
$x + 0.4\,r$) and draws both residual histories together with the solver's recorded
one. Linear mixing reaches the same state but needs several times more iterations;
the solver's record and this notebook's Anderson run lie on top of each other.
```

In [12]:

```
energy_lin, _, history_lin, _ = solve_n8(LAM1, mixing="linear")
fig, ax = plt.subplots()
ax.semilogy([h[0] for h in history_lin], [h[1] for h in history_lin], "s-",
            color=PALETTE[1], ms=4, lw=1.5, label="linear mixing, $\\beta = 0.4$")
ax.semilogy([h[0] for h in history], [h[1] for h in history], "o-",
            color=PALETTE[0], ms=7, lw=2.0, label="Anderson mixing (this notebook)")
ax.semilogy([r[0] for r in recorded_history], [r[1] for r in recorded_history], "x",
            color="black", ms=9, label="Anderson mixing (Rust solver, record)")
ax.axhline(1e-11, color="0.4", ls=":", lw=1.2, label="tolerance $10^{-11}$")
ax.set_xlabel("iteration")
ax.set_ylabel("residual (units of $m$)")
ax.set_title("Self-consistent loop for $N = 8$, $\\lambda = +\\lambda_1$")
ax.legend(fontsize=8)
save_figure(fig, "scf_mixing",
            "Residual of the self-consistent loop (vertical axis, logarithmic, units "
            "of $m$) against the iteration (horizontal axis) for $N = 8$ with the "
            "coupling $+\\lambda_1$: linear mixing (orange squares) needs about "
            "four times as many iterations as Anderson mixing (blue dots); the "
            "crosses are the residuals recorded by the Rust solver, which agree with "
            "this notebook's Anderson run.")
report("iterations: Anderson / linear", f"{len(history)} / {len(history_lin)}")
report("|E(linear) - E(Anderson)|", f"{abs(energy_lin - energy):.1e}")
check(abs(energy_lin - energy) < 1e-12 and len(history_lin) > 2 * len(history),
      "linear mixing reaches the same energy, with more than twice the iterations")
```

Out [12]:

```
Figure 15e.5 saved as Revision/textbook/figures/15e_5_scf_mixing.png
RESULT iterations: Anderson / linear = 13 / 47
RESULT |E(linear) - E(Anderson)| = 8.0e-15
PASS linear mixing reaches the same energy, with more than twice the iterations
```

Text cell:

```
## 14. All four couplings, and an exact symmetry

The next cell solves $N = 8$ for $+\lambda_1$, $-\lambda_1$, $+\lambda_2$,
$-\lambda_2$, compares the energies and occupied levels with the record, and computes
the lowest empty level (the LUMO: the brane band at $k = 0.25$ in the final
potentials) for $+\lambda_1$. Then it checks an **exact symmetry** of this state:
changing the sign of $\lambda$ changes the sign of the energy. The reason, line by
line. (1) At $k = 0$ the real system of block $j$ is $a' = M a - j(\varepsilon -
v)\,b$, $b' = j(\varepsilon - v)\,a - M b$. (2) Put $\tilde a = a$, $\tilde b = -b$,
$\tilde\varepsilon = -\varepsilon$, $\tilde v = -v$, the same $M$ and the same $j$;
then $\tilde\varepsilon - \tilde v = -(\varepsilon - v)$, and the two equations
become $\tilde a' = M\tilde a - j(\tilde\varepsilon - \tilde v)\,\tilde b$ and
$\tilde b' = j(\tilde\varepsilon - \tilde v)\,\tilde a - M\tilde b$ (each sign
change appears twice and cancels): $(a, -b)$ is an orbital of the problem with
$(M, -v)$ at the level $-\varepsilon$, with the same boundary conditions ($b = 0$
```

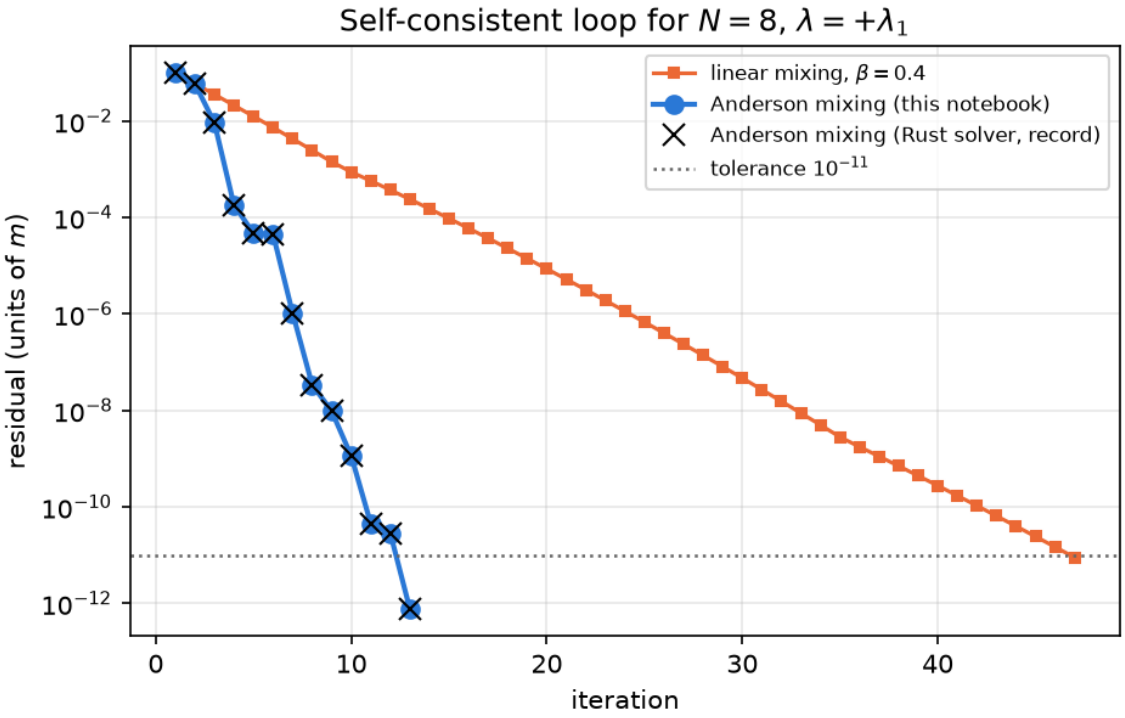

Figure 409: Residual of the self-consistent loop (vertical axis, logarithmic, units of m) against the iteration (horizontal axis) for $N = 8$ with the coupling $+\lambda_1$: linear mixing (orange squares) needs about four times as many iterations as Anderson mixing (blue dots); the crosses are the residuals recorded by the Rust solver, which agree with this notebook's Anderson run. (Notebook 15e, figure 5.)

```
stays $b = 0$) and the same label ($\Phi \to -\Phi$, and the label 0 goes to 0).
(3) Its density $a^2 + b^2$ is the same and its scalar density $2jab$ changes sign,
so $n \to n$ and $S \to -S$. (4) Under $\lambda \to -\lambda$ the potentials of
these densities are $M - m = \tfrac{15}{16}(-\lambda)(-S)$, unchanged, and
$-\tfrac{1}{16}(-\lambda)n = -v$: exactly the potentials assumed in (2), so the
mapped orbitals form a self-consistent state of $-\lambda$ (the loop started from
the free state finds exactly this one, as the check shows). (5) Its energy:
$\sum g f\varepsilon$ changes sign with every level, and
$e_{int} = \lambda(\tfrac{15}{32}S^2 - \tfrac{1}{32}n^2)$ changes sign with
$\lambda$ while $S^2$ and $n^2$ stay; so $E_{KS}(-\lambda) = -E_{KS}(\lambda)$.
```

In [13]:

```
tags = {"lamp1": LAM1, "lamm1": -LAM1, "lamp2": LAM2, "lamm2": -LAM2}
results = {}
worst_e, worst_homo = 0.0, 0.0
for tag, lam in tags.items():
    e, lv, hist, xf = solve_n8(lam)
    results[tag] = (e, lv[1], xf)
    row = ground[f"N8_{tag}_a00"]
    worst_e = max(worst_e, abs(e - float(row["E_KS"])))
    worst_homo = max(worst_homo, abs(lv[1] - float(row["HOMO"])))
    say(f"{tag}: lambda {lam:+.5f}, E_KS {e:+.12e}, HOMO {lv[1]:+.12e},"
        f" {len(hist)} it.")
xf = results["lamp1"][2]
lumo = find_level(0.25, 1, "even", 0, 0.0, (MASS + xf[:POINTS]).tolist(),
                  xf[POINTS:].tolist(), 0.43)
report("LUMO of N8_lamp1_a00 (brane band at k = 0.25)", f"{lumo:.13f}")
antisym = max(abs(results["lamp1"][0] + results["lamm1"][0]),
              abs(results["lamp2"][0] + results["lamm2"][0]))
report("largest |E(+lambda) + E(-lambda)|", f"{antisym:.1e}")
check(worst_e < 1e-14 and worst_homo < 1e-13,
      "E_KS and HOMO of all four couplings reproduced", record=f"{SUMMARY}, rows N8")
check(abs(lumo - float(ground["N8_lamp1_a00"]["LUMO"])) < 1e-12,
      "the LUMO of N8_lamp1_a00 reproduced", record=f"{SUMMARY}, column LUMO")
check(antisym < 1e-15, "E_KS(-lambda) = -E_KS(+lambda) for N = 8")
```

Out [13]:

```
lamp1: lambda +0.01946, E_KS -9.868426190876e-04, HOMO -2.455627077441e-04, 13 it.
lamm1: lambda -0.01946, E_KS +9.868426190876e-04, HOMO +2.455627077441e-04, 13 it.
lamp2: lambda +0.05838, E_KS -2.862652173608e-03, HOMO -6.910857390418e-04, 17 it.
lamm2: lambda -0.05838, E_KS +2.862652173608e-03, HOMO +6.910857390418e-04, 17 it.
RESULT LUMO of N8_lamp1_a00 (brane band at k = 0.25) = 0.4306983508491
RESULT largest |E(+lambda) + E(-lambda)| = 0.0e+00
PASS E_KS and HOMO of all four couplings reproduced
     reproduces Revision/kohn_sham/results/ground/summary.csv, rows N8
PASS the LUMO of N8_lamp1_a00 reproduced
     reproduces Revision/kohn_sham/results/ground/summary.csv, column LUMO
PASS E_KS(-lambda) = -E_KS(+lambda) for N = 8
```

Text cell:

```
## 15. The self-consistent potentials of N = 8

The last figure shows the converged potentials of the four couplings. The potential
$v = -\lambda n/16$ follows the proper density of the zero modes, which grows toward
the tip roughly like $e^{2y}e^{-6y} = e^{-4y}$ (the free orbital is $e^{y}$, the
proper volume $e^{6y}$). The mass shift is smaller: the scalar density $S$ of the
zero modes vanishes without interaction (there $b = 0$) and is created only by the
interaction itself; it also vanishes at the tip, where $b(-L) = 0$.
The check confirms the symmetry of the previous section in the potentials:
$M - m$ is the same and $v$ changes sign under $\lambda \to -\lambda$.
```

In [14]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
labels = {"lamp1": "$+\\lambda_1$", "lamm1": "$-\\lambda_1$",
          "lamp2": "$+\\lambda_2$", "lamm2": "$-\\lambda_2$"}
for colour, (tag, label) in zip(PALETTE, labels.items()):
    xf = results[tag][2]
    style = "-" if tag.startswith("lamp") else "--"
    left.plot(Y, xf[:POINTS], style, color=colour, lw=1.8, label=label)
    right.plot(Y, xf[POINTS:], style, color=colour, lw=1.8, label=label)
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("$M(y) - m$ (units of $m$)")
left.set_title("Mass shift")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("$v(y)$ (units of $m$)")
right.set_title("Potential")
right.legend()
save_figure(fig, "n8_potentials",
            "The self-consistent mass shift $M(y) - m$ (left) and potential $v(y)$ "
            "(right), vertical axes in units of $m$, of the state $N = 8$ for the "
            "couplings $\\pm\\lambda_1$ and $\\pm\\lambda_2$, against the hidden "
            "coordinate $y$. The potential grows toward the tip roughly like "
            "$e^{-4y}$ and "
            "changes sign with $\\lambda$; the mass shift is smaller and is the same "
            "for $+\\lambda$ and $-\\lambda$ (the dashed curves lie on the solid ones).")
plus, minus = results["lamp1"][2], results["lamm1"][2]  # potentials x of +-lambda_1
mirror = max(float(np.max(np.abs(plus[:POINTS] - minus[:POINTS]))),
             float(np.max(np.abs(plus[POINTS:] + minus[POINTS:]))))
report("largest violation of the potential symmetry", f"{mirror:.1e}")
check(mirror < 1e-12, "M - m is even and v is odd under lambda -> -lambda")
```

Out [14]:

```
Figure 15e.6 saved as Revision/textbook/figures/15e_6_n8_potentials.png
RESULT largest violation of the potential symmetry = 0.0e+00
PASS M - m is even and v is odd under lambda -> -lambda
```

Text cell:

```
## 16. The last check
```

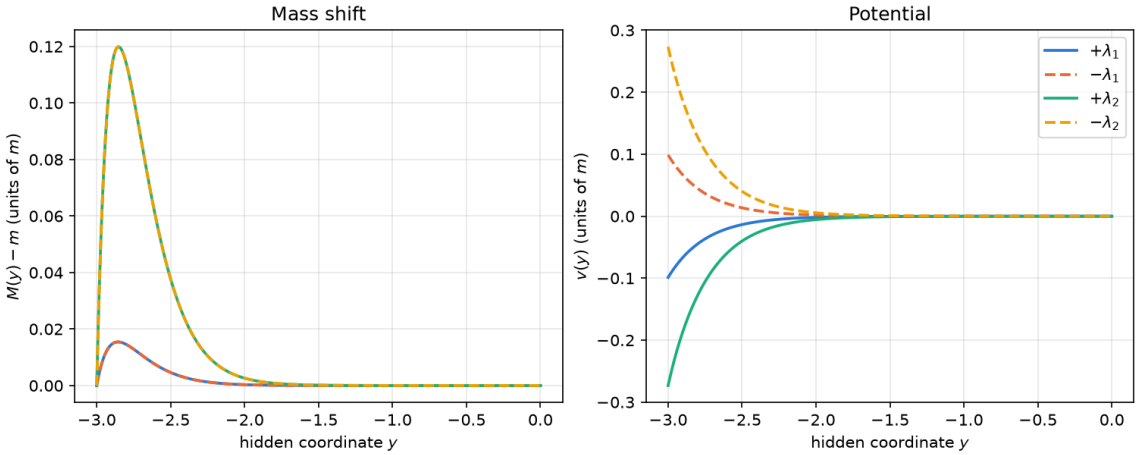

Figure 410: The self-consistent mass shift $M(y) - m$ (left) and potential $v(y)$ (right), vertical axes in units of m, of the state $N = 8$ for the couplings $\pm\lambda_1$ and $\pm\lambda_2$, against the hidden coordinate y. The potential grows toward the tip roughly like e^{-4y} and changes sign with λ; the mass shift is smaller and is the same for $+\lambda$ and $-\lambda$ (the dashed curves lie on the solid ones). (Notebook 15e, figure 6.)

```
The last cell checks that every figure file of this notebook exists and prints the
number of checks that passed.
```

In [15]:

```
NAMES = ["phase_function", "orbitals", "rk4_convergence", "brane_band", "scf_mixing",
         "n8_potentials"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15e_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

Out [15]:

```
PASS every figure file of this notebook exists
ALL 18 CHECKS PASSED (notebook 15e)
```

Text cell:

```
## 17. What this notebook showed

- The Pruefer angle turns the search for levels into counting: the phase function
  $\Phi(\varepsilon)$ increases strictly, every level is the crossing with one target
  $l\pi$ or $\pi/2 + l\pi$, and no level can be missed (PROVED by the comparison
  argument of section 4; COMPUTED here for the free spectrum).
- RK4 shooting on the solver's grid reproduces the exact free levels (within
  $5 \times 10^{-9}$ for $|\varepsilon| < 4$), the solver's own numbers (within
```

```
  $10^{-12}$) and the fourth order of the method (error ratio 16).
- The brane band has the slope $c\,e^{-a_{4,0}}$ with $c = 1.90514825365$, and the
  slice enters only through $k e^{-a_{4,0}}$ (the rescaling identity): along the
  deflating history every momentum is redshifted.
- A few dozen lines of Python with Anderson mixing reproduce the interacting $N = 8$
  ground states of the Rust solver: energies, levels and the residuals of the loop.
  Linear mixing converges too, but much more slowly.
- For $N = 8$ the energy is exactly odd in $\lambda$, a symmetry of the zero-mode
  state found and checked here.
```

15.9 Line-by-line walk-through of Notebook 15e

The notebook has 15 code cells, In [1] to In [15]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted, then explained. Docstrings (the texts in triple quotes under a `def` line, which say what a function does) and some comment lines are left out of the quotations, and long caption strings in calls of `save_figure` are shortened: the quotation shows the first line of the call and the first line of the caption, and then a line `...)` that stands for the rest of the caption and the closing parenthesis; the complete cells are printed in Section 15.8. Every code cell is preceded in the notebook by a text cell that says what it does.

In [1], the set-up cell. Every line that starts with `#` is a **comment**, which Python skips. The first part of the cell, down to the lines of `-` and `=` signs, is the complete run instructions of Section 15.7 again, as comments, so that the notebook file carries its own instructions. The code starts after the heading THE SET-UP; it computes no physics and is the same in every notebook of the book, except for the line that names the notebook (the notebooks that run a Rust program add two imports and one function, explained in Section 15.14). The other walk-throughs of this chapter refer back to this paragraph.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of a package) so that the code can use it. `json`, `os`, `textwrap` and `pathlib` come with Python. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or a folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These lines load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python code), which show a picture file below a cell.

```
NOTEBOOK_ID = "15e"  # this notebook: chapter 15, example e
```

A **variable** is a name for a value; this line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"15e"`. The figure files are named after it.

```
def find_repository_root():
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. `Path.cwd()` is the folder in which Jupyter runs the notebook (the folder that holds it), and `.resolve()` writes it as a complete address. `here.parents` lists the folders above it; `[here, *here.parents]` is the list that starts with `here` and continues with them (the star unpacks one list into another). The `for` loop visits these folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains the file `Revision/textbook/requirements.txt` (the list of the book's packages) is the repository, and `return` hands it back. If no folder qualifies, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
REPO = find_repository_root()
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The first line calls the function and names its result `REPO`. It is never printed, because it differs from computer to computer, while the printed output of a notebook must not. The second line chooses where files are written. `os.environ` holds the **environment variables** of the program (named texts that it receives from the computer); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and the default `str(REPO)` (the repository folder as a string) otherwise. When you run the notebook the variable is not set, so the files go into the repository; the book's checking tool sets it to a scratch folder, so that a check never changes a file that git tracks. (Notebook 15e writes nothing else. The notebooks 15a, 15b and 15d, which run the Rust solver, also write the solver's raw output into the folders `Revision/kohn_sham/solver/target/textbook_15a`, `textbook_15b` and `textbook_15d` next to it, about 5.9 MB, 0.9 MB and 0.6 MB; git ignores these folders, and they are written during a check too.)

```
def repository_file(relative):
    return REPO / relative


def output_file(relative):
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions. `repository_file("Revision/...")` gives the full path of a repository file, for reading a Revision record. `output_file("Revision/...")` gives the full path at which to write

a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder that will hold it, together with any missing folder above it, and does nothing if the folder exists.

```
def say(text):
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of the book; `textwrap.fill` breaks the text at blanks and starts every line after the first with four blanks. This is why some printed lines below continue on an indented second line.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

matplotlib reads personal settings from a file on your computer if you have one; `matplotlib.rcdefaults()` returns to the built-in settings, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets four settings for every figure: the size (7.0 by 4.2 inches), the size of the letters (10 points) and a faint grid of lines behind the curves (`grid.alpha` 0.3 means 30 per cent opaque). The braces make a **dictionary**: pairs of a key and a value, written `key: value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/15e.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes the captions file with the empty dictionary `{}` and a line end; `encoding="utf-8"` and `newline="\n"` make the file the same bytes on every operating system.

```
def save_figure(fig, name, caption):
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` is called once for every figure. `setdefault(name, default)` returns the number already given to this figure name, or gives it the next number (one more than the number of figures so far) and returns that; so the figures are numbered 1, 2, 3, ... in the order in which they are first saved. The file name is, for example, `15e_1_phase_function.png`. `fig.savefig` writes the picture: `dpi=150` is the resolution (150 dots per inch), `bbox_inches="tight"` cuts away the empty margin, and `metadata={"Software": None}` stores no program name in the file, so that every run writes exactly the same bytes. `plt.close(fig)` forgets the figure (otherwise Jupyter would draw it a second time). The caption is stored in `CAPTIONS`, and the whole dictionary is written again to the captions file (`json.dumps` turns it into text, `indent=1` puts one entry per line, `sort_keys=True` sorts them). `display(Image(...))` shows the saved picture below the cell, and the last line prints, for example, Figure 15e.1 saved as Revision/textbook/figures/15e_1_phase_function.png.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection, in square brackets). `check` is the function behind every check: if the statement `condition` is false (`False`), `raise AssertionError(...)` stops the notebook with an error that names the check; if it is true, the name is appended to `PASSED` and the line PASS name is printed, followed by a line naming the Revision record and its check when the argument `record` is given. (Python's own `assert` statement is not used, because Python started with the option that optimises the code skips it.)

```
def report(label, value, unit=""):
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds the unit when one is given. `all_checks_passed` prints the last line of the notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one output line of In [1], Set-up of notebook 15e complete.

In [2], the grid.

```
import csv  # reads the tables (CSV files) of the Revision record
import math  # exp, sqrt, atan2, pi for single numbers
```

```
import numpy as np  # arrays of numbers
```

`csv` reads tables stored as comma-separated text, `math` holds the mathematical functions for single numbers, and numpy (under the short name `np`) works with **arrays**, lists of numbers on which arithmetic acts entry by entry.

```
H, MASS, L = 1.0, 1.0, 3.0  # the units H = m = 1 and the tip cutoff L = 3
DK = 0.25  # the spacing of the 3-momenta
VOL7 = (2.0 * math.pi / DK) ** 3  # proper 7-volume per unit e^{6Hy} (v_t = 1)
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

The first line gives three names at once: $H = 1$, $m = 1$ (called `MASS`, because the letter m is used for other things below) and $L = 3$. `DK` is $\Delta k = 0.25$. `VOL7` is $\mathrm{Vol}_7 = (2\pi/\Delta k)^3 = 15875.2137$ (Section 15.2); `**` is the power. `PALETTE` is a list of eight colours, written as hexadecimal colour codes.

```
def make_grid(steps):
    points = 2 * steps + 1  # step ends and step midpoints
    y = [-L * ((points - 1 - f) / (points - 1)) for f in range(points)]
    half = 0.5 * L / steps  # the distance between two fine points
    weights = [(4.0 if f % 2 == 1 else 2.0) * half / 3.0 for f in range(points)]
    weights[0] = weights[-1] = half / 3.0  # Simpson: 1, 4, 2, 4, ..., 4, 1 (x h/6)
    return {"steps": steps, "h": L / steps, "points": points, "y": y,
            "ew": [math.exp(-H * v) for v in y], "e6": [math.exp(6 * H * v) for v in y],
            "simpson": np.array(weights)}
```

`make_grid` builds the solver's grid for a given number of RK4 steps. The fine grid has `points` $= 2G+1$ points (the step ends and the step midpoints). The second line is a **list comprehension** (a list built by a `for` inside square brackets): for $f = 0, 1, \ldots, 2G$ it computes $y_f = -L\,(2G - f)/(2G)$, which is $-L$ for $f = 0$ and 0 for $f = 2G$; this is the formula $y_f = -L + f\,h/2$ of Section 15.3 written so that the last point is exactly 0. `half` is the spacing $h/2$. The weights of Simpson's rule with spacing $h/2$ are $\frac{h/2}{3}$ times 4 at the odd points (`f % 2 == 1`: the remainder of f divided by 2 is 1), 2 at the inner even points and 1 at the two ends; the next line sets the two ends (`weights[-1]` is the last entry). The function returns a dictionary with the number of steps, the step $h = L/G$, the number of points, the coordinates, the factors e^{-Hy} (for κ; key `"ew"`) and e^{6Hy} (the proper volume factor; key `"e6"`) at every fine point, and the Simpson weights as an array.

```
GRID = make_grid(900)  # the canonical grid of the solver
Y = np.array(GRID["y"])
E6 = np.array(GRID["e6"])
simpson_test = float(np.sum(GRID["simpson"] * np.exp(Y)))
say(f"fine grid: {GRID['points']} points, step h = {GRID['h']:.6f}")
report("Simpson integral of e^y from -3 to 0", f"{simpson_test:.15f}")
check(abs(simpson_test - (1.0 - math.exp(-3.0))) < 1e-13,
      "Simpson's rule on the fine grid integrates e^y exactly to 1e-13")
```

`GRID` is the canonical grid ($G = 900$); `Y` and `E6` are its coordinates and volume factors as arrays. `simpson_test` is Simpson's rule applied to e^y: the sum of weight times value over the 1801 points

(`np.sum` adds the entries of an array; `float` turns the result into an ordinary number). The exact integral is $\int_{-3}^{0} e^{y}\,dy = 1-e^{-3} = 0.950212931632136$. The format `:.6f` prints six digits after the point, `:.15f` fifteen. Out [2] prints 1801 points, the step 0.003333 and the integral 0.950212931632177, which differs from the exact value by 4×10^{-14}, so the check passes.

In [3], one shooting integration.

```
def rhs(m_y, kk, je, a, b):
    return m_y * a - (kk + je) * b, (je - kk) * a - m_y * b
```

`rhs` (right-hand side) returns the pair (a', b') of the block equation for given values of M (`m_y`), κk (`kk`), $j(\varepsilon - v)$ (`je`), a and b: $a' = Ma - (\kappa k + j(\varepsilon - v))b$ and $b' = (j(\varepsilon - v) - \kappa k)a - Mb$, exactly Section 15.2.

```
def shoot(eps, k, j, a4, mass, pot, grid=GRID, store=False):
    h = grid["h"]
    kk = k * math.exp(-a4)  # k e^{-a4,0}; kappa k = kk e^{-Hy}
    coef = [(mass[f], kk * grid["ew"][f], j * (eps - pot[f]))
            for f in range(grid["points"])]
```

`shoot` integrates from the tip to the brane for the trial energy `eps`, the momentum `k`, the block type `j`, the slice `a4` and the potentials `mass` (M) and `pot` (v), given as lists on the fine grid. The arguments `grid=GRID` and `store=False` have **default values**, used when the caller does not give them. `kk` is $k\,e^{-a_{4,0}}$, so that κk at a point is `kk` times e^{-Hy}. `coef` is a list with one triple $(M, \kappa k, j(\varepsilon - v))$ per fine point, computed once before the integration.

```
    a, b = 1.0, 0.0  # the tip condition b(-L) = 0
    theta, raw = 0.0, 0.0  # the followed angle and the last angle from atan2
    area, r2_prev = 0.0, 1.0  # integral of r^2 (trapezoid) and r^2 at the last node
    nodes = [(a, b)] if store else None
```

The start values: $(a, b) = (1, 0)$ at the tip (the tip condition), the followed angle $\theta = 0$ and the last raw angle 0, the integral of r^2 so far 0, and $r^2 = 1$ at the tip. If `store` is true, the list `nodes` keeps the values at the step ends, starting with the tip; otherwise it is `None` (Python's word for "nothing").

```
    for i in range(grid["steps"]):
        c0, c1, c2 = coef[2 * i], coef[2 * i + 1], coef[2 * i + 2]  # end, mid, end
        k1a, k1b = rhs(*c0, a, b)
        k2a, k2b = rhs(*c1, a + 0.5 * h * k1a, b + 0.5 * h * k1b)
        k3a, k3b = rhs(*c1, a + 0.5 * h * k2a, b + 0.5 * h * k2b)
        k4a, k4b = rhs(*c2, a + h * k3a, b + h * k3b)
        a += h / 6.0 * (k1a + 2.0 * k2a + 2.0 * k3a + k4a)
        b += h / 6.0 * (k1b + 2.0 * k2b + 2.0 * k3b + k4b)
```

The loop makes the G RK4 steps. Step i runs from the fine point $2i$ to the fine point $2i + 2$; its midpoint is the fine point $2i + 1$, so `c0`, `c1`, `c2` are the coefficients at the start, the middle and the end of the step. `rhs(*c0, a, b)` passes the three numbers of the triple `c0` as the first three arguments (the star unpacks it). The four lines compute the four slopes of the classical Runge-Kutta

rule of Chapter 2: k_1 at the start, k_2 and k_3 at the midpoint (from the half-step predictions with k_1 and with k_2), k_4 at the end (from the full-step prediction with k_3). The last two lines advance a and b by $\frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$; `a += x` means `a = a + x`.

```
        new = math.atan2(b, a)
        change = new - raw
        if change > math.pi:  # bring the change of angle into (-pi, pi]
            change -= 2.0 * math.pi
        elif change <= -math.pi:
            change += 2.0 * math.pi
        theta += change
        raw = new
```

These lines follow the Pruefer angle. `math.atan2(b, a)` is the angle of the point (a, b), a number between $-\pi$ and π; it jumps by 2π when the point crosses the negative a-axis. One step is so short that the true angle changes by much less than π; so the change `new - raw` is brought into the interval from $-\pi$ to π by adding or subtracting 2π, and that change is added to `theta`. In this way `theta` follows the angle continuously and may grow beyond 2π (Section 15.3).

```
        r2 = a * a + b * b
        area += 0.5 * h * (r2_prev + r2)  # trapezoid rule over this step
        r2_prev = r2
        if store:
            nodes.append((a, b))
    return j * theta, area / r2_prev, nodes
```

`r2` is $r^2 = a^2 + b^2$ at the new step end; the trapezoid rule adds $\frac{h}{2}(r_i^2 + r_{i+1}^2)$ for the step. When `store` is true, the new values are appended to `nodes`. After the loop the function returns three things: $\Phi = j\,\theta(0)$, the derivative $d\Phi/d\varepsilon = \int r^2 dy / r(0)^2$ (`r2_prev` now holds $r(0)^2$), and the stored values.

```
FREE_MASS = [MASS] * GRID["points"]  # M = m everywhere (no interaction)
FREE_POT = [0.0] * GRID["points"]  # v = 0 everywhere
phi_zero, _, _ = shoot(0.0, 0.0, 1, 0.0, FREE_MASS, FREE_POT)
report("Phi(0) for k = 0, j = +1 (the zero mode: b stays 0)", phi_zero)
check(phi_zero == 0.0, "at eps = 0 and k = 0 the Pruefer angle stays exactly 0")
```

`[MASS] * n` is a list of n copies of 1.0: the free potentials $M = m$ and $v = 0$ on the 1801 fine points. The shot at $\varepsilon = 0$, $k = 0$, $j = +1$, $a_{4,0} = 0$ is the zero mode; the underscores `_` receive the two results that are not needed. With $k = 0$ and $\varepsilon - v = 0$ the equation for b is $b' = -Mb$, and $b = 0$ at the tip; so every RK4 stage gives exactly $b = 0$, `atan2(0, a)` is exactly 0 for $a > 0$, and $\Phi = 0$ without any rounding error. Out [3] prints the RESULT line with the value 0.0 and the PASS line. The text cell that follows the cell in the notebook repeats the derivation of $d\Phi/d\varepsilon$ of Section 15.3.

In [4], the phase function.

```
energies = np.linspace(-4.0, 4.0, 401)  # 401 trial energies
phase = np.array([shoot(e, 0.0, 1, 0.0, FREE_MASS, FREE_POT)[0] for e in energies])
```

`np.linspace(-4.0, 4.0, 401)` makes 401 equally spaced numbers from -4 to 4 (spacing 0.02). For each of them the cell shoots with $k = 0$, $j = +1$ and no interaction and keeps the first result, Φ (`[0]` takes the first entry of the returned triple).

```
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(energies, phase / math.pi, color=PALETTE[0], lw=2.0,
        label="$\\Phi(\\varepsilon)/\\pi$, $k = 0$, $j = +1$")
for whole in range(-4, 5):
    ax.axhline(whole, color=PALETTE[1], lw=0.8, alpha=0.7)  # even targets l pi
    ax.axhline(whole + 0.5, color=PALETTE[2], lw=0.8, ls="--", alpha=0.7)  # odd
ax.plot([], [], color=PALETTE[1], lw=0.8, label="even targets $l$")
ax.plot([], [], color=PALETTE[2], lw=0.8, ls="--", label="odd targets $l + 1/2$")
```

`plt.subplots` makes a figure `fig` with one drawing area `ax` of 7.5 by 4.5 inches. `ax.plot` draws Φ/π against ε as a blue line of width 2 (`lw`); the `label` is the text of the legend, written in the math notation of matplotlib (a doubled backslash in a Python string is one backslash). The loop draws horizontal lines (`axhline`) at the whole numbers -4 to 4 (the even targets $l\pi$, divided by π) and dashed lines (the line style `ls` set to two hyphens) half-way between them (the odd targets); `alpha=0.7` makes them slightly transparent. The two `ax.plot([], [], ...)` lines draw nothing (empty lists) but put the two kinds of lines into the legend.

```
ax.set_xlabel("trial energy $\\varepsilon$ (units of $m$)")
ax.set_ylabel("$\\Phi(\\varepsilon) / \\pi$")
ax.set_title("The phase function counts the levels")
ax.set_ylim(-3.2, 3.2)
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "phase_function",
            "The phase function $\\Phi(\\varepsilon)/\\pi$ (vertical axis) of the "
...)
check(bool(np.all(np.diff(phase) > 0.0)), "Phi increases strictly at all 400 steps")
```

These lines label the axes, set the title, show the vertical range from -3.2 to 3.2 and draw the legend in the upper left corner. `save_figure` saves Figure 15e.1 with its caption (shortened here; the complete caption is printed under the figure in Section 15.8). `np.diff(phase)` is the array of the 400 differences between neighbouring values; `> 0.0` compares each with 0, and `np.all` is true when every comparison is true. Out [4] prints the figure line and PASS: Φ increases at all 400 steps, as Section 15.3 proved. In the figure the curve is nearly flat between the levels ± 1.29 around $\varepsilon = 0$ (there it passes the zero mode at $\Phi = 0$) and climbs steeply through each level; this is the gap of the free field.

In [5], Newton's method inside a bracket.

```
def target(parity, label):
    return label * math.pi + (0.0 if parity == "even" else 0.5 * math.pi)
```

The target t_l of a level: $l\pi$ for even parity, $\pi/2 + l\pi$ for odd parity. `a if condition else b` is Python's choice between two values.

```
def find_level(k, j, parity, label, a4, mass, pot, guess, grid=GRID, tol=1e-13):
    t = target(parity, label)
    def g(e):  # Phi - target and its derivative at the energy e
        phi, dphi, _ = shoot(e, k, j, a4, mass, pot, grid)
        return phi - t, dphi
    ec = guess
    gc, dc = g(ec)
    if gc == 0.0:
        return ec
    step = min(max(abs(gc / dc) * 1.2, 1e-4), 2.0)  # the first trial step
```

`find_level` returns the energy of the level with the given momentum, block type, parity and label in the given potentials, starting from the energy `guess`. The inner function `g` shoots at the energy `e` and returns $\Phi - t_l$ and $d\Phi/d\varepsilon$. `ec`, `gc`, `dc` are the current energy, its value of $\Phi - t_l$ and its derivative. If the guess hits the target exactly, it is returned at once. The first trial step is 1.2 times the Newton estimate $|\Phi - t_l|/\Phi'$ of the distance to the root, but at least 10^{-4} and at most 2.

```
    if gc < 0.0:  # Phi is too small: walk up until Phi - t >= 0
        lo = ec
        while True:
            e = lo + step
            ge, de = g(e)
            if ge >= 0.0:
                hi = e  # now [lo, hi] brackets the root
                if abs(ge) < abs(gc):
                    ec, gc, dc = e, ge, de  # keep the better end for Newton
                break
            lo, ec, gc, dc = e, e, ge, de
            step *= 2.0  # double the step
```

If Φ is below the target, the root lies higher (because Φ increases). The loop `while True` repeats until `break`: it tries the energy `lo + step`; if $\Phi - t_l$ is no longer negative there, the root lies between `lo` and this energy, which becomes `hi`, and the end with the smaller $|\Phi - t_l|$ becomes the start of Newton's method; otherwise the trial energy becomes the new `lo` and the step is doubled. Doubling reaches any distance in a few steps.

```
    else:  # Phi is too large: walk down until Phi - t <= 0
        hi = ec
        while True:
            e = hi - step
            ge, de = g(e)
            if ge <= 0.0:
                lo = e
                if abs(ge) < abs(gc):
                    ec, gc, dc = e, ge, de
                break
            hi, ec, gc, dc = e, e, ge, de
            step *= 2.0
    if gc == 0.0:
        return ec
```

The mirror image: if Φ is above the target, the walk goes down until $\Phi - t_l \le 0$. Afterwards `[lo, hi]` brackets the root; if one end hit it exactly, it is returned.

```
    previous = math.inf
    for _ in range(300):
        if hi - lo <= tol:
            break
        new = ec - gc / dc  # the Newton step
        bisect = not lo < new < hi or abs(gc) > 0.5 * previous
        if bisect:
            new = 0.5 * (lo + hi)  # bisection instead
        previous = abs(gc)
        moved = abs(new - ec)  # how far this step moved the energy
        ec = new
        gc, dc = g(ec)
        if gc == 0.0:
            break
        if gc < 0.0:
            lo = ec
        else:
            hi = ec
        if moved <= tol and not bisect:
            break  # a Newton step that moves less than tol: converged
    return ec
```

The refinement, at most 300 steps (`for _ in range(300)` repeats 300 times without using the counter). It stops when the bracket is shorter than the tolerance 10^{-13}. Otherwise it computes the Newton step $\varepsilon - (\Phi - t_l)/\Phi'$. `bisect` is true when the Newton point is not strictly inside the bracket, or when the previous step did not halve $|\Phi - t_l|$ (`previous` holds the earlier value; it starts as infinity, `math.inf`, so the first step is never rejected for this reason); then the midpoint of the bracket is used instead. After the step the new energy is evaluated, the bracket end on the same side of the root is replaced (the sign of $\Phi - t_l$ tells the side), and the loop stops when a Newton step moved the energy by at most the tolerance. This is the solver's rule of Section 15.3. The cell defines functions only and prints nothing.

In [6], the exact levels at $k = 0$ and the solver's levels.

```
def odd_momentum(n):
    lo, hi = (n - 0.5) * math.pi / L, n * math.pi / L
    f = lambda p: MASS * math.sin(p * L) + p * math.cos(p * L)
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        lo, hi = (mid, hi) if f(lo) * f(mid) > 0.0 else (lo, mid)
    return 0.5 * (lo + hi)
```

The odd levels need the roots p of $\tan(pL) = -p/M$ (Section 15.3). Multiplied by $M\cos(pL)$ this is $f(p) = M\sin(pL) + p\cos(pL) = 0$, which has no division. The n-th root lies between $(n - \frac{1}{2})\pi/L$ and $n\pi/L$: at the left end $\cos(pL) = 0$ and $f = \pm M$, at the right end $\sin(pL) = 0$ and $f = \pm p$ with the opposite sign, so f changes sign in between. `lambda p: ...` defines a small function without a name. The loop is **bisection**: if f has the same sign at `lo` and at the midpoint (their product is

positive), the root lies in the right half, otherwise in the left half. After 200 halvings the interval is far below rounding size, and its midpoint is returned.

```
def exact_level(parity, label):
    if parity == "even":
        n = abs(label)
        return math.copysign(math.sqrt(MASS ** 2 + (n * math.pi / L) ** 2), label) \
            if n else 0.0
    n = label + 1 if label >= 0 else -label  # labels 0, 1, ... and -1, -2, ...
    return math.copysign(math.sqrt(MASS ** 2 + odd_momentum(n) ** 2), label + 0.5)
```

The exact free level at $k = 0$ with a given parity and label. For even parity the label l belongs to $n = |l|$ and the level is $\pm\sqrt{M^2 + (n\pi/L)^2}$ with the sign of l (`math.copysign(x, s)` is $|x|$ with the sign of s); the label 0 is the zero mode, $\varepsilon = 0$ (`if n else 0.0`: a nonzero n counts as true). The backslash at the end of a line continues the statement on the next line. For odd parity the labels $0, 1, 2, \ldots$ are the positive levels with $n = l + 1$, and the labels $-1, -2, \ldots$ the negative ones with $n = -l$; the sign is that of $l + \frac{1}{2}$, which is positive for $l \geq 0$ and negative for $l \leq -1$.

```
RECORD_FREE = "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
recorded = {}
for line in repository_file(RECORD_FREE).read_text(encoding="utf-8").split("\n")[1:]:
    if line:
        m_, l_, parity, label, eps_num, eps_exact, _ = line.split(",")
        if float(m_) == 1.0 and float(l_) == 3.0:
            recorded[(parity, int(label))] = (float(eps_num), float(eps_exact))
```

The record file of the solver's free spectra at $k = 0$ is read as text, split into lines, and the first line (the column names) is skipped (`[1:]` takes everything from the second entry on). Each nonempty line is split at the commas into seven fields: the mass, L, the parity, the label, the solver's numerical level, the exact level and one more column that is not needed. Only the rows with $M = 1$ and $L = 3$ are kept (the record also tests other masses and lengths), in a dictionary whose key is the pair (parity, label).

```
worst_exact, worst_record = 0.0, 0.0
say("parity label   shooting          exact              difference")
for parity in ("even", "odd"):
    for label in range(-3, 6):
        exact = exact_level(parity, label)
        found = find_level(0.0, 1, parity, label, 0.0, FREE_MASS, FREE_POT, exact + 0.01)
        say(f"{parity:5} {label:3d}  {found:17.13f}  {exact:17.13f}"
            f"  {found - exact:10.2e}")
        if abs(exact) < 4.0:
            worst_exact = max(worst_exact, abs(found - exact))
        if (parity, label) in recorded:
            worst_record = max(worst_record, abs(found - recorded[(parity, label)][0]))
```

For both parities and the labels -3 to 5 the cell computes the exact level, finds the level by shooting (with the guess 0.01 above the exact value), and prints a table row: the parity in 5 characters, the label in 3 digits, both levels with 13 digits after the point, and their difference in the exponent

format `.2e` (two digits after the point and a power of ten). The largest difference is kept for $|\varepsilon| < 4$ (`worst_exact`; the record uses the same range for its tolerance), and the largest difference to the solver's recorded value for every row present in the record (`worst_record`).

```
report("largest |shooting - exact| for |eps| < 4", f"{worst_exact:.2e}")
report("largest |this notebook - solver record|", f"{worst_record:.2e}")
check(worst_exact < 5e-9,
      "the shooting levels equal the exact levels within 5e-9 for |eps| < 4",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_k0_analytic_spectra")
check(worst_record < 1e-12, "the levels equal the solver's levels within 1e-12",
      record=f"{RECORD_FREE}, column eps_numeric")
```

Out [6] shows the table. The differences grow with the energy, from 10^{-13} near ± 1.3 to 6.7×10^{-9} at the odd label 5 ($\varepsilon = 5.90$): the error of RK4 grows like $(h\varepsilon)^4$, because a higher level oscillates faster. The largest difference for $|\varepsilon| < 4$ is 7.23×10^{-10}, the value of the record's check free_k0_analytic_spectra. The first check therefore tests the tolerance 5×10^{-9} only for $|\varepsilon| < 4$, as the record does, and its PASS line says so (the check name is written on its own line of the call because the call would otherwise be longer than a line of the page); for $4 \le |\varepsilon| < 7$ the record allows 3×10^{-7}, far above the 6.7×10^{-9} of the table. The largest difference to the solver's own numbers is 7.99×10^{-15}: the Python code and the Rust program do the same arithmetic in a slightly different order. Both checks pass. The even label 0 is printed as -0.0000000000000, a difference of -1.4×10^{-19} from the exact zero, left by Newton's method.

In [7], orbitals with Hermite midpoints.

```
def orbital(eps, k, j, a4, mass, pot, grid=GRID):
    _, _, nodes = shoot(eps, k, j, a4, mass, pot, grid, store=True)
    h = grid["h"]
    kk = k * math.exp(-a4)
    a = np.zeros(grid["points"])
    b = np.zeros(grid["points"])
    a[0::2] = [p[0] for p in nodes]  # the step ends
    b[0::2] = [p[1] for p in nodes]
```

`orbital` returns the normalised orbital of a level on the whole fine grid. It shoots once more at the level's energy and keeps the values at the $G + 1$ step ends. `np.zeros(n)` is an array of n zeros. `a[0::2]` is every second entry starting with the first (the fine indices $0, 2, 4, \ldots$, the step ends), and the step-end values are written there.

```
    f = np.arange(0, grid["points"], 2)  # fine indices of the step ends
    m_y = np.array(mass)[f]
    kap = kk * np.array(grid["ew"])[f]
    je = j * (eps - np.array(pot)[f])
    da = m_y * a[f] - (kap + je) * b[f]  # derivatives from the equation
    db = (je - kap) * a[f] - m_y * b[f]
```

`np.arange(0, n, 2)` is the array $0, 2, 4, \ldots$ of the step-end indices; indexing an array with it picks those entries. So `m_y`, `kap` and `je` are M, κk and $j(\varepsilon - v)$ at the step ends, and `da`, `db` are the

derivatives a', b' there, computed from the block equation itself (this is where the Hermite formula gets its derivatives).

```
    a[1::2] = 0.5 * (a[f][:-1] + a[f][1:]) + h / 8.0 * (da[:-1] - da[1:])
    b[1::2] = 0.5 * (b[f][:-1] + b[f][1:]) + h / 8.0 * (db[:-1] - db[1:])
    norm = math.sqrt(float(np.sum(grid["simpson"] * (a * a + b * b))))
    return a / norm, b / norm
```

`a[1::2]` are the midpoints. For each step, `a[f][:-1]` is the value at its start (all step ends but the last) and `a[f][1:]` the value at its end (all but the first); the line is the Hermite formula $\frac{1}{2}(u_0+u_1)+\frac{h}{8}(u_0'-u_1')$ of Section 15.3, for all steps at once. Then Simpson's rule gives $\int (a^2+b^2)\,dy$, and both components are divided by its square root, so the returned orbital is normalised.

```
zero_a, zero_b = orbital(0.0, 0.0, 1, 0.0, FREE_MASS, FREE_POT)
bulk_eps = find_level(0.0, 1, "even", 1, 0.0, FREE_MASS, FREE_POT, 1.4)
bulk_a, bulk_b = orbital(bulk_eps, 0.0, 1, 0.0, FREE_MASS, FREE_POT)
band_eps = find_level(0.5, 1, "even", 0, 0.0, FREE_MASS, FREE_POT, 0.8)
band_a, band_b = orbital(band_eps, 0.5, 1, 0.0, FREE_MASS, FREE_POT)
```

Three orbitals of the free block $j=+1$: the zero mode ($\varepsilon=0$, $k=0$); the first even bulk level at $k=0$ (label 1, guess 1.4; its energy is 1.4479719, Out [6]); and the brane-band level at $k=0.5$ (the shell $n_2=4$; label 0, guess 0.8). Their energies appear in the panel titles of the figure.

```
fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.6), sharey=True,
                         layout="constrained")
for ax, (a, b, title) in zip(axes, [
        (zero_a, zero_b, "zero mode, $k = 0$, $\\varepsilon = 0$"),
        (bulk_a, bulk_b, f"bulk, $k = 0$, $\\varepsilon = {bulk_eps:.4f}$"),
        (band_a, band_b, f"brane band, $k = 0.5$, $\\varepsilon = {band_eps:.4f}$")]):
    ax.plot(Y, a, color=PALETTE[0], lw=2.0, label="$a(y)$")
    ax.plot(Y, b, color=PALETTE[1], lw=2.0, ls="--", label="$b(y)$")
    ax.set_title(title, fontsize=9)
    ax.set_xlabel("hidden coordinate $y$")
axes[0].set_ylabel("orbital (normalised)")
axes[0].legend()
save_figure(fig, "orbitals",
            "Three normalised orbitals of the free block $j = +1$ (components $a$, "
...)
```

`plt.subplots(1, 3, ...)` makes one row of three drawing areas (`axes`), which share the vertical axis (`sharey=True`); `layout="constrained"` spaces them so that no labels overlap. `zip` pairs each drawing area with one triple (the two components and a title), and the loop draws a solid and b dashed against y. The first panel gets the vertical label and the legend, and `save_figure` saves Figure 15e.2. The student should see that the zero mode is e^{y} and sits at the brane with $b=0$ everywhere, that the bulk orbital fills the whole interval, and that the band orbital is again bound to the brane; every b vanishes at both ends.

```
exact_zero = math.sqrt(2.0 * MASS / (1.0 - math.exp(-2.0 * MASS * L))) * np.exp(MASS * Y)
zero_error = max(float(np.max(np.abs(zero_a - exact_zero))),
```

```
                float(np.max(np.abs(zero_b))))
report("largest deviation of the zero mode from its exact form", f"{zero_error:.2e}")
check(zero_error < 1e-11, "the zero mode is (e^{My}, 0), normalised, to 1e-11",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_zero_mode_exact")
```

`exact_zero` is the normalised zero mode $\sqrt{2M/(1-e^{-2ML})}\,e^{My}$ of Section 15.3 on the fine grid. `zero_error` is the larger of the largest deviation of a from it and the largest $|b|$. Out [7] prints 1.35×10^{-12}, the value of the record's check free_zero_mode_exact, and PASS.

In [8], the order 4 of RK4.

```
STEPS = [100, 200, 400, 800]
errors = {2: [], 3: []}
for steps in STEPS:
    grid = make_grid(steps)
    mass = [MASS] * grid["points"]
    pot = [0.0] * grid["points"]
    for label in errors:
        found = find_level(0.0, 1, "even", label, 0.0, mass, pot,
                           exact_level("even", label), grid=grid)
        errors[label].append(abs(found - exact_level("even", label)))
```

For the grids of 100, 200, 400 and 800 steps the cell finds the even levels with the labels 2 and 3 at $k = 0$ (exact values $\sqrt{1+(2\pi/3)^2} = 2.3208815$ and $\sqrt{1+\pi^2} = 3.2969083$) and appends the size of the error to a list for each label. Looping over a dictionary (`for label in errors`) visits its keys, 2 and 3.

```
ratios = []
for label, errs in errors.items():
    steps_ratios = [errs[i] / errs[i + 1] for i in range(len(errs) - 1)]
    ratios += steps_ratios
    say(f"label {label}: errors " + ", ".join(f"{e:.2e}" for e in errs)
        + "; ratios " + ", ".join(f"{r:.2f}" for r in steps_ratios))
```

For each label the three ratios of the error on one grid to the error on the grid with twice as many steps are computed, collected in `ratios` and printed (`", ".join(...)` writes the numbers separated by commas). Out [8] shows the errors 2.45×10^{-7}, 1.53×10^{-8}, 9.59×10^{-10}, 5.99×10^{-11} for the label 2 and ratios between 15.96 and 16.00.

```
steps_h = [L / s for s in STEPS]
fig, ax = plt.subplots()
for colour, (label, errs) in zip(PALETTE, errors.items()):
    ax.loglog(steps_h, errs, "o-", color=colour, lw=2.0, ms=7,
              label=f"even level $l = {label}$")
ax.loglog(steps_h, [errors[3][0] * (s / steps_h[0]) ** 4 for s in steps_h], "--",
          color="0.4", lw=1.2, label="slope 4: error $\\propto h^4$")
```

`steps_h` are the steps $h = 3/G$. `ax.loglog` draws with logarithmic scales on both axes; the style `"o-"` draws dots joined by lines, `ms` is the size of the dots. On such axes a power law error $= Ch^4$ is

a straight line of slope 4; the dashed grey line (`color="0.4"` is a grey) is that law through the first error of the label 3.

```
ax.set_xlabel("step $h$ (units of $1/m$)")
ax.set_ylabel("|shooting level - exact level| (units of $m$)")
ax.set_title("RK4: halving the step divides the error by 16")
ax.legend()
save_figure(fig, "rk4_convergence",
            "Error of two shooting levels (vertical axis, logarithmic, units of $m$) "
...)
middle = sorted(ratios)[len(ratios) // 2]
report("median error ratio for halving the step", f"{middle:.2f}")
check(15.0 < middle < 17.0, "the error ratio is 16 within 1 (fourth order)",
      record="Revision/kohn_sham/reports/ks-rust-determinism.json, check "
             "refined_free_spectra_convergence_order")
```

After the labels and the legend, `save_figure` saves Figure 15e.3. `sorted(ratios)` sorts the six ratios; the entry at the position `len(ratios) // 2` (`//` divides and drops the remainder; here position 3) is the **median**, the middle value. It is 16.00, the ratio 2^4 of a fourth-order method, the same number as in the record's check refined_free_spectra_convergence_order. In the figure the dots lie on lines parallel to the dashed line of slope 4.

In [9], the brane band and the redshift.

```
THEORY = json.loads(repository_file("Revision/kohn_sham/ks-theory.json").read_text(
    encoding="utf-8"))  # the theory file of the solver
C_RECORD = float(THEORY["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
c_formula = (2 * MASS / (2 * MASS - H) * (1 - math.exp(-(2 * MASS - H) * L))
             / (1 - math.exp(-2 * MASS * L)))
```

`json.loads` turns the text of the theory file into Python dictionaries and lists. `C_RECORD` is the recorded slope $c = 1.9051482536448664$, and `c_formula` computes $c = \frac{2M}{2M-H} \cdot \frac{1-e^{-(2M-H)L}}{1-e^{-2ML}}$ of Section 15.4 again.

```
def band(k, a4):
    return find_level(k, 1, "even", 0, a4, FREE_MASS, FREE_POT, 1.9 * k * math.exp(-a4))
```

`band(k, a4)` is the brane-band level: block type +1, even parity, label 0, free potentials, with the guess $1.9\,k\,e^{-a_{4,0}}$ from the small-k law.

```
momenta = np.linspace(0.0, 2.0, 41)
fig, ax = plt.subplots()
slopes = []
for shade, a4 in zip(("#86b6ef", "#2a78d6", "#104281"), (0.0, 1.0, 2.0)):
    curve = [band(k, a4) for k in momenta]
    ax.plot(momenta, curve, "o-", color=shade, ms=4, lw=1.8,
            label=f"$a_{{4,0}} = {a4:.0f}$")
    ax.plot(momenta, c_formula * momenta * math.exp(-a4), ":", color=shade, lw=1.2)
```

For the slices $a_{4,0} = 0, 1, 2$ (in light, middle and dark blue) the cell computes the band at 41 momenta from 0 to 2 and draws it, together with the dotted tangent $c\,k\,e^{-a_{4,0}}$. In an f-string a doubled brace `{{` prints one brace, so the label reads $a_{4,0} = 0$ and so on.

```
    s1, s2 = band(1e-4, a4) / 1e-4, band(2e-4, a4) / 2e-4  # s(k) and s(2k)
    slopes.append((4.0 * s1 - s2) / 3.0 * math.exp(a4))  # slope times e^{a4,0}
    if a4 == 0.0:
        s4 = band(4e-4, a4) / 4e-4  # s(4k), to measure how the error grows
        growth = (s4 - s2) / (s2 - s1)  # 4 if the error of s is proportional to k^2
```

Still inside the loop: $s(k) = \varepsilon(k)/k$ at $k = 10^{-4}$ and 2×10^{-4}. The band is an odd function of k (the text cell before In [9] derives this from the two block symmetries), so $\varepsilon = ck - dk^3 + \ldots$ and $s(k) = c - dk^2 + \ldots$; the **Richardson extrapolation** $\frac{1}{3}(4s(k) - s(2k))$ removes the k^2 term: $4(c - dk^2) - (c - 4dk^2) = 3c$. Multiplied by $e^{a_{4,0}}$ it must be the slice-independent c. At the first slice the cell also computes $s(4k)$ and the ratio of the changes $(s(4k) - s(2k))/(s(2k) - s(k))$, which is $(16 - 4)/(4 - 1) = 4$ if the error of s is proportional to k^2.

```
ax.plot([], [], ":", color="0.4", label="$c\\,k\\,e^{-a_{4,0}}$")
ax.set_xlabel("3-momentum $k$ (units of $m$)")
ax.set_ylabel("brane-band level $\\varepsilon$ (units of $m$)")
ax.set_title("The brane band redshifts along the history")
ax.legend()
save_figure(fig, "brane_band",
            "The brane-band level $\\varepsilon(k)$ (vertical axis, units of $m$) "
...)
rescale = max(abs(band(k, a4) - band(k * math.exp(-a4), 0.0))
              for k in (0.25, 1.0, 2.5) for a4 in (0.5, 1.0, 2.0))
```

After the legend entry for the dotted tangents and the labels, `save_figure` saves Figure 15e.4. `rescale` is the largest violation of the rescaling identity $\varepsilon(k; a_{4,0}) = \varepsilon(k\,e^{-a_{4,0}}; 0)$ for three momenta and three slices; the two `for` clauses inside `max(...)` run over all nine pairs.

```
report("growth of the error of eps(k)/k when k doubles", f"{growth:.4f}")
report("slope c times e^{a4,0} at a4,0 = 0, 1, 2",
       ", ".join(f"{s:.10f}" for s in slopes))
report("largest violation of the rescaling identity", f"{rescale:.1e}")
check(abs(c_formula - C_RECORD) < 1e-14 and
      max(abs(s - C_RECORD) for s in slopes) < 1e-9 * C_RECORD,
      "the brane-band slope is c e^{-a4,0} with c = 1.9051482536",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_brane_band_slope")
check(rescale < 1e-12, "eps(k, a4,0) = eps(k e^{-a4,0}, 0) within 1e-12",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_rescaling_relation_and_band_monotone")
```

Out [9] prints the growth factor 4.0000, the three extrapolated slopes, each 1.9051482536, and the violation 0.0, and both checks pass. The violation is exactly zero, not merely small, because the two calls do the same arithmetic: `shoot` uses k only through the product $k\,e^{-a_{4,0}}$, and $k\,e^{-a_{4,0}}$ at the slice $a_{4,0}$ and $(k\,e^{-a_{4,0}})\,e^{0}$ at the slice 0 are the same number. In Figure 15e.4 each band curve is

the previous one with the momentum axis stretched by $e = 2.718$, and every curve lies below its dotted tangent.

In [10], the self-consistent loop.

```
POINTS = GRID["points"]
P_FACTOR = 1.0 / (E6 * VOL7)  # P = e^{-6Hy}/Vol_7 on the fine grid
```

`POINTS` is 1801, and `P_FACTOR` is the array of $P = e^{-6Hy}/\mathrm{Vol}_7$ at the fine points (Section 15.2).

```
def iteration(x, lam, a4, guesses):
    mass = (MASS + x[:POINTS]).tolist()
    pot = x[POINTS:].tolist()
    n = np.zeros(POINTS)
    s = np.zeros(POINTS)
    levels = {}
```

`iteration` performs one Kohn-Sham iteration for $N = 8$. The input `x` is the array of the 2×1801 unknowns: its first half (`x[:POINTS]`) is $M - m$ and its second half (`x[POINTS:]`) is v; they become the lists `mass` and `pot` that `shoot` expects (`.tolist()`). The densities n and S start at zero, and `levels` will hold the two occupied levels.

```
    for j in (1, -1):  # the two block types, each level four-fold (g = 4)
        eps = find_level(0.0, j, "even", 0, a4, mass, pot, guesses.get(j, 0.0))
        a, b = orbital(eps, 0.0, j, a4, mass, pot)
        weight = 0.5 * 4.0 * 1.0  # w g f with w = 1/2 (the Z2 doubling)
        n += weight * P_FACTOR * (a * a + b * b)
        s += weight * P_FACTOR * j * 2.0 * a * b
        levels[j] = eps
```

For each block type the occupied level is the zero mode: $k = 0$, even parity, label 0. Its energy is found starting from the previous iteration's energy (`guesses.get(j, 0.0)` gives 0 in the first iteration, when `guesses` is empty), and its orbital is computed. The weight $wgf = \frac{1}{2} \cdot 4 \cdot 1 = 2$ multiplies the orbital densities $P(a^2 + b^2)$ and $P\, j\, 2ab$, which are added to n and S (the formulas of Section 15.2).

```
    x_out = np.concatenate([lam * (15.0 / 16.0) * s, lam * (-1.0 / 16.0) * n])
    e_int = lam * (15.0 / 32.0 * s * s - 1.0 / 32.0 * n * n)
    energy = sum(4.0 * e for e in levels.values()) \
        - 2.0 * VOL7 * float(np.sum(GRID["simpson"] * E6 * e_int))
    return float(np.max(np.abs(x_out - x))), x_out, energy, levels
```

The output potentials are $\frac{15}{16}\lambda S$ and $-\frac{1}{16}\lambda n$, joined into one array (`np.concatenate`). `e_int` is the interaction energy density, and `energy` is $E_{KS} = \sum gf\varepsilon - 2\,\mathrm{Vol}_7 \int e^{6Hy} e_{int}\, dy$ with $gf = 4$ for each of the two levels and Simpson's rule for the integral. The function returns the size of the residual (the largest $|x_{out} - x|$), the output potentials, the energy and the levels.

```
class Anderson:
    def __init__(self, depth=6, beta=0.4):
        self.depth, self.beta = depth, beta
        self.xs, self.rs, self.best = [], [], math.inf
```

A **class** defines a new kind of object that keeps data between calls. An `Anderson` object stores the depth 6, the factor $\beta = 0.4$, the lists of the last inputs `xs` and residuals `rs`, and the smallest residual size seen so far, `best`. `__init__` runs when the object is made; `self` is the object itself.

```
    def next(self, x, r):
        size = float(np.max(np.abs(r)))
        if size > 10.0 * self.best and self.xs:
            self.xs, self.rs = [], []  # the residual grew tenfold: start again
        self.best = min(self.best, size)
        self.xs = (self.xs + [x.copy()])[-self.depth:]
        self.rs = (self.rs + [r.copy()])[-self.depth:]
        count = len(self.xs)
```

`next` receives the current input and residual and returns the next input. If the residual is more than ten times the smallest one seen and there is a history, the history is forgotten (an empty list counts as false, a nonempty one as true). The new pair is appended, and `[-self.depth:]` keeps only the last six. `count` is the number p of stored pairs.

```
        system = np.zeros((count + 1, count + 1))
        gram = np.array([[float(np.dot(u, v)) for v in self.rs] for u in self.rs])
        system[:count, :count] = gram + 1e-12 * np.max(np.diag(gram)) * np.eye(count)
        system[:count, count] = 1.0
        system[count, :count] = 1.0
        right = np.zeros(count + 1)
        right[count] = 1.0
        weights = np.linalg.solve(system, right)[:count]
        return sum(c * (xi + self.beta * ri)
                   for c, xi, ri in zip(weights, self.xs, self.rs))
```

These lines build and solve the bordered system of Section 15.5. `gram` is the Gram matrix $G_{ik} = r_i \cdot r_k$ (`np.dot` is the dot product of two arrays). The upper left block of `system` is G plus 10^{-12} times its largest diagonal entry on the diagonal (`np.eye(count)` is the unit matrix), the last column and the last row (without the corner) are ones, and the right side is $(0, \ldots, 0, 1)$. `np.linalg.solve` solves the linear system; its first p entries are the weights c_i (the last one is $-\nu$). The function returns $\sum_i c_i(x_i + \beta r_i)$.

```
def solve_n8(lam, a4=0.0, mixing="anderson", tol=1e-11, limit=200):
    x = np.zeros(2 * POINTS)
    mixer = Anderson()
    guesses, history = {}, []
    for it in range(1, limit + 1):
        residual, x_out, energy, levels = iteration(x, lam, a4, guesses)
        guesses = levels
        history.append((it, residual, energy))
```

```
        if residual <= tol:
            return energy, levels, history, x
        r = x_out - x
        x = mixer.next(x, r) if mixing == "anderson" else x + 0.4 * r
    raise RuntimeError("no convergence")
```

`solve_n8` runs the loop for the coupling `lam`. It starts from the free potentials ($x = 0$), makes a new `Anderson` object, and repeats at most 200 iterations: one iteration, its levels as the next guesses, its number, residual and energy appended to `history`; if the residual is at most 10^{-11}, the energy, levels, history and potentials are returned; otherwise the next input is made by Anderson mixing or, if `mixing` is `"linear"`, by linear mixing $x + 0.4\,r$. If the loop never converges, a `RuntimeError` stops the notebook. The cell defines functions and the class only and prints nothing.

In [11], the state $N = 8$ with $+\lambda_1$.

```
params = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))
calib = {int(c["N"]): c for c in params["couplingCalibration"]["values"]}
LAM1, LAM2 = calib[8]["lambda1"], calib[8]["lambda2"]
```

The parameters file of the record is read; `calib` is a **dictionary comprehension** that files the calibration entries under their particle number (8, 136, 688), and `LAM1`, `LAM2` are the couplings $\lambda_1 = 0.01946$ and $\lambda_2 = 0.05838$ of $N = 8$ (Section 15.2).

```
energy, levels, history, x_final = solve_n8(LAM1)
say("iteration   residual        E_KS")
for it, residual, e in history:
    say(f"{it:9d}   {residual:9.3e}   {e:.13e}")
```

The loop is run with $+\lambda_1$ and its history is printed, one line per iteration: the iteration number, the residual and the energy. Out [11] shows the residual falling from 0.1 in the first iteration to 7.5×10^{-13} in the thirteenth, and the energy settling at $-9.8684261908764 \times 10^{-4}$.

```
SUMMARY = "Revision/kohn_sham/results/ground/summary.csv"
with open(repository_file(SUMMARY), newline="", encoding="utf-8") as handle:
    ground = {row["id"]: row for row in csv.DictReader(handle)}
runs = {run["id"]: run for run in json.loads(repository_file(
    "Revision/kohn_sham/results/ground/runs.json").read_text(encoding="utf-8"))}
recorded_history = runs["N8_lamp1_a00"]["scfHistory_iteration_residual_E"]
early = max(abs(h[1] - r[1]) / r[1] for h, r in zip(history[:6], recorded_history[:6]))
```

`with open(...) as handle:` opens the record's table of ground states and closes it again after the indented block; `csv.DictReader` reads each row as a dictionary from the column names to the values, and the rows are filed under their state name. The file `runs.json` holds for every ground state the residual and energy of each iteration; `recorded_history` is that list for `N8_lamp1_a00`. `early` is the largest relative difference between this notebook's residuals and the recorded ones in the first six iterations.

```
report("E_KS of N = 8 with lambda_1", f"{energy:.15e}")
report("iterations (this notebook / record)",
       f"{len(history)} / {len(recorded_history)}")
report("largest relative difference of the first six residuals", f"{early:.1e}")
check(abs(energy - float(ground["N8_lamp1_a00"]["E_KS"])) < 1e-14
      and abs(levels[1] - float(ground["N8_lamp1_a00"]["HOMO"])) < 1e-13,
      "E_KS and HOMO of N8_lamp1_a00 reproduced", record=f"{SUMMARY}, row N8_lamp1_a00")
check(early < 1e-5, "the first six residuals equal the recorded ones within 1e-5",
      record="Revision/kohn_sham/results/ground/runs.json, N8_lamp1_a00")
check(abs(levels[1] - levels[-1]) < 1e-13, "the levels of j = +1 and j = -1 are equal")
```

The cell reports $E_{KS} = -9.868426190876385 \times 10^{-4}$, 13 iterations here and in the record, and the relative difference 3.0×10^{-7} of the first six residuals (the later residuals are differences of nearly equal numbers and depend on the order of the arithmetic, as the text cell explains). The three checks pass: the energy and the occupied level (HOMO) agree with the record within 10^{-14} and 10^{-13}, the early residuals within 10^{-5}, and the levels of the two block types are equal, as the symmetry $\sigma_3 h_j(k) \sigma_3 = h_{-j}(-k)$ demands at $k = 0$.

In [12], Anderson against linear mixing.

```
energy_lin, _, history_lin, _ = solve_n8(LAM1, mixing="linear")
fig, ax = plt.subplots()
ax.semilogy([h[0] for h in history_lin], [h[1] for h in history_lin], "s-",
            color=PALETTE[1], ms=4, lw=1.5, label="linear mixing, $\\beta = 0.4$")
ax.semilogy([h[0] for h in history], [h[1] for h in history], "o-",
            color=PALETTE[0], ms=7, lw=2.0, label="Anderson mixing (this notebook)")
ax.semilogy([r[0] for r in recorded_history], [r[1] for r in recorded_history], "x",
            color="black", ms=9, label="Anderson mixing (Rust solver, record)")
ax.axhline(1e-11, color="0.4", ls=":", lw=1.2, label="tolerance $10^{-11}$")
```

The same state is solved with linear mixing. `ax.semilogy` draws with a logarithmic vertical axis: the residual against the iteration for linear mixing (orange squares, `"s-"`), for this notebook's Anderson run (blue dots) and for the solver's recorded run (black crosses, `"x"` without a line), and a dotted line at the tolerance.

```
ax.set_xlabel("iteration")
ax.set_ylabel("residual (units of $m$)")
ax.set_title("Self-consistent loop for $N = 8$, $\\lambda = +\\lambda_1$")
ax.legend(fontsize=8)
save_figure(fig, "scf_mixing",
            "Residual of the self-consistent loop (vertical axis, logarithmic, units "
...)
report("iterations: Anderson / linear", f"{len(history)} / {len(history_lin)}")
report("|E(linear) - E(Anderson)|", f"{abs(energy_lin - energy):.1e}")
check(abs(energy_lin - energy) < 1e-12 and len(history_lin) > 2 * len(history),
      "linear mixing reaches the same energy, with more than twice the iterations")
```

`save_figure` saves Figure 15e.5. Out [12] reports 13 iterations for Anderson mixing and 47 for linear mixing, and an energy difference of 8.0×10^{-15}: both reach the same state, linear mixing with about four times the work. The check passes. In the figure the black crosses of the Rust solver sit on the blue dots.

In [13], the four couplings and the symmetry.

```
tags = {"lamp1": LAM1, "lamm1": -LAM1, "lamp2": LAM2, "lamm2": -LAM2}
results = {}
worst_e, worst_homo = 0.0, 0.0
for tag, lam in tags.items():
    e, lv, hist, xf = solve_n8(lam)
    results[tag] = (e, lv[1], xf)
    row = ground[f"N8_{tag}_a00"]
    worst_e = max(worst_e, abs(e - float(row["E_KS"])))
    worst_homo = max(worst_homo, abs(lv[1] - float(row["HOMO"])))
    say(f"{tag}: lambda {lam:+.5f}, E_KS {e:+.12e}, HOMO {lv[1]:+.12e},"
        f" {len(hist)} it.")
```

The dictionary `tags` maps the four run tags to the couplings $\pm\lambda_1$, $\pm\lambda_2$. For each, the loop is solved, the energy, the level of $j = +1$ and the final potentials are kept in `results`, the differences to the record's row (for example `N8_lamm1_a00`) are tracked, and a line is printed; the format `+.5f` always prints the sign. Out [13] shows that E_{KS} and the HOMO of $-\lambda$ are exactly the negatives of those of $+\lambda$, with 13 iterations for $\pm\lambda_1$ and 17 for $\pm\lambda_2$.

```
xf = results["lamp1"][2]
lumo = find_level(0.25, 1, "even", 0, 0.0, (MASS + xf[:POINTS]).tolist(),
                  xf[POINTS:].tolist(), 0.43)
report("LUMO of N8_lamp1_a00 (brane band at k = 0.25)", f"{lumo:.13f}")
antisym = max(abs(results["lamp1"][0] + results["lamm1"][0]),
              abs(results["lamp2"][0] + results["lamm2"][0]))
report("largest |E(+lambda) + E(-lambda)|", f"{antisym:.1e}")
```

The lowest empty level of the state with $+\lambda_1$ is the brane-band level of the first shell, $k = 0.25$, in the converged potentials; it is found with the guess 0.43 (the free gap). `antisym` is the larger of $|E(+\lambda_1) + E(-\lambda_1)|$ and $|E(+\lambda_2) + E(-\lambda_2)|$. Out [13] prints the LUMO 0.4306983508491 and the value 0.0.

```
check(worst_e < 1e-14 and worst_homo < 1e-13,
      "E_KS and HOMO of all four couplings reproduced", record=f"{SUMMARY}, rows N8")
check(abs(lumo - float(ground["N8_lamp1_a00"]["LUMO"])) < 1e-12,
      "the LUMO of N8_lamp1_a00 reproduced", record=f"{SUMMARY}, column LUMO")
check(antisym < 1e-15, "E_KS(-lambda) = -E_KS(+lambda) for N = 8")
```

The three checks pass: the four energies and levels agree with the record, the LUMO too, and the energy is odd in λ, the exact symmetry derived in Section 15.5 (the text cell before the code cell repeats the derivation). The sum is exactly zero, not merely small, because the loop for $-\lambda$ performs exactly the arithmetic of the loop for $+\lambda$ with the signs of ε, v and b reversed, and a change of sign is exact in computer arithmetic.

In [14], the self-consistent potentials.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
labels = {"lamp1": "$+\\lambda_1$", "lamm1": "$-\\lambda_1$",
```

```
          "lamp2": "$+\\lambda_2$", "lamm2": "$-\\lambda_2$"}
for colour, (tag, label) in zip(PALETTE, labels.items()):
    xf = results[tag][2]
    style = "-" if tag.startswith("lamp") else "--"
    left.plot(Y, xf[:POINTS], style, color=colour, lw=1.8, label=label)
    right.plot(Y, xf[POINTS:], style, color=colour, lw=1.8, label=label)
```

Two drawing areas side by side, named `left` and `right`. For each coupling the converged potentials are drawn: $M - m$ on the left and v on the right, solid for positive couplings and dashed for negative ones (`tag.startswith("lamp")` is true for the tags of $+\lambda$).

```
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("$M(y) - m$ (units of $m$)")
left.set_title("Mass shift")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("$v(y)$ (units of $m$)")
right.set_title("Potential")
right.legend()
save_figure(fig, "n8_potentials",
            "The self-consistent mass shift $M(y) - m$ (left) and potential $v(y)$ "
...)
```

The axis labels and titles, and `save_figure` for Figure 15e.6. The student should see the potential $v = -\lambda n/16$ grow toward the tip roughly like e^{-4y}: the zero mode is e^{y}, so its coordinate density is e^{2y}, and dividing by the proper volume factor e^{6y} gives e^{-4y}. This growth toward the tip is why the interacting $N = 8$ results depend strongly on the cutoff L (Section 15.2, the paragraph on the cutoff). The mass shift is smaller, because the scalar density of the zero modes vanishes without interaction ($b = 0$) and is created only by the interaction itself.

```
plus, minus = results["lamp1"][2], results["lamm1"][2]  # potentials x of +-lambda_1
mirror = max(float(np.max(np.abs(plus[:POINTS] - minus[:POINTS]))),
             float(np.max(np.abs(plus[POINTS:] + minus[POINTS:]))))
report("largest violation of the potential symmetry", f"{mirror:.1e}")
check(mirror < 1e-12, "M - m is even and v is odd under lambda -> -lambda")
```

The symmetry of Section 15.5 predicts that $M - m$ is the same for $\pm\lambda$ and v changes sign. `mirror` is the larger of the largest $|(M-m)_+ - (M-m)_-|$ and the largest $|v_+ + v_-|$. Out [14] prints 0.0 and PASS; in the figure the dashed curves of the mass shift lie on the solid ones.

In [15], the last check.

```
NAMES = ["phase_function", "orbitals", "rk4_convergence", "brane_band", "scf_mixing",
         "n8_potentials"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15e_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

`NAMES` lists the six figure names in order. `enumerate(NAMES, start=1)` pairs each name with its number 1, 2, ...; the list comprehension with `if not ...` collects the names whose file does not

exist. The check requires that list to be empty, and `all_checks_passed()` prints the last line, ALL 18 CHECKS PASSED (notebook 15e).

15.10 The Rust solver and its canonical matrix

Notebook 15e showed the method on the smallest state. The Revision record computes every state of the canonical matrix with a compiled program, the **Rust solver** `revision_ks_solver`, whose source is the folder `Revision/kohn_sham/solver` of the repository. Rust is a programming language whose programs are translated once into machine code by its tool cargo and then run much faster than Python; the solver needs no packages from outside the repository, so cargo downloads nothing.

What is in the program. The source is split into modules, one file each in `Revision/kohn_sham/solver/src`:

- `theory.rs` reads the theory file `Revision/kohn_sham/ks-theory.json` and the gamma matrices `Revision/algebra/gammas.json` and checks again, in floating point, that the block basis of Chapter 14 really makes every operator of the problem block diagonal (check block_reduction_numeric);
- `shoot.rs` holds the shooting with the Pruefer angle and the safeguarded Newton root of Section 15.3;
- `scf.rs` holds the label sets, the aufbau, the densities and the self-consistent loop with Anderson mixing of Sections 15.4 and 15.5;
- `mermin.rs` finds the chemical potential at a temperature (Section 15.26);
- `analysis.rs` computes the energy-momentum tensor, its identities, the adiabaticity measure and the particle-hole lists;
- `spectrum.rs` computes the free spectra and their exact checks;
- `runs.rs` runs the whole canonical matrix;
- `json.rs`, `sha256.rs`, `report.rs` and `model.rs` write the results deterministically, compute fingerprints, collect the checks and hold the parameters.

The two ways to run it. The command `revision_ks_solver all` computes the whole canonical matrix of Section 15.2 and writes it into a folder of results. For each of the 75 ground states it solves the state itself, the same state at the four neighbouring slices $a_{4,0} \pm \delta$ and $a_{4,0} \pm 2\delta$ with $\delta = 0.002$ (for the derivatives along the history), the first excited state (Delta-SCF), the partner state of the rescaling identity (for the 60 states with $a_{4,0} > 0$), and the variant with the exact Fock exchange (for the 60 states with $\lambda \neq 0$); for each of the 135 thermal states it solves the state and its four neighbours at the temperatures $T \pm 0.01T$ and $T \pm 0.02T$ (for the temperature derivatives). It then evaluates its 42 checks, prints each as a line PASS - name or FAIL - name on its error stream, writes them into a report, and prints SUCCESS or FAILURE as the last line of its output stream. The command `revision_ks_solver single` solves one state with chosen parameters (m, λ, $a_{4,0}$, N and, if wanted, T) and writes its levels, energies and energy-momentum integrals as a JSON file and, if wanted, its profiles as a CSV file; Notebooks 15b and 15d use it.

The results. The folder `Revision/kohn_sham/results` holds the canonical matrix: `parameters.json` (all parameters and the calibration of Section 15.2), `manifest.json` (the list of every result file with its sha256 fingerprint, a 64-digit number computed from the bytes of a file, so that two files with the same fingerprint are identical), `spectrum/` (the free spectra), `ground/` (the 75 ground states: a summary table, the history of every self-consistent loop, the level tables, the profiles of the densities and the energy-momentum tensor at the 151 points $y = -3, -2.98, \ldots, 0$, and the integrals of the tensor), `excited/` (Delta-SCF and the particle-hole excitations), `adiabatic/` (Section 15.21), `rescaling/`, `exx/` (the exact-Fock variant) and `thermo/` (the 135 thermal states). Every number is written with 16 significant digits and no time stamp or path, so that a second run writes the same bytes.

How the record checks itself. Three reports belong to the solver. `Revision/kohn_sham/reports/ks-rust-solver.json` holds the 42 checks of the run itself, all PASS: the theory input, the free spectra (exact levels, zero mode, band slope, block symmetries, labels, tip angle, rescaling), every state (convergence, particle number, boundary conditions, the two energy forms, the energy-momentum identities, closed shells, completeness of the label sets), the derivatives along the history, the excited states, the exact-Fock variant, the thermodynamics and the chemical potential. `Revision/kohn_sham/reports/ks-rust-determinism.json` (14 checks, all PASS) compares a second run with the canonical one (all 244 files, the 243 result files and the manifest, byte-identical; check repeat_byte_identical) and a **refined run** with twice the RK4 steps and ten times smaller tolerances, against tolerances fixed before the comparison (10^{-8} for levels, energies and thermodynamics, 10^{-6} for profiles and derived quantities): for example E_{KS} agrees to 1.9×10^{-12} relative (check refined_ground_energies) and 23724 levels agree label by label to $2.1 \times 10^{-9}\, m$ (check refined_eigenvalues). `Revision/kohn_sham/reports/ks-rust-mermin-roots.json` compares every chemical potential with a 40-digit root (Section 15.26). An independent cross-check by a second program written in Python is the subject of Chapter 16.

Threads. The solver distributes independent states over the processor cores (at most 22 threads) and merges the results in a fixed order, so the number of threads does not change any output byte. The record's run of the canonical matrix took 78.1 s on an idle machine with 22 threads (`Revision/kohn_sham/solver/README.md`, Timings); the whole Notebook 15a took 164 s and 232 s in its two recorded runs on the machine that built the book, while other programs were running (section 4.5 of `Revision/textbook/notebooks/15a_canonical_matrix.PROVENANCE.md`).

15.11 Example: running the canonical matrix (Notebook 15a)

Notebook 15a builds the solver with cargo on the student's computer, runs `revision_ks_solver all` into a folder that git ignores, and checks the new run against the committed record in three ways: the solver's own 42 checks must all pass and match the record's list; the new run must write the same 243 result files (and on the computer that built the book they are byte-identical); and the key numbers must agree within the tolerances that the record fixed in advance. Then it reads the new results and draws eight figures: the levels of $N = 136$ along the history (Figure 15a.1), the gaps (Figure 15a.2), the orbital relaxation of Delta-SCF (Figure 15a.3), the total energies and the interaction shifts (Figure 15a.4), the densities (Figure 15a.5), the self-consistent potentials (Figure 15a.6), the convergence of the self-consistent loop (Figure 15a.7) and the particle-hole excitations (Figure 15a.8). It needs Rust (the run instructions below explain how to install it), takes two to four minutes on a computer with many cores and up to about ten minutes on a laptop, and ends with ALL 25 CHECKS PASSED (notebook 15a).

What to look for: in Figure 15a.1 the blue brane-band levels that fall from slice to slice while the black levels at $k = 0$ stay flat; in Figure 15a.2 the gaps that fall almost, but not quite, like the dashed lines $e^{-a_{4,0}}$; in Figure 15a.5 that the particles sit near the brane although the proper density is largest at the tip.

15.12 How to run Notebook 15a

Step 1. What this notebook does and what it needs.

Notebook 15a (Running the Rust Kohn-Sham solver over the canonical matrix) is the file `Revision/textbook/notebooks/15a_canonical_matrix.ipynb` of the repository Dirac_claude. It builds the Rust Kohn-Sham solver of the repository with cargo (a full build of about a minute when the program is missing, a second when it is up to date), runs it over the whole canonical matrix (75 ground states and 135 thermal states of the Kohn-Sham gas of dirac16complex along the deflating history), checks that the new results agree with the committed Revision record within tolerances fixed in advance, and draws the Kohn-Sham levels, the gaps, the energies, the densities, the self-consistent potentials, the convergence of the self-consistent loop and the particle-hole excitations. The solver writes its raw output (about 6 MB) into the folder `Revision/kohn_sham/solver/target/textbook_15a`, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and

accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/kohn_sham/solver/target/release/revision_ks_solver` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15a_canonical_matrix.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15a_canonical_matrix.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 15a_canonical_matrix.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/15a.captions.json`
- `Revision/textbook/figures/15a_1_levels_history.png`
- `Revision/textbook/figures/15a_2_gaps_history.png`
- `Revision/textbook/figures/15a_3_delta_scf.png`
- `Revision/textbook/figures/15a_4_energy_history.png`
- `Revision/textbook/figures/15a_5_densities.png`
- `Revision/textbook/figures/15a_6_potentials.png`
- `Revision/textbook/figures/15a_7_scf_convergence.png`
- `Revision/textbook/figures/15a_8_particle_hole.png`

It changes no other file of the repository except the Rust build folder `target` next to each `Cargo.toml` it builds (inside it the notebook keeps the raw output of its program runs in `Revision/kohn_sham/solver/target/textbook_15a`, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 25 CHECKS PASSED (notebook 15a)
```

and the notebook must show 8 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/15a_canonical_matrix.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 15a_canonical_matrix.ipynb
python -m nbconvert --execute --inplace 15a_canonical_matrix.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.

- The cell that runs the canonical matrix shows the label with the star for several minutes: this is normal. The solver computes 210 states and uses up to 22 processor cores; on the computer that built the notebook (22 cores) the whole notebook takes two to four minutes, and on a computer with 4 cores it can need up to about 10 minutes. Wait until the label shows a number.
- An AssertionError names one of the comparisons with the record: the notebook prints the largest difference just above the error. Differences far below the tolerance are rounding effects of another computer; a larger difference means that the solver or its input files were changed. Get the stored versions back and run the notebook again.

```
git checkout -- Revision/kohn_sham
```

- You want the disk space of the solver output back: the folder `Revision/kohn_sham/solver/target/textbook_15a` holds only the raw output of the last run (ignored by git); delete it at any time, the notebook writes it again.

15.13 Notebook 15a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/15a_canonical_matrix.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This notebook runs the **Rust Kohn-Sham solver** of the repository over its whole
**canonical matrix**: the Kohn-Sham states of the fermion field dirac16complex in
the author's primordial gravitational field, for three particle numbers, five
couplings and five instants of the deflating history (75 ground states), and for
135 states at three temperatures. Then it

- checks that the solver reports every one of its own checks as PASS (the
  committed record holds 42 of them);
- compares every new result with the committed Revision record (the folder
  Revision/kohn_sham/results), number by number, with tolerances that were fixed in
  advance;
- prints the energies and gaps of the ground states along the history;
- draws eight figures: the Kohn-Sham levels along the history, the gaps, the
  Delta-SCF excitation energies, the total energies, the densities, the
  self-consistent potentials, the convergence of the self-consistent loop and the
  particle-hole excitations.

The run takes two to four minutes on a computer with many processor cores
(up to about ten minutes on a laptop). Everything it writes outside the folder of
```

the figures goes into the Rust build folder
‘Revision/kohn_sham/solver/target/textbook_15a‘, which git ignores.

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 15a (Running the Rust Kohn-Sham solver over the canonical matrix) is the file
‘Revision/textbook/notebooks/15a_canonical_matrix.ipynb‘ of the repository Dirac_claude.
It builds the Rust Kohn-Sham solver of the repository with cargo (a full build of about a
minute when the program is missing, a second when it is up to date), runs it over the
whole canonical matrix (75 ground states and 135 thermal states of the Kohn-Sham gas of
dirac16complex along the deflating history), checks that the new results agree with the
committed Revision record within tolerances fixed in advance, and draws the Kohn-Sham
levels, the gaps, the energies, the densities, the self-consistent potentials, the
convergence of the self-consistent loop and the particle-hole excitations. The solver
writes its raw output (about 6 MB) into the folder
‘Revision/kohn_sham/solver/target/textbook_15a‘, which git ignores. It needs a computer
with Windows 11, macOS or Linux, an internet connection for the installation, the program
Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages
run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program
cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver,
which is part of the repository and is built on your computer.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):
```

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘ again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/kohn_sham/solver/target/release/revision_ks_solver‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15a_canonical_matrix.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15a_canonical_matrix.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp]

Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 15a_canonical_matrix.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/15a.captions.json‘
- ‘Revision/textbook/figures/15a_1_levels_history.png‘
- ‘Revision/textbook/figures/15a_2_gaps_history.png‘
- ‘Revision/textbook/figures/15a_3_delta_scf.png‘
- ‘Revision/textbook/figures/15a_4_energy_history.png‘
- ‘Revision/textbook/figures/15a_5_densities.png‘
- ‘Revision/textbook/figures/15a_6_potentials.png‘
- ‘Revision/textbook/figures/15a_7_scf_convergence.png‘
- ‘Revision/textbook/figures/15a_8_particle_hole.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds (inside it the notebook keeps the raw output of its program runs in ‘Revision/kohn_sham/solver/target/textbook_15a‘, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 25 CHECKS PASSED (notebook 15a)
```

and the notebook must show 8 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/15a_canonical_matrix.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was

  downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 15a_canonical_matrix.ipynb
python -m nbconvert --execute --inplace 15a_canonical_matrix.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- The cell that runs the canonical matrix shows the label with the star for several minutes: this is normal. The solver computes 210 states and uses up to 22 processor cores; on the computer that built the notebook (22 cores) the whole notebook takes two to four minutes, and on a computer with 4 cores it can need up to about 10 minutes. Wait until the label shows a number.
- An AssertionError names one of the comparisons with the record: the notebook prints the largest difference just above the error. Differences far below the tolerance are rounding effects of another computer; a larger difference means that the solver or its input files were changed. Get the stored versions back and run the notebook again.

```
git checkout -- Revision/kohn_sham
```

- You want the disk space of the solver output back: the folder ‘Revision/kohn_sham/solver/target/textbook_15a‘ holds only the raw output of the last run (ignored by git); delete it at any time, the notebook writes it again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 15a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 15a (Running the Rust Kohn-Sham solver over the canonical matrix) is the file
# Revision/textbook/notebooks/15a_canonical_matrix.ipynb of the repository Dirac_claude.
# It builds the Rust Kohn-Sham solver of the repository with cargo (a full build of about
# a minute when the program is missing, a second when it is up to date), runs it over the
# whole canonical matrix (75 ground states and 135 thermal states of the Kohn-Sham gas of
# dirac16complex along the deflating history), checks that the new results agree with the
# committed Revision record within tolerances fixed in advance, and draws the Kohn-Sham
# levels, the gaps, the energies, the densities, the self-consistent potentials, the
# convergence of the self-consistent loop and the particle-hole excitations. The solver
# writes its raw output (about 6 MB) into the folder
# Revision/kohn_sham/solver/target/textbook_15a, which git ignores. It needs a computer
# with Windows 11, macOS or Linux, an internet connection for the installation, the
# program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It also needs Rust
# (the program cargo, version 1.91.1 or newer), because it runs the Rust program
# revision_ks_solver, which is part of the repository and is built on your computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
```

```
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

```
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
#
# Windows, macOS and Linux:
#     cargo --version
#
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
#     cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
#
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
```

```
# Revision/kohn_sham/solver/target/release/revision_ks_solver (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 15a_canonical_matrix.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 15a_canonical_matrix.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 15a_canonical_matrix.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/15a.captions.json
# - Revision/textbook/figures/15a_1_levels_history.png
# - Revision/textbook/figures/15a_2_gaps_history.png
# - Revision/textbook/figures/15a_3_delta_scf.png
# - Revision/textbook/figures/15a_4_energy_history.png
# - Revision/textbook/figures/15a_5_densities.png
# - Revision/textbook/figures/15a_6_potentials.png
# - Revision/textbook/figures/15a_7_scf_convergence.png
# - Revision/textbook/figures/15a_8_particle_hole.png
#
```

```
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds (inside it the notebook keeps the raw output of its program
# runs in Revision/kohn_sham/solver/target/textbook_15a, which git ignores); running it
# headless or saving it in JupyterLab also rewrites the notebook file itself. It does not
# use the internet while it runs. The files it writes are the same files that are stored
# in the repository (on another computer a figure may differ in a few bytes, which is
# harmless). To get the stored versions back, run this command in the repository folder
# (it also undoes every change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 25 CHECKS PASSED (notebook 15a)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/15a_canonical_matrix.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 15a_canonical_matrix.ipynb
#     python -m nbconvert --execute --inplace 15a_canonical_matrix.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
```

```
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - The cell that runs the canonical matrix shows the label with the star for several
#   minutes: this is normal. The solver computes 210 states and uses up to 22 processor
#   cores; on the computer that built the notebook (22 cores) the whole notebook takes
#   two to four minutes, and on a computer with 4 cores it can need up to about 10
#   minutes. Wait until the label shows a number.
# - An AssertionError names one of the comparisons with the record: the notebook prints
#   the largest difference just above the error. Differences far below the tolerance are
#   rounding effects of another computer; a larger difference means that the solver or
#   its input files were changed. Get the stored versions back and run the notebook
#   again.
#     git checkout -- Revision/kohn_sham
# - You want the disk space of the solver output back: the folder
#   Revision/kohn_sham/solver/target/textbook_15a holds only the raw output of the last
#   run (ignored by git); delete it at any time, the notebook writes it again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "15a"  # this notebook: chapter 15, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
```

```
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
```

```
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
```

```
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 15a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Kohn-Sham state**: an approximate state of many identical fermions, built from
  one-particle wave functions (*orbitals*) that each solve a one-particle equation
  in a common *effective potential*; the potential depends on the densities of the
  occupied orbitals, so the equations must be solved *self-consistently*.
- **Level**: an allowed energy $\varepsilon$ of the one-particle equation. A level is
  **occupied** if particles sit in it ($f = 1$) and **empty** if not ($f = 0$).
- **Degeneracy** $g$: the number of different orbitals that share one level.
- **HOMO, LUMO, Kohn-Sham gap**: the highest occupied and the lowest unoccupied
  level, and their difference $\Delta_{KS} = \varepsilon_{LUMO} - \varepsilon_{HOMO}$.
- **Delta-SCF**: the energy of the first excited state minus the energy of the
  ground state, both computed self-consistently (SCF = self-consistent field).
- **Slice** $a_{4,0}$: one instant of the history, the value of the metric function
  $a_4$ at that instant. The **history** is $a_4 = A H x_4$ with $A = 1$.
- **Canonical matrix**: the fixed list of states the solver computes: particle
  numbers $N = 8, 136, 688$; couplings $\lambda = 0, \pm\lambda_1, \pm\lambda_2$;
  slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$; and temperatures $T = 0.01, 0.02, 0.05$
  (in units of the mass $m$) for $\lambda = 0, \pm\lambda_1$.
- **Proper density**: a number of particles (or an energy) per unit of proper
  7-volume. **Coordinate density**: the same per unit of the coordinate $y$ (it
  contains the volume factor $e^{6Hy}$).
- **Brane** and **tip**: the two ends $y = 0$ and $y = -L = -3$ of the hidden
  coordinate.
- **Shell** $n_2$: the 3-space momenta form a lattice $\vec k = \Delta k\,\vec n$
  with $\vec n$ a vector of three whole numbers; a shell collects the $r_3(n_2)$
  vectors with the same $n_2 = |\vec n|^2$, so $|\vec k| = \Delta k \sqrt{n_2}$.
- **Residual**: in the self-consistent loop, the largest change of the potential
  from one iteration to the next; the loop stops when it is below $10^{-11}$.
- **Anderson mixing**: a rule that builds the next potential from several earlier
  ones; it converges much faster than simple mixing.
- **Manifest, sha256**: the solver writes a list (the manifest) of all its files
  with a 64-digit *fingerprint* (sha256) of each; two files with the same
  fingerprint are identical byte for byte.
- **Tolerance**: the largest difference that a comparison accepts; the tolerances
  used here were fixed in advance in the Revision record.
```

Text cell:

```
## 4. The physical and mathematical situation

**The field and the metric.** The author's metric has eight coordinates $x_0$ to
$x_7$. The directions $x_1, x_2, x_3$ are ordinary 3-space, with the scale factor
$e^{a_4} \sin^{1/6} z$; $x_4$ is the time; $x_5, x_6, x_7$ are the three **extra
times**, with the scale factor $e^{-a_4} \sin^{1/6} z$, so they **deflate
exponentially** while $a_4$ grows; $x_0$ is the hidden space direction, with
$z = 6 H x_0$ between $0$ and $\pi/2$. The Kohn-Sham model uses the hidden
coordinate $y = \ln(\sin z)/(6H)$, which runs from the tip $y = -L$ (the model cuts
the space there, with $L = 3$) to the brane $y = 0$ (the end $z = \pi/2$ of the
patch).

**The equation the solver solves.** For one 3-momentum $k = |\vec k|$ and one of
two block types $j = \pm 1$ the 16-component field reduces exactly to a pair of
real functions $a(y), b(y)$ (an orbital) with

$$a' = M a - (\kappa k + j(\varepsilon - v))\, b, \qquad
b' = (j(\varepsilon - v) - \kappa k)\, a - M b,$$

where the prime is $d/dy$, $\kappa = e^{-Hy - a_{4,0}}$, $M(y) = m + \tfrac{15}{16}
\lambda S(y)$ is the effective mass and $v(y) = -\tfrac{1}{16} \lambda n(y)$ the
effective potential, with $n$ and $S$ the proper number and scalar densities of
the occupied orbitals. The slice $a_{4,0}$ enters only through $\kappa$: along the
history every 3-momentum is **redshifted**, $k \to k e^{-a_{4,0}}$.

**Boundary conditions.** At the tip $b(-L) = 0$ (a regular tip, a choice). At the
brane the Z2 mirror, which is ASSUMED: $b(0) = 0$ (even parity) or $a(0) = 0$ (odd
parity).

**Status of the history.** The history $a_4 = A H x_4$ with $A = 1$ is a PRESCRIBED
BACKGROUND: it is not solved for, and the Kohn-Sham states are not an admissible
source of the equations for $a_4$ (the Revision record
Revision/field_equations_a4/reports/ks-source-conditions.json). The states are
**instantaneous** (adiabatic) states at each slice.

**Parameters** (units $H = m = 1$): $L = 3$, $\Delta k = 0.25$, $N = 8$ (the eight
zero modes at $k = 0$), $N = 136$ and $N = 688$ (closed shells of the brane band),
and per $N$ the couplings $\lambda_1, \lambda_2$ that make the first-order
potential $0.1\,m$ and $0.3\,m$ where it is largest, over every point and every
slice, so that it stays below these values along the whole history
(Revision/kohn_sham/results/parameters.json).
Particles fill the positive branch and the zero modes, a CONVENTION whose
justification is OPEN. $N$ counts the particles of the doubled system (universe and
Z2 image); one patch holds $N/2$.

All numbers of this notebook are COMPUTED by the solver; the notebook checks that
they agree with the committed record.
```

Text cell:

```
## 5. Building the solver
```

The next cell imports the packages of this notebook and builds the Rust program ‘revision_ks_solver‘ with cargo (the helper ‘rust_program‘ of the set-up cell runs ‘cargo build --release‘ for the crate; the first build takes about a minute, later builds a second). ‘PALETTE‘ holds the colours of the figures: eight distinct hues in a fixed order, and five shades of blue for the five slices of the history (light for the earliest, dark for the latest).

In [2]:

```
import csv  # reads the tables (CSV files) that the solver writes
import math  # exp, sqrt and pi for single numbers

import numpy as np  # arrays of numbers

SOLVER_MANIFEST = "Revision/kohn_sham/solver/Cargo.toml"  # the crate of the solver
program = rust_program(SOLVER_MANIFEST, "revision_ks_solver")  # build it, get its path

PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # blue, orange, aqua, yellow, magenta, green, ...
SLICE_SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light->dark
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]  # the five slices a4,0 of the history
say("Packages imported; colours defined.")
```

Out [2]:

```
Rust program revision_ks_solver is built and ready.
Packages imported; colours defined.
```

Text cell:

```
## 6. Running the canonical matrix

The next cell runs ‘revision_ks_solver all‘. The option ‘--root‘ tells the program where the repository is (it reads the theory file ‘Revision/kohn_sham/ks-theory.json‘, the gamma matrices ‘Revision/algebra/gammas.json‘ and one fixture file from there); ‘--out‘ and ‘--report‘ name the folder of the results and the file of the check report. Both go into the folder ‘textbook_15a‘ inside the Rust build folder ‘target‘, so the committed record is never touched. A folder left over from an interrupted run is removed first (the solver refuses to write into a folder without its manifest). The solver prints one line per check on its error stream and ‘SUCCESS‘ as its last line on its output stream; the cell counts those lines. This cell takes two to three minutes on a computer with many cores (up to about ten minutes on a laptop).
```

In [3]:

```
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_15a"  # git ignores it
NEW_RESULTS = RUN_FOLDER / "results"  # the new canonical matrix
```

```
NEW_REPORT = RUN_FOLDER / "ks-rust-solver.json"  # the new check report
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
if NEW_RESULTS.exists() and not (NEW_RESULTS / "manifest.json").exists():
    shutil.rmtree(NEW_RESULTS)  # remains of an interrupted run
completed = subprocess.run(
    [str(program), "all", "--root", str(REPO), "--out", str(NEW_RESULTS),
     "--report", str(NEW_REPORT)],
    capture_output=True, text=True, encoding="utf-8", errors="replace")
stdout_lines = completed.stdout.strip().split("\n")  # what it printed on stdout
stderr_lines = completed.stderr.split("\n")  # its check lines and timings
pass_count = sum(1 for line in stderr_lines if line.startswith("PASS - "))
fail_count = sum(1 for line in stderr_lines if line.startswith("FAIL - "))
say(f"exit code {completed.returncode}, last line {stdout_lines[-1]!r}, "
    f"{pass_count} PASS lines, {fail_count} FAIL lines")
check(completed.returncode == 0 and stdout_lines[-1] == "SUCCESS" and fail_count == 0,
      "the solver ran the whole canonical matrix and reported SUCCESS")
```

Out [3]:

```
exit code 0, last line 'SUCCESS', 42 PASS lines, 0 FAIL lines
PASS the solver ran the whole canonical matrix and reported SUCCESS
```

Text cell:

```
## 7. The solver's own check report

The solver writes its checks into a JSON report. The next cell reads the new report
and the committed one, prints how many checks of each group there are (the group
is the first word of the check name, for example `ground` or `thermo`), and checks
that every check passes, that the new report has the same summary (number of
checks, of PASS and of FAIL) as the committed report, and that it lists the same
checks in the same order. Whether the two reports are identical byte for byte is
printed as a RESULT line: on the computer that built this book they are; on another
computer the last digits of some reported deviations may differ.
```

In [4]:

```
RECORD_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
new_report = json.loads(NEW_REPORT.read_text(encoding="utf-8"))
old_report = json.loads(repository_file(RECORD_REPORT).read_text(encoding="utf-8"))
groups = {}  # first word of a check name -> [number of checks, number of PASS]
for item in new_report["checks"]:
    group = item["name"].split("_")[0]
    groups.setdefault(group, [0, 0])
    groups[group][0] += 1
    groups[group][1] += item["verdict"] == "PASS"  # True counts as 1
for group, (count, passed) in groups.items():
    say(f"  {group:10} {count:2d} checks, {passed:2d} PASS")
new_names = [item["name"] for item in new_report["checks"]]
old_names = [item["name"] for item in old_report["checks"]]
summary = new_report["summary"]  # {"checks": ..., "pass": ..., "fail": ...}
```

```
report("checks in the new report / PASS / FAIL",
       f"{summary['checks']} / {summary['pass']} / {summary['fail']}")
check(summary == old_report["summary"] and summary["fail"] == 0
      and summary["pass"] == summary["checks"] == len(new_names),
      f"the new report holds the {summary['checks']} checks of the record, all PASS",
      record=f"{RECORD_REPORT}, summary")
check(new_names == old_names, "the new report has the same checks as the record")
report("new report byte-identical to the record",
       NEW_REPORT.read_bytes() == repository_file(RECORD_REPORT).read_bytes())
```

Out [4]:

```
  theory      1 checks,  1 PASS
  gamma       1 checks,  1 PASS
  block       1 checks,  1 PASS
  free        7 checks,  7 PASS
  adiabatic   2 checks,  2 PASS
  thermo     11 checks, 11 PASS
  t3          1 checks,  1 PASS
  emt         5 checks,  5 PASS
  excited     2 checks,  2 PASS
  exx         2 checks,  2 PASS
  ground      7 checks,  7 PASS
  rescaling   2 checks,  2 PASS
RESULT checks in the new report / PASS / FAIL = 42 / 42 / 0
PASS the new report holds the 42 checks of the record, all PASS
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, summary
PASS the new report has the same checks as the record
RESULT new report byte-identical to the record = True
```

Text cell:

```
## 8. Comparing every result file with the record

The solver's manifest lists every result file with its sha256 fingerprint. The next
cell checks that the new run wrote exactly the same set of files as the record and
counts how many have the same fingerprint (byte-identical files).
```

In [5]:

```
RECORD_RESULTS = "Revision/kohn_sham/results"


def manifest_of(folder):
    """{file path: sha256} of the manifest.json in folder."""
    data = json.loads((folder / "manifest.json").read_text(encoding="utf-8"))
    return {entry["path"]: entry["sha256"] for entry in data["files"]}


old_manifest = manifest_of(repository_file(RECORD_RESULTS))
new_manifest = manifest_of(NEW_RESULTS)
```

```
check(sorted(old_manifest) == sorted(new_manifest),
      f"the new run wrote the same {len(old_manifest)} result files as the record",
      record=f"{RECORD_RESULTS}/manifest.json")
same = sum(1 for path in old_manifest if new_manifest[path] == old_manifest[path])
report("result files byte-identical to the record", f"{same} of {len(old_manifest)}")
```

Out [5]:

```
PASS the new run wrote the same 243 result files as the record
     reproduces Revision/kohn_sham/results/manifest.json
RESULT result files byte-identical to the record = 243 of 243
```

Text cell:

```
Byte identity can fail on another computer only through the last digit of a number
(operating systems compute $e^x$ and $\ln x$ with slightly different rounding). The
meaningful comparison is therefore numerical. The next cell reads four tables of the
new run and of the record and computes the largest difference of their key columns:
the energies $E_{KS}$ of the 75 ground states (relative to $\max(|E|, 1)$), their
HOMO, LUMO and gap (absolute, in units of $m$), the Delta-SCF energies, the
chemical potential $\mu$, energy, entropy and free energy of the 135 thermal states,
and the adiabaticity measure $Q_{max}$. The tolerances are those fixed in advance in
Revision/kohn_sham/reports/ks-rust-determinism.json: $10^{-8}$ for eigenvalues,
energies and thermodynamics, $10^{-6}$ for derived quantities.
```

In [6]:

```
def read_table(folder, name):
    """The rows of the CSV table folder/name as a dictionary id -> row."""
    with open(folder / name, newline="", encoding="utf-8") as handle:
        return {row["id"]: row for row in csv.DictReader(handle)}


def worst(table, columns, relative):
    """Largest difference between the new run and the record in these columns."""
    new = read_table(NEW_RESULTS, table)
    old = read_table(repository_file(RECORD_RESULTS), table)
    assert sorted(new) == sorted(old), f"{table}: the state lists differ"
    largest = 0.0
    for key in old:
        for column in columns:
            a, b = float(new[key][column]), float(old[key][column])
            scale = max(abs(b), 1.0) if relative else 1.0
            largest = max(largest, abs(a - b) / scale)
    return largest


COMPARISONS = [  # (table, columns, relative?, tolerance, what)
    ("ground/summary.csv", ["E_KS"], True, 1e-8, "ground-state energies"),
    ("ground/summary.csv", ["HOMO", "LUMO", "KS_gap"], False, 1e-8,
     "HOMO, LUMO and gaps"),
```

```
    ("excited/summary.csv", ["delta_SCF"], False, 1e-8, "Delta-SCF energies"),
    ("thermo/thermodynamics.csv", ["mu", "E", "entropy", "F"], True, 1e-8,
     "thermodynamics"),
    ("adiabatic/adiabaticity.csv", ["Q_max"], True, 1e-6, "adiabaticity Q_max"),
]
for table, columns, relative, tolerance, what in COMPARISONS:
    largest = worst(table, columns, relative)
    report(f"largest difference, {what}", f"{largest:.1e}")
    check(largest <= tolerance, f"{what} agree with the record within {tolerance:.0e}",
          record=f"{RECORD_RESULTS}/{table}")
```

Out [6]:

```
RESULT largest difference, ground-state energies = 0.0e+00
PASS ground-state energies agree with the record within 1e-08
     reproduces Revision/kohn_sham/results/ground/summary.csv
RESULT largest difference, HOMO, LUMO and gaps = 0.0e+00
PASS HOMO, LUMO and gaps agree with the record within 1e-08
     reproduces Revision/kohn_sham/results/ground/summary.csv
RESULT largest difference, Delta-SCF energies = 0.0e+00
PASS Delta-SCF energies agree with the record within 1e-08
     reproduces Revision/kohn_sham/results/excited/summary.csv
RESULT largest difference, thermodynamics = 0.0e+00
PASS thermodynamics agree with the record within 1e-08
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv
RESULT largest difference, adiabaticity Q_max = 0.0e+00
PASS adiabaticity Q_max agree with the record within 1e-06
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv
```

Text cell:

```
## 9. The ground states along the history

From here on the notebook reads the NEW results (which equal the record). The next
cell prints, for the three particle numbers without interaction ($\lambda = 0$),
the energy $E_{KS}$, the HOMO, the LUMO and the gap at the five slices.
```

In [7]:

```
ground = read_table(NEW_RESULTS, "ground/summary.csv")  # the 75 ground states
excited = read_table(NEW_RESULTS, "excited/summary.csv")  # their Delta-SCF


def state_id(n, tag, a4):
    """The solver's name of a state, e.g. N136_lam0_a10 for N = 136, a4,0 = 1."""
    return f"N{n}_{tag}_a{round(10 * a4):02d}"


say("   N  a4,0        E_KS        HOMO        LUMO     KS gap")
for n in (8, 136, 688):
    for a4 in SLICES:
```

```
        row = ground[state_id(n, "lam0", a4)]
        say(f"{n:4d}  {a4:4.1f}  {float(row['E_KS']):10.5f}  {float(row['HOMO']):10.6f}"
            f"  {float(row['LUMO']):10.6f}  {float(row['KS_gap']):9.6f}")
report("KS gap of N = 8 at a4,0 = 0, 1, 2", ", ".join(
    f"{float(ground[state_id(8, 'lam0', a)]['KS_gap']):.7f}" for a in (0, 1, 2)))
```

Out [7]:

```
   N  a4,0        E_KS        HOMO        LUMO     KS gap
   8   0.0     0.00000    0.000000    0.430734   0.430734
   8   0.5     0.00000    0.000000    0.272645   0.272645
   8   1.0     0.00000    0.000000    0.170349   0.170349
   8   1.5     0.00000    0.000000    0.105015   0.105015
   8   2.0     0.00000    0.000000    0.064159   0.064159
 136   0.0    80.28222    0.799646    0.882320   0.082674
 136   0.5    51.24519    0.512555    0.566393   0.053838
 136   1.0    32.38412    0.325768    0.360757   0.034989
 136   1.5    20.20308    0.204636    0.227280   0.022644
 136   2.0    12.44507    0.126766    0.141223   0.014457
 688   0.0   680.44125    1.247113    1.292293   0.045180
 688   0.5   437.51291    0.803752    0.835207   0.031455
 688   1.0   279.42489    0.515229    0.535712   0.020484
 688   1.5   176.73282    0.327505    0.340814   0.013310
 688   2.0   110.38667    0.205760    0.214370   0.008610
RESULT KS gap of N = 8 at a4,0 = 0, 1, 2 = 0.4307337, 0.1703493, 0.0641594
```

Text cell:

```
## 10. The Kohn-Sham levels along the history

The next cell reads, for $N = 136$ and $\lambda = 0$, the level tables of the five
slices. Each level has a **key**: its shell $n_2$, block type $j$, parity and
Pruefer label $l$ (the label numbers the levels of one sector in the order of their
energies, and a level keeps its label when the potentials change continuously, so
the label identifies the same level at every slice). The figure follows each key across
the slices: the **brane band** (the lowest even level of $j = +1$ in each shell
$n_2 \ge 1$) comes down as the history proceeds, while the levels at $k = 0$ do not
move at all, because $a_{4,0}$ enters only through $\kappa k$. The dashed line is
the small-$k$ law $\varepsilon \approx c\, k\, e^{-a_{4,0}}$ for the first shell
($k = 0.25$), with the slope $c = 1.9051482536$ of Revision/kohn_sham/ks-theory.json.

The checks: every $k = 0$ level is the same at all five slices (to $10^{-12}$), and
every brane-band level present at all slices decreases strictly along the history.
```

In [8]:

```
def read_levels(n, tag, a4):
    """{key: (eps, f)} of one state; key = (n2, j, parity, label)."""
    path = NEW_RESULTS / "ground/levels" / f"{state_id(n, tag, a4)}.csv"
    with open(path, newline="", encoding="utf-8") as handle:
        return {(int(r["n2"]), int(r["j"]), r["parity"], int(r["label"])):
```

```
                (float(r["eps"]), float(r["f"])) for r in csv.DictReader(handle)}


levels = [read_levels(136, "lam0", a4) for a4 in SLICES]  # one dictionary per slice
THEORY = json.loads(repository_file("Revision/kohn_sham/ks-theory.json").read_text(
    encoding="utf-8"))  # the theory file of the solver
C_SLOPE = float(THEORY["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])  # c
fig, ax = plt.subplots(figsize=(7.5, 5.0))
all_keys = sorted(set().union(*levels))
for key in all_keys:
    points = [(a4, lv[key]) for a4, lv in zip(SLICES, levels) if key in lv]
    xs = [p[0] for p in points]
    es = [p[1][0] for p in points]
    if min(es) > 1.6:
        continue  # only the part of the spectrum near the Fermi level
    band = key[1] == 1 and key[2] == "even" and key[3] == 0 and key[0] >= 1
    colour = PALETTE[0] if band else ("black" if key[0] == 0 else "0.6")
    ax.plot(xs, es, "-", color=colour, lw=1.0, alpha=0.8)
    for x, (e, f) in points:
        ax.plot([x], [e], "o", ms=5, color=colour,
                markerfacecolor=colour if f > 0.5 else "white")
a_fine = np.linspace(0.0, 2.0, 101)
ax.plot(a_fine, C_SLOPE * 0.25 * np.exp(-a_fine), "--", color=PALETTE[1], lw=1.5,
        label="$c\\,k\\,e^{-a_{4,0}}$ for $k = 0.25$")
ax.plot([], [], "o-", color=PALETTE[0], label="brane band ($j=+1$, even, $l=0$)")
ax.plot([], [], "o-", color="black", label="levels at $k = 0$")
ax.plot([], [], "o-", color="0.6", label="other levels")
ax.plot([], [], "o", color="0.3", markerfacecolor="white", label="open: empty")
ax.set_ylim(-0.05, 1.6)
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("level $\\varepsilon$ (units of $m$)")
ax.set_title("Kohn-Sham levels of $N = 136$, $\\lambda = 0$, along the history")
ax.legend(fontsize=8, loc="center right")
save_figure(fig, "levels_history",
            "Kohn-Sham levels $\\varepsilon$ (vertical axis, units of $m$) of the "
            "state $N = 136$, $\\lambda = 0$ at the five slices $a_{4,0} = 0$ to $2$ "
            "(horizontal axis), each level followed by its key; filled dots are "
            "occupied, open dots empty. The brane band (blue) comes down as the "
            "3-momenta redshift like $k e^{-a_{4,0}}$, the dashed curve is the "
            "small-$k$ law for the first shell, and the levels at $k = 0$ (black) do "
            "not move.")
k0_keys = [key for key in levels[0] if key[0] == 0]
k0_spread = max(abs(lv[key][0] - levels[0][key][0]) for lv in levels for key in k0_keys)
band_keys = [key for key in all_keys if all(key in lv for lv in levels)
             and key[1] == 1 and key[2] == "even" and key[3] == 0 and key[0] >= 1]
decreasing = all(levels[i + 1][key][0] < levels[i][key][0]
                 for key in band_keys for i in range(4))
report("number of brane-band levels followed through all five slices", len(band_keys))
check(k0_spread <= 1e-12, "the k = 0 levels are the same at every slice")
check(decreasing, "every brane-band level decreases strictly along the history")
```

Out [8]:

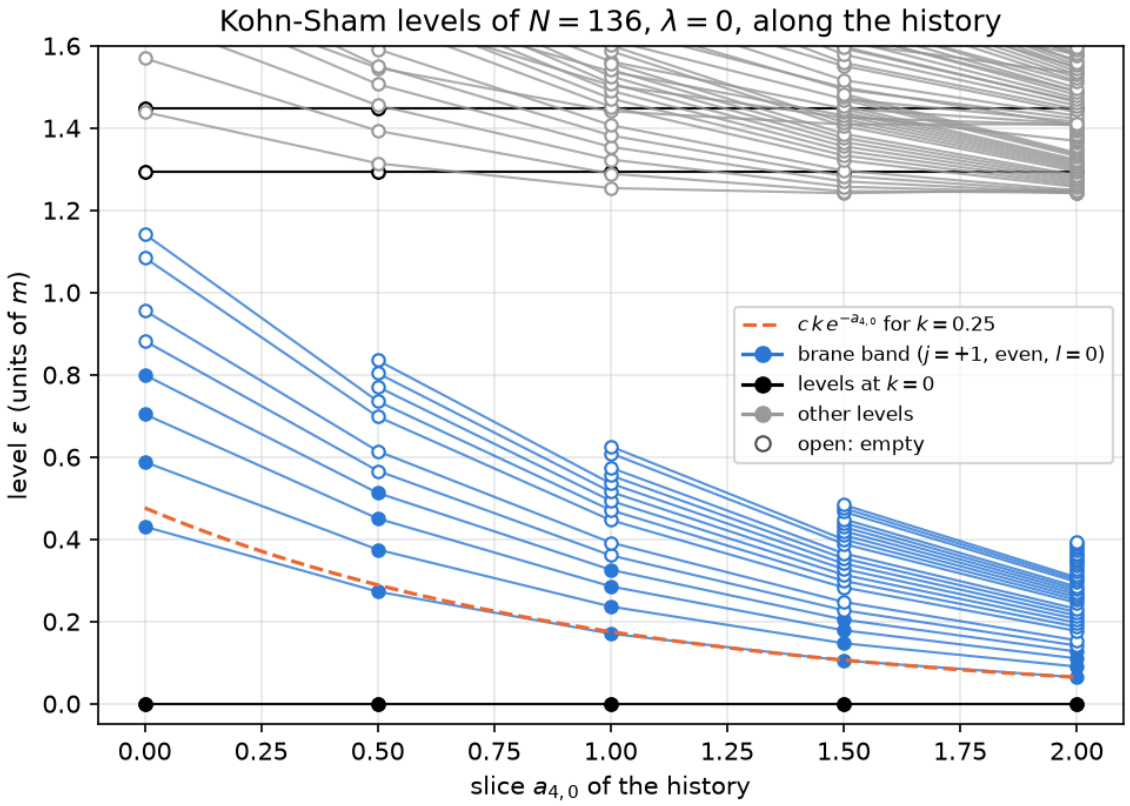

Figure 411: Kohn-Sham levels ε (vertical axis, units of m) of the state $N = 136$, $\lambda = 0$ at the five slices $a_{4,0} = 0$ to 2 (horizontal axis), each level followed by its key; filled dots are occupied, open dots empty. The brane band (blue) comes down as the 3-momenta redshift like $ke^{-a_{4,0}}$, the dashed curve is the small-k law for the first shell, and the levels at $k = 0$ (black) do not move. (Notebook 15a, figure 1.)

```
Figure 15a.1 saved as Revision/textbook/figures/15a_1_levels_history.png
RESULT number of brane-band levels followed through all five slices = 8
PASS the k = 0 levels are the same at every slice
PASS every brane-band level decreases strictly along the history
```

Text cell:

```
## 11. The gaps along the history

The next cell draws the Kohn-Sham gap of $N = 8$, $136$ and $688$ (without
interaction) against the slice on a logarithmic vertical axis, together with dashed
lines proportional to $e^{-a_{4,0}}$ through the first point of each curve. Every gap
shrinks along the history, somewhat more slowly than $e^{-a_{4,0}}$: the gap is the
distance between two brane-band levels (for $N = 688$ at $a_{4,0} = 0$ between a band
level and the bulk level at $k = 0$), and the band levels approach the straight line
$c\,k\,e^{-a_{4,0}}$ only when the redshifted momenta $k e^{-a_{4,0}}$ are small.
Without interaction the levels do not change when one particle is
moved, so the Delta-SCF energy must equal the gap exactly; the solver recorded this
as its check ‘excited_delta_scf_free_equals_gap‘.
```

In [9]:

```
fig, ax = plt.subplots()
for colour, n in zip(PALETTE, (8, 136, 688)):
    gaps = [float(ground[state_id(n, "lam0", a4)]["KS_gap"]) for a4 in SLICES]
    ax.plot(SLICES, gaps, "o-", color=colour, lw=1.5, label=f"$N = {n}$")
    ax.plot(a_fine, gaps[0] * np.exp(-a_fine), "--", color=colour, lw=1.0)
ax.set_yscale("log")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("Kohn-Sham gap $\\Delta_{KS}$ (units of $m$)")
ax.set_title("The gap closes along the history (dashed: $\\propto e^{-a_{4,0}}$)")
ax.legend()
save_figure(fig, "gaps_history",
            "Kohn-Sham gap $\\Delta_{KS}$ (vertical axis, logarithmic, units of $m$) "
            "of the states $N = 8$, $136$ and $688$ without interaction at the five "
            "slices $a_{4,0}$ (horizontal axis). Dashed lines fall like "
            "$e^{-a_{4,0}}$ from the first point of each curve: every gap shrinks "
            "along the history, somewhat more slowly than this, because the levels "
            "that bound it belong to the brane band, whose momenta redshift like "
            "$k e^{-a_{4,0}}$ but whose energy is linear in the momentum only for "
            "small momenta.")
free_dscf = max(abs(float(excited[state_id(n, "lam0", a)]["delta_SCF_minus_gap"]))
                for n in (8, 136, 688) for a in SLICES)
report("largest |Delta-SCF - gap| without interaction", f"{free_dscf:.1e}")
check(free_dscf <= 1e-10, "Delta-SCF equals the gap without interaction",
      record=f"{RECORD_REPORT}, check excited_delta_scf_free_equals_gap")
shrinking = all(float(ground[state_id(n, "lam0", SLICES[i + 1])]["KS_gap"])
                < float(ground[state_id(n, "lam0", SLICES[i])]["KS_gap"])
                for n in (8, 136, 688) for i in range(4))
check(shrinking, "every gap shrinks from slice to slice")
```

Out [9]:

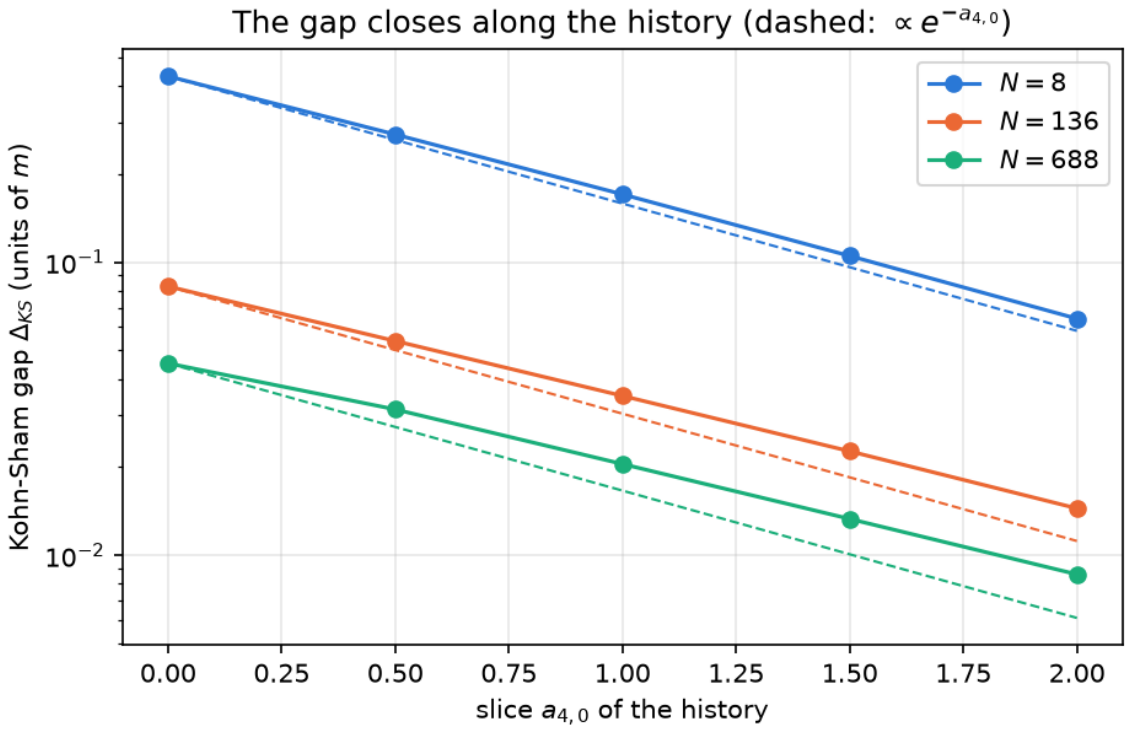

Figure 412: Kohn-Sham gap Δ_{KS} (vertical axis, logarithmic, units of m) of the states $N = 8$, 136 and 688 without interaction at the five slices $a_{4,0}$ (horizontal axis). Dashed lines fall like $e^{-a_{4,0}}$ from the first point of each curve: every gap shrinks along the history, somewhat more slowly than this, because the levels that bound it belong to the brane band, whose momenta redshift like $ke^{-a_{4,0}}$ but whose energy is linear in the momentum only for small momenta. (Notebook 15a, figure 2.)

```
Figure 15a.2 saved as Revision/textbook/figures/15a_2_gaps_history.png
RESULT largest |Delta-SCF - gap| without interaction = 2.0e-11
PASS Delta-SCF equals the gap without interaction
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    excited_delta_scf_free_equals_gap
PASS every gap shrinks from slice to slice
```

Text cell:

```
With interaction the orbitals relax when a particle is moved, and Delta-SCF differs
from the gap. The next cell draws the difference $\Delta_{SCF} - \Delta_{KS}$ for
the four couplings $\pm\lambda_1$, $\pm\lambda_2$ and the three particle numbers. The
vertical axis is *symmetric logarithmic*: logarithmic for large positive and
negative values and linear near zero (between $-10^{-9}$ and $10^{-9}$), so values of
both signs and very different sizes fit on one axis.
```

In [10]:

```
fig, axes = plt.subplots(1, 3, figsize=(10.0, 3.8), sharey=True,
                         layout="constrained")
for ax, n in zip(axes, (8, 136, 688)):
    for colour, tag, label in zip(PALETTE, ("lamp1", "lamm1", "lamp2", "lamm2"),
                                  ("$+\\lambda_1$", "$-\\lambda_1$", "$+\\lambda_2$",
                                   "$-\\lambda_2$")):
        diff = [float(excited[state_id(n, tag, a)]["delta_SCF_minus_gap"])
                for a in SLICES]
        ax.plot(SLICES, diff, "o-", color=colour, lw=1.5, label=label)
    ax.set_yscale("symlog", linthresh=1e-9)
    ax.set_title(f"$N = {n}$")
    ax.set_xlabel("slice $a_{4,0}$")
axes[0].set_ylabel("$\\Delta_{SCF} - \\Delta_{KS}$ (units of $m$)")
axes[0].legend(fontsize=8)
largest_id = max(excited, key=lambda i: abs(float(excited[i]["delta_SCF_minus_gap"])))
largest_relax = abs(float(excited[largest_id]["delta_SCF_minus_gap"]))
mantissa, power = f"{largest_relax:.1e}".split("e")  # e.g. "5.5", "-04"
save_figure(fig, "delta_scf",
            "Orbital relaxation: the Delta-SCF excitation energy minus the Kohn-Sham "
            "gap (vertical axis, symmetric logarithmic, units of $m$) for the "
            "couplings $\\pm\\lambda_1$ and $\\pm\\lambda_2$ at the five slices "
            "(horizontal axis), for $N = 8$, $136$ and $688$. It is small everywhere; "
            f"it is largest, ${mantissa} \\times 10^{{{int(power)}}}\\,m$, for "
            "$N = 688$ with $+\\lambda_2$ at $a_{4,0} = 0$, where the lowest empty "
            "level is a bulk level at $k = 0$.")
report("largest |Delta-SCF - gap| over the 75 states",
       f"{largest_relax:.3e} ({largest_id})")
check(largest_id == "N688_lamp2_a00",
      "the largest relaxation belongs to N = 688, +lambda_2, a4,0 = 0 (the caption)")
check(largest_relax < 1e-3, "the orbital relaxation is below 0.001 m in every state")
```

Out [10]:

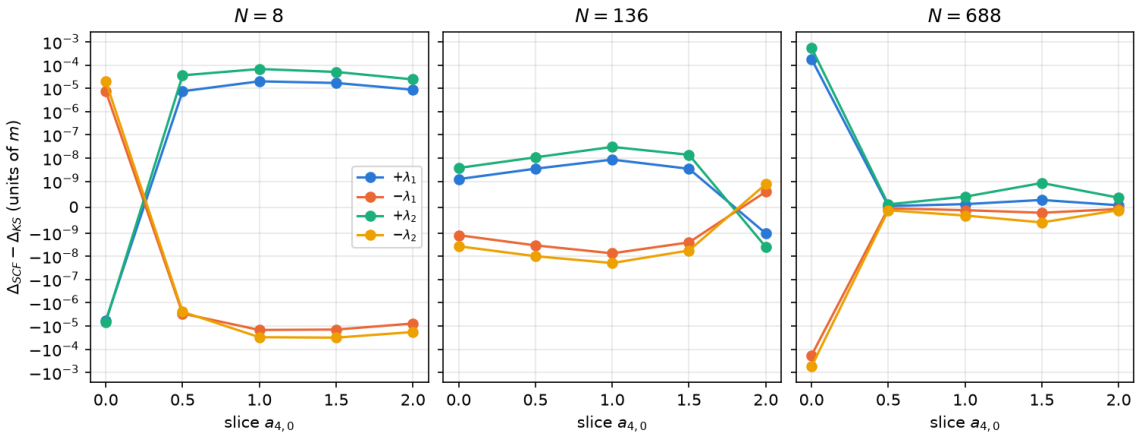

Figure 413: Orbital relaxation: the Delta-SCF excitation energy minus the Kohn-Sham gap (vertical axis, symmetric logarithmic, units of m) for the couplings $\pm\lambda_1$ and $\pm\lambda_2$ at the five slices (horizontal axis), for $N = 8$, 136 and 688. It is small everywhere; it is largest, $5.5 \times 10^{-4}\, m$, for $N = 688$ with $+\lambda_2$ at $a_{4,0} = 0$, where the lowest empty level is a bulk level at $k = 0$. (Notebook 15a, figure 3.)

```
Figure 15a.3 saved as Revision/textbook/figures/15a_3_delta_scf.png
RESULT largest |Delta-SCF - gap| over the 75 states = 5.496e-04 (N688_lamp2_a00)
PASS the largest relaxation belongs to N = 688, +lambda_2, a4,0 = 0 (the caption)
PASS the orbital relaxation is below 0.001 m in every state
```

Text cell:

```
## 12. The total energy along the history

The next cell draws $E_{KS}$ of $N = 136$ and $688$ without interaction (left) and
the change of $E_{KS}$ caused by the interaction, $E_{KS}(\lambda) - E_{KS}(0)$, for
$N = 136$ (right). The energy of the gas falls along the history because the brane
band redshifts. A repulsive coupling ($\lambda > 0$) raises the energy and an
attractive one lowers it, for $N = 136$ and $N = 688$; the check confirms this order
at every slice. (For $N = 8$ the order is reversed: the eight zero modes have no
scalar density $S$, so to first order in $\lambda$ the only interaction energy is
the exchange term $-\tfrac{1}{32}\lambda n^2$, which is negative for $\lambda > 0$.)
```

In [11]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
for colour, n in zip(PALETTE, (136, 688)):
    energies = [float(ground[state_id(n, "lam0", a)]["E_KS"]) for a in SLICES]
    left.plot(SLICES, energies, "o-", color=colour, lw=1.5, label=f"$N = {n}$")
left.set_yscale("log")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("$E_{KS}$ (units of $m$)")
left.set_title("Energy without interaction")
left.legend()
tags = ("lamp2", "lamp1", "lamm1", "lamm2")
labels = ("$+\\lambda_2$", "$+\\lambda_1$", "$-\\lambda_1$", "$-\\lambda_2$")
```

```
largest_shift = 0.0  # the largest |E(lambda) - E(0)| of N = 136
for colour, tag, label in zip(PALETTE, tags, labels):
    shift = [float(ground[state_id(136, tag, a)]["E_KS"])
             - float(ground[state_id(136, "lam0", a)]["E_KS"]) for a in SLICES]
    largest_shift = max([largest_shift] + [abs(s) for s in shift])
    right.plot(SLICES, shift, "o-", color=colour, lw=1.5, label=label)
e136 = [float(ground[state_id(136, "lam0", a)]["E_KS"]) for a in SLICES]
right.axhline(0.0, color="0.4", lw=0.8)
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$E_{KS}(\\lambda) - E_{KS}(0)$ (units of $m$)")
right.set_title("Interaction energy shift, $N = 136$")
right.legend(fontsize=8)
save_figure(fig, "energy_history",
            "Left: the Kohn-Sham energy $E_{KS}$ (vertical axis, logarithmic, units "
            "of $m$) of $N = 136$ and $N = 688$ without interaction at the five "
            "slices; it falls along the history as the brane band redshifts. Right: "
            "the energy shift caused by the couplings $\\pm\\lambda_1$ and "
            "$\\pm\\lambda_2$ for $N = 136$ (vertical axis, units of $m$): repulsion "
            f"raises and attraction lowers the energy, by at most "
            f"${largest_shift:.3f}\\,m$ out of ${min(e136):.0f}$ to "
            f"${max(e136):.0f}\\,m$.")
report("largest |E(lambda) - E(0)| of N = 136", f"{largest_shift:.4f}")
ordered = all(
    float(ground[state_id(n, "lamp2", a)]["E_KS"])
    > float(ground[state_id(n, "lamp1", a)]["E_KS"])
    > float(ground[state_id(n, "lam0", a)]["E_KS"])
    > float(ground[state_id(n, "lamm1", a)]["E_KS"])
    > float(ground[state_id(n, "lamm2", a)]["E_KS"])
    for n in (136, 688) for a in SLICES)
check(ordered, "E(+lambda2) > E(+lambda1) > E(0) > E(-lambda1) > E(-lambda2), "
               "N = 136 and 688")
falling = all(float(ground[state_id(n, "lam0", SLICES[i + 1])]["E_KS"])
              < float(ground[state_id(n, "lam0", SLICES[i])]["E_KS"])
              for n in (136, 688) for i in range(4))
check(falling, "the energy of N = 136 and 688 falls from slice to slice")
```

Out [11]:

```
Figure 15a.4 saved as Revision/textbook/figures/15a_4_energy_history.png
RESULT largest |E(lambda) - E(0)| of N = 136 = 0.0318
PASS E(+lambda2) > E(+lambda1) > E(0) > E(-lambda1) > E(-lambda2), N = 136 and 688
PASS the energy of N = 136 and 688 falls from slice to slice
```

Text cell:

```
## 13. Where the particles are: the densities

The solver writes for every ground state the **profiles**: the densities and the
energy-momentum tensor at the 151 points $y = -3, -2.98, \dots, 0$. The next cell
reads the proper number density $n(y)$ of $N = 136$ without interaction at the
slices $0$, $1$ and $2$ and draws it (left, logarithmic) together with the number of
particles per unit of $y$, $2\,\mathrm{Vol}_7\, e^{6Hy} n(y)$ (right), where
```

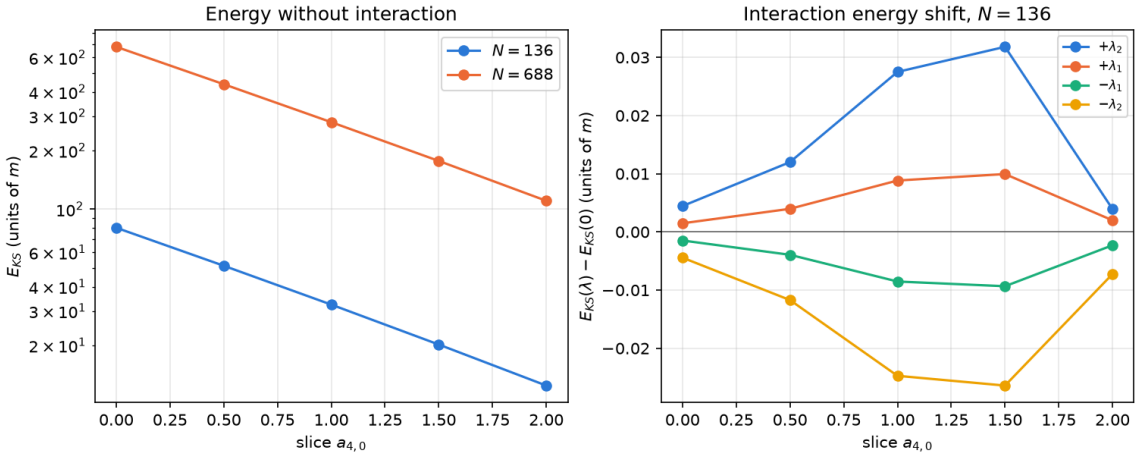

Figure 414: Left: the Kohn-Sham energy E_{KS} (vertical axis, logarithmic, units of m) of $N = 136$ and $N = 688$ without interaction at the five slices; it falls along the history as the brane band redshifts. Right: the energy shift caused by the couplings $\pm\lambda_1$ and $\pm\lambda_2$ for $N = 136$ (vertical axis, units of m): repulsion raises and attraction lowers the energy, by at most $0.032\,m$ out of 12 to $80\,m$. (Notebook 15a, figure 4.)

```
$\mathrm{Vol}_7 = (2\pi/\Delta k)^3$ is the proper 7-volume of the box per unit of
$e^{6Hy}$. The proper density is largest at the tip (the proper volume
$e^{6Hy}$ is tiny there), but the particles themselves sit near the brane; along the
history they spread toward the tip.

The integral of the right-hand curve over $y$ must be $N = 136$. It is computed with
**Simpson's rule** on the 151 points (step $h = 0.02$): $\int f\,dy \approx
\tfrac{h}{3}(f_0 + 4 f_1 + 2 f_2 + 4 f_3 + \dots + 4 f_{149} + f_{150})$.
```

In [12]:

```
VOL7 = (2.0 * math.pi / 0.25) ** 3  # proper 7-volume per unit e^{6Hy} (v_t = 1)


def read_profile(n, tag, a4):
    """The profile table of one state as a dictionary column -> numpy array."""
    path = NEW_RESULTS / "ground/profiles" / f"{state_id(n, tag, a4)}.csv"
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}


def simpson(values, step):
    """Simpson's rule for equally spaced values (an odd number of points)."""
    weights = np.full(len(values), 2.0)
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    return step / 3.0 * float(np.sum(weights * values))


fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
```

```
counts = []
for shade, a4 in zip((SLICE_SHADES[0], SLICE_SHADES[2], SLICE_SHADES[4]), (0, 1, 2)):
    prof = read_profile(136, "lam0", a4)
    y = prof["y"]
    per_y = 2.0 * VOL7 * np.exp(6.0 * y) * prof["n"]  # particles per unit y
    counts.append(simpson(per_y, y[1] - y[0]))
    left.plot(y, prof["n"], color=shade, lw=1.8, label=f"$a_{{4,0}} = {a4}$")
    right.plot(y, per_y, color=shade, lw=1.8, label=f"$a_{{4,0}} = {a4}$")
left.set_yscale("log")
left.set_xlabel("hidden coordinate $y$ (tip at $-3$, brane at $0$)")
left.set_ylabel("proper density $n(y)$")
left.set_title("Proper number density")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("particles per unit $y$")
right.set_title("$2\\,\\mathrm{Vol}_7\\, e^{6Hy} n(y)$, area $= N$")
right.legend()
save_figure(fig, "densities",
            "Left: the proper number density $n(y)$ (vertical axis, logarithmic, "
            "particles per unit proper 7-volume) of $N = 136$ without interaction "
            "at $a_{4,0} = 0$, $1$, $2$ against the hidden coordinate $y$ (horizontal "
            "axis). Right: the particles per unit $y$, "
            "$2\\,\\mathrm{Vol}_7 e^{6Hy} n(y)$, whose area is $N = 136$. The particles "
            "sit near the brane $y = 0$ and spread toward the tip along the history; "
            "the proper density is largest at the tip, where the proper volume is "
            "small.")
report("particle number from the profiles at a4,0 = 0, 1, 2",
       ", ".join(f"{c:.6f}" for c in counts))
check(max(abs(c - 136.0) for c in counts) < 1e-5,
      "Simpson's rule on the profiles gives N = 136 at every slice",
      record=f"{RECORD_REPORT}, check ground_N_conservation")
```

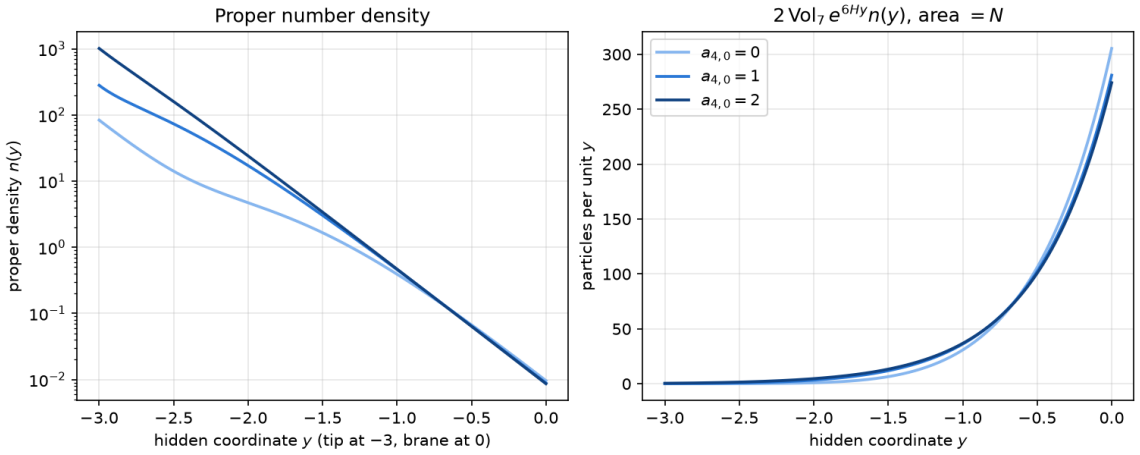

Figure 415: Left: the proper number density $n(y)$ (vertical axis, logarithmic, particles per unit proper 7-volume) of $N = 136$ without interaction at $a_{4,0} = 0$, 1, 2 against the hidden coordinate y (horizontal axis). Right: the particles per unit y, $2\,\mathrm{Vol}_7 e^{6Hy} n(y)$, whose area is $N = 136$. The particles sit near the brane $y = 0$ and spread toward the tip along the history; the proper density is largest at the tip, where the proper volume is small. (Notebook 15a, figure 5.)

Out [12]:

```
Figure 15a.5 saved as Revision/textbook/figures/15a_5_densities.png
RESULT particle number from the profiles at a4,0 = 0, 1, 2 = 136.000002, 136.000002,
    136.000002
PASS Simpson's rule on the profiles gives N = 136 at every slice
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    ground_N_conservation
```

Text cell:

```
## 14. The self-consistent potentials

With interaction the effective mass $M(y) = m + \tfrac{15}{16}\lambda S(y)$ and the
potential $v(y) = -\tfrac{1}{16}\lambda n(y)$ differ from $m$ and $0$. The next cell
draws $M - m$ and $v$ for $N = 136$ with the repulsive coupling $+\lambda_2$ at the
five slices. Both are largest in the tip region and become much larger along the
history (the largest $|v|$ grows at every slice; the largest $|M - m|$ grows up to
$a_{4,0} = 1.5$ and is a little smaller at $a_{4,0} = 2$), because the redshifted
brane-band orbitals spread toward the tip, where the proper densities are large.
This is why the couplings were calibrated over the whole history (so that the
first-order potential stays below $0.3\,m$ at every slice).
The check compares, for each slice, the largest $|M - m|$ on the 151 profile points
with the value `max_abs_Meff_minus_m` that the solver recorded on its fine grid of
1801 points (every profile point is also a point of the fine grid): the profile
value cannot be larger, and since the maximum lies between two profile points it
is smaller by a little, at most 0.2 percent.
```

In [13]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
ratios = []  # profile maximum / recorded maximum, one per slice
for shade, a4 in zip(SLICE_SHADES, SLICES):
    prof = read_profile(136, "lamp2", a4)
    left.plot(prof["y"], prof["M_eff"] - 1.0, color=shade, lw=1.8,
              label=f"$a_{{4,0}} = {a4}$")
    right.plot(prof["y"], prof["v_v"], color=shade, lw=1.8)
    recorded = float(ground[state_id(136, "lamp2", a4)]["max_abs_Meff_minus_m"])
    ratios.append(float(np.max(np.abs(prof["M_eff"] - 1.0))) / recorded)
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("$M(y) - m$ (units of $m$)")
left.set_title("Mass shift, $N = 136$, $+\\lambda_2$")
left.legend(fontsize=8)
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("$v(y)$ (units of $m$)")
right.set_title("Exchange potential, $N = 136$, $+\\lambda_2$")
m_max = [float(ground[state_id(136, "lamp2", a4)]["max_abs_Meff_minus_m"])
         for a4 in SLICES]  # the solver's largest |M - m| at each slice
v_max = [float(ground[state_id(136, "lamp2", a4)]["max_abs_v_v"]) for a4 in SLICES]
peak = int(np.argmax(m_max))  # the slice where |M - m| is largest
save_figure(fig, "potentials",
```

```
            "The self-consistent mass shift $M(y) - m$ (left) and potential $v(y)$ "
            "(right), vertical axes in units of $m$, of $N = 136$ with the repulsive "
            "coupling $+\\lambda_2$ at the five slices (light blue: $a_{4,0} = 0$, "
            "dark blue: $a_{4,0} = 2$) against the hidden coordinate $y$. Both are "
            "concentrated in the tip region and become much larger along the "
            f"history: the largest $|M - m|$ grows from ${m_max[0]:.3f}\\,m$ at "
            f"$a_{{4,0}} = 0$ to ${m_max[peak]:.3f}\\,m$ at "
            f"$a_{{4,0}} = {SLICES[peak]}$ (${m_max[-1]:.3f}\\,m$ at "
            f"$a_{{4,0}} = 2$), the largest $|v|$ from ${v_max[0]:.3f}\\,m$ to "
            f"${v_max[-1]:.3f}\\,m$; both stay below $0.4\\,m$.")
report("profile maximum / recorded maximum of |M - m|",
       ", ".join(f"{r:.5f}" for r in ratios))
check(all(0.998 <= r <= 1.0 + 1e-12 for r in ratios),
      "the profile maxima of |M - m| lie within 0.2 percent below the recorded ones",
      record=f"{RECORD_RESULTS}/ground/summary.csv, column max_abs_Meff_minus_m")
report("largest |M - m| at the five slices", ", ".join(f"{v:.3f}" for v in m_max))
report("largest |v| at the five slices", ", ".join(f"{v:.3f}" for v in v_max))
check(all(v_max[i + 1] > v_max[i] for i in range(4)) and SLICES[peak] == 1.5
      and max(m_max + v_max) < 0.4,
      "|v| grows at every slice, |M - m| peaks at a4,0 = 1.5, both stay below 0.4 m",
      record=f"{RECORD_RESULTS}/ground/summary.csv, columns max_abs_Meff_minus_m "
             "and max_abs_v_v")
```

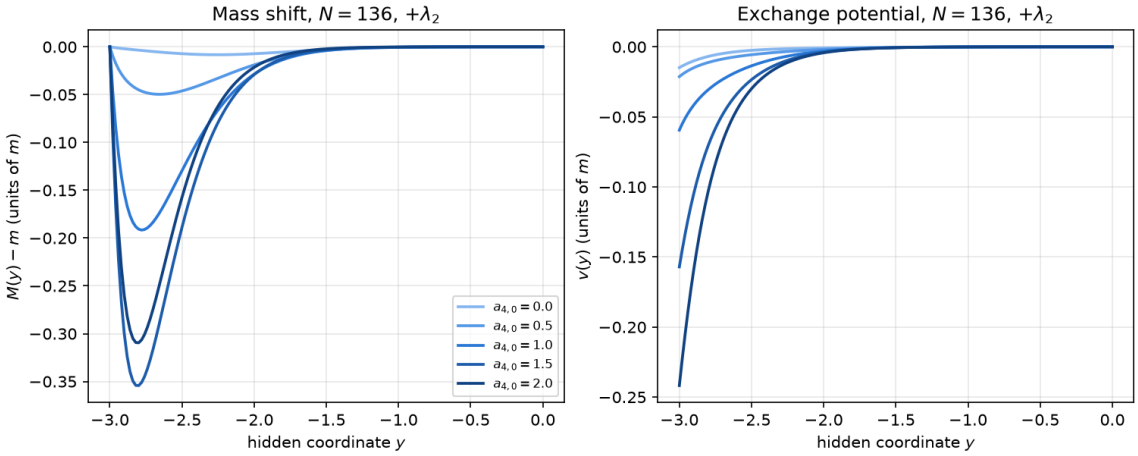

Figure 416: The self-consistent mass shift $M(y) - m$ (left) and potential $v(y)$ (right), vertical axes in units of m, of $N = 136$ with the repulsive coupling $+\lambda_2$ at the five slices (light blue: $a_{4,0} = 0$, dark blue: $a_{4,0} = 2$) against the hidden coordinate y. Both are concentrated in the tip region and become much larger along the history: the largest $|M - m|$ grows from $0.008\,m$ at $a_{4,0} = 0$ to $0.355\,m$ at $a_{4,0} = 1.5$ ($0.310\,m$ at $a_{4,0} = 2$), the largest $|v|$ from $0.015\,m$ to $0.242\,m$; both stay below $0.4\,m$. (Notebook 15a, figure 6.)

Out [13]:

```
Figure 15a.6 saved as Revision/textbook/figures/15a_6_potentials.png
RESULT profile maximum / recorded maximum of |M - m| = 0.99997, 0.99996, 0.99997,
    0.99893, 0.99891
PASS the profile maxima of |M - m| lie within 0.2 percent below the recorded ones
```

```
     reproduces Revision/kohn_sham/results/ground/summary.csv, column
    max_abs_Meff_minus_m
RESULT largest |M - m| at the five slices = 0.008, 0.050, 0.192, 0.355, 0.310
RESULT largest |v| at the five slices = 0.015, 0.021, 0.059, 0.157, 0.242
PASS |v| grows at every slice, |M - m| peaks at a4,0 = 1.5, both stay below 0.4 m
     reproduces Revision/kohn_sham/results/ground/summary.csv, columns
    max_abs_Meff_minus_m and max_abs_v_v
```

Text cell:

```
## 15. How fast the self-consistent loop converges

The file ‘ground/runs.json‘ records for every ground state the residual of each
iteration of the self-consistent loop. The next cell draws it for the strongest
couplings $\pm\lambda_2$ at the last slice $a_{4,0} = 2$ (the hardest cases), with
the tolerance $10^{-11}$ as a dotted line, and checks that all 75 ground states
converged directly (path ‘direct‘: no fallback was needed) with a final residual at
or below the tolerance (zero for $\lambda = 0$, where one iteration suffices).
```

In [14]:

```
runs = json.loads((NEW_RESULTS / "ground/runs.json").read_text(encoding="utf-8"))
by_id = {run["id"]: run for run in runs}
fig, ax = plt.subplots()
styles = {"lamp2": "-", "lamm2": "--"}
for colour, n in zip(PALETTE, (8, 136, 688)):
    for tag, style in styles.items():
        history = by_id[state_id(n, tag, 2.0)]["scfHistory_iteration_residual_E"]
        ax.plot([h[0] for h in history], [h[1] for h in history], style, marker="o",
                ms=4, color=colour, lw=1.5,
                label=f"$N = {n}$, {'+' if tag == 'lamp2' else '-'}$\\lambda_2$")
ax.axhline(1e-11, color="0.3", ls=":", lw=1.2, label="tolerance $10^{-11}$")
ax.set_yscale("log")
ax.set_xlabel("iteration")
ax.set_ylabel("residual: largest change of the potential (units of $m$)")
ax.set_title("Anderson mixing, $a_{4,0} = 2$, couplings $\\pm\\lambda_2$")
ax.legend(fontsize=8, ncol=2)
save_figure(fig, "scf_convergence",
            "Convergence of the self-consistent loop with Anderson mixing: the "
            "residual (vertical axis, logarithmic, units of $m$) at each iteration "
            "(horizontal axis) for $N = 8$, $136$, $688$ with the couplings "
            "$+\\lambda_2$ (solid) and $-\\lambda_2$ (dashed) at $a_{4,0} = 2$; the "
            "dotted line is the tolerance $10^{-11}$. The residual falls by about one "
            "order of magnitude per iteration, and every loop ends below the "
            "tolerance after at most 17 iterations. For $N = 8$ the curves of "
            "$+\\lambda_2$ and $-\\lambda_2$ coincide, a consequence of an exact "
            "symmetry of the zero-mode state.")
direct = all(run["path"] == "direct" for run in runs)
final = max(run["scfHistory_iteration_residual_E"][-1][1] for run in runs)
longest = max(run["iterations"] for run in runs)
report("largest final residual / largest number of iterations",
       f"{final:.2e} / {longest}")
```

```
check(len(runs) == 75 and direct and final <= 1e-11,
      "all 75 ground states converged directly below the tolerance 1e-11",
      record=f"{RECORD_REPORT}, check ground_scf_converged")
```

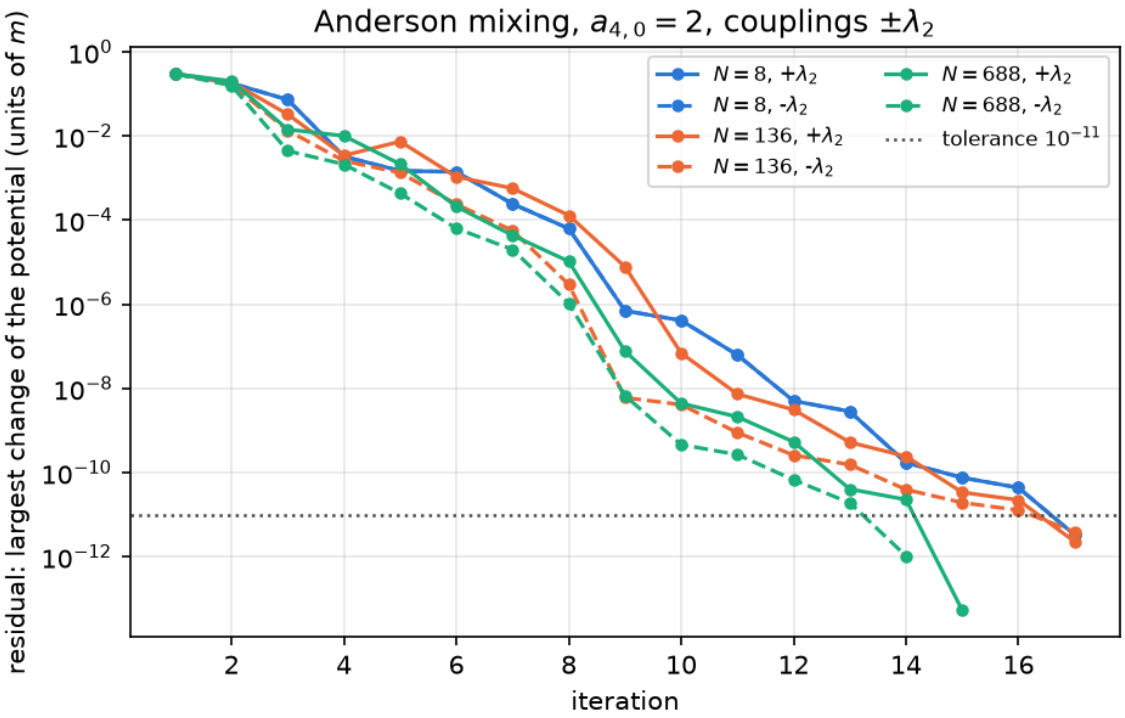

Figure 417: Convergence of the self-consistent loop with Anderson mixing: the residual (vertical axis, logarithmic, units of m) at each iteration (horizontal axis) for $N = 8$, 136, 688 with the couplings $+\lambda_2$ (solid) and $-\lambda_2$ (dashed) at $a_{4,0} = 2$; the dotted line is the tolerance 10^{-11}. The residual falls by about one order of magnitude per iteration, and every loop ends below the tolerance after at most 17 iterations. For $N = 8$ the curves of $+\lambda_2$ and $-\lambda_2$ coincide, a consequence of an exact symmetry of the zero-mode state. (Notebook 15a, figure 7.)

Out [14]:

```
Figure 15a.7 saved as Revision/textbook/figures/15a_7_scf_convergence.png
RESULT largest final residual / largest number of iterations = 9.34e-12 / 17
PASS all 75 ground states converged directly below the tolerance 1e-11
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    ground_scf_converged
```

Text cell:

```
## 16. The excitations of the gas along the history

For every ground state the solver also lists its lowest **particle-hole
excitations** (up to 24, in the files ‘excited/particle-hole/<state>.csv‘): one
particle is taken out of an occupied level (the *hole*) and put into an empty level
```

(the *particle*); the excitation energy is $\Delta\varepsilon =
\varepsilon_{particle} - \varepsilon_{hole}$, and the *multiplicity* is the number
of ways to do it, $g_{hole} \times g_{particle}$. Each excitation is named by its
two levels, written ‘n2:j:parity:label‘. The next cell draws the excitations of
$N = 136$ without interaction at the five slices and makes three checks over all
75 states: (1) the lowest excitation is the Kohn-Sham gap; (2) every excitation
listed at all five slices of a series gets cheaper from slice to slice; (3) none of
the listed excitations stays inside one *sector* (the same shell n_2, block type
j and parity). The third point matters for the history: the exact evolution
along $a_4 = AHx_4$ keeps the momentum, the block type and the parity of every
orbital, so the history alone cannot create any of these excitations. It can only
cause jumps inside a sector; the cell prints the energy of the jump with the
largest adiabaticity number Q of each state (column ‘Q_max_delta_eps‘ of
‘adiabatic/adiabaticity.csv‘), which is much larger.

In [15]:

```
TAGS = ("lam0", "lamp1", "lamm1", "lamp2", "lamm2")  # the five couplings


def read_pairs(n, tag, a4):
    """{(hole, particle): (energy, multiplicity, same sector?)} of one state."""
    path = NEW_RESULTS / "excited/particle-hole" / f"{state_id(n, tag, a4)}.csv"
    with open(path, newline="", encoding="utf-8") as handle:
        return {(r["hole_levels"], r["particle_levels"]):
                (float(r["delta_eps"]), float(r["multiplicity"]), r["same_sector"])
                for r in csv.DictReader(handle)}


lists = {(n, tag, a4): read_pairs(n, tag, a4)
         for n in (8, 136, 688) for tag in TAGS for a4 in SLICES}  # 75 states
fig, ax = plt.subplots()
for a4 in SLICES:
    values = list(lists[(136, "lam0", a4)].values())
    ax.scatter([a4] * len(values), [v[0] for v in values],
               s=[v[1] / 40.0 for v in values],  # marker area ~ multiplicity
               color=PALETTE[0], alpha=0.3, edgecolors=PALETTE[0])
lowest = [min(v[0] for v in lists[(136, "lam0", a4)].values()) for a4 in SLICES]
ax.plot(SLICES, lowest, "o-", color=PALETTE[1], lw=1.8, ms=5,
        label="lowest excitation (the Kohn-Sham gap)")
ax.plot(a_fine, lowest[0] * np.exp(-a_fine), "--", color="0.4", lw=1.0,
        label="$\\propto e^{-a_{4,0}}$")
ax.scatter([], [], s=60, color=PALETTE[0], alpha=0.3, edgecolors=PALETTE[0],
           label="excitations (area: multiplicity)")
ax.set_yscale("log")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("excitation energy $\\Delta\\varepsilon$ (units of $m$)")
ax.set_title("Particle-hole excitations of $N = 136$, $\\lambda = 0$")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "particle_hole",
            "The lowest particle-hole excitation energies (vertical axis, "
            "logarithmic, units of $m$) of the free state $N = 136$ at the five "
            "slices $a_{4,0}$ (horizontal axis); the area of a marker is proportional "
```

```
            "to the multiplicity of the excitation. Orange: the lowest excitation, "
            "which is the Kohn-Sham gap; dashed: a fall like $e^{-a_{4,0}}$ from the "
            "first gap. The whole ladder moves down along the history because the "
            "brane-band levels on both sides of each excitation redshift; none of "
            "these excitations stays inside one sector, so the history alone cannot "
            "create them.")
lowest_is_gap = all(
    abs(min(v[0] for v in lists[key].values())
        - float(ground[state_id(*key)]["KS_gap"])) <= 1e-12 for key in lists)
cheaper = True
for n in (8, 136, 688):
    for tag in TAGS:
        series = [lists[(n, tag, a4)] for a4 in SLICES]
        common = set(series[0]).intersection(*series[1:])  # listed at every slice
        cheaper = cheaper and all(series[i + 1][key][0] < series[i][key][0]
                                  for key in common for i in range(4))
count = sum(len(pairs) for pairs in lists.values())
inside = sum(1 for pairs in lists.values() for v in pairs.values() if v[2] != "false")
jumps = [float(row["Q_max_delta_eps"]) for row in
         read_table(NEW_RESULTS, "adiabatic/adiabaticity.csv").values()
         if row["Q_max_delta_eps"] != "null"]  # null: no jump (N = 8)
report("particle-hole excitations listed / inside one sector", f"{count} / {inside}")
report("lowest excitation of N = 136 at a4,0 = 0 and 2",
       f"{lowest[0]:.5f}, {lowest[-1]:.5f}")
report("energy of the largest-Q jump inside a sector (smallest, largest)",
       f"{min(jumps):.3f}, {max(jumps):.3f}")
check(lowest_is_gap, "in all 75 states the lowest particle-hole energy is the KS gap",
      record=f"{RECORD_RESULTS}/excited/particle-hole and ground/summary.csv")
check(cheaper, "every excitation listed at all five slices gets cheaper along the "
               "history")
check(count >= len(lists) and inside == 0,
      f"none of the {count} listed excitations stays inside one sector",
      record=f"{RECORD_RESULTS}/excited/particle-hole, column same_sector")
```

Out [15]:

```
Figure 15a.8 saved as Revision/textbook/figures/15a_8_particle_hole.png
RESULT particle-hole excitations listed / inside one sector = 1610 / 0
RESULT lowest excitation of N = 136 at a4,0 = 0 and 2 = 0.08267, 0.01446
RESULT energy of the largest-Q jump inside a sector (smallest, largest) = 1.488, 2.253
PASS in all 75 states the lowest particle-hole energy is the KS gap
     reproduces Revision/kohn_sham/results/excited/particle-hole and ground/summary.csv
PASS every excitation listed at all five slices gets cheaper along the history
PASS none of the 1610 listed excitations stays inside one sector
     reproduces Revision/kohn_sham/results/excited/particle-hole, column same_sector
```

Text cell:

```
## 17. The last check

The last cell checks that every figure file of this notebook exists in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

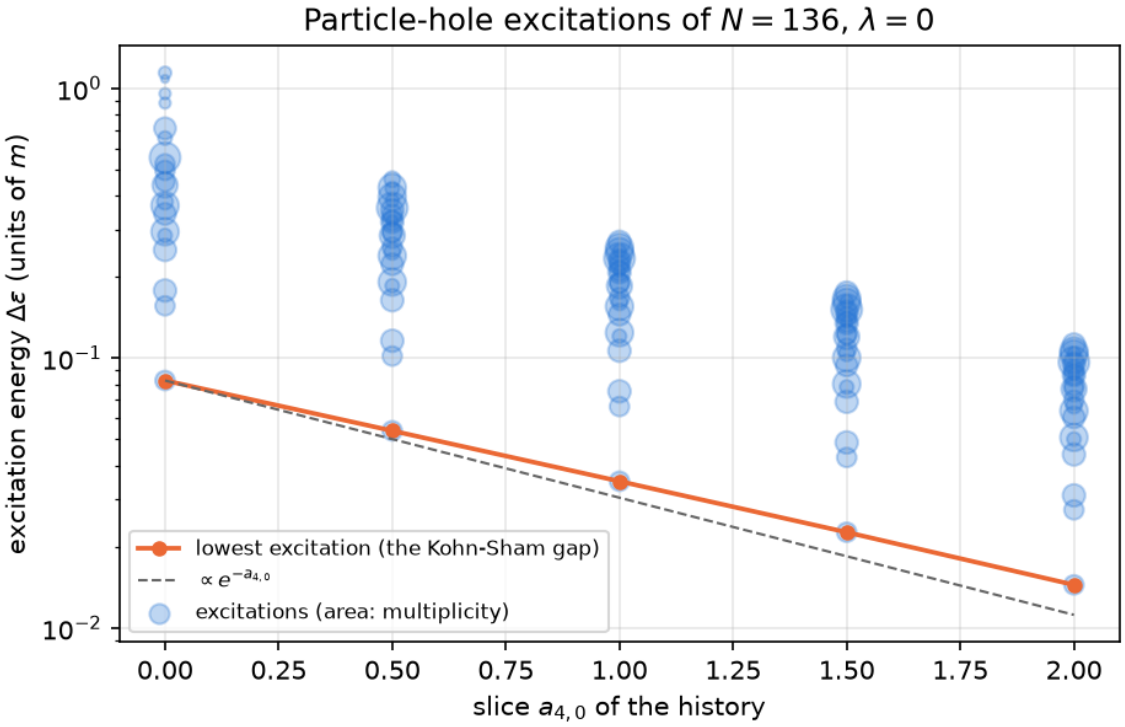

Figure 418: The lowest particle-hole excitation energies (vertical axis, logarithmic, units of m) of the free state $N = 136$ at the five slices $a_{4,0}$ (horizontal axis); the area of a marker is proportional to the multiplicity of the excitation. Orange: the lowest excitation, which is the Kohn-Sham gap; dashed: a fall like $e^{-a_{4,0}}$ from the first gap. The whole ladder moves down along the history because the brane-band levels on both sides of each excitation redshift; none of these excitations stays inside one sector, so the history alone cannot create them. (Notebook 15a, figure 8.)

In [16]:

```
NAMES = ["levels_history", "gaps_history", "delta_scf", "energy_history",
         "densities", "potentials", "scf_convergence",
         "particle_hole"]  # the figures, in order
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15a_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

Out [16]:

```
PASS every figure file of this notebook exists
ALL 25 CHECKS PASSED (notebook 15a)
```

Text cell:

```
## 18. What this notebook showed

- The Rust solver, built on this computer, reproduces the whole canonical matrix of
  the Revision record: the same files, every one of its own checks PASS, and every
  key number within the tolerances fixed in advance (COMPUTED).
- Along the deflating history the brane band redshifts like $k e^{-a_{4,0}}$, the
  levels at $k = 0$ stay where they are, the gaps close at nearly the rate
  $e^{-a_{4,0}}$, and the energy of the gas falls.
- Without interaction Delta-SCF equals the gap exactly; with interaction the orbital
  relaxation stays below $10^{-3}\,m$. The lowest particle-hole excitation is the
  gap, every listed excitation gets cheaper along the history, and none of them
  stays inside one sector, so the history alone cannot create them.
- The particles sit near the brane, the proper densities and the self-consistent
  potentials are largest at the tip, and the self-consistent loop converges in at
  most 17 iterations.
- These are instantaneous states on a PRESCRIBED background; the brane is ASSUMED;
  the filling is a CONVENTION; whether the gas really follows the instantaneous
  states along the history is the question of adiabaticity, which the solver
  measures with the number $Q_{max}$ compared above.
```

15.14 Line-by-line walk-through of Notebook 15a

The notebook has 16 code cells, In [1] to In [16]. As in Section 15.9, docstrings are left out of the quotations, and long caption strings in calls of `save_figure` are shortened: the quotation shows the first line of the call and the first line of the caption, and then a line `...)` that stands for the rest of the caption and the closing parenthesis; the complete cells are printed in Section 15.13.

In [1], the set-up cell. It is the set-up cell of Notebook 15e (explained line by line in Section 15.9), with three differences: the comment lines at the top hold the run instructions of this notebook (Section 15.12), the line `NOTEBOOK_ID = "15a"` names it, and, because this notebook runs a Rust program, it has two more imports and one more function:

```
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs
```

`shutil` is a module of Python for files and programs; its function `shutil.which` finds a program on the computer, and `shutil.rmtree` deletes a folder with everything in it. `subprocess` runs another program from Python and collects what it prints.

```
def rust_program(manifest, binary):
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
```

`rust_program` builds a Rust program and returns where it is. `manifest` is the repository path of the crate's file `Cargo.toml` (a **crate** is a Rust project; `Cargo.toml` describes it), and `binary` is the name of the program. If `shutil.which("cargo")` finds no program cargo, the function stops with a message that says how to install Rust. `crate` is the folder that holds `Cargo.toml` (`.parent` is the folder above a path).

```
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
```

`subprocess.run([...])` runs the command given as a list of words: cargo builds the crate in release mode (the fast, optimised translation) into the crate's own folder `target`. `capture_output=True` collects what cargo prints instead of showing it, `text=True` with `encoding="utf-8"` turns it into text, and `errors="replace"` replaces any byte that is not valid text. When cargo has nothing to do (the program is up to date) this takes about a second; the first build takes a minute or two. A program reports success with the **return code** 0; any other code means failure, and then the last 3000 characters of cargo's error stream (`stderr[-3000:]`) are printed and the notebook stops.

```
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")
```

The program lies in `target/release`; on Windows its file name ends with `.exe`, on Linux and macOS it has no ending, so both names are tried. The function prints that the program is ready and returns its path.

In [2], building the solver.

```
import csv  # reads the tables (CSV files) that the solver writes
import math  # exp, sqrt and pi for single numbers

import numpy as np  # arrays of numbers

SOLVER_MANIFEST = "Revision/kohn_sham/solver/Cargo.toml"  # the crate of the solver
program = rust_program(SOLVER_MANIFEST, "revision_ks_solver")  # build it, get its path
```

After the imports (Section 15.9, In [2]) the cell builds the solver and keeps the path of the program in `program`.

```
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # blue, orange, aqua, yellow, magenta, green, ...
SLICE_SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light->dark
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]  # the five slices a4,0 of the history
say("Packages imported; colours defined.")
```

`PALETTE` holds eight distinct colours, `SLICE_SHADES` five shades of blue for the five slices (light for the earliest, dark for the latest), and `SLICES` the slices $a_{4,0}$. Out [2] prints that the program is built and the line of the last statement.

In [3], running the canonical matrix.

```
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_15a"  # git ignores it
NEW_RESULTS = RUN_FOLDER / "results"  # the new canonical matrix
NEW_REPORT = RUN_FOLDER / "ks-rust-solver.json"  # the new check report
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
if NEW_RESULTS.exists() and not (NEW_RESULTS / "manifest.json").exists():
    shutil.rmtree(NEW_RESULTS)  # remains of an interrupted run
```

The new run is written into the folder `textbook_15a` inside the solver's build folder `target`, which git ignores; the committed record is never touched. The folder is created if necessary. If a results folder exists without its `manifest.json`, an earlier run was interrupted, and the solver would refuse to write into it; `shutil.rmtree` removes it.

```
completed = subprocess.run(
    [str(program), "all", "--root", str(REPO), "--out", str(NEW_RESULTS),
     "--report", str(NEW_REPORT)],
    capture_output=True, text=True, encoding="utf-8", errors="replace")
stdout_lines = completed.stdout.strip().split("\n")  # what it printed on stdout
stderr_lines = completed.stderr.split("\n")  # its check lines and timings
```

The solver runs the command `all`. Its option root (written, like every option, with two hyphens in front) tells it where the repository is (it reads the theory file, the gamma matrices and the fixture of 40-digit chemical potentials from there), and the options out and report name the folder of the results and the file of the check report. This is the longest step of the notebook. The printed text is split into lines: `.strip()` removes the blank space at the ends, `.split("\n")` cuts at the line ends.

```
pass_count = sum(1 for line in stderr_lines if line.startswith("PASS - "))
fail_count = sum(1 for line in stderr_lines if line.startswith("FAIL - "))
say(f"exit code {completed.returncode}, last line {stdout_lines[-1]!r}, "
    f"{pass_count} PASS lines, {fail_count} FAIL lines")
check(completed.returncode == 0 and stdout_lines[-1] == "SUCCESS" and fail_count == 0,
      "the solver ran the whole canonical matrix and reported SUCCESS")
```

`sum(1 for line in ... if ...)` counts the lines that start with PASS - and with FAIL -. The format `!r` prints the last output line with quotes. Out [3] prints exit code 0, the last line SUCCESS, 42 PASS lines and 0 FAIL lines, and the check passes.

In [4], the solver's check report.

```
RECORD_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
new_report = json.loads(NEW_REPORT.read_text(encoding="utf-8"))
old_report = json.loads(repository_file(RECORD_REPORT).read_text(encoding="utf-8"))
groups = {}  # first word of a check name -> [number of checks, number of PASS]
for item in new_report["checks"]:
    group = item["name"].split("_")[0]
    groups.setdefault(group, [0, 0])
    groups[group][0] += 1
    groups[group][1] += item["verdict"] == "PASS"  # True counts as 1
for group, (count, passed) in groups.items():
    say(f"  {group:10} {count:2d} checks, {passed:2d} PASS")
```

The new report and the committed one are read. Each check has a name, a verdict and a detail; the first word of the name (the text before the first underscore) is its group. For each group the cell counts the checks and the passed ones (a comparison gives `True` or `False`, which count as 1 and 0 when added). Out [4] lists the twelve groups, for example free with 7 checks and thermo with 11, all passed.

```
new_names = [item["name"] for item in new_report["checks"]]
old_names = [item["name"] for item in old_report["checks"]]
summary = new_report["summary"]  # {"checks": ..., "pass": ..., "fail": ...}
report("checks in the new report / PASS / FAIL",
       f"{summary['checks']} / {summary['pass']} / {summary['fail']}")
check(summary == old_report["summary"] and summary["fail"] == 0
      and summary["pass"] == summary["checks"] == len(new_names),
      f"the new report holds the {summary['checks']} checks of the record, all PASS",
      record=f"{RECORD_REPORT}, summary")
check(new_names == old_names, "the new report has the same checks as the record")
report("new report byte-identical to the record",
       NEW_REPORT.read_bytes() == repository_file(RECORD_REPORT).read_bytes())
```

The lists of check names of the two reports and the summary of the new one are formed. The first check requires the same summary as the record, no failure, and as many passes as checks; the number 42 is not typed into the notebook but read from the report. The second requires the same check names in the same order. The last line compares the two report files byte for byte (`read_bytes` reads a file as raw bytes); Out [4] prints True: on the computer that built the book the new

report is identical to the record. On another computer the last digits of some reported deviations may differ, which is why this is a RESULT and not a check.

In [5], every result file.

```
RECORD_RESULTS = "Revision/kohn_sham/results"


def manifest_of(folder):
    data = json.loads((folder / "manifest.json").read_text(encoding="utf-8"))
    return {entry["path"]: entry["sha256"] for entry in data["files"]}


old_manifest = manifest_of(repository_file(RECORD_RESULTS))
new_manifest = manifest_of(NEW_RESULTS)
check(sorted(old_manifest) == sorted(new_manifest),
      f"the new run wrote the same {len(old_manifest)} result files as the record",
      record=f"{RECORD_RESULTS}/manifest.json")
same = sum(1 for path in old_manifest if new_manifest[path] == old_manifest[path])
report("result files byte-identical to the record", f"{same} of {len(old_manifest)}")
```

`manifest_of` reads a manifest and returns a dictionary from each file path to its sha256 fingerprint. `sorted(old_manifest)` is the sorted list of the paths; the check requires the same set of files. `same` counts the files whose fingerprints agree. Out [5]: the same 243 result files, all 243 byte-identical to the record.

In [6], the numerical comparison.

```
def read_table(folder, name):
    with open(folder / name, newline="", encoding="utf-8") as handle:
        return {row["id"]: row for row in csv.DictReader(handle)}
```

`read_table` reads a CSV table of the results and files its rows under the state name in the column `id`.

```
def worst(table, columns, relative):
    new = read_table(NEW_RESULTS, table)
    old = read_table(repository_file(RECORD_RESULTS), table)
    assert sorted(new) == sorted(old), f"{table}: the state lists differ"
    largest = 0.0
    for key in old:
        for column in columns:
            a, b = float(new[key][column]), float(old[key][column])
            scale = max(abs(b), 1.0) if relative else 1.0
            largest = max(largest, abs(a - b) / scale)
    return largest
```

`worst` returns the largest difference between the new run and the record in the given columns of a table, over all states. The `assert` statement stops with the given message if the two tables do not list the same states (a safety net inside a helper, not one of the notebook's checks). For a relative comparison each difference is divided by the larger of $|b|$ and 1, so that numbers near zero are compared absolutely.

```
COMPARISONS = [  # (table, columns, relative?, tolerance, what)
    ("ground/summary.csv", ["E_KS"], True, 1e-8, "ground-state energies"),
    ("ground/summary.csv", ["HOMO", "LUMO", "KS_gap"], False, 1e-8,
     "HOMO, LUMO and gaps"),
    ("excited/summary.csv", ["delta_SCF"], False, 1e-8, "Delta-SCF energies"),
    ("thermo/thermodynamics.csv", ["mu", "E", "entropy", "F"], True, 1e-8,
     "thermodynamics"),
    ("adiabatic/adiabaticity.csv", ["Q_max"], True, 1e-6, "adiabaticity Q_max"),
]
for table, columns, relative, tolerance, what in COMPARISONS:
    largest = worst(table, columns, relative)
    report(f"largest difference, {what}", f"{largest:.1e}")
    check(largest <= tolerance, f"{what} agree with the record within {tolerance:.0e}",
          record=f"{RECORD_RESULTS}/{table}")
```

`COMPARISONS` lists five comparisons, each a tuple (a fixed group of values in parentheses): the table, its columns, whether the comparison is relative, the tolerance (the values fixed in advance in `Revision/kohn_sham/reports/ks-rust-determinism.json`) and a description. The loop unpacks each tuple into five names, computes the largest difference, reports it and checks it. Out [6] reports 0.0 for all five and five PASS lines: on this computer the new numbers are exactly the recorded ones.

In [7], the ground states along the history.

```
ground = read_table(NEW_RESULTS, "ground/summary.csv")  # the 75 ground states
excited = read_table(NEW_RESULTS, "excited/summary.csv")  # their Delta-SCF


def state_id(n, tag, a4):
    return f"N{n}_{tag}_a{round(10 * a4):02d}"
```

From here on the notebook reads the new results. `state_id` builds the solver's name of a state: `round(10 * a4)` is ten times the slice as a whole number, and the format `02d` writes it with two digits, so $N = 136$, `lam0`, $a_{4,0} = 1$ gives `N136_lam0_a10`.

```
say("   N  a4,0        E_KS        HOMO        LUMO     KS gap")
for n in (8, 136, 688):
    for a4 in SLICES:
        row = ground[state_id(n, "lam0", a4)]
        say(f"{n:4d}  {a4:4.1f}  {float(row['E_KS']):10.5f}  {float(row['HOMO']):10.6f}"
            f"  {float(row['LUMO']):10.6f}  {float(row['KS_gap']):9.6f}")
report("KS gap of N = 8 at a4,0 = 0, 1, 2", ", ".join(
    f"{float(ground[state_id(8, 'lam0', a)]['KS_gap']):.7f}" for a in (0, 1, 2)))
```

For the three particle numbers without interaction and the five slices the cell prints a table of E_{KS}, the HOMO, the LUMO and the gap, and then the gap of $N = 8$ at $a_{4,0} = 0, 1, 2$ with seven digits. Out [7] is the table of Section 15.15. For $N = 8$ the energy is exactly 0 at every slice (the zero modes have $\varepsilon = 0$ and there is no interaction), and the gaps are 0.4307337, 0.1703493, 0.0641594.

In [8], the levels along the history.

```
def read_levels(n, tag, a4):
    path = NEW_RESULTS / "ground/levels" / f"{state_id(n, tag, a4)}.csv"
    with open(path, newline="", encoding="utf-8") as handle:
        return {(int(r["n2"]), int(r["j"]), r["parity"], int(r["label"])):
                (float(r["eps"]), float(r["f"])) for r in csv.DictReader(handle)}
```

`read_levels` reads the level table of one state and returns a dictionary whose key is the **key** of a level, the four numbers (shell n_2, block type j, parity, Pruefer label l), and whose value is the pair (level, occupation).

```
levels = [read_levels(136, "lam0", a4) for a4 in SLICES]  # one dictionary per slice
THEORY = json.loads(repository_file("Revision/kohn_sham/ks-theory.json").read_text(
    encoding="utf-8"))  # the theory file of the solver
C_SLOPE = float(THEORY["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])  # c
fig, ax = plt.subplots(figsize=(7.5, 5.0))
all_keys = sorted(set().union(*levels))
```

`levels` holds the level tables of $N = 136$, $\lambda = 0$ at the five slices. `C_SLOPE` is the band slope c from the theory file. `set().union(*levels)` is the set of all keys that occur at any slice (a **set** is a collection without repetitions), sorted into a list.

```
for key in all_keys:
    points = [(a4, lv[key]) for a4, lv in zip(SLICES, levels) if key in lv]
    xs = [p[0] for p in points]
    es = [p[1][0] for p in points]
    if min(es) > 1.6:
        continue  # only the part of the spectrum near the Fermi level
    band = key[1] == 1 and key[2] == "even" and key[3] == 0 and key[0] >= 1
    colour = PALETTE[0] if band else ("black" if key[0] == 0 else "0.6")
    ax.plot(xs, es, "-", color=colour, lw=1.0, alpha=0.8)
    for x, (e, f) in points:
        ax.plot([x], [e], "o", ms=5, color=colour,
                markerfacecolor=colour if f > 0.5 else "white")
```

For each key the cell collects the slices at which the level exists (the label sets differ a little from slice to slice) with its level and occupation. Levels that stay above $1.6\,m$ are skipped (`continue` jumps to the next key). A key belongs to the brane band if it is $j = +1$, even, label 0 and $n_2 \geq 1$; band levels are drawn blue, levels at $k = 0$ black and all others grey. Each level is drawn as a thin line through its points, and each point as a dot that is filled when the level is occupied ($f > 0.5$) and open (white inside) when it is empty.

```
a_fine = np.linspace(0.0, 2.0, 101)
ax.plot(a_fine, C_SLOPE * 0.25 * np.exp(-a_fine), "--", color=PALETTE[1], lw=1.5,
        label="$c\\,k\\,e^{-a_{4,0}}$ for $k = 0.25$")
ax.plot([], [], "o-", color=PALETTE[0], label="brane band ($j=+1$, even, $l=0$)")
ax.plot([], [], "o-", color="black", label="levels at $k = 0$")
ax.plot([], [], "o-", color="0.6", label="other levels")
ax.plot([], [], "o", color="0.3", markerfacecolor="white", label="open: empty")
ax.set_ylim(-0.05, 1.6)
```

```
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("level $\\varepsilon$ (units of $m$)")
ax.set_title("Kohn-Sham levels of $N = 136$, $\\lambda = 0$, along the history")
ax.legend(fontsize=8, loc="center right")
save_figure(fig, "levels_history",
            "Kohn-Sham levels $\\varepsilon$ (vertical axis, units of $m$) of the "
...)
```

`a_fine` holds 101 slices from 0 to 2; the dashed orange curve is the small-k law $c\,k\,e^{-a_{4,0}}$ for the first shell. The empty plots make the legend entries, and the remaining lines set the range, labels and title and save Figure 15a.1.

```
k0_keys = [key for key in levels[0] if key[0] == 0]
k0_spread = max(abs(lv[key][0] - levels[0][key][0]) for lv in levels for key in k0_keys)
band_keys = [key for key in all_keys if all(key in lv for lv in levels)
             and key[1] == 1 and key[2] == "even" and key[3] == 0 and key[0] >= 1]
decreasing = all(levels[i + 1][key][0] < levels[i][key][0]
                 for key in band_keys for i in range(4))
report("number of brane-band levels followed through all five slices", len(band_keys))
check(k0_spread <= 1e-12, "the k = 0 levels are the same at every slice")
check(decreasing, "every brane-band level decreases strictly along the history")
```

`k0_spread` is the largest change of a level at $k = 0$ between the first slice and any other. `band_keys` are the band levels present at all five slices, and `decreasing` is true when each of them is lower at each slice than at the previous one (`all(...)` is true when every comparison is true). Out [8]: 8 band levels are followed through all slices, the $k = 0$ levels do not move, and every band level falls; both are consequences of the rescaling identity (Section 15.4). In Figure 15a.1 the lowest blue level (the first shell) lies below the dashed curve at $a_{4,0} = 0$, where the momentum 0.25 is not yet small enough for the straight-line law, and approaches it along the history as the redshifted momentum shrinks.

In [9], the gaps.

```
fig, ax = plt.subplots()
for colour, n in zip(PALETTE, (8, 136, 688)):
    gaps = [float(ground[state_id(n, "lam0", a4)]["KS_gap"]) for a4 in SLICES]
    ax.plot(SLICES, gaps, "o-", color=colour, lw=1.5, label=f"$N = {n}$")
    ax.plot(a_fine, gaps[0] * np.exp(-a_fine), "--", color=colour, lw=1.0)
ax.set_yscale("log")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("Kohn-Sham gap $\\Delta_{KS}$ (units of $m$)")
ax.set_title("The gap closes along the history (dashed: $\\propto e^{-a_{4,0}}$)")
ax.legend()
save_figure(fig, "gaps_history",
            "Kohn-Sham gap $\\Delta_{KS}$ (vertical axis, logarithmic, units of $m$) "
...)
```

For each particle number the gaps at the five slices are drawn as dots joined by lines, with a dashed curve $\Delta(0)\,e^{-a_{4,0}}$ through the first point; `set_yscale("log")` makes the vertical axis logarithmic, on which $e^{-a_{4,0}}$ is a straight line. This is Figure 15a.2.

```
free_dscf = max(abs(float(excited[state_id(n, "lam0", a)]["delta_SCF_minus_gap"]))
                for n in (8, 136, 688) for a in SLICES)
report("largest |Delta-SCF - gap| without interaction", f"{free_dscf:.1e}")
check(free_dscf <= 1e-10, "Delta-SCF equals the gap without interaction",
      record=f"{RECORD_REPORT}, check excited_delta_scf_free_equals_gap")
shrinking = all(float(ground[state_id(n, "lam0", SLICES[i + 1])]["KS_gap"])
                < float(ground[state_id(n, "lam0", SLICES[i])]["KS_gap"])
                for n in (8, 136, 688) for i in range(4))
check(shrinking, "every gap shrinks from slice to slice")
```

`free_dscf` is the largest $|\Delta_{SCF} - \Delta_{KS}|$ of the 15 free states; Out [9] gives 2.0×10^{-11}, the value of the record's check, below its tolerance 10^{-10}. The second check confirms that every gap shrinks from slice to slice.

In [10], the orbital relaxation.

```
fig, axes = plt.subplots(1, 3, figsize=(10.0, 3.8), sharey=True,
                         layout="constrained")
for ax, n in zip(axes, (8, 136, 688)):
    for colour, tag, label in zip(PALETTE, ("lamp1", "lamm1", "lamp2", "lamm2"),
                                  ("$+\\lambda_1$", "$-\\lambda_1$", "$+\\lambda_2$",
                                   "$-\\lambda_2$")):
        diff = [float(excited[state_id(n, tag, a)]["delta_SCF_minus_gap"])
                for a in SLICES]
        ax.plot(SLICES, diff, "o-", color=colour, lw=1.5, label=label)
    ax.set_yscale("symlog", linthresh=1e-9)
    ax.set_title(f"$N = {n}$")
    ax.set_xlabel("slice $a_{4,0}$")
axes[0].set_ylabel("$\\Delta_{SCF} - \\Delta_{KS}$ (units of $m$)")
axes[0].legend(fontsize=8)
```

Three panels, one per particle number; in each, the relaxation $\Delta_{SCF} - \Delta_{KS}$ of the four interacting couplings at the five slices. The **symmetric logarithmic** axis (`"symlog"`) is logarithmic for large values of either sign and linear between -10^{-9} and 10^{-9} (`linthresh`), so that small values of both signs fit on one axis.

```
largest_id = max(excited, key=lambda i: abs(float(excited[i]["delta_SCF_minus_gap"])))
largest_relax = abs(float(excited[largest_id]["delta_SCF_minus_gap"]))
mantissa, power = f"{largest_relax:.1e}".split("e")  # e.g. "5.5", "-04"
save_figure(fig, "delta_scf",
            "Orbital relaxation: the Delta-SCF excitation energy minus the Kohn-Sham "
...)
report("largest |Delta-SCF - gap| over the 75 states",
       f"{largest_relax:.3e} ({largest_id})")
check(largest_id == "N688_lamp2_a00",
      "the largest relaxation belongs to N = 688, +lambda_2, a4,0 = 0 (the caption)")
check(largest_relax < 1e-3, "the orbital relaxation is below 0.001 m in every state")
```

`max(excited, key=...)` returns the state name whose relaxation has the largest size. The number is written in the format `.1e` and split at the letter e into the digits and the power of ten, which the

caption of Figure 15a.3 prints as 5.5×10^{-4} (so the caption's number is computed, not typed). Out [10]: the largest relaxation is $5.496 \times 10^{-4}\,m$, in `N688_lamp2_a00`, where the lowest empty level is the bulk level at $k = 0$, and every relaxation is below $0.001\,m$.

In [11], the total energy.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
for colour, n in zip(PALETTE, (136, 688)):
    energies = [float(ground[state_id(n, "lam0", a)]["E_KS"]) for a in SLICES]
    left.plot(SLICES, energies, "o-", color=colour, lw=1.5, label=f"$N = {n}$")
left.set_yscale("log")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("$E_{KS}$ (units of $m$)")
left.set_title("Energy without interaction")
left.legend()
```

The left panel draws E_{KS} of the free states $N = 136$ and 688 at the five slices on a logarithmic axis.

```
tags = ("lamp2", "lamp1", "lamm1", "lamm2")
labels = ("$+\\lambda_2$", "$+\\lambda_1$", "$-\\lambda_1$", "$-\\lambda_2$")
largest_shift = 0.0  # the largest |E(lambda) - E(0)| of N = 136
for colour, tag, label in zip(PALETTE, tags, labels):
    shift = [float(ground[state_id(136, tag, a)]["E_KS"])
             - float(ground[state_id(136, "lam0", a)]["E_KS"]) for a in SLICES]
    largest_shift = max([largest_shift] + [abs(s) for s in shift])
    right.plot(SLICES, shift, "o-", color=colour, lw=1.5, label=label)
e136 = [float(ground[state_id(136, "lam0", a)]["E_KS"]) for a in SLICES]
right.axhline(0.0, color="0.4", lw=0.8)
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$E_{KS}(\\lambda) - E_{KS}(0)$ (units of $m$)")
right.set_title("Interaction energy shift, $N = 136$")
right.legend(fontsize=8)
save_figure(fig, "energy_history",
            "Left: the Kohn-Sham energy $E_{KS}$ (vertical axis, logarithmic, units "
...)
```

The right panel draws, for $N = 136$, the change $E_{KS}(\lambda) - E_{KS}(0)$ of the four couplings at the five slices, and keeps the largest size of these changes in `largest_shift` (the list `[largest_shift] + [...]` joins the old largest value with the new sizes). `e136` are the free energies of $N = 136$; the caption of Figure 15a.4 prints their smallest and largest value and `largest_shift`.

```
report("largest |E(lambda) - E(0)| of N = 136", f"{largest_shift:.4f}")
ordered = all(
    float(ground[state_id(n, "lamp2", a)]["E_KS"])
    > float(ground[state_id(n, "lamp1", a)]["E_KS"])
    > float(ground[state_id(n, "lam0", a)]["E_KS"])
    > float(ground[state_id(n, "lamm1", a)]["E_KS"])
    > float(ground[state_id(n, "lamm2", a)]["E_KS"])
    for n in (136, 688) for a in SLICES)
check(ordered, "E(+lambda2) > E(+lambda1) > E(0) > E(-lambda1) > E(-lambda2), "
               "N = 136 and 688")
```

```
falling = all(float(ground[state_id(n, "lam0", SLICES[i + 1])]["E_KS"])
              < float(ground[state_id(n, "lam0", SLICES[i])]["E_KS"])
              for n in (136, 688) for i in range(4))
check(falling, "the energy of N = 136 and 688 falls from slice to slice")
```

Out [11]: the largest shift is $0.0318\,m$, out of energies from 12 to $80\,m$. Python allows a chain of comparisons, `a > b > c`, which is true when every neighbouring pair is ordered; the first check confirms that repulsion raises and attraction lowers the energy, more for the stronger coupling, at every slice for $N = 136$ and 688. The second confirms that the free energies fall from slice to slice. For $N = 8$ the order is reversed (the text cell before the code cell explains why, and Section 15.5 gives the exact symmetry).

In [12], the densities.

```
VOL7 = (2.0 * math.pi / 0.25) ** 3  # proper 7-volume per unit e^{6Hy} (v_t = 1)


def read_profile(n, tag, a4):
    path = NEW_RESULTS / "ground/profiles" / f"{state_id(n, tag, a4)}.csv"
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}
```

`read_profile` reads the profile table of a state with `np.genfromtxt`, which turns a CSV file into an array whose columns can be addressed by the names in the first line (`names=True`); the function returns a dictionary from each column name (`y`, `n`, `S`, `M_eff`, `v_v`, `rho`, `p3`, ...) to its array of 151 values.

```
def simpson(values, step):
    weights = np.full(len(values), 2.0)
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    return step / 3.0 * float(np.sum(weights * values))
```

`simpson` is Simpson's rule for an odd number of equally spaced values: weights 1, 4, 2, 4, ..., 4, 1 times $h/3$ (`np.full(n, 2.0)` is an array of n twos).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
counts = []
for shade, a4 in zip((SLICE_SHADES[0], SLICE_SHADES[2], SLICE_SHADES[4]), (0, 1, 2)):
    prof = read_profile(136, "lam0", a4)
    y = prof["y"]
    per_y = 2.0 * VOL7 * np.exp(6.0 * y) * prof["n"]  # particles per unit y
    counts.append(simpson(per_y, y[1] - y[0]))
    left.plot(y, prof["n"], color=shade, lw=1.8, label=f"$a_{{4,0}} = {a4}$")
    right.plot(y, per_y, color=shade, lw=1.8, label=f"$a_{{4,0}} = {a4}$")
```

For $N = 136$, $\lambda = 0$ at the slices 0, 1, 2 the cell reads the profile, forms the number of particles per unit y in the doubled box, $2\,\mathrm{Vol}_7\, e^{6Hy} n(y)$, integrates it with Simpson's rule (the spacing is `y[1] - y[0]` $= 0.02$), and draws the proper density on the left and the particles per unit y on the right.

```
left.set_yscale("log")
left.set_xlabel("hidden coordinate $y$ (tip at $-3$, brane at $0$)")
left.set_ylabel("proper density $n(y)$")
left.set_title("Proper number density")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("particles per unit $y$")
right.set_title("$2\\,\\mathrm{Vol}_7\\, e^{6Hy} n(y)$, area $= N$")
right.legend()
save_figure(fig, "densities",
            "Left: the proper number density $n(y)$ (vertical axis, logarithmic, "
...)
report("particle number from the profiles at a4,0 = 0, 1, 2",
       ", ".join(f"{c:.6f}" for c in counts))
check(max(abs(c - 136.0) for c in counts) < 1e-5,
      "Simpson's rule on the profiles gives N = 136 at every slice",
      record=f"{RECORD_REPORT}, check ground_N_conservation")
```

After the labels, `save_figure` saves Figure 15a.5. Out [12] gives 136.000002 at all three slices: the integral of the coarse 151-point profile reproduces N to 2×10^{-6} (the solver's own check ground_N_conservation, on its fine grid, gives 1.5×10^{-15}).

In [13], the self-consistent potentials.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
ratios = []  # profile maximum / recorded maximum, one per slice
for shade, a4 in zip(SLICE_SHADES, SLICES):
    prof = read_profile(136, "lamp2", a4)
    left.plot(prof["y"], prof["M_eff"] - 1.0, color=shade, lw=1.8,
              label=f"$a_{{4,0}} = {a4}$")
    right.plot(prof["y"], prof["v_v"], color=shade, lw=1.8)
    recorded = float(ground[state_id(136, "lamp2", a4)]["max_abs_Meff_minus_m"])
    ratios.append(float(np.max(np.abs(prof["M_eff"] - 1.0))) / recorded)
```

For $N = 136$ with $+\lambda_2$ the cell draws $M - m$ (left) and v (right) at the five slices, and for each slice divides the largest $|M - m|$ on the 151 profile points by the largest value that the solver recorded on its 1801 fine points.

```
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("$M(y) - m$ (units of $m$)")
left.set_title("Mass shift, $N = 136$, $+\\lambda_2$")
left.legend(fontsize=8)
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("$v(y)$ (units of $m$)")
right.set_title("Exchange potential, $N = 136$, $+\\lambda_2$")
m_max = [float(ground[state_id(136, "lamp2", a4)]["max_abs_Meff_minus_m"])
         for a4 in SLICES]  # the solver's largest |M - m| at each slice
v_max = [float(ground[state_id(136, "lamp2", a4)]["max_abs_v_v"]) for a4 in SLICES]
peak = int(np.argmax(m_max))  # the slice where |M - m| is largest
save_figure(fig, "potentials",
            "The self-consistent mass shift $M(y) - m$ (left) and potential $v(y)$ "
...)
```

`m_max` and `v_max` are the recorded largest $|M - m|$ and $|v|$ at the five slices, and `np.argmax` gives the position of the largest entry of `m_max`. The caption of Figure 15a.6 prints these numbers.

```
report("profile maximum / recorded maximum of |M - m|",
       ", ".join(f"{r:.5f}" for r in ratios))
check(all(0.998 <= r <= 1.0 + 1e-12 for r in ratios),
      "the profile maxima of |M - m| lie within 0.2 percent below the recorded ones",
      record=f"{RECORD_RESULTS}/ground/summary.csv, column max_abs_Meff_minus_m")
report("largest |M - m| at the five slices", ", ".join(f"{v:.3f}" for v in m_max))
report("largest |v| at the five slices", ", ".join(f"{v:.3f}" for v in v_max))
check(all(v_max[i + 1] > v_max[i] for i in range(4)) and SLICES[peak] == 1.5
      and max(m_max + v_max) < 0.4,
      "|v| grows at every slice, |M - m| peaks at a4,0 = 1.5, both stay below 0.4 m",
      record=f"{RECORD_RESULTS}/ground/summary.csv, columns max_abs_Meff_minus_m "
             "and max_abs_v_v")
```

Every profile point is also a point of the fine grid, so the profile maximum cannot exceed the recorded one; it is smaller when the true maximum lies between two profile points. Out [13]: the ratios lie between 0.99891 and 0.99997; the largest $|M - m|$ is 0.008, 0.050, 0.192, 0.355, 0.310 m and the largest $|v|$ is 0.015, 0.021, 0.059, 0.157, 0.242 m at the five slices. Both checks pass (`m_max + v_max` joins the two lists into one).

In [14], the convergence of the loop.

```
runs = json.loads((NEW_RESULTS / "ground/runs.json").read_text(encoding="utf-8"))
by_id = {run["id"]: run for run in runs}
fig, ax = plt.subplots()
styles = {"lamp2": "-", "lamm2": "--"}
for colour, n in zip(PALETTE, (8, 136, 688)):
    for tag, style in styles.items():
        history = by_id[state_id(n, tag, 2.0)]["scfHistory_iteration_residual_E"]
        ax.plot([h[0] for h in history], [h[1] for h in history], style, marker="o",
                ms=4, color=colour, lw=1.5,
                label=f"$N = {n}$, {'+' if tag == 'lamp2' else '-'}$\\lambda_2$")
```

`runs.json` is a list with one entry per ground state; `by_id` files them under their names. For the strongest couplings $\pm\lambda_2$ at the last slice (the hardest cases) the residual of every iteration is drawn, solid for $+\lambda_2$ and dashed for $-\lambda_2$.

```
ax.axhline(1e-11, color="0.3", ls=":", lw=1.2, label="tolerance $10^{-11}$")
ax.set_yscale("log")
ax.set_xlabel("iteration")
ax.set_ylabel("residual: largest change of the potential (units of $m$)")
ax.set_title("Anderson mixing, $a_{4,0} = 2$, couplings $\\pm\\lambda_2$")
ax.legend(fontsize=8, ncol=2)
save_figure(fig, "scf_convergence",
            "Convergence of the self-consistent loop with Anderson mixing: the "
...)
direct = all(run["path"] == "direct" for run in runs)
final = max(run["scfHistory_iteration_residual_E"][-1][1] for run in runs)
longest = max(run["iterations"] for run in runs)
```

```
report("largest final residual / largest number of iterations",
       f"{final:.2e} / {longest}")
check(len(runs) == 75 and direct and final <= 1e-11,
      "all 75 ground states converged directly below the tolerance 1e-11",
      record=f"{RECORD_REPORT}, check ground_scf_converged")
```

After the tolerance line and the labels (`ncol=2` puts the legend in two columns), `save_figure` saves Figure 15a.7. `direct` is true when every run took the path `direct` (no fallback was needed), `final` is the largest last residual and `longest` the largest number of iterations. Out [14]: 9.34×10^{-12} and 17, and the check passes. In the figure the curves of $N = 8$ for $+\lambda_2$ and $-\lambda_2$ coincide, as the exact symmetry of Section 15.5 predicts.

In [15], the particle-hole excitations.

```
TAGS = ("lam0", "lamp1", "lamm1", "lamp2", "lamm2")  # the five couplings


def read_pairs(n, tag, a4):
    path = NEW_RESULTS / "excited/particle-hole" / f"{state_id(n, tag, a4)}.csv"
    with open(path, newline="", encoding="utf-8") as handle:
        return {(r["hole_levels"], r["particle_levels"]):
                (float(r["delta_eps"]), float(r["multiplicity"]), r["same_sector"])
                for r in csv.DictReader(handle)}


lists = {(n, tag, a4): read_pairs(n, tag, a4)
         for n in (8, 136, 688) for tag in TAGS for a4 in SLICES}  # 75 states
```

`read_pairs` reads the list of the lowest particle-hole excitations of a state: the key is the pair (hole levels, particle levels), each written `n2:j:parity:label`, and the value is the excitation energy $\Delta\varepsilon = \varepsilon_{particle} - \varepsilon_{hole}$, the multiplicity $g_{hole}\, g_{particle}$ and the word `true` or `false` that says whether the two levels lie in the same sector. `lists` holds these lists for all 75 ground states.

```
fig, ax = plt.subplots()
for a4 in SLICES:
    values = list(lists[(136, "lam0", a4)].values())
    ax.scatter([a4] * len(values), [v[0] for v in values],
               s=[v[1] / 40.0 for v in values],  # marker area ~ multiplicity
               color=PALETTE[0], alpha=0.3, edgecolors=PALETTE[0])
lowest = [min(v[0] for v in lists[(136, "lam0", a4)].values()) for a4 in SLICES]
ax.plot(SLICES, lowest, "o-", color=PALETTE[1], lw=1.8, ms=5,
        label="lowest excitation (the Kohn-Sham gap)")
ax.plot(a_fine, lowest[0] * np.exp(-a_fine), "--", color="0.4", lw=1.0,
        label="$\\propto e^{-a_{4,0}}$")
```

`ax.scatter` draws markers whose areas are given by `s`: here the multiplicity divided by 40, so a larger marker is a more degenerate excitation. For $N = 136$ without interaction all listed excitations are drawn at their slice, the lowest excitation of each slice is joined by an orange line, and a dashed curve falls like $e^{-a_{4,0}}$ from the first one.

```
ax.scatter([], [], s=60, color=PALETTE[0], alpha=0.3, edgecolors=PALETTE[0],
           label="excitations (area: multiplicity)")
ax.set_yscale("log")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("excitation energy $\\Delta\\varepsilon$ (units of $m$)")
ax.set_title("Particle-hole excitations of $N = 136$, $\\lambda = 0$")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "particle_hole",
            "The lowest particle-hole excitation energies (vertical axis, "
...)
```

A legend entry, the axes and `save_figure` for Figure 15a.8.

```
lowest_is_gap = all(
    abs(min(v[0] for v in lists[key].values())
        - float(ground[state_id(*key)]["KS_gap"])) <= 1e-12 for key in lists)
cheaper = True
for n in (8, 136, 688):
    for tag in TAGS:
        series = [lists[(n, tag, a4)] for a4 in SLICES]
        common = set(series[0]).intersection(*series[1:])  # listed at every slice
        cheaper = cheaper and all(series[i + 1][key][0] < series[i][key][0]
                                  for key in common for i in range(4))
```

`lowest_is_gap` is true when in every state the lowest listed excitation equals the Kohn-Sham gap (`state_id(*key)` unpacks the triple $(N, \text{tag}, a_{4,0})$ into the three arguments). For each of the 15 series (particle number and coupling), `common` is the set of excitations listed at all five slices (`intersection` keeps what all sets share), and `cheaper` stays true only if each of them is cheaper at each slice than at the previous one.

```
count = sum(len(pairs) for pairs in lists.values())
inside = sum(1 for pairs in lists.values() for v in pairs.values() if v[2] != "false")
jumps = [float(row["Q_max_delta_eps"]) for row in
         read_table(NEW_RESULTS, "adiabatic/adiabaticity.csv").values()
         if row["Q_max_delta_eps"] != "null"]  # null: no jump (N = 8)
report("particle-hole excitations listed / inside one sector", f"{count} / {inside}")
report("lowest excitation of N = 136 at a4,0 = 0 and 2",
       f"{lowest[0]:.5f}, {lowest[-1]:.5f}")
report("energy of the largest-Q jump inside a sector (smallest, largest)",
       f"{min(jumps):.3f}, {max(jumps):.3f}")
```

`count` is the number of listed excitations of all 75 states, and `inside` the number of those whose two levels lie in one sector. `jumps` collects, from the adiabaticity table, the energy of the jump with the largest adiabaticity number Q in each state (Section 15.21); for $N = 8$ there is no such jump and the table says `null`. Out [15]: 1610 excitations, none inside one sector; the lowest excitation of $N = 136$ falls from 0.08267 to 0.01446; the in-sector jumps cost between 1.488 and 2.253 m.

```
check(lowest_is_gap, "in all 75 states the lowest particle-hole energy is the KS gap",
      record=f"{RECORD_RESULTS}/excited/particle-hole and ground/summary.csv")
```

```
check(cheaper, "every excitation listed at all five slices gets cheaper along the "
               "history")
check(count >= len(lists) and inside == 0,
      f"none of the {count} listed excitations stays inside one sector",
      record=f"{RECORD_RESULTS}/excited/particle-hole, column same_sector")
```

The three checks pass. The meaning of the third: the exact evolution along the history keeps the momentum, the block type and the parity of every orbital (Section 15.21), so the motion of the background alone cannot create any of the 1610 excitations; it can only cause jumps inside a sector, and the jump with the largest Q costs between $1.488\,m$ and $2.253\,m$.

In [16], the last check.

```
NAMES = ["levels_history", "gaps_history", "delta_scf", "energy_history",
         "densities", "potentials", "scf_convergence",
         "particle_hole"]  # the figures, in order
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15a_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

The same as In [15] of Notebook 15e (Section 15.9), with the eight figure names of this notebook in order (the comment says so) and the file names `15a_<number>_<name>.png`: `enumerate(NAMES, start=1)` pairs each name with its number, the list comprehension collects the names whose file does not exist, the check requires that list to be empty, and `all_checks_passed()` prints the last line. The cell prints PASS every figure file of this notebook exists and ALL 25 CHECKS PASSED (notebook 15a).

15.15 What the canonical matrix shows

All numbers of this section are COMPUTED by the Rust solver and reproduced by Notebook 15a; the cell is named with each.

The ground states without interaction (Notebook 15a, Out [7]; `Revision/kohn_sham/results/ground/summary.csv`):

N	$a_{4,0}$	E_{KS}	**HOMO**	**LUMO**	Δ_{KS}
8	0	0	0	0.430734	0.430734
8	1	0	0	0.170349	0.170349
8	2	0	0	0.064159	0.064159
136	0	80.28222	0.799646	0.882320	0.082674
136	1	32.38412	0.325768	0.360757	0.034989
136	2	12.44507	0.126766	0.141223	0.014457
688	0	680.44125	1.247113	1.292293	0.045180
688	1	279.42489	0.515229	0.535712	0.020484
688	2	110.38667	0.205760	0.214370	0.008610

How to read it. For $N = 8$ the eight particles sit in the zero modes at $\varepsilon = 0$, so $E_{KS} = 0$ at every slice, and the gap is the first band level (Section 15.4). For $N = 136$ the HOMO is the band level of

the shell $n_2 = 4$ and the LUMO that of $n_2 = 5$; both redshift, and the energy, the sum of $g\varepsilon$ over the occupied band levels, falls from 80.28 to 12.45, by a factor 6.45. For $N = 688$ at $a_{4,0} = 0$ the LUMO is 1.292293, the bulk level at $k = 0$, which does not move; at later slices the band levels of the shells 12 and higher have come down below it, and the LUMO is again a band level. (Section 15.21 shows that a filling of $N = 696$ would cross the Fermi level for this reason.)

The gaps close more slowly than $e^{-a_{4,0}}$. Figure 15a.2 shows all gaps falling, each a little more slowly than its dashed line. The reason is the bending of the band (Section 15.4 and Exercise 2): a gap is the distance between two band levels, and a band level falls exactly like $e^{-a_{4,0}}$ only where it is a straight line, at small redshifted momenta.

Delta-SCF and the orbital relaxation. Without interaction $\Delta_{SCF} = \Delta_{KS}$ to $2.0 \times 10^{-11}\, m$ (Out [9]). With interaction the relaxation $\Delta_{SCF} - \Delta_{KS}$ is at most $5.5 \times 10^{-4}\, m$ (Out [10], `N688_lam p2_a00`), in every state far below the gap itself: the first excited state is well described by moving one particle between frozen orbitals.

The interaction is a perturbation. For $N = 136$ the couplings change the energy by at most $0.0318\, m$ (Out [11]), and the self-consistent potentials stay below $0.4\, m$ (Out [13]). The potentials grow along the history (the largest $|v|$ from 0.015 to $0.242\, m$ for $+\lambda_2$), as the calibration of Section 15.2 anticipated, and they live near the tip, where the proper densities are large (Figure 15a.6). All this holds at the cutoff $L = 3$, where the couplings were calibrated; at larger L the interaction effects grow: for $N = 136$ at $a_{4,0} = 2$ the shift of the energy by $-\lambda_1$ is at large L about 21 times its size at $L = 3$, and the shift by $+\lambda_1$ is at $L = 5.5$, the last L before its iteration fails, about 28 times its size at $L = 3$; its limit is not established (Section 15.2, the paragraph on the cutoff). The self-consistent loop converged directly in all 75 states in at most 17 iterations (Out [14]).

Where the particles are. Figure 15a.5 makes the most important picture of the gas visible: the particles themselves sit near the brane (right panel), but the proper density, particles per proper 7-volume, is largest at the tip (left panel), because the proper volume factor e^{6Hy} is tiny there ($e^{-18} = 1.5 \times 10^{-8}$ at $y = -3$). Along the history the redshifted band orbitals reach further toward the tip.

Excitations. In every state the cheapest way to excite the gas is the Kohn-Sham gap, all listed excitations get cheaper along the history, and none of them can be caused by the history alone (Out [15]); the next sections ask how much energy the history does put into the gas (Section 15.16) and how strongly it can disturb it (Section 15.21).

What this does not show. These are instantaneous states on a PRESCRIBED background, with the ASSUMED brane and the CONVENTION of filling. Whether a real gas follows them along the history is the question of adiabaticity (Section 15.21); whether the gas could make the history itself is answered in Section 15.16: it could not.

15.16 The energy-momentum tensor of the gas, and why the gas cannot drive the history

What the tensor is. Chapter 9 introduced the **energy-momentum tensor** $T^{\mu}{}_{\nu}$: at every point a table of 8×8 numbers that says how much energy and momentum there is and how it flows. For the Kohn-Sham states it is diagonal, and its diagonal holds the **energy density** $\rho = -T^{x_4}{}_{x_4}$ and the **pressures** $p_\mu = T^{\mu}{}_{\mu}$ (no sum over μ; the sign convention of the Revision record). Because of the symmetries of the states only four different numbers occur at each y: p_3, the pressure along

each of the three directions of 3-space; p_t, the pressure along each of the three extra times; p_0, the pressure along the hidden direction; and ρ. They are proper densities (per proper 7-volume) and depend on y.

The formulas (ks-theory.json, emt; checks emt_orbital_components and emt_trace_identity). With the orbital densities $n_o = P(a^2+b^2)$, $s_o = P\,j\,2ab$ and $t_o = P\,j\,(a^2-b^2)$ (the last one is the current along the momentum; $P = e^{-6Hy}/\mathrm{Vol}_7$ as in Section 15.2) and sums over the occupied levels with the weights wgf:

$$\rho = \sum wgf\,\varepsilon\,n_o - e_{int}, \qquad p_3 = \sum wgf\,\tfrac{\kappa k}{3}\,t_o + e_{int}, \qquad p_t = e_{int},$$

$$p_0 = \sum wgf\,[(\varepsilon - v)\,n_o - M s_o - \kappa k\,t_o] + e_{int}.$$

Three remarks make these formulas plausible. (1) The energy density is the level times the number density, summed, minus the interaction energy density (the same double counting as in Section 15.5). (2) A quantum moving along x_1 pushes on a wall across x_1 with $\kappa k\,t_o$; a closed shell of momenta contains every direction equally often (a cube has the same number of vectors along each axis), so the pressure along each of the three directions is one third of it. (3) The states belong to the **good sector**: they have no momentum along the extra times, so the extra-time pressure has no kinetic part at all, and only the interaction contributes, $p_t = e_{int}$. Without interaction $p_t = 0$.

The conservation law along the hidden direction. The conservation law of the tensor states that its covariant divergence vanishes, $\nabla_\mu T^\mu{}_\nu = 0$ (Chapter 9). In flat spacetime it says that energy and momentum are neither created nor destroyed, they only flow. In the author's metric, which depends on the time x_4 through a_4, it is a local balance law: the energy of the gas is not conserved, it changes along the history (the energy-change law below; for a universe, Section 20.22 (d)). We write out its y component for a diagonal tensor that depends only on y, line by line. The covariant divergence of a mixed tensor is

$$\nabla_\mu T^\mu{}_\nu = \partial_\mu T^\mu{}_\nu + \Gamma^\mu{}_{\mu\sigma} T^\sigma{}_\nu - \Gamma^\sigma{}_{\mu\nu} T^\mu{}_\sigma,$$

with the Christoffel symbols Γ (Chapters 3 and 9; summed over repeated indices). For a diagonal metric Chapter 9 gives $\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu})$ (no sum over λ).

$$\nabla_\mu T^\mu{}_y = \partial_y T^y{}_y + \Gamma^\mu{}_{\mu y}\,T^y{}_y - \sum_\nu \Gamma^\nu{}_{\nu y}\,T^\nu{}_\nu.$$

Rule: set $\nu = y$; for a diagonal tensor only $\sigma = y$ survives in the middle term and only $\sigma = \mu$ in the last, and only the y derivative of $T^y{}_y$ is nonzero. (The index ν in the sum runs over the eight directions; the letter λ is kept for the coupling.)

$$\Gamma^\mu{}_{\mu y} = \partial_y \ln\sqrt{|g|} = \partial_y(6Hy) = 6H.$$

Rule: for a diagonal metric $\Gamma^\mu{}_{\mu y} = \sum_\mu \frac{1}{2g_{\mu\mu}}\partial_y g_{\mu\mu} = \frac{1}{2}\sum_\mu \partial_y \ln|g_{\mu\mu}| = \frac{1}{2}\partial_y \ln|\det g| = \partial_y \ln\sqrt{|g|}$ (the formula above with $\lambda = \mu$ and $\nu = y$, where the first and the last term cancel; $\partial \ln Q = \partial Q/Q$; the logarithm of a product is the sum of the logarithms), and $\sqrt{|g|} = e^{6Hy}$ (Chapter 14).

$$\Gamma^{\nu}{}_{\nu y} = \tfrac{1}{2} g^{\nu\nu}\partial_y g_{\nu\nu} = \tfrac{1}{2}\partial_y \ln|g_{\nu\nu}| = H \quad (\nu = x_1, x_2, x_3, x_5, x_6, x_7), \qquad \Gamma^{x_4}{}_{x_4 y} = \Gamma^{y}{}_{yy} = 0.$$

Rule: the same formula for one value of ν (no sum) gives half the derivative of $\ln|g_{\nu\nu}|$; the six warped directions carry the factor e^{2Hy} in the line element $ds^2 = e^{2Hy}[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 + dx_7^2)] - dx_4^2 + dy^2$, and $\tfrac{1}{2}\partial_y \ln e^{2Hy} = H$; $g_{x_4x_4} = -1$ and $g_{yy} = 1$ are constant.

$$p_0' + 6H\,p_0 - 3H\,p_3 - 3H\,p_t = 0.$$

Rule: $T^{y}{}_{y} = p_0$; the six warped terms give H times p_3 three times and H times p_t three times.

$$(e^{6Hy}p_0)' = 3H\,e^{6Hy}\,(p_3 + p_t).$$

Rule: $(e^{6Hy}p_0)' = e^{6Hy}(p_0' + 6Hp_0)$ (product and chain rule); multiply the previous line by e^{6Hy}. This is the **conservation law along** y (ks-theory.json, emt.conservationY). Integrated from the tip to the brane it reads $[e^{6Hy}p_0]_{-L}^{0} = 3H\int_{-L}^{0} e^{6Hy}(p_3 + p_t)\,dy$. It is PROVED for every self-consistent state (check emt_y_conservation_selfconsistent of `Revision/kohn_sham/reports/ks-theory-python.json`: the force terms $-M'S - v'n$ of the orbitals cancel exactly against the derivative of e_{int} when the potentials are the self-consistent ones) and COMPUTED for all 75 ground states of the record (checks emt_y_conservation_pointwise, worst relative residual 1.95×10^{-8}, and emt_y_conservation_integrated, worst 1.42×10^{-11}, of `Revision/kohn_sham/reports/ks-rust-solver.json`).

The energy change along the history. Now the x_4 component. The metric depends on x_4 through $a_4(x_4)$, and the states are instantaneous states that follow it. Line by line:

$$\nabla_\mu T^{\mu}{}_{x_4} = \partial_{x_4} T^{x_4}{}_{x_4} + \Gamma^{\mu}{}_{\mu x_4} T^{x_4}{}_{x_4} - \sum_{\nu} \Gamma^{\nu}{}_{\nu x_4} T^{\nu}{}_{\nu}.$$

Rule: as before with y replaced by x_4, now with the time derivative of $T^{x_4}{}_{x_4}$ (the flux $T^{y}{}_{x_4}$ along y vanishes for the instantaneous states, ks-theory.json emt.offDiagonal).

$$\Gamma^{\mu}{}_{\mu x_4} = \partial_{x_4} \ln\sqrt{|g|} = 0.$$

Rule: $\sqrt{|g|} = e^{6Hy}$ does not depend on a_4: 3-space grows like e^{3a_4} and the three extra times shrink like e^{-3a_4}, so the proper 7-volume stays the same (check geometry_sqrt_det).

$$\Gamma^{\nu}{}_{\nu x_4} = \tfrac{1}{2}\partial_{x_4} \ln|g_{\nu\nu}| = +a_4' \ (\nu = x_1, x_2, x_3), \quad -a_4' \ (\nu = x_5, x_6, x_7), \quad 0 \ (\nu = x_4, y).$$

Rule: the inflating directions carry e^{2a_4} and the deflating extra times e^{-2a_4}; $\tfrac{1}{2}\partial_{x_4} \ln e^{\pm 2a_4} = \pm a_4'$, with $a_4' = da_4/dx_4$.

$$-\partial_{x_4}\rho - (3a_4'\,p_3 - 3a_4'\,p_t) = 0, \qquad \partial_{x_4}\rho = -3a_4'\,(p_3 - p_t).$$

Rule: $T^{x_4}{}_{x_4} = -\rho$; insert the symbols; then move the bracket to the right side.

$$\frac{\partial \rho}{\partial a_4} = -3\,(p_3 - p_t), \qquad \frac{dE}{da_4} = -3 \cdot 2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy}\,(p_3 - p_t)\,dy.$$

Rule: along the history $\partial_{x_4} = a_4'\,\partial/\partial a_4$ (chain rule), divide by a_4'; then multiply by $2\,\mathrm{Vol}_7 e^{6Hy}$ and integrate over y, with $E = 2\,\mathrm{Vol}_7 \int e^{6Hy}\rho\,dy = E_{KS}$ (check emt_energy_integral of the solver: relative difference at most 1.6×10^{-15} in all 75 states). This is the **energy-change law** (ks-theory.json, emt.energyChange; PROVED, check emt_x4_component; COMPUTED for all 75 states against differences of the self-consistent energies at neighbouring slices with fixed occupations, check emt_energy_change_dE_da4, worst 1.5×10^{-10}).

How to read it: work. Write the law as $dE = -3\,P_3\,da_4 + 3\,P_t\,da_4$ with the integrated pressures $P_3 = 2\,\mathrm{Vol}_7 \int e^{6Hy} p_3\,dy$ and P_t likewise. When 3-space inflates by da_4, each of its three directions stretches by the factor e^{da_4}, and a gas that pushes outward with the pressure p_3 does work on the expanding space and loses that energy, as a gas in an expanding cylinder pushes a piston. When the three extra times deflate by the same amount, the gas gains $3p_t\,da_4$ per unit volume. Without interaction $p_t = 0$, so the free gas only loses energy, at the rate $3P_3$: this is why E_{KS} falls along the history (Section 15.15). With interaction $p_t = e_{int}$ can have either sign, and the deflation of the extra times can give energy to the gas or take it away; Notebook 15b counts 29 interacting ground states with a positive and 31 with a negative integrated p_t (Out [11]).

Why the gas cannot be the source of the history. In the author's theory the function $a_4(x_4)$ is not free: it must solve the field equations for a_4 (Chapter 12), whose right-hand side is the energy-momentum tensor of the matter. The record `Revision/field_equations_a4/reports/ks-source-conditions.json` lists, under its key conditions, what a source must satisfy:

- (C1) every component of the source is independent of x_0, that is, of y;
- (C2) $p_3 + p_t = 2p_0$ at every point (an algebraic identity of the left-hand side of the a_4 equations, which every source must share);
- (C3) for the linear history $a_4 = AHx_4 + a_0$ used here: $p_3 = p_t = p_0$ and a constant ρ.

The Kohn-Sham gas fails all three. C1: its densities vary strongly with y (Figure 15a.5); the record finds $(\max\rho - \min\rho)/\max|T| \geq 0.0497$ in every state with a nonzero tensor (check ks_profiles_depend_on_x0). C2: $\max|p_3 + p_t - 2p_0|/\max|T|$ lies between 2.09 and 3.99 (check ks_profiles_violate_algebraic_condition), and even the integrated components violate it: $(\int p_3 + \int p_t)/(2\int p_0)$ is 0.339767, 0.259690, 0.239714 for $N = 136$, $\lambda = 0$ at $a_{4,0} = 0, 1, 2$, never close to 1 (check ks_integrals_violate_algebraic_condition). C3: along the history ρ falls (the integral from 80.28 to 12.45 for $N = 136$) and $p_3 \neq p_t$ (check ks_history_is_a_prescribed_background); indeed the energy-change law says that ρ can only stay constant if $p_3 = p_t$. The five free states of $N = 8$ are not covered by these checks: their tensor vanishes identically (only the zero modes are filled), a zero source, for which Einstein gravity has no vacuum solution with $H > 0$ (check ks_zero_source_states_listed). Notebook 15b reproduces these numbers from its own computations (Out [12]). Hence the status: the history $a_4 = AHx_4$ is a **PRESCRIBED BACKGROUND**, and the gas is a **test field** on it, without back-reaction; quantities derived along the history, such as the ratios of integrated pressures below, are not consequences of the coupled field equations. Chapter 17 returns to this question with the full source conditions.

Integrated pressures. The ratio $\int p_3 / \int \rho$ of the free gas is 0.2966, 0.3100, 0.3264 for $N = 136$ at $a_{4,0} = 0, 1, 2$ (Notebook 15b, Out [9]; `Revision/kohn_sham/results/ground/emt-integrals.csv`). It rises toward 1/3, the value for massless particles, because the band levels are nearly linear in the momentum ($\varepsilon \approx c\,k\,e^{-a_{4,0}}$), like massless particles, and become more so as the momenta redshift. These are integrated components on a prescribed background, not an equation of state that an observer in 3-space would measure.

15.17 Example: the energy-momentum profiles (Notebook 15b)

Notebook 15b runs the solver's command `single` for two states, $N = 136$ without interaction at $a_{4,0} = 1$ and $N = 136$ with $+\lambda_2$ at $a_{4,0} = 2$, and checks that their profiles equal the committed ones. It draws the four components as proper densities (Figure 15b.1) and as the densities that are integrated over y (Figure 15b.2), checks the conservation law along y point by point and integrated (Figure 15b.3), solves $N = 136$ at 17 slices and checks the energy-change law at every slice and integrated over the whole history (Figure 15b.4), draws the integrated pressure ratios (Figure 15b.5) and the spreading of the energy toward the tip (Figure 15b.6), shows the interaction terms (Figure 15b.7), and tests the three source conditions on its own numbers. It needs Rust, takes about half a minute, and ends with ALL 18 CHECKS PASSED (notebook 15b).

What to look for: in Figure 15b.1 that the proper densities are huge at the tip and that p_0 changes sign; in Figure 15b.2 that the same quantities times the volume factor sit near the brane; in Figure 15b.3 two curves that lie on top of each other; in Figure 15b.4 short orange lines, computed from the pressures alone, that touch the curve of the energies.

15.18 How to run Notebook 15b

Step 1. What this notebook does and what it needs.

Notebook 15b (Energy-momentum profiles and the conservation law along the hidden direction) is the file `Revision/textbook/notebooks/15b_emt_profiles.ipynb` of the repository Dirac_claude. It runs the Rust Kohn-Sham solver for chosen states of the Kohn-Sham gas of dirac16complex, reads the profiles of the energy density and of the three pressures along the hidden coordinate, checks them against the committed Revision record, verifies the conservation law of the energy-momentum tensor along the hidden direction point by point and integrated, integrates the profiles to the total energy, and checks the energy-change identity dE/da4 = minus three times the integrated difference of the 3-space and extra-time pressures along the deflating history. It also tests, on its own numbers, the three conditions that a source of the equations for a4 must meet, and reproduces the Revision record that shows that the Kohn-Sham states fail them (the history is a prescribed background). Before it runs the solver, the notebook builds it with cargo (a full build of about a minute when the program is missing, a second when it is up to date). The solver writes its output files (about 1 MB) into the folder `Revision/kohn_sham/solver/target/textbook_15b`, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because

it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/kohn_sham/solver/target/release/revision_ks_solver` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses

the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15b_emt_profiles.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15b_emt_profiles.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 15b_emt_profiles.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/15b.captions.json`
- `Revision/textbook/figures/15b_1_proper_emt.png`
- `Revision/textbook/figures/15b_2_coordinate_emt.png`
- `Revision/textbook/figures/15b_3_y_conservation.png`
- `Revision/textbook/figures/15b_4_energy_slopes.png`

- Revision/textbook/figures/15b_5_integrated_ratios.png
- Revision/textbook/figures/15b_6_energy_spreading.png
- Revision/textbook/figures/15b_7_interaction_terms.png

It changes no other file of the repository except the Rust build folder target next to each Cargo.toml it builds (inside it the notebook keeps the raw output of its program runs in Revision/kohn_sham/solver/target/textbook_15b, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 18 CHECKS PASSED (notebook 15b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file Revision/textbook/notebooks/15b_emt_profiles.ipynb inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with git clone.
- Windows: "running scripts is disabled on this system": run the line with Set-ExecutionPolicy and then the activation line again. "py is not recognized": use python instead of py -3; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run sudo apt install python3-venv and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 15b_emt_profiles.ipynb
python -m nbconvert --execute --inplace 15b_emt_profiles.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- You want the disk space of the solver output back: the folder `Revision/kohn_sham/solver/target/textbook_15b` holds only the raw output of the last run (21 files, about 0.9 MB), which git ignores; delete it at any time, the notebook writes it again.

15.19 Notebook 15b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/15b_emt_profiles.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A Kohn-Sham state of dirac16complex carries energy and momentum. The
**energy-momentum tensor** of the state has four different diagonal components: the
energy density $\rho$ and the pressures $p_3$ (along 3-space), $p_t$ (along the
three extra times) and $p_0$ (along the hidden direction). They depend on the
hidden coordinate $y$. This notebook
```

```
- runs the Rust Kohn-Sham solver for two states and checks that the profiles it
  writes are those of the committed Revision record;
- draws the four components, as proper densities and as the densities that are
  integrated over $y$;
- checks the **conservation law along the hidden direction**,
  $(e^{6Hy} p_0)' = 3H e^{6Hy}(p_3 + p_t)$, point by point and integrated;
- integrates the energy density to the total Kohn-Sham energy;
- solves the state $N = 136$ at 17 slices of the deflating history and checks the
  **energy-change identity**
  $dE/da_4 = -3 \cdot 2\,\mathrm{Vol}_7 \int e^{6Hy}(p_3 - p_t)\,dy$: the energy of
  the gas changes only through the difference between the 3-space pressure and the
  extra-time pressure;
- shows how the interaction adds the same energy density to $p_3$, $p_t$ and $p_0$;
- tests, on its own numbers, the three conditions that a source of the equations
  for $a_4$ must meet, and reproduces the Revision record that shows they fail.

It draws seven figures and takes about half a minute.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 15b (Energy-momentum profiles and the conservation law along the hidden
direction) is the file ‘Revision/textbook/notebooks/15b_emt_profiles.ipynb‘ of the
repository Dirac_claude. It runs the Rust Kohn-Sham solver for chosen states of the
Kohn-Sham gas of dirac16complex, reads the profiles of the energy density and of the
three pressures along the hidden coordinate, checks them against the committed Revision
record, verifies the conservation law of the energy-momentum tensor along the hidden
direction point by point and integrated, integrates the profiles to the total energy, and
checks the energy-change identity dE/da4 = minus three times the integrated difference of
the 3-space and extra-time pressures along the deflating history. It also tests, on its
own numbers, the three conditions that a source of the equations for a4 must meet, and
reproduces the Revision record that shows that the Kohn-Sham states fail them (the
history is a prescribed background). Before it runs the solver, the notebook builds it
with cargo (a full build of about a minute when the program is missing, a second when it
is up to date). The solver writes its output files (about 1 MB) into the folder
‘Revision/kohn_sham/solver/target/textbook_15b‘, which git ignores. It needs a computer
with Windows 11, macOS or Linux, an internet connection for the installation, the program
Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages
run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program
cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver,
which is part of the repository and is built on your computer.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
```

terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned

versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘ again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/kohn_sham/solver/target/release/revision_ks_solver‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15b_emt_profiles.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python

3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15b_emt_profiles.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 15b_emt_profiles.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/15b.captions.json‘
- ‘Revision/textbook/figures/15b_1_proper_emt.png‘
- ‘Revision/textbook/figures/15b_2_coordinate_emt.png‘
- ‘Revision/textbook/figures/15b_3_y_conservation.png‘
- ‘Revision/textbook/figures/15b_4_energy_slopes.png‘
- ‘Revision/textbook/figures/15b_5_integrated_ratios.png‘
- ‘Revision/textbook/figures/15b_6_energy_spreading.png‘
- ‘Revision/textbook/figures/15b_7_interaction_terms.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds (inside it the notebook keeps the raw output of its program runs in ‘Revision/kohn_sham/solver/target/textbook_15b‘, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
```

```
    ALL 18 CHECKS PASSED (notebook 15b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/15b_emt_profiles.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 15b_emt_profiles.ipynb
  python -m nbconvert --execute --inplace 15b_emt_profiles.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- You want the disk space of the solver output back: the folder ‘Revision/kohn_sham/solver/target/textbook_15b‘ holds only the raw output of the last run (21 files, about 0.9 MB), which git ignores; delete it at any time, the notebook writes it again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 15b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 15b (Energy-momentum profiles and the conservation law along the hidden
# direction) is the file Revision/textbook/notebooks/15b_emt_profiles.ipynb of the
# repository Dirac_claude. It runs the Rust Kohn-Sham solver for chosen states of the
# Kohn-Sham gas of dirac16complex, reads the profiles of the energy density and of the
# three pressures along the hidden coordinate, checks them against the committed Revision
# record, verifies the conservation law of the energy-momentum tensor along the hidden
# direction point by point and integrated, integrates the profiles to the total energy,
# and checks the energy-change identity dE/da4 = minus three times the integrated
# difference of the 3-space and extra-time pressures along the deflating history. It also
# tests, on its own numbers, the three conditions that a source of the equations for a4
# must meet, and reproduces the Revision record that shows that the Kohn-Sham states fail
# them (the history is a prescribed background). Before it runs the solver, the notebook
# builds it with cargo (a full build of about a minute when the program is missing, a
# second when it is up to date). The solver writes its output files (about 1 MB) into the
# folder Revision/kohn_sham/solver/target/textbook_15b, which git ignores. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It also needs Rust
# (the program cargo, version 1.91.1 or newer), because it runs the Rust program
# revision_ks_solver, which is part of the repository and is built on your computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
```

```
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
```

```
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
```

```
#
# Windows, macOS and Linux:
#     cargo --version
#
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
#     cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
#
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
# Revision/kohn_sham/solver/target/release/revision_ks_solver (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 15b_emt_profiles.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 15b_emt_profiles.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 15b_emt_profiles.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
```

```
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/15b.captions.json
# - Revision/textbook/figures/15b_1_proper_emt.png
# - Revision/textbook/figures/15b_2_coordinate_emt.png
# - Revision/textbook/figures/15b_3_y_conservation.png
# - Revision/textbook/figures/15b_4_energy_slopes.png
# - Revision/textbook/figures/15b_5_integrated_ratios.png
# - Revision/textbook/figures/15b_6_energy_spreading.png
# - Revision/textbook/figures/15b_7_interaction_terms.png
#
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds (inside it the notebook keeps the raw output of its program
# runs in Revision/kohn_sham/solver/target/textbook_15b, which git ignores); running it
# headless or saving it in JupyterLab also rewrites the notebook file itself. It does not
# use the internet while it runs. The files it writes are the same files that are stored
# in the repository (on another computer a figure may differ in a few bytes, which is
# harmless). To get the stored versions back, run this command in the repository folder
# (it also undoes every change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 18 CHECKS PASSED (notebook 15b)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/15b_emt_profiles.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
```

```
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 15b_emt_profiles.ipynb
#     python -m nbconvert --execute --inplace 15b_emt_profiles.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - You want the disk space of the solver output back: the folder
#   Revision/kohn_sham/solver/target/textbook_15b holds only the raw output of the last
#   run (21 files, about 0.9 MB), which git ignores; delete it at any time, the notebook
#   writes it again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "15b"  # this notebook: chapter 15, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
```

```
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
```

```
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
```

```
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 15b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Energy-momentum tensor** $T^\mu{}_\nu$: a table of $8 \times 8$ numbers at each
  point that says how much energy and momentum there is and how it flows. Its
  diagonal holds the **energy density** $\rho = -T^{x_4}{}_{x_4}$ and the
  **pressures** $p_\mu = T^{\mu}{}_{\mu}$ (no sum), the force per area in the
  direction $\mu$ (the sign convention of the Revision record).
- $p_3$: the pressure along each of the three directions of 3-space ($x_1, x_2,
  x_3$); $p_t$: along each extra time ($x_5, x_6, x_7$); $p_0$: along the hidden
  direction.
- **Interaction energy density** $e_{int} = \lambda(\tfrac{15}{32}S^2 -
  \tfrac{1}{32}n^2)$: the energy of the contact interaction per proper 7-volume.
- **Proper density** (per proper 7-volume) and **coordinate density** (per unit of
  $y$ and of the coordinates $x_1 \dots x_7$; it is $e^{6Hy}$ times the proper
  density, because the proper 7-volume of a coordinate box is $e^{6Hy}$ times its
  coordinate volume).
- **Conservation law**: the statement $\nabla_\mu T^\mu{}_\nu = 0$ (the covariant
  divergence vanishes). In flat spacetime it says that energy and momentum are
  neither created nor destroyed, they only flow. In the author's metric, which
  depends on the time $x_4$ through $a_4$, it is a local balance law: the energy is
  not conserved, it changes along the history (sections 4 and 9). The momenta along
  $x_1, x_2, x_3, x_5, x_6, x_7$, coordinates on which the metric does not depend,
  have totals that are constant, but only under ASSUMED boundary conditions
  (Section 20.22 of the book).
- **Finite difference**: an approximation of a derivative from values at nearby
  points; the fourth-order formula
  $f'(y_i) \approx [f_{i-2} - 8 f_{i-1} + 8 f_{i+1} - f_{i+2}]/(12h)$ has an error
  proportional to $h^4$.
- **Symmetric logarithmic axis**: logarithmic for large positive and negative values
  and linear near zero, so that both signs fit on one axis.
- **Tangent**: the straight line that touches a curve at one point; its slope is the
  derivative there.
```

Text cell:

```
## 4. The physical and mathematical situation
```

Geometry. In the hidden coordinate y (tip $y = -L = -3$, brane $y = 0$) the author's metric reads (Revision record Revision/kohn_sham/ks-theory.json)

$$ds^2 = e^{2Hy}\left[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2
+ dx_7^2)\right] - dx_4^2 + dy^2 .$$

3-space (x_1, x_2, x_3) inflates with e^{a_4} while the three extra times (x_5, x_6, x_7) deflate with e^{-a_4}; the proper 7-volume factor is $\sqrt{|g|} = e^{6Hy}$, the same at every slice $a_{4,0}$.

The tensor of a Kohn-Sham state (ks-theory.json, emt; sums over the occupied levels with the weights $w g f$, $w = 1/2$; n_o, s_o, t_o are the number, scalar and current densities of one orbital):

$$\rho = \sum w g f\,\varepsilon\,n_o - e_{int}, \qquad
p_3 = \sum w g f\,\tfrac{\kappa |k|}{3}\,t_o + e_{int}, \qquad p_t = e_{int},$$
$$p_0 = \sum w g f\left[(\varepsilon - v)\,n_o - M s_o - \kappa|k|\,t_o\right]
+ e_{int} .$$

The extra-time pressure contains no kinetic part: the states belong to the *good sector*, which has no momentum along the extra times.

Conservation along y. For a tensor that is diagonal and depends only on y, the y component of $\nabla_\mu T^\mu{}_\nu = 0$ is

$$\partial_y T^y{}_y + \Gamma^\mu{}_{\mu y} T^y{}_y
- \sum_\nu \Gamma^\nu{}_{\nu y} T^\nu{}_\nu = 0$$

(the index ν runs over the eight directions; the letter λ is kept for the coupling). Line by line:
$\Gamma^\mu{}_{\mu y} = \partial_y \ln\sqrt{|g|} = 6H$ (the derivative of $\ln e^{6Hy}$); $\Gamma^\nu{}_{\nu y} = \tfrac12 g^{\nu\nu}\partial_y g_{\nu\nu} = H$ for each of the six warped directions $x_1, x_2, x_3, x_5, x_6, x_7$ (their $g_{\nu\nu}$ contains the factor e^{2Hy}, and half the derivative of $\ln e^{2Hy}$ is H) and 0 for x_4 and y; the six warped terms are $3H p_3$ (3-space) and $3H p_t$ (extra times); so

$$p_0' + 6H p_0 - 3H p_3 - 3H p_t = 0, \quad\text{that is}\quad
(e^{6Hy} p_0)' = 3H\,e^{6Hy}(p_3 + p_t),$$

because $(e^{6Hy}p_0)' = e^{6Hy}(p_0' + 6Hp_0)$ (product and chain rule). Integrated from tip to brane: $[e^{6Hy}p_0]_{-L}^{0} = 3H \int_{-L}^{0} e^{6Hy}(p_3 + p_t)\,dy$.

Energy change along the history. The x_4 component of the same law reads $\partial_{x_4} T^{x_4}{}_{x_4} + \Gamma^\mu{}_{\mu x_4} T^{x_4}{}_{x_4} - \sum_\nu \Gamma^\nu{}_{\nu x_4} T^\nu{}_\nu = 0$. Line by line:
$T^{x_4}{}_{x_4} = -\rho$ (the sign convention); $\Gamma^\mu{}_{\mu x_4} = \partial_{x_4}\ln\sqrt{|g|} = 3a_4' - 3a_4' = 0$ (3-space grows like e^{3a_4}, the extra times shrink like e^{-3a_4}, so the 7-volume is constant); $\Gamma^\nu{}_{\nu x_4} = \tfrac12 g^{\nu\nu}\partial_{x_4} g_{\nu\nu} = +a_4'$ for the three inflating directions ($g_{\nu\nu} \propto e^{2a_4}$) and $-a_4'$ for the three deflating extra times ($g_{\nu\nu} \propto e^{-2a_4}$), 0 for x_4 and y. So $-\partial_{x_4}\rho - (3a_4' p_3 - 3a_4' p_t) = 0$, that is $\partial_{x_4}\rho = -3 a_4'(p_3 - p_t)$. Along the history a_4 grows with x_4,

```
so $\partial_{x_4} = a_4'\,\partial/\partial a_4$ (chain rule) and
$\partial\rho/\partial a_4 = -3(p_3 - p_t)$. Multiplying by $2\,\mathrm{Vol}_7
e^{6Hy}$ and integrating over $y$ gives, for the total energy
$E = 2\,\mathrm{Vol}_7\int e^{6Hy}\rho\,dy$ of the doubled system of states that
follow the history adiabatically (the solver checks it with the Hellmann-Feynman
theorem),

$$\frac{dE}{da_4} = -3 \cdot 2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy}(p_3 - p_t)\,dy .$$

Read as work: when 3-space inflates by $da_4$, the gas loses the energy
$3\,p_3\,da_4$ per unit proper volume (the work done by its 3-space pressure), and
when the three extra times deflate by the same amount it gains $3\,p_t\,da_4$.
Here $p_t = e_{int}$ is the interaction energy density alone: it is zero without
interaction and can have either sign (positive or negative integrals both occur in
the record), so the deflation can return energy to the gas or take more away.

**Status.** The tensor is COMPUTED by the solver; the two identities are exact and
are checked here numerically. The history $a_4 = A H x_4$ is a PRESCRIBED
BACKGROUND: the Kohn-Sham tensor depends on $y$ (hence on $x_0$) and has
$p_3 + p_t \ne 2 p_0$, so it is not an admissible source of the equations for $a_4$
(Revision record Revision/field_equations_a4/reports/ks-source-conditions.json).
Units: $H = m = 1$, $L = 3$, $\mathrm{Vol}_7 = (2\pi/0.25)^3$.
```

Text cell:

```
## 5. Building the solver and producing two profiles

The next cell builds the solver (‘rust_program‘, about a second when it is up to
date) and defines a helper ‘solve_state‘ that runs the subcommand ‘single‘ for one
state: ‘--m‘, ‘--lambda‘, ‘--a4‘, ‘--N‘ give the mass, coupling, slice and particle
number, ‘--out‘ names the JSON file of the results and ‘--profiles‘ the CSV file of
the profiles. The files go into the folder ‘textbook_15b‘ inside the Rust build
folder ‘target‘, which git ignores.
```

In [2]:

```
import csv  # reads the tables (CSV files)
import math  # exp and pi for single numbers

import numpy as np  # arrays of numbers

program = rust_program("Revision/kohn_sham/solver/Cargo.toml", "revision_ks_solver")
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_15b"  # ignored by git
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
SLICE_SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light->dark
VOL7 = (2.0 * math.pi / 0.25) ** 3  # proper 7-volume per unit e^{6Hy}
H = 1.0


def solve_state(name, n, lam, a4, profiles=False):
```

```
    """Run "revision_ks_solver single" for one state; return its JSON record."""
    out = RUN_FOLDER / f"{name}.json"
    command = [str(program), "single", "--root", str(REPO), "--m", "1",
               "--lambda", repr(lam), "--a4", repr(a4), "--N", repr(float(n)),
               "--out", str(out)]
    if profiles:
        command += ["--profiles", str(RUN_FOLDER / f"{name}.csv")]
    done = subprocess.run(command, capture_output=True, text=True)
    if done.returncode != 0 or done.stdout.strip().split("\n")[-1] != "SUCCESS":
        raise RuntimeError(f"the solver failed for {name}: {done.stderr[-500:]}")
    return json.loads(out.read_text(encoding="utf-8"))


params = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))
LAMBDA2_136 = {int(c["N"]): c for c in params["couplingCalibration"]["values"]}[136][
    "lambda2"]  # the coupling lambda_2 of N = 136
free_state = solve_state("N136_lam0_a10", 136, 0.0, 1.0, profiles=True)
strong_state = solve_state("N136_lamp2_a20", 136, LAMBDA2_136, 2.0, profiles=True)
say(f"solved N136_lam0_a10 ({free_state['iterations']} iteration) and N136_lamp2_a20 "
    f"({strong_state['iterations']} iterations, lambda = {LAMBDA2_136})")
```

Out [2]:

```
Rust program revision_ks_solver is built and ready.
solved N136_lam0_a10 (1 iteration) and N136_lamp2_a20 (17 iterations, lambda = 0.002789)
```

Text cell:

```
The next cell reads the two new profile files and the corresponding committed files
of the record (‘ground/profiles/N136_lam0_a10.csv‘ and ‘N136_lamp2_a20.csv‘) and
compares every column. The largest difference is measured relative to the largest
value of each column; the tolerance $10^{-9}$ is far below the accuracy of the
profiles themselves ($10^{-6}$, the tolerance of the record’s refined comparison).
Whether the files are byte-identical is printed as a RESULT line.
```

In [3]:

```
RECORD_PROFILES = "Revision/kohn_sham/results/ground/profiles"


def read_profile(path):
    """A profile table as a dictionary column -> numpy array."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}


worst = 0.0
identical = []
for name in ("N136_lam0_a10", "N136_lamp2_a20"):
    new = read_profile(RUN_FOLDER / f"{name}.csv")
```

```
    old = read_profile(repository_file(f"{RECORD_PROFILES}/{name}.csv"))
    for column in old:
        scale = max(float(np.max(np.abs(old[column]))), 1e-300)
        worst = max(worst, float(np.max(np.abs(new[column] - old[column]))) / scale)
    identical.append((RUN_FOLDER / f"{name}.csv").read_bytes()
                     == repository_file(f"{RECORD_PROFILES}/{name}.csv").read_bytes())
say("columns: " + ", ".join(old))
report("largest relative difference to the record", f"{worst:.1e}")
report("profile files byte-identical to the record", identical)
check(worst < 1e-9, "the new profiles equal the committed profiles within 1e-9",
      record=f"{RECORD_PROFILES}/N136_lam0_a10.csv and N136_lamp2_a20.csv")
```

Out [3]:

```
columns: y, n, S, Q, M_eff, v_v, w_Q, e_int, rho, p3, p_t, p0
RESULT largest relative difference to the record = 0.0e+00
RESULT profile files byte-identical to the record = [True, True]
PASS the new profiles equal the committed profiles within 1e-9
     reproduces Revision/kohn_sham/results/ground/profiles/N136_lam0_a10.csv and
    N136_lamp2_a20.csv
```

Text cell:

```
## 6. The four components as proper densities

The next cell draws $\rho$, $p_3$, $p_t$ and $p_0$ of the state $N = 136$,
$\lambda = 0$, $a_{4,0} = 1$ against $y$. The values span six orders of magnitude and
$p_0$ changes sign, so the vertical axis is symmetric logarithmic (linear between
$-10^{-3}$ and $10^{-3}$). Without interaction $p_t = e_{int} = 0$ exactly; the
check confirms it.
```

In [4]:

```
prof = read_profile(RUN_FOLDER / "N136_lam0_a10.csv")
y = prof["y"]
step = float(y[1] - y[0])  # 0.02
COMPONENTS = [("rho", "$\\rho$"), ("p3", "$p_3$"), ("p_t", "$p_t$"), ("p0", "$p_0$")]
fig, ax = plt.subplots()
for colour, (column, label) in zip(PALETTE, COMPONENTS):
    ax.plot(y, prof[column], color=colour, lw=2.0, label=label)
ax.set_yscale("symlog", linthresh=1e-3)
ax.set_xlabel("hidden coordinate $y$ (tip at $-3$, brane at $0$)")
ax.set_ylabel("proper density (units of $m^8$)")
ax.set_title("Energy density and pressures, $N = 136$, $\\lambda = 0$, $a_{4,0} = 1$")
ax.legend()
save_figure(fig, "proper_emt",
            "The energy density $\\rho$ and the pressures $p_3$, $p_t$, $p_0$ of the "
            "Kohn-Sham state $N = 136$, $\\lambda = 0$, $a_{4,0} = 1$ as proper "
            "densities (vertical axis, symmetric logarithmic, units of $m^8$) "
            "against the hidden coordinate $y$ (horizontal axis). All are largest "
            "at the tip, where the proper volume is small; $p_0$ is negative near "
```

```
            "the tip and changes sign near $y = -2.45$, and $p_t$ is zero without "
            "interaction.")
report("p_t without interaction: largest |p_t|", float(np.max(np.abs(prof["p_t"]))))
check(float(np.max(np.abs(prof["p_t"]))) == 0.0, "p_t = e_int = 0 when lambda = 0")
```

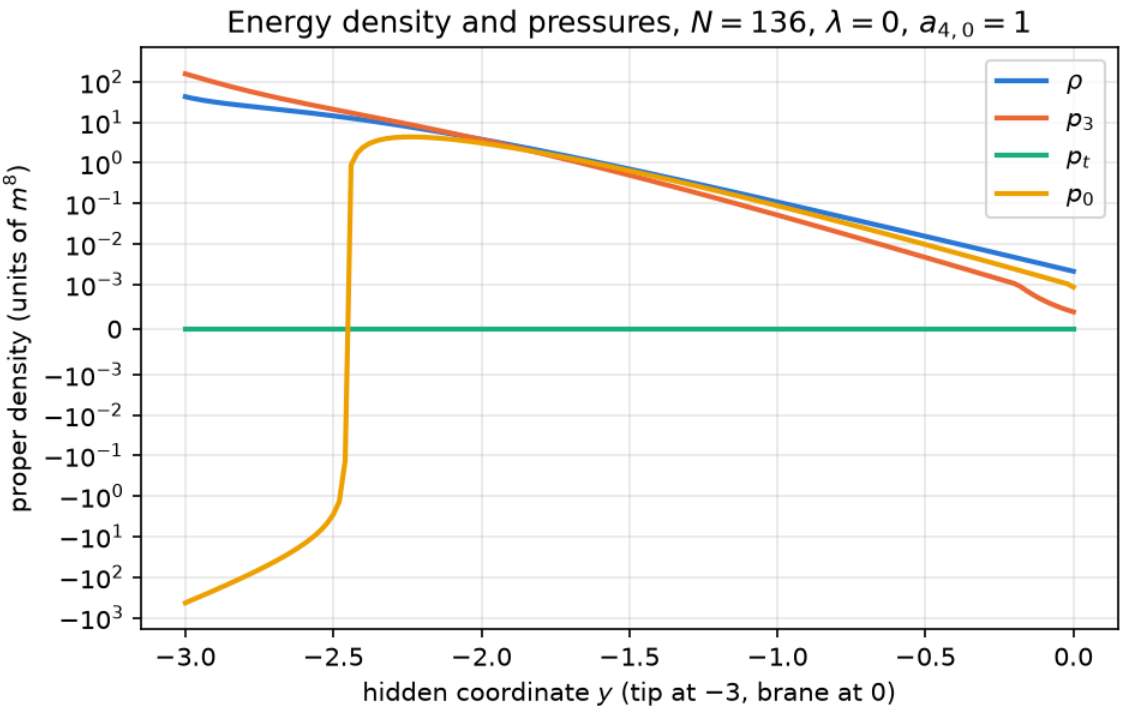

Figure 419: The energy density ρ and the pressures p_3, p_t, p_0 of the Kohn-Sham state $N = 136$, $\lambda = 0$, $a_{4,0} = 1$ as proper densities (vertical axis, symmetric logarithmic, units of m^8) against the hidden coordinate y (horizontal axis). All are largest at the tip, where the proper volume is small; p_0 is negative near the tip and changes sign near $y = -2.45$, and p_t is zero without interaction. (Notebook 15b, figure 1.)

Out [4]:

```
Figure 15b.1 saved as Revision/textbook/figures/15b_1_proper_emt.png
RESULT p_t without interaction: largest |p_t| = 0.0
PASS p_t = e_int = 0 when lambda = 0
```

Text cell:

```
## 7. What is integrated: the coordinate densities

Multiplying by $2\,\mathrm{Vol}_7\, e^{6Hy}$ turns a proper density into the amount
per unit of $y$ in the whole box of the doubled system. The next cell draws these
densities (now on a linear axis) and integrates them with Simpson's rule on the 151
points: the integral of the energy density must be the Kohn-Sham energy $E_{KS}$ of
the solver's JSON record (and of the committed table 'ground/summary.csv'), and the
integrals of the pressures must agree with the committed table
```

‘ground/emt-integrals.csv‘. The solver integrates on a finer grid (1801 points), so
the comparison allows a relative difference of 10^{-6}.

In [5]:

```
def simpson(values, h):
    """Simpson's rule on equally spaced points (an odd number of them)."""
    weights = np.full(len(values), 2.0)
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    return h / 3.0 * float(np.sum(weights * values))


def read_rows(relative):
    """The rows of a committed CSV table of the record as a dictionary by id."""
    with open(repository_file(relative), newline="", encoding="utf-8") as handle:
        return {row["id"]: row for row in csv.DictReader(handle)}


EMT_TABLE = "Revision/kohn_sham/results/ground/emt-integrals.csv"
emt_rows = read_rows(EMT_TABLE)
summary_rows = read_rows("Revision/kohn_sham/results/ground/summary.csv")
weight = 2.0 * VOL7 * np.exp(6.0 * H * y)  # 2 Vol_7 e^{6Hy}
fig, ax = plt.subplots()
integrals = {}
for colour, (column, label) in zip(PALETTE, COMPONENTS):
    ax.plot(y, weight * prof[column], color=colour, lw=2.0, label=label)
    integrals[column] = simpson(weight * prof[column], step)
ax.axhline(0.0, color="0.4", lw=0.8)
ax.set_xlabel("hidden coordinate $y$")
ax.set_ylabel("$2\\,\\mathrm{Vol}_7\\, e^{6Hy} \\times$ component (units of $m$)")
ax.set_title("The integrands: energy and pressures per unit $y$")
ax.legend()
save_figure(fig, "coordinate_emt",
            "The energy density and the pressures of the state $N = 136$, "
            "$\\lambda = 0$, $a_{4,0} = 1$ multiplied by $2\\,\\mathrm{Vol}_7 "
            "e^{6Hy}$ (vertical axis, linear, units of $m$ per unit $y$) against "
            "$y$ (horizontal axis). In this form the energy and the pressures sit "
            "near the brane; the area under $\\rho$ is the Kohn-Sham energy "
            "$E_{KS} = 32.384$.")
row = emt_rows["N136_lam0_a10"]
rel = lambda a, b: abs(a - b) / max(abs(b), 1e-300)
worst_int = max(rel(integrals[c], float(row[f"int_{c}"])) for c in ("rho", "p3", "p0"))
report("E_KS from the profile / from the solver",
       f"{integrals['rho']:.8f} / {free_state['E_KS']:.8f}")
report("largest relative difference of the integrals to the record", f"{worst_int:.1e}")
check(rel(free_state["E_KS"], float(summary_rows["N136_lam0_a10"]["E_KS"])) < 1e-12,
      "the solver's E_KS equals the record",
      record="Revision/kohn_sham/results/ground/summary.csv, N136_lam0_a10")
check(rel(integrals["rho"], free_state["E_KS"]) < 1e-6 and worst_int < 1e-6,
      "Simpson's rule on the profile gives E_KS and the recorded integrals",
      record=f"{EMT_TABLE}, columns int_rho, int_p3, int_p0")
```

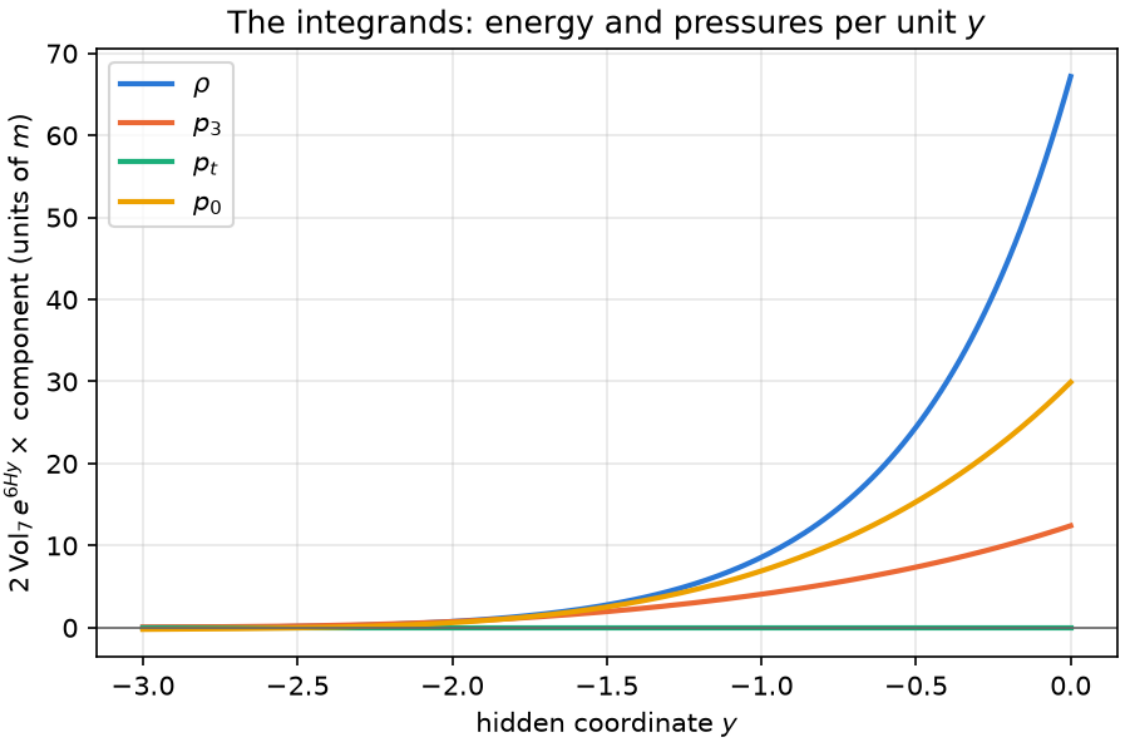

Figure 420: The energy density and the pressures of the state $N = 136$, $\lambda = 0$, $a_{4,0} = 1$ multiplied by $2\,\mathrm{Vol}_7 e^{6Hy}$ (vertical axis, linear, units of m per unit y) against y (horizontal axis). In this form the energy and the pressures sit near the brane; the area under ρ is the Kohn-Sham energy $E_{KS} = 32.384$. (Notebook 15b, figure 2.)

Out [5]:

```
Figure 15b.2 saved as Revision/textbook/figures/15b_2_coordinate_emt.png
RESULT E_KS from the profile / from the solver = 32.38411614 / 32.38411569
RESULT largest relative difference of the integrals to the record = 1.4e-08
PASS the solver's E_KS equals the record
     reproduces Revision/kohn_sham/results/ground/summary.csv, N136_lam0_a10
PASS Simpson's rule on the profile gives E_KS and the recorded integrals
     reproduces Revision/kohn_sham/results/ground/emt-integrals.csv, columns int_rho,
    int_p3, int_p0
```

Text cell:

```
## 8. The conservation law along the hidden direction

The next cell computes the left side $(e^{6Hy}p_0)'$ by the fourth-order finite
difference (one-sided fourth-order formulas at the two ends, as in the solver) and
the right side $3H e^{6Hy}(p_3 + p_t)$, draws both, and draws their difference
divided by the largest of the two sides. It also compares the jump
$[e^{6Hy}p_0]_{-3}^{0}$ with the integral of the right side. The solver checked the
same law on its fine grid (record checks ‘emt_y_conservation_pointwise‘ and
‘emt_y_conservation_integrated‘); on the 151 points of the profile the finite
difference is less accurate, so the tolerances here are $10^{-4}$ (point by point)
and $10^{-6}$ (integrated).
```

In [6]:

```
def derivative4(f, h):
    """Fourth-order finite-difference derivative (one-sided at the two ends)."""
    d = np.zeros(len(f))
    d[2:-2] = (f[:-4] - 8.0 * f[1:-3] + 8.0 * f[3:-1] - f[4:]) / (12.0 * h)
    d[0] = (-25 * f[0] + 48 * f[1] - 36 * f[2] + 16 * f[3] - 3 * f[4]) / (12.0 * h)
    d[1] = (-3 * f[0] - 10 * f[1] + 18 * f[2] - 6 * f[3] + f[4]) / (12.0 * h)
    d[-1] = (25 * f[-1] - 48 * f[-2] + 36 * f[-3] - 16 * f[-4] + 3 * f[-5]) / (12.0 * h)
    d[-2] = (3 * f[-1] + 10 * f[-2] - 18 * f[-3] + 6 * f[-4] - f[-5]) / (12.0 * h)
    return d


def conservation(p):
    """(left side, right side, pointwise residual, integrated residual) of the law."""
    e6 = np.exp(6.0 * H * p["y"])
    left = derivative4(e6 * p["p0"], step)
    right = 3.0 * H * e6 * (p["p3"] + p["p_t"])
    scale = max(float(np.max(np.abs(left))), float(np.max(np.abs(right))))
    jump = e6[-1] * p["p0"][-1] - e6[0] * p["p0"][0]
    integral = simpson(right, step)
    return left, right, np.abs(left - right) / scale, abs(jump - integral) / abs(jump)


left, right, residual, integrated = conservation(prof)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
```

```
ax1.plot(y, left, color=PALETTE[0], lw=3.0, label="$(e^{6Hy}p_0)'$")
ax1.plot(y, right, color=PALETTE[1], lw=1.5, ls="--",
         label="$3H\\,e^{6Hy}(p_3 + p_t)$")
ax1.set_xlabel("hidden coordinate $y$")
ax1.set_ylabel("units of $m^9$")
ax1.set_title("The two sides of the law")
ax1.legend()
ax2.semilogy(y, np.maximum(residual, 1e-16), color=PALETTE[2], lw=1.5)
ax2.axhline(1e-4, color="0.4", ls=":", lw=1.2, label="tolerance $10^{-4}$")
ax2.set_xlabel("hidden coordinate $y$")
ax2.set_ylabel("|left - right| / largest value")
ax2.set_title("Relative residual on the 151 points")
ax2.legend()
save_figure(fig, "y_conservation",
            "Left: the two sides of the conservation law along the hidden "
            "direction, $(e^{6Hy}p_0)'$ (solid) and $3H e^{6Hy}(p_3 + p_t)$ "
            "(dashed), for $N = 136$, $\\lambda = 0$, $a_{4,0} = 1$ (vertical axis, "
            "units of $m^9$; horizontal axis $y$); the curves lie on top of each "
            "other. Right: their difference relative to the largest value "
            "(logarithmic), the error of the finite difference on the coarse "
            "profile grid, far below the tolerance.")
report("largest pointwise residual / integrated residual",
       f"{float(np.max(residual)):.1e} / {integrated:.1e}")
check(float(np.max(residual)) < 1e-4 and integrated < 1e-6,
      "(e^{6Hy} p0)' = 3H e^{6Hy} (p3 + p_t) holds for N136_lam0_a10",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, checks "
             "emt_y_conservation_pointwise and emt_y_conservation_integrated")
```

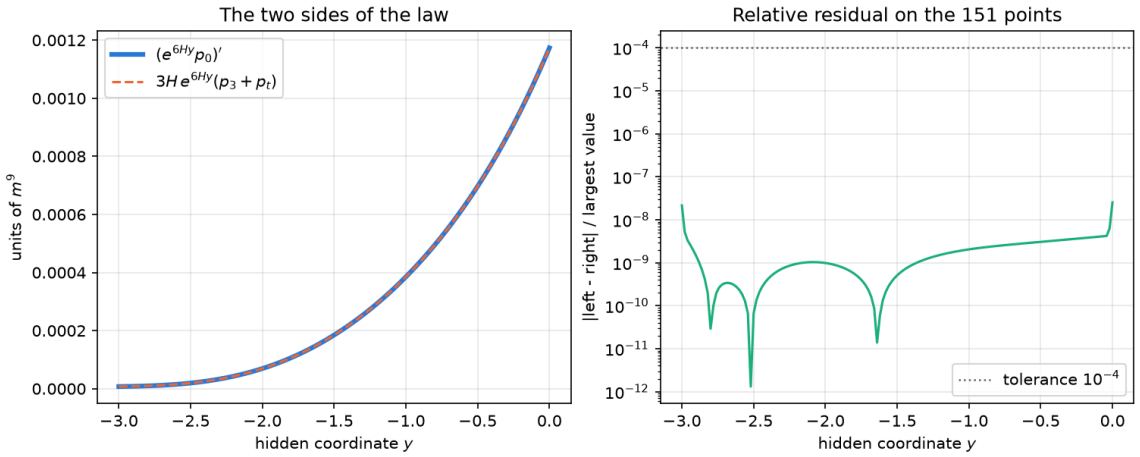

Figure 421: Left: the two sides of the conservation law along the hidden direction, $(e^{6Hy}p_0)'$ (solid) and $3He^{6Hy}(p_3 + p_t)$ (dashed), for $N = 136$, $\lambda = 0$, $a_{4,0} = 1$ (vertical axis, units of m^9; horizontal axis y); the curves lie on top of each other. Right: their difference relative to the largest value (logarithmic), the error of the finite difference on the coarse profile grid, far below the tolerance. (Notebook 15b, figure 3.)

Out [6]:

```
Figure 15b.3 saved as Revision/textbook/figures/15b_3_y_conservation.png
```

```
RESULT largest pointwise residual / integrated residual = 2.5e-08 / 6.1e-10
PASS (e^{6Hy} p0)' = 3H e^{6Hy} (p3 + p_t) holds for N136_lam0_a10
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, checks
    emt_y_conservation_pointwise and emt_y_conservation_integrated
```

Text cell:

```
## 9. The energy along the history and its slope

The next cell solves the state $N = 136$ without interaction at 17 slices,
$a_{4,0} = 0, 0.125, 0.25, \dots, 2$ (about a third of a second each), and reads
from each JSON record the energy $E_{KS}$ and the integrals
$2\,\mathrm{Vol}_7\int e^{6Hy}(\cdot)\,dy$ of $\rho$, $p_3$, $p_t$, $p_0$ that the
solver computed on its fine grid. From them it forms the slope
$-3(\int p_3 - \int p_t)$ of the energy-change identity. At the five slices of the
canonical matrix the energies and integrals must equal the committed tables, and
the slopes must equal the derivative $dE/da_4$ that the solver obtained by
finite differences between neighbouring slices (column ‘dE_da4_finite_difference‘
of ‘adiabatic/adiabaticity.csv‘).
```

In [7]:

```
DENSE = [i / 8.0 for i in range(17)]  # the slices 0, 0.125, ..., 2
history = []
for a4 in DENSE:
    record = solve_state(f"N136_lam0_dense_{round(1000 * a4):04d}", 136, 0.0, a4)
    ints = record["emtIntegrals_2Vol7_int_e6Hy"]  # 2 Vol_7 int e^{6Hy} (...) dy
    history.append({"a4": a4, "E": record["E_KS"], "rho": ints["rho"],
                    "p3": ints["p3"], "p_t": ints["p_t"], "p0": ints["p0"],
                    "slope": -3.0 * (ints["p3"] - ints["p_t"])})
adiabatic_rows = read_rows("Revision/kohn_sham/results/adiabatic/adiabaticity.csv")
worst_e, worst_slope = 0.0, 0.0
say("a4,0       E_KS     dE/da4 (EMT)   dE/da4 (record, finite differences)")
for item in history:
    if item["a4"] in (0.0, 0.5, 1.0, 1.5, 2.0):
        key = f"N136_lam0_a{round(10 * item['a4']):02d}"
        recorded_slope = float(adiabatic_rows[key]["dE_da4_finite_difference"])
        worst_e = max(worst_e, rel(item["E"], float(summary_rows[key]["E_KS"])),
                      rel(item["p3"], float(emt_rows[key]["int_p3"])))
        worst_slope = max(worst_slope, rel(item["slope"], recorded_slope))
        say(f"{item['a4']:4.2f}  {item['E']:10.5f}  {item['slope']:12.6f}"
            f"  {recorded_slope:12.6f}")
report("largest relative difference of E and int p3 to the record", f"{worst_e:.1e}")
report("largest relative difference of the slopes", f"{worst_slope:.1e}")
check(worst_e < 1e-12, "E_KS and the integrals at the five slices equal the record",
      record="Revision/kohn_sham/results/ground/summary.csv and emt-integrals.csv")
check(worst_slope < 1e-8, "dE/da4 = -3 (int p3 - int p_t) at the five slices",
      record="Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column "
             "dE_da4_finite_difference")
```

Out [7]:

```
a4,0       E_KS     dE/da4 (EMT)   dE/da4 (record, finite differences)
0.00    80.28222    -71.439756    -71.439756
0.50    51.24519    -46.470842    -46.470842
1.00    32.38412    -30.119224    -30.119224
1.50    20.20308    -19.339665    -19.339665
2.00    12.44507    -12.186143    -12.186143
RESULT largest relative difference of E and int p3 to the record = 0.0e+00
RESULT largest relative difference of the slopes = 3.7e-12
PASS E_KS and the integrals at the five slices equal the record
     reproduces Revision/kohn_sham/results/ground/summary.csv and emt-integrals.csv
PASS dE/da4 = -3 (int p3 - int p_t) at the five slices
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column
    dE_da4_finite_difference
```

Text cell:

```
The identity also holds between the slices. The next cell integrates the slopes over
the whole history with Simpson's rule on the 17 slices (step $0.125$) and compares
the result with the change of the energy itself, $E(2) - E(0)$: the energy lost by
the gas along the history is exactly the work done by the pressure difference. The
figure shows $E_{KS}$ at the 17 slices with short tangent lines whose slopes are
computed from the pressures alone.
```

In [8]:

```
slopes = np.array([item["slope"] for item in history])
energies = np.array([item["E"] for item in history])
work = simpson(slopes, 0.125)  # integral of dE/da4 over the history
change = energies[-1] - energies[0]
fig, ax = plt.subplots()
ax.plot(DENSE, energies, "o", color=PALETTE[0], ms=8, label="$E_{KS}$ (solver)")
for a4, e, s in zip(DENSE, energies, slopes):
    ax.plot([a4 - 0.05, a4 + 0.05], [e - 0.05 * s, e + 0.05 * s], color=PALETTE[1],
            lw=2.5)
ax.plot([], [], color=PALETTE[1], lw=2.0,
        label="slope $-3 \\cdot 2\\mathrm{Vol}_7 \\int e^{6Hy}(p_3 - p_t)$")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("$E_{KS}$ (units of $m$)")
ax.set_title("$N = 136$, $\\lambda = 0$: the pressures give the slope of the energy")
ax.legend()
save_figure(fig, "energy_slopes",
            "The Kohn-Sham energy $E_{KS}$ of $N = 136$ without interaction "
            "(vertical axis, units of $m$) at 17 slices $a_{4,0}$ of the history "
            "(horizontal axis), with short tangent lines whose slopes are computed "
            "from the pressures alone, $dE/da_4 = -3 \\cdot 2\\,\\mathrm{Vol}_7 "
            "\\int e^{6Hy}(p_3 - p_t)\\,dy$: every line touches the curve of the "
            "energies, which falls from $80.3$ to $12.4$.")
report("E(2) - E(0) / integral of the slopes", f"{change:.6f} / {work:.6f}")
check(rel(work, change) < 1e-5,
      "the integrated slope equals E(2) - E(0) within 1e-5 (relative)",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "emt_energy_change_dE_da4")
```

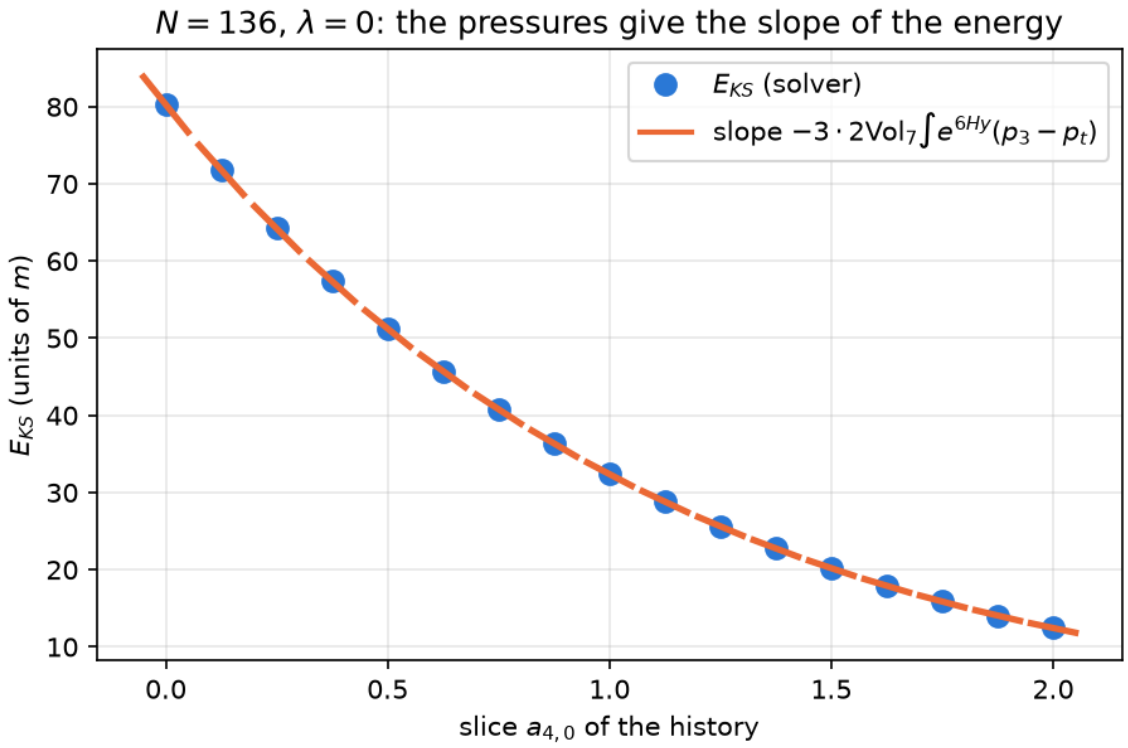

Figure 422: The Kohn-Sham energy E_{KS} of $N = 136$ without interaction (vertical axis, units of m) at 17 slices $a_{4,0}$ of the history (horizontal axis), with short tangent lines whose slopes are computed from the pressures alone, $dE/da_4 = -3 \cdot 2\,\mathrm{Vol}_7 \int e^{6Hy}(p_3 - p_t)\,dy$: every line touches the curve of the energies, which falls from 80.3 to 12.4. (Notebook 15b, figure 4.)

Out [8]:

```
Figure 15b.4 saved as Revision/textbook/figures/15b_4_energy_slopes.png
RESULT E(2) - E(0) / integral of the slopes = -67.837152 / -67.837207
PASS the integrated slope equals E(2) - E(0) within 1e-5 (relative)
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    emt_energy_change_dE_da4
```

Text cell:

```
## 10. The integrated pressures along the history

The next cell divides the integrated pressures by the integrated energy density.
These ratios are the integrated components only: what a 3-space observer would call
an equation of state is not computed here (it needs the averaging over the hidden
direction and the extra times, which is the subject of the Revision dark-sector
work), and the history is a prescribed background. The figure shows
$\int p_3/\int\rho$ and $\int p_0/\int\rho$ for $N = 136$ at the 17 slices (lines)
and for $N = 136$ and $N = 688$ at the five slices of the record (markers). The check
compares the ratios of $N = 136$ at the five slices with the committed table.
```

In [9]:

```
ratio3 = np.array([item["p3"] / item["rho"] for item in history])
ratio8 = np.array([item["p0"] / item["rho"] for item in history])
fig, ax = plt.subplots()
ax.plot(DENSE, ratio3, color=PALETTE[0], lw=2.0,
        label="$\\int p_3 / \\int \\rho$, $N = 136$")
ax.plot(DENSE, ratio8, color=PALETTE[1], lw=2.0,
        label="$\\int p_0 / \\int \\rho$, $N = 136$")
worst_ratio = 0.0
for marker, n in (("o", 136), ("s", 688)):
    keys = [f"N{n}_lam0_a{round(10 * a):02d}" for a in (0.0, 0.5, 1.0, 1.5, 2.0)]
    r3 = [float(emt_rows[k]["int_p3"]) / float(emt_rows[k]["int_rho"]) for k in keys]
    r8 = [float(emt_rows[k]["int_p0"]) / float(emt_rows[k]["int_rho"]) for k in keys]
    ax.plot([0, 0.5, 1, 1.5, 2], r3, marker, color=PALETTE[0], ms=8,
            markerfacecolor="white", label=f"record, $N = {n}$")
    ax.plot([0, 0.5, 1, 1.5, 2], r8, marker, color=PALETTE[1], ms=8,
            markerfacecolor="white")
    if n == 136:
        worst_ratio = max(abs(r3[i] - ratio3[4 * i]) for i in range(5))
ax.axhline(1.0 / 3.0, color="0.4", ls=":", lw=1.2, label="$1/3$")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("ratio of integrals")
ax.set_title("Integrated pressures over integrated energy, $\\lambda = 0$")
ax.legend(fontsize=8)
save_figure(fig, "integrated_ratios",
            "The integrated 3-space pressure (blue) and hidden-direction pressure "
            "(orange), each divided by the integrated energy density (vertical "
            "axis, pure numbers), against the slice $a_{4,0}$ (horizontal axis), for "
            "$N = 136$ without interaction at 17 slices (lines) and for $N = 136$ "
```

```
            "(circles) and $N = 688$ (squares) from the record. The 3-space ratio "
            "rises from about $0.30$ toward the value $1/3$ of a gas of massless "
            "particles as the momenta redshift. These are integrated components "
            "on a prescribed background, not an observed equation of state.")
report("int p3 / int rho of N = 136 at a4,0 = 0, 1, 2",
       ", ".join(f"{ratio3[i]:.4f}" for i in (0, 8, 16)))
report("int p0 / int rho of N = 136 at a4,0 = 0, 1, 2",
       ", ".join(f"{ratio8[i]:.4f}" for i in (0, 8, 16)))
check(worst_ratio < 1e-12, "the ratios at the five slices equal the record",
      record=f"{EMT_TABLE}, int_p3 / int_rho")
check(bool(np.all(np.diff(ratio3) > 0.0)) and ratio3[-1] < 1.0 / 3.0,
      "int p3 / int rho rises along the history and stays below 1/3")
```

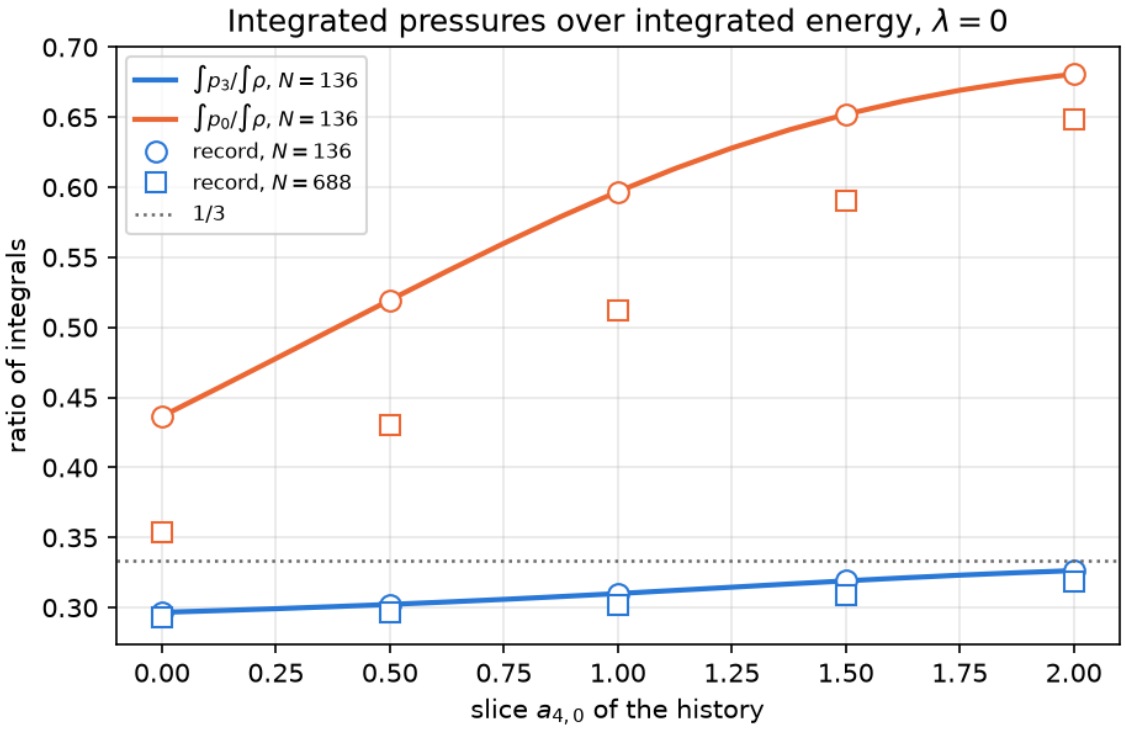

Figure 423: The integrated 3-space pressure (blue) and hidden-direction pressure (orange), each divided by the integrated energy density (vertical axis, pure numbers), against the slice $a_{4,0}$ (horizontal axis), for $N = 136$ without interaction at 17 slices (lines) and for $N = 136$ (circles) and $N = 688$ (squares) from the record. The 3-space ratio rises from about 0.30 toward the value 1/3 of a gas of massless particles as the momenta redshift. These are integrated components on a prescribed background, not an observed equation of state. (Notebook 15b, figure 5.)

Out [9]:

```
Figure 15b.5 saved as Revision/textbook/figures/15b_5_integrated_ratios.png
RESULT int p3 / int rho of N = 136 at a4,0 = 0, 1, 2 = 0.2966, 0.3100, 0.3264
RESULT int p0 / int rho of N = 136 at a4,0 = 0, 1, 2 = 0.4365, 0.5969, 0.6808
PASS the ratios at the five slices equal the record
     reproduces Revision/kohn_sham/results/ground/emt-integrals.csv, int_p3 / int_rho
PASS int p3 / int rho rises along the history and stays below 1/3
```

Text cell:

```
## 11. Where the energy sits along the history

The next cell reads the committed profiles of $N = 136$, $\lambda = 0$ at the five
slices and draws the coordinate energy density divided by the total energy,
$2\,\mathrm{Vol}_7 e^{6Hy}\rho(y)/E_{KS}$, whose area is 1. As the brane band
redshifts, its orbitals reach further from the brane toward the tip. The check
measures this with the mean position $\langle y \rangle = \int y \cdot (\text{this
density})\,dy$, which must decrease from slice to slice.
```

In [10]:

```
fig, ax = plt.subplots()
means = []
for shade, a4 in zip(SLICE_SHADES, (0.0, 0.5, 1.0, 1.5, 2.0)):
    p = read_profile(repository_file(f"{RECORD_PROFILES}/N136_lam0_a"
                                     f"{round(10 * a4):02d}.csv"))
    density = 2.0 * VOL7 * np.exp(6.0 * H * p["y"]) * p["rho"]
    density = density / simpson(density, step)  # area 1
    means.append(simpson(p["y"] * density, step))
    ax.plot(p["y"], density, color=shade, lw=2.0, label=f"$a_{{4,0}} = {a4}$")
ax.set_xlabel("hidden coordinate $y$")
ax.set_ylabel("$2\\,\\mathrm{Vol}_7 e^{6Hy}\\rho / E_{KS}$ (units of $m$)")
ax.set_title("The energy spreads toward the tip along the history")
ax.legend()
save_figure(fig, "energy_spreading",
            "The energy per unit $y$ divided by the total energy, "
            "$2\\,\\mathrm{Vol}_7 e^{6Hy}\\rho(y)/E_{KS}$ (vertical axis, area 1), "
            "of $N = 136$ without interaction at the five slices $a_{4,0} = 0$ to "
            "$2$ (light to dark blue), against $y$ (horizontal axis). The energy is "
            "concentrated near the brane, and its distribution moves toward the tip "
            "as the history proceeds.")
report("mean position <y> of the energy at the five slices",
       ", ".join(f"{m:.4f}" for m in means))
check(all(means[i + 1] < means[i] for i in range(4)),
      "the mean position of the energy moves toward the tip from slice to slice")
```

Out [10]:

```
Figure 15b.6 saved as Revision/textbook/figures/15b_6_energy_spreading.png
RESULT mean position <y> of the energy at the five slices = -0.3873, -0.4263, -0.4556,
    -0.4746, -0.4849
PASS the mean position of the energy moves toward the tip from slice to slice
```

Text cell:

```
## 12. The interaction terms

With interaction the energy density $e_{int}$ enters every component: it is
```

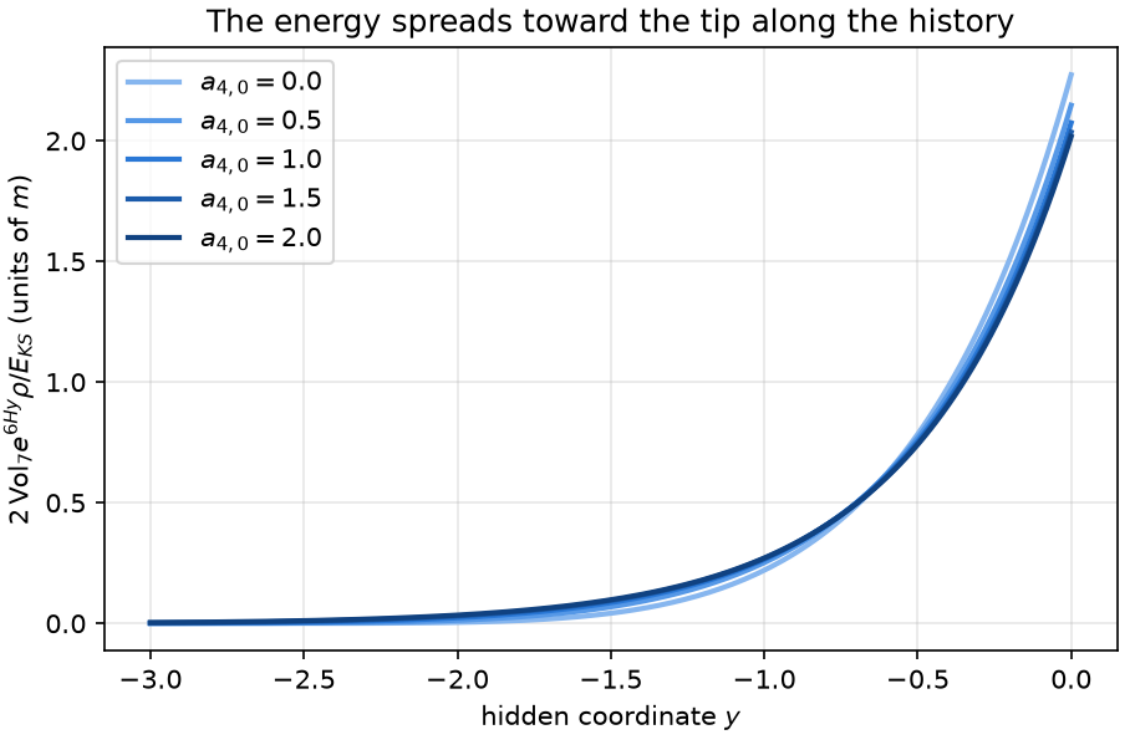

Figure 424: The energy per unit y divided by the total energy, $2\,\mathrm{Vol}_7 e^{6Hy}\rho(y)/E_{KS}$ (vertical axis, area 1), of $N = 136$ without interaction at the five slices $a_{4,0} = 0$ to 2 (light to dark blue), against y (horizontal axis). The energy is concentrated near the brane, and its distribution moves toward the tip as the history proceeds. (Notebook 15b, figure 6.)

```
subtracted in $\rho$ and added to $p_3$, $p_0$ and, alone, forms $p_t$. The next
cell uses the strongly interacting state $N = 136$, $+\lambda_2$, $a_{4,0} = 2$: it
draws $e_{int}$ and its two parts $\tfrac{15}{32}\lambda S^2$ and
$-\tfrac{1}{32}\lambda n^2$ (computed here from the profile columns $S$ and $n$),
checks that the column $e_{int}$ of the profile is this sum and that $p_t = e_{int}$
exactly, and checks the conservation law for this state too (where $p_t \ne 0$).
Finally it counts, among the 60 interacting ground states of the committed table
‘ground/emt-integrals.csv‘, how many have a positive and how many a negative
integrated extra-time pressure $2\,\mathrm{Vol}_7\int e^{6Hy}p_t\,dy$: both signs
occur, so the deflation of the extra times can give energy to the gas or take
energy from it.
```

In [11]:

```
sp = read_profile(RUN_FOLDER / "N136_lamp2_a20.csv")
hartree = LAMBDA2_136 * 15.0 / 32.0 * sp["S"] ** 2  # (15/32) lambda S^2
exchange = -LAMBDA2_136 / 32.0 * sp["n"] ** 2  # -(1/32) lambda n^2
fig, ax = plt.subplots()
ax.plot(sp["y"], sp["e_int"], color=PALETTE[0], lw=3.0, label="$e_{int} = p_t$")
ax.plot(sp["y"], hartree, color=PALETTE[1], lw=1.5, ls="--",
        label="$\\frac{15}{32}\\lambda S^2$")
ax.plot(sp["y"], exchange, color=PALETTE[2], lw=1.5, ls="--",
        label="$-\\frac{1}{32}\\lambda n^2$")
ax.set_yscale("symlog", linthresh=1e-2)
ax.set_xlabel("hidden coordinate $y$")
ax.set_ylabel("proper energy density (units of $m^8$)")
ax.set_title("Interaction energy density, $N = 136$, $+\\lambda_2$, $a_{4,0} = 2$")
ax.legend()
save_figure(fig, "interaction_terms",
            "The interaction energy density $e_{int}$, which is also the extra-time "
            "pressure $p_t$ (solid), and its two parts, the positive term "
            "$\\frac{15}{32}\\lambda S^2$ and the negative term "
            "$-\\frac{1}{32}\\lambda n^2$ (dashed), for $N = 136$ with the "
            "repulsive coupling $+\\lambda_2$ at $a_{4,0} = 2$ (vertical axis, "
            "symmetric logarithmic, proper units $m^8$; horizontal axis $y$). The "
            "interaction lives near the tip, where the proper densities are large.")
sum_error = float(np.max(np.abs(hartree + exchange - sp["e_int"]))) / float(
    np.max(np.abs(sp["e_int"])))
_, _, residual2, integrated2 = conservation(sp)
report("largest |e_int - (15/32 lambda S^2 - 1/32 lambda n^2)| / max|e_int|",
       f"{sum_error:.1e}")
report("conservation law with interaction: pointwise / integrated residual",
       f"{float(np.max(residual2)):.1e} / {integrated2:.1e}")
check(sum_error < 1e-12 and bool(np.all(sp["p_t"] == sp["e_int"])),
      "e_int = lambda (15/32 S^2 - 1/32 n^2) and p_t = e_int")
check(float(np.max(residual2)) < 1e-4 and integrated2 < 1e-6,
      "the conservation law holds for the interacting state N136_lamp2_a20",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, checks "
             "emt_y_conservation_pointwise and emt_y_conservation_integrated")
int_pt = [float(row["int_p_t"]) for key, row in emt_rows.items()
          if "_lam0_" not in key]  # the 60 interacting ground states
positive = sum(1 for value in int_pt if value > 0.0)
negative = sum(1 for value in int_pt if value < 0.0)
```

```
report("interacting states with integrated p_t > 0 / < 0", f"{positive} / {negative}")
check(len(int_pt) == 60 and positive > 0 and negative > 0,
      "the integrated extra-time pressure takes both signs",
      record=f"{EMT_TABLE}, column int_p_t")
```

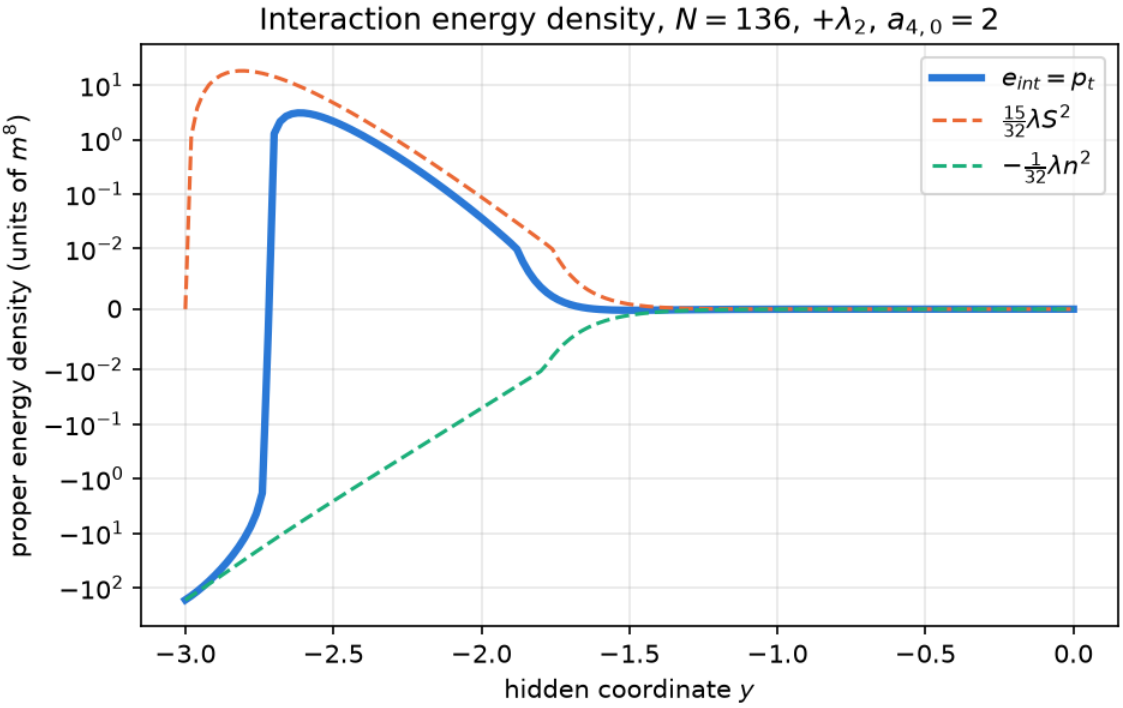

Figure 425: The interaction energy density e_{int}, which is also the extra-time pressure p_t (solid), and its two parts, the positive term $\frac{15}{32}\lambda S^2$ and the negative term $-\frac{1}{32}\lambda n^2$ (dashed), for $N = 136$ with the repulsive coupling $+\lambda_2$ at $a_{4,0} = 2$ (vertical axis, symmetric logarithmic, proper units m^8; horizontal axis y). The interaction lives near the tip, where the proper densities are large. (Notebook 15b, figure 7.)

Out [11]:

```
Figure 15b.7 saved as Revision/textbook/figures/15b_7_interaction_terms.png
RESULT largest |e_int - (15/32 lambda S^2 - 1/32 lambda n^2)| / max|e_int| = 6.8e-16
RESULT conservation law with interaction: pointwise / integrated residual = 5.0e-06 /
    2.3e-08
PASS e_int = lambda (15/32 S^2 - 1/32 n^2) and p_t = e_int
PASS the conservation law holds for the interacting state N136_lamp2_a20
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, checks
    emt_y_conservation_pointwise and emt_y_conservation_integrated
RESULT interacting states with integrated p_t > 0 / < 0 = 29 / 31
PASS the integrated extra-time pressure takes both signs
     reproduces Revision/kohn_sham/results/ground/emt-integrals.csv, column int_p_t
```

Text cell:

```
## 13. Why this tensor cannot drive the history

The field equations for $a_4$ (the Einstein-Lovelock equations of the author's
metric) accept a source only under three conditions (Revision record
Revision/field_equations_a4/reports/ks-source-conditions.json, key `conditions`):

- (C1) every component of the source is independent of $x_0$, that is, of $y$;
- (C2) $p_3 + p_t = 2 p_0$ at every point;
- (C3) for the linear history $a_4 = AHx_4 + a_0$ used here: $p_3 = p_t = p_0$
  and a constant $\rho$.

The next cell tests the three conditions on this notebook's own numbers: the
profile of $N = 136$, $\lambda = 0$, $a_{4,0} = 1$ for C1 and C2 point by point,
and the 17 slices of the history for C2 after the integration over $y$ and for C3.
It prints the profile at $y = -3, -1.5, 0$ and checks these values and the
integrated ratios $(\int p_3 + \int p_t)/(2\int p_0)$ at $a_{4,0} = 0, 1, 2$
against the numbers printed in the record's checks (6 significant digits). All
three conditions fail, as the record says: the Kohn-Sham gas is a test field on a
PRESCRIBED BACKGROUND, not a source that would produce the deflating history.
```

In [12]:

```
import re  # finds the numbers inside the record's text

SOURCE = "Revision/field_equations_a4/reports/ks-source-conditions.json"
source = json.loads(repository_file(SOURCE).read_text(encoding="utf-8"))
details = {item["name"]: item["detail"] for item in source["checks"]}
t_scale = max(float(np.max(np.abs(prof[c]))) for c in ("rho", "p3", "p_t", "p0"))
c1 = (float(np.max(prof["rho"])) - float(np.min(prof["rho"]))) / t_scale  # C1 test
c2 = prof["p3"] + prof["p_t"] - 2.0 * prof["p0"]  # zero everywhere if C2 held
number = r"(-?[0-9.]+(?:e[-+]?[0-9]+)?)"  # a number as the record prints it
examples = re.findall(
    rf"y = {number}: rho = {number}, p3 = {number}, p_t = {number}, "
    rf"p0 = {number}, p3 \+ p_t - 2 p0 = {number}",
    details["ks_profiles_violate_algebraic_condition"])
say("    y         rho          p3      p_t          p0  p3 + p_t - 2 p0")
worst_example = 0.0
for example in examples:
    y0, values = float(example[0]), [float(v) for v in example[1:]]
    i = int(np.argmin(np.abs(y - y0)))  # the profile point at this y
    mine = [float(prof["rho"][i]), float(prof["p3"][i]), float(prof["p_t"][i]),
            float(prof["p0"][i]), float(c2[i])]
    say(f"{y[i] + 0.0:5.1f}  {mine[0]:10.6g}  {mine[1]:10.6g}"  # + 0.0 turns -0 into 0
        f"  {mine[2] + 0.0:7.3g}  {mine[3]:10.6g}  {mine[4]:15.6g}")
    for a, b in zip(mine, values):  # 6 significant digits: relative 5e-6
        worst_example = max(worst_example, abs(a - b) / max(abs(b), 1e-300)
                            if b != 0.0 else abs(a))
ratios_c2 = {item["a4"]: (item["p3"] + item["p_t"]) / (2.0 * item["p0"])
             for item in history}
recorded_c2 = {int(key[-2:]) / 10.0: float(value) for key, value in re.findall(
    r"(N136_lam0_a[0-9][0-9]): ([0-9.]+)",
    details["ks_integrals_violate_algebraic_condition"])}
worst_c2 = max(abs(ratios_c2[a4] / value - 1.0) for a4, value in recorded_c2.items())
```

```
report("C1: (max rho - min rho) / max|T| for N136_lam0_a10", f"{c1:.4f}")
report("C2 integrated: (int p3 + int p_t)/(2 int p0) at a4,0 = 0, 1, 2",
       ", ".join(f"{ratios_c2[a]:.6f}" for a in (0.0, 1.0, 2.0)))
report("C3: int rho at a4,0 = 0 and 2; int p3 and int p_t at a4,0 = 1",
       f"{history[0]['rho']:.4f}, {history[-1]['rho']:.4f}; "
       f"{history[8]['p3']:.4f}, {history[8]['p_t']:.1f}")
check(c1 > 0.01 and len(examples) == 3 and worst_example < 5e-6
      and float(np.min(np.abs(c2[[0, len(c2) // 2, -1]]))) > 0.0,
      "C1 and C2 fail for N136_lam0_a10; the record's example values reproduced",
      record=f"{SOURCE}, checks ks_profiles_depend_on_x0 and "
             "ks_profiles_violate_algebraic_condition")
check(len(recorded_c2) == 3 and worst_c2 < 5e-6
      and all(abs(r - 1.0) > 0.5 for r in ratios_c2.values()),
      "C2 fails after integration at all 17 slices; the record's ratios reproduced",
      record=f"{SOURCE}, check ks_integrals_violate_algebraic_condition")
check(history[-1]["rho"] < 0.2 * history[0]["rho"]
      and all(item["p3"] > 0.25 * item["rho"] and item["p_t"] == 0.0
              for item in history),
      "C3 fails: rho changes along the history and p3 differs from p_t",
      record=f"{SOURCE}, check ks_history_is_a_prescribed_background")
```

Out [12]:

```
    y         rho          p3      p_t          p0  p3 + p_t - 2 p0
 -3.0     43.0702     158.268        0    -431.733          1021.73
 -1.5    0.707818    0.498835        0    0.643324        -0.787813
  0.0  0.00211576  0.000391237        0  0.000942049      -0.00149286
RESULT C1: (max rho - min rho) / max|T| for N136_lam0_a10 = 0.0998
RESULT C2 integrated: (int p3 + int p_t)/(2 int p0) at a4,0 = 0, 1, 2 = 0.339767,
    0.259690, 0.239714
RESULT C3: int rho at a4,0 = 0 and 2; int p3 and int p_t at a4,0 = 1 = 80.2822, 12.4451;
    10.0397, 0.0
PASS C1 and C2 fail for N136_lam0_a10; the record's example values reproduced
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, checks
    ks_profiles_depend_on_x0 and ks_profiles_violate_algebraic_condition
PASS C2 fails after integration at all 17 slices; the record's ratios reproduced
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_integrals_violate_algebraic_condition
PASS C3 fails: rho changes along the history and p3 differs from p_t
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_history_is_a_prescribed_background
```

Text cell:

```
## 14. The last check

The last cell checks that every figure file of this notebook exists and prints the
number of checks that passed.
```

In [13]:

```
NAMES = ["proper_emt", "coordinate_emt", "y_conservation", "energy_slopes",
         "integrated_ratios", "energy_spreading", "interaction_terms"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15b_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

Out [13]:

```
PASS every figure file of this notebook exists
ALL 18 CHECKS PASSED (notebook 15b)
```

Text cell:

```
## 15. What this notebook showed

- The solver's profiles of the energy density and the three pressures are
  reproduced on this computer, and their integrals give the Kohn-Sham energy
  (COMPUTED).
- The conservation law along the hidden direction,
  $(e^{6Hy}p_0)' = 3H e^{6Hy}(p_3 + p_t)$, holds point by point and integrated, with
  and without interaction (an exact identity, checked numerically).
- Along the deflating history the energy of the gas changes exactly by the work of
  the pressure difference, $dE/da_4 = -3 \cdot 2\,\mathrm{Vol}_7\int e^{6Hy}(p_3 -
  p_t)\,dy$: the 3-space pressure takes energy from the gas as 3-space inflates,
  and the extra-time pressure, which is only the interaction energy density (zero
  without interaction, of either sign with it), adds or removes a small amount as
  the extra times deflate.
- The integrated ratio $\int p_3/\int\rho$ rises toward $1/3$ as the momenta
  redshift; the energy moves toward the tip.
- The history is PRESCRIBED: these components are not an admissible source of the
  equations for $a_4$ (they depend on $y$, $p_3 + p_t \ne 2p_0$, and along the
  history $\rho$ changes and $p_3 \ne p_t$; checked here on the notebook's own
  numbers), and no equation of state seen by an observer is claimed here.
```

15.20 Line-by-line walk-through of Notebook 15b

The notebook has 13 code cells, In [1] to In [13]. As before, docstrings are left out of the quotations and long caption strings are shortened; the complete cells are printed in Section 15.19.

In [1], the set-up cell. It is the set-up cell of Notebook 15a (Section 15.14, which explains the two Rust lines and the function `rust_program`; the rest is explained in Section 15.9), with the run instructions of this notebook (Section 15.18) in its comments and `NOTEBOOK_ID = "15b"`.

In [2], the solver and two profiles.

```
import csv  # reads the tables (CSV files)
import math  # exp and pi for single numbers

import numpy as np  # arrays of numbers
```

```
program = rust_program("Revision/kohn_sham/solver/Cargo.toml", "revision_ks_solver")
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_15b"  # ignored by git
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
```

The imports, the solver built (about a second when it is up to date), and the folder `textbook_15b` inside the solver's build folder, which git ignores, for the files of this notebook's runs.

```
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
SLICE_SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light->dark
VOL7 = (2.0 * math.pi / 0.25) ** 3  # proper 7-volume per unit e^{6Hy}
H = 1.0
```

The colours (as in Notebook 15a), $\mathrm{Vol}_7 = 15875.2137$ and $H = 1$.

```
def solve_state(name, n, lam, a4, profiles=False):
    out = RUN_FOLDER / f"{name}.json"
    command = [str(program), "single", "--root", str(REPO), "--m", "1",
               "--lambda", repr(lam), "--a4", repr(a4), "--N", repr(float(n)),
               "--out", str(out)]
    if profiles:
        command += ["--profiles", str(RUN_FOLDER / f"{name}.csv")]
    done = subprocess.run(command, capture_output=True, text=True)
    if done.returncode != 0 or done.stdout.strip().split("\n")[-1] != "SUCCESS":
        raise RuntimeError(f"the solver failed for {name}: {done.stderr[-500:]}")
    return json.loads(out.read_text(encoding="utf-8"))
```

`solve_state` runs the solver's command `single` for one state and returns its JSON record as a dictionary. The command list holds the program, the word `single`, the repository, the mass 1, the coupling, the slice and the particle number; `repr(x)` writes a number with all the digits needed to read it back exactly. If profiles are wanted, the option that names a CSV file for them is appended (`+=` extends the list). If the program fails, or its last printed line is not SUCCESS, the notebook stops with the end of its error message.

```
params = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                    .read_text(encoding="utf-8"))
LAMBDA2_136 = {int(c["N"]): c for c in params["couplingCalibration"]["values"]}[136][
    "lambda2"]  # the coupling lambda_2 of N = 136
free_state = solve_state("N136_lam0_a10", 136, 0.0, 1.0, profiles=True)
strong_state = solve_state("N136_lamp2_a20", 136, LAMBDA2_136, 2.0, profiles=True)
say(f"solved N136_lam0_a10 ({free_state['iterations']} iteration) and N136_lamp2_a20 "
    f"({strong_state['iterations']} iterations, lambda = {LAMBDA2_136})")
```

The coupling $\lambda_2 = 0.002789$ of $N = 136$ is read from the record. Two states are solved with profiles: the free state at $a_{4,0} = 1$ and the strongly interacting state at $a_{4,0} = 2$. Out [2] reports 1 iteration for the free state (without interaction one iteration is exact) and 17 for the other.

In [3], comparison with the committed profiles.

```
RECORD_PROFILES = "Revision/kohn_sham/results/ground/profiles"


def read_profile(path):
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}
```

`read_profile` is the function of Notebook 15a (Section 15.14, In [12]), here with a path as its argument.

```
worst = 0.0
identical = []
for name in ("N136_lam0_a10", "N136_lamp2_a20"):
    new = read_profile(RUN_FOLDER / f"{name}.csv")
    old = read_profile(repository_file(f"{RECORD_PROFILES}/{name}.csv"))
    for column in old:
        scale = max(float(np.max(np.abs(old[column]))), 1e-300)
        worst = max(worst, float(np.max(np.abs(new[column] - old[column]))) / scale)
    identical.append((RUN_FOLDER / f"{name}.csv").read_bytes()
                     == repository_file(f"{RECORD_PROFILES}/{name}.csv").read_bytes())
```

For both states and every column the largest difference between the new and the committed profile is divided by the largest value of the column (or by 10^{-300}, so that a column of zeros does not cause a division by zero), and the largest such ratio is kept. The byte identity of each pair of files is recorded in the list `identical`.

```
say("columns: " + ", ".join(old))
report("largest relative difference to the record", f"{worst:.1e}")
report("profile files byte-identical to the record", identical)
check(worst < 1e-9, "the new profiles equal the committed profiles within 1e-9",
      record=f"{RECORD_PROFILES}/N136_lam0_a10.csv and N136_lamp2_a20.csv")
```

Out [3] lists the twelve columns (y, n, S, Q, M, v, w_Q, e_{int}, ρ, p_3, p_t, p_0; Q and w_Q belong to the exact-Fock variant), a difference of 0.0, the files byte-identical, and PASS.

In [4], the proper densities.

```
prof = read_profile(RUN_FOLDER / "N136_lam0_a10.csv")
y = prof["y"]
step = float(y[1] - y[0])  # 0.02
COMPONENTS = [("rho", "$\\rho$"), ("p3", "$p_3$"), ("p_t", "$p_t$"), ("p0", "$p_0$")]
fig, ax = plt.subplots()
for colour, (column, label) in zip(PALETTE, COMPONENTS):
    ax.plot(y, prof[column], color=colour, lw=2.0, label=label)
ax.set_yscale("symlog", linthresh=1e-3)
```

The free profile is read; `y` are its 151 coordinates with the spacing `step` $= 0.02$. `COMPONENTS` pairs the column names with their labels, and the four components are drawn on a symmetric logarithmic axis (linear between -10^{-3} and 10^{-3}), because they span six powers of ten and p_0 changes sign.

```
ax.set_xlabel("hidden coordinate $y$ (tip at $-3$, brane at $0$)")
ax.set_ylabel("proper density (units of $m^8$)")
ax.set_title("Energy density and pressures, $N = 136$, $\\lambda = 0$, $a_{4,0} = 1$")
ax.legend()
save_figure(fig, "proper_emt",
            "The energy density $\\rho$ and the pressures $p_3$, $p_t$, $p_0$ of the "
...)
report("p_t without interaction: largest |p_t|", float(np.max(np.abs(prof["p_t"]))))
check(float(np.max(np.abs(prof["p_t"]))) == 0.0, "p_t = e_int = 0 when lambda = 0")
```

The labels (a proper energy density has the unit m^8 in eight dimensions), Figure 15b.1, and the check that p_t is exactly zero without interaction (Out [4]). In the figure all components are largest at the tip, and p_0 is negative near the tip and changes sign near $y = -2.45$.

In [5], the coordinate densities and their integrals.

```
def simpson(values, h):
    weights = np.full(len(values), 2.0)
    weights[1::2] = 4.0
    weights[0] = weights[-1] = 1.0
    return h / 3.0 * float(np.sum(weights * values))


def read_rows(relative):
    with open(repository_file(relative), newline="", encoding="utf-8") as handle:
        return {row["id"]: row for row in csv.DictReader(handle)}
```

`simpson` is Simpson's rule (Section 15.14, In [12]); `read_rows` reads a committed table of the record and files its rows by state name.

```
EMT_TABLE = "Revision/kohn_sham/results/ground/emt-integrals.csv"
emt_rows = read_rows(EMT_TABLE)
summary_rows = read_rows("Revision/kohn_sham/results/ground/summary.csv")
weight = 2.0 * VOL7 * np.exp(6.0 * H * y)  # 2 Vol_7 e^{6Hy}
fig, ax = plt.subplots()
integrals = {}
for colour, (column, label) in zip(PALETTE, COMPONENTS):
    ax.plot(y, weight * prof[column], color=colour, lw=2.0, label=label)
    integrals[column] = simpson(weight * prof[column], step)
```

The record's table of the integrals $2\,\mathrm{Vol}_7 \int e^{6Hy}(\cdot)\,dy$ and its summary table are read. `weight` is $2\,\mathrm{Vol}_7 e^{6Hy}$, which turns a proper density into the amount per unit y in the doubled box. Each component times the weight is drawn and integrated.

```
ax.axhline(0.0, color="0.4", lw=0.8)
ax.set_xlabel("hidden coordinate $y$")
ax.set_ylabel("$2\\,\\mathrm{Vol}_7\\, e^{6Hy} \\times$ component (units of $m$)")
ax.set_title("The integrands: energy and pressures per unit $y$")
ax.legend()
save_figure(fig, "coordinate_emt",
```

```
            "The energy density and the pressures of the state $N = 136$, "
...)
row = emt_rows["N136_lam0_a10"]
rel = lambda a, b: abs(a - b) / max(abs(b), 1e-300)
worst_int = max(rel(integrals[c], float(row[f"int_{c}"])) for c in ("rho", "p3", "p0"))
```

A zero line, labels, Figure 15b.2. `rel` is the relative difference of two numbers, and `worst_int` the largest relative difference between this notebook's integrals of ρ, p_3, p_0 and the record's columns `int_rho`, `int_p3`, `int_p0`.

```
report("E_KS from the profile / from the solver",
       f"{integrals['rho']:.8f} / {free_state['E_KS']:.8f}")
report("largest relative difference of the integrals to the record", f"{worst_int:.1e}")
check(rel(free_state["E_KS"], float(summary_rows["N136_lam0_a10"]["E_KS"])) < 1e-12,
      "the solver's E_KS equals the record",
      record="Revision/kohn_sham/results/ground/summary.csv, N136_lam0_a10")
check(rel(integrals["rho"], free_state["E_KS"]) < 1e-6 and worst_int < 1e-6,
      "Simpson's rule on the profile gives E_KS and the recorded integrals",
      record=f"{EMT_TABLE}, columns int_rho, int_p3, int_p0")
```

Out [5]: the integral of the energy density over the 151 profile points is 32.38411614 against the solver's $E_{KS} = 32.38411569$ (a relative difference of 1.4×10^{-8}, the error of Simpson's rule on the coarse profile grid); both checks pass. In Figure 15b.2 the energy and the pressures sit near the brane.

In [6], the conservation law along y.

```
def derivative4(f, h):
    d = np.zeros(len(f))
    d[2:-2] = (f[:-4] - 8.0 * f[1:-3] + 8.0 * f[3:-1] - f[4:]) / (12.0 * h)
    d[0] = (-25 * f[0] + 48 * f[1] - 36 * f[2] + 16 * f[3] - 3 * f[4]) / (12.0 * h)
    d[1] = (-3 * f[0] - 10 * f[1] + 18 * f[2] - 6 * f[3] + f[4]) / (12.0 * h)
    d[-1] = (25 * f[-1] - 48 * f[-2] + 36 * f[-3] - 16 * f[-4] + 3 * f[-5]) / (12.0 * h)
    d[-2] = (3 * f[-1] + 10 * f[-2] - 18 * f[-3] + 6 * f[-4] - f[-5]) / (12.0 * h)
    return d
```

`derivative4` computes the derivative of equally spaced values with error proportional to h^4. At the inner points it uses $f'(y_i) \approx [f_{i-2} - 8f_{i-1} + 8f_{i+1} - f_{i+2}]/(12h)$; `f[:-4]`, `f[1:-3]`, `f[3:-1]` and `f[4:]` are the arrays shifted so that position i of each holds f_{i-2}, f_{i-1}, f_{i+1}, f_{i+2} for the points 2 to 148. At the two first and two last points, where two neighbours on one side are missing, one-sided formulas of the same order use five points on one side. (Each of these formulas is the derivative of the polynomial of degree 4 through the five points.)

```
def conservation(p):
    e6 = np.exp(6.0 * H * p["y"])
    left = derivative4(e6 * p["p0"], step)
    right = 3.0 * H * e6 * (p["p3"] + p["p_t"])
    scale = max(float(np.max(np.abs(left))), float(np.max(np.abs(right))))
    jump = e6[-1] * p["p0"][-1] - e6[0] * p["p0"][0]
    integral = simpson(right, step)
    return left, right, np.abs(left - right) / scale, abs(jump - integral) / abs(jump)
```

`conservation` returns, for a profile, the left side $(e^{6Hy}p_0)'$, the right side $3H\,e^{6Hy}(p_3+p_t)$, their difference at every point divided by the largest value of the two sides, and the integrated residual: the jump $[e^{6Hy}p_0]_{-3}^{0}$ compared with the integral of the right side, relative to the jump.

```
left, right, residual, integrated = conservation(prof)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
ax1.plot(y, left, color=PALETTE[0], lw=3.0, label="$(e^{6Hy}p_0)'$")
ax1.plot(y, right, color=PALETTE[1], lw=1.5, ls="--",
         label="$3H\\,e^{6Hy}(p_3 + p_t)$")
ax1.set_xlabel("hidden coordinate $y$")
ax1.set_ylabel("units of $m^9$")
ax1.set_title("The two sides of the law")
ax1.legend()
```

The left panel draws both sides, the left side as a thick line and the right side dashed on top of it.

```
ax2.semilogy(y, np.maximum(residual, 1e-16), color=PALETTE[2], lw=1.5)
ax2.axhline(1e-4, color="0.4", ls=":", lw=1.2, label="tolerance $10^{-4}$")
ax2.set_xlabel("hidden coordinate $y$")
ax2.set_ylabel("|left - right| / largest value")
ax2.set_title("Relative residual on the 151 points")
ax2.legend()
save_figure(fig, "y_conservation",
            "Left: the two sides of the conservation law along the hidden "
...)
report("largest pointwise residual / integrated residual",
       f"{float(np.max(residual)):.1e} / {integrated:.1e}")
check(float(np.max(residual)) < 1e-4 and integrated < 1e-6,
      "(e^{6Hy} p0)' = 3H e^{6Hy} (p3 + p_t) holds for N136_lam0_a10",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, checks "
             "emt_y_conservation_pointwise and emt_y_conservation_integrated")
```

The right panel draws the relative residual on a logarithmic axis (`np.maximum(residual, 1e-16)` raises exact zeros to 10^{-16}, because a logarithmic axis cannot show 0), with the tolerance 10^{-4} as a dotted line; then Figure 15b.3. Out [6]: the largest pointwise residual is 2.5×10^{-8} and the integrated one 6.1×10^{-10}, far below the tolerances; the law holds.

In [7], the energy at 17 slices and its slope.

```
DENSE = [i / 8.0 for i in range(17)]  # the slices 0, 0.125, ..., 2
history = []
for a4 in DENSE:
    record = solve_state(f"N136_lam0_dense_{round(1000 * a4):04d}", 136, 0.0, a4)
    ints = record["emtIntegrals_2Vol7_int_e6Hy"]  # 2 Vol_7 int e^{6Hy} (...) dy
    history.append({"a4": a4, "E": record["E_KS"], "rho": ints["rho"],
                    "p3": ints["p3"], "p_t": ints["p_t"], "p0": ints["p0"],
                    "slope": -3.0 * (ints["p3"] - ints["p_t"])})
```

The free state $N=136$ is solved at the 17 slices $0, 0.125, \ldots, 2$ (the names hold a thousand times the slice with four digits). From each JSON record the cell keeps the energy, the integrals of the

four components that the solver computed on its fine grid, and the slope $-3(\int p_3 - \int p_t)$ that the energy-change law predicts for dE/da_4.

```
adiabatic_rows = read_rows("Revision/kohn_sham/results/adiabatic/adiabaticity.csv")
worst_e, worst_slope = 0.0, 0.0
say("a4,0       E_KS     dE/da4 (EMT)   dE/da4 (record, finite differences)")
for item in history:
    if item["a4"] in (0.0, 0.5, 1.0, 1.5, 2.0):
        key = f"N136_lam0_a{round(10 * item['a4']):02d}"
        recorded_slope = float(adiabatic_rows[key]["dE_da4_finite_difference"])
        worst_e = max(worst_e, rel(item["E"], float(summary_rows[key]["E_KS"])),
                      rel(item["p3"], float(emt_rows[key]["int_p3"])))
        worst_slope = max(worst_slope, rel(item["slope"], recorded_slope))
        say(f"{item['a4']:4.2f}  {item['E']:10.5f}  {item['slope']:12.6f}"
            f"  {recorded_slope:12.6f}")
```

At the five slices of the canonical matrix the cell compares the energy and $\int p_3$ with the record, and the predicted slope with the derivative that the solver obtained from the energies at neighbouring slices (Richardson differences with the step $\delta = 0.002$, column `dE_da4_finite_difference`), and prints a table.

```
report("largest relative difference of E and int p3 to the record", f"{worst_e:.1e}")
report("largest relative difference of the slopes", f"{worst_slope:.1e}")
check(worst_e < 1e-12, "E_KS and the integrals at the five slices equal the record",
      record="Revision/kohn_sham/results/ground/summary.csv and emt-integrals.csv")
check(worst_slope < 1e-8, "dE/da4 = -3 (int p3 - int p_t) at the five slices",
      record="Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column "
             "dE_da4_finite_difference")
```

Out [7]: the energies and integrals equal the record exactly, and the slopes from the pressures agree with the finite differences to 3.7×10^{-12}; the slope is -71.44 at $a_{4,0} = 0$ and -12.19 at $a_{4,0} = 2$.

In [8], the slopes integrated over the history.

```
slopes = np.array([item["slope"] for item in history])
energies = np.array([item["E"] for item in history])
work = simpson(slopes, 0.125)  # integral of dE/da4 over the history
change = energies[-1] - energies[0]
```

Simpson's rule on the 17 slopes (spacing 0.125) integrates dE/da_4 from 0 to 2; the result must equal the change $E(2) - E(0)$ of the energy.

```
fig, ax = plt.subplots()
ax.plot(DENSE, energies, "o", color=PALETTE[0], ms=8, label="$E_{KS}$ (solver)")
for a4, e, s in zip(DENSE, energies, slopes):
    ax.plot([a4 - 0.05, a4 + 0.05], [e - 0.05 * s, e + 0.05 * s], color=PALETTE[1],
            lw=2.5)
ax.plot([], [], color=PALETTE[1], lw=2.0,
        label="slope $-3 \\cdot 2\\mathrm{Vol}_7 \\int e^{6Hy}(p_3 - p_t)$")
```

The energies are drawn as dots; through each dot a short line from $a_{4,0} - 0.05$ to $a_{4,0} + 0.05$ with the predicted slope s (it rises by $0.05\,s$ on each side). An empty plot makes the legend entry.

```
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("$E_{KS}$ (units of $m$)")
ax.set_title("$N = 136$, $\\lambda = 0$: the pressures give the slope of the energy")
ax.legend()
save_figure(fig, "energy_slopes",
            "The Kohn-Sham energy $E_{KS}$ of $N = 136$ without interaction "
...)
report("E(2) - E(0) / integral of the slopes", f"{change:.6f} / {work:.6f}")
check(rel(work, change) < 1e-5,
      "the integrated slope equals E(2) - E(0) within 1e-5 (relative)",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "emt_energy_change_dE_da4")
```

Figure 15b.4; Out [8]: $E(2) - E(0) = -67.837152$ and the integral of the slopes -67.837207, a relative difference of 8×10^{-7} (the error of Simpson's rule with the coarse spacing 0.125). The energy the gas loses along the history is the work of its pressures.

In [9], the integrated pressure ratios.

```
ratio3 = np.array([item["p3"] / item["rho"] for item in history])
ratio8 = np.array([item["p0"] / item["rho"] for item in history])
fig, ax = plt.subplots()
ax.plot(DENSE, ratio3, color=PALETTE[0], lw=2.0,
        label="$\\int p_3 / \\int \\rho$, $N = 136$")
ax.plot(DENSE, ratio8, color=PALETTE[1], lw=2.0,
        label="$\\int p_0 / \\int \\rho$, $N = 136$")
```

The ratios $\int p_3 / \int \rho$ and $\int p_0 / \int \rho$ at the 17 slices, drawn as lines.

```
worst_ratio = 0.0
for marker, n in (("o", 136), ("s", 688)):
    keys = [f"N{n}_lam0_a{round(10 * a):02d}" for a in (0.0, 0.5, 1.0, 1.5, 2.0)]
    r3 = [float(emt_rows[k]["int_p3"]) / float(emt_rows[k]["int_rho"]) for k in keys]
    r8 = [float(emt_rows[k]["int_p0"]) / float(emt_rows[k]["int_rho"]) for k in keys]
    ax.plot([0, 0.5, 1, 1.5, 2], r3, marker, color=PALETTE[0], ms=8,
            markerfacecolor="white", label=f"record, $N = {n}$")
    ax.plot([0, 0.5, 1, 1.5, 2], r8, marker, color=PALETTE[1], ms=8,
            markerfacecolor="white")
    if n == 136:
        worst_ratio = max(abs(r3[i] - ratio3[4 * i]) for i in range(5))
```

The same ratios from the record's table at the five canonical slices are drawn as open circles ($N = 136$) and squares ($N = 688$). For $N = 136$ they are compared with this notebook's values; `ratio3[4 * i]` is the slice $0.5\,i$, because the dense slices have the spacing 0.125.

```
ax.axhline(1.0 / 3.0, color="0.4", ls=":", lw=1.2, label="$1/3$")
ax.set_xlabel("slice $a_{4,0}$ of the history")
```

```
ax.set_ylabel("ratio of integrals")
ax.set_title("Integrated pressures over integrated energy, $\\lambda = 0$")
ax.legend(fontsize=8)
save_figure(fig, "integrated_ratios",
            "The integrated 3-space pressure (blue) and hidden-direction pressure "
...)
report("int p3 / int rho of N = 136 at a4,0 = 0, 1, 2",
       ", ".join(f"{ratio3[i]:.4f}" for i in (0, 8, 16)))
report("int p0 / int rho of N = 136 at a4,0 = 0, 1, 2",
       ", ".join(f"{ratio8[i]:.4f}" for i in (0, 8, 16)))
check(worst_ratio < 1e-12, "the ratios at the five slices equal the record",
      record=f"{EMT_TABLE}, int_p3 / int_rho")
check(bool(np.all(np.diff(ratio3) > 0.0)) and ratio3[-1] < 1.0 / 3.0,
      "int p3 / int rho rises along the history and stays below 1/3")
```

A dotted line at 1/3, Figure 15b.5, the ratios at $a_{4,0} = 0, 1, 2$ (the dense indices 0, 8, 16) and two checks. Out [9]: $\int p_3 / \int \rho = 0.2966$, 0.3100, 0.3264 and $\int p_0 / \int \rho = 0.4365$, 0.5969, 0.6808; the ratios equal the record, and the 3-space ratio rises at every slice but stays below 1/3 (Section 15.16 explains why).

In [10], where the energy sits.

```
fig, ax = plt.subplots()
means = []
for shade, a4 in zip(SLICE_SHADES, (0.0, 0.5, 1.0, 1.5, 2.0)):
    p = read_profile(repository_file(f"{RECORD_PROFILES}/N136_lam0_a"
                                     f"{round(10 * a4):02d}.csv"))
    density = 2.0 * VOL7 * np.exp(6.0 * H * p["y"]) * p["rho"]
    density = density / simpson(density, step)  # area 1
    means.append(simpson(p["y"] * density, step))
    ax.plot(p["y"], density, color=shade, lw=2.0, label=f"$a_{{4,0}} = {a4}$")
```

For the five committed profiles of $N = 136$ without interaction the energy per unit y, $2\,\mathrm{Vol}_7 e^{6Hy}\rho$, is divided by its integral (so that the area under each curve is 1) and drawn; its mean position $\langle y \rangle = \int y \cdot (\text{density})\, dy$ is computed with Simpson's rule.

```
ax.set_xlabel("hidden coordinate $y$")
ax.set_ylabel("$2\\,\\mathrm{Vol}_7 e^{6Hy}\\rho / E_{KS}$ (units of $m$)")
ax.set_title("The energy spreads toward the tip along the history")
ax.legend()
save_figure(fig, "energy_spreading",
            "The energy per unit $y$ divided by the total energy, "
...)
report("mean position <y> of the energy at the five slices",
       ", ".join(f"{m:.4f}" for m in means))
check(all(means[i + 1] < means[i] for i in range(4)),
      "the mean position of the energy moves toward the tip from slice to slice")
```

Figure 15b.6; Out [10]: $\langle y \rangle = -0.3873$, -0.4263, -0.4556, -0.4746, -0.4849, moving toward the tip from slice to slice, as the redshifted band orbitals spread.

In [11], the interaction terms.

```
sp = read_profile(RUN_FOLDER / "N136_lamp2_a20.csv")
hartree = LAMBDA2_136 * 15.0 / 32.0 * sp["S"] ** 2  # (15/32) lambda S^2
exchange = -LAMBDA2_136 / 32.0 * sp["n"] ** 2  # -(1/32) lambda n^2
fig, ax = plt.subplots()
ax.plot(sp["y"], sp["e_int"], color=PALETTE[0], lw=3.0, label="$e_{int} = p_t$")
ax.plot(sp["y"], hartree, color=PALETTE[1], lw=1.5, ls="--",
        label="$\\frac{15}{32}\\lambda S^2$")
ax.plot(sp["y"], exchange, color=PALETTE[2], lw=1.5, ls="--",
        label="$-\\frac{1}{32}\\lambda n^2$")
ax.set_yscale("symlog", linthresh=1e-2)
```

For the strongly interacting profile the two parts of the interaction energy density are computed from the columns S and n: the positive part $\frac{15}{32}\lambda S^2$ (named `hartree` in the code; it holds the Hartree term and the scalar part of the exchange, Section 15.2) and the negative part $-\frac{1}{32}\lambda n^2$. Both are drawn dashed, with the column e_{int} as a thick line, on a symmetric logarithmic axis.

```
ax.set_xlabel("hidden coordinate $y$")
ax.set_ylabel("proper energy density (units of $m^8$)")
ax.set_title("Interaction energy density, $N = 136$, $+\\lambda_2$, $a_{4,0} = 2$")
ax.legend()
save_figure(fig, "interaction_terms",
            "The interaction energy density $e_{int}$, which is also the extra-time "
...)
sum_error = float(np.max(np.abs(hartree + exchange - sp["e_int"]))) / float(
    np.max(np.abs(sp["e_int"])))
_, _, residual2, integrated2 = conservation(sp)
report("largest |e_int - (15/32 lambda S^2 - 1/32 lambda n^2)| / max|e_int|",
       f"{sum_error:.1e}")
report("conservation law with interaction: pointwise / integrated residual",
       f"{float(np.max(residual2)):.1e} / {integrated2:.1e}")
```

Figure 15b.7; `sum_error` compares the sum of the two parts with the column e_{int}, and the conservation law is evaluated again for this state, in which $p_t \neq 0$.

```
check(sum_error < 1e-12 and bool(np.all(sp["p_t"] == sp["e_int"])),
      "e_int = lambda (15/32 S^2 - 1/32 n^2) and p_t = e_int")
check(float(np.max(residual2)) < 1e-4 and integrated2 < 1e-6,
      "the conservation law holds for the interacting state N136_lamp2_a20",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, checks "
             "emt_y_conservation_pointwise and emt_y_conservation_integrated")
int_pt = [float(row["int_p_t"]) for key, row in emt_rows.items()
          if "_lam0_" not in key]  # the 60 interacting ground states
positive = sum(1 for value in int_pt if value > 0.0)
negative = sum(1 for value in int_pt if value < 0.0)
report("interacting states with integrated p_t > 0 / < 0", f"{positive} / {negative}")
check(len(int_pt) == 60 and positive > 0 and negative > 0,
      "the integrated extra-time pressure takes both signs",
      record=f"{EMT_TABLE}, column int_p_t")
```

Out [11]: the sum matches to 6.8×10^{-16} and $p_t = e_{int}$ exactly; the conservation law holds with residuals 5.0×10^{-6} (pointwise) and 2.3×10^{-8} (integrated). Among the 60 interacting ground states

(every row whose name does not contain `_lam0_`) the integrated extra-time pressure is positive in 29 and negative in 31. In Figure 15b.7 the interaction lives near the tip.

In [12], the three source conditions.

```
import re  # finds the numbers inside the record's text

SOURCE = "Revision/field_equations_a4/reports/ks-source-conditions.json"
source = json.loads(repository_file(SOURCE).read_text(encoding="utf-8"))
details = {item["name"]: item["detail"] for item in source["checks"]}
```

`re` is Python's module for **regular expressions**, patterns that find pieces of text. The source-conditions record is read, and `details` files the detail text of each of its checks under the check name.

```
t_scale = max(float(np.max(np.abs(prof[c]))) for c in ("rho", "p3", "p_t", "p0"))
c1 = (float(np.max(prof["rho"])) - float(np.min(prof["rho"]))) / t_scale  # C1 test
c2 = prof["p3"] + prof["p_t"] - 2.0 * prof["p0"]  # zero everywhere if C2 held
```

For the free profile at $a_{4,0} = 1$: `t_scale` is the largest size of any component, `c1` the variation of ρ along y relative to it (zero if C1 held), and `c2` the array $p_3 + p_t - 2p_0$ (zero everywhere if C2 held).

```
number = r"(-?[0-9.]+(?:e[-+]?[0-9]+)?)"  # a number as the record prints it
examples = re.findall(
    rf"y = {number}: rho = {number}, p3 = {number}, p_t = {number}, "
    rf"p0 = {number}, p3 \+ p_t - 2 p0 = {number}",
    details["ks_profiles_violate_algebraic_condition"])
```

`number` is a pattern for a number as the record prints it: an optional minus sign (`-?`), digits and points (`[0-9.]+`), and an optional exponent such as `e-05`; the parentheses mark the part to be returned. A string with the prefix `rf` is both raw (backslashes are kept) and an f-string, so `{number}` is replaced by the pattern; `\+` stands for a literal plus sign. `re.findall` returns, for every place in the detail text that matches, the six numbers: y, ρ, p_3, p_t, p_0 and $p_3 + p_t - 2p_0$. The record prints three such examples, at $y = -3$, -1.5 and 0.

```
say("    y         rho          p3      p_t          p0  p3 + p_t - 2 p0")
worst_example = 0.0
for example in examples:
    y0, values = float(example[0]), [float(v) for v in example[1:]]
    i = int(np.argmin(np.abs(y - y0)))  # the profile point at this y
    mine = [float(prof["rho"][i]), float(prof["p3"][i]), float(prof["p_t"][i]),
            float(prof["p0"][i]), float(c2[i])]
    say(f"{y[i] + 0.0:5.1f}  {mine[0]:10.6g}  {mine[1]:10.6g}"  # + 0.0 turns -0 into 0
        f"  {mine[2] + 0.0:7.3g}  {mine[3]:10.6g}  {mine[4]:15.6g}")
    for a, b in zip(mine, values):  # 6 significant digits: relative 5e-6
        worst_example = max(worst_example, abs(a - b) / max(abs(b), 1e-300)
                            if b != 0.0 else abs(a))
```

For each example the profile point nearest to its y is found (`np.argmin` gives the position of the smallest distance), this notebook's five values there are printed (the format `g` chooses the shorter of the fixed and the exponent form; adding 0.0 turns a negative zero into 0), and the largest relative difference to the record's six-digit numbers is kept (an absolute difference where the record's value is 0).

```
ratios_c2 = {item["a4"]: (item["p3"] + item["p_t"]) / (2.0 * item["p0"])
             for item in history}
recorded_c2 = {int(key[-2:]) / 10.0: float(value) for key, value in re.findall(
    r"(N136_lam0_a[0-9][0-9]): ([0-9.]+)",
    details["ks_integrals_violate_algebraic_condition"])}
worst_c2 = max(abs(ratios_c2[a4] / value - 1.0) for a4, value in recorded_c2.items())
```

`ratios_c2` holds $(\int p_3 + \int p_t)/(2\int p_0)$ at the 17 slices; C2 integrated would make it 1. The record prints this ratio for `N136_lam0_a00`, `a10` and `a20`; the pattern finds the three names and numbers, the slice is read from the last two digits of the name (`key[-2:]`), and `worst_c2` is the largest relative difference to this notebook's values.

```
report("C1: (max rho - min rho) / max|T| for N136_lam0_a10", f"{c1:.4f}")
report("C2 integrated: (int p3 + int p_t)/(2 int p0) at a4,0 = 0, 1, 2",
       ", ".join(f"{ratios_c2[a]:.6f}" for a in (0.0, 1.0, 2.0)))
report("C3: int rho at a4,0 = 0 and 2; int p3 and int p_t at a4,0 = 1",
       f"{history[0]['rho']:.4f}, {history[-1]['rho']:.4f}; "
       f"{history[8]['p3']:.4f}, {history[8]['p_t']:.1f}")
```

Three RESULT lines: the C1 measure, the C2 ratios at three slices, and the numbers that violate C3 ($\int \rho$ at the first and last slice; $\int p_3$ and $\int p_t$ at $a_{4,0} = 1$).

```
check(c1 > 0.01 and len(examples) == 3 and worst_example < 5e-6
      and float(np.min(np.abs(c2[[0, len(c2) // 2, -1]]))) > 0.0,
      "C1 and C2 fail for N136_lam0_a10; the record's example values reproduced",
      record=f"{SOURCE}, checks ks_profiles_depend_on_x0 and "
             "ks_profiles_violate_algebraic_condition")
check(len(recorded_c2) == 3 and worst_c2 < 5e-6
      and all(abs(r - 1.0) > 0.5 for r in ratios_c2.values()),
      "C2 fails after integration at all 17 slices; the record's ratios reproduced",
      record=f"{SOURCE}, check ks_integrals_violate_algebraic_condition")
check(history[-1]["rho"] < 0.2 * history[0]["rho"]
      and all(item["p3"] > 0.25 * item["rho"] and item["p_t"] == 0.0
              for item in history),
      "C3 fails: rho changes along the history and p3 differs from p_t",
      record=f"{SOURCE}, check ks_history_is_a_prescribed_background")
```

The three checks: C1 and C2 fail for this profile (the variation is more than 1 per cent, and $p_3 + p_t - 2p_0$ is nonzero at the tip, the middle point and the brane; `c2[[0, len(c2) // 2, -1]]` picks these three entries), and the record's three examples are reproduced to six digits; C2 fails after integration at all 17 slices (the ratio differs from 1 by more than 0.5) and the record's three ratios are reproduced; C3 fails ($\int \rho$ falls to less than a fifth, and $\int p_3$ is more than a quarter of $\int \rho$ while $\int p_t = 0$). Out [12] prints the table: at the tip $\rho = 43.07$, $p_3 = 158.3$, $p_0 = -431.7$ and

$p_3 + p_t - 2p_0 = 1021.7$; the C1 measure 0.0998; the C2 ratios 0.339767, 0.259690, 0.239714; and $\int \rho = 80.2822$ and 12.4451.

In [13], the last check.

```
NAMES = ["proper_emt", "coordinate_emt", "y_conservation", "energy_slopes",
         "integrated_ratios", "energy_spreading", "interaction_terms"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15b_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

As in Notebook 15e (Section 15.9, In [15]), with the seven figure names of this notebook in order and the file names `15b_<number>_<name>.png`: the names whose file does not exist are collected, the check requires that list to be empty, and `all_checks_passed()` prints the last line, ALL 18 CHECKS PASSED (notebook 15b).

15.21 Adiabaticity: does the gas follow the moving background?

The question. Every state computed so far is an **instantaneous** state: at the slice $a_{4,0}$ the solver finds the ground state of the Hamiltonian $h(a_{4,0})$ of that instant, as if the background stood still. But it does not stand still: along the history $a_4 = AHx_4$ grows with the time x_4, 3-space inflates and the extra times deflate. A system whose Hamiltonian changes slowly enough stays in the eigenstate that continues its initial one (the **adiabatic theorem** of quantum mechanics; Chapter 14 introduced this adiabatic picture); a system whose Hamiltonian changes too fast is kicked into higher levels. Which case applies here must be measured.

The exact evolution keeps the sector. In the mean field the orbitals obey exactly (ks-theory.json, adiabaticity.exactEvolution; check ansatz_removes_spin_connection, which holds for a time-dependent a_4)

$$i\,\frac{\partial \chi}{\partial x_4} = h(a_4(x_4))\,\chi, \qquad h_j = j\Big[-i\sigma_1\frac{d}{dy} + M\sigma_2 + \kappa k\sigma_3\Big] + v, \qquad \kappa = e^{-Hy-a_4(x_4)}.$$

At every instant h acts within one block, at one 3-momentum, and keeps the brane condition; so the evolution never moves a particle to another momentum, block type or parity. **Transitions are possible only inside a sector**, between the levels of one (n_2, j, parity). Notebook 15a showed that each of the lowest particle-hole excitations of the gas (up to 24 per state, 1610 in all) changes the sector (Section 15.15); the in-sector jumps are much more expensive: the one with the largest Q costs between $1.488\,m$ and $2.253\,m$ in the 50 states that have one (all but the 25 states with $N = 8$).

Where the measure Q comes from, line by line. Expand the evolving orbital in the instantaneous orbitals ϕ_m, with $h\,\phi_m = \varepsilon_m\phi_m$ at every instant, as $\chi = \sum_m c_m(x_4)\,e^{-i\theta_m}\phi_m$ with the phases $\theta_m = \int^{x_4} \varepsilon_m\,dx_4$; a dot means d/dx_4.

$$i\dot\chi = \sum_m \left(i\dot c_m + \varepsilon_m c_m\right)e^{-i\theta_m}\phi_m + i\sum_m c_m e^{-i\theta_m}\dot\phi_m.$$

Rule: the product rule for the three factors; $\dot\theta_m = \varepsilon_m$.

$$h\chi = \sum_m \varepsilon_m c_m e^{-i\theta_m}\phi_m.$$

Rule: $h\phi_m = \varepsilon_m\phi_m$.

$$\dot c_m = -\sum_n c_n\, e^{i(\theta_m-\theta_n)}\,\langle m|\dot\phi_n\rangle.$$

Rule: set the two lines equal (the evolution equation), the terms $\varepsilon_m c_m$ cancel, take the inner product with ϕ_m (the orbitals are orthonormal, so $\langle m|\phi_n\rangle$ is 1 for $n=m$ and 0 otherwise; $\langle m|X\rangle$ means $\int \phi_m^\dagger X\,dy$), multiply by $-i\,e^{i\theta_m}$, and use $-i\cdot i = 1$.

$$\langle m|\dot\phi_n\rangle = \frac{\langle m|\dot h|n\rangle}{\varepsilon_n-\varepsilon_m} = \dot a_4\,\frac{\langle m|\partial_a h|n\rangle}{\varepsilon_n-\varepsilon_m} \qquad (m\neq n).$$

Rule: differentiate $h\phi_n = \varepsilon_n\phi_n$: $\dot h\phi_n + h\dot\phi_n = \dot\varepsilon_n\phi_n + \varepsilon_n\dot\phi_n$; take the inner product with ϕ_m, where $\langle m|h\dot\phi_n\rangle = \varepsilon_m\langle m|\dot\phi_n\rangle$ (h is self-adjoint, Section 15.2) and $\langle m|\phi_n\rangle = 0$; solve for $\langle m|\dot\phi_n\rangle$; finally $\dot h = \dot a_4\,\partial h/\partial a_4$ (chain rule), written $\partial_a h$ (check adiabatic_offdiagonal_identity).

$$|c_m| \approx \Big|\int \dot a_4\,\frac{\langle m|\partial_a h|n\rangle}{\varepsilon_n-\varepsilon_m}\,e^{i(\varepsilon_m-\varepsilon_n)x_4}\,dx_4\Big| \approx \dot a_4\,\frac{|\langle m|\partial_a h|n\rangle|}{(\varepsilon_n-\varepsilon_m)^2}.$$

Rule: start with $c_n = 1$ and all other c zero, and keep on the right of the third line only the term of the occupied orbital n (first order); over a short stretch of time the factor in front of the exponential hardly changes, and $\int e^{i\omega x_4}dx_4 = e^{i\omega x_4}/(i\omega)$, whose size is $1/|\omega|$ with $\omega = \varepsilon_m-\varepsilon_n$.

With $\dot a_4 = AH$ this is the **adiabaticity measure** of the record (ks-theory.json, adiabaticity.measure):

$$Q_{nm} = AH\,\frac{|\langle n|\partial_a h|m\rangle|}{(\varepsilon_n-\varepsilon_m)^2}, \qquad \text{transition probability} \approx Q_{nm}^2,$$

for n occupied and m empty in the same sector. The evolution is adiabatic when every $Q_{nm} \ll 1$. This is an estimate of first order, not a solution of the time-dependent problem, which stays OPEN.

The derivative of the Hamiltonian. Without interaction only κ depends on a_4, and $\partial\kappa/\partial a_4 = -\kappa$ (the derivative of e^{-Hy-a_4}). So

$$\partial_a h_j = -j\,\kappa k\,\sigma_3, \qquad \langle n|\partial_a h|m\rangle = -j\,k\,e^{-a_{4,0}}\int_{-L}^{0} e^{-Hy}\,(a_n a_m - b_n b_m)\,dy,$$

because for $\chi = (a, ib)$ the product $\chi_n^\dagger\sigma_3\chi_m$ is $a_n a_m + (-ib_n)(-ib_m) = a_n a_m - b_n b_m$. With interaction the self-consistent potentials change with a_4 too, and $\partial_a h$ gains $j(\partial_a M)\sigma_2 + \partial_a v$, which the solver obtains from the states at $a_{4,0}\pm\delta$ and $\pm 2\delta$ (ks-theory.json, adiabaticity.derivative). At $k=0$, $\partial_a h = 0$: the zero modes do not feel the history at all, and for $N=8$ every Q is exactly zero.

Hellmann-Feynman. For $m=n$ the same differentiation gives the **Hellmann-Feynman theorem** of Chapter 13, $d\varepsilon_n/da_4 = \langle n|\partial_a h|n\rangle$ (take the inner product of the differentiated eigenequation with ϕ_n itself; the terms with $\dot\phi_n$ cancel because h is self-adjoint and $\langle n|n\rangle = 1$). Summed

over the occupied levels it gives $dE/da_4 = \sum g f \langle n|\partial_a h|n\rangle$ for the free gas, the same as the energy-change law of Section 15.16 (check adiabatic_hellmann_feynman of `Revision/kohn_sham/reports/ks-theory-python.json`; the solver checks it for all 75 states, worst deviation $2.6 \times 10^{-9}\,m$, check adiabatic_hellmann_feynman of `Revision/kohn_sham/reports/ks-rust-solver.json`).

What the record finds (COMPUTED; `Revision/kohn_sham/results/adiabatic/adiabaticity.csv`, reproduced by Notebook 15c). Over the whole canonical matrix $Q_{max} \leq 0.0935$, so every transition probability is below 0.009; the largest Q belongs to the jump from the brane-band level of the highest occupied shell to the first bulk level of the same sector, at the start of the history ($N = 688$, $a_{4,0} = 0$), and Q_{max} falls along the history because the matrix element carries the factor $k\,e^{-a_{4,0}}$. Within the sectors, the instantaneous states are followed adiabatically to a good approximation.

Fermi-level crossings. Adiabatic evolution keeps the occupation of every level. If along the history an empty level of one sector comes down below an occupied level of another sector, the instantaneous ground state (aufbau) would move particles from one to the other, but the evolution cannot: the two levels lie in different sectors. Such a **Fermi-level crossing** makes the instantaneous ground state the wrong state. The canonical matrix has none (the record file `adiabatic/fermi-level-crossings.csv`: no change of the occupied set in any of its 60 rows). The record demonstrates the effect with $N = 696 = 688 + 8$ (check adiabatic_crossing_flag_demonstration): at $a_{4,0} = 0$ the eight extra particles sit in the odd bulk level at $k = 0$ ($\varepsilon = 1.2923$, which does not move), while the band level of the shell $n_2 = 12$ lies just above it and comes down; after the crossing the adiabatically continued state lies above the instantaneous aufbau state by $3.66\,m$ at $a_{4,0} = 0.5$, growing to $8.62\,m$ at $a_{4,0} = 2$ (`adiabatic/crossing-demo.csv`). There the instantaneous picture fails.

15.22 Example: the adiabaticity measure (Notebook 15c)

Notebook 15c needs no Rust. It defines the solver's shooting method for the free problem in Python, solves again all 270 levels of the occupied sectors of the free states $N = 136$ and $N = 688$ at the five slices, and computes Q_{nm} for every allowed pair. It draws the two orbitals of the pair with the largest Q and the integrand of their matrix element (Figure 15c.1), every Q against the shell (Figure 15c.2), Q_{max} of the whole matrix along the history (Figure 15c.3), the Hellmann-Feynman theorem (Figure 15c.4), the Fermi-level crossing of $N = 696$ (Figure 15c.5) and a heat map of Q_{max} (Figure 15c.6). It takes about 15 seconds and ends with ALL 12 CHECKS PASSED (notebook 15c).

What to look for: in Figure 15c.1 that the band orbital sits at the brane while the bulk orbital fills the interval, so that their overlap is small; in Figure 15c.2 that all points lie below 0.1; in Figure 15c.5 how quickly the band level crosses the fixed bulk level.

15.23 How to run Notebook 15c

Step 1. What this notebook does and what it needs.

Notebook 15c (Adiabaticity along the deflating history) is the file `Revision/textbook/notebooks/15c_adiabaticity.ipynb` of the repository Dirac_claude. It explains when the instantaneous Kohn-Sham states of dirac16complex can be followed along the deflating history (the adiabatic approximation), recomputes in plain Python the adiabaticity measure Q of every allowed transition of the free states N = 136 and N = 688 at the five slices, reproduces the largest Q, its pair of

levels, its energy difference and its matrix element from the committed Revision record, checks the Hellmann-Feynman theorem and the energy derivative along the history, and reproduces the Fermi-level crossing of the demonstration state N = 696. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15c_adiabaticity.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15c_adiabaticity.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 15c_adiabaticity.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/15c.captions.json`
- `Revision/textbook/figures/15c_1_transition_orbitals.png`
- `Revision/textbook/figures/15c_2_q_by_pair.png`
- `Revision/textbook/figures/15c_3_q_history.png`
- `Revision/textbook/figures/15c_4_hellmann_feynman.png`
- `Revision/textbook/figures/15c_5_fermi_crossing.png`
- `Revision/textbook/figures/15c_6_q_map.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 12 CHECKS PASSED (notebook 15c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/15c_adiabaticity.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 15c_adiabaticity.ipynb
python -m nbconvert --execute --inplace 15c_adiabaticity.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

15.24 Notebook 15c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/15c_adiabaticity.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Kohn-Sham states of the Revision record are **instantaneous** states: at each
```

```
slice $a_{4,0}$ of the history the solver finds the ground state of the Hamiltonian
of that instant. But the history moves on: $a_4 = A H x_4$ grows with the time
$x_4$. Does the gas follow the instantaneous ground state, or does the motion of
the background kick particles into higher levels? This is the question of
**adiabaticity**. This notebook

- explains the measure $Q_{nm}$ of the Revision record and where it comes from;
- recomputes in plain Python, with the solver's own shooting method, every
  allowed transition of the free states $N = 136$ and $N = 688$ at the five slices,
  and reproduces the largest value $Q_{max}$, the pair of levels, their energy
  difference and the matrix element recorded by the Rust solver;
- checks the Hellmann-Feynman theorem $d\varepsilon_n/da_4 = \langle n|\partial_a
  h|n\rangle$ and the energy derivative $dE/da_4$;
- reproduces the **Fermi-level crossing** of the demonstration state $N = 696$, where
  the instantaneous ground state and the adiabatically followed state differ.

It draws six figures and needs no Rust.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 15c (Adiabaticity along the deflating history) is the file
‘Revision/textbook/notebooks/15c_adiabaticity.ipynb‘ of the repository Dirac_claude. It
explains when the instantaneous Kohn-Sham states of dirac16complex can be followed along
the deflating history (the adiabatic approximation), recomputes in plain Python the
adiabaticity measure Q of every allowed transition of the free states N = 136 and N = 688
at the five slices, reproduces the largest Q, its pair of levels, its energy difference
and its matrix element from the committed Revision record, checks the Hellmann-Feynman
theorem and the energy derivative along the history, and reproduces the Fermi-level
crossing of the demonstration state N = 696. It needs a computer with Windows 11, macOS
or Linux, an internet connection for the installation, the program Git, and Python 3.12
or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program
that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
```

open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
```

```
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one

goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15c_adiabaticity.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15c_adiabaticity.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 15c_adiabaticity.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/15c.captions.json‘
- ‘Revision/textbook/figures/15c_1_transition_orbitals.png‘
- ‘Revision/textbook/figures/15c_2_q_by_pair.png‘
- ‘Revision/textbook/figures/15c_3_q_history.png‘
- ‘Revision/textbook/figures/15c_4_hellmann_feynman.png‘
- ‘Revision/textbook/figures/15c_5_fermi_crossing.png‘
- ‘Revision/textbook/figures/15c_6_q_map.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS every figure file of this notebook exists
    ALL 12 CHECKS PASSED (notebook 15c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/15c_adiabaticity.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 15c_adiabaticity.ipynb
      python -m nbconvert --execute --inplace 15c_adiabaticity.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers

‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 15c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 15c (Adiabaticity along the deflating history) is the file
# Revision/textbook/notebooks/15c_adiabaticity.ipynb of the repository Dirac_claude. It
# explains when the instantaneous Kohn-Sham states of dirac16complex can be followed
# along the deflating history (the adiabatic approximation), recomputes in plain Python
# the adiabaticity measure Q of every allowed transition of the free states N = 136 and N
# = 688 at the five slices, reproduces the largest Q, its pair of levels, its energy
# difference and its matrix element from the committed Revision record, checks the
# Hellmann-Feynman theorem and the energy derivative along the history, and reproduces
# the Fermi-level crossing of the demonstration state N = 696. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages
# run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
```

```
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 15c_adiabaticity.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
```

```
#     jupyter nbconvert --to notebook --execute --inplace 15c_adiabaticity.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 15c_adiabaticity.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/15c.captions.json
# - Revision/textbook/figures/15c_1_transition_orbitals.png
# - Revision/textbook/figures/15c_2_q_by_pair.png
# - Revision/textbook/figures/15c_3_q_history.png
# - Revision/textbook/figures/15c_4_hellmann_feynman.png
# - Revision/textbook/figures/15c_5_fermi_crossing.png
# - Revision/textbook/figures/15c_6_q_map.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 12 CHECKS PASSED (notebook 15c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/15c_adiabaticity.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
```

```
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 15c_adiabaticity.ipynb
#     python -m nbconvert --execute --inplace 15c_adiabaticity.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "15c"  # this notebook: chapter 15, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
```

```
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 15c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Instantaneous (adiabatic) state**: the ground state of the Hamiltonian
  $h(a_{4,0})$ of one instant of the history (one slice $a_{4,0}$); the history
  itself keeps moving, with 3-space inflating and the extra times deflating.
- **Adiabatic**: a change of the Hamiltonian slow enough that a system in an
  eigenstate stays in the eigenstate that continues it (the adiabatic theorem).
- **Transition**: a jump of a particle from an occupied level $n$ to an empty level
  $m$; its **amplitude** is a number whose square is the probability of the jump.
- **Matrix element** $\langle n|X|m\rangle$: for orbitals $\chi_n = (a_n, i b_n)$
```

```
  the integral $\int \chi_n^\dagger X \chi_m\,dy$; for $X = \sigma_3$ it is
  $\int (a_n a_m - b_n b_m)\,dy$.
- **Sector**: the shell $n_2$, the block type $j$ and the brane parity. The exact
  evolution conserves all three, so transitions happen only inside a sector.
- **Adiabaticity measure** $Q_{nm} = A H |\langle n|\partial_a h|m\rangle| /
  (\varepsilon_n - \varepsilon_m)^2$: the first-order amplitude of the jump
  $n \to m$; $Q_{max}$ is the largest over all allowed pairs.
- **Hellmann-Feynman theorem**: the derivative of a level with respect to a
  parameter equals the matrix element of the derivative of the Hamiltonian.
- **Aufbau**: filling the lowest levels with the $N$ particles. **Fermi-level
  crossing**: an empty level comes down below an occupied one along the history,
  so the aufbau filling changes. **Adiabatically continued state**: the state
  that keeps the occupations of the first slice.
- **Open shell**: a degenerate group of levels only partly filled.
- **Heat map**: a table of numbers drawn as coloured squares.
```

Text cell:

```
## 4. The physical and mathematical situation

In the author's metric 3-space $x_1, x_2, x_3$ inflates with $e^{a_4}$ and the three
extra times $x_5, x_6, x_7$ deflate with $e^{-a_4}$; the history is $a_4 = A H x_4$
with $A = 1$, a PRESCRIBED BACKGROUND (Revision record ks-theory.json,
adiabaticity.historyStatus). In the hidden coordinate $y$ the Kohn-Sham orbitals of a
sector obey exactly (ks-theory.json, adiabaticity.exactEvolution)

$$i\,\frac{\partial \chi}{\partial x_4} = h(a_4(x_4))\,\chi, \qquad
h_j = j\left[-i\sigma_1 \frac{d}{dy} + M\sigma_2 + \kappa k \sigma_3\right] + v,
\qquad \kappa = e^{-Hy - a_4}.$$

**Where $Q$ comes from.** Write $\chi = \sum_m c_m(x_4)\,e^{-i\theta_m}\,\phi_m$ with
the instantaneous orbitals $h\phi_m = \varepsilon_m\phi_m$ and the phases
$\theta_m = \int \varepsilon_m\,dx_4$. Inserting into the equation and projecting
on $\phi_m$ gives, line by line: (1) the left side gives
$i\dot c_m + \varepsilon_m c_m + i\sum_n c_n e^{i(\theta_m - \theta_n)}\langle
m|\dot\phi_n\rangle$ (product rule; the dot is $d/dx_4$); (2) the right side gives
$\varepsilon_m c_m$; (3) so $\dot c_m = -\sum_n c_n e^{i(\theta_m-\theta_n)}\langle
m|\dot\phi_n\rangle$. (4) Differentiating $h\phi_n = \varepsilon_n\phi_n$ and
projecting on $\phi_m$ ($m \ne n$) gives $\langle m|\dot\phi_n\rangle = \dot a_4
\langle m|\partial_a h|n\rangle/(\varepsilon_n - \varepsilon_m)$. (5) Starting from
$c_n = 1$, integrating (3) over a time in which the phase $e^{i(\varepsilon_m -
\varepsilon_n)x_4}$ turns many times gives an amplitude of size
$|\langle m|\dot\phi_n\rangle|/|\varepsilon_m - \varepsilon_n|$ (the integral of
$e^{i\omega t}$ is $e^{i\omega t}/(i\omega)$), that is, with $\dot a_4 = AH$,

$$Q_{nm} = A H\,\frac{|\langle n|\partial_a h|m\rangle|}{(\varepsilon_n -
\varepsilon_m)^2}, \qquad \text{transition probability} \approx Q_{nm}^2 .$$

The evolution is adiabatic when every $Q_{nm} \ll 1$.

**The derivative of $h$.** Without interaction ($\lambda = 0$) only $\kappa$ depends
on $a_4$, and $\partial\kappa/\partial a_4 = -\kappa$, so
$\partial_a h_j = -j\kappa k\sigma_3$ and
```

```
$\langle n|\partial_a h|m\rangle = -j k e^{-a_{4,0}}\int e^{-Hy}(a_n a_m - b_n b_m)\,dy$.
(With interaction the self-consistent potentials change too; the solver includes
their derivatives. This notebook recomputes the free states $\lambda = 0$ and shows
the recorded values of all states.)

**Hellmann-Feynman.** For $m = n$ the same steps give
$d\varepsilon_n/da_4 = \langle n|\partial_a h|n\rangle$, and the energy of the free
gas, $E = \sum g f\varepsilon$, changes by $dE/da_4 = \sum g f\langle n|\partial_a
h|n\rangle$.

The non-adiabatic (time-dependent) problem itself is OPEN; $Q$ is a first-order
estimate. Units $H = m = 1$, $L = 3$, $\Delta k = 0.25$.
```

Text cell:

```
## 5. The solver's shooting method in Python

The next cell defines the solver's method for the free problem ($M = m$, $v = 0$):
the grid of $G = 900$ RK4 steps on $[-3, 0]$ with its fine grid of step ends and
midpoints, `shoot` (integrate the block equation from the tip, where $b = 0$, to the
brane, and follow the Pruefer angle $\theta = \mathrm{atan2}(b, a)$; the phase
$\Phi = j\theta(0)$ increases with $\varepsilon$ and equals $l\pi$ (even parity) or
$\pi/2 + l\pi$ (odd parity) at the level with label $l$; `shoot` also returns the
derivative $d\Phi/d\varepsilon = \int r^2 dy/r(0)^2$, $r^2 = a^2 + b^2$, with the
integral by the trapezoid rule on the step ends, as in the solver), `find_level`
(the solver's Newton method inside a bracket for $\Phi(\varepsilon) = $ target: walk
with doubling steps until the target is bracketed, then Newton steps, with bisection
whenever a Newton step leaves the bracket or the previous step did not halve the
error; stop when the bracket or a Newton step is below $10^{-13}$), and `orbital`
(the normalised orbital on the fine grid, with cubic Hermite midpoints). The
comments say what each line does.
```

In [2]:

```
import csv  # reads the tables (CSV files) of the Revision record
import math  # exp, sqrt, atan2, pi for single numbers

import numpy as np  # arrays of numbers

L, STEPS = 3.0, 900  # tip cutoff and number of RK4 steps (the solver's values)
POINTS = 2 * STEPS + 1  # step ends and midpoints
STEP = L / STEPS
Y = np.array([-L * ((POINTS - 1 - f) / (POINTS - 1)) for f in range(POINTS)])
EW_LIST = [math.exp(-y) for y in Y.tolist()]  # e^{-Hy} with H = 1, as the solver
EW = np.array(EW_LIST)
SIMPSON = np.full(POINTS, 2.0 * STEP / 6.0)  # Simpson weights on the fine grid
SIMPSON[1::2] = 4.0 * STEP / 6.0
SIMPSON[0] = SIMPSON[-1] = STEP / 6.0
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

```
def shoot(eps, k, j, a4, store=False):
    """Free block (M = 1, v = 0): integrate tip -> brane; (Phi, dPhi/deps, nodes)."""
    kk = k * math.exp(-a4)
    a, b, theta, raw, area, r2_prev = 1.0, 0.0, 0.0, 0.0, 0.0, 1.0
    je = j * eps  # j (eps - v) with v = 0
    nodes = [(a, b)] if store else None
    for i in range(STEPS):
        k0 = kk * EW_LIST[2 * i]  # kappa k at the start of the step
        k1 = kk * EW_LIST[2 * i + 1]  # at its midpoint
        k2 = kk * EW_LIST[2 * i + 2]  # at its end
        p1a, p1b = a - (k0 + je) * b, (je - k0) * a - b  # (a', b') with M = 1
        aa, bb = a + 0.5 * STEP * p1a, b + 0.5 * STEP * p1b
        p2a, p2b = aa - (k1 + je) * bb, (je - k1) * aa - bb
        aa, bb = a + 0.5 * STEP * p2a, b + 0.5 * STEP * p2b
        p3a, p3b = aa - (k1 + je) * bb, (je - k1) * aa - bb
        aa, bb = a + STEP * p3a, b + STEP * p3b
        p4a, p4b = aa - (k2 + je) * bb, (je - k2) * aa - bb
        a += STEP / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)
        b += STEP / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
        new = math.atan2(b, a)  # the Pruefer angle, followed continuously:
        change = new - raw  # bring the change of angle into (-pi, pi]
        if change > math.pi:
            change -= 2.0 * math.pi
        elif change <= -math.pi:
            change += 2.0 * math.pi
        theta, raw = theta + change, new
        r2 = a * a + b * b
        area, r2_prev = area + 0.5 * STEP * (r2_prev + r2), r2  # trapezoid rule
        if store:
            nodes.append((a, b))
    return j * theta, area / r2_prev, nodes  # Phi and dPhi/deps = int r^2 / r(0)^2


def find_level(k, j, parity, label, a4, guess, tol=1e-13):
    """The level with this label: the solver's Newton method inside a bracket."""
    target = label * math.pi + (0.0 if parity == "even" else 0.5 * math.pi)
    e = guess
    g, d = shoot(e, k, j, a4)[:2]
    g -= target
    if g == 0.0:
        return e
    step = min(max(abs(g / d) * 1.2, 1e-4), 2.0)  # the first trial step
    if g < 0:  # Phi too small: walk up with doubling steps until Phi >= target
        lo = e
        while True:
            trial = lo + step
            gt, dt = shoot(trial, k, j, a4)[:2]
            gt -= target
            if gt >= 0:
                hi = trial  # [lo, hi] brackets the level
                if abs(gt) < abs(g):
                    e, g, d = trial, gt, dt  # keep the better end for Newton
                break
            lo, e, g, d = trial, trial, gt, dt
```

```
            step *= 2.0
    else:  # Phi too large: walk down until Phi <= target
        hi = e
        while True:
            trial = hi - step
            gt, dt = shoot(trial, k, j, a4)[:2]
            gt -= target
            if gt <= 0:
                lo = trial
                if abs(gt) < abs(g):
                    e, g, d = trial, gt, dt
                break
            hi, e, g, d = trial, trial, gt, dt
            step *= 2.0
    previous = math.inf
    while hi - lo > tol and g != 0.0:
        new = e - g / d  # Newton step, or bisection if it leaves the bracket
        bisect = not lo < new < hi or abs(g) > 0.5 * previous
        if bisect:
            new = 0.5 * (lo + hi)
        previous = abs(g)
        moved = abs(new - e)
        e = new
        g, d = shoot(e, k, j, a4)[:2]
        g -= target
        if g < 0:
            lo = e
        elif g > 0:
            hi = e
        if moved <= tol and not bisect:
            break  # a Newton step that moves less than tol: converged
    return e


def orbital(eps, k, j, a4):
    """Normalised orbital (a, b) on the fine grid; Hermite midpoints, Simpson norm."""
    nodes = np.array(shoot(eps, k, j, a4, store=True)[2])
    kap = k * math.exp(-a4) * EW[0::2]
    a_n, b_n = nodes[:, 0], nodes[:, 1]
    da = a_n - (kap + j * eps) * b_n  # derivatives at the step ends
    db = (j * eps - kap) * a_n - b_n
    a = np.zeros(POINTS)
    b = np.zeros(POINTS)
    a[0::2], b[0::2] = a_n, b_n
    a[1::2] = 0.5 * (a_n[:-1] + a_n[1:]) + STEP / 8.0 * (da[:-1] - da[1:])
    b[1::2] = 0.5 * (b_n[:-1] + b_n[1:]) + STEP / 8.0 * (db[:-1] - db[1:])
    norm = math.sqrt(float(np.sum(SIMPSON * (a * a + b * b))))
    return a / norm, b / norm


def da_h(k, j, a4, first, second):
    """<first| d_a h |second> = -j k e^{-a4} int e^{-y} (a1 a2 - b1 b2) dy."""
    s3 = first[0] * second[0] - first[1] * second[1]
    return -j * k * math.exp(-a4) * float(np.sum(SIMPSON * EW * s3))
```

Text cell:

```
## 6. The levels of the record, solved again

The next cell reads the level table of a free ground state from the record (for
example ‘ground/levels/N136_lam0_a00.csv‘: for each level its shell $n_2$, the
number $r_3$ of momentum vectors in the shell, $k$, $j$, the parity, the label $l$,
the energy and the occupation $f$), keeps the sectors that contain an occupied
level (only there can a particle jump), and solves every level of these sectors
again in Python. Each energy must agree with the solver’s to $10^{-12}$.
```

In [3]:

```
LEVELS = "Revision/kohn_sham/results/ground/levels"
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]


def state_id(n, a4, tag="lam0"):
    return f"N{n}_{tag}_a{round(10 * a4):02d}"


def solve_state(n, a4):
    """{sector: {label: (eps, f, k, r3, orbital)}} for the sectors with particles."""
    rows = list(csv.DictReader(open(repository_file(f"{LEVELS}/{state_id(n, a4)}.csv"),
                                    newline="", encoding="utf-8")))
    sectors = {}
    for row in rows:
        sector = (int(row["n2"]), int(row["j"]), row["parity"])
        sectors.setdefault(sector, []).append(row)
    solved, worst = {}, 0.0
    for sector, members in sectors.items():
        if all(float(row["f"]) < 0.5 for row in members):
            continue  # no particle in this sector: no transition starts here
        solved[sector] = {}
        for row in members:
            k, eps_rec = float(row["k"]), float(row["eps"])
            eps = find_level(k, sector[1], sector[2], int(row["label"]), a4, eps_rec)
            worst = max(worst, abs(eps - eps_rec))
            solved[sector][int(row["label"])] = (
                eps, float(row["f"]), k, int(row["r3"]), orbital(eps, k, sector[1], a4))
    return solved, worst


states = {}
worst_level = 0.0
for n in (136, 688):
    for a4 in SLICES:
        states[(n, a4)], worst = solve_state(n, a4)
        worst_level = max(worst_level, worst)
count = sum(len(levels) for state in states.values() for levels in state.values())
report("levels solved again (10 states)", count)
report("largest |eps(Python) - eps(solver)|", f"{worst_level:.1e}")
```

```
check(worst_level < 1e-12, "every level equals the solver's level within 1e-12",
      record=f"{LEVELS}/N136_lam0_a*.csv and N688_lam0_a*.csv, column eps")
```

Out [3]:

```
RESULT levels solved again (10 states) = 270
RESULT largest |eps(Python) - eps(solver)| = 9.3e-14
PASS every level equals the solver's level within 1e-12
     reproduces Revision/kohn_sham/results/ground/levels/N136_lam0_a*.csv and
    N688_lam0_a*.csv, column eps
```

Text cell:

```
## 7. The pair of levels with the largest Q

For $N = 136$ at $a_{4,0} = 0$ the record names the pair with the largest $Q$: from
the brane-band level of the shell $n_2 = 4$ ($k = 0.5$, the highest occupied shell,
label 0) to the first bulk level of the same sector (label 1). The next cell draws
the two orbitals and the integrand $\kappa k(a_n a_m - b_n b_m)$ of their matrix
element: the band orbital is bound to the brane, the bulk orbital oscillates
across the interval, so their overlap partly cancels, and the energy difference
($2.05\,m$) is large.
```

In [4]:

```
sector = (4, 1, "even")
band = states[(136, 0.0)][sector][0]  # (eps, f, k, r3, (a, b)) of label 0
bulk = states[(136, 0.0)][sector][1]  # label 1
k = band[2]
integrand = k * EW * (band[4][0] * bulk[4][0] - band[4][1] * bulk[4][1])
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
left.plot(Y, band[4][0], color=PALETTE[0], lw=2.0, label="band: $a$")
left.plot(Y, band[4][1], color=PALETTE[0], lw=1.5, ls="--", label="band: $b$")
left.plot(Y, bulk[4][0], color=PALETTE[1], lw=2.0, label="bulk: $a$")
left.plot(Y, bulk[4][1], color=PALETTE[1], lw=1.5, ls="--", label="bulk: $b$")
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("orbital (normalised)")
left.set_title(f"$n_2 = 4$: band $\\varepsilon = {band[0]:.4f}$, "
               f"bulk $\\varepsilon = {bulk[0]:.4f}$", fontsize=10)
left.legend(fontsize=8)
right.plot(Y, integrand, color=PALETTE[2], lw=2.0)
right.fill_between(Y, integrand, color=PALETTE[2], alpha=0.25)
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("$\\kappa k\\,(a_n a_m - b_n b_m)$ (units of $m^2$)")
right.set_title("Integrand of the matrix element")
save_figure(fig, "transition_orbitals",
            "Left: the orbitals (components $a$ solid, $b$ dashed; vertical axis, "
            "normalised) of the occupied brane-band level (blue) and the empty bulk "
            "level (orange) of the sector $n_2 = 4$, $j = +1$, even parity, of the "
            "free state $N = 136$ at $a_{4,0} = 0$, against $y$ (horizontal axis). "
            "Right: the integrand of their matrix element $\\langle n|\\partial_a h|"
```

```
            "m\\rangle$ (units of $m^2$); the size of its area is $0.3589\\,m$, and "
            "with the energy difference $2.053\\,m$ it gives $Q = 0.0851$, the "
            "largest of this state.")
element = abs(da_h(k, 1, 0.0, band[4], bulk[4]))
report("|<band| d_a h |bulk>| and Q for N = 136, a4,0 = 0",
       f"{element:.13f}, {element / (band[0] - bulk[0]) ** 2:.13f}")
```

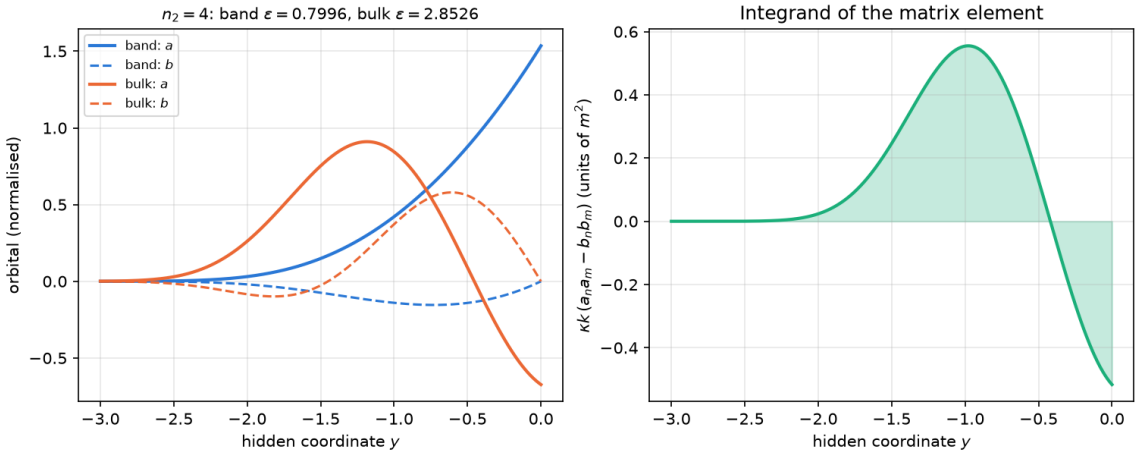

Figure 426: Left: the orbitals (components a solid, b dashed; vertical axis, normalised) of the occupied brane-band level (blue) and the empty bulk level (orange) of the sector $n_2 = 4$, $j = +1$, even parity, of the free state $N = 136$ at $a_{4,0} = 0$, against y (horizontal axis). Right: the integrand of their matrix element $\langle n|\partial_a h|m\rangle$ (units of m^2); the size of its area is $0.3589\,m$, and with the energy difference $2.053\,m$ it gives $Q = 0.0851$, the largest of this state. (Notebook 15c, figure 1.)

Out [4]:

```
Figure 15c.1 saved as Revision/textbook/figures/15c_1_transition_orbitals.png
RESULT |<band| d_a h |bulk>| and Q for N = 136, a4,0 = 0 = 0.3588612167741,
    0.0851487731074
```

Text cell:

```
## 8. Q for every allowed pair, and the comparison with the record

The next cell computes $Q_{nm}$ for every pair (occupied $n$, empty $m$, same sector)
of the ten free states $N = 136, 688$ at the five slices, finds the largest, and
compares the value, the pair (written as the record writes it,
‘n2:j:parity:label -> n2:j:parity:label‘), the energy difference and the matrix
element with the columns ‘Q_max‘, ‘Q_max_pair‘, ‘Q_max_delta_eps‘ and
‘Q_max_matrix_element‘ of ‘adiabatic/adiabaticity.csv‘.
```

In [5]:

```
ADIABATIC = "Revision/kohn_sham/results/adiabatic/adiabaticity.csv"
with open(repository_file(ADIABATIC), newline="", encoding="utf-8") as handle:
```

```
    adiabatic = {row["id"]: row for row in csv.DictReader(handle)}


def key_text(sector, label):
    return f"{sector[0]}:{sector[1]:+d}:{sector[2]}:{label}"


pairs = {}  # (n, a4) -> list of (Q, n2, label m) for the figure
worst_q, same_pairs = 0.0, True
say("   N  a4,0   Q_max    pair                              delta eps   matrix el.")
for (n, a4), state in states.items():
    best = (0.0, "", 0.0, 0.0)
    pairs[(n, a4)] = []
    for sector, levels in state.items():
        for ln, (en, fn, k, _, on) in levels.items():
            if fn < 0.5:
                continue
            for lm, (em, fm, _, _, om) in levels.items():
                if fm > 0.5:
                    continue
                element = abs(da_h(k, sector[1], a4, on, om))
                q = element / (en - em) ** 2  # A H = 1
                pairs[(n, a4)].append((q, sector[0], lm))
                if q > best[0]:
                    best = (q, f"{key_text(sector, ln)} -> {key_text(sector, lm)}",
                            abs(en - em), element)
    row = adiabatic[state_id(n, a4)]
    for value, column in ((best[0], "Q_max"), (best[2], "Q_max_delta_eps"),
                          (best[3], "Q_max_matrix_element")):
        worst_q = max(worst_q, abs(value / float(row[column]) - 1.0))
    same_pairs = same_pairs and best[1] == row["Q_max_pair"]
    say(f"{n:4d}  {a4:4.1f}  {best[0]:.5f}  {best[1]:28}  {best[2]:9.6f}"
        f"  {best[3]:9.6f}")
report("largest relative difference to the record", f"{worst_q:.1e}")
check(worst_q < 1e-9, "Q_max, its energy difference and matrix element reproduced",
      record=f"{ADIABATIC}, columns Q_max, Q_max_delta_eps, Q_max_matrix_element")
check(same_pairs, "the pair with the largest Q is the recorded pair",
      record=f"{ADIABATIC}, column Q_max_pair")
```

Out [5]:

```
   N  a4,0   Q_max    pair                           delta eps   matrix el.
 136   0.0  0.08515  4:+1:even:0 -> 4:+1:even:1     2.052930   0.358861
 136   0.5  0.07456  4:+1:even:0 -> 4:+1:even:1     1.888369   0.265876
 136   1.0  0.06362  4:+1:even:0 -> 4:+1:even:1     1.745737   0.193883
 136   1.5  0.05275  4:+1:even:0 -> 4:+1:even:1     1.616341   0.137809
 136   2.0  0.03839  4:+1:even:0 -> 4:+1:even:1     1.524416   0.089223
 688   0.0  0.09345  11:+1:even:0 -> 11:+1:even:1   2.253405   0.474527
 688   0.5  0.08526  11:+1:even:0 -> 11:+1:even:1   2.055018   0.360061
 688   1.0  0.07469  11:+1:even:0 -> 11:+1:even:1   1.890122   0.266836
 688   1.5  0.06374  11:+1:even:0 -> 11:+1:even:1   1.747327   0.194603
 688   2.0  0.05289  11:+1:even:0 -> 11:+1:even:1   1.617688   0.138417
RESULT largest relative difference to the record = 6.8e-14
```

```
PASS Q_max, its energy difference and matrix element reproduced
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, columns Q_max,
    Q_max_delta_eps, Q_max_matrix_element
PASS the pair with the largest Q is the recorded pair
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max_pair
```

Text cell:

```
The next cell draws all pairs of $N = 136$ and $N = 688$ at $a_{4,0} = 0$: $Q$
against the shell $n_2$ of the sector, coloured by the label $m$ of the empty
level. In every shell the jump to the next bulk level (label 1) dominates, and it
grows with $n_2$: the matrix element is proportional to $k$. The largest $Q$ belongs
to the highest occupied shell ($n_2 = 4$ for $N = 136$, $n_2 = 11$ for $N = 688$).
The zero modes ($n_2 = 0$) cannot jump at all: there $k = 0$ and $\partial_a h = 0$.
```

In [6]:

```
fig, ax = plt.subplots()
for marker, n in (("o", 136), ("s", 688)):
    for colour, label in zip(PALETTE, (1, 2)):
        points = [(n2, q) for q, n2, lm in pairs[(n, 0.0)] if lm == label]
        ax.plot([p[0] for p in points], [max(p[1], 1e-6) for p in points], marker,
                color=colour, ms=8 if n == 136 else 6,
                markerfacecolor=colour if n == 136 else "white",
                label=f"$N = {n}$, to label {label}")
ax.set_yscale("log")
ax.set_xlabel("shell $n_2$ of the sector ($k = 0.25\\sqrt{n_2}$)")
ax.set_ylabel("$Q_{nm}$ (pure number)")
ax.set_title("Every allowed transition at $a_{4,0} = 0$, $\\lambda = 0$")
ax.legend(fontsize=8)
save_figure(fig, "q_by_pair",
            "The adiabaticity measure $Q_{nm}$ (vertical axis, logarithmic, pure "
            "number) of every allowed transition of the free states $N = 136$ "
            "(filled) and $N = 688$ (open) at $a_{4,0} = 0$, against the shell "
            "$n_2$ of the sector (horizontal axis); colour: the label of the empty "
            "level. Transitions at $k = 0$ vanish and are drawn at the bottom "
            "($10^{-6}$). All values lie below $0.1$, so the instantaneous states "
            "are followed adiabatically to a good approximation.")
largest = max(q for values in pairs.values() for q, _, _ in values)
report("largest Q of all free pairs (10 states)", f"{largest:.5f}")
check(largest < 0.1, "every Q of the free states is below 0.1")
```

Out [6]:

```
Figure 15c.2 saved as Revision/textbook/figures/15c_2_q_by_pair.png
RESULT largest Q of all free pairs (10 states) = 0.09345
PASS every Q of the free states is below 0.1
```

Text cell:

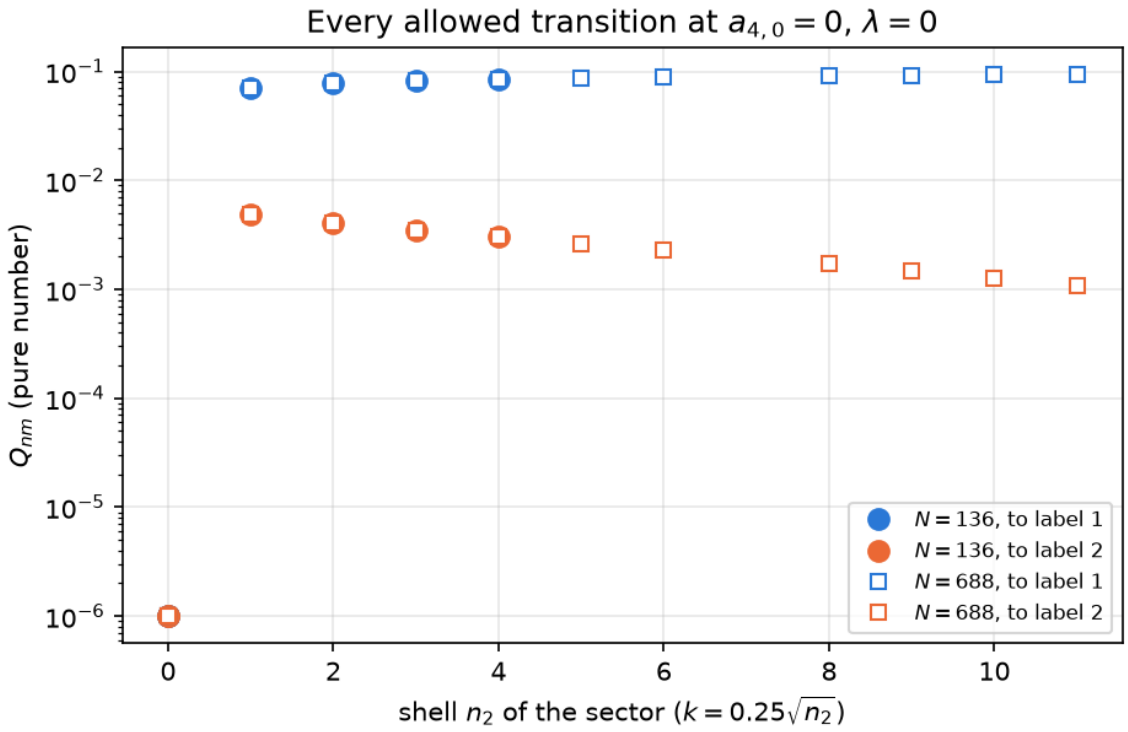

Figure 427: The adiabaticity measure Q_{nm} (vertical axis, logarithmic, pure number) of every allowed transition of the free states $N = 136$ (filled) and $N = 688$ (open) at $a_{4,0} = 0$, against the shell n_2 of the sector (horizontal axis); colour: the label of the empty level. Transitions at $k = 0$ vanish and are drawn at the bottom (10^{-6}). All values lie below 0.1, so the instantaneous states are followed adiabatically to a good approximation. (Notebook 15c, figure 2.)

```
## 9. Q along the history, for every state of the matrix

The next cell reads $Q_{max}$ of all 75 ground states from the record (with
interaction the self-consistent potentials contribute too) and draws it against the
slice, together with the values computed here for $\lambda = 0$ (black crosses).
$Q_{max}$ falls along the history, because the matrix element carries the factor
$k e^{-a_{4,0}}$. The right panel shows the transition probability $Q_{max}^2$.
```

In [7]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
tags = [("lam0", "$0$"), ("lamp1", "$+\\lambda_1$"), ("lamm1", "$-\\lambda_1$"),
        ("lamp2", "$+\\lambda_2$"), ("lamm2", "$-\\lambda_2$")]
record_max = 0.0
for style, n in (("-", 136), ("--", 688)):
    for colour, (tag, label) in zip(PALETTE, tags):
        values = [float(adiabatic[state_id(n, a4, tag)]["Q_max"]) for a4 in SLICES]
        record_max = max(record_max, max(values))
        left.plot(SLICES, values, style, color=colour, lw=1.5, marker="o", ms=4,
                  label=f"$N = {n}$, $\\lambda = $ {label}")
        right.plot(SLICES, [v * v for v in values], style, color=colour, lw=1.5,
                   marker="o", ms=4)
    mine = [max(q for q, _, _ in pairs[(n, a4)]) for a4 in SLICES]
    left.plot(SLICES, mine, "x", color="black", ms=10, mew=2)
left.plot([], [], "x", color="black", ms=10, mew=2, label="this notebook")
left.set_yscale("log")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("$Q_{max}$")
left.set_title("Largest $Q$ (record)")
left.legend(fontsize=7, ncol=2)
right.set_yscale("log")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$Q_{max}^2$")
right.set_title("Transition probability estimate")
save_figure(fig, "q_history",
            "Left: the largest adiabaticity measure $Q_{max}$ (vertical axis, "
            "logarithmic) of the 50 ground states with $N = 136$ (solid) and "
            "$N = 688$ (dashed) for the five couplings, against the slice "
            "(horizontal axis), from the record; black crosses: the values "
            "recomputed here for $\\lambda = 0$. Right: the transition probability "
            "estimate $Q_{max}^2$, below $0.009$ everywhere. Both fall along the "
            "history; for $N = 8$ every $Q$ is exactly zero (only $k = 0$ is "
            "occupied).")
n8_zero = all(float(adiabatic[state_id(8, a4, tag)]["Q_max"]) == 0.0
              for a4 in SLICES for tag, _ in tags)
report("largest Q_max of the whole matrix (record)", f"{record_max:.4f}")
check(record_max <= 0.0935 and n8_zero,
      "Q_max <= 0.0935 over the matrix, and Q = 0 exactly for N = 8",
      record=f"{ADIABATIC}, column Q_max")
falling = all(max(q for q, _, _ in pairs[(n, SLICES[i + 1])])
              < max(q for q, _, _ in pairs[(n, SLICES[i])])
              for n in (136, 688) for i in range(4))
check(falling, "Q_max of the free states falls from slice to slice")
```

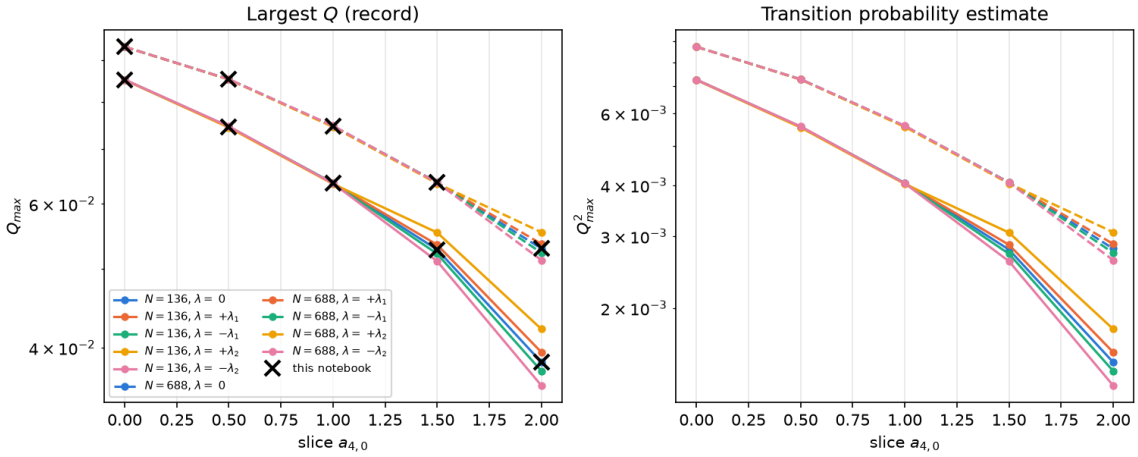

Figure 428: Left: the largest adiabaticity measure Q_{max} (vertical axis, logarithmic) of the 50 ground states with $N = 136$ (solid) and $N = 688$ (dashed) for the five couplings, against the slice (horizontal axis), from the record; black crosses: the values recomputed here for $\lambda = 0$. Right: the transition probability estimate Q^2_{max}, below 0.009 everywhere. Both fall along the history; for $N = 8$ every Q is exactly zero (only $k = 0$ is occupied). (Notebook 15c, figure 3.)

Out [7]:

```
Figure 15c.3 saved as Revision/textbook/figures/15c_3_q_history.png
RESULT largest Q_max of the whole matrix (record) = 0.0935
PASS Q_max <= 0.0935 over the matrix, and Q = 0 exactly for N = 8
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max
PASS Q_max of the free states falls from slice to slice
```

Text cell:

```
## 10. The Hellmann-Feynman theorem and the energy derivative

The next cell takes the occupied brane-band levels of the free state $N = 688$ at
$a_{4,0} = 1$ (shells $n_2 = 1$ to $11$), computes each level also at
$a_{4,0} \pm \delta$ and $\pm 2\delta$ with $\delta = 0.002$ (the solver's step),
forms the **Richardson** combination of the two central differences,
$\tfrac{1}{3}[4 D(\delta) - D(2\delta)]$ with $D(s) = [\varepsilon(a+s) -
\varepsilon(a-s)]/(2s)$ (it removes the error proportional to $\delta^2$), and
compares with the matrix element $\langle n|\partial_a h|n\rangle$. Then it adds up
$dE/da_4 = \sum g f \langle n|\partial_a h|n\rangle$ over all occupied levels,
with the degeneracy $g = 4 r_3(n_2)$, and compares with the derivative the solver
obtained by finite differences of $E$ (column 'dE_da4_finite_difference').
```

In [8]:

```
DELTA = 2e-3
a0 = 1.0
state = states[(688, a0)]
```

```
shells, finite, element = [], [], []
for sector, levels in sorted(state.items()):
    if sector[0] == 0:
        continue  # the zero modes do not depend on a4,0
    eps, f, k, r3, orb = levels[0]  # the occupied brane-band level (label 0)
    near = {s: find_level(k, sector[1], sector[2], 0, a0 + s, eps)
            for s in (DELTA, -DELTA, 2 * DELTA, -2 * DELTA)}
    d1 = (near[DELTA] - near[-DELTA]) / (2 * DELTA)
    d2 = (near[2 * DELTA] - near[-2 * DELTA]) / (4 * DELTA)
    shells.append(k)
    finite.append((4.0 * d1 - d2) / 3.0)
    element.append(da_h(k, sector[1], a0, orb, orb))
hf_dev = max(abs(a - b) for a, b in zip(finite, element))
de_da = sum(4.0 * levels[l][3] * levels[l][1] * da_h(levels[l][2], sector[1], a0,
                                                     levels[l][4], levels[l][4])
            for sector, levels in state.items() for l in levels)
recorded = float(adiabatic[state_id(688, a0)]["dE_da4_finite_difference"])
fig, ax = plt.subplots()
ax.plot(shells, finite, "o", color=PALETTE[0], ms=9, label="finite differences")
ax.plot(shells, element, "x", color=PALETTE[1], ms=10, mew=2,
        label="$\\langle n|\\partial_a h|n\\rangle$")
ax.set_xlabel("3-momentum $k$ of the shell (units of $m$)")
ax.set_ylabel("$d\\varepsilon/da_4$ (units of $m$)")
ax.set_title("Hellmann-Feynman: $N = 688$, $\\lambda = 0$, $a_{4,0} = 1$")
ax.legend()
save_figure(fig, "hellmann_feynman",
            "The derivative $d\\varepsilon/da_4$ of the eleven occupied brane-band "
            "levels of the free state $N = 688$ at $a_{4,0} = 1$ (vertical axis, "
            "units of $m$) against their momentum $k$ (horizontal axis): circles "
            "from Richardson finite differences of the levels, crosses from the "
            "matrix element of $\\partial_a h = -j\\kappa k\\sigma_3$. They agree "
            "(the Hellmann-Feynman theorem); every level falls along the history, "
            "faster for larger $k$.")
report("largest |finite difference - matrix element|", f"{hf_dev:.1e}")
report("dE/da4 of N688_lam0_a10: Hellmann-Feynman / record",
       f"{de_da:.10f} / {recorded:.10f}")
check(hf_dev < 1e-7, "d eps/da4 = <n| d_a h |n> for the eleven levels within 1e-7",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "adiabatic_hellmann_feynman")
check(abs(de_da / recorded - 1.0) < 1e-9, "dE/da4 = sum g f <n| d_a h |n>",
      record=f"{ADIABATIC}, N688_lam0_a10, column dE_da4_finite_difference")
```

Out [8]:

```
Figure 15c.4 saved as Revision/textbook/figures/15c_4_hellmann_feynman.png
RESULT largest |finite difference - matrix element| = 1.7e-12
RESULT dE/da4 of N688_lam0_a10: Hellmann-Feynman / record = -253.0160345318 /
    -253.0160345309
PASS d eps/da4 = <n| d_a h |n> for the eleven levels within 1e-7
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    adiabatic_hellmann_feynman
PASS dE/da4 = sum g f <n| d_a h |n>
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, N688_lam0_a10,
```

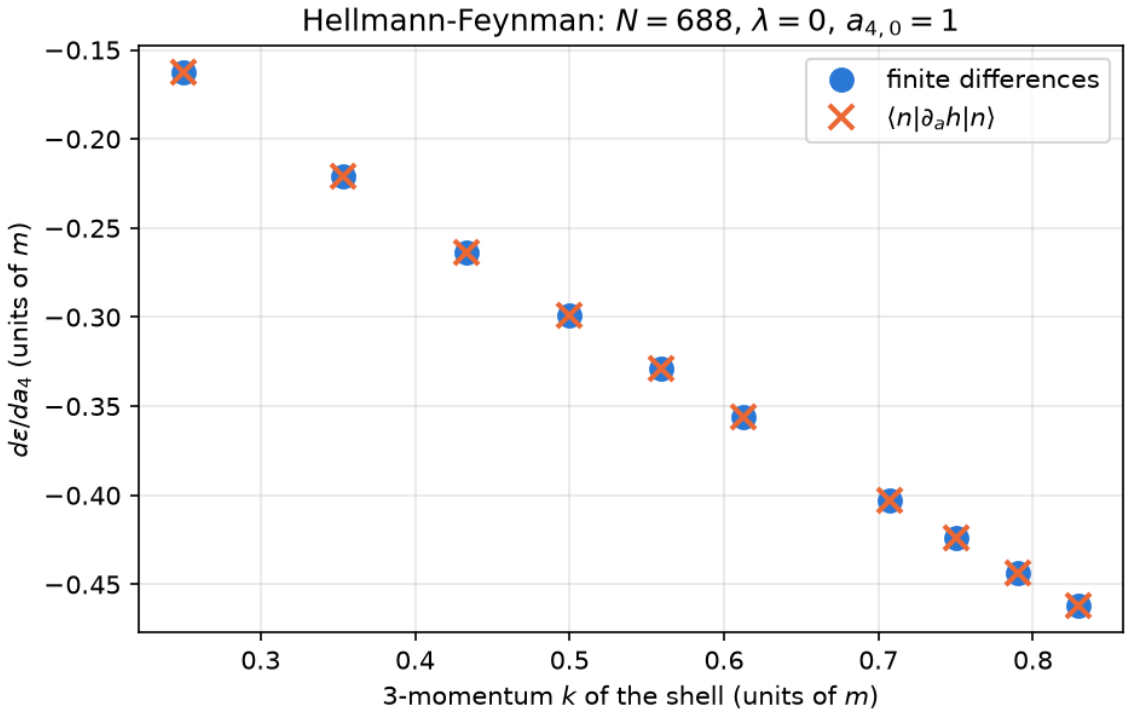

Figure 429: The derivative $d\varepsilon/da_4$ of the eleven occupied brane-band levels of the free state $N = 688$ at $a_{4,0} = 1$ (vertical axis, units of m) against their momentum k (horizontal axis): circles from Richardson finite differences of the levels, crosses from the matrix element of $\partial_a h = -j\kappa k\sigma_3$. They agree (the Hellmann-Feynman theorem); every level falls along the history, faster for larger k. (Notebook 15c, figure 4.)

```
    column dE_da4_finite_difference
```

Text cell:

```
## 11. When the Fermi level is crossed

Adiabatic evolution keeps the occupation of every level. If an empty level comes
down below an occupied one along the history (a **Fermi-level crossing**), the
instantaneous ground state fills the new lower level, but the adiabatically
evolving gas cannot get there: the two levels lie in different sectors, and the
exact evolution never moves a particle between sectors. The canonical matrix has no
such crossing (the record file ‘adiabatic/fermi-level-crossings.csv‘). The record’s
demonstration uses $N = 696 = 688 + 8$: at $a_{4,0} = 0$ the eight extra particles
sit in the odd bulk level at $k = 0$ ($\varepsilon = 1.2923$, which does not move),
while the brane band of the shell $n_2 = 12$ ($k = 0.25\sqrt{12} = 0.866$, 32
orbitals) lies above it and comes down along the history. The next cell computes
both levels, finds the slice where they cross by bisection, and computes the
energy by which the adiabatically continued state lies above the instantaneous
ground state, $8\,(\varepsilon_{odd} - \varepsilon_{12})$ (the eight particles of
the $k = 0$ level would sit in the band level instead), at the five slices.
```

In [9]:

```
K12 = 0.25 * math.sqrt(12.0)
eps_odd = find_level(0.0, 1, "odd", 0, 0.0, 1.29)  # the k = 0 odd bulk level


def band12(a4):
    """The brane-band level of the shell n2 = 12 at the slice a4."""
    return find_level(K12, 1, "even", 0, a4, 1.9 * K12 * math.exp(-a4))


lo, hi = 0.0, 0.5  # bisection for band12(a4) = eps_odd
for _ in range(45):
    mid = 0.5 * (lo + hi)
    lo, hi = (mid, hi) if band12(mid) > eps_odd else (lo, mid)
crossing = 0.5 * (lo + hi)
dense = np.linspace(0.0, 2.0, 41)
band_curve = [band12(a) for a in dense]
with open(repository_file("Revision/kohn_sham/results/adiabatic/crossing-demo.csv"),
          newline="", encoding="utf-8") as handle:
    demo = {float(row["a4"]): row for row in csv.DictReader(handle)}
mine = {a4: max(0.0, 8.0 * (eps_odd - band12(a4))) for a4 in SLICES}
worst_demo = max(abs(mine[a4] - float(demo[a4]["difference"])) for a4 in SLICES)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
left.plot(dense, band_curve, color=PALETTE[0], lw=2.0,
          label="brane band $n_2 = 12$ (32 orbitals)")
left.axhline(eps_odd, color=PALETTE[1], lw=2.0, label="odd bulk level, $k = 0$ (8)")
left.axvline(crossing, color="0.4", ls=":", lw=1.2, label="crossing")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("level (units of $m$)")
left.set_title("Two levels near the Fermi level, $N = 696$")
```

```
left.legend(fontsize=8)
right.plot(SLICES, [float(demo[a]["difference"]) for a in SLICES], "o", color="black",
           ms=9, label="record")
right.plot(SLICES, [mine[a] for a in SLICES], "x", color=PALETTE[0], ms=10, mew=2,
           label="$8(\\varepsilon_{odd} - \\varepsilon_{12})$, this notebook")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$E_{continued} - E_{aufbau}$ (units of $m$)")
right.set_title("Energy above the instantaneous ground state")
right.legend(fontsize=8)
save_figure(fig, "fermi_crossing",
            "The Fermi-level crossing of the demonstration state $N = 696$ without "
            "interaction. Left: the brane-band level of the shell $n_2 = 12$ (blue) "
            "falls along the history and crosses the fixed odd bulk level at $k = 0$ "
            "(orange) almost at once, at $a_{4,0} = 0.0029$ (dotted line), because "
            "at $a_{4,0} = 0$ it lies only $0.0032\\,m$ above it; vertical axis in "
            "units of $m$. Right: the energy by which the adiabatically continued "
            "state lies "
            "above the instantaneous aufbau state (vertical axis, units of $m$), "
            "from the record (dots) and from the two levels (crosses), at the five "
            "slices.")
report("band level n2 = 12 minus the odd level at a4,0 = 0",
       f"{band_curve[0] - eps_odd:.10f}")
report("crossing slice a4,0", f"{crossing:.6f}")
report("largest |8 (eps_odd - eps_12) - recorded difference|", f"{worst_demo:.1e}")
check(0.0 < crossing < 0.5, "the crossing lies between the slices 0 and 0.5",
      record="Revision/kohn_sham/results/adiabatic/crossing-demo.csv, flags at 0 "
             "and 0.5")
check(worst_demo < 1e-9, "the energy differences of the demonstration reproduced",
      record="Revision/kohn_sham/results/adiabatic/crossing-demo.csv, column "
             "difference")
```

Out [9]:

```
Figure 15c.5 saved as Revision/textbook/figures/15c_5_fermi_crossing.png
RESULT band level n2 = 12 minus the odd level at a4,0 = 0 = 0.0032325390
RESULT crossing slice a4,0 = 0.002854
RESULT largest |8 (eps_odd - eps_12) - recorded difference| = 2.0e-11
PASS the crossing lies between the slices 0 and 0.5
     reproduces Revision/kohn_sham/results/adiabatic/crossing-demo.csv, flags at 0 and
    0.5
PASS the energy differences of the demonstration reproduced
     reproduces Revision/kohn_sham/results/adiabatic/crossing-demo.csv, column difference
```

Text cell:

```
## 12. The whole matrix at a glance

The last figure is a heat map of $Q_{max}$ from the record: one row per series
(particle number and coupling), one column per slice, darker for larger values.
The check reads the record file of Fermi-level crossings and confirms that none of
its 60 rows (15 series, 4 steps between slices) reports a change of the occupied set.
```

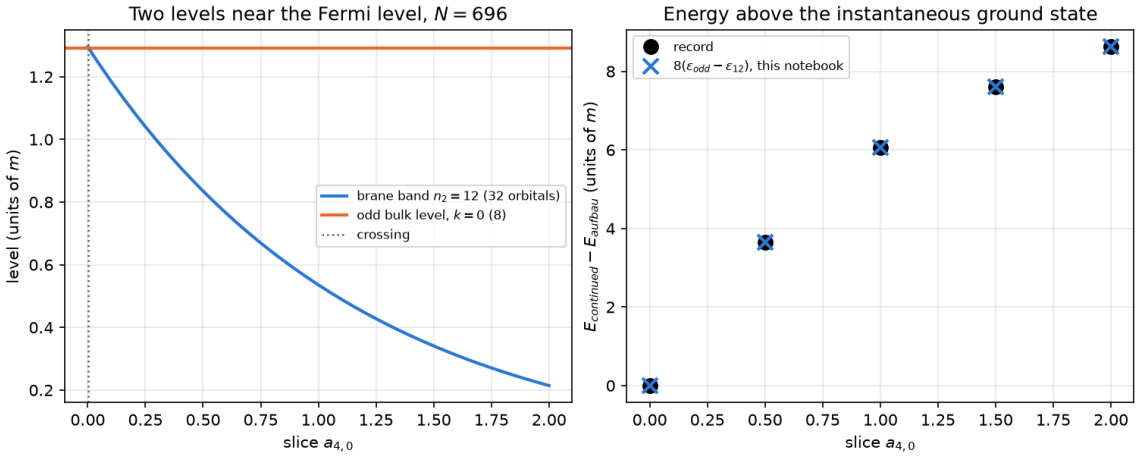

Figure 430: The Fermi-level crossing of the demonstration state $N = 696$ without interaction. Left: the brane-band level of the shell $n_2 = 12$ (blue) falls along the history and crosses the fixed odd bulk level at $k = 0$ (orange) almost at once, at $a_{4,0} = 0.0029$ (dotted line), because at $a_{4,0} = 0$ it lies only $0.0032\,m$ above it; vertical axis in units of m. Right: the energy by which the adiabatically continued state lies above the instantaneous aufbau state (vertical axis, units of m), from the record (dots) and from the two levels (crosses), at the five slices. (Notebook 15c, figure 5.)

In [10]:

```
rows, names = [], []
for n in (8, 136, 688):
    for tag, label in tags:
        rows.append([float(adiabatic[state_id(n, a4, tag)]["Q_max"]) for a4 in SLICES])
        names.append(f"N = {n}, {tag}")
table = np.array(rows)
fig, ax = plt.subplots(figsize=(6.5, 6.0))
image = ax.imshow(table, cmap="Blues", aspect="auto", vmin=0.0, vmax=0.1)
ax.grid(False)  # no grid lines across the coloured squares
ax.set_xticks(range(5), [f"{a:.1f}" for a in SLICES])
ax.set_yticks(range(len(names)), names, fontsize=8)
for i in range(table.shape[0]):
    for jj in range(table.shape[1]):
        ax.text(jj, i, f"{table[i, jj]:.3f}", ha="center", va="center", fontsize=7,
                color="white" if table[i, jj] > 0.06 else "black")
ax.set_xlabel("slice $a_{4,0}$")
ax.set_title("$Q_{max}$ of the 75 ground states (record)")
fig.colorbar(image, ax=ax, label="$Q_{max}$")
save_figure(fig, "q_map",
            "Heat map of the largest adiabaticity measure $Q_{max}$ of the 75 ground "
            "states of the canonical matrix (record): rows are the particle numbers "
            "$N = 8$, $136$, $688$ with the couplings $0$, $\\pm\\lambda_1$, "
            "$\\pm\\lambda_2$ (tags lam0, lamp1, lamm1, lamp2, lamm2), columns the "
            "slices $a_{4,0}$; the printed numbers are the values. Every value is "
            "below $0.1$, the largest at the start of the history.")
with open(repository_file("Revision/kohn_sham/results/adiabatic/"
                          "fermi-level-crossings.csv"), newline="",
          encoding="utf-8") as handle:
```

```
    crossings = list(csv.DictReader(handle))
changed = sum(row["occupied_set_changed"] == "true" for row in crossings)
report("rows of the crossing record / rows with a changed occupied set",
       f"{len(crossings)} / {changed}")
check(len(crossings) == 60 and changed == 0,
      "no Fermi-level crossing in the canonical matrix",
      record="Revision/kohn_sham/results/adiabatic/fermi-level-crossings.csv")
```

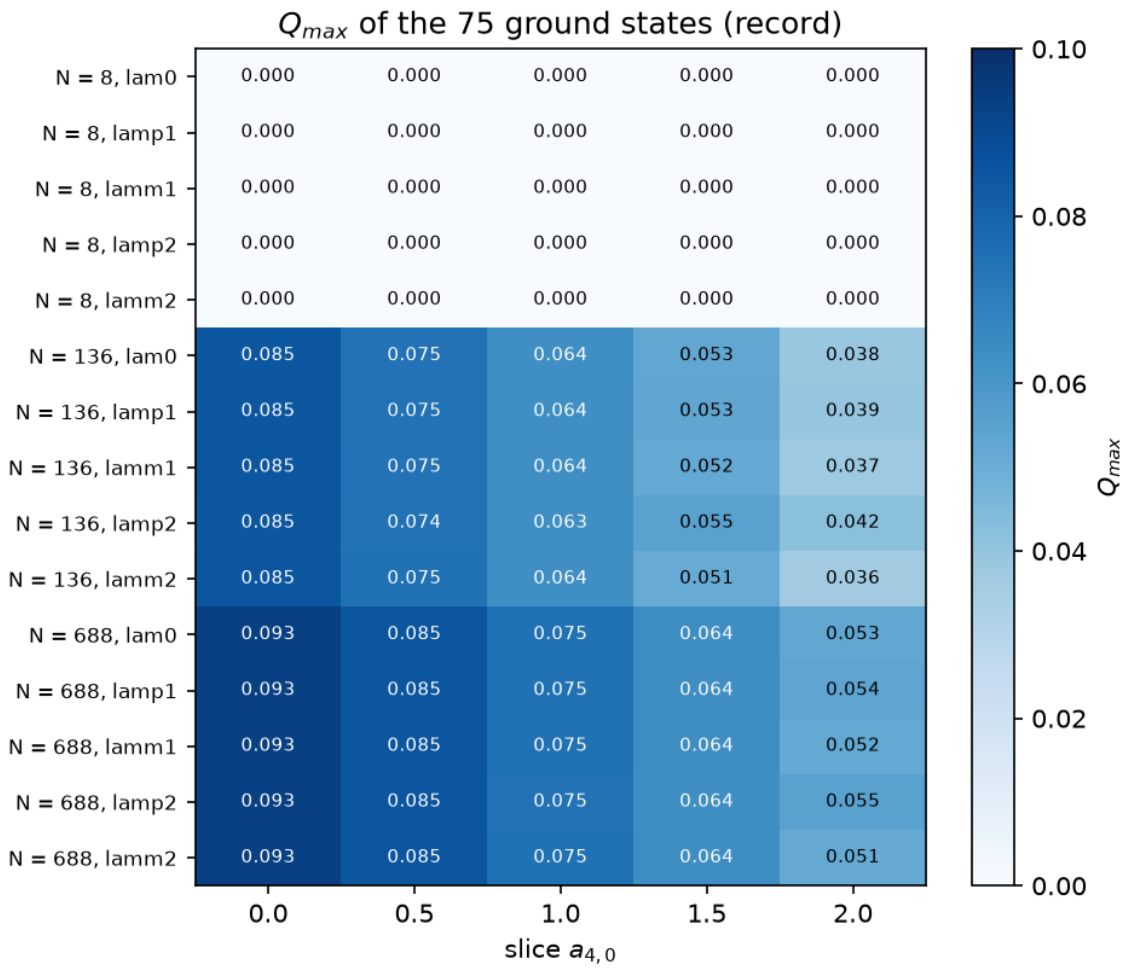

Figure 431: Heat map of the largest adiabaticity measure Q_{max} of the 75 ground states of the canonical matrix (record): rows are the particle numbers $N = 8$, 136, 688 with the couplings 0, $\pm\lambda_1$, $\pm\lambda_2$ (tags lam0, lamp1, lamm1, lamp2, lamm2), columns the slices $a_{4,0}$; the printed numbers are the values. Every value is below 0.1, the largest at the start of the history. (Notebook 15c, figure 6.)

Out [10]:

```
Figure 15c.6 saved as Revision/textbook/figures/15c_6_q_map.png
RESULT rows of the crossing record / rows with a changed occupied set = 60 / 0
PASS no Fermi-level crossing in the canonical matrix
     reproduces Revision/kohn_sham/results/adiabatic/fermi-level-crossings.csv
```

Text cell:

```
## 13. The last check

The last cell checks that every figure file of this notebook exists and prints the
number of checks that passed.
```

In [11]:

```
NAMES = ["transition_orbitals", "q_by_pair", "q_history", "hellmann_feynman",
         "fermi_crossing", "q_map"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15c_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

Out [11]:

```
PASS every figure file of this notebook exists
ALL 12 CHECKS PASSED (notebook 15c)
```

Text cell:

```
## 14. What this notebook showed

- The adiabaticity measure $Q_{nm} = AH|\langle n|\partial_a h|m\rangle|/
  (\varepsilon_n - \varepsilon_m)^2$ is the first-order amplitude of a jump between
  instantaneous levels of one sector; its derivation needs only the product rule and
  the integral of a rotating phase.
- Recomputed in plain Python, every level, the largest $Q$, its pair, its energy
  difference and its matrix element of the free states agree with the Rust
  solver's record (COMPUTED).
- $Q_{max} \le 0.0935$ over the whole canonical matrix (transition probability
  below $0.009$) and it decreases along the history: the instantaneous states are
  followed adiabatically, within the sectors, to a good approximation.
- The Hellmann-Feynman theorem holds, and the energy falls along the history at the
  rate $\sum g f\langle n|\partial_a h|n\rangle$.
- When a level crosses the Fermi level (the demonstration $N = 696$), the
  instantaneous ground state is not the state that the evolution reaches: there the
  instantaneous picture fails, by $3.66$ to $8.62\,m$ in energy. The full
  time-dependent problem is OPEN.
```

15.25 Line-by-line walk-through of Notebook 15c

The notebook has 11 code cells, In [1] to In [11]; the complete cells are printed in Section 15.24.

In [1], the set-up cell. It is the set-up cell of Notebook 15e, explained line by line in Section 15.9, with the run instructions of this notebook (Section 15.23) in its comments and `NOTEBOOK_ID = "15c"`.

In [2], the solver's method for the free problem.

```
import csv  # reads the tables (CSV files) of the Revision record
import math  # exp, sqrt, atan2, pi for single numbers

import numpy as np  # arrays of numbers

L, STEPS = 3.0, 900  # tip cutoff and number of RK4 steps (the solver's values)
POINTS = 2 * STEPS + 1  # step ends and midpoints
STEP = L / STEPS
Y = np.array([-L * ((POINTS - 1 - f) / (POINTS - 1)) for f in range(POINTS)])
EW_LIST = [math.exp(-y) for y in Y.tolist()]  # e^{-Hy} with H = 1, as the solver
EW = np.array(EW_LIST)
```

The imports; $L = 3$ and $G = 900$ steps; the 1801 fine points `Y` (the formula of Notebook 15e, Section 15.9 In [2]); and the factor e^{-Hy} at every fine point, once as a list (fast for single numbers in the loop) and once as an array.

```
SIMPSON = np.full(POINTS, 2.0 * STEP / 6.0)  # Simpson weights on the fine grid
SIMPSON[1::2] = 4.0 * STEP / 6.0
SIMPSON[0] = SIMPSON[-1] = STEP / 6.0
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

The Simpson weights $\frac{h}{6}$ times $1, 4, 2, 4, \ldots, 4, 1$ on the fine grid (Section 15.3), and the colours.

```
def shoot(eps, k, j, a4, store=False):
    kk = k * math.exp(-a4)
    a, b, theta, raw, area, r2_prev = 1.0, 0.0, 0.0, 0.0, 0.0, 1.0
    je = j * eps  # j (eps - v) with v = 0
    nodes = [(a, b)] if store else None
```

The free version of `shoot` of Notebook 15e: the mass is 1 and $v = 0$ everywhere, so they need not be passed. The start values are those of Section 15.9 (In [3]), and `je` is $j\varepsilon$.

```
    for i in range(STEPS):
        k0 = kk * EW_LIST[2 * i]  # kappa k at the start of the step
        k1 = kk * EW_LIST[2 * i + 1]  # at its midpoint
        k2 = kk * EW_LIST[2 * i + 2]  # at its end
        p1a, p1b = a - (k0 + je) * b, (je - k0) * a - b  # (a', b') with M = 1
        aa, bb = a + 0.5 * STEP * p1a, b + 0.5 * STEP * p1b
        p2a, p2b = aa - (k1 + je) * bb, (je - k1) * aa - bb
        aa, bb = a + 0.5 * STEP * p2a, b + 0.5 * STEP * p2b
        p3a, p3b = aa - (k1 + je) * bb, (je - k1) * aa - bb
        aa, bb = a + STEP * p3a, b + STEP * p3b
        p4a, p4b = aa - (k2 + je) * bb, (je - k2) * aa - bb
        a += STEP / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)
        b += STEP / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
```

The RK4 step written out without the helper `rhs`: κk at the start, the middle and the end of the step; the four slopes (a', b') of the block equation with $M = 1$, each at the prediction made with the previous slope (`aa`, `bb`); and the weighted average $\frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$.

```
        new = math.atan2(b, a)  # the Pruefer angle, followed continuously:
        change = new - raw  # bring the change of angle into (-pi, pi]
        if change > math.pi:
            change -= 2.0 * math.pi
        elif change <= -math.pi:
            change += 2.0 * math.pi
        theta, raw = theta + change, new
        r2 = a * a + b * b
        area, r2_prev = area + 0.5 * STEP * (r2_prev + r2), r2  # trapezoid rule
        if store:
            nodes.append((a, b))
    return j * theta, area / r2_prev, nodes  # Phi and dPhi/deps = int r^2 / r(0)^2
```

The Pruefer angle followed continuously, the trapezoid sum of r^2, the stored values, and the result $(\Phi, d\Phi/d\varepsilon, \text{nodes})$, as in Section 15.9.

```
def find_level(k, j, parity, label, a4, guess, tol=1e-13):
    target = label * math.pi + (0.0 if parity == "even" else 0.5 * math.pi)
    e = guess
    g, d = shoot(e, k, j, a4)[:2]
    g -= target
    if g == 0.0:
        return e
    step = min(max(abs(g / d) * 1.2, 1e-4), 2.0)  # the first trial step
```

`find_level` is the safeguarded Newton method of Section 15.9 (In [5]) for the free problem: the target, the value $g = \Phi - t_l$ and the derivative d at the guess (`[:2]` keeps the first two results of `shoot`), and the first trial step.

```
    if g < 0:  # Phi too small: walk up with doubling steps until Phi >= target
        lo = e
        while True:
            trial = lo + step
            gt, dt = shoot(trial, k, j, a4)[:2]
            gt -= target
            if gt >= 0:
                hi = trial  # [lo, hi] brackets the level
                if abs(gt) < abs(g):
                    e, g, d = trial, gt, dt  # keep the better end for Newton
                break
            lo, e, g, d = trial, trial, gt, dt
            step *= 2.0
    else:  # Phi too large: walk down until Phi <= target
        hi = e
        while True:
            trial = hi - step
            gt, dt = shoot(trial, k, j, a4)[:2]
            gt -= target
            if gt <= 0:
                lo = trial
                if abs(gt) < abs(g):
```

```
                e, g, d = trial, gt, dt
            break
        hi, e, g, d = trial, trial, gt, dt
        step *= 2.0
```

The walk with doubling steps up or down until the target is bracketed, exactly as in Notebook 15e.

```
    previous = math.inf
    while hi - lo > tol and g != 0.0:
        new = e - g / d  # Newton step, or bisection if it leaves the bracket
        bisect = not lo < new < hi or abs(g) > 0.5 * previous
        if bisect:
            new = 0.5 * (lo + hi)
        previous = abs(g)
        moved = abs(new - e)
        e = new
        g, d = shoot(e, k, j, a4)[:2]
        g -= target
        if g < 0:
            lo = e
        elif g > 0:
            hi = e
        if moved <= tol and not bisect:
            break  # a Newton step that moves less than tol: converged
    return e
```

The refinement by Newton steps with bisection as the safeguard, written as a `while` loop that runs as long as the bracket is longer than the tolerance and the target is not hit exactly; otherwise as in Notebook 15e.

```
def orbital(eps, k, j, a4):
    nodes = np.array(shoot(eps, k, j, a4, store=True)[2])
    kap = k * math.exp(-a4) * EW[0::2]
    a_n, b_n = nodes[:, 0], nodes[:, 1]
    da = a_n - (kap + j * eps) * b_n  # derivatives at the step ends
    db = (j * eps - kap) * a_n - b_n
    a = np.zeros(POINTS)
    b = np.zeros(POINTS)
    a[0::2], b[0::2] = a_n, b_n
    a[1::2] = 0.5 * (a_n[:-1] + a_n[1:]) + STEP / 8.0 * (da[:-1] - da[1:])
    b[1::2] = 0.5 * (b_n[:-1] + b_n[1:]) + STEP / 8.0 * (db[:-1] - db[1:])
    norm = math.sqrt(float(np.sum(SIMPSON * (a * a + b * b))))
    return a / norm, b / norm
```

The normalised orbital on the fine grid, as in Section 15.9 (In [7]): the stored step-end values become a two-column array (`nodes[:, 0]` is its first column, a; `nodes[:, 1]` the second, b), the derivatives come from the block equation, the midpoints from the Hermite formula, and Simpson's rule normalises.

```
def da_h(k, j, a4, first, second):
    s3 = first[0] * second[0] - first[1] * second[1]
    return -j * k * math.exp(-a4) * float(np.sum(SIMPSON * EW * s3))
```

`da_h` is the matrix element $\langle n|\partial_a h|m\rangle = -j\,k\,e^{-a_{4,0}}\int e^{-Hy}(a_n a_m - b_n b_m)\,dy$ of Section 15.21, with the integral by Simpson's rule; `first` and `second` are orbitals, each a pair (a, b) of arrays. The cell defines functions only and prints nothing.

In [3], the levels of the record solved again.

```
LEVELS = "Revision/kohn_sham/results/ground/levels"
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]


def state_id(n, a4, tag="lam0"):
    return f"N{n}_{tag}_a{round(10 * a4):02d}"
```

The folder of the level tables, the five slices, and the state names as in Notebook 15a (here the coupling tag comes last, with the default `lam0`).

```
def solve_state(n, a4):
    rows = list(csv.DictReader(open(repository_file(f"{LEVELS}/{state_id(n, a4)}.csv"),
                                    newline="", encoding="utf-8")))
    sectors = {}
    for row in rows:
        sector = (int(row["n2"]), int(row["j"]), row["parity"])
        sectors.setdefault(sector, []).append(row)
```

`solve_state` reads the level table of a free state into a list of rows and groups the rows by sector: `setdefault(sector, [])` returns the list of that sector, creating an empty one the first time, and the row is appended to it.

```
    solved, worst = {}, 0.0
    for sector, members in sectors.items():
        if all(float(row["f"]) < 0.5 for row in members):
            continue  # no particle in this sector: no transition starts here
        solved[sector] = {}
        for row in members:
            k, eps_rec = float(row["k"]), float(row["eps"])
            eps = find_level(k, sector[1], sector[2], int(row["label"]), a4, eps_rec)
            worst = max(worst, abs(eps - eps_rec))
            solved[sector][int(row["label"])] = (
                eps, float(row["f"]), k, int(row["r3"]), orbital(eps, k, sector[1], a4))
    return solved, worst
```

Sectors without an occupied level are skipped: no transition can start there. For every level of the other sectors the energy is found again in Python, starting from the recorded value, and stored with its occupation, momentum, number of directions r_3 and orbital, under its sector and label; `worst` is the largest difference to the record.

```
states = {}
worst_level = 0.0
for n in (136, 688):
```

```
    for a4 in SLICES:
        states[(n, a4)], worst = solve_state(n, a4)
        worst_level = max(worst_level, worst)
count = sum(len(levels) for state in states.values() for levels in state.values())
report("levels solved again (10 states)", count)
report("largest |eps(Python) - eps(solver)|", f"{worst_level:.1e}")
check(worst_level < 1e-12, "every level equals the solver's level within 1e-12",
      record=f"{LEVELS}/N136_lam0_a*.csv and N688_lam0_a*.csv, column eps")
```

The ten free states $N = 136,688$ at the five slices are solved. Out [3]: 270 levels, the largest difference to the solver 9.3×10^{-14}, PASS.

In [4], the pair with the largest Q.

```
sector = (4, 1, "even")
band = states[(136, 0.0)][sector][0]  # (eps, f, k, r3, (a, b)) of label 0
bulk = states[(136, 0.0)][sector][1]  # label 1
k = band[2]
integrand = k * EW * (band[4][0] * bulk[4][0] - band[4][1] * bulk[4][1])
```

For $N = 136$ at $a_{4,0} = 0$ the record names the pair `4:+1:even:0 -> 4:+1:even:1`: in the sector $n_2 = 4$ ($k = 0.5$), $j = +1$, even, the occupied band level (label 0) and the first empty bulk level (label 1). `band[4]` is the orbital (a, b) of the band level; `integrand` is $\kappa k\,(a_n a_m - b_n b_m)$ at $a_{4,0} = 0$.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
left.plot(Y, band[4][0], color=PALETTE[0], lw=2.0, label="band: $a$")
left.plot(Y, band[4][1], color=PALETTE[0], lw=1.5, ls="--", label="band: $b$")
left.plot(Y, bulk[4][0], color=PALETTE[1], lw=2.0, label="bulk: $a$")
left.plot(Y, bulk[4][1], color=PALETTE[1], lw=1.5, ls="--", label="bulk: $b$")
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("orbital (normalised)")
left.set_title(f"$n_2 = 4$: band $\\varepsilon = {band[0]:.4f}$, "
               f"bulk $\\varepsilon = {bulk[0]:.4f}$", fontsize=10)
left.legend(fontsize=8)
```

The left panel draws both orbitals (a solid, b dashed), with their energies in the title.

```
right.plot(Y, integrand, color=PALETTE[2], lw=2.0)
right.fill_between(Y, integrand, color=PALETTE[2], alpha=0.25)
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("$\\kappa k\\,(a_n a_m - b_n b_m)$ (units of $m^2$)")
right.set_title("Integrand of the matrix element")
save_figure(fig, "transition_orbitals",
            "Left: the orbitals (components $a$ solid, $b$ dashed; vertical axis, "
...)
element = abs(da_h(k, 1, 0.0, band[4], bulk[4]))
report("|<band| d_a h |bulk>| and Q for N = 136, a4,0 = 0",
       f"{element:.13f}, {element / (band[0] - bulk[0]) ** 2:.13f}")
```

The right panel draws the integrand and shades the area between it and zero (`fill_between`), whose signed size is the matrix element; then Figure 15c.1. Out [4]: $|\langle n|\partial_a h|m\rangle| = 0.3588612$ and

$Q = 0.3588612/(2.052930)^2 = 0.0851488$. The band orbital sits at the brane and the bulk orbital oscillates, so the integrand changes sign and partly cancels; together with the large energy difference this keeps Q small.

In [5], Q for every allowed pair.

```
ADIABATIC = "Revision/kohn_sham/results/adiabatic/adiabaticity.csv"
with open(repository_file(ADIABATIC), newline="", encoding="utf-8") as handle:
    adiabatic = {row["id"]: row for row in csv.DictReader(handle)}


def key_text(sector, label):
    return f"{sector[0]}:{sector[1]:+d}:{sector[2]}:{label}"
```

The record's adiabaticity table is read. `key_text` writes a level as the record writes it, `n2:j:parity:label`; the format `+d` prints the block type with its sign.

```
pairs = {}  # (n, a4) -> list of (Q, n2, label m) for the figure
worst_q, same_pairs = 0.0, True
say("   N  a4,0   Q_max    pair                            delta eps   matrix el.")
for (n, a4), state in states.items():
    best = (0.0, "", 0.0, 0.0)
    pairs[(n, a4)] = []
    for sector, levels in state.items():
        for ln, (en, fn, k, _, on) in levels.items():
            if fn < 0.5:
                continue
            for lm, (em, fm, _, _, om) in levels.items():
                if fm > 0.5:
                    continue
                element = abs(da_h(k, sector[1], a4, on, om))
                q = element / (en - em) ** 2  # A H = 1
                pairs[(n, a4)].append((q, sector[0], lm))
                if q > best[0]:
                    best = (q, f"{key_text(sector, ln)} -> {key_text(sector, lm)}",
                            abs(en - em), element)
```

For each of the ten states and each occupied sector, the two inner loops run over every occupied level n (`fn` its occupation; empty ones are skipped) and every empty level m (occupied ones are skipped) of the same sector. For each pair the matrix element and $Q = |\langle n|\partial_a h|m\rangle|/(\varepsilon_n - \varepsilon_m)^2$ (with $AH = 1$) are computed and stored for the figure, and `best` keeps the largest Q with its pair, energy difference and matrix element.

```
    row = adiabatic[state_id(n, a4)]
    for value, column in ((best[0], "Q_max"), (best[2], "Q_max_delta_eps"),
                          (best[3], "Q_max_matrix_element")):
        worst_q = max(worst_q, abs(value / float(row[column]) - 1.0))
    same_pairs = same_pairs and best[1] == row["Q_max_pair"]
    say(f"{n:4d}  {a4:4.1f}  {best[0]:.5f}  {best[1]:28}  {best[2]:9.6f}"
        f"  {best[3]:9.6f}")
```

Still inside the loop over the states: the largest Q, its energy difference and its matrix element are compared with the record, the pair's name too, and a table row is printed.

```
report("largest relative difference to the record", f"{worst_q:.1e}")
check(worst_q < 1e-9, "Q_max, its energy difference and matrix element reproduced",
      record=f"{ADIABATIC}, columns Q_max, Q_max_delta_eps, Q_max_matrix_element")
check(same_pairs, "the pair with the largest Q is the recorded pair",
      record=f"{ADIABATIC}, column Q_max_pair")
```

Out [5] shows the table: Q_{max} falls from 0.08515 to 0.03839 for $N = 136$ and from 0.09345 to 0.05289 for $N = 688$ along the history, always for the jump from the band level of the highest occupied shell ($n_2 = 4$, respectively 11) to the bulk level above it in the same sector. The largest relative difference to the record is 6.8×10^{-14}, and the pairs are the recorded ones.

In [6], every pair at the first slice.

```
fig, ax = plt.subplots()
for marker, n in (("o", 136), ("s", 688)):
    for colour, label in zip(PALETTE, (1, 2)):
        points = [(n2, q) for q, n2, lm in pairs[(n, 0.0)] if lm == label]
        ax.plot([p[0] for p in points], [max(p[1], 1e-6) for p in points], marker,
                color=colour, ms=8 if n == 136 else 6,
                markerfacecolor=colour if n == 136 else "white",
                label=f"$N = {n}$, to label {label}")
```

For $N = 136$ (filled circles) and $N = 688$ (open squares) at $a_{4,0} = 0$, the cell draws Q of every pair against the shell of its sector, coloured by the label of the empty level (1 or 2). Values below 10^{-6} (the transitions at $k = 0$, which are exactly zero) are drawn at 10^{-6}, because a logarithmic axis cannot show zero.

```
ax.set_yscale("log")
ax.set_xlabel("shell $n_2$ of the sector ($k = 0.25\\sqrt{n_2}$)")
ax.set_ylabel("$Q_{nm}$ (pure number)")
ax.set_title("Every allowed transition at $a_{4,0} = 0$, $\\lambda = 0$")
ax.legend(fontsize=8)
save_figure(fig, "q_by_pair",
            "The adiabaticity measure $Q_{nm}$ (vertical axis, logarithmic, pure "
...)
largest = max(q for values in pairs.values() for q, _, _ in values)
report("largest Q of all free pairs (10 states)", f"{largest:.5f}")
check(largest < 0.1, "every Q of the free states is below 0.1")
```

Figure 15c.2, and the largest Q over all pairs of the ten states: 0.09345 (Out [6]), below 0.1. In the figure the jump to the label 1 dominates in every shell and grows with n_2, because the matrix element is proportional to k.

In [7], Q_{max} of the whole matrix.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
tags = [("lam0", "$0$"), ("lamp1", "$+\\lambda_1$"), ("lamm1", "$-\\lambda_1$"),
```

```
        ("lamp2", "$+\\lambda_2$"), ("lamm2", "$-\\lambda_2$")]
record_max = 0.0
for style, n in (("-", 136), ("--", 688)):
    for colour, (tag, label) in zip(PALETTE, tags):
        values = [float(adiabatic[state_id(n, a4, tag)]["Q_max"]) for a4 in SLICES]
        record_max = max(record_max, max(values))
        left.plot(SLICES, values, style, color=colour, lw=1.5, marker="o", ms=4,
                  label=f"$N = {n}$, $\\lambda = $ {label}")
        right.plot(SLICES, [v * v for v in values], style, color=colour, lw=1.5,
                   marker="o", ms=4)
```

For $N = 136$ (solid) and 688 (dashed) and the five couplings the recorded Q_{max} at the five slices is drawn on the left and its square, the transition probability, on the right; `record_max` keeps the largest value.

```
    mine = [max(q for q, _, _ in pairs[(n, a4)]) for a4 in SLICES]
    left.plot(SLICES, mine, "x", color="black", ms=10, mew=2)
left.plot([], [], "x", color="black", ms=10, mew=2, label="this notebook")
left.set_yscale("log")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("$Q_{max}$")
left.set_title("Largest $Q$ (record)")
left.legend(fontsize=7, ncol=2)
right.set_yscale("log")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$Q_{max}^2$")
right.set_title("Transition probability estimate")
save_figure(fig, "q_history",
            "Left: the largest adiabaticity measure $Q_{max}$ (vertical axis, "
...)
```

The values recomputed in this notebook for $\lambda = 0$ are drawn as black crosses (`mew` is the width of the cross lines), then the legend, axes and Figure 15c.3.

```
n8_zero = all(float(adiabatic[state_id(8, a4, tag)]["Q_max"]) == 0.0
              for a4 in SLICES for tag, _ in tags)
report("largest Q_max of the whole matrix (record)", f"{record_max:.4f}")
check(record_max <= 0.0935 and n8_zero,
      "Q_max <= 0.0935 over the matrix, and Q = 0 exactly for N = 8",
      record=f"{ADIABATIC}, column Q_max")
falling = all(max(q for q, _, _ in pairs[(n, SLICES[i + 1])])
              < max(q for q, _, _ in pairs[(n, SLICES[i])])
              for n in (136, 688) for i in range(4))
check(falling, "Q_max of the free states falls from slice to slice")
```

Out [7]: the largest Q_{max} of the matrix is 0.0935, every Q of $N = 8$ is exactly zero (only $k = 0$ is occupied, where $\partial_a h = 0$), and Q_{max} of the free states falls from slice to slice.

In [8], the Hellmann-Feynman theorem.

```
DELTA = 2e-3
a0 = 1.0
```

```
state = states[(688, a0)]
shells, finite, element = [], [], []
for sector, levels in sorted(state.items()):
    if sector[0] == 0:
        continue  # the zero modes do not depend on a4,0
    eps, f, k, r3, orb = levels[0]  # the occupied brane-band level (label 0)
    near = {s: find_level(k, sector[1], sector[2], 0, a0 + s, eps)
            for s in (DELTA, -DELTA, 2 * DELTA, -2 * DELTA)}
```

For the free state $N = 688$ at $a_{4,0} = 1$ the cell visits the occupied sectors in sorted order, skips the shell $n_2 = 0$, and takes the occupied band level (label 0) of each shell; it solves this level again at the four slices 1 ± 0.002 and 1 ± 0.004.

```
    d1 = (near[DELTA] - near[-DELTA]) / (2 * DELTA)
    d2 = (near[2 * DELTA] - near[-2 * DELTA]) / (4 * DELTA)
    shells.append(k)
    finite.append((4.0 * d1 - d2) / 3.0)
    element.append(da_h(k, sector[1], a0, orb, orb))
hf_dev = max(abs(a - b) for a, b in zip(finite, element))
```

`d1` and `d2` are the central differences $D(\delta)$ and $D(2\delta)$, $D(s) = [\varepsilon(a+s) - \varepsilon(a-s)]/(2s)$; their error is proportional to s^2, so the Richardson combination $\frac{1}{3}(4D(\delta) - D(2\delta))$ removes it (as in Section 15.9, In [9]). `element` is the diagonal matrix element $\langle n|\partial_a h|n\rangle$, and `hf_dev` the largest difference between the two.

```
de_da = sum(4.0 * levels[l][3] * levels[l][1] * da_h(levels[l][2], sector[1], a0,
                                                     levels[l][4], levels[l][4])
            for sector, levels in state.items() for l in levels)
recorded = float(adiabatic[state_id(688, a0)]["dE_da4_finite_difference"])
```

`de_da` is $\sum g f \langle n|\partial_a h|n\rangle$ over all levels of the occupied sectors, with $g = 4r_3$ (`levels[l][3]` is r_3, `levels[l][1]` the occupation, which is 0 for the empty levels); `recorded` is the derivative of E that the solver obtained by finite differences.

```
fig, ax = plt.subplots()
ax.plot(shells, finite, "o", color=PALETTE[0], ms=9, label="finite differences")
ax.plot(shells, element, "x", color=PALETTE[1], ms=10, mew=2,
        label="$\\langle n|\\partial_a h|n\\rangle$")
ax.set_xlabel("3-momentum $k$ of the shell (units of $m$)")
ax.set_ylabel("$d\\varepsilon/da_4$ (units of $m$)")
ax.set_title("Hellmann-Feynman: $N = 688$, $\\lambda = 0$, $a_{4,0} = 1$")
ax.legend()
save_figure(fig, "hellmann_feynman",
            "The derivative $d\\varepsilon/da_4$ of the eleven occupied brane-band "
...)
report("largest |finite difference - matrix element|", f"{hf_dev:.1e}")
report("dE/da4 of N688_lam0_a10: Hellmann-Feynman / record",
       f"{de_da:.10f} / {recorded:.10f}")
check(hf_dev < 1e-7, "d eps/da4 = <n| d_a h |n> for the eleven levels within 1e-7",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
```

```
            "adiabatic_hellmann_feynman")
check(abs(de_da / recorded - 1.0) < 1e-9, "dE/da4 = sum g f <n| d_a h |n>",
      record=f"{ADIABATIC}, N688_lam0_a10, column dE_da4_finite_difference")
```

Figure 15c.4 draws both derivatives of the eleven band levels against their momentum: circles and crosses coincide, and every level falls, faster for larger k. Out [8]: the largest difference is 1.7×10^{-12}, and $dE/da_4 = -253.0160345318$ against the record's -253.0160345309; both checks pass.

In [9], the Fermi-level crossing of $N = 696$.

```
K12 = 0.25 * math.sqrt(12.0)
eps_odd = find_level(0.0, 1, "odd", 0, 0.0, 1.29)  # the k = 0 odd bulk level


def band12(a4):
    return find_level(K12, 1, "even", 0, a4, 1.9 * K12 * math.exp(-a4))
```

`K12` is the momentum $0.25\sqrt{12} = 0.866$ of the shell $n_2 = 12$; `eps_odd` the odd bulk level at $k = 0$ (it does not depend on the slice); `band12` the band level of the shell 12 at a given slice.

```
lo, hi = 0.0, 0.5  # bisection for band12(a4) = eps_odd
for _ in range(45):
    mid = 0.5 * (lo + hi)
    lo, hi = (mid, hi) if band12(mid) > eps_odd else (lo, mid)
crossing = 0.5 * (lo + hi)
dense = np.linspace(0.0, 2.0, 41)
band_curve = [band12(a) for a in dense]
```

Bisection finds the slice at which the falling band level meets the bulk level: while the band level is still above, the crossing lies later. 45 halvings of the interval from 0 to 0.5 locate it far below the printed digits. `band_curve` is the band level at 41 slices for the figure.

```
with open(repository_file("Revision/kohn_sham/results/adiabatic/crossing-demo.csv"),
          newline="", encoding="utf-8") as handle:
    demo = {float(row["a4"]): row for row in csv.DictReader(handle)}
mine = {a4: max(0.0, 8.0 * (eps_odd - band12(a4))) for a4 in SLICES}
worst_demo = max(abs(mine[a4] - float(demo[a4]["difference"])) for a4 in SLICES)
```

The record's demonstration table is read, filed by slice. Without interaction the adiabatically continued state keeps its eight particles in the bulk level, while the instantaneous aufbau state puts them into the lower band level; the energy difference is therefore $8(\varepsilon_{odd} - \varepsilon_{12})$ after the crossing and 0 before it (`max(0.0, ...)`). `worst_demo` compares this with the record's column `difference`.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
left.plot(dense, band_curve, color=PALETTE[0], lw=2.0,
          label="brane band $n_2 = 12$ (32 orbitals)")
left.axhline(eps_odd, color=PALETTE[1], lw=2.0, label="odd bulk level, $k = 0$ (8)")
left.axvline(crossing, color="0.4", ls=":", lw=1.2, label="crossing")
```

```
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("level (units of $m$)")
left.set_title("Two levels near the Fermi level, $N = 696$")
left.legend(fontsize=8)
```

The left panel: the band level of the shell 12 (degeneracy $4r_3(12) = 4 \cdot 8 = 32$), the fixed bulk level (8-fold) as a horizontal line, and the crossing as a dotted vertical line (`axvline`).

```
right.plot(SLICES, [float(demo[a]["difference"]) for a in SLICES], "o", color="black",
           ms=9, label="record")
right.plot(SLICES, [mine[a] for a in SLICES], "x", color=PALETTE[0], ms=10, mew=2,
           label="$8(\\varepsilon_{odd} - \\varepsilon_{12})$, this notebook")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$E_{continued} - E_{aufbau}$ (units of $m$)")
right.set_title("Energy above the instantaneous ground state")
right.legend(fontsize=8)
save_figure(fig, "fermi_crossing",
            "The Fermi-level crossing of the demonstration state $N = 696$ without "
...)
```

The right panel: the recorded energy differences (black dots) and this notebook's (blue crosses) at the five slices; then Figure 15c.5.

```
report("band level n2 = 12 minus the odd level at a4,0 = 0",
       f"{band_curve[0] - eps_odd:.10f}")
report("crossing slice a4,0", f"{crossing:.6f}")
report("largest |8 (eps_odd - eps_12) - recorded difference|", f"{worst_demo:.1e}")
check(0.0 < crossing < 0.5, "the crossing lies between the slices 0 and 0.5",
      record="Revision/kohn_sham/results/adiabatic/crossing-demo.csv, flags at 0 "
             "and 0.5")
check(worst_demo < 1e-9, "the energy differences of the demonstration reproduced",
      record="Revision/kohn_sham/results/adiabatic/crossing-demo.csv, column "
             "difference")
```

Out [9]: at $a_{4,0} = 0$ the band level lies only $0.0032325\,m$ above the bulk level, so the crossing comes almost at once, at $a_{4,0} = 0.002854$; the energy differences agree with the record to 2.0×10^{-11}. Both checks pass.

In [10], the heat map.

```
rows, names = [], []
for n in (8, 136, 688):
    for tag, label in tags:
        rows.append([float(adiabatic[state_id(n, a4, tag)]["Q_max"]) for a4 in SLICES])
        names.append(f"N = {n}, {tag}")
table = np.array(rows)
```

A table of Q_{max} with one row per series (15 rows: three particle numbers times five couplings) and one column per slice, with a name for each row.

```
fig, ax = plt.subplots(figsize=(6.5, 6.0))
image = ax.imshow(table, cmap="Blues", aspect="auto", vmin=0.0, vmax=0.1)
ax.grid(False)  # no grid lines across the coloured squares
ax.set_xticks(range(5), [f"{a:.1f}" for a in SLICES])
ax.set_yticks(range(len(names)), names, fontsize=8)
for i in range(table.shape[0]):
    for jj in range(table.shape[1]):
        ax.text(jj, i, f"{table[i, jj]:.3f}", ha="center", va="center", fontsize=7,
                color="white" if table[i, jj] > 0.06 else "black")
```

`ax.imshow` draws the table as coloured squares, from white (0) to dark blue (0.1); `aspect="auto"` lets the squares fill the drawing area. The ticks are labelled with the slices and the row names, and every square gets its value printed in its middle (`table.shape` is the pair (rows, columns); white letters on the dark squares).

```
ax.set_xlabel("slice $a_{4,0}$")
ax.set_title("$Q_{max}$ of the 75 ground states (record)")
fig.colorbar(image, ax=ax, label="$Q_{max}$")
save_figure(fig, "q_map",
            "Heat map of the largest adiabaticity measure $Q_{max}$ of the 75 ground "
...)
with open(repository_file("Revision/kohn_sham/results/adiabatic/"
                          "fermi-level-crossings.csv"), newline="",
          encoding="utf-8") as handle:
    crossings = list(csv.DictReader(handle))
changed = sum(row["occupied_set_changed"] == "true" for row in crossings)
report("rows of the crossing record / rows with a changed occupied set",
       f"{len(crossings)} / {changed}")
check(len(crossings) == 60 and changed == 0,
      "no Fermi-level crossing in the canonical matrix",
      record="Revision/kohn_sham/results/adiabatic/fermi-level-crossings.csv")
```

A colour scale, Figure 15c.6, and the check that none of the 60 rows of the record's crossing table (15 series, four steps between neighbouring slices) reports a change of the occupied set (Out [10]: 60 / 0).

In [11], the last check.

```
NAMES = ["transition_orbitals", "q_by_pair", "q_history", "hellmann_feynman",
         "fermi_crossing", "q_map"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15c_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

As in Notebook 15e (Section 15.9, In [15]), with the six figure names of this notebook in order and the file names `15c_<number>_<name>.png`: the names whose file does not exist are collected, the check requires that list to be empty, and `all_checks_passed()` prints the last line, ALL 12 CHECKS PASSED (notebook 15c).

15.26 The gas at a temperature

Mermin's functional. At a temperature $T > 0$ the particles no longer fill exactly the lowest levels. Mermin's finite-temperature form of Kohn-Sham theory (Chapter 13) occupies each level with the **Fermi-Dirac** probability

$$f(x) = \frac{1}{1 + e^{x}}, \qquad x = \frac{\varepsilon - \mu}{T},$$

where the **chemical potential** μ is fixed by the particle number, $\sum g f = N$. The function f is 1 far below μ, 0 far above, and $\frac{1}{2}$ at $\varepsilon = \mu$; it falls from nearly 1 to nearly 0 over a few T (Figure 15d.5 shows it). A useful identity: $1 - f(x) = f(-x)$, because $1 - \frac{1}{1+e^{x}} = \frac{e^{x}}{1+e^{x}} = \frac{1}{e^{-x}+1}$. The functions of state of the doubled system are (ks-theory.json, thermodynamics):

$$E = \sum g f \varepsilon - 2\,\mathrm{Vol}_7 \int e^{6Hy} e_{int}\, dy, \qquad S = -\sum g\,[f \ln f + (1 - f)\ln(1 - f)],$$

$$F = E - TS, \qquad \Omega = -T \sum g \ln\left(1 + e^{-(\varepsilon-\mu)/T}\right) - 2\,\mathrm{Vol}_7 \int e^{6Hy} e_{int}\, dy = F - \mu N,$$

the energy, the **entropy** (a measure of how many arrangements of the particles the occupations allow), the **free energy** and the **grand potential**. The last equality is Exercise 5. The self-consistent loop is the one of Section 15.5 with the Fermi-Dirac occupations in place of the aufbau. A note on the record: the theory file writes the particle-number condition as $\sum w g f = N$ with $w = \frac{1}{2}$, which contradicts its own densities, energy and grand potential; the solver uses $\sum g f = N$, the particle number of the doubled system, and records the discrepancy (`Revision/kohn_sham/results/parameters.json`, conventions.particleNumber).

The record's thermal states. 135 states: $N = 8,136,688$, the couplings $0, \pm\lambda_1$, the five slices and $T = 0.01, 0.02, 0.05$ (Section 15.2). Their label sets contain every particle level whose free occupation is above 10^{-12} (the thermal window; check thermo_window_cut), and the solver checks $\sum g f = N$ to 3.1×10^{-12} (check thermo_N_conservation), the two forms of Ω (check thermo_grand_potential_two_forms) and the identities $C_V = T\,dS/dT = dE/dT$ and $-dF/dT = S$ with differences between neighbouring temperatures (checks thermo_CV_identity and thermo_entropy_identity).

The heat capacity in closed form. The **heat capacity** $C_V = dE/dT$ at fixed N says how much energy one unit of temperature costs. Without interaction the levels do not depend on T, and C_V has a closed form. With the shorthand $f_i = f(x_i)$, $d_i = \varepsilon_i - \mu$ and $w_i = g_i f_i (1 - f_i)$, line by line:

$$\frac{df}{dx} = -\frac{e^{x}}{(1 + e^{x})^2} = -f(1 - f).$$

Rule: the derivative of $1/(1 + e^{x})$ by the chain rule; then $\frac{e^{x}}{(1+e^{x})^2} = \frac{1}{1+e^{x}} \cdot \frac{e^{x}}{1+e^{x}} = f\,(1 - f)$.

$$\frac{dx_i}{dT} = -\frac{d_i}{T^2} - \frac{1}{T}\frac{d\mu}{dT}.$$

Rule: $x_i = (\varepsilon_i - \mu)/T$ with fixed ε_i; the quotient rule, and μ changes with T because N is fixed.

$$0 = \frac{dN}{dT} = \sum_i g_i \frac{df}{dx}(x_i)\frac{dx_i}{dT} = -\sum_i w_i \frac{dx_i}{dT}, \qquad \frac{d\mu}{dT} = -\frac{\sum_i w_i d_i}{T\sum_i w_i}.$$

Rule: differentiate $\sum g_i f_i = N$; insert the first line; then insert the second line and solve for $d\mu/dT$.

$$C_V = \frac{dE}{dT} = -\sum_i w_i \varepsilon_i \frac{dx_i}{dT} = -\sum_i w_i d_i \frac{dx_i}{dT} = \frac{1}{T^2}\Big[\sum_i w_i d_i^2 - \frac{(\sum_i w_i d_i)^2}{\sum_i w_i}\Big].$$

Rule: differentiate $E = \sum g_i f_i \varepsilon_i$ in the same way; replacing ε_i by $d_i = \varepsilon_i - \mu$ subtracts $\mu \sum w_i\, dx_i/dT$, which is zero by the previous line; then insert the second line and the value of $d\mu/dT$. In the same way $dN/d\mu = \sum_i g_i \frac{df}{dx}(x_i)\cdot(-\frac{1}{T}) = \frac{1}{T}\sum_i w_i$ at fixed T.

$T\,dS/dT = dE/dT$ and $-dF/dT = S$. The entropy of one level is $-g[f\ln f + (1-f)\ln(1-f)]$; its derivative with respect to f is $-g[\ln f + 1 - \ln(1-f) - 1] = g\ln\frac{1-f}{f} = g\,x$, because $\frac{1-f}{f} = e^x$. So, line by line,

$$\frac{dS}{dT} = \sum_i g_i x_i \frac{df}{dx}(x_i)\frac{dx_i}{dT} = -\sum_i w_i x_i \frac{dx_i}{dT} = -\frac{1}{T}\sum_i w_i d_i \frac{dx_i}{dT} = \frac{1}{T}\frac{dE}{dT}.$$

Rule: the chain rule with the derivative just found; $df/dx = -f(1-f)$; $x_i = d_i/T$; and the previous derivation of dE/dT. Hence $C_V = T\,dS/dT$, and $\frac{dF}{dT} = \frac{dE}{dT} - S - T\frac{dS}{dT} = -S$ (the product rule on TS). The solver reports C_V as $T\,dS/dT$, because the entropy is computed from the occupations and is well conditioned, while dE/dT is a difference of nearly equal energies.

The two-level formula for $N = 8$. At $a_{4,0} = 0$ and low temperature only two groups of levels matter for $N = 8$: the eight zero modes at $\varepsilon = 0$ and the 24 orbitals of the first band shell at $\varepsilon = \Delta = 0.4307$ (the gap). The particles that leave the zero modes must enter the band, line by line:

$$8\,(1 - f(-\mu/T)) = 24\,f((\Delta-\mu)/T).$$

Rule: the holes in the zero modes equal the particles in the band, because N is fixed.

$$8\,e^{-\mu/T} = 24\,e^{-(\Delta-\mu)/T}.$$

Rule: $1 - f(x) = f(-x)$, and $f(x) \approx e^{-x}$ for $x \gg 1$; here μ/T and $(\Delta-\mu)/T$ are about 21 and 22.

$$\ln 8 - \frac{\mu}{T} = \ln 24 - \frac{\Delta-\mu}{T}, \qquad \mu = \frac{\Delta}{2} - \frac{T}{2}\ln 3.$$

Rule: take the logarithm of both sides; collect μ on one side ($-2\mu/T = \ln 3 - \Delta/T$, since $\ln 24 - \ln 8 = \ln 3$) and divide by $-2/T$. At $T \to 0$ the chemical potential sits in the middle of the gap; it falls linearly with the slope $-\frac{1}{2}\ln 3$, the logarithm of the ratio of the degeneracies. At $T = 0.01$ the formula gives 0.209873777875, the record 0.209873776387 (Notebook 15d, Out [5]); the difference 1.5×10^{-9} is the size of the neglected terms.

Why μ must be computed with care. Near a closed shell at low temperature almost every level is either full or empty. The direct count $\sum gf - N$ then adds numbers very close to 1 and subtracts

N; in double precision (the computer's numbers, about 16 significant digits) the result is only known to about ϵN, where $\epsilon = 2^{-52} = 2.2 \times 10^{-16}$ is the rounding unit, while the count changes with μ only at the rate $dN/d\mu$. A root of the direct count is therefore uncertain by about $\epsilon N/(dN/d\mu)$. For $N = 8$ at $a_{4,0} = 0$, $T = 0.01$ the record has $dN/d\mu = 1.23 \times 10^{-6}$ (`Revision/kohn_sham/results/thermo/thermodynamics.csv`), so this uncertainty is $2.2 \times 10^{-16} \cdot 8/(1.23 \times 10^{-6}) = 1.4 \times 10^{-9}\, m$, a million times worse than the rounding unit. An earlier version of the solver used the direct count, and the independent cross-check of Chapter 16 found its μ off by $8.3 \times 10^{-10}\, m$ in the state `N8_lamm1_a00_T10` (`Revision/kohn_sham/solver/README.md`, History).

The balance form, line by line. Let $\mathcal{S}$ be the set of the levels that are filled at $T = 0$ and $d = N - \sum_{\mathcal{S}} g$ (zero for a closed shell). Then

$$\sum_i g_i f(x_i) - N = \sum_{i \notin \mathcal{S}} g_i f(x_i) + \sum_{i \in \mathcal{S}} g_i (1 - f(-x_i)) - N = P - H_l - d,$$

with the thermal particles above the filled levels $P = \sum_{i \notin \mathcal{S}} g_i f(x_i)$ and the thermal holes below them $H_l = \sum_{i \in \mathcal{S}} g_i f(-x_i)$. Rule: split the sum into the levels outside and inside $\mathcal{S}$; inside use $f(x) = 1 - f(-x)$; then $\sum_{\mathcal{S}} g_i - N = -d$. The condition $\sum gf = N$ becomes $P = H_l + d$, and for a closed shell simply $\ln P = \ln H_l$: the thermal particles balance the thermal holes. No numbers close to 1 are added and no large number is subtracted; each hole factor is computed as $f(-x)$ directly, never as $1 - f$. The solver solves $\ln(P + d_-) - \ln(H_l + d_+) = 0$ with $d_\pm = \max(\pm d, 0)$, adding each sum in the form $\ln \sum e^{t_i} = t_{max} + \ln \sum e^{t_i - t_{max}}$ (the **log-sum-exp**, which never overflows or underflows), by Newton's method in a bracket and then bisection down to neighbouring double-precision numbers (`Revision/kohn_sham/solver/src/mermin.rs`). The record checks every one of the 135 roots against a root computed with 40 significant digits by mpmath: all lie within their rounding bound, the largest difference is $2.4 \times 10^{-16}\, m$ (`Revision/kohn_sham/reports/ks-rust-mermin-roots.json`, check solver_mu_within_rounding_bound).

The sea holes: where the filling convention stops being reasonable. The CONVENTION of Section 15.2 never occupies the negative branch, not even thermally. How good is that? Without interaction the sea's brane band (block type $j = -1$, even parity, label 0) has the levels $-\varepsilon_{band}(n_2)$ (Section 15.4). If the sea were treated thermally, each of its orbitals would be empty with the probability $1 - f((-\varepsilon_{band} - \mu)/T) = f((\varepsilon_{band} + \mu)/T) = 1/(1 + e^{(\mu + \varepsilon_{band})/T})$; such an empty sea orbital is a **thermal hole**, which a full quantum treatment would count as an antiparticle of the same universe. The record computes the number of these holes for every thermal state as a DIAGNOSTIC (check thermo_sea_hole_diagnostic_computed): it exceeds 1 per cent of N in 15 of the 135 states, all at $a_{4,0} \geq 1$, and reaches $30.98\, N$ in `N8_lamm1_a20_T50` ($N = 8$, $-\lambda_1$, $a_{4,0} = 2$, $T = 0.05$). There the band has been squeezed so close to $\varepsilon = 0$ that the temperature excites the sea as easily as the particles, μ itself is negative (-0.0977), and the particle-only ensemble is outside its range of validity; a thermal treatment of the sea is OPEN. These thermal holes are excitations inside one universe; they say nothing about pairs of universes or about the matter-antimatter asymmetry.

15.27 Example: thermodynamics (Notebook 15d)

Notebook 15d runs the solver's command `single` for 20 free thermal states ($N = 8$ and 136) and reads their final levels exactly. From these levels alone it recomputes, in plain Python, the chemical potential (with mpmath at 40 digits), E, S, F, Ω, C_V and $dN/d\mu$, and compares them with the record. It shows why the direct count is too coarse (Figure 15d.1), draws $\mu(T)$ with

the two-level formula (Figure 15d.2), F and S along the history (Figure 15d.3), the heat capacity (Figure 15d.4), the occupations (Figure 15d.5), the sea-hole diagnostic (Figure 15d.6) and the effect of the interaction on F (Figure 15d.7). It needs Rust, takes about half a minute, and ends with ALL 15 CHECKS PASSED (notebook 15d).

What to look for: in Figure 15d.1 the orange staircase of the double-precision count against the smooth blue line of the exact count; in Figure 15d.2 the straight start of $\mu(T)$ from the middle of the gap; in Figure 15d.4 how the heat capacity of $N = 8$ wakes up at ever lower temperatures as the history closes the gap; in Figure 15d.6 the values above the 1 per cent line late in the history.

15.28 How to run Notebook 15d

Step 1. What this notebook does and what it needs.

Notebook 15d (Thermodynamics of the Kohn-Sham gas along the deflating history) is the file `Revision/textbook/notebooks/15d_thermodynamics.ipynb` of the repository Dirac_claude. It builds the Rust Kohn-Sham solver with cargo (a full build of about a minute when the program is missing, a second when it is up to date), runs it for the free thermal states N = 8 and N = 136 of dirac16complex at the slices of the deflating history, recomputes from the solver's levels the chemical potential (at 40 digits and with the well-conditioned double-precision balance), the energy, entropy, free energy, grand potential, heat capacity and the sea-hole diagnostic of the committed Revision record, and draws the chemical potential, free energy, entropy and heat capacity as functions of the temperature. The solver writes its output files (about 0.6 MB) into the folder `Revision/kohn_sham/solver/target/textbook_15d`, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
```

```
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/kohn_sham/solver/target/release/revision_ks_solver` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 15d_thermodynamics.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below.

To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 15d_thermodynamics.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 15d_thermodynamics.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/15d.captions.json`
- `Revision/textbook/figures/15d_1_root_conditioning.png`
- `Revision/textbook/figures/15d_2_chemical_potential.png`
- `Revision/textbook/figures/15d_3_free_energy_entropy.png`
- `Revision/textbook/figures/15d_4_heat_capacity.png`
- `Revision/textbook/figures/15d_5_occupations.png`
- `Revision/textbook/figures/15d_6_sea_holes.png`
- `Revision/textbook/figures/15d_7_interaction_free_energy.png`

It changes no other file of the repository except the Rust build folder `target` next to each `Cargo.toml` it builds (inside it the notebook keeps the raw output of its program runs in `Revision/kohn_sham/solver/target/textbook_15d`, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 15 CHECKS PASSED (notebook 15d)
```

and the notebook must show 7 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/15d_thermodynamics.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 15d_thermodynamics.ipynb
python -m nbconvert --execute --inplace 15d_thermodynamics.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- You want the disk space of the solver output back: the folder `Revision/kohn_sham/solver/target/textbook_15d` holds only the raw output of the last run (40 files, about 0.6 MB), which git ignores; delete it at any time, the notebook writes it again.

15.29 Notebook 15d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/15d_thermodynamics.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

At a temperature $T > 0$ the particles of the Kohn-Sham gas no longer fill exactly
the lowest levels: each level is occupied with the **Fermi-Dirac** probability
$f = 1/(1 + e^{(\varepsilon - \mu)/T})$, where the **chemical potential** $\mu$ is
fixed by the particle number. This is Mermin's finite-temperature version of
Kohn-Sham theory. The Revision record holds 135 such states (three particle numbers,
three couplings, five slices of the history, three temperatures). This notebook

- runs the Rust solver for the free states ($\lambda = 0$) of $N = 8$ and $N = 136$
  and reads its levels;
- recomputes from these levels, in plain Python, $\mu$ (at 40 digits with mpmath),
  the energy $E$, the entropy $S$, the free energy $F = E - TS$, the grand
  potential $\Omega$, the heat capacity $C_V$ and $dN/d\mu$, and compares all of
  them with the record;
- explains why the solver computes $\mu$ from a **balance of particles and holes**
  instead of the direct count, and shows the difference;
- draws $\mu$, $F$, $S$ and $C_V$ as continuous functions of $T$, the occupations at
  three temperatures, the sea-hole diagnostic of the filling convention, and the
  effect of the interaction on $F$.

It draws seven figures and takes about half a minute.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**
```

Notebook 15d (Thermodynamics of the Kohn-Sham gas along the deflating history) is the file ‘Revision/textbook/notebooks/15d_thermodynamics.ipynb‘ of the repository Dirac_claude. It builds the Rust Kohn-Sham solver with cargo (a full build of about a minute when the program is missing, a second when it is up to date), runs it for the free thermal states N = 8 and N = 136 of dirac16complex at the slices of the deflating history, recomputes from the solver’s levels the chemical potential (at 40 digits and with the well-conditioned double-precision balance), the energy, entropy, free energy, grand potential, heat capacity and the sea-hole diagnostic of the committed Revision record, and draws the chemical potential, free energy, entropy and heat capacity as functions of the temperature. The solver writes its output files (about 0.6 MB) into the folder ‘Revision/kohn_sham/solver/target/textbook_15d‘, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘ again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
    cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/kohn_sham/solver/target/release/revision_ks_solver‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 15d_thermodynamics.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 15d_thermodynamics.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 15d_thermodynamics.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/15d.captions.json‘
- ‘Revision/textbook/figures/15d_1_root_conditioning.png‘

- ‘Revision/textbook/figures/15d_2_chemical_potential.png‘
- ‘Revision/textbook/figures/15d_3_free_energy_entropy.png‘
- ‘Revision/textbook/figures/15d_4_heat_capacity.png‘
- ‘Revision/textbook/figures/15d_5_occupations.png‘
- ‘Revision/textbook/figures/15d_6_sea_holes.png‘
- ‘Revision/textbook/figures/15d_7_interaction_free_energy.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds (inside it the notebook keeps the raw output of its program runs in ‘Revision/kohn_sham/solver/target/textbook_15d‘, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 15 CHECKS PASSED (notebook 15d)
```

and the notebook must show 7 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/15d_thermodynamics.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command

below does what ‘jupyter lab‘ does, the second what the headless command does:

```
    python -m jupyterlab 15d_thermodynamics.ipynb
    python -m nbconvert --execute --inplace 15d_thermodynamics.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- You want the disk space of the solver output back: the folder ‘Revision/kohn_sham/solver/target/textbook_15d‘ holds only the raw output of the last run (40 files, about 0.6 MB), which git ignores; delete it at any time, the notebook writes it again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 15d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 15d (Thermodynamics of the Kohn-Sham gas along the deflating history) is the
# file Revision/textbook/notebooks/15d_thermodynamics.ipynb of the repository
# Dirac_claude. It builds the Rust Kohn-Sham solver with cargo (a full build of about a
# minute when the program is missing, a second when it is up to date), runs it for the
# free thermal states N = 8 and N = 136 of dirac16complex at the slices of the deflating
# history, recomputes from the solver's levels the chemical potential (at 40 digits and
# with the well-conditioned double-precision balance), the energy, entropy, free energy,
# grand potential, heat capacity and the sea-hole diagnostic of the committed Revision
# record, and draws the chemical potential, free energy, entropy and heat capacity as
# functions of the temperature. The solver writes its output files (about 0.6 MB) into
# the folder Revision/kohn_sham/solver/target/textbook_15d, which git ignores. It needs a
# computer with Windows 11, macOS or Linux, an internet connection for the installation,
# the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
# with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
# matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
# and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the
```

```
# other packages run Jupyter, the program that shows and runs notebooks. It also needs
# Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program
# revision_ks_solver, which is part of the repository and is built on your computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
```

```
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
```

```
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
#
# Windows, macOS and Linux:
#     cargo --version
#
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
#     cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
#
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
# Revision/kohn_sham/solver/target/release/revision_ks_solver (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 15d_thermodynamics.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
```

```
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 15d_thermodynamics.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 15d_thermodynamics.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/15d.captions.json
# - Revision/textbook/figures/15d_1_root_conditioning.png
# - Revision/textbook/figures/15d_2_chemical_potential.png
# - Revision/textbook/figures/15d_3_free_energy_entropy.png
# - Revision/textbook/figures/15d_4_heat_capacity.png
# - Revision/textbook/figures/15d_5_occupations.png
# - Revision/textbook/figures/15d_6_sea_holes.png
# - Revision/textbook/figures/15d_7_interaction_free_energy.png
#
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds (inside it the notebook keeps the raw output of its program
# runs in Revision/kohn_sham/solver/target/textbook_15d, which git ignores); running it
# headless or saving it in JupyterLab also rewrites the notebook file itself. It does not
# use the internet while it runs. The files it writes are the same files that are stored
# in the repository (on another computer a figure may differ in a few bytes, which is
# harmless). To get the stored versions back, run this command in the repository folder
# (it also undoes every change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 15 CHECKS PASSED (notebook 15d)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
```

```
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/15d_thermodynamics.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 15d_thermodynamics.ipynb
#     python -m nbconvert --execute --inplace 15d_thermodynamics.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - You want the disk space of the solver output back: the folder
#   Revision/kohn_sham/solver/target/textbook_15d holds only the raw output of the last
#   run (40 files, about 0.6 MB), which git ignores; delete it at any time, the notebook
#   writes it again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "15d"  # this notebook: chapter 15, example d


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
```

```
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 15d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Temperature** $T$: measured in units of the mass $m$ (Boltzmann's constant is
  1).
- **Fermi-Dirac occupation** $f(x) = 1/(1 + e^{x})$ with $x = (\varepsilon - \mu)/T$:
  the probability that a level of energy $\varepsilon$ is occupied. It is 1 far below
  $\mu$, 0 far above, and $1/2$ at $\varepsilon = \mu$. A useful identity:
  $1 - f(x) = f(-x)$.
- **Chemical potential** $\mu$: the number that makes the occupations add up to the
  particle number, $\sum g f = N$ ($g$ the degeneracy of a level).
- **Entropy** $S = -\sum g[f\ln f + (1 - f)\ln(1 - f)]$: a measure of how many
  microscopic arrangements the gas can take.
- **Free energy** $F = E - TS$ and **grand potential** $\Omega = F - \mu N$.
- **Heat capacity** $C_V = dE/dT = T\,dS/dT$ at fixed $N$: how much energy one
  unit of temperature costs.
- **Activated**: a gas whose lowest empty level lies a gap $\Delta$ above the
  highest occupied one; at $T \ll \Delta$ the thermal excitations are suppressed like
```

```
  $e^{-\Delta/(2T)}$.
- **Hole**: an empty place in a level that is normally occupied.
- **Sea**: the levels of the negative branch, which the filling CONVENTION of the
  record leaves out (normal ordering); **sea holes** would be thermal antiparticles.
- **Double precision**: the computer's usual numbers, with about 16 significant
  digits; **40 digits**: mpmath's numbers with as many digits as asked for.
- **Conditioning**: how strongly a result reacts to small errors of the input.
```

Text cell:

```
## 4. The physical and mathematical situation

The Kohn-Sham states of dirac16complex live in the author's metric, with 3-space
$x_1, x_2, x_3$ inflating like $e^{a_4}$, the three extra times $x_5, x_6, x_7$
deflating like $e^{-a_4}$, the time $x_4$ and the hidden direction $x_0$ (coordinate
$y$). The history $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND, and each thermal state
is an instantaneous state at one slice $a_{4,0}$.

**Mermin's functional** (Revision record ks-theory.json, thermodynamics): with the
levels $\varepsilon_i$ (degeneracy $g_i$) of the Kohn-Sham Hamiltonian,

$$\sum_i g_i f_i = N, \quad E = \sum_i g_i f_i\varepsilon_i -
2\,\mathrm{Vol}_7\!\int e^{6Hy}e_{int}\,dy, \quad
S = -\sum_i g_i[f_i\ln f_i + (1 - f_i)\ln(1 - f_i)],$$
$$F = E - TS, \qquad \Omega = -T\sum_i g_i\ln(1 + e^{-(\varepsilon_i - \mu)/T})
- 2\,\mathrm{Vol}_7\!\int e^{6Hy}e_{int}\,dy = F - \mu N .$$

Without interaction ($\lambda = 0$) the levels do not depend on $T$ and $e_{int} = 0$,
so everything follows from the levels. Then the heat capacity has a closed form:
differentiating $\sum g f = N$ at fixed $N$ gives $d\mu/dT$, and inserting it into
$dE/dT$ gives, with $w_i = g_i f_i(1 - f_i)$ and $d_i = \varepsilon_i - \mu$,

$$C_V = \frac{1}{T^2}\left[\sum_i w_i d_i^2 - \frac{(\sum_i w_i d_i)^2}{\sum_i
w_i}\right], \qquad \frac{dN}{d\mu} = \frac{1}{T}\sum_i w_i .$$

(Line by line: $df/dx = -f(1-f)$; $dx_i/dT = -d_i/T^2 - (d\mu/dT)/T$; fixed $N$ means
$\sum w_i\,dx_i/dT = 0$, so $d\mu/dT = -\sum w_i d_i/(T\sum w_i)$; then
$dE/dT = -\sum w_i\varepsilon_i\,dx_i/dT$, and replacing $\varepsilon_i$ by $d_i$
changes nothing because $\sum w_i\,dx_i/dT = 0$.)

**The filling convention** (a CONVENTION whose justification is OPEN): particles fill
only the positive branch and the zero modes; the negative branch (the sea) is never
occupied thermally. The record measures how good this is: the **sea holes** that the
sea's brane band would carry at the same $\mu$ and $T$.

**Parameters**: $H = m = 1$, $L = 3$, $\Delta k = 0.25$, $T = 0.01, 0.02, 0.05$; $N$
counts the doubled system (universe and Z2 image, the brane being ASSUMED).
```

Text cell:

```
## 5. Building the solver and computing the free thermal states
```

The next cell builds the solver and defines ‘thermal_state‘, which runs ‘revision_ks_solver single‘ with ‘--T‘ (the temperature) and ‘--margin 0.2‘ (the label set of the canonical matrix, so that the state is exactly the recorded one), and with ‘--mermin-levels‘, which writes the final levels as decimal numbers that read back into exactly the solver’s numbers. It returns the solver’s μ, its levels with degeneracies, and the full level list (shell, block type, parity, label, energy, degeneracy, occupation). The cell then computes the 18 free states $N = 8, 136$, $a_{4,0} = 0, 1, 2$, $T = 0.01, 0.02, 0.05$, and two more for $N = 136$ at $a_{4,0} = 0.5$ and 1.5 ($T = 0.05$), and compares each μ with the record.

In [2]:

```
import csv  # reads the tables (CSV files)
import math  # exp, log for single numbers

import mpmath  # numbers with 40 digits
import numpy as np  # arrays of numbers

program = rust_program("Revision/kohn_sham/solver/Cargo.toml", "revision_ks_solver")
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_15d"  # git ignores it
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
SHADES = {0.0: "#86b6ef", 0.5: "#5598e7", 1.0: "#2a78d6", 1.5: "#1c5cab",
          2.0: "#104281"}  # one shade of blue per slice, light to dark
TEMPS = (0.01, 0.02, 0.05)


def state_id(n, a4, t, tag="lam0"):
    return f"N{n}_{tag}_a{round(10 * a4):02d}_T{round(1000 * t)}"


def thermal_state(n, a4, t):
    """Run the solver for one free thermal state; (mu, eps, deg, levels)."""
    name = state_id(n, a4, t)
    out, lev = RUN_FOLDER / f"{name}.json", RUN_FOLDER / f"{name}-levels.json"
    done = subprocess.run(
        [str(program), "single", "--root", str(REPO), "--m", "1", "--lambda", "0",
         "--a4", repr(a4), "--N", repr(float(n)), "--T", repr(t), "--margin", "0.2",
         "--out", str(out), "--mermin-levels", str(lev)],
        capture_output=True, text=True)
    if done.returncode != 0:
        raise RuntimeError(f"the solver failed for {name}: {done.stderr[-500:]}")
    exact = json.loads(lev.read_text(encoding="utf-8"))
    full = json.loads(out.read_text(encoding="utf-8"))
    eps = [float(e) for e, _ in exact["levels_eps_deg"]]  # exactly the solver’s
    deg = [float(g) for _, g in exact["levels_eps_deg"]]
    return float(exact["mu"]), eps, deg, full["levels_n2_j_parity_label_eps_deg_f"]


THERMO = "Revision/kohn_sham/results/thermo/thermodynamics.csv"
with open(repository_file(THERMO), newline="", encoding="utf-8") as handle:
    record = {row["id"]: row for row in csv.DictReader(handle)}
```

```
states = {}
for n in (8, 136):
    for a4 in (0.0, 1.0, 2.0):
        for t in TEMPS:
            states[(n, a4, t)] = thermal_state(n, a4, t)
for a4 in (0.5, 1.5):
    states[(136, a4, 0.05)] = thermal_state(136, a4, 0.05)
worst_mu = max(abs(mu - float(record[state_id(n, a4, t)]["mu"]))
               for (n, a4, t), (mu, _, _, _) in states.items())
report("free thermal states computed", len(states))
report("largest |mu(solver now) - mu(record)|", f"{worst_mu:.1e}")
check(worst_mu < 1e-12, f"the solver reproduces the recorded mu of {len(states)} states",
      record=f"{THERMO}, column mu")
```

Out [2]:

```
Rust program revision_ks_solver is built and ready.
RESULT free thermal states computed = 20
RESULT largest |mu(solver now) - mu(record)| = 5.6e-17
PASS the solver reproduces the recorded mu of 20 states
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, column mu
```

Text cell:

```
## 6. The chemical potential at 40 digits, and the other functions of state

The next cell solves $\sum g f = N$ again with mpmath at 40 significant digits
(Newton's method started at the solver's $\mu$), so that rounding plays no role at
all. The solver's $\mu$ must lie within its own **rounding bound** (column
`mu_rounding_bound` of the record) of this exact root. Then it computes $E$, $S$,
$F$, $\Omega$, $C_V$ (closed form) and $dN/d\mu$ in double precision at the solver's
$\mu$ and compares them with the record. The record's $C_V$ is a numerical
derivative $T\,dS/dT$ (Richardson differences), so the comparison allows
$10^{-4}$; the others agree to $10^{-12}$.
```

In [3]:

```
def exact_root(eps, deg, n, t, guess):
    """The root of sum g f = N with 40 significant digits (Newton's method)."""
    with mpmath.workdps(40):
        levels = [(mpmath.mpf(e), mpmath.mpf(g)) for e, g in zip(eps, deg)]
        temp, mu = mpmath.mpf(t), mpmath.mpf(guess)
        for _ in range(60):
            occ = [(g, 1 / (1 + mpmath.exp((e - mu) / temp))) for e, g in levels]
            count = sum(g * f for g, f in occ) - n
            slope = sum(g * f * (1 - f) for g, f in occ) / temp  # dN/dmu
            step = count / slope
            mu -= step
            if abs(step) < mpmath.mpf(10) ** -36:
                break
        return mu
```

```
def functions_of_state(eps, deg, mu, t):
    """E, S, F, Omega, C_V, dN/dmu of a free state (double precision, stable forms)."""
    e, g = np.array(eps), np.array(deg)
    x = (e - mu) / t
    small = np.exp(-np.abs(x))  # e^{-|x|}, never overflows
    f_abs = small / (1.0 + small)  # f(|x|)
    f = np.where(x > 0, f_abs, 1.0 - f_abs)  # f(x)
    energy = float(np.sum(g * f * e))
    # -[f ln f + (1-f) ln(1-f)] = ln(1 + e^{-|x|}) + |x| f(|x|)
    entropy = float(np.sum(g * (np.log1p(small) + np.abs(x) * f_abs)))
    # ln(1 + e^{-x}) = max(-x, 0) + ln(1 + e^{-|x|})
    omega = -t * float(np.sum(g * (np.maximum(-x, 0.0) + np.log1p(small))))
    w = g * f_abs * (1.0 - f_abs)  # g f (1 - f), the same for x and -x
    d = e - mu
    cv = float((np.sum(w * d * d) - np.sum(w * d) ** 2 / np.sum(w)) / t ** 2)
    return energy, entropy, energy - t * entropy, omega, cv, float(np.sum(w) / t)


rel = lambda a, b: abs(a - b) / max(abs(b), 1.0)  # relative to max(|b|, 1)
worst = {"root": 0.0, "E S F Omega": 0.0, "C_V": 0.0, "dN/dmu": 0.0}
for (n, a4, t), (mu, eps, deg, _) in states.items():
    row = record[state_id(n, a4, t)]
    root = exact_root(eps, deg, n, t, mu)
    worst["root"] = max(worst["root"],
                        float(abs(mu - root)) / float(row["mu_rounding_bound"]))
    e_, s_, f_, o_, cv, dn = functions_of_state(eps, deg, mu, t)
    worst["E S F Omega"] = max(worst["E S F Omega"], rel(e_, float(row["E"])),
                               rel(s_, float(row["entropy"])), rel(f_, float(row["F"])),
                               rel(o_, float(row["Omega_direct"])))
    worst["C_V"] = max(worst["C_V"], abs(cv / float(row["C_V"]) - 1.0))
    worst["dN/dmu"] = max(worst["dN/dmu"], abs(dn / float(row["dN_dmu"]) - 1.0))
report("largest |mu - 40-digit root| / recorded rounding bound", f"{worst['root']:.2f}")
for key in ("E S F Omega", "C_V", "dN/dmu"):
    report(f"largest relative difference, {key}", f"{worst[key]:.1e}")
check(worst["root"] <= 1.0, "mu lies within its rounding bound of the 40-digit root",
      record="Revision/kohn_sham/reports/ks-rust-mermin-roots.json and "
             f"{THERMO}, column mu_rounding_bound")
check(worst["E S F Omega"] < 1e-12,
      f"E, S, F and Omega of {len(states)} states reproduced",
      record=f"{THERMO}, columns E, entropy, F, Omega_direct")
check(worst["C_V"] < 1e-4, "the closed-form C_V reproduces the recorded T dS/dT",
      record=f"{THERMO}, column C_V")
check(worst["dN/dmu"] < 1e-9, "dN/dmu reproduced", record=f"{THERMO}, column dN_dmu")
```

Out [3]:

```
RESULT largest |mu - 40-digit root| / recorded rounding bound = 0.07
RESULT largest relative difference, E S F Omega = 3.6e-14
RESULT largest relative difference, C_V = 1.6e-05
RESULT largest relative difference, dN/dmu = 2.0e-15
PASS mu lies within its rounding bound of the 40-digit root
```

```
     reproduces Revision/kohn_sham/reports/ks-rust-mermin-roots.json and
    Revision/kohn_sham/results/thermo/thermodynamics.csv, column mu_rounding_bound
PASS E, S, F and Omega of 20 states reproduced
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, columns E, entropy,
    F, Omega_direct
PASS the closed-form C_V reproduces the recorded T dS/dT
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, column C_V
PASS dN/dmu reproduced
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, column dN_dmu
```

Text cell:

```
## 7. Why the solver uses a balance instead of the direct count

Near a closed shell at low temperature almost every level is either full ($f
\approx 1$) or empty ($f \approx 0$). The direct count $\sum g f - N$ then adds
numbers close to 1 and subtracts $N$: in double precision the result is a multiple
of the rounding step of $N$ (about $10^{-15}$), while the true value changes with
$\mu$ only at the rate $dN/d\mu$, which for $N = 8$ at $a_{4,0} = 0$,
$T = 0.01$ is about $1.2 \times 10^{-6}$. So the direct count cannot locate $\mu$
better than about $10^{-15}/(1.2 \times 10^{-6})$, a few times $10^{-10}$ to
$10^{-9}$ (the record found an error of $8.3 \times 10^{-10}$ in the solver's
earlier version, which used the direct count). The solver
instead balances the thermal **particles** above the filled levels against the
thermal **holes** below them, $P = \sum_{above} g f(x)$ and $H_l = \sum_{below} g
f(-x)$ (each hole factor computed as $f(-x)$, never as $1 - f$), and solves
$\ln P = \ln H_l$; no large numbers are subtracted. The next cell computes the
direct count in double precision and the exact count at 40 digits at 61 values of
$\mu$ within $\pm 3 \times 10^{-9}$ of the root, and finds the root of the balance
in double precision by bisection down to neighbouring numbers.
```

In [4]:

```
def balance_root(eps, deg, n, t):
    """mu from ln P = ln H (particles above = holes below the T = 0 filling)."""
    order = np.argsort(eps, kind="stable")
    e, g = np.array(eps)[order], np.array(deg)[order]
    filled = int(np.searchsorted(np.cumsum(g), n - 1e-9)) + 1  # T = 0 filling
    below_e, below_g, above_e, above_g = e[:filled], g[:filled], e[filled:], g[filled:]

    def log_sum(log_terms):  # ln(sum e^{t}) without overflow or underflow
        top = float(np.max(log_terms))
        return top + math.log(float(np.sum(np.exp(log_terms - top))))

    def log_f(x):  # ln f(x) = -ln(1 + e^{x}), stable for every x
        return -(np.maximum(x, 0.0) + np.log1p(np.exp(-np.abs(x))))

    def balance(mu):
        return (log_sum(np.log(above_g) + log_f((above_e - mu) / t))
                - log_sum(np.log(below_g) + log_f(-(below_e - mu) / t)))

    lo, hi = float(e[0]) - 1.0, float(e[-1]) + 1.0
```

```
    while lo < 0.5 * (lo + hi) < hi:  # bisection down to neighbouring numbers
        mid = 0.5 * (lo + hi)
        lo, hi = (mid, hi) if balance(mid) < 0.0 else (lo, mid)
    return lo


mu8, eps8, deg8, _ = states[(8, 0.0, 0.01)]
root8 = exact_root(eps8, deg8, 8, 0.01, mu8)
offsets = np.linspace(-3e-9, 3e-9, 61)
direct, exact = [], []
for off in offsets:
    x = (np.array(eps8) - (mu8 + off)) / 0.01
    direct.append(float(np.sum(np.array(deg8) / (1.0 + np.exp(x)))) - 8.0)
    with mpmath.workdps(40):
        m_ = mpmath.mpf(mu8) + mpmath.mpf(off)
        exact.append(float(sum(mpmath.mpf(g) / (1 + mpmath.exp((mpmath.mpf(e) - m_)
                                                               / mpmath.mpf(0.01)))
                               for e, g in zip(eps8, deg8)) - 8))
fig, ax = plt.subplots()
ax.plot(offsets * 1e9, direct, "s-", color=PALETTE[1], ms=4, lw=1.2,
        label="direct count, double precision")
ax.plot(offsets * 1e9, exact, color=PALETTE[0], lw=2.0, label="exact count, 40 digits")
ax.axvline(float(root8 - mu8) * 1e9, color="0.3", ls=":", lw=1.2, label="exact root")
ax.axhline(0.0, color="0.6", lw=0.8)
ax.set_xlabel("$\\mu$ minus the solver's $\\mu$ (units of $10^{-9}\\,m$)")
ax.set_ylabel("$\\sum g f - N$")
ax.set_title("$N = 8$, $a_{4,0} = 0$, $T = 0.01$: the direct count is too coarse")
ax.legend(fontsize=8)
zero_at = [off for off, value in zip(offsets, direct) if value == 0.0]  # direct = 0
if zero_at:  # where the double-precision count says "exactly N particles"
    miss = min(abs(off - float(root8 - mu8)) for off in zero_at)  # nearest such point
    where = ("is zero only on a short interval that misses the true root by at "
             f"least ${miss * 1e10:.0f} \\times 10^{{-10}}\\,m$")
    report("distance of the zeros of the direct count from the true root",
           f"at least {miss:.1e} m")
else:  # (on another computer the rounding steps may fall differently)
    where = "is never exactly zero at the sampled points"
save_figure(fig, "root_conditioning",
            "The particle-number condition $\\sum g f - N$ (vertical axis, of size "
            "$10^{-15}$) against $\\mu$ near its root (horizontal axis, in units of "
            "$10^{-9}\\,m$) for the activated state $N = 8$, $a_{4,0} = 0$, "
            "$T = 0.01$: computed directly in double precision it moves in steps of "
            f"the rounding unit of $N$ (orange squares) and {where}, while the exact "
            "count (blue line, 40 digits) crosses zero at one point, the true root, "
            "which the balance of particles and holes finds.")
worst_balance = 0.0
for (n, a4, t), (mu, eps, deg, _) in states.items():
    root = exact_root(eps, deg, n, t, mu)
    worst_balance = max(worst_balance, float(abs(balance_root(eps, deg, n, t) - root)))
zero_width = (np.sum(np.array(direct) == 0.0)) * (offsets[1] - offsets[0])
report("width of the interval where the double-precision direct count is 0",
       f"{zero_width:.1e} m")
report(f"largest |balance root - 40-digit root| ({len(states)} states)",
       f"{worst_balance:.1e}")
```

```
check(worst_balance < 1e-15, "the double-precision balance finds mu to 1e-15",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "thermo_mu_well_conditioned_root")
```

Out [4]:

```
RESULT distance of the zeros of the direct count from the true root = at least 3.0e-10 m
```

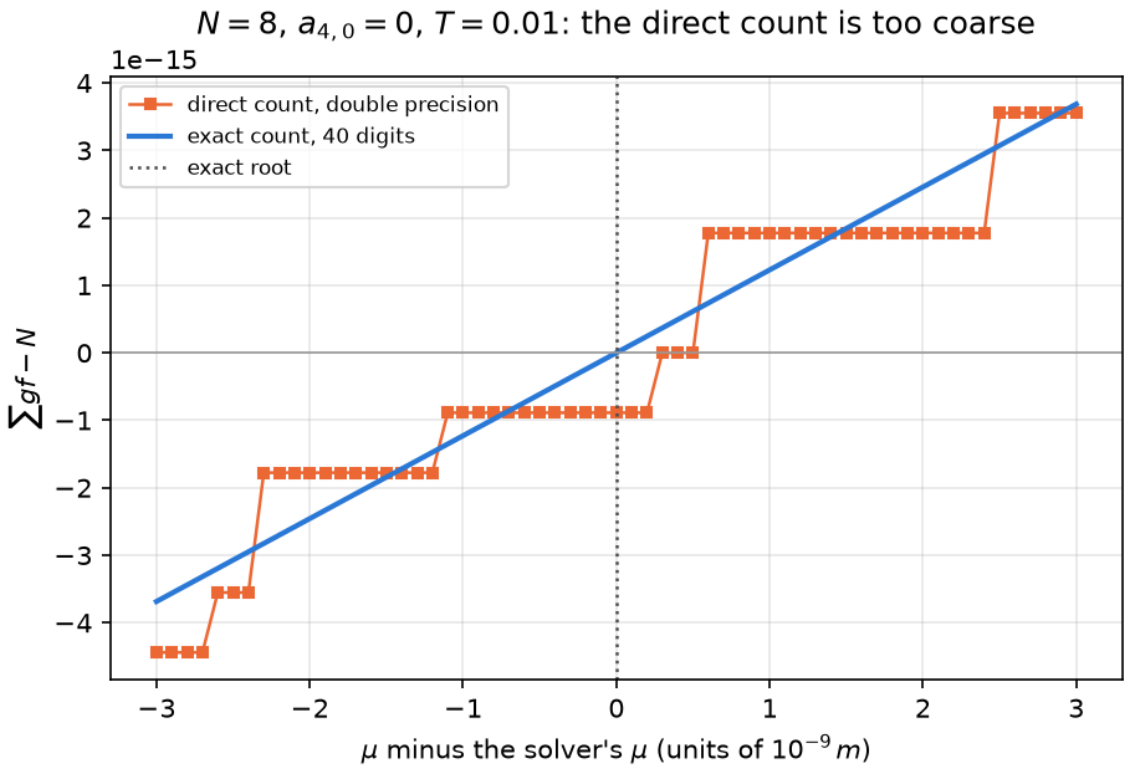

Figure 432: The particle-number condition $\sum gf - N$ (vertical axis, of size 10^{-15}) against μ near its root (horizontal axis, in units of $10^{-9}\,m$) for the activated state $N = 8$, $a_{4,0} = 0$, $T = 0.01$: computed directly in double precision it moves in steps of the rounding unit of N (orange squares) and is zero only on a short interval that misses the true root by at least $3 \times 10^{-10}\,m$, while the exact count (blue line, 40 digits) crosses zero at one point, the true root, which the balance of particles and holes finds. (Notebook 15d, figure 1.)

Out [4]:

```
Figure 15d.1 saved as Revision/textbook/figures/15d_1_root_conditioning.png
RESULT width of the interval where the double-precision direct count is 0 = 3.0e-10 m
RESULT largest |balance root - 40-digit root| (20 states) = 9.3e-17
PASS the double-precision balance finds mu to 1e-15
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    thermo_mu_well_conditioned_root
```

Text cell:

```
## 8. The chemical potential as a function of temperature

With ‘balance_root‘ the next cell computes $\mu$ at 49 temperatures from $0.002$ to
$0.05$ for $N = 8$ at $a_{4,0} = 0$, $1$, $2$, using the level set of the solver at
$T = 0.05$ (the largest window; at lower temperatures the extra levels are empty to
far below the rounding). For $a_{4,0} = 0$ the two-level picture explains $\mu$:
the eight zero modes ($\varepsilon = 0$) lose as many particles as the 24 orbitals of
the first band level ($\varepsilon = \Delta = 0.4307$), $8e^{-\mu/T} =
24e^{-(\Delta - \mu)/T}$, so $\mu = \Delta/2 - (T/2)\ln 3$ (the logarithm of the
ratio of degeneracies). The check compares this formula with the record at
$T = 0.01$.
```

In [5]:

```
T_GRID = np.linspace(0.002, 0.05, 49)


def curves(n, a4):
    """mu, E, S, F, C_V on T_GRID from the solver's T = 0.05 level set."""
    _, eps, deg, _ = states[(n, a4, 0.05)]
    rows = []
    for t in T_GRID:
        mu = balance_root(eps, deg, n, t)
        energy, entropy, free, _, cv, _ = functions_of_state(eps, deg, mu, t)
        rows.append((mu, energy, entropy, free, cv))
    return np.array(rows)  # columns: mu, E, S, F, C_V


GROUND = "Revision/kohn_sham/results/ground/summary.csv"
with open(repository_file(GROUND), newline="", encoding="utf-8") as handle:
    ground = {row["id"]: row for row in csv.DictReader(handle)}
gap8 = float(ground["N8_lam0_a00"]["KS_gap"])
two_level = gap8 / 2.0 - 0.5 * T_GRID * math.log(3.0)
curve8 = {a4: curves(8, a4) for a4 in (0.0, 1.0, 2.0)}
fig, ax = plt.subplots()
for a4 in (0.0, 1.0, 2.0):
    ax.plot(T_GRID, curve8[a4][:, 0], color=SHADES[a4], lw=2.0,
            label=f"$a_{{4,0}} = {a4:.0f}$")
    ax.plot(TEMPS, [float(record[state_id(8, a4, t)]["mu"]) for t in TEMPS], "o",
            color=SHADES[a4], ms=8, markerfacecolor="white")
ax.plot(T_GRID, two_level, "--", color=PALETTE[1], lw=1.5,
        label="$\\Delta/2 - (T/2)\\ln 3$, $a_{4,0} = 0$")
ax.plot([], [], "o", color="0.4", markerfacecolor="white", label="record")
ax.set_xlabel("temperature $T$ (units of $m$)")
ax.set_ylabel("chemical potential $\\mu$ (units of $m$)")
ax.set_title("$N = 8$, $\\lambda = 0$: the chemical potential")
ax.legend(fontsize=8)
save_figure(fig, "chemical_potential",
            "The chemical potential $\\mu$ (vertical axis, units of $m$) of the free "
            "state $N = 8$ against the temperature $T$ (horizontal axis, units of "
            "$m$) at the slices $a_{4,0} = 0$, $1$, $2$ (lines: this notebook; open "
            "circles: the record). At $T \\to 0$ the chemical potential sits in the "
            "middle of the gap, $\\Delta/2$, and falls linearly with the slope "
```

```
            "$-(\\ln 3)/2$ (dashed, the two-level formula); the gap and with it "
            "$\\mu$ shrink along the history.")
formula = gap8 / 2.0 - 0.005 * math.log(3.0)
recorded = float(record[state_id(8, 0.0, 0.01)]["mu"])
report("mu of N8_lam0_a00_T10: two-level formula / record",
       f"{formula:.12f} / {recorded:.12f}")
check(abs(formula - recorded) < 1e-8, "the two-level formula gives mu to 1e-8",
      record=f"{THERMO}, N8_lam0_a00_T10")
```

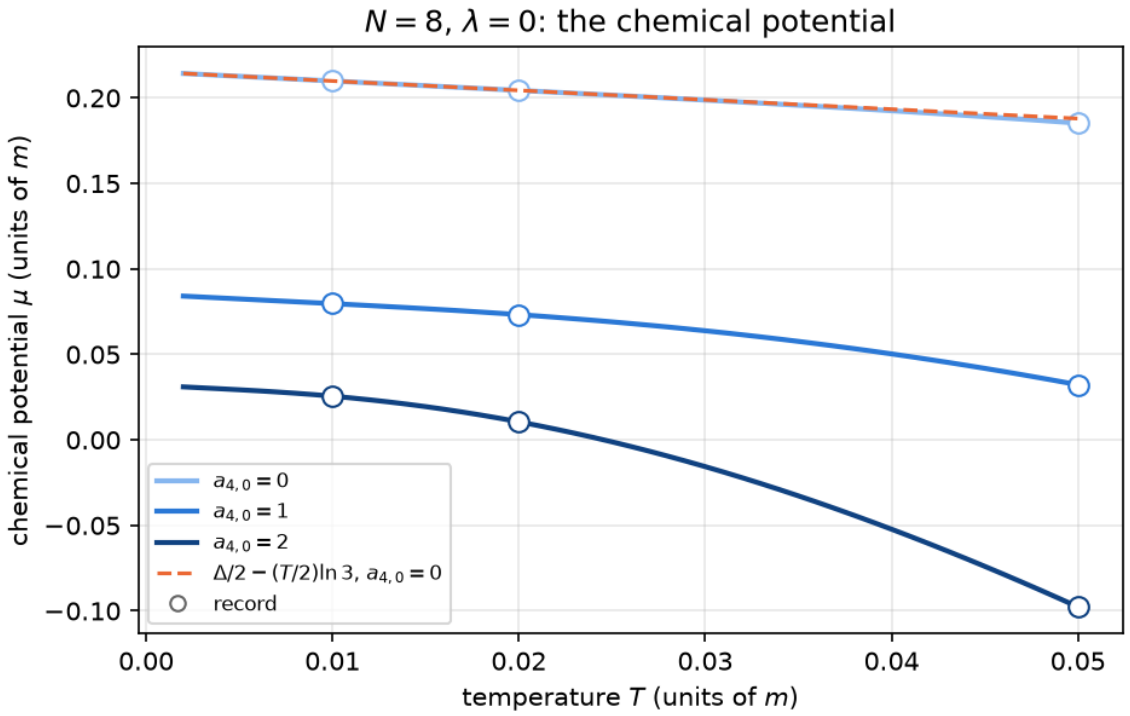

Figure 433: The chemical potential μ (vertical axis, units of m) of the free state $N = 8$ against the temperature T (horizontal axis, units of m) at the slices $a_{4,0} = 0$, 1, 2 (lines: this notebook; open circles: the record). At $T \to 0$ the chemical potential sits in the middle of the gap, $\Delta/2$, and falls linearly with the slope $-(\ln 3)/2$ (dashed, the two-level formula); the gap and with it μ shrink along the history. (Notebook 15d, figure 2.)

Out [5]:

```
Figure 15d.2 saved as Revision/textbook/figures/15d_2_chemical_potential.png
RESULT mu of N8_lam0_a00_T10: two-level formula / record = 0.209873777875 /
    0.209873776387
PASS the two-level formula gives mu to 1e-8
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, N8_lam0_a00_T10
```

Text cell:

```
## 9. Free energy and entropy along the history
```

```
The next cell computes the curves of $N = 136$ at the five slices and draws the
free energy measured from the ground-state energy, $F(T) - E_0$, and the entropy
$S(T)$; the record's values are the open circles. It also checks the thermodynamic
identity $-dF/dT = S$ with central differences $[F(T + h) - F(T - h)]/(2h)$,
$h = 10^{-3}T$, at every temperature of the grid.
```

In [6]:

```
curve136 = {a4: curves(136, a4) for a4 in (0.0, 0.5, 1.0, 1.5, 2.0)}
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
for a4, values in curve136.items():
    e0 = float(ground[f"N136_lam0_a{round(10 * a4):02d}"]["E_KS"])
    left.plot(T_GRID, values[:, 3] - e0, color=SHADES[a4], lw=2.0,
              label=f"$a_{{4,0}} = {a4}$")
    right.plot(T_GRID, values[:, 2], color=SHADES[a4], lw=2.0)
    if a4 in (0.0, 1.0, 2.0):
        ids = [state_id(136, a4, t) for t in TEMPS]
        left.plot(TEMPS, [float(record[i]["F"]) - e0 for i in ids], "o",
                  color=SHADES[a4], ms=7, markerfacecolor="white")
        right.plot(TEMPS, [float(record[i]["entropy"]) for i in ids], "o",
                   color=SHADES[a4], ms=7, markerfacecolor="white")
left.set_xlabel("temperature $T$")
left.set_ylabel("$F - E_0$ (units of $m$)")
left.set_title("Free energy, $N = 136$, $\\lambda = 0$")
left.legend(fontsize=8)
right.set_xlabel("temperature $T$")
right.set_ylabel("entropy $S$")
right.set_title("Entropy (circles: record)")
save_figure(fig, "free_energy_entropy",
            "Left: the free energy measured from the ground-state energy, "
            "$F - E_0$ (vertical axis, units of $m$), of the free state $N = 136$ at "
            "the five slices (light to dark blue) against the temperature (horizontal "
            "axis, units of $m$). Right: the entropy $S$ (pure number). Circles: the "
            "record. Along the history the levels crowd together, so the same "
            "temperature excites more particles: the entropy grows and the free "
            "energy falls faster.")
_, eps, deg, _ = states[(136, 1.0, 0.05)]
worst_fs = 0.0
for t in T_GRID:
    h = 1e-3 * t
    f_plus = functions_of_state(eps, deg, balance_root(eps, deg, 136, t + h), t + h)[2]
    f_minus = functions_of_state(eps, deg, balance_root(eps, deg, 136, t - h), t - h)[2]
    s = functions_of_state(eps, deg, balance_root(eps, deg, 136, t), t)[1]
    worst_fs = max(worst_fs, abs(-(f_plus - f_minus) / (2 * h) - s) / s)
report("largest relative |-dF/dT - S|, N = 136, a4,0 = 1", f"{worst_fs:.1e}")
check(worst_fs < 1e-4, "-dF/dT = S along the whole temperature grid",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "thermo_entropy_identity")
rising = all(bool(np.all(np.diff(v[:, 2]) > 0)) and bool(np.all(np.diff(v[:, 3]) < 0))
             for v in curve136.values())
check(rising, "S rises and F falls with T at every slice")
```

Out [6]:

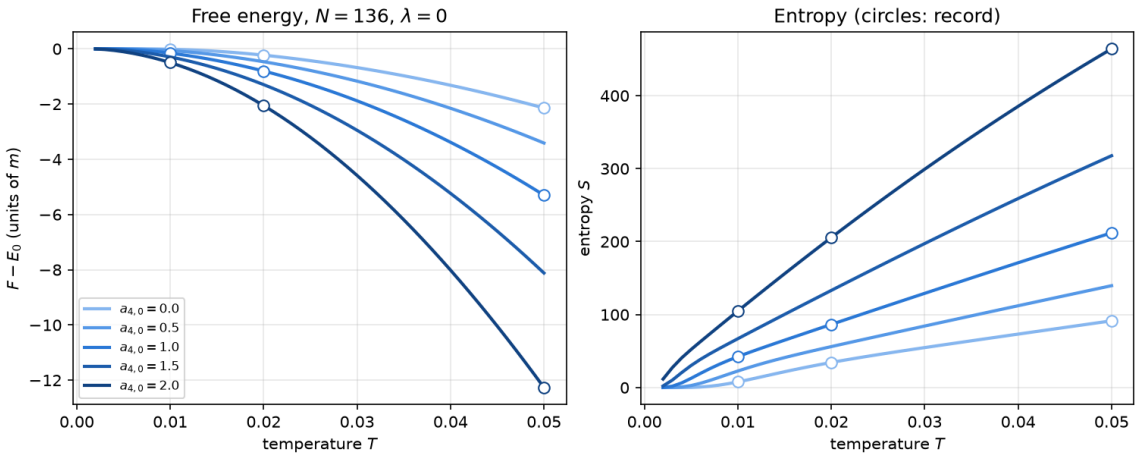

Figure 434: Left: the free energy measured from the ground-state energy, $F - E_0$ (vertical axis, units of m), of the free state $N = 136$ at the five slices (light to dark blue) against the temperature (horizontal axis, units of m). Right: the entropy S (pure number). Circles: the record. Along the history the levels crowd together, so the same temperature excites more particles: the entropy grows and the free energy falls faster. (Notebook 15d, figure 3.)

```
Figure 15d.3 saved as Revision/textbook/figures/15d_3_free_energy_entropy.png
RESULT largest relative |-dF/dT - S|, N = 136, a4,0 = 1 = 7.5e-06
PASS -dF/dT = S along the whole temperature grid
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    thermo_entropy_identity
PASS S rises and F falls with T at every slice
```

Text cell:

```
## 10. The heat capacity

The next cell draws $C_V(T)$ (closed form) for $N = 8$ at three slices (left) and
$N = 136$ at five slices (right), on logarithmic vertical axes, with the record's
values. For $N = 8$ at $a_{4,0} = 0$ the gas is **activated**: below $T \approx 0.02$
the heat capacity is tiny, because the first excitation costs the gap $0.43\,m$.
Along the history the gap closes and the heat capacity rises at lower temperatures.
$N = 136$ has a much smaller gap ($0.083\,m$ at $a_{4,0} = 0$, $0.014\,m$ at $2$) and
a large heat capacity already at $T = 0.01$.
```

In [7]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
for a4, values in curve8.items():
    left.plot(T_GRID, values[:, 4], color=SHADES[a4], lw=2.0,
              label=f"$a_{{4,0}} = {a4:.0f}$")
    left.plot(TEMPS, [float(record[state_id(8, a4, t)]["C_V"]) for t in TEMPS], "o",
              color=SHADES[a4], ms=7, markerfacecolor="white")
for a4, values in curve136.items():
    right.plot(T_GRID, values[:, 4], color=SHADES[a4], lw=2.0,
```

```
                label=f"$a_{{4,0}} = {a4}$")
    if a4 in (0.0, 1.0, 2.0):
        right.plot(TEMPS, [float(record[state_id(136, a4, t)]["C_V"]) for t in TEMPS],
                   "o", color=SHADES[a4], ms=7, markerfacecolor="white")
for ax, n in ((left, 8), (right, 136)):
    ax.set_yscale("log")
    ax.set_xlabel("temperature $T$ (units of $m$)")
    ax.set_title(f"Heat capacity, $N = {n}$, $\\lambda = 0$")
    ax.legend(fontsize=8)
left.set_ylabel("$C_V$ (pure number)")
save_figure(fig, "heat_capacity",
            "The heat capacity $C_V = dE/dT$ (vertical axis, logarithmic, pure "
            "number) of the free states $N = 8$ (left, three slices) and $N = 136$ "
            "(right, five slices) against the temperature (horizontal axis, units "
            "of $m$); circles: the record. The gas $N = 8$ is activated across its "
            "gap of $0.43\\,m$ at $a_{4,0} = 0$; as the history closes the gaps, "
            "the heat capacity rises at ever lower temperatures.")
positive = all(bool(np.all(v[:, 4] > 0)) for v in list(curve8.values())
               + list(curve136.values()))
check(positive, "C_V > 0 at every temperature and slice")
```

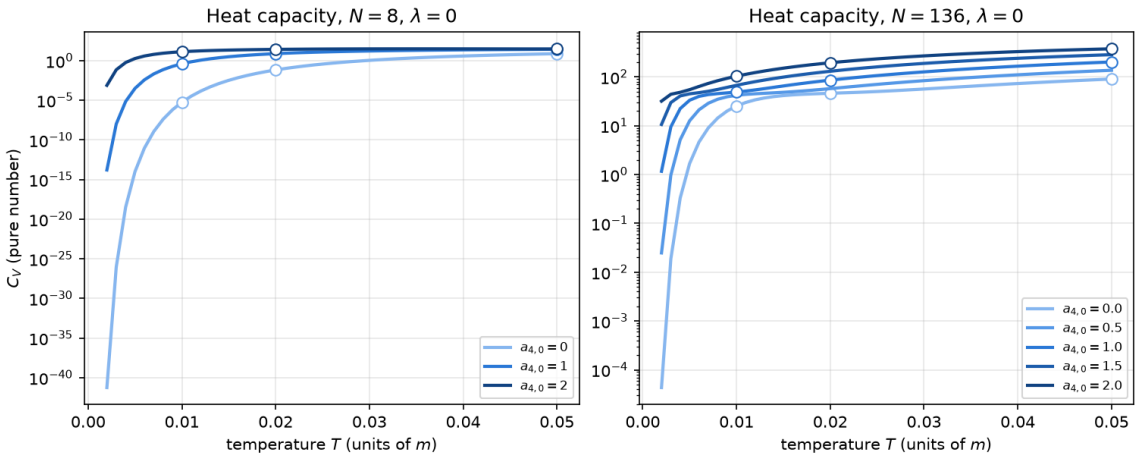

Figure 435: The heat capacity $C_V = dE/dT$ (vertical axis, logarithmic, pure number) of the free states $N = 8$ (left, three slices) and $N = 136$ (right, five slices) against the temperature (horizontal axis, units of m); circles: the record. The gas $N = 8$ is activated across its gap of $0.43\,m$ at $a_{4,0} = 0$; as the history closes the gaps, the heat capacity rises at ever lower temperatures. (Notebook 15d, figure 4.)

Out [7]:

```
Figure 15d.4 saved as Revision/textbook/figures/15d_4_heat_capacity.png
PASS C_V > 0 at every temperature and slice
```

Text cell:

```
## 11. What the occupations look like
```

```
The next cell draws the occupation $f$ of every level of $N = 136$ at
$a_{4,0} = 1$ against its energy, at the three temperatures of the record, with the
chemical potential marked: the step of the Fermi-Dirac function softens over a width
of a few $T$. The check confirms that the occupations add up to $N = 136$.
```

In [8]:

```
fig, ax = plt.subplots()
sums = []
for colour, t in zip(PALETTE, TEMPS):
    mu, eps, deg, levels = states[(136, 1.0, t)]
    e = np.array(eps)
    f = 1.0 / (1.0 + np.exp(np.minimum((e - mu) / t, 700.0)))
    sums.append(float(np.sum(np.array(deg) * f)))
    order = np.argsort(e)
    ax.plot(e[order], f[order], "o-", color=colour, ms=4, lw=1.0, label=f"$T = {t}$")
    ax.axvline(mu, color=colour, ls=":", lw=1.2)
ax.set_xlim(0.2, 0.5)
ax.set_xlabel("level $\\varepsilon$ (units of $m$)")
ax.set_ylabel("occupation $f$")
ax.set_title("$N = 136$, $a_{4,0} = 1$: Fermi-Dirac occupations (dotted: $\\mu$)")
ax.legend()
save_figure(fig, "occupations",
            "The occupation $f$ of the levels of the free state $N = 136$ at "
            "$a_{4,0} = 1$ (vertical axis) against their energy (horizontal axis, "
            "units of $m$, near the Fermi level) at the temperatures $T = 0.01$, "
            "$0.02$, $0.05$; dotted lines: the chemical potentials. The step from "
            "full to empty widens with the temperature, over a few $T$.")
report("sum g f at the three temperatures", ", ".join(f"{s:.12f}" for s in sums))
check(max(abs(s - 136.0) for s in sums) < 1e-9, "the occupations add up to N = 136",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "thermo_N_conservation")
```

Out [8]:

```
Figure 15d.5 saved as Revision/textbook/figures/15d_5_occupations.png
RESULT sum g f at the three temperatures = 136.000000000000, 136.000000000000,
    136.000000000000
PASS the occupations add up to N = 136
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    thermo_N_conservation
```

Text cell:

```
## 12. The filling convention and its sea holes

The convention fills only the positive branch. The sea's brane band (block type
$j = -1$, even parity, label 0) has, without interaction, exactly the energies
$-\varepsilon_{band}(n_2)$ of the brane band of $j = +1$ (the block Hamiltonians obey
$h_{-1} = -h_{+1}$ when $v = 0$). If the sea were treated thermally, its levels would
carry holes with the probability $1 - f(-\varepsilon_{band}) =
```

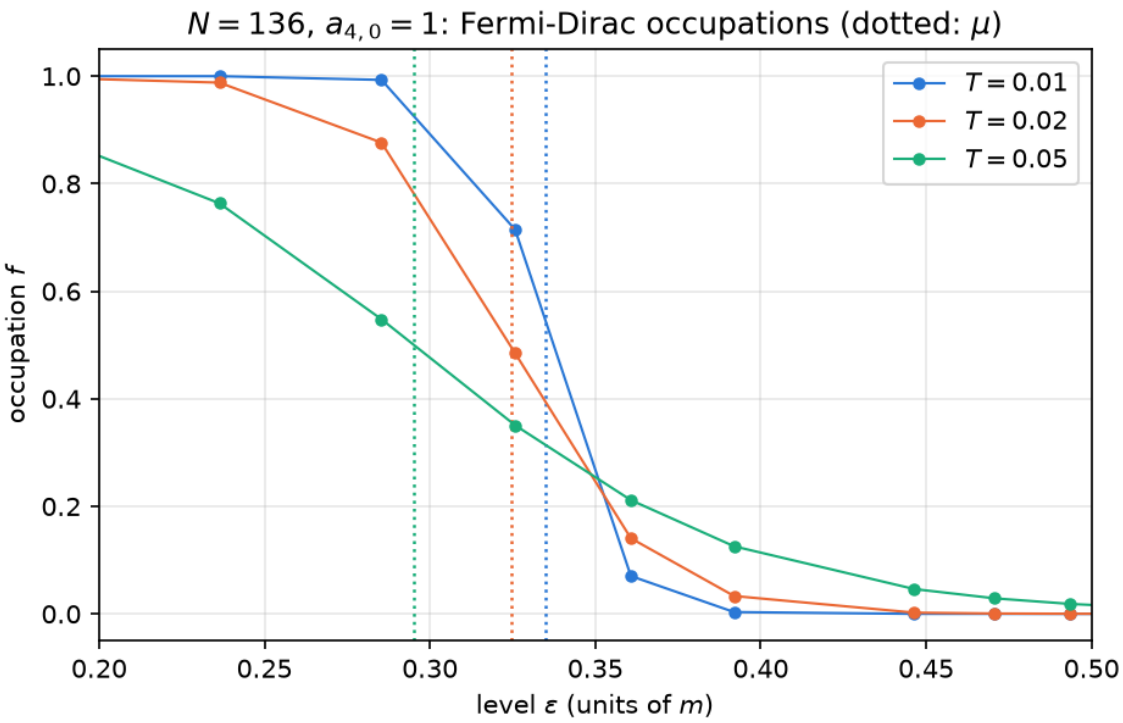

Figure 436: The occupation f of the levels of the free state $N = 136$ at $a_{4,0} = 1$ (vertical axis) against their energy (horizontal axis, units of m, near the Fermi level) at the temperatures $T = 0.01$, 0.02, 0.05; dotted lines: the chemical potentials. The step from full to empty widens with the temperature, over a few T. (Notebook 15d, figure 5.)

```
1/(1 + e^{(\mu + \varepsilon_{band})/T})$. The next cell adds up
$4 r_3(n_2)$ times this probability over the shells $n_2 \ge 1$ of the level set and
compares with the record's column 'sea_holes_excluded' for $N = 8$ at $T = 0.05$. The
figure shows the record's number of sea holes per particle for $N = 8$ and $136$ at
all slices and temperatures: above about 1 percent (dotted line) the particle-only
ensemble is outside its range of validity, and a thermal treatment of the sea
(particle-antiparticle pairs) would be needed.
```

In [9]:

```
def shell_sizes(largest):
    """r3(n2): the number of whole-number vectors with n2 = |n|^2, n2 <= largest."""
    reach = math.isqrt(largest) + 1
    sizes = {}
    for x in range(-reach, reach + 1):
        for y in range(-reach, reach + 1):
            for z in range(-reach, reach + 1):
                s = x * x + y * y + z * z
                if s <= largest:
                    sizes[s] = sizes.get(s, 0) + 1
    return sizes


worst_sea, mine = 0.0, {}
for a4 in (0.0, 1.0, 2.0):
    mu, _, _, levels = states[(8, a4, 0.05)]
    band = {lv[0]: lv[4] for lv in levels
            if lv[1] == 1 and lv[2] == "even" and lv[3] == 0 and lv[0] >= 1}
    sizes = shell_sizes(max(lv[0] for lv in levels))
    holes = sum(4 * sizes[n2] / (1.0 + math.exp((mu + band[n2]) / 0.05))
                for n2 in sizes if n2 >= 1)
    mine[a4] = holes / 8.0
    recorded = float(record[state_id(8, a4, 0.05)]["sea_holes_excluded"])
    worst_sea = max(worst_sea, abs(holes / recorded - 1.0))
fig, ax = plt.subplots()
plotted = []  # every value drawn, to measure their range
for marker, n in (("o", 8), ("s", 136)):
    for colour, t in zip(PALETTE, TEMPS):
        values = [float(record[state_id(n, a, t)]["sea_holes_over_N"])
                  for a in (0.0, 0.5, 1.0, 1.5, 2.0)]
        plotted += values
        ax.plot([0, 0.5, 1, 1.5, 2], np.maximum(values, 1e-60), marker + "-",
                color=colour, ms=6, lw=1.2,
                markerfacecolor=colour if n == 8 else "white",
                label=f"$N = {n}$, $T = {t}$")
ax.plot(list(mine), list(mine.values()), "x", color="black", ms=11, mew=2,
        label="this notebook, $N = 8$, $T = 0.05$")
ax.axhline(0.01, color="0.3", ls=":", lw=1.2, label="1 percent")
ax.set_yscale("log")
ax.set_ylim(1e-60, 1e4)  # the smallest recorded value is about 2e-56
ax.set_xlabel("slice $a_{4,0}$")
ax.set_ylabel("sea holes per particle")
ax.set_title("Diagnostic of the filling convention ($\\lambda = 0$)")
```

```
ax.legend(fontsize=7, ncol=2, loc="lower right")
smallest = min(v for v in plotted if v > 0.0)  # the smallest nonzero value
powers = int(math.log10(max(plotted) / smallest))  # whole powers of ten spanned
save_figure(fig, "sea_holes",
            "The number of thermal holes that the excluded sea brane band would "
            "carry, per particle (vertical axis, logarithmic), for $N = 8$ (filled) "
            "and $N = 136$ (open) at $T = 0.01$, $0.02$, $0.05$ against the slice "
            "(horizontal axis), from the record; crosses: recomputed here. The values "
            f"span more than {powers} powers of ten: at low temperature and early in "
            "the history the sea is practically full. Above the dotted 1 percent "
            "line, reached late in the history at the higher temperatures, the "
            "particle-only convention is outside its range of validity.")
report("smallest and largest sea holes per particle drawn",
       f"{smallest:.2e}, {max(plotted):.2e}")
report("sea holes per particle, N = 8, T = 0.05, a4,0 = 0, 1, 2",
       ", ".join(f"{v:.4g}" for v in mine.values()))
check(worst_sea < 1e-9, "the sea-hole diagnostic reproduced",
      record=f"{THERMO}, column sea_holes_excluded")
```

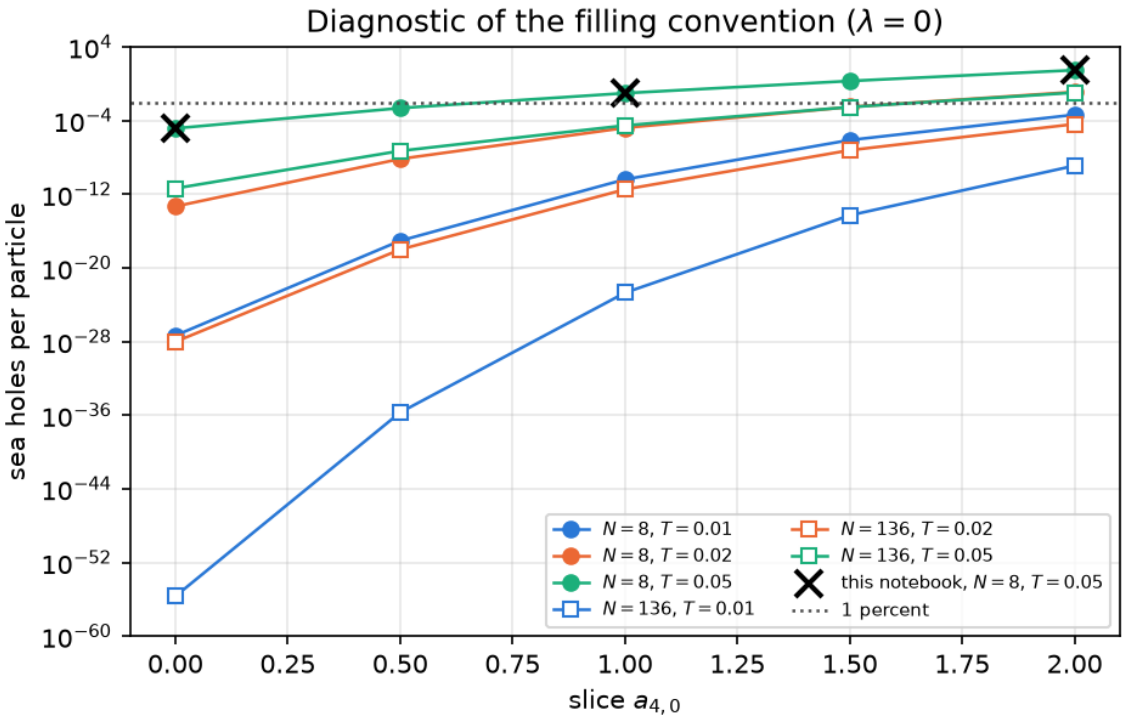

Figure 437: The number of thermal holes that the excluded sea brane band would carry, per particle (vertical axis, logarithmic), for $N = 8$ (filled) and $N = 136$ (open) at $T = 0.01$, 0.02, 0.05 against the slice (horizontal axis), from the record; crosses: recomputed here. The values span more than 57 powers of ten: at low temperature and early in the history the sea is practically full. Above the dotted 1 percent line, reached late in the history at the higher temperatures, the particle-only convention is outside its range of validity. (Notebook 15d, figure 6.)

Out [9]:

```
Figure 15d.6 saved as Revision/textbook/figures/15d_6_sea_holes.png
RESULT smallest and largest sea holes per particle drawn = 2.10e-56, 3.08e+01
RESULT sea holes per particle, N = 8, T = 0.05, a4,0 = 0, 1, 2 = 1.464e-05, 0.09746, 30.8
PASS the sea-hole diagnostic reproduced
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, column
    sea_holes_excluded
```

Text cell:

```
## 13. The interaction at finite temperature

The last figure uses only the record: the change of the free energy caused by the
couplings $\pm\lambda_1$, $F(\lambda) - F(0)$, for $N = 8$, $136$ and $688$ at the
first slice and the three temperatures. The two checks: the interaction changes $F$
by less than $0.005\,m$; and repulsion ($+\lambda_1$) raises it for $N = 136$ and
$688$ but lowers it for $N = 8$, attraction the other way round (the zero-mode state
$N = 8$ has no scalar density without interaction, so to first order its
interaction energy is the exchange term $-\tfrac{1}{32}\lambda n^2$, negative for
$\lambda > 0$).
```

In [10]:

```
fig, ax = plt.subplots()
shifts = []
by_series = {}  # (N, tag) -> the three values of F(lambda) - F(0)
for colour, n in zip(PALETTE, (8, 136, 688)):
    for tag, style in (("lamp1", "-"), ("lamm1", "--")):
        values = [float(record[state_id(n, 0.0, t, tag)]["F"])
                  - float(record[state_id(n, 0.0, t)]["F"]) for t in TEMPS]
        shifts += values
        by_series[(n, tag)] = values
        sign = "+" if tag == "lamp1" else "-"
        ax.plot(TEMPS, values, style, marker="o", color=colour, ms=6, lw=1.5,
                label=f"$N = {n}$, ${sign}\\lambda_1$")
ax.set_yscale("symlog", linthresh=1e-4)
ax.axhline(0.0, color="0.5", lw=0.8)
ax.set_xlabel("temperature $T$ (units of $m$)")
ax.set_ylabel("$F(\\lambda) - F(0)$ (units of $m$)")
ax.set_title("Effect of the interaction on the free energy, $a_{4,0} = 0$")
ax.legend(fontsize=8, ncol=2)
up = by_series[(688, "lamp1")]  # N = 688, +lambda_1, at T = 0.01, 0.02, 0.05
save_figure(fig, "interaction_free_energy",
            "The change of the free energy caused by the couplings $+\\lambda_1$ "
            "(solid) and $-\\lambda_1$ (dashed), $F(\\lambda) - F(0)$ (vertical axis, "
            "symmetric logarithmic, units of $m$), for $N = 8$, $136$, $688$ at "
            "$a_{4,0} = 0$ against the temperature (horizontal axis), from the "
            "record. The effect is small and changes only slowly with $T$ (for "
            f"$N = 688$ and $+\\lambda_1$ it grows from ${up[0]:.4f}\\,m$ at "
            f"$T = 0.01$ to ${up[-1]:.4f}\\,m$ at $T = 0.05$); its sign follows the "
            "sign of the coupling, the other way round for $N = 8$.")
report("largest |F(lambda) - F(0)| at a4,0 = 0", f"{max(abs(s) for s in shifts):.4f}")
check(max(abs(s) for s in shifts) < 0.005,
```

```
      "the interaction changes F by less than 0.005 at a4,0 = 0",
      record=f"{THERMO}, column F")
raises = all(v > 0.0 for n in (136, 688) for v in by_series[(n, "lamp1")]) and all(
    v < 0.0 for n in (136, 688) for v in by_series[(n, "lamm1")])
lowers8 = all(v < 0.0 for v in by_series[(8, "lamp1")]) and all(
    v > 0.0 for v in by_series[(8, "lamm1")])
check(raises and lowers8,
      "repulsion raises F for N = 136 and 688 and lowers it for N = 8",
      record=f"{THERMO}, column F")
```

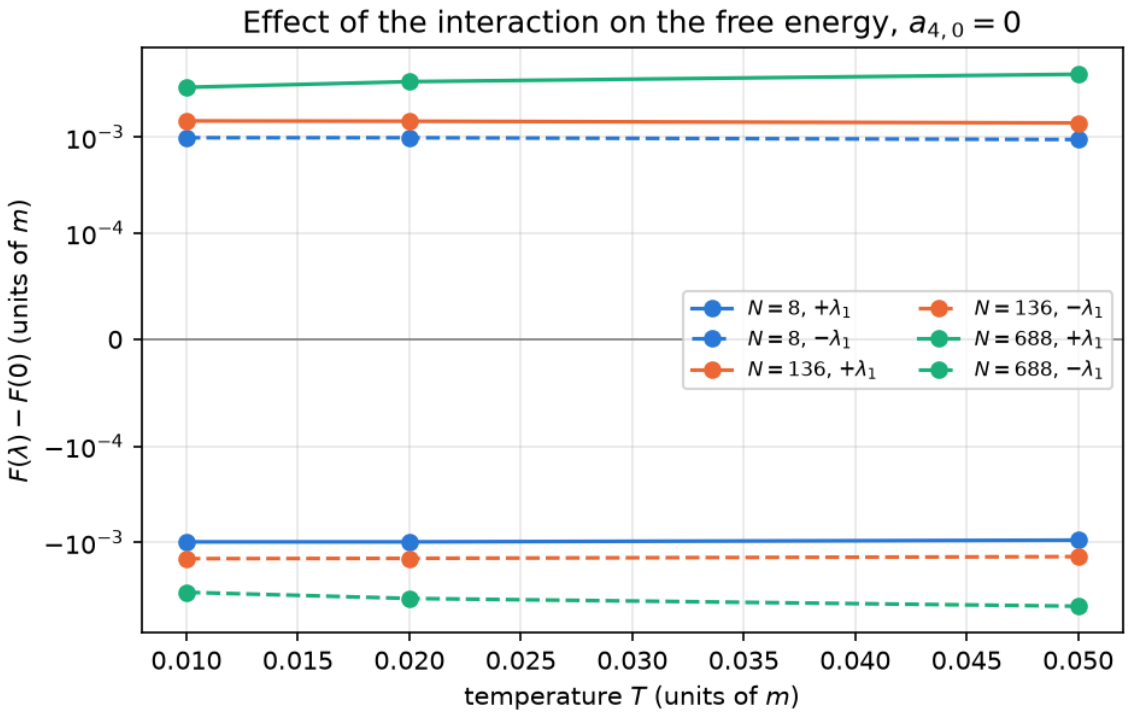

Figure 438: The change of the free energy caused by the couplings $+\lambda_1$ (solid) and $-\lambda_1$ (dashed), $F(\lambda) - F(0)$ (vertical axis, symmetric logarithmic, units of m), for $N = 8$, 136, 688 at $a_{4,0} = 0$ against the temperature (horizontal axis), from the record. The effect is small and changes only slowly with T (for $N = 688$ and $+\lambda_1$ it grows from $0.0033\,m$ at $T = 0.01$ to $0.0045\,m$ at $T = 0.05$); its sign follows the sign of the coupling, the other way round for $N = 8$. (Notebook 15d, figure 7.)

Out [10]:

```
Figure 15d.7 saved as Revision/textbook/figures/15d_7_interaction_free_energy.png
RESULT largest |F(lambda) - F(0)| at a4,0 = 0 = 0.0046
PASS the interaction changes F by less than 0.005 at a4,0 = 0
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, column F
PASS repulsion raises F for N = 136 and 688 and lowers it for N = 8
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, column F
```

Text cell:

```
## 14. The last check

The last cell checks that every figure file of this notebook exists and prints the
number of checks that passed.
```

In [11]:

```
NAMES = ["root_conditioning", "chemical_potential", "free_energy_entropy",
         "heat_capacity", "occupations", "sea_holes", "interaction_free_energy"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/15d_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

Out [11]:

```
PASS every figure file of this notebook exists
ALL 15 CHECKS PASSED (notebook 15d)
```

Text cell:

```
## 15. What this notebook showed

- From the solver's levels alone, plain Python reproduces the chemical potential,
  energy, entropy, free energy, grand potential, heat capacity and $dN/d\mu$ of the
  free thermal states of the record (COMPUTED).
- The solver's chemical potential lies within its rounding bound of the 40-digit
  root; the direct count in double precision could not do this, which is why the
  solver balances thermal particles against thermal holes.
- $\mu$ starts in the middle of the gap and falls with $T$; along the deflating
  history the gaps close, so the entropy rises, the free energy falls faster and the
  heat capacity rises at lower temperatures; $-dF/dT = S$ holds.
- The particle-only filling is a CONVENTION; late in the history at the higher
  temperatures the excluded sea would carry many holes (up to 31 per particle for
  $N = 8$), and there the convention is outside its range of validity (OPEN: a
  thermal treatment of the sea).
- These are instantaneous states on a PRESCRIBED background, with the brane
  ASSUMED.
```

15.30 Line-by-line walk-through of Notebook 15d

The notebook has 11 code cells, In [1] to In [11]; the complete cells are printed in Section 15.29.

In [1], the set-up cell. It is the set-up cell of Notebook 15a (Section 15.14 for the Rust lines, Section 15.9 for the rest), with the run instructions of this notebook (Section 15.28) in its comments and `NOTEBOOK_ID = "15d"`.

In [2], the solver and the free thermal states.

```
import csv  # reads the tables (CSV files)
import math  # exp, log for single numbers

import mpmath  # numbers with 40 digits
import numpy as np  # arrays of numbers

program = rust_program("Revision/kohn_sham/solver/Cargo.toml", "revision_ks_solver")
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_15d"  # git ignores it
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
```

The imports, among them mpmath, a package for numbers with as many digits as asked for; the solver is built, and the folder `textbook_15d` inside its build folder receives this notebook's runs.

```
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
SHADES = {0.0: "#86b6ef", 0.5: "#5598e7", 1.0: "#2a78d6", 1.5: "#1c5cab",
          2.0: "#104281"}  # one shade of blue per slice, light to dark
TEMPS = (0.01, 0.02, 0.05)


def state_id(n, a4, t, tag="lam0"):
    return f"N{n}_{tag}_a{round(10 * a4):02d}_T{round(1000 * t)}"
```

The colours, one shade per slice in a dictionary, the three temperatures of the record, and the names of the thermal states, for example `N8_lam0_a00_T10` for $N = 8$, $\lambda = 0$, $a_{4,0} = 0$, $T = 0.01$.

```
def thermal_state(n, a4, t):
    name = state_id(n, a4, t)
    out, lev = RUN_FOLDER / f"{name}.json", RUN_FOLDER / f"{name}-levels.json"
    done = subprocess.run(
        [str(program), "single", "--root", str(REPO), "--m", "1", "--lambda", "0",
         "--a4", repr(a4), "--N", repr(float(n)), "--T", repr(t), "--margin", "0.2",
         "--out", str(out), "--mermin-levels", str(lev)],
        capture_output=True, text=True)
    if done.returncode != 0:
        raise RuntimeError(f"the solver failed for {name}: {done.stderr[-500:]}")
```

`thermal_state` runs `single` for one free thermal state. Compared with Notebook 15b it adds the temperature, the margin 0.2 of the canonical thermal runs (so that the label set, and therefore the state, is exactly the recorded one), and the option that writes the final levels and μ into a second file as decimal numbers that read back into exactly the solver's double-precision numbers.

```
    exact = json.loads(lev.read_text(encoding="utf-8"))
    full = json.loads(out.read_text(encoding="utf-8"))
    eps = [float(e) for e, _ in exact["levels_eps_deg"]]  # exactly the solver's
    deg = [float(g) for _, g in exact["levels_eps_deg"]]
    return float(exact["mu"]), eps, deg, full["levels_n2_j_parity_label_eps_deg_f"]
```

The two files are read; `eps` and `deg` are the levels and their degeneracies, and the function returns the solver's μ, these two lists, and the full level list (shell, block type, parity, label, energy, degeneracy, occupation) from the main JSON file.

```
THERMO = "Revision/kohn_sham/results/thermo/thermodynamics.csv"
with open(repository_file(THERMO), newline="", encoding="utf-8") as handle:
    record = {row["id"]: row for row in csv.DictReader(handle)}
states = {}
for n in (8, 136):
    for a4 in (0.0, 1.0, 2.0):
        for t in TEMPS:
            states[(n, a4, t)] = thermal_state(n, a4, t)
for a4 in (0.5, 1.5):
    states[(136, a4, 0.05)] = thermal_state(136, a4, 0.05)
```

The record's table of the 135 thermal states is read, and 20 free states are computed: $N = 8$ and 136 at the slices 0, 1, 2 and the three temperatures (18 states), and $N = 136$ at $a_{4,0} = 0.5$ and 1.5 with $T = 0.05$.

```
worst_mu = max(abs(mu - float(record[state_id(n, a4, t)]["mu"]))
               for (n, a4, t), (mu, _, _, _) in states.items())
report("free thermal states computed", len(states))
report("largest |mu(solver now) - mu(record)|", f"{worst_mu:.1e}")
check(worst_mu < 1e-12, f"the solver reproduces the recorded mu of {len(states)} states",
      record=f"{THERMO}, column mu")
```

Out [2]: 20 states, and the new μ agrees with the record to 5.6×10^{-17} (the record's column holds 16 significant digits).

In [3], the 40-digit root and the functions of state.

```
def exact_root(eps, deg, n, t, guess):
    with mpmath.workdps(40):
        levels = [(mpmath.mpf(e), mpmath.mpf(g)) for e, g in zip(eps, deg)]
        temp, mu = mpmath.mpf(t), mpmath.mpf(guess)
        for _ in range(60):
            occ = [(g, 1 / (1 + mpmath.exp((e - mu) / temp))) for e, g in levels]
            count = sum(g * f for g, f in occ) - n
            slope = sum(g * f * (1 - f) for g, f in occ) / temp  # dN/dmu
            step = count / slope
            mu -= step
            if abs(step) < mpmath.mpf(10) ** -36:
                break
        return mu
```

`exact_root` solves $\sum gf = N$ with 40 significant digits: inside `with mpmath.workdps(40):` every mpmath number (`mpmath.mpf`) carries 40 digits. Starting at the solver's μ, Newton's method takes the step $(\sum gf - N)/(dN/d\mu)$ with $dN/d\mu = \sum gf(1-f)/T$ (Section 15.26), at most 60 times, until the step is below 10^{-36}. With 40 digits the subtraction of N loses nothing that matters.

```
def functions_of_state(eps, deg, mu, t):
    e, g = np.array(eps), np.array(deg)
    x = (e - mu) / t
    small = np.exp(-np.abs(x))  # e^{-|x|}, never overflows
```

```
    f_abs = small / (1.0 + small)  # f(|x|)
    f = np.where(x > 0, f_abs, 1.0 - f_abs)  # f(x)
    energy = float(np.sum(g * f * e))
```

`functions_of_state` computes the functions of state of a free state in double precision, in forms that never overflow. `small` is $e^{-|x|}$, at most 1; `f_abs` is $f(|x|) = e^{-|x|}/(1+e^{-|x|})$; `np.where(conditi on, a, b)` takes a where the condition holds and b elsewhere, so `f` is $f(x)$ ($f(|x|)$ for $x > 0$ and $1 - f(|x|)$ otherwise). The energy is $\sum gf\varepsilon$ (no interaction).

```
    # -[f ln f + (1-f) ln(1-f)] = ln(1 + e^{-|x|}) + |x| f(|x|)
    entropy = float(np.sum(g * (np.log1p(small) + np.abs(x) * f_abs)))
    # ln(1 + e^{-x}) = max(-x, 0) + ln(1 + e^{-|x|})
    omega = -t * float(np.sum(g * (np.maximum(-x, 0.0) + np.log1p(small))))
```

The two comment lines state the formulas that the next lines use. The entropy of one level is $-[f \ln f + (1-f)\ln(1-f)] = \ln(1+e^{-|x|}) + |x|\, f(|x|)$. (For $x \geq 0$: $\ln f = -x - \ln(1+e^{-x})$ and $\ln(1-f) = -\ln(1+e^{-x})$; inserting, the two logarithms add to $\ln(1+e^{-x})$ and the rest is xf; for $x < 0$ the roles of f and $1-f$ exchange.) `np.log1p(s)` is $\ln(1+s)$, accurate also for tiny s. The grand potential uses $\ln(1+e^{-x}) = \max(-x, 0) + \ln(1+e^{-|x|})$.

```
    w = g * f_abs * (1.0 - f_abs)  # g f (1 - f), the same for x and -x
    d = e - mu
    cv = float((np.sum(w * d * d) - np.sum(w * d) ** 2 / np.sum(w)) / t ** 2)
    return energy, entropy, energy - t * entropy, omega, cv, float(np.sum(w) / t)
```

`w` is $w_i = g_i f_i (1 - f_i)$ (the product $f(1-f)$ is the same for x and $-x$), `cv` the closed form of C_V derived in Section 15.26, and the function returns E, S, $F = E - TS$, Ω, C_V and $dN/d\mu = \sum w/T$.

```
rel = lambda a, b: abs(a - b) / max(abs(b), 1.0)  # relative to max(|b|, 1)
worst = {"root": 0.0, "E S F Omega": 0.0, "C_V": 0.0, "dN/dmu": 0.0}
for (n, a4, t), (mu, eps, deg, _) in states.items():
    row = record[state_id(n, a4, t)]
    root = exact_root(eps, deg, n, t, mu)
    worst["root"] = max(worst["root"],
                        float(abs(mu - root)) / float(row["mu_rounding_bound"]))
    e_, s_, f_, o_, cv, dn = functions_of_state(eps, deg, mu, t)
    worst["E S F Omega"] = max(worst["E S F Omega"], rel(e_, float(row["E"])),
                               rel(s_, float(row["entropy"])), rel(f_, float(row["F"])),
                               rel(o_, float(row["Omega_direct"])))
    worst["C_V"] = max(worst["C_V"], abs(cv / float(row["C_V"]) - 1.0))
    worst["dN/dmu"] = max(worst["dN/dmu"], abs(dn / float(row["dN_dmu"]) - 1.0))
```

For each of the 20 states: the 40-digit root, the distance of the solver's μ from it measured in units of the recorded rounding bound (column `mu_rounding_bound`, the solver's first-order estimate of the rounding error of its root), and the functions of state compared with the record's columns (relative to the larger of the value and 1). The dictionary `worst` keeps the largest deviation of each kind.

```
report("largest |mu - 40-digit root| / recorded rounding bound", f"{worst['root']:.2f}")
for key in ("E S F Omega", "C_V", "dN/dmu"):
```

```
    report(f"largest relative difference, {key}", f"{worst[key]:.1e}")
check(worst["root"] <= 1.0, "mu lies within its rounding bound of the 40-digit root",
      record="Revision/kohn_sham/reports/ks-rust-mermin-roots.json and "
             f"{THERMO}, column mu_rounding_bound")
check(worst["E S F Omega"] < 1e-12,
      f"E, S, F and Omega of {len(states)} states reproduced",
      record=f"{THERMO}, columns E, entropy, F, Omega_direct")
check(worst["C_V"] < 1e-4, "the closed-form C_V reproduces the recorded T dS/dT",
      record=f"{THERMO}, column C_V")
check(worst["dN/dmu"] < 1e-9, "dN/dmu reproduced", record=f"{THERMO}, column dN_dmu")
```

Out [3]: μ lies within 0.07 of its rounding bound of the exact root; E, S, F, Ω agree to 3.6×10^{-14}; the closed-form C_V agrees with the record's Richardson difference $T\,dS/dT$ to 1.6×10^{-5} (the accuracy of the difference quotient); $dN/d\mu$ to 2.0×10^{-15}. All four checks pass.

In [4], why the balance is needed.

```
def balance_root(eps, deg, n, t):
    order = np.argsort(eps, kind="stable")
    e, g = np.array(eps)[order], np.array(deg)[order]
    filled = int(np.searchsorted(np.cumsum(g), n - 1e-9)) + 1  # T = 0 filling
    below_e, below_g, above_e, above_g = e[:filled], g[:filled], e[filled:], g[filled:]
```

`balance_root` finds μ from the balance of Section 15.26 in double precision. The levels are sorted by energy (`np.argsort` gives the order; `kind="stable"` keeps equal levels in their original order). `np.cumsum(g)` is the running total of the degeneracies, and `np.searchsorted` finds the position where it reaches N; the first `filled` levels are the set $\mathcal{S}$ filled at $T = 0$ (the states here are closed shells, so $d = 0$).

```
    def log_sum(log_terms):  # ln(sum e^{t}) without overflow or underflow
        top = float(np.max(log_terms))
        return top + math.log(float(np.sum(np.exp(log_terms - top))))

    def log_f(x):  # ln f(x) = -ln(1 + e^{x}), stable for every x
        return -(np.maximum(x, 0.0) + np.log1p(np.exp(-np.abs(x))))
```

`log_sum` is the log-sum-exp $\ln \sum e^{t_i} = t_{max} + \ln \sum e^{t_i - t_{max}}$, in which every exponential is at most 1. `log_f` is $\ln f(x) = -\ln(1 + e^x) = -[\max(x, 0) + \ln(1 + e^{-|x|})]$, which works for every x, however large.

```
    def balance(mu):
        return (log_sum(np.log(above_g) + log_f((above_e - mu) / t))
                - log_sum(np.log(below_g) + log_f(-(below_e - mu) / t)))

    lo, hi = float(e[0]) - 1.0, float(e[-1]) + 1.0
    while lo < 0.5 * (lo + hi) < hi:  # bisection down to neighbouring numbers
        mid = 0.5 * (lo + hi)
        lo, hi = (mid, hi) if balance(mid) < 0.0 else (lo, mid)
    return lo
```

`balance(mu)` is $\ln P - \ln H_l$: each term $g\,f$ is written as $e^{\ln g + \ln f}$, so that $\ln P$ = log-sum-exp of $\ln g + \ln f(x)$ over the levels above, and $\ln H_l$ the same with $f(-x)$ over the levels below. It increases with μ (more particles above, fewer holes below). Bisection between a value below all levels and one above them halves the interval until its midpoint is no longer strictly between its ends, that is, until `lo` and `hi` are neighbouring double-precision numbers.

```
mu8, eps8, deg8, _ = states[(8, 0.0, 0.01)]
root8 = exact_root(eps8, deg8, 8, 0.01, mu8)
offsets = np.linspace(-3e-9, 3e-9, 61)
direct, exact = [], []
for off in offsets:
    x = (np.array(eps8) - (mu8 + off)) / 0.01
    direct.append(float(np.sum(np.array(deg8) / (1.0 + np.exp(x)))) - 8.0)
    with mpmath.workdps(40):
        m_ = mpmath.mpf(mu8) + mpmath.mpf(off)
        exact.append(float(sum(mpmath.mpf(g) / (1 + mpmath.exp((mpmath.mpf(e) - m_)
                                                               / mpmath.mpf(0.01)))
                               for e, g in zip(eps8, deg8)) - 8))
```

For the activated state $N = 8$, $a_{4,0} = 0$, $T = 0.01$ the cell evaluates the count $\sum g f - 8$ at 61 values of μ within $\pm 3 \times 10^{-9}$ of the solver's μ: once directly in double precision (`direct`) and once with 40 digits (`exact`).

```
fig, ax = plt.subplots()
ax.plot(offsets * 1e9, direct, "s-", color=PALETTE[1], ms=4, lw=1.2,
        label="direct count, double precision")
ax.plot(offsets * 1e9, exact, color=PALETTE[0], lw=2.0, label="exact count, 40 digits")
ax.axvline(float(root8 - mu8) * 1e9, color="0.3", ls=":", lw=1.2, label="exact root")
ax.axhline(0.0, color="0.6", lw=0.8)
ax.set_xlabel("$\\mu$ minus the solver's $\\mu$ (units of $10^{-9}\\,m$)")
ax.set_ylabel("$\\sum g f - N$")
ax.set_title("$N = 8$, $a_{4,0} = 0$, $T = 0.01$: the direct count is too coarse")
ax.legend(fontsize=8)
```

Both counts are drawn against the offset in units of 10^{-9}, with the exact root as a dotted vertical line and a zero line.

```
zero_at = [off for off, value in zip(offsets, direct) if value == 0.0]  # direct = 0
if zero_at:  # where the double-precision count says "exactly N particles"
    miss = min(abs(off - float(root8 - mu8)) for off in zero_at)  # nearest such point
    where = ("is zero only on a short interval that misses the true root by at "
             f"least ${miss * 1e10:.0f} \\times 10^{{-10}}\\,m$")
    report("distance of the zeros of the direct count from the true root",
           f"at least {miss:.1e} m")
else:  # (on another computer the rounding steps may fall differently)
    where = "is never exactly zero at the sampled points"
save_figure(fig, "root_conditioning",
            "The particle-number condition $\\sum g f - N$ (vertical axis, of size "
...)
```

`zero_at` collects the sampled values of μ at which the double-precision count is exactly zero; `miss` is the smallest distance of such a point from the true root. The caption of Figure 15d.1 is completed with a sentence that depends on what was found (on another computer the rounding steps may fall differently). Out [4] reports that these zeros miss the true root by at least $3.0 \times 10^{-10}\,m$.

```
worst_balance = 0.0
for (n, a4, t), (mu, eps, deg, _) in states.items():
    root = exact_root(eps, deg, n, t, mu)
    worst_balance = max(worst_balance, float(abs(balance_root(eps, deg, n, t) - root)))
zero_width = (np.sum(np.array(direct) == 0.0)) * (offsets[1] - offsets[0])
report("width of the interval where the double-precision direct count is 0",
       f"{zero_width:.1e} m")
report(f"largest |balance root - 40-digit root| ({len(states)} states)",
       f"{worst_balance:.1e}")
check(worst_balance < 1e-15, "the double-precision balance finds mu to 1e-15",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "thermo_mu_well_conditioned_root")
```

For all 20 states the root of the balance is compared with the 40-digit root, and the width of the interval where the direct count is zero is estimated from the number of sampled zeros times the spacing. Out [4]: the width is $3.0 \times 10^{-10}\,m$, while the balance finds every root to $9.3 \times 10^{-17}\,m$, about three million times better. In Figure 15d.1 the orange squares jump in steps of the rounding unit of N (size 10^{-15}) while the blue line crosses zero at one point.

In [5], the chemical potential against the temperature.

```
T_GRID = np.linspace(0.002, 0.05, 49)


def curves(n, a4):
    _, eps, deg, _ = states[(n, a4, 0.05)]
    rows = []
    for t in T_GRID:
        mu = balance_root(eps, deg, n, t)
        energy, entropy, free, _, cv, _ = functions_of_state(eps, deg, mu, t)
        rows.append((mu, energy, entropy, free, cv))
    return np.array(rows)  # columns: mu, E, S, F, C_V
```

`T_GRID` holds 49 temperatures from 0.002 to 0.05. `curves` takes the levels of the state at $T = 0.05$ (the largest thermal window; at lower temperatures the extra levels are empty far below the rounding) and computes μ, E, S, F and C_V at every temperature of the grid, returned as an array with one row per temperature.

```
GROUND = "Revision/kohn_sham/results/ground/summary.csv"
with open(repository_file(GROUND), newline="", encoding="utf-8") as handle:
    ground = {row["id"]: row for row in csv.DictReader(handle)}
gap8 = float(ground["N8_lam0_a00"]["KS_gap"])
two_level = gap8 / 2.0 - 0.5 * T_GRID * math.log(3.0)
curve8 = {a4: curves(8, a4) for a4 in (0.0, 1.0, 2.0)}
```

The gap $\Delta = 0.4307337$ of `N8_lam0_a00` is read from the record's ground-state table, the two-level formula $\Delta/2 - (T/2)\ln 3$ of Section 15.26 is evaluated on the grid, and the curves of $N = 8$ are computed at three slices.

```
fig, ax = plt.subplots()
for a4 in (0.0, 1.0, 2.0):
    ax.plot(T_GRID, curve8[a4][:, 0], color=SHADES[a4], lw=2.0,
            label=f"$a_{{4,0}} = {a4:.0f}$")
    ax.plot(TEMPS, [float(record[state_id(8, a4, t)]["mu"]) for t in TEMPS], "o",
            color=SHADES[a4], ms=8, markerfacecolor="white")
ax.plot(T_GRID, two_level, "--", color=PALETTE[1], lw=1.5,
        label="$\\Delta/2 - (T/2)\\ln 3$, $a_{4,0} = 0$")
ax.plot([], [], "o", color="0.4", markerfacecolor="white", label="record")
ax.set_xlabel("temperature $T$ (units of $m$)")
ax.set_ylabel("chemical potential $\\mu$ (units of $m$)")
ax.set_title("$N = 8$, $\\lambda = 0$: the chemical potential")
ax.legend(fontsize=8)
save_figure(fig, "chemical_potential",
            "The chemical potential $\\mu$ (vertical axis, units of $m$) of the free "
...)
```

$\mu(T)$ at the three slices (`curve8[a4][:, 0]` is the first column of the array), the record's values as open circles, and the two-level formula dashed; then Figure 15d.2.

```
formula = gap8 / 2.0 - 0.005 * math.log(3.0)
recorded = float(record[state_id(8, 0.0, 0.01)]["mu"])
report("mu of N8_lam0_a00_T10: two-level formula / record",
       f"{formula:.12f} / {recorded:.12f}")
check(abs(formula - recorded) < 1e-8, "the two-level formula gives mu to 1e-8",
      record=f"{THERMO}, N8_lam0_a00_T10")
```

The formula at $T = 0.01$ ($T/2 = 0.005$): 0.209873777875 against the record's 0.209873776387 (Out [5]); the check passes. In the figure, μ starts at $\Delta/2$ and falls along the dashed line; at the later slices the gap and with it μ are smaller.

In [6], free energy and entropy.

```
curve136 = {a4: curves(136, a4) for a4 in (0.0, 0.5, 1.0, 1.5, 2.0)}
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
for a4, values in curve136.items():
    e0 = float(ground[f"N136_lam0_a{round(10 * a4):02d}"]["E_KS"])
    left.plot(T_GRID, values[:, 3] - e0, color=SHADES[a4], lw=2.0,
              label=f"$a_{{4,0}} = {a4}$")
    right.plot(T_GRID, values[:, 2], color=SHADES[a4], lw=2.0)
```

The curves of $N = 136$ at the five slices; on the left the free energy measured from the ground-state energy E_0 of the same slice (column 3 of the curves minus `e0`), on the right the entropy (column 2).

```
    if a4 in (0.0, 1.0, 2.0):
        ids = [state_id(136, a4, t) for t in TEMPS]
```

```
        left.plot(TEMPS, [float(record[i]["F"]) - e0 for i in ids], "o",
                  color=SHADES[a4], ms=7, markerfacecolor="white")
        right.plot(TEMPS, [float(record[i]["entropy"]) for i in ids], "o",
                   color=SHADES[a4], ms=7, markerfacecolor="white")
left.set_xlabel("temperature $T$")
left.set_ylabel("$F - E_0$ (units of $m$)")
left.set_title("Free energy, $N = 136$, $\\lambda = 0$")
left.legend(fontsize=8)
right.set_xlabel("temperature $T$")
right.set_ylabel("entropy $S$")
right.set_title("Entropy (circles: record)")
save_figure(fig, "free_energy_entropy",
            "Left: the free energy measured from the ground-state energy, "
...)
```

Where the record has thermal states (slices 0, 1, 2), its values are drawn as open circles; then the labels and Figure 15d.3.

```
_, eps, deg, _ = states[(136, 1.0, 0.05)]
worst_fs = 0.0
for t in T_GRID:
    h = 1e-3 * t
    f_plus = functions_of_state(eps, deg, balance_root(eps, deg, 136, t + h), t + h)[2]
    f_minus = functions_of_state(eps, deg, balance_root(eps, deg, 136, t - h), t - h)[2]
    s = functions_of_state(eps, deg, balance_root(eps, deg, 136, t), t)[1]
    worst_fs = max(worst_fs, abs(-(f_plus - f_minus) / (2 * h) - s) / s)
```

The identity $-dF/dT = S$ of Section 15.26 is tested for $N = 136$ at $a_{4,0} = 1$ at every temperature of the grid: the central difference $[F(T+h) - F(T-h)]/(2h)$ with $h = 10^{-3}T$ (each F at its own μ; index 2 of the returned tuple is F, index 1 is S) is compared with S.

```
report("largest relative |-dF/dT - S|, N = 136, a4,0 = 1", f"{worst_fs:.1e}")
check(worst_fs < 1e-4, "-dF/dT = S along the whole temperature grid",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "thermo_entropy_identity")
rising = all(bool(np.all(np.diff(v[:, 2]) > 0)) and bool(np.all(np.diff(v[:, 3]) < 0))
             for v in curve136.values())
check(rising, "S rises and F falls with T at every slice")
```

Out [6]: the largest relative deviation is 7.5×10^{-6}; and at every slice S rises and F falls with T at every step of the grid. In Figure 15d.3 the later slices (darker) have the larger entropy at the same temperature: the history crowds the levels together, so the same temperature excites more particles.

In [7], the heat capacity.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0), layout="constrained")
for a4, values in curve8.items():
    left.plot(T_GRID, values[:, 4], color=SHADES[a4], lw=2.0,
              label=f"$a_{{4,0}} = {a4:.0f}$")
    left.plot(TEMPS, [float(record[state_id(8, a4, t)]["C_V"]) for t in TEMPS], "o",
```

```
            color=SHADES[a4], ms=7, markerfacecolor="white")
for a4, values in curve136.items():
    right.plot(T_GRID, values[:, 4], color=SHADES[a4], lw=2.0,
               label=f"$a_{{4,0}} = {a4}$")
    if a4 in (0.0, 1.0, 2.0):
        right.plot(TEMPS, [float(record[state_id(136, a4, t)]["C_V"]) for t in TEMPS],
                   "o", color=SHADES[a4], ms=7, markerfacecolor="white")
```

The closed-form C_V (column 4 of the curves) of $N = 8$ at three slices on the left and of $N = 136$ at five slices on the right, with the record's values as circles.

```
for ax, n in ((left, 8), (right, 136)):
    ax.set_yscale("log")
    ax.set_xlabel("temperature $T$ (units of $m$)")
    ax.set_title(f"Heat capacity, $N = {n}$, $\\lambda = 0$")
    ax.legend(fontsize=8)
left.set_ylabel("$C_V$ (pure number)")
save_figure(fig, "heat_capacity",
            "The heat capacity $C_V = dE/dT$ (vertical axis, logarithmic, pure "
...)
positive = all(bool(np.all(v[:, 4] > 0)) for v in list(curve8.values())
               + list(curve136.values()))
check(positive, "C_V > 0 at every temperature and slice")
```

Both panels get a logarithmic axis, labels and a legend; Figure 15d.4; and the check that C_V is positive everywhere (the closed form is a weighted variance of the d_i divided by T^2, so it cannot be negative). In the figure the gas $N = 8$ at $a_{4,0} = 0$ is **activated**: its heat capacity is tiny below $T \approx 0.02$, because each excitation costs the gap $0.43\,m$ and is suppressed like $e^{-\Delta/(2T)}$; as the history closes the gap, the heat capacity wakes up at ever lower temperatures. $N = 136$, with its much smaller gap, has a large heat capacity already at $T = 0.01$.

In [8], the occupations.

```
fig, ax = plt.subplots()
sums = []
for colour, t in zip(PALETTE, TEMPS):
    mu, eps, deg, levels = states[(136, 1.0, t)]
    e = np.array(eps)
    f = 1.0 / (1.0 + np.exp(np.minimum((e - mu) / t, 700.0)))
    sums.append(float(np.sum(np.array(deg) * f)))
    order = np.argsort(e)
    ax.plot(e[order], f[order], "o-", color=colour, ms=4, lw=1.0, label=f"$T = {t}$")
    ax.axvline(mu, color=colour, ls=":", lw=1.2)
```

For $N = 136$ at $a_{4,0} = 1$ and the three temperatures the occupation f of every level is computed (`np.minimum(..., 700.0)` caps the exponent, because e^{710} would overflow), the sum $\sum g f$ is kept, and f is drawn against the level, sorted by energy, with μ as a dotted vertical line.

```
ax.set_xlim(0.2, 0.5)
ax.set_xlabel("level $\\varepsilon$ (units of $m$)")
```

```
ax.set_ylabel("occupation $f$")
ax.set_title("$N = 136$, $a_{4,0} = 1$: Fermi-Dirac occupations (dotted: $\\mu$)")
ax.legend()
save_figure(fig, "occupations",
            "The occupation $f$ of the levels of the free state $N = 136$ at "
...)
report("sum g f at the three temperatures", ", ".join(f"{s:.12f}" for s in sums))
check(max(abs(s - 136.0) for s in sums) < 1e-9, "the occupations add up to N = 136",
      record="Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "thermo_N_conservation")
```

The horizontal range is limited to the levels near the Fermi level (0.2 to 0.5); Figure 15d.5; Out [8]: the sums are 136.000000000000 at all three temperatures. In the figure the step from full to empty widens with T.

In [9], the sea holes.

```
def shell_sizes(largest):
    reach = math.isqrt(largest) + 1
    sizes = {}
    for x in range(-reach, reach + 1):
        for y in range(-reach, reach + 1):
            for z in range(-reach, reach + 1):
                s = x * x + y * y + z * z
                if s <= largest:
                    sizes[s] = sizes.get(s, 0) + 1
    return sizes
```

`shell_sizes` counts $r_3(n_2)$ for all n_2 up to `largest` by brute force: it visits every vector of whole numbers in a cube large enough (`math.isqrt` is the whole-number square root) and adds one to the count of its $n_2 = x^2 + y^2 + z^2$.

```
worst_sea, mine = 0.0, {}
for a4 in (0.0, 1.0, 2.0):
    mu, _, _, levels = states[(8, a4, 0.05)]
    band = {lv[0]: lv[4] for lv in levels
            if lv[1] == 1 and lv[2] == "even" and lv[3] == 0 and lv[0] >= 1}
    sizes = shell_sizes(max(lv[0] for lv in levels))
    holes = sum(4 * sizes[n2] / (1.0 + math.exp((mu + band[n2]) / 0.05))
                for n2 in sizes if n2 >= 1)
    mine[a4] = holes / 8.0
    recorded = float(record[state_id(8, a4, 0.05)]["sea_holes_excluded"])
    worst_sea = max(worst_sea, abs(holes / recorded - 1.0))
```

For $N = 8$ at $T = 0.05$ and three slices: `band` maps each shell $n_2 \geq 1$ to its band level ($j = +1$, even, label 0; `lv[4]` is the energy in the full level list), the shell sizes are counted up to the largest shell of the level set, and the number of sea holes is $\sum 4 r_3(n_2)/(1 + e^{(\mu+\varepsilon_{band})/T})$ over these shells (Section 15.26). It is divided by 8 for the holes per particle and compared with the record's column `sea_holes_excluded`.

```
fig, ax = plt.subplots()
plotted = []  # every value drawn, to measure their range
for marker, n in (("o", 8), ("s", 136)):
    for colour, t in zip(PALETTE, TEMPS):
        values = [float(record[state_id(n, a, t)]["sea_holes_over_N"])
                  for a in (0.0, 0.5, 1.0, 1.5, 2.0)]
        plotted += values
        ax.plot([0, 0.5, 1, 1.5, 2], np.maximum(values, 1e-60), marker + "-",
                color=colour, ms=6, lw=1.2,
                markerfacecolor=colour if n == 8 else "white",
                label=f"$N = {n}$, $T = {t}$")
```

The record's holes per particle for $N = 8$ (filled circles) and $N = 136$ (open squares), the three temperatures and the five slices, joined by lines; values below 10^{-60} would be drawn at 10^{-60} (the smallest recorded value is about 2×10^{-56}, so none is clipped).

```
ax.plot(list(mine), list(mine.values()), "x", color="black", ms=11, mew=2,
        label="this notebook, $N = 8$, $T = 0.05$")
ax.axhline(0.01, color="0.3", ls=":", lw=1.2, label="1 percent")
ax.set_yscale("log")
ax.set_ylim(1e-60, 1e4)  # the smallest recorded value is about 2e-56
ax.set_xlabel("slice $a_{4,0}$")
ax.set_ylabel("sea holes per particle")
ax.set_title("Diagnostic of the filling convention ($\\lambda = 0$)")
ax.legend(fontsize=7, ncol=2, loc="lower right")
smallest = min(v for v in plotted if v > 0.0)  # the smallest nonzero value
powers = int(math.log10(max(plotted) / smallest))  # whole powers of ten spanned
save_figure(fig, "sea_holes",
            "The number of thermal holes that the excluded sea brane band would "
...)
```

This notebook's three values as black crosses (`list(mine)` is the list of the slices, the keys of the dictionary), the 1 per cent line, the axes, and the number of whole powers of ten between the smallest and the largest value, which the caption of Figure 15d.6 prints.

```
report("smallest and largest sea holes per particle drawn",
       f"{smallest:.2e}, {max(plotted):.2e}")
report("sea holes per particle, N = 8, T = 0.05, a4,0 = 0, 1, 2",
       ", ".join(f"{v:.4g}" for v in mine.values()))
check(worst_sea < 1e-9, "the sea-hole diagnostic reproduced",
      record=f"{THERMO}, column sea_holes_excluded")
```

Out [9]: the values drawn span from 2.10×10^{-56} to 30.8 holes per particle (57 powers of ten); for $N = 8$ at $T = 0.05$ they are 1.464×10^{-5}, 0.09746 and 30.8 at the slices 0, 1, 2; the record is reproduced. At $a_{4,0} = 2$ the excluded sea would carry about 31 holes for each particle: there the CONVENTION is outside its range of validity.

In [10], the interaction at a temperature.

```
fig, ax = plt.subplots()
shifts = []
```

```
by_series = {}  # (N, tag) -> the three values of F(lambda) - F(0)
for colour, n in zip(PALETTE, (8, 136, 688)):
    for tag, style in (("lamp1", "-"), ("lamm1", "--")):
        values = [float(record[state_id(n, 0.0, t, tag)]["F"])
                  - float(record[state_id(n, 0.0, t)]["F"]) for t in TEMPS]
        shifts += values
        by_series[(n, tag)] = values
        sign = "+" if tag == "lamp1" else "-"
        ax.plot(TEMPS, values, style, marker="o", color=colour, ms=6, lw=1.5,
                label=f"$N = {n}$, ${sign}\\lambda_1$")
```

From the record alone: the change $F(\lambda) - F(0)$ of the free energy for $\pm\lambda_1$, $N = 8, 136, 688$, at $a_{4,0} = 0$ and the three temperatures, drawn solid for $+\lambda_1$ and dashed for $-\lambda_1$, and kept in `shifts` and, by series, in `by_series`.

```
ax.set_yscale("symlog", linthresh=1e-4)
ax.axhline(0.0, color="0.5", lw=0.8)
ax.set_xlabel("temperature $T$ (units of $m$)")
ax.set_ylabel("$F(\\lambda) - F(0)$ (units of $m$)")
ax.set_title("Effect of the interaction on the free energy, $a_{4,0} = 0$")
ax.legend(fontsize=8, ncol=2)
up = by_series[(688, "lamp1")]  # N = 688, +lambda_1, at T = 0.01, 0.02, 0.05
save_figure(fig, "interaction_free_energy",
            "The change of the free energy caused by the couplings $+\\lambda_1$ "
...)
```

A symmetric logarithmic axis (linear between -10^{-4} and 10^{-4}), the zero line, labels, and Figure 15d.7, whose caption prints the values of `up`, 0.0033 and $0.0045\,m$ at $T = 0.01$ and 0.05.

```
report("largest |F(lambda) - F(0)| at a4,0 = 0", f"{max(abs(s) for s in shifts):.4f}")
check(max(abs(s) for s in shifts) < 0.005,
      "the interaction changes F by less than 0.005 at a4,0 = 0",
      record=f"{THERMO}, column F")
raises = all(v > 0.0 for n in (136, 688) for v in by_series[(n, "lamp1")]) and all(
    v < 0.0 for n in (136, 688) for v in by_series[(n, "lamm1")])
lowers8 = all(v < 0.0 for v in by_series[(8, "lamp1")]) and all(
    v > 0.0 for v in by_series[(8, "lamm1")])
check(raises and lowers8,
      "repulsion raises F for N = 136 and 688 and lowers it for N = 8",
      record=f"{THERMO}, column F")
```

Out [10]: the largest change is $0.0046\,m$; repulsion raises F for $N = 136$ and 688 and lowers it for $N = 8$, whose zero modes have no scalar density without interaction, so that to first order only the exchange term $-\frac{1}{32}\lambda n^2$ acts (Section 15.5 gives the exact symmetry at $T = 0$).

In [11], the last check.

```
NAMES = ["root_conditioning", "chemical_potential", "free_energy_entropy",
         "heat_capacity", "occupations", "sea_holes", "interaction_free_energy"]
missing = [name for number, name in enumerate(NAMES, start=1)
```

```
        if not output_file(f"{FIGURE_FOLDER}/15d_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

As in Notebook 15e (Section 15.9, In [15]), with the seven figure names of this notebook in order and the file names `15d_<number>_<name>.png`: the names whose file does not exist are collected, the check requires that list to be empty, and `all_checks_passed()` prints the last line, ALL 15 CHECKS PASSED (notebook 15d).

15.31 What we proved, what we computed, what we assumed

PROVED (exact statements; derived line by line in this chapter or in Chapter 14, and verified by sympy in `Revision/kohn_sham/reports/ks-theory-python.json` and by WolframScript in `Revision/kohn_sham/reports/ks-theory-wolfram.json`):

statement	where	record check
the real block equation, its boundary conditions, self-adjointness, real levels	Section 15.2	`block_hamiltonian`, `bc_brane_parity_conditions`, `bc_self_adjoint_boundary_term`
the Pruefer phase increases strictly and runs over all values: every level has its own label, none is missed	Section 15.3	derived here; the solver's check `free_particle_branch_labels`
the exact levels at $k = 0$ and the zero mode $\sqrt{2M/(1 - e^{-2ML})}\, e^{My}$	Section 15.3	`bc_exact_k0_spectra`
the Hermite midpoint formula and the Anderson weights	Sections 15.3, 15.5	derived here
the brane-band slope $c = 2/(1 + e^{-3}) = 1.9051482536$ (times $e^{-a_{4,0}}$)	Section 15.4	`brane_band_slope`
the rescaling identity: a later slice is the first slice with redshifted momenta	Section 15.4	`rescaling_identity`
$E_{KS} = \sum gf\varepsilon - 2\,\mathrm{Vol}_7 \int e^{6Hy} e_{int}\, dy$ (double counting)	Section 15.5	`ks_onshell_lagrangian`
$E_{KS}(-\lambda) = -E_{KS}(\lambda)$ for the mapped zero-mode state of $N = 8$	Section 15.5	derived here
the energy-momentum tensor of the states	Section 15.16	`emt_orbital_components`, `emt_trace_identity`
the conservation law $(e^{6Hy} p_0)' = 3H\, e^{6Hy}(p_3 + p_t)$	Section 15.16	`emt_y_conservation_selfconsistent`
the energy-change law $dE/da_4 = -3 \cdot 2\,\mathrm{Vol}_7 \int e^{6Hy}(p_3 - p_t)\, dy$	Section 15.16	`emt_x4_component`

statement	where	record check
the Hellmann-Feynman theorem and the first-order estimate Q_{nm}	Section 15.21	`adiabatic_hellmann_feynman`, `adiabatic_offdiagonal_identity`
the closed form of C_V, $C_V = T\,dS/dT$, $-dF/dT = S$, $\Omega = F - \mu N$, the balance form of $\sum gf = N$	Section 15.26, Exercise 5	derived here

COMPUTED (numerical, with the record file and the measured agreement):

- The canonical matrix: 75 ground states and 135 thermal states, every one of the 42 checks of `Revision/kohn_sham/reports/ks-rust-solver.json` PASS; a repeat run byte-identical and a refined run within tolerances fixed in advance (`Revision/kohn_sham/reports/ks-rust-determinism.json`, 14 checks PASS); every chemical potential within its rounding bound of a 40-digit root (`Revision/kohn_sham/reports/ks-rust-mermin-roots.json`). Notebook 15a reran the matrix and found every compared number equal to the record (Out [5], Out [6]).

- The method by hand (Notebook 15e): the exact $k = 0$ levels to 7.2×10^{-10}, the solver's levels to 8×10^{-15}, the RK4 error ratio 16.00, the band slope at three slices, and the interacting $N = 8$ states to 10^{-14}.

- Along the history: the $k = 0$ levels do not move, the band levels and all gaps fall (the gap of $N = 8$: 0.4307337, 0.1703493, 0.0641594 at $a_{4,0} = 0, 1, 2$), the energy of the free gas falls ($N = 136$: 80.28 to 12.45), and the integrated ratio $\int p_3 / \int \rho$ rises toward 1/3 (0.2966 to 0.3264).

- Delta-SCF equals the gap without interaction (2.0×10^{-11}); the orbital relaxation is at most $5.5 \times 10^{-4}\,m$; the self-consistent loops converge in at most 17 iterations.

- The conservation law and the energy-change law hold numerically in every state (solver checks `emt_y_conservation_pointwise`, `emt_y_conservation_integrated` and `emt_energy_change_dE_da4`; Notebook 15b).

- $Q_{max} \leq 0.0935$ over the matrix (transition probability below 0.009), no Fermi-level crossing in the matrix, and the demonstration $N = 696$ where the instantaneous ground state is wrong by 3.66 to $8.62\,m$ (Notebook 15c).

- The thermodynamics of the free states from the levels alone (Notebook 15d), and the sea-hole diagnostic: more than 1 per cent of N in 15 of the 135 thermal states, at most $30.98\,N$.

- The dependence on the cutoff from $L = 3$ to 6 of 29 recorded states (`Revision/kohn_sham/tip_convergence/tip-convergence.json`, 7 of 7 checks PASS; Section 15.2): the free results with $k \neq 0$ converge faster than exponentially, the levels at $k = 0$ only like $1/L^2$, and the interacting $N = 8$ energy ($\lambda = \pm\lambda_1$) goes from $\mp 9.868 \times 10^{-4}$ at $L = 3$ to $\mp 3.0046 \times 10^{-3}$ at large L. These are numbers of the record; the notebooks of this chapter compute at $L = 3$ only.

- The five notebooks print 88 PASS lines together (25, 18, 12, 15 and 18).

ASSUMED:

- the good sector (no dependence on the extra times) and the Z2 mirror brane at $y = 0$;
- CHOSEN: the regular tip $b(-L) = 0$ at the cutoff $L = 3$. Its effect is COMPUTED in the record (`Revision/kohn_sham/tip_convergence/`, $L = 3$ to 6, 7 of 7 checks PASS; Section 15.2, the paragraph on the cutoff): the recorded free results, except $N = 688$ at $a_{4,0} = 0$, lie within 1.4 per cent (E_{KS}) and 12 per cent (the integral of p_0) of their large-L values; the interaction effects do not, because the proper zero-mode density grows toward the tip like $e^{(6H-2m)|y|}$: the recorded $N = 8$ interaction energy $\mp 9.868 \times 10^{-4}$ ($\lambda = \pm\lambda_1$) is about one third of its large-L value $\mp 3.0046 \times 10^{-3}$, the interaction shift of $N = 136$ at $a_{4,0} = 2$ is about 21 times its $L = 3$ value at large L ($-\lambda_1$) and about 28 times at $L = 5.5$ ($+\lambda_1$, limit not established), and the self-consistent iteration fails at some larger L for 13 of the 19 interacting states studied. The couplings (calibrated at $L = 3$; the same rule at larger L drives λ_1 to zero) and $N = 688$ (bulk edge at $L = 3$) are themselves choices at $L = 3$;
- CONVENTION: particles fill the positive branch and the zero modes, the sea is never occupied, not even thermally; its justification is OPEN, and the sea-hole diagnostic shows where it fails;
- PRESCRIBED BACKGROUND: the history $a_4 = AHx_4$ with $A = 1$; the gas is a test field on it and cannot be its source (it violates the conditions C1, C2, C3 of `Revision/field_equations_a4/reports/ks-source-conditions.json`);
- the functional: Hartree plus the exact local exchange of the uniform gas, no correlation; the instantaneous (adiabatic) states, supported but not proved by $Q_{max} \ll 1$.

HYPOTHESIS: none.

OPEN: the time-dependent (non-adiabatic) evolution of the gas; a thermal treatment of the sea; the justification of the filling convention; the limit $L \to \infty$ wherever the record does not establish it ($N = 688$ at $a_{4,0} = 0$, the 7 interacting states of Section 15.2 whose limit is not established, whether self-consistent states exist where the iteration fails, and every state not computed at $L \neq 3$); what drives the history, since this gas cannot (Chapter 22 lists these problems). Nothing in this chapter concerns the creation of universes, pairs of universes or the matter-antimatter asymmetry.

15.32 Exercises

Exercise 1 (closed shells). (a) Show that $r_3(3) = 8$ and $r_3(5) = 24$. (b) Starting with the eight zero modes and filling the brane band shell by shell, compute the closed shells up to $n_2 = 5$. (c) Which of them is nearest to 688/4, and why is it the record's N_{mid}?

Answer. (a) $n_1^2 + n_2^2 + n_3^2 = 3$ only with three squares equal to 1, so $(\pm 1, \pm 1, \pm 1)$: $2^3 = 8$ vectors. For 5 the squares must be $4 + 1 + 0$: the number 2 can stand in any of the 3 places, the number 1 in any of the 2 remaining places, and each has two signs, so $3 \cdot 2 \cdot 2 \cdot 2 = 24$ vectors. (b) With $g = 4r_3(n_2)$ and $r_3(1), \ldots, r_3(5) = 6, 12, 8, 6, 24$: $8 + 24 = 32$; $32 + 48 = 80$; $80 + 32 = 112$; $112 + 24 = 136$; $136 + 96 = 232$. Rule: add $4r_3(n_2)$ for each new shell. (c) $688/4 = 172$; the distances are $|136 - 172| = 36$ and $|232 - 172| = 60$, so 136 is nearest, the rule of `Revision/kohn_sham/results/parameters.json` (particleNumbers.rule).

Exercise 2 (why the gap shrinks more slowly than $e^{-a_{4,0}}$). Let $s(k) = \varepsilon_{band}(k)/k$ and assume that s decreases as k grows. (a) Use the rescaling identity to show that $\Delta_{KS}(a_{4,0})/\Delta_{KS}(0) > e^{-a_{4,0}}$ for $N = 8$ and $a_{4,0} > 0$. (b) Compute s at the momenta $0.25\,e^{-a_{4,0}}$, $a_{4,0} = 0, 1, 2$, from the gaps 0.4307337, 0.1703493, 0.0641594 (Notebook 15a, Out [7]), and compare with $c = 1.9051482536$.

Answer. (a) Section 15.4: $\Delta_{KS}(a_{4,0}) = \varepsilon_{band}(k_a)$ with $k_a = 0.25\,e^{-a_{4,0}}$, so

$$\frac{\Delta_{KS}(a_{4,0})}{\Delta_{KS}(0)} = \frac{k_a\, s(k_a)}{0.25\, s(0.25)} = e^{-a_{4,0}}\,\frac{s(k_a)}{s(0.25)} > e^{-a_{4,0}}.$$

Rule: write each band level as $k\,s(k)$; $k_a/0.25 = e^{-a_{4,0}}$; and $s(k_a) > s(0.25)$ because $k_a < 0.25$ and s decreases. (b) $s(0.25) = 0.4307337/0.25 = 1.722935$; $s(0.0919699) = 0.1703493/0.0919699 = 1.852229$; $s(0.0338338) = 0.0641594/0.0338338 = 1.896310$. Rule: divide each gap by its momentum ($0.25\,e^{-1} = 0.0919699$, $0.25\,e^{-2} = 0.0338338$). The values grow as k shrinks and approach $c = 1.905148$, the slope at $k = 0$: the assumption holds on these points, and the ratios $1.852229/1.722935 = 1.0750$ and $1.896310/1.722935 = 1.1006$ are exactly the factors by which the measured ratios 0.39549 and 0.14895 exceed $e^{-1} = 0.36788$ and $e^{-2} = 0.13534$.

Exercise 3 (counting levels with the phase). For the free block $j = +1$ at $k = 0$ use the exact levels of Section 15.3 ($M = 1$, $L = 3$): the even levels $\sqrt{1 + (n\pi/3)^2}$ and the odd levels 1.2922928, 2.0106286 (Notebook 15e, Out [6]). (a) How many levels lie strictly between $\varepsilon = 0$ and $\varepsilon = 2$? (b) Between which two multiples of $\pi/2$ does $\Phi(2)$ lie? (c) Why is $\Phi(0) = 0$ exactly?

Answer. (a) Even: $n = 1$ gives $\sqrt{1 + 1.0966} = 1.4479719 < 2$, $n = 2$ gives $\sqrt{1 + 4.3865} = 2.3208815 > 2$. Odd: $1.2922928 < 2 < 2.0106286$. So two levels: the odd label 0 (target $\pi/2$) and the even label 1 (target π). (b) Φ increases strictly, has passed the targets $\pi/2$ (at 1.2923) and π (at 1.4480), but not yet $3\pi/2$ (the odd label 1, at 2.0106); so $\pi < \Phi(2) < 3\pi/2$, that is $1 < \Phi(2)/\pi < 1.5$, as Figure 15e.1 shows. (c) At $\varepsilon = 0$, $k = 0$, $v = 0$ the angle equation is $\theta' = -M \sin 2\theta$ with $\theta(-L) = 0$; the constant $\theta = 0$ solves it, and a solution of such an equation is fixed by its start value, so $\theta(0) = 0$. (This is the zero mode, $b = 0$; Notebook 15e checks that the computed value is exactly 0.0, Out [3].)

Exercise 4 (Anderson weights for two residuals). (a) For $p = 2$ solve the bordered system of Section 15.5 and show $c_1 = (G_{22} - G_{12})/(G_{11} - 2G_{12} + G_{22})$, $c_2 = 1 - c_1$. (b) Evaluate it for the residuals $r_1 = (2, 1)$ and $r_2 = (1, -1)$, and compare $|c_1 r_1 + c_2 r_2|^2$ with $|r_1|^2$ and $|r_2|^2$.

Answer. (a) With $c_2 = 1 - c_1$, $F = |c_1 r_1 + (1 - c_1) r_2|^2 = c_1^2 G_{11} + 2c_1(1 - c_1) G_{12} + (1 - c_1)^2 G_{22}$. Rule: expand the square with the dot products $G_{ik} = r_i \cdot r_k$. Then $dF/dc_1 = 2c_1 G_{11} + 2(1 - 2c_1) G_{12} - 2(1 - c_1) G_{22} = 0$. Rule: differentiate each term. Collecting c_1: $c_1(G_{11} - 2G_{12} + G_{22}) = G_{22} - G_{12}$, which is the claim (the same as eliminating ν from the bordered system). (b) $G_{11} = 4 + 1 = 5$, $G_{22} = 1 + 1 = 2$, $G_{12} = 2 - 1 = 1$, so $c_1 = (2 - 1)/(5 - 2 + 2) = 0.2$ and $c_2 = 0.8$. The combined residual is $0.2\,(2, 1) + 0.8\,(1, -1) = (1.2, -0.6)$ with $|\cdot|^2 = 1.44 + 0.36 = 1.8$, smaller than $|r_2|^2 = 2$ and $|r_1|^2 = 5$; check: $F(c_1) = 5c_1^2 - 2c_1 + 2$ has its minimum 1.8 at $c_1 = 0.2$.

Exercise 5 (the grand potential). For fixed levels and without interaction show that, level by level, $\varepsilon f - T S_1 - \mu f = -T \ln(1 + e^{-x})$, where $S_1 = -[f \ln f + (1 - f) \ln(1 - f)]$ and $x = (\varepsilon - \mu)/T$; conclude $\Omega = E - TS - \mu N$.

Answer. Line by line. $1 - f = f(-x) = 1/(1 + e^{-x})$, so $\ln(1 - f) = -\ln(1 + e^{-x})$. Rule: the identity $1 - f(x) = f(-x)$ of Section 15.26. Next, $\ln f - \ln(1 - f) = \ln \frac{f}{1-f} = -x$. Rule: $\frac{1-f}{f} = e^{x}$. Hence $S_1 = -f[\ln f - \ln(1 - f)] - \ln(1 - f) = fx - \ln(1 - f)$. Rule: write $f \ln f +$

$(1-f)\ln(1-f) = f[\ln f - \ln(1-f)] + \ln(1-f)$. Then $TS_1 = (\varepsilon-\mu)f - T\ln(1-f)$, because $Tx = \varepsilon - \mu$, and $\varepsilon f - TS_1 - \mu f = T\ln(1-f) = -T\ln(1+e^{-x})$. Summing with the degeneracies: $E - TS - \mu N = -T\sum g\ln(1+e^{-x}) = \Omega$, with $E = \sum gf\varepsilon$ and $N = \sum gf$. With interaction both sides contain the same term $-2\,\mathrm{Vol}_7 \int e^{6Hy} e_{int}\, dy$ (Section 15.26); the solver checks the equality in all 135 states (check thermo_grand_potential_two_forms, worst relative difference 1.7×10^{-14}).

Exercise 6 (the energy along the history from its slope). Notebook 15b, Out [7], gives for $N = 136$ without interaction $E(0) = 80.28222$, $E(0.5) = 51.24519$ and the slopes $dE/da_4 = -71.439756$ at $a_{4,0} = 0$ and -46.470842 at $a_{4,0} = 0.5$. (a) Estimate $E(0.5)$ from $E(0)$ with Euler's rule (one step with the slope at the start) and with the trapezoid rule (the mean of the two slopes). (b) Explain the signs of the errors.

Answer. (a) Euler: $80.28222 + 0.5 \cdot (-71.439756) = 80.28222 - 35.71988 = 44.56234$, too low by 6.68. Trapezoid: $80.28222 + 0.5 \cdot \frac{1}{2}(-71.439756 - 46.470842) = 80.28222 - 29.47765 = 50.80457$, too low by 0.44. Rule: $E(a+h) \approx E(a) + h\,E'(a)$ (Euler) and $E(a+h) \approx E(a) + \frac{h}{2}[E'(a) + E'(a+h)]$ (trapezoid), Chapter 2. (b) The slope becomes less negative along the history (the curve of Figure 15b.4 is convex, it bends upward). Euler uses the steepest slope, that of the start, for the whole step and so overestimates the drop. The trapezoid rule averages the two end slopes; but the slope rises fast at first and then more slowly, so between the two ends it lies above the straight line that joins its end values, and the average of the two end slopes is more negative than the true average slope: the drop is again overestimated, by much less. Notebook 15b uses Simpson's rule on 17 slices instead and obtains $E(2) - E(0)$ to 8×10^{-7} relative (Out [8]).

Exercise 7 (adiabaticity). (a) From Notebook 15c, Out [5], compute Q and the transition probability for $N = 136$ at $a_{4,0} = 0$ (matrix element 0.358861, energy difference 2.052930) and at $a_{4,0} = 2$ (matrix element 0.089223, energy difference 1.524416). (b) Why is $Q = 0$ exactly for $N = 8$? (c) Use the two-level formula of Section 15.26 to estimate μ of $N = 8$ at $a_{4,0} = 0$ and $T = 0.02$; the record has 0.2043728 (`Revision/kohn_sham/results/thermo/thermodynamics.csv`, `N8_lam0_a00_T20`). Why is the agreement worse than at $T = 0.01$?

Answer. (a) $Q = 0.358861/2.052930^2 = 0.358861/4.214522 = 0.085149$, probability $Q^2 = 0.00725$; at $a_{4,0} = 2$: $Q = 0.089223/2.323844 = 0.038395$, probability 0.00147. Rule: $Q = AH|\langle n|\partial_a h|m\rangle|/(\varepsilon_n - \varepsilon_m)^2$ with $AH = 1$ (Section 15.21). Both are far below 1: the gas follows its instantaneous states within the sectors. (b) For $N = 8$ only the zero modes at $k = 0$ are occupied, so transitions could start only in the sectors with $k = 0$, where the free $\partial_a h = -j\kappa k\sigma_3$ vanishes. With interaction the occupied orbitals are still the $k = 0$ orbitals, which do not feel the slice at all (the slice enters only through κk), so the self-consistent potentials do not change with a_4 either, and $\partial_a h = 0$ on these sectors: every matrix element vanishes. (c) $\mu = 0.4307337/2 - 0.01\ln 3 = 0.2153668 - 0.0109861 = 0.2043807$, against the record's 0.2043728: a difference of 8×10^{-6}, against 1.5×10^{-9} at $T = 0.01$. Rule: the formula keeps only the first band shell and uses $f(x) \approx e^{-x}$; at the higher temperature the next band shells (at 0.588 and above) are less suppressed, and the neglected terms grow like $e^{-(\varepsilon-\mu)/T}$, so the estimate is less accurate.

16 Checking the solution independently: the reference solver and the cross-check

Chapter 15 solved the Kohn-Sham equations of the field dirac16complex in the author's primordial universe with one program, the Revision Rust solver, and read physics from its numbers: levels,

energies, densities, pressures, heat capacities, all along the history in which ordinary 3-space inflates and the three extra times x_5, x_6, x_7 deflate exponentially. This chapter asks the question that every such calculation must answer before its numbers may be called results: how do we know that they are right? The Revision record answers it with a second program that shares no code with the first and solves the same equations by a different method (the **reference solver**), and with a third program that compares the two, number by number, with a rule for the allowed difference that was written down before any comparison was made (the **cross-check**). Three complete notebooks, 16a, 16b and 16c, repeat the essential steps and reproduce the numbers of the record.

16.1 What this chapter does

Why checking is a chapter of its own. A number printed by a program is not yet a result. Between the equations and the printed number stand a grid that replaces the continuous hidden direction by finitely many points, numbers in the computer that carry only about sixteen significant digits, thousands of lines of code in which a single wrong sign could hide, and the choices of the model itself. The first three can be tested by solving the same equations a second time in a completely different way: if two independent programs agree to many digits, within errors that each of them has measured, it is very unlikely that both are wrong in the same way. The fourth, the model, cannot be tested this way, because both programs share it; Section 16.25 says precisely what that means. The cross-check of the record did not only confirm the Rust solver: its first run failed, and the failure exposed a rounding error in the computation of the chemical potential that no test of either program alone had seen. This chapter tells that story too (Sections 16.20 to 16.24).

What is done, in order.

- Why a second program is needed and what agreement between two programs can and cannot show (Section 16.2).
- The two solvers side by side: what each one computes and how (Section 16.3).
- How the error of a grid calculation is measured: the order of convergence and the ratio test (Section 16.4), Richardson extrapolation and the uncertainty of the reference solver (Section 16.5), the canonical and refined runs that measure the uncertainty of the Rust solver (Section 16.6).
- The tolerance rule of the cross-check, derived line by line, and what the full cross-check of the record found (Sections 16.7 and 16.8).
- Notebook 16a, which runs the reference solver on five states, runs the Rust solver twice for each of them, applies the tolerance rule to 5140 comparisons and reproduces the worst cases of the record (Sections 16.9 to 16.12).
- Inside the reference solver: the exact levels of the free problem, the staggered grid, the matrix, counting eigenvalues, the exact discrete zero mode and the slope of the brane band (Sections 16.13 to 16.15), with Notebook 16b, which rebuilds the method by hand (Sections 16.16 to 16.19).
- How the cross-check caught a rounding error in the chemical potential, why sixteen digits were not enough, and how the computation was repaired (Section 16.20), with Notebook 16c (Sections 16.21 to 16.24).

- What the cross-check does not test, the closing summary and the exercises (Sections 16.25 to 16.27).

The three notebooks.

- Notebook 16a (needs Rust; 51 checks, 7 figures): the reference solver on a subset of five states, the Rust solver with its canonical and its refined numerics, and the tolerance rule on 5140 comparisons.
- Notebook 16b (no Rust; 31 checks, 7 figures): the method of the reference solver by hand on the free problem: the exact levels, the staggered grid, the matrix, Sturm counts and bisection, the zero mode, Richardson extrapolation, the brane-band slope.
- Notebook 16c (no Rust; 26 checks, 6 figures): the chemical potential with 40 digits, the rounding staircase of the direct sum, the faulty value of the first cross-check reproduced, the repair, and the rounding bounds of 45 states.

Notebook 16a is placed first because it shows the whole check at work; 16b then opens the reference solver and 16c the chemical potential.

The status of every statement. Every statement of this chapter carries one of the labels of Chapter 0.

- PROVED: the formulas of the method, each derived line by line in this chapter: the ratio test, Richardson extrapolation and why its uncertainty over-estimates the error, the factor 16/15 of the Rust uncertainty, the tolerance rule as a consequence of the triangle inequality, the exact free levels (in the record: check bc_exact_k0_spectra of `Revision/kohn_sham/reports/ks-theory-python.json`), the staggered matrix and its symmetry, the pivots of the Sturm count, the exact discrete zero mode, the Hellmann-Feynman slope of the brane band (check brane_band_slope of the same report), the slope $dN/d\mu$ of the Mermin condition, Newton's method, the well-conditioned rewriting of the Mermin condition and the first-order rounding bounds. Two facts of linear algebra are quoted, not proved: Sylvester's law of inertia and the spectral theorem for symmetric matrices.
- COMPUTED: every number of the comparisons. The reference solver's own checks are in `Revision/kohn_sham/reports/ks-reference.json` (37 checks, all PASS), the cross-check in `Revision/kohn_sham/reports/ks-crosscheck.json` (31 checks, all PASS, 162691 comparisons) and its table `Revision/kohn_sham/reports/ks-crosscheck-table.csv`; the notebooks reproduce them, with the measured differences given with each number.
- ASSUMED: that the grid errors have the assumed form (even powers of the cell width for the reference, fourth order for the Rust solver). This is not proved for the self-consistent problem; it is tested by the ratio test on every level and by a fourth grid for one state (Sections 16.4 and 16.5). Also ASSUMED, as in Chapters 14 and 15: the good sector and the Z2 mirror brane at $y = 0$; CHOSEN: the regular tip at $y = -L$ with $L = 3$; CONVENTION, justification OPEN: which levels count as particles.
- PRESCRIBED BACKGROUND: the history $a_4 = AHx_4$ with $A = 1$ (the Kohn-Sham states violate the conditions that the a_4 field equations put on their source: `Revision/field_equations_a4/reports/ks-source-conditions.json`).

- OPEN: the time-dependent (non-adiabatic) problem. HYPOTHESIS: none is used in this chapter.

What this chapter does not touch. Nothing in this chapter concerns pairs of universes, their creation, or matter and antimatter. The cross-check compares two programs that solve the same Kohn-Sham equations; it says nothing about whether any universe is created, in pairs or otherwise. The pairing theorems and their exact scope are the subject of Chapters 18 to 21.

Notation and units. The author's coordinates are $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates with the scale factor $e^{a_4} \sin^{1/6} z$; x_4 is the time; x_5, x_6, x_7 are the three extra times, which deflate exponentially with the scale factor $e^{-a_4} \sin^{1/6} z$; x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$. The hidden coordinate is $y = \ln(\sin z)/(6H)$; it runs from the **tip** $y = -L$ to the **brane** $y = 0$. A **slice** is one instant of the history, and $a_{4,0}$ is the value of a_4 there. In all numbers $H = 1$ and $m = 1$, so energies and temperatures are in units of m and lengths in units of $1/H$. A state of the record is named like N136_lamp2_a20: $N = 136$ quanta, the coupling $+\lambda_2$ (the tags lam0, lamp1, lamm1, lamp2, lamm2 stand for $\lambda = 0, +\lambda_1, -\lambda_1, +\lambda_2, -\lambda_2$), the slice $a_{4,0} = 2.0$ (the two digits are ten times the slice); a thermal state ends in T10, T20 or T50 for the temperatures $T = 0.01$, 0.02, 0.05.

16.2 Why a second program, and what agreement can show

Four kinds of error. Suppose a program prints a number x for a quantity whose exact value is X. The **error** of x is the difference $x - X$. It has four sources.

- The **discretisation error**: the program replaces the continuous interval $-L \le y \le 0$ by finitely many points, so it solves a nearby problem, not the exact one. This error shrinks when the points are made denser.
- The **rounding error**: the computer stores each number with about sixteen significant digits (Section 16.20 explains exactly how), so almost every operation makes a tiny error, and long sums can make larger ones.
- **Mistakes in the program**: a wrong sign, a wrong index, a forgotten term. They do not shrink when the grid is refined.
- **Errors of the model or of its inputs**: a wrong formula in the theory, a wrong boundary condition, a wrong constant. The program then solves the wrong equations correctly.

What a second program tests. The record's reference solver is a second program written for this purpose. It shares no code with the Rust solver; it is written in another language (Python instead of Rust); it uses a different numerical method (a matrix on a grid instead of step-by-step integration, Section 16.3); and its only physics input is the theory file `Revision/kohn_sham/ks-theory.jso n`, which the Rust solver reads as well. Discretisation errors of two different methods are different, rounding errors take different paths, and a mistake in one program is very unlikely to be repeated, with the same size, in the other. So if the two programs agree within their measured errors, then, with high confidence, neither has a discretisation, rounding or programming error larger than those errors. But an error of the fourth kind is shared: both programs read the same theory file and use the same boundary conditions, so they would agree even if the model were wrong. The cross-check

tests the numerics, not the physics model. It also compares the two programs at the same tip cutoff, $L = 3$ (CHOSEN): agreement tests how accurately both programs solve the equations with $L = 3$, not how the results change when L grows. That dependence is measured separately, in `Revision/kohn_sham/tip_convergence/`, where, for two of the recorded states, the reference solver agrees with the Rust solver at $L = 3$ and at $L = 4$ in all 28 comparisons (Section 16.25).

Uncertainty, tolerance, ratio. Neither program knows its own error exactly (otherwise it could subtract it). What each can do is measure an **uncertainty**: a number U that, for reasons given in Sections 16.5 and 16.6, should be at least as large as the size $|x - X|$ of its error. The **tolerance** is the largest difference between the two programs that the comparison allows; Section 16.7 derives it from the two uncertainties. The **ratio** of a comparison is the difference of the two programs divided by the tolerance; the comparison passes when the ratio is at most 1.

Why the tolerance must be fixed in advance. A tolerance that is chosen after looking at the differences can always be made large enough for everything to pass, and then the comparison tests nothing. The record therefore fixes the tolerance rule in the cross-check program before any comparison and never adjusts it to a result (`Revision/kohn_sham/checker/crosscheck_ks.py`, its header, and the README of `Revision/kohn_sham/checker`). The rule was tested by its first use: it failed in one state, the tolerance was not widened, and the cause was found and repaired (Section 16.20).

16.3 The two solvers

The equations both solve. Both programs solve the Kohn-Sham problem of Chapters 14 and 15. At a slice $a_{4,0}$ an orbital of block type $j = \pm 1$ with 3-momentum of length k is a pair of real functions $a(y)$, $b(y)$ that obey

$$a' = Ma - \big(\kappa k + j(\varepsilon - v)\big)\, b, \qquad b' = \big(j(\varepsilon - v) - \kappa k\big)\, a - Mb, \qquad \kappa = e^{-Hy - a_{4,0}},$$

where the prime is d/dy, ε is the level, $M(y) = m + \frac{15}{16}\lambda S(y)$ the effective mass and $v(y) = -\frac{1}{16}\lambda n(y)$ the potential (`Revision/kohn_sham/ks-theory.json`, `blockEquation.realForm` and `exchange.kohnShamPotentials`). The boundary conditions are the regular tip $b(-L) = 0$ (CHOSEN) and the ASSUMED Z2 brane: even orbitals have $b(0) = 0$, odd orbitals $a(0) = 0$. The densities n and S are sums over the occupied orbitals, so the potentials depend on the solution, and the problem is solved again and again until it reproduces itself (self-consistency). At a temperature $T > 0$ the occupations are Fermi-Dirac numbers $f = 1/(1 + e^{(\varepsilon-\mu)/T})$, and the chemical potential μ is fixed by $\sum g f = N$. The list of states, the **canonical matrix**, holds 75 ground states (three particle numbers, five couplings, five slices) and 135 thermal states (three particle numbers, three couplings, five slices, three temperatures).

The Rust solver (Chapter 15). For a trial energy it integrates the two equations from the tip to the brane with the Runge-Kutta rule RK4 in $G = 900$ steps, counts levels with the Pruefer angle, and finds each level by Newton's method inside a bracket to a tolerance of 10^{-13}; self-consistency is reached with Anderson mixing to a residual of 10^{-11} (`Revision/kohn_sham/results/parameters.json`, numerics).

The reference solver (`Revision/kohn_sham/reference/run_reference.py` with the module `ks_fd.py` in the same folder). It places the two functions of an orbital on a **staggered grid** of

G cells: a at the centres of the cells and b at their ends. The differential equations then become the eigenvalue problem of a symmetric **tridiagonal matrix** with $2G - 1$ rows (a matrix whose only nonzero entries are on the main diagonal and its two neighbours). It finds each eigenvalue by its number, by counting how many eigenvalues lie below a trial value (the **Sturm count**) and halving an interval (**bisection**), so that no level can be missed; it then sharpens each eigenvalue and computes its eigenvector. Every quantity is computed on three grids, $G = 300$, 600 and 1200, and the three results are combined by **Richardson extrapolation** (Section 16.5) into a value R with a measured uncertainty U. Self-consistency is reached with Anderson mixing to a residual of 10^{-12}. The reference re-derives from their stated rules even the inputs it could have copied: the particle numbers 136 and 688 and the couplings λ_1, λ_2 (for $N = 8$: 0.01946 and 0.05838; `Revision/kohn_sham/reports/ks-reference.json`, check `parameters_calibration`).

Side by side.

	Rust solver	**reference solver**
language	Rust	Python with numpy
method	shooting with RK4 from the tip to the brane	staggered finite differences, one matrix per sector
finding a level	Pruefer count and Newton's method	Sturm count and bisection
grid	$G = 900$ steps	$G = 300$, 600, 1200 cells
error of one grid	proportional to h^4	proportional to h^2
uncertainty	canonical minus refined run (Section 16.6)	Richardson extrapolation (Section 16.5)
self-consistency	residual at most 10^{-11}	residual at most 10^{-12}
results	`Revision/kohn_sham/results`	`Revision/kohn_sham/reference/results`

What the reference computes for each state. A ground-state **job** (one unit of work of the program) solves the state, its Delta-SCF excited state (one quantum moved from the highest occupied to the lowest empty group of levels), four neighbouring slices $a_{4,0} \pm 0.002$ and $a_{4,0} \pm 0.004$ at fixed occupations (for the derivative dE/da_4 and the adiabaticity measure of Chapter 15), and the list of the lowest particle-hole excitations, each on the three grids. A thermal job solves the state and four neighbouring temperatures $T(1 \pm 0.01)$, $T(1 \pm 0.02)$ (for the heat capacity). The program also solves an exact-exchange variant of the 60 states with $\lambda \neq 0$, the rescaling partners of the 60 states with $a_{4,0} > 0$, a demonstration of a level crossing, and one state on a fourth grid (Section 16.5). Its report `Revision/kohn_sham/reports/ks-reference.json` records 37 checks of the reference on itself, all PASS.

16.4 Order of convergence: the ratio test

The error of a grid calculation. Let a grid have cells of width h ($h = L/G$ for G cells), and let $x(h)$ be a number computed on it whose exact value is X. For the staggered grid of the reference the error has an expansion in even powers of h (Section 16.14 shows where the even powers come from):

$$x(h) = X + c\,h^2 + d\,h^4 + e\,h^6 + \ldots,$$

where the numbers c, d, e do not depend on h. Such a method is said to be of **second order** (or to have **order of convergence** 2), because for small h the first term $c\,h^2$ dominates: halving h divides the error by about $2^2 = 4$. The expansion is ASSUMED for the self-consistent problem; the ratio test below checks it on the computed numbers themselves.

The ratio test, line by line. Compute x on three grids with the cell widths h, $h/2$ and $h/4$ (for the reference $G = 300$, 600, 1200, that is $h = 0.01$, 0.005, 0.0025).

$$x(h) - x(h/2) = c\,h^2 - c\,\tfrac{h^2}{4} + d\,h^4 - d\,\tfrac{h^4}{16} + \cdots = \tfrac{3}{4}\,c\,h^2 + \tfrac{15}{16}\,d\,h^4 + \ldots$$

Rule: subtract the expansion with $h/2$ in place of h from the expansion with h; X cancels, and $(h/2)^2 = h^2/4$, $(h/2)^4 = h^4/16$.

$$x(h/2) - x(h/4) = \tfrac{3}{4}\,c\,\tfrac{h^2}{4} + \tfrac{15}{16}\,d\,\tfrac{h^4}{16} + \ldots$$

Rule: the same subtraction with $h/2$ in place of h, so every h^2 gets a factor $\frac{1}{4}$ and every h^4 a factor $\frac{1}{16}$.

$$\frac{x(h) - x(h/2)}{x(h/2) - x(h/4)} = \frac{\frac{3}{4}c\,h^2\,(1 + \frac{5}{4}\frac{d}{c}h^2 + \ldots)}{\frac{3}{16}c\,h^2\,(1 + \frac{5}{16}\frac{d}{c}h^2 + \ldots)} = 4\left(1 + \tfrac{15}{16}\,\tfrac{d}{c}\,h^2 + \ldots\right).$$

Rule: take $\frac{3}{4}ch^2$ and $\frac{3}{16}ch^2$ out of the two brackets ($\frac{15}{16}/\frac{3}{4} = \frac{5}{4}$ and $\frac{15}{256}/\frac{3}{16} = \frac{5}{16}$), then divide, using $1/(1+s) = 1 - s + \ldots$ for small s, so that $\frac{5}{4} - \frac{5}{16} = \frac{15}{16}$.

So the **convergence ratio** $(x(h) - x(h/2))/(x(h/2) - x(h/4))$ is 4 up to a correction of order h^2. A ratio close to 4 shows that the grids are fine enough for the h^2 term to dominate (the **asymptotic regime**), which is what Richardson extrapolation needs. The test uses only computed numbers, not the unknown X.

An example from the record. Notebook 16a prints the Kohn-Sham energy of the state N136_lamp2_a20 on the three grids: $E_{KS}(300) = 12.449082943357$, $E_{KS}(600) = 12.449020346521$, $E_{KS}(1200) = 12.449004697554$. The two differences are 6.2597×10^{-5} and 1.5649×10^{-5}, and their ratio is 4.00006: second order, in the asymptotic regime. The reference applies the test to every level of every ground state and requires the median ratio of each state to lie within 0.05 of 4; the worst state is within 7.4×10^{-5} of 4 (`Revision/kohn_sham/reports/ks-reference.json`, check `richardson_asymptotic_ratio`).

16.5 Richardson extrapolation and the uncertainty of the reference

The idea. If we know the form of the error, we can combine results on several grids so that the leading error terms cancel. This is **Richardson extrapolation**, named after the meteorologist Lewis Fry Richardson, who used it in 1911.

One step, line by line. Start from the expansion of Section 16.4 on the grids h and $h/2$.

$$x(h/2) = X + c\,\tfrac{h^2}{4} + d\,\tfrac{h^4}{16} + \ldots$$

Rule: put $h/2$ in place of h in the expansion.

$$4\,x(h/2) - x(h) = 3X + (1-1)\,c\,h^2 + (\tfrac{4}{16} - 1)\,d\,h^4 + \cdots = 3X - \tfrac{3}{4}\,d\,h^4 + \ldots$$

Rule: multiply the line above by 4 and subtract the expansion for h; the $c\,h^2$ terms cancel because $4 \cdot \frac{1}{4} = 1$.

$$r(h) = \frac{4\,x(h/2) - x(h)}{3} = X - \tfrac{1}{4}\,d\,h^4 + \ldots$$

Rule: divide by 3. The **one-step value** $r(h)$ has no h^2 error left; its error starts with h^4.

The second step, line by line. The same step on the finer pair $h/2$, $h/4$ gives

$$r(h/2) = \frac{4\,x(h/4) - x(h/2)}{3} = X - \tfrac{1}{4}\,d\,\tfrac{h^4}{16} + \cdots = X - \tfrac{1}{64}\,d\,h^4 + \ldots$$

Rule: the line before, with $h/2$ in place of h, so h^4 becomes $h^4/16$.

$$16\,r(h/2) - r(h) = 15X + (\,-\tfrac{16}{64} + \tfrac{1}{4})\,d\,h^4 + \cdots = 15X + \ldots$$

Rule: multiply by 16 and subtract $r(h)$; the h^4 terms cancel because $16 \cdot \frac{1}{64} = \frac{1}{4}$. What remains are terms of order h^6.

$$R = \frac{16\,r(h/2) - r(h)}{15} = X + O(h^6).$$

Rule: divide by 15. The notation $O(h^6)$ means terms that are at most a constant times h^6.

The same value in one line. Insert the two one-step values:

$$16\,r(h/2) - r(h) = \frac{64\,x(h/4) - 16\,x(h/2)}{3} - \frac{4\,x(h/2) - x(h)}{3} = \frac{64\,x(h/4) - 20\,x(h/2) + x(h)}{3}.$$

Rule: $16 \cdot \frac{4x(h/4) - x(h/2)}{3}$ multiplied out, then the two fractions with the same denominator subtracted, collecting $-16 - 4 = -20$ for $x(h/2)$.

$$R = \frac{64\,x(h/4) - 20\,x(h/2) + x(h)}{45}.$$

Rule: divide by 15, and $3 \cdot 15 = 45$. The three weights add up to $(64 - 20 + 1)/45 = 1$, as they must: if all three grids gave the same number, R would be that number.

The uncertainty. The reference states as the uncertainty of R

$$U = \bigl|R - r(h/2)\bigr| + 2 \cdot 10^{-12}\max(1, |R|).$$

Why this is a safe estimate, line by line:

$$R - r(h/2) = \Big(X + O(h^6)\Big) - \Big(X - \tfrac{1}{64} d\, h^4 + O(h^6)\Big) = \tfrac{1}{64}\, d\, h^4 + O(h^6).$$

Rule: subtract the two expansions derived above; X cancels.

So the first term of U is the size of the h^4 error of the finest one-step value $r(h/2)$, while R itself has only an h^6 error. When h is small, h^6 is much smaller than h^4, so U is larger than the error of R: it over-estimates it. The second term, $2 \cdot 10^{-12}$ times the size of R (at least 1), is a **floor** for the rounding errors of the computer, which do not shrink with h.

An example. For N136_lamp2_a20 the one-step values are $r(h) = 12.448999480909$ and $r(h/2) = 12.448999481232$, the three-grid value is $R = 12.448999481253$, and $U = 4.6 \times 10^{-11}$ (Notebook 16a, In [6]; the record has $R = 12.44899948125322$ with $U = 4.6 \times 10^{-11}$, `Revision/kohn_sham/reference/README.md`). The single grid $G = 1200$ is 5.2×10^{-6} away from R; two Richardson steps have gained five significant digits.

Testing the uncertainty on a fourth grid. The argument above rests on the ASSUMED form of the error. The record tests it. One state, N136_lamp2_a20 (the strongest coupling, with the largest densities at the tip), is solved on a fourth grid as well, $G = 2400$. From $(300, 600, 1200)$ the three-grid value R_{123} with its uncertainty U_{123} is formed, and from $(600, 1200, 2400)$ the three-grid value R_{234}. The error of R_{234} is of order $(h/2)^6 = h^6/64$, so it is about 64 times smaller than the error of R_{123}; hence $|R_{123} - R_{234}|$ is practically the error of R_{123}, and the test is whether it is at most U_{123}. For all 19 classes of quantities (energies, levels, integrals, values at the tip and at the brane, five profiles with 151 points each) it is: the largest ratio $|R_{123} - R_{234}|/U_{123}$ is 0.787, for the value of the pressure p_0 at the tip (`Revision/kohn_sham/reports/ks-reference.json`, check `richardson_uncertainty_validated`). Notebook 16a repeats this test (In [7], Figure 16a.2). The uncertainty is therefore an honest estimate, and it is not hugely larger than the error either, which a useful estimate must also be.

16.6 The uncertainty of the Rust solver: canonical and refined runs

The idea. The Rust solver uses the Runge-Kutta rule RK4, whose error over the whole interval is proportional to h^4 for the step width $h = L/G$ (Chapter 2). If the step is halved, the error is divided by $2^4 = 16$. The record therefore runs every state twice: with the **canonical numerics** of the committed results and with the **refined numerics**, which halve the step and tighten the other tolerances. The canonical numerics are $G = 900$ steps, a root tolerance of 10^{-13} and a self-consistency tolerance of 10^{-11}; the refined numerics are $G = 1800$, 10^{-14} and 10^{-12} (`Revision/kohn_sham/checker/measure_rust_refinement.py`, which writes `Revision/kohn_sham/checker/rust-refinement.json`).

The factor 16/15, line by line. Let e be the error of the canonical result x_c, so $x_c = X + e$, and let the refined result be x_r.

$$x_r = X + \tfrac{e}{16}.$$

Rule: halving the step of a fourth-order method divides the error by $2^4 = 16$ (ASSUMED error model; the other tolerances are tightened so that they do not spoil it).

$$x_c - x_r = e - \tfrac{e}{16} = \tfrac{15}{16}\, e.$$

Rule: subtract the two lines; X cancels.

$$e = \tfrac{16}{15}\,(x_c - x_r), \qquad U_{Rust} = \tfrac{16}{15}\,|x_c - x_r|.$$

Rule: multiply by $\frac{16}{15}$; the size of the canonical error is the **Rust uncertainty**.

So the difference between the two runs, which can be measured, gives the error of the canonical run, which cannot. For example, Notebook 16a (In [13]) measures $|x_c - x_r| = 7.529 \times 10^{-10}$ for the Kohn-Sham energy of N688_lam0_a00, so $U_{Rust} = \frac{16}{15} \cdot 7.529 \times 10^{-10} = 8.03 \times 10^{-10}$.

Two conditions. First, the measured difference applies to the committed results only if the canonical run made for the measurement reproduces them. The record checks this for every state: the canonical runs reproduce the committed matrix with every difference zero (`Revision/kohn_sham/reports/ks-crosscheck.json`, check `rust_refinement_applies_to_matrix`). Second, some quantities (the Delta-SCF energy, the adiabaticity measure, the finite-difference dE/da_4, the heat capacity) are not printed by the single runs of the measurement. For them the cross-check uses the largest canonical-minus-refined difference over the whole canonical matrix, taken from the Rust solver's determinism report; the values are listed in `Revision/kohn_sham/reports/ks-crosscheck.json` under `rust_matrix_wide_uncertainties` (for example 2.135×10^{-9} for the levels and 5.689×10^{-9}, relative, for the heat capacity).

16.7 The tolerance rule, fixed in advance

The rule. For every number that both programs compute, the cross-check requires

$$|x_{Rust} - x_{ref}| \le 3\,(U_{ref} + U_{Rust}) + 10^{-12} \cdot s,$$

where U_{ref} is the Richardson uncertainty of the reference, U_{Rust} the measured uncertainty of the Rust solver, and the **scale** s is $\max(1, |x_{ref}|)$ for a single number and the largest value of the profile for a point of a profile. The right side is the **tolerance**, and the left side divided by the right side is the **ratio**.

Why the two uncertainties are added, line by line. Let X be the exact value.

$$x_{Rust} - x_{ref} = (x_{Rust} - X) - (x_{ref} - X).$$

Rule: add and subtract X.

$$|x_{Rust} - x_{ref}| \le |x_{Rust} - X| + |x_{ref} - X|.$$

Rule: the **triangle inequality** $|p - q| \le |p| + |q|$ for any two numbers p, q.

$$|x_{Rust} - x_{ref}| \le U_{Rust} + U_{ref}.$$

Rule: each uncertainty is at least the size of its program's error (that is what an uncertainty is meant to be).

So if both uncertainties are honest, the difference can never exceed their sum. The rule allows three times the sum, because the uncertainties are estimates, not proven bounds (Sections 16.5 and 16.6 explain the assumptions behind them): the factor 3 keeps a comparison from failing merely because an estimate was a little too small, while a real mistake, which usually produces differences many times the uncertainties, still fails. The last term, 10^{-12} times the scale, is a floor for rounding, which neither uncertainty measures.

Propagated uncertainties, line by line. Some compared quantities are built from others, and their uncertainties are added up the same way. The free energy is $F = E - TS$, so

$$F_{Rust} - F = (E_{Rust} - E) - T\,(S_{Rust} - S), \qquad U_F = U_E + T\,U_S.$$

Rule: subtract the exact $F = E - TS$ from the computed one, then apply the triangle inequality as above. In the same way the grand potential $\Omega = F - \mu N$ gets $U_\Omega = U_F + N\,U_\mu$. The derivative of the energy along the history has the energy-momentum form $dE/da_4 = -6\,\mathrm{Vol}_7 \int e^{6Hy}(p_3 - p_t)\,dy = -3\,(I_3 - I_t)$, where I_3 and I_t are the integrals $2\,\mathrm{Vol}_7 \int e^{6Hy} p_3\,dy$ and $2\,\mathrm{Vol}_7 \int e^{6Hy} p_t\,dy$ that the solvers report; hence its uncertainty is $3\,(U_{I_3} + U_{I_t})$. The exchange difference $\Delta E_x = \frac{\lambda}{32}\,2\,\mathrm{Vol}_7 \int e^{6Hy} Q^2\,dy$ is quadratic in the profile Q, and a relative change δ of Q changes Q^2 by the relative amount $(1+\delta)^2 - 1 = 2\delta + \delta^2 \approx 2\delta$; hence its uncertainty is twice the relative uncertainty of Q times $|\Delta E_x|$.

A floor for difference quotients. The heat capacity is also computed as dE/dT, a **difference quotient**: energies at the temperatures $T \pm dT$ (and $T \pm 2dT$) are subtracted and divided by multiples of $dT = 0.01\,T$. Each energy is a sum over N occupied states of levels that are computed only to a tolerance (the Rust solver finds each level to its root tolerance 10^{-13}; the reference stops its self-consistency at 10^{-12}), so each energy carries a small noise of up to about N times that tolerance, and dividing by dT magnifies the noise to about $N \times \text{tolerance}/dT$. The cross-check adds this floor for each solver: $N \cdot 10^{-13}/dT$ for the Rust solver and $N \cdot 10^{-12}/dT$ for the reference (in its own report each solver uses the same floor for its identity $C_V = dE/dT$). For $N = 8$ and $T = 0.05$, where $dT = 5 \times 10^{-4}$, the two floors are 1.6×10^{-9} and 1.6×10^{-8}.

A worked comparison. Notebook 16a (In [14]) shows every piece of one comparison, the Kohn-Sham energy of N8_lamm2_a00. Rust gives 0.002862652173608389 and the reference 0.002862652173734779, a difference of 1.264×10^{-13}. The uncertainties are $U_{ref} = 2.096 \times 10^{-12}$ and $U_{Rust} = 1.214 \times 10^{-13}$. The tolerance is

$$3\,(2.096 + 0.1214) \times 10^{-12} + 10^{-12} \cdot 1 = 6.652 \times 10^{-12} + 1.000 \times 10^{-12} = 7.652 \times 10^{-12},$$

Rule: insert the numbers; the scale is $\max(1, 0.00286) = 1$. The ratio is $1.264 \times 10^{-13} / 7.652 \times 10^{-12} = 0.0165$: the two programs agree to less than a sixtieth of what the rule allows. (The notebook works with the unrounded uncertainties, $U_{ref} = 2.0955 \times 10^{-12}$ and $U_{Rust} = 1.2145 \times 10^{-13}$, and therefore prints the tolerance 7.651×10^{-12}; the last digit differs only because the four-digit inputs above are rounded. The ratio is 0.0165 either way.)

16.8 What the full cross-check found

The cross-check program `Revision/kohn_sham/checker/crosscheck_ks.py` compares the full canonical matrix. Its report `Revision/kohn_sham/reports/ks-crosscheck.json` holds 31 checks, all PASS, with 162691 comparisons in all (run of 2026-10-08). Twenty of the checks are classes of comparisons; for each, the table gives the number of comparisons and the largest ratio found (from the details of the checks in that report).

check	what is compared	comparisons	largest ratio
`ground_occupations_and_groups`	occupied levels, occupations, HOMO and LUMO groups	300	identical
`ground_energies`	E_{KS}, E_{band}, E_{int}	225	0.148
`ground_homo_lumo_gap`	HOMO, LUMO, Kohn-Sham gap	225	0.001
`ground_eigenvalues`	every level present in both label sets	9616	0.324
`excited_delta_scf`	Delta-SCF energy, excited energy	150	0.296
`excited_particle_hole_lists`	particle-hole excitations	4842	0.001
`emt_integrals`	the five energy-momentum integrals	375	0.225
`emt_brane_tip_values`	ρ, p_3, p_t, p_0 at the brane and the tip	600	0.495
`ground_profiles`	ten profiles at 151 points	113250	0.495
`exchange_delta_E_x`	the exchange difference ΔE_x	75	0.044
`adiabatic_dE_da4`	dE/da_4 in two forms	150	0.122
`adiabatic_Q_max`	the adiabaticity measure and its pair	225	0.314
`exx_variant_scf`	the exact-exchange variant	240	0.128
`rescaling_partners`	the rescaling partners	180	0.137
`crossing_demonstration`	the level-crossing demonstration	31	0.111
`thermo_state_functions`	μ, E, S, F, Ω in two forms	810	0.284
`thermo_derivatives`	C_V, dE/dT, $-dF/dT$	405	0.226
`thermo_sea_hole_diagnostic`	the sea-hole diagnostic of the filling convention	270	0.048
`thermo_levels`	every level of every thermal state, label by label	34378	0.312
`thermo_mu_high_precision`	μ recomputed with 40 digits, and Ω with it	270	0.050

How the comparisons are counted. The column adds up to 166617, more than the 162691 comparisons of the report, because the two numbers count slightly different things. The report counts the 5447 rows of the comparison table `ks-crosscheck-table.csv` (one row for each compared single quantity, such as an energy, an integral or a chemical potential, with both values, both uncertainties, the tolerance and the ratio) plus the 9616 level, 113250 profile and 34378 thermal-level comparisons, which are summarised in the report instead of being written as rows: $5447 + 9616 + 113250 + 34378 = 162691$. The column counts, for each class, every test the class makes. This gives two differences. (i) The other 17 classes of the table also make 3934 yes/no tests that write no row of the comparison table: whether the two programs occupy the same levels, find the same HOMO and LUMO groups, assign the same particle-hole excitations with the same multiplicities, and similar flags. (ii) 8 of the 5447 rows belong to two checks that are not in the table: the bulk edge (`parameters_particle_numbers`) and seven coupling strengths (`parameters_couplings`). So the column is $162691 - 8 + 3934 = 166617$.

The other eleven checks are preconditions, consistency tests and diagnostics: the three input reports pass every check (`inputs_all_pass`: at the time of the cross-check the reference report had 37 of 37, the Rust solver report 42 of 42 and the Rust determinism report 14 of 14); the canonical single runs reproduce the committed matrix (`rust_refinement_applies_to_matrix`, and its exact-exchange variant); both solvers solve the same problem and read the same theory file (`problem_definition_identical`); the reference re-derives the same particle numbers and couplings (`parameters_particle_numbers`, `parameters_couplings`); the floating-point chemical potentials lie within their rounding bounds (`thermo_mu_rounding_diagnostic`, Section 16.20); the Rust solver keeps the rounding bound of μ that it states itself (`thermo_mu_rust_stated_bound`, explained below); and the reference's own output is reproducible: the cross-check runs the reference a second time into a fresh folder and finds all 340 result files and the report byte for byte identical (`reference_outputs_lf_only`, `reference_manifest`, `reference_repeat_byte_identical`).

The two newest checks. The cross-check was run again on 2026-10-08 against the current Rust solver, and two checks were added (`Revision/kohn_sham/checker/README.md`, History 3); the 29 former checks gave the same results as before.

- `thermo_levels` compares every level of every thermal state, label by label, exactly as `ground_eigenvalues` does for the ground states: 33973 levels in the 135 thermal states, with the same tolerance rule and with $U_{Rust} = \frac{16}{15}$ times the largest canonical-minus-refined difference over the levels of the state. It also makes three yes/no tests in each of the 135 states: every level with an occupation $f \geq 10^{-12}$ in one program's set is present in the other's; the Rust labels of each sector start where the Rust ground-state files say they start; and the reference's ranks of each sector start at 0. These $3 \cdot 135 = 405$ tests bring the count to $33973 + 405 = 34378$. The largest ratio is 0.312 (N688_lamm1_a00_T50, level 0:+1:odd:2: Rust 2.90931115629902, reference 2.90931115615719, difference 1.418×10^{-10}, tolerance 4.548×10^{-10}).

- `thermo_mu_rust_stated_bound` is not a comparison of the two programs but a test of a promise. For every thermal state the Rust solver writes, next to μ, a bound on the rounding error of its own root, the column `mu_rounding_bound` of `Revision/kohn_sham/results/thermo/thermodynamics.csv` (the bound B_{well} of Section 16.20). The check recomputes the root of $\sum gf = N$ with 40 digits on the Rust levels and requires $|\mu_{Rust} - \text{root}| \leq$ `mu_rounding_bound` in all 135 states (two yes/no tests per state, 270 in all). It must use the exact doubles of the Rust run, not the printed 16-digit numbers, because rounding the printed numbers to 16 digits alone changes μ by up to about 6×10^{-16}, as much as the bound itself.

The largest ratio $|\mu_{Rust} - \text{root}|/\text{bound}$ is 0.149 (N688_lam0_a00_T10: 1.028×10^{-16} against 6.909×10^{-16}); for N8_lamm1_a00_T10, the state of Section 16.20, it is 2.519×10^{-17} against 3.154×10^{-16}.

Notebook 16a (In [2]) reads both checks from the report and prints their sizes.

What the table shows. No comparison of the whole matrix uses even half of its tolerance: the largest ratio is 0.495, for the energy density ρ at the tip of N8_lamm2_a00, where the densities are largest and both programs are least accurate. Many ratios are far smaller. Every one of these comparisons is made at the same tip cutoff $L = 3$; the dependence on L is measured separately (Section 16.25). Status: COMPUTED; the record files are named above, and Notebook 16a reproduces the worst case of six of the classes.

16.9 Example: the reference solver on a subset and the cross-check (Notebook 16a)

Notebook 16a repeats the cross-check for a **subset** of five states: three ground states and two thermal states, four of them chosen because the full cross-check found its largest ratios there, and the fifth (N688_lam0_a00) because it has the most quanta. It runs the reference program of the repository on these states and on the fourth grid, checks that every new number reproduces the committed reference files, shows the convergence and Richardson extrapolation at work, builds and runs the Rust solver with its canonical and its refined numerics, applies the tolerance rule (Section 16.7) to 5140 comparisons, and reproduces 97 rows of the committed comparison table and the worst cases of six classes of the full cross-check. It needs Rust, takes two to six minutes on a fast computer (more when other programs keep it busy) and about ten minutes on a typical laptop, draws seven figures and ends with the line ALL 51 CHECKS PASSED (notebook 16a).

16.10 How to run Notebook 16a

Step 1. What this notebook does and what it needs.

Notebook 16a (Running the reference solver on a subset and the cross-check) is the file `Revision/textbook/notebooks/16a_reference_crosscheck.ipynb` of the repository Dirac_claude. It runs the independent Python reference solver of the repository on five Kohn-Sham states of dirac16complex in the deflating primordial field (three ground states and two thermal states, four of them chosen because the full cross-check found its largest differences there) and on a fourth, finer grid for one state, checks that every new result reproduces the committed reference record, shows how the grids converge and how Richardson extrapolation removes the grid error, builds the Rust solver and runs it with its canonical and its refined numerics to measure its uncertainty, applies the tolerance rule of the cross-check to 5140 comparisons, and reproduces the corresponding rows and worst cases of the committed cross-check report. The raw outputs of the Rust program go into a temporary folder of the operating system (outside the repository), which the notebook deletes as soon as it has read them. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the

program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/kohn_sham/solver/target/release/revision_ks_solver` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses

the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 16a_reference_crosscheck.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 16a_reference_crosscheck.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 16a_reference_crosscheck.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/16a.captions.json`
- `Revision/textbook/figures/16a_1_grid_convergence.png`
- `Revision/textbook/figures/16a_2_uncertainty_validated.png`
- `Revision/textbook/figures/16a_3_eigenvalue_ratios.png`
- `Revision/textbook/figures/16a_4_level_agreement.png`

- `Revision/textbook/figures/16a_5_profile_agreement.png`
- `Revision/textbook/figures/16a_6_uncertainty_budget.png`
- `Revision/textbook/figures/16a_7_ratios_by_class.png`

It changes no other file of the repository except the Rust build folder `target` next to each `Cargo.toml` it builds; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 51 CHECKS PASSED (notebook 16a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/16a_reference_crosscheck.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 16a_reference_crosscheck.ipynb
python -m nbconvert --execute --inplace 16a_reference_crosscheck.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- The cell that runs the reference solver on the three ground states shows the label with the star for a minute or longer: this is normal. The state N136_lamp2_a20 alone takes up to two minutes on a fast computer and up to five minutes on a laptop, because the reference solves 354 levels on three grids, each with its excited state and four neighbouring slices. Wait until the label shows a number.
- A line reports that a re-run is not identical byte for byte, but the PASS line after it appears: this is harmless. Another computer may round the last digit of a few numbers differently; the check allows differences of one part in a billion, far below every uncertainty of the record.
- An AssertionError names one of the comparisons with the record: the notebook prints the numbers just above the error. A large difference means that the reference program, the Rust solver or the record was changed. Get the stored versions back and run the notebook again.

```
git checkout -- Revision/kohn_sham
```

- On Windows the cell that builds the Rust solver stops with the cargo message failed to remove file and Access is denied: the program revision_ks_solver is still running in another window or terminal, and Windows does not let cargo replace a running program. Wait until that run has finished (or close it), then run the cell again.

- The run was stopped (by an error, or with Kernel > Interrupt) while the cell that runs the Rust solver was busy: the raw outputs of the Rust runs may then be left behind in a folder whose name starts with textbook_16a_, inside the folder for temporary files of your operating system (in Python, tempfile.gettempdir() names it). The notebook deletes that folder only when the cell finishes. Delete the leftover folder by hand; every run makes a new one.

16.11 Notebook 16a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/16a_reference_crosscheck.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Two different computer programs solve the same Kohn-Sham equations of the fermion
field dirac16complex in the author's primordial gravitational field: the **Rust
solver** (whose results the previous chapter used) and an independent **Python
reference solver**, which shares no code with it and uses a different numerical
method. The **cross-check** compares the two, number by number, with a tolerance rule
that was fixed before any comparison was made. This notebook does that work itself
for a **subset** of five states:

- it runs the reference program of the repository on three ground states and two
  thermal states and checks that the new results reproduce the committed reference
  record;
- it shows how the reference results converge on its three grids, how **Richardson
  extrapolation** removes the grid error and measures what is left, and it tests that
  measured uncertainty on a fourth, finer grid;
- it builds the Rust solver, runs each state twice (with the **canonical** and with
  the **refined** numerics) and measures the Rust uncertainty from the difference;
- it applies the tolerance rule of the cross-check to 5140 comparisons (energies,
  levels, profiles, energy-momentum integrals, adiabaticity, thermodynamics) and
  reproduces the rows and the worst cases of the committed cross-check report;
- it draws seven figures: grid convergence, the fourth-grid test of the uncertainty,
  the convergence ratios of the levels, the agreement of the levels, the agreement
  of a density profile, the uncertainty budget, and all ratios of the subset by
  class.

Four of the five states were chosen because the full cross-check of all 210 states
found its largest differences in them; the fifth (N688_lam0_a00) has the largest
number of particles. The run takes two to six minutes on a fast computer (more
when other programs keep it busy) and about ten minutes on a typical laptop.
```

Text cell:

```
## 2. How to run this notebook
```

Step 1. What this notebook does and what it needs.

Notebook 16a (Running the reference solver on a subset and the cross-check) is the file ‘Revision/textbook/notebooks/16a_reference_crosscheck.ipynb‘ of the repository Dirac_claude. It runs the independent Python reference solver of the repository on five Kohn-Sham states of dirac16complex in the deflating primordial field (three ground states and two thermal states, four of them chosen because the full cross-check found its largest differences there) and on a fourth, finer grid for one state, checks that every new result reproduces the committed reference record, shows how the grids converge and how Richardson extrapolation removes the grid error, builds the Rust solver and runs it with its canonical and its refined numerics to measure its uncertainty, applies the tolerance rule of the cross-check to 5140 comparisons, and reproduces the corresponding rows and worst cases of the committed cross-check report. The raw outputs of the Rust program go into a temporary folder of the operating system (outside the repository), which the notebook deletes as soon as it has read them. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a

higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
```

```
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘ again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
    curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
    cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
    cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/kohn_sham/solver/target/release/revision_ks_solver‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 16a_reference_crosscheck.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 16a_reference_crosscheck.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 16a_reference_crosscheck.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/16a.captions.json‘
- ‘Revision/textbook/figures/16a_1_grid_convergence.png‘
- ‘Revision/textbook/figures/16a_2_uncertainty_validated.png‘
- ‘Revision/textbook/figures/16a_3_eigenvalue_ratios.png‘
- ‘Revision/textbook/figures/16a_4_level_agreement.png‘
- ‘Revision/textbook/figures/16a_5_profile_agreement.png‘
- ‘Revision/textbook/figures/16a_6_uncertainty_budget.png‘
- ‘Revision/textbook/figures/16a_7_ratios_by_class.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 51 CHECKS PASSED (notebook 16a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/16a_reference_crosscheck.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 16a_reference_crosscheck.ipynb
      python -m nbconvert --execute --inplace 16a_reference_crosscheck.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- The cell that runs the reference solver on the three ground states shows the label with the star for a minute or longer: this is normal. The state N136_lamp2_a20 alone takes up to two minutes on a fast computer and up to five minutes on a laptop, because the reference solves 354 levels on three grids, each with its excited state and four neighbouring slices. Wait until the label shows a number.
- A line reports that a re-run is not identical byte for byte, but the PASS line after it appears: this is harmless. Another computer may round the last digit of a few numbers differently; the check allows differences of one part in a billion, far below every uncertainty of the record.
- An AssertionError names one of the comparisons with the record: the notebook prints the numbers just above the error. A large difference means that the reference program, the Rust solver or the record was changed. Get the stored versions back and run the notebook again.

      git checkout -- Revision/kohn_sham

- On Windows the cell that builds the Rust solver stops with the cargo message failed to remove file and Access is denied: the program revision_ks_solver is still running in another window or terminal, and Windows does not let cargo replace a running program. Wait until that run has finished (or close it), then run the cell again.
- The run was stopped (by an error, or with Kernel > Interrupt) while the cell that runs the Rust solver was busy: the raw outputs of the Rust runs may then be left behind in a folder whose name starts with textbook_16a_, inside the folder for temporary files of your operating system (in Python, tempfile.gettempdir() names it). The notebook deletes that folder only when the cell finishes. Delete the leftover folder by hand; every run makes a new one.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers

‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 16a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 16a (Running the reference solver on a subset and the cross-check) is the file
# Revision/textbook/notebooks/16a_reference_crosscheck.ipynb of the repository
# Dirac_claude. It runs the independent Python reference solver of the repository on five
# Kohn-Sham states of dirac16complex in the deflating primordial field (three ground
# states and two thermal states, four of them chosen because the full cross-check found
# its largest differences there) and on a fourth, finer grid for one state, checks that
# every new result reproduces the committed reference record, shows how the grids
# converge and how Richardson extrapolation removes the grid error, builds the Rust
# solver and runs it with its canonical and its refined numerics to measure its
# uncertainty, applies the tolerance rule of the cross-check to 5140 comparisons, and
# reproduces the corresponding rows and worst cases of the committed cross-check report.
# The raw outputs of the Rust program go into a temporary folder of the operating system
# (outside the repository), which the notebook deletes as soon as it has read them. It
# needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and
# matplotlib; the other packages run Jupyter, the program that shows and runs notebooks.
# It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the
# Rust program revision_ks_solver, which is part of the repository and is built on your
# computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
```

```
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
```

```
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
#
# Windows, macOS and Linux:
#     cargo --version
#
```

```
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
#     cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
#
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
# Revision/kohn_sham/solver/target/release/revision_ks_solver (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 16a_reference_crosscheck.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 16a_reference_crosscheck.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 16a_reference_crosscheck.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/16a.captions.json
# - Revision/textbook/figures/16a_1_grid_convergence.png
```

```
# - Revision/textbook/figures/16a_2_uncertainty_validated.png
# - Revision/textbook/figures/16a_3_eigenvalue_ratios.png
# - Revision/textbook/figures/16a_4_level_agreement.png
# - Revision/textbook/figures/16a_5_profile_agreement.png
# - Revision/textbook/figures/16a_6_uncertainty_budget.png
# - Revision/textbook/figures/16a_7_ratios_by_class.png
#
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds; running it headless or saving it in JupyterLab also rewrites
# the notebook file itself. It does not use the internet while it runs. The files it
# writes are the same files that are stored in the repository (on another computer a
# figure may differ in a few bytes, which is harmless). To get the stored versions back,
# run this command in the repository folder (it also undoes every change you made
# yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 51 CHECKS PASSED (notebook 16a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/16a_reference_crosscheck.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 16a_reference_crosscheck.ipynb
#     python -m nbconvert --execute --inplace 16a_reference_crosscheck.ipynb
```

```
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - The cell that runs the reference solver on the three ground states shows the label
#   with the star for a minute or longer: this is normal. The state N136_lamp2_a20 alone
#   takes up to two minutes on a fast computer and up to five minutes on a laptop,
#   because the reference solves 354 levels on three grids, each with its excited state
#   and four neighbouring slices. Wait until the label shows a number.
# - A line reports that a re-run is not identical byte for byte, but the PASS line after
#   it appears: this is harmless. Another computer may round the last digit of a few
#   numbers differently; the check allows differences of one part in a billion, far below
#   every uncertainty of the record.
# - An AssertionError names one of the comparisons with the record: the notebook prints
#   the numbers just above the error. A large difference means that the reference
#   program, the Rust solver or the record was changed. Get the stored versions back and
#   run the notebook again.
#     git checkout -- Revision/kohn_sham
# - On Windows the cell that builds the Rust solver stops with the cargo message failed
#   to remove file and Access is denied: the program revision_ks_solver is still running
#   in another window or terminal, and Windows does not let cargo replace a running
#   program. Wait until that run has finished (or close it), then run the cell again.
# - The run was stopped (by an error, or with Kernel > Interrupt) while the cell that
#   runs the Rust solver was busy: the raw outputs of the Rust runs may then be left
#   behind in a folder whose name starts with textbook_16a_, inside the folder for
#   temporary files of your operating system (in Python, tempfile.gettempdir() names it).
#   The notebook deletes that folder only when the cell finishes. Delete the leftover
#   folder by hand; every run makes a new one.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
```

```
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "16a"  # this notebook: chapter 16, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
```

```
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
```

```
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 16a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Solver**: a program that computes the solution of equations. The **Rust solver**
  integrates the orbital equation step by step from one end of the hidden coordinate
  to the other (*shooting*). The **reference solver** writes the same equation as one
  large matrix on a grid and finds its eigenvalues.
- **Grid**: equally spaced points on the interval $-L \le y \le 0$. $G$ is the number
  of cells, $h = L/G$ the width of one cell. The reference uses $G = 300$, $600$ and
  $1200$; one state is also solved on $G = 2400$ (the **fourth grid**).
- **Grid error**: the difference between a number computed on a grid and the exact
  number. For the reference it shrinks like $h^2$.
- **Richardson extrapolation**: a combination of the results on several grids in
  which the leading grid errors cancel. $R$ is the extrapolated value, $U$ its
  measured **uncertainty** (how far it may still be from the exact value).
- **Validation**: a test of a stated uncertainty against a more accurate number.
- **Canonical numerics** of the Rust solver: $G = 900$ RK4 steps and its standard
  tolerances. **Refined numerics**: $G = 1800$ steps and ten times smaller
  tolerances. $U_{Rust}$ is the uncertainty of the canonical Rust result measured
  from the two.
- **Tolerance**: the largest difference allowed between the two solvers.
  **Ratio**: the difference divided by the tolerance; a comparison passes when the
```

```
  ratio is at most 1.
- **State id**: a name such as `N136_lamp2_a20`: $N = 136$ particles, coupling
  $+\lambda_2$ (`lam0`: $\lambda = 0$; `lamp1`, `lamm1`: $\pm\lambda_1$; `lamp2`,
  `lamm2`: $\pm\lambda_2$), slice $a_{4,0} = 2.0$ (the digits are ten times
  $a_{4,0}$). A thermal state ends with `_T10`, `_T20` or `_T50`: the temperature
  $T = 0.01$, $0.02$ or $0.05$ (in units of the mass $m$).
- **Level key** `n2:j:parity:rank`: the shell $n_2 = |\vec n|^2$ of the 3-space
  momentum, the block type $j = \pm 1$, the brane parity (even or odd), and the
  **rank**, the position of the level among the particle levels of its sector
  (0 = the lowest). The Rust solver numbers levels by a **label**; the rank is the
  label minus the label of the lowest particle level (`label_min`).
- **Profile**: a density as a function of $y$, stored at the 151 points
  $y = -3 + 0.02\,i$. **Brane value** and **tip value**: its values at $y = 0$ and
  $y = -3$.
- **JSON** and **CSV**: two text formats for stored numbers (JSON: named entries in
  braces; CSV: a table, one line per row, entries separated by commas).
- **Record**: a committed Revision file whose numbers this notebook reproduces.
  **Byte for byte identical**: two files that agree in every character.
- $E_{KS}$: the Kohn-Sham energy; $E_{band} = \sum g f \varepsilon$ the sum of the
  occupied levels; $E_{int}$ the interaction energy; $E_{KS} = E_{band} - E_{int}$.
  **HOMO**, **LUMO**: the highest occupied and lowest unoccupied level; the
  **KS gap** is their difference. **Delta-SCF**: the energy of the first excited
  state minus $E_{KS}$.
- $\int\rho$, $\int p_3$, $\int p_t$, $\int p_0$, $\int n$: the energy density, the
  three pressures and the particle density integrated over the hidden coordinate
  with the volume factor, $2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy}(\dots)\,dy$.
- $dE/da_4$: how fast the energy changes along the history; $Q_{max}$: the
  **adiabaticity** measure (small means the slow-change approximation holds);
  $\Delta E_x$: the difference between the exact Fock exchange and the uniform-gas
  exchange.
- $\mu$, $E$, $S$, $F = E - TS$, $\Omega$: chemical potential, energy, entropy, free
  energy and grand potential of a thermal state; $C_V = T\,dS/dT$ its heat capacity.
```

Text cell:

```
## 4. The physical and mathematical situation

**The equations being solved.** The author's coordinates are $x_1, x_2, x_3$
(3-space, scale factor $e^{a_4} \sin^{1/6} z$), $x_4$ (the time), $x_5, x_6, x_7$
(the three extra times, which DEFLATE exponentially: scale factor
$e^{-a_4} \sin^{1/6} z$ with $a_4$ increasing) and $x_0$ (the hidden direction,
$z = 6Hx_0$, written through $y = \ln(\sin z)/(6H)$ in $-L \le y \le 0$). Along the
PRESCRIBED BACKGROUND history $a_4 = A H x_4$ ($A = 1$) the Kohn-Sham states are
computed at five instants, the **slices** $a_{4,0} = 0, 0.5, 1, 1.5, 2$. At a slice
each orbital $\chi(y)$ solves the $2\times 2$ block equation
$$h_j\,\chi = \varepsilon\,\chi, \qquad h_j = j\left[-i\sigma_1 \frac{d}{dy}
+ M\sigma_2 + \kappa k\,\sigma_3\right] + v, \qquad \kappa = e^{-Hy - a_{4,0}},$$
with the ASSUMED $Z_2$ brane at $y = 0$, a regular tip at $y = -L = -3$, and the
self-consistent potentials $M = m + \tfrac{15}{16}\lambda S$ and
$v = -\tfrac{1}{16}\lambda n$ (units $H = m = 1$).

**Two independent solvers.** The Rust solver shoots with RK4 ($G = 900$ steps); the
reference solver puts the two components of $\chi$ on a staggered grid, turns the
```

```
equation into a symmetric matrix of size $2G - 1$, and computes every quantity on
$G = 300$, $600$ and $1200$. Its error behaves like $c\,h^2 + d\,h^4 + \dots$, so the
Richardson value
$$R = \frac{64\,x(1200) - 20\,x(600) + x(300)}{45}$$
removes the $h^2$ and $h^4$ terms, and its uncertainty is
$U_{ref} = |R - (4\,x(1200) - x(600))/3| + 2\cdot 10^{-12}\max(1, |R|)$.

**The Rust uncertainty.** RK4 has errors of order $h^4$. Halving the step divides
the error $e$ by $2^4 = 16$, so canonical minus refined is $e - e/16 = \tfrac{15}{16}e$
and the canonical error is $e = \tfrac{16}{15}\,|x_{canonical} - x_{refined}|$. This
is $U_{Rust}$.

**The tolerance rule** (fixed in the cross-check program before any comparison):
$$|x_{Rust} - x_{ref}| \le 3\,(U_{ref} + U_{Rust}) + 10^{-12}\cdot\text{scale},$$
with scale $= \max(1, |x|)$ for single numbers and the largest value of the profile
for profile points. The **ratio** is the left side divided by the right side.

**The subset.** The committed cross-check compared all 210 states (75 ground, 135
thermal) and passed all 31 checks. This notebook recomputes five of them:

| state | $N$ | $\lambda$ | $a_{4,0}$ | $T$ | why it is in the subset |
| --- | --- | --- | --- | --- | --- |
| N8_lamm2_a00 | 8 | $-\lambda_2$ | 0 | 0 | largest ratio of all, at the tip |
| N688_lam0_a00 | 688 | 0 | 0 | 0 | the largest number of particles |
| N136_lamp2_a20 | 136 | $+\lambda_2$ | 2 | 0 | largest ratios of levels and $Q_{max}$ |
| N8_lamm1_a00_T10 | 8 | $-\lambda_1$ | 0 | 0.01 | where the first cross-check failed |
| N8_lam0_a15_T50 | 8 | 0 | 1.5 | 0.05 | largest thermal ratios |

The values of $\lambda_1$ and $\lambda_2$ depend on $N$; the next cells read them.
What the cross-check can and cannot show is said at the end of the notebook.
```

Text cell:

```
## 5. The records and the problem definition of both solvers

The next cell reads the three committed reports (reference, Rust solver,
cross-check) and checks that every check in them passed. Then it compares the
problem definitions in the two ‘parameters.json‘ files (the physical constants
$H$, $m$, $L$, $\Delta k$, $v_t$, the tip angle, the history, the slices, the
temperatures, and the fingerprint of the theory file both programs read) and the
six couplings that each solver derived on its own from the same rule. This repeats
the cross-check’s checks ‘problem_definition_identical‘ and (for the couplings)
‘parameters_couplings‘. It also prints the sizes of the two checks that the
cross-check gained on 2026-10-08: ‘thermo_levels‘ (every level of every thermal
state, label by label) and ‘thermo_mu_rust_stated_bound‘ (the Rust solver’s stated
rounding bound of $\mu$, tested against roots computed with 40 digits).
```

In [2]:

```
import csv  # reads tables stored as CSV files (comma-separated values)
import re  # finds patterns in text (used to read numbers out of report sentences)
import sys  # the list of folders in which Python looks for modules
```

```
import tempfile  # makes temporary folders outside the repository

import numpy as np  # arrays of numbers

KS = "Revision/kohn_sham"  # the folder of the Kohn-Sham record (repository path)
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def read_csv(relative):
    """Read a CSV file of the repository: a list of rows, each row a dictionary from
    the column names to the texts in that row."""
    with open(repository_file(relative), newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle))


REPORTS = {"reference solver": read_json(f"{KS}/reports/ks-reference.json"),
           "Rust solver": read_json(f"{KS}/reports/ks-rust-solver.json"),
           "cross-check": read_json(f"{KS}/reports/ks-crosscheck.json")}
counts = {}
for label, rep in REPORTS.items():
    summary = rep["summary"]  # {"checks": ..., "pass": ..., "fail": ...}
    counts[label] = (summary["pass"], summary["checks"])
    say(f"{label}: {summary['pass']} of {summary['checks']} checks PASS")
check(counts == {"reference solver": (37, 37), "Rust solver": (42, 42),
                 "cross-check": (31, 31)},
      "the three committed reports pass every check (37, 42 and 31)")
CROSS = {c["name"]: c for c in REPORTS["cross-check"]["checks"]}  # name -> check
for name in ["thermo_levels", "thermo_mu_rust_stated_bound"]:  # the two newest
    detail = CROSS[name]["detail"]  # the report's sentence about this check
    size = re.search(r"(\d+) comparisons", detail).group(1)  # a number in the text
    say(f"cross-check {name}: {size} comparisons, verdict {CROSS[name]['verdict']}")

RUST_PARAMS = read_json(f"{KS}/results/parameters.json")  # the Rust solver's
REF_PARAMS = read_json(f"{KS}/reference/results/parameters.json")  # the reference's
pairs = [("H", "H"), ("m", "m"), ("L_tipCutoff", "L"), ("dk", "dk"), ("v_t", "vt"),
         ("tipTheta", "tipTheta"), ("historyA", "historyA"),
         ("slicesA4", "slicesA4"), ("temperatures", "temperatures")]
differ = [rust_name for rust_name, ref_name in pairs
          if RUST_PARAMS["physics"][rust_name] != REF_PARAMS["physics"][ref_name]]
same_theory = (RUST_PARAMS["theoryInputs"]["ksTheorySha256"]
               == REF_PARAMS["theoryInputs"]["ksTheorySha256"])
for rust_name, ref_name in pairs:
    say(f"  {rust_name:13s} Rust {RUST_PARAMS['physics'][rust_name]}   reference "
        f"{REF_PARAMS['physics'][ref_name]}")
check(not differ and same_theory,
      "both solvers solve the same problem and read the same ks-theory.json",
      record=f"{KS}/reports/ks-crosscheck.json, check problem_definition_identical")
```

```
LAMBDAS = {}  # N -> {"lamp1": +lambda_1, ...}: the couplings of the reference
rust_cal = {c["N"]: c for c in RUST_PARAMS["couplingCalibration"]["values"]}
same_couplings = True
for cal in REF_PARAMS["derived"]["calibration"]:
    N = cal["N"]
    LAMBDAS[N] = {"lam0": 0.0, "lamp1": cal["lambda1"], "lamm1": -cal["lambda1"],
                  "lamp2": cal["lambda2"], "lamm2": -cal["lambda2"]}
    same_couplings &= (cal["lambda1"] == rust_cal[N]["lambda1"]
                       and cal["lambda2"] == rust_cal[N]["lambda2"])
    say(f"N = {N:5.0f}: lambda_1 = {cal['lambda1']}, lambda_2 = {cal['lambda2']}")
check(same_couplings, "the six couplings derived by the two solvers are identical",
      record=f"{KS}/reports/ks-crosscheck.json, check parameters_couplings")
```

Out [2]:

```
reference solver: 37 of 37 checks PASS
Rust solver: 42 of 42 checks PASS
cross-check: 31 of 31 checks PASS
PASS the three committed reports pass every check (37, 42 and 31)
cross-check thermo_levels: 34378 comparisons, verdict PASS
cross-check thermo_mu_rust_stated_bound: 270 comparisons, verdict PASS
  H             Rust 1.0   reference 1.0
  m             Rust 1.0   reference 1.0
  L_tipCutoff   Rust 3.0   reference 3.0
  dk            Rust 0.25   reference 0.25
  v_t           Rust 1.0   reference 1.0
  tipTheta      Rust 0.0   reference 0.0
  historyA      Rust 1.0   reference 1.0
  slicesA4      Rust [0.0, 0.5, 1.0, 1.5, 2.0]   reference [0.0, 0.5, 1.0, 1.5, 2.0]
  temperatures  Rust [0.01, 0.02, 0.05]   reference [0.01, 0.02, 0.05]
PASS both solvers solve the same problem and read the same ks-theory.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    problem_definition_identical
N =     8: lambda_1 = 0.01946, lambda_2 = 0.05838
N =   136: lambda_1 = 0.0009298, lambda_2 = 0.002789
N =   688: lambda_1 = 0.0001846, lambda_2 = 0.0005538
PASS the six couplings derived by the two solvers are identical
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check parameters_couplings
```

Text cell:

```
## 6. The reference program and the five states

The reference program is the Python file ‘Revision/kohn_sham/reference/run_reference.py‘
(it uses ‘ks_fd.py‘ in the same folder). Its **jobs** compute one state each: a
ground-state job solves the state, its Delta-SCF excited state, four neighbouring
slices $a_{4,0} \pm 0.002$, $\pm 0.004$ (for $dE/da_4$ and the adiabaticity
measure) and its particle-hole list, all on the three grids; a thermal job solves the
state and four neighbouring temperatures $T(1 \pm 0.01)$, $T(1 \pm 0.02)$ (for
$C_V$). The next cell makes Python find the program, imports it **without running
it** (importing only defines its functions), reads the functional coefficients it
takes from ‘ks-theory.json‘, and writes down the five states as the program’s job
```

```
descriptions: the id, $N$, the coupling tag and value, the slice, the first-order
mean-field size $\sigma$ (0, 0.1 or 0.3; it sets how many levels are kept), and the
temperature.
```

In [3]:

```
sys.dont_write_bytecode = True  # do not write a __pycache__ folder into the record
sys.path.insert(0, str(repository_file(f"{KS}/reference")))  # Python looks here too
import run_reference as RR  # noqa: E402  the reference program (Revision code)

CO = RR.theory_coefficients()  # coefficients read and checked from ks-theory.json
say(f"functional coefficients: M_eff = m + {CO['cM']} lambda S, "
    f"v_v = {CO['cV']} lambda n, e_int = lambda ({CO['eS2']} S^2 + {CO['eN2']} n^2)")


def job_spec(N, tag, a4, T=None):
    """The reference program's description of one state (a dictionary)."""
    spec = {"id": RR.run_id(N, tag, a4, T), "N": N, "tag": tag,
            "lam": LAMBDAS[N][tag], "a4": a4, "sigma": RR.SIGMA[tag]}
    if T is not None:
        spec["T"] = T
    return spec


SPECS = {}
for args in [(8.0, "lamm2", 0.0), (688.0, "lam0", 0.0), (136.0, "lamp2", 2.0),
             (8.0, "lamm1", 0.0, 0.01), (8.0, "lam0", 1.5, 0.05)]:
    spec = job_spec(*args)
    SPECS[spec["id"]] = spec
GROUND_IDS = ["N8_lamm2_a00", "N688_lam0_a00", "N136_lamp2_a20"]
THERMAL_IDS = ["N8_lamm1_a00_T10", "N8_lam0_a15_T50"]
print("state               N   lambda       a4,0  sigma  T")
for sid, spec in SPECS.items():
    print(f"{sid:18s} {spec['N']:4.0f}  {spec['lam']:+.5f}  {spec['a4']:4.1f}  "
          f"{spec['sigma']:4.1f}   {spec.get('T', 0.0):.2f}")
check(sorted(SPECS) == sorted(GROUND_IDS + THERMAL_IDS),
      "the five job descriptions have the ids of the subset")
```

Out [3]:

```
functional coefficients: M_eff = m + 15/16 lambda S, v_v = -1/16 lambda n, e_int = lambda
    (15/32 S^2 + -1/32 n^2)
state               N   lambda       a4,0  sigma  T
N8_lamm2_a00          8  -0.05838   0.0   0.3   0.00
N688_lam0_a00       688  +0.00000   0.0   0.0   0.00
N136_lamp2_a20      136  +0.00279   2.0   0.3   0.00
N8_lamm1_a00_T10      8  -0.01946   0.0   0.1   0.01
N8_lam0_a15_T50       8  +0.00000   1.5   0.0   0.05
PASS the five job descriptions have the ids of the subset
```

Text cell:

7. Running the reference solver on the three ground states

The next cell runs the reference program's ground-state job for each of the three ground states. One addition: the job calls the program's function `_ground_on_grid` once for each grid and keeps only the extrapolated levels. To draw the convergence of every level we want the levels on each grid as well, so the cell **wraps** that function: it puts in its place a small function of our own that calls the original one, keeps a copy of the levels it returns, and passes its result on unchanged. After the three jobs the original function is put back. The results are the contents of the JSON files the reference program writes (`RR.jsonable` turns the arrays into plain numbers, as the program does before it writes a file). This cell takes one to three minutes on a fast computer.

In [4]:

```
PER_GRID = {}  # (state id, G) -> the levels of the state on grid G (an array)
original_ground_on_grid = RR._ground_on_grid  # the reference's function for one grid


def ground_on_grid_recorded(co, spec, G, labels):
    """Call the reference's own function for one grid and keep a copy of its levels."""
    result = original_ground_on_grid(co, spec, G, labels)
    PER_GRID[(spec["id"], G)] = result["eps"].copy()
    return result


RR._ground_on_grid = ground_on_grid_recorded  # the job now calls our wrapper
NEW = {}  # state id -> the new reference result (the content of its JSON file)
for sid in GROUND_IDS:
    NEW[sid] = RR.jsonable(RR.ground_job(CO, SPECS[sid])["data"])
    E = NEW[sid]["scalars"]["E_KS"]
    say(f"{sid}: {NEW[sid]['levels_in_set']} levels, E_KS = {E['value']:.13f} "
        f"(U = {E['U']:.1e})")
RR._ground_on_grid = original_ground_on_grid  # put the original function back
check(len(PER_GRID) == 9, "the levels of three states on three grids were kept")
```

Out [4]:

```
N8_lamm2_a00: 22 levels, E_KS = 0.0028626521737 (U = 2.1e-12)
N688_lam0_a00: 137 levels, E_KS = 680.4412465815219 (U = 3.3e-09)
N136_lamp2_a20: 354 levels, E_KS = 12.4489994812532 (U = 4.6e-11)
PASS the levels of three states on three grids were kept
```

Text cell:

The next cell compares each new result with the committed file `Revision/kohn_sham/reference/results/ground/<id>.json`. The function `compare_records` walks through both structures at the same time: the same names in every dictionary, the same length of every list, the same texts and truth values, and every number equal within one part in a billion (10^{-9} relative). It counts the numbers it compared. The function `check_reproduction` also writes the new

result as text exactly as the reference program writes its files and reports
whether that text is identical to the committed file **byte for byte** (on the
computer that built this book it is; the cross-check's own repeat run found the
same for all 340 files).

In [5]:

```
def compare_records(new, old, problems, where="top"):
    """Compare two JSON structures; return the number of numbers compared and append
    a description of every difference to the list problems."""
    if isinstance(new, bool) or isinstance(old, bool) or new is None or \
            isinstance(new, str):
        if new != old:
            problems.append(f"{where}: {new!r} != {old!r}")
        return 0
    if isinstance(new, (int, float)):
        if not isinstance(old, (int, float)) or \
                abs(new - old) > 1e-9 * max(1.0, abs(old)):
            problems.append(f"{where}: {new!r} != {old!r}")
        return 1
    if isinstance(new, dict):
        if sorted(new) != sorted(old):
            problems.append(f"{where}: different names")
            return 0
        return sum(compare_records(new[k], old[k], problems, f"{where}/{k}")
                   for k in new)
    if len(new) != len(old):  # a list
        problems.append(f"{where}: lengths {len(new)} and {len(old)}")
        return 0
    return sum(compare_records(a, b, problems, f"{where}[{i}]")
               for i, (a, b) in enumerate(zip(new, old)))


def check_reproduction(label, data, relative):
    """Check the new result data (named label) against its committed file."""
    committed = repository_file(relative).read_text(encoding="utf-8")
    problems = []
    numbers = compare_records(data, json.loads(committed), problems)
    text = json.dumps(data, indent=1, ensure_ascii=True) + "\n"  # as written
    say(f"{label}: {numbers} numbers compared; identical byte for byte: "
        f"{text == committed}")
    for line in problems[:5]:  # the first differences, if there are any
        say(f"  difference {line}")
    check(not problems,
          f"the re-run of {label} reproduces {relative.split('/results/')[1]}",
          record=f"{KS}/reports/ks-crosscheck.json, check "
                 "reference_repeat_byte_identical")


for sid in GROUND_IDS:
    check_reproduction(sid, NEW[sid], f"{KS}/reference/results/ground/{sid}.json")
```

Out [5]:

```
N8_lamm2_a00: 3597 numbers compared; identical byte for byte: True
PASS the re-run of N8_lamm2_a00 reproduces ground/N8_lamm2_a00.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
N688_lam0_a00: 4513 numbers compared; identical byte for byte: True
PASS the re-run of N688_lam0_a00 reproduces ground/N688_lam0_a00.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
N136_lamp2_a20: 5815 numbers compared; identical byte for byte: True
PASS the re-run of N136_lamp2_a20 reproduces ground/N136_lamp2_a20.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
```

Text cell:

```
## 8. Richardson extrapolation at work

Write $x(h)$ for a number computed on a grid with cell width $h$ and $X$ for its exact
value. For the staggered grid the error has only even powers of $h$:
$$x(h) = X + c\,h^2 + d\,h^4 + \dots$$
The three grids have $h$, $h/2$ and $h/4$ ($G = 300$, $600$, $1200$). Line by line:

- Halving $h$ gives $x(h/2) = X + c\,h^2/4 + d\,h^4/16 + \dots$ (put $h/2$ in place of
  $h$).
- Then $4\,x(h/2) - x(h) = 3X + (4/16 - 1)\,d\,h^4 + \dots$ (the $c\,h^2$ terms cancel,
  because $4 \cdot c h^2/4 = c h^2$).
- So $r(h) = \frac{4\,x(h/2) - x(h)}{3} = X - \frac{d\,h^4}{4} + \dots$: one
  Richardson step leaves an $h^4$ error.
- The same step on the finer pair gives $r(h/2) = X - \frac{d\,h^4}{64} + \dots$.
- Then $\frac{16\,r(h/2) - r(h)}{15} = X + \dots$ (the $h^4$ terms cancel, because
  $16/64 = 1/4$), and multiplying out gives
  $R = \frac{64\,x(h/4) - 20\,x(h/2) + x(h)}{45}$.
- The leftover error of the finest one-step value $r(h/2)$ is about $|R - r(h/2)|$;
  that is the uncertainty $U$ (plus a floor $2\cdot 10^{-12}\max(1,|R|)$ for
  rounding). Because $R$ has removed one more error term than $r(h/2)$, $U$ is meant
  to over-estimate the error of $R$; section 9 tests this on a fourth grid.
- Because $x(h) - x(h/2) \approx \tfrac34 c h^2$ and
  $x(h/2) - x(h/4) \approx \tfrac{3}{16} c h^2$, their **ratio is 4** when the grids
  are fine enough for the $h^2$ term to dominate (the *asymptotic regime*).

The next cell prints $E_{KS}$ of each ground state on the three grids, computes $R$
and $U$ with our own lines (both written forms of $R$), checks that they equal the
values in the reference result, and prints the ratio of the differences.
```

In [6]:

```
def richardson(x1, x2, x3):
    """Three-grid Richardson value R and its uncertainty U (x1 on the coarsest grid),
    in the two-step form of the reference program; also the two one-step values."""
    r_fine = (4.0 * x3 - x2) / 3.0  # one step on the grids h/2 and h/4
    r_coarse = (4.0 * x2 - x1) / 3.0  # one step on the grids h and h/2
    R = (16.0 * r_fine - r_coarse) / 15.0  # the second step
```

```
    U = abs(R - r_fine) + 2e-12 * max(1.0, abs(R))
    return R, U, r_coarse, r_fine


CONVERGENCE = {}  # state id -> (E_KS on the three grids, R)
for sid in GROUND_IDS:
    xs = [grid["scalars"]["E_KS"] for grid in NEW[sid]["per_grid"]]
    R, U, _, _ = richardson(*xs)
    R_one_line = (64.0 * xs[2] - 20.0 * xs[1] + xs[0]) / 45.0
    stored = NEW[sid]["scalars"]["E_KS"]
    ratio = (xs[0] - xs[1]) / (xs[1] - xs[2])
    CONVERGENCE[sid] = (xs, R)
    say(f"{sid}: E_KS(300) = {xs[0]:.12f}, E_KS(600) = {xs[1]:.12f}, "
        f"E_KS(1200) = {xs[2]:.12f}")
    say(f"    R = {R:.13f}, U = {U:.1e}, ratio of the differences = {ratio:.5f}")
    check(abs(R - stored["value"]) <= 1e-14 * max(1.0, abs(R))
          and abs(U - stored["U"]) <= 1e-14 * max(1.0, abs(R))
          and abs(R_one_line - R) <= 1e-13 * max(1.0, abs(R)),
          f"our Richardson lines give the reference value and U of E_KS ({sid})")
```

Out [6]:

```
N8_lamm2_a00: E_KS(300) = 0.002862539189, E_KS(600) = 0.002862623929, E_KS(1200) =
    0.002862645113
    R = 0.0028626521737, U = 2.1e-12, ratio of the differences = 4.00020
PASS our Richardson lines give the reference value and U of E_KS (N8_lamm2_a00)
N688_lam0_a00: E_KS(300) = 680.448036126842, E_KS(600) = 680.442943944824, E_KS(1200) =
    680.441670920908
    R = 680.4412465815219, U = 3.3e-09, ratio of the differences = 4.00007
PASS our Richardson lines give the reference value and U of E_KS (N688_lam0_a00)
N136_lamp2_a20: E_KS(300) = 12.449082943357, E_KS(600) = 12.449020346521, E_KS(1200) =
    12.449004697554
    R = 12.4489994812532, U = 4.6e-11, ratio of the differences = 4.00006
PASS our Richardson lines give the reference value and U of E_KS (N136_lamp2_a20)
```

Text cell:

```
## 9. A fourth grid tests the uncertainty

Is $U$ really larger than the error of $R$? The reference program answers this with
its job ‘validation_job‘: it solves one state, N136_lamp2_a20 (the strongest coupling
with the largest densities at the tip), on four grids $G = 300, 600, 1200, 2400$,
forms the three-grid value twice, $R_{123}$ from $(300, 600, 1200)$ and $R_{234}$
from $(600, 1200, 2400)$, and compares $|R_{123} - R_{234}|$ with the stated
uncertainty $U_{123}$ for 19 classes of quantities (energies, HOMO, LUMO, integrals,
tip and brane values, all levels, five profiles). $R_{234}$ is much closer to the
exact value than $R_{123}$ (its grids are twice as fine and its leftover error
shrinks like $h^6$), so $|R_{123} - R_{234}|$ is practically the error of $R_{123}$.

The next cell runs that job again (about 40 seconds) and compares its result with the
committed file ‘Revision/kohn_sham/reference/results/validation/N136_lamp2_a20.json‘.
To draw the four grids we again use a wrapper: the job calls the function
```

‘observables‘ of ‘ks_fd‘ once on each grid, and our wrapper keeps E_{KS}, the
levels and the tip value of p_0 of each grid.

In [7]:

```
FOUR_GRIDS = {}  # G -> (E_KS, levels, p0 at the tip) of N136_lamp2_a20 on grid G
original_observables = RR.K.observables  # the function of ks_fd (RR.K is ks_fd)


def observables_recorded(state):
    """Call ks_fd's own function and keep E_KS, the levels and p0 at the tip."""
    scalars, profiles = original_observables(state)
    FOUR_GRIDS[state.grid.G] = (scalars["E_KS"], state.eps.copy(),
                                scalars["p0_tip"])
    return scalars, profiles


RR.K.observables = observables_recorded  # the job now calls our wrapper
VALIDATION = RR.jsonable(RR.validation_job(CO, SPECS["N136_lamp2_a20"])["data"])
RR.K.observables = original_observables  # put the original function back
check_reproduction("the fourth-grid validation", VALIDATION,
                   f"{KS}/reference/results/validation/N136_lamp2_a20.json")
same = all(abs(FOUR_GRIDS[G][0] - grid["scalars"]["E_KS"]) <= 1e-12
           for G, grid in zip((300, 600, 1200), NEW["N136_lamp2_a20"]["per_grid"]))
check(sorted(FOUR_GRIDS) == [300, 600, 1200, 2400] and same,
      "the validation run repeats the ground job's E_KS on the three common grids")
classes = VALIDATION["classes"]
for name, c in classes.items():
    say(f"  {name:20s} {c['elements']:4d} element(s): max |R123 - R234| / U123 = "
        f"{c['max_ratio_to_U123']:.2e}")
worst = max(classes, key=lambda name: classes[name]["max_ratio_to_U123"])
detail = next(c["detail"] for c in REPORTS["reference solver"]["checks"]
              if c["name"] == "richardson_uncertainty_validated")
stated = float(re.search(r"U123 = ([0-9.]+) \((\w+)\)", detail).group(1))
say(f"largest ratio {classes[worst]['max_ratio_to_U123']:.3f} ({worst}); "
    f"the report states {stated}")
check(all(c["all_within_U123"] for c in classes.values())
      and len(classes) == 19 and worst == "p0_tip"
      and abs(classes[worst]["max_ratio_to_U123"] - stated) <= 5e-4,
      "on the fourth grid every three-grid value moves by less than its U",
      record=f"{KS}/reports/ks-reference.json, check richardson_uncertainty_validated")
```

Out [7]:

```
the fourth-grid validation: 99 numbers compared; identical byte for byte: True
PASS the re-run of the fourth-grid validation reproduces validation/N136_lamp2_a20.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
PASS the validation run repeats the ground job's E_KS on the three common grids
  E_KS                    1 element(s): max |R123 - R234| / U123 = 7.66e-04
  E_int                   1 element(s): max |R123 - R234| / U123 = 1.02e-03
  HOMO                    1 element(s): max |R123 - R234| / U123 = 1.22e-04
```

```
  LUMO                   1 element(s): max |R123 - R234| / U123 = 2.40e-04
  int_rho                1 element(s): max |R123 - R234| / U123 = 5.73e-04
  int_p3                 1 element(s): max |R123 - R234| / U123 = 0.00e+00
  int_p0                 1 element(s): max |R123 - R234| / U123 = 2.04e-03
  dE_da4_emt             1 element(s): max |R123 - R234| / U123 = 1.46e-04
  deltaE_x_exact_fock    1 element(s): max |R123 - R234| / U123 = 1.99e-04
  rho_tip                1 element(s): max |R123 - R234| / U123 = 4.38e-01
  p0_tip                 1 element(s): max |R123 - R234| / U123 = 7.87e-01
  rho_brane              1 element(s): max |R123 - R234| / U123 = 2.32e-06
  p0_brane               1 element(s): max |R123 - R234| / U123 = 1.59e-06
  eigenvalues          354 element(s): max |R123 - R234| / U123 = 2.73e-03
  profile_n            151 element(s): max |R123 - R234| / U123 = 6.61e-01
  profile_S            151 element(s): max |R123 - R234| / U123 = 2.22e-02
  profile_rho          151 element(s): max |R123 - R234| / U123 = 4.38e-01
  profile_p3           151 element(s): max |R123 - R234| / U123 = 4.37e-01
  profile_p0           151 element(s): max |R123 - R234| / U123 = 7.87e-01
largest ratio 0.787 (p0_tip); the report states 0.787
PASS on the fourth grid every three-grid value moves by less than its U
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    richardson_uncertainty_validated
```

Text cell:

```
The next figure shows convergence as a picture. Left: for the three ground states, the
distance $|x(G) - R|$ of $E_{KS}$ on each grid from the extrapolated value, against
the cell width $h = 3/G$, on logarithmic axes; a straight line of slope 2 means an
error proportional to $h^2$. Right: N136_lamp2_a20 on all four grids, every distance
measured from the best value $R_{234}$: the single grids (slope 2), the three
one-step values $r$ from neighbouring pairs of grids (slope 4), and the three-grid
value $R_{123}$ next to its stated uncertainty $U_{123}$. The dashed guide lines
have slopes 2 and 4 and pass through the first point of their series.
```

In [8]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.3))
h = np.array([3.0 / 300, 3.0 / 600, 3.0 / 1200])  # the cell widths
for colour, sid in zip(PALETTE, GROUND_IDS):
    xs, R = CONVERGENCE[sid]
    left.loglog(h, [abs(x - R) for x in xs], "o-", color=colour, lw=1.5, ms=6,
                label=sid)
    first = abs(xs[0] - R)  # a guide of slope 2, a factor 3 below the first point
    left.loglog(h, first / 3.0 * (h / h[0]) ** 2, "--", color=colour, lw=0.9)
left.set_xlabel("cell width $h = L/G$")
left.set_ylabel("$|E_{KS}(G) - R|$ (units of $m$)")
left.set_title("three grids, three states")
left.legend(fontsize=8)

grids = [300, 600, 1200, 2400]
e4 = [FOUR_GRIDS[G][0] for G in grids]  # E_KS of N136_lamp2_a20 on the four grids
h4 = np.array([3.0 / G for G in grids])
R123, U123, _, _ = richardson(*e4[:3])
R234, _, _, _ = richardson(*e4[1:])
```

```
single = [abs(x - R234) for x in e4]
one_step = [abs((4.0 * e4[i + 1] - e4[i]) / 3.0 - R234) for i in range(3)]
right.loglog(h4, single, "o-", color=PALETTE[0], lw=1.5, ms=6,
             label="single grid $x(G)$")
right.loglog(h4, single[0] / 3.0 * (h4 / h4[0]) ** 2, "--", color=PALETTE[0],
             lw=0.9, label="slope 2 (shifted down)")
right.loglog(h4[:3], one_step, "s-", color=PALETTE[1], lw=1.5, ms=6,
             label="one step $r$ (pair $G$, $2G$)")
right.loglog(h4[:3], one_step[0] / 3.0 * (h4[:3] / h4[0]) ** 4, "--",
             color=PALETTE[1], lw=0.9, label="slope 4 (shifted down)")
right.loglog([h4[0]], [max(abs(R123 - R234), 1e-16)], "D", color=PALETTE[2], ms=8,
             label="three grids $R_{123}$")
right.loglog([h4[0]], [U123], "_", color="k", ms=16, mew=2,
             label="its uncertainty $U_{123}$")
right.set_xlabel("cell width $h$ of the coarsest grid used")
right.set_ylabel("distance from $R_{234}$ (units of $m$)")
right.set_title("N136_lamp2_a20 on four grids")
right.legend(fontsize=7, loc="center right")
for ax, ticks in ((left, h), (right, h4)):  # tick labels at the grids' h only
    ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
    ax.set_xticks(ticks)
    ax.set_xticklabels([f"{t:g}" for t in ticks])
fig.tight_layout()
save_figure(fig, "grid_convergence",
            "Convergence of the reference solver for the Kohn-Sham energy $E_{KS}$, "
            "logarithmic axes, energies in units of the mass $m$. Left: distance of "
            "the single-grid value from the Richardson value $R$ against the cell "
            "width $h = 3/G$ for $G = 300, 600, 1200$ for the three ground states; "
            "the points run parallel to the dashed lines of slope 2 (drawn a factor 3 "
            "lower), an error proportional to $h^2$. Right: the state N136_lamp2_a20 "
            "also on $G = 2400$, every "
            "distance measured from the four-grid best value $R_{234}$: single grids "
            "fall with slope 2, one Richardson step with slope 4, and the three-grid "
            "value $R_{123}$ (diamond) lies far below its stated uncertainty "
            "$U_{123}$ (bar), so $U$ is a safe over-estimate.")
check(all(abs(CONVERGENCE[s][0][0] - CONVERGENCE[s][1])
          > abs(CONVERGENCE[s][0][2] - CONVERGENCE[s][1]) for s in GROUND_IDS)
      and single[0] > single[1] > single[2] > single[3]
      and one_step[0] > one_step[1] > one_step[2] and abs(R123 - R234) <= U123,
      "the errors shrink grid by grid and R123 lies within U123 of R234")
```

Out [8]:

```
Figure 16a.1 saved as Revision/textbook/figures/16a_1_grid_convergence.png
PASS the errors shrink grid by grid and R123 lies within U123 of R234
```

Text cell:

```
The next figure draws the result of the validation for all 19 classes: for each
class the largest value of $|R_{123} - R_{234}|/U_{123}$ over its elements. A bar
that stays left of the line 1 means that the stated uncertainty covers the change.
A class whose largest value is exactly zero (the two three-grid values agree in every
digit) is drawn at $10^{-8}$.
```

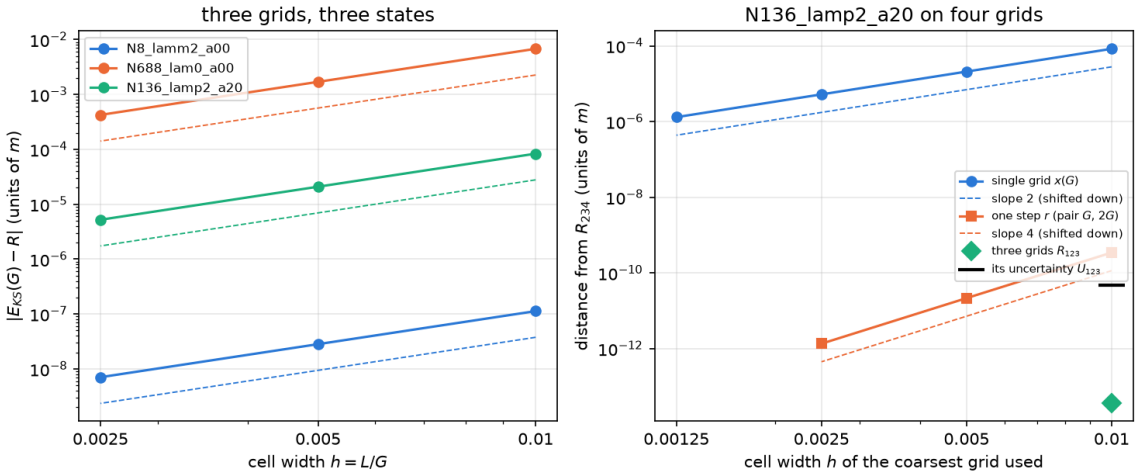

Figure 439: Convergence of the reference solver for the Kohn-Sham energy E_{KS}, logarithmic axes, energies in units of the mass m. Left: distance of the single-grid value from the Richardson value R against the cell width $h = 3/G$ for $G = 300, 600, 1200$ for the three ground states; the points run parallel to the dashed lines of slope 2 (drawn a factor 3 lower), an error proportional to h^2. Right: the state N136_lamp2_a20 also on $G = 2400$, every distance measured from the four-grid best value R_{234}: single grids fall with slope 2, one Richardson step with slope 4, and the three-grid value R_{123} (diamond) lies far below its stated uncertainty U_{123} (bar), so U is a safe over-estimate. (Notebook 16a, figure 1.)

In [9]:

```
names = list(classes)  # the 19 classes in the order of the record
values = [max(classes[n]["max_ratio_to_U123"], 1e-8) for n in names]
fig, ax = plt.subplots(figsize=(7.5, 6.2))
colours = [PALETTE[1] if n == worst else PALETTE[0] for n in names]
ax.barh(range(len(names)), values, color=colours, height=0.65)
ax.set_xscale("log")
ax.axvline(1.0, color="k", lw=1.2)
ax.set_yticks(range(len(names)))
ax.set_yticklabels([f"{n} ({classes[n]['elements']})" for n in names], fontsize=8)
ax.invert_yaxis()  # the first class at the top
ax.set_xlim(1e-9, 3.0)
for i, n in enumerate(names):
    if classes[n]["max_ratio_to_U123"] == 0.0:  # mark a class that agrees exactly
        ax.text(1.5e-8, i, "exactly 0", va="center", fontsize=8)
ax.set_xlabel("largest $|R_{123} - R_{234}|$ / $U_{123}$ of the class")
ax.set_title("Fourth-grid test of the uncertainty (N136_lamp2_a20)")
save_figure(fig, "uncertainty_validated",
            "Validation of the reference uncertainty on a fourth grid for the state "
            "N136_lamp2_a20: for each of the 19 classes of quantities (the number of "
            "elements in brackets) the largest change of the three-grid value between "
            "the grids $(300, 600, 1200)$ and $(600, 1200, 2400)$, divided by the "
            "stated uncertainty $U_{123}$, on a logarithmic axis; a class that agrees "
            f"exactly is drawn at $10^{{-8}}$ and marked. All bars end left of the "
            "line 1; the largest, "
            f"{classes[worst]['max_ratio_to_U123']:.3f}, is the value of $p_0$ at the "
```

```
"tip, where the densities are largest.")
```

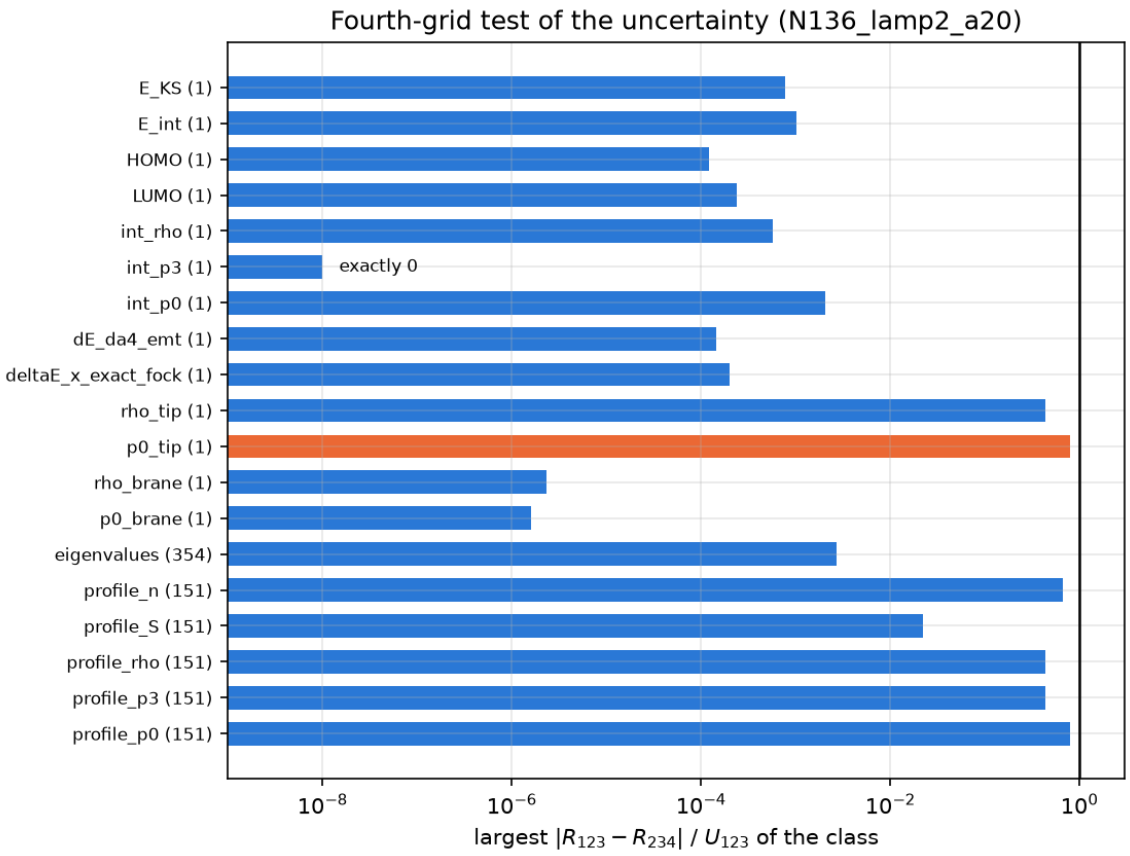

Figure 440: Validation of the reference uncertainty on a fourth grid for the state N136_lamp2_a20: for each of the 19 classes of quantities (the number of elements in brackets) the largest change of the three-grid value between the grids $(300, 600, 1200)$ and $(600, 1200, 2400)$, divided by the stated uncertainty U_{123}, on a logarithmic axis; a class that agrees exactly is drawn at 10^{-8} and marked. All bars end left of the line 1; the largest, 0.787, is the value of p_0 at the tip, where the densities are largest. (Notebook 16a, figure 2.)

Out [9]:

```
Figure 16a.2 saved as Revision/textbook/figures/16a_2_uncertainty_validated.png
```

Text cell:

```
## 10. Every level converges like $h^2$

The ratio test can be made for every level, not only for $E_{KS}$. The next cell
computes, for every level of the three ground states, the ratio
$(x(300) - x(600))/(x(600) - x(1200))$ with the reference program's own function
`RR.asym_ratio` (it leaves out levels whose differences are below $10^{-10}$, where
rounding would dominate), and compares the median, smallest and largest ratio with
```

the values stored in the record (entry ‘consistency‘). Then it draws, for every level, the distance of its ratio from 4 against the level’s energy. The reference’s check ‘richardson_asymptotic_ratio‘ requires the median to lie within 0.05 of 4.

In [10]:

```
level_points = []  # (state, level energy, ratio) of every level with a ratio
for sid in GROUND_IDS:
    eps = [PER_GRID[(sid, G)] for G in (300, 600, 1200)]
    ratios = RR.asym_ratio(*eps)  # the reference's own function
    stored = NEW[sid]["consistency"]["ratio_eps"]
    mine = (len(ratios), float(np.median(ratios)), min(ratios), max(ratios))
    say(f"{sid}: {mine[0]} levels, median ratio {mine[1]:.6f}, smallest "
        f"{mine[2]:.6f}, largest {mine[3]:.6f}")
    check(mine == (stored["count"], stored["median"], stored["min"], stored["max"])
          and abs(mine[1] - 4.0) <= 0.05,
          f"the levels of {sid} converge like h^2 (median ratio within 0.05 of 4)",
          record=f"{KS}/reports/ks-reference.json, check richardson_asymptotic_ratio")
    floor = 1e-10  # the same selection as asym_ratio: both differences above 1e-10
    d1, d2 = eps[0] - eps[1], eps[1] - eps[2]
    keep = (np.abs(d1) > floor) & (np.abs(d2) > floor)
    energies = np.asarray(NEW[sid]["levels"]["eps"])[keep]
    level_points += [(sid, e, r) for e, r in zip(energies, d1[keep] / d2[keep])]

fig, ax = plt.subplots(figsize=(7.5, 4.3))
for colour, sid in zip(PALETTE, GROUND_IDS):
    pts = [(e, abs(r - 4.0)) for s, e, r in level_points if s == sid]
    ax.semilogy([p[0] for p in pts], [p[1] for p in pts], "o", color=colour, ms=4,
                alpha=0.8, label=f"{sid} ({len(pts)} levels)")
ax.axhline(0.05, color="k", lw=1.0, ls="--", label="limit 0.05 of the median test")
ax.set_xlabel("level energy $\\varepsilon$ (Richardson value, units of $m$)")
ax.set_ylabel("$|$ratio $- 4|$")
ax.set_title("Every level converges like $h^2$")
ax.legend(fontsize=8)
worst_level = max(abs(r - 4.0) for _, _, r in level_points)
save_figure(fig, "eigenvalue_ratios",
            "Distance from 4 of the convergence ratio $(x(300) - x(600))/(x(600) - "
            "x(1200))$ of every level of the three ground states, against the "
            "level energy in units of $m$ (vertical axis logarithmic). A ratio of 4 "
            "means an error proportional to $h^2$. Most levels lie between "
            "$10^{-5}$ and $3\\times10^{-4}$ from 4, with no trend in the energy; "
            "the levels nearest $\\varepsilon = 0$ deviate by $5\\times10^{-4}$ to "
            "$7\\times10^{-4}$. The largest deviations belong to a group of levels "
            "of N136_lamp2_a20 between $\\varepsilon = 1.2$ and $2.1$; even the worst "
            f"level, at {worst_level:.4f}, stays below the limit 0.05 that the "
            "reference applies to the median.")
check(worst_level < 0.05, "every single level has a ratio within 0.05 of 4")
```

Out [10]:

```
N8_lamm2_a00: 22 levels, median ratio 4.000053, smallest 3.999954, largest 4.000506
PASS the levels of N8_lamm2_a00 converge like h^2 (median ratio within 0.05 of 4)
```

```
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    richardson_asymptotic_ratio
N688_lam0_a00: 135 levels, median ratio 4.000064, smallest 3.999959, largest 4.000220
PASS the levels of N688_lam0_a00 converge like h^2 (median ratio within 0.05 of 4)
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    richardson_asymptotic_ratio
N136_lamp2_a20: 354 levels, median ratio 4.000055, smallest 3.999269, largest 4.033570
PASS the levels of N136_lamp2_a20 converge like h^2 (median ratio within 0.05 of 4)
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    richardson_asymptotic_ratio
```

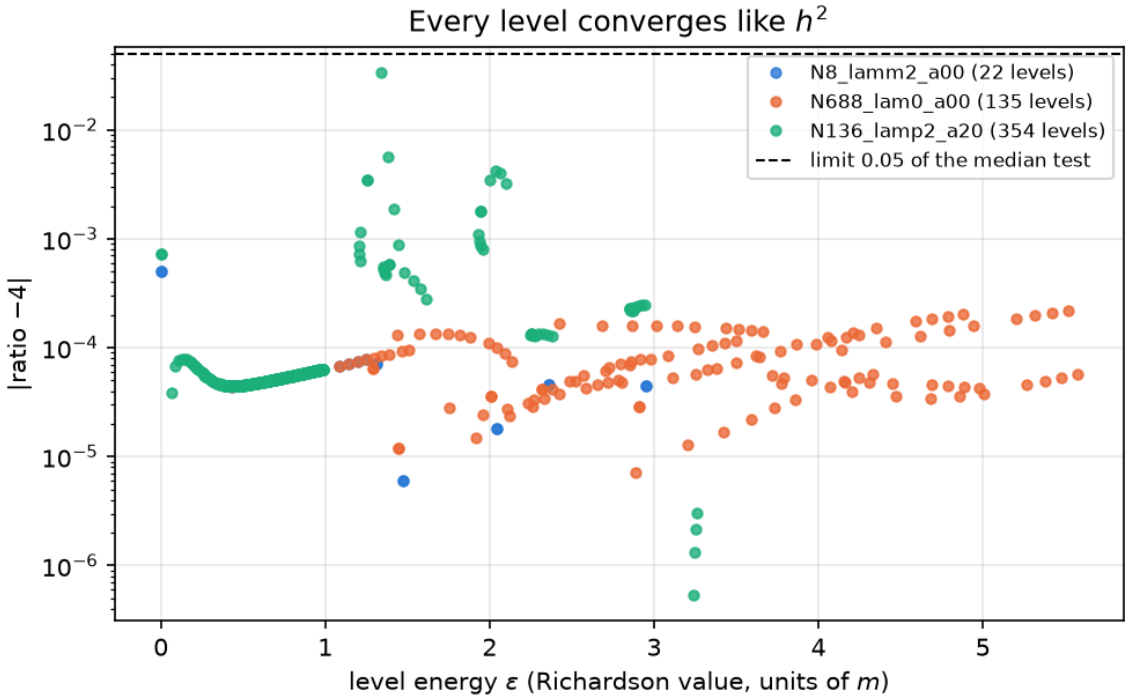

Figure 441: Distance from 4 of the convergence ratio $(x(300) - x(600))/(x(600) - x(1200))$ of every level of the three ground states, against the level energy in units of m (vertical axis logarithmic). A ratio of 4 means an error proportional to h^2. Most levels lie between 10^{-5} and 3×10^{-4} from 4, with no trend in the energy; the levels nearest $\varepsilon = 0$ deviate by 5×10^{-4} to 7×10^{-4}. The largest deviations belong to a group of levels of N136_lamp2_a20 between $\varepsilon = 1.2$ and 2.1; even the worst level, at 0.0336, stays below the limit 0.05 that the reference applies to the median. (Notebook 16a, figure 3.)

Out [10]:

```
Figure 16a.3 saved as Revision/textbook/figures/16a_3_eigenvalue_ratios.png
PASS every single level has a ratio within 0.05 of 4
```

Text cell:

```
## 11. The two thermal states on the reference
```

```
The next cell runs the reference program's thermal job for the two thermal states,
compares the results with the committed files
`Revision/kohn_sham/reference/results/thermo/<id>.json` in the same way, and prints
the chemical potential $\mu$, the energy $E$ and the heat capacity $C_V$ with their
uncertainties. The state N8_lamm1_a00_T10 is the one in which the first cross-check
failed (the Rust value of $\mu$ was $8.3\times10^{-10}$ too small, about 100 times
the tolerance); Notebook 16c of this chapter tells that story and reproduces the
faulty value digit for digit.
```

In [11]:

```
for sid in THERMAL_IDS:
    NEW[sid] = RR.jsonable(RR.thermo_job(CO, SPECS[sid])["data"])
    th = NEW[sid]["thermo"]
    say(f"{sid}: {NEW[sid]['levels_in_set']} levels; mu = {th['mu']['value']:.13f} "
        f"(U {th['mu']['U']:.1e}), E = {th['E']['value']:.12f} (U {th['E']['U']:.1e}),"
        f" C_V = {th['C_V']['value']:.9f} (U {th['C_V']['U']:.1e})")
    check_reproduction(sid, NEW[sid], f"{KS}/reference/results/thermo/{sid}.json")
```

Out [11]:

```
N8_lamm1_a00_T10: 8 levels; mu = 0.2100104497343 (U 2.3e-12), E = 0.000986845293 (U
    2.0e-12), C_V = 0.000005755 (U 2.0e-12)
N8_lamm1_a00_T10: 259 numbers compared; identical byte for byte: True
PASS the re-run of N8_lamm1_a00_T10 reproduces thermo/N8_lamm1_a00_T10.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
N8_lam0_a15_T50: 474 levels; mu = -0.0304029805513 (U 2.1e-12), E = 0.950242831973 (U
    2.6e-12), C_V = 32.379779710 (U 8.0e-11)
N8_lam0_a15_T50: 6537 numbers compared; identical byte for byte: True
PASS the re-run of N8_lam0_a15_T50 reproduces thermo/N8_lam0_a15_T50.json
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    reference_repeat_byte_identical
```

Text cell:

```
## 12. The Rust solver: canonical and refined runs

The next cell builds the Rust solver with cargo (a second when it is already built)
and runs its command `single` for each state twice: with the canonical numerics and
with `--refined`. The arguments are exactly those of the cross-check's measurement
program `measure_rust_refinement.py`: the mass $m = 1$, the coupling, the slice, $N$,
the temperature for thermal states, and the **margin**, which decides how many levels
above the Fermi level are kept ($0.25 + 2\sigma$ at $T = 0$, $0.2 + 2\sigma$ at
$T > 0$). Each run writes a JSON file (energies, levels, integrals) and a CSV file
(the profiles) into a new **temporary folder** of the operating system, outside the
repository; the cell reads them back and then deletes the folder, so the runs leave
no file behind. It prints each command without the two output-file options (their
folder differs from run to run).
```

In [12]:

```
SOLVER = rust_program(f"{KS}/solver/Cargo.toml", "revision_ks_solver")
RUN_FOLDER = Path(tempfile.mkdtemp(prefix="textbook_16a_"))  # a new, empty folder


def run_single(sid, refined):
    """Run ‘revision_ks_solver single‘ for state sid; return (results, profiles,
    the arguments without the output files)."""
    spec = SPECS[sid]
    margin = (0.2 if "T" in spec else 0.25) + 2.0 * spec["sigma"]
    stem = f"{sid}_{’refined’ if refined else ’canonical’}"
    arguments = ["single", "--m", "1", "--lambda", repr(spec["lam"]),
                 "--a4", repr(spec["a4"]), "--N", repr(spec["N"]),
                 "--margin", repr(margin)]
    if "T" in spec:
        arguments += ["--T", repr(spec["T"])]
    if refined:
        arguments.append("--refined")
    files = ["--out", f"{RUN_FOLDER / stem}.json",
             "--profiles", f"{RUN_FOLDER / stem}.csv"]
    done = subprocess.run([str(SOLVER)] + arguments + files, cwd=str(REPO),
                          capture_output=True, text=True)
    if done.returncode != 0 or not done.stdout.strip().endswith("SUCCESS"):
        print(done.stderr[-2000:])
        raise RuntimeError(f"the Rust run {stem} failed")
    results = json.loads((RUN_FOLDER / f"{stem}.json").read_text(encoding="utf-8"))
    with open(RUN_FOLDER / f"{stem}.csv", newline="", encoding="utf-8") as handle:
        profiles = [{k: float(v) for k, v in row.items()}
                    for row in csv.DictReader(handle)]
    return results, profiles, arguments


RUST = {}  # (state id, "canonical" or "refined") -> (results, profiles)
for sid in SPECS:
    for refined in (False, True):
        results, profiles, arguments = run_single(sid, refined)
        RUST[(sid, "refined" if refined else "canonical")] = (results, profiles)
    say("revision_ks_solver " + " ".join(arguments))  # the refined command
shutil.rmtree(RUN_FOLDER)  # delete the temporary folder and the raw outputs in it
check(len(RUST) == 10, "ten Rust runs (five states, two numerics) completed")
```

Out [12]:

```
Rust program revision_ks_solver is built and ready.
revision_ks_solver single --m 1 --lambda -0.05838 --a4 0.0 --N 8.0 --margin 0.85
    --refined
revision_ks_solver single --m 1 --lambda 0.0 --a4 0.0 --N 688.0 --margin 0.25 --refined
revision_ks_solver single --m 1 --lambda 0.002789 --a4 2.0 --N 136.0 --margin 0.85
    --refined
revision_ks_solver single --m 1 --lambda -0.01946 --a4 0.0 --N 8.0 --margin 0.4 --T 0.01
    --refined
revision_ks_solver single --m 1 --lambda 0.0 --a4 1.5 --N 8.0 --margin 0.2 --T 0.05
    --refined
PASS ten Rust runs (five states, two numerics) completed
```

Text cell:

```
The measured differences are only meaningful if the canonical ‘single‘ run gives
exactly the committed canonical results. The next cell checks this, as the
cross-check’s check ‘rust_refinement_applies_to_matrix‘ does: for the ground states
$E_{KS}$, the five energy-momentum integrals, every level and every profile point;
for the thermal states $E$, $\mu$ and the entropy; each must agree with the committed
files within $10^{-12}$ (relative to $\max(1,|x|)$ or to the profile’s largest value).
Then it computes the canonical-minus-refined differences of $E_{KS}$, of the levels and
(for thermal states) of $\mu$ and checks that they equal the values recorded in
‘Revision/kohn_sham/checker/rust-refinement.json‘.
```

In [13]:

```
SUMMARY = {r["id"]: r for r in read_csv(f"{KS}/results/ground/summary.csv")}
EMT = {r["id"]: r for r in read_csv(f"{KS}/results/ground/emt-integrals.csv")}
EXCITED = {r["id"]: r for r in read_csv(f"{KS}/results/excited/summary.csv")}
ADIABATIC = {r["id"]: r for r in read_csv(f"{KS}/results/adiabatic/adiabaticity.csv")}
THERMO = {r["id"]: r for r in read_csv(f"{KS}/results/thermo/thermodynamics.csv")}
REFINEMENT = {(s["kind"], s["id"]): s
              for s in read_json(f"{KS}/checker/rust-refinement.json")["states"]}


def rel(a, b):
    """|a - b| relative to max(1, |b|)."""
    return abs(a - b) / max(1.0, abs(b))


def level_map(results):
    """{(n2, j, parity, label): eps} of the levels of a ‘single‘ run."""
    return {tuple(l[:4]): l[4] for l in results["levels_n2_j_parity_label_eps_deg_f"]}


for sid in SPECS:
    canon, canon_prof = RUST[(sid, "canonical")]
    refined, _ = RUST[(sid, "refined")]
    if sid in GROUND_IDS:
        worst = rel(canon["E_KS"], float(SUMMARY[sid]["E_KS"]))
        integrals = canon["emtIntegrals_2Vol7_int_e6Hy"]
        for k in ("rho", "p3", "p_t", "p0", "n"):
            worst = max(worst, rel(integrals[k], float(EMT[sid]["int_" + k])))
        committed = {(int(x["n2"]), int(x["j"]), x["parity"], int(x["label"])):
                     float(x["eps"])
                     for x in read_csv(f"{KS}/results/ground/levels/{sid}.csv")}
        mine = level_map(canon)
        worst = max([worst] + [abs(mine[k] - committed[k]) for k in mine
                               if k in committed])
        stored_prof = read_csv(f"{KS}/results/ground/profiles/{sid}.csv")
        for name in ("n", "S", "rho", "p3", "p_t", "p0"):
            top = max(abs(float(row[name])) for row in stored_prof)
            worst = max([worst] + [abs(a[name] - float(b[name])) / max(top, 1e-300)
                                   for a, b in zip(canon_prof, stored_prof)])
```

```
        kind = "ground"
    else:
        row = THERMO[sid]
        worst = max(rel(canon["E_KS"], float(row["E"])),
                    abs(canon["mu_or_fermi_level"] - float(row["mu"])),
                    abs(canon["entropy"] - float(row["entropy"]))
                    / max(1e-6, abs(float(row["entropy"]))))
        kind = "thermo"
    rec = REFINEMENT[(kind, sid)]
    lc, lr = level_map(canon), level_map(refined)
    level_diff = max(abs(lc[k] - lr[k]) for k in lc if k in lr)
    e_diff = abs(canon["E_KS"] - refined["E_KS"])
    say(f"{sid}: largest difference from the committed results {worst:.1e}; "
        f"|canonical - refined|: E_KS {e_diff:.3e}, levels {level_diff:.3e}")
    same = (abs(e_diff - rec["scalars"]["E_KS"]["abs_diff"])
            <= 1e-12 * max(1.0, abs(canon["E_KS"]))
            and abs(level_diff - rec["levels"]["max_abs_diff"]) <= 1e-12)
    if kind == "thermo":
        mu_diff = abs(canon["mu_or_fermi_level"] - refined["mu_or_fermi_level"])
        same = same and abs(mu_diff - rec["scalars"]["mu"]["abs_diff"]) <= 1e-12
    check(worst <= 1e-12, f"the canonical run of {sid} reproduces the committed "
                          "Rust results",
          record=f"{KS}/reports/ks-crosscheck.json, check "
                 "rust_refinement_applies_to_matrix")
    check(same, f"canonical minus refined of {sid} reproduces the measured values",
          record=f"{KS}/checker/rust-refinement.json, state {kind} {sid}")
```

Out [13]:

```
N8_lamm2_a00: largest difference from the committed results 0.0e+00; |canonical -
    refined|: E_KS 1.139e-13, levels 3.738e-10
PASS the canonical run of N8_lamm2_a00 reproduces the committed Rust results
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    rust_refinement_applies_to_matrix
PASS canonical minus refined of N8_lamm2_a00 reproduces the measured values
     reproduces Revision/kohn_sham/checker/rust-refinement.json, state ground
    N8_lamm2_a00
N688_lam0_a00: largest difference from the committed results 0.0e+00; |canonical -
    refined|: E_KS 7.529e-10, levels 2.135e-09
PASS the canonical run of N688_lam0_a00 reproduces the committed Rust results
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    rust_refinement_applies_to_matrix
PASS canonical minus refined of N688_lam0_a00 reproduces the measured values
     reproduces Revision/kohn_sham/checker/rust-refinement.json, state ground
    N688_lam0_a00
N136_lamp2_a20: largest difference from the committed results 0.0e+00; |canonical -
    refined|: E_KS 1.797e-11, levels 2.711e-10
PASS the canonical run of N136_lamp2_a20 reproduces the committed Rust results
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    rust_refinement_applies_to_matrix
PASS canonical minus refined of N136_lamp2_a20 reproduces the measured values
     reproduces Revision/kohn_sham/checker/rust-refinement.json, state ground
    N136_lamp2_a20
```

```
N8_lamm1_a00_T10: largest difference from the committed results 0.0e+00; |canonical -
    refined|: E_KS 8.223e-15, levels 1.267e-12
PASS the canonical run of N8_lamm1_a00_T10 reproduces the committed Rust results
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    rust_refinement_applies_to_matrix
PASS canonical minus refined of N8_lamm1_a00_T10 reproduces the measured values
     reproduces Revision/kohn_sham/checker/rust-refinement.json, state thermo
    N8_lamm1_a00_T10
N8_lam0_a15_T50: largest difference from the committed results 0.0e+00; |canonical -
    refined|: E_KS 2.423e-10, levels 2.225e-12
PASS the canonical run of N8_lam0_a15_T50 reproduces the committed Rust results
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    rust_refinement_applies_to_matrix
PASS canonical minus refined of N8_lam0_a15_T50 reproduces the measured values
     reproduces Revision/kohn_sham/checker/rust-refinement.json, state thermo
    N8_lam0_a15_T50
```

Text cell:

```
## 13. The tolerance rule in a few lines of Python

The next cell writes the rule as a function. ‘compare‘ receives the class, the case
name, the Rust value, the reference value, the two uncertainties and (for profile
points) the scale; it computes the tolerance and the ratio exactly as the cross-check
program does and stores a row in the list ‘ROWS‘. The cell also computes, for every
state, the Rust uncertainties $U_{Rust} = \tfrac{16}{15}|canonical - refined|$ of
every quantity the ‘single‘ command reports, and reads the **matrix-wide** Rust
uncertainties that the cross-check uses for quantities that ‘single‘ does not report
(Delta-SCF, $Q_{max}$, $dE/da_4$ by differences, $C_V$); they are the largest
canonical-minus-refined differences of the whole Rust matrix, recorded in the
cross-check report. Finally it shows every piece of one comparison, $E_{KS}$ of
N8_lamm2_a00.
```

In [14]:

```
RK4_FACTOR = 16.0 / 15.0  # canonical error = (16/15) |canonical - refined|
ROWS = []  # (class, case, x_Rust, x_ref, U_ref, U_Rust, tolerance, |diff|, ratio)


def compare(cls, case, x_rust, x_ref, u_ref, u_rust, scale=None):
    """The tolerance rule of the cross-check; returns the ratio."""
    scale = max(1.0, abs(x_ref)) if scale is None else scale
    tolerance = 3.0 * (u_ref + u_rust) + 1e-12 * scale
    diff = abs(x_rust - x_ref)
    ratio = diff / tolerance
    ROWS.append((cls, case, x_rust, x_ref, u_ref, u_rust, tolerance, diff, ratio))
    return ratio


PROFILE_NAMES = ("n", "S", "Q", "M_eff", "v_v", "e_int", "rho", "p3", "p_t", "p0")
```

```
def rust_uncertainties(sid):
    """U_Rust of every quantity reported by ‘single‘, for state sid."""
    (canon, cp), (refined, rp) = RUST[(sid, "canonical")], RUST[(sid, "refined")]
    U = {k: RK4_FACTOR * abs(canon[k] - refined[k])
         for k in ("E_KS", "E_band", "E_int", "entropy")}
    U["mu"] = RK4_FACTOR * abs(canon["mu_or_fermi_level"]
                               - refined["mu_or_fermi_level"])
    ci, ri = canon["emtIntegrals_2Vol7_int_e6Hy"], refined["emtIntegrals_2Vol7_int_e6Hy"]
    for k in ("rho", "p3", "p_t", "p0", "n"):
        U["int_" + k] = RK4_FACTOR * abs(ci[k] - ri[k])
    for k in ("rho", "p3", "p_t", "p0"):  # profile end points: brane and tip
        U[k + "_brane"] = RK4_FACTOR * abs(cp[-1][k] - rp[-1][k])
        U[k + "_tip"] = RK4_FACTOR * abs(cp[0][k] - rp[0][k])
    lc, lr = level_map(canon), level_map(refined)
    U["levels"] = RK4_FACTOR * max(abs(lc[k] - lr[k]) for k in lc if k in lr)
    for name in PROFILE_NAMES:  # (U of the profile, largest |value|)
        U["profile_" + name] = (
            RK4_FACTOR * max(abs(a[name] - b[name]) for a, b in zip(cp, rp)),
            max(abs(a[name]) for a in cp))
    return U


U_RUST = {sid: rust_uncertainties(sid) for sid in SPECS}
WIDE = REPORTS["cross-check"]["rust_matrix_wide_uncertainties"]
say("matrix-wide Rust differences: " + ", ".join(
    f"{k.replace('refined_', '')} {v:.2e}" for k, v in sorted(WIDE.items())))

sid = "N8_lamm2_a00"  # one comparison, piece by piece
x_rust = float(SUMMARY[sid]["E_KS"])
x_ref = NEW[sid]["scalars"]["E_KS"]["value"]
u_ref = NEW[sid]["scalars"]["E_KS"]["U"]
u_rust = U_RUST[sid]["E_KS"]
ratio = compare("ground_energies", f"{sid} E_KS", x_rust, x_ref, u_ref, u_rust)
say(f"E_KS of {sid}: Rust {x_rust!r}, reference {x_ref!r}")
say(f"    |diff| = {abs(x_rust - x_ref):.3e}, U_ref = {u_ref:.3e}, "
    f"U_Rust = {u_rust:.3e}")
say(f"    tolerance = 3 (U_ref + U_Rust) + 1e-12 max(1, |x|) = {ROWS[-1][6]:.3e}, "
    f"ratio = {ratio:.4f}")
check(ratio <= 1.0, f"E_KS of {sid} agrees within the tolerance")
```

Out [14]:

```
matrix-wide Rust differences: adiabatic_derivatives 9.02e-10, delta_scf 5.82e-11,
    eigenvalues 2.14e-09, ground_energies 1.90e-12, ground_homo_lumo_gap 1.30e-12,
    heat_capacity 5.69e-09, profiles 2.80e-09, thermodynamics 2.49e-10
E_KS of N8_lamm2_a00: Rust 0.002862652173608389, reference 0.002862652173734779
    |diff| = 1.264e-13, U_ref = 2.096e-12, U_Rust = 1.214e-13
    tolerance = 3 (U_ref + U_Rust) + 1e-12 max(1, |x|) = 7.651e-12, ratio = 0.0165
PASS E_KS of N8_lamm2_a00 agrees within the tolerance
```

Text cell:

14. The ground-state comparisons

The next cell makes, for the three ground states, the comparisons of the cross-check program's ground-state classes (the class names are the names of its checks):

- `ground_energies`: E_{KS}, E_{band}, E_{int} with U_{Rust} measured;
- `ground_homo_lumo_gap`: HOMO and LUMO with U_{Rust} = the largest level uncertainty of the state, the gap with twice that;
- `ground_eigenvalues`: every level present in both label sets, key by key (a Rust label is turned into a rank by subtracting `label_min`);
- `excited_delta_scf`: Delta-SCF and $E_{excited}$, with the matrix-wide Delta-SCF uncertainty;
- `ground_occupations_and_groups` (not a ratio): the same occupied levels.

The uncertainties were measured in the cells above. The comparisons of the energy-momentum tensor follow in the cell after this one.

In [15]:

```
def rank_key(n2, j, parity, rank):
    """The level key n2:j:parity:rank as a text, for example 1:-1:even:2."""
    return f"{int(n2)}:{'+1' if int(j) > 0 else '-1'}:{parity}:{int(rank)}"


U_DSCF = RK4_FACTOR * WIDE["refined_delta_scf"]  # matrix-wide Delta-SCF uncertainty
for sid in GROUND_IDS:
    sc, U = NEW[sid]["scalars"], U_RUST[sid]
    for k in ("E_KS", "E_band", "E_int"):
        if not (sid == "N8_lamm2_a00" and k == "E_KS"):  # made in section 13
            compare("ground_energies", f"{sid} {k}", float(SUMMARY[sid][k]),
                    sc[k]["value"], sc[k]["U"], U[k])
    for k, u in (("HOMO", U["levels"]), ("LUMO", U["levels"]),
                 ("KS_gap", 2.0 * U["levels"])):
        compare("ground_homo_lumo_gap", f"{sid} {k}", float(SUMMARY[sid][k]),
                sc[k]["value"], sc[k]["U"], u)
    rust_levels = {}  # key -> (eps, occupation) of the committed Rust levels
    for x in read_csv(f"{KS}/results/ground/levels/{sid}.csv"):
        rank = int(x["label"]) - int(x["label_min"])
        rust_levels[rank_key(x["n2"], x["j"], x["parity"], rank)] = (
            float(x["eps"]), float(x["f"]))
    lev = NEW[sid]["levels"]
    ref_levels = {rank_key(*k): (e, u, f)
                  for k, e, u, f in zip(lev["keys"], lev["eps"], lev["U"], lev["f"])}
    common = sorted(set(rust_levels) & set(ref_levels))
    for k in common:
        compare("ground_eigenvalues", f"{sid} level {k}", rust_levels[k][0],
                ref_levels[k][0], ref_levels[k][1], U["levels"])
    occupied_rust = sorted(k for k, v in rust_levels.items() if v[1] > 0)
    occupied_ref = sorted(k for k, v in ref_levels.items() if v[2] > 0)
    compare("excited_delta_scf", f"{sid} delta_SCF", float(EXCITED[sid]["delta_SCF"]),
            sc["delta_SCF"]["value"], sc["delta_SCF"]["U"], U_DSCF)
    compare("excited_delta_scf", f"{sid} E_excited", float(EXCITED[sid]["E_excited"]),
            sc["E_excited"]["value"], sc["E_excited"]["U"], U["E_KS"] + U_DSCF)
```

```
    say(f"{sid}: {len(rust_levels)} Rust levels, {len(ref_levels)} reference levels, "
        f"{len(common)} compared; {len(occupied_rust)} occupied in both")
    check(occupied_rust == occupied_ref, f"the same occupied levels in {sid}",
          record=f"{KS}/reports/ks-crosscheck.json, check "
                 "ground_occupations_and_groups")
```

Out [15]:

```
N8_lamm2_a00: 64 Rust levels, 22 reference levels, 22 compared; 2 occupied in both
PASS the same occupied levels in N8_lamm2_a00
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    ground_occupations_and_groups
N688_lam0_a00: 164 Rust levels, 137 reference levels, 137 compared; 12 occupied in both
PASS the same occupied levels in N688_lam0_a00
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    ground_occupations_and_groups
N136_lamp2_a20: 1248 Rust levels, 354 reference levels, 354 compared; 6 occupied in both
PASS the same occupied levels in N136_lamp2_a20
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    ground_occupations_and_groups
```

Text cell:

```
The next cell adds the comparisons of the energy-momentum tensor and of the
adiabaticity, again in the classes of the cross-check program:

- ‘emt_integrals‘: the five integrals $\int\rho$, $\int p_3$, $\int p_t$, $\int p_0$,
  $\int n$;
- ‘emt_brane_tip_values‘: $\rho$, $p_3$, $p_t$, $p_0$ at the brane and at the tip;
- ‘ground_profiles‘: the ten profiles at all 151 points, with the scale = the
  largest value of the profile;
- ‘exchange_delta_E_x‘: $\Delta E_x$, whose Rust uncertainty is derived from the
  measured uncertainty of the profile $Q$ (twice its relative size, because
  $\Delta E_x$ is quadratic in $Q$);
- ‘adiabatic_dE_da4‘: $dE/da_4$ in its energy-momentum form (uncertainty from the
  integrals of $p_3$ and $p_t$) and as a difference quotient (matrix-wide);
- ‘adiabatic_Q_max‘: $Q_{max}$, and when it is not zero also its matrix element and
  its level spacing; the maximising pair of levels must be the same.

The pair of levels of $Q_{max}$ is printed with the Rust labels and the reference
ranks; for the sector of that pair (even parity, $j = +1$) the lowest particle label
is 0, so labels and ranks coincide.
```

In [16]:

```
U_ADIABATIC = RK4_FACTOR * WIDE["refined_adiabatic_derivatives"]  # relative
for sid in GROUND_IDS:
    sc, U, e = NEW[sid]["scalars"], U_RUST[sid], EMT[sid]
    for k in ("rho", "p3", "p_t", "p0", "n"):
        compare("emt_integrals", f"{sid} int_{k}", float(e["int_" + k]),
                sc["int_" + k]["value"], sc["int_" + k]["U"], U["int_" + k])
```

```
    for k in ("rho", "p3", "p_t", "p0"):
        for end in ("brane", "tip"):
            name = f"{k}_{end}"
            compare("emt_brane_tip_values", f"{sid} {name}", float(e[name]),
                    sc[name]["value"], sc[name]["U"], U[name])
    stored = read_csv(f"{KS}/results/ground/profiles/{sid}.csv")
    for name in PROFILE_NAMES:
        values = [float(row[name]) for row in stored]
        top = max(max(abs(v) for v in values), 1e-300)  # the scale of the profile
        ref = NEW[sid]["profiles"][name]
        for i, row in enumerate(stored):
            compare("ground_profiles", f"{sid} {name}(y = {float(row['y']):.2f})",
                    values[i], ref["value"][i], ref["U"][i],
                    U["profile_" + name][0], scale=top)
    u_q, top_q = U["profile_Q"]
    dex = float(e["deltaE_x_exact_fock"])
    compare("exchange_delta_E_x", f"{sid} deltaE_x", dex,
            sc["deltaE_x_exact_fock"]["value"], sc["deltaE_x_exact_fock"]["U"],
            RK4_FACTOR * 2.0 * (u_q / RK4_FACTOR) / max(top_q, 1e-300) * abs(dex))
    a = ADIABATIC[sid]
    compare("adiabatic_dE_da4", f"{sid} dE_da4_emt", float(a["dE_da4_emt"]),
            sc["dE_da4_emt"]["value"], sc["dE_da4_emt"]["U"],
            3.0 * (U["int_p3"] + U["int_p_t"]))
    fd = float(a["dE_da4_finite_difference"])
    compare("adiabatic_dE_da4", f"{sid} dE_da4_finite_difference", fd,
            sc["dE_da4_fd"]["value"], sc["dE_da4_fd"]["U"], U_ADIABATIC * abs(fd))
    q_rust, ad = float(a["Q_max"]), NEW[sid]["adiabatic"]
    compare("adiabatic_Q_max", f"{sid} Q_max", q_rust, ad["Q_max"], ad["U_Q_max"],
            U_ADIABATIC * abs(q_rust))
    if q_rust > 0:
        top = ad["top"][0]  # the reference's maximising pair
        hole, particle = [s.strip() for s in a["Q_max_pair"].split("->")]
        say(f"{sid}: Q_max = {q_rust:.10f} for the pair {hole} -> {particle} (Rust), "
            f"{top['hole']} -> {top['particle']} (reference)")
        check((hole, particle) == (top["hole"], top["particle"]),
              f"both solvers find the same maximising pair in {sid}")
        me = float(a["Q_max_matrix_element"])
        compare("adiabatic_Q_max", f"{sid} Q_max matrix element", me,
                top["matrix_element"], top["U_matrix_element"], U_ADIABATIC * abs(me))
        compare("adiabatic_Q_max", f"{sid} Q_max delta eps", float(a["Q_max_delta_eps"]),
                top["delta_eps"], top["U_delta_eps"], 2.0 * U["levels"])
ground_rows = [r for r in ROWS if r[1].split()[0] in GROUND_IDS]
say(f"{len(ground_rows)} ground-state comparisons so far")
check(all(r[8] <= 1.0 for r in ground_rows),
      "every ground-state comparison of the subset passes the tolerance rule")
```

Out [16]:

```
N688_lam0_a00: Q_max = 0.0934505917 for the pair 11:+1:even:0 -> 11:+1:even:1 (Rust),
    11:+1:even:0 -> 11:+1:even:1 (reference)
PASS both solvers find the same maximising pair in N688_lam0_a00
N136_lamp2_a20: Q_max = 0.0421562349 for the pair 4:+1:even:0 -> 4:+1:even:1 (Rust),
    4:+1:even:0 -> 4:+1:even:1 (reference)
```

```
PASS both solvers find the same maximising pair in N136_lamp2_a20
5122 ground-state comparisons so far
PASS every ground-state comparison of the subset passes the tolerance rule
```

Text cell:

```
## 15. The thermal comparisons

The next cell makes the thermal comparisons of the two thermal states, in the
cross-check classes ‘thermo_state_functions‘ ($\mu$, $E$, $S$, $F$, $\Omega$ in both
of its forms) and ‘thermo_derivatives‘ ($C_V = T\,dS/dT$, $dE/dT$, $-dF/dT$). The
Rust uncertainties of $F = E - TS$ and $\Omega = F - \mu N$ are added up from those
of $E$, $S$ and $\mu$ ($U_F = U_E + T U_S$, $U_\Omega = U_F + N U_\mu$). $dE/dT$ and
$-dF/dT$ are difference quotients of energies over $dT = 0.01\,T$, so each solver’s
noise floor is added: $N \times$ (the Rust root tolerance $10^{-13}$) $/dT$ and
$N \times 10^{-12}/dT$ for the reference.

Three classes of the full cross-check are not repeated here: the particle-hole
lists, the exact-Fock variant and the rescaling partners (other solver runs that this
subset does not make), and for thermal states the sea-hole diagnostic and the
40-digit chemical potential (Notebook 16c computes the latter for all 45 thermal
states with $N = 8$, among them N8_lamm1_a00_T10).
```

In [17]:

```
ROOT_TOLERANCE = RUST_PARAMS["numerics"]["rootTolerance"]  # 1e-13
U_HEAT = RK4_FACTOR * WIDE["refined_heat_capacity"]  # relative, matrix-wide
for sid in THERMAL_IDS:
    th, U, row = NEW[sid]["thermo"], U_RUST[sid], THERMO[sid]
    T, N = NEW[sid]["T"], NEW[sid]["N"]
    u_f = U["E_KS"] + T * U["entropy"]
    for k, u in (("mu", U["mu"]), ("E", U["E_KS"]), ("entropy", U["entropy"]),
                 ("F", u_f), ("Omega_direct", u_f + N * U["mu"]),
                 ("Omega_F_minus_muN", u_f + N * U["mu"])):
        compare("thermo_state_functions", f"{sid} {k}", float(row[k]),
                th[k]["value"], th[k]["U"], u)
    dT = 0.01 * T
    for k in ("C_V", "C_V_from_dEdT", "minus_dFdT"):
        x = float(row[k])
        quotient = k != "C_V"  # a difference quotient of energies
        compare("thermo_derivatives", f"{sid} {k}", x, th[k]["value"],
                th[k]["U"] + (N * 1e-12 / dT if quotient else 0.0),
                U_HEAT * max(abs(x), 1e-6)
                + (N * ROOT_TOLERANCE / dT if quotient else 0.0))
    r = next(r for r in ROWS if r[1] == f"{sid} mu")
    say(f"{sid}: mu Rust {r[2]!r}, reference {r[3]!r}, ratio {r[8]:.4f}")
thermal_rows = [r for r in ROWS if r[1].split()[0] in THERMAL_IDS]
say(f"{len(thermal_rows)} thermal comparisons; {len(ROWS)} comparisons in all")
check(len(ROWS) == 5140 and all(r[8] <= 1.0 for r in ROWS),
      "all 5140 comparisons of the subset pass the tolerance rule")
```

Out [17]:

```
N8_lamm1_a00_T10: mu Rust 0.2100104497339036, reference 0.21001044973430544, ratio 0.0438
N8_lam0_a15_T50: mu Rust -0.03040298055041425, reference -0.03040298055126662, ratio
    0.0825
18 thermal comparisons; 5140 comparisons in all
PASS all 5140 comparisons of the subset pass the tolerance rule
```

Text cell:

```
## 16. Reproducing the committed cross-check report

The cross-check program wrote every comparison of single numbers into the table
‘Revision/kohn_sham/reports/ks-crosscheck-table.csv‘ (levels and profile points are
too many; the report keeps only their worst case). The next cell finds every one of
our comparisons in that table and checks that our tolerance and ratio agree with the
stored ones to the four significant digits the table keeps. Then it reads, from the
sentences of the cross-check report, the worst ratio of six classes and the case
where it occurred, and checks that our subset contains exactly that case with that
ratio: the hardest comparisons of the whole cross-check were made again here.
```

In [18]:

```
TABLE = {r["case"]: r for r in read_csv(f"{KS}/reports/ks-crosscheck-table.csv")}
matched, largest = 0, 0.0
for cls, case, *_, tolerance, diff, ratio in ROWS:
    if case in TABLE:
        matched += 1
        stored = TABLE[case]
        largest = max(largest, abs(ratio - float(stored["ratio"])))
        if abs(ratio - float(stored["ratio"])) > 6e-4 or \
                abs(tolerance - float(stored["tolerance"])) > 1e-3 * tolerance:
            say(f"differs: {case}: {ratio:.4f} vs {stored['ratio']}")
say(f"{matched} of our comparisons are rows of the committed table; the largest "
    f"difference of the ratio is {largest:.1e}")
check(matched == 97 and largest <= 6e-4,
      "our tolerances and ratios equal the 97 matching rows of the committed table",
      record=f"{KS}/reports/ks-crosscheck-table.csv")

WORST_CASES = {}  # class -> (worst ratio in the report, its case)
pattern = re.compile(r"worst \|diff\|/tolerance ([0-9.]+) \((.+?): Rust ")
for c in REPORTS["cross-check"]["checks"]:
    found = pattern.search(c["detail"])
    if found and c["name"] in ("ground_eigenvalues", "ground_profiles",
                               "emt_brane_tip_values", "adiabatic_Q_max",
                               "thermo_state_functions", "thermo_derivatives"):
        WORST_CASES[c["name"]] = (float(found.group(1)), found.group(2))
for cls, (ratio, case) in sorted(WORST_CASES.items()):
    mine = max((r for r in ROWS if r[0] == cls), key=lambda r: r[8])
    say(f"{cls}: report {ratio:.3f} ({case}); this notebook {mine[8]:.3f} "
        f"({mine[1]})")
    check(mine[1] == case and abs(mine[8] - ratio) <= 6e-4,
          f"the worst case of {cls} is reproduced",
          record=f"{KS}/reports/ks-crosscheck.json, check {cls}")
```

Out [18]:

```
97 of our comparisons are rows of the committed table; the largest difference of the
    ratio is 5.0e-05
PASS our tolerances and ratios equal the 97 matching rows of the committed table
     reproduces Revision/kohn_sham/reports/ks-crosscheck-table.csv
adiabatic_Q_max: report 0.314 (N136_lamp2_a20 Q_max matrix element); this notebook 0.314
    (N136_lamp2_a20 Q_max matrix element)
PASS the worst case of adiabatic_Q_max is reproduced
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check adiabatic_Q_max
emt_brane_tip_values: report 0.495 (N8_lamm2_a00 rho_tip); this notebook 0.495
    (N8_lamm2_a00 rho_tip)
PASS the worst case of emt_brane_tip_values is reproduced
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check emt_brane_tip_values
ground_eigenvalues: report 0.324 (N136_lamp2_a20 level 1:-1:even:2); this notebook 0.324
    (N136_lamp2_a20 level 1:-1:even:2)
PASS the worst case of ground_eigenvalues is reproduced
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check ground_eigenvalues
ground_profiles: report 0.495 (N8_lamm2_a00 rho(y = -3.00)); this notebook 0.495
    (N8_lamm2_a00 rho(y = -3.00))
PASS the worst case of ground_profiles is reproduced
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check ground_profiles
thermo_derivatives: report 0.226 (N8_lam0_a15_T50 C_V); this notebook 0.226
    (N8_lam0_a15_T50 C_V)
PASS the worst case of thermo_derivatives is reproduced
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check thermo_derivatives
thermo_state_functions: report 0.284 (N8_lam0_a15_T50 E); this notebook 0.284
    (N8_lam0_a15_T50 E)
PASS the worst case of thermo_state_functions is reproduced
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    thermo_state_functions
```

Text cell:

```
## 17. The agreement in pictures

The next cell draws the levels of N136_lamp2_a20, the state with the largest level
ratio: for each of its compared levels the ratio $|x_{Rust} - x_{ref}|/$tolerance
against its energy. Every point lies below the line ratio = 1.
```

In [19]:

```
rows = [r for r in ROWS if r[0] == "ground_eigenvalues"
        and r[1].startswith("N136_lamp2_a20 ")]
worst = max(rows, key=lambda r: r[8])
fig, ax = plt.subplots(figsize=(7.5, 4.3))
ax.semilogy([r[3] for r in rows], [max(r[8], 1e-6) for r in rows], "o",
            color=PALETTE[0], ms=4, alpha=0.8, label="one level of N136_lamp2_a20")
ax.semilogy([worst[3]], [worst[8]], "o", color=PALETTE[1], ms=9,
            label=f"largest ratio {worst[8]:.3f}: level "
                  f"{worst[1].split('level ')[1]}")
```

```
ax.axhline(1.0, color="k", lw=1.2, label="ratio 1: the tolerance")
ax.set_ylim(1e-6, 3.0)
ax.set_xlabel("level energy $\\varepsilon$ (reference value, units of $m$)")
ax.set_ylabel("$|\\varepsilon_{Rust} - \\varepsilon_{ref}|$ / tolerance")
ax.set_title(f"{len(rows)} levels of N136_lamp2_a20 compared")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "level_agreement",
            "Agreement of the two solvers for every level of the ground state "
            "N136_lamp2_a20 ($N = 136$, $\\lambda = +\\lambda_2$, $a_{4,0} = 2$): the "
            "difference of the two level energies divided by the tolerance $3(U_{ref} "
            "+ U_{Rust}) + 10^{-12}\\max(1, |\\varepsilon|)$, against the level energy "
            "in units of $m$ (vertical axis logarithmic; ratios below one millionth "
            "are drawn at $10^{-6}$). Every point lies below the line 1, so every "
            f"level passes; the largest ratio, {worst[8]:.3f}, is the largest of all "
            "9616 level comparisons of the full cross-check.")
check(len(rows) == 354 and worst[8] < 1.0,
      "all 354 levels of N136_lamp2_a20 agree within the tolerance")
```

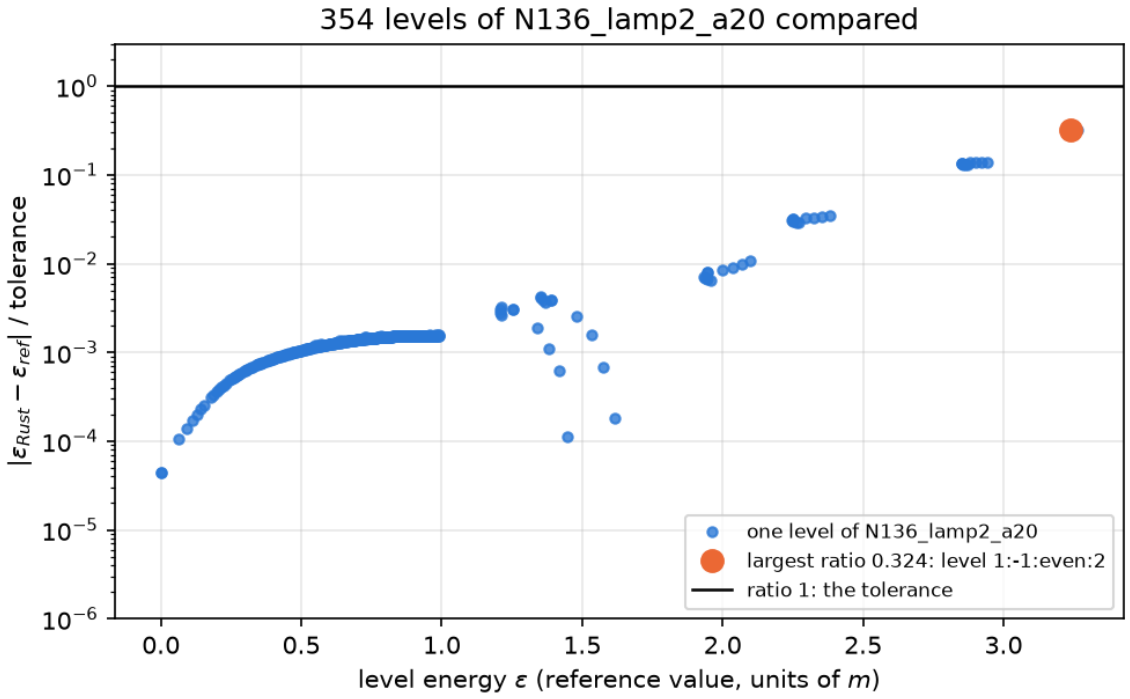

Figure 442: Agreement of the two solvers for every level of the ground state N136_lamp2_a20 ($N = 136$, $\lambda = +\lambda_2$, $a_{4,0} = 2$): the difference of the two level energies divided by the tolerance $3(U_{ref} + U_{Rust}) + 10^{-12}\max(1, |\varepsilon|)$, against the level energy in units of m (vertical axis logarithmic; ratios below one millionth are drawn at 10^{-6}). Every point lies below the line 1, so every level passes; the largest ratio, 0.324, is the largest of all 9616 level comparisons of the full cross-check. (Notebook 16a, figure 4.)

Out [19]:

```
Figure 16a.4 saved as Revision/textbook/figures/16a_4_level_agreement.png
PASS all 354 levels of N136_lamp2_a20 agree within the tolerance
```

Text cell:

```
The next cell draws the profile in which the largest ratio of the whole cross-check
occurred: the energy density $\rho(y)$ of N8_lamm2_a00. Top: $\rho$ from both
solvers (the reference at every fifth point). Bottom: the ratio at each of the 151
points for $\rho$ and for $p_0$.
```

In [20]:

```
sid = "N8_lamm2_a00"
stored = read_csv(f"{KS}/results/ground/profiles/{sid}.csv")
y = np.array([float(row["y"]) for row in stored])
fig, (top, bottom) = plt.subplots(2, 1, figsize=(7.5, 6.6), sharex=True)
top.plot(y, [float(row["rho"]) for row in stored], color=PALETTE[0], lw=1.8,
         label="Rust solver")
top.plot(y[::5], NEW[sid]["profiles"]["rho"]["value"][::5], "o", color=PALETTE[1],
         ms=5, mfc="none", label="reference solver")
top.set_ylabel("$\\rho(y)$ (proper energy density, units of $m$)")
top.set_title("Energy density of N8_lamm2_a00")
top.legend(fontsize=8)
largest = 0.0
for colour, name in ((PALETTE[0], "rho"), (PALETTE[2], "p0")):
    ratios = [r[8] for r in ROWS if r[0] == "ground_profiles"
              and r[1].startswith(f"{sid} {name}(")]
    largest = max(largest, max(ratios))
    bottom.semilogy(y, np.maximum(ratios, 1e-6), "-", color=colour, lw=1.5,
                    label=f"profile {name}")
bottom.axhline(1.0, color="k", lw=1.2, label="ratio 1: the tolerance")
bottom.set_ylim(1e-6, 3.0)
bottom.set_xlabel("hidden coordinate $y$ (tip $y = -3$, brane $y = 0$)")
bottom.set_ylabel("$|$Rust $-$ reference$|$ / tolerance")
bottom.legend(fontsize=8, loc="center right")
fig.tight_layout()
save_figure(fig, "profile_agreement",
            "The profile with the largest ratio of the whole cross-check. Top: the "
            "proper energy density $\\rho(y)$ of the ground state N8_lamm2_a00 "
            "($N = 8$, $\\lambda = -\\lambda_2$, $a_{4,0} = 0$) from the Rust solver "
            "(line) and the reference solver (circles), in units of $m$, against the "
            "hidden coordinate $y$. Bottom: the difference divided by the tolerance "
            "at each of the 151 points for $\\rho$ and $p_0$ (logarithmic). The "
            f"largest ratio, {largest:.3f}, is at the tip $y = -3$, where the "
            "densities are largest and both solvers are least accurate.")
```

Out [20]:

```
Figure 16a.5 saved as Revision/textbook/figures/16a_5_profile_agreement.png
```

Text cell:

```
The next cell shows where the tolerances come from. For every comparison of single
numbers and of levels in the subset it places a point at the reference uncertainty
```

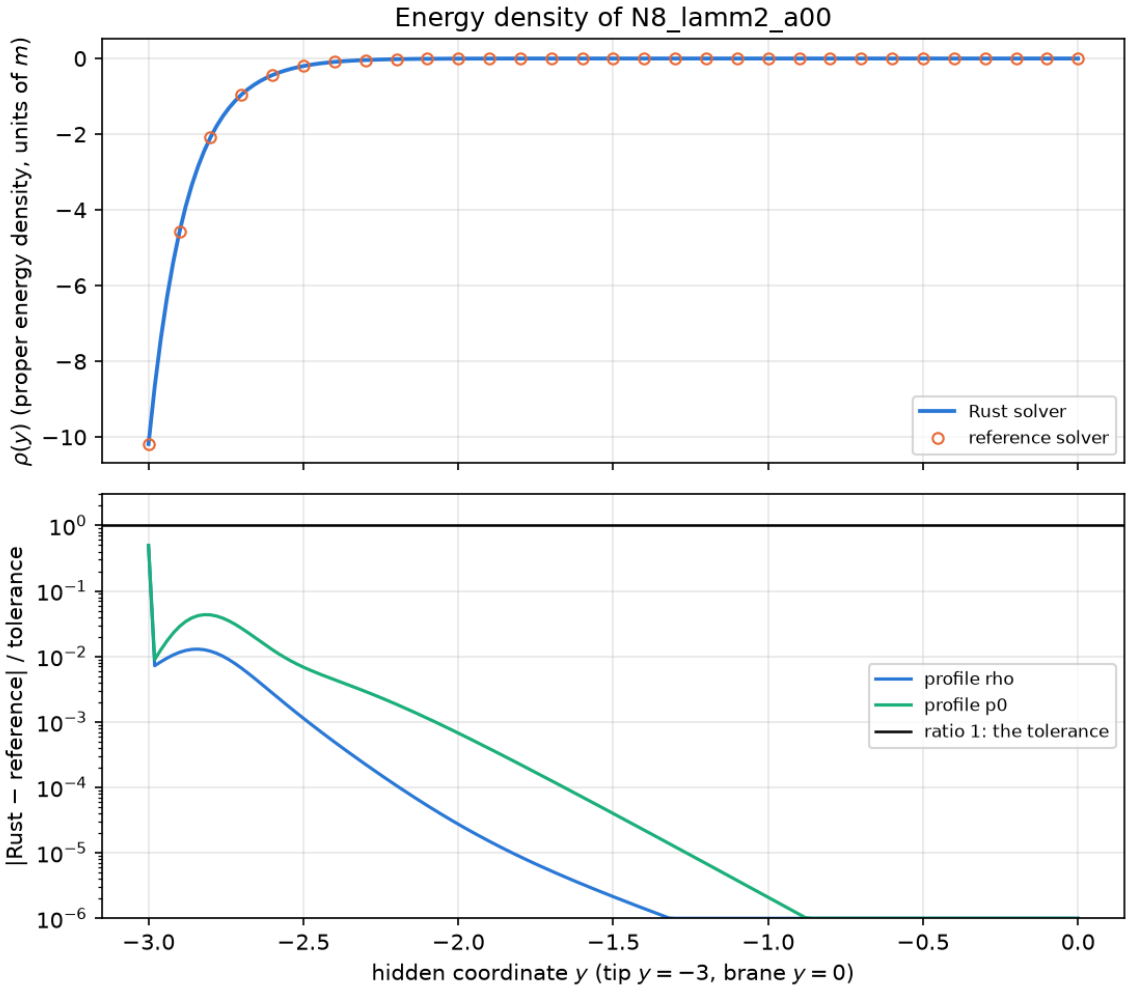

Figure 443: The profile with the largest ratio of the whole cross-check. Top: the proper energy density $\rho(y)$ of the ground state N8_lamm2_a00 ($N = 8$, $\lambda = -\lambda_2$, $a_{4,0} = 0$) from the Rust solver (line) and the reference solver (circles), in units of m, against the hidden coordinate y. Bottom: the difference divided by the tolerance at each of the 151 points for ρ and p_0 (logarithmic). The largest ratio, 0.495, is at the tip $y = -3$, where the densities are largest and both solvers are least accurate. (Notebook 16a, figure 5.)

U_{ref} (horizontal) and the Rust uncertainty U_{Rust} (vertical). Points above the diagonal are comparisons in which the Rust uncertainty dominates the tolerance.

In [21]:

```
groups = {"ground states, single numbers": [], "ground states, levels": [],
          "thermal states": []}
for r in ROWS:
    if r[0] == "ground_profiles" or r[4] <= 0.0 or r[5] <= 0.0:
        continue  # profile points are drawn elsewhere; a zero U has no logarithm
    if r[1].split()[0] in THERMAL_IDS:
        groups["thermal states"].append(r)
    elif r[0] == "ground_eigenvalues":
        groups["ground states, levels"].append(r)
    else:
        groups["ground states, single numbers"].append(r)
fig, ax = plt.subplots(figsize=(6.6, 5.6))
for colour, (label, rows) in zip(PALETTE, groups.items()):
    ax.loglog([r[4] for r in rows], [r[5] for r in rows], "o", color=colour, ms=4,
              alpha=0.75, label=f"{label} ({len(rows)})")
span = np.array([1e-18, 1e-5])
ax.loglog(span, span, "k-", lw=1.0, label="$U_{Rust} = U_{ref}$")
ax.set_xlabel("$U_{ref}$ (Richardson uncertainty of the reference)")
ax.set_ylabel("$U_{Rust}$ (16/15 of canonical minus refined)")
ax.set_title("Who sets the tolerance?")
ax.legend(fontsize=8)
above = sum(1 for rows in groups.values() for r in rows if r[5] > r[4])
total = sum(len(rows) for rows in groups.values())
save_figure(fig, "uncertainty_budget",
            "The two uncertainties that make the tolerance, for every comparison of "
            "single numbers and of levels in the five states of the subset: the "
            "Richardson uncertainty $U_{ref}$ of the reference solver (horizontal) "
            "and the measured uncertainty $U_{Rust}$ of the Rust solver (vertical), "
            "both logarithmic and in the units of the quantity. Points above the "
            "diagonal are dominated by the Rust uncertainty, points below it by the "
            f"reference; {above} of the {total} points lie above it. Comparisons in "
            "which one uncertainty is exactly zero are left out.")
say(f"{above} of {total} drawn comparisons have U_Rust > U_ref")
check(total > 500, "the uncertainty budget has more than 500 comparisons")
```

Out [21]:

```
Figure 16a.6 saved as Revision/textbook/figures/16a_6_uncertainty_budget.png
547 of 602 drawn comparisons have U_Rust > U_ref
PASS the uncertainty budget has more than 500 comparisons
```

Text cell:

```
The last figure gathers all 5140 comparisons of the subset by class. Each dot is one
comparison, spread a little sideways so that dots do not hide each other; ratios
below one millionth (including exact agreement) are drawn at $10^{-6}$. The orange
diamond marks the largest ratio of each class.
```

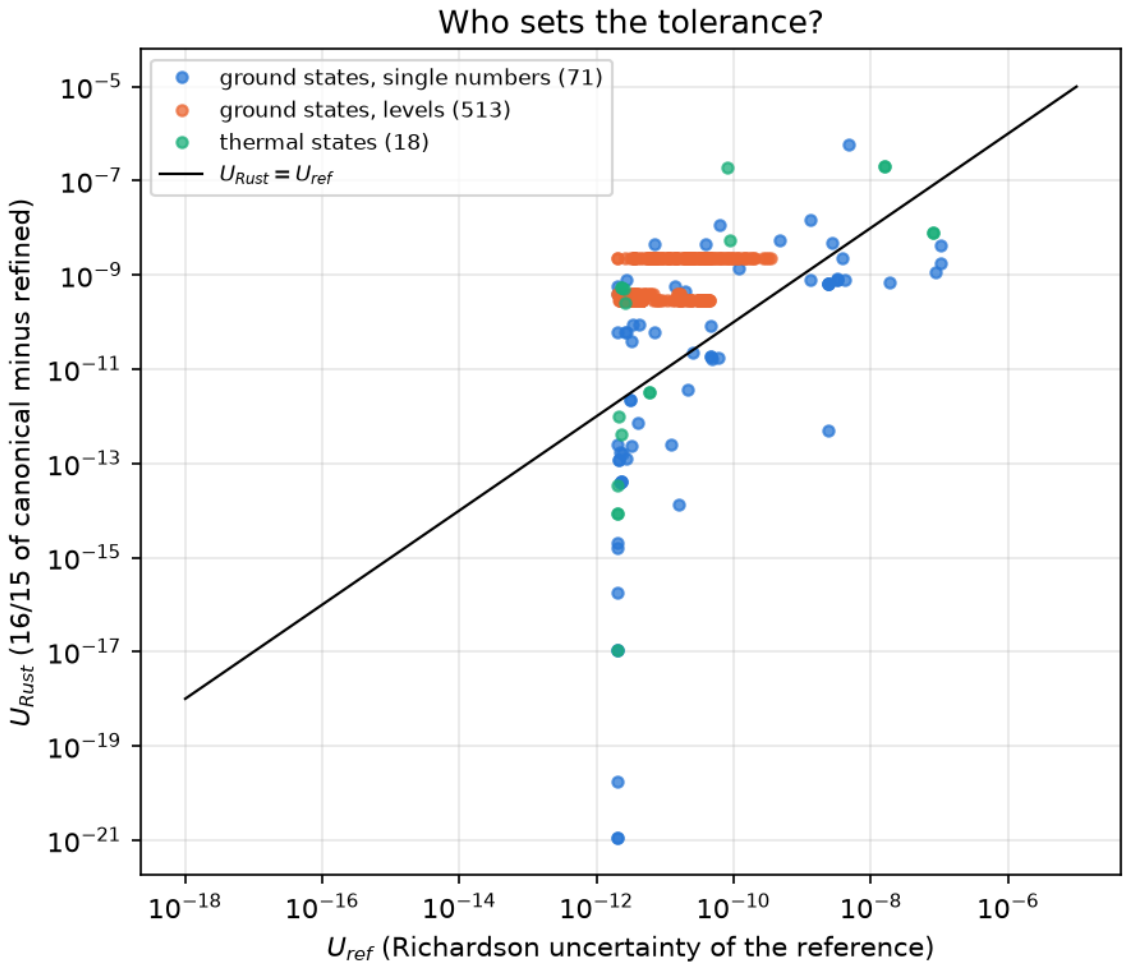

Figure 444: The two uncertainties that make the tolerance, for every comparison of single numbers and of levels in the five states of the subset: the Richardson uncertainty U_{ref} of the reference solver (horizontal) and the measured uncertainty U_{Rust} of the Rust solver (vertical), both logarithmic and in the units of the quantity. Points above the diagonal are dominated by the Rust uncertainty, points below it by the reference; 547 of the 602 points lie above it. Comparisons in which one uncertainty is exactly zero are left out. (Notebook 16a, figure 6.)

In [22]:

```
classes = ["ground_energies", "ground_homo_lumo_gap", "ground_eigenvalues",
           "excited_delta_scf", "emt_integrals", "emt_brane_tip_values",
           "ground_profiles", "exchange_delta_E_x", "adiabatic_dE_da4",
           "adiabatic_Q_max", "thermo_state_functions", "thermo_derivatives"]
rng = np.random.default_rng(12345)  # fixed seed: the same picture in every run
fig, ax = plt.subplots(figsize=(9.0, 5.0))
worst_by_class = {}
for i, cls in enumerate(classes):
    ratios = np.array([r[8] for r in ROWS if r[0] == cls])
    jitter = rng.uniform(-0.28, 0.28, size=len(ratios))
    ax.semilogy(i + jitter, np.maximum(ratios, 1e-6), "o", color=PALETTE[0], ms=2.5,
                alpha=0.35)
    worst_by_class[cls] = float(ratios.max())
    ax.semilogy([i], [max(ratios.max(), 1e-6)], "D", color=PALETTE[1], ms=7)
    ax.text(i, 1.6, f"{len(ratios)}", ha="center", fontsize=7)
ax.axhline(1.0, color="k", lw=1.2)
ax.set_xticks(range(len(classes)))
ax.set_xticklabels([c.replace("_", " ") for c in classes], rotation=40, ha="right",
                   fontsize=8)
ax.set_ylim(1e-6, 4.0)
ax.set_ylabel("$|x_{Rust} - x_{ref}|$ / tolerance")
ax.set_title("All comparisons of the subset (number of comparisons above each class)")
fig.tight_layout()
top_class = max(worst_by_class, key=worst_by_class.get)
save_figure(fig, "ratios_by_class",
            f"All {len(ROWS)} comparisons of the five states of the subset, grouped "
            "by the class of the cross-check: each dot is one ratio of the difference "
            "of the two solvers to the tolerance (logarithmic; ratios below "
            "$10^{-6}$ drawn at $10^{-6}$), the diamond is the largest ratio of the "
            "class, the number above a class counts its comparisons, and the black "
            "line is the tolerance. Every dot lies below it; the largest ratio, "
            f"{worst_by_class[top_class]:.3f}, belongs to the tip value of the energy "
            "density of N8_lamm2_a00.")
for cls in classes:
    say(f"  {cls:24s} largest ratio {worst_by_class[cls]:.4f}")
top_row = max(ROWS, key=lambda r: r[8])  # the comparison with the largest ratio
say(f"the largest ratio of the subset: {top_row[8]:.4f} ({top_row[1]})")
check(max(worst_by_class.values()) < 0.5
      and top_row[1] == "N8_lamm2_a00 rho_tip",
      "no comparison uses half of its tolerance; the largest is rho at the tip")
```

Out [22]:

```
Figure 16a.7 saved as Revision/textbook/figures/16a_7_ratios_by_class.png
  ground_energies          largest ratio 0.1315
  ground_homo_lumo_gap     largest ratio 0.0007
  ground_eigenvalues       largest ratio 0.3244
  excited_delta_scf        largest ratio 0.0610
  emt_integrals            largest ratio 0.2088
  emt_brane_tip_values     largest ratio 0.4952
```

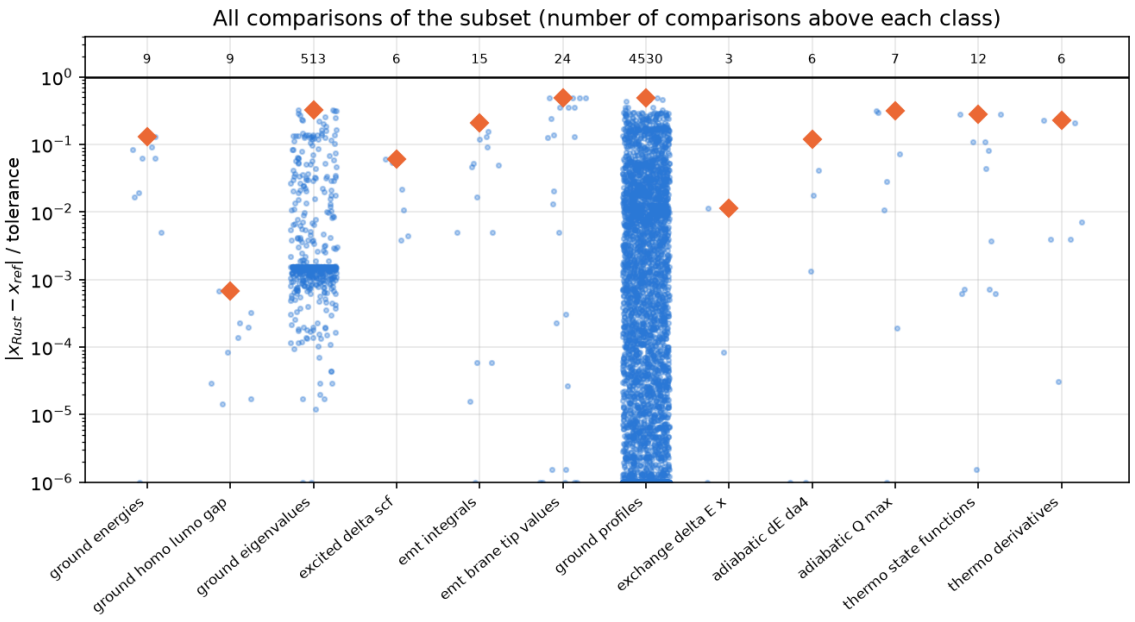

Figure 445: All 5140 comparisons of the five states of the subset, grouped by the class of the cross-check: each dot is one ratio of the difference of the two solvers to the tolerance (logarithmic; ratios below 10^{-6} drawn at 10^{-6}), the diamond is the largest ratio of the class, the number above a class counts its comparisons, and the black line is the tolerance. Every dot lies below it; the largest ratio, 0.495, belongs to the tip value of the energy density of N8_lamm2_a00. (Notebook 16a, figure 7.)

```
  ground_profiles          largest ratio 0.4952
  exchange_delta_E_x       largest ratio 0.0115
  adiabatic_dE_da4         largest ratio 0.1216
  adiabatic_Q_max          largest ratio 0.3142
  thermo_state_functions   largest ratio 0.2843
  thermo_derivatives       largest ratio 0.2257
the largest ratio of the subset: 0.4952 (N8_lamm2_a00 rho_tip)
PASS no comparison uses half of its tolerance; the largest is rho at the tip
```

Text cell:

```
## 18. The last check

The next cell confirms that every figure of this notebook was written, and prints
the number of checks that passed.
```

In [23]:

```
figure_files = [f"{FIGURE_FOLDER}/16a_{k}_{name}.png" for k, name in enumerate(
    ["grid_convergence", "uncertainty_validated", "eigenvalue_ratios",
     "level_agreement", "profile_agreement", "uncertainty_budget",
     "ratios_by_class"], start=1)]
check(all(output_file(f).is_file() for f in figure_files),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [23]:

```
PASS every figure file of this notebook exists
ALL 51 CHECKS PASSED (notebook 16a)
```

Text cell:

```
## 19. What this notebook showed

- The reference program, run again here on five states and on the fourth grid,
  reproduces the committed reference results: every number within one part in a
  billion (on the computer that built the book, byte for byte).
- Its grids converge like $h^2$: the differences shrink by a factor 4 from grid to
  grid, for $E_{KS}$ and for every level; two Richardson steps remove the $h^2$ and
  $h^4$ errors, and $U$ measures what is left. On a fourth grid every three-grid
  value of N136_lamp2_a20 moves by less than its $U$ (at most 0.787 of it, at the
  tip), so $U$ is a safe estimate of the remaining error.
- The Rust solver, run again with its canonical numerics, reproduces its committed
  results, and its refined run reproduces the recorded canonical-minus-refined
  differences, from which $U_{Rust} = \tfrac{16}{15}|canonical - refined|$.
- With the tolerance rule $|x_{Rust} - x_{ref}| \le 3(U_{ref} + U_{Rust}) +
  10^{-12}\,$scale, fixed before the comparison, all 5140 comparisons of the subset
  pass; our tolerances and ratios equal the 97 rows of the committed table, and the
```

```
  worst cases of six classes of the full cross-check (largest ratio 0.495) are
  reproduced exactly.
- What this does NOT show: both solvers implement the same functional, the same
  ASSUMED $Z_2$ brane, the same tip cutoff and the same filling CONVENTION, and both
  compute instantaneous (adiabatic) states along a PRESCRIBED BACKGROUND history. An
  error in those common inputs cannot be detected by comparing the two programs. The
  cross-check tests the numerics, not the physics model.
```

16.12 Line-by-line walk-through of Notebook 16a

The notebook has 23 code cells, In [1] to In [23]. This section explains every line of every one of them, in order. In the notebook each code cell is preceded by a text cell that says what the cell does; the numbers that the cells print are in Section 16.11 under the labels Out [k].

In [1], the set-up cell. It is the same in every notebook of the book except for the notebook's name and, in notebooks that run a Rust program, two more imports and the function `rust_program`; the walk-throughs of Notebooks 16b and 16c refer back to this paragraph. Its first 299 lines repeat the complete run instructions of Section 16.10 as **comment lines**: every line that starts with `#` is skipped by Python and is there only so that the notebook file carries its own instructions. Two lines of `=` signs around the title THE SET-UP mark where the code begins.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "16a"  # this notebook: chapter 16, example a
```

`import` loads a **module** (a part of Python or of an installed package) so that the cell can use it; the text after `#` on each line says what the module is for. `json` reads and writes JSON files, the text format in which most Revision records are stored (names and numbers in braces). `os` gives access to the operating system, here to read an environment variable. `textwrap` breaks long printed text into lines. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on Windows, macOS and Linux. `matplotlib` is the plotting package, and `matplotlib.pyplot` its drawing functions, called `plt` by convention. `Image` and `display` show a saved picture below a cell. `shutil` can find a program on the computer (here cargo, the Rust build tool) and `subprocess` can run one. The last line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"16a"`; the figure files are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
```

```
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called; the text in triple quotes under the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. The list `[here, *here.parents]` holds this folder followed by its parent, the parent of that, and so on up to the top of the disk (the star unpacks the parents into the list); the `for` loop takes them one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back; if none does, `raise` stops the notebook with an error message that says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`REPO` is the repository folder; it is never printed, because it differs from computer to computer while the printed output of the notebook must be the same everywhere. `os.environ.get(name, default)` reads an **environment variable** (a named text that a program receives from the computer) or returns the default; when you run the notebook the variable TEXTBOOK_OUTPUT_ROOT is not set, so `OUTPUT_ROOT` is the repository, while the book's checking tool sets it to a scratch folder so that a check never changes the repository. `repository_file` gives the full path of a repository file

for reading; `output_file` gives the full path at which to write one, after creating its folder (`mkdir` with `parents=True` creates missing parent folders too, and `exist_ok=True` makes it do nothing if the folder exists). `say` prints a text in lines of at most 89 characters (the width of a page of the book), continuation lines starting with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

`matplotlib.rcdefaults()` returns to the built-in plotting settings, so that the figures are the same on every computer, and `plt.rcParams.update` sets the figure size (7.0 by 4.2 inches), the letter size (10 points) and a faint grid. The braces `{...}` make a **dictionary**: pairs of a key and a value, written `key:  value`. A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/16a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary `{}` into the captions file; `newline="\n"` stores the same line end on every operating system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` numbers the figures 1, 2, 3, ... in the order in which they are saved (`setdefault` returns the number already stored for this name, or stores one more than the count so far), saves the figure as a PNG file with 150 dots per inch, cut to its content and without the program's name in the file (so that two runs write the same bytes), closes it, records its caption in the captions file

(`json.dumps` turns the dictionary into JSON text, with one entry per line and the keys sorted), shows the saved picture below the cell, and prints where it was saved. The caption given to it is the caption that the book prints under the figure.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

`PASSED` is an empty **list** (an ordered collection, in square brackets). `check` is the function behind every check of the notebook: if the statement `condition` is false, `raise AssertionError(...)` stops the notebook with an error that names the check; if it is true, the name is appended to `PASSED` and the line PASS followed by the name is printed, and a second line names the Revision record and its check when the argument `record` is given (`None` is Python's word for "nothing"). Python's own `assert` statement is not used, because Python started with the option `-O` skips it. `report` prints a key number as a line that starts with RESULT. `all_checks_passed` prints the last line of the notebook with the number of checks that passed; `len` is the length of a list.

```
def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
```

```
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")
```

`rust_program` builds a Rust program and returns where it is. `shutil.which("cargo")` looks for the program cargo; if it is not found (`is None`), the function stops with a message that says how to install Rust. `crate` is the folder that holds the file `Cargo.toml` (`.parent` removes the last part of a path). `subprocess.run([...])` runs cargo with the arguments in the list, exactly as if the release build command for this manifest were typed into a terminal; `capture_output=True` keeps cargo's messages instead of printing them, and `text=True` with `encoding="utf-8"` reads them as text. The **return code** of a program is 0 when it succeeded; otherwise the function prints the last 3000 characters of cargo's error messages (`[-3000:]` takes the end of a string) and stops. The `for` loop tries the program's name without and with the ending `.exe` (Linux and macOS have no ending, Windows has `.exe`) and returns the first one that exists, after printing that the program is ready.

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

The last statement prints the one output line of In [1].

In [2], the records and the problem definition of both solvers.

```
import csv  # reads tables stored as CSV files (comma-separated values)
import re  # finds patterns in text (used to read numbers out of report sentences)
import sys  # the list of folders in which Python looks for modules
import tempfile  # makes temporary folders outside the repository

import numpy as np  # arrays of numbers

KS = "Revision/kohn_sham"  # the folder of the Kohn-Sham record (repository path)
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

Four more modules: `csv` reads tables stored as CSV files (comma-separated values: one line per row, the entries separated by commas), `re` finds patterns in text (**regular expressions**, used below to read numbers out of the sentences of a report), `sys` gives access to the list of folders in which Python looks for modules, and `tempfile` makes temporary folders in the place the operating system keeps for temporary files, outside the repository (In [12] uses it for the raw outputs of the Rust runs). `numpy`, called `np`, holds arrays of numbers. `KS` is the folder of the Kohn-Sham record, and `PALETTE` a list of eight colours, written as hexadecimal codes, used in a fixed order by all figures.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
```

```
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def read_csv(relative):
    """Read a CSV file of the repository: a list of rows, each row a dictionary from
    the column names to the texts in that row."""
    with open(repository_file(relative), newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle))
```

Two small reading functions. `read_json` reads a repository file as text and turns it into Python objects with `json.loads` (dictionaries, lists, numbers and strings). `read_csv` opens a CSV file (`with open(...) as handle:` opens it and closes it again when the indented block ends), and `csv.DictReader` reads it row by row, each row a dictionary from the column names of the first line to the texts in that row; `list(...)` collects all rows.

```
REPORTS = {"reference solver": read_json(f"{KS}/reports/ks-reference.json"),
           "Rust solver": read_json(f"{KS}/reports/ks-rust-solver.json"),
           "cross-check": read_json(f"{KS}/reports/ks-crosscheck.json")}
counts = {}
for label, rep in REPORTS.items():
    summary = rep["summary"]  # {"checks": ..., "pass": ..., "fail": ...}
    counts[label] = (summary["pass"], summary["checks"])
    say(f"{label}: {summary['pass']} of {summary['checks']} checks PASS")
check(counts == {"reference solver": (37, 37), "Rust solver": (42, 42),
                 "cross-check": (31, 31)},
      "the three committed reports pass every check (37, 42 and 31)")
CROSS = {c["name"]: c for c in REPORTS["cross-check"]["checks"]}  # name -> check
for name in ["thermo_levels", "thermo_mu_rust_stated_bound"]:  # the two newest
    detail = CROSS[name]["detail"]  # the report's sentence about this check
    size = re.search(r"(\d+) comparisons", detail).group(1)  # a number in the text
    say(f"cross-check {name}: {size} comparisons, verdict {CROSS[name]['verdict']}")
```

`REPORTS` is a dictionary of the three committed reports: the reference solver's self-checks, the Rust solver's self-checks, and the cross-check. The loop goes through its pairs (`.items()`); each report has an entry `summary` with the numbers of checks and of PASS verdicts, which the loop stores in `counts` as a **tuple** (an unchangeable pair in round brackets) and prints. The check requires exactly 37 of 37, 42 of 42 and 31 of 31; Out [2] shows the three lines and the PASS line. `CROSS` is a dictionary from the name of each check of the cross-check to the check itself, which is again a dictionary with three entries: `name`, `verdict` (PASS or FAIL) and `detail`, the sentence in which the report describes what was compared and what was found (the `{... for c in ...}` form builds a dictionary in one line, as a list comprehension builds a list). The loop takes the two checks that were added to the cross-check on 2026-10-08 (Section 16.8). For each, `detail` is its sentence, and `re.search` looks in it for the pattern `(\d+) comparisons`: `\d` stands for any digit, `+` for one or more of them, and the round brackets mark the part to keep, which `.group(1)` returns (the `r` before the quotes keeps the backslash as it is). The two printed lines give 34378 and 270 comparisons and the verdict that the report records for each check, PASS; this agrees with the check above, which found 31 PASS verdicts among 31 checks. These two lines are printed information, not checks of the notebook: they do not count among its 51 checks.

```
RUST_PARAMS = read_json(f"{KS}/results/parameters.json")  # the Rust solver's
REF_PARAMS = read_json(f"{KS}/reference/results/parameters.json")  # the reference's
```

```
pairs = [("H", "H"), ("m", "m"), ("L_tipCutoff", "L"), ("dk", "dk"), ("v_t", "vt"),
         ("tipTheta", "tipTheta"), ("historyA", "historyA"),
         ("slicesA4", "slicesA4"), ("temperatures", "temperatures")]
differ = [rust_name for rust_name, ref_name in pairs
          if RUST_PARAMS["physics"][rust_name] != REF_PARAMS["physics"][ref_name]]
same_theory = (RUST_PARAMS["theoryInputs"]["ksTheorySha256"]
               == REF_PARAMS["theoryInputs"]["ksTheorySha256"])
for rust_name, ref_name in pairs:
    say(f"  {rust_name:13s} Rust {RUST_PARAMS['physics'][rust_name]}   reference "
        f"{REF_PARAMS['physics'][ref_name]}")
check(not differ and same_theory,
      "both solvers solve the same problem and read the same ks-theory.json",
      record=f"{KS}/reports/ks-crosscheck.json, check problem_definition_identical")
```

`RUST_PARAMS` and `REF_PARAMS` are the `parameters.json` files of the two solvers. The two programs name some constants differently (the tip cutoff is `L_tipCutoff` in one and `L` in the other), so `pairs` lists the names side by side. `differ` is built by a **list comprehension**, `[expression for ... in ... if condition]`, which collects the Rust name of every pair whose two values are not equal (`!=`); it must be empty. `same_theory` compares the SHA-256 fingerprints (a 64-character number computed from every byte of a file; two different files practically never have the same one) of the theory file that each solver read. The loop prints the nine pairs of values, the name padded to 13 characters (`:13s`). The check requires no difference and the same theory file, and names the cross-check's check `problem_definition_identical`, which it reproduces.

```
LAMBDAS = {}  # N -> {"lamp1": +lambda_1, ...}: the couplings of the reference
rust_cal = {c["N"]: c for c in RUST_PARAMS["couplingCalibration"]["values"]}
same_couplings = True
for cal in REF_PARAMS["derived"]["calibration"]:
    N = cal["N"]
    LAMBDAS[N] = {"lam0": 0.0, "lamp1": cal["lambda1"], "lamm1": -cal["lambda1"],
                  "lamp2": cal["lambda2"], "lamm2": -cal["lambda2"]}
    same_couplings &= (cal["lambda1"] == rust_cal[N]["lambda1"]
                       and cal["lambda2"] == rust_cal[N]["lambda2"])
    say(f"N = {N:5.0f}: lambda_1 = {cal['lambda1']}, lambda_2 = {cal['lambda2']}")
check(same_couplings, "the six couplings derived by the two solvers are identical",
      record=f"{KS}/reports/ks-crosscheck.json, check parameters_couplings")
```

Each solver derived the couplings λ_1 and λ_2 for each particle number on its own, from the same rule (Section 16.3). `rust_cal` is a **dictionary comprehension** that files the Rust values under their N. The loop goes through the reference's values, stores for each N the five couplings of the tags lam0, lamp1, lamm1, lamp2, lamm2 in `LAMBDAS` (they are used by the next cells), and compares with the Rust values; `same_couplings &= ...` keeps `same_couplings` true only while every comparison is true (`&=` combines with "and"). `{N:5.0f}` prints N in five places without decimals. Out [2] shows $\lambda_1 = 0.01946$, $\lambda_2 = 0.05838$ for $N = 8$, 0.0009298 and 0.002789 for $N = 136$, and 0.0001846 and 0.0005538 for $N = 688$, identical in both solvers.

In [3], the reference program and the five states.

```
sys.dont_write_bytecode = True  # do not write a __pycache__ folder into the record
sys.path.insert(0, str(repository_file(f"{KS}/reference")))  # Python looks here too
```

```
import run_reference as RR  # noqa: E402  the reference program (Revision code)

CO = RR.theory_coefficients()  # coefficients read and checked from ks-theory.json
say(f"functional coefficients: M_eff = m + {CO['cM']} lambda S, "
    f"v_v = {CO['cV']} lambda n, e_int = lambda ({CO['eS2']} S^2 + {CO['eN2']} n^2)")
```

`sys.dont_write_bytecode = True` stops Python from writing a folder of translated files (`__pycache__`) next to the imported program, which would otherwise appear inside the Revision record. `sys.path.insert(0, ...)` puts the folder of the reference program at the front of the list of folders in which Python looks for modules, so that `import run_reference as RR` finds `Revision/kohn_sham/reference/run_reference.py` and makes it available under the short name `RR`. Importing does not run the program: its last lines start the computation only when it is run as a program, so the import only defines its functions. (The comment `noqa: E402` tells a style checker that an import in the middle of a cell is intended.) `RR.theory_coefficients()` reads the coefficients of the functional from `ks-theory.json` as exact fractions; the printed line, Out [3], shows $M_{eff} = m + \frac{15}{16}\lambda S$, $v_v = -\frac{1}{16}\lambda n$ and $e_{int} = \lambda(\frac{15}{32}S^2 - \frac{1}{32}n^2)$, the formulas of Chapter 14.

```
def job_spec(N, tag, a4, T=None):
    """The reference program's description of one state (a dictionary)."""
    spec = {"id": RR.run_id(N, tag, a4, T), "N": N, "tag": tag,
            "lam": LAMBDAS[N][tag], "a4": a4, "sigma": RR.SIGMA[tag]}
    if T is not None:
        spec["T"] = T
    return spec
```

`job_spec` writes one state as the reference program describes it: a dictionary with the id (made by the program's own function `RR.run_id`, so that the names agree with the record's file names), N, the coupling tag and its value from `LAMBDAS`, the slice, the first-order mean-field size σ of the tag (`RR.SIGMA`: 0 for lam0, 0.1 for $\pm\lambda_1$, 0.3 for $\pm\lambda_2$; it decides how many levels the label set keeps), and, for a thermal state, the temperature. `T=None` makes the temperature optional.

```
SPECS = {}
for args in [(8.0, "lamm2", 0.0), (688.0, "lam0", 0.0), (136.0, "lamp2", 2.0),
             (8.0, "lamm1", 0.0, 0.01), (8.0, "lam0", 1.5, 0.05)]:
    spec = job_spec(*args)
    SPECS[spec["id"]] = spec
GROUND_IDS = ["N8_lamm2_a00", "N688_lam0_a00", "N136_lamp2_a20"]
THERMAL_IDS = ["N8_lamm1_a00_T10", "N8_lam0_a15_T50"]
print("state               N   lambda       a4,0  sigma  T")
for sid, spec in SPECS.items():
    print(f"{sid:18s} {spec['N']:4.0f}  {spec['lam']:+.5f}  {spec['a4']:4.1f}  "
          f"{spec['sigma']:4.1f}   {spec.get('T', 0.0):.2f}")
check(sorted(SPECS) == sorted(GROUND_IDS + THERMAL_IDS),
      "the five job descriptions have the ids of the subset")
```

The loop makes the five job descriptions; `job_spec(*args)` unpacks each tuple into the arguments of the function, and the descriptions are filed under their ids in `SPECS`. `GROUND_IDS` and `THERMAL_IDS` list the five ids in the order used below. The table printed by the loop (Out [3]) shows each state's N, coupling (with sign and five decimals, `:+.5f`), slice, σ and temperature (`spec.get("T",`

0.0) gives 0 when a state has no temperature). The check compares the sorted ids with the five ids of the subset.

In [4], the reference solver on the three ground states.

```
PER_GRID = {}  # (state id, G) -> the levels of the state on grid G (an array)
original_ground_on_grid = RR._ground_on_grid  # the reference's function for one grid


def ground_on_grid_recorded(co, spec, G, labels):
    """Call the reference's own function for one grid and keep a copy of its levels."""
    result = original_ground_on_grid(co, spec, G, labels)
    PER_GRID[(spec["id"], G)] = result["eps"].copy()
    return result
```

The reference's ground-state job calls the program's internal function `_ground_on_grid` once per grid and keeps only the extrapolated levels. To draw the convergence of every level (In [10]) we want the levels on each grid as well. `original_ground_on_grid` keeps the program's function under a second name. The new function `ground_on_grid_recorded` takes the same four arguments, calls the original function, stores a copy of the levels it returned (`result["eps"]`, a numpy array; `.copy()` makes an independent copy) in `PER_GRID` under the pair (state id, grid), and passes the result on unchanged. Such a function, which calls another and adds something around it, is called a **wrapper**.

```
RR._ground_on_grid = ground_on_grid_recorded  # the job now calls our wrapper
NEW = {}  # state id -> the new reference result (the content of its JSON file)
for sid in GROUND_IDS:
    NEW[sid] = RR.jsonable(RR.ground_job(CO, SPECS[sid])["data"])
    E = NEW[sid]["scalars"]["E_KS"]
    say(f"{sid}: {NEW[sid]['levels_in_set']} levels, E_KS = {E['value']:.13f} "
        f"(U = {E['U']:.1e})")
RR._ground_on_grid = original_ground_on_grid  # put the original function back
check(len(PER_GRID) == 9, "the levels of three states on three grids were kept")
```

`RR._ground_on_grid = ground_on_grid_recorded` puts the wrapper in the place of the program's function: the job looks the function up by its name in its module each time, so from now on it calls the wrapper. The loop runs the job `RR.ground_job` for each ground state; its result is a dictionary whose entry `data` is the content of the JSON file that the program writes, and `RR.jsonable` turns its numpy arrays into plain Python numbers exactly as the program does before writing. Each line of Out [4] prints the number of levels kept and the Richardson value of E_{KS} with its uncertainty: N8_lamm2_a00 has 22 levels and $E_{KS} = 0.0028626521737$ ($U = 2.1 \times 10^{-12}$), N688_lam0_a00 has 137 levels and $E_{KS} = 680.4412465815219$ ($U = 3.3 \times 10^{-9}$), N136_lamp2_a20 has 354 levels and $E_{KS} = 12.4489994812532$ ($U = 4.6 \times 10^{-11}$). Then the original function is put back, and the check confirms that nine level arrays (three states, three grids) were kept. This cell runs for one to three minutes on a fast computer: N136_lamp2_a20 alone solves 354 levels on three grids, each with its excited state and four neighbouring slices.

In [5], comparing the new results with the committed files.

```
def compare_records(new, old, problems, where="top"):
    """Compare two JSON structures; return the number of numbers compared and append
```

```
    a description of every difference to the list problems."""
    if isinstance(new, bool) or isinstance(old, bool) or new is None or \
            isinstance(new, str):
        if new != old:
            problems.append(f"{where}: {new!r} != {old!r}")
        return 0
    if isinstance(new, (int, float)):
        if not isinstance(old, (int, float)) or \
                abs(new - old) > 1e-9 * max(1.0, abs(old)):
            problems.append(f"{where}: {new!r} != {old!r}")
        return 1
```

`compare_records` walks through two JSON structures at the same time and counts the numbers it compared. This first part handles the simplest cases. `isinstance(x, bool)` asks whether `x` is a truth value; a truth value, a missing value (`None`) or a text must be equal, and every inequality is appended to the list `problems` as a line naming the place (`{new!r}` writes the value as Python would type it). A number (`int` or `float`) must be within one part in a billion of the committed one, measured relative to $\max(1, |old|)$; it counts as one compared number. (A backslash at the end of a line continues the statement on the next line.)

```
    if isinstance(new, dict):
        if sorted(new) != sorted(old):
            problems.append(f"{where}: different names")
            return 0
        return sum(compare_records(new[k], old[k], problems, f"{where}/{k}")
                   for k in new)
    if len(new) != len(old):  # a list
        problems.append(f"{where}: lengths {len(new)} and {len(old)}")
        return 0
    return sum(compare_records(a, b, problems, f"{where}[{i}]")
               for i, (a, b) in enumerate(zip(new, old)))
```

For a dictionary both must have the same names (sorted, because the order does not matter); then the function calls itself for each entry, with the place extended by `/name`, and adds up the counts with `sum(...)`. A function that calls itself is **recursive**: it goes down into the nested structure until it reaches single values. Anything else is a list: both must have the same length, and the function compares them entry by entry (`zip` pairs the entries, `enumerate` numbers them, and the place is extended by `[i]`).

```
def check_reproduction(label, data, relative):
    """Check the new result data (named label) against its committed file."""
    committed = repository_file(relative).read_text(encoding="utf-8")
    problems = []
    numbers = compare_records(data, json.loads(committed), problems)
    text = json.dumps(data, indent=1, ensure_ascii=True) + "\n"  # as written
    say(f"{label}: {numbers} numbers compared; identical byte for byte: "
        f"{text == committed}")
    for line in problems[:5]:  # the first differences, if there are any
        say(f"  difference {line}")
    check(not problems,
```

```
        f"the re-run of {label} reproduces {relative.split('/results/')[1]}",
        record=f"{KS}/reports/ks-crosscheck.json, check "
               "reference_repeat_byte_identical")
```

`check_reproduction` reads the committed file as text, compares its content with the new result, and also writes the new result as text exactly as the reference program writes its files (`json.dumps` with an indentation of one blank and only ASCII characters, plus a final line end) to report whether the two texts are identical byte for byte. It prints the number of numbers compared and that answer, then the first five differences, if any (`problems[:5]` is the first five entries of a list). The check requires no difference; its name gives the file name after `/results/` (`.split('/results/')[1]` cuts the path at that text and takes the second piece), and its record line names the cross-check's check `reference_repeat_byte_identical`, which made the same test for all files.

```
for sid in GROUND_IDS:
    check_reproduction(sid, NEW[sid], f"{KS}/reference/results/ground/{sid}.json")
```

The loop applies this to the three ground states. Out [5] shows 3597, 4513 and 5815 numbers compared, every file identical byte for byte, and three PASS lines.

In [6], Richardson extrapolation at work.

```
def richardson(x1, x2, x3):
    """Three-grid Richardson value R and its uncertainty U (x1 on the coarsest grid),
    in the two-step form of the reference program; also the two one-step values."""
    r_fine = (4.0 * x3 - x2) / 3.0  # one step on the grids h/2 and h/4
    r_coarse = (4.0 * x2 - x1) / 3.0  # one step on the grids h and h/2
    R = (16.0 * r_fine - r_coarse) / 15.0  # the second step
    U = abs(R - r_fine) + 2e-12 * max(1.0, abs(R))
    return R, U, r_coarse, r_fine
```

`richardson` is the two-step extrapolation of Section 16.5 written in Python: `r_fine` is the one-step value $r(h/2)$ of the two finer grids, `r_coarse` the one-step value $r(h)$, `R` the second step $(16\,r(h/2) - r(h))/15$, and `U` the uncertainty $|R - r(h/2)| + 2 \cdot 10^{-12} \max(1, |R|)$. The function returns all four numbers.

```
CONVERGENCE = {}  # state id -> (E_KS on the three grids, R)
for sid in GROUND_IDS:
    xs = [grid["scalars"]["E_KS"] for grid in NEW[sid]["per_grid"]]
    R, U, _, _ = richardson(*xs)
    R_one_line = (64.0 * xs[2] - 20.0 * xs[1] + xs[0]) / 45.0
    stored = NEW[sid]["scalars"]["E_KS"]
    ratio = (xs[0] - xs[1]) / (xs[1] - xs[2])
    CONVERGENCE[sid] = (xs, R)
    say(f"{sid}: E_KS(300) = {xs[0]:.12f}, E_KS(600) = {xs[1]:.12f}, "
        f"E_KS(1200) = {xs[2]:.12f}")
    say(f"    R = {R:.13f}, U = {U:.1e}, ratio of the differences = {ratio:.5f}")
    check(abs(R - stored["value"]) <= 1e-14 * max(1.0, abs(R))
          and abs(U - stored["U"]) <= 1e-14 * max(1.0, abs(R))
          and abs(R_one_line - R) <= 1e-13 * max(1.0, abs(R)),
          f"our Richardson lines give the reference value and U of E_KS ({sid})")
```

For each ground state the list comprehension takes E_{KS} from the three per-grid records of the new result (`per_grid` holds one record per grid, coarsest first). `R, U, _, _ = richardson(*xs)` keeps the first two returned numbers (the underscore is a name for values that are not needed). `R_one_line` is the one-line form $(64x(h/4) - 20x(h/2) + x(h))/45$ of Section 16.5, `stored` the value and uncertainty that the reference program wrote, and `ratio` the convergence ratio of Section 16.4. The three values are kept in `CONVERGENCE` for the figure of In [8]. The two printed lines per state (Out [6]) show the three grid values, R, U and the ratio: 4.00020, 4.00007 and 4.00006. The check requires our R and U to equal the program's within 10^{-14} (relative) and the two written forms of R to agree within 10^{-13}: Section 16.5's algebra and the program agree.

In [7], a fourth grid tests the uncertainty.

```
FOUR_GRIDS = {}  # G -> (E_KS, levels, p0 at the tip) of N136_lamp2_a20 on grid G
original_observables = RR.K.observables  # the function of ks_fd (RR.K is ks_fd)


def observables_recorded(state):
    """Call ks_fd's own function and keep E_KS, the levels and p0 at the tip."""
    scalars, profiles = original_observables(state)
    FOUR_GRIDS[state.grid.G] = (scalars["E_KS"], state.eps.copy(),
                                scalars["p0_tip"])
    return scalars, profiles
```

A second wrapper, for the function `observables` of the module `ks_fd` (the reference program imported this module under the name `K`, so it is `RR.K` here). The validation job calls it once on each of the four grids; the wrapper keeps the grid's E_{KS}, a copy of its levels and the value of p_0 at the tip in `FOUR_GRIDS`, under the number of cells `state.grid.G`.

```
RR.K.observables = observables_recorded  # the job now calls our wrapper
VALIDATION = RR.jsonable(RR.validation_job(CO, SPECS["N136_lamp2_a20"])["data"])
RR.K.observables = original_observables  # put the original function back
check_reproduction("the fourth-grid validation", VALIDATION,
                   f"{KS}/reference/results/validation/N136_lamp2_a20.json")
same = all(abs(FOUR_GRIDS[G][0] - grid["scalars"]["E_KS"]) <= 1e-12
           for G, grid in zip((300, 600, 1200), NEW["N136_lamp2_a20"]["per_grid"]))
check(sorted(FOUR_GRIDS) == [300, 600, 1200, 2400] and same,
      "the validation run repeats the ground job's E_KS on the three common grids")
```

The wrapper is put in place, the program's job `RR.validation_job` solves N136_lamp2_a20 on $G = 300$, 600, 1200 and 2400 (about 40 seconds), and the original function is put back. `check_reproduction` compares the result with the committed file `validation/N136_lamp2_a20.json` (Out [7]: 99 numbers, identical byte for byte). The second check confirms that the validation run repeats the ground job's E_{KS} on the three common grids to 10^{-12} (`all(...)` is true when every comparison in the brackets is true) and that the four grids were recorded.

```
classes = VALIDATION["classes"]
for name, c in classes.items():
    say(f"  {name:20s} {c['elements']:4d} element(s): max |R123 - R234| / U123 = "
        f"{c['max_ratio_to_U123']:.2e}")
```

```
worst = max(classes, key=lambda name: classes[name]["max_ratio_to_U123"])
detail = next(c["detail"] for c in REPORTS["reference solver"]["checks"]
              if c["name"] == "richardson_uncertainty_validated")
stated = float(re.search(r"U123 = ([0-9.]+) \((\w+)\)", detail).group(1))
say(f"largest ratio {classes[worst]['max_ratio_to_U123']:.3f} ({worst}); "
    f"the report states {stated}")
check(all(c["all_within_U123"] for c in classes.values())
      and len(classes) == 19 and worst == "p0_tip"
      and abs(classes[worst]["max_ratio_to_U123"] - stated) <= 5e-4,
      "on the fourth grid every three-grid value moves by less than its U",
      record=f"{KS}/reports/ks-reference.json, check richardson_uncertainty_validated")
```

`classes` holds the 19 quantity classes of the validation, each with its number of elements and the largest ratio $|R_{123} - R_{234}|/U_{123}$; the loop prints them (Out [7]: from 0 for the integral of p_3 to 0.787 for p_0 at the tip). `max(classes, key=...)` finds the class with the largest ratio: the `key` is a **lambda**, a one-line function written in place, that gives each class name its ratio. `next(... for c in ... if ...)` takes the first check of the reference report whose name is `richardson_uncertainty_validated`, and `re.search` reads the number after the text `U123 =` in its detail sentence: in the pattern, `[0-9.]+` means one or more digits or points, round brackets capture what they match (`.group(1)` returns it), `\(` and `\)` are literal brackets, and `\w+` is a word. The check requires every element of every class to lie within its U_{123}, 19 classes, the largest ratio at `p0_tip`, and our largest ratio equal to the one stated in the report within 5×10^{-4}.

In [8], convergence as a picture (Figure 16a.1).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.3))
h = np.array([3.0 / 300, 3.0 / 600, 3.0 / 1200])  # the cell widths
for colour, sid in zip(PALETTE, GROUND_IDS):
    xs, R = CONVERGENCE[sid]
    left.loglog(h, [abs(x - R) for x in xs], "o-", color=colour, lw=1.5, ms=6,
                label=sid)
    first = abs(xs[0] - R)  # a guide of slope 2, a factor 3 below the first point
    left.loglog(h, first / 3.0 * (h / h[0]) ** 2, "--", color=colour, lw=0.9)
left.set_xlabel("cell width $h = L/G$")
left.set_ylabel("$|E_{KS}(G) - R|$ (units of $m$)")
left.set_title("three grids, three states")
left.legend(fontsize=8)
```

`plt.subplots(1, 2, ...)` makes a figure with two sets of axes side by side, `left` and `right`, 10 by 4.3 inches. `h` is the numpy array of the three cell widths 3/300, 3/600, 3/1200. For each ground state (`zip` pairs the colours with the ids) the left axes draw the distances $|E_{KS}(G) - R|$ against h with **logarithmic axes** (`loglog`: equal distances on the axis are equal factors), as circles joined by lines (`"o-"`, line width 1.5, marker size 6), and a dashed **guide line** proportional to h^2 that starts a factor 3 below the first point. On logarithmic axes a power law Ch^q is a straight line of slope q, so points parallel to the guide show an error proportional to h^2. The labels, title and legend complete the panel.

```
grids = [300, 600, 1200, 2400]
e4 = [FOUR_GRIDS[G][0] for G in grids]  # E_KS of N136_lamp2_a20 on the four grids
h4 = np.array([3.0 / G for G in grids])
```

```
R123, U123, _, _ = richardson(*e4[:3])
R234, _, _, _ = richardson(*e4[1:])
single = [abs(x - R234) for x in e4]
one_step = [abs((4.0 * e4[i + 1] - e4[i]) / 3.0 - R234) for i in range(3)]
```

For the right panel: `e4` is E_{KS} of N136_lamp2_a20 on the four grids (kept by the wrapper of In [7]), `h4` their cell widths, `R123` and `U123` the three-grid value and uncertainty of the three coarser grids (`e4[:3]` is the first three entries), and `R234` the three-grid value of the three finer ones (`e4[1:]` is everything after the first). `single` holds the distance of each single-grid value from R_{234}, and `one_step` the distance of each one-step value $(4x(2G) - x(G))/3$ from it.

```
right.loglog(h4, single, "o-", color=PALETTE[0], lw=1.5, ms=6,
             label="single grid $x(G)$")
right.loglog(h4, single[0] / 3.0 * (h4 / h4[0]) ** 2, "--", color=PALETTE[0],
             lw=0.9, label="slope 2 (shifted down)")
right.loglog(h4[:3], one_step, "s-", color=PALETTE[1], lw=1.5, ms=6,
             label="one step $r$ (pair $G$, $2G$)")
right.loglog(h4[:3], one_step[0] / 3.0 * (h4[:3] / h4[0]) ** 4, "--",
             color=PALETTE[1], lw=0.9, label="slope 4 (shifted down)")
right.loglog([h4[0]], [max(abs(R123 - R234), 1e-16)], "D", color=PALETTE[2], ms=8,
             label="three grids $R_{123}$")
right.loglog([h4[0]], [U123], "_", color="k", ms=16, mew=2,
             label="its uncertainty $U_{123}$")
right.set_xlabel("cell width $h$ of the coarsest grid used")
right.set_ylabel("distance from $R_{234}$ (units of $m$)")
right.set_title("N136_lamp2_a20 on four grids")
right.legend(fontsize=7, loc="center right")
```

The right panel draws the single-grid distances with a dashed slope-2 guide, the one-step distances (squares) with a dashed slope-4 guide (h^4 is the error of a one-step value, Section 16.5), the distance $|R_{123} - R_{234}|$ as a diamond (`max(..., 1e-16)` keeps a zero away from the logarithmic axis) and the uncertainty U_{123} as a short horizontal bar (marker `"_"`, black, edge width 2).

```
for ax, ticks in ((left, h), (right, h4)):  # tick labels at the grids' h only
    ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
    ax.set_xticks(ticks)
    ax.set_xticklabels([f"{t:g}" for t in ticks])
fig.tight_layout()
```

For both panels the tick labels are set at the grids' cell widths only: `NullFormatter` removes the labels of the small in-between ticks, `set_xticks` puts ticks at the given values, and `set_xticklabels` writes them in the shortest form (`:g`). `fig.tight_layout()` arranges the panels so that their labels do not overlap.

```
save_figure(fig, "grid_convergence",
            "Convergence of the reference solver for the Kohn-Sham energy $E_{KS}$, "
            "logarithmic axes, energies in units of the mass $m$. Left: distance of "
            "the single-grid value from the Richardson value $R$ against the cell "
            "width $h = 3/G$ for $G = 300, 600, 1200$ for the three ground states; "
```

```
            "the points run parallel to the dashed lines of slope 2 (drawn a factor 3 "
            "lower), an error proportional to $h^2$. Right: the state N136_lamp2_a20 "
            "also on $G = 2400$, every "
            "distance measured from the four-grid best value $R_{234}$: single grids "
            "fall with slope 2, one Richardson step with slope 4, and the three-grid "
            "value $R_{123}$ (diamond) lies far below its stated uncertainty "
            "$U_{123}$ (bar), so $U$ is a safe over-estimate.")
check(all(abs(CONVERGENCE[s][0][0] - CONVERGENCE[s][1])
          > abs(CONVERGENCE[s][0][2] - CONVERGENCE[s][1]) for s in GROUND_IDS)
      and single[0] > single[1] > single[2] > single[3]
      and one_step[0] > one_step[1] > one_step[2] and abs(R123 - R234) <= U123,
      "the errors shrink grid by grid and R123 lies within U123 of R234")
```

`save_figure` saves Figure 16a.1 with its caption (Python joins strings written next to each other into one). What the student should see: on the left, three straight lines parallel to their slope-2 guides, the errors of the three states falling by a factor 4 from grid to grid although their sizes differ by five powers of ten; on the right, the slope-2 line of the single grids, the slope-4 line of one Richardson step, and the diamond of R_{123}, only 3.6×10^{-14} from R_{234} (`Revision/kohn_sham/reference/results/validation/N136_lamp2_a20.json`, class E_KS), far below the bar of its uncertainty $U_{123} = 4.6 \times 10^{-11}$: the uncertainty is a safe over-estimate. The check confirms what the picture shows: for each state the error shrinks from the coarsest to the finest grid, the single-grid and one-step distances shrink monotonically, and R_{123} lies within U_{123} of R_{234}.

In [9], the fourth-grid test as a bar chart (Figure 16a.2).

```
names = list(classes)  # the 19 classes in the order of the record
values = [max(classes[n]["max_ratio_to_U123"], 1e-8) for n in names]
fig, ax = plt.subplots(figsize=(7.5, 6.2))
colours = [PALETTE[1] if n == worst else PALETTE[0] for n in names]
ax.barh(range(len(names)), values, color=colours, height=0.65)
ax.set_xscale("log")
ax.axvline(1.0, color="k", lw=1.2)
ax.set_yticks(range(len(names)))
ax.set_yticklabels([f"{n} ({classes[n]['elements']})" for n in names], fontsize=8)
ax.invert_yaxis()  # the first class at the top
ax.set_xlim(1e-9, 3.0)
```

`names` is the list of the 19 class names in the order of the record, and `values` their largest ratios, with exact zeros raised to 10^{-8} so that they can be drawn on a logarithmic axis. `colours` marks the worst class in orange (the second palette colour) and the others in blue. `barh` draws horizontal bars, one per class, at the heights 0 to 18; `set_xscale("log")` makes the horizontal axis logarithmic; `axvline(1.0, ...)` draws the vertical black line at ratio 1; the class names with their numbers of elements become the labels of the vertical axis; `invert_yaxis()` puts the first class at the top; and the horizontal axis runs from 10^{-9} to 3.

```
for i, n in enumerate(names):
    if classes[n]["max_ratio_to_U123"] == 0.0:  # mark a class that agrees exactly
        ax.text(1.5e-8, i, "exactly 0", va="center", fontsize=8)
ax.set_xlabel("largest $|R_{123} - R_{234}|$ / $U_{123}$ of the class")
ax.set_title("Fourth-grid test of the uncertainty (N136_lamp2_a20)")
```

```
save_figure(fig, "uncertainty_validated",
            "Validation of the reference uncertainty on a fourth grid for the state "
            "N136_lamp2_a20: for each of the 19 classes of quantities (the number of "
            "elements in brackets) the largest change of the three-grid value between "
            "the grids $(300, 600, 1200)$ and $(600, 1200, 2400)$, divided by the "
            "stated uncertainty $U_{123}$, on a logarithmic axis; a class that agrees "
            f"exactly is drawn at $10^{{-8}}$ and marked. All bars end left of the "
            "line 1; the largest, "
            f"{classes[worst]['max_ratio_to_U123']:.3f}, is the value of $p_0$ at the "
            "tip, where the densities are largest.")
```

The loop writes "exactly 0" next to every class whose ratio is exactly zero (here only the integral of p_3). After the axis label and the title, `save_figure` saves Figure 16a.2. The caption is an f-string that inserts the largest ratio, `{{-8}}` writing a literal pair of braces. What the student should see: every bar ends left of the line 1. The energies, the integrals and the levels end far to the left (ratios between about 10^{-4} and 3×10^{-3}, and exactly 0 for the integral of p_3), and the two values at the brane even further left (about 2×10^{-6}). Only the two values at the tip (ρ: 0.438, p_0: 0.787) and four of the five profiles, which include the tip as their first point (n: 0.661, ρ: 0.438, p_3: 0.437, p_0: 0.787; for ρ and p_0 the largest ratio is the tip value itself), come within a factor of about 2 of the line; the profile of S stays at 0.022. The largest ratio (p_0 at the tip, 0.787) is within a factor of 1.3 of the line. The tip, where the densities are largest, is where the grids work hardest. (All these ratios are printed in Out [7].)

In [10], every level converges like h^2 (Figure 16a.3).

```
level_points = []  # (state, level energy, ratio) of every level with a ratio
for sid in GROUND_IDS:
    eps = [PER_GRID[(sid, G)] for G in (300, 600, 1200)]
    ratios = RR.asym_ratio(*eps)  # the reference's own function
    stored = NEW[sid]["consistency"]["ratio_eps"]
    mine = (len(ratios), float(np.median(ratios)), min(ratios), max(ratios))
    say(f"{sid}: {mine[0]} levels, median ratio {mine[1]:.6f}, smallest "
        f"{mine[2]:.6f}, largest {mine[3]:.6f}")
    check(mine == (stored["count"], stored["median"], stored["min"], stored["max"])
          and abs(mine[1] - 4.0) <= 0.05,
          f"the levels of {sid} converge like h^2 (median ratio within 0.05 of 4)",
          record=f"{KS}/reports/ks-reference.json, check richardson_asymptotic_ratio")
```

For each ground state `eps` is the list of the level arrays on the three grids kept by the wrapper of In [4], and `RR.asym_ratio` (the reference's own function) computes the convergence ratio of Section 16.4 for every level whose two differences exceed 10^{-10} (smaller differences are dominated by rounding). `mine` collects the number of ratios, their **median** (the middle value when they are sorted; `np.median`), the smallest and the largest, and prints them. The check requires these four numbers to equal the ones the program stored in the entry `consistency` of the result, and the median to lie within 0.05 of 4, the criterion of the reference's check `richardson_asymptotic_ratio`. Out [10] shows medians of 4.000053, 4.000064 and 4.000055.

```
    floor = 1e-10  # the same selection as asym_ratio: both differences above 1e-10
    d1, d2 = eps[0] - eps[1], eps[1] - eps[2]
    keep = (np.abs(d1) > floor) & (np.abs(d2) > floor)
```

```
    energies = np.asarray(NEW[sid]["levels"]["eps"])[keep]
    level_points += [(sid, e, r) for e, r in zip(energies, d1[keep] / d2[keep])]
```

The same selection by hand, to draw it: `d1` and `d2` are the two differences of every level (numpy subtracts arrays entry by entry), `keep` is a **mask**, an array of truth values that is true where both differences exceed the floor (`&` combines two masks with "and"), and `energies` are the Richardson values of the kept levels (`array[keep]` keeps the entries where the mask is true). `level_points` collects one tuple (state, level energy, ratio) per level.

```
fig, ax = plt.subplots(figsize=(7.5, 4.3))
for colour, sid in zip(PALETTE, GROUND_IDS):
    pts = [(e, abs(r - 4.0)) for s, e, r in level_points if s == sid]
    ax.semilogy([p[0] for p in pts], [p[1] for p in pts], "o", color=colour, ms=4,
                alpha=0.8, label=f"{sid} ({len(pts)} levels)")
ax.axhline(0.05, color="k", lw=1.0, ls="--", label="limit 0.05 of the median test")
ax.set_xlabel("level energy $\\varepsilon$ (Richardson value, units of $m$)")
ax.set_ylabel("$|$ratio $- 4|$")
ax.set_title("Every level converges like $h^2$")
ax.legend(fontsize=8)
```

The figure draws, for each state, the distance of every ratio from 4 against the level energy, on a logarithmic vertical axis (`semilogy`), with the limit 0.05 of the median test as a dashed line. Two backslashes in a Python string give one backslash in the text, so `"\\varepsilon"` reaches matplotlib as the LaTeX command for ε.

```
worst_level = max(abs(r - 4.0) for _, _, r in level_points)
save_figure(fig, "eigenvalue_ratios",
            "Distance from 4 of the convergence ratio $(x(300) - x(600))/(x(600) - "
            "x(1200))$ of every level of the three ground states, against the "
            "level energy in units of $m$ (vertical axis logarithmic). A ratio of 4 "
            "means an error proportional to $h^2$. Most levels lie between "
            "$10^{-5}$ and $3\\times10^{-4}$ from 4, with no trend in the energy; "
            "the levels nearest $\\varepsilon = 0$ deviate by $5\\times10^{-4}$ to "
            "$7\\times10^{-4}$. The largest deviations belong to a group of levels "
            "of N136_lamp2_a20 between $\\varepsilon = 1.2$ and $2.1$; even the worst "
            f"level, at {worst_level:.4f}, stays below the limit 0.05 that the "
            "reference applies to the median.")
check(worst_level < 0.05, "every single level has a ratio within 0.05 of 4")
```

`worst_level` is the largest distance from 4 of any single level (0.0336). `save_figure` saves Figure 16a.3, and the check requires every single level, not only the median, to lie within 0.05 of 4. What the student should see: most levels lie between 10^{-5} and 3×10^{-4} from 4 (the long band of N136_lamp2_a20 between $\varepsilon = 0.06$ and 1 lies at 4×10^{-5} to 8×10^{-5}), and there is no trend with the energy: the levels of N688_lam0_a00 up to $\varepsilon = 5.6$ stay between 7×10^{-6} and 2.2×10^{-4}, and the highest levels of N136_lamp2_a20, near $\varepsilon = 3.25$, come closest to 4 of all (5×10^{-7} to 3×10^{-6}). The levels nearest $\varepsilon = 0$ deviate by 5×10^{-4} (N8_lamm2_a00, its largest ratio 4.000506 of Out [10]) to 7×10^{-4}. The largest deviations, 10^{-3} to 0.034, belong to a group of levels of N136_lamp2_a20 between $\varepsilon = 1.2$ and 2.1. Section 16.4 says what such a deviation measures: the ratio is $4\,(1 + \frac{15}{16}\frac{d}{c}h^2)$, so $|\text{ratio} - 4| = \frac{15}{4}\,|\frac{d}{c}|\,h^2$. The worst deviation, 0.0336, therefore means

$|\frac{d}{c}|\,h^2 = 0.0336/3.75 = 0.009$ on the coarsest grid ($h = 0.01$): there the h^4 term of the error is about 1% of the h^2 term, still small, but much larger than for most levels, whose deviations of 10^{-5} to 3×10^{-4} mean $|\frac{d}{c}|\,h^2$ between 3×10^{-6} and 8×10^{-5}. Even the worst level stays below the dashed line.

In [11], the two thermal states.

```
for sid in THERMAL_IDS:
    NEW[sid] = RR.jsonable(RR.thermo_job(CO, SPECS[sid])["data"])
    th = NEW[sid]["thermo"]
    say(f"{sid}: {NEW[sid]['levels_in_set']} levels; mu = {th['mu']['value']:.13f} "
        f"(U {th['mu']['U']:.1e}), E = {th['E']['value']:.12f} (U {th['E']['U']:.1e}),"
        f" C_V = {th['C_V']['value']:.9f} (U {th['C_V']['U']:.1e})")
    check_reproduction(sid, NEW[sid], f"{KS}/reference/results/thermo/{sid}.json")
```

For each thermal state the reference's thermal job is run (it solves the state on the three grids at T and at four neighbouring temperatures), its result is turned into plain numbers and kept in `NEW`, and the chemical potential μ, the energy E and the heat capacity C_V are printed with their uncertainties (`:.13f`, `:.12f` and `:.9f` give 13, 12 and 9 decimals, `:.1e` one decimal in powers of ten). Then `check_reproduction` compares with the committed thermal file. Out [11]: for N8_lamm1_a00_T10, $\mu = 0.2100104497343$ ($U = 2.3 \times 10^{-12}$), and for N8_lam0_a15_T50, $\mu = -0.0304029805513$ and $C_V = 32.379779710$; both files are reproduced byte for byte. N8_lamm1_a00_T10 is the state in which the first cross-check failed (Section 16.20).

In [12], the Rust solver: canonical and refined runs.

```
SOLVER = rust_program(f"{KS}/solver/Cargo.toml", "revision_ks_solver")
RUN_FOLDER = Path(tempfile.mkdtemp(prefix="textbook_16a_"))  # a new, empty folder
```

`rust_program` (In [1]) builds the Rust solver with cargo, which takes a second when the program is up to date, and returns the path of the program, `SOLVER`. `tempfile.mkdtemp` creates a new, empty **temporary folder** in the place the operating system keeps for temporary files (outside the repository; its name starts with `textbook_16a_` and ends with random characters) and returns its name, which `Path` turns into a path: `RUN_FOLDER`. The raw outputs of the runs go there, so they never enter the repository.

```
def run_single(sid, refined):
    """Run `revision_ks_solver single` for state sid; return (results, profiles,
    the arguments without the output files)."""
    spec = SPECS[sid]
    margin = (0.2 if "T" in spec else 0.25) + 2.0 * spec["sigma"]
    stem = f"{sid}_{'refined' if refined else 'canonical'}"
    arguments = ["single", "--m", "1", "--lambda", repr(spec["lam"]),
                 "--a4", repr(spec["a4"]), "--N", repr(spec["N"]),
                 "--margin", repr(margin)]
    if "T" in spec:
        arguments += ["--T", repr(spec["T"])]
    if refined:
        arguments.append("--refined")
```

`run_single` runs the solver's command `single` for one state. The **margin** decides how many levels above the Fermi level the label set keeps: $0.25 + 2\sigma$ at $T = 0$ and $0.2 + 2\sigma$ at $T > 0$, exactly as in the cross-check's measurement program (`"T" in spec` asks whether the dictionary has the key `"T"`). `stem` is the base of the two output file names, the state id followed by `_canonical` or `_refined`. `arguments` is the list of the program's arguments: the command, the mass 1, the coupling, the slice, N and the margin, each number written with `repr` (the shortest decimal text that gives back exactly the same double); a thermal state adds its temperature, and a refined run adds the option that switches to the refined numerics of Section 16.6.

```
    files = ["--out", f"{RUN_FOLDER / stem}.json",
             "--profiles", f"{RUN_FOLDER / stem}.csv"]
    done = subprocess.run([str(SOLVER)] + arguments + files, cwd=str(REPO),
                          capture_output=True, text=True)
    if done.returncode != 0 or not done.stdout.strip().endswith("SUCCESS"):
        print(done.stderr[-2000:])
        raise RuntimeError(f"the Rust run {stem} failed")
    results = json.loads((RUN_FOLDER / f"{stem}.json").read_text(encoding="utf-8"))
    with open(RUN_FOLDER / f"{stem}.csv", newline="", encoding="utf-8") as handle:
        profiles = [{k: float(v) for k, v in row.items()}
                    for row in csv.DictReader(handle)]
    return results, profiles, arguments
```

`files` adds the two output options: a JSON file with the energies, levels and integrals, and a CSV file with the profiles, both in `RUN_FOLDER` (`RUN_FOLDER / stem` joins the folder and the name). `subprocess.run` runs the program from the repository folder (`cwd=...`) and keeps its messages. If the program failed (a return code other than 0, or a last line other than SUCCESS; `.strip()` removes blanks and line ends), the function prints the end of the error messages and stops. Otherwise it reads the JSON file back and reads the CSV file as a list of rows, each a dictionary from the column names to numbers (`float(v)` turns each text into a number). It returns the two and the argument list.

```
RUST = {}  # (state id, "canonical" or "refined") -> (results, profiles)
for sid in SPECS:
    for refined in (False, True):
        results, profiles, arguments = run_single(sid, refined)
        RUST[(sid, "refined" if refined else "canonical")] = (results, profiles)
    say("revision_ks_solver " + " ".join(arguments))  # the refined command
shutil.rmtree(RUN_FOLDER)  # delete the temporary folder and the raw outputs in it
check(len(RUST) == 10, "ten Rust runs (five states, two numerics) completed")
```

The double loop runs every state twice, canonical (`refined` false) and refined (true), and keeps the results in the dictionary `RUST` under the pair (state id, kind). After each state it prints the refined command without the output files (whose folder differs from run to run): Out [12]. All results are now held in `RUST`, so `shutil.rmtree` deletes the temporary folder with everything in it (`rmtree` removes a whole folder tree); the runs leave no file behind. The check confirms ten runs.

In [13], do the runs reproduce the committed Rust results?

```
SUMMARY = {r["id"]: r for r in read_csv(f"{KS}/results/ground/summary.csv")}
EMT = {r["id"]: r for r in read_csv(f"{KS}/results/ground/emt-integrals.csv")}
```

```
EXCITED = {r["id"]: r for r in read_csv(f"{KS}/results/excited/summary.csv")}
ADIABATIC = {r["id"]: r for r in read_csv(f"{KS}/results/adiabatic/adiabaticity.csv")}
THERMO = {r["id"]: r for r in read_csv(f"{KS}/results/thermo/thermodynamics.csv")}
REFINEMENT = {(s["kind"], s["id"]): s
              for s in read_json(f"{KS}/checker/rust-refinement.json")["states"]}
```

The committed canonical Rust results are read: the ground-state summary, the energy-momentum integrals, the excited-state summary, the adiabaticity table and the thermodynamics table, each filed under the state id. `REFINEMENT` holds the cross-check's measurement file `rust-refinement.json`, filed under the pair (kind, id).

```
def rel(a, b):
    """|a - b| relative to max(1, |b|)."""
    return abs(a - b) / max(1.0, abs(b))


def level_map(results):
    """{(n2, j, parity, label): eps} of the levels of a ‘single‘ run."""
    return {tuple(l[:4]): l[4] for l in results["levels_n2_j_parity_label_eps_deg_f"]}
```

`rel(a, b)` is the difference of `a` and `b` relative to $\max(1, |b|)$. `level_map` turns the list of levels of a single run (each level a list of n_2, j, parity, label, energy, degeneracy, occupation) into a dictionary from the first four entries (`tuple(l[:4])`, the key of the level) to its energy (`l[4]`).

```
for sid in SPECS:
    canon, canon_prof = RUST[(sid, "canonical")]
    refined, _ = RUST[(sid, "refined")]
    if sid in GROUND_IDS:
        worst = rel(canon["E_KS"], float(SUMMARY[sid]["E_KS"]))
        integrals = canon["emtIntegrals_2Vol7_int_e6Hy"]
        for k in ("rho", "p3", "p_t", "p0", "n"):
            worst = max(worst, rel(integrals[k], float(EMT[sid]["int_" + k])))
        committed = {(int(x["n2"]), int(x["j"]), x["parity"], int(x["label"])):
                     float(x["eps"])
                     for x in read_csv(f"{KS}/results/ground/levels/{sid}.csv")}
        mine = level_map(canon)
        worst = max([worst] + [abs(mine[k] - committed[k]) for k in mine
                               if k in committed])
        stored_prof = read_csv(f"{KS}/results/ground/profiles/{sid}.csv")
        for name in ("n", "S", "rho", "p3", "p_t", "p0"):
            top = max(abs(float(row[name])) for row in stored_prof)
            worst = max([worst] + [abs(a[name] - float(b[name])) / max(top, 1e-300)
                                   for a, b in zip(canon_prof, stored_prof)])
        kind = "ground"
```

For each state the canonical and refined results are taken from `RUST` (`refined, _ = ...` drops the refined profiles, which are not needed here). For a ground state `worst` collects the largest difference between the canonical run and the committed results: E_{KS} relative; the five energy-momentum integrals relative; every level present in both (`committed` is a dictionary from the level key to the committed energy, read from the state's levels file); and every point of six profiles, measured relative

to the largest value of that profile (`max(top, 1e-300)` avoids a division by zero for a profile that is zero everywhere).

```
    else:
        row = THERMO[sid]
        worst = max(rel(canon["E_KS"], float(row["E"])),
                    abs(canon["mu_or_fermi_level"] - float(row["mu"])),
                    abs(canon["entropy"] - float(row["entropy"]))
                    / max(1e-6, abs(float(row["entropy"]))))
        kind = "thermo"
```

For a thermal state `worst` is the largest of three differences: E relative, μ absolute, and the entropy relative to $\max(10^{-6}, |S|)$.

```
    rec = REFINEMENT[(kind, sid)]
    lc, lr = level_map(canon), level_map(refined)
    level_diff = max(abs(lc[k] - lr[k]) for k in lc if k in lr)
    e_diff = abs(canon["E_KS"] - refined["E_KS"])
    say(f"{sid}: largest difference from the committed results {worst:.1e}; "
        f"|canonical - refined|: E_KS {e_diff:.3e}, levels {level_diff:.3e}")
    same = (abs(e_diff - rec["scalars"]["E_KS"]["abs_diff"])
            <= 1e-12 * max(1.0, abs(canon["E_KS"]))
            and abs(level_diff - rec["levels"]["max_abs_diff"]) <= 1e-12)
    if kind == "thermo":
        mu_diff = abs(canon["mu_or_fermi_level"] - refined["mu_or_fermi_level"])
        same = same and abs(mu_diff - rec["scalars"]["mu"]["abs_diff"]) <= 1e-12
    check(worst <= 1e-12, f"the canonical run of {sid} reproduces the committed "
                          "Rust results",
          record=f"{KS}/reports/ks-crosscheck.json, check "
                 "rust_refinement_applies_to_matrix")
    check(same, f"canonical minus refined of {sid} reproduces the measured values",
          record=f"{KS}/checker/rust-refinement.json, state {kind} {sid}")
```

`rec` is the measurement of this state in `rust-refinement.json`. `level_diff` is the largest canonical-minus-refined difference over the levels present in both runs, and `e_diff` the difference of E_{KS}. The printed line (Out [13]) shows both and `worst`, which is exactly 0 in all five states. `same` asks whether these differences equal the recorded ones within 10^{-12}; for thermal states the difference of μ must also agree. The first check reproduces the cross-check's check `rust_refinement_applies_to_matrix`: the canonical run gives exactly the committed results, so the measured differences apply to them. The second reproduces the state's entry of the measurement file. For example N688_lam0_a00 has $|x_c - x_r| = 7.529 \times 10^{-10}$ for E_{KS}, the number used in Section 16.6.

In [14], the tolerance rule in a few lines.

```
RK4_FACTOR = 16.0 / 15.0  # canonical error = (16/15) |canonical - refined|
ROWS = []  # (class, case, x_Rust, x_ref, U_ref, U_Rust, tolerance, |diff|, ratio)


def compare(cls, case, x_rust, x_ref, u_ref, u_rust, scale=None):
    """The tolerance rule of the cross-check; returns the ratio."""
```

```
    scale = max(1.0, abs(x_ref)) if scale is None else scale
    tolerance = 3.0 * (u_ref + u_rust) + 1e-12 * scale
    diff = abs(x_rust - x_ref)
    ratio = diff / tolerance
    ROWS.append((cls, case, x_rust, x_ref, u_ref, u_rust, tolerance, diff, ratio))
    return ratio
```

RK4_FACTOR is the factor $\frac{16}{15}$ of Section 16.6. ROWS will hold one tuple per comparison. compare is the tolerance rule of Section 16.7: the scale is $\max(1, |x_{ref}|)$ unless a scale is given (for profile points), the tolerance is $3(U_{ref} + U_{Rust}) + 10^{-12}\cdot$scale, the difference is $|x_{Rust} - x_{ref}|$, and the ratio is their quotient. The function stores all nine numbers of the comparison in ROWS and returns the ratio.

```
PROFILE_NAMES = ("n", "S", "Q", "M_eff", "v_v", "e_int", "rho", "p3", "p_t", "p0")


def rust_uncertainties(sid):
    """U_Rust of every quantity reported by ‘single‘, for state sid."""
    (canon, cp), (refined, rp) = RUST[(sid, "canonical")], RUST[(sid, "refined")]
    U = {k: RK4_FACTOR * abs(canon[k] - refined[k])
         for k in ("E_KS", "E_band", "E_int", "entropy")}
    U["mu"] = RK4_FACTOR * abs(canon["mu_or_fermi_level"]
                               - refined["mu_or_fermi_level"])
    ci, ri = canon["emtIntegrals_2Vol7_int_e6Hy"], refined["emtIntegrals_2Vol7_int_e6Hy"]
    for k in ("rho", "p3", "p_t", "p0", "n"):
        U["int_" + k] = RK4_FACTOR * abs(ci[k] - ri[k])
    for k in ("rho", "p3", "p_t", "p0"):  # profile end points: brane and tip
        U[k + "_brane"] = RK4_FACTOR * abs(cp[-1][k] - rp[-1][k])
        U[k + "_tip"] = RK4_FACTOR * abs(cp[0][k] - rp[0][k])
    lc, lr = level_map(canon), level_map(refined)
    U["levels"] = RK4_FACTOR * max(abs(lc[k] - lr[k]) for k in lc if k in lr)
    for name in PROFILE_NAMES:  # (U of the profile, largest |value|)
        U["profile_" + name] = (
            RK4_FACTOR * max(abs(a[name] - b[name]) for a, b in zip(cp, rp)),
            max(abs(a[name]) for a in cp))
    return U
```

PROFILE_NAMES are the ten profiles of the record: n, S, Q, M_{eff}, v_v, e_{int}, ρ, p_3, p_t, p_0. rust_unc ertainties computes $U_{Rust} = \frac{16}{15}|x_c - x_r|$ for every quantity that the single runs report: the three energies and the entropy (a dictionary comprehension), μ, the five integrals, the values of ρ, p_3, p_t, p_0 at the brane (the last profile point, index -1) and at the tip (the first, index 0), the levels (one number: $\frac{16}{15}$ times the largest difference over all levels of the state), and for each profile a pair: $\frac{16}{15}$ times the largest difference over its points, and the largest value of the profile.

```
U_RUST = {sid: rust_uncertainties(sid) for sid in SPECS}
WIDE = REPORTS["cross-check"]["rust_matrix_wide_uncertainties"]
say("matrix-wide Rust differences: " + ", ".join(
    f"{k.replace('refined_', '')} {v:.2e}" for k, v in sorted(WIDE.items())))
```

U_RUST holds these uncertainties for the five states. WIDE is the dictionary of matrix-wide differences recorded in the cross-check report for the quantities that single runs do not report (Section 16.6);

the printed line lists them, with the prefix `refined_` removed from each name (`.replace`), sorted by name (Out [14]).

```
sid = "N8_lamm2_a00"  # one comparison, piece by piece
x_rust = float(SUMMARY[sid]["E_KS"])
x_ref = NEW[sid]["scalars"]["E_KS"]["value"]
u_ref = NEW[sid]["scalars"]["E_KS"]["U"]
u_rust = U_RUST[sid]["E_KS"]
ratio = compare("ground_energies", f"{sid} E_KS", x_rust, x_ref, u_ref, u_rust)
say(f"E_KS of {sid}: Rust {x_rust!r}, reference {x_ref!r}")
say(f"    |diff| = {abs(x_rust - x_ref):.3e}, U_ref = {u_ref:.3e}, "
    f"U_Rust = {u_rust:.3e}")
say(f"    tolerance = 3 (U_ref + U_Rust) + 1e-12 max(1, |x|) = {ROWS[-1][6]:.3e}, "
    f"ratio = {ratio:.4f}")
check(ratio <= 1.0, f"E_KS of {sid} agrees within the tolerance")
```

One comparison piece by piece: E_{KS} of N8_lamm2_a00 from the committed Rust summary and from the new reference result, with U_{ref} from the reference and U_{Rust} from the measurement. The three printed lines (Out [14]) are the numbers of the worked comparison of Section 16.7: difference 1.264×10^{-13}, $U_{ref} = 2.096 \times 10^{-12}$, $U_{Rust} = 1.214 \times 10^{-13}$, tolerance 7.651×10^{-12}, ratio 0.0165 (`ROWS[-1][6]` is the tolerance of the last comparison). The check requires the ratio to be at most 1.

In [15], the ground-state comparisons.

```
def rank_key(n2, j, parity, rank):
    """The level key n2:j:parity:rank as a text, for example 1:-1:even:2."""
    return f"{int(n2)}:{'+1' if int(j) > 0 else '-1'}:{parity}:{int(rank)}"
```

`rank_key` writes the key of a level as a text such as `1:-1:even:2`: the shell n_2, the block type with its sign, the parity and the **rank**. The rank numbers the levels of one sector from its lowest particle level, which gets rank 0, upwards $1, 2, \ldots$ and downwards $-1, -2, \ldots$; for the Rust solver it is the Pruefer label l of Chapter 15 minus l_{min}, the label of the lowest particle level of the sector, and the reference solver counts its levels in the same way (Section 16.13 works out the ranks of the free problem).

```
U_DSCF = RK4_FACTOR * WIDE["refined_delta_scf"]  # matrix-wide Delta-SCF uncertainty
for sid in GROUND_IDS:
    sc, U = NEW[sid]["scalars"], U_RUST[sid]
    for k in ("E_KS", "E_band", "E_int"):
        if not (sid == "N8_lamm2_a00" and k == "E_KS"):  # made in section 13
            compare("ground_energies", f"{sid} {k}", float(SUMMARY[sid][k]),
                    sc[k]["value"], sc[k]["U"], U[k])
    for k, u in (("HOMO", U["levels"]), ("LUMO", U["levels"]),
                 ("KS_gap", 2.0 * U["levels"])):
        compare("ground_homo_lumo_gap", f"{sid} {k}", float(SUMMARY[sid][k]),
                sc[k]["value"], sc[k]["U"], u)
```

`U_DSCF` is the matrix-wide Delta-SCF uncertainty. For each ground state: the three energies are compared (skipping E_{KS} of N8_lamm2_a00, which In [14] already compared), and HOMO, LUMO and the gap, each with the largest level uncertainty of the state, the gap with twice that (it is a difference of two levels).

```
    rust_levels = {}  # key -> (eps, occupation) of the committed Rust levels
    for x in read_csv(f"{KS}/results/ground/levels/{sid}.csv"):
        rank = int(x["label"]) - int(x["label_min"])
        rust_levels[rank_key(x["n2"], x["j"], x["parity"], rank)] = (
            float(x["eps"]), float(x["f"]))
    lev = NEW[sid]["levels"]
    ref_levels = {rank_key(*k): (e, u, f)
                  for k, e, u, f in zip(lev["keys"], lev["eps"], lev["U"], lev["f"])}
    common = sorted(set(rust_levels) & set(ref_levels))
    for k in common:
        compare("ground_eigenvalues", f"{sid} level {k}", rust_levels[k][0],
                ref_levels[k][0], ref_levels[k][1], U["levels"])
```

The committed Rust levels of the state are read and filed under their keys, with the Rust label turned into a rank by subtracting `label_min`, the label of the lowest particle level of its sector; each entry holds the energy and the occupation. The reference levels are filed the same way, with energy, uncertainty and occupation (`zip` walks through four lists at once). `common` is the sorted list of the keys present in both (`&` of two sets is their common part), and every common level is compared with the level uncertainty of the state.

```
    occupied_rust = sorted(k for k, v in rust_levels.items() if v[1] > 0)
    occupied_ref = sorted(k for k, v in ref_levels.items() if v[2] > 0)
    compare("excited_delta_scf", f"{sid} delta_SCF", float(EXCITED[sid]["delta_SCF"]),
            sc["delta_SCF"]["value"], sc["delta_SCF"]["U"], U_DSCF)
    compare("excited_delta_scf", f"{sid} E_excited", float(EXCITED[sid]["E_excited"]),
            sc["E_excited"]["value"], sc["E_excited"]["U"], U["E_KS"] + U_DSCF)
    say(f"{sid}: {len(rust_levels)} Rust levels, {len(ref_levels)} reference levels, "
        f"{len(common)} compared; {len(occupied_rust)} occupied in both")
    check(occupied_rust == occupied_ref, f"the same occupied levels in {sid}",
          record=f"{KS}/reports/ks-crosscheck.json, check "
                 "ground_occupations_and_groups")
```

`occupied_rust` and `occupied_ref` are the sorted keys of the occupied levels of each solver. The Delta-SCF energy and the energy of the excited state are compared (the latter with the sum of the uncertainties of E_{KS} and of the Delta-SCF energy). The printed line (Out [15]) shows how many levels each solver kept and how many were compared: the Rust solver keeps more levels than the reference (64 against 22 for N8_lamm2_a00), and every reference level has a Rust partner. The check requires the same occupied levels, reproducing the check `ground_occupations_and_groups`.

In [16], energy-momentum tensor and adiabaticity.

```
U_ADIABATIC = RK4_FACTOR * WIDE["refined_adiabatic_derivatives"]  # relative
for sid in GROUND_IDS:
    sc, U, e = NEW[sid]["scalars"], U_RUST[sid], EMT[sid]
    for k in ("rho", "p3", "p_t", "p0", "n"):
        compare("emt_integrals", f"{sid} int_{k}", float(e["int_" + k]),
                sc["int_" + k]["value"], sc["int_" + k]["U"], U["int_" + k])
    for k in ("rho", "p3", "p_t", "p0"):
        for end in ("brane", "tip"):
            name = f"{k}_{end}"
            compare("emt_brane_tip_values", f"{sid} {name}", float(e[name]),
                    sc[name]["value"], sc[name]["U"], U[name])
```

`U_ADIABATIC` is the matrix-wide relative uncertainty of the derivatives along the history. For each ground state the five integrals are compared, each with its measured U_{Rust}, and the four quantities ρ, p_3, p_t, p_0 at the brane and at the tip (eight comparisons; `name` is for example `rho_tip`).

```
stored = read_csv(f"{KS}/results/ground/profiles/{sid}.csv")
for name in PROFILE_NAMES:
    values = [float(row[name]) for row in stored]
    top = max(max(abs(v) for v in values), 1e-300)  # the scale of the profile
    ref = NEW[sid]["profiles"][name]
    for i, row in enumerate(stored):
        compare("ground_profiles", f"{sid} {name}(y = {float(row['y']):.2f})",
                values[i], ref["value"][i], ref["U"][i],
                U["profile_" + name][0], scale=top)
```

The committed Rust profiles of the state are read; for each of the ten profiles `top` is its largest value (the scale of Section 16.7) and `ref` the reference profile with its uncertainties. Every one of the 151 points is compared, its case named with the profile and the position, for example `rho(y = -3.00)`.

```
u_q, top_q = U["profile_Q"]
dex = float(e["deltaE_x_exact_fock"])
compare("exchange_delta_E_x", f"{sid} deltaE_x", dex,
        sc["deltaE_x_exact_fock"]["value"], sc["deltaE_x_exact_fock"]["U"],
        RK4_FACTOR * 2.0 * (u_q / RK4_FACTOR) / max(top_q, 1e-300) * abs(dex))
```

The exchange difference ΔE_x, with the uncertainty derived in Section 16.7: twice the relative uncertainty of the profile Q times $|\Delta E_x|$. The code multiplies by `RK4_FACTOR` and divides `u_q` by it again; the two factors cancel, and what remains is $2\,(U_Q / \max|Q|)\,|\Delta E_x|$, where U_Q already contains the factor $\frac{16}{15}$.

```
a = ADIABATIC[sid]
compare("adiabatic_dE_da4", f"{sid} dE_da4_emt", float(a["dE_da4_emt"]),
        sc["dE_da4_emt"]["value"], sc["dE_da4_emt"]["U"],
        3.0 * (U["int_p3"] + U["int_p_t"]))
fd = float(a["dE_da4_finite_difference"])
compare("adiabatic_dE_da4", f"{sid} dE_da4_finite_difference", fd,
        sc["dE_da4_fd"]["value"], sc["dE_da4_fd"]["U"], U_ADIABATIC * abs(fd))
q_rust, ad = float(a["Q_max"]), NEW[sid]["adiabatic"]
compare("adiabatic_Q_max", f"{sid} Q_max", q_rust, ad["Q_max"], ad["U_Q_max"],
        U_ADIABATIC * abs(q_rust))
```

The derivative dE/da_4 in its energy-momentum form, with the uncertainty $3(U_{I_3} + U_{I_t})$ derived in Section 16.7, and as a difference quotient, with the matrix-wide relative uncertainty. Then the adiabaticity measure Q_{max} of Chapter 15.

```
if q_rust > 0:
    top = ad["top"][0]  # the reference's maximising pair
    hole, particle = [s.strip() for s in a["Q_max_pair"].split("->")]
    say(f"{sid}: Q_max = {q_rust:.10f} for the pair {hole} -> {particle} (Rust), "
        f"{top['hole']} -> {top['particle']} (reference)")
```

```
        check((hole, particle) == (top["hole"], top["particle"]),
              f"both solvers find the same maximising pair in {sid}")
        me = float(a["Q_max_matrix_element"])
        compare("adiabatic_Q_max", f"{sid} Q_max matrix element", me,
                top["matrix_element"], top["U_matrix_element"], U_ADIABATIC * abs(me))
        compare("adiabatic_Q_max", f"{sid} Q_max delta eps", float(a["Q_max_delta_eps"]),
                top["delta_eps"], top["U_delta_eps"], 2.0 * U["levels"])
```

When Q_{max} is not zero (it is zero in both solvers for the states with $N = 8$: their eight quanta sit at zero 3-momentum, where the slice $a_{4,0}$ does not enter the equations, so every matrix element of the derivative along the history vanishes; `Revision/kohn_sham/reports/ks-crosscheck.json`, check `adiabatic_Q_max`), the maximising pair of levels must be the same in both solvers. The Rust table writes the pair as `hole -> particle`; `.split("->")` cuts the text at the arrow and `.strip()` removes the blanks. The printed lines (Out [16]) show $Q_{max} = 0.0934505917$ for N688_lam0_a00 and 0.0421562349 for N136_lamp2_a20, with the same pairs in both solvers; the check requires this. Then the matrix element and the level spacing of the pair are compared.

```
ground_rows = [r for r in ROWS if r[1].split()[0] in GROUND_IDS]
say(f"{len(ground_rows)} ground-state comparisons so far")
check(all(r[8] <= 1.0 for r in ground_rows),
      "every ground-state comparison of the subset passes the tolerance rule")
```

`ground_rows` are the comparisons whose case begins with a ground-state id (`.split()[0]` is the first word). Out [16] reports 5122 of them, and the check requires every ratio to be at most 1.

In [17], the thermal comparisons.

```
ROOT_TOLERANCE = RUST_PARAMS["numerics"]["rootTolerance"]  # 1e-13
U_HEAT = RK4_FACTOR * WIDE["refined_heat_capacity"]  # relative, matrix-wide
for sid in THERMAL_IDS:
    th, U, row = NEW[sid]["thermo"], U_RUST[sid], THERMO[sid]
    T, N = NEW[sid]["T"], NEW[sid]["N"]
    u_f = U["E_KS"] + T * U["entropy"]
    for k, u in (("mu", U["mu"]), ("E", U["E_KS"]), ("entropy", U["entropy"]),
                 ("F", u_f), ("Omega_direct", u_f + N * U["mu"]),
                 ("Omega_F_minus_muN", u_f + N * U["mu"])):
        compare("thermo_state_functions", f"{sid} {k}", float(row[k]),
                th[k]["value"], th[k]["U"], u)
```

`ROOT_TOLERANCE` is the Rust root tolerance 10^{-13}, read from its parameters, and `U_HEAT` the matrix-wide relative uncertainty of the heat capacity. For each thermal state, `u_f` is $U_F = U_E + T\,U_S$ (Section 16.7), and six state functions are compared: μ, E, S, F, and Ω in its two forms, both with $U_\Omega = U_F + N\,U_\mu$.

```
    dT = 0.01 * T
    for k in ("C_V", "C_V_from_dEdT", "minus_dFdT"):
        x = float(row[k])
        quotient = k != "C_V"  # a difference quotient of energies
        compare("thermo_derivatives", f"{sid} {k}", x, th[k]["value"],
```

```
            th[k]["U"] + (N * 1e-12 / dT if quotient else 0.0),
            U_HEAT * max(abs(x), 1e-6)
            + (N * ROOT_TOLERANCE / dT if quotient else 0.0))
```

The three derivatives with respect to T: $C_V = T\,dS/dT$ and the two difference quotients dE/dT and $-dF/dT$, for which the noise floors of Section 16.7 are added, $N \cdot 10^{-12}/dT$ to the reference and $N \cdot 10^{-13}/dT$ to the Rust solver, with $dT = 0.01\,T$ (`x if condition else y` takes `x` when the condition holds and `y` otherwise).

```
    r = next(r for r in ROWS if r[1] == f"{sid} mu")
    say(f"{sid}: mu Rust {r[2]!r}, reference {r[3]!r}, ratio {r[8]:.4f}")
thermal_rows = [r for r in ROWS if r[1].split()[0] in THERMAL_IDS]
say(f"{len(thermal_rows)} thermal comparisons; {len(ROWS)} comparisons in all")
check(len(ROWS) == 5140 and all(r[8] <= 1.0 for r in ROWS),
      "all 5140 comparisons of the subset pass the tolerance rule")
```

`next(...)` finds the comparison of μ of the state, and its two values and ratio are printed (Out [17]: for N8_lamm1_a00_T10 the two values of μ differ by 4.0×10^{-13}, ratio 0.0438). The last check counts all comparisons, 5140 (5122 ground and 18 thermal), and requires every ratio to be at most 1.

In [18], reproducing the committed cross-check report.

```
TABLE = {r["case"]: r for r in read_csv(f"{KS}/reports/ks-crosscheck-table.csv")}
matched, largest = 0, 0.0
for cls, case, *_, tolerance, diff, ratio in ROWS:
    if case in TABLE:
        matched += 1
        stored = TABLE[case]
        largest = max(largest, abs(ratio - float(stored["ratio"])))
        if abs(ratio - float(stored["ratio"])) > 6e-4 or \
                abs(tolerance - float(stored["tolerance"])) > 1e-3 * tolerance:
            say(f"differs: {case}: {ratio:.4f} vs {stored['ratio']}")
say(f"{matched} of our comparisons are rows of the committed table; the largest "
    f"difference of the ratio is {largest:.1e}")
check(matched == 97 and largest <= 6e-4,
      "our tolerances and ratios equal the 97 matching rows of the committed table",
      record=f"{KS}/reports/ks-crosscheck-table.csv")
```

The cross-check wrote every comparison of single numbers into its table `ks-crosscheck-table.csv` (levels and profile points are too many and appear only in the report). `TABLE` files the rows under their case names. The loop goes through our comparisons, unpacking each tuple into its class, case, tolerance, difference and ratio (`*_` collects the four middle entries, which are not needed); whenever a case is a row of the table, it counts it and compares our ratio and tolerance with the stored ones, which the table keeps with four significant digits. Out [18]: 97 of our comparisons are rows of the table, and the largest difference of a ratio is 5.0×10^{-5}. The check requires 97 matches within 6×10^{-4}.

```
WORST_CASES = {}  # class -> (worst ratio in the report, its case)
pattern = re.compile(r"worst \|diff\|/tolerance ([0-9.]+) \((.+?): Rust ")
```

```
for c in REPORTS["cross-check"]["checks"]:
    found = pattern.search(c["detail"])
    if found and c["name"] in ("ground_eigenvalues", "ground_profiles",
                               "emt_brane_tip_values", "adiabatic_Q_max",
                               "thermo_state_functions", "thermo_derivatives"):
        WORST_CASES[c["name"]] = (float(found.group(1)), found.group(2))
for cls, (ratio, case) in sorted(WORST_CASES.items()):
    mine = max((r for r in ROWS if r[0] == cls), key=lambda r: r[8])
    say(f"{cls}: report {ratio:.3f} ({case}); this notebook {mine[8]:.3f} "
        f"({mine[1]})")
    check(mine[1] == case and abs(mine[8] - ratio) <= 6e-4,
          f"the worst case of {cls} is reproduced",
          record=f"{KS}/reports/ks-crosscheck.json, check {cls}")
```

`re.compile` prepares a pattern that reads, from the detail sentence of a check of the cross-check report, the worst ratio and its case: `\|` is a literal vertical bar, and `(.+?)` captures the shortest text up to `:  Rust` (the question mark makes the match as short as possible). For six classes the worst ratio and case are kept in `WORST_CASES`. For each, `mine` is our comparison with the largest ratio in that class. Out [18] prints both: the worst case of the whole cross-check in each of the six classes is a comparison of our subset, with the same ratio (0.314, 0.495, 0.324, 0.495, 0.226, 0.284). The six checks require the same case and the same ratio within 6×10^{-4}.

In [19], the levels of N136_lamp2_a20 (Figure 16a.4).

```
rows = [r for r in ROWS if r[0] == "ground_eigenvalues"
        and r[1].startswith("N136_lamp2_a20 ")]
worst = max(rows, key=lambda r: r[8])
fig, ax = plt.subplots(figsize=(7.5, 4.3))
ax.semilogy([r[3] for r in rows], [max(r[8], 1e-6) for r in rows], "o",
            color=PALETTE[0], ms=4, alpha=0.8, label="one level of N136_lamp2_a20")
ax.semilogy([worst[3]], [worst[8]], "o", color=PALETTE[1], ms=9,
            label=f"largest ratio {worst[8]:.3f}: level "
                  f"{worst[1].split('level ')[1]}")
ax.axhline(1.0, color="k", lw=1.2, label="ratio 1: the tolerance")
ax.set_ylim(1e-6, 3.0)
ax.set_xlabel("level energy $\\varepsilon$ (reference value, units of $m$)")
ax.set_ylabel("$|\\varepsilon_{Rust} - \\varepsilon_{ref}|$ / tolerance")
ax.set_title(f"{len(rows)} levels of N136_lamp2_a20 compared")
ax.legend(fontsize=8, loc="lower right")
```

`rows` are the 354 level comparisons of N136_lamp2_a20 (`startswith` tests the beginning of the case name), and `worst` the one with the largest ratio. The figure draws each level's ratio against its reference energy on a logarithmic vertical axis (ratios below 10^{-6} raised to 10^{-6}), the worst level as a large orange circle with its key in the legend (`.split('level ')[1]` takes the text after `level`), and the line ratio 1.

```
save_figure(fig, "level_agreement",
            "Agreement of the two solvers for every level of the ground state "
            "N136_lamp2_a20 ($N = 136$, $\\lambda = +\\lambda_2$, $a_{4,0} = 2$): the "
            "difference of the two level energies divided by the tolerance $3(U_{ref} "
            "+ U_{Rust}) + 10^{-12}\\max(1, |\\varepsilon|)$, against the level energy "
```

```
            "in units of $m$ (vertical axis logarithmic; ratios below one millionth "
            "are drawn at $10^{-6}$). Every point lies below the line 1, so every "
            f"level passes; the largest ratio, {worst[8]:.3f}, is the largest of all "
            "9616 level comparisons of the full cross-check.")
check(len(rows) == 354 and worst[8] < 1.0,
      "all 354 levels of N136_lamp2_a20 agree within the tolerance")
```

`save_figure` saves Figure 16a.4. What the student should see: every point lies below the line 1. The ratios rise with the level energy, from below 10^{-4} near 0 to 0.324 for the level 1:-1:even:2 at 3.24, because the higher levels' orbitals oscillate faster and both solvers are less accurate there; the largest ratio is the largest of all 9616 level comparisons of the full cross-check. The check requires 354 levels and the largest ratio below 1.

In [20], the profile with the largest ratio (Figure 16a.5).

```
sid = "N8_lamm2_a00"
stored = read_csv(f"{KS}/results/ground/profiles/{sid}.csv")
y = np.array([float(row["y"]) for row in stored])
fig, (top, bottom) = plt.subplots(2, 1, figsize=(7.5, 6.6), sharex=True)
top.plot(y, [float(row["rho"]) for row in stored], color=PALETTE[0], lw=1.8,
         label="Rust solver")
top.plot(y[::5], NEW[sid]["profiles"]["rho"]["value"][::5], "o", color=PALETTE[1],
         ms=5, mfc="none", label="reference solver")
top.set_ylabel("$\\rho(y)$ (proper energy density, units of $m$)")
top.set_title("Energy density of N8_lamm2_a00")
top.legend(fontsize=8)
```

The committed Rust profiles of N8_lamm2_a00 are read, `y` holds the 151 positions $-3, -2.98, \ldots, 0$, and a figure with two panels one above the other is made (`sharex=True` gives them the same horizontal axis). The upper panel draws the Rust energy density as a line and the reference energy density at every fifth point (`[::5]`) as open circles (`mfc="none"`: no fill).

```
largest = 0.0
for colour, name in ((PALETTE[0], "rho"), (PALETTE[2], "p0")):
    ratios = [r[8] for r in ROWS if r[0] == "ground_profiles"
              and r[1].startswith(f"{sid} {name}(")]
    largest = max(largest, max(ratios))
    bottom.semilogy(y, np.maximum(ratios, 1e-6), "-", color=colour, lw=1.5,
                    label=f"profile {name}")
bottom.axhline(1.0, color="k", lw=1.2, label="ratio 1: the tolerance")
bottom.set_ylim(1e-6, 3.0)
bottom.set_xlabel("hidden coordinate $y$ (tip $y = -3$, brane $y = 0$)")
bottom.set_ylabel("$|$Rust $-$ reference$|$ / tolerance")
bottom.legend(fontsize=8, loc="center right")
fig.tight_layout()
```

The lower panel draws, for ρ and for p_0, the ratio of each of the 151 point comparisons against y (`np.maximum` raises the small ratios to 10^{-6} point by point), and the line ratio 1; `largest` keeps the largest ratio of the two profiles.

```
save_figure(fig, "profile_agreement",
            "The profile with the largest ratio of the whole cross-check. Top: the "
            "proper energy density $\\rho(y)$ of the ground state N8_lamm2_a00 "
            "($N = 8$, $\\lambda = -\\lambda_2$, $a_{4,0} = 0$) from the Rust solver "
            "(line) and the reference solver (circles), in units of $m$, against the "
            "hidden coordinate $y$. Bottom: the difference divided by the tolerance "
            "at each of the 151 points for $\\rho$ and $p_0$ (logarithmic). The "
            f"largest ratio, {largest:.3f}, is at the tip $y = -3$, where the "
            "densities are largest and both solvers are least accurate.")
```

`save_figure` saves Figure 16a.5. What the student should see: in the upper panel the energy density of this state is negative and steep near the tip (it reaches about -10.2 at $y = -3$) and practically zero over most of the interval, and the circles of the reference lie on the Rust line; in the lower panel the ratios fall from 0.495 at the tip to below 10^{-6} towards the brane. The largest ratio of the whole cross-check sits at the tip because that is where the densities, and with them both programs' errors, are largest.

In [21], who sets the tolerance? (Figure 16a.6).

```
groups = {"ground states, single numbers": [], "ground states, levels": [],
          "thermal states": []}
for r in ROWS:
    if r[0] == "ground_profiles" or r[4] <= 0.0 or r[5] <= 0.0:
        continue  # profile points are drawn elsewhere; a zero U has no logarithm
    if r[1].split()[0] in THERMAL_IDS:
        groups["thermal states"].append(r)
    elif r[0] == "ground_eigenvalues":
        groups["ground states, levels"].append(r)
    else:
        groups["ground states, single numbers"].append(r)
```

The comparisons are sorted into three groups for the figure: ground-state single numbers, ground-state levels and thermal states. Profile points are left out (they appear in Figure 16a.7), and so are comparisons in which one of the two uncertainties is exactly zero, because zero has no logarithm (`continue` skips to the next comparison).

```
fig, ax = plt.subplots(figsize=(6.6, 5.6))
for colour, (label, rows) in zip(PALETTE, groups.items()):
    ax.loglog([r[4] for r in rows], [r[5] for r in rows], "o", color=colour, ms=4,
              alpha=0.75, label=f"{label} ({len(rows)})")
span = np.array([1e-18, 1e-5])
ax.loglog(span, span, "k-", lw=1.0, label="$U_{Rust} = U_{ref}$")
ax.set_xlabel("$U_{ref}$ (Richardson uncertainty of the reference)")
ax.set_ylabel("$U_{Rust}$ (16/15 of canonical minus refined)")
ax.set_title("Who sets the tolerance?")
ax.legend(fontsize=8)
above = sum(1 for rows in groups.values() for r in rows if r[5] > r[4])
total = sum(len(rows) for rows in groups.values())
```

Each group is drawn as points at the position (U_{ref}, U_{Rust}) on logarithmic axes, with the diagonal $U_{Rust} = U_{ref}$ as a black line. `above` counts the points above the diagonal and `total` all points.

```
save_figure(fig, "uncertainty_budget",
            "The two uncertainties that make the tolerance, for every comparison of "
            "single numbers and of levels in the five states of the subset: the "
            "Richardson uncertainty $U_{ref}$ of the reference solver (horizontal) "
            "and the measured uncertainty $U_{Rust}$ of the Rust solver (vertical), "
            "both logarithmic and in the units of the quantity. Points above the "
            "diagonal are dominated by the Rust uncertainty, points below it by the "
            f"reference; {above} of the {total} points lie above it. Comparisons in "
            "which one uncertainty is exactly zero are left out.")
say(f"{above} of {total} drawn comparisons have U_Rust > U_ref")
check(total > 500, "the uncertainty budget has more than 500 comparisons")
```

`save_figure` saves Figure 16a.6, and the printed line (Out [21]) says that 547 of the 602 drawn comparisons have $U_{Rust} > U_{ref}$. What the student should see: most points lie above the diagonal, so in most comparisons the measured Rust uncertainty is the larger part of the tolerance. The reference uncertainties of the levels lie between their floor $2 \cdot 10^{-12}$ and about 4×10^{-10} (the largest, 3.5×10^{-10}, belongs to a high level of N688_lam0_a00; `Revision/kohn_sham/reference/results/ground/N688_lam0_a00.json`), while the Rust level uncertainties form horizontal rows, one value per state (the largest level difference of the state, Section 16.6). The reference, with its three grids and two Richardson steps, is usually the more accurate program, and the tolerance is set mainly by the Rust solver. The check requires more than 500 drawn comparisons.

In [22], all comparisons by class (Figure 16a.7).

```
classes = ["ground_energies", "ground_homo_lumo_gap", "ground_eigenvalues",
           "excited_delta_scf", "emt_integrals", "emt_brane_tip_values",
           "ground_profiles", "exchange_delta_E_x", "adiabatic_dE_da4",
           "adiabatic_Q_max", "thermo_state_functions", "thermo_derivatives"]
rng = np.random.default_rng(12345)  # fixed seed: the same picture in every run
fig, ax = plt.subplots(figsize=(9.0, 5.0))
worst_by_class = {}
```

`classes` lists the twelve classes of the cross-check that the subset fills. `rng` is a **random-number generator** with a fixed starting value (the **seed** 12345), so that the "random" numbers it gives are the same in every run and the figure is the same byte for byte. A figure of 9 by 5 inches is made, and `worst_by_class` will hold each class's largest ratio.

```
for i, cls in enumerate(classes):
    ratios = np.array([r[8] for r in ROWS if r[0] == cls])
    jitter = rng.uniform(-0.28, 0.28, size=len(ratios))
    ax.semilogy(i + jitter, np.maximum(ratios, 1e-6), "o", color=PALETTE[0], ms=2.5,
                alpha=0.35)
    worst_by_class[cls] = float(ratios.max())
    ax.semilogy([i], [max(ratios.max(), 1e-6)], "D", color=PALETTE[1], ms=7)
    ax.text(i, 1.6, f"{len(ratios)}", ha="center", fontsize=7)
```

For each class `ratios` is the array of its ratios. `rng.uniform(-0.28, 0.28, size=...)` gives one random number between -0.28 and 0.28 per ratio; adding it to the class's position spreads the dots a little sideways so that they do not hide one another (**jitter**). The dots are drawn small and half transparent (`alpha=0.35`), the class's largest ratio as an orange diamond, and the number of comparisons of the class as a small text above the plot area.

```
ax.axhline(1.0, color="k", lw=1.2)
ax.set_xticks(range(len(classes)))
ax.set_xticklabels([c.replace("_", " ") for c in classes], rotation=40, ha="right",
                   fontsize=8)
ax.set_ylim(1e-6, 4.0)
ax.set_ylabel("$|x_{Rust} - x_{ref}|$ / tolerance")
ax.set_title("All comparisons of the subset (number of comparisons above each class)")
fig.tight_layout()
```

The line ratio 1 is drawn; the class names, with underscores replaced by blanks, are written under their positions, turned by 40 degrees and aligned at their right ends; the vertical axis runs from 10^{-6} to 4.

```
top_class = max(worst_by_class, key=worst_by_class.get)
save_figure(fig, "ratios_by_class",
            f"All {len(ROWS)} comparisons of the five states of the subset, grouped "
            "by the class of the cross-check: each dot is one ratio of the difference "
            "of the two solvers to the tolerance (logarithmic; ratios below "
            "$10^{-6}$ drawn at $10^{-6}$), the diamond is the largest ratio of the "
            "class, the number above a class counts its comparisons, and the black "
            "line is the tolerance. Every dot lies below it; the largest ratio, "
            f"{worst_by_class[top_class]:.3f}, belongs to the tip value of the energy "
            "density of N8_lamm2_a00.")
for cls in classes:
    say(f"  {cls:24s} largest ratio {worst_by_class[cls]:.4f}")
top_row = max(ROWS, key=lambda r: r[8])  # the comparison with the largest ratio
say(f"the largest ratio of the subset: {top_row[8]:.4f} ({top_row[1]})")
check(max(worst_by_class.values()) < 0.5
      and top_row[1] == "N8_lamm2_a00 rho_tip",
      "no comparison uses half of its tolerance; the largest is rho at the tip")
```

`top_class` is the class with the largest ratio (`max` with `key=worst_by_class.get` compares the classes by their values). `save_figure` saves Figure 16a.7, the loop prints each class's largest ratio (Out [22]), and `top_row` is the single comparison with the largest ratio of all 5140: 0.4952, the energy density at the tip of N8_lamm2_a00. What the student should see: every dot of every class lies below the line 1; the 4530 profile points fill a dense column from 10^{-6} to just below 0.5; and no class comes closer to the line than about one half. The check requires every largest ratio below 0.5 and the overall largest at `N8_lamm2_a00 rho_tip`.

In [23], the last check.

```
figure_files = [f"{FIGURE_FOLDER}/16a_{k}_{name}.png" for k, name in enumerate(
    ["grid_convergence", "uncertainty_validated", "eigenvalue_ratios",
     "level_agreement", "profile_agreement", "uncertainty_budget",
     "ratios_by_class"], start=1)]
check(all(output_file(f).is_file() for f in figure_files),
      "every figure file of this notebook exists")
all_checks_passed()
```

`enumerate(..., start=1)` numbers the seven figure names from 1, and the list comprehension builds their file names, for example `Revision/textbook/figures/16a_1_grid_convergence.png`.

The check confirms that each file exists where the notebook writes (`output_file`), and `all_checks_passed()` prints the last line, ALL 51 CHECKS PASSED (notebook 16a): three checks each in In [2], In [5], In [6], In [7], In [15] and In [16], four in In [10], ten in In [13], seven in In [18], two in In [11], and one each in In [3], In [4], In [8], In [12], In [14], In [17], In [19], In [21], In [22] and In [23].

16.13 Inside the reference solver I: the free problem and its exact levels

Why a problem with known answers. A grid method can be tested best on a problem whose exact answers are known, because then its error can be measured directly instead of estimated. The Kohn-Sham problem has such a special case: no interaction ($\lambda = 0$, so $M = m$ and $v = 0$) and no 3-momentum ($k = 0$). We call it the **free problem at zero momentum**. With $k = 0$ the factor κ drops out of the equations of Section 16.3, and so does the slice $a_{4,0}$:

$$a' = Ma - j\varepsilon\, b, \qquad b' = j\varepsilon\, a - Mb,$$

with a constant $M > 0$ (in the numbers $M = m = 1$, $L = 3$). Its exact levels are derived in this section (in the record: `Revision/kohn_sham/ks-theory.json`, `boundaryConditions.exactK0Spectra`, verified by check bc_exact_k0_spectra of `Revision/kohn_sham/reports/ks-theory-python.json`; status PROVED). Notebook 16b checks the derivation with the symbolic algebra package sympy and uses the levels to measure the errors of the reference's grid.

Even parity, line by line. The conditions are $b(-L) = 0$ (tip) and $b(0) = 0$ (brane). Let $\varepsilon \neq 0$; recall $j = \pm 1$, so $j^2 = 1$.

$$j\, b' = \varepsilon\, a - jM\, b \quad \Longrightarrow \quad a = \frac{j\,(b' + Mb)}{\varepsilon}.$$

Rule: multiply the second equation by j (using $j^2 = 1$), move jMb to the left side and divide by ε.

$$\frac{j\,(b'' + Mb')}{\varepsilon} = M\,\frac{j\,(b' + Mb)}{\varepsilon} - j\varepsilon\, b.$$

Rule: insert this a into the first equation $a' = Ma - j\varepsilon b$; the derivative of a is $j(b'' + Mb')/\varepsilon$ because M and ε are constants.

$$b'' + Mb' = Mb' + M^2 b - \varepsilon^2 b.$$

Rule: multiply both sides by ε/j (which equals $j\varepsilon$, since $1/j = j$).

$$b'' = -\left(\varepsilon^2 - M^2\right) b.$$

Rule: cancel Mb' on both sides and collect the terms with b.

If $\varepsilon^2 > M^2$, write $p^2 = \varepsilon^2 - M^2$ with $p > 0$. The equation $b'' = -p^2 b$ has the solutions $\alpha \sin(p(y + L)) + \beta \cos(p(y + L))$ (differentiate twice to check). The tip condition $b(-L) = 0$ gives $\beta = 0$, and since the equations are linear we may take $\alpha = 1$:

$$b = \sin\big(p\,(y+L)\big), \qquad b(0) = \sin(pL) = 0 \quad\Longrightarrow\quad p = \frac{n\pi}{L},\ n = 1, 2, 3, \ldots$$

Rule: the sine vanishes exactly at the whole multiples of π. Hence the even levels are

$$\varepsilon = \pm\sqrt{M^2 + \Big(\frac{n\pi}{L}\Big)^2}, \qquad n = 1, 2, 3, \ldots$$

Rule: $\varepsilon^2 = M^2 + p^2$, and both signs of the square root are allowed. (For $\varepsilon^2 \le M^2$ the solutions with $b(-L) = 0$ are $\sinh(q(y+L))$ or $y + L$, which do not vanish again at $y = 0$; so there are no other nonzero levels.) For $M = 1$, $L = 3$ the first one is $\sqrt{1 + (\pi/3)^2} = \sqrt{2.096623} = 1.447972$.

The zero mode, line by line. For $\varepsilon = 0$ the two equations no longer mix a and b:

$$a' = Ma, \qquad b' = -Mb \quad\Longrightarrow\quad a = A\,e^{My}, \qquad b = B\,e^{-My}.$$

Rule: the solution of $f' = cf$ is $f = f(0)\,e^{cy}$ (Chapter 2). The tip condition $b(-L) = B\,e^{ML} = 0$ forces $B = 0$, so $b = 0$ everywhere and the brane condition $b(0) = 0$ holds automatically: $\varepsilon = 0$ is an even level, the **zero mode**, whose orbital $a = A\,e^{My}$ grows towards the brane. Its normalisation $\int_{-L}^{0}(a^2 + b^2)\,dy = 1$ fixes A:

$$A^2\int_{-L}^{0} e^{2My}\,dy = A^2\Big[\frac{e^{2My}}{2M}\Big]_{-L}^{0} = A^2\,\frac{1 - e^{-2ML}}{2M} = 1 \quad\Longrightarrow\quad A = \sqrt{\frac{2M}{1 - e^{-2ML}}}.$$

Rule: the antiderivative of e^{2My} is $e^{2My}/(2M)$, evaluated at 0 and at $-L$; then solve for $A > 0$. For $M = 1$, $L = 3$: $A = \sqrt{2/(1 - e^{-6})} = 1.415970$.

Odd parity, line by line. Now the conditions are $b(-L) = 0$ and $a(0) = 0$. The derivation of $b'' = -(\varepsilon^2 - M^2)b$ did not use any brane condition, so again $b = \sin(p(y+L))$ and $a = j(b' + Mb)/\varepsilon$:

$$a = \frac{j\,\big(p\cos(p(y+L)) + M\sin(p(y+L))\big)}{\varepsilon}, \qquad a(0) = 0 \iff p\cos(pL) + M\sin(pL) = 0.$$

Rule: differentiate b ($b' = p\cos(p(y+L))$), insert, and set $y = 0$; the factor j/ε is not zero.

$$\tan(pL) = -\frac{p}{M}.$$

Rule: divide by $M\cos(pL)$ (which is not zero at a root: if $\cos(pL) = 0$ then $\sin(pL) = \pm 1$ and the left side would be $\pm M \neq 0$).

No odd level with $\varepsilon^2 \le M^2$, line by line. The sine above assumed $\varepsilon^2 > M^2$. We use the **hyperbolic sine and cosine**, $\sinh t = (e^t - e^{-t})/2$ and $\cosh t = (e^t + e^{-t})/2$: each is the derivative of the other, $\sinh 0 = 0$, and both are positive for $t > 0$. The three remaining cases give no level.

$$\varepsilon = 0: \qquad a = A\,e^{My}, \quad b = 0, \qquad a(0) = A = 0.$$

Rule: with $\varepsilon = 0$ the equations are $a' = Ma$ and $b' = -Mb$; as for the even zero mode, $b(-L) = 0$ gives $b = 0$ and $a = A\,e^{My}$; the odd brane condition $a(0) = 0$ then forces $A = 0$. Only the trivial solution $a = b = 0$ is left: odd parity has no zero mode.

$$0 < \varepsilon^2 < M^2: \qquad b = \sinh(q(y+L)), \qquad a(0) = \frac{j\,(q\cosh(qL) + M\sinh(qL))}{\varepsilon} \neq 0.$$

Rule: now $b'' = -(\varepsilon^2 - M^2)b = q^2 b$ with $q = \sqrt{M^2 - \varepsilon^2} > 0$, and $\sinh(q(y+L))$ solves it (differentiate twice) and vanishes at the tip; insert into $a = j(b' + Mb)/\varepsilon$ and set $y = 0$. Both terms of the bracket are positive, because q, M and L are positive, so $a(0) = 0$ is impossible.

$$\varepsilon^2 = M^2: \qquad b = y + L, \qquad a(0) = \frac{j\,(1 + ML)}{\varepsilon} \neq 0.$$

Rule: now $b'' = 0$, so b is a straight line, and the one that vanishes at the tip is $y + L$ (times a constant); insert as before, with $b' = 1$; $1 + ML > 0$. Hence every odd level has $|\varepsilon| > M$: the lowest positive odd level is $+\sqrt{M^2 + p_0^2} = 1.2923$, the bulk edge, and this is why odd parity has no zero mode (Section 16.19).

One root in each interval. Let $F(p) = M\sin(pL) + p\cos(pL)$ and $l = 0, 1, 2, \ldots$. At $pL = (l + \frac{1}{2})\pi$ we have cos $= 0$ and sin $= (-1)^l$, so $F = M(-1)^l$. At $pL = (l+1)\pi$ we have sin $= 0$ and cos $= (-1)^{l+1}$, so $F = p\,(-1)^{l+1}$. The two values have opposite signs, so F has a root p_l between them (a continuous function that changes sign has a zero in between: the intermediate value theorem). There is only one: on that interval $\tan(pL)$ increases from $-\infty$ to 0 while $-p/M$ decreases, so they meet once; and on the intervals $l\pi < pL < (l + \frac{1}{2})\pi$ there is none, because there $\tan(pL) > 0 > -p/M$. So the odd levels are $\varepsilon = \pm\sqrt{M^2 + p_l^2}$, $l = 0, 1, 2, \ldots$, with p_l the root in $((l + \frac{1}{2})\pi/L, (l+1)\pi/L)$. For $M = 1$, $L = 3$ the lowest is 1.292292828069 (Notebook 16b, Out [2]); this is the **bulk edge** of Chapter 15, the lowest level that does not belong to the brane band.

Ranks. The levels of one sector (one parity and one block type) are numbered from the lowest **particle** level, which gets the **rank** 0, upwards $1, 2, \ldots$ and downwards $-1, -2, \ldots$. By the convention of `ks-theory.json` the brane zero modes are particles. So for even parity rank 0 is the zero mode, rank $n \geq 1$ is $+\sqrt{M^2 + (n\pi/L)^2}$ and rank $-n$ its negative; for odd parity rank $r \geq 0$ is $+\sqrt{M^2 + p_r^2}$ and rank $-r - 1$ its negative. Notebook 16b computes the ranks -3 to 5 of both parities (Out [2]); for example the even levels of ranks 1 to 5 are 1.447972, 2.320881, 3.296908, 4.306502 and 5.330625.

16.14 Inside the reference solver II: the staggered grid and the matrix

The equations as an eigenvalue problem, line by line. A matrix method needs the equations in the form "level times orbital equals an operator applied to the orbital".

$$\varepsilon\, a = j\,(b' + Mb).$$

Rule: from the second equation $b' = j\varepsilon a - Mb$, multiply by j and move jMb over (as in Section 16.13).

$$\varepsilon\, b = j\,(-a' + Ma).$$

Rule: from the first equation $a' = Ma - j\varepsilon b$, multiply by $-j$ ($-ja' = -jMa + \varepsilon b$, using $j^2 = 1$) and solve for εb. (With a 3-momentum and a potential the record's operator is the same with $+jK + v$ added to the first line and $-jK + v$ to the second, $K = \kappa k$: `Revision/kohn_sham/reference/ks_fd.py`, its header.)

The staggered grid. Divide $-L \le y \le 0$ into G equal **cells** of width $h = L/G$. The cell ends, the **nodes**, are $y_i = -L + i\,h$ for $i = 0, \ldots, G$; the cell centres, the **half nodes**, are $y_{p+1/2} = -L + (p + \frac{1}{2})\,h$ for $p = 0, \ldots, G-1$. The reference stores a at the half nodes and b at the nodes: $u_p \approx a(y_{p+1/2})$ and $w_i \approx b(y_i)$. At the two end nodes b is zero for even parity ($w_0 = 0$ at the tip and $w_G = 0$ at the brane), so they are not unknowns. The unknowns, in the order of y, are

$$(u_0,\ w_1,\ u_1,\ w_2,\ \ldots,\ w_{G-1},\ u_{G-1}),$$

G values of u and $G-1$ values of w, $2G-1$ in all (Figure 16b.1 draws them for $G = 6$). Such a grid, on which the two components live on alternating points, is called **staggered**. On an ordinary grid, with both components at the same points and the centred difference $(f_{i+1} - f_{i-1})/(2h)$, a zigzag pattern $+1, -1, +1, \ldots$ has a difference of zero and produces spurious extra levels (the **doubling** problem of naive discretisations of Dirac-type equations; the record's reference README names it as the reason for the staggering, `Revision/kohn_sham/reference/README.md`). On the staggered grid every difference connects neighbours one half cell apart, and no such pattern escapes.

Centred differences and averages, line by line. Let f be a smooth function. Taylor's theorem (Chapter 2) gives

$$f(y \pm \tfrac{h}{2}) = f \pm \tfrac{h}{2}\, f' + \tfrac{h^2}{8}\, f'' \pm \tfrac{h^3}{48}\, f''' + \tfrac{h^4}{384}\, f'''' \pm \ldots,$$

Rule: $f(y+s) = f + sf' + \frac{s^2}{2} f'' + \frac{s^3}{6} f''' + \frac{s^4}{24} f'''' + \ldots$ with $s = \pm h/2$, so $\frac{s^2}{2} = \frac{h^2}{8}$, $\frac{s^3}{6} = \pm\frac{h^3}{48}$, $\frac{s^4}{24} = \frac{h^4}{384}$.

$$\frac{f(y + \frac{h}{2}) - f(y - \frac{h}{2})}{h} = f' + \frac{h^2}{24}\, f''' + \ldots$$

Rule: subtract the two expansions; the even terms cancel and the odd terms double ($2 \cdot \frac{h}{2} = h$, $2 \cdot \frac{h^3}{48} = \frac{h^3}{24}$), then divide by h.

$$\frac{f(y + \frac{h}{2}) + f(y - \frac{h}{2})}{2} = f + \frac{h^2}{8}\, f'' + \ldots$$

Rule: add the two expansions; now the odd terms cancel; divide by 2.

So the **centred difference** and the **centred average** of the two neighbours half a cell away are second-order approximations of f' and f at the midpoint, and their errors contain only even powers of h (the next terms are h^4, h^6, ...). This is where the even-power expansion of Section 16.4 comes from.

The rows of the matrix, line by line. Write the first eigenvalue equation at the half node $y_{p+1/2}$, the place where u_p lives; its neighbours of b are w_p and w_{p+1}:

$$\varepsilon\, u_p = j\Big(\frac{w_{p+1}-w_p}{h} + M\,\frac{w_p+w_{p+1}}{2}\Big) = j\Big(-\frac{1}{h}+\frac{M}{2}\Big)\,w_p + j\Big(\frac{1}{h}+\frac{M}{2}\Big)\,w_{p+1}.$$

Rule: replace b' by the centred difference and b by the centred average, then collect the coefficients of w_p and w_{p+1}. Write the second equation at the node y_i, where w_i lives; its neighbours of a are u_{i-1} and u_i:

$$\varepsilon\, w_i = j\Big(-\frac{u_i-u_{i-1}}{h} + M\,\frac{u_{i-1}+u_i}{2}\Big) = j\Big(\frac{1}{h}+\frac{M}{2}\Big)\,u_{i-1} + j\Big(-\frac{1}{h}+\frac{M}{2}\Big)\,u_i.$$

Rule: the same replacements for a' and a, then collect.

The matrix is tridiagonal and symmetric. In the order of the unknowns each row couples only to its two neighbours in the list (a u to the w on either side, a w to the u on either side), so the matrix T of the problem $Tz = \varepsilon z$ is **tridiagonal**; at $k = 0$ its diagonal is zero. It is also **symmetric** ($T_{rc} = T_{cr}$): the coefficient of w_{p+1} in the row of u_p is $j(\frac{1}{h}+\frac{M}{2})$, and the coefficient of u_p in the row of w_{p+1} (put $i = p+1$, so $u_{i-1} = u_p$) is the same number; likewise the coefficient of w_p in the row of u_p and of u_p in the row of w_p (put $i = p$) are both $j(-\frac{1}{h}+\frac{M}{2})$. So the entry between the neighbours number r and $r+1$ of the list is

$$T_{r,r+1} = T_{r+1,r} = \begin{cases} j\,(\frac{1}{h}+\frac{M}{2}) & r \text{ even (a } u \text{ followed by a } w), \\ j\,(-\frac{1}{h}+\frac{M}{2}) & r \text{ odd (a } w \text{ followed by a } u). \end{cases}$$

For $G = 4$ ($h = 0.75$) and $M = 1$ the two values are 1.8333 and -0.8333 (Notebook 16b, Out [4] prints the whole 7×7 matrix). Symmetry matters: a real symmetric matrix has only real eigenvalues and an orthonormal set of eigenvectors (the **spectral theorem**, quoted from linear algebra), as the levels of a self-adjoint problem must; and the counting method of Section 16.15 works only for symmetric matrices.

Odd parity: the rotated frame. For odd parity the brane condition is $a(0) = 0$, but a lives at half nodes, so no unknown sits at $y = 0$ to be set to zero. The reference rotates the two components before it discretises, so that both conditions become "b-type component zero at the ends" again. With the Pauli matrices $\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $\sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, $\sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ and the complex orbital $\chi = (a, ib)$ of Chapter 14, write $\chi = R\,\psi$ with $\psi = (u, iw)$ and $R = e^{i\phi\sigma_1}$ for an angle $\phi(y)$.

$$R = e^{i\phi\sigma_1} = \cos\phi + i\sin\phi\,\sigma_1.$$

Rule: the series $e^X = 1 + X + X^2/2 + \ldots$ with $X = i\phi\sigma_1$ and $\sigma_1^2 = 1$ splits into the even terms, the series of $\cos\phi$, and the odd terms, $i\sigma_1$ times the series of $\sin\phi$.

$$\begin{pmatrix} a \\ ib \end{pmatrix} = \begin{pmatrix} \cos\phi\, u + i\sin\phi\cdot iw \\ \cos\phi\cdot iw + i\sin\phi\, u \end{pmatrix} = \begin{pmatrix} \cos\phi\, u - \sin\phi\, w \\ i\,(\sin\phi\, u + \cos\phi\, w) \end{pmatrix}.$$

Rule: σ_1 exchanges the two components of (u, iw), and $i \cdot i = -1$. So $a = \cos\phi\, u - \sin\phi\, w$ and $b = \sin\phi\, u + \cos\phi\, w$. The reference chooses $\phi = j\,\frac{\pi}{2}\,\frac{y+L}{L}$: at the tip $\phi = 0$, so $b = w$ and $b(-L) = 0$ means $w(-L) = 0$; at the brane $\phi = j\pi/2$, so $\cos\phi = 0$, $\sin\phi = j$, $a = -j\,w$, and $a(0) = 0$ means $w(0) = 0$. Both conditions are again "$w = 0$ at the ends", and the same staggered grid works.

The rotated operator, line by line. The operator of Section 16.3 is $h_j = j[-i\sigma_1 \frac{d}{dy} + M\sigma_2 + K\sigma_3] + v$, and $h_j\chi = \varepsilon\chi$ becomes $R^{-1}h_j R\,\psi = \varepsilon\psi$, with $R^{-1} = \cos\phi - i\sin\phi\,\sigma_1$.

$$R^{-1}\Big(-i\sigma_1\frac{d}{dy}\Big)(R\psi) = -i\sigma_1(\psi' + R^{-1}R'\,\psi) = -i\sigma_1\psi' - i\sigma_1\cdot i\phi'\sigma_1\,\psi = -i\sigma_1\psi' + \phi'\,\psi.$$

Rule: the product rule $(R\psi)' = R'\psi + R\psi'$; σ_1 commutes with R; $R^{-1}R' = i\phi'\sigma_1$ (differentiate $e^{i\phi\sigma_1}$); and $-i\cdot i\cdot\sigma_1^2 = 1$. The derivative of the rotation adds the plain number ϕ'.

$$R^{-1}\sigma_2 R = (\cos\phi - i\sin\phi\,\sigma_1)(\cos\phi\,\sigma_2 + \sin\phi\,\sigma_3) = \cos 2\phi\,\sigma_2 + \sin 2\phi\,\sigma_3.$$

Rule: first $\sigma_2 R = \cos\phi\,\sigma_2 + i\sin\phi\,\sigma_2\sigma_1 = \cos\phi\,\sigma_2 + \sin\phi\,\sigma_3$ (because $\sigma_2\sigma_1 = -i\sigma_3$); then multiply out with $\sigma_1\sigma_2 = i\sigma_3$, $\sigma_1\sigma_3 = -i\sigma_2$, and use $\cos^2\phi - \sin^2\phi = \cos 2\phi$, $2\sin\phi\cos\phi = \sin 2\phi$. In the same way $R^{-1}\sigma_3 R = \cos 2\phi\,\sigma_3 - \sin 2\phi\,\sigma_2$ (Notebook 16b checks both with sympy, In [9]).

$$R^{-1}(M\sigma_2 + K\sigma_3)R = \big(M\cos 2\phi - K\sin 2\phi\big)\,\sigma_2 + \big(M\sin 2\phi + K\cos 2\phi\big)\,\sigma_3 = m_2\,\sigma_2 + k_2\,\sigma_3.$$

Rule: insert the two rotated matrices and collect the coefficients of σ_2 and σ_3. So the rotated operator has the same form, $j[-i\sigma_1\frac{d}{dy} + \phi' + m_2\sigma_2 + k_2\sigma_3] + v$, with the new mass m_2 and momentum term k_2. In the matrix, $j(\phi' + k_2) + v$ stands on the diagonal of the u rows and $j(\phi' - k_2) + v$ on the diagonal of the w rows (σ_3 is $+1$ on the first component and -1 on the second), and M in the neighbour entries is replaced by m_2, taken at the half node of the pair so that the matrix stays symmetric. For even parity $\phi = 0$, and everything reduces to the matrix above, with $\pm jK + v$ on the diagonal. The reference chooses the angle $j\phi$ for block type j so that the exact symmetry between the two block types at $k = 0$ survives on the grid: their levels agree exactly, to 0 in the record (`Revision/kohn_sham/reports/ks-reference.json`, check `free_k0_block_type_symmetry`).

16.15 Inside the reference solver III: counting eigenvalues, the zero mode and the brane band

The pivots, line by line. How does one find eigenvalue number i of a matrix with thousands of rows without computing all of them? By counting. Let T be tridiagonal with diagonal $d_1, \ldots, d_n$ and neighbour entries $o_1, \ldots, o_{n-1}$, and let x be a trial number. Write $T - x\mathbb{1}$ (where $\mathbb{1}$ is the unit matrix) as a product $\mathcal{L}D\mathcal{L}^T$, with $\mathcal{L}$ (a script letter, because L is the length 3 of the interval) having ones on its diagonal and one band below it, and D diagonal with entries $q_1, \ldots, q_n$, the **pivots**. For two rows:

$$\begin{pmatrix} d_1 - x & o_1 \\ o_1 & d_2 - x \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ \ell & 1 \end{pmatrix}\begin{pmatrix} q_1 & 0 \\ 0 & q_2 \end{pmatrix}\begin{pmatrix} 1 & \ell \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} q_1 & \ell q_1 \\ \ell q_1 & \ell^2 q_1 + q_2 \end{pmatrix}.$$

Rule: multiply the three matrices (row times column).

$$q_1 = d_1 - x, \qquad \ell = \frac{o_1}{q_1}, \qquad q_2 = d_2 - x - \ell^2 q_1 = d_2 - x - \frac{o_1^2}{q_1}.$$

Rule: compare the entries of the two sides, first the top left, then the off-diagonal, then the bottom right. The same comparison, row after row, gives for every n

$$q_1 = d_1 - x, \qquad q_r = d_r - x - \frac{o_{r-1}^2}{q_{r-1}} \quad (r = 2, \ldots, n).$$

Rule: each new row of $\mathcal{L}D\mathcal{L}^T$ involves only the previous pivot, because T has only one band on each side of the diagonal.

Sylvester's law of inertia and the Sturm count. A theorem of linear algebra that we quote without proof, **Sylvester's law of inertia**, says: if $A = \mathcal{L}D\mathcal{L}^T$ with an invertible $\mathcal{L}$, then A and D have the same number of negative eigenvalues. The eigenvalues of D are its diagonal entries, the pivots, and the eigenvalues of $T - x\mathbb{1}$ are $\varepsilon - x$ for the eigenvalues ε of T. Hence

$$c(x) = (\text{number of negative pivots}) = (\text{number of eigenvalues of } T \text{ below } x).$$

Rule: $\varepsilon - x < 0$ exactly when $\varepsilon < x$. The function $c(x)$ is the **Sturm count**; it jumps by one at every eigenvalue (Figure 16b.2 draws it as a staircase). If a pivot happens to be exactly zero, the next one would divide by zero; the reference then replaces it by the tiny negative number -10^{-290}, which is the same as moving x by a tiny amount. A worked example with three rows is Exercise 4 of Section 16.27.

Bisection. To find eigenvalue number i (counting from 0 at the lowest), start with an interval $[lo, hi]$ that contains every eigenvalue. Take the midpoint mid. If $c(mid) \le i$, at most i eigenvalues lie below mid, so eigenvalue number i lies at or above it: replace lo by mid; otherwise replace hi by mid. Each step halves the interval and keeps the eigenvalue inside, so no eigenvalue can be missed or counted twice. After n steps an interval of width W has shrunk to $W/2^n$; to reach a width τ one needs $n \ge \log_2(W/\tau)$ steps.

The starting interval: Gershgorin's bound, line by line. Let $Tz = \varepsilon z$ with $z \neq 0$, and let r be the row in which $|z_r|$ is largest.

$$(\varepsilon - T_{rr})\, z_r = \sum_{c \neq r} T_{rc}\, z_c.$$

Rule: row r of $Tz = \varepsilon z$, with the diagonal term moved to the left.

$$|\varepsilon - T_{rr}|\, |z_r| \le \sum_{c \neq r} |T_{rc}|\, |z_c| \le \Big(\sum_{c \neq r} |T_{rc}| \Big)\, |z_r|.$$

Rule: the triangle inequality, then $|z_c| \le |z_r|$ for every c.

$$|\varepsilon - T_{rr}| \le \rho_r = \sum_{c \ne r} |T_{rc}|.$$

Rule: divide by $|z_r| > 0$. So every eigenvalue lies within the **radius** ρ_r of some diagonal entry (Gershgorin's theorem), and the interval from $\min(d-\rho) - 1$ to $\max(d+\rho) + 1$ contains them all. For $G = 150$ ($h = 0.02$) the neighbour entries are 50.5 and -49.5, every inner radius is 100, and the interval is $[-101, 101]$, of width $W = 202$. The notebook stops when the width is below $10^{-14}\max(1, |lo|)$, so for a level near zero $\tau = 10^{-14}$ and $n \ge \log_2(202/10^{-14}) = 54.2$: 55 halvings, the number Notebook 16b prints (Out [5]).

The exact discrete zero mode, line by line. At $k = 0$ the grid problem keeps an eigenvalue that is exactly zero. Put $\varepsilon = 0$ and all $w_i = 0$. The rows of the u then read $0 = 0$. The row of w_i demands

$$\Big(\frac{1}{h} + \frac{M}{2}\Big)\, u_{i-1} + \Big(-\frac{1}{h} + \frac{M}{2}\Big)\, u_i = 0 \quad \Longrightarrow \quad u_i = q\, u_{i-1}, \qquad q = \frac{1 + Mh/2}{1 - Mh/2}.$$

Rule: multiply by h and solve for u_i. Hence $u_p = q^p\, u_0$: a discrete exponential, with $b = 0$ exactly, as in the continuum. How close is it to e^{My}?

$$\ln q = \ln\left(1 + \tfrac{Mh}{2}\right) - \ln\left(1 - \tfrac{Mh}{2}\right) = 2\Big(\tfrac{Mh}{2} + \tfrac{1}{3}\big(\tfrac{Mh}{2}\big)^3 + \dots\Big) = Mh + \frac{(Mh)^3}{12} + \dots$$

Rule: the series $\ln(1+t) = t - \frac{t^2}{2} + \frac{t^3}{3} - \dots$ with $t = \pm Mh/2$; in the difference the even powers cancel and the odd powers double. So $q^p = e^{p \ln q} = e^{Mph\,(1 + (Mh)^2/12 + \dots)}$: the exponent is M times the distance ph from the first half node, up to a relative error $(Mh)^2/12$, an error of order h^2. For $G = 150$ the grid values lie within 6.9×10^{-5} of $A\, e^{My}$ (Notebook 16b, Out [7]). In the record the zero mode's eigenvalue is at most 1.3×10^{-39} and the extrapolated profile agrees with $A\, e^{My}$ to 4.0×10^{-15} (`Revision/kohn_sham/reports/ks-reference.json`, check free_zero_mode).

The slope of the brane band: Hellmann-Feynman, line by line. Now switch on a small 3-momentum k. The zero mode moves away from zero and becomes the lowest level of the **brane band**, $\varepsilon \approx c\,k$ for small k (Chapter 15). Its slope $c = d\varepsilon/dk$ at $k = 0$ follows from a general fact about symmetric matrices. Let $T(k)z(k) = \varepsilon(k)z(k)$ with $z^T z = 1$, and write a prime for d/dk.

$$T'z + Tz' = \varepsilon' z + \varepsilon z'.$$

Rule: differentiate both sides with the product rule.

$$z^T T' z + z^T T z' = \varepsilon'\, z^T z + \varepsilon\, z^T z'.$$

Rule: multiply from the left by the row z^T.

$$z^T T' z + \varepsilon\, z^T z' = \varepsilon' + \varepsilon\, z^T z' \quad \Longrightarrow \quad \varepsilon' = z^T\, T'\, z.$$

Rule: $z^T T = (T^T z)^T = (Tz)^T = \varepsilon z^T$ because T is symmetric, and $z^T z = 1$; then cancel $\varepsilon z^T z'$. This is the **Hellmann-Feynman theorem** for matrices: the derivative of a level is the expectation

value of the derivative of the matrix. For even parity and $j = +1$ the momentum enters only the diagonal, $+\kappa(y)k$ in the u rows and $-\kappa(y)k$ in the w rows, so T' is diagonal with $\pm\kappa$. The zero mode has $w = 0$, and its normalised vector holds $u_p\sqrt{h}$, so

$$c = \sum_p \kappa(y_{p+1/2})\, u_p^2\, h \;\longrightarrow\; \int_{-L}^{0} \kappa\, a^2\, dy \quad (h \to 0).$$

Rule: only the u entries contribute; the sum is the midpoint rule for the integral. In the continuum, with $a = A\, e^{My}$ and $\kappa = e^{-Hy-a_{4,0}}$:

$$c = e^{-a_{4,0}} A^2 \int_{-L}^{0} e^{(2M-H)y}\, dy = e^{-a_{4,0}}\, \frac{2M}{1-e^{-2ML}}\, \frac{1-e^{-(2M-H)L}}{2M-H}.$$

Rule: $\kappa a^2 = e^{-a_{4,0}} A^2 e^{-Hy} e^{2My}$; integrate the exponential as for the normalisation; insert A^2 from Section 16.13. The theory file `Revision/kohn_sham/ks-theory.json` holds this formula under the name `checksNumeric.braneBandSlopeFormula`, and the record proves it in the check brane_band_slope of its Python report `Revision/kohn_sham/reports/ks-theory-python.json` (status PROVED). For $M = H = 1$ and $L = 3$ it gives:

$$c(0) = \frac{2\,(1-e^{-3})}{1-e^{-6}} = \frac{2 \cdot 0.950213}{0.997521} = 1.905148.$$

Rule: insert the numbers; $e^{-3} = 0.049787$ and $e^{-6} = 0.002479$.

The meaning of the factor $e^{-a_{4,0}}$. The weight $\kappa = e^{-Hy-a_{4,0}}$ is one over the 3-space scale factor $e^{a_4} \sin^{1/6} z = e^{a_4+Hy}$ (because $\sin z = e^{6Hy}$, so $\sin^{1/6} z = e^{Hy}$). Along the history $a_4 = AHx_4$ the 3-space scale factor grows while the three extra times deflate with e^{-a_4} (the 7-volume stays constant), and the energy carried by a 3-momentum is redshifted by $e^{-a_{4,0}}$: the brane band flattens. From slice to slice ($\Delta a_{4,0} = 0.5$) the slope shrinks by the factor $e^{-0.5} = 0.6065$: 1.905148, 1.155531, 0.700865, 0.425096, 0.257834. The reference's grid reproduces the formula to 2.6×10^{-15}, relative (`Revision/kohn_sham/reports/ks-reference.json`, check `free_brane_band_slope`).

16.16 Example: the reference method by hand (Notebook 16b)

Notebook 16b opens the reference solver and rebuilds its method by hand on the free problem of Section 16.13, where every answer is known: it derives the exact levels and checks them with sympy, draws the staggered grid, builds the matrix and checks it entry by entry against the reference's own function, finds eigenvalues with Sturm counts and bisection, constructs the exact discrete zero mode, treats the odd parity in the rotated frame, measures the convergence on five grids and the effect of one and two Richardson steps, computes the slope of the brane band at the five slices of the history, and finally runs the reference program's own free-field job and checks its six free-field criteria. It needs no Rust, runs in less than a minute, draws seven figures and ends with the line ALL 31 CHECKS PASSED (notebook 16b).

16.17 How to run Notebook 16b

Step 1. What this notebook does and what it needs.

Notebook 16b (The reference method by hand: staggered grid, Sturm counts, Richardson) is the file `Revision/textbook/notebooks/16b_staggered_grid.ipynb` of the repository Dirac_claude. It builds, line by line, the staggered finite-difference matrix that the independent reference solver uses, for the free Kohn-Sham orbital equation at zero 3-momentum, a problem whose levels are known exactly; it finds the levels by Sturm counts and bisection, shows that the grid error shrinks like the square of the cell width and that two Richardson steps remove it, constructs the exact discrete zero mode at the brane, computes the slope of the brane band at the five slices of the deflating history, and checks every number against the committed record of the reference solver, whose free-field checks it also runs again. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 16b_staggered_grid.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 45 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 16b_staggered_grid.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 16b_staggered_grid.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/16b.captions.json`
- `Revision/textbook/figures/16b_1_staggered_grid.png`
- `Revision/textbook/figures/16b_2_sturm_staircase.png`
- `Revision/textbook/figures/16b_3_orbitals.png`
- `Revision/textbook/figures/16b_4_matrix_heat_maps.png`
- `Revision/textbook/figures/16b_5_convergence_ladder.png`
- `Revision/textbook/figures/16b_6_error_ratios.png`
- `Revision/textbook/figures/16b_7_brane_band_slope.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 31 CHECKS PASSED (notebook 16b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/16b_staggered_grid.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 16b_staggered_grid.ipynb
python -m nbconvert --execute --inplace 16b_staggered_grid.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cell that computes the levels on five grids runs for a minute or longer: this is normal on a slow computer: the Sturm count is a loop over up to 4799 matrix rows, repeated about 60 times for every grid. Wait until the label shows a number.
- An AssertionError names a comparison with the record free-checks.json: the reference program or its record was changed. Get the stored versions back and run the notebook again.

```
git checkout -- Revision/kohn_sham
```

16.18 Notebook 16b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/16b_staggered_grid.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Notebook 16a ran the reference solver as a finished program. This notebook opens
it up and rebuilds its method by hand on the one problem whose answers are known
exactly: the **free** orbital equation (no interaction, $\lambda = 0$) with zero
3-momentum ($k = 0$) and constant mass $M = m = 1$. Step by step it

- derives the exact levels of that problem (and checks the derivation with sympy);
- places the two components of an orbital on a **staggered grid** and turns the
  differential equation into a **symmetric tridiagonal matrix**, and checks that
  the matrix is exactly the one the reference program builds;
- finds the eigenvalues by **Sturm counts** and **bisection**, the method of the
  reference, and checks them against a standard eigenvalue routine;
- constructs the **exact zero mode** of the discrete problem by hand;
- treats the odd parity with the reference's **rotated frame**;
- shows on five grids that the error shrinks like $h^2$ and that two
  **Richardson** steps remove it down to rounding;
- computes the slope of the **brane band** at the five slices of the deflating
  history and compares it with the exact formula of `ks-theory.json`;
- runs the reference program's own free-field job again and checks that it
  reproduces the committed record and passes the six free-field checks.

It draws seven figures and runs in less than a minute.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 16b (The reference method by hand: staggered grid, Sturm counts, Richardson) is
the file `Revision/textbook/notebooks/16b_staggered_grid.ipynb` of the repository
Dirac_claude. It builds, line by line, the staggered finite-difference matrix that the
independent reference solver uses, for the free Kohn-Sham orbital equation at zero
3-momentum, a problem whose levels are known exactly; it finds the levels by Sturm counts
and bisection, shows that the grid error shrinks like the square of the cell width and
that two Richardson steps remove it, constructs the exact discrete zero mode at the
brane, computes the slope of the brane band at the five slices of the deflating history,
and checks every number against the committed record of the reference solver, whose
free-field checks it also runs again. It needs a computer with Windows 11, macOS or
Linux, an internet connection for the installation, the program Git, and Python 3.12 or
```

newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 16b_staggered_grid.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 45 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 16b_staggered_grid.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 16b_staggered_grid.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/16b.captions.json‘
- ‘Revision/textbook/figures/16b_1_staggered_grid.png‘
- ‘Revision/textbook/figures/16b_2_sturm_staircase.png‘
- ‘Revision/textbook/figures/16b_3_orbitals.png‘
- ‘Revision/textbook/figures/16b_4_matrix_heat_maps.png‘
- ‘Revision/textbook/figures/16b_5_convergence_ladder.png‘
- ‘Revision/textbook/figures/16b_6_error_ratios.png‘
- ‘Revision/textbook/figures/16b_7_brane_band_slope.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 31 CHECKS PASSED (notebook 16b)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/16b_staggered_grid.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs

  jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
  programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
  Start them through Python itself, in the folder of the notebook: the first command
  below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 16b_staggered_grid.ipynb
      python -m nbconvert --execute --inplace 16b_staggered_grid.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
  loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
  first run after the installation: this is a message, not an error; the run continues
  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- The cell that computes the levels on five grids runs for a minute or longer: this is
  normal on a slow computer: the Sturm count is a loop over up to 4799 matrix rows,
  repeated about 60 times for every grid. Wait until the label shows a number.
- An AssertionError names a comparison with the record free-checks.json: the reference
  program or its record was changed. Get the stored versions back and run the notebook
  again.

      git checkout -- Revision/kohn_sham

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 16b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 16b (The reference method by hand: staggered grid, Sturm counts, Richardson)
# is the file Revision/textbook/notebooks/16b_staggered_grid.ipynb of the repository
# Dirac_claude. It builds, line by line, the staggered finite-difference matrix that the
# independent reference solver uses, for the free Kohn-Sham orbital equation at zero
# 3-momentum, a problem whose levels are known exactly; it finds the levels by Sturm
# counts and bisection, shows that the grid error shrinks like the square of the cell
# width and that two Richardson steps remove it, constructs the exact discrete zero mode
# at the brane, computes the slope of the brane band at the five slices of the deflating
# history, and checks every number against the committed record of the reference solver,
# whose free-field checks it also runs again. It needs a computer with Windows 11, macOS
# or Linux, an internet connection for the installation, the program Git, and Python 3.12
# or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
```

```
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
```

```
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
```

```
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 16b_staggered_grid.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 45
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 16b_staggered_grid.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 16b_staggered_grid.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/16b.captions.json
# - Revision/textbook/figures/16b_1_staggered_grid.png
# - Revision/textbook/figures/16b_2_sturm_staircase.png
# - Revision/textbook/figures/16b_3_orbitals.png
# - Revision/textbook/figures/16b_4_matrix_heat_maps.png
# - Revision/textbook/figures/16b_5_convergence_ladder.png
# - Revision/textbook/figures/16b_6_error_ratios.png
# - Revision/textbook/figures/16b_7_brane_band_slope.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
```

```
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 31 CHECKS PASSED (notebook 16b)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/16b_staggered_grid.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 16b_staggered_grid.ipynb
#     python -m nbconvert --execute --inplace 16b_staggered_grid.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - The cell that computes the levels on five grids runs for a minute or longer: this is
#   normal on a slow computer: the Sturm count is a loop over up to 4799 matrix rows,
```

```
#   repeated about 60 times for every grid. Wait until the label shows a number.
# - An AssertionError names a comparison with the record free-checks.json: the reference
#   program or its record was changed. Get the stored versions back and run the notebook
#   again.
#     git checkout -- Revision/kohn_sham
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "16b"  # this notebook: chapter 16, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
```

```
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
```

```
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 16b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Orbital**: a one-particle wave function; here a pair of real functions $a(y)$,
  $b(y)$ of the hidden coordinate $y$ in $-L \le y \le 0$ ($L = 3$).
- **Level**: an allowed energy $\varepsilon$ of the orbital equation.
- **Exact** (or analytic) **value**: a number obtained by algebra, without a grid.
- **Grid**: the points that divide $-L \le y \le 0$ into $G$ equal **cells** of
  width $h = L/G$. **Nodes** are the cell ends $y_i = -L + i\,h$
  ($i = 0, \dots, G$), **half nodes** the cell centres
  $y_{p+1/2} = -L + (p + \tfrac12)\,h$ ($p = 0, \dots, G - 1$).
- **Staggered grid**: $a$ is stored at the half nodes, $b$ at the nodes.
- **Matrix** $T$: a square table of numbers; **symmetric**: $T_{rc} = T_{cr}$;
  **tridiagonal**: only the main diagonal and its two neighbours are not zero.
- **Eigenvalue** and **eigenvector**: a number $\varepsilon$ and a column $z \ne 0$
  with $T z = \varepsilon z$. A symmetric matrix of size $n$ has $n$ real
  eigenvalues.
- **Sturm count** $c(x)$: the number of eigenvalues of $T$ below a number $x$.
- **Bisection**: halving an interval that contains the wanted value until it is
  small enough.
- **Order of convergence**: the power $q$ in an error $\approx C h^q$; the reference
  has $q = 2$.
- **Richardson extrapolation**: a combination of results on several grids in which
  the leading errors cancel.
- **Machine epsilon** $\epsilon_{mach} = 2^{-52} \approx 2.2\times10^{-16}$: the
  relative size of the rounding error of one operation in the computer's numbers.
- **Parity**: the condition at the brane $y = 0$: **even** $b(0) = 0$, **odd**
  $a(0) = 0$ (the ASSUMED $Z_2$ brane). **Block type** $j = \pm 1$: the two kinds of
```

```
  $2\times 2$ blocks.
- **Zero mode**: a level with $\varepsilon = 0$ exactly.
- **Brane band**: the lowest even levels at small 3-momentum $k$, whose orbitals sit
  at the brane; its **slope** is $d\varepsilon/dk$ at $k = 0$.
```

Text cell:

```
## 4. The physical and mathematical situation

The Kohn-Sham orbitals of dirac16complex in the author's primordial field depend on
the hidden coordinate $y = \ln(\sin z)/(6H)$ (with $z = 6Hx_0$; $y = 0$ is the brane,
$y = -L$ the tip). With $\chi = (a, i\,b)$ the block equation of `ks-theory.json`
(entry `blockEquation.realForm`) reads, for the free problem ($v = 0$, constant
$M$), units $H = m = 1$:
$$a' = M a - \left(\kappa k + j\varepsilon\right) b, \qquad
b' = \left(j\varepsilon - \kappa k\right) a - M b,$$
where $' = d/dy$, $\kappa = e^{-Hy - a_{4,0}}$ and $k$ is the size of the 3-momentum.
Here $\kappa$ is one over the 3-space scale factor $e^{a_4}\sin^{1/6} z = e^{a_4 + Hy}$:
along the history $a_4$ grows, 3-space inflates, the three extra times
$x_5, x_6, x_7$ deflate with $e^{-a_4}$, and the 3-momenta are redshifted by
$e^{-a_{4,0}}$. The boundary conditions are the regular tip $b(-L) = 0$ (chosen)
and the ASSUMED $Z_2$ brane: $b(0) = 0$ for even and $a(0) = 0$ for odd orbitals.

For $k = 0$ the momentum term drops out, $\kappa$ disappears, and the levels are known
exactly. That makes this problem the reference solver's test bench: every error of
the grid can be measured against the exact answer. The interacting states of the
canonical matrix (Notebook 16a) use the same matrix with $k \ne 0$ and
self-consistent $M(y)$ and $v(y)$, where no exact answer exists.
```

Text cell:

```
## 5. The exact levels, line by line

**Even parity** ($b(-L) = 0$ and $b(0) = 0$), $k = 0$, $j^2 = 1$:

- The second equation $b' = j\varepsilon a - M b$ gives
  $a = j\,(b' + M b)/\varepsilon$ for $\varepsilon \ne 0$ (multiply by $j$ and use
  $j^2 = 1$).
- Put this $a$ into the first equation $a' = M a - j\varepsilon b$:
  $j(b'' + M b')/\varepsilon = M j (b' + M b)/\varepsilon - j\varepsilon b$.
- Multiply by $\varepsilon/j$: $b'' + M b' = M b' + M^2 b - \varepsilon^2 b$.
- Cancel $M b'$: $b'' = -(\varepsilon^2 - M^2)\,b$.
- With $p^2 = \varepsilon^2 - M^2 > 0$ the solution with $b(-L) = 0$ is
  $b = \sin(p\,(y + L))$ (a sine that starts at zero at the tip).
- $b(0) = 0$ needs $\sin(pL) = 0$, so $p = n\pi/L$ with $n = 1, 2, \dots$, and
  $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$.

**The zero mode** ($\varepsilon = 0$): the equations become $a' = M a$ and
$b' = -M b$, so $a = A\,e^{My}$ and $b = B\,e^{-My}$; $b(-L) = 0$ forces $B = 0$.
So $b = 0$ everywhere and $a = A\,e^{My}$, a mode that grows towards the brane. With
$\int_{-L}^{0} a^2\,dy = 1$ we get $A^2 (1 - e^{-2ML})/(2M) = 1$, so
$A = \sqrt{2M/(1 - e^{-2ML})}$.
```

```
**Odd parity** ($b(-L) = 0$ and $a(0) = 0$): again $b = \sin(p\,(y+L))$, and then
$a = j\,(p\cos(p\,(y+L)) + M \sin(p\,(y+L)))/\varepsilon$. The condition $a(0) = 0$
is $p\cos(pL) + M\sin(pL) = 0$, that is $\tan(pL) = -p/M$, with one root $p_l$ in
each interval $((l + \tfrac12)\pi/L, (l+1)\pi/L)$, $l = 0, 1, 2, \dots$, and
$\varepsilon = \pm\sqrt{M^2 + p_l^2}$. These are the formulas of `ks-theory.json`,
entry `boundaryConditions.exactKOSpectra`.

**Odd parity has no level with $\varepsilon^2 \le M^2$**, so all its levels have
$|\varepsilon| > M$:
- $\varepsilon = 0$: as for the zero mode, $a = A\,e^{My}$ and $b = 0$; then
  $a(0) = A = 0$, so only the trivial solution $a = b = 0$ is left: no zero mode.
- $0 < \varepsilon^2 < M^2$: now $b'' = q^2 b$ with $q^2 = M^2 - \varepsilon^2 > 0$,
  and the solution with $b(-L) = 0$ is $b = \sinh(q\,(y+L))$; then
  $a(0) = j\,(q\cosh(qL) + M\sinh(qL))/\varepsilon$, which is not 0, because both
  terms in the bracket are positive.
- $\varepsilon^2 = M^2$: now $b'' = 0$, so $b = y + L$, and
  $a(0) = j\,(1 + ML)/\varepsilon$, again not 0.

**Ranks.** The levels of a sector are counted from the lowest **particle** level
(rank 0): for even parity that is the zero mode (by the convention of
`ks-theory.json`, the brane zero modes are particles), then $n = 1, 2, \dots$ are
ranks $1, 2, \dots$ and the negative levels are ranks $-1, -2, \dots$; for odd parity
rank $r \ge 0$ is $+\sqrt{M^2 + p_r^2}$ and rank $-r-1$ is its negative.

The next cell checks the two solutions with sympy (it substitutes them into both
equations and simplifies the residuals to zero) and computes the exact levels of
ranks $-3$ to $5$ for $M = 1$, $L = 3$, finding the odd roots by bisection.
```

In [2]:

```
import math  # functions of single numbers (sqrt, sin, pi, ...)
import sys  # the list of folders in which Python looks for modules

import numpy as np  # arrays of numbers
import sympy as sp  # exact symbolic algebra

PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
M_VALUE, L_VALUE = 1.0, 3.0  # the mass M = m = 1 and the tip cutoff L = 3

y, p, eps = sp.symbols("y p epsilon", real=True)  # real symbols
M, L = sp.symbols("M L", positive=True)  # M > 0 and L > 0 (needed by the integral)
for j_value in (1, -1):
    j = sp.Integer(j_value)
    b = sp.sin(p * (y + L))  # the even and odd solution for b
    a = j * (sp.diff(b, y) + M * b) / eps  # a from the second equation
    on_shell = {eps: sp.sqrt(M ** 2 + p ** 2)}  # epsilon^2 = M^2 + p^2
    first = sp.diff(a, y) - (M * a - j * eps * b)  # a' - (M a - j eps b)
    second = sp.diff(b, y) - (j * eps * a - M * b)  # b' - (j eps a - M b)
    residuals = [sp.simplify(r.subs(on_shell)) for r in (first, second)]
    check(residuals == [0, 0],
          f"sympy: b = sin(p(y+L)), a = j(b' + M b)/eps solve both equations (j = "
```

```
          f"{j_value}) when eps^2 = M^2 + p^2")
zero_a = sp.exp(M * y)  # the zero mode a = e^{My}, b = 0
check(sp.simplify(sp.diff(zero_a, y) - M * zero_a) == 0,
      "sympy: a = e^(My), b = 0 solves the equations with eps = 0")
norm = sp.integrate(zero_a ** 2, (y, -L, 0))  # int_{-L}^0 e^{2My} dy
check(sp.simplify(norm - (1 - sp.exp(-2 * M * L)) / (2 * M)) == 0,
      "sympy: int e^(2My) dy over -L..0 equals (1 - e^(-2ML))/(2M)")


def odd_root(l, M=M_VALUE, L=L_VALUE):
    """The root p_l of p cos(pL) + M sin(pL) = 0 in ((l + 1/2) pi/L, (l + 1) pi/L),
    by bisection (200 halvings: far below the rounding of a double)."""
    lo, hi = (l + 0.5) * math.pi / L, (l + 1) * math.pi / L
    f = lambda q: M * math.sin(q * L) + q * math.cos(q * L)
    f_lo = f(lo)
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        if (f(mid) > 0) == (f_lo > 0):  # same sign as at lo: the root is above mid
            lo, f_lo = mid, f(mid)
        else:
            hi = mid
    return 0.5 * (lo + hi)


RANKS = np.arange(-3, 6)  # the ranks -3, ..., 5 of the record
P_ODD = [odd_root(l) for l in range(12)]
EXACT_EVEN = np.array([0.0 if r == 0 else
                       math.copysign(math.sqrt(M_VALUE ** 2
                                               + (abs(r) * math.pi / L_VALUE) ** 2),
                                     r) for r in RANKS])
EXACT_ODD = np.array([math.sqrt(M_VALUE ** 2 + P_ODD[r] ** 2) if r >= 0 else
                      -math.sqrt(M_VALUE ** 2 + P_ODD[-r - 1] ** 2) for r in RANKS])
print("rank    even level      odd level")
for r, e, o in zip(RANKS, EXACT_EVEN, EXACT_ODD):
    print(f"{r:4d}  {e:+.12f}  {o:+.12f}")
check(all(abs(math.tan(q * L_VALUE) + q / M_VALUE) < 1e-9 for q in P_ODD),
      "the twelve odd roots satisfy tan(pL) = -p/M")
```

Out [2]:

```
PASS sympy: b = sin(p(y+L)), a = j(b' + M b)/eps solve both equations (j = 1) when eps^2
    = M^2 + p^2
PASS sympy: b = sin(p(y+L)), a = j(b' + M b)/eps solve both equations (j = -1) when eps^2
    = M^2 + p^2
PASS sympy: a = e^(My), b = 0 solves the equations with eps = 0
PASS sympy: int e^(2My) dy over -L..0 equals (1 - e^(-2ML))/(2M)
rank    even level      odd level
  -3  -3.296908309476  -2.911935875354
  -2  -2.320881480155  -2.010628560046
  -1  -1.447971930402  -1.292292828069
   0  +0.000000000000  +1.292292828069
   1  +1.447971930402  +2.010628560046
   2  +2.320881480155  +2.911935875354
```

```
   3  +3.296908309476  +3.882990032622
   4  +4.306502453234  +4.884575630821
   5  +5.330625458687  +5.901680085768
PASS the twelve odd roots satisfy tan(pL) = -p/M
```

Text cell:

```
## 6. The staggered grid

The reference stores $a$ at the $G$ half nodes and $b$ at the $G - 1$ inner nodes;
$b$ at the two end nodes is not stored, it is zero: $b(-L) = 0$ (tip) and, for even
parity, $b(0) = 0$ (brane). Write $u_p \approx a(y_{p+1/2})$ and
$w_i \approx b(y_i)$, so $w_0 = w_G = 0$. The unknowns are listed in the order of
$y$: $(u_0, w_1, u_1, w_2, \dots, w_{G-1}, u_{G-1})$, $2G - 1$ numbers in all. The
next cell draws this grid for $G = 6$.
```

In [3]:

```
G_SHOW = 6
h_show = L_VALUE / G_SHOW
nodes = -L_VALUE + h_show * np.arange(G_SHOW + 1)  # y_0, ..., y_G
halves = -L_VALUE + h_show * (np.arange(G_SHOW) + 0.5)  # y_{1/2}, ..., y_{G-1/2}
fig, ax = plt.subplots(figsize=(9.0, 3.0))
ax.axhline(0.0, color="k", lw=1.0)
ax.plot(halves, np.zeros(G_SHOW), "o", color=PALETTE[0], ms=10,
        label="$u_p \\approx a(y_{p+1/2})$ (half nodes)")
ax.plot(nodes[1:-1], np.zeros(G_SHOW - 1), "s", color=PALETTE[1], ms=9,
        label="$w_i \\approx b(y_i)$ (inner nodes)")
ax.plot(nodes[[0, -1]], [0.0, 0.0], "s", color=PALETTE[1], ms=9, mfc="white",
        label="$w_0 = w_G = 0$ (tip and brane, even parity)")
for p_index, yp in enumerate(halves):
    ax.annotate(f"$u_{p_index}$", (yp, 0.0), (yp, 0.25), ha="center", fontsize=9)
for i_index, yi in enumerate(nodes):
    ax.annotate(f"$w_{i_index}$", (yi, 0.0), (yi, -0.35), ha="center", fontsize=9)
ax.annotate("tip $y = -L$", (nodes[0], 0.0), (nodes[0], 0.55), ha="center")
ax.annotate("brane $y = 0$", (nodes[-1], 0.0), (nodes[-1], 0.55), ha="center")
ax.annotate("", (nodes[1], -0.75), (nodes[0], -0.75),
            arrowprops={"arrowstyle": "<->"})
ax.text(0.5 * (nodes[0] + nodes[1]), -0.95, "$h = L/G$", ha="center")
ax.set_ylim(-1.2, 0.9)
ax.set_xlim(-L_VALUE - 0.3, 0.3)
ax.set_yticks([])
ax.set_xlabel("hidden coordinate $y$")
ax.legend(fontsize=8, loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=3)
save_figure(fig, "staggered_grid",
            "The staggered grid of the reference solver for $G = 6$ cells of width "
            "$h = L/G = 0.5$ on the hidden coordinate $-3 \\le y \\le 0$: the first "
            "orbital component $a$ is stored at the cell centres (circles, $u_p$), "
            "the second component $b$ at the inner cell ends (squares, $w_i$); at the "
            "tip and at the brane $b$ is zero (open squares) and is not an unknown. "
            "The $2G - 1 = 11$ unknowns alternate $u_0, w_1, u_1, \\dots, u_5$.")
```

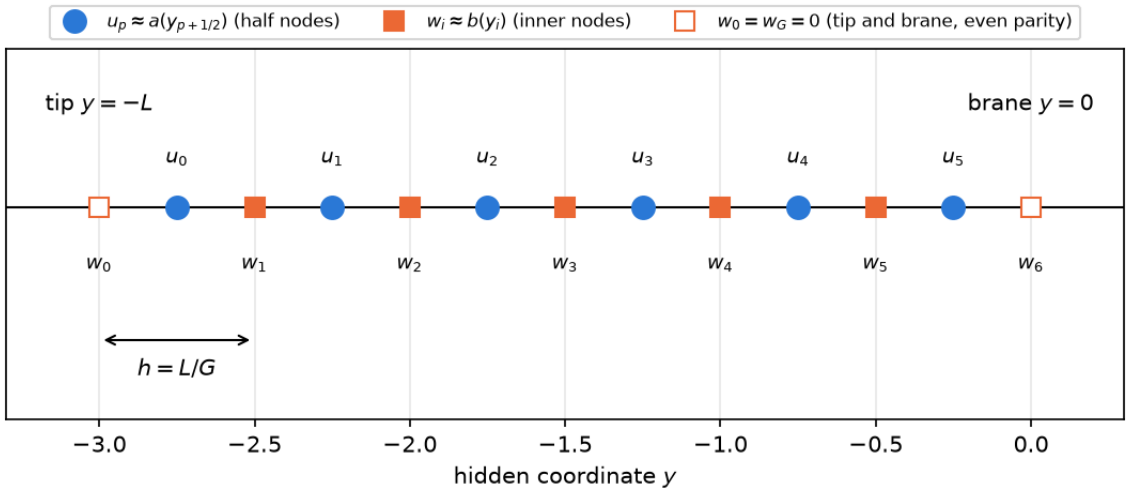

Figure 446: The staggered grid of the reference solver for $G = 6$ cells of width $h = L/G = 0.5$ on the hidden coordinate $-3 \le y \le 0$: the first orbital component a is stored at the cell centres (circles, u_p), the second component b at the inner cell ends (squares, w_i); at the tip and at the brane b is zero (open squares) and is not an unknown. The $2G - 1 = 11$ unknowns alternate $u_0, w_1, u_1, \ldots, u_5$. (Notebook 16b, figure 1.)

Out [3]:

```
Figure 16b.1 saved as Revision/textbook/figures/16b_1_staggered_grid.png
```

Text cell:

```
## 7. From the differential equation to a matrix

The equations of section 4 at $k = 0$, written as an eigenvalue problem
$\varepsilon\,(a, b) = h\,(a, b)$, are (multiply the second by $j$ and the first by
$-j$, using $j^2 = 1$)
$$\varepsilon\,a = j\,(b' + M b), \qquad \varepsilon\,b = j\,(-a' + M a).$$
**Discretise** each at the place where its left side is stored, with centred
differences (the value halfway between two neighbours, and the difference of the two
neighbours divided by their distance $h$):

- at the half node $y_{p+1/2}$ (row of $u_p$): $b' \approx (w_{p+1} - w_p)/h$ and
  $b \approx (w_p + w_{p+1})/2$, so
  $\varepsilon\,u_p = j(-\tfrac1h + \tfrac M2)\,w_p + j(\tfrac1h + \tfrac M2)\,w_{p+1}$;
- at the node $y_i$ (row of $w_i$): $a' \approx (u_i - u_{i-1})/h$ and
  $a \approx (u_{i-1} + u_i)/2$, so
  $\varepsilon\,w_i = j(\tfrac1h + \tfrac M2)\,u_{i-1} + j(-\tfrac1h + \tfrac M2)\,u_i$.

In the order $(u_0, w_1, u_1, \dots)$ every row couples only to its two neighbours,
so the matrix $T$ is **tridiagonal**: the diagonal is zero (no $\varepsilon$-free
term at $k = 0$), and the entry between neighbours number $r$ and $r + 1$ is
$j(\tfrac1h + \tfrac M2)$ when $r$ is even (a $u$ followed by a $w$) and
$j(-\tfrac1h + \tfrac M2)$ when $r$ is odd (a $w$ followed by a $u$). The coefficient
```

```
of $w_{p+1}$ in the row of $u_p$ equals the coefficient of $u_p$ in the row of
$w_{p+1}$, and likewise for $w_p$: $T$ is **symmetric**, so its eigenvalues are real,
as the levels of a self-adjoint problem must be. Centred differences and averages
have errors of order $h^2$ with only even powers of $h$, which is what Richardson
extrapolation needs.

The next cell builds the diagonal and the off-diagonal of $T$ for even parity, prints
the whole matrix for $G = 4$, and compares the arrays for $G = 4$ and $G = 300$ with
those of the reference's own function `sector_arrays` in `ks_fd.py` (for $\lambda = 0$,
$k = 0$, even parity, $j = +1$).
```

In [4]:

```
sys.dont_write_bytecode = True  # do not write a __pycache__ folder into the record
sys.path.insert(0, str(repository_file("Revision/kohn_sham/reference")))
import ks_fd as K  # noqa: E402  the reference solver's module (Revision code)


def even_matrix(G, j=1.0, M=M_VALUE, L=L_VALUE):
    """Diagonal d (2G - 1 numbers) and off-diagonal o (2G - 2 numbers) of T for
    even parity, k = 0, constant M."""
    h = L / G
    d = np.zeros(2 * G - 1)  # the diagonal is zero at k = 0
    r = np.arange(2 * G - 2)  # the number of the first of two neighbours
    o = np.where(r % 2 == 0, j * (1.0 / h + M / 2.0), j * (-1.0 / h + M / 2.0))
    return d, o


d4, o4 = even_matrix(4)
T4 = np.diag(d4) + np.diag(o4, 1) + np.diag(o4, -1)  # the full 7 x 7 matrix
print("T for G = 4 (h = 0.75), even parity, j = +1:")
for row in T4:
    print("  " + " ".join(f"{x:+7.3f}" for x in row))
for G in (4, 300):
    d, o = even_matrix(G)
    phys = K.Phys(m=M_VALUE, L=L_VALUE, H=1.0)  # the reference's parameters
    grid = K.Grid(phys, G)
    sector = K.Sectors([(0, 1, 1, 0)])  # (n2 = 0, r3 = 1, j = +1, even)
    alpha, beta = K.sector_arrays(grid, phys, sector, np.full(grid.n, M_VALUE),
                                  np.zeros(grid.n))
    check(np.array_equal(alpha[:, 0], d) and np.array_equal(beta[:, 0], o),
          f"G = {G}: our matrix equals the reference's (ks_fd.sector_arrays), "
          "entry by entry")
check(np.array_equal(T4, T4.T), "the matrix is symmetric")
```

Out [4]:

```
T for G = 4 (h = 0.75), even parity, j = +1:
   +0.000  +1.833  +0.000  +0.000  +0.000  +0.000  +0.000
   +1.833  +0.000  -0.833  +0.000  +0.000  +0.000  +0.000
   +0.000  -0.833  +0.000  +1.833  +0.000  +0.000  +0.000
   +0.000  +0.000  +1.833  +0.000  -0.833  +0.000  +0.000
```

```
   +0.000  +0.000  +0.000  -0.833  +0.000  +1.833  +0.000
   +0.000  +0.000  +0.000  +0.000  +1.833  +0.000  -0.833
   +0.000  +0.000  +0.000  +0.000  +0.000  -0.833  +0.000
PASS G = 4: our matrix equals the reference's (ks_fd.sector_arrays), entry by entry
PASS G = 300: our matrix equals the reference's (ks_fd.sector_arrays), entry by entry
PASS the matrix is symmetric
```

Text cell:

```
## 8. Eigenvalues by counting: the Sturm count and bisection

How does one find eigenvalue number $i$ of a matrix with thousands of rows without
computing all of them? The reference counts. For a number $x$ form the **pivots**
$$q_1 = d_1 - x, \qquad q_r = d_r - x - \frac{o_{r-1}^2}{q_{r-1}}
\quad (r = 2, \dots, n).$$
They are the numbers that appear on the diagonal when $T - x\,\mathbb{1}$ is written
as $\mathcal{L} D \mathcal{L}^T$ ($\mathcal{L}$ with ones on the diagonal, $D$
diagonal; a script letter, because $L$ is the length 3), by elimination row after
row. **Sylvester's law of inertia** (a theorem of linear algebra that we quote, not
prove) says that $T - x\,\mathbb{1}$ and $D$ have the same number of negative
eigenvalues; the eigenvalues of $T - x\,\mathbb{1}$ are $\varepsilon - x$; so the
number of negative pivots is the number $c(x)$ of eigenvalues of $T$ below $x$. A
pivot that is exactly zero is replaced by a tiny negative number (as in `ks_fd`).

**Bisection** then finds eigenvalue number $i$ (counting from 0): start with an
interval $[lo, hi]$ with $c(lo) \le i < c(hi)$ (the **Gershgorin** bounds
$\min(d - |o_{left}| - |o_{right}|) - 1$ and $\max(d + \dots) + 1$ contain every
eigenvalue); take the midpoint; if $c(mid) \le i$ the eigenvalue is above $mid$,
otherwise below; keep the half that contains it; repeat until the interval is
shorter than $10^{-14}\max(1, |lo|)$. No eigenvalue can be missed or counted twice.

The next cell writes both as functions, works on many matrices or many $x$ at once
(numpy arrays), and compares the result for $G = 150$ with `numpy.linalg.eigvalsh`,
a standard routine that computes all eigenvalues of a symmetric matrix.
```

In [5]:

```
def sturm_count(d, o, x):
    """Number of eigenvalues below x of the tridiagonal matrix (d, o), for every
    entry of the array x."""
    x = np.asarray(x, dtype=float)
    o2 = o * o
    q = d[0] - x  # the first pivot
    q = np.where(np.abs(q) < 1e-290, -1e-290, q)  # never divide by an exact zero
    count = (q < 0).astype(int)
    for r in range(1, len(d)):
        q = d[r] - x - o2[r - 1] / q  # the next pivot
        q = np.where(np.abs(q) < 1e-290, -1e-290, q)
        count += q < 0
    return count
```

```
def bisection(d, o, index, rel_tol=1e-14):
    """Eigenvalues number index (an array of integers, 0 = the lowest) of (d, o)."""
    index = np.asarray(index)
    radius = np.zeros_like(d)
    radius[:-1] += np.abs(o)
    radius[1:] += np.abs(o)
    lo = np.full(len(index), np.min(d - radius) - 1.0)  # Gershgorin: below all
    hi = np.full(len(index), np.max(d + radius) + 1.0)  # Gershgorin: above all
    steps = 0
    while np.any(hi - lo > rel_tol * np.maximum(1.0, np.abs(lo))):
        mid = 0.5 * (lo + hi)
        above = sturm_count(d, o, mid) <= index  # True: the eigenvalue is above mid
        lo = np.where(above, mid, lo)
        hi = np.where(above, hi, mid)
        steps += 1
    return 0.5 * (lo + hi), steps


d150, o150 = even_matrix(150)
T150 = np.diag(d150) + np.diag(o150, 1) + np.diag(o150, -1)
all_eigs = np.linalg.eigvalsh(T150)  # every eigenvalue, sorted
test_x = np.linspace(-6.0, 6.0, 241) + 0.0123  # 241 test points, none at a level
counts_ok = np.array_equal(sturm_count(d150, o150, test_x),
                           np.searchsorted(all_eigs, test_x))
first_particle = int(sturm_count(d150, o150, np.array([-1e-9]))[0])
found, steps = bisection(d150, o150, first_particle + RANKS)
say(f"G = 150: {len(all_eigs)} eigenvalues; {first_particle} lie below -1e-9, so the "
    f"lowest particle level (rank 0) is eigenvalue number {first_particle}")
say(f"bisection: {steps} halvings; largest difference from eigvalsh "
    f"{np.max(np.abs(found - all_eigs[first_particle + RANKS])):.1e}")
check(counts_ok, "the Sturm count equals the number of eigvalsh eigenvalues below x "
                 "at 241 test points")
check(np.max(np.abs(found - all_eigs[first_particle + RANKS])) < 1e-12,
      "bisection with Sturm counts finds the same nine levels as eigvalsh")
```

Out [5]:

```
G = 150: 299 eigenvalues; 149 lie below -1e-9, so the lowest particle level (rank 0) is
    eigenvalue number 149
bisection: 55 halvings; largest difference from eigvalsh 3.8e-14
PASS the Sturm count equals the number of eigvalsh eigenvalues below x at 241 test points
PASS bisection with Sturm counts finds the same nine levels as eigvalsh
```

Text cell:

```
The next figure shows the Sturm count as a staircase for a coarse grid ($G = 30$, 59
eigenvalues): $c(x)$ jumps by one at every eigenvalue. The exact levels of section 5
are drawn as dashed lines: the low eigenvalues of the grid sit almost exactly on
them, the high ones (whose orbitals oscillate on the scale of a few cells) are
visibly shifted.
```

In [6]:

```
d30, o30 = even_matrix(30)
xs = np.linspace(-4.5, 4.5, 3601)  # a fine set of x values
stairs = sturm_count(d30, o30, xs)
eigs30 = np.linalg.eigvalsh(np.diag(d30) + np.diag(o30, 1) + np.diag(o30, -1))
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.step(xs, stairs, where="post", color=PALETTE[0], lw=1.6,
        label="Sturm count $c(x)$, $G = 30$")
shown = eigs30[np.abs(eigs30) < 4.5]
ax.plot(shown, np.searchsorted(eigs30, shown) + 0.5, "o", color=PALETTE[1], ms=5,
        label="eigenvalues of the matrix")
exact_lines = [0.0] + [s * math.sqrt(1.0 + (n * math.pi / 3.0) ** 2)
                       for n in range(1, 5) for s in (1, -1)]
for k, e in enumerate(sorted(exact_lines)):
    ax.axvline(e, color="k", lw=0.8, ls="--",
               label="exact levels (even parity)" if k == 0 else None)
ax.set_xlabel("$x$ (units of $m$)")
ax.set_ylabel("number of eigenvalues below $x$")
ax.set_title("Counting eigenvalues: the Sturm staircase")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "sturm_staircase",
            "The Sturm count $c(x)$, the number of eigenvalues below $x$ of the "
            "even-parity matrix with $G = 30$ cells ($M = 1$, $L = 3$, $k = 0$), "
            "against $x$ in units of $m$. The count rises by one at each eigenvalue "
            "(circles, from a standard eigenvalue routine); the dashed lines are the "
            "exact levels $0$ and $\\pm\\sqrt{1 + (n\\pi/3)^2}$. Bisection finds "
            "eigenvalue number $i$ by asking only how many eigenvalues lie below a "
            "trial value.")
check(int(sturm_count(d30, o30, np.array([4.5]))[0]
          - sturm_count(d30, o30, np.array([-4.5]))[0]) == len(shown),
      "the staircase rises by one for each eigenvalue in the window")
```

Out [6]:

```
Figure 16b.2 saved as Revision/textbook/figures/16b_2_sturm_staircase.png
PASS the staircase rises by one for each eigenvalue in the window
```

Text cell:

```
## 9. The exact discrete zero mode, and the orbitals

At $k = 0$ the grid problem has an eigenvalue that is **exactly** zero. Put
$\varepsilon = 0$ and $w_i = 0$ for every $i$: the rows of the $u$ are then
satisfied, and the row of $w_i$ demands
$(\tfrac1h + \tfrac M2)\,u_{i-1} + (-\tfrac1h + \tfrac M2)\,u_i = 0$, that is
$$u_i = q\,u_{i-1}, \qquad q = \frac{1 + Mh/2}{1 - Mh/2}.$$
So $u_p = q^p\,u_0$: a discrete exponential. Because
$\ln q = \ln(1 + Mh/2) - \ln(1 - Mh/2) = Mh + (Mh)^3/12 + \dots$, it follows the exact
zero mode $e^{My}$ up to an error of order $h^2$, and $b = 0$ exactly, as in the
continuum. A practical point: computing $q$ first and then $q^p$ multiplies the
rounding error of $q$ (about $10^{-16}$) by $p$, up to $10^{-13}$ for $p = 1200$;
computing $u_p = e^{p \ln q}$ with $\ln q$ from the two logarithms
(`numpy.log1p(x)` $= \ln(1 + x)$, accurate also for small $x$) avoids that.
```

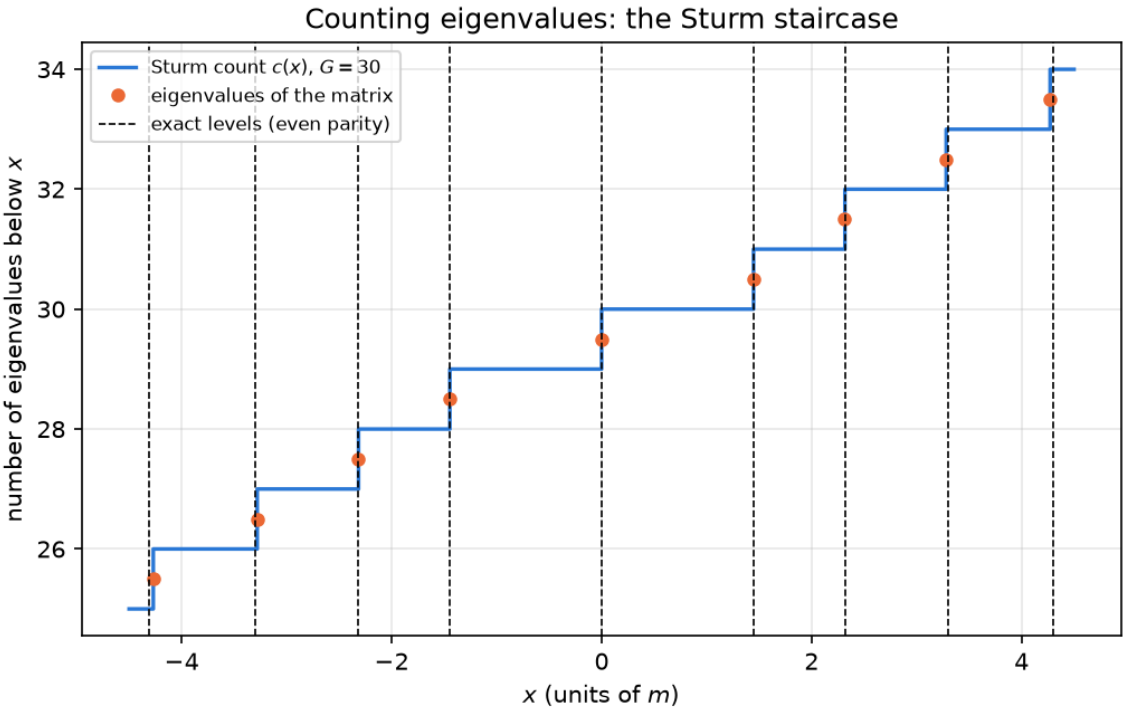

Figure 447: The Sturm count $c(x)$, the number of eigenvalues below x of the even-parity matrix with $G = 30$ cells ($M = 1$, $L = 3$, $k = 0$), against x in units of m. The count rises by one at each eigenvalue (circles, from a standard eigenvalue routine); the dashed lines are the exact levels 0 and $\pm\sqrt{1 + (n\pi/3)^2}$. Bisection finds eigenvalue number i by asking only how many eigenvalues lie below a trial value. (Notebook 16b, figure 2.)

The next cell builds the discrete zero mode for $G = 150$, checks $T z = 0$ to
rounding, and also takes from ‘numpy.linalg.eigh‘ (eigenvalues and eigenvectors) the
orbital of the even level of rank 1, $\varepsilon = \sqrt{1 + (\pi/3)^2}$. An
eigenvector z of length 1 holds $u_p\sqrt h$ and $w_i\sqrt h$, so that
$\sum (u^2 + w^2)\,h = 1$ approximates $\int (a^2 + b^2)\,dy = 1$.

In [7]:

```
def zero_mode(G, M=M_VALUE, L=L_VALUE):
    """The exact discrete zero mode: u_p = exp(p ln q), normalised so that
    sum u^2 h = 1; returns (half nodes y, u)."""
    h = L / G
    log_q = np.log1p(M * h / 2.0) - np.log1p(-M * h / 2.0)  # ln q, accurately
    u = np.exp(np.arange(G) * log_q)
    u /= math.sqrt(np.sum(u * u) * h)
    return -L + (np.arange(G) + 0.5) * h, u


y_half, u0 = zero_mode(150)
z = np.zeros(2 * 150 - 1)
z[0::2] = u0  # the u entries; every w entry stays 0
residual = np.max(np.abs(T150 @ z)) / np.max(np.abs(z))
A = math.sqrt(2.0 * M_VALUE / (1.0 - math.exp(-2.0 * M_VALUE * L_VALUE)))
shape_error = np.max(np.abs(u0 - A * np.exp(M_VALUE * y_half)))
say(f"discrete zero mode, G = 150: |T z| / |z| = {residual:.1e}; largest distance "
    f"from A e^(My) at the half nodes {shape_error:.2e} (A = {A:.6f})")
check(residual < 1e-12, "the discrete zero mode is an exact eigenvector with "
                        "eigenvalue 0 (to rounding), with b = 0")
check(shape_error < 1e-3, "the discrete zero mode follows A e^(My) to order h^2")

values, vectors = np.linalg.eigh(T150)
level = first_particle + 1  # rank 1
vec = vectors[:, level] / math.sqrt(L_VALUE / 150)  # u and w with sum (u^2+w^2) h = 1
vec *= np.sign(vec[-1])  # fix the overall sign: u at the brane positive
u1, w1 = vec[0::2], vec[1::2]
y_node = -L_VALUE + np.arange(1, 150) * (L_VALUE / 150)
p1 = math.pi / L_VALUE
e1 = math.sqrt(M_VALUE ** 2 + p1 ** 2)
b_exact = lambda yy: np.sin(p1 * (yy + L_VALUE))
a_exact = lambda yy: (p1 * np.cos(p1 * (yy + L_VALUE))
                      + M_VALUE * np.sin(p1 * (yy + L_VALUE))) / e1
norm_exact = math.sqrt(L_VALUE)  # int (a^2 + b^2) dy = L for these a and b
scale = np.sign(a_exact(np.array([y_half[-1]]))[0]) / norm_exact
say(f"level of rank 1: eigenvalue {values[level]:.10f}, exact {e1:.10f}")
check(np.max(np.abs(u1 - scale * a_exact(y_half))) < 1e-3
      and np.max(np.abs(w1 - scale * b_exact(y_node))) < 1e-3,
      "the eigenvector of rank 1 follows the exact orbital (a, b) to order h^2")
```

Out [7]:

```
discrete zero mode, G = 150: |T z| / |z| = 1.5e-14; largest distance from A e^(My) at the
    half nodes 6.93e-05 (A = 1.415970)
```

```
PASS the discrete zero mode is an exact eigenvector with eigenvalue 0 (to rounding), with
    b = 0
PASS the discrete zero mode follows A e^(My) to order h^2
level of rank 1: eigenvalue 1.4479202214, exact 1.4479719304
PASS the eigenvector of rank 1 follows the exact orbital (a, b) to order h^2
```

Text cell:

```
The next figure draws both orbitals: left the zero mode $a(y)$ (with $b = 0$), right
the two components of the level of rank 1; the grid values are drawn at every fifth
point, the exact functions as lines.
```

In [8]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
fine = np.linspace(-L_VALUE, 0.0, 400)
left.plot(fine, A * np.exp(M_VALUE * fine), color="k", lw=1.2,
          label="exact $a = A\\,e^{My}$")
left.plot(y_half[::5], u0[::5], "o", color=PALETTE[0], ms=5,
          label="grid $u_p$, $G = 150$")
left.axhline(0.0, color=PALETTE[1], lw=2.0, label="$b = 0$ (exactly)")
left.set_xlabel("$y$")
left.set_ylabel("orbital component")
left.set_title("zero mode, $\\varepsilon = 0$")
left.legend(fontsize=8)
right.plot(fine, scale * a_exact(fine), color="k", lw=1.2, label="exact $a$")
right.plot(fine, scale * b_exact(fine), color="k", lw=1.2, ls="--",
           label="exact $b$")
right.plot(y_half[::5], u1[::5], "o", color=PALETTE[0], ms=5, label="grid $u_p$")
right.plot(y_node[::5], w1[::5], "s", color=PALETTE[1], ms=5, label="grid $w_i$")
right.set_xlabel("$y$")
right.set_title(f"rank 1, $\\varepsilon = {e1:.4f}$")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "orbitals",
            "Orbitals of the free problem at zero 3-momentum ($M = 1$, $L = 3$), "
            "normalised to $\\int (a^2 + b^2)\\,dy = 1$, against the hidden coordinate "
            "$y$ (tip $-3$, brane $0$). Left: the zero mode, $a = A e^{My}$ with "
            "$A = \\sqrt{2M/(1 - e^{-2ML})}$ and $b = 0$, localised at the brane; the "
            "circles are the exact discrete zero mode $u_p = q^p u_0$ of the grid "
            "with $G = 150$. Right: the even level of rank 1, "
            "$\\varepsilon = \\sqrt{1 + (\\pi/3)^2}$: its $b$ vanishes at the tip "
            "and at the brane, and the grid values lie on the exact curves.")
```

Out [8]:

```
Figure 16b.3 saved as Revision/textbook/figures/16b_3_orbitals.png
```

Text cell:

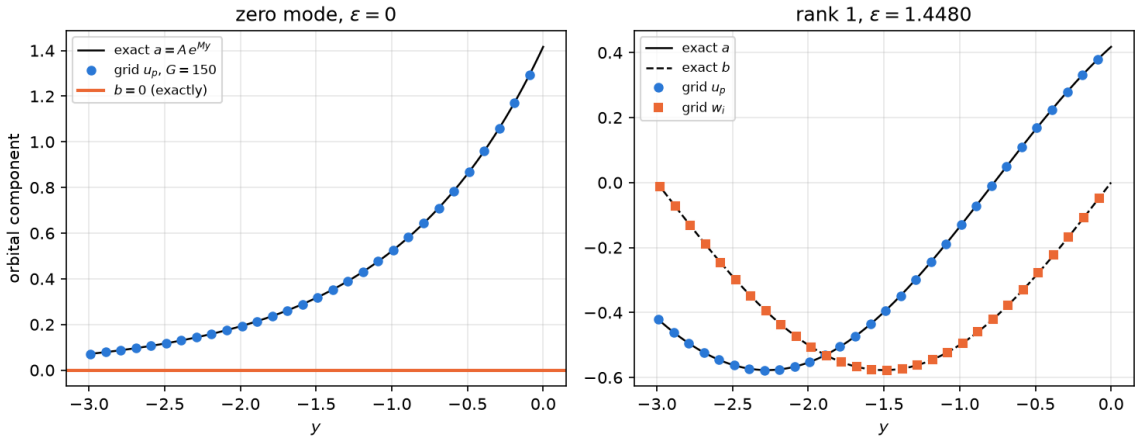

Figure 448: Orbitals of the free problem at zero 3-momentum ($M = 1$, $L = 3$), normalised to $\int (a^2 + b^2)\, dy = 1$, against the hidden coordinate y (tip -3, brane 0). Left: the zero mode, $a = Ae^{My}$ with $A = \sqrt{2M/(1 - e^{-2ML})}$ and $b = 0$, localised at the brane; the circles are the exact discrete zero mode $u_p = q^p u_0$ of the grid with $G = 150$. Right: the even level of rank 1, $\varepsilon = \sqrt{1 + (\pi/3)^2}$: its b vanishes at the tip and at the brane, and the grid values lie on the exact curves. (Notebook 16b, figure 3.)

```
## 10. Odd parity: the rotated frame

For odd parity the brane condition is $a(0) = 0$, but $a$ is stored at half nodes, so
no unknown sits at $y = 0$ to be set to zero. The reference therefore rotates the
two components before discretising. With the Pauli matrices
$\sigma_1, \sigma_2, \sigma_3$ and an angle $\phi(y)$ write
$\chi = e^{i\phi\sigma_1}\psi$, $\psi = (u, i\,w)$. Then:

- $e^{i\phi\sigma_1} = \cos\phi + i\sin\phi\,\sigma_1$ (because $\sigma_1^2 = 1$), so
  $a = \cos\phi\,u - \sin\phi\,w$ and $b = \sin\phi\,u + \cos\phi\,w$.
- Choose $\phi = j\,\frac{\pi}{2}\,\frac{y + L}{L}$: at the tip $\phi = 0$, so
  $b = w$ and $b(-L) = 0$ means $w(-L) = 0$; at the brane $\phi = j\pi/2$, so
  $a = -j\,w$ and $a(0) = 0$ means $w(0) = 0$. Both conditions are again "$w = 0$ at
  the ends", and the same staggered grid works.
- The equation for $\psi$ has the same form, $h' = j[-i\sigma_1\,d/dy + \phi' +
  m_2\sigma_2 + k_2\sigma_3] + v$, with $m_2 = M\cos 2\phi - K\sin 2\phi$ and
  $k_2 = M\sin 2\phi + K\cos 2\phi$ ($K = \kappa k$): the derivative of
  $e^{i\phi\sigma_1}$ gives the extra number $\phi'$, and the rotation turns
  $\sigma_2$ into $\cos 2\phi\,\sigma_2 + \sin 2\phi\,\sigma_3$ and $\sigma_3$ into
  $\cos 2\phi\,\sigma_3 - \sin 2\phi\,\sigma_2$. The next cell checks these two
  identities with sympy.
- In the matrix, $\phi' + k_2$ goes on the diagonal of the $u$ rows,
  $\phi' - k_2$ on the diagonal of the $w$ rows ($\sigma_3 = $ diag$(1, -1)$), both
  times $j$, and $M$ in the off-diagonal entries is replaced by $m_2$ (taken at the
  half node of the pair). For even parity $\phi = 0$ and everything reduces to
  section 7, with $\pm j K$ on the diagonal when $k \ne 0$.

The cell then writes the general function `sector_matrix` (both parities, any $k$
and slice $a_{4,0}$) and checks it against `ks_fd.sector_arrays` for both parities,
both block types, and $k = 0$ and $k = 0.5$.
```

In [9]:

```
phi = sp.symbols("phi", real=True)
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices
s2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
rot = sp.cos(phi) * sp.eye(2) + sp.I * sp.sin(phi) * s1  # e^{i phi sigma1}
back = sp.cos(phi) * sp.eye(2) - sp.I * sp.sin(phi) * s1  # e^{-i phi sigma1}
ok2 = sp.simplify(back * s2 * rot - (sp.cos(2 * phi) * s2 + sp.sin(2 * phi) * s3))
ok3 = sp.simplify(back * s3 * rot - (sp.cos(2 * phi) * s3 - sp.sin(2 * phi) * s2))
check(ok2 == sp.zeros(2, 2) and ok3 == sp.zeros(2, 2),
      "sympy: the rotation turns sigma2 into cos(2 phi) sigma2 + sin(2 phi) sigma3 "
      "and sigma3 into cos(2 phi) sigma3 - sin(2 phi) sigma2")


def sector_matrix(G, odd, j=1.0, k=0.0, a4=0.0, M=M_VALUE, L=L_VALUE, H=1.0):
    """Diagonal and off-diagonal of T for one sector: parity (odd True/False),
    block type j, 3-momentum k, slice a4; constant M, v = 0."""
    h = L / G
    r = np.arange(2 * G - 1)  # the position of each unknown in the list
    y_r = -L + (r + 1) * h / 2.0  # its y (half node for even r, node for odd r)
    is_u = r % 2 == 0
    K_r = np.exp(-H * y_r - a4) * k  # K = kappa k
    angle = j * 0.5 * math.pi * (y_r + L) / L if odd else np.zeros_like(y_r)
    dangle = j * math.pi / (2.0 * L) if odd else 0.0  # phi'
    m2 = M * np.cos(2 * angle) - K_r * np.sin(2 * angle)
    k2 = M * np.sin(2 * angle) + K_r * np.cos(2 * angle)
    d = np.where(is_u, j * (dangle + k2), j * (dangle - k2))
    pair = np.arange(2 * G - 2)  # the pair (r, r + 1)
    u_of_pair = np.where(pair % 2 == 0, pair, pair + 1)  # its half node
    sign = np.where(pair % 2 == 0, 1.0, -1.0)  # +1/h (u then w), -1/h (w then u)
    o = j * (sign / h + 0.5 * m2[u_of_pair])
    return d, o


agree = []
for odd in (False, True):
    for j_value in (1.0, -1.0):
        for n2, r3, k_value in ((0, 1, 0.0), (4, 6, 0.5)):  # |k| = 0.25 sqrt(n2)
            phys = K.Phys(m=M_VALUE, L=L_VALUE, H=1.0)
            grid = K.Grid(phys, 300)
            sector = K.Sectors([(n2, r3, j_value, int(odd))])
            alpha, beta = K.sector_arrays(grid, phys, sector,
                                          np.full(grid.n, M_VALUE), np.zeros(grid.n))
            d, o = sector_matrix(300, odd, j=j_value, k=k_value)
            agree.append(max(np.max(np.abs(alpha[:, 0] - d)),
                             np.max(np.abs(beta[:, 0] - o))))
say(f"largest difference from ks_fd.sector_arrays over 8 sectors: {max(agree):.1e}")
check(max(agree) <= 1e-12,
      "sector_matrix equals the reference's matrices for both parities, both block "
      "types, k = 0 and k = 0.5")
```

Out [9]:

```
PASS sympy: the rotation turns sigma2 into cos(2 phi) sigma2 + sin(2 phi) sigma3 and
    sigma3 into cos(2 phi) sigma3 - sin(2 phi) sigma2
largest difference from ks_fd.sector_arrays over 8 sectors: 0.0e+00
PASS sector_matrix equals the reference's matrices for both parities, both block types, k
    = 0 and k = 0.5
```

Text cell:

```
The next figure shows the two matrices for $G = 8$ (15 unknowns, $j = +1$, $k = 0$)
as heat maps: the colour of square $(r, c)$ is the entry $T_{rc}$. Both are
tridiagonal and symmetric. In the even matrix the diagonal is zero and the
off-diagonal entries alternate between $1/h + M/2$ and $-1/h + M/2$; in the odd
matrix the rotation adds $\phi' \pm M\sin 2\phi$ on the diagonal.
```

In [10]:

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.6))
for ax, odd, title in ((axes[0], False, "even parity"), (axes[1], True, "odd parity")):
    d, o = sector_matrix(8, odd)
    T = np.diag(d) + np.diag(o, 1) + np.diag(o, -1)
    image = ax.imshow(T, cmap="RdBu_r", vmin=-3.5, vmax=3.5)
    ax.set_title(f"$T$, {title}, $G = 8$")
    ax.set_xticks(range(0, 15, 2))  # whole numbers: rows and columns 0 to 14
    ax.set_yticks(range(0, 15, 2))
    ax.set_xlabel("column $c$")
    ax.set_ylabel("row $r$")
fig.colorbar(image, ax=axes, shrink=0.85, label="entry $T_{rc}$ (units of $m$)")
save_figure(fig, "matrix_heat_maps",
            "The reference matrices $T$ for $G = 8$ cells ($h = 0.375$, $M = 1$, "
            "$L = 3$, $j = +1$, $k = 0$) as heat maps, entry $T_{rc}$ in units of $m$ "
            "by colour (blue negative, red positive, white zero). Left, even parity: "
            "only the two neighbouring diagonals are filled, alternating "
            "$1/h + M/2 = 3.17$ and $-1/h + M/2 = -2.17$. Right, odd parity in the "
            "rotated frame: the diagonal carries $\\phi' \\pm M\\sin 2\\phi$ and the "
            "off-diagonal entries are $\\pm 1/h + \\tfrac12 M\\cos 2\\phi$, with "
            "$\\phi$ at the half node of the pair; $M\\cos 2\\phi$ goes from $+M$ at "
            "the tip to $-M$ at the brane, so the red entries fall from 3.16 to "
            "2.25 and the blue ones from -2.25 to -3.16, without a change of sign.")
```

Out [10]:

```
Figure 16b.4 saved as Revision/textbook/figures/16b_4_matrix_heat_maps.png
```

Text cell:

```
## 11. Five grids: second order, and Richardson removes the error

The next cell finds the levels of ranks $-3$ to $5$ of both parities ($j = +1$) on
$G = 150, 300, 600, 1200, 2400$ by Sturm counts and bisection, and their distances
```

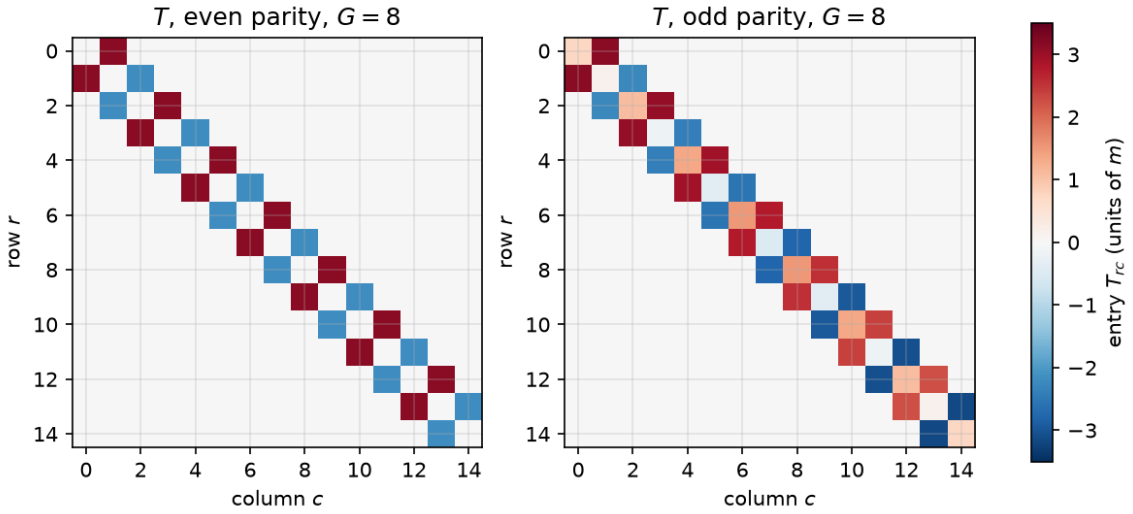

Figure 449: The reference matrices T for $G = 8$ cells ($h = 0.375$, $M = 1$, $L = 3$, $j = +1$, $k = 0$) as heat maps, entry T_{rc} in units of m by colour (blue negative, red positive, white zero). Left, even parity: only the two neighbouring diagonals are filled, alternating $1/h + M/2 = 3.17$ and $-1/h + M/2 = -2.17$. Right, odd parity in the rotated frame: the diagonal carries $\phi' \pm M \sin 2\phi$ and the off-diagonal entries are $\pm 1/h + \frac{1}{2} M \cos 2\phi$, with ϕ at the half node of the pair; $M \cos 2\phi$ goes from $+M$ at the tip to $-M$ at the brane, so the red entries fall from 3.16 to 2.25 and the blue ones from -2.25 to -3.16, without a change of sign. (Notebook 16b, figure 4.)

```
from the exact levels. It then forms, from $G = 300, 600, 1200$, the Richardson value
$R$ and its uncertainty $U$ exactly as the reference does, and compares them, level
by level, with the record ‘free-checks.json‘ (rows with $M = 1$, $L = 3$, $j = +1$).
This cell takes about ten seconds; the largest matrix has 4799 rows.
```

In [11]:

```
GRIDS = (150, 300, 600, 1200, 2400)
EXACT = np.concatenate([EXACT_EVEN, EXACT_ODD])  # 18 levels: even, then odd
LEVELS = {}  # G -> the 18 levels on that grid
for G in GRIDS:
    found = []
    for odd in (False, True):
        d, o = sector_matrix(G, odd)
        lowest_particle = int(sturm_count(d, o, np.array([-1e-9]))[0])
        values_g, _ = bisection(d, o, lowest_particle + RANKS)
        found.append(values_g)
    LEVELS[G] = np.concatenate(found)
    say(f"G = {G:4d}: largest |level - exact| = "
        f"{np.max(np.abs(LEVELS[G] - EXACT)):.3e}")


def richardson(x1, x2, x3):
    """Three-grid Richardson value and uncertainty, as in the reference program."""
    r_fine, r_coarse = (4.0 * x3 - x2) / 3.0, (4.0 * x2 - x1) / 3.0
    R = (16.0 * r_fine - r_coarse) / 15.0
```

```
    return R, np.abs(R - r_fine) + 2e-12 * np.maximum(1.0, np.abs(R))


R18, U18 = richardson(LEVELS[300], LEVELS[600], LEVELS[1200])
record = json.loads(repository_file(
    "Revision/kohn_sham/reference/results/free-checks.json").read_text(encoding="utf-8"))
rows = [r for r in record["analytic"]["rows"]
        if float(r[0]) == 1.0 and float(r[1]) == 3.0 and r[3] == 1]
order = [(par, rank) for par in ("even", "odd") for rank in RANKS]
check([(r[2], r[4]) for r in rows] == order, "the record has our 18 rows in our order")
dev_R = max(abs(float(r[5]) - R18[i]) for i, r in enumerate(rows))
dev_U = max(abs(float(r[6]) - U18[i]) for i, r in enumerate(rows))
dev_1200 = max(abs(float(r[9]) - (LEVELS[1200][i] - EXACT[i]))
               for i, r in enumerate(rows))
say(f"against the record: R differs by at most {dev_R:.1e}, U by {dev_U:.1e}, "
    f"G = 1200 minus exact by {dev_1200:.1e}")
say(f"Richardson value minus exact: at most {np.max(np.abs(R18 - EXACT)):.1e} "
    f"(single grid G = 1200: {np.max(np.abs(LEVELS[1200] - EXACT)):.1e})")
check(dev_R < 1e-12 and dev_U < 1e-12 and dev_1200 < 1e-12,
      "our levels, R and U reproduce the 18 rows of the record",
      record="Revision/kohn_sham/reference/results/free-checks.json, analytic rows")
check(np.max(np.abs(R18 - EXACT)) <= 1e-11,
      "the Richardson values equal the exact levels to 1e-11",
      record="Revision/kohn_sham/reports/ks-reference.json, check "
             "free_k0_analytic_spectra")
```

Out [11]:

```
G =  150: largest |level - exact| = 2.607e-03
G =  300: largest |level - exact| = 6.518e-04
G =  600: largest |level - exact| = 1.629e-04
G = 1200: largest |level - exact| = 4.074e-05
G = 2400: largest |level - exact| = 1.018e-05
PASS the record has our 18 rows in our order
against the record: R differs by at most 8.9e-15, U by 1.3e-15, G = 1200 minus exact by
    6.2e-15
Richardson value minus exact: at most 1.2e-14 (single grid G = 1200: 4.1e-05)
PASS our levels, R and U reproduce the 18 rows of the record
     reproduces Revision/kohn_sham/reference/results/free-checks.json, analytic rows
PASS the Richardson values equal the exact levels to 1e-11
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    free_k0_analytic_spectra
```

Text cell:

```
The next figure is the **Richardson ladder** for three levels: the even level of
rank 1, the odd level of rank 0 and the even level of rank 5 (the highest, whose
orbital oscillates fastest). For each, against the cell width $h$: the error of the
single grids (slope 2), of one Richardson step from two neighbouring grids (slope 4),
and of the three-grid value (two steps), which reaches the rounding floor of about
$10^{-14}$ on the coarsest triple for the two lower levels and from the second
triple on for the level of rank 5.
```

In [12]:

```
hs = np.array([L_VALUE / G for G in GRIDS])
picks = [("even", 1), ("odd", 0), ("even", 5)]
fig, ax = plt.subplots(figsize=(8.0, 6.4))
for colour, (par, rank) in zip(PALETTE, picks):
    i = order.index((par, rank))
    single = np.array([abs(LEVELS[G][i] - EXACT[i]) for G in GRIDS])
    one = np.array([abs((4.0 * LEVELS[GRIDS[g + 1]][i] - LEVELS[GRIDS[g]][i]) / 3.0
                        - EXACT[i]) for g in range(4)])
    two = np.array([abs(richardson(LEVELS[GRIDS[g]][i], LEVELS[GRIDS[g + 1]][i],
                                   LEVELS[GRIDS[g + 2]][i])[0] - EXACT[i])
                    for g in range(3)])
    label = f"{par} rank {rank} ($\\varepsilon = {EXACT[i]:.3f}$)"
    ax.loglog(hs, single, "o-", color=colour, lw=1.4, label=f"{label}: single grid")
    ax.loglog(hs[:4], one, "s--", color=colour, lw=1.2, label="one step")
    ax.loglog(hs[:3], np.maximum(two, 1e-16), "^:", color=colour, lw=1.2,
              label="two steps")
ax.axhline(1e-14, color="k", lw=0.8, ls="-.", label="rounding floor $10^{-14}$")
ax.set_xlabel("cell width $h$ of the coarsest grid used (units of $1/H$)")
ax.set_ylabel("distance from the exact level (units of $m$)")
ax.set_title("The Richardson ladder: slopes 2, 4 and then rounding")
ax.set_ylim(1e-16, 1e-1)
ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
ax.set_xticks(hs)  # tick labels only at the five cell widths
ax.set_xticklabels([f"{t:g}" for t in hs])
ax.legend(fontsize=7, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.13))
save_figure(fig, "convergence_ladder",
            "Distance of the computed level from the exact level, in units of $m$, "
            "against the cell width $h = 3/G$ for $G = 150$ to $2400$, logarithmic "
            "axes, for three levels of the free problem: circles, single grids "
            "(slope 2, error proportional to $h^2$); squares, one Richardson step "
            "$(4x(h/2) - x(h))/3$ (slope 4); triangles, the three-grid value of the "
            "reference (two steps), at the rounding floor near $10^{-14}$ on the "
            "coarsest triple for the two lower levels and from the second triple on "
            "for rank 5. "
            "The high level of rank 5 has the largest errors, because its orbital "
            "varies fastest from cell to cell.")
```

Out [12]:

```
Figure 16b.5 saved as Revision/textbook/figures/16b_5_convergence_ladder.png
```

Text cell:

```
The next cell computes the **convergence ratio**
$(x(G) - x(2G))/(x(2G) - x(4G))$ for every one of the 18 levels and three triples of
grids, and draws it against the level energy. Second order means a ratio of 4; the
reference's check `free_convergence_order_two` requires every ratio of the triple
$(300, 600, 1200)$ (for all three choices of $M$ and $L$) to lie in $[3.99, 4.01]$.
```

In [13]:

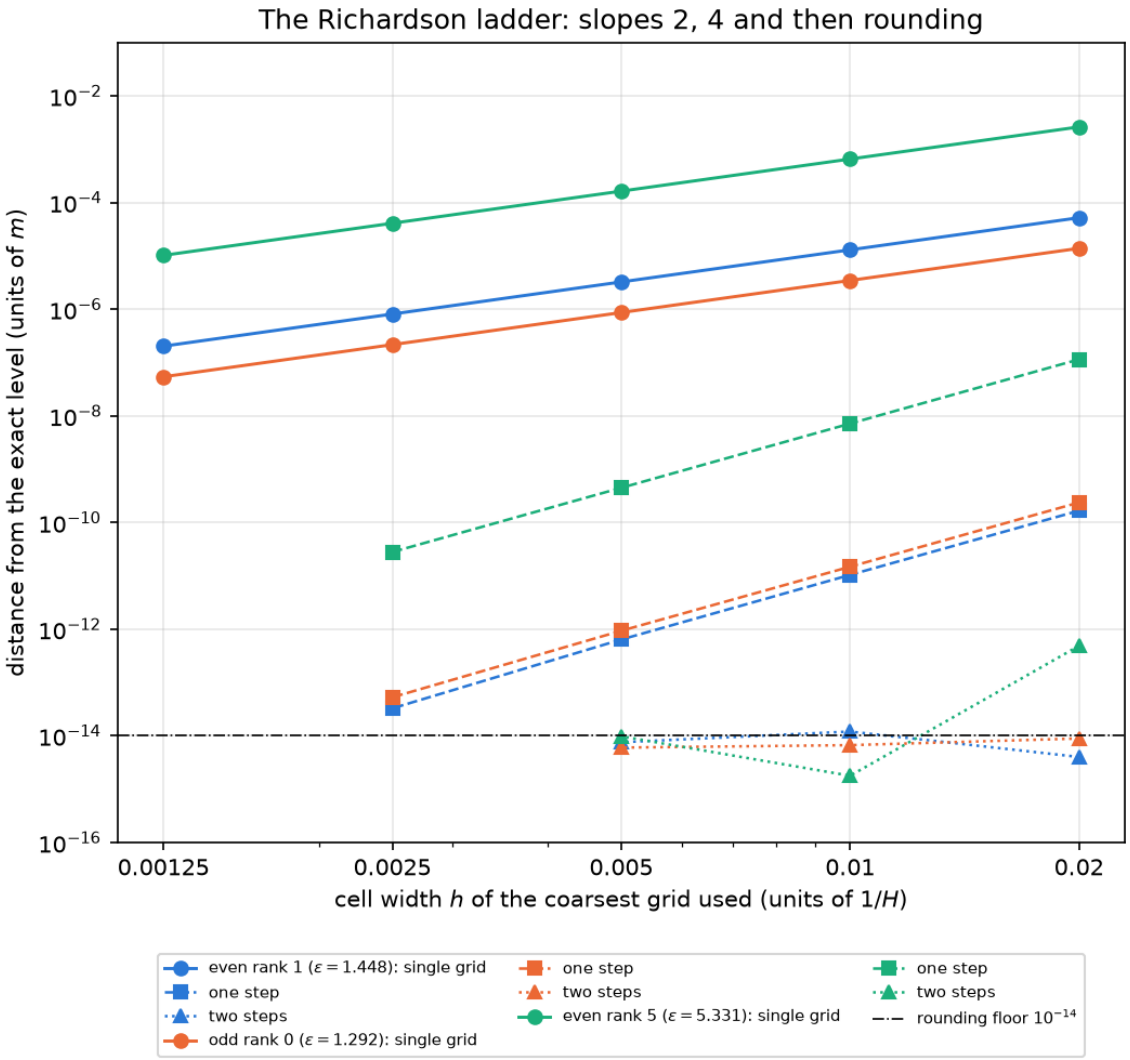

Figure 450: Distance of the computed level from the exact level, in units of m, against the cell width $h = 3/G$ for $G = 150$ to 2400, logarithmic axes, for three levels of the free problem: circles, single grids (slope 2, error proportional to h^2); squares, one Richardson step $(4x(h/2) - x(h))/3$ (slope 4); triangles, the three-grid value of the reference (two steps), at the rounding floor near 10^{-14} on the coarsest triple for the two lower levels and from the second triple on for rank 5. The high level of rank 5 has the largest errors, because its orbital varies fastest from cell to cell. (Notebook 16b, figure 5.)

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
worst = {}
for colour, (g1, g2, g3) in zip(PALETTE, ((150, 300, 600), (300, 600, 1200),
                                          (600, 1200, 2400))):
    d1, d2 = LEVELS[g1] - LEVELS[g2], LEVELS[g2] - LEVELS[g3]
    keep = (np.abs(d1) > 1e-10) & (np.abs(d2) > 1e-10)  # not the zero mode
    ratio = d1[keep] / d2[keep]
    worst[(g1, g2, g3)] = float(np.max(np.abs(ratio - 4.0)))
    ax.plot(EXACT[keep], ratio, "o", color=colour, ms=6,
            label=f"grids {g1}, {g2}, {g3}: largest $|$ratio $- 4|$ = "
                  f"{worst[(g1, g2, g3)]:.1e}")
ax.axhline(4.0, color="k", lw=1.0)
ax.set_xlabel("exact level $\\varepsilon$ (units of $m$)")
ax.set_ylabel("$(x(G) - x(2G))/(x(2G) - x(4G))$")
ax.set_title("Second-order convergence: every ratio tends to 4")
ax.legend(fontsize=8)
save_figure(fig, "error_ratios",
            "The convergence ratio $(x(G) - x(2G))/(x(2G) - x(4G))$ of the 17 nonzero "
            "levels of ranks $-3$ to $5$ (both parities, $M = 1$, $L = 3$, $k = 0$) "
            "against the exact level in units of $m$, for three triples of grids. A "
            "ratio of 4 means an error proportional to $h^2$; the ratios approach 4 "
            "as the grids get finer, fastest for the low levels. The zero mode (even "
            "parity, rank 0) is exact on every grid and has no ratio.")
say("largest |ratio - 4| per triple: " + ", ".join(
    f"{k}: {v:.2e}" for k, v in worst.items()))
check(worst[(300, 600, 1200)] <= 0.01
      and worst[(600, 1200, 2400)] < worst[(300, 600, 1200)]
      and int(np.sum(keep)) == 17,  # 18 levels minus the one zero mode
      "the ratios lie within 0.01 of 4 and approach 4 on finer grids",
      record="Revision/kohn_sham/reports/ks-reference.json, check "
             "free_convergence_order_two")
```

Out [13]:

```
Figure 16b.6 saved as Revision/textbook/figures/16b_6_error_ratios.png
largest |ratio - 4| per triple: (150, 300, 600): 6.57e-04, (300, 600, 1200): 1.64e-04,
    (600, 1200, 2400): 4.10e-05
PASS the ratios lie within 0.01 of 4 and approach 4 on finer grids
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    free_convergence_order_two
```

Text cell:

```
## 12. The brane band along the deflating history

Now switch on a small 3-momentum $k$. The zero mode is no longer exactly at zero: it
becomes the bottom of the **brane band**, $\varepsilon \approx c\,k$ for small $k$.
How large is the slope $c = d\varepsilon/dk$ at $k = 0$?

- **Hellmann-Feynman on the matrix.** If $T(k) z = \varepsilon(k) z$ with
  $z^T z = 1$, differentiating gives $d\varepsilon/dk = z^T (dT/dk) z$ (the terms
  with $dz/dk$ cancel because $T$ is symmetric and $z^T z = 1$). For even parity
```

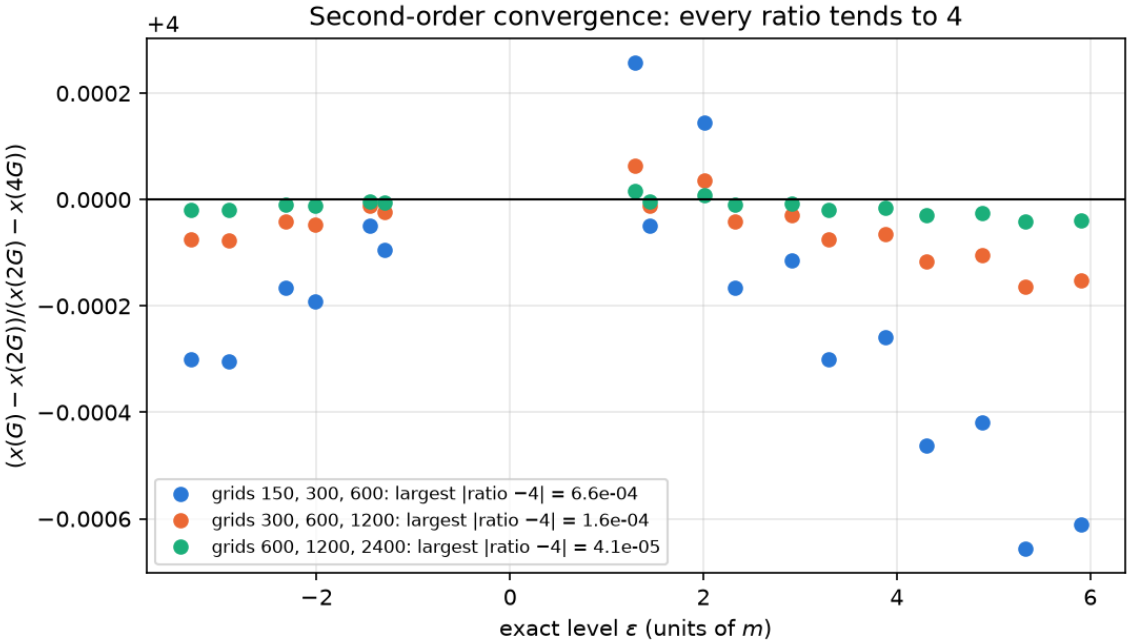

Figure 451: The convergence ratio $(x(G) - x(2G))/(x(2G) - x(4G))$ of the 17 nonzero levels of ranks -3 to 5 (both parities, $M = 1$, $L = 3$, $k = 0$) against the exact level in units of m, for three triples of grids. A ratio of 4 means an error proportional to h^2; the ratios approach 4 as the grids get finer, fastest for the low levels. The zero mode (even parity, rank 0) is exact on every grid and has no ratio. (Notebook 16b, figure 6.)

```
  and $j = +1$, $dT/dk$ is diagonal: $+\kappa(y)$ in the $u$ rows and $-\kappa(y)$ in
  the $w$ rows (from $\pm j K$, $K = \kappa k$). The zero mode has $w = 0$, so
  $c = \sum_p \kappa(y_{p+1/2})\,u_p^2\,h$.
- **In the continuum** this is $c = \int_{-L}^{0} \kappa\,a^2\,dy$ with
  $a = A e^{My}$ and $\kappa = e^{-Hy - a_{4,0}}$:
  $c = e^{-a_{4,0}} A^2 \int_{-L}^{0} e^{(2M - H)y}\,dy
  = e^{-a_{4,0}}\,\frac{2M}{1 - e^{-2ML}}\,\frac{1 - e^{-(2M - H)L}}{2M - H}$,
  the formula ‘checksNumeric.braneBandSlopeFormula‘ of ‘ks-theory.json‘.
- **The meaning.** The factor $e^{-a_{4,0}}$ is the redshift of the 3-momentum: along
  the history $a_4 = A H x_4$, 3-space inflates by $e^{a_4}$ while the three extra
  times deflate by $e^{-a_4}$ (the 7-volume stays constant), and the brane band
  flattens as $e^{-a_{4,0}}$.

The next cell computes $c$ on $G = 300, 600, 1200$ with the discrete zero mode of
section 9, extrapolates with Richardson, and compares with the formula and with the
record (entry ‘brane_band_slope‘ of ‘free-checks.json‘) at the five slices
$a_{4,0} = 0, 0.5, 1, 1.5, 2$.
```

In [14]:

```
SLICES = (0.0, 0.5, 1.0, 1.5, 2.0)


def slope_formula(a4, M=M_VALUE, H=1.0, L=L_VALUE):
    """c = e^(-a4) (2M/(1 - e^(-2ML))) (1 - e^(-(2M - H)L))/(2M - H)."""
    return (math.exp(-a4) * 2.0 * M / (1.0 - math.exp(-2.0 * M * L))
            * (1.0 - math.exp(-(2.0 * M - H) * L)) / (2.0 * M - H))


per_grid = {}
for G in (300, 600, 1200):
    yh, u = zero_mode(G)
    weight = u * u * (L_VALUE / G)  # u_p^2 h, summing to 1
    per_grid[G] = np.array([np.sum(np.exp(-yh - a4) * weight) for a4 in SLICES])
R_slope, U_slope = richardson(per_grid[300], per_grid[600], per_grid[1200])
formula = np.array([slope_formula(a4) for a4 in SLICES])
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json").read_text(
    encoding="utf-8"))
stated = float(theory["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
rec_rows = record["brane_band_slope"]["rows"]
print("a4,0   slope (grid, Richardson)   formula            record")
for i, a4 in enumerate(SLICES):
    print(f"{a4:4.1f}   {R_slope[i]:.15f}      {formula[i]:.15f}  "
          f"{float(rec_rows[i][1]):.15f}")
rel_formula = np.max(np.abs(R_slope - formula) / formula)
rel_record = max(abs(float(r[1]) - R_slope[i]) / formula[i]
                 for i, r in enumerate(rec_rows))
say(f"largest relative difference: from the formula {rel_formula:.1e}, from the "
    f"record {rel_record:.1e}")
print("stated in ks-theory.json (checksNumeric): c at a4,0 = 0 is "
      + theory["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
check(rel_formula <= 1e-11 and abs(slope_formula(0.0) - stated) <= 1e-15 * stated,
      "the grid slope equals c e^(-a4,0) of ks-theory.json at the five slices",
```

```
      record="Revision/kohn_sham/reports/ks-reference.json, check "
             "free_brane_band_slope")
check(rel_record <= 1e-13, "our slopes reproduce the record's brane_band_slope rows",
      record="Revision/kohn_sham/reference/results/free-checks.json, brane_band_slope")
check(np.allclose(R_slope[1:] / R_slope[:-1], math.exp(-0.5), rtol=1e-12, atol=0),
      "from slice to slice the slope shrinks by exactly e^(-0.5)")
```

Out [14]:

```
a4,0   slope (grid, Richardson)   formula            record
 0.0   1.905148253644867      1.905148253644867  1.905148253644862
 0.5   1.155530827133593      1.155530827133592  1.155530827133590
 1.0   0.700864874899623      0.700864874899623  0.700864874899621
 1.5   0.425096034942281      0.425096034942280  0.425096034942279
 2.0   0.257833778514766      0.257833778514766  0.257833778514766
largest relative difference: from the formula 3.8e-16, from the record 2.6e-15
stated in ks-theory.json (checksNumeric): c at a4,0 = 0 is 1.9051482536448664
PASS the grid slope equals c e^(-a4,0) of ks-theory.json at the five slices
     reproduces Revision/kohn_sham/reports/ks-reference.json, check free_brane_band_slope
PASS our slopes reproduce the record's brane_band_slope rows
     reproduces Revision/kohn_sham/reference/results/free-checks.json, brane_band_slope
PASS from slice to slice the slope shrinks by exactly e^(-0.5)
```

Text cell:

```
The next figure draws the slope against the slice: the grid values (after
Richardson) on the exact formula, on a logarithmic vertical axis, where
$e^{-a_{4,0}}$ is a straight line. A second panel shows, for $a_{4,0} = 0$, the
single-grid values approaching the exact slope like $h^2$.
```

In [15]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
a_fine = np.linspace(0.0, 2.0, 100)
left.semilogy(a_fine, [slope_formula(a) for a in a_fine], color="k", lw=1.2,
              label="formula $c\\,e^{-a_{4,0}}$")
left.semilogy(SLICES, R_slope, "o", color=PALETTE[0], ms=8,
              label="grid, three-grid Richardson value")
left.set_xlabel("slice $a_{4,0}$ (3-space inflates, extra times deflate)")
left.set_ylabel("brane-band slope $d\\varepsilon/dk$ at $k = 0$")
left.set_title("The brane band flattens as $e^{-a_{4,0}}$")
left.legend(fontsize=8)
hs3 = np.array([L_VALUE / G for G in (300, 600, 1200)])
right.loglog(hs3, [abs(per_grid[G][0] - formula[0]) for G in (300, 600, 1200)],
             "o-", color=PALETTE[1], lw=1.5, ms=7,
             label="single grid, $a_{4,0} = 0$")
right.loglog(hs3, abs(per_grid[300][0] - formula[0]) / 3.0 * (hs3 / hs3[0]) ** 2,
             "k--", lw=0.9, label="slope 2 (drawn a factor 3 lower)")
right.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
right.set_xticks(hs3)  # tick labels only at the three cell widths
right.set_xticklabels([f"{t:g}" for t in hs3])
```

```
right.set_xlabel("cell width $h$")
right.set_ylabel("$|c(G) - c|$")
right.set_title("grid error of the slope")
right.legend(fontsize=8)
fig.tight_layout()
save_figure(fig, "brane_band_slope",
            "The slope $d\\varepsilon/dk$ at $k = 0$ of the brane band (even parity, "
            "$j = +1$, $M = H = 1$, $L = 3$). Left: against the slice $a_{4,0}$ of the "
            "history, logarithmic vertical axis; the Richardson values of the grid "
            "(circles) lie on the formula of ks-theory.json, $c(0)\\,e^{-a_{4,0}}$ "
            "with $c(0) = 1.90515$: the 3-momentum is redshifted as 3-space inflates "
            "while the three extra times deflate. Right: the single-grid values at "
            "$a_{4,0} = 0$ approach the exact slope with an error proportional to "
            "$h^2$ (parallel to the dashed line of slope 2).")
```

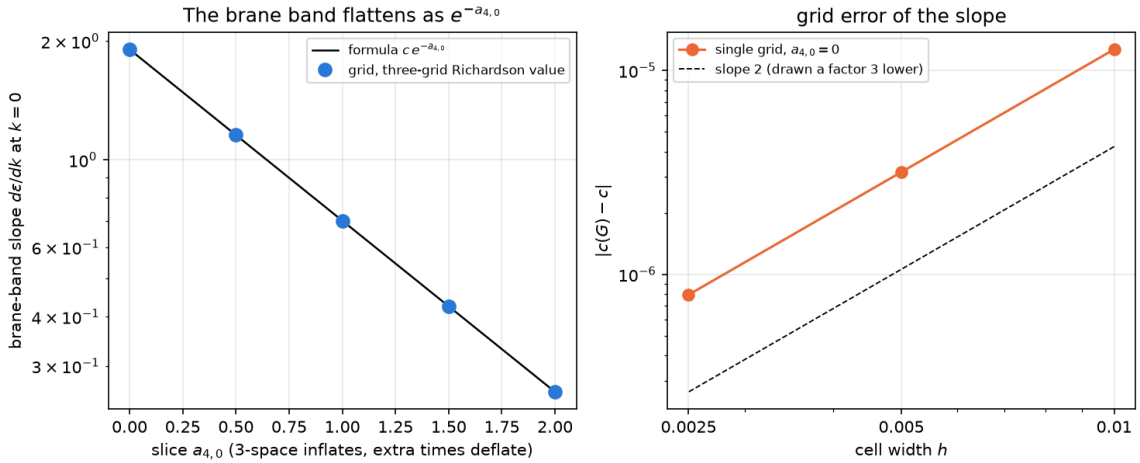

Figure 452: The slope $d\varepsilon/dk$ at $k = 0$ of the brane band (even parity, $j = +1$, $M = H = 1$, $L = 3$). Left: against the slice $a_{4,0}$ of the history, logarithmic vertical axis; the Richardson values of the grid (circles) lie on the formula of ks-theory.json, $c(0)\,e^{-a_{4,0}}$ with $c(0) = 1.90515$: the 3-momentum is redshifted as 3-space inflates while the three extra times deflate. Right: the single-grid values at $a_{4,0} = 0$ approach the exact slope with an error proportional to h^2 (parallel to the dashed line of slope 2). (Notebook 16b, figure 7.)

Out [15]:

```
Figure 16b.7 saved as Revision/textbook/figures/16b_7_brane_band_slope.png
```

Text cell:

```
## 13. The reference program's own free-field job

Finally the next cell runs the reference program's job 'free_checks_job' (about ten
seconds), which computes all of this for three choices of $(M, L)$, both block types
and both parities, the zero mode, the slope and the particle branch; compares its
result with the committed record 'free-checks.json'; and applies the six free-field
criteria of the reference report, with the thresholds of the reference program.
```

In [16]:

```
import run_reference as RR  # noqa: E402  the reference program (Revision code)

CO = RR.theory_coefficients()  # coefficients read and checked from ks-theory.json
free = RR.jsonable(RR.free_checks_job(CO)["data"])
committed = repository_file(
    "Revision/kohn_sham/reference/results/free-checks.json").read_text(encoding="utf-8")
text = json.dumps(free, indent=1, ensure_ascii=True) + "\n"  # as the program writes
say(f"the new free-field result is identical to the record byte for byte: "
    f"{text == committed}")
same = json.loads(text) == json.loads(committed)
if not same:  # another computer may round the last digits differently
    old = json.loads(committed)["analytic"]
    same = all(abs(float(a[5]) - float(b[5])) < 1e-12
               for a, b in zip(free["analytic"]["rows"], old["rows"]))
check(same, "the reference's free-field job reproduces free-checks.json",
      record="Revision/kohn_sham/reference/results/free-checks.json")
an, zm = free["analytic"], free["zero_mode"]
bb, pb = free["brane_band_slope"], free["particle_branch"]
criteria = {
    "free_k0_analytic_spectra": an["max_error_richardson"] <= 1e-11,
    "free_convergence_order_two": 3.99 <= an["ratio_min"] and an["ratio_max"] <= 4.01,
    "free_k0_block_type_symmetry": an["jsym_max"] <= 1e-13
    and zm["jsym_profile"] <= 1e-13,
    "free_zero_mode": zm["max_abs_eps"] <= 1e-13 and zm["max_w_over_u"] <= 1e-12
    and zm["profile_richardson_max_error"] <= 1e-9,
    "free_brane_band_slope": bb["max_rel"] <= 1e-11
    and bb["formula_vs_theory_number"] <= 1e-15,
    "free_particle_branch": pb["violations"] == 0
    and pb["closest_to_zero_away_from_zero_modes"] > 1e-3,
}
say(f"max error of the Richardson values {an['max_error_richardson']:.1e}; ratios "
    f"{an['ratio_min']:.5f} to {an['ratio_max']:.5f}; zero mode |eps| <= "
    f"{zm['max_abs_eps']:.1e}; slope {bb['max_rel']:.1e} relative")
report = json.loads(repository_file(
    "Revision/kohn_sham/reports/ks-reference.json").read_text(encoding="utf-8"))
verdicts = {c["name"]: c["verdict"] for c in report["checks"]}
for name, ok in criteria.items():
    check(ok and verdicts[name] == "PASS", f"{name} holds for the new run",
          record=f"Revision/kohn_sham/reports/ks-reference.json, check {name}")
```

Out [16]:

```
the new free-field result is identical to the record byte for byte: True
PASS the reference's free-field job reproduces free-checks.json
     reproduces Revision/kohn_sham/reference/results/free-checks.json
max error of the Richardson values 2.8e-14; ratios 3.99977 to 4.00019; zero mode |eps| <=
    1.3e-39; slope 2.6e-15 relative
PASS free_k0_analytic_spectra holds for the new run
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    free_k0_analytic_spectra
```

```
PASS free_convergence_order_two holds for the new run
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    free_convergence_order_two
PASS free_k0_block_type_symmetry holds for the new run
     reproduces Revision/kohn_sham/reports/ks-reference.json, check
    free_k0_block_type_symmetry
PASS free_zero_mode holds for the new run
     reproduces Revision/kohn_sham/reports/ks-reference.json, check free_zero_mode
PASS free_brane_band_slope holds for the new run
     reproduces Revision/kohn_sham/reports/ks-reference.json, check free_brane_band_slope
PASS free_particle_branch holds for the new run
     reproduces Revision/kohn_sham/reports/ks-reference.json, check free_particle_branch
```

Text cell:

```
## 14. The last check

The next cell confirms that every figure of this notebook was written, and prints
the number of checks that passed.
```

In [17]:

```
figure_files = [f"{FIGURE_FOLDER}/16b_{k}_{name}.png" for k, name in enumerate(
    ["staggered_grid", "sturm_staircase", "orbitals", "matrix_heat_maps",
     "convergence_ladder", "error_ratios", "brane_band_slope"], start=1)]
check(all(output_file(f).is_file() for f in figure_files),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [17]:

```
PASS every figure file of this notebook exists
ALL 31 CHECKS PASSED (notebook 16b)
```

Text cell:

```
## 15. What this notebook showed

- The free orbital equation at $k = 0$ has exact levels: $0$ and
  $\pm\sqrt{M^2 + (n\pi/L)^2}$ (even), $\pm\sqrt{M^2 + p^2}$ with
  $\tan(pL) = -p/M$ (odd); sympy confirms the solutions.
- On the staggered grid the equation becomes a symmetric tridiagonal matrix, built
  here line by line; it is exactly the reference's matrix (also for odd parity in
  the rotated frame and for $k \ne 0$).
- The Sturm count gives the number of eigenvalues below any $x$, so bisection finds
  every level by its number and none can be missed.
- The discrete problem keeps an exact zero mode, $u_p = q^p u_0$, $b = 0$.
- The grid error is proportional to $h^2$ (ratios within 0.01 of 4); two Richardson
  steps bring all 18 levels to within $10^{-11}$ of the exact values, and the values
```

```
  and uncertainties reproduce the record ‘free-checks.json‘.
- The brane-band slope is $c\,e^{-a_{4,0}}$ with $c = 1.90515$ for $M = H = 1$,
  $L = 3$: the 3-momentum redshifts as 3-space inflates and the extra times deflate.
- The reference program’s own free-field job reproduces its record and passes its
  six free-field checks.
- What this does NOT show: the free problem tests the numerical method. The boundary
  conditions (the ASSUMED $Z_2$ brane, the chosen regular tip) and the model itself
  are inputs, not results, of this test.
```

16.19 Line-by-line walk-through of Notebook 16b

The notebook has 17 code cells, In [1] to In [17]; the numbers they print are in Section 16.18.

In [1], the set-up cell. It is the set-up cell of Notebook 16a, explained line by line in Section 16.12, without the two imports `shutil` and `subprocess` and without the function `rust_program`, because this notebook runs no Rust program. Its comment lines repeat the run instructions of Section 16.17, and its only other difference is the name:

```
NOTEBOOK_ID = "16b"  # this notebook: chapter 16, example b
```

In [2], the exact levels.

```
import math  # functions of single numbers (sqrt, sin, pi, ...)
import sys  # the list of folders in which Python looks for modules

import numpy as np  # arrays of numbers
import sympy as sp  # exact symbolic algebra

PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
M_VALUE, L_VALUE = 1.0, 3.0  # the mass M = m = 1 and the tip cutoff L = 3
```

`math` holds functions of single numbers ($\sqrt{}$, sin, π, ...), `sys` the list of module folders, `numpy` arrays, and `sympy`, called `sp`, exact symbolic algebra: it computes with letters and formulas instead of numbers. `PALETTE` is the list of figure colours of Notebook 16a. The last line gives two names at once: `M_VALUE` is the mass $M = m = 1$ and `L_VALUE` the tip cutoff $L = 3$.

```
y, p, eps = sp.symbols("y p epsilon", real=True)  # real symbols
M, L = sp.symbols("M L", positive=True)  # M > 0 and L > 0 (needed by the integral)
for j_value in (1, -1):
    j = sp.Integer(j_value)
    b = sp.sin(p * (y + L))  # the even and odd solution for b
    a = j * (sp.diff(b, y) + M * b) / eps  # a from the second equation
    on_shell = {eps: sp.sqrt(M ** 2 + p ** 2)}  # epsilon^2 = M^2 + p^2
    first = sp.diff(a, y) - (M * a - j * eps * b)  # a’ - (M a - j eps b)
    second = sp.diff(b, y) - (j * eps * a - M * b)  # b’ - (j eps a - M b)
    residuals = [sp.simplify(r.subs(on_shell)) for r in (first, second)]
    check(residuals == [0, 0],
          f"sympy: b = sin(p(y+L)), a = j(b’ + M b)/eps solve both equations (j = "
          f"{j_value}) when eps^2 = M^2 + p^2")
```

`sp.symbols` creates the symbols y, p, ε (declared real) and M, L (declared positive; the integral below needs $M > 0$). For both block types ($j = 1$ and $j = -1$, made an exact sympy integer by `sp.Integer`) the cell writes the solution of Section 16.13: $b = \sin(p(y + L))$ and $a = j(b' + Mb)/\varepsilon$ (`sp.diff(b, y)` is the derivative b'). `on_shell` is a dictionary that replaces ε by $\sqrt{M^2 + p^2}$. `first` and `second` are the two equations written as "left side minus right side", which must be zero for a solution; `.subs(on_shell)` inserts ε, and `sp.simplify` brings each to its simplest form. The check requires both to be exactly 0: the formulas of Section 16.13 solve both equations whenever $\varepsilon^2 = M^2 + p^2$, for both block types (Out [2], two PASS lines).

```
zero_a = sp.exp(M * y)  # the zero mode a = e^{My}, b = 0
check(sp.simplify(sp.diff(zero_a, y) - M * zero_a) == 0,
      "sympy: a = e^(My), b = 0 solves the equations with eps = 0")
norm = sp.integrate(zero_a ** 2, (y, -L, 0))  # int_{-L}^0 e^{2My} dy
check(sp.simplify(norm - (1 - sp.exp(-2 * M * L)) / (2 * M)) == 0,
      "sympy: int e^(2My) dy over -L..0 equals (1 - e^(-2ML))/(2M)")
```

The zero mode: $a = e^{My}$ must satisfy $a' - Ma = 0$, which sympy confirms, and `sp.integrate` computes $\int_{-L}^{0} e^{2My}\,dy$ symbolically, which must equal $(1 - e^{-2ML})/(2M)$, the normalisation integral of Section 16.13.

```
def odd_root(l, M=M_VALUE, L=L_VALUE):
    """The root p_l of p cos(pL) + M sin(pL) = 0 in ((l + 1/2) pi/L, (l + 1) pi/L),
    by bisection (200 halvings: far below the rounding of a double)."""
    lo, hi = (l + 0.5) * math.pi / L, (l + 1) * math.pi / L
    f = lambda q: M * math.sin(q * L) + q * math.cos(q * L)
    f_lo = f(lo)
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        if (f(mid) > 0) == (f_lo > 0):  # same sign as at lo: the root is above mid
            lo, f_lo = mid, f(mid)
        else:
            hi = mid
    return 0.5 * (lo + hi)
```

`odd_root(l)` finds the root p_l of $F(p) = M\sin(pL) + p\cos(pL)$ in the interval $((l+\frac{1}{2})\pi/L, (l+1)\pi/L)$, which Section 16.13 proved to contain exactly one root, by bisection: `f` is F written as a **lambda** (a one-line function); `f_lo` is its value at the left end; each of the 200 steps takes the midpoint and keeps the half in which F changes sign (if $F(mid)$ has the sign of $F(lo)$, the root lies above mid). Two hundred halvings shrink the interval far below the spacing of the computer's numbers, so the result is the root to the last digit.

```
RANKS = np.arange(-3, 6)  # the ranks -3, ..., 5 of the record
P_ODD = [odd_root(l) for l in range(12)]
EXACT_EVEN = np.array([0.0 if r == 0 else
                       math.copysign(math.sqrt(M_VALUE ** 2
                                               + (abs(r) * math.pi / L_VALUE) ** 2),
                                     r) for r in RANKS])
EXACT_ODD = np.array([math.sqrt(M_VALUE ** 2 + P_ODD[r] ** 2) if r >= 0 else
                      -math.sqrt(M_VALUE ** 2 + P_ODD[-r - 1] ** 2) for r in RANKS])
```

```
print("rank    even level      odd level")
for r, e, o in zip(RANKS, EXACT_EVEN, EXACT_ODD):
    print(f"{r:4d}  {e:+.12f}  {o:+.12f}")
check(all(abs(math.tan(q * L_VALUE) + q / M_VALUE) < 1e-9 for q in P_ODD),
      "the twelve odd roots satisfy tan(pL) = -p/M")
```

`RANKS` is the array of the ranks -3 to 5, `P_ODD` the first twelve odd roots. `EXACT_EVEN` holds the even levels by rank: 0 for rank 0, and $\sqrt{M^2 + (|r|\pi/L)^2}$ with the sign of the rank (`math.copysign(x, r)` gives x the sign of r). `EXACT_ODD` holds the odd levels: $+\sqrt{M^2 + p_r^2}$ for $r \geq 0$ and $-\sqrt{M^2 + p_{-r-1}^2}$ for negative ranks (rank -1 belongs to p_0). The table of Out [2] prints both with twelve decimals and a sign (`:+.12f`); for example the even level of rank 1 is $+1.447971930402$ and the odd level of rank 0 is $+1.292292828069$. The check confirms that every odd root satisfies $\tan(pL) = -p/M$ to 10^{-9}.

In [3], the staggered grid as a picture (Figure 16b.1).

```
G_SHOW = 6
h_show = L_VALUE / G_SHOW
nodes = -L_VALUE + h_show * np.arange(G_SHOW + 1)  # y_0, ..., y_G
halves = -L_VALUE + h_show * (np.arange(G_SHOW) + 0.5)  # y_{1/2}, ..., y_{G-1/2}
fig, ax = plt.subplots(figsize=(9.0, 3.0))
ax.axhline(0.0, color="k", lw=1.0)
ax.plot(halves, np.zeros(G_SHOW), "o", color=PALETTE[0], ms=10,
        label="$u_p \\approx a(y_{p+1/2})$ (half nodes)")
ax.plot(nodes[1:-1], np.zeros(G_SHOW - 1), "s", color=PALETTE[1], ms=9,
        label="$w_i \\approx b(y_i)$ (inner nodes)")
ax.plot(nodes[[0, -1]], [0.0, 0.0], "s", color=PALETTE[1], ms=9, mfc="white",
        label="$w_0 = w_G = 0$ (tip and brane, even parity)")
```

For $G = 6$ cells, `h_show` is the cell width 0.5, `nodes` the seven cell ends $y_0, \ldots, y_6$ and `halves` the six cell centres (`np.arange(n)` is the array $0, 1, \ldots, n-1$, and adding 0.5 moves to the centres). A wide, low figure is made; `axhline` draws the y axis as a horizontal line. The half nodes are drawn as blue circles (the values u_p), the inner nodes as orange squares (`nodes[1:-1]` leaves out the first and the last; the values w_i), and the two end nodes as open squares (`mfc="white"`), because there $b = 0$ is known and is not an unknown. Each label explains its marker.

```
for p_index, yp in enumerate(halves):
    ax.annotate(f"$u_{p_index}$", (yp, 0.0), (yp, 0.25), ha="center", fontsize=9)
for i_index, yi in enumerate(nodes):
    ax.annotate(f"$w_{i_index}$", (yi, 0.0), (yi, -0.35), ha="center", fontsize=9)
ax.annotate("tip $y = -L$", (nodes[0], 0.0), (nodes[0], 0.55), ha="center")
ax.annotate("brane $y = 0$", (nodes[-1], 0.0), (nodes[-1], 0.55), ha="center")
ax.annotate("", (nodes[1], -0.75), (nodes[0], -0.75),
            arrowprops={"arrowstyle": "<->"})
ax.text(0.5 * (nodes[0] + nodes[1]), -0.95, "$h = L/G$", ha="center")
ax.set_ylim(-1.2, 0.9)
ax.set_xlim(-L_VALUE - 0.3, 0.3)
ax.set_yticks([])
ax.set_xlabel("hidden coordinate $y$")
ax.legend(fontsize=8, loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=3)
```

`annotate(text, point, text_position)` writes a text near a point: the names $u_0, \dots, u_5$ above the circles and $w_0, \dots, w_6$ below the nodes, and the words tip and brane above the two ends. An empty annotation with `arrowprops` draws a double arrow over the first cell, labelled $h = L/G$. The axes are set (no ticks on the vertical axis, `set_yticks([])`) and the legend is placed above the plot (`bbox_to_anchor` gives its position relative to the axes, `ncol=3` makes three columns).

```
save_figure(fig, "staggered_grid",
            "The staggered grid of the reference solver for $G = 6$ cells of width "
            "$h = L/G = 0.5$ on the hidden coordinate $-3 \\le y \\le 0$: the first "
            "orbital component $a$ is stored at the cell centres (circles, $u_p$), "
            "the second component $b$ at the inner cell ends (squares, $w_i$); at the "
            "tip and at the brane $b$ is zero (open squares) and is not an unknown. "
            "The $2G - 1 = 11$ unknowns alternate $u_0, w_1, u_1, \\dots, u_5$.")
```

`save_figure` saves Figure 16b.1. What the student should see: along the hidden coordinate from the tip at -3 to the brane at 0, circles and squares alternate; the $2G - 1 = 11$ unknowns are, in order, $u_0, w_1, u_1, \dots, w_5, u_5$; every circle has a square on each side, so every difference of Section 16.14 connects neighbours half a cell apart.

In [4], from the differential equation to a matrix.

```
sys.dont_write_bytecode = True  # do not write a __pycache__ folder into the record
sys.path.insert(0, str(repository_file("Revision/kohn_sham/reference")))
import ks_fd as K  # noqa: E402  the reference solver's module (Revision code)
```

As in Notebook 16a (Section 16.12, In [3]): no translated files are written into the record, the reference folder is added to the module search list, and the reference solver's module `ks_fd.py` is imported under the name `K`.

```
def even_matrix(G, j=1.0, M=M_VALUE, L=L_VALUE):
    """Diagonal d (2G - 1 numbers) and off-diagonal o (2G - 2 numbers) of T for
    even parity, k = 0, constant M."""
    h = L / G
    d = np.zeros(2 * G - 1)  # the diagonal is zero at k = 0
    r = np.arange(2 * G - 2)  # the number of the first of two neighbours
    o = np.where(r % 2 == 0, j * (1.0 / h + M / 2.0), j * (-1.0 / h + M / 2.0))
    return d, o
```

`even_matrix(G)` builds the tridiagonal matrix of Section 16.14 for even parity, $k = 0$ and constant M in its compact form: the diagonal `d` ($2G-1$ zeros, `np.zeros`) and the neighbour entries `o` ($2G-2$ numbers). `r` numbers the pairs of neighbours; `r % 2` is the remainder of r divided by 2, so `r % 2 == 0` is true for even r, and `np.where(condition, x, y)` takes $j(\frac{1}{h} + \frac{M}{2})$ where it is true and $j(-\frac{1}{h} + \frac{M}{2})$ where it is false: exactly the entries $T_{r,r+1}$ derived in Section 16.14.

```
d4, o4 = even_matrix(4)
T4 = np.diag(d4) + np.diag(o4, 1) + np.diag(o4, -1)  # the full 7 x 7 matrix
print("T for G = 4 (h = 0.75), even parity, j = +1:")
for row in T4:
    print("  " + " ".join(f"{x:+7.3f}" for x in row))
```

For $G = 4$ the full matrix is assembled: `np.diag(d4)` puts the diagonal into a square matrix, and `np.diag(o4, 1)` and `np.diag(o4, -1)` put the neighbour entries one place above and one place below it. The loop prints the 7×7 matrix row by row, each entry with sign and three decimals in seven places (`:+7.3f`); Out [4] shows the alternating entries $+1.833$ and -0.833 next to the zero diagonal.

```
for G in (4, 300):
    d, o = even_matrix(G)
    phys = K.Phys(m=M_VALUE, L=L_VALUE, H=1.0)  # the reference's parameters
    grid = K.Grid(phys, G)
    sector = K.Sectors([(0, 1, 1, 0)])  # (n2 = 0, r3 = 1, j = +1, even)
    alpha, beta = K.sector_arrays(grid, phys, sector, np.full(grid.n, M_VALUE),
                                  np.zeros(grid.n))
    check(np.array_equal(alpha[:, 0], d) and np.array_equal(beta[:, 0], o),
          f"G = {G}: our matrix equals the reference's (ks_fd.sector_arrays), "
          "entry by entry")
check(np.array_equal(T4, T4.T), "the matrix is symmetric")
```

For $G = 4$ and $G = 300$ the cell builds the same matrix with the reference's own function: `K.Phys` holds the reference's parameters ($m = 1$, $L = 3$, $H = 1$), `K.Grid` its grid, and `K.Sectors` a list of sectors, here one, ($n_2 = 0, r_3 = 1, j = +1$, even). `K.sector_arrays` returns the diagonals `alpha` and the neighbour entries `beta` of every sector's matrix (one column per sector), for the effective mass $M = 1$ at every position (`np.full(n, value)` is an array of n equal values) and the potential $v = 0$. The check requires our arrays to be **exactly** equal to the reference's (`np.array_equal`), entry by entry; the last check confirms that the 7×7 matrix equals its transpose `T4.T` (rows and columns exchanged), that is, that it is symmetric.

In [5], eigenvalues by counting: the Sturm count and bisection.

```
def sturm_count(d, o, x):
    """Number of eigenvalues below x of the tridiagonal matrix (d, o), for every
    entry of the array x."""
    x = np.asarray(x, dtype=float)
    o2 = o * o
    q = d[0] - x  # the first pivot
    q = np.where(np.abs(q) < 1e-290, -1e-290, q)  # never divide by an exact zero
    count = (q < 0).astype(int)
    for r in range(1, len(d)):
        q = d[r] - x - o2[r - 1] / q  # the next pivot
        q = np.where(np.abs(q) < 1e-290, -1e-290, q)
        count += q < 0
    return count
```

`sturm_count(d, o, x)` is the pivot recursion of Section 16.15 for every entry of the array `x` at once. `np.asarray(x, dtype=float)` makes sure `x` is an array of numbers, and `o2` holds the squares o_r^2. `q` starts as the first pivot $d_1 - x$; `np.where` replaces a pivot whose size is below 10^{-290} by -10^{-290} (never divide by an exact zero); `count` starts as 1 where the pivot is negative and 0 elsewhere (`.astype(int)` turns truth values into the numbers 1 and 0). The loop computes each next pivot $q_r = d_r - x - o_{r-1}^2/q_{r-1}$ and adds 1 to the count wherever it is negative (`count += q < 0` adds the truth values as numbers). The returned count is the number of eigenvalues below each x.

```
def bisection(d, o, index, rel_tol=1e-14):
    """Eigenvalues number index (an array of integers, 0 = the lowest) of (d, o)."""
    index = np.asarray(index)
    radius = np.zeros_like(d)
    radius[:-1] += np.abs(o)
    radius[1:] += np.abs(o)
    lo = np.full(len(index), np.min(d - radius) - 1.0)  # Gershgorin: below all
    hi = np.full(len(index), np.max(d + radius) + 1.0)  # Gershgorin: above all
    steps = 0
    while np.any(hi - lo > rel_tol * np.maximum(1.0, np.abs(lo))):
        mid = 0.5 * (lo + hi)
        above = sturm_count(d, o, mid) <= index  # True: the eigenvalue is above mid
        lo = np.where(above, mid, lo)
        hi = np.where(above, hi, mid)
        steps += 1
    return 0.5 * (lo + hi), steps
```

`bisection(d, o, index)` finds the eigenvalues with the numbers in the array `index` (0 is the lowest), all at once. `radius` is Gershgorin's radius of every row: each row gets the size of the neighbour entry on its right (`radius[:-1] += ...`, all rows but the last) and on its left (`radius[1:] += ...`, all rows but the first). `lo` and `hi` start one unit beyond Gershgorin's bounds, so every eigenvalue lies inside (Section 16.15). The `while` loop runs as long as some interval is wider than $10^{-14} \max(1, |lo|)$ (`np.any` is true if any entry is true). In each step `above` is true where at most `index` eigenvalues lie below the midpoint, that is, where the wanted eigenvalue lies at or above it; there `lo` moves up to the midpoint, elsewhere `hi` moves down. `steps` counts the halvings. The function returns the midpoints of the final intervals and the number of steps.

```
d150, o150 = even_matrix(150)
T150 = np.diag(d150) + np.diag(o150, 1) + np.diag(o150, -1)
all_eigs = np.linalg.eigvalsh(T150)  # every eigenvalue, sorted
test_x = np.linspace(-6.0, 6.0, 241) + 0.0123  # 241 test points, none at a level
counts_ok = np.array_equal(sturm_count(d150, o150, test_x),
                           np.searchsorted(all_eigs, test_x))
first_particle = int(sturm_count(d150, o150, np.array([-1e-9]))[0])
found, steps = bisection(d150, o150, first_particle + RANKS)
```

For $G = 150$ the full matrix `T150` (299×299) is assembled, and `np.linalg.eigvalsh` computes all its eigenvalues with a standard routine for symmetric matrices, sorted. `test_x` holds 241 trial values between -6 and 6, shifted by 0.0123 so that none falls on a level. `counts_ok` compares our Sturm counts with the number of eigenvalues below each trial value found by `np.searchsorted` (it tells where each `x` would be inserted into the sorted list, which is exactly that number). `first_particle` is the Sturm count at -10^{-9}, the number of eigenvalues below zero: the zero mode, the lowest particle level (rank 0), has this number. Bisection then finds the nine levels of ranks -3 to 5.

```
say(f"G = 150: {len(all_eigs)} eigenvalues; {first_particle} lie below -1e-9, so the "
    f"lowest particle level (rank 0) is eigenvalue number {first_particle}")
say(f"bisection: {steps} halvings; largest difference from eigvalsh "
    f"{np.max(np.abs(found - all_eigs[first_particle + RANKS])):.1e}")
check(counts_ok, "the Sturm count equals the number of eigvalsh eigenvalues below x "
                 "at 241 test points")
```

```
check(np.max(np.abs(found - all_eigs[first_particle + RANKS])) < 1e-12,
      "bisection with Sturm counts finds the same nine levels as eigvalsh")
```

Out [5]: 299 eigenvalues, 149 of them below -10^{-9}, so the zero mode is eigenvalue number 149; bisection needed 55 halvings (the number estimated in Section 16.15) and agrees with the standard routine to 3.8×10^{-14}. The two checks require the Sturm counts to agree at all 241 trial values and the nine bisection levels to agree with `eigvalsh` to 10^{-12}.

In [6], the Sturm staircase (Figure 16b.2).

```
d30, o30 = even_matrix(30)
xs = np.linspace(-4.5, 4.5, 3601)  # a fine set of x values
stairs = sturm_count(d30, o30, xs)
eigs30 = np.linalg.eigvalsh(np.diag(d30) + np.diag(o30, 1) + np.diag(o30, -1))
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.step(xs, stairs, where="post", color=PALETTE[0], lw=1.6,
        label="Sturm count $c(x)$, $G = 30$")
shown = eigs30[np.abs(eigs30) < 4.5]
ax.plot(shown, np.searchsorted(eigs30, shown) + 0.5, "o", color=PALETTE[1], ms=5,
        label="eigenvalues of the matrix")
```

For a coarse grid, $G = 30$ (59 eigenvalues), `xs` holds 3601 values of x between -4.5 and 4.5, `stairs` the Sturm count at each, and `eigs30` all eigenvalues. `ax.step(..., where="post")` draws the count as a staircase whose value changes just after each x. `shown` keeps the eigenvalues inside the window (`np.abs(eigs30) < 4.5` is a mask), and each is drawn as a circle half-way up its step (`searchsorted` gives the count below it, plus 0.5).

```
exact_lines = [0.0] + [s * math.sqrt(1.0 + (n * math.pi / 3.0) ** 2)
                       for n in range(1, 5) for s in (1, -1)]
for k, e in enumerate(sorted(exact_lines)):
    ax.axvline(e, color="k", lw=0.8, ls="--",
               label="exact levels (even parity)" if k == 0 else None)
ax.set_xlabel("$x$ (units of $m$)")
ax.set_ylabel("number of eigenvalues below $x$")
ax.set_title("Counting eigenvalues: the Sturm staircase")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "sturm_staircase",
            "The Sturm count $c(x)$, the number of eigenvalues below $x$ of the "
            "even-parity matrix with $G = 30$ cells ($M = 1$, $L = 3$, $k = 0$), "
            "against $x$ in units of $m$. The count rises by one at each eigenvalue "
            "(circles, from a standard eigenvalue routine); the dashed lines are the "
            "exact levels $0$ and $\\pm\\sqrt{1 + (n\\pi/3)^2}$. Bisection finds "
            "eigenvalue number $i$ by asking only how many eigenvalues lie below a "
            "trial value.")
```

`exact_lines` holds the exact even levels of Section 16.13 in the window: 0 and $\pm\sqrt{1 + (n\pi/3)^2}$ for $n = 1$ to 4 (a list comprehension with two `for` parts makes both signs for each n). Each is drawn as a dashed vertical line, and only the first gets a legend label (`label=... if k == 0 else None`). After the axis labels, title and legend, `save_figure` saves Figure 16b.2. What the student should see: the staircase rises by exactly one at each eigenvalue; near zero the eigenvalues of the coarse

matrix sit on the dashed exact levels (the zero mode exactly), while the outermost ones, at about ± 4.3, are visibly shifted from them, because their orbitals oscillate on the scale of a few cells and the grid error is larger.

```
check(int(sturm_count(d30, o30, np.array([4.5]))[0]
          - sturm_count(d30, o30, np.array([-4.5]))[0]) == len(shown),
      "the staircase rises by one for each eigenvalue in the window")
```

The check: the count at 4.5 minus the count at -4.5 equals the number of eigenvalues in the window.

In [7], the exact discrete zero mode and an orbital.

```
def zero_mode(G, M=M_VALUE, L=L_VALUE):
    """The exact discrete zero mode: u_p = exp(p ln q), normalised so that
    sum u^2 h = 1; returns (half nodes y, u)."""
    h = L / G
    log_q = np.log1p(M * h / 2.0) - np.log1p(-M * h / 2.0)  # ln q, accurately
    u = np.exp(np.arange(G) * log_q)
    u /= math.sqrt(np.sum(u * u) * h)
    return -L + (np.arange(G) + 0.5) * h, u
```

`zero_mode(G)` builds the discrete zero mode of Section 16.15 as $u_p = e^{p \ln q}$. `np.log1p(t)` computes $\ln(1+t)$ accurately also for small t, so `log_q` is $\ln(1 + Mh/2) - \ln(1 - Mh/2) = \ln q$ without first forming q: computing q and then q^p would multiply the rounding error of q (about 10^{-16}) by p, up to 10^{-13} for $p = 1200$. The vector is normalised so that $\sum u_p^2 h = 1$ (`u /= x` divides every entry by x), the discrete form of $\int a^2\, dy = 1$. The function returns the half nodes and the values.

```
y_half, u0 = zero_mode(150)
z = np.zeros(2 * 150 - 1)
z[0::2] = u0  # the u entries; every w entry stays 0
residual = np.max(np.abs(T150 @ z)) / np.max(np.abs(z))
A = math.sqrt(2.0 * M_VALUE / (1.0 - math.exp(-2.0 * M_VALUE * L_VALUE)))
shape_error = np.max(np.abs(u0 - A * np.exp(M_VALUE * y_half)))
say(f"discrete zero mode, G = 150: |T z| / |z| = {residual:.1e}; largest distance "
    f"from A e^(My) at the half nodes {shape_error:.2e} (A = {A:.6f})")
check(residual < 1e-12, "the discrete zero mode is an exact eigenvector with "
                        "eigenvalue 0 (to rounding), with b = 0")
check(shape_error < 1e-3, "the discrete zero mode follows A e^(My) to order h^2")
```

For $G = 150$ the zero mode is placed into a vector `z` of length 299: the u entries at the even positions (`z[0::2]` takes every second entry from the first) and zeros at the w positions. `T150 @ z` is the product of the matrix with the vector (`@` is matrix multiplication), and `residual` is its largest entry relative to the largest entry of `z`; it must vanish if z is an eigenvector with eigenvalue 0. `A` is the continuum normalisation of Section 16.13 and `shape_error` the largest distance of the grid values from $A\, e^{My}$ at the half nodes. Out [7]: residual 1.5×10^{-14} (rounding), distance 6.93×10^{-5}, $A = 1.415970$. The two checks require the residual below 10^{-12} and the distance below 10^{-3} (second order: with $h = 0.02$, $h^2 = 4 \times 10^{-4}$).

```
values, vectors = np.linalg.eigh(T150)
level = first_particle + 1  # rank 1
```

```
vec = vectors[:, level] / math.sqrt(L_VALUE / 150)  # u and w with sum (u^2+w^2) h = 1
vec *= np.sign(vec[-1])  # fix the overall sign: u at the brane positive
u1, w1 = vec[0::2], vec[1::2]
y_node = -L_VALUE + np.arange(1, 150) * (L_VALUE / 150)
p1 = math.pi / L_VALUE
e1 = math.sqrt(M_VALUE ** 2 + p1 ** 2)
b_exact = lambda yy: np.sin(p1 * (yy + L_VALUE))
a_exact = lambda yy: (p1 * np.cos(p1 * (yy + L_VALUE))
                      + M_VALUE * np.sin(p1 * (yy + L_VALUE))) / e1
norm_exact = math.sqrt(L_VALUE)  # int (a^2 + b^2) dy = L for these a and b
scale = np.sign(a_exact(np.array([y_half[-1]]))[0]) / norm_exact
```

`np.linalg.eigh` returns all eigenvalues and eigenvectors (one per column) of `T150`. `level` is the number of the level of rank 1, one above the zero mode. Its eigenvector has length 1; dividing by $\sqrt{h}$ turns it into grid values u_p, w_i with $\sum(u^2 + w^2)h = 1$, and multiplying by the sign of its last entry makes u at the brane positive (an eigenvector is fixed only up to its sign). `u1` and `w1` are its u and w entries, `y_node` the 149 inner nodes. The exact orbital of rank 1 is $b = \sin(p_1(y + L))$ and $a = (p_1 \cos(p_1(y + L)) + M \sin(p_1(y + L)))/\varepsilon_1$ with $p_1 = \pi/L$ (Section 16.13, $j = 1$); its integral $\int (a^2 + b^2)\, dy$ equals L, so `scale` divides by $\sqrt{L}$ and gives the exact orbital the sign of the grid's.

```
say(f"level of rank 1: eigenvalue {values[level]:.10f}, exact {e1:.10f}")
check(np.max(np.abs(u1 - scale * a_exact(y_half))) < 1e-3
      and np.max(np.abs(w1 - scale * b_exact(y_node))) < 1e-3,
      "the eigenvector of rank 1 follows the exact orbital (a, b) to order h^2")
```

Out [7] prints the eigenvalue 1.4479202214 next to the exact level 1.4479719304 (the grid error 5.2×10^{-5} is of order h^2), and the check requires the grid values of both components within 10^{-3} of the exact functions.

In [8], the orbitals as a picture (Figure 16b.3).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.0))
fine = np.linspace(-L_VALUE, 0.0, 400)
left.plot(fine, A * np.exp(M_VALUE * fine), color="k", lw=1.2,
          label="exact $a = A\\,e^{My}$")
left.plot(y_half[::5], u0[::5], "o", color=PALETTE[0], ms=5,
          label="grid $u_p$, $G = 150$")
left.axhline(0.0, color=PALETTE[1], lw=2.0, label="$b = 0$ (exactly)")
left.set_xlabel("$y$")
left.set_ylabel("orbital component")
left.set_title("zero mode, $\\varepsilon = 0$")
left.legend(fontsize=8)
```

The left panel draws the exact zero mode $A\, e^{My}$ as a black line on 400 points, the grid values u_p at every fifth half node as circles, and the line $b = 0$ in orange.

```
right.plot(fine, scale * a_exact(fine), color="k", lw=1.2, label="exact $a$")
right.plot(fine, scale * b_exact(fine), color="k", lw=1.2, ls="--",
           label="exact $b$")
right.plot(y_half[::5], u1[::5], "o", color=PALETTE[0], ms=5, label="grid $u_p$")
```

```
right.plot(y_node[::5], w1[::5], "s", color=PALETTE[1], ms=5, label="grid $w_i$")
right.set_xlabel("$y$")
right.set_title(f"rank 1, $\\varepsilon = {e1:.4f}$")
right.legend(fontsize=8)
fig.tight_layout()
```

The right panel draws the exact a (solid) and b (dashed) of rank 1 and the grid values u_p (circles) and w_i (squares), every fifth point; its title shows the level.

```
save_figure(fig, "orbitals",
            "Orbitals of the free problem at zero 3-momentum ($M = 1$, $L = 3$), "
            "normalised to $\\int (a^2 + b^2)\\,dy = 1$, against the hidden coordinate "
            "$y$ (tip $-3$, brane $0$). Left: the zero mode, $a = A e^{My}$ with "
            "$A = \\sqrt{2M/(1 - e^{-2ML})}$ and $b = 0$, localised at the brane; the "
            "circles are the exact discrete zero mode $u_p = q^p u_0$ of the grid "
            "with $G = 150$. Right: the even level of rank 1, "
            "$\\varepsilon = \\sqrt{1 + (\\pi/3)^2}$: its $b$ vanishes at the tip "
            "and at the brane, and the grid values lie on the exact curves.")
```

`save_figure` saves Figure 16b.3. What the student should see: on the left, the zero mode rises from 0.07 at the tip to 1.42 at the brane, an orbital bound to the brane, with b exactly zero; on the right, the orbital of rank 1, whose b vanishes at both ends (the even boundary conditions) while a does not; in both panels the grid points lie on the exact curves.

In [9], odd parity in the rotated frame.

```
phi = sp.symbols("phi", real=True)
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices
s2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
rot = sp.cos(phi) * sp.eye(2) + sp.I * sp.sin(phi) * s1  # e^{i phi sigma1}
back = sp.cos(phi) * sp.eye(2) - sp.I * sp.sin(phi) * s1  # e^{-i phi sigma1}
ok2 = sp.simplify(back * s2 * rot - (sp.cos(2 * phi) * s2 + sp.sin(2 * phi) * s3))
ok3 = sp.simplify(back * s3 * rot - (sp.cos(2 * phi) * s3 - sp.sin(2 * phi) * s2))
check(ok2 == sp.zeros(2, 2) and ok3 == sp.zeros(2, 2),
      "sympy: the rotation turns sigma2 into cos(2 phi) sigma2 + sin(2 phi) sigma3 "
      "and sigma3 into cos(2 phi) sigma3 - sin(2 phi) sigma2")
```

The Pauli matrices are written as sympy matrices, `rot` is $R = \cos\phi + i\sin\phi\,\sigma_1$ and `back` its inverse $R^{-1} = \cos\phi - i\sin\phi\,\sigma_1$ (`sp.eye(2)` is the 2×2 unit matrix and `sp.I` the imaginary unit). `ok2` and `ok3` are $R^{-1}\sigma_2 R - (\cos 2\phi\,\sigma_2 + \sin 2\phi\,\sigma_3)$ and $R^{-1}\sigma_3 R - (\cos 2\phi\,\sigma_3 - \sin 2\phi\,\sigma_2)$, simplified; the check requires both to be the zero matrix: the two rotation rules of Section 16.14.

```
def sector_matrix(G, odd, j=1.0, k=0.0, a4=0.0, M=M_VALUE, L=L_VALUE, H=1.0):
    """Diagonal and off-diagonal of T for one sector: parity (odd True/False),
    block type j, 3-momentum k, slice a4; constant M, v = 0."""
    h = L / G
    r = np.arange(2 * G - 1)  # the position of each unknown in the list
    y_r = -L + (r + 1) * h / 2.0  # its y (half node for even r, node for odd r)
    is_u = r % 2 == 0
```

```
    K_r = np.exp(-H * y_r - a4) * k  # K = kappa k
    angle = j * 0.5 * math.pi * (y_r + L) / L if odd else np.zeros_like(y_r)
    dangle = j * math.pi / (2.0 * L) if odd else 0.0  # phi'
    m2 = M * np.cos(2 * angle) - K_r * np.sin(2 * angle)
    k2 = M * np.sin(2 * angle) + K_r * np.cos(2 * angle)
    d = np.where(is_u, j * (dangle + k2), j * (dangle - k2))
    pair = np.arange(2 * G - 2)  # the pair (r, r + 1)
    u_of_pair = np.where(pair % 2 == 0, pair, pair + 1)  # its half node
    sign = np.where(pair % 2 == 0, 1.0, -1.0)  # +1/h (u then w), -1/h (w then u)
    o = j * (sign / h + 0.5 * m2[u_of_pair])
    return d, o
```

`sector_matrix` builds the matrix of any sector, both parities, any block type j, 3-momentum k and slice $a_{4,0}$, for constant M and $v = 0$. `r` numbers the $2G-1$ unknowns and `y_r` gives each its position: the unknown number r lies at $-L + (r+1)h/2$, a half node for even r and a node for odd r; `is_u` marks the u unknowns. `K_r` is $K = \kappa k$ at each position. For odd parity `angle` is $\phi = j\,\frac{\pi}{2}\,\frac{y+L}{L}$ and `dangle` its derivative $\phi' = j\pi/(2L)$; for even parity both are zero. `m2` and `k2` are $m_2 = M\cos 2\phi - K\sin 2\phi$ and $k_2 = M\sin 2\phi + K\cos 2\phi$. The diagonal is $j(\phi' + k_2)$ in the u rows and $j(\phi' - k_2)$ in the w rows. For the neighbour entries, `pair` numbers the pairs $(r, r+1)$, `u_of_pair` is the position of the u in each pair (the first for even r, the second for odd r), and `sign` gives $+1/h$ for a u followed by a w and $-1/h$ for a w followed by a u; the entry is $j(\pm\frac{1}{h} + \frac{1}{2}m_2)$ with m_2 at the half node of the pair. These are the formulas of Section 16.14.

```
agree = []
for odd in (False, True):
    for j_value in (1.0, -1.0):
        for n2, r3, k_value in ((0, 1, 0.0), (4, 6, 0.5)):  # |k| = 0.25 sqrt(n2)
            phys = K.Phys(m=M_VALUE, L=L_VALUE, H=1.0)
            grid = K.Grid(phys, 300)
            sector = K.Sectors([(n2, r3, j_value, int(odd))])
            alpha, beta = K.sector_arrays(grid, phys, sector,
                                          np.full(grid.n, M_VALUE), np.zeros(grid.n))
            d, o = sector_matrix(300, odd, j=j_value, k=k_value)
            agree.append(max(np.max(np.abs(alpha[:, 0] - d)),
                             np.max(np.abs(beta[:, 0] - o))))
say(f"largest difference from ks_fd.sector_arrays over 8 sectors: {max(agree):.1e}")
check(max(agree) <= 1e-12,
      "sector_matrix equals the reference's matrices for both parities, both block "
      "types, k = 0 and k = 0.5")
```

The loops compare `sector_matrix` with the reference's `K.sector_arrays` for both parities (`int(odd)` is 1 for odd), both block types, and two momenta: $k = 0$ (shell $n_2 = 0$) and the shell $n_2 = 4$ with $r_3(4) = 6$ points, where $|k| = 0.25\sqrt{4} = 0.5$. `agree` collects the largest difference of each of the 8 comparisons. Out [9]: the largest is 0.0; the check requires at most 10^{-12}.

In [10], the matrices as heat maps (Figure 16b.4).

```
fig, axes = plt.subplots(1, 2, figsize=(10.0, 4.6))
for ax, odd, title in ((axes[0], False, "even parity"), (axes[1], True, "odd parity")):
    d, o = sector_matrix(8, odd)
```

```
    T = np.diag(d) + np.diag(o, 1) + np.diag(o, -1)
    image = ax.imshow(T, cmap="RdBu_r", vmin=-3.5, vmax=3.5)
    ax.set_title(f"$T$, {title}, $G = 8$")
    ax.set_xticks(range(0, 15, 2))  # whole numbers: rows and columns 0 to 14
    ax.set_yticks(range(0, 15, 2))
    ax.set_xlabel("column $c$")
    ax.set_ylabel("row $r$")
fig.colorbar(image, ax=axes, shrink=0.85, label="entry $T_{rc}$ (units of $m$)")
```

For $G = 8$ (15 unknowns, $h = 0.375$) the even and the odd matrix are assembled and drawn as **heat maps**: `imshow` paints entry T_{rc} as a coloured square in row r and column c, with the colour scale `"RdBu_r"` from blue (negative) through white (zero) to red (positive), fixed between -3.5 and 3.5 so that both panels use the same colours. The ticks are placed at the even numbers 0 to 14, and `fig.colorbar` adds one colour scale for both panels.

```
save_figure(fig, "matrix_heat_maps",
            "The reference matrices $T$ for $G = 8$ cells ($h = 0.375$, $M = 1$, "
            "$L = 3$, $j = +1$, $k = 0$) as heat maps, entry $T_{rc}$ in units of $m$ "
            "by colour (blue negative, red positive, white zero). Left, even parity: "
            "only the two neighbouring diagonals are filled, alternating "
            "$1/h + M/2 = 3.17$ and $-1/h + M/2 = -2.17$. Right, odd parity in the "
            "rotated frame: the diagonal carries $\\phi' \\pm M\\sin 2\\phi$ and the "
            "off-diagonal entries are $\\pm 1/h + \\tfrac12 M\\cos 2\\phi$, with "
            "$\\phi$ at the half node of the pair; $M\\cos 2\\phi$ goes from $+M$ at "
            "the tip to $-M$ at the brane, so the red entries fall from 3.16 to "
            "2.25 and the blue ones from -2.25 to -3.16, without a change of sign.")
```

`save_figure` saves Figure 16b.4. What the student should see: both matrices are filled only on the two diagonals next to the main diagonal (tridiagonal), and each is its own mirror image across the main diagonal (symmetric). In the even matrix the main diagonal is white (zero) and the neighbour entries alternate between red, $1/h + M/2 = 3.17$, and blue, $-1/h + M/2 = -2.17$. In the odd matrix the rotation puts $\phi' \pm M \sin 2\phi$ on the main diagonal, and the neighbour entries are $j(\pm 1/h + \frac{1}{2} m_2)$ with $m_2 = M \cos 2\phi$ (only half of m_2 enters, as in the even matrix only half of M), where ϕ is taken at the half node of the pair (`u_of_pair` in In [9]). Since m_2 goes from $+M$ at the tip to $-M$ at the brane, the entries move between the limits $1/h + M/2 = 3.17$ and $1/h - M/2 = 2.17$ (red) and $-1/h + M/2 = -2.17$ and $-1/h - M/2 = -3.17$ (blue). The half nodes do not reach the ends, so the drawn values stop a little short of these limits: with $\phi = \frac{\pi}{2} \frac{y+L}{L}$, the first red entry has $y + L = h/2$, so $\cos 2\phi = \cos(\pi/16) = 0.981$ and the entry is $2.667 + 0.490 = 3.16$; the last red entry has $y + L = 13h/2$, so $\cos 2\phi = \cos(13\pi/16) = -0.831$ and the entry is $2.667 - 0.416 = 2.25$. The blue entries run in the same way from -2.25 to -3.16. No entry changes sign, so red and blue keep alternating everywhere, but the red becomes lighter and the blue darker from the top left to the bottom right.

In [11], five grids and Richardson extrapolation.

```
GRIDS = (150, 300, 600, 1200, 2400)
EXACT = np.concatenate([EXACT_EVEN, EXACT_ODD])  # 18 levels: even, then odd
LEVELS = {}  # G -> the 18 levels on that grid
for G in GRIDS:
    found = []
```

```
    for odd in (False, True):
        d, o = sector_matrix(G, odd)
        lowest_particle = int(sturm_count(d, o, np.array([-1e-9]))[0])
        values_g, _ = bisection(d, o, lowest_particle + RANKS)
        found.append(values_g)
    LEVELS[G] = np.concatenate(found)
    say(f"G = {G:4d}: largest |level - exact| = "
        f"{np.max(np.abs(LEVELS[G] - EXACT)):.3e}")
```

`GRIDS` holds the five grids $G = 150$ to 2400 and `EXACT` the 18 exact levels (nine even, then nine odd; `np.concatenate` joins arrays). For each grid and each parity the matrix is built, the zero-mode number is found by a Sturm count at -10^{-9}, and bisection finds the nine levels of ranks -3 to 5; the 18 levels are kept in `LEVELS`, and the largest distance from the exact levels is printed. Out [11]: 2.607×10^{-3} for $G = 150$, then 6.518×10^{-4}, 1.629×10^{-4}, 4.074×10^{-5} and 1.018×10^{-5}: each halving of h divides the error by 4, as second order demands. This cell takes about ten seconds; the largest matrix has 4799 rows.

```
def richardson(x1, x2, x3):
    """Three-grid Richardson value and uncertainty, as in the reference program."""
    r_fine, r_coarse = (4.0 * x3 - x2) / 3.0, (4.0 * x2 - x1) / 3.0
    R = (16.0 * r_fine - r_coarse) / 15.0
    return R, np.abs(R - r_fine) + 2e-12 * np.maximum(1.0, np.abs(R))
```

`richardson` is the reference's three-grid extrapolation of Section 16.5, now for whole arrays at once (`np.abs` and `np.maximum` work entry by entry).

```
R18, U18 = richardson(LEVELS[300], LEVELS[600], LEVELS[1200])
record = json.loads(repository_file(
    "Revision/kohn_sham/reference/results/free-checks.json").read_text(encoding="utf-8"))
rows = [r for r in record["analytic"]["rows"]
        if float(r[0]) == 1.0 and float(r[1]) == 3.0 and r[3] == 1]
order = [(par, rank) for par in ("even", "odd") for rank in RANKS]
check([(r[2], r[4]) for r in rows] == order, "the record has our 18 rows in our order")
dev_R = max(abs(float(r[5]) - R18[i]) for i, r in enumerate(rows))
dev_U = max(abs(float(r[6]) - U18[i]) for i, r in enumerate(rows))
dev_1200 = max(abs(float(r[9]) - (LEVELS[1200][i] - EXACT[i]))
               for i, r in enumerate(rows))
say(f"against the record: R differs by at most {dev_R:.1e}, U by {dev_U:.1e}, "
    f"G = 1200 minus exact by {dev_1200:.1e}")
say(f"Richardson value minus exact: at most {np.max(np.abs(R18 - EXACT)):.1e} "
    f"(single grid G = 1200: {np.max(np.abs(LEVELS[1200] - EXACT)):.1e})")
check(dev_R < 1e-12 and dev_U < 1e-12 and dev_1200 < 1e-12,
      "our levels, R and U reproduce the 18 rows of the record",
      record="Revision/kohn_sham/reference/results/free-checks.json, analytic rows")
check(np.max(np.abs(R18 - EXACT)) <= 1e-11,
      "the Richardson values equal the exact levels to 1e-11",
      record="Revision/kohn_sham/reports/ks-reference.json, check "
             "free_k0_analytic_spectra")
```

`R18` and `U18` are the Richardson values and uncertainties of the 18 levels from $G = 300$, 600, 1200. `record` is the committed file `free-checks.json`; `rows` keeps its rows with $m = 1$, $L = 3$ and $j = +1$

(each row lists m, L, parity, j, rank, R, U, the exact level, R minus exact, and the $G = 1200$ value minus exact). The first check confirms that these are our 18 rows in our order (`order` lists the pairs (parity, rank)). `dev_R`, `dev_U` and `dev_1200` are the largest differences between the record and our R, our U and our $G = 1200$ errors. Out [11]: 8.9×10^{-15}, 1.3×10^{-15} and 6.2×10^{-15}, so the second check (all below 10^{-12}) reproduces the record. The Richardson values lie within 1.2×10^{-14} of the exact levels, while the best single grid $G = 1200$ is 4.1×10^{-5} away: two Richardson steps gain about nine digits. The third check requires 10^{-11}, the criterion of the reference's check `free_k0_analytic_spectra`.

In [12], the Richardson ladder (Figure 16b.5).

```
hs = np.array([L_VALUE / G for G in GRIDS])
picks = [("even", 1), ("odd", 0), ("even", 5)]
fig, ax = plt.subplots(figsize=(8.0, 6.4))
for colour, (par, rank) in zip(PALETTE, picks):
    i = order.index((par, rank))
    single = np.array([abs(LEVELS[G][i] - EXACT[i]) for G in GRIDS])
    one = np.array([abs((4.0 * LEVELS[GRIDS[g + 1]][i] - LEVELS[GRIDS[g]][i]) / 3.0
                        - EXACT[i]) for g in range(4)])
    two = np.array([abs(richardson(LEVELS[GRIDS[g]][i], LEVELS[GRIDS[g + 1]][i],
                                   LEVELS[GRIDS[g + 2]][i])[0] - EXACT[i])
                    for g in range(3)])
    label = f"{par} rank {rank} ($\\varepsilon = {EXACT[i]:.3f}$)"
    ax.loglog(hs, single, "o-", color=colour, lw=1.4, label=f"{label}: single grid")
    ax.loglog(hs[:4], one, "s--", color=colour, lw=1.2, label="one step")
    ax.loglog(hs[:3], np.maximum(two, 1e-16), "^:", color=colour, lw=1.2,
              label="two steps")
```

`hs` holds the five cell widths. For three levels (even rank 1, odd rank 0 and even rank 5; `order.index` finds a level's position), the cell computes three error series against the exact level: `single`, the five single grids; `one`, the four one-step values $(4x(h/2) - x(h))/3$ of neighbouring pairs; and `two`, the three three-grid values of neighbouring triples. Each series is drawn on logarithmic axes in the level's colour: circles with solid lines, squares with dashed lines, triangles with dotted lines (`"^:"`); zero errors are raised to 10^{-16}.

```
ax.axhline(1e-14, color="k", lw=0.8, ls="-.", label="rounding floor $10^{-14}$")
ax.set_xlabel("cell width $h$ of the coarsest grid used (units of $1/H$)")
ax.set_ylabel("distance from the exact level (units of $m$)")
ax.set_title("The Richardson ladder: slopes 2, 4 and then rounding")
ax.set_ylim(1e-16, 1e-1)
ax.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
ax.set_xticks(hs)  # tick labels only at the five cell widths
ax.set_xticklabels([f"{t:g}" for t in hs])
ax.legend(fontsize=7, ncol=3, loc="upper center", bbox_to_anchor=(0.5, -0.13))
```

The dash-dotted line at 10^{-14} marks the rounding floor. The axes, the tick labels at the five cell widths (as in Notebook 16a, In [8]) and a legend in three columns below the plot (`bbox_to_anchor=(0.5, -0.13)` places it under the axes) complete the figure.

```
save_figure(fig, "convergence_ladder",
            "Distance of the computed level from the exact level, in units of $m$, "
```

```
            "against the cell width $h = 3/G$ for $G = 150$ to $2400$, logarithmic "
            "axes, for three levels of the free problem: circles, single grids "
            "(slope 2, error proportional to $h^2$); squares, one Richardson step "
            "$(4x(h/2) - x(h))/3$ (slope 4); triangles, the three-grid value of the "
            "reference (two steps), at the rounding floor near $10^{-14}$ on the "
            "coarsest triple for the two lower levels and from the second triple on "
            "for rank 5. "
            "The high level of rank 5 has the largest errors, because its orbital "
            "varies fastest from cell to cell.")
```

`save_figure` saves Figure 16b.5. What the student should see: the single-grid errors fall along straight lines of slope 2 (a factor 4 per halving), the one-step errors along lines of slope 4 (a factor 16), and the two-step errors reach the rounding floor of about 10^{-14} at once, for the two lower levels already on the coarsest triple and for the level of rank 5 from the second triple on; there they scatter instead of falling further, because at the floor rounding, not the grid, sets the error. The high level of rank 5 has the largest errors in every series, because its orbital varies fastest from cell to cell.

In [13], every ratio tends to 4 (Figure 16b.6).

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
worst = {}
for colour, (g1, g2, g3) in zip(PALETTE, ((150, 300, 600), (300, 600, 1200),
                                          (600, 1200, 2400))):
    d1, d2 = LEVELS[g1] - LEVELS[g2], LEVELS[g2] - LEVELS[g3]
    keep = (np.abs(d1) > 1e-10) & (np.abs(d2) > 1e-10)  # not the zero mode
    ratio = d1[keep] / d2[keep]
    worst[(g1, g2, g3)] = float(np.max(np.abs(ratio - 4.0)))
    ax.plot(EXACT[keep], ratio, "o", color=colour, ms=6,
            label=f"grids {g1}, {g2}, {g3}: largest $|$ratio $- 4|$ = "
                  f"{worst[(g1, g2, g3)]:.1e}")
ax.axhline(4.0, color="k", lw=1.0)
ax.set_xlabel("exact level $\\varepsilon$ (units of $m$)")
ax.set_ylabel("$(x(G) - x(2G))/(x(2G) - x(4G))$")
ax.set_title("Second-order convergence: every ratio tends to 4")
ax.legend(fontsize=8)
```

For three triples of grids the convergence ratio of Section 16.4 is computed for all 18 levels; `keep` leaves out the one level whose differences are below 10^{-10}, the zero mode (even parity, rank 0), which is exact on every grid, so 17 ratios remain per triple (the odd parity has no zero mode: its rank 0 is the bulk edge 1.2923). `worst` keeps each triple's largest distance from 4, and the ratios are drawn against the exact level energies, one colour per triple, with the line 4.

```
save_figure(fig, "error_ratios",
            "The convergence ratio $(x(G) - x(2G))/(x(2G) - x(4G))$ of the 17 nonzero "
            "levels of ranks $-3$ to $5$ (both parities, $M = 1$, $L = 3$, $k = 0$) "
            "against the exact level in units of $m$, for three triples of grids. A "
            "ratio of 4 means an error proportional to $h^2$; the ratios approach 4 "
            "as the grids get finer, fastest for the low levels. The zero mode (even "
            "parity, rank 0) is exact on every grid and has no ratio.")
say("largest |ratio - 4| per triple: " + ", ".join(
    f"{k}: {v:.2e}" for k, v in worst.items()))
```

```
check(worst[(300, 600, 1200)] <= 0.01
      and worst[(600, 1200, 2400)] < worst[(300, 600, 1200)]
      and int(np.sum(keep)) == 17,  # 18 levels minus the one zero mode
      "the ratios lie within 0.01 of 4 and approach 4 on finer grids",
      record="Revision/kohn_sham/reports/ks-reference.json, check "
             "free_convergence_order_two")
```

`save_figure` saves Figure 16b.6, and the printed line (Out [13]) gives the largest distances from 4: 6.57×10^{-4}, 1.64×10^{-4} and 4.10×10^{-5} for the triples $(150, 300, 600)$, $(300, 600, 1200)$ and $(600, 1200, 2400)$, each four times smaller than the one before, as the correction $\frac{15}{16}\frac{d}{c}h^2$ of Section 16.4 predicts. What the student should see: the 17 ratios of each triple lie close to 4, the finer triples closer, and the high levels deviate most. The check requires the middle triple within 0.01 of 4 (the reference's criterion `free_convergence_order_two` asks for $[3.99, 4.01]$), the finest triple closer still, and exactly 17 kept levels (`np.sum(keep)` counts the true entries of the mask of the last triple; `int` makes it a whole number), so that only the zero mode was left out.

In [14], the brane band along the deflating history.

```
SLICES = (0.0, 0.5, 1.0, 1.5, 2.0)


def slope_formula(a4, M=M_VALUE, H=1.0, L=L_VALUE):
    """c = e^(-a4) (2M/(1 - e^(-2ML))) (1 - e^(-(2M - H)L))/(2M - H)."""
    return (math.exp(-a4) * 2.0 * M / (1.0 - math.exp(-2.0 * M * L))
            * (1.0 - math.exp(-(2.0 * M - H) * L)) / (2.0 * M - H))
```

`SLICES` are the five slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$, and `slope_formula` is the slope $c(a_{4,0})$ derived in Section 16.15.

```
per_grid = {}
for G in (300, 600, 1200):
    yh, u = zero_mode(G)
    weight = u * u * (L_VALUE / G)  # u_p^2 h, summing to 1
    per_grid[G] = np.array([np.sum(np.exp(-yh - a4) * weight) for a4 in SLICES])
R_slope, U_slope = richardson(per_grid[300], per_grid[600], per_grid[1200])
```

For the grids 300, 600, 1200 the discrete zero mode is built; `weight` holds $u_p^2 h$, which adds up to 1; and the slope at each slice is $\sum \kappa(y_{p+1/2})\, u_p^2\, h$ with $\kappa = e^{-y-a_{4,0}}$ ($H = 1$), the discrete Hellmann-Feynman formula of Section 16.15. Richardson extrapolation over the three grids gives `R_slope`.

```
formula = np.array([slope_formula(a4) for a4 in SLICES])
theory = json.loads(repository_file("Revision/kohn_sham/ks-theory.json").read_text(
    encoding="utf-8"))
stated = float(theory["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
rec_rows = record["brane_band_slope"]["rows"]
print("a4,0   slope (grid, Richardson)   formula            record")
for i, a4 in enumerate(SLICES):
    print(f"{a4:4.1f}   {R_slope[i]:.15f}      {formula[i]:.15f}  "
          f"{float(rec_rows[i][1]):.15f}")
```

`formula` holds the exact slopes. `theory` is the committed theory file `ks-theory.json`, and `stated` its number `checksNumeric.braneBandSlope_M1_H1_L3_a0`, the slope at $a_{4,0} = 0$ written in the file. `rec_rows` are the rows of the record's entry brane_band_slope. The table of Out [14] prints, for each slice, the grid value, the formula and the record with fifteen decimals; at $a_{4,0} = 0$ the three agree to fourteen decimals, 1.90514825364486, and at $a_{4,0} = 2$ all three read 0.257833778514766.

```
rel_formula = np.max(np.abs(R_slope - formula) / formula)
rel_record = max(abs(float(r[1]) - R_slope[i]) / formula[i]
                 for i, r in enumerate(rec_rows))
say(f"largest relative difference: from the formula {rel_formula:.1e}, from the "
    f"record {rel_record:.1e}")
print("stated in ks-theory.json (checksNumeric): c at a4,0 = 0 is "
      + theory["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
check(rel_formula <= 1e-11 and abs(slope_formula(0.0) - stated) <= 1e-15 * stated,
      "the grid slope equals c e^(-a4,0) of ks-theory.json at the five slices",
      record="Revision/kohn_sham/reports/ks-reference.json, check "
             "free_brane_band_slope")
check(rel_record <= 1e-13, "our slopes reproduce the record's brane_band_slope rows",
      record="Revision/kohn_sham/reference/results/free-checks.json, brane_band_slope")
check(np.allclose(R_slope[1:] / R_slope[:-1], math.exp(-0.5), rtol=1e-12, atol=0),
      "from slice to slice the slope shrinks by exactly e^(-0.5)")
```

`rel_formula` and `rel_record` are the largest relative differences from the formula and from the record: 3.8×10^{-16} and 2.6×10^{-15} (Out [14]). The line printed next shows the number written in `ks-theory.json`. The first check requires the grid slope to equal the formula to 10^{-11} and the formula at $a_{4,0} = 0$ to equal the stated number to 10^{-15}, the reference's criterion `free_bra ne_band_slope`; the second reproduces the record's rows; the third confirms that from slice to slice the slope shrinks by exactly $e^{-0.5}$ (`np.allclose` with a relative tolerance 10^{-12}; `R_slope[1:] / R_slope[:-1]` divides each slope by the one before).

In [15], the brane band as a picture (Figure 16b.7).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2))
a_fine = np.linspace(0.0, 2.0, 100)
left.semilogy(a_fine, [slope_formula(a) for a in a_fine], color="k", lw=1.2,
              label="formula $c\\,e^{-a_{4,0}}$")
left.semilogy(SLICES, R_slope, "o", color=PALETTE[0], ms=8,
              label="grid, three-grid Richardson value")
left.set_xlabel("slice $a_{4,0}$ (3-space inflates, extra times deflate)")
left.set_ylabel("brane-band slope $d\\varepsilon/dk$ at $k = 0$")
left.set_title("The brane band flattens as $e^{-a_{4,0}}$")
left.legend(fontsize=8)
```

The left panel draws the formula $c\,e^{-a_{4,0}}$ on 100 slices as a black line and the five Richardson values as circles, on a logarithmic vertical axis, where a function e^{-a} is a straight line.

```
hs3 = np.array([L_VALUE / G for G in (300, 600, 1200)])
right.loglog(hs3, [abs(per_grid[G][0] - formula[0]) for G in (300, 600, 1200)],
             "o-", color=PALETTE[1], lw=1.5, ms=7,
             label="single grid, $a_{4,0} = 0$")
```

```
right.loglog(hs3, abs(per_grid[300][0] - formula[0]) / 3.0 * (hs3 / hs3[0]) ** 2,
             "k--", lw=0.9, label="slope 2 (drawn a factor 3 lower)")
right.xaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
right.set_xticks(hs3)  # tick labels only at the three cell widths
right.set_xticklabels([f"{t:g}" for t in hs3])
right.set_xlabel("cell width $h$")
right.set_ylabel("$|c(G) - c|$")
right.set_title("grid error of the slope")
right.legend(fontsize=8)
fig.tight_layout()
```

The right panel draws, for $a_{4,0} = 0$, the distance of the single-grid slopes from the exact slope against the cell width, with a dashed line of slope 2 drawn a factor 3 lower, and tick labels at the three cell widths.

```
save_figure(fig, "brane_band_slope",
            "The slope $d\\varepsilon/dk$ at $k = 0$ of the brane band (even parity, "
            "$j = +1$, $M = H = 1$, $L = 3$). Left: against the slice $a_{4,0}$ of the "
            "history, logarithmic vertical axis; the Richardson values of the grid "
            "(circles) lie on the formula of ks-theory.json, $c(0)\\,e^{-a_{4,0}}$ "
            "with $c(0) = 1.90515$: the 3-momentum is redshifted as 3-space inflates "
            "while the three extra times deflate. Right: the single-grid values at "
            "$a_{4,0} = 0$ approach the exact slope with an error proportional to "
            "$h^2$ (parallel to the dashed line of slope 2).")
```

`save_figure` saves Figure 16b.7. What the student should see: on the left, the five circles lie exactly on a straight line falling by a factor $e^{-2} = 0.135$ from 1.905 to 0.258: the brane band flattens as 3-space inflates and the extra times deflate, because the 3-momentum is redshifted; on the right, the grid error of the slope falls with slope 2, from about 1.3×10^{-5} to 8×10^{-7}.

In [16], the reference program's own free-field job.

```
import run_reference as RR  # noqa: E402  the reference program (Revision code)

CO = RR.theory_coefficients()  # coefficients read and checked from ks-theory.json
free = RR.jsonable(RR.free_checks_job(CO)["data"])
committed = repository_file(
    "Revision/kohn_sham/reference/results/free-checks.json").read_text(encoding="utf-8")
text = json.dumps(free, indent=1, ensure_ascii=True) + "\n"  # as the program writes
say(f"the new free-field result is identical to the record byte for byte: "
    f"{text == committed}")
```

The reference program is imported (as in Notebook 16a), and its job `free_checks_job` is run: it computes the exact-level comparison for three choices of (M, L), both block types and both parities on five grids, the zero mode, the brane-band slope and the split of every sector into particle and sea levels (about ten seconds). Its result is written as text exactly as the program writes the file and compared with the committed file; Out [16] says that the two are identical byte for byte.

```
same = json.loads(text) == json.loads(committed)
if not same:  # another computer may round the last digits differently
```

```
    old = json.loads(committed)["analytic"]
    same = all(abs(float(a[5]) - float(b[5])) < 1e-12
               for a, b in zip(free["analytic"]["rows"], old["rows"]))
check(same, "the reference's free-field job reproduces free-checks.json",
      record="Revision/kohn_sham/reference/results/free-checks.json")
```

If the texts were not identical (another computer may round the last digits differently), `same` would still accept the result when the decoded contents are equal or the Richardson values of all rows agree to 10^{-12}. The check requires this.

```
an, zm = free["analytic"], free["zero_mode"]
bb, pb = free["brane_band_slope"], free["particle_branch"]
criteria = {
    "free_k0_analytic_spectra": an["max_error_richardson"] <= 1e-11,
    "free_convergence_order_two": 3.99 <= an["ratio_min"] and an["ratio_max"] <= 4.01,
    "free_k0_block_type_symmetry": an["jsym_max"] <= 1e-13
    and zm["jsym_profile"] <= 1e-13,
    "free_zero_mode": zm["max_abs_eps"] <= 1e-13 and zm["max_w_over_u"] <= 1e-12
    and zm["profile_richardson_max_error"] <= 1e-9,
    "free_brane_band_slope": bb["max_rel"] <= 1e-11
    and bb["formula_vs_theory_number"] <= 1e-15,
    "free_particle_branch": pb["violations"] == 0
    and pb["closest_to_zero_away_from_zero_modes"] > 1e-3,
}
```

`an`, `zm`, `bb` and `pb` are the four parts of the result. `criteria` is a dictionary from the names of the six free-field checks of the reference report to their conditions, with exactly the thresholds of the reference program: the Richardson levels within 10^{-11} of the exact ones; all convergence ratios in $[3.99, 4.01]$; the two block types' spectra and zero-mode profiles equal to 10^{-13}; the zero mode's eigenvalue below 10^{-13}, its w entries below 10^{-12} of its u entries and its extrapolated profile within 10^{-9} of $A\,e^{My}$; the slope within 10^{-11} (relative) of the formula, and the formula equal to the stated number; and no level on the wrong side of zero, with no level within 10^{-3} of zero apart from the zero modes.

```
say(f"max error of the Richardson values {an['max_error_richardson']:.1e}; ratios "
    f"{an['ratio_min']:.5f} to {an['ratio_max']:.5f}; zero mode |eps| <= "
    f"{zm['max_abs_eps']:.1e}; slope {bb['max_rel']:.1e} relative")
report = json.loads(repository_file(
    "Revision/kohn_sham/reports/ks-reference.json").read_text(encoding="utf-8"))
verdicts = {c["name"]: c["verdict"] for c in report["checks"]}
for name, ok in criteria.items():
    check(ok and verdicts[name] == "PASS", f"{name} holds for the new run",
          record=f"Revision/kohn_sham/reports/ks-reference.json, check {name}")
```

The printed line (Out [16]) gives the largest error of the Richardson levels (2.8×10^{-14}, over all three (M, L)), the range of the ratios (3.99977 to 4.00019), the size of the zero mode's eigenvalue (1.3×10^{-39}) and the slope's relative error (2.6×10^{-15}); these are the numbers of the reference report. The report's verdicts are read into a dictionary, and the loop makes one check per criterion: it must hold for the new run and the report's verdict must be PASS. Six PASS lines follow, each naming the check of `Revision/kohn_sham/reports/ks-reference.json` that it reproduces.

In [17], the last check.

```
figure_files = [f"{FIGURE_FOLDER}/16b_{k}_{name}.png" for k, name in enumerate(
    ["staggered_grid", "sturm_staircase", "orbitals", "matrix_heat_maps",
     "convergence_ladder", "error_ratios", "brane_band_slope"], start=1)]
check(all(output_file(f).is_file() for f in figure_files),
      "every figure file of this notebook exists")
all_checks_passed()
```

As In [23] of Notebook 16a, for the seven figures; it prints ALL 31 CHECKS PASSED (notebook 16b): five checks in In [2], three each in In [4], In [7], In [11] and In [14], two each in In [5] and In [9], seven in In [16], and one each in In [6], In [13] and In [17].

16.20 How the cross-check caught a rounding error: the chemical potential

What happened. The first cross-check of the record compared a representative subset of the canonical matrix (18 of the 75 ground states and 8 of the 135 thermal states). It failed one check, `thermo_state_functions`, in one state, N8_lamm1_a00_T10: the Rust solver gave the chemical potential $\mu = 0.2100104489071649$ and the reference 0.2100104497343054, a difference of 8.27×10^{-10} against a tolerance of 7.97×10^{-12}, a ratio of about 104 (`Revision/kohn_sham/checker/README.md`, History 1). The tolerance was not widened. Instead the cause was found: the levels of both solvers were right, and the Rust solver's computation of μ from them was wrong in the tenth decimal, because of the way ordinary computer numbers round. This section explains every step of that diagnosis; Notebook 16c repeats it with numbers it computes itself.

The Mermin condition and its slope, line by line. At a temperature $T > 0$ the occupation of a level ε_i is the Fermi-Dirac number $f(x_i)$ with $x_i = (\varepsilon_i - \mu)/T$ and $f(x) = 1/(1 + e^x)$, and each level holds g_i states (its **degeneracy**; in this model $g = 4r_3(n_2)$, Chapter 15). The chemical potential is the root of the **Mermin condition**

$$N(\mu) = \sum_i g_i \, f\Big(\frac{\varepsilon_i - \mu}{T}\Big) = N.$$

How fast does $N(\mu)$ change with μ?

$$f'(x) = -\frac{e^x}{(1 + e^x)^2}.$$

Rule: $f = (1 + e^x)^{-1}$ and the chain rule: the derivative of u^{-1} is $-u^{-2}u'$, with $u' = e^x$.

$$\frac{e^x}{(1 + e^x)^2} = \frac{1}{1 + e^x} \cdot \frac{e^x}{1 + e^x} = f(x)\,(1 - f(x)).$$

Rule: split the fraction, and $1 - f = \frac{1+e^x-1}{1+e^x} = \frac{e^x}{1+e^x}$.

$$\frac{dN}{d\mu} = \sum_i g_i \, f'(x_i) \, \frac{dx_i}{d\mu} = \frac{1}{T} \sum_i g_i \, f(x_i)\,(1 - f(x_i)).$$

Rule: the chain rule with $dx_i/d\mu = -1/T$, and $f' = -f(1-f)$. Every term is positive, so $N(\mu)$ increases with μ and has at most one root. As μ goes to $-\infty$ every f tends to 0, and as μ goes to $+\infty$ every f tends to 1, so if the levels hold more than N states the root exists (intermediate value theorem). We write $N' = dN/d\mu$.

The activated regime, line by line. In N8_lamm1_a00_T10 the eight quanta exactly fill the two lowest levels, $\varepsilon_0 = 0.000245562708$ (two levels of 4 states each, $g_0 = 8$ together); the next level is $\varepsilon_1 = 0.430761462627$ with $g_1 = 24$ states (Notebook 16c, Out [2]). The **gap** between them, 0.4305, is 43 times the temperature $T = 0.01$. When a gap is many times T the state is in the **activated regime**: only very few quanta are thermally lifted across it. Write H_{th} for the **thermal holes** (the subscript th, for thermal, keeps them apart from the author's constant H), the missing quanta below μ, and P for the **thermal particles** above it:

$$H_{th} = g_0\,(1 - f(x_0)) = \frac{g_0}{1 + e^{(\mu-\varepsilon_0)/T}} \approx g_0\, e^{-(\mu-\varepsilon_0)/T}, \qquad P = g_1\, f(x_1) \approx g_1\, e^{-(\varepsilon_1-\mu)/T}.$$

Rule: $1 - f(x) = 1/(1 + e^{-x})$, and $1/(1 + e^s) \approx e^{-s}$ when s is large (here $s \approx 21$, so the error is a relative $e^{-21} \approx 10^{-9}$). Since $g_0 = N$, the Mermin condition $g_0(1 - H_{th}/g_0) + P = N$ says simply $P = H_{th}$: as many quanta above the gap as are missing below it. Taking logarithms of $g_0\, e^{-(\mu-\varepsilon_0)/T} = g_1\, e^{-(\varepsilon_1-\mu)/T}$:

$$\ln g_0 - \frac{\mu - \varepsilon_0}{T} = \ln g_1 - \frac{\varepsilon_1 - \mu}{T} \quad \Longrightarrow \quad \mu = \frac{\varepsilon_0 + \varepsilon_1}{2} - \frac{T}{2}\,\ln\frac{g_1}{g_0}.$$

Rule: $\ln(ge^{-s}) = \ln g - s$; collect the terms with μ ($2\mu/T$ on one side) and solve. With the numbers, $(\varepsilon_0 + \varepsilon_1)/2 = 0.215503513$ and $\frac{T}{2}\ln 3 = 0.005493061$, so $\mu \approx 0.210010451$: μ sits near the middle of the gap, shifted a little towards the lower level because the upper level has three times more states. (This simple estimate is within 1.5×10^{-9} of the exact root; Exercise 7 finds the small correction.) Then

$$N' = \frac{1}{T}\sum_i g_i f_i (1 - f_i) \approx \frac{H_{th} + P}{T} = \frac{2H_{th}}{T} = \frac{2 \cdot 6.21 \times 10^{-9}}{0.01} = 1.24 \times 10^{-6}.$$

Rule: for a full level $f(1-f) \approx 1 - f$, for an empty level $f(1-f) \approx f$; then $P = H_{th}$, and $H_{th} = 6.21 \times 10^{-9}$ (Notebook 16c, Out [3]). So $N(\mu)$ is extremely flat: moving μ by 10^{-9} changes $N(\mu)$ by only 1.2×10^{-15}. Conversely, an error δ in a computed value of $N(\mu) - N$ moves the computed root by δ/N', about 8×10^5 times δ (the root of the tangent line moves by the error divided by the slope). The problem is **ill-conditioned**: its answer reacts strongly to small errors of its input.

Numbers in a computer. A **double**, the ordinary number of a computer, is stored as a sign, 53 binary digits and an exponent (the IEEE 754 standard). Between two neighbouring powers of two, $2^k \le x < 2^{k+1}$, the doubles are equally spaced, 2^{k-52} apart. Between 1 and 2 the spacing is $2^{-52} = 2.2 \times 10^{-16}$, the **machine epsilon** ϵ_{mach}; between 4 and 8 it is $2^{-50} = 8.9 \times 10^{-16}$; between 8 and 16 it is $2^{-49} = 1.8 \times 10^{-15}$. The result of every operation is **rounded** to the nearest double. A sum whose exact value is close to 8 can therefore come out only as a multiple of 2^{-50} (just below 8) or of 2^{-49} (just above 8).

The staircase, line by line. Near its root the exact sum is $S(\mu) = 8 + N'\,(\mu - \mu_{root})$. Added up directly in doubles, the computed residual $D(\mu) = S(\mu) - 8$ is exactly zero whenever the computed sum is the double 8 itself, and that happens whenever S lies within half a spacing of 8:

$$8 - 2^{-51} \le S(\mu) \le 8 + 2^{-50}.$$

Rule: rounding to the nearest double; half the spacing below 8 is $\frac{1}{2}2^{-50} = 2^{-51}$, half the spacing above is $\frac{1}{2}2^{-49} = 2^{-50}$.

$$\Delta\mu = \frac{2^{-51} + 2^{-50}}{N'} = \frac{3 \cdot 2^{-51}}{N'} = \frac{1.33 \times 10^{-15}}{1.242 \times 10^{-6}} = 1.07 \times 10^{-9}.$$

Rule: an interval of length $3 \cdot 2^{-51}$ in S is an interval of length $3 \cdot 2^{-51}/N'$ in μ, because S changes by N' per unit of μ. So in the computer the direct residual is a **staircase**: constant over intervals of about 10^{-9} in μ, jumping by one spacing between them, and exactly zero over a whole interval about 10^{-9} wide. Notebook 16c measures this interval as 1.05×10^{-9} wide, from 8.25×10^{-10} below the root to 2.30×10^{-10} above it; its position is shifted from the ideal one by the rounding of the individual terms and of the partial sums along the way (Figure 16c.2).

Why bisection then stops at the wrong place. Bisection on the residual asks at each midpoint only one question: is $D(mid) < 0$? If yes, the root is above mid; if not, at or below it. On the zero interval the answer is always "not below zero", so every midpoint inside it becomes the new upper end, and the search converges to the **left end** of the zero interval, about 8.3×10^{-10} below the true root. That is the faulty value of the first cross-check: Notebook 16c reproduces 0.2100104489071649 in all sixteen digits by bisection on the direct sum (In [7]). The record's Rust solver now writes for every thermal state the root that the former direct count would give on the same final levels, minus the present μ, in the column `mu_direct_count_minus_mu` of `Revision/kohn_sham/results/thermo/thermodynamics.csv`; for N8_lamm1_a00_T10 it is -8.267×10^{-10}.

Deciding which value is right: 40 digits and Newton's method. To decide, the checker recomputed μ from each solver's own final levels with 40 significant digits, using the package mpmath, which computes with as many digits as one asks for (slowly, in software). With 40 digits the rounding of the sum is about $10^{-40}N$, which moves the root by about $10^{-40}N/N' < 10^{-33}$: far below anything a double can hold. The root is found by **Newton's method**. Line by line: let μ_k be a guess.

$$N(\mu) \approx N(\mu_k) + N'(\mu_k)\,(\mu - \mu_k).$$

Rule: near μ_k a smooth function is close to its tangent line (Taylor's theorem to first order).

$$N(\mu_k) + N'(\mu_k)\,(\mu_{k+1} - \mu_k) = N \quad \Longrightarrow \quad \mu_{k+1} = \mu_k - \frac{N(\mu_k) - N}{N'(\mu_k)}.$$

Rule: the next guess is where the tangent line reaches the value N; subtract $N(\mu_k)$ and divide by $N'(\mu_k)$.

Why Newton's method doubles the digits, line by line. Let $e_k = \mu_k - \mu^*$ be the error of the guess, μ^* the root, and write N', N'' for the derivatives at μ^*.

$$N(\mu_k) - N = N' e_k + \tfrac{1}{2} N'' e_k^2 + \dots, \qquad N'(\mu_k) = N' + N'' e_k + \dots$$

Rule: Taylor's theorem around μ^*, with $N(\mu^*) = N$.

$$e_{k+1} = e_k - \frac{N' e_k + \frac{1}{2} N'' e_k^2}{N' + N'' e_k} = e_k - e_k\Big(1 + \tfrac{1}{2}\tfrac{N''}{N'} e_k\Big)\Big(1 - \tfrac{N''}{N'} e_k\Big) + \dots = \frac{N''}{2N'}\, e_k^2 + \dots$$

Rule: subtract μ^* from Newton's formula; take $N' e_k$ out of the numerator and N' out of the denominator; $1/(1+s) = 1 - s + \dots$; multiply out, keeping terms up to e_k^2. The new error is proportional to the square of the old one (**quadratic convergence**): once the error is small, the number of correct digits roughly doubles at every step. Notebook 16c (Out [4]) starts in the middle of the gap and shows the distances 5.5×10^{-3}, 4.9×10^{-4}, 4.0×10^{-7}, 2.1×10^{-16} and 1.5×10^{-34}.

The diagnosis. The 40-digit root on the Rust levels is 0.21001044973390364544 and on the reference levels 0.21001044973430543050; they differ by 4.0×10^{-13}, which is how much the two solvers' levels differ (each is uncertain by about 10^{-12}). The faulty Rust value was 8.3×10^{-10} below both. So the levels were right and the root-finding was wrong (the cross-check's check `thermo_mu_high_precision` now compares these roots in all 270 level sets, worst ratio 0.050: `Revision/kohn_sham/reports/ks-crosscheck.json`). The reference had had the same defect: in its first complete run its μ missed its own 40-digit root by 1.15×10^{-9} in the same state (`Revision/kohn_sham/reference/README.md`, History); it was repaired in the same way.

Why the measured uncertainty did not warn. Before the repair the canonical and the refined Rust runs computed μ with the same direct count, so both made almost the same rounding error, and their difference, from which U_{Rust} is measured (Section 16.6), did not contain it. An uncertainty measured by repeating a computation measures only the errors that the repetition changes. After the repair the refined run computes the root with a second, exactly equivalent form whose rounding takes a different path, so that their difference now contains the rounding error of the root (`Revision/kohn_sham/reports/ks-rust-determinism.json`, check `refined_mermin_root_path`). This is the deeper lesson of the episode, and the reason why a truly independent program is worth the effort.

The repair: a well-conditioned form, line by line. The trouble is that the direct sum adds numbers of size 1 (the full occupations) to get a total near 8 and then subtracts 8, so its rounding errors are of size $\epsilon_{mach} \cdot 8$, while the interesting part, $P - H_{th}$, is about 10^{-9} in size. The repair rewrites the residual exactly so that the large numbers are added only as whole numbers. Split the levels into those below μ ($x_i < 0$) and those above ($x_i \geq 0$).

$$f(x) + f(-x) = \frac{1}{1+e^x} + \frac{1}{1+e^{-x}} = \frac{1}{1+e^x} + \frac{e^x}{e^x+1} = 1.$$

Rule: multiply the numerator and the denominator of the second fraction by e^x; then the two fractions have the same denominator.

$$\sum_i g_i f(x_i) - N = \sum_{\text{below}} g_i (1 - f(-x_i)) + \sum_{\text{above}} g_i f(x_i) - N.$$

Rule: below μ write each occupation as $f(x_i) = 1 - f(-x_i)$.

$$W(\mu) = \Big(\sum_{\text{below}} g_i - N \Big) - \sum_{\text{below}} g_i \, f(-x_i) + \sum_{\text{above}} g_i \, f(x_i) = -d - H_{th} + P.$$

Rule: collect the terms; $d = N - \sum_{\text{below}} g_i$ is a whole number, H_{th} the thermal holes and P the thermal particles. The value of W is exactly that of the direct residual; only the rounding differs. The bracket d is a sum of whole numbers, which doubles add without any error. H_{th} and P are sums of small positive numbers, each computed with a relative error of about ϵ_{mach}, so near the root W is known to about $\epsilon_{mach}(P + H_{th})$, here a few times 10^{-24}, instead of $\epsilon_{mach} \cdot 8$. The reference solver's module uses exactly this form (`mermin_residual` in `Revision/kohn_sham/reference/ks_fd.py`); the Rust solver uses its logarithm, $\ln(P + d_-) - \ln(H_{th} + d_+)$ with $d_\pm = \max(\pm d, 0)$, which is nearly a straight line in μ (the form LogBalance of `Revision/kohn_sham/solver/src/mermin.rs`), and its refined run uses the linear form (LinearDeviation).

The rounding bounds, line by line. How large can the error of a computed root be?

$$N' \, (\mu_c - \mu^*) + \delta R = 0 \quad \Longrightarrow \quad |\mu_c - \mu^*| = \frac{|\delta R|}{N'}.$$

Rule: near the root the computed residual is the tangent line plus its rounding error δR, and the computed root μ_c is where it vanishes. A sum of n terms in doubles has a rounding error of at most about $n \, \epsilon_{mach}$ times the size of the terms. For the direct sum the terms add up to N, so $|\delta R| \lesssim n \, \epsilon_{mach} N$; for the well-conditioned form they add up to $P + H_{th} + |d|$. The Rust solver's documentation states the complete first-order bounds with generous constants (`Revision/kohn_sham/solver/src/mermin.rs`, its header; n the number of levels, g_{max} the largest degeneracy, $\langle |\varepsilon - \mu| \rangle$ the mean distance of the levels from μ weighted with $gf(1-f)$, and $\Lambda = \max(|\ln(P + d_-)|, |\ln(H_{th} + d_+)|)$):

$$B_{direct} = \epsilon_{mach} \Big[(n+2) \, \frac{N}{N'} + 3 \, \langle |\varepsilon - \mu| \rangle + T(\ln g_{max} + 3) + 2|\mu| \Big],$$

$$B_{well} = \epsilon_{mach} \Big[(n + 2 + \Lambda) \, \frac{P + H_{th} + |d|}{N'} + 3 \, \langle |\varepsilon - \mu| \rangle + T(\ln g_{max} + 3) + 2|\mu| \Big].$$

The extra terms count the rounding of the arguments x_i, of the logarithms and of the returned double. For N8_lamm1_a00_T10 ($n = 7$ Rust levels, $N' = 1.242072 \times 10^{-6}$) the first term of B_{direct} is $9 \cdot 2.22 \times 10^{-16} \cdot 8 / 1.242 \times 10^{-6} = 1.29 \times 10^{-8}$, and the record states $B_{direct} = 1.287 \times 10^{-8}$ and $B_{well} = 3.154 \times 10^{-16}$ (`Revision/kohn_sham/reports/ks-rust-mermin-roots.json`, state N8_lamm1_a00_T10). The faulty error, 8.27×10^{-10}, lies well inside B_{direct}: it is about 0.58 of the shift $\epsilon_{mach} N / N' = 1.43 \times 10^{-9}$ that a single rounding of the size $\epsilon_{mach} N$ causes. The repaired Rust value differs from the 40-digit root by 2.5×10^{-17} (same report, muRun_minus_root), inside B_{well}; over all 135 thermal states the repaired μ lies within its bound (same report, check `solver_mu_within_rounding_bound`), and in the cross-check the largest distance of a Rust μ from the 40-digit root on its own levels is 6.7×10^{-16} (`Revision/kohn_sham/reports/ks-crosscheck.json`, check `thermo_mu_rounding_diagnostic`).

Where the danger lies. The bound B_{direct} is large only where N' is tiny, that is, where a gap is many times T. Among the 45 thermal states with $N = 8$ this happens only at the first slice and the lowest temperature, where the bound reaches 10^{-8}; along the history 3-space inflates while the

extra times deflate, the 3-momenta are redshifted by $e^{-a_{4,0}}$, the levels above the gap move down, the gap closes, N' grows, and the bound falls to about 10^{-15} (Figure 16c.5). The three states with the largest direct-sum errors are the three couplings at $a_{4,0} = 0$ and $T = 0.01$: -8.27×10^{-10}, $+2.72 \times 10^{-10}$ and $+6.96 \times 10^{-11}$ (Notebook 16c, Out [11]; the same numbers are in `Revision/kohn_sham/reports/ks-rust-solver.json`, check `thermo_mu_well_conditioned_root`).

Status. PROVED: the slope formula, the uniqueness of the root, the exact rewriting $W = -d - H_{th} + P$, Newton's formula and its quadratic convergence; the rounding bounds are first-order estimates derived in the Rust documentation and confirmed against 40-digit roots for every thermal state (check `solver_mu_within_rounding_bound`). COMPUTED: every number of this section, from the records named with it and reproduced by Notebook 16c.

16.21 Example: the Mermin root (Notebook 16c)

Notebook 16c reconstructs the diagnosis and the repair from the records. For N8_lamm1_a00_T10 it reads the final levels of both solvers, computes μ with 40 digits by Newton's method, evaluates the direct residual in doubles near the root and shows the staircase, reproduces the faulty value of the first cross-check by bisection on the direct sum and the correct value by bisection on the well-conditioned form, and evaluates the rounding bounds. Then it repeats the 40-digit comparison of the cross-check for all 45 thermal states with eight quanta, for the levels of both solvers, and reproduces the recorded roots, bounds, tolerances and ratios. It needs no Rust, runs in about half a minute, draws six figures and ends with the line ALL 26 CHECKS PASSED (notebook 16c).

16.22 How to run Notebook 16c

Step 1. What this notebook does and what it needs.

Notebook 16c (How the cross-check caught a rounding error: the Mermin root) is the file `Revision/textbook/notebooks/16c_mermin_root.ipynb` of the repository Dirac_claude. The first cross-check of the two Kohn-Sham solvers failed on the chemical potential of one thermal state. This notebook reconstructs the diagnosis and the repair from the records: it computes the chemical potential of that state with 40 significant digits by Newton's method from the final levels of both solvers, shows why ordinary computer numbers locate it only to about one part in a billion when the occupations are simply added up, reproduces the faulty value of the first cross-check by bisection on that direct sum and the correct value by the well-conditioned form that both solvers use now, computes the rounding bounds that predict such errors, and repeats the 40-digit comparison of the cross-check for all 45 thermal states with eight particles, reproducing the recorded roots, bounds, tolerances and ratios. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs

the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is

changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 16c_mermin_root.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 16c_mermin_root.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 16c_mermin_root.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/16c.captions.json`

- `Revision/textbook/figures/16c_1_levels_and_occupations.png`
- `Revision/textbook/figures/16c_2_residual_staircase.png`
- `Revision/textbook/figures/16c_3_convergence_paths.png`
- `Revision/textbook/figures/16c_4_errors_versus_bound.png`
- `Revision/textbook/figures/16c_5_conditioning_map.png`
- `Revision/textbook/figures/16c_6_crosscheck_ratios.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 26 CHECKS PASSED (notebook 16c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/16c_mermin_root.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 16c_mermin_root.ipynb
python -m nbconvert --execute --inplace 16c_mermin_root.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cell that computes the 40-digit roots of the 45 states runs for half a minute or longer: this is normal. Numbers with 40 digits are computed by Python itself, not by the processor, so each operation is about a thousand times slower; the cell does about a hundred thousand of them. Wait until the label shows a number.
- An AssertionError names a comparison with a record: the reference results, the Rust results or a report were changed. Get the stored versions back and run the notebook again.

```
git checkout -- Revision/kohn_sham
```

16.23 Notebook 16c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/16c_mermin_root.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

A thermal Kohn-Sham state (temperature $T > 0$) fills its levels with the
Fermi-Dirac occupations, and one number, the **chemical potential** $\mu$, is fixed
```

by the condition that the occupations add up to the particle number N. The first cross-check of the two Kohn-Sham solvers (the Rust solver and the independent Python reference solver) FAILED on exactly this number: in the state N8_lamm1_a00_T10 the two values of μ differed by 8.27×10^{-10} (in units of the mass m), about 100 times the tolerance that had been fixed before the comparison. This notebook reconstructs the diagnosis and the repair from the records:

- it reads the final levels of that state from both solvers and computes μ again with **40 significant digits** (Newton's method, with the package mpmath); this decides which value was right;
- it shows why the ordinary numbers of a computer (about 16 digits) cannot locate μ better than about 10^{-9} when the occupations are simply added up: the computed sum is a **staircase** with steps of about 10^{-15};
- it reproduces the faulty value of the first cross-check, in all the digits the record prints, by bisection on that direct sum, and the correct value by the **well-conditioned** form of the condition that both solvers use now;
- it computes the **rounding bounds** that predict the size of such errors and reproduces those of the Rust solver's 40-digit report;
- for all 45 thermal states with $N = 8$ it repeats the cross-check's 40-digit comparison for both solvers, reproduces its table rows, and shows where the direct sum fails;
- it draws six figures.

It needs no Rust and runs in about half a minute.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 16c (How the cross-check caught a rounding error: the Mermin root) is the file ‘Revision/textbook/notebooks/16c_mermin_root.ipynb‘ of the repository Dirac_claude. The first cross-check of the two Kohn-Sham solvers failed on the chemical potential of one thermal state. This notebook reconstructs the diagnosis and the repair from the records: it computes the chemical potential of that state with 40 significant digits by Newton's method from the final levels of both solvers, shows why ordinary computer numbers locate it only to about one part in a billion when the occupations are simply added up, reproduces the faulty value of the first cross-check by bisection on that direct sum and the correct value by the well-conditioned form that both solvers use now, computes the rounding bounds that predict such errors, and repeats the 40-digit comparison of the cross-check for all 45 thermal states with eight particles, reproducing the recorded roots, bounds, tolerances and ratios. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows

PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment,

and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 16c_mermin_root.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 16c_mermin_root.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 16c_mermin_root.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/16c.captions.json‘
- ‘Revision/textbook/figures/16c_1_levels_and_occupations.png‘
- ‘Revision/textbook/figures/16c_2_residual_staircase.png‘
- ‘Revision/textbook/figures/16c_3_convergence_paths.png‘
- ‘Revision/textbook/figures/16c_4_errors_versus_bound.png‘
- ‘Revision/textbook/figures/16c_5_conditioning_map.png‘

- ‘Revision/textbook/figures/16c_6_crosscheck_ratios.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 26 CHECKS PASSED (notebook 16c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/16c_mermin_root.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 16c_mermin_root.ipynb
  python -m nbconvert --execute --inplace 16c_mermin_root.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues

  normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The cell that computes the 40-digit roots of the 45 states runs for half a minute or longer: this is normal. Numbers with 40 digits are computed by Python itself, not by the processor, so each operation is about a thousand times slower; the cell does about a hundred thousand of them. Wait until the label shows a number.
- An AssertionError names a comparison with a record: the reference results, the Rust results or a report were changed. Get the stored versions back and run the notebook again.

```
    git checkout -- Revision/kohn_sham
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 16c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 16c (How the cross-check caught a rounding error: the Mermin root) is the file
# Revision/textbook/notebooks/16c_mermin_root.ipynb of the repository Dirac_claude. The
# first cross-check of the two Kohn-Sham solvers failed on the chemical potential of one
# thermal state. This notebook reconstructs the diagnosis and the repair from the
# records: it computes the chemical potential of that state with 40 significant digits by
# Newton’s method from the final levels of both solvers, shows why ordinary computer
# numbers locate it only to about one part in a billion when the occupations are simply
# added up, reproduces the faulty value of the first cross-check by bisection on that
# direct sum and the correct value by the well-conditioned form that both solvers use
# now, computes the rounding bounds that predict such errors, and repeats the 40-digit
# comparison of the cross-check for all 45 thermal states with eight particles,
# reproducing the recorded roots, bounds, tolerances and ratios. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, mpmath and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
```

```
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
```

```
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
```

```
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 16c_mermin_root.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 1 minute
# on a typical laptop. Scroll to the end and compare the last printed lines with the
# lines in the step "What the notebook writes and what you must see" below. To keep the
# results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the
# menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 16c_mermin_root.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 16c_mermin_root.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/16c.captions.json
# - Revision/textbook/figures/16c_1_levels_and_occupations.png
# - Revision/textbook/figures/16c_2_residual_staircase.png
# - Revision/textbook/figures/16c_3_convergence_paths.png
# - Revision/textbook/figures/16c_4_errors_versus_bound.png
# - Revision/textbook/figures/16c_5_conditioning_map.png
# - Revision/textbook/figures/16c_6_crosscheck_ratios.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
```

```
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 26 CHECKS PASSED (notebook 16c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/16c_mermin_root.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 16c_mermin_root.ipynb
#     python -m nbconvert --execute --inplace 16c_mermin_root.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - The cell that computes the 40-digit roots of the 45 states runs for half a minute or
#   longer: this is normal. Numbers with 40 digits are computed by Python itself, not by
#   the processor, so each operation is about a thousand times slower; the cell does
#   about a hundred thousand of them. Wait until the label shows a number.
# - An AssertionError names a comparison with a record: the reference results, the Rust
#   results or a report were changed. Get the stored versions back and run the notebook
#   again.
#     git checkout -- Revision/kohn_sham
#
# ------------------------------------------------------------------------------------
```

```
# ============================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ============================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "16c"  # this notebook: chapter 16, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
```

```
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 16c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Level** $\varepsilon_i$: an allowed one-particle energy of the Kohn-Sham problem
  (units of the mass $m$). **Degeneracy** $g_i$: the number of states with that
  energy. In this model $g = 4\,r_3(n_2)$, where $n_2 = |\vec n|^2$ labels the shell
  of the 3-space momentum and $r_3(n_2)$ counts the whole-number points
  $(a, b, c)$ with $a^2 + b^2 + c^2 = n_2$ (the factor 4 counts four blocks of the
  spinor equation).
- **Fermi-Dirac occupation** $f(x) = 1/(1 + e^x)$ with $x = (\varepsilon - \mu)/T$:
  the average number of particles in one state; it lies between 0 and 1.
- **Chemical potential** $\mu$: the energy at which the occupation is one half.
- **Mermin condition**: $N(\mu) = \sum_i g_i\,f(x_i) = N$, the equation that fixes
  $\mu$. Its solution is called the **root**.
- **Gap**: an energy interval without levels. When the gap is much larger than $T$
  the state is in the **activated regime**: very few particles are thermally lifted
  across it.
- **Thermal holes** $H_{th} = \sum_{\varepsilon_i < \mu} g_i\,(1 - f(x_i))$ (missing
  particles below $\mu$) and **thermal particles**
  $P = \sum_{\varepsilon_i > \mu} g_i\,f(x_i)$ (particles above $\mu$).
- **Newton's method** and **bisection**: two ways to find a root, explained where
  they are used.
- **Double**: the ordinary number of a computer, with 53 binary digits (about 16
  decimal digits). **Machine epsilon** $\epsilon_{mach} = 2^{-52} \approx
  2.2\times10^{-16}$: the relative distance between neighbouring doubles near 1.
- **Rounding error**: the difference between the exact result of an operation and
  the double that the computer stores.
- **40-digit arithmetic**: numbers with 40 significant decimal digits, computed by
  the package mpmath (slow, but exact far beyond what is needed here).
- **Residual**: the left side minus the right side of an equation,
  $N(\mu) - N$; it is zero at the root.
- **Conditioning**: how strongly a small error of the residual moves the root.
  **Rounding bound**: an upper limit for the error of a computed root.
- **Tolerance** and **ratio** of the cross-check: the largest difference allowed
  between the two solvers, $3(U_{ref} + U_{Rust}) + 10^{-12}\max(1, |x|)$, and the
```

  difference divided by it; a comparison passes when the ratio is at most 1.
- **State id** such as ‘N8_lamm1_a00_T10‘: $N = 8$ particles, coupling
  $-\lambda_1$ (‘lam0‘: $\lambda = 0$; ‘lamp1‘: $+\lambda_1$; ‘lamm1‘:
  $-\lambda_1$), slice $a_{4,0} = 0$ (the digits are ten times $a_{4,0}$),
  temperature $T = 0.01$ (‘T10‘; ‘T20‘: 0.02; ‘T50‘: 0.05; units of m).

Text cell:

```
## 4. The physical and mathematical situation

**The states.** The Kohn-Sham states of the fermion field dirac16complex are computed
at five instants (slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$) of the PRESCRIBED BACKGROUND
history $a_4 = A H x_4$ of the author's metric: 3-space $x_1, x_2, x_3$ inflates with
the scale factor $e^{a_4}\sin^{1/6} z$, while the three extra times $x_5, x_6, x_7$
DEFLATE exponentially with $e^{-a_4}\sin^{1/6} z$; $x_4$ is the time and $x_0$ the
hidden direction. The 135 thermal states of the canonical matrix have
$N = 8, 136, 688$ particles, couplings $\lambda = 0, \pm\lambda_1$, the five slices
and $T = 0.01, 0.02, 0.05$. For each, both solvers compute the self-consistent levels
$\varepsilon_i$ and then $\mu$ from the Mermin condition
$$N(\mu) = \sum_i g_i\, f\!\left(\frac{\varepsilon_i - \mu}{T}\right) = N,
\qquad f(x) = \frac{1}{1 + e^x}.$$

**The slope of $N(\mu)$, line by line.**

- $f(x) = (1 + e^x)^{-1}$, so by the chain rule
  $f'(x) = -(1 + e^x)^{-2}\,e^x$.
- $\dfrac{e^x}{(1 + e^x)^2} = \dfrac{1}{1 + e^x}\cdot\dfrac{e^x}{1 + e^x}
  = f(x)\,(1 - f(x))$, because $1 - f(x) = \dfrac{e^x}{1 + e^x}$.
- So $f'(x) = -f(x)(1 - f(x))$.
- With $x = (\varepsilon - \mu)/T$ we have $dx/d\mu = -1/T$, and the chain rule gives
  $\dfrac{dN}{d\mu} = \sum_i g_i\, f'(x_i)\,\left(-\dfrac{1}{T}\right)
  = \dfrac{1}{T}\sum_i g_i\, f(x_i)(1 - f(x_i))$.
- Every term is positive, so $N(\mu)$ increases with $\mu$ and the root is unique.

**The activated regime.** In N8_lamm1_a00_T10 the eight particles exactly fill the
two lowest levels (each 4-fold, at $\varepsilon \approx 0.00025$); the next level
lies at $0.43$, so the gap is 43 times $T = 0.01$. Then $\mu$ sits near the middle
of the gap, where the few thermal holes below balance the few thermal particles
above, and $f(1 - f)$ is tiny for every level: $dN/d\mu \approx 10^{-6}$. A flat
$N(\mu)$ makes the root hard to locate: an error $\delta$ of the computed residual
moves the root by $\delta/(dN/d\mu)$ (the tangent line), a million times more.

**The history (records).** The cross-check compares every number of the two solvers
with the rule $|x_{Rust} - x_{ref}| \le 3(U_{ref} + U_{Rust}) +
10^{-12}\max(1,|x|)$, fixed in the program before any comparison. Its first run, on
a subset of the states, failed the check ‘thermo_state_functions‘ in
N8_lamm1_a00_T10: the Rust $\mu$ was $0.2100104489071649$, the reference $\mu$
$0.2100104497343054$, a difference of $8.27\times10^{-10}$ against a tolerance of
$7.97\times10^{-12}$. The tolerance was NOT widened. Instead the checker recomputed
$\mu$ with 40 digits from each solver's own levels: the two 40-digit roots agreed to
$4\times10^{-13}$, so the levels agreed and the Rust root was wrong. The Rust
solver computed $\mu$ by bisection on the direct sum $\sum g f - N$; it was repaired
(file ‘solver/src/mermin.rs‘). The reference solver had the same defect in its first
```

```
complete run (its $\mu$ missed its own 40-digit root by $1.15\times10^{-9}$ in the same
state); it was repaired the same way. This notebook repeats each step of that
diagnosis with numbers it computes itself.
```

Text cell:

```
## 5. The levels of the state N8_lamm1_a00_T10

The next cell reads, for the state N8_lamm1_a00_T10, the final levels of the Rust
solver (stored with its canonical numerics in the cross-check's measurement file
‘rust-refinement.json‘), the final levels of the reference solver (its result file
‘thermo/N8_lamm1_a00_T10.json‘; the degeneracy is computed from the shell $n_2$ as
$4\,r_3(n_2)$), and the committed values of $\mu$ of both solvers. It also reads the
numbers of the history from the two README files with a text pattern, so that the
notebook uses the recorded values, not copies typed by hand. It prints the levels
side by side and checks that both solvers have the same levels (within their
uncertainties) and the same filling.
```

In [2]:

```
import csv  # reads tables stored as CSV files (comma-separated values)
import math  # functions of single numbers (isqrt, log10, ...)
import re  # finds patterns in text (used to read numbers out of sentences)
import sys  # the list of folders in which Python looks for modules
from functools import lru_cache  # remembers the results of a function

import mpmath as mp  # numbers with as many digits as we ask for
import numpy as np  # arrays of numbers

KS = "Revision/kohn_sham"  # the folder of the Kohn-Sham record (repository path)
STATE = "N8_lamm1_a00_T10"  # the state in which the first cross-check failed
EPS_MACH = 2.0 ** -52  # machine epsilon: neighbouring doubles near 1 differ by this
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def read_csv(relative):
    """Read a CSV file of the repository: a list of rows, each row a dictionary from
    the column names to the texts in that row."""
    with open(repository_file(relative), newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle))


@lru_cache(maxsize=None)
def r3(n2):
    """The number of whole-number points (a, b, c) with a^2 + b^2 + c^2 = n2."""
    R = math.isqrt(n2) + 1  # no coordinate can be larger than this
```

```
    return sum(1 for a in range(-R, R + 1) for b in range(-R, R + 1)
               for c in range(-R, R + 1) if a * a + b * b + c * c == n2)


RUST_MU = {r["id"]: float(r["mu"])
           for r in read_csv(f"{KS}/results/thermo/thermodynamics.csv")}
REFINEMENT = {s["id"]: s for s in read_json(f"{KS}/checker/rust-refinement.json")
              ["states"] if s["kind"] == "thermo"}


def state_data(sid):
    """Everything this notebook needs about one thermal state, from the records."""
    rust = REFINEMENT[sid]  # the Rust run with the canonical numerics
    ref = read_json(f"{KS}/reference/results/thermo/{sid}.json")
    T, N = rust["canonical_T_N"]
    return {"id": sid, "T": T, "N": N,
            "eps_rust": np.array([e for e, g in rust["canonical_levels_eps_deg"]]),
            "g_rust": np.array([g for e, g in rust["canonical_levels_eps_deg"]]),
            "eps_ref": np.array(ref["levels"]["eps"]),
            "g_ref": np.array([4.0 * r3(k[0]) for k in ref["levels"]["keys"]]),
            "keys_ref": ref["levels"]["keys"],
            "U_levels_ref": max(ref["levels"]["U"]),  # largest level uncertainty
            "mu_rust": RUST_MU[sid], "mu_ref": ref["thermo"]["mu"]["value"],
            "U_mu_ref": ref["thermo"]["mu"]["U"]}


S = state_data(STATE)
print(" i  key (n2:j:parity:rank)   g   eps reference        eps Rust")
for i, key in enumerate(S["keys_ref"]):
    rust = f"{S['eps_rust'][i]:.15f}" if i < len(S["eps_rust"]) else "(not kept)"
    print(f"{i:2d}  {key[0]}:{key[1]:+d}:{key[2]}:{key[3]}"
          f"{'':12s}{S['g_ref'][i]:3.0f}   {S['eps_ref'][i]:.15f}   {rust}")
common = len(S["eps_rust"])  # the Rust label set keeps one level fewer
say(f"T = {S['T']}, N = {S['N']}; committed mu: Rust {S['mu_rust']!r}, reference "
    f"{S['mu_ref']!r}")
check(np.array_equal(S["g_rust"], S["g_ref"][:common])
      and np.max(np.abs(S["eps_rust"] - S["eps_ref"][:common])) < 1e-11,
      "both solvers have the same levels (to 1e-11) and degeneracies")
check(np.sum(S["g_rust"][S["eps_rust"] < 0.2]) == S["N"],
      "the eight particles exactly fill the two levels below the gap")

checker_history = repository_file(f"{KS}/checker/README.md").read_text(
    encoding="utf-8")
found = re.search(r"Rust.s mu was ([0-9.]+) and the reference.s\s+([0-9.]+) "
                  r"\(\|diff\| ([0-9.e-]+) against a tolerance of ([0-9.e-]+)\)",
                  checker_history)
MU_OLD_RUST, MU_OLD_REF, DIFF_OLD, TOL_OLD = (float(x) for x in found.groups())
reference_history = repository_file(f"{KS}/reference/README.md").read_text(
    encoding="utf-8")
REF_OLD_MISS = float(re.search(r"this mu differed by ([0-9.e-]+) from the 40-digit",
                               reference_history).group(1))
say(f"history: Rust mu {MU_OLD_RUST!r}, reference mu {MU_OLD_REF!r}, |diff| "
    f"{DIFF_OLD:.3g}, tolerance {TOL_OLD:.3g}; reference first run missed its root "
    f"by {REF_OLD_MISS:.3g}")
```

```
check(abs(MU_OLD_RUST - MU_OLD_REF) - DIFF_OLD < 5e-13 and DIFF_OLD > TOL_OLD,
      "the recorded difference exceeds the recorded tolerance",
      record=f"{KS}/checker/README.md, History 1")
```

Out [2]:

```
 i  key (n2:j:parity:rank)   g   eps reference        eps Rust
 0  0:+1:even:0              4   0.000245562707576   0.000245562707576
 1  0:-1:even:0              4   0.000245562707576   0.000245562707576
 2  1:+1:even:0             24   0.430761462627430   0.430761462626626
 3  2:+1:even:0             48   0.587973061749916   0.587973061748854
 4  3:+1:even:0             32   0.704111587085657   0.704111587084438
 5  4:+1:even:0             24   0.799655571340359   0.799655571339050
 6  5:+1:even:0             96   0.882327953674313   0.882327953672962
 7  6:+1:even:0             96   0.956014380953183   (not kept)
T = 0.01, N = 8.0; committed mu: Rust 0.2100104497339036, reference 0.21001044973430544
PASS both solvers have the same levels (to 1e-11) and degeneracies
PASS the eight particles exactly fill the two levels below the gap
history: Rust mu 0.2100104489071649, reference mu 0.2100104497343054, |diff| 8.27e-10,
    tolerance 7.97e-12; reference first run missed its root by 1.15e-09
PASS the recorded difference exceeds the recorded tolerance
     reproduces Revision/kohn_sham/checker/README.md, History 1
```

Text cell:

```
The next figure shows the state. Left: the occupation $f$ as a function of the
energy, a step from 1 to 0 that is $T = 0.01$ wide and sits at $\mu$, with the
levels as vertical lines; the two lowest levels are full, all others empty, and
$\mu$ lies in the middle of the gap. Right: the same occupations on a logarithmic
scale: the thermal holes $g(1 - f)$ of the two full levels and the thermal particles
$g f$ of the empty levels, at the committed $\mu$ of the reference. The holes and the
particles are each about $6\times10^{-9}$ and balance each other; the empty levels
above the first one hardly matter.
```

In [3]:

```
sys.dont_write_bytecode = True  # do not write a __pycache__ folder into the record
sys.path.insert(0, str(repository_file(f"{KS}/reference")))  # Python looks here too
import ks_fd as K  # noqa: E402  the reference solver's module (Revision code)

eps, g, T, N = S["eps_ref"], S["g_ref"], S["T"], S["N"]
mu = S["mu_ref"]
x = (eps - mu) / T
below = eps < mu
holes = g * K.fermi(-x)  # g (1 - f): f(-x) = 1 - f(x), computed without subtracting
particles = g * K.fermi(x)  # g f
fig, (left, right) = plt.subplots(1, 2, figsize=(10.5, 4.2))
energies = np.linspace(-0.05, 1.0, 2001)
left.plot(energies, K.fermi((energies - mu) / T), color=PALETTE[0], lw=1.8,
          label="occupation $f((\\varepsilon - \\mu)/T)$")
for e in np.unique(eps):  # one dotted line per energy, labelled with its states
```

```
    left.axvline(e, color="k", lw=0.8, ls=":")
    states = g[eps == e]
    label = (f"g = {states[0]:.0f}" if len(states) == 1
             else f"{len(states)} levels, g = {states[0]:.0f} each")
    left.text(e + 0.008, 0.08, label, rotation=90, fontsize=8)
left.axvline(mu, color=PALETTE[1], lw=1.4, ls="--", label=f"$\\mu = {mu:.6f}$")
left.annotate("", (eps[2], 1.12), (eps[1], 1.12), arrowprops={"arrowstyle": "<->"})
left.text(0.5 * (eps[1] + eps[2]), 1.15, "gap $= 43\\,T$", ha="center")
left.set_ylim(-0.05, 1.25)
left.set_xlabel("energy $\\varepsilon$ (units of $m$)")
left.set_ylabel("occupation $f$")
left.set_title("N8_lamm1_a00_T10: levels and occupation")
left.legend(fontsize=8, loc="center right", framealpha=1.0)
right.semilogy(eps[below], holes[below], "v", color=PALETTE[1], ms=9,
               label=f"holes $g(1-f)$, sum {np.sum(holes[below]):.3e}")
right.semilogy(eps[~below], particles[~below], "^", color=PALETTE[0], ms=9,
               label=f"particles $g f$, sum {np.sum(particles[~below]):.3e}")
right.axvline(mu, color=PALETTE[1], lw=1.4, ls="--")
right.set_xlabel("energy $\\varepsilon$ (units of $m$)")
right.set_ylabel("thermal holes and particles")
right.set_title("the balance that fixes $\\mu$")
right.legend(fontsize=8, loc="upper right")
fig.tight_layout()
save_figure(fig, "levels_and_occupations",
            "The thermal Kohn-Sham state N8_lamm1_a00_T10 ($N = 8$, $\\lambda = "
            "-\\lambda_1$, $a_{4,0} = 0$, $T = 0.01\\,m$) of the reference solver. "
            "Left: the Fermi-Dirac occupation against the energy in units of $m$, "
            "with the levels as dotted lines and their degeneracies $g$; the two "
            "lowest levels hold the eight particles, the next level lies 43 times "
            "$T$ higher, and $\\mu$ (dashed) sits in the middle of this gap. Right: "
            "the thermal holes of the full levels and the thermal particles of the "
            "empty ones on a logarithmic axis; both sums are about $6 \\times "
            "10^{-9}$ and cancel, which is the Mermin condition.")
balance = np.sum(holes[below]) - np.sum(particles[~below])
say(f"holes {np.sum(holes[below]):.6e}, particles {np.sum(particles[~below]):.6e}, "
    f"difference {balance:.2e}")
check(abs(balance) < 1e-15 and np.sum(g[below]) == N,
      "at the committed mu the thermal holes and particles balance (to 1e-15)")
```

Out [3]:

```
Figure 16c.1 saved as Revision/textbook/figures/16c_1_levels_and_occupations.png
holes 6.210359e-09, particles 6.210359e-09, difference -6.62e-24
PASS at the committed mu the thermal holes and particles balance (to 1e-15)
```

Text cell:

```
## 6. The root with 40 digits: Newton's method

**Newton's method, line by line.** Suppose $\mu_k$ is a guess for the root.

- Near $\mu_k$ the function is close to its tangent line:
```

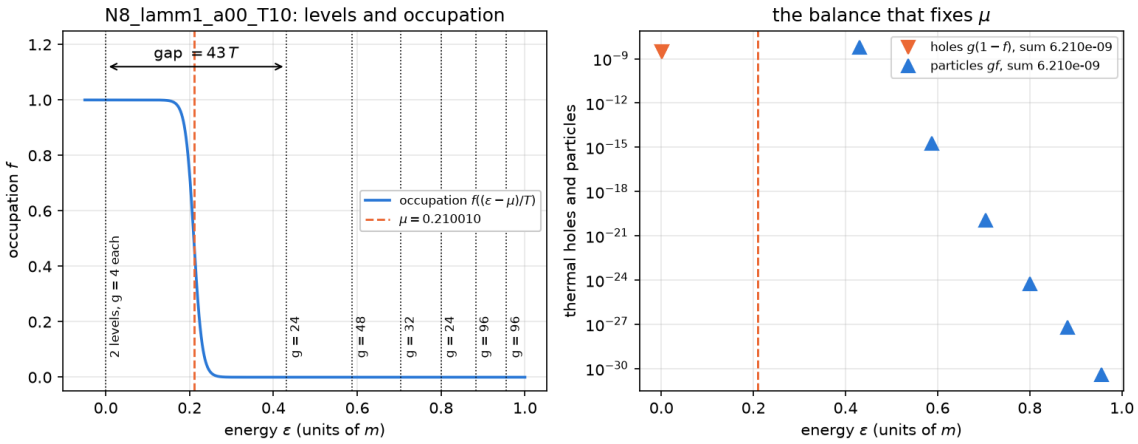

Figure 453: The thermal Kohn-Sham state N8_lamm1_a00_T10 ($N = 8$, $\lambda = -\lambda_1$, $a_{4,0} = 0$, $T = 0.01\,m$) of the reference solver. Left: the Fermi-Dirac occupation against the energy in units of m, with the levels as dotted lines and their degeneracies g; the two lowest levels hold the eight particles, the next level lies 43 times T higher, and μ (dashed) sits in the middle of this gap. Right: the thermal holes of the full levels and the thermal particles of the empty ones on a logarithmic axis; both sums are about 6×10^{-9} and cancel, which is the Mermin condition. (Notebook 16c, figure 1.)

```
  $N(\mu) \approx N(\mu_k) + N'(\mu_k)\,(\mu - \mu_k)$, where $N' = dN/d\mu$
  (section 4).
- Ask where the tangent line reaches the value $N$:
  $N(\mu_k) + N'(\mu_k)(\mu - \mu_k) = N$.
- Solve for $\mu$ (subtract $N(\mu_k)$, divide by $N'(\mu_k)$):
  $\mu_{k+1} = \mu_k - \dfrac{N(\mu_k) - N}{N'(\mu_k)}$. That is the next guess.
- Close to the root the error of the tangent line is of second order in the
  distance, so each step roughly doubles the number of correct digits
  (**quadratic convergence**).

The next cell writes Newton's method with mpmath numbers of 40 significant digits,
starts in the middle of the gap, and stops when a step is below $10^{-32}$. In 40-digit
arithmetic the rounding of the sum is about $10^{-40} N$, which moves the root by
about $10^{-40}\,N/(dN/d\mu) < 10^{-33}$: far below anything a double can hold (two
40-digit computations that add in a different order may therefore differ by about
$10^{-33}$, so the cell compares 40-digit roots to $10^{-32}$). The cell computes
the root from the Rust levels and from the reference levels and compares them with
three records: the fixture `mermin-roots-40digit.json` of the
Rust solver (made from exactly the same doubles), the checker's own 40-digit
function `mu_high_precision` (imported from `crosscheck_ks.py`), and the row
`N8_lamm1_a00_T10 mu_high_precision` of the cross-check table.
```

In [4]:

```
mp.mp.dps = 40  # every mpmath number now carries 40 significant digits


def mermin_root_40(eps, g, N, T, start):
    """Newton's method for the root mu of sum g f((eps - mu)/T) = N with 40 digits.
```

```
    Returns (the root, dN/dmu there, the list of all guesses)."""
    E = [mp.mpf(float(e)) for e in eps]  # the levels, exactly the given doubles
    G = [mp.mpf(float(x)) for x in g]
    T40, N40, mu = mp.mpf(float(T)), mp.mpf(float(N)), mp.mpf(float(start))
    guesses = [mu]
    for _ in range(60):
        total, slope = mp.mpf(0), mp.mpf(0)
        for e, gi in zip(E, G):
            x = (e - mu) / T40
            if x > 2000:  # f < e^(-2000): invisible even with 40 digits
                continue
            f = 1 / (1 + mp.exp(x))
            total += gi * f  # N(mu)
            slope += gi * f * (1 - f) / T40  # dN/dmu
        step = (total - N40) / slope  # where the tangent line reaches N
        mu -= step
        guesses.append(mu)
        if abs(step) < mp.mpf(10) ** -32:  # the first 32 digits are settled
            break
    return mu, slope, guesses


start = 0.5 * (S["eps_rust"][1] + S["eps_rust"][2])  # the middle of the gap
ROOT_R, SLOPE_R, NEWTON_PATH = mermin_root_40(S["eps_rust"], S["g_rust"], 8, 0.01,
                                              start)
ROOT_F, SLOPE_F, _ = mermin_root_40(S["eps_ref"], S["g_ref"], 8, 0.01, start)
print("Newton steps from the middle of the gap (Rust levels):")
for k, guess in enumerate(NEWTON_PATH):
    distance = mp.nstr(abs(guess - ROOT_R), 3)  # from the final root
    print(f"  mu_{k} = {mp.nstr(guess, 34)}   distance {distance}")
report("40-digit root on the Rust levels", mp.nstr(ROOT_R, 30))
report("40-digit root on the reference levels", mp.nstr(ROOT_F, 30))
report("dN/dmu at the root (Rust levels)", mp.nstr(SLOPE_R, 7))

fixture = next(s for s in read_json(f"{KS}/solver/tools/mermin-roots-40digit.json")
               ["fixture"] if s["id"] == STATE)
same_doubles = [float(e) for e, _ in fixture["levels"]] == list(S["eps_rust"])
check(same_doubles and abs(ROOT_R - mp.mpf(fixture["root40"])) < mp.mpf(10) ** -32,
      "the root on the Rust levels equals the fixture root40 to 1e-32",
      record=f"{KS}/solver/tools/mermin-roots-40digit.json, state {STATE}")

sys.path.insert(0, str(repository_file(f"{KS}/checker")))
import crosscheck_ks as CC  # noqa: E402  the cross-check program (Revision code)

checker_root, _ = CC.mu_high_precision(list(S["eps_rust"]), list(S["g_rust"]), 8.0,
                                       0.01, S["mu_rust"])
row = {r["case"]: r for r in read_csv(f"{KS}/reports/ks-crosscheck-table.csv")}[
    f"{STATE} mu_high_precision"]
say(f"table row: Rust {row['rust']}, reference {row['reference']}, |diff| "
    f"{row['abs_diff']}; ours: |diff| {mp.nstr(abs(ROOT_R - ROOT_F), 4)}")
check(checker_root == float(ROOT_R) and abs(float(ROOT_R) - float(row["rust"])) < 1e-16
      and abs(float(ROOT_F) - float(row["reference"])) < 1e-16,
      "our 40-digit roots equal the checker's function and its table row",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_mu_high_precision")
```

```
check(abs(abs(ROOT_R - ROOT_F) - mp.mpf(row["abs_diff"])) < 1e-16
      and abs(ROOT_R - ROOT_F) < 1e-12,
      "the two roots differ by only 4e-13: the levels agree, the old Rust mu did not")
```

Out [4]:

```
Newton steps from the middle of the gap (Rust levels):
  mu_0 = 0.21550351266710093822709382038738 11   distance 0.00549
  mu_1 = 0.21050351154940823024055502775625 98   distance 0.000493
  mu_2 = 0.21001084890652912944339947788981 16   distance 3.99e-7
  mu_3 = 0.21001044973390385745722899787003 48   distance 2.12e-16
  mu_4 = 0.21001044973390364543877468139392 22   distance 1.48e-34
  mu_5 = 0.21001044973390364543877468139392 2    distance 0.0
RESULT 40-digit root on the Rust levels = 0.21001044973390364543877468139 4
RESULT 40-digit root on the reference levels = 0.21001044973430543050080534941 7
RESULT dN/dmu at the root (Rust levels) = 1.242072e-6
PASS the root on the Rust levels equals the fixture root40 to 1e-32
     reproduces Revision/kohn_sham/solver/tools/mermin-roots-40digit.json, state
    N8_lamm1_a00_T10
table row: Rust 2.100104497339036e-01, reference 2.100104497343054e-01, |diff| 4.018e-13;
    ours: |diff| 4.018e-13
PASS our 40-digit roots equal the checker's function and its table row
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    thermo_mu_high_precision
PASS the two roots differ by only 4e-13: the levels agree, the old Rust mu did not
```

Text cell:

```
So the correct $\mu$ of this state is $0.21001044973390\dots$ on the Rust levels and
$0.21001044973430\dots$ on the reference levels; they differ by $4\times10^{-13}$,
which is how much the two solvers' levels differ (each is uncertain by about
$10^{-12}$). The faulty value of the first cross-check, $0.2100104489071649$, is
$8.3\times10^{-10}$ below both: the levels were right, the computation of $\mu$
from them was wrong. The next section shows why.
```

Text cell:

```
## 7. Why 16 digits are not enough: the staircase

**How a computer adds.** A double has 53 binary digits. Between 4 and 8 the doubles
are spaced $2^{2 - 52} = 2^{-50} \approx 8.9\times10^{-16}$ apart, between 8 and 16
they are $2^{-49} \approx 1.8\times10^{-15}$ apart. When the computer adds up the
occupations $\sum g f$, whose exact value is close to 8, the result is rounded to one
of these doubles. Now count how much the exact sum changes when $\mu$ moves by
$10^{-9}$: by $dN/d\mu \times 10^{-9} \approx 1.2\times10^{-6}\times10^{-9} =
1.2\times10^{-15}$, about one spacing. So in the computer the **direct residual**
$D(\mu) = \sum g f - N$ cannot follow $\mu$ smoothly: it stays constant over
intervals of about $10^{-9}$ in $\mu$ and then jumps by one spacing. It is a
**staircase**, and in particular it is exactly zero on a whole interval of $\mu$
about $10^{-9}$ wide. A root search that looks at $D$ can stop anywhere in that
interval.
```

```
The next cell evaluates, at 1201 values of $\mu$ within $3\times10^{-9}$ of the
40-digit root (on the Rust levels): the direct residual $D(\mu)$ in doubles; the
**well-conditioned residual** $W(\mu)$ of the reference module `ks_fd` (its function
`mermin_residual`, explained in section 8); and the exact residual with 40 digits.
```

In [5]:

```
eps_r, g_r = S["eps_rust"], S["g_rust"]


def direct_residual(mu, eps, g, N, T):
    """sum g f - N, added up directly in doubles (the method of the first runs)."""
    return float(np.sum(g * K.fermi((eps - mu) / T))) - N


def exact_residual(mu40, eps, g, N, T):
    """sum g f - N with 40 digits (mu40 an mpmath number)."""
    return sum(mp.mpf(float(gi)) / (1 + mp.exp((mp.mpf(float(e)) - mu40) / T))
               for e, gi in zip(eps, g)) - N


shifts = np.linspace(-3e-9, 3e-9, 1201)  # distances from the root, units of m
mu_star = float(ROOT_R)  # the double nearest to the 40-digit root
D = np.array([direct_residual(mu_star + d, eps_r, g_r, 8.0, 0.01) for d in shifts])
W = np.array([K.mermin_residual(eps_r, g_r, 8.0, 0.01, mu_star + d) for d in shifts])
X = np.array([float(exact_residual(mp.mpf(mu_star) + mp.mpf(float(d)), eps_r, g_r,
                                   8, mp.mpf("0.01"))) for d in shifts])
steps = np.unique(D)  # the different values the direct residual takes
zero = shifts[D == 0.0]  # where the computed direct residual is exactly zero
say(f"the direct residual takes {len(steps)} different values in this window: "
    + ", ".join(f"{v / 2.0 ** -50:+.0f}" for v in steps) + " times 2^-50")
say(f"it is exactly zero for mu - root from {zero.min():.3e} to {zero.max():.3e}, "
    f"an interval {zero.max() - zero.min():.2e} wide")
say(f"largest |W - exact| = {np.max(np.abs(W - X)):.1e}, largest |D - exact| = "
    f"{np.max(np.abs(D - X)):.1e}")
check(np.all(np.mod(D / 2.0 ** -50, 1.0) == 0.0) and len(steps) <= 12,
      "the direct residual is a staircase: whole multiples of 2^-50 only")
check(3e-10 < zero.max() - zero.min() < 3e-9,
      "the direct residual vanishes on an interval about 1e-9 wide")
check(np.max(np.abs(W - X)) < 1e-20,
      "the well-conditioned residual follows the exact one to 1e-20")
```

Out [5]:

```
the direct residual takes 6 different values in this window: -4, -3, -2, +0, +2, +4 times
    2^-50
it is exactly zero for mu - root from -8.250e-10 to 2.300e-10, an interval 1.05e-09 wide
largest |W - exact| = 3.9e-23, largest |D - exact| = 1.5e-15
PASS the direct residual is a staircase: whole multiples of 2^-50 only
PASS the direct residual vanishes on an interval about 1e-9 wide
PASS the well-conditioned residual follows the exact one to 1e-20
```

Text cell:

```
The next figure draws the three residuals in the window. The direct residual
(orange) moves in steps of $2^{-50}$ and $2^{-49}$; the well-conditioned residual
(blue) lies on the exact straight line through the root (black, dashed); the grey band
marks where the direct residual is exactly zero; the red line marks the faulty
$\mu$ of the first cross-check.
```

In [6]:

```
fig, ax = plt.subplots(figsize=(8.5, 5.0))
scale_mu, scale_r = 1e-9, 1e-15  # plot units: 1e-9 m and 1e-15 particles
ax.axvspan(zero.min() / scale_mu, zero.max() / scale_mu, color="0.85",
           label="direct residual exactly 0")
ax.step(shifts / scale_mu, D / scale_r, where="mid", color=PALETTE[1], lw=1.8,
        label="direct residual $D$ in doubles")
ax.plot(shifts / scale_mu, W / scale_r, color=PALETTE[0], lw=2.5, alpha=0.8,
        label="well-conditioned residual $W$ in doubles")
ax.plot(shifts / scale_mu, X / scale_r, "k--", lw=1.0, label="exact (40 digits)")
ax.axvline((MU_OLD_RUST - mu_star) / scale_mu, color=PALETTE[7], lw=1.5,
           label="mu of the first cross-check (Rust)")
ax.axhline(0.0, color="k", lw=0.6)
ax.set_xlabel("$\\mu$ minus the 40-digit root (units of $10^{-9}\\,m$)")
ax.set_ylabel("$N(\\mu) - N$ (units of $10^{-15}$)")
ax.set_title("N8_lamm1_a00_T10: the computed residual near the root")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "residual_staircase",
            "The residual $N(\\mu) - N$ of the Mermin condition of the state "
            "N8_lamm1_a00_T10 (Rust levels) within $3 \\times 10^{-9}\\,m$ of the "
            "40-digit root, horizontal axis in units of $10^{-9}\\,m$, vertical axis "
            "in units of $10^{-15}$ particles. Added up directly in ordinary computer "
            "numbers (orange) the sum near 8 can only change in steps of $2^{-50}$ "
            "or $2^{-49}$, so the residual is a staircase that is exactly zero on an "
            "interval about $10^{-9}$ wide (grey); the faulty $\\mu$ of the first "
            "cross-check (red) is the left end of that interval. The "
            "well-conditioned form (blue) follows the exact line (dashed) and "
            "crosses zero at the root.")
```

Out [6]:

```
Figure 16c.2 saved as Revision/textbook/figures/16c_2_residual_staircase.png
```

Text cell:

```
## 8. Bisection on the two forms: the faulty value, and the repair

**Bisection.** If a function is increasing, negative at $lo$ and not negative at
$hi$, its root lies between them. Take the midpoint $mid$; if the function is
negative there, the root lies above it ($lo = mid$), otherwise at or below it
($hi = mid$). Every step halves the interval. In doubles one stops when $lo$ and
```

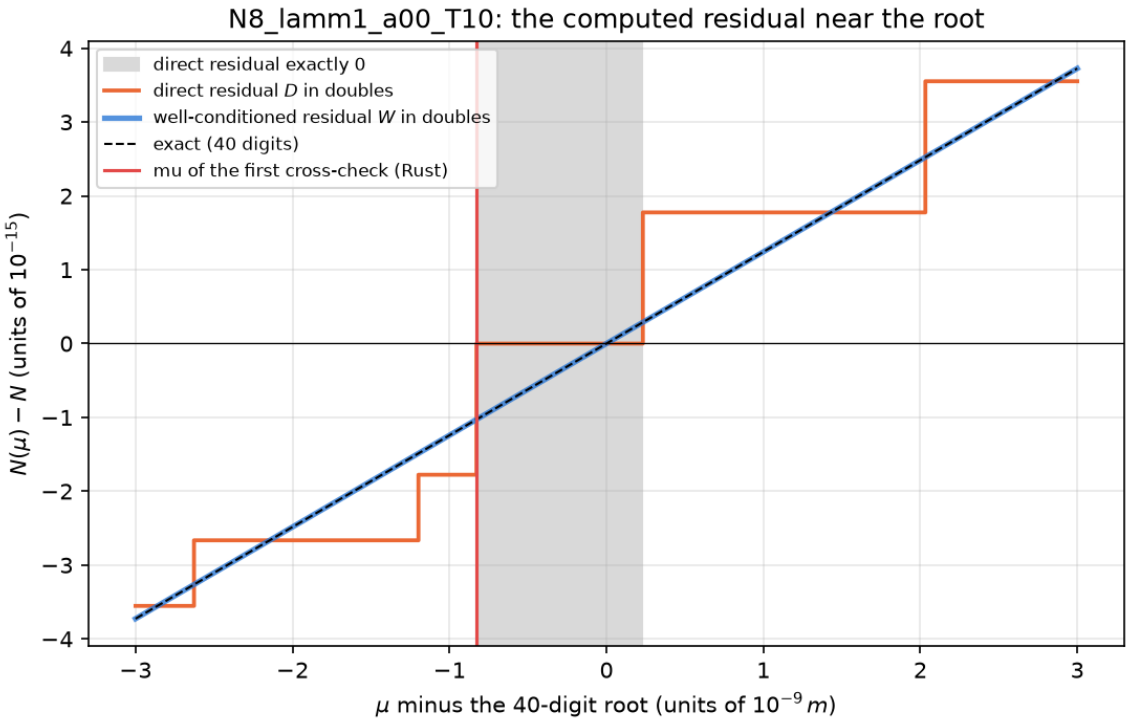

Figure 454: The residual $N(\mu) - N$ of the Mermin condition of the state N8_lamm1_a00_T10 (Rust levels) within $3 \times 10^{-9}\,m$ of the 40-digit root, horizontal axis in units of $10^{-9}\,m$, vertical axis in units of 10^{-15} particles. Added up directly in ordinary computer numbers (orange) the sum near 8 can only change in steps of 2^{-50} or 2^{-49}, so the residual is a staircase that is exactly zero on an interval about 10^{-9} wide (grey); the faulty μ of the first cross-check (red) is the left end of that interval. The well-conditioned form (blue) follows the exact line (dashed) and crosses zero at the root. (Notebook 16c, figure 2.)

```
$hi$ are neighbouring doubles (the midpoint can no longer be different from both).
This is how both solvers searched for $\mu$, starting from an interval that contains
every level.

**The well-conditioned form, line by line.** Split the sum into the levels below
$\mu$ ($x_i < 0$) and the levels above ($x_i \ge 0$).

- $f(x) + f(-x) = \dfrac{1}{1 + e^x} + \dfrac{1}{1 + e^{-x}}
  = \dfrac{1}{1 + e^x} + \dfrac{e^x}{e^x + 1} = 1$ (multiply the second fraction by
  $e^x/e^x$), so $f(x) = 1 - f(-x)$.
- Below $\mu$ write each occupation as $1 - f(-x_i)$:
  $\sum_i g_i f(x_i) - N = \sum_{below} g_i\,(1 - f(-x_i)) + \sum_{above} g_i\,f(x_i)
  - N$.
- Collect the ones:
  $W(\mu) = \Big(\sum_{below} g_i - N\Big) - \sum_{below} g_i\,f(-x_i)
  + \sum_{above} g_i\,f(x_i)$.
- The first bracket is a sum of whole numbers, which doubles add exactly. The second
  sum is $H_{th}$, the thermal holes, and the third is $P$, the thermal particles; both
  are sums of small positive numbers, each computed with a relative error of about
  $10^{-16}$. Near the root $W$ is small and is known to about $10^{-16}$ of the size
  of $H_{th}$ and $P$ (here $6\times10^{-9}$), instead of $10^{-16}$ of $N = 8$.

The value is the same as before (it is an exact rewriting); only the rounding is
different. The next cell runs bisection on both forms with the Rust levels, starting
from the interval the reference uses, and records every midpoint.
```

In [7]:

```
def bisection(residual, lo, hi):
    """Bisection for the root of an increasing function with residual(lo) < 0 and
    residual(hi) >= 0, down to neighbouring doubles; returns (root, all midpoints)."""
    midpoints = []
    while True:
        mid = 0.5 * (lo + hi)
        if mid <= lo or mid >= hi:  # lo and hi are neighbouring doubles: done
            break
        midpoints.append(mid)
        if residual(mid) < 0.0:
            lo = mid  # the root lies above mid
        else:
            hi = mid  # the root lies at or below mid
    return 0.5 * (lo + hi), midpoints


def bracket(eps, T):
    """The starting interval of the reference: every level, 60 T and 1 m around."""
    return float(eps.min()) - 60.0 * T - 1.0, float(eps.max()) + 60.0 * T + 1.0


MU_DIRECT, PATH_DIRECT = bisection(
    lambda mu: direct_residual(mu, eps_r, g_r, 8.0, 0.01), *bracket(eps_r, 0.01))
MU_WELL, PATH_WELL = bisection(
    lambda mu: K.mermin_residual(eps_r, g_r, 8.0, 0.01, mu), *bracket(eps_r, 0.01))
```

```
_, mu_ks_fd = K.mermin(eps_r, g_r, 8.0, 0.01)  # the reference's own function
say(f"bisection, direct sum:      mu = {MU_DIRECT!r} after {len(PATH_DIRECT)} "
    f"halvings; minus root {float(MU_DIRECT - ROOT_R):+.3e}")
say(f"bisection, well-conditioned: mu = {MU_WELL!r} after {len(PATH_WELL)} "
    f"halvings; minus root {float(MU_WELL - ROOT_R):+.3e}")
say(f"recorded mu of the first cross-check (Rust): {MU_OLD_RUST!r}; committed "
    f"repaired Rust mu: {S['mu_rust']!r}")
report("direct-sum bisection minus the 40-digit root",
       f"{float(MU_DIRECT - ROOT_R):.4e}", "m")
check(abs(MU_DIRECT - MU_OLD_RUST) <= 5e-17,
      "bisection on the direct sum gives the recorded faulty mu (all 16 digits)",
      record=f"{KS}/checker/README.md, History 1 (Rust mu 0.2100104489071649)")
check(MU_WELL == mu_ks_fd and abs(MU_WELL - ROOT_R) < 1e-16,
      "bisection on the well-conditioned form (ks_fd.mermin) finds the root to 1e-16")
check(abs(S["mu_rust"] - MU_WELL) < 1e-16,
      "the committed repaired Rust mu equals the well-conditioned root",
      record=f"{KS}/results/thermo/thermodynamics.csv, state {STATE}")
mu_direct_ref, _ = bisection(
    lambda mu: direct_residual(mu, S["eps_ref"], S["g_ref"], 8.0, 0.01),
    *bracket(S["eps_ref"], 0.01))
say(f"the direct sum on the reference levels misses their root by "
    f"{float(mu_direct_ref - ROOT_F):+.3e}")
```

Out [7]:

```
bisection, direct sum:      mu = 0.21001044890716491 after 57 halvings; minus root
    -8.267e-10
bisection, well-conditioned: mu = 0.21001044973390365 after 57 halvings; minus root
    +2.568e-18
recorded mu of the first cross-check (Rust): 0.2100104489071649; committed repaired Rust
    mu: 0.2100104497339036
RESULT direct-sum bisection minus the 40-digit root = -8.2674e-10 m
PASS bisection on the direct sum gives the recorded faulty mu (all 16 digits)
     reproduces Revision/kohn_sham/checker/README.md, History 1 (Rust mu
    0.2100104489071649)
PASS bisection on the well-conditioned form (ks_fd.mermin) finds the root to 1e-16
PASS the committed repaired Rust mu equals the well-conditioned root
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, state
    N8_lamm1_a00_T10
the direct sum on the reference levels misses their root by -8.271e-10
```

Text cell:

```
The direct sum also misses on the reference levels, by almost the same amount
(printed above): their levels agree with the Rust levels to about $10^{-12}$, so
their staircase has a zero interval at almost the same place, and bisection again
stops at its left end. (The reference's own first complete run missed by
$1.15\times10^{-9}$; it computed $\mu$ on each of its three grids and then combined
them, so that number is not the same computation and is not reproduced here.)

The next figure follows the three searches step by step: the distance of each guess
from the 40-digit root, on a logarithmic axis. Bisection halves the distance at every
```

```
step (a straight line on this axis); on the direct sum it stops improving near
$10^{-9}$, because from there on it only explores the zero interval of the
staircase; on the well-conditioned form it goes on down to the spacing of the doubles
near $\mu$, about $3\times10^{-17}$. Newton's method with 40 digits needs only a few
steps, and the number of correct digits roughly doubles at each.
```

In [8]:

```
fig, ax = plt.subplots(figsize=(8.5, 5.0))
floor = 1e-36  # distances that are exactly zero are drawn here
for colour, path, label, size in (
        (PALETTE[0], PATH_WELL, "bisection, well-conditioned form", 6),
        (PALETTE[1], PATH_DIRECT, "bisection, direct sum", 3)):  # drawn on top
    distance = [max(float(abs(mp.mpf(m) - ROOT_R)), floor) for m in path]
    ax.semilogy(range(1, len(path) + 1), distance, "o-", color=colour, ms=size,
                lw=1.2, label=f"{label} ({len(path)} steps)")
newton = [max(float(abs(m - ROOT_R)), floor) for m in NEWTON_PATH]
ax.semilogy(range(len(newton)), newton, "s-", color=PALETTE[2], ms=6, lw=1.5,
            label="Newton with 40 digits")
ax.axhline(abs(MU_OLD_RUST - float(ROOT_R)), color=PALETTE[7], lw=1.0, ls="--",
           label="error of the first cross-check")
ax.axhline(math.ulp(mu_star), color="k", lw=0.8, ls=":",
           label="spacing of the doubles near $\\mu$")
ax.set_ylim(floor / 3.0, 10.0)
ax.set_xlabel("step")
ax.set_ylabel("distance from the 40-digit root (units of $m$)")
ax.set_title("Three searches for $\\mu$ in N8_lamm1_a00_T10")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "convergence_paths",
            "The distance of each guess for $\\mu$ from the 40-digit root, units "
            "of $m$, logarithmic, against the step number, for the state "
            "N8_lamm1_a00_T10 with the Rust levels. Bisection halves the distance at "
            "every step; on the direct sum (orange) it stalls at $8.3 \\times "
            "10^{-10}$, the error of the first cross-check (red dashed), because "
            "the computed sum is zero on a whole interval; on the well-conditioned "
            "form (blue) it reaches the spacing of the doubles (dotted). Newton's "
            "method with 40 digits (green) doubles the number of correct digits at "
            "each step; zero distances are drawn at $10^{-36}$.")
check(min(float(abs(mp.mpf(m) - ROOT_R)) for m in PATH_DIRECT[-20:]) > 5e-10
      and float(abs(NEWTON_PATH[-2] - ROOT_R)) < 1e-32 and len(NEWTON_PATH) <= 7,
      "the direct bisection stalls above 5e-10; Newton reaches 1e-32 in 5 steps")
```

Out [8]:

```
Figure 16c.3 saved as Revision/textbook/figures/16c_3_convergence_paths.png
PASS the direct bisection stalls above 5e-10; Newton reaches 1e-32 in 5 steps
```

Text cell:

```
## 9. The rounding bounds: predicting the error
```

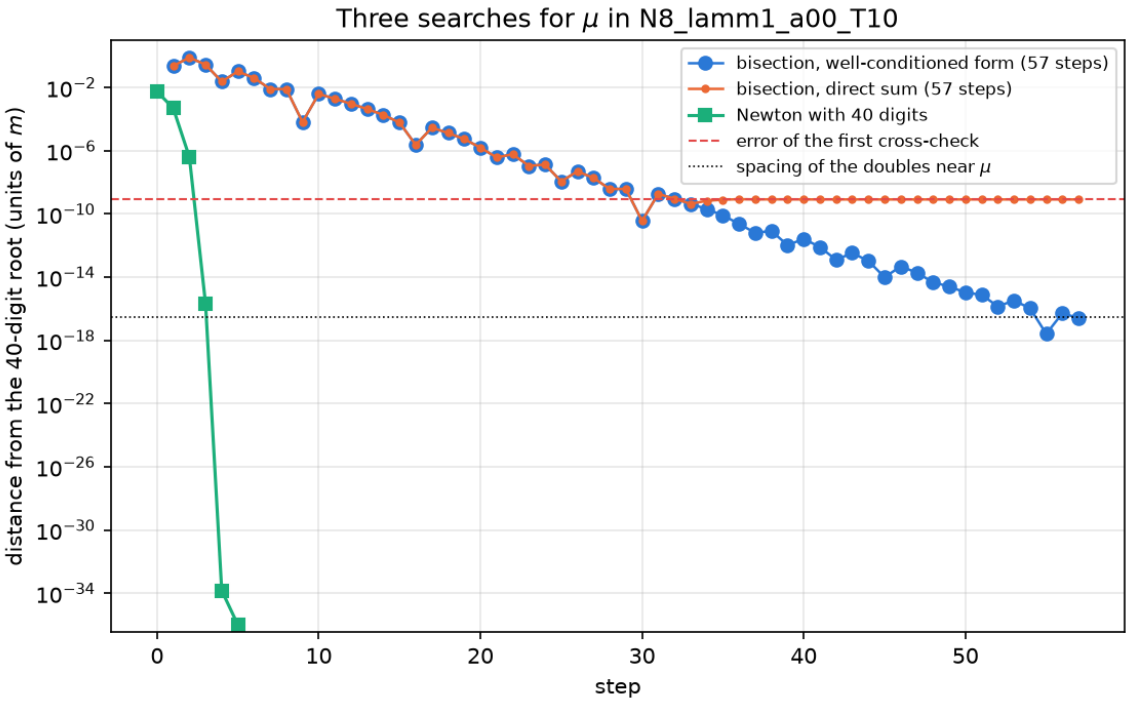

Figure 455: The distance of each guess for μ from the 40-digit root, units of m, logarithmic, against the step number, for the state N8_lamm1_a00_T10 with the Rust levels. Bisection halves the distance at every step; on the direct sum (orange) it stalls at 8.3×10^{-10}, the error of the first cross-check (red dashed), because the computed sum is zero on a whole interval; on the well-conditioned form (blue) it reaches the spacing of the doubles (dotted). Newton's method with 40 digits (green) doubles the number of correct digits at each step; zero distances are drawn at 10^{-36}. (Notebook 16c, figure 3.)

```
How large can the error of a computed root be? Line by line:

- Suppose the computed residual is off by $\delta R$ (rounding). Near the root the
  residual is the tangent line $N'(\mu - \mu_{root})$, so the computed root sits
  where $N'(\mu - \mu_{root}) + \delta R = 0$, that is $\mu - \mu_{root} =
  -\delta R/N'$.
- Adding $n$ numbers in doubles makes an error of at most about $n\,\epsilon_{mach}$
  times the size of the numbers. For the direct sum the numbers are the
  occupations, of total size $N$, so $|\delta R| \lesssim n\,\epsilon_{mach} N$ and
  $|\mu - \mu_{root}| \lesssim n\,\epsilon_{mach} N / N'$.
- For the well-conditioned form the numbers are $H_{th}$, $P$ and the whole number
  $d = N - \sum_{below} g$ (zero at the root of this state), so $N$ is replaced by
  $P + H_{th} + |d|$.

The Rust solver's documentation (file `solver/src/mermin.rs`) states the complete
bounds, with generous constants. Besides the summation they count the rounding of
the arguments $x_i = (\varepsilon_i - \mu)/T$ (the term with
$\langle|\varepsilon - \mu|\rangle$), the logarithms of the degeneracies and a few
constants (the term with $T$), the returned double (the term with $|\mu|$), and, for
the well-conditioned form, the logarithms with which the solver balances the two
sides $A = P + d_-$ and $B = H_{th} + d_+$ ($d_\pm = \max(\pm d, 0)$; at the root
$A = B$):
$$B_{direct} = \epsilon_{mach}\Big[(n + 2)\,\frac{N}{N'} + 3\,\langle|\varepsilon
- \mu|\rangle + T(\ln g_{max} + 3) + 2|\mu|\Big],$$
$$B_{well} = \epsilon_{mach}\Big[(n + 2 + \Lambda)\,\frac{P + H_{th} + |d|}{N'}
+ 3\,\langle|\varepsilon - \mu|\rangle + T(\ln g_{max} + 3) + 2|\mu|\Big],$$
where $n$ is the number of levels, $g_{max}$ the largest degeneracy,
$\Lambda = \max(|\ln A|, |\ln B|)$, and $\langle\cdot\rangle$ the average weighted with
$g f(1 - f)$. The next cell evaluates both at the root and compares them with the
entry of this state in the Rust solver's 40-digit report `ks-rust-mermin-roots.json`
(which prints 4 digits).
```

In [9]:

```
def rounding_bounds(eps, g, N, T, mu):
    """dN/dmu, the bound of the direct sum and the bound of the well-conditioned form
    at mu (the formulas of solver/src/mermin.rs), and P, H_th, d."""
    x = (eps - mu) / T
    f_plus, f_minus = K.fermi(x), K.fermi(-x)  # f(x) and 1 - f(x) = f(-x)
    weight = g * f_plus * f_minus  # g f (1 - f): how strongly a level reacts to mu
    slope = float(np.sum(weight)) / T  # dN/dmu
    mean_distance = float(np.sum(weight * np.abs(eps - mu)) / np.sum(weight))
    below = x < 0.0
    particles = float(np.sum(g[~below] * f_plus[~below]))  # P
    holes = float(np.sum(g[below] * f_minus[below]))  # H_th
    d = N - float(np.sum(g[below]))  # a whole number
    n = len(eps)
    big_l = max(abs(math.log(particles + max(-d, 0.0))),  # |ln A|
                abs(math.log(holes + max(d, 0.0))))  # |ln B|
    common = EPS_MACH * (3.0 * mean_distance + T * (math.log(float(np.max(g))) + 3.0)
                         + 2.0 * abs(mu))
    direct = EPS_MACH * (n + 2) * N / slope + common
    well = EPS_MACH * (n + 2 + big_l) * (particles + holes + abs(d)) / slope + common
```

```
    return {"slope": slope, "direct": direct, "well": well, "P": particles,
            "H_th": holes, "d": d, "n": n, "Lambda": big_l}


MERMIN_REPORT = read_json(f"{KS}/reports/ks-rust-mermin-roots.json")
RECORD = {s["id"]: s for s in MERMIN_REPORT["states"]}
b = rounding_bounds(eps_r, g_r, 8.0, 0.01, mu_star)
rec = RECORD[STATE]
say(f"n = {b['n']} levels; P = {b['P']:.6e}, H_th = {b['H_th']:.6e}, d = {b['d']:.0f}, "
    f"Lambda = {b['Lambda']:.3f}; dN/dmu = {b['slope']:.6e} (record {rec['dN_dmu']})")
say(f"B_direct = {b['direct']:.4e} (record {rec['boundDirectCount']}); B_well = "
    f"{b['well']:.4e} (record {rec['boundWellConditioned']})")
one_step = EPS_MACH * 8.0 / b["slope"]  # one rounding of size eps_mach N, as mu
say(f"one rounding of size eps_mach N moves the root by {one_step:.3e}; the error "
    f"of the first cross-check is {DIFF_OLD / one_step:.2f} of that")
check(abs(b["slope"] / float(rec["dN_dmu"]) - 1) < 1e-5
      and abs(b["direct"] / float(rec["boundDirectCount"]) - 1) < 1e-3
      and abs(b["well"] / float(rec["boundWellConditioned"]) - 1) < 1e-3,
      "dN/dmu and both rounding bounds reproduce the Rust 40-digit report",
      record=f"{KS}/reports/ks-rust-mermin-roots.json, state {STATE}")
check(abs(MU_OLD_RUST - float(ROOT_R)) <= b["direct"]
      and abs(S["mu_rust"] - float(ROOT_R)) <= b["well"] + 1e-16,
      "the old error lies within B_direct, the repaired mu within B_well")
```

Out [9]:

```
n = 7 levels; P = 6.210359e-09, H_th = 6.210359e-09, d = 0, Lambda = 18.897; dN/dmu =
    1.242072e-06 (record 1.242072e-06)
B_direct = 1.2871e-08 (record 1.287e-08); B_well = 3.1539e-16 (record 3.154e-16)
one rounding of size eps_mach N moves the root by 1.430e-09; the error of the first
    cross-check is 0.58 of that
PASS dN/dmu and both rounding bounds reproduce the Rust 40-digit report
     reproduces Revision/kohn_sham/reports/ks-rust-mermin-roots.json, state
    N8_lamm1_a00_T10
PASS the old error lies within B_direct, the repaired mu within B_well
```

Text cell:

```
So the error of the first cross-check was no accident: it has exactly the size that
the rounding of the direct sum allows. $8.27\times10^{-10}$ is about 0.6 of the shift
that a single rounding of size $\epsilon_{mach} N$ causes, and far inside
$B_{direct} \approx 1.3\times10^{-8}$. The well-conditioned form shrinks the bound
to about $3\times10^{-16}$ (printed above), a factor of about $4\times10^{7}$.
```

Text cell:

```
## 10. All 45 thermal states with eight particles

Is N8_lamm1_a00_T10 special? The next cell repeats everything for the 45 thermal
states with $N = 8$ (3 couplings, 5 slices, 3 temperatures), for the levels of both
solvers: the 40-digit root (90 roots; this takes about half a minute), $dN/d\mu$ and
```

```
the two bounds, the bisection on the direct sum and on the well-conditioned form,
and the cross-check's comparisons. It reproduces four records:

- the 40-digit roots, $dN/d\mu$ and the two bounds of `ks-rust-mermin-roots.json`
  (the level files keep 16 digits, so the roots agree to about $10^{-17}$, not
  $10^{-35}$);
- the rows `<state> mu_high_precision` of the cross-check table: both 40-digit roots,
  the tolerance with $U_{ref}$ = the largest level uncertainty of the reference and
  $U_{Rust} = \tfrac{16}{15}\times$ the largest canonical-minus-refined level
  difference, and the ratio;
- the rows `<state> mu` (check `thermo_state_functions`): the two committed values of
  $\mu$ with $U_{ref}$ of the reference's $\mu$ and $U_{Rust} = \tfrac{16}{15}|\mu_
  {canonical} - \mu_{refined}|$;
- the check `thermo_mu_rounding_diagnostic`: each committed $\mu$ lies within
  $n\,\epsilon_{mach} N/N'$ of the root on its own levels (plus $3U$ for the
  reference, whose $\mu$ is a combination of three grids).
```

In [10]:

```
TABLE = {r["case"]: r for r in read_csv(f"{KS}/reports/ks-crosscheck-table.csv")}


def tolerance(x_ref, u_ref, u_rust):
    """The tolerance rule of the cross-check for single numbers."""
    return 3.0 * (u_ref + u_rust) + 1e-12 * max(1.0, abs(x_ref))


ROWS = []  # one dictionary per state
for sid in sorted(s for s in REFINEMENT if s.startswith("N8_")):
    s = state_data(sid)
    N, T = s["N"], s["T"]
    root_r, _, _ = mermin_root_40(s["eps_rust"], s["g_rust"], N, T, s["mu_rust"])
    root_f, _, _ = mermin_root_40(s["eps_ref"], s["g_ref"], N, T, s["mu_ref"])
    r = {"id": sid, "N": N, "T": T, "root_r": root_r, "root_f": root_f,
         "b_r": rounding_bounds(s["eps_rust"], s["g_rust"], N, T, float(root_r)),
         "b_f": rounding_bounds(s["eps_ref"], s["g_ref"], N, T, float(root_f))}
    for side in ("r", "f"):  # the two bisections on each solver's levels
        e, gg = (s["eps_rust"], s["g_rust"]) if side == "r" else (s["eps_ref"],
                                                                  s["g_ref"])
        root = float(r["root_" + side])
        r["direct_" + side] = bisection(
            lambda mu: direct_residual(mu, e, gg, N, T), *bracket(e, T))[0] - root
        r["well_" + side] = bisection(
            lambda mu: K.mermin_residual(e, gg, N, T, mu), *bracket(e, T))[0] - root
    r["committed_r"] = float(mp.mpf(s["mu_rust"]) - root_r)
    r["committed_f"] = float(mp.mpf(s["mu_ref"]) - root_f)
    r["U_mu_ref"] = s["U_mu_ref"]
    u_lev_rust = 16.0 / 15.0 * REFINEMENT[sid]["levels"]["max_abs_diff"]
    u_mu_rust = 16.0 / 15.0 * REFINEMENT[sid]["scalars"]["mu"]["abs_diff"]
    tol_hp = tolerance(float(root_f), s["U_levels_ref"], u_lev_rust)
    tol_mu = tolerance(s["mu_ref"], s["U_mu_ref"], u_mu_rust)
    r["ratio_hp"] = float(abs(root_r - root_f)) / tol_hp
    r["ratio_mu"] = abs(s["mu_rust"] - s["mu_ref"]) / tol_mu
```

```
    r["tol_hp"], r["tol_mu"] = tol_hp, tol_mu
    r["n_r"], r["n_f"] = len(s["eps_rust"]), len(s["eps_ref"])
    ROWS.append(r)

print("the nine states with the largest direct-sum bound (Rust levels):")
print("state                 n   dN/dmu     B_direct   direct error  B_well")
for r in sorted(ROWS, key=lambda r: -r["b_r"]["direct"])[:9]:
    print(f"{r['id']:20s} {r['n_r']:3d}  {r['b_r']['slope']:.3e}  "
          f"{r['b_r']['direct']:.3e}  {r['direct_r']:+.3e}    {r['b_r']['well']:.3e}")
```

Out [10]:

```
the nine states with the largest direct-sum bound (Rust levels):
state                 n   dN/dmu     B_direct   direct error  B_well
N8_lamp1_a00_T10       7  1.216e-06  1.315e-08  +6.959e-11    3.155e-16
N8_lamm1_a00_T10       7  1.242e-06  1.287e-08  -8.267e-10    3.154e-16
N8_lam0_a00_T10        5  1.229e-06  1.012e-08  +2.722e-10    3.095e-16
N8_lamp1_a05_T10      13  3.302e-03  8.069e-12  -2.050e-13    2.239e-16
N8_lamm1_a05_T10      13  3.359e-03  7.932e-12  -1.527e-15    2.239e-16
N8_lam0_a05_T10        8  3.329e-03  5.337e-12  -3.366e-13    2.123e-16
N8_lamp1_a00_T20      11  2.903e-02  7.957e-13  -2.803e-15    3.628e-16
N8_lamm1_a00_T20      11  2.934e-02  7.872e-13  -1.768e-14    3.627e-16
N8_lam0_a00_T20        8  2.919e-02  6.089e-13  +1.621e-14    3.484e-16
```

Text cell:

The next cell makes the checks against the four records and against the bounds.

In [11]:

```
worst = {"root": 0.0, "slope": 0.0, "bounds": 0.0, "hp": 0.0, "tol": 0.0,
         "ratio": 0.0}
for r in ROWS:
    rec = RECORD[r["id"]]
    worst["root"] = max(worst["root"], float(abs(r["root_r"] - mp.mpf(rec["root40"]))))
    worst["slope"] = max(worst["slope"],
                         abs(r["b_r"]["slope"] / float(rec["dN_dmu"]) - 1))
    worst["bounds"] = max(worst["bounds"],
                          abs(r["b_r"]["direct"] / float(rec["boundDirectCount"]) - 1),
                          abs(r["b_r"]["well"] / float(rec["boundWellConditioned"])
                              - 1))
    hp, mu_row = TABLE[f"{r['id']} mu_high_precision"], TABLE[f"{r['id']} mu"]
    worst["hp"] = max(worst["hp"], abs(float(r["root_r"]) - float(hp["rust"])),
                      abs(float(r["root_f"]) - float(hp["reference"])))
    worst["tol"] = max(worst["tol"], abs(r["tol_hp"] / float(hp["tolerance"]) - 1),
                       abs(r["tol_mu"] / float(mu_row["tolerance"]) - 1))
    worst["ratio"] = max(worst["ratio"], abs(r["ratio_hp"] - float(hp["ratio"])),
                         abs(r["ratio_mu"] - float(mu_row["ratio"])))
say("largest differences from the records: " + ", ".join(
    f"{k} {v:.1e}" for k, v in worst.items()))
check(worst["root"] < 1e-16,
```

```
      "the 45 roots on the Rust levels reproduce root40 (to the 16-digit levels)",
      record=f"{KS}/reports/ks-rust-mermin-roots.json, states (root40)")
check(worst["slope"] < 1e-5 and worst["bounds"] < 1e-3,
      "dN/dmu and both bounds of all 45 states reproduce the Rust report",
      record=f"{KS}/reports/ks-rust-mermin-roots.json, states (bounds)")
check(worst["hp"] < 2e-16 and worst["tol"] < 1e-3 and worst["ratio"] <= 1e-4,
      "both 40-digit roots, tolerances and ratios reproduce 90 table rows",
      record=f"{KS}/reports/ks-crosscheck-table.csv, rows mu, mu_high_precision")
check(all(r["ratio_hp"] <= 1.0 and r["ratio_mu"] <= 1.0 for r in ROWS),
      "after the repair every mu comparison of the 45 states passes",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_state_functions")
diag_r = [abs(r["committed_r"]) / (r["n_r"] * EPS_MACH * r["N"] / r["b_r"]["slope"])
          for r in ROWS]
diag_f = [abs(r["committed_f"]) / (r["n_f"] * EPS_MACH * r["N"] / r["b_f"]["slope"]
                                   + 3.0 * r["U_mu_ref"]) for r in ROWS]
say(f"committed mu minus root, as a fraction of the diagnostic bound: Rust at most "
    f"{max(diag_r):.1e}, reference at most {max(diag_f):.1e}")
check(max(diag_r) <= 1.0 and max(diag_f) <= 1.0,
      "both solvers' committed mu lie within the rounding diagnostic",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_mu_rounding_diagnostic")
check(all(abs(r["direct_" + s]) <= r["b_" + s]["direct"]
          and abs(r["well_" + s]) <= r["b_" + s]["well"] for r in ROWS
          for s in ("r", "f")),
      "in all 90 level sets each bisection lies within its own bound")
big = sorted(ROWS, key=lambda r: -abs(r["direct_r"]))[:3]
say("largest direct-sum errors: " + ", ".join(
    f"{r['id']} {r['direct_r']:+.2e}" for r in big))
check({r["id"] for r in big} == {"N8_lam0_a00_T10", "N8_lamm1_a00_T10",
                                 "N8_lamp1_a00_T10"},
      "the direct sum fails worst in the three states with the largest gap/T")
```

Out [11]:

```
largest differences from the records: root 2.8e-17, slope 4.5e-07, bounds 4.2e-04, hp
    5.6e-17, tol 3.9e-04, ratio 5.0e-05
PASS the 45 roots on the Rust levels reproduce root40 (to the 16-digit levels)
     reproduces Revision/kohn_sham/reports/ks-rust-mermin-roots.json, states (root40)
PASS dN/dmu and both bounds of all 45 states reproduce the Rust report
     reproduces Revision/kohn_sham/reports/ks-rust-mermin-roots.json, states (bounds)
PASS both 40-digit roots, tolerances and ratios reproduce 90 table rows
     reproduces Revision/kohn_sham/reports/ks-crosscheck-table.csv, rows mu,
    mu_high_precision
PASS after the repair every mu comparison of the 45 states passes
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    thermo_state_functions
committed mu minus root, as a fraction of the diagnostic bound: Rust at most 1.3e-02,
    reference at most 1.4e-05
PASS both solvers' committed mu lie within the rounding diagnostic
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    thermo_mu_rounding_diagnostic
PASS in all 90 level sets each bisection lies within its own bound
largest direct-sum errors: N8_lamm1_a00_T10 -8.27e-10, N8_lam0_a00_T10 +2.72e-10,
    N8_lamp1_a00_T10 +6.96e-11
```

```
PASS the direct sum fails worst in the three states with the largest gap/T
```

Text cell:

```
The next figure collects the errors of all 180 computed roots (45 states, two
solvers' levels, two methods; plus the committed values) against the
conditioning scale $\epsilon_{mach} N/N'$, the shift of the root caused by one
rounding of size $\epsilon_{mach} N$. The direct sum (orange and red) scatters
below the line of slope 1, reaching it where the conditioning is worst; the
well-conditioned form and the committed values (blue and green) stay near
$10^{-17}$ whatever the conditioning. Errors that are exactly zero are drawn at
$10^{-19}$.
```

In [12]:

```
scale = np.array([EPS_MACH * r["N"] / r["b_r"]["slope"] for r in ROWS])
scale_f = np.array([EPS_MACH * r["N"] / r["b_f"]["slope"] for r in ROWS])
fig, ax = plt.subplots(figsize=(8.0, 6.0))
low = 1e-19  # exact zeros are drawn here
series = [(scale, "direct_r", "o", PALETTE[1], "direct sum, Rust levels"),
          (scale_f, "direct_f", "s", PALETTE[7], "direct sum, reference levels"),
          (scale, "well_r", "o", PALETTE[0], "well-conditioned, Rust levels"),
          (scale_f, "well_f", "s", PALETTE[6], "well-conditioned, reference levels"),
          (scale, "committed_r", "^", PALETTE[2], "committed Rust mu")]
for xs, key, marker, colour, label in series:
    ys = np.maximum([abs(r[key]) for r in ROWS], low)
    ax.loglog(xs, ys, marker, color=colour, ms=5, alpha=0.8, label=label)
line = np.array([1e-16, 1e-8])
ax.loglog(line, line, "k-", lw=1.0, label="error = $\\epsilon_{mach} N / N'$")
i_old = [r["id"] for r in ROWS].index(STATE)
ax.loglog([scale[i_old]], [DIFF_OLD], "*", color="k", ms=14,
          label="first cross-check (recorded)")
ax.set_xlabel("$\\epsilon_{mach} N / (dN/d\\mu)$ (units of $m$)")
ax.set_ylabel("$|\\mu$ computed $-$ 40-digit root$|$ (units of $m$)")
ax.set_title("45 thermal states with $N = 8$: error against conditioning")
ax.legend(fontsize=7, loc="upper left")
save_figure(fig, "errors_versus_bound",
            "Errors of the computed chemical potential, in units of $m$, against "
            "the conditioning scale $\\epsilon_{mach} N/(dN/d\\mu)$ for the 45 "
            "thermal states with $N = 8$, both axes logarithmic. Bisection on the "
            "direct sum (orange: Rust levels, red: reference levels) has errors "
            "that grow with the conditioning scale and approach the line of slope 1 "
            "in the worst states; the star is the recorded error of the first "
            "cross-check. The well-conditioned form (blue, purple) and the "
            "committed repaired Rust values (green) stay near $10^{-17}$ for every "
            "state; exact zeros are drawn at $10^{-19}$.")
```

Out [12]:

```
Figure 16c.4 saved as Revision/textbook/figures/16c_4_errors_versus_bound.png
```

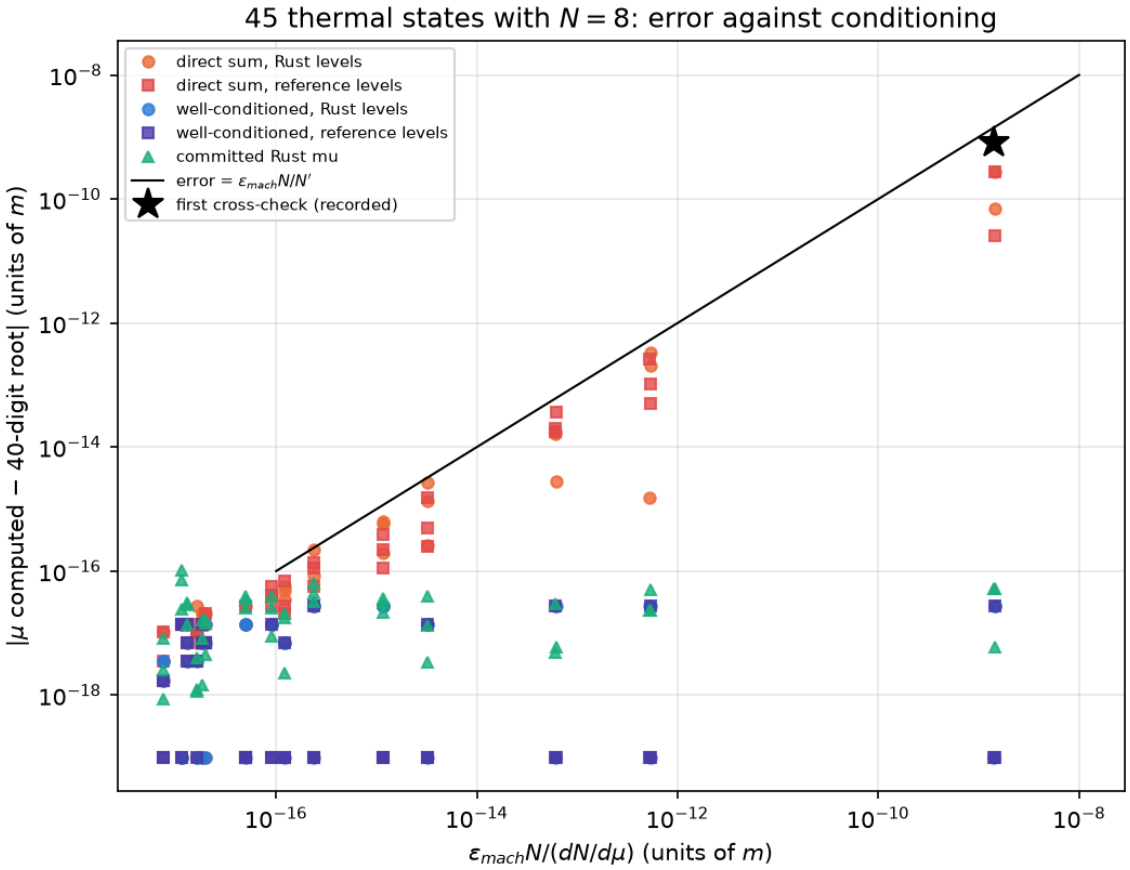

Figure 456: Errors of the computed chemical potential, in units of m, against the conditioning scale $\epsilon_{mach} N/(dN/d\mu)$ for the 45 thermal states with $N = 8$, both axes logarithmic. Bisection on the direct sum (orange: Rust levels, red: reference levels) has errors that grow with the conditioning scale and approach the line of slope 1 in the worst states; the star is the recorded error of the first cross-check. The well-conditioned form (blue, purple) and the committed repaired Rust values (green) stay near 10^{-17} for every state; exact zeros are drawn at 10^{-19}. (Notebook 16c, figure 4.)

Text cell:

```
Where in the canonical matrix is the direct sum dangerous? The next figure shows
$B_{direct}$ of the 45 states as three maps (one per coupling), with the slice
$a_{4,0}$ downwards and the temperature across; the colour and the number in each
square give $\log_{10} B_{direct}$. At $a_{4,0} = 0$ and $T = 0.01$ the gap is 43
times $T$ and the bound is $10^{-8}$. Along the history 3-space inflates and the
extra times deflate: the 3-momenta are redshifted by $e^{-a_{4,0}}$, the levels above
the gap move down, the gap shrinks, $dN/d\mu$ grows, and the bound falls towards
$10^{-15}$.
```

In [13]:

```
tags = [("lam0", "$\\lambda = 0$"), ("lamp1", "$\\lambda = +\\lambda_1$"),
        ("lamm1", "$\\lambda = -\\lambda_1$")]
slices, temps = ["a00", "a05", "a10", "a15", "a20"], ["T10", "T20", "T50"]
by_id = {r["id"]: r for r in ROWS}
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.6))
for ax, (tag, title) in zip(axes, tags):
    grid = np.array([[math.log10(by_id[f"N8_{tag}_{a}_{t}"]["b_r"]["direct"])
                      for t in temps] for a in slices])
    image = ax.imshow(grid, cmap="viridis", vmin=-15.0, vmax=-7.5, aspect="auto")
    ax.grid(False)  # no grid lines across the coloured squares
    for i in range(5):
        for j in range(3):
            ax.text(j, i, f"{grid[i, j]:.1f}", ha="center", va="center",
                    color="w" if grid[i, j] < -11.5 else "k", fontsize=9)
    ax.set_xticks(range(3))
    ax.set_xticklabels(["0.01", "0.02", "0.05"])
    ax.set_yticks(range(5))
    ax.set_yticklabels(["0", "0.5", "1", "1.5", "2"])
    ax.set_xlabel("temperature $T$ (units of $m$)")
    ax.set_title(title)
axes[0].set_ylabel("slice $a_{4,0}$")
fig.colorbar(image, ax=axes, shrink=0.9, label="$\\log_{10} B_{direct}$ (units of $m$)")
save_figure(fig, "conditioning_map",
            "The rounding bound $B_{direct}$ of the direct sum, as $\\log_{10}$ of "
            "its value in units of $m$, for the 45 thermal states with $N = 8$ (Rust "
            "levels): one map per coupling, the slice $a_{4,0}$ of the deflating "
            "history downwards, the temperature across. The direct sum is dangerous "
            "only at low temperature early in the history, where the gap is 43 "
            "times $T$ and the bound reaches $10^{-8}$; later the redshift of the "
            "3-momenta closes the gap and the bound falls to about $10^{-15}$.")
check(max(by_id[f"N8_{t}_a00_T10"]["b_r"]["direct"] for t, _ in tags)
      == max(r["b_r"]["direct"] for r in ROWS)
      and all(by_id[f"N8_{t}_a00_T10"]["b_r"]["direct"]
              > 100.0 * by_id[f"N8_{t}_a10_T10"]["b_r"]["direct"] for t, _ in tags),
      "the bound is largest at a4,0 = 0, T = 0.01 and falls along the history")
```

Out [13]:

```
Figure 16c.5 saved as Revision/textbook/figures/16c_5_conditioning_map.png
PASS the bound is largest at a4,0 = 0, T = 0.01 and falls along the history
```

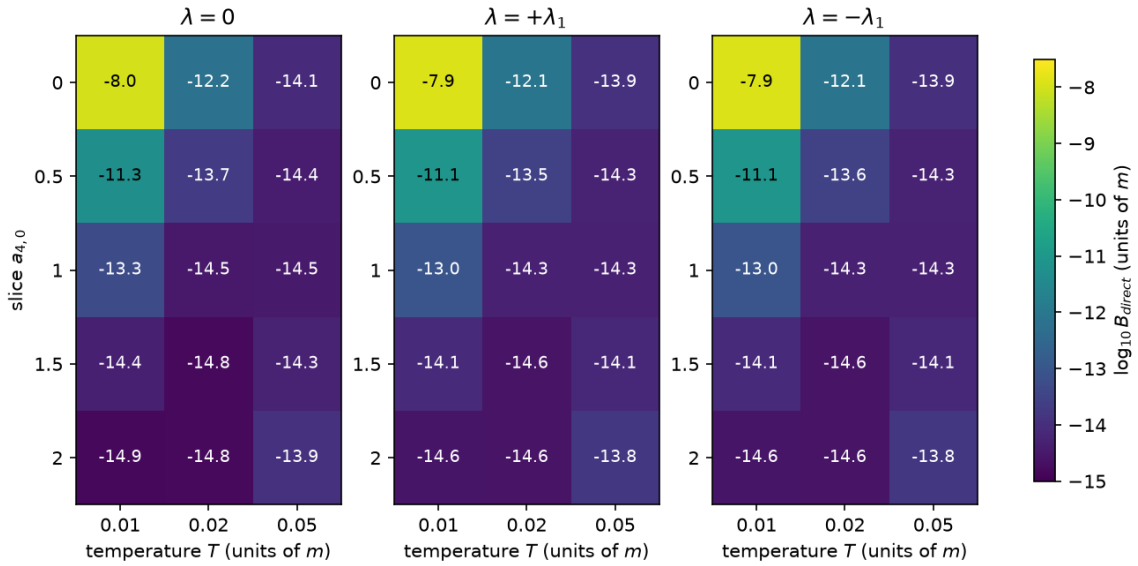

Figure 457: The rounding bound B_{direct} of the direct sum, as $\log_{10}$ of its value in units of m, for the 45 thermal states with $N = 8$ (Rust levels): one map per coupling, the slice $a_{4,0}$ of the deflating history downwards, the temperature across. The direct sum is dangerous only at low temperature early in the history, where the gap is 43 times T and the bound reaches 10^{-8}; later the redshift of the 3-momenta closes the gap and the bound falls to about 10^{-15}. (Notebook 16c, figure 5.)

Text cell:

```
## 11. What the tolerance rule did

The last figure is the cross-check's view. For each of the 45 states it shows the
ratio of the $\mu$ comparison (‘thermo_state_functions‘, circles) and of the
40-digit comparison (‘thermo_mu_high_precision‘, squares), both computed above, and
the ratio of the failed first comparison, $8.27\times10^{-10}/7.97\times10^{-12}$
(red star). The rule was fixed before the comparison and was not changed afterwards:
the failure was explained (the 40-digit roots), the program was repaired, and the
same rule then passed.
```

In [14]:

```
order = sorted(range(len(ROWS)), key=lambda i: -ROWS[i]["b_r"]["direct"])
ratio_old = DIFF_OLD / TOL_OLD  # the ratio of the first, failed comparison
fig, ax = plt.subplots(figsize=(9.5, 4.8))
positions = np.arange(len(order))
ax.semilogy(positions, [ROWS[i]["ratio_mu"] for i in order], "o", color=PALETTE[0],
            ms=5, label="$\\mu$ (committed values), after the repair")
ax.semilogy(positions, [ROWS[i]["ratio_hp"] for i in order], "s", color=PALETTE[2],
            ms=5, mfc="none", label="$\\mu$ with 40 digits on each solver's levels")
k_old = order.index([r["id"] for r in ROWS].index(STATE))
ax.semilogy([k_old], [ratio_old], "*", color=PALETTE[7], ms=16,
            label=f"first cross-check, before the repair: {ratio_old:.0f}")
ax.axhline(1.0, color="k", lw=1.2, label="ratio 1: the tolerance")
```

```
ax.set_xticks(positions[::4])
ax.set_xticklabels([ROWS[i]["id"].replace("N8_", "") for i in order][::4],
                   rotation=40, ha="right", fontsize=7)
ax.set_xlabel("state (sorted by the bound of the direct sum, largest first)")
ax.set_ylabel("$|\\mu_{Rust} - \\mu_{ref}|$ / tolerance")
ax.set_ylim(1e-4, 1e3)
ax.legend(fontsize=8, loc="upper right")
fig.tight_layout()
save_figure(fig, "crosscheck_ratios",
            "The cross-check of the chemical potential for the 45 thermal states with "
            "$N = 8$: the difference of the two solvers divided by the tolerance "
            "fixed in advance, logarithmic, the states sorted by the rounding bound "
            "of the direct sum. Circles: the committed values after the repair; "
            "squares: the 40-digit roots on each solver's levels; all lie far below "
            "the line 1. The star is the first comparison in N8_lamm1_a00_T10, "
            f"before the repair, with the ratio {ratio_old:.0f}: the rule caught a "
            "rounding error of the Rust solver, and the repair, not a wider "
            "tolerance, removed it.")
report_detail = next(c["detail"] for c in read_json(
    f"{KS}/reports/ks-crosscheck.json")["checks"]
    if c["name"] == "thermo_mu_high_precision")
worst_all = float(re.search(r"worst \|diff\|/tolerance ([0-9.]+)",
                            report_detail).group(1))
say(f"first comparison: ratio {ratio_old:.1f}. After the repair, over these 45 "
    f"states: mu at most {max(r['ratio_mu'] for r in ROWS):.4f}, 40-digit roots at "
    f"most {max(r['ratio_hp'] for r in ROWS):.4f} (the report, over all 135 states: "
    f"40-digit roots at most {worst_all})")
check(ratio_old > 1.0 and max(r["ratio_hp"] for r in ROWS) <= worst_all + 5e-4
      and max(r["ratio_mu"] for r in ROWS) < 0.1,
      "the first comparison failed the rule; after the repair all ratios are small",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_mu_high_precision")
```

Out [14]:

```
Figure 16c.6 saved as Revision/textbook/figures/16c_6_crosscheck_ratios.png
first comparison: ratio 103.8. After the repair, over these 45 states: mu at most 0.0825,
    40-digit roots at most 0.0270 (the report, over all 135 states: 40-digit roots at
    most 0.05)
PASS the first comparison failed the rule; after the repair all ratios are small
     reproduces Revision/kohn_sham/reports/ks-crosscheck.json, check
    thermo_mu_high_precision
```

Text cell:

```
## 12. The last check

The next cell confirms that every figure of this notebook was written, and prints
the number of checks that passed.
```

In [15]:

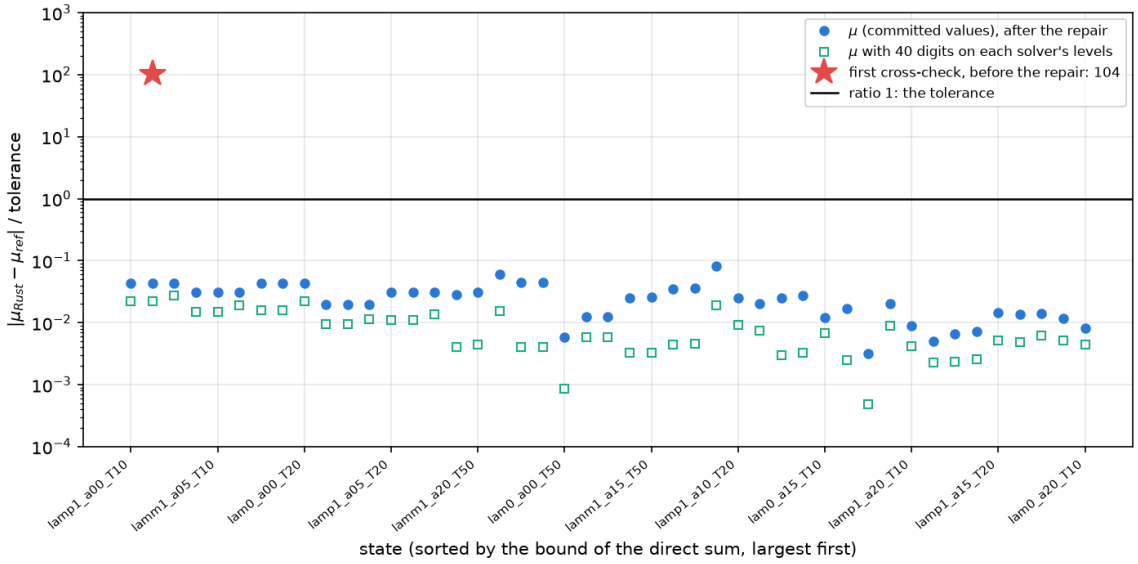

Figure 458: The cross-check of the chemical potential for the 45 thermal states with $N = 8$: the difference of the two solvers divided by the tolerance fixed in advance, logarithmic, the states sorted by the rounding bound of the direct sum. Circles: the committed values after the repair; squares: the 40-digit roots on each solver's levels; all lie far below the line 1. The star is the first comparison in N8_lamm1_a00_T10, before the repair, with the ratio 104: the rule caught a rounding error of the Rust solver, and the repair, not a wider tolerance, removed it. (Notebook 16c, figure 6.)

```
figure_files = [f"{FIGURE_FOLDER}/16c_{k}_{name}.png" for k, name in enumerate(
    ["levels_and_occupations", "residual_staircase", "convergence_paths",
     "errors_versus_bound", "conditioning_map", "crosscheck_ratios"], start=1)]
check(all(output_file(f).is_file() for f in figure_files),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [15]:

```
PASS every figure file of this notebook exists
ALL 26 CHECKS PASSED (notebook 16c)
```

Text cell:

```
## 13. What this notebook showed

- The chemical potential of a thermal state is the root of
  $\sum g f((\varepsilon - \mu)/T) = N$; its slope $dN/d\mu = \sum g f(1 - f)/T$ is
  tiny when the gap is many times $T$ (activated regime), so a small error of the
  residual moves the root a lot.
- With 40 digits (Newton's method) the root of N8_lamm1_a00_T10 is
  $0.21001044973390\dots$ on the Rust levels and differs by $4\times10^{-13}$ on the
  reference levels; this reproduces the cross-check table, the checker's own
  function and the Rust fixture.
```

```
- Added up directly in doubles, the residual is a staircase in steps of $2^{-50}$
  that is exactly zero on an interval about $10^{-9}$ wide; bisection on it returns
  the recorded faulty value $0.2100104489071649$ of the first cross-check in all the
  digits the record prints.
- The exact rewriting $W = (\sum_{below} g - N) - H_{th} + P$ is computed with relative
  precision; bisection on it gives the root to $10^{-16}$, the committed repaired
  values of both solvers.
- The rounding bounds of the Rust documentation predict both errors
  ($B_{direct} \approx 1.3\times10^{-8}$, $B_{well} \approx 3\times10^{-16}$ for
  this state); our evaluation of them reproduces the Rust 40-digit report for all
  45 states with $N = 8$; the direct sum is dangerous only early in the history and
  at low $T$.
- The cross-check's tolerance, fixed in advance, caught the error (ratio about
  100); after the repair every ratio of the chemical potential in these 45 states is
  below 0.1, with the same rule.
- What this does NOT show: the 40-digit roots test the computation of $\mu$ from
  given levels. The levels themselves, the functional, the ASSUMED $Z_2$ brane, the
  filling CONVENTION and the PRESCRIBED BACKGROUND history are inputs that both
  solvers share; this notebook does not test them.
```

16.24 Line-by-line walk-through of Notebook 16c

The notebook has 15 code cells, In [1] to In [15]; the numbers they print are in Section 16.23.

In [1], the set-up cell. As in Notebook 16b, it is the set-up cell of Notebook 16a (Section 16.12) without the Rust helper; its comment lines repeat the run instructions of Section 16.22, and its name line is

```
NOTEBOOK_ID = "16c"  # this notebook: chapter 16, example c
```

In [2], the levels of N8_lamm1_a00_T10 and the history.

```
import csv  # reads tables stored as CSV files (comma-separated values)
import math  # functions of single numbers (isqrt, log10, ...)
import re  # finds patterns in text (used to read numbers out of sentences)
import sys  # the list of folders in which Python looks for modules
from functools import lru_cache  # remembers the results of a function

import mpmath as mp  # numbers with as many digits as we ask for
import numpy as np  # arrays of numbers

KS = "Revision/kohn_sham"  # the folder of the Kohn-Sham record (repository path)
STATE = "N8_lamm1_a00_T10"  # the state in which the first cross-check failed
EPS_MACH = 2.0 ** -52  # machine epsilon: neighbouring doubles near 1 differ by this
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

Besides the modules of Notebook 16a, `math` gives functions of single numbers (here `isqrt`, the whole-number square root, `log`, `log10` and `ulp`), and `lru_cache` from the module `functools` makes a function remember its results. `mpmath`, called `mp`, computes with as many digits as one asks for.

STATE is the state of the failed comparison, and EPS_MACH is the machine epsilon 2^{-52} (** is the power).

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def read_csv(relative):
    """Read a CSV file of the repository: a list of rows, each row a dictionary from
    the column names to the texts in that row."""
    with open(repository_file(relative), newline="", encoding="utf-8") as handle:
        return list(csv.DictReader(handle))
```

The two reading functions of Notebook 16a (Section 16.12, In [2]).

```
@lru_cache(maxsize=None)
def r3(n2):
    """The number of whole-number points (a, b, c) with a^2 + b^2 + c^2 = n2."""
    R = math.isqrt(n2) + 1  # no coordinate can be larger than this
    return sum(1 for a in range(-R, R + 1) for b in range(-R, R + 1)
               for c in range(-R, R + 1) if a * a + b * b + c * c == n2)
```

r3(n2) counts the points (a, b, c) with whole-number coordinates and $a^2 + b^2 + c^2 = n_2$, the number $r_3(n_2)$ of Chapter 15. No coordinate can be larger than $\sqrt{n_2}$, so the three nested ranges from $-R$ to R with $R = \lfloor\sqrt{n_2}\rfloor + 1$ cover all candidates, and sum(1 for ... if ...) counts those that satisfy the equation. The line @lru_cache(maxsize=None) above the function is a **decorator**: it wraps the function so that a result, once computed for some n_2, is remembered and returned at once the next time.

```
RUST_MU = {r["id"]: float(r["mu"])
           for r in read_csv(f"{KS}/results/thermo/thermodynamics.csv")}
REFINEMENT = {s["id"]: s for s in read_json(f"{KS}/checker/rust-refinement.json")
              ["states"] if s["kind"] == "thermo"}
```

RUST_MU files the committed Rust chemical potentials of all thermal states under their ids (from the column mu of thermodynamics.csv). REFINEMENT files the thermal entries of the cross-check's measurement file rust-refinement.json; each holds, among other things, the final levels and degeneracies of the canonical Rust run, from which the checker computed its 40-digit roots.

```
def state_data(sid):
    """Everything this notebook needs about one thermal state, from the records."""
    rust = REFINEMENT[sid]  # the Rust run with the canonical numerics
    ref = read_json(f"{KS}/reference/results/thermo/{sid}.json")
    T, N = rust["canonical_T_N"]
    return {"id": sid, "T": T, "N": N,
            "eps_rust": np.array([e for e, g in rust["canonical_levels_eps_deg"]]),
            "g_rust": np.array([g for e, g in rust["canonical_levels_eps_deg"]]),
            "eps_ref": np.array(ref["levels"]["eps"]),
```

```
            "g_ref": np.array([4.0 * r3(k[0]) for k in ref["levels"]["keys"]]),
            "keys_ref": ref["levels"]["keys"],
            "U_levels_ref": max(ref["levels"]["U"]),  # largest level uncertainty
            "mu_rust": RUST_MU[sid], "mu_ref": ref["thermo"]["mu"]["value"],
            "U_mu_ref": ref["thermo"]["mu"]["U"]}
```

`state_data(sid)` collects everything the notebook needs about one thermal state: T and N; the Rust levels and degeneracies (two list comprehensions over the pairs (energy, degeneracy)); the reference levels from its result file, with the degeneracies computed as $4r_3(n_2)$ from each level's shell (`k[0]` is the first entry of the level's key); the keys; the largest level uncertainty of the reference; and the committed μ of both solvers with the reference's uncertainty of μ.

```
S = state_data(STATE)
print(" i  key (n2:j:parity:rank)   g   eps reference        eps Rust")
for i, key in enumerate(S["keys_ref"]):
    rust = f"{S['eps_rust'][i]:.15f}" if i < len(S["eps_rust"]) else "(not kept)"
    print(f"{i:2d}  {key[0]}:{key[1]:+d}:{key[2]}:{key[3]}"
          f"{'':12s}{S['g_ref'][i]:3.0f}   {S['eps_ref'][i]:.15f}   {rust}")
common = len(S["eps_rust"])  # the Rust label set keeps one level fewer
say(f"T = {S['T']}, N = {S['N']}; committed mu: Rust {S['mu_rust']!r}, reference "
    f"{S['mu_ref']!r}")
check(np.array_equal(S["g_rust"], S["g_ref"][:common])
      and np.max(np.abs(S["eps_rust"] - S["eps_ref"][:common])) < 1e-11,
      "both solvers have the same levels (to 1e-11) and degeneracies")
check(np.sum(S["g_rust"][S["eps_rust"] < 0.2]) == S["N"],
      "the eight particles exactly fill the two levels below the gap")
```

`S` holds the data of N8_lamm1_a00_T10. The table of Out [2] prints each reference level with its key, degeneracy and energy, and the Rust level next to it (the braces with an empty text and the format `12s` print twelve blanks; `{key[1]:+d}` prints the block type with its sign); the reference keeps one level more than the Rust solver (`"(not kept)"`). The first check requires the same degeneracies and levels that agree to 10^{-11}; they agree to about 10^{-12}. The second requires that the levels below 0.2 hold exactly the eight quanta: the two levels at 0.000245562708 with 4 states each.

```
checker_history = repository_file(f"{KS}/checker/README.md").read_text(
    encoding="utf-8")
found = re.search(r"Rust.s mu was ([0-9.]+) and the reference.s\s+([0-9.]+) "
                  r"\(\|diff\| ([0-9.e-]+) against a tolerance of ([0-9.e-]+)\)",
                  checker_history)
MU_OLD_RUST, MU_OLD_REF, DIFF_OLD, TOL_OLD = (float(x) for x in found.groups())
reference_history = repository_file(f"{KS}/reference/README.md").read_text(
    encoding="utf-8")
REF_OLD_MISS = float(re.search(r"this mu differed by ([0-9.e-]+) from the 40-digit",
                               reference_history).group(1))
say(f"history: Rust mu {MU_OLD_RUST!r}, reference mu {MU_OLD_REF!r}, |diff| "
    f"{DIFF_OLD:.3g}, tolerance {TOL_OLD:.3g}; reference first run missed its root "
    f"by {REF_OLD_MISS:.3g}")
check(abs(MU_OLD_RUST - MU_OLD_REF) - DIFF_OLD < 5e-13 and DIFF_OLD > TOL_OLD,
      "the recorded difference exceeds the recorded tolerance",
      record=f"{KS}/checker/README.md, History 1")
```

The numbers of the history are read from the two README files with regular expressions, so that the notebook uses the recorded values, not copies typed by hand. In the first pattern a point `.` matches any single character (here the apostrophe of the words "Rust's" and "reference's"), `\s+` one or more blanks or line breaks (the README breaks this sentence across two lines), `\(` and `\|` a literal bracket and vertical bar, and each `(...)` captures a number; `found.groups()` returns the four captured texts, which the **generator** `(float(x) for x in ...)` turns into four numbers given four names at once. The second pattern reads how far the reference's first complete run missed its own root. Out [2] prints the history: Rust $\mu = 0.2100104489071649$, reference $\mu = 0.2100104497343054$, difference 8.27×10^{-10}, tolerance 7.97×10^{-12}; the reference's first run missed its root by 1.15×10^{-9}. The check confirms that the recorded difference is the difference of the two recorded values (to 5×10^{-13}, the rounding of the printed digits) and exceeds the tolerance.

In [3], the state as a picture (Figure 16c.1).

```
sys.dont_write_bytecode = True  # do not write a __pycache__ folder into the record
sys.path.insert(0, str(repository_file(f"{KS}/reference")))  # Python looks here too
import ks_fd as K  # noqa: E402  the reference solver's module (Revision code)
```

The reference module `ks_fd` is imported as `K`, as in Notebook 16b.

```
eps, g, T, N = S["eps_ref"], S["g_ref"], S["T"], S["N"]
mu = S["mu_ref"]
x = (eps - mu) / T
below = eps < mu
holes = g * K.fermi(-x)  # g (1 - f): f(-x) = 1 - f(x), computed without subtracting
particles = g * K.fermi(x)  # g f
```

With the reference levels and committed μ, `x` holds the arguments $(\varepsilon_i - \mu)/T$ and `below` marks the levels below μ. `K.fermi` is the reference's Fermi function; it computes $e^{-|x|}$ first and returns $e^{-|x|}/(1+e^{-|x|})$ for $x > 0$ and $1/(1+e^{-|x|})$ otherwise, so it never computes the exponential of a large positive number, which would overflow. `holes` are $g(1-f) = g\,f(-x)$, computed directly as $f(-x)$ and never as a subtraction $1 - f$ (which would lose all digits for a full level), and `particles` are $g\,f$.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.5, 4.2))
energies = np.linspace(-0.05, 1.0, 2001)
left.plot(energies, K.fermi((energies - mu) / T), color=PALETTE[0], lw=1.8,
          label="occupation $f((\\varepsilon - \\mu)/T)$")
for e in np.unique(eps):  # one dotted line per energy, labelled with its states
    left.axvline(e, color="k", lw=0.8, ls=":")
    states = g[eps == e]
    label = (f"g = {states[0]:.0f}" if len(states) == 1
             else f"{len(states)} levels, g = {states[0]:.0f} each")
    left.text(e + 0.008, 0.08, label, rotation=90, fontsize=8)
left.axvline(mu, color=PALETTE[1], lw=1.4, ls="--", label=f"$\\mu = {mu:.6f}$")
left.annotate("", (eps[2], 1.12), (eps[1], 1.12), arrowprops={"arrowstyle": "<->"})
left.text(0.5 * (eps[1] + eps[2]), 1.15, "gap $= 43\\,T$", ha="center")
left.set_ylim(-0.05, 1.25)
left.set_xlabel("energy $\\varepsilon$ (units of $m$)")
```

```
left.set_ylabel("occupation $f$")
left.set_title("N8_lamm1_a00_T10: levels and occupation")
left.legend(fontsize=8, loc="center right", framealpha=1.0)
```

The left panel draws the occupation $f((\varepsilon-\mu)/T)$ on 2001 energies from -0.05 to 1, a dotted vertical line at each distinct level (`np.unique`), labelled with its degeneracy, turned by 90 degrees (two levels share the lowest energy, so their label says "2 levels, g = 4 each"), μ as a dashed orange line, and a double arrow over the gap between the levels number 1 and 2 with the text "gap = $43\,T$".

```
right.semilogy(eps[below], holes[below], "v", color=PALETTE[1], ms=9,
               label=f"holes $g(1-f)$, sum {np.sum(holes[below]):.3e}")
right.semilogy(eps[~below], particles[~below], "^", color=PALETTE[0], ms=9,
               label=f"particles $g f$, sum {np.sum(particles[~below]):.3e}")
right.axvline(mu, color=PALETTE[1], lw=1.4, ls="--")
right.set_xlabel("energy $\\varepsilon$ (units of $m$)")
right.set_ylabel("thermal holes and particles")
right.set_title("the balance that fixes $\\mu$")
right.legend(fontsize=8, loc="upper right")
fig.tight_layout()
```

The right panel draws the holes of the levels below μ (triangles pointing down) and the particles of the levels above it (triangles up) on a logarithmic axis, with their sums in the legend.

```
save_figure(fig, "levels_and_occupations",
            "The thermal Kohn-Sham state N8_lamm1_a00_T10 ($N = 8$, $\\lambda = "
            "-\\lambda_1$, $a_{4,0} = 0$, $T = 0.01\\,m$) of the reference solver. "
            "Left: the Fermi-Dirac occupation against the energy in units of $m$, "
            "with the levels as dotted lines and their degeneracies $g$; the two "
            "lowest levels hold the eight particles, the next level lies 43 times "
            "$T$ higher, and $\\mu$ (dashed) sits in the middle of this gap. Right: "
            "the thermal holes of the full levels and the thermal particles of the "
            "empty ones on a logarithmic axis; both sums are about $6 \\times "
            "10^{-9}$ and cancel, which is the Mermin condition.")
```

`save_figure` saves Figure 16c.1. What the student should see: on the left, the occupation is a step from 1 to 0 about $T = 0.01$ wide, centred at $\mu = 0.210010$, in the middle of a gap 43 times wider; the two lowest levels lie at the top of the step, all others at the bottom. On the right, the holes of the lowest levels and the particles of the level at 0.43 are both about 6×10^{-9}; the particles of the higher levels fall off steeply (10^{-15}, 10^{-20}, ...), because each level's particles shrink by $e^{-\Delta\varepsilon/T}$.

```
balance = np.sum(holes[below]) - np.sum(particles[~below])
say(f"holes {np.sum(holes[below]):.6e}, particles {np.sum(particles[~below]):.6e}, "
    f"difference {balance:.2e}")
check(abs(balance) < 1e-15 and np.sum(g[below]) == N,
      "at the committed mu the thermal holes and particles balance (to 1e-15)")
```

`balance` is the holes minus the particles; Out [3] prints both, 6.210359×10^{-9}, and their difference, -6.6×10^{-24}. The check requires a balance below 10^{-15} and the levels below μ to hold exactly N states: the Mermin condition in the form $P = H_{th}$ of Section 16.20.

In [4], the root with 40 digits: Newton's method.

```
mp.mp.dps = 40  # every mpmath number now carries 40 significant digits
```

`mp.mp.dps = 40` sets the working precision of mpmath to 40 decimal places (significant digits) for every following computation.

```
def mermin_root_40(eps, g, N, T, start):
    """Newton's method for the root mu of sum g f((eps - mu)/T) = N with 40 digits.
    Returns (the root, dN/dmu there, the list of all guesses)."""
    E = [mp.mpf(float(e)) for e in eps]  # the levels, exactly the given doubles
    G = [mp.mpf(float(x)) for x in g]
    T40, N40, mu = mp.mpf(float(T)), mp.mpf(float(N)), mp.mpf(float(start))
    guesses = [mu]
    for _ in range(60):
        total, slope = mp.mpf(0), mp.mpf(0)
        for e, gi in zip(E, G):
            x = (e - mu) / T40
            if x > 2000:  # f < e^(-2000): invisible even with 40 digits
                continue
            f = 1 / (1 + mp.exp(x))
            total += gi * f  # N(mu)
            slope += gi * f * (1 - f) / T40  # dN/dmu
        step = (total - N40) / slope  # where the tangent line reaches N
        mu -= step
        guesses.append(mu)
        if abs(step) < mp.mpf(10) ** -32:  # the first 32 digits are settled
            break
    return mu, slope, guesses
```

`mermin_root_40` is Newton's method of Section 16.20 in 40-digit numbers. `mp.mpf(float(e))` turns each level into an mpmath number exactly equal to the given double (no digit is invented or lost). The loop makes at most 60 steps. In each it adds up $N(\mu)$ (`total`) and $N'(\mu) = \sum gf(1-f)/T$ (`slope`) over the levels, skipping levels with $x > 2000$, whose occupation e^{-2000} is invisible even with 40 digits. `step` is $(N(\mu) - N)/N'(\mu)$, and the new guess is μ minus the step; every guess is kept. The loop stops when a step is below 10^{-32} (`mp.mpf(10) ** -32`): then the first 32 digits are settled. The function returns the root, the slope there and all guesses.

```
start = 0.5 * (S["eps_rust"][1] + S["eps_rust"][2])  # the middle of the gap
ROOT_R, SLOPE_R, NEWTON_PATH = mermin_root_40(S["eps_rust"], S["g_rust"], 8, 0.01,
                                              start)
ROOT_F, SLOPE_F, _ = mermin_root_40(S["eps_ref"], S["g_ref"], 8, 0.01, start)
print("Newton steps from the middle of the gap (Rust levels):")
for k, guess in enumerate(NEWTON_PATH):
    distance = mp.nstr(abs(guess - ROOT_R), 3)  # from the final root
    print(f"  mu_{k} = {mp.nstr(guess, 34)}   distance {distance}")
report("40-digit root on the Rust levels", mp.nstr(ROOT_R, 30))
report("40-digit root on the reference levels", mp.nstr(ROOT_F, 30))
report("dN/dmu at the root (Rust levels)", mp.nstr(SLOPE_R, 7))
```

Newton's method starts in the middle of the gap, at $(\varepsilon_1 + \varepsilon_2)/2$ of the Rust levels (positions 1 and 2 of the list), and is run on the Rust and on the reference levels. The loop prints every guess with

34 digits and its distance from the final root (`mp.nstr(x, n)` writes x with n significant digits): the distances 5.49×10^{-3}, 4.93×10^{-4}, 3.99×10^{-7}, 2.12×10^{-16}, 1.48×10^{-34} and 0 show the doubling of the correct digits (Out [4]). The three RESULT lines give the two roots with 30 digits and $N' = 1.242072 \times 10^{-6}$.

```
fixture = next(s for s in read_json(f"{KS}/solver/tools/mermin-roots-40digit.json")
               ["fixture"] if s["id"] == STATE)
same_doubles = [float(e) for e, _ in fixture["levels"]] == list(S["eps_rust"])
check(same_doubles and abs(ROOT_R - mp.mpf(fixture["root40"])) < mp.mpf(10) ** -32,
      "the root on the Rust levels equals the fixture root40 to 1e-32",
      record=f"{KS}/solver/tools/mermin-roots-40digit.json, state {STATE}")
```

`fixture` is the entry of this state in the Rust solver's 40-digit fixture `mermin-roots-40digit.jso n`, made from the exact doubles of its final levels. `same_doubles` confirms that the fixture's levels are exactly our Rust levels, and the check requires our root to equal the fixture's `root40` to 10^{-32}.

```
sys.path.insert(0, str(repository_file(f"{KS}/checker")))
import crosscheck_ks as CC  # noqa: E402  the cross-check program (Revision code)

checker_root, _ = CC.mu_high_precision(list(S["eps_rust"]), list(S["g_rust"]), 8.0,
                                       0.01, S["mu_rust"])
row = {r["case"]: r for r in read_csv(f"{KS}/reports/ks-crosscheck-table.csv")}[
    f"{STATE} mu_high_precision"]
say(f"table row: Rust {row['rust']}, reference {row['reference']}, |diff| "
    f"{row['abs_diff']}; ours: |diff| {mp.nstr(abs(ROOT_R - ROOT_F), 4)}")
check(checker_root == float(ROOT_R) and abs(float(ROOT_R) - float(row["rust"])) < 1e-16
      and abs(float(ROOT_F) - float(row["reference"])) < 1e-16,
      "our 40-digit roots equal the checker's function and its table row",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_mu_high_precision")
```

The cross-check program `crosscheck_ks.py` is imported as `CC` (its folder is added to the search list first), and its own 40-digit function `mu_high_precision` computes the root on the Rust levels, starting from the committed μ. `row` is the row `N8_lamm1_a00_T10 mu_high_precision` of the cross-check table (a dictionary comprehension over all rows, then indexed by the case name). The printed line compares the table's two roots and their difference, 4.018×10^{-13}, with ours. The check requires our root, converted to a double, to equal the checker's exactly and both roots to equal the table's to 10^{-16}.

```
check(abs(abs(ROOT_R - ROOT_F) - mp.mpf(row["abs_diff"])) < 1e-16
      and abs(ROOT_R - ROOT_F) < 1e-12,
      "the two roots differ by only 4e-13: the levels agree, the old Rust mu did not")
```

The last check: our two roots differ by the table's 4.018×10^{-13} and by less than 10^{-12}. The levels of the two solvers agree; the old Rust μ, 8.3×10^{-10} below both roots, did not.

In [5], why 16 digits are not enough: the staircase.

```
eps_r, g_r = S["eps_rust"], S["g_rust"]
```

```
def direct_residual(mu, eps, g, N, T):
    """sum g f - N, added up directly in doubles (the method of the first runs)."""
    return float(np.sum(g * K.fermi((eps - mu) / T))) - N


def exact_residual(mu40, eps, g, N, T):
    """sum g f - N with 40 digits (mu40 an mpmath number)."""
    return sum(mp.mpf(float(gi)) / (1 + mp.exp((mp.mpf(float(e)) - mu40) / T))
               for e, gi in zip(eps, g)) - N
```

`direct_residual` is the residual $\sum gf - N$ added up directly in doubles, the method of the first runs (`np.sum` of the products, `float` to make it an ordinary number). `exact_residual` is the same sum in 40-digit numbers, for a 40-digit μ.

```
shifts = np.linspace(-3e-9, 3e-9, 1201)  # distances from the root, units of m
mu_star = float(ROOT_R)  # the double nearest to the 40-digit root
D = np.array([direct_residual(mu_star + d, eps_r, g_r, 8.0, 0.01) for d in shifts])
W = np.array([K.mermin_residual(eps_r, g_r, 8.0, 0.01, mu_star + d) for d in shifts])
X = np.array([float(exact_residual(mp.mpf(mu_star) + mp.mpf(float(d)), eps_r, g_r,
                                   8, mp.mpf("0.01"))) for d in shifts])
steps = np.unique(D)  # the different values the direct residual takes
zero = shifts[D == 0.0]  # where the computed direct residual is exactly zero
```

`shifts` holds 1201 distances from the root between -3×10^{-9} and 3×10^{-9}, 5×10^{-12} apart. `mu_star` is the double nearest to the 40-digit root. At each shifted μ the cell evaluates the direct residual `D`, the well-conditioned residual `W` of Section 16.20 with the reference's function `K.mermin_residual`, and the exact residual `X` with 40 digits (the shift is added in 40-digit numbers, and the temperature is given as the exact text `"0.01"`). `steps` holds the different values that `D` takes (`np.unique` sorts them and removes repetitions), and `zero` the shifts at which `D` is exactly zero.

```
say(f"the direct residual takes {len(steps)} different values in this window: "
    + ", ".join(f"{v / 2.0 ** -50:+.0f}" for v in steps) + " times 2^-50")
say(f"it is exactly zero for mu - root from {zero.min():.3e} to {zero.max():.3e}, "
    f"an interval {zero.max() - zero.min():.2e} wide")
say(f"largest |W - exact| = {np.max(np.abs(W - X)):.1e}, largest |D - exact| = "
    f"{np.max(np.abs(D - X)):.1e}")
check(np.all(np.mod(D / 2.0 ** -50, 1.0) == 0.0) and len(steps) <= 12,
      "the direct residual is a staircase: whole multiples of 2^-50 only")
check(3e-10 < zero.max() - zero.min() < 3e-9,
      "the direct residual vanishes on an interval about 1e-9 wide")
check(np.max(np.abs(W - X)) < 1e-20,
      "the well-conditioned residual follows the exact one to 1e-20")
```

Out [5]: the direct residual takes only six values in this window, -4, -3, -2, 0, $+2$ and $+4$ times 2^{-50}. Above 8 it moves in steps of $2^{-49} = 2 \cdot 2^{-50}$, the spacing of the doubles there (Section 16.20). Below 8 it moves in steps of one spacing, 2^{-50}, except at the last step: the value $-1 \cdot 2^{-50}$, that is the sum $8 - 2^{-50}$, never occurs in this window, and the residual jumps from -2 straight to 0 times 2^{-50}. Which doubles the computed sum can reach is decided by the rounding of the individual terms and of the partial sums along the way, not only by the spacing near 8; this is also why the

zero interval is not centred on the root. It is exactly zero from 8.250×10^{-10} below the root to 2.300×10^{-10} above it, an interval 1.05×10^{-9} wide (the estimate of Section 16.20 is 1.07×10^{-9}). The well-conditioned residual differs from the exact one by at most 3.9×10^{-23}, the direct one by up to 1.5×10^{-15}. The three checks require: every value of `D` a whole multiple of 2^{-50} (`np.mod(x, 1.0)` is the fractional part) and at most 12 different values; a zero interval between 3×10^{-10} and 3×10^{-9} wide; and `W` within 10^{-20} of the exact residual.

In [6], the staircase as a picture (Figure 16c.2).

```
fig, ax = plt.subplots(figsize=(8.5, 5.0))
scale_mu, scale_r = 1e-9, 1e-15  # plot units: 1e-9 m and 1e-15 particles
ax.axvspan(zero.min() / scale_mu, zero.max() / scale_mu, color="0.85",
           label="direct residual exactly 0")
ax.step(shifts / scale_mu, D / scale_r, where="mid", color=PALETTE[1], lw=1.8,
        label="direct residual $D$ in doubles")
ax.plot(shifts / scale_mu, W / scale_r, color=PALETTE[0], lw=2.5, alpha=0.8,
        label="well-conditioned residual $W$ in doubles")
ax.plot(shifts / scale_mu, X / scale_r, "k--", lw=1.0, label="exact (40 digits)")
ax.axvline((MU_OLD_RUST - mu_star) / scale_mu, color=PALETTE[7], lw=1.5,
           label="mu of the first cross-check (Rust)")
ax.axhline(0.0, color="k", lw=0.6)
ax.set_xlabel("$\\mu$ minus the 40-digit root (units of $10^{-9}\\,m$)")
ax.set_ylabel("$N(\\mu) - N$ (units of $10^{-15}$)")
ax.set_title("N8_lamm1_a00_T10: the computed residual near the root")
ax.legend(fontsize=8, loc="upper left")
```

The plot units are 10^{-9} for μ and 10^{-15} for the residual. `axvspan` shades the zero interval in light grey (the colour `"0.85"` is a grey level). `ax.step(..., where="mid")` draws the direct residual as a staircase whose steps change half-way between the sample points; the well-conditioned residual is a thick blue line and the exact one a thin dashed black line. A red vertical line marks the faulty μ of the first cross-check, measured from the root.

```
save_figure(fig, "residual_staircase",
            "The residual $N(\\mu) - N$ of the Mermin condition of the state "
            "N8_lamm1_a00_T10 (Rust levels) within $3 \\times 10^{-9}\\,m$ of the "
            "40-digit root, horizontal axis in units of $10^{-9}\\,m$, vertical axis "
            "in units of $10^{-15}$ particles. Added up directly in ordinary computer "
            "numbers (orange) the sum near 8 can only change in steps of $2^{-50}$ "
            "or $2^{-49}$, so the residual is a staircase that is exactly zero on an "
            "interval about $10^{-9}$ wide (grey); the faulty $\\mu$ of the first "
            "cross-check (red) is the left end of that interval. The "
            "well-conditioned form (blue) follows the exact line (dashed) and "
            "crosses zero at the root.")
```

`save_figure` saves Figure 16c.2. What the student should see: the exact residual is a straight line through zero at the root, with slope $N' = 1.24 \times 10^{-6}$ (in these units 1.24 per unit); the blue well-conditioned residual lies on it everywhere; the orange direct residual is a staircase with steps of $2^{-50} = 0.89$ and $2^{-49} = 1.78$ units, exactly zero on the grey interval; and the red line, the faulty μ, is the left end of that interval, where bisection on the direct sum must stop.

In [7], bisection on the two forms: the faulty value and the repair.

```
def bisection(residual, lo, hi):
    """Bisection for the root of an increasing function with residual(lo) < 0 and
    residual(hi) >= 0, down to neighbouring doubles; returns (root, all midpoints)."""
    midpoints = []
    while True:
        mid = 0.5 * (lo + hi)
        if mid <= lo or mid >= hi:  # lo and hi are neighbouring doubles: done
            break
        midpoints.append(mid)
        if residual(mid) < 0.0:
            lo = mid  # the root lies above mid
        else:
            hi = mid  # the root lies at or below mid
    return 0.5 * (lo + hi), midpoints
```

`bisection(residual, lo, hi)` is the bisection of Section 16.20 for an increasing function given as an argument (in Python a function can be passed like any other value). The `while True:` loop runs until `break`: if the midpoint is not strictly between `lo` and `hi`, the two are neighbouring doubles and nothing is left to halve. Otherwise the midpoint is recorded, and the half is kept in which the root lies: above the midpoint if the residual is negative there, at or below it otherwise.

```
def bracket(eps, T):
    """The starting interval of the reference: every level, 60 T and 1 m around."""
    return float(eps.min()) - 60.0 * T - 1.0, float(eps.max()) + 60.0 * T + 1.0
```

`bracket` gives the starting interval that the reference uses: from the lowest level minus $60T$ minus 1 to the highest level plus $60T$ plus 1, which surely contains the root.

```
MU_DIRECT, PATH_DIRECT = bisection(
    lambda mu: direct_residual(mu, eps_r, g_r, 8.0, 0.01), *bracket(eps_r, 0.01))
MU_WELL, PATH_WELL = bisection(
    lambda mu: K.mermin_residual(eps_r, g_r, 8.0, 0.01, mu), *bracket(eps_r, 0.01))
_, mu_ks_fd = K.mermin(eps_r, g_r, 8.0, 0.01)  # the reference's own function
say(f"bisection, direct sum:      mu = {MU_DIRECT!r} after {len(PATH_DIRECT)} "
    f"halvings; minus root {float(MU_DIRECT - ROOT_R):+.3e}")
say(f"bisection, well-conditioned: mu = {MU_WELL!r} after {len(PATH_WELL)} "
    f"halvings; minus root {float(MU_WELL - ROOT_R):+.3e}")
say(f"recorded mu of the first cross-check (Rust): {MU_OLD_RUST!r}; committed "
    f"repaired Rust mu: {S['mu_rust']!r}")
report("direct-sum bisection minus the 40-digit root",
       f"{float(MU_DIRECT - ROOT_R):.4e}", "m")
```

The two bisections on the Rust levels: on the direct residual and on the well-conditioned one (each residual is given as a lambda of μ alone). `K.mermin` is the reference's own root finder, which bisects the well-conditioned form. Out [7]: the direct sum gives $\mu = 0.21001044890716491$, 8.267×10^{-10} below the root, and the well-conditioned form 0.21001044973390365, 2.6×10^{-18} from it; both after 57 halvings. That number is no accident: the starting interval reaches from -1.600 to 2.482, so it is 4.08 wide; the doubles near 0.21 are $2^{-55} = 2.8 \times 10^{-17}$ apart; and $4.08/2^{57} = 2.8 \times 10^{-17}$, so after 57 halvings `lo` and `hi` are neighbouring doubles. The RESULT line repeats the error of the direct sum.

```
check(abs(MU_DIRECT - MU_OLD_RUST) <= 5e-17,
      "bisection on the direct sum gives the recorded faulty mu (all 16 digits)",
      record=f"{KS}/checker/README.md, History 1 (Rust mu 0.2100104489071649)")
check(MU_WELL == mu_ks_fd and abs(MU_WELL - ROOT_R) < 1e-16,
      "bisection on the well-conditioned form (ks_fd.mermin) finds the root to 1e-16")
check(abs(S["mu_rust"] - MU_WELL) < 1e-16,
      "the committed repaired Rust mu equals the well-conditioned root",
      record=f"{KS}/results/thermo/thermodynamics.csv, state {STATE}")
```

The three checks: the direct-sum bisection reproduces the recorded faulty $\mu = 0.2100104489071649$ of the first cross-check to 5×10^{-17}, that is, in all sixteen digits that the record prints; the well-conditioned bisection gives exactly the result of the reference's own function and lies within 10^{-16} of the root; and the committed, repaired Rust μ equals the well-conditioned root to 10^{-16}.

```
mu_direct_ref, _ = bisection(
    lambda mu: direct_residual(mu, S["eps_ref"], S["g_ref"], 8.0, 0.01),
    *bracket(S["eps_ref"], 0.01))
say(f"the direct sum on the reference levels misses their root by "
    f"{float(mu_direct_ref - ROOT_F):+.3e}")
```

The direct sum on the reference levels misses their own root by almost the same amount, 8.271×10^{-10}: the two level sets agree to about 10^{-12}, so their staircases have zero intervals at almost the same place, and bisection stops at the left end again.

In [8], three searches compared (Figure 16c.3).

```
fig, ax = plt.subplots(figsize=(8.5, 5.0))
floor = 1e-36  # distances that are exactly zero are drawn here
for colour, path, label, size in (
        (PALETTE[0], PATH_WELL, "bisection, well-conditioned form", 6),
        (PALETTE[1], PATH_DIRECT, "bisection, direct sum", 3)):  # drawn on top
    distance = [max(float(abs(mp.mpf(m) - ROOT_R)), floor) for m in path]
    ax.semilogy(range(1, len(path) + 1), distance, "o-", color=colour, ms=size,
                lw=1.2, label=f"{label} ({len(path)} steps)")
newton = [max(float(abs(m - ROOT_R)), floor) for m in NEWTON_PATH]
ax.semilogy(range(len(newton)), newton, "s-", color=PALETTE[2], ms=6, lw=1.5,
            label="Newton with 40 digits")
```

For the two bisections the distance of every midpoint from the 40-digit root is drawn against the step number on a logarithmic axis (distances that are exactly zero are raised to 10^{-36}); the direct-sum path is drawn second, with smaller markers, so that it lies on top. The Newton guesses are drawn as green squares.

```
ax.axhline(abs(MU_OLD_RUST - float(ROOT_R)), color=PALETTE[7], lw=1.0, ls="--",
           label="error of the first cross-check")
ax.axhline(math.ulp(mu_star), color="k", lw=0.8, ls=":",
           label="spacing of the doubles near $\\mu$")
ax.set_ylim(floor / 3.0, 10.0)
ax.set_xlabel("step")
ax.set_ylabel("distance from the 40-digit root (units of $m$)")
```

```
ax.set_title("Three searches for $\\mu$ in N8_lamm1_a00_T10")
ax.legend(fontsize=8, loc="upper right")
```

Two horizontal lines mark the error of the first cross-check (red, dashed) and the spacing of the doubles near μ (`math.ulp(x)` is the distance from x to the next double; dotted).

```
save_figure(fig, "convergence_paths",
            "The distance of each guess for $\\mu$ from the 40-digit root, units "
            "of $m$, logarithmic, against the step number, for the state "
            "N8_lamm1_a00_T10 with the Rust levels. Bisection halves the distance at "
            "every step; on the direct sum (orange) it stalls at $8.3 \\times "
            "10^{-10}$, the error of the first cross-check (red dashed), because "
            "the computed sum is zero on a whole interval; on the well-conditioned "
            "form (blue) it reaches the spacing of the doubles (dotted). Newton's "
            "method with 40 digits (green) doubles the number of correct digits at "
            "each step; zero distances are drawn at $10^{-36}$.")
check(min(float(abs(mp.mpf(m) - ROOT_R)) for m in PATH_DIRECT[-20:]) > 5e-10
      and float(abs(NEWTON_PATH[-2] - ROOT_R)) < 1e-32 and len(NEWTON_PATH) <= 7,
      "the direct bisection stalls above 5e-10; Newton reaches 1e-32 in 5 steps")
```

`save_figure` saves Figure 16c.3. What the student should see: the two bisection paths coincide for about 30 steps, halving the distance at every step on average (a straight line on this axis); then the direct-sum path stops falling and stays at 8.3×10^{-10} (it now only explores the zero interval of the staircase), while the well-conditioned path continues down to the spacing of the doubles; Newton's method with 40 digits reaches 10^{-34} in four steps. The check requires the last 20 direct-sum midpoints to stay more than 5×10^{-10} from the root, Newton's next-to-last guess to be within 10^{-32}, and at most 7 Newton guesses.

In [9], the rounding bounds.

```
def rounding_bounds(eps, g, N, T, mu):
    """dN/dmu, the bound of the direct sum and the bound of the well-conditioned form
    at mu (the formulas of solver/src/mermin.rs), and P, H_th, d."""
    x = (eps - mu) / T
    f_plus, f_minus = K.fermi(x), K.fermi(-x)  # f(x) and 1 - f(x) = f(-x)
    weight = g * f_plus * f_minus  # g f (1 - f): how strongly a level reacts to mu
    slope = float(np.sum(weight)) / T  # dN/dmu
    mean_distance = float(np.sum(weight * np.abs(eps - mu)) / np.sum(weight))
    below = x < 0.0
    particles = float(np.sum(g[~below] * f_plus[~below]))  # P
    holes = float(np.sum(g[below] * f_minus[below]))  # H_th
    d = N - float(np.sum(g[below]))  # a whole number
    n = len(eps)
    big_l = max(abs(math.log(particles + max(-d, 0.0))),  # |ln A|
                abs(math.log(holes + max(d, 0.0))))  # |ln B|
    common = EPS_MACH * (3.0 * mean_distance + T * (math.log(float(np.max(g))) + 3.0)
                         + 2.0 * abs(mu))
    direct = EPS_MACH * (n + 2) * N / slope + common
    well = EPS_MACH * (n + 2 + big_l) * (particles + holes + abs(d)) / slope + common
    return {"slope": slope, "direct": direct, "well": well, "P": particles,
            "H_th": holes, "d": d, "n": n, "Lambda": big_l}
```

`rounding_bounds` evaluates the two bounds of Section 16.20 at a given μ. `f_plus` and `f_minus` are $f(x)$ and $f(-x) = 1 - f(x)$; `weight` is $g\,f(1-f)$, how strongly each level reacts to μ; `slope` is $N' = \sum g f(1-f)/T$; `mean_distance` is $\langle|\varepsilon - \mu|\rangle$, the mean distance of the levels from μ weighted with `weight`. `particles` is P (the levels above μ; `~below` reverses the mask), `holes` is H_{th}, and `d` the whole number $N - \sum_{\text{below}} g$. `big_l` is $\Lambda = \max(|\ln A|, |\ln B|)$ with $A = P + \max(-d, 0)$ and $B = H_{th} + \max(d, 0)$, the two sides of the balance. `common` holds the three small terms that both bounds share, and `direct` and `well` are B_{direct} and B_{well}.

```
MERMIN_REPORT = read_json(f"{KS}/reports/ks-rust-mermin-roots.json")
RECORD = {s["id"]: s for s in MERMIN_REPORT["states"]}
b = rounding_bounds(eps_r, g_r, 8.0, 0.01, mu_star)
rec = RECORD[STATE]
say(f"n = {b['n']} levels; P = {b['P']:.6e}, H_th = {b['H_th']:.6e}, d = {b['d']:.0f}, "
    f"Lambda = {b['Lambda']:.3f}; dN/dmu = {b['slope']:.6e} (record {rec['dN_dmu']})")
say(f"B_direct = {b['direct']:.4e} (record {rec['boundDirectCount']}); B_well = "
    f"{b['well']:.4e} (record {rec['boundWellConditioned']})")
```

`MERMIN_REPORT` is the Rust solver's 40-digit report `ks-rust-mermin-roots.json`, and `RECORD` files its states by id. The bounds are evaluated at the double nearest to the root, and Out [9] prints them next to the record: $n = 7$ levels, $P = H_{th} = 6.210359 \times 10^{-9}$, $d = 0$, $\Lambda = 18.897$ (which is $|\ln(6.21 \times 10^{-9})|$), $N' = 1.242072 \times 10^{-6}$ (record the same), $B_{direct} = 1.2871 \times 10^{-8}$ (record 1.287×10^{-8}) and $B_{well} = 3.1539 \times 10^{-16}$ (record 3.154×10^{-16}).

```
one_step = EPS_MACH * 8.0 / b["slope"]  # one rounding of size eps_mach N, as mu
say(f"one rounding of size eps_mach N moves the root by {one_step:.3e}; the error "
    f"of the first cross-check is {DIFF_OLD / one_step:.2f} of that")
```

`one_step` is the shift $\epsilon_{mach} N/N' = 1.430 \times 10^{-9}$ that a single rounding of the size $\epsilon_{mach} N$ causes; the error of the first cross-check is 0.58 of it (Out [9]).

```
check(abs(b["slope"] / float(rec["dN_dmu"]) - 1) < 1e-5
      and abs(b["direct"] / float(rec["boundDirectCount"]) - 1) < 1e-3
      and abs(b["well"] / float(rec["boundWellConditioned"]) - 1) < 1e-3,
      "dN/dmu and both rounding bounds reproduce the Rust 40-digit report",
      record=f"{KS}/reports/ks-rust-mermin-roots.json, state {STATE}")
check(abs(MU_OLD_RUST - float(ROOT_R)) <= b["direct"]
      and abs(S["mu_rust"] - float(ROOT_R)) <= b["well"] + 1e-16,
      "the old error lies within B_direct, the repaired mu within B_well")
```

The first check reproduces the record: N' to a relative 10^{-5} and both bounds to 10^{-3} (the report prints four digits). The second confirms that the old error lies within B_{direct} and the repaired μ within B_{well}.

In [10], all 45 thermal states with eight quanta.

```
TABLE = {r["case"]: r for r in read_csv(f"{KS}/reports/ks-crosscheck-table.csv")}


def tolerance(x_ref, u_ref, u_rust):
    """The tolerance rule of the cross-check for single numbers."""
    return 3.0 * (u_ref + u_rust) + 1e-12 * max(1.0, abs(x_ref))
```

TABLE files the rows of the cross-check table by case, and tolerance is the rule of Section 16.7 for single numbers.

```
ROWS = []  # one dictionary per state
for sid in sorted(s for s in REFINEMENT if s.startswith("N8_")):
    s = state_data(sid)
    N, T = s["N"], s["T"]
    root_r, _, _ = mermin_root_40(s["eps_rust"], s["g_rust"], N, T, s["mu_rust"])
    root_f, _, _ = mermin_root_40(s["eps_ref"], s["g_ref"], N, T, s["mu_ref"])
    r = {"id": sid, "N": N, "T": T, "root_r": root_r, "root_f": root_f,
         "b_r": rounding_bounds(s["eps_rust"], s["g_rust"], N, T, float(root_r)),
         "b_f": rounding_bounds(s["eps_ref"], s["g_ref"], N, T, float(root_f))}
```

The loop goes through the 45 thermal states whose id begins with N8 (startswith), sorted. For each it collects the data, computes the 40-digit roots on the Rust and on the reference levels (Newton's method, started from each solver's committed μ), and evaluates both bounds at both roots.

```
    for side in ("r", "f"):  # the two bisections on each solver's levels
        e, gg = (s["eps_rust"], s["g_rust"]) if side == "r" else (s["eps_ref"],
                                                                  s["g_ref"])
        root = float(r["root_" + side])
        r["direct_" + side] = bisection(
            lambda mu: direct_residual(mu, e, gg, N, T), *bracket(e, T))[0] - root
        r["well_" + side] = bisection(
            lambda mu: K.mermin_residual(e, gg, N, T, mu), *bracket(e, T))[0] - root
```

For each solver's levels (side is "r" for Rust and "f" for the reference) it runs bisection on the direct sum and on the well-conditioned form and stores each result minus the root ([0] takes the root from the returned pair). The lambdas are used immediately inside the loop, so each sees the current levels e and degeneracies gg.

```
    r["committed_r"] = float(mp.mpf(s["mu_rust"]) - root_r)
    r["committed_f"] = float(mp.mpf(s["mu_ref"]) - root_f)
    r["U_mu_ref"] = s["U_mu_ref"]
    u_lev_rust = 16.0 / 15.0 * REFINEMENT[sid]["levels"]["max_abs_diff"]
    u_mu_rust = 16.0 / 15.0 * REFINEMENT[sid]["scalars"]["mu"]["abs_diff"]
    tol_hp = tolerance(float(root_f), s["U_levels_ref"], u_lev_rust)
    tol_mu = tolerance(s["mu_ref"], s["U_mu_ref"], u_mu_rust)
    r["ratio_hp"] = float(abs(root_r - root_f)) / tol_hp
    r["ratio_mu"] = abs(s["mu_rust"] - s["mu_ref"]) / tol_mu
    r["tol_hp"], r["tol_mu"] = tol_hp, tol_mu
    r["n_r"], r["n_f"] = len(s["eps_rust"]), len(s["eps_ref"])
    ROWS.append(r)
```

committed_r and committed_f are the committed μ of each solver minus its 40-digit root. The cell then rebuilds two comparisons of the cross-check: the 40-digit roots, with U_{ref} the reference's largest level uncertainty and U_{Rust} $\frac{16}{15}$ times the largest canonical-minus-refined level difference (because μ is a weighted mean of the levels, its uncertainty is bounded by theirs: Revision/kohn_sham/checker/crosscheck_ks.py, comment before the 40-digit check), and the committed values of μ, with the reference's U of μ and $\frac{16}{15}|\mu_c - \mu_r|$; it keeps both tolerances and ratios and the numbers of levels.

```
print("the nine states with the largest direct-sum bound (Rust levels):")
print("state                 n   dN/dmu     B_direct   direct error  B_well")
for r in sorted(ROWS, key=lambda r: -r["b_r"]["direct"])[:9]:
    print(f"{r['id']:20s} {r['n_r']:3d}  {r['b_r']['slope']:.3e}  "
          f"{r['b_r']['direct']:.3e}  {r['direct_r']:+.3e}    {r['b_r']['well']:.3e}")
```

The table of Out [10] lists the nine states with the largest B_{direct} (Rust levels), sorted by it (`key=lamb da r: -...` sorts from the largest down): their number of levels, N', B_{direct}, the error of the direct sum and B_{well}. The three states at $a_{4,0} = 0$, $T = 0.01$ have $N' \approx 1.2 \times 10^{-6}$ and bounds near 10^{-8}; the next ones, at $a_{4,0} = 0.5$ or $T = 0.02$, have N' thousands of times larger and bounds below 10^{-11}.

In [11], the checks against the records.

```
worst = {"root": 0.0, "slope": 0.0, "bounds": 0.0, "hp": 0.0, "tol": 0.0,
         "ratio": 0.0}
for r in ROWS:
    rec = RECORD[r["id"]]
    worst["root"] = max(worst["root"], float(abs(r["root_r"] - mp.mpf(rec["root40"]))))
    worst["slope"] = max(worst["slope"],
                         abs(r["b_r"]["slope"] / float(rec["dN_dmu"]) - 1))
    worst["bounds"] = max(worst["bounds"],
                          abs(r["b_r"]["direct"] / float(rec["boundDirectCount"]) - 1),
                          abs(r["b_r"]["well"] / float(rec["boundWellConditioned"])
                              - 1))
    hp, mu_row = TABLE[f"{r['id']} mu_high_precision"], TABLE[f"{r['id']} mu"]
    worst["hp"] = max(worst["hp"], abs(float(r["root_r"]) - float(hp["rust"])),
                      abs(float(r["root_f"]) - float(hp["reference"])))
    worst["tol"] = max(worst["tol"], abs(r["tol_hp"] / float(hp["tolerance"]) - 1),
                       abs(r["tol_mu"] / float(mu_row["tolerance"]) - 1))
    worst["ratio"] = max(worst["ratio"], abs(r["ratio_hp"] - float(hp["ratio"])),
                         abs(r["ratio_mu"] - float(mu_row["ratio"])))
```

`worst` collects six largest differences: of our Rust roots from the report's `root40`; of N' and of the two bounds from the report (relative); of both 40-digit roots from the table's rows `mu_high_precision`; of our tolerances from the table's (relative); and of our ratios from the table's, for both rows of each state (`mu_high_precision` and `mu`).

```
say("largest differences from the records: " + ", ".join(
    f"{k} {v:.1e}" for k, v in worst.items()))
check(worst["root"] < 1e-16,
      "the 45 roots on the Rust levels reproduce root40 (to the 16-digit levels)",
      record=f"{KS}/reports/ks-rust-mermin-roots.json, states (root40)")
check(worst["slope"] < 1e-5 and worst["bounds"] < 1e-3,
      "dN/dmu and both bounds of all 45 states reproduce the Rust report",
      record=f"{KS}/reports/ks-rust-mermin-roots.json, states (bounds)")
check(worst["hp"] < 2e-16 and worst["tol"] < 1e-3 and worst["ratio"] <= 1e-4,
      "both 40-digit roots, tolerances and ratios reproduce 90 table rows",
      record=f"{KS}/reports/ks-crosscheck-table.csv, rows mu, mu_high_precision")
check(all(r["ratio_hp"] <= 1.0 and r["ratio_mu"] <= 1.0 for r in ROWS),
      "after the repair every mu comparison of the 45 states passes",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_state_functions")
```

Out [11] prints them: roots 2.8×10^{-17}, slope 4.5×10^{-7}, bounds 4.2×10^{-4}, table roots 5.6×10^{-17}, tolerances 3.9×10^{-4}, ratios 5.0×10^{-5}. The roots agree only to about 10^{-17}, not 10^{-35}, because the level files keep sixteen significant digits; the bounds and tolerances agree to the four digits the records print. The four checks reproduce the report (roots, then N' and bounds), the 90 table rows, and the cross-check's verdict that after the repair every comparison of μ in these 45 states passes.

```
diag_r = [abs(r["committed_r"]) / (r["n_r"] * EPS_MACH * r["N"] / r["b_r"]["slope"])
          for r in ROWS]
diag_f = [abs(r["committed_f"]) / (r["n_f"] * EPS_MACH * r["N"] / r["b_f"]["slope"]
                                   + 3.0 * r["U_mu_ref"]) for r in ROWS]
say(f"committed mu minus root, as a fraction of the diagnostic bound: Rust at most "
    f"{max(diag_r):.1e}, reference at most {max(diag_f):.1e}")
check(max(diag_r) <= 1.0 and max(diag_f) <= 1.0,
      "both solvers' committed mu lie within the rounding diagnostic",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_mu_rounding_diagnostic")
```

The diagnostic of the cross-check (`thermo_mu_rounding_diagnostic`): each committed μ minus the root on its own levels, divided by the conditioning bound $n\,\epsilon_{mach} N/N'$ (for the reference plus $3U$, because its μ combines three grids). Out [11]: at most 1.3×10^{-2} for Rust and 1.4×10^{-5} for the reference; the check requires at most 1.

```
check(all(abs(r["direct_" + s]) <= r["b_" + s]["direct"]
          and abs(r["well_" + s]) <= r["b_" + s]["well"] for r in ROWS
          for s in ("r", "f")),
      "in all 90 level sets each bisection lies within its own bound")
big = sorted(ROWS, key=lambda r: -abs(r["direct_r"]))[:3]
say("largest direct-sum errors: " + ", ".join(
    f"{r['id']} {r['direct_r']:+.2e}" for r in big))
check({r["id"] for r in big} == {"N8_lam0_a00_T10", "N8_lamm1_a00_T10",
                                 "N8_lamp1_a00_T10"},
      "the direct sum fails worst in the three states with the largest gap/T")
```

The next check confirms that in all 90 level sets (45 states, two solvers) each bisection lies within its own bound: the direct sum within B_{direct} and the well-conditioned form within B_{well}. `big` holds the three states with the largest direct-sum errors on the Rust levels, printed in Out [11] (-8.27×10^{-10}, $+2.72 \times 10^{-10}$, $+6.96 \times 10^{-11}$), and the last check requires them to be the three states at $a_{4,0} = 0$ and $T = 0.01$, where the gap is largest compared with T (a set `{...}` of ids is compared, so the order does not matter).

In [12], error against conditioning (Figure 16c.4).

```
scale = np.array([EPS_MACH * r["N"] / r["b_r"]["slope"] for r in ROWS])
scale_f = np.array([EPS_MACH * r["N"] / r["b_f"]["slope"] for r in ROWS])
fig, ax = plt.subplots(figsize=(8.0, 6.0))
low = 1e-19  # exact zeros are drawn here
series = [(scale, "direct_r", "o", PALETTE[1], "direct sum, Rust levels"),
          (scale_f, "direct_f", "s", PALETTE[7], "direct sum, reference levels"),
          (scale, "well_r", "o", PALETTE[0], "well-conditioned, Rust levels"),
          (scale_f, "well_f", "s", PALETTE[6], "well-conditioned, reference levels"),
          (scale, "committed_r", "^", PALETTE[2], "committed Rust mu")]
```

```
for xs, key, marker, colour, label in series:
    ys = np.maximum([abs(r[key]) for r in ROWS], low)
    ax.loglog(xs, ys, marker, color=colour, ms=5, alpha=0.8, label=label)
```

`scale` and `scale_f` hold the conditioning scale $\epsilon_{mach} N/N'$ of each state for the Rust and the reference levels. `series` lists five sets of points with their key, marker, colour and label; the loop draws each error (raised to 10^{-19} if exactly zero) against the conditioning scale on logarithmic axes.

```
line = np.array([1e-16, 1e-8])
ax.loglog(line, line, "k-", lw=1.0, label="error = $\\epsilon_{mach} N / N'$")
i_old = [r["id"] for r in ROWS].index(STATE)
ax.loglog([scale[i_old]], [DIFF_OLD], "*", color="k", ms=14,
          label="first cross-check (recorded)")
ax.set_xlabel("$\\epsilon_{mach} N / (dN/d\\mu)$ (units of $m$)")
ax.set_ylabel("$|\\mu$ computed $-$ 40-digit root$|$ (units of $m$)")
ax.set_title("45 thermal states with $N = 8$: error against conditioning")
ax.legend(fontsize=7, loc="upper left")
```

The black line is "error equals conditioning scale", a line of slope 1, and the star is the recorded error of the first cross-check at the scale of its state (`.index(STATE)` finds the state's position).

```
save_figure(fig, "errors_versus_bound",
            "Errors of the computed chemical potential, in units of $m$, against "
            "the conditioning scale $\\epsilon_{mach} N/(dN/d\\mu)$ for the 45 "
            "thermal states with $N = 8$, both axes logarithmic. Bisection on the "
            "direct sum (orange: Rust levels, red: reference levels) has errors "
            "that grow with the conditioning scale and approach the line of slope 1 "
            "in the worst states; the star is the recorded error of the first "
            "cross-check. The well-conditioned form (blue, purple) and the "
            "committed repaired Rust values (green) stay near $10^{-17}$ for every "
            "state; exact zeros are drawn at $10^{-19}$.")
```

`save_figure` saves Figure 16c.4. What the student should see: the errors of the direct sum (orange and red) grow with the conditioning scale and come within a factor of about 2 of the line of slope 1 in the worst states, where the star of the first cross-check sits; the well-conditioned form (blue, purple) and the committed repaired Rust values (green) stay near 10^{-17} whatever the conditioning, many of them exactly zero (the bottom row).

In [13], where the direct sum is dangerous (Figure 16c.5).

```
tags = [("lam0", "$\\lambda = 0$"), ("lamp1", "$\\lambda = +\\lambda_1$"),
        ("lamm1", "$\\lambda = -\\lambda_1$")]
slices, temps = ["a00", "a05", "a10", "a15", "a20"], ["T10", "T20", "T50"]
by_id = {r["id"]: r for r in ROWS}
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.6))
for ax, (tag, title) in zip(axes, tags):
    grid = np.array([[math.log10(by_id[f"N8_{tag}_{a}_{t}"]["b_r"]["direct"])
                      for t in temps] for a in slices])
    image = ax.imshow(grid, cmap="viridis", vmin=-15.0, vmax=-7.5, aspect="auto")
    ax.grid(False)  # no grid lines across the coloured squares
```

`tags` pairs the three coupling tags with their titles, `slices` and `temps` list the id parts of the five slices and three temperatures, and `by_id` files the 45 rows by id. For each coupling, `grid` is a 5×3 table of $\log_{10} B_{direct}$, slices down, temperatures across, built by a nested list comprehension; `imshow` paints it with the colour scale `viridis` between -15 and -7.5, and `ax.grid(False)` removes the grid lines over the squares.

```
    for i in range(5):
        for j in range(3):
            ax.text(j, i, f"{grid[i, j]:.1f}", ha="center", va="center",
                    color="w" if grid[i, j] < -11.5 else "k", fontsize=9)
    ax.set_xticks(range(3))
    ax.set_xticklabels(["0.01", "0.02", "0.05"])
    ax.set_yticks(range(5))
    ax.set_yticklabels(["0", "0.5", "1", "1.5", "2"])
    ax.set_xlabel("temperature $T$ (units of $m$)")
    ax.set_title(title)
axes[0].set_ylabel("slice $a_{4,0}$")
fig.colorbar(image, ax=axes, shrink=0.9, label="$\\log_{10} B_{direct}$ (units of $m$)")
```

Each square gets its number with one decimal, written in white on the dark squares and in black on the light ones; the ticks are labelled with the temperatures and the slices, and one colour scale serves the three maps.

```
save_figure(fig, "conditioning_map",
            "The rounding bound $B_{direct}$ of the direct sum, as $\\log_{10}$ of "
            "its value in units of $m$, for the 45 thermal states with $N = 8$ (Rust "
            "levels): one map per coupling, the slice $a_{4,0}$ of the deflating "
            "history downwards, the temperature across. The direct sum is dangerous "
            "only at low temperature early in the history, where the gap is 43 "
            "times $T$ and the bound reaches $10^{-8}$; later the redshift of the "
            "3-momenta closes the gap and the bound falls to about $10^{-15}$.")
check(max(by_id[f"N8_{t}_a00_T10"]["b_r"]["direct"] for t, _ in tags)
      == max(r["b_r"]["direct"] for r in ROWS)
      and all(by_id[f"N8_{t}_a00_T10"]["b_r"]["direct"]
              > 100.0 * by_id[f"N8_{t}_a10_T10"]["b_r"]["direct"] for t, _ in tags),
      "the bound is largest at a4,0 = 0, T = 0.01 and falls along the history")
```

`save_figure` saves Figure 16c.5. What the student should see: in each of the three maps only the top left square ($a_{4,0} = 0$, $T = 0.01$) is bright, at about -8, and its neighbours ($a_{4,0} = 0.5$ or $T = 0.02$) are near -11 to -12; everywhere else the bound lies between about 10^{-15} and 10^{-13}. Along the deflating history the gap closes, because the redshift of the 3-momenta brings the levels of the brane band down towards the filled levels. The check requires the largest bound of all 45 to be one of the three states at $a_{4,0} = 0$, $T = 0.01$, and each of them to be more than 100 times its value at $a_{4,0} = 1$.

In [14], what the tolerance rule did (Figure 16c.6).

```
order = sorted(range(len(ROWS)), key=lambda i: -ROWS[i]["b_r"]["direct"])
ratio_old = DIFF_OLD / TOL_OLD  # the ratio of the first, failed comparison
fig, ax = plt.subplots(figsize=(9.5, 4.8))
```

```
positions = np.arange(len(order))
ax.semilogy(positions, [ROWS[i]["ratio_mu"] for i in order], "o", color=PALETTE[0],
            ms=5, label="$\\mu$ (committed values), after the repair")
ax.semilogy(positions, [ROWS[i]["ratio_hp"] for i in order], "s", color=PALETTE[2],
            ms=5, mfc="none", label="$\\mu$ with 40 digits on each solver's levels")
k_old = order.index([r["id"] for r in ROWS].index(STATE))
ax.semilogy([k_old], [ratio_old], "*", color=PALETTE[7], ms=16,
            label=f"first cross-check, before the repair: {ratio_old:.0f}")
ax.axhline(1.0, color="k", lw=1.2, label="ratio 1: the tolerance")
```

`order` sorts the 45 states by B_{direct}, largest first, and `ratio_old` is the ratio of the failed first comparison, $8.27 \times 10^{-10} / 7.97 \times 10^{-12} = 103.8$. The ratios of the committed μ (circles) and of the 40-digit roots (open squares, `mfc="none"`) are drawn for each state in that order, the old ratio as a red star at the position of N8_lamm1_a00_T10, and the line ratio 1.

```
ax.set_xticks(positions[::4])
ax.set_xticklabels([ROWS[i]["id"].replace("N8_", "") for i in order][::4],
                   rotation=40, ha="right", fontsize=7)
ax.set_xlabel("state (sorted by the bound of the direct sum, largest first)")
ax.set_ylabel("$|\\mu_{Rust} - \\mu_{ref}|$ / tolerance")
ax.set_ylim(1e-4, 1e3)
ax.legend(fontsize=8, loc="upper right")
fig.tight_layout()
```

Every fourth state is named on the horizontal axis (without the common prefix N8_), and the vertical axis runs from 10^{-4} to 10^{3}.

```
save_figure(fig, "crosscheck_ratios",
            "The cross-check of the chemical potential for the 45 thermal states with "
            "$N = 8$: the difference of the two solvers divided by the tolerance "
            "fixed in advance, logarithmic, the states sorted by the rounding bound "
            "of the direct sum. Circles: the committed values after the repair; "
            "squares: the 40-digit roots on each solver's levels; all lie far below "
            "the line 1. The star is the first comparison in N8_lamm1_a00_T10, "
            f"before the repair, with the ratio {ratio_old:.0f}: the rule caught a "
            "rounding error of the Rust solver, and the repair, not a wider "
            "tolerance, removed it.")
```

`save_figure` saves Figure 16c.6. What the student should see: the star of the first comparison, at about 104, lies two orders of magnitude above the line 1; after the repair, the same rule gives ratios between about 5×10^{-4} and 0.08 for every state. The rule caught the error, and the repair, not a wider tolerance, removed it.

```
report_detail = next(c["detail"] for c in read_json(
    f"{KS}/reports/ks-crosscheck.json")["checks"]
    if c["name"] == "thermo_mu_high_precision")
worst_all = float(re.search(r"worst \|diff\|/tolerance ([0-9.]+)",
                            report_detail).group(1))
say(f"first comparison: ratio {ratio_old:.1f}. After the repair, over these 45 "
    f"states: mu at most {max(r['ratio_mu'] for r in ROWS):.4f}, 40-digit roots at "
```

```
    f"most {max(r['ratio_hp'] for r in ROWS):.4f} (the report, over all 135 states: "
    f"40-digit roots at most {worst_all})")
check(ratio_old > 1.0 and max(r["ratio_hp"] for r in ROWS) <= worst_all + 5e-4
      and max(r["ratio_mu"] for r in ROWS) < 0.1,
      "the first comparison failed the rule; after the repair all ratios are small",
      record=f"{KS}/reports/ks-crosscheck.json, check thermo_mu_high_precision")
```

The worst ratio of the 40-digit comparison over all 135 thermal states is read from the detail of the cross-check's check `thermo_mu_high_precision` (0.05). Out [14] prints the old ratio, 103.8, and the largest ratios over the 45 states after the repair: 0.0825 for the committed μ and 0.0270 for the 40-digit roots. The check requires the old ratio above 1, our 40-digit ratios no larger than the report's worst, and every ratio of the committed μ below 0.1.

In [15], the last check.

```
figure_files = [f"{FIGURE_FOLDER}/16c_{k}_{name}.png" for k, name in enumerate(
    ["levels_and_occupations", "residual_staircase", "convergence_paths",
     "errors_versus_bound", "conditioning_map", "crosscheck_ratios"], start=1)]
check(all(output_file(f).is_file() for f in figure_files),
      "every figure file of this notebook exists")
all_checks_passed()
```

As In [23] of Notebook 16a, for the six figures; it prints ALL 26 CHECKS PASSED (notebook 16c): seven checks in In [11], three each in In [2], In [4], In [5] and In [7], two in In [9], and one each in In [3], In [8], In [13], In [14] and In [15].

16.25 What the cross-check does not test

The cross-check is a test of the **numerics**: of the discretisation, the rounding and the programming of the Rust solver, against a second program that differs in all three. Everything that both programs take from the same source is outside its reach, and the record says so (`Revision/kohn_sham/reference/README.md`, the section on what the reference does not establish). Precisely:

- **The functional.** Both solvers read the same theory file `Revision/kohn_sham/ks-theory.json`: the contact interaction with Hartree and the exact local exchange of the uniform gas, and no correlation (Chapters 13 and 14). If that functional were a poor approximation for the field, both programs would agree on the wrong answer.
- **The boundary conditions.** Both use the ASSUMED Z2 mirror brane at $y = 0$ and the CHOSEN regular tip at the cutoff $y = -L$ with $L = 3$. The cross-check compares the two solvers at this same cutoff, so it cannot show how the results depend on L. That dependence is measured separately, in `Revision/kohn_sham/tip_convergence/` (report `tip-convergence.json`, 7 of 7 checks PASS): a copy of the Rust solver that reads L from the environment is run from $L = 3$ to 6 for 29 of the recorded states, and the reference solver, run for two of these states (N8_lamp1_a00 and N136_lam0_a20) at $L = 3$ and at $L = 4$, agrees with it in all 28 comparisons, seven quantities for each state and each L (the Kohn-Sham energy E_{KS}, the interaction energy, HOMO, LUMO and the integrals of p_0, n and ρ; check `reference_solver_at_two_L`). Its worst ratio, 0.005, is measured against that study's own tolerance

$10\,(U_{ref} + U_{Rust}) + 10^{-9}\max(1, |x|)$, which is looser than the rule of Section 16.7, so it cannot be compared with the 0.495 of the cross-check. The study finds that the recorded free results, except $N = 688$ at $a_{4,0} = 0$ (whose energy does not converge up to $L = 6$), lie within 1.4% (E_{KS}) and 12% (the energy-momentum integral of p_0) of their large-L values, while the interaction effects depend strongly on L: the recorded Kohn-Sham energy E_{KS} of the interacting states with $N = 8$ is about one third of its large-L value. Agreement of the two programs at $L = 3$ says nothing about these differences.

- **The filling convention.** Both fill only the positive branch and the brane zero modes and never populate the sea, not even thermally (a CONVENTION whose justification is OPEN). The sea-hole diagnostic, which both programs compute and which agree with each other (worst ratio 0.048), shows where this convention leaves its range of validity: in 15 of the 135 thermal states the excluded sea would carry more than 1% of N, at most $30.98N$ (`Revision/kohn_sham/reports/ks-crosscheck.json`, check `thermo_sea_hole_diagnostic`). Agreement of the two programs on this diagnostic says nothing about whether the convention is right.

- **The history and the adiabatic states.** Both compute instantaneous (adiabatic) Kohn-Sham states along the PRESCRIBED BACKGROUND history $a_4 = AHx_4$ with $A = 1$. The history is not solved for: the Kohn-Sham states violate the conditions that the a_4 field equations put on their source (`Revision/field_equations_a4/reports/ks-source-conditions.json`, Chapter 17), and the time-dependent (non-adiabatic) problem is OPEN.

- **The good sector.** Both assume that the quanta do not depend on the extra times x_5, x_6, x_7 (ASSUMED, Chapter 14).

Of these inputs, only the tip cutoff has been varied in a study of its own, and only in part: L from 3 to 6 for 29 recorded states in `Revision/kohn_sham/tip_convergence/` (above). Several of the other inputs are probed by diagnostics, which can show where an input fails but cannot confirm it: the filling convention by the sea-hole diagnostic and the prescribed history by the source conditions of the a_4 equations (both above), the exchange part of the functional by the exchange difference ΔE_x and the exact-Fock variant (Section 14.27), and the instantaneous (adiabatic) states by the adiabaticity measure Q_{max} and the search for Fermi-level crossings (Section 15.21). Even for the tip cutoff the large-L limit is not established for $N = 688$ at $a_{4,0} = 0$, nor for the 7 interacting states whose self-consistent iteration fails before the successive differences of E_{KS} meet that study's rule for a geometric tail, nor, in 8 further interacting states, for some quantities other than E_{KS} (the interaction energy and the integrals of p_3 and p_t among them). A test that could confirm the other inputs would need something other than a second program for the same equations: an exact functional for the quantised field, a justification or a replacement of the filling convention, a solution of the coupled problem for a_4 and the gas, a time-dependent calculation. These are OPEN problems (Chapter 22). Nothing in this chapter concerns pairs of universes, their creation, or matter and antimatter.

16.26 What we proved, what we computed, what we assumed

PROVED (derived line by line in this chapter; where the Revision record holds the statement, its report and check are named):

- The convergence ratio $(x(h) - x(h/2))/(x(h/2) - x(h/4)) = 4(1 + \frac{15}{16}\frac{d}{c}h^2 + \dots)$ for an error with even powers of h (Section 16.4).
- Richardson extrapolation: $r(h) = (4x(h/2) - x(h))/3 = X - \frac{1}{4}dh^4 + \dots$, $R = (64x(h/4) - 20x(h/2) + x(h))/45 = X + O(h^6)$, and $R - r(h/2) = \frac{1}{64}dh^4 + O(h^6)$, so that the stated U over-estimates the error of R when h is small (Section 16.5).
- The Rust uncertainty $U_{Rust} = \frac{16}{15}|x_c - x_r|$ for a fourth-order method whose step is halved, and in general the factor $2^p/(2^p - 1)$ for order p (Section 16.6, Exercise 3).
- The tolerance rule as a consequence of the triangle inequality, and the propagated uncertainties $U_F = U_E + TU_S$, $U_\Omega = U_F + NU_\mu$, $3(U_{I_3} + U_{I_t})$ for dE/da_4, and the factor 2 for ΔE_x (Section 16.7).
- The exact free levels at zero momentum: 0 and $\pm\sqrt{M^2 + (n\pi/L)^2}$ for even parity, $\pm\sqrt{M^2 + p_l^2}$ with $\tan(p_l L) = -p_l/M$ (one root in each interval) for odd parity, and the zero mode $A\,e^{My}$ with $A = \sqrt{2M/(1 - e^{-2ML})}$ (Section 16.13; `Revision/kohn_sham/reports/ks-theory-python.json`, check bc_exact_k0_spectra).
- The staggered discretisation: second-order centred differences and averages with even-power errors, the symmetric tridiagonal matrix, the rotated frame for odd parity with $m_2 = M\cos 2\phi - K\sin 2\phi$, $k_2 = M\sin 2\phi + K\cos 2\phi$ and the extra term ϕ' (Section 16.14).
- The pivot recursion $q_r = d_r - x - o_{r-1}^2/q_{r-1}$ and, with Sylvester's law, the Sturm count; Gershgorin's bound; the number of bisection steps (Section 16.15).
- The exact discrete zero mode $u_p = q^p u_0$, $q = (1 + Mh/2)/(1 - Mh/2)$, $\ln q = Mh + (Mh)^3/12 + \dots$; the Hellmann-Feynman theorem for symmetric matrices; the brane-band slope $c = e^{-a_{4,0}}\frac{2M}{1-e^{-2ML}}\frac{1-e^{-(2M-H)L}}{2M-H}$ (Section 16.15; same report, check brane_band_slope).
- The slope $dN/d\mu = \frac{1}{T}\sum gf(1-f) > 0$ and the uniqueness of the chemical potential; the two-level estimate $\mu \approx \frac{\varepsilon_0+\varepsilon_1}{2} - \frac{T}{2}\ln\frac{g_1}{g_0}$; Newton's method and its quadratic convergence; the width $3 \cdot 2^{-51}/N'$ of the zero interval of the direct sum near $N = 8$; the exact rewriting $W = -d - H_{th} + P$ of the Mermin residual (Section 16.20).

Quoted, not proved here: Sylvester's law of inertia and the spectral theorem for symmetric matrices (Sections 16.14 and 16.15). The first-order rounding bounds B_{direct} and B_{well} are derived in the Rust solver's documentation (`Revision/kohn_sham/solver/src/mermin.rs`); this chapter derives their main terms.

COMPUTED (numbers from the records named, each reproduced by a notebook of the chapter):

- The reference solver's self-checks: 37 checks, all PASS (`Revision/kohn_sham/reports/ks-reference.json`); among them the exact free levels reproduced to 2.8×10^{-14}, convergence ratios between 3.99977 and 4.00019, the brane-band slope to 2.6×10^{-15}, and the fourth-grid validation with the largest ratio 0.787 (Notebooks 16a and 16b).
- The full cross-check: 31 checks, all PASS, 162691 comparisons, no ratio above 0.495 (the report `Revision/kohn_sham/reports/ks-crosscheck.json` and its table `ks-crosscheck-table.csv` in the same folder). Notebook 16a reproduces, for five states, the reference results

byte for byte, the Rust canonical results exactly, the canonical-minus-refined differences of `Revision/kohn_sham/checker/rust-refinement.json`, 97 table rows, and the worst cases of six classes; all 5140 of its comparisons pass, the largest with the ratio 0.4952.

- The chemical potential: the faulty value 0.2100104489071649 of the first cross-check reproduced in all sixteen digits by bisection on the direct sum; the 40-digit roots 0.21001044973390… (Rust levels) and 0.21001044973430… (reference levels), 4.0×10^{-13} apart; the bounds $B_{direct} = 1.287 \times 10^{-8}$ and $B_{well} = 3.154 \times 10^{-16}$ of N8_lamm1_a00_T10 and of all 45 thermal states with $N = 8$ (`Revision/kohn_sham/reports/ks-rust-mermin-roots.json`; Notebook 16c).

ASSUMED:

- that the grid errors have the assumed forms (even powers of h for the reference, fourth order for the Rust solver), tested by the ratio test on every level and by a fourth grid for one state, not proved for the self-consistent problem;
- that the measured uncertainties bound the errors (the factor 3 of the tolerance rule allows for estimates that are somewhat too small); Section 16.20 shows one way this can fail: a repeated computation that shares a rounding error does not measure it;
- the inputs shared by both programs, listed in Section 16.25: the functional, the Z2 mirror brane, the good sector; CHOSEN: the regular tip and $L = 3$ (the cross-check compares the two programs at this L; the dependence on L is measured separately, Section 16.25); CONVENTION, justification OPEN: the filling of the levels; PRESCRIBED BACKGROUND: the history $a_4 = AHx_4$.

HYPOTHESIS: none is used in this chapter. **OPEN:** the time-dependent (non-adiabatic) problem, the justification of the filling convention, any test that could confirm the shared inputs beyond the study of the tip cutoff L (29 recorded states in `Revision/kohn_sham/tip_convergence/`) and the diagnostics of Section 16.25, which can show where an input fails but cannot confirm it, and the large-L limit wherever that study does not establish it. It establishes the limit (numerically, with uncertainties) only for the states and quantities that its `README.md` marks converged or with a geometric tail, and no limit at all for $N = 688$ at $a_{4,0} = 0$ or for the 7 interacting states whose self-consistent iteration fails before the successive differences of E_{KS} meet its rule for a geometric tail. In 8 further interacting states some quantities other than E_{KS} do not meet the rule (the interaction energy, the integrals of p_3 and p_t, LUMO, the Kohn-Sham gap; `tip-convergence.json`, `analysis.fixed`), and the recorded states and quantities that the study does not compute at $L \neq 3$ are not covered.

16.27 Exercises

Exercise 1 (Richardson by hand). Notebook 16a prints the Kohn-Sham energy of N8_lamm2_a00 on the three grids: $E_{KS}(300) = 0.002862539189$, $E_{KS}(600) = 0.002862623929$, $E_{KS}(1200) = 0.002862645113$. Compute the convergence ratio, the one-step values $r(h)$ and $r(h/2)$, the three-grid value R and its uncertainty U. Compare R with the notebook's 0.0028626521737 and explain the difference.

Answer. The differences are $x(h)-x(h/2)=-8.4740\times10^{-8}$ and $x(h/2)-x(h/4)=-2.1184\times10^{-8}$, and their ratio is 4.0002: second order. Then $r(h)=(4\cdot0.002862623929-0.002862539189)/3=(0.011450495716-0.002862539189)/3=0.008587956527/3=0.0028626521757$, and $r(h/2)=(4\cdot0.002862645113-0.002862623929)/3=(0.011450580452-0.002862623929)/3=0.008587956523/3=0.0028626521743$. So $R=(16\,r(h/2)-r(h))/15=r(h/2)+(r(h/2)-r(h))/15=0.0028626521743-0.0000000000001=0.0028626521742$, and $U=|R-r(h/2)|+2\cdot10^{-12}=0.9\times10^{-13}+2\times10^{-12}=2.1\times10^{-12}$, as in the notebook. Our R differs from the notebook's 0.0028626521737 by about 5×10^{-13}, because the printed grid values are rounded to 10^{-12}: the weights of R are $\frac{64}{45}$, $-\frac{20}{45}$ and $\frac{1}{45}$, so rounding errors of up to 0.5×10^{-12} in the inputs can move R by up to $\frac{64+20+1}{45}\cdot0.5\times10^{-12}=0.94\times10^{-12}$. The notebook computes with the unrounded values. (Useful identity: $R=r(h/2)+(r(h/2)-r(h))/15$, which follows from $16r(h/2)-r(h)=15r(h/2)+(r(h/2)-r(h))$.)

Exercise 2 (the worst comparison). The worst comparison of the whole cross-check is the energy density at the tip of N8_lamm2_a00: Rust gives -10.1822650611886, the reference -10.182265056614, and the tolerance is 9.238×10^{-9} (`Revision/kohn_sham/reports/ks-crosscheck.json`, check emt_brane_tip_values). Compute the difference and the ratio, and the sum $U_{ref}+U_{Rust}$ of the two uncertainties.

Answer. $|x_{Rust}-x_{ref}|=10.1822650611886-10.182265056614=4.5746\times10^{-9}$. The ratio is $4.5746\times10^{-9}/9.238\times10^{-9}=0.495$, the largest ratio of the record. The tolerance is $3(U_{ref}+U_{Rust})+10^{-12}\max(1,10.18)$, so $U_{ref}+U_{Rust}=(9.238\times10^{-9}-1.018\times10^{-11})/3=3.076\times10^{-9}$. The difference is about 1.5 times this sum. Since the difference can exceed the sum of the two UNCERTAINTIES only if at least one error exceeds its uncertainty (Section 16.7: the difference never exceeds the sum of the two errors), at least one of the two estimates is too small at the tip, by a factor of up to about 1.5. The factor 3 of the rule absorbs this, which is why it is there. In the whole cross-check only two classes have a largest ratio above $\frac{1}{3}$, where the factor 3 is needed at all: emt_brane_tip_values and ground_profiles, both with 0.495 at this tip value; the next largest class maximum is 0.324 (ground_eigenvalues; the table of Section 16.8).

Exercise 3 (the factor for a method of order p). A method has the error $x(h)=X+c\,h^p+\ldots$. It is run with the step h (result x_c) and with $h/2$ (result x_r). Show that the error of x_c is $\frac{2^p}{2^p-1}(x_c-x_r)$, and evaluate the factor for $p=2$ and $p=4$. Use it to compute U_{Rust} for E_{KS} of N688_lam0_a00, where Notebook 16a measures $|x_c-x_r|=7.529\times10^{-10}$.

Answer. $x_c=X+e$ with $e=ch^p$, and $x_r=X+c(h/2)^p=X+e/2^p$. Subtracting, $x_c-x_r=e(1-2^{-p})=e\,\frac{2^p-1}{2^p}$, so $e=\frac{2^p}{2^p-1}(x_c-x_r)$. For $p=2$ the factor is $\frac{4}{3}$, for $p=4$ it is $\frac{16}{15}$, the factor of Section 16.6. For N688_lam0_a00: $U_{Rust}=\frac{16}{15}\cdot7.529\times10^{-10}=8.031\times10^{-10}$. (The same algebra gives Richardson's first step: $X=x_r-(x_c-x_r)/(2^p-1)$, which for $p=2$ is $(4x_r-x_c)/3$.)

Exercise 4 (a Sturm count by hand). Take the even-parity matrix of Section 16.14 for $G=2$ cells ($h=1.5$, $M=1$, $j=+1$): the unknowns are u_0,w_1,u_1, and $T=\begin{pmatrix}0&a&0\\a&0&b\\0&b&0\end{pmatrix}$ with $a=\frac{1}{h}+\frac{M}{2}$, $b=-\frac{1}{h}+\frac{M}{2}$. (i) Compute a, b and the three eigenvalues. (ii) Compute the pivots at $x=0.5$ and the Sturm count $c(0.5)$. (iii) Construct the discrete zero mode. (iv) Compare the positive eigenvalue with the exact level 1.447972.

Answer. (i) $a=\frac{2}{3}+\frac{1}{2}=\frac{7}{6}$ and $b=-\frac{2}{3}+\frac{1}{2}=-\frac{1}{6}$. The determinant of $T-\varepsilon\mathbb{1}$, expanded along the first row, is $-\varepsilon(\varepsilon^2-b^2)-a(-a\varepsilon)=-\varepsilon(\varepsilon^2-a^2-b^2)$, so the eigenvalues are 0 and $\pm\sqrt{a^2+b^2}=\pm\sqrt{\frac{49}{36}+\frac{1}{36}}=\pm\sqrt{\frac{50}{36}}=\pm1.178511$. (ii) $q_1=0-0.5=-0.5$; $q_2=0-0.5-a^2/q_1=$

$-0.5+1.361111/0.5 = -0.5+2.722222 = 2.222222$; $q_3 = 0-0.5-b^2/q_2 = -0.5-0.027778/2.222222 = -0.5-0.0125 = -0.5125$. Two pivots are negative, so $c(0.5) = 2$: indeed two eigenvalues, -1.178511 and 0, lie below 0.5. (iii) $q = (1+Mh/2)/(1-Mh/2) = 1.75/0.25 = 7$, so $u_1 = 7u_0$ and $w_1 = 0$. Check the row of w_1: $a\,u_0 + b\,u_1 = \frac{7}{6}u_0 - \frac{1}{6}\cdot 7u_0 = 0$. Normalised with $\sum u^2 h = 1$: $u_0^2(1+49)\cdot 1.5 = 1$, so $u_0 = 0.11547$, $u_1 = 0.80829$. (iv) The coarse grid gives 1.178511 instead of 1.447972, an error of 0.27: with only two cells the h^2 error is large, and Notebook 16b shows how it shrinks by a factor 4 per halving of h.

Exercise 5 (how many halvings). For the even-parity matrix with $G = 300$ cells ($M = 1$, $L = 3$, $k = 0$), find Gershgorin's starting interval and the number of bisection steps needed to locate the zero mode to 10^{-14}.

Answer. $h = 3/300 = 0.01$, so the neighbour entries are $\frac{1}{h} + \frac{1}{2} = 100.5$ and $-\frac{1}{h} + \frac{1}{2} = -99.5$. Every inner row has the radius $100.5 + 99.5 = 200$ and the diagonal is zero, so the interval is $[\min(d-\rho) - 1, \max(d+\rho) + 1] = [-201, 201]$, of width $W = 402$. After n halvings the width is $402/2^n$, and $402/2^n \le 10^{-14}$ needs $2^n \ge 4.02 \times 10^{16}$, that is $n \ge \log_2(4.02 \times 10^{16}) = 55.2$: 56 halvings. (For $G = 150$ the same reasoning gives 55, the number of Notebook 16b; doubling G doubles W and costs one more halving.)

Exercise 6 (the accuracy of the discrete zero mode). For $G = 150$ ($h = 0.02$, $M = 1$), compute $\ln q$ to nine decimals with the series of Section 16.15 and compare with Mh. By how much does the discrete exponential grow faster than e^{My} across the whole interval $L = 3$, and is this consistent with the distance 6.93×10^{-5} that Notebook 16b measures?

Answer. $\ln q = Mh + (Mh)^3/12 + \cdots = 0.02 + 0.000008/12 = 0.02 + 0.000000667 = 0.020000667$ (the next term, $(Mh)^5/80 = 4 \times 10^{-11}$, does not change these digits). So the discrete exponential grows like $e^{1.0000333\,y}$ instead of e^y: over the 150 cells the exponents differ by $150 \cdot 6.67 \times 10^{-7} = 1.0 \times 10^{-4}$. Two effects of order h^2 make up the measured distance, and both raise the discrete values near the brane, where a is largest ($A\,e^{My} = 1.402$ at the last half node $y = -0.01$). (i) The rate. A mode e^{ry} normalised to $\int a^2 dy = 1$ has $a(0)^2 = 2r/(1 - e^{-2rL})$; increasing $r = M$ by the relative amount $(Mh)^2/12 = 3.33 \times 10^{-5}$ raises $\ln a$ near the brane by about $(\frac{1}{2M} - \frac{Le^{-2ML}}{1-e^{-2ML}} + y) \cdot 3.33 \times 10^{-5} = (0.5 - 0.0075 - 0.01) \cdot 3.33 \times 10^{-5} = 1.61 \times 10^{-5}$, that is by $1.61 \times 10^{-5} \cdot 1.402 = 2.3 \times 10^{-5}$ in absolute size. (ii) The normalisation. The notebook normalises with the midpoint sum $\sum u^2 h$, not with the integral. Over one cell centred at y_p the integral of e^{2My} is $e^{2My_p}(e^{Mh} - e^{-Mh})/(2M) = h\,e^{2My_p}\,\frac{\sinh(Mh)}{Mh} = h\,e^{2My_p}(1 + \frac{(Mh)^2}{6} + \dots)$, so the sum is smaller than the integral by the relative amount $(Mh)^2/6 = 6.7 \times 10^{-5}$; dividing by the square root of a number that is too small by this amount raises the normalised values by half of it, 3.3×10^{-5}, which is 4.7×10^{-5} at the value 1.402. Together: $2.3 \times 10^{-5} + 4.7 \times 10^{-5} = 7.0 \times 10^{-5}$, consistent with the measured 6.93×10^{-5}. (To test the split yourself, compute the half nodes `y = -L + (np.arange(G) + 0.5) * h` at the start of the function `zero_mode` of Notebook 16b and change the function in two ways. First, with `u = np.exp(M * y)`, the values e^{My} with the exact rate M, and the normalisation by the sum kept, only part (ii) remains: the distance printed by the notebook drops to about 4.7×10^{-5}. Second, with `r = log_q / h` and `u = np.exp(r * y)`, the values e^{ry} with $r = \ln q/h$, divided by `math.sqrt((1 - math.exp(-2 * r * L)) / (2 * r))`, the square root of the exact integral of e^{2ry} from $-L$ to 0, in place of the sum, only part (i) remains: about 2.3×10^{-5}. The values $u_p = q^p$ of the notebook are $e^{r(y_p+L-h/2)}$, larger than e^{ry_p} by a constant factor, so they must be replaced by e^{ry} before the exact integral is used.) Both are of order h^2: $(Mh)^2/12$ and $(Mh)^2/6$.

Exercise 7 (the chemical potential of N8_lamm1_a00_T10 by hand). With the levels $\varepsilon_0 = 0.000245562708$ ($g_0 = 8$), $\varepsilon_1 = 0.430761462627$ ($g_1 = 24$), $\varepsilon_2 = 0.587973061749$ ($g_2 = 48$) and

$T = 0.01$: (i) evaluate the two-level estimate of Section 16.20; (ii) show that the thermal particles of the level ε_2 lower μ by about $\frac{T}{2}P_2/P_1$ and evaluate the correction; (iii) compare with the 40-digit root 0.2100104497339.

Answer. (i) $\mu_0 = \frac{0.000245562708+0.430761462627}{2} - 0.005\ln 3 = 0.215503512668 - 0.005493061443 = 0.210010451225$. (ii) The Mermin condition $H_{th} = P_1 + P_2$ reads $g_0 e^{-(\mu-\varepsilon_0)/T} = g_1 e^{-(\varepsilon_1-\mu)/T}(1 + P_2/P_1)$. Taking logarithms as in Section 16.20 gives $\mu = \mu_0 - \frac{T}{2}\ln(1 + P_2/P_1) \approx \mu_0 - \frac{T}{2}\,\frac{P_2}{P_1}$, because $\ln(1 + s) \approx s$ for small s. With $\mu \approx 0.2100104$: $P_2 = 48\,e^{-(0.587973-0.210010)/0.01} = 48\,e^{-37.796} = 48{\cdot}3.85{\times}10^{-17} = 1.85{\times}10^{-15}$ and $P_1 = H_{th} = 6.21{\times}10^{-9}$, so $P_2/P_1 = 2.97{\times}10^{-7}$ and the correction is $0.005 \cdot 2.97 \times 10^{-7} = 1.49 \times 10^{-9}$. (iii) $\mu \approx 0.210010451225 - 0.000000001488 = 0.210010449737$, within about 3×10^{-12} of the 40-digit root 0.2100104497339 (the remaining difference comes from the approximations $1/(1 + e^s) \approx e^{-s}$ and $\ln(1 + s) \approx s$, and from rounding the levels to twelve decimals). The direct sum in doubles can hardly see the level ε_2: its particles, 1.8×10^{-15}, are one or two spacings of the doubles near 8, so the direct sum blurs an effect that moves μ by 1.5×10^{-9}, the size of the error of the first cross-check.

Exercise 8 (another activated state). For the state N8_lam0_a00_T10 the record gives $n = 5$ levels and $dN/d\mu = 1.229 \times 10^{-6}$ (`Revision/kohn_sham/reports/ks-rust-mermin-roots.json`). Estimate the width of the interval of μ on which the direct residual is exactly zero, and the main term of B_{direct}. Compare with the error of the direct sum for this state printed by Notebook 16c ($+2.72 \times 10^{-10}$).

Answer. The width is $3 \cdot 2^{-51}/N' = 1.332 \times 10^{-15}/1.229 \times 10^{-6} = 1.08 \times 10^{-9}$, as for N8_lamm1_a00_T10, because N' is almost the same. The main term of the bound is $(n + 2)\,\epsilon_{mach} N/N' = 7 \cdot 2.220 \times 10^{-16} \cdot 8/1.229 \times 10^{-6} = 1.012 \times 10^{-8}$, the record's 1.012×10^{-8} (the other terms are about 10^{-16}). The actual error, 2.72×10^{-10}, lies inside the zero interval's width and far inside the bound. Its sign is positive here: where the zero interval lies relative to the root depends on how the individual terms of the sum are rounded, so bisection can stop above or below the root. A bound tells how large an error can be; it does not tell its sign.

17 The a4 equations with the Kohn-Sham source: a prescribed background

Chapters 14 to 16 computed the Kohn-Sham states of the field dirac16complex in the author's primordial universe, at five instants of the history $a_4 = Hx_4$ along which 3-space inflates and the three extra times deflate exponentially. Chapter 12 derived the field equations that gravity imposes on the function $a_4(x_4)$. This chapter puts the two together. It asks one question: if the energy and momentum of the Kohn-Sham gas are put on the right-hand side of the field equations of gravity, is the history $a_4 = Hx_4$ a solution? The answer is no, and the chapter proves exactly why. Two complete notebooks repeat every step: Notebook 17a tests the three conditions that the field equations impose on every source against all 75 recorded Kohn-Sham ground states, and Notebook 17b shows that the failure is forced by the conservation of energy and momentum itself.

17.1 What this chapter does, and why

The question. In Einstein's theory, and in its generalisation used in this book, matter and geometry are tied together: the curvature of the metric equals a constant times the energy and momentum of the matter. In Chapters 14 and 15 we did something weaker. We took the author's metric with the history $a_4 = Hx_4$ as GIVEN, and we computed how a gas of dirac16complex quanta settles into its lowest state at five instants of that history. We never asked whether the gas, through its own energy and momentum, would make the metric that it moves in. This chapter asks that question.

Why it matters. If the answer were yes, the history $a_4 = Hx_4$, with the gas in it, would be a solution of the coupled equations of gravity and matter: a possible universe of this theory. Every energy, pressure and equation of state computed along it would then be a prediction of the theory. If the answer is no, the history is a **prescribed background**: a stage that we chose, on which the gas moves as a **test field** whose own gravity is ignored. Its energies and pressures are still well-defined numbers, but they describe a gas in a chosen geometry, not a self-consistent universe. The Revision record states the answer in `Revision/field_equations_a4/reports/ks-source-conditions.json`: no recorded Kohn-Sham state is an admissible source (the record tests the 75 ground states, the only states whose profiles it stores), and the history is a prescribed background without back-reaction. This chapter derives that statement from the beginning and checks every number of it.

What is derived, in order.

- The field equations of a_4 and the most general source they allow (Section 17.3).
- The three **source conditions**: no dependence on the hidden coordinate x_0 (C1, Section 17.4), the algebraic condition $p_3 + p_t = 2p_0$ (C2, Section 17.5), and, for the linear history $a_4 = AHx_4 + a_0$, equal pressures and a constant energy density (C3, Section 17.6). They are derived on the background $b_4 = 1$ of the author's metric of 2026-10-14, on which the Kohn-Sham states are computed; the record states that the verdict holds for every second function $b_4(x_4)$: C1 holds unchanged, so the x_0-dependent Kohn-Sham profiles are not an admissible source for any b_4 (`Revision/field_equations_a4/reports/ks-source-conditions.json`, check `ks_verdict_holds_for_every_b4`; Section 12.12 gives the field equations with b_4).
- What the 75 recorded Kohn-Sham ground states are and which numbers of them we use (Section 17.7); then Notebook 17a, which evaluates C1, C2 and C3 on all of them (Sections 17.8 to 17.11).
- Why the states fail: every source of the field equations must be conserved, and what the Kohn-Sham states conserve (Section 17.12); the conservation law in the hidden coordinate y (Section 17.13); for a conserved source C2 says exactly that the hidden pressure p_0 is flat (Section 17.14); the integrated form and the averaged ratio (Section 17.15); the energy exchange along the history and the propagation of the constraint, which turn C3 into $p_3 = p_t$ (Section 17.16).
- The field equations averaged over the hidden direction: the moment equations, which of them can hold together, their first integral, and what can and cannot be integrated with the Kohn-Sham source, within a stated approximation (Section 17.17); then Notebook 17b (Sections 17.18 to 17.21).

- What "prescribed background" means, and what remains open (Section 17.22); the summary of statuses (Section 17.23); exercises with complete answers (Section 17.24).

The two notebooks.

notebook	what it computes	PASS lines	figures
17a	the three source conditions derived from the record, evaluated on the 75 recorded Kohn-Sham ground states	16	7
17b	the conservation law in the author's metric; why a conserved source fails C2 and C3; the averaged equations and their first integral	20	6

Neither needs Rust: both read the stored results of the Revision Rust solver and the Revision reports, and compute everything else with numpy and sympy.

The status of every statement. As everywhere in this book, every statement carries one of five labels: PROVED (exact, with the verifier file and check name), COMPUTED (numerical, with its measured error), ASSUMED, HYPOTHESIS, OPEN. In this chapter:

- PROVED: the three source conditions (Sections 17.4 to 17.6), the conservation law and its consequences (Sections 17.12 to 17.16), the moment equations and their first integral (Section 17.17), and the absence of a solution without matter in Einstein gravity.
- COMPUTED: every statement about the 75 recorded ground states (Notebooks 17a and 17b), each with the record file it reproduces.
- ASSUMED: the history $a_4 = Hx_4$ on which the Kohn-Sham states were computed (it is the prescribed background); the good sector, the Z2 mirror at the brane and the tip cutoff of the Kohn-Sham model (Chapter 14); in Section 17.17, the approximation that keeps only two of the averaged field equations (it is stated there with its error).
- OPEN: what metric the Kohn-Sham gas would produce with back-reaction, and whether any state of dirac16complex is an admissible source of the author's metric (Section 17.22).
- HYPOTHESIS: none in this chapter.

Honesty about what this chapter is not. Nothing in this chapter concerns the creation of universes, pairs of universes, or matter and antimatter. In particular the sign changes that appear in it (a pressure that changes sign along the hidden direction, the states with couplings $+\lambda$ and $-\lambda$ in figure 4 of Notebook 17b) are properties of particular recorded states; they are not the pairing theorems of Chapters 18 to 20.

17.2 The words and symbols of this chapter

Each word is defined in plain terms here; the later sections make each definition precise with formulas.

- **Coordinates** $x_0, \ldots, x_7$, in the author's order: x_1, x_2, x_3 are ordinary 3-space, which inflates; x_4 is the time; x_5, x_6, x_7 are the three **extra times**, which behave like time and **deflate exponentially**; x_0 is the hidden space direction. The angle $z = 6Hx_0$ runs from 0 to $\pi/2$; $H > 0$ is a constant of the author's metric, with the unit of an inverse length.

- **Prime**: on a_4 it means the derivative with respect to the time, $a_4' = da_4/dx_4$, $a_4'' = d^2a_4/dx_4^2$; the code writes them `ad1` and `ad2`. On a function of the hidden coordinate it means d/dy; we always say which.

- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: a second way to number the points of the hidden direction, used by the Kohn-Sham computations. It grows when x_0 grows. $y = 0$ is the end $z = \pi/2$, called the **brane**; $y \to -\infty$ is the end $z \to 0$, called the **tip**. The Kohn-Sham computations stop at $y = -L = -3$, the **tip cutoff**; the region $-3 \le y \le 0$ is the **patch**.

- **Metric**, **scale factor**: the table $g_{\mu\nu}$ that turns coordinate steps into squared lengths, and the number $\sqrt{|g_{\mu\mu}|}$ by which a step along one direction must be multiplied to give a length.

- **Field equations of gravity**: $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^{\mu}{}_{\nu} + \Lambda\, \delta^{\mu}_{\nu} = \kappa\, T^{\mu}{}_{\nu}$. On the left are the three **Lovelock tensors** $E_{(1)}$ (the Einstein tensor), $E_{(2)}$ (the Gauss-Bonnet tensor) and $E_{(3)}$, built from the curvature of the metric, weighted by the **couplings** $\alpha_1, \alpha_2, \alpha_3$, and the **cosmological constant** Λ. $\kappa > 0$ measures the strength of gravity. **Einstein gravity** is $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$. The symbol δ^{μ}_{ν} is 1 when $\mu = \nu$ and 0 otherwise.

- **Energy-momentum tensor** $T^{\mu}{}_{\nu}$, also called the **source**: an 8×8 table at every point that says how much energy and momentum the matter has and how they flow. Its diagonal parts are the **energy density** $\rho = -T^{x_4}{}_{x_4}$ and the **pressures** p_3 (each direction of 3-space), p_t (each extra time) and p_0 (the hidden direction); its possible **mixed** parts are $q_{40} = T^{x_4}{}_{x_0}$ and $q_{04} = T^{x_0}{}_{x_4}$.

- **Admissible source**: a source for which the field equations of the author's metric can hold. **Source conditions** C1, C2, C3: the three conditions that an admissible source must meet (Sections 17.4 to 17.6).

- **Linear member**: the history $a_4 = AHx_4 + a_0$ with constants A (the **slope**) and a_0. For $A > 0$ the extra-time scale factor e^{-a_4} shrinks exponentially. The history of the Kohn-Sham computations is $A = 1$, $a_0 = 0$.

- **Slice**: one instant of the history, labelled by the value $a_{4,0}$ of a_4 there; the Kohn-Sham states were computed at the five slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$.

- **Kohn-Sham state**: a many-quantum state of dirac16complex computed with the Kohn-Sham method (Chapters 13 to 15). A recorded state is named like `N136_lamm2_a20`: particle number $N = 136$, coupling $\lambda = -\lambda_2$ (`lamm2`; `lamm1` is $-\lambda_1$, `lam0` is $\lambda = 0$, `lamp1` is $+\lambda_1$, `lamp2` is $+\lambda_2$), slice $a_{4,0} = 2.0$ (`a20`).

- Words of the Kohn-Sham method used again here (Chapters 13 to 15 define them in full): an **orbital** is the wave of one quantum, and an **eigen-orbital** one that solves the Kohn-Sham equation with a definite energy; the **Fermi level** is the energy that separates the occupied orbitals from the empty ones; a state is **self-consistent** when the potentials computed from its own density are the potentials its orbitals were computed with; the **good sector** is the set of orbitals that do not depend on the extra times x_5, x_6, x_7; the **Z2 mirror** is the ASSUMED rule that the hidden direction continues beyond the brane as a mirror image of the computed patch, with the field there fixed by the field on the computed side (Chapter 14 states the rule; record `Revision/kohn_sham/ks-theory.json`, key `boundaryConditions.brane`).

- More words of the Kohn-Sham computations: a **shell** is a group of orbitals with the same energy, and a **closed shell** is a particle number N that fills every orbital up to some energy completely and none above it (so the ground state is unique); the **first-order mean-field potential** is the potential that one quantum feels from the others when the interaction is counted once, to first order in the coupling λ, with the density of the free gas; **fixed occupations** means that the same orbitals stay filled when a_4 is changed a little, even if their energies move.

- **Least squares**: the straight line through a set of points that makes the sum of the squared vertical distances of the points from the line as small as possible.

- **Moment** of a field equation: the equation multiplied by the volume weight e^{6Hy} and integrated over the patch; it involves the source only through its weighted means $\bar{\rho}$, $\bar{p}_3$, $\bar{p}_t$, $\bar{p}_0$. **First integral**: an equation with only first derivatives that every solution of a second-order equation obeys. **Cubic Hermite interpolation**: between two tabulated points, the cubic polynomial with the given values and the given slopes at both ends. **Turning point**: the value of a_4 where a_4' reaches 0. **Branch point**: a value of a_4' where $F(a_4') = 0$, so that the evolution equation cannot be solved for a_4''.

- **Profile**: a component of the source as a function of y, stored at 151 points. **Proper density**: an amount per unit of proper volume, the volume that rulers measure.

- **max|T|**: the largest absolute value of ρ, p_3, p_t and p_0 of a state over the whole patch. We divide by it to compare states of very different size.

- **Integral over the patch** $\int X$: the total amount of a density X in the computed region, $2\,\mathrm{Vol}_7 \int_{-3}^{0} e^{6Hy} X \, dy$ (Section 17.7). **Weighted mean** $\bar{X}$: the integral divided by the proper volume of the patch.

- **Covariant divergence** $\nabla_\mu T^\mu{}_\nu$: the curved-space form of "the change of a quantity in time plus its outflow through the walls of a small box". **Conservation**: zero covariant divergence. **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu}$: combinations of first derivatives of the metric that say how the coordinate directions turn and stretch from point to point. **Bianchi identity**: the left-hand side of the field equations has zero covariant divergence for every metric.

- **Constraint**: the x_4 component of the field equations (it contains a_4' but no a_4''). **Evolution equation**: the 3-space component minus the extra-time component, $a_4'' F(a_4') = \kappa(p_3 - p_t)$.

- **Prescribed background**: a metric that is chosen, not solved for. **Test field**: matter that moves in a prescribed background while its own gravity is ignored. **Back-reaction**: the effect of the matter's own gravity on the metric; a test field has none.

- **Finite difference**: an estimate of a derivative from neighbouring values in a table. Its **order of accuracy** p: its error falls like h^p when the step h shrinks. **Simpson's rule**: an estimate of an integral from equally spaced values, of order 4.
- **Heat map**: a table drawn as coloured squares, one colour per value. **Symmetric logarithmic axis**: an axis that is logarithmic for large positive and large negative values and linear in a narrow band around zero, so that both signs and many orders of magnitude can be shown.
- **sha256 fingerprint**: a 64-digit code computed from the bytes of a file; two files with the same fingerprint are, for all practical purposes, identical.
- **Units**: as in the Revision runs, $H = 1$ and the fermion mass $m = 1$. Lengths are in units of $1/H$; energy densities and pressures (energy per unit of 7-dimensional volume) in units of m^8; integrals over the patch (energies) in units of m.

17.3 The field equations of a4 and their source

The metric. The author's metric is diagonal; in the order $x_0, \dots, x_7$ its entries are

$$g = \mathrm{diag}(e^{2a_4}\sin^{1/3} z,\ e^{2a_4}\sin^{1/3} z,\ e^{2a_4}\sin^{1/3} z,\ -1,\\ -e^{-2a_4}\sin^{1/3} z,\ -e^{-2a_4}\sin^{1/3} z,\ -e^{-2a_4}\sin^{1/3} z,\ \cot^2 z),$$

with $z = 6Hx_0$ and one free function $a_4(x_4)$. The 3-space scale factor is $e^{a_4}\sin^{1/6} z$ and the extra-time scale factor $e^{-a_4}\sin^{1/6} z$: when a_4 grows, 3-space inflates and the extra times deflate.

The source. Chapter 12 allowed the most general source that the symmetry of the metric suggests:

$$T^{\mu}{}_{\nu} = \mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0) + q_{40}\ (\mu = x_4, \nu = x_0) + q_{04}\ (\mu = x_0, \nu = x_4),$$

where every one of $\rho, p_3, p_t, p_0, q_{40}, q_{04}$ may a priori depend on x_4 and on x_0.

The left-hand sides. The Revision record `Revision/field_equations_a4/a4-equations.json` (key `lovelockTensors`) stores every component of the three Lovelock tensors of the author's metric. Writing $a' = a_4'$ and $a'' = a_4''$ for short, the components we need are:

$$E_{(1)}{}^{x_1}{}_{x_1} = 15H^2 - 3a'^2 + a'', \qquad E_{(1)}{}^{x_5}{}_{x_5} = 15H^2 - 3a'^2 - a'',$$
$$E_{(1)}{}^{x_0}{}_{x_0} = 15H^2 - 3a'^2, \qquad E_{(1)}{}^{x_4}{}_{x_4} = 3a'^2 + 21H^2,$$

$$E_{(2)}{}^{x_1}{}_{x_1} = 12a'^4 + 168a'^2H^2 - 180H^4 + a''(-24a'^2 - 40H^2),$$
$$E_{(2)}{}^{x_5}{}_{x_5} = 12a'^4 + 168a'^2H^2 - 180H^4 - a''(-24a'^2 - 40H^2),$$
$$E_{(2)}{}^{x_0}{}_{x_0} = 12a'^4 + 168a'^2H^2 - 180H^4,$$
$$E_{(2)}{}^{x_4}{}_{x_4} = -36a'^4 - 120a'^2H^2 - 420H^4,$$

$$
\begin{aligned}
{E_{(3)}}^{x_1}{}_{x_1} &= -72a'^6 - 648a'^4H^2 - 1944a'^2H^4 + 360H^6 \\
&\quad + a''(360a'^4 + 432a'^2H^2 + 360H^4), \\
{E_{(3)}}^{x_5}{}_{x_5} &= -72a'^6 - 648a'^4H^2 - 1944a'^2H^4 + 360H^6 \\
&\quad - a''(360a'^4 + 432a'^2H^2 + 360H^4), \\
{E_{(3)}}^{x_0}{}_{x_0} &= -72a'^6 - 648a'^4H^2 - 1944a'^2H^4 + 360H^6, \\
{E_{(3)}}^{x_4}{}_{x_4} &= 360a'^6 + 648a'^4H^2 + 1080a'^2H^4 + 2520H^6.
\end{aligned}
$$

We have only regrouped the record's terms: in each x_1x_1 and x_5x_5 component we collected the terms that contain a'' (Exercise 1 asks you to check this against the record's formulas term by term). The components x_2x_2 and x_3x_3 equal x_1x_1, the components x_6x_6 and x_7x_7 equal x_5x_5, and every off-diagonal component, x_4x_0 included, is zero (record: `allOtherComponents`; PROVED, Wolfram report `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `independent_components`; sympy report `Revision/field_equations_a4/reports/python-a4-report.json`, check `other_components_vanish`).

A pattern. Look at the three blocks. For every order k there are two polynomials, P_k in a' and H and Q_k in a' and H, such that

$$
{E_{(k)}}^{x_1}{}_{x_1} = P_k + a''Q_k, \qquad {E_{(k)}}^{x_5}{}_{x_5} = P_k - a''Q_k, \qquad {E_{(k)}}^{x_0}{}_{x_0} = P_k,
$$

namely $P_1 = 15H^2 - 3a'^2$, $Q_1 = 1$; $P_2 = 12a'^4 + 168a'^2H^2 - 180H^4$, $Q_2 = -24a'^2 - 40H^2$; $P_3 = -72a'^6 - 648a'^4H^2 - 1944a'^2H^4 + 360H^6$, $Q_3 = 360a'^4 + 432a'^2H^2 + 360H^4$. We write the weighted sums as

$$
e_{hh} = \sum_{k=1}^{3} \alpha_k {E_{(k)}}^{h}{}_{h} \text{ (no sum over } h), \qquad \mathcal{P} = \sum_{k=1}^{3} \alpha_k P_k, \qquad \mathcal{Q} = \sum_{k=1}^{3} \alpha_k Q_k,
$$

so that $e_{11} = \mathcal{P} + a''\mathcal{Q}$, $e_{55} = \mathcal{P} - a''\mathcal{Q}$ and $e_{00} = \mathcal{P}$.

The equations. With these abbreviations the 64 field equations reduce to four diagonal equations and two mixed ones:

$$
\begin{aligned}
&\text{constraint } (x_4): && e_{44} + \Lambda = -\kappa\rho, \\
&\text{3-space } (x_1 = x_2 = x_3): && \mathcal{P} + a''\mathcal{Q} + \Lambda = \kappa p_3, \\
&\text{extra times } (x_5 = x_6 = x_7): && \mathcal{P} - a''\mathcal{Q} + \Lambda = \kappa p_t, \\
&\text{hidden } (x_0): && \mathcal{P} + \Lambda = \kappa p_0, \\
&\text{mixed } (x_4x_0 \text{ and } x_0x_4): && 0 = \kappa q_{40}, \quad 0 = \kappa q_{04}.
\end{aligned}
$$

The minus sign in the constraint comes from $T^{x_4}{}_{x_4} = -\rho$. Subtracting the extra-time equation from the 3-space equation gives the **evolution equation**:

$$
(\mathcal{P} + a''\mathcal{Q} + \Lambda) - (\mathcal{P} - a''\mathcal{Q} + \Lambda) = \kappa p_3 - \kappa p_t
$$

(rule: subtract the two equations side by side)

$$2a''\mathcal{Q} = \kappa(p_3 - p_t), \qquad \text{that is} \qquad a_4'' F(a_4') = \kappa(p_3 - p_t), \quad F = 2\mathcal{Q}$$

(rule: $\mathcal{P}$ and Λ cancel; $a''\mathcal{Q} - (-a''\mathcal{Q}) = 2a''\mathcal{Q}$). Written out, $F = 2\alpha_1 - \alpha_2(48a'^2 + 80H^2) + \alpha_3(720a'^4 + 864a'^2H^2 + 720H^4)$, the record's `evolution_F` (PROVED: checks `evolution_factorises` of the sympy report and `evolution_factorises_a4pp_times_F` of the Wolfram report). In Einstein gravity $F = 2$ and the evolution equation is $2a_4'' = \kappa(p_3 - p_t)$.

17.4 Condition C1: a source must not depend on x0

The argument. Read the left-hand side of each equation of Section 17.3. It is built from $a_4'(x_4)$, $a_4''(x_4)$, the constant H and the constants α_k and Λ; it contains no x_0, not even through $\cot z$ or $\sin z$. (The metric itself depends on x_0 through z; these components of the Lovelock tensors, with one index up and one down, do not. PROVED: Wolfram report, checks `P1_structure`, `P2_structure`, `P3_structure`, which state that each tensor $P_{(k)}$ is diagonal and free of x_0; the record defines $E_{(k)} = -P_{(k)}/2^{k+1}$, a constant multiple, so the same holds for $E_{(k)}$ (record `a4-equations.json`, key `conventions.lovelock`); reproduced in Notebook 17a, In [4].) Now take the hidden equation at two points with the same time x_4 and two different hidden coordinates x_0 and $\tilde{x}_0$:

$$\kappa\, p_0(x_4, x_0) = \mathcal{P}(x_4) + \Lambda = \kappa\, p_0(x_4, \tilde{x}_0)$$

(rule: the hidden equation at both points; its left-hand side is the same number at both, because it does not depend on x_0)

$$p_0(x_4, x_0) = p_0(x_4, \tilde{x}_0)$$

(rule: divide by κ, which is not zero). The same three lines with the constraint, the 3-space and the extra-time equations give the same for ρ, p_3 and p_t. The mixed equations say $q_{40} = q_{04} = 0$ outright. So:

C1. Every component of an admissible source is independent of x_0, and the mixed components q_{40} and q_{04} vanish. (PROVED; record `Revision/field_equations_a4/a4-equations.json`, key `generalSource.x0_dependence`.)

The same in the coordinate y. The Kohn-Sham states are stored as functions of $y = \ln(\sin z)/(6H)$. By the chain rule

$$\frac{dy}{dx_0} = \frac{1}{6H} \cdot \frac{\cos z}{\sin z} \cdot 6H = \cot z,$$

(rule: the chain rule; the derivative of $\ln u$ is $1/u$ times the derivative of u, the derivative of $\sin z$ is $\cos z$, and $dz/dx_0 = 6H$) and $\cot z > 0$ for $0 < z < \pi/2$. So y grows strictly with x_0: every value of y belongs to exactly one value of x_0. A function that takes different values at two values of y takes different values at the two corresponding values of x_0. "Depends on y" and "depends on x_0" are the same statement.

How the record measures it. For a state with a nonzero source the record computes the **spread** of the energy density,

$$\frac{\max_y \rho - \min_y \rho}{\max |T|},$$

which is 0 exactly when ρ does not depend on y. C1 needs 0; the record calls a state dependent on x_0 when the spread exceeds its tolerance 10^{-6}.

17.5 Condition C2: the algebraic condition

The derivation. Add the 3-space and the extra-time equations and subtract twice the hidden equation:

$$(\mathcal{P} + a''\mathcal{Q} + \Lambda) + (\mathcal{P} - a''\mathcal{Q} + \Lambda) - 2(\mathcal{P} + \Lambda) = \kappa p_3 + \kappa p_t - 2\kappa p_0$$

(rule: add and subtract the equations side by side; the same operation on both sides keeps an equation true)

$$(\mathcal{P} + \mathcal{P} - 2\mathcal{P}) + (a''\mathcal{Q} - a''\mathcal{Q}) + (\Lambda + \Lambda - 2\Lambda) = \kappa(p_3 + p_t - 2p_0)$$

(rule: collect equal kinds of terms on the left; take out the common factor κ on the right)

$$0 = \kappa(p_3 + p_t - 2p_0)$$

(rule: each bracket on the left is zero). Divided by $\kappa \neq 0$:

C2. Every admissible source has $p_3 + p_t = 2p_0$ at every point and every time. (PROVED; Wolfram report, check `algebraic_identity_x1_plus_x5_minus_2x0`; sympy report, check `algebraic_identity`; reproduced in Notebook 17a, In [4].)

Remarks. C2 holds for every choice of the couplings α_k, of Λ and of the history a_4: it is a condition on the matter alone. It does not involve the energy density. For Einstein gravity the identity behind it is a two-line computation: $(15H^2 - 3a'^2 + a'') + (15H^2 - 3a'^2 - a'') - 2(15H^2 - 3a'^2) = 30H^2 - 6a'^2 - 30H^2 + 6a'^2 = 0$. We call

$$V = p_3 + p_t - 2p_0$$

the **violation** of C2. The record measures its size by $\max_y |V| / \max |T|$ and, for a source averaged over the hidden direction, by the ratio $R = (\int p_3 + \int p_t)/(2 \int p_0)$, which an averaged source satisfying C2 would have equal to 1.

17.6 Condition C3: what the linear member demands

The linear member. For $a_4 = AHx_4 + a_0$:

$$a_4' = AH, \qquad a_4'' = 0$$

(rule: the derivative of $AHx_4 + a_0$ is the constant AH, and the derivative of a constant is 0).

The pressures are equal. The evolution equation of Section 17.3 becomes

$$0 \cdot F(AH) = \kappa(p_3 - p_t) \quad \Longrightarrow \quad p_3 = p_t$$

(rule: insert $a_4'' = 0$; zero times any number is zero; divide by κ). With C2:

$$p_3 + p_3 = 2p_0 \quad \Longrightarrow \quad p_0 = p_3$$

(rule: insert $p_t = p_3$ into $p_3 + p_t = 2p_0$ and divide by 2). So $p_3 = p_t = p_0$; we call the common value p.

The energy density and the pressure are constant. In the constraint, $e_{44} = \sum_k \alpha_k {E_{(k)}}^{x_4}{}_{x_4}$ contains only a' and H (Section 17.3), and $a' = AH$ is a constant. So $-\kappa\rho = e_{44}(AH) + \Lambda$ is the same number at every time. In the same way the hidden equation gives $\kappa p = \mathcal{P}(AH) + \Lambda$, the same number at every time.

C3. Along the linear member $a_4 = AHx_4 + a_0$ an admissible source has $p_3 = p_t = p_0$, and ρ and these pressures do not change with x_4. (PROVED; sympy and Wolfram reports, check `linear_member_equal_pressures`; reproduced in Notebook 17a, In [4].) In Einstein gravity the record gives the values: $\kappa\rho = -(21 + 3A^2)H^2 - \Lambda$ and $\kappa p = (15 - 3A^2)H^2 + \Lambda$ (record `a4-equations.json`, keys `linearMember.rhoEinstein` and `linearMember.pEinstein`). For the history of the Kohn-Sham computations, $A = 1$, this is $\kappa\rho = -24H^2 - \Lambda$ and $\kappa p = 12H^2 + \Lambda$.

What C3 asks of a gas. A gas on the linear history must keep its energy density constant while 3-space inflates and the extra times deflate, and it must push equally hard along 3-space, along the extra times and along the hidden direction, at every instant. Section 17.16 shows that the first demand follows from the second: once the pressures p_3 and p_t are equal, conservation keeps ρ constant.

No source at all. Could the empty source $T = 0$ be admissible? In Einstein gravity the constraint and the hidden equation read $3a'^2 + 21H^2 + \Lambda = -\kappa\rho$ and $15H^2 - 3a'^2 + \Lambda = \kappa p_0$. With $\rho = p_0 = 0$:

$$(3a'^2 + 21H^2 + \Lambda) - (15H^2 - 3a'^2 + \Lambda) = 0 - 0$$

(rule: subtract the second equation from the first, with $\rho = p_0 = 0$ on the right)

$$6a'^2 + 6H^2 = 0$$

(rule: $\Lambda - \Lambda = 0$, $3a'^2 + 3a'^2 = 6a'^2$, $21H^2 - 15H^2 = 6H^2$). A square of a real number is never negative, and $6H^2 > 0$ for $H > 0$, so the left side is positive: there is no solution. (PROVED; Wolfram report, check `einstein_no_vacuum_solution`; reproduced in Notebook 17a, In [19].) The author's metric always needs matter.

17.7 The recorded Kohn-Sham states as a candidate source

What was computed. The Revision Rust solver (Chapter 15) computed, at each slice $a_{4,0}$ of the history $a_4 = Hx_4$, the self-consistent Kohn-Sham ground state of N quanta of dirac16complex in the author's metric with a_4 held at its slice value $a_{4,0}$ (the stationary-slice, adiabatic ansatz of Chapter 14; between the slices the extra times keep deflating). It used the good sector (no dependence on the extra times), the Z2 mirror at the brane and the tip cutoff at $y = -3$, all ASSUMED (Chapter 14). The recorded states are all combinations of

- three particle numbers: $N = 8$ (the eight brane zero modes, the levels of zero 3-momentum and zero energy) and the closed shells $N = 136$ and $N = 688$ (record `Revision/kohn_sham/results/parameters.json`, key `particleNumbers`);
- five couplings: $\lambda = 0$, $\pm\lambda_1$ and $\pm\lambda_2$, chosen for each N so that the largest first-order mean-field potential along the whole history is about $0.1\,m$ and $0.3\,m$ (the same file, key `couplingCalibration`; λ is rounded to four significant digits, so the recorded potentials lie very slightly off these values, in either direction: for $N = 8$, $\lambda_1 = 0.01946$ and $\lambda_2 = 0.05838$ give $0.1000011\,m$ and $0.3000034\,m$ (above), for $N = 136$, $\lambda_1 = 0.0009298$ and $\lambda_2 = 0.002789$ give $0.0999984\,m$ and $0.2999522\,m$ (below), and for $N = 688$, $\lambda_1 = 0.0001846$ and $\lambda_2 = 0.0005538$ give $0.1000001\,m$ and $0.3000004\,m$, again above);
- five slices: $a_{4,0} = 0, 0.5, 1, 1.5, 2$.

That makes $3 \times 5 \times 5 = 75$ states. They are the ground states, at zero temperature, of the canonical matrix of Chapter 15. That matrix also holds 135 thermal states, at the three temperatures 0.01, 0.02 and 0.05 (Chapter 15); the record stores no profiles for them (the folder `Revision/kohn_sham/results/thermo` holds only the table `thermodynamics.csv`), so they are not tested in this chapter (Section 17.17). In this chapter "the recorded states" are the 75 ground states.

What is stored for each state. A **profile** file `Revision/kohn_sham/results/ground/profiles/<name>.csv` with the proper densities at the 151 points $y = -3, -2.98, \ldots, 0$ (step 0.02), among them the four columns `rho`, `p3`, `p_t`, `p0`; and a row of the table `Revision/kohn_sham/results/ground/emt-integrals.csv` with the integrals over the patch, the values at the brane and at the tip, and check columns of the solver.

Why the profiles can be compared with the field equations directly. The field equations use $T^{\mu}{}_{\nu}$ with one index up and one down. In the frame of rulers (the diagonal vielbein of Chapter 6) a component is $T^{a}{}_{b} = e^{a}{}_{\mu}\, e_{b}{}^{\nu}\, T^{\mu}{}_{\nu}$, and for a diagonal vielbein a diagonal component picks up one factor and its inverse: $T^{a}{}_{a} = T^{\mu}{}_{\mu}$ (no sum). So the proper densities of the profiles are exactly the ρ, p_3, p_t, p_0 of the field equations.

The Kohn-Sham source in words. The record `Revision/kohn_sham/ks-theory.json` (key `emt`) gives the four components as sums over the occupied orbitals. Two facts matter here. First, $p_t = e_{\text{int}}$, the interaction energy density: in the good sector no orbital moves along the extra times, so there is no extra-time kinetic pressure; for $\lambda = 0$ the interaction vanishes and $p_t = 0$ exactly. Second, p_3 is the kinetic pressure of the motion along 3-space plus e_{int}, so for a gas whose quanta move along 3-space p_3 is in general not zero; for the recorded states with $\lambda = 0$ its integral over the patch is positive at every slice (Notebook 17a, In [16] and figure 7). The mixed component, there written $T^{x_4}{}_{y}$, vanishes for every eigen-orbital (key `emt.offDiagonal`), so the Kohn-Sham states

meet the part $q_{40} = q_{04} = 0$ of C1. The question is the dependence on x_0, and conditions C2 and C3.

The integral over the patch. A small region of the hidden direction between y and $y + dy$, times the coordinate box of 3-space and of the extra times, has the proper 7-volume

$$dV_7 = \sqrt{|g|}\; d^3x\, d^3x_t\, dy = e^{6Hy}\, \ell^3 v_t\, dy = \mathrm{Vol}_7\, e^{6Hy}\, dy$$

(rule: in the coordinate y the product of the seven scale factors other than that of the time is $(e^{a_4}e^{Hy})^3(e^{-a_4}e^{Hy})^3 \cdot 1 = e^{6Hy}$, Chapter 14; the coordinate box of 3-space is a cube of side ℓ and that of the extra times has the coordinate volume v_t). The runs use $\ell = 8\pi$ (from the momentum spacing $\Delta k = 2\pi/\ell = 0.25$) and $v_t = 1$, so $\mathrm{Vol}_7 = (8\pi)^3 = 512\pi^3 \approx 15875.2$ (record `parameters.json`, key `physics.Vol7`). The total amount of a density X in the patch and its mirror copy is therefore

$$\int X \;=\; 2\,\mathrm{Vol}_7 \int_{-3}^{0} e^{6Hy}\, X(y)\, dy,$$

the columns `int_rho`, `int_p3`, `int_p_t`, `int_p0` of the table. The factor 2 counts the mirror copy of the ASSUMED Z2 brane. For $X = \rho$ the integral is the energy E of the Kohn-Sham state (`ks-theory.json`, key `emt.integrated`). The proper volume of the patch and its copy is

$$2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy}\, dy = 2\,\mathrm{Vol}_7 \left[\frac{e^{6Hy}}{6H}\right]_{-L}^{0} = 2\,\mathrm{Vol}_7\, \frac{1 - e^{-6HL}}{6H}$$

(rule: an antiderivative of e^{cy} is e^{cy}/c; then the fundamental theorem of calculus, upper value minus lower value, with $e^0 = 1$). The **weighted mean** of X is the amount divided by the volume:

$$\bar{X} = \frac{\int X}{2\,\mathrm{Vol}_7\,(1 - e^{-6HL})/(6H)}.$$

Note how fast the weight e^{6Hy} falls toward the tip: at $y = -L = -3$ (with $H = 1$) it is $e^{-18} \approx 1.5 \times 10^{-8}$.

The prediction of the conditions. C1 asks for flat profiles, C2 for $V = 0$ at every point (or, averaged, $R = 1$), C3 for a constant $\int \rho$ along the history and equal pressures. Notebook 17a checks each against the record.

17.8 Example: the three conditions tested on 75 recorded ground states

The first notebook derives the three conditions from the Lovelock components stored in the record, with exact computer algebra; reads the 75 recorded ground states; and reproduces every number of the five checks on the Kohn-Sham states of `Revision/field_equations_a4/reports/ks-source-conditions.json` (its sixth check, `ks_verdict_holds_for_every_b4`, is about the metric with b_4). It prints 16 PASS lines and draws seven figures: the hidden coordinate and the energy profiles; a heat map of the spread of ρ (C1); the violation profiles and a heat map of their size (C2);

three points of one state as bars; the averaged ratio R; and the integrals along the history (C3). It runs in about 15 seconds on a typical laptop and needs no Rust.

17.9 How to run Notebook 17a

Step 1. What this notebook does and what it needs.

Notebook 17a (The source conditions of the a4 equations on the recorded Kohn-Sham states) is the file `Revision/textbook/notebooks/17a_source_conditions.ipynb` of the repository Dirac_claude. It derives with sympy, from the Lovelock components stored in the Revision record of the a4 field equations, the three conditions that every source of the author's metric must meet (C1: no dependence on the hidden coordinate x0; C2: p3 + p_t = 2 p0; C3: for the linear member a4 = A H x4, equal pressures and a constant energy density). It then evaluates the three conditions on the 75 committed Kohn-Sham ground states of dirac16complex, reproduces every number of the five checks of the Revision record ks-source-conditions.json, shows that no recorded Kohn-Sham ground state is an admissible source, so that the Kohn-Sham history is a prescribed background, and draws seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 17a_source_conditions.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 17a_source_conditions.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 17a_source_conditions.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/17a.captions.json`
- `Revision/textbook/figures/17a_1_rho_profiles.png`
- `Revision/textbook/figures/17a_2_c1_map.png`
- `Revision/textbook/figures/17a_3_violation_profiles.png`
- `Revision/textbook/figures/17a_4_c2_map.png`
- `Revision/textbook/figures/17a_5_three_points.png`
- `Revision/textbook/figures/17a_6_integrated_ratio.png`
- `Revision/textbook/figures/17a_7_history_integrals.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 16 CHECKS PASSED (notebook 17a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/17a_source_conditions.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 17a_source_conditions.ipynb
python -m nbconvert --execute --inplace 17a_source_conditions.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming ks-source-conditions.json or a profile file: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

17.10 Notebook 17a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/17a_source_conditions.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The author's metric has one free function, $a_4(x_4)$, and the Einstein-Lovelock
field equations decide which **source** (energy density and pressures of matter)
can produce it. The Revision record of these equations shows that their left-hand
sides do not depend on the hidden coordinate $x_0$ and obey one algebraic identity.
So every admissible source must satisfy three conditions:

- **C1**: no component of the source depends on $x_0$ (and the mixed components
  $q_{40}$ and $q_{04}$ are zero);
- **C2**: $p_3 + p_t = 2p_0$ (3-space pressure plus extra-time pressure equals
  twice the hidden-direction pressure);
- **C3**: for the linear member $a_4 = AHx_4 + a_0$ (the history used by the
  Kohn-Sham computations): $p_3 = p_t = p_0$ and a constant energy density.

This notebook

- derives C1, C2 and C3 with sympy from the Lovelock components of the record;
- reads the 75 Kohn-Sham ground states of the Revision record (their profiles
  along the hidden coordinate and their integrals);
- evaluates C1, C2 (point by point and after integration over $x_0$) and C3 on
  every state and reproduces every number of the five checks of the record
  `ks-source-conditions.json`;
- concludes, as the record does, that no recorded Kohn-Sham ground state is an
  admissible source, so that the Kohn-Sham history $a_4 = AHx_4$ is a
  **prescribed background** (the gas is a test field without back-reaction);
- draws seven teaching plots. It takes about fifteen seconds.
```

Text cell:

```
## 2. How to run this notebook
```

Step 1. What this notebook does and what it needs.

Notebook 17a (The source conditions of the a4 equations on the recorded Kohn-Sham states) is the file ‘Revision/textbook/notebooks/17a_source_conditions.ipynb‘ of the repository Dirac_claude. It derives with sympy, from the Lovelock components stored in the Revision record of the a4 field equations, the three conditions that every source of the author’s metric must meet (C1: no dependence on the hidden coordinate x0; C2: p3 + p_t = 2 p0; C3: for the linear member a4 = A H x4, equal pressures and a constant energy density). It then evaluates the three conditions on the 75 committed Kohn-Sham ground states of dirac16complex, reproduces every number of the five checks of the Revision record ks-source-conditions.json, shows that no recorded Kohn-Sham ground state is an admissible source, so that the Kohn-Sham history is a prescribed background, and draws seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 17a_source_conditions.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 17a_source_conditions.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 17a_source_conditions.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/17a.captions.json‘
- ‘Revision/textbook/figures/17a_1_rho_profiles.png‘
- ‘Revision/textbook/figures/17a_2_c1_map.png‘
- ‘Revision/textbook/figures/17a_3_violation_profiles.png‘
- ‘Revision/textbook/figures/17a_4_c2_map.png‘
- ‘Revision/textbook/figures/17a_5_three_points.png‘
- ‘Revision/textbook/figures/17a_6_integrated_ratio.png‘
- ‘Revision/textbook/figures/17a_7_history_integrals.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 16 CHECKS PASSED (notebook 17a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/17a_source_conditions.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

    ```
    python -m jupyterlab 17a_source_conditions.ipynb
    python -m nbconvert --execute --inplace 17a_source_conditions.ipynb
    ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming ks-source-conditions.json or a profile file: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 17a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 17a (The source conditions of the a4 equations on the recorded Kohn-Sham
# states) is the file Revision/textbook/notebooks/17a_source_conditions.ipynb of the
# repository Dirac_claude. It derives with sympy, from the Lovelock components stored in
# the Revision record of the a4 field equations, the three conditions that every source
```

```
# of the author’s metric must meet (C1: no dependence on the hidden coordinate x0; C2: p3
# + p_t = 2 p0; C3: for the linear member a4 = A H x4, equal pressures and a constant
# energy density). It then evaluates the three conditions on the 75 committed Kohn-Sham
# ground states of dirac16complex, reproduces every number of the five checks of the
# Revision record ks-source-conditions.json, shows that no recorded Kohn-Sham ground
# state is an admissible source, so that the Kohn-Sham history is a prescribed
# background, and draws seven teaching plots. It needs a computer with Windows 11, macOS
# or Linux, an internet connection for the installation, the program Git, and Python 3.12
# or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
```

```
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
```

```
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 17a_source_conditions.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 17a_source_conditions.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 17a_source_conditions.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
```

```
# - Revision/textbook/figures/17a.captions.json
# - Revision/textbook/figures/17a_1_rho_profiles.png
# - Revision/textbook/figures/17a_2_c1_map.png
# - Revision/textbook/figures/17a_3_violation_profiles.png
# - Revision/textbook/figures/17a_4_c2_map.png
# - Revision/textbook/figures/17a_5_three_points.png
# - Revision/textbook/figures/17a_6_integrated_ratio.png
# - Revision/textbook/figures/17a_7_history_integrals.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 16 CHECKS PASSED (notebook 17a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/17a_source_conditions.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 17a_source_conditions.ipynb
```

```
#     python -m nbconvert --execute --inplace 17a_source_conditions.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming ks-source-conditions.json or a profile file: the notebook
#   reads the Revision records of the repository. Run it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone,
#   not on a copy of the notebook file alone.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "17a"  # this notebook: chapter 17, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 17a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ = ordinary 3-space,
  which inflates (scale factor $e^{a_4}\sin^{1/6}z$); $x_4$ = the time;
  $x_5, x_6, x_7$ = the three **extra times**, which **deflate exponentially**
  (scale factor $e^{-a_4}\sin^{1/6}z$, with $a_4$ increasing); $x_0$ = the hidden
  space direction, with $z = 6Hx_0$ between $0$ and $\pi/2$.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: another way to number the points
  of the hidden direction. $y = 0$ is the end $z = \pi/2$ (the **brane**), and
  $y \to -\infty$ is the end $z \to 0$ (the **tip**). The computations stop at
  $y = -L = -3$ (the **tip cutoff**). Since $y$ grows when $x_0$ grows, "depends
  on $y$" and "depends on $x_0$" mean the same thing.
- **Energy-momentum tensor** $T^\mu{}_\nu$: an $8 \times 8$ table at each point
  that says how much energy and momentum matter has and how they flow. For our
  sources it is diagonal: $T = \mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t,
  p_0)$ in the order $x_0, \dots, x_7$, plus possible mixed entries $q_{40}$ (row
  $x_4$, column $x_0$) and $q_{04}$ (row $x_0$, column $x_4$).
```

```
- **Energy density** $\rho$ and **pressures** $p_3$ (each direction of 3-space),
  $p_t$ (each extra time), $p_0$ (the hidden direction).
- **Source**: the matter whose $T$ stands on the right-hand side of the field
  equations. **Admissible source**: a $T$ for which the field equations of the
  author's metric can hold.
- **Profile**: a component of $T$ as a function of $y$, stored at the 151 points
  $y = -3, -2.98, \dots, 0$. **Proper density**: per unit of proper 7-volume
  (the volume that rulers measure). The diagonal components $T^\mu{}_\mu$ (one
  index up, one down) are the same in the coordinates and in the frame of rulers,
  so the profiles are directly the $\rho$ and $p$ of the field equations.
- **Kohn-Sham state**: a many-fermion state of dirac16complex computed with the
  Kohn-Sham method (density functional theory) by the Rust solver of the Revision
  record. A state is labelled by its particle number $N$ (8, 136 or 688), its
  self-coupling $\lambda$ ($0$, $\pm\lambda_1$, $\pm\lambda_2$) and its **slice**
  $a_{4,0}$, the value of $a_4$ at which it was computed ($0, 0.5, 1, 1.5, 2$).
  The name `N136_lamp2_a20` means $N = 136$, $\lambda = +\lambda_2$,
  $a_{4,0} = 2.0$ (`lamm1` is $-\lambda_1$, `lam0` is $\lambda = 0$).
- **History**: the sequence of slices along $a_4 = AHx_4$ with $A = 1$.
  **Linear member**: this choice of $a_4$, a straight line in $x_4$.
- **Prescribed background**: a metric that is GIVEN, not solved for. A field that
  moves in it but whose own gravity is ignored is a **test field**; ignoring its
  gravity means ignoring its **back-reaction** on the metric.
- **Integral over the patch**: $2\,\mathrm{Vol}_7\int_{-3}^{0} e^{6Hy}\,X\,dy$, the
  total amount of $X$ in the computed region. Here $\mathrm{Vol}_7 = \ell^3 v_t$ is
  the coordinate volume of the six directions $x_1, x_2, x_3$ (a box of side
  $\ell$) and $x_5, x_6, x_7$ (coordinate volume $v_t$); times $e^{6Hy}\,dy$ it is
  the proper 7-volume of a thin slab of the hidden direction. The factor 2 counts
  the mirror copy of the ASSUMED $Z_2$ brane.
- **max|T|**: the largest absolute value of $\rho$, $p_3$, $p_t$, $p_0$ of a state
  over all $y$; we divide by it to compare states of very different size.
- **Status labels**: PROVED (exact), COMPUTED (numerical), ASSUMED (a choice),
  OPEN (not known).
- Units: $H = 1$, $m = 1$ (the fermion mass): lengths in units of $1/H$, energy
  densities and pressures in units of $m^8$, integrals over the patch (energies)
  in units of $m$.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's metric (order $x_0, \dots, x_7$, $z = 6Hx_0$) is diagonal with entries
$e^{2a_4}\sin^{1/3}z$ (three times), $-1$, $-e^{-2a_4}\sin^{1/3}z$ (three times),
$\cot^2 z$. The Einstein-Lovelock field equations are

$$\sum_{k=1}^{3}\alpha_k E_{(k)}{}^{\mu}{}_{\nu} + \Lambda\,\delta^\mu_\nu =
\kappa\,T^\mu{}_\nu ,$$

with the Lovelock tensors $E_{(1)}$ (Einstein), $E_{(2)}$ (Gauss-Bonnet), $E_{(3)}$
(third order), couplings $\alpha_k$, cosmological constant $\Lambda$ and strength of
gravity $\kappa$. The Revision record gives every component of the left-hand side
as a polynomial in $a_4'$, $a_4''$ and $H$ (the prime is $d/dx_4$). The diagonal
components give four equations:
```

```
$$\textstyle\sum_k\alpha_kE_{(k)}{}^{x_4}{}_{x_4} + \Lambda = -\kappa\rho,\quad
\sum_k\alpha_kE_{(k)}{}^{x_1}{}_{x_1} + \Lambda = \kappa p_3,$$
$$\textstyle\sum_k\alpha_kE_{(k)}{}^{x_5}{}_{x_5} + \Lambda = \kappa p_t,\quad
\sum_k\alpha_kE_{(k)}{}^{x_0}{}_{x_0} + \Lambda = \kappa p_0 .$$

The left-hand sides contain no $x_0$, so the right-hand sides cannot either (C1).
The left-hand sides obey $E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5} = 2E^{x_0}{}_{x_0}$
for every $k$; adding the $x_1$ and $x_5$ equations and subtracting twice the $x_0$
equation gives $\kappa(p_3 + p_t - 2p_0) = 0$ (C2; $\Lambda$ cancels because
$\Lambda + \Lambda - 2\Lambda = 0$). For $a_4 = AHx_4 + a_0$ we have $a_4' = AH$
(a constant) and $a_4'' = 0$; then the $x_1$, $x_5$ and $x_0$ left-hand sides
coincide and none of them changes with $x_4$ (C3).

The Kohn-Sham computations of the Revision record use exactly this linear member:
they solve the Kohn-Sham equations of dirac16complex at five slices $a_{4,0}$ of the
history $a_4 = Hx_4$ (instantaneous, adiabatic states, in the good sector without
extra-time momenta, with the ASSUMED $Z_2$ mirror at the brane and a tip cutoff at
$y = -3$). The question of this notebook: is the energy-momentum tensor of these
states an admissible source, so that the history would be a solution of the
coupled equations? The answer of the record is no; we check every number of it.
```

Text cell:

```
## 5. The Revision records and the helpers that read them

The next cell imports the packages and defines four helpers. ‘read_json‘ reads a
JSON file of the repository. ‘record_entry‘ finds a check by its name in a Revision
report. ‘reproduces‘ is a check that passes only when this notebook’s own result
holds AND the named checks of the report have the verdict PASS; it prints the
report file and the check names. ‘tex_number‘ writes a number in powers of ten for
the captions of the figures. The cell then reads the record on the Kohn-Sham
source conditions, prints its conclusion and checks that its six checks passed:
the five on the Kohn-Sham states, which this notebook reproduces, and the sixth,
‘ks_verdict_holds_for_every_b4‘, which states that the verdict holds for the
author’s metric with any second function $b_4(x_4)$ of 2026-10-14 (the conditions
C1 to C3 below are derived on the background $b_4 = 1$).
```

In [2]:

```
import csv  # reads tables stored as CSV files (comma-separated values)

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

SOURCE_REPORT = "Revision/field_equations_a4/reports/ks-source-conditions.json"
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
PY_A4 = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy checks
WL_A4 = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram
GROUND = "Revision/kohn_sham/results/ground"  # the 75 Kohn-Sham ground states


def read_json(relative):
```

```
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_entry(report_file, name):
    """The entry {name, verdict, detail} of a check of a report, None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry
    return None


def reproduces(condition, name, report_file, record_names):
    """A check that also requires the named checks of the report to be PASS."""
    found = all((record_entry(report_file, n) or {}).get("verdict") == "PASS"
                for n in record_names)
    names_text = ", ".join(record_names)  # the check names, separated by commas
    check(condition and found, name, record=f"{report_file}, check {names_text}")


def tex_number(value, digits=2):
    """A number for a caption in powers of ten: 0.0123 -> 1.2 \\times 10^{-2}."""
    mantissa, exponent = f"{value:.{digits - 1}e}".split("e")
    return f"{mantissa} \\times 10^{{{int(exponent)}}}"


source_record = read_json(SOURCE_REPORT)
summary = source_record["summary"]  # how many checks the record has, how many pass
passed, total = summary["pass"], summary["checks"]
report("checks of the record ks-source-conditions.json", f"{passed} of {total} PASS")
say("its conclusion: " + source_record["conclusion"])
check(summary == {"checks": 6, "pass": 6, "fail": 0},
      "the record on the Kohn-Sham source conditions has 6 checks, all PASS")
```

Out [2]:

```
RESULT checks of the record ks-source-conditions.json = 6 of 6 PASS
its conclusion: No Kohn-Sham state recorded in Revision/kohn_sham is an admissible source
    of the author's metric: C1 and C2 fail for every nonzero state (C2 also after
    integration over x0), and the Kohn-Sham history uses the linear member as a
    prescribed background, without back-reaction.  With the metric of 2026-10-14 (b4[x4])
    C1 alone excludes every recorded state for every b4 (check
    ks_verdict_holds_for_every_b4); C2 is the b4 = 1 form.
PASS the record on the Kohn-Sham source conditions has 6 checks, all PASS
```

Text cell:

```
## 6. The three conditions, derived from the a4 equations

The record stores each Lovelock component as a formula in the Wolfram Language:
‘ad1‘ stands for $a_4’$, ‘ad2‘ for $a_4’’$, ‘^‘ for a power. The next cell turns
the components $x_1x_1$, $x_4x_4$, $x_5x_5$, $x_0x_0$ and $x_4x_0$ of
```

```
$E_{(1)}, E_{(2)}, E_{(3)}$ into sympy expressions and prints those of Einstein's
tensor $E_{(1)}$ (they are the left-hand sides of Einstein gravity).
```

In [3]:

```
equations = read_json(EQUATIONS)
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4'' (the record's names)
H = sp.symbols("H", positive=True)  # the constant H of the metric
NAMES = {"ad1": ad1, "ad2": ad2, "H": H}  # how the record's names become symbols


def parse(text):
    """A formula of the record (Wolfram Language text) as a sympy expression."""
    return sp.sympify(text.replace("^", "**"), locals=NAMES)


COMPONENTS = ("x1x1", "x4x4", "x5x5", "x0x0", "x4x0")
E = {k: {c: parse(equations["lovelockTensors"][f"E{k}"][c]["input"])
         for c in COMPONENTS}
     for k in (1, 2, 3)}  # E[k][component]: the Lovelock tensor of order k
for c in COMPONENTS:
    say(f"E_(1)^{c[:2]}_{c[2:]} = {E[1][c]}")
```

Out [3]:

```
E_(1)^x1_x1 = 15*H**2 - 3*ad1**2 + ad2
E_(1)^x4_x4 = 21*H**2 + 3*ad1**2
E_(1)^x5_x5 = 15*H**2 - 3*ad1**2 - ad2
E_(1)^x0_x0 = 15*H**2 - 3*ad1**2
E_(1)^x4_x0 = 0
```

Text cell:

```
The next cell checks the three conditions on the left-hand sides, for all three
Lovelock orders.

- C1: every component contains only the symbols $a_4'$, $a_4''$, $H$ (no $x_0$,
  not even through $\cot z$), and the mixed $x_4x_0$ and $x_0x_4$ components are
  zero, so the source must have $q_{40} = q_{04} = 0$.
- C2: $E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5} - 2E^{x_0}{}_{x_0}$ expands to zero.
- C3: with $a_4'' = 0$ the $x_1$, $x_5$, $x_0$ components are equal, and the
  $x_4$ component contains $a_4'$ and $H$ only (constant when $a_4' = AH$).
```

In [4]:

```
allowed = {ad1, ad2, H}  # the only symbols a left-hand side may contain
no_x0 = all(E[k][c].free_symbols <= allowed for k in E for c in COMPONENTS)
reproduces(no_x0 and all(E[k]["x4x0"] == 0 for k in E),
           "C1: the left-hand sides are free of x0 and the x4-x0 one is 0",
```

```
          WL_A4, ["P1_structure", "P2_structure", "P3_structure"])
identity = [sp.expand(E[k]["x1x1"] + E[k]["x5x5"] - 2 * E[k]["x0x0"]) for k in E]
say(f"E^x1_x1 + E^x5_x5 - 2 E^x0_x0 for k = 1, 2, 3: {identity}")
reproduces(all(value == 0 for value in identity),
          "C2: E^x1_x1 + E^x5_x5 = 2 E^x0_x0 for every order k",
          WL_A4, ["algebraic_identity_x1_plus_x5_minus_2x0"])
linear = {ad2: 0}  # the linear member: a4'' = 0 (and a4' = A H, a constant)
equal = all(sp.expand((E[k]["x1x1"] - E[k]["x0x0"]).subs(linear)) == 0
            and sp.expand((E[k]["x5x5"] - E[k]["x0x0"]).subs(linear)) == 0
            for k in E)
rho_side = all(E[k]["x4x4"].free_symbols <= {ad1, H} for k in E)
reproduces(equal and rho_side,
          "C3: for a4'' = 0 the pressures are equal and rho is constant",
          PY_A4, ["linear_member_equal_pressures"])
```

Out [4]:

```
PASS C1: the left-hand sides are free of x0 and the x4-x0 one is 0
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    P1_structure, P2_structure, P3_structure
E^x1_x1 + E^x5_x5 - 2 E^x0_x0 for k = 1, 2, 3: [0, 0, 0]
PASS C2: E^x1_x1 + E^x5_x5 = 2 E^x0_x0 for every order k
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    algebraic_identity_x1_plus_x5_minus_2x0
PASS C3: for a4'' = 0 the pressures are equal and rho is constant
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    linear_member_equal_pressures
```

Text cell:

```
## 7. The recorded Kohn-Sham states and the hidden coordinate

The next cell reads the 75 profile files of the record (one per state; columns
‘y‘, ‘rho‘, ‘p3‘, ‘p_t‘, ‘p0‘ among others), the table of integrals, and the
parameters of the computations. For every state it computes max|T|, the largest
absolute value of the four components, and sorts the states into those with a
nonzero tensor and those with $T = 0$ everywhere (no source at all). It also prints
what the Kohn-Sham theory record says about the mixed components (there written
$T^{x_4}{}_y$): they vanish for every eigen-orbital, so the Kohn-Sham states meet
the part $q_{40} = q_{04} = 0$ of C1; the question is the $x_0$ dependence.
```

In [5]:

```
COLUMNS = ("rho", "p3", "p_t", "p0")  # the four diagonal components of the source


def read_profile(path):
    """A profile table as a dictionary: column name -> numpy array (151 values)."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}
```

```
files = sorted(repository_file(f"{GROUND}/profiles").glob("*.csv"),
               key=lambda path: path.name)  # in alphabetical order, as the record
profiles = {path.stem: read_profile(path) for path in files}
with repository_file(f"{GROUND}/emt-integrals.csv").open(
        encoding="utf-8", newline="") as handle:
    integrals = {row["id"]: row for row in csv.DictReader(handle)}
physics = read_json("Revision/kohn_sham/results/parameters.json")["physics"]
scale = {sid: float(max(np.max(np.abs(prof[c])) for c in COLUMNS))
         for sid, prof in profiles.items()}  # max|T| of every state
zero = [sid for sid in profiles if scale[sid] == 0.0]  # T = 0 everywhere
nonzero = [sid for sid in profiles if scale[sid] > 0.0]
y = profiles["N136_lam0_a10"]["y"]  # the grid of the hidden coordinate
same_grid = all(np.array_equal(prof["y"], y) for prof in profiles.values())
report("states read", len(profiles))
report("states with a nonzero tensor", len(nonzero))
report("grid of y", f"{len(y)} points from {y[0]:g} to {y[-1] + 0.0:g}, "
       f"step {y[1] - y[0]:.2f}")
H_value, L_cut, VOL7 = physics["H"], physics["L_tipCutoff"], physics["Vol7"]
report("H, tip cutoff L, Vol_7", f"{H_value:g}, {L_cut:g}, {VOL7:.6g}")
say("ks-theory.json, emt.offDiagonal: "
    + read_json("Revision/kohn_sham/ks-theory.json")["emt"]["offDiagonal"])
check(len(profiles) == 75 and len(nonzero) == 70 and len(zero) == 5 and same_grid
      and sorted(integrals) == sorted(profiles),
      "75 states on one grid (70 nonzero, 5 zero); every state has its integrals")
```

Out [5]:

```
RESULT states read = 75
RESULT states with a nonzero tensor = 70
RESULT grid of y = 151 points from -3 to 0, step 0.02
RESULT H, tip cutoff L, Vol_7 = 1, 3, 15875.2
ks-theory.json, emt.offDiagonal: T^x4_y = 0 for eigen-orbitals (K_x4y = P (eps - v/2)
    c_o, c_o = 0)
PASS 75 states on one grid (70 nonzero, 5 zero); every state has its integrals
```

Text cell:

```
The next cell draws two pictures. Left: the hidden coordinate
$y = \ln(\sin z)/(6H)$ against $z = 6Hx_0$ (logarithmic horizontal axis), with the
computed patch from the tip cutoff $y = -3$ to the brane $y = 0$; the cutoff sits
at $z = \arcsin(e^{-18})$, so the patch covers almost all of the hidden direction.
Right: the energy density $\rho(y)$, divided by max|T|, of the state $N = 136$,
$\lambda = 0$ at the five slices (logarithmic vertical axis; the cell first checks
that $\rho$ is positive at every point, so that a logarithmic axis can show it).
An admissible source would be a horizontal line (C1).
```

In [6]:

```
SLICES = {"a00": 0.0, "a05": 0.5, "a10": 1.0, "a15": 1.5, "a20": 2.0}  # a4,0
SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light -> dark
```

```
history136 = [f"N136_lam0_{tag}" for tag in SLICES]  # N = 136, lambda = 0
falls = [float(np.max(profiles[s]["rho"]) / np.min(profiles[s]["rho"]))
         for s in history136]  # how much rho changes along y, slice by slice
check(all(float(np.min(profiles[s]["rho"])) > 0.0 for s in history136),
      "N = 136, lambda = 0: rho is positive at every point of every slice")
z = np.logspace(-9, np.log10(np.pi / 2), 400)  # z from 1e-9 to pi/2
z_tip = float(np.arcsin(np.exp(-18.0)))  # where y = -3 (6 H y = -18 with H = 1)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.plot(z, np.log(np.sin(z)) / 6.0, color="#2a78d6", lw=2.0)
left.axvspan(z_tip, np.pi / 2, color="#1baf7a", alpha=0.15,
             label="computed patch, $-3 \\leq y \\leq 0$")
left.axhline(-3.0, color="0.4", ls="--", lw=1.0)
left.set_xscale("log")
left.set_xlabel("$z = 6Hx_0$ (logarithmic axis)")
left.set_ylabel("hidden coordinate $y$ (units of $1/H$)")
left.set_title("$y = \\ln(\\sin z)/(6H)$, $H = 1$")
left.legend(loc="lower right", fontsize=8)
for shade, sid, a40 in zip(SHADES, history136, SLICES.values()):
    right.plot(y, profiles[sid]["rho"] / scale[sid], color=shade, lw=1.8,
               label=f"$a_{{4,0}} = {a40:g}$")
right.axhline(1e-3, color="0.4", ls=":", lw=1.2,
              label="admissible: constant (example)")
right.set_yscale("log")
right.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
right.set_ylabel("$\\rho(y)$ / max|T|")
right.set_title("$N = 136$, $\\lambda = 0$")
right.legend(fontsize=7, loc="upper right")
fig.tight_layout()
save_figure(fig, "rho_profiles",
            "Left: the hidden coordinate $y = \\ln(\\sin z)/(6H)$ against $z = 6Hx_0$ "
            "(logarithmic axis, $H = 1$); the shaded band is the computed patch from "
            "the tip cutoff $y = -3$ to the brane $y = 0$, which covers $z$ from "
            f"${z_tip / 1e-8:.1f} \\times 10^{{-8}}$ to $\\pi/2$. Right: the energy "
            "density $\\rho(y)$ of the Kohn-Sham state $N = 136$, $\\lambda = 0$ "
            "divided by the largest component of the state, at the five slices "
            "$a_{4,0} = 0$ to $2$ (logarithmic vertical axis). The largest value of "
            f"$\\rho$ is {min(falls):.0f} to ${tex_number(max(falls))}$ times its "
            "smallest "
            "value, depending on the slice, while condition C1 of the field "
            "equations demands a horizontal line such as the dotted one.")
```

Out [6]:

```
PASS N = 136, lambda = 0: rho is positive at every point of every slice
```

Out [6]:

```
Figure 17a.1 saved as Revision/textbook/figures/17a_1_rho_profiles.png
```

Text cell:

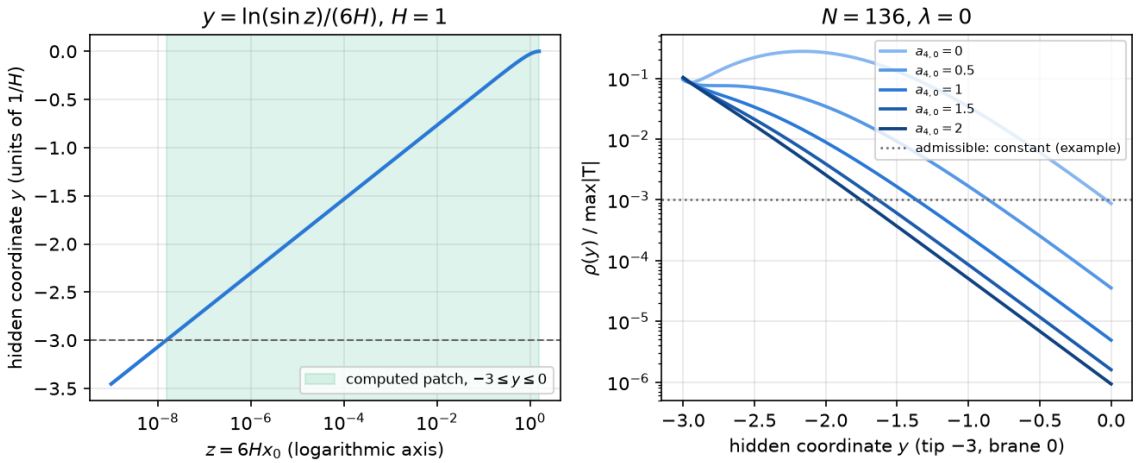

Figure 459: Left: the hidden coordinate $y = \ln(\sin z)/(6H)$ against $z = 6Hx_0$ (logarithmic axis, $H = 1$); the shaded band is the computed patch from the tip cutoff $y = -3$ to the brane $y = 0$, which covers z from 1.5×10^{-8} to $\pi/2$. Right: the energy density $\rho(y)$ of the Kohn-Sham state $N = 136$, $\lambda = 0$ divided by the largest component of the state, at the five slices $a_{4,0} = 0$ to 2 (logarithmic vertical axis). The largest value of ρ is 319 to 1.1×10^5 times its smallest value, depending on the slice, while condition C1 of the field equations demands a horizontal line such as the dotted one. (Notebook 17a, figure 1.)

```
## 8. Condition C1: the source must not depend on $x_0$

The record measures the $x_0$ dependence of a state by the spread of its energy
density, $(\max_y\rho - \min_y\rho)/\max|T|$. For an admissible source this number
is $0$. The next cell computes it for the 70 nonzero states, finds the smallest
value, and checks that it is far above the record's tolerance $10^{-6}$ and that
the number and the state name printed by the record (six significant digits) are
exactly ours.
```

In [7]:

```
TOL = 1e-6  # the relative tolerance of the record
spread = {sid: float(np.max(profiles[sid]["rho"]) - np.min(profiles[sid]["rho"]))
          / scale[sid] for sid in nonzero}
smallest = min(spread, key=spread.get)  # the state that depends least on x0
largest = max(spread, key=spread.get)  # the state that depends most on x0
report("smallest spread of rho / max|T|", f"{spread[smallest]:.6g} ({smallest})")
report("largest spread of rho / max|T|", f"{spread[largest]:.6g} ({largest})")
detail = record_entry(SOURCE_REPORT, "ks_profiles_depend_on_x0")["detail"]
text = f">= {spread[smallest]:.6g} (smallest: {smallest})"  # as the record prints it
reproduces(all(value > TOL for value in spread.values()) and text in detail
           and "75 ground-state profiles" in detail
           and "70 with a nonzero" in detail,
           "C1 fails: every nonzero state depends on x0",
           SOURCE_REPORT, ["ks_profiles_depend_on_x0"])
```

Out [7]:

```
RESULT smallest spread of rho / max|T| = 0.0497329 (N136_lamm2_a20)
RESULT largest spread of rho / max|T| = 0.995037 (N8_lamm1_a00)
PASS C1 fails: every nonzero state depends on x0
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_profiles_depend_on_x0
```

Text cell:

```
The next cell draws the spread for all 75 states as a coloured table (a **heat
map**): one row per particle number and coupling, one column per slice. Grey cells
are the five states with $T = 0$. The colour scale is logarithmic. The helper
‘describe‘ writes a state name as mathematics for the caption.
```

In [8]:

```
from matplotlib.colors import LogNorm  # a logarithmic colour scale

TAGS = [("m2", "-\\lambda_2"), ("m1", "-\\lambda_1"), ("0", "0"),
        ("p1", "+\\lambda_1"), ("p2", "+\\lambda_2")]  # the couplings, in order
ROWS = [(n, tag, label) for n in (8, 136, 688) for tag, label in TAGS]


def describe(sid):
    """The state name N136_lamm2_a20 written as N = 136, lambda = -lambda_2, ..."""
    n, lam, slice_tag = sid.split("_")
    label = dict(TAGS)[lam[3:]]  # lam[3:] is m2, m1, 0, p1 or p2
    return (f"$N = {n[1:]}$, $\\lambda = {label}$, "
            f"$a_{{4,0}} = {SLICES[slice_tag]:g}$")


def state_table(values):
    """A 15 x 5 array of values[state] (NaN for the states with T = 0)."""
    table = np.full((len(ROWS), len(SLICES)), np.nan)
    for i, (n, tag, _) in enumerate(ROWS):
        for j, slice_tag in enumerate(SLICES):
            sid = f"N{n}_lam{tag}_{slice_tag}"
            if sid in values:
                table[i, j] = values[sid]
    return table


def draw_table(ax, table, norm, fmt):
    """Draw a state table as a heat map with the value written in every cell."""
    cmap = plt.get_cmap("viridis").copy()
    cmap.set_bad("#d9d9d9")  # grey for NaN (the states with T = 0)
    image = ax.imshow(np.ma.masked_invalid(table), cmap=cmap, norm=norm,
                      aspect="auto")
    for i in range(table.shape[0]):
        for j in range(table.shape[1]):
            value = table[i, j]
            if np.isnan(value):
                ax.text(j, i, "T = 0", ha="center", va="center", fontsize=7)
```

```
                continue
            light = norm(value) > 0.6  # yellow-green cells: black text reads best
            ax.text(j, i, format(value, fmt), ha="center", va="center", fontsize=7,
                    color="black" if light else "white")
    ax.set_xticks(range(len(SLICES)), [f"{a:g}" for a in SLICES.values()])
    ax.set_yticks(range(len(ROWS)),
                  [f"$N = {n}$, $\\lambda = {label}$" for n, _, label in ROWS],
                  fontsize=7)
    ax.set_xlabel("slice $a_{4,0}$")
    ax.grid(False)
    return image


n8_spread = [value for sid, value in spread.items() if sid.startswith("N8_")]
low8, high8 = f"{min(n8_spread):.3f}", f"{max(n8_spread):.3f}"  # three decimals
n8_text = (f"all have the spread ${low8}$" if low8 == high8  # equal when rounded
           else f"have spreads of ${low8}$ to ${high8}$")
fig, ax = plt.subplots(figsize=(6.4, 6.0))
image = draw_table(ax, state_table(spread), LogNorm(0.04, 1.0), ".3f")
fig.colorbar(image, ax=ax, label="spread of $\\rho$ / max|T| (C1 needs 0)")
ax.set_title("Condition C1: dependence on the hidden direction")
save_figure(fig, "c1_map",
            "The spread $(\\max_y \\rho - \\min_y \\rho)/\\max|T|$ of the energy "
            "density along the hidden coordinate for all 75 recorded Kohn-Sham "
            "ground states: rows are the particle numbers $N = 8, 136, 688$ with "
            "the five couplings, columns the slices $a_{4,0}$ (logarithmic colour "
            "scale, pure numbers). "
            "Condition C1 needs $0$ in every cell; the smallest value is "
            f"${spread[smallest]:.4f}$ ({describe(smallest)}), and the $N = 8$ "
            f"states, made of brane zero modes, {n8_text}, almost the whole "
            "size of the tensor. Grey cells: the five states with no source at all.")
```

Out [8]:

```
Figure 17a.2 saved as Revision/textbook/figures/17a_2_c1_map.png
```

Text cell:

```
## 9. Condition C2 point by point: $p_3 + p_t = 2p_0$

The next cell forms the **violation profile** $V(y) = p_3 + p_t - 2p_0$ of every
nonzero state (an admissible source has $V = 0$ at every $y$) and its size
$\max_y|V|/\max|T|$. It checks that the size is far from zero for every state and
that the smallest and largest values, with their states, are those of the record.
```

In [9]:

```
violation = {sid: profiles[sid]["p3"] + profiles[sid]["p_t"]
             - 2.0 * profiles[sid]["p0"] for sid in nonzero}  # V(y)
size = {sid: float(np.max(np.abs(violation[sid]))) / scale[sid] for sid in nonzero}
low = min(size, key=size.get)  # the state closest to C2
```

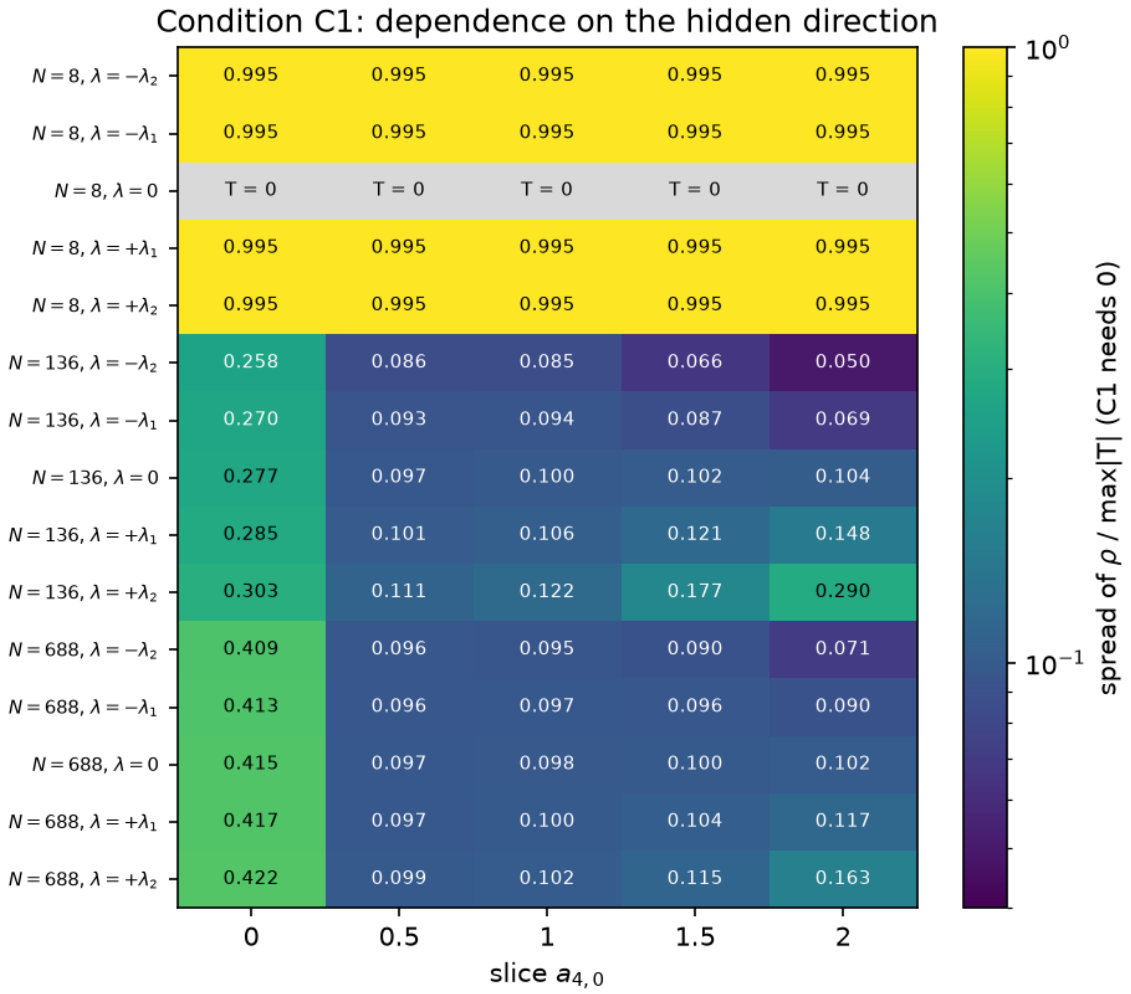

Figure 460: The spread $(\max_y \rho - \min_y \rho)/\max|T|$ of the energy density along the hidden coordinate for all 75 recorded Kohn-Sham ground states: rows are the particle numbers $N = 8, 136, 688$ with the five couplings, columns the slices $a_{4,0}$ (logarithmic colour scale, pure numbers). Condition C1 needs 0 in every cell; the smallest value is 0.0497 ($N = 136$, $\lambda = -\lambda_2$, $a_{4,0} = 2$), and the $N = 8$ states, made of brane zero modes, all have the spread 0.995, almost the whole size of the tensor. Grey cells: the five states with no source at all. (Notebook 17a, figure 2.)

```
high = max(size, key=size.get)  # the state farthest from C2
report("smallest max|V| / max|T|", f"{size[low]:.6g} ({low})")
report("largest max|V| / max|T|", f"{size[high]:.6g} ({high})")
detail = record_entry(SOURCE_REPORT, "ks_profiles_violate_algebraic_condition")[
    "detail"]
text = f"between {size[low]:.6g} ({low}) and {size[high]:.6g} ({high})"
reproduces(all(value > TOL for value in size.values()) and text in detail,
           "C2 fails point by point for every nonzero state",
           SOURCE_REPORT, ["ks_profiles_violate_algebraic_condition"])
```

Out [9]:

```
RESULT smallest max|V| / max|T| = 2.09192 (N136_lamp2_a20)
RESULT largest max|V| / max|T| = 3.99006 (N8_lamm1_a00)
PASS C2 fails point by point for every nonzero state
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_profiles_violate_algebraic_condition
```

Text cell:

```
The next cell draws the violation profiles $V(y)/\max|T|$ of the states with
$\lambda = 0$ for $N = 136$ (left) and $N = 688$ (right), at the five slices. The
vertical axis is **symmetric logarithmic**: logarithmic for large positive and
negative values, linear between $-10^{-7}$ and $10^{-7}$, so that both signs and
seven orders of magnitude are visible. The dashed line is the value $0$ that C2
demands. Before drawing, the cell counts how often each plotted profile changes
its sign between neighbouring grid points, and how large $|V|$ is at the tip and
at the brane; the caption quotes these numbers.
```

In [10]:

```
plotted = [f"N{n}_lam0_{tag}" for n in (136, 688) for tag in SLICES]
signs = {sid: np.sign(violation[sid]) for sid in plotted}  # +1, -1 (or 0)
changes = {sid: int(np.sum(signs[sid][1:] * signs[sid][:-1] < 0)) for sid in plotted}
at_tip = [abs(float(violation[sid][0])) / scale[sid] for sid in plotted]
at_brane = [abs(float(violation[sid][-1])) / scale[sid] for sid in plotted]
say(f"sign changes of the ten plotted profiles: {sorted(set(changes.values()))}")
check(all(count == 1 for count in changes.values())
      and min(at_brane) > 0.0,
      "each plotted V(y) changes sign exactly once and is nonzero at the brane")
fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8), sharey=True)
for ax, n in zip(axes, (136, 688)):
    for shade, (tag, a40) in zip(SHADES, SLICES.items()):
        sid = f"N{n}_lam0_{tag}"
        ax.plot(y, violation[sid] / scale[sid], color=shade, lw=1.8,
                label=f"$a_{{4,0}} = {a40:g}$")
    ax.axhline(0.0, color="#e34948", ls="--", lw=1.2, label="C2: $V = 0$")
    ax.set_yscale("symlog", linthresh=1e-7)
    ax.set_yticks([-1e-1, -1e-3, -1e-5, -1e-7, 0.0, 1e-7, 1e-5, 1e-3, 1e-1, 1e1])
    ax.set_xlabel("hidden coordinate $y$")
    ax.set_title(f"$N = {n}$, $\\lambda = 0$")
```

```
axes[0].set_ylabel("$V = p_3 + p_t - 2p_0$, divided by max|T|")
axes[1].legend(fontsize=7, loc="upper right")
fig.tight_layout()
save_figure(fig, "violation_profiles",
            "The violation profiles $V(y) = p_3 + p_t - 2p_0$ of the Kohn-Sham ground "
            "states with $\\lambda = 0$, divided by the largest component of each "
            "state, for $N = 136$ (left) and $N = 688$ (right) at the five slices "
            "$a_{4,0}$ (horizontal axis: the hidden coordinate $y$; vertical axis "
            "symmetric logarithmic, linear between $-10^{-7}$ and $10^{-7}$). "
            "Condition C2 of the field equations demands the dashed line $V = 0$; "
            f"instead $|V|$ is {min(at_tip):.2f} to {max(at_tip):.2f} times max|T| "
            "at the tip, every profile changes sign exactly once, and at the brane "
            f"$|V|$/max|T| is still between ${tex_number(min(at_brane))}$ and "
            f"${tex_number(max(at_brane))}$, small but not zero.")
```

Out [10]:

```
sign changes of the ten plotted profiles: [1]
PASS each plotted V(y) changes sign exactly once and is nonzero at the brane
```

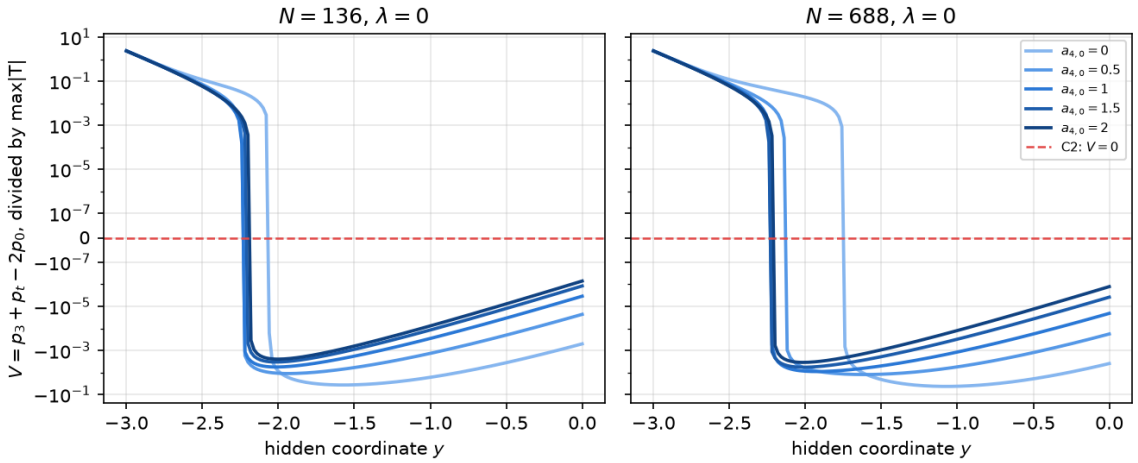

Figure 461: The violation profiles $V(y) = p_3 + p_t - 2p_0$ of the Kohn-Sham ground states with $\lambda = 0$, divided by the largest component of each state, for $N = 136$ (left) and $N = 688$ (right) at the five slices $a_{4,0}$ (horizontal axis: the hidden coordinate y; vertical axis symmetric logarithmic, linear between -10^{-7} and 10^{-7}). Condition C2 of the field equations demands the dashed line $V = 0$; instead $|V|$ is 2.36 to 2.37 times max|T| at the tip, every profile changes sign exactly once, and at the brane $|V|$/max|T| is still between 7.1×10^{-7} and 4.0×10^{-3}, small but not zero. (Notebook 17a, figure 3.)

Out [10]:

```
Figure 17a.3 saved as Revision/textbook/figures/17a_3_violation_profiles.png
```

Text cell:

```
The next cell draws the size $\max_y|V|/\max|T|$ for all 75 states as a heat map,
in the same layout as the map of C1 (linear colour scale).
```

In [11]:

```
from matplotlib.colors import Normalize  # a linear colour scale

fig, ax = plt.subplots(figsize=(6.4, 6.0))
image = draw_table(ax, state_table(size), Normalize(2.0, 4.0), ".3f")
fig.colorbar(image, ax=ax, label="max$|p_3 + p_t - 2p_0|$ / max|T| (C2 needs 0)")
ax.set_title("Condition C2: $p_3 + p_t = 2p_0$ point by point")
save_figure(fig, "c2_map",
            "The size $\\max_y|p_3 + p_t - 2p_0|/\\max|T|$ of the violation of "
            "condition C2 for all 75 recorded Kohn-Sham ground states (rows: particle "
            "number and coupling; columns: the slice $a_{4,0}$; linear colour scale, "
            "pure numbers). C2 needs $0$; every nonzero state violates it by "
            f"${size[low]:.2f}$ to ${size[high]:.2f}$ times its largest component. "
            "The $N = 8$ states, made of brane zero modes, have the same value at "
            "every slice; grey cells: no source.")
```

Out [11]:

```
Figure 17a.4 saved as Revision/textbook/figures/17a_4_c2_map.png
```

Text cell:

```
## 10. A worked example: three points of one state

The record illustrates the failure with the state $N = 136$, $\lambda = 0$,
$a_{4,0} = 1$ at three points: next to the tip ($y = -3$), in the middle
($y = -1.5$) and at the brane ($y = 0$). The next cell takes the grid point nearest
to each, prints the four components and $V$ with six significant digits, and checks
that the record prints exactly the same text. (A printed ‘-0‘ is the number zero
stored with a minus sign; it equals $0$.)
```

In [12]:

```
example = profiles["N136_lam0_a10"]
POINTS = (-3.0, -1.5, 0.0)  # tip, middle, brane
detail = record_entry(SOURCE_REPORT, "ks_profiles_violate_algebraic_condition")[
    "detail"]
rows = []
for target in POINTS:
    i = int(np.argmin(np.abs(example["y"] - target)))  # nearest grid point
    values = {c: float(example[c][i]) for c in ("y",) + COLUMNS}
    values["V"] = values["p3"] + values["p_t"] - 2.0 * values["p0"]
    rows.append(values)
    text = ("y = {y:.6g}: rho = {rho:.6g}, p3 = {p3:.6g}, p_t = {p_t:.6g}, "
            "p0 = {p0:.6g}, p3 + p_t - 2 p0 = {V:.6g}").format(**values)
```

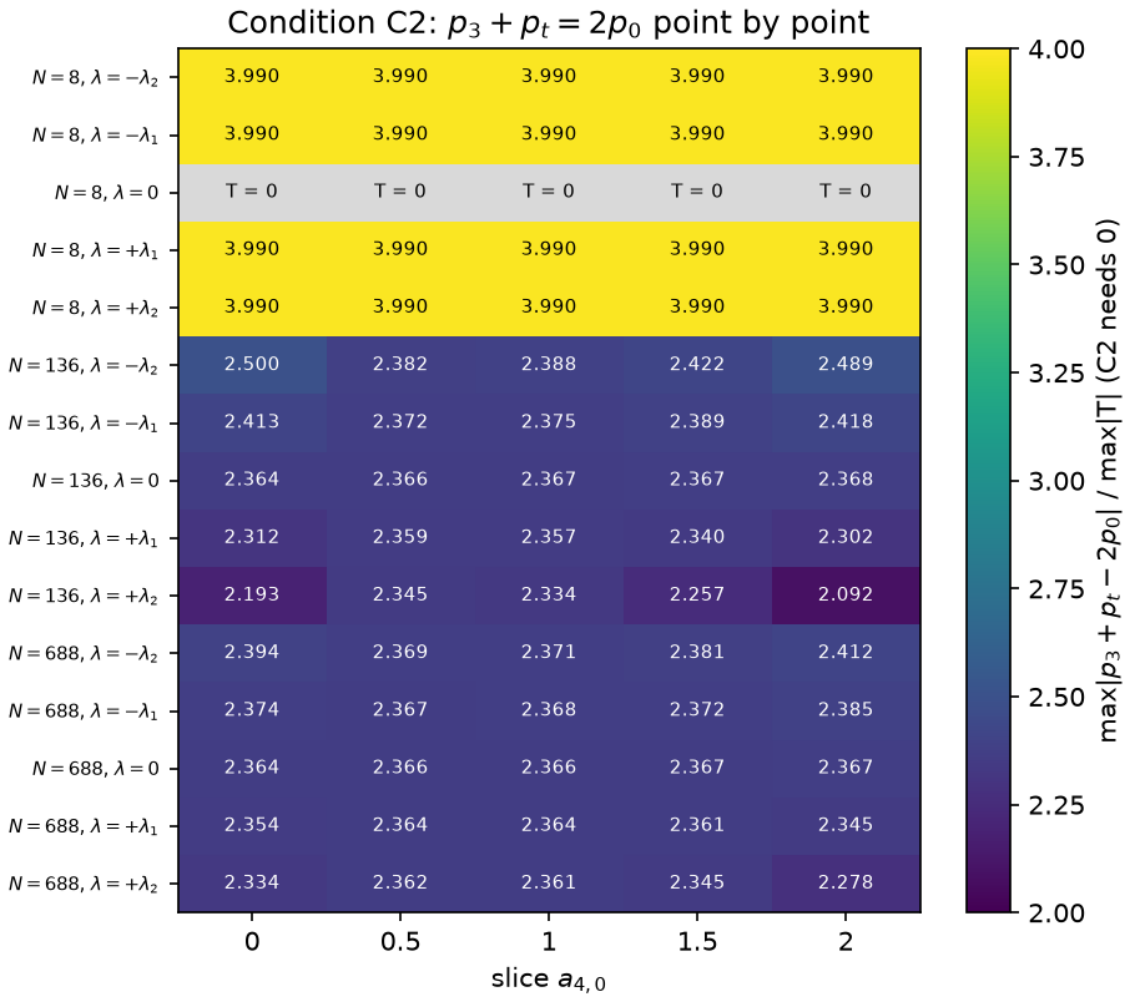

Figure 462: The size $\max_y |p_3 + p_t - 2p_0| / \max |T|$ of the violation of condition C2 for all 75 recorded Kohn-Sham ground states (rows: particle number and coupling; columns: the slice $a_{4,0}$; linear colour scale, pure numbers). C2 needs 0; every nonzero state violates it by 2.09 to 3.99 times its largest component. The $N = 8$ states, made of brane zero modes, have the same value at every slice; grey cells: no source. (Notebook 17a, figure 4.)

```
    say(text)
    values["in_record"] = text in detail  # the record prints the same text?
reproduces(all(r["in_record"] for r in rows) and all(r["V"] != 0 for r in rows),
           "the three example points of N136_lam0_a10 equal the record",
           SOURCE_REPORT, ["ks_profiles_violate_algebraic_condition"])
```

Out [12]:

```
y = -3: rho = 43.0702, p3 = 158.268, p_t = -0, p0 = -431.733, p3 + p_t - 2 p0 = 1021.73
y = -1.5: rho = 0.707818, p3 = 0.498835, p_t = 0, p0 = 0.643324, p3 + p_t - 2 p0 =
    -0.787813
y = -0: rho = 0.00211576, p3 = 0.000391237, p_t = -0, p0 = 0.000942049, p3 + p_t - 2 p0 =
    -0.00149286
PASS the three example points of N136_lam0_a10 equal the record
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_profiles_violate_algebraic_condition
```

Text cell:

```
The next cell draws, for each of the three points, the two sides of C2 as bars:
$p_3 + p_t$ and $2p_0$. C2 demands two bars of equal height at every point.
```

In [13]:

```
fig, axes = plt.subplots(1, 3, figsize=(9.0, 3.4))
for ax, values, where in zip(axes, rows, ("tip", "middle", "brane")):
    sides = [values["p3"] + values["p_t"], 2.0 * values["p0"]]
    bars = ax.bar(["$p_3 + p_t$", "$2p_0$"], sides, color=["#2a78d6", "#eb6834"])
    for bar, value in zip(bars, sides):
        ax.annotate(f"{value:.4g}", (bar.get_x() + bar.get_width() / 2, value),
                    ha="center", va="bottom" if value >= 0 else "top", fontsize=8)
    ax.axhline(0.0, color="0.3", lw=0.8)
    y_here = values["y"] + 0.0  # + 0.0 turns -0 into 0
    ax.set_title(f"{where}: $y = {y_here:g}$", fontsize=10)
    ax.margins(y=0.18)
axes[0].set_ylabel("pressure (units of $m^8$)")
fig.suptitle("$N = 136$, $\\lambda = 0$, $a_{4,0} = 1$: the two sides of C2")
fig.tight_layout()
tip, middle, brane = rows
factors = [2.0 * r["p0"] / (r["p3"] + r["p_t"]) for r in (middle, brane)]
tip_left, tip_right = tip["p3"] + tip["p_t"], 2.0 * tip["p0"]  # the two sides
save_figure(fig, "three_points",
            "The two sides of condition C2, $p_3 + p_t$ (blue) and $2p_0$ (orange), "
            "of the Kohn-Sham state $N = 136$, $\\lambda = 0$, $a_{4,0} = 1$ at the "
            "tip $y = -3$, in the middle $y = -1.5$ and at the brane $y = 0$ "
            "(vertical axes: proper pressure in units of $m^8$, each panel with its "
            "own scale). C2 demands equal bars; near the tip the two sides even have "
            f"opposite signs (${tip_left:.1f}$ against "
            f"${tip_right:.1f}$), in the middle $2p_0$ is {factors[0]:.2f} "
            f"times $p_3 + p_t$ and at the brane {factors[1]:.2f} times.")
```

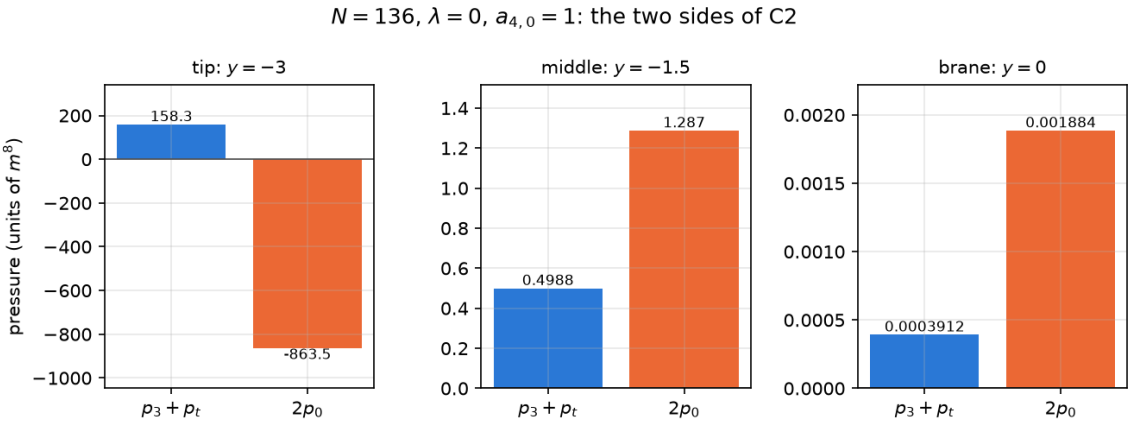

Figure 463: The two sides of condition C2, $p_3 + p_t$ (blue) and $2p_0$ (orange), of the Kohn-Sham state $N = 136$, $\lambda = 0$, $a_{4,0} = 1$ at the tip $y = -3$, in the middle $y = -1.5$ and at the brane $y = 0$ (vertical axes: proper pressure in units of m^8, each panel with its own scale). C2 demands equal bars; near the tip the two sides even have opposite signs (158.3 against -863.5), in the middle $2p_0$ is 2.58 times $p_3 + p_t$ and at the brane 4.82 times. (Notebook 17a, figure 5.)

Out [13]:

```
Figure 17a.5 saved as Revision/textbook/figures/17a_5_three_points.png
```

Text cell:

```
## 11. Condition C2 after integration over $x_0$

Could an average over the hidden direction rescue C2? A source that is averaged
over $x_0$ (a "dimensionally reduced" source) would need
$\int p_3 + \int p_t = 2\int p_0$, the integrals taken over the patch with the
proper-volume weight. The next cell computes the ratio
$(\int p_3 + \int p_t)/(2\int p_0)$ from the record's table of integrals for every
nonzero state. C2 would need the ratio $1$.
```

In [14]:

```
def integral(sid, column):
    """The integral 2 Vol_7 int e^{6Hy} X dy of column X from the record's table."""
    return float(integrals[sid][column])


ratio = {sid: (integral(sid, "int_p3") + integral(sid, "int_p_t"))
         / (2.0 * integral(sid, "int_p0"))
         for sid in nonzero if integral(sid, "int_p0") != 0.0}
closest = min(ratio, key=lambda sid: abs(ratio[sid] - 1.0))
report("ratio closest to 1", f"{ratio[closest]:.6g} ({closest})")
report("range of the ratio", f"{min(ratio.values()):.4f} to {max(ratio.values()):.4f}")
history_text = ", ".join(f"{sid}: {ratio[sid]:.6g}" for sid in
```

```
                        ("N136_lam0_a00", "N136_lam0_a10", "N136_lam0_a20"))
say("the history N = 136, lambda = 0: " + history_text)
detail = record_entry(SOURCE_REPORT, "ks_integrals_violate_algebraic_condition")[
    "detail"]
reproduces(len(ratio) == len(nonzero)
           and all(abs(r - 1.0) > TOL for r in ratio.values())
           and f"(closest to 1: {ratio[closest]:.6g} at {closest})" in detail
           and history_text in detail,
           "C2 fails also after integration over x0, for every nonzero state",
           SOURCE_REPORT, ["ks_integrals_violate_algebraic_condition"])
```

Out [14]:

```
RESULT ratio closest to 1 = 0.414328 (N688_lamm2_a00)
RESULT range of the ratio = 0.1072 to 0.4143
the history N = 136, lambda = 0: N136_lam0_a00: 0.339767, N136_lam0_a10: 0.25969,
    N136_lam0_a20: 0.239714
PASS C2 fails also after integration over x0, for every nonzero state
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_integrals_violate_algebraic_condition
```

Text cell:

```
The next cell draws the integrated ratio against the slice $a_{4,0}$ for every
particle number (colour) and coupling (line style and marker). The red dashed line
is the value $1$ that an averaged source would need.
```

In [15]:

```
N_COLOURS = {8: "#1baf7a", 136: "#2a78d6", 688: "#eb6834"}
STYLES = {"m2": ("v", ":"), "m1": ("<", "-."), "0": ("o", "-"),
          "p1": (">", "--"), "p2": ("^", (0, (1, 3)))}  # marker, line style
fig, ax = plt.subplots()
for n, tag, label in ROWS:
    ids = [f"N{n}_lam{tag}_{s}" for s in SLICES]
    if ids[0] not in ratio:
        continue  # N = 8, lambda = 0: no source
    marker, style = STYLES[tag]
    ax.plot(list(SLICES.values()), [ratio[sid] for sid in ids], marker=marker,
            ls=style, color=N_COLOURS[n], lw=1.3, ms=4)
ax.axhline(1.0, color="#e34948", ls="--", lw=1.4)
ax.text(1.0, 0.95, "C2 after integration needs 1", color="#e34948", ha="center",
        va="top", fontsize=9)
for n, colour in N_COLOURS.items():
    ax.plot([], [], color=colour, lw=2.0, label=f"$N = {n}$")
for tag, label in TAGS:
    marker, style = STYLES[tag]
    ax.plot([], [], color="0.3", marker=marker, ls=style, label=f"$\\lambda = {label}$")
ax.set_xlabel("slice $a_{4,0}$")
ax.set_ylabel("$(\\int p_3 + \\int p_t) / (2\\int p_0)$")
ax.set_ylim(0.0, 1.1)
```

```
ax.legend(fontsize=7, ncol=2, loc="center right")
near = max(max(ratio[f"N{n}_lam{tag}_{s}"] for tag, _ in TAGS)
           - min(ratio[f"N{n}_lam{tag}_{s}"] for tag, _ in TAGS)
           for n in (136, 688) for s in SLICES)  # spread over the couplings
save_figure(fig, "integrated_ratio",
            "The ratio $(\\int p_3 + \\int p_t)/(2\\int p_0)$ of the pressures "
            "integrated over the patch with the proper-volume weight, for every "
            "nonzero recorded Kohn-Sham state, against the slice $a_{4,0}$ "
            "(colours: particle number; line styles and markers: coupling; pure "
            "numbers). A source averaged over $x_0$ would need the value $1$ (red "
            "dashed line); the states lie between "
            f"${min(ratio.values()):.3f}$ and ${max(ratio.values()):.3f}$, so even "
            "the average violates condition C2. For $N = 136$ and $N = 688$ the "
            "five couplings give almost the same ratio (they differ by at most "
            f"${near:.4f}$ at one slice), so their lines lie on top of each other.")
```

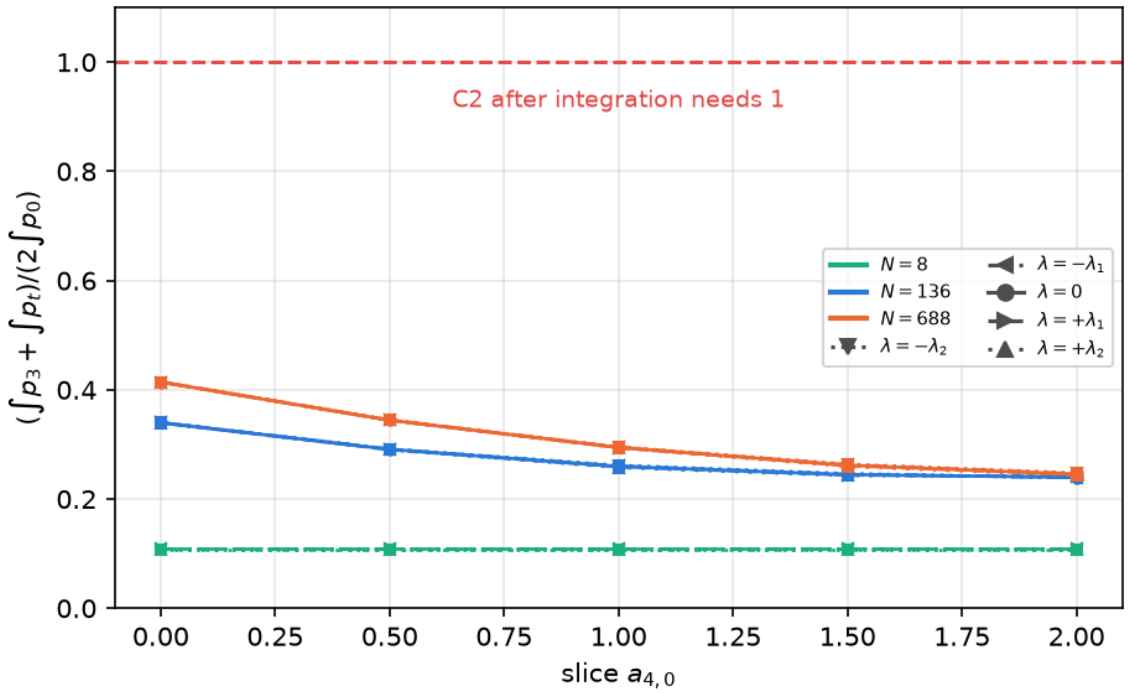

Figure 464: The ratio $(\int p_3 + \int p_t)/(2\int p_0)$ of the pressures integrated over the patch with the proper-volume weight, for every nonzero recorded Kohn-Sham state, against the slice $a_{4,0}$ (colours: particle number; line styles and markers: coupling; pure numbers). A source averaged over x_0 would need the value 1 (red dashed line); the states lie between 0.107 and 0.414, so even the average violates condition C2. For $N = 136$ and $N = 688$ the five couplings give almost the same ratio (they differ by at most 0.0045 at one slice), so their lines lie on top of each other. (Notebook 17a, figure 6.)

Out [15]:

```
Figure 17a.6 saved as Revision/textbook/figures/17a_6_integrated_ratio.png
```

Text cell:

12. Condition C3: the linear member along the history

The Kohn-Sham history is the linear member $a_4 = AHx_4$ with $A = 1$. For it the field equations need $p_3 = p_t = p_0$ and a constant ρ (C3). The record tests this on the integrals of the six states $N = 136$ and $N = 688$ with $\lambda = 0$ at the slices 0, 1 and 2. The next cell prints their integrated ρ, p_3, p_t, checks that ρ changes from slice to slice and that $p_3 \ne p_t$ (with the record's tolerance), and that the record prints the same numbers.

In [16]:

```
HISTORY = ["N136_lam0_a00", "N136_lam0_a10", "N136_lam0_a20",
           "N688_lam0_a00", "N688_lam0_a10", "N688_lam0_a20"]
pieces = []
for sid in HISTORY:
    r, p3, pt = (integral(sid, c) for c in ("int_rho", "int_p3", "int_p_t"))
    pieces.append(f"{sid}: int rho = {r:.6g}, int p3 = {p3:.6g}, int p_t = {pt:.6g}")
    say(pieces[-1])
rho_constant = all(
    abs(integral(a, "int_rho") - integral(b, "int_rho"))
    <= TOL * abs(integral(a, "int_rho"))
    for a, b in (("N136_lam0_a00", "N136_lam0_a10"),
                 ("N688_lam0_a00", "N688_lam0_a10")))
p_equal = all(abs(integral(s, "int_p3") - integral(s, "int_p_t"))
              <= TOL * max(abs(integral(s, "int_p3")), 1.0) for s in HISTORY)
detail = record_entry(SOURCE_REPORT, "ks_history_is_a_prescribed_background")[
    "detail"]
reproduces(not rho_constant and not p_equal and "; ".join(pieces) in detail,
           "C3 fails: along the history rho changes and p3 differs from p_t",
           SOURCE_REPORT, ["ks_history_is_a_prescribed_background"])
```

Out [16]:

```
N136_lam0_a00: int rho = 80.2822, int p3 = 23.8133, int p_t = 0
N136_lam0_a10: int rho = 32.3841, int p3 = 10.0397, int p_t = 0
N136_lam0_a20: int rho = 12.4451, int p3 = 4.06205, int p_t = 0
N688_lam0_a00: int rho = 680.441, int p3 = 199.305, int p_t = 0
N688_lam0_a10: int rho = 279.425, int p3 = 84.3387, int p_t = 0
N688_lam0_a20: int rho = 110.387, int p3 = 35.1354, int p_t = 0
PASS C3 fails: along the history rho changes and p3 differs from p_t
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_history_is_a_prescribed_background
```

Text cell:

The next cell draws the four integrated components of the states with $\lambda = 0$ at all five slices, for $N = 136$ (left) and $N = 688$ (right). C3 would need a horizontal ρ line and the three pressure lines on top of each other. Without interaction $p_t = e_{int} = 0$ exactly (the interaction energy density e_{int} is the only extra-time pressure of the good sector), so the p_t line lies on the axis.

In [17]:

```
PARTS = [("int_rho", "$\\int\\rho$", "#2a78d6", "o"),
         ("int_p3", "$\\int p_3$", "#eb6834", "s"),
         ("int_p_t", "$\\int p_t$", "#1baf7a", "^"),
         ("int_p0", "$\\int p_0$", "#4a3aa7", "D")]
fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
for ax, n in zip(axes, (136, 688)):
    ids = [f"N{n}_lam0_{s}" for s in SLICES]
    for column, label, colour, marker in PARTS:
        ax.plot(list(SLICES.values()), [integral(sid, column) for sid in ids],
                color=colour, marker=marker, lw=1.8, label=label)
    ax.set_xlabel("slice $a_{4,0}$ (the history $a_4 = Hx_4$)")
    ax.set_title(f"$N = {n}$, $\\lambda = 0$")
axes[0].set_ylabel("$2\\,\\mathrm{Vol}_7\\int e^{6Hy} X\\,dy$ (units of $m$)")
axes[1].legend(fontsize=8)
fig.tight_layout()
drop = [integral(f"N{n}_lam0_a00", "int_rho") / integral(f"N{n}_lam0_a20", "int_rho")
        for n in (136, 688)]
save_figure(fig, "history_integrals",
            "The energy density and the three pressures of the Kohn-Sham states with "
            "$\\lambda = 0$, integrated over the patch with the proper-volume weight, "
            "at the five slices of the history $a_4 = Hx_4$ (left $N = 136$, right "
            "$N = 688$; vertical axis in units of $m$ with $H = 1$; along the "
            "history 3-space inflates as $e^{a_4}$ and the extra times deflate as "
            "$e^{-a_4}$). The linear member "
            "needs a constant $\\rho$ and equal pressures (condition C3); instead "
            f"$\\int\\rho$ falls by a factor of {drop[0]:.2f} ($N = 136$) and "
            f"{drop[1]:.2f} ($N = 688$) from $a_{{4,0}} = 0$ to $2$, $\\int p_3$ stays "
            "above $\\int p_t = 0$, and $\\int p_0$ is different again.")
```

Out [17]:

```
Figure 17a.7 saved as Revision/textbook/figures/17a_7_history_integrals.png
```

Text cell:

```
The states $N = 8$ need a separate remark. They consist of the eight brane zero
modes, which have zero 3-momentum $k$. The slice $a_{4,0}$ enters the Kohn-Sham
equations only through the redshifted momentum $k e^{-a_{4,0}}$, while the proper
box stays the same (the exact rescaling identity of the Kohn-Sham record), so for
$k = 0$ nothing changes: these states are the same at every slice. The next cell
confirms that their profiles are identical at all five slices and that they have
$p_3 = p_t$; so they satisfy the part "constant $\rho$ and $p_3 = p_t$" of C3. They
are nevertheless not admissible: $p_0$ differs from $p_3$, and they fail C1 and C2
(the maps of sections 8 and 9).
```

In [18]:

```
same, p3_equals_pt, p0_differs = True, True, True
for tag in ("m2", "m1", "p1", "p2"):
```

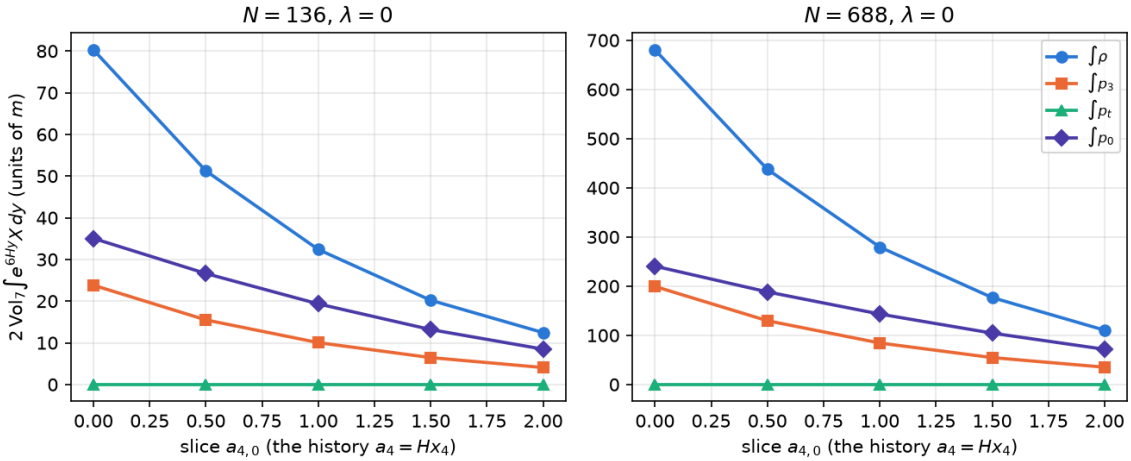

Figure 465: The energy density and the three pressures of the Kohn-Sham states with $\lambda = 0$, integrated over the patch with the proper-volume weight, at the five slices of the history $a_4 = Hx_4$ (left $N = 136$, right $N = 688$; vertical axis in units of m with $H = 1$; along the history 3-space inflates as e^{a_4} and the extra times deflate as e^{-a_4}). The linear member needs a constant ρ and equal pressures (condition C3); instead $\int \rho$ falls by a factor of 6.45 ($N = 136$) and 6.16 ($N = 688$) from $a_{4,0} = 0$ to 2, $\int p_3$ stays above $\int p_t = 0$, and $\int p_0$ is different again. (Notebook 17a, figure 7.)

```
    first = profiles[f"N8_lam{tag}_a00"]
    for slice_tag in SLICES:
        prof = profiles[f"N8_lam{tag}_{slice_tag}"]
        same = same and all(np.array_equal(prof[c], first[c]) for c in COLUMNS)
    p3_equals_pt = p3_equals_pt and np.array_equal(first["p3"], first["p_t"])
    p0_differs = p0_differs and float(np.max(np.abs(first["p0"] - first["p3"]))) > 0
int_p3_n8 = integral("N8_lamp1_a00", "int_p3")  # N = 8, lambda = +lambda_1
int_p0_n8 = integral("N8_lamp1_a00", "int_p0")
say(f"N = 8, lambda = +lambda_1: int p3 = {int_p3_n8:.6g}, int p0 = {int_p0_n8:.6g}")
check(same and p3_equals_pt and p0_differs,
      "N = 8: the same state at every slice, p3 = p_t, but p0 differs from p3")
```

Out [18]:

```
N = 8, lambda = +lambda_1: int p3 = -0.000977659, int p0 = -0.00889138
PASS N = 8: the same state at every slice, p3 = p_t, but p0 differs from p3
```

Text cell:

```
## 13. The states with no source at all

Five states have $T = 0$ at every point: $N = 8$ with $\lambda = 0$ (zero modes
with zero energy and no interaction). Is "no source" admissible? In Einstein
gravity ($\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$) the record's $x_4$ and $x_0$
equations read $3a_4'^2 + 21H^2 + \Lambda = -\kappa\rho$ and
$-3a_4'^2 + 15H^2 + \Lambda = \kappa p_0$. With $\rho = p_0 = 0$, subtracting the
second from the first gives $6a_4'^2 + 6H^2 = 0$, impossible for real $a_4'$ and
```

$H > 0$: the left side is a square times 6 plus a positive number. The next cell repeats this with sympy, from the record's text of the two equations (sympy is told that a_4' is real and H positive, and it decides that the difference is positive), and checks the list of zero states against the record.

In [19]:

```
kappa, rho, p0, Lam = sp.symbols("kappa rho p0 Lam", real=True)
NAMES.update({"kappa": kappa, "rho": rho, "p0": p0, "Lam": Lam})


def equation(text):
    """An equation (left == right) of the record as the expression left - right."""
    left, right = text.split("==")
    return parse(left) - parse(right)


einstein = equations["einstein"]
x4_equation = equation(einstein["constraint_x4"]["input"]).subs({rho: 0})
x0_equation = equation(einstein["hidden_x0"]["input"]).subs({p0: 0})
difference = sp.expand(x4_equation - x0_equation)  # Lambda cancels
say(f"x4 equation minus x0 equation with rho = p0 = 0: {difference} = 0")
say(f"is the left side positive for real a4' and H > 0? {difference.is_positive}")
reproduces(difference == 6 * ad1 ** 2 + 6 * H ** 2 and difference.is_positive is True,
           "Einstein gravity: no source at all (T = 0) has no solution for H > 0",
           WL_A4, ["einstein_no_vacuum_solution"])
detail = record_entry(SOURCE_REPORT, "ks_zero_source_states_listed")["detail"]
reproduces(", ".join(zero) in detail and f"{len(zero)} states" in detail,
           "the five states with T = 0 are those listed by the record",
           SOURCE_REPORT, ["ks_zero_source_states_listed"])
```

Out [19]:

```
x4 equation minus x0 equation with rho = p0 = 0: 6*H**2 + 6*ad1**2 = 0
is the left side positive for real a4' and H > 0? True
PASS Einstein gravity: no source at all (T = 0) has no solution for H > 0
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    einstein_no_vacuum_solution
PASS the five states with T = 0 are those listed by the record
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_zero_source_states_listed
```

Text cell:

```
## 14. The last check

The last cell checks that the seven figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [20]:

```
names = ["rho_profiles", "c1_map", "violation_profiles", "c2_map", "three_points",
         "integrated_ratio", "history_integrals"]
paths = [output_file(f"{FIGURE_FOLDER}/17a_{k}_{name}.png")
         for k, name in enumerate(names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [20]:

```
PASS every figure file of this notebook exists
ALL 16 CHECKS PASSED (notebook 17a)
```

Text cell:

```
## 15. What this notebook showed

- PROVED (exact algebra with sympy on the Lovelock components stored in the a4
  record): the left-hand sides of the Einstein-Lovelock equations of the author's
  metric are free of
  $x_0$ and have no $x_4x_0$ component (C1), obey
  $E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5} = 2E^{x_0}{}_{x_0}$ for every order (C2:
  $p_3 + p_t = 2p_0$), and for the linear member need equal pressures and a
  constant energy density (C3).
- COMPUTED (the 75 recorded Kohn-Sham ground states; every number of the five
  checks of the record `ks-source-conditions.json` reproduced): every nonzero
  state depends on $x_0$ (the spread of $\rho$ is at least $0.0497$ of max|T|),
  violates C2 point by point (by $2.09$ to $3.99$ times max|T|) and after
  integration over $x_0$ (ratios between $0.107$ and $0.414$ instead of $1$), and
  along the history $\rho$ changes and $p_3 \ne p_t$. The $N = 8$ states do not
  change along the history, but they too fail C1 and C2.
- PROVED (Einstein gravity): $T = 0$ is not a solution for $H > 0$, so the five
  states without a source are not admissible either.
- Hence no recorded Kohn-Sham ground state is an admissible source (the 135
  thermal states of the canonical matrix store no profiles and are not tested).
  The Kohn-Sham history $a_4 = Hx_4$ is a PRESCRIBED BACKGROUND (ASSUMED, not
  solved for): the gas is a test field without back-reaction, and energies,
  pressures and equations of state computed along it are not consequences of the
  coupled field equations.
- WHY (worked out in Notebook 17b): every source of the field equations must be
  conserved, and for a conserved source C2 says exactly that $p_0$ does not
  change along $x_0$, while C3 says exactly that no energy is exchanged with the
  inflating 3-space and the deflating extra times ($p_3 = p_t$). The Kohn-Sham
  states obey the conservation law of the hidden direction at every point, and
  the energy law of the time direction only for the total energy of the patch;
  their $p_0$ varies strongly along $x_0$ and their $p_3$ exceeds $p_t$.
- NOT shown: that no state of dirac16complex could ever be a source of the
  author's metric, or what metric the Kohn-Sham gas would produce with
  back-reaction (OPEN).
```

17.11 Line-by-line walk-through of Notebook 17a

The notebook has 20 code cells, In [1] to In [20]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted, then explained. Where a line repeats a pattern already explained, we say so and explain only what is new. Each figure is described after the cell that draws it.

In [1], the set-up cell. Every line that starts with `#` is a **comment**: Python skips it. The comment lines at the top of the cell are the complete run instructions of Section 17.9, so that the notebook file carries its own instructions even when it is copied without the book. The code starts after the comment line that announces THE SET-UP (the same in every notebook of this book; it computes no physics).

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module**, a part of Python or of an installed package, so that the code may use it. `json`, `os`, `textwrap` and `pathlib` come with Python. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system. The text after `#` on each line is a comment.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell.

```
NOTEBOOK_ID = "17a"  # this notebook: chapter 17, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotation marks) `"17a"`. The figure files and the last line of the notebook are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs each time it is called. The text in triple quotation marks under the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd()` is the folder in which Jupyter runs the notebook, and `.resolve()` writes it as a complete address. `here.parents` lists the folders above it; `[here, *here.parents]` is a **list** (an ordered collection in square brackets) that starts with `here` and continues with those folders (the star unpacks one list into another). The `for` loop visits the folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains `Revision/textbook/requirements.txt` is the repository, and `return` hands it back to the caller. If no folder does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The first code line calls the function and names its result `REPO`. The second chooses where files are written: `os.environ` holds the **environment variables** of the program (named texts it receives from the computer), and `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and `str(REPO)` (the repository folder as a string) otherwise. When you run the notebook the variable is not set, so files go into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions. `repository_file` gives the full path of a file of the repository, for reading a Revision record. `output_file` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder of the file and every missing folder above it, and does nothing if they exist.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters; `textwrap.fill` breaks the text at blanks and starts every line after the first with four blanks. That is why long printed lines of this notebook continue on an indented second line.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings of matplotlib, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets the default size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid (`grid.alpha` 0.3 means 30 per cent opaque). The braces make a **dictionary**: pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: a name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/17a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty JSON dictionary and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored as bytes and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns `len(FIGURE_NUMBERS) + 1`, one more than the number of figures so far: the figures are numbered 1, 2, 3, and a cell run twice keeps its number. The file name joins the notebook id, the number and the name, for example `17a_1_rho_profiles.png`. `fig.savefig` writes a PNG picture with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory, because Jupyter would

otherwise draw it a second time. The caption is stored, the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys), `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` starts as an empty list. `check` is the function behind every check of the notebook. If `condition` is false, `raise AssertionError(...)` stops the notebook with an error that names the check. Otherwise the name is appended to `PASSED`, the line PASS name is printed, and, when `record` is given, a second line names the Revision record that the check reproduces. `record=None` makes that argument optional.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds the unit only when one is given. `all_checks_passed` prints the last line of the notebook; `len(PASSED)` is the number of checks that passed. The last statement prints the one output line of In [1]: Set-up of notebook 17a complete.

In [2], the Revision records and the helpers that read them.

```
import csv  # reads tables stored as CSV files (comma-separated values)

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
```

`csv` comes with Python and reads tables stored as text with commas between the entries. numpy (short name `np`) works with **arrays**, tables of numbers on which arithmetic acts entry by entry. sympy (short name `sp`) does exact algebra with symbols.

```
SOURCE_REPORT = "Revision/field_equations_a4/reports/ks-source-conditions.json"
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
PY_A4 = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy checks
WL_A4 = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram
GROUND = "Revision/kohn_sham/results/ground"  # the 75 Kohn-Sham ground states
```

Five names for the Revision records used below: the record on the Kohn-Sham source conditions (the record this notebook reproduces), the record of the a_4 equations, its two verification reports (sympy and Wolfram), and the folder of the recorded Kohn-Sham ground states.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_entry(report_file, name):
    """The entry {name, verdict, detail} of a check of a report, None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry
    return None
```

`read_json` reads a JSON file of the repository as text and turns it into Python dictionaries and lists (`json.loads`). Every Revision report has a list `checks` whose entries have a `name`, a `verdict` (PASS or FAIL) and a `detail` text. `record_entry` walks through that list and returns the entry with the given name; `None` (Python's "nothing") if there is none.

```
def reproduces(condition, name, report_file, record_names):
    """A check that also requires the named checks of the report to be PASS."""
    found = all((record_entry(report_file, n) or {}).get("verdict") == "PASS"
                for n in record_names)
    names_text = ", ".join(record_names)  # the check names, separated by commas
    check(condition and found, name, record=f"{report_file}, check {names_text}")
```

`reproduces` is the check used whenever a result of the notebook reproduces a Revision record. `all(... for n in record_names)` is true when the expression holds for every name in the list. For one name, `record_entry(...) or {}` is the entry, or an empty dictionary if the entry is missing; `.get("verdict")` reads its verdict (or `None`). So `found` is true only when every named check of the report exists and has the verdict PASS. `", ".join(...)` joins the names with commas. The check passes only when the notebook's own `condition` holds AND the record agrees; its second printed line names the report and the checks.

```
def tex_number(value, digits=2):
    """A number for a caption in powers of ten: 0.0123 -> 1.2 \\times 10^{-2}."""
    mantissa, exponent = f"{value:.{digits - 1}e}".split("e")
    return f"{mantissa} \\times 10^{{{int(exponent)}}}"
```

`tex_number` writes a number in powers of ten for the captions. The format `.1e` (here `digits - 1 = 1`) writes `0.0123` as `1.2e-02`; `.split("e")` cuts this text at the letter e into `"1.2"` and `"-02"`; `int("-02")` is the whole number -2. The result is the text `1.2 \times 10^{-2}`, which the book prints as 1.2×10^{-2}. (In a Python string a backslash is written twice, and in an f-string a brace is written twice, so `{{{...}}}` gives one literal brace around the value.)

```
source_record = read_json(SOURCE_REPORT)
summary = source_record["summary"]  # how many checks the record has, how many pass
passed, total = summary["pass"], summary["checks"]
report("checks of the record ks-source-conditions.json", f"{passed} of {total} PASS")
say("its conclusion: " + source_record["conclusion"])
check(summary == {"checks": 6, "pass": 6, "fail": 0},
      "the record on the Kohn-Sham source conditions has 6 checks, all PASS")
```

The record is read; its `summary` dictionary says how many checks it has and how many passed. `passed, total = a, b` gives two names at once. The cell prints the count and the record's conclusion, and checks that the record has 6 checks, all PASS. Output: RESULT checks of the record ks-source-conditions.json = 6 of 6 PASS, the record's conclusion (no recorded Kohn-Sham state is an admissible source; C1 and C2 fail for every nonzero state, C2 also after integration over x_0; the history is a prescribed background without back-reaction), and the first PASS line.

In [3], the Lovelock components of the record as sympy expressions.

```
equations = read_json(EQUATIONS)
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4'' (the record's names)
H = sp.symbols("H", positive=True)  # the constant H of the metric
NAMES = {"ad1": ad1, "ad2": ad2, "H": H}  # how the record's names become symbols
```

The record of the a_4 equations is read. `sp.symbols` makes sympy **symbols**, letters that sympy computes with exactly: `ad1` and `ad2` stand for a_4' and a_4'' and are declared real numbers; `H` is declared positive. `NAMES` tells sympy which symbol each name of the record means.

```
def parse(text):
    """A formula of the record (Wolfram Language text) as a sympy expression."""
    return sp.sympify(text.replace("^", "**"), locals=NAMES)
```

The record stores its formulas as text in the Wolfram Language, where a power is written `^`; Python writes `**`. `parse` replaces the one by the other and lets `sp.sympify` read the text as a sympy expression, with the names of `NAMES`.

```
COMPONENTS = ("x1x1", "x4x4", "x5x5", "x0x0", "x4x0")
E = {k: {c: parse(equations["lovelockTensors"][f"E{k}"][c]["input"])
         for c in COMPONENTS}
     for k in (1, 2, 3)}  # E[k][component]: the Lovelock tensor of order k
for c in COMPONENTS:
    say(f"E_(1)^{c[:2]}_{c[2:]} = {E[1][c]}")
```

`COMPONENTS` is a **tuple** (a fixed list in round brackets) of the five components needed. The next line is a **dictionary comprehension** inside another: for each order $k = 1, 2, 3$ and each component `c`

it reads the record's text `equations["lovelockTensors"]["E1"]["x1x1"]["input"]` (and so on) and parses it. `E[2]["x5x5"]` is then $E_{(2)}{}^{x_5}{}_{x_5}$. The loop prints the five components of Einstein's tensor; `c[:2]` is the first two characters of the name (`x1`) and `c[2:]` the rest. Output: `E_(1)^x1_x1 = 15*H**2 - 3*ad1**2 + ad2`, `E_(1)^x4_x4 = 21*H**2 + 3*ad1**2`, `E_(1)^x5_x5 = 15*H**2 - 3*ad1**2 - ad2`, `E_(1)^x0_x0 = 15*H**2 - 3*ad1**2` and `E_(1)^x4_x0 = 0`, the Einstein components of Section 17.3.

In [4], the three conditions on the left-hand sides.

```
allowed = {ad1, ad2, H}  # the only symbols a left-hand side may contain
no_x0 = all(E[k][c].free_symbols <= allowed for k in E for c in COMPONENTS)
reproduces(no_x0 and all(E[k]["x4x0"] == 0 for k in E),
           "C1: the left-hand sides are free of x0 and the x4-x0 one is 0",
           WL_A4, ["P1_structure", "P2_structure", "P3_structure"])
```

`{ad1, ad2, H}` is a **set**, a collection without order. `.free_symbols` is the set of symbols that an expression contains, and `<=` between sets means "is contained in". So `no_x0` is true when no component of any order contains anything but a_4', a_4'' and H. (A dependence on x_0 would have to appear as a symbol such as `x0` or `cc`.) Together with "every x_4x_0 component is zero" this is the left-hand side of C1 (Section 17.4); the check also requires the three structure checks of the Wolfram report to be PASS.

```
identity = [sp.expand(E[k]["x1x1"] + E[k]["x5x5"] - 2 * E[k]["x0x0"]) for k in E]
say(f"E^x1_x1 + E^x5_x5 - 2 E^x0_x0 for k = 1, 2, 3: {identity}")
reproduces(all(value == 0 for value in identity),
           "C2: E^x1_x1 + E^x5_x5 = 2 E^x0_x0 for every order k",
           WL_A4, ["algebraic_identity_x1_plus_x5_minus_2x0"])
```

For each order, `sp.expand` multiplies out $E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5} - 2E^{x_0}{}_{x_0}$ and collects equal terms. The printed list is `[0, 0, 0]`: the identity behind C2 (Section 17.5) holds for each order separately.

```
linear = {ad2: 0}  # the linear member: a4'' = 0 (and a4' = A H, a constant)
equal = all(sp.expand((E[k]["x1x1"] - E[k]["x0x0"]).subs(linear)) == 0
            and sp.expand((E[k]["x5x5"] - E[k]["x0x0"]).subs(linear)) == 0
            for k in E)
rho_side = all(E[k]["x4x4"].free_symbols <= {ad1, H} for k in E)
reproduces(equal and rho_side,
           "C3: for a4'' = 0 the pressures are equal and rho is constant",
           PY_A4, ["linear_member_equal_pressures"])
```

`.subs(linear)` replaces a_4'' by 0. Then `equal` is true when, for every order, the x_1 and x_5 components both equal the x_0 component; so the three pressure equations have the same left-hand side and give $p_3 = p_t = p_0$. `rho_side` is true when the x_4 components contain only a_4' and H; with $a_4' = AH$ constant the constraint then gives a constant ρ. That is C3 (Section 17.6), reproduced from the sympy report. Output: three PASS lines, each followed by the record it reproduces.

In [5], the recorded Kohn-Sham states.

```
COLUMNS = ("rho", "p3", "p_t", "p0")  # the four diagonal components of the source
```

```
def read_profile(path):
    """A profile table as a dictionary: column name -> numpy array (151 values)."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}
```

`COLUMNS` names the four columns of a profile file that hold the source. `np.genfromtxt` reads a table of numbers from a text file; `delimiter=","` says that commas separate the entries, and `names=True` that the first line holds the column names. `data.dtype.names` lists them, and the function returns a dictionary from each name to its column, an array of 151 numbers.

```
files = sorted(repository_file(f"{GROUND}/profiles").glob("*.csv"),
               key=lambda path: path.name)  # in alphabetical order, as the record
profiles = {path.stem: read_profile(path) for path in files}
```

`.glob("*.csv")` finds every file whose name ends in `.csv` in the folder of profiles; `sorted(..., key=lambda path: path.name)` puts them in alphabetical order of their names (a `lambda` is a one-line function without a name; here it returns the file name). `path.stem` is the file name without `.csv`, the name of the state, such as `N136_lam0_a10`. So `profiles["N136_lam0_a10"]["p0"]` is the hidden pressure of that state at the 151 points.

```
with repository_file(f"{GROUND}/emt-integrals.csv").open(
        encoding="utf-8", newline="") as handle:
    integrals = {row["id"]: row for row in csv.DictReader(handle)}
physics = read_json("Revision/kohn_sham/results/parameters.json")["physics"]
```

`with ... as handle:` opens the table of integrals and closes it again at the end of the indented block. `csv.DictReader` reads it row by row, each row as a dictionary from the column name to the entry (as text); `integrals` maps the state name in the column `id` to its row. `physics` is the part of the solver's parameter record with H, the tip cutoff and Vol_7.

```
scale = {sid: float(max(np.max(np.abs(prof[c])) for c in COLUMNS))
         for sid, prof in profiles.items()}  # max|T| of every state
zero = [sid for sid in profiles if scale[sid] == 0.0]  # T = 0 everywhere
nonzero = [sid for sid in profiles if scale[sid] > 0.0]
```

For each state (`sid`, the state's name, and `prof`, its profile), `np.abs` takes absolute values entry by entry, `np.max` the largest of a column, and `max(...)` the largest of the four columns: this is max|T|. A **list comprehension** `[sid for sid in profiles if ...]` collects the names that satisfy a condition: `zero` holds the states whose tensor vanishes everywhere, `nonzero` the others.

```
y = profiles["N136_lam0_a10"]["y"]  # the grid of the hidden coordinate
same_grid = all(np.array_equal(prof["y"], y) for prof in profiles.values())
report("states read", len(profiles))
report("states with a nonzero tensor", len(nonzero))
report("grid of y", f"{len(y)} points from {y[0]:g} to {y[-1] + 0.0:g}, "
       f"step {y[1] - y[0]:.2f}")
```

`y` is the column of grid points of one state; `np.array_equal` checks that every state uses exactly the same points. `y[0]` is the first point and `y[-1]` the last (a negative index counts from the end). The last point is stored as -0 (a zero with a minus sign, which a computer can hold); adding `0.0` turns it into $+0$, so that it prints as 0. The format `:g` writes a number in its shortest form and `:.2f` with two decimals.

```
H_value, L_cut, VOL7 = physics["H"], physics["L_tipCutoff"], physics["Vol7"]
report("H, tip cutoff L, Vol_7", f"{H_value:g}, {L_cut:g}, {VOL7:.6g}")
say("ks-theory.json, emt.offDiagonal: "
    + read_json("Revision/kohn_sham/ks-theory.json")["emt"]["offDiagonal"])
check(len(profiles) == 75 and len(nonzero) == 70 and len(zero) == 5 and same_grid
      and sorted(integrals) == sorted(profiles),
      "75 states on one grid (70 nonzero, 5 zero); every state has its integrals")
```

The three numbers of the run are read and printed; `:.6g` keeps six significant digits. The cell prints the Kohn-Sham theory record's statement on the mixed component, and checks the counts, the common grid, and that the table of integrals has exactly the same states (`sorted` of a dictionary is the sorted list of its keys). Output: 75 states, 70 with a nonzero tensor, 151 points from -3 to 0 with step 0.02; $H = 1$, $L = 3$, $\mathrm{Vol}_7 = 15875.2$; the statement that ${T^{x_4}}_y = 0$ for eigen-orbitals; and a PASS line.

In [6], the hidden coordinate and the energy profiles (figure 1).

```
SLICES = {"a00": 0.0, "a05": 0.5, "a10": 1.0, "a15": 1.5, "a20": 2.0}  # a4,0
SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light -> dark
history136 = [f"N136_lam0_{tag}" for tag in SLICES]  # N = 136, lambda = 0
```

`SLICES` maps the slice tag of a state name to the value $a_{4,0}$. `SHADES` are five colours from light to dark blue, written as **hexadecimal colour codes** (two digits each for red, green and blue). `history136` lists the five states with $N = 136$, $\lambda = 0$; a loop over a dictionary visits its keys in the order they were written.

```
falls = [float(np.max(profiles[s]["rho"]) / np.min(profiles[s]["rho"]))
         for s in history136]  # how much rho changes along y, slice by slice
check(all(float(np.min(profiles[s]["rho"])) > 0.0 for s in history136),
      "N = 136, lambda = 0: rho is positive at every point of every slice")
```

For each of the five states, `falls` holds the ratio of the largest to the smallest value of ρ along y. The check confirms that ρ is positive everywhere for these states, which a logarithmic axis needs (the logarithm of zero or of a negative number does not exist). Output: the PASS line.

```
z = np.logspace(-9, np.log10(np.pi / 2), 400)  # z from 1e-9 to pi/2
z_tip = float(np.arcsin(np.exp(-18.0)))  # where y = -3 (6 H y = -18 with H = 1)
```

`np.logspace(a, b, 400)` gives 400 numbers from 10^a to 10^b, equally spaced on a logarithmic scale: here from 10^{-9} to $\pi/2$. The tip cutoff $y = -3$ sits where $\sin z = e^{6Hy} = e^{-18}$, that is $z = \arcsin(e^{-18})$, about 1.5×10^{-8}.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.plot(z, np.log(np.sin(z)) / 6.0, color="#2a78d6", lw=2.0)
left.axvspan(z_tip, np.pi / 2, color="#1baf7a", alpha=0.15,
             label="computed patch, $-3 \\leq y \\leq 0$")
left.axhline(-3.0, color="0.4", ls="--", lw=1.0)
left.set_xscale("log")
left.set_xlabel("$z = 6Hx_0$ (logarithmic axis)")
left.set_ylabel("hidden coordinate $y$ (units of $1/H$)")
left.set_title("$y = \\ln(\\sin z)/(6H)$, $H = 1$")
left.legend(loc="lower right", fontsize=8)
```

`plt.subplots(1, 2, ...)` makes a figure with one row of two panels (called **axes** in matplotlib), 9.0 by 3.8 inches; the two panels are named `left` and `right`. The left panel draws $y = \ln(\sin z)/6$ (with $H = 1$) against z; `lw` is the line width. `axvspan` shades the vertical band from the tip cutoff to $\pi/2$, the computed patch, with 15 per cent opacity; `axhline` draws a dashed grey horizontal line (the line style `ls` given as two hyphens) at $y = -3$. `set_xscale("log")` makes the horizontal axis logarithmic. The labels and the title are written with `$...$`, which matplotlib prints as mathematics (a backslash is written twice inside a Python string). `legend` shows the label of the shaded band.

```
for shade, sid, a40 in zip(SHADES, history136, SLICES.values()):
    right.plot(y, profiles[sid]["rho"] / scale[sid], color=shade, lw=1.8,
               label=f"$a_{{4,0}} = {a40:g}$")
right.axhline(1e-3, color="0.4", ls=":", lw=1.2,
              label="admissible: constant (example)")
right.set_yscale("log")
right.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
right.set_ylabel("$\\rho(y)$ / max|T|")
right.set_title("$N = 136$, $\\lambda = 0$")
right.legend(fontsize=7, loc="upper right")
fig.tight_layout()
```

`zip` walks through three lists side by side: a colour, a state name and its slice value. Each pass draws $\rho(y)$ divided by max|T| of the state. A dotted line (`ls=":"`) at 10^{-3} shows, as an example, what an admissible profile looks like: a constant. `set_yscale("log")` makes the vertical axis logarithmic. `fig.tight_layout()` arranges the panels so that their labels do not overlap.

```
save_figure(fig, "rho_profiles",
            "Left: the hidden coordinate $y = \\ln(\\sin z)/(6H)$ against $z = 6Hx_0$ "
            "(logarithmic axis, $H = 1$); the shaded band is the computed patch from "
            "the tip cutoff $y = -3$ to the brane $y = 0$, which covers $z$ from "
            f"${z_tip / 1e-8:.1f} \\times 10^{{-8}}$ to $\\pi/2$. Right: the energy "
            "density $\\rho(y)$ of the Kohn-Sham state $N = 136$, $\\lambda = 0$ "
            "divided by the largest component of the state, at the five slices "
            "$a_{4,0} = 0$ to $2$ (logarithmic vertical axis). The largest value of "
            f"$\\rho$ is {min(falls):.0f} to ${tex_number(max(falls))}$ times its "
            "smallest "
            "value, depending on the slice, while condition C1 of the field "
            "equations demands a horizontal line such as the dotted one.")
```

`save_figure` saves the figure and records the caption. The caption is written as twelve strings next to each other, one per line inside the round brackets; Python joins strings written next to each other

into one string, so the twelve lines make one caption. Most of them are plain strings. Two start with `f` and contain computed numbers in braces: `z_tip / 1e-8` written with one decimal (`:.1f`) gives 1.5, so the text reads 1.5×10^{-8} (the doubled braces `{{-8}}` print one pair of braces, which the book's mathematics needs); `min(falls)` with no decimals (`:.0f`) gives 319, and `tex_numb er(max(falls))` gives 1.1×10^{5}. The backslashes are doubled because a single backslash in a Python string starts a special character. Output: figure 17a.1 and the line Figure 17a.1 saved as Revision/textbook/figures/17a_1_rho_profiles.png.

What figure 17a.1 shows. Left: the curve $y(z)$ is a straight line on the logarithmic z axis, because $y = \ln(\sin z)/6 \approx \ln(z)/6$ for small z; the shaded patch $-3 \le y \le 0$ covers z from 1.5×10^{-8} to $\pi/2$, almost the whole hidden direction. Right: the energy density of the five states $N = 136$, $\lambda = 0$ on a logarithmic axis. Near the tip it is about a tenth of max|T|; toward the brane it falls by two to five powers of ten, the more the later the slice (the largest value is 319 to 1.1×10^{5} times the smallest). Condition C1 demands a horizontal line such as the dotted example. The student should see at a glance that C1 fails, and by a huge margin.

In [7], condition C1 measured.

```
TOL = 1e-6  # the relative tolerance of the record
spread = {sid: float(np.max(profiles[sid]["rho"]) - np.min(profiles[sid]["rho"]))
          / scale[sid] for sid in nonzero}
smallest = min(spread, key=spread.get)  # the state that depends least on x0
largest = max(spread, key=spread.get)  # the state that depends most on x0
report("smallest spread of rho / max|T|", f"{spread[smallest]:.6g} ({smallest})")
report("largest spread of rho / max|T|", f"{spread[largest]:.6g} ({largest})")
```

`TOL` is the record's tolerance. `spread` holds, for each of the 70 nonzero states, the spread of ρ divided by max|T| (Section 17.4). `min(spread, key=spread.get)` returns the name whose value is smallest (`spread.get` looks a value up by its name), and `max` the largest. Output: the smallest spread is 0.0497329 (state `N136_lamm2_a20`), the largest 0.995037 (`N8_lamm1_a00`).

```
detail = record_entry(SOURCE_REPORT, "ks_profiles_depend_on_x0")["detail"]
text = f">= {spread[smallest]:.6g} (smallest: {smallest})"  # as the record prints it
reproduces(all(value > TOL for value in spread.values()) and text in detail
           and "75 ground-state profiles" in detail
           and "70 with a nonzero" in detail,
           "C1 fails: every nonzero state depends on x0",
           SOURCE_REPORT, ["ks_profiles_depend_on_x0"])
```

`detail` is the record's text for its check `ks_profiles_depend_on_x0`. `text` is our smallest value written exactly as the record writes it; `text in detail` is true when the record contains this text. The check passes when every spread exceeds the tolerance and the record states our numbers and counts. Output: PASS C1 fails: every nonzero state depends on x0, reproducing the record's check. (COMPUTED; the smallest spread, 0.0497, is fifty thousand times the tolerance.)

In [8], the map of C1 (figure 2).

```
from matplotlib.colors import LogNorm  # a logarithmic colour scale
```

```
TAGS = [("m2", "-\\lambda_2"), ("m1", "-\\lambda_1"), ("0", "0"),
        ("p1", "+\\lambda_1"), ("p2", "+\\lambda_2")]  # the couplings, in order
ROWS = [(n, tag, label) for n in (8, 136, 688) for tag, label in TAGS]
```

`LogNorm` turns values into colours on a logarithmic scale. `TAGS` pairs each coupling tag of the state names with its mathematical label. `ROWS` lists the 15 rows of the map: each particle number with each coupling.

```
def describe(sid):
    """The state name N136_lamm2_a20 written as N = 136, lambda = -lambda_2, ..."""
    n, lam, slice_tag = sid.split("_")
    label = dict(TAGS)[lam[3:]]  # lam[3:] is m2, m1, 0, p1 or p2
    return (f"$N = {n[1:]}$, $\\lambda = {label}$, "
            f"$a_{{4,0}} = {SLICES[slice_tag]:g}$")
```

`describe` turns a state name into mathematics for a caption. `sid.split("_")` cuts `N136_lamm2_a20` at the underscores into `N136`, `lamm2` and `a20`. `n[1:]` drops the letter N; `lam[3:]` drops `lam` and leaves `m2`; `dict(TAGS)` turns the list of pairs into a dictionary, which gives $-\lambda_2$; `SLICES["a20"]` is 2.0, printed as 2.

```
def state_table(values):
    """A 15 x 5 array of values[state] (NaN for the states with T = 0)."""
    table = np.full((len(ROWS), len(SLICES)), np.nan)
    for i, (n, tag, _) in enumerate(ROWS):
        for j, slice_tag in enumerate(SLICES):
            sid = f"N{n}_lam{tag}_{slice_tag}"
            if sid in values:
                table[i, j] = values[sid]
    return table
```

`state_table` arranges a value per state into a table of 15 rows and 5 columns. `np.full(shape, n p.nan)` starts with every entry equal to NaN ("not a number", the computer's mark for a missing value). `enumerate` counts the rows $i = 0, 1, \ldots$ while it walks through them; `_` is a name for a part we do not need. The state name is rebuilt from the row and column, and its value is entered if it exists; the five states with $T = 0$ stay NaN.

```
def draw_table(ax, table, norm, fmt):
    """Draw a state table as a heat map with the value written in every cell."""
    cmap = plt.get_cmap("viridis").copy()
    cmap.set_bad("#d9d9d9")  # grey for NaN (the states with T = 0)
    image = ax.imshow(np.ma.masked_invalid(table), cmap=cmap, norm=norm,
                      aspect="auto")
```

`draw_table` draws such a table as a heat map. `viridis` is a colour scale from dark purple through blue and green to yellow; `.copy()` makes a private copy, and `set_bad` colours missing entries grey. `np.ma.masked_invalid` marks the NaN entries as missing; `ax.imshow` draws the table as coloured squares with the given colour scale `norm`; `aspect="auto"` lets the squares stretch to fill the panel.

```
    for i in range(table.shape[0]):
        for j in range(table.shape[1]):
            value = table[i, j]
            if np.isnan(value):
                ax.text(j, i, "T = 0", ha="center", va="center", fontsize=7)
                continue
            light = norm(value) > 0.6  # yellow-green cells: black text reads best
            ax.text(j, i, format(value, fmt), ha="center", va="center", fontsize=7,
                    color="black" if light else "white")
```

Two loops visit every square (`table.shape[0]` is the number of rows, `table.shape[1]` of columns). A missing entry is labelled `T = 0` and `continue` jumps to the next square. Otherwise the value is written in the middle of its square (`ha` and `va` are the horizontal and vertical alignment) with the format `fmt`; `norm(value)` is the position of the value on the colour scale between 0 and 1, and on the light upper part the text is black, elsewhere white.

```
    ax.set_xticks(range(len(SLICES)), [f"{a:g}" for a in SLICES.values()])
    ax.set_yticks(range(len(ROWS)),
                  [f"$N = {n}$, $\\lambda = {label}$" for n, _, label in ROWS],
                  fontsize=7)
    ax.set_xlabel("slice $a_{4,0}$")
    ax.grid(False)
    return image
```

The column marks are labelled with the slice values and the row marks with N and λ; the grid of the default settings is switched off over the heat map; the drawn image is returned, so that a colour bar can be attached to it.

```
n8_spread = [value for sid, value in spread.items() if sid.startswith("N8_")]
low8, high8 = f"{min(n8_spread):.3f}", f"{max(n8_spread):.3f}"  # three decimals
n8_text = (f"all have the spread ${low8}$" if low8 == high8  # equal when rounded
           else f"have spreads of ${low8}$ to ${high8}$")
```

The spreads of the 20 nonzero $N = 8$ states are collected; their smallest and largest are written with three decimals. If the two texts agree, the caption says that all have the same spread; otherwise it gives the range. (This is a conditional expression, `a if condition else b`.)

```
fig, ax = plt.subplots(figsize=(6.4, 6.0))
image = draw_table(ax, state_table(spread), LogNorm(0.04, 1.0), ".3f")
fig.colorbar(image, ax=ax, label="spread of $\\rho$ / max|T| (C1 needs 0)")
ax.set_title("Condition C1: dependence on the hidden direction")
```

A single panel; the spread table is drawn with a logarithmic colour scale from 0.04 to 1 and three decimals in each square; a **colour bar** beside it translates colours into values; the title names the condition.

```
save_figure(fig, "c1_map",
            "The spread $(\\max_y \\rho - \\min_y \\rho)/\\max|T|$ of the energy "
```

```
            "density along the hidden coordinate for all 75 recorded Kohn-Sham "
            "ground states: rows are the particle numbers $N = 8, 136, 688$ with "
            "the five couplings, columns the slices $a_{4,0}$ (logarithmic colour "
            "scale, pure numbers). "
            "Condition C1 needs $0$ in every cell; the smallest value is "
            f"${spread[smallest]:.4f}$ ({describe(smallest)}), and the $N = 8$ "
            f"states, made of brane zero modes, {n8_text}, almost the whole "
            "size of the tensor. Grey cells: the five states with no source at all.")
```

The caption, joined from its strings as in In [6], inserts three computed pieces: the smallest spread with four decimals (`:.4f`), 0.0497; the state written by `describe`, ($N = 136$, $\lambda = -\lambda_2$, $a_{4,0} = 2$); and the $N = 8$ text, "all have the spread 0.995". Output: figure 17a.2 and its saved line.

What figure 17a.2 shows. Rows: $N = 8, 136, 688$, each with the couplings $-\lambda_2$ to $+\lambda_2$; columns: the five slices; colour and number: the spread of ρ divided by max|T| (a pure number). C1 needs 0 in every square. The $N = 8$ rows are yellow, 0.995 everywhere: their energy density varies by almost the whole size of their tensor. The $N = 136$ and $N = 688$ rows lie between 0.05 and 0.42, largest at the first slice. The grey row, $N = 8$ with $\lambda = 0$, has no source at all. No square is close to 0.

In [9], condition C2 point by point.

```
violation = {sid: profiles[sid]["p3"] + profiles[sid]["p_t"]
             - 2.0 * profiles[sid]["p0"] for sid in nonzero}  # V(y)
size = {sid: float(np.max(np.abs(violation[sid]))) / scale[sid] for sid in nonzero}
low = min(size, key=size.get)  # the state closest to C2
high = max(size, key=size.get)  # the state farthest from C2
report("smallest max|V| / max|T|", f"{size[low]:.6g} ({low})")
report("largest max|V| / max|T|", f"{size[high]:.6g} ({high})")
```

For each nonzero state, `violation` is the array $V(y) = p_3 + p_t - 2p_0$ at the 151 points (numpy adds and multiplies arrays entry by entry), and `size` its largest absolute value divided by max|T|. Output: the smallest size is 2.09192 (`N136_lamp2_a20`) and the largest 3.99006 (`N8_lamm1_a00`).

```
detail = record_entry(SOURCE_REPORT, "ks_profiles_violate_algebraic_condition")[
    "detail"]
text = f"between {size[low]:.6g} ({low}) and {size[high]:.6g} ({high})"
reproduces(all(value > TOL for value in size.values()) and text in detail,
           "C2 fails point by point for every nonzero state",
           SOURCE_REPORT, ["ks_profiles_violate_algebraic_condition"])
```

As in In [7]: the record's text must contain our two numbers and states exactly. Output: PASS C2 fails point by point for every nonzero state, reproducing the record's check. (COMPUTED.) A violation of 2 to 4 times the largest component is not a small error: the two sides of C2 are not even of the same size.

In [10], the violation profiles (figure 3).

```
plotted = [f"N{n}_lam0_{tag}" for n in (136, 688) for tag in SLICES]
signs = {sid: np.sign(violation[sid]) for sid in plotted}  # +1, -1 (or 0)
changes = {sid: int(np.sum(signs[sid][1:] * signs[sid][:-1] < 0)) for sid in plotted}
```

`plotted` lists the ten states with $\lambda = 0$ and $N = 136$ or 688. `np.sign` gives $+1$, -1 or 0 for each entry. `signs[sid][1:]` is the array without its first entry and `signs[sid][:-1]` without its last, so their product, entry by entry, is the product of the signs of neighbouring points; it is negative exactly where V changes sign between two neighbours. `np.sum(... < 0)` counts these places (true counts as 1).

```
at_tip = [abs(float(violation[sid][0])) / scale[sid] for sid in plotted]
at_brane = [abs(float(violation[sid][-1])) / scale[sid] for sid in plotted]
say(f"sign changes of the ten plotted profiles: {sorted(set(changes.values()))}")
check(all(count == 1 for count in changes.values())
      and min(at_brane) > 0.0,
      "each plotted V(y) changes sign exactly once and is nonzero at the brane")
```

`at_tip` and `at_brane` hold $|V|/\max|T|$ at the first point ($y = -3$) and at the last ($y = 0$). `set(...)` keeps each different count once, and `sorted` puts them in order. Output: `sign changes of the ten plotted profiles: [1]`, and a PASS line: every plotted profile changes sign exactly once and is not zero at the brane.

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8), sharey=True)
for ax, n in zip(axes, (136, 688)):
    for shade, (tag, a40) in zip(SHADES, SLICES.items()):
        sid = f"N{n}_lam0_{tag}"
        ax.plot(y, violation[sid] / scale[sid], color=shade, lw=1.8,
                label=f"$a_{{4,0}} = {a40:g}$")
    ax.axhline(0.0, color="#e34948", ls="--", lw=1.2, label="C2: $V = 0$")
    ax.set_yscale("symlog", linthresh=1e-7)
    ax.set_yticks([-1e-1, -1e-3, -1e-5, -1e-7, 0.0, 1e-7, 1e-5, 1e-3, 1e-1, 1e1])
    ax.set_xlabel("hidden coordinate $y$")
    ax.set_title(f"$N = {n}$, $\\lambda = 0$")
axes[0].set_ylabel("$V = p_3 + p_t - 2p_0$, divided by max|T|")
axes[1].legend(fontsize=7, loc="upper right")
fig.tight_layout()
```

Two panels that share their vertical axis (`sharey=True`), one per particle number. In each, the five violation profiles divided by max|T| are drawn in the five shades, and a red dashed line marks $V = 0$, the value C2 demands. `set_yscale("symlog", linthresh=1e-7)` makes the vertical axis symmetric logarithmic: linear between -10^{-7} and 10^{-7}, logarithmic beyond, in both directions. `set_yticks` chooses the marks. `axes[0]` is the left panel and `axes[1]` the right one.

```
save_figure(fig, "violation_profiles",
            "The violation profiles $V(y) = p_3 + p_t - 2p_0$ of the Kohn-Sham ground "
            "states with $\\lambda = 0$, divided by the largest component of each "
            "state, for $N = 136$ (left) and $N = 688$ (right) at the five slices "
            "$a_{4,0}$ (horizontal axis: the hidden coordinate $y$; vertical axis "
            "symmetric logarithmic, linear between $-10^{-7}$ and $10^{-7}$). "
            "Condition C2 of the field equations demands the dashed line $V = 0$; "
            f"instead $|V|$ is {min(at_tip):.2f} to {max(at_tip):.2f} times max|T| "
            "at the tip, every profile changes sign exactly once, and at the brane "
            f"$|V|$/max|T| is still between ${tex_number(min(at_brane))}$ and "
            f"${tex_number(max(at_brane))}$, small but not zero.")
```

The caption, joined from its strings as in In [6], inserts the range of $|V|/\max|T|$ at the tip with two decimals, 2.36 to 2.37, and at the brane in powers of ten, 7.1×10^{-7} to 4.0×10^{-3}. Output: figure 17a.3 and its saved line.

What figure 17a.3 shows. Horizontal axis: y from the tip cutoff to the brane (units of $1/H$); vertical axis: $V/\max|T|$, a pure number, on the symmetric logarithmic scale. Every profile starts at the tip with V about 2.4 times max|T| and positive, drops steeply through zero once, near $y \approx -2.2$ (for the first slice at $y \approx -2.05$ when $N = 136$ and $y \approx -1.75$ when $N = 688$), stays negative, and rises toward zero at the brane without reaching it. C2 demands the red dashed line. Section 17.14 will explain the single change of sign: V is the slope of p_0, and p_0 has one maximum.

In [11], the map of C2 (figure 4).

```
from matplotlib.colors import Normalize  # a linear colour scale

fig, ax = plt.subplots(figsize=(6.4, 6.0))
image = draw_table(ax, state_table(size), Normalize(2.0, 4.0), ".3f")
fig.colorbar(image, ax=ax, label="max$|p_3 + p_t - 2p_0|$ / max|T| (C2 needs 0)")
ax.set_title("Condition C2: $p_3 + p_t = 2p_0$ point by point")
save_figure(fig, "c2_map",
            "The size $\\max_y|p_3 + p_t - 2p_0|/\\max|T|$ of the violation of "
            "condition C2 for all 75 recorded Kohn-Sham ground states (rows: particle "
            "number and coupling; columns: the slice $a_{4,0}$; linear colour scale, "
            "pure numbers). C2 needs $0$; every nonzero state violates it by "
            f"${size[low]:.2f}$ to ${size[high]:.2f}$ times its largest component. "
            "The $N = 8$ states, made of brane zero modes, have the same value at "
            "every slice; grey cells: no source.")
```

`Normalize` is imported first (a `from ... import` line may stand anywhere in a cell). The same heat map as In [8], now of `size` with a linear colour scale (`Normalize`) from 2 to 4, with its colour bar and title; the caption, joined from its strings as in In [6], inserts the range with two decimals, 2.09 to 3.99. Output: figure 17a.4 and its saved line.

What figure 17a.4 shows. The same rows and columns as figure 17a.2; colour and number: $\max_y |V|/\max|T|$, a pure number. C2 needs 0. The $N = 8$ rows show 3.990 in every square (the same state at every slice, In [18]); the $N = 136$ and $N = 688$ rows lie between 2.09 and 2.50. Every nonzero state violates C2 by more than twice its own largest component.

In [12], a worked example: three points of one state.

```
example = profiles["N136_lam0_a10"]
POINTS = (-3.0, -1.5, 0.0)  # tip, middle, brane
detail = record_entry(SOURCE_REPORT, "ks_profiles_violate_algebraic_condition")[
    "detail"]
rows = []
```

The state $N = 136$, $\lambda = 0$, $a_{4,0} = 1$, which the record uses as its example; the three values of y the record quotes; the record's text; and an empty list for the results.

```
for target in POINTS:
    i = int(np.argmin(np.abs(example["y"] - target)))  # nearest grid point
```

```
    values = {c: float(example[c][i]) for c in ("y",) + COLUMNS}
    values["V"] = values["p3"] + values["p_t"] - 2.0 * values["p0"]
    rows.append(values)
    text = ("y = {y:.6g}: rho = {rho:.6g}, p3 = {p3:.6g}, p_t = {p_t:.6g}, "
            "p0 = {p0:.6g}, p3 + p_t - 2 p0 = {V:.6g}").format(**values)
    say(text)
    values["in_record"] = text in detail  # the record prints the same text?
```

For each target value of y: `np.abs(example["y"] - target)` is the distance of every grid point from it, and `np.argmin` the position of the smallest distance, the nearest grid point i. `values` collects y, ρ, p_3, p_t, p_0 there (`("y",) + COLUMNS` joins two tuples) and V. `"...".format(**values)` fills the named places `{y:.6g}` and so on from the dictionary, with six significant digits as the record prints them. The line is printed, and `in_record` notes whether the record contains exactly this text.

```
reproduces(all(r["in_record"] for r in rows) and all(r["V"] != 0 for r in rows),
           "the three example points of N136_lam0_a10 equal the record",
           SOURCE_REPORT, ["ks_profiles_violate_algebraic_condition"])
```

The check passes when the record contains all three lines and $V \neq 0$ at all three points. Output:

point	ρ	p_3	p_t	p_0	$V = p_3 + p_t - 2p_0$
tip, $y = -3$	43.0702	158.268	-0	-431.733	1021.73
middle, $y = -1.5$	0.707818	0.498835	0	0.643324	-0.787813
brane, $y = -0$	0.00211576	0.000391237	-0	0.000942049	-0.00149286

(units of m^8), and the PASS line. A printed -0 is a zero stored with a minus sign; it equals 0. Check one line by hand: at the tip $V = 158.268 + 0 - 2 \times (-431.733) = 158.268 + 863.466 = 1021.734$, which rounds to the printed 1021.73. (COMPUTED; the record's check `ks_profiles_violate_algebraic_condition` prints the same three lines.)

In [13], the two sides of C2 as bars (figure 5).

```
fig, axes = plt.subplots(1, 3, figsize=(9.0, 3.4))
for ax, values, where in zip(axes, rows, ("tip", "middle", "brane")):
    sides = [values["p3"] + values["p_t"], 2.0 * values["p0"]]
    bars = ax.bar(["$p_3 + p_t$", "$2p_0$"], sides, color=["#2a78d6", "#eb6834"])
```

Three panels, one per point. In each, `sides` holds the two sides of C2, and `ax.bar` draws them as a blue and an orange bar with mathematical labels under them; `bars` keeps the two drawn bars.

```
    for bar, value in zip(bars, sides):
        ax.annotate(f"{value:.4g}", (bar.get_x() + bar.get_width() / 2, value),
                    ha="center", va="bottom" if value >= 0 else "top", fontsize=8)
    ax.axhline(0.0, color="0.3", lw=0.8)
    y_here = values["y"] + 0.0  # + 0.0 turns -0 into 0
    ax.set_title(f"{where}: $y = {y_here:g}$", fontsize=10)
    ax.margins(y=0.18)
```

`ax.annotate` writes the value of each bar, with four significant digits, at the middle of the bar's top (`bar.get_x()` is its left edge and `bar.get_width()` its width), above a positive bar and below a negative one. A thin line marks zero; the title names the point; `margins(y=0.18)` leaves 18 per cent of free space above and below, so that the numbers fit.

```
axes[0].set_ylabel("pressure (units of $m^8$)")
fig.suptitle("$N = 136$, $\\lambda = 0$, $a_{4,0} = 1$: the two sides of C2")
fig.tight_layout()
tip, middle, brane = rows
factors = [2.0 * r["p0"] / (r["p3"] + r["p_t"]) for r in (middle, brane)]
tip_left, tip_right = tip["p3"] + tip["p_t"], 2.0 * tip["p0"]  # the two sides
```

`suptitle` writes a title over the whole figure. `tip, middle, brane = rows` gives the three dictionaries names. `factors` holds $2p_0/(p_3+p_t)$ at the middle and at the brane, and `tip_left`, `tip_righ t` the two sides at the tip.

```
save_figure(fig, "three_points",
            "The two sides of condition C2, $p_3 + p_t$ (blue) and $2p_0$ (orange), "
            "of the Kohn-Sham state $N = 136$, $\\lambda = 0$, $a_{4,0} = 1$ at the "
            "tip $y = -3$, in the middle $y = -1.5$ and at the brane $y = 0$ "
            "(vertical axes: proper pressure in units of $m^8$, each panel with its "
            "own scale). C2 demands equal bars; near the tip the two sides even have "
            f"opposite signs (${tip_left:.1f}$ against "
            f"${tip_right:.1f}$), in the middle $2p_0$ is {factors[0]:.2f} "
            f"times $p_3 + p_t$ and at the brane {factors[1]:.2f} times.")
```

The caption, joined from its strings as in In [6], inserts the two sides at the tip with one decimal, 158.3 against −863.5, and the two factors with two decimals, 2.58 and 4.82. Output: figure 17a.5 and its saved line.

What figure 17a.5 shows. Three panels for the tip, the middle and the brane of the state $N = 136$, $\lambda = 0$, $a_{4,0} = 1$; vertical axes: pressure in units of m^8, each with its own scale. C2 demands two bars of equal height in each panel. At the tip the two sides do not even have the same sign; in the middle $2p_0$ is 2.58 times $p_3 + p_t$ (since $1.286648/0.498835 = 2.579$); at the brane it is 4.82 times. Exercise 2 repeats this arithmetic.

In [14], condition C2 after integration over x0.

```
def integral(sid, column):
    """The integral 2 Vol_7 int e^{6Hy} X dy of column X from the record's table."""
    return float(integrals[sid][column])
```

`integral` reads one integral of a state from the table, as a number (`float` turns the text into a number).

```
ratio = {sid: (integral(sid, "int_p3") + integral(sid, "int_p_t"))
         / (2.0 * integral(sid, "int_p0"))
         for sid in nonzero if integral(sid, "int_p0") != 0.0}
closest = min(ratio, key=lambda sid: abs(ratio[sid] - 1.0))
report("ratio closest to 1", f"{ratio[closest]:.6g} ({closest})")
report("range of the ratio", f"{min(ratio.values()):.4f} to {max(ratio.values()):.4f}")
```

`ratio` holds $R = (\int p_3 + \int p_t)/(2\int p_0)$ for every nonzero state (skipping, as a precaution, a state with $\int p_0 = 0$, which would make the division impossible; there is none). `closest` is the state whose ratio is nearest to 1: the `lambda` returns the distance from 1. Output: the ratio closest to 1 is 0.414328 (`N688_lamm2_a00`), and all ratios lie between 0.1072 and 0.4143.

```
history_text = ", ".join(f"{sid}: {ratio[sid]:.6g}" for sid in
                         ("N136_lam0_a00", "N136_lam0_a10", "N136_lam0_a20"))
say("the history N = 136, lambda = 0: " + history_text)
detail = record_entry(SOURCE_REPORT, "ks_integrals_violate_algebraic_condition")[
    "detail"]
reproduces(len(ratio) == len(nonzero)
           and all(abs(r - 1.0) > TOL for r in ratio.values())
           and f"(closest to 1: {ratio[closest]:.6g} at {closest})" in detail
           and history_text in detail,
           "C2 fails also after integration over x0, for every nonzero state",
           SOURCE_REPORT, ["ks_integrals_violate_algebraic_condition"])
```

The ratios of three states of the history are printed as the record prints them: 0.339767, 0.25969 and 0.239714 at the slices 0, 1 and 2. The check requires a ratio for every nonzero state, every ratio away from 1, and the record's text to contain our numbers. Output: PASS C2 fails also after integration over x0, for every nonzero state. (COMPUTED.) So even a source averaged over the hidden direction, a "dimensionally reduced" source, would violate C2.

In [15], the averaged ratio (figure 6).

```
N_COLOURS = {8: "#1baf7a", 136: "#2a78d6", 688: "#eb6834"}
STYLES = {"m2": ("v", ":"), "m1": ("<", "-."), "0": ("o", "-"),
          "p1": (">", "--"), "p2": ("^", (0, (1, 3)))}  # marker, line style
fig, ax = plt.subplots()
```

A colour for each particle number (green, blue, orange), and for each coupling a marker (a triangle pointing down, left, a circle, right, up) and a line style (dotted, dash-dot, solid, dashed, and `(0, (1, 3))`, which means dots of length 1 separated by gaps of 3). One panel of the default size.

```
for n, tag, label in ROWS:
    ids = [f"N{n}_lam{tag}_{s}" for s in SLICES]
    if ids[0] not in ratio:
        continue  # N = 8, lambda = 0: no source
    marker, style = STYLES[tag]
    ax.plot(list(SLICES.values()), [ratio[sid] for sid in ids], marker=marker,
            ls=style, color=N_COLOURS[n], lw=1.3, ms=4)
```

For each of the 15 rows the five state names of the series are built; the series without a source is skipped; the ratio is drawn against the slice values, with the marker and line style of the coupling and the colour of the particle number (`ms` is the marker size).

```
ax.axhline(1.0, color="#e34948", ls="--", lw=1.4)
ax.text(1.0, 0.95, "C2 after integration needs 1", color="#e34948", ha="center",
        va="top", fontsize=9)
```

```
for n, colour in N_COLOURS.items():
    ax.plot([], [], color=colour, lw=2.0, label=f"$N = {n}$")
for tag, label in TAGS:
    marker, style = STYLES[tag]
    ax.plot([], [], color="0.3", marker=marker, ls=style, label=f"$\\lambda = {label}$")
```

A red dashed line at 1 with a short text under it. The two loops draw nothing (`ax.plot([], [])` has no points) but create the entries of the legend: one per colour and one per marker and line style.

```
ax.set_xlabel("slice $a_{4,0}$")
ax.set_ylabel("$(\\int p_3 + \\int p_t) / (2\\int p_0)$")
ax.set_ylim(0.0, 1.1)
ax.legend(fontsize=7, ncol=2, loc="center right")
near = max(max(ratio[f"N{n}_lam{tag}_{s}"] for tag, _ in TAGS)
           - min(ratio[f"N{n}_lam{tag}_{s}"] for tag, _ in TAGS)
           for n in (136, 688) for s in SLICES)  # spread over the couplings
```

Axis labels, the vertical range 0 to 1.1, a legend in two columns. `near` is, over the ten combinations of $N \in \{136, 688\}$ and a slice, the largest difference between the five couplings' ratios (the inner `max(...) - min(...)` is the difference for one combination, the outer `max` the largest of the ten).

```
save_figure(fig, "integrated_ratio",
            "The ratio $(\\int p_3 + \\int p_t)/(2\\int p_0)$ of the pressures "
            "integrated over the patch with the proper-volume weight, for every "
            "nonzero recorded Kohn-Sham state, against the slice $a_{4,0}$ "
            "(colours: particle number; line styles and markers: coupling; pure "
            "numbers). A source averaged over $x_0$ would need the value $1$ (red "
            "dashed line); the states lie between "
            f"${min(ratio.values()):.3f}$ and ${max(ratio.values()):.3f}$, so even "
            "the average violates condition C2. For $N = 136$ and $N = 688$ the "
            "five couplings give almost the same ratio (they differ by at most "
            f"${near:.4f}$ at one slice), so their lines lie on top of each other.")
```

The caption, joined from its strings as in In [6], inserts the range of the ratios with three decimals, 0.107 to 0.414, and `near` with four decimals, 0.0045. Output: figure 17a.6 and its saved line.

What figure 17a.6 shows. Horizontal axis: the slice $a_{4,0}$; vertical axis: R, a pure number. A source averaged over x_0 would need the red dashed line at 1. The $N = 8$ states lie flat between 0.107 and 0.110 (the same value at every slice; the four nonzero couplings differ slightly); the $N = 136$ states fall from 0.34 to 0.24 and the $N = 688$ states from 0.41 to 0.25 along the history; the five couplings of each of these two N lie on top of each other. All points are far below 1. Section 17.15 explains why: by the integrated conservation law, $R = b_0/\bar{p}_0$ exactly, where $b_0 = [p_0(0) - e^{-6HL}p_0(-L)]/(1 - e^{-6HL})$ is the brane value of p_0 with a tip term; for $N = 136$ and 688 the tip term supplies at most a few per cent of R, for $N = 8$ about a third.

In [16], condition C3 along the history.

```
HISTORY = ["N136_lam0_a00", "N136_lam0_a10", "N136_lam0_a20",
           "N688_lam0_a00", "N688_lam0_a10", "N688_lam0_a20"]
pieces = []
```

```
for sid in HISTORY:
    r, p3, pt = (integral(sid, c) for c in ("int_rho", "int_p3", "int_p_t"))
    pieces.append(f"{sid}: int rho = {r:.6g}, int p3 = {p3:.6g}, int p_t = {pt:.6g}")
    say(pieces[-1])
```

The six states that the record tests. For each, the three integrals are read (the round brackets with `for` make a **generator**, whose three values are given the names `r`, `p3`, `pt`), written as the record writes them, stored, and printed (`pieces[-1]` is the line just added).

```
rho_constant = all(
    abs(integral(a, "int_rho") - integral(b, "int_rho"))
    <= TOL * abs(integral(a, "int_rho"))
    for a, b in (("N136_lam0_a00", "N136_lam0_a10"),
                 ("N688_lam0_a00", "N688_lam0_a10")))
p_equal = all(abs(integral(s, "int_p3") - integral(s, "int_p_t"))
              <= TOL * max(abs(integral(s, "int_p3")), 1.0) for s in HISTORY)
```

`rho_constant` is true only if the energy at slice 1 equals the energy at slice 0, within the relative tolerance, for both particle numbers; `p_equal` only if $\int p_3 = \int p_t$ for all six states (with the tolerance taken relative to $\int p_3$, or absolute when that is below 1). These are the record's own tests of C3.

```
detail = record_entry(SOURCE_REPORT, "ks_history_is_a_prescribed_background")[
    "detail"]
reproduces(not rho_constant and not p_equal and "; ".join(pieces) in detail,
           "C3 fails: along the history rho changes and p3 differs from p_t",
           SOURCE_REPORT, ["ks_history_is_a_prescribed_background"])
```

The check passes when both C3 tests fail and the record prints the same six lines, joined by semicolons. Output:

state	$\int \rho$	$\int p_3$	$\int p_t$
`N136_lam0_a00`	80.2822	23.8133	0
`N136_lam0_a10`	32.3841	10.0397	0
`N136_lam0_a20`	12.4451	4.06205	0
`N688_lam0_a00`	680.441	199.305	0
`N688_lam0_a10`	279.425	84.3387	0
`N688_lam0_a20`	110.387	35.1354	0

(units of m), and PASS C3 fails: along the history rho changes and p3 differs from p_t. (COMPUTED; record check `ks_history_is_a_prescribed_background`.)

In [17], the integrals along the history (figure 7).

```
PARTS = [("int_rho", "$\\int\\rho$", "#2a78d6", "o"),
         ("int_p3", "$\\int p_3$", "#eb6834", "s"),
         ("int_p_t", "$\\int p_t$", "#1baf7a", "^"),
         ("int_p0", "$\\int p_0$", "#4a3aa7", "D")]
fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
```

```
for ax, n in zip(axes, (136, 688)):
    ids = [f"N{n}_lam0_{s}" for s in SLICES]
    for column, label, colour, marker in PARTS:
        ax.plot(list(SLICES.values()), [integral(sid, column) for sid in ids],
                color=colour, marker=marker, lw=1.8, label=label)
    ax.set_xlabel("slice $a_{4,0}$ (the history $a_4 = Hx_4$)")
    ax.set_title(f"$N = {n}$, $\\lambda = 0$")
axes[0].set_ylabel("$2\\,\\mathrm{Vol}_7\\int e^{6Hy} X\\,dy$ (units of $m$)")
axes[1].legend(fontsize=8)
fig.tight_layout()
```

`PARTS` lists, for each of the four integrals, its column, its label, a colour and a marker (circle, square, triangle, diamond). Two panels, one per particle number; in each, the four integrals of the five states with $\lambda = 0$ are drawn against the slice.

```
drop = [integral(f"N{n}_lam0_a00", "int_rho") / integral(f"N{n}_lam0_a20", "int_rho")
        for n in (136, 688)]
```

`drop` is the factor by which the energy falls from the first to the last slice: $80.2822/12.4451 = 6.45$ for $N = 136$ and $680.441/110.387 = 6.16$ for $N = 688$.

```
save_figure(fig, "history_integrals",
            "The energy density and the three pressures of the Kohn-Sham states with "
            "$\\lambda = 0$, integrated over the patch with the proper-volume weight, "
            "at the five slices of the history $a_4 = Hx_4$ (left $N = 136$, right "
            "$N = 688$; vertical axis in units of $m$ with $H = 1$; along the "
            "history 3-space inflates as $e^{a_4}$ and the extra times deflate as "
            "$e^{-a_4}$). The linear member "
            "needs a constant $\\rho$ and equal pressures (condition C3); instead "
            f"$\\int\\rho$ falls by a factor of {drop[0]:.2f} ($N = 136$) and "
            f"{drop[1]:.2f} ($N = 688$) from $a_{{4,0}} = 0$ to $2$, $\\int p_3$ stays "
            "above $\\int p_t = 0$, and $\\int p_0$ is different again.")
```

The caption, joined from its strings as in In [6], inserts both factors with two decimals. In the f-string the subscript is written `a_{{4,0}}`, because inside an f-string a single brace would start a computed piece; the doubled braces print as one pair. Output: figure 17a.7 and its saved line.

What figure 17a.7 shows. Horizontal axis: the slice $a_{4,0}$ of the history $a_4 = Hx_4$ (3-space inflates as e^{a_4}, the extra times deflate as e^{-a_4}); vertical axis: the integrals over the patch in units of m. C3 needs a horizontal blue line ($\int \rho$ constant) and the three pressure lines on top of each other. Instead $\int \rho$ falls by a factor of about 6 over the history, $\int p_3$ (orange) lies well above $\int p_t = 0$ (green, on the axis: without interaction the extra times carry no pressure), and $\int p_0$ (purple) is different again. Section 17.16 shows that the fall of the energy is exactly the work done by the unbalanced pressure, $\int p_3 > \int p_t$.

In [18], the states N = 8.

```
same, p3_equals_pt, p0_differs = True, True, True
for tag in ("m2", "m1", "p1", "p2"):
    first = profiles[f"N8_lam{tag}_a00"]
```

```
    for slice_tag in SLICES:
        prof = profiles[f"N8_lam{tag}_{slice_tag}"]
        same = same and all(np.array_equal(prof[c], first[c]) for c in COLUMNS)
    p3_equals_pt = p3_equals_pt and np.array_equal(first["p3"], first["p_t"])
    p0_differs = p0_differs and float(np.max(np.abs(first["p0"] - first["p3"]))) > 0
```

Three flags start true. For each nonzero coupling of $N = 8$, the profiles of all five slices are compared with the first, column by column and bit for bit (`np.array_equal`); then the cell tests $p_3 = p_t$ exactly and $p_0 \neq p_3$ somewhere. The `and` keeps a flag true only if it stays true for every case.

```
int_p3_n8 = integral("N8_lamp1_a00", "int_p3")  # N = 8, lambda = +lambda_1
int_p0_n8 = integral("N8_lamp1_a00", "int_p0")
say(f"N = 8, lambda = +lambda_1: int p3 = {int_p3_n8:.6g}, int p0 = {int_p0_n8:.6g}")
check(same and p3_equals_pt and p0_differs,
      "N = 8: the same state at every slice, p3 = p_t, but p0 differs from p3")
```

Two integrals of one $N = 8$ state are printed as an example: $\int p_3 = -0.000977659$ and $\int p_0 = -0.00889138$, so p_0 is not p_3. Output: that line and a PASS line. Why these states do not change: the eight zero modes have zero 3-momentum, and the slice enters the Kohn-Sham equations only through the redshifted momentum $ke^{-a_{4,0}}$ (the exact rescaling identity of Chapter 14), so for $k = 0$ nothing changes. They satisfy the part "constant ρ and $p_3 = p_t$" of C3, but they fail C1 and C2 (In [7] and In [9]) and have $p_0 \neq p_3$.

In [19], the states with no source at all.

```
kappa, rho, p0, Lam = sp.symbols("kappa rho p0 Lam", real=True)
NAMES.update({"kappa": kappa, "rho": rho, "p0": p0, "Lam": Lam})


def equation(text):
    """An equation (left == right) of the record as the expression left - right."""
    left, right = text.split("==")
    return parse(left) - parse(right)
```

Four more real symbols (κ, ρ, p_0, Λ) are made and added to the names that `parse` knows (`update` adds pairs to a dictionary). The record writes an equation as `left == right`; `equation` cuts the text at `==` and returns left minus right, which is zero exactly when the equation holds.

```
einstein = equations["einstein"]
x4_equation = equation(einstein["constraint_x4"]["input"]).subs({rho: 0})
x0_equation = equation(einstein["hidden_x0"]["input"]).subs({p0: 0})
difference = sp.expand(x4_equation - x0_equation)  # Lambda cancels
say(f"x4 equation minus x0 equation with rho = p0 = 0: {difference} = 0")
say(f"is the left side positive for real a4' and H > 0? {difference.is_positive}")
```

The record's Einstein constraint `3*ad1^2 + 21*H^2 + Lam == -(kappa*rho)` and hidden equation `-3*ad1^2 + 15*H^2 + Lam == kappa*p0` become expressions with $\rho = 0$ and $p_0 = 0$; their difference, multiplied out, is the derivation of Section 17.6. `.is_positive` asks sympy whether the expression is positive for all allowed values of its symbols (a_4' real, H positive); sympy answers `True`. Output: `6*H**2 + 6*ad1**2 = 0` and `True`.

```
reproduces(difference == 6 * ad1 ** 2 + 6 * H ** 2 and difference.is_positive is True,
           "Einstein gravity: no source at all (T = 0) has no solution for H > 0",
           WL_A4, ["einstein_no_vacuum_solution"])
detail = record_entry(SOURCE_REPORT, "ks_zero_source_states_listed")["detail"]
reproduces(", ".join(zero) in detail and f"{len(zero)} states" in detail,
           "the five states with T = 0 are those listed by the record",
           SOURCE_REPORT, ["ks_zero_source_states_listed"])
```

The first check confirms the difference and its positivity (PROVED; Wolfram report, check `einstein_no_vacuum_solution`). The second confirms that the record lists exactly our five states with $T = 0$: `N8_lam0_a00`, `N8_lam0_a05`, `N8_lam0_a10`, `N8_lam0_a15`, `N8_lam0_a20`. These eight zero modes have zero energy and, without interaction, no source at all; by the first check, "no source" is not admissible either. Output: two PASS lines with their records.

In [20], the last check.

```
names = ["rho_profiles", "c1_map", "violation_profiles", "c2_map", "three_points",
         "integrated_ratio", "history_integrals"]
paths = [output_file(f"{FIGURE_FOLDER}/17a_{k}_{name}.png")
         for k, name in enumerate(names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

The seven figure names; `enumerate(names, 1)` counts them from 1, so the paths are `17a_1_rho_profiles.png` to `17a_7_history_integrals.png` in the folder where the notebook wrote them. The check confirms that every file exists, and `all_checks_passed()` prints the last line. Output: PASS every figure file of this notebook exists and ALL 16 CHECKS PASSED (notebook 17a).

The count. The 16 PASS lines are: one in In [2], three in In [4], one each in In [5], In [6], In [7], In [9], In [10], In [12], In [14], In [16], In [18], two in In [19], and one in In [20]. Ten of them reproduce a Revision record and name it.

17.12 Why the states fail: every source must be conserved

Notebook 17a shows THAT the recorded Kohn-Sham ground states fail all three conditions. We now ask WHY. The answer comes from one of the deepest facts about the field equations of gravity.

The Bianchi identity. For every metric, the left-hand side of the field equations has zero covariant divergence:

$$\nabla_\mu {E_{(k)}}^\mu{}_\nu = 0 \quad (k = 1, 2, 3, \text{ every } \nu), \qquad \nabla_\mu(\Lambda\, \delta^\mu_\nu) = 0.$$

The first statement is a theorem about the Lovelock tensors; for the author's metric with an arbitrary $a_4(x_4)$ it is PROVED in the record (checks `E1_divergence_free`, `E2_divergence_free`, `E3_divergence_free` of both the Wolfram and the sympy report). The second holds because Λ is a constant and the covariant derivative of δ^μ_ν is zero (the table δ^μ_ν, 1 on the diagonal and 0 elsewhere, is the same in every coordinate system).

The consequence. Take the covariant divergence of both sides of the field equations:

$$\sum_k \alpha_k \nabla_\mu E_{(k)}{}^\mu{}_\nu + \nabla_\mu(\Lambda\,\delta^\mu_\nu) = \kappa\,\nabla_\mu T^\mu{}_\nu$$

(rule: the same operation on both sides of an equation keeps it true; the divergence of a sum is the sum of the divergences, and a constant factor comes out)

$$0 = \kappa\,\nabla_\mu T^\mu{}_\nu$$

(rule: the Bianchi identity makes every term on the left zero). Since $\kappa \neq 0$: **every source of the field equations is conserved**, $\nabla_\mu T^\mu{}_\nu = 0$. Conservation is NECESSARY.

What the Kohn-Sham states conserve. A field that obeys its own TIME-DEPENDENT field equation in a given metric has a conserved energy-momentum tensor (Section 9.9: it is the Noether identity, which follows from the invariance of the action under a change of coordinates). The Kohn-Sham states are not such solutions: each one solves the instantaneous problem of one slice (the stationary-slice ansatz of Chapter 14), so the Noether argument does not apply to them, and the record proves exactly this much:

- along the hidden direction they are conserved at every point (the y law of Section 17.13, for every self-consistent state: `Revision/kohn_sham/reports/ks-theory-python.json`, check `emt_y_conservation_selfconsistent`; the Rust solver checks it on its grid of points and integrated over the patch: `Revision/kohn_sham/reports/ks-rust-solver.json`, checks `emt_y_conservation_pointwise` and `emt_y_conservation_integrated`);
- in the time direction only the TOTAL energy of the patch obeys the energy-change law (the same theory report, check `emt_x4_component`, which states the law for the energy of the 7-volume; the solver compares the derivative of that energy with respect to a_4, computed by finite differences in a_4, with the law: check `emt_energy_change_dE_da4`). Point by point the time law fails: the instantaneous states carry no energy flux along the hidden direction (their mixed component vanishes), so nothing moves energy from one value of y to another, and the energy at a given y changes along the history differently from what the pressures there demand (Section 17.16 shows the numbers).

So conservation is not SUFFICIENT. Even a source that obeyed every conservation law could fail the field equations, and the Kohn-Sham states, which obey the hidden law everywhere and the time law for their total energy, do fail. The next four sections show precisely what the field equations of the author's metric demand beyond conservation, and why the Kohn-Sham states cannot give it.

17.13 The conservation law in the coordinate y

The covariant divergence. For any metric and any table $T^\mu{}_\nu$,

$$\nabla_\mu T^\mu{}_\nu = \sum_\mu \partial_\mu T^\mu{}_\nu + \sum_{\mu,\lambda} \Gamma^\mu{}_{\mu\lambda}\,T^\lambda{}_\nu - \sum_{\mu,\lambda} \Gamma^\lambda{}_{\mu\nu}\,T^\mu{}_\lambda,$$

with the Christoffel symbols

$$\Gamma^{\lambda}{}_{\mu\nu} = \frac{1}{2}\sum_{\sigma} g^{\lambda\sigma}(\partial_\mu g_{\sigma\nu} + \partial_\nu g_{\sigma\mu} - \partial_\sigma g_{\mu\nu}),$$

where ∂_μ is the partial derivative with respect to the coordinate x_μ and $g^{\lambda\sigma}$ is the inverse metric (the summation index is called σ here, because κ is the strength of gravity). The first sum is the ordinary "change plus outflow"; the two others correct for the stretching and turning of the coordinate directions.

A lemma for a diagonal metric. Write the diagonal entries as $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ with signs $\eta_{\mu\mu} = \pm 1$ and positive scale factors f_μ. For a diagonal metric the inverse is diagonal, $g^{\lambda\lambda} = 1/g_{\lambda\lambda}$, so only $\sigma = \lambda$ survives in the sum:

$$\Gamma^{\lambda}{}_{\mu\nu} = \frac{1}{2g_{\lambda\lambda}}(\partial_\mu g_{\lambda\nu} + \partial_\nu g_{\lambda\mu} - \partial_\lambda g_{\mu\nu}) \quad \text{(no sum over } \lambda\text{)}.$$

Now set $\lambda = \mu$:

$$\Gamma^{\mu}{}_{\mu\nu} = \frac{1}{2g_{\mu\mu}}(\partial_\mu g_{\mu\nu} + \partial_\nu g_{\mu\mu} - \partial_\mu g_{\mu\nu})$$

(rule: replace λ by μ everywhere)

$$= \frac{\partial_\nu g_{\mu\mu}}{2g_{\mu\mu}}$$

(rule: the first and the third term in the bracket are equal and cancel)

$$= \frac{\eta_{\mu\mu}\, 2 f_\mu\, \partial_\nu f_\mu}{2\,\eta_{\mu\mu} f_\mu^2} = \frac{\partial_\nu f_\mu}{f_\mu} = \partial_\nu \ln f_\mu$$

(rule: $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ and the chain rule $\partial_\nu(f_\mu^2) = 2 f_\mu\, \partial_\nu f_\mu$; cancel $2\eta_{\mu\mu} f_\mu$; the derivative of $\ln f$ is f'/f).

The divergence of a diagonal tensor. (Section 9.9 derived this formula for every diagonal metric and applied it in the coordinate x_0; here we apply it in the coordinate y, and we repeat the short derivation so that this section can be read on its own.) Let $T^{\mu}{}_{\nu}$ be diagonal ($T^{\mu}{}_{\nu} = 0$ for $\mu \neq \nu$). In the three sums of the divergence only the terms with a diagonal entry survive: in the first only $\mu = \nu$; in the second only $\lambda = \nu$; in the third only $\lambda = \mu$. So

$$\nabla_\mu T^{\mu}{}_{\nu} = \partial_\nu T^{\nu}{}_{\nu} + \sum_{\mu} \Gamma^{\mu}{}_{\mu\nu}\, T^{\nu}{}_{\nu} - \sum_{\mu} \Gamma^{\mu}{}_{\mu\nu}\, T^{\mu}{}_{\mu}$$

(rule: drop the zero terms; no sum over ν)

$$= \partial_\nu T^{\nu}{}_{\nu} + \sum_{\mu} (\partial_\nu \ln f_\mu)(T^{\nu}{}_{\nu} - T^{\mu}{}_{\mu})$$

(rule: the lemma, and the two sums combined into one).

The author's metric in the coordinate y. In the coordinate y the metric is (Chapter 14)

$$ds^2 = e^{2Hy}\big[e^{2a_4}(dx_1^2+dx_2^2+dx_3^2) - e^{-2a_4}(dx_5^2+dx_6^2+dx_7^2)\big] - dx_4^2 + dy^2,$$

so the scale factors are $f_i = e^{a_4+Hy}$ for 3-space, $f_4 = 1$, $f_t = e^{-a_4+Hy}$ for the extra times and $f_y = 1$. Their logarithms are $\ln f_i = a_4 + Hy$, $\ln f_t = -a_4 + Hy$, $\ln f_4 = \ln f_y = 0$, and their derivatives:

$$\partial_4 \ln f_i = a_4', \quad \partial_4 \ln f_t = -a_4', \quad \partial_y \ln f_i = \partial_y \ln f_t = H, \quad \text{all others } 0.$$

Take a diagonal source $T = \mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0)$ whose entries depend on x_4 and y.

The component $\nu = y$. Here $T^y{}_y = p_0$. The three 3-space directions contribute $H(p_0 - p_3)$ each, the three extra times $H(p_0 - p_t)$ each, and x_4 and y contribute nothing (their weight $\partial_y \ln f$ is zero):

$$\nabla_\mu T^\mu{}_y = \partial_y p_0 + 3H(p_0 - p_3) + 3H(p_0 - p_t)$$

(rule: the formula above with $\nu = y$)

$$= p_0' + 6Hp_0 - 3H(p_3 + p_t)$$

(rule: multiply out and collect; the prime on p_0 is $\partial/\partial y$).

The component $\nu = x_4$. Here $T^{x_4}{}_{x_4} = -\rho$:

$$\nabla_\mu T^\mu{}_{x_4} = \partial_4(-\rho) + 3a_4'(-\rho - p_3) + 3(-a_4')(-\rho - p_t)$$

(rule: the formula with $\nu = x_4$; 3-space has weight a_4' and entry p_3, the extra times weight $-a_4'$ and entry p_t)

$$= -\partial_4\rho - 3a_4'\rho - 3a_4'p_3 + 3a_4'\rho + 3a_4'p_t = -\partial_4\rho - 3a_4'(p_3 - p_t)$$

(rule: multiply out; $-3a_4'\rho + 3a_4'\rho = 0$; factor out $-3a_4'$).

The other six components contain only derivatives with respect to $x_1, x_2, x_3, x_5, x_6, x_7$, of which nothing depends, and weights $\partial_{x_1} \ln f = 0$; they vanish identically.

The conservation law. A diagonal source in the author's metric is conserved exactly when

$$\partial_4\rho = -3a_4'\,(p_3 - p_t) \qquad \text{and} \qquad p_0' + 6Hp_0 = 3H(p_3 + p_t).$$

(PROVED: `Revision/kohn_sham/reports/ks-theory-python.json`, checks `emt_y_conservation_selfconsistent` and `emt_x4_component`; recorded in `Revision/kohn_sham/ks-theory.json`, keys `emt.conservationY` and `emt.energyChange`; derived again with sympy in Notebook 17b, In [9].)

The same in the author's coordinate x_0. The record of the a_4 equations writes the law in the coordinate x_0, for the general source with mixed entries (key `generalSource`, equations `conservation_x4` and `conservation_x0`):

$$\nabla_\mu T^\mu{}_{x_4} = -\partial_4\rho - 3a_4'(p_3 - p_t) + \partial_0 q_{04} - 6H\tan z\, q_{04},$$
$$\nabla_\mu T^\mu{}_{x_0} = \partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t) + \partial_4 q_{40}$$

(PROVED: Wolfram and sympy reports, check `conservation_components`; sympy report, check `json_conservation`; derived again in Notebook 17b, In [7] and In [8]). With $q_{40} = q_{04} = 0$ the two forms agree. By the chain rule of Section 17.4, $\partial_0 = (dy/dx_0)\,\partial_y = \cot z\,\partial_y$, so

$$\partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t) = \cot z\,[p_0' + 6Hp_0 - 3H(p_3 + p_t)]$$

(rule: write $\partial_0 p_0 = \cot z\, p_0'$ and take out the common factor $\cot z$; $3H \cdot 2p_0 = 6Hp_0$). Since $\cot z > 0$ on the patch, the x_0 law and the y law say the same.

17.14 For a conserved source, C2 says that p0 is flat

The key step. Take a conserved diagonal source and solve the y law for $p_3 + p_t$:

$$p_3 + p_t = \frac{p_0' + 6Hp_0}{3H}$$

(rule: divide both sides of $p_0' + 6Hp_0 = 3H(p_3 + p_t)$ by $3H$, which is not zero). Insert this into the violation of C2:

$$V = p_3 + p_t - 2p_0$$

(rule: the definition of V, Section 17.5)

$$= \frac{p_0' + 6Hp_0}{3H} - 2p_0$$

(rule: replace $p_3 + p_t$ by its value from the conservation law)

$$= \frac{p_0'}{3H} + 2p_0 - 2p_0 = \frac{p_0'}{3H}$$

(rule: split the fraction, $6Hp_0/(3H) = 2p_0$, and cancel). So, for EVERY conserved diagonal source,

$$p_3 + p_t - 2p_0 = \frac{1}{3H}\,\frac{\partial p_0}{\partial y}.$$

(PROVED: Notebook 17b, In [10], with sympy, from the conservation law of In [9].)

What it means.

- For a conserved source, C2 holds at a point exactly when p_0 is flat there ($\partial p_0/\partial y = 0$). C2 is not an extra demand on top of conservation: it is the p_0 part of C1. A conserved source that meets C1 meets C2 automatically.
- For the Kohn-Sham states the violation of C2 is, point by point, the slope of their hidden pressure divided by $3H$. Their p_0 changes by several powers of ten along y (figure 2 of Notebook 17b: the largest $|p_0|$ is 2933 to 2.3×10^6 times the smallest, for $N = 136$, $\lambda = 0$), so V cannot be small.
- The single change of sign of V in figure 17a.3 is now clear: V is zero exactly where p_0 has its maximum. The Kohn-Sham p_0 is negative near the tip, rises steeply, has one maximum (near $y \approx -2.2$ for most slices), and then falls toward the brane; so $V > 0$ on the tip side of the maximum and $V < 0$ on the brane side.
- The identity is COMPUTED on all 70 nonzero states with fourth-order finite differences of the stored profiles: $|p_0'/(3H) - V|$ is at most 1.05×10^{-4} of max|T|, and the difference falls like the fourth power of the step, as an error of the differences must (Notebook 17b, In [11] and In [13]).

A conserved profile that is not flat. Exercise 6 shows that, when $p_3 + p_t = P$ is a constant, every conserved p_0 has the form $p_0 = P/2 + c\,e^{-6Hy}$ with a constant c, and only $c = 0$ satisfies C2. Conservation allows many profiles; the field equations of the author's metric allow only the flat one.

17.15 The integrated law and the averaged condition

The integrated law. Multiply the y law by e^{6Hy}:

$$e^{6Hy}p_0' + 6He^{6Hy}p_0 = 3H\,e^{6Hy}(p_3 + p_t)$$

(rule: multiply both sides by the positive number e^{6Hy})

$$\frac{d}{dy}(e^{6Hy}p_0) = 3H\,e^{6Hy}(p_3 + p_t)$$

(rule: the product rule read backwards, $(e^{6Hy}p_0)' = e^{6Hy}p_0' + 6He^{6Hy}p_0$). Integrate from the tip cutoff $y = -L$ to the brane $y = 0$:

$$e^0\,p_0(0) - e^{-6HL}\,p_0(-L) = 3H\int_{-L}^{0} e^{6Hy}(p_3 + p_t)\,dy$$

(rule: the fundamental theorem of calculus, the integral of a derivative is the difference of the values at the ends). Multiply by $2\,\mathrm{Vol}_7$ and use the notation $\int X$ of Section 17.7:

$$2\,\mathrm{Vol}_7[p_0(0) - e^{-6HL}p_0(-L)] = 3H\Big(\int p_3 + \int p_t\Big).$$

(PROVED from the y law. COMPUTED on the 70 nonzero states with the brane values, tip values and integrals of the record's table: the two sides agree to a relative difference of at most

1.421×10^{-11}; the solver's own relative difference of the same two sides, column `ycons_integrated_rel` (computed from its unrounded numbers and divided by the largest of the terms involved), is at most 1.422×10^{-11}, the value of the solver check `emt_y_conservation_integrated`; Notebook 17b, In [14].)

Why the averaged ratio is not 1. The averaged form of C2 asks for $R = (\int p_3 + \int p_t)/(2\int p_0) = 1$. By the definition of the weighted mean (Section 17.7),

$$\int p_0 = 2\,\mathrm{Vol}_7\,\bar{p}_0\,\frac{1 - e^{-6HL}}{6H}.$$

Divide the integrated law by $2\int p_0$:

$$R = \frac{\int p_3 + \int p_t}{2\int p_0} = \frac{2\,\mathrm{Vol}_7\,[p_0(0) - e^{-6HL}p_0(-L)]/(3H)}{2 \cdot 2\,\mathrm{Vol}_7\,\bar{p}_0\,(1 - e^{-6HL})/(6H)}$$

(rule: the numerator from the integrated law divided by $3H$; the denominator from the line above)

$$= \frac{p_0(0) - e^{-6HL}\,p_0(-L)}{(1 - e^{-6HL})\,\bar{p}_0}$$

(rule: cancel $2\,\mathrm{Vol}_7$; $\frac{1/(3H)}{2/(6H)} = \frac{6H}{6H} = 1$). We call the numerator, divided by $1 - e^{-6HL}$, the **boundary value** of p_0,

$$b_0 = \frac{p_0(0) - e^{-6HL}\,p_0(-L)}{1 - e^{-6HL}}, \qquad R = \frac{b_0}{\bar{p}_0} \quad \text{exactly.}$$

The averaged C2 asks that the boundary value of the hidden pressure equal its weighted mean. That is true for a flat p_0 and false for the Kohn-Sham states, for two different reasons.

- For the states with 3-momentum ($N = 136$ and $N = 688$) the weight e^{6Hy} puts most of the mean in the last stretch before the brane (at $y = -1$ the weight is already $e^{-6} \approx 0.0025$): Notebook 17b, In [15], finds that the last unit, $-1 \le y \le 0$, supplies between 0.812 and 0.932 of the integral of $e^{6Hy}p_0$ for $N = 136$ and between 0.816 and 0.969 for $N = 688$. There their p_0 falls toward the brane, so its boundary value lies below its mean.
- For the $N = 8$ brane zero modes the size of p_0 falls almost like e^{-6Hy} over most of the patch (the shape of the term $c\,e^{-6Hy}$ in Exercise 6), so the product $e^{6Hy}p_0$ is nearly flat there and the mean comes from the whole patch: the last unit supplies only 0.439 to 0.441 of the integral (In [15]). A nearly flat $e^{6Hy}p_0$ makes the mean much larger than the brane value. To see the size, suppose $e^{6Hy}p_0$ were exactly a constant c; then $p_0(0) = c$, while $\bar{p}_0 = cL/[(1 - e^{-6HL})/(6H)] \approx 6HL\,c = 18\,c$ (rule: the integral of the constant c from $-L$ to 0 is cL; then divide by the volume factor of Section 17.7, use $1 - e^{-6HL} \approx 1$, and $H = 1$, $L = 3$). The brane value alone would then give the ratio $1/18 \approx 0.056$; the recorded states give 0.074 (next paragraph). For these states the tip term is not small either: at the tip $e^{6Hy}p_0$ is not tiny compared with its brane value (it has the opposite sign there), so the term $-e^{-6HL}p_0(-L)$ supplies about a third of b_0 (next paragraph). This is the difference from the states with 3-momentum, whose $e^{6Hy}p_0$ is concentrated near the brane.

Is the tip term small? The factor $e^{-6HL} = e^{-18} \approx 1.5 \times 10^{-8}$ is tiny, but near the tip $|p_0|$ is enormous, so the answer depends on the state. Notebook 17b, In [15], measures the share $1 - p_0(0)/b_0$ of R that the tip term supplies: from 4.1×10^{-5} to 0.039 for $N = 136$, from 5.3×10^{-6} to 0.025 for $N = 688$, but 0.31 to 0.33 for the 20 nonzero $N = 8$ states. So $R \approx p_0(\text{brane})/\bar{p}_0$ is a fair description of the states with 3-momentum and a poor one for the brane zero modes: for them the brane value alone would give ratios from 0.074 (the caption of figure 17b.4), and the tip term raises them to the values $R = 0.107$ to 0.110 of figure 17a.6. The exact formula reproduces every ratio of Notebook 17a to a relative difference of 1.4×10^{-11} (Notebook 17b, In [15]); Exercise 5 computes one by hand.

17.16 The time direction: energy exchange and the propagation of the constraint

The energy-change law. For a source that does not depend on x_0, the x_4 law of Section 17.13 reads

$$\rho' = -3a_4' \, (p_3 - p_t)$$

(the prime is d/dx_4). In words: while 3-space inflates ($a_4' > 0$), a 3-space pressure p_3 takes energy out of the source, and while the extra times deflate, an extra-time pressure p_t puts energy in. The volume of the seven directions other than the time is constant (the factors e^{3a_4} and e^{-3a_4} cancel), so, unlike a gas in an ordinary expanding universe, there is no term with ρ itself: only the difference of the two pressures moves the energy. This is the first law of thermodynamics, "change of energy equals minus pressure times change of volume", applied to the two families of directions (an interpretation of the PROVED identity; Section 9.10 works it out).

For the Kohn-Sham gas the record states only the integrated version (`Revision/kohn_sham/ks-theory.json`, key `emt.energyChange`): the energy $E = \int \rho$ of the state changes as

$$\frac{dE}{dx_4} = -3a_4' \Big(\int p_3 - \int p_t \Big).$$

Along the history $a_4 = AHx_4 + a_0$ the slice is $a_{4,0} = a_4(x_4)$, so by the chain rule

$$\frac{dE}{dx_4} = \frac{dE}{da_{4,0}} \cdot \frac{da_4}{dx_4} = AH \, \frac{dE}{da_{4,0}}$$

(rule: the chain rule; $da_4/dx_4 = AH$), and dividing the two expressions for dE/dx_4 by $AH \neq 0$,

$$\frac{dE}{da_{4,0}} = -3 \Big(\int p_3 - \int p_t \Big).$$

(Solver check `emt_energy_change_dE_da4` of the report named in Section 17.12: the derivative that the solver computes from extra self-consistent states at $a_4 \pm \delta$ and $a_4 \pm 2\delta$ around each slice, with $\delta = 0.002$ and fixed occupations, agrees with the right-hand side in all 75 cases; the difference, divided by the larger of the sum of the absolute orbital energies and m, is at most 1.505×10^{-10}.) For $\lambda = 0$, $\int p_t = 0$ and $\int p_3 > 0$, so the energy must FALL along the history. For $N = 136$ at the first slice the rate is $-3 \times 23.8133 = -71.44$ (units of m per unit of $a_{4,0}$). Over the whole history the

energy falls from 80.2822 to 12.4451; Simpson's rule applied to the five recorded rates reproduces the change -67.8372 to a relative error of 2.1×10^{-4}, and for all ten series that change, to at most 2.2×10^{-4} (COMPUTED; Notebook 17b, In [18]; Exercise 7).

Point by point the time law fails. Notebook 17b, In [20], tests $\partial\rho/\partial a_4 = -3(p_3 - p_t)$ at every grid point of the slice $a_{4,0} = 1$, with a fourth-order difference in a_4 over the five slices. For $N = 136$, $\lambda = 0$ it finds at the tip $\partial\rho/\partial a_4 = 82.97$ against $-3(p_3 - p_t) = -474.8$ (opposite signs: the energy density near the tip GROWS along the history while its pressure says it should fall), in the middle ($y = -1.5$) -0.4505 against -1.497, and at the brane -0.002068 against -0.001174. In all ten series with 3-momentum the ratio of the two sides at the tip is negative (-0.327 to -0.106), while the two sides integrated over the patch with the weight e^{6Hy} agree to 1.3×10^{-3}, the accuracy of a difference with step 0.5 (COMPUTED). The reason is the one given in Section 17.12: an instantaneous state has no energy flux q_{04} along y, so the energy of each layer of the patch cannot be balanced by flow into its neighbours; only the total is balanced. A genuinely time-dependent state of the gas would need such a flux, $q_{04} \neq 0$, and C1 forbids it: a further reason why the history is only a prescribed background.

The constraint propagates. Write the constraint and the evolution equation of Section 17.3 as two expressions that must vanish:

$$\mathcal{C} = e_{44}(a_4') + \Lambda + \kappa\rho, \qquad \mathcal{E} = a_4''\,F(a_4') - \kappa(p_3 - p_t).$$

First a fact about e_{44}: its derivative with respect to $a' = a_4'$ is

$$\frac{de_{44}}{da'} = \alpha_1 \cdot 6a' + \alpha_2(-144a'^3 - 240a'H^2) + \alpha_3(2160a'^5 + 2592a'^3H^2 + 2160a'H^4)$$

(rule: differentiate each term of ${E_{(k)}}^{x_4}{}_{x_4}$ of Section 17.3; the derivative of a'^n is na'^{n-1})

$$= 6a'\big[\alpha_1 - \alpha_2(24a'^2 + 40H^2) + \alpha_3(360a'^4 + 432a'^2H^2 + 360H^4)\big] = 6a'\mathcal{Q} = 3a'F$$

(rule: take out the common factor $6a'$; the bracket is $\mathcal{Q}$ of Section 17.3, and $F = 2\mathcal{Q}$). Now differentiate $\mathcal{C}$ along the time, for a source that depends on x_4 only:

$$\frac{d\mathcal{C}}{dx_4} = \frac{de_{44}}{da'}\,a_4'' + \kappa\rho'$$

(rule: the chain rule for $e_{44}(a_4'(x_4))$; Λ is constant)

$$= 3a_4'F\,a_4'' - 3\kappa a_4'(p_3 - p_t)$$

(rule: the fact above, and the energy-change law $\rho' = -3a_4'(p_3 - p_t)$)

$$= 3a_4'\big[a_4''F - \kappa(p_3 - p_t)\big] = 3a_4'\,\mathcal{E}$$

(rule: take out the common factor $3a_4'$). (PROVED: Wolfram report, check `constraint_propagation_bianchi`; sympy report, check `bianchi_x4`; reproduced for all couplings in Notebook 17b, In [17].)

What it means. If the evolution equation holds at all times ($\mathcal{E} = 0$), a constraint that holds at one time holds at all times: this is how the Bianchi identity keeps the field equations consistent. Along the linear member ($a_4'' = 0$, $a_4' = AH$) it gives

$$\frac{d\mathcal{C}}{dx_4} = -3AH\kappa\,(p_3 - p_t).$$

A conserved source with $p_3 \neq p_t$ therefore makes the constraint drift: even if it held at one instant, it would fail at the next. The constraint of the linear member can hold at all times only if $p_3 = p_t$; then $\rho' = 0$ and, with C2, $p_0 = p_3$. For a conserved source along the linear member, C3 therefore reduces to the single demand $p_3 = p_t$: the constant ρ then follows from conservation, and $p_0 = p_3$ from C2. The Kohn-Sham gas at $\lambda = 0$ has $p_t = 0$ and $\int p_3 > 0$: its energy falls along the history, which the linear member forbids.

Summary of Sections 17.12 to 17.16. For a conserved diagonal source in the author's metric:

condition	what it says in general	what it says for a conserved source	the Kohn-Sham states
C1	no component depends on x_0	ρ, p_3, p_t flat; p_0 flat is C2	fail: profiles vary by powers of ten
C2	$p_3 + p_t = 2p_0$	p_0 flat	fail: $V = p_0'/(3H)$ is large
C3	$p_3 = p_t = p_0$, ρ constant	$p_3 = p_t$ (then ρ constant, $p_0 = p_3$)	fail: $\int p_3 > \int p_t$, E falls by a factor of about 6

The Kohn-Sham source is conserved along the hidden direction at every point (y law), and in the time direction only for the total energy of the patch (dE/dx_4 law); it is NOT ADMISSIBLE.

17.17 The equations averaged over the hidden direction: what can and cannot be integrated

Sections 17.4 to 17.16 show that the field equations of the author's metric cannot hold, point by point, with any recorded Kohn-Sham ground state as the source. The Revision record `Revision/field_equations_a4/ks_source/` (script `ks_source_a4.py`, report `reports/ks-source-a4.json` with 23 checks, all PASS) asks the next questions. How far is the source from one that does not depend on x_0? Which weaker, averaged equations can it satisfy? And what do those averaged equations say about $a_4(x_4)$, in particular about the exponential deflation of the extra times? This section derives the answers; Notebook 17b, In [21] to In [23], reproduces the numbers it needs.

The x_0 identity once more. The record proves for every Lovelock order k that ${E_{(k)}}^{x_1}{}_{x_1} + {E_{(k)}}^{x_5}{}_{x_5} - 2{E_{(k)}}^{x_0}{}_{x_0} = 0$ (check `A1_lovelock_identity_x1_plus_x5_equals_2x0`; it is the identity behind C2 in Section 17.5), and that the y law of Section 17.13 gives, for a source that obeys it, $p_3 + p_t - 2p_0 = p_0'(y)/(3H)$ exactly (check `A6_x0_conservation_in_y`; it is Section 17.14). The record then evaluates the y law on all 70 nonzero profiles with fourth-order differences and finds it satisfied to 6.63404×10^{-5} of the largest term (check `B2_x0_conservation_on_every_profile`). This refines the wording of the first record `ks-source-conditions.json` (check `B0_wave1_report_verified`): the recorded states do NOT violate the conservation identity of the hidden direction;

what they violate is $p_3 + p_t = 2p_0$, which is that identity combined with the independence of x_0 (PROVED and COMPUTED, as stated).

How far from an x_0-independent source? The projection mismatch. Among all functions that do not depend on y, which one is closest to a profile $X(y)$? Measure the distance with the volume weight, $D(c) = \int_{-L}^{0} e^{6Hy}\,(X-c)^2\,dy$ for a constant c. Its derivative with respect to c is

$$\frac{dD}{dc} = -2\int_{-L}^{0} e^{6Hy}\,(X-c)\,dy$$

(rule: differentiate under the integral sign; the derivative of $(X-c)^2$ with respect to c is $-2(X-c)$), and it vanishes when

$$\int_{-L}^{0} e^{6Hy} X\,dy = c\int_{-L}^{0} e^{6Hy}\,dy, \qquad c = \frac{\int_{-L}^{0} e^{6Hy} X\,dy}{\int_{-L}^{0} e^{6Hy}\,dy} = \bar{X}$$

(rule: split the integral, take the constant c out, divide by the positive number $\int e^{6Hy}dy$). So the closest flat profile is the weighted mean $\bar{X}$ of Section 17.7; it is called the **orthogonal projection** of X on the flat functions. The record measures the remaining part in two ways: $\mu_1 = \int e^{6Hy}|X-\bar{X}|\,dy / \int e^{6Hy}|X|\,dy$ (the **L1 measure**) and $\mu = \sqrt{D(\bar{X})/D(0)}$ (the **L2 measure**); both are 0 for a flat profile. Over the 70 nonzero states the energy density has μ_1 between 0.583068 and 0.763572 and μ between 0.754082 and 1 (check `B4_hidden_direction_mismatch`; the record prefers the L1 measure, because the L2 measure is dominated by the large values near the tip and so depends more on the tip cutoff). At least about 58 per cent of the energy density deviates from its average: the source is not close to an x_0-independent one (COMPUTED).

The moment equations. Take any one of the field equations of Section 17.3, say the constraint $e_{44} + \Lambda = -\kappa\rho$, multiply it by e^{6Hy} and integrate over the patch:

$$\int_{-L}^{0} e^{6Hy}\,(e_{44}+\Lambda)\,dy = -\kappa\int_{-L}^{0} e^{6Hy}\rho\,dy$$

(rule: the same operation on both sides keeps an equation true)

$$(e_{44}+\Lambda)\,W = -\kappa\,\bar{\rho}\,W, \qquad W = \int_{-L}^{0} e^{6Hy}\,dy = \frac{1-e^{-6HL}}{6H}$$

(rule: the left side does not depend on y and comes out of the integral; on the right, the integral is $W\bar{\rho}$ by the definition of the weighted mean)

$$e_{44} + \Lambda = -\kappa\,\bar{\rho}$$

(rule: divide by $W > 0$). Each field equation gives in this way its **moment**: the same equation with every component of the source replaced by its weighted mean. Every exact solution would satisfy all of them, so they are NECESSARY conditions, weaker than the field equations themselves (PROVED: it is one line of algebra). The combination $x_1 + x_5 - 2x_0$ of the moments reads $0 = \kappa(\bar{p}_3 + \bar{p}_t - 2\bar{p}_0)$, the averaged C2. The record measures its failure by the relative defect $|\bar{p}_3 + \bar{p}_t - 2\bar{p}_0| / (|\bar{p}_3| + |\bar{p}_t| + 2|\bar{p}_0|)$,

which lies between 0.414086 (`N688_lamm2_a00`) and 0.806379 over the 70 nonzero states (check `C1_averaged_algebraic_condition_fails`; reproduced in Notebook 17b, In [21]). No choice of $\kappa \neq 0$ satisfies all the moments: even after averaging, the Kohn-Sham source is not admissible (COMPUTED).

Which moments can hold together. Write $P(X) = \sum_k \alpha_k E_{(k)}{}^{x_4}{}_{x_4}$ evaluated at $a_4'^2 = X$ (the x_4 components contain a_4' only through its square, Section 17.3). In Einstein gravity $P(X) = 3X + 21H^2$. The record proves $3F(a_4') = 2\,dP/dX$ at $X = a_4'^2$ for all couplings (check `A3_F_equals_two_thirds_dP_dX`; in Einstein gravity $3 \times 2 = 2 \times 3$), the fact $de_{44}/da' = 3a'F$ of Section 17.16 in another form. The averaged source obeys the averaged energy relation $d\bar{\rho}/da_4 = -3(\bar{p}_3 - \bar{p}_t)$, because the time law holds for the total energy of the patch (check `C2_averaged_energy_relation`: the solver's derivative agrees with it to 1.53537×10^{-10}, Simpson's rule over the slices to 2.70425×10^{-4}, and there are no Fermi-level crossings along the history). Then the derivation of Section 17.16 goes through with the averages:

$$\frac{d}{dx_4}\Big[P(a_4'^2) + \Lambda + \kappa\bar{\rho}(a_4)\Big] = 3a_4'\Big[a_4''F(a_4') - \kappa(\bar{p}_3 - \bar{p}_t)\Big]$$

(PROVED: check `A4_constraint_propagation_with_averaged_source`). So the x_4 moment (the constraint) and the $x_1 - x_5$ moment (the evolution equation) are consistent with each other. The x_0 moment is not. In Einstein gravity the x_4 and x_0 moments read

$$3a_4'^2 + 21H^2 + \Lambda = -\kappa\bar{\rho}, \qquad 15H^2 - 3a_4'^2 + \Lambda = \kappa\bar{p}_0;$$

adding them gives

$$36H^2 + 2\Lambda = \kappa(\bar{p}_0 - \bar{\rho})$$

(rule: add the two equations side by side and move $-\kappa\bar{\rho}$ to the right; $3a_4'^2 - 3a_4'^2 = 0$, $21H^2 + 15H^2 = 36H^2$). The left side is a constant, so $\bar{p}_0 - \bar{\rho}$ would have to stay constant along the history. In the ten series with 3-momentum it changes by 0.909317 to 0.915045 of its largest value (check `C3_x0_moment_inconsistent_with_history`): every set of moments that contains the x_0 moment is inconsistent for them (COMPUTED). For the $N = 8$ states the averages do not change along the history, but their averaged C2 still fails.

The approximation, stated. The record therefore keeps the consistent pair, the x_4 moment and the $x_1 - x_5$ moment, and drops the x_0 moment and the $x_1 + x_5 - 2x_0$ moment. This is an APPROXIMATION (ASSUMED), and its error is not small: the dropped averaged C2 fails by the defects 0.41 to 0.81 quoted above, and the dropped x_0 moment, evaluated along the solved histories, fails by 0.109607 to 1.87301 of the largest term of that equation (check `D7_dropped_x0_moment_residual_and_adiabaticity`). Everything below holds only within this approximation, for the zero-temperature instantaneous states at the computed slices $0 \leq a_4 \leq 2$.

The first integral. For $a_4' \neq 0$ the pair is equivalent to the single first-order equation $P(a_4'^2) + \Lambda = -\kappa\bar{\rho}(a_4)$ (the bracket of the identity above is constant, and the constraint makes the constant zero). In Einstein gravity, with the initial rate $a_4'(0) = H$ (the rate of the history on which the states were computed) and the **source strength** $\sigma_0 = \kappa\bar{\rho}(0)/H^2$, the constraint at $a_4 = 0$ fixes Λ:

$$3H^2 + 21H^2 + \Lambda = -\sigma_0 H^2 \quad \Longrightarrow \quad \Lambda = -24H^2 - \sigma_0 H^2$$

(rule: insert $a_4' = H$ and $\kappa\bar{\rho}(0) = \sigma_0 H^2$; move the numbers to the right). Insert this Λ into the constraint at a later a_4:

$$3a_4'^2 + 21H^2 - 24H^2 - \sigma_0 H^2 = -\kappa\bar{\rho}(a_4) = -\sigma_0 H^2 \, \frac{\bar{\rho}(a_4)}{\bar{\rho}(0)}$$

(rule: $\kappa = \sigma_0 H^2/\bar{\rho}(0)$ by the definition of σ_0)

$$\Big(\frac{a_4'}{H}\Big)^2 = 1 + \frac{\sigma_0}{3}\Big(1 - \frac{\bar{\rho}(a_4)}{\bar{\rho}(0)}\Big)$$

(rule: move $-3H^2 - \sigma_0 H^2$ to the right, divide by $3H^2$). (PROVED; the record reproduces it at every slice reached by every Einstein case, that is up to the turning point where a case halts, to 1.6946×10^{-15}, check `D2_einstein_first_integral`.) The gas changes $a_4'^2$ only through the fraction $1 - \bar{\rho}(a_4)/\bar{\rho}(0)$ of its averaged energy density that it has lost. Over the computed range the record finds that $a_4'^2$ changes by at most $0.281661\,|\sigma_0|H^2$ (check `D5_einstein_sign_of_the_effect`; among the four integrated series with 3-momentum listed below, the largest change belongs to $N = 136$, $\lambda = 0$, whose lost fraction at $a_4 = 2$ is therefore $3 \times 0.281661 = 0.845$).

What can be integrated, and what it gives. The record integrates the pair for three theories of gravity (Einstein, $\alpha_1 = 1$; Einstein-Gauss-Bonnet, $\alpha_2 H^2 = 1/80$; a cubic Lovelock case, $\alpha_2 H^2 = 1/80$, $\alpha_3 H^4 = 1/4000$), five series of states (four of the ten series with 3-momentum, $N = 136$, $\lambda = 0$ and $N = 688$, $\lambda = 0, \pm\lambda_2$, and the series $N = 8$, $+\lambda_2$; the other six series with 3-momentum, $N = 136$ with $\lambda = \pm\lambda_1, \pm\lambda_2$ and $N = 688$ with $\lambda = \pm\lambda_1$, are not integrated; record table `results/ks-source-a4-cases.csv`) and seven parameter choices ($\sigma_0 = \pm 10, \pm 1, \pm 1/10$, and $\Lambda = 0$), by two methods: the first integral, and a fourth-order Runge-Kutta integration of the evolution equation with a separately interpolated source. The two agree to 5.49953×10^{-4} (check `D1_two_integration_methods_agree`). Between the slices $\bar{\rho}$ is taken from the cubic Hermite interpolant with the exact slopes $-3(\bar{p}_3 - \bar{p}_t)$. The results (COMPUTED, within the approximation):

- For $N = 136$, $\lambda = 0$ in Einstein gravity, $a_4'(2)/a_4'(0) = 1.95362$ for $\sigma_0 = 10$, 1.1321 for $\sigma_0 = 1$ and 0.847549 for $\sigma_0 = -1$. A positive $\kappa\bar{\rho}$ speeds the deflation of the extra times up; a negative one slows it down (Notebook 17b, In [22], reproduces these three numbers from the first integral; figure 17b.6).
- With $\Lambda = 0$ the constraint at $a_4 = 0$ needs $\sigma_0 = -24$ in Einstein gravity ($\sigma_0 < 0$, that is $\kappa\bar{\rho}(0) < 0$, in all three theories; for the four integrated series with 3-momentum $\bar{\rho}(0) > 0$, so this forces $\kappa < 0$, record table, column `kappa`), and for these four series the deflation HALTS: for $N = 136$, $\lambda = 0$, a_4' reaches 0 at the turning point $a_4 = 0.149623$ (Einstein), 0.0724197 (Einstein-Gauss-Bonnet), 0.102772 (cubic), and the three integrated series with $N = 688$ halt likewise, at 0.151681 to 0.151697 (Einstein), 0.0733735 to 0.0733808 (Einstein-Gauss-Bonnet) and 0.104149 to 0.104159 (cubic) (check `D3_lambda_zero_cases_halt`; Notebook 17b, In [22], finds the Einstein value for $N = 136$, and 0.397998 for $\sigma_0 = -10$, by bisection). The integration stops at the turning point; what follows it is not computed (OPEN).

- In Einstein-Gauss-Bonnet gravity $F = 1 - \frac{3}{5}a_4'^2/H^2$ vanishes at $a_4'^2 = \frac{5}{3}H^2$ (check `D0_gravity_branches`; in Einstein gravity $F = 2$ and in the cubic case $F > 0$ never vanish). For $\sigma_0 = 1$ and $\sigma_0 = 10$ the first integral drives $a_4'^2$ up to this **branch point** inside the computed range, where the evolution equation cannot be solved for a_4'' (8 cases: the four integrated series with 3-momentum, two strengths each): for $N = 136$, $\lambda = 0$ at $a_4 = 0.249607$ ($\sigma_0 = 1$) and 0.0226924 ($\sigma_0 = 10$) (check `D6_branch_points_only_for_egb`). For $\sigma_0 = 1/10$ these four series stay regular up to $a_4 = 2$, where a_4'/H is between 1.07629 and 1.07705, and the $N = 8$ series keeps $a_4' = H$ for every strength (record table, columns `event` and `rate_end`).
- For the $N = 8$ states the averages do not change with a_4 and $\bar{p}_3 = \bar{p}_t$, so the evolution equation gives $a_4'' = 0$: exactly the linear member $a_4 = A_0 H x_4 + a_0$, with the rate A_0 chosen initially and Λ absorbing $\kappa\bar{\rho}$ (check `D4_constant_source_gives_linear_member`); their averaged C2 still fails.

What cannot be integrated, or was not computed. The full coupled problem, the author's metric with the Kohn-Sham source, has no solution at all (C1 fails), so nothing about a_4 is DERIVED from it (PROVED, Sections 17.4 to 17.14). The x_0 moment cannot be kept (shown above). Not computed (OPEN): the source beyond $a_4 = 2$, where no Kohn-Sham states exist (the record states only a CONDITIONAL: if $\bar{\rho}$ tended to 0 there, Einstein gravity would give $a_4'^2 \to (-\Lambda - 21H^2)/3$, a late-time exponential deflation at a rate set by Λ and H alone); the evolution after a turning point; the six series with 3-momentum that are not integrated; thermal states (only the ground states, at zero temperature, are used); the time-dependent, non-adiabatic problem; and a source with the profile the equations require, or a metric with x_0-dependent warp functions that could carry the Kohn-Sham profile.

So, does the dirac16complex source drive the exponential deflation of the extra times? Exactly: the question has no solution to examine, because no recorded Kohn-Sham ground state is an admissible source. Within the stated approximation: no. The equations are unchanged under $x_4 \to -x_4$, because F and P contain a_4' only through even powers (check `A3_F_equals_two_thirds_dP_dX`); that reversal turns deflation of the extra times into inflation, so the sign of the rate and its size are initial data, and Λ is fixed by them through the constraint. The gas only modifies a deflation that is already there, by a bounded amount set by the energy it loses; with $\Lambda = 0$ it even halts it at a turning point, where the integration stops (what follows is not computed). Exponential deflation at a constant rate is allowed (the $N = 8$ states give exactly the linear member) but not selected. This is the answer of the record `Revision/field_equations_a4/ks_source/README.md`, reproduced here step by step.

17.18 Example: the conservation laws, the averaged equations, and why the states fail

The second notebook makes Sections 17.12 to 17.17 concrete. It first checks that the Kohn-Sham source rests on the author's eight real 16×16 gamma matrices, the very file that the Rust solver read; then it derives with sympy the covariant divergence of a general source in the author's metric, in the coordinate x_0 and in the coordinate y, and compares it with the two Revision records; it proves $V = p_0'/(3H)$ and $d\mathcal{C}/dx_4 = 3a_4'\mathcal{E}$; and it tests the identities on the 75 recorded ground states, point by point with a convergence study, integrated over the patch, and along the deflating history, where it also shows that the time law fails point by point; finally it computes the hidden-direction averages, the defect of the averaged C2 and the first integral of the averaged equations of Einstein

gravity, and reproduces the record's rates and turning points. It prints 20 PASS lines and draws six figures. It runs in about 20 seconds on a typical laptop and needs no Rust.

17.19 How to run Notebook 17b

Step 1. What this notebook does and what it needs.

Notebook 17b (The conservation laws and the source conditions: why the Kohn-Sham states fail) is the file `Revision/textbook/notebooks/17b_conserved_sources.ipynb` of the repository Dirac_claude. It checks that the Kohn-Sham source of chapter 17 rests on the eight real 16 x 16 gamma matrices of the author (the very file the Rust solver read), derives with sympy the conservation law of a source in the author's metric, in the coordinate x0 and in the hidden coordinate y, and compares it with the Revision records. It proves that for a conserved source the violation p3 + p_t - 2 p0 of condition C2 equals the slope of p0 along y divided by 3H, and that the time derivative of the constraint is 3 a4' times the evolution equation, so that the linear member needs p3 = p_t. It then tests these identities on the 75 recorded Kohn-Sham ground states: point by point with fourth-order differences and a convergence study, integrated with the brane and tip values, and along the deflating history with Simpson's rule. The Kohn-Sham states obey the conservation law of the hidden direction at every point and the energy law of the time direction for the total energy of the patch (not point by point), and still fail the source conditions. Finally it averages the source over the hidden direction and integrates the averaged a4 equations of Einstein gravity with their first integral, an approximation whose numbers it checks against the Revision record. Six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 17b_conserved_sources.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 17b_conserved_sources.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 17b_conserved_sources.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/17b.captions.json`
- `Revision/textbook/figures/17b_1_eight_gammas.png`
- `Revision/textbook/figures/17b_2_slope_identity.png`
- `Revision/textbook/figures/17b_3_difference_order.png`
- `Revision/textbook/figures/17b_4_brane_and_mean.png`
- `Revision/textbook/figures/17b_5_energy_exchange.png`
- `Revision/textbook/figures/17b_6_first_integral.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 17b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/17b_conserved_sources.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 17b_conserved_sources.ipynb
python -m nbconvert --execute --inplace 17b_conserved_sources.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming gammas.json, a report or a profile file: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.
- the check of the sha256 fingerprint of gammas.json fails: the file was changed, for example by an editor or a tool that converted its line endings. Restore it with the command below, run in the repository folder.

```
git checkout HEAD Revision/algebra/gammas.json
```

17.20 Notebook 17b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/17b_conserved_sources.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Notebook 17a showed that no recorded Kohn-Sham ground state of dirac16complex is an
admissible source of the author's metric: the states fail the three source
conditions C1, C2, C3 of the $a_4$ equations. This notebook asks WHY, and the
answer is the **conservation law** of energy and momentum. Every source of the
field equations must be conserved; the Kohn-Sham states obey the conservation law
of the hidden direction at every point, and the energy law of the time direction
only for the total energy of the patch; and for a conserved source the conditions
take a very simple form. The notebook

- checks that the Kohn-Sham source is built on the author's eight REAL
  $16 \times 16$ gamma matrices (the very file that the Rust solver read);
- derives with sympy the conservation law of a source in the author's metric, in
  the coordinate $x_0$ and in the hidden coordinate $y$, and compares it with the
  Revision records;
- PROVES that for a conserved source the violation of C2 is the slope of $p_0$:
  $p_3 + p_t - 2p_0 = p_0'(y)/(3H)$; so C2 holds exactly where $p_0$ is flat;
- PROVES that the time derivative of the constraint equation is $3a_4'$ times the
  evolution equation, so that the linear member $a_4 = AHx_4$ needs $p_3 = p_t$;
- tests these identities on the 75 recorded Kohn-Sham ground states: point by
  point (with a convergence study of the differences), integrated over the
  hidden direction, and along the deflating history, where only the energy of the
  whole patch obeys the time law;
- averages the source over the hidden direction, shows that the averaged C2 still
  fails, and integrates the averaged $a_4$ equations of Einstein gravity with
```

```
  their first integral (an APPROXIMATION, as the Revision record states);
- draws six teaching plots. It takes about twenty seconds.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 17b (The conservation laws and the source conditions: why the Kohn-Sham states
fail) is the file ‘Revision/textbook/notebooks/17b_conserved_sources.ipynb‘ of the
repository Dirac_claude. It checks that the Kohn-Sham source of chapter 17 rests on the
eight real 16 x 16 gamma matrices of the author (the very file the Rust solver read),
derives with sympy the conservation law of a source in the author’s metric, in the
coordinate x0 and in the hidden coordinate y, and compares it with the Revision records.
It proves that for a conserved source the violation p3 + p_t - 2 p0 of condition C2
equals the slope of p0 along y divided by 3H, and that the time derivative of the
constraint is 3 a4’ times the evolution equation, so that the linear member needs p3 =
p_t. It then tests these identities on the 75 recorded Kohn-Sham ground states: point by
point with fourth-order differences and a convergence study, integrated with the brane
and tip values, and along the deflating history with Simpson’s rule. The Kohn-Sham states
obey the conservation law of the hidden direction at every point and the energy law of
the time direction for the total energy of the patch (not point by point), and still fail
the source conditions. Finally it averages the source over the hidden direction and
integrates the averaged a4 equations of Einstein gravity with their first integral, an
approximation whose numbers it checks against the Revision record. Six teaching plots. It
needs a computer with Windows 11, macOS or Linux, an internet connection for the
installation, the program Git, and Python 3.12 or newer (the notebooks were built with
Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0,
mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2,
ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and
matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It
does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 17b_conserved_sources.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 17b_conserved_sources.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 17b_conserved_sources.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/17b.captions.json‘
- ‘Revision/textbook/figures/17b_1_eight_gammas.png‘
- ‘Revision/textbook/figures/17b_2_slope_identity.png‘
- ‘Revision/textbook/figures/17b_3_difference_order.png‘
- ‘Revision/textbook/figures/17b_4_brane_and_mean.png‘
- ‘Revision/textbook/figures/17b_5_energy_exchange.png‘
- ‘Revision/textbook/figures/17b_6_first_integral.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS every figure file of this notebook exists
    ALL 20 CHECKS PASSED (notebook 17b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/17b_conserved_sources.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 17b_conserved_sources.ipynb
  python -m nbconvert --execute --inplace 17b_conserved_sources.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming gammas.json, a report or a profile file: the notebook reads the Revision records of the repository. Run it inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone, not on a copy of the notebook file alone.
- the check of the sha256 fingerprint of gammas.json fails: the file was changed, for example by an editor or a tool that converted its line endings. Restore it with the command below, run in the repository folder.

```
    git checkout HEAD Revision/algebra/gammas.json
```

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 17b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 17b (The conservation laws and the source conditions: why the Kohn-Sham states
# fail) is the file Revision/textbook/notebooks/17b_conserved_sources.ipynb of the
# repository Dirac_claude. It checks that the Kohn-Sham source of chapter 17 rests on the
# eight real 16 x 16 gamma matrices of the author (the very file the Rust solver read),
# derives with sympy the conservation law of a source in the author's metric, in the
# coordinate x0 and in the hidden coordinate y, and compares it with the Revision
# records. It proves that for a conserved source the violation p3 + p_t - 2 p0 of
# condition C2 equals the slope of p0 along y divided by 3H, and that the time derivative
# of the constraint is 3 a4' times the evolution equation, so that the linear member
# needs p3 = p_t. It then tests these identities on the 75 recorded Kohn-Sham ground
# states: point by point with fourth-order differences and a convergence study,
# integrated with the brane and tip values, and along the deflating history with
# Simpson's rule. The Kohn-Sham states obey the conservation law of the hidden direction
# at every point and the energy law of the time direction for the total energy of the
# patch (not point by point), and still fail the source conditions. Finally it averages
# the source over the hidden direction and integrates the averaged a4 equations of
# Einstein gravity with their first integral, an approximation whose numbers it checks
# against the Revision record. Six teaching plots. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
```

```
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 17b_conserved_sources.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
```

```
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 17b_conserved_sources.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 17b_conserved_sources.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/17b.captions.json
# - Revision/textbook/figures/17b_1_eight_gammas.png
# - Revision/textbook/figures/17b_2_slope_identity.png
# - Revision/textbook/figures/17b_3_difference_order.png
# - Revision/textbook/figures/17b_4_brane_and_mean.png
# - Revision/textbook/figures/17b_5_energy_exchange.png
# - Revision/textbook/figures/17b_6_first_integral.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 20 CHECKS PASSED (notebook 17b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
```

```
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/17b_conserved_sources.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 17b_conserved_sources.ipynb
#     python -m nbconvert --execute --inplace 17b_conserved_sources.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming gammas.json, a report or a profile file: the notebook
#   reads the Revision records of the repository. Run it inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone,
#   not on a copy of the notebook file alone.
# - the check of the sha256 fingerprint of gammas.json fails: the file was changed, for
#   example by an editor or a tool that converted its line endings. Restore it with the
#   command below, run in the repository folder.
#     git checkout HEAD Revision/algebra/gammas.json
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "17b"  # this notebook: chapter 17, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
```

```
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 17b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ = ordinary 3-space,
  which inflates (scale factor $e^{a_4}\sin^{1/6}z$); $x_4$ = the time;
  $x_5, x_6, x_7$ = the three **extra times**, which **deflate exponentially**
  (scale factor $e^{-a_4}\sin^{1/6}z$, with $a_4$ increasing); $x_0$ = the hidden
  space direction, $z = 6Hx_0$ between $0$ and $\pi/2$.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: another way to number the points
  of the hidden direction; $y = 0$ is the **brane** ($z = \pi/2$), $y = -3$ the
  **tip cutoff** of the computations. A **chart** is a choice of coordinates.
- **Gamma matrices** $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$: eight square tables
  of numbers with $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\,1$ (the
  **Clifford relation**), $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$. The
  author's are real $16 \times 16$ **signed permutation matrices**: each row and
  each column holds exactly one nonzero entry, $+1$ or $-1$.
- **sha256 fingerprint**: a 64-digit code computed from the bytes of a file; two
  files with the same fingerprint are, for all practical purposes, identical.
- **Energy-momentum tensor** $T^\mu{}_\nu$ (the **source**): here
  $T = \mathrm{diag}(p_3, p_3, p_3, -\rho, p_t, p_t, p_t, p_0)$ in the order
  $x_0, \dots, x_7$, with energy density $\rho$ and pressures $p_3$ (3-space),
  $p_t$ (extra times), $p_0$ (hidden direction); possibly mixed entries $q_{40}$
  (row $x_4$, column $x_0$) and $q_{04}$.
- **Christoffel symbols** $\Gamma^\lambda{}_{\mu\nu} = \tfrac12 g^{\lambda\kappa}
  (\partial_\mu g_{\kappa\nu} + \partial_\nu g_{\kappa\mu} -
  \partial_\kappa g_{\mu\nu})$: numbers built from the first derivatives of the
  metric that say how the coordinate directions turn from point to point.
- **Covariant divergence** $\nabla_\mu T^\mu{}_\nu = \partial_\mu T^\mu{}_\nu +
  \Gamma^\mu{}_{\mu\lambda}T^\lambda{}_\nu - \Gamma^\lambda{}_{\mu\nu}
  T^\mu{}_\lambda$ (sum over repeated indices): the curved-space version of "what
  flows out of a small box". The **conservation law** is
  $\nabla_\mu T^\mu{}_\nu = 0$ for every $\nu$.
- **Bianchi identity**: the left-hand sides of the Einstein-Lovelock equations
  have zero covariant divergence for EVERY metric; hence every source of the
  equations must be conserved.
- **Source conditions** (from the $a_4$ equations): **C1** no component of the
  source depends on $x_0$ and $q_{40} = q_{04} = 0$; **C2** $p_3 + p_t = 2p_0$;
  **C3** for the linear member $a_4 = AHx_4 + a_0$: $p_3 = p_t = p_0$ and a
  constant $\rho$. **Admissible source**: one that meets them.
- **Constraint** (the $x_4$ equation, no $a_4''$) and **evolution equation**
  ($a_4''\,F(a_4') = \kappa(p_3 - p_t)$) of the $a_4$ equations.
- **Finite difference** of fourth order: the slope of a tabulated function from
  five neighbouring values,
```

```
  $f'(y) \approx [-f(y+2h) + 8f(y+h) - 8f(y-h) + f(y-2h)]/(12h)$; its error falls
  like $h^4$ when the step $h$ shrinks (**order of accuracy** 4).
- **Simpson's rule**: the integral of a function over $[a, a+2h]$ is about
  $(h/3)[f(a) + 4f(a+h) + f(a+2h)]$; error like $h^4$.
- **Weighted mean**: $\int e^{6Hy}p_0\,dy / \int e^{6Hy}dy$, the average of
  $p_0$ over the proper volume.
- **Kohn-Sham state**, **slice** $a_{4,0}$, **history** $a_4 = AHx_4$ with
  $A = 1$, **prescribed background**, **test field**, **back-reaction**: as in
  Notebook 17a; a state name such as `N136_lamp2_a20` means $N = 136$,
  $\lambda = +\lambda_2$, $a_{4,0} = 2.0$.
- **Least squares**: the straight line through a set of points that makes the sum
  of the squared vertical distances of the points from the line as small as
  possible.
- **Weighted average over the hidden direction** $\bar X$ (also called the
  **moment** of $X$): the integral of $e^{6Hy}X$ over the patch divided by the
  integral of $e^{6Hy}$. A **moment equation** is a field equation multiplied by
  $e^{6Hy}$ and integrated over the patch.
- **First integral**: an equation with only first derivatives that every solution
  of a second-order equation obeys; here $a_4'^2$ as a function of $a_4$.
- **Cubic Hermite interpolation**: between two tabulated points, the cubic
  polynomial with the given values AND the given slopes at both ends.
- **Turning point**: the value of $a_4$ where $a_4'$ reaches $0$; there the
  deflation of the extra times halts.
- **Status labels**: PROVED (exact), COMPUTED (numerical, with its measured
  error), ASSUMED (including a stated approximation), OPEN. Units: $H = 1$,
  $m = 1$.
```

Text cell:

```
## 4. The physical and mathematical situation

The field equations of gravity, $\sum_k\alpha_kE_{(k)}{}^\mu{}_\nu +
\Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu$, have a left-hand side whose covariant
divergence vanishes identically (the Bianchi identity; the Revision record checks
it for all three Lovelock tensors of the author's metric). Taking the divergence
of both sides gives $\kappa\nabla_\mu T^\mu{}_\nu = 0$: a source must be
conserved. Conservation is therefore NECESSARY. This notebook shows that it is not
SUFFICIENT, and that it explains the failure of the Kohn-Sham states.

The Kohn-Sham states are computed in the prescribed background $a_4 = Hx_4$ (the
extra times deflate as $e^{-a_4}$, 3-space inflates as $e^{a_4}$). A field that
solves its own TIME-DEPENDENT field equation in a given metric has a conserved
energy-momentum tensor. The Kohn-Sham states are not such solutions: each one
solves the instantaneous problem of one slice (the stationary-slice ansatz). So
the Revision record proves less, and exactly this: the conservation law of the
hidden direction for every self-consistent state, at every point; and the
energy-change law of the time direction only INTEGRATED over the hidden
direction, for the instantaneous states followed with fixed occupations (they
carry no energy flux along $y$). The Rust solver checks both numerically. We will
see:

1. in the hidden direction, conservation reads
   $p_0' + 6Hp_0 = 3H(p_3 + p_t)$ (the prime is $d/dy$); hence the violation of
```

```
   C2 is $V = p_3 + p_t - 2p_0 = p_0'/(3H)$. For a conserved source without mixed
   entries, C2 and "$p_0$ does not depend on $x_0$" (part of C1) are the SAME
   condition. The Kohn-Sham $p_0$ changes by orders of magnitude along $y$, so C2
   must fail;
2. in the time direction, conservation reads $\rho' = -3a_4'(p_3 - p_t)$ (the
   prime is $d/dx_4$): energy flows between the source and the expanding and
   deflating directions unless $p_3 = p_t$. The Kohn-Sham gas has
   $p_3 > 0 = p_t$ (at $\lambda = 0$), so its energy must change along the
   history, which C3 forbids. For the Kohn-Sham states this law holds for the
   energy of the whole patch but fails point by point (section 12).
```

Text cell:

```
## 5. The Revision records and the helpers that read them

The next cell imports the packages and names the Revision records used below. It
defines four helpers: ‘read_json‘ reads a JSON file of the repository,
‘record_entry‘ finds a check by its name in a Revision report, ‘reproduces‘ is
a check that passes only when this notebook's own result holds AND the named
checks of the report have the verdict PASS, and ‘tex_number‘ writes a number in
powers of ten for the captions. Then it counts the checks of the seven reports and
requires that all of them passed.
```

In [2]:

```
import csv  # reads tables stored as CSV files (comma-separated values)
import hashlib  # computes the sha256 fingerprint of a file

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols

GAMMAS = "Revision/algebra/gammas.json"  # the eight real 16 x 16 gammas
ALGEBRA = "Revision/algebra/reports/python-algebra.json"  # their checks
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
PY_A4 = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy
WL_A4 = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram
SOURCE_REPORT = "Revision/field_equations_a4/reports/ks-source-conditions.json"
KS_SOURCE = "Revision/field_equations_a4/ks_source/reports/ks-source-a4.json"
KS_THEORY = "Revision/kohn_sham/ks-theory.json"  # the Kohn-Sham theory record
KS_PY = "Revision/kohn_sham/reports/ks-theory-python.json"  # its sympy checks
KS_RUST = "Revision/kohn_sham/reports/ks-rust-solver.json"  # the solver checks
GROUND = "Revision/kohn_sham/results/ground"  # the 75 Kohn-Sham ground states


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_entry(report_file, name):
    """The entry {name, verdict, detail} of a check of a report, None if absent."""
    for entry in read_json(report_file)["checks"]:
```

```
        if entry["name"] == name:
            return entry
    return None


def reproduces(condition, name, report_file, record_names):
    """A check that also requires the named checks of the report to be PASS."""
    found = all((record_entry(report_file, n) or {}).get("verdict", "").upper()
                == "PASS" for n in record_names)  # some reports write "pass"
    names_text = ", ".join(record_names)  # the check names, separated by commas
    check(condition and found, name, record=f"{report_file}, check {names_text}")


def tex_number(value, digits=2):
    """A number for a caption in powers of ten: 0.0123 -> 1.2 \\times 10^{-2}."""
    mantissa, exponent = f"{value:.{digits - 1}e}".split("e")
    return f"{mantissa} \\times 10^{{{int(exponent)}}}"


REPORTS = [ALGEBRA, PY_A4, WL_A4, SOURCE_REPORT, KS_SOURCE, KS_PY, KS_RUST]
every_pass = True
for report_file in REPORTS:
    verdicts = [entry["verdict"].upper()
                for entry in read_json(report_file)["checks"]]
    passed = verdicts.count("PASS")  # how many checks of this report passed
    every_pass = every_pass and passed == len(verdicts)
    report(report_file.split("/")[-1], f"{passed} of {len(verdicts)} checks PASS")
check(every_pass, "every check of the seven Revision reports used here is PASS")
```

Out [2]:

```
RESULT python-algebra.json = 35 of 35 checks PASS
RESULT python-a4-report.json = 63 of 63 checks PASS
RESULT wolfram-a4-report.json = 64 of 64 checks PASS
RESULT ks-source-conditions.json = 6 of 6 checks PASS
RESULT ks-source-a4.json = 23 of 23 checks PASS
RESULT ks-theory-python.json = 58 of 58 checks PASS
RESULT ks-rust-solver.json = 42 of 42 checks PASS
PASS every check of the seven Revision reports used here is PASS
```

Text cell:

```
## 6. The eight real $16 \times 16$ gamma matrices behind the Kohn-Sham source

dirac16complex is a 16-component spinor field, and its field equation, its
Lagrangian and its energy-momentum tensor are written with the author's eight
real $16 \times 16$ gamma matrices. The Kohn-Sham solver reduces the 16-component
equation to $2 \times 2$ blocks; the solver's own check `block_reduction_numeric`
confirms that its blocks are exactly the 16-component Hamiltonian written in
another basis. The next cell reads the matrices from the Revision record
`gammas.json` and checks: there are eight; each is $16 \times 16$; every entry is
a JSON integer (so the matrices are REAL) equal to $-1$, $0$ or $+1$; each is a
signed permutation matrix.
```

In [3]:

```
gamma_record = read_json(GAMMAS)
entries = [v for matrix in gamma_record["gamma"] for row in matrix for v in row]
real_integers = all(isinstance(v, int) for v in entries)  # no complex entries
gamma = [np.array(matrix, dtype=float) for matrix in gamma_record["gamma"]]
eta = np.array(gamma_record["eta"], dtype=float)  # +1 or -1 for x0, ..., x7
shapes = sorted({matrix.shape for matrix in gamma})  # should be one shape only
values = sorted({int(v) for v in entries})  # the entries that occur
one_per_line = all(np.all(np.count_nonzero(matrix, axis=0) == 1)
                   and np.all(np.count_nonzero(matrix, axis=1) == 1)
                   for matrix in gamma)  # one nonzero entry per row and column
report("number of gamma matrices", len(gamma))
report("their shapes", shapes)
report("the entries that occur", values)
report("eta in the order x0, ..., x7", [int(v) for v in eta])
reproduces(len(gamma) == 8 and shapes == [(16, 16)] and real_integers
           and values == [-1, 0, 1] and one_per_line,
           "eight real 16 x 16 signed permutation matrices",
           ALGEBRA, ["reality_signed_permutations"])
```

Out [3]:

```
RESULT number of gamma matrices = 8
RESULT their shapes = [(16, 16)]
RESULT the entries that occur = [-1, 0, 1]
RESULT eta in the order x0, ..., x7 = [1, 1, 1, 1, -1, -1, -1, -1]
PASS eight real 16 x 16 signed permutation matrices
     reproduces Revision/algebra/reports/python-algebra.json, check
    reality_signed_permutations
```

Text cell:

```
The next cell checks the Clifford relation
$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\,1$ for all $8 \times 8 = 64$
ordered pairs $(a, b)$ (the products of integer matrices are exact in floating
point), and it computes the sha256 fingerprint of the file ‘gammas.json‘. The Rust
solver printed the first 16 digits of the fingerprint of the file it read into
its check ‘gamma_fixture_numeric‘; equal digits show that the Kohn-Sham states
were computed with exactly these matrices.
```

In [4]:

```
identity16 = np.eye(16)  # the 16 x 16 unit matrix
deviation = 0.0  # the largest deviation from the Clifford relation
for a in range(8):
    for b in range(8):
        anticommutator = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
        target = 2.0 * eta[a] * identity16 if a == b else 0.0 * identity16
        deviation = max(deviation, float(np.max(np.abs(anticommutator - target))))
```

```
report("largest deviation from the Clifford relation, 64 pairs", deviation)
reproduces(deviation == 0.0, "the Clifford relation holds exactly for all 64 pairs",
           ALGEBRA, ["clifford_relation"])
fingerprint = hashlib.sha256(repository_file(GAMMAS).read_bytes()).hexdigest()
report("sha256 of gammas.json, first 16 digits", fingerprint[:16])
solver_detail = record_entry(KS_RUST, "gamma_fixture_numeric")["detail"]
reproduces(f"(sha256 {fingerprint[:16]})" in solver_detail,
           "the Rust Kohn-Sham solver read exactly this file of gammas",
           KS_RUST, ["gamma_fixture_numeric", "block_reduction_numeric"])
```

Out [4]:

```
RESULT largest deviation from the Clifford relation, 64 pairs = 0.0
PASS the Clifford relation holds exactly for all 64 pairs
     reproduces Revision/algebra/reports/python-algebra.json, check clifford_relation
RESULT sha256 of gammas.json, first 16 digits = c99277bab4793c42
PASS the Rust Kohn-Sham solver read exactly this file of gammas
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    gamma_fixture_numeric, block_reduction_numeric
```

Text cell:

```
The next cell draws the eight matrices as coloured tables (heat maps): red is
$+1$, blue is $-1$, white is $0$. The title of each panel says which coordinate
the matrix belongs to and the sign of its square, $(\gamma^a)^2 = \eta^{aa}\,1$.
```

In [5]:

```
LABELS = ["x_1", "x_2", "x_3", "x_4", "x_5", "x_6", "x_7", "x_0"]
fig, axes = plt.subplots(2, 4, figsize=(9.6, 5.4))
for a, ax in enumerate(axes.flat):
    image = ax.imshow(gamma[a], cmap="RdBu_r", vmin=-1.0, vmax=1.0)
    sign = "+1" if eta[a] > 0 else "-1"  # the square of this gamma matrix
    ax.set_title(f"$\\gamma^{{({LABELS[a]})}}$, square ${sign}$", fontsize=10)
    ax.set_xticks([0, 15], ["1", "16"])  # column numbers 1 and 16
    ax.set_yticks([0, 15], ["1", "16"])  # row numbers 1 and 16
    ax.tick_params(labelsize=7)  # small numbers, so that they do not collide
    ax.grid(False)  # no grid lines over the entries
fig.colorbar(image, ax=axes, shrink=0.75, label="matrix entry", ticks=[-1, 0, 1])
nonzero_count = int(sum(np.count_nonzero(matrix) for matrix in gamma))
save_figure(fig, "eight_gammas",
            "The author's eight real $16 \\times 16$ gamma matrices "
            "$\\gamma^{(x_0)}$ to $\\gamma^{(x_7)}$ as heat maps (rows 1 to 16 down, "
            "columns 1 to 16 across; red $+1$, blue $-1$, white $0$). Each row and "
            "each column holds exactly one nonzero entry, so the eight matrices "
            f"have {nonzero_count} nonzero entries in all; the squares are $+1$ "
            "for the space-like directions $x_0, x_1, x_2, x_3$ and $-1$ for the "
            "time $x_4$ and the deflating extra times $x_5, x_6, x_7$. These are "
            "the matrices with which the Kohn-Sham source of this chapter was "
            "computed.")
```

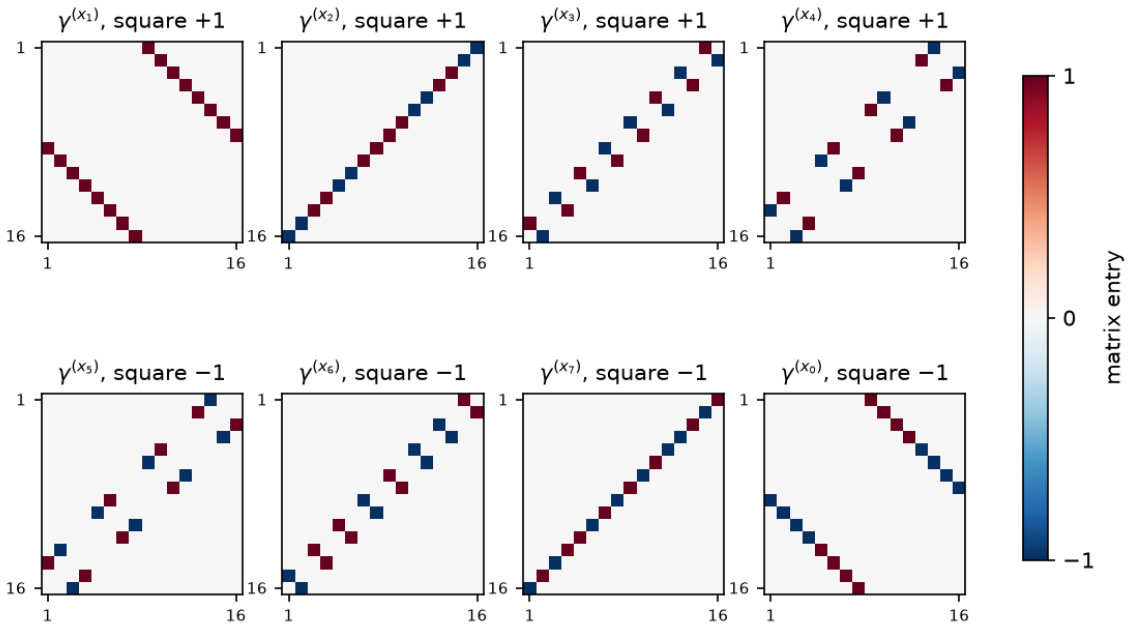

Figure 466: The author's eight real 16×16 gamma matrices $\gamma^{(x_0)}$ to $\gamma^{(x_7)}$ as heat maps (rows 1 to 16 down, columns 1 to 16 across; red $+1$, blue -1, white 0). Each row and each column holds exactly one nonzero entry, so the eight matrices have 128 nonzero entries in all; the squares are $+1$ for the space-like directions x_0, x_1, x_2, x_3 and -1 for the time x_4 and the deflating extra times x_5, x_6, x_7. These are the matrices with which the Kohn-Sham source of this chapter was computed. (Notebook 17b, figure 1.)

Out [5]:

```
Figure 17b.1 saved as Revision/textbook/figures/17b_1_eight_gammas.png
```

Text cell:

```
## 7. The hidden coordinate $y$

The author's metric has $g_{00} = \cot^2 z$ with $z = 6Hx_0$, and the warp factor
$\sin^{1/3}z$ in front of the six directions $x_1, x_2, x_3, x_5, x_6, x_7$. The
Kohn-Sham computations use $y = \ln(\sin z)/(6H)$ instead of $x_0$. The next cell
derives, step by step with sympy:

1. $dy/dx_0 = \cot z$ (the chain rule: $d\ln(\sin z)/dz = \cot z$ and
   $dz/dx_0 = 6H$);
2. $g_{yy} = g_{00}\,(dx_0/dy)^2 = \cot^2 z/\cot^2 z = 1$: $y$ measures proper
   length along the hidden direction;
3. $\sin z = e^{6Hy}$, so the warp factor is $\sin^{1/3}z = e^{2Hy}$;
4. $\sqrt{|\det g|} = \cos z$ in the $x_0$ chart and $e^{6Hy}$ in the $y$ chart
   (the factors $e^{\pm 2a_4}$ of 3-space and of the extra times cancel in the
   determinant: the 7-volume does not change while 3-space inflates and the
   extra times deflate).

To let sympy simplify powers safely, $\sin z$ and $\cos z$ are written as
positive symbols $s$ and $c$ (both are positive for $0 < z < \pi/2$).
```

In [6]:

```
H = sp.symbols("H", positive=True)  # the constant H of the metric
X0 = sp.symbols("x0", positive=True)  # the author's hidden coordinate
z = 6 * H * X0  # the angle z = 6 H x0
y_of_x0 = sp.log(sp.sin(z)) / (6 * H)  # the hidden coordinate y
dy_dx0 = sp.simplify(sp.diff(y_of_x0, X0))  # chain rule
g_yy = sp.simplify(sp.cot(z) ** 2 / dy_dx0 ** 2)  # g00 times (dx0/dy)^2
say(f"dy/dx0 = {dy_dx0},  g_yy = {g_yy}")
s, c, a4_value = sp.symbols("s c a4", positive=True)  # s = sin z, c = cos z
y_of_s = sp.log(s) / (6 * H)  # y written with s = sin z
warp = sp.simplify(sp.exp(2 * H * y_of_s))  # e^{2Hy} in terms of s
say(f"e^(2Hy) = {warp}  (the warp factor sin^(1/3) z)")
factors = ([sp.exp(2 * a4_value) * s ** sp.Rational(1, 3)] * 3 + [1]
           + [sp.exp(-2 * a4_value) * s ** sp.Rational(1, 3)] * 3)  # |g_ii|, i < 8
root_x0 = sp.simplify(sp.sqrt(sp.prod(factors) * (c / s) ** 2))  # g00 = cot^2 z
root_y = sp.simplify(sp.sqrt(sp.prod(factors) * 1))  # g_yy = 1
say(f"sqrt|det g| in the x0 chart = {root_x0};  in the y chart = {root_y}")
reproduces(sp.simplify(dy_dx0 - sp.cot(z)) == 0 and g_yy == 1
           and warp == s ** sp.Rational(1, 3) and root_x0 == c and root_y == s,
           "dy/dx0 = cot z, g_yy = 1, sin^(1/3) z = e^(2Hy), sqrt|g| = cos z or e^(6Hy)",
           KS_PY, ["geometry_hidden_coordinate", "geometry_sqrt_det",
                   "geometry_warped_form"])
```

Out [6]:

```
dy/dx0 = 1/tan(6*H*x0),  g_yy = 1
e^(2Hy) = s**(1/3)  (the warp factor sin^(1/3) z)
sqrt|det g| in the x0 chart = c;  in the y chart = s
PASS dy/dx0 = cot z, g_yy = 1, sin^(1/3) z = e^(2Hy), sqrt|g| = cos z or e^(6Hy)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    geometry_hidden_coordinate, geometry_sqrt_det, geometry_warped_form
```

Text cell:

```
## 8. The conservation law of a source in the author's metric

The next cell defines two helpers that work for any diagonal metric:
‘christoffel‘ computes all $8 \times 8 \times 8 = 512$ Christoffel symbols with
the formula of section 3, and ‘divergence‘ computes the eight components
$\nabla_\mu T^\mu{}_\nu$, $\nu = 1, \dots, 8$, of the covariant divergence of a
table $T^\mu{}_\nu$ (stored as ‘T[mu, nu]‘). It then builds the author's metric in
the $x_0$ chart with a general function $a_4(x_4)$, and the most general source
the $a_4$ record allows: $\rho, p_3, p_t, p_0, q_{40}, q_{04}$, each a function of
$x_4$ and $x_0$. Finally it prints the eight components of the divergence, written
in the record's notation: ‘ad1‘ is $a_4'$, ‘d4rho‘ is $\partial\rho/\partial x_4$,
‘d0p0‘ is $\partial p_0/\partial x_0$, ‘cc‘ is $\cot z$, and so on (the helper
‘readable‘ collects the terms of each source quantity and simplifies the factor in
front of it with $\sin 2z = 2\sin z\cos z$ and $\tan z = \sin z/\cos z$).
```

In [7]:

```
def christoffel(metric, chart):
    """All Gamma^l_{mn} of a diagonal metric, as a nested list [l][m][n]."""
    inverse = [1 / metric[k, k] for k in range(8)]  # diagonal: g^kk = 1 / g_kk
    return [[[sp.simplify(inverse[l] * (sp.diff(metric[l, m], chart[n])
                                        + sp.diff(metric[l, n], chart[m])
                                        - sp.diff(metric[m, n], chart[l])) / 2)
              for n in range(8)] for m in range(8)] for l in range(8)]


def divergence(T, metric, chart):
    """The 8 components nabla_mu T^mu_nu of a table T[mu, nu] = T^mu_nu."""
    G = christoffel(metric, chart)
    result = []
    for nu in range(8):
        total = 0
        for mu in range(8):
            total += sp.diff(T[mu, nu], chart[mu])  # d_mu T^mu_nu
            for lam in range(8):
                total += G[mu][mu][lam] * T[lam, nu]  # Gamma^mu_{mu lam} T^lam_nu
                total -= G[lam][mu][nu] * T[mu, lam]  # Gamma^lam_{mu nu} T^mu_lam
        result.append(sp.simplify(total))
    return result


coords = sp.symbols("x0:8", real=True)  # x0, ..., x7 (the x0 chart)
x4, x0 = coords[4], coords[0]
```

```
a4 = sp.Function("a4")(x4)  # the free function of the metric
zz = 6 * H * x0  # z = 6 H x0
wz = sp.sin(zz) ** sp.Rational(1, 3)  # the warp factor sin^(1/3) z
metric_x0 = sp.diag(*([sp.cot(zz) ** 2] + [sp.exp(2 * a4) * wz] * 3 + [-1]
                      + [-sp.exp(-2 * a4) * wz] * 3))  # x0, ..., x7 (b4 = 1)
names = ("rho", "p3", "pt", "p0", "q40", "q04")
rho, p3, pt, p0, q40, q04 = (sp.Function(n)(x4, x0) for n in names)
T_x0 = sp.diag(p0, p3, p3, p3, -rho, pt, pt, pt)  # the diagonal part
T_x0[4, 0] = q40  # T^x4_x0
T_x0[0, 4] = q04  # T^x0_x4
div_x0 = divergence(T_x0, metric_x0, coords)
notation = {sp.Derivative(f, v): sp.Symbol(f"d{v.name[1]}{f.func.__name__}")
            for f in (rho, p3, pt, p0, q40, q04) for v in (x4, x0)}
notation[sp.Derivative(a4, x4)] = sp.Symbol("ad1")  # a4'


def in_record_notation(expression):
    """Write derivatives as d4rho, d0p0, ... and functions as plain symbols."""
    expression = expression.subs(notation)
    return expression.subs({f: sp.Symbol(f.func.__name__)
                            for f in (rho, p3, pt, p0, q40, q04)})


cc = sp.Symbol("cc")  # the record writes cot z as cc


def readable(expression):
    """A component as a sum of source symbols times simplified coefficients."""
    plain = sp.expand(in_record_notation(expression))
    sources = sorted(plain.free_symbols - {H, x0, sp.Symbol("ad1")}, key=str)
    total = sp.Integer(0)  # the sum, built term by term
    for symbol, factor in sp.collect(plain, sources, evaluate=False).items():
        factor = factor.subs(sp.sin(2 * zz), 2 * sp.sin(zz) * sp.cos(zz))
        factor = sp.simplify(factor.subs(sp.tan(zz), sp.sin(zz) / sp.cos(zz)))
        factor = sp.simplify(factor.subs(sp.sin(zz), sp.cos(zz) / cc))
        total += factor * symbol  # sin z = cos z / cot z
    return total.subs(sp.tan(zz), 1 / cc)


for nu, component in enumerate(div_x0):
    say(f"nabla_mu T^mu_x{nu} = {readable(component)}")
```

Out [7]:

```
nabla_mu T^mu_x0 = 6*H*cc*p0 - 3*H*cc*p3 - 3*H*cc*pt + d0p0 + d4q40
nabla_mu T^mu_x1 = 0
nabla_mu T^mu_x2 = 0
nabla_mu T^mu_x3 = 0
nabla_mu T^mu_x4 = -6*H*q04/cc - 3*ad1*p3 + 3*ad1*pt + d0q04 - d4rho
nabla_mu T^mu_x5 = 0
nabla_mu T^mu_x6 = 0
nabla_mu T^mu_x7 = 0
```

Text cell:

```
Six of the eight components vanish identically. The two others are the
conservation laws in the time direction $x_4$ and in the hidden direction $x_0$.
The next cell reads the record's two equations ‘conservation_x4‘ and
‘conservation_x0‘ (Wolfram Language text; ‘cc‘ stands for $\cot z$) and checks
that they equal ours (the difference is rewritten with cosines and simplified to
zero; one term needs the identity $\tan z + \cot z = 2/\sin 2z$).
```

In [8]:

```
equations = read_json(EQUATIONS)
RECORD_NAMES = {"ad1": sp.Derivative(a4, x4), "H": H, "cc": sp.cot(zz),
                "rho": rho, "p3": p3, "pt": pt, "p0": p0, "q40": q40, "q04": q04}
RECORD_NAMES.update({str(symbol): derivative
                     for derivative, symbol in notation.items()})


def record_equation(text):
    """An equation left == right of the record as the expression left - right."""
    left, right = text.replace("^", "**").split("==")
    return (sp.sympify(left, locals=RECORD_NAMES)
            - sp.sympify(right, locals=RECORD_NAMES))


same = []
for key, nu in (("conservation_x4", 4), ("conservation_x0", 0)):
    recorded = record_equation(equations["generalSource"][key]["input"])
    difference = sp.simplify((div_x0[nu] - recorded).rewrite(sp.cos))
    same.append(difference == 0)
    say(f"{key}: ours minus the record = {difference}")
zero_components = all(div_x0[nu] == 0 for nu in (1, 2, 3, 5, 6, 7))
reproduces(all(same) and zero_components,
           "our divergence equals the record: x4 and x0 laws, six zero components",
           PY_A4, ["conservation_components", "json_conservation"])
```

Out [8]:

```
conservation_x4: ours minus the record = 0
conservation_x0: ours minus the record = 0
PASS our divergence equals the record: x4 and x0 laws, six zero components
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    conservation_components, json_conservation
```

Text cell:

```
## 9. The conservation law in the coordinate $y$, and what it does to C2

The Kohn-Sham states have $q_{40} = q_{04} = 0$ (their mixed component vanishes
for every eigen-orbital) and depend on $y$. In the $y$ chart the author's metric
```

```
is, by section 7,
$e^{2Hy}[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 +
dx_7^2)] - dx_4^2 + dy^2$. The next cell computes the covariant divergence of a
diagonal source $T(x_4, y)$ in this chart with the same helper and checks the two
surviving components against the formulas

$$\nabla_\mu T^\mu{}_{x_4} = -\partial_4\rho - 3a_4'(p_3 - p_t), \qquad
\nabla_\mu T^\mu{}_y = p_0' + 6Hp_0 - 3H(p_3 + p_t)$$

of the Kohn-Sham theory record (the prime on $p_0$ is $\partial/\partial y$). It
prints the components with short names: `ad1` is $a_4'$, `d4rho` is
$\partial\rho/\partial x_4$ and `dyp0` is $\partial p_0/\partial y$.
```

In [9]:

```
yc = sp.symbols("y", real=True)  # the hidden coordinate y
chart_y = [yc] + list(coords[1:])  # y (for x0), x1, ..., x7
wy = sp.exp(2 * H * yc)  # the warp factor e^{2Hy} = sin^(1/3) z
metric_y = sp.diag(*([1] + [sp.exp(2 * a4) * wy] * 3 + [-1]
                     + [-sp.exp(-2 * a4) * wy] * 3))
RHO, P3, PT, P0 = (sp.Function(n)(x4, yc) for n in ("rho", "p3", "pt", "p0"))
T_y = sp.diag(P0, P3, P3, P3, -RHO, PT, PT, PT)  # a diagonal source T(x4, y)
div_y = divergence(T_y, metric_y, chart_y)
law_x4 = -sp.diff(RHO, x4) - 3 * sp.diff(a4, x4) * (P3 - PT)
law_y = sp.diff(P0, yc) + 6 * H * P0 - 3 * H * (P3 + PT)
derivatives = {sp.Derivative(RHO, x4): sp.Symbol("d4rho"),
               sp.Derivative(P0, yc): sp.Symbol("dyp0"),
               sp.Derivative(a4, x4): sp.Symbol("ad1")}  # short names
functions = {f: sp.Symbol(f.func.__name__) for f in (RHO, P3, PT, P0)}


def short(expression):
    """Write the derivatives as d4rho, dyp0, ad1 and the functions as symbols."""
    return expression.subs(derivatives).subs(functions)


say(f"nabla_mu T^mu_x4 = {short(div_y[4])}")
say(f"nabla_mu T^mu_y = {short(div_y[0])}")
others_zero = all(div_y[nu] == 0 for nu in (1, 2, 3, 5, 6, 7))
theory = read_json(KS_THEORY)["emt"]  # the energy-momentum part of the record
say("ks-theory.json, emt.conservationY: " + theory["conservationY"])
reproduces(sp.simplify(div_y[4] - law_x4) == 0 and sp.simplify(div_y[0] - law_y) == 0
           and others_zero
           and "6H p0 = 3H (p3 + p_t)" in theory["conservationY"],
           "in the y chart: the x4 law and dp0/dy + 6H p0 = 3H (p3 + p_t)",
           KS_PY, ["emt_y_conservation_selfconsistent", "emt_x4_component"])
```

Out [9]:

```
nabla_mu T^mu_x4 = -3*ad1*p3 + 3*ad1*pt - d4rho
nabla_mu T^mu_y = 6*H*p0 - 3*H*p3 - 3*H*pt + dyp0
ks-theory.json, emt.conservationY: p0' + 6H p0 = 3H (p3 + p_t) for every self-consistent
```

```
    state (nabla_mu T^mu_y = 0): solver check
PASS in the y chart: the x4 law and dp0/dy + 6H p0 = 3H (p3 + p_t)
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check
    emt_y_conservation_selfconsistent, emt_x4_component
```

Text cell:

```
Now the key step. Solve the $y$ law for $p_3$:
$p_3 = (p_0' + 6Hp_0)/(3H) - p_t$. Put this into the violation of C2:

$$V = p_3 + p_t - 2p_0 = \frac{p_0' + 6Hp_0}{3H} - 2p_0 = \frac{p_0'}{3H}.$$

The first equality is the definition of $V$; the second replaces $p_3 + p_t$ by
its value from the conservation law; the third cancels $6Hp_0/(3H) = 2p_0$
against $-2p_0$. So, for EVERY conserved diagonal source: **C2 holds at a point
exactly when $p_0$ is flat there.** C2 is not an extra demand on a conserved
source; it is the $p_0$ part of C1. The next cell repeats the algebra with
sympy.
```

In [10]:

```
p3_from_law = sp.solve(sp.Eq(law_y, 0), P3)[0]  # p3 from the y conservation law
V = P3 + PT - 2 * P0  # the violation of C2
V_conserved = sp.simplify(V.subs(P3, p3_from_law))
say(f"p3 from the conservation law: {short(p3_from_law)}")
say(f"V = p3 + p_t - 2 p0 for a conserved source: {short(V_conserved)}")
check(sp.simplify(V_conserved - sp.diff(P0, yc) / (3 * H)) == 0,
      "PROVED: for a conserved source V = p3 + p_t - 2 p0 = (dp0/dy) / (3H)")
```

Out [10]:

```
p3 from the conservation law: 2*p0 - pt + dyp0/(3*H)
V = p3 + p_t - 2 p0 for a conserved source: dyp0/(3*H)
PASS PROVED: for a conserved source V = p3 + p_t - 2 p0 = (dp0/dy) / (3H)
```

Text cell:

```
## 10. Testing $V = p_0'/(3H)$ on the recorded Kohn-Sham states

The next cell reads the 75 profile files of the Kohn-Sham record (151 points
$y = -3, -2.98, \dots, 0$; columns `rho`, `p3`, `p_t`, `p0` among others), the
table of integrals and the parameters. It defines `slope`, the fourth-order
difference of section 3 with the step $s \cdot h$ ($h = 0.02$, $s = 1, 2, \dots$),
evaluated at the inner points (two points at each end have no neighbours on one
side). For every state with a nonzero tensor it compares $V$ with $p_0'/(3H)$ and
divides the largest difference by max|T| (the largest absolute value of $\rho$,
$p_3$, $p_t$, $p_0$ of the state).
```

In [11]:

```
COLUMNS = ("rho", "p3", "p_t", "p0")  # the four diagonal components


def read_profile(path):
    """A profile table as a dictionary: column name -> numpy array (151 values)."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}


files = sorted(repository_file(f"{GROUND}/profiles").glob("*.csv"),
               key=lambda path: path.name)
profiles = {path.stem: read_profile(path) for path in files}
with repository_file(f"{GROUND}/emt-integrals.csv").open(
        encoding="utf-8", newline="") as handle:
    integrals = {row["id"]: row for row in csv.DictReader(handle)}
physics = read_json("Revision/kohn_sham/results/parameters.json")["physics"]
H_value, L_cut, VOL7 = physics["H"], physics["L_tipCutoff"], physics["Vol7"]
scale = {sid: float(max(np.max(np.abs(prof[c])) for c in COLUMNS))
         for sid, prof in profiles.items()}  # max|T| of every state
nonzero = [sid for sid in profiles if scale[sid] > 0.0]
y = profiles["N136_lam0_a10"]["y"]  # the common grid of y
h = float(y[1] - y[0])  # the grid step


def slope(values, s=1):
    """dv/dy at the points i = 2s, ..., 150 - 2s (fourth order, step s h)."""
    v = values
    return (-v[4 * s:] + 8 * v[3 * s:-s] - 8 * v[s:-3 * s] + v[:-4 * s]) / (12 * h * s)


def violation(prof):
    """V(y) = p3 + p_t - 2 p0 of a profile."""
    return prof["p3"] + prof["p_t"] - 2.0 * prof["p0"]


residual = {sid: float(np.max(np.abs(slope(profiles[sid]["p0"]) / (3 * H_value)
                                     - violation(profiles[sid])[2:-2])))
            / scale[sid] for sid in nonzero}
worst = max(residual, key=residual.get)  # the state with the largest difference
report("states with a nonzero tensor", len(nonzero))
report("grid step h", f"{h:.2f}")
report("largest |(dp0/dy)/(3H) - V| / max|T| over 70 states", f"{residual[worst]:.2e}",
       f"({worst})")
check(len(nonzero) == 70 and residual[worst] < 1e-3,
      "V = (dp0/dy)/(3H) on every state to 1e-3 of max|T| (fourth-order differences)")
```

Out [11]:

```
RESULT states with a nonzero tensor = 70
RESULT grid step h = 0.02
RESULT largest |(dp0/dy)/(3H) - V| / max|T| over 70 states = 1.05e-04 (N8_lamm1_a00)
PASS V = (dp0/dy)/(3H) on every state to 1e-3 of max|T| (fourth-order differences)
```

Text cell:

```
The next cell draws the identity for the history $N = 136$, $\lambda = 0$. Left:
$p_0(y)$ at the five slices (divided by max|T|): for C2, by section 9, these
curves would have to be horizontal. Right: $V(y)$ (lines) and $p_0'(y)/(3H)$
(dots, every fifth grid point) of the slice $a_{4,0} = 1$. Both vertical axes are
symmetric logarithmic (logarithmic for large positive and negative values,
linear near zero), so that both signs and many orders of magnitude are visible.
```

In [12]:

```
SLICES = {"a00": 0.0, "a05": 0.5, "a10": 1.0, "a15": 1.5, "a20": 2.0}  # a4,0
SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light to dark
history136 = [f"N136_lam0_{tag}" for tag in SLICES]
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.0))
for shade, sid, a40 in zip(SHADES, history136, SLICES.values()):
    left.plot(y, profiles[sid]["p0"] / scale[sid], color=shade, lw=1.8,
              label=f"$a_{{4,0}} = {a40:g}$")
left.set_yscale("symlog", linthresh=1e-6)
left.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
left.set_ylabel("$p_0(y)$ / max|T|")
left.set_title("$N = 136$, $\\lambda = 0$: $p_0$ is far from flat")
left.legend(fontsize=7, loc="upper right")
example = profiles["N136_lam0_a10"]
inner = y[2:-2]  # the points where the slope is known
right.plot(y, violation(example) / scale["N136_lam0_a10"], color="#2a78d6",
           lw=2.0, label="$V = p_3 + p_t - 2p_0$")
right.plot(inner[::5], (slope(example["p0"]) / (3 * H_value))[::5]
           / scale["N136_lam0_a10"], "o", color="#eb6834", ms=3.5,
           label="$p_0'(y)/(3H)$")
right.axhline(0.0, color="#e34948", ls="--", lw=1.0, label="C2: $V = 0$")
right.set_yscale("symlog", linthresh=1e-6)
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("divided by max|T|")
right.set_title("$a_{4,0} = 1$: the two sides of the identity")
right.legend(fontsize=7, loc="upper right")
fig.tight_layout()
p0_range = [float(np.max(np.abs(profiles[s]["p0"])) / np.min(np.abs(profiles[s]["p0"])))
            for s in history136]  # how far p0 is from flat, per slice
save_figure(fig, "slope_identity",
            "Left: the hidden-direction pressure $p_0(y)$ of the Kohn-Sham states "
            "$N = 136$, $\\lambda = 0$ at the five slices $a_{4,0} = 0$ to $2$, "
            "divided by the largest component of each state (horizontal axis: the "
            "hidden coordinate $y$; vertical axis symmetric logarithmic, linear "
            "between $-10^{-6}$ and $10^{-6}$); the largest $|p_0|$ is "
            f"{min(p0_range):.0f} to ${tex_number(max(p0_range))}$ times the "
            "smallest, while "
            "condition C2 of a conserved source demands a flat $p_0$. Right: for "
            "$a_{4,0} = 1$ the violation $V = p_3 + p_t - 2p_0$ (blue line) and the "
            "slope $p_0'(y)/(3H)$ from fourth-order differences (orange dots) lie "
            "on top of each other, as the conservation law demands; the dashed "
            "line is the value $0$ that C2 needs.")
```

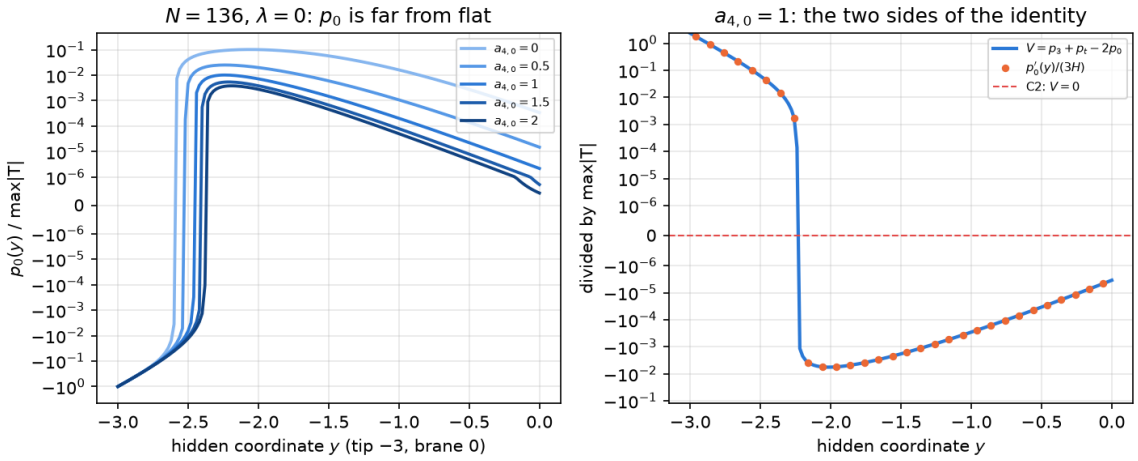

Figure 467: Left: the hidden-direction pressure $p_0(y)$ of the Kohn-Sham states $N = 136$, $\lambda = 0$ at the five slices $a_{4,0} = 0$ to 2, divided by the largest component of each state (horizontal axis: the hidden coordinate y; vertical axis symmetric logarithmic, linear between -10^{-6} and 10^{-6}); the largest $|p_0|$ is 2933 to 2.3×10^6 times the smallest, while condition C2 of a conserved source demands a flat p_0. Right: for $a_{4,0} = 1$ the violation $V = p_3 + p_t - 2p_0$ (blue line) and the slope $p_0'(y)/(3H)$ from fourth-order differences (orange dots) lie on top of each other, as the conservation law demands; the dashed line is the value 0 that C2 needs. (Notebook 17b, figure 2.)

Out [12]:

```
Figure 17b.2 saved as Revision/textbook/figures/17b_2_slope_identity.png
```

Text cell:

```
How do we know that the small differences of the last test are errors of the
finite differences and not a failure of conservation? By changing the step. The
next cell evaluates $|p_0'/(3H) - V|/\max|T|$ at the four points
$y = -2.4, -1.8, -1.2, -0.6$ (grid indices 30, 60, 90, 120) with the steps
$s \cdot h$, $s = 1, 2, 3, 5, 6$, for five states, and fits a straight line to
$\log(\text{difference})$ against $\log(\text{step})$. A fourth-order difference
of an exact identity must give the slope $4$.
```

In [13]:

```
STEPS = np.array([1, 2, 3, 5, 6])  # the step is s h
POINTS = [30, 60, 90, 120]  # grid indices of y = -2.4, -1.8, -1.2, -0.6
STUDY = ["N8_lamp1_a00", "N136_lam0_a10", "N136_lamp2_a20", "N688_lam0_a10",
         "N688_lamm2_a00"]


def difference_at_points(sid, s):
    """max over POINTS of |(dp0/dy)/(3H) - V| / max|T| with the step s h."""
    prof = profiles[sid]
    derivative = slope(prof["p0"], s) / (3 * H_value)  # index i - 2s is point i
```

```
    return max(abs(float(derivative[i - 2 * s] - violation(prof)[i]))
               for i in POINTS) / scale[sid]


errors = {sid: np.array([difference_at_points(sid, s) for s in STEPS])
          for sid in STUDY}
orders = {sid: float(np.polyfit(np.log(STEPS * h), np.log(errors[sid]), 1)[0])
          for sid in STUDY}  # the slope of the straight-line fit
for sid in STUDY:
    say(f"{sid}: differences " + ", ".join(f"{e:.1e}" for e in errors[sid])
        + f";  fitted order {orders[sid]:.2f}")
check(all(3.8 < order < 4.2 for order in orders.values()),
      "the differences fall like (step)^4: they are errors of the differences")
fig, ax = plt.subplots(figsize=(6.4, 4.4))
for sid in STUDY:
    ax.loglog(STEPS * h, errors[sid], "o-", lw=1.4, ms=4,
              label=f"{sid} (order {orders[sid]:.2f})")
reference = errors["N136_lam0_a10"][0] * (STEPS / STEPS[0]) ** 4.0
ax.loglog(STEPS * h, reference, "k--", lw=1.0, label="slope 4 (reference)")
ax.set_xlabel("step of the difference $s h$ (units of $1/H$)")
ax.set_ylabel("max $|p_0'/(3H) - V|$ / max|T|")
ax.legend(fontsize=7)
save_figure(fig, "difference_order",
            "The difference between $p_0'(y)/(3H)$, computed with fourth-order "
            "differences of step $sh$ ($h = 0.02$, $s = 1, 2, 3, 5, 6$), and the "
            "violation $V = p_3 + p_t - 2p_0$, at the four points $y = -2.4$, "
            "$-1.8$, $-1.2$, $-0.6$, divided by the largest component of the state, "
            "for five recorded Kohn-Sham states (logarithmic axes; the step in units "
            "of $1/H$). The fitted slopes lie between "
            f"{min(orders.values()):.2f} and {max(orders.values()):.2f}, the order "
            "4 of the differences: the small differences are errors of the finite "
            "differences, and the conservation law itself holds.")
```

Out [13]:

```
N8_lamp1_a00: differences 6.9e-07, 1.1e-05, 5.8e-05, 4.8e-04, 1.0e-03;  fitted order 4.08
N136_lam0_a10: differences 4.6e-07, 7.5e-06, 3.8e-05, 3.0e-04, 6.5e-04;  fitted order
    4.04
N136_lamp2_a20: differences 5.4e-07, 8.6e-06, 4.4e-05, 3.4e-04, 7.2e-04;  fitted order
    4.02
N688_lam0_a10: differences 4.6e-07, 7.4e-06, 3.8e-05, 3.0e-04, 6.4e-04;  fitted order
    4.04
N688_lamm2_a00: differences 4.6e-07, 7.5e-06, 3.8e-05, 3.1e-04, 6.5e-04;  fitted order
    4.04
PASS the differences fall like (step)^4: they are errors of the differences
```

Out [13]:

```
Figure 17b.3 saved as Revision/textbook/figures/17b_3_difference_order.png
```

Text cell:

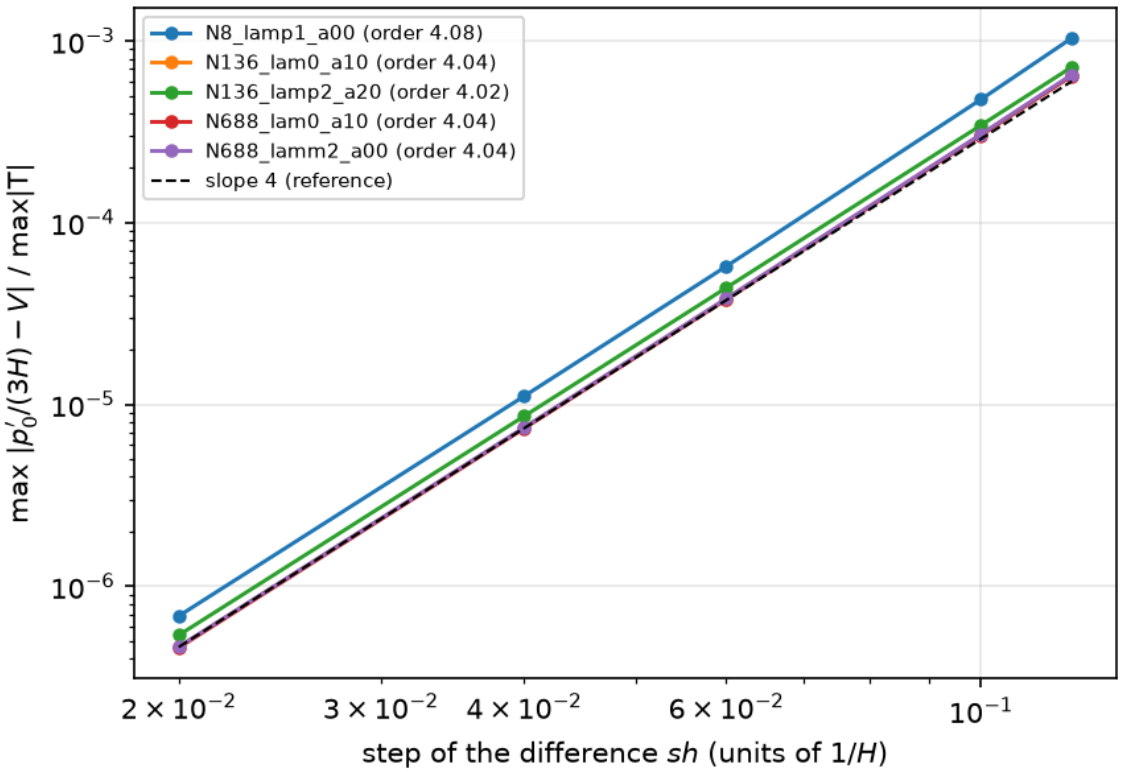

Figure 468: The difference between $p_0'(y)/(3H)$, computed with fourth-order differences of step sh ($h = 0.02$, $s = 1, 2, 3, 5, 6$), and the violation $V = p_3 + p_t - 2p_0$, at the four points $y = -2.4$, -1.8, -1.2, -0.6, divided by the largest component of the state, for five recorded Kohn-Sham states (logarithmic axes; the step in units of $1/H$). The fitted slopes lie between 4.02 and 4.08, the order 4 of the differences: the small differences are errors of the finite differences, and the conservation law itself holds. (Notebook 17b, figure 3.)

```
## 11. The integrated form, and why the integrated ratio of C2 is not 1

Multiply the $y$ law by $e^{6Hy}$: since $(e^{6Hy}p_0)' = e^{6Hy}(p_0' + 6Hp_0)$
(product rule), the law becomes $(e^{6Hy}p_0)' = 3He^{6Hy}(p_3 + p_t)$. Integrate
from the tip $y = -L$ to the brane $y = 0$ (fundamental theorem of calculus) and
multiply by $2\,\mathrm{Vol}_7$ (the volume factor of the record's integrals):

$$2\,\mathrm{Vol}_7\left[p_0(0) - e^{-6HL}p_0(-L)\right] =
3H\left(\textstyle\int p_3 + \int p_t\right),$$

where $\int X$ means $2\,\mathrm{Vol}_7\int_{-L}^{0}e^{6Hy}X\,dy$, the column
`int_X` of the record's table. The next cell checks this with the table's brane
values, tip values and integrals for all 70 nonzero states (relative difference
of the two sides, divided by the right side). The table also has a column
`ycons_integrated_rel`: the solver's own relative difference of the same two
sides, computed from its unrounded numbers and divided by the largest of the terms
involved (not by the right side alone), so it is a close cousin of our number,
not the same number by definition; the cell checks that its largest value is the
one printed in the solver's check
`emt_y_conservation_integrated` (two mirror states, $\lambda = \pm\lambda_2$ with
$N = 8$, share this largest value; the record names one of them). Our
differences, computed from numbers printed with 16 digits, are of the same tiny
size, about $10^{-11}$.
```

In [14]:

```
def number(sid, column):
    """A number of the record's table of integrals."""
    return float(integrals[sid][column])


tip_weight = float(np.exp(-6.0 * H_value * L_cut))  # e^{-6HL}, about 1.5e-8
integrated = {}
for sid in nonzero:
    left_side = 2 * VOL7 * (number(sid, "p0_brane") - tip_weight * number(sid, "p0_tip"))
    right_side = 3 * H_value * (number(sid, "int_p3") + number(sid, "int_p_t"))
    integrated[sid] = abs(left_side - right_side) / abs(right_side)
worst_integrated = max(integrated, key=integrated.get)
report("largest relative difference of the integrated law",
       f"{integrated[worst_integrated]:.3e} ({worst_integrated})")
column = {sid: number(sid, "ycons_integrated_rel") for sid in nonzero}
column_worst = max(column, key=column.get)  # the solver's own largest value
report("largest value of the column ycons_integrated_rel",
       f"{column[column_worst]:.3e} ({column_worst})")
solver = record_entry(KS_RUST, "emt_y_conservation_integrated")["detail"]
say("the solver record: " + solver)
named = solver.split("worst value ")[1].split("(")[1].split(")")[0]  # its state
tie = abs(column[named] - column[column_worst]) < 1e-15  # mirror states tie
report("the state named by the record and its column value",
       f"{named}, {column[named]:.3e}")
reproduces(integrated[worst_integrated] < 1e-10 and tie
           and f"worst value {column[column_worst]:.3e}" in solver,
           "the integrated conservation law holds for all 70 states",
```

```
          KS_RUST, ["emt_y_conservation_integrated"])
```

Out [14]:

```
RESULT largest relative difference of the integrated law = 1.421e-11 (N8_lamm2_a00)
RESULT largest value of the column ycons_integrated_rel = 1.422e-11 (N8_lamm2_a00)
the solver record: [e^{6Hy} p0]_{-L}^{0} = 3H int e^{6Hy} (p3 + p_t) dy (nabla_mu T^mu_y
    = 0): relative difference; 75 cases; worst value 1.422e-11 (N8_lamp2_a00); tolerance
    1.0e-7; failures: none
RESULT the state named by the record and its column value = N8_lamp2_a00, 1.422e-11
PASS the integrated conservation law holds for all 70 states
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    emt_y_conservation_integrated
```

Text cell:

```
Notebook 17a found that the integrated ratio $R = (\int p_3 + \int p_t)/(2\int
p_0)$ lies between about $0.1$ and $0.4$ instead of $1$. The integrated law
explains this number. Divide it by $2\int p_0$ and write $\int p_0 =
2\,\mathrm{Vol}_7\,\bar p_0\,(1 - e^{-6HL})/(6H)$, where $\bar p_0$ is the
weighted mean of $p_0$ (because $\int_{-L}^0 e^{6Hy}dy = (1 - e^{-6HL})/(6H)$):

$$R = \frac{b_0}{\bar p_0}, \qquad b_0 = \frac{p_0(0) - e^{-6HL}p_0(-L)}
{1 - e^{-6HL}}.$$

We call $b_0$ the **boundary value** of $p_0$: the brane value with the small tip
term. So the averaged C2 asks that the boundary value of $p_0$ equal its mean:
true for a flat $p_0$, false for the Kohn-Sham states. Is the tip term negligible,
so that $b_0 \approx p_0(0)$? The weight $e^{-6HL} = e^{-18}$ is tiny, but near the
tip $|p_0|$ is enormous. The next cell computes $R$ both ways for every nonzero
state, checks the record's closest value, and measures the share of $R$ that the
tip term supplies. It also measures, from the profiles, which part of the weighted
mean of $p_0$ comes from the last unit before the brane, $-1 \le y \le 0$.
```

In [15]:

```
mean_p0 = {sid: number(sid, "int_p0") / (2 * VOL7 * (1 - tip_weight) / (6 * H_value))
           for sid in nonzero}  # the weighted mean of p0
ratio = {sid: (number(sid, "int_p3") + number(sid, "int_p_t"))
         / (2 * number(sid, "int_p0")) for sid in nonzero}  # as in Notebook 17a
boundary = {sid: (number(sid, "p0_brane") - tip_weight * number(sid, "p0_tip"))
            / (1 - tip_weight) for sid in nonzero}  # the boundary value b0
from_brane = {sid: boundary[sid] / mean_p0[sid] for sid in nonzero}  # R = b0/mean
brane_only = {sid: number(sid, "p0_brane") / mean_p0[sid] for sid in nonzero}
tip_share = {sid: 1.0 - brane_only[sid] / from_brane[sid] for sid in nonzero}
step = float(y[1] - y[0])  # the grid step of y, 0.02


def simpson_rule(values):
    """Simpson's rule with the step of y (for an odd number of points)."""
    return step / 3 * (values[0] + values[-1] + 4 * values[1:-1:2].sum()
```

```
                    + 2 * values[2:-1:2].sum())


last_unit = y >= -1.0 - 1e-9  # the last unit before the brane: 51 points
near_share, table_gap = {}, {}
for sid in nonzero:
    weighted = np.exp(6 * H_value * y) * profiles[sid]["p0"]  # e^{6Hy} p0
    whole = simpson_rule(weighted)  # the integral over the whole patch
    near_share[sid] = simpson_rule(weighted[last_unit]) / whole
    table_gap[sid] = abs(2 * VOL7 * whole / number(sid, "int_p0") - 1.0)
agreement = max(abs(from_brane[sid] / ratio[sid] - 1.0) for sid in nonzero)
closest = min(ratio, key=lambda sid: abs(ratio[sid] - 1.0))
report("largest relative difference of the two forms of R", f"{agreement:.1e}")
report("R closest to 1", f"{ratio[closest]:.6g} ({closest})")
report("largest relative difference, Simpson's integral of p0 and int_p0",
       f"{max(table_gap.values()):.1e}")
for n in (8, 136, 688):
    shares = [tip_share[sid] for sid in nonzero if sid.startswith(f"N{n}_")]
    report(f"share of R from the tip term, N = {n}",
           f"{min(shares):.2g} to {max(shares):.2g}")
    nears = [near_share[sid] for sid in nonzero if sid.startswith(f"N{n}_")]
    report(f"share of the mean of p0 from -1 <= y <= 0, N = {n}",
           f"{min(nears):.3f} to {max(nears):.3f}")
detail = record_entry(SOURCE_REPORT, "ks_integrals_violate_algebraic_condition")[
    "detail"]
reproduces(agreement < 1e-9
           and f"(closest to 1: {ratio[closest]:.6g} at {closest})" in detail,
           "R = b0 / mean of p0 (b0: brane value with the tip term), 70 states",
           SOURCE_REPORT, ["ks_integrals_violate_algebraic_condition"])
```

Out [15]:

```
RESULT largest relative difference of the two forms of R = 1.4e-11
RESULT R closest to 1 = 0.414328 (N688_lamm2_a00)
RESULT largest relative difference, Simpson's integral of p0 and int_p0 = 4.7e-09
RESULT share of R from the tip term, N = 8 = 0.31 to 0.33
RESULT share of the mean of p0 from -1 <= y <= 0, N = 8 = 0.439 to 0.441
RESULT share of R from the tip term, N = 136 = 4.1e-05 to 0.039
RESULT share of the mean of p0 from -1 <= y <= 0, N = 136 = 0.812 to 0.932
RESULT share of R from the tip term, N = 688 = 5.3e-06 to 0.025
RESULT share of the mean of p0 from -1 <= y <= 0, N = 688 = 0.816 to 0.969
PASS R = b0 / mean of p0 (b0: brane value with the tip term), 70 states
     reproduces Revision/field_equations_a4/reports/ks-source-conditions.json, check
    ks_integrals_violate_algebraic_condition
```

Text cell:

```
The next cell draws the boundary value $|b_0|$ against the mean $|\bar p_0|$ for
all 70 states (logarithmic axes; filled markers where $\bar p_0$ is positive,
larger open rings where it is negative). Their ratio is exactly $R$, so the
averaged C2 would put every point on the diagonal. The cell also lists the states
with a negative mean $\bar p_0$ and checks that every point lies below the
diagonal ($0 < b_0/\bar p_0 < 1$).
```

In [16]:

```
N_COLOURS = {8: "#1baf7a", 136: "#2a78d6", 688: "#eb6834"}
fig, ax = plt.subplots(figsize=(6.4, 5.0))
for n, colour in N_COLOURS.items():
    for positive in (True, False):
        ids = [sid for sid in nonzero if sid.startswith(f"N{n}_")
               and (mean_p0[sid] > 0) == positive]  # one sign at a time
        if not ids:
            continue
        ax.loglog([abs(mean_p0[sid]) for sid in ids],
                  [abs(boundary[sid]) for sid in ids], "o",
                  ms=5 if positive else 10,  # rings around the mirror states
                  color=colour, mfc=colour if positive else "none",
                  label=f"$N = {n}$" + ("" if positive else ", mean $< 0$"))
line = np.array([1e-7, 1.0])  # the range of the diagonal
ax.loglog(line, line, "--", color="#e34948", lw=1.2,
          label="averaged C2: boundary value = mean")
ax.set_xlabel("weighted mean $|\\bar p_0|$ (units of $m^8$)")
ax.set_ylabel("boundary value $|b_0|$ (units of $m^8$)")
ax.legend(fontsize=8, loc="upper left")
negative = sorted(sid for sid in nonzero if mean_p0[sid] < 0)  # mean of p0 < 0
say("states with a negative mean of p0: " + ", ".join(negative))
expected = sorted(sid for sid in nonzero if sid.startswith("N8_lamp"))
plotted = {sid: boundary[sid] / mean_p0[sid] for sid in nonzero}  # = R
save_figure(fig, "brane_and_mean",
            "The boundary value $b_0 = (p_0(0) - e^{-6HL}p_0(-L))/(1 - e^{-6HL})$ "
            "of the hidden-direction pressure against its weighted mean, for the "
            "70 recorded Kohn-Sham states with a nonzero source (logarithmic "
            "axes, units of $m^8$; colours: particle number; open rings: the "
            "$N = 8$ states with $\\lambda > 0$, whose mean and brane value of "
            "$p_0$ are negative; they enclose their partners with $-\\lambda$, "
            "which have the same absolute values; the 20 nonzero $N = 8$ states "
            "fall on only two places). By the integrated "
            "conservation law $b_0/\\bar p_0$ is exactly the averaged ratio $R$, "
            "and the averaged condition C2 holds only on the dashed diagonal; "
            "every state lies below it, with ratios between "
            f"{min(plotted.values()):.3f} and {max(plotted.values()):.3f}. The brane "
            "value alone gives the ratios "
            f"{min(brane_only.values()):.3f} to {max(brane_only.values()):.3f}: for "
            "the $N = 8$ states the tip term supplies about a third of $R$.")
check(negative == expected and all(0.0 < plotted[sid] < 1.0 for sid in nonzero),
      "every point lies below the diagonal; mean of p0 < 0 only for N = 8, "
      "lambda > 0")
```

Out [16]:

```
states with a negative mean of p0: N8_lamp1_a00, N8_lamp1_a05, N8_lamp1_a10,
    N8_lamp1_a15, N8_lamp1_a20, N8_lamp2_a00, N8_lamp2_a05, N8_lamp2_a10, N8_lamp2_a15,
    N8_lamp2_a20
```

Out [16]:

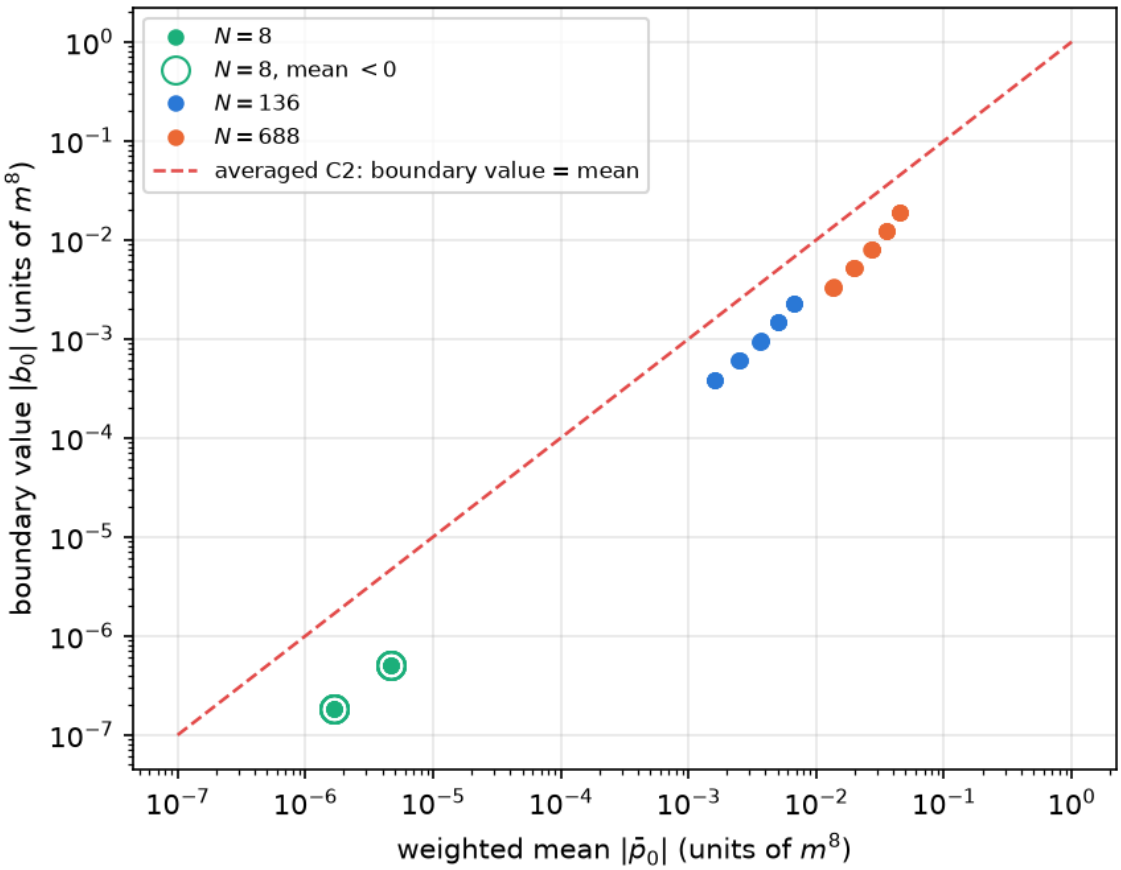

Figure 469: The boundary value $b_0 = (p_0(0) - e^{-6HL}p_0(-L))/(1 - e^{-6HL})$ of the hidden-direction pressure against its weighted mean, for the 70 recorded Kohn-Sham states with a nonzero source (logarithmic axes, units of m^8; colours: particle number; open rings: the $N = 8$ states with $\lambda > 0$, whose mean and brane value of p_0 are negative; they enclose their partners with $-\lambda$, which have the same absolute values; the 20 nonzero $N = 8$ states fall on only two places). By the integrated conservation law $b_0/\bar{p}_0$ is exactly the averaged ratio R, and the averaged condition C2 holds only on the dashed diagonal; every state lies below it, with ratios between 0.107 and 0.414. The brane value alone gives the ratios 0.074 to 0.414: for the $N = 8$ states the tip term supplies about a third of R. (Notebook 17b, figure 4.)

```
Figure 17b.4 saved as Revision/textbook/figures/17b_4_brane_and_mean.png
PASS every point lies below the diagonal; mean of p0 < 0 only for N = 8, lambda > 0
```

Text cell:

```
## 12. The time direction: energy exchange and condition C3

The $x_4$ law of section 9 for a source that does not depend on $x_0$ reads
$\rho' = -3a_4'(p_3 - p_t)$. In words: when 3-space inflates ($a_4' > 0$) a
3-space pressure $p_3$ takes energy out of the source, and when the extra times
deflate an extra-time pressure $p_t$ puts energy in. For the linear member
$a_4' = AH \neq 0$, so a constant $\rho$ (part of C3) holds exactly when
$p_3 = p_t$ (another part of C3).

The $a_4$ equations know this. Write the constraint as
$\mathcal C = \sum_k\alpha_kE_{(k)}{}^{x_4}{}_{x_4} + \Lambda + \kappa\rho = 0$
and the evolution equation as $\mathcal E = a_4''F - \kappa(p_3 - p_t) = 0$. The
left-hand side of $\mathcal C$ depends on $x_4$ only through $a_4'$, so by the
chain rule $d\mathcal C/dx_4 = (\partial\mathcal C/\partial a_4')\,a_4'' +
\kappa\rho'$. The next cell inserts the record's Lovelock components and $F$,
uses $\rho' = -3a_4'(p_3 - p_t)$, and checks
$d\mathcal C/dx_4 = 3a_4'\,\mathcal E$ for all couplings $\alpha_1, \alpha_2,
\alpha_3$. Along the linear member ($a_4'' = 0$) this gives
$d\mathcal C/dx_4 = -3\kappa AH(p_3 - p_t)$: the constraint can hold at all
times only if $p_3 = p_t$.
```

In [17]:

```
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
alpha = sp.symbols("alpha1:4", real=True)  # the Lovelock couplings
kappa, p3s, pts = sp.symbols("kappa p3 pt", real=True)  # kappa and the pressures
LOCALS = {"ad1": ad1, "ad2": ad2, "H": H, "alpha1": alpha[0], "alpha2": alpha[1],
          "alpha3": alpha[2]}


def parse(text):
    """A formula of the record (Wolfram Language text) as a sympy expression."""
    return sp.sympify(text.replace("^", "**"), locals=LOCALS)


E44 = sum(alpha[k - 1] * parse(equations["lovelockTensors"][f"E{k}"]["x4x4"]["input"])
          for k in (1, 2, 3))  # sum_k alpha_k E_(k)^x4_x4
F = parse(equations["generalSource"]["evolution_F"]["input"])
rho_dot = -3 * ad1 * (p3s - pts)  # the x4 conservation law
constraint_dot = sp.diff(E44, ad1) * ad2 + kappa * rho_dot  # chain rule
evolution = ad2 * F - kappa * (p3s - pts)
say(f"d/d(a4') of the x4 component: {sp.factor(sp.diff(E44, ad1))}")
say(f"3 a4' F: {sp.factor(3 * ad1 * F)}")
reproduces(sp.expand(constraint_dot - 3 * ad1 * evolution) == 0,
           "PROVED: dC/dx4 = 3 (da4/dx4) times the evolution equation, any alpha_k",
           WL_A4, ["constraint_propagation_bianchi"])
linear = sp.expand(constraint_dot.subs({ad2: 0, ad1: sp.Symbol("A") * H}))
say(f"along the linear member a4 = A H x4: dC/dx4 = {sp.factor(linear)}")
```

Out [17]:

```
d/d(a4') of the x4 component: 6*ad1*(360*H**4*alpha3 + 432*H**2*ad1**2*alpha3 -
    40*H**2*alpha2 + 360*ad1**4*alpha3 - 24*ad1**2*alpha2 + alpha1)
3 a4' F: 6*ad1*(360*H**4*alpha3 + 432*H**2*ad1**2*alpha3 - 40*H**2*alpha2 +
    360*ad1**4*alpha3 - 24*ad1**2*alpha2 + alpha1)
PASS PROVED: dC/dx4 = 3 (da4/dx4) times the evolution equation, any alpha_k
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    constraint_propagation_bianchi
along the linear member a4 = A H x4: dC/dx4 = -3*A*H*kappa*(p3 - pt)
```

Text cell:

```
For the Kohn-Sham gas only the integrated version of the $x_4$ law is recorded
(the cell after the next two shows why: point by point it fails):
$dE/da_{4,0} = -3\left(\int p_3 - \int p_t\right)$ with the total energy
$E = \int\rho$ (the Rust solver checks it with tiny steps of $a_4$). The recorded
slices are $0.5$ apart, so we can test it with Simpson's rule over the whole
history: $E(2) - E(0)$ must equal the integral of $-3(\int p_3 - \int p_t)$ from
$0$ to $2$. This is fair only if each state is followed continuously from slice
to slice (no level crosses the Fermi level); the record of the history lists no
such crossing. The next cell does this for all 15 series (three particle numbers,
five couplings).
```

In [18]:

```
history = read_json("Revision/kohn_sham/results/adiabatic/history.json")
no_crossing = all(not series["fermiLevelCrossings"] for series in history["series"])
TAGS = ["lamm2", "lamm1", "lam0", "lamp1", "lamp2"]  # the five couplings
simpson_error, energies, drives = {}, {}, {}
for n in (8, 136, 688):
    for tag in TAGS:
        ids = [f"N{n}_{tag}_{s}" for s in SLICES]
        E = np.array([number(sid, "int_rho") for sid in ids])  # E at the slices
        drive = np.array([-3 * (number(sid, "int_p3") - number(sid, "int_p_t"))
                          for sid in ids])  # dE/da4 at the slices
        simpson = 0.5 / 3 * (drive[0] + 4 * drive[1] + 2 * drive[2]
                             + 4 * drive[3] + drive[4])  # step 0.5, from 0 to 2
        energies[(n, tag)], drives[(n, tag)] = E, drive
        change = E[-1] - E[0]
        simpson_error[(n, tag)] = (abs(simpson - change) / abs(change)
                                   if change != 0.0 else abs(simpson))
moving = [key for key in simpson_error if key[0] != 8]
largest = max(simpson_error[key] for key in moving)
unchanged = all(np.all(energies[(8, tag)] == energies[(8, tag)][0])
                and np.all(drives[(8, tag)] == 0.0) for tag in TAGS)
for n in (136, 688):
    E, error = energies[(n, "lam0")], simpson_error[(n, "lam0")]
    say(f"N = {n}, lambda = 0: E(0) = {E[0]:.6g}, E(2) = {E[-1]:.6g}, "
        f"change {E[-1] - E[0]:.6g}, relative Simpson error {error:.1e}")
report("largest relative Simpson error, N = 136 and 688", f"{largest:.1e}")
```

```
reproduces(no_crossing and largest < 1e-3 and unchanged,
           "E(2) - E(0) = integral of -3 (int p3 - int p_t) along the history",
           KS_RUST, ["emt_energy_change_dE_da4"])
```

Out [18]:

```
N = 136, lambda = 0: E(0) = 80.2822, E(2) = 12.4451, change -67.8372, relative Simpson
    error 2.1e-04
N = 688, lambda = 0: E(0) = 680.441, E(2) = 110.387, change -570.055, relative Simpson
    error 2.2e-04
RESULT largest relative Simpson error, N = 136 and 688 = 2.2e-04
PASS E(2) - E(0) = integral of -3 (int p3 - int p_t) along the history
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    emt_energy_change_dE_da4
```

Text cell:

```
The next cell draws the energy along the history for $N = 136$ and $N = 688$
with $\lambda = 0$, divided by its value at $a_{4,0} = 0$ (left), together with
the values predicted from $E(0)$ by Simpson's rule (crosses at $a_{4,0} = 1$ and
$2$) and the constant energy that C3 would need; and the relative Simpson error
of all ten moving series (right). The $N = 8$ series do not move: their
pressures obey $p_3 = p_t$ exactly, so their energy is the same at every slice.
```

In [19]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.0))
slice_values = np.array(list(SLICES.values()))
for n, marker in ((136, "o"), (688, "s")):
    E, drive = energies[(n, "lam0")], drives[(n, "lam0")]
    to_one = E[0] + 0.5 / 3 * (drive[0] + 4 * drive[1] + drive[2])  # 0 to 1
    to_two = to_one + 0.5 / 3 * (drive[2] + 4 * drive[3] + drive[4])  # 1 to 2
    left.plot(slice_values, E / E[0], marker=marker, color=N_COLOURS[n], lw=1.6,
              label=f"$N = {n}$: recorded $E$")
    left.plot([1.0, 2.0], [to_one / E[0], to_two / E[0]], "x", color="black",
              ms=9, mew=1.6)
left.plot([], [], "x", color="black", label="from $E(0)$ by Simpson")
left.axhline(1.0, color="#e34948", ls="--", lw=1.2, label="C3: constant energy")
left.set_xlabel("slice $a_{4,0}$ (history $a_4 = Hx_4$)")
left.set_ylabel("$E(a_{4,0}) / E(0)$")
left.set_ylim(0.0, 1.1)
left.legend(fontsize=7, loc="lower left")
keys = [(n, tag) for n in (136, 688) for tag in TAGS]
labels = [f"{n}, {tag[3:]}" for n, tag in keys]  # N and the coupling tag
right.bar(range(len(keys)), [simpson_error[key] * 1e4 for key in keys],
          color=[N_COLOURS[n] for n, _ in keys])
right.set_xticks(range(len(keys)), labels, rotation=60, fontsize=7)
right.set_xlabel("series ($N$, coupling: m2 $= -\\lambda_2$, ..., p2 $= +\\lambda_2$)")
right.set_ylabel("relative Simpson error ($10^{-4}$)")
fig.tight_layout()
fall = [energies[(n, "lam0")][0] / energies[(n, "lam0")][-1] for n in (136, 688)]
```

```
save_figure(fig, "energy_exchange",
            "Left: the total energy $E = \\int\\rho$ of the Kohn-Sham states with "
            "$\\lambda = 0$ along the history $a_4 = Hx_4$, divided by its value at "
            "$a_{4,0} = 0$ (circles $N = 136$, squares $N = 688$; pure numbers), "
            "the values predicted from $E(0)$ by integrating "
            "$dE/da_4 = -3(\\int p_3 - \\int p_t)$ with Simpson's rule (crosses), "
            "and the constant energy that condition C3 of the linear member needs "
            f"(dashed). The energy falls by factors of {fall[0]:.2f} and "
            f"{fall[1]:.2f}: as 3-space inflates and the extra times deflate, the "
            "gas with $p_3 > p_t = 0$ gives up energy. Right: the relative error of "
            "Simpson's rule for $E(2) - E(0)$ in all ten moving series, in units of "
            f"$10^{{-4}}$ (largest ${tex_number(largest)}$): the energy-change law "
            "holds to "
            "the accuracy of the five slices.")
```

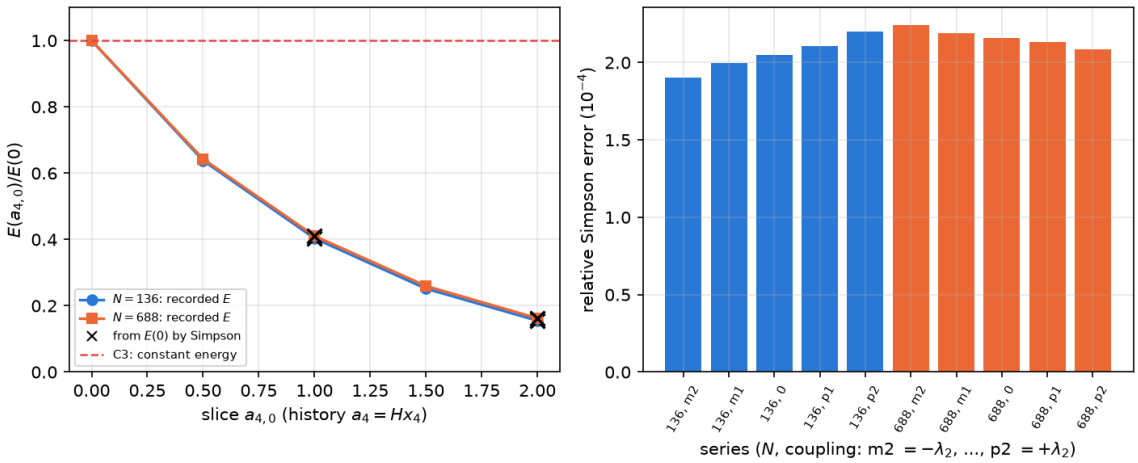

Figure 470: Left: the total energy $E = \int \rho$ of the Kohn-Sham states with $\lambda = 0$ along the history $a_4 = Hx_4$, divided by its value at $a_{4,0} = 0$ (circles $N = 136$, squares $N = 688$; pure numbers), the values predicted from $E(0)$ by integrating $dE/da_4 = -3(\int p_3 - \int p_t)$ with Simpson's rule (crosses), and the constant energy that condition C3 of the linear member needs (dashed). The energy falls by factors of 6.45 and 6.16: as 3-space inflates and the extra times deflate, the gas with $p_3 > p_t = 0$ gives up energy. Right: the relative error of Simpson's rule for $E(2) - E(0)$ in all ten moving series, in units of 10^{-4} (largest 2.2×10^{-4}): the energy-change law holds to the accuracy of the five slices. (Notebook 17b, figure 5.)

Out [19]:

```
Figure 17b.5 saved as Revision/textbook/figures/17b_5_energy_exchange.png
```

Text cell:

```
The energy law was tested above for the energy $E$ of the whole patch. Does it
also hold point by point, $\partial\rho/\partial a_4 = -3(p_3 - p_t)$ at every
$y$ (with $a_4' = H$ along the history)? The next cell takes the ten moving
series, estimates $\partial\rho/\partial a_4$ at the slice $a_{4,0} = 1$ at every
```

```
grid point with the fourth-order difference of section 3 in the variable $a_4$
(step $0.5$, the five slices), and compares it with $-3(p_3 - p_t)$ of that
slice: at three points, and integrated over the patch with the weight $e^{6Hy}$
(Simpson's rule in $y$). The record says that the instantaneous states carry no
energy flux along $y$ ($T^{x_4}{}_y = 0$), so nothing moves energy from one value
of $y$ to another: the total can be right while the pointwise law fails.
```

In [20]:

```
def change_with_a4(n, tag):
    """d rho / d a4 at the slice a4,0 = 1 at every y (fourth order, step 0.5)."""
    v = [profiles[f"N{n}_{tag}_{s}"]["rho"] for s in SLICES]  # rho at 5 slices
    return (v[0] - 8 * v[1] + 8 * v[3] - v[4]) / (12 * 0.5)


weight = np.exp(6 * H_value * y)  # the volume weight e^(6Hy) on the grid


def patch_integral(values):
    """Simpson's rule for the integral of e^(6Hy) times values over the patch."""
    f = weight * values
    return h / 3 * (f[0] + f[-1] + 4 * f[1:-1:2].sum() + 2 * f[2:-1:2].sum())


tip_ratio, integral_gap = {}, {}
for n, tag in moving:
    middle = profiles[f"N{n}_{tag}_a10"]  # the slice a4,0 = 1
    left_side = change_with_a4(n, tag)  # d rho / d a4 at every y
    right_side = -3 * (middle["p3"] - middle["p_t"])  # what the law needs there
    tip_ratio[(n, tag)] = float(left_side[0] / right_side[0])  # at y = -3
    integral_gap[(n, tag)] = float(abs(patch_integral(left_side)
                                       / patch_integral(right_side) - 1.0))
slice_one = profiles["N136_lam0_a10"]
rate136 = change_with_a4(136, "lam0")
for index, place in ((0, "-3 (tip)"), (75, "-1.5"), (150, "0 (brane)")):
    needed = -3 * (slice_one["p3"][index] - slice_one["p_t"][index])
    say(f"N = 136, lambda = 0, a4,0 = 1, y = {place}: d rho/d a4 = "
        f"{rate136[index]:.4g}, -3 (p3 - p_t) = {needed:.4g}")
report("ratio of the two sides at the tip, 10 series",
       f"{min(tip_ratio.values()):.3f} to {max(tip_ratio.values()):.3f}")
report("largest relative difference of the two patch integrals",
       f"{max(integral_gap.values()):.1e}")
reproduces(all(r < 0.0 for r in tip_ratio.values())
           and max(integral_gap.values()) < 5e-3,
           "the x4 law fails point by point, holds for the energy of the patch",
           KS_PY, ["emt_x4_component"])
```

Out [20]:

```
N = 136, lambda = 0, a4,0 = 1, y = -3 (tip): d rho/d a4 = 82.97, -3 (p3 - p_t) = -474.8
N = 136, lambda = 0, a4,0 = 1, y = -1.5: d rho/d a4 = -0.4505, -3 (p3 - p_t) = -1.497
N = 136, lambda = 0, a4,0 = 1, y = 0 (brane): d rho/d a4 = -0.002068, -3 (p3 - p_t) =
```

```
    -0.001174
RESULT ratio of the two sides at the tip, 10 series = -0.327 to -0.106
RESULT largest relative difference of the two patch integrals = 1.3e-03
PASS the x4 law fails point by point, holds for the energy of the patch
     reproduces Revision/kohn_sham/reports/ks-theory-python.json, check emt_x4_component
```

Text cell:

```
## 13. The equations averaged over the hidden direction (an approximation)

No recorded ground state meets C1, so the $a_4$ equations cannot hold point by point
with the Kohn-Sham source. The Revision record in the folder
‘Revision/field_equations_a4/ks_source‘ asks the next weaker question: multiply
every field equation by the volume weight $e^{6Hy}$ and integrate it over the
patch. Because the left-hand sides do not depend on $y$, each one becomes the
same equation with the source replaced by its weighted average $\bar X$ (its
**moment**). These moment equations are NECESSARY conditions: every exact
solution obeys them. The next cell computes the averages
$\bar X = \int X / (2\,\mathrm{Vol}_7 W)$, $W = (1 - e^{-6HL})/(6H)$, of every
state from the record’s integrals, compares $\bar\rho$ with the record’s table of
averages, and evaluates the averaged C2 (the $x_1 + x_5 - 2x_0$ moment) through
its relative defect
$|\bar p_3 + \bar p_t - 2\bar p_0|/(|\bar p_3| + |\bar p_t| + 2|\bar p_0|)$,
which is $0$ when the averaged C2 holds.
```

In [21]:

```
MOMENTS = "Revision/field_equations_a4/ks_source/results/ks-source-moments.csv"
W = (1 - tip_weight) / (6 * H_value)  # the integral of e^(6Hy) over the patch


def mean(sid, column):
    """The weighted average of a column, from the record’s integral."""
    return number(sid, f"int_{column}") / (2 * VOL7 * W)


with repository_file(MOMENTS).open(encoding="utf-8", newline="") as handle:
    moments = {row["id"]: row for row in csv.DictReader(handle)}
agree = max(abs(mean(sid, "rho") / float(moments[sid]["rho_bar"]) - 1.0)
            for sid in nonzero)
defect = {}
for sid in nonzero:
    p3b, ptb, p0b = (mean(sid, column) for column in ("p3", "p_t", "p0"))
    defect[sid] = abs(p3b + ptb - 2 * p0b) / (abs(p3b) + abs(ptb) + 2 * abs(p0b))
low, high = min(defect, key=defect.get), max(defect, key=defect.get)
report("largest relative difference of rho_bar from the record", f"{agree:.1e}")
report("defect of the averaged C2",
       f"{defect[low]:.6g} ({low}) to {defect[high]:.6g}")
detail = record_entry(KS_SOURCE, "C1_averaged_algebraic_condition_fails")["detail"]
reproduces(agree < 1e-6
           and f"[{defect[low]:.6g} ({low}), {defect[high]:.6g}]" in detail,
           "the averaged C2 fails by a defect of order 1 in all 70 states",
           KS_SOURCE, ["C1_averaged_algebraic_condition_fails"])
```

Out [21]:

```
RESULT largest relative difference of rho_bar from the record = 6.7e-08
RESULT defect of the averaged C2 = 0.414086 (N688_lamm2_a00) to 0.806379
PASS the averaged C2 fails by a defect of order 1 in all 70 states
     reproduces Revision/field_equations_a4/ks_source/reports/ks-source-a4.json, check
    C1_averaged_algebraic_condition_fails
```

Text cell:

```
Which moment equations can hold together? The Revision record proves (checks
‘A4_constraint_propagation_with_averaged_source‘, ‘C2_averaged_energy_relation‘
and ‘C3_x0_moment_inconsistent_with_history‘) that the $x_4$ moment (the
constraint) and the $x_1 - x_5$ moment (the evolution equation) are consistent,
because the averages obey $d\bar\rho/da_4 = -3(\bar p_3 - \bar p_t)$; every set
that contains the $x_0$ moment is not. Keeping the consistent pair and dropping
the $x_0$ and $x_1 + x_5 - 2x_0$ moments is an APPROXIMATION; its error is the
dropped residuals, which the record finds to be of order 1. In Einstein gravity
the constraint is $3a_4'^2 + 21H^2 + \Lambda = -\kappa\bar\rho(a_4)$. At $a_4 = 0$
with the initial rate $a_4'(0) = H$ it fixes $\Lambda = -24H^2 - \sigma_0H^2$,
where $\sigma_0 = \kappa\bar\rho(0)/H^2$ is the source strength; subtracting the
two gives the **first integral**

$$\left(\frac{a_4'}{H}\right)^2 = 1 + \frac{\sigma_0}{3}\left(1 -
\frac{\bar\rho(a_4)}{\bar\rho(0)}\right).$$

The next cell evaluates it for the series $N = 136$, $\lambda = 0$: at the slice
$a_4 = 2$ for $\sigma_0 = 10, 1, -1$; and between the slices with the cubic
Hermite interpolant of $\bar\rho$ whose slopes are the exact
$-3(\bar p_3 - \bar p_t)$, to find by bisection the turning point where $a_4'$
reaches $0$ for $\sigma_0 = -10$ and for $\Lambda = 0$ ($\sigma_0 = -24$). It
compares every number with the record's table of integrated cases.
```

In [22]:

```
CASES = "Revision/field_equations_a4/ks_source/results/ks-source-a4-cases.csv"
series = "N136_lam0"  # the canonical series
ids = [f"{series}_{s}" for s in SLICES]  # its five states
rho_bar = np.array([mean(sid, "rho") for sid in ids])
rho_slope = np.array([-3 * (mean(sid, "p3") - mean(sid, "p_t")) for sid in ids])


def rho_between(a):
    """rho_bar(a4) between the slices: cubic Hermite with the exact slopes."""
    k = min(int(a / 0.5), 3)  # the interval [0.5 k, 0.5 k + 0.5] that holds a
    t = (a - 0.5 * k) / 0.5  # the position inside it, from 0 to 1
    return ((2 * t**3 - 3 * t**2 + 1) * rho_bar[k]
            + (t**3 - 2 * t**2 + t) * 0.5 * rho_slope[k]
            + (-2 * t**3 + 3 * t**2) * rho_bar[k + 1]
            + (t**3 - t**2) * 0.5 * rho_slope[k + 1])
```

```
def rate_squared(a, sigma0):
    """(a4'/H)^2 at a4 = a from the Einstein first integral, a4'(0) = H."""
    return 1.0 + sigma0 / 3.0 * (1.0 - rho_between(a) / rho_bar[0])


def turning_point(sigma0):
    """The a4 in [0, 0.5] where (a4'/H)^2 reaches 0, by bisection."""
    low_end, high_end = 0.0, 0.5
    for _ in range(60):  # each step halves the interval
        centre = (low_end + high_end) / 2
        if rate_squared(centre, sigma0) > 0.0:
            low_end = centre
        else:
            high_end = centre
    return low_end


with repository_file(CASES).open(encoding="utf-8", newline="") as handle:
    cases = {(row["gravity"], row["series"], row["case"]): row
             for row in csv.DictReader(handle)}
rate_gap, turn_gap, halts = 0.0, 0.0, {}
for sigma0 in (10, 1, -1):
    ours = rate_squared(2.0, sigma0) ** 0.5  # a4'/H at a4 = 2
    theirs = float(cases[("einstein", series, f"sigma0 = {sigma0}")]["rate_a20"])
    rate_gap = max(rate_gap, abs(ours / theirs - 1.0))
    say(f"sigma0 = {sigma0:3d}: a4'/H at a4 = 2 is {ours:.6g} (record {theirs:.6g})")
for sigma0, case in ((-10, "sigma0 = -10"), (-24, "Lambda = 0")):
    halts[sigma0] = turning_point(sigma0)
    theirs = float(cases[("einstein", series, case)]["a4_end"])
    turn_gap = max(turn_gap, abs(halts[sigma0] / theirs - 1.0))
    say(f"{case}: the deflation halts at a4 = {halts[sigma0]:.6g} "
        f"(record {theirs:.6g})")
reproduces(rate_gap < 1e-8, "Einstein first integral: a4'/H at a4 = 2 as recorded",
           KS_SOURCE, ["D2_einstein_first_integral"])
reproduces(turn_gap < 1e-6, "turning points for sigma0 = -10 and Lambda = 0",
           KS_SOURCE, ["D3_lambda_zero_cases_halt"])
```

Out [22]:

```
sigma0 =  10: a4'/H at a4 = 2 is 1.95362 (record 1.95362)
sigma0 =   1: a4'/H at a4 = 2 is 1.1321 (record 1.1321)
sigma0 =  -1: a4'/H at a4 = 2 is 0.847549 (record 0.847549)
sigma0 = -10: the deflation halts at a4 = 0.397998 (record 0.397998)
Lambda = 0: the deflation halts at a4 = 0.149623 (record 0.149623)
PASS Einstein first integral: a4'/H at a4 = 2 as recorded
     reproduces Revision/field_equations_a4/ks_source/reports/ks-source-a4.json, check
    D2_einstein_first_integral
PASS turning points for sigma0 = -10 and Lambda = 0
     reproduces Revision/field_equations_a4/ks_source/reports/ks-source-a4.json, check
    D3_lambda_zero_cases_halt
```

Text cell:

```
The next cell draws the first integral: $a_4'/H$ against $a_4$ over the computed
range $0 \le a_4 \le 2$ for five source strengths, with the turning points and the
constant rate of the linear member $a_4 = Hx_4$ (the history on which the
Kohn-Sham states were computed).
```

In [23]:

```
grid = np.linspace(0.0, 2.0, 401)  # a4 from 0 to 2 in steps of 0.005
STRENGTHS = [(10, "#eb6834"), (1, "#f2a541"), (-1, "#5598e7"), (-10, "#1c5cab"),
             (-24, "#104281")]  # sigma0 and its colour
fig, ax = plt.subplots(figsize=(6.4, 4.4))
for sigma0, colour in STRENGTHS:
    values = np.array([rate_squared(a, sigma0) for a in grid])
    keep = values >= 0.0  # a4'^2 cannot be negative: the curve ends at the turning point
    label = ("$\\Lambda = 0$ ($\\sigma_0 = -24$)" if sigma0 == -24
             else f"$\\sigma_0 = {sigma0}$")
    x_values, rates = grid[keep], np.sqrt(values[keep])
    if sigma0 in halts:  # end the curve exactly at its turning point
        x_values, rates = np.append(x_values, halts[sigma0]), np.append(rates, 0.0)
    ax.plot(x_values, rates, color=colour, lw=1.8, label=label)
ax.plot(list(halts.values()), [0.0, 0.0], "kx", ms=8, mew=1.6,
        label="turning points ($a_4' = 0$)")
ax.axhline(1.0, color="#e34948", ls="--", lw=1.2,
           label="linear member $a_4 = Hx_4$")
ax.set_xlabel("$a_4$ (the extra times scale as $e^{-a_4}$)")
ax.set_ylabel("$a_4'/H$")
ax.legend(fontsize=7, loc="upper left")
end_rates = {s: rate_squared(2.0, s) ** 0.5 for s in (10, 1, -1)}
save_figure(fig, "first_integral",
            "The rate $a_4'/H$ of the deflation of the extra times against $a_4$ "
            "(pure numbers), from the first integral of the averaged $a_4$ "
            "equations of Einstein gravity with the Kohn-Sham source $N = 136$, "
            "$\\lambda = 0$ (an APPROXIMATION: the $x_0$ moments are dropped), "
            "initial rate $a_4'(0) = H$, for the source strengths "
            "$\\sigma_0 = \\kappa\\bar\\rho(0)/H^2 = 10, 1, -1, -10$ and for "
            "$\\Lambda = 0$ ($\\sigma_0 = -24$). The dashed line is the constant "
            "rate of the history $a_4 = Hx_4$. A positive $\\sigma_0$ speeds the "
            f"deflation up ($a_4'/H = {end_rates[10]:.3f}$ and "
            f"${end_rates[1]:.3f}$ at $a_4 = 2$), a negative one slows it down "
            f"(${end_rates[-1]:.3f}$ for $\\sigma_0 = -1$); for $\\sigma_0 = -10$ "
            f"and for $\\Lambda = 0$ it halts at $a_4 = {halts[-10]:.3f}$ and "
            f"${halts[-24]:.3f}$ (crosses). The source does not create the "
            "deflation: the initial rate is chosen.")
```

Out [23]:

```
Figure 17b.6 saved as Revision/textbook/figures/17b_6_first_integral.png
```

Text cell:

```
## 14. The last check
```

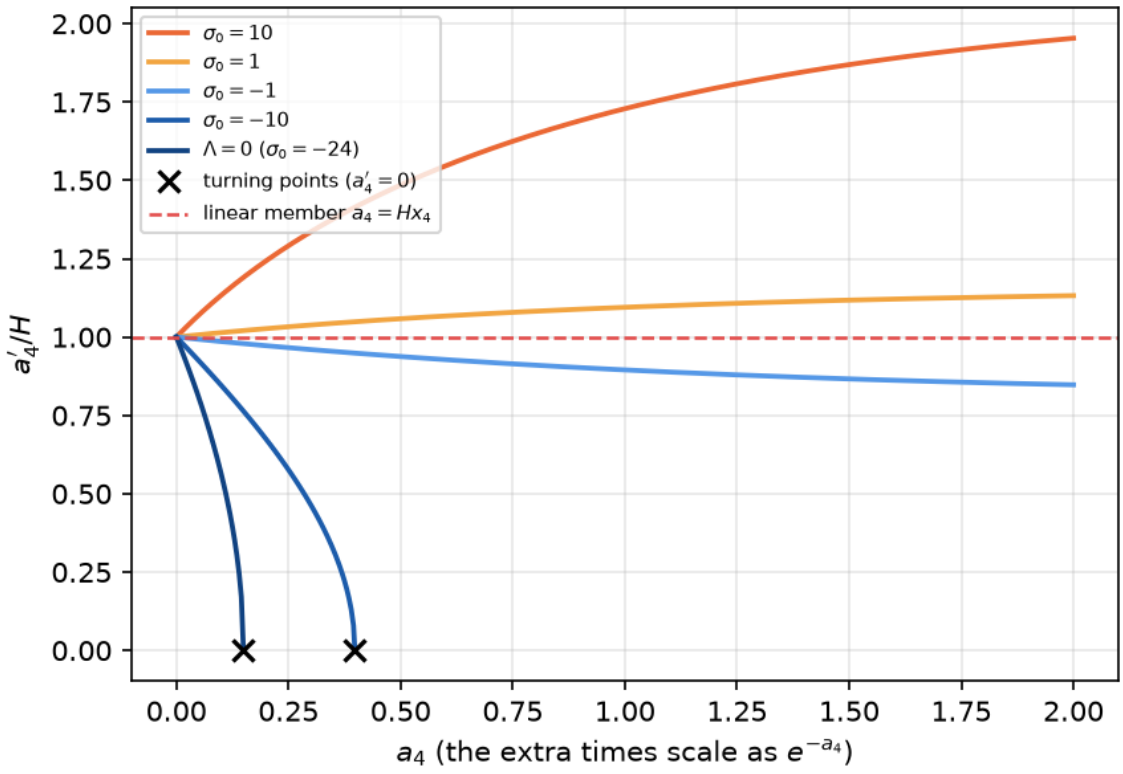

Figure 471: The rate a_4'/H of the deflation of the extra times against a_4 (pure numbers), from the first integral of the averaged a_4 equations of Einstein gravity with the Kohn-Sham source $N = 136$, $\lambda = 0$ (an APPROXIMATION: the x_0 moments are dropped), initial rate $a_4'(0) = H$, for the source strengths $\sigma_0 = \kappa\bar{\rho}(0)/H^2 = 10, 1, -1, -10$ and for $\Lambda = 0$ ($\sigma_0 = -24$). The dashed line is the constant rate of the history $a_4 = Hx_4$. A positive σ_0 speeds the deflation up ($a_4'/H = 1.954$ and 1.132 at $a_4 = 2$), a negative one slows it down (0.848 for $\sigma_0 = -1$); for $\sigma_0 = -10$ and for $\Lambda = 0$ it halts at $a_4 = 0.398$ and 0.150 (crosses). The source does not create the deflation: the initial rate is chosen. (Notebook 17b, figure 6.)

```
The last cell checks that the six figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [24]:

```
figure_names = ["eight_gammas", "slope_identity", "difference_order",
                "brane_and_mean", "energy_exchange", "first_integral"]
paths = [output_file(f"{FIGURE_FOLDER}/17b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [24]:

```
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 17b)
```

Text cell:

```
## 15. What this notebook showed

- COMPUTED and exact: the Kohn-Sham source rests on the author's eight REAL
  $16 \times 16$ gamma matrices (signed permutation matrices with the Clifford
  relation for all 64 pairs); the Rust solver read exactly this file (equal sha256
  digits).
- PROVED (sympy, equal to the $a_4$ record and to the Kohn-Sham theory record):
  in the author's metric a diagonal source is conserved when
  $\rho' = -3a_4'(p_3 - p_t)$ (time direction) and
  $p_0' + 6Hp_0 = 3H(p_3 + p_t)$ (hidden direction, coordinate $y$).
- PROVED: for a conserved source the violation of C2 is
  $p_3 + p_t - 2p_0 = p_0'(y)/(3H)$, so C2 is the statement "$p_0$ is flat"; and
  $d\mathcal C/dx_4 = 3a_4'\,\mathcal E$, so the linear member needs $p_3 = p_t$
  and a constant $\rho$.
- COMPUTED (the 75 recorded ground states): the Kohn-Sham states obey the conservation
  law of the hidden direction point by point (to about $10^{-4}$ of max|T| with
  fourth-order differences of step 0.02, the error falling with the fourth power
  of the step) and integrated over the patch (to about $10^{-11}$). The energy
  law of the time direction holds for the energy of the whole patch along the
  history (Simpson's rule, to about $2 \times 10^{-4}$), but NOT point by point:
  at the tip its two sides even have opposite signs. The instantaneous states
  carry no energy flux along $y$; a genuinely time-dependent state would need one
  ($q_{04} \neq 0$), and C1 forbids it.
- Hence the Kohn-Sham source is NOT ADMISSIBLE: its $p_0$ is far from flat (so
  C1 and C2 fail together) and its $p_3 > p_t$ makes its energy change along the
  history (so C3 fails). Conservation is necessary for a source of the $a_4$
  equations, not sufficient. The history $a_4 = Hx_4$ remains a PRESCRIBED
  BACKGROUND (ASSUMED), and the Kohn-Sham gas a test field without
  back-reaction.
- COMPUTED within an ASSUMED approximation (the Revision record `ks_source`,
  reproduced): averaged over the hidden direction, C2 still fails by a defect of
```

```
  order 1. Keeping only the consistent pair of moment equations (constraint and
  evolution) gives the first integral; in Einstein gravity the Kohn-Sham source
  changes a deflation that is
  already there by a bounded amount, and with $\Lambda = 0$ it halts it at
  $a_4 \approx 0.15$. It does not start or select the exponential deflation of
  the extra times: the initial rate is chosen.
- NOT shown (OPEN): what metric the Kohn-Sham gas would produce with
  back-reaction; whether any state of dirac16complex is an admissible source of
  the author's metric.
```

17.21 Line-by-line walk-through of Notebook 17b

The notebook has 24 code cells, In [1] to In [24]. As in Section 17.11, every line or small group of lines is quoted and explained, and each figure is described after the cell that draws it.

In [1], the set-up cell. It is the set-up cell of Notebook 17a, word for word, except for two things: the comment lines at the top hold the run instructions of Notebook 17b (Section 17.19), and the line `NOTEBOOK_ID = "17b"` names this notebook. Every line is explained in Section 17.11 under In [1]. The cell prints its one line, Set-up of notebook 17b complete: repository folder found, helpers defined.

In [2], the Revision records and the helpers.

```
import csv  # reads tables stored as CSV files (comma-separated values)
import hashlib  # computes the sha256 fingerprint of a file

import numpy as np  # arrays of numbers
import sympy as sp  # exact algebra with symbols
```

As in Notebook 17a, plus `hashlib`, a module of Python that computes fingerprints of data (In [4]).

```
GAMMAS = "Revision/algebra/gammas.json"  # the eight real 16 x 16 gammas
ALGEBRA = "Revision/algebra/reports/python-algebra.json"  # their checks
EQUATIONS = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
PY_A4 = "Revision/field_equations_a4/reports/python-a4-report.json"  # sympy
WL_A4 = "Revision/field_equations_a4/reports/wolfram-a4-report.json"  # Wolfram
SOURCE_REPORT = "Revision/field_equations_a4/reports/ks-source-conditions.json"
KS_SOURCE = "Revision/field_equations_a4/ks_source/reports/ks-source-a4.json"
KS_THEORY = "Revision/kohn_sham/ks-theory.json"  # the Kohn-Sham theory record
KS_PY = "Revision/kohn_sham/reports/ks-theory-python.json"  # its sympy checks
KS_RUST = "Revision/kohn_sham/reports/ks-rust-solver.json"  # the solver checks
GROUND = "Revision/kohn_sham/results/ground"  # the 75 Kohn-Sham ground states
```

Eleven names for the Revision records used below: the author's gamma matrices and the report of their checks; the record of the a_4 equations and its two reports; the record on the Kohn-Sham source conditions; the report of the a_4 equations with the Kohn-Sham source (Section 17.17); the Kohn-Sham theory record, its sympy report and the report of the Rust solver; and the folder of the recorded states.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_entry(report_file, name):
    """The entry {name, verdict, detail} of a check of a report, None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry
    return None


def reproduces(condition, name, report_file, record_names):
    """A check that also requires the named checks of the report to be PASS."""
    found = all((record_entry(report_file, n) or {}).get("verdict", "").upper()
                == "PASS" for n in record_names)  # some reports write "pass"
    names_text = ", ".join(record_names)  # the check names, separated by commas
    check(condition and found, name, record=f"{report_file}, check {names_text}")


def tex_number(value, digits=2):
    """A number for a caption in powers of ten: 0.0123 -> 1.2 \\times 10^{-2}."""
    mantissa, exponent = f"{value:.{digits - 1}e}".split("e")
    return f"{mantissa} \\times 10^{{{int(exponent)}}}"
```

The four helpers of Notebook 17a (Section 17.11, In [2]), with one change in `reproduces`: `.get("verdict", "")` returns an empty string when an entry has no verdict, and `.upper()` turns the verdict into capital letters, because the report of the algebra writes its verdicts as `pass` while the other reports write `PASS`.

```
REPORTS = [ALGEBRA, PY_A4, WL_A4, SOURCE_REPORT, KS_SOURCE, KS_PY, KS_RUST]
every_pass = True
for report_file in REPORTS:
    verdicts = [entry["verdict"].upper()
                for entry in read_json(report_file)["checks"]]
    passed = verdicts.count("PASS")  # how many checks of this report passed
    every_pass = every_pass and passed == len(verdicts)
    report(report_file.split("/")[-1], f"{passed} of {len(verdicts)} checks PASS")
check(every_pass, "every check of the seven Revision reports used here is PASS")
```

For each of the seven reports the cell collects the verdicts, counts the PASS among them (`.count`), keeps `every_pass` true only if all passed, and prints the file name (`split("/")[-1]`, the part after the last slash) with the count. Output: seven RESULT lines, one per report, each of the form "name of the report = n of n checks PASS", and the line PASS every check of the seven Revision reports used here is PASS. The numbers n are the numbers of checks that the reports held when the notebook was executed; they are printed in the notebook text above. The notebook does not require particular counts, because a report may gain checks when the Revision record is extended: the check asks only that every check of every report passes.

In [3], the eight gamma matrices are real signed permutation matrices.

```
gamma_record = read_json(GAMMAS)
entries = [v for matrix in gamma_record["gamma"] for row in matrix for v in row]
real_integers = all(isinstance(v, int) for v in entries)  # no complex entries
gamma = [np.array(matrix, dtype=float) for matrix in gamma_record["gamma"]]
eta = np.array(gamma_record["eta"], dtype=float)  # +1 or -1 for x0, ..., x7
```

The record `gammas.json` stores the eight matrices as lists of rows of numbers. The list comprehension with three `for` parts walks through every matrix, every row of it and every entry of the row, and collects all $8 \times 16 \times 16 = 2048$ entries. `isinstance(v, int)` is true when an entry is a whole number; a complex entry would be stored differently, so `real_integers` confirms that the matrices are REAL. Then each matrix becomes a numpy array of decimal numbers (`dtype=float`), and `eta` holds the eight signs of the frame metric.

```
shapes = sorted({matrix.shape for matrix in gamma})  # should be one shape only
values = sorted({int(v) for v in entries})  # the entries that occur
one_per_line = all(np.all(np.count_nonzero(matrix, axis=0) == 1)
                   and np.all(np.count_nonzero(matrix, axis=1) == 1)
                   for matrix in gamma)  # one nonzero entry per row and column
```

`matrix.shape` is the pair (rows, columns); a set keeps each different shape once. `values` is the sorted set of the different entries. `np.count_nonzero(matrix, axis=0)` counts the nonzero entries of every column and `axis=1` of every row; `one_per_line` is true when every column and every row of every matrix holds exactly one nonzero entry: then each matrix is a **signed permutation matrix**, which moves the 16 components of a spinor into a new order and changes some of their signs.

```
report("number of gamma matrices", len(gamma))
report("their shapes", shapes)
report("the entries that occur", values)
report("eta in the order x0, ..., x7", [int(v) for v in eta])
reproduces(len(gamma) == 8 and shapes == [(16, 16)] and real_integers
           and values == [-1, 0, 1] and one_per_line,
           "eight real 16 x 16 signed permutation matrices",
           ALGEBRA, ["reality_signed_permutations"])
```

Output: 8 matrices, the single shape (16, 16), the entries -1, 0, 1, $\eta = (1,1,1,1,-1,-1,-1,-1)$ in the order $x_0, \ldots, x_7$, and the PASS line, reproducing the algebra report's check `reality_signed_permutations`.

In [4], the Clifford relation and the fingerprint of the file.

```
identity16 = np.eye(16)  # the 16 x 16 unit matrix
deviation = 0.0  # the largest deviation from the Clifford relation
for a in range(8):
    for b in range(8):
        anticommutator = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
        target = 2.0 * eta[a] * identity16 if a == b else 0.0 * identity16
        deviation = max(deviation, float(np.max(np.abs(anticommutator - target))))
report("largest deviation from the Clifford relation, 64 pairs", deviation)
reproduces(deviation == 0.0, "the Clifford relation holds exactly for all 64 pairs",
           ALGEBRA, ["clifford_relation"])
```

`np.eye(16)` is the unit matrix. The two loops visit all $8 \times 8 = 64$ ordered pairs (a, b); `@` is matrix multiplication. The **anticommutator** $\gamma^a\gamma^b + \gamma^b\gamma^a$ is compared with $2\eta^{aa}$ times the unit matrix when $a = b$ and with the zero matrix otherwise, and `deviation` keeps the largest difference of any entry. Products and sums of matrices of whole numbers are exact in the computer, so the deviation must be exactly 0. Output: RESULT ... = 0.0 and the PASS line (algebra report, check `clifford_relation`).

```
fingerprint = hashlib.sha256(repository_file(GAMMAS).read_bytes()).hexdigest()
report("sha256 of gammas.json, first 16 digits", fingerprint[:16])
solver_detail = record_entry(KS_RUST, "gamma_fixture_numeric")["detail"]
reproduces(f"(sha256 {fingerprint[:16]})" in solver_detail,
           "the Rust Kohn-Sham solver read exactly this file of gammas",
           KS_RUST, ["gamma_fixture_numeric", "block_reduction_numeric"])
```

`read_bytes()` reads the file as raw bytes, `hashlib.sha256(...)` computes its fingerprint and `.hexdigest()` writes it as 64 hexadecimal digits (the digits 0 to 9 and the letters a to f). The Rust solver printed the first 16 digits of the fingerprint of the file it read into its check `gamma_fixture_numeric`; the check passes when the solver's text contains exactly ours, and when the solver's check `block_reduction_numeric` passed (it confirms that the solver's 2×2 blocks are the 16-component Hamiltonian written in another basis). Output: the 16 digits `c99277bab4793c42` and PASS the Rust Kohn-Sham solver read exactly this file of gammas. So the Kohn-Sham source of this chapter was computed with the author's eight real 16×16 matrices and no others.

In [5], the eight matrices as heat maps (figure 1).

```
LABELS = ["x_1", "x_2", "x_3", "x_4", "x_5", "x_6", "x_7", "x_0"]
fig, axes = plt.subplots(2, 4, figsize=(9.6, 5.4))
for a, ax in enumerate(axes.flat):
    image = ax.imshow(gamma[a], cmap="RdBu_r", vmin=-1.0, vmax=1.0)
    sign = "+1" if eta[a] > 0 else "-1"  # the square of this gamma matrix
    ax.set_title(f"$\\gamma^{{({LABELS[a]})}}$, square ${sign}$", fontsize=10)
```

A figure of two rows of four panels; `axes.flat` walks through the eight panels row by row, and `enumerate` numbers them $a = 0, \ldots, 7$ (the matrix of x_{a+1}). `imshow` draws the matrix with the colour scale `RdBu_r` (red, white, blue reversed: red for $+1$, white for 0, blue for -1), fixed between -1 and 1 by `vmin` and `vmax`. The title names the matrix and the sign of its square, $(\gamma^a)^2 = \eta^{aa}$ times the unit matrix.

```
    ax.set_xticks([0, 15], ["1", "16"])  # column numbers 1 and 16
    ax.set_yticks([0, 15], ["1", "16"])  # row numbers 1 and 16
    ax.tick_params(labelsize=7)  # small numbers, so that they do not collide
    ax.grid(False)  # no grid lines over the entries
fig.colorbar(image, ax=axes, shrink=0.75, label="matrix entry", ticks=[-1, 0, 1])
nonzero_count = int(sum(np.count_nonzero(matrix) for matrix in gamma))
```

Only the first and the last row and column are numbered (Python counts from 0, the book from 1); one colour bar serves all eight panels (`ax=axes`, shrunk to 75 per cent). `nonzero_count` counts the nonzero entries of all eight matrices: `np.count_nonzero` counts them in one matrix, and `sum` adds the eight counts.

```
save_figure(fig, "eight_gammas",
            "The author's eight real $16 \\times 16$ gamma matrices "
            "$\\gamma^{(x_0)}$ to $\\gamma^{(x_7)}$ as heat maps (rows 1 to 16 down, "
            "columns 1 to 16 across; red $+1$, blue $-1$, white $0$). Each row and "
            "each column holds exactly one nonzero entry, so the eight matrices "
            f"have {nonzero_count} nonzero entries in all; the squares are $+1$ "
            "for the space-like directions $x_0, x_1, x_2, x_3$ and $-1$ for the "
            "time $x_4$ and the deflating extra times $x_5, x_6, x_7$. These are "
            "the matrices with which the Kohn-Sham source of this chapter was "
            "computed.")
```

The caption is joined from its strings, as in Notebook 17a, In [6]; its one computed piece is `nonzero_count`, $8 \times 16 = 128$. Output: figure 17b.1 and its saved line.

What figure 17b.1 shows. Eight 16×16 tables, rows 1 to 16 downwards and columns 1 to 16 across, entries coloured red $(+1)$, blue (-1) or white (0). Every panel has exactly 16 coloured squares, one in each row and each column; the patterns and the signs differ from matrix to matrix (some lie along the diagonal from the lower left to the upper right, others in two blocks away from the diagonal). The squares are $+1$ for the space-like directions x_0, x_1, x_2, x_3 and -1 for the time x_4 and the deflating extra times x_5, x_6, x_7. The student should see that these are honest real matrices of whole numbers, the same that the Kohn-Sham source was computed with.

In [6], the hidden coordinate y, derived.

```
H = sp.symbols("H", positive=True)  # the constant H of the metric
X0 = sp.symbols("x0", positive=True)  # the author's hidden coordinate
z = 6 * H * X0  # the angle z = 6 H x0
y_of_x0 = sp.log(sp.sin(z)) / (6 * H)  # the hidden coordinate y
dy_dx0 = sp.simplify(sp.diff(y_of_x0, X0))  # chain rule
g_yy = sp.simplify(sp.cot(z) ** 2 / dy_dx0 ** 2)  # g00 times (dx0/dy)^2
say(f"dy/dx0 = {dy_dx0},  g_yy = {g_yy}")
```

Symbols for H and x_0, both positive; the angle z and $y = \ln(\sin z)/(6H)$ as sympy expressions. `sp.diff` differentiates with respect to x_0 (sympy applies the chain rule) and `sp.simplify` simplifies: the result is $\cot z$, printed as `1/tan(6*H*x0)`. The metric entry in the new coordinate is $g_{yy} = g_{00}\,(dx_0/dy)^2 = \cot^2 z / \cot^2 z = 1$ (rule: a length $\sqrt{g_{00}}\,dx_0$ is the same in both coordinates, and $dx_0 = dy/(dy/dx_0)$). Output: `dy/dx0 = 1/tan(6*H*x0),  g_yy = 1`.

```
s, c, a4_value = sp.symbols("s c a4", positive=True)  # s = sin z, c = cos z
y_of_s = sp.log(s) / (6 * H)  # y written with s = sin z
warp = sp.simplify(sp.exp(2 * H * y_of_s))  # e^{2Hy} in terms of s
say(f"e^(2Hy) = {warp}  (the warp factor sin^(1/3) z)")
```

To let sympy simplify powers safely, $\sin z$ and $\cos z$ are replaced by positive symbols s and c (both are positive on the patch), and a_4 by a positive symbol (only for this simplification; any value would do, since it cancels). Then $e^{2Hy} = e^{2H\ln(s)/(6H)} = e^{\ln(s)/3} = s^{1/3}$ (rule: $e^{\ln s} = s$ and $e^{k\ln s} = s^k$). Output: `e^(2Hy) = s**(1/3)`: the warp factor $\sin^{1/3} z$ is e^{2Hy}.

```
factors = ([sp.exp(2 * a4_value) * s ** sp.Rational(1, 3)] * 3 + [1]
           + [sp.exp(-2 * a4_value) * s ** sp.Rational(1, 3)] * 3)  # |g_ii|, i < 8
```

```
root_x0 = sp.simplify(sp.sqrt(sp.prod(factors) * (c / s) ** 2))  # g00 = cot^2 z
root_y = sp.simplify(sp.sqrt(sp.prod(factors) * 1))  # g_yy = 1
say(f"sqrt|det g| in the x0 chart = {root_x0};  in the y chart = {root_y}")
```

`factors` lists the absolute values of the seven diagonal entries other than g_{00} (`[x] * 3` repeats an entry three times; `sp.Rational(1, 3)` is the exact fraction 1/3). `sp.prod` multiplies them: $e^{6a_4}s \cdot 1 \cdot e^{-6a_4}s = s^2$. Times $g_{00} = \cot^2 z = (c/s)^2$ this is c^2, whose square root is $c = \cos z$; times $g_{yy} = 1$ it is s^2, whose square root is $s = \sin z = e^{6Hy}$. Output: `sqrt|det g| in the x0 chart = c; in the y chart = s`.

```
reproduces(sp.simplify(dy_dx0 - sp.cot(z)) == 0 and g_yy == 1
           and warp == s ** sp.Rational(1, 3) and root_x0 == c and root_y == s,
           "dy/dx0 = cot z, g_yy = 1, sin^(1/3) z = e^(2Hy), sqrt|g| = cos z or e^(6Hy)",
           KS_PY, ["geometry_hidden_coordinate", "geometry_sqrt_det",
                   "geometry_warped_form"])
```

The check collects the four results and reproduces three checks of the Kohn-Sham theory report. Output: the PASS line.

In [7], the covariant divergence of a general source in the coordinate x0.

```
def christoffel(metric, chart):
    """All Gamma^l_{mn} of a diagonal metric, as a nested list [l][m][n]."""
    inverse = [1 / metric[k, k] for k in range(8)]  # diagonal: g^kk = 1 / g_kk
    return [[[sp.simplify(inverse[l] * (sp.diff(metric[l, m], chart[n])
                                        + sp.diff(metric[l, n], chart[m])
                                        - sp.diff(metric[m, n], chart[l])) / 2)
              for n in range(8)] for m in range(8)] for l in range(8)]
```

`christoffel` computes all $8 \times 8 \times 8 = 512$ symbols $\Gamma^l{}_{mn}$ of a diagonal metric with the formula of Section 17.13, $\Gamma^l{}_{mn} = \frac{1}{2g_{ll}}(\partial_m g_{ln} + \partial_n g_{lm} - \partial_l g_{mn})$. `chart` is the list of the eight coordinates; `metric[l, m]` is an entry of the sympy matrix; the three nested list comprehensions build a list of lists of lists, so that `G[l][m][n]` is $\Gamma^l{}_{mn}$.

```
def divergence(T, metric, chart):
    """The 8 components nabla_mu T^mu_nu of a table T[mu, nu] = T^mu_nu."""
    G = christoffel(metric, chart)
    result = []
    for nu in range(8):
        total = 0
        for mu in range(8):
            total += sp.diff(T[mu, nu], chart[mu])  # d_mu T^mu_nu
            for lam in range(8):
                total += G[mu][mu][lam] * T[lam, nu]  # Gamma^mu_{mu lam} T^lam_nu
                total -= G[lam][mu][nu] * T[mu, lam]  # Gamma^lam_{mu nu} T^mu_lam
        result.append(sp.simplify(total))
    return result
```

`divergence` programs the definition of Section 17.13 term by term, without the shortcut of the lemma: for each ν it adds $\partial_\mu T^\mu{}_\nu$, adds $\Gamma^\mu{}_{\mu\lambda} T^\lambda{}_\nu$ and subtracts $\Gamma^\lambda{}_{\mu\nu} T^\mu{}_\lambda$ over all μ and λ (`+=` adds

to the running total, `-=` subtracts), simplifies, and appends the result. The table may be any 8×8 matrix, with mixed entries too.

```
coords = sp.symbols("x0:8", real=True)  # x0, ..., x7 (the x0 chart)
x4, x0 = coords[4], coords[0]
a4 = sp.Function("a4")(x4)  # the free function of the metric
zz = 6 * H * x0  # z = 6 H x0
wz = sp.sin(zz) ** sp.Rational(1, 3)  # the warp factor sin^(1/3) z
metric_x0 = sp.diag(*([sp.cot(zz) ** 2] + [sp.exp(2 * a4) * wz] * 3 + [-1]
                      + [-sp.exp(-2 * a4) * wz] * 3))  # x0, ..., x7 (b4 = 1)
```

`sp.symbols("x0:8")` makes the eight symbols `x0` to `x7` (the range 1:9 stops before 9); position 3 is x_4 and position 7 is x_0. `sp.Function("a4")(x4)` is an unknown function $a_4(x_4)$, which sympy differentiates as a_4'. `sp.diag(*list)` builds the diagonal matrix of the author's metric from the list of its eight entries (the star passes the entries one by one).

```
names = ("rho", "p3", "pt", "p0", "q40", "q04")
rho, p3, pt, p0, q40, q04 = (sp.Function(n)(x4, x0) for n in names)
T_x0 = sp.diag(p0, p3, p3, p3, -rho, pt, pt, pt)  # the diagonal part
T_x0[4, 0] = q40  # T^x4_x0
T_x0[0, 4] = q04  # T^x0_x4
div_x0 = divergence(T_x0, metric_x0, coords)
```

The six parts of the most general source are unknown functions of x_4 and x_0. The table is the diagonal matrix with the two mixed entries placed in row x_4, column x_0 and the reverse. `div_x0` is the list of the eight components of its divergence.

```
notation = {sp.Derivative(f, v): sp.Symbol(f"d{v.name[1]}{f.func.__name__}")
            for f in (rho, p3, pt, p0, q40, q04) for v in (x4, x0)}
notation[sp.Derivative(a4, x4)] = sp.Symbol("ad1")  # a4'


def in_record_notation(expression):
    """Write derivatives as d4rho, d0p0, ... and functions as plain symbols."""
    expression = expression.subs(notation)
    return expression.subs({f: sp.Symbol(f.func.__name__)
                            for f in (rho, p3, pt, p0, q40, q04)})
```

To print the result in the record's notation, each partial derivative is replaced by a short symbol: `v.name[1]` is the digit of the coordinate (4 or 8) and `f.func.__name__` the name of the function, so $\partial\rho/\partial x_4$ becomes `d4rho` and $\partial p_0/\partial x_0$ becomes `d0p0`; a_4' becomes `ad1`. `in_record_notation` makes these replacements and then writes each function as a plain symbol.

```
cc = sp.Symbol("cc")  # the record writes cot z as cc


def readable(expression):
    """A component as a sum of source symbols times simplified coefficients."""
    plain = sp.expand(in_record_notation(expression))
```

```
    sources = sorted(plain.free_symbols - {H, x0, sp.Symbol("ad1")}, key=str)
    total = sp.Integer(0)  # the sum, built term by term
    for symbol, factor in sp.collect(plain, sources, evaluate=False).items():
        factor = factor.subs(sp.sin(2 * zz), 2 * sp.sin(zz) * sp.cos(zz))
        factor = sp.simplify(factor.subs(sp.tan(zz), sp.sin(zz) / sp.cos(zz)))
        factor = sp.simplify(factor.subs(sp.sin(zz), sp.cos(zz) / cc))
        total += factor * symbol  # sin z = cos z / cot z
    return total.subs(sp.tan(zz), 1 / cc)
```

`readable` writes a component as a sum "coefficient times source quantity", with the coefficients in terms of `cc` $= \cot z$ as in the record. After multiplying out, `sources` is the sorted list of the source symbols that occur (all symbols except H, x_0 and `ad1`; `key=str` sorts by name). `sp.collect(..., evaluate=False)` returns a dictionary from each source symbol to its coefficient. Each coefficient is rewritten with $\sin 2z = 2\sin z\cos z$ and $\tan z = \sin z/\cos z$, simplified, and then written with $\sin z = \cos z/\cot z$, so that only $\cot z$ remains; the terms are added up, and a remaining $\tan z$ is written as $1/\cot z$.

```
for nu, component in enumerate(div_x0):
    say(f"nabla_mu T^mu_x{nu} = {readable(component)}")
```

The eight components are printed. Output: six zeros, and

`nabla_mu T^mu_x0 = 6*H*cc*p0 - 3*H*cc*p3 - 3*H*cc*pt + d0p0 + d4q40`,

`nabla_mu T^mu_x4 = -6*H*q04/cc - 3*ad1*p3 + 3*ad1*pt + d0q04 - d4rho`,

which are the two laws of Section 17.13 in the coordinate x_0 (since $1/\cot z = \tan z$).

In [8], comparison with the record.

```
equations = read_json(EQUATIONS)
RECORD_NAMES = {"ad1": sp.Derivative(a4, x4), "H": H, "cc": sp.cot(zz),
                "rho": rho, "p3": p3, "pt": pt, "p0": p0, "q40": q40, "q04": q04}
RECORD_NAMES.update({str(symbol): derivative
                     for derivative, symbol in notation.items()})
```

`RECORD_NAMES` translates the names of the record's text back into sympy objects: `ad1` into the derivative a_4', `cc` into $\cot z$, the source names into the functions, and (through `update` with the dictionary `notation` read backwards) `d4rho` into $\partial\rho/\partial x_4$ and so on.

```
def record_equation(text):
    """An equation left == right of the record as the expression left - right."""
    left, right = text.replace("^", "**").split("==")
    return (sp.sympify(left, locals=RECORD_NAMES)
            - sp.sympify(right, locals=RECORD_NAMES))
```

As `equation` in Notebook 17a: the record's text `left == right` becomes the expression left minus right.

```
same = []
for key, nu in (("conservation_x4", 4), ("conservation_x0", 0)):
    recorded = record_equation(equations["generalSource"][key]["input"])
    difference = sp.simplify((div_x0[nu] - recorded).rewrite(sp.cos))
    same.append(difference == 0)
    say(f"{key}: ours minus the record = {difference}")
zero_components = all(div_x0[nu] == 0 for nu in (1, 2, 3, 5, 6, 7))
reproduces(all(same) and zero_components,
           "our divergence equals the record: x4 and x0 laws, six zero components",
           PY_A4, ["conservation_components", "json_conservation"])
```

For the record's two laws (positions 3 and 7 are x_4 and x_0) the difference between our component and the record's is rewritten with cosines only (`.rewrite(sp.cos)`) and simplified; sympy finds 0 (one term needs the identity $\tan z + \cot z = 2/\sin 2z$, which the rewriting handles). Output: `conservation_x4:  ours minus the record = 0`, the same for `conservation_x0`, and the PASS line reproducing the sympy report's checks `conservation_components` and `json_conservation`. (PROVED.)

In [9], the conservation law in the coordinate y.

```
yc = sp.symbols("y", real=True)  # the hidden coordinate y
chart_y = [yc] + list(coords[1:])  # y (for x0), x1, ..., x7
wy = sp.exp(2 * H * yc)  # the warp factor e^{2Hy} = sin^(1/3) z
metric_y = sp.diag(*([1] + [sp.exp(2 * a4) * wy] * 3 + [-1]
                     + [-sp.exp(-2 * a4) * wy] * 3))
```

The coordinate y replaces x_0 (`coords[:7]` are the first seven coordinates); the metric is the warped form of Section 17.13, with $g_{yy} = 1$.

```
RHO, P3, PT, P0 = (sp.Function(n)(x4, yc) for n in ("rho", "p3", "pt", "p0"))
T_y = sp.diag(P0, P3, P3, P3, -RHO, PT, PT, PT)  # a diagonal source T(x4, y)
div_y = divergence(T_y, metric_y, chart_y)
law_x4 = -sp.diff(RHO, x4) - 3 * sp.diff(a4, x4) * (P3 - PT)
law_y = sp.diff(P0, yc) + 6 * H * P0 - 3 * H * (P3 + PT)
```

A diagonal source whose four parts are unknown functions of x_4 and y (capital names, to keep them apart from those of In [7]); its divergence with the same helper; and the two laws of Section 17.13 typed in by hand, for comparison.

```
derivatives = {sp.Derivative(RHO, x4): sp.Symbol("d4rho"),
               sp.Derivative(P0, yc): sp.Symbol("dyp0"),
               sp.Derivative(a4, x4): sp.Symbol("ad1")}  # short names
functions = {f: sp.Symbol(f.func.__name__) for f in (RHO, P3, PT, P0)}


def short(expression):
    """Write the derivatives as d4rho, dyp0, ad1 and the functions as symbols."""
    return expression.subs(derivatives).subs(functions)
```

Short names for printing, as in In [7]: `dyp0` is $\partial p_0/\partial y$.

```
say(f"nabla_mu T^mu_x4 = {short(div_y[4])}")
say(f"nabla_mu T^mu_y = {short(div_y[0])}")
others_zero = all(div_y[nu] == 0 for nu in (1, 2, 3, 5, 6, 7))
theory = read_json(KS_THEORY)["emt"]  # the energy-momentum part of the record
say("ks-theory.json, emt.conservationY: " + theory["conservationY"])
reproduces(sp.simplify(div_y[4] - law_x4) == 0 and sp.simplify(div_y[0] - law_y) == 0
           and others_zero
           and "6H p0 = 3H (p3 + p_t)" in theory["conservationY"],
           "in the y chart: the x4 law and dp0/dy + 6H p0 = 3H (p3 + p_t)",
           KS_PY, ["emt_y_conservation_selfconsistent", "emt_x4_component"])
```

The two surviving components are printed, the six others are confirmed to vanish, the record's statement of the y law is printed, and the check requires our components to equal the hand-typed laws and the record to state the same law. Output: `nabla_mu T^mu_x4 = -3*ad1*p3 + 3*ad1*pt - d4rho`, `nabla_mu T^mu_y = -3*H*p3 + 6*H*p0 - 3*H*pt + dyp0`, the record's line `p0' + 6H p0 = 3H (p3 + p_t) for every self-consistent state`, and the PASS line. (PROVED.)

In [10], the key identity.

```
p3_from_law = sp.solve(sp.Eq(law_y, 0), P3)[0]  # p3 from the y conservation law
V = P3 + PT - 2 * P0  # the violation of C2
V_conserved = sp.simplify(V.subs(P3, p3_from_law))
say(f"p3 from the conservation law: {short(p3_from_law)}")
say(f"V = p3 + p_t - 2 p0 for a conserved source: {short(V_conserved)}")
check(sp.simplify(V_conserved - sp.diff(P0, yc) / (3 * H)) == 0,
      "PROVED: for a conserved source V = p3 + p_t - 2 p0 = (dp0/dy) / (3H)")
```

`sp.Eq(law_y, 0)` is the equation "the y law equals 0", and `sp.solve(..., P3)` solves it for p_3; it returns a list of solutions, of which `[0]` takes the first (and only) one: $p_3 = 2p_0 - p_t + p_0'/(3H)$. Inserting it into V gives $p_0'/(3H)$, the derivation of Section 17.14. Output: `p3 from the conservation law: 2*p0 - pt + dyp0/(3*H)`, `V = p3 + p_t - 2 p0 for a conserved source:  dyp0/(3*H)` and the PASS line. (PROVED.)

In [11], the identity on the recorded states.

```
COLUMNS = ("rho", "p3", "p_t", "p0")  # the four diagonal components


def read_profile(path):
    """A profile table as a dictionary: column name -> numpy array (151 values)."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}


files = sorted(repository_file(f"{GROUND}/profiles").glob("*.csv"),
               key=lambda path: path.name)
profiles = {path.stem: read_profile(path) for path in files}
with repository_file(f"{GROUND}/emt-integrals.csv").open(
        encoding="utf-8", newline="") as handle:
    integrals = {row["id"]: row for row in csv.DictReader(handle)}
physics = read_json("Revision/kohn_sham/results/parameters.json")["physics"]
```

```
H_value, L_cut, VOL7 = physics["H"], physics["L_tipCutoff"], physics["Vol7"]
scale = {sid: float(max(np.max(np.abs(prof[c])) for c in COLUMNS))
         for sid, prof in profiles.items()}  # max|T| of every state
nonzero = [sid for sid in profiles if scale[sid] > 0.0]
y = profiles["N136_lam0_a10"]["y"]  # the common grid of y
h = float(y[1] - y[0])  # the grid step
```

These lines read the 75 profiles, the table of integrals and the parameters, and compute max|T| and the list of nonzero states, exactly as Notebook 17a did (Section 17.11, In [5]). The only new line is the grid step `h`, the distance between two neighbouring grid points, 0.02.

```
def slope(values, s=1):
    """dv/dy at the points i = 2s, ..., 150 - 2s (fourth order, step s h)."""
    v = values
    return (-v[4 * s:] + 8 * v[3 * s:-s] - 8 * v[s:-3 * s] + v[:-4 * s]) / (12 * h * s)
```

`slope` estimates the derivative of a tabulated function with the **fourth-order central difference** of step $\delta = sh$:

$$f'(y) \approx \frac{-f(y+2\delta) + 8f(y+\delta) - 8f(y-\delta) + f(y-2\delta)}{12\,\delta}.$$

Why fourth order? Expand each value in a Taylor series around y: $f(y \pm \delta) = f \pm \delta f' + \frac{\delta^2}{2} f'' \pm \frac{\delta^3}{6} f''' + \frac{\delta^4}{24} f'''' \pm \frac{\delta^5}{120} f^{(5)} + \ldots$, and the same with 2δ. Line by line:

$$8\big[f(y+\delta) - f(y-\delta)\big] = 16\delta f' + \tfrac{16}{6}\delta^3 f''' + \tfrac{16}{120}\delta^5 f^{(5)} + \ldots$$

(rule: in the difference the even powers cancel and the odd powers double)

$$-\big[f(y+2\delta) - f(y-2\delta)\big] = -4\delta f' - \tfrac{16}{6}\delta^3 f''' - \tfrac{64}{120}\delta^5 f^{(5)} - \ldots$$

(rule: the same with 2δ: $2 \cdot 2\delta = 4\delta$, $2(2\delta)^3/6 = 16\delta^3/6$, $2(2\delta)^5/120 = 64\delta^5/120$)

$$\text{sum} = 12\delta f' - \tfrac{48}{120}\delta^5 f^{(5)} + \ldots, \qquad \frac{\text{sum}}{12\delta} = f' - \frac{\delta^4}{30} f^{(5)} + \ldots$$

(rule: add; the δ^3 terms cancel; divide by 12δ, and $\frac{48}{120 \cdot 12} = \frac{1}{30}$). The error is proportional to δ^4: halving the step divides it by 16. In the code, for the array `v` of 151 values, `v[4*s:]` are the values $f(y_i + 2\delta)$ for the points $i = 2s, \ldots, 150 - 2s$, `v[3*s:-s]` the values $f(y_i + \delta)$, `v[s:-3*s]` the values $f(y_i - \delta)$ and `v[:-4*s]` the values $f(y_i - 2\delta)$; all four arrays have $151 - 4s$ entries, and the formula acts on them entry by entry. The first and last $2s$ points have no neighbours on one side and get no slope.

```
def violation(prof):
    """V(y) = p3 + p_t - 2 p0 of a profile."""
    return prof["p3"] + prof["p_t"] - 2.0 * prof["p0"]
```

```
residual = {sid: float(np.max(np.abs(slope(profiles[sid]["p0"]) / (3 * H_value)
                                     - violation(profiles[sid])[2:-2])))
            / scale[sid] for sid in nonzero}
worst = max(residual, key=residual.get)  # the state with the largest difference
```

`violation` returns $V(y)$ of a profile. For each nonzero state, `residual` is the largest difference between $p_0'/(3H)$ (from `slope` with $s = 1$) and V at the inner points (`[2:-2]` drops two points at each end, to match), divided by max|T|. `worst` is the state with the largest value.

```
report("states with a nonzero tensor", len(nonzero))
report("grid step h", f"{h:.2f}")
report("largest |(dp0/dy)/(3H) - V| / max|T| over 70 states", f"{residual[worst]:.2e}",
       f"({worst})")
check(len(nonzero) == 70 and residual[worst] < 1e-3,
      "V = (dp0/dy)/(3H) on every state to 1e-3 of max|T| (fourth-order differences)")
```

The third `report` passes the state's name in brackets as its third argument, the place of the unit, so that it is printed after the number. `:.2e` writes a number with two decimals in the computer's power-of-ten form, `1.05e-04` for 1.05×10^{-4}. Output: 70 states, $h = 0.02$, the largest difference 1.05×10^{-4} of max|T| (state `N8_lamm1_a00`), and the PASS line. (COMPUTED; the bound 10^{-3} was fixed before the run. The solver, on its own grid of 900 steps of $3/900 = 1/300$ (six times finer than the stored grid of 150 steps of 0.02), checks the same law with fourth-order differences to a relative residual of at most 1.951×10^{-8}: `Revision/kohn_sham/reports/ks-rust-solver.json`, check `emt_y_conservation_pointwise`; the number of steps is the record `Revision/kohn_sham/results/parameters.json`, key `numerics.rk4Steps`.)

In [12], the identity drawn (figure 2).

```
SLICES = {"a00": 0.0, "a05": 0.5, "a10": 1.0, "a15": 1.5, "a20": 2.0}  # a4,0
SHADES = ["#86b6ef", "#5598e7", "#2a78d6", "#1c5cab", "#104281"]  # light to dark
history136 = [f"N136_lam0_{tag}" for tag in SLICES]
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.0))
for shade, sid, a40 in zip(SHADES, history136, SLICES.values()):
    left.plot(y, profiles[sid]["p0"] / scale[sid], color=shade, lw=1.8,
              label=f"$a_{{4,0}} = {a40:g}$")
left.set_yscale("symlog", linthresh=1e-6)
left.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
left.set_ylabel("$p_0(y)$ / max|T|")
left.set_title("$N = 136$, $\\lambda = 0$: $p_0$ is far from flat")
left.legend(fontsize=7, loc="upper right")
```

The slices, shades and the history $N = 136$, $\lambda = 0$ as in Notebook 17a. The left panel draws $p_0 / \max|T|$ of the five states on a symmetric logarithmic axis, linear between -10^{-6} and 10^{-6}.

```
example = profiles["N136_lam0_a10"]
inner = y[2:-2]  # the points where the slope is known
right.plot(y, violation(example) / scale["N136_lam0_a10"], color="#2a78d6",
```

```
           lw=2.0, label="$V = p_3 + p_t - 2p_0$")
right.plot(inner[::5], (slope(example["p0"]) / (3 * H_value))[::5]
           / scale["N136_lam0_a10"], "o", color="#eb6834", ms=3.5,
           label="$p_0'(y)/(3H)$")
right.axhline(0.0, color="#e34948", ls="--", lw=1.0, label="C2: $V = 0$")
right.set_yscale("symlog", linthresh=1e-6)
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("divided by max|T|")
right.set_title("$a_{4,0} = 1$: the two sides of the identity")
right.legend(fontsize=7, loc="upper right")
fig.tight_layout()
```

The right panel draws, for the slice $a_{4,0} = 1$, the violation V as a blue line over all points and $p_0'/(3H)$ as orange dots at every fifth inner point (`[::5]` takes every fifth entry), both divided by max|T|, with the red dashed line $V = 0$ and the same kind of axis.

```
p0_range = [float(np.max(np.abs(profiles[s]["p0"])) / np.min(np.abs(profiles[s]["p0"])))
            for s in history136]  # how far p0 is from flat, per slice
```

`p0_range` holds, per slice, the largest $|p_0|$ divided by the smallest.

```
save_figure(fig, "slope_identity",
            "Left: the hidden-direction pressure $p_0(y)$ of the Kohn-Sham states "
            "$N = 136$, $\\lambda = 0$ at the five slices $a_{4,0} = 0$ to $2$, "
            "divided by the largest component of each state (horizontal axis: the "
            "hidden coordinate $y$; vertical axis symmetric logarithmic, linear "
            "between $-10^{-6}$ and $10^{-6}$); the largest $|p_0|$ is "
            f"{min(p0_range):.0f} to ${tex_number(max(p0_range))}$ times the "
            "smallest, while "
            "condition C2 of a conserved source demands a flat $p_0$. Right: for "
            "$a_{4,0} = 1$ the violation $V = p_3 + p_t - 2p_0$ (blue line) and the "
            "slope $p_0'(y)/(3H)$ from fourth-order differences (orange dots) lie "
            "on top of each other, as the conservation law demands; the dashed "
            "line is the value $0$ that C2 needs.")
```

The caption, joined from its strings, inserts the range of `p0_range`: the smallest with no decimals, 2933, and the largest in powers of ten, 2.3×10^6. The prime in the text $p_0'(y)$ is an ordinary character of a string written in double quotation marks. Output: figure 17b.2 and its saved line.

What figure 17b.2 shows. Left: horizontal axis y (units of $1/H$), vertical axis $p_0/\max|T|$ (a pure number, symmetric logarithmic). At the tip p_0 is negative and as large as max|T| (the -10^0 at the lower left); it rises steeply, crosses zero between $y \approx -2.6$ and $y \approx -2.35$, reaches a maximum and falls toward the brane by several powers of ten. For C2, by Section 17.14, every curve would have to be horizontal. Right: the blue line V and the orange dots $p_0'/(3H)$ lie on top of each other along the whole patch, positive on the tip side of the maximum of p_0 and negative on the brane side: the student sees the identity $V = p_0'/(3H)$ at work, and why V changes sign once.

In [13], the differences have order 4 (figure 3).

```
STEPS = np.array([1, 2, 3, 5, 6])  # the step is s h
POINTS = [30, 60, 90, 120]  # grid indices of y = -2.4, -1.8, -1.2, -0.6
```

```
STUDY = ["N8_lamp1_a00", "N136_lam0_a10", "N136_lamp2_a20", "N688_lam0_a10",
         "N688_lamm2_a00"]
```

Five step multipliers s; four grid points (index i is $y = -3 + 0.02\,i$: index 30 is $y = -2.4$, and so on); five states of all three particle numbers.

```
def difference_at_points(sid, s):
    """max over POINTS of |(dp0/dy)/(3H) - V| / max|T| with the step s h."""
    prof = profiles[sid]
    derivative = slope(prof["p0"], s) / (3 * H_value)  # index i - 2s is point i
    return max(abs(float(derivative[i - 2 * s] - violation(prof)[i]))
               for i in POINTS) / scale[sid]
```

For one state and one step: the slope array starts at the grid point $2s$, so its entry `i - 2*s` belongs to the grid point i; the function returns the largest difference $|p_0'/(3H) - V|$ at the four points, divided by max|T|.

```
errors = {sid: np.array([difference_at_points(sid, s) for s in STEPS])
          for sid in STUDY}
orders = {sid: float(np.polyfit(np.log(STEPS * h), np.log(errors[sid]), 1)[0])
          for sid in STUDY}  # the slope of the straight-line fit
for sid in STUDY:
    say(f"{sid}: differences " + ", ".join(f"{e:.1e}" for e in errors[sid])
        + f";  fitted order {orders[sid]:.2f}")
check(all(3.8 < order < 4.2 for order in orders.values()),
      "the differences fall like (step)^4: they are errors of the differences")
```

`errors` holds the five differences of each state. If the difference is an error of order 4, it is $C\delta^4$ with a constant C, so $\ln(\text{difference}) = \ln C + 4\ln\delta$: a straight line of slope 4 in the logarithms. `np.polyfit(x, y, 1)` fits a straight line $y \approx c_1 x + c_0$ by least squares and returns $[c_1, c_0]$; `[0]` takes the slope. Output: one line per state, for example `N136_lam0_a10:  differences 4.6e-07, 7.5e-06, 3.8e-05, 3.0e-04, 6.5e-04; fitted order 4.04`; the fitted orders are 4.08, 4.04, 4.02, 4.04 and 4.04; and the PASS line. (COMPUTED.) Had the conservation law failed, the differences would stop falling at the size of the failure; instead they fall like δ^4 down to 5×10^{-7} of max|T| at the smallest step.

```
fig, ax = plt.subplots(figsize=(6.4, 4.4))
for sid in STUDY:
    ax.loglog(STEPS * h, errors[sid], "o-", lw=1.4, ms=4,
              label=f"{sid} (order {orders[sid]:.2f})")
reference = errors["N136_lam0_a10"][0] * (STEPS / STEPS[0]) ** 4.0
ax.loglog(STEPS * h, reference, "k--", lw=1.0, label="slope 4 (reference)")
ax.set_xlabel("step of the difference $s h$ (units of $1/H$)")
ax.set_ylabel("max $|p_0'/(3H) - V|$ / max|T|")
ax.legend(fontsize=7)
```

`ax.loglog` draws with both axes logarithmic; `"o-"` means circles joined by lines, and each state's legend entry carries its fitted order. The black dashed reference line (the format string is the letter

k, for black, and two hyphens, for dashes) starts at the first point of the state `N136_lam0_a10` and grows like the fourth power of the step: `STEPS / STEPS[0]` is the step relative to the smallest one, and `** 4.0` raises it to the fourth power.

```
save_figure(fig, "difference_order",
            "The difference between $p_0'(y)/(3H)$, computed with fourth-order "
            "differences of step $sh$ ($h = 0.02$, $s = 1, 2, 3, 5, 6$), and the "
            "violation $V = p_3 + p_t - 2p_0$, at the four points $y = -2.4$, "
            "$-1.8$, $-1.2$, $-0.6$, divided by the largest component of the state, "
            "for five recorded Kohn-Sham states (logarithmic axes; the step in units "
            "of $1/H$). The fitted slopes lie between "
            f"{min(orders.values()):.2f} and {max(orders.values()):.2f}, the order "
            "4 of the differences: the small differences are errors of the finite "
            "differences, and the conservation law itself holds.")
```

The caption, joined from its strings, inserts the smallest and the largest fitted order with two decimals, 4.02 and 4.08. Output: figure 17b.3 and its saved line.

What figure 17b.3 shows. Horizontal axis: the step sh from 0.02 to 0.12 (units of $1/H$); vertical axis: the largest difference divided by max|T| (pure number); both logarithmic. Five straight lines, parallel to the dashed reference of slope 4: the differences are pure errors of the finite differences, and the conservation law itself holds for the recorded states.

In [14], the integrated law.

```
def number(sid, column):
    """A number of the record's table of integrals."""
    return float(integrals[sid][column])


tip_weight = float(np.exp(-6.0 * H_value * L_cut))  # e^{-6HL}, about 1.5e-8
integrated = {}
for sid in nonzero:
    left_side = 2 * VOL7 * (number(sid, "p0_brane") - tip_weight * number(sid, "p0_tip"))
    right_side = 3 * H_value * (number(sid, "int_p3") + number(sid, "int_p_t"))
    integrated[sid] = abs(left_side - right_side) / abs(right_side)
worst_integrated = max(integrated, key=integrated.get)
report("largest relative difference of the integrated law",
       f"{integrated[worst_integrated]:.3e} ({worst_integrated})")
```

`number` reads a number of the table. `tip_weight` is $e^{-6HL} = e^{-18}$. For each nonzero state the two sides of the integrated law of Section 17.15 are computed from the table's columns `p0_brane`, `p0_tip`, `int_p3` and `int_p_t`, and their relative difference is stored. Output: the largest relative difference is 1.421×10^{-11} (state `N8_lamm2_a00`).

```
column = {sid: number(sid, "ycons_integrated_rel") for sid in nonzero}
column_worst = max(column, key=column.get)  # the solver's own largest value
report("largest value of the column ycons_integrated_rel",
       f"{column[column_worst]:.3e} ({column_worst})")
solver = record_entry(KS_RUST, "emt_y_conservation_integrated")["detail"]
say("the solver record: " + solver)
```

The table also has a column `ycons_integrated_rel`: the solver's own relative difference of the same two sides, computed from its unrounded internal numbers and divided by the largest of the terms involved (the integral, the boundary term, the tip term and the size of the cancelling terms), not by the right side alone as our number is; so the two are close cousins, not the same quantity by definition. Its largest value is 1.422×10^{-11}, also at `N8_lamm2_a00`, against our 1.421×10^{-11}. The text of the solver's check `emt_y_conservation_integrated` is printed: 75 cases, worst value 1.422×10^{-11} (`N8_lamp2_a00`), tolerance 10^{-7}, no failures.

```
named = solver.split("worst value ")[1].split("(")[1].split(")")[0]  # its state
tie = abs(column[named] - column[column_worst]) < 1e-15  # mirror states tie
report("the state named by the record and its column value",
       f"{named}, {column[named]:.3e}")
reproduces(integrated[worst_integrated] < 1e-10 and tie
           and f"worst value {column[column_worst]:.3e}" in solver,
           "the integrated conservation law holds for all 70 states",
           KS_RUST, ["emt_y_conservation_integrated"])
```

The state named by the record is cut out of its text: after `worst value` , after the next `(`, before the next `)`. The record names `N8_lamp2_a00`, our maximum is `N8_lamm2_a00`: the two states with $N = 8$ and $\lambda = \pm\lambda_2$ have tensors of opposite sign and equal size (the record's table: for example $\int \rho = \mp 0.00286265$), so their relative differences are equal, and `tie` confirms it to 10^{-15}. The check passes when our largest difference is below 10^{-10}, the tie holds and the record's worst value equals the column's. Output: the state and its value, and the PASS line. (COMPUTED. The pair $\pm\lambda$ of these zero-mode states is a property of these particular recorded states; it is not one of the pairing theorems of Chapters 18 to 20.)

In [15], why the averaged ratio is not 1.

```
mean_p0 = {sid: number(sid, "int_p0") / (2 * VOL7 * (1 - tip_weight) / (6 * H_value))
           for sid in nonzero}  # the weighted mean of p0
ratio = {sid: (number(sid, "int_p3") + number(sid, "int_p_t"))
         / (2 * number(sid, "int_p0")) for sid in nonzero}  # as in Notebook 17a
boundary = {sid: (number(sid, "p0_brane") - tip_weight * number(sid, "p0_tip"))
            / (1 - tip_weight) for sid in nonzero}  # the boundary value b0
from_brane = {sid: boundary[sid] / mean_p0[sid] for sid in nonzero}  # R = b0/mean
brane_only = {sid: number(sid, "p0_brane") / mean_p0[sid] for sid in nonzero}
tip_share = {sid: 1.0 - brane_only[sid] / from_brane[sid] for sid in nonzero}
```

Six dictionaries: the weighted mean $\bar{p}_0$ of Section 17.7; the ratio R as Notebook 17a computed it; the boundary value $b_0 = [p_0(0) - e^{-6HL} p_0(-L)]/(1 - e^{-6HL})$ of Section 17.15; R computed the other way, $b_0/\bar{p}_0$; the ratio that the brane value alone would give, $p_0(0)/\bar{p}_0$; and the share of R that the tip term supplies, $1 - p_0(0)/b_0$ (the quotient of the last two ratios, subtracted from 1).

```
step = float(y[1] - y[0])  # the grid step of y, 0.02


def simpson_rule(values):
    """Simpson's rule with the step of y (for an odd number of points)."""
    return step / 3 * (values[0] + values[-1] + 4 * values[1:-1:2].sum()
                       + 2 * values[2:-1:2].sum())
```

`step` is the distance between neighbouring points of the grid `y`, 0.02 (`float` turns the numpy number into an ordinary Python number). `simpson_rule` integrates a list of values given at the points of such a grid with **Simpson's rule**. On one pair of steps of width h the rule is $\int_a^{a+2h} f\,dy \approx \frac{h}{3}[f(a) + 4f(a+h) + f(a+2h)]$: the exact integral of the parabola through the three points (its error falls like h^4). For a grid with an even number of steps, that is an odd number of points $f_0, f_1, \ldots, f_{2n}$, the pairs are added: every inner point with an odd index is the middle of a pair and gets the weight 4, and every inner point with an even index is the end of one pair and the start of the next and gets the weight $1 + 1 = 2$:

$$\int f\,dy \approx \frac{h}{3}\big[f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 2f_{2n-2} + 4f_{2n-1} + f_{2n}\big].$$

In the code, `values[0]` and `values[-1]` are the first and the last value (the index -1 counts from the end); `values[1:-1:2]` takes every second value from index 1 up to, but not including, the last one (the odd indices); `values[2:-1:2]` does the same from index 2 (the even inner indices); `.sum()` adds the values taken.

```
last_unit = y >= -1.0 - 1e-9  # the last unit before the brane: 51 points
near_share, table_gap = {}, {}
for sid in nonzero:
    weighted = np.exp(6 * H_value * y) * profiles[sid]["p0"]  # e^{6Hy} p0
    whole = simpson_rule(weighted)  # the integral over the whole patch
    near_share[sid] = simpson_rule(weighted[last_unit]) / whole
    table_gap[sid] = abs(2 * VOL7 * whole / number(sid, "int_p0") - 1.0)
```

`y >= -1.0 - 1e-9` compares every point of the grid with -1 and gives an array of `True` and `False` values, `True` for the 51 points $y = -1, -0.98, \ldots, 0$ (the tiny 10^{-9} makes sure that the point $y = -1$ counts even if the file stores it with a rounding error in the last digit). These 51 points have 50 steps, an even number, as Simpson's rule needs; the whole grid has 151 points and 150 steps. Two empty dictionaries are made, and the loop fills them for every nonzero state. `weighted` holds the 151 values of $e^{6Hy}p_0$; `whole` is its integral from $y = -3$ to 0. `weighted[last_unit]` keeps only the values at the points where `last_unit` is `True`, so `near_share` is the integral over $-1 \le y \le 0$ divided by the integral over the whole patch. Because the weighted mean $\bar{p}_0$ is this integral divided by a fixed number (Section 17.7), `near_share` is also the share of $\bar{p}_0$ that comes from the last unit before the brane. `table_gap` is a control: the table's `int_p0` is $2\,\mathrm{Vol}_7 \int e^{6Hy}p_0\,dy$ (Section 17.7), so `2 * VOL7 * whole` should equal it, and the relative difference is stored.

```
agreement = max(abs(from_brane[sid] / ratio[sid] - 1.0) for sid in nonzero)
closest = min(ratio, key=lambda sid: abs(ratio[sid] - 1.0))
report("largest relative difference of the two forms of R", f"{agreement:.1e}")
report("R closest to 1", f"{ratio[closest]:.6g} ({closest})")
report("largest relative difference, Simpson's integral of p0 and int_p0",
       f"{max(table_gap.values()):.1e}")
for n in (8, 136, 688):
    shares = [tip_share[sid] for sid in nonzero if sid.startswith(f"N{n}_")]
    report(f"share of R from the tip term, N = {n}",
           f"{min(shares):.2g} to {max(shares):.2g}")
    nears = [near_share[sid] for sid in nonzero if sid.startswith(f"N{n}_")]
    report(f"share of the mean of p0 from -1 <= y <= 0, N = {n}",
```

```
           f"{min(nears):.3f} to {max(nears):.3f}")
detail = record_entry(SOURCE_REPORT, "ks_integrals_violate_algebraic_condition")[
    "detail"]
reproduces(agreement < 1e-9
           and f"(closest to 1: {ratio[closest]:.6g} at {closest})" in detail,
           "R = b0 / mean of p0 (b0: brane value with the tip term), 70 states",
           SOURCE_REPORT, ["ks_integrals_violate_algebraic_condition"])
```

`agreement` is the largest relative difference of the two forms of R over all nonzero states. The third `report` prints the largest value of `table_gap`. The loop collects, for each particle number, the tip shares of its states (`startswith` tests the beginning of the name) and prints the smallest and the largest with two significant digits (`.2g`); then it does the same for the shares of the mean from the last unit, with three decimals (`.3f`). Output: 1.4×10^{-11}; the ratio closest to 1 is again 0.414328 (`N688_lamm2_a00`), the record's number; 4.7×10^{-9}, so the notebook's integral of the profile and the solver's table describe the same number; then, for each particle number, two lines: the tip shares, 0.31 to 0.33 for $N = 8$, 4.1×10^{-5} to 0.039 for $N = 136$ and 5.3×10^{-6} to 0.025 for $N = 688$, and the shares of the mean from $-1 \le y \le 0$, 0.439 to 0.441 for $N = 8$, 0.812 to 0.932 for $N = 136$ and 0.816 to 0.969 for $N = 688$; and the PASS line. (COMPUTED. So for the states with 3-momentum most of the mean of p_0 comes from the last unit before the brane and the tip term matters little; for the brane zero modes $N = 8$ less than half of the mean comes from there, and the tip term supplies about a third of R, Section 17.15.)

In [16], the boundary value against the mean (figure 4).

```
N_COLOURS = {8: "#1baf7a", 136: "#2a78d6", 688: "#eb6834"}
fig, ax = plt.subplots(figsize=(6.4, 5.0))
for n, colour in N_COLOURS.items():
    for positive in (True, False):
        ids = [sid for sid in nonzero if sid.startswith(f"N{n}_")
               and (mean_p0[sid] > 0) == positive]  # one sign at a time
        if not ids:
            continue
        ax.loglog([abs(mean_p0[sid]) for sid in ids],
                  [abs(boundary[sid]) for sid in ids], "o",
                  ms=5 if positive else 10,  # rings around the mirror states
                  color=colour, mfc=colour if positive else "none",
                  label=f"$N = {n}$" + ("" if positive else ", mean $< 0$"))
```

For each particle number and each sign of $\bar{p}_0$, the states of that kind are collected (`(mean > 0) == positive` selects one sign); an empty group is skipped. The points are drawn on logarithmic axes, which need positive numbers, so absolute values are used: horizontal $|\bar{p}_0|$, vertical the boundary value $|b_0|$ of In [15]. Their ratio is exactly R. States with a positive mean get small filled circles; states with a negative mean get larger rings (`mfc`, the marker face colour, is `"none"`, so only the outline is drawn).

```
line = np.array([1e-7, 1.0])  # the range of the diagonal
ax.loglog(line, line, "--", color="#e34948", lw=1.2,
          label="averaged C2: boundary value = mean")
ax.set_xlabel("weighted mean $|\\bar p_0|$ (units of $m^8$)")
ax.set_ylabel("boundary value $|b_0|$ (units of $m^8$)")
```

```
ax.legend(fontsize=8, loc="upper left")
negative = sorted(sid for sid in nonzero if mean_p0[sid] < 0)  # mean of p0 < 0
say("states with a negative mean of p0: " + ", ".join(negative))
expected = sorted(sid for sid in nonzero if sid.startswith("N8_lamp"))
```

The red dashed diagonal is where the boundary value equals the mean, the averaged C2 ($R = 1$). The cell prints the states with a negative mean, and `expected` lists the ten $N = 8$ states with $\lambda > 0$ (their names start with `N8_lamp`).

```
plotted = {sid: boundary[sid] / mean_p0[sid] for sid in nonzero}  # = R
save_figure(fig, "brane_and_mean",
            "The boundary value $b_0 = (p_0(0) - e^{-6HL}p_0(-L))/(1 - e^{-6HL})$ "
            "of the hidden-direction pressure against its weighted mean, for the "
            ...)
check(negative == expected and all(0.0 < plotted[sid] < 1.0 for sid in nonzero),
      "every point lies below the diagonal; mean of p0 < 0 only for N = 8, "
      "lambda > 0")
```

`plotted` holds the ratio of the vertical to the horizontal value of every point, $b_0/\bar{p}_0$, which is R. The caption is quoted shortened (it is printed in full under the figure); joined from its strings, it inserts the range of the plotted ratios with three decimals, 0.107 to 0.414, and the range that the brane value alone would give, 0.074 to 0.414 (`min` and `max` of the dictionaries `plotted` and `brane_only`). The check confirms that the states with a negative mean are exactly the expected ten, and that every plotted ratio lies strictly between 0 and 1 (Python allows the chained comparison `0.0 < r < 1.0`). Output: the list `N8_lamp1_a00` to `N8_lamp2_a20`, figure 17b.4 with its saved line, and then the PASS line (the check comes after the figure).

What figure 17b.4 shows. Both axes logarithmic, in units of m^8. The averaged C2 would put every point on the dashed diagonal. All 70 points lie below it: the boundary value of p_0 is between about a tenth and four tenths of its mean, the ratios $R = 0.107$ to 0.414 of Notebook 17a. For the $N = 136$ and $N = 688$ states the boundary value is practically the brane value; for the $N = 8$ states the brane value alone would be only 0.074 of the mean, and the tip term raises the ratio to about 0.11 (near the tip p_0 has the sign opposite to its mean, so $-e^{-6HL}p_0(-L)$ has the sign of the mean). The blue ($N = 136$) and orange ($N = 688$) points form two short chains, one point per slice (the couplings overlap); the 20 nonzero $N = 8$ states, equal at every slice, fall on only two places, where the filled points of $-\lambda_1$ and $-\lambda_2$ sit inside the rings of $+\lambda_1$ and $+\lambda_2$ (same size, opposite sign: for $+\lambda$ the mean and the brane value of p_0 are negative, while p_0 at the tip is positive).

In [17], the constraint propagates.

```
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
alpha = sp.symbols("alpha1:4", real=True)  # the Lovelock couplings
kappa, p3s, pts = sp.symbols("kappa p3 pt", real=True)  # kappa and the pressures
LOCALS = {"ad1": ad1, "ad2": ad2, "H": H, "alpha1": alpha[0], "alpha2": alpha[1],
          "alpha3": alpha[2]}
```

```
def parse(text):
    """A formula of the record (Wolfram Language text) as a sympy expression."""
    return sp.sympify(text.replace("^", "**"), locals=LOCALS)
```

New symbols: a_4' and a_4'', the three couplings (`"alpha1:4"` makes `alpha1`, `alpha2`, `alpha3`), κ and two pressures (named `p3s` and `pts` in the code, to keep them apart from the functions of In [7]). `parse` reads a formula of the record, as in Notebook 17a.

```
E44 = sum(alpha[k - 1] * parse(equations["lovelockTensors"][f"E{k}"]["x4x4"]["input"])
          for k in (1, 2, 3))  # sum_k alpha_k E_(k)^x4_x4
F = parse(equations["generalSource"]["evolution_F"]["input"])
rho_dot = -3 * ad1 * (p3s - pts)  # the x4 conservation law
constraint_dot = sp.diff(E44, ad1) * ad2 + kappa * rho_dot  # chain rule
evolution = ad2 * F - kappa * (p3s - pts)
```

`E44` is $e_{44} = \sum_k \alpha_k E_{(k)}{}^{x_4}{}_{x_4}$ from the record, `F` the record's F, `rho_dot` the energy-change law, `constraint_dot` the derivative $d\mathcal{C}/dx_4 = (de_{44}/da')\, a_4'' + \kappa\rho'$ of Section 17.16, and `evolution` the expression $\mathcal{E}$.

```
say(f"d/d(a4') of the x4 component: {sp.factor(sp.diff(E44, ad1))}")
say(f"3 a4' F: {sp.factor(3 * ad1 * F)}")
reproduces(sp.expand(constraint_dot - 3 * ad1 * evolution) == 0,
           "PROVED: dC/dx4 = 3 (da4/dx4) times the evolution equation, any alpha_k",
           WL_A4, ["constraint_propagation_bianchi"])
linear = sp.expand(constraint_dot.subs({ad2: 0, ad1: sp.Symbol("A") * H}))
say(f"along the linear member a4 = A H x4: dC/dx4 = {sp.factor(linear)}")
```

`sp.factor` writes an expression as a product of factors. The two printed lines show de_{44}/da' and $3a'F$; they are the same, $6a'(\alpha_1 - 24\alpha_2 a'^2 - 40\alpha_2 H^2 + 360\alpha_3 a'^4 + 432\alpha_3 a'^2 H^2 + 360\alpha_3 H^4)$, the fact derived by hand in Section 17.16. The check confirms $d\mathcal{C}/dx_4 - 3a_4'\mathcal{E} = 0$ for all couplings (PROVED; Wolfram report, check `constraint_propagation_bianchi`). Then $a_4'' = 0$ and $a_4' = AH$ are inserted. Output: the two factored lines, the PASS line and `along the linear member a4 = A H x4:  dC/dx4 = -3*A*H*kappa*(p3 - pt)`.

In [18], the energy exchange along the history.

```
history = read_json("Revision/kohn_sham/results/adiabatic/history.json")
no_crossing = all(not series["fermiLevelCrossings"] for series in history["series"])
TAGS = ["lamm2", "lamm1", "lam0", "lamp1", "lamp2"]  # the five couplings
simpson_error, energies, drives = {}, {}, {}
```

The solver's record of the history lists, for each of the 15 series, the places where a level crosses the Fermi level between slices; `not series["fermiLevelCrossings"]` is true when that list is empty. Comparing the energies of different slices is fair only if each state is followed continuously, so `no_crossing` must be true. `TAGS` are the coupling parts of the names; three empty dictionaries will collect the results.

```
for n in (8, 136, 688):
    for tag in TAGS:
        ids = [f"N{n}_{tag}_{s}" for s in SLICES]
        E = np.array([number(sid, "int_rho") for sid in ids])  # E at the slices
        drive = np.array([-3 * (number(sid, "int_p3") - number(sid, "int_p_t"))
```

```
                    for sid in ids])  # dE/da4 at the slices
        simpson = 0.5 / 3 * (drive[0] + 4 * drive[1] + 2 * drive[2]
                             + 4 * drive[3] + drive[4])  # step 0.5, from 0 to 2
        energies[(n, tag)], drives[(n, tag)] = E, drive
        change = E[-1] - E[0]
        simpson_error[(n, tag)] = (abs(simpson - change) / abs(change)
                                   if change != 0.0 else abs(simpson))
```

For each of the 15 series: the energies $E = \int \rho$ at the five slices; the rates $dE/da_{4,0} = -3(\int p_3 - \int p_t)$ of Section 17.16; and **Simpson's rule** for the integral of the rate from $a_{4,0} = 0$ to 2. On one pair of steps of width h, Simpson's rule is $\int_a^{a+2h} f \approx \frac{h}{3}[f(a) + 4f(a+h) + f(a+2h)]$ (the exact integral of the parabola through the three points; its error falls like h^4). With $h = 0.5$ on $[0, 1]$ and on $[1, 2]$ and the two added, the middle value $f(1)$ appears twice: $\frac{0.5}{3}[f_0 + 4f_1 + 2f_2 + 4f_3 + f_4]$. The results are stored under the key `(n, tag)` (a tuple can be a key of a dictionary). The relative error compares Simpson's integral with the recorded change $E(2) - E(0)$; when the change is zero (the $N = 8$ series) the absolute value is stored instead.

```
moving = [key for key in simpson_error if key[0] != 8]
largest = max(simpson_error[key] for key in moving)
unchanged = all(np.all(energies[(8, tag)] == energies[(8, tag)][0])
                and np.all(drives[(8, tag)] == 0.0) for tag in TAGS)
```

The ten series with $N = 136$ or 688 move; `largest` is their largest relative error. `unchanged` confirms that every $N = 8$ series has the same energy at every slice and zero rate (their $p_3 = p_t$, Notebook 17a, In [18]).

```
for n in (136, 688):
    E, error = energies[(n, "lam0")], simpson_error[(n, "lam0")]
    say(f"N = {n}, lambda = 0: E(0) = {E[0]:.6g}, E(2) = {E[-1]:.6g}, "
        f"change {E[-1] - E[0]:.6g}, relative Simpson error {error:.1e}")
report("largest relative Simpson error, N = 136 and 688", f"{largest:.1e}")
reproduces(no_crossing and largest < 1e-3 and unchanged,
           "E(2) - E(0) = integral of -3 (int p3 - int p_t) along the history",
           KS_RUST, ["emt_energy_change_dE_da4"])
```

Output: for $N = 136$, $\lambda = 0$: $E(0) = 80.2822$, $E(2) = 12.4451$, change -67.8372, relative Simpson error 2.1×10^{-4}; for $N = 688$: 680.441, 110.387, -570.055, 2.2×10^{-4}; the largest relative error of the ten moving series, 2.2×10^{-4}; and the PASS line, which also requires the solver's check `emt_energy_change_dE_da4` (the same law with tiny steps of a_4) to be PASS. (COMPUTED. The remaining error is that of Simpson's rule with only five slices, not of the law: the solver checks the law with steps of 0.002 in a_4 to 1.5×10^{-10}, measured as in Section 17.16.)

In [19], the energy along the history (figure 5).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.6, 4.0))
slice_values = np.array(list(SLICES.values()))
for n, marker in ((136, "o"), (688, "s")):
    E, drive = energies[(n, "lam0")], drives[(n, "lam0")]
    to_one = E[0] + 0.5 / 3 * (drive[0] + 4 * drive[1] + drive[2])  # 0 to 1
```

```
    to_two = to_one + 0.5 / 3 * (drive[2] + 4 * drive[3] + drive[4])  # 1 to 2
    left.plot(slice_values, E / E[0], marker=marker, color=N_COLOURS[n], lw=1.6,
              label=f"$N = {n}$: recorded $E$")
    left.plot([1.0, 2.0], [to_one / E[0], to_two / E[0]], "x", color="black",
              ms=9, mew=1.6)
```

For $N = 136$ (circles) and $N = 688$ (squares) with $\lambda = 0$, the left panel draws $E/E(0)$ at the five slices, and black crosses (`"x"`, with marker edge width `mew`) at the energies predicted from $E(0)$ by one Simpson step to $a_{4,0} = 1$ and a second to $a_{4,0} = 2$.

```
left.plot([], [], "x", color="black", label="from $E(0)$ by Simpson")
left.axhline(1.0, color="#e34948", ls="--", lw=1.2, label="C3: constant energy")
left.set_xlabel("slice $a_{4,0}$ (history $a_4 = Hx_4$)")
left.set_ylabel("$E(a_{4,0}) / E(0)$")
left.set_ylim(0.0, 1.1)
left.legend(fontsize=7, loc="lower left")
```

A legend entry for the crosses (an empty plot, as in Notebook 17a, In [15]), the red dashed line of the constant energy that C3 needs, labels, the vertical range and the legend.

```
keys = [(n, tag) for n in (136, 688) for tag in TAGS]
labels = [f"{n}, {tag[3:]}" for n, tag in keys]  # N and the coupling tag
right.bar(range(len(keys)), [simpson_error[key] * 1e4 for key in keys],
          color=[N_COLOURS[n] for n, _ in keys])
right.set_xticks(range(len(keys)), labels, rotation=60, fontsize=7)
right.set_xlabel("series ($N$, coupling: m2 $= -\\lambda_2$, ..., p2 $= +\\lambda_2$)")
right.set_ylabel("relative Simpson error ($10^{-4}$)")
fig.tight_layout()
```

The right panel draws one bar per moving series: its relative Simpson error times 10^4, coloured by N; the bars are labelled with N and the coupling tag (`tag[3:]` drops `lam`), turned by 60 degrees.

```
fall = [energies[(n, "lam0")][0] / energies[(n, "lam0")][-1] for n in (136, 688)]
```

`fall` is the factor by which the energy falls from the first to the last slice, for $N = 136$ and $N = 688$.

```
save_figure(fig, "energy_exchange",
            "Left: the total energy $E = \\int\\rho$ of the Kohn-Sham states with "
            "$\\lambda = 0$ along the history $a_4 = Hx_4$, divided by its value at "
            "$a_{4,0} = 0$ (circles $N = 136$, squares $N = 688$; pure numbers), "
            "the values predicted from $E(0)$ by integrating "
            "$dE/da_4 = -3(\\int p_3 - \\int p_t)$ with Simpson's rule (crosses), "
            "and the constant energy that condition C3 of the linear member needs "
            f"(dashed). The energy falls by factors of {fall[0]:.2f} and "
            f"{fall[1]:.2f}: as 3-space inflates and the extra times deflate, the "
            "gas with $p_3 > p_t = 0$ gives up energy. Right: the relative error of "
            "Simpson's rule for $E(2) - E(0)$ in all ten moving series, in units of "
            f"$10^{{-4}}$ (largest ${tex_number(largest)}$): the energy-change law "
            "holds to "
            "the accuracy of the five slices.")
```

The caption, joined from its strings, inserts the two factors with two decimals, 6.45 and 6.16, and the largest relative Simpson error of In [18] in powers of ten, 2.2×10^{-4} (`$10^{{-4}}$` in an f-string prints as 10^{-4}). Output: figure 17b.5 and its saved line.

What figure 17b.5 shows. Left: horizontal axis the slice $a_{4,0}$; vertical axis $E/E(0)$, a pure number. The recorded energies of $N = 136$ and $N = 688$ fall to about 0.4 at $a_{4,0} = 1$ and to about 0.16 at $a_{4,0} = 2$, and the crosses predicted from $E(0)$ by the energy-change law sit on them; C3 would need the dashed line at 1. As 3-space inflates and the extra times deflate, the gas, with $p_t = 0$ and $\int p_3 > 0$ (the caption's short form is $p_3 > p_t = 0$), gives up energy, exactly as the energy law of the patch demands. Right: the relative Simpson errors of the ten moving series, between about 1.9 and 2.2 in units of 10^{-4}: the energy-change law holds to the accuracy that five slices allow.

In [20], the time law tested point by point.

```
def change_with_a4(n, tag):
    """d rho / d a4 at the slice a4,0 = 1 at every y (fourth order, step 0.5)."""
    v = [profiles[f"N{n}_{tag}_{s}"]["rho"] for s in SLICES]  # rho at 5 slices
    return (v[0] - 8 * v[1] + 8 * v[3] - v[4]) / (12 * 0.5)
```

`change_with_a4` collects, for one series, the five profiles of ρ at the slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$ (each an array of 151 values, one per grid point y) and applies the fourth-order central difference of In [11] in the variable a_4, centred on the middle slice $a_{4,0} = 1$ with the step $\delta = 0.5$: $[f(0) - 8f(0.5) + 8f(1.5) - f(2)]/(12 \times 0.5)$, the formula of In [11] with $f(1 - 2\delta) = f(0)$, $f(1 - \delta) = f(0.5)$, $f(1 + \delta) = f(1.5)$ and $f(1 + 2\delta) = f(2)$ (the middle value $f(1)$ does not enter). Because the arrays are subtracted element by element, the result is $\partial\rho/\partial a_4$ at every grid point at once.

```
weight = np.exp(6 * H_value * y)  # the volume weight e^(6Hy) on the grid


def patch_integral(values):
    """Simpson's rule for the integral of e^(6Hy) times values over the patch."""
    f = weight * values
    return h / 3 * (f[0] + f[-1] + 4 * f[1:-1:2].sum() + 2 * f[2:-1:2].sum())
```

`weight` is e^{6Hy} at the 151 grid points. `patch_integral` is Simpson's rule on the grid of step $h = 0.02$ (150 intervals, an even number): the two end values once, the values at odd positions (`f[1:-1:2]`, every second value from index 1 to the last but one) four times, the values at even inner positions (`f[2:-1:2]`) twice, all times $h/3$.

```
tip_ratio, integral_gap = {}, {}
for n, tag in moving:
    middle = profiles[f"N{n}_{tag}_a10"]  # the slice a4,0 = 1
    left_side = change_with_a4(n, tag)  # d rho / d a4 at every y
    right_side = -3 * (middle["p3"] - middle["p_t"])  # what the law needs there
    tip_ratio[(n, tag)] = float(left_side[0] / right_side[0])  # at y = -3
    integral_gap[(n, tag)] = float(abs(patch_integral(left_side)
                                       / patch_integral(right_side) - 1.0))
```

For each of the ten moving series of In [18] (`moving`, $N = 136$ and 688): the left side $\partial\rho/\partial a_4$ and the right side $-3(p_3 - p_t)$ of the pointwise law at the slice $a_{4,0} = 1$ (along the history $a_4' = H = 1$,

so $\partial\rho/\partial x_4 = \partial\rho/\partial a_4$). `tip_ratio` stores their ratio at the first grid point, the tip $y = -3$ (index 0); `integral_gap` stores the relative difference of the two sides after both are integrated over the patch with the weight e^{6Hy}, which is the law for the energy of the patch.

```
slice_one = profiles["N136_lam0_a10"]
rate136 = change_with_a4(136, "lam0")
for index, place in ((0, "-3 (tip)"), (75, "-1.5"), (150, "0 (brane)")):
    needed = -3 * (slice_one["p3"][index] - slice_one["p_t"][index])
    say(f"N = 136, lambda = 0, a4,0 = 1, y = {place}: d rho/d a4 = "
        f"{rate136[index]:.4g}, -3 (p3 - p_t) = {needed:.4g}")
```

For the canonical state $N = 136$, $\lambda = 0$, the two sides are printed at three grid points: index 0 ($y = -3$), 75 ($y = -3 + 75 \times 0.02 = -1.5$) and 150 ($y = 0$), with four significant digits (`.4g`). Output: at the tip 82.97 against -474.8; in the middle -0.4505 against -1.497; at the brane -0.002068 against -0.001174.

```
report("ratio of the two sides at the tip, 10 series",
       f"{min(tip_ratio.values()):.3f} to {max(tip_ratio.values()):.3f}")
report("largest relative difference of the two patch integrals",
       f"{max(integral_gap.values()):.1e}")
reproduces(all(r < 0.0 for r in tip_ratio.values())
           and max(integral_gap.values()) < 5e-3,
           "the x4 law fails point by point, holds for the energy of the patch",
           KS_PY, ["emt_x4_component"])
```

Output: the tip ratios lie between -0.327 and -0.106 (negative in every series: the two sides have opposite signs), and the patch integrals agree to 1.3×10^{-3}. The check passes when every tip ratio is negative and the integrals agree to better than 5×10^{-3} (the accuracy that a difference with the step 0.5 allows), and the theory report's check `emt_x4_component`, which states the law for the energy of the 7-volume, is PASS. (COMPUTED; Section 17.16 explains the result.)

In [21], the hidden-direction averages and the averaged C2.

```
MOMENTS = "Revision/field_equations_a4/ks_source/results/ks-source-moments.csv"
W = (1 - tip_weight) / (6 * H_value)  # the integral of e^(6Hy) over the patch


def mean(sid, column):
    """The weighted average of a column, from the record's integral."""
    return number(sid, f"int_{column}") / (2 * VOL7 * W)
```

`MOMENTS` names the record's table of hidden-direction averages (one row per state). `W` is $\int_{-L}^{0} e^{6Hy}dy = (1 - e^{-6HL})/(6H)$ of Section 17.17. `mean` turns a column `int_X` of the table of integrals into the weighted mean $\bar{X} = \int X/(2\,\mathrm{Vol}_7 W)$ of Section 17.7.

```
with repository_file(MOMENTS).open(encoding="utf-8", newline="") as handle:
    moments = {row["id"]: row for row in csv.DictReader(handle)}
agree = max(abs(mean(sid, "rho") / float(moments[sid]["rho_bar"]) - 1.0)
            for sid in nonzero)
```

The record's table is read into a dictionary by state name, as the table of integrals was in In [11]. `agree` is the largest relative difference between our $\bar{\rho}$ and the record's column `rho_bar` over the 70 nonzero states (the record computes its averages with Simpson's rule on the profiles, we from the solver's integrals, so they agree only to the accuracy of that rule).

```
defect = {}
for sid in nonzero:
    p3b, ptb, p0b = (mean(sid, column) for column in ("p3", "p_t", "p0"))
    defect[sid] = abs(p3b + ptb - 2 * p0b) / (abs(p3b) + abs(ptb) + 2 * abs(p0b))
low, high = min(defect, key=defect.get), max(defect, key=defect.get)
```

For each state the three averaged pressures are computed (the parentheses make a **generator**, which produces the three values one after the other, and the three names receive them in order), and the relative defect of the averaged C2 of Section 17.17 is stored. `low` and `high` are the states with the smallest and the largest defect.

```
report("largest relative difference of rho_bar from the record", f"{agree:.1e}")
report("defect of the averaged C2",
       f"{defect[low]:.6g} ({low}) to {defect[high]:.6g}")
detail = record_entry(KS_SOURCE, "C1_averaged_algebraic_condition_fails")["detail"]
reproduces(agree < 1e-6
           and f"[{defect[low]:.6g} ({low}), {defect[high]:.6g}]" in detail,
           "the averaged C2 fails by a defect of order 1 in all 70 states",
           KS_SOURCE, ["C1_averaged_algebraic_condition_fails"])
```

Output: $\bar{\rho}$ agrees with the record to 6.7×10^{-8}; the defect runs from 0.414086 (`N688_lamm2_a00`) to 0.806379. The check passes when the averages agree to 10^{-6} and the text of the record's check contains exactly this range, written as the record writes it, `[0.414086 (N688_lamm2_a00), 0.806379]`. (COMPUTED.)

In [22], the first integral and the turning points.

```
CASES = "Revision/field_equations_a4/ks_source/results/ks-source-a4-cases.csv"
series = "N136_lam0"  # the canonical series
ids = [f"{series}_{s}" for s in SLICES]  # its five states
rho_bar = np.array([mean(sid, "rho") for sid in ids])
rho_slope = np.array([-3 * (mean(sid, "p3") - mean(sid, "p_t")) for sid in ids])
```

`CASES` names the record's table of integrated histories. For the series $N = 136$, $\lambda = 0$ the cell collects $\bar{\rho}$ at the five slices and its slope $d\bar{\rho}/da_4 = -3(\bar{p}_3 - \bar{p}_t)$ there (the averaged energy relation of Section 17.17).

```
def rho_between(a):
    """rho_bar(a4) between the slices: cubic Hermite with the exact slopes."""
    k = min(int(a / 0.5), 3)  # the interval [0.5 k, 0.5 k + 0.5] that holds a
    t = (a - 0.5 * k) / 0.5  # the position inside it, from 0 to 1
    return ((2 * t**3 - 3 * t**2 + 1) * rho_bar[k]
            + (t**3 - 2 * t**2 + t) * 0.5 * rho_slope[k]
            + (-2 * t**3 + 3 * t**2) * rho_bar[k + 1]
            + (t**3 - t**2) * 0.5 * rho_slope[k + 1])
```

`rho_between` is the cubic Hermite interpolant. `int(a / 0.5)` is the number of the interval of length 0.5 that contains a (`min(..., 3)` keeps $a = 2$ in the last interval), and `t` runs from 0 at its left end to 1 at its right end. The four cubic polynomials in `t` are the standard Hermite weights: at $t = 0$ they are $1, 0, 0, 0$ and at $t = 1$ they are $0, 0, 1, 0$, so the curve passes through both tabulated values; their derivatives are chosen so that the slope of the curve (with respect to a, hence the factor 0.5, the interval length) equals the given slopes at both ends. `t**3` is t^3.

```
def rate_squared(a, sigma0):
    """(a4'/H)^2 at a4 = a from the Einstein first integral, a4'(0) = H."""
    return 1.0 + sigma0 / 3.0 * (1.0 - rho_between(a) / rho_bar[0])
```

The first integral of Section 17.17, $(a_4'/H)^2 = 1 + (\sigma_0/3)(1 - \bar{\rho}(a_4)/\bar{\rho}(0))$.

```
def turning_point(sigma0):
    """The a4 in [0, 0.5] where (a4'/H)^2 reaches 0, by bisection."""
    low_end, high_end = 0.0, 0.5
    for _ in range(60):  # each step halves the interval
        centre = (low_end + high_end) / 2
        if rate_squared(centre, sigma0) > 0.0:
            low_end = centre
        else:
            high_end = centre
    return low_end
```

Bisection: start with an interval whose left end has $(a_4'/H)^2 > 0$ and whose right end has $(a_4'/H)^2 < 0$; test the centre and keep the half in which the sign still changes. After 60 halvings the interval is $0.5/2^{60} \approx 4 \times 10^{-19}$ long, far below the precision of the numbers. (`_` is the name Python programmers give a loop variable that is not used.)

```
with repository_file(CASES).open(encoding="utf-8", newline="") as handle:
    cases = {(row["gravity"], row["series"], row["case"]): row
             for row in csv.DictReader(handle)}
rate_gap, turn_gap, halts = 0.0, 0.0, {}
```

The record's table is read into a dictionary whose key is the triple (gravity, series, case), for example `("einstein", "N136_lam0", "sigma0 = 1")`. Two numbers will hold the largest relative differences from the record, and `halts` the turning points.

```
for sigma0 in (10, 1, -1):
    ours = rate_squared(2.0, sigma0) ** 0.5  # a4'/H at a4 = 2
    theirs = float(cases[("einstein", series, f"sigma0 = {sigma0}")]["rate_a20"])
    rate_gap = max(rate_gap, abs(ours / theirs - 1.0))
    say(f"sigma0 = {sigma0:3d}: a4'/H at a4 = 2 is {ours:.6g} (record {theirs:.6g})")
```

For $\sigma_0 = 10, 1, -1$ the rate at $a_4 = 2$ (`** 0.5` is the square root) is compared with the record's column `rate_a20`, the rate a_4'/H at the slice $a_4 = 2$ (`{sigma0:3d}` prints the integer in a field of three characters, so the lines line up). Output: 1.95362, 1.1321 and 0.847549, each equal to the record's value.

```
for sigma0, case in ((-10, "sigma0 = -10"), (-24, "Lambda = 0")):
    halts[sigma0] = turning_point(sigma0)
    theirs = float(cases[("einstein", series, case)]["a4_end"])
    turn_gap = max(turn_gap, abs(halts[sigma0] / theirs - 1.0))
    say(f"{case}: the deflation halts at a4 = {halts[sigma0]:.6g} "
        f"(record {theirs:.6g})")
```

For $\sigma_0 = -10$ and for $\Lambda = 0$ (which needs $\sigma_0 = -24$, Section 17.17) the turning point is found and compared with the record's column `a4_end`, where the record's integration stopped. Output: 0.397998 and 0.149623, equal to the record's values.

```
reproduces(rate_gap < 1e-8, "Einstein first integral: a4'/H at a4 = 2 as recorded",
           KS_SOURCE, ["D2_einstein_first_integral"])
reproduces(turn_gap < 1e-6, "turning points for sigma0 = -10 and Lambda = 0",
           KS_SOURCE, ["D3_lambda_zero_cases_halt"])
```

Two checks: the rates agree with the record to 10^{-8} and the turning points to 10^{-6} (relative), and the record's checks `D2_einstein_first_integral` and `D3_lambda_zero_cases_halt` are PASS. Output: two PASS lines. (COMPUTED, within the ASSUMED approximation of Section 17.17. The agreement is not a test of the physics of the approximation, only of its arithmetic: the notebook and the record use the same first integral and the same interpolant.)

In [23], the first integral drawn (figure 6).

```
grid = np.linspace(0.0, 2.0, 401)  # a4 from 0 to 2 in steps of 0.005
STRENGTHS = [(10, "#eb6834"), (1, "#f2a541"), (-1, "#5598e7"), (-10, "#1c5cab"),
             (-24, "#104281")]  # sigma0 and its colour
fig, ax = plt.subplots(figsize=(6.4, 4.4))
```

`np.linspace(0.0, 2.0, 401)` gives 401 equally spaced values of a_4 from 0 to 2; `STRENGTHS` pairs each source strength with a colour (warm for $\sigma_0 > 0$, blue for $\sigma_0 < 0$).

```
for sigma0, colour in STRENGTHS:
    values = np.array([rate_squared(a, sigma0) for a in grid])
    keep = values >= 0.0  # a4'^2 cannot be negative: the curve ends at the turning point
    label = ("$\\Lambda = 0$ ($\\sigma_0 = -24$)" if sigma0 == -24
             else f"$\\sigma_0 = {sigma0}$")
    x_values, rates = grid[keep], np.sqrt(values[keep])
    if sigma0 in halts:  # end the curve exactly at its turning point
        x_values, rates = np.append(x_values, halts[sigma0]), np.append(rates, 0.0)
    ax.plot(x_values, rates, color=colour, lw=1.8, label=label)
```

For each strength the cell computes $(a_4'/H)^2$ on the grid and keeps only the points where it is not negative (`keep` is an array of True and False; `grid[keep]` picks the values where it is True): beyond a turning point the first integral would give $(a_4'/H)^2 < 0$, which no real rate has; the record's integration stops at the turning point, and what follows it is not computed (Section 17.17). The label is written in LaTeX. For the two strengths with a turning point the point $(a_4^*, 0)$ is appended (`np.append`), so that the curve ends exactly there; then the curve of a_4'/H (the square root) is drawn.

```
ax.plot(list(halts.values()), [0.0, 0.0], "kx", ms=8, mew=1.6,
        label="turning points ($a_4' = 0$)")
ax.axhline(1.0, color="#e34948", ls="--", lw=1.2,
           label="linear member $a_4 = Hx_4$")
ax.set_xlabel("$a_4$ (the extra times scale as $e^{-a_4}$)")
ax.set_ylabel("$a_4'/H$")
ax.legend(fontsize=7, loc="upper left")
end_rates = {s: rate_squared(2.0, s) ** 0.5 for s in (10, 1, -1)}
```

Black crosses mark the two turning points; the red dashed line is the constant rate of the history $a_4 = Hx_4$; then the axis labels and the legend. `end_rates` holds the rates at $a_4 = 2$ for the caption.

```
save_figure(fig, "first_integral",
            "The rate $a_4'/H$ of the deflation of the extra times against $a_4$ "
            "(pure numbers), from the first integral of the averaged $a_4$ "
            ...)
```

The caption, quoted shortened (it is printed in full under the figure), inserts the rates 1.954, 1.132 and 0.848 and the turning points 0.398 and 0.150 with three decimals. Output: figure 17b.6 and its saved line.

What figure 17b.6 shows. Horizontal axis: a_4 from 0 to 2 (pure number; the extra-time scale factor is proportional to e^{-a_4}); vertical axis: a_4'/H (pure number). All five curves start at 1, the chosen initial rate. With $\sigma_0 = 10$ the rate grows to 1.954 at $a_4 = 2$, with $\sigma_0 = 1$ to 1.132, with $\sigma_0 = -1$ it falls to 0.848; the curves flatten toward the right because the gas has already given up most of its averaged energy there (only the lost fraction enters the first integral). With $\sigma_0 = -10$ and with $\Lambda = 0$ the rate drops steeply to 0 at $a_4 = 0.398$ and 0.150 (crosses): the deflation halts, and the curves end there because the integration stops at the turning point; what follows it is not computed. None of the curves is selected by the equations: the starting value 1 is a choice, and the dashed line, the history on which the Kohn-Sham states were computed, is a solution of the approximate equations only for a source that does not change, like the $N = 8$ states. (COMPUTED within the ASSUMED approximation of Section 17.17.)

In [24], the last check.

```
figure_names = ["eight_gammas", "slope_identity", "difference_order",
                "brane_and_mean", "energy_exchange", "first_integral"]
paths = [output_file(f"{FIGURE_FOLDER}/17b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

As In [20] of Notebook 17a, for the six figures. Output: PASS every figure file of this notebook exists and ALL 20 CHECKS PASSED (notebook 17b).

The count. The 20 PASS lines are: one each in In [2] and In [3], two in In [4], one each in In [6], In [8], In [9], In [10], In [11], In [13], In [14], In [15], In [16], In [17], In [18], In [20] and In [21], two in In [22], and one in In [24]. Fourteen of them reproduce a Revision record and name it.

17.22 What a prescribed background is, and what remains open

What the record establishes. In the words of the record `Revision/field_equations_a4/reports/ks-source-conditions.json`: no Kohn-Sham state recorded in `Revision/kohn_sham` is an admissible source of the author's metric; C1 and C2 fail for every nonzero state (C2 also after integration over x_0), and the Kohn-Sham history uses the linear member as a prescribed background, without back-reaction. The record of the a_4 equations says the same (`Revision/field_equations_a4/a4-equations.json`, key `fields.dirac16complex.kohnSham`), and so does the Kohn-Sham theory record (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.historyStatus`). Sections 17.12 to 17.16 add the reason: the Kohn-Sham source is conserved along the hidden direction at every point, and in the time direction for the total energy of the patch (not point by point); but its hidden pressure is far from flat and its 3-space pressure is not balanced by an equal extra-time pressure ($\int p_3 > \int p_t$ along the history), and for a conserved source these are exactly the failures of C2 and C3. Section 17.17 adds that averaging over the hidden direction does not rescue it: the averaged C2 still fails, and the averaged equations can be integrated only after two of them are dropped, an approximation whose error is of order 1.

What "prescribed background" means. A prescribed background is a metric chosen by hand, in which matter is placed and studied while its own gravity is ignored. The matter is then a **test field**, and the effect it would have on the metric, its **back-reaction**, is left out. This is a common and useful approximation in physics, much like computing the motion of a cork on a river whose flow is given: the cork follows the flow, and nobody asks how the cork changes the river. Quantum fields in a given expanding universe are usually studied this way. The approximation is good when the matter is too dilute to change the metric noticeably; whether that holds here cannot be decided without the back-reaction itself.

What follows for the other chapters. Every number computed along the history $a_4 = Hx_4$ in Chapters 14 to 16 (the Kohn-Sham levels, the energies and gaps, the temperatures, the energy densities, pressures and equations of state) is a property of a test gas in a chosen geometry. It is correctly computed (COMPUTED, with the accuracies stated there), but it is not a consequence of the coupled field equations of gravity and matter. Any equation of state derived from it, in particular for the dark-sector hypotheses discussed in Chapter 22, inherits this status: it describes the gas in the prescribed background, not a self-consistent universe.

What this chapter does NOT say.

- It does not say that the author's metric cannot be a solution of the field equations. Chapter 12 shows what source the linear member requires, and builds exact examples with a condensate of the commuting field dirac16complex00, whose pressures are equal and do not depend on x_0 (a computation of the book, Section 12.26).
- It does not say that no state of dirac16complex can be an admissible source. Only the 75 recorded ground states (zero temperature), whose profiles the record stores, are tested; the 135 thermal states of the canonical matrix (Chapter 15) store no profiles and are not tested. The record of the a_4 equations states what such a state would need: $p_3 + p_t = 2p_0$, every expectation value independent of x_0, and every off-diagonal expectation value zero (keys `fields.dirac16complex.kohnSham` and `fields.dirac16complex.offDiagonal`); for the linear member, in addition, equal expectation values of the 3-space and the extra-time kinetic terms (key `fields.dirac16complex.homogeneousSingleMode`). It also states that

no state with unequal kinetic terms that meets the other conditions, and so would drive a_4'', is constructed in the Revision record (key `fields.dirac16complex.evolution`); and the many-quantum states of dirac16complex that the Revision record does construct, the Kohn-Sham states, fail the conditions (this chapter). Whether an admissible state exists is OPEN.

- It does not say what metric the Kohn-Sham gas would produce. That needs back-reaction, which nobody has computed. It is OPEN.
- It says nothing about the creation of universes, about pairs of universes of masses $+m$ and $-m$, or about matter and antimatter. Those questions are treated, with the precise statement of what is proved and what is not, in Chapters 18 to 21.

What back-reaction would require. By C1, a source that depends on x_0 cannot sit on the right-hand side of the field equations of a metric whose left-hand side does not depend on x_0. A self-consistent treatment therefore needs a metric with more freedom: functions of both the time x_4 and the hidden coordinate (for example separate warp factors for 3-space and for the extra times that depend on y), the Kohn-Sham equations in that metric, and the field equations, all solved together. Beyond that, the Kohn-Sham states themselves are instantaneous (adiabatic) states; the time-dependent problem along the history is also OPEN. Chapter 22 describes how a student could attack both.

17.23 What we proved, what we computed, what we assumed

PROVED (exact; each with the record file and check, and the notebook cell that reproduces it):

statement	**where verified**	**notebook**
C1: the left-hand sides are diagonal and free of x_0, so a source must be independent of x_0 with $q_{40} = q_{04} = 0$	`wolfram-a4-report.json`: `P1_structure`, `P2_structure`, `P3_structure`, `independent_components`	17a, In [4]
C2: $p_3 + p_t = 2p_0$ for every source	`wolfram-a4-report.json`: `algebraic_identity_x1_plus_x5_minus_2x0`; `python-a4-report.json`: `algebraic_identity`	17a, In [4]
C3: the linear member needs $p_3 = p_t = p_0$ and constant ρ	`python-a4-report.json` and `wolfram-a4-report.json`: `linear_member_equal_pressures`	17a, In [4]
Einstein gravity: $T = 0$ has no solution for $H > 0$	`wolfram-a4-report.json`: `einstein_no_vacuum_solution`	17a, In [19]
the Bianchi identity: every source must be conserved	both reports: `E1_divergence_free`, `E2_divergence_free`, `E3_divergence_free`	(Section 17.12)

statement	where verified	notebook
the conservation law in the coordinates x_0 and y	`python-a4-report.json`: `conservation_components`, `json_conservation`; `ks-theory-python.json`: `emt_y_conservation_selfconsistent`, `emt_x4_component`	17b, In [7] to In [9]
for a conserved source $p_3 + p_t - 2p_0 = p_0'(y)/(3H)$	derived in Section 17.14	17b, In [10]
$d\mathcal{C}/dx_4 = 3a_4' \mathcal{E}$ for all couplings; along the linear member $-3AH\kappa(p_3 - p_t)$	`wolfram-a4-report.json`: `constraint_propagation_bianchi`; `python-a4-report.json`: `bianchi_x4`	17b, In [17]
the moment equations; $3F = 2\,dP/dX$; the constraint propagates with the averaged source; the Einstein first integral	`ks-source-a4.json`: `A1_lovelock_identity_x1_plus_x5_equals_2x0`, `A3_F_equals_two_thirds_dP_dX`, `A4_constraint_propagation_with_averaged_source`, `A6_x0_conservation_in_y`, `D2_einstein_first_integral`	(Section 17.17); 17b, In [22]

(The report files lie in `Revision/field_equations_a4/reports/`, `Revision/field_equations_a4/ks_source/reports/` and `Revision/kohn_sham/reports/`.)

COMPUTED (numerical, on the 75 recorded Kohn-Sham ground states; each reproduces the record named):

result	measured value	record and check	notebook
every nonzero state depends on x_0	spread of ρ at least 0.0497329 of $\max\|T\|$	`ks-source-conditions.json`: `ks_profiles_depend_on_x0`	17a, In [7]
C2 fails point by point	$\max\|V\|/\max\|T\|$ from 2.09192 to 3.99006	`ks_profiles_violate_algebraic_condition`	17a, In [9], In [12]
C2 fails after integration over x_0	R from 0.1072 to 0.4143 (closest to 1: 0.414328)	`ks_integrals_violate_algebraic_condition`	17a, In [14]
C3 fails along the history	$\int \rho$ falls by factors 6.45 and 6.16; $\int p_3 > \int p_t = 0$	`ks_history_is_a_prescribed_background`	17a, In [16], In [17]
five states have no source at all	$N = 8$, $\lambda = 0$, all slices	`ks_zero_source_states_listed`	17a, In [19]

result	measured value	record and check	notebook
the source rests on the author's real gammas	Clifford deviation exactly 0; sha256 prefix c99277bab4793c42	`python-algebra.json`: `reality_signed_permutations`, `clifford_relation`; `ks-rust-solver.json`: `gamma_fixture_numeric`	17b, In [3], In [4]
$V = p_0'/(3H)$ on the stored grid	at most 1.05×10^{-4} of $\max\|T\|$; fitted order 4.02 to 4.08	(the book's own test)	17b, In [11], In [13]
the integrated conservation law	relative difference at most 1.421×10^{-11}	`ks-rust-solver.json`: `emt_y_conservation_integrated`	17b, In [14]
R equals the boundary value b_0 of p_0 over its mean	agreement to 1.4×10^{-11}; tip share 0.31 to 0.33 for $N = 8$, at most 0.039 otherwise; share of $\bar{p}_0$ from $-1 \le y \le 0$: 0.439 to 0.441 for $N = 8$, at least 0.812 otherwise	`ks_integrals_violate_algebraic_condition`	17b, In [15]
the energy-change law of the patch along the history	Simpson's rule to at most 2.2×10^{-4}	`ks-rust-solver.json`: `emt_energy_change_dE_da4`	17b, In [18]
the time law fails point by point	tip ratio of the two sides -0.327 to -0.106; patch integrals agree to 1.3×10^{-3}	`ks-theory-python.json`: `emt_x4_component`	17b, In [20]
the averaged C2 fails	defect 0.414086 to 0.806379	`ks-source-a4.json`: `C1_averaged_algebraic_condition_fails`	17b, In [21]
first integral, Einstein, $N = 136$, $\lambda = 0$ (within the approximation)	$a_4'(2)/H = 1.95362$, 1.1321, 0.847549 for $\sigma_0 = 10, 1, -1$; halts at $a_4 = 0.397998$ ($\sigma_0 = -10$) and 0.149623 ($\Lambda = 0$)	`ks-source-a4.json`: `D2_einstein_first_integral`, `D3_lambda_zero_cases_halt`	17b, In [22]

ASSUMED.

- The history $a_4 = Hx_4$ ($A = 1$) on which the Kohn-Sham states were computed: it is a PRESCRIBED BACKGROUND, chosen and not solved for (this chapter shows that it is not a solution with the Kohn-Sham source).
- The ingredients of the Kohn-Sham model (Chapter 14): the good sector without extra-time dependence, the Z2 mirror at the brane, the tip cutoff $y = -3$, the box of 3-space and the

chosen particle numbers and couplings.

- In Section 17.17 only: the truncation of the averaged equations to the x_4 moment and the $x_1 - x_5$ moment (an approximation whose error, the dropped residuals, is of order 1), the initial rate $a_4'(0) = H$, and the interpolation of $\bar{\rho}$ between the slices.
- The overall sign convention $\sigma_T = +1$ of the source (record `a4-equations.json`, key `fields.emtConvention`). It does not affect C1, C2 or C3, which do not change when every component of T changes sign.

HYPOTHESIS. None in this chapter.

OPEN. What metric the Kohn-Sham gas would produce with back-reaction; whether any state of dirac16complex is an admissible source of the author's metric; the time-dependent (non-adiabatic) evolution of the gas along the history; the source beyond $a_4 = 2$ and at nonzero temperature; the evolution after a turning point (the record's integration stops there, Section 17.17); the six series with 3-momentum that are not integrated.

17.24 Exercises

Exercise 1 (the pattern of the Lovelock components). The record stores ${E_{(2)}}^{x_1}{}_{x_1} = 12a'^4 - 24a'^2a'' + 168a'^2H^2 - 40a''H^2 - 180H^4$, ${E_{(2)}}^{x_5}{}_{x_5} = 12a'^4 + 24a'^2a'' + 168a'^2H^2 + 40a''H^2 - 180H^4$ and ${E_{(2)}}^{x_0}{}_{x_0} = 12a'^4 + 168a'^2H^2 - 180H^4$. (a) Show that they have the form $P_2 + a''Q_2$, $P_2 - a''Q_2$, P_2, and find P_2 and Q_2. (b) Verify C2 for $k = 2$. (c) Find the Gauss-Bonnet part of F.

Answer. (a) In ${E_{(2)}}^{x_1}{}_{x_1}$ the terms without a'' are $12a'^4 + 168a'^2H^2 - 180H^4$, which is exactly ${E_{(2)}}^{x_0}{}_{x_0}$; call it P_2. The terms with a'' are $-24a'^2a'' - 40a''H^2 = a''(-24a'^2 - 40H^2)$ (rule: take out the common factor a''); call the bracket $Q_2 = -24a'^2 - 40H^2$. In ${E_{(2)}}^{x_5}{}_{x_5}$ the terms without a'' are again P_2, and the terms with a'' are $+24a'^2a'' + 40a''H^2 = -a''Q_2$. (b) $(P_2 + a''Q_2) + (P_2 - a''Q_2) - 2P_2 = 0$: the a'' terms cancel and $P_2 + P_2 - 2P_2 = 0$. (c) ${E_{(2)}}^{x_1}{}_{x_1} - {E_{(2)}}^{x_5}{}_{x_5} = 2a''Q_2 = a''(-48a'^2 - 80H^2)$, so the Gauss-Bonnet part of F is $\alpha_2(-48a'^2 - 80H^2)$, as in the record's `evolution_F`.

Exercise 2 (the two sides of C2 at three points). Use the three lines printed by Notebook 17a, In [12], for the state $N = 136$, $\lambda = 0$, $a_{4,0} = 1$. (a) Compute V in the middle and at the brane. (b) Compute $2p_0/(p_3 + p_t)$ at both points. (c) At the tip, compute $V/\max|T|$, given that the largest component of this state is $|p_0|$ at the tip (figure 2 of Notebook 17b shows $p_0/\max|T| = -1$ there).

Answer. (a) Middle: $V = 0.498835 + 0 - 2 \times 0.643324 = 0.498835 - 1.286648 = -0.787813$. Brane: $V = 0.000391237 + 0 - 2 \times 0.000942049 = 0.000391237 - 0.001884098 = -0.001492861$, printed as -0.00149286. (b) Middle: $1.286648/0.498835 = 2.579$, the 2.58 of figure 17a.5; brane: $0.001884098/0.000391237 = 4.816$, the 4.82. (c) $\max|T| = 431.733$, so $V/\max|T| = 1021.73/431.733 = 2.367$, inside the range 2.36 to 2.37 quoted in the caption of figure 17a.3.

Exercise 3 (the two forms of the hidden conservation law). Starting from the record's law in the coordinate x_0 for a diagonal source, $\partial_0 p_0 + 3H\cot z\,(2p_0 - p_3 - p_t) = 0$, derive the law in the coordinate y. Why may you divide by $\cot z$?

Answer. By the chain rule, $\partial_0 p_0 = (dy/dx_0)\,dp_0/dy = \cot z\,p_0'$, because $dy/dx_0 = \cot z$ (Section 17.4). The law becomes $\cot z\,p_0' + 3H\cot z\,(2p_0 - p_3 - p_t) = 0$, that is $\cot z\,[p_0' + 6Hp_0 - 3H(p_3 + p_t)] = 0$ (rule: take out the common factor; $3H \cdot 2p_0 = 6Hp_0$). On the patch $0 < z < \pi/2$ both $\cos z$ and $\sin z$ are positive, so $\cot z > 0$ and we may divide by it: $p_0' + 6Hp_0 = 3H(p_3 + p_t)$.

Exercise 4 (C2 and C1 for a conserved source). A conserved diagonal source in the author's metric has $p_0 = 5$ at every point (units of m^8, $H = 1$) and $p_t = 1$. (a) What is p_3? (b) Does it satisfy C2? (c) Can a conserved source with a flat p_0 still fail C1?

Answer. (a) The y law with $p_0' = 0$: $0 + 6 \times 5 = 3(p_3 + 1)$, so $p_3 + 1 = 10$ and $p_3 = 9$. (b) $p_3 + p_t = 10 = 2p_0$: yes, as Section 17.14 predicts for a flat p_0. (c) Yes. C1 also asks that ρ, p_3 and p_t do not depend on x_0. The hidden law fixes only the sum $p_3 + p_t$ and says nothing about ρ. For example $p_0 = 5$, $p_t = 1 + \sin y$ and $p_3 = 9 - \sin y$ satisfy the hidden law ($6 \times 5 = 3 \times 10$) and C2, but p_3 and p_t depend on y, so C1 fails. C2 is only the p_0 part of C1.

Exercise 5 (the averaged ratio by hand). For the state $N = 136$, $\lambda = 0$, $a_{4,0} = 0$ the record's table `Revision/kohn_sham/results/ground/emt-integrals.csv` gives $\int p_3 = 23.8133$, $\int p_t = 0$, $\int p_0 = 35.0435$, $p_0(\text{brane}) = 0.00224994$ and $p_0(\text{tip}) = -6.59995$; and $\text{Vol}_7 = 15875.2$, $H = 1$, $L = 3$. (a) Compute R from the integrals. (b) Compute the weighted mean $\bar{p}_0$. (c) Compute R from the brane and tip values and the mean, with the formula of Section 17.15. (d) Check the integrated conservation law.

Answer. (a) $R = (23.8133 + 0)/(2 \times 35.0435) = 23.8133/70.0870 = 0.33977$, the 0.339767 of the record. (b) The volume factor is $2\,\text{Vol}_7(1 - e^{-18})/6 = 31750.4 \times (1 - 1.5 \times 10^{-8})/6 = 5291.73$ (the e^{-18} changes nothing at this precision), so $\bar{p}_0 = 35.0435/5291.73 = 0.0066223$. (c) $e^{-18} \times (-6.59995) = -1.005 \times 10^{-7}$, so the numerator is $0.00224994 + 0.0000001005 = 0.00225004$, and $R = 0.00225004/0.0066223 = 0.33977$: the same number, as the conservation law demands. The brane value is about a third of the mean. (d) Left side: $2\,\text{Vol}_7 \times 0.00225004 = 31750.4 \times 0.00225004 = 71.440$; right side: $3H \times 23.8133 = 71.440$. They agree to the digits given.

Exercise 6 (every conserved profile with a constant sum). Suppose $p_3 + p_t = P$ is a constant. (a) Find every $p_0(y)$ that satisfies the y law. (b) Which of them satisfies C2? (c) Write the answer in the coordinate z.

Answer. (a) The law is $p_0' + 6Hp_0 = 3HP$. Multiply by e^{6Hy}: $(e^{6Hy}p_0)' = 3HPe^{6Hy}$ (rule: the product rule read backwards). Integrate: $e^{6Hy}p_0 = 3HP\,e^{6Hy}/(6H) + c = \frac{P}{2}e^{6Hy} + c$ with a constant c (rule: an antiderivative of e^{6Hy} is $e^{6Hy}/(6H)$; integration adds a constant). Divide by e^{6Hy}: $p_0 = \frac{P}{2} + c\,e^{-6Hy}$. Check: $p_0' = -6Hc\,e^{-6Hy}$, and $p_0' + 6Hp_0 = -6Hce^{-6Hy} + 3HP + 6Hce^{-6Hy} = 3HP$. (b) $V = p_0'/(3H) = -2c\,e^{-6Hy}$, which is zero everywhere only for $c = 0$: only the flat profile $p_0 = P/2$ satisfies C2. (c) Since $e^{6Hy} = \sin z$, $p_0 = P/2 + c/\sin z$: every non-flat conserved profile grows like $1/\sin z$ toward the tip.

Exercise 7 (Simpson's rule along the history). The record's table gives, for $N = 136$, $\lambda = 0$ at $a_{4,0} = 0, 0.5, 1, 1.5, 2$: $\int p_3 = 23.8133$, 15.4903, 10.0397, 6.44655, 4.06205 and $\int p_t = 0$; and $E(0) = 80.2822$, $E(2) = 12.4451$. (a) Compute the five rates $dE/da_{4,0}$. (b) Integrate them with Simpson's rule from 0 to 2 and compare with $E(2) - E(0)$. (c) Do the same with the trapezoidal rule, $h[\frac{1}{2}f_0 + f_1 + f_2 + f_3 + \frac{1}{2}f_4]$, and compare.

Answer. (a) $dE/da_{4,0} = -3\int p_3$: -71.4399, -46.4709, -30.1191, -19.33965, -12.18615. (b) $\frac{0.5}{3}[-71.4399 + 4(-46.4709) + 2(-30.1191) + 4(-19.33965) - 12.18615] = \frac{1}{6}(-71.4399 - 185.8836 - 60.2382 - 77.3586 - 12.18615) = \frac{1}{6}(-407.10645) = -67.8511$. The recorded change is $12.4451 - 80.2822 = -67.8371$ (Notebook 17b prints -67.8372, because it subtracts the unrounded numbers of the table). The difference is 0.0140, a relative error of $0.0140/67.837 = 2.1 \times 10^{-4}$, the value of Notebook 17b, In [18]. (c) $0.5[-35.71995 - 46.4709 - 30.1191 - 19.33965 - 6.093075] = 0.5 \times (-137.7427) = -68.8713$, a relative error of $1.034/67.837 = 1.5 \times 10^{-2}$, about seventy times larger: Simpson's rule, of order 4, is far better than the trapezoidal rule, of order 2, with the same five

values.

Exercise 8 (the order of the finite differences). For the state `N136_lam0_a10` Notebook 17b, In [13], found the difference 4.6×10^{-7} of max|T| at the step 0.02. Predict the differences at the steps 0.04, 0.06 and 0.10 if the error is of order 4, and compare with the printed values 7.5×10^{-6}, 3.8×10^{-5} and 3.0×10^{-4}.

Answer. An error of order 4 is $C\delta^4$, so doubling the step multiplies it by $2^4 = 16$: $16 \times 4.6 \times 10^{-7} = 7.4 \times 10^{-6}$. Tripling: $3^4 = 81$, $81 \times 4.6 \times 10^{-7} = 3.7 \times 10^{-5}$. Five times: $5^4 = 625$, $625 \times 4.6 \times 10^{-7} = 2.9 \times 10^{-4}$. The predictions agree with the printed values to within a few per cent, and the printed values are a little larger, more so for the larger steps (for the step 0.10 the printed 3.0×10^{-4} was at least 2.95×10^{-4} before rounding, while every number that rounds to 4.6×10^{-7}, at most 4.65×10^{-7}, gives at most $625 \times 4.65 \times 10^{-7} = 2.91 \times 10^{-4}$: the rounding cannot explain the gap). The excess is the next term of the error: a fourth-order difference has the error $C\delta^4 + D\delta^6 + \ldots$, and the term $D\delta^6$, small beside $C\delta^4$ for small δ, grows faster with the step. That is why the fitted order is 4.04 and not exactly 4.00. A failure of the conservation law would show up as a floor below which the differences stop falling; there is none.

Exercise 9 (the constraint in Einstein gravity). In Einstein gravity $\mathcal{C} = 3a_4'^2 + 21H^2 + \Lambda + \kappa\rho$ and $\mathcal{E} = 2a_4'' - \kappa(p_3 - p_t)$. (a) Show by hand that $d\mathcal{C}/dx_4 = 3a_4'\,\mathcal{E}$ for a source that obeys $\rho' = -3a_4'(p_3 - p_t)$. (b) For the linear member with $A = 1$ and the Kohn-Sham gas at $\lambda = 0$, what is the sign of $d\mathcal{C}/dx_4$, and what does it mean?

Answer. (a) $d\mathcal{C}/dx_4 = 6a_4'a_4'' + \kappa\rho'$ (rule: the chain rule for $a_4'^2$; H and Λ are constants) $= 6a_4'a_4'' - 3\kappa a_4'(p_3 - p_t)$ (rule: insert the conservation law) $= 3a_4'[2a_4'' - \kappa(p_3 - p_t)] = 3a_4'\,\mathcal{E}$ (rule: take out $3a_4'$). (b) With $a_4'' = 0$ and $a_4' = H$: $d\mathcal{C}/dx_4 = -3H\kappa(p_3 - p_t)$. At $\lambda = 0$ the gas has $p_t = 0$ exactly, and its p_3 is not zero (at the three points of In [12] of Notebook 17a it is positive, and its integral $\int p_3$ is positive at every slice, In [16]). Wherever $p_3 \neq 0$, $d\mathcal{C}/dx_4 \neq 0$, and integrated over the patch it is negative: even if the constraint held at one slice, it would be violated at the next. The linear member cannot carry this gas as its source.

Exercise 10 (the size of the patch). (a) Compute e^{-18} from $e^{-6} \approx 0.0024788$, and the angle z of the tip cutoff. (b) What fraction of the proper volume of the patch lies in $-1 \leq y \leq 0$?

Answer. (a) $e^{-18} = (e^{-6})^3 \approx 0.0024788^3 = 1.523 \times 10^{-8}$ (rule: $e^{3u} = (e^u)^3$). The tip cutoff has $\sin z = e^{6Hy} = e^{-18}$; for so small a number $\arcsin u \approx u$, so $z \approx 1.5 \times 10^{-8}$, the value in the caption of figure 17a.1. (b) The proper volume between $y = a$ and $y = 0$ is proportional to $\int_a^0 e^{6y}dy = (1 - e^{6a})/6$. For $a = -1$: $(1 - e^{-6})/6$; for the whole patch, $a = -3$: $(1 - e^{-18})/6$. The fraction is $(1 - 0.0024788)/(1 - 1.5 \times 10^{-8}) = 0.9975$: more than 99.7 per cent of the volume lies within one unit of the brane. The integral of a density, however, also depends on how large the density is where the volume is. For the states with 3-momentum ($N = 136$ and $N = 688$) the volume wins: Notebook 17b, In [15], finds that the last unit supplies between 0.812 and 0.969 of the integral of $e^{6Hy}p_0$, so their weighted mean $\bar{p}_0$ is decided near the brane. For the $N = 8$ brane zero modes it does not: their p_0 grows toward the tip almost like e^{-6Hy}, so $e^{6Hy}p_0$ is nearly flat, the last unit supplies only 0.439 to 0.441 of the integral, and the whole patch, the tip included, matters (Section 17.15).

18 The pairing theorems T1, T2 and Q

The author asked for a proof that "Universes of masses {+mass, -mass} are created in pairs", for the fermion field dirac16complex and for the commuting field dirac16complex00. This chapter gives, with complete line-by-line proofs, everything that the equations of this book actually establish about such pairs, and it says exactly where that stops. Three theorems are proved. **T1** (the chirality pairing): multiplying a field by the chirality matrix Γ turns every configuration of the theory with mass m and coupling λ into a configuration of the theory with $-m$ and $-\lambda$, in every gravitational field, and reverses its energy, momentum, stresses, current and charge. **T2** (the mirror pairing): multiplying by the gamma matrix of the hidden direction and reflecting the hidden coordinate across the surface $z = \pi/2$ turns a solution with $(-m, \lambda)$ into a solution with (m, λ) at EQUAL, not opposite, energy and charge; this uses the Z2 mirror construction, which is an assumption. **Q** (the quantum reading): for the quantised field dirac16complex the T1 image carries the opposite Krein matrix $-B$, so it is the same quantum system relabelled, while two independently quantised universes of masses $+m$ and $-m$ have identical one-particle spectra (the $\lambda = 0$ spectra, in flat 4+4 space or with frozen coefficients at a point) and energies that do not cancel. All three are exact maps between sets of solutions. None of them describes a process in which a universe, or a pair of universes, comes into being: no creation process, rate, probability or amplitude follows from these equations. Section 18.28 lists precisely what is not established.

18.1 What this chapter does

The question, and the honest answer. In this book a **universe of mass** m means a configuration of one of the two fields, with mass parameter m and coupling λ, in the author's primordial gravitational field; usually it is a **solution**, a configuration that obeys the field equations. Two universes are **paired** when an explicit invertible rule takes every solution of the theory with mass m to a solution of the theory with mass $-m$, and takes its Lagrangian, its energy-momentum tensor and its current to stated multiples of those of the partner. The author's request asks for more: that such universes are *created* in pairs at the big bang. That statement is a HYPOTHESIS of the author. The field equations of this book are equations for a field on a given gravitational background, together with their canonical quantisation; no equation of the book produces a universe from anything, so the creation of pairs cannot be proved from them, and it is not proved here. What can be proved is that the solutions come in partnered families, and with which signs their energies and charges are partnered. That is the content of this chapter.

The plan. Sections 18.2 to 18.4 set the stage: the words, the author's metric, its spin connection, the Lagrangian, the field equation, the energy-momentum tensor and the current. Section 18.5 proves four matrix lemmas; everything later rests on them. Notebook 18a checks them exactly (Sections 18.6 to 18.9). Sections 18.10 to 18.13 state and prove T1; Sections 18.14 to 18.16 state and prove T2. Notebook 18b proves both theorems again, as exact polynomial identities, in the author's metric with an arbitrary deflating history (Sections 18.17 to 18.20). Section 18.21 derives a family of exact solutions, the fields that depend only on the time; Sections 18.22 and 18.23 state and prove Q. Notebook 18c solves the field equations numerically and shows each theorem at work on actual solutions (Sections 18.24 to 18.27). Section 18.28 states what is not established, Section 18.29 collects the status of every result, and Sections 18.30 and 18.31 give exercises and their complete worked answers. The Kohn-Sham version of the pairing, theorem T3, is the subject of Chapter 19; the corollary C1 and the question of the title of Chapter 20, "Do universes come in pairs?", are

treated there; matter and antimatter are the subject of Chapter 21.

The three worked examples. Each is a complete Jupyter notebook, complete in itself; you may run them in any order.

notebook	what it computes	checks	figures
18a	the author's gammas rebuilt from their formulas; the four matrix lemmas, exactly	16	7
18b	T1 and T2 as exact sympy identities in the author's metric, both statistics	20	5
18c	T1, T2 and Q on actual solutions: Runge-Kutta, spectra, Krein norms	21	7

Notation. The author's coordinates are written $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates (scale factor $e^{a_4} \sin^{1/6} z$); x_4 is the time; x_5, x_6, x_7 are the three extra times, which are time-like and DEFLATE EXPONENTIALLY (scale factor $e^{-a_4} \sin^{1/6} z$, with a_4 increasing); x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$ and $H > 0$ the author's constant. The author's metric of 2026-10-14 has a second function $b_4(x_4) > 0$ in the extra-time factor (Chapter 3); this chapter works on the background $b_4 = 1$, and Sections 18.3, 18.16 and 18.21 say what a changing b_4 changes. The flat frame metric is $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \ldots, x_7$; its entry for the direction a is η_{aa}, and $\eta_{ab} = 0$ for $a \neq b$. The gamma matrices are the author's eight real 16×16 matrices; the one of the direction x_a is written $\gamma^{(x_a)}$, or $\gamma^{(a)}$ for short. I_{16} (also written 1) is the 16×16 identity matrix and I_8 the 8×8 one. For a matrix M, M^T is its transpose (rows and columns exchanged), M^* the matrix of the complex-conjugate entries and $M^\dagger = (M^T)^*$; for a real matrix $M^\dagger = M^T$. The letters m and λ are the mass and the coupling of the field. A statement is labelled PROVED (exact, with the verifying record and its check), COMPUTED (a numerical result with its accuracy), ASSUMED, HYPOTHESIS or OPEN.

The records this chapter uses. Every formula and number comes from these Revision records or from the chapter's own notebooks, which reproduce the records where they overlap (each such check prints the record file and the check name):

record	what it holds
`Revision/algebra/gammas.json`	the eight gammas, C, Γ and B
`Revision/pairing/pairing-theory.json`	the theorems T1, T2, Q with hypotheses, proofs and data tables
`Revision/pairing/reports/wolfram-pairing.json`	the WolframScript verification of T1, T2, Q (101 checks, all PASS)
`Revision/pairing/reports/python-pairing.json`	the independent sympy verification (66 checks, all PASS)
`Revision/theory/reports/`	the field-theory checks used here: `python-field-theory.json`, `wolfram-field-theory.json`, `python-scope.json`

record	what it holds
`Revision/lead_checks/reports/charge-conjugation-and-u1.json`	the charge-conjugation matrices and the charge (12 checks, all PASS)
`Revision/kohn_sham/results/parameters.json`	the canonical deflating history $a_4 = AHx_4$, $A = 1$

The Wolfram side and the sympy side share no code; the sympy side rebuilds the gammas from the author's formulas and compares them entry by entry with `gammas.json`. A report is a file in which a verifying program has recorded every check with its name, its verdict and a detail text; every check cited in this chapter has the verdict PASS. Below, a report is named by its file name only, for example "`python-pairing.json`, check `gammas.Gamma`". The Revision document that collects these proofs is `Revision/docs/PAIR_CREATION_PROOFS.md`; this chapter follows its logic and adds every intermediate line.

18.2 The words of this chapter

- **Configuration**: any field Ψ, that is, 16 components $\Psi_1(x), \ldots, \Psi_{16}(x)$ given at every point x, whether or not it obeys the field equations. A statement that holds for every configuration holds **off shell**; one that holds only for solutions holds **on shell**. An off-shell identity holds in particular on shell.
- **Statistics**: dirac16complex has complex ANTICOMMUTING components, called **Grassmann** numbers ($\theta_1\theta_2 = -\theta_2\theta_1$, so $\theta\theta = 0$); dirac16complex00 has complex COMMUTING components (ordinary complex numbers). "Both statistics" means: the statement and its proof hold for both kinds of components.
- **Parameters**: the mass m and the coupling λ of the potential $U(S) = \frac{\lambda}{2}S^2$. The theory with the parameters (m, λ) and the theory with $(-m, -\lambda)$ are two DIFFERENT theories: their Lagrangians are different functions of the field.
- **Map**: a rule that turns a field Ψ into a new field Ψ'. All maps of this chapter are of the form $\Psi'(x) = M\Psi(x)$ with a constant 16×16 matrix M, possibly combined with a change of the coordinate x_0.
- **Bilinear**: an expression $\Psi^\dagger X \Phi = \sum_{A,B} \Psi^*_A X_{AB} \Phi_B$ with a matrix X of numbers or of ordinary functions; X is its **kernel**.
- **Signed permutation matrix**: a matrix with exactly one nonzero entry, $+1$ or -1, in each row and in each column. Multiplying a column of 16 numbers by it reorders the numbers and changes the signs of some of them. Each of the author's gammas is such a matrix.
- **Projector**: a matrix P with $PP = P$. It keeps one part of a column and removes the rest; doing it twice changes nothing more. The **chiral projectors** $P_- = \frac{1}{2}(1-\Gamma)$ and $P_+ = \frac{1}{2}(1+\Gamma)$ keep the components 1 to 8, respectively 9 to 16, and set the other eight to zero.
- **Dirac adjoint**: the row $\bar\Psi = \Psi^\dagger C$, with the charge matrix C of Chapter 5. The **scalar** is $S = \bar\Psi\Psi = \Psi^\dagger C \Psi$.
- **Chirality**: the matrix $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\cdots\gamma^{(x_7)}$, the product of all eight gammas in the author's order (the author's T16[8]).

- **Chiral halves**: the components 1 to 8 of Ψ, written ψ_- (where $\Gamma = -1$), and 9 to 16, written ψ_+ (where $\Gamma = +1$).
- **Reflection** R_n: the 8×8 diagonal matrix with -1 in place n and $+1$ elsewhere; it reverses the direction n and keeps η: $R_n \eta R_n = \eta$.
- **Pin(4,4)**: the group of all products of unit vectors $v = \sum_a v_a \gamma^{(a)}$ with $\sum_a \eta_{aa} v_a^2 = \pm 1$ (Chapter 5). Each single gamma is such a unit vector, so every product of gammas belongs to Pin(4,4).
- **Character** of a map M: the sign χ in $M^\dagger C M = \chi C$. It says whether S keeps ($\chi = +1$) or reverses ($\chi = -1$) its sign under $\Psi \to M\Psi$.
- **Krein matrix** $B = -iC\gamma^{(x_4)}$: the charge density is $\Psi^\dagger B \Psi$, and B is the matrix of the canonical anticommutator of the quantised field (Chapter 10). B has eight eigenvalues $+1$ and eight -1; the number $u^\dagger B u$ of a column u is its **Krein norm**, which can be negative. The **Krein sign** of a map M is the sign σ in $MBM^\dagger = \sigma B$.
- **Patch, mirror patch, brane**: $z = 6Hx_0$ runs over the **patch** $0 < z < \pi/2$. The map $z \to \pi - z$ takes it to the **mirror patch** $\pi/2 < z < \pi$. The surface $z = \pi/2$ between them is the **brane**. Gluing the mirror patch to the patch at the brane is the **Z2 construction** of the Revision record; it is ASSUMED.
- **Isometry**: a change of coordinates that does not change the metric.
- **Jet**: at one point, the values of the 16 components, of their 16 complex conjugates and of their $8 \times 16 + 8 \times 16$ first derivatives: 288 numbers. A Lagrangian at a point is a polynomial in these 288 numbers.
- **Negative control**: a computation that must FAIL. It shows that a test can detect a wrong statement.
- **Junction condition**: an equation that says how the fields and the geometry must match across a surface, here across the brane $z = \pi/2$. **Brane tension**: an energy per unit area carried by such a surface itself (for the brane, per unit of its seven-dimensional area).
- **Legendre transform**: the rule that turns a Lagrangian into the energy (Hamiltonian) density: for each field, its time derivative times its canonical momentum, summed over the fields, minus the Lagrangian.
- Words used only in Section 18.28, in the list of what is NOT established: a **vacuum** is a state with no particles, and **vacuum decay** is its change into a state of lower energy; **tunnelling** is a quantum transition through a region that the classical equations forbid, for example through an energy barrier; a **cross-section** is a number, with the units of an area, that measures how likely a collision or a reaction is; a **Bogoliubov coefficient** says how much of the particles of one choice of vacuum the vacuum of another choice contains, and so how many particles a changing background creates; the **wave function of the universe** is a quantum state assigned to a whole universe, its geometry included; a **path integral** computes a quantum amplitude by adding one contribution for every possible history of the fields.

18.3 The stage: the author's metric, its frame and its spin connection

The metric. The author's line element, in their coordinates, is

$$\begin{aligned} ds^2 = {} & \cot^2 z\, dx_0^2 + e^{2a_4} \sin^{1/3} z\, (dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2 \\ & - e^{-2a_4} \sin^{1/3} z\, (dx_5^2 + dx_6^2 + dx_7^2), \end{aligned}$$

with $a_4 = a_4(x_4)$ and $z = 6Hx_0$. This is the author's metric of 2026-10-14 on its background $b_4 = 1$: the metric has a second function $b_4(x_4) > 0$, which multiplies the three extra-time components, $-b_4 e^{-2a_4} \sin^{1/3} z\, (dx_5^2 + dx_6^2 + dx_7^2)$ (Chapter 3; `wolfram-field-theory-b4.json`, check `metric_is_the_authors_b4`). The pairing records and the three notebooks of this chapter work at $b_4 = 1$; the end of this section and Sections 18.16 and 18.21 say what a changing b_4 changes. The metric is diagonal: $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ with the **vielbein factors**

$$f_0 = \cot z, \quad f_1 = f_2 = f_3 = e^{a_4} \sin^{1/6} z, \quad f_4 = 1, \quad f_5 = f_6 = f_7 = e^{-a_4} \sin^{1/6} z.$$

The **diagonal vielbein** is $e^a{}_\mu = f_\mu \delta^a_\mu$, its inverse $e^\mu{}_a = \delta^\mu_a / f_a$, and the curved gammas are $\gamma^\mu = e^\mu{}_a \gamma^{(a)} = \gamma^{(\mu)} / f_\mu$ (no sum) and $\gamma_\mu = g_{\mu\mu} \gamma^\mu = \eta_{\mu\mu} f_\mu \gamma^{(\mu)}$. The volume factor is the product of the eight factors:

$$\sqrt{|g|} = f_0 f_1 \cdots f_7 = \cot z \cdot \left(e^{a_4} \sin^{1/6} z\right)^3 \cdot 1 \cdot \left(e^{-a_4} \sin^{1/6} z\right)^3.$$

The rule $\sqrt{|\det g|} = \prod_\mu |g_{\mu\mu}|^{1/2}$ for a diagonal metric gave this line.

$$\sqrt{|g|} = e^{3a_4 - 3a_4} \sin^{6/6} z \cdot \frac{\cos z}{\sin z} = \cos z.$$

The powers of e^{a_4} cancel (the inflation of 3-space is compensated by the deflation of the extra times), the six powers $\sin^{1/6} z$ multiply to $\sin z$, and $\cot z = \cos z / \sin z$. So $\sqrt{|g|} = \cos z$ for every history a_4 (record: `wolfram-pairing.json`, check `primordial_vielbein`; Notebook 18b checks it). With a changing b_4 each extra-time factor carries $b_4^{1/2}$, and $\sqrt{|g|} = b_4^{3/2} \cos z$ (`wolfram-field-theory-b4.json`, check `sqrt_det_g`).

The canonical spin connection, derived for a diagonal vielbein. A spinor field needs a **spin connection** $\omega_{\mu ab}$ to define its covariant derivative (Chapter 6). The canonical one is fixed by the **vielbein postulate**, which says that the gammas are covariantly constant; its solution is

$$\omega_\mu{}^a{}_b = e^a{}_\nu (\partial_\mu e^\nu{}_b + \Gamma^\nu{}_{\mu\lambda} e^\lambda{}_b), \qquad \omega_{\mu ab} = \eta_{aa}\, \omega_\mu{}^a{}_b,$$

where $\Gamma^\nu{}_{\mu\lambda}$ are the Christoffel symbols of the metric (Chapter 3). (In this chapter the symbol Γ WITH indices is a Christoffel symbol; WITHOUT indices it is the chirality matrix.) For a diagonal metric the Christoffel symbols with three different indices vanish, and for $a \neq b$

$$\Gamma^a{}_{ab} = \frac{\partial_b g_{aa}}{2 g_{aa}}, \qquad \Gamma^a{}_{bb} = -\frac{\partial_a g_{bb}}{2 g_{aa}}.$$

These are the Christoffel formula $\Gamma^{\nu}{}_{\mu\lambda} = \frac{1}{2} g^{\nu\nu}(\partial_\mu g_{\nu\lambda} + \partial_\lambda g_{\nu\mu} - \partial_\nu g_{\mu\lambda})$ with a diagonal g, in which only the terms with two equal indices survive. Now take a direction a and a different direction b, and compute the component of the connection whose derivative index is $\mu = a$:

$$\omega_a{}^a{}_b = e^a{}_a(\partial_a e^a{}_b + \Gamma^a{}_{a\lambda} e^\lambda{}_b) = f_a\Big(0 + \Gamma^a{}_{ab}\frac{1}{f_b}\Big).$$

The inverse vielbein is diagonal, so $e^a{}_b = 0$ for $a \neq b$ and $e^\lambda{}_b = \delta^\lambda_b / f_b$ keeps only $\lambda = b$.

$$\Gamma^a{}_{ab} = \frac{\partial_b(\eta_{aa} f_a^2)}{2\eta_{aa} f_a^2} = \frac{2\eta_{aa} f_a\, \partial_b f_a}{2\eta_{aa} f_a^2} = \frac{\partial_b f_a}{f_a}.$$

We inserted $g_{aa} = \eta_{aa} f_a^2$ and used the chain rule $\partial_b(f_a^2) = 2 f_a \partial_b f_a$.

$$\omega_a{}^a{}_b = f_a\,\frac{\partial_b f_a}{f_a}\,\frac{1}{f_b} = \frac{\partial_b f_a}{f_b}, \qquad \omega_{a,ab} = \eta_{aa}\frac{\partial_b f_a}{f_b}.$$

The first equation inserts the line before into the line before that; the second lowers the index a with η_{aa}. Now the component with the derivative index $\mu = b$, line by line:

$$\omega_b{}^a{}_b = e^a{}_a(\partial_b e^a{}_b + \Gamma^a{}_{b\lambda} e^\lambda{}_b) = f_a\Big(0 + \Gamma^a{}_{bb}\frac{1}{f_b}\Big).$$

The same two facts as before: $e^a{}_\nu$ keeps only $\nu = a$, $e^a{}_b = 0$ for $a \neq b$, and $e^\lambda{}_b = \delta^\lambda_b / f_b$ keeps only $\lambda = b$.

$$\Gamma^a{}_{bb} = -\frac{\partial_a(\eta_{bb} f_b^2)}{2\eta_{aa} f_a^2} = -\frac{2\eta_{bb} f_b\, \partial_a f_b}{2\eta_{aa} f_a^2} = -\eta_{aa}\eta_{bb}\,\frac{f_b\,\partial_a f_b}{f_a^2}.$$

We inserted $g_{bb} = \eta_{bb} f_b^2$ and $g_{aa} = \eta_{aa} f_a^2$ into the second Christoffel formula, used the chain rule, and replaced $1/\eta_{aa}$ by η_{aa} (it is $+1$ or -1).

$$\omega_b{}^a{}_b = f_a\Big(-\eta_{aa}\eta_{bb}\frac{f_b\,\partial_a f_b}{f_a^2}\Big)\frac{1}{f_b} = -\eta_{aa}\eta_{bb}\,\frac{\partial_a f_b}{f_a}, \qquad \omega_{b,ab} = \eta_{aa}\,\omega_b{}^a{}_b = -\eta_{bb}\,\frac{\partial_a f_b}{f_a}.$$

The first equation inserts the line before and cancels f_b and one f_a; the second lowers the index a with η_{aa} and uses $\eta_{aa}^2 = 1$. Compare with the formula $\omega_{a,ab} = \eta_{aa}\,\partial_b f_a / f_b$ found first: exchanging the names a and b in it gives $\omega_{b,ba} = \eta_{bb}\,\partial_a f_b / f_a$, so $\omega_{b,ab} = -\omega_{b,ba}$, the antisymmetry $\omega_{\mu ab} = -\omega_{\mu ba}$. Components whose three indices are all different contain a Christoffel symbol with three different indices and vanish, and $\omega_\mu{}^a{}_a = f_a(-\partial_\mu f_a / f_a^2 + \partial_\mu f_a / f_a^2) = 0$. So **the only nonzero components are $\omega_{a,ab} = \eta_{aa}\,\partial_b f_a / f_b$ and $\omega_{a,ba} = -\omega_{a,ab}$, for $a \neq b$.**

The twelve components in the author's metric. The factors depend only on x_4 (through a_4) and on x_0 (through z), and $\partial_0 = 6H\,d/dz$ because $z = 6Hx_0$. Write $a_4' = da_4/dx_4$.

- $a = 1$, $b = 4$: $\eta_{11} = 1$, $\partial_4 f_1 = a_4' e^{a_4} \sin^{1/6} z$ (chain rule), $f_4 = 1$, so $\omega_{x_1,x_1x_4} = a_4' e^{a_4} \sin^{1/6} z$.

- $a=1$, $b=0$: $\partial_0 f_1 = 6H\,e^{a_4}\,\frac{1}{6}\sin^{-5/6} z\cos z = He^{a_4}\sin^{-5/6} z\cos z$ (power rule and chain rule); dividing by $f_0 = \cos z/\sin z$ gives $\omega_{x_1,x_1x_0} = He^{a_4}\sin^{1/6} z$, and by antisymmetry $\omega_{x_1,x_0x_1} = -He^{a_4}\sin^{1/6} z$.
- $a=5$, $b=4$: $\eta_{55}=-1$, $\partial_4 f_5 = -a_4' e^{-a_4}\sin^{1/6} z$, so $\omega_{x_5,x_5x_4} = +a_4' e^{-a_4}\sin^{1/6} z$, and by antisymmetry $\omega_{x_5,x_4x_5} = -a_4' e^{-a_4}\sin^{1/6} z$.
- $a=5$, $b=0$: $\partial_0 f_5 = He^{-a_4}\sin^{-5/6} z\cos z$; with $\eta_{55}=-1$ and the division by f_0: $\omega_{x_5,x_5x_0} = -He^{-a_4}\sin^{1/6} z$, and by antisymmetry $\omega_{x_5,x_0x_5} = +He^{-a_4}\sin^{1/6} z$.
- x_2, x_3 repeat x_1, and x_6, x_7 repeat x_5. $f_4 = 1$ is constant and f_0 depends only on x_0, so $a=4$ and $a=0$ give nothing.

That is $3\times 2 + 3\times 2 = 12$ nonzero components with $a<b$ in the order $x_0,\ldots,x_7$ (the hidden index first: ω_{x_1,x_0x_1}, ω_{x_1,x_1x_4}, ω_{x_5,x_0x_5}, ω_{x_5,x_4x_5}, ...), exactly the list of the record (`python-pairing.json`, check `geometry.spin_connection_components`); Notebooks 18b and 18c reproduce the list character for character. Note that $\Omega_{x_4} = \Omega_{x_0} = 0$: no component has the derivative index 4 or 0.

What enters the field equation: $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$. The spinor connection is $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$ with the generators $S^{ab} = \frac{1}{4}[\gamma^{(a)},\gamma^{(b)}]$, which equal $\frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ for $a\neq b$ (two different gammas anticommute, so $\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)} = 2\gamma^{(a)}\gamma^{(b)}$). For the inflating direction x_1:

$$\Omega_{x_1} = \omega_{x_1,x_1x_4}S^{14} + \omega_{x_1,x_1x_0}S^{10} = \tfrac{1}{2}\omega_{x_1,x_1x_4}\gamma^{(1)}\gamma^{(4)} + \tfrac{1}{2}\omega_{x_1,x_1x_0}\gamma^{(1)}\gamma^{(0)}.$$

The two terms $\omega_{\mu ab}S^{ab}$ and $\omega_{\mu ba}S^{ba}$ of the sum are equal (both factors change sign), which cancels the $\frac{1}{2}$ of the definition; then $S^{ab} = \frac{1}{2}\gamma^{(a)}\gamma^{(b)}$.

$$\gamma^{x_1}\Omega_{x_1} = \frac{\gamma^{(1)}}{f_1}\,\Omega_{x_1} = \frac{1}{2f_1}\big(\omega_{x_1,x_1x_4}\gamma^{(4)} + \omega_{x_1,x_1x_0}\gamma^{(0)}\big) = \frac{a_4'}{2}\gamma^{(4)} + \frac{H}{2}\gamma^{(0)}.$$

We used $\gamma^{(1)}\gamma^{(1)} = \eta_{11} = +1$ (Clifford relation), and the factor $1/f_1 = e^{-a_4}\sin^{-1/6} z$ cancels the factor $e^{a_4}\sin^{1/6} z$ of both components. For the deflating extra time x_5:

$$\gamma^{x_5}\Omega_{x_5} = \frac{\gamma^{(5)}}{2f_5}\big(\omega_{x_5,x_4x_5}\gamma^{(4)}\gamma^{(5)} + \omega_{x_5,x_5x_0}\gamma^{(5)}\gamma^{(0)}\big).$$

This is the same construction with the two components of x_5 ($S^{45} = \frac{1}{2}\gamma^{(4)}\gamma^{(5)}$ and $S^{50} = \frac{1}{2}\gamma^{(5)}\gamma^{(0)}$).

$$\gamma^{(5)}\gamma^{(4)}\gamma^{(5)} = -\gamma^{(4)}\gamma^{(5)}\gamma^{(5)} = -\gamma^{(4)}(-1) = \gamma^{(4)}, \qquad \gamma^{(5)}\gamma^{(5)}\gamma^{(0)} = -\gamma^{(0)}.$$

Two different gammas anticommute, and a time-like gamma squares to $\eta_{55} = -1$.

$$\gamma^{x_5}\Omega_{x_5} = \frac{1}{2f_5}\big(-a_4' e^{-a_4}\sin^{1/6} z\,\gamma^{(4)} + He^{-a_4}\sin^{1/6} z\,\gamma^{(0)}\big) = -\frac{a_4'}{2}\gamma^{(4)} + \frac{H}{2}\gamma^{(0)}.$$

We inserted the two components and cancelled $f_5 = e^{-a_4}\sin^{1/6} z$. Adding the three inflating and the three deflating directions (and nothing from x_4 and x_0):

$$\gamma^\mu\Omega_\mu = 3\Big(\frac{a_4'}{2}\gamma^{(4)} + \frac{H}{2}\gamma^{(0)}\Big) + 3\Big(-\frac{a_4'}{2}\gamma^{(4)} + \frac{H}{2}\gamma^{(0)}\Big) = 3H\gamma^{(x_0)}.$$

The a_4' terms of the three inflating directions and of the three deflating extra times cancel exactly; the hidden-direction terms add. This is PROVED for every history $a_4(x_4)$ (`python-field-theory.json`, check `gamma_mu_Omega_mu_equals_3H_gamma_x0`; `wolfram-field-theory.json`, check `gammaOmega_equals_3H_gamma_x0`; Notebook 18b recomputes it direction by direction and draws it in its figure 1). The value $3H\gamma^{(x_0)}$ belongs to the diagonal vielbein: in another frame it changes (`python-scope.json`; Chapter 8). None of the theorems below depends on it: T1 holds for every connection, and T2 uses only the transformation of the connection.

With a changing b_4. In the metric with $b_4(x_4)$ the extra-time factors are $f_5 = f_6 = f_7 = e^{-a_4} b_4^{1/2} \sin^{1/6} z$. Each extra time then contributes $(-\frac{a_4'}{2} + \frac{b_4'}{4b_4})\gamma^{(x_4)} + \frac{H}{2}\gamma^{(x_0)}$, and

$$\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)} + \frac{3}{4}\,\frac{b_4'}{b_4}\,\gamma^{(x_4)}$$

(Chapter 7; the checks `gammaOmega_per_direction` and `gammaOmega_total` of `python-field-theory-b4.json` and of `wolfram-field-theory-b4.json`). The extra term vanishes on the background $b_4 = 1$ and for every constant b_4.

18.4 The Lagrangian, the field equation, the energy-momentum tensor and the current

The Lagrangian. Both fields have the same Lagrangian density; only the statistics of the components differ:

$$\mathcal{L}_{m,\lambda}[\Psi] = \sqrt{|g|}\,\Big[K[\Psi] - mS - \frac{\lambda}{2}S^2\Big], \qquad K = \frac{1}{2}(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi), \qquad S = \bar\Psi\Psi,$$

with the covariant derivatives $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$, and a sum over $\mu = 0, \ldots, 7$ (the eight coordinates $x_0, \ldots, x_7$) in K. K is the **kinetic term**, mS the **mass term**, $\frac{\lambda}{2}S^2$ the **potential**. For a general potential $U(S)$ we write $\mathcal{L}_{m,U}$. The Lagrangian is real for both statistics (Chapter 7).

The kinetic term written out. Inserting $\gamma^\mu = e^\mu{}_a\gamma^{(a)}$ and the two covariant derivatives into K:

$$K = \frac{1}{2}\sum_{\mu,a} e^\mu{}_a(\bar\Psi\gamma^{(a)}\partial_\mu\Psi - \partial_\mu\bar\Psi\gamma^{(a)}\Psi) + \frac{1}{2}\sum_{\mu,a} e^\mu{}_a(\bar\Psi\gamma^{(a)}\Omega_\mu\Psi + \bar\Psi\Omega_\mu\gamma^{(a)}\Psi).$$

The derivative parts give the first sum; the connection parts $+\bar\Psi\gamma^{(a)}\Omega_\mu\Psi$ (from $D_\mu\Psi$) and $-(-\bar\Psi\Omega_\mu)\gamma^{(a)}\Psi$ (from $D_\mu\bar\Psi$) give the second.

$$K = \frac{1}{2}\sum_{\mu,a} e^\mu{}_a(\bar\Psi\gamma^{(a)}\partial_\mu\Psi - \partial_\mu\bar\Psi\gamma^{(a)}\Psi) + \frac{1}{4}\sum_{\mu,a,b,c} e^\mu{}_a\,\omega_{\mu bc}\,\bar\Psi\{\gamma^{(a)}, S^{bc}\}\Psi.$$

We inserted $\Omega_\mu = \frac{1}{2}\sum_{b,c}\omega_{\mu bc}S^{bc}$ and wrote $\gamma^{(a)}S^{bc} + S^{bc}\gamma^{(a)} = \{\gamma^{(a)}, S^{bc}\}$, the **anticommutator**. This form shows the structure that every proof below uses: K is a sum of **coefficient functions** of the gravitational field ($e^\mu{}_a$, $\omega_{\mu bc}$; and $\sqrt{|g|}$ in front of $\mathcal{L}$) times **bilinears** whose kernels are $C\gamma^{(a)}$ and $C\gamma^{(a)}S^{bc}$, $CS^{bc}\gamma^{(a)}$, products of one or three gammas after C.

The field equation. Chapter 7 derived the Euler-Lagrange equations of $\mathcal{L}_{m,U}$; for both statistics they are

$$E_{m,U}[\Psi] \equiv \gamma^\mu D_\mu\Psi - \big(m + U'(S)\big)\Psi = 0, \qquad (D_\mu\bar\Psi)\gamma^\mu = -\big(m + U'(S)\big)\bar\Psi,$$

with $U'(S) = \lambda S$ for $U = \frac{\lambda}{2}S^2$; we write $E_{m,\lambda}$. The record proves that the Euler-Lagrange expression of Ψ^*_A derived from $\mathcal{L}$ is exactly $\sqrt{|g|}\,[CE_{m,\lambda}]_A$ (`python-pairing.json`, check `T1.metric.commuting.euler_lagrange_derived` and its Grassmann twin; Notebook 18b repeats it). In the author's metric at $b_4 = 1$, with $\gamma^\mu = \gamma^{(\mu)}/f_\mu$, $\partial_0 = 6H\,d/dz$ and $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$, the field equation reads

$$\begin{aligned} &e^{-a_4}\sin^{-1/6} z\,\big(\gamma^{(1)}\partial_1 + \gamma^{(2)}\partial_2 + \gamma^{(3)}\partial_3\big)\Psi + \gamma^{(4)}\partial_4\Psi \\ &\quad + e^{a_4}\sin^{-1/6} z\,\big(\gamma^{(5)}\partial_5 + \gamma^{(6)}\partial_6 + \gamma^{(7)}\partial_7\big)\Psi \\ &\quad + \tan z\,\gamma^{(0)}\partial_0\Psi + 3H\gamma^{(0)}\Psi = (m + \lambda S)\Psi. \end{aligned}$$

The derivative terms are $\gamma^\mu\partial_\mu\Psi$ with $1/f_\mu$ written out ($1/\cot z = \tan z$); the last term on the left is $\gamma^\mu\Omega_\mu\Psi$. The deflation of the extra times appears as the growing factor e^{a_4} in front of their derivatives. With a changing b_4 that factor is $e^{a_4}b_4^{-1/2}\sin^{-1/6} z$, and the left side gains the term $\frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}\Psi$ (`wolfram-field-theory-b4.json`, checks `Dirac_operator_explicit_G` and `Dirac_operator_explicit_C`; Chapter 7).

The energy-momentum tensor and the current. The pairing record uses the symmetric tensor

$$T_{\mu\nu} = \frac{1}{4}\big(\bar\Psi\gamma_\mu D_\nu\Psi + \bar\Psi\gamma_\nu D_\mu\Psi - (D_\mu\bar\Psi)\gamma_\nu\Psi - (D_\nu\bar\Psi)\gamma_\mu\Psi\big) - g_{\mu\nu}\frac{\mathcal{L}}{\sqrt{|g|}},$$

which is minus the tensor of Chapter 9; in this convention the energy density is $\rho = -T_{x_4x_4}$ and the pressure of a direction $\mu \neq x_4$ is $p_\mu = -g^{\mu\mu}T_{\mu\mu}$. The current is a multiple of $\bar\Psi\gamma^\mu\Psi$: the theory record and Notebook 18c use $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, whose time component is the charge density $J^{x_4} = -i\Psi^\dagger C\gamma^{(x_4)}\Psi = \Psi^\dagger B\Psi$, with the charge $Q = \int\cos z\,\Psi^\dagger B\Psi\,d^7x$ of a slice $x_4 = \text{const}$ (the integral runs over the seven coordinates other than x_4, with x_0 over the patch; in a formula the letter Q always means this charge, while the plain name Q, without a formula, is the quantum theorem of Section 18.22); the sympy pairing checker and Notebook 18b use $i\bar\Psi\gamma^\mu\Psi$, the Wolfram checker $\bar\Psi\gamma^\mu\Psi$. Every statement of this chapter about T and J has the form $T' = \pm T$ or $J' = \pm J$; such a statement is unchanged when T or J is multiplied by any fixed number, so the different conventions do not matter.

18.5 Four matrix lemmas

Every map of this chapter multiplies the field by a constant matrix. The lemmas below say what such a multiplication does to the adjoint, to bilinears, to the connection and to the Krein matrix. They are exact identities between the author's matrices. We use only the Clifford relation

$\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta_{ab} I_{16}$, the reality of the gammas, $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$, and the facts about C proved in Chapter 5: $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$ is real and symmetric, $CC = 1$, and

$$C\gamma^{(a)}C^{-1} = -(\gamma^{(a)})^T \qquad \text{for every } a.$$

Rule P (moving a gamma through a product). If X is a product of k gammas, $\gamma^{(a)}X = \pm X\gamma^{(a)}$, with one factor -1 for each factor of X different from $\gamma^{(a)}$ (they anticommute) and $+1$ for each factor equal to $\gamma^{(a)}$ (it commutes with itself). This is the Clifford relation applied once per factor.

Lemma 1 (the chirality). Γ is real and symmetric, $\Gamma\Gamma = 1$, $\Gamma\gamma^{(a)} = -\gamma^{(a)}\Gamma$ for every a, $\Gamma C = C\Gamma$ and $\Gamma S^{ab} = S^{ab}\Gamma$. Hence $\Gamma\Omega_\mu = \Omega_\mu\Gamma$ for EVERY connection $\Omega_\mu = \frac{1}{2}\sum \omega_{\mu ab} S^{ab}$, and

$$D_\mu(\Gamma\Psi) = \Gamma D_\mu\Psi, \qquad D_\mu(\bar\Psi\Gamma) = (D_\mu\bar\Psi)\Gamma.$$

Proof. Γ is the product of all eight different gammas. Moving $\gamma^{(b)}$ through it passes the seven factors $\gamma^{(a)}$, $a \neq b$, and the factor $\gamma^{(b)}$ itself; by Rule P the sign is $(-1)^7 = -1$, so $\gamma^{(b)}\Gamma = -\Gamma\gamma^{(b)}$. C is a product of four gammas and S^{ab} $(a \neq b)$ is $\frac{1}{2}$ times a product of two; moving Γ through them costs $(-1)^4 = +1$ and $(-1)^2 = +1$, so both commute with Γ ($S^{aa} = 0$ trivially). The product Γ of the author's gammas is computed exactly: $\Gamma = \mathrm{diag}(-I_8, I_8)$, which is real, symmetric and squares to 1. Since every Ω_μ is a combination of the S^{ab} with ordinary numbers as coefficients, Γ commutes with it. Finally

$$D_\mu(\Gamma\Psi) = \partial_\mu(\Gamma\Psi) + \Omega_\mu\Gamma\Psi = \Gamma\partial_\mu\Psi + \Gamma\Omega_\mu\Psi = \Gamma D_\mu\Psi.$$

The constant Γ passes through ∂_μ (a constant factor comes out of a derivative), and $\Omega_\mu\Gamma = \Gamma\Omega_\mu$. The adjoint statement, written out from the right:

$$D_\mu(\bar\Psi\Gamma) = \partial_\mu(\bar\Psi\Gamma) - \bar\Psi\Gamma\Omega_\mu = (\partial_\mu\bar\Psi)\Gamma - \bar\Psi\Omega_\mu\Gamma = (\partial_\mu\bar\Psi - \bar\Psi\Omega_\mu)\Gamma = (D_\mu\bar\Psi)\Gamma.$$

The first equality is the rule $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$ of Section 18.4, applied to the row $\bar\Psi\Gamma$ (by Lemma 2 below it is the adjoint of $\Gamma\Psi$); the second takes the constant Γ out of ∂_μ and uses $\Gamma\Omega_\mu = \Omega_\mu\Gamma$; the third takes out the common factor Γ on the right, and the last recognises $D_\mu\bar\Psi$. QED.

statement	**status**	**where it is verified**
Lemma 1	PROVED	`wolfram-pairing.json`, check `Gamma_properties`; `python-pairing.json`, checks `gammas.Gamma` and `gammas.Gamma_ant icommutes`; Notebook 18a, In [6] and In [7]

Lemma 2 (bilinears under $\Psi \to \Gamma\Psi$). For $\Psi' = \Gamma\Psi$: $\bar\Psi' = \bar\Psi\Gamma$, and $\bar\Psi' X \Psi' = (-1)^k\, \bar\Psi X \Psi$ for every product X of k gammas. In particular S is unchanged and every bilinear with the kernel $C\gamma^{(a)}$, $C\gamma^{(a)}S^{bc}$ or $CS^{bc}\gamma^{(a)}$ changes sign. Moreover $\Gamma B\Gamma = -B$.

Proof. Line by line:

$$\bar{\Psi}' = (\Gamma\Psi)^\dagger C = \Psi^\dagger \Gamma^\dagger C.$$

This is the definition of the adjoint and the rule $(MN)^\dagger = N^\dagger M^\dagger$ (the dagger of a product reverses the order).

$$\Psi^\dagger \Gamma^\dagger C = \Psi^\dagger \Gamma C = \Psi^\dagger C \Gamma = \bar{\Psi}\Gamma.$$

Γ is real and symmetric, so $\Gamma^\dagger = \Gamma^T = \Gamma$; then $\Gamma C = C\Gamma$ (Lemma 1); then the definition $\bar{\Psi} = \Psi^\dagger C$.

$$\bar{\Psi}' X \Psi' = \bar{\Psi}\,\Gamma X \Gamma\,\Psi.$$

We inserted $\bar{\Psi}' = \bar{\Psi}\Gamma$ and $\Psi' = \Gamma\Psi$.

$$\Gamma X \Gamma = (-1)^k X\,\Gamma\Gamma = (-1)^k X.$$

Moving the left Γ through the k factors of X costs $(-1)^k$ (Lemma 1: Γ anticommutes with each gamma); then $\Gamma\Gamma = 1$. For S, $X = 1$ and $k = 0$; for the kinetic kernels $k = 1$; for the connection kernels $\gamma^{(a)} S^{bc}$ and $S^{bc}\gamma^{(a)}$ with $b \neq c$, $k = 3$ (when a equals b or c, two factors combine to η_{aa} and one gamma is left, $k = 1$: odd in every case). Finally

$$\Gamma B \Gamma = -i\,\Gamma C \gamma^{(x_4)} \Gamma = -i\,C\,\Gamma\gamma^{(x_4)}\Gamma = -i\,C(-\gamma^{(x_4)}) = +iC\gamma^{(x_4)} = -B.$$

The definition $B = -iC\gamma^{(x_4)}$; then $\Gamma C = C\Gamma$; then the line before with $k = 1$. QED. Nowhere were two field components exchanged: only the numbers of the matrices were moved, and every Ψ^* still stands to the left of every Ψ. So Lemma 2 holds for commuting and for Grassmann components alike.

statement	**status**	**where it is verified**
Lemma 2 for S and the 456 kinetic and connection kernels	PROVED	`python-pairing.json`, check `T1.general_field.matrix_identities`; `wolfram-pairing.json`, checks `T1_kernel_scalar` and `T1_kernel_kinetic`
Lemma 2 for all 512 triples (a, b, c) and the field-equation kernels	PROVED	`wolfram-pairing.json`, checks `T1_kernel_connection` and `T1_kernel_field_equation`
$\Gamma B \Gamma = -B$	PROVED	`wolfram-pairing.json`, check `Q_Krein_metric_of_images`; `python-pairing.json`, check `Q.image_krein_metric`

statement	status	where it is verified
the sign $(-1)^k$ for all 256 products of different gammas	PROVED	Notebook 18a, In [9] and In [10]

Lemma 3 (the eight reflections). Let n be one of the eight directions and R_n its reflection.

1. $(\gamma^{(n)})^T C \gamma^{(n)} = -\eta_{nn} C$.
2. $\gamma^{(n)}\gamma^{(a)}(\gamma^{(n)})^{-1} = -(R_n)_{aa}\gamma^{(a)}$, and $\gamma^{(n)} S^{ab} (\gamma^{(n)})^{-1} = (R_n)_{aa}(R_n)_{bb} S^{ab}$.
3. $P_n := \Gamma\gamma^{(n)}$ is plus or minus the product of the seven other gammas, hence an element of Pin(4,4); $P_n^{-1}\gamma^{(a)}P_n = (R_n)_{aa}\gamma^{(a)}$ (it **covers** the reflection R_n: it acts on the gammas exactly as R_n acts on the directions); its character is $\chi_n = -\eta_{nn}$, that is $P_n^\dagger C P_n = -\eta_{nn} C$: -1 for the space-like x_0, x_1, x_2, x_3 and $+1$ for the time-like x_4, x_5, x_6, x_7; and $\Gamma P_n = \gamma^{(n)}$.

Proof of 1. Multiply $C\gamma^{(n)}C^{-1} = -(\gamma^{(n)})^T$ on the right by C: $C\gamma^{(n)} = -(\gamma^{(n)})^T C$. Then

$$(\gamma^{(n)})^T C \gamma^{(n)} = (\gamma^{(n)})^T\big(-(\gamma^{(n)})^T C\big) = -\big((\gamma^{(n)})^T\big)^2 C = -\big((\gamma^{(n)})^2\big)^T C = -\eta_{nn} C.$$

We replaced $C\gamma^{(n)}$, collected the two transposes, used $(M^T)^2 = (M^2)^T$ and the Clifford relation $(\gamma^{(n)})^2 = \eta_{nn} I_{16}$.

Proof of 2. $(\gamma^{(n)})^{-1} = \eta_{nn}\gamma^{(n)}$, because $\gamma^{(n)}\eta_{nn}\gamma^{(n)} = \eta_{nn}^2 = 1$. For $a \neq n$: $\gamma^{(n)}\gamma^{(a)}(\gamma^{(n)})^{-1} = -\gamma^{(a)}\gamma^{(n)}(\gamma^{(n)})^{-1} = -\gamma^{(a)}$ (anticommute, then cancel), and $-(R_n)_{aa} = -1$. For $a = n$: $\gamma^{(n)}\gamma^{(n)}(\gamma^{(n)})^{-1} = \gamma^{(n)}$, and $-(R_n)_{nn} = +1$. The statement for $S^{ab} = \frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ $(a \neq b)$ follows by inserting $(\gamma^{(n)})^{-1}\gamma^{(n)} = 1$ between the two factors: the two signs $-(R_n)_{aa}$ and $-(R_n)_{bb}$ multiply to $(R_n)_{aa}(R_n)_{bb}$.

Proof of 3. $\Gamma\gamma^{(n)}$ is the ordered product of the eight gammas followed by $\gamma^{(n)}$. Moving the last factor to its place next to the factor $\gamma^{(n)}$ inside Γ passes some of the other gammas, each costing a sign (Rule P); then $\gamma^{(n)}\gamma^{(n)} = \eta_{nn}$. So $P_n = \pm\eta_{nn}\times$ (the product of the seven other gammas in their order), a product of unit vectors, which lies in Pin(4,4). Next, with $(\gamma^{(n)})^{-1} = \eta_{nn}\gamma^{(n)}$ and $\Gamma^{-1} = \Gamma$:

$$P_n^{-1}\gamma^{(a)}P_n = (\gamma^{(n)})^{-1}\Gamma\gamma^{(a)}\Gamma\gamma^{(n)} = -(\gamma^{(n)})^{-1}\gamma^{(a)}\gamma^{(n)} = (R_n)_{aa}\gamma^{(a)}.$$

The inverse of a product reverses the order; $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}$ (Lemma 2 with $k = 1$); and item 2, read with $(\gamma^{(n)})^{-1}$ on the left, which gives the same signs. For the character, P_n is real, so $P_n^\dagger = P_n^T = (\gamma^{(n)})^T\Gamma^T = (\gamma^{(n)})^T\Gamma$:

$$P_n^\dagger C P_n = (\gamma^{(n)})^T\,\Gamma C \Gamma\,\gamma^{(n)} = (\gamma^{(n)})^T C \gamma^{(n)} = -\eta_{nn} C.$$

We used $\Gamma C\Gamma = C\Gamma\Gamma = C$ (Lemma 1) and item 1. Finally $\Gamma P_n = \Gamma\Gamma\gamma^{(n)} = \gamma^{(n)}$. QED. The signs of P_n relative to the product of the other seven gammas, in the order of Γ, are $-1, +1, -1, +1, +1, -1, +1, -1$ for $n = x_0, \ldots, x_7$ (computed exactly; they equal the data table `reflections` of `pairing-theory.json`).

statement	status	where it is verified
Lemma 3, items 1 to 3	PROVED	`wolfram-pairing.json`, checks `T2_Pn_in_Pin44`, `T2_Pn_covers_the_reflection`, `T2_character_of_Pn`, `T2_Gamma_times_Pn_is_gamma_n` and `T2_kernels_gamma_n_with_frame_reflection`
the reflection table	PROVED	`python-pairing.json`, checks `T2.general_field.matrix_identities`, `T2.general_field.reflection_table` and `compare.theory.reflection_table`; Notebook 18a, In [12] and In [13]

Lemma 4 (the Krein signs of the maps). $MBM^\dagger = \sigma_M B$, with $\sigma_\Gamma = -1$; $\sigma_{\gamma^{(n)}} = +1$ for $n = x_0, x_1, x_2, x_3, x_4$ and -1 for $n = x_5, x_6, x_7$; and $\sigma_{P_n} = -\sigma_{\gamma^{(n)}}$.

Proof. Γ is real and symmetric, so $\Gamma B \Gamma^\dagger = \Gamma B \Gamma = -B$ by Lemma 2. For $M = \gamma^{(n)}$: transposing $C\gamma^{(n)}C^{-1} = -(\gamma^{(n)})^T$ and using $C^T = C = C^{-1}$ gives $\gamma^{(n)}C = -C(\gamma^{(n)})^T$. Then

$$\begin{aligned}\gamma^{(n)} B (\gamma^{(n)})^\dagger &= -i\,\gamma^{(n)} C \gamma^{(x_4)} (\gamma^{(n)})^T = i\,C(\gamma^{(n)})^T \gamma^{(x_4)} (\gamma^{(n)})^T \\ &= i\,\eta_{nn}^2\, C\gamma^{(n)}\gamma^{(x_4)}\gamma^{(n)} = i\,C\gamma^{(n)}\gamma^{(x_4)}\gamma^{(n)}.\end{aligned}$$

The definition of B and $(\gamma^{(n)})^\dagger = (\gamma^{(n)})^T$ (real matrix); then the relation $\gamma^{(n)}C = -C(\gamma^{(n)})^T$; then $(\gamma^{(n)})^T = \eta_{nn}\gamma^{(n)}$ twice; then $\eta_{nn}^2 = 1$. For $n \neq 4$: $\gamma^{(n)}\gamma^{(x_4)}\gamma^{(n)} = -\gamma^{(x_4)}\gamma^{(n)}\gamma^{(n)} = -\eta_{nn}\gamma^{(x_4)}$, so the result is $-i\eta_{nn}C\gamma^{(x_4)} = \eta_{nn}B$: $+B$ for x_0, x_1, x_2, x_3 and $-B$ for x_5, x_6, x_7. For $n = 4$: $\gamma^{(4)}\gamma^{(4)}\gamma^{(4)} = -\gamma^{(4)}$, so the result is $-iC\gamma^{(4)} = +B$. For $P_n = \Gamma\gamma^{(n)}$: $P_n B P_n^\dagger = \Gamma\,\gamma^{(n)} B (\gamma^{(n)})^\dagger\,\Gamma = \sigma_{\gamma^{(n)}}\Gamma B \Gamma = -\sigma_{\gamma^{(n)}} B$. QED.

statement	status	where it is verified
Lemma 4, the 17 Krein signs	PROVED	`wolfram-pairing.json`, check `Q_Krein_metric_of_images`; `python-pairing.json`, checks `compare.theory.krein_signs` and `Q.T2_image_keeps_B`; data table `Krein_signs_M_B_Mdagger` of `pairing-theory.json`; Notebook 18a, In [17]

The block form. Split $\Psi = (\psi_-, \psi_+)$ into its components 1 to 8 and 9 to 16. Because every gamma anticommutes with $\Gamma = \mathrm{diag}(-I_8, I_8)$, every gamma has nonzero entries only in the two off-diagonal

8×8 blocks (Chapter 5): it exchanges the two halves. The author's $\gamma^{(x_0)}$ has the identity I_8 in both off-diagonal blocks, so $\gamma^{(x_0)}(\psi_-, \psi_+) = (\psi_+, \psi_-)$. C, a product of four gammas, commutes with Γ and is block diagonal; for the author's gammas $C = \mathrm{diag}(-\sigma, \sigma)$ with a real symmetric 8×8 matrix σ, so

$$S = \Psi^\dagger C \Psi = -\psi_-^\dagger \sigma \psi_- + \psi_+^\dagger \sigma \psi_+ .$$

Reading this line: reversing the sign of one half (Γ) keeps S; exchanging the two halves ($\gamma^{(x_0)}$) reverses S. The two halves are the two inequivalent irreducible 8-dimensional representations of Spin(4,4) (Chapter 5).

statement	status	where it is verified
the block form: every gamma exchanges the halves, $\gamma^{(x_0)}$ swaps them, $C = \mathrm{diag}(-\sigma, \sigma)$	PROVED	Notebook 18a, In [16]
the two halves are the two inequivalent irreducible representations of Spin(4,4), exchanged by every gamma	PROVED	`python-algebra.json`, checks `spin_halves_irreducible`, `spin_halves_inequivalent` and `reflections_exchange_halves`; `wolfram-algebra.json`, check `Spin_Pin_reflection_swaps_halves`

18.6 Example: Notebook 18a checks the four lemmas

Notebook 18a first answers the question that every later step depends on: are the matrices of the Revision record really the author's? It rebuilds the eight matrices T16A[0], ..., T16A[7] from the author's own formulas (the tau matrices), maps the author's frame order (0 = hidden, 1 to 3 = 3-space, 4 = time, 5 to 7 = extra times) to the coordinates $x_0, \ldots, x_7$, and compares them entry by entry with `Revision/algebra/gammas.json`; it checks that they are real signed permutation matrices with entries -1, 0, $+1$ and that the 64 Clifford relations hold. Then it proves Lemmas 1 to 4 and the block form with exact whole-number arithmetic, compares the reflection table and the table of Krein signs with the data tables of the pairing record row by row, and draws seven teaching figures: the eight gammas, the sign flip $\Gamma\gamma^{(x_1)}\Gamma = -\gamma^{(x_1)}$, the parity of the 256 products, the reflection table, the chiral projectors, the block form and the Krein signs. Its last line is ALL 16 CHECKS PASSED (notebook 18a); it runs in about 15 seconds.

18.7 How to run Notebook 18a

Step 1. What this notebook does and what it needs.

Notebook 18a (The matrix facts behind the pairing theorems T1, T2 and Q) is the file `Revision/textbook/notebooks/18a_pairing_matrices.ipynb` of the repository Dirac_claude. It rebuilds the author's eight real 16 by 16 gamma matrices from the author's own formulas (the tau matrices),

checks that they equal the matrices of the Revision record, that every entry is -1, 0 or +1 and that they obey the Clifford relations, and builds C, the chirality Gamma, the chiral projectors, the Krein matrix B and the generators S^ab. With exact whole-number matrix arithmetic it then proves the four lemmas on which the pairing theorems rest: Gamma anticommutes with every gamma and commutes with C and with every S^ab; under the map from Psi to Gamma Psi a bilinear with k gamma factors gets the sign (-1)^k, so the scalar S is unchanged while all 456 kinetic and connection matrices of the Lagrangian change sign; the eight reflections P_n = Gamma gamma^n lie in Pin(4,4), act on the gammas as the reflections of the eight directions and have the character -1 for the space-like and +1 for the time-like directions; and the Krein signs of the 17 maps. It reproduces the reflection table and the Krein table of the Revision pairing record and the matrix checks of its two reports, and draws seven teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 18a_pairing_matrices.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 18a_pairing_matrices.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 18a_pairing_matrices.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/18a.captions.json`
- `Revision/textbook/figures/18a_1_eight_gammas.png`
- `Revision/textbook/figures/18a_2_chirality_flips_gammas.png`
- `Revision/textbook/figures/18a_3_parity_of_products.png`
- `Revision/textbook/figures/18a_4_reflection_table.png`
- `Revision/textbook/figures/18a_5_chiral_projectors.png`
- `Revision/textbook/figures/18a_6_chiral_blocks.png`
- `Revision/textbook/figures/18a_7_krein_signs.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 16 CHECKS PASSED (notebook 18a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/18a_pairing_matrices.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 18a_pairing_matrices.ipynb
python -m nbconvert --execute --inplace 18a_pairing_matrices.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for gammas.json or pairing-theory.json: the notebook reads four files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.

18.8 Notebook 18a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/18a_pairing_matrices.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The pairing theorems of this chapter say what happens to the Lagrangian, the field
equations, the energy-momentum tensor and the current of the fields dirac16complex
and dirac16complex00 when the field $\Psi$ is replaced by $M\Psi$ for a fixed
$16 \times 16$ matrix $M$. Every step of their proofs is a statement about such
matrices. This notebook first shows that the matrices used are exactly the
author's: it rebuilds the eight real $16 \times 16$ gamma matrices from the
author's own formulas and compares them entry by entry with the Revision record.
Then it proves all the matrix statements of the theorems exactly, with
whole-number arithmetic (no rounding anywhere):

- **Lemma 1 (the chirality).** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots
  \gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$ is real and symmetric, $\Gamma^2 = 1$,
  $\Gamma$ anticommutes with each of the eight gammas and commutes with $C$ and with
  every generator $S^{ab}$; hence it commutes with every spin connection.
- **Lemma 2 (bilinears).** Under $\Psi \to \Gamma\Psi$ a bilinear
  $\bar\Psi M\Psi$ whose matrix $M$ is a product of $k$ gammas is multiplied by
  $(-1)^k$. The notebook checks this for all 256 products of different gammas and
  for all 456 matrices that occur in the kinetic and connection terms of the
  Lagrangian (they all change sign), while the scalar $S = \bar\Psi\Psi$ does not.
- **Lemma 3 (reflections).** For each direction $n$ the matrix
  $P_n = \Gamma\gamma^{(n)}$ is, up to a sign, the product of the other seven
  gammas (so it belongs to the group Pin(4,4)); it acts on the gammas as the
  reflection of the direction $n$; its *character* is $-1$ for the space-like
  directions $x_0, x_1, x_2, x_3$ and $+1$ for the time-like ones
  $x_4, x_5, x_6, x_7$. The notebook rebuilds the reflection table of the Revision
  pairing record row by row.
- **Lemma 4 (Krein signs).** $MBM^\dagger = \pm B$ for the 17 maps
  $M = \Gamma, \gamma^{(n)}, P_n$, with the signs of the Revision record.

It also builds the two chiral projectors and the block form behind the theorems
(the two chiral halves) and draws seven teaching figures.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 18a (The matrix facts behind the pairing theorems T1, T2 and Q) is the file
```

‘Revision/textbook/notebooks/18a_pairing_matrices.ipynb‘ of the repository Dirac_claude. It rebuilds the author’s eight real 16 by 16 gamma matrices from the author’s own formulas (the tau matrices), checks that they equal the matrices of the Revision record, that every entry is -1, 0 or +1 and that they obey the Clifford relations, and builds C, the chirality Gamma, the chiral projectors, the Krein matrix B and the generators S^ab. With exact whole-number matrix arithmetic it then proves the four lemmas on which the pairing theorems rest: Gamma anticommutes with every gamma and commutes with C and with every S^ab; under the map from Psi to Gamma Psi a bilinear with k gamma factors gets the sign (-1)^k, so the scalar S is unchanged while all 456 kinetic and connection matrices of the Lagrangian change sign; the eight reflections P_n = Gamma gamma^n lie in Pin(4,4), act on the gammas as the reflections of the eight directions and have the character -1 for the space-like and +1 for the time-like directions; and the Krein signs of the 17 maps. It reproduces the reflection table and the Krein table of the Revision pairing record and the matrix checks of its two reports, and draws seven teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 18a_pairing_matrices.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 18a_pairing_matrices.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 18a_pairing_matrices.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/18a.captions.json‘
- ‘Revision/textbook/figures/18a_1_eight_gammas.png‘
- ‘Revision/textbook/figures/18a_2_chirality_flips_gammas.png‘
- ‘Revision/textbook/figures/18a_3_parity_of_products.png‘
- ‘Revision/textbook/figures/18a_4_reflection_table.png‘
- ‘Revision/textbook/figures/18a_5_chiral_projectors.png‘
- ‘Revision/textbook/figures/18a_6_chiral_blocks.png‘
- ‘Revision/textbook/figures/18a_7_krein_signs.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 16 CHECKS PASSED (notebook 18a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
    python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/18a_pairing_matrices.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
    python -m jupyterlab 18a_pairing_matrices.ipynb
    python -m nbconvert --execute --inplace 18a_pairing_matrices.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for gammas.json or pairing-theory.json: the notebook reads four files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository (a notebook copied alone to another folder cannot find them). Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 18a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 18a (The matrix facts behind the pairing theorems T1, T2 and Q) is the file
```

```
# Revision/textbook/notebooks/18a_pairing_matrices.ipynb of the repository Dirac_claude.
# It rebuilds the author's eight real 16 by 16 gamma matrices from the author's own
# formulas (the tau matrices), checks that they equal the matrices of the Revision
# record, that every entry is -1, 0 or +1 and that they obey the Clifford relations, and
# builds C, the chirality Gamma, the chiral projectors, the Krein matrix B and the
# generators S^ab. With exact whole-number matrix arithmetic it then proves the four
# lemmas on which the pairing theorems rest: Gamma anticommutes with every gamma and
# commutes with C and with every S^ab; under the map from Psi to Gamma Psi a bilinear
# with k gamma factors gets the sign (-1)^k, so the scalar S is unchanged while all 456
# kinetic and connection matrices of the Lagrangian change sign; the eight reflections
# P_n = Gamma gamma^n lie in Pin(4,4), act on the gammas as the reflections of the eight
# directions and have the character -1 for the space-like and +1 for the time-like
# directions; and the Krein signs of the 17 maps. It reproduces the reflection table and
# the Krein table of the Revision pairing record and the matrix checks of its two
# reports, and draws seven teaching figures. It needs a computer with Windows 11, macOS
# or Linux, an internet connection for the installation, the program Git, and Python 3.12
# or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
# these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
```

```
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
```

```
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 18a_pairing_matrices.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 18a_pairing_matrices.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
```

```
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 18a_pairing_matrices.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/18a.captions.json
# - Revision/textbook/figures/18a_1_eight_gammas.png
# - Revision/textbook/figures/18a_2_chirality_flips_gammas.png
# - Revision/textbook/figures/18a_3_parity_of_products.png
# - Revision/textbook/figures/18a_4_reflection_table.png
# - Revision/textbook/figures/18a_5_chiral_projectors.png
# - Revision/textbook/figures/18a_6_chiral_blocks.png
# - Revision/textbook/figures/18a_7_krein_signs.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 16 CHECKS PASSED (notebook 18a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/18a_pairing_matrices.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
```

```
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 18a_pairing_matrices.ipynb
#     python -m nbconvert --execute --inplace 18a_pairing_matrices.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for gammas.json or pairing-theory.json: the notebook reads four
#   files of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository (a notebook copied
#   alone to another folder cannot find them). Clone the repository again and open the
#   notebook there.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "18a"  # this notebook: chapter 18, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
```

```
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 18a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Matrix, entry, product**: a matrix is a table of numbers; $M_{AB}$ is the entry
  in row $A$ and column $B$; in code ‘M @ N‘ is the matrix product and
  ‘np.eye(16)‘ the identity $I_{16}$ (1 on the diagonal, 0 elsewhere).
- **Transpose, dagger**: $M^T$ exchanges rows and columns; $M^\dagger = (M^T)^*$
  also replaces every $i$ by $-i$. For a real matrix $M^\dagger = M^T$.
- **Real matrix**: a matrix whose entries are real numbers (no $i$). The author’s
```

```
  gammas are even simpler: every entry is $-1$, $0$ or $+1$.
- **Signed permutation matrix**: a matrix with exactly one nonzero entry, $+1$ or
  $-1$, in every row and in every column. Multiplying a column of 16 numbers by it
  reorders the numbers and changes some signs.
- **Symmetric, antisymmetric**: $M^T = M$, respectively $M^T = -M$.
- **Commute, anticommute**: $MN = NM$, respectively $MN = -NM$.
- **Gamma matrices** $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$: the author's real
  $16 \times 16$ matrices with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}
  = 2\eta^{ab}I_{16}$ (the *Clifford relation*), $\eta = \mathrm{diag}(+1, +1, +1,
  +1, -1, -1, -1, -1)$ in the order $x_0, \dots, x_7$.
- **Space-like, time-like direction**: $\eta_{nn} = +1$ ($x_1, x_2, x_3$ and the
  hidden direction $x_0$), respectively $\eta_{nn} = -1$ (the time $x_4$ and the
  three EXTRA TIMES $x_5, x_6, x_7$, which deflate exponentially in the author's
  metric).
- **$C$**: $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, the
  product of the four space-like gammas (the author's sigma16). The Dirac adjoint
  is $\bar\Psi = \Psi^\dagger C$.
- **Chirality** $\Gamma$: the product of all eight gammas in the order
  $x_0, x_1, \dots, x_7$ (the author's T16A[8]).
- **Projector**: a matrix $P$ with $P^2 = P$. The *chiral projectors*
  $P_- = \frac12(1 - \Gamma)$ and $P_+ = \frac12(1 + \Gamma)$ (the author's P_L and
  P_R) keep one half of the 16 components and set the other half to zero.
- **$B$**: $B = -iC\gamma^{(x_4)}$, the Krein matrix; the charge density of the
  field is $\Psi^\dagger B\Psi$ and $B$ is the matrix of the canonical
  anticommutator of the quantised field.
- **Generators** $S^{ab} = \frac14[\gamma^{(a)}, \gamma^{(b)}]$, where
  $[M, N] = MN - NM$ is the commutator; for $a \neq b$, $S^{ab} = \frac12
  \gamma^{(a)}\gamma^{(b)}$. A spin connection is $\Omega_\mu = \frac12\sum_{a,b}
  \omega_{\mu ab}S^{ab}$ with numbers or functions $\omega_{\mu ab}$.
- **Bilinear**: an expression $\Psi^\dagger X\Phi = \sum_{A,B}\Psi_A^* X_{AB}
  \Phi_B$ with a matrix $X$ of numbers; $X$ is called its *kernel*.
- **Reflection** $R_n$: the $8 \times 8$ diagonal matrix with $-1$ in place $n$
  and $+1$ elsewhere; it reverses the direction $n$.
- **Pin(4,4)**: the group of all products of unit vectors $\sum_a v_a\gamma^{(a)}$
  with $\sum_a\eta_{aa}v_a^2 = \pm1$ and their inverses; each single
  $\gamma^{(a)}$ is such a unit vector, so every product of gammas belongs to it.
- **Character** $\chi$ of a map $M$: the sign in $M^\dagger CM = \chi C$; it tells
  whether the scalar $S = \bar\Psi\Psi$ keeps ($\chi = +1$) or changes ($\chi = -1$)
  its sign under $\Psi \to M\Psi$.
- **Krein sign** $\sigma$ of a map $M$: the sign in $MBM^\dagger = \sigma B$.
- **Chiral halves**: the components 1 to 8 of $\Psi$ (where $\Gamma = -1$, written
  $\psi_-$) and 9 to 16 (where $\Gamma = +1$, written $\psi_+$).
- **Exact**: the gammas, $C$, $\Gamma$, $4S^{ab}$ and $B/i$ have whole-number
  entries; numpy multiplies whole numbers without rounding, so every equality
  checked here is an exact identity, a proof for that identity.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's coordinates are $x_1, x_2, x_3$ (ordinary 3-space, inflating),
$x_4$ (the time), $x_5, x_6, x_7$ (the three extra times, deflating exponentially:
their scale factor is $e^{-a_4}\sin^{1/6}z$ with $a_4$ increasing) and $x_0$ (the
```

hidden space direction, $z = 6Hx_0$). Both fields have the Lagrangian density

$$\mathcal{L}_{m,\lambda}[\Psi] = \sqrt{|g|}\,\Big[K - mS - \frac{\lambda}{2}S^2
\Big],\qquad K = \frac12\big(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)
\gamma^\mu\Psi\big),\qquad S = \bar\Psi\Psi ,$$

with $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and $\gamma^\mu =
\sum_a e^\mu{}_a\gamma^{(a)}$. The pairing theorems are:

- **T1**: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$ in
  every gravitational field; solutions of the (m, λ) equations go to
  solutions of the $(-m, -\lambda)$ equations; the energy-momentum tensor and the
  current change sign.
- **T2**: with the reflection of a space-like direction n,
  $\mathcal{L}_{m,\lambda}[\gamma^{(n)}\Psi;\,R_ne] =
  +\mathcal{L}_{-m,\lambda}[\Psi;\,e]$: the mass changes sign, the coupling does
  not, and the energy-momentum tensor is NOT reversed.
- **Q** (the quantised dirac16complex): the image $\Gamma\Psi$ carries the Krein
  matrix $-B$ instead of B.

Why matrix identities are enough, line by line. Let $\Psi' = M\Psi$ with a
constant real matrix M. The dagger of a product reverses the order:

$$\Psi'^\dagger = (M\Psi)^\dagger = \Psi^\dagger M^\dagger = \Psi^\dagger M^T .$$

Multiply on the right by C to get the adjoint of the new field:

$$\bar\Psi' = \Psi'^\dagger C = \Psi^\dagger M^TC .$$

For $M = \Gamma$: $\Gamma^T = \Gamma$ (Lemma 1) and $\Gamma C = C\Gamma$ (Lemma 1),
so $\bar\Psi' = \Psi^\dagger\Gamma C = \Psi^\dagger C\Gamma = \bar\Psi\Gamma$. A
bilinear with a matrix X in the middle therefore becomes

$$\bar\Psi' X\Psi' = \bar\Psi\,\Gamma X\Gamma\,\Psi .$$

If X is a product of k gammas, moving the left Γ through the k
factors costs k signs (Lemma 1: $\Gamma\gamma^{(a)} = -\gamma^{(a)}\Gamma$), and
then $\Gamma\Gamma = 1$:

$$\Gamma X\Gamma = (-1)^k X\,\Gamma\Gamma = (-1)^k X .$$

So $S = \bar\Psi\Psi$ ($k = 0$) is unchanged, and every kinetic term
$\bar\Psi\gamma^{(a)}\partial_\mu\Psi$ ($k = 1$) changes sign. Only constant
numbers were moved, never two field components, so the same lines hold for
commuting components (dirac16complex00) and for anticommuting (Grassmann)
components (dirac16complex). The coefficients $\sqrt{|g|}$, $e^\mu{}_a$ and
$\omega_{\mu ab}$ of the gravitational field are ordinary functions that multiply
these bilinears; they are untouched by the map. That is why T1 holds in EVERY
gravitational field, and why this notebook needs only matrices.

Text cell:

5. The author's eight real gamma matrices, C, Γ and B

The next cell imports numpy, defines the helpers that read the Revision records, reads the author's gamma matrices from `Revision/algebra/gammas.json` (whole numbers) and checks the 64 Clifford relations. The helper `reproduces` is a check that passes only when this notebook's own result holds AND the named checks of the named Revision reports have the verdict PASS; it prints the report files and the check names.

In [2]:

```
import itertools  # all subsets of a list (for the 256 products of gammas)
from math import comb  # binomial coefficients: comb(8, k) = number of k-subsets

import numpy as np  # arrays and matrices

GAMMAS = "Revision/algebra/gammas.json"
THEORY = "Revision/pairing/pairing-theory.json"
PY = "Revision/pairing/reports/python-pairing.json"  # the sympy verifier's report
WL = "Revision/pairing/reports/wolfram-pairing.json"  # the Wolfram verifier's report


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def verdict(report_file, name):
    """The recorded verdict (PASS or FAIL) of the check name of a report."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper()
    return "MISSING"


def reproduces(condition, name, *sources, table=None):
    """check(condition, name), which also requires every named check of every
    source (report_file, [check names]) to have the recorded verdict PASS; table
    names a data table of the theory record that the result equals."""
    recorded = all(verdict(report_file, n) == "PASS"
                   for report_file, names in sources for n in names)
    parts = [f"{report_file}, check {', '.join(names)}"
             for report_file, names in sources]
    if table is not None:
        parts.insert(0, f"{THEORY}, data table {table}")
    check(condition and recorded, name, record="; ".join(parts))


fixture = read_json(GAMMAS)
gamma = {a: np.array(fixture["gamma"][a], dtype=np.int64) for a in range(8)}
eta = {a: int(fixture["eta"][a]) for a in range(8)}  # +1 or -1
I16 = np.eye(16, dtype=np.int64)
```

```
def same(M, N):
    """True when the two matrices are equal entry by entry (exact for integers)."""
    return np.array_equal(M, N)


say("eta = " + str([eta[a] for a in range(8)]) + " for x0, ..., x7")
reproduces(all(same(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                    2 * (eta[a] if a == b else 0) * I16)
               for a in range(8) for b in range(8)),
           "the 64 Clifford relations hold exactly",
           (WL, ["fixture_Clifford_relation"]), (PY, ["gammas.clifford"]))
```

Out [2]:

```
eta = [1, 1, 1, 1, -1, -1, -1, -1] for x0, ..., x7
PASS the 64 Clifford relations hold exactly
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    fixture_Clifford_relation; Revision/pairing/reports/python-pairing.json, check
    gammas.clifford
```

Text cell:

```
**Are these the author's matrices?** The author's notebook defines its gamma
matrices T16A[0], ..., T16A[7] by explicit formulas, and the next cell repeats
those formulas step by step and compares the result with the record. The steps:

1. For $h = 1, 2, 3$ two $4 \times 4$ matrices with entries
   $(s_h)_{pq} = Q_a - Q_b$ and $(t_h)_{pq} = Q_a + Q_b$ ($p, q = 1, \dots, 4$),
   where $Q_a$ is the sign of the arrangement $(h, p, q, 4)$ (the *permutation
   sign*: $+1$ or $-1$ according to whether an even or odd number of exchanges
   sorts it, and 0 when two of the four numbers are equal) and
   $Q_b = \delta_{p4}\delta_{qh} - \delta_{ph}\delta_{q4}$ with the Kronecker
   delta ($\delta_{pq} = 1$ for $p = q$, else 0).
2. Eight $8 \times 8$ matrices $\tau_0 = I_8$, $\tau_h = \begin{pmatrix} 0 & s_h
   \\ s_h & 0 \end{pmatrix}$, $\tau_{7-h} = \begin{pmatrix} 0 & t_h \\ -t_h & 0
   \end{pmatrix}$ ($h = 1, 2, 3$) and $\tau_7 = \tau_1\tau_2\cdots\tau_6$.
3. $\bar\tau_A = \sigma\tau_A^T\sigma$ with $\sigma = \begin{pmatrix} 0 & I_4 \\
   I_4 & 0 \end{pmatrix}$.
4. $\mathrm{T16A}[A] = \begin{pmatrix} 0 & \bar\tau_A \\ \tau_A & 0
   \end{pmatrix}$, $A = 0, \dots, 7$.
5. The author's notebook counts its frame from 0 (0 = hidden direction, 1 to 3 =
   3-space, 4 = time, 5 to 7 = extra times); in the author's coordinates
   $\gamma^{(x_0)} = $ T16A[0], $\gamma^{(x_n)} = $ T16A[n] for $n = 1, \dots, 7$.

The cell then checks that the record holds exactly eight matrices, each with 16
rows and 16 columns, each REAL with every entry equal to $-1$, $0$ or $+1$, each a
signed permutation matrix, with $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$, and that
they equal the rebuilt T16A entry by entry.
```

In [3]:

```
def perm_sign(seq):
    """The permutation sign of a list of numbers: +1 (even number of exchanges
    sorts it), -1 (odd number), 0 (two entries are equal)."""
    if len(set(seq)) != len(seq):
        return 0
    inversions = sum(1 for i in range(len(seq)) for j in range(i + 1, len(seq))
                     if seq[i] > seq[j])  # pairs in the wrong order
    return -1 if inversions % 2 else 1


def kd(p, q):
    """The Kronecker delta: 1 if p = q, else 0."""
    return 1 if p == q else 0


I4, Z4 = np.eye(4, dtype=np.int64), np.zeros((4, 4), dtype=np.int64)
I8, Z8 = np.eye(8, dtype=np.int64), np.zeros((8, 8), dtype=np.int64)
s4, t4 = {}, {}
for h in (1, 2, 3):  # step 1: the self-dual and anti-self-dual 4 x 4 blocks
    Qa = np.array([[perm_sign([h, p, q, 4]) for q in range(1, 5)]
                   for p in range(1, 5)])
    Qb = np.array([[kd(p, 4) * kd(q, h) - kd(p, h) * kd(q, 4) for q in range(1, 5)]
                   for p in range(1, 5)])
    s4[h], t4[h] = Qa - Qb, Qa + Qb
tau = {0: I8}  # step 2: the eight tau matrices
for h in (1, 2, 3):
    tau[h] = np.block([[Z4, s4[h]], [s4[h], Z4]])
    tau[7 - h] = np.block([[Z4, t4[h]], [-t4[h], Z4]])
tau[7] = tau[1] @ tau[2] @ tau[3] @ tau[4] @ tau[5] @ tau[6]
sigma4 = np.block([[Z4, I4], [I4, Z4]])  # step 3: tau-bar
taubar = {A: sigma4 @ tau[A].T @ sigma4 for A in range(8)}
T16A = {A: np.block([[Z8, taubar[A]], [tau[A], Z8]]) for A in range(8)}  # step 4
frame_index = {a: a for a in range(8)}  # step 5: x_a -> A = a
rebuilt_equal = all(same(T16A[frame_index[a]], gamma[a]) for a in range(8))
raw = fixture["gamma"]  # the numbers exactly as stored in the record
eight_real = (len(raw) == 8 and all(len(M) == 16 and all(len(row) == 16 for row in M)
                                    for M in raw)
              and all(type(x) is int and x in (-1, 0, 1)
                      for M in raw for row in M for x in row))
signed_perm = all((np.abs(gamma[a]).sum(axis=0) == 1).all()
                  and (np.abs(gamma[a]).sum(axis=1) == 1).all() for a in range(8))
transpose_rule = all(same(gamma[a].T, eta[a] * gamma[a]) for a in range(8))
say(f"8 matrices of 16 x 16 real entries -1, 0, +1: {eight_real}; signed "
    f"permutation matrices: {signed_perm}; transpose = eta_aa times the matrix: "
    f"{transpose_rule}; equal to the author's T16A rebuilt from the formulas: "
    f"{rebuilt_equal}")
reproduces(eight_real and signed_perm and transpose_rule and rebuilt_equal,
           "the record holds the author's eight real 16 x 16 gamma matrices",
           (PY, ["gammas.equal_wolfram_fixture"]))
```

Out [3]:

```
8 matrices of 16 x 16 real entries -1, 0, +1: True; signed permutation matrices: True;
```

```
    transpose = eta_aa times the matrix: True; equal to the author's T16A rebuilt from
    the formulas: True
PASS the record holds the author's eight real 16 x 16 gamma matrices
     reproduces Revision/pairing/reports/python-pairing.json, check
    gammas.equal_wolfram_fixture
```

Text cell:

```
The next cell draws the eight gamma matrices as coloured grids (*heat maps*): red
for $+1$, blue for $-1$, light grey for 0. It first defines the colours of all
figures of this notebook (a blue, an orange, a green and a grey that stay
distinguishable for colour-blind readers, and the blue-grey-red colour scale for
matrices) and a helper ‘draw_matrix‘ that draws one matrix with thin lines that
separate the two chiral halves (rows and columns 1 to 8 and 9 to 16).
```

In [4]:

```
from matplotlib.colors import LinearSegmentedColormap  # colour scales

BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])


def draw_matrix(ax, matrix, title, ticks=True):
    """Heat map of a real 16 x 16 matrix with entries between -1 and +1."""
    image = ax.imshow(matrix, cmap=DIVERGING, vmin=-1.0, vmax=1.0)
    places = [0, 7, 15] if ticks else []  # label rows and columns 1, 8, 16
    ax.set_xticks(places, [str(t + 1) for t in places])
    ax.set_yticks(places, [str(t + 1) for t in places])
    ax.axhline(7.5, color=GREY, linewidth=0.8)  # the border between the halves
    ax.axvline(7.5, color=GREY, linewidth=0.8)
    ax.grid(False)  # no grid lines on top of the squares
    ax.set_title(title, fontsize=10)
    return image


fig, axes = plt.subplots(2, 4, figsize=(11.0, 6.0))
for a, ax in zip(range(8), axes.flat):
    kind = "space-like" if eta[a] == 1 else "time-like"
    image = draw_matrix(ax, gamma[a], f"$\\gamma^{{(x_{a})}}$ ({kind})")
fig.colorbar(image, ax=list(axes.flat), shrink=0.7, label="entry")
save_figure(fig, "eight_gammas",
            "The author's eight real 16 by 16 gamma matrices, one heat map each, in "
            "the order $x_0$ to $x_7$ of the author's coordinates (top row: "
            "$x_1, x_2, x_3, x_4$; bottom row: $x_5, x_6, x_7, x_0$). Horizontal "
            "axis: column number, vertical axis: row number; red is $+1$, blue is "
            "$-1$, light grey is 0; thin lines separate the components 1 to 8 "
            "from 9 to 16. Every row and every column holds exactly one red or "
            "blue square (signed permutation matrices), no entry is complex, and "
            "every matrix has its squares only in the two off-diagonal blocks, so "
            "each gamma exchanges the two halves of a 16-component field.")
```

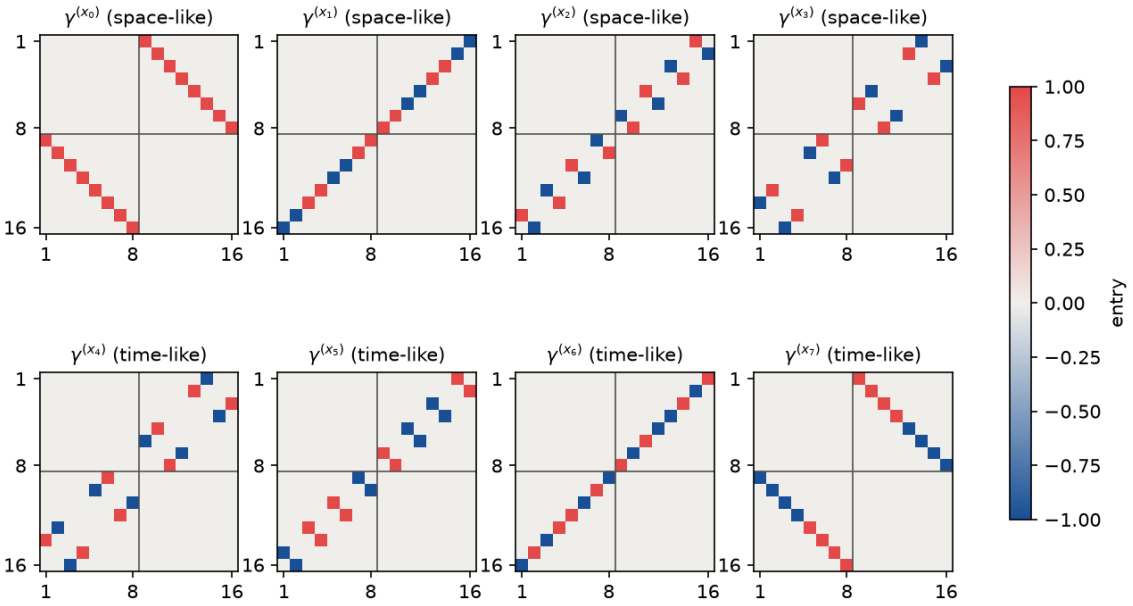

Figure 472: The author's eight real 16 by 16 gamma matrices, one heat map each, in the order x_0 to x_7 of the author's coordinates (top row: x_1, x_2, x_3, x_4; bottom row: x_5, x_6, x_7, x_0). Horizontal axis: column number, vertical axis: row number; red is $+1$, blue is -1, light grey is 0; thin lines separate the components 1 to 8 from 9 to 16. Every row and every column holds exactly one red or blue square (signed permutation matrices), no entry is complex, and every matrix has its squares only in the two off-diagonal blocks, so each gamma exchanges the two halves of a 16-component field. (Notebook 18a, figure 1.)

Out [4]:

```
Figure 18a.1 saved as Revision/textbook/figures/18a_1_eight_gammas.png
```

Text cell:

```
The next cell builds $C$, $\Gamma$ and $B$ from the gammas and compares them with
the matrices stored in the same record. $B = -iC\gamma^{(x_4)}$ is $i$ times a real
whole-number matrix; the cell keeps that real matrix as `B_over_i` (so that
$B = i\,\cdot$ `B_over_i`) and also forms the complex matrix `B`. It checks the
basic facts used by the pairing record: $C$ real symmetric with $C^2 = 1$, every
$C\gamma^{(a)}$ antisymmetric, $B$ Hermitian with $B^2 = 1$, and
$C\gamma^{(x_4)} = iB$.
```

In [5]:

```
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the four space-like gammas
Gamma = I16.copy()
for a in range(8):  # the order x0, ..., x7 of the factors of Gamma
    Gamma = Gamma @ gamma[a]
B_over_i = -(C @ gamma[4])  # B = -i C gamma^(x4) = i * (-C gamma^(x4))
B = 1j * B_over_i  # the complex matrix B (entries 0, +i, -i)
B_file = np.array(fixture["B"]["re"]) + 1j * np.array(fixture["B"]["im"])
stored = (same(C, np.array(fixture["C"])) and same(Gamma, np.array(fixture["Gamma"]))
          and np.array_equal(B, B_file))
reproduces(stored, "C, Gamma and B built here equal the matrices of the gammas record",
           (WL, ["fixture_definitions"]))
basic = (same(C.T, C) and same(C @ C, I16)
         and all(same((C @ gamma[a]).T, -(C @ gamma[a])) for a in range(8))
         and np.array_equal(B.conj().T, B) and np.array_equal(B @ B, np.eye(16))
         and np.array_equal(C @ gamma[4], 1j * B))
reproduces(basic, "C real symmetric, C^2 = 1, B Hermitian, B^2 = 1, C gamma^(x4) = i B",
           (WL, ["C_and_B_basic"]), (PY, ["gammas.C", "gammas.B"]))
```

Out [5]:

```
PASS C, Gamma and B built here equal the matrices of the gammas record
     reproduces Revision/pairing/reports/wolfram-pairing.json, check fixture_definitions
PASS C real symmetric, C^2 = 1, B Hermitian, B^2 = 1, C gamma^(x4) = i B
     reproduces Revision/pairing/reports/wolfram-pairing.json, check C_and_B_basic;
    Revision/pairing/reports/python-pairing.json, check gammas.C, gammas.B
```

Text cell:

```
## 6. Lemma 1: the chirality $\Gamma$

The next cell checks every part of Lemma 1 exactly: $\Gamma =
\mathrm{diag}(-I_8, I_8)$; $\Gamma$ is real and symmetric and $\Gamma^2 = 1$;
$\Gamma\gamma^{(a)} = -\gamma^{(a)}\Gamma$ for each of the eight gammas;
```

```
$\Gamma C = C\Gamma$; and $\Gamma S^{ab} = S^{ab}\Gamma$ for all 64 pairs
$(a, b)$. Since $S^{ab} = \frac14(\gamma^{(a)}\gamma^{(b)} -
\gamma^{(b)}\gamma^{(a)})$ has quarters and halves as entries, the cell uses
$4S^{ab}$, which has whole-number entries; $\Gamma$ commutes with $S^{ab}$ exactly
when it commutes with $4S^{ab}$.

Why these hold: $\Gamma$ is the product of all eight different gammas. Moving
$\gamma^{(b)}$ through it passes seven factors that anticommute with $\gamma^{(b)}$
and one ($\gamma^{(b)}$ itself) that commutes with it: the sign is $(-1)^7 = -1$.
$C$ (four gammas) and $S^{ab}$ (two gammas when $a \neq b$) contain an even number
of gammas, so $\Gamma$ passes them with the sign $(-1)^{\mathrm{even}} = +1$.
```

In [6]:

```
four_S = {(a, b): gamma[a] @ gamma[b] - gamma[b] @ gamma[a]  # 4 S^ab, whole numbers
          for a in range(8) for b in range(8)}
diagonal = same(Gamma, np.diag([-1] * 8 + [1] * 8))  # diag(-I8, I8)
real_symmetric_involution = same(Gamma.T, Gamma) and same(Gamma @ Gamma, I16)
anticommutes = all(same(Gamma @ gamma[a], -gamma[a] @ Gamma) for a in range(8))
commutes_C = same(Gamma @ C, C @ Gamma)
commutes_S = all(same(Gamma @ M, M @ Gamma) for M in four_S.values())  # 64 pairs
say(f"Gamma = diag(-I8, I8): {diagonal}; real symmetric with Gamma^2 = 1: "
    f"{real_symmetric_involution}; anticommutes with all 8 gammas: {anticommutes}; "
    f"commutes with C: {commutes_C}; commutes with all 64 S^ab: {commutes_S}")
reproduces(diagonal and real_symmetric_involution and anticommutes and commutes_C
           and commutes_S, "Lemma 1: the chirality Gamma",
           (WL, ["Gamma_properties"]),
           (PY, ["gammas.Gamma", "gammas.Gamma_anticommutes"]))
```

Out [6]:

```
Gamma = diag(-I8, I8): True; real symmetric with Gamma^2 = 1: True; anticommutes with all
    8 gammas: True; commutes with C: True; commutes with all 64 S^ab: True
PASS Lemma 1: the chirality Gamma
     reproduces Revision/pairing/reports/wolfram-pairing.json, check Gamma_properties;
    Revision/pairing/reports/python-pairing.json, check gammas.Gamma,
    gammas.Gamma_anticommutes
```

Text cell:

```
Lemma 1 says that $\Gamma$ commutes with EVERY spin connection
$\Omega_\mu = \frac12\sum_{a,b}\omega_{\mu ab}S^{ab}$, whatever the numbers
$\omega_{\mu ab}$ are, because it commutes with each $S^{ab}$. The next cell
illustrates this with numbers: it draws an arbitrary antisymmetric table
$\omega_{ab} = -\omega_{ba}$ of random numbers (with a fixed seed, so that every run
draws the same numbers), forms $\Omega = \frac12\sum_{a,b}\omega_{ab}S^{ab}$ and
measures the largest entry of $\Gamma\Omega - \Omega\Gamma$. This is an
illustration (floating-point numbers); the proof is the exact check above.
```

In [7]:

```
rng = np.random.default_rng(12345)  # a fixed seed: the same numbers every run
omega = rng.normal(size=(8, 8))  # rows and columns 0..7 stand for x0, ..., x7
omega = omega - omega.T  # antisymmetric: omega[a, b] = -omega[b, a]
Omega = sum(0.5 * omega[a, b] * four_S[(a, b)] / 4.0
            for a in range(8) for b in range(8))  # (1/2) omega_ab S^ab
commutator = np.abs(Gamma @ Omega - Omega @ Gamma).max()
check(commutator < 1e-12 and np.abs(Omega).max() > 0.1,
      "Gamma commutes with a random spin connection Omega (largest entry of the "
      "commutator below 1e-12)")
```

Out [7]:

```
PASS Gamma commutes with a random spin connection Omega (largest entry of the commutator
    below 1e-12)
```

Text cell:

```
The next cell draws three heat maps: $\Gamma$ (left), the gamma of the first
direction $\gamma^{(x_1)}$ (middle) and $\Gamma\gamma^{(x_1)}\Gamma$ (right). The
right picture is the middle one with every colour reversed: conjugating by
$\Gamma$ reverses the sign of a gamma. The cell first checks this for all eight
gammas.
```

In [8]:

```
flips = all(same(Gamma @ gamma[a] @ Gamma, -gamma[a]) for a in range(8))
check(flips, "Gamma gamma^(a) Gamma = -gamma^(a) for all eight directions")
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], Gamma, "$\\Gamma = \\mathrm{diag}(-I_8, I_8)$")
draw_matrix(axes[1], gamma[1], "$\\gamma^{(x_1)}$")
image = draw_matrix(axes[2], Gamma @ gamma[1] @ Gamma,
                    "$\\Gamma\\gamma^{(x_1)}\\Gamma = -\\gamma^{(x_1)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "chirality_flips_gammas",
            "Heat maps of three 16 by 16 matrices: the chirality $\\Gamma$ (left), "
            "the gamma matrix $\\gamma^{(x_1)}$ of the first direction of 3-space "
            "(middle) and $\\Gamma\\gamma^{(x_1)}\\Gamma$ (right). Horizontal axis: "
            "column number, vertical axis: row number; red is $+1$, blue is $-1$, "
            "light grey is 0; thin lines separate the chiral halves, components 1 "
            "to 8 and 9 to 16. $\\Gamma$ is $-1$ on the first half and $+1$ on the "
            "second. $\\gamma^{(x_1)}$ has its entries only in the two off-diagonal "
            "blocks: it exchanges the halves. The right picture is the middle one "
            "with every colour reversed, because $\\Gamma$ anticommutes with every "
            "gamma. This sign is the whole content of the pairing theorem T1.")
```

Out [8]:

```
PASS Gamma gamma^(a) Gamma = -gamma^(a) for all eight directions
```

Out [8]:

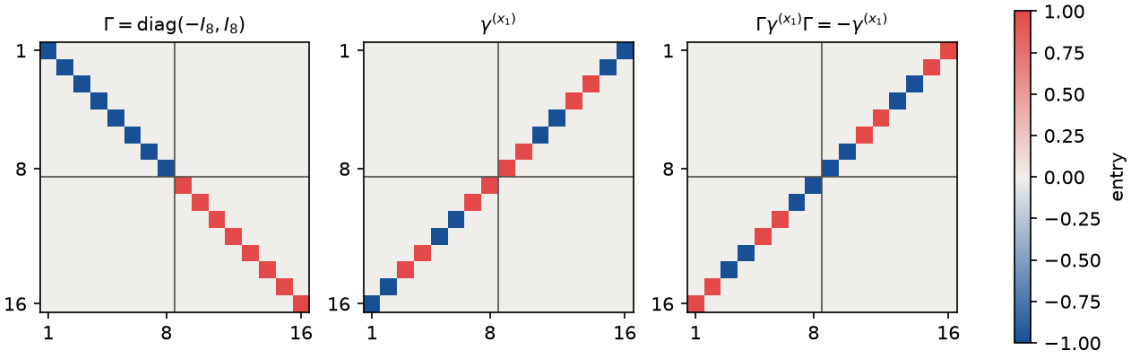

Figure 473: Heat maps of three 16 by 16 matrices: the chirality Γ (left), the gamma matrix $\gamma^{(x_1)}$ of the first direction of 3-space (middle) and $\Gamma\gamma^{(x_1)}\Gamma$ (right). Horizontal axis: column number, vertical axis: row number; red is $+1$, blue is -1, light grey is 0; thin lines separate the chiral halves, components 1 to 8 and 9 to 16. Γ is -1 on the first half and $+1$ on the second. $\gamma^{(x_1)}$ has its entries only in the two off-diagonal blocks: it exchanges the halves. The right picture is the middle one with every colour reversed, because Γ anticommutes with every gamma. This sign is the whole content of the pairing theorem T1. (Notebook 18a, figure 2.)

```
Figure 18a.2 saved as Revision/textbook/figures/18a_2_chirality_flips_gammas.png
```

Text cell:

```
## 7. Lemma 2: the sign of every bilinear under $\Psi \to \Gamma\Psi$

Section 4 showed that the map $\Psi \to \Gamma\Psi$ turns a bilinear
$\bar\Psi X\Psi$ into $\bar\Psi\,\Gamma X\Gamma\,\Psi$. The next cell forms all
$2^8 = 256$ products $X = \gamma^{(a_1)}\cdots\gamma^{(a_k)}$ of different gammas
($a_1 < \dots < a_k$, $k = 0, \dots, 8$; the empty product $k = 0$ is $I_{16}$),
determines for each the sign $s$ in $\Gamma X\Gamma = sX$, and checks that
$s = (-1)^k$ every time. It also counts the products of each length $k$; there are
$\binom{8}{k}$ of them (the number of ways to choose $k$ of 8 directions).
```

In [9]:

```
def sign_between(M, N):
    """+1 if M = N, -1 if M = -N, 0 otherwise (exact comparison)."""
    if same(M, N):
        return 1
    if same(M, -N):
        return -1
    return 0


parity_ok = True
counts = {k: {1: 0, -1: 0} for k in range(9)}  # k -> number of products per sign
for k in range(9):
    for subset in itertools.combinations(range(8), k):
```

```
        X = I16.copy()
        for a in subset:  # the product gamma^(a1) ... gamma^(ak)
            X = X @ gamma[a]
        s = sign_between(Gamma @ X @ Gamma, X)
        parity_ok = parity_ok and s == (-1) ** k
        counts[k][s] += 1
say("number of products of k different gammas, by the sign s: "
    + "; ".join(f"k={k}: {counts[k][1]} with s=+1, {counts[k][-1]} with s=-1"
                for k in range(9)))
check(parity_ok and all(counts[k][1] + counts[k][-1] == comb(8, k) for k in range(9)),
      "Gamma X Gamma = (-1)^k X for all 256 products X of k different gammas")
```

Out [9]:

```
number of products of k different gammas, by the sign s: k=0: 1 with s=+1, 0 with s=-1;
    k=1: 0 with s=+1, 8 with s=-1; k=2: 28 with s=+1, 0 with s=-1; k=3: 0 with s=+1, 56
    with s=-1; k=4: 70 with s=+1, 0 with s=-1; k=5: 0 with s=+1, 56 with s=-1; k=6: 28
    with s=+1, 0 with s=-1; k=7: 0 with s=+1, 8 with s=-1; k=8: 1 with s=+1, 0 with s=-1
PASS Gamma X Gamma = (-1)^k X for all 256 products X of k different gammas
```

Text cell:

```
The next cell checks the matrices that actually occur in the Lagrangian. Written
out with the vielbein $e^\mu{}_a$ and the connection $\omega_{\mu bc}$, the kinetic
term is

$$K = \frac12\sum_{\mu,a}e^\mu{}_a\big(\bar\Psi\gamma^{(a)}\partial_\mu\Psi -
\partial_\mu\bar\Psi\gamma^{(a)}\Psi\big) + \frac14\sum_{\mu,a,b,c}e^\mu{}_a
\omega_{\mu bc}\,\bar\Psi\{\gamma^{(a)}, S^{bc}\}\Psi ,$$

where $\{M, N\} = MN + NM$. As bilinears in $\Psi^\dagger$ and $\Psi$ their kernels
are $C\gamma^{(a)}$ (8 matrices) and $C\gamma^{(a)}S^{bc}$, $CS^{bc}\gamma^{(a)}$
with $b < c$ (224 each): 456 matrices in all, the list of the sympy verifier.
Since $\Gamma$ is real and symmetric, the kernel $X$ of $\Psi^\dagger X\Psi$
becomes $\Gamma^TX\Gamma = \Gamma X\Gamma$. The cell checks $\Gamma X\Gamma = -X$
for all 456 kernels and $\Gamma C\Gamma = +C$ for the kernel of $S$. It also checks
the form used by the Wolfram verifier, $\Gamma C\{\gamma^{(a)}, S^{bc}\}\Gamma =
-C\{\gamma^{(a)}, S^{bc}\}$ for all $8^3 = 512$ triples $(a, b, c)$, the
field-equation kernels $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}$ and
$\Gamma S^{bc}\Gamma = S^{bc}$, and $\Gamma B\Gamma = -B$ (the last part of
Lemma 2). Products with $S^{bc}$ are formed with $4S^{bc}$ (whole numbers).
```

In [10]:

```
kinetic = [C @ gamma[a] for a in range(8)]
connection = [C @ gamma[a] @ four_S[(b, c)] for a in range(8)
              for b in range(8) for c in range(b + 1, 8)]
connection += [C @ four_S[(b, c)] @ gamma[a] for a in range(8)
               for b in range(8) for c in range(b + 1, 8)]
kernels = kinetic + connection
all_flip = all(same(Gamma @ X @ Gamma, -X) for X in kernels)
```

```
scalar_kept = same(Gamma @ C @ Gamma, C)
triples = [C @ (gamma[a] @ four_S[(b, c)] + four_S[(b, c)] @ gamma[a])
           for a in range(8) for b in range(8) for c in range(8)]
triples_flip = all(same(Gamma @ X @ Gamma, -X) for X in triples)
field_kernels = (all(same(Gamma @ gamma[a] @ Gamma, -gamma[a]) for a in range(8))
                 and all(same(Gamma @ M @ Gamma, M) for M in four_S.values()))
krein_reversed = np.array_equal(Gamma @ B @ Gamma, -B)
report("number of kinetic and connection kernels", len(kernels))
report("number of triples (a, b, c)", len(triples))
reproduces(len(kernels) == 456 and all_flip and scalar_kept,
           "Lemma 2: all 456 kinetic and connection kernels flip, C is kept",
           (PY, ["T1.general_field.matrix_identities"]),
           (WL, ["T1_kernel_scalar", "T1_kernel_kinetic"]))
reproduces(len(triples) == 512 and triples_flip and field_kernels and krein_reversed,
           "Lemma 2: 512 connection triples flip; field-equation kernels; GBG = -B",
           (WL, ["T1_kernel_connection", "T1_kernel_field_equation"]),
           (PY, ["Q.image_krein_metric"]))
```

Out [10]:

```
RESULT number of kinetic and connection kernels = 456
RESULT number of triples (a, b, c) = 512
PASS Lemma 2: all 456 kinetic and connection kernels flip, C is kept
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.general_field.matrix_identities; Revision/pairing/reports/wolfram-pairing.json,
    check T1_kernel_scalar, T1_kernel_kinetic
PASS Lemma 2: 512 connection triples flip; field-equation kernels; GBG = -B
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    T1_kernel_connection, T1_kernel_field_equation; Revision/pairing/reports/python-
    pairing.json, check Q.image_krein_metric
```

Text cell:

```
The next cell draws the result of the 256 products as a bar chart: for each number
$k$ of gamma factors, a bar of height $\binom{8}{k}$ (the number of such products),
red when the bilinear keeps its sign under $\Psi \to \Gamma\Psi$ ($k$ even) and
blue when it changes sign ($k$ odd). Labels name the bilinears of the theory that
belong to some of the bars.
```

In [11]:

```
ks = np.arange(9)
heights = [comb(8, int(k)) for k in ks]
colours = ["#e34948" if k % 2 == 0 else "#184f95" for k in ks]
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.bar(ks, heights, color=colours, width=0.7)
ax.set_xticks(ks)
ax.set_xlabel("number $k$ of gamma factors in the matrix $X$ of $\\bar\\Psi X\\Psi$")
ax.set_ylabel("number of products $\\binom{8}{k}$")
ax.set_ylim(0, 85)
notes = {0: "$S = \\bar\\Psi\\Psi$", 1: "kinetic, $J^\\mu$, $\\Psi^\\dagger B\\Psi$",
```

```
         3: "connection", 8: "$\\bar\\Psi\\Gamma\\Psi$"}
for k, text in notes.items():
    ax.text(k, heights[k] + 3, text, ha="center", fontsize=8.5, rotation=90,
            va="bottom")
ax.bar([0], [0], color="#e34948", label="sign kept, $(-1)^k = +1$")
ax.bar([0], [0], color="#184f95", label="sign reversed, $(-1)^k = -1$")
ax.legend(loc="upper right")
save_figure(fig, "parity_of_products",
            "The 256 products $X$ of different gamma matrices, sorted by the number "
            "$k$ of factors (horizontal axis); the bar height is the number of "
            "such products, $\\binom{8}{k}$ (vertical axis). Red bars: the "
            "bilinear $\\bar\\Psi X\\Psi$ keeps its value under the chirality map "
            "$\\Psi \\to \\Gamma\\Psi$; blue bars: it changes sign. The sign is "
            "$(-1)^k$ for every one of the 256 products. The scalar $S$ sits in "
            "the red bar $k = 0$; the kinetic terms, the current $J^\\mu$ and the "
            "charge density $\\Psi^\\dagger B\\Psi$ in the blue bar $k = 1$; the "
            "spin-connection terms in the blue bar $k = 3$. This is why T1 "
            "reverses the kinetic term but not the mass term.")
```

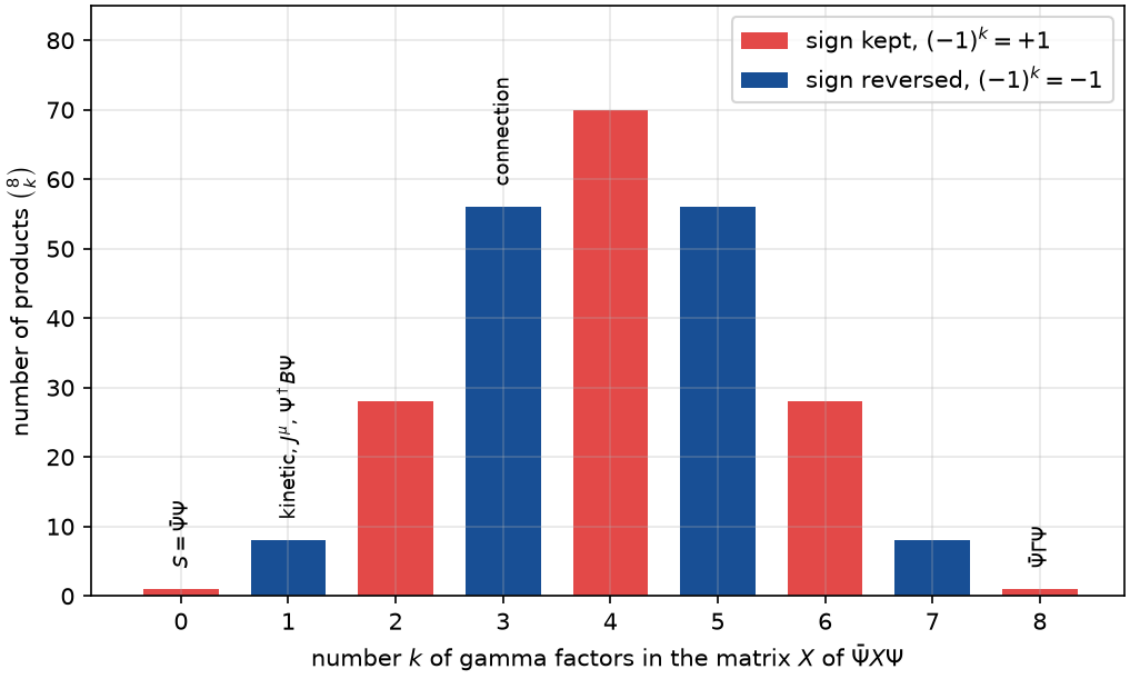

Figure 474: The 256 products X of different gamma matrices, sorted by the number k of factors (horizontal axis); the bar height is the number of such products, $\binom{8}{k}$ (vertical axis). Red bars: the bilinear $\bar\Psi X\Psi$ keeps its value under the chirality map $\Psi \to \Gamma\Psi$; blue bars: it changes sign. The sign is $(-1)^k$ for every one of the 256 products. The scalar S sits in the red bar $k = 0$; the kinetic terms, the current J^μ and the charge density $\Psi^\dagger B\Psi$ in the blue bar $k = 1$; the spin-connection terms in the blue bar $k = 3$. This is why T1 reverses the kinetic term but not the mass term. (Notebook 18a, figure 3.)

Out [11]:

```
Figure 18a.3 saved as Revision/textbook/figures/18a_3_parity_of_products.png
```

Text cell:

```
## 8. Lemma 3: the eight reflections and their Pin(4,4) lifts

For each direction $n = x_0, \dots, x_7$ the next cell forms $P_n = \Gamma
\gamma^{(n)}$ and checks, exactly:

1. $P_n$ equals $+$ or $-$ the product of the other seven gammas in the order of
   $\Gamma$ ($x_0, x_1, \dots, x_7$ without $n$), so it is a product of unit
   vectors and lies in Pin(4,4); the signs are recorded.
2. $P_n$ is a signed permutation matrix, so $P_n^{-1} = P_n^T$, and
   $P_n^{-1}\gamma^{(a)}P_n = (R_n)_{aa}\gamma^{(a)}$ for all $a$: $P_n$ acts on the
   gammas exactly as the reflection $R_n$ of the direction $n$.
3. The character: $P_n^\dagger CP_n = \chi_nC$ with $\chi_n = -\eta_{nn}$.
4. $\Gamma P_n = \gamma^{(n)}$ (because $\Gamma\Gamma = 1$): "$\Gamma$ combined with
   the reflection $P_n$" acts on the field as the single matrix $\gamma^{(n)}$.
5. For $M = \gamma^{(n)}$ together with the reflected frame: the scalar sign
   $M^TCM = -\eta_{nn}C$; the kinetic kernels $M^TC\gamma^{(a)}M =
   \eta_{nn}(R_n)_{aa}C\gamma^{(a)}$ (the factor $(R_n)_{aa}$ is absorbed by the
   reflected frame, so the kinetic term gets the sign $\eta_{nn}$); and the 512
   connection kernels $M^TC\{\gamma^{(a)}, S^{bc}\}M = \eta_{nn}(R_n)_{aa}
   (R_n)_{bb}(R_n)_{cc}C\{\gamma^{(a)}, S^{bc}\}$ (the connection of the reflected
   frame is $R_n\omega R_n$, which absorbs the other two factors).

From items 3 and 5: for a space-like $n$ the kinetic term is unchanged and $S$
changes sign, so $\mathcal{L}_{m,\lambda} \to \mathcal{L}_{-m,\lambda}$ (T2); for a
time-like $n$ the kinetic term changes sign and $S$ does not, so
$\mathcal{L}_{m,\lambda} \to -\mathcal{L}_{-m,-\lambda}$ (a map of the T1 type).
```

In [12]:

```
order = list(range(8))  # the order x0, ..., x7 of the factors of Gamma
names = {a: f"x{a}" for a in range(8)}
rows = []  # one row of the reflection table per direction n
lemma3_ok = True
for n in range(8):
    P = Gamma @ gamma[n]
    others = I16.copy()
    for a in order:
        if a != n:
            others = others @ gamma[a]  # the product of the other seven gammas
    R = {a: (-1 if a == n else 1) for a in range(8)}  # the diagonal of R_n
    orthogonal = same(P.T @ P, I16)  # so P^-1 = P^T
    covers = all(same(P.T @ gamma[a] @ P, R[a] * gamma[a]) for a in range(8))
    M = gamma[n]  # Gamma P_n = gamma^(n)
    kinetic_sign = eta[n]
    kinetic_ok = all(same(M.T @ C @ gamma[a] @ M, kinetic_sign * R[a] * C @ gamma[a])
                     for a in range(8))
    connection_ok = all(
        same(M.T @ C @ (gamma[a] @ four_S[(b, c)] + four_S[(b, c)] @ gamma[a]) @ M,
```

```
            kinetic_sign * R[a] * R[b] * R[c]
            * C @ (gamma[a] @ four_S[(b, c)] + four_S[(b, c)] @ gamma[a]))
        for a in range(8) for b in range(8) for c in range(8))
    row = {"direction": names[n], "eta_nn": eta[n],
           "product_sign": sign_between(P, others),
           "character": sign_between(P.T @ C @ P, C),
           "S_sign": sign_between(M.T @ C @ M, C),
           "kinetic_sign": kinetic_sign if kinetic_ok else 0}
    row["map"] = ("(m, lambda) -> (-m, lambda), L -> +L (T2)"
                  if (row["S_sign"], row["kinetic_sign"]) == (-1, 1)
                  else "(m, lambda) -> (-m, -lambda), L -> -L (T1-type)")
    rows.append(row)
    lemma3_ok = (lemma3_ok and orthogonal and covers and connection_ok and kinetic_ok
                 and same(Gamma @ P, gamma[n]) and row["character"] == -eta[n]
                 and row["product_sign"] in (1, -1))
for row in rows:
    say(f"{row['direction']}: eta {row['eta_nn']:+d}, P_n = {row['product_sign']:+d} "
        f"x (other seven), character {row['character']:+d}, S -> "
        f"{row['S_sign']:+d} S, kinetic {row['kinetic_sign']:+d}: {row['map']}")
product_signs = [r["product_sign"] for r in rows]
reproduces(lemma3_ok and product_signs == [-1, 1, -1, 1, 1, -1, 1, -1],
           "Lemma 3: P_n in Pin(4,4), covers R_n, character -eta_nn, Gamma P_n = g^n",
           (WL, ["T2_Pn_in_Pin44", "T2_Pn_covers_the_reflection", "T2_character_of_Pn",
                 "T2_Gamma_times_Pn_is_gamma_n",
                 "T2_kernels_gamma_n_with_frame_reflection"]),
           (PY, ["T2.general_field.matrix_identities",
                 "T2.general_field.reflection_table"]))
```

Out [12]:

```
x0: eta +1, P_n = -1 x (other seven), character -1, S -> -1 S, kinetic +1: (m, lambda) ->
    (-m, lambda), L -> +L (T2)
x1: eta +1, P_n = +1 x (other seven), character -1, S -> -1 S, kinetic +1: (m, lambda) ->
    (-m, lambda), L -> +L (T2)
x2: eta +1, P_n = -1 x (other seven), character -1, S -> -1 S, kinetic +1: (m, lambda) ->
    (-m, lambda), L -> +L (T2)
x3: eta +1, P_n = +1 x (other seven), character -1, S -> -1 S, kinetic +1: (m, lambda) ->
    (-m, lambda), L -> +L (T2)
x4: eta -1, P_n = +1 x (other seven), character +1, S -> +1 S, kinetic -1: (m, lambda) ->
    (-m, -lambda), L -> -L (T1-type)
x5: eta -1, P_n = -1 x (other seven), character +1, S -> +1 S, kinetic -1: (m, lambda) ->
    (-m, -lambda), L -> -L (T1-type)
x6: eta -1, P_n = +1 x (other seven), character +1, S -> +1 S, kinetic -1: (m, lambda) ->
    (-m, -lambda), L -> -L (T1-type)
x7: eta -1, P_n = -1 x (other seven), character +1, S -> +1 S, kinetic -1: (m, lambda) ->
    (-m, -lambda), L -> -L (T1-type)
PASS Lemma 3: P_n in Pin(4,4), covers R_n, character -eta_nn, Gamma P_n = g^n
     reproduces Revision/pairing/reports/wolfram-pairing.json, check T2_Pn_in_Pin44,
    T2_Pn_covers_the_reflection, T2_character_of_Pn, T2_Gamma_times_Pn_is_gamma_n,
    T2_kernels_gamma_n_with_frame_reflection; Revision/pairing/reports/python-
    pairing.json, check T2.general_field.matrix_identities,
    T2.general_field.reflection_table
```

Text cell:

```
The next cell compares the table just computed with the data table ‘reflections‘
of the Revision record ‘Revision/pairing/pairing-theory.json‘, field by field
(the sign of $\eta_{nn}$, the sign of $P_n$ against the product of the other seven,
the character, the sign of $S$, the kinetic sign and the resulting map of the mass
and the coupling). It also reads the status line of the record, which must say
that all checks of its report passed.
```

In [13]:

```
theory = read_json(THEORY)
recorded_rows = theory["data"]["reflections"]
keys = [("eta_nn", "eta_nn"),
        ("product_sign", "P_n_equals_sign_times_product_of_other_seven"),
        ("character", "character_of_P_n"), ("S_sign", "S_sign_under_gamma_n"),
        ("kinetic_sign", "kinetic_sign_under_gamma_n_with_frame_reflection"),
        ("map", "mass_coupling_map")]
table_equal = len(recorded_rows) == 8 and all(
    recorded["direction"] == mine["direction"]
    and all(mine[k_mine] == recorded[k_rec] for k_mine, k_rec in keys)
    for mine, recorded in zip(rows, recorded_rows))
say("status of the theory record: " + theory["status"])
reproduces(table_equal and theory["status"] == "all checks of the report passed",
           "the reflection table equals the record’s, row by row",
           (PY, ["compare.theory.reflection_table"]), table="reflections")
```

Out [13]:

```
status of the theory record: all checks of the report passed
PASS the reflection table equals the record’s, row by row
     reproduces Revision/pairing/pairing-theory.json, data table reflections;
    Revision/pairing/reports/python-pairing.json, check compare.theory.reflection_table
```

Text cell:

```
The next cell draws the reflection table: eight rows (the directions), five
columns of signs (red $+1$, blue $-1$) and, at the right, the map that the row
produces.
```

In [14]:

```
columns = ["$\\eta_{nn}$", "sign of $P_n$", "character $\\chi_n$",
           "$S \\to \\pm S$", "kinetic $\\pm$"]
grid = np.array([[r["eta_nn"], r["product_sign"], r["character"], r["S_sign"],
                  r["kinetic_sign"]] for r in rows], dtype=float)
fig, ax = plt.subplots(figsize=(8.6, 5.6))
ax.imshow(grid, cmap=DIVERGING, vmin=-1.0, vmax=1.0, aspect="auto")
for i in range(8):
```

```
    for j in range(5):
        ax.text(j, i, f"{int(grid[i, j]):+d}", ha="center", va="center",
                color="white", fontsize=10)
    label = "T2: $(-m, \\lambda)$, $+\\mathcal{L}$" if rows[i]["S_sign"] == -1 \
        else "T1-type: $(-m, -\\lambda)$, $-\\mathcal{L}$"
    ax.text(5.0, i, label, ha="left", va="center", fontsize=9)
ax.set_xticks(range(5), columns, fontsize=9)
ax.set_yticks(range(8), [f"$n = x_{a}$" for a in range(8)])
ax.set_xlim(-0.5, 7.4)
ax.grid(False)
ax.set_title("reflection of the direction $n$ with $\\Psi \\to \\gamma^{(n)}\\Psi$")
save_figure(fig, "reflection_table",
            "The reflection table of Lemma 3, one row for each direction $n$ (the "
            "space-like $x_0, x_1, x_2, x_3$ and the time-like $x_4$ to $x_7$). "
            "Columns: the sign $\\eta_{nn}$ of the direction; the sign of "
            "$P_n = \\Gamma\\gamma^{(n)}$ relative to the product of the other "
            "seven gammas; the character $\\chi_n$ in $P_n^\\dagger CP_n = "
            "\\chi_nC$; the sign of $S$ and of the kinetic term under "
            "$\\Psi \\to \\gamma^{(n)}\\Psi$ with the reflected frame. Red is "
            "$+1$, blue is $-1$. The character is always $-\\eta_{nn}$. A "
            "space-like reflection keeps the kinetic term and reverses $S$: the "
            "mass changes sign and the coupling does not (theorem T2). A "
            "time-like reflection does the opposite, which gives a map of the T1 "
            "type. The table equals the Revision record row by row.")
```

Out [14]:

```
Figure 18a.4 saved as Revision/textbook/figures/18a_4_reflection_table.png
```

Text cell:

```
## 9. The chiral projectors and the two halves

The author's notebook also defines the two *chiral projectors*
$P_L = \frac12(1 - \Gamma)$ and $P_R = \frac12(1 + \Gamma)$; we write them
$P_- = P_L$ and $P_+ = P_R$. Since $\Gamma = \mathrm{diag}(-I_8, I_8)$,
$P_- = \mathrm{diag}(I_8, 0)$ keeps the components 1 to 8 ($\psi_-$) and $P_+ =
\mathrm{diag}(0, I_8)$ keeps 9 to 16 ($\psi_+$). The next cell checks, exactly:

- $P_\pm^2 = P_\pm$ (projectors), $P_- + P_+ = 1$, $P_-P_+ = 0$, each of rank 8;
- $\Gamma = P_+ - P_-$: the T1 image is $\Gamma\Psi = P_+\Psi - P_-\Psi$, the field
  with the relative sign of its two halves reversed;
- $\gamma^{(a)}P_- = P_+\gamma^{(a)}$ for all eight gammas: a gamma takes the
  $\psi_-$ half into the $\psi_+$ half (and back), because it anticommutes with
  $\Gamma$;
- $CP_\pm = P_\pm C$ and $S^{ab}P_\pm = P_\pm S^{ab}$: $C$ and the generators of
  Spin(4,4) keep each half, which is why the two halves are separate
  representations of Spin(4,4).
```

In [15]:

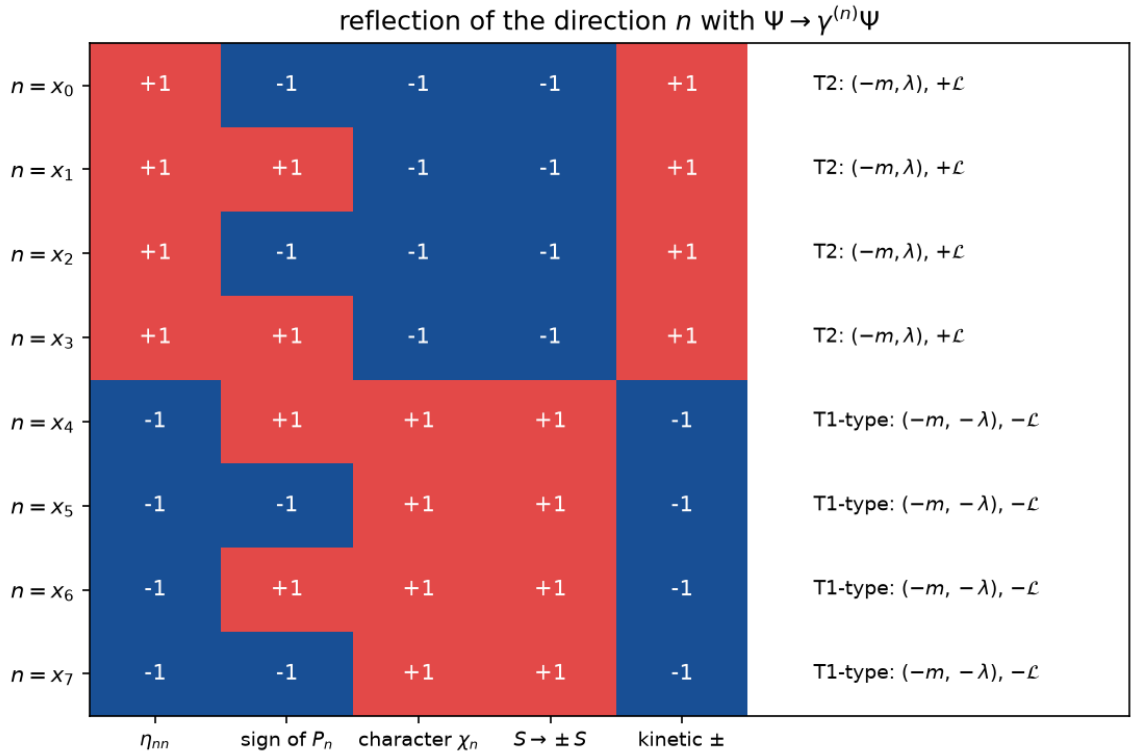

Figure 475: The reflection table of Lemma 3, one row for each direction n (the space-like x_0, x_1, x_2, x_3 and the time-like x_4 to x_7). Columns: the sign η_{nn} of the direction; the sign of $P_n = \Gamma\gamma^{(n)}$ relative to the product of the other seven gammas; the character χ_n in $P_n^\dagger C P_n = \chi_n C$; the sign of S and of the kinetic term under $\Psi \to \gamma^{(n)}\Psi$ with the reflected frame. Red is $+1$, blue is -1. The character is always $-\eta_{nn}$. A space-like reflection keeps the kinetic term and reverses S: the mass changes sign and the coupling does not (theorem T2). A time-like reflection does the opposite, which gives a map of the T1 type. The table equals the Revision record row by row. (Notebook 18a, figure 4.)

```
P_minus = (I16 - Gamma) // 2  # whole numbers 0 and 1: the author's P_L
P_plus = (I16 + Gamma) // 2  # the author's P_R
projectors = (same(P_minus @ P_minus, P_minus) and same(P_plus @ P_plus, P_plus)
              and same(P_minus + P_plus, I16) and not (P_minus @ P_plus).any()
              and int(np.trace(P_minus)) == 8 and int(np.trace(P_plus)) == 8)
gamma_is_difference = same(Gamma, P_plus - P_minus)
gammas_swap = all(same(gamma[a] @ P_minus, P_plus @ gamma[a]) for a in range(8))
halves_kept = (same(C @ P_minus, P_minus @ C)
               and all(same(M @ P_minus, P_minus @ M) for M in four_S.values()))
say(f"projectors of rank 8: {projectors}; Gamma = P+ - P-: {gamma_is_difference}; "
    f"every gamma maps the minus half to the plus half: {gammas_swap}; C and all "
    f"S^ab keep each half: {halves_kept}")
check(projectors and gamma_is_difference and gammas_swap and halves_kept,
      "the chiral projectors: P-^2 = P-, P+^2 = P+, Gamma = P+ - P-, gammas swap")
fig, axes = plt.subplots(1, 4, figsize=(12.0, 3.6))
draw_matrix(axes[0], P_minus, "$P_- = \\frac{1}{2}(1 - \\Gamma)$")
draw_matrix(axes[1], P_plus, "$P_+ = \\frac{1}{2}(1 + \\Gamma)$")
draw_matrix(axes[2], gamma[1] @ P_minus, "$\\gamma^{(x_1)}P_-$")
image = draw_matrix(axes[3], P_plus @ gamma[1], "$P_+\\gamma^{(x_1)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "chiral_projectors",
            "Heat maps of the two chiral projectors and of one gamma between them; "
            "columns horizontal, rows vertical, red $+1$, blue $-1$, grey 0, thin "
            "lines between the halves. $P_-$ (first picture, the author's P_L) is "
            "the identity on the components 1 to 8 and zero elsewhere; $P_+$ "
            "(second, the author's P_R) is the identity on 9 to 16. Their "
            "difference is $\\Gamma$. The third and fourth pictures are equal: "
            "$\\gamma^{(x_1)}P_- = P_+\\gamma^{(x_1)}$, so the gamma takes a field "
            "that lives in the first half into the second half. The T1 image "
            "$\\Gamma\\Psi = P_+\\Psi - P_-\\Psi$ reverses the relative sign of "
            "the two halves.")
```

Out [15]:

```
projectors of rank 8: True; Gamma = P+ - P-: True; every gamma maps the minus half to the
    plus half: True; C and all S^ab keep each half: True
PASS the chiral projectors: P-^2 = P-, P+^2 = P+, Gamma = P+ - P-, gammas swap
```

Out [15]:

```
Figure 18a.5 saved as Revision/textbook/figures/18a_5_chiral_projectors.png
```

Text cell:

```
The next cell checks the block form behind the theorems. Split
$\Psi = (\psi_-, \psi_+)$ into its first and second eight components. Then

- $\Gamma(\psi_-, \psi_+) = (-\psi_-, \psi_+)$: the T1 image reverses the
  relative sign of the two halves;
- every gamma has nonzero entries only in the two off-diagonal $8 \times 8$ blocks
  (it exchanges the halves), and $\gamma^{(x_0)}$ has the identity $I_8$ in both
```

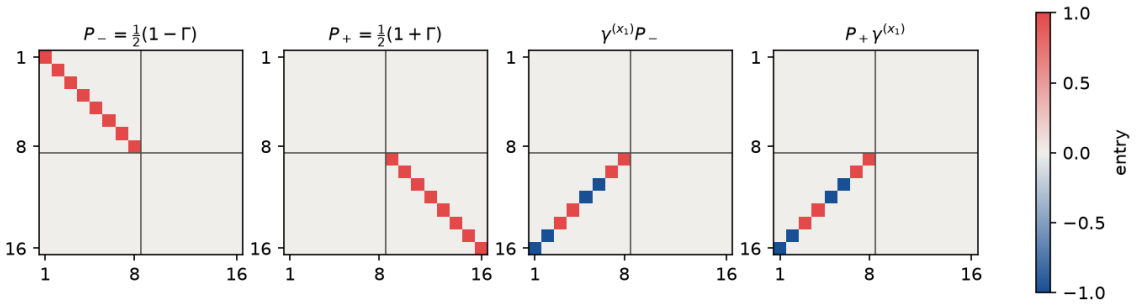

Figure 476: Heat maps of the two chiral projectors and of one gamma between them; columns horizontal, rows vertical, red +1, blue −1, grey 0, thin lines between the halves. P_- (first picture, the author's P_L) is the identity on the components 1 to 8 and zero elsewhere; P_+ (second, the author's P_R) is the identity on 9 to 16. Their difference is Γ. The third and fourth pictures are equal: $\gamma^{(x_1)} P_- = P_+ \gamma^{(x_1)}$, so the gamma takes a field that lives in the first half into the second half. The T1 image $\Gamma\Psi = P_+\Psi - P_-\Psi$ reverses the relative sign of the two halves. (Notebook 18a, figure 5.)

```
  of them, so $\gamma^{(x_0)}(\psi_-, \psi_+) = (\psi_+, \psi_-)$: the T2 image
  exchanges the two halves;
- $C$ (a product of four gammas, an even number of exchanges) is block diagonal,
  $C = \mathrm{diag}(-\sigma, \sigma)$ with an $8 \times 8$ matrix $\sigma$, so
  $S = -\psi_-^\dagger\sigma\psi_- + \psi_+^\dagger\sigma\psi_+$. Exchanging the
  halves therefore reverses $S$, and reversing one half keeps $S$.
```

In [16]:

```
def blocks(M):
    """The four 8 x 8 blocks of a 16 x 16 matrix: upper left, upper right, lower
    left, lower right."""
    return M[:8, :8], M[:8, 8:], M[8:, :8], M[8:, 8:]


off_diagonal = all(not blocks(gamma[a])[0].any() and not blocks(gamma[a])[3].any()
                   for a in range(8))  # .any() is False when all entries are 0
g0_swaps = (same(blocks(gamma[0])[1], I8) and same(blocks(gamma[0])[2], I8))
C_ul, C_ur, C_ll, C_lr = blocks(C)
C_block_diagonal = not C_ur.any() and not C_ll.any() and same(C_ul, -C_lr)
P0 = Gamma @ gamma[0]
say(f"every gamma off-diagonal: {off_diagonal}; gamma^(x0) = [[0, I8], [I8, 0]]: "
    f"{g0_swaps}; C = diag(-sigma, sigma): {C_block_diagonal}")
check(off_diagonal and g0_swaps and C_block_diagonal,
      "block form: gammas exchange the halves, gamma^(x0) swaps them, C is diagonal")
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], C, "$C = \\mathrm{diag}(-\\sigma, \\sigma)$")
draw_matrix(axes[1], gamma[0], "$\\gamma^{(x_0)}$: exchanges the halves")
image = draw_matrix(axes[2], P0, "$P_0 = \\Gamma\\gamma^{(x_0)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "chiral_blocks",
            "Heat maps of $C$ (left), $\\gamma^{(x_0)}$ (middle) and the reflection "
```

```
"$P_0 = \\Gamma\\gamma^{(x_0)}$ of the hidden direction (right); axes "
"as in the first figure (columns horizontal, rows vertical, red $+1$, "
"blue $-1$, grey 0), thin lines between the chiral halves. $C$ lives "
"in the two diagonal blocks, with opposite signs: "
"$S = -\\psi_-^\\dagger\\sigma\\psi_- + \\psi_+^\\dagger\\sigma\\psi_+$. "
"$\\gamma^{(x_0)}$ is the identity in both off-diagonal blocks: it "
"exchanges $\\psi_-$ and $\\psi_+$, which reverses $S$ (the T2 image). "
"$P_0$ differs from $\\gamma^{(x_0)}$ by the sign of one block, the "
"work of $\\Gamma$.")
```

Out [16]:

```
every gamma off-diagonal: True; gamma^(x0) = [[0, I8], [I8, 0]]: True; C = diag(-sigma,
    sigma): True
PASS block form: gammas exchange the halves, gamma^(x0) swaps them, C is diagonal
```

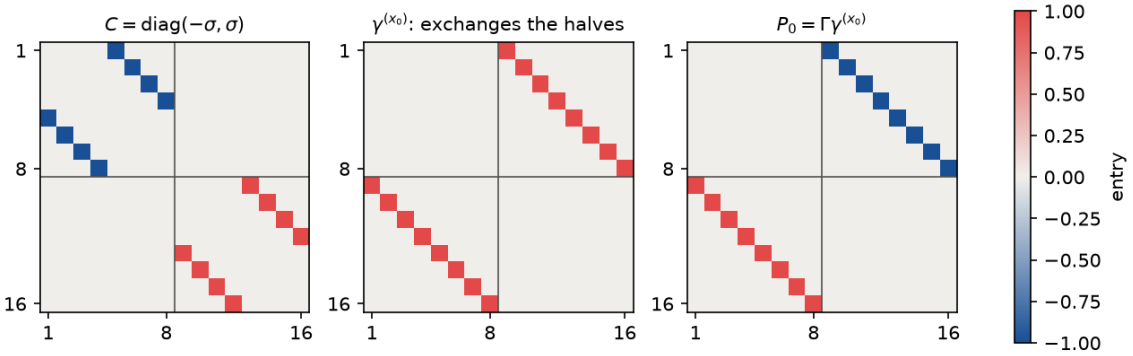

Figure 477: Heat maps of C (left), $\gamma^{(x_0)}$ (middle) and the reflection $P_0 = \Gamma\gamma^{(x_0)}$ of the hidden direction (right); axes as in the first figure (columns horizontal, rows vertical, red $+1$, blue -1, grey 0), thin lines between the chiral halves. C lives in the two diagonal blocks, with opposite signs: $S = -\psi_-^\dagger\sigma\psi_- + \psi_+^\dagger\sigma\psi_+$. $\gamma^{(x_0)}$ is the identity in both off-diagonal blocks: it exchanges ψ_- and ψ_+, which reverses S (the T2 image). P_0 differs from $\gamma^{(x_0)}$ by the sign of one block, the work of Γ. (Notebook 18a, figure 6.)

Out [16]:

```
Figure 18a.6 saved as Revision/textbook/figures/18a_6_chiral_blocks.png
```

Text cell:

```
## 10. Lemma 4: the Krein signs of the 17 maps

For the quantised field dirac16complex the canonical anticommutator is
$\{\Psi_A, \Psi_B^\dagger\} = B_{AB}\,\delta/\sqrt{|g|}$. For the image field
$M\Psi$ it becomes $(MBM^\dagger)_{AB}\,\delta/\sqrt{|g|}$. The next cell computes
the sign $\sigma$ in $MBM^\dagger = \sigma B$ for the 17 maps $M = \Gamma$,
$M = \gamma^{(n)}$ and $M = P_n$, and compares the list with the data table
‘Krein_signs_M_B_Mdagger‘ of the pairing record. It uses $B = i\,$‘B_over_i‘:
$MBM^\dagger = i\,M\,$‘B_over_i‘$\,M^T$ for a real $M$, so whole numbers suffice.
```

In [17]:

```
maps = [("Gamma", Gamma)]
maps += [(f"gamma^x{n}", gamma[n]) for n in range(8)]
maps += [(f"P_x{n}", Gamma @ gamma[n]) for n in range(8)]
krein = {name: sign_between(M @ B_over_i @ M.T, B_over_i) for name, M in maps}
say("M B M^dagger = sigma B: " + ", ".join(f"{name} {s:+d}"
                                           for name, s in krein.items()))
recorded_krein = {row["map"]: row["sign"]
                  for row in theory["data"]["Krein_signs_M_B_Mdagger"]}
reproduces(krein == recorded_krein and krein["Gamma"] == -1 and krein["gamma^x0"] == 1,
           "Lemma 4: the 17 Krein signs equal the record's; Gamma -1, gamma^(x0) +1",
           (WL, ["Q_Krein_metric_of_images"]),
           (PY, ["compare.theory.krein_signs", "Q.T2_image_keeps_B"]),
           table="Krein_signs_M_B_Mdagger")
```

Out [17]:

```
M B M^dagger = sigma B: Gamma -1, gamma^x0 +1, gamma^x1 +1, gamma^x2 +1, gamma^x3 +1,
    gamma^x4 +1, gamma^x5 -1, gamma^x6 -1, gamma^x7 -1, P_x0 -1, P_x1 -1, P_x2 -1, P_x3
    -1, P_x4 -1, P_x5 +1, P_x6 +1, P_x7 +1
PASS Lemma 4: the 17 Krein signs equal the record's; Gamma -1, gamma^(x0) +1
     reproduces Revision/pairing/pairing-theory.json, data table Krein_signs_M_B_Mdagger;
    Revision/pairing/reports/wolfram-pairing.json, check Q_Krein_metric_of_images;
    Revision/pairing/reports/python-pairing.json, check compare.theory.krein_signs,
    Q.T2_image_keeps_B
```

Text cell:

```
The next cell draws the imaginary parts of $B$, of $\Gamma B\Gamma^\dagger$ and of
$\gamma^{(x_0)}B\gamma^{(x_0)\dagger}$ (all three are purely imaginary). The image
under $\Gamma$ (the T1 map) carries $-B$; the image under $\gamma^{(x_0)}$ (the T2
map) carries $+B$.
```

In [18]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], B_over_i, "imaginary part of $B$")
draw_matrix(axes[1], Gamma @ B_over_i @ Gamma.T,
            "imaginary part of $\\Gamma B\\Gamma^\\dagger = -B$")
image = draw_matrix(axes[2], gamma[0] @ B_over_i @ gamma[0].T,
                    "imaginary part of $\\gamma^{(x_0)}B\\gamma^{(x_0)\\dagger} = B$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry divided by $i$")
save_figure(fig, "krein_signs",
            "Heat maps of the imaginary parts of three 16 by 16 matrices (their "
            "real parts are zero): the Krein matrix $B = -iC\\gamma^{(x_4)}$ (left), "
            "$\\Gamma B\\Gamma^\\dagger$ (middle) and "
            "$\\gamma^{(x_0)}B\\gamma^{(x_0)\\dagger}$ (right); columns horizontal, "
            "rows vertical, red $+1$, blue $-1$, grey 0. The middle picture is the "
            "left one with all colours reversed: the chirality image "
```

```
"$\\Gamma\\Psi$ of a quantised field has the anticommutator $-B$. The "
"right picture equals the left one: the mirror image "
"$\\gamma^{(x_0)}\\Psi$ of theorem T2 keeps $+B$, so it is an ordinary "
"copy of the theory with equal energies.")
```

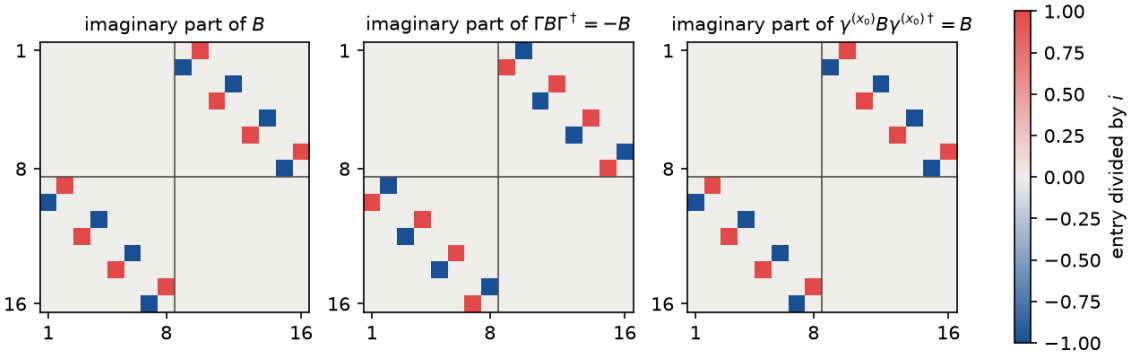

Figure 478: Heat maps of the imaginary parts of three 16 by 16 matrices (their real parts are zero): the Krein matrix $B = -iC\gamma^{(x_4)}$ (left), $\Gamma B \Gamma^\dagger$ (middle) and $\gamma^{(x_0)} B \gamma^{(x_0)\dagger}$ (right); columns horizontal, rows vertical, red $+1$, blue -1, grey 0. The middle picture is the left one with all colours reversed: the chirality image $\Gamma\Psi$ of a quantised field has the anticommutator $-B$. The right picture equals the left one: the mirror image $\gamma^{(x_0)}\Psi$ of theorem T2 keeps $+B$, so it is an ordinary copy of the theory with equal energies. (Notebook 18a, figure 7.)

Out [18]:

```
Figure 18a.7 saved as Revision/textbook/figures/18a_7_krein_signs.png
```

Text cell:

```
## 11. The last check

The last cell checks that the seven figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [19]:

```
figure_names = ["eight_gammas", "chirality_flips_gammas", "parity_of_products",
                "reflection_table", "chiral_projectors", "chiral_blocks",
                "krein_signs"]
paths = [output_file(f"{FIGURE_FOLDER}/18a_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [19]:

```
PASS every figure file of this notebook exists
ALL 16 CHECKS PASSED (notebook 18a)
```

Text cell:

```
## 12. What this notebook showed

- PROVED (exact whole-number comparison): the eight matrices used by the Revision
  record, and by every notebook of this chapter, are the author's own: they are
  real $16 \times 16$ signed permutation matrices with entries $-1, 0, +1$, they
  obey the 64 Clifford relations of signature (4,4), and they equal the matrices
  T16A[0..7] rebuilt here from the author's formulas, in the author's coordinates
  $x_0, \dots, x_7$.
- PROVED (exact whole-number identities of the author's gammas, the same as the
  Revision pairing record's): Lemma 1, $\Gamma = \mathrm{diag}(-I_8, I_8)$
  anticommutes with every gamma and commutes with $C$, with every $S^{ab}$ and
  therefore with every spin connection.
- PROVED: Lemma 2, under $\Psi \to \Gamma\Psi$ a bilinear with $k$ gamma factors
  gets the sign $(-1)^k$ (all 256 products); all 456 kinetic and connection
  kernels of the Lagrangian change sign while the scalar $S$ is unchanged; and
  $\Gamma B\Gamma = -B$. Together with the linearity of the Lagrangian in the
  coefficients of the gravitational field this is the proof of theorem T1 in every
  gravitational field, for commuting and for Grassmann components.
- PROVED: Lemma 3, the reflection table: $P_n = \Gamma\gamma^{(n)}$ lies in
  Pin(4,4), acts on the gammas as the reflection $R_n$, has the character
  $-\eta_{nn}$; a space-like reflection maps $(m, \lambda)$ to $(-m, \lambda)$ with
  $\mathcal{L} \to +\mathcal{L}$ (theorem T2), a time-like one gives a map of the
  T1 type. The table equals the record's row by row.
- PROVED: the chiral projectors $P_\mp = \frac12(1 \mp \Gamma)$ split the 16
  components into two halves of 8; every gamma exchanges the halves, $C$ and the
  $S^{ab}$ keep them.
- PROVED: Lemma 4, the Krein signs of the 17 maps, equal to the record's: the T1
  image carries $-B$, the T2 image $+B$.
- NOT shown here: anything about a particular solution, the author's metric or
  the creation of universes; the matrix lemmas are the algebraic input of the
  theorems, which map solutions to solutions and say nothing about how any
  universe comes into being.
```

18.9 Line-by-line walk-through of Notebook 18a

The notebook has 19 code cells, In [1] to In [19]. This section explains every line of every one of them. The words of the notebook's section 3 are defined there, and in this chapter as follows: matrix and transpose in the Notation paragraph of Section 18.1 (matrices themselves in Chapter 1); signed permutation matrix, projector, character and Krein sign in Section 18.2; the characters and the Krein signs of the maps again in Lemmas 3 and 4 of Section 18.5.

In [1], the set-up cell. Its first part repeats the complete run instructions of Section 18.7 as **comment lines**: every line that starts with `#` is a comment, which Python skips; they are there so that the notebook file carries its own instructions. The code starts below the line THE SET-UP between two lines of `=` signs. This code is the same in every notebook of the book except for the line that names the notebook; it is explained here once, and the walk-throughs of Notebooks 18b

and 18c refer back to this explanation. The texts in triple quotes below a `def` line are **docstrings**: they describe the function and do nothing when the code runs.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json` reads and writes JSON files (text files that hold names, lists and numbers), `os` reads environment variables, `textwrap` breaks long texts into lines, and `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system. matplotlib is the plotting package; its drawing functions are loaded under the short name `plt`. `Image` and `display` come from IPython, the part of Jupyter that runs Python code; together they show a saved picture below a cell.

```
NOTEBOOK_ID = "18a"  # this notebook: chapter 18, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"18a"`; the figure files and the last printed line use it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. `here.parents` lists its parent folder, the parent of that, and so on; `[here, *here.parents]` is the list that starts with `here` and continues with all of them. The `for` loop takes the folders one after the other; `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If there is none, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

`REPO` is the repository folder found by the function; it is never printed, because it differs from computer to computer while the printed output of a notebook must not. `OUTPUT_ROOT` is the folder below which files are written. `os.environ` holds the **environment variables** (named texts that a program receives from the computer); `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and the repository folder otherwise. When you run the notebook the variable is not set; the book's checking tool sets it to a scratch folder, so that a check never changes the repository.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

`repository_file("Revision/...")` returns the full path of a repository file, for reading a Revision record. `output_file("Revision/...")` returns the full path at which to write a file; before that, `path.parent.mkdir(parents=True, exist_ok=True)` creates its folder (and every missing folder above it) and does nothing if the folder exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a page of the book: `textwrap.fill` breaks the text at blanks, and every line after the first starts with four blanks. `str(text)` first turns any value into text.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings of matplotlib, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets the size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid behind the curves. The braces make a **dictionary**, a collection of pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: each name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/18a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary `{}` and a line end into the captions file, which `save_figure` then fills; `encoding="utf-8"` fixes how letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` is called once for every figure. `setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far; so the figures are numbered 1, 2, 3, and a cell run twice keeps its number. The file name joins the notebook id, the number and the name, for example `18a_1_eight_gammas.png`. `fig.savefig` writes the picture as a PNG file with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory (otherwise Jupyter would draw it a second time). The caption is stored, and the whole dictionary of captions is written to the captions file (`json.dumps` turns it into JSON text with sorted keys). `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
```

```
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list**, an ordered collection written with square brackets. `check` is the function behind every check of the book. `condition` is either `True` or `False`. If it is false, `raise AssertionError(...)` stops the notebook with an error that names the check (Python's own statement `assert` is not used here, because Python started with the option `-O` skips it). If it is true, the name is appended to `PASSED` and the line PASS name is printed; when the optional third argument `record` names a Revision record, a second line says which record and which check the result reproduces.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT, followed by the unit when one is given (`a if condition else b` is `a` when the condition holds and `b` otherwise). `all_checks_passed` prints the last line of the notebook with the number of checks that passed (`len` is the length of a list). The last statement prints the single output line of In [1].

In [2], the record helpers, the gammas and the Clifford relations.

```
import itertools  # all subsets of a list (for the 256 products of gammas)
from math import comb  # binomial coefficients: comb(8, k) = number of k-subsets

import numpy as np  # arrays and matrices

GAMMAS = "Revision/algebra/gammas.json"
THEORY = "Revision/pairing/pairing-theory.json"
PY = "Revision/pairing/reports/python-pairing.json"  # the sympy verifier's report
WL = "Revision/pairing/reports/wolfram-pairing.json"  # the Wolfram verifier's report
```

`itertools` provides `combinations`, used in In [9] to list all subsets of the eight directions; `comb(8, k)` is the binomial coefficient $\binom{8}{k}$, the number of ways to choose k things out of 8. numpy, the

package for arrays and matrices, is loaded under the short name `np`. The four constants are the repository paths of the four Revision records the notebook reads: the gammas, the pairing theory record and the two reports of the pairing verifiers.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def verdict(report_file, name):
    """The recorded verdict (PASS or FAIL) of the check name of a report."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"].upper()
    return "MISSING"
```

`read_json` reads a record file as text and turns it into Python objects (`json.loads`). `verdict` reads a report and looks through its list `"checks"`; each entry is a dictionary with the keys `name`, `verdict` and `detail`. It returns the verdict of the check with the given name in capital letters (`.upper()`; the two pairing reports write PASS, but some other Revision reports write pass in small letters, and the comparison should not depend on that), or `"MISSING"` if the report has no such check.

```
def reproduces(condition, name, *sources, table=None):
    """check(condition, name), which also requires every named check of every
    source (report_file, [check names]) to have the recorded verdict PASS; table
    names a data table of the theory record that the result equals."""
    recorded = all(verdict(report_file, n) == "PASS"
                   for report_file, names in sources for n in names)
    parts = [f"{report_file}, check {', '.join(names)}"
             for report_file, names in sources]
    if table is not None:
        parts.insert(0, f"{THEORY}, data table {table}")
    check(condition and recorded, name, record="; ".join(parts))
```

`reproduces` is the check used whenever a result repeats a Revision record. `*sources` collects any number of pairs (report file, list of check names). `recorded` is true only when every named check of every named report has the verdict PASS: `all(...)` is true when every item of the following **generator expression** (a `for` loop written inside parentheses) is true. `parts` builds the text that names the reports and checks (`", ".join(names)` joins the names with commas); if a data table of the theory record is named, its text is put first. Finally `check` is called: the check passes only when the notebook's own result (`condition`) holds AND the record says PASS.

```
fixture = read_json(GAMMAS)
gamma = {a: np.array(fixture["gamma"][a], dtype=np.int64) for a in range(8)}
eta = {a: int(fixture["eta"][a]) for a in range(8)}  # +1 or -1
I16 = np.eye(16, dtype=np.int64)


def same(M, N):
    """True when the two matrices are equal entry by entry (exact for integers)."""
    return np.array_equal(M, N)
```

`fixture` is the dictionary read from `gammas.json`. Its entry `"gamma"` is a list of eight matrices, each a list of 16 rows of 16 whole numbers; the **dictionary comprehension** `{a: ... for a in range(8)}` stores the matrix of the direction x_a under the key `a`, as a numpy array of 64-bit whole numbers (`dtype=np.int64`). With whole numbers every product below is exact: nothing is rounded. `range(8)` runs through 0, 1, ..., 7 (the end is excluded), and `fixture["gamma"][a]` takes the entry number `a`: the record lists the matrices in the order $x_0, \ldots, x_7$, and Python counts list places from 0. `eta` holds η_{aa}, and `I16` is the identity matrix (`np.eye`). `same(M, N)` is true when two matrices are equal entry by entry; for whole numbers that comparison is exact.

```
say("eta = " + str([eta[a] for a in range(8)]) + " for x0, ..., x7")
reproduces(all(same(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                    2 * (eta[a] if a == b else 0) * I16)
                for a in range(8) for b in range(8)),
           "the 64 Clifford relations hold exactly",
           (WL, ["fixture_Clifford_relation"]), (PY, ["gammas.clifford"]))
```

The first line prints the list of the eight values η_{aa}: `[1, 1, 1, 1, -1, -1, -1, -1]`, the signature (4,4) in the order $x_0, \ldots, x_7$. The second statement checks the Clifford relation for all $8 \times 8 = 64$ pairs (a, b): `@` is the matrix product, so `gamma[a] @ gamma[b] + gamma[b] @ gamma[a]` is $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}$, compared with $2\eta_{aa} I_{16}$ when $a = b$ and with the zero matrix otherwise. It names the Wolfram check `fixture_Clifford_relation` and the sympy check `gammas.clifford`. Out [2] shows the PASS line and the record line.

In [3], are these the author's matrices? The markdown cell before it lists the author's five construction steps; the code follows them one by one.

```
def perm_sign(seq):
    """The permutation sign of a list of numbers: +1 (even number of exchanges
    sorts it), -1 (odd number), 0 (two entries are equal)."""
    if len(set(seq)) != len(seq):
        return 0
    inversions = sum(1 for i in range(len(seq)) for j in range(i + 1, len(seq))
                     if seq[i] > seq[j])  # pairs in the wrong order
    return -1 if inversions % 2 else 1


def kd(p, q):
    """The Kronecker delta: 1 if p = q, else 0."""
    return 1 if p == q else 0
```

`perm_sign` computes the **permutation sign** of a list of numbers. `set(seq)` keeps each different number once; if it is shorter than the list, two entries are equal and the sign is 0. Otherwise the code counts the **inversions**, the pairs of places $i < j$ whose numbers stand in the wrong order (`seq[i] > seq[j]`); sorting the list needs an even number of exchanges exactly when this count is even, so the sign is $+1$ for an even and -1 for an odd count (`inversions % 2` is the remainder after division by 2). `kd(p, q)` is the Kronecker delta.

```
I4, Z4 = np.eye(4, dtype=np.int64), np.zeros((4, 4), dtype=np.int64)
I8, Z8 = np.eye(8, dtype=np.int64), np.zeros((8, 8), dtype=np.int64)
```

```
s4, t4 = {}, {}
for h in (1, 2, 3):  # step 1: the self-dual and anti-self-dual 4 x 4 blocks
    Qa = np.array([[perm_sign([h, p, q, 4]) for q in range(1, 5)]
                   for p in range(1, 5)])
    Qb = np.array([[kd(p, 4) * kd(q, h) - kd(p, h) * kd(q, 4) for q in range(1, 5)]
                   for p in range(1, 5)])
    s4[h], t4[h] = Qa - Qb, Qa + Qb
```

`np.eye(4)` and `np.zeros((4, 4))` are the 4×4 identity and zero matrices; the line with `I8, Z8` makes the 8×8 ones. Step 1: for $h = 1, 2, 3$, `Qa` is the 4×4 table of the permutation signs of the arrangements $(h, p, q, 4)$ for $p, q = 1, \ldots, 4$ (rows p, columns q), and `Qb` the table $\delta_{p4}\delta_{qh} - \delta_{ph}\delta_{q4}$; the author's blocks are $s_h = Q_a - Q_b$ and $t_h = Q_a + Q_b$.

```
tau = {0: I8}  # step 2: the eight tau matrices
for h in (1, 2, 3):
    tau[h] = np.block([[Z4, s4[h]], [s4[h], Z4]])
    tau[7 - h] = np.block([[Z4, t4[h]], [-t4[h], Z4]])
tau[7] = tau[1] @ tau[2] @ tau[3] @ tau[4] @ tau[5] @ tau[6]
sigma4 = np.block([[Z4, I4], [I4, Z4]])  # step 3: tau-bar
taubar = {A: sigma4 @ tau[A].T @ sigma4 for A in range(8)}
T16A = {A: np.block([[Z8, taubar[A]], [tau[A], Z8]]) for A in range(8)}  # step 4
frame_index = {a: a for a in range(8)}  # step 5: x_a -> A = a
rebuilt_equal = all(same(T16A[frame_index[a]], gamma[a]) for a in range(8))
```

Step 2: `tau[0]` is I_8; for $h = 1, 2, 3$, `np.block` assembles an 8×8 matrix from four 4×4 blocks: τ_h has s_h in both off-diagonal blocks, τ_{7-h} has t_h above and $-t_h$ below; τ_7 is the product $\tau_1\tau_2\cdots\tau_6$. Step 3: $\bar{\tau}_A = \sigma\tau_A^T\sigma$ with σ the 8×8 matrix with I_4 in both off-diagonal blocks (`.T` is the transpose). Step 4: the 16×16 matrix T16A[A] has $\bar{\tau}_A$ in the upper-right and τ_A in the lower-left block. Step 5: `frame_index` maps the author's coordinate x_a to their frame number, which is the same number a: the author's notebook counts its frame in the order $x_0, \ldots, x_7$, with 0 the hidden direction. `rebuilt_equal` is true when every rebuilt matrix equals the record's matrix of the same direction.

```
raw = fixture["gamma"]  # the numbers exactly as stored in the record
eight_real = (len(raw) == 8 and all(len(M) == 16 and all(len(row) == 16 for row in M)
                                    for M in raw)
              and all(type(x) is int and x in (-1, 0, 1)
                      for M in raw for row in M for x in row))
signed_perm = all((np.abs(gamma[a]).sum(axis=0) == 1).all()
                  and (np.abs(gamma[a]).sum(axis=1) == 1).all() for a in range(8))
transpose_rule = all(same(gamma[a].T, eta[a] * gamma[a]) for a in range(8))
say(f"8 matrices of 16 x 16 real entries -1, 0, +1: {eight_real}; signed "
    f"permutation matrices: {signed_perm}; transpose = eta_aa times the matrix: "
    f"{transpose_rule}; equal to the author's T16A rebuilt from the formulas: "
    f"{rebuilt_equal}")
reproduces(eight_real and signed_perm and transpose_rule and rebuilt_equal,
           "the record holds the author's eight real 16 x 16 gamma matrices",
           (PY, ["gammas.equal_wolfram_fixture"]))
```

`raw` is the list of numbers exactly as stored in the record (before numpy touched them). `eight_real` checks that there are 8 matrices, each with 16 rows of 16 entries, and that every entry is a

Python whole number (`type(x) is int`, so no complex number and no fraction) equal to -1, 0 or $+1$. `signed_perm` checks the defining property of a signed permutation matrix: the absolute values `np.abs` sum to 1 in every column (`axis=0`) and in every row (`axis=1`). `transpose_rule` checks $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$: the space-like gammas are symmetric and the time-like ones antisymmetric. The cell prints the four results and checks them together against `gammas.equal_wolfram_fixture`, the sympy check that compared its own rebuilt gammas with the record. Out [3] prints True four times and the PASS line: **the record holds the author's eight real** 16×16 **matrices**, and every computation of this chapter uses them.

In [4], the first figure: the eight gammas.

```
from matplotlib.colors import LinearSegmentedColormap  # colour scales

BLUE, ORANGE, GREEN, GREY = "#2a78d6", "#eb6834", "#1baf7a", "#52514e"
DIVERGING = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#f0efec", "#e34948"])
```

`LinearSegmentedColormap.from_list` makes a **colour scale** that runs from dark blue (for -1) through light grey (0) to red ($+1$). The four colour constants are a blue, an orange, a green and a grey that colour-blind readers can tell apart; they are written as hexadecimal colour codes.

```
def draw_matrix(ax, matrix, title, ticks=True):
    """Heat map of a real 16 x 16 matrix with entries between -1 and +1."""
    image = ax.imshow(matrix, cmap=DIVERGING, vmin=-1.0, vmax=1.0)
    places = [0, 7, 15] if ticks else []  # label rows and columns 1, 8, 16
    ax.set_xticks(places, [str(t + 1) for t in places])
    ax.set_yticks(places, [str(t + 1) for t in places])
    ax.axhline(7.5, color=GREY, linewidth=0.8)  # the border between the halves
    ax.axvline(7.5, color=GREY, linewidth=0.8)
    ax.grid(False)  # no grid lines on top of the squares
    ax.set_title(title, fontsize=10)
    return image
```

`draw_matrix` draws one matrix as a **heat map**, a grid of coloured squares, one per entry. `ax` is one panel of a figure. `imshow` draws the squares with the colour scale fixed between -1 and $+1$ (`vmin`, `vmax`), so that equal numbers always get equal colours. The tick marks label the rows and columns 1, 8 and 16 (Python counts from 0, so the places are 0, 7 and 15 and the labels add 1). `axhline(7.5)` and `axvline(7.5)` draw thin lines between the rows 8 and 9 and the columns 8 and 9, the border between the two chiral halves. `ax.grid(False)` removes the grid, and the title is written above the panel. The function returns the drawn image, which the colour bar needs.

```
fig, axes = plt.subplots(2, 4, figsize=(11.0, 6.0))
for a, ax in zip(range(8), axes.flat):
    kind = "space-like" if eta[a] == 1 else "time-like"
    image = draw_matrix(ax, gamma[a], f"$\\gamma^{{(x_{a})}}$ ({kind})")
fig.colorbar(image, ax=list(axes.flat), shrink=0.7, label="entry")
save_figure(fig, "eight_gammas",
            "The author's eight real 16 by 16 gamma matrices, one heat map each, in "
            "the order $x_0$ to $x_7$ of the author's coordinates (top row: "
```

```
            "$x_1, x_2, x_3, x_4$; bottom row: $x_5, x_6, x_7, x_0$). Horizontal "
            "axis: column number, vertical axis: row number; red is $+1$, blue is "
            "$-1$, light grey is 0; thin lines separate the components 1 to 8 "
            "from 9 to 16. Every row and every column holds exactly one red or "
            "blue square (signed permutation matrices), no entry is complex, and "
            "every matrix has its squares only in the two off-diagonal blocks, so "
            "each gamma exchanges the two halves of a 16-component field.")
```

`plt.subplots(2, 4)` makes a figure with two rows of four panels, 11 by 6 inches. `zip` pairs the directions $x_0, \ldots, x_7$ (`range(8)`) with the panels (`axes.flat` lists the panels row by row). Each panel is titled with $\gamma^{(x_a)}$ and the word space-like or time-like; in the f-string, double braces `{{` and `}}` print single braces, and `\\` prints one backslash, so the title is LaTeX code. `fig.colorbar` adds the colour key. `save_figure` saves figure 18a.1 with its caption. **What you see in Figure 18a.1:** in every panel each row and each column has exactly one coloured square (signed permutation matrices), the squares lie only in the upper-right and lower-left 8×8 blocks (each gamma exchanges the two halves), and there are only the colours of $+1$ and -1: no entry is complex.

In [5], C, Γ and B.

```
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # the four space-like gammas
Gamma = I16.copy()
for a in range(8):  # the order x0, ..., x7 of the factors of Gamma
    Gamma = Gamma @ gamma[a]
B_over_i = -(C @ gamma[4])  # B = -i C gamma^(x4) = i * (-C gamma^(x4))
B = 1j * B_over_i  # the complex matrix B (entries 0, +i, -i)
B_file = np.array(fixture["B"]["re"]) + 1j * np.array(fixture["B"]["im"])
stored = (same(C, np.array(fixture["C"])) and same(Gamma, np.array(fixture["Gamma"]))
          and np.array_equal(B, B_file))
reproduces(stored, "C, Gamma and B built here equal the matrices of the gammas record",
           (WL, ["fixture_definitions"]))
```

C is the product of the four space-like gammas in the author's order x_0, x_1, x_2, x_3. Γ starts as a copy of the identity (`.copy()`, so that `I16` itself is not changed) and is multiplied on the right by the eight gammas in the order $x_0, x_1, \ldots, x_7$. Because $B = -iC\gamma^{(x_4)}$ is i times the real whole-number matrix $-C\gamma^{(x_4)}$, the cell keeps that real matrix as `B_over_i` and forms the complex matrix `B` with `1j`, Python's name for i. The record stores B as a real part `"re"` and an imaginary part `"im"`, and `B_file` rebuilds it. `stored` compares the three matrices built here with the record's; the check names the Wolfram check `fixture_definitions`.

```
basic = (same(C.T, C) and same(C @ C, I16)
         and all(same((C @ gamma[a]).T, -(C @ gamma[a])) for a in range(8))
         and np.array_equal(B.conj().T, B) and np.array_equal(B @ B, np.eye(16))
         and np.array_equal(C @ gamma[4], 1j * B))
reproduces(basic, "C real symmetric, C^2 = 1, B Hermitian, B^2 = 1, C gamma^(x4) = i B",
           (WL, ["C_and_B_basic"]), (PY, ["gammas.C", "gammas.B"]))
```

`basic` checks five facts: $C^T = C$, $CC = 1$, every $C\gamma^{(a)}$ antisymmetric, $B^\dagger = B$ (`.conj().T` is the conjugate transpose) with $BB = 1$ (`np.eye(16)` is the identity), and $C\gamma^{(x_4)} = iB$. The last one is how B enters the time-derivative term of the Lagrangian (Section 18.23). Out [5] shows two PASS lines.

In [6], Lemma 1.

```
four_S = {(a, b): gamma[a] @ gamma[b] - gamma[b] @ gamma[a]  # 4 S^ab, whole numbers
          for a in range(8) for b in range(8)}
diagonal = same(Gamma, np.diag([-1] * 8 + [1] * 8))  # diag(-I8, I8)
real_symmetric_involution = same(Gamma.T, Gamma) and same(Gamma @ Gamma, I16)
anticommutes = all(same(Gamma @ gamma[a], -gamma[a] @ Gamma) for a in range(8))
commutes_C = same(Gamma @ C, C @ Gamma)
commutes_S = all(same(Gamma @ M, M @ Gamma) for M in four_S.values())  # 64 pairs
say(f"Gamma = diag(-I8, I8): {diagonal}; real symmetric with Gamma^2 = 1: "
    f"{real_symmetric_involution}; anticommutes with all 8 gammas: {anticommutes}; "
    f"commutes with C: {commutes_C}; commutes with all 64 S^ab: {commutes_S}")
reproduces(diagonal and real_symmetric_involution and anticommutes and commutes_C
           and commutes_S, "Lemma 1: the chirality Gamma",
           (WL, ["Gamma_properties"]),
           (PY, ["gammas.Gamma", "gammas.Gamma_anticommutes"]))
```

`four_S` holds $4S^{ab} = \gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)}$ for all 64 pairs; with the factor 4 the entries are whole numbers, and a matrix commutes with S^{ab} exactly when it commutes with $4S^{ab}$. `np.diag([-1] * 8 + [1] * 8)` is the diagonal matrix with eight entries -1 and eight $+1$ (`[-1] * 8` repeats the list eight times, `+` joins lists). The five results are: $\Gamma = \mathrm{diag}(-I_8, I_8)$; $\Gamma^T = \Gamma$ and $\Gamma\Gamma = 1$; $\Gamma\gamma^{(a)} = -\gamma^{(a)}\Gamma$ for all eight a; $\Gamma C = C\Gamma$; $\Gamma S^{ab} = S^{ab}\Gamma$ for all 64 pairs. Out [6] prints True five times and the PASS line for Lemma 1, with the records `Gamma_properties`, `gammas.Gamma` and `gammas.Gamma_anticommutes`.

In [7], Γ commutes with a random connection.

```
rng = np.random.default_rng(12345)  # a fixed seed: the same numbers every run
omega = rng.normal(size=(8, 8))  # rows and columns 0..7 stand for x0, ..., x7
omega = omega - omega.T  # antisymmetric: omega[a, b] = -omega[b, a]
Omega = sum(0.5 * omega[a, b] * four_S[(a, b)] / 4.0
            for a in range(8) for b in range(8))  # (1/2) omega_ab S^ab
commutator = np.abs(Gamma @ Omega - Omega @ Gamma).max()
check(commutator < 1e-12 and np.abs(Omega).max() > 0.1,
      "Gamma commutes with a random spin connection Omega (largest entry of the "
      "commutator below 1e-12)")
```

`np.random.default_rng(12345)` is a random-number generator with a fixed **seed**, so every run draws the same numbers. `rng.normal(size=(8, 8))` draws an array of 8×8 numbers from the normal (bell-curve) distribution; row and column a stand for x_a. `omega - omega.T` makes it antisymmetric, $\omega_{ab} = -\omega_{ba}$, as a connection must be. `Omega` is $\frac{1}{2}\sum_{a,b}\omega_{ab}S^{ab}$, with S^{ab} written as `four_S / 4.0`; `sum(...)` adds the 64 matrices. `commutator` is the largest absolute entry of $\Gamma\Omega - \Omega\Gamma$. The check requires it below 10^{-12} and requires Ω itself not to be tiny (largest entry above 0.1), so that the test is not empty. This is an illustration with floating-point numbers; the proof is the exact check of In [6].

In [8], the second figure: Γ reverses every gamma.

```
flips = all(same(Gamma @ gamma[a] @ Gamma, -gamma[a]) for a in range(8))
check(flips, "Gamma gamma^(a) Gamma = -gamma^(a) for all eight directions")
```

`flips` checks $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}$ for all eight directions, the identity behind T1.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], Gamma, "$\\Gamma = \\mathrm{diag}(-I_8, I_8)$")
draw_matrix(axes[1], gamma[1], "$\\gamma^{(x_1)}$")
image = draw_matrix(axes[2], Gamma @ gamma[1] @ Gamma,
                    "$\\Gamma\\gamma^{(x_1)}\\Gamma = -\\gamma^{(x_1)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "chirality_flips_gammas",
            "Heat maps of three 16 by 16 matrices: the chirality $\\Gamma$ (left), "
            "the gamma matrix $\\gamma^{(x_1)}$ of the first direction of 3-space "
            "(middle) and $\\Gamma\\gamma^{(x_1)}\\Gamma$ (right). Horizontal axis: "
            "column number, vertical axis: row number; red is $+1$, blue is $-1$, "
            "light grey is 0; thin lines separate the chiral halves, components 1 "
            "to 8 and 9 to 16. $\\Gamma$ is $-1$ on the first half and $+1$ on the "
            "second. $\\gamma^{(x_1)}$ has its entries only in the two off-diagonal "
            "blocks: it exchanges the halves. The right picture is the middle one "
            "with every colour reversed, because $\\Gamma$ anticommutes with every "
            "gamma. This sign is the whole content of the pairing theorem T1.")
```

A figure with three panels: Γ, $\gamma^{(x_1)}$ and $\Gamma\gamma^{(x_1)}\Gamma$, drawn with `draw_matrix`, a colour key, and `save_figure`. **What you see in Figure 18a.2:** Γ is blue on the first eight diagonal places and red on the last eight; $\gamma^{(x_1)}$ has its squares only in the off-diagonal blocks; the right panel is the middle one with every colour reversed. Why: Γ multiplies the rows 1 to 8 by -1 from the left and the columns 1 to 8 by -1 from the right, and every square of an off-diagonal block lies in exactly one of these rows or columns.

In [9], Lemma 2 for all 256 products.

```
def sign_between(M, N):
    """+1 if M = N, -1 if M = -N, 0 otherwise (exact comparison)."""
    if same(M, N):
        return 1
    if same(M, -N):
        return -1
    return 0
```

`sign_between(M, N)` returns $+1$ if $M = N$, -1 if $M = -N$ and 0 otherwise, with exact comparisons.

```
parity_ok = True
counts = {k: {1: 0, -1: 0} for k in range(9)}  # k -> number of products per sign
for k in range(9):
    for subset in itertools.combinations(range(8), k):
        X = I16.copy()
        for a in subset:  # the product gamma^(a1) ... gamma^(ak)
            X = X @ gamma[a]
        s = sign_between(Gamma @ X @ Gamma, X)
        parity_ok = parity_ok and s == (-1) ** k
        counts[k][s] += 1
```

`counts[k]` counts, for each length $k = 0, \ldots, 8$, the products with the sign $+1$ and with the sign -1. `itertools.combinations(range(8), k)` lists every set of k different directions in increasing

order. For each, `X` is the product of their gammas (the empty product, $k = 0$, is the identity). `s` is the sign in $\Gamma X \Gamma = sX$; `parity_ok` stays true only if `s` equals $(-1)^k$ every time (`**` is the power).

```
say("number of products of k different gammas, by the sign s: "
    + "; ".join(f"k={k}: {counts[k][1]} with s=+1, {counts[k][-1]} with s=-1"
                for k in range(9)))
check(parity_ok and all(counts[k][1] + counts[k][-1] == comb(8, k) for k in range(9)),
      "Gamma X Gamma = (-1)^k X for all 256 products X of k different gammas")
```

The cell prints the counts and checks that $s = (-1)^k$ always and that the number of products of length k is $\binom{8}{k}$. Out [9] reads: $k = 0$: 1 product with $s = +1$; $k = 1$: 8 with $s = -1$; $k = 2$: 28 with $+1$; $k = 3$: 56 with -1; $k = 4$: 70 with $+1$; $k = 5$: 56 with -1; $k = 6$: 28 with $+1$; $k = 7$: 8 with -1; $k = 8$: 1 with $+1$. The total is $2^8 = 256$.

In [10], the kernels of the Lagrangian.

```
kinetic = [C @ gamma[a] for a in range(8)]
connection = [C @ gamma[a] @ four_S[(b, c)] for a in range(8)
              for b in range(8) for c in range(b + 1, 8)]
connection += [C @ four_S[(b, c)] @ gamma[a] for a in range(8)
               for b in range(8) for c in range(b + 1, 8)]
kernels = kinetic + connection
all_flip = all(same(Gamma @ X @ Gamma, -X) for X in kernels)
scalar_kept = same(Gamma @ C @ Gamma, C)
```

`kinetic` holds the 8 kernels $C\gamma^{(a)}$ of the kinetic bilinears. `connection` holds $C\gamma^{(a)}(4S^{bc})$ and $C(4S^{bc})\gamma^{(a)}$ for all a and all $b < c$: $8 \times 28 = 224$ of each kind (`+=` appends the second list to the first). With the 8 kinetic kernels these are $8 + 224 + 224 = 456$ matrices, the list of the sympy verifier. `all_flip` checks $\Gamma X \Gamma = -X$ for all of them; since Γ is real and symmetric, this is the change of the kernel of $\Psi^\dagger X \Psi$ under $\Psi \to \Gamma\Psi$. `scalar_kept` checks $\Gamma C \Gamma = C$, the kernel of S.

```
triples = [C @ (gamma[a] @ four_S[(b, c)] + four_S[(b, c)] @ gamma[a])
           for a in range(8) for b in range(8) for c in range(8)]
triples_flip = all(same(Gamma @ X @ Gamma, -X) for X in triples)
field_kernels = (all(same(Gamma @ gamma[a] @ Gamma, -gamma[a]) for a in range(8))
                 and all(same(Gamma @ M @ Gamma, M) for M in four_S.values()))
krein_reversed = np.array_equal(Gamma @ B @ Gamma, -B)
```

`triples` holds the form of the Wolfram verifier, $C\{\gamma^{(a)}, 4S^{bc}\}$ for all $8^3 = 512$ triples (a, b, c), and `triples_flip` checks that each changes sign. `field_kernels` checks the two kernels of the field equation: $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}$ and $\Gamma S^{bc}\Gamma = S^{bc}$. `krein_reversed` checks $\Gamma B \Gamma = -B$ (the last part of Lemma 2).

```
report("number of kinetic and connection kernels", len(kernels))
report("number of triples (a, b, c)", len(triples))
reproduces(len(kernels) == 456 and all_flip and scalar_kept,
           "Lemma 2: all 456 kinetic and connection kernels flip, C is kept",
           (PY, ["T1.general_field.matrix_identities"]),
           (WL, ["T1_kernel_scalar", "T1_kernel_kinetic"]))
```

```
reproduces(len(triples) == 512 and triples_flip and field_kernels and krein_reversed,
           "Lemma 2: 512 connection triples flip; field-equation kernels; GBG = -B",
           (WL, ["T1_kernel_connection", "T1_kernel_field_equation"]),
           (PY, ["Q.image_krein_metric"]))
```

The cell prints the two counts, 456 and 512, as RESULT lines and makes two checks with their records. Out [10] shows both PASS lines.

In [11], the third figure: the parity of the 256 products.

```
ks = np.arange(9)
heights = [comb(8, int(k)) for k in ks]
colours = ["#e34948" if k % 2 == 0 else "#184f95" for k in ks]
fig, ax = plt.subplots(figsize=(8.0, 4.6))
ax.bar(ks, heights, color=colours, width=0.7)
ax.set_xticks(ks)
ax.set_xlabel("number $k$ of gamma factors in the matrix $X$ of $\\bar\\Psi X\\Psi$")
ax.set_ylabel("number of products $\\binom{8}{k}$")
ax.set_ylim(0, 85)
notes = {0: "$S = \\bar\\Psi\\Psi$", 1: "kinetic, $J^\\mu$, $\\Psi^\\dagger B\\Psi$",
         3: "connection", 8: "$\\bar\\Psi\\Gamma\\Psi$"}
for k, text in notes.items():
    ax.text(k, heights[k] + 3, text, ha="center", fontsize=8.5, rotation=90,
            va="bottom")
ax.bar([0], [0], color="#e34948", label="sign kept, $(-1)^k = +1$")
ax.bar([0], [0], color="#184f95", label="sign reversed, $(-1)^k = -1$")
ax.legend(loc="upper right")
```

`ks` is the array $0, 1, \ldots, 8$ and `heights` the numbers $\binom{8}{k}$. `colours` is red for even k and blue for odd k (`k % 2 == 0` tests evenness). `ax.bar` draws one bar per k. The axis labels are set, the vertical axis runs to 85, and `notes` writes, rotated by 90 degrees above four bars, which bilinear of the theory belongs there: S at $k = 0$; the kinetic terms, the current and the charge density at $k = 1$; the connection terms at $k = 3$; $\bar\Psi\Gamma\Psi$ at $k = 8$. The two bars of height zero only create the two entries of the legend.

```
save_figure(fig, "parity_of_products",
            "The 256 products $X$ of different gamma matrices, sorted by the number "
            "$k$ of factors (horizontal axis); the bar height is the number of "
            "such products, $\\binom{8}{k}$ (vertical axis). Red bars: the "
            "bilinear $\\bar\\Psi X\\Psi$ keeps its value under the chirality map "
            "$\\Psi \\to \\Gamma\\Psi$; blue bars: it changes sign. The sign is "
            "$(-1)^k$ for every one of the 256 products. The scalar $S$ sits in "
            "the red bar $k = 0$; the kinetic terms, the current $J^\\mu$ and the "
            "charge density $\\Psi^\\dagger B\\Psi$ in the blue bar $k = 1$; the "
            "spin-connection terms in the blue bar $k = 3$. This is why T1 "
            "reverses the kinetic term but not the mass term.")
```

The figure is saved as 18a.3. **What you see in Figure 18a.3:** a symmetric row of bars of heights 1, 8, 28, 56, 70, 56, 28, 8, 1, alternately red and blue. Why it matters: the mass term sits in a red bar (unchanged by Γ) and the kinetic term in blue bars (reversed); that difference is the whole of theorem T1.

In [12], Lemma 3: the reflection table.

```
order = list(range(8))  # the order x0, ..., x7 of the factors of Gamma
names = {a: f"x{a}" for a in range(8)}
rows = []  # one row of the reflection table per direction n
lemma3_ok = True
for n in range(8):
    P = Gamma @ gamma[n]
    others = I16.copy()
    for a in order:
        if a != n:
            others = others @ gamma[a]  # the product of the other seven gammas
    R = {a: (-1 if a == n else 1) for a in range(8)}  # the diagonal of R_n
    orthogonal = same(P.T @ P, I16)  # so P^-1 = P^T
    covers = all(same(P.T @ gamma[a] @ P, R[a] * gamma[a]) for a in range(8))
```

`order` is the order of the factors of Γ, and `names` gives the labels x0, ..., x7. For each direction n: `P` is $P_n = \Gamma\gamma^{(n)}$; `others` is the product of the seven other gammas in the order of Γ; `R` is the diagonal of the reflection R_n (`-1 if a == n else 1`). `orthogonal` checks $P_n^T P_n = 1$, so that $P_n^{-1} = P_n^T$ (true for every signed permutation matrix); `covers` checks $P_n^{-1}\gamma^{(a)}P_n = (R_n)_{aa}\gamma^{(a)}$ for all eight a: P_n acts on the gammas as the reflection of the direction n (Lemma 3, item 3).

```
    M = gamma[n]  # Gamma P_n = gamma^(n)
    kinetic_sign = eta[n]
    kinetic_ok = all(same(M.T @ C @ gamma[a] @ M, kinetic_sign * R[a] * C @ gamma[a])
                     for a in range(8))
    connection_ok = all(
        same(M.T @ C @ (gamma[a] @ four_S[(b, c)] + four_S[(b, c)] @ gamma[a]) @ M,
             kinetic_sign * R[a] * R[b] * R[c]
             * C @ (gamma[a] @ four_S[(b, c)] + four_S[(b, c)] @ gamma[a]))
        for a in range(8) for b in range(8) for c in range(8))
```

`M` is $\gamma^{(n)} = \Gamma P_n$, the matrix of the T2 map. `kinetic_ok` checks $M^T C\gamma^{(a)} M = \eta_{nn}(R_n)_{aa} C\gamma^{(a)}$ for all a: with the reflected frame the factor $(R_n)_{aa}$ is absorbed (Section 18.15), so the kinetic term gets the sign η_{nn}. `connection_ok` checks the same for the 512 connection kernels, with the three factors $(R_n)_{aa}(R_n)_{bb}(R_n)_{cc}$, two of which are absorbed by the reflected connection $R_n\omega R_n$.

```
    row = {"direction": names[n], "eta_nn": eta[n],
           "product_sign": sign_between(P, others),
           "character": sign_between(P.T @ C @ P, C),
           "S_sign": sign_between(M.T @ C @ M, C),
           "kinetic_sign": kinetic_sign if kinetic_ok else 0}
    row["map"] = ("(m, lambda) -> (-m, lambda), L -> +L (T2)"
                  if (row["S_sign"], row["kinetic_sign"]) == (-1, 1)
                  else "(m, lambda) -> (-m, -lambda), L -> -L (T1-type)")
    rows.append(row)
    lemma3_ok = (lemma3_ok and orthogonal and covers and connection_ok and kinetic_ok
                 and same(Gamma @ P, gamma[n]) and row["character"] == -eta[n]
                 and row["product_sign"] in (1, -1))
```

One row of the table: the direction, η_{nn}, the sign of P_n against the product of the other seven, the character (the sign in $P_n^T C P_n = \chi C$), the sign of S under $\Psi \to \gamma^{(n)}\Psi$ (the sign in $M^T C M = \pm C$), and the kinetic sign (0 if the kinetic check failed, which would make the table differ from the record). `row["map"]` writes the resulting map of the parameters: if S changes sign and the kinetic term does not, $(m, \lambda) \to (-m, \lambda)$ with $\mathcal{L} \to +\mathcal{L}$ (T2); otherwise a map of the T1 type. `lemma3_ok` collects all the conditions, including $\Gamma P_n = \gamma^{(n)}$ and the character $-\eta_{nn}$.

```
for row in rows:
    say(f"{row['direction']}: eta {row['eta_nn']:+d}, P_n = {row['product_sign']:+d} "
        f"x (other seven), character {row['character']:+d}, S -> "
        f"{row['S_sign']:+d} S, kinetic {row['kinetic_sign']:+d}: {row['map']}")
product_signs = [r["product_sign"] for r in rows]
reproduces(lemma3_ok and product_signs == [-1, 1, -1, 1, 1, -1, 1, -1],
           "Lemma 3: P_n in Pin(4,4), covers R_n, character -eta_nn, Gamma P_n = g^n",
           (WL, ["T2_Pn_in_Pin44", "T2_Pn_covers_the_reflection", "T2_character_of_Pn",
                 "T2_Gamma_times_Pn_is_gamma_n",
                 "T2_kernels_gamma_n_with_frame_reflection"]),
           (PY, ["T2.general_field.matrix_identities",
                 "T2.general_field.reflection_table"]))
```

The loop prints one line per direction (`:+d` writes a whole number with its sign); then the check requires all conditions and the recorded signs $-1, +1, -1, +1, +1, -1, +1, -1$, with five Wolfram checks and two sympy checks. Out [12] shows the eight rows: the space-like x_0, x_1, x_2, x_3 have character -1, reverse S and keep the kinetic term (T2); the time-like $x_4, \ldots, x_7$ have character $+1$, keep S and reverse the kinetic term (T1 type).

In [13], the table against the record.

```
theory = read_json(THEORY)
recorded_rows = theory["data"]["reflections"]
keys = [("eta_nn", "eta_nn"),
        ("product_sign", "P_n_equals_sign_times_product_of_other_seven"),
        ("character", "character_of_P_n"), ("S_sign", "S_sign_under_gamma_n"),
        ("kinetic_sign", "kinetic_sign_under_gamma_n_with_frame_reflection"),
        ("map", "mass_coupling_map")]
table_equal = len(recorded_rows) == 8 and all(
    recorded["direction"] == mine["direction"]
    and all(mine[k_mine] == recorded[k_rec] for k_mine, k_rec in keys)
    for mine, recorded in zip(rows, recorded_rows))
say("status of the theory record: " + theory["status"])
reproduces(table_equal and theory["status"] == "all checks of the report passed",
           "the reflection table equals the record's, row by row",
           (PY, ["compare.theory.reflection_table"]), table="reflections")
```

`theory` is the pairing theory record; `recorded_rows` its data table `reflections`, a list of eight dictionaries. `keys` pairs each field name of the notebook's rows with the field name of the record. `table_equal` requires eight recorded rows and, for each pair of rows (`zip` pairs them in order), the same direction and equal values in all six fields. The cell prints the status line of the record ("all checks of the report passed") and checks the table together with the sympy check `compare.theory.reflection_table`.

In [14], the fourth figure: the reflection table.

```
columns = ["$\\eta_{nn}$", "sign of $P_n$", "character $\\chi_n$",
           "$S \\to \\pm S$", "kinetic $\\pm$"]
grid = np.array([[r["eta_nn"], r["product_sign"], r["character"], r["S_sign"],
                  r["kinetic_sign"]] for r in rows], dtype=float)
fig, ax = plt.subplots(figsize=(8.6, 5.6))
ax.imshow(grid, cmap=DIVERGING, vmin=-1.0, vmax=1.0, aspect="auto")
for i in range(8):
    for j in range(5):
        ax.text(j, i, f"{int(grid[i, j]):+d}", ha="center", va="center",
                color="white", fontsize=10)
    label = "T2: $(-m, \\lambda)$, $+\\mathcal{L}$" if rows[i]["S_sign"] == -1 \
        else "T1-type: $(-m, -\\lambda)$, $-\\mathcal{L}$"
    ax.text(5.0, i, label, ha="left", va="center", fontsize=9)
ax.set_xticks(range(5), columns, fontsize=9)
ax.set_yticks(range(8), [f"$n = x_{a}$" for a in range(8)])
ax.set_xlim(-0.5, 7.4)
ax.grid(False)
ax.set_title("reflection of the direction $n$ with $\\Psi \\to \\gamma^{(n)}\\Psi$")
```

`columns` names the five columns of signs; `grid` is the 8×5 array of the signs (`dtype=float` because `imshow` draws numbers). `imshow` colours each sign (red $+1$, blue -1; `aspect="auto"` lets the cells be wider than high). The double loop writes each sign as white text in its cell; then the map of each row is written to the right of the grid (a backslash at the end of a line continues the statement on the next line). The axis ticks name the columns and the rows; `set_xlim(-0.5, 7.4)` leaves room for the text on the right.

```
save_figure(fig, "reflection_table",
            "The reflection table of Lemma 3, one row for each direction $n$ (the "
            "space-like $x_0, x_1, x_2, x_3$ and the time-like $x_4$ to $x_7$). "
            "Columns: the sign $\\eta_{nn}$ of the direction; the sign of "
            "$P_n = \\Gamma\\gamma^{(n)}$ relative to the product of the other "
            "seven gammas; the character $\\chi_n$ in $P_n^\\dagger CP_n = "
            "\\chi_nC$; the sign of $S$ and of the kinetic term under "
            "$\\Psi \\to \\gamma^{(n)}\\Psi$ with the reflected frame. Red is "
            "$+1$, blue is $-1$. The character is always $-\\eta_{nn}$. A "
            "space-like reflection keeps the kinetic term and reverses $S$: the "
            "mass changes sign and the coupling does not (theorem T2). A "
            "time-like reflection does the opposite, which gives a map of the T1 "
            "type. The table equals the Revision record row by row.")
```

The figure is saved as 18a.4. **What you see in Figure 18a.4:** the columns of the character, of the sign of S and of η_{nn} form the same pattern up to sign: character $= -\eta_{nn}$ in every row; the four space-like rows read "T2", the four time-like rows "T1-type".

In [15], the fifth figure: the chiral projectors.

```
P_minus = (I16 - Gamma) // 2  # whole numbers 0 and 1: the author's P_L
P_plus = (I16 + Gamma) // 2  # the author's P_R
projectors = (same(P_minus @ P_minus, P_minus) and same(P_plus @ P_plus, P_plus)
```

```
             and same(P_minus + P_plus, I16) and not (P_minus @ P_plus).any()
             and int(np.trace(P_minus)) == 8 and int(np.trace(P_plus)) == 8)
gamma_is_difference = same(Gamma, P_plus - P_minus)
gammas_swap = all(same(gamma[a] @ P_minus, P_plus @ gamma[a]) for a in range(8))
halves_kept = (same(C @ P_minus, P_minus @ C)
               and all(same(M @ P_minus, P_minus @ M) for M in four_S.values()))
say(f"projectors of rank 8: {projectors}; Gamma = P+ - P-: {gamma_is_difference}; "
    f"every gamma maps the minus half to the plus half: {gammas_swap}; C and all "
    f"S^ab keep each half: {halves_kept}")
check(projectors and gamma_is_difference and gammas_swap and halves_kept,
      "the chiral projectors: P-^2 = P-, P+^2 = P+, Gamma = P+ - P-, gammas swap")
```

`P_minus` is $\frac{1}{2}(1-\Gamma)$; since $1-\Gamma$ has entries 0 and 2, the whole-number division `// 2` gives exactly 0 and 1. `projectors` checks $P_-P_- = P_-$, $P_+P_+ = P_+$, $P_- + P_+ = 1$, $P_-P_+ = 0$ (`.any()` is false when all entries are 0, so `not (...).any()` means "is the zero matrix") and that each has trace 8 (the trace of a projector is its rank, the number of components it keeps). `gamma_is_difference` checks $\Gamma = P_+ - P_-$; `gammas_swap` checks $\gamma^{(a)}P_- = P_+\gamma^{(a)}$ for all a; `halves_kept` checks that C and all S^{ab} commute with P_- (hence also with $P_+ = 1 - P_-$). One check combines them.

```
fig, axes = plt.subplots(1, 4, figsize=(12.0, 3.6))
draw_matrix(axes[0], P_minus, "$P_- = \\frac{1}{2}(1 - \\Gamma)$")
draw_matrix(axes[1], P_plus, "$P_+ = \\frac{1}{2}(1 + \\Gamma)$")
draw_matrix(axes[2], gamma[1] @ P_minus, "$\\gamma^{(x_1)}P_-$")
image = draw_matrix(axes[3], P_plus @ gamma[1], "$P_+\\gamma^{(x_1)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "chiral_projectors",
            "Heat maps of the two chiral projectors and of one gamma between them; "
            "columns horizontal, rows vertical, red $+1$, blue $-1$, grey 0, thin "
            "lines between the halves. $P_-$ (first picture, the author's P_L) is "
            "the identity on the components 1 to 8 and zero elsewhere; $P_+$ "
            "(second, the author's P_R) is the identity on 9 to 16. Their "
            "difference is $\\Gamma$. The third and fourth pictures are equal: "
            "$\\gamma^{(x_1)}P_- = P_+\\gamma^{(x_1)}$, so the gamma takes a field "
            "that lives in the first half into the second half. The T1 image "
            "$\\Gamma\\Psi = P_+\\Psi - P_-\\Psi$ reverses the relative sign of "
            "the two halves.")
```

Four heat maps: P_-, P_+, $\gamma^{(x_1)}P_-$ and $P_+\gamma^{(x_1)}$; saved as 18a.5. **What you see in Figure 18a.5:** P_- is red on the first eight diagonal places, P_+ on the last eight; the third and the fourth panels are identical and occupy only the lower-left block: a field that lives in the first half is moved by the gamma into the second half.

In [16], the sixth figure: the block form.

```
def blocks(M):
    """The four 8 x 8 blocks of a 16 x 16 matrix: upper left, upper right, lower
    left, lower right."""
    return M[:8, :8], M[:8, 8:], M[8:, :8], M[8:, 8:]
```

`blocks(M)` returns the four 8×8 blocks of a 16×16 matrix: `M[:8, :8]` is rows 1 to 8 and columns 1 to 8 (in Python, `:8` means the places 0 to 7), `M[:8, 8:]` rows 1 to 8 and columns 9 to 16, and so on.

```
off_diagonal = all(not blocks(gamma[a])[0].any() and not blocks(gamma[a])[3].any()
                   for a in range(8))  # .any() is False when all entries are 0
g0_swaps = (same(blocks(gamma[0])[1], I8) and same(blocks(gamma[0])[2], I8))
C_ul, C_ur, C_ll, C_lr = blocks(C)
C_block_diagonal = not C_ur.any() and not C_ll.any() and same(C_ul, -C_lr)
P0 = Gamma @ gamma[0]
say(f"every gamma off-diagonal: {off_diagonal}; gamma^(x0) = [[0, I8], [I8, 0]]: "
    f"{g0_swaps}; C = diag(-sigma, sigma): {C_block_diagonal}")
check(off_diagonal and g0_swaps and C_block_diagonal,
      "block form: gammas exchange the halves, gamma^(x0) swaps them, C is diagonal")
```

`off_diagonal` checks that the upper-left and lower-right blocks of every gamma are zero. `g0_swaps` checks that both off-diagonal blocks of $\gamma^{(x_0)}$ are I_8, so that $\gamma^{(x_0)}(\psi_-, \psi_+) = (\psi_+, \psi_-)$. `C_block_diagonal` checks that the off-diagonal blocks of C vanish and that its upper-left block is minus its lower-right block: $C = \mathrm{diag}(-\sigma, \sigma)$. `P0` is $P_0 = \Gamma\gamma^{(x_0)}$, drawn below. One check combines the three facts.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], C, "$C = \\mathrm{diag}(-\\sigma, \\sigma)$")
draw_matrix(axes[1], gamma[0], "$\\gamma^{(x_0)}$: exchanges the halves")
image = draw_matrix(axes[2], P0, "$P_0 = \\Gamma\\gamma^{(x_0)}$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry")
save_figure(fig, "chiral_blocks",
            "Heat maps of $C$ (left), $\\gamma^{(x_0)}$ (middle) and the reflection "
            "$P_0 = \\Gamma\\gamma^{(x_0)}$ of the hidden direction (right); axes "
            "as in the first figure (columns horizontal, rows vertical, red $+1$, "
            "blue $-1$, grey 0), thin lines between the chiral halves. $C$ lives "
            "in the two diagonal blocks, with opposite signs: "
            "$S = -\\psi_-^\\dagger\\sigma\\psi_- + \\psi_+^\\dagger\\sigma\\psi_+$. "
            "$\\gamma^{(x_0)}$ is the identity in both off-diagonal blocks: it "
            "exchanges $\\psi_-$ and $\\psi_+$, which reverses $S$ (the T2 image). "
            "$P_0$ differs from $\\gamma^{(x_0)}$ by the sign of one block, the "
            "work of $\\Gamma$.")
```

Three heat maps, C, $\gamma^{(x_0)}$ and P_0; saved as 18a.6. **What you see in Figure 18a.6:** C fills the two diagonal blocks with patterns of opposite colours; $\gamma^{(x_0)}$ is two red diagonals in the off-diagonal blocks; P_0 is $\gamma^{(x_0)}$ with one of its two blocks turned blue. Why: $S = -\psi_-^\dagger\sigma\psi_- + \psi_+^\dagger\sigma\psi_+$, so swapping the halves (the T2 image) reverses S.

In [17], Lemma 4: the 17 Krein signs.

```
maps = [("Gamma", Gamma)]
maps += [(f"gamma^x{n}", gamma[n]) for n in range(8)]
maps += [(f"P_x{n}", Gamma @ gamma[n]) for n in range(8)]
krein = {name: sign_between(M @ B_over_i @ M.T, B_over_i) for name, M in maps}
say("M B M^dagger = sigma B: " + ", ".join(f"{name} {s:+d}"
                                           for name, s in krein.items()))
```

`maps` is a list of 17 pairs (name, matrix): Γ, the eight $\gamma^{(n)}$ and the eight P_n. For a real M, $MBM^\dagger = i\,M(B/i)M^T$, so the sign σ in $MBM^\dagger = \sigma B$ is the sign between `M @ B_over_i @ M.T` and `B_over_i`, exact whole numbers. `krein` stores the 17 signs and the cell prints them in one line.

```
recorded_krein = {row["map"]: row["sign"]
                  for row in theory["data"]["Krein_signs_M_B_Mdagger"]}
reproduces(krein == recorded_krein and krein["Gamma"] == -1 and krein["gamma^x0"] == 1,
           "Lemma 4: the 17 Krein signs equal the record's; Gamma -1, gamma^(x0) +1",
           (WL, ["Q_Krein_metric_of_images"]),
           (PY, ["compare.theory.krein_signs", "Q.T2_image_keeps_B"]),
           table="Krein_signs_M_B_Mdagger")
```

`recorded_krein` turns the record's data table `Krein_signs_M_B_Mdagger` into a dictionary from the map's name to its sign; the check requires the two dictionaries to be equal (same names, same signs) and names the Wolfram check `Q_Krein_metric_of_images` and the sympy checks `compare.theory.krein_signs` and `Q.T2_image_keeps_B`. Out [17] lists: Γ: -1; $\gamma^{(x_0)}, \gamma^{(x_1)}, \gamma^{(x_2)}, \gamma^{(x_3)}, \gamma^{(x_4)}$: $+1$; $\gamma^{(x_5)}, \gamma^{(x_6)}, \gamma^{(x_7)}$: -1; $P_{x_0}, \ldots, P_{x_4}$: -1; $P_{x_5}, P_{x_6}, P_{x_7}$: $+1$.

In [18], the seventh figure: the Krein signs.

```
fig, axes = plt.subplots(1, 3, figsize=(11.0, 4.0))
draw_matrix(axes[0], B_over_i, "imaginary part of $B$")
draw_matrix(axes[1], Gamma @ B_over_i @ Gamma.T,
            "imaginary part of $\\Gamma B\\Gamma^\\dagger = -B$")
image = draw_matrix(axes[2], gamma[0] @ B_over_i @ gamma[0].T,
                    "imaginary part of $\\gamma^{(x_0)}B\\gamma^{(x_0)\\dagger} = B$")
fig.colorbar(image, ax=list(axes), shrink=0.8, label="entry divided by $i$")
save_figure(fig, "krein_signs",
            "Heat maps of the imaginary parts of three 16 by 16 matrices (their "
            "real parts are zero): the Krein matrix $B = -iC\\gamma^{(x_4)}$ (left), "
            "$\\Gamma B\\Gamma^\\dagger$ (middle) and "
            "$\\gamma^{(x_0)}B\\gamma^{(x_0)\\dagger}$ (right); columns horizontal, "
            "rows vertical, red $+1$, blue $-1$, grey 0. The middle picture is the "
            "left one with all colours reversed: the chirality image "
            "$\\Gamma\\Psi$ of a quantised field has the anticommutator $-B$. The "
            "right picture equals the left one: the mirror image "
            "$\\gamma^{(x_0)}\\Psi$ of theorem T2 keeps $+B$, so it is an ordinary "
            "copy of the theory with equal energies.")
```

Three heat maps of the imaginary parts (that is, of `B_over_i` and its images): B, $\Gamma B \Gamma^T$ and $\gamma^{(x_0)} B \gamma^{(x_0)T}$; saved as 18a.7. **What you see in Figure 18a.7:** the middle panel is the left one with every colour reversed (the T1 image carries $-B$), and the right panel is identical to the left one (the T2 image keeps $+B$). Section 18.23 explains what this means for the quantised field.

In [19], the last check.

```
figure_names = ["eight_gammas", "chirality_flips_gammas", "parity_of_products",
                "reflection_table", "chiral_projectors", "chiral_blocks",
                "krein_signs"]
paths = [output_file(f"{FIGURE_FOLDER}/18a_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

`figure_names` lists the seven figure names in order; `enumerate(figure_names, 1)` pairs each with its number, starting at 1, so `paths` holds the seven file paths. The check requires all seven files to exist, and `all_checks_passed()` prints the last line: ALL 16 CHECKS PASSED (notebook 18a).

What Notebook 18a established. PROVED by exact whole-number arithmetic: the matrices of the record are the author's (rebuilt from their formulas); Lemma 1; Lemma 2 for all 256 products and all 456 Lagrangian kernels; Lemma 3 with the reflection table of the record; the block form; Lemma 4 with the 17 Krein signs of the record. Nothing in the notebook concerns a particular solution or any process that creates a universe.

18.10 Theorem T1: the chirality pairing

Hypotheses.

- (H1) The gravitational field: ANY vielbein $e^a{}_\mu$ (so any metric $g_{\mu\nu} = \sum_a \eta_{aa} e^a{}_\mu e^a{}_\nu$ of signature (4,4)) and ANY spin connection $\omega_{\mu ab} = -\omega_{\mu ba}$, in particular the canonical connection of the author's metric with any history $a_4(x_4)$ and any second function $b_4(x_4)$. The field is a fixed background: it is not varied, and both members of the pair live in the SAME field.
- (H2) The statistics: Ψ has Grassmann components (dirac16complex) or commuting components (dirac16complex00). The map acts at the same point, $\Psi'(x) = \Gamma\Psi(x)$, with no change of coordinates.
- (H3) The potential: $U(S) = \frac{\lambda}{2} S^2$; more generally any function U (commuting field) or any polynomial U (Grassmann field), mapped to $-U$.
- (H4) $\mathcal{L}$, E, T and J as in Section 18.4, with any fixed overall sign or factor of T and J.

Theorem T1. Under (H1) to (H4), for both fields:

- (T1a) $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$ for every configuration (off shell); for a general potential $\mathcal{L}_{m,U}[\Gamma\Psi] = -\mathcal{L}_{-m,-U}[\Psi]$. Separately, $S[\Gamma\Psi] = S[\Psi]$ and $K[\Gamma\Psi] = -K[\Psi]$.
- (T1b) $E_{m,\lambda}[\Gamma\Psi] = -\Gamma E_{-m,-\lambda}[\Psi]$: $\Gamma\Psi$ solves the field equations with (m, λ) if and only if Ψ solves those with $(-m, -\lambda)$. Equivalently, because $\Gamma\Gamma = 1$: Ψ solves the (m, λ) equations if and only if $\Gamma\Psi$ solves the $(-m, -\lambda)$ equations.
- (T1c) $T_{\mu\nu}^{(-m,-\lambda)}[\Gamma\Psi] = -T_{\mu\nu}^{(m,\lambda)}[\Psi]$ and $J^\mu[\Gamma\Psi] = -J^\mu[\Psi]$, at every point, off and on shell.
- (T1d) The pair (Ψ with (m, λ); $\Gamma\Psi$ with $(-m, -\lambda)$) has the total energy-momentum density $T + T' = 0$, the total current $J + J' = 0$ and the total charge $Q + Q' = 0$, as classical bilinears: ordinary numbers for dirac16complex00, elements of the Grassmann algebra for dirac16complex.

Status: PROVED. Records: `wolfram-pairing.json`, the 51 checks of the T1 group (four gravitational fields, both statistics; their names begin with `T1_`, plus `Gamma_properties`); `python-pairing.json`, the 23 checks whose names begin with `T1.` (the comparison check `compare.theorem_T1` lists the 17 of them that confirm the Wolfram theorem independently); the theorem record `pairing-theory.json`, theorem T1. Notebook 18b proves it again in the author's metric (In [10] to In [12]); Notebook 18c shows it on actual solutions (In [7] to In [11]).

18.11 The proof of T1, line by line

Step 1: the adjoint and the derivatives. By Lemma 2, $\overline{\Gamma\Psi} = \bar{\Psi}\Gamma$. By Lemma 1,

$$D_\mu(\Gamma\Psi) = \Gamma D_\mu\Psi, \qquad D_\mu\overline{\Gamma\Psi} = D_\mu(\bar{\Psi}\Gamma) = (D_\mu\bar{\Psi})\Gamma.$$

These hold for every connection, because Γ commutes with every S^{ab}.

Step 2: the scalar and the kinetic term. Insert $\Psi' = \Gamma\Psi$ into S:

$$S[\Gamma\Psi] = \overline{\Gamma\Psi}\,\Gamma\Psi = \bar{\Psi}\Gamma\Gamma\Psi = \bar{\Psi}\Psi = S[\Psi].$$

The first equality is the definition of S; the second is Step 1; the third is $\Gamma\Gamma = 1$ (Lemma 1). Now the first half of the kinetic term:

$$\overline{\Gamma\Psi}\,\gamma^\mu D_\mu(\Gamma\Psi) = \bar{\Psi}\,\Gamma\gamma^\mu\Gamma\,D_\mu\Psi = -\bar{\Psi}\gamma^\mu D_\mu\Psi.$$

The first equality is Step 1 twice. The second uses $\Gamma\gamma^\mu\Gamma = -\gamma^\mu$: $\gamma^\mu = \sum_a e^\mu{}_a\gamma^{(a)}$ is a combination of gammas with ordinary functions as coefficients, and each $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}$ (Lemma 2 with $k = 1$). The second half changes sign in the same way:

$$(D_\mu\overline{\Gamma\Psi})\gamma^\mu\,\Gamma\Psi = (D_\mu\bar{\Psi})\,\Gamma\gamma^\mu\Gamma\,\Psi = -(D_\mu\bar{\Psi})\gamma^\mu\Psi.$$

Subtracting the two lines and multiplying by $\frac{1}{2}$ gives

$$K[\Gamma\Psi] = -K[\Psi].$$

In the written-out form of Section 18.4, K is a sum of coefficient functions of the gravitational field times bilinears with an odd number of gammas; each such bilinear changes sign by Lemma 2, whatever the coefficients are.

Step 3: the Lagrangian. $U(S)$ is unchanged because S is. So

$$\mathcal{L}_{m,U}[\Gamma\Psi] = \sqrt{|g|}\,[\,-K - mS - U(S)].$$

This inserts Step 2 into the definition of $\mathcal{L}$; $\sqrt{|g|}$ is a property of the field, not of Ψ.

$$\sqrt{|g|}\,[\,-K - mS - U(S)] = -\sqrt{|g|}\,[K - (-m)S - (-U)(S)] = -\mathcal{L}_{-m,-U}[\Psi].$$

We took out the factor -1 and recognised the Lagrangian with the mass $-m$ and the potential $-U$. For $U = \frac{\lambda}{2}S^2$, $-U = \frac{-\lambda}{2}S^2$, which is (T1a).

Step 4: the field equations. Using Step 1 and $\gamma^\mu\Gamma = -\Gamma\gamma^\mu$:

$$\gamma^\mu D_\mu(\Gamma\Psi) = \gamma^\mu\Gamma D_\mu\Psi = -\Gamma\gamma^\mu D_\mu\Psi.$$

Insert this and $S[\Gamma\Psi] = S$ into $E_{m,\lambda}$:

$$E_{m,\lambda}[\Gamma\Psi] = -\Gamma\gamma^\mu D_\mu\Psi - (m+\lambda S)\Gamma\Psi = -\Gamma\big[\gamma^\mu D_\mu\Psi - (-m-\lambda S)\Psi\big] = -\Gamma E_{-m,-\lambda}[\Psi].$$

The middle step takes out $-\Gamma$ (a constant matrix commutes with the numbers m, λ, S); the last is the definition of E with $(-m,-\lambda)$. Since Γ is invertible, $E_{m,\lambda}[\Gamma\Psi] = 0$ exactly when $E_{-m,-\lambda}[\Psi] = 0$: this is (T1b). The same holds for the Euler-Lagrange expressions derived directly from $\mathcal{L}$, whatever their explicit form: (T1a) is an identity between functions of Ψ, and $\Psi \to \Gamma\Psi$ is a constant invertible linear substitution, so by the chain rule the Euler-Lagrange expressions of $\mathcal{L}_{m,\lambda}$ at $\Gamma\Psi$, multiplied by Γ, equal minus those of $\mathcal{L}_{-m,-\lambda}$ at Ψ; they vanish together. (The overall sign of a Lagrangian does not change its Euler-Lagrange equations: $-\mathcal{L}$ and $\mathcal{L}$ have the same solutions.)

Step 5: the energy-momentum tensor and the current. $T_{\mu\nu}$ is a sum of the kinetic bilinears $\bar\Psi\gamma_\mu D_\nu\Psi$ and $(D_\mu\bar\Psi)\gamma_\nu\Psi$ (one gamma each) and of $-g_{\mu\nu}\mathcal{L}/\sqrt{|g|}$. By Step 2 each kinetic bilinear changes sign under $\Psi \to \Gamma\Psi$: the lines of Step 2 hold with the free index ν on D_ν and with $\gamma_\mu = \sum_\rho g_{\mu\rho}\gamma^\rho$ in place of γ^μ, because γ_μ is again a combination of gammas with ordinary functions as coefficients, so $\Gamma\gamma_\mu\Gamma = -\gamma_\mu$; for example $\overline{\Gamma\Psi}\,\gamma_\mu D_\nu(\Gamma\Psi) = \bar\Psi\,\Gamma\gamma_\mu\Gamma\,D_\nu\Psi = -\bar\Psi\gamma_\mu D_\nu\Psi$. By Step 3 with m replaced by $-m$ and λ by $-\lambda$, $\mathcal{L}_{-m,-\lambda}[\Gamma\Psi] = -\mathcal{L}_{m,\lambda}[\Psi]$. Therefore

$$T^{(-m,-\lambda)}_{\mu\nu}[\Gamma\Psi] = -\frac{1}{4}(\cdots)_\Psi - g_{\mu\nu}\frac{-\mathcal{L}_{m,\lambda}[\Psi]}{\sqrt{|g|}} = -T^{(m,\lambda)}_{\mu\nu}[\Psi],$$

where $(\cdots)_\Psi$ is the bracket of kinetic bilinears of Ψ. The current is a multiple of $\bar\Psi\gamma^\mu\Psi$, a bilinear with one gamma, so $J^\mu[\Gamma\Psi] = -J^\mu[\Psi]$; and the charge $Q = \int\sqrt{|g|}\,J^{x_4}\,d^7x$ changes sign with it. This is (T1c).

Step 6: the pair. Add (T1c) to the quantities of Ψ: $T^{(m,\lambda)}[\Psi]+T^{(-m,-\lambda)}[\Gamma\Psi] = 0$, $J[\Psi]+J[\Gamma\Psi] = 0$, $Q + Q' = 0$. This is (T1d).

Step 7: both statistics. $\Gamma = \mathrm{diag}(-I_8, I_8)$ multiplies each component by $+1$ or -1. No two Grassmann components are ever exchanged, so every line above holds word for word for anticommuting components, as identities between elements of the Grassmann algebra.

The proof used only Lemmas 1 and 2 and the fact that $\mathcal{L}$, T and J are sums of coefficient functions of the gravitational field ($\sqrt{|g|}$, $e^\mu{}_a$, $\omega_{\mu ab}$) times bilinears. It therefore holds in every gravitational field (H1). QED.

18.12 T1 in the block form of the author's field

In the author's field the proof can also be read off the field equation itself. In the chiral block form $\Psi = (\psi_-, \psi_+)$ every gamma is block off-diagonal; write its upper-right block $\bar\tau^a$ and its lower-left block τ^a (for the author's gammas these are their matrices $\bar\tau_A$ and τ_A, and $\bar\tau^0 = \tau^0 = I_8$). Then $\gamma^{(a)}(\psi_-,\psi_+) = (\bar\tau^a\psi_+, \tau^a\psi_-)$, and the 16 equations of Section 18.4 become the two 8-component equations

$$\sum_a f_a^{-1}\bar\tau^a\partial_a\psi_+ + 3H\psi_+ = V\psi_-, \qquad \sum_a f_a^{-1}\tau^a\partial_a\psi_- + 3H\psi_- = V\psi_+, \qquad V = m+\lambda S.$$

The first equation is the upper half (rows 1 to 8) of $\gamma^\mu\partial_\mu\Psi + 3H\gamma^{(x_0)}\Psi = V\Psi$, the second the lower half; $3H\gamma^{(x_0)}$ contributes $3H\psi_+$ to the upper and $3H\psi_-$ to the lower half because both blocks of $\gamma^{(x_0)}$ are I_8. Now apply T1: $\Gamma(\psi_-,\psi_+) = (-\psi_-,\psi_+)$, $S = -\psi_-^\dagger\sigma\psi_- + \psi_+^\dagger\sigma\psi_+$ is unchanged (the sign of ψ_- appears twice), and with the parameters $(-m,-\lambda)$ the coefficient becomes $-m-\lambda S = -V$. The two equations for the new field with $(-m,-\lambda)$ read

$$\sum_a f_a^{-1}\bar\tau^a\partial_a\psi_+ + 3H\psi_+ = (-V)(-\psi_-), \qquad -\Big(\sum_a f_a^{-1}\tau^a\partial_a\psi_- + 3H\psi_-\Big) = -V\psi_+,$$

which are the old equations: the first is unchanged because $(-V)(-\psi_-) = V\psi_-$, and the second is the old one multiplied by -1. None of the gravitational factors, $e^{\mp a_4}\sin^{-1/6}z$, $\tan z$ and $3H$, changed. This is T1 in the author's metric, PROVED again in this form by the records `wolfram-pairing.json`, checks `T1_field_equation_covariance_primordial_commuting` and `T1_field_equation_covariance_primordial_grassmann`.

18.13 What T1 says, and what it does not say

Both m and λ must change sign. The mass term keeps its sign under Γ only if m is replaced by $-m$, and the potential only if λ is replaced by $-\lambda$. The **negative controls** of the record show that nothing weaker works: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{m,\lambda}[\Psi] \neq 0$ and $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,\lambda}[\Psi] \neq 0$ (`wolfram-pairing.json`, checks `T1_Lagrangian_negative_controls_primordial_commuting` and its Grassmann, diagonal8 and pointwise twins; `python-pairing.json`, checks `T1.metric.commuting.negative_controls` and `T1.metric.grassmann.negative_controls`; Notebook 18b, In [10] and In [11]; Notebook 18c, In [7], where the wrong partner $(-m,+\lambda)$ drifts away from $\Gamma\Psi$ by 1.641). So for $\lambda \neq 0$, T1 is not a pure $+m$ / $-m$ pairing; for $\lambda = 0$ (free fields) it pairs $+m$ with $-m$ exactly.

The partner's energy density, pressures and equations of state. By (T1c), $\rho' = -\rho$ and $p'_\mu = -p_\mu$ in every direction. For homogeneous solutions (Section 18.21) the record gives $\rho = mS + \frac{\lambda}{2}S^2$ and $p = \frac{\lambda}{2}S^2$ in every direction (`python-field-theory.json`, check `commuting_homogeneous_on_shell_rho_p`); the partner has $S' = S$ and $\rho' = -mS - \frac{\lambda}{2}S^2$, $p' = -\frac{\lambda}{2}S^2$. Every ratio of two components is the same for both members, in particular the equations of state $w_3 = p_3/\rho$, $w_t = p_t/\rho$ and $w_0 = p_0/\rho$.

T1 is not a symmetry of one theory. A **symmetry** maps the solutions of one theory to solutions of the SAME theory. T1 changes the parameters and the sign of the action; it maps the theory (m,λ) onto the theory $(-m,-\lambda)$. Because $-\mathcal{L}$ and $\mathcal{L}$ have the same field equations, $\Gamma\Psi$ is a solution of the $(-m,-\lambda)$ theory; nothing more is asserted.

What the map does to the components. Γ is -1 on one irreducible half of Spin(4,4) and $+1$ on the other; the partner differs from Ψ only by the relative sign of its two inequivalent halves (Figure 18a.5). Γ is itself an element of Pin(4,4), a product of eight unit vectors.

T1 and charge conjugation (rule of this book: charge conjugation is a matrix). Chapter 5 derived the two charge-conjugation matrices of the author's gammas. Every matrix M with $M(\gamma^{(a)})^* = s\,\gamma^{(a)}M$ for all a is a multiple of 1 when $s = +1$ and a multiple of Γ when $s = -1$ (`charge-conjugation-and-u1.json`, checks `intertwiners_same_mass` and `intertwiners_reversed_mass`). This gives two charge-conjugation matrices: $\mathcal{C}_+ = C$, with $\mathcal{C}_+^{-1}\gamma^{(a)}\mathcal{C}_+ = -(\gamma^{(a)})^T$, which keeps the mass and gives the conjugate field $\Psi^c = \mathcal{C}_+\bar\Psi^T = \Psi^*$; and $\mathcal{C}_- = \Gamma C$, with $\mathcal{C}_-^{-1}\gamma^{(a)}\mathcal{C}_- = +(\gamma^{(a)})^T$, which REVERSES the mass and gives $\Psi^c = \mathcal{C}_-\bar\Psi^T = \Gamma\Psi^*$ (checks `charge_c`

onjugation_matrix_plus and charge_conjugation_matrix_minus). Because the author's gammas, C and the spin connection are REAL, plain complex conjugation of a REAL field does nothing: a real commuting field is its own $\mathcal{C}_+$ conjugate and carries no U(1) charge ($J^\mu = 0$ identically), so for a real field complex conjugation cannot be what exchanges matter and antimatter (checks representation_real and real_fields_charge_conjugation). For a COMPLEX field the conjugations are defined by the matrices and act differently: the same-mass conjugation by $\mathcal{C}_+$, whose result happens to be $\Psi^c = \mathcal{C}_+\bar{\Psi}^T = \Psi^*$, keeps S and REVERSES J for commuting components, $(S, J) \to (S, -J)$; for Grassmann components it gives $(S, J) \to (-S, +J)$ as classical bilinears, and normal ordering adds no sign (it subtracts the vacuum value, a number), so the quantised field has the same $(S, J) \to (-S, +J)$ (the record's corrected detail text of that check; Section 5.34 and Section 21.26 make the Fock-space computation); the mass-reversing conjugation by $\mathcal{C}_-$, $\Psi^c = \Gamma\Psi^*$, gives $(S, +J)$ for commuting and $(-S, -J)$ for Grassmann components (check bilinears_under_charge_conjugation). In every case the charge conjugation is the matrix $\mathcal{C}_+$ or $\mathcal{C}_-$ applied to $\bar{\Psi}^T$; complex conjugation by itself is not called charge conjugation in this book. For such a real field, $\Psi^* = \Psi$, so the $\mathcal{C}_-$ conjugate $\Gamma\Psi^*$ is exactly the T1 image $\Gamma\Psi$: **on real fields, T1 is the mass-reversing charge conjugation by the matrix** Γ, and it maps solutions with (m, λ) to solutions with $(-m, -\lambda)$ (same check). For complex fields the two maps differ: T1 is linear ($\Psi \to \Gamma\Psi$) and reverses J, while the $\mathcal{C}_-$ conjugation is antilinear ($\Psi \to \Gamma\Psi^*$); for commuting components it keeps J (check bilinears_under_charge_conjugation). For the quantised field the conjugation that preserves the canonical anticommutator is $\Psi \to \Gamma\Psi^{\dagger T}$, and it too reverses the mass (check quantum_charge_conjugation_unitary_type). Chapter 21 uses these facts for matter and antimatter.

18.14 Theorem T2: the mirror pairing

T1 reverses the energy and needs $\lambda \to -\lambda$. Is there a map that pairs (m, λ) with $(-m, \lambda)$, the same coupling? Lemma 3 points to it: the reflections of the space-like directions have the character -1, so they reverse S and keep the kinetic term.

Hypotheses.

- (H5) n is a space-like frame direction (x_0, x_1, x_2 or x_3), and $P_n = \Gamma\gamma^{(n)}$ is the Pin(4,4) lift of the reflection R_n, of character -1 (Lemma 3).
- (H6) General-field version: the frame is reflected, $e' = R_n e$, that is ${e'}^a{}_\mu = (R_n)_{aa} e^a{}_\mu$ (the same metric, since $R_n \eta R_n = \eta$, and the same $\sqrt{|g|}$), with its own canonical connection; and $\Psi'(x) = \Gamma P_n \Psi(x) = \gamma^{(n)}\Psi(x)$ at the same point.
- (H7) Author's-field version: the map ϕ: $x_0 \to \pi/(6H) - x_0$, that is $z \to \pi - z$, takes the patch $0 < z < \pi/2$ onto the mirror patch $\pi/2 < z < \pi$; each patch carries its own positive vielbein ($f_0 = |\cot z|$, which is $-\cot z$ on the mirror patch); and $\Psi'(\phi(x)) = \gamma^{(x_0)}\Psi(x)$. The brane $z = \pi/2$ is a degenerate surface of the metric ($g_{00} = \cot^2 z = 0$ and $\sqrt{|g|} = \cos z = 0$ there); extending the field to the mirror patch is the Z2 construction, ASSUMED.
- (H8) Both statistics; $U(S) = \frac{\lambda}{2}S^2$, or any even function of S.

Theorem T2. Under (H5) to (H8), for both fields:

- (T2a) $\mathcal{L}_{m,\lambda}[\gamma^{(n)}\Psi;\, R_n e] = +\mathcal{L}_{-m,\lambda}[\Psi;\, e]$ in every gravitational field; in the author's field $\mathcal{L}_{m,\lambda}[\Psi'](\phi(x)) = \mathcal{L}_{-m,\lambda}[\Psi](x)$.
- (T2b) $S' = -S$, and $E_{m,\lambda}[\gamma^{(n)}\Psi;\, R_n e] = -\gamma^{(n)} E_{-m,\lambda}[\Psi;\, e]$: Ψ solves the $(-m,\lambda)$ equations if and only if its image solves the (m,λ) equations (in the author's field: on the patch and on the mirror patch respectively).
- (T2c) In the general-field version $T'_{\mu\nu} = T^{(-m,\lambda)}_{\mu\nu}[\Psi]$ and $J'^{\mu} = J^{\mu}[\Psi]$. In the author's field the coordinate components pick up the reflection of x_0: $T_{\mu\nu}[\Psi'](\phi(x)) = \Lambda_\mu \Lambda_\nu\, T^{(-m,\lambda)}_{\mu\nu}[\Psi](x)$ and $J'^{\mu} = \Lambda_\mu J^{\mu}$, with $\Lambda = \mathrm{diag}(-1,1,1,1,1,1,1,1)$. The energy density, the charge density and every component without an index x_0 are EQUAL, not opposite. T2 pairs $+m$ with $-m$ at equal energy-momentum.
- (T2d) For a time-like n (character $+1$) the same construction gives $\mathcal{L} \to -\mathcal{L}_{-m,-\lambda}$, a map of the T1 type. The character -1 is what makes T2 a pairing $(m,\lambda) \to (-m,\lambda)$ with $\mathcal{L} \to +\mathcal{L}$.

Status: PROVED, the author's-field version under the ASSUMED Z2 construction. Records: `wolfram-pairing.json`, the 28 checks of the T2 group (the frame reflections of all eight directions in a general field at a point, both statistics, and the mirror in the author's field); `python-pairing.json`, the 16 checks of the T2 group (names beginning with `T2.` and the two geometry checks of the mirror); theorem T2 of `pairing-theory.json`. Notebook 18b proves it in the author's metric (In [15] to In [19]); Notebook 18c shows it on actual solutions (In [12] and In [13]).

18.15 The proof of T2 in a general field

Fix a space-like n in the end, but keep η_{nn} general as long as possible, so that (T2d) comes out too.

Step 1: the geometry. The reflected frame gives the same metric:

$$g'_{\mu\nu} = \sum_a \eta_{aa}\,(R_n)_{aa} e^a{}_\mu\,(R_n)_{aa} e^a{}_\nu = \sum_a \eta_{aa} e^a{}_\mu e^a{}_\nu = g_{\mu\nu}.$$

$(R_n)^2_{aa} = 1$. Its canonical connection is $R_n \omega R_n$: in the formula $\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu e^\nu{}_b + \Gamma^\nu{}_{\mu\lambda} e^\lambda{}_b)$ the new vielbein carries a factor $(R_n)_{aa}$, its inverse a factor $(R_n)_{bb}$, the constant factors come out of ∂_μ, and the Christoffel symbols of the unchanged metric are unchanged:

$$\omega'_{\mu ab} = (R_n)_{aa}(R_n)_{bb}\,\omega_{\mu ab}.$$

By Lemma 3, item 2, $(R_n)_{aa}(R_n)_{bb} S^{ab} = \gamma^{(n)} S^{ab} (\gamma^{(n)})^{-1}$, so

$$\Omega'_\mu = \frac{1}{2}\sum_{a,b} (R_n)_{aa}(R_n)_{bb}\,\omega_{\mu ab} S^{ab} = \gamma^{(n)}\,\Omega_\mu\,(\gamma^{(n)})^{-1},$$

and therefore $D'_\mu(\gamma^{(n)}\Psi) = \gamma^{(n)}\partial_\mu\Psi + \gamma^{(n)}\Omega_\mu(\gamma^{(n)})^{-1}\gamma^{(n)}\Psi = \gamma^{(n)} D_\mu \Psi$.

Step 2: the adjoint. $\gamma^{(n)}$ is real with $(\gamma^{(n)})^T = \eta_{nn}\gamma^{(n)}$, and from $C\gamma^{(n)}C^{-1} = -(\gamma^{(n)})^T = -\eta_{nn}\gamma^{(n)}$ we get $C\gamma^{(n)} = -\eta_{nn}\gamma^{(n)}C$ (multiply on the right by C) and then $\gamma^{(n)}C = -\eta_{nn}C\gamma^{(n)}$ (multiply both sides by $-\eta_{nn}$ and use $\eta_{nn}^2 = 1$). Then

$$\bar\Psi' = \Psi^\dagger(\gamma^{(n)})^T C = \eta_{nn}\Psi^\dagger\gamma^{(n)}C = \eta_{nn}\Psi^\dagger(-\eta_{nn}C\gamma^{(n)}) = -\bar\Psi\gamma^{(n)}.$$

The definition of the adjoint with $(M\Psi)^\dagger = \Psi^\dagger M^T$ for a real M; the transpose rule; the relation just derived; $\eta_{nn}^2 = 1$. Now the derivative of the adjoint, with the connection Ω'_μ of Step 1:

$$D'_\mu\bar\Psi' = \partial_\mu\bar\Psi' - \bar\Psi'\Omega'_\mu = -(\partial_\mu\bar\Psi)\gamma^{(n)} + \bar\Psi\gamma^{(n)}\gamma^{(n)}\Omega_\mu(\gamma^{(n)})^{-1}.$$

The definition $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$; then $\bar\Psi' = -\bar\Psi\gamma^{(n)}$ (the constant $\gamma^{(n)}$ comes out of ∂_μ) and $\Omega'_\mu = \gamma^{(n)}\Omega_\mu(\gamma^{(n)})^{-1}$ (Step 1); the two minus signs of the second term multiply to $+1$.

$$\bar\Psi\gamma^{(n)}\gamma^{(n)}\Omega_\mu(\gamma^{(n)})^{-1} = \eta_{nn}\,\bar\Psi\Omega_\mu\,\eta_{nn}\gamma^{(n)} = \bar\Psi\Omega_\mu\gamma^{(n)}.$$

The Clifford relation $\gamma^{(n)}\gamma^{(n)} = \eta_{nn}$, the inverse $(\gamma^{(n)})^{-1} = \eta_{nn}\gamma^{(n)}$ (Lemma 3, proof of 2), and $\eta_{nn}^2 = 1$.

$$D'_\mu\bar\Psi' = -(\partial_\mu\bar\Psi)\gamma^{(n)} + \bar\Psi\Omega_\mu\gamma^{(n)} = -(\partial_\mu\bar\Psi - \bar\Psi\Omega_\mu)\gamma^{(n)} = -(D_\mu\bar\Psi)\gamma^{(n)}.$$

We inserted the line before, took out the common factor $-(\cdots)\gamma^{(n)}$ on the right, and recognised $D_\mu\bar\Psi$.

Step 3: the gammas of the reflected frame. $\gamma'^\mu = \sum_a e'^\mu{}_a\gamma^{(a)} = \sum_a e^\mu{}_a(R_n)_{aa}\gamma^{(a)}$. For $a \neq n$, $(R_n)_{aa}\gamma^{(a)}\gamma^{(n)} = \gamma^{(a)}\gamma^{(n)} = -\gamma^{(n)}\gamma^{(a)}$; for $a = n$, $(R_n)_{nn}\gamma^{(n)}\gamma^{(n)} = -\gamma^{(n)}\gamma^{(n)}$. So in both cases $(R_n)_{aa}\gamma^{(a)}\gamma^{(n)} = -\gamma^{(n)}\gamma^{(a)}$, and summing with the coefficients $e^\mu{}_a$:

$$\gamma'^\mu\gamma^{(n)} = -\gamma^{(n)}\gamma^\mu.$$

Step 4: the bilinears. Insert Steps 1 to 3:

$$\bar\Psi'\gamma'^\mu D'_\mu\Psi' = -\bar\Psi\gamma^{(n)}\,\gamma'^\mu\gamma^{(n)}D_\mu\Psi = -\bar\Psi\gamma^{(n)}(-\gamma^{(n)}\gamma^\mu)D_\mu\Psi = \eta_{nn}\,\bar\Psi\gamma^\mu D_\mu\Psi.$$

The first step inserts $\bar\Psi' = -\bar\Psi\gamma^{(n)}$ (Step 2), $D'_\mu\Psi' = \gamma^{(n)}D_\mu\Psi$ (Step 1); the second is Step 3; the last step is $\gamma^{(n)}\gamma^{(n)} = \eta_{nn}$ and $(-1)(-1) = 1$. The second half of K, line by line:

$$(D'_\mu\bar\Psi')\gamma'^\mu\Psi' = -(D_\mu\bar\Psi)\gamma^{(n)}\gamma'^\mu\gamma^{(n)}\Psi.$$

We inserted $D'_\mu\bar\Psi' = -(D_\mu\bar\Psi)\gamma^{(n)}$ (Step 2) and $\Psi' = \gamma^{(n)}\Psi$.

$$\gamma^{(n)}\gamma'^\mu\gamma^{(n)} = \gamma^{(n)}\big(-\gamma^{(n)}\gamma^\mu\big) = -\eta_{nn}\gamma^\mu.$$

Step 3 replaced $\gamma'^\mu\gamma^{(n)}$ by $-\gamma^{(n)}\gamma^\mu$; then $\gamma^{(n)}\gamma^{(n)} = \eta_{nn}$.

$$(D'_\mu\bar\Psi')\gamma'^\mu\Psi' = -(D_\mu\bar\Psi)\big(-\eta_{nn}\gamma^\mu\big)\Psi = \eta_{nn}\,(D_\mu\bar\Psi)\gamma^\mu\Psi.$$

We inserted the line before; the two minus signs multiply to +1. Both halves of K are multiplied by η_{nn}, so $K' = \eta_{nn} K$. And

$$S' = \bar{\Psi}'\Psi' = -\bar{\Psi}\gamma^{(n)}\gamma^{(n)}\Psi = -\eta_{nn} S.$$

Step 5: the Lagrangian. For a space-like n, $\eta_{nn} = +1$: $K' = K$ and $S' = -S$, so

$$\mathcal{L}_{m,\lambda}[\Psi'; e'] = \sqrt{|g|}\Big[K - m(-S) - \frac{\lambda}{2}(-S)^2\Big] = \sqrt{|g|}\Big[K - (-m)S - \frac{\lambda}{2}S^2\Big] = \mathcal{L}_{-m,\lambda}[\Psi; e],$$

which is (T2a); the potential is even in S, so it does not change. For a time-like n, $\eta_{nn} = -1$: $K' = -K$ and $S' = S$, so $\mathcal{L}_{m,\lambda}[\Psi'; e'] = \sqrt{|g|}[-K - mS - \frac{\lambda}{2}S^2] = -\mathcal{L}_{-m,-\lambda}[\Psi; e]$, which is (T2d).

Step 6: the field equations (space-like n). With Step 1, Step 3 and $S' = -S$:

$$E_{m,\lambda}[\Psi'; e'] = \gamma'^{\mu}\gamma^{(n)} D_\mu\Psi - (m - \lambda S)\gamma^{(n)}\Psi = -\gamma^{(n)}\big[\gamma^\mu D_\mu\Psi - (-m + \lambda S)\Psi\big] = -\gamma^{(n)} E_{-m,\lambda}[\Psi; e],$$

which is (T2b), since $\gamma^{(n)}$ is invertible.

Step 7: the energy-momentum tensor and the current. The tensor of Section 18.4 contains the kinetic bilinears with free indices and the lowered gammas $\gamma_\mu = \sum_\rho g_{\mu\rho}\gamma^\rho$. The reflected frame has the same metric (Step 1), so $\gamma'_\mu = \sum_\rho g_{\mu\rho}\gamma'^\rho$, and the line $\gamma^{(n)}\gamma'^\rho\gamma^{(n)} = -\eta_{nn}\gamma^\rho$ of Step 4, multiplied by $g_{\mu\rho}$ and summed over ρ, gives

$$\gamma^{(n)}\gamma'_\mu\gamma^{(n)} = \sum_\rho g_{\mu\rho}\,\gamma^{(n)}\gamma'^\rho\gamma^{(n)} = -\eta_{nn}\sum_\rho g_{\mu\rho}\gamma^\rho = -\eta_{nn}\gamma_\mu.$$

Now the first kind of kinetic bilinear, line by line:

$$\bar{\Psi}'\gamma'_\mu D'_\nu\Psi' = -\bar{\Psi}\gamma^{(n)}\gamma'_\mu\gamma^{(n)} D_\nu\Psi = -\bar{\Psi}\big(-\eta_{nn}\gamma_\mu\big) D_\nu\Psi = \eta_{nn}\,\bar{\Psi}\gamma_\mu D_\nu\Psi.$$

The first step inserts $\bar{\Psi}' = -\bar{\Psi}\gamma^{(n)}$ (Step 2) and $D'_\nu\Psi' = \gamma^{(n)} D_\nu\Psi$ (Step 1); the second is the line before; the two minus signs multiply to +1. The second kind:

$$(D'_\mu\bar{\Psi}')\gamma'_\nu\Psi' = -(D_\mu\bar{\Psi})\gamma^{(n)}\gamma'_\nu\gamma^{(n)}\Psi = -(D_\mu\bar{\Psi})\big(-\eta_{nn}\gamma_\nu\big)\Psi = \eta_{nn}\,(D_\mu\bar{\Psi})\gamma_\nu\Psi.$$

The first step inserts $D'_\mu\bar{\Psi}' = -(D_\mu\bar{\Psi})\gamma^{(n)}$ (Step 2) and $\Psi' = \gamma^{(n)}\Psi$; the second is the line before with ν in place of μ. For a space-like n, $\eta_{nn} = +1$, so all four kinetic bilinears of $T_{\mu\nu}$ (the two lines with μ and ν exchanged give the other two) are unchanged. Insert them, $g'_{\mu\nu} = g_{\mu\nu}$, $\sqrt{|g'|} = \sqrt{|g|}$ and Step 5 into the tensor of Section 18.4, the tensor of the theory (m, λ) evaluated at Ψ' in the frame e':

$$T'_{\mu\nu} = \frac{1}{4}\big(\bar{\Psi}\gamma_\mu D_\nu\Psi + \bar{\Psi}\gamma_\nu D_\mu\Psi - (D_\mu\bar{\Psi})\gamma_\nu\Psi - (D_\nu\bar{\Psi})\gamma_\mu\Psi\big) - g_{\mu\nu}\frac{\mathcal{L}_{-m,\lambda}[\Psi; e]}{\sqrt{|g|}} = T^{(-m,\lambda)}_{\mu\nu}[\Psi].$$

The kinetic bracket contains neither m nor λ; only the Lagrangian of the last term changes, and Step 5 turns it into $\mathcal{L}_{-m,\lambda}[\Psi; e]$. Likewise $\bar{\Psi}'\gamma'^{\mu}\Psi' = -\bar{\Psi}\gamma^{(n)}\gamma'^{\mu}\gamma^{(n)}\Psi = \eta_{nn}\bar{\Psi}\gamma^{\mu}\Psi$ gives $J' = J$. This is (T2c) in the general-field version.

Step 8: both statistics. $\gamma^{(n)}$ is a constant signed permutation matrix: it reorders the components and changes some signs, but in every bilinear each Ψ^* still stands to the left of each Ψ. No two Grassmann components are exchanged, so Steps 1 to 7 hold for both statistics. QED.

18.16 T2 in the author's field: the mirror across the brane

The mirror is an isometry. Under $z \to \pi - z$: $\sin(\pi - z) = \sin z$, so the factors $\sin^{1/3} z$ of the 3-space and extra-time components are unchanged; $\cot(\pi - z) = -\cot z$, so $g_{00} = \cot^2 z$ is unchanged; the factors $e^{\pm 2a_4}$ (and the second function $b_4(x_4)$ of the metric with b_4) do not depend on z. So every $g_{\mu\mu}$ takes the same value at the mirror point (`python-pairing.json`, check `geometry.mirror_isometry`; `wolfram-pairing.json`, check `T2_mirror_is_isometry`; Notebook 18b, In [15], and its Figure 18b.4).

The pulled-back frame is the reflected frame. On the mirror patch the positive hidden vielbein factor is $f_0 = -\cot z$ (positive there, because $\cot z < 0$ for $\pi/2 < z < \pi$). Pull the hidden leg $e^0 = f_0\, dx_0$ back by ϕ: $x_0 \to \pi/(6H) - x_0$ replaces dx_0 by $-dx_0$ and z by $\pi - z$:

$$-\cot(\pi - z)\, d\big(\tfrac{\pi}{6H} - x_0\big) = \cot z\,(-dx_0) = -e^0.$$

The other seven legs are unchanged. So the pulled-back vielbein is $R_0 e$, the frame reflected in the direction x_0, and $\sqrt{|g|}\, d^8x$ is unchanged ($|\cos z|$ is unchanged and the Jacobian of ϕ has modulus 1). Pulling the Lagrangian of the mirror patch back by ϕ therefore gives exactly the frame-reflected Lagrangian of Section 18.15 with $n = x_0$, which proves (T2a) and (T2b) in the author's field. The coordinate components of T and J pick up the Jacobian R_0 of ϕ, one factor -1 for each index x_0, which gives (T2c) with $\Lambda = R_0$. QED.

The spin-connection term on the mirror patch. With $f_0 = -\cot z$ the twelve components of Section 18.3 keep their form, except that the six components with an index x_0 change sign (they contain $1/f_0$). In the sum of Section 18.3 the a_4' terms still cancel and the hidden terms add with the opposite sign: $\gamma^{\mu}\Omega_{\mu} = -3H\gamma^{(x_0)}$ on the mirror patch (Notebook 18c, In [3]). In block form, $\gamma^{(x_0)}(\psi_-, \psi_+) = (\psi_+, \psi_-)$: the mirror image exchanges the two inequivalent Spin(4,4) halves, and $S = -\psi_-^{\dagger}\sigma\psi_- + \psi_+^{\dagger}\sigma\psi_+$ changes sign.

Which matrix makes the mirror a pairing? The sympy record searched four candidates P for the map $\Psi'(\phi(x)) = P\Psi(x)$ and all eight sign choices of a relation $\mathcal{L}^{\text{mirror}}_{m,\lambda}[\Psi'] = \sigma\mathcal{L}^{\text{patch}}_{\pm m,\pm\lambda}[\Psi]$. The result (`python-pairing.json`, checks `T2.metric.commuting.reflection_table` and `T2.metric.grassmann.reflection_table`; Notebook 18b, In [17], Figure 18b.5):

matrix P	**relation found**
1 (the plain mirror)	none
Γ	none
$\gamma^{(x_0)} = \Gamma P_0$	$+\mathcal{L}_{-m,\lambda}$: theorem T2
$\Gamma\gamma^{(x_0)} = P_0$ (the reflection alone)	$-\mathcal{L}_{m,-\lambda}$

The plain mirror fails because the pull-back reverses the hidden leg of the frame: the part of the kinetic term along x_0 (and the connection terms with one index x_0) changes sign while the rest does not, so the pulled-back Lagrangian is neither + nor − a Lagrangian of the patch. The matrix $\gamma^{(x_0)}$ repairs exactly this sign (Steps 3 and 4 of Section 18.15 with $n = x_0$).

The eight reflections. For completeness, the frame reflections of all eight directions (the data table `reflections` of `pairing-theory.json`, recomputed exactly by Notebook 18a, In [12] and In [13], Figure 18a.4):

direction	η_{nn}	**character of** P_n	S **under** $\gamma^{(n)}$	**kinetic term**	**map**
x_0	$+1$	-1	$-S$	$+K$	$(-m, \lambda)$, $+\mathcal{L}$ (T2)
x_1	$+1$	-1	$-S$	$+K$	$(-m, \lambda)$, $+\mathcal{L}$ (T2)
x_2	$+1$	-1	$-S$	$+K$	$(-m, \lambda)$, $+\mathcal{L}$ (T2)
x_3	$+1$	-1	$-S$	$+K$	$(-m, \lambda)$, $+\mathcal{L}$ (T2)
x_4	-1	$+1$	$+S$	$-K$	$(-m, -\lambda)$, $-\mathcal{L}$ (T1 type)
x_5	-1	$+1$	$+S$	$-K$	$(-m, -\lambda)$, $-\mathcal{L}$ (T1 type)
x_6	-1	$+1$	$+S$	$-K$	$(-m, -\lambda)$, $-\mathcal{L}$ (T1 type)
x_7	-1	$+1$	$+S$	$-K$	$(-m, -\lambda)$, $-\mathcal{L}$ (T1 type)

In the author's field only the reflection of x_0 is realised by an isometry that maps the patch onto another patch, the mirror across the Z2 brane; the reflections of x_1, x_2, x_3 are statements about frames.

With a changing b_4. The general-field version of T2 (Section 18.15) holds for every vielbein, so also for the author's metric with any $b_4(x_4)$, and so does T1. The mirror $z \to \pi - z$ does not touch x_4: $b_4(x_4)$ is unchanged by it, and the mirror is still an isometry. The records check the author's-field version of T2 at $b_4 = 1$ only; this book has no recorded check of it with a changing b_4.

What T2 says and does not say. T2 pairs a solution with mass $-m$ on the patch with a solution with mass $+m$ on the mirror patch, at the SAME coupling and with EQUAL energy density and charge density: nothing cancels in a T2 pair. The gluing of the two patches at the degenerate brane is an assumption; no junction condition (an equation that says how the fields and the geometry must match across the brane), brane tension (an energy per unit area carried by the brane itself) or matching of the field across the brane is derived (`pairing-theory.json`, the list `not_esta blished`).

18.17 Example: Notebook 18b proves T1 and T2 in the author's metric

Notebook 18b repeats the proofs of Sections 18.11 and 18.15 as exact computer algebra, without taking any step on trust. It builds the author's metric on the background $b_4 = 1$ with an UNKNOWN

history $a_4(x_4)$ (so every result holds for every deflating history), computes its Christoffel symbols and canonical spin connection with sympy, checks the vielbein postulate and the twelve components of Section 18.3, and recomputes $\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ direction by direction. Then it writes the Lagrangian, the field operator, all 36 components of $T_{\mu\nu}$ and the 8 components of J^μ as polynomials in the 288 values of the first jet, in an algebra that can treat the jet values as commuting numbers (dirac16complex00) or as Grassmann numbers (dirac16complex), and proves T1 and T2 as identities in which every coefficient is zero. It reproduces 35 checks of `python-pairing.json` and the corresponding Wolfram checks, including the numbers of monomials of the Lagrangian recorded by the Wolfram verifier, 408 for commuting and 392 for Grassmann components, and draws five figures. Its last line is ALL 20 CHECKS PASSED (notebook 18b); it runs in about 2 minutes.

18.18 How to run Notebook 18b

Step 1. What this notebook does and what it needs.

Notebook 18b (T1 and T2 proved symbolically in the author's deflating metric) is the file `Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb` of the repository Dirac_claude. With sympy it builds the author's metric with an arbitrary history a4(x4) of the three exponentially deflating extra times, its diagonal vielbein and its canonical spin connection, and checks the connection (antisymmetry, the vielbein postulate, its nonzero components, gamma^mu Omega_mu = 3 H gamma^(x0) with the deflation terms cancelling). It then writes the Lagrangian, the field equation, the energy-momentum tensor and the current of the two fields on the first jet, with commuting components (dirac16complex00) and with Grassmann components (dirac16complex), and proves as exact polynomial identities the pairing theorem T1 (L_m,lambda of Gamma Psi = minus L_-m,-lambda of Psi, the field equations, T and J reversed, the pair sums zero, a general cubic potential, a random general field) with its negative controls, and the pairing theorem T2 across the Z2 mirror z to pi minus z (L to plus L_-m,lambda, S reversed, T and J pulled back, not reversed). It reproduces 35 checks of the sympy pairing report and the monomial counts and checks of the Wolfram pairing report, and draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 18b_t1_t2_symbolic.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 18b_t1_t2_symbolic.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 18b_t1_t2_symbolic.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/18b.captions.json`
- `Revision/textbook/figures/18b_1_connection_history.png`
- `Revision/textbook/figures/18b_2_lagrangian_cancellation.png`
- `Revision/textbook/figures/18b_3_emt_signs.png`
- `Revision/textbook/figures/18b_4_mirror_metric.png`
- `Revision/textbook/figures/18b_5_mirror_candidates.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 18b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 18b_t1_t2_symbolic.ipynb
python -m nbconvert --execute --inplace 18b_t1_t2_symbolic.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for gammas.json or one of the pairing reports: the notebook reads four files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository. Clone the repository again and open the notebook there.
- a cell runs for several minutes: the symbolic checks take about a minute and a half in all on the computer on which the book was built; on a slow computer they may take five minutes. Wait for the PASS lines; the star in the margin of a running cell turns into a number when it ends.

18.19 Notebook 18b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The pairing theorems T1 and T2 are statements about the Lagrangian, the field
equations, the energy-momentum tensor and the current of the two fields
dirac16complex (anticommuting, Grassmann components) and dirac16complex00
(commuting components). Notebook 18a proved the matrix facts behind them. This
notebook proves the theorems themselves, in the author's primordial
gravitational field, as exact identities computed with sympy:

1. It builds the author's metric with an ARBITRARY history $a_4(x_4)$ (the three
   extra times $x_5, x_6, x_7$ deflate exponentially as $a_4$ grows), its
   diagonal vielbein and its canonical spin connection, and checks the connection.
2. It writes every quantity of the theory as a polynomial in the values of the
   field and of its first derivatives at a point (the *first jet*), with
   commuting components and with Grassmann components.
3. **T1**: it proves $\mathcal{L}_{m,\lambda}[\Gamma\Psi] =
```

```
   -\mathcal{L}_{-m,-\lambda}[\Psi]$, the map of the field equations, $T_{\mu\nu}
   \to -T_{\mu\nu}$ and $J^\mu \to -J^\mu$, so that the pair has zero total
   energy-momentum and current; and it shows with *negative controls* that the
   coupling $\lambda$ must change sign too.
4. **T2**: it proves that the reflection of the hidden direction across the
   brane $z = \pi/2$, with $\Psi \to \gamma^{(x_0)}\Psi$, maps a solution with
   $(-m, \lambda)$ to a solution with $(m, \lambda)$, reverses $S$ and keeps the
   energy-momentum tensor and the current (pulled back, not reversed).

Every result is compared with the checks of the two Revision pairing reports
(the sympy report and the Wolfram report), and five figures show what cancels and
what does not.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 18b (T1 and T2 proved symbolically in the author's deflating metric) is the file
`Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb` of the repository Dirac_claude.
With sympy it builds the author's metric with an arbitrary history a4(x4) of the three
exponentially deflating extra times, its diagonal vielbein and its canonical spin
connection, and checks the connection (antisymmetry, the vielbein postulate, its nonzero
components, gamma^mu Omega_mu = 3 H gamma^(x0) with the deflation terms cancelling). It
then writes the Lagrangian, the field equation, the energy-momentum tensor and the
current of the two fields on the first jet, with commuting components (dirac16complex00)
and with Grassmann components (dirac16complex), and proves as exact polynomial identities
the pairing theorem T1 (L_m,lambda of Gamma Psi = minus L_-m,-lambda of Psi, the field
equations, T and J reversed, the pair sums zero, a general cubic potential, a random
general field) with its negative controls, and the pairing theorem T2 across the Z2
mirror z to pi minus z (L to plus L_-m,lambda, S reversed, T and J pulled back, not
reversed). It reproduces 35 checks of the sympy pairing report and the monomial counts
and checks of the Wolfram pairing report, and draws five teaching figures. It needs a
computer with Windows 11, macOS or Linux, an internet connection for the installation,
the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5)
with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0,
matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0
and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".
```

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
```

```
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 18b_t1_t2_symbolic.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 18b_t1_t2_symbolic.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 18b_t1_t2_symbolic.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/18b.captions.json‘
- ‘Revision/textbook/figures/18b_1_connection_history.png‘
- ‘Revision/textbook/figures/18b_2_lagrangian_cancellation.png‘
- ‘Revision/textbook/figures/18b_3_emt_signs.png‘
- ‘Revision/textbook/figures/18b_4_mirror_metric.png‘
- ‘Revision/textbook/figures/18b_5_mirror_candidates.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS every figure file of this notebook exists
    ALL 20 CHECKS PASSED (notebook 18b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 18b_t1_t2_symbolic.ipynb
      python -m nbconvert --execute --inplace 18b_t1_t2_symbolic.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for gammas.json or one of the pairing reports: the notebook reads four files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository. Clone the repository

  again and open the notebook there.
- a cell runs for several minutes: the symbolic checks take about a minute and a half in all on the computer on which the book was built; on a slow computer they may take five minutes. Wait for the PASS lines; the star in the margin of a running cell turns into a number when it ends.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 18b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 18b (T1 and T2 proved symbolically in the author's deflating metric) is the
# file Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb of the repository
# Dirac_claude. With sympy it builds the author's metric with an arbitrary history a4(x4)
# of the three exponentially deflating extra times, its diagonal vielbein and its
# canonical spin connection, and checks the connection (antisymmetry, the vielbein
# postulate, its nonzero components, gamma^mu Omega_mu = 3 H gamma^(x0) with the
# deflation terms cancelling). It then writes the Lagrangian, the field equation, the
# energy-momentum tensor and the current of the two fields on the first jet, with
# commuting components (dirac16complex00) and with Grassmann components (dirac16complex),
# and proves as exact polynomial identities the pairing theorem T1 (L_m,lambda of Gamma
# Psi = minus L_-m,-lambda of Psi, the field equations, T and J reversed, the pair sums
# zero, a general cubic potential, a random general field) with its negative controls,
# and the pairing theorem T2 across the Z2 mirror z to pi minus z (L to plus L_-m,lambda,
# S reversed, T and J pulled back, not reversed). It reproduces 35 checks of the sympy
# pairing report and the monomial counts and checks of the Wolfram pairing report, and
# draws five teaching figures. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
```

```
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
```

```
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 18b_t1_t2_symbolic.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
```

```
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 18b_t1_t2_symbolic.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 18b_t1_t2_symbolic.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/18b.captions.json
# - Revision/textbook/figures/18b_1_connection_history.png
# - Revision/textbook/figures/18b_2_lagrangian_cancellation.png
# - Revision/textbook/figures/18b_3_emt_signs.png
# - Revision/textbook/figures/18b_4_mirror_metric.png
# - Revision/textbook/figures/18b_5_mirror_candidates.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 20 CHECKS PASSED (notebook 18b)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
```

```
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/18b_t1_t2_symbolic.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 18b_t1_t2_symbolic.ipynb
#     python -m nbconvert --execute --inplace 18b_t1_t2_symbolic.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for gammas.json or one of the pairing reports: the notebook reads
#   four files of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository. Clone the
#   repository again and open the notebook there.
# - a cell runs for several minutes: the symbolic checks take about a minute and a half
#   in all on the computer on which the book was built; on a slow computer they may take
#   five minutes. Wait for the PASS lines; the star in the margin of a running cell turns
#   into a number when it ends.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "18b"  # this notebook: chapter 18, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
```

```
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 18b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Metric, vielbein**: the metric $g_{\mu\nu}$ measures lengths and times; a
  *vielbein* $e^a{}_\mu$ writes it as $g_{\mu\nu} = \sum_a\eta_{aa}e^a{}_\mu
  e^a{}_\nu$ with $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$. For the
  author's diagonal metric $e^a{}_\mu = f_\mu\delta^a_\mu$ with
  $f_\mu = \sqrt{|g_{\mu\mu}|}$.
- **Christoffel symbols** $\Gamma^\nu{}_{\mu\lambda}$: the numbers (functions)
  built from the first derivatives of the metric that say how the coordinate
  directions turn from point to point.
- **Canonical spin connection** $\omega_{\mu ab}$: the connection that makes the
  gammas $\gamma^\mu = e^\mu{}_a\gamma^{(a)}$ covariantly constant (the *vielbein
  postulate*); it is antisymmetric in $a, b$. $\Omega_\mu =
  \frac12\sum_{a,b}\omega_{\mu ab}S^{ab}$ enters the covariant derivative
  $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$.
- **Jet**: at one point, the 16 components $\Psi_A$, their complex conjugates
  $\Psi_A^*$, and the $8 \times 16$ first derivatives $\partial_\mu\Psi_A$ and
  $\partial_\mu\Psi_A^*$: 288 numbers. Every quantity of this notebook at that
  point is a polynomial in these 288 numbers, with coefficients that depend on the
  point.
- **Generator, monomial**: the notebook treats each of the 288 jet values as a
  symbol, a *generator*; a *monomial* is a product of generators, and a
  polynomial is a sum of monomials times coefficients.
- **Grassmann generators**: symbols that anticommute, $\theta_1\theta_2 =
  -\theta_2\theta_1$, so $\theta_1\theta_1 = 0$. They model the anticommuting
  components of dirac16complex.
- **Identity in the jets**: a polynomial whose every coefficient is zero. It is
  zero for EVERY value of the jet, that is, for every field at every point,
  whether the field solves the field equations or not (*off shell*). An identity
  off shell holds in particular for solutions (*on shell*).
- **Euler-Lagrange expression**: $\partial\mathcal{L}/\partial\Psi^*_A -
  \sum_\mu\partial_\mu\big(\partial\mathcal{L}/\partial(\partial_\mu\Psi^*_A)\big)$;
  the field equations say it is zero.
- **Negative control**: a computation that must FAIL; it shows that the test can
  detect a wrong statement.
- **Isometry**: a change of coordinates that does not change the metric.
- **Patch, mirror patch, brane**: the author's coordinate $z = 6Hx_0$ runs over
  the *patch* $0 < z < \pi/2$. The map $z \to \pi - z$ takes it to the *mirror
  patch* $\pi/2 < z < \pi$; the surface $z = \pi/2$ between them is the *brane*,
  where $g_{00} = \cot^2 z = 0$. Gluing the mirror patch to the patch there is
  the Z2 construction, an ASSUMPTION of the Revision record.
```

Text cell:

```
## 4. The physical and mathematical situation

**The metric.** In the author's coordinates ($x_1, x_2, x_3$ 3-space, $x_4$ the
time, $x_5, x_6, x_7$ the extra times, $x_0$ hidden, $z = 6Hx_0$)

$$ds^2 = e^{2a_4}\sin^{1/3}z\,(dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2
- e^{-2a_4}\sin^{1/3}z\,(dx_5^2 + dx_6^2 + dx_7^2) + \cot^2 z\,dx_0^2 ,$$

with $a_4 = a_4(x_4)$. As $a_4$ grows, 3-space inflates ($e^{a_4}$) and the three
extra times deflate exponentially ($e^{-a_4}$). The vielbein factors are
$f_1 = f_2 = f_3 = e^{a_4}\sin^{1/6}z$, $f_4 = 1$, $f_5 = f_6 = f_7 =
e^{-a_4}\sin^{1/6}z$, $f_0 = \cot z$, and $\sqrt{|g|} = f_0f_2\cdots f_7 =
\cos z$. The notebook keeps $a_4$ as an unknown function of $x_4$, so every result
holds for every history, deflating or not.

**The theory.** With $\bar\Psi = \Psi^\dagger C$, $S = \bar\Psi\Psi$ and
$D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$,

$$\mathcal{L}_{m,\lambda} = \sqrt{|g|}\Big[\tfrac12\big(\bar\Psi\gamma^\mu
D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi\big) - mS - \tfrac{\lambda}{2}S^2\Big],
\qquad E_{m,\lambda} = \gamma^\mu D_\mu\Psi - (m + \lambda S)\Psi ,$$

$$T_{\mu\nu} = \tfrac14\big(\bar\Psi\gamma_\mu D_\nu\Psi + \bar\Psi\gamma_\nu
D_\mu\Psi - (D_\mu\bar\Psi)\gamma_\nu\Psi - (D_\nu\bar\Psi)\gamma_\mu\Psi\big)
- g_{\mu\nu}\mathcal{L}/\sqrt{|g|},\qquad J^\mu = i\bar\Psi\gamma^\mu\Psi .$$

The field equation is $E_{m,\lambda} = 0$. $T_{\mu\nu}$ and $J^\mu$ are written
exactly as in the sympy pairing report of the Revision record; another overall
sign or factor changes nothing below, because every statement is of the form
$T' = \pm T$, $J' = \pm J$. (In this convention the energy density of the theory
record is $\rho = -T_{x_4x_4}$.)

**Why a polynomial identity in the jet is a proof.** Every quantity above, at a
point, is a polynomial in the 288 jet values whose coefficients are functions of
$x_4$ and $z$ (through $a_4$, $a_4' = da_4/dx_4$, $\sin z$, $\cot z$, $H$). If
the polynomial $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,-\lambda}
[\Psi]$ has only zero coefficients, it vanishes for every field, at every point,
for every history $a_4$: that IS the theorem. For dirac16complex the jet values
are Grassmann generators; the same computation in an algebra of anticommuting
symbols proves the theorem for that field.
```

Text cell:

```
## 5. The gamma matrices as exact sympy matrices

The next cell imports sympy and numpy, defines the helpers that read the Revision
records (`reproduces` passes only when the notebook's own result holds AND every
named check of the named report has the verdict PASS), reads the author's gamma
matrices and builds $C$, $\Gamma$ and $S^{ab} = \frac14[\gamma^{(a)},
\gamma^{(b)}]$ as exact sympy matrices (whole numbers and halves, no rounding).
```

In [2]:

```
import itertools  # all combinations of signs
import random  # Python's random numbers with a fixed seed (the random field)

import numpy as np  # numbers for the figures
import sympy as sp  # exact symbolic algebra

GAMMAS = "Revision/algebra/gammas.json"
PY = "Revision/pairing/reports/python-pairing.json"  # the sympy verifier's report
WL = "Revision/pairing/reports/wolfram-pairing.json"  # the Wolfram verifier's report
PARAMETERS = "Revision/kohn_sham/results/parameters.json"


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


RECORDS = {PY: read_json(PY), WL: read_json(WL)}


def recorded(report_file, name):
    """The recorded entry (verdict and detail) of the check name of a report."""
    for entry in RECORDS[report_file]["checks"]:
        if entry["name"] == name:
            return entry
    return {"verdict": "MISSING", "detail": ""}


def reproduces(condition, name, *sources):
    """check(condition, name), which also requires every named check of every
    source (report_file, [check names]) to have the recorded verdict PASS."""
    ok = all(recorded(f, n)["verdict"].upper() == "PASS"
             for f, names in sources for n in names)
    text = "; ".join(f"{f}, check {', '.join(names)}" for f, names in sources)
    check(condition and ok, name, record=text)


fixture = read_json(GAMMAS)
G = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # gamma^(x_a), a = 0..7
ETA = {a: int(fixture["eta"][a]) for a in range(8)}
I16 = sp.eye(16)
C = G[0] * G[1] * G[2] * G[3]  # the four space-like gammas
GAM = G[0] * G[1] * G[2] * G[3] * G[4] * G[5] * G[6] * G[7]  # the chirality Gamma
SAB = {(a, b): (G[a] * G[b] - G[b] * G[a]) / 4  # the generators S^ab
       for a in range(8) for b in range(8)}
check(all(G[a] * G[b] + G[b] * G[a] == (2 * ETA[a] if a == b else 0) * I16
          for a in range(8) for b in range(8))
      and GAM == sp.diag(*([-1] * 8 + [1] * 8)),
      "eight real 16 x 16 gammas with the Clifford relations; Gamma = diag(-I8, I8)")
```

Out [2]:

```
PASS eight real 16 x 16 gammas with the Clifford relations; Gamma = diag(-I8, I8)
```

Text cell:

```
## 6. The author's metric and its canonical spin connection

The next cell defines the symbols: the time $x_4$, the hidden angle $z$, the
author's constant $H$, the mass $m$, the coupling $\lambda$ and the unknown
function $a_4(x_4)$. A derivative along $x_0$ is a derivative along $z$ times
$6H$ (because $z = 6Hx_0$); nothing depends on $x_1, x_2, x_3, x_5, x_6, x_7$.

The class `Geometry` then computes, for the diagonal vielbein with the factors
$f_\mu$ of section 4 (with $f_0 = s_0\cot z$, where the sign $s_0 = +1$ on the
patch and $s_0 = -1$ on the mirror patch keeps $f_0$ positive):

- the metric $g_{\mu\mu} = \eta_{\mu\mu}f_\mu^2$ and $\sqrt{|g|} = f_0\cdots f_7$;
- the Christoffel symbols of a diagonal metric,
  $\Gamma^\nu{}_{\mu\lambda} = \frac{1}{2g_{\nu\nu}}\big(\delta_{\nu\lambda}
  \partial_\mu g_{\nu\nu} + \delta_{\nu\mu}\partial_\lambda g_{\nu\nu} -
  \delta_{\mu\lambda}\partial_\nu g_{\mu\mu}\big)$;
- the canonical spin connection $\omega_\mu{}^a{}_b = e^a{}_\nu\big(\partial_\mu
  e^\nu{}_b + \Gamma^\nu{}_{\mu\lambda}e^\lambda{}_b\big)$ with the inverse
  vielbein $e^\nu{}_b = \delta^\nu_b/f_b$, lowered to $\omega_{\mu ab} =
  \eta_{aa}\,\omega_\mu{}^a{}_b$;
- $\Omega_\mu = \frac12\sum_{a,b}\omega_{\mu ab}S^{ab}$, the curved gammas
  $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ and $\gamma_\mu = g_{\mu\mu}\gamma^\mu$.

Every expression is simplified exactly by sympy.
```

In [3]:

```
x4, z, H = sp.symbols("x4 z H", real=True)
m, lam = sp.symbols("m lambda", real=True)
a4 = sp.Function("a4")(x4)  # the history: any function of the time x4


def d(expr, mu):
    """The derivative of a coefficient along the coordinate x_mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # z = 6 H x0
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7


class Geometry:
    """The author's metric with the diagonal vielbein; s0 = +1 on the patch,
    s0 = -1 on the mirror patch (then f0 = -cot z > 0)."""

    def __init__(self, s0):
        up, down = sp.exp(a4), sp.exp(-a4)  # inflating and deflating factors
        s6 = sp.sin(z) ** sp.Rational(1, 6)
```

```
        f = {0: s0 * sp.cot(z), 1: up * s6, 2: up * s6, 3: up * s6,
             4: sp.Integer(1), 5: down * s6, 6: down * s6, 7: down * s6}
        self.f = f
        self.g = {mu: ETA[mu] * f[mu] ** 2 for mu in f}  # g_mu mu
        self.sqrtg = sp.simplify(sp.Mul(*f.values()))  # sqrt|g|
        g = self.g
        self.chr = {}  # Christoffel symbols Gamma^nu_(mu lambda)
        for nu, mu, la in itertools.product(range(8), repeat=3):
            v = 0
            if nu == la:
                v += d(g[nu], mu)
            if nu == mu:
                v += d(g[nu], la)
            if mu == la:
                v -= d(g[mu], nu)
            self.chr[nu, mu, la] = sp.simplify(v / (2 * g[nu]))
        self.om = {}  # omega_mu ab
        for mu, a, b in itertools.product(range(8), repeat=3):
            v = self.chr[a, mu, b] / f[b]  # e^a_nu Gamma^nu_mu lam e^lam_b ...
            if a == b:
                v += d(1 / f[b], mu)  # ... + e^a_nu d_mu e^nu_b
            self.om[mu, a, b] = sp.simplify(ETA[a] * f[a] * v)
        self.Om = {}  # Omega_mu = (1/2) omega_mu ab S^ab
        for mu in range(8):
            M = sp.zeros(16)
            for a, b in itertools.product(range(8), repeat=2):
                if self.om[mu, a, b] != 0:
                    M += self.om[mu, a, b] * SAB[a, b] / 2
            self.Om[mu] = M.applyfunc(sp.simplify)
        self.gup = {mu: G[mu] / f[mu] for mu in f}  # gamma^mu
        self.glow = {mu: ETA[mu] * f[mu] * G[mu] for mu in f}  # gamma_mu


patch = Geometry(1)  # the author's patch 0 < z < pi/2
author = {0: sp.cot(z) ** 2, 1: sp.exp(2 * a4) * sp.sin(z) ** sp.Rational(1, 3),
          4: -1, 5: -sp.exp(-2 * a4) * sp.sin(z) ** sp.Rational(1, 3)}
author.update({2: author[1], 3: author[1], 6: author[5], 7: author[5]})
metric_ok = all(sp.simplify(patch.g[mu] - author[mu]) == 0 for mu in range(8))
say(f"sqrt|g| on the patch = {patch.sqrtg}")
reproduces(metric_ok and sp.simplify(patch.sqrtg - sp.cos(z)) == 0,
           "the vielbein gives the author's metric and sqrt|g| = cos z",
           (WL, ["primordial_vielbein"]))
```

Out [3]:

```
sqrt|g| on the patch = cos(z)
PASS the vielbein gives the author's metric and sqrt|g| = cos z
     reproduces Revision/pairing/reports/wolfram-pairing.json, check primordial_vielbein
```

Text cell:

```
The next cell checks the connection in three ways. (1) $\omega_{\mu ab} =
-\omega_{\mu ba}$ for all $\mu, a, b$. (2) The vielbein postulate: for all
```

```
$\mu, \nu$, $\partial_\mu\gamma^\nu + \sum_\lambda\Gamma^\nu{}_{\mu\lambda}
\gamma^\lambda + \Omega_\mu\gamma^\nu - \gamma^\nu\Omega_\mu = 0$ (64 matrix
equations). (3) It lists the nonzero components $\omega_{\mu ab}$ with $a < b$,
writing $a_4'$ as ‘a4p‘, in the format of the Revision report, and compares the
list with the recorded one word for word.
```

In [4]:

```
COORDS = {a: f"x{a}" for a in range(8)}
antisymmetric = all(sp.simplify(patch.om[mu, a, b] + patch.om[mu, b, a]) == 0
                    for mu, a, b in itertools.product(range(8), repeat=3))
postulate = True
for mu, nu in itertools.product(range(8), repeat=2):
    M = patch.gup[nu].applyfunc(lambda e: d(e, mu))  # d_mu gamma^nu
    for la in range(8):
        if patch.chr[nu, mu, la] != 0:
            M = M + patch.chr[nu, mu, la] * patch.gup[la]
    M = M + patch.Om[mu] * patch.gup[nu] - patch.gup[nu] * patch.Om[mu]
    postulate = postulate and all(sp.simplify(e) == 0 for e in M)
a4p = sp.Symbol("a4p")  # a short name for the derivative a4'


def short(expr):
    """Write a4' as a4p and a4(x4) as a4, as in the Revision report."""
    expr = sp.simplify(expr).subs(sp.Derivative(a4, x4), a4p)
    return expr.subs(a4, sp.Symbol("a4"))


nonzero = [(mu, a, b) for mu in range(8) for a in range(8)
           for b in range(a + 1, 8) if sp.simplify(patch.om[mu, a, b]) != 0]
listing = "; ".join(f"omega_{COORDS[mu]} {COORDS[a]}{COORDS[b]} = "
                    f"{short(patch.om[mu, a, b])}" for mu, a, b in nonzero)
for mu, a, b in nonzero:
    say(f"omega_{COORDS[mu]} {COORDS[a]}{COORDS[b]} = {short(patch.om[mu, a, b])}")
same_list = recorded(PY, "geometry.spin_connection_components")["detail"] == (
    "nonzero components (a < b): " + listing)
reproduces(antisymmetric and postulate, "the canonical spin connection is "
           "antisymmetric and obeys the vielbein postulate",
           (PY, ["geometry.omega_antisymmetric",
                 "geometry.gamma_covariantly_constant"]),
           (WL, ["connection_primordial"]))
reproduces(len(nonzero) == 12 and same_list,
           "the 12 nonzero connection components equal the recorded list",
           (PY, ["geometry.spin_connection_components"]))
```

Out [4]:

```
omega_x1 x0x1 = -H*exp(a4)*sin(z)**(1/6)
omega_x1 x1x4 = a4p*exp(a4)*sin(z)**(1/6)
omega_x2 x0x2 = -H*exp(a4)*sin(z)**(1/6)
omega_x2 x2x4 = a4p*exp(a4)*sin(z)**(1/6)
omega_x3 x0x3 = -H*exp(a4)*sin(z)**(1/6)
```

```
omega_x3 x3x4 = a4p*exp(a4)*sin(z)**(1/6)
omega_x5 x0x5 = H*exp(-a4)*sin(z)**(1/6)
omega_x5 x4x5 = -a4p*exp(-a4)*sin(z)**(1/6)
omega_x6 x0x6 = H*exp(-a4)*sin(z)**(1/6)
omega_x6 x4x6 = -a4p*exp(-a4)*sin(z)**(1/6)
omega_x7 x0x7 = H*exp(-a4)*sin(z)**(1/6)
omega_x7 x4x7 = -a4p*exp(-a4)*sin(z)**(1/6)
PASS the canonical spin connection is antisymmetric and obeys the vielbein postulate
     reproduces Revision/pairing/reports/python-pairing.json, check
    geometry.omega_antisymmetric, geometry.gamma_covariantly_constant;
    Revision/pairing/reports/wolfram-pairing.json, check connection_primordial
PASS the 12 nonzero connection components equal the recorded list
     reproduces Revision/pairing/reports/python-pairing.json, check
    geometry.spin_connection_components
```

Text cell:

```
The next cell computes $\gamma^\mu\Omega_\mu = \sum_\mu\gamma^\mu\Omega_\mu$,
direction by direction. Each inflating direction $x_1, x_2, x_3$ contributes
$\frac{a_4'}{2}\gamma^{(x_4)} + \frac{H}{2}\gamma^{(x_0)}$ and each deflating extra
time $x_5, x_6, x_7$ contributes $-\frac{a_4'}{2}\gamma^{(x_4)} +
\frac{H}{2}\gamma^{(x_0)}$: the six $a_4'$ terms cancel, the six $H$ terms add up
to $3H\gamma^{(x_0)}$. The cell extracts the two coefficients of each contribution
with traces ($\mathrm{tr}(\gamma^{(a)}\gamma^{(b)}) = 16\eta_{aa}\delta_{ab}$, so
the coefficient of $\gamma^{(a)}$ in $X$ is $\mathrm{tr}(\gamma^{(a)}X)/
(16\eta_{aa})$) and checks the sum.
```

In [5]:

```
def coefficient(X, a):
    """The coefficient of gamma^(a) in the matrix X (exact, by a trace)."""
    return sp.simplify((G[a] * X).trace() / (16 * ETA[a]))


contributions = {mu: (patch.gup[mu] * patch.Om[mu]).applyfunc(sp.simplify)
                 for mu in range(8)}
for mu in range(8):
    c4, c0 = coefficient(contributions[mu], 4), coefficient(contributions[mu], 0)
    rest = (contributions[mu] - c4 * G[4] - c0 * G[0]).applyfunc(sp.simplify)
    assert rest == sp.zeros(16)  # nothing but gamma^(x4) and gamma^(x0)
    say(f"direction {COORDS[mu]}: coefficient of gamma^(x4) = {short(c4)}, "
        f"of gamma^(x0) = {short(c0)}")
total = sp.zeros(16)
for mu in range(8):
    total += contributions[mu]
check(total.applyfunc(sp.simplify) == 3 * H * G[0],
      "gamma^mu Omega_mu = 3 H gamma^(x0): the a4' terms of the deflation cancel")
```

Out [5]:

```
direction x0: coefficient of gamma^(x4) = 0, of gamma^(x0) = 0
direction x1: coefficient of gamma^(x4) = a4p/2, of gamma^(x0) = H/2
```

```
direction x2: coefficient of gamma^(x4) = a4p/2, of gamma^(x0) = H/2
direction x3: coefficient of gamma^(x4) = a4p/2, of gamma^(x0) = H/2
direction x4: coefficient of gamma^(x4) = 0, of gamma^(x0) = 0
direction x5: coefficient of gamma^(x4) = -a4p/2, of gamma^(x0) = H/2
direction x6: coefficient of gamma^(x4) = -a4p/2, of gamma^(x0) = H/2
direction x7: coefficient of gamma^(x4) = -a4p/2, of gamma^(x0) = H/2
PASS gamma^mu Omega_mu = 3 H gamma^(x0): the a4' terms of the deflation cancel
```

Text cell:

```
The next cell draws the connection along the canonical deflating history of the
Revision Kohn-Sham record, $a_4 = AHx_4$ with $A = 1$ and $H = 1$ (read from
`Revision/kohn_sham/results/parameters.json`), at the hidden angle $z = \pi/4$.
Left: the two kinds of connection coefficients, $\omega_{x_1,x_1x_4} =
a_4'e^{a_4}\sin^{1/6}z$ of an inflating direction (it grows) and
$-\omega_{x_5,x_4x_5} = a_4'e^{-a_4}\sin^{1/6}z$ of a deflating extra time (it
decays). Right: their contributions to the coefficient of $\gamma^{(x_4)}$ in
$\gamma^\mu\Omega_\mu$, which do NOT depend on $x_4$ (the factor $1/f_\mu =
e^{\mp a_4}/\sin^{1/6}z$ of $\gamma^\mu$ undoes the growth and the decay) and
cancel exactly.
```

In [6]:

```
params = read_json(PARAMETERS)["physics"]
A_hist, H_hist = params["historyA"], params["H"]  # 1.0 and 1.0
history = {a4: A_hist * H_hist * x4, H: H_hist}  # a4 = A H x4


def along(expr, times, zv=sp.pi / 4):
    """Evaluate expr along the history at the times x4 (numbers for the figure)."""
    e = expr.subs(sp.Derivative(a4, x4), A_hist * H_hist).subs(history)
    f = sp.lambdify(x4, e.subs(z, zv), "numpy")
    return np.broadcast_to(np.asarray(f(times), dtype=float), times.shape)


times = np.linspace(0.0, 2.0, 201)  # x4 from 0 to 2, a4 from 0 to 2
infl = along(patch.om[1, 1, 4], times)
defl = along(-patch.om[5, 4, 5], times)
c_infl = along(3 * coefficient(contributions[1], 4), times)
c_defl = along(3 * coefficient(contributions[5], 4), times)
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
axes[0].semilogy(times, infl, color="#2a78d6", label="inflating, $x_1$")
axes[0].semilogy(times, defl, color="#eb6834", label="deflating extra time, $x_5$")
axes[0].set_xlabel("time $x_4$ (units $1/H$); $a_4 = AHx_4$")
axes[0].set_ylabel("connection coefficient (units $H$)")
axes[0].legend()
axes[1].plot(times, c_infl, color="#2a78d6", label="$x_1, x_2, x_3$ together")
axes[1].plot(times, c_defl, color="#eb6834", label="$x_5, x_6, x_7$ together")
axes[1].plot(times, c_infl + c_defl, color="#52514e", linestyle="--", label="sum")
axes[1].set_xlabel("time $x_4$ (units $1/H$)")
axes[1].set_ylabel("coefficient of $\\gamma^{(x_4)}$ in $\\gamma^\\mu\\Omega_\\mu$")
axes[1].set_ylim(-2.0, 2.0)
```

```
axes[1].legend(loc="center right")
save_figure(fig, "connection_history",
            "The spin connection of the author's metric along the canonical "
            "deflating history $a_4 = AHx_4$ with $A = 1$, $H = 1$, at $z = \\pi/4$. "
            "Left, logarithmic vertical axis: the coefficient "
            "$a_4'\\prime e^{a_4}\\sin^{1/6}z$ of an inflating 3-space direction "
            "grows and the coefficient $a_4'\\prime e^{-a_4}\\sin^{1/6}z$ of a "
            "deflating extra time decays, both in units of $H$, against the time "
            "$x_4$ in units of $1/H$. Right: their contributions to the "
            "coefficient of $\\gamma^{(x_4)}$ in $\\gamma^\\mu\\Omega_\\mu$, "
            "$+3AH/2$ from the three inflating and $-3AH/2$ from the three "
            "deflating directions, constant in time; their sum, dashed, is zero, "
            "so only $3H\\gamma^{(x_0)}$ survives.")
```

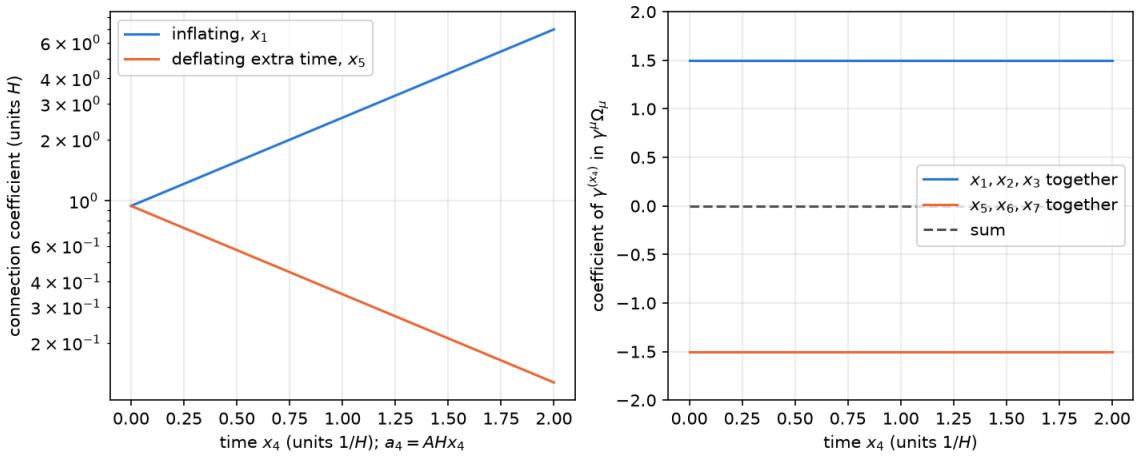

Figure 479: The spin connection of the author's metric along the canonical deflating history $a_4 = AHx_4$ with $A = 1$, $H = 1$, at $z = \pi/4$. Left, logarithmic vertical axis: the coefficient $a_4' e^{a_4} \sin^{1/6} z$ of an inflating 3-space direction grows and the coefficient $a_4' e^{-a_4} \sin^{1/6} z$ of a deflating extra time decays, both in units of H, against the time x_4 in units of $1/H$. Right: their contributions to the coefficient of $\gamma^{(x_4)}$ in $\gamma^\mu \Omega_\mu$, $+3AH/2$ from the three inflating and $-3AH/2$ from the three deflating directions, constant in time; their sum, dashed, is zero, so only $3H\gamma^{(x_0)}$ survives. (Notebook 18b, figure 1.)

Out [6]:

```
Figure 18b.1 saved as Revision/textbook/figures/18b_1_connection_history.png
```

Text cell:

```
## 7. An algebra for the jets: commuting or Grassmann

The next cell defines a small algebra of polynomials in numbered generators. An
element is a Python dictionary: each key is a monomial, written as the sorted
tuple of its generator numbers, and each value is its sympy coefficient. Adding
merges the dictionaries. Multiplying multiplies every monomial of the first
factor with every monomial of the second. For COMMUTING generators the product
```

monomial is just the sorted joined tuple. For GRASSMANN generators (‘odd=True‘)
two more rules apply: a monomial that contains a generator twice is zero
($\theta\theta = 0$), and sorting the joined tuple costs the sign $(-1)^p$, where
p is the number of pairs that must be exchanged (each exchange of two
anticommuting symbols gives a minus sign). ‘is_zero‘ decides whether every
coefficient vanishes, first with ‘expand‘ and, if needed, with ‘simplify‘.

In [7]:

```
class Jet:
    """A polynomial in numbered generators with sympy coefficients."""

    __slots__ = ("terms", "odd")

    def __init__(self, terms, odd):
        self.terms = terms  # {(generator numbers, sorted): coefficient}
        self.odd = odd  # True: Grassmann generators, False: commuting ones

    @staticmethod
    def symbol(number, odd):
        return Jet({(number,): sp.Integer(1)}, odd)

    def __add__(self, other):
        terms = dict(self.terms)
        for key, c in other.terms.items():
            terms[key] = terms[key] + c if key in terms else c
        return Jet(terms, self.odd)

    def __neg__(self):
        return Jet({key: -c for key, c in self.terms.items()}, self.odd)

    def __sub__(self, other):
        return self + (-other)

    def times(self, number):
        """Multiply by a number or a sympy coefficient."""
        if number == 0:
            return Jet({}, self.odd)
        return Jet({key: number * c for key, c in self.terms.items()}, self.odd)

    def __mul__(self, other):
        terms = {}
        for k1, c1 in self.terms.items():
            for k2, c2 in other.terms.items():
                sign = 1
                if self.odd:
                    if set(k1) & set(k2):  # a generator twice: theta theta = 0
                        continue
                    swaps = sum(1 for p in k1 for q in k2 if p > q)
                    sign = -1 if swaps % 2 else 1  # sorting costs (-1)^swaps
                key = tuple(sorted(k1 + k2))
                c = sign * c1 * c2
                terms[key] = terms[key] + c if key in terms else c
```

```
        return Jet(terms, self.odd)

    def derivative(self, number):
        """The left derivative with respect to the generator number."""
        terms = {}
        for key, c in self.terms.items():
            if number not in key:
                continue
            place = key.index(number)
            rest = key[:place] + key[place + 1:]
            if self.odd:  # move the generator to the front first
                c = c if place % 2 == 0 else -c
            else:  # an ordinary power rule
                c = key.count(number) * c
            terms[rest] = terms[rest] + c if rest in terms else c
        return Jet(terms, self.odd)

    def substitute(self, rule):
        """Replace symbols in every coefficient (rule: {old: new})."""
        return Jet({key: c.subs(rule) for key, c in self.terms.items()}, self.odd)

    def is_zero(self):
        """True when every coefficient is exactly zero."""
        for c in self.terms.values():
            e = sp.expand(c)
            if e != 0 and sp.simplify(e) != 0:
                return False
        return True


def zero(odd):
    return Jet({}, odd)


def all_zero(items):
    return all(item.is_zero() for item in items)
```

Text cell:

```
The next cell defines the matrix operations on columns of 16 algebra elements (matrix
times column, row times matrix, row times column) and the first jet. The 288
generators are numbered: $\Psi^*_A$ is $A$ ($A = 0, \dots, 15$), $\Psi_A$ is $16 +
A$, $\partial_\mu\Psi^*_A$ is $32 + 16\mu + A$ and $\partial_\mu\Psi_A$ is $160 +
16\mu + A$ ($\mu = 0, \dots, 7$ for $x_0, \dots, x_7$). The function ‘transform‘
returns the jet of the new field $\Psi'(x) = P\Psi(Rx)$: the components are
multiplied by the matrix $P$ (their conjugates by $P^*$), and the derivative along
$x_\mu$ gets the sign ‘signs[mu]‘ of the coordinate reflection $R$ (all $+1$ for T1;
$-1$ for $x_0$ in the mirror map of T2).
```

In [8]:

```
def mat_vec(M, v, odd):
    """The matrix M times the column v of algebra elements."""
```

```
    out = []
    for A in range(16):
        acc = zero(odd)
        for B in range(16):
            if M[A, B] != 0:
                acc = acc + v[B].times(M[A, B])
        out.append(acc)
    return out


def row_mat(v, M, odd):
    """The row v of algebra elements times the matrix M."""
    out = []
    for B in range(16):
        acc = zero(odd)
        for A in range(16):
            if M[A, B] != 0:
                acc = acc + v[A].times(M[A, B])
        out.append(acc)
    return out


def dot(u, v, odd):
    """The row u times the column v (the order of the factors is kept)."""
    acc = zero(odd)
    for p, q in zip(u, v):
        if p.terms and q.terms:
            acc = acc + p * q
    return acc


def number_psi_star(A):
    return A


def number_psi(A):
    return 16 + A


def number_dpsi_star(mu, A):
    return 32 + 16 * mu + A


def number_dpsi(mu, A):
    return 160 + 16 * mu + A


def jets(odd):
    """The first jet (Psi^*, Psi, d Psi^*, d Psi) as algebra generators."""
    return ([Jet.symbol(number_psi_star(A), odd) for A in range(16)],
            [Jet.symbol(number_psi(A), odd) for A in range(16)],
            {mu: [Jet.symbol(number_dpsi_star(mu, A), odd) for A in range(16)]
             for mu in range(8)},
            {mu: [Jet.symbol(number_dpsi(mu, A), odd) for A in range(16)]
```

```
             for mu in range(8)})


NO_REFLECTION = {mu: 1 for mu in range(8)}
MIRROR = {mu: (-1 if mu == 0 else 1) for mu in range(8)}  # x0 -> const - x0


def transform(J, P, signs, odd):
    """The jet of Psi'(x) = P Psi(R x)."""
    psi_star, psi, dpsi_star, dpsi = J
    Pc = P.conjugate()
    return (mat_vec(Pc, psi_star, odd), mat_vec(P, psi, odd),
            {mu: [x.times(signs[mu]) for x in mat_vec(Pc, dpsi_star[mu], odd)]
             for mu in range(8)},
            {mu: [x.times(signs[mu]) for x in mat_vec(P, dpsi[mu], odd)]
             for mu in range(8)})
```

Text cell:

```
## 8. The Lagrangian, the field equation, $T_{\mu\nu}$ and $J^\mu$ on the jet

The next cell writes the quantities of section 4 on a jet `J` in a geometry
`geo`, line by line as in the formulas: the covariant derivatives
$D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and $D_\mu\bar\Psi =
(\partial_\mu\Psi^*)^TC - \Psi^{*T}C\Omega_\mu$; the scalar $S = \Psi^{*T}C\Psi$;
the Lagrangian (its argument `upoly`, used once, replaces $\frac{\lambda}{2}S^2$ by
a general cubic $U(S) = u_1S + u_2S^2 + u_3S^3$); the field-equation operator
$E$; the energy-momentum tensor $T_{\mu\nu}$ for $\mu \le \nu$ (36 components);
the current $J^\mu$; and the Euler-Lagrange expressions derived from the
Lagrangian. The derivation uses the rule $\partial_\mu(c\,\Psi_B) =
(\partial_\mu c)\Psi_B + c\,\partial_\mu\Psi_B$ for a coefficient $c$ that
depends on the point (the metric functions).
```

In [9]:

```
def covariant(geo, J, odd):
    """D_mu Psi and D_mu Psibar for mu = 1..8."""
    psi_star, psi, dpsi_star, dpsi = J
    Dpsi = {mu: [p + q for p, q in zip(dpsi[mu], mat_vec(geo.Om[mu], psi, odd))]
            for mu in range(8)}
    Dbar = {mu: [p - q for p, q in zip(row_mat(dpsi_star[mu], C, odd),
                                       row_mat(psi_star, C * geo.Om[mu], odd))]
            for mu in range(8)}
    return Dpsi, Dbar


def scalar(J, odd):
    """S = Psibar Psi = Psi^dagger C Psi."""
    return dot(J[0], mat_vec(C, J[1], odd), odd)


def lagrangian(geo, J, odd, mass, coup, upoly=None, density=True):
```

```
    """L = sqrt|g| [ K - m S - U(S) ] (density=False: without sqrt|g|)."""
    Dpsi, Dbar = covariant(geo, J, odd)
    kin = zero(odd)
    for mu in range(8):
        kin = kin + dot(J[0], mat_vec(C * geo.gup[mu], Dpsi[mu], odd), odd)
        kin = kin - dot(Dbar[mu], mat_vec(geo.gup[mu], J[1], odd), odd)
    S = scalar(J, odd)
    L = kin.times(sp.Rational(1, 2)) - S.times(mass)
    if upoly is None:  # U = (lambda/2) S^2
        L = L - (S * S).times(coup / 2)
    else:  # U = u1 S + u2 S^2 + u3 S^3
        L = L - S.times(upoly[1]) - (S * S).times(upoly[2])
        L = L - (S * S * S).times(upoly[3])
    return L.times(geo.sqrtg) if density else L


def field_operator(geo, J, odd, mass, coup):
    """E = gamma^mu D_mu Psi - (m + lambda S) Psi (16 algebra elements)."""
    Dpsi, _ = covariant(geo, J, odd)
    S = scalar(J, odd)
    out = [zero(odd) for _ in range(16)]
    for mu in range(8):
        out = [p + q for p, q in zip(out, mat_vec(geo.gup[mu], Dpsi[mu], odd))]
    return [out[A] - J[1][A].times(mass) - (S * J[1][A]).times(coup)
            for A in range(16)]


def emt(geo, J, odd, mass, coup):
    """T_mu nu for mu <= nu (36 components)."""
    Dpsi, Dbar = covariant(geo, J, odd)
    L = lagrangian(geo, J, odd, mass, coup, density=False)
    T = {}
    for mu in range(8):
        for nu in range(mu, 8):
            t = (dot(J[0], mat_vec(C * geo.glow[mu], Dpsi[nu], odd), odd)
                 + dot(J[0], mat_vec(C * geo.glow[nu], Dpsi[mu], odd), odd)
                 - dot(Dbar[mu], mat_vec(geo.glow[nu], J[1], odd), odd)
                 - dot(Dbar[nu], mat_vec(geo.glow[mu], J[1], odd), odd))
            t = t.times(sp.Rational(1, 4))
            if mu == nu:
                t = t - L.times(geo.g[mu])
            T[mu, nu] = t
    return T


def current(geo, J, odd):
    """J^mu = i Psibar gamma^mu Psi (8 components)."""
    return {mu: dot(J[0], mat_vec(sp.I * C * geo.gup[mu], J[1], odd), odd)
            for mu in range(8)}


def euler_lagrange(geo, J, odd, mass, coup):
    """dL/dPsi^*_A - sum_mu d_mu (dL/d(d_mu Psi^*_A)), A = 0..15."""
    L = lagrangian(geo, J, odd, mass, coup)
```

```
    out = []
    for A in range(16):
        e = L.derivative(number_psi_star(A))
        for mu in range(8):
            q = L.derivative(number_dpsi_star(mu, A))  # c Psi_B, linear in Psi
            for key, c in q.terms.items():
                B = key[0] - 16  # the component Psi_B of this term
                e = e - Jet({key: d(c, mu)}, odd)  # (d_mu c) Psi_B
                e = e - Jet({(number_dpsi(mu, B),): c}, odd)  # c d_mu Psi_B
        out.append(e)
    return out
```

Text cell:

```
## 9. Theorem T1 in the author's metric, both fields

The next cell defines the complete T1 test for one statistics and runs it for
commuting components (dirac16complex00). With $\Psi' = \Gamma\Psi$ (the jet
transformed by $\Gamma$, no coordinate change) it checks, each as an identity in
the 288 jet values:

1. $S[\Gamma\Psi] - S[\Psi] = 0$;
2. $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,-\lambda}[\Psi] = 0$,
   and it counts the monomials of $\mathcal{L}$ with a nonzero coefficient (the
   Wolfram report records this count);
3. negative controls: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] +
   \mathcal{L}_{m,\lambda}[\Psi] \neq 0$ and $\mathcal{L}_{m,\lambda}[\Gamma\Psi] +
   \mathcal{L}_{-m,\lambda}[\Psi] \neq 0$ (the mass AND the coupling must change
   sign);
4. the Euler-Lagrange expressions derived from $\mathcal{L}$ equal
   $\sqrt{|g|}\,[CE_{m,\lambda}]_A$ for all 16 $A$ (so $E = 0$ IS the field
   equation);
5. $E_{m,\lambda}[\Gamma\Psi] + \Gamma E_{-m,-\lambda}[\Psi] = 0$ (16 components):
   $\Gamma\Psi$ solves the $(m, \lambda)$ equations exactly when $\Psi$ solves
   the $(-m, -\lambda)$ equations;
6. $T_{\mu\nu}[\Gamma\Psi; m, \lambda] + T_{\mu\nu}[\Psi; -m, -\lambda] = 0$ and,
   for the pair, $T_{\mu\nu}[\Psi; m, \lambda] + T_{\mu\nu}[\Gamma\Psi; -m,
   -\lambda] = 0$ (36 components each);
7. $J^\mu[\Gamma\Psi] + J^\mu[\Psi] = 0$ (8 components).

The cell also stores the monomial counts used by the figures. To count the
nonzero coefficients quickly it evaluates each coefficient at one point with
numbers (an exact zero gives 0; this count is a description, not a proof).
```

In [10]:

```
SAMPLE = {z: sp.Rational(7, 10), H: sp.Rational(13, 10), m: sp.Rational(11, 10),
          lam: sp.Rational(3, 5)}  # a sample point for counting only
SAMPLE_A4 = {"a4p": sp.Rational(9, 10), "a4": sp.Rational(2, 5)}


def count_nonzero(element):
```

```
    """The number of monomials whose coefficient is not zero at the sample."""
    n = 0
    for c in element.terms.values():
        c = c.subs(sp.Derivative(a4, x4), SAMPLE_A4["a4p"])  # a4' first,
        c = c.subs(a4, SAMPLE_A4["a4"]).subs(SAMPLE)  # then a4 and the rest
        value = complex(sp.N(c, 30))
        n += abs(value) > 1e-20
    return n


COUNTS = {}  # the monomial counts for the figures


def t1_checks(odd):
    stat = "grassmann" if odd else "commuting"
    field = "dirac16complex" if odd else "dirac16complex00"
    tag = f"primordial_{stat}"
    J = jets(odd)
    JG = transform(J, GAM, NO_REFLECTION, odd)
    s_ok = (scalar(JG, odd) - scalar(J, odd)).is_zero()
    L_image = lagrangian(patch, JG, odd, m, lam)
    L_plain = lagrangian(patch, J, odd, m, lam)
    L_partner = lagrangian(patch, J, odd, -m, -lam)
    L_wrong = lagrangian(patch, J, odd, -m, lam)
    n_L = count_nonzero(L_plain)
    COUNTS[stat] = {"L": n_L, "L[Gamma Psi]": count_nonzero(L_image),
                    "T1 sum": count_nonzero(L_image + L_partner),
                    "control (m, l)": count_nonzero(L_image + L_plain),
                    "control (-m, l)": count_nonzero(L_image + L_wrong)}
    wl_count = f"L has {n_L} monomials" in recorded(WL, f"T1_Lagrangian_{tag}")[
        "detail"]
    report(f"{field}: monomials of L with a nonzero coefficient", n_L)
    reproduces(s_ok and (L_image + L_partner).is_zero() and wl_count,
               f"T1 {field}: S kept, L_m,l[Gamma Psi] = -L_-m,-l[Psi]",
               (PY, [f"T1.metric.{stat}.S_invariant", f"T1.metric.{stat}.lagrangian"]),
               (WL, [f"T1_scalar_and_kinetic_{tag}", f"T1_Lagrangian_{tag}"]))
    controls = (not (L_image + L_plain).is_zero()
                and not (L_image + L_wrong).is_zero())
    reproduces(controls, f"T1 {field}: the negative controls fail as they must",
               (PY, [f"T1.metric.{stat}.negative_controls"]),
               (WL, [f"T1_Lagrangian_negative_controls_{tag}"]))
    E = field_operator(patch, J, odd, m, lam)
    CE = mat_vec(C, E, odd)
    EL = euler_lagrange(patch, J, odd, m, lam)
    derived = all_zero([EL[A] - CE[A].times(patch.sqrtg) for A in range(16)])
    E_image = field_operator(patch, JG, odd, m, lam)
    E_partner = mat_vec(GAM, field_operator(patch, J, odd, -m, -lam), odd)
    mapped = all_zero([E_image[A] + E_partner[A] for A in range(16)])
    reproduces(derived and mapped,
               f"T1 {field}: EL = sqrt|g| C E; E_m,l[Gamma Psi] = -Gamma E_-m,-l",
               (PY, [f"T1.metric.{stat}.euler_lagrange_derived",
                     f"T1.metric.{stat}.euler_lagrange_map"]),
               (WL, [f"T1_Euler_Lagrange_derived_{tag}",
                     f"T1_field_equation_covariance_{tag}"]))
```

```
    T_image = emt(patch, JG, odd, m, lam)
    T_partner = emt(patch, J, odd, -m, -lam)
    T_plain = emt(patch, J, odd, m, lam)
    T_pair = emt(patch, JG, odd, -m, -lam)
    emt_ok = all_zero([T_image[k] + T_partner[k] for k in T_image])
    pair_ok = all_zero([T_plain[k] + T_pair[k] for k in T_plain])
    COUNTS[stat]["T"] = {k: count_nonzero(T_plain[k]) for k in T_plain}
    COUNTS[stat]["T1 sign"] = {k: -1 for k in T_plain if emt_ok}
    J_image, J_plain = current(patch, JG, odd), current(patch, J, odd)
    current_ok = all_zero([J_image[mu] + J_plain[mu] for mu in range(8)])
    reproduces(emt_ok and pair_ok and current_ok and len(T_image) == 36,
               f"T1 {field}: T -> -T (36), pair T sum 0, J -> -J (8)",
               (PY, [f"T1.metric.{stat}.emt", f"T1.metric.{stat}.pair_total_emt_zero",
                     f"T1.metric.{stat}.current"]),
               (WL, [f"T1_energy_momentum_{tag}", f"T1_current_{tag}"]))


t1_checks(False)  # dirac16complex00: commuting components
```

Out [10]:

```
RESULT dirac16complex00: monomials of L with a nonzero coefficient = 408
PASS T1 dirac16complex00: S kept, L_m,l[Gamma Psi] = -L_-m,-l[Psi]
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.S_invariant, T1.metric.commuting.lagrangian;
    Revision/pairing/reports/wolfram-pairing.json, check
    T1_scalar_and_kinetic_primordial_commuting, T1_Lagrangian_primordial_commuting
PASS T1 dirac16complex00: the negative controls fail as they must
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.negative_controls; Revision/pairing/reports/wolfram-pairing.json,
    check T1_Lagrangian_negative_controls_primordial_commuting
PASS T1 dirac16complex00: EL = sqrt|g| C E; E_m,l[Gamma Psi] = -Gamma E_-m,-l
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.euler_lagrange_derived, T1.metric.commuting.euler_lagrange_map;
    Revision/pairing/reports/wolfram-pairing.json, check
    T1_Euler_Lagrange_derived_primordial_commuting,
    T1_field_equation_covariance_primordial_commuting
PASS T1 dirac16complex00: T -> -T (36), pair T sum 0, J -> -J (8)
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.emt, T1.metric.commuting.pair_total_emt_zero,
    T1.metric.commuting.current; Revision/pairing/reports/wolfram-pairing.json, check
    T1_energy_momentum_primordial_commuting, T1_current_primordial_commuting
```

Text cell:

```
The next cell runs the same test with Grassmann components (dirac16complex).
Because $\Gamma$ only multiplies each component by $+1$ or $-1$, no two
anticommuting factors are ever exchanged by the map, and all checks pass in the
same way; the Lagrangian has fewer monomials, because every monomial of $S^2$ that
contains one generator twice vanishes ($\theta\theta = 0$).
```

In [11]:

```
t1_checks(True)  # dirac16complex: Grassmann components
say("monomials of L with a nonzero coefficient: commuting "
    f"{COUNTS['commuting']['L']}, Grassmann {COUNTS['grassmann']['L']}")
```

Out [11]:

```
RESULT dirac16complex: monomials of L with a nonzero coefficient = 392
PASS T1 dirac16complex: S kept, L_m,l[Gamma Psi] = -L_-m,-l[Psi]
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.grassmann.S_invariant, T1.metric.grassmann.lagrangian;
    Revision/pairing/reports/wolfram-pairing.json, check
    T1_scalar_and_kinetic_primordial_grassmann, T1_Lagrangian_primordial_grassmann
PASS T1 dirac16complex: the negative controls fail as they must
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.grassmann.negative_controls; Revision/pairing/reports/wolfram-pairing.json,
    check T1_Lagrangian_negative_controls_primordial_grassmann
PASS T1 dirac16complex: EL = sqrt|g| C E; E_m,l[Gamma Psi] = -Gamma E_-m,-l
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.grassmann.euler_lagrange_derived, T1.metric.grassmann.euler_lagrange_map;
    Revision/pairing/reports/wolfram-pairing.json, check
    T1_Euler_Lagrange_derived_primordial_grassmann,
    T1_field_equation_covariance_primordial_grassmann
PASS T1 dirac16complex: T -> -T (36), pair T sum 0, J -> -J (8)
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.grassmann.emt, T1.metric.grassmann.pair_total_emt_zero,
    T1.metric.grassmann.current; Revision/pairing/reports/wolfram-pairing.json, check
    T1_energy_momentum_primordial_grassmann, T1_current_primordial_grassmann
monomials of L with a nonzero coefficient: commuting 408, Grassmann 392
```

Text cell:

```
The next cell adds two more T1 tests. (a) A general potential: with
$U(S) = u_1S + u_2S^2 + u_3S^3$ (three arbitrary coefficients),
$\mathcal{L}_{m,U}[\Gamma\Psi] + \mathcal{L}_{-m,-U}[\Psi] = 0$, for both
statistics. (b) A general gravitational field, not the author's: an arbitrary
non-diagonal vielbein and an arbitrary connection at one point, made of random
fractions with the fixed seed 20261001 of the Revision report (64 entries
$e^\mu{}_a$, 28 independent $\omega_{\mu ab}$ for each $\mu$, and
$\sqrt{|g|}$), where the Lagrangian identity and the map of the field operator
must hold too, for both statistics.
```

In [12]:

```
u1, u2, u3 = sp.symbols("u1 u2 u3", real=True)
cubic_ok = True
for odd in (False, True):
    J = jets(odd)
    JG = transform(J, GAM, NO_REFLECTION, odd)
    L1 = lagrangian(patch, JG, odd, m, 0, upoly={1: u1, 2: u2, 3: u3})
    L2 = lagrangian(patch, J, odd, -m, 0, upoly={1: -u1, 2: -u2, 3: -u3})
    cubic_ok = cubic_ok and (L1 + L2).is_zero()
```

```
reproduces(cubic_ok, "T1 with a general cubic potential: L_m,U[Gamma Psi] = "
           "-L_-m,-U[Psi]", (PY, ["T1.metric.commuting.general_potential",
                                  "T1.metric.grassmann.general_potential"]))

rng = random.Random(20261001)  # the seed of the Revision report


def fraction():
    return sp.Rational(rng.randint(-9, 9), rng.randint(1, 5))


class RandomField:
    """A general gravitational field at one point (only what L and E use)."""


field = RandomField()
E8 = [[fraction() for _ in range(8)] for _ in range(8)]  # e^mu_a, not diagonal
field.gup = {mu: sum((E8[mu][a] * G[a] for a in range(8)), sp.zeros(16))
             for mu in range(8)}
field.Om = {}
for mu in range(8):
    M = sp.zeros(16)
    for a in range(8):
        for b in range(a + 1, 8):
            M += fraction() * SAB[a, b]  # an antisymmetric omega
    field.Om[mu] = M
field.sqrtg = abs(fraction()) + 1
random_ok = True
for odd in (False, True):
    J = jets(odd)
    JG = transform(J, GAM, NO_REFLECTION, odd)
    L_sum = (lagrangian(field, JG, odd, m, lam)
             + lagrangian(field, J, odd, -m, -lam))
    E1 = field_operator(field, JG, odd, m, lam)
    E2 = mat_vec(GAM, field_operator(field, J, odd, -m, -lam), odd)
    random_ok = random_ok and all_zero([L_sum] + [E1[A] + E2[A] for A in range(16)])
reproduces(random_ok, "T1 in a random general field (seed 20261001), both fields",
           (PY, ["T1.general_field.random_instance.commuting",
                 "T1.general_field.random_instance.grassmann"]))
```

Out [12]:

```
PASS T1 with a general cubic potential: L_m,U[Gamma Psi] = -L_-m,-U[Psi]
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.general_potential, T1.metric.grassmann.general_potential
PASS T1 in a random general field (seed 20261001), both fields
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.general_field.random_instance.commuting,
    T1.general_field.random_instance.grassmann
```

Text cell:

```
The next cell draws what cancels. For each statistics it shows five bars: the
number of monomials (with a nonzero coefficient) of $\mathcal{L}_{m,\lambda}
[\Psi]$ and of $\mathcal{L}_{m,\lambda}[\Gamma\Psi]$; of the T1 sum
$\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,-\lambda}[\Psi]$ (zero);
and of the two negative controls, where the mass term and the $S^2$ term (first
control) or only the $S^2$ term (second control) survive.
```

In [13]:

```
labels = ["L", "L[Gamma Psi]", "T1 sum", "control (m, l)", "control (-m, l)"]
texts = ["$\\mathcal{L}_{m,\\lambda}[\\Psi]$",
         "$\\mathcal{L}_{m,\\lambda}[\\Gamma\\Psi]$",
         "T1: $+\\,\\mathcal{L}_{-m,-\\lambda}[\\Psi]$",
         "control: $+\\,\\mathcal{L}_{m,\\lambda}[\\Psi]$",
         "control: $+\\,\\mathcal{L}_{-m,\\lambda}[\\Psi]$"]
fig, ax = plt.subplots(figsize=(9.0, 4.6))
xs = np.arange(len(labels))
for shift, stat, colour in [(-0.18, "commuting", "#2a78d6"),
                            (0.18, "grassmann", "#eb6834")]:
    values = [COUNTS[stat][k] for k in labels]
    ax.bar(xs + shift, values, width=0.36, color=colour,
           label="commuting (dirac16complex00)" if stat == "commuting"
           else "Grassmann (dirac16complex)")
    for x, v in zip(xs + shift, values):
        ax.text(x, v + 6, str(v), ha="center", fontsize=8)
ax.set_xticks(xs, texts, fontsize=8.5, rotation=12)
ax.set_ylabel("monomials with a nonzero coefficient")
ax.set_ylim(0, 480)
ax.legend(loc="upper right")
save_figure(fig, "lagrangian_cancellation",
            "What cancels in theorem T1, in the author's metric with an arbitrary "
            "deflating history. Vertical axis: the number of monomials in the 288 "
            "jet values whose coefficient is not zero. The first two bars of each "
            "colour: the Lagrangian of a field and of its chirality image, "
            "$408$ monomials for commuting components and $392$ for Grassmann "
            "components. Third: the T1 sum with the reversed mass and coupling, "
            "exactly zero. Fourth and fifth, the negative controls: with the "
            "same mass and coupling the mass term and the $S^2$ term survive; "
            "with only the mass reversed the $S^2$ term survives. So both $m$ and "
            "$\\lambda$ must change sign.")
```

Out [13]:

```
Figure 18b.2 saved as Revision/textbook/figures/18b_2_lagrangian_cancellation.png
```

Text cell:

```
The next cell draws the 36 components of the energy-momentum tensor. Left: the
number of monomials of each component $T_{\mu\nu}[\Psi; m, \lambda]$ of a field
with commuting components (the table is symmetric, $T_{\nu\mu} = T_{\mu\nu}$, and
is drawn in full). Right: the sign $s$ with $T_{\mu\nu}[\Gamma\Psi; -m, -\lambda]
```

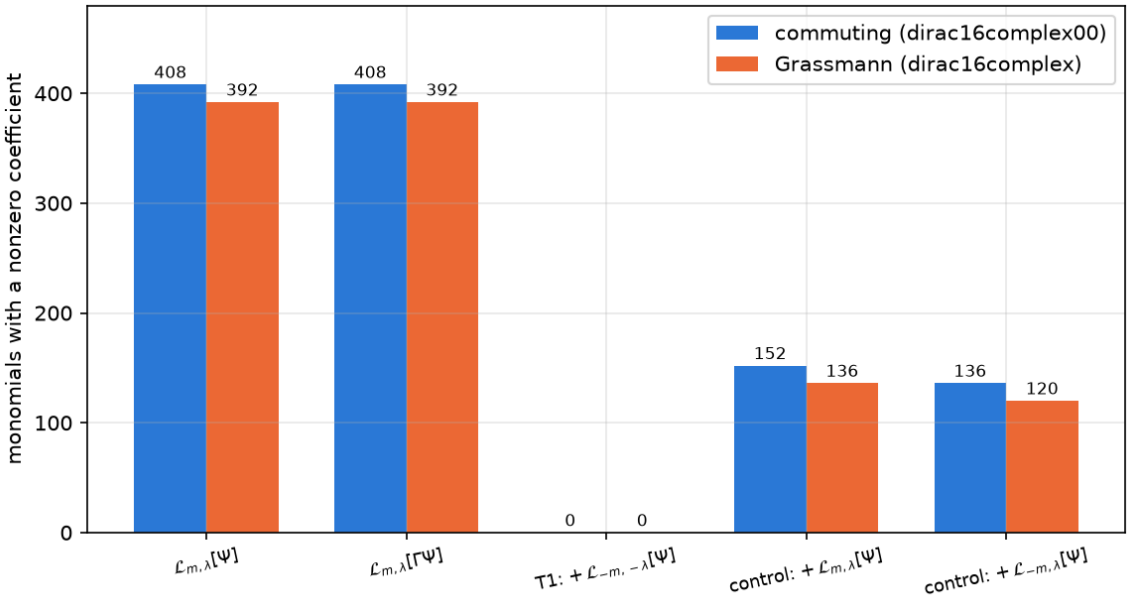

Figure 480: What cancels in theorem T1, in the author's metric with an arbitrary deflating history. Vertical axis: the number of monomials in the 288 jet values whose coefficient is not zero. The first two bars of each colour: the Lagrangian of a field and of its chirality image, 408 monomials for commuting components and 392 for Grassmann components. Third: the T1 sum with the reversed mass and coupling, exactly zero. Fourth and fifth, the negative controls: with the same mass and coupling the mass term and the S^2 term survive; with only the mass reversed the S^2 term survives. So both m and λ must change sign. (Notebook 18b, figure 2.)

```
= s\,T_{\mu\nu}[\Psi; m, \lambda]$, which the cell above proved to be $-1$ for
every component: the T1 partner carries exactly the opposite energy density,
momentum density and pressures.
```

In [14]:

```
counts = np.zeros((8, 8))
signs = np.zeros((8, 8))
for (mu, nu), n in COUNTS["commuting"]["T"].items():
    counts[mu, nu] = counts[nu, mu] = n
for (mu, nu), s in COUNTS["commuting"]["T1 sign"].items():
    signs[mu, nu] = signs[nu, mu] = s
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.6))
image = axes[0].imshow(counts, cmap="Blues")
fig.colorbar(image, ax=axes[0], shrink=0.85, label="monomials")
axes[1].imshow(signs, cmap="coolwarm", vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        axes[0].text(j, i, str(int(counts[i, j])), ha="center", va="center",
                     fontsize=7, color="black" if counts[i, j] < 200 else "white")
        axes[1].text(j, i, f"{int(signs[i, j]):+d}", ha="center", va="center",
                     fontsize=8, color="white")
for ax, title in [(axes[0], "monomials of $T_{\\mu\\nu}[\\Psi; m, \\lambda]$"),
                  (axes[1], "sign of the T1 partner's $T_{\\mu\\nu}$")]:
    ax.set_xticks(range(8), [f"$x_{a}$" for a in range(8)])
    ax.set_yticks(range(8), [f"$x_{a}$" for a in range(8)])
    ax.grid(False)
    ax.set_title(title, fontsize=10)
save_figure(fig, "emt_signs",
            "The 36 independent components of the energy-momentum tensor in the "
            "author's metric, row index $\\mu$ and column index $\\nu$ from $x_0$ "
            "to $x_7$. Left: the number of jet monomials in each component for a "
            "field with commuting components; the diagonal components carry the "
            "whole Lagrangian, the off-diagonal ones only kinetic terms. Right: "
            "the sign that relates the component of the T1 partner "
            "$\\Gamma\\Psi$ with $(-m, -\\lambda)$ to that of $\\Psi$ with "
            "$(m, \\lambda)$, proved to be $-1$ everywhere: energy density, "
            "momentum densities and pressures are all reversed, and the pair has "
            "a zero total.")
```

Out [14]:

```
Figure 18b.3 saved as Revision/textbook/figures/18b_3_emt_signs.png
```

Text cell:

```
## 10. Theorem T2: the mirror across the brane $z = \pi/2$

The next cell builds the mirror patch $\pi/2 < z < \pi$ with its own positive
vielbein ($f_0 = -\cot z > 0$ there) and the patch geometry *evaluated at the
mirror point*: every coefficient function of the patch with $z$ replaced by
```

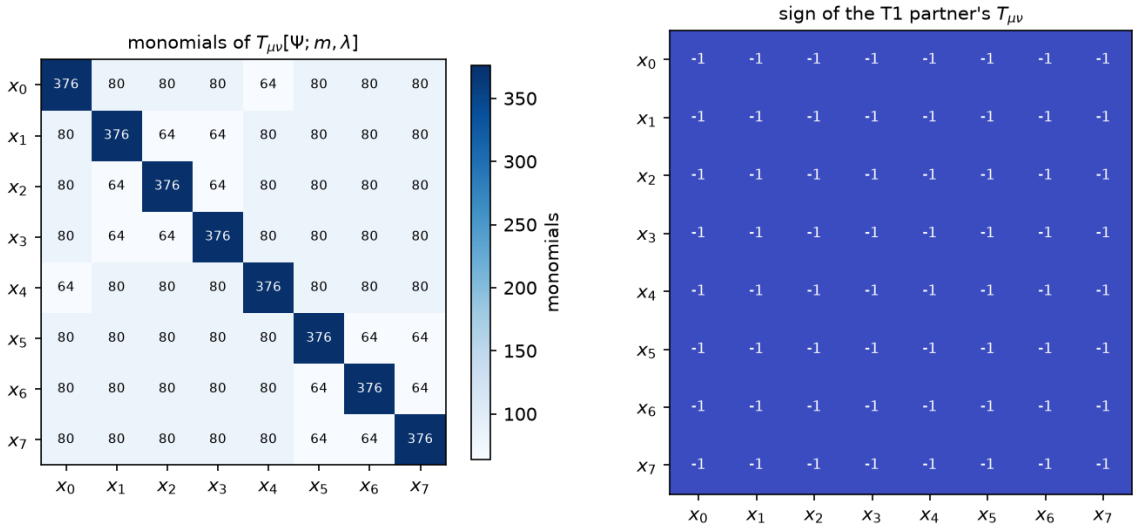

Figure 481: The 36 independent components of the energy-momentum tensor in the author's metric, row index μ and column index ν from x_0 to x_7. Left: the number of jet monomials in each component for a field with commuting components; the diagonal components carry the whole Lagrangian, the off-diagonal ones only kinetic terms. Right: the sign that relates the component of the T1 partner $\Gamma\Psi$ with $(-m, -\lambda)$ to that of Ψ with (m, λ), proved to be -1 everywhere: energy density, momentum densities and pressures are all reversed, and the pair has a zero total. (Notebook 18b, figure 3.)

```
$\pi - z$. It checks that $z \to \pi - z$ (that is, $x_0 \to \pi/(6H) - x_0$) is
an isometry, $g_{\mu\mu}(\pi - z) = g_{\mu\mu}(z)$ for all eight components, and
that the brane is a degenerate surface: $g_{00} \to 0$ and $\sqrt{|g|} \to 0$ as
$z \to \pi/2$. Finally it checks the mirror patch's own connection (antisymmetry
and the vielbein postulate).
```

In [15]:

```
mirror = Geometry(-1)  # the mirror patch pi/2 < z < pi, f0 = -cot z > 0
to_mirror = {z: sp.pi - z}


class Reflected:
    """The patch's coefficient functions evaluated at the mirror point pi - z."""


reflected = Reflected()
reflected.sqrtg = sp.expand(patch.sqrtg.subs(to_mirror))
reflected.g = {mu: sp.expand(patch.g[mu].subs(to_mirror)) for mu in range(8)}
for name in ("Om", "gup", "glow"):
    setattr(reflected, name, {mu: getattr(patch, name)[mu].subs(to_mirror)
                              .applyfunc(sp.expand) for mu in range(8)})
isometry = all(sp.simplify(patch.g[mu].subs(to_mirror) - patch.g[mu]) == 0
               for mu in range(8))
degenerate = (sp.limit(patch.g[0], z, sp.pi / 2) == 0
```

```
              and sp.limit(patch.sqrtg, z, sp.pi / 2) == 0)
reproduces(isometry and degenerate, "z -> pi - z is an isometry; the brane "
           "z = pi/2 is degenerate (g00 = 0, sqrt|g| = 0)",
           (PY, ["geometry.mirror_isometry", "geometry.brane_degenerate"]),
           (WL, ["T2_mirror_is_isometry"]))
mirror_postulate = all(
    sp.simplify(mirror.om[mu, a, b] + mirror.om[mu, b, a]) == 0
    for mu, a, b in itertools.product(range(8), repeat=3))
for mu, nu in itertools.product(range(8), repeat=2):
    M = mirror.gup[nu].applyfunc(lambda e: d(e, mu))
    for la in range(8):
        if mirror.chr[nu, mu, la] != 0:
            M = M + mirror.chr[nu, mu, la] * mirror.gup[la]
    M = M + mirror.Om[mu] * mirror.gup[nu] - mirror.gup[nu] * mirror.Om[mu]
    mirror_postulate = mirror_postulate and all(sp.simplify(e) == 0 for e in M)
say(f"sqrt|g| on the mirror patch = {mirror.sqrtg} (positive there, cos z < 0)")
reproduces(mirror_postulate, "the mirror patch's own connection is canonical",
           (WL, ["connection_mirror_patch"]))
```

Out [15]:

```
PASS z -> pi - z is an isometry; the brane z = pi/2 is degenerate (g00 = 0, sqrt|g| = 0)
     reproduces Revision/pairing/reports/python-pairing.json, check
    geometry.mirror_isometry, geometry.brane_degenerate;
    Revision/pairing/reports/wolfram-pairing.json, check T2_mirror_is_isometry
sqrt|g| on the mirror patch = -cos(z) (positive there, cos z < 0)
PASS the mirror patch's own connection is canonical
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    connection_mirror_patch
```

Text cell:

```
The next cell draws the metric functions on both patches against $z$ from 0 to
$\pi$: they are mirror-symmetric about the brane $z = \pi/2$, where $g_{00}$ and
$\sqrt{|g|}$ vanish.
```

In [16]:

```
zs = np.linspace(0.02, np.pi - 0.02, 400)
fig, ax = plt.subplots(figsize=(8.0, 4.4))
ax.plot(zs, np.sin(zs) ** (1.0 / 3.0), color="#2a78d6",
        label="$\\sin^{1/3}z$ (factor of $g_{11}$ and $-g_{55}$)")
ax.plot(zs, 1.0 / np.tan(zs) ** 2, color="#eb6834", label="$g_{00} = \\cot^2 z$")
ax.plot(zs, np.abs(np.cos(zs)), color="#1baf7a", label="$\\sqrt{|g|} = |\\cos z|$")
ax.axvline(np.pi / 2, color="#52514e", linestyle="--")
ax.text(np.pi / 2 + 0.04, 2.6, "brane $z = \\pi/2$", fontsize=9)
ax.axvspan(0.0, np.pi / 2, color="#2a78d6", alpha=0.05)
ax.text(0.25, 2.6, "patch", fontsize=9)
ax.text(2.35, 2.6, "mirror patch", fontsize=9)
ax.set_xlim(0.0, np.pi)
ax.set_ylim(0.0, 3.0)
```

```
ax.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax.set_ylabel("metric function (no unit)")
ax.legend(loc="center left", bbox_to_anchor=(0.02, 0.62), fontsize=8.5)
save_figure(fig, "mirror_metric",
            "The functions of the hidden angle that enter the author's metric, "
            "against $z = 6Hx_0$ from 0 to $\\pi$ (horizontal axis, radians; "
            "vertical axis, pure numbers): $\\sin^{1/3}z$, the factor of the "
            "3-space and extra-time components; $g_{00} = \\cot^2 z$; and "
            "$\\sqrt{|g|} = |\\cos z|$. The shaded half is the author's patch, the "
            "other half its mirror patch. Every curve is mirror-symmetric about "
            "the dashed brane $z = \\pi/2$, so $z \\to \\pi - z$ is an isometry. "
            "At the brane $g_{00}$ and $\\sqrt{|g|}$ vanish: the metric is "
            "degenerate there, and gluing the two patches is an assumption.")
```

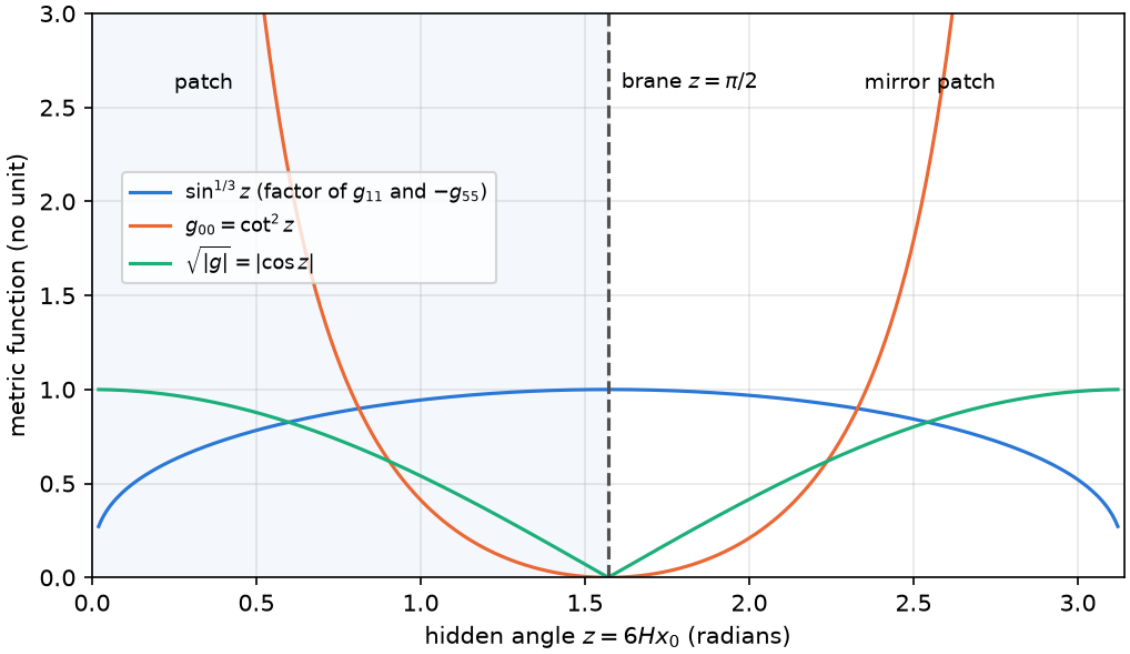

Figure 482: The functions of the hidden angle that enter the author's metric, against $z = 6Hx_0$ from 0 to π (horizontal axis, radians; vertical axis, pure numbers): $\sin^{1/3} z$, the factor of the 3-space and extra-time components; $g_{00} = \cot^2 z$; and $\sqrt{|g|} = |\cos z|$. The shaded half is the author's patch, the other half its mirror patch. Every curve is mirror-symmetric about the dashed brane $z = \pi/2$, so $z \to \pi - z$ is an isometry. At the brane g_{00} and $\sqrt{|g|}$ vanish: the metric is degenerate there, and gluing the two patches is an assumption. (Notebook 18b, figure 4.)

Out [16]:

```
Figure 18b.4 saved as Revision/textbook/figures/18b_4_mirror_metric.png
```

Text cell:

```
The next cell searches, as the sympy report does, for a relation between the
Lagrangian on the mirror patch of the new field $\Psi'(x) = P\Psi(Rx)$ ($R$: the
```

```
mirror map, which reverses the derivative along $x_0$) and the Lagrangian on the
patch at the mirror point, $\mathcal{L}^{\rm mirror}_{m,\lambda}[\Psi'] =
\sigma\,\mathcal{L}^{\rm patch}_{\pm m,\pm\lambda}[\Psi]$, with the eight sign
choices of $(\sigma, \pm m, \pm\lambda)$, for the four candidates $P = 1$,
$\Gamma$, $\gamma^{(x_0)}$ and $\Gamma\gamma^{(x_0)}$. The expected answer: no
relation for $P = 1$ and $P = \Gamma$; $P = \gamma^{(x_0)}$ gives
$+\mathcal{L}_{-m,\lambda}$ (theorem T2); $P = \Gamma\gamma^{(x_0)}$ (the
reflection $P_0$ alone) gives $-\mathcal{L}_{m,-\lambda}$.
```

In [17]:

```
CANDIDATES = [("1", I16), ("Gamma", GAM), ("gamma^(x0)", G[0]),
              ("Gamma gamma^(x0)", GAM * G[0])]
FOUND = {}
for odd in (False, True):
    J = jets(odd)
    for name, P in CANDIDATES:
        JP = transform(J, P, MIRROR, odd)
        L_mirror = lagrangian(mirror, JP, odd, m, lam)
        hit = None
        for sigma, sm, sl in itertools.product((1, -1), repeat=3):
            L_patch = lagrangian(reflected, J, odd, sm * m, sl * lam)
            if (L_mirror - L_patch.times(sigma)).is_zero():
                hit = (sigma, sm, sl)
                break
        FOUND[odd, name] = hit
for name, _ in CANDIDATES:
    hit = FOUND[False, name]
    text = "no relation" if hit is None else (
        f"L = {'+' if hit[0] > 0 else '-'}L_({'' if hit[1] > 0 else '-'}m, "
        f"{'' if hit[2] > 0 else '-'}lambda)")
    say(f"P = {name}: {text} (the same for Grassmann components: "
        f"{FOUND[True, name] == hit})")
expected = {"1": None, "Gamma": None, "gamma^(x0)": (1, -1, 1),
            "Gamma gamma^(x0)": (-1, 1, -1)}
reproduces(all(FOUND[odd, name] == expected[name]
               for odd in (False, True) for name, _ in CANDIDATES),
           "T2: only gamma^(x0) maps (m, lambda) to (-m, lambda) with L -> +L",
           (PY, ["T2.metric.commuting.reflection_table",
                 "T2.metric.grassmann.reflection_table"]),
           (WL, ["T2_mirror_Lagrangian_commuting", "T2_mirror_Lagrangian_grassmann"]))
```

Out [17]:

```
P = 1: no relation (the same for Grassmann components: True)
P = Gamma: no relation (the same for Grassmann components: True)
P = gamma^(x0): L = +L_(-m, lambda) (the same for Grassmann components: True)
P = Gamma gamma^(x0): L = -L_(m, -lambda) (the same for Grassmann components: True)
PASS T2: only gamma^(x0) maps (m, lambda) to (-m, lambda) with L -> +L
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.reflection_table, T2.metric.grassmann.reflection_table;
    Revision/pairing/reports/wolfram-pairing.json, check T2_mirror_Lagrangian_commuting,
    T2_mirror_Lagrangian_grassmann
```

Text cell:

```
The next cell completes T2 for $P = \gamma^{(x_0)}$, both statistics: the field
equations, $E^{\rm mirror}_{m,\lambda}[\Psi'] = -\gamma^{(x_0)}E^{\rm patch}
_{-m,\lambda}[\Psi]$ (so $\Psi$ solves the $(-m, \lambda)$ equations on the patch
exactly when its mirror image solves the $(m, \lambda)$ equations on the mirror
patch); the energy-momentum tensor, $T_{\mu\nu}[\Psi'] = \Lambda_\mu\Lambda_\nu
T_{\mu\nu}[\Psi; -m, \lambda]$ with $\Lambda = \mathrm{diag}(-1, 1, 1, 1, 1, 1,
1, 1)$ (the ordinary change of a tensor under the reflection of $x_0$; no other sign
change, so the energy density is EQUAL); the current, $J'^\mu = \Lambda_\mu
J^\mu$; and $S[\gamma^{(x_0)}\Psi] = -S[\Psi]$. The cell also records the sign of
every component of $T_{\mu\nu}$ for the figure.
```

In [18]:

```
t2_ok = True
T2_SIGNS = {}
for odd in (False, True):
    J = jets(odd)
    JP = transform(J, G[0], MIRROR, odd)
    E1 = field_operator(mirror, JP, odd, m, lam)
    E2 = mat_vec(G[0], field_operator(reflected, J, odd, -m, lam), odd)
    el_ok = all_zero([E1[A] + E2[A] for A in range(16)])
    T1_ = emt(mirror, JP, odd, m, lam)
    T2_ = emt(reflected, J, odd, -m, lam)
    emt_ok = all_zero([T1_[k] - T2_[k].times(MIRROR[k[0]] * MIRROR[k[1]])
                       for k in T1_])
    if not odd:
        T2_SIGNS = {k: MIRROR[k[0]] * MIRROR[k[1]] for k in T1_ if emt_ok}
    J1, J2 = current(mirror, JP, odd), current(reflected, J, odd)
    current_ok = all_zero([J1[mu] - J2[mu].times(MIRROR[mu]) for mu in range(8)])
    s_odd = (scalar(JP, odd) + scalar(J, odd)).is_zero()
    stat = "grassmann" if odd else "commuting"
    say(f"{stat}: field equations mapped {el_ok}, T pulled back {emt_ok}, "
        f"J pulled back {current_ok}, S reversed {s_odd}")
    t2_ok = t2_ok and el_ok and emt_ok and current_ok and s_odd
reproduces(t2_ok, "T2: field equations mapped, T and J pulled back (equal energy), "
           "S reversed", (PY, [f"T2.metric.{s}.{c}" for s in ("commuting", "grassmann")
                               for c in ("euler_lagrange_map", "emt", "current",
                                         "S_odd")]),
           (WL, ["T2_mirror_energy_momentum_and_current_commuting",
                 "T2_mirror_energy_momentum_and_current_grassmann",
                 "T2_mirror_Euler_Lagrange_commuting",
                 "T2_mirror_Euler_Lagrange_grassmann"]))
```

Out [18]:

```
commuting: field equations mapped True, T pulled back True, J pulled back True, S
    reversed True
grassmann: field equations mapped True, T pulled back True, J pulled back True, S
    reversed True
```

```
PASS T2: field equations mapped, T and J pulled back (equal energy), S reversed
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.euler_lagrange_map, T2.metric.commuting.emt,
    T2.metric.commuting.current, T2.metric.commuting.S_odd,
    T2.metric.grassmann.euler_lagrange_map, T2.metric.grassmann.emt,
    T2.metric.grassmann.current, T2.metric.grassmann.S_odd;
    Revision/pairing/reports/wolfram-pairing.json, check
    T2_mirror_energy_momentum_and_current_commuting,
    T2_mirror_energy_momentum_and_current_grassmann, T2_mirror_Euler_Lagrange_commuting,
    T2_mirror_Euler_Lagrange_grassmann
```

Text cell:

```
The next cell draws two summaries of T2. Left: the search of the cell before last
as a grid, one row per candidate matrix $P$ and one column per sign choice
$(\sigma, \pm m, \pm\lambda)$; a filled square marks the relation that holds.
Right: the sign $s$ in $T_{\mu\nu}[\text{mirror image}] = s\,T_{\mu\nu}[\Psi;
-m, \lambda]$, $+1$ except for the seven mixed components with one $x_0$ index.
```

In [19]:

```
choices = list(itertools.product((1, -1), repeat=3))
grid = np.zeros((len(CANDIDATES), len(choices)))
for i, (name, _) in enumerate(CANDIDATES):
    if FOUND[False, name] is not None:
        grid[i, choices.index(FOUND[False, name])] = 1.0
t2_signs = np.zeros((8, 8))
for (mu, nu), s in T2_SIGNS.items():
    t2_signs[mu, nu] = t2_signs[nu, mu] = s
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4),
                         gridspec_kw={"width_ratios": [1.4, 1.0]})
axes[0].imshow(grid, cmap="Greens", vmin=0, vmax=1.4, aspect="auto")
axes[0].set_xticks(range(len(choices)),
                   [f"{'+' if s > 0 else '-'}L\n{'' if a > 0 else '-'}m\n"
                    f"{'' if b > 0 else '-'}$\\lambda$" for s, a, b in choices],
                   fontsize=8)
axes[0].set_yticks(range(len(CANDIDATES)),
                   ["$P = 1$", "$P = \\Gamma$", "$P = \\gamma^{(x_0)}$",
                    "$P = \\Gamma\\gamma^{(x_0)}$"])
axes[0].set_title("relation $\\mathcal{L}^{mirror}_{m,\\lambda}[P\\Psi] = "
                  "\\sigma\\mathcal{L}_{\\pm m,\\pm\\lambda}[\\Psi]$", fontsize=10)
axes[0].grid(False)
axes[1].imshow(t2_signs, cmap="coolwarm", vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        axes[1].text(j, i, f"{int(t2_signs[i, j]):+d}", ha="center", va="center",
                     fontsize=8, color="white")
axes[1].set_xticks(range(8), [f"$x_{a}$" for a in range(8)])
axes[1].set_yticks(range(8), [f"$x_{a}$" for a in range(8)])
axes[1].set_title("sign of the mirror image's $T_{\\mu\\nu}$", fontsize=10)
axes[1].grid(False)
save_figure(fig, "mirror_candidates",
```

```
"Theorem T2 in the author's metric. Left: for four candidate matrices "
"$P$ (rows), the eight possible relations between the Lagrangian of "
"the mirror image on the mirror patch and the Lagrangian of $\\Psi$ on "
"the patch (columns: overall sign $\\sigma$, sign of $m$, sign of "
"$\\lambda$); a green square marks the relation that holds exactly. "
"$P = 1$ and $P = \\Gamma$ give none; $P = \\gamma^{(x_0)}$ gives "
"$+\\mathcal{L}_{-m,\\lambda}$, theorem T2; $P = \\Gamma\\gamma^{(x_0)}$ "
"gives $-\\mathcal{L}_{m,-\\lambda}$. Right: the sign of each component "
"of the mirror image's $T_{\\mu\\nu}$ relative to that of the "
"$(-m, \\lambda)$ field, $+1$ except the mixed components with one "
"$x_0$ index: the energy density is equal, not opposite.")
```

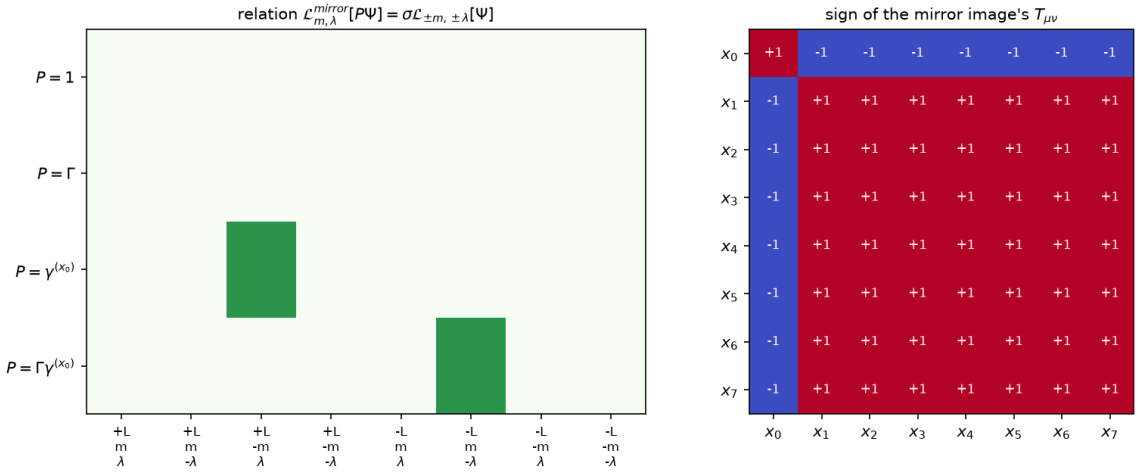

Figure 483: Theorem T2 in the author's metric. Left: for four candidate matrices P (rows), the eight possible relations between the Lagrangian of the mirror image on the mirror patch and the Lagrangian of Ψ on the patch (columns: overall sign σ, sign of m, sign of λ); a green square marks the relation that holds exactly. $P = 1$ and $P = \Gamma$ give none; $P = \gamma^{(x_0)}$ gives $+\mathcal{L}_{-m,\lambda}$, theorem T2; $P = \Gamma\gamma^{(x_0)}$ gives $-\mathcal{L}_{m,-\lambda}$. Right: the sign of each component of the mirror image's $T_{\mu\nu}$ relative to that of the $(-m, \lambda)$ field, $+1$ except the mixed components with one x_0 index: the energy density is equal, not opposite. (Notebook 18b, figure 5.)

Out [19]:

```
Figure 18b.5 saved as Revision/textbook/figures/18b_5_mirror_candidates.png
```

Text cell:

```
## 11. The last check

The last cell checks that the five figure files exist and prints the number of
checks that passed.
```

In [20]:

```
figure_names = ["connection_history", "lagrangian_cancellation", "emt_signs",
                "mirror_metric", "mirror_candidates"]
paths = [output_file(f"{FIGURE_FOLDER}/18b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [20]:

```
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 18b)
```

Text cell:

```
## 12. What this notebook showed

- PROVED (exact sympy identities in the first jet, for an arbitrary history
  $a_4(x_4)$ of the deflating extra times): the diagonal vielbein gives the
  author's metric with $\sqrt{|g|} = \cos z$; its canonical spin connection has
  exactly the 12 nonzero components of the Revision record and obeys the vielbein
  postulate; $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$, because the $a_4'$ terms of
  the three inflating directions and of the three deflating extra times cancel.
- PROVED, theorem T1, for dirac16complex00 (commuting) and dirac16complex
  (Grassmann) in the author's metric: $S[\Gamma\Psi] = S[\Psi]$;
  $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$ (the
  Lagrangian has 408, respectively 392, monomials, as in the Wolfram report); the
  Euler-Lagrange expressions are $\sqrt{|g|}\,CE$; $\Gamma\Psi$ solves the
  $(m, \lambda)$ equations exactly when $\Psi$ solves the $(-m, -\lambda)$ ones;
  all 36 components of $T_{\mu\nu}$ and all 8 of $J^\mu$ change sign, so the pair
  has zero total energy-momentum and current as classical bilinears. The negative
  controls fail: both $m$ and $\lambda$ must change sign. The same holds for a
  general cubic potential and in a random general gravitational field.
- PROVED, theorem T2: with the mirror $z \to \pi - z$ (an isometry; the brane
  $z = \pi/2$ is degenerate) and $\Psi \to \gamma^{(x_0)}\Psi$, the Lagrangian of
  the image with $(m, \lambda)$ on the mirror patch equals the Lagrangian of the
  field with $(-m, \lambda)$ on the patch; the field equations are mapped;
  $T_{\mu\nu}$ and $J^\mu$ are pulled back (equal energy density, not opposite)
  and $S$ is reversed. Neither $1$ nor $\Gamma$ gives such a relation.
- ASSUMED: the gluing of the mirror patch to the patch at the degenerate brane
  (the Z2 construction). The gravitational field is a fixed background in both
  theorems; nothing here derives back-reaction.
- NOT shown, and not following from these identities: that any universe is
  created, in pairs or otherwise. T1 and T2 map solutions to solutions; they
  contain no process, rate or amplitude.
```

18.20 Line-by-line walk-through of Notebook 18b

The notebook has 20 code cells. **In [1]** is the set-up cell, word for word the set-up cell of Notebook 18a explained line by line in Section 18.9, except that its comment lines hold the run instructions

of Section 18.18 and its line `NOTEBOOK_ID = "18b"` names this notebook, so its figures are saved as `18b_<k>_<name>.png` and their captions in `18b.captions.json`. It prints one line, Set-up of notebook 18b complete: repository folder found, helpers defined.

The idea of the notebook in one paragraph. At one point x, every quantity of the theory (the Lagrangian, the 16 components of the field operator, the 36 components of $T_{\mu\nu}$, the 8 of J^μ) is a polynomial in the 288 numbers of the **first jet**: the 16 values Ψ_A, the 16 values Ψ^*_A and the $2 \times 8 \times 16 = 256$ first derivatives $\partial_\mu \Psi_A$, $\partial_\mu \Psi^*_A$. The coefficients of the polynomial are functions of the point: of x_4 through $a_4(x_4)$ and a'_4, of z, and of H, m, λ. If a polynomial has only zero coefficients, it is zero for EVERY field at EVERY point and for EVERY history a_4. So proving "$\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,-\lambda}[\Psi]$ has only zero coefficients" proves T1 in the author's metric. To handle dirac16complex, the 288 jet values are made ANTICOMMUTING symbols; the same computation then proves the theorem for Grassmann components.

In [2], the exact gammas.

```
import itertools  # all combinations of signs
import random  # Python's random numbers with a fixed seed (the random field)

import numpy as np  # numbers for the figures
import sympy as sp  # exact symbolic algebra

GAMMAS = "Revision/algebra/gammas.json"
PY = "Revision/pairing/reports/python-pairing.json"  # the sympy verifier's report
WL = "Revision/pairing/reports/wolfram-pairing.json"  # the Wolfram verifier's report
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
```

`itertools` gives `product`, which lists all combinations (used for index triples and sign choices); `random` is Python's own random-number module (used with a fixed seed in In [12]); numpy is used only for the figures; **sympy** (`sp`) is the package for exact symbolic algebra: it computes with symbols such as z and H, with fractions and with functions, without rounding. The four constants name the records the notebook reads: the gammas, the two pairing reports, and the Kohn-Sham parameter file that holds the canonical history.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


RECORDS = {PY: read_json(PY), WL: read_json(WL)}


def recorded(report_file, name):
    """The recorded entry (verdict and detail) of the check name of a report."""
    for entry in RECORDS[report_file]["checks"]:
        if entry["name"] == name:
            return entry
    return {"verdict": "MISSING", "detail": ""}


def reproduces(condition, name, *sources):
```

```
    """check(condition, name), which also requires every named check of every
    source (report_file, [check names]) to have the recorded verdict PASS."""
    ok = all(recorded(f, n)["verdict"].upper() == "PASS"
             for f, names in sources for n in names)
    text = "; ".join(f"{f}, check {', '.join(names)}" for f, names in sources)
    check(condition and ok, name, record=text)
```

`read_json` is the reader of Notebook 18a. `RECORDS` reads the two reports once and keeps them. `recorded(report_file, name)` returns the whole entry of a check (its verdict and its detail text), or a placeholder with the verdict MISSING. `reproduces` works as in Notebook 18a: the check passes only when the notebook's own result holds and every named record check has the verdict PASS.

```
fixture = read_json(GAMMAS)
G = {a: sp.Matrix(fixture["gamma"][a]) for a in range(8)}  # gamma^(x_a), a = 0..7
ETA = {a: int(fixture["eta"][a]) for a in range(8)}
I16 = sp.eye(16)
C = G[0] * G[1] * G[2] * G[3]  # the four space-like gammas
GAM = G[0] * G[1] * G[2] * G[3] * G[4] * G[5] * G[6] * G[7]  # the chirality Gamma
SAB = {(a, b): (G[a] * G[b] - G[b] * G[a]) / 4  # the generators S^ab
       for a in range(8) for b in range(8)}
check(all(G[a] * G[b] + G[b] * G[a] == (2 * ETA[a] if a == b else 0) * I16
          for a in range(8) for b in range(8))
      and GAM == sp.diag(*([-1] * 8 + [1] * 8)),
      "eight real 16 x 16 gammas with the Clifford relations; Gamma = diag(-I8, I8)")
```

`sp.Matrix(...)` makes an exact sympy matrix from the lists of whole numbers of the record, one for each direction; `G[a]` is $\gamma^{(x_a)}$. `ETA` holds η_{aa} and `I16` is the exact identity. `C` and `GAM` (Γ) are the products of Section 18.5, written with `*`, which is the matrix product for sympy matrices. `SAB` holds the 64 generators $S^{ab} = \frac{1}{4}[\gamma^{(a)}, \gamma^{(b)}]$ exactly (entries 0 and $\pm\frac{1}{2}$). The check confirms the 64 Clifford relations and $\Gamma = \mathrm{diag}(-I_8, I_8)$ (`sp.diag(*list)` makes a diagonal matrix from the entries of the list). Out [2] shows one PASS line.

In [3], the metric and its canonical connection.

```
x4, z, H = sp.symbols("x4 z H", real=True)
m, lam = sp.symbols("m lambda", real=True)
a4 = sp.Function("a4")(x4)  # the history: any function of the time x4
```

`sp.symbols` creates the symbols x_4, z, H, m and λ, declared real. `sp.Function("a4")(x4)` is an UNKNOWN function $a_4(x_4)$: sympy differentiates it formally, writing its derivative as `Derivative(a4(x4), x4)`. Leaving a_4 unknown is what makes every result of this notebook hold for every history, in particular for the deflating ones.

```
def d(expr, mu):
    """The derivative of a coefficient along the coordinate x_mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # z = 6 H x0
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7
```

`d(expr, mu)` is the derivative of a coefficient function along the coordinate x_μ: along x_4 it is `sp.diff(expr, x4)`; along x_0 it is $6H\,d/dz$, the chain rule for $z = 6Hx_0$; along the other six coordinates it is zero, because no coefficient depends on them.

```
class Geometry:
    """The author's metric with the diagonal vielbein; s0 = +1 on the patch,
    s0 = -1 on the mirror patch (then f0 = -cot z > 0)."""

    def __init__(self, s0):
        up, down = sp.exp(a4), sp.exp(-a4)  # inflating and deflating factors
        s6 = sp.sin(z) ** sp.Rational(1, 6)
        f = {0: s0 * sp.cot(z), 1: up * s6, 2: up * s6, 3: up * s6,
             4: sp.Integer(1), 5: down * s6, 6: down * s6, 7: down * s6}
        self.f = f
        self.g = {mu: ETA[mu] * f[mu] ** 2 for mu in f}  # g_mu mu
        self.sqrtg = sp.simplify(sp.Mul(*f.values()))  # sqrt|g|
        g = self.g
```

A **class** is a recipe for an object that holds several named values; `Geometry(s0)` builds one, and `__init__` is the code that runs when it is built (`self` is the object being built). The factors f_μ of Section 18.3 are stored in the dictionary `f`, with $f_0 = s_0 \cot z$: $s_0 = +1$ on the patch and -1 on the mirror patch, where $\cot z < 0$, so that f_0 is positive on both. `self.g` holds $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$, and `self.sqrtg` the product of the eight factors (`sp.Mul(*values)` multiplies them), simplified exactly.

```
        self.chr = {}  # Christoffel symbols Gamma^nu_(mu lambda)
        for nu, mu, la in itertools.product(range(8), repeat=3):
            v = 0
            if nu == la:
                v += d(g[nu], mu)
            if nu == mu:
                v += d(g[nu], la)
            if mu == la:
                v -= d(g[mu], nu)
            self.chr[nu, mu, la] = sp.simplify(v / (2 * g[nu]))
```

The Christoffel symbols of a diagonal metric, $\Gamma^\nu{}_{\mu\lambda} = \frac{1}{2g_{\nu\nu}}(\delta_{\nu\lambda}\partial_\mu g_{\nu\nu} + \delta_{\nu\mu}\partial_\lambda g_{\nu\nu} - \delta_{\mu\lambda}\partial_\nu g_{\mu\mu})$, for all $8^3 = 512$ index triples (`itertools.product(range(8), repeat=3)` lists them). Each `if` adds one of the three terms when its Kronecker delta is 1. This is the formula of Section 18.3 written for all indices at once.

```
        self.om = {}  # omega_mu ab
        for mu, a, b in itertools.product(range(8), repeat=3):
            v = self.chr[a, mu, b] / f[b]  # e^a_nu Gamma^nu_mu lam e^lam_b ...
            if a == b:
                v += d(1 / f[b], mu)  # ... + e^a_nu d_mu e^nu_b
            self.om[mu, a, b] = sp.simplify(ETA[a] * f[a] * v)
```

The canonical connection $\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu e^\nu{}_b + \Gamma^\nu{}_{\mu\lambda} e^\lambda{}_b)$ for the diagonal vielbein: $e^\lambda{}_b = \delta^\lambda_b / f_b$ turns the second term into $\Gamma^a{}_{\mu b}/f_b$; the first term exists only for $a = b$, where it is $\partial_\mu(1/f_b)$; the factor $e^a{}_a = f_a$ multiplies both; and η_{aa} lowers the index a. So `self.om[mu, a, b]` is $\omega_{\mu ab}$.

```
        self.Om = {}  # Omega_mu = (1/2) omega_mu ab S^ab
        for mu in range(8):
            M = sp.zeros(16)
            for a, b in itertools.product(range(8), repeat=2):
                if self.om[mu, a, b] != 0:
                    M += self.om[mu, a, b] * SAB[a, b] / 2
            self.Om[mu] = M.applyfunc(sp.simplify)
        self.gup = {mu: G[mu] / f[mu] for mu in f}  # gamma^mu
        self.glow = {mu: ETA[mu] * f[mu] * G[mu] for mu in f}  # gamma_mu
```

$\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$, summed over the nonzero components only and simplified entry by entry (`applyfunc` applies a function to every entry of a matrix). `gup` holds the curved gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ and `glow` the lowered ones $\gamma_\mu = \eta_{\mu\mu}f_\mu\gamma^{(\mu)}$.

```
patch = Geometry(1)  # the author's patch 0 < z < pi/2
author = {0: sp.cot(z) ** 2, 1: sp.exp(2 * a4) * sp.sin(z) ** sp.Rational(1, 3),
          4: -1, 5: -sp.exp(-2 * a4) * sp.sin(z) ** sp.Rational(1, 3)}
author.update({2: author[1], 3: author[1], 6: author[5], 7: author[5]})
metric_ok = all(sp.simplify(patch.g[mu] - author[mu]) == 0 for mu in range(8))
say(f"sqrt|g| on the patch = {patch.sqrtg}")
reproduces(metric_ok and sp.simplify(patch.sqrtg - sp.cos(z)) == 0,
           "the vielbein gives the author's metric and sqrt|g| = cos z",
           (WL, ["primordial_vielbein"]))
```

`patch` is the geometry of the author's patch. `author` writes the eight components of the author's metric exactly as the author gave them ($e^{2a_4}\sin^{1/3} z$ for 3-space, -1 for the time, $-e^{-2a_4}\sin^{1/3} z$ for the extra times, $\cot^2 z$ for the hidden direction); `update` fills in the repeated ones. `metric_ok` checks that the vielbein reproduces all eight. Out [3] prints $\sqrt{|g|} = \cos z$ and the PASS line with the Wolfram check `primordial_vielbein`.

In [4], the connection checked three ways.

```
COORDS = {a: f"x{a}" for a in range(8)}
antisymmetric = all(sp.simplify(patch.om[mu, a, b] + patch.om[mu, b, a]) == 0
                    for mu, a, b in itertools.product(range(8), repeat=3))
postulate = True
for mu, nu in itertools.product(range(8), repeat=2):
    M = patch.gup[nu].applyfunc(lambda e: d(e, mu))  # d_mu gamma^nu
    for la in range(8):
        if patch.chr[nu, mu, la] != 0:
            M = M + patch.chr[nu, mu, la] * patch.gup[la]
    M = M + patch.Om[mu] * patch.gup[nu] - patch.gup[nu] * patch.Om[mu]
    postulate = postulate and all(sp.simplify(e) == 0 for e in M)
```

`antisymmetric` checks $\omega_{\mu ab} + \omega_{\mu ba} = 0$ for all 512 triples. The loop then checks the vielbein postulate for all 64 pairs (μ, ν): `M` starts as $\partial_\mu\gamma^\nu$ (the derivative of each entry; `lambda e: d(e, mu)` is a one-line function), adds $\sum_\lambda \Gamma^\nu{}_{\mu\lambda}\gamma^\lambda$ and the commutator $\Omega_\mu\gamma^\nu - \gamma^\nu\Omega_\mu$; every entry of the result must simplify to zero.

```
a4p = sp.Symbol("a4p")  # a short name for the derivative a4'
```

```
def short(expr):
    """Write a4' as a4p and a4(x4) as a4, as in the Revision report."""
    expr = sp.simplify(expr).subs(sp.Derivative(a4, x4), a4p)
    return expr.subs(a4, sp.Symbol("a4"))


nonzero = [(mu, a, b) for mu in range(8) for a in range(8)
           for b in range(a + 1, 8) if sp.simplify(patch.om[mu, a, b]) != 0]
listing = "; ".join(f"omega_{COORDS[mu]} {COORDS[a]}{COORDS[b]} = "
                    f"{short(patch.om[mu, a, b])}" for mu, a, b in nonzero)
for mu, a, b in nonzero:
    say(f"omega_{COORDS[mu]} {COORDS[a]}{COORDS[b]} = {short(patch.om[mu, a, b])}")
same_list = recorded(PY, "geometry.spin_connection_components")["detail"] == (
    "nonzero components (a < b): " + listing)
```

`a4p` is a plain symbol used to write a_4' in the format of the record, and `short` replaces the derivative of a_4 by `a4p` and the function $a_4(x_4)$ by the plain symbol `a4`. `nonzero` lists the triples (μ, a, b) with $a < b$ whose component is not zero; `listing` writes them in one line, separated by semicolons, and the loop prints them one per line. `same_list` compares that line, character for character, with the detail text of the sympy check `geometry.spin_connection_components`.

```
reproduces(antisymmetric and postulate, "the canonical spin connection is "
           "antisymmetric and obeys the vielbein postulate",
           (PY, ["geometry.omega_antisymmetric",
                 "geometry.gamma_covariantly_constant"]),
           (WL, ["connection_primordial"]))
reproduces(len(nonzero) == 12 and same_list,
           "the 12 nonzero connection components equal the recorded list",
           (PY, ["geometry.spin_connection_components"]))
```

Two checks: the connection is antisymmetric and canonical (three records), and it has exactly the 12 recorded components. Out [4] prints the twelve components with $a < b$, which are those derived by hand in Section 18.3: $\omega_{x_i,x_0x_i} = -He^{a_4}\sin^{1/6} z$ and $\omega_{x_i,x_ix_4} = a_4' e^{a_4}\sin^{1/6} z$ for each inflating direction x_i, $\omega_{x_t,x_0x_t} = He^{-a_4}\sin^{1/6} z$ and $\omega_{x_t,x_4x_t} = -a_4' e^{-a_4}\sin^{1/6} z$ for each deflating extra time x_t.

In [5], $\gamma\hat{}\mu\ \Omega_\mu$ direction by direction.

```
def coefficient(X, a):
    """The coefficient of gamma^(a) in the matrix X (exact, by a trace)."""
    return sp.simplify((G[a] * X).trace() / (16 * ETA[a]))
```

`coefficient(X, a)` extracts the coefficient of $\gamma^{(a)}$ in a matrix X. The rule behind it: the **trace** (the sum of the diagonal entries) of $\gamma^{(a)}\gamma^{(b)}$ is $16\eta_{aa}$ if $a = b$ and 0 otherwise. So if $X = \sum_b c_b\gamma^{(b)} +$ (products of several gammas), then $\mathrm{tr}(\gamma^{(a)}X) = 16\eta_{aa}c_a$, and $c_a = \mathrm{tr}(\gamma^{(a)}X)/(16\eta_{aa})$.

```
contributions = {mu: (patch.gup[mu] * patch.Om[mu]).applyfunc(sp.simplify)
                 for mu in range(8)}
```

```
for mu in range(8):
    c4, c0 = coefficient(contributions[mu], 4), coefficient(contributions[mu], 0)
    rest = (contributions[mu] - c4 * G[4] - c0 * G[0]).applyfunc(sp.simplify)
    assert rest == sp.zeros(16)  # nothing but gamma^(x4) and gamma^(x0)
    say(f"direction {COORDS[mu]}: coefficient of gamma^(x4) = {short(c4)}, "
        f"of gamma^(x0) = {short(c0)}")
total = sp.zeros(16)
for mu in range(8):
    total += contributions[mu]
check(total.applyfunc(sp.simplify) == 3 * H * G[0],
      "gamma^mu Omega_mu = 3 H gamma^(x0): the a4' terms of the deflation cancel")
```

`contributions[mu]` is $\gamma^{\mu}\Omega_{\mu}$ for one direction (no sum). For each direction the loop extracts the coefficients of $\gamma^{(x_4)}$ and $\gamma^{(x_0)}$ and asserts that nothing else is left (`assert` stops the notebook if the condition is false). Then `total` adds the eight contributions and the check requires $3H\gamma^{(x_0)}$. Out [5] prints, exactly as derived in Section 18.3: $a_4'/2$ and $H/2$ for x_1, x_2, x_3; $-a_4'/2$ and $H/2$ for x_5, x_6, x_7; zero for x_0 and x_4.

In [6], Figure 18b.1: the connection along the deflating history.

```
params = read_json(PARAMETERS)["physics"]
A_hist, H_hist = params["historyA"], params["H"]  # 1.0 and 1.0
history = {a4: A_hist * H_hist * x4, H: H_hist}  # a4 = A H x4


def along(expr, times, zv=sp.pi / 4):
    """Evaluate expr along the history at the times x4 (numbers for the figure)."""
    e = expr.subs(sp.Derivative(a4, x4), A_hist * H_hist).subs(history)
    f = sp.lambdify(x4, e.subs(z, zv), "numpy")
    return np.broadcast_to(np.asarray(f(times), dtype=float), times.shape)
```

`params` is the physics block of the Kohn-Sham parameter record; it gives the canonical history $a_4 = AHx_4$ with $A = 1$ and $H = 1$. `history` is a substitution rule. `along(expr, times)` turns a sympy expression into numbers for the figure: it replaces a_4' by AH (the derivative of AHx_4), then a_4 by AHx_4 and H by its value, sets $z = \pi/4$, and `sp.lambdify` turns the result into a numpy function of x_4. `np.broadcast_to` makes sure that a constant expression also returns one number per time.

```
times = np.linspace(0.0, 2.0, 201)  # x4 from 0 to 2, a4 from 0 to 2
infl = along(patch.om[1, 1, 4], times)
defl = along(-patch.om[5, 4, 5], times)
c_infl = along(3 * coefficient(contributions[1], 4), times)
c_defl = along(3 * coefficient(contributions[5], 4), times)
```

`times` holds 201 evenly spaced times from 0 to 2 (`np.linspace`). `infl` is the component $\omega_{x_1,x_1x_4} = a_4' e^{a_4} \sin^{1/6} z$ of an inflating direction and `defl` the component $-\omega_{x_5,x_4x_5} = a_4' e^{-a_4} \sin^{1/6} z$ of a deflating extra time. `c_infl` and `c_defl` are three times the coefficient of $\gamma^{(x_4)}$ from one inflating and from one deflating direction: the contributions of the three directions of each kind.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
axes[0].semilogy(times, infl, color="#2a78d6", label="inflating, $x_1$")
```

```
axes[0].semilogy(times, defl, color="#eb6834", label="deflating extra time, $x_5$")
axes[0].set_xlabel("time $x_4$ (units $1/H$); $a_4 = AHx_4$")
axes[0].set_ylabel("connection coefficient (units $H$)")
axes[0].legend()
axes[1].plot(times, c_infl, color="#2a78d6", label="$x_1, x_2, x_3$ together")
axes[1].plot(times, c_defl, color="#eb6834", label="$x_5, x_6, x_7$ together")
axes[1].plot(times, c_infl + c_defl, color="#52514e", linestyle="--", label="sum")
axes[1].set_xlabel("time $x_4$ (units $1/H$)")
axes[1].set_ylabel("coefficient of $\\gamma^{(x_4)}$ in $\\gamma^\\mu\\Omega_\\mu$")
axes[1].set_ylim(-2.0, 2.0)
axes[1].legend(loc="center right")
save_figure(fig, "connection_history",
            "The spin connection of the author's metric along the canonical "
            "deflating history $a_4 = AHx_4$ with $A = 1$, $H = 1$, at $z = \\pi/4$. "
            "Left, logarithmic vertical axis: the coefficient "
            "$a_4^\\prime e^{a_4}\\sin^{1/6}z$ of an inflating 3-space direction "
            "grows and the coefficient $a_4^\\prime e^{-a_4}\\sin^{1/6}z$ of a "
            "deflating extra time decays, both in units of $H$, against the time "
            "$x_4$ in units of $1/H$. Right: their contributions to the "
            "coefficient of $\\gamma^{(x_4)}$ in $\\gamma^\\mu\\Omega_\\mu$, "
            "$+3AH/2$ from the three inflating and $-3AH/2$ from the three "
            "deflating directions, constant in time; their sum, dashed, is zero, "
            "so only $3H\\gamma^{(x_0)}$ survives.")
```

The left panel draws the two components with a logarithmic vertical axis (`semilogy`), the right panel the two contributions and their sum (dashed). **What you see in Figure 18b.1:** on the left a rising straight line (the inflating component grows like e^{a_4}, a straight line on a logarithmic axis) and a falling one (the deflating component decays like e^{-a_4}); on the right two horizontal lines at $+3AH/2 = +1.5$ and -1.5 and their sum at zero. Why: the factor $1/f_\mu$ of γ^μ undoes the growth and the decay exactly, and the two kinds of directions contribute with opposite signs; only $3H\gamma^{(x_0)}$ survives.

In [7], an algebra for the jets.

```
class Jet:
    """A polynomial in numbered generators with sympy coefficients."""

    __slots__ = ("terms", "odd")

    def __init__(self, terms, odd):
        self.terms = terms  # {(generator numbers, sorted): coefficient}
        self.odd = odd  # True: Grassmann generators, False: commuting ones

    @staticmethod
    def symbol(number, odd):
        return Jet({(number,): sp.Integer(1)}, odd)
```

An object of the class `Jet` is a polynomial: `terms` is a dictionary whose keys are **monomials**, written as sorted tuples of generator numbers (the tuple `(3, 17)` is the product of generators 3 and 17), and whose values are sympy coefficients. `odd` says whether the generators anticommute (Grassmann) or commute. `__slots__` only fixes the two attribute names. `Jet.symbol(number,`

odd) is the polynomial consisting of one generator with coefficient 1 (@staticmethod marks a function that belongs to the class but needs no object).

```
    def __add__(self, other):
        terms = dict(self.terms)
        for key, c in other.terms.items():
            terms[key] = terms[key] + c if key in terms else c
        return Jet(terms, self.odd)

    def __neg__(self):
        return Jet({key: -c for key, c in self.terms.items()}, self.odd)

    def __sub__(self, other):
        return self + (-other)

    def times(self, number):
        """Multiply by a number or a sympy coefficient."""
        if number == 0:
            return Jet({}, self.odd)
        return Jet({key: number * c for key, c in self.terms.items()}, self.odd)
```

__add__ defines what + does: copy the terms of the first polynomial and add the coefficients of the second, monomial by monomial (terms[key] + c if key in terms else c adds when the monomial is already present and inserts it otherwise). __neg__ defines -p (every coefficient negated) and __sub__ defines p - q as p + (-q). times(number) multiplies every coefficient by a number or a sympy expression; multiplying by 0 returns the empty polynomial.

```
    def __mul__(self, other):
        terms = {}
        for k1, c1 in self.terms.items():
            for k2, c2 in other.terms.items():
                sign = 1
                if self.odd:
                    if set(k1) & set(k2):  # a generator twice: theta theta = 0
                        continue
                    swaps = sum(1 for p in k1 for q in k2 if p > q)
                    sign = -1 if swaps % 2 else 1  # sorting costs (-1)^swaps
                key = tuple(sorted(k1 + k2))
                c = sign * c1 * c2
                terms[key] = terms[key] + c if key in terms else c
        return Jet(terms, self.odd)
```

__mul__ defines p * q: every monomial of p is multiplied by every monomial of q. For commuting generators the product monomial is the sorted joined tuple. For Grassmann generators two more rules apply. If the two monomials share a generator, the product is zero ($\theta\theta = 0$) and is skipped (set(k1) & set(k2) is the set of common generators). Otherwise the joined tuple must be sorted, and each exchange of two anticommuting generators costs a sign: the number of exchanges needed is the number of pairs (p, q) with p from the first and q from the second monomial and $p > q$, and the sign is $(-1)^{\text{swaps}}$.

```
    def derivative(self, number):
        """The left derivative with respect to the generator number."""
        terms = {}
        for key, c in self.terms.items():
            if number not in key:
                continue
            place = key.index(number)
            rest = key[:place] + key[place + 1:]
            if self.odd:  # move the generator to the front first
                c = c if place % 2 == 0 else -c
            else:  # an ordinary power rule
                c = key.count(number) * c
            terms[rest] = terms[rest] + c if rest in terms else c
        return Jet(terms, self.odd)
```

`derivative(number)` is the **left derivative** with respect to one generator: in each monomial that contains it, the generator is removed. For commuting generators this is the power rule (the coefficient is multiplied by the number of times the generator occurs). For Grassmann generators the generator is first moved to the front of the monomial, which passes the `place` generators before it and costs $(-1)^{\text{place}}$; then it is removed. The left derivative is the one used for the Euler-Lagrange expressions of a Grassmann field.

```
    def substitute(self, rule):
        """Replace symbols in every coefficient (rule: {old: new})."""
        return Jet({key: c.subs(rule) for key, c in self.terms.items()}, self.odd)

    def is_zero(self):
        """True when every coefficient is exactly zero."""
        for c in self.terms.values():
            e = sp.expand(c)
            if e != 0 and sp.simplify(e) != 0:
                return False
        return True


def zero(odd):
    return Jet({}, odd)


def all_zero(items):
    return all(item.is_zero() for item in items)
```

`substitute` replaces symbols in every coefficient (a helper that the later cells do not need). `is_zero` decides whether a polynomial is zero: every coefficient is first expanded (`sp.expand` multiplies out products and powers) and, if that does not give 0, simplified (`sp.simplify` uses identities such as $\sin^2 + \cos^2 = 1$). If any coefficient is not zero, the answer is False. `zero(odd)` is the empty polynomial and `all_zero(items)` tests a whole list.

In [8], matrices acting on columns of polynomials, and the jet.

```
def mat_vec(M, v, odd):
    """The matrix M times the column v of algebra elements."""
```

```
    out = []
    for A in range(16):
        acc = zero(odd)
        for B in range(16):
            if M[A, B] != 0:
                acc = acc + v[B].times(M[A, B])
        out.append(acc)
    return out


def row_mat(v, M, odd):
    """The row v of algebra elements times the matrix M."""
    out = []
    for B in range(16):
        acc = zero(odd)
        for A in range(16):
            if M[A, B] != 0:
                acc = acc + v[A].times(M[A, B])
        out.append(acc)
    return out


def dot(u, v, odd):
    """The row u times the column v (the order of the factors is kept)."""
    acc = zero(odd)
    for p, q in zip(u, v):
        if p.terms and q.terms:
            acc = acc + p * q
    return acc
```

A spinor at one point is a column of 16 polynomials. `mat_vec(M, v)` is the matrix M times the column v: component A is $\sum_B M_{AB} v_B$ (only the nonzero entries are used). `row_mat(v, M)` is the row v times M: component B is $\sum_A v_A M_{AB}$. `dot(u, v)` is the row u times the column v, $\sum_A u_A v_A$, with the factor of u always written LEFT of the factor of v; this order matters for Grassmann generators, and it is the order of the bilinears $\Psi^\dagger X \Psi$.

```
def number_psi_star(A):
    return A


def number_psi(A):
    return 16 + A


def number_dpsi_star(mu, A):
    return 32 + 16 * mu + A


def number_dpsi(mu, A):
    return 160 + 16 * mu + A


def jets(odd):
```

```
    """The first jet (Psi^*, Psi, d Psi^*, d Psi) as algebra generators."""
    return ([Jet.symbol(number_psi_star(A), odd) for A in range(16)],
            [Jet.symbol(number_psi(A), odd) for A in range(16)],
            {mu: [Jet.symbol(number_dpsi_star(mu, A), odd) for A in range(16)]
             for mu in range(8)},
            {mu: [Jet.symbol(number_dpsi(mu, A), odd) for A in range(16)]
             for mu in range(8)})
```

The numbering of the 288 generators: Ψ^*_A is number A ($A = 0, \ldots, 15$; Python counts from 0), Ψ_A is $16+A$, $\partial_\mu \Psi^*_A$ is $32+16\mu+A$ and $\partial_\mu \Psi_A$ is $160+16\mu+A$, for $\mu = 0, \ldots, 7$ (the coordinates $x_0, \ldots, x_7$). `jets(odd)` returns the jet as four parts: the column Ψ^*, the column Ψ, and two dictionaries of columns for the derivatives.

```
NO_REFLECTION = {mu: 1 for mu in range(8)}
MIRROR = {mu: (-1 if mu == 0 else 1) for mu in range(8)}  # x0 -> const - x0


def transform(J, P, signs, odd):
    """The jet of Psi'(x) = P Psi(R x)."""
    psi_star, psi, dpsi_star, dpsi = J
    Pc = P.conjugate()
    return (mat_vec(Pc, psi_star, odd), mat_vec(P, psi, odd),
            {mu: [x.times(signs[mu]) for x in mat_vec(Pc, dpsi_star[mu], odd)]
             for mu in range(8)},
            {mu: [x.times(signs[mu]) for x in mat_vec(P, dpsi[mu], odd)]
             for mu in range(8)})
```

`NO_REFLECTION` and `MIRROR` hold the signs of a coordinate reflection R: all $+1$, or -1 for x_0 only (the mirror $x_0 \to \pi/(6H) - x_0$). `transform(J, P, signs)` returns the jet of the new field $\Psi'(x) = P\Psi(Rx)$: the components are multiplied by P, their conjugates by P^* (`P.conjugate()`, the matrix of conjugate entries; for the real matrices used here it equals P), and each derivative along x_μ also gets the sign of the reflection of x_μ, by the chain rule.

In [9], the theory on the jet.

```
def covariant(geo, J, odd):
    """D_mu Psi and D_mu Psibar for mu = 1..8."""
    psi_star, psi, dpsi_star, dpsi = J
    Dpsi = {mu: [p + q for p, q in zip(dpsi[mu], mat_vec(geo.Om[mu], psi, odd))]
            for mu in range(8)}
    Dbar = {mu: [p - q for p, q in zip(row_mat(dpsi_star[mu], C, odd),
                                       row_mat(psi_star, C * geo.Om[mu], odd))]
            for mu in range(8)}
    return Dpsi, Dbar
```

`covariant` returns, for each μ, the columns $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and the rows $D_\mu\bar\Psi = (\partial_\mu\Psi^*)^T C - \Psi^{*T} C \Omega_\mu$; the second is $\partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$ written with $\bar\Psi = \Psi^\dagger C$ (the row of the conjugates times C).

```
def scalar(J, odd):
    """S = Psibar Psi = Psi^dagger C Psi."""
    return dot(J[0], mat_vec(C, J[1], odd), odd)
```

$S = \Psi^\dagger C \Psi$: the row of conjugates times the column $C\Psi$.

```
def lagrangian(geo, J, odd, mass, coup, upoly=None, density=True):
    """L = sqrt|g| [ K - m S - U(S) ] (density=False: without sqrt|g|)."""
    Dpsi, Dbar = covariant(geo, J, odd)
    kin = zero(odd)
    for mu in range(8):
        kin = kin + dot(J[0], mat_vec(C * geo.gup[mu], Dpsi[mu], odd), odd)
        kin = kin - dot(Dbar[mu], mat_vec(geo.gup[mu], J[1], odd), odd)
    S = scalar(J, odd)
    L = kin.times(sp.Rational(1, 2)) - S.times(mass)
    if upoly is None:  # U = (lambda/2) S^2
        L = L - (S * S).times(coup / 2)
    else:  # U = u1 S + u2 S^2 + u3 S^3
        L = L - S.times(upoly[1]) - (S * S).times(upoly[2])
        L = L - (S * S * S).times(upoly[3])
    return L.times(geo.sqrtg) if density else L
```

The Lagrangian of Section 18.4, term by term: `kin` adds, for each μ, $\bar\Psi\gamma^\mu D_\mu\Psi$ (the row $\Psi^\dagger$ times $C\gamma^\mu D_\mu\Psi$) and subtracts $(D_\mu\bar\Psi)\gamma^\mu\Psi$. Then $\mathcal{L}/\sqrt{|g|} = \frac{1}{2}K_{\rm sum} - mS - U(S)$, with $U = \frac{\lambda}{2}S^2$, or, when the argument `upoly` is given, the cubic $U = u_1 S + u_2 S^2 + u_3 S^3$. With `density=True` the result is multiplied by $\sqrt{|g|}$.

```
def field_operator(geo, J, odd, mass, coup):
    """E = gamma^mu D_mu Psi - (m + lambda S) Psi (16 algebra elements)."""
    Dpsi, _ = covariant(geo, J, odd)
    S = scalar(J, odd)
    out = [zero(odd) for _ in range(16)]
    for mu in range(8):
        out = [p + q for p, q in zip(out, mat_vec(geo.gup[mu], Dpsi[mu], odd))]
    return [out[A] - J[1][A].times(mass) - (S * J[1][A]).times(coup)
            for A in range(16)]
```

`field_operator` returns the 16 components of $E = \gamma^\mu D_\mu\Psi - (m + \lambda S)\Psi$: the sum over μ of $\gamma^\mu D_\mu\Psi$, minus $m\Psi_A$, minus $\lambda S\Psi_A$.

```
def emt(geo, J, odd, mass, coup):
    """T_mu nu for mu <= nu (36 components)."""
    Dpsi, Dbar = covariant(geo, J, odd)
    L = lagrangian(geo, J, odd, mass, coup, density=False)
    T = {}
    for mu in range(8):
        for nu in range(mu, 8):
            t = (dot(J[0], mat_vec(C * geo.glow[mu], Dpsi[nu], odd), odd)
                 + dot(J[0], mat_vec(C * geo.glow[nu], Dpsi[mu], odd), odd)
                 - dot(Dbar[mu], mat_vec(geo.glow[nu], J[1], odd), odd)
                 - dot(Dbar[nu], mat_vec(geo.glow[mu], J[1], odd), odd))
            t = t.times(sp.Rational(1, 4))
            if mu == nu:
                t = t - L.times(geo.g[mu])
            T[mu, nu] = t
    return T
```

`emt` returns the 36 components $T_{\mu\nu}$ with $\mu \le \nu$ of the tensor of Section 18.4: the four kinetic bilinears with the lowered gammas, times $\frac{1}{4}$, and, on the diagonal, minus $g_{\mu\mu}\mathcal{L}/\sqrt{|g|}$ (the off-diagonal $g_{\mu\nu}$ vanish).

```
def current(geo, J, odd):
    """J^mu = i Psibar gamma^mu Psi (8 components)."""
    return {mu: dot(J[0], mat_vec(sp.I * C * geo.gup[mu], J[1], odd), odd)
            for mu in range(8)}
```

`current` returns the 8 components $J^\mu = i\bar{\Psi}\gamma^\mu\Psi$ (the convention of the sympy pairing report; `sp.I` is i).

```
def euler_lagrange(geo, J, odd, mass, coup):
    """dL/dPsi^*_A - sum_mu d_mu (dL/d(d_mu Psi^*_A)), A = 0..15."""
    L = lagrangian(geo, J, odd, mass, coup)
    out = []
    for A in range(16):
        e = L.derivative(number_psi_star(A))
        for mu in range(8):
            q = L.derivative(number_dpsi_star(mu, A))  # c Psi_B, linear in Psi
            for key, c in q.terms.items():
                B = key[0] - 16  # the component Psi_B of this term
                e = e - Jet({key: d(c, mu)}, odd)  # (d_mu c) Psi_B
                e = e - Jet({(number_dpsi(mu, B),): c}, odd)  # c d_mu Psi_B
        out.append(e)
    return out
```

`euler_lagrange` derives the Euler-Lagrange expressions from the Lagrangian itself, without using the formula for E: for each A, the derivative of $\mathcal{L}$ with respect to the generator Ψ^*_A, minus $\sum_\mu \partial_\mu$ of the derivative with respect to $\partial_\mu\Psi^*_A$. That second derivative, `q`, is a sum of terms $c\,\Psi_B$ with coefficient functions c of the point (the kinetic term contains $\partial_\mu\Psi^*$ only multiplied by Ψ). Its total derivative is computed with the product rule, $\partial_\mu(c\Psi_B) = (\partial_\mu c)\Psi_B + c\,\partial_\mu\Psi_B$: the first piece keeps the monomial and differentiates the coefficient; the second replaces Ψ_B by the generator of $\partial_\mu\Psi_B$.

In [10], theorem T1 for dirac16complex00.

```
SAMPLE = {z: sp.Rational(7, 10), H: sp.Rational(13, 10), m: sp.Rational(11, 10),
          lam: sp.Rational(3, 5)}  # a sample point for counting only
SAMPLE_A4 = {"a4p": sp.Rational(9, 10), "a4": sp.Rational(2, 5)}


def count_nonzero(element):
    """The number of monomials whose coefficient is not zero at the sample."""
    n = 0
    for c in element.terms.values():
        c = c.subs(sp.Derivative(a4, x4), SAMPLE_A4["a4p"])  # a4' first,
        c = c.subs(a4, SAMPLE_A4["a4"]).subs(SAMPLE)  # then a4 and the rest
        value = complex(sp.N(c, 30))
        n += abs(value) > 1e-20
    return n
```

```
COUNTS = {}  # the monomial counts for the figures
```

Counting the monomials of a polynomial whose coefficients are long expressions is slow if every coefficient is simplified. `count_nonzero` evaluates each coefficient at one sample point (the fractions in `SAMPLE` and `SAMPLE_A4`: $z = 0.7$, $H = 1.3$, $m = 1.1$, $\lambda = 0.6$, $a_4' = 0.9$, $a_4 = 0.4$) with 30 digits (`sp.N(c, 30)`) and counts those that are not zero. The count only describes the size of a polynomial; it is not part of any proof. `COUNTS` keeps the counts for the figures.

```
def t1_checks(odd):
    stat = "grassmann" if odd else "commuting"
    field = "dirac16complex" if odd else "dirac16complex00"
    tag = f"primordial_{stat}"
    J = jets(odd)
    JG = transform(J, GAM, NO_REFLECTION, odd)
    s_ok = (scalar(JG, odd) - scalar(J, odd)).is_zero()
    L_image = lagrangian(patch, JG, odd, m, lam)
    L_plain = lagrangian(patch, J, odd, m, lam)
    L_partner = lagrangian(patch, J, odd, -m, -lam)
    L_wrong = lagrangian(patch, J, odd, -m, lam)
    n_L = count_nonzero(L_plain)
    COUNTS[stat] = {"L": n_L, "L[Gamma Psi]": count_nonzero(L_image),
                    "T1 sum": count_nonzero(L_image + L_partner),
                    "control (m, l)": count_nonzero(L_image + L_plain),
                    "control (-m, l)": count_nonzero(L_image + L_wrong)}
    wl_count = f"L has {n_L} monomials" in recorded(WL, f"T1_Lagrangian_{tag}")[
        "detail"]
    report(f"{field}: monomials of L with a nonzero coefficient", n_L)
    reproduces(s_ok and (L_image + L_partner).is_zero() and wl_count,
               f"T1 {field}: S kept, L_m,l[Gamma Psi] = -L_-m,-l[Psi]",
               (PY, [f"T1.metric.{stat}.S_invariant", f"T1.metric.{stat}.lagrangian"]),
               (WL, [f"T1_scalar_and_kinetic_{tag}", f"T1_Lagrangian_{tag}"]))
```

`t1_checks(odd)` runs the whole T1 test for one statistics. `stat`, `field` and `tag` are the names used in the record's check names. `J` is the jet and `JG` the jet of $\Gamma\Psi$. `s_ok` tests $S[\Gamma\Psi] - S[\Psi] = 0$. Then four Lagrangians: of the image with (m, λ), of Ψ with (m, λ), with $(-m, -\lambda)$ (the T1 partner) and with $(-m, \lambda)$ (a wrong partner). `n_L` is the number of monomials of $\mathcal{L}_{m,\lambda}[\Psi]$, and `COUNTS[stat]` stores the counts of five polynomials for Figure 18b.2. `wl_count` checks that the detail text of the Wolfram check `T1_Lagrangian_primordial_<stat>` contains the same number ("L has 408 monomials"). The first check requires S kept, $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,-\lambda}[\Psi] = 0$ in all coefficients, and the agreement of the count.

```
    controls = (not (L_image + L_plain).is_zero()
                and not (L_image + L_wrong).is_zero())
    reproduces(controls, f"T1 {field}: the negative controls fail as they must",
               (PY, [f"T1.metric.{stat}.negative_controls"]),
               (WL, [f"T1_Lagrangian_negative_controls_{tag}"]))
```

The negative controls: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{m,\lambda}[\Psi]$ and $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,\lambda}[\Psi]$ must NOT be zero.

```
    E = field_operator(patch, J, odd, m, lam)
    CE = mat_vec(C, E, odd)
    EL = euler_lagrange(patch, J, odd, m, lam)
    derived = all_zero([EL[A] - CE[A].times(patch.sqrtg) for A in range(16)])
    E_image = field_operator(patch, JG, odd, m, lam)
    E_partner = mat_vec(GAM, field_operator(patch, J, odd, -m, -lam), odd)
    mapped = all_zero([E_image[A] + E_partner[A] for A in range(16)])
    reproduces(derived and mapped,
               f"T1 {field}: EL = sqrt|g| C E; E_m,l[Gamma Psi] = -Gamma E_-m,-l",
               (PY, [f"T1.metric.{stat}.euler_lagrange_derived",
                     f"T1.metric.{stat}.euler_lagrange_map"]),
               (WL, [f"T1_Euler_Lagrange_derived_{tag}",
                     f"T1_field_equation_covariance_{tag}"]))
```

`E` is the field operator, `CE` is C times it, and `EL` the Euler-Lagrange expressions derived from $\mathcal{L}$. `derived` checks $\mathrm{EL}_A = \sqrt{|g|}\,(CE)_A$ for all 16 A: the field equation $E = 0$ IS the Euler-Lagrange equation of this Lagrangian. `mapped` checks $E_{m,\lambda}[\Gamma\Psi] + \Gamma E_{-m,-\lambda}[\Psi] = 0$, statement (T1b).

```
    T_image = emt(patch, JG, odd, m, lam)
    T_partner = emt(patch, J, odd, -m, -lam)
    T_plain = emt(patch, J, odd, m, lam)
    T_pair = emt(patch, JG, odd, -m, -lam)
    emt_ok = all_zero([T_image[k] + T_partner[k] for k in T_image])
    pair_ok = all_zero([T_plain[k] + T_pair[k] for k in T_plain])
    COUNTS[stat]["T"] = {k: count_nonzero(T_plain[k]) for k in T_plain}
    COUNTS[stat]["T1 sign"] = {k: -1 for k in T_plain if emt_ok}
    J_image, J_plain = current(patch, JG, odd), current(patch, J, odd)
    current_ok = all_zero([J_image[mu] + J_plain[mu] for mu in range(8)])
    reproduces(emt_ok and pair_ok and current_ok and len(T_image) == 36,
               f"T1 {field}: T -> -T (36), pair T sum 0, J -> -J (8)",
               (PY, [f"T1.metric.{stat}.emt", f"T1.metric.{stat}.pair_total_emt_zero",
                     f"T1.metric.{stat}.current"]),
               (WL, [f"T1_energy_momentum_{tag}", f"T1_current_{tag}"]))
```

Four tensors: of the image with (m, λ), of Ψ with $(-m, -\lambda)$, of Ψ with (m, λ) and of the image with $(-m, -\lambda)$. `emt_ok` checks $T[\Gamma\Psi; m, \lambda] + T[\Psi; -m, -\lambda] = 0$ for all 36 components (T1c, read from the other side), and `pair_ok` checks the pair statement $T[\Psi; m, \lambda] + T[\Gamma\Psi; -m, -\lambda] = 0$ (T1d). The counts of the 36 components and the proved signs -1 are stored for Figure 18b.3. `current_ok` checks $J[\Gamma\Psi] + J[\Psi] = 0$ for the 8 components. One check combines them, with three sympy and two Wolfram records.

```
t1_checks(False)  # dirac16complex00: commuting components
```

The test is run for commuting components. Out [10] prints RESULT dirac16complex00: monomials of L with a nonzero coefficient = 408 and four PASS lines.

In [11], theorem T1 for dirac16complex.

```
t1_checks(True)  # dirac16complex: Grassmann components
say("monomials of L with a nonzero coefficient: commuting "
    f"{COUNTS['commuting']['L']}, Grassmann {COUNTS['grassmann']['L']}")
```

The same test with Grassmann generators, then a line with both counts. Out [11]: 392 monomials and four PASS lines; the last line reads commuting 408, Grassmann 392. Why fewer: in S^2, products in which the same Grassmann generator occurs twice vanish ($\theta\theta = 0$), so some monomials of the commuting case are absent.

In [12], a general potential and a random gravitational field.

```
u1, u2, u3 = sp.symbols("u1 u2 u3", real=True)
cubic_ok = True
for odd in (False, True):
    J = jets(odd)
    JG = transform(J, GAM, NO_REFLECTION, odd)
    L1 = lagrangian(patch, JG, odd, m, 0, upoly={1: u1, 2: u2, 3: u3})
    L2 = lagrangian(patch, J, odd, -m, 0, upoly={1: -u1, 2: -u2, 3: -u3})
    cubic_ok = cubic_ok and (L1 + L2).is_zero()
reproduces(cubic_ok, "T1 with a general cubic potential: L_m,U[Gamma Psi] = "
           "-L_-m,-U[Psi]", (PY, ["T1.metric.commuting.general_potential",
                                  "T1.metric.grassmann.general_potential"]))
```

Three symbols u_1, u_2, u_3 for a general cubic potential. For both statistics: the Lagrangian of the image with (m, U) (the coupling argument is 0 and unused when `upoly` is given) plus the Lagrangian of Ψ with $(-m, -U)$, where $-U$ has the coefficients $-u_1, -u_2, -u_3$, must vanish. This is (T1a) for a general potential.

```
rng = random.Random(20261001)  # the seed of the Revision report


def fraction():
    return sp.Rational(rng.randint(-9, 9), rng.randint(1, 5))


class RandomField:
    """A general gravitational field at one point (only what L and E use)."""


field = RandomField()
E8 = [[fraction() for _ in range(8)] for _ in range(8)]  # e^mu_a, not diagonal
field.gup = {mu: sum((E8[mu][a] * G[a] for a in range(8)), sp.zeros(16))
             for mu in range(8)}
field.Om = {}
for mu in range(8):
    M = sp.zeros(16)
    for a in range(8):
        for b in range(a + 1, 8):
            M += fraction() * SAB[a, b]  # an antisymmetric omega
    field.Om[mu] = M
field.sqrtg = abs(fraction()) + 1
```

`random.Random(20261001)` is a random-number generator with the seed number used by the Revision report (the instance drawn here is the notebook's own). `fraction()` returns a random fraction p/q with p from -9 to 9 and q from 1 to 5. `RandomField` is an empty class that serves

as a container. The field gets 64 random entries $e^{\mu}{}_{a}$ (a non-diagonal vielbein; `field.gup[mu]` is $\gamma^{\mu} = \sum_a e^{\mu}{}_{a}\gamma^{(a)}$), for each μ a connection $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$ with 28 random independent components (antisymmetry is built in by summing over $a < b$ only), and a positive volume factor. It is not the author's field and not a solution of anything: it stands for an arbitrary gravitational field at one point.

```
random_ok = True
for odd in (False, True):
    J = jets(odd)
    JG = transform(J, GAM, NO_REFLECTION, odd)
    L_sum = (lagrangian(field, JG, odd, m, lam)
             + lagrangian(field, J, odd, -m, -lam))
    E1 = field_operator(field, JG, odd, m, lam)
    E2 = mat_vec(GAM, field_operator(field, J, odd, -m, -lam), odd)
    random_ok = random_ok and all_zero([L_sum] + [E1[A] + E2[A] for A in range(16)])
reproduces(random_ok, "T1 in a random general field (seed 20261001), both fields",
           (PY, ["T1.general_field.random_instance.commuting",
                 "T1.general_field.random_instance.grassmann"]))
```

For both statistics: the T1 identity of the Lagrangian and the map of the 16 components of the field operator must hold in this random field. This shows, by an example with no special structure, why T1 holds in every gravitational field: the proof never used the form of the coefficients. Out [12] shows two PASS lines.

In [13], Figure 18b.2: what cancels.

```
labels = ["L", "L[Gamma Psi]", "T1 sum", "control (m, 1)", "control (-m, 1)"]
texts = ["$\\mathcal{L}_{m,\\lambda}[\\Psi]$",
         "$\\mathcal{L}_{m,\\lambda}[\\Gamma\\Psi]$",
         "T1: $+\\,\\mathcal{L}_{-m,-\\lambda}[\\Psi]$",
         "control: $+\\,\\mathcal{L}_{m,\\lambda}[\\Psi]$",
         "control: $+\\,\\mathcal{L}_{-m,\\lambda}[\\Psi]$"]
fig, ax = plt.subplots(figsize=(9.0, 4.6))
xs = np.arange(len(labels))
for shift, stat, colour in [(-0.18, "commuting", "#2a78d6"),
                            (0.18, "grassmann", "#eb6834")]:
    values = [COUNTS[stat][k] for k in labels]
    ax.bar(xs + shift, values, width=0.36, color=colour,
           label="commuting (dirac16complex00)" if stat == "commuting"
           else "Grassmann (dirac16complex)")
    for x, v in zip(xs + shift, values):
        ax.text(x, v + 6, str(v), ha="center", fontsize=8)
ax.set_xticks(xs, texts, fontsize=8.5, rotation=12)
ax.set_ylabel("monomials with a nonzero coefficient")
ax.set_ylim(0, 480)
ax.legend(loc="upper right")
save_figure(fig, "lagrangian_cancellation",
            "What cancels in theorem T1, in the author's metric with an arbitrary "
            "deflating history. Vertical axis: the number of monomials in the 288 "
            "jet values whose coefficient is not zero. The first two bars of each "
            "colour: the Lagrangian of a field and of its chirality image, "
            "$408$ monomials for commuting components and $392$ for Grassmann "
```

```
            "components. Third: the T1 sum with the reversed mass and coupling, "
            "exactly zero. Fourth and fifth, the negative controls: with the "
            "same mass and coupling the mass term and the $S^2$ term survive; "
            "with only the mass reversed the $S^2$ term survives. So both $m$ and "
            "$\\lambda$ must change sign.")
```

Five groups of two bars (commuting in blue, Grassmann in orange; `xs + shift` moves the two bars of a group apart), with the number written above each bar. **What you see in Figure 18b.2:** 408 and 392 monomials for $\mathcal{L}_{m,\lambda}[\Psi]$ and the same numbers for $\mathcal{L}_{m,\lambda}[\Gamma\Psi]$; zero for the T1 sum; and nonzero bars for the two negative controls. In the first control only the mass term and the S^2 term survive (the kinetic terms cancel, the potential terms add), in the second only the S^2 term. Why: Γ reverses exactly the kinetic part, so a cancellation needs both $m \to -m$ and $\lambda \to -\lambda$.

In [14], Figure 18b.3: the 36 components of the tensor.

```
counts = np.zeros((8, 8))
signs = np.zeros((8, 8))
for (mu, nu), n in COUNTS["commuting"]["T"].items():
    counts[mu, nu] = counts[nu, mu] = n
for (mu, nu), s in COUNTS["commuting"]["T1 sign"].items():
    signs[mu, nu] = signs[nu, mu] = s
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.6))
image = axes[0].imshow(counts, cmap="Blues")
fig.colorbar(image, ax=axes[0], shrink=0.85, label="monomials")
axes[1].imshow(signs, cmap="coolwarm", vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        axes[0].text(j, i, str(int(counts[i, j])), ha="center", va="center",
                     fontsize=7, color="black" if counts[i, j] < 200 else "white")
        axes[1].text(j, i, f"{int(signs[i, j]):+d}", ha="center", va="center",
                     fontsize=8, color="white")
for ax, title in [(axes[0], "monomials of $T_{\\mu\\nu}[\\Psi; m, \\lambda]$"),
                  (axes[1], "sign of the T1 partner's $T_{\\mu\\nu}$")]:
    ax.set_xticks(range(8), [f"$x_{a}$" for a in range(8)])
    ax.set_yticks(range(8), [f"$x_{a}$" for a in range(8)])
    ax.grid(False)
    ax.set_title(title, fontsize=10)
```

`counts` and `signs` are 8×8 arrays filled symmetrically from the 36 stored components ($T_{\nu\mu} = T_{\mu\nu}$). The left panel draws the counts with a blue colour scale and writes each number in its cell (white text on dark cells); the right panel draws the proved signs. The ticks name the coordinates.

```
save_figure(fig, "emt_signs",
            "The 36 independent components of the energy-momentum tensor in the "
            "author's metric, row index $\\mu$ and column index $\\nu$ from $x_0$ "
            "to $x_7$. Left: the number of jet monomials in each component for a "
            "field with commuting components; the diagonal components carry the "
            "whole Lagrangian, the off-diagonal ones only kinetic terms. Right: "
            "the sign that relates the component of the T1 partner "
            "$\\Gamma\\Psi$ with $(-m, -\\lambda)$ to that of $\\Psi$ with "
            "$(m, \\lambda)$, proved to be $-1$ everywhere: energy density, "
```

```
            "momentum densities and pressures are all reversed, and the pair has "
            "a zero total.")
```

What you see in Figure 18b.3: on the left, 376 monomials in each diagonal component (those components contain the whole Lagrangian through $-g_{\mu\mu}\mathcal{L}/\sqrt{|g|}$) and 64 or 80 in each off-diagonal one (only kinetic bilinears); on the right, -1 in every one of the 64 cells. The T1 partner carries exactly the opposite energy density, momentum densities and pressures.

In [15], the mirror patch.

```
mirror = Geometry(-1)  # the mirror patch pi/2 < z < pi, f0 = -cot z > 0
to_mirror = {z: sp.pi - z}


class Reflected:
    """The patch's coefficient functions evaluated at the mirror point pi - z."""


reflected = Reflected()
reflected.sqrtg = sp.expand(patch.sqrtg.subs(to_mirror))
reflected.g = {mu: sp.expand(patch.g[mu].subs(to_mirror)) for mu in range(8)}
for name in ("Om", "gup", "glow"):
    setattr(reflected, name, {mu: getattr(patch, name)[mu].subs(to_mirror)
                              .applyfunc(sp.expand) for mu in range(8)})
```

`mirror` is the geometry of the mirror patch ($s_0 = -1$, so $f_0 = -\cot z > 0$ there). `to_mirror` is the substitution $z \to \pi - z$. `reflected` holds the patch's coefficient functions evaluated at the mirror point: $\sqrt{|g|}$, the metric, Ω_μ and both kinds of curved gammas, each with z replaced by $\pi - z$ and expanded (`setattr(object, name, value)` stores a value under a name given as a string).

```
isometry = all(sp.simplify(patch.g[mu].subs(to_mirror) - patch.g[mu]) == 0
               for mu in range(8))
degenerate = (sp.limit(patch.g[0], z, sp.pi / 2) == 0
              and sp.limit(patch.sqrtg, z, sp.pi / 2) == 0)
reproduces(isometry and degenerate, "z -> pi - z is an isometry; the brane "
           "z = pi/2 is degenerate (g00 = 0, sqrt|g| = 0)",
           (PY, ["geometry.mirror_isometry", "geometry.brane_degenerate"]),
           (WL, ["T2_mirror_is_isometry"]))
```

`isometry` checks $g_{\mu\mu}(\pi - z) = g_{\mu\mu}(z)$ for all eight components; `degenerate` checks with limits (`sp.limit`) that $g_{00} \to 0$ and $\sqrt{|g|} \to 0$ as $z \to \pi/2$. One check, three records.

```
mirror_postulate = all(
    sp.simplify(mirror.om[mu, a, b] + mirror.om[mu, b, a]) == 0
    for mu, a, b in itertools.product(range(8), repeat=3))
for mu, nu in itertools.product(range(8), repeat=2):
    M = mirror.gup[nu].applyfunc(lambda e: d(e, mu))
    for la in range(8):
        if mirror.chr[nu, mu, la] != 0:
            M = M + mirror.chr[nu, mu, la] * mirror.gup[la]
```

```
    M = M + mirror.Om[mu] * mirror.gup[nu] - mirror.gup[nu] * mirror.Om[mu]
    mirror_postulate = mirror_postulate and all(sp.simplify(e) == 0 for e in M)
say(f"sqrt|g| on the mirror patch = {mirror.sqrtg} (positive there, cos z < 0)")
reproduces(mirror_postulate, "the mirror patch's own connection is canonical",
           (WL, ["connection_mirror_patch"]))
```

The mirror patch's own connection is antisymmetric and obeys the vielbein postulate (the same test as In [4]). Out [15] prints $\sqrt{|g|} = -\cos z$ on the mirror patch, positive there because $\cos z < 0$ for $\pi/2 < z < \pi$, and two PASS lines.

In [16], Figure 18b.4: the metric on both patches.

```
zs = np.linspace(0.02, np.pi - 0.02, 400)
fig, ax = plt.subplots(figsize=(8.0, 4.4))
ax.plot(zs, np.sin(zs) ** (1.0 / 3.0), color="#2a78d6",
        label="$\\sin^{1/3}z$ (factor of $g_{11}$ and $-g_{55}$)")
ax.plot(zs, 1.0 / np.tan(zs) ** 2, color="#eb6834", label="$g_{00} = \\cot^2 z$")
ax.plot(zs, np.abs(np.cos(zs)), color="#1baf7a", label="$\\sqrt{|g|} = |\\cos z|$")
ax.axvline(np.pi / 2, color="#52514e", linestyle="--")
ax.text(np.pi / 2 + 0.04, 2.6, "brane $z = \\pi/2$", fontsize=9)
ax.axvspan(0.0, np.pi / 2, color="#2a78d6", alpha=0.05)
ax.text(0.25, 2.6, "patch", fontsize=9)
ax.text(2.35, 2.6, "mirror patch", fontsize=9)
ax.set_xlim(0.0, np.pi)
ax.set_ylim(0.0, 3.0)
ax.set_xlabel("hidden angle $z = 6Hx_0$ (radians)")
ax.set_ylabel("metric function (no unit)")
ax.legend(loc="center left", bbox_to_anchor=(0.02, 0.62), fontsize=8.5)
save_figure(fig, "mirror_metric",
            "The functions of the hidden angle that enter the author's metric, "
            "against $z = 6Hx_0$ from 0 to $\\pi$ (horizontal axis, radians; "
            "vertical axis, pure numbers): $\\sin^{1/3}z$, the factor of the "
            "3-space and extra-time components; $g_{00} = \\cot^2 z$; and "
            "$\\sqrt{|g|} = |\\cos z|$. The shaded half is the author's patch, the "
            "other half its mirror patch. Every curve is mirror-symmetric about "
            "the dashed brane $z = \\pi/2$, so $z \\to \\pi - z$ is an isometry. "
            "At the brane $g_{00}$ and $\\sqrt{|g|}$ vanish: the metric is "
            "degenerate there, and gluing the two patches is an assumption.")
```

400 values of z from just above 0 to just below π (the end points are avoided because $\cot^2 z$ is infinite there). Three curves: $\sin^{1/3} z$, $g_{00} = \cot^2 z$ (`1.0 / np.tan(zs) ** 2`) and $|\cos z|$; a dashed vertical line at the brane, a faint shading of the patch and three labels. **What you see in Figure 18b.4:** every curve is mirror-symmetric about $z = \pi/2$, so $z \to \pi - z$ is an isometry; at the brane $\sin^{1/3} z$ reaches its maximum 1 while g_{00} and $\sqrt{|g|}$ touch zero: the metric is degenerate there, which is why gluing the two patches is an assumption and not a result.

In [17], which matrix makes the mirror a pairing.

```
CANDIDATES = [("1", I16), ("Gamma", GAM), ("gamma^(x0)", G[0]),
              ("Gamma gamma^(x0)", GAM * G[0])]
FOUND = {}
```

```
for odd in (False, True):
    J = jets(odd)
    for name, P in CANDIDATES:
        JP = transform(J, P, MIRROR, odd)
        L_mirror = lagrangian(mirror, JP, odd, m, lam)
        hit = None
        for sigma, sm, sl in itertools.product((1, -1), repeat=3):
            L_patch = lagrangian(reflected, J, odd, sm * m, sl * lam)
            if (L_mirror - L_patch.times(sigma)).is_zero():
                hit = (sigma, sm, sl)
                break
        FOUND[odd, name] = hit
```

Four candidate matrices: 1, Γ, $\gamma^{(x_0)}$ and $\Gamma\gamma^{(x_0)} = P_0$. For each statistics and each candidate, `JP` is the jet of $\Psi'(x) = P\Psi(Rx)$ with the mirror reflection, and `L_mirror` its Lagrangian with (m, λ) in the mirror geometry. The inner loop tries the eight sign choices (σ, s_m, s_λ) and compares with $\sigma\,\mathcal{L}_{s_m m, s_\lambda \lambda}[\Psi]$ in the reflected patch geometry; `break` stops at the first relation that holds exactly. `FOUND` stores the result (or `None`).

```
for name, _ in CANDIDATES:
    hit = FOUND[False, name]
    text = "no relation" if hit is None else (
        f"L = {'+' if hit[0] > 0 else '-'}L_({'' if hit[1] > 0 else '-'}m, "
        f"{'' if hit[2] > 0 else '-'}lambda)")
    say(f"P = {name}: {text} (the same for Grassmann components: "
        f"{FOUND[True, name] == hit})")
```

For each candidate the cell prints the relation found for commuting components and whether the Grassmann result is the same (the nested f-string writes the signs as + or − and leaves out a + in front of m and λ).

```
expected = {"1": None, "Gamma": None, "gamma^(x0)": (1, -1, 1),
            "Gamma gamma^(x0)": (-1, 1, -1)}
reproduces(all(FOUND[odd, name] == expected[name]
               for odd in (False, True) for name, _ in CANDIDATES),
           "T2: only gamma^(x0) maps (m, lambda) to (-m, lambda) with L -> +L",
           (PY, ["T2.metric.commuting.reflection_table",
                 "T2.metric.grassmann.reflection_table"]),
           (WL, ["T2_mirror_Lagrangian_commuting", "T2_mirror_Lagrangian_grassmann"]))
```

The expected results of the sympy record: none for 1 and Γ; $(+1, -1, +1)$, that is $+\mathcal{L}_{-m,\lambda}$, for $\gamma^{(x_0)}$; $(-1, +1, -1)$, that is $-\mathcal{L}_{m,-\lambda}$, for P_0. Out [17] shows exactly these four lines and the PASS line.

In [18], T2 completed.

```
t2_ok = True
T2_SIGNS = {}
for odd in (False, True):
    J = jets(odd)
    JP = transform(J, G[0], MIRROR, odd)
```

```
    E1 = field_operator(mirror, JP, odd, m, lam)
    E2 = mat_vec(G[0], field_operator(reflected, J, odd, -m, lam), odd)
    el_ok = all_zero([E1[A] + E2[A] for A in range(16)])
    T1_ = emt(mirror, JP, odd, m, lam)
    T2_ = emt(reflected, J, odd, -m, lam)
    emt_ok = all_zero([T1_[k] - T2_[k].times(MIRROR[k[0]] * MIRROR[k[1]])
                       for k in T1_])
    if not odd:
        T2_SIGNS = {k: MIRROR[k[0]] * MIRROR[k[1]] for k in T1_ if emt_ok}
    J1, J2 = current(mirror, JP, odd), current(reflected, J, odd)
    current_ok = all_zero([J1[mu] - J2[mu].times(MIRROR[mu]) for mu in range(8)])
    s_odd = (scalar(JP, odd) + scalar(J, odd)).is_zero()
    stat = "grassmann" if odd else "commuting"
    say(f"{stat}: field equations mapped {el_ok}, T pulled back {emt_ok}, "
        f"J pulled back {current_ok}, S reversed {s_odd}")
    t2_ok = t2_ok and el_ok and emt_ok and current_ok and s_odd
```

For both statistics with $P = \gamma^{(x_0)}$ and the mirror reflection: `el_ok` checks $E^{\mathrm{mirror}}_{m,\lambda}[\Psi'] + \gamma^{(x_0)} E^{\mathrm{patch}}_{-m,\lambda}[\Psi] = 0$ (T2b); `emt_ok` checks $T_{\mu\nu}[\Psi'] = \Lambda_\mu \Lambda_\nu T_{\mu\nu}[\Psi; -m, \lambda]$ with $\Lambda =$ `MIRROR` (T2c), and for commuting components the signs $\Lambda_\mu \Lambda_\nu$ are stored for the figure; `current_ok` checks $J'^\mu = \Lambda_\mu J^\mu$; `s_odd` checks $S[\gamma^{(x_0)}\Psi] + S[\Psi] = 0$. The cell prints the four results for each statistics.

```
reproduces(t2_ok, "T2: field equations mapped, T and J pulled back (equal energy), "
           "S reversed", (PY, [f"T2.metric.{s}.{c}" for s in ("commuting", "grassmann")
                               for c in ("euler_lagrange_map", "emt", "current",
                                         "S_odd")]),
           (WL, ["T2_mirror_energy_momentum_and_current_commuting",
                 "T2_mirror_energy_momentum_and_current_grassmann",
                 "T2_mirror_Euler_Lagrange_commuting",
                 "T2_mirror_Euler_Lagrange_grassmann"]))
```

One check with eight sympy and four Wolfram records (the list comprehension builds the eight sympy names). Out [18] prints True eight times and the PASS line.

In [19], Figure 18b.5: the two summaries of T2.

```
choices = list(itertools.product((1, -1), repeat=3))
grid = np.zeros((len(CANDIDATES), len(choices)))
for i, (name, _) in enumerate(CANDIDATES):
    if FOUND[False, name] is not None:
        grid[i, choices.index(FOUND[False, name])] = 1.0
t2_signs = np.zeros((8, 8))
for (mu, nu), s in T2_SIGNS.items():
    t2_signs[mu, nu] = t2_signs[nu, mu] = s
```

`choices` lists the eight sign choices in the order of the search. `grid` has one row per candidate and one column per choice, with 1 where a relation was found (`choices.index` finds the column). `t2_signs` is the 8×8 array of the stored signs $\Lambda_\mu \Lambda_\nu$.

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4),
                         gridspec_kw={"width_ratios": [1.4, 1.0]})
```

```
axes[0].imshow(grid, cmap="Greens", vmin=0, vmax=1.4, aspect="auto")
axes[0].set_xticks(range(len(choices)),
                   [f"{'+' if s > 0 else '-'}L\n{'' if a > 0 else '-'}m\n"
                    f"{'' if b > 0 else '-'}$\\lambda$" for s, a, b in choices],
                   fontsize=8)
axes[0].set_yticks(range(len(CANDIDATES)),
                   ["$P = 1$", "$P = \\Gamma$", "$P = \\gamma^{(x_0)}$",
                    "$P = \\Gamma\\gamma^{(x_0)}$"])
axes[0].set_title("relation $\\mathcal{L}^{mirror}_{m,\\lambda}[P\\Psi] = "
                  "\\sigma\\mathcal{L}_{\\pm m,\\pm\\lambda}[\\Psi]$", fontsize=10)
axes[0].grid(False)
axes[1].imshow(t2_signs, cmap="coolwarm", vmin=-1, vmax=1)
for i in range(8):
    for j in range(8):
        axes[1].text(j, i, f"{int(t2_signs[i, j]):+d}", ha="center", va="center",
                     fontsize=8, color="white")
axes[1].set_xticks(range(8), [f"$x_{a}$" for a in range(8)])
axes[1].set_yticks(range(8), [f"$x_{a}$" for a in range(8)])
axes[1].set_title("sign of the mirror image's $T_{\\mu\\nu}$", fontsize=10)
axes[1].grid(False)
```

Two panels of different widths (`width_ratios`). The left one draws `grid` in shades of green, with the sign choices as three-line column labels (`\n` starts a new line); the right one draws the signs with their values written in the cells.

```
save_figure(fig, "mirror_candidates",
            "Theorem T2 in the author's metric. Left: for four candidate matrices "
            "$P$ (rows), the eight possible relations between the Lagrangian of "
            "the mirror image on the mirror patch and the Lagrangian of $\\Psi$ on "
            "the patch (columns: overall sign $\\sigma$, sign of $m$, sign of "
            "$\\lambda$); a green square marks the relation that holds exactly. "
            "$P = 1$ and $P = \\Gamma$ give none; $P = \\gamma^{(x_0)}$ gives "
            "$+\\mathcal{L}_{-m,\\lambda}$, theorem T2; $P = \\Gamma\\gamma^{(x_0)}$ "
            "gives $-\\mathcal{L}_{m,-\\lambda}$. Right: the sign of each component "
            "of the mirror image's $T_{\\mu\\nu}$ relative to that of the "
            "$(-m, \\lambda)$ field, $+1$ except the mixed components with one "
            "$x_0$ index: the energy density is equal, not opposite.")
```

What you see in Figure 18b.5: on the left, the rows of 1 and Γ are empty, the row of $\gamma^{(x_0)}$ has its green square under $+\mathcal{L}$, $-m$, $+\lambda$, and the row of P_0 under $-\mathcal{L}$, $+m$, $-\lambda$; on the right, $+1$ everywhere except the 14 cells of the row and the column x_0 off the diagonal (the seven mixed components with one index x_0, drawn twice), where the sign is -1. The energy density ($T_{x_4x_4}$, sign $+1$) of the mirror partner is EQUAL to that of the field it mirrors.

In [20], the last check.

```
figure_names = ["connection_history", "lagrangian_cancellation", "emt_signs",
                "mirror_metric", "mirror_candidates"]
paths = [output_file(f"{FIGURE_FOLDER}/18b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
```

```
      "every figure file of this notebook exists")
all_checks_passed()
```

The five figure files must exist; the last line is ALL 20 CHECKS PASSED (notebook 18b).

What Notebook 18b established. PROVED, as exact identities in the 288 jet values, for an arbitrary history $a_4(x_4)$ and for both statistics: the canonical connection and $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$; T1 in the author's metric (Lagrangian, field equations derived from the Lagrangian, all 36 components of T, all 8 of J, the pair totals, the negative controls), for a general cubic potential and in a random general field; T2 across the mirror, with the search that singles out $\gamma^{(x_0)}$. ASSUMED: the gluing at the degenerate brane. NOT shown: any process that creates a universe; the identities map solutions to solutions.

18.21 A family of exact solutions: the fields that depend only on the time

The theorems speak about every solution. To SEE them at work we need actual solutions, and the field equation of the author's metric has a family of exact ones: the **homogeneous** fields, which depend on the time x_4 only. We derive them line by line for the commuting field dirac16complex00, whose components are ordinary complex numbers (the Revision record also gives the Grassmann version). The family is derived on the background $b_4 = 1$; the end of the next paragraphs says what a changing b_4 does to it.

The field equation becomes an ordinary differential equation. For $\Psi = \Psi(x_4)$ every derivative except ∂_4 vanishes, and $\gamma^{x_4} = \gamma^{(x_4)}/f_4 = \gamma^{(x_4)}$. The spin connection still acts, through $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ (Section 18.3). With $V = m + \lambda S$ the field equation of Section 18.4 is therefore

$$\gamma^{(x_4)}\frac{d\Psi}{dx_4} + 3H\gamma^{(x_0)}\Psi = V\Psi.$$

Subtract $3H\gamma^{(x_0)}\Psi$ and multiply on the left by $-\gamma^{(x_4)}$; since $\gamma^{(x_4)}\gamma^{(x_4)} = \eta_{44} = -1$, the left side becomes $d\Psi/dx_4$:

$$\frac{d\Psi}{dx_4} = M\Psi, \qquad M = -\gamma^{(x_4)}\big(V - 3H\gamma^{(x_0)}\big) = -V\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}.$$

S is constant. M is real (V is real because $S = \Psi^\dagger C\Psi$ is real for a real symmetric C, and the gammas are real), so $M^\dagger = M^T$. By the product rule,

$$\frac{dS}{dx_4} = \Big(\frac{d\Psi}{dx_4}\Big)^\dagger C\Psi + \Psi^\dagger C\frac{d\Psi}{dx_4} = \Psi^\dagger\big(M^TC + CM\big)\Psi.$$

Now $(\gamma^{(x_4)})^T = -\gamma^{(x_4)}$ and $(\gamma^{(x_0)})^T = \gamma^{(x_0)}$, and from $C\gamma^{(a)}C^{-1} = -(\gamma^{(a)})^T$: C commutes with $\gamma^{(x_4)}$ and anticommutes with $\gamma^{(x_0)}$. Hence

$$M^T = V\gamma^{(x_4)} + 3H(\gamma^{(x_0)})^T(\gamma^{(x_4)})^T = V\gamma^{(x_4)} - 3H\gamma^{(x_0)}\gamma^{(x_4)} = V\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}.$$

The transpose of a product reverses the order; then the two transpose rules; then $\gamma^{(x_0)}\gamma^{(x_4)} = -\gamma^{(x_4)}\gamma^{(x_0)}$.

$$M^TC + CM = VC\gamma^{(x_4)} - 3HC\gamma^{(x_4)}\gamma^{(x_0)} - VC\gamma^{(x_4)} + 3HC\gamma^{(x_4)}\gamma^{(x_0)} = 0.$$

We moved C to the left in M^TC: past $\gamma^{(x_4)}$ without a sign and past $\gamma^{(x_0)}$ with a sign. So $dS/dx_4 = 0$: S keeps its initial value S_0, and $V = m + \lambda S_0$ and M are constant along the solution (`python-field-theory.json`, check `exact_nonlinear_homogeneous_solution`).

The square of M. Move the second $\gamma^{(x_4)}$ to the left through $(V - 3H\gamma^{(x_0)})$, which turns it into $(V + 3H\gamma^{(x_0)})$:

$$M^2 = \gamma^{(x_4)}(V-3H\gamma^{(x_0)})\gamma^{(x_4)}(V-3H\gamma^{(x_0)}) = \gamma^{(x_4)}\gamma^{(x_4)}(V+3H\gamma^{(x_0)})(V-3H\gamma^{(x_0)}) = -(V^2-9H^2).$$

The two signs in front of $\gamma^{(x_4)}$ cancel; then $\gamma^{(x_4)}\gamma^{(x_4)} = -1$ and $(\gamma^{(x_0)})^2 = 1$ in the product of the brackets. So $M^2 = (9H^2 - V^2)I_{16}$.

The exact solution. When $V^2 > 9H^2$ put $w = \sqrt{V^2 - 9H^2}$, so that $M^2 = -w^2$. Then

$$\Psi(x_4) = \cos(wx_4)\,\Psi(0) + \frac{\sin(wx_4)}{w}\,M\Psi(0).$$

Check: the derivative is $-w\sin(wx_4)\Psi(0) + \cos(wx_4)M\Psi(0)$, and $M\Psi(x_4) = \cos(wx_4)M\Psi(0) + \frac{\sin(wx_4)}{w}M^2\Psi(0)$ is the same, because $M^2 = -w^2$. At $x_4 = 0$ the formula gives $\Psi(0)$. The solution **oscillates** with the frequency w. When $V^2 < 9H^2$, cosine and sine become cosh and sinh and the solution **grows** exponentially: inside the window $|m + \lambda S_0| < 3H$ these modes are not oscillations but growth (`python-scope.json`, check `good_sector_x0_independent_modes_without_boundary_condition`; Chapter 8). PROVED: `python-field-theory.json`, check `exact_nonlinear_homogeneous_solution`; `wolfram-field-theory.json`, check `exact_solution_nonlinear_homogeneous_C`. The history a_4 does not enter M, but it enters the energy-momentum tensor through the vielbein factors $e^{\pm a_4}$ and through the spin connection (Notebook 18c, Figure 18c.3).

With a changing b_4. In the metric with $b_4(x_4)$ the homogeneous equation gains one term: $d\Psi/dx_4 = M\Psi - \frac{3}{4}(b_4'/b_4)\Psi$ (`wolfram-field-theory-b4.json`, check `evolution_form_C`, which states that the extra term is the factor $b_4^{-3/4}$). Writing $\Psi = b_4^{-3/4}\Phi$ removes it: Φ obeys $d\Phi/dx_4 = M\Phi$, where M contains $V = m + \lambda S[\Psi] = m + \lambda b_4^{-3/2}S[\Phi]$. The argument above still gives $dS[\Phi]/dx_4 = 0$, because it never used that V is constant. So for $\lambda = 0$ the solutions are $b_4^{-3/4}$ times those of $b_4 = 1$; for $\lambda \neq 0$ the coefficient V changes with b_4, the matrices M at different times do not commute, and this book does not solve that case. (DERIVED here from the recorded equation; there is no record check of this paragraph of its own.)

What the theory record says about these solutions. For homogeneous solutions, on shell, the energy density is $\rho = mS + \frac{\lambda}{2}S^2$, every pressure (3-space, extra times, hidden direction) is $p = \frac{\lambda}{2}S^2$, and the mixed component $T_{x_4x_0}$ vanishes (`python-field-theory.json`, checks `commuting_homogeneous_on_shell_rho_p` and `commuting_T_x4x0_homogeneous`). The charge density $J^{x_4} = \Psi^\dagger B\Psi$ is NOT constant: the conservation law $\partial_4 J^{x_4} + \frac{1}{\sqrt{|g|}}\partial_0(\sqrt{|g|}J^{x_0}) = 0$ holds with a flow along the hidden direction. For a homogeneous field $J^{x_0} = -i\bar\Psi\gamma^{(x_0)}\Psi/f_0 = \tan z\, q_0$ with the **frame current** $q_0 = -i\bar\Psi\gamma^{(x_0)}\Psi$ (a number that depends on x_4 only; it is NOT the charge Q), and $\sqrt{|g|}J^{x_0} = \cos z \tan z\, q_0 = \sin z\, q_0$, so

$$\frac{1}{\sqrt{|g|}}\partial_0(\sqrt{|g|}J^{x_0}) = \frac{6H\cos z\, q_0}{\cos z} = 6Hq_0, \qquad \frac{dJ^{x_4}}{dx_4} = -6Hq_0.$$

The derivative $\partial_0 = 6H\, d/dz$ acts only on $\sin z$, because q_0 does not depend on x_0; the second equation is the conservation law solved for $\partial_4 J^{x_4}$. Notebook 18c checks this law along its solution (`python-field-theory.json`, check `commuting_current_conservation`).

The local law does not make the total charge constant. The law above is LOCAL: it says that charge can leave a small region only by flowing through its surface. The total charge of the patch is constant only if nothing flows through the boundary of the patch. For a homogeneous field take a box of coordinate volume V_6 in the six directions $x_1, x_2, x_3, x_5, x_6, x_7$ (for a homogeneous field the current J^μ is the same at every point of the slice, so whatever flows in through one face of the box in these directions flows out through the opposite face: these faces add nothing), and the whole patch $0 < z < \pi/2$ in x_0. Its charge is

$$Q(x_4) = \int \cos z\, J^{x_4}\, d^7x = V_6\, J^{x_4}(x_4) \int_0^{\pi/(12H)} \cos(6Hx_0)\, dx_0 = V_6\, J^{x_4}(x_4)\, \frac{1}{6H}.$$

J^{x_4} does not depend on the seven coordinates of the slice, so it comes out of the integral; the integral over the six directions of the box gives V_6; the substitution $z = 6Hx_0$, $dx_0 = dz/(6H)$, gives $\frac{1}{6H}\int_0^{\pi/2}\cos z\, dz = \frac{1}{6H}[\sin z]_0^{\pi/2} = \frac{1}{6H}$.

$$\frac{dQ}{dx_4} = \frac{V_6}{6H}\,\frac{dJ^{x_4}}{dx_4} = \frac{V_6}{6H}\,(-6Hq_0) = -V_6\, q_0.$$

We differentiated the line before (V_6 and H are constants) and inserted the law $dJ^{x_4}/dx_4 = -6Hq_0$. The same result comes from integrating the flow: $\int_0^{\pi/(12H)} \partial_0(\sin z\, q_0)\, dx_0 = [\sin z\, q_0]_{z=0}^{z=\pi/2} = q_0$, because $\sin z = 0$ at the tip $z = 0$ and $\sin z = 1$ at the brane $z = \pi/2$. So the charge of the patch changes at the rate $-V_6 q_0$, which is the charge flowing out through the brane. Notebook 18c prints the range of q_0 along its solution, from -0.3454 to 0.3454 (Out [10]): it is not zero, and the charge density J^{x_4} of Figure 18c.3 oscillates accordingly. The Revision record states the same boundary term: the flux $\sin z\, u^\dagger M_0 v$ "vanishes at the tip z -> 0 but not at the patch end z = pi/2" (`python-scope.json`, check `good_sector_hermiticity_up_to_the_brane_flux`). The T2 mirror partner has the EQUAL charge density at every time (T2c), so the charge of the mirror patch changes at the same rate, and the two glued patches together change at twice the rate: gluing does not stop the flow. A constant total charge needs a **no-flux condition** at the brane, $\sqrt{|g|}J^{x_0} = 0$ at $z = \pi/2$, that is $q_0 = 0$ for these solutions. No such condition is derived in this book: it would be part of the junction condition at the brane, which is OPEN (Section 18.28, item 7).

What T1 and T2 predict for this family. T1: if Ψ solves the equation with (m, λ), then $\Gamma\Psi$ solves it with $(-m, -\lambda)$. Directly: Γ anticommutes with $\gamma^{(x_4)}$ and commutes with $\gamma^{(x_4)}\gamma^{(x_0)}$, and $S[\Gamma\Psi] = S$, so $\Gamma M_{m,\lambda}\Gamma = +V\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)} = M_{-m,-\lambda}$: the partner's generator is the similar matrix $\Gamma M\Gamma$, with the same eigenvalues. T2: on the mirror patch the connection term is $-3H\gamma^{(x_0)}$ (Section 18.16), so the equation there has $M^{\text{mirror}} = -V\gamma^{(x_4)} - 3H\gamma^{(x_4)}\gamma^{(x_0)}$; with $S[\gamma^{(x_0)}\Psi] = -S$ and the parameters $(-m, \lambda)$, $V \to -m - \lambda S = -V$, and $\gamma^{(x_0)}M_{m,\lambda}\gamma^{(x_0)} = V\gamma^{(x_4)} - 3H\gamma^{(x_4)}\gamma^{(x_0)}$, which is exactly $M^{\text{mirror}}_{-m,\lambda}$. So $\gamma^{(x_0)}\Psi$ solves the mirror equation with $(-m, \lambda)$. (We used $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)} = -\gamma^{(x_4)}$ and $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_0)} = -\gamma^{(x_4)}\gamma^{(x_0)}$.) Notebook 18c checks both by solving the partner equations independently.

18.22 The quantum reading Q

From fields to quantum fields, in one paragraph. Chapter 10 quantised dirac16complex canonically, with x_4 as the time. The recipe reads off the term of the Lagrangian that contains the time derivative. With $\gamma^{x_4} = \gamma^{(x_4)}$ and $C\gamma^{(x_4)} = iB$ (Notebook 18a, In [5]), the time-derivative part of K is

$$\frac{1}{2}(\Psi^\dagger C\gamma^{(x_4)}\partial_4\Psi - \partial_4\Psi^\dagger C\gamma^{(x_4)}\Psi) = \frac{i}{2}(\Psi^\dagger B\partial_4\Psi - \partial_4\Psi^\dagger B\Psi),$$

so the Lagrangian contains $\frac{i}{2}\sqrt{|g|}\,\Psi^\dagger N\partial_4\Psi$ (plus its partner term) with the **velocity kernel** $N = B$. The canonical rule then fixes the anticommutator of the field operators on a slice $x_4 = \text{const}$: the momentum is $\pi = i\sqrt{|g|}\Psi^\dagger N$, and $\{\Psi_A(x), \pi_B(y)\} = i\delta_{AB}\delta^7(x-y)$ forces

$$\{\Psi_A(x), \Psi_B^\dagger(y)\} = (N^{-1})_{AB}\,\frac{\delta^7(x-y)}{\sqrt{|g|}} = B_{AB}\,\frac{\delta^7(x-y)}{\sqrt{|g|}},$$

because $N^{-1} = B^{-1} = B$ ($BB = 1$). Here $\{X, Y\} = XY + YX$ is the anticommutator of two operators and δ^7 the delta function of the seven coordinates other than x_4. Because B has eight eigenvalues $+1$ and eight -1, the state space carries an indefinite **Krein** form when $\Psi^\dagger$ is read as the ordinary adjoint (Chapter 10).

statement	status	where it is verified
the canonical anticommutator is $B\,\delta^7/\sqrt{\lvert g\rvert}$	PROVED	`python-field-theory.json`, check `canonical_anticommutator_B`; `python-pairing.json`, check `Q.canonical_anticommutator`

One-particle waves. In flat 4+4 space ($H = 0$, a_4 constant, all $f_\mu = 1$) and for $\lambda = 0$, a plane wave $\Psi = u\,e^{-iwx_4 + i\sum_{a\neq 4} k_a x_a}$ turns the field equation $\sum_a \gamma^{(a)}\partial_a\Psi = m\Psi$ into $-iw\gamma^{(x_4)}u + i\sum_{a\neq 4} k_a\gamma^{(a)}u = mu$. Multiply on the left by $-i\gamma^{(x_4)}$ and use $\gamma^{(x_4)}\gamma^{(x_4)} = -1$:

$$wu = h_m(k)u, \qquad h_m(k) = -im\gamma^{(x_4)} - \gamma^{(x_4)}\sum_{a\neq x_4} k_a\gamma^{(a)}.$$

The frequencies w are the eigenvalues of the **one-particle matrix** $h_m(k)$. Its square: write $h = -\gamma^{(x_4)}(im + K)$ with $K = \sum_{a\neq 4} k_a\gamma^{(a)}$; moving $\gamma^{(x_4)}$ through $(im + K)$ turns it into $(im - K)$, so

$$\begin{aligned} h^2 &= \gamma^{(x_4)}\gamma^{(x_4)}(im - K)(im + K) = -(-m^2 - K^2) = m^2 + K^2 \\ &= (m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2)I_{16}. \end{aligned}$$

$K^2 = \sum_a k_a^2\eta_{aa}$ by the Clifford relation (the mixed products cancel in pairs). So $h^2 = w^2 I_{16}$ with $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$: real frequencies when $w^2 > 0$; imaginary ones, growing modes, when the extra-time momentum is large enough to make $w^2 < 0$.

Hypotheses.

- (HQ1) dirac16complex is canonically quantised with x_4 as the evolution time and the anticommutator fixed by the velocity kernel of its Lagrangian, as above.
- (HQ2) dirac16complex00 is a classical field and has no quantum reading.

The statements.

- (Q1) The chirality image $\chi = \Gamma\Psi$ carries the Krein matrix $-B$: $\{\chi_A, \chi_B^\dagger\} = -B_{AB}\,\delta^7/\sqrt{|g|}$. This is also the anticommutator demanded by its own Lagrangian $\mathcal{L}_{m,\lambda}[\Gamma\chi] = -\mathcal{L}_{-m,-\lambda}[\chi]$.
- (Q2) The image's own generators (energy, momentum and charge from its own Lagrangian) coincide with those of Ψ: the pair Ψ, $\Gamma\Psi$ is ONE quantum system, and the T1 identity $T^{(-m,-\lambda)}[\Gamma\Psi] = -T^{(m,\lambda)}[\Psi]$ is an identity between operators of that one system.
- (Q3) An independently quantised universe with $(-m, -\lambda)$ has the anticommutator $+B$ (from its own Lagrangian $\mathcal{L}_{-m,-\lambda}$), its own vacuum and its own generators. It cannot be identified with $\Gamma\Psi$. On the product state space the generators of two independent universes add, $P_{\text{total}} = P_1 \otimes 1 + 1 \otimes P_2$, and no cancellation $P_1 + P_2 = 0$ follows.
- (Q4) The one-particle matrices satisfy $\Gamma h_m(k)\Gamma = h_{-m}(k)$ and $\gamma^{(x_0)} h_m(k) \gamma^{(x_0)} = h_{-m}(R_0 k)$ ($R_0 k$: the momentum with k_0 reversed): the $+m$ and $-m$ one-particle spectra are IDENTICAL, not opposite. For every real frequency each of the two eigenspaces (for $+w$ and for $-w$) has dimension 8 and Krein inertia (4,4), for $+m$ and $-m$ alike; for imaginary or zero frequency the eigenspaces are Krein-neutral (B vanishes on them). The same similarity holds in a general field at a point, with frozen coefficients.
- (Q5) The T2 image $\gamma^{(x_0)}\Psi$ keeps the anticommutator $+B$: the mirror $(-m, \lambda)$ universe is an ordinary, independently quantisable copy with equal energies.

Status: PROVED under (HQ1). Records: `wolfram-pairing.json`, the 12 checks of the Q group (names beginning with `Q_`); `python-pairing.json`, the 10 checks whose names begin with `Q.`; theorem Q of `pairing-theory.json`. Notebook 18a checks the Krein signs (In [17], In [18]); Notebook 18c checks Q4 and Q5 on plane waves (In [16] to In [20]). Chapter 10 treats the same quantum reading in its own Notebook 10g.

18.23 The proofs of Q1 to Q5

Q1. The field operators of the image are $\chi_A = \sum_C \Gamma_{AC}\Psi_C$ and $\chi_B^\dagger = \sum_D \Psi_D^\dagger \Gamma_{BD}$ (Γ is real). The anticommutator is linear in each argument, so

$$\{\chi_A, \chi_B^\dagger\} = \sum_{C,D} \Gamma_{AC}\{\Psi_C, \Psi_D^\dagger\}\Gamma_{BD} = \sum_{C,D} \Gamma_{AC} B_{CD} \Gamma_{DB}\,\frac{\delta^7}{\sqrt{|g|}} = (\Gamma B \Gamma)_{AB}\,\frac{\delta^7}{\sqrt{|g|}} = -B_{AB}\,\frac{\delta^7}{\sqrt{|g|}}.$$

We used $\Gamma_{BD} = \Gamma_{DB}$ (Γ symmetric) and Lemma 2, $\Gamma B \Gamma = -B$. For the image's own Lagrangian: written in terms of χ, the Lagrangian of the theory is $\mathcal{L}_{m,\lambda}[\Psi] = \mathcal{L}_{m,\lambda}[\Gamma\chi]$; its velocity term is

$\frac{i}{2}\sqrt{|g|}\,\chi^\dagger\Gamma B\Gamma\partial_4\chi = \frac{i}{2}\sqrt{|g|}\,\chi^\dagger(-B)\partial_4\chi$, so its velocity kernel is $N=-B$ and the canonical rule gives the anticommutator $N^{-1}=-B$. The two agree.

Q2. The image's own Lagrangian, evaluated at $\chi=\Gamma\Psi$, is $\mathcal{L}_{m,\lambda}[\Gamma\Gamma\Psi]=\mathcal{L}_{m,\lambda}[\Psi]$: the same function of the same operators. Its energy density (obtained by the Legendre transform: for each field, its time derivative times its canonical momentum, summed, minus the Lagrangian) and its Noether densities are therefore the same operators as those of Ψ. At the one-particle level: the energy of a plane wave is $\Psi^\dagger\mathcal{E}_m(k)\Psi$ with the **energy kernel** $\mathcal{E}_m(k)=Bh_m(k)$, and the evolution is $i\partial_4\Psi=$ (anticommutator matrix)(energy kernel)$\Psi=B\mathcal{E}_m\Psi=h_m\Psi$. The image's own Lagrangian is $-\mathcal{L}_{-m,-\lambda}[\chi]$, so its anticommutator matrix is $-B$ (Q1) and its energy kernel is $-\mathcal{E}_{-m}$; its evolution generator is

$$(-B)(-\mathcal{E}_{-m})=B\mathcal{E}_{-m}=h_{-m}=\Gamma h_m\Gamma,$$

which is exactly how $\Gamma\Psi$ must evolve: $i\partial_4(\Gamma\Psi)=\Gamma h_m\Psi=(\Gamma h_m\Gamma)(\Gamma\Psi)$. The sign of the Krein matrix and the sign of the energy kernel compensate: the image is the same quantum system relabelled.

Q3. An independent universe χ_2 with the Lagrangian $\mathcal{L}_{-m,-\lambda}[\chi_2]$ has the velocity kernel $+B$ and therefore $\{\chi_2,\chi_2^\dagger\}=+B\,\delta^7/\sqrt{|g|}$, while $\{\Gamma\Psi_1,(\Gamma\Psi_1)^\dagger\}=-B\,\delta^7/\sqrt{|g|}$. Since $B\neq -B$, $\chi_2=\Gamma\Psi_1$ is impossible. Independence also requires $\{\Psi_1,\chi_2^\dagger\}=0$, whereas $\{\Gamma\Psi_1,\Psi_1^\dagger\}=\Gamma B\,\delta^7/\sqrt{|g|}$ is a matrix of rank 16, not zero. On the product of the two state spaces the generators of the two systems add; the T1 identity relates operators of universe 1 only and gives no relation $P_2=-P_1$. At zero momentum the one-particle generator of the two universes together has the eigenvalues $-m$ and $+m$, 16 times each, all nonzero for $m\neq 0$: nothing cancels.

Q4. $\Gamma\gamma^{(x_4)}\Gamma=-\gamma^{(x_4)}$ (one gamma) and $\Gamma\gamma^{(x_4)}\gamma^{(a)}\Gamma=\gamma^{(x_4)}\gamma^{(a)}$ (two gammas), so

$$\Gamma h_m(k)\Gamma=+im\gamma^{(x_4)}-\gamma^{(x_4)}\sum_{a\neq 4}k_a\gamma^{(a)}=h_{-m}(k).$$

With $(\gamma^{(x_0)})^2=1$: $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)}=-\gamma^{(x_4)}$; for $a\neq 0,4$, $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(a)}\gamma^{(x_0)}=\gamma^{(x_4)}\gamma^{(a)}$ (two exchanges); and $\gamma^{(x_0)}\gamma^{(x_4)}\gamma^{(x_0)}\gamma^{(x_0)}=-\gamma^{(x_4)}\gamma^{(x_0)}$. So the mass term and the k_0 term change sign and the others do not: $\gamma^{(x_0)}h_m(k)\gamma^{(x_0)}=h_{-m}(R_0k)$. **Similar matrices have equal spectra**: if $hu=wu$, then $(MhM^{-1})(Mu)=w(Mu)$. So $+m$ and $-m$ have the same frequencies.

The Krein inertia, in three steps. (i) $Bh_m=h_m^\dagger B$; this follows from the reality of the gammas and the relations of C with $\gamma^{(x_4)}$ and $\gamma^{(a)}$ (Exercise 18.5 asks you to verify it). For a real $w>0$ the matrices $P_\pm=\frac{1}{2}(1\pm h/w)$ are the projectors onto the eigenspaces of $\pm w$ (because $h^2=w^2$), and

$$P_+^\dagger BP_-=\tfrac{1}{4}(1+h^\dagger/w)B(1-h/w)=\tfrac{1}{4}B(1+h/w)(1-h/w)=\tfrac{1}{4}B(1-h^2/w^2)=0.$$

We moved B to the left with $h^\dagger B=Bh$, then multiplied out; $h^2=w^2$. So the two eigenspaces are B-orthogonal, and B is nondegenerate on each (it is invertible on the whole space). (ii) Without extra-time momentum ($k_5=k_6=k_7=0$) B commutes with h, and $\operatorname{tr}B=\operatorname{tr}(Bh)=0$, so $\operatorname{tr}(BP_\pm)=0$: on each 8-dimensional eigenspace B is an involution with trace 0, which has four eigenvalues $+1$ and four -1, Krein inertia (4,4). (iii) The region $w^2>0$ is connected and contains the case without extra-time momentum (lowering k_5,k_6,k_7 to zero only increases w^2),

and the projectors change continuously with k; a nondegenerate form cannot change its inertia continuously, so (4,4) holds at every real frequency. For an imaginary w: if $hu = wu$ and $hv = wv$, then $w\,u^\dagger Bv = u^\dagger Bhv = u^\dagger h^\dagger Bv = \bar{w}\,u^\dagger Bv$ ($\bar{w}$ the complex conjugate), so $(w - \bar{w})u^\dagger Bv = 0$ and $u^\dagger Bv = 0$: the eigenspace is **Krein-neutral**. For $w = 0$, $h^2 = 0$ with rank 8 and the null space of h equals its range, also B-neutral. The records are listed in the table at the end of this section. The recorded samples, all reproduced by Notebook 18c (In [16]):

m	**momentum** $(k_0, \ldots, k_7)$	w	**dimensions of the $+w$ and $-w$ eigenspaces**	**Krein inertia on $+w$ and on $-w$**
± 2	(4, 1, 2, 0, 0, 0, 0, 0)	5	8 and 8	(4,4) and (4,4)
± 3	(0, 0, 0, 0, 0, 0, 0, 0)	3	8 and 8	(4,4) and (4,4)
± 1	(0, 1, 0, 0, 0, 0, 0, 0)	$\sqrt{2}$	8 and 8	(4,4) and (4,4)
± 2	(0, 0, 0, 0, 0, 1, 0, 0)	$\sqrt{3}$	8 and 8	(4,4) and (4,4)

(Each row stands for the two recorded samples with $+m$ and $-m$; the entry k_4 is not used. The last row has an extra-time momentum and still a real frequency: $w^2 = 4 - 1 = 3$.)

Q5. Lemma 4 gives $\gamma^{(x_0)} B (\gamma^{(x_0)})^\dagger = +B$, so the anticommutator of $\gamma^{(x_0)}\Psi$ is $+B\,\delta^7/\sqrt{|g|}$, the same as that of an ordinary field. QED.

statement	**status**	**where it is verified**
Q1: the T1 image carries $-B$, also by its own Lagrangian	PROVED	`wolfram-pairing.json`, checks `Q_Krein_metric_of_images`, `Q_symplectic_kernel_commuting` and `Q_symplectic_kernel_grassmann`; `python-pairing.json`, checks `Q.image_krein_metric` and `Q.image_own_quantisation`
Q2: the image's own generators are those of Ψ	PROVED	`wolfram-pairing.json`, checks `Q_generators_of_the_image_commuting` and `Q_generators_of_the_image_grassmann`; `python-pairing.json`, check `Q.image_generators_same_dynamics`
Q3: no identification and no cancellation of independent universes	PROVED	`wolfram-pairing.json`, check `Q_no_identification_of_independent_universes`; `python-pairing.json`, check `Q.no_cancellation_independent_universes`

statement	status	where it is verified
Q4: the maps of h_m and the dispersion $h^2 = w^2$	PROVED	`wolfram-pairing.json`, checks `Q_one_particle_flat_dispersion`, `Q_one_particle_maps` and `Q_one_particle_general_field`; `python-pairing.json`, check `Q.one_particle_maps`
Q4: Krein inertia (4,4) at every real frequency	PROVED	`wolfram-pairing.json`, checks `Q_one_particle_Krein_signatures` and `Q_one_particle_Krein_inertia_real_frequencies`; `python-pairing.json`, checks `Q.one_particle_Krein_inertia` and `Q.one_particle_Krein_inertia_proof`
Q4: imaginary and zero frequencies are Krein-neutral	PROVED	`wolfram-pairing.json`, check `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`; `python-pairing.json`, check `Q.one_particle_complex_frequency_Krein_neutral`
Q5: the T2 image keeps $+B$	PROVED	`wolfram-pairing.json`, check `Q_Krein_metric_of_images`; `python-pairing.json`, check `Q.T2_image_keeps_B`
the eight samples of the table above	PROVED (exact arithmetic of the record); COMPUTED again in floating point by Notebook 18c	data table `one_particle_flat` of `pairing-theory.json`; Notebook 18c, In [16]

What Q does not say. Q1 to Q5 are statements about the canonical anticommutator. They neither use nor establish a positive-norm state space for either universe. The theory record constructs, separately, a positive Fock representation in the good sector without extra-time momentum (Chapter 10); it is not used here. And the vanishing total energy-momentum and charge of a T1 pair holds for classical bilinears and as an operator identity within ONE quantum system; it does not hold for two independently quantised universes, whose generators add without cancelling.

18.24 Example: Notebook 18c sees the theorems in numbers

Notebook 18c takes one homogeneous solution of dirac16complex00 in the author's metric (Section 18.21), with values chosen as an ILLUSTRATION: units in which $m = 1$, $H = 0.2$, $\lambda = 0.3$, the

hidden angle $z_0 = \pi/4$, and an initial value of 16 complex numbers drawn with the fixed seed 2026. Only the history $a_4 = AHx_4$ with $A = 1$ is read from the Revision record `Revision/kohn_sham/results/parameters.json`. The notebook solves the equation with the classical Runge-Kutta method of fourth order and checks it against the exact formula; solves the T1 partner equation $(-m, -\lambda)$ and the mirror equation $(-m, \lambda)$ INDEPENDENTLY and finds $\Gamma\Psi$ and $\gamma^{(x_0)}\Psi$; computes S, all 8 components of J^μ and all 36 of $T_{\mu\nu}$ along the deflating history; compares the spectra of the evolution matrices; and, for the quantum reading, reproduces the plane-wave samples of the pairing record and follows the Krein norms of single eigenvectors under Γ and $\gamma^{(x_0)}$. Its last line is ALL 21 CHECKS PASSED (notebook 18c); it runs in about 30 seconds.

18.25 How to run Notebook 18c

Step 1. What this notebook does and what it needs.

Notebook 18c (T1 and T2 numerically: solutions, partners, currents and spectra) is the file `Revision/textbook/notebooks/18c_pairs_numerically.ipynb` of the repository Dirac_claude. It takes the exact family of solutions of the field equation of dirac16complex00 in the author's metric that depend on the time x4 only (recorded in the Revision theory reports), integrates one solution with the Runge-Kutta method and compares it with the exact formula, and then solves its two partners independently: the T1 partner with mass and coupling reversed, starting from Gamma times the initial value, and the T2 partner on the mirror patch beyond the brane, starting from gamma^(x0) times the initial value. It checks that the partners are exactly Gamma Psi and gamma^(x0) Psi, that the T1 partner has the opposite charge density, current and energy-momentum tensor (all 36 components, along the canonical deflating history a4 = A H x4), that the T2 partner has the same ones and the opposite scalar S, that the charge obeys its local conservation law (with a flow through the brane, so the total charge of the patch is not constant), and that the spectra of the partners are equal. For the quantum reading Q it reproduces the one-particle samples of the Revision pairing record (frequencies, eigenspace dimensions, Krein inertia (4,4), the Krein-neutral imaginary and zero frequencies) and shows that the chirality image reverses every Krein norm while the mirror image keeps it. Seven teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 18c_pairs_numerically.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 18c_pairs_numerically.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 18c_pairs_numerically.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/18c.captions.json`
- `Revision/textbook/figures/18c_1_rk4_convergence.png`
- `Revision/textbook/figures/18c_2_partner_components.png`
- `Revision/textbook/figures/18c_3_t1_pair_densities.png`
- `Revision/textbook/figures/18c_4_t2_mirror_densities.png`
- `Revision/textbook/figures/18c_5_family_spectrum.png`
- `Revision/textbook/figures/18c_6_flat_dispersion.png`

- `Revision/textbook/figures/18c_7_krein_norms.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 21 CHECKS PASSED (notebook 18c)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/18c_pairs_numerically.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 18c_pairs_numerically.ipynb
python -m nbconvert --execute --inplace 18c_pairs_numerically.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for gammas.json or one of the reports: the notebook reads nine files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository. Clone the repository again and open the notebook there.

18.26 Notebook 18c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/18c_pairs_numerically.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Notebook 18b proved the pairing theorems T1 and T2 as identities. This notebook
SEES them in numbers. It uses an exact family of solutions of the field equation
of dirac16complex00 (16 commuting complex components) in the author's metric: the
fields that depend on the time $x_4$ only. For such a field the 16 field equations
become 16 ordinary differential equations in $x_4$, which a computer can solve.

1. It solves one such field $\Psi$ (mass $m$, coupling $\lambda$) with the
   Runge-Kutta method and checks it against the exact formula of the Revision
   theory record.
2. **T1.** It solves, independently, the theory with $(-m, -\lambda)$ starting
   from $\Gamma\Psi(0)$, and finds exactly $\Gamma\Psi(x_4)$ at every time; it
   computes the charge density, the current and all 36 components of the
   energy-momentum tensor of both along the canonical deflating history
   $a_4 = AHx_4$ of the Revision record: they are exactly opposite.
3. **T2.** It solves, independently, the theory with $(-m, \lambda)$ on the
   mirror patch beyond the brane $z = \pi/2$, starting from
   $\gamma^{(x_0)}\Psi(0)$, and finds exactly $\gamma^{(x_0)}\Psi(x_4)$; the charge
   density and the energy density are EQUAL, the scalar $S$ is opposite.
4. **Spectra.** It shows that the evolution generators of a universe and of its
   partners have the same eigenvalues, and, for plane waves in flat 4+4 space (the
```

```
   quantum reading Q), it reproduces the one-particle samples of the Revision
   pairing record and shows how $\Gamma$ reverses every Krein norm while
   $\gamma^{(x_0)}$ keeps it.

Seven figures show the solutions, the partners, their densities and spectra.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 18c (T1 and T2 numerically: solutions, partners, currents and spectra) is the
file ‘Revision/textbook/notebooks/18c_pairs_numerically.ipynb‘ of the repository
Dirac_claude. It takes the exact family of solutions of the field equation of
dirac16complex00 in the author’s metric that depend on the time x4 only (recorded in the
Revision theory reports), integrates one solution with the Runge-Kutta method and
compares it with the exact formula, and then solves its two partners independently: the
T1 partner with mass and coupling reversed, starting from Gamma times the initial value,
and the T2 partner on the mirror patch beyond the brane, starting from gamma^(x0) times
the initial value. It checks that the partners are exactly Gamma Psi and gamma^(x0) Psi,
that the T1 partner has the opposite charge density, current and energy-momentum tensor
(all 36 components, along the canonical deflating history a4 = A H x4), that the T2
partner has the same ones and the opposite scalar S, that the charge obeys its local
conservation law (with a flow through the brane, so the total charge of the patch is not
constant), and that the spectra of the partners are equal. For the quantum reading Q it
reproduces the one-particle samples of the Revision pairing record (frequencies,
eigenspace dimensions, Krein inertia (4,4), the Krein-neutral imaginary and zero
frequencies) and shows that the chirality image reverses every Krein norm while the
mirror image keeps it. Seven teaching figures. It needs a computer with Windows 11, macOS
or Linux, an internet connection for the installation, the program Git, and Python 3.12
or newer (the notebooks were built with Python 3.14.5) with these packages at exactly
these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the
program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):
```

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 18c_pairs_numerically.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 18c_pairs_numerically.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 18c_pairs_numerically.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/18c.captions.json‘
- ‘Revision/textbook/figures/18c_1_rk4_convergence.png‘
- ‘Revision/textbook/figures/18c_2_partner_components.png‘
- ‘Revision/textbook/figures/18c_3_t1_pair_densities.png‘
- ‘Revision/textbook/figures/18c_4_t2_mirror_densities.png‘
- ‘Revision/textbook/figures/18c_5_family_spectrum.png‘
- ‘Revision/textbook/figures/18c_6_flat_dispersion.png‘
- ‘Revision/textbook/figures/18c_7_krein_norms.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 21 CHECKS PASSED (notebook 18c)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/18c_pairs_numerically.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 18c_pairs_numerically.ipynb
  python -m nbconvert --execute --inplace 18c_pairs_numerically.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" for gammas.json or one of the reports: the notebook reads nine files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository. Clone the repository again and open the notebook there.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 18c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 18c (T1 and T2 numerically: solutions, partners, currents and spectra) is the
# file Revision/textbook/notebooks/18c_pairs_numerically.ipynb of the repository
# Dirac_claude. It takes the exact family of solutions of the field equation of
# dirac16complex00 in the author's metric that depend on the time x4 only (recorded in
# the Revision theory reports), integrates one solution with the Runge-Kutta method and
# compares it with the exact formula, and then solves its two partners independently: the
# T1 partner with mass and coupling reversed, starting from Gamma times the initial
# value, and the T2 partner on the mirror patch beyond the brane, starting from
# gamma^(x0) times the initial value. It checks that the partners are exactly Gamma Psi
# and gamma^(x0) Psi, that the T1 partner has the opposite charge density, current and
# energy-momentum tensor (all 36 components, along the canonical deflating history a4 = A
# H x4), that the T2 partner has the same ones and the opposite scalar S, that the charge
# obeys its local conservation law (with a flow through the brane, so the total charge of
# the patch is not constant), and that the spectra of the partners are equal. For the
# quantum reading Q it reproduces the one-particle samples of the Revision pairing record
# (frequencies, eigenspace dimensions, Krein inertia (4,4), the Krein-neutral imaginary
# and zero frequencies) and shows that the chirality image reverses every Krein norm
# while the mirror image keeps it. Seven teaching figures. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
```

```
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

```
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 18c_pairs_numerically.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
```

```
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 18c_pairs_numerically.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 18c_pairs_numerically.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/18c.captions.json
# - Revision/textbook/figures/18c_1_rk4_convergence.png
# - Revision/textbook/figures/18c_2_partner_components.png
# - Revision/textbook/figures/18c_3_t1_pair_densities.png
# - Revision/textbook/figures/18c_4_t2_mirror_densities.png
# - Revision/textbook/figures/18c_5_family_spectrum.png
# - Revision/textbook/figures/18c_6_flat_dispersion.png
# - Revision/textbook/figures/18c_7_krein_norms.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 21 CHECKS PASSED (notebook 18c)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
```

```
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/18c_pairs_numerically.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 18c_pairs_numerically.ipynb
#     python -m nbconvert --execute --inplace 18c_pairs_numerically.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" for gammas.json or one of the reports: the notebook reads nine
#   files of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository. Clone the
#   repository again and open the notebook there.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

```
NOTEBOOK_ID = "18c"  # this notebook: chapter 18, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
```

```
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 18c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Ordinary differential equation (ODE)**: an equation for functions of ONE
  variable (here the time $x_4$) and their first derivatives.
- **Runge-Kutta method (RK4)**: a recipe that advances the solution of an ODE by
  a step $h$ using four evaluations of the right-hand side; its error after a fixed
  time shrinks like $h^4$ (*fourth order*).
- **Homogeneous field**: a field that does not depend on $x_1, x_2, x_3, x_5, x_6,
  x_7, x_0$, only on the time $x_4$.
- **Scalar** $S = \bar\Psi\Psi = \Psi^\dagger C\Psi$; **charge density**
  $J^{x_4} = \Psi^\dagger B\Psi$ with the Krein matrix $B = -iC\gamma^{(x_4)}$;
  **current** $J^\mu = -i\bar\Psi\gamma^\mu\Psi$ (the convention of the theory
  record, so that $J^{x_4}$ is the charge density).
- **Energy-momentum tensor** $T_{\mu\nu}$: 36 independent numbers at each point
  that say how much energy and momentum there is and how it flows; in the
  convention of the pairing record used here the energy density is
  $\rho = -T_{x_4x_4}$.
- **Deflating history**: $a_4 = AHx_4$ with $A = 1$ (the canonical choice of the
  Revision Kohn-Sham record): as $x_4$ grows, 3-space inflates by $e^{a_4}$ and
  the three extra times $x_5, x_6, x_7$ deflate by $e^{-a_4}$.
- **Patch, mirror patch, brane**: $z = 6Hx_0$ runs over the patch $0 < z <
  \pi/2$; the mirror patch $\pi/2 < z < \pi$ is its image under $z \to \pi - z$;
  the brane $z = \pi/2$ joins them (an ASSUMED construction).
- **Eigenvalue, eigenvector**: $Mu = wu$ with a number $w$ and a column $u \neq
  0$; the set of eigenvalues is the *spectrum* of $M$.
- **Similar matrices**: $N = PMP^{-1}$; they have the same spectrum.
- **Krein norm** of a column $u$: the real number $u^\dagger Bu$, which can be
  positive or negative because $B$ has eight eigenvalues $+1$ and eight $-1$.
- **Krein inertia** of a set of columns: how many independent directions have a
  positive and how many a negative Krein norm.
```

Text cell:

```
## 4. The physical and mathematical situation

**The field equation for homogeneous fields.** The field equation of both fields
is $\gamma^\mu D_\mu\Psi = (m + \lambda S)\Psi$. For a field that depends on $x_4$
only, every derivative except $\partial_4$ vanishes, $\gamma^{x_4} =
\gamma^{(x_4)}$ (the vielbein factor of $x_4$ is 1), and the spin connection
contributes $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ on the patch (the deflation
terms cancel). So the 16 field equations are EXACTLY

$$\gamma^{(x_4)}\frac{d\Psi}{dx_4} + 3H\gamma^{(x_0)}\Psi = (m + \lambda S)\Psi .$$

Multiply on the left by $-\gamma^{(x_4)}$ and use $(\gamma^{(x_4)})^2 = -1$:
```

```
$$\frac{d\Psi}{dx_4} = M\Psi,\qquad M = -\gamma^{(x_4)}\big((m + \lambda S) -
3H\gamma^{(x_0)}\big) = -(m + \lambda S)\gamma^{(x_4)} +
3H\gamma^{(x_4)}\gamma^{(x_0)} .$$

The Revision theory record proves that $S$ is constant along such a solution, so
$M$ is a constant matrix with $M^2 = \big(9H^2 - (m + \lambda S)^2\big)I_{16}$,
and the exact solution is

$$\Psi(x_4) = \cos(wx_4)\,\Psi(0) + \frac{\sin(wx_4)}{w}\,M\Psi(0),\qquad
w = \sqrt{(m + \lambda S)^2 - 9H^2}$$

when $(m + \lambda S)^2 > 9H^2$ (an oscillation; otherwise the cosine and sine
become cosh and sinh, a growth). The history $a_4$ does not enter this
equation, but it enters the energy-momentum tensor through the vielbein factors
$e^{\pm a_4}$ and the spin connection, as the figures will show. (The value
$3H\gamma^{(x_0)}$ belongs to the diagonal vielbein; $S$, $J^\mu$ and
$T_{\mu\nu}$, the quantities plotted here, do not depend on that choice.)

**The mirror patch.** On the mirror patch the hidden vielbein factor is
$-\cot z > 0$ and the spin connection term changes sign: $\gamma^\mu\Omega_\mu =
-3H\gamma^{(x_0)}$. The equation there is $d\Psi/dx_4 = -\gamma^{(x_4)}\big((m +
\lambda S) + 3H\gamma^{(x_0)}\big)\Psi$.

**What the theorems predict.** T1: if $\Psi$ solves the $(m, \lambda)$ equation,
$\Gamma\Psi$ solves the $(-m, -\lambda)$ equation, with $T \to -T$ and
$J \to -J$. T2: if $\Psi$ solves the $(m, \lambda)$ equation on the patch,
$\gamma^{(x_0)}\Psi$ (taken at the mirror point) solves the $(-m, \lambda)$
equation on the mirror patch, with $S \to -S$ and $T$, $J$ pulled back (the
components with one $x_0$ index change sign, all others are equal).

**Plane waves in flat 4+4 space.** For the quantum reading Q the Revision record
uses the one-particle matrix $h_m(k) = -im\gamma^{(x_4)} - \gamma^{(x_4)}
\sum_{a\neq x_4}k_a\gamma^{(a)}$ of a plane wave $e^{-iwx_4 + ik\cdot x}$; its
eigenvalues are the frequencies $w$, with $h_m^2 = (m^2 + k_0^2 + k_1^2 + k_2^2 +
k_3^2 - k_5^2 - k_6^2 - k_7^2)I_{16}$.
```

Text cell:

```
## 5. The matrices

The next cell imports numpy and sympy, defines the record helpers (‘reproduces‘
passes only when the notebook’s own result holds AND every named check of the
named reports has the verdict PASS), reads the author’s eight real gamma matrices
and builds $C$, $\Gamma$ and $B$ as numpy arrays of floating-point numbers
(whole numbers and $\pm i$, which floating-point numbers store exactly).
```

In [2]:

```
import numpy as np  # arrays, matrices, eigenvalues
import sympy as sp  # exact symbolic algebra for the spin connection
```

```
GAMMAS = "Revision/algebra/gammas.json"
PY = "Revision/pairing/reports/python-pairing.json"
WL = "Revision/pairing/reports/wolfram-pairing.json"
THEORY = "Revision/pairing/pairing-theory.json"
TH_PY = "Revision/theory/reports/python-field-theory.json"
TH_WL = "Revision/theory/reports/wolfram-field-theory.json"
SCOPE = "Revision/theory/reports/python-scope.json"
PARAMETERS = "Revision/kohn_sham/results/parameters.json"


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


RECORDS = {f: read_json(f) for f in (PY, WL, TH_PY, TH_WL, SCOPE)}


def recorded(report_file, name):
    """The recorded entry (verdict and detail) of the check name of a report."""
    for entry in RECORDS[report_file]["checks"]:
        if entry["name"] == name:
            return entry
    return {"verdict": "MISSING", "detail": ""}


def reproduces(condition, name, *sources):
    """check(condition, name), which also requires every named check of every
    source (report_file, [check names]) to have the recorded verdict PASS."""
    ok = all(recorded(f, n)["verdict"].upper() == "PASS"
             for f, names in sources for n in names)
    text = "; ".join(f"{f}, check {', '.join(names)}" for f, names in sources)
    check(condition and ok, name, record=text)


fixture = read_json(GAMMAS)
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
ETA = {a: int(fixture["eta"][a]) for a in range(8)}
I16 = np.eye(16)
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]
Gamma = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5] @ gamma[6] \
    @ gamma[7]
B = -1j * C @ gamma[4]  # the Krein matrix
check(all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                         2.0 * (ETA[a] if a == b else 0) * I16)
          for a in range(8) for b in range(8))
      and np.array_equal(Gamma, np.diag([-1.0] * 8 + [1.0] * 8))
      and np.array_equal(B.conj().T, B),
      "eight real 16 x 16 gammas (Clifford relations), Gamma = diag(-I8, I8), "
      "B Hermitian")
```

Out [2]:

```
PASS eight real 16 x 16 gammas (Clifford relations), Gamma = diag(-I8, I8), B Hermitian
```

Text cell:

```
## 6. The spin connection on the patch and on the mirror patch

For a diagonal vielbein $e^a{}_\mu = f_\mu\delta^a_\mu$ whose factors depend on
$x_4$ and $x_0$ only, the canonical spin connection has one simple formula: for
two different directions $a$ and $b$,

$$\omega_{a,ab} = \eta_{aa}\,\frac{\partial_bf_a}{f_b},\qquad
\omega_{a,ba} = -\omega_{a,ab},$$

and every other component is zero (the first index is the direction of the
derivative $D_a$). The next cell writes the author's factors $f_1 = f_2 = f_3 =
e^{a_4}\sin^{1/6}z$, $f_4 = 1$, $f_5 = f_6 = f_7 = e^{-a_4}\sin^{1/6}z$, $f_0 =
s_0\cot z$ ($s_0 = +1$ on the patch, $-1$ on the mirror patch) with sympy, lists
the nonzero components on the patch in the format of the sympy pairing report
($a_4'$ written `a4p`) and compares the list with the recorded one word for word.
It then computes $\gamma^\mu\Omega_\mu$ in the *divergence form* of the theory
record, $\gamma^\mu\Omega_\mu = \frac{1}{2\sqrt{|g|}}\sum_\mu\partial_\mu
\big(\sqrt{|g|}\gamma^{(\mu)}/f_\mu\big)$, on both patches: $+3H\gamma^{(x_0)}$
and $-3H\gamma^{(x_0)}$.
```

In [3]:

```
x4, z, H_sym = sp.symbols("x4 z H", real=True)
a4 = sp.Function("a4")(x4)
a4p = sp.Symbol("a4p")


def d(expr, b):
    """The derivative of a coefficient along the coordinate x_b (z = 6 H x0)."""
    if b == 4:
        return sp.diff(expr, x4)
    if b == 0:
        return 6 * H_sym * sp.diff(expr, z)
    return sp.Integer(0)


def factors(s0):
    s6 = sp.sin(z) ** sp.Rational(1, 6)
    f = {a: sp.exp(a4) * s6 for a in (1, 2, 3)}
    f.update({a: sp.exp(-a4) * s6 for a in (5, 6, 7)})
    f.update({4: sp.Integer(1), 0: s0 * sp.cot(z)})
    return f


def connection(s0):
    """{(mu, a, b) with a < b: omega_mu ab} for the diagonal vielbein."""
    f, om = factors(s0), {}
    for a in range(8):
```

```
        for b in range(8):
            if a != b:
                value = sp.simplify(ETA[a] * d(f[a], b) / f[b])
                if value != 0:  # omega_a,ab (or -omega_a,ba when b < a)
                    om[(a, a, b) if a < b else (a, b, a)] = (
                        value if a < b else -value)
    return dict(sorted(om.items()))


def short(expr):
    expr = sp.simplify(expr).subs(sp.Derivative(a4, x4), a4p)
    return expr.subs(a4, sp.Symbol("a4"))


patch_om, mirror_om = connection(1), connection(-1)
listing = "; ".join(f"omega_x{mu} x{a}x{b} = {short(v)}"
                    for (mu, a, b), v in patch_om.items())
say("patch: " + listing)
same = recorded(PY, "geometry.spin_connection_components")["detail"] == (
    "nonzero components (a < b): " + listing)
reproduces(len(patch_om) == 12 and same,
           "the formula gives the 12 recorded spin connection components",
           (PY, ["geometry.spin_connection_components"]))
flips = all(sp.simplify(mirror_om[k] + patch_om[k]) == 0 if 0 in k[1:]
            else sp.simplify(mirror_om[k] - patch_om[k]) == 0 for k in patch_om)
divergence = {}
for s0 in (1, -1):
    f = factors(s0)
    sqrtg = sp.simplify(sp.Mul(*f.values()))
    divergence[s0] = {mu: sp.simplify(d(sqrtg / f[mu], mu) / (2 * sqrtg))
                      for mu in range(8)}
say(f"divergence form, patch: coefficient of gamma^(x0) = {divergence[1][0]}, of "
    f"gamma^(x4) = {divergence[1][4]}; mirror patch: {divergence[-1][0]}, "
    f"{divergence[-1][4]}")
reproduces(flips and divergence[1][0] == 3 * H_sym and divergence[-1][0] == -3 * H_sym
           and all(divergence[s][mu] == 0 for s in (1, -1) for mu in range(1, 8)),
           "gamma^mu Omega_mu = +3 H gamma^(x0) (patch), -3 H gamma^(x0) (mirror)",
           (TH_PY, ["gamma_mu_Omega_mu_equals_3H_gamma_x0",
                    "divergence_of_sqrtg_gamma"]),
           (TH_WL, ["gammaOmega_equals_3H_gamma_x0", "gammaOmega_divergence_form"]))
```

Out [3]:

```
patch: omega_x1 x0x1 = -H*exp(a4)*sin(z)**(1/6); omega_x1 x1x4 =
    a4p*exp(a4)*sin(z)**(1/6); omega_x2 x0x2 = -H*exp(a4)*sin(z)**(1/6); omega_x2 x2x4 =
    a4p*exp(a4)*sin(z)**(1/6); omega_x3 x0x3 = -H*exp(a4)*sin(z)**(1/6); omega_x3 x3x4 =
    a4p*exp(a4)*sin(z)**(1/6); omega_x5 x0x5 = H*exp(-a4)*sin(z)**(1/6); omega_x5 x4x5 =
    -a4p*exp(-a4)*sin(z)**(1/6); omega_x6 x0x6 = H*exp(-a4)*sin(z)**(1/6); omega_x6 x4x6
    = -a4p*exp(-a4)*sin(z)**(1/6); omega_x7 x0x7 = H*exp(-a4)*sin(z)**(1/6); omega_x7
    x4x7 = -a4p*exp(-a4)*sin(z)**(1/6)
PASS the formula gives the 12 recorded spin connection components
     reproduces Revision/pairing/reports/python-pairing.json, check
    geometry.spin_connection_components
```

```
divergence form, patch: coefficient of gamma^(x0) = 3*H, of gamma^(x4) = 0; mirror patch:
    -3*H, 0
PASS gamma^mu Omega_mu = +3 H gamma^(x0) (patch), -3 H gamma^(x0) (mirror)
     reproduces Revision/theory/reports/python-field-theory.json, check
    gamma_mu_Omega_mu_equals_3H_gamma_x0, divergence_of_sqrtg_gamma;
    Revision/theory/reports/wolfram-field-theory.json, check
    gammaOmega_equals_3H_gamma_x0, gammaOmega_divergence_form
```

Text cell:

```
The next cell fixes the numbers. The history is the canonical one of the Revision
Kohn-Sham record, $a_4 = AHx_4$ with $A = 1$ (read from
‘Revision/kohn_sham/results/parameters.json‘). The other values are an
ILLUSTRATION chosen by this notebook (the theorems hold for every value): units in
which $m = 1$, $H = 0.2$, $\lambda = 0.3$, the hidden angle $z_0 = \pi/4$ on the
patch (its mirror point is $3\pi/4$), and an initial value $\Psi(0)$ of 16
complex numbers drawn with the fixed seed 2026, weighted towards the eigenvalue
$+1$ of $C$ (so that $S = \Psi^\dagger C\Psi$ is clearly positive: $C$ is real and
symmetric with $C^2 = 1$, its projectors are $P_{C\pm} = \frac12(1 \pm C)$, and
$S = |P_{C+}\Psi|^2 - |P_{C-}\Psi|^2$), and scaled to length 1. With these values
$(m + \lambda S)^2 > 9H^2$, so the solution oscillates. The cell then turns
the spin connection into numerical $16 \times 16$ matrices
$\Omega_\mu = \sum_{a<b}\omega_{\mu ab}S^{ab}$ at any time $x_4$ and angle $z$,
and defines the curved gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ and
$\gamma_\mu = \eta_{\mu\mu}f_\mu\gamma^{(\mu)}$.
```

In [4]:

```
A_hist = read_json(PARAMETERS)["physics"]["historyA"]  # A = 1
m, H, lam = 1.0, 0.2, 0.3  # an illustration (units m = 1)
Z0 = np.pi / 4  # the patch point; the mirror point is pi - Z0
rng = np.random.default_rng(2026)
raw = rng.normal(size=16) + 1j * rng.normal(size=16)
P_C = 0.5 * (I16 + C)  # the projector on the eigenvalue +1 of C
psi0 = 2.0 * P_C @ raw + 0.5 * (I16 - P_C) @ raw  # mostly in the +1 part: S > 0
psi0 = psi0 / np.linalg.norm(psi0)  # length 1
S0 = float((psi0.conj() @ C @ psi0).real)  # S of the initial value
report("S of the initial value", f"{S0:.6f}")
report("m + lambda S", f"{m + lam * S0:.6f}")
report("3 H", f"{3 * H:.6f}")

S_AB = {(a, b): 0.5 * gamma[a] @ gamma[b] for a in range(8) for b in range(8)}
history = {sp.Derivative(a4, x4): A_hist * H_sym}  # a4' = A H


def numeric(table):
    """Turn the sympy components into functions of (x4, z) for this H and A."""
    out = {}
    for key, value in table.items():
        e = value.subs(history).subs(a4, A_hist * H_sym * x4).subs(H_sym, H)
        out[key] = sp.lambdify((x4, z), e, "numpy")
    return out
```

```
OMEGA = {1: numeric(patch_om), -1: numeric(mirror_om)}
FACTOR = {s0: {mu: sp.lambdify((x4, z), e.subs(a4, A_hist * H_sym * x4)
                                .subs(H_sym, H), "numpy")
               for mu, e in factors(s0).items()} for s0 in (1, -1)}


def geometry_at(t, zv, s0):
    """Numbers at the time t and angle zv: f, g, gamma^mu, gamma_mu, Omega_mu."""
    f = {mu: float(FACTOR[s0][mu](t, zv)) for mu in range(8)}
    Om = {mu: np.zeros((16, 16)) for mu in range(8)}
    for (mu, a, b), fn in OMEGA[s0].items():
        Om[mu] = Om[mu] + float(fn(t, zv)) * S_AB[a, b]
    return {"f": f, "g": {mu: ETA[mu] * f[mu] ** 2 for mu in f},
            "sqrtg": abs(np.prod(list(f.values()))),
            "up": {mu: gamma[mu] / f[mu] for mu in f},
            "down": {mu: ETA[mu] * f[mu] * gamma[mu] for mu in f}, "Om": Om}


for s0, zv in ((1, Z0), (-1, np.pi - Z0)):
    geo = geometry_at(1.3, zv, s0)
    total = sum(geo["up"][mu] @ geo["Om"][mu] for mu in range(8))
    assert np.allclose(total, 3 * s0 * H * gamma[0], atol=1e-12)
check(abs(m + lam * S0) > 3 * H,
      "|m + lambda S| > 3 H: the solution and both partners oscillate")
```

Out [4]:

```
RESULT S of the initial value = 0.897319
RESULT m + lambda S = 1.269196
RESULT 3 H = 0.600000
PASS |m + lambda S| > 3 H: the solution and both partners oscillate
```

Text cell:

```
## 7. The solution: Runge-Kutta against the exact formula

The next cell defines the right-hand side $M\Psi$ of the ODE (with $s_0 = +1$ on
the patch and $-1$ on the mirror patch), one RK4 step and an integrator that
returns the solution at every step, and the exact formula of section 4. It
integrates $\Psi$ from $x_4 = 0$ to $x_4 = 20$ with 2000 steps of length 0.01 and
checks: $S$ stays equal to $S(0)$; the RK4 solution agrees with the exact formula;
$M^2 = (9H^2 - (m + \lambda S)^2)I_{16}$. It repeats the integration with 250 to
4000 steps to measure the order of the method (the error should fall by
$2^4 = 16$ when the step is halved).
```

In [5]:

```
X_END = 20.0  # integrate from x4 = 0 to 20 (a4 from 0 to A H 20 = 4)
```

```
def generator(psi, mass, coup, s0):
    """The matrix M of d psi/dx4 = M psi (s0 = +1 patch, -1 mirror patch)."""
    S = (psi.conj() @ C @ psi).real
    return -gamma[4] @ ((mass + coup * S) * I16 - 3 * s0 * H * gamma[0])


def rk4(psi_start, mass, coup, s0, steps):
    """The RK4 solution at x4 = 0, h, 2h, ..., X_END (steps + 1 rows)."""
    h = X_END / steps
    out = np.empty((steps + 1, 16), dtype=complex)
    psi = psi_start.astype(complex)
    out[0] = psi
    for n in range(steps):
        k1 = generator(psi, mass, coup, s0) @ psi
        k2 = generator(psi + 0.5 * h * k1, mass, coup, s0) @ (psi + 0.5 * h * k1)
        k3 = generator(psi + 0.5 * h * k2, mass, coup, s0) @ (psi + 0.5 * h * k2)
        k4 = generator(psi + h * k3, mass, coup, s0) @ (psi + h * k3)
        psi = psi + h / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
        out[n + 1] = psi
    return out


def exact(psi_start, mass, coup, s0, times):
    """The exact solution cos(w x4) psi(0) + sin(w x4)/w M psi(0)."""
    M = generator(psi_start, mass, coup, s0)
    S = (psi_start.conj() @ C @ psi_start).real
    w = np.sqrt((mass + coup * S) ** 2 - 9 * H ** 2)
    return (np.cos(w * times)[:, None] * psi_start[None, :]
            + (np.sin(w * times) / w)[:, None] * (M @ psi_start)[None, :])


STEPS = 2000
times = np.linspace(0.0, X_END, STEPS + 1)
psi = rk4(psi0, m, lam, 1, STEPS)  # the universe of mass m on the patch
S_t = np.einsum("ti,ij,tj->t", psi.conj(), C, psi).real
error = np.abs(psi - exact(psi0, m, lam, 1, times)).max()
M0 = generator(psi0, m, lam, 1)
square_ok = np.allclose(M0 @ M0, (9 * H ** 2 - (m + lam * S0) ** 2) * I16, atol=1e-13)
errors = {}
for n in (250, 500, 1000, 2000, 4000):
    sol = rk4(psi0, m, lam, 1, n)
    errors[n] = float(np.abs(sol[-1] - exact(psi0, m, lam, 1, np.array([X_END]))[0])
                      .max())
orders = [np.log2(errors[n] / errors[2 * n]) for n in (250, 500, 1000, 2000)]
say("RK4 error at x4 = 20 for 250, 500, 1000, 2000, 4000 steps: "
    + ", ".join(f"{errors[n]:.1e}" for n in errors))
say("measured orders: " + ", ".join(f"{p:.2f}" for p in orders))
report("largest RK4 error with 2000 steps (all times, all components)",
       f"{error:.1e}")
reproduces(np.abs(S_t - S0).max() < 1e-9 and error < 1e-7 and square_ok,
           "S is constant and RK4 equals the exact homogeneous solution (2000 steps)",
           (TH_PY, ["exact_nonlinear_homogeneous_solution"]),
           (TH_WL, ["exact_solution_nonlinear_homogeneous_C"]),
```

```
          (SCOPE, ["good_sector_x0_independent_modes_without_boundary_condition"]))
check(all(3.8 < p < 4.2 for p in orders), "RK4 is of fourth order here")
```

Out [5]:

```
RK4 error at x4 = 20 for 250, 500, 1000, 2000, 4000 steps: 2.0e-05, 1.2e-06, 6.9e-08,
    4.3e-09, 2.6e-10
measured orders: 4.10, 4.06, 4.03, 4.01
RESULT largest RK4 error with 2000 steps (all times, all components) = 4.3e-09
PASS S is constant and RK4 equals the exact homogeneous solution (2000 steps)
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_nonlinear_homogeneous_solution; Revision/theory/reports/wolfram-field-
    theory.json, check exact_solution_nonlinear_homogeneous_C;
    Revision/theory/reports/python-scope.json, check
    good_sector_x0_independent_modes_without_boundary_condition
PASS RK4 is of fourth order here
```

Text cell:

```
The next cell draws the convergence of the method: the error at $x_4 = 20$
against the step length on logarithmic axes, with a line of slope 4 for
comparison.
```

In [6]:

```
steps_list = sorted(errors)
hs = np.array([X_END / n for n in steps_list])
errs = np.array([errors[n] for n in steps_list])
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.loglog(hs, errs, "o-", color="#2a78d6", label="RK4 error at $x_4 = 20$")
ax.loglog(hs, errs[-1] * (hs / hs[-1]) ** 4, "--", color="#52514e",
          label="slope 4: error $\\propto h^4$")
ax.set_xlabel("step length $h$ (units $1/m$)")
ax.set_ylabel("largest error of a component")
ax.legend()
save_figure(fig, "rk4_convergence",
            "Accuracy of the Runge-Kutta solution of the homogeneous field equation "
            "in the author's metric: the largest difference between a component of "
            "the computed field and of the exact solution of the Revision theory "
            "record at the time $x_4 = 20$ (vertical axis, logarithmic), against "
            "the step length $h$ in units of $1/m$ (horizontal axis, logarithmic), "
            "for 250 to 4000 steps. The points follow the dashed line of slope 4: "
            "halving the step divides the error by 16, the fourth order of the "
            "method. With 2000 steps the error is far below every difference "
            "studied in this notebook.")
```

Out [6]:

```
Figure 18c.1 saved as Revision/textbook/figures/18c_1_rk4_convergence.png
```

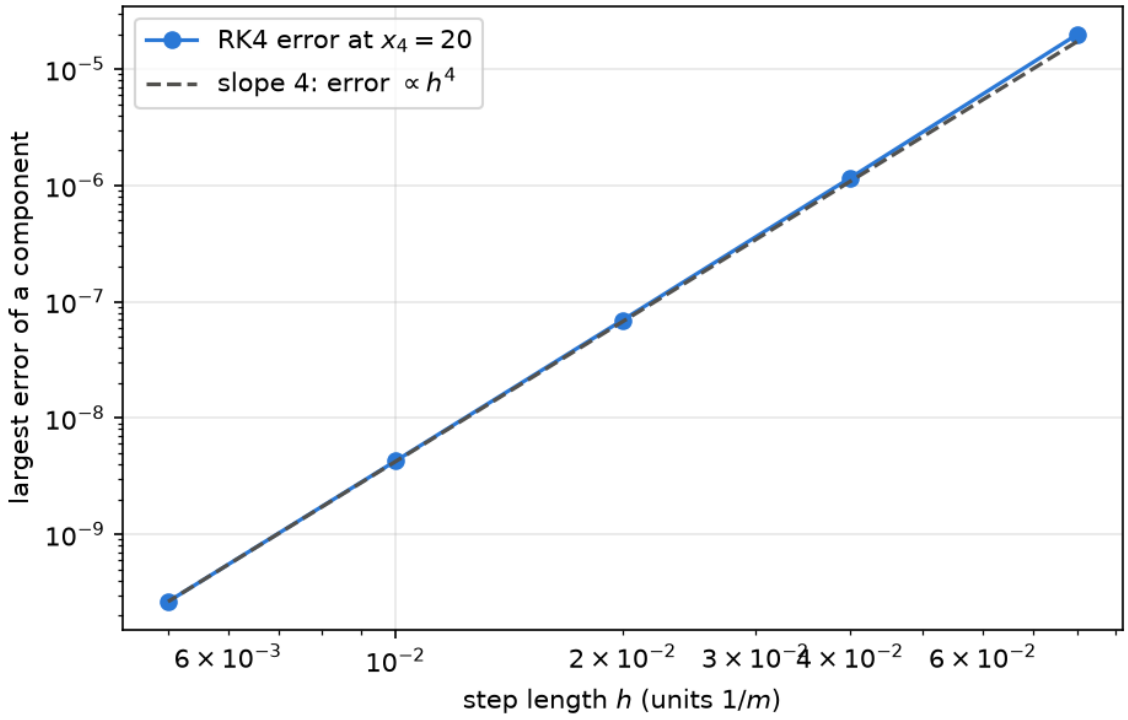

Figure 484: Accuracy of the Runge-Kutta solution of the homogeneous field equation in the author's metric: the largest difference between a component of the computed field and of the exact solution of the Revision theory record at the time $x_4 = 20$ (vertical axis, logarithmic), against the step length h in units of $1/m$ (horizontal axis, logarithmic), for 250 to 4000 steps. The points follow the dashed line of slope 4: halving the step divides the error by 16, the fourth order of the method. With 2000 steps the error is far below every difference studied in this notebook. (Notebook 18c, figure 1.)

Text cell:

```
## 8. Theorem T1 in numbers

The next cell solves, INDEPENDENTLY, the theory with $(-m, -\lambda)$ on the patch,
starting from $\Gamma\Psi(0)$, and compares the result $\Phi(x_4)$ with
$\Gamma\Psi(x_4)$ at all 2001 times. Since $\Gamma = \mathrm{diag}(-I_8, I_8)$,
T1 predicts $\Phi_A = -\Psi_A$ for the components $A = 1, \dots, 8$ and
$\Phi_A = +\Psi_A$ for $A = 9, \dots, 16$. As a negative control the cell also
solves the WRONG partner theory $(-m, +\lambda)$ from the same start, which must
NOT give $\Gamma\Psi$: the coupling has to change sign together with the mass.
```

In [7]:

```
phi = rk4(Gamma @ psi0, -m, -lam, 1, STEPS)  # the T1 partner, solved on its own
t1_gap = np.abs(phi - psi @ Gamma.T).max()  # row by row: Gamma psi(x4)
wrong = rk4(Gamma @ psi0, -m, lam, 1, STEPS)  # the negative control (-m, +lambda)
wrong_gap = np.abs(wrong - psi @ Gamma.T).max()
report("largest difference of the wrong partner (-m, +lambda) from Gamma Psi",
       f"{wrong_gap:.3f}")
check(t1_gap < 1e-12, "T1: the (-m, -lambda) solution from Gamma Psi(0) is Gamma "
      "Psi(x4) at all 2001 times (difference below 1e-12)")
check(wrong_gap > 0.1, "negative control: the (-m, +lambda) solution from Gamma "
      "Psi(0) is not Gamma Psi(x4)")
```

Out [7]:

```
RESULT largest difference of the wrong partner (-m, +lambda) from Gamma Psi = 1.641
PASS T1: the (-m, -lambda) solution from Gamma Psi(0) is Gamma Psi(x4) at all 2001 times
    (difference below 1e-12)
PASS negative control: the (-m, +lambda) solution from Gamma Psi(0) is not Gamma Psi(x4)
```

Text cell:

```
The next cell draws two components: $A = 1$ (in the first half, where $\Gamma =
-1$) and $A = 9$ (in the second half, where $\Gamma = +1$), real parts, for the
field $\Psi$ (lines) and for its independently computed partner $\Phi$ (dots).
```

In [8]:

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0), sharey=True)
every = slice(0, STEPS + 1, 40)  # a dot every 40 steps
for ax, A in zip(axes, (0, 8)):
    ax.plot(times, psi[:, A].real, color="#2a78d6",
            label=f"$\\Psi_{{{A + 1}}}$, mass $m$, coupling $\\lambda$")
    ax.plot(times[every], phi[every, A].real, "o", color="#eb6834", markersize=3.5,
            label=f"$\\Phi_{{{A + 1}}}$, mass $-m$, coupling $-\\lambda$")
    ax.plot(times, wrong[:, A].real, "--", color="#52514e", linewidth=0.9,
            label="wrong partner: $-m$, $+\\lambda$")
```

```
    ax.set_xlabel("time $x_4$ (units $1/m$)")
    ax.set_title(f"component {A + 1}: " + ("$\\Phi = -\\Psi$" if A < 8
                                           else "$\\Phi = +\\Psi$"), fontsize=10)
    ax.legend(loc="lower left", fontsize=8)
axes[0].set_ylabel("real part of the component")
save_figure(fig, "partner_components",
            "A solution and its T1 partner, computed separately. Lines: the real "
            "part of component 1 (left) and component 9 (right) of the homogeneous "
            "solution $\\Psi$ with mass $m = 1$ and coupling $\\lambda = 0.3$, "
            "against the time $x_4$ in units of $1/m$. Dots: the same components "
            "of the solution $\\Phi$ of the theory with $-m$ and $-\\lambda$, "
            "started from $\\Gamma\\Psi(0)$ and integrated on its own. In the first "
            "half of the components $\\Phi = -\\Psi$, in the second half "
            "$\\Phi = +\\Psi$: the partner is exactly $\\Gamma\\Psi$ at every time, "
            "as theorem T1 says. Dashed: the negative control with $-m$ and "
            "$+\\lambda$, which drifts away because it oscillates with another "
            "frequency.")
```

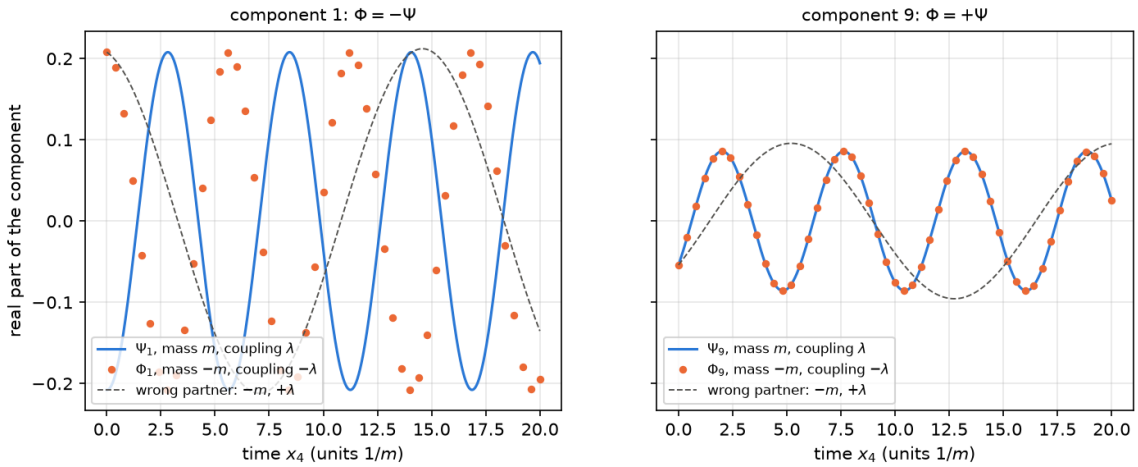

Figure 485: A solution and its T1 partner, computed separately. Lines: the real part of component 1 (left) and component 9 (right) of the homogeneous solution Ψ with mass $m = 1$ and coupling $\lambda = 0.3$, against the time x_4 in units of $1/m$. Dots: the same components of the solution Φ of the theory with $-m$ and $-\lambda$, started from $\Gamma\Psi(0)$ and integrated on its own. In the first half of the components $\Phi = -\Psi$, in the second half $\Phi = +\Psi$: the partner is exactly $\Gamma\Psi$ at every time, as theorem T1 says. Dashed: the negative control with $-m$ and $+\lambda$, which drifts away because it oscillates with another frequency. (Notebook 18c, figure 2.)

Out [8]:

```
Figure 18c.2 saved as Revision/textbook/figures/18c_2_partner_components.png
```

Text cell:

```
The next cell computes, at every 20th time, the bilinears of a field in the
geometry of the point $(x_4, z)$: $S$; the current $J^\mu = -i\bar\Psi\gamma^\mu
```

```
\Psi$ (8 components); and the energy-momentum tensor of the pairing record,

$$T_{\mu\nu} = \tfrac14\big(\bar\Psi\gamma_\mu D_\nu\Psi + \bar\Psi\gamma_\nu
D_\mu\Psi - (D_\mu\bar\Psi)\gamma_\nu\Psi - (D_\nu\bar\Psi)\gamma_\mu\Psi\big) -
g_{\mu\nu}\mathcal{L}/\sqrt{|g|},$$

with $D_4\Psi = d\Psi/dx_4 = M\Psi$ (the field equation), $D_\mu\Psi =
\Omega_\mu\Psi$ for the other directions (the field does not depend on them; the
spin connection still acts), $D_\mu\bar\Psi = (D_\mu\Psi)^\dagger C$, and
$\mathcal{L}/\sqrt{|g|} = \frac12\sum_\mu(\bar\Psi\gamma^\mu D_\mu\Psi -
(D_\mu\bar\Psi)\gamma^\mu\Psi) - mS - \frac{\lambda}{2}S^2$. The history
$a_4 = AHx_4$ enters through the factors $f_\mu$ and $\Omega_\mu$. The cell then
checks, at all sampled times, that the T1 partner has $J^\mu[\Phi] = -J^\mu[\Psi]$
for all 8 components and $T_{\mu\nu}[\Phi; -m, -\lambda] =
-T_{\mu\nu}[\Psi; m, \lambda]$ for all 36 components.
```

In [9]:

```
def bilinears(field, mass, coup, t, zv, s0):
    """S, J^mu (8 numbers) and T_mu nu (8 x 8) of a homogeneous field at (t, zv)."""
    geo = geometry_at(t, zv, s0)
    D = {mu: geo["Om"][mu] @ field for mu in range(8)}
    D[4] = D[4] + generator(field, mass, coup, s0) @ field  # d field / dx4
    bar = field.conj() @ C  # the adjoint row Psibar = Psi^dagger C
    Dbar = {mu: D[mu].conj() @ C for mu in range(8)}
    S = (bar @ field).real
    L = 0.5 * sum(bar @ geo["up"][mu] @ D[mu] - Dbar[mu] @ geo["up"][mu] @ field
                  for mu in range(8)) - mass * S - 0.5 * coup * S ** 2
    T = np.zeros((8, 8), dtype=complex)
    for mu in range(8):
        for nu in range(8):
            T[mu, nu] = 0.25 * (
                bar @ geo["down"][mu] @ D[nu] + bar @ geo["down"][nu] @ D[mu]
                - Dbar[mu] @ geo["down"][nu] @ field
                - Dbar[nu] @ geo["down"][mu] @ field)
            if mu == nu:
                T[mu, nu] -= geo["g"][mu] * L
    J = np.array([-1j * bar @ geo["up"][mu] @ field for mu in range(8)])
    assert np.abs(T.imag).max() < 1e-12 and np.abs(J.imag).max() < 1e-12
    return S, J.real, T.real


SAMPLE = range(0, STEPS + 1, 20)  # 101 sampled times
sampled = times[list(SAMPLE)]
data = {"psi": [bilinears(psi[n], m, lam, times[n], Z0, 1) for n in SAMPLE],
        "phi": [bilinears(phi[n], -m, -lam, times[n], Z0, 1) for n in SAMPLE]}
scale_T = max(np.abs(T).max() for _, _, T in data["psi"])
scale_J = max(np.abs(J).max() for _, J, _ in data["psi"])
t1_T = max(np.abs(Tp + Tq).max() for (_, _, Tp), (_, _, Tq)
           in zip(data["psi"], data["phi"])) / scale_T
t1_J = max(np.abs(Jp + Jq).max() for (_, Jp, _), (_, Jq, _)
           in zip(data["psi"], data["phi"])) / scale_J
t1_S = max(abs(Sp - Sq) for (Sp, _, _), (Sq, _, _) in zip(data["psi"], data["phi"]))
```

```
reproduces(t1_T < 1e-12 and t1_J < 1e-12 and t1_S < 1e-12,
           "T1: S kept, J -> -J (8) and T -> -T (36) at all 101 sampled times",
           (PY, ["T1.metric.commuting.emt", "T1.metric.commuting.current"]))
```

Out [9]:

```
PASS T1: S kept, J -> -J (8) and T -> -T (36) at all 101 sampled times
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.emt, T1.metric.commuting.current
```

Text cell:

```
The next cell checks three facts of the theory record about homogeneous solutions,
at all sampled times: the energy density is $\rho = -T_{x_4x_4} = mS +
\frac{\lambda}{2}S^2$; the three kinds of pressure $p_\mu = -g^{\mu\mu}T_{\mu\mu}$
(3-space, extra times, hidden direction) all equal $\frac{\lambda}{2}S^2$; and the
mixed component $T_{x_4x_0}$ vanishes. It also checks the LOCAL conservation law
of the charge: the charge density can change in time only by flowing along the
hidden direction, $\partial_4J^{x_4} + \frac{1}{\sqrt{|g|}}\partial_0(\sqrt{|g|}
J^{x_0}) = 0$; for a homogeneous field $J^{x_0} = \tan z\,q_0$ with the frame
current $q_0 = -i\bar\Psi\gamma^{(x_0)}\Psi$ (a number that does not depend on
$x_0$) and $\sqrt{|g|} = \cos z$, so the law reads $\partial_4J^{x_4} = -6Hq_0$,
where $\partial_4J^{x_4} = 2\,\mathrm{Re}(\Psi^\dagger BM\Psi)$ follows from the
field equation. The flow through the brane $z = \pi/2$ is $\sqrt{|g|}J^{x_0} =
\sin z\,q_0 = q_0$ there; the cell prints the range of $q_0$: it is not zero, so
the total charge of the patch is NOT constant for this solution (charge flows
through the brane), although the local law holds exactly.
```

In [10]:

```
rho_ok = p_ok = mixed_ok = flow_ok = True
q0_values = []  # the frame current q0 at the sampled times
for n, (S, J, T) in zip(SAMPLE, data["psi"]):
    geo = geometry_at(times[n], Z0, 1)
    rho_ok = rho_ok and abs(-T[4, 4] - (m * S + 0.5 * lam * S ** 2)) < 1e-12
    p = [-T[mu, mu] / geo["g"][mu] for mu in (0, 1, 2, 3, 5, 6, 7)]
    p_ok = p_ok and max(abs(x - 0.5 * lam * S ** 2) for x in p) < 1e-12
    mixed_ok = mixed_ok and abs(T[4, 0]) < 1e-12 * scale_T
    field = psi[n]
    dJ4 = 2 * (field.conj() @ B @ generator(field, m, lam, 1) @ field).real
    q0 = (-1j * field.conj() @ C @ gamma[0] @ field).real
    q0_values.append(q0)
    flow_ok = flow_ok and abs(dJ4 + 6 * H * q0) < 1e-12
reproduces(rho_ok and p_ok and mixed_ok,
           "rho = m S + (1/2) S^2, all pressures (1/2) S^2, T_x4x0 = 0 (homogeneous)",
           (TH_PY, ["commuting_homogeneous_on_shell_rho_p",
                    "commuting_T_x4x0_homogeneous"]))
reproduces(flow_ok, "local charge conservation: d4 J^x4 = -6 H q0 (flow along x0)",
           (TH_PY, ["commuting_current_conservation"]))
say(f"q0 lies between {min(q0_values):.4f} and {max(q0_values):.4f}: the flow "
    "sin z q0 through the brane z = pi/2 is not zero, so the charge of the "
    "patch changes in time")
```

Out [10]:

```
PASS rho = m S + (1/2) S^2, all pressures (1/2) S^2, T_x4x0 = 0 (homogeneous)
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_homogeneous_on_shell_rho_p, commuting_T_x4x0_homogeneous
PASS local charge conservation: d4 J^x4 = -6 H q0 (flow along x0)
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_current_conservation
q0 lies between -0.3454 and 0.3454: the flow sin z q0 through the brane z = pi/2 is not
    zero, so the charge of the patch changes in time
```

Text cell:

```
The next cell draws the pair of T1 in three panels, against the time $x_4$:
the charge density $J^{x_4}$, the energy density $\rho = -T_{x_4x_4}$, and the
mixed component $T_{x_4x_1}$ (a flow of momentum along the inflating direction
$x_1$, which contains the spin connection and the factor $e^{a_4}$ of the
deflating history). Each panel shows the field, its T1 partner and their sum.
```

In [11]:

```
def series(name, pick):
    return np.array([pick(S, J, T) for S, J, T in data[name]])


panels = [("charge density $J^{x_4}$", lambda S, J, T: J[4]),
          ("energy density $\\rho = -T_{x_4x_4}$", lambda S, J, T: -T[4, 4]),
          ("$T_{x_4x_1}$ (history-dependent)", lambda S, J, T: T[4, 1])]
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0))
for ax, (title, pick) in zip(axes, panels):
    a, b = series("psi", pick), series("phi", pick)
    ax.plot(sampled, a, color="#2a78d6", label="$\\Psi$: $(m, \\lambda)$")
    ax.plot(sampled, b, color="#eb6834", label="T1 partner: $(-m, -\\lambda)$")
    ax.plot(sampled, a + b, color="#52514e", linestyle="--", label="sum")
    ax.set_title(title, fontsize=10)
    ax.set_xlabel("time $x_4$ (units $1/m$)")
axes[0].legend(fontsize=8, loc="lower left")
save_figure(fig, "t1_pair_densities",
            "The T1 pair in numbers, along the deflating history $a_4 = AHx_4$ "
            "($A = 1$, $H = 0.2$, $m = 1$, $\\lambda = 0.3$, at $z = \\pi/4$). "
            "Horizontal axes: the time $x_4$ in units of $1/m$. Left: the charge "
            "density $J^{x_4}$; middle: the energy density $\\rho$; right: the "
            "mixed component $T_{x_4x_1}$, which grows with the factor $e^{a_4}$ "
            "of the inflating direction. Blue: the solution with $(m, \\lambda)$; "
            "orange: its partner $\\Gamma\\Psi$ with $(-m, -\\lambda)$; dashed: "
            "their sum. Every orange curve is the mirror image of the blue one in "
            "the horizontal axis, and the sum is zero at every time: the pair has "
            "zero total charge and energy-momentum as classical bilinears.")
```

Out [11]:

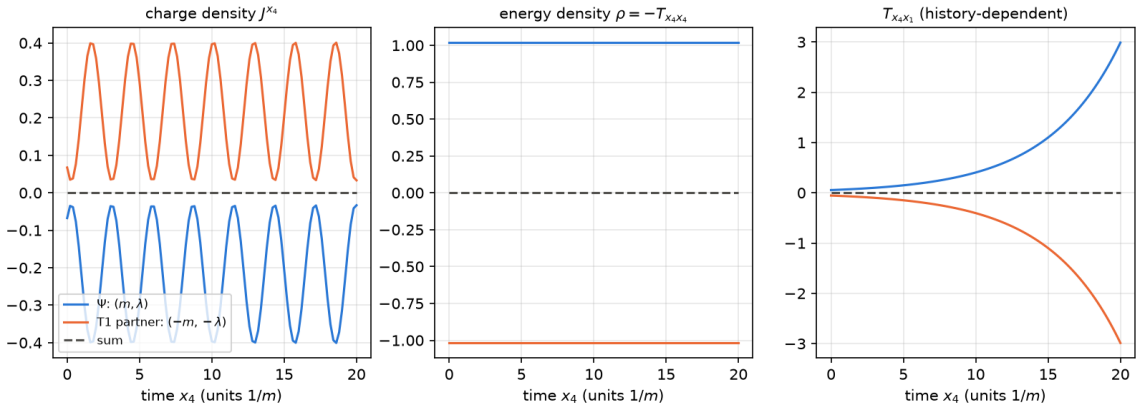

Figure 486: The T1 pair in numbers, along the deflating history $a_4 = AHx_4$ ($A = 1$, $H = 0.2$, $m = 1$, $\lambda = 0.3$, at $z = \pi/4$). Horizontal axes: the time x_4 in units of $1/m$. Left: the charge density J^{x_4}; middle: the energy density ρ; right: the mixed component $T_{x_4x_1}$, which grows with the factor e^{a_4} of the inflating direction. Blue: the solution with (m, λ); orange: its partner $\Gamma\Psi$ with $(-m, -\lambda)$; dashed: their sum. Every orange curve is the mirror image of the blue one in the horizontal axis, and the sum is zero at every time: the pair has zero total charge and energy-momentum as classical bilinears. (Notebook 18c, figure 3.)

```
Figure 18c.3 saved as Revision/textbook/figures/18c_3_t1_pair_densities.png
```

Text cell:

```
## 9. Theorem T2 in numbers: the mirror partner

The next cell solves, INDEPENDENTLY, the theory with $(-m, \lambda)$ on the mirror
patch (the spin connection term is $-3H\gamma^{(x_0)}$ there), starting from
$\gamma^{(x_0)}\Psi(0)$, and compares the result $X(x_4)$ with
$\gamma^{(x_0)}\Psi(x_4)$. Then it computes the bilinears of $X$ at the mirror
point $z = \pi - z_0$ and checks the T2 predictions at all sampled times:
$S[X] = -S[\Psi]$; $J^\mu[X] = \Lambda_\mu J^\mu[\Psi]$ and $T_{\mu\nu}[X] =
\Lambda_\mu\Lambda_\nu T_{\mu\nu}[\Psi]$ with $\Lambda = \mathrm{diag}(-1, 1, 1, 1,
1, 1, 1, 1)$ (only the components with an $x_0$ index change sign; the charge
density and the energy density are EQUAL).
```

In [12]:

```
chi = rk4(gamma[0] @ psi0, -m, lam, -1, STEPS)  # the mirror partner, on its own
t2_gap = np.abs(chi - psi @ gamma[0].T).max()
data["chi"] = [bilinears(chi[n], -m, lam, times[n], np.pi - Z0, -1) for n in SAMPLE]
LAM = np.array([-1, 1, 1, 1, 1, 1, 1, 1], dtype=float)  # x0, ..., x7
t2_S = max(abs(Sp + Sx) for (Sp, _, _), (Sx, _, _) in zip(data["psi"], data["chi"]))
t2_J = max(np.abs(Jx - LAM * Jp).max() for (_, Jp, _), (_, Jx, _)
           in zip(data["psi"], data["chi"])) / scale_J
t2_T = max(np.abs(Tx - np.outer(LAM, LAM) * Tp).max() for (_, _, Tp), (_, _, Tx)
           in zip(data["psi"], data["chi"])) / scale_T
check(t2_gap < 1e-12, "T2: the (-m, lambda) mirror solution from gamma^(x0) Psi(0) "
```

```
      "is gamma^(x0) Psi(x4) at all 2001 times")
reproduces(t2_S < 1e-12 and t2_J < 1e-12 and t2_T < 1e-12,
           "T2: S -> -S, J and T pulled back (charge and energy density equal)",
           (PY, ["T2.metric.commuting.S_odd", "T2.metric.commuting.emt",
                 "T2.metric.commuting.current"]))
```

Out [12]:

```
PASS T2: the (-m, lambda) mirror solution from gamma^(x0) Psi(0) is gamma^(x0) Psi(x4) at
    all 2001 times
PASS T2: S -> -S, J and T pulled back (charge and energy density equal)
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.S_odd, T2.metric.commuting.emt, T2.metric.commuting.current
```

Text cell:

```
The next cell draws the T2 pair: the scalar $S$, the charge density $J^{x_4}$ and
the mixed component $T_{x_4x_1}$ of the field on the patch and of its mirror
partner on the mirror patch.
```

In [13]:

```
panels = [("scalar $S$", lambda S, J, T: S),
          ("charge density $J^{x_4}$", lambda S, J, T: J[4]),
          ("$T_{x_4x_1}$ (history-dependent)", lambda S, J, T: T[4, 1])]
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0))
for ax, (title, pick) in zip(axes, panels):
    ax.plot(sampled, series("psi", pick), color="#2a78d6",
            label="$\\Psi$ on the patch: $(m, \\lambda)$")
    ax.plot(sampled[::4], series("chi", pick)[::4], "o", color="#1baf7a",
            markersize=4, label="mirror partner: $(-m, \\lambda)$")
    ax.set_title(title, fontsize=10)
    ax.set_xlabel("time $x_4$ (units $1/m$)")
axes[0].set_ylim(-1.1, 1.1)
axes[0].legend(fontsize=8, loc="center left")
save_figure(fig, "t2_mirror_densities",
            "The T2 pair in numbers, same values as in the previous figure. Blue "
            "line: the solution $\\Psi$ with $(m, \\lambda)$ on the patch at "
            "$z = \\pi/4$; green dots: its mirror partner $\\gamma^{(x_0)}\\Psi$, "
            "solved on its own with $(-m, \\lambda)$ on the mirror patch at "
            "$z = 3\\pi/4$. Horizontal axes: the time $x_4$ in units of $1/m$. "
            "Left: the scalar $S$, constant and exactly opposite for the partner. "
            "Middle and right: the charge density $J^{x_4}$ and the component "
            "$T_{x_4x_1}$, EQUAL for the partner. T2 pairs the mass $m$ with $-m$ "
            "at the same charge and energy-momentum; nothing cancels.")
```

Out [13]:

```
Figure 18c.4 saved as Revision/textbook/figures/18c_4_t2_mirror_densities.png
```

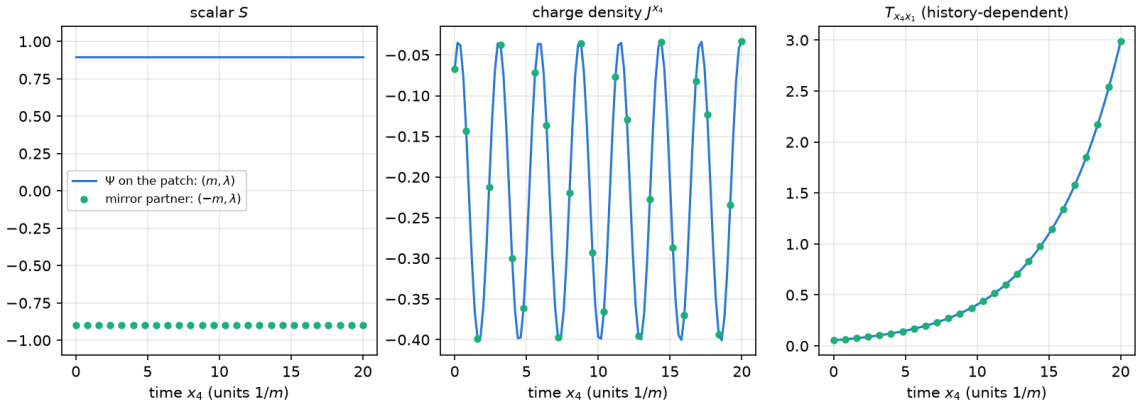

Figure 487: The T2 pair in numbers, same values as in the previous figure. Blue line: the solution Ψ with (m, λ) on the patch at $z = \pi/4$; green dots: its mirror partner $\gamma^{(x_0)}\Psi$, solved on its own with $(-m, \lambda)$ on the mirror patch at $z = 3\pi/4$. Horizontal axes: the time x_4 in units of $1/m$. Left: the scalar S, constant and exactly opposite for the partner. Middle and right: the charge density J^{x_4} and the component $T_{x_4x_1}$, EQUAL for the partner. T2 pairs the mass m with $-m$ at the same charge and energy-momentum; nothing cancels. (Notebook 18c, figure 4.)

Text cell:

```
## 10. Spectra: the partners evolve with the same eigenvalues

The evolution generator $M$ of the homogeneous family is a constant matrix along a
solution. The next cell computes its 16 eigenvalues for the field (mass $m$,
coupling $\lambda$, scalar $S$), for the T1 partner ($-m$, $-\lambda$, the same
$S$) and for the T2 partner on the mirror patch ($-m$, $\lambda$, scalar $-S$),
and checks that the three spectra are equal: $\Gamma M\Gamma$ and
$\gamma^{(x_0)}M\gamma^{(x_0)}$ are exactly the partners' generators (similar
matrices). It then repeats the computation for masses from $-2$ to $2$ at the same
$\lambda$ and $S$: the eigenvalues are $\pm iw$ (8 times each) with $w^2 = (m +
\lambda S)^2 - 9H^2$, real (an oscillation) outside the window
$|m + \lambda S| < 3H$ and imaginary (a growth) inside it, the same for every
universe and its partner.
```

In [14]:

```
def spectrum(Mx):
    """The 16 eigenvalues, sorted by imaginary part, then real part (rounded)."""
    ev = np.linalg.eigvals(Mx)
    return ev[np.lexsort((np.round(ev.real, 9), np.round(ev.imag, 9)))]


M_psi = generator(psi0, m, lam, 1)
M_t1 = generator(Gamma @ psi0, -m, -lam, 1)
M_t2 = generator(gamma[0] @ psi0, -m, lam, -1)
similar = (np.array_equal(Gamma @ M_psi @ Gamma, M_t1)
           and np.allclose(gamma[0] @ M_psi @ gamma[0], M_t2, atol=1e-15))
sp_psi, sp_t1, sp_t2 = spectrum(M_psi), spectrum(M_t1), spectrum(M_t2)
```

```
w_exact = np.sqrt((m + lam * S0) ** 2 - 9 * H ** 2)
report("frequency w of the solution and of both partners", f"{w_exact:.6f}")
check(similar and np.abs(sp_psi - sp_t1).max() < 1e-12
      and np.abs(sp_psi - sp_t2).max() < 1e-12
      and np.allclose(np.sort(np.abs(sp_psi.imag)), w_exact, atol=1e-12),
      "the generators of the field and of its T1 and T2 partners are similar: "
      "equal spectra +-i w")
masses = np.linspace(-2.0, 2.0, 161)
family = {}
for label, sign_m, sign_l in (("field", 1, 1), ("T1 partner", -1, -1)):
    rows = []
    for mv in masses:
        V = sign_m * mv + sign_l * lam * S0
        Mx = -V * gamma[4] + 3 * H * gamma[4] @ gamma[0]
        rows.append(spectrum(Mx))
    family[label] = np.array(rows)
check(np.abs(family["field"] - family["T1 partner"]).max() < 1e-9,
      "for every mass from -2 to 2 the field and its T1 partner have equal spectra")
```

Out [14]:

```
RESULT frequency w of the solution and of both partners = 1.118417
PASS the generators of the field and of its T1 and T2 partners are similar: equal spectra
    +-i w
PASS for every mass from -2 to 2 the field and its T1 partner have equal spectra
```

Text cell:

```
The next cell draws these spectra against the mass: the largest imaginary part
(the oscillation frequency $w$) and the largest real part (the growth rate) of the
eigenvalues, for the field and for its T1 partner.
```

In [15]:

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
for label, colour, style in (("field", "#2a78d6", "-"),
                             ("T1 partner", "#eb6834", "o")):
    rows = family[label]
    kw = {"markersize": 3} if style == "o" else {}
    ax.plot(masses, rows.imag.max(axis=1), style, color=colour,
            label=f"frequency $w$, {label}", **kw)
    ax.plot(masses, rows.real.max(axis=1), style, color=colour, alpha=0.45,
            label=f"growth rate, {label}", **kw)
ax.axvline(m, color="#52514e", linestyle=":", label="the solution of this notebook")
ax.set_xlabel("mass $m$ of the field (the partner has $-m$, $-\\lambda$)")
ax.set_ylabel("eigenvalue part (units $m = 1$)")
ax.legend(fontsize=8, loc="upper center")
save_figure(fig, "family_spectrum",
            "The spectrum of the evolution generator of the homogeneous solutions, "
            "against the mass $m$ (horizontal axis) at the fixed coupling "
            "$\\lambda = 0.3$ and scalar $S$ of this notebook, $H = 0.2$. Dark "
```

```
"curves: the oscillation frequency $w$, the largest imaginary part of "
"the eigenvalues; light curves: the growth rate, the largest real part. "
"Lines: the field with $(m, \\lambda)$; dots: its T1 partner with "
"$(-m, -\\lambda)$. They coincide everywhere. Inside the window "
"$|m + \\lambda S| < 3H$ the frequency is zero and the solutions grow; "
"outside it they oscillate. The dotted line marks the solution "
"plotted above.")
```

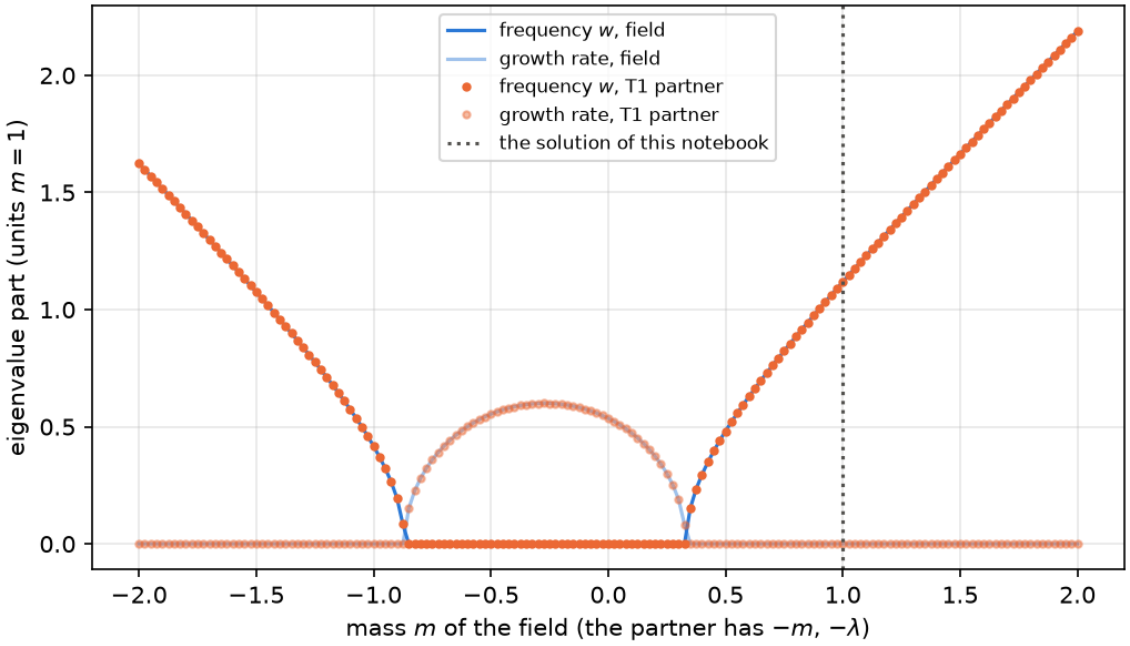

Figure 488: The spectrum of the evolution generator of the homogeneous solutions, against the mass m (horizontal axis) at the fixed coupling $\lambda = 0.3$ and scalar S of this notebook, $H = 0.2$. Dark curves: the oscillation frequency w, the largest imaginary part of the eigenvalues; light curves: the growth rate, the largest real part. Lines: the field with (m, λ); dots: its T1 partner with $(-m, -\lambda)$. They coincide everywhere. Inside the window $|m + \lambda S| < 3H$ the frequency is zero and the solutions grow; outside it they oscillate. The dotted line marks the solution plotted above. (Notebook 18c, figure 5.)

Out [15]:

```
Figure 18c.5 saved as Revision/textbook/figures/18c_5_family_spectrum.png
```

Text cell:

```
## 11. Plane waves in flat 4+4 space: the quantum reading Q

The next cell builds the one-particle matrix $h_m(k)$ of the Revision pairing
record and checks, numerically: $h_m^2 = w^2I_{16}$ with $w^2 = m^2 + k_0^2 + k_1^2
+ k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$ (for 50 random momenta); the maps
$\Gamma h_m(k)\Gamma = h_{-m}(k)$ and $\gamma^{(x_0)}h_m(k)\gamma^{(x_0)} =
```

```
h_{-m}(R_0k)$, where $R_0k$ reverses $k_0$; and it reproduces the eight samples
of the data table ‘one_particle_flat‘ of the pairing record: the frequency $w$,
the dimension of the eigenspaces of $+w$ and $-w$, and the *Krein inertia* of
each (the numbers of positive and negative eigenvalues of the $8 \times 8$ matrix
$V^\dagger BV$, where the 8 columns of $V$ span the eigenspace).
```

In [16]:

```
def h_matrix(mass, k):
    """h_m(k) = -i m gamma^(x4) - gamma^(x4) sum_(a != 4) k_a gamma^(a)."""
    out = -1j * mass * gamma[4]
    for a in (0, 1, 2, 3, 5, 6, 7):
        out = out - k[a] * gamma[4] @ gamma[a]
    return out


def w_squared(mass, k):
    return mass ** 2 + sum(k[a] ** 2 for a in (0, 1, 2, 3)) - sum(
        k[a] ** 2 for a in (5, 6, 7))


def eigenspace(Mx, value, tol=1e-9):
    """Orthonormal columns spanning the null space of Mx - value I."""
    _, sv, vh = np.linalg.svd(Mx - value * I16)
    return vh[sv < tol].conj().T


def inertia(V):
    """(positive, negative) eigenvalue counts of V^dagger B V."""
    ev = np.linalg.eigvalsh(V.conj().T @ B @ V)
    return int((ev > 1e-9).sum()), int((ev < -1e-9).sum())


R0 = np.array([-1, 1, 1, 1, 1, 1, 1, 1], dtype=float)  # the reflection of x0
rand = np.random.default_rng(7)
square_ok = maps_ok = True
for _ in range(50):
    k, mv = rand.normal(size=8), rand.normal()
    h = h_matrix(mv, k)
    square_ok = square_ok and np.allclose(h @ h, w_squared(mv, k) * I16, atol=1e-12)
    maps_ok = (maps_ok and np.allclose(Gamma @ h @ Gamma, h_matrix(-mv, k))
               and np.allclose(gamma[0] @ h @ gamma[0], h_matrix(-mv, R0 * k)))
samples = read_json(THEORY)["data"]["one_particle_flat"]["samples"]
rows_ok = True
for row in samples:
    k = np.array(row["k"], dtype=float)
    w = float(sp.sympify(row["w"].replace("Sqrt[", "sqrt(").replace("]", ")")))
    h = h_matrix(row["m"], k)
    Vp, Vn = eigenspace(h, w), eigenspace(h, -w)
    mine = (np.isclose(np.sqrt(w_squared(row["m"], k)), w), Vp.shape[1],
            Vn.shape[1], inertia(Vp), inertia(Vn))
    rows_ok = rows_ok and mine == (True, row["dim_plus_w"], row["dim_minus_w"],
                                   tuple(row["B_inertia_plus_w"]),
```

```
                                tuple(row["B_inertia_minus_w"]))
    (pp, pn), (qp, qn) = inertia(Vp), inertia(Vn)
    say(f"m = {row['m']:+d}, k = {row['k']}: w = {w:.6f}, dimensions "
        f"{Vp.shape[1]} and {Vn.shape[1]}, Krein inertia ({pp},{pn}) and "
        f"({qp},{qn})")
reproduces(square_ok and maps_ok,
           "h_m^2 = w^2 I16; Gamma h_m Gamma = h_-m; gamma^(x0) h_m gamma^(x0) = "
           "h_-m(R0 k)", (WL, ["Q_one_particle_flat_dispersion", "Q_one_particle_maps"]),
           (PY, ["Q.one_particle_maps"]))
reproduces(rows_ok and len(samples) == 8,
           "the eight recorded samples: w, dimensions 8 and 8, Krein inertia (4,4)",
           (WL, ["Q_one_particle_Krein_signatures"]),
           (PY, ["compare.theory.one_particle", "Q.one_particle_Krein_inertia_proof"]))
```

Out [16]:

```
m = +2, k = [4, 1, 2, 0, 0, 0, 0, 0]: w = 5.000000, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = -2, k = [4, 1, 2, 0, 0, 0, 0, 0]: w = 5.000000, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = +3, k = [0, 0, 0, 0, 0, 0, 0, 0]: w = 3.000000, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = -3, k = [0, 0, 0, 0, 0, 0, 0, 0]: w = 3.000000, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = +1, k = [0, 1, 0, 0, 0, 0, 0, 0]: w = 1.414214, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = -1, k = [0, 1, 0, 0, 0, 0, 0, 0]: w = 1.414214, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = +2, k = [0, 0, 0, 0, 0, 1, 0, 0]: w = 1.732051, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
m = -2, k = [0, 0, 0, 0, 0, 1, 0, 0]: w = 1.732051, dimensions 8 and 8, Krein inertia
    (4,4) and (4,4)
PASS h_m^2 = w^2 I16; Gamma h_m Gamma = h_-m; gamma^(x0) h_m gamma^(x0) = h_-m(R0 k)
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_one_particle_flat_dispersion, Q_one_particle_maps; Revision/pairing/reports/python-
    pairing.json, check Q.one_particle_maps
PASS the eight recorded samples: w, dimensions 8 and 8, Krein inertia (4,4)
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_one_particle_Krein_signatures; Revision/pairing/reports/python-pairing.json, check
    compare.theory.one_particle, Q.one_particle_Krein_inertia_proof
```

Text cell:

```
The next cell checks the three recorded samples with imaginary or zero frequency
(an extra-time momentum $k_5$ larger than the mass): $m = 1$ with $k_5 = 2$
($w^2 = -3$), with $k_1 = 1, k_5 = 2$ ($w^2 = -2$), and with $k_5 = 1$
($w^2 = 0$). For an imaginary $w$ each eigenspace has dimension 8 and the Krein
form $V^\dagger BV$ is ZERO on it (*Krein-neutral*); for $w = 0$ the matrix $h$
has the null space of dimension 8, also Krein-neutral, and $h^2 = 0$.
```

In [17]:

```
neutral_ok = True
for k in ([0, 0, 0, 0, 0, 2, 0, 0], [0, 1, 0, 0, 0, 2, 0, 0],
          [0, 0, 0, 0, 0, 1, 0, 0]):  # (k0, ..., k7)
    k = np.array(k, dtype=float)
    h, w2 = h_matrix(1.0, k), w_squared(1.0, k)
    if w2 == 0:
        spaces = [eigenspace(h, 0.0)]
        neutral_ok = neutral_ok and np.allclose(h @ h, 0.0)
    else:
        w = 1j * np.sqrt(-w2)
        spaces = [eigenspace(h, w), eigenspace(h, -w)]
    dims = [V.shape[1] for V in spaces]
    largest = max(np.abs(V.conj().T @ B @ V).max() for V in spaces)
    neutral_ok = neutral_ok and all(dim == 8 for dim in dims) and largest < 1e-9
    say(f"k = {k.astype(int).tolist()}: w^2 = {w2:+.0f}, eigenspace dimensions "
        f"{dims}, Krein form zero on them: {largest < 1e-9}")
reproduces(neutral_ok, "imaginary and zero frequencies: eigenspaces of dimension 8 "
           "are Krein-neutral",
           (PY, ["Q.one_particle_complex_frequency_Krein_neutral"]),
           (WL, ["Q_one_particle_complex_and_zero_frequencies_Krein_neutral"]))
```

Out [17]:

```
k = [0, 0, 0, 0, 0, 2, 0, 0]: w^2 = -3, eigenspace dimensions [8, 8], Krein form zero on
    them: True
k = [0, 1, 0, 0, 0, 2, 0, 0]: w^2 = -2, eigenspace dimensions [8, 8], Krein form zero on
    them: True
k = [0, 0, 0, 0, 0, 1, 0, 0]: w^2 = +0, eigenspace dimensions [8], Krein form zero on
    them: True
PASS imaginary and zero frequencies: eigenspaces of dimension 8 are Krein-neutral
     reproduces Revision/pairing/reports/python-pairing.json, check
    Q.one_particle_complex_frequency_Krein_neutral; Revision/pairing/reports/wolfram-
    pairing.json, check Q_one_particle_complex_and_zero_frequencies_Krein_neutral
```

Text cell:

```
The next cell draws the mapped spectra of plane waves: the eigenvalues of
$h_m(k)$ for the mass $m = 1$ (lines) and of $h_{-m}(k)$ (dots), against a
3-space momentum $k_1$ (left) and against an extra-time momentum $k_5$ (right).
The cell first checks that the two sets of 16 eigenvalues agree at every momentum
(away from the single point $k_5 = 1$, where $w = 0$ and the eigenvalues of a
matrix with $h^2 = 0$ cannot be computed accurately in floating point).
```

In [18]:

```
ks = np.linspace(0.0, 3.0, 121)
curves = {}
for label, mass in (("plus", 1.0), ("minus", -1.0)):
    for axis in (1, 5):
        rows = []
        for kv in ks:
```

```
            k = np.zeros(8)
            k[axis] = kv
            rows.append(spectrum(h_matrix(mass, k)))
        curves[label, axis] = np.array(rows)
away = np.abs(ks - 1.0) > 0.05
equal = max(np.abs(curves["plus", axis][away] - curves["minus", axis][away]).max()
            for axis in (1, 5))
check(equal < 1e-6, "the plane-wave spectra of the masses +1 and -1 are equal")
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for ax, axis, name in ((axes[0], 1, "3-space momentum $k_1$"),
                       (axes[1], 5, "extra-time momentum $k_5$")):
    ax.plot(ks, np.sqrt(np.clip(1.0 + (ks ** 2 if axis == 1 else -ks ** 2), 0, None)),
            color="#2a78d6", label="real part, mass $+1$")
    ax.plot(ks, np.sqrt(np.clip(-1.0 + (ks ** 2 if axis == 5 else -ks ** 2), 0,
                                None)), color="#eb6834",
            label="imaginary part, mass $+1$")
    dots = curves["minus", axis]
    ax.plot(ks[::5], np.abs(dots.real).max(axis=1)[::5], "o", color="#2a78d6",
            markersize=3.5, alpha=0.6, label="mass $-1$ (dots)")
    ax.plot(ks[::5], np.abs(dots.imag).max(axis=1)[::5], "o", color="#eb6834",
            markersize=3.5, alpha=0.6)
    ax.set_xlabel(name + " (units $m$)")
    ax.set_ylim(-0.1, 3.3)
axes[0].set_ylabel("frequency $|w|$: real and imaginary parts")
axes[0].legend(fontsize=8, loc="upper left")
save_figure(fig, "flat_dispersion",
            "Mapped spectra of plane waves in flat 4+4 space: the frequencies, the "
            "eigenvalues of the one-particle matrix $h_m(k)$, for the mass $+1$ "
            "(lines) and $-1$ (dots), against a 3-space momentum $k_1$ (left) and "
            "an extra-time momentum $k_5$ (right), in units of the mass. Blue: "
            "the real part $\\sqrt{1 + k_1^2}$ or $\\sqrt{1 - k_5^2}$; orange: the "
            "imaginary part, which appears when the extra-time momentum exceeds the "
            "mass (a growing mode). Each value is 8-fold, with both signs. The "
            "dots lie on the lines: the universes of mass $+m$ and $-m$ have the "
            "same one-particle spectrum, because $\\Gamma h_m\\Gamma = h_{-m}$.")
```

Out [18]:

```
PASS the plane-wave spectra of the masses +1 and -1 are equal
```

Out [18]:

```
Figure 18c.6 saved as Revision/textbook/figures/18c_6_flat_dispersion.png
```

Text cell:

```
The next cell looks at the eigenvectors themselves, at the first recorded sample
($m = 2$, $k = (1, 2, 0, 0, 0, 0, 0, 4)$, $w = 5$). In each eigenspace it chooses
8 columns $u$ with Krein norms $u^\dagger Bu = \pm1$ (the eigenvectors of
$V^\dagger BV$, rescaled), 16 in all. Then: $\Gamma u$ is an eigenvector of
$h_{-m}(k)$ with the same frequency and the OPPOSITE Krein norm (Lemma 4,
```

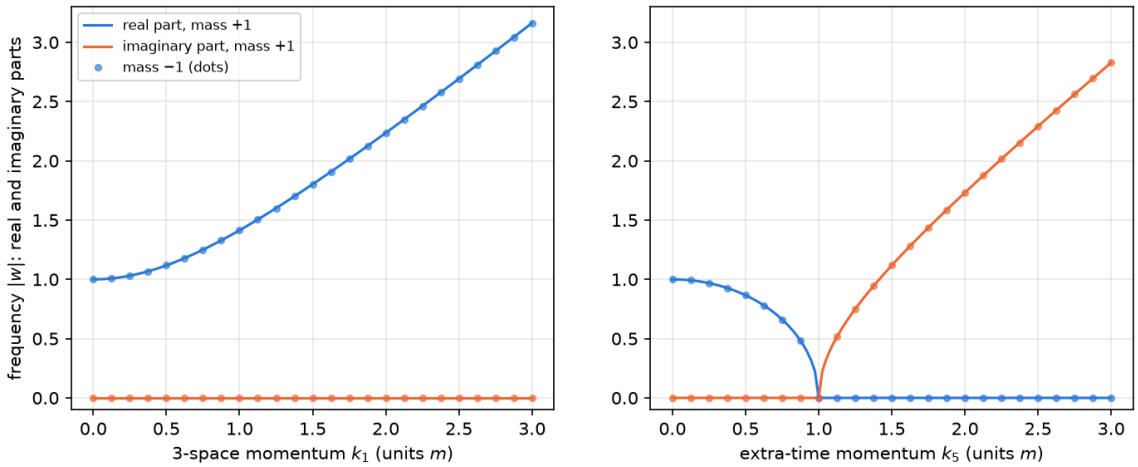

Figure 489: Mapped spectra of plane waves in flat 4+4 space: the frequencies, the eigenvalues of the one-particle matrix $h_m(k)$, for the mass $+1$ (lines) and -1 (dots), against a 3-space momentum k_1 (left) and an extra-time momentum k_5 (right), in units of the mass. Blue: the real part $\sqrt{1+k_1^2}$ or $\sqrt{1-k_5^2}$; orange: the imaginary part, which appears when the extra-time momentum exceeds the mass (a growing mode). Each value is 8-fold, with both signs. The dots lie on the lines: the universes of mass $+m$ and $-m$ have the same one-particle spectrum, because $\Gamma h_m \Gamma = h_{-m}$. (Notebook 18c, figure 6.)

```
$\Gamma B\Gamma = -B$); $\gamma^{(x_0)}u$ is an eigenvector of $h_{-m}(R_8k)$
with the same frequency and the SAME Krein norm ($\gamma^{(x_0)}B
\gamma^{(x_0)} = B$).
```

In [19]:

```
row = samples[0]
k = np.array(row["k"], dtype=float)
w = 5.0
h = h_matrix(row["m"], k)
columns, freqs = [], []
for value in (w, -w):
    V = eigenspace(h, value)
    ev, U = np.linalg.eigh(V.conj().T @ B @ V)  # diagonalise the Krein form
    for j in np.argsort(-ev):  # positive norms first
        u = V @ U[:, j] / np.sqrt(abs(ev[j]))  # Krein norm +1 or -1
        columns.append(u)
        freqs.append(value)
norms = np.array([(u.conj() @ B @ u).real for u in columns])
norms_G = np.array([((Gamma @ u).conj() @ B @ (Gamma @ u)).real for u in columns])
norms_0 = np.array([((gamma[0] @ u).conj() @ B @ (gamma[0] @ u)).real
                    for u in columns])
eig_G = all(np.allclose(h_matrix(-row["m"], k) @ (Gamma @ u), f * (Gamma @ u))
            for u, f in zip(columns, freqs))
eig_0 = all(np.allclose(h_matrix(-row["m"], R0 * k) @ (gamma[0] @ u),
                        f * (gamma[0] @ u)) for u, f in zip(columns, freqs))
say("Krein norms of the 16 columns: " + " ".join(f"{x:+.0f}" for x in norms))
```

```
reproduces(eig_G and eig_0 and np.allclose(norms_G, -norms)
           and np.allclose(norms_0, norms) and int((norms > 0).sum()) == 8,
           "Gamma u: eigenvector of h_-m, Krein norm reversed; gamma^(x0) u: kept",
           (WL, ["Q_Krein_metric_of_images"]),
           (PY, ["Q.image_krein_metric", "Q.T2_image_keeps_B"]))
```

Out [19]:

```
Krein norms of the 16 columns: +1 +1 +1 +1 -1 -1 -1 -1 +1 +1 +1 +1 -1 -1 -1 -1
PASS Gamma u: eigenvector of h_-m, Krein norm reversed; gamma^(x0) u: kept
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_Krein_metric_of_images; Revision/pairing/reports/python-pairing.json, check
    Q.image_krein_metric, Q.T2_image_keeps_B
```

Text cell:

```
The next cell draws the 16 Krein norms and those of the two images as bars.
```

In [20]:

```
fig, ax = plt.subplots(figsize=(10.0, 4.2))
idx = np.arange(16)
ax.bar(idx - 0.27, norms, width=0.27, color="#2a78d6", label="$u^\\dagger Bu$")
ax.bar(idx, norms_G, width=0.27, color="#eb6834",
       label="$(\\Gamma u)^\\dagger B(\\Gamma u)$: T1 image")
ax.bar(idx + 0.27, norms_0, width=0.27, color="#1baf7a",
       label="$(\\gamma^{(x_0)}u)^\\dagger B(\\gamma^{(x_0)}u)$: T2 image")
ax.axvline(7.5, color="#52514e", linewidth=0.8)
ax.text(3.5, 1.35, "frequency $+5$", ha="center", fontsize=9)
ax.text(11.5, 1.35, "frequency $-5$", ha="center", fontsize=9)
ax.set_xticks(idx, [str(j + 1) for j in idx])
ax.set_xlabel("column number $j$ (eight per eigenspace)")
ax.set_ylabel("Krein norm")
ax.set_ylim(-1.6, 1.6)
ax.legend(fontsize=8, loc="lower left", ncol=3)
save_figure(fig, "krein_norms",
            "Krein norms of plane-wave eigenvectors and of their images, for the "
            "recorded sample $m = 2$, $k = (1, 2, 0, 0, 0, 0, 0, 4)$, frequency "
            "$w = \\pm5$. Horizontal axis: the 16 eigenvector columns $u$, eight "
            "for $+5$ and eight for $-5$; vertical axis: the Krein norm. Blue: "
            "$u^\\dagger Bu$, four $+1$ and four $-1$ in each eigenspace (Krein "
            "inertia (4,4)). Orange: the chirality image $\\Gamma u$, an "
            "eigenvector of the mass $-m$ with every norm reversed. Green: the "
            "mirror image $\\gamma^{(x_0)}u$, an eigenvector of the mass $-m$ with "
            "every norm kept. This is the quantum reading Q at the level of single "
            "modes.")
```

Out [20]:

```
Figure 18c.7 saved as Revision/textbook/figures/18c_7_krein_norms.png
```

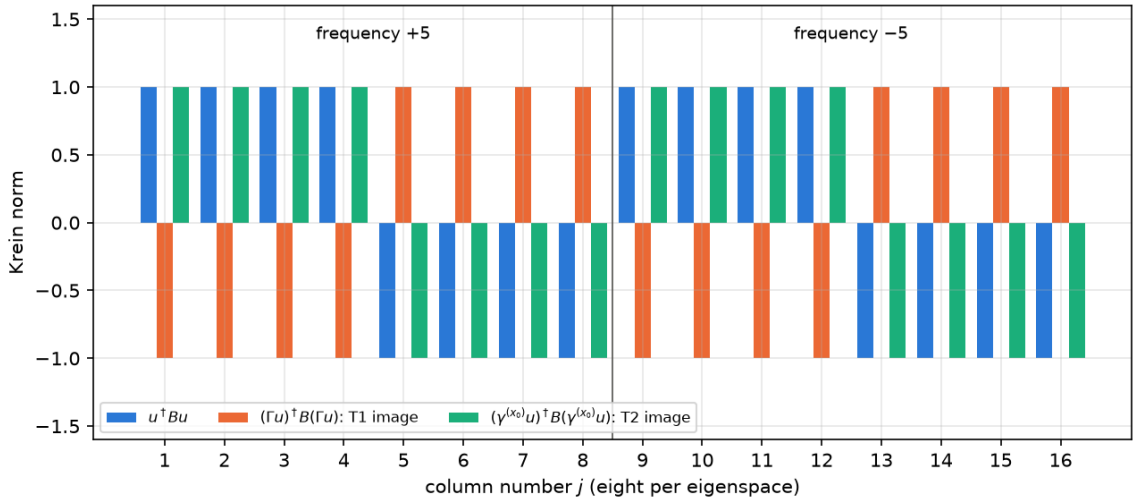

Figure 490: Krein norms of plane-wave eigenvectors and of their images, for the recorded sample $m = 2$, $k = (1, 2, 0, 0, 0, 0, 0, 4)$, frequency $w = \pm 5$. Horizontal axis: the 16 eigenvector columns u, eight for $+5$ and eight for -5; vertical axis: the Krein norm. Blue: $u^\dagger Bu$, four $+1$ and four -1 in each eigenspace (Krein inertia (4,4)). Orange: the chirality image Γu, an eigenvector of the mass $-m$ with every norm reversed. Green: the mirror image $\gamma^{(x_0)}u$, an eigenvector of the mass $-m$ with every norm kept. This is the quantum reading Q at the level of single modes. (Notebook 18c, figure 7.)

Text cell:

```
## 12. The last check

The last cell checks that the seven figure files exist and prints the number of
checks that passed.
```

In [21]:

```
figure_names = ["rk4_convergence", "partner_components", "t1_pair_densities",
                "t2_mirror_densities", "family_spectrum", "flat_dispersion",
                "krein_norms"]
paths = [output_file(f"{FIGURE_FOLDER}/18c_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [21]:

```
PASS every figure file of this notebook exists
ALL 21 CHECKS PASSED (notebook 18c)
```

Text cell:

```
## 13. What this notebook showed

- COMPUTED (RK4, fourth order measured, checked against the exact solution of the
  Revision theory record): a homogeneous solution of dirac16complex00 in the
  author's metric with $m = 1$, $H = 0.2$, $\lambda = 0.3$ (illustrative values),
  with constant $S$, oscillating at the frequency $w = \sqrt{(m + \lambda S)^2 -
  9H^2}$, and with the charge flowing along the hidden direction exactly as the
  local conservation law requires. The flow through the brane $z = \pi/2$ is not
  zero, so the total charge of the patch is not constant for this solution; a
  constant total charge would need a no-flux condition at the brane, which is
  not derived (OPEN).
- COMPUTED, theorem T1: the theory with $(-m, -\lambda)$, solved on its own from
  $\Gamma\Psi(0)$, gives $\Gamma\Psi(x_4)$ at all times; its charge density,
  current and all 36 components of $T_{\mu\nu}$ along the deflating history
  $a_4 = AHx_4$ are exactly opposite, so the pair sums to zero at every time (as
  classical bilinears).
- COMPUTED, theorem T2: the theory with $(-m, \lambda)$ on the mirror patch, solved
  on its own from $\gamma^{(x_0)}\Psi(0)$, gives $\gamma^{(x_0)}\Psi(x_4)$; $S$ is
  opposite, the charge density and the energy density are EQUAL, the other
  components are pulled back by the reflection of $x_0$.
- COMPUTED and PROVED (similar matrices): the field and its partners have equal
  spectra; for plane waves in flat 4+4 space the masses $+m$ and $-m$ have the
  same frequencies, every real frequency has Krein inertia (4,4) in the eight
  recorded samples, the imaginary and zero frequencies are Krein-neutral, and
  $\Gamma$ reverses every Krein norm while $\gamma^{(x_0)}$ keeps it (the quantum
  reading Q).
- ASSUMED: the mirror patch glued at the degenerate brane (Z2 construction); the
  gravitational field is a fixed background.
- NOT shown: any process that creates a universe or a pair. The partners here are
  solutions of OTHER parameter sets, computed from the first one; nothing in these
  equations makes them appear.
```

18.27 Line-by-line walk-through of Notebook 18c

The notebook has 21 code cells. **In [1]** is the set-up cell, word for word the set-up cell of Notebook 18a explained line by line in Section 18.9, except that its comment lines hold the run instructions of Section 18.25 and its line `NOTEBOOK_ID = "18c"` names this notebook, so its figures are saved as `18c_<k>_<name>.png` and their captions in `18c.captions.json`. It prints one line, Set-up of notebook 18c complete: repository folder found, helpers defined.

The idea of the notebook in one paragraph. Notebooks 18a and 18b computed with exact numbers: whole numbers, fractions and symbols. This notebook computes with **floating-point numbers**, the ordinary numbers of a computer, which carry about 16 significant decimal digits and are rounded after every operation. So an identity such as $\Phi = \Gamma\Psi$ is checked by requiring the largest difference to be smaller than a **tolerance**, usually 10^{-12}: far below the size of the quantities compared (about 0.1 to 3) and far above the rounding errors (about 10^{-16}). The notebook takes one solution of the homogeneous family of Section 18.21, computes it in two independent ways (step by step with the Runge-Kutta method, and from the exact formula), then solves the two partner equations of T1 and T2 on their own, and compares the solutions and all their bilinears along the deflating history.

In [2], the records and the matrices.

```
import numpy as np  # arrays, matrices, eigenvalues
import sympy as sp  # exact symbolic algebra for the spin connection

GAMMAS = "Revision/algebra/gammas.json"
PY = "Revision/pairing/reports/python-pairing.json"
WL = "Revision/pairing/reports/wolfram-pairing.json"
THEORY = "Revision/pairing/pairing-theory.json"
TH_PY = "Revision/theory/reports/python-field-theory.json"
TH_WL = "Revision/theory/reports/wolfram-field-theory.json"
SCOPE = "Revision/theory/reports/python-scope.json"
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
```

numpy (`np`) is the package for arrays of floating-point numbers: matrix products, eigenvalues, the singular value decomposition. sympy (`sp`) is used once more, in In [3], to compute the spin connection exactly before it is turned into numbers. The eight constants are the repository paths of the eight Revision records the notebook reads: the gammas; the two reports of the pairing verifiers and the pairing theory record (for the plane-wave samples of the quantum reading); the two field-theory reports and the scope report (for the exact homogeneous solution and the conservation of the charge); and the Kohn-Sham parameter record, which holds the canonical deflating history.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


RECORDS = {f: read_json(f) for f in (PY, WL, TH_PY, TH_WL, SCOPE)}


def recorded(report_file, name):
    """The recorded entry (verdict and detail) of the check name of a report."""
    for entry in RECORDS[report_file]["checks"]:
        if entry["name"] == name:
            return entry
    return {"verdict": "MISSING", "detail": ""}


def reproduces(condition, name, *sources):
    """check(condition, name), which also requires every named check of every
    source (report_file, [check names]) to have the recorded verdict PASS."""
    ok = all(recorded(f, n)["verdict"].upper() == "PASS"
             for f, names in sources for n in names)
    text = "; ".join(f"{f}, check {', '.join(names)}" for f, names in sources)
    check(condition and ok, name, record=text)
```

The record helpers of Notebook 18b. `read_json` reads a record. `RECORDS` reads the five reports once, with the file name as the key (a dictionary comprehension). `recorded(report_file, name)` returns the entry of a named check, or a placeholder with the verdict MISSING. `reproduces` passes only when the notebook's own result holds AND every named check has the recorded verdict PASS; `.upper()` makes the comparison independent of capitalisation, which matters here, because the field-theory report writes its verdicts as pass in small letters.

```
fixture = read_json(GAMMAS)
gamma = {a: np.array(fixture["gamma"][a], dtype=float) for a in range(8)}
ETA = {a: int(fixture["eta"][a]) for a in range(8)}
I16 = np.eye(16)
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]
Gamma = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5] @ gamma[6] \
    @ gamma[7]
B = -1j * C @ gamma[4]  # the Krein matrix
check(all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                         2.0 * (ETA[a] if a == b else 0) * I16)
          for a in range(8) for b in range(8))
      and np.array_equal(Gamma, np.diag([-1.0] * 8 + [1.0] * 8))
      and np.array_equal(B.conj().T, B),
      "eight real 16 x 16 gammas (Clifford relations), Gamma = diag(-I8, I8), "
      "B Hermitian")
```

The gammas are read as numpy arrays of floating-point numbers (`dtype=float`). Their entries are -1, 0 and $+1$, which floating-point numbers store exactly, and sums and products of such whole numbers are again exact; so `np.array_equal`, an exact comparison, is still correct for them. `C`, `Gamma` and `B` are built as in Section 18.5 (a backslash at the end of a line continues the statement on the next line; `1j` is i). The check confirms the 64 Clifford relations, $\Gamma = \mathrm{diag}(-I_8, I_8)$ and $B^\dagger = B$ (`.conj().T` is the conjugate transpose). Out [2] shows one PASS line.

In [3], the spin connection on the patch and on the mirror patch.

```
x4, z, H_sym = sp.symbols("x4 z H", real=True)
a4 = sp.Function("a4")(x4)
a4p = sp.Symbol("a4p")


def d(expr, b):
    """The derivative of a coefficient along the coordinate x_b (z = 6 H x0)."""
    if b == 4:
        return sp.diff(expr, x4)
    if b == 0:
        return 6 * H_sym * sp.diff(expr, z)
    return sp.Integer(0)
```

As in Notebook 18b: the symbols x_4, z and H (here named `H_sym`, because from In [4] on the name `H` holds the number 0.2), the unknown history $a_4(x_4)$, the plain symbol `a4p` for a_4', and the derivative `d(expr, b)` along the coordinate x_b: d/dx_4 along x_4, $6H\,d/dz$ along x_0 (because $z = 6Hx_0$), and zero along the six coordinates on which no coefficient depends.

```
def factors(s0):
    s6 = sp.sin(z) ** sp.Rational(1, 6)
    f = {a: sp.exp(a4) * s6 for a in (1, 2, 3)}
    f.update({a: sp.exp(-a4) * s6 for a in (5, 6, 7)})
    f.update({4: sp.Integer(1), 0: s0 * sp.cot(z)})
    return f
```

`factors(s0)` returns the eight vielbein factors of Section 18.3: $e^{a_4}\sin^{1/6} z$ for x_1, x_2, x_3; $e^{-a_4}\sin^{1/6} z$ for the deflating extra times x_5, x_6, x_7; 1 for x_4; and $s_0 \cot z$ for x_0, with $s_0 = +1$ on the patch and

$s_0 = -1$ on the mirror patch, where $\cot z < 0$, so that $f_0 > 0$ on both. `f.update({...})` adds entries to the dictionary.

```
def connection(s0):
    """{(mu, a, b) with a < b: omega_mu ab} for the diagonal vielbein."""
    f, om = factors(s0), {}
    for a in range(8):
        for b in range(8):
            if a != b:
                value = sp.simplify(ETA[a] * d(f[a], b) / f[b])
                if value != 0:  # omega_a,ab (or -omega_a,ba when b < a)
                    om[(a, a, b) if a < b else (a, b, a)] = (
                        value if a < b else -value)
    return dict(sorted(om.items()))
```

`connection(s0)` uses the short formula derived in Section 18.3, $\omega_{a,ab} = \eta_{aa}\,\partial_b f_a / f_b$ for $a \neq b$, instead of the general Christoffel computation of Notebook 18b. Each nonzero value is stored under the key (μ, a, b) with $a < b$, the format of the record: when $a < b$ the value is $\omega_{a,ab}$ itself, stored under (a, a, b); when $b < a$ the component with the smaller index first is $\omega_{a,ba} = -\omega_{a,ab}$, stored under (a, b, a). `dict(sorted(...))` orders the keys, so that the list is printed in the order of the record.

```
def short(expr):
    expr = sp.simplify(expr).subs(sp.Derivative(a4, x4), a4p)
    return expr.subs(a4, sp.Symbol("a4"))


patch_om, mirror_om = connection(1), connection(-1)
listing = "; ".join(f"omega_x{mu} x{a}x{b} = {short(v)}"
                    for (mu, a, b), v in patch_om.items())
say("patch: " + listing)
same = recorded(PY, "geometry.spin_connection_components")["detail"] == (
    "nonzero components (a < b): " + listing)
reproduces(len(patch_om) == 12 and same,
           "the formula gives the 12 recorded spin connection components",
           (PY, ["geometry.spin_connection_components"]))
```

`short` writes a_4' as `a4p` and $a_4(x_4)$ as `a4`, as in Notebook 18b. The twelve components of the patch are printed in one line (the output wraps it) and compared, character for character, with the detail text of the sympy check `geometry.spin_connection_components`; the check requires 12 components and the agreement. So two different computations, the general Christoffel formula of Notebook 18b and the short formula of this cell, reproduce the same record.

```
flips = all(sp.simplify(mirror_om[k] + patch_om[k]) == 0 if 0 in k[1:]
            else sp.simplify(mirror_om[k] - patch_om[k]) == 0 for k in patch_om)
```

`flips` compares the two patches component by component: the six components with an index x_0 (`0 in k[1:]` looks at the two frame indices of the key) must change sign, because they contain $1/f_0$, and the six others must be equal.

```
divergence = {}
for s0 in (1, -1):
    f = factors(s0)
    sqrtg = sp.simplify(sp.Mul(*f.values()))
    divergence[s0] = {mu: sp.simplify(d(sqrtg / f[mu], mu) / (2 * sqrtg))
                      for mu in range(8)}
say(f"divergence form, patch: coefficient of gamma^(x0) = {divergence[1][0]}, of "
    f"gamma^(x4) = {divergence[1][4]}; mirror patch: {divergence[-1][0]}, "
    f"{divergence[-1][4]}")
reproduces(flips and divergence[1][0] == 3 * H_sym and divergence[-1][0] == -3 * H_sym
           and all(divergence[s][mu] == 0 for s in (1, -1) for mu in range(1, 8)),
           "gamma^mu Omega_mu = +3 H gamma^(x0) (patch), -3 H gamma^(x0) (mirror)",
           (TH_PY, ["gamma_mu_Omega_mu_equals_3H_gamma_x0",
                    "divergence_of_sqrtg_gamma"]),
           (TH_WL, ["gammaOmega_equals_3H_gamma_x0", "gammaOmega_divergence_form"]))
```

The **divergence form** of the theory record, $\gamma^\mu\Omega_\mu = \frac{1}{2\sqrt{|g|}}\sum_\mu \partial_\mu(\sqrt{|g|}\,\gamma^{(\mu)}/f_\mu)$, computed on both patches: `divergence[s0][mu]` is the coefficient of $\gamma^{(\mu)}$, $\partial_\mu(\sqrt{|g|}/f_\mu)/(2\sqrt{|g|})$. Why this formula holds for a diagonal vielbein, line by line. Only the components $\omega_{\mu,\mu b}$ (and their partners $\omega_{\mu,b\mu} = -\omega_{\mu,\mu b}$) are nonzero (Section 18.3), so

$$\gamma^\mu\Omega_\mu = \sum_\mu \frac{\gamma^{(\mu)}}{f_\mu}\sum_{b\neq\mu}\omega_{\mu,\mu b}S^{\mu b}.$$

The two equal terms $\omega_{\mu,\mu b}S^{\mu b}$ and $\omega_{\mu,b\mu}S^{b\mu}$ of $\Omega_\mu = \frac12\sum_{a,b}\omega_{\mu ab}S^{ab}$ cancel the factor $\frac12$, as in Section 18.3.

$$\gamma^{(\mu)}S^{\mu b} = \tfrac12\gamma^{(\mu)}\gamma^{(\mu)}\gamma^{(b)} = \tfrac12\eta_{\mu\mu}\gamma^{(b)}.$$

$S^{\mu b} = \frac12\gamma^{(\mu)}\gamma^{(b)}$ for $b\neq\mu$; then the Clifford relation $\gamma^{(\mu)}\gamma^{(\mu)} = \eta_{\mu\mu}$.

$$\gamma^\mu\Omega_\mu = \sum_b \gamma^{(b)}\sum_{\mu\neq b}\frac{\eta_{\mu\mu}}{2f_\mu}\,\eta_{\mu\mu}\frac{\partial_b f_\mu}{f_b} = \sum_b\frac{\gamma^{(b)}}{2f_b}\sum_{\mu\neq b}\frac{\partial_b f_\mu}{f_\mu}.$$

We inserted $\omega_{\mu,\mu b} = \eta_{\mu\mu}\partial_b f_\mu/f_b$, exchanged the order of the two sums, and used $\eta_{\mu\mu}^2 = 1$.

$$\sum_{\mu\neq b}\frac{\partial_b f_\mu}{f_\mu} = \partial_b\ln\prod_{\mu\neq b}f_\mu = \partial_b\ln\frac{\sqrt{|g|}}{f_b}, \qquad \frac{1}{2f_b}\,\partial_b\ln\frac{\sqrt{|g|}}{f_b} = \frac{1}{2\sqrt{|g|}}\,\partial_b\frac{\sqrt{|g|}}{f_b}.$$

The first equation is the rule $\partial\ln F = \partial F/F$ applied to each factor (the logarithm of a product is the sum of the logarithms), with $\sqrt{|g|} = \prod_\mu f_\mu$; the second is the same rule read backwards, with $F = \sqrt{|g|}/f_b$ and $\frac{1}{f_b}\cdot\frac{f_b}{\sqrt{|g|}} = \frac{1}{\sqrt{|g|}}$. Together they give the divergence form. On the patch, $\sqrt{|g|}/f_0 = \cos z/\cot z = \sin z$ and $\partial_0\sin z = 6H\cos z$, so the coefficient of $\gamma^{(x_0)}$ is $6H\cos z/(2\cos z) = 3H$; $\sqrt{|g|}/f_4 = \cos z$ does not depend on x_4, so the coefficient of $\gamma^{(x_4)}$ is 0. On the mirror patch $\sqrt{|g|} = -\cos z$ and $f_0 = -\cot z$ give the same quotient $\sin z$, but it is divided by $2\sqrt{|g|} = -2\cos z$:

the coefficient is $-3H$. The check requires the sign flips, the two coefficients $\pm 3H$ and zero for the seven other directions, and names two checks of each field-theory report. Out [3] prints the twelve components, then $3H$, *0, -3*H, 0, and two PASS lines.

In [4], the numbers of the illustration and the geometry as numbers.

```
A_hist = read_json(PARAMETERS)["physics"]["historyA"]  # A = 1
m, H, lam = 1.0, 0.2, 0.3  # an illustration (units m = 1)
Z0 = np.pi / 4  # the patch point; the mirror point is pi - Z0
rng = np.random.default_rng(2026)
raw = rng.normal(size=16) + 1j * rng.normal(size=16)
P_C = 0.5 * (I16 + C)  # the projector on the eigenvalue +1 of C
psi0 = 2.0 * P_C @ raw + 0.5 * (I16 - P_C) @ raw  # mostly in the +1 part: S > 0
psi0 = psi0 / np.linalg.norm(psi0)  # length 1
S0 = float((psi0.conj() @ C @ psi0).real)  # S of the initial value
report("S of the initial value", f"{S0:.6f}")
report("m + lambda S", f"{m + lam * S0:.6f}")
report("3 H", f"{3 * H:.6f}")
```

`A_hist` is the number $A = 1$ of the canonical history $a_4 = AHx_4$, read from the Kohn-Sham parameter record. The next two lines fix the ILLUSTRATION chosen by this notebook: $m = 1$ (so lengths and times are measured in units of $1/m$), $H = 0.2$, $\lambda = 0.3$ and the patch point $z_0 = \pi/4$. `np.random.default_rng(2026)` is a random-number generator with the fixed seed 2026; `raw` is a column of 16 complex numbers whose real and imaginary parts are drawn from the normal distribution. The initial value is built so that $S > 0$. Since C is real and symmetric with $CC = 1$, the matrices $P_{C\pm} = \frac{1}{2}(1 \pm C)$ are projectors: $P_{C\pm}^\dagger = P_{C\pm}$, $P_{C\pm}P_{C\pm} = P_{C\pm}$, $P_{C+}P_{C-} = \frac{1}{4}(1 - CC) = 0$ and $P_{C+} - P_{C-} = C$. Hence

$$S = \Psi^\dagger C \Psi = \Psi^\dagger P_{C+} \Psi - \Psi^\dagger P_{C-} \Psi = |P_{C+}\Psi|^2 - |P_{C-}\Psi|^2,$$

where $|v|^2 = v^\dagger v$ is the squared length; the last step writes $\Psi^\dagger P_{C\pm} \Psi = \Psi^\dagger P_{C\pm}^\dagger P_{C\pm} \Psi = (P_{C\pm}\Psi)^\dagger (P_{C\pm}\Psi)$. Weighting the part of P_{C+} by 2 and the part of P_{C-} by 0.5 makes the first term larger. `np.linalg.norm` is the length of a column, and dividing by it gives a column of length 1. `S0` is S of the initial value (`.real` drops the imaginary part, which is zero up to rounding because C is real and symmetric). Out [4] prints $S = 0.897319$, $m + \lambda S = 1.269196$ and $3H = 0.600000$.

```
S_AB = {(a, b): 0.5 * gamma[a] @ gamma[b] for a in range(8) for b in range(8)}
history = {sp.Derivative(a4, x4): A_hist * H_sym}  # a4' = A H


def numeric(table):
    """Turn the sympy components into functions of (x4, z) for this H and A."""
    out = {}
    for key, value in table.items():
        e = value.subs(history).subs(a4, A_hist * H_sym * x4).subs(H_sym, H)
        out[key] = sp.lambdify((x4, z), e, "numpy")
    return out


OMEGA = {1: numeric(patch_om), -1: numeric(mirror_om)}
```

```
FACTOR = {s0: {mu: sp.lambdify((x4, z), e.subs(a4, A_hist * H_sym * x4)
                                .subs(H_sym, H), "numpy")
               for mu, e in factors(s0).items()} for s0 in (1, -1)}
```

`S_AB` holds $S^{ab} = \frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ (correct for $a \neq b$; the entries with $a = b$ are never used). `history` replaces a_4' by AH. `numeric(table)` turns each sympy component into a numpy function of (x_4, z): it substitutes $a_4' = AH$, then $a_4 = AHx_4$ and $H = 0.2$, and `sp.lambdify` makes a function that evaluates the resulting expression for numbers. `OMEGA` holds these functions for both patches (keys 1 and -1), and `FACTOR` the eight vielbein factors of both patches as functions of (x_4, z).

```
def geometry_at(t, zv, s0):
    """Numbers at the time t and angle zv: f, g, gamma^mu, gamma_mu, Omega_mu."""
    f = {mu: float(FACTOR[s0][mu](t, zv)) for mu in range(8)}
    Om = {mu: np.zeros((16, 16)) for mu in range(8)}
    for (mu, a, b), fn in OMEGA[s0].items():
        Om[mu] = Om[mu] + float(fn(t, zv)) * S_AB[a, b]
    return {"f": f, "g": {mu: ETA[mu] * f[mu] ** 2 for mu in f},
            "sqrtg": abs(np.prod(list(f.values()))),
            "up": {mu: gamma[mu] / f[mu] for mu in f},
            "down": {mu: ETA[mu] * f[mu] * gamma[mu] for mu in f}, "Om": Om}
```

`geometry_at(t, zv, s0)` returns the numbers of the geometry at the time $x_4 = t$ and the angle $z =$ `zv`: the eight factors f_μ (`float` turns a numpy number into a Python number); the matrices $\Omega_\mu = \sum_{a<b} \omega_{\mu ab} S^{ab}$; the metric components $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$; $\sqrt{|g|}$ (`abs` of the product of the factors); the curved gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ (key `"up"`) and $\gamma_\mu = \eta_{\mu\mu} f_\mu \gamma^{(\mu)}$ (key `"down"`).

```
for s0, zv in ((1, Z0), (-1, np.pi - Z0)):
    geo = geometry_at(1.3, zv, s0)
    total = sum(geo["up"][mu] @ geo["Om"][mu] for mu in range(8))
    assert np.allclose(total, 3 * s0 * H * gamma[0], atol=1e-12)
check(abs(m + lam * S0) > 3 * H,
      "|m + lambda S| > 3 H: the solution and both partners oscillate")
```

A numerical test of the matrices just built: at the time 1.3 on each patch, $\sum_\mu \gamma^\mu \Omega_\mu$ must equal $3 s_0 H \gamma^{(x_0)}$ entry by entry to 10^{-12} (`np.allclose`; `assert` stops the notebook if not). Then the check $|m + \lambda S| = 1.269 > 3H = 0.6$: by Section 18.21 the solution oscillates. The partners oscillate too, because their generators have the same eigenvalues (In [14]). Out [4] ends with this PASS line.

In [5], the solution: Runge-Kutta against the exact formula.

```
X_END = 20.0  # integrate from x4 = 0 to 20 (a4 from 0 to A H 20 = 4)


def generator(psi, mass, coup, s0):
    """The matrix M of d psi/dx4 = M psi (s0 = +1 patch, -1 mirror patch)."""
    S = (psi.conj() @ C @ psi).real
    return -gamma[4] @ ((mass + coup * S) * I16 - 3 * s0 * H * gamma[0])
```

`X_END` is the end time 20. Along the history $a_4 = AHx_4$ with $AH = 0.2$, a_4 runs from 0 to 4, so between $x_4 = 0$ and 20 the extra times deflate by the factor $e^{-4} \approx 0.018$ and 3-space inflates by $e^4 \approx$

54.6. `generator(psi, mass, coup, s0)` returns the matrix $M = -\gamma^{(x_4)}\big((m+\lambda S)I_{16} - 3s_0 H\gamma^{(x_0)}\big)$ of Section 18.21. S is computed from the column `psi` itself, because the equation is nonlinear (M depends on the field through S), and $s_0 = -1$ selects the mirror patch, where the connection term is $-3H\gamma^{(x_0)}$.

```
def rk4(psi_start, mass, coup, s0, steps):
    """The RK4 solution at x4 = 0, h, 2h, ..., X_END (steps + 1 rows)."""
    h = X_END / steps
    out = np.empty((steps + 1, 16), dtype=complex)
    psi = psi_start.astype(complex)
    out[0] = psi
    for n in range(steps):
        k1 = generator(psi, mass, coup, s0) @ psi
        k2 = generator(psi + 0.5 * h * k1, mass, coup, s0) @ (psi + 0.5 * h * k1)
        k3 = generator(psi + 0.5 * h * k2, mass, coup, s0) @ (psi + 0.5 * h * k2)
        k4 = generator(psi + h * k3, mass, coup, s0) @ (psi + h * k3)
        psi = psi + h / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
        out[n + 1] = psi
    return out
```

`rk4` is the **classical Runge-Kutta method of fourth order** (Chapter 2). With the step h and $F(\Psi) = M(\Psi)\Psi$, one step from Ψ_n at the time $x_4 = nh$ computes four slopes,

$$k_1 = F(\Psi_n), \qquad k_2 = F\big(\Psi_n + \tfrac{h}{2}k_1\big), \qquad k_3 = F\big(\Psi_n + \tfrac{h}{2}k_2\big), \qquad k_4 = F(\Psi_n + hk_3),$$

and moves on to $\Psi_{n+1} = \Psi_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$. `np.empty` reserves an array of `steps + 1` rows of 16 complex numbers; row n receives Ψ_n. `astype(complex)` makes a complex copy of the start column.

```
def exact(psi_start, mass, coup, s0, times):
    """The exact solution cos(w x4) psi(0) + sin(w x4)/w M psi(0)."""
    M = generator(psi_start, mass, coup, s0)
    S = (psi_start.conj() @ C @ psi_start).real
    w = np.sqrt((mass + coup * S) ** 2 - 9 * H ** 2)
    return (np.cos(w * times)[:, None] * psi_start[None, :]
            + (np.sin(w * times) / w)[:, None] * (M @ psi_start)[None, :])
```

`exact` evaluates the formula $\Psi(x_4) = \cos(wx_4)\Psi(0) + \frac{\sin(wx_4)}{w}M\Psi(0)$ with $w = \sqrt{(m+\lambda S)^2 - 9H^2}$ (Section 18.21) at many times at once: `[:, None]` makes a column of the time values and `[None, :]` a row of the components, and their product is the table with one row per time (numpy's **broadcasting**).

```
STEPS = 2000
times = np.linspace(0.0, X_END, STEPS + 1)
psi = rk4(psi0, m, lam, 1, STEPS)  # the universe of mass m on the patch
S_t = np.einsum("ti,ij,tj->t", psi.conj(), C, psi).real
error = np.abs(psi - exact(psi0, m, lam, 1, times)).max()
M0 = generator(psi0, m, lam, 1)
square_ok = np.allclose(M0 @ M0, (9 * H ** 2 - (m + lam * S0) ** 2) * I16, atol=1e-13)
```

The solution with 2000 steps of length 0.01, at the 2001 times `times`, evenly spaced from 0 to 20 (`np.linspace`). `S_t` is S at every time: `np.einsum("ti,ij,tj->t", ...)` computes, for each time t, the sum $\sum_{i,j} \Psi^*_{ti} C_{ij} \Psi_{tj}$ (the letters say which indices are summed and which one is kept). `error` is the largest difference between the Runge-Kutta table and the exact one, over all times and components. `square_ok` checks $M^2 = (9H^2 - (m + \lambda S)^2) I_{16}$ numerically.

```
errors = {}
for n in (250, 500, 1000, 2000, 4000):
    sol = rk4(psi0, m, lam, 1, n)
    errors[n] = float(np.abs(sol[-1] - exact(psi0, m, lam, 1, np.array([X_END]))[0])
                      .max())
orders = [np.log2(errors[n] / errors[2 * n]) for n in (250, 500, 1000, 2000)]
say("RK4 error at x4 = 20 for 250, 500, 1000, 2000, 4000 steps: "
    + ", ".join(f"{errors[n]:.1e}" for n in errors))
say("measured orders: " + ", ".join(f"{p:.2f}" for p in orders))
report("largest RK4 error with 2000 steps (all times, all components)",
       f"{error:.1e}")
```

The **order** of the method is measured. If the error at the end time behaves like $e(h) = ch^p$, halving the step gives $e(h/2) = ch^p/2^p$, so $e(h)/e(h/2) = 2^p$ and $p = \log_2 \left(e(h)/e(h/2)\right)$. The loop solves with 250, 500, 1000, 2000 and 4000 steps and stores the error at $x_4 = 20$; `orders` holds the four values of p. Out [5] prints the errors 2.0e-05, 1.2e-06, 6.9e-08, 4.3e-09 and 2.6e-10 (2.0e-05 means 2.0×10^{-5}), the measured orders 4.10, 4.06, 4.03 and 4.01, and the largest error with 2000 steps over all times and components, 4.3e-09.

```
reproduces(np.abs(S_t - S0).max() < 1e-9 and error < 1e-7 and square_ok,
           "S is constant and RK4 equals the exact homogeneous solution (2000 steps)",
           (TH_PY, ["exact_nonlinear_homogeneous_solution"]),
           (TH_WL, ["exact_solution_nonlinear_homogeneous_C"]),
           (SCOPE, ["good_sector_x0_independent_modes_without_boundary_condition"]))
check(all(3.8 < p < 4.2 for p in orders), "RK4 is of fourth order here")
```

The first check requires S to stay constant to 10^{-9}, the error to be below 10^{-7}, and the square of M; it names the exact homogeneous solution of the two field-theory reports and the scope report's check on these modes. The second requires every measured order between 3.8 and 4.2. COMPUTED: the Runge-Kutta solution agrees with the exact formula of the record to 4.3×10^{-9}, and the method is of fourth order.

In [6], Figure 18c.1: the convergence of the method.

```
steps_list = sorted(errors)
hs = np.array([X_END / n for n in steps_list])
errs = np.array([errors[n] for n in steps_list])
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.loglog(hs, errs, "o-", color="#2a78d6", label="RK4 error at $x_4 = 20$")
ax.loglog(hs, errs[-1] * (hs / hs[-1]) ** 4, "--", color="#52514e",
          label="slope 4: error $\\propto h^4$")
ax.set_xlabel("step length $h$ (units $1/m$)")
ax.set_ylabel("largest error of a component")
ax.legend()
```

`hs` holds the five step lengths $20/n$ and `errs` the five errors. `ax.loglog` draws with logarithmic axes on both sides: a power law $e = ch^p$ becomes a straight line of slope p, because $\log e = \log c + p \log h$. The dashed comparison line $e_{\text{last}}(h/h_{\text{last}})^4$ has the slope 4 and passes through the last point.

```
save_figure(fig, "rk4_convergence",
            "Accuracy of the Runge-Kutta solution of the homogeneous field equation "
            "in the author's metric: the largest difference between a component of "
            "the computed field and of the exact solution of the Revision theory "
            "record at the time $x_4 = 20$ (vertical axis, logarithmic), against "
            "the step length $h$ in units of $1/m$ (horizontal axis, logarithmic), "
            "for 250 to 4000 steps. The points follow the dashed line of slope 4: "
            "halving the step divides the error by 16, the fourth order of the "
            "method. With 2000 steps the error is far below every difference "
            "studied in this notebook.")
```

What you see in Figure 18c.1: five points on a straight line parallel to the dashed line of slope 4, falling from about 2×10^{-5} at $h = 0.08$ to about 3×10^{-10} at $h = 0.005$. Why it matters for the theorems: the differences studied below are either zero by a theorem (they must come out below 10^{-12}) or of order 1 (the negative control). Moreover the map $\Psi \to \Gamma\Psi$ commutes with every step of the method: the slopes of the T1 partner are Γ times the slopes of the field, because $F_{-m,-\lambda}(\Gamma\Psi) = \Gamma F_{m,\lambda}(\Psi)$ (the same computation as $\Gamma M \Gamma$ in Section 18.21), and in the same way $F^{\text{mirror}}_{-m,\lambda}(\gamma^{(x_0)}\Psi) = \gamma^{(x_0)} F_{m,\lambda}(\Psi)$. So a partner computed with the same steps equals the mapped field up to rounding, whatever the integration error; the comparison with the exact formula shows, in addition, that both are the true solutions to 10^{-9}.

In [7], theorem T1 in numbers.

```
phi = rk4(Gamma @ psi0, -m, -lam, 1, STEPS)  # the T1 partner, solved on its own
t1_gap = np.abs(phi - psi @ Gamma.T).max()  # row by row: Gamma psi(x4)
wrong = rk4(Gamma @ psi0, -m, lam, 1, STEPS)  # the negative control (-m, +lambda)
wrong_gap = np.abs(wrong - psi @ Gamma.T).max()
report("largest difference of the wrong partner (-m, +lambda) from Gamma Psi",
       f"{wrong_gap:.3f}")
check(t1_gap < 1e-12, "T1: the (-m, -lambda) solution from Gamma Psi(0) is Gamma "
      "Psi(x4) at all 2001 times (difference below 1e-12)")
check(wrong_gap > 0.1, "negative control: the (-m, +lambda) solution from Gamma "
      "Psi(0) is not Gamma Psi(x4)")
```

`phi` is the solution of the partner theory $(-m, -\lambda)$ started from $\Gamma\Psi(0)$, computed by the same Runge-Kutta function with no knowledge of `psi`. In `psi @ Gamma.T` each row of the table `psi` is $\Psi(x_4)$ written as a row, and a row times Γ^T is $(\Gamma\Psi)^T$; so this table holds $\Gamma\Psi(x_4)$ at all 2001 times, and `t1_gap` is its largest difference from `phi`. `wrong` is the negative control, the theory $(-m, +\lambda)$ from the same start, and `wrong_gap` its largest difference from $\Gamma\Psi$. Out [7] prints RESULT largest difference of the wrong partner (-m, +lambda) from Gamma Psi = 1.641 and two PASS lines: the T1 partner equals $\Gamma\Psi$ to better than 10^{-12} at every time, while the wrong partner is off by 1.641, as much as the field itself (the column has length 1). Why it fails: with S unchanged its coefficient is $V = -m + \lambda S = -1 + 0.269196 = -0.730804$ instead of -1.269196, so it oscillates with the frequency $\sqrt{0.730804^2 - 0.36} = 0.4172$ instead of 1.1184.

In [8], Figure 18c.2: the partner, component by component.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0), sharey=True)
every = slice(0, STEPS + 1, 40)  # a dot every 40 steps
for ax, A in zip(axes, (0, 8)):
    ax.plot(times, psi[:, A].real, color="#2a78d6",
            label=f"$\\Psi_{{{A + 1}}}$, mass $m$, coupling $\\lambda$")
    ax.plot(times[every], phi[every, A].real, "o", color="#eb6834", markersize=3.5,
            label=f"$\\Phi_{{{A + 1}}}$, mass $-m$, coupling $-\\lambda$")
    ax.plot(times, wrong[:, A].real, "--", color="#52514e", linewidth=0.9,
            label="wrong partner: $-m$, $+\\lambda$")
    ax.set_xlabel("time $x_4$ (units $1/m$)")
    ax.set_title(f"component {A + 1}: " + ("$\\Phi = -\\Psi$" if A < 8
                                           else "$\\Phi = +\\Psi$"), fontsize=10)
    ax.legend(loc="lower left", fontsize=8)
axes[0].set_ylabel("real part of the component")
```

Two panels side by side that share the vertical axis (`sharey=True`). `every` is a **slice** that takes every 40th of the 2001 times, so that the dots do not hide the line. The loop draws component 1 (Python place 0) in the left panel and component 9 (place 8) in the right one: the real part of Ψ_A as a blue line, of the partner Φ_A as orange dots, and of the wrong partner as a thin dashed line. In the f-strings, `{{{A + 1}}}` prints a brace, the number $A+1$ and a brace, so that the label reads Ψ_1 or Ψ_9 in LaTeX. The title of each panel says which relation holds in that half.

```
save_figure(fig, "partner_components",
            "A solution and its T1 partner, computed separately. Lines: the real "
            "part of component 1 (left) and component 9 (right) of the homogeneous "
            "solution $\\Psi$ with mass $m = 1$ and coupling $\\lambda = 0.3$, "
            "against the time $x_4$ in units of $1/m$. Dots: the same components "
            "of the solution $\\Phi$ of the theory with $-m$ and $-\\lambda$, "
            "started from $\\Gamma\\Psi(0)$ and integrated on its own. In the first "
            "half of the components $\\Phi = -\\Psi$, in the second half "
            "$\\Phi = +\\Psi$: the partner is exactly $\\Gamma\\Psi$ at every time, "
            "as theorem T1 says. Dashed: the negative control with $-m$ and "
            "$+\\lambda$, which drifts away because it oscillates with another "
            "frequency.")
```

What you see in Figure 18c.2: on the left, the dots lie on the mirror image of the blue line in the horizontal axis ($\Phi_1 = -\Psi_1$: they start near $+0.21$ where the line starts near -0.21); on the right, the dots lie on the line ($\Phi_9 = +\Psi_9$). The dashed wrong partner starts at the same values as the dots but oscillates with the period $2\pi/0.4172 \approx 15.1$ instead of $2\pi/1.1184 \approx 5.62$ and drifts away. This is $\Gamma = \mathrm{diag}(-I_8, I_8)$ at work on an actual solution.

In [9], the bilinears of a homogeneous field, and T1 for them.

```
def bilinears(field, mass, coup, t, zv, s0):
    """S, J^mu (8 numbers) and T_mu nu (8 x 8) of a homogeneous field at (t, zv)."""
    geo = geometry_at(t, zv, s0)
    D = {mu: geo["Om"][mu] @ field for mu in range(8)}
    D[4] = D[4] + generator(field, mass, coup, s0) @ field  # d field / dx4
    bar = field.conj() @ C  # the adjoint row Psibar = Psi^dagger C
    Dbar = {mu: D[mu].conj() @ C for mu in range(8)}
    S = (bar @ field).real
```

`bilinears(field, mass, coup, t, zv, s0)` computes, for one column `field` at the time `t` and the angle `zv`, every quantity the theorems speak about. `geo` holds the geometry there. The covariant derivatives: a homogeneous field has $\partial_\mu\Psi = 0$ for $\mu \neq 4$, so $D_\mu\Psi = \Omega_\mu\Psi$, and $\partial_4\Psi = M\Psi$ by the field equation, which the line `D[4] = D[4] + ...` adds to $\Omega_4\Psi$ (zero here). `bar` is the row $\bar\Psi = \Psi^\dagger C$ (`field.conj() @ C`: the conjugate column used as a row, times C). `Dbar` uses $D_\mu\bar\Psi = (D_\mu\Psi)^\dagger C$, which follows from the reality of Ω_μ:

$$(D_\mu\Psi)^\dagger C = (\partial_\mu\Psi)^\dagger C + \Psi^\dagger\Omega_\mu^T C = \partial_\mu\bar\Psi - \Psi^\dagger C\Omega_\mu = D_\mu\bar\Psi.$$

The rule $(XY)^\dagger = Y^\dagger X^\dagger$ with $\Omega_\mu^\dagger = \Omega_\mu^T$ (Ω_μ is real); then $\Omega_\mu^T C = -C\Omega_\mu$ (Exercise 18.4 derives it from $C\gamma^{(a)}C^{-1} = -(\gamma^{(a)})^T$); then the definition of $D_\mu\bar\Psi$. `S` is $\bar\Psi\Psi$.

```
    L = 0.5 * sum(bar @ geo["up"][mu] @ D[mu] - Dbar[mu] @ geo["up"][mu] @ field
                  for mu in range(8)) - mass * S - 0.5 * coup * S ** 2
    T = np.zeros((8, 8), dtype=complex)
    for mu in range(8):
        for nu in range(8):
            T[mu, nu] = 0.25 * (
                bar @ geo["down"][mu] @ D[nu] + bar @ geo["down"][nu] @ D[mu]
                - Dbar[mu] @ geo["down"][nu] @ field
                - Dbar[nu] @ geo["down"][mu] @ field)
            if mu == nu:
                T[mu, nu] -= geo["g"][mu] * L
    J = np.array([-1j * bar @ geo["up"][mu] @ field for mu in range(8)])
    assert np.abs(T.imag).max() < 1e-12 and np.abs(J.imag).max() < 1e-12
    return S, J.real, T.real
```

`L` is $\mathcal{L}/\sqrt{|g|}$ of Section 18.4: half the sum over μ of $\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi$, minus mS, minus $\frac{\lambda}{2}S^2$. `T` is the 8×8 table of $T_{\mu\nu}$ of the pairing record (Section 18.4), built from the lowered gammas γ_μ (`geo["down"]`), with $-g_{\mu\mu}\mathcal{L}/\sqrt{|g|}$ added on the diagonal; it is stored at the place `[mu, nu]` (the place k is x_k). `J` holds $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, the convention of the theory record, for $\mu = 0, \ldots, 7$. For a commuting field all these numbers are real (the Lagrangian is real, Chapter 7); `assert` confirms that their imaginary parts are below 10^{-12}, and the function returns the real parts.

```
SAMPLE = range(0, STEPS + 1, 20)  # 101 sampled times
sampled = times[list(SAMPLE)]
data = {"psi": [bilinears(psi[n], m, lam, times[n], Z0, 1) for n in SAMPLE],
        "phi": [bilinears(phi[n], -m, -lam, times[n], Z0, 1) for n in SAMPLE]}
scale_T = max(np.abs(T).max() for _, _, T in data["psi"])
scale_J = max(np.abs(J).max() for _, J, _ in data["psi"])
t1_T = max(np.abs(Tp + Tq).max() for (_, _, Tp), (_, _, Tq)
           in zip(data["psi"], data["phi"])) / scale_T
t1_J = max(np.abs(Jp + Jq).max() for (_, Jp, _), (_, Jq, _)
           in zip(data["psi"], data["phi"])) / scale_J
t1_S = max(abs(Sp - Sq) for (Sp, _, _), (Sq, _, _) in zip(data["psi"], data["phi"]))
reproduces(t1_T < 1e-12 and t1_J < 1e-12 and t1_S < 1e-12,
           "T1: S kept, J -> -J (8) and T -> -T (36) at all 101 sampled times",
           (PY, ["T1.metric.commuting.emt", "T1.metric.commuting.current"]))
```

SAMPLE takes every 20th time, $x_4 = 0, 0.2, 0.4, \ldots, 20$: 101 times. data holds the bilinears of the field with (m, λ) and of its T1 partner with $(-m, -\lambda)$ at the same point z_0 of the patch. scale_T and scale_J are the largest absolute entries of T and J of the field over all sampled times; the differences are divided by them, so that the test measures a relative error. t1_T is the largest relative entry of $T[\Psi] + T[\Phi]$, t1_J that of $J[\Psi] + J[\Phi]$, and t1_S the largest difference of the two values of S. The check requires all three below 10^{-12} and names the sympy checks T1.metric.commuting.emt and T1.metric.commuting.current. Out [9] shows the PASS line. COMPUTED on an actual solution along the deflating history: the 36 independent components of $T_{\mu\nu}$ (the table is symmetric) and the 8 of J^μ of the partner are the negatives of those of the field, and S is the same.

In [10], three facts of the theory record, and the local conservation law of the charge.

```
rho_ok = p_ok = mixed_ok = flow_ok = True
q0_values = []  # the frame current q0 at the sampled times
for n, (S, J, T) in zip(SAMPLE, data["psi"]):
    geo = geometry_at(times[n], Z0, 1)
    rho_ok = rho_ok and abs(-T[4, 4] - (m * S + 0.5 * lam * S ** 2)) < 1e-12
    p = [-T[mu, mu] / geo["g"][mu] for mu in (0, 1, 2, 3, 5, 6, 7)]
    p_ok = p_ok and max(abs(x - 0.5 * lam * S ** 2) for x in p) < 1e-12
    mixed_ok = mixed_ok and abs(T[4, 0]) < 1e-12 * scale_T
    field = psi[n]
    dJ4 = 2 * (field.conj() @ B @ generator(field, m, lam, 1) @ field).real
    q0 = (-1j * field.conj() @ C @ gamma[0] @ field).real
    q0_values.append(q0)
    flow_ok = flow_ok and abs(dJ4 + 6 * H * q0) < 1e-12
```

q0_values starts as an empty list; the loop appends one number to it at each sampled time. The loop runs over the sampled times of the field (zip pairs the time indices with the stored bilinears). T[4, 4] is $T_{x_4x_4}$ (place 4 is x_4), so -T[4, 4] is the energy density ρ of the pairing convention, compared with $mS + \frac{\lambda}{2}S^2$. p holds the seven pressures $p_\mu = -g^{\mu\mu}T_{\mu\mu} = -T_{\mu\mu}/g_{\mu\mu}$ of x_0, of x_1, x_2, x_3 and of the deflating extra times x_5, x_6, x_7, each compared with $\frac{\lambda}{2}S^2$. T[4, 0] is $T_{x_4x_0}$, which must vanish (relative to scale_T). Then the conservation law of Section 18.21: dJ4 is $\partial_4 J^{x_4}$, computed from the field equation $\partial_4\Psi = M\Psi$,

$$\frac{d}{dx_4}(\Psi^\dagger B\Psi) = (M\Psi)^\dagger B\Psi + \Psi^\dagger BM\Psi = 2\,\mathrm{Re}(\Psi^\dagger BM\Psi).$$

The product rule; then $(M\Psi)^\dagger B\Psi = \Psi^\dagger M^\dagger B\Psi$ is the complex conjugate of $\Psi^\dagger B^\dagger M\Psi = \Psi^\dagger BM\Psi$ (B is Hermitian), and a number plus its complex conjugate is twice its real part. q0 is the frame current $q_0 = -i\bar\Psi\gamma^{(x_0)}\Psi$ of Section 18.21 (not the charge Q), q0_values.append(q0) stores it, and the local law of Section 18.21 requires $\partial_4 J^{x_4} = -6Hq_0$.

```
reproduces(rho_ok and p_ok and mixed_ok,
           "rho = m S + (1/2) S^2, all pressures (1/2) S^2, T_x4x0 = 0 (homogeneous)",
           (TH_PY, ["commuting_homogeneous_on_shell_rho_p",
                    "commuting_T_x4x0_homogeneous"]))
reproduces(flow_ok, "local charge conservation: d4 J^x4 = -6 H q0 (flow along x0)",
           (TH_PY, ["commuting_current_conservation"]))
say(f"q0 lies between {min(q0_values):.4f} and {max(q0_values):.4f}: the flow "
```

```
    "sin z q0 through the brane z = pi/2 is not zero, so the charge of the "
    "patch changes in time")
```

Two checks and one printed line. The first check names two checks of the field-theory report `python-field-theory.json`, one for the energy density and the pressures and one for $T_{x_4x_0}$ of homogeneous solutions; the second names its check of the LOCAL conservation law of the charge. Out [10] shows both PASS lines. The `say` line prints the smallest and the largest value of q_0 over the 101 sampled times (`min` and `max` of the list; `:.4f` writes four decimals): -0.3454 and 0.3454. At the brane $z = \pi/2$ the flow $\sqrt{|g|}J^{x_0} = \sin z\, q_0$ equals q_0, so it is not zero, and by Section 18.21 the charge of the patch changes at the rate $-V_6 q_0$: the local law holds exactly, but the total charge of the patch is NOT constant for this solution. With the numbers of Out [4] the energy density is $\rho = mS + \frac{\lambda}{2}S^2 = 0.897319 + 0.15 \times 0.805181 = 1.0181$ and every pressure is $0.15 \times 0.805181 = 0.1208$; Exercise 18.2 continues this computation.

In [11], Figure 18c.3: the T1 pair in numbers.

```
def series(name, pick):
    return np.array([pick(S, J, T) for S, J, T in data[name]])


panels = [("charge density $J^{x_4}$", lambda S, J, T: J[4]),
          ("energy density $\\rho = -T_{x_4x_4}$", lambda S, J, T: -T[4, 4]),
          ("$T_{x_4x_1}$ (history-dependent)", lambda S, J, T: T[4, 1])]
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0))
for ax, (title, pick) in zip(axes, panels):
    a, b = series("psi", pick), series("phi", pick)
    ax.plot(sampled, a, color="#2a78d6", label="$\\Psi$: $(m, \\lambda)$")
    ax.plot(sampled, b, color="#eb6834", label="T1 partner: $(-m, -\\lambda)$")
    ax.plot(sampled, a + b, color="#52514e", linestyle="--", label="sum")
    ax.set_title(title, fontsize=10)
    ax.set_xlabel("time $x_4$ (units $1/m$)")
axes[0].legend(fontsize=8, loc="lower left")
```

`series(name, pick)` returns the time series of one quantity; `pick` is a small function that chooses it from (S, J, T). `panels` lists three choices, each a title and a one-line function (`lambda S, J, T: ...`): J^{x_4} (`J[4]`), $\rho = -T_{x_4x_4}$ and $T_{x_4x_1}$ (`T[4, 1]`). For each panel the field (blue), its T1 partner (orange) and their sum (dashed) are drawn against the sampled times.

```
save_figure(fig, "t1_pair_densities",
            "The T1 pair in numbers, along the deflating history $a_4 = AHx_4$ "
            "($A = 1$, $H = 0.2$, $m = 1$, $\\lambda = 0.3$, at $z = \\pi/4$). "
            "Horizontal axes: the time $x_4$ in units of $1/m$. Left: the charge "
            "density $J^{x_4}$; middle: the energy density $\\rho$; right: the "
            "mixed component $T_{x_4x_1}$, which grows with the factor $e^{a_4}$ "
            "of the inflating direction. Blue: the solution with $(m, \\lambda)$; "
            "orange: its partner $\\Gamma\\Psi$ with $(-m, -\\lambda)$; dashed: "
            "their sum. Every orange curve is the mirror image of the blue one in "
            "the horizontal axis, and the sum is zero at every time: the pair has "
            "zero total charge and energy-momentum as classical bilinears.")
```

What you see in Figure 18c.3: on the left, the blue charge density oscillates between about -0.04 and -0.40 and the orange one between $+0.04$ and $+0.40$, seven times between $x_4 = 0$ and 20 (the density is quadratic in the field, so it oscillates with the period $\pi/w = 2.81$, half the period of the field); in the middle, two constant lines at $+1.018$ and -1.018; on the right, the blue component rises from about 0.05 to 3 and the orange one falls from -0.05 to -3. In each panel the dashed sum is zero at every time. Why the right panel grows: $T_{x_4x_1}$ contains $\gamma_{x_1} = f_1\gamma^{(x_1)}$ and the connection Ω_{x_1}, both proportional to $e^{a_4} = e^{0.2x_4}$, which grows by the factor $e^4 \approx 54.6$ between $x_4 = 0$ and 20; this is where the deflating history enters the energy-momentum tensor. COMPUTED: $T + T' = 0$ and $J + J' = 0$ at every sampled time.

In [12], theorem T2 in numbers: the mirror partner.

```
chi = rk4(gamma[0] @ psi0, -m, lam, -1, STEPS)  # the mirror partner, on its own
t2_gap = np.abs(chi - psi @ gamma[0].T).max()
data["chi"] = [bilinears(chi[n], -m, lam, times[n], np.pi - Z0, -1) for n in SAMPLE]
LAM = np.array([-1, 1, 1, 1, 1, 1, 1, 1], dtype=float)  # x0, ..., x7
t2_S = max(abs(Sp + Sx) for (Sp, _, _), (Sx, _, _) in zip(data["psi"], data["chi"]))
t2_J = max(np.abs(Jx - LAM * Jp).max() for (_, Jp, _), (_, Jx, _)
           in zip(data["psi"], data["chi"])) / scale_J
t2_T = max(np.abs(Tx - np.outer(LAM, LAM) * Tp).max() for (_, _, Tp), (_, _, Tx)
           in zip(data["psi"], data["chi"])) / scale_T
```

`chi` is the mirror partner: the theory $(-m, \lambda)$ on the mirror patch ($s_0 = -1$), started from $\gamma^{(x_0)}\Psi(0)$ and solved on its own. `t2_gap` compares it with $\gamma^{(x_0)}\Psi(x_4)$ (`psi @ gamma[0].T`, as in In [7]). `data["chi"]` holds its bilinears at the MIRROR point $z = \pi - z_0 = 3\pi/4$, in the geometry of the mirror patch. `LAM` is $\Lambda = R_0 = \mathrm{diag}(-1, 1, 1, 1, 1, 1, 1, 1)$. `t2_S` measures $S[X] + S[\Psi]$, which must vanish because S is reversed; `t2_J` measures $J[X] - \Lambda J[\Psi]$ and `t2_T` measures $T[X] - \Lambda_\mu\Lambda_\nu T[\Psi]$ (`np.outer(LAM, LAM)` is the 8×8 table of the products $\Lambda_\mu\Lambda_\nu$), both relative.

```
check(t2_gap < 1e-12, "T2: the (-m, lambda) mirror solution from gamma^(x0) Psi(0) "
      "is gamma^(x0) Psi(x4) at all 2001 times")
reproduces(t2_S < 1e-12 and t2_J < 1e-12 and t2_T < 1e-12,
           "T2: S -> -S, J and T pulled back (charge and energy density equal)",
           (PY, ["T2.metric.commuting.S_odd", "T2.metric.commuting.emt",
                 "T2.metric.commuting.current"]))
```

Two checks: the mirror partner is $\gamma^{(x_0)}\Psi$ at all 2001 times, and S is reversed while J and T are pulled back, with the sympy checks `T2.metric.commuting.S_odd`, `T2.metric.commuting.emt` and `T2.metric.commuting.current`. Out [12] shows both PASS lines. Note what is compared: the field at $z_0 = \pi/4$ on the patch and its partner at $3\pi/4$ on the mirror patch, the two points that the mirror $z \to \pi - z$ exchanges.

In [13], Figure 18c.4: the T2 pair in numbers.

```
panels = [("scalar $S$", lambda S, J, T: S),
          ("charge density $J^{x_4}$", lambda S, J, T: J[4]),
          ("$T_{x_4x_1}$ (history-dependent)", lambda S, J, T: T[4, 1])]
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0))
for ax, (title, pick) in zip(axes, panels):
```

```
    ax.plot(sampled, series("psi", pick), color="#2a78d6",
            label="$\\Psi$ on the patch: $(m, \\lambda)$")
    ax.plot(sampled[::4], series("chi", pick)[::4], "o", color="#1baf7a",
            markersize=4, label="mirror partner: $(-m, \\lambda)$")
    ax.set_title(title, fontsize=10)
    ax.set_xlabel("time $x_4$ (units $1/m$)")
axes[0].set_ylim(-1.1, 1.1)
axes[0].legend(fontsize=8, loc="center left")
```

Three panels, S, J^{x_4} and $T_{x_4x_1}$, with the field as a blue line and the mirror partner as green dots at every fourth sampled time (`[::4]` takes every fourth entry). `set_ylim(-1.1, 1.1)` fixes the vertical range of the first panel so that both values of S are visible.

```
save_figure(fig, "t2_mirror_densities",
            "The T2 pair in numbers, same values as in the previous figure. Blue "
            "line: the solution $\\Psi$ with $(m, \\lambda)$ on the patch at "
            "$z = \\pi/4$; green dots: its mirror partner $\\gamma^{(x_0)}\\Psi$, "
            "solved on its own with $(-m, \\lambda)$ on the mirror patch at "
            "$z = 3\\pi/4$. Horizontal axes: the time $x_4$ in units of $1/m$. "
            "Left: the scalar $S$, constant and exactly opposite for the partner. "
            "Middle and right: the charge density $J^{x_4}$ and the component "
            "$T_{x_4x_1}$, EQUAL for the partner. T2 pairs the mass $m$ with $-m$ "
            "at the same charge and energy-momentum; nothing cancels.")
```

What you see in Figure 18c.4: on the left two constant lines, $S = 0.897$ for the field and $S = -0.897$ for the partner; in the middle and on the right the green dots lie ON the blue curves: the mirror partner has the same charge density and the same $T_{x_4x_1}$. Compare Figure 18c.3, where the T1 partner was the mirror image in the horizontal axis. T2 pairs m with $-m$ at EQUAL charge and energy-momentum: nothing cancels in a T2 pair.

In [14], the spectra of the generators.

```
def spectrum(Mx):
    """The 16 eigenvalues, sorted by imaginary part, then real part (rounded)."""
    ev = np.linalg.eigvals(Mx)
    return ev[np.lexsort((np.round(ev.real, 9), np.round(ev.imag, 9)))]


M_psi = generator(psi0, m, lam, 1)
M_t1 = generator(Gamma @ psi0, -m, -lam, 1)
M_t2 = generator(gamma[0] @ psi0, -m, lam, -1)
similar = (np.array_equal(Gamma @ M_psi @ Gamma, M_t1)
           and np.allclose(gamma[0] @ M_psi @ gamma[0], M_t2, atol=1e-15))
sp_psi, sp_t1, sp_t2 = spectrum(M_psi), spectrum(M_t1), spectrum(M_t2)
w_exact = np.sqrt((m + lam * S0) ** 2 - 9 * H ** 2)
report("frequency w of the solution and of both partners", f"{w_exact:.6f}")
check(similar and np.abs(sp_psi - sp_t1).max() < 1e-12
      and np.abs(sp_psi - sp_t2).max() < 1e-12
      and np.allclose(np.sort(np.abs(sp_psi.imag)), w_exact, atol=1e-12),
      "the generators of the field and of its T1 and T2 partners are similar: "
      "equal spectra +-i w")
```

`spectrum(Mx)` returns the 16 eigenvalues of a matrix (`np.linalg.eigvals`) in a fixed order: `np.lexsort` sorts by its last key first, here by the imaginary part and then by the real part, both rounded to 9 decimals so that rounding differences do not change the order. `M_psi`, `M_t1` and `M_t2` are the generators of the field, of the T1 partner and of the mirror partner at the start. `similar` checks $\Gamma M \Gamma = M_{\mathrm{T1}}$ with the exact comparison `np.array_equal` (Γ only changes the signs of rows and columns, which floating-point arithmetic does exactly) and $\gamma^{(x_0)} M \gamma^{(x_0)} = M_{\mathrm{T2}}$ to 10^{-15}. Similar matrices have the same eigenvalues (Section 18.23), and the check compares the three sorted spectra. It also requires the absolute imaginary parts to equal w: since $M^2 = -w^2 I_{16}$, every eigenvalue μ of M obeys $\mu^2 = -w^2$, so $\mu = \pm iw$. Out [14] prints the frequency $w = 1.118417$ of the solution and of both partners, and the PASS line.

```
masses = np.linspace(-2.0, 2.0, 161)
family = {}
for label, sign_m, sign_l in (("field", 1, 1), ("T1 partner", -1, -1)):
    rows = []
    for mv in masses:
        V = sign_m * mv + sign_l * lam * S0
        Mx = -V * gamma[4] + 3 * H * gamma[4] @ gamma[0]
        rows.append(spectrum(Mx))
    family[label] = np.array(rows)
check(np.abs(family["field"] - family["T1 partner"]).max() < 1e-9,
      "for every mass from -2 to 2 the field and its T1 partner have equal spectra")
```

The same comparison for a whole family: 161 masses from -2 to 2 (`np.linspace(-2.0, 2.0, 161)`), at the fixed λ and S_0. For the field the generator has $V = m + \lambda S_0$; for the T1 partner the mass $-m$ and the coupling $-\lambda$ give $V = -m - \lambda S_0$. The generator $-V\gamma^{(x_4)} + 3H\gamma^{(x_4)}\gamma^{(x_0)}$ is built directly and its sorted spectrum stored, one row per mass. The check requires the two tables to agree to 10^{-9} at every mass; Out [14] ends with its PASS line.

In [15], Figure 18c.5: the spectrum against the mass.

```
fig, ax = plt.subplots(figsize=(8.0, 4.4))
for label, colour, style in (("field", "#2a78d6", "-"),
                             ("T1 partner", "#eb6834", "o")):
    rows = family[label]
    kw = {"markersize": 3} if style == "o" else {}
    ax.plot(masses, rows.imag.max(axis=1), style, color=colour,
            label=f"frequency $w$, {label}", **kw)
    ax.plot(masses, rows.real.max(axis=1), style, color=colour, alpha=0.45,
            label=f"growth rate, {label}", **kw)
ax.axvline(m, color="#52514e", linestyle=":", label="the solution of this notebook")
ax.set_xlabel("mass $m$ of the field (the partner has $-m$, $-\\lambda$)")
ax.set_ylabel("eigenvalue part (units $m = 1$)")
ax.legend(fontsize=8, loc="upper center")
```

For the field (lines, style `"-"`) and the partner (dots, style `"o"`; `**kw` passes the marker size only for the dots): the largest imaginary part of the 16 eigenvalues at each mass (`rows.imag.max(axis=1)`, the oscillation frequency w) in full colour, and the largest real part (the growth rate) in a lighter shade (`alpha=0.45`). A dotted vertical line marks the mass $m = 1$ of the solution of the notebook.

```
save_figure(fig, "family_spectrum",
            "The spectrum of the evolution generator of the homogeneous solutions, "
            "against the mass $m$ (horizontal axis) at the fixed coupling "
            "$\\lambda = 0.3$ and scalar $S$ of this notebook, $H = 0.2$. Dark "
            "curves: the oscillation frequency $w$, the largest imaginary part of "
            "the eigenvalues; light curves: the growth rate, the largest real part. "
            "Lines: the field with $(m, \\lambda)$; dots: its T1 partner with "
            "$(-m, -\\lambda)$. They coincide everywhere. Inside the window "
            "$|m + \\lambda S| < 3H$ the frequency is zero and the solutions grow; "
            "outside it they oscillate. The dotted line marks the solution "
            "plotted above.")
```

What you see in Figure 18c.5: the dots lie on the lines everywhere. Outside a window of masses the growth rate is zero and the frequency $w = \sqrt{(m + \lambda S_0)^2 - 9H^2}$ rises from 0 at the edges of the window towards the straight lines $|m + \lambda S_0|$; at the dotted line, $m = 1$, it is 1.118. Inside the window $|m + \lambda S_0| < 3H$, that is $-0.869 < m < 0.331$, the frequency is zero and the growth rate is an arc with the top $3H = 0.6$ at $m = -\lambda S_0 = -0.269$. Why an arc: inside the window the eigenvalues are $\pm\kappa$ with $\kappa^2 = 9H^2 - (m + \lambda S_0)^2$, so $\kappa^2 + (m + \lambda S_0)^2 = (3H)^2$, the equation of a circle of radius $3H$ (drawn with different scales on the two axes). The field and its T1 partner have the same spectrum for every mass: T1 never turns an oscillating universe into a growing one.

In [16], plane waves in flat space: the quantum reading Q.

```
def h_matrix(mass, k):
    """h_m(k) = -i m gamma^(x4) - gamma^(x4) sum_(a != 4) k_a gamma^(a)."""
    out = -1j * mass * gamma[4]
    for a in (0, 1, 2, 3, 5, 6, 7):
        out = out - k[a] * gamma[4] @ gamma[a]
    return out


def w_squared(mass, k):
    return mass ** 2 + sum(k[a] ** 2 for a in (0, 1, 2, 3)) - sum(
        k[a] ** 2 for a in (5, 6, 7))


def eigenspace(Mx, value, tol=1e-9):
    """Orthonormal columns spanning the null space of Mx - value I."""
    _, sv, vh = np.linalg.svd(Mx - value * I16)
    return vh[sv < tol].conj().T


def inertia(V):
    """(positive, negative) eigenvalue counts of V^dagger B V."""
    ev = np.linalg.eigvalsh(V.conj().T @ B @ V)
    return int((ev > 1e-9).sum()), int((ev < -1e-9).sum())
```

Four helpers. The first, `h_matrix(mass, k)`, builds the one-particle matrix of Section 18.22,

$$h_m(k) = -im\gamma^{(x_4)} - \gamma^{(x_4)} \sum_{a \neq 4} k_a \gamma^{(a)},$$

where `k[a]` is k_a (the entry k_4 is never used). `w_squared` returns $m^2+k_0^2+k_1^2+k_2^2+k_3^2-k_5^2-k_6^2-k_7^2$. `eigenspace(Mx, value)` returns orthonormal columns that span the eigenspace of the eigenvalue `value`, that is the null space of $M_x - wI_{16}$. It uses the **singular value decomposition** `np.linalg.svd`, which writes a matrix as $A = U\Sigma V^\dagger$ with U and V unitary (orthonormal columns) and Σ diagonal with the singular values $\sigma_j \geq 0$; since $Av_j = \sigma_j u_j$ for the columns u_j of U and v_j of V, the columns v_j with $\sigma_j = 0$ (in floating point: below 10^{-9}) span the null space. numpy returns $V^\dagger$ (`vh`), so the selected rows are conjugated and transposed into columns. `inertia(V)` counts the positive and the negative eigenvalues of the 8×8 Hermitian matrix $V^\dagger B V$, the Krein form restricted to the eigenspace (`np.linalg.eigvalsh` computes the eigenvalues of a Hermitian matrix).

```
R0 = np.array([-1, 1, 1, 1, 1, 1, 1, 1], dtype=float)  # the reflection of x0
rand = np.random.default_rng(7)
square_ok = maps_ok = True
for _ in range(50):
    k, mv = rand.normal(size=8), rand.normal()
    h = h_matrix(mv, k)
    square_ok = square_ok and np.allclose(h @ h, w_squared(mv, k) * I16, atol=1e-12)
    maps_ok = (maps_ok and np.allclose(Gamma @ h @ Gamma, h_matrix(-mv, k))
               and np.allclose(gamma[0] @ h @ gamma[0], h_matrix(-mv, R0 * k)))
```

`R0` reverses k_0. With the fixed seed 7, fifty random masses and momenta are drawn; for each, `square_ok` checks $h^2 = w^2 I_{16}$ and `maps_ok` checks $\Gamma h_m(k) \Gamma = h_{-m}(k)$ and $\gamma^{(x_0)} h_m(k) \gamma^{(x_0)} = h_{-m}(R_0 k)$ (`R0 * k` multiplies entry by entry).

```
samples = read_json(THEORY)["data"]["one_particle_flat"]["samples"]
rows_ok = True
for row in samples:
    k = np.array(row["k"], dtype=float)
    w = float(sp.sympify(row["w"].replace("Sqrt[", "sqrt(").replace("]", ")")))
    h = h_matrix(row["m"], k)
    Vp, Vn = eigenspace(h, w), eigenspace(h, -w)
    mine = (np.isclose(np.sqrt(w_squared(row["m"], k)), w), Vp.shape[1],
            Vn.shape[1], inertia(Vp), inertia(Vn))
    rows_ok = rows_ok and mine == (True, row["dim_plus_w"], row["dim_minus_w"],
                                   tuple(row["B_inertia_plus_w"]),
                                   tuple(row["B_inertia_minus_w"]))
    (pp, pn), (qp, qn) = inertia(Vp), inertia(Vn)
    say(f"m = {row['m']:+d}, k = {row['k']}: w = {w:.6f}, dimensions "
        f"{Vp.shape[1]} and {Vn.shape[1]}, Krein inertia ({pp},{pn}) and "
        f"({qp},{qn})")
```

`samples` is the data table `one_particle_flat` of `pairing-theory.json`: eight rows, each with m, k, the frequency w as text (for example `Sqrt[2]`, the Wolfram way of writing $\sqrt{2}$), the dimensions of the two eigenspaces and their Krein inertias. The text of w is translated into sympy's `sqrt(2)`, read by `sp.sympify` and turned into a number by `float`. For each row the cell computes the eigenspaces of $+w$ and $-w$, their dimensions (`shape[1]`, the number of columns) and their inertias; `mine` collects them in the order of the record, and `rows_ok` requires equality with the record. One line per sample is printed.

```
reproduces(square_ok and maps_ok,
           "h_m^2 = w^2 I16; Gamma h_m Gamma = h_-m; gamma^(x0) h_m gamma^(x0) = "
```

```
           "h_-m(R0 k)", (WL, ["Q_one_particle_flat_dispersion", "Q_one_particle_maps"]),
           (PY, ["Q.one_particle_maps"]))
reproduces(rows_ok and len(samples) == 8,
           "the eight recorded samples: w, dimensions 8 and 8, Krein inertia (4,4)",
           (WL, ["Q_one_particle_Krein_signatures"]),
           (PY, ["compare.theory.one_particle", "Q.one_particle_Krein_inertia_proof"]))
```

Two checks with their records. Out [16] prints the eight samples, each with the dimensions 8 and 8 and the Krein inertias (4,4) and (4,4), and two PASS lines. COMPUTED in floating point, they agree with the exact samples of the record.

In [17], imaginary and zero frequencies.

```
neutral_ok = True
for k in ([0, 0, 0, 0, 0, 2, 0, 0], [0, 1, 0, 0, 0, 2, 0, 0],
          [0, 0, 0, 0, 0, 1, 0, 0]):  # (k0, ..., k7)
    k = np.array(k, dtype=float)
    h, w2 = h_matrix(1.0, k), w_squared(1.0, k)
    if w2 == 0:
        spaces = [eigenspace(h, 0.0)]
        neutral_ok = neutral_ok and np.allclose(h @ h, 0.0)
    else:
        w = 1j * np.sqrt(-w2)
        spaces = [eigenspace(h, w), eigenspace(h, -w)]
    dims = [V.shape[1] for V in spaces]
    largest = max(np.abs(V.conj().T @ B @ V).max() for V in spaces)
    neutral_ok = neutral_ok and all(dim == 8 for dim in dims) and largest < 1e-9
    say(f"k = {k.astype(int).tolist()}: w^2 = {w2:+.0f}, eigenspace dimensions "
        f"{dims}, Krein form zero on them: {largest < 1e-9}")
```

Three momenta with an extra-time component k_5 at $m = 1$: $w^2 = 1 - 4 = -3$, $w^2 = 1 + 1 - 4 = -2$ and $w^2 = 1 - 1 = 0$. For $w^2 < 0$ the frequencies are $\pm i\sqrt{-w^2}$ (`1j * np.sqrt(-w2)`) and both eigenspaces are computed; for $w^2 = 0$ the eigenspace of 0 is computed and $h^2 = 0$ is checked. `largest` is the largest absolute entry of $V^\dagger B V$ over the eigenspaces; **Krein-neutral** means that it vanishes. The check requires the dimension 8 everywhere and `largest` below 10^{-9}.

```
reproduces(neutral_ok, "imaginary and zero frequencies: eigenspaces of dimension 8 "
           "are Krein-neutral",
           (PY, ["Q.one_particle_complex_frequency_Krein_neutral"]),
           (WL, ["Q_one_particle_complex_and_zero_frequencies_Krein_neutral"]))
```

Out [17] prints the three cases (eigenspace dimensions [8, 8], [8, 8] and [8]; Krein form zero on them: True) and the PASS line, with the checks `Q.one_particle_complex_frequency_Krein_neutral` and `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`. These are the growing extra-time waves: they carry no Krein norm at all (Section 18.23).

In [18], Figure 18c.6: the mapped spectra of plane waves.

```
ks = np.linspace(0.0, 3.0, 121)
curves = {}
```

```
for label, mass in (("plus", 1.0), ("minus", -1.0)):
    for axis in (1, 5):
        rows = []
        for kv in ks:
            k = np.zeros(8)
            k[axis] = kv
            rows.append(spectrum(h_matrix(mass, k)))
        curves[label, axis] = np.array(rows)
away = np.abs(ks - 1.0) > 0.05
equal = max(np.abs(curves["plus", axis][away] - curves["minus", axis][away]).max()
            for axis in (1, 5))
check(equal < 1e-6, "the plane-wave spectra of the masses +1 and -1 are equal")
```

The cell uses 121 momenta from 0 to 3. For the masses $+1$ and -1, and for a momentum along x_1 (`axis = 1`) or along x_5 (`axis = 5`), the 16 sorted eigenvalues of $h_m(k)$ are stored. `away` excludes the momenta within 0.05 of $k = 1$: there the extra-time wave has $w = 0$ and $h^2 = 0$, and the eigenvalues of such a matrix are very sensitive to rounding (a change of its entries by 10^{-16} can move them by about 10^{-8}, the square root). The check requires the two spectra to agree to 10^{-6} at all other momenta.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2))
for ax, axis, name in ((axes[0], 1, "3-space momentum $k_1$"),
                       (axes[1], 5, "extra-time momentum $k_5$")):
    ax.plot(ks, np.sqrt(np.clip(1.0 + (ks ** 2 if axis == 1 else -ks ** 2), 0, None)),
            color="#2a78d6", label="real part, mass $+1$")
    ax.plot(ks, np.sqrt(np.clip(-1.0 + (ks ** 2 if axis == 5 else -ks ** 2), 0,
                                None)), color="#eb6834",
            label="imaginary part, mass $+1$")
    dots = curves["minus", axis]
    ax.plot(ks[::5], np.abs(dots.real).max(axis=1)[::5], "o", color="#2a78d6",
            markersize=3.5, alpha=0.6, label="mass $-1$ (dots)")
    ax.plot(ks[::5], np.abs(dots.imag).max(axis=1)[::5], "o", color="#eb6834",
            markersize=3.5, alpha=0.6)
    ax.set_xlabel(name + " (units $m$)")
    ax.set_ylim(-0.1, 3.3)
axes[0].set_ylabel("frequency $|w|$: real and imaginary parts")
axes[0].legend(fontsize=8, loc="upper left")
```

Two panels. The lines are the exact formulas for the mass $+1$: the real part $\sqrt{1 + k_1^2}$ (left) or $\sqrt{1 - k_5^2}$ (right, where it exists) and the imaginary part, zero on the left and $\sqrt{k_5^2 - 1}$ on the right for $k_5 > 1$; `np.clip(x, 0, None)` replaces negative numbers by 0 before the square root. The dots are the largest absolute real part and the largest absolute imaginary part of the computed eigenvalues for the mass -1, at every fifth momentum.

```
save_figure(fig, "flat_dispersion",
            "Mapped spectra of plane waves in flat 4+4 space: the frequencies, the "
            "eigenvalues of the one-particle matrix $h_m(k)$, for the mass $+1$ "
            "(lines) and $-1$ (dots), against a 3-space momentum $k_1$ (left) and "
            "an extra-time momentum $k_5$ (right), in units of the mass. Blue: "
            "the real part $\\sqrt{1 + k_1^2}$ or $\\sqrt{1 - k_5^2}$; orange: the "
```

```
        "imaginary part, which appears when the extra-time momentum exceeds the "
        "mass (a growing mode). Each value is 8-fold, with both signs. The "
        "dots lie on the lines: the universes of mass $+m$ and $-m$ have the "
        "same one-particle spectrum, because $\\Gamma h_m\\Gamma = h_{-m}$.")
```

What you see in Figure 18c.6: on the left the frequency rises from 1 along $\sqrt{1+k_1^2}$ and the imaginary part stays zero; on the right the real part falls from 1 to 0 at $k_5 = 1$ and from there the imaginary part rises along $\sqrt{k_5^2 - 1}$, a growing wave whose growth rate increases without limit as k_5 grows. The dots of the mass -1 lie on the lines of the mass $+1$ everywhere: the universes of masses $+m$ and $-m$ have the same one-particle spectrum (Q4), because $\Gamma h_m \Gamma = h_{-m}$.

In [19], the Krein norms of single modes.

```
row = samples[0]
k = np.array(row["k"], dtype=float)
w = 5.0
h = h_matrix(row["m"], k)
columns, freqs = [], []
for value in (w, -w):
    V = eigenspace(h, value)
    ev, U = np.linalg.eigh(V.conj().T @ B @ V)  # diagonalise the Krein form
    for j in np.argsort(-ev):  # positive norms first
        u = V @ U[:, j] / np.sqrt(abs(ev[j]))  # Krein norm +1 or -1
        columns.append(u)
        freqs.append(value)
```

The first recorded sample, $m = 2$, $k = (k_0, \ldots, k_7) = (4, 1, 2, 0, 0, 0, 0, 0)$, with $w = \sqrt{4 + 16 + 1 + 4} = 5$. For each of the two eigenspaces ($+5$ and -5), `V` holds 8 orthonormal columns. The Krein form on the eigenspace is the 8×8 Hermitian matrix $V^\dagger B V$; `np.linalg.eigh` returns its eigenvalues `ev` and an orthonormal set of eigenvectors, the columns of `U`. The loop takes them in the order of decreasing eigenvalue (`np.argsort(-ev)`) and forms $u = VU_j/\sqrt{|\epsilon_j|}$, where U_j is column j of `U` and ϵ_j its eigenvalue. Its Krein norm is

$$u^\dagger B u = \frac{U_j^\dagger (V^\dagger B V) U_j}{|\epsilon_j|} = \frac{\epsilon_j U_j^\dagger U_j}{|\epsilon_j|} = \pm 1.$$

The first step inserts u; the second uses $(V^\dagger B V)U_j = \epsilon_j U_j$; the last uses $U_j^\dagger U_j = 1$. So the 16 columns have the Krein norms $+1$ or -1, and `freqs` records the frequency of each.

```
norms = np.array([(u.conj() @ B @ u).real for u in columns])
norms_G = np.array([((Gamma @ u).conj() @ B @ (Gamma @ u)).real for u in columns])
norms_0 = np.array([((gamma[0] @ u).conj() @ B @ (gamma[0] @ u)).real
                    for u in columns])
eig_G = all(np.allclose(h_matrix(-row["m"], k) @ (Gamma @ u), f * (Gamma @ u))
            for u, f in zip(columns, freqs))
eig_0 = all(np.allclose(h_matrix(-row["m"], R0 * k) @ (gamma[0] @ u),
                        f * (gamma[0] @ u)) for u, f in zip(columns, freqs))
say("Krein norms of the 16 columns: " + " ".join(f"{x:+.0f}" for x in norms))
reproduces(eig_G and eig_0 and np.allclose(norms_G, -norms)
```

```
          and np.allclose(norms_0, norms) and int((norms > 0).sum()) == 8,
          "Gamma u: eigenvector of h_-m, Krein norm reversed; gamma^(x0) u: kept",
          (WL, ["Q_Krein_metric_of_images"]),
          (PY, ["Q.image_krein_metric", "Q.T2_image_keeps_B"]))
```

`norms`, `norms_G` and `norms_0` are the Krein norms of u, of Γu and of $\gamma^{(x_0)}u$. `eig_G` checks that Γu is an eigenvector of $h_{-m}(k)$ with the same frequency, and `eig_0` that $\gamma^{(x_0)}u$ is one of $h_{-m}(R_0 k)$ (the maps of Q4). The check requires these, $(\Gamma u)^\dagger B(\Gamma u) = -u^\dagger B u$, $(\gamma^{(x_0)}u)^\dagger B(\gamma^{(x_0)}u) = u^\dagger B u$, and eight positive norms among the sixteen. Why the norms behave so: $(Mu)^\dagger B(Mu) = u^\dagger M^\dagger B M u$, and for the real symmetric matrices $M = \Gamma$ and $M = \gamma^{(x_0)}$ Lemma 4 gives $M^\dagger B M = M B M^\dagger = \sigma_M B$ with $\sigma_\Gamma = -1$ and $\sigma_{\gamma^{(x_0)}} = +1$ (Exercise 18.6). Out [19] prints the norms $+1, +1, +1, +1, -1, -1, -1, -1$ twice, once for each eigenspace, and the PASS line.

In [20], Figure 18c.7: the Krein norms as bars.

```
fig, ax = plt.subplots(figsize=(10.0, 4.2))
idx = np.arange(16)
ax.bar(idx - 0.27, norms, width=0.27, color="#2a78d6", label="$u^\\dagger Bu$")
ax.bar(idx, norms_G, width=0.27, color="#eb6834",
       label="$(\\Gamma u)^\\dagger B(\\Gamma u)$: T1 image")
ax.bar(idx + 0.27, norms_0, width=0.27, color="#1baf7a",
       label="$(\\gamma^{(x_0)}u)^\\dagger B(\\gamma^{(x_0)}u)$: T2 image")
ax.axvline(7.5, color="#52514e", linewidth=0.8)
ax.text(3.5, 1.35, "frequency $+5$", ha="center", fontsize=9)
ax.text(11.5, 1.35, "frequency $-5$", ha="center", fontsize=9)
ax.set_xticks(idx, [str(j + 1) for j in idx])
ax.set_xlabel("column number $j$ (eight per eigenspace)")
ax.set_ylabel("Krein norm")
ax.set_ylim(-1.6, 1.6)
ax.legend(fontsize=8, loc="lower left", ncol=3)
```

Three bars per column: the norm of u (blue, shifted left by 0.27), of Γu (orange, in the middle) and of $\gamma^{(x_0)}u$ (green, shifted right). A thin vertical line separates the two eigenspaces and two labels name their frequencies; `set_xticks` labels the columns 1 to 16.

```
save_figure(fig, "krein_norms",
            "Krein norms of plane-wave eigenvectors and of their images, for the "
            "recorded sample $m = 2$, $k = (1, 2, 0, 0, 0, 0, 0, 4)$, frequency "
            "$w = \\pm5$. Horizontal axis: the 16 eigenvector columns $u$, eight "
            "for $+5$ and eight for $-5$; vertical axis: the Krein norm. Blue: "
            "$u^\\dagger Bu$, four $+1$ and four $-1$ in each eigenspace (Krein "
            "inertia (4,4)). Orange: the chirality image $\\Gamma u$, an "
            "eigenvector of the mass $-m$ with every norm reversed. Green: the "
            "mirror image $\\gamma^{(x_0)}u$, an eigenvector of the mass $-m$ with "
            "every norm kept. This is the quantum reading Q at the level of single "
            "modes.")
```

What you see in Figure 18c.7: in each eigenspace four blue bars at $+1$ and four at -1 (the Krein inertia (4,4)); every orange bar points the other way; every green bar equals its blue bar. The chirality image (T1) of a mode with a positive Krein norm is a mode of the mass $-m$ with

a negative norm; the mirror image (T2) keeps the norm. This is Q1 and Q5 at the level of single modes.

In [21], the last check.

```
figure_names = ["rk4_convergence", "partner_components", "t1_pair_densities",
                "t2_mirror_densities", "family_spectrum", "flat_dispersion",
                "krein_norms"]
paths = [output_file(f"{FIGURE_FOLDER}/18c_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

The seven figure files must exist; the last line is ALL 21 CHECKS PASSED (notebook 18c).

What Notebook 18c established. The following results are COMPUTED for one homogeneous solution of the commuting field dirac16complex00, with the illustrative values ($m = 1$, $H = 0.2$, $\lambda = 0.3$, $z_0 = \pi/4$, seed 2026) along the canonical deflating history $a_4 = AHx_4$ ($A = 1$, from the Revision record): the Runge-Kutta solution equals the exact solution of the theory record to 4.3×10^{-9} (fourth order measured); its charge density obeys the local conservation law, with a nonzero flow through the brane, so the charge of the patch is not constant; the T1 partner, solved on its own, is $\Gamma\Psi$, with opposite charge density, current and energy-momentum tensor at every time, while the wrong partner $(-m, +\lambda)$ is not; the T2 partner on the mirror patch is $\gamma^{(x_0)}\Psi$, with S reversed and EQUAL charge and energy densities; the generators of the field and of both partners have equal spectra; and for plane waves the eight recorded samples of Q are reproduced, the growing waves are Krein-neutral, Γ reverses every Krein norm and $\gamma^{(x_0)}$ keeps it. ASSUMED: the mirror patch glued at the degenerate brane, and a fixed gravitational field. NOT shown: any process that creates a universe or a pair. The partners are solutions of OTHER parameter sets, computed from the first one; nothing in these equations makes them appear.

18.28 What the pairing theorems do not establish

T1, T2 and Q are exact. Their content is precisely this: explicit invertible maps between the solutions of two theories with different parameters, with stated signs for the Lagrangian, the energy-momentum tensor, the current and the canonical anticommutator. Both Revision verifiers wrote, independently of each other, a list of what these theorems do NOT establish, and a comparison check confirms that the two lists cover the same topics (`python-pairing.json`, check `compare.theory.not_established`; the lists are the entries `not_established` of `pairing-theory.json` and of `python-pairing.json`). Merged, with the reason for each item:

1. **No creation process.** The theorems map solutions to solutions and quantities to quantities. Nothing in these equations produces a universe, a pair of universes, or a change of the number of universes; no transition from "no universe" to "two universes", no initial state, no vacuum decay (the change of a state without particles into one of lower energy) and no tunnelling process (a quantum transition through a region that the classical equations forbid) is derived.
2. **No rate, probability or amplitude.** No transition amplitude, probability, cross-section (a measure, with the units of an area, of how likely a reaction is), rate or Bogoliubov coefficient

(the number that says how many particles a changing background creates from a vacuum) for creating universes of masses $+m$ and $-m$ is computed or implied; no wave function of the universe (a quantum state of a whole universe, geometry included) and no path integral (a sum over every possible history of the fields) is part of the theorems.

3. **No dynamical necessity.** No equation and no conservation law forces the partner to exist. A single universe of mass $+m$ is an equally valid solution without its partner. T1 and T2 are correspondences between the solutions of two parameter sets, not a mechanism.

4. **T1 is not a symmetry of one theory, and for $\lambda \neq 0$ it is not a pure $+m$ / $-m$ pairing.** It changes the parameters to $(-m, -\lambda)$ and the sign of the action. The pairing $(m, \lambda) \to (-m, \lambda)$ at fixed coupling is T2, at EQUAL, not opposite, energy-momentum.

5. **The zero total of a T1 pair has a limited meaning.** $T + T' = 0$, $J + J' = 0$ and $Q + Q' = 0$ hold for classical bilinears (a configuration and its image) and as an operator identity within ONE quantum system (Q2). They do not hold for two independently quantised universes, whose generators add without cancelling (Q3).

6. **Test field only.** The gravitational field is fixed and the same for both members. The back-reaction through the field equations for a_4 is not part of the theorems. The only statement about it is the corollary C1 of Chapter 20: a T1 pair taken as the complete classical source of the author's metric is a zero source, and the Einstein equations then have no solution for $H > 0$. That is a statement about sources in one common geometry, not a derivation that the geometry, or the pair, is created.

7. **The Z2 brane is assumed.** The mirror across $z = \pi/2$ uses the ASSUMED Z2 construction; the metric is degenerate there ($g_{00} = 0$ and $\sqrt{|g|} = 0$), and no junction condition, brane tension or matching of the field across the brane is derived.

8. **Quantum positivity is not established.** Every real-frequency eigenspace of the one-particle matrix has the Krein inertia (4,4), and the growing waves are Krein-neutral (Q4); a positive-norm Fock space for either universe is not established by these theorems.

9. **dirac16complex00 is a classical field**: no quantum statement is made for it.

10. **The Kohn-Sham level is separate.** Theorem T3 is proved in Chapter 19 with its own hypotheses (the ASSUMED Z2 brane, instantaneous mean-field Kohn-Sham states); nothing in this chapter establishes it.

The author's hypothesis. The statement that the big bang CREATES universes in pairs of masses $+m$ and $-m$ is the author's HYPOTHESIS. This chapter proves that the solutions come in partnered families (T1 and T2) and how a quantised partner must be read (Q); it does not prove, and the equations of this book cannot prove, that any universe is created, in pairs or otherwise. Chapter 20 takes up the question "Do universes come in pairs?" and states the hypothesis as a hypothesis.

Matter and antimatter. Where this chapter touches matter and antimatter, the exact statements are these. PROVED: the theory has an exact U(1) symmetry, and its current obeys the exact LOCAL conservation law $\partial_\mu(\cos z\, J^\mu) = 0$ on shell (`charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity`; Notebook 18c checks the local law along its solution in In [10], with the frame current q_0). The charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, d^7x$ of a slice $x_4 = \text{const}$ is constant only if

no charge flows through the boundary of the patch, that is through the brane $z = \pi/2$. There the flow $\sqrt{|g|}J^{x_0} = \sin z\,(-i\bar{\Psi}\gamma^{(x_0)}\Psi)$ does not vanish in general (`python-scope.json`, check `good_sector_hermiticity_up_to_the_brane_flux`: the boundary term vanishes at the tip $z \to 0$ but not at $z = \pi/2$). The homogeneous solutions of Section 18.21 exchange charge through the brane: their patch charge changes at the rate $-V_6 q_0$, and Notebook 18c prints q_0 between -0.3454 and 0.3454 (Out [10]; the oscillating charge density of Figure 18c.3); the T2 mirror partner has the equal charge density, so gluing the two patches does not restore a constant total. Hence the statement "no net charge can be generated inside one universe" holds only under a **no-flux condition at the brane**, $\sqrt{|g|}J^{x_0} = 0$ at $z = \pi/2$; that condition is ASSUMED where it is used, and deriving it is OPEN: it belongs to the junction condition at the brane, which is not derived (item 7). A T1 partner carries the opposite charge, so a T1 pair has total charge zero as classical bilinears (T1d); this is a universe and anti-universe statement about solutions. Such ideas form a class in the published literature; one example is L. Boyle, K. Finn and N. Turok, "CPT-Symmetric Universe", Phys. Rev. Lett. 121, 251301 (2018); it is cited only as an example of the class, and nothing in this chapter is taken from it. On real fields T1 is the mass-reversing charge conjugation by the matrix Γ (Section 18.13). The theory as built does NOT solve the matter-antimatter problem. In 1967 Sakharov showed that an excess of matter can grow from an equal start only if three conditions hold: a process that changes the baryon number, a violation of the symmetries C and CP, and a departure from thermal equilibrium (Chapter 21). The theory as built has no baryons, no process that changes the baryon number (its U(1) current is conserved locally, and globally under the no-flux condition at the brane), no violation of CP built in or computed, and no computation of a departure from thermal equilibrium. To solve the problem it would need all of these, and a computed excess of matter that agrees with the measured one. Every scenario in which our universe is one member of such a pair, or in which the pairing explains the excess of matter, is a HYPOTHESIS. Chapter 21 treats matter and antimatter from zero.

18.29 What we proved, what we computed, what we assumed

Proved. Exactly, for both fields unless something else is said, each with the Revision record that verifies it and the notebook of this chapter that reproduces it:

statement	**where it is verified**	**notebook**
the gammas of the record are the author's eight real 16×16 signed permutation matrices, rebuilt from their formulas	`python-pairing.json`, check `gammas.equal_wolfram_fixture`	18a
$\sqrt{\|g\|} = \cos z$; the 12 components of the canonical spin connection; $\gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ for every history a_4 (the deflation terms cancel), $-3H\gamma^{(x_0)}$ on the mirror patch	`wolfram-pairing.json`, check `primordial_vielbein`; `python-pairing.json`, check `geometry.spin_connection_components`; `python-field-theory.json`, checks `gamma_mu_Omega_mu_equals_3H_gamma_x0` and `divergence_of_sqrtg_gamma`	18b, 18c

statement	where it is verified	notebook
with a changing $b_4(x_4)$: $\sqrt{\lvert g\rvert} = b_4^{3/2}\cos z$ and $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$	`wolfram-field-theory-b4.json`, checks `sqrt_det_g` and `gammaOmega_total`; `python-field-theory-b4.json`, check `gammaOmega_total`	Chapter 7 (07b)
Lemma 1: Γ real, symmetric, $\Gamma\Gamma = 1$, anticommuting with every gamma, commuting with C and every S^{ab}	`wolfram-pairing.json`, check `Gamma_properties`; `python-pairing.json`, checks `gammas.Gamma` and `gammas.Gamma_anticommutes`	18a
Lemma 2: $\bar\Psi X\Psi \to (-1)^k\bar\Psi X\Psi$ under $\Psi\to\Gamma\Psi$; $\Gamma B\Gamma = -B$	`wolfram-pairing.json`, checks `T1_kernel_scalar`, `T1_kernel_kinetic`, `T1_kernel_connection` and `T1_kernel_field_equation`; `python-pairing.json`, checks `T1.general_field.matrix_identities` and `Q.image_krein_metric`	18a
Lemma 3: the eight reflections P_n in Pin(4,4), their characters $-\eta_{nn}$, the reflection table	`wolfram-pairing.json`, checks `T2_Pn_in_Pin44`, `T2_Pn_covers_the_reflection`, `T2_character_of_Pn` and `T2_Gamma_times_Pn_is_gamma_n`; `python-pairing.json`, checks `T2.general_field.reflection_table` and `compare.theory.reflection_table`	18a
Lemma 4: the 17 Krein signs	`wolfram-pairing.json`, check `Q_Krein_metric_of_images`; `python-pairing.json`, check `compare.theory.krein_signs`	18a
T1 (T1a to T1d), in every gravitational field taken as a fixed background, both statistics	`wolfram-pairing.json`, the 51 checks of T1; `python-pairing.json`, the 23 checks whose names begin with `T1.`, and `compare.theory.theorem_T1`	18b, 18c
T2 (T2a to T2d); the author's-field version under the ASSUMED Z2 construction	`wolfram-pairing.json`, the 28 checks of T2; `python-pairing.json`, the 16 checks of T2 and `compare.theory.theorem_T2`	18b, 18c

statement	where it is verified	notebook
Q1 to Q5, for the canonically quantised dirac16complex	`wolfram-pairing.json`, the 12 checks whose names begin with `Q_`; `python-pairing.json`, the 10 checks whose names begin with `Q.`, and `compare.theory.theorem_Q`	18a, 18c
the homogeneous solutions: S constant, $M^2 = (9H^2 - (m+\lambda S)^2) I_{16}$, the exact formula; $\rho = mS + \frac{\lambda}{2}S^2$, every pressure $\frac{\lambda}{2}S^2$, $T_{x_4 x_0} = 0$; the LOCAL conservation law of the charge, $dJ^{x_4}/dx_4 = -6Hq_0$ with $q_0 = -i\bar{\Psi}\gamma^{(x_0)}\Psi$ (for these solutions the flow $\sin z\, q_0$ through the brane does not vanish, so the charge of the patch is not constant)	`python-field-theory.json`, checks `exact_nonlinear_homogeneous_solution`, `commuting_homogeneous_on_shell_rho_p`, `commuting_T_x4x0_homogeneous` and `commuting_current_conservation`; `wolfram-field-theory.json`, check `exact_solution_nonlinear_homogeneous_C`	18c
the two charge-conjugation matrices $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$; on real fields T1 is the mass-reversing conjugation by Γ; the exact U(1) symmetry and its LOCAL conservation law $\partial_\mu(\cos z\, J^\mu) = 0$ (a constant total charge needs the no-flux condition at the brane, listed under Assumed and Open)	`charge-conjugation-and-u1.json`, checks `intertwiners_same_mass`, `intertwiners_reversed_mass`, `charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus`, `real_fields_charge_conjugation` and `u1_noether_matrix_identity`	none (Chapters 5 and 21)

Computed. Numerical results of the notebooks, with their accuracy:

- Notebook 18a: every result is exact whole-number arithmetic, except the illustration of In [7], where Γ commutes with a random connection to below 10^{-12}.
- Notebook 18b: every result is exact (sympy). The monomial counts, evaluated at one sample point, are 408 (commuting) and 392 (Grassmann) for the Lagrangian, equal to the counts recorded by the Wolfram verifier, and 376 (diagonal) or 64 and 80 (off-diagonal) for the components of $T_{\mu\nu}$; they describe the size of the polynomials and prove nothing.
- Notebook 18c (illustrative values $m = 1$, $H = 0.2$, $\lambda = 0.3$, $z_0 = \pi/4$, seed 2026; history $A = 1$ from the record): $S = 0.897319$, $m + \lambda S = 1.269196$, $w = 1.118417$; Runge-Kutta errors at $x_4 = 20$ for 250 to 4000 steps from 2.0×10^{-5} to 2.6×10^{-10}, measured orders 4.10, 4.06, 4.03 and 4.01, largest error with 2000 steps 4.3×10^{-9} against the exact formula of the record; the T1 partner equals $\Gamma\Psi$ and the T2 partner equals $\gamma^{(x_0)}\Psi$ to below 10^{-12} at all 2001 times, while the wrong partner $(-m, +\lambda)$ is off by 1.641; S, J^μ and $T_{\mu\nu}$ of the partners as predicted,

to a relative 10^{-12} at 101 sampled times; the local conservation law $dJ^{x_4}/dx_4 = -6Hq_0$ to 10^{-12} at the same times, with the frame current q_0 between -0.3454 and 0.3454, so a nonzero flow through the brane; the eight plane-wave samples of the record reproduced (dimensions 8 and 8, Krein inertia (4,4)); the plane-wave spectra of the masses ± 1 equal to 10^{-6} at 121 momenta (away from $k_5 = 1$); the Krein norms ± 1 of the 16 modes of the first sample reversed by Γ and kept by $\gamma^{(x_0)}$.

Assumed.

- The gravitational field is a fixed background (a test field) in every theorem of this chapter: it is not varied, and both members of a pair live in the same field.
- The Z2 mirror construction: gluing the mirror patch to the patch at the degenerate brane $z = \pi/2$; it enters T2 in the author's field, the mirror universe of Q5 and the mirror partner of Notebook 18c (`pairing-theory.json`, theorem T2, its hypothesis on the primordial field).
- For Q: the canonical quantisation of dirac16complex with x_4 as the time and with anticommutators (part of its definition as a fermion field); flat 4+4 space, or frozen coefficients at one point, for the one-particle statements.
- The history $a_4 = AHx_4$ with $A = 1$ used in the figures is the PRESCRIBED BACKGROUND of the Revision Kohn-Sham record (`parameters.json`; `ks-source-conditions.json`, check `ks_history_is_a_prescribed_background`). The theorems themselves hold for every history.
- The value $3H\gamma^{(x_0)}$ of $\gamma^\mu \Omega_\mu$ belongs to the diagonal vielbein (`python-scope.json`; Chapter 8); none of the theorems depends on it.
- The background $b_4 = 1$ of the author's metric of 2026-10-14, in every statement about the author's field and in all three notebooks. T1 and the general-field version of T2 hold for every vielbein, so for any $b_4(x_4)$ as well (Sections 18.3, 18.16 and 18.21).
- The illustrative values of Notebook 18c.
- Wherever a constant total charge of one universe is used (the statement that no net charge can be generated inside one universe): the no-flux condition $\sqrt{|g|}J^{x_0} = 0$ at the brane $z = \pi/2$. It is not derived (see Open).

Hypothesis.

- That the big bang creates universes in pairs of masses $+m$ and $-m$: the author's HYPOTHESIS, not derived from any equation of this book.
- That our universe has an anti-universe partner, or that the pairing explains the observed excess of matter over antimatter.

Open.

- Any creation process, rate or amplitude for universes, which these equations do not contain.

- The gravitational back-reaction of a pair: the pair as part of the source of the field equations for a_4 (C1 treats only the case in which a T1 pair is the complete source).
- A junction condition at the brane $z = \pi/2$ that would replace the assumed Z2 construction.
- The no-flux condition $\sqrt{|g|}J^{x_0} = 0$ at the brane, which a constant total charge of one universe needs; the homogeneous solutions of Section 18.21 violate it (Notebook 18c, Out [10]), and no equation of this book imposes it.
- A positive Hilbert space for the whole quantised field (all momenta, the curved metric), and the fate of the growing extra-time waves.

18.30 Exercises

Exercise 18.1. The factors of Γ are $\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}\gamma^{(x_4)}\gamma^{(x_5)}\gamma^{(x_6)}\gamma^{(x_7)}$, in this order. Use Rule P (Section 18.5) to write $P_n = \Gamma\gamma^{(n)}$ as a sign times the product of the seven other gammas in the order of Γ, for $n = x_1$, x_4, x_5 and x_0. Find the general rule for the sign and check it against all eight signs $-1, +1, -1, +1, +1, -1, +1, -1$ of the record.

Exercise 18.2. Notebook 18c has $S = 0.897319$, $m = 1$ and $\lambda = 0.3$ (Out [4]). For homogeneous solutions the record gives $\rho = mS + \frac{\lambda}{2}S^2$ and $p = \frac{\lambda}{2}S^2$ in every direction. (a) Compute ρ, p and the equation of state $w = p/\rho$ of the solution, and compare w with the formula $w = \lambda S/(2m + \lambda S)$ of the record. (b) The same for the T1 partner ($-m$, $-\lambda$, the same S). (c) The same for the T2 partner ($-m$, λ, the scalar $-S$). (d) Add the energy densities of each pair.

Exercise 18.3. (a) With $m + \lambda S = 1.269196$ and $3H = 0.6$, compute the frequency w of the solution of Notebook 18c, the period of the field and the period of its charge density J^{x_4}, and the number of oscillations of J^{x_4} between $x_4 = 0$ and 20 (Figure 18c.3). (b) Compute the coefficient V, the frequency and the period of the wrong partner $(-m, +\lambda)$ started from $\Gamma\Psi(0)$ (Figure 18c.2). (c) For which masses m (at the same λ and S) do the homogeneous solutions grow instead of oscillating, and what is the largest growth rate (Figure 18c.5)?

Exercise 18.4. (a) From $C\gamma^{(a)}C^{-1} = -(\gamma^{(a)})^T$ show that $(S^{ab})^T C = -CS^{ab}$ for $a \neq b$. (b) Conclude that $\Omega_\mu^T C = -C\Omega_\mu$ for every real connection, and that $D_\mu\bar{\Psi} = (D_\mu\Psi)^\dagger C$, the formula used by Notebook 18c in In [9].

Exercise 18.5. Show that $Bh_m(k) = h_m(k)^\dagger B$ for real m and real momenta k, with $h_m(k) = -im\gamma^{(x_4)} - \gamma^{(x_4)}K$, $K = \sum_{a\neq 4} k_a\gamma^{(a)}$ and $B = -iC\gamma^{(x_4)}$. Use only the reality of the gammas, $(\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$, the Clifford relation and $C\gamma^{(a)}C^{-1} = -(\gamma^{(a)})^T$.

Exercise 18.6. Let $h_m(k)u = wu$ with a real w and the Krein norm $u^\dagger Bu = 1$. (a) Show that Γu is an eigenvector of $h_{-m}(k)$ with the same w and the Krein norm -1. (b) Show that $\gamma^{(x_0)}u$ is an eigenvector of $h_{-m}(R_0 k)$ with the same w and the Krein norm $+1$. (c) Which bars of Figure 18c.7 show (a) and (b)?

Exercise 18.7. Write $\mathcal{L}_{m,\lambda}[\Psi] = \sqrt{|g|}\,[K - mS - \frac{\lambda}{2}S^2]$. (a) Compute $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{m,\lambda}[\Psi]$, $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,\lambda}[\Psi]$ and $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,-\lambda}[\Psi]$. (b) Which terms survive in each, and which bars of Figure 18b.2 show it? (c) For which λ is T1 a pure $+m$ / $-m$ pairing?

Exercise 18.8. (a) Compute $\sin^{1/6} z$, f_0 and $\sqrt{|g|}$ at $z = \pi/4$ on the patch and at $z = 3\pi/4$ on the mirror patch, and check that the metric takes the same values at the two points. (b) Use the

divergence form of Section 18.27 to compute the coefficient of $\gamma^{(x_0)}$ in $\gamma^\mu\Omega_\mu$ on the mirror patch, with $H = 0.2$. (c) Pull the hidden leg $e^0 = f_0\,dx_0$ of the mirror patch back by ϕ: $x_0 \to \pi/(6H) - x_0$, and say which frame reflection results.

Exercise 18.9. A student writes: "Theorem T1 proves that the big bang creates every universe of mass m together with a universe of mass $-m$: the two have opposite energies, so the pair costs no energy, and nothing forbids it." Name every step of this sentence that the equations of this book do not establish, and say what they do establish instead.

18.31 Answers to the exercises

Answer 18.1. Write $\gamma^{(a)}$ as γ_a for short. For $n = x_1$: $\Gamma\gamma_1 = \gamma_0\gamma_1\gamma_2\gamma_3\gamma_4\gamma_5\gamma_6\gamma_7\gamma_1$. Move the last γ_1 to the left past $\gamma_7, \gamma_6, \gamma_5, \gamma_4, \gamma_3, \gamma_2$: six different gammas, each exchange costs -1 (Rule P), so the sign is $(-1)^6 = +1$, and $\Gamma\gamma_1 = \gamma_0\gamma_1\gamma_1\gamma_2\gamma_3\gamma_4\gamma_5\gamma_6\gamma_7$. Then $\gamma_1\gamma_1 = \eta_{11} = +1$ (Clifford relation): $P_{x_1} = +\gamma_0\gamma_2\gamma_3\gamma_4\gamma_5\gamma_6\gamma_7$, sign $+1$. For $n = x_4$: three exchanges (past $\gamma_7, \gamma_6, \gamma_5$), $(-1)^3 = -1$, then $\gamma_4\gamma_4 = \eta_{44} = -1$: the sign is $(-1)(-1) = +1$. For $n = x_5$: two exchanges, $(+1)$, then $\eta_{55} = -1$: the sign is -1. For $n = x_0$: seven exchanges (past $\gamma_7, \dots, \gamma_1$), $(-1)^7 = -1$, then $\eta_{00} = +1$: the sign is -1, $P_{x_0} = -\gamma_1\gamma_2\cdots\gamma_7$. General rule: the sign is $(-1)^{N_n}\eta_{nn}$, where N_n is the number of factors that stand to the right of $\gamma^{(n)}$ in Γ, $N_n = 7 - n$. With $N = 7, 6, 5, 4, 3, 2, 1, 0$ for $x_0, \dots, x_7$ and $\eta = (+1, +1, +1, +1, -1, -1, -1, -1)$: x_0: $(-1)^7 = -1$; x_1: $+1$; x_2: $(-1)^5 = -1$; x_3: $+1$; x_4: $(-1)^3(-1) = +1$; x_5: $(+1)(-1) = -1$; x_6: $(-1)(-1) = +1$; x_7: $(+1)(-1) = -1$. These are the eight signs of the record (data table `reflections` of `pairing-theory.json`; Notebook 18a, Out [12]).

Answer 18.2. $S^2 = 0.897319^2 = 0.805181$. (a) $p = \frac{0.3}{2} \times 0.805181 = 0.120777$; $\rho = 1 \times 0.897319 + 0.120777 = 1.018096$; $w = 0.120777/1.018096 = 0.11863$. The record's formula: $\lambda S/(2m + \lambda S) = 0.269196/2.269196 = 0.11863$, the same, as it must be: dividing p and ρ by $S/2$ gives $w = \lambda S/(2m + \lambda S)$. (b) With $-m$ and $-\lambda$ and the same S: $\rho' = -mS - \frac{\lambda}{2}S^2 = -1.018096$ and $p' = -\frac{\lambda}{2}S^2 = -0.120777$ (for $U = -\frac{\lambda}{2}S^2$, $p' = SU' - U = -\lambda S^2 + \frac{\lambda}{2}S^2$); $w' = p'/\rho' = 0.11863$, the same equation of state, as T1 says (Section 18.13). (c) With $-m$, λ and the scalar $-S$: $\rho'' = (-m)(-S) + \frac{\lambda}{2}(-S)^2 = 1.018096$ and $p'' = \frac{\lambda}{2}S^2 = 0.120777$: equal to those of the solution. (d) $\rho + \rho' = 0$: the T1 pair has zero total energy density as classical bilinears. $\rho + \rho'' = 2.036192$: nothing cancels in a T2 pair.

Answer 18.3. (a) $w^2 = 1.269196^2 - 0.6^2 = 1.610858 - 0.36 = 1.250858$, so $w = 1.11842$ (Out [14] prints 1.118417, computed with the unrounded S). The field oscillates with the period $2\pi/w = 5.618$. $J^{x_4} = \Psi^\dagger B\Psi$ is quadratic in $\Psi = \cos(wx_4)\Psi(0) + \frac{\sin(wx_4)}{w}M\Psi(0)$, so it contains $\cos^2$, $\sin^2$ and $\sin\cos$, which are combinations of 1, $\cos(2wx_4)$ and $\sin(2wx_4)$: its period is $\pi/w = 2.809$, and $20/2.809 = 7.1$, so seven full oscillations, as in Figure 18c.3. (b) $S[\Gamma\Psi] = S$, so $V = -m + \lambda S = -1 + 0.269196 = -0.730804$; $w^2 = 0.534075 - 0.36 = 0.174075$, $w = 0.41722$, period $2\pi/0.41722 = 15.06$: in Figure 18c.2 the dashed curve completes about $20/15.06 = 1.3$ oscillations. (c) The solutions grow when $(m + \lambda S)^2 < 9H^2$, that is $-0.6 < m + 0.269196 < 0.6$, or $-0.869196 < m < 0.330804$. There $M^2 = (9H^2 - V^2)I_{16}$ is positive, the eigenvalues are $\pm\kappa$ with $\kappa = \sqrt{9H^2 - V^2}$, and the largest growth rate is $3H = 0.6$, at $V = 0$, that is $m = -0.269196$: the top of the arc in Figure 18c.5.

Answer 18.4. (a) The given relation, solved for the transpose: $(\gamma^{(a)})^T = -C\gamma^{(a)}C^{-1}$. For $a \neq b$, $S^{ab} = \frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ and

$$(S^{ab})^T = \tfrac{1}{2}(\gamma^{(b)})^T(\gamma^{(a)})^T = \tfrac{1}{2}\big(-C\gamma^{(b)}C^{-1}\big)\big(-C\gamma^{(a)}C^{-1}\big)$$
$$= \tfrac{1}{2}C\gamma^{(b)}\gamma^{(a)}C^{-1} = -\tfrac{1}{2}C\gamma^{(a)}\gamma^{(b)}C^{-1} = -CS^{ab}C^{-1}.$$

The transpose of a product reverses the order; the two minus signs multiply to $+1$; $C^{-1}C = 1$; two different gammas anticommute. Multiplying on the right by C gives $(S^{ab})^TC = -CS^{ab}$. (b) $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$ with real numbers $\omega_{\mu ab}$; transposing is linear, so $\Omega_\mu^T C = \frac{1}{2}\sum\omega_{\mu ab}(S^{ab})^TC = -C\Omega_\mu$. Then, with C constant and $\Omega_\mu^\dagger = \Omega_\mu^T$ (real),

$$(D_\mu\Psi)^\dagger C = (\partial_\mu\Psi)^\dagger C + \Psi^\dagger\Omega_\mu^T C = \partial_\mu(\Psi^\dagger C) - \Psi^\dagger C\Omega_\mu = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu = D_\mu\bar\Psi.$$

Answer 18.5. Step 1, the adjoint of h. The gammas are real, so $(\gamma^{(a)})^\dagger = (\gamma^{(a)})^T = \eta_{aa}\gamma^{(a)}$; in particular $(\gamma^{(x_4)})^\dagger = -\gamma^{(x_4)}$. Write $\tilde K = K^\dagger = \sum_{a\neq 4}k_a\eta_{aa}\gamma^{(a)}$ (k_a real). With $(XY)^\dagger = Y^\dagger X^\dagger$ and $(-i)^* = i$:

$$h^\dagger = im(\gamma^{(x_4)})^\dagger - K^\dagger(\gamma^{(x_4)})^\dagger = -im\gamma^{(x_4)} + \tilde K\gamma^{(x_4)}.$$

Step 2, two facts about C. From $C\gamma^{(a)}C^{-1} = -(\gamma^{(a)})^T = -\eta_{aa}\gamma^{(a)}$: for $a = 4$ ($\eta_{44} = -1$), C commutes with $\gamma^{(x_4)}$; and summing with the coefficients k_a, $CKC^{-1} = -\tilde K$, that is $CK = -\tilde KC$. Step 3, the two products, using $\gamma^{(x_4)}\gamma^{(x_4)} = -1$:

$$Bh = -iC\gamma^{(x_4)}\big(-im\gamma^{(x_4)} - \gamma^{(x_4)}K\big) = (-i)(-i)m\,C\gamma^{(x_4)}\gamma^{(x_4)} + iC\gamma^{(x_4)}\gamma^{(x_4)}K = mC - iCK,$$

$$h^\dagger B = \big(-im\gamma^{(x_4)} + \tilde K\gamma^{(x_4)}\big)(-iC\gamma^{(x_4)}) = -m\,\gamma^{(x_4)}C\gamma^{(x_4)} - i\tilde K\gamma^{(x_4)}C\gamma^{(x_4)} = mC + i\tilde KC.$$

In the second line $\gamma^{(x_4)}C\gamma^{(x_4)} = C\gamma^{(x_4)}\gamma^{(x_4)} = -C$ (Step 2). By Step 2, $-iCK = i\tilde KC$, so $Bh = h^\dagger B$. (The result $Bh = mC - iCK$ is the energy kernel $\mathcal{E}_m(k)$ of Section 18.23, which the sympy record writes as $m\,C - i\,k_aC\gamma^{(a)}$.)

Answer 18.6. (a) By Q4, $\Gamma h_m(k)\Gamma = h_{-m}(k)$, and $\Gamma\Gamma = 1$, so $h_{-m}(k)\Gamma u = \Gamma h_m(k)\Gamma\Gamma u = \Gamma h_m(k)u = w\,\Gamma u$. The Krein norm: $(\Gamma u)^\dagger B(\Gamma u) = u^\dagger\Gamma^\dagger B\Gamma u = u^\dagger\Gamma B\Gamma u = -u^\dagger Bu = -1$, because Γ is real and symmetric ($\Gamma^\dagger = \Gamma$) and $\Gamma B\Gamma = -B$ (Lemma 2). (b) By Q4, $\gamma^{(x_0)}h_m(k)\gamma^{(x_0)} = h_{-m}(R_0k)$, and $\gamma^{(x_0)}\gamma^{(x_0)} = \eta_{00} = 1$, so $h_{-m}(R_0k)\gamma^{(x_0)}u = \gamma^{(x_0)}h_m(k)u = w\,\gamma^{(x_0)}u$. $\gamma^{(x_0)}$ is real and symmetric (space-like), so $(\gamma^{(x_0)}u)^\dagger B(\gamma^{(x_0)}u) = u^\dagger\gamma^{(x_0)}B\gamma^{(x_0)}u = u^\dagger Bu = 1$ by Lemma 4 ($\sigma_{\gamma^{(x_0)}} = +1$). (c) In Figure 18c.7 every orange bar (Γu) is the blue bar (u) turned upside down, which is (a), and every green bar ($\gamma^{(x_0)}u$) equals the blue bar, which is (b).

Answer 18.7. (a) By T1, $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = \sqrt{|g|}\,[-K - mS - \frac{\lambda}{2}S^2]$ (K reversed, S kept). Adding $\mathcal{L}_{m,\lambda}[\Psi] = \sqrt{|g|}\,[K - mS - \frac{\lambda}{2}S^2]$ gives $-\sqrt{|g|}\,(2mS + \lambda S^2)$. Adding $\mathcal{L}_{-m,\lambda}[\Psi] = \sqrt{|g|}\,[K + mS - \frac{\lambda}{2}S^2]$ gives $-\sqrt{|g|}\,\lambda S^2$. Adding $\mathcal{L}_{-m,-\lambda}[\Psi] = \sqrt{|g|}\,[K + mS + \frac{\lambda}{2}S^2]$ gives 0. (b) In the first sum the kinetic terms cancel and the mass term and the S^2 term survive: the fourth pair of bars of Figure 18b.2. In the second only the S^2 term survives: the fifth pair, the shortest nonzero bars. The third sum is the zero bar of T1. (c) For $\lambda = 0$ the second sum vanishes, and T1 pairs $(m, 0)$ with $(-m, 0)$: a pure $+m$ / $-m$ pairing of free fields.

Answer 18.8. (a) $\sin(\pi/4) = \sin(3\pi/4) = 1/\sqrt{2} = 0.707107$, so $\sin^{1/6} z = 2^{-1/12} = 0.943874$ at both points, and the components $e^{\pm 2a_4} \sin^{1/3} z$ are equal. On the patch $f_0 = \cot(\pi/4) = 1$; on the mirror patch $f_0 = -\cot(3\pi/4) = -(-1) = 1$; so $g_{00} = f_0^2 = 1$ at both. $\sqrt{|g|} = \cos(\pi/4) = 0.707107$ on the patch and $-\cos(3\pi/4) = 0.707107$ on the mirror patch. The metric takes the same values: the mirror is an isometry. (b) On the mirror patch $\sqrt{|g|}/f_0 = (-\cos z)/(-\cot z) = \sin z$, whose derivative along x_0 is $6H \cos z$; divided by $2\sqrt{|g|} = -2\cos z$ this gives $-3H = -0.6$. (c) At the image point the hidden leg is $-\cot(\pi - z)\, d(\frac{\pi}{6H} - x_0) = \cot z\, (-dx_0) = -\cot z\, dx_0 = -e^0$, because $\cot(\pi - z) = -\cot z$ and $d(\text{const} - x_0) = -dx_0$. The other seven legs are unchanged, so the pulled-back frame is the original frame with the direction x_0 reflected, $R_0 e$ (Section 18.16).

Answer 18.9. Step by step. "Theorem T1 proves that the big bang creates": no. T1 maps the solutions of the theory (m, λ) onto those of $(-m, -\lambda)$; no equation of this book describes the creation of a universe, in pairs or otherwise, and no process, rate, probability or amplitude follows (Section 18.28, items 1 and 2). "every universe of mass m together with a universe of mass $-m$": no. A universe of mass m is an equally valid solution without any partner (item 3), and the T1 partner has the coupling $-\lambda$, so for $\lambda \neq 0$ it is not merely a universe of mass $-m$ (item 4). "the two have opposite energies": true for the classical bilinears of a configuration and its image, and as an operator identity within one quantum system (T1c, Q2); false for two independently quantised universes, whose energies add without cancelling (Q3; item 5). "so the pair costs no energy": the theorems contain no energy balance of a creation process; the only statement about the pair as a source of gravity is the corollary C1 of Chapter 20, which says that a T1 pair taken as the complete source of the author's metric is a zero source, for which the Einstein equations have no solution with $H > 0$ (item 6). "and nothing forbids it": not established either way; the equations neither forbid nor produce it. What the equations do establish: the exact maps T1 and T2 between solution sets, with their signs, and the quantum reading Q. That the big bang creates universes in pairs remains the author's HYPOTHESIS.

19 T3: the Kohn-Sham universes of mass $+M$ and $-M$

Chapter 18 proved exact pairing theorems for the fields themselves: multiplied by the chirality matrix Γ, a field configuration of the theory with the mass m and the coupling λ becomes a configuration of the theory with $-m$ and $-\lambda$ (theorem T1). But the numbers that Chapters 15 and 16 computed, the levels, energies, densities and pressures of a gas of quanta of the field dirac16complex in the author's primordial universe, are not field configurations. They are **Kohn-Sham states**: quanta that fill one-quantum levels in a common, self-made mean field, with densities computed by the expectation-value rule of the quantised field, with boundary conditions at the two ends of the hidden direction, at one instant of the history in which ordinary 3-space inflates while the three extra times x_5, x_6, x_7 deflate exponentially. For these states the pairing question must be asked again, and it has a different answer. Theorem **T3** of the Revision record says: every self-consistent Kohn-Sham state of the universe with the bare mass $+M$ has an exact partner in the universe with the bare mass $-M$, with the **same** coupling $+\lambda$ and a transformed boundary condition at the tip, and the partner has the **same** levels, occupations, energies and energy-momentum profiles, while its scalar density is the mirror image. This chapter proves T3 line by line, shows by exact formulas what goes wrong when the boundary condition is not transformed, and watches the theorem at work in two complete notebooks: one that repeats every step of the proof by hand, and one that runs the Revision Rust Kohn-Sham solver 62 times on universes of mass $+M$, their partners and two kinds of wrong partners. Like every pairing theorem of this book, T3 is an exact map between two sets of

solutions; it does not describe the creation of anything, and Section 19.22 says precisely what it does not establish.

19.1 What this chapter does

Why a separate theorem for the Kohn-Sham states. A theorem about a field equation does not automatically carry over to an approximation of it. Three things are new at the Kohn-Sham level. First, the densities that make the mean field are not the classical bilinears of the field but expectation values in the quantised theory, computed with the indefinite matrix B of Chapter 10; the chirality matrix Γ reverses the sign of B, and that sign decides which partner the theorem finds. Second, the Kohn-Sham problem has boundary conditions: the ASSUMED Z2 mirror at the brane $y = 0$ and a chosen condition at the tip $y = -L$; a map of solutions must map the boundary conditions too. Third, a Kohn-Sham state is self-consistent: the mean field is made by the occupied orbitals, so the map must carry the whole loop from densities to potentials to orbitals and back. T3 handles all three, and the answer is the pairing $(m, \lambda, \theta) \to (-m, +\lambda, \pi - \theta)$ at EQUAL energies, not the T1 pairing $(m, \lambda) \to (-m, -\lambda)$ at opposite energies.

What is done, in order.

- The words of the chapter (Section 19.2).
- The Kohn-Sham problem of one orbital: the slice, the hidden coordinate, the eight blocks, the block Hamiltonian, the boundary conditions (Section 19.3); and of the whole gas: the densities, the mean field, the occupations, the energies and the energy-momentum profiles (Section 19.4).
- Why the chirality Γ acts on the blocks as the Pauli matrix σ_2, and what it does to the Krein sign (Section 19.5).
- The proof of T3 in four steps: the orbitals (Section 19.6), the boundary conditions (Section 19.7), the densities and the mean field, with the reason why the coupling must keep its sign (Section 19.8), and self-consistency with the equality of every energy and every energy-momentum profile (Section 19.9).
- The theorem with its hypotheses and its verification records, and its completion of 2026-10-08: the gaps that an adversarial verification found and closed, statement S6 (the 16-component expectation rule: every component of the energy-momentum tensor and of the current is unchanged), and the two numerical demonstrations of the record (Section 19.10); why T3 is not T1, and the mirror copy inside the Z2 orbifold (Section 19.11).
- The exact levels at zero 3-momentum, and the three things that go wrong when the tip is not transformed: the zero modes move to the tip, a level appears inside the mass gap, and the brane band gets a seven times steeper slope (Section 19.12).
- Notebook 19b, the proof by hand (Sections 19.13 to 19.16).
- The four universes A, B, C, D of the Rust runs and Notebook 19a (Sections 19.17 to 19.21).
- What T3 says about pairs of universes and what it does not (Section 19.22), the closing summary (Section 19.23) and exercises with complete answers (Section 19.24).

The two notebooks.

notebook	what it computes	Rust	PASS lines	figures
19b	the gammas and the block map; every step of the proof of T3 in exact sympy algebra; the exact zero-momentum spectra of A, B and the control C; shooting; the brane-band slopes	no	19	6
19a	the gammas and the block map; 62 runs of the Rust Kohn-Sham solver for A, B, C, D with 8, 136 and 688 quanta, along the history, for five couplings and three temperatures; comparison with the record's numerical demonstration of T3	yes	37	10

Notebook 19b is placed first, because it follows the proof step by step; Notebook 19a then shows the theorem in the full self-consistent computation. Each notebook is complete in itself; you may run them in either order.

The status of every statement. Every statement of this chapter carries one of the labels of Chapter 0.

- PROVED: theorem T3 itself, with every step derived line by line in Sections 19.5 to 19.12. In the Revision record T3 is proved by WolframScript (`Revision/pairing/kohn_sham/reports/wolfram-t3.json`, 10 of 10 checks PASS) and by sympy (`Revision/pairing/kohn_sham/reports/python-t3.json`, 13 of 13 checks PASS); the theorem record with its hypotheses, statement, proof and limits is `Revision/pairing/kohn_sham/t3-theory.json`. An adversarial verification (2026-10-08) found no error in the statement or the proof, but gaps in its verification; the **completion** record `Revision/pairing/kohn_sham/t3-completion.json` closes the three exact ones (`Revision/pairing/kohn_sham/reports/wolfram-t3-completion.json`, 3 of 3 checks PASS; `Revision/pairing/kohn_sham/reports/python-t3-completion.json`, 7 of 7): the Kohn-Sham coefficients read from the record in both engines (Section 19.8), the

filling convention carried onto the partner (Section 19.9), and statement S6, the 16-component expectation rule (Section 19.10). The facts about the blocks come from the Kohn-Sham theory reports `Revision/kohn_sham/reports/ks-theory-python.json` (in its present state 58 of 58 checks PASS) and `Revision/kohn_sham/reports/ks-theory-wolfram.json` (46 of 46).

- COMPUTED: every number of the Rust solver and of the notebooks, with its measured difference. The record's two numerical demonstrations of T3 (Section 19.10): `Revision/pairing/kohn_sham/reports/t3-rust-demo.json` (the Rust solver on all 210 states of the canonical matrix, 7 of 7 checks PASS) and `Revision/pairing/kohn_sham/reports/t3-reference-demo.json` (the independent reference solver on 18 states, 7 of 7). The solver's own T3 self-test is the check t3_block_map_solver_selftest of `Revision/kohn_sham/reports/ks-rust-solver.json` (in its present state 42 of 42 checks PASS). All of them are demonstrations, not part of the proof, and the Rust agreement is at the rounding level by construction (Section 19.17).

- ASSUMED: the good sector (no dependence on the extra times); the Z2 mirror construction at the brane $y = 0$; the cut of the hidden direction at the tip $y = -L$ with $L = 3$ and the chosen tip condition (T3 needs its transformed form); the mean field of Hartree plus the exact exchange of the uniform gas, without correlation. CONVENTION: which levels count as particles (the filling convention, Section 19.4); its justification is OPEN.

- PRESCRIBED BACKGROUND: the history $a_4 = AHx_4$ with $A = 1$, on the background $b_4 = 1$ of the author's metric of 2026-10-14 (Chapter 14); the Kohn-Sham states are not an admissible source of the a_4 field equations (`Revision/field_equations_a4/reports/ks-source-conditions.json`, with the checks `ks_profiles_violate_algebraic_condition` and `ks_history_is_a_prescribed_background`).

- OPEN: the time-dependent (non-adiabatic) Kohn-Sham problem; the justification of the filling convention.

- HYPOTHESIS: the author's statement that universes are created in pairs is a HYPOTHESIS. This chapter neither uses it nor proves it.

Notation and units. The author's coordinates are $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates with the scale factor $e^{a_4} \sin^{1/6} z$; x_4 is the time; x_5, x_6, x_7 are the three extra times, which are time-like and deflate exponentially with the scale factor $e^{-a_4} \sin^{1/6} z$ while a_4 grows; x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$ and the author's constant $H > 0$. The frame metric is $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in this order, and $\gamma^{(x_a)}$ is the author's real 16×16 gamma matrix of the direction x_a (Chapters 4 and 5), with $\gamma^{(x_a)}\gamma^{(x_b)} + \gamma^{(x_b)}\gamma^{(x_a)} = 2\eta_{ab}$ times the unit matrix. The **universe of mass** $+M$ of the title is the Kohn-Sham problem with the bare mass $m = +M$, the **universe of mass** $-M$ the one with $m = -M$; in all numbers $M = 1$ and $H = 1$, so energies, momenta and temperatures are in units of $|m|$, lengths in units of $1/H$. A state of the record is named like N136_lamp1_a10: $N = 136$ quanta, the coupling $+\lambda_1$ (the tags lam0, lamp1, lamm1, lamp2, lamm2 stand for $\lambda = 0, +\lambda_1, -\lambda_1, +\lambda_2, -\lambda_2$), the slice $a_{4,0} = 1.0$ (the two digits are ten times the slice). The two couplings are fixed per particle number in `Revision/kohn_sham/results/parameters.json` so that the first-order mean field stays below 0.1 and 0.3 along the whole history: for $N = 8$, $\lambda_1 = 0.01946$ and $\lambda_2 = 0.05838$; for $N = 136$, $\lambda_1 = 0.0009298$ and $\lambda_2 = 0.002789$; for $N = 688$, $\lambda_1 = 0.0001846$ and $\lambda_2 = 0.0005538$.

19.2 The words of this chapter

- **Kohn-Sham state**: an approximate state of N identical quanta built from one-quantum wave functions, the **orbitals**, each of which solves a one-quantum equation in a common **mean field**; the mean field is made from the densities of the occupied orbitals.
- **Self-consistent**: the orbitals reproduce the mean field that made them. A computer finds such a state by repeating "densities, then mean field, then orbitals, then densities" until nothing changes.
- **Slice**: one instant of the history $a_4 = AHx_4$; $a_{4,0}$ is the value of a_4 there. An **instantaneous** (adiabatic) Kohn-Sham state is the self-consistent state of the Kohn-Sham problem written at that one instant, with $a_4 = a_{4,0}$; along the history the extra times keep deflating, and the states are computed slice after slice.
- **Hidden coordinate** y: $y = \ln(\sin z)/(6H)$; the **brane** is $y = 0$ ($z = \pi/2$), the **tip** is the cut $y = -L$ with $L = 3$.
- **Block**: one of the eight pairs of spinor components into which the 16 components split exactly; labelled (j, s_2, s_3) with three signs. The **block type** is $j = \pm 1$. An orbital of a block is a pair of functions $\chi(y) = (\chi_1(y), \chi_2(y))$.
- **Level** ε: an allowed energy of one orbital; **occupation** f between 0 and 1: how much of a quantum sits in that orbital; **degeneracy** g: how many orbitals share one level.
- **Brane parity**: even means $\chi_2(0) = 0$, odd means $\chi_1(0) = 0$; both kinds of orbitals belong to every state. **Tip angle** θ: the angle of the tip condition $(1 - Q(\theta))\chi(-L) = 0$.
- **Zero mode**: an orbital with $\varepsilon = 0$ at zero 3-momentum. **Brane band**: the levels that grow out of the zero modes when the 3-momentum is switched on. **Mass gap**: the energies $-|M| < \varepsilon < |M|$, in which a free quantum of mass M in an unbounded space has no level.
- **Image**: the orbital, state or problem obtained by applying the map of T3. **Partner**: the problem whose solutions are exactly the images. A map is an **involution** when applying it twice gives back the start.
- **Krein sign** of a block: the number ± 1 by which the matrix B acts in the block.
- **Negative control**: a computation that must FAIL to agree; it shows that an agreement is not automatic. In this chapter the **control** C is the universe of mass $-M$ with the UNtransformed tip, and the **wrong partner** D the one with the reversed coupling.
- **Characteristic function**: a function of ε whose zeros are exactly the levels.
- **Hellmann-Feynman rule**: a small change δh of a Hamiltonian moves a non-degenerate level by $\langle\chi|\delta h|\chi\rangle/\langle\chi|\chi\rangle$ to first order.

19.3 The Kohn-Sham problem of one orbital

This section collects, from Chapter 14 and the record `Revision/kohn_sham/ks-theory.json`, every formula of the problem for one orbital that the proof will use. Each is PROVED in the Kohn-Sham theory reports; the check names are given.

The slice and the hidden coordinate. The Kohn-Sham states are computed at the slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$ of the history $a_4 = AHx_4$, $A = 1$ (`Revision/kohn_sham/results/parameters.json`). In the hidden coordinate y the author's metric is

$$ds^2 = e^{2Hy}\left[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 + dx_7^2)\right] - dx_4^2 + dy^2,$$

the volume factor is $\sqrt{|g|} = e^{6Hy}$ (the inflation e^{3a_4} of 3-space and the deflation e^{-3a_4} of the extra times cancel). This is the author's metric on the background $b_4 = 1$, on which the Kohn-Sham record works (Chapter 14): the metric of 2026-10-14 has a second function $b_4(x_4)$ that multiplies the extra-time components (Chapter 3), and T3 and all its records are statements on this background. A 3-momentum k enters every equation through the **momentum weight**

$$\kappa(y) = e^{-Hy - a_{4,0}}.$$

(Checks geometry_hidden_coordinate, geometry_sqrt_det and geometry_warped_form.) The slice enters the problem only through κ. The proof below never changes κ, so it works at every slice.

The orbital. In the good sector (no dependence on the extra times) an orbital of 3-momentum $\mathbf{k} = (k, 0, 0)$ along x_1 is

$$\Psi_{16} = \frac{e^{-i\varepsilon x_4}\, e^{ikx_1}\, e^{-3Hy}}{\sqrt{\ell^3 v_t}}\, V_\beta\, \chi(y), \qquad \int_{-L}^{0} \chi^\dagger \chi \, dy = 1,$$

where V_β are the two columns of the block β in the unitary 16×16 matrix V of the record, ℓ is the size of the 3-torus of 3-space and v_t the coordinate volume of the extra times. A rotation of 3-space does not change the levels, so turning $\mathbf{k}$ along x_1 loses nothing (check rotation_invariance). The allowed momenta are Δk times triples of whole numbers, with $\Delta k = 0.25$; a **shell** is the set of momenta with the same $|\mathbf{k}|^2 = \Delta k^2 n_2$.

The eight blocks. The three matrices $J = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_4)}$, $K_1 = \gamma^{(x_2)}\gamma^{(x_3)}$ and $K_2 = \gamma^{(x_5)}\gamma^{(x_6)}$ commute with each other and with the Hamiltonian; their eigenvalues $j = \pm 1$, is_2 and is_3 ($s_2, s_3 = \pm 1$) label eight blocks of two components each (checks blocks_commuting_set, blocks_projectors, blocks_basis_unitary; Section 14.5). In the basis V_β of the block (j, s_2, s_3) the matrices that the proof needs act as 2×2 matrices or numbers (check blocks_forms):

matrix	**in the block** (j, s_2, s_3)
$\gamma^{(x_0)}$	σ_3
$\gamma^{(x_0)}\gamma^{(x_4)}$	$j\sigma_1$
$B = -iC\gamma^{(x_4)}$	the number js_2
C	$s_2\sigma_2$
BC	$j\sigma_2$
$B\gamma^{(x_0)}$	$js_2\sigma_3$

Here $\sigma_1, \sigma_2, \sigma_3$ are the Pauli matrices

$$\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Each squares to the unit matrix **1**, two different ones anticommute ($\sigma_a\sigma_b = -\sigma_b\sigma_a$ for $a \neq b$), each is Hermitian ($\sigma_a^\dagger = \sigma_a$, where † means: transpose and replace every entry by its complex conjugate), and their products are $\sigma_1\sigma_2 = i\sigma_3$, $\sigma_2\sigma_3 = i\sigma_1$, $\sigma_3\sigma_1 = i\sigma_2$, with the opposite sign in the reversed order.

The block Hamiltonian. In a block of type j the orbital solves $h_j\chi = \varepsilon\chi$ with

$$h_j(M, v) = j\left[-i\sigma_1\frac{d}{dy} + M(y)\,\sigma_2 + \kappa(y)\,k\,\sigma_3\right] + v(y),$$

where $M(y) = M_{\text{eff}}(y)$ is the effective mass and $v(y) = v_v(y)$ the potential of the mean field (Section 19.4); the Hamiltonian depends on j but not on s_2, s_3, so there are two types of blocks, four blocks of each type (check block_hamiltonian). We write $h_j(M, v)$ when we want to show which mass and potential it contains. Multiplying $h_j\chi = \varepsilon\chi$ from the left by $ij\sigma_1$ gives the first-order form (check block_ode_equivalent; Section 14.5)

$$\frac{d\chi}{dy} = N\chi, \qquad N = M\sigma_3 - \kappa k\,\sigma_2 + ij(\varepsilon - v)\,\sigma_1,$$

and with $\chi = (a, ib)$, a and b real, the **real form**

$$\frac{da}{dy} = Ma - \big(\kappa k + j(\varepsilon - v)\big)\,b, \qquad \frac{db}{dy} = \big(j(\varepsilon - v) - \kappa k\big)\,a - Mb.$$

The boundary conditions. At the brane the record ASSUMES the Z2 mirror (orbifold) construction: the field on the mirror side $y > 0$ is the reflected field, $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$, and continuity at $y = 0$ gives $(1 \mp \gamma^{(x_0)})\chi(0) = 0$, that is, in every block,

$$\text{even parity: } \chi_2(0) = 0 \quad (b(0) = 0), \qquad \text{odd parity: } \chi_1(0) = 0 \quad (a(0) = 0).$$

Both parities are solved and filled together; they are the two sectors of the doubled system "universe plus mirror image" (check bc_brane_parity_conditions). At the tip the record CHOOSES a regular condition from the family

$$(1 - Q(\theta))\chi(-L) = 0, \qquad Q(\theta) = \cos\theta\,\sigma_3 + \sin\theta\,\sigma_2,$$

with the canonical angle $\theta = 0$: $(1 - \sigma_3)\chi(-L) = 0$, that is $\chi_2(-L) = 0$ ($b(-L) = 0$). For $\theta = \pi$, $Q(\pi) = -\sigma_3$ and the condition is $(1 + \sigma_3)\chi(-L) = 0$, that is $\chi_1(-L) = 0$ ($a(-L) = 0$) (check bc_tip_family).

Why these conditions make sense: the current. For two pairs of functions ϕ, χ the record proves (check bc_self_adjoint_boundary_term)

$$\phi^\dagger(h_j\chi) - (h_j\phi)^\dagger\chi = \frac{d}{dy}\Big[-ij\,\phi^\dagger\sigma_1\chi\Big].$$

Integrated from $-L$ to 0, the left side becomes the difference between "h_j acting to the right" and "h_j acting to the left", and the right side the value of the bracket at $y = 0$ minus its value at $y = -L$.

If both boundary conditions make the **current** $\phi^\dagger\sigma_1\chi$ vanish at the ends, the two integrals are equal: h_j is self-adjoint, its levels are real and its orbitals of different levels are orthogonal. Both brane conditions and every tip condition of the family do this; Exercise 19.4 proves it for the tip.

19.4 The Kohn-Sham problem of the whole gas

The expectation-value rule. The quantised field (Chapter 10) gives every density as an expectation value. For the quasi-free states of the Kohn-Sham model the record uses the rule

$$\langle\Psi^\dagger X\Psi\rangle = \mathrm{Tr}(X\rho), \qquad \rho = \sum_{\text{occupied}} f\,u\,u^\dagger B,$$

where X is a 16×16 matrix, u runs over the occupied 16-component orbitals, f is the occupation and $B=-iC\gamma^{(x_4)}$ is the Krein matrix ($B^2=1$, eigenvalues $+1$ eight times and -1 eight times). For one orbital, line by line:

$$\mathrm{Tr}(X\,u\,u^\dagger B) = \mathrm{Tr}((u^\dagger B)(Xu)) = u^\dagger BXu.$$

Rule: the trace of a product does not change when the factors are moved around in a circle, $\mathrm{Tr}(PR)=\mathrm{Tr}(RP)$, here with $P=Xu$ (a column) and $R=u^\dagger B$ (a row); a row times a column is a single number, its own trace. (Check gas_densities.) The three densities that matter are the **number density** (the U(1) charge density, whose integral counts the quanta), the **scalar density** and a third density Q:

$$n=\langle\Psi^\dagger B\Psi\rangle=\sum f\,u^\dagger BBu=\sum f\,u^\dagger u, \qquad S=\langle\bar\Psi\Psi\rangle=\langle\Psi^\dagger C\Psi\rangle=\sum f\,u^\dagger BCu,$$

$$Q=\langle\Psi^\dagger B\gamma^{(x_0)}\Psi\rangle=\sum f\,u^\dagger BB\gamma^{(x_0)}u=\sum f\,u^\dagger\gamma^{(x_0)}u.$$

Rule: $X=B$, C, $B\gamma^{(x_0)}$ in the line above, and $BB=B^2=1$.

The densities of one orbital. Put in $u=\Psi_{16}$ of Section 19.3. The phases $e^{-i\varepsilon x_4}e^{ikx_1}$ cancel against their conjugates, and e^{-3Hy} appears twice, so

$$u^\dagger Xu=\frac{e^{-6Hy}}{\ell^3 v_t}\,\chi^\dagger(V_\beta^\dagger XV_\beta)\chi = P\,\chi^\dagger(V_\beta^\dagger XV_\beta)\chi, \qquad P=\frac{e^{-6Hy}}{\ell^3v_t}.$$

The middle matrix is the block form of X (the table of Section 19.3). With $V_\beta^\dagger V_\beta=\mathbf{1}$, $BC\to j\sigma_2$ and $\gamma^{(x_0)}\to\sigma_3$:

$$n_o=P\,\chi^\dagger\chi, \qquad s_o=P\,\chi^\dagger j\sigma_2\chi, \qquad q_o=P\,\chi^\dagger\sigma_3\chi.$$

The record adds two more (`ks-theory.json`, densities.perOrbital): the density of the 3-momentum current along $\mathbf{k}$, $t_o=P\,\chi^\dagger j\sigma_3\chi=j\,q_o$, and the current along y, $c_o=P\,\chi^\dagger j\sigma_1\chi$. In the real form $\chi=(a,ib)$ they are simple: $\sigma_2\chi=(-i\cdot ib,\ ia)=(b,ia)$ and $\chi^\dagger=(a,-ib)$, so

$$\chi^\dagger\chi = a^2 + b^2, \qquad \chi^\dagger\sigma_2\chi = ab + (-ib)(ia) = 2ab,$$

$$\chi^\dagger\sigma_3\chi = a^2 - b^2, \qquad \chi^\dagger\sigma_1\chi = a(ib) + (-ib)a = 0.$$

Rule: multiply out, with $i \cdot i = -1$. So $c_o = 0$ for every orbital of real form: no current flows along the hidden direction.

The totals. Each level ε of the block type j, the brane parity p and the momentum shell n_2 has the degeneracy $g = 4r_3(n_2)$ (four blocks (s_2, s_3) of the type j times the number $r_3(n_2)$ of lattice momenta in the shell). Then

$$n(y) = \sum_{\text{levels}} w\, g\, f\, n_o(y), \qquad S(y) = \sum_{\text{levels}} w\, g\, f\, s_o(y),$$

and the same for $t(y)$ and $Q(y)$, with $w = w_{Z2} = \frac{1}{2}$: every orbital is normalised on the patch $-L \le y \le 0$, which is half of the doubled interval of the Z2 construction. The particle number is $N = \sum gf$ over all levels of both parities; it counts the quanta of the doubled system, $N/2$ per copy (`ks-theory.json`, densities.total; the conventions of `Revision/kohn_sham/results/parameters.json`).

The mean field. The contact interaction is $U = \frac{\lambda}{2}S^2$. For the quasi-free states of the model, Hartree plus the exact exchange of the uniform gas give the **interaction energy density** (check exchange_uniform_gas; Chapter 14)

$$e_{\text{int}}(n, S) = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2,$$

without a correlation term. The two potentials are its derivatives, line by line:

$$M_{\text{eff}} = m + \frac{\partial e_{\text{int}}}{\partial S} = m + \frac{15}{32} \cdot 2\,\lambda S = m + \frac{15}{16}\lambda S, \qquad v_v = \frac{\partial e_{\text{int}}}{\partial n} = -\frac{1}{32} \cdot 2\,\lambda n = -\frac{\lambda n}{16}.$$

Rule: the derivative of cx^2 is $2cx$; the bare mass m is added to the derivative with respect to S because the mass term of the Lagrangian is mS (check ks_potentials). Note for later: e_{int} contains S only as S^2, and $M_{\text{eff}} - m$ is proportional to λS.

The occupations. At the temperature T the occupations are Mermin's (Fermi's) $f = 1/(1 + e^{(\varepsilon-\mu)/T})$, with the chemical potential μ fixed by $\sum gf = N$; at $T = 0$ the lowest particle levels are filled. Which levels count as particle levels is a CONVENTION of the record (`ks-theory.json`, thermodynamics.fillingConvention): the levels of the positive branch of the problem without interaction, together with the zero modes at zero 3-momentum, each followed continuously as λ is switched on; the negative branch is the normal-ordered sea and contributes nothing. Its justification is OPEN.

The energies. With $\text{Vol}_7 = \ell^3 v_t$ the record defines (`ks-theory.json`, thermodynamics)

$$E_{KS} = \sum gf\varepsilon - 2\,\text{Vol}_7 \int_{-L}^{0} e^{6Hy} e_{\text{int}}\, dy, \qquad S_{\text{ent}} = -\sum g[f \ln f + (1-f)\ln(1-f)],$$

$$\Omega = -T\sum g\ln\left(1+e^{-(\varepsilon-\mu)/T}\right) - 2\,\mathrm{Vol}_7\int_{-L}^{0} e^{6Hy}e_{\mathrm{int}}\,dy, \qquad F=\Omega+\mu N = E_{KS}-TS_{\mathrm{ent}}:$$

the Kohn-Sham energy, the entropy, the grand potential and the free energy (Exercise 19.7 derives the last equality).

The energy-momentum profiles. The energy density and the three pressures of the gas, per unit proper volume, are (`ks-theory.json`, emt; check emt_orbital_components)

$$\rho = \sum wgf\,\varepsilon\,n_o - e_{\mathrm{int}}, \qquad p_3 = \sum wgf\,\frac{\kappa|k|}{3}\,t_o + e_{\mathrm{int}}, \qquad p_t = e_{\mathrm{int}},$$

$$p_0 = \sum wgf\Big[(\varepsilon-v_v)\,n_o - M_{\mathrm{eff}}\,s_o - \kappa|k|\,t_o\Big] + e_{\mathrm{int}}:$$

ρ is the energy density, p_3 the pressure of 3-space, p_t the pressure of the deflating extra times and p_0 the pressure of the hidden direction. Every self-consistent state obeys the conservation law along y, $p_0' + 6Hp_0 = 3H(p_3+p_t)$ (check emt_y_conservation_selfconsistent).

The loop. A self-consistent state is found by repeating: from $n(y)$ and $S(y)$ build $M_{\mathrm{eff}}(y)$ and $v_v(y)$; solve $h_j\chi = \varepsilon\chi$ for both block types, every momentum shell and both parities with the boundary conditions; fill the levels; compute new $n(y)$ and $S(y)$. A state that reproduces itself is a **fixed point** of this loop. Chapter 15 describes how the Rust solver does it.

19.5 The chirality Γ on the blocks

Γ and the gammas. The chirality is the product of all eight gammas in the author's order,

$$\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}\gamma^{(x_4)}\gamma^{(x_5)}\gamma^{(x_6)}\gamma^{(x_7)} = \mathrm{diag}(-1,\ldots,-1,+1,\ldots,+1),$$

with -1 eight times and then $+1$ eight times (`Revision/algebra/reports/wolfram-algebra.json`, check Gamma_diag). Moving one gamma $\gamma^{(x_c)}$ from the left of Γ to its right passes the seven other factors, each with a sign -1, and itself, without a sign: $\gamma^{(x_c)}\Gamma = (-1)^7\Gamma\gamma^{(x_c)} = -\Gamma\gamma^{(x_c)}$. So Γ **anticommutes with every gamma**, hence with every product of an odd number of gammas, and **commutes with every product of an even number**. It is real and diagonal, so $\Gamma^\dagger = \Gamma$ and $\Gamma^2 = 1$.

Γ moves every block to its partner block. $J = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_4)}$ has three factors, K_1 and K_2 have two. Let v be a column of the block (j, s_2, s_3), that is $Jv = jv$, $K_1v = is_2v$, $K_2v = is_3v$. Then, line by line:

$$J(\Gamma v) = -\Gamma Jv = -j\,(\Gamma v), \qquad K_1(\Gamma v) = \Gamma K_1 v = is_2\,(\Gamma v), \qquad K_2(\Gamma v) = is_3\,(\Gamma v).$$

Rule: Γ anticommutes with the odd product J and commutes with the even products K_1, K_2; then use the eigenvalue equations of v. So Γv lies in the block $(-j, s_2, s_3)$: the **partner block**, with the opposite type and the same s_2, s_3 (check blocks_relation_to_Gamma).

Γ acts as $\pm\sigma 2$ between the partner blocks. Let V_b be the two basis columns of the block $b = (j, s_2, s_3)$ and V_p those of the partner block $p = (-j, s_2, s_3)$. Because ΓV_b lies in the block p, it is a combination of the columns V_p: $\Gamma V_b = V_p G$ for some 2×2 matrix G. The block forms say $\gamma^{(x_0)} V_b = V_b \sigma_3$ and $\gamma^{(x_0)} V_p = V_p \sigma_3$ (in both blocks $\gamma^{(x_0)}$ is σ_3), and $\gamma^{(x_0)}\gamma^{(x_4)} V_b = V_b(j\sigma_1)$, $\gamma^{(x_0)}\gamma^{(x_4)} V_p = V_p(-j\sigma_1)$ (the type of p is $-j$). Now line by line:

$$\Gamma\gamma^{(x_0)} V_b = \Gamma V_b \sigma_3 = V_p G \sigma_3, \qquad -\gamma^{(x_0)}\Gamma V_b = -\gamma^{(x_0)} V_p G = -V_p \sigma_3 G.$$

Rule: the block forms and the definition of G. The two left sides are equal, because Γ anticommutes with $\gamma^{(x_0)}$; the columns of V_p are independent, so

$$G\sigma_3 = -\sigma_3 G.$$

The same with the even product $\gamma^{(x_0)}\gamma^{(x_4)}$, which commutes with Γ:

$$\Gamma(\gamma^{(x_0)}\gamma^{(x_4)}) V_b = V_p G\,(j\sigma_1), \qquad (\gamma^{(x_0)}\gamma^{(x_4)})\Gamma V_b = V_p(-j\sigma_1)\,G, \qquad \text{so} \qquad G\sigma_1 = -\sigma_1 G.$$

Rule: equate the two lines and divide by $j = \pm 1$. Every 2×2 matrix can be written as $G = g_0\mathbf{1} + g_1\sigma_1 + g_2\sigma_2 + g_3\sigma_3$ with four numbers $g_0, \ldots, g_3$ (four numbers for four entries). Multiply $G\sigma_3 = -\sigma_3 G$ from the left by σ_3: $\sigma_3 G \sigma_3 = -G$. Line by line:

$$\sigma_3 G \sigma_3 = g_0\mathbf{1} - g_1\sigma_1 - g_2\sigma_2 + g_3\sigma_3 = -g_0\mathbf{1} - g_1\sigma_1 - g_2\sigma_2 - g_3\sigma_3.$$

Rule: $\sigma_3\sigma_3\sigma_3 = \sigma_3$, and $\sigma_3\sigma_a\sigma_3 = -\sigma_a\sigma_3\sigma_3 = -\sigma_a$ for $a = 1, 2$; the right side is $-G$. Comparing the coefficients gives $g_0 = -g_0$ and $g_3 = -g_3$, so $g_0 = g_3 = 0$. The same steps with σ_1 ($\sigma_1 G \sigma_1 = g_0 + g_1\sigma_1 - g_2\sigma_2 - g_3\sigma_3 = -G$) give $g_0 = g_1 = 0$. Hence

$$G = \tau\,\sigma_2, \qquad |\tau| = 1,$$

where $|\tau| = 1$ because Γ and V are unitary, so G is unitary too. The record computes the number τ for all eight blocks: $\tau = s_2$, that is $\Gamma V_{(j,s_2,s_3)} = V_{(-j,s_2,s_3)}\,(s_2\sigma_2)$ (checks blocks_relation_to_Gamma of `ks-theory-python.json`, T3_Gamma_is_the_block_map of `wolfram-t3.json`, T3.Gamma_is_the_block_map of `python-t3.json`). Notebooks 19a and 19b compute it again (Notebook 19b, figure 2, shows where the entries sit).

The map of T3. For a 16-component orbital Ψ_{16} built from χ in the block (j, s_2, s_3), the orbital $\Gamma\Psi_{16}$ is built from $s_2\sigma_2\chi$ in the block $(-j, s_2, s_3)$, at the same 3-momentum and the same point. The constant sign s_2 changes no density (every density contains the orbital twice, and $s_2^2 = 1$), so we drop it. On the reduced problem the chirality is the map

$$(\chi,\ j) \longrightarrow (\sigma_2\chi,\ -j), \qquad \text{in the real form} \qquad (a, b, j) \longrightarrow (b, a, -j),$$

because $\sigma_2(a, ib) = (b, ia)$: the map exchanges a and b and reverses the block type.

Γ **reverses the Krein sign.** In the block (j, s_2, s_3) the matrix B is the number js_2, its Krein sign; in the partner block it is $-js_2$. In 16 components: $B = -iC\gamma^{(x_4)}$ is $-i$ times a product of five gammas (four in C, one more), an odd product, so $\Gamma B = -B\Gamma$ and

$$\Gamma B \Gamma^\dagger = \Gamma B \Gamma = -B\Gamma\Gamma = -B$$

(`Revision/pairing/reports/python-pairing.json`, check Q.image_krein_metric). This one sign is why T3 differs from T1 (Section 19.11).

19.6 Step 1 of the proof: an orbital and its image

The block Hamiltonian under $\sigma 2$. We conjugate $h_j(M, v)$ with σ_2, that is, we compute $\sigma_2 h_j(M, v)\sigma_2$, term by term. The matrix σ_2 is constant, so it can be moved past the derivative: $\frac{d}{dy}(\sigma_2\chi) = \sigma_2 \frac{d\chi}{dy}$. Line by line:

$$\sigma_2\Big(-i\sigma_1\frac{d}{dy}\Big)\sigma_2 = -i\,\sigma_2\sigma_1\sigma_2\,\frac{d}{dy} = +i\sigma_1\frac{d}{dy}.$$

Rule: $\sigma_2\sigma_1\sigma_2 = -\sigma_1\sigma_2\sigma_2 = -\sigma_1$ (the two anticommute, and $\sigma_2^2 = \mathbf{1}$).

$$\sigma_2(M\sigma_2)\sigma_2 = M\,\sigma_2\sigma_2\sigma_2 = M\sigma_2, \qquad \sigma_2(\kappa k\,\sigma_3)\sigma_2 = \kappa k\,\sigma_2\sigma_3\sigma_2 = -\kappa k\,\sigma_3, \qquad \sigma_2\,v\,\sigma_2 = v.$$

Rule: $\sigma_2^2 = \mathbf{1}$; $\sigma_2\sigma_3\sigma_2 = -\sigma_3\sigma_2\sigma_2 = -\sigma_3$; the numbers M, κk, v (functions of y) commute with every matrix. Adding the four terms, with the factor j in front of the first three:

$$\sigma_2 h_j(M, v)\sigma_2 = j\Big[+i\sigma_1\frac{d}{dy} + M\sigma_2 - \kappa k\,\sigma_3\Big] + v = (-j)\Big[-i\sigma_1\frac{d}{dy} + (-M)\sigma_2 + \kappa k\,\sigma_3\Big] + v = h_{-j}(-M, v).$$

Rule: take the factor -1 out of the bracket and give it to j; then compare with the definition of h_j in Section 19.3. This holds for every function $M(y)$, $\kappa(y)$, $v(y)$ and every k.

The image orbital. Suppose $h_j(M, v)\chi = \varepsilon\chi$. Then, line by line:

$$h_{-j}(-M, v)(\sigma_2\chi) = \sigma_2 h_j(M, v)\sigma_2\,\sigma_2\chi = \sigma_2 h_j(M, v)\chi = \sigma_2\,\varepsilon\chi = \varepsilon\,(\sigma_2\chi).$$

Rule: replace $h_{-j}(-M, v)$ by $\sigma_2 h_j(M, v)\sigma_2$ (the line above); $\sigma_2\sigma_2 = \mathbf{1}$; the eigenvalue equation; a number commutes with σ_2. So $\sigma_2\chi$ **is an orbital of the partner block type $-j$, in the mass $-M$ and the SAME potential v, with the SAME level ε and the same 3-momentum**. Because $\sigma_2^2 = \mathbf{1}$, the argument also runs backwards. Without the change of the mass the identity fails: $h_{-j}(-M, v) - h_{-j}(M, v) = (-j)(-2M)\sigma_2 = 2jM\sigma_2$, which is not zero for $M \neq 0$. Status: PROVED; checks T3_block_hamiltonian_map of `wolfram-t3.json` and T3.block_hamiltonian_map of `python-t3.json` (each with this control), and block_Gamma_map of both Kohn-Sham theory reports.

The same in the first-order form. The solver integrates $d\chi/dy = N\chi$. Line by line, with $N_j(M) = M\sigma_3 - \kappa k\sigma_2 + ij(\varepsilon - v)\sigma_1$:

$$\sigma_2 N_j(M)\sigma_2 = M\sigma_2\sigma_3\sigma_2 - \kappa k\,\sigma_2\sigma_2\sigma_2 + ij(\varepsilon - v)\sigma_2\sigma_1\sigma_2 = -M\sigma_3 - \kappa k\,\sigma_2 - ij(\varepsilon - v)\sigma_1 = N_{-j}(-M).$$

Rule: the three conjugation rules above; then compare with $N_{-j}(-M) = (-M)\sigma_3 - \kappa k\sigma_2 + i(-j)(\varepsilon - v)\sigma_1$. Hence if $d\chi/dy = N_j(M)\chi$, then $\frac{d}{dy}(\sigma_2\chi) = \sigma_2 N_j(M)\sigma_2\,\sigma_2\chi = N_{-j}(-M)\,(\sigma_2\chi)$ (checks T3_ode_map and T3.ode_map).

The same in the real form. In the real form the map is $(a, b, j, M) \to (b, a, -j, -M)$. Check it directly: call the image $A = b$, $\mathcal{B} = a$. The real form with $(-j, -M)$ asks

$$\frac{dA}{dy} = (-M)A - \big(\kappa k + (-j)(\varepsilon - v)\big)\mathcal{B} = -Mb + \big(j(\varepsilon - v) - \kappa k\big)a,$$

and this is exactly the second equation of the original, $db/dy = (j(\varepsilon - v) - \kappa k)a - Mb$. Likewise

$$\frac{d\mathcal{B}}{dy} = \big((-j)(\varepsilon - v) - \kappa k\big)A - (-M)\mathcal{B} = Ma - \big(\kappa k + j(\varepsilon - v)\big)b,$$

which is the first equation of the original, da/dy. Rule: insert $A = b$, $\mathcal{B} = a$ and multiply out the signs. Notebook 19b uses this form for its shooting.

Every slice at once. The slice $a_{4,0}$ and the 3-momentum enter only through κk, which the map does not touch. So everything in this section, and therefore T3, holds at every slice of the deflating history, and for every 3-momentum.

19.7 Step 2 of the proof: the boundary conditions

The tip. Line by line:

$$\begin{aligned}\sigma_2 Q(\theta)\sigma_2 &= \cos\theta\,\sigma_2\sigma_3\sigma_2 + \sin\theta\,\sigma_2\sigma_2\sigma_2 = -\cos\theta\,\sigma_3 + \sin\theta\,\sigma_2 \\ &= \cos(\pi - \theta)\,\sigma_3 + \sin(\pi - \theta)\,\sigma_2 = Q(\pi - \theta).\end{aligned}$$

Rule: the conjugation rules of Section 19.6; then $\cos(\pi - \theta) = -\cos\theta$ and $\sin(\pi - \theta) = \sin\theta$. Therefore

$$\sigma_2(1 - Q(\theta))\chi = \sigma_2\chi - \sigma_2 Q(\theta)\sigma_2\,\sigma_2\chi = \big(1 - Q(\pi - \theta)\big)\,\sigma_2\chi.$$

Rule: insert $\sigma_2\sigma_2 = \mathbf{1}$ between $Q(\theta)$ and χ. Since σ_2 is invertible, the left side vanishes exactly when $(1 - Q(\theta))\chi = 0$. So χ **obeys the tip condition with the angle** θ **exactly when** $\sigma_2\chi$ **obeys the tip condition with the angle** $\pi - \theta$. The canonical $\theta = 0$ ($\chi_2(-L) = 0$) becomes $\theta = \pi$ ($\chi_1(-L) = 0$); in components this is plain, because $\sigma_2\chi = (-i\chi_2,\ i\chi_1)$ has the first component $-i\chi_2$. A control: the solution $(1, 0)$ of the condition $\theta = 0$ has the image $\sigma_2(1, 0) = (0, i)$, and $(1 - Q(0))(0, i) = (1 - \sigma_3)(0, i) = (0, 2i) \neq 0$: the image violates the UNtransformed condition, while

$(1-Q(\pi))(0,i) = (1+\sigma_3)(0,i) = (0,0)$. Status: PROVED; checks T3_tip_condition_map and T3.tip_condition_map.

The brane. Line by line:

$$\sigma_2(1-\sigma_3)\sigma_2 = \sigma_2\sigma_2 - \sigma_2\sigma_3\sigma_2 = 1+\sigma_3.$$

So the even condition $(1-\sigma_3)\chi(0)=0$, that is $\chi_2(0)=0$, holds exactly when $(1+\sigma_3)\sigma_2\chi(0)=0$, the odd condition for the image. **The map exchanges the two brane parities.** Both parities belong to every Kohn-Sham problem of the record (they are the two sectors of the ASSUMED Z2 construction, solved and filled together), so the set of boundary problems is mapped onto itself and only the names "even" and "odd" are exchanged. The tip is the only boundary condition that really changes.

The current and the norm. For the image orbitals of type $-j$, line by line:

$$(\sigma_2\phi)^\dagger(\sigma_2\chi) = \phi^\dagger\sigma_2\sigma_2\chi = \phi^\dagger\chi, \qquad -i(-j)\,(\sigma_2\phi)^\dagger\sigma_1(\sigma_2\chi) = ij\,\phi^\dagger\sigma_2\sigma_1\sigma_2\chi = -ij\,\phi^\dagger\sigma_1\chi.$$

Rule: $(\sigma_2\phi)^\dagger = \phi^\dagger\sigma_2^\dagger = \phi^\dagger\sigma_2$; $\sigma_2\sigma_1\sigma_2 = -\sigma_1$. The norm and the current of Section 19.3 are unchanged. So normalised orbitals go to normalised orbitals, orthogonal ones to orthogonal ones, and the image problem is self-adjoint with real levels, like the original. Status: PROVED; checks T3_brane_parities_exchanged and T3.brane_parities_exchanged.

What Steps 1 and 2 give together. Write $\mathcal{P}_j(M,v,\theta,p)$ for the boundary-value problem of the block type j with the mass $M(y)$, the potential $v(y)$, the tip angle θ and the brane parity p. Then χ is a normalised eigen-orbital of $\mathcal{P}_j(M,v,\theta,\text{even})$ with the level ε exactly when $\sigma_2\chi$ is a normalised eigen-orbital of $\mathcal{P}_{-j}(-M,v,\pi-\theta,\text{odd})$ with the same level, and the same with "even" and "odd" exchanged. The correspondence is one to one and onto, because σ_2 undoes itself. So **the two problems have the same levels, level by level, with the same multiplicities**. The degeneracies agree too: the map sends the four blocks of the type j one to one onto the four blocks of the type $-j$, at the same momenta.

19.8 Step 3 of the proof: the densities and the mean field

The densities of one orbital. Under $(\chi, j) \to (\sigma_2\chi, -j)$ the five densities of Section 19.4 become, line by line:

$$n_o \to P(\sigma_2\chi)^\dagger(\sigma_2\chi) = P\chi^\dagger\sigma_2\sigma_2\chi = P\chi^\dagger\chi = +n_o,$$

$$s_o \to P(\sigma_2\chi)^\dagger(-j)\sigma_2(\sigma_2\chi) = -jP\chi^\dagger\sigma_2\sigma_2\sigma_2\chi = -jP\chi^\dagger\sigma_2\chi = -s_o,$$

$$t_o \to -jP\chi^\dagger\sigma_2\sigma_3\sigma_2\chi = -jP\chi^\dagger(-\sigma_3)\chi = +t_o,$$

$$q_o \to P\chi^\dagger\sigma_2\sigma_3\sigma_2\chi = -q_o, \qquad c_o \to -jP\chi^\dagger\sigma_2\sigma_1\sigma_2\chi = +c_o.$$

Rule: $(\sigma_2\chi)^\dagger = \chi^\dagger\sigma_2$; $\sigma_2^2 = \mathbf{1}$; $\sigma_2\sigma_3\sigma_2 = -\sigma_3$, $\sigma_2\sigma_1\sigma_2 = -\sigma_1$; the factor P is the same because the map acts at the same point. In the real form it is even simpler: $(a, b, j) \to (b, a, -j)$ turns $a^2 + b^2$ into $b^2 + a^2$ (no change), $2jab$ into $2(-j)ba$ (sign change) and $a^2 - b^2$ into $b^2 - a^2$ (sign change). Status: PROVED; checks T3_orbital_densities and T3.orbital_densities (for arbitrary complex χ and both j).

The densities of the gas. If the image state has the same levels, degeneracies and occupations (Step 4 shows that it does), the sums of Section 19.4 give

$$n(y) \to n(y), \qquad t(y) \to t(y), \qquad S(y) \to -S(y), \qquad Q(y) \to -Q(y).$$

Which partner closes the mean field. For the image orbitals to be orbitals of the partner problem, Step 1 demands that the partner's mean field be $(-M_{\text{eff}}, v_v)$: the opposite mass and the same potential. Let the partner have the bare mass m' and the coupling λ'; its mean field, built from the image densities $(n, -S)$, is $m' + \frac{15}{16}\lambda'(-S)$ and $-\lambda' n/16$. The potential decides the coupling, line by line:

$$-\frac{\lambda' n}{16} = -\frac{\lambda n}{16} \quad \Longrightarrow \quad \lambda' = \lambda \qquad (n \neq 0).$$

Rule: multiply by $-16/n$. Then the mass decides the bare mass:

$$m' - \frac{15}{16}\lambda S = -m - \frac{15}{16}\lambda S \quad \Longrightarrow \quad m' = -m.$$

Rule: insert $\lambda' = \lambda$, and add $\frac{15}{16}\lambda S$ to both sides. Check, line by line, that $(m', \lambda') = (-m, +\lambda)$ works:

$$M'_{\text{eff}} = -m + \frac{15}{16}\lambda(-S) = -\Big(m + \frac{15}{16}\lambda S\Big) = -M_{\text{eff}}, \qquad v'_v = -\frac{\lambda n}{16} = v_v,$$

$$e_{\text{int}}(n, -S) = \frac{15}{32}\lambda(-S)^2 - \frac{1}{32}\lambda n^2 = e_{\text{int}}(n, S).$$

Rule: take -1 out of the bracket; $(-S)^2 = S^2$. **The coupling keeps its sign; only the bare mass is reversed.** With the parameter change of T1, $(m', \lambda') = (-m, -\lambda)$, the mean field comes out wrong:

$$-m + \frac{15}{16}(-\lambda)(-S) = -M_{\text{eff}} + \frac{15}{8}\lambda S, \qquad -\frac{(-\lambda)n}{16} = v_v + \frac{\lambda n}{8},$$

both off whenever $\lambda S \neq 0$ or $\lambda n \neq 0$. Status: PROVED; checks T3_mean_field_map and T3.mean_field_map (with the control $(-m, -\lambda)$). The sympy check also verifies that the potentials are the derivatives of e_{int}. Notebook 19b, figure 3, draws these three mass lines; Notebook 19a, figure 9, shows the energies of $(-m, -\lambda)$.

A gap in the verification of this step, and how it was closed. The adversarial verification of 2026-10-08 (`Revision/pairing/kohn_sham/t3-completion.json`, entry adversarial_verification,

Gap 1) found two weaknesses in the Wolfram verifier of T3 as it then stood: it wrote the four coefficients $\frac{15}{16}$, $-\frac{1}{16}$, $\frac{15}{32}$, $-\frac{1}{32}$ into its script instead of reading them from `Revision/kohn_sham/ks-theory.json`, and its clause about v_v compared an expression with itself (`vvf[lam, n] === vvf[lam, n]`), which is true for every v_v whatever. So, for v_v, the statement "two independent verifiers" did not hold at that time: only the sympy check, which reads the coefficients from the record, verified the potential. The algebra above was never in doubt; what was missing was an independent machine check of it. The completion closes the gap with the checks T3C_mean_field_coefficients_from_ks_theory (`Revision/pairing/kohn_sham/reports/wolfram-t3-completion.json`) and T3C.mean_field_coefficients_from_ks_theory (`Revision/pairing/kohn_sham/reports/python-t3-completion.json`): both engines read the potentials from `ks-theory.json`, check that $M_{\text{eff}} - m = \partial e_{\text{int}}/\partial S$ and $v_v = \partial e_{\text{int}}/\partial n$ (the two derivatives of Section 19.4), and repeat the map $M_{\text{eff}}[-m, +\lambda, -S] = -M_{\text{eff}}[m, \lambda, S]$, e_{int} even in S, v_v free of S, with the control $(-m, -\lambda)$, which fails. Afterwards, on the same day, the T3 verifier itself was fixed at the root: it now reads the coefficients from `ks-theory.json`, and its check T3_mean_field_map writes v_v as a function of (m, λ, n, S) and compares its values at the original and at the image arguments, so that a wrong coefficient makes the check fail; it still has 10 of 10 checks PASS (`Revision/pairing/kohn_sham/README.md`; `Revision/pairing/kohn_sham/wolfram/WOLFRAMSCRIPT_PROVENANCE.md`, section 6.5). Status of Step 3 now: PROVED, by two independent verifiers in the T3 records and again by the completion checks in both engines.

19.9 Step 4 of the proof: self-consistency, energies and energy-momentum

Let A be a self-consistent Kohn-Sham state of the problem with (m, λ, θ) at a slice $a_{4,0}$, with N quanta at the temperature T: its orbitals χ with the levels ε and the occupations f give the densities n, S, these give M_{eff}, v_v, and the eigen-orbitals of this mean field with both parities and the tip angle θ are again the orbitals χ. Define the state B: the orbitals $\sigma_2\chi$ in the partner blocks, with the same occupations. We show that B is a self-consistent state of the problem with $(-m, +\lambda, \pi - \theta)$, at the same slice, with the same N and T.

- (i) The densities of B are $(n, -S)$ (Step 3).
- (ii) The partner problem builds from them the mean field $(-M_{\text{eff}}, v_v)$ (Step 3).
- (iii) In this mean field, with the tip angle $\pi - \theta$ and both parities, the eigen-orbitals of the partner problem are exactly the orbitals $\sigma_2\chi$, with the same levels and degeneracies (Steps 1 and 2).
- (iv) The occupations. A Mermin occupation depends only on ε, μ and T. The particle number $N(\mu) = \sum g f(\varepsilon; \mu, T)$ is the same function of μ for both problems, because the levels and degeneracies are the same, so the same μ solves $N(\mu) = N$. Which levels are particle levels is decided by following the problem without interaction; at $\lambda = 0$ the two free problems ($M = m$ and $M = -m$, $v = 0$) are themselves mapped onto each other by Steps 1 and 2, so every image level carries the label of its original. In particular the zero mode $(e^{my}, 0)$ of A goes to $\sigma_2(e^{my}, 0) = (0, ie^{my})$, a zero mode of B. And because the coupling is the same, the continuous path from $\lambda = 0$ to λ along which the labels are followed is the same path for both problems. Hence the partner problem gives every orbital $\sigma_2\chi$ the occupation f. This item was the second gap of the adversarial verification (`t3-completion.json`, Gap 2): the record's proof used the filling convention (hypothesis H5, defined for the member with $m > 0$ and $\theta = 0$) for

the image with the remark that both block types and both parities belong to each problem, without showing that the image's set of particle levels is the image of the original's set. The argument of this item is what the completion checks: for every coupling λ' on the path from 0 to λ and any $S(y)$, $\sigma_2 h_j(m + \frac{15}{16}\lambda' S)\sigma_2 = h_{-j}(-(m + \frac{15}{16}\lambda' S))$ for both j (Step 1 with the mass $m + \frac{15}{16}\lambda' S$), so the map keeps the levels with $\varepsilon > 0$ at $\lambda = 0$ and commutes with the continuation in λ; the brane zero mode $(e^{My}, 0)$ goes to $(0, ie^{My})$, the brane zero mode of the image in its own boundary conditions (odd parity, tip angle π); with the untransformed tip the image mode violates the tip condition (the control). Checks T3C_filling_convention_mapped and T3C.filling_convention_mapped (statement "H5 for the pair" of `t3-completion.json`). Status: PROVED; the filling convention itself remains a CONVENTION whose justification is OPEN.

- (v) So the densities that the partner problem computes from its own orbitals and occupations are $(n, -S)$, the densities we started from in (ii): B is a fixed point of the partner's loop, a **self-consistent state**.

Applying the map twice gives back A, because $\sigma_2^2 = \mathbf{1}$ and $(-(-m), \lambda, \pi - (\pi - \theta)) = (m, \lambda, \theta)$: the map is an **involution**, and every self-consistent state of the partner problem is the image of a self-consistent state of the original. The self-consistent states of the two problems are in one-to-one correspondence.

Equal energies. Line by line, for the quantities of Section 19.4:

- $\sum g f \varepsilon$ is the same: the same g, f and ε.
- $e_{\text{int}}(n, -S) = e_{\text{int}}(n, S)$ at every y (Step 3), so $2\,\text{Vol}_7 \int e^{6Hy} e_{\text{int}}\, dy$ is the same.
- Hence E_{KS} is the same; S_{ent} is the same (it contains only g and f); Ω is the same (the same levels, μ, T and interaction integral); and $F = \Omega + \mu N$ is the same.

Equal energy-momentum profiles. Term by term in the formulas of Section 19.4:

- $\rho = \sum w g f\, \varepsilon n_o - e_{\text{int}}$: n_o and e_{int} are unchanged, so $\rho(y)$ is unchanged.
- $p_3 = \sum w g f\, \frac{\kappa|k|}{3} t_o + e_{\text{int}}$: t_o is unchanged, and so are κ and $|k|$; $p_3(y)$ is unchanged.
- $p_t = e_{\text{int}}$: unchanged.
- $p_0 = \sum w g f [(\varepsilon - v_v) n_o - M_{\text{eff}} s_o - \kappa|k| t_o] + e_{\text{int}}$: the only term with odd factors is $M_{\text{eff}} s_o$, and it becomes $(-M_{\text{eff}})(-s_o) = M_{\text{eff}} s_o$, a product of two odd factors; $p_0(y)$ is unchanged.

So the four profiles are equal at every point y, while $S(y)$, $Q(y)$ and $M_{\text{eff}}(y)$ change sign and $n(y)$, $t(y)$, $v_v(y)$, $e_{\text{int}}(y)$ do not. The Kohn-Sham gap (the lowest empty level minus the highest occupied level) is a difference of two levels and is the same as well. Status: PROVED; checks `T3_energies_and_emt_profiles_equal` of `wolfram-t3.json` and `T3.energies_and_emt_profiles_equal` of `python-t3.json`; the sympy check takes three general occupied orbitals of both block types with arbitrary complex χ, levels, occupations, degeneracies and momenta, and also verifies the control: with $(-m, -\lambda)$ the energy density differs. These are the four diagonal components of the energy-momentum tensor of the 2×2 reduction; that every other component and the current are equal too is statement S6 of the completion (Section 19.10; the third gap of the adversarial verification).

19.10 Theorem T3: the statement, its hypotheses and its records

Hypotheses (`Revision/pairing/kohn_sham/t3-theory.json`, hypotheses H1 to H6, in words):

- H1, the problem: the instantaneous Kohn-Sham problem of `Revision/kohn_sham/ks-theory.json` at a fixed slice $a_{4,0}$ of the author's metric, in the good sector, with a lattice of 3-momenta and the block Hamiltonians h_j on $-L \le y \le 0$.
- H2, the functional: Hartree plus the exact exchange of the uniform gas, $M_{\text{eff}} = m + \frac{15}{16}\lambda S$, $v_v = -\lambda n/16$, no correlation; the energy-momentum tensor of that record.
- H3, the brane, ASSUMED: the Z2 mirror construction at $y = 0$; both parities are solved and filled together with the weight $w_{Z2} = \frac{1}{2}$.
- H4, the tip, chosen: $(1 - Q(\theta))\chi(-L) = 0$.
- H5, the occupations: Mermin's at fixed N, with the filling convention of the record.
- H6, the same remaining data for both members: the same H, L, $a_{4,0}$, lattice, extra-time volume, N and T; mean-field (quasi-free) states only.

Theorem T3 (the Kohn-Sham level of the pairing). Under H1 to H6, the map $(\chi, j) \to (\sigma_2\chi, -j)$ at the same 3-momentum, which is the chirality Γ of the 16-component orbital (up to the sign s_2 of each block), with the two brane parities exchanged and the tip angle $\theta \to \pi - \theta$, maps every self-consistent instantaneous Kohn-Sham state with (m, λ, θ) onto a self-consistent Kohn-Sham state with $(-m, +\lambda, \pi - \theta)$, and conversely (the map is an involution). The two states have:

- S1: the same levels, orbital by orbital: the orbital of the type j and the parity p goes to the type $-j$ and the other parity, at the same momentum and the same level;
- S2: with the untransformed tip the spectra differ (Section 19.12);
- S3: the densities transform as $n \to n$, $t \to t$, $S \to -S$, $Q \to -Q$, so $M_{\text{eff}} \to -M_{\text{eff}}$ and $v_v \to v_v$; with $(-m, -\lambda)$ instead of $(-m, +\lambda)$ the map fails;
- S4: equal levels with their degeneracies, occupations, chemical potential, particle number, entropy, Kohn-Sham energy E_{KS}, grand potential Ω and free energy F, and equal profiles $\rho(y)$, $p_3(y)$, $p_t(y)$, $p_0(y)$; $S(y)$ and $Q(y)$ change sign;
- S5: inside the ASSUMED Z2 orbifold the mirror copy of a self-consistent state carries $(-m, +\lambda)$ (Section 19.11).

The records. Each step of the proof is a named check of two independent verifiers that share no code (with one exception at the time of the adversarial verification, the v_v clause of Step 3, Section 19.8, closed by the completion below). Every check below has the verdict PASS.

step	what it shows	wolfram-t3.json	python-t3.json
1	$\sigma_2 h_j(M)\sigma_2 = h_{-j}(-M)$, also for N	T3_block_hamiltonian_map, T3_ode_map	T3.block_hamiltonian_map, T3.ode_map
2	tip $\theta \to \pi - \theta$; parities exchanged; current and norm kept	T3_tip_condition_map, T3_brane_parities_exchanged	T3.tip_condition_map, T3.brane_parities_exchanged
3	n, t even, S, Q odd; the mean field closes for $(-m, +\lambda)$ only	T3_orbital_densities, T3_mean_field_map	T3.orbital_densities, T3.mean_field_map
4	equal energies and profiles	T3_energies_and_emt_profiles_equal	T3.energies_and_emt_profiles_equal
confirmations	exact $k = 0$ spectra; Γ is the block map; the mirror copy	T3_exact_k0_spectra, T3_Gamma_is_the_block_map, T3_z2_mirror_copy_carries_minus_m_plus_lambda	T3.exact_k0_spectra, T3.Gamma_is_the_block_map, T3.z2_mirror_copy_carries_minus_m_plus_lambda
comparison	the sympy side confirms the Wolfram theorem record and its list of limits	none	compare.t3_theory.theorem, compare.t3_theory.not_established
numbers	the Rust solver's self-test (a confirmation, not a proof)	none	T3.rust_selftest_numerical_confirmation

So `wolfram-t3.json` has 10 checks and `python-t3.json` 13 (the ten counterparts, the two comparisons and the numerical confirmation). The record `Revision/docs/PAIR_CREATION_PROOFS.md` (its section 8) collects the same proof.

The numerical confirmation. The Rust solver's report `Revision/kohn_sham/reports/ks-rust-solver.json` holds the check t3_block_map_solver_selftest: the problems $(m, \lambda, \theta = 0)$ and $(-m, \lambda, \theta = \pi)$, solved independently at the slice $a_{4,0} = 1$, gave the same sorted levels, occupations, Kohn-Sham energies and integrated energy-momentum components and the opposite S, with the worst difference 9.95×10^{-14} against the tolerance 10^{-9} fixed in advance. For $N = 8$ and $\lambda = 0.01946$: $E_{KS} = -9.868426190876 \times 10^{-4}$ against $-9.868426190868 \times 10^{-4}$ (80 levels), while the control with the untransformed tip gave -2.0145243719. For $N = 136$ and $\lambda = 0.0009298$: $E_{KS} = 32.39294915318$ in both runs (160 levels), control 29.283751318. Status: COMPUTED; the report itself labels it "NOT a proof of T3". Notebook 19a reproduces these numbers. This self-test of two states was the only numerical confirmation recorded with T3 (the fourth gap of the adversarial verification); the completion replaces it by the two demonstrations below.

The completion of 2026-10-08. An adversarial verification re-ran both T3 verifiers (they reproduce `t3-theory.json` and the two T3 reports byte for byte) and re-derived every step of the proof by hand. It found no error in the statement or the proof of T3, and five gaps, listed in `Revision/pairing/kohn_sham/t3-completion.json`, entry adversarial_verification. The completion is a separate record; T3, its hypotheses H1 to H6 and its statements S1 to S5 are unchanged. Its exact checks are those of `Revision/pairing/kohn_sham/reports/wolfram-t3-completion.json`

(3 of 3 PASS) and `Revision/pairing/kohn_sham/reports/python-t3-completion.json` (7 of 7 PASS):

gap	what was missing	Wolfram check	sympy check
1	the coefficients of the mean field were typed into the Wolfram script, and its v_v clause compared an expression with itself (Section 19.8; the script has since been fixed at the root)	`T3C_mean_field_coefficients_from_ks_theory`	`T3C.mean_field_coefficients_from_ks_theory`
2	the filling convention was used for the partner without showing that its particle levels are the images of the original's (Section 19.9, item (iv))	`T3C_filling_convention_mapped`	`T3C.filling_convention_mapped`
3	only the four diagonal energy-momentum components of the 2×2 reduction were proved equal (statement S6 below)	`T3C_krein_rule_16_component`	`T3C.krein_rule_16_component`
4	the numerical confirmation was the two-state self-test (the demonstrations below)	none	`T3C.rust_demo_numerical_demonstration`, `T3C.reference_demo_numerical_demonstration`
5	`t3-theory.json` does not name the history of the numerical states a PRESCRIBED BACKGROUND; the completion record does	none	none
all	the T3 reports pass; the sympy side confirms the completion record	none	`T3C.t3_reports_pass`, `compare.t3_completion`

Statement S6: the 16-component expectation rule. Steps 3 and 4 work with the 2×2 blocks and the four diagonal components ρ, p_3, p_t, p_0. S6 says: with the expectation rule $\rho = \sum f\, uu^\dagger B$ of Section 19.4 (here ρ is the 16×16 matrix of the rule, not the energy density), the image state, whose orbitals are $u' = \Gamma u$, has $\rho' = -\Gamma\rho\Gamma$, so n is unchanged, S and Q change sign, and every component of the energy-momentum tensor and of the current $J^a = \langle\bar{\Psi}\gamma^{(x_a)}\Psi\rangle$ is unchanged. The derivation, line by line. First, because $B^2 = 1$, multiplying $\rho = \sum f\, uu^\dagger B$ from the right by B gives

$$\sum f\, uu^\dagger = \rho B.$$

Rule: $BB = 1$. Then, for the image orbitals,

$$\rho' = \sum f\,(\Gamma u)(\Gamma u)^\dagger B = \sum f\,\Gamma u u^\dagger \Gamma^\dagger B = \Gamma\Big(\sum f\, uu^\dagger\Big)\Gamma B = \Gamma\rho B\Gamma B.$$

Rule: $(\Gamma u)^\dagger = u^\dagger\Gamma^\dagger$; $\Gamma^\dagger = \Gamma$ (real and diagonal, Section 19.5); the constant matrix Γ comes out of the sum; the line before. Now $\Gamma B = -B\Gamma$ (Section 19.5), so

$$B\Gamma B = -\Gamma BB = -\Gamma, \qquad \rho' = \Gamma\rho(B\Gamma B) = -\Gamma\rho\Gamma.$$

Rule: replace $B\Gamma$ by $-\Gamma B$, then $BB = 1$. The expectation value of any 16×16 matrix X in the image state is then

$$\langle X\rangle' = \mathrm{Tr}(X\rho') = -\mathrm{Tr}(X\Gamma\rho\Gamma) = -\mathrm{Tr}(\Gamma X\Gamma\rho).$$

Rule: the trace does not change when the factors are moved around in a circle; here the last factor Γ is moved to the front. Section 19.5 showed $\Gamma X\Gamma = +X$ when X is a number times a product of an even number of gammas, and $\Gamma X\Gamma = -X$ for an odd number. Hence **an even product changes sign**, $\langle X\rangle' = -\langle X\rangle$, and **an odd product is unchanged**, $\langle X\rangle' = +\langle X\rangle$. Line by line for the quantities of the theory:

quantity	**matrix X**	**gammas in X**	**in the image state**
charge density n	$B = -iC\gamma^{(x_4)}$	5, odd	unchanged
scalar density S	C	4, even	sign reversed
density Q	$B\gamma^{(x_0)}$	6, even	sign reversed
current J^a	$C\gamma^{(x_a)}$	5, odd	unchanged
kinetic bilinears $\bar\Psi\gamma^{(x_a)}\partial\Psi$	$C\gamma^{(x_a)}$	5, odd	unchanged
spin-connection bilinears $\bar\Psi\gamma^{(x_a)}S^{bc}\Psi$	$C\gamma^{(x_a)}S^{bc}$	7, odd	unchanged

Rule for the table: C has four gammas, so $B = -iC\gamma^{(x_4)}$ has five, $B\gamma^{(x_0)}$ six and $C\gamma^{(x_a)}$ five, and $S^{bc} = \frac{1}{4}(\gamma^{(x_b)}\gamma^{(x_c)} - \gamma^{(x_c)}\gamma^{(x_b)})$ adds two; a constant factor such as $-i$ or $\frac{1}{4}$ does not matter, and Γ is constant, so it passes through the derivative ∂. The energy-momentum tensor is built from the kinetic and spin-connection bilinears (unchanged), from the mass term mS and from $U = \frac{\lambda}{2}S^2$: with $m\to -m$ and $S\to -S$ the product mS is unchanged, and with the same λ so is U. So every component of the 16-component energy-momentum tensor and every component of the current are the same in the two members, while S and Q change sign. The record verifies each ingredient with the author's matrices of `Revision/algebra/gammas.json`: $\Gamma^2 = 1$, $\Gamma B\Gamma = -B$, $\Gamma C\Gamma = C$, $\Gamma\gamma^{(x_a)}\Gamma = -\gamma^{(x_a)}$ for all eight a, $\Gamma S^{ab}\Gamma = S^{ab}$ for all 64 pairs, $(\Gamma u)(\Gamma u)^\dagger B = -\Gamma(uu^\dagger B)\Gamma$ for a general 16-component u, and the $8 + 512$ kinetic bilinears. Status: PROVED; checks T3C_krein_rule_16_component and T3C.krein_rule_16_component. S6 is an equality of expectation values in Kohn-Sham (quasi-free) states with the fixed Krein matrix B, not a statement about two independently quantised universes.

The two numerical demonstrations (status COMPUTED; demonstrations, not part of the proof). Each solves, at the slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$ of the PRESCRIBED BACKGROUND history, three members of every state: **plus**, the universe (m, λ) with the tip angle 0 (the A of Section 19.17); **image**, the universe $(-m, +\lambda)$ with the tip angle π (B); and **control**, $(-m, +\lambda)$ with the untransformed tip angle 0 (C).

- The Rust solver (`Revision/pairing/kohn_sham/reports/t3-rust-demo.json`, 7 of 7 checks PASS; table `Revision/pairing/kohn_sham/numerics/results/t3-rust-states.csv`) on all 210 states of the canonical matrix: 75 ground states ($N = 8,136,688$; $\lambda = 0, \pm\lambda_1, \pm\lambda_2$; five slices) and 135 Mermin states ($\lambda = 0, \pm\lambda_1$ at $T = 0.01, 0.02, 0.05$), 630 runs in all. The plus members reproduce the committed canonical matrix to 2.538×10^{-10}. Image equals plus to 2.179×10^{-13} in the ground states and 3.877×10^{-12} in the thermal states (tolerance 10^{-9}): the levels label by label with the sector map $(n_2, j, \text{parity}) \to (n_2, -j, \text{other parity})$, the occupations, E_{KS}, μ, the entropy, Ω, F, the gap, the energy-momentum integrals, the even profiles point by point, and S, Q, M_{eff} with the opposite sign. The control differs from plus: its density profile $n(y)$ differs by at least 4.750 times the maximum of n (state N8_lamm1_a15) in the 196 states where the solver found a control state; in the other 14 states, all with $\lambda < 0$, the self-consistency loop of the control diverges, which the record reports as such and not as a proof that no such state exists. A second control, the T1 parameters $(-m, -\lambda)$ with the tip angle π, differs from plus by at least 1.21×10^{-2} in all 150 states with $\lambda \neq 0$: the Kohn-Sham partner carries $+\lambda$. The agreement of image and plus is at the rounding level **by construction**, because the solver's shooting is covariant under the map (Section 19.17 shows why); it tests the solver's handling of the transformed boundary conditions, the labels, the filling convention, the self-consistency loop and the Mermin root, not the discretisation.
- The independent reference solver (`Revision/pairing/kohn_sham/reports/t3-reference-demo.json`, 7 of 7 checks PASS; Chapter 16): finite differences on grids of 300, 600 and 1200 points, combined by Richardson extrapolation (the combination of the three grids in which the leading discretisation error cancels), on 18 states (13 ground, 5 Mermin). Its frame for the image is not the discrete image of the plus frame, so on a single grid the levels of the two members differ by up to 2.23×10^{-4}; after the extrapolation image equals plus to 2.043×10^{-14} (ground) and 3.038×10^{-14} (thermal). This is the test that does not depend on the discretisation: a statement about the continuum problem. The exact zero-momentum image spectra are reproduced to 2.80×10^{-14}; the control's density differs by at least 5.296 times the maximum (3 states without a converged control, all with $\lambda < 0$); and the $-M$ universe of the reference solver equals the $-M$ universe of the Rust solver to 1.255×10^{-11}.

What the completion adds to the list of what is not established (`t3-completion.json`, entry not_established): the demonstrations show the equalities only on the states solved, to their stated tolerances, and control states for which a solver found no self-consistent solution are reported as such, not as a proof that none exists; S6 is not a statement about two independently quantised universes; the filling convention is a CONVENTION whose justification is OPEN. Section 19.22 collects the whole list.

19.11 Why T3 is not T1, and the mirror copy in the Z2 orbifold

Two theorems, one matrix. T1 (Chapter 18) and T3 are both built on the chirality Γ, yet T1 pairs (m, λ) with $(-m, -\lambda)$ at opposite energies and T3 pairs (m, λ, θ) with $(-m, +\lambda, \pi - \theta)$ at equal energies. There is no contradiction, because they speak about different quantities.

- T1 is about the **classical bilinears** $\Psi^\dagger X \Psi$ of the field. Under $\Psi \to \Gamma\Psi$ such a bilinear becomes $\Psi^\dagger \Gamma X \Gamma \Psi$, and $\Gamma X \Gamma = +X$ for an even product of gammas, $-X$ for an odd one (Section 19.5).
- T3 is about the **expectation values** $u^\dagger B X u$ of the quantised theory (Section 19.4), which contain one more factor, the Krein matrix B. Under $u \to \Gamma u$ they become $u^\dagger \Gamma B X \Gamma u = u^\dagger (\Gamma B \Gamma)(\Gamma X \Gamma) u = -u^\dagger B (\Gamma X \Gamma) u$, because $\Gamma B \Gamma = -B$ (Section 19.5) and $\Gamma\Gamma = 1$.

So every sign of T1 is reversed at the Kohn-Sham level. Line by line for the two densities that matter:

quantity	**matrix X**	**gammas in X**	**classical bilinear (T1)**	**Kohn-Sham density (T3)**
scalar density S	C	4, even	unchanged	sign reversed
charge density n	B	5, odd	sign reversed	unchanged

The classical scalar density is unchanged under Γ (`Revision/pairing/reports/python-pairing.json`, checks T1.metric.grassmann.S_invariant and T1.metric.commuting.S_invariant) and the classical current is reversed (checks T1.metric.grassmann.current and T1.metric.commuting.current); then self-consistency of the mass term $m + \lambda S$ in the field equation $\gamma^\mu D_\mu \Psi = (m + \lambda S)\Psi$, whose left side changes sign under Γ, requires $(m, \lambda) \to (-m, -\lambda)$. At the Kohn-Sham level S changes sign and n does not, and the same requirement gives $(m, \lambda) \to (-m, +\lambda)$, as Step 3 found. The record states this comparison as an interpretation, labelled as such and not as a theorem: in its parameters the Kohn-Sham map is of the T2 type (same coupling, equal energies), although its matrix is the chirality of T1 (`t3-theory.json`, entry relation_to_T1_T2; section 8.4 of the record `Revision/docs/PAIR_CREATION_PROOFS.md`).

The mirror copy inside the Z2 orbifold. The brane condition of Section 19.3 comes from the ASSUMED Z2 construction: the patch $-L \le y \le 0$ is glued at $y = 0$ to its mirror image $0 \le y \le L$. On the doubled interval the record uses the reflection P_A: $\chi(y) \to \phi(y) = \sigma_3 \chi(-y)$. Write $h(M, \kappa, v) = -i\sigma_1 \frac{d}{dy} + M\sigma_2 + \kappa k \sigma_3 + v$ (the type j is a common factor and plays no role here). Line by line, with $\frac{d\phi}{dy}(y) = -\sigma_3 \chi'(-y)$ (chain rule):

$$
\begin{aligned}
h(-M(-y), \kappa(-y), v(-y))\phi = {} & i\sigma_1\sigma_3\chi'(-y) - M(-y)\sigma_2\sigma_3\chi(-y) \\
& + \kappa(-y)k\,\sigma_3\sigma_3\chi(-y) + v(-y)\sigma_3\chi(-y),
\end{aligned}
$$

$$
\begin{aligned}
\sigma_3\big(h(M, \kappa, v)\chi\big)(-y) = {} & -i\sigma_3\sigma_1\chi'(-y) + M(-y)\sigma_3\sigma_2\chi(-y) \\
& + \kappa(-y)k\,\sigma_3\sigma_3\chi(-y) + v(-y)\sigma_3\chi(-y).
\end{aligned}
$$

Rule: insert ϕ in the first line; multiply $h\chi$ at the point $-y$ by σ_3 in the second. The two lines are equal term by term, because $\sigma_1\sigma_3 = -\sigma_3\sigma_1$ and $\sigma_2\sigma_3 = -\sigma_3\sigma_2$. So P_A maps an orbital of

the mass function $M(y)$ onto an orbital of the mass function $-M(-y)$, with the potential $v(-y)$ (check bc_mirror_map_PA of the Kohn-Sham theory reports). The doubled problem has one mass function on the whole interval, so it is symmetric under P_A exactly when $M_{\text{eff}}(y) = -M_{\text{eff}}(-y)$: the mass is **odd** across the brane. Under σ_3, n_o is unchanged and s_o changes sign ($\sigma_3\sigma_2\sigma_3 = -\sigma_2$), so on the mirror side $n(y) = n(-y)$ and $S(y) = -S(-y)$. If the mirror side has the parameters $(\tilde{m}, \tilde{\lambda})$, its mass at a point $y > 0$ is, line by line,

$$\tilde{m} + \frac{15}{16}\tilde{\lambda}S(y) = \tilde{m} - \frac{15}{16}\tilde{\lambda}S(-y),$$

and it must equal $-M_{\text{eff}}(-y) = -m - \frac{15}{16}\lambda S(-y)$ for every value of $S(-y)$; its potential $-\tilde{\lambda}n(y)/16 = -\tilde{\lambda}n(-y)/16$ must equal $v_v(-y) = -\lambda n(-y)/16$. Comparing the terms with and without the densities gives $\tilde{\lambda} = \lambda$ and $\tilde{m} = -m$: **inside the ASSUMED orbifold, the mirror copy of a self-consistent state of the universe of mass $+M$ is a universe of mass $-M$ with the same coupling**, the statement S5 (checks T3_z2_mirror_copy_carries_minus_m_plus_lambda and T3.z2_mirror_copy_carries_minus_m_plus_lambda; bc_mirror_parities_of_densities). This is a property of the assumed geometric construction, not a derivation that such a pair exists or was created.

19.12 The exact levels at zero momentum, and what goes wrong without the transformed tip

Without the 3-momentum ($k = 0$) and without the interaction ($M_{\text{eff}} = M$ constant, $v_v = 0$) the block equation can be solved exactly. This gives T3 a second, independent confirmation, and it shows exactly what happens to the **control** C: the universe of mass $-M$ with the UNtransformed tip angle $\theta = 0$.

The square of N. With $k = 0$ and $v = 0$, $N = M\sigma_3 + ij\varepsilon\sigma_1$. Line by line:

$$N^2 = M^2\sigma_3^2 + Mij\varepsilon\,(\sigma_3\sigma_1 + \sigma_1\sigma_3) + (ij\varepsilon)^2\sigma_1^2 = M^2\,\mathbf{1} + 0 - \varepsilon^2\,\mathbf{1} = (M^2 - \varepsilon^2)\,\mathbf{1}.$$

Rule: multiply out the square of a sum of two matrices, keeping the order of the factors; $\sigma_3^2 = \sigma_1^2 = \mathbf{1}$; $\sigma_3\sigma_1 + \sigma_1\sigma_3 = 0$; $(ij)^2 = i^2j^2 = -1$. Write $p = \sqrt{M^2 - \varepsilon^2}$; inside the mass gap ($\varepsilon^2 < M^2$) p is real, outside it $p = ir$ with the real $r = \sqrt{\varepsilon^2 - M^2}$.

The solution. $\chi(y) = e^{Ny}\chi(0)$, with the matrix exponential $e^{Ny} = \sum_{n\geq 0}(Ny)^n/n!$. The even powers are $N^{2r} = p^{2r}\mathbf{1}$ and the odd ones $N^{2r+1} = p^{2r}N$, so

$$e^{Ny} = \sum_{r\geq 0}\frac{(py)^{2r}}{(2r)!}\,\mathbf{1} + \sum_{r\geq 0}\frac{p^{2r}y^{2r+1}}{(2r+1)!}\,N = \cosh(py)\,\mathbf{1} + \frac{\sinh(py)}{p}\,N.$$

Rule: the series $\cosh x = \sum x^{2r}/(2r)!$ and $\sinh x = \sum x^{2r+1}/(2r+1)!$. Check: $\frac{d}{dy}e^{Ny} = p\sinh(py) + \cosh(py)N$ and $Ne^{Ny} = \cosh(py)N + \frac{\sinh(py)}{p}N^2 = \cosh(py)N + p\sinh(py)$: the same. At the tip $y = -L$, since cosh is even and sinh odd,

$$\chi(-L) = \Big[\cosh(pL)\,\mathbf{1} - \frac{\sinh(pL)}{p}\,N\Big]\chi(0).$$

The characteristic functions. Start at the brane with the parity, $\chi(0) = (1, 0)$ (even) or $(0, 1)$ (odd), go to the tip, and take the component that the tip condition must make zero: as a function of ε this is the **characteristic function** $D(\varepsilon)$, and its zeros are the levels. The matrix N has the columns $N(1, 0) = (M, ij\varepsilon)$ and $N(0, 1) = (ij\varepsilon, -M)$. Line by line, for the three universes:

universe	**mass, type, tip**	**even:** $\chi(0) = (1, 0)$	**odd:** $\chi(0) = (0, 1)$
A	M, j, $\theta = 0$: $\chi_2(-L) = 0$	$D = -ij\varepsilon \sinh(pL)/p$	$D = \cosh(pL) + M \sinh(pL)/p$
B	$-M$, $-j$, $\theta = \pi$: $\chi_1(-L) = 0$	$D = \cosh(pL) + M \sinh(pL)/p$	$D = +ij\varepsilon \sinh(pL)/p$
C	$-M$, j, $\theta = 0$: $\chi_2(-L) = 0$	$D = -ij\varepsilon \sinh(pL)/p$	$D = \cosh(pL) - M \sinh(pL)/p$

Rule, for example for B odd: with the mass $-M$ and the type $-j$, $N(0, 1) = (-ij\varepsilon, +M)$, and the first component of $\chi(-L)$ is $0 \cdot \cosh(pL) - \frac{\sinh(pL)}{p}(-ij\varepsilon) = ij\varepsilon \sinh(pL)/p$. The other entries follow the same way. Reading the table: **B even is A odd, and B odd is minus A even**: the two problems have the same levels, with the parities exchanged, as T3 says. **C even is A even, but C odd is not A odd**: the control differs in the odd sector. Status: PROVED; checks T3_exact_k0_spectra and T3.exact_k0_spectra (identically in ε, M, L and for both j, with the control).

The levels of A (and B). Even: $\varepsilon \sinh(pL)/p = 0$. Inside the gap $p > 0$ and $\sinh(pL) > 0$, so only $\varepsilon = 0$: the **zero mode**. Outside the gap $\sinh(irL)/(ir) = \sin(rL)/r$, so $\sin(rL) = 0$, $r = n\pi/L$ and $\varepsilon = \pm\sqrt{M^2 + (n\pi/L)^2}$, $n = 1, 2, 3, \ldots$ Odd: $\cosh(pL) + M \sinh(pL)/p = 0$. Inside the gap both terms are positive (for $M > 0$): **no level in the gap**. Outside, $\cos(rL) + M \sin(rL)/r = 0$, that is $\tan(rL) = -r/M$ (check bc_exact_k0_spectra of the Kohn-Sham theory reports).

The levels of C, and the level inside the gap. Odd: $\cosh(pL) - M \sinh(pL)/p = 0$. Inside the gap, with $p = q$ real, this is $\cosh(qL) = M \sinh(qL)/q$, that is

$$\tanh(qL) = \frac{q}{M}.$$

The function $u(q) = \tanh(qL) - q/M$ has $u(0) = 0$, the slope $u'(0) = L - 1/M = 2$ for $M = 1$, $L = 3$ (positive), and $u(M) = \tanh(ML) - 1 < 0$; it is concave for $q > 0$ (because tanh is), so it has exactly one root q in $(0, M)$. The level is, line by line,

$$\varepsilon_b^2 = M^2 - q^2 = M^2 - M^2 \tanh^2(qL) = \frac{M^2}{\cosh^2(qL)}, \qquad \varepsilon_b = \frac{M}{\cosh(qL)}.$$

Rule: $q = M \tanh(qL)$ from the condition; $1 - \tanh^2 = 1/\cosh^2$. For $M = 1$, $L = 3$: $q = 0.994901528453$ and $\varepsilon_b = 0.100851121375$ (Notebook 19a, Out [18]; Notebook 19b finds the same level as a zero of D). Outside the gap the odd levels of C obey $\tan(rL) = +r/M$, and no odd level of C equals an odd level of A: if $\cos(rL) + M \sin(rL)/r = 0$ and $\cos(rL) - M \sin(rL)/r = 0$ both held, their sum would give $\cos(rL) = 0$ and their difference $\sin(rL) = 0$, impossible because $\cos^2 + \sin^2 = 1$.

The numbers. For $M = 1$, $L = 3$, below $\varepsilon = 4$ (Notebook 19b, Out [13] and Out [16]):

sector	levels
A even = B odd = C even	0, 1.4479719304, 2.3208814802, 3.2969083095
A odd = B even	1.2922928281, 2.0106285600, 2.9119358754, 3.8829900326
C odd	0.1008511214, 1.6875789860, 2.6839784554, 3.7115109122

The even levels are $\sqrt{1+(n\pi/3)^2}$ for $n = 1, 2, 3$. The levels of A reproduce the analytic table `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv` to 8.9×10^{-16}, and the lowest odd level 1.2922928281 is the "bulk edge" of `Revision/kohn_sham/results/parameters.json`. For $v = 0$ the levels of the type $-j$ are the negatives of those of the type j, so each ladder is symmetric about zero.

Where the zero modes live. At $\varepsilon = 0$, $N = M\sigma_3$, so $\chi_1' = M\chi_1$ and $\chi_2' = -M\chi_2$. For A (even, $\chi_2(0) = 0$) this forces $\chi_2 = 0$ everywhere and $\chi = (e^{My}, 0)$, which also obeys the tip condition $\chi_2(-L) = 0$: a zero mode **largest at the brane**. For B (mass $-M$, so $\chi_1' = -M\chi_1$, $\chi_2' = M\chi_2$; odd, $\chi_1(0) = 0$) it gives $\chi = (0, e^{My})$, again largest at the brane, the image $\sigma_2(e^{My}, 0) = (0, ie^{My})$ up to the factor i. For C (mass $-M$, even) it gives $\chi = (e^{-My}, 0)$: a zero mode **largest at the tip**. The coordinate density $e^{6Hy}n(y)$ of a zero mode is $|\chi|^2/(\ell^3 v_t)$, proportional to e^{2My} for A and B and to e^{-2My} for C; its value at the brane divided by its value at the tip is $e^{2ML} = e^6 = 403.4288$ for A and B and $e^{-6} = 0.002478752$ for C (Notebook 19a, Out [17]).

The brane band and its slope. When a small 3-momentum k is switched on, the zero mode becomes the lowest level of the **brane band**. Its slope follows from the Hellmann-Feynman rule with $\delta h = \frac{\partial h_j}{\partial k}k = jk\kappa\sigma_3$, $\kappa = e^{-Hy}$ at the slice $a_{4,0} = 0$. For A ($j = 1$, $\chi = (e^{My}, 0)$, $\chi^\dagger\sigma_3\chi = e^{2My}$), line by line:

$$c = \frac{\int_{-L}^{0} e^{-Hy}e^{2My}\,dy}{\int_{-L}^{0} e^{2My}\,dy} = \frac{(1-e^{-(2M-H)L})/(2M-H)}{(1-e^{-2ML})/(2M)} = \frac{2M}{2M-H}\cdot\frac{1-e^{-(2M-H)L}}{1-e^{-2ML}}.$$

Rule: $\int_{-L}^{0} e^{\alpha y}dy = (1-e^{-\alpha L})/\alpha$, with $\alpha = 2M - H$ and $\alpha = 2M$. For B ($j = -1$, $\chi = (0, ie^{My})$): $j\chi^\dagger\sigma_3\chi = (-1)(-e^{2My}) = e^{2My}$, the same integrand: **the same slope**. For C ($j = 1$, $\chi = (e^{-My}, 0)$):

$$c_{\text{ctrl}} = \frac{\int_{-L}^{0} e^{-(2M+H)y}\,dy}{\int_{-L}^{0} e^{-2My}\,dy} = \frac{2M}{2M+H}\cdot\frac{e^{(2M+H)L}-1}{e^{2ML}-1}.$$

Rule: $\int_{-L}^{0} e^{-\beta y}dy = (e^{\beta L}-1)/\beta$. For $M = H = 1$, $L = 3$: $c = 2/(1+e^{-3}) = 1.9051482536449$ (`ks-theory.json`, entry checksNumeric; check brane_band_slope; `Revision/kohn_sham/results/spectrum/brane-band-slope.csv`) and $c_{\text{ctrl}} = 13.4219751976$, about seven times larger (Notebook 19b, Out [19]). The reason is visible in the integrands: the control's zero mode lives at the tip, where the redshift weight $\kappa = e^{-Hy}$ is largest (e^3 at $y = -3$), so a 3-momentum costs it much more energy. At a later slice both slopes are multiplied by $e^{-a_{4,0}}$, the redshift.

Summary of the control. With the untransformed tip, three things go wrong at once: the zero modes move from the brane to the tip; a level appears inside the mass gap, $\varepsilon_b = 0.1009$; and the brane band starts seven times steeper. The self-consistent states of C therefore differ from those

of A in every number, as Notebook 19a shows. T3 is not an automatic symmetry of the solver: it needs exactly the transformed tip.

19.13 Example: the proof of T3 by hand (Notebook 19b)

Notebook 19b repeats Sections 19.5 to 19.12 with the computer, as far as possible exactly. It reads the author's eight gammas from the record, checks with whole numbers that they are real signed permutation matrices obeying the anticommutation rule, forms the chirality Γ, and shows that Γ maps every block onto its partner block as $\pm\sigma_2$ (Section 19.5). With sympy it re-derives the four steps of the proof (Sections 19.6 to 19.9: the block Hamiltonian, the tip and brane conditions, the densities, the mean field), each time with the negative control, and reproduces the corresponding checks of `python-t3.json`. It computes the six characteristic functions of Section 19.12 symbolically, finds their zeros numerically, compares the levels of A with the analytic table of the record, finds the control's level inside the mass gap, finds all these levels a second time by shooting (Runge-Kutta integration from the tip to the brane), and measures the slopes of the brane band of A, B and C against the closed forms. It needs Python only (numpy, sympy, matplotlib), takes about 50 seconds (24.2 s in the recorded build on the computer that built this book), draws six figures and ends with the line ALL 19 CHECKS PASSED (notebook 19b).

19.14 How to run Notebook 19b

Step 1. What this notebook does and what it needs.

Notebook 19b (Theorem T3 by hand: the block map, the exact spectra and the control) is the file `Revision/textbook/notebooks/19b_t3_by_hand.ipynb` of the repository Dirac_claude. It reads the eight real 16 x 16 gamma matrices of the record, checks that they are real signed permutation matrices that obey the anticommutation rule of the signature (4,4), builds their product, the chirality Gamma, and shows that Gamma maps each 2 x 2 Kohn-Sham block onto its partner block as the Pauli matrix sigma2. It then re-derives with exact algebra (sympy) every step of the proof of theorem T3: the block Hamiltonian, the boundary conditions, the densities and the mean field. It computes the exact zero-momentum spectra of the universe of mass +M, of its T3 partner and of the negative control with the untransformed tip, finds the level that only the control has inside the mass gap, and solves the brane band at small momenta by shooting, comparing its slope with the closed forms. It reproduces the corresponding Revision records and draws six figures. Python only; no Rust is needed. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at

the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 19b_t3_by_hand.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 50 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 19b_t3_by_hand.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 19b_t3_by_hand.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/19b.captions.json`

- `Revision/textbook/figures/19b_1_eight_gammas.png`
- `Revision/textbook/figures/19b_2_gamma_block_map.png`
- `Revision/textbook/figures/19b_3_mean_field_map.png`
- `Revision/textbook/figures/19b_4_characteristic_functions.png`
- `Revision/textbook/figures/19b_5_k0_ladders.png`
- `Revision/textbook/figures/19b_6_brane_band.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 19 CHECKS PASSED (notebook 19b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/19b_t3_by_hand.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".

- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 19b_t3_by_hand.ipynb
python -m nbconvert --execute --inplace 19b_t3_by_hand.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- A sympy cell runs for more than a minute: the exact simplifications take a few seconds on a normal computer. If a cell runs much longer, check that the installed sympy is the version pinned in the requirements file, and run the notebook again.
- A shooting cell shows the label with the star for half a minute: this is normal: the shooting cells integrate the equations for thousands of levels at once (3000 Runge-Kutta steps each). Wait until the label shows a number.

19.15 Notebook 19b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/19b_t3_by_hand.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Theorem **T3** of the Revision record pairs every self-consistent Kohn-Sham state of
the universe with the bare mass $+m$, the coupling $\lambda$ and the tip angle
$\theta$ with a state of the universe with the bare mass $-m$, the **same** coupling
$+\lambda$ and the tip angle $\pi - \theta$, with equal levels, energies and
```

energy-momentum profiles and the opposite scalar density. This notebook does the proof by hand, with exact algebra where possible and with small, transparent numerical computations where a number is needed. It

- reads the author's eight real 16×16 gamma matrices, draws them, checks that they are real and obey the anticommutation rule, and builds their product, the chirality Γ;
- shows that Γ maps each 2×2 block of the Kohn-Sham problem onto its partner block as the Pauli matrix σ_2: this is the map of T3;
- re-derives every algebraic step of the proof of T3 exactly with sympy;
- computes the exact spectra at zero momentum of three universes: A (mass $+m$, tip angle 0), its T3 partner B (mass $-m$, tip angle π) and the control C (mass $-m$, tip angle 0, the tip NOT transformed), and finds the level that only C has inside the mass gap;
- solves the lowest band at small momenta by shooting and compares its slope with formulas derived here.

It needs Python only and takes about a minute.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 19b (Theorem T3 by hand: the block map, the exact spectra and the control) is the file ‘Revision/textbook/notebooks/19b_t3_by_hand.ipynb‘ of the repository Dirac_claude. It reads the eight real 16 x 16 gamma matrices of the record, checks that they are real signed permutation matrices that obey the anticommutation rule of the signature (4,4), builds their product, the chirality Gamma, and shows that Gamma maps each 2 x 2 Kohn-Sham block onto its partner block as the Pauli matrix sigma2. It then re-derives with exact algebra (sympy) every step of the proof of theorem T3: the block Hamiltonian, the boundary conditions, the densities and the mean field. It computes the exact zero-momentum spectra of the universe of mass +M, of its T3 partner and of the negative control with the untransformed tip, finds the level that only the control has inside the mass gap, and solves the brane band at small momenta by shooting, comparing its slope with the closed forms. It reproduces the corresponding Revision records and draws six figures. Python only; no Rust is needed. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the

default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
```

```
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 19b_t3_by_hand.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 50 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 19b_t3_by_hand.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 19b_t3_by_hand.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/19b.captions.json‘
- ‘Revision/textbook/figures/19b_1_eight_gammas.png‘
- ‘Revision/textbook/figures/19b_2_gamma_block_map.png‘
- ‘Revision/textbook/figures/19b_3_mean_field_map.png‘
- ‘Revision/textbook/figures/19b_4_characteristic_functions.png‘
- ‘Revision/textbook/figures/19b_5_k0_ladders.png‘
- ‘Revision/textbook/figures/19b_6_brane_band.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on

another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 19 CHECKS PASSED (notebook 19b)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/19b_t3_by_hand.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 19b_t3_by_hand.ipynb
  python -m nbconvert --execute --inplace 19b_t3_by_hand.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel

  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.
- A sympy cell runs for more than a minute: the exact simplifications take a few seconds
  on a normal computer. If a cell runs much longer, check that the installed sympy is the
  version pinned in the requirements file, and run the notebook again.
- A shooting cell shows the label with the star for half a minute: this is normal: the
  shooting cells integrate the equations for thousands of levels at once (3000
  Runge-Kutta steps each). Wait until the label shows a number.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 19b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 19b (Theorem T3 by hand: the block map, the exact spectra and the control) is
# the file Revision/textbook/notebooks/19b_t3_by_hand.ipynb of the repository
# Dirac_claude. It reads the eight real 16 x 16 gamma matrices of the record, checks that
# they are real signed permutation matrices that obey the anticommutation rule of the
# signature (4,4), builds their product, the chirality Gamma, and shows that Gamma maps
# each 2 x 2 Kohn-Sham block onto its partner block as the Pauli matrix sigma2. It then
# re-derives with exact algebra (sympy) every step of the proof of theorem T3: the block
# Hamiltonian, the boundary conditions, the densities and the mean field. It computes the
# exact zero-momentum spectra of the universe of mass +M, of its T3 partner and of the
# negative control with the untransformed tip, finds the level that only the control has
# inside the mass gap, and solves the brane band at small momenta by shooting, comparing
# its slope with the closed forms. It reproduces the corresponding Revision records and
# draws six figures. Python only; no Rust is needed. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
```

```
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
```

```
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 19b_t3_by_hand.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
```

```
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 50
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 19b_t3_by_hand.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 19b_t3_by_hand.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/19b.captions.json
# - Revision/textbook/figures/19b_1_eight_gammas.png
# - Revision/textbook/figures/19b_2_gamma_block_map.png
# - Revision/textbook/figures/19b_3_mean_field_map.png
# - Revision/textbook/figures/19b_4_characteristic_functions.png
# - Revision/textbook/figures/19b_5_k0_ladders.png
# - Revision/textbook/figures/19b_6_brane_band.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 19 CHECKS PASSED (notebook 19b)
#
# and the notebook must show 6 figures below the cells that draw them.
#
```

```
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/19b_t3_by_hand.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 19b_t3_by_hand.ipynb
#     python -m nbconvert --execute --inplace 19b_t3_by_hand.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - A sympy cell runs for more than a minute: the exact simplifications take a few
#   seconds on a normal computer. If a cell runs much longer, check that the installed
#   sympy is the version pinned in the requirements file, and run the notebook again.
# - A shooting cell shows the label with the star for half a minute: this is normal: the
#   shooting cells integrate the equations for thousands of levels at once (3000
#   Runge-Kutta steps each). Wait until the label shows a number.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
```

```
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "19b"  # this notebook: chapter 19, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
```

```
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")
```

```
say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 19b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ = ordinary 3-space, which
  inflates; $x_4$ = the time; $x_5, x_6, x_7$ = the three **extra times**, which
  **deflate exponentially**; $x_0$ = the hidden space direction.
- **Hidden coordinate** $y$: it runs from the **tip** $y = -L$ ($L = 3$) to the
  **brane** $y = 0$.
- **Matrix**: a square table of numbers; **heat map**: a picture of a matrix in which
  every entry is a coloured square. **Real** matrix: all entries are real numbers.
  **Signed permutation matrix**: every row and every column holds exactly one entry
  $\pm 1$, all others are $0$.
- **Gamma matrices** $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$: the author's eight real
  $16 \times 16$ matrices with $\gamma^a\gamma^b + \gamma^b\gamma^a =
  2\eta^{ab}\mathbf{1}$, where $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$
  in the order $x_0, \dots, x_7$ and $\mathbf{1}$ is the unit matrix.
  **Chirality** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$.
- **Pauli matrices** $\sigma_1 = \begin{pmatrix}0&1\\1&0\end{pmatrix}$,
  $\sigma_2 = \begin{pmatrix}0&-i\\i&0\end{pmatrix}$,
  $\sigma_3 = \begin{pmatrix}1&0\\0&-1\end{pmatrix}$. Each squares to $\mathbf{1}$,
  and any two of them anticommute ($\sigma_a\sigma_b = -\sigma_b\sigma_a$ for
  $a \ne b$).
- **Block**: one of the eight pairs of components into which the 16 components of
  the Kohn-Sham problem split; labelled $(j, s_2, s_3)$ with signs $\pm 1$.
- **Orbital** $\chi(y) = (\chi_1(y), \chi_2(y))$: a two-component wave function of
  one block; **level** $\varepsilon$: its energy.
- **Brane parity**: even means $\chi_2(0) = 0$, odd means $\chi_1(0) = 0$. **Tip
  angle** $\theta$: the boundary condition $(1 - Q(\theta))\chi(-L) = 0$ with
  $Q(\theta) = \cos\theta\,\sigma_3 + \sin\theta\,\sigma_2$.
- **Characteristic function**: a function of $\varepsilon$ whose zeros are exactly
  the levels.
- **Shooting**: integrate the equation from one end with the boundary condition
  there and adjust $\varepsilon$ until the condition at the other end holds.
- **Mass gap**: the energies $-m < \varepsilon < m$, where a free particle of mass
  $m$ has no level.
- **sympy**: a Python package that computes with exact symbols and fractions.
- Status labels: PROVED (exact), COMPUTED (numerical), ASSUMED (a choice),
  HYPOTHESIS, OPEN (not known).
- Units: $H = 1$ and $|m| = 1$.
```

Text cell:

```
## 4. The physical and mathematical situation
```

In the instantaneous Kohn-Sham model of the fermion field dirac16complex in the author's deflating primordial field (record 'Revision/kohn_sham/ks-theory.json'), an orbital of block type $j = \pm 1$ with 3-momentum k solves

$$h_j\chi = \varepsilon\chi,\qquad h_j = j\Big[-i\sigma_1\frac{d}{dy}
+ M_{eff}(y)\,\sigma_2 + \kappa(y)\,k\,\sigma_3\Big] + v_v(y)$$

on $-L \le y \le 0$, with $\kappa = e^{-Hy - a_{4,0}}$ (the redshift of the 3-momenta while 3-space inflates and the extra times deflate), the effective mass $M_{eff} = m + \frac{15}{16}\lambda S$ and the potential $v_v = -\frac{1}{16}\lambda n$. Move everything except the derivative to the right:
$-ij\sigma_1\,d\chi/dy = (\varepsilon - v_v)\chi - j(M_{eff}\sigma_2 +
\kappa k\sigma_3)\chi$. Multiply from the left by $ij\sigma_1$; on the left $(ij\sigma_1)(-ij\sigma_1) = -i^2j^2\sigma_1^2 = \mathbf{1}$, on the right use $j^2 = 1$, $\sigma_1\sigma_2 = i\sigma_3$ and $\sigma_1\sigma_3 = -i\sigma_2$:
$-ij^2(M_{eff}\,i\sigma_3 - \kappa k\,i\sigma_2) = M_{eff}\sigma_3 - \kappa k\sigma_2$. The result is the first-order form of the record, $d\chi/dy = N\chi$ with $N = M_{eff}\sigma_3 - \kappa k\sigma_2 + ij(\varepsilon - v_v)\sigma_1$. Writing $\chi = (a, ib)$ with real a, b it is real:

$$\frac{da}{dy} = M_{eff}\,a - (\kappa k + j(\varepsilon - v_v))\,b,\qquad
\frac{db}{dy} = (j(\varepsilon - v_v) - \kappa k)\,a - M_{eff}\,b .$$

The boundary conditions: at the brane the Z_2 mirror (ASSUMED), even or odd parity, both solved; at the tip $(1 - Q(\theta))\chi(-L) = 0$ (chosen), which is $\chi_2(-L) = 0$ for $\theta = 0$ and $\chi_1(-L) = 0$ for $\theta = \pi$.

T3 (PROVED; records 't3-theory.json', 'wolfram-t3.json', 'python-t3.json'): the map $(\chi, j) \to (\sigma_2\chi, -j)$ at the same momentum, with the brane parities exchanged and $\theta \to \pi - \theta$, sends every self-consistent state with (m, λ, θ) to one with $(-m, +\lambda, \pi - \theta)$, with equal levels, occupations, energies and energy-momentum profiles; S and Q change sign. In the real form, $\sigma_2(a, ib) = (b, ia)$: the map simply exchanges a and b and reverses j.

The three universes of this notebook ($m = 1$, no interaction where spectra are computed): A has the mass $+1$ and the tip angle 0; B, its T3 partner, the mass -1 and the tip angle π; C, the control, the mass -1 and the tip angle 0.

What is assumed and what is not shown. The Z_2 brane is ASSUMED, the tip is a chosen cutoff, the states are instantaneous mean-field states along a PRESCRIBED BACKGROUND history. T3 maps solutions onto solutions; no creation process, rate or amplitude of a pair of universes follows from it.

Text cell:

5. The eight real gamma matrices

The next cell imports the packages, reads the record 'Revision/algebra/gammas.json' and stores the eight matrices as arrays of **integers**, so that every product below is computed exactly. A complex matrix would be stored in the record as a pair (real part, imaginary part); a plain list of rows means a real matrix. The cell checks that every entry is -1, 0 or $+1$ and that every matrix is a signed

```
permutation matrix (exactly one nonzero entry in every row and every column).
```

In [2]:

```
import csv  # reads the tables (CSV files) of the record
import math  # sqrt, pi, exp, sinh, cosh, tanh of single numbers

import numpy as np  # arrays of numbers
import sympy as sp  # exact symbolic algebra
from matplotlib.colors import ListedColormap  # a colour map with three colours

GAMMAS = "Revision/algebra/gammas.json"  # the record of the gamma matrices
ALGEBRA = "Revision/algebra/reports/wolfram-algebra.json"  # its check report
fixture = json.loads(repository_file(GAMMAS).read_text(encoding="utf-8"))
stored_real = all(isinstance(g, list) for g in fixture["gamma"])  # no complex part
gamma = [np.array(g, dtype=int) for g in fixture["gamma"]]  # gamma[k] is xk
eta = [int(e) for e in fixture["eta"]]  # +1 +1 +1 +1 -1 -1 -1 -1 (x0, ..., x7)
names = fixture["coordinates"]  # "x0", ..., "x7"
values = sorted({int(x) for g in gamma for x in g.flat})  # the values that occur
one_per_line = all(
    np.array_equal(np.abs(g).sum(axis=0), np.ones(16, dtype=int))
    and np.array_equal(np.abs(g).sum(axis=1), np.ones(16, dtype=int))
    for g in gamma)  # every column and every row holds one entry +1 or -1
say(f"{len(gamma)} matrices of size {gamma[0].shape}, entries {values}, "
    f"eta = {eta}")
check(stored_real and values == [-1, 0, 1] and one_per_line,
      "the eight gammas are real signed permutation matrices",
      record=f"{ALGEBRA}, checks reality and signed_permutation_matrices")
```

Out [2]:

```
8 matrices of size (16, 16), entries [-1, 0, 1], eta = [1, 1, 1, 1, -1, -1, -1, -1]
PASS the eight gammas are real signed permutation matrices
     reproduces Revision/algebra/reports/wolfram-algebra.json, checks reality and
    signed_permutation_matrices
```

Text cell:

```
The next cell draws the eight matrices as heat maps: blue squares are $-1$, white
squares $0$, orange squares $+1$. The title of each picture names the coordinate and
its sign $\eta$ (space-like $+1$, time-like $-1$).
```

In [3]:

```
THREE = ListedColormap(["#2a78d6", "#ffffff", "#eb6834"])  # -1, 0, +1
fig, axes = plt.subplots(2, 4, figsize=(9.0, 4.9))
for a, ax in enumerate(axes.flat):
    ax.imshow(gamma[a], cmap=THREE, vmin=-1, vmax=1, interpolation="nearest")
    ax.set_title(f"$\\gamma^{{({names[a][0]}_{names[a][1]})}}$, "
```

```
                f"$\\eta = {eta[a]:+d}$", fontsize=9)
    ax.set_xticks([0, 15])
    ax.set_yticks([0, 15])
    ax.tick_params(labelsize=7)
    ax.grid(False)
fig.tight_layout()
save_figure(fig, "eight_gammas",
            "The author's eight real $16 \\times 16$ gamma matrices "
            "$\\gamma^{(x_0)}, \\dots, \\gamma^{(x_7)}$ as heat maps (row index "
            "down, column index across, both from 0 to 15): blue is $-1$, white "
            "$0$, orange $+1$. Each matrix has exactly one nonzero entry in every "
            "row and every column. The four with $\\eta = +1$ (3-space and the "
            "hidden direction) are symmetric about the diagonal; the four with "
            "$\\eta = -1$ (the time and the three deflating extra times) are "
            "antisymmetric.")
```

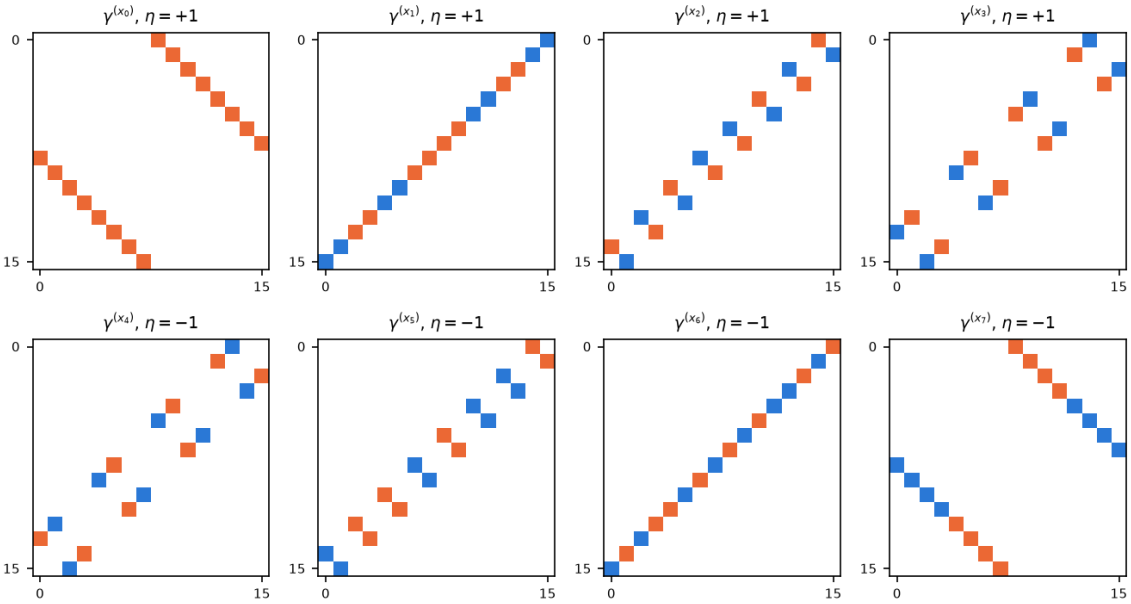

Figure 491: The author's eight real 16×16 gamma matrices $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$ as heat maps (row index down, column index across, both from 0 to 15): blue is -1, white 0, orange $+1$. Each matrix has exactly one nonzero entry in every row and every column. The four with $\eta = +1$ (3-space and the hidden direction) are symmetric about the diagonal; the four with $\eta = -1$ (the time and the three deflating extra times) are antisymmetric. (Notebook 19b, figure 1.)

Out [3]:

```
Figure 19b.1 saved as Revision/textbook/figures/19b_1_eight_gammas.png
```

Text cell:

```
The next cell checks the anticommutation rule
$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\mathbf{1}$ for all $8 \times 8 = 64$
```

```
pairs, exactly in integers. It also checks the symmetry pattern seen in the
figure. Why must it hold? A signed permutation matrix $P$ satisfies $P^TP =
\mathbf{1}$ (the transpose $P^T$, rows and columns exchanged, is its inverse), and the
rule with $a = b$ says $\gamma^a\gamma^a = \eta^{aa}\mathbf{1}$, so $\gamma^a$ is its
own inverse up to the sign $\eta^{aa}$. Hence $(\gamma^a)^T = \eta^{aa}\gamma^a$:
symmetric for $\eta = +1$, antisymmetric for $\eta = -1$.
```

In [4]:

```
unit = np.eye(16, dtype=int)
worst = 0  # largest violation of the anticommutation rule (an integer)
for a in range(8):
    for b in range(8):
        anti = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
        rule = 2 * eta[a] * (a == b) * unit  # 2 eta^ab times the unit matrix
        worst = max(worst, int(np.max(np.abs(anti - rule))))
pattern = all(np.array_equal(g.T, e * g) for g, e in zip(gamma, eta))
say(f"largest violation of the anticommutation rule over 64 pairs: {worst}")
check(worst == 0 and pattern,
      "g^a g^b + g^b g^a = 2 eta^ab exactly; g^T = eta g (symmetry pattern)",
      record=f"{ALGEBRA}, checks Clifford_relation and symmetry_pattern")
```

Out [4]:

```
largest violation of the anticommutation rule over 64 pairs: 0
PASS g^a g^b + g^b g^a = 2 eta^ab exactly; g^T = eta g (symmetry pattern)
     reproduces Revision/algebra/reports/wolfram-algebra.json, checks Clifford_relation
    and symmetry_pattern
```

Text cell:

```
## 6. The chirality and the map of T3

The next cell multiplies the eight matrices in the author's order,
$\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\cdots\gamma^{(x_7)}$, and checks:
$\Gamma$ is the diagonal matrix with $-1$ in the first eight places and $+1$ in the
last eight (the matrix stored in the record), $\Gamma^2 = \mathbf{1}$, and $\Gamma$
anticommutes with every gamma matrix. Then it reads the block basis $V$ of the
record `ks-theory.json`: 16 columns, two for each block $(j, s_2, s_3)$, written as
text (`0`, `1`, `-1`, `I`, `-I`, times $1/(2\sqrt2)$). It checks that $V$ is unitary
($V^\dagger V = \mathbf{1}$, where $V^\dagger$ is the transpose with every entry
complex-conjugated) and that $\Gamma$ sends the two columns of block
$(j, s_2, s_3)$ to the two columns of block $(-j, s_2, s_3)$ times $s_2\sigma_2$.
```

In [5]:

```
Gamma = gamma[0]  # gamma^(x0) first ...
for a in range(1, 8):
    Gamma = Gamma @ gamma[a]  # ... times gamma^(x1), ..., gamma^(x7)
```

```
diagonal = np.diag([-1] * 8 + [1] * 8)
anticommutes = all(np.array_equal(Gamma @ g, -(g @ Gamma)) for g in gamma)
check(np.array_equal(Gamma, diagonal) and np.array_equal(Gamma @ Gamma, unit)
      and np.array_equal(Gamma, np.array(fixture["Gamma"])) and anticommutes,
      "Gamma = diag(-1 (8 times), +1 (8 times)), Gamma^2 = 1, Gamma g^a = -g^a Gamma",
      record=f"{ALGEBRA}, checks Gamma_diag and Gamma_anticommutes_with_gammas")

KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
ENTRY = {"0": 0.0, "1": 1.0, "-1": -1.0, "I": 1j, "-I": -1j}  # the record's text
V = np.array([[ENTRY[x] for x in row]
              for row in ks["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
V = V / (2.0 * math.sqrt(2.0))  # the record stores 2 sqrt(2) V
labels = [tuple(int(x) for x in label) for label in ks["blockBasis"]["labels"]]
pauli1 = np.array([[0, 1], [1, 0]], dtype=complex)
pauli2 = np.array([[0, -1j], [1j, 0]])
pauli3 = np.array([[1, 0], [0, -1]], dtype=complex)
unitary = float(np.max(np.abs(V.conj().T @ V - np.eye(16))))
worst_map, worst_form = 0.0, 0.0
for b, (j, s2, s3) in enumerate(labels):
    p = labels.index((-j, s2, s3))  # the partner block
    Vb, Vp = V[:, 2 * b:2 * b + 2], V[:, 2 * p:2 * p + 2]
    worst_map = max(worst_map, float(np.max(np.abs(Gamma @ Vb
                                                   - Vp @ (s2 * pauli2)))))
    g0 = Vb.conj().T @ gamma[0] @ Vb  # gamma^(x0) inside block b
    g04 = Vb.conj().T @ gamma[0] @ gamma[4] @ Vb  # gamma^(x0) gamma^(x4) there
    worst_form = max(worst_form, float(np.max(np.abs(g0 - pauli3))),
                     float(np.max(np.abs(g04 - j * pauli1))))
say(f"largest deviations: unitarity {unitary:.1e}, block map {worst_map:.1e}, "
    f"forms gamma^(x0) = sigma3 and gamma^(x0) gamma^(x4) = j sigma1 "
    f"{worst_form:.1e}")
check(unitary < 1e-12 and worst_map < 1e-12 and worst_form < 1e-12,
      "Gamma maps block (j, s2, s3) onto block (-j, s2, s3) as s2 sigma2",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.Gamma_is_the_block_map")
```

Out [5]:

```
PASS Gamma = diag(-1 (8 times), +1 (8 times)), Gamma^2 = 1, Gamma g^a = -g^a Gamma
     reproduces Revision/algebra/reports/wolfram-algebra.json, checks Gamma_diag and
    Gamma_anticommutes_with_gammas
largest deviations: unitarity 1.1e-16, block map 0.0e+00, forms gamma^(x0) = sigma3 and
    gamma^(x0) gamma^(x4) = j sigma1 1.1e-16
PASS Gamma maps block (j, s2, s3) onto block (-j, s2, s3) as s2 sigma2
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.Gamma_is_the_block_map
```

Text cell:

```
The next cell draws two matrices. Left: $\Gamma$ itself. Right: $\Gamma$ written in
the block basis, $V^\dagger\Gamma V$. Its entries are $0$ or $\pm i$ (the cell checks
that the real part vanishes), so the picture shows the imaginary part. The
```

```
$2 \times 2$ squares sit off the diagonal: block $b$ is joined to its partner block
with the opposite $j$, and each square is $\pm\sigma_2$.
```

In [6]:

```
in_blocks = V.conj().T @ Gamma @ V  # Gamma in the block basis
real_part = float(np.max(np.abs(in_blocks.real)))
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 4.4))
left.imshow(Gamma, cmap=THREE, vmin=-1, vmax=1, interpolation="nearest")
left.set_title("$\\Gamma = \\gamma^{(x_0)}\\gamma^{(x_1)}\\cdots\\gamma^{(x_7)}$",
               fontsize=10)
right.imshow(in_blocks.imag, cmap=THREE, vmin=-1, vmax=1, interpolation="nearest")
right.set_title("imaginary part of $V^\\dagger\\Gamma V$ (block basis)", fontsize=10)
tags = [f"({j:+d},{s2:+d},{s3:+d})" for j, s2, s3 in labels]
right.set_xticks([2 * b + 0.5 for b in range(8)])
right.set_xticklabels(tags, rotation=90, fontsize=6)
right.set_yticks([2 * b + 0.5 for b in range(8)])
right.set_yticklabels(tags, fontsize=6)
for edge in range(1, 8):
    right.axhline(2 * edge - 0.5, color="0.6", lw=0.5)
    right.axvline(2 * edge - 0.5, color="0.6", lw=0.5)
for ax in (left, right):
    ax.grid(False)
left.set_xticks([0, 7, 8, 15])
left.set_yticks([0, 7, 8, 15])
fig.tight_layout()
save_figure(fig, "gamma_block_map",
            "Left: the chirality $\\Gamma$, the product of the eight real gamma "
            "matrices, as a heat map (blue $-1$, white $0$, orange $+1$; indices 0 "
            "to 15): $-1$ on the first eight diagonal places, $+1$ on the last "
            "eight. Right: the imaginary part of $\\Gamma$ in the Kohn-Sham block "
            "basis, with the block labels $(j, s_2, s_3)$ on the axes. The nonzero "
            "$2 \\times 2$ squares join each block to the block with the opposite "
            "$j$ and the same $s_2, s_3$, and each square is $\\pm\\sigma_2$: this "
            "is the map $(\\chi, j) \\to (\\sigma_2\\chi, -j)$ of theorem T3.")
report("largest real part of V^dagger Gamma V", f"{real_part:.1e}")
```

Out [6]:

```
Figure 19b.2 saved as Revision/textbook/figures/19b_2_gamma_block_map.png
RESULT largest real part of V^dagger Gamma V = 2.2e-17
```

Text cell:

```
## 7. The proof of T3, step by step, in exact algebra

**Step 1: the block Hamiltonian.** Conjugating a Pauli matrix with $\sigma_2$ gives
$\sigma_2\sigma_1\sigma_2 = -\sigma_1$, $\sigma_2\sigma_2\sigma_2 = \sigma_2$,
$\sigma_2\sigma_3\sigma_2 = -\sigma_3$ (because $\sigma_2$ anticommutes with
$\sigma_1$ and $\sigma_3$ and $\sigma_2^2 = \mathbf{1}$). The constant matrix
$\sigma_2$ also passes through $d/dy$. So
```

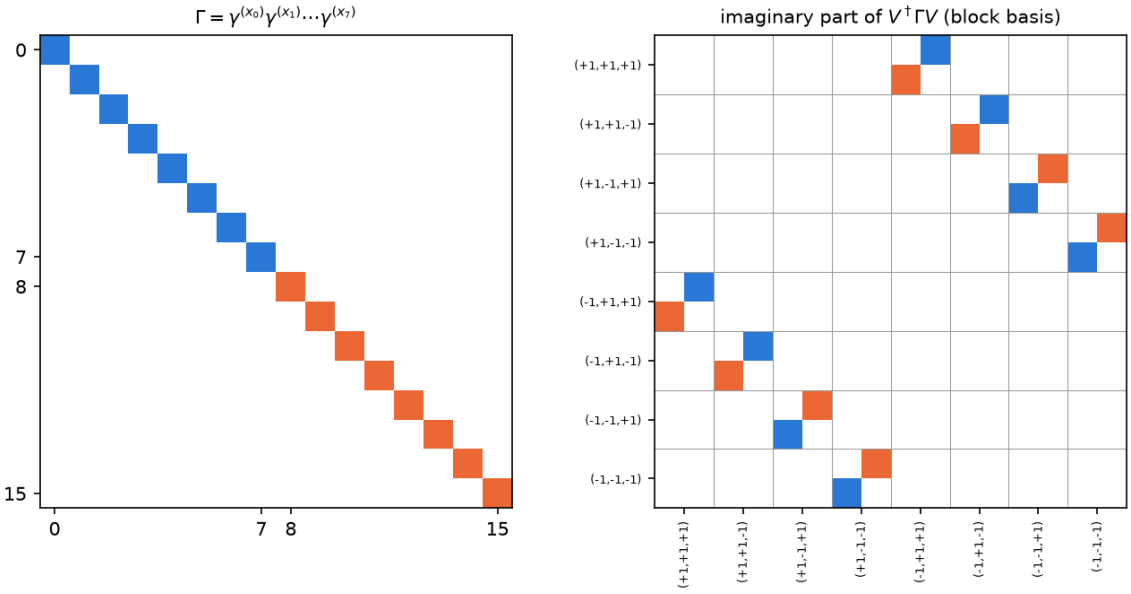

Figure 492: Left: the chirality Γ, the product of the eight real gamma matrices, as a heat map (blue -1, white 0, orange $+1$; indices 0 to 15): -1 on the first eight diagonal places, $+1$ on the last eight. Right: the imaginary part of Γ in the Kohn-Sham block basis, with the block labels (j, s_2, s_3) on the axes. The nonzero 2×2 squares join each block to the block with the opposite j and the same s_2, s_3, and each square is $\pm\sigma_2$: this is the map $(\chi, j) \to (\sigma_2\chi, -j)$ of theorem T3. (Notebook 19b, figure 2.)

```
$$\sigma_2 h_j(M)\sigma_2 = j\Big[+i\sigma_1\frac{d}{dy} + M\sigma_2 -
\kappa k\sigma_3\Big] + v_v = (-j)\Big[-i\sigma_1\frac{d}{dy} + (-M)\sigma_2 +
\kappa k\sigma_3\Big] + v_v = h_{-j}(-M).$$

If $h_j(M)\chi = \varepsilon\chi$, then $h_{-j}(-M)(\sigma_2\chi) =
\sigma_2h_j(M)\sigma_2\sigma_2\chi = \sigma_2h_j(M)\chi = \varepsilon\,\sigma_2\chi$:
the image is an orbital of the partner block with the mass $-M$ and the SAME level.
The next cell checks the three conjugation rules and the identity
$h_{-j}(-M)(\sigma_2\chi) = \sigma_2h_j(M)\chi$ for arbitrary functions $M(y)$,
$\kappa(y)$, $v_v(y)$, $\chi_1(y)$, $\chi_2(y)$ and both $j$, with sympy; as a
control it checks that without $M \to -M$ the identity fails. It also checks the
same statement for the first-order matrix: $\sigma_2N_j(M)\sigma_2 = N_{-j}(-M)$.
```

In [7]:

```
y = sp.Symbol("y", real=True)  # the hidden coordinate
k, eps, vv = sp.symbols("k epsilon v_v", real=True)  # momentum, level, potential
Mf, kappa, vf = (sp.Function(name)(y) for name in ("M", "kappa", "v"))
chi = sp.Matrix([sp.Function("chi1")(y), sp.Function("chi2")(y)])  # any orbital
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices, exactly
s2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
one = sp.eye(2)
```

```
def h(j, mass, c):
    """h_j(mass) applied to the two-component function c."""
    return j * (-sp.I * s1 * c.diff(y) + mass * s2 * c + kappa * k * s3 * c) + vf * c


def is_zero(matrix):
    """True when every entry simplifies to exactly 0."""
    return all(sp.simplify(sp.expand(entry)) == 0 for entry in matrix)


rules = (s2 * s1 * s2 == -s1 and s2 * s2 * s2 == s2 and s2 * s3 * s2 == -s3)
maps = all(is_zero(h(-j, -Mf, s2 * chi) - s2 * h(j, Mf, chi)) for j in (1, -1))
control = not is_zero(h(-1, Mf, s2 * chi) - s2 * h(1, Mf, chi))  # no M -> -M
Ms = sp.Symbol("M", real=True)
jj = sp.Symbol("j", real=True)


def N(j, mass):
    """The first-order matrix N of d chi/dy = N chi (constant data)."""
    return mass * s3 - sp.Symbol("kappa") * k * s2 + sp.I * j * (eps - vv) * s1


ode = (s2 * N(jj, Ms) * s2 - N(-jj, -Ms)).applyfunc(sp.expand) == sp.zeros(2, 2)
say(f"conjugation rules: {rules}; block map for both j: {maps}; control fails "
    f"as it must: {control}; first-order form: {ode}")
check(rules and maps and control,
      "sigma2 h_j(M) sigma2 = h_(-j)(-M) for arbitrary M(y), kappa(y), v(y)",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.block_hamiltonian_map")
check(ode, "sigma2 N_j(M) sigma2 = N_(-j)(-M) for the first-order form",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check T3.ode_map")
```

Out [7]:

```
conjugation rules: True; block map for both j: True; control fails as it must: True;
    first-order form: True
PASS sigma2 h_j(M) sigma2 = h_(-j)(-M) for arbitrary M(y), kappa(y), v(y)
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.block_hamiltonian_map
PASS sigma2 N_j(M) sigma2 = N_(-j)(-M) for the first-order form
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check T3.ode_map
```

Text cell:

```
**Step 2: the boundary conditions.** At the tip, $\sigma_2Q(\theta)\sigma_2 =
\cos\theta\,\sigma_2\sigma_3\sigma_2 + \sin\theta\,\sigma_2 = -\cos\theta\,\sigma_3 +
\sin\theta\,\sigma_2 = Q(\pi - \theta)$, because $\cos(\pi - \theta) = -\cos\theta$
and $\sin(\pi - \theta) = \sin\theta$. Hence $(1 - Q(\theta))\chi = 0$ holds exactly
when $(1 - Q(\pi - \theta))\sigma_2\chi = \sigma_2(1 - Q(\theta))\chi = 0$: the image
obeys the tip condition with the angle $\pi - \theta$. At the brane, $\sigma_2(1 -
\sigma_3)\sigma_2 = 1 + \sigma_3$: the even condition $(1 - \sigma_3)\chi(0) = 0$
```

```
(that is $\chi_2(0) = 0$) becomes the odd one $(1 + \sigma_3)\sigma_2\chi(0) = 0$.
The next cell checks both, and as a control that the image $\sigma_2(1, 0)$ of the
solution $(1, 0)$ of the condition $\theta = 0$ violates the UNtransformed condition
and obeys the transformed one.
```

In [8]:

```
theta = sp.Symbol("theta", real=True)


def Q(angle):
    """The tip matrix Q(angle) = cos(angle) sigma3 + sin(angle) sigma2."""
    return sp.cos(angle) * s3 + sp.sin(angle) * s2


tip = (s2 * Q(theta) * s2 - Q(sp.pi - theta)).applyfunc(sp.simplify) == sp.zeros(2, 2)
e1 = sp.Matrix([1, 0])  # the vector (1, 0)
zero2 = sp.zeros(2, 1)
tip_control = ((one - Q(0)) * e1 == zero2 and (one - Q(0)) * (s2 * e1) != zero2
               and (one - Q(sp.pi)) * (s2 * e1) == zero2)
brane = s2 * (one - s3) * s2 == one + s3
say(f"tip: sigma2 Q(theta) sigma2 = Q(pi - theta): {tip}; control: {tip_control}; "
    f"brane parities exchanged: {brane}")
check(tip and tip_control and brane,
      "tip angle theta -> pi - theta and the brane parities are exchanged",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, checks "
             "T3.tip_condition_map and T3.brane_parities_exchanged")
```

Out [8]:

```
tip: sigma2 Q(theta) sigma2 = Q(pi - theta): True; control: True; brane parities
    exchanged: True
PASS tip angle theta -> pi - theta and the brane parities are exchanged
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, checks
    T3.tip_condition_map and T3.brane_parities_exchanged
```

Text cell:

```
**Step 3: the densities.** Per orbital the record ‘ks-theory.json‘ uses (up to a
common positive factor) $n_o = \chi^\dagger\chi$, $s_o = j\chi^\dagger\sigma_2\chi$,
$t_o = j\chi^\dagger\sigma_3\chi$, $q_o = \chi^\dagger\sigma_3\chi$ and
$c_o = j\chi^\dagger\sigma_1\chi$, where $\chi^\dagger$ is the row of the complex
conjugates. Under $(\chi, j) \to (\sigma_2\chi, -j)$, with $\sigma_2^\dagger =
\sigma_2$: $n_o \to \chi^\dagger\sigma_2\sigma_2\chi = n_o$;
$s_o \to -j\chi^\dagger\sigma_2\sigma_2\sigma_2\chi = -s_o$;
$t_o \to -j\chi^\dagger\sigma_2\sigma_3\sigma_2\chi = +t_o$;
$q_o \to \chi^\dagger\sigma_2\sigma_3\sigma_2\chi = -q_o$;
$c_o \to -j\chi^\dagger\sigma_2\sigma_1\sigma_2\chi = +c_o$. The next cell checks the
signs $(+, -, +, -, +)$ for an arbitrary complex $\chi = (a_1 + ib_1, a_2 + ib_2)$ and
both $j$.
```

In [9]:

```
a1, b1, a2, b2 = sp.symbols("a1 b1 a2 b2", real=True)
psi = sp.Matrix([a1 + sp.I * b1, a2 + sp.I * b2])  # an arbitrary complex orbital


def densities(j, c):
    """(n, s, t, q, c) of one orbital c of block type j."""
    dag = c.H  # the row of complex conjugates
    return [sp.expand((dag * m * c)[0, 0]) for m in
            (one, j * s2, j * s3, s3, j * s1)]


SIGNS = (1, -1, 1, -1, 1)  # expected: n, t, c even; s, q odd
signs_ok = all(sp.expand(after - sign * before) == 0
               for j in (1, -1)
               for after, before, sign in zip(densities(-j, s2 * psi),
                                              densities(j, psi), SIGNS))
say(f"n = {densities(1, psi)[0]},  s (j = +1) = {densities(1, psi)[1]}")
check(signs_ok, "under the map n, t, c keep their sign, s and q change it",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.orbital_densities")
```

Out [9]:

```
n = a1**2 + a2**2 + b1**2 + b2**2,  s (j = +1) = 2*a1*b2 - 2*a2*b1
PASS under the map n, t, c keep their sign, s and q change it
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.orbital_densities
```

Text cell:

```
**Step 4: the mean field.** The record gives $M_{eff} = m + c_M\lambda S$ and
$v_v = c_v\lambda n$ with $c_M = 15/16$ and $c_v = -1/16$, and the interaction energy
density $e_{int} = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2$. The image
state has the densities $(n, -S)$ (step 3). For it to be self-consistent in the
partner block, its effective mass must be $-M_{eff}$ (step 1) and its potential
$v_v$ unchanged. With the bare mass $-m$ and the SAME coupling,
$-m + c_M\lambda(-S) = -(m + c_M\lambda S) = -M_{eff}$, and $c_v\lambda n$ is
unchanged. With $(-m, -\lambda)$ instead, $-m + c_M(-\lambda)(-S) = -m +
c_M\lambda S \ne -M_{eff}$ whenever $\lambda S \ne 0$. The next cell reads the two
coefficients from the record and checks this, the evenness of $e_{int}$ in $S$, and
that $M_{eff} - m$ and $v_v$ are the derivatives of $e_{int}$ with respect to $S$
and $n$.
```

In [10]:

```
potentials = ks["exchange"]["kohnShamPotentials"]
cM = sp.Rational(potentials["Meff_coefficient_of_lambda_S"])  # 15/16
cv = sp.Rational(potentials["vv_coefficient_of_lambda_n"])  # -1/16
```

```
m, lam, S, n = sp.symbols("m lambda S n", real=True)


def M_eff(mass, coupling, scalar):
    """The effective mass m + cM lambda S of the record."""
    return mass + cM * coupling * scalar


e_int = sp.Rational(15, 32) * lam * S ** 2 - sp.Rational(1, 32) * lam * n ** 2
partner = sp.expand(M_eff(-m, lam, -S) + M_eff(m, lam, S)) == 0
wrong = sp.expand(M_eff(-m, -lam, -S) + M_eff(m, lam, S)) != 0
even = sp.expand(e_int.subs(S, -S) - e_int) == 0
derivatives = (sp.expand(sp.diff(e_int, S) - (M_eff(m, lam, S) - m)) == 0
               and sp.expand(sp.diff(e_int, n) - cv * lam * n) == 0)
say(f"cM = {cM}, cv = {cv}; (-m, +lambda) works: {partner}; (-m, -lambda) fails: "
    f"{wrong}; e_int even in S: {even}; derivatives: {derivatives}")
check(partner and wrong and even and derivatives and (cM, cv) == (
    sp.Rational(15, 16), sp.Rational(-1, 16)),
    "M_eff(-m, +lambda, -S) = -M_eff(m, lambda, S); (-m, -lambda) fails",
    record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
           "T3.mean_field_map")
```

Out [10]:

```
cM = 15/16, cv = -1/16; (-m, +lambda) works: True; (-m, -lambda) fails: True; e_int even
    in S: True; derivatives: True
PASS M_eff(-m, +lambda, -S) = -M_eff(m, lambda, S); (-m, -lambda) fails
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.mean_field_map
```

Text cell:

```
The next cell draws step 4. For every value of the scalar density $S$ (horizontal
axis) it draws the mass that the image needs, $-M_{eff}(m, \lambda, S)$, the mass
that the partner universe $(-m, +\lambda)$ gives to the image density $-S$, and the
mass that the universe $(-m, -\lambda)$ would give. The coupling
$\lambda = 0.8$ is exaggerated on purpose so that the slopes can be seen; the
identity holds for every $\lambda$.
```

In [11]:

```
S_values = np.linspace(-1.0, 1.0, 201)
lam_demo, m_demo, cM_value = 0.8, 1.0, float(cM)
need = -(m_demo + cM_value * lam_demo * S_values)  # -M_eff(m, lambda, S)
give_partner = -m_demo + cM_value * lam_demo * (-S_values)  # (-m, +lambda) at -S
give_wrong = -m_demo + cM_value * (-lam_demo) * (-S_values)  # (-m, -lambda) at -S
fig, ax = plt.subplots()
ax.plot(S_values, need, color="#2a78d6", lw=3.0,
        label="needed: $-M_{eff}(m, \\lambda, S)$")
ax.plot(S_values, give_partner, color="#eb6834", lw=1.6, ls="--",
        label="partner $(-m, +\\lambda)$ at $-S$")
```

```
ax.plot(S_values, give_wrong, color="#eda100", lw=1.6, ls=":",
        label="$(-m, -\\lambda)$ at $-S$")
ax.set_xlabel("scalar density $S$ (units of $|m|^7$)")
ax.set_ylabel("effective mass (units of $|m|$)")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "mean_field_map",
            "Step 4 of the proof of T3: the effective mass that the image state "
            "needs, $-M_{eff}(m, \\lambda, S)$ (blue), against the scalar density "
            "$S$, with $m = 1$ and an exaggerated coupling $\\lambda = 0.8$ "
            "(units of $|m|$ and $|m|^7$). The partner universe with the bare mass "
            "$-m$ and the same coupling $+\\lambda$ gives exactly this mass to the "
            "image density $-S$ (orange dashed, on top of the blue line); the "
            "universe with $(-m, -\\lambda)$ gives a line with the opposite slope "
            "(yellow dotted), which agrees only at $S = 0$.")
check(float(np.max(np.abs(need - give_partner))) < 1e-15
      and float(np.max(np.abs(need - give_wrong))) > 1.0,
      "the drawn lines: the partner gives the needed mass, (-m, -lambda) does not")
```

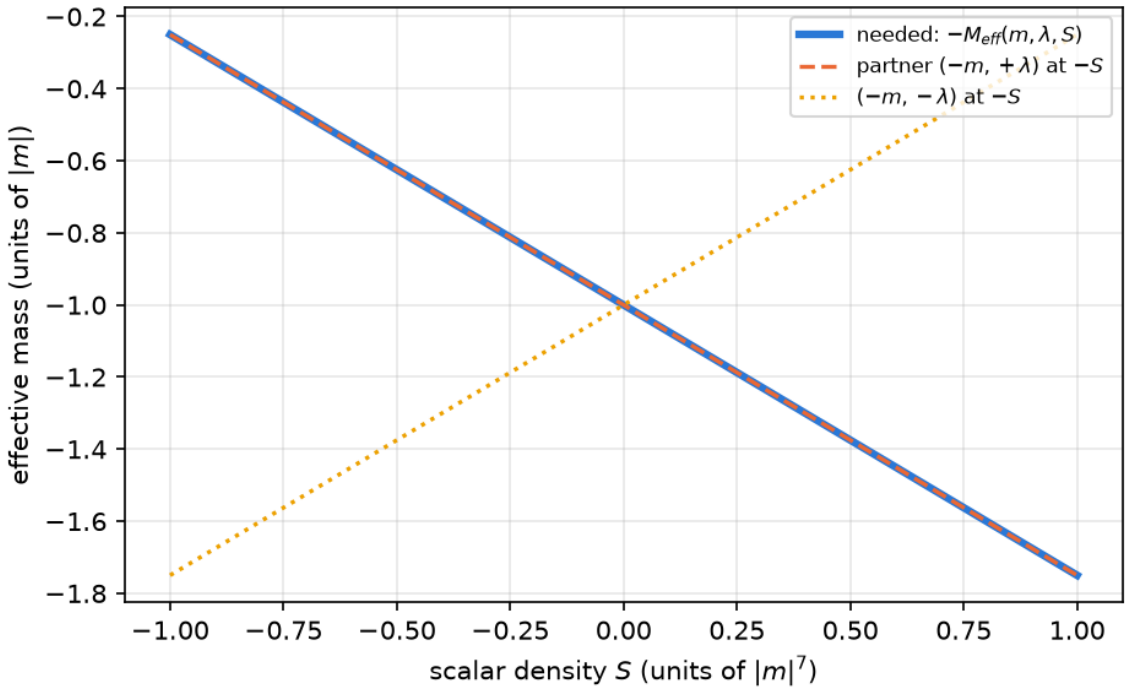

Figure 493: Step 4 of the proof of T3: the effective mass that the image state needs, $-M_{eff}(m, \lambda, S)$ (blue), against the scalar density S, with $m = 1$ and an exaggerated coupling $\lambda = 0.8$ (units of $|m|$ and $|m|^7$). The partner universe with the bare mass $-m$ and the same coupling $+\lambda$ gives exactly this mass to the image density $-S$ (orange dashed, on top of the blue line); the universe with $(-m, -\lambda)$ gives a line with the opposite slope (yellow dotted), which agrees only at $S = 0$. (Notebook 19b, figure 3.)

Out [11]:

```
Figure 19b.3 saved as Revision/textbook/figures/19b_3_mean_field_map.png
PASS the drawn lines: the partner gives the needed mass, (-m, -lambda) does not
```

Text cell:

```
## 8. The exact spectra at zero momentum

Take $k = 0$, no interaction ($M_{eff} = M$ constant, $v_v = 0$). Then
$N = M\sigma_3 + ij\varepsilon\sigma_1$ and $N^2 = M^2\sigma_3^2 +
(ij\varepsilon)^2\sigma_1^2 + Mij\varepsilon(\sigma_3\sigma_1 + \sigma_1\sigma_3) =
(M^2 - \varepsilon^2)\mathbf{1}$, because $\sigma_3^2 = \sigma_1^2 = \mathbf{1}$,
$(ij)^2 = -1$ and $\sigma_3\sigma_1 + \sigma_1\sigma_3 = 0$. Write
$p = \sqrt{M^2 - \varepsilon^2}$. In the series $e^{Ny} = \sum_n (Ny)^n/n!$ the even
powers give $p^{2r}y^{2r}\mathbf{1}$ and the odd ones $p^{2r}y^{2r+1}N$, so

$$\chi(y) = e^{Ny}\chi(0) = \Big[\cosh(py)\,\mathbf{1} +
\frac{\sinh(py)}{p}\,N\Big]\chi(0).$$

Start at the brane with the parity ($\chi(0) = (1, 0)$ for even, $(0, 1)$ for odd),
go to the tip $y = -L$ and ask the tip condition: the component that must vanish
there, as a function of $\varepsilon$, is the **characteristic function**, and its
zeros are the levels. For A ($M$, $j$, $\theta = 0$, the second component must
vanish) and B ($-M$, $-j$, $\theta = \pi$, the first component must vanish) and the
control C ($-M$, $j$, $\theta = 0$) the next cell computes all six functions with
sympy and checks: A even $= -$(B odd), A odd $=$ B even (so A and B have the SAME
levels, sector by sector after exchanging the parities); C even $=$ A even, but C
odd $\ne$ A odd (the control differs in the odd sector).
```

In [12]:

```
Mp, L = sp.symbols("M L", positive=True)
yy = sp.Symbol("yy", real=True)
p = sp.sqrt(Mp ** 2 - eps ** 2)


def propagator(j, mass):
    """exp(N yy) for k = 0, v = 0: cosh(p yy) 1 + sinh(p yy)/p N."""
    N0 = mass * s3 + sp.I * j * eps * s1
    return sp.cosh(p * yy) * one + sp.sinh(p * yy) / p * N0


e2 = sp.Matrix([0, 1])  # the vector (0, 1)
at_tip = {yy: -L}
D = {}  # characteristic functions, for j = +1
D["A even"] = (propagator(1, Mp) * e1)[1].subs(at_tip)  # chi2(-L)
D["A odd"] = (propagator(1, Mp) * e2)[1].subs(at_tip)
D["B even"] = (propagator(-1, -Mp) * e1)[0].subs(at_tip)  # chi1(-L)
D["B odd"] = (propagator(-1, -Mp) * e2)[0].subs(at_tip)
D["C even"] = (propagator(1, -Mp) * e1)[1].subs(at_tip)  # chi2(-L)
D["C odd"] = (propagator(1, -Mp) * e2)[1].subs(at_tip)
for name in D:
    say(f"D({name}) = {sp.simplify(D[name])}")
pairs_equal = (sp.simplify(D["A even"] + D["B odd"]) == 0
               and sp.simplify(D["A odd"] - D["B even"]) == 0)
control_differs = (sp.simplify(D["C even"] - D["A even"]) == 0
```

```
                  and sp.simplify(D["C odd"] - D["A odd"]) != 0)
check(pairs_equal and control_differs,
      "k = 0: A and B have equal characteristic functions; C differs (odd sector)",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.exact_k0_spectra")
```

Out [12]:

```
D(A even) = -I*epsilon*sinh(L*sqrt(M**2 - epsilon**2))/sqrt(M**2 - epsilon**2)
D(A odd) = M*sinh(L*sqrt(M**2 - epsilon**2))/sqrt(M**2 - epsilon**2) + cosh(L*sqrt(M**2 -
    epsilon**2))
D(B even) = M*sinh(L*sqrt(M**2 - epsilon**2))/sqrt(M**2 - epsilon**2) + cosh(L*sqrt(M**2
    - epsilon**2))
D(B odd) = I*epsilon*sinh(L*sqrt(M**2 - epsilon**2))/sqrt(M**2 - epsilon**2)
D(C even) = -I*epsilon*sinh(L*sqrt(M**2 - epsilon**2))/sqrt(M**2 - epsilon**2)
D(C odd) = -M*sinh(L*sqrt(M**2 - epsilon**2))/sqrt(M**2 - epsilon**2) + cosh(L*sqrt(M**2
    - epsilon**2))
PASS k = 0: A and B have equal characteristic functions; C differs (odd sector)
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.exact_k0_spectra
```

Text cell:

```
Dropping the constant factor $-ij$ of the even function, the three different
functions are, with $C = \cosh(pL)$ and $\Sigma = \sinh(pL)/p$:
A even $= \varepsilon\,\Sigma$, A odd $= C + M\Sigma$, C odd $= C - M\Sigma$. For
$\varepsilon^2 > M^2$, $p = ir$ with $r = \sqrt{\varepsilon^2 - M^2}$, and
$\cosh(irL) = \cos(rL)$, $\sinh(irL)/(ir) = \sin(rL)/r$: all three are real for real
$\varepsilon$. The next cell evaluates them for $M = 1$, $L = 3$ on a fine grid of
$\varepsilon$ between $0$ and $4$, finds every zero (a sign change between two grid
points, refined by bisection), and compares the levels of A with the analytic levels
of the record ‘Revision/kohn_sham/results/spectrum/free-k0-analytic.csv‘ (even:
$\varepsilon = 0$ and $\sqrt{M^2 + (n\pi/L)^2}$; odd: $\tan(rL) = -r/M$).
```

In [13]:

```
M_BARE, L_TIP = 1.0, 3.0  # the mass |m| and the tip distance L of the record


def C_and_Sigma(e):
    """cosh(pL) and sinh(pL)/p for the level(s) e, real for every real e."""
    pc = np.sqrt(M_BARE ** 2 - np.asarray(e, dtype=float) ** 2 + 0j)  # complex p
    small = np.abs(pc) < 1e-12  # p = 0: sinh(pL)/p -> L
    safe = np.where(small, 1.0, pc)
    sigma = np.where(small, L_TIP, np.sinh(safe * L_TIP) / safe)
    return np.cosh(pc * L_TIP).real, sigma.real


def f_A_even(e):
    return np.asarray(e) * C_and_Sigma(e)[1]
```

```
def f_A_odd(e):
    c, sig = C_and_Sigma(e)
    return c + M_BARE * sig


def f_C_odd(e):
    c, sig = C_and_Sigma(e)
    return c - M_BARE * sig


def zeros(f, top=4.0, points=40001):
    """All sign changes of f on (0, top], each refined by 80 bisection steps."""
    grid = np.linspace(1e-9, top, points)
    values = f(grid)
    found = []
    for i in np.nonzero(np.sign(values[:-1]) * np.sign(values[1:]) < 0)[0]:
        low, high = grid[i], grid[i + 1]
        for _ in range(80):
            middle = 0.5 * (low + high)
            if np.sign(f(middle)) == np.sign(f(low)):
                low = middle
            else:
                high = middle
        found.append(0.5 * (low + high))
    return found


levels = {"A even": [0.0] + zeros(f_A_even), "A odd": zeros(f_A_odd),
          "C odd": zeros(f_C_odd)}
for name, values in levels.items():
    say(f"{name}: " + ", ".join(f"{v:.10f}" for v in values))
SPECTRUM = "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
with repository_file(SPECTRUM).open(encoding="utf-8", newline="") as handle:
    rows = [r for r in csv.DictReader(handle)
            if float(r["m"]) == 1.0 and float(r["L"]) == 3.0]
recorded = {parity: sorted(float(r["eps_analytic"]) for r in rows
                           if r["parity"] == parity
                           and 0.0 <= float(r["eps_analytic"]) < 4.0)
            for parity in ("even", "odd")}
worst = max(max(abs(x - y) for x, y in zip(levels["A even"], recorded["even"])),
            max(abs(x - y) for x, y in zip(levels["A odd"], recorded["odd"])))
report("largest difference of the levels of A from the record", f"{worst:.1e}")
check(len(levels["A even"]) == len(recorded["even"])
      and len(levels["A odd"]) == len(recorded["odd"]) and worst < 1e-12,
      "the k = 0 levels of A are the analytic levels of the record",
      record=f"{SPECTRUM}, rows m = 1, L = 3")
```

Out [13]:

```
A even: 0.0000000000, 1.4479719304, 2.3208814802, 3.2969083095
A odd: 1.2922928281, 2.0106285600, 2.9119358754, 3.8829900326
C odd: 0.1008511214, 1.6875789860, 2.6839784554, 3.7115109122
```

```
RESULT largest difference of the levels of A from the record = 8.9e-16
PASS the k = 0 levels of A are the analytic levels of the record
     reproduces Revision/kohn_sham/results/spectrum/free-k0-analytic.csv, rows m = 1, L =
    3
```

Text cell:

```
The control's odd function $C - M\Sigma$ has a zero inside the mass gap
$0 < \varepsilon < M$, where A has none. There $p = q$ is real and the zero is
$\cosh(qL) = M\sinh(qL)/q$, that is $\tanh(qL) = q/M$; the level is
$\varepsilon_b = \sqrt{M^2 - q^2} = M/\cosh(qL)$ (because $q^2 = M^2\tanh^2(qL)$
and $1 - \tanh^2 = 1/\cosh^2$). For A the condition would be $\tanh(qL) = -q/M$,
impossible for $q > 0$. The next cell checks the two forms of $\varepsilon_b$, that
the control's odd levels above the gap are NOT those of A, and draws the three
functions (left: the even function, common to all three universes after the
exchange of parities; right: the odd functions of A and C).
```

In [14]:

```
eps_b = levels["C odd"][0]  # the first zero: inside the gap?
q_b = math.sqrt(M_BARE ** 2 - eps_b ** 2)
two_forms = (abs(math.tanh(q_b * L_TIP) - q_b / M_BARE) < 1e-12
             and abs(M_BARE / math.cosh(q_b * L_TIP) - eps_b) < 1e-12)
report("sub-gap level of the control eps_b", f"{eps_b:.12f}", "|m|")
above = levels["C odd"][1:]  # the control's odd levels above the gap
distinct = min(abs(x - y) for x in above for y in levels["A odd"]) > 1e-3
check(0.0 < eps_b < M_BARE and two_forms and distinct,
      "only the control has a level in the gap, eps_b = M/cosh(qL), tanh(qL) = q/M")

grid = np.linspace(0.0, 4.0, 2001)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.plot(grid, f_A_even(grid), color="#2a78d6", lw=2.0,
          label="$\\varepsilon\\,\\sinh(pL)/p$ (even A = odd B = even C)")
left.plot(levels["A even"], np.zeros(len(levels["A even"])), "o", color="#2a78d6")
right.plot(grid, f_A_odd(grid), color="#2a78d6", lw=2.0,
           label="A odd $= $ B even: $\\cosh(pL) + M\\sinh(pL)/p$")
right.plot(grid, f_C_odd(grid), color="#1baf7a", lw=1.6, ls=":",
           label="C odd: $\\cosh(pL) - M\\sinh(pL)/p$")
right.plot(levels["A odd"], np.zeros(len(levels["A odd"])), "o", color="#2a78d6")
right.plot(levels["C odd"], np.zeros(len(levels["C odd"])), "^", color="#1baf7a")
for ax in (left, right):
    ax.axhline(0.0, color="0.3", lw=0.8)
    ax.axvline(M_BARE, color="0.5", lw=0.8, ls="--")  # the edge of the mass gap
    ax.set_xlabel("level $\\varepsilon$ (units of $|m|$)")
    ax.set_ylim(-4.0, 6.0)
    ax.legend(fontsize=7, loc="upper right")
left.set_ylabel("characteristic function")
fig.tight_layout()
save_figure(fig, "characteristic_functions",
            "The characteristic functions of the zero-momentum spectra for "
            "$M = 1$, $L = 3$ (horizontal axis: the level $\\varepsilon$ in units "
            "of $|m|$; the dashed vertical line is the edge $\\varepsilon = M$ of "
```

```
"the mass gap). Left: the function of the even sector of A, which is "
"also that of the odd sector of the partner B and of the even sector of "
"the control C; its zeros (dots) are $0$ and $\\sqrt{M^2 + "
"(n\\pi/L)^2}$. Right: the odd sector of A (equal to the even sector of "
"B) and the odd sector of the control C. Only C has a zero inside the "
"gap, the level $\\varepsilon_b = 0.1009$, and its other zeros differ "
"from those of A.")
```

Out [14]:

```
RESULT sub-gap level of the control eps_b = 0.100851121375 |m|
PASS only the control has a level in the gap, eps_b = M/cosh(qL), tanh(qL) = q/M
```

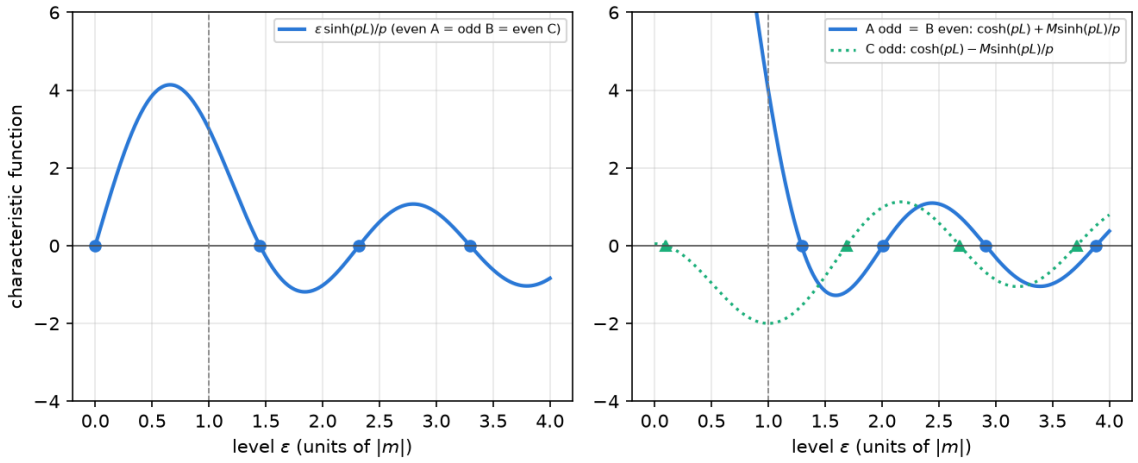

Figure 494: The characteristic functions of the zero-momentum spectra for $M = 1$, $L = 3$ (horizontal axis: the level ε in units of $|m|$; the dashed vertical line is the edge $\varepsilon = M$ of the mass gap). Left: the function of the even sector of A, which is also that of the odd sector of the partner B and of the even sector of the control C; its zeros (dots) are 0 and $\sqrt{M^2 + (n\pi/L)^2}$. Right: the odd sector of A (equal to the even sector of B) and the odd sector of the control C. Only C has a zero inside the gap, the level $\varepsilon_b = 0.1009$, and its other zeros differ from those of A. (Notebook 19b, figure 4.)

Out [14]:

```
Figure 19b.4 saved as Revision/textbook/figures/19b_4_characteristic_functions.png
```

Text cell:

```
## 9. The same levels without formulas: shooting at zero momentum

The levels of section 8 came from formulas. The next cells find them again by
**shooting**, which needs no formula and works for every $k$ and every potential.
The real equations of section 4 are integrated from the tip, where the tip
condition fixes the start ($(a, b) = (1, 0)$ for $\theta = 0$, that is
$\chi_2(-L) = 0$; $(a, b) = (0, 1)$ for $\theta = \pi$), to the brane with the
```

classical Runge-Kutta method (RK4, 3000 steps of length 0.001). A level is a value of ε for which the brane condition holds: $b(0) = 0$ (even) or $a(0) = 0$ (odd). The function ‘brane_value‘ returns the component that must vanish; it works on whole numpy arrays of problems at once (every array entry is one problem: its level, momentum, mass, block type, tip angle and parity). The function ‘find_levels‘ scans ε on a grid, keeps every sign change, and refines each one with the **Illinois method** (a safeguarded secant rule: keep a bracket with a sign change, take the zero of the straight line through its ends as the new point, keep the part with the sign change, and halve the stored value at an end that is kept twice in a row, so that both ends move).

In [15]:

```
H, STEPS = 1.0, 3000  # the constant H and the number of RK4 steps


def brane_value(e, kk, mass, j, tip, odd):
    """Shoot from the tip y = -L to the brane y = 0 at the slice a4,0 = 0.  All
    arguments are numpy arrays of equal shape (one entry per problem); returns
    a(0) where odd is 1 and b(0) where odd is 0."""
    a = np.where(tip == 0.0, 1.0, 0.0)  # theta = 0: (a, b) = (1, 0) at the tip
    b = 1.0 - a  # theta = pi: (a, b) = (0, 1)
    step = L_TIP / STEPS

    def slope(yv, a, b):
        w = math.exp(-H * yv) * kk  # kappa(y) k
        return mass * a - (w + j * e) * b, (j * e - w) * a - mass * b

    for i in range(STEPS):
        yv = -L_TIP + i * step
        ka1, kb1 = slope(yv, a, b)
        ka2, kb2 = slope(yv + step / 2, a + step / 2 * ka1, b + step / 2 * kb1)
        ka3, kb3 = slope(yv + step / 2, a + step / 2 * ka2, b + step / 2 * kb2)
        ka4, kb4 = slope(yv + step, a + step * ka3, b + step * kb3)
        a = a + step / 6 * (ka1 + 2 * ka2 + 2 * ka3 + ka4)
        b = b + step / 6 * (kb1 + 2 * kb2 + 2 * kb3 + kb4)
    return np.where(odd == 1.0, a, b)


def find_levels(problems, top, points=2001, rounds=16, first_only=False):
    """problems: rows (k, mass, j, tip, odd).  Returns (row index, level) for every
    sign change of brane_value on (0, top] (only the lowest one per row when
    first_only is True)."""
    P = np.array(problems, dtype=float)
    grid = np.linspace(1e-9, top, points)
    columns = [np.repeat(P[:, c:c + 1], points, axis=1) for c in range(5)]
    values = brane_value(np.tile(grid, (len(P), 1)), *columns)
    rows, cols = np.nonzero(np.sign(values[:, :-1]) * np.sign(values[:, 1:]) < 0)
    if first_only:  # keep the first sign change of every row
        keep = np.concatenate(([True], rows[1:] != rows[:-1]))
        rows, cols = rows[keep], cols[keep]
    lo, hi = grid[cols], grid[cols + 1]  # brackets with a sign change
    f_lo, f_hi = values[rows, cols], values[rows, cols + 1]
```

```
    data = [P[rows, c] for c in range(5)]  # the problem of every root
    kept = np.zeros(len(rows))  # which end was kept last: +1 high, -1 low
    new = lo
    for _ in range(rounds):
        new = (lo * f_hi - hi * f_lo) / (f_hi - f_lo)  # zero of the straight line
        f_new = brane_value(new, *data)
        move_low = np.sign(f_new) == np.sign(f_lo)  # the zero lies above new
        f_hi = np.where(move_low & (kept == 1), f_hi / 2, f_hi)  # Illinois rule
        f_lo = np.where(~move_low & (kept == -1), f_lo / 2, f_lo)
        lo, f_lo = np.where(move_low, new, lo), np.where(move_low, f_new, f_lo)
        hi, f_hi = np.where(move_low, hi, new), np.where(move_low, f_hi, f_new)
        kept = np.where(move_low, 1, -1)
    return rows, new


say("brane_value and find_levels defined")
```

Out [15]:

```
brane_value and find_levels defined
```

Text cell:

```
The next cell solves the six sectors at $k = 0$: for A (mass $+1$, $j = +1$, tip
angle $0$) and the control C (mass $-1$, $j = +1$, tip angle $0$) the even and the
odd parity, for B (mass $-1$, $j = -1$, tip angle $\pi$) the even and the odd parity
of the partner block. It compares every level with section 8: A's sectors with the
formulas of A, B's even sector with A's odd one and B's odd sector with A's even one
(T3 exchanges the parities), C's even sector with A's even one and C's odd sector
with the control's own formula. It also checks the zero modes: the brane value is
exactly $0$ at $\varepsilon = 0$ in A even, B odd and C even.
```

In [16]:

```
SECTORS = {  # name: (mass, j, tip angle, odd)
    "A even": (1.0, 1.0, 0.0, 0.0), "A odd": (1.0, 1.0, 0.0, 1.0),
    "B even": (-1.0, -1.0, math.pi, 0.0), "B odd": (-1.0, -1.0, math.pi, 1.0),
    "C even": (-1.0, 1.0, 0.0, 0.0), "C odd": (-1.0, 1.0, 0.0, 1.0)}
names_k0 = list(SECTORS)
rows, found = find_levels([(0.0, *SECTORS[name]) for name in names_k0], top=4.0)
shot = {name: sorted(found[rows == r]) for r, name in enumerate(names_k0)}
zero_mode = {name: float(brane_value(np.zeros(1), np.zeros(1),
                                     *(np.full(1, x) for x in SECTORS[name]))[0])
             for name in ("A even", "B odd", "C even")}
for name in ("A even", "B odd", "C even"):
    shot[name] = [0.0] + shot[name]  # the zero mode at eps = 0
expected = {"A even": levels["A even"], "A odd": levels["A odd"],
            "B even": levels["A odd"], "B odd": levels["A even"],
            "C even": levels["A even"], "C odd": levels["C odd"]}
worst_shot = 0.0
for name in names_k0:
```

```
    say(f"{name}: " + ", ".join(f"{e:.10f}" for e in shot[name]))
    if len(shot[name]) != len(expected[name]):
        worst_shot = math.inf  # a level is missing or extra
    else:
        worst_shot = max(worst_shot, max(abs(x - y) for x, y in
                                         zip(shot[name], expected[name])))
report("largest difference shooting - formula at k = 0", f"{worst_shot:.1e}")
check(worst_shot < 1e-9 and all(v == 0.0 for v in zero_mode.values()),
      "shooting finds exactly the formula levels of A, B and C at k = 0")
```

Out [16]:

```
A even: 0.0000000000, 1.4479719304, 2.3208814802, 3.2969083095
A odd: 1.2922928281, 2.0106285600, 2.9119358754, 3.8829900326
B even: 1.2922928281, 2.0106285600, 2.9119358754, 3.8829900326
B odd: 0.0000000000, 1.4479719304, 2.3208814802, 3.2969083095
C even: 0.0000000000, 1.4479719304, 2.3208814802, 3.2969083095
C odd: 0.1008511214, 1.6875789860, 2.6839784554, 3.7115109122
RESULT largest difference shooting - formula at k = 0 = 5.9e-12
PASS shooting finds exactly the formula levels of A, B and C at k = 0
```

Text cell:

```
The next cell draws the three ladders found by shooting (both signs: at $k = 0$ the
levels come in pairs $\pm\varepsilon$, because the functions of section 8 are even
or odd in $\varepsilon$) and checks that the ladders of A and B agree level by level
to $10^{-12}$, although the two were computed with different masses, block types,
tip conditions and parities.
```

In [17]:

```
COLOUR = {"A": "#2a78d6", "B": "#eb6834", "C": "#1baf7a"}
fig, ax = plt.subplots(figsize=(7.0, 5.0))
for x, kind in enumerate("ABC"):
    for parity, style in (("even", "-"), ("odd", "--")):
        for e in shot[f"{kind} {parity}"]:
            for sign in ((1,) if e == 0.0 else (1, -1)):
                ax.plot([x - 0.3, x + 0.3], [sign * e, sign * e],
                        color=COLOUR[kind], ls=style, lw=2.0)
ax.axhspan(-M_BARE, M_BARE, color="0.9", zorder=0)  # the mass gap
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(["A: $+m$, $\\theta = 0$", "B: $-m$, $\\theta = \\pi$",
                    "C: $-m$, $\\theta = 0$ (control)"])
ax.set_xlim(-0.6, 2.6)
ax.set_ylabel("level $\\varepsilon$ at $k = 0$ (units of $|m|$)")
ax.grid(axis="x", visible=False)
save_figure(fig, "k0_ladders",
            "The levels at zero momentum of the universes A (mass $+m$, tip angle "
            "0), B (mass $-m$, tip angle $\\pi$, the T3 partner) and C (mass $-m$, "
            "tip angle 0, the control), for $m = 1$, $L = 3$, no interaction, found "
            "by shooting (vertical axis: the level in units of $|m|$; solid lines "
```

```
            "even parity, dashed lines odd parity; grey band: the mass gap "
            "$|\\varepsilon| < m$). A and B have the same ladder with the parities "
            "exchanged; the control has different odd levels and the pair "
            "$\\pm\\varepsilon_b$ inside the gap.")
ladder_A = np.array(sorted(shot["A even"] + shot["A odd"]))
ladder_B = np.array(sorted(shot["B even"] + shot["B odd"]))
check(len(ladder_A) == len(ladder_B)
      and float(np.max(np.abs(ladder_A - ladder_B))) < 1e-12,
      "the shooting ladders of A and B agree level by level")
```

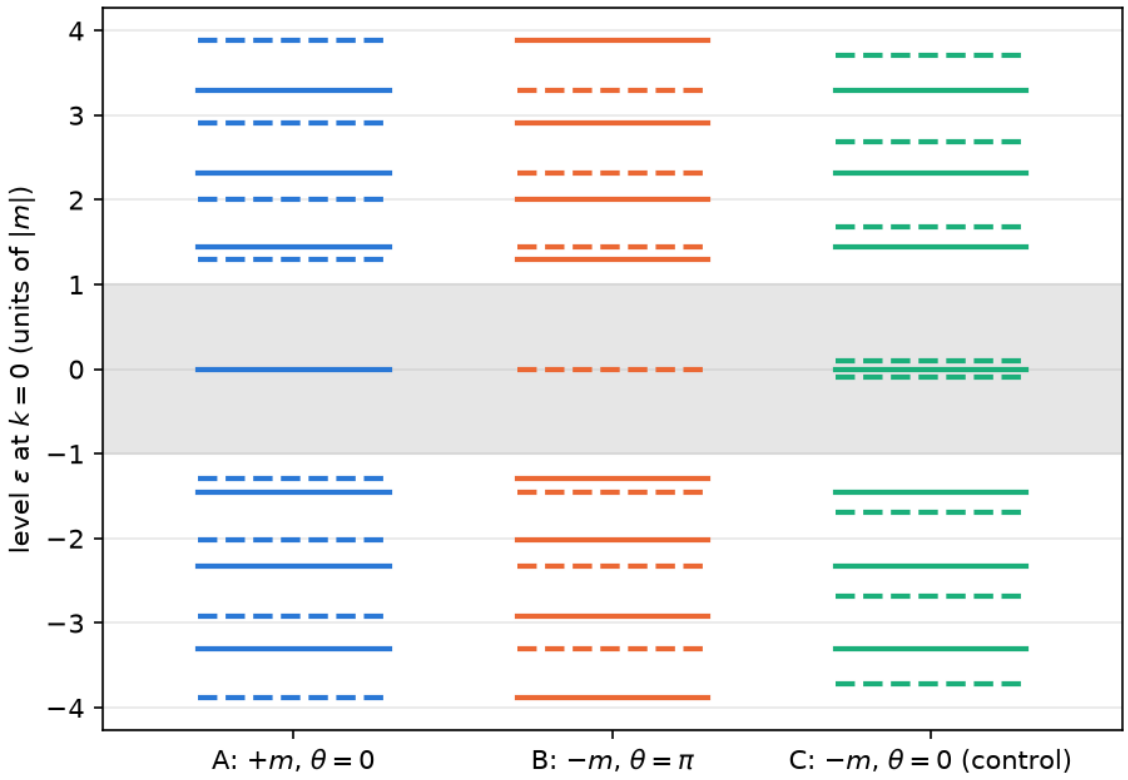

Figure 495: The levels at zero momentum of the universes A (mass $+m$, tip angle 0), B (mass $-m$, tip angle π, the T3 partner) and C (mass $-m$, tip angle 0, the control), for $m = 1$, $L = 3$, no interaction, found by shooting (vertical axis: the level in units of $|m|$; solid lines even parity, dashed lines odd parity; grey band: the mass gap $|\varepsilon| < m$). A and B have the same ladder with the parities exchanged; the control has different odd levels and the pair $\pm\varepsilon_b$ inside the gap. (Notebook 19b, figure 5.)

Out [17]:

```
Figure 19b.5 saved as Revision/textbook/figures/19b_5_k0_ladders.png
PASS the shooting ladders of A and B agree level by level
```

Text cell:

```
## 10. Small momenta: the brane band
```

```
At small $k$ the zero mode of A turns into the **brane band**, a level that grows
linearly with $k$. Its slope follows from first-order perturbation theory (the
Hellmann-Feynman rule: a small change $\delta h$ of $h$ moves a level by
$\langle\chi|\delta h|\chi\rangle/\langle\chi|\chi\rangle$, where
$\langle\chi|X|\chi\rangle = \int_{-L}^{0}\chi^\dagger X\chi\,dy$). Here
$\delta h = jk\kappa\sigma_3$ with $\kappa = e^{-Hy}$ at $a_{4,0} = 0$. For A,
$j = 1$ and $\chi = (e^{My}, 0)$, so $\chi^\dagger\sigma_3\chi = e^{2My}$ and

$$c = \frac{\int_{-L}^0 e^{(2M - H)y}dy}{\int_{-L}^0 e^{2My}dy} =
\frac{2M}{2M - H}\,\frac{1 - e^{-(2M - H)L}}{1 - e^{-2ML}} .$$

For B, $j = -1$ and $\chi = (0, ie^{My})$, so $j\chi^\dagger\sigma_3\chi =
(-1)(-e^{2My})$: the same slope. For the control C, $j = 1$ and $\chi =
(e^{-My}, 0)$, living at the tip where $\kappa$ is large:

$$c_{ctrl} = \frac{\int_{-L}^0 e^{-(2M + H)y}dy}{\int_{-L}^0 e^{-2My}dy} =
\frac{2M}{2M + H}\,\frac{e^{(2M + H)L} - 1}{e^{2ML} - 1} .$$

The next cell finds, by shooting, the lowest positive level of the band sector (A:
$j = +1$, even; B: $j = -1$, odd; C: $j = +1$, even) at 25 momenta between $0.01$
and $0.25$, for the three universes in one computation.
```

In [18]:

```
BAND = {"A": (1.0, 1.0, 0.0, 0.0), "B": (-1.0, -1.0, math.pi, 1.0),
        "C": (-1.0, 1.0, 0.0, 0.0)}  # mass, j, tip angle, odd
k_plot = np.linspace(0.0, 0.25, 26)[1:]  # 25 momenta for the figure
problems = [(kk, *BAND[kind]) for kind in "ABC" for kk in k_plot]
rows, found = find_levels(problems, top=2.0, points=401, first_only=True)
band = {kind: found[i * len(k_plot):(i + 1) * len(k_plot)]
        for i, kind in enumerate("ABC")}
band_gap = float(np.max(np.abs(band["A"] - band["B"])))
report("largest |eps_A - eps_B| on the band", f"{band_gap:.1e}")
check(len(rows) == 3 * len(k_plot) and band_gap < 1e-10
      and float(np.min(np.abs(band["A"] - band["C"]))) > 1e-3,
      "the band of B equals that of A at every momentum; the control differs")
```

Out [18]:

```
RESULT largest |eps_A - eps_B| on the band = 0.0e+00
PASS the band of B equals that of A at every momentum; the control differs
```

Text cell:

```
The next cell measures the slopes at $k \to 0$. The band is an odd function of $k$,
$\varepsilon(k) = ck + dk^3 + \dots$, so with $\varepsilon(k)/k = c + dk^2$ at $k$
and $2k$ the combination $[4\varepsilon(k)/k - \varepsilon(2k)/(2k)]/3$ removes the
$k^2$ term (Richardson extrapolation). It uses $k = 10^{-4}$ and $2 \times 10^{-4}$
and compares with the closed forms and with the slope stored in the records
(‘ks-theory.json‘, key ‘checksNumeric‘, and
‘Revision/kohn_sham/results/spectrum/brane-band-slope.csv‘).
```

In [19]:

```
small = 1e-4
problems = [(kk, *BAND[kind]) for kind in "ABC" for kk in (small, 2 * small)]
rows, found = find_levels(problems, top=0.01, points=401, first_only=True)
slopes = {kind: (4 * found[2 * i] / small - found[2 * i + 1] / (2 * small)) / 3
          for i, kind in enumerate("ABC")}
c_A = (2 * M_BARE / (2 * M_BARE - H) * (1 - math.exp(-(2 * M_BARE - H) * L_TIP))
       / (1 - math.exp(-2 * M_BARE * L_TIP)))
c_C = (2 * M_BARE / (2 * M_BARE + H) * (math.exp((2 * M_BARE + H) * L_TIP) - 1)
       / (math.exp(2 * M_BARE * L_TIP) - 1))
c_record = float(ks["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
SLOPES = "Revision/kohn_sham/results/spectrum/brane-band-slope.csv"
with repository_file(SLOPES).open(encoding="utf-8", newline="") as handle:
    c_csv = float(next(csv.DictReader(handle))["c_theory"])  # the row a4,0 = 0
say(f"closed forms: c = {c_A:.13f}, c_ctrl = {c_C:.10f}; records: "
    f"{c_record:.13f} and {c_csv:.13f}")
for kind in "ABC":
    say(f"{kind}: shooting slope {slopes[kind]:.10f}")
check(len(rows) == 6 and abs(c_A - c_record) < 1e-13 and abs(c_A - c_csv) < 1e-13
      and abs(slopes["A"] / c_A - 1) < 1e-7 and abs(slopes["B"] / c_A - 1) < 1e-7,
      "the brane-band slope of A and of its partner B is c = 1.9051482536",
      record=f"{KS_THEORY}, checksNumeric braneBandSlope_M1_H1_L3_a0, and {SLOPES}")
check(abs(slopes["C"] / c_C - 1) < 1e-6 and c_C > 7 * c_A,
      "the control's band starts with the much larger slope c_ctrl = 13.42")
```

Out [19]:

```
closed forms: c = 1.9051482536449, c_ctrl = 13.4219751976; records: 1.9051482536449 and
    1.9051482536449
A: shooting slope 1.9051482536
B: shooting slope 1.9051482536
C: shooting slope 13.4219751976
PASS the brane-band slope of A and of its partner B is c = 1.9051482536
     reproduces Revision/kohn_sham/ks-theory.json, checksNumeric
    braneBandSlope_M1_H1_L3_a0, and Revision/kohn_sham/results/spectrum/brane-band-
    slope.csv
PASS the control's band starts with the much larger slope c_ctrl = 13.42
```

Text cell:

```
The next cell draws the lowest positive level of the band sector against the
momentum for the three universes, with the straight lines $ck$ and $c_{ctrl}k$.
```

In [20]:

```
fig, ax = plt.subplots()
ax.plot(k_plot, band["A"], "o-", color=COLOUR["A"], lw=2.2, ms=6,
        label="A: $+m$, $\\theta = 0$")
ax.plot(k_plot, band["B"], "s--", color=COLOUR["B"], lw=1.4, ms=3,
```

```
        label="B: $-m$, $\\theta = \\pi$ (T3 partner)")
ax.plot(k_plot, band["C"], "^:", color=COLOUR["C"], lw=1.4, ms=5,
        label="C: $-m$, $\\theta = 0$ (control)")
k_line = np.linspace(0.0, 0.25, 51)
ax.plot(k_line, c_A * k_line, "k--", lw=0.8, label="$c\\,k$, $c = 1.905$")
short = k_line[k_line < 0.11]  # the control's line leaves the picture early
ax.plot(short, c_C * short, "k:", lw=0.8, label="$c_{ctrl}\\,k$, $c_{ctrl} = 13.42$")
ax.set_xlabel("3-space momentum $k$ (units of $|m|$)")
ax.set_ylabel("lowest positive level $\\varepsilon$ (units of $|m|$)")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "brane_band",
            "The lowest positive level of the band sector against the 3-space "
            "momentum $k$ (both in units of $|m|$) at the slice $a_{4,0} = 0$, for "
            "$m = 1$, $L = 3$, no interaction, found by shooting: A (blue circles), "
            "its T3 partner B (orange squares, on top of A) and the control C (aqua "
            "triangles). A and B start with the slope $c = 1.905$ of the brane band "
            "(dashed); the control's zero mode lives at the tip, where the redshift "
            "factor $e^{-Hy}$ is largest, so its level rises with the slope "
            "$c_{ctrl} = 13.42$ (dotted) and soon bends below the first level of "
            "the bulk.")
```

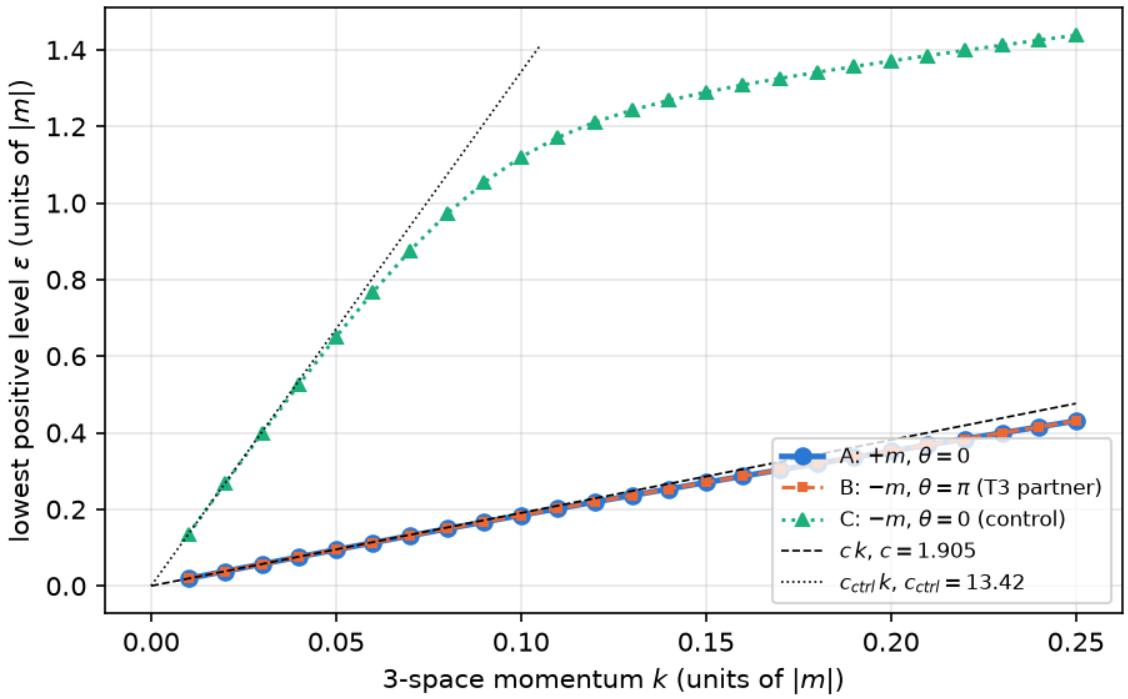

Figure 496: The lowest positive level of the band sector against the 3-space momentum k (both in units of $|m|$) at the slice $a_{4,0} = 0$, for $m = 1$, $L = 3$, no interaction, found by shooting: A (blue circles), its T3 partner B (orange squares, on top of A) and the control C (aqua triangles). A and B start with the slope $c = 1.905$ of the brane band (dashed); the control's zero mode lives at the tip, where the redshift factor e^{-Hy} is largest, so its level rises with the slope $c_{ctrl} = 13.42$ (dotted) and soon bends below the first level of the bulk. (Notebook 19b, figure 6.)

Out [20]:

```
Figure 19b.6 saved as Revision/textbook/figures/19b_6_brane_band.png
```

Text cell:

```
## 11. The last check

The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [21]:

```
paths = [output_file(f"{FIGURE_FOLDER}/{name}.png") for name in
         ["19b_1_eight_gammas", "19b_2_gamma_block_map", "19b_3_mean_field_map",
          "19b_4_characteristic_functions", "19b_5_k0_ladders", "19b_6_brane_band"]]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [21]:

```
PASS every figure file of this notebook exists
ALL 19 CHECKS PASSED (notebook 19b)
```

Text cell:

```
## 12. What this notebook showed

- PROVED in the records and re-checked here exactly (integers and sympy): the
  author's eight gamma matrices are real $16 \times 16$ signed permutation matrices
  with $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\mathbf{1}$; the chirality
  $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$ is
  $\mathrm{diag}(-1, \dots, -1, +1, \dots, +1)$ and maps each Kohn-Sham block
  $(j, s_2, s_3)$ onto $(-j, s_2, s_3)$ as $s_2\sigma_2$.
- PROVED (sympy, reproducing the checks of `python-t3.json`): the four steps of T3:
  $\sigma_2h_j(M)\sigma_2 = h_{-j}(-M)$; the tip angle goes to $\pi - \theta$ and the
  brane parities are exchanged; $n$, $t$ keep and $S$, $Q$ change their sign; the
  mean field closes exactly for $(-m, +\lambda)$ and not for $(-m, -\lambda)$.
- PROVED (sympy) and COMPUTED (roots): at zero momentum A and B have the same
  levels; the control C, with the untransformed tip, has different odd levels and a
  level inside the mass gap, $\varepsilon_b = m/\cosh(qL)$ with $\tanh(qL) = q/m$.
- COMPUTED (shooting) against closed forms: the brane band of A and B starts with the
  slope $c = 1.9051482536$ of the record; the control's band with
  $c_{ctrl} = 13.42$, because its zero mode lives at the tip.
- ASSUMED: the $Z_2$ brane; the tip condition is a choice (T3 needs the transformed
  tip angle); instantaneous mean-field states on a PRESCRIBED BACKGROUND history.
- NOT shown: that a universe is created, in pairs or otherwise. T3 maps solutions
  onto solutions; it gives no creation process, rate or amplitude.
```

19.16 Line-by-line walk-through of Notebook 19b

The notebook has 21 code cells, In [1] to In [21]. This section explains every line of every one of them, in order; the numbers that the cells print are in Section 19.15 under the labels Out [k]. In the notebook each code cell is preceded by a text cell that says what the cell does.

In [1], the set-up cell. It is the same in every notebook of the book except for the notebook's name; in notebooks that run a Rust program it has two more imports and one more function (the walk-through of Notebook 19a in Section 19.21 explains them). Its first 236 lines repeat the complete run instructions of Section 19.14 as **comment lines**: Python skips every line that starts with `#`; they are there so that the notebook file carries its own instructions. Line 237, a line of dashes, ends the instructions, and two lines of `=` signs around the title THE SET-UP mark where the code begins.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "19b"  # this notebook: chapter 19, example b
```

`import` loads a **module** (a part of Python or of an installed package) so that the cell can use it; the text after `#` on each line says what it is for. `json` reads and writes JSON files, the text format in which most Revision records are stored. `os` gives access to the operating system, here to read an environment variable. `textwrap` breaks long printed text into lines. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on Windows, macOS and Linux. `matplotlib` is the plotting package and `matplotlib.pyplot` its drawing functions, called `plt` by convention. `Image` and `display` show a saved picture below a cell. The last line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"19b"`; the figure files are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called; the text in triple quotes under the `def` line is its **docstring**, a description that Python stores but does not run. `Path.cwd`

`()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. The list `[here, *here.parents]` holds this folder followed by its parent, the parent of that, and so on up to the top of the disk (the star unpacks the parents into the list); the `for` loop takes them one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back; if none does, `raise` stops the notebook with an error message that says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`REPO` is the repository folder; it is never printed, because it differs from computer to computer while the printed output must be the same everywhere. `os.environ.get(name, default)` reads an **environment variable** (a named text that a program receives from the computer) or returns the default; when you run the notebook the variable TEXTBOOK_OUTPUT_ROOT is not set, so `OUTPUT_ROOT` is the repository, while the book's checking tool sets it to a scratch folder so that a check never changes the repository. `repository_file` gives the full path of a repository file for reading; `output_file` gives the full path at which to write one, after creating its folder (`mkdir` with `parents=True` creates missing parent folders too, and `exist_ok=True` makes it do nothing if the folder exists). `say` prints a text in lines of at most 89 characters, continuation lines starting with four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
```

```
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

`matplotlib.rcdefaults()` returns to the built-in plotting settings, so that the figures are the same on every computer, and `plt.rcParams.update` sets the figure size (7.0 by 4.2 inches), the letter size (10 points) and a faint grid. The braces `{...}` make a **dictionary**: pairs of a key and a value, written `key:  value`. A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/19b.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary `{}` into the captions file; `newline="\n"` stores the same line end on every operating system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` numbers the figures 1, 2, 3, ... in the order in which they are saved (`setdefault` returns the number already stored for this name, or stores one more than the count so far), saves the figure as a PNG file with 150 dots per inch, cut to its content and without the program's name in the file (so that two runs write the same bytes), closes it, records its caption in the captions file (`json.dumps` turns the dictionary into JSON text, one entry per line, keys sorted), shows the saved picture below the cell and prints where it was saved. The caption given to it is the caption that the book prints under the figure.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
```

```
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`PASSED` is an empty **list** (an ordered collection, in square brackets). `check` is the function behind every check of the notebook: if the statement `condition` is false, `raise AssertionError(...)` stops the notebook with an error that names the check; if it is true, the name is appended to `PASSED`, the line PASS followed by the name is printed, and a second line names the Revision record and its check when the argument `record` is given (`None` is Python's word for "nothing"). Python's own `assert` statement is not used, because Python started with the option `-O` skips it. `report` prints a key number as a line that starts with RESULT. `all_checks_passed` prints the last line of the notebook with the number of checks that passed (`len` is the length of a list). The last statement prints the one output line of In [1].

In [2], the eight gammas as whole numbers.

```
import csv  # reads the tables (CSV files) of the record
import math  # sqrt, pi, exp, sinh, cosh, tanh of single numbers

import numpy as np  # arrays of numbers
import sympy as sp  # exact symbolic algebra
from matplotlib.colors import ListedColormap  # a colour map with three colours
```

`csv` reads tables stored as CSV files (comma-separated values: one line per row, the entries separated by commas). `math` computes functions of single numbers. `numpy`, called `np`, holds arrays of numbers and multiplies matrices; `sympy`, called `sp`, computes with exact symbols, fractions and functions. `ListedColormap` makes a colour scale out of a short list of colours; it is used for the heat maps.

```
GAMMAS = "Revision/algebra/gammas.json"  # the record of the gamma matrices
ALGEBRA = "Revision/algebra/reports/wolfram-algebra.json"  # its check report
fixture = json.loads(repository_file(GAMMAS).read_text(encoding="utf-8"))
stored_real = all(isinstance(g, list) for g in fixture["gamma"])  # no complex part
gamma = [np.array(g, dtype=int) for g in fixture["gamma"]]  # gamma[k] is xk
```

```
eta = [int(e) for e in fixture["eta"]]  # +1 +1 +1 +1 -1 -1 -1 -1 (x0, ..., x7)
names = fixture["coordinates"]  # "x0", ..., "x7"
values = sorted({int(x) for g in gamma for x in g.flat})  # the values that occur
```

`GAMMAS` and `ALGEBRA` are the repository paths of the gamma record and of its Wolfram check report. `read_text` reads the record as text and `json.loads` turns it into Python objects: `fixture` is a dictionary whose keys are the names stored in the record. The record stores a real matrix as a plain list of rows and a complex one as a pair of real and imaginary parts (a dictionary), so `stored_real` is true when all eight entries of `fixture["gamma"]` are plain lists (`isinstance(g, list)` asks whether `g` is a list; `all` is true when every one is). `gamma` is the list of the eight matrices as numpy arrays of whole numbers (`dtype=int`), so that every product below is exact; the record lists the matrices in the order $x_0, \ldots, x_7$ and Python counts list positions from 0, so `gamma[k]` is $\gamma^{(x_k)}$: `gamma[0]` is $\gamma^{(x_0)}$ of the hidden direction and `gamma[7]` is $\gamma^{(x_7)}$. `eta` holds the eight signs of the frame metric, `names` the coordinate names. The **set comprehension** `{int(x) for g in gamma for x in g.flat}` collects every value that occurs in any matrix (`g.flat` runs through all 256 entries; a set keeps each value once), and `sorted` puts them in order.

```
one_per_line = all(
    np.array_equal(np.abs(g).sum(axis=0), np.ones(16, dtype=int))
    and np.array_equal(np.abs(g).sum(axis=1), np.ones(16, dtype=int))
    for g in gamma)  # every column and every row holds one entry +1 or -1
say(f"{len(gamma)} matrices of size {gamma[0].shape}, entries {values}, "
    f"eta = {eta}")
check(stored_real and values == [-1, 0, 1] and one_per_line,
      "the eight gammas are real signed permutation matrices",
      record=f"{ALGEBRA}, checks reality and signed_permutation_matrices")
```

`np.abs(g)` replaces every entry by its absolute value; `.sum(axis=0)` adds each column, `.sum(axis=1)` each row. If the entries are -1, 0, $+1$ and every column and every row sums to 1 in absolute value, then every row and every column holds exactly one nonzero entry ± 1: the matrix is a **signed permutation matrix**. `one_per_line` is true when this holds for all eight. Out [2] shows the line printed by `say` (eight 16×16 matrices, entries $-1, 0, 1$, and η) and the PASS line, which reproduces the checks reality and signed_permutation_matrices of `wolfram-algebra.json`.

In [3], the eight heat maps (Figure 19b.1).

```
THREE = ListedColormap(["#2a78d6", "#ffffff", "#eb6834"])  # -1, 0, +1
fig, axes = plt.subplots(2, 4, figsize=(9.0, 4.9))
for a, ax in enumerate(axes.flat):
    ax.imshow(gamma[a], cmap=THREE, vmin=-1, vmax=1, interpolation="nearest")
    ax.set_title(f"$\\gamma^{{({names[a][0]}_{names[a][1]})}}$, "
                 f"$\\eta = {eta[a]:+d}$", fontsize=9)
    ax.set_xticks([0, 15])
    ax.set_yticks([0, 15])
    ax.tick_params(labelsize=7)
    ax.grid(False)
fig.tight_layout()
```

A colour is written `"#rrggbb"`, three two-digit numbers in base 16 for its red, green and blue parts; `THREE` is a colour scale with blue for -1, white for 0 and orange for $+1$. `plt.subplots(2, 4, ...)`

makes a figure with a grid of two rows of four **axes** (drawing areas), 9.0 by 4.9 inches. `enumerate(axes.flat)` runs through the eight axes and counts them, $a = 0, \ldots, 7$. `imshow` draws a matrix as a picture, one coloured square per entry, with the colour scale fixed from -1 to $+1$ (`vmin`, `vmax`) and sharp squares (`interpolation="nearest"`). The title is an f-string that writes, for example, $\gamma^{(x_1)}$, $\eta = +1$: `names[a][0]` is the letter x and `names[a][1]` the digit, the doubled braces `{{` and `}}` produce single braces in the text, and `:+d` writes a whole number with its sign. The ticks are put only at 0 and 15, in small letters, and the grid is switched off. `tight_layout` arranges the eight pictures without overlaps.

```
save_figure(fig, "eight_gammas",
            "The author's eight real $16 \\times 16$ gamma matrices "
            "$\\gamma^{(x_0)}, \\dots, \\gamma^{(x_7)}$ as heat maps (row index "
            "down, column index across, both from 0 to 15): blue is $-1$, white "
            "$0$, orange $+1$. Each matrix has exactly one nonzero entry in every "
            "row and every column. The four with $\\eta = +1$ (3-space and the "
            "hidden direction) are symmetric about the diagonal; the four with "
            "$\\eta = -1$ (the time and the three deflating extra times) are "
            "antisymmetric.")
```

`save_figure` saves Figure 19b.1 with its caption (Python joins strings written next to each other into one; a doubled backslash writes one backslash). What the student should see: eight square pictures of 16×16 coloured squares; in each row and each column exactly one square is coloured, the rest white; the four pictures of x_0, x_1, x_2 and x_3 are mirror-symmetric about the diagonal from top left to bottom right, and the four of $x_4, \ldots, x_7$ are antisymmetric (a blue square on one side of the diagonal faces an orange one on the other). In [4] explains why.

In [4], the anticommutation rule, exactly.

```
unit = np.eye(16, dtype=int)
worst = 0  # largest violation of the anticommutation rule (an integer)
for a in range(8):
    for b in range(8):
        anti = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
        rule = 2 * eta[a] * (a == b) * unit  # 2 eta^ab times the unit matrix
        worst = max(worst, int(np.max(np.abs(anti - rule))))
pattern = all(np.array_equal(g.T, e * g) for g, e in zip(gamma, eta))
say(f"largest violation of the anticommutation rule over 64 pairs: {worst}")
check(worst == 0 and pattern,
      "g^a g^b + g^b g^a = 2 eta^ab exactly; g^T = eta g (symmetry pattern)",
      record=f"{ALGEBRA}, checks Clifford_relation and symmetry_pattern")
```

`np.eye(16, dtype=int)` is the 16×16 unit matrix of whole numbers. The two loops run over all $8 \times 8 = 64$ pairs (a, b); `@` multiplies matrices. `anti` is $\gamma^a\gamma^b + \gamma^b\gamma^a$ and `rule` is $2\eta^{ab}$ times the unit matrix: `(a == b)` is `True` (counted as 1) when $a = b$ and `False` (0) otherwise. `worst` keeps the largest absolute difference of any entry; with whole numbers it is exact. `pattern` checks the symmetry pattern seen in Figure 19b.1: `g.T` is the transpose, `zip(gamma, eta)` pairs each matrix with its sign, and the test is $(\gamma^a)^T = \eta^{aa}\gamma^a$. The reason, given in the text cell before the code: a signed permutation matrix P obeys $P^TP = \mathbf{1}$, and $\gamma^a\gamma^a = \eta^{aa}\mathbf{1}$, so $(\gamma^a)^T = (\gamma^a)^T\gamma^a\gamma^a\eta^{aa} = \eta^{aa}\gamma^a$. Out [4] prints the largest violation, 0, and the PASS line, which reproduces the checks Clifford_relation and symmetry_pattern of `wolfram-algebra.json`.

In [5], the chirality and the block map (Section 19.5).

```
Gamma = gamma[0]  # gamma^(x0) first ...
for a in range(1, 8):
    Gamma = Gamma @ gamma[a]  # ... times gamma^(x1), ..., gamma^(x7)
diagonal = np.diag([-1] * 8 + [1] * 8)
anticommutes = all(np.array_equal(Gamma @ g, -(g @ Gamma)) for g in gamma)
check(np.array_equal(Gamma, diagonal) and np.array_equal(Gamma @ Gamma, unit)
      and np.array_equal(Gamma, np.array(fixture["Gamma"])) and anticommutes,
      "Gamma = diag(-1 (8 times), +1 (8 times)), Gamma^2 = 1, Gamma g^a = -g^a Gamma",
      record=f"{ALGEBRA}, checks Gamma_diag and Gamma_anticommutes_with_gammas")
```

`Gamma` starts as $\gamma^{(x_0)}$ and is multiplied from the right by $\gamma^{(x_1)}, \ldots, \gamma^{(x_7)}$ in turn: the author's order of Section 19.5. `[-1] * 8 + [1] * 8` is the list of eight -1 followed by eight $+1$, and `np.diag` makes the diagonal matrix with these entries. `anticommutes` tests $\Gamma\gamma^a = -\gamma^a\Gamma$ for all eight. The check requires all four facts: Γ is this diagonal matrix, $\Gamma^2 = \mathbf{1}$, it equals the matrix stored in the record under the key `Gamma`, and it anticommutes with every gamma; it reproduces the checks Gamma_diag and Gamma_anticommutes_with_gammas of `wolfram-algebra.json`.

```
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
ENTRY = {"0": 0.0, "1": 1.0, "-1": -1.0, "I": 1j, "-I": -1j}  # the record's text
V = np.array([[ENTRY[x] for x in row]
              for row in ks["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
V = V / (2.0 * math.sqrt(2.0))  # the record stores 2 sqrt(2) V
labels = [tuple(int(x) for x in label) for label in ks["blockBasis"]["labels"]]
pauli1 = np.array([[0, 1], [1, 0]], dtype=complex)
pauli2 = np.array([[0, -1j], [1j, 0]])
pauli3 = np.array([[1, 0], [0, -1]], dtype=complex)
unitary = float(np.max(np.abs(V.conj().T @ V - np.eye(16))))
```

`ks` is the Kohn-Sham theory record. It stores the block basis as $2\sqrt{2}\,V$, entry by entry as text: `"0"`, `"1"`, `"-1"`, `"I"`, `"-I"`; the dictionary `ENTRY` translates each text into a number (Python writes the imaginary unit i as `1j`). The nested **list comprehension** `[[ENTRY[x] for x in row] for row in ...]` translates every entry of every row, `np.array` makes a 16×16 array of complex numbers, and the next line divides by $2\sqrt{2}$. `labels` is the list of the eight block labels (j, s_2, s_3) as **tuples** (fixed groups of numbers in round brackets) of whole numbers. `pauli1`, `pauli2`, `pauli3` are the Pauli matrices as complex arrays. `unitary` is the largest entry of $|V^\dagger V - \mathbf{1}|$: `.conj()` takes the complex conjugate of every entry and `.T` transposes, so `V.conj().T` is $V^\dagger$; for a unitary matrix this is 0 up to rounding.

```
worst_map, worst_form = 0.0, 0.0
for b, (j, s2, s3) in enumerate(labels):
    p = labels.index((-j, s2, s3))  # the partner block
    Vb, Vp = V[:, 2 * b:2 * b + 2], V[:, 2 * p:2 * p + 2]
    worst_map = max(worst_map, float(np.max(np.abs(Gamma @ Vb
                                                   - Vp @ (s2 * pauli2)))))
    g0 = Vb.conj().T @ gamma[0] @ Vb  # gamma^(x0) inside block b
    g04 = Vb.conj().T @ gamma[0] @ gamma[4] @ Vb  # gamma^(x0) gamma^(x4) there
    worst_form = max(worst_form, float(np.max(np.abs(g0 - pauli3))),
                     float(np.max(np.abs(g04 - j * pauli1))))
```

The loop runs over the eight blocks: `b` is the number of the block and `(j, s2, s3)` its label. `labels.index((-j, s2, s3))` finds the number `p` of the partner block. `V[:, 2 * b:2 * b + 2]` takes all rows and the two columns $2b$ and $2b+1$ (a **slice** `start:stop` runs up to, but not including, `stop`): the basis V_b of the block, and V_p likewise. `worst_map` keeps the largest entry of $|\Gamma V_b - V_p(s_2\sigma_2)|$: the statement $\Gamma V_b = V_p\, s_2\sigma_2$ of Section 19.5. `g0` is $V_b^\dagger \gamma^{(x_0)} V_b$, the block form of $\gamma^{(x_0)}$, and `g04` the block form of $\gamma^{(x_0)}\gamma^{(x_4)}$ (`gamma[4]` is $\gamma^{(x_4)}$); `worst_form` keeps their largest deviations from σ_3 and $j\sigma_1$, the two block forms that the proof of Section 19.5 uses.

```
say(f"largest deviations: unitarity {unitary:.1e}, block map {worst_map:.1e}, "
    f"forms gamma^(x0) = sigma3 and gamma^(x0) gamma^(x4) = j sigma1 "
    f"{worst_form:.1e}")
check(unitary < 1e-12 and worst_map < 1e-12 and worst_form < 1e-12,
      "Gamma maps block (j, s2, s3) onto block (-j, s2, s3) as s2 sigma2",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.Gamma_is_the_block_map")
```

`:.1e` writes a number in exponent form with one digit after the point. Out [5] shows the deviations: unitarity 1.1×10^{-16} (rounding of the division by $2\sqrt{2}$), block map exactly 0, block forms 1.1×10^{-16}; the check, with the tolerance 10^{-12}, reproduces T3.Gamma_is_the_block_map of `python-t3.json`. So the computer confirms $\tau = s_2$ of Section 19.5 for all eight blocks.

In [6], Γ in the block basis (Figure 19b.2).

```
in_blocks = V.conj().T @ Gamma @ V  # Gamma in the block basis
real_part = float(np.max(np.abs(in_blocks.real)))
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 4.4))
left.imshow(Gamma, cmap=THREE, vmin=-1, vmax=1, interpolation="nearest")
left.set_title("$\\Gamma = \\gamma^{(x_0)}\\gamma^{(x_1)}\\cdots\\gamma^{(x_7)}$",
               fontsize=10)
right.imshow(in_blocks.imag, cmap=THREE, vmin=-1, vmax=1, interpolation="nearest")
right.set_title("imaginary part of $V^\\dagger\\Gamma V$ (block basis)", fontsize=10)
```

$V^\dagger\Gamma V$ is Γ written in the block basis: its entry in row r and column c is the component along the r-th basis column of the image of the c-th one. `real_part` is the largest absolute real part of its entries. The figure has two axes side by side, `left` and `right` (the pair returned by `subplots(1, 2, ...)` is unpacked into two names). The left one shows Γ, the right one the imaginary part of $V^\dagger\Gamma V$ (`.imag`), both with the three-colour scale.

```
tags = [f"({j:+d},{s2:+d},{s3:+d})" for j, s2, s3 in labels]
right.set_xticks([2 * b + 0.5 for b in range(8)])
right.set_xticklabels(tags, rotation=90, fontsize=6)
right.set_yticks([2 * b + 0.5 for b in range(8)])
right.set_yticklabels(tags, fontsize=6)
for edge in range(1, 8):
    right.axhline(2 * edge - 0.5, color="0.6", lw=0.5)
    right.axvline(2 * edge - 0.5, color="0.6", lw=0.5)
for ax in (left, right):
    ax.grid(False)
left.set_xticks([0, 7, 8, 15])
left.set_yticks([0, 7, 8, 15])
fig.tight_layout()
```

`tags` are the block labels written like `(+1,+1,+1)`. On the right picture each label is put in the middle of its two rows and columns ($2b + 0.5$), turned by 90 degrees on the horizontal axis. `axhline` and `axvline` draw thin grey lines (`"0.6"` is a grey level between black 0 and white 1, `lw` the line width) on the borders between the blocks, so that the eight 2×2 squares can be seen. The grid is switched off in both pictures, and the left one gets ticks at 0, 7, 8 and 15, where Γ changes from -1 to $+1$.

```
save_figure(fig, "gamma_block_map",
            "Left: the chirality $\\Gamma$, the product of the eight real gamma "
            "matrices, as a heat map (blue $-1$, white $0$, orange $+1$; indices 0 "
            "to 15): $-1$ on the first eight diagonal places, $+1$ on the last "
            "eight. Right: the imaginary part of $\\Gamma$ in the Kohn-Sham block "
            "basis, with the block labels $(j, s_2, s_3)$ on the axes. The nonzero "
            "$2 \\times 2$ squares join each block to the block with the opposite "
            "$j$ and the same $s_2, s_3$, and each square is $\\pm\\sigma_2$: this "
            "is the map $(\\chi, j) \\to (\\sigma_2\\chi, -j)$ of theorem T3.")
report("largest real part of V^dagger Gamma V", f"{real_part:.1e}")
```

`save_figure` saves Figure 19b.2, and `report` prints that the real part is at most 2.2×10^{-17} (Out [6]), so the imaginary part is the whole matrix. What the student should see: on the left a diagonal line, blue in the upper half and orange in the lower half; on the right no square on the diagonal (no block is mapped into itself) but, in the upper-right and lower-left quarters, eight 2×2 squares, each joining a block (j, s_2, s_3) to $(-j, s_2, s_3)$. Each square holds the pattern of σ_2, whose imaginary part is -1 above and $+1$ below its diagonal, or its negative: the blocks with $s_2 = +1$ show $+\sigma_2$, those with $s_2 = -1$ show $-\sigma_2$. This is $G = s_2\sigma_2$ of Section 19.5.

In [7], Step 1 of the proof in sympy (Section 19.6).

```
y = sp.Symbol("y", real=True)  # the hidden coordinate
k, eps, vv = sp.symbols("k epsilon v_v", real=True)  # momentum, level, potential
Mf, kappa, vf = (sp.Function(name)(y) for name in ("M", "kappa", "v"))
chi = sp.Matrix([sp.Function("chi1")(y), sp.Function("chi2")(y)])  # any orbital
s1 = sp.Matrix([[0, 1], [1, 0]])  # the Pauli matrices, exactly
s2 = sp.Matrix([[0, -sp.I], [sp.I, 0]])
s3 = sp.Matrix([[1, 0], [0, -1]])
one = sp.eye(2)
```

`sp.Symbol("y", real=True)` makes an exact symbol y that sympy treats as a real number; `sp.symbols` makes several at once: k, ε and a constant potential v_v. `sp.Function(name)(y)` makes an **undetermined function** of y: sympy knows nothing about it except that it depends on y, so an identity proved with it holds for every function. `Mf`, `kappa`, `vf` are the functions $M(y)$, $\kappa(y)$, $v(y)$ (a generator expression in round brackets is unpacked into the three names), and `chi` is an arbitrary orbital $(\chi_1(y), \chi_2(y))$ as a sympy column. `s1`, `s2`, `s3` are the exact Pauli matrices (`sp.I` is i) and `one` the 2×2 unit matrix.

```
def h(j, mass, c):
    """h_j(mass) applied to the two-component function c."""
    return j * (-sp.I * s1 * c.diff(y) + mass * s2 * c + kappa * k * s3 * c) + vf * c
```

```
def is_zero(matrix):
    """True when every entry simplifies to exactly 0."""
    return all(sp.simplify(sp.expand(entry)) == 0 for entry in matrix)
```

`h` applies the block Hamiltonian h_j of Section 19.3, with the mass function `mass`, to a two-component function `c`: `c.diff(y)` is the derivative dc/dy, and the products `*` of sympy matrices are matrix products. `is_zero` is true when every entry of a matrix, multiplied out (`expand`) and simplified (`simplify`), is exactly 0.

```
rules = (s2 * s1 * s2 == -s1 and s2 * s2 * s2 == s2 and s2 * s3 * s2 == -s3)
maps = all(is_zero(h(-j, -Mf, s2 * chi) - s2 * h(j, Mf, chi)) for j in (1, -1))
control = not is_zero(h(-1, Mf, s2 * chi) - s2 * h(1, Mf, chi))  # no M -> -M
Ms = sp.Symbol("M", real=True)
jj = sp.Symbol("j", real=True)
```

`rules` tests the three conjugation rules $\sigma_2\sigma_1\sigma_2 = -\sigma_1$, $\sigma_2^3 = \sigma_2$, $\sigma_2\sigma_3\sigma_2 = -\sigma_3$ of Section 19.6 (`==` compares two exact matrices). `maps` tests, for both block types, the identity $h_{-j}(-M)(\sigma_2\chi) - \sigma_2 h_j(M)\chi = 0$ for the arbitrary functions: this is the statement "the image of every orbital is an orbital of the partner" in its strongest form, as an identity of differential operators. `control` is the negative control: without $M \to -M$ the difference is not zero. `Ms` and `jj` are a constant mass M and a symbolic type j for the next test.

```
def N(j, mass):
    """The first-order matrix N of d chi/dy = N chi (constant data)."""
    return mass * s3 - sp.Symbol("kappa") * k * s2 + sp.I * j * (eps - vv) * s1


ode = (s2 * N(jj, Ms) * s2 - N(-jj, -Ms)).applyfunc(sp.expand) == sp.zeros(2, 2)
say(f"conjugation rules: {rules}; block map for both j: {maps}; control fails "
    f"as it must: {control}; first-order form: {ode}")
```

`N` is the matrix $N_j(M) = M\sigma_3 - \kappa k\sigma_2 + ij(\varepsilon - v)\sigma_1$ of the first-order form, with a constant symbol κ. `ode` tests $\sigma_2 N_j(M)\sigma_2 - N_{-j}(-M) = 0$ with the symbolic j: `.applyfunc(sp.expand)` multiplies out every entry, and the result is compared with the 2×2 zero matrix. The `say` line prints the four truth values (Out [7]: all `True`).

```
check(rules and maps and control,
      "sigma2 h_j(M) sigma2 = h_(-j)(-M) for arbitrary M(y), kappa(y), v(y)",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.block_hamiltonian_map")
check(ode, "sigma2 N_j(M) sigma2 = N_(-j)(-M) for the first-order form",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check T3.ode_map")
```

Two checks, reproducing T3.block_hamiltonian_map and T3.ode_map of `python-t3.json`.

In [8], Step 2: the tip and the brane (Section 19.7).

```
theta = sp.Symbol("theta", real=True)


def Q(angle):
    """The tip matrix Q(angle) = cos(angle) sigma3 + sin(angle) sigma2."""
    return sp.cos(angle) * s3 + sp.sin(angle) * s2


tip = (s2 * Q(theta) * s2 - Q(sp.pi - theta)).applyfunc(sp.simplify) == sp.zeros(2, 2)
e1 = sp.Matrix([1, 0])  # the vector (1, 0)
zero2 = sp.zeros(2, 1)
tip_control = ((one - Q(0)) * e1 == zero2 and (one - Q(0)) * (s2 * e1) != zero2
               and (one - Q(sp.pi)) * (s2 * e1) == zero2)
brane = s2 * (one - s3) * s2 == one + s3
```

`theta` is a symbolic angle and `Q` the tip matrix $Q(\theta)$. `tip` tests $\sigma_2 Q(\theta)\sigma_2 = Q(\pi-\theta)$ for every θ (sympy simplifies $\cos(\pi-\theta)$ to $-\cos\theta$). `tip_control` is the control of Section 19.7: $(1,0)$ obeys the condition $\theta = 0$; its image $\sigma_2(1,0)$ violates it (`!=` means "is not equal") and obeys the condition $\theta = \pi$. `brane` tests $\sigma_2(1-\sigma_3)\sigma_2 = 1+\sigma_3$: the parities are exchanged.

```
say(f"tip: sigma2 Q(theta) sigma2 = Q(pi - theta): {tip}; control: {tip_control}; "
    f"brane parities exchanged: {brane}")
check(tip and tip_control and brane,
      "tip angle theta -> pi - theta and the brane parities are exchanged",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, checks "
             "T3.tip_condition_map and T3.brane_parities_exchanged")
```

Out [8] prints three `True` and the PASS line, reproducing two checks of `python-t3.json`.

In [9], Step 3: the densities of one orbital (Section 19.8).

```
a1, b1, a2, b2 = sp.symbols("a1 b1 a2 b2", real=True)
psi = sp.Matrix([a1 + sp.I * b1, a2 + sp.I * b2])  # an arbitrary complex orbital


def densities(j, c):
    """(n, s, t, q, c) of one orbital c of block type j."""
    dag = c.H  # the row of complex conjugates
    return [sp.expand((dag * m * c)[0, 0]) for m in
            (one, j * s2, j * s3, s3, j * s1)]
```

Four real symbols make an arbitrary complex orbital $\psi = (a_1 + ib_1, a_2 + ib_2)$ at one point. `densitie s` returns the five densities n_o, s_o, t_o, q_o, c_o without the common factor P: `c.H` is the conjugate transpose $c^\dagger$ (a row), and for each of the five matrices $\mathbf{1}$, $j\sigma_2$, $j\sigma_3$, σ_3, $j\sigma_1$ the product $c^\dagger X c$ is a 1×1 matrix whose only entry `[0, 0]` is multiplied out.

```
SIGNS = (1, -1, 1, -1, 1)  # expected: n, t, c even; s, q odd
signs_ok = all(sp.expand(after - sign * before) == 0
               for j in (1, -1)
               for after, before, sign in zip(densities(-j, s2 * psi),
```

```
                                            densities(j, psi), SIGNS))
say(f"n = {densities(1, psi)[0]},  s (j = +1) = {densities(1, psi)[1]}")
check(signs_ok, "under the map n, t, c keep their sign, s and q change it",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.orbital_densities")
```

`SIGNS` lists the expected signs of Section 19.8. For both j the test compares each density of the image $(\sigma_2\psi, -j)$ with the sign times the density of (ψ, j); `zip` runs through the three lists side by side. The `say` line prints two of the densities as formulas (Out [9]): $n_o = a_1^2 + a_2^2 + b_1^2 + b_2^2$ and $s_o = 2a_1b_2 - 2a_2b_1$ for $j = +1$ (sympy writes powers as `**`). For a real-form orbital, $a_1 = a$, $b_2 = b$ and $b_1 = a_2 = 0$, these are $a^2 + b^2$ and $2ab$, as in Section 19.4. The check reproduces T3.orbital_densities.

In [10], Step 4 of the record: the mean field (Section 19.8).

```
potentials = ks["exchange"]["kohnShamPotentials"]
cM = sp.Rational(potentials["Meff_coefficient_of_lambda_S"])  # 15/16
cv = sp.Rational(potentials["vv_coefficient_of_lambda_n"])  # -1/16
m, lam, S, n = sp.symbols("m lambda S n", real=True)


def M_eff(mass, coupling, scalar):
    """The effective mass m + cM lambda S of the record."""
    return mass + cM * coupling * scalar
```

The two coefficients are read from the Kohn-Sham record as texts, `"15/16"` and `"-1/16"`, and `sp.Rational` turns them into exact fractions. m, λ, S, n are symbols, and `M_eff` builds $M_{\text{eff}} = m + c_M\lambda S$ for any bare mass, coupling and scalar density.

```
e_int = sp.Rational(15, 32) * lam * S ** 2 - sp.Rational(1, 32) * lam * n ** 2
partner = sp.expand(M_eff(-m, lam, -S) + M_eff(m, lam, S)) == 0
wrong = sp.expand(M_eff(-m, -lam, -S) + M_eff(m, lam, S)) != 0
even = sp.expand(e_int.subs(S, -S) - e_int) == 0
derivatives = (sp.expand(sp.diff(e_int, S) - (M_eff(m, lam, S) - m)) == 0
               and sp.expand(sp.diff(e_int, n) - cv * lam * n) == 0)
```

`e_int` is $e_{\text{int}} = \frac{15}{32}\lambda S^2 - \frac{1}{32}\lambda n^2$. `partner` tests $M_{\text{eff}}(-m, +\lambda, -S) = -M_{\text{eff}}(m, \lambda, S)$; `wrong` tests that $(-m, -\lambda)$ does NOT give it; `even` tests $e_{\text{int}}(n, -S) = e_{\text{int}}(n, S)$ (`.subs(S, -S)` replaces S by $-S$); `derivatives` tests $\partial e_{\text{int}}/\partial S = M_{\text{eff}} - m$ and $\partial e_{\text{int}}/\partial n = v_v$ (`sp.diff` differentiates).

```
say(f"cM = {cM}, cv = {cv}; (-m, +lambda) works: {partner}; (-m, -lambda) fails: "
    f"{wrong}; e_int even in S: {even}; derivatives: {derivatives}")
check(partner and wrong and even and derivatives and (cM, cv) == (
    sp.Rational(15, 16), sp.Rational(-1, 16)),
    "M_eff(-m, +lambda, -S) = -M_eff(m, lambda, S); (-m, -lambda) fails",
    record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
           "T3.mean_field_map")
```

Out [10] prints the two coefficients and four `True`; the check also requires the coefficients to be exactly $15/16$ and $-1/16$, and reproduces T3.mean_field_map.

In [11], the mean-field map as a picture (Figure 19b.3).

```
S_values = np.linspace(-1.0, 1.0, 201)
lam_demo, m_demo, cM_value = 0.8, 1.0, float(cM)
need = -(m_demo + cM_value * lam_demo * S_values)  # -M_eff(m, lambda, S)
give_partner = -m_demo + cM_value * lam_demo * (-S_values)  # (-m, +lambda) at -S
give_wrong = -m_demo + cM_value * (-lam_demo) * (-S_values)  # (-m, -lambda) at -S
```

`np.linspace(-1.0, 1.0, 201)` is an array of 201 equally spaced values of S from -1 to 1. The coupling 0.8 is deliberately large so that the slopes are visible. The three arrays are, for every S, the mass the image needs, $-M_{\text{eff}}(m, \lambda, S)$, the mass the partner $(-m, +\lambda)$ gives to the image density $-S$, and the mass $(-m, -\lambda)$ would give (numpy computes each formula for all 201 values at once).

```
fig, ax = plt.subplots()
ax.plot(S_values, need, color="#2a78d6", lw=3.0,
        label="needed: $-M_{eff}(m, \\lambda, S)$")
ax.plot(S_values, give_partner, color="#eb6834", lw=1.6, ls="--",
        label="partner $(-m, +\\lambda)$ at $-S$")
ax.plot(S_values, give_wrong, color="#eda100", lw=1.6, ls=":",
        label="$(-m, -\\lambda)$ at $-S$")
ax.set_xlabel("scalar density $S$ (units of $|m|^7$)")
ax.set_ylabel("effective mass (units of $|m|$)")
ax.legend(fontsize=8, loc="upper right")
```

`plot` draws a curve through the points; `lw` is the line width, `ls` the line style (two hyphens in quotes mean dashed, a colon `":"` dotted), and `label` the text for the **legend** (the box that names the curves), which `legend` draws in the upper right corner. The axis labels name the quantities and units.

```
save_figure(fig, "mean_field_map",
            "Step 4 of the proof of T3: the effective mass that the image state "
            "needs, $-M_{eff}(m, \\lambda, S)$ (blue), against the scalar density "
            "$S$, with $m = 1$ and an exaggerated coupling $\\lambda = 0.8$ "
            "(units of $|m|$ and $|m|^7$). The partner universe with the bare mass "
            "$-m$ and the same coupling $+\\lambda$ gives exactly this mass to the "
            "image density $-S$ (orange dashed, on top of the blue line); the "
            "universe with $(-m, -\\lambda)$ gives a line with the opposite slope "
            "(yellow dotted), which agrees only at $S = 0$.")
check(float(np.max(np.abs(need - give_partner))) < 1e-15
      and float(np.max(np.abs(need - give_wrong))) > 1.0,
      "the drawn lines: the partner gives the needed mass, (-m, -lambda) does not")
```

`save_figure` saves Figure 19b.3; the check confirms what the picture shows. What the student should see: a thick blue line falling from -0.25 at $S = -1$ to -1.75 at $S = 1$ (slope $-\frac{15}{16} \cdot 0.8 = -0.75$), the orange dashed line exactly on top of it, and the yellow dotted line rising with the opposite slope, crossing the blue one only at $S = 0$, where both are $-m = -1$. (The caption's "Step 4" is the numbering of the record's proof list, where the mean field belongs to the third item; in this chapter it is part of Step 3.)

In [12], the characteristic functions at zero momentum (Section 19.12).

```
Mp, L = sp.symbols("M L", positive=True)
yy = sp.Symbol("yy", real=True)
p = sp.sqrt(Mp ** 2 - eps ** 2)


def propagator(j, mass):
    """exp(N yy) for k = 0, v = 0: cosh(p yy) 1 + sinh(p yy)/p N."""
    N0 = mass * s3 + sp.I * j * eps * s1
    return sp.cosh(p * yy) * one + sp.sinh(p * yy) / p * N0
```

M and L are positive symbols, `yy` the position, and $p = \sqrt{M^2 - \varepsilon^2}$. `propagator` is the matrix $e^{Ny} = \cosh(py)\mathbf{1} + \frac{\sinh(py)}{p} N$ of Section 19.12 with $N = M\sigma_3 + ij\varepsilon\sigma_1$, for any type and mass.

```
e2 = sp.Matrix([0, 1])  # the vector (0, 1)
at_tip = {yy: -L}
D = {}  # characteristic functions, for j = +1
D["A even"] = (propagator(1, Mp) * e1)[1].subs(at_tip)  # chi2(-L)
D["A odd"] = (propagator(1, Mp) * e2)[1].subs(at_tip)
D["B even"] = (propagator(-1, -Mp) * e1)[0].subs(at_tip)  # chi1(-L)
D["B odd"] = (propagator(-1, -Mp) * e2)[0].subs(at_tip)
D["C even"] = (propagator(1, -Mp) * e1)[1].subs(at_tip)  # chi2(-L)
D["C odd"] = (propagator(1, -Mp) * e2)[1].subs(at_tip)
for name in D:
    say(f"D({name}) = {sp.simplify(D[name])}")
```

The six entries of the table of Section 19.12, with $j = +1$: the propagator applied to the start at the brane, $(1, 0)$ for even or $(0, 1)$ for odd, evaluated at the tip ($y = -L$, the substitution `at_tip`), and the component that the tip condition must make zero: the second (index 1) for $\theta = 0$, the first (index 0) for $\theta = \pi$. A has $(M, +1, \theta = 0)$, B $(-M, -1, \theta = \pi)$, C $(-M, +1, \theta = 0)$. The loop prints them (Out [12]); sympy writes `I` for i, `sqrt` for the square root and `epsilon` for ε. Compare with the table of Section 19.12: D(A even) $= -i\varepsilon \sinh(pL)/p$, D(A odd) $= D$(B even) $= \cosh(pL) + M \sinh(pL)/p$, D(B odd) $= +i\varepsilon \sinh(pL)/p$, D(C even) $=$ D(A even), D(C odd) $= \cosh(pL) - M \sinh(pL)/p$.

```
pairs_equal = (sp.simplify(D["A even"] + D["B odd"]) == 0
               and sp.simplify(D["A odd"] - D["B even"]) == 0)
control_differs = (sp.simplify(D["C even"] - D["A even"]) == 0
                   and sp.simplify(D["C odd"] - D["A odd"]) != 0)
check(pairs_equal and control_differs,
      "k = 0: A and B have equal characteristic functions; C differs (odd sector)",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.exact_k0_spectra")
```

`pairs_equal` tests A even $= -$B odd and A odd $=$ B even, identically in ε, M, L; `control_differs` tests C even $=$ A even and C odd $\neq$ A odd. The check reproduces T3.exact_k0_spectra.

In [13], the zeros of the characteristic functions.

```
M_BARE, L_TIP = 1.0, 3.0  # the mass |m| and the tip distance L of the record
```

```
def C_and_Sigma(e):
    """cosh(pL) and sinh(pL)/p for the level(s) e, real for every real e."""
    pc = np.sqrt(M_BARE ** 2 - np.asarray(e, dtype=float) ** 2 + 0j)  # complex p
    small = np.abs(pc) < 1e-12  # p = 0: sinh(pL)/p -> L
    safe = np.where(small, 1.0, pc)
    sigma = np.where(small, L_TIP, np.sinh(safe * L_TIP) / safe)
    return np.cosh(pc * L_TIP).real, sigma.real
```

From here on the numbers $M = 1$, $L = 3$ of the record are used. `C_and_Sigma` computes the two building blocks $\cosh(pL)$ and $\Sigma = \sinh(pL)/p$ for one level or an array of levels. Adding `0j` makes the number under the square root complex, so that for $\varepsilon^2 > M^2$ numpy returns $p = ir$ instead of an error; as Section 19.12 showed, $\cosh(irL) = \cos(rL)$ and $\sinh(irL)/(ir) = \sin(rL)/r$ are real, so `.real` keeps everything. At $p = 0$ the quotient $\sinh(pL)/p$ has the limit L; `np.where(small, a, b)` takes `a` where `small` is true and `b` elsewhere, and `safe` avoids a division by zero.

```
def f_A_even(e):
    return np.asarray(e) * C_and_Sigma(e)[1]


def f_A_odd(e):
    c, sig = C_and_Sigma(e)
    return c + M_BARE * sig


def f_C_odd(e):
    c, sig = C_and_Sigma(e)
    return c - M_BARE * sig
```

The three different characteristic functions, without the constant factor $-ij$ of the even one: $\varepsilon\Sigma$, $\cosh(pL) + M\Sigma$ and $\cosh(pL) - M\Sigma$.

```
def zeros(f, top=4.0, points=40001):
    """All sign changes of f on (0, top], each refined by 80 bisection steps."""
    grid = np.linspace(1e-9, top, points)
    values = f(grid)
    found = []
    for i in np.nonzero(np.sign(values[:-1]) * np.sign(values[1:]) < 0)[0]:
        low, high = grid[i], grid[i + 1]
        for _ in range(80):
            middle = 0.5 * (low + high)
            if np.sign(f(middle)) == np.sign(f(low)):
                low = middle
            else:
                high = middle
        found.append(0.5 * (low + high))
    return found
```

`zeros` finds every zero of a function between 10^{-9} and `top`. It evaluates the function on 40001 grid points; `values[:-1]` are all values but the last and `values[1:]` all but the first, so their signs multiplied are negative exactly where the sign changes between two neighbouring points, and

`np.nonzero(...)[0]` lists those positions. Each sign change is refined by **bisection**: halve the interval, keep the half whose ends still have opposite signs, 80 times (the interval then is 2^{-80} times its first length, far below the rounding of the computer). The middle of the last interval is the zero.

```
levels = {"A even": [0.0] + zeros(f_A_even), "A odd": zeros(f_A_odd),
          "C odd": zeros(f_C_odd)}
for name, values in levels.items():
    say(f"{name}: " + ", ".join(f"{v:.10f}" for v in values))
```

The levels of the three sectors; the zero mode $\varepsilon = 0$ of the even sector is put in front by hand (the grid starts just above 0). `", ".join(...)` writes the numbers separated by commas, each with ten digits. Out [13] shows the three ladders of the table in Section 19.12.

```
SPECTRUM = "Revision/kohn_sham/results/spectrum/free-k0-analytic.csv"
with repository_file(SPECTRUM).open(encoding="utf-8", newline="") as handle:
    rows = [r for r in csv.DictReader(handle)
            if float(r["m"]) == 1.0 and float(r["L"]) == 3.0]
recorded = {parity: sorted(float(r["eps_analytic"]) for r in rows
                           if r["parity"] == parity
                           and 0.0 <= float(r["eps_analytic"]) < 4.0)
            for parity in ("even", "odd")}
```

The record's table of the analytic free levels is read with `csv.DictReader`, which gives each row as a dictionary from the column names to the texts (`with ... as handle:` opens the file and closes it again at the end of the block). Only the rows with $m = 1$ and $L = 3$ are kept. `recorded` holds, for each parity, the sorted recorded levels between 0 and 4.

```
worst = max(max(abs(x - y) for x, y in zip(levels["A even"], recorded["even"])),
            max(abs(x - y) for x, y in zip(levels["A odd"], recorded["odd"])))
report("largest difference of the levels of A from the record", f"{worst:.1e}")
check(len(levels["A even"]) == len(recorded["even"])
      and len(levels["A odd"]) == len(recorded["odd"]) and worst < 1e-12,
      "the k = 0 levels of A are the analytic levels of the record",
      record=f"{SPECTRUM}, rows m = 1, L = 3")
```

`worst` is the largest difference between our levels of A and the recorded ones: 8.9×10^{-16} (Out [13]). The check also requires the same number of levels.

In [14], the control's level in the gap (Figure 19b.4).

```
eps_b = levels["C odd"][0]  # the first zero: inside the gap?
q_b = math.sqrt(M_BARE ** 2 - eps_b ** 2)
two_forms = (abs(math.tanh(q_b * L_TIP) - q_b / M_BARE) < 1e-12
             and abs(M_BARE / math.cosh(q_b * L_TIP) - eps_b) < 1e-12)
report("sub-gap level of the control eps_b", f"{eps_b:.12f}", "|m|")
above = levels["C odd"][1:]  # the control's odd levels above the gap
distinct = min(abs(x - y) for x in above for y in levels["A odd"]) > 1e-3
check(0.0 < eps_b < M_BARE and two_forms and distinct,
      "only the control has a level in the gap, eps_b = M/cosh(qL), tanh(qL) = q/M")
```

`eps_b` is the lowest odd level of the control and `q_b` $= \sqrt{M^2 - \varepsilon_b^2}$. `two_forms` tests the two statements of Section 19.12: $\tanh(qL) = q/M$ and $\varepsilon_b = M/\cosh(qL)$. `above` holds the other odd levels of C, and `distinct` tests that none of them is within 10^{-3} of an odd level of A. The check requires $0 < \varepsilon_b < M$ (inside the gap) as well. Out [14] prints $\varepsilon_b = 0.100851121375$.

```
grid = np.linspace(0.0, 4.0, 2001)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
left.plot(grid, f_A_even(grid), color="#2a78d6", lw=2.0,
          label="$\\varepsilon\\,\\sinh(pL)/p$ (even A = odd B = even C)")
left.plot(levels["A even"], np.zeros(len(levels["A even"])), "o", color="#2a78d6")
right.plot(grid, f_A_odd(grid), color="#2a78d6", lw=2.0,
           label="A odd $= $ B even: $\\cosh(pL) + M\\sinh(pL)/p$")
right.plot(grid, f_C_odd(grid), color="#1baf7a", lw=1.6, ls=":",
           label="C odd: $\\cosh(pL) - M\\sinh(pL)/p$")
right.plot(levels["A odd"], np.zeros(len(levels["A odd"])), "o", color="#2a78d6")
right.plot(levels["C odd"], np.zeros(len(levels["C odd"])), "^", color="#1baf7a")
```

The left picture draws the even function on 2001 points from 0 to 4 and marks its zeros, the levels, with dots on the axis (`"o"`; `np.zeros(n)` gives the height 0 for every dot). The right picture draws the odd functions of A and of the control C and marks their zeros with dots and triangles (`"^"`).

```
for ax in (left, right):
    ax.axhline(0.0, color="0.3", lw=0.8)
    ax.axvline(M_BARE, color="0.5", lw=0.8, ls="--")  # the edge of the mass gap
    ax.set_xlabel("level $\\varepsilon$ (units of $|m|$)")
    ax.set_ylim(-4.0, 6.0)
    ax.legend(fontsize=7, loc="upper right")
left.set_ylabel("characteristic function")
fig.tight_layout()
save_figure(fig, "characteristic_functions",
            "The characteristic functions of the zero-momentum spectra for "
            "$M = 1$, $L = 3$ (horizontal axis: the level $\\varepsilon$ in units "
            "of $|m|$; the dashed vertical line is the edge $\\varepsilon = M$ of "
            "the mass gap). Left: the function of the even sector of A, which is "
            "also that of the odd sector of the partner B and of the even sector of "
            "the control C; its zeros (dots) are $0$ and $\\sqrt{M^2 + "
            "(n\\pi/L)^2}$. Right: the odd sector of A (equal to the even sector of "
            "B) and the odd sector of the control C. Only C has a zero inside the "
            "gap, the level $\\varepsilon_b = 0.1009$, and its other zeros differ "
            "from those of A.")
```

In both pictures a horizontal line marks zero and a dashed vertical line the edge of the mass gap $\varepsilon = M$; the vertical range is cut to -4 to 6 because the functions grow like cosh inside the gap. `save_figure` saves Figure 19b.4. What the student should see: on the left a curve that is positive inside the gap (its only zero there is the zero mode at $\varepsilon = 0$) and oscillates like $\varepsilon \sin(rL)/r$ outside, crossing zero at 1.448, 2.321, 3.297; on the right the blue curve of A, which starts at $\cosh(3) + \sinh(3) = e^3$ (above the drawn range), stays above zero in the whole gap and crosses zero only above it, and the aqua dotted curve of the control, which starts at $\cosh(3) - \sinh(3) = e^{-3} = 0.05$, falls through zero at $\varepsilon_b = 0.1009$ inside the gap, and crosses zero above the gap at other places than the blue curve.

In [15], shooting: the functions `brane_value` and `find_levels`.

```
H, STEPS = 1.0, 3000  # the constant H and the number of RK4 steps


def brane_value(e, kk, mass, j, tip, odd):
    """Shoot from the tip y = -L to the brane y = 0 at the slice a4,0 = 0.  All
    arguments are numpy arrays of equal shape (one entry per problem); returns
    a(0) where odd is 1 and b(0) where odd is 0."""
    a = np.where(tip == 0.0, 1.0, 0.0)  # theta = 0: (a, b) = (1, 0) at the tip
    b = 1.0 - a  # theta = pi: (a, b) = (0, 1)
    step = L_TIP / STEPS
```

Shooting finds levels without a formula: start at the tip with the values that the tip condition allows, integrate the real form of Section 19.3 to the brane, and ask whether the brane condition holds. `brane_value` does this for many problems at once: every argument is a numpy array, and each array position is one problem (its level `e`, momentum `kk`, mass, type `j`, tip angle and parity). At the tip, $\theta = 0$ demands $b(-L) = 0$, so the start is $(a, b) = (1, 0)$; $\theta = \pi$ demands $a(-L) = 0$, so the start is $(0, 1)$ (the overall size does not matter, the equations are linear). The step length is $L/3000 = 0.001$.

```
    def slope(yv, a, b):
        w = math.exp(-H * yv) * kk  # kappa(y) k
        return mass * a - (w + j * e) * b, (j * e - w) * a - mass * b
```

`slope` is the right side of the real form at the point `yv`, with $v = 0$ and the momentum weight $\kappa k = e^{-Hy}k$ at the slice $a_{4,0} = 0$: it returns $da/dy = Ma - (\kappa k + j\varepsilon)b$ and $db/dy = (j\varepsilon - \kappa k)a - Mb$.

```
    for i in range(STEPS):
        yv = -L_TIP + i * step
        ka1, kb1 = slope(yv, a, b)
        ka2, kb2 = slope(yv + step / 2, a + step / 2 * ka1, b + step / 2 * kb1)
        ka3, kb3 = slope(yv + step / 2, a + step / 2 * ka2, b + step / 2 * kb2)
        ka4, kb4 = slope(yv + step, a + step * ka3, b + step * kb3)
        a = a + step / 6 * (ka1 + 2 * ka2 + 2 * ka3 + ka4)
        b = b + step / 6 * (kb1 + 2 * kb2 + 2 * kb3 + kb4)
    return np.where(odd == 1.0, a, b)
```

The loop makes 3000 steps of the classical **Runge-Kutta method** of fourth order (RK4, Chapter 2): from the slope at the start of a step, two slopes at its middle and one at its end, the step adds the weighted mean $\frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ times the step length. At the brane the function returns the component that must vanish there: $a(0)$ for the odd parity, $b(0)$ for the even parity. A level is a value of ε at which this **brane value** is zero.

```
def find_levels(problems, top, points=2001, rounds=16, first_only=False):
    """problems: rows (k, mass, j, tip, odd).  Returns (row index, level) for every
    sign change of brane_value on (0, top] (only the lowest one per row when
    first_only is True)."""
```

```
    P = np.array(problems, dtype=float)
    grid = np.linspace(1e-9, top, points)
    columns = [np.repeat(P[:, c:c + 1], points, axis=1) for c in range(5)]
    values = brane_value(np.tile(grid, (len(P), 1)), *columns)
```

`find_levels` takes a list of problems, each a row $(k, M, j, \theta, \text{odd})$, and a top energy. `P` is the table of the problems, one row each. The scan grid has `points` energies between 10^{-9} and `top`. To shoot all problems at all grid energies in one call, two tables of the same shape are made: `np.tile(grid, (len(P), 1))` repeats the grid once per problem (one row per problem), and `np.repeat(P[:, c:c + 1], points, axis=1)` repeats column `c` of the problem table along each row; `*columns` passes the five tables as the five last arguments. `values` holds the brane value of every problem at every grid energy.

```
    rows, cols = np.nonzero(np.sign(values[:, :-1]) * np.sign(values[:, 1:]) < 0)
    if first_only:  # keep the first sign change of every row
        keep = np.concatenate(([True], rows[1:] != rows[:-1]))
        rows, cols = rows[keep], cols[keep]
    lo, hi = grid[cols], grid[cols + 1]  # brackets with a sign change
    f_lo, f_hi = values[rows, cols], values[rows, cols + 1]
    data = [P[rows, c] for c in range(5)]  # the problem of every root
```

The sign changes between neighbouring grid energies are found as in In [13], now in every row at once: `rows` and `cols` list the problem and the grid position of each sign change, in order. With `first_only`, only the first sign change of each problem is kept (an entry is kept when its row differs from the row of the entry before it). Each sign change gives a **bracket** `lo`, `hi` with the brane values `f_lo`, `f_hi` of opposite signs, and `data` lists the problem data of each bracket.

```
    kept = np.zeros(len(rows))  # which end was kept last: +1 high, -1 low
    new = lo
    for _ in range(rounds):
        new = (lo * f_hi - hi * f_lo) / (f_hi - f_lo)  # zero of the straight line
        f_new = brane_value(new, *data)
        move_low = np.sign(f_new) == np.sign(f_lo)  # the zero lies above new
        f_hi = np.where(move_low & (kept == 1), f_hi / 2, f_hi)  # Illinois rule
        f_lo = np.where(~move_low & (kept == -1), f_lo / 2, f_lo)
        lo, f_lo = np.where(move_low, new, lo), np.where(move_low, f_new, f_lo)
        hi, f_hi = np.where(move_low, hi, new), np.where(move_low, f_hi, f_new)
        kept = np.where(move_low, 1, -1)
    return rows, new
```

All brackets are refined together, 16 rounds, by the **Illinois method**. In each round the new point is the zero of the straight line through $(\text{lo}, f_{\text{lo}})$ and $(\text{hi}, f_{\text{hi}})$: $\text{new} = (\text{lo}\, f_{\text{hi}} - \text{hi}\, f_{\text{lo}})/(f_{\text{hi}} - f_{\text{lo}})$. Its brane value decides which end moves: if it has the sign of the low end, the zero lies above `new` and the low end moves there (`move_low`), otherwise the high end moves. A plain straight-line rule can keep one end fixed for many rounds and converge slowly; the Illinois rule halves the stored value at an end that is kept twice in a row (the high end when the low end moved again, `& (kept == 1)`; the low end in the opposite case, `~` meaning "not"), which pulls the next straight line toward that end. `kept` remembers which end was kept. The function returns the problem number and the level of every root.

```
say("brane_value and find_levels defined")
```

The cell prints one line (Out [15]).

In [16], the six sectors at k = 0 by shooting.

```
SECTORS = {  # name: (mass, j, tip angle, odd)
    "A even": (1.0, 1.0, 0.0, 0.0), "A odd": (1.0, 1.0, 0.0, 1.0),
    "B even": (-1.0, -1.0, math.pi, 0.0), "B odd": (-1.0, -1.0, math.pi, 1.0),
    "C even": (-1.0, 1.0, 0.0, 0.0), "C odd": (-1.0, 1.0, 0.0, 1.0)}
names_k0 = list(SECTORS)
rows, found = find_levels([(0.0, *SECTORS[name]) for name in names_k0], top=4.0)
shot = {name: sorted(found[rows == r]) for r, name in enumerate(names_k0)}
```

`SECTORS` gives the data of the six problems: A has the mass +1, type +1 and tip angle 0; B the mass −1, type −1 and tip angle π; C the mass −1, type +1 and tip angle 0; each in both parities. `find_levels` shoots all six at $k = 0$ up to $\varepsilon = 4$. `found[rows == r]` picks the levels of the problem `r` (a comparison of an array with a number gives an array of truth values, which selects the matching entries), and `shot` stores them sorted under the sector's name.

```
zero_mode = {name: float(brane_value(np.zeros(1), np.zeros(1),
                                     *(np.full(1, x) for x in SECTORS[name]))[0])
             for name in ("A even", "B odd", "C even")}
for name in ("A even", "B odd", "C even"):
    shot[name] = [0.0] + shot[name]  # the zero mode at eps = 0
expected = {"A even": levels["A even"], "A odd": levels["A odd"],
            "B even": levels["A odd"], "B odd": levels["A even"],
            "C even": levels["A even"], "C odd": levels["C odd"]}
```

The zero modes sit exactly at $\varepsilon = 0$, where the scan cannot see a sign change. So `zero_mode` evaluates the brane value at $\varepsilon = 0$, $k = 0$ in the three sectors that have one (`np.full(1, x)` is an array of length one holding `x`), and the zero level is added to their lists. `expected` says what each sector must give according to Section 19.12: A its own levels; B even the levels of A odd and B odd those of A even (T3 exchanges the parities); C even those of A even; C odd the control's own levels.

```
worst_shot = 0.0
for name in names_k0:
    say(f"{name}: " + ", ".join(f"{e:.10f}" for e in shot[name]))
    if len(shot[name]) != len(expected[name]):
        worst_shot = math.inf  # a level is missing or extra
    else:
        worst_shot = max(worst_shot, max(abs(x - y) for x, y in
                                         zip(shot[name], expected[name])))
report("largest difference shooting - formula at k = 0", f"{worst_shot:.1e}")
check(worst_shot < 1e-9 and all(v == 0.0 for v in zero_mode.values()),
      "shooting finds exactly the formula levels of A, B and C at k = 0")
```

The loop prints the six ladders (Out [16]) and keeps the largest difference from the formula levels; a missing or extra level would make it infinite (`math.inf`). The largest difference is 5.9×10^{-12}, the

error of 3000 Runge-Kutta steps, and the brane values at $\varepsilon = 0$ are exactly 0: at $\varepsilon = 0$, $k = 0$ the starting value has no component that could grow into the forbidden one (for example, for A even b stays exactly 0 along the whole integration).

In [17], the ladders (Figure 19b.5).

```
COLOUR = {"A": "#2a78d6", "B": "#eb6834", "C": "#1baf7a"}
fig, ax = plt.subplots(figsize=(7.0, 5.0))
for x, kind in enumerate("ABC"):
    for parity, style in (("even", "-"), ("odd", "--")):
        for e in shot[f"{kind} {parity}"]:
            for sign in ((1,) if e == 0.0 else (1, -1)):
                ax.plot([x - 0.3, x + 0.3], [sign * e, sign * e],
                        color=COLOUR[kind], ls=style, lw=2.0)
ax.axhspan(-M_BARE, M_BARE, color="0.9", zorder=0)  # the mass gap
```

`COLOUR` gives each universe its colour: blue A, orange B, aqua C. For each universe (at the horizontal positions 0, 1, 2) and each parity (solid lines for even, dashed for odd) every level is drawn as a short horizontal line, and, except the zero mode, also its negative: at $k = 0$ the levels come in pairs $\pm\varepsilon$ (the characteristic functions are even or odd in ε). `axhspan` shades the mass gap $-1 < \varepsilon < 1$ in light grey behind everything (`zorder=0`).

```
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(["A: $+m$, $\\theta = 0$", "B: $-m$, $\\theta = \\pi$",
                    "C: $-m$, $\\theta = 0$ (control)"])
ax.set_xlim(-0.6, 2.6)
ax.set_ylabel("level $\\varepsilon$ at $k = 0$ (units of $|m|$)")
ax.grid(axis="x", visible=False)
save_figure(fig, "k0_ladders",
            "The levels at zero momentum of the universes A (mass $+m$, tip angle "
            "0), B (mass $-m$, tip angle $\\pi$, the T3 partner) and C (mass $-m$, "
            "tip angle 0, the control), for $m = 1$, $L = 3$, no interaction, found "
            "by shooting (vertical axis: the level in units of $|m|$; solid lines "
            "even parity, dashed lines odd parity; grey band: the mass gap "
            "$|\\varepsilon| < m$). A and B have the same ladder with the parities "
            "exchanged; the control has different odd levels and the pair "
            "$\\pm\\varepsilon_b$ inside the gap.")
```

The three ladders are labelled under the horizontal axis, the vertical grid lines are switched off, and `save_figure` saves Figure 19b.5. What the student should see: the ladders of A and B at exactly the same heights, but where A has a solid line B has a dashed one and the other way round (the exchanged parities); the zero mode at 0; nothing else inside the grey gap for A and B; the control C with the zero mode, two dashed lines at ± 0.1009 inside the gap, and dashed lines above the gap at heights different from those of A.

```
ladder_A = np.array(sorted(shot["A even"] + shot["A odd"]))
ladder_B = np.array(sorted(shot["B even"] + shot["B odd"]))
check(len(ladder_A) == len(ladder_B)
      and float(np.max(np.abs(ladder_A - ladder_B))) < 1e-12,
      "the shooting ladders of A and B agree level by level")
```

The two complete ladders (both parities, sorted) of A and B are compared level by level; they agree to better than 10^{-12} although they were shot with different masses, types, tip conditions and parities.

In [18], the brane band at small momenta.

```
BAND = {"A": (1.0, 1.0, 0.0, 0.0), "B": (-1.0, -1.0, math.pi, 1.0),
        "C": (-1.0, 1.0, 0.0, 0.0)}  # mass, j, tip angle, odd
k_plot = np.linspace(0.0, 0.25, 26)[1:]  # 25 momenta for the figure
problems = [(kk, *BAND[kind]) for kind in "ABC" for kk in k_plot]
rows, found = find_levels(problems, top=2.0, points=401, first_only=True)
band = {kind: found[i * len(k_plot):(i + 1) * len(k_plot)]
        for i, kind in enumerate("ABC")}
```

`BAND` gives the sector of the zero mode of each universe: A type $+1$ even, B type -1 odd (its image), C type $+1$ even. `k_plot` are the 25 momenta $0.01, 0.02, \ldots, 0.25$ (26 points from 0 to 0.25 without the first). The 75 problems are shot together, and only the lowest positive level of each is kept (`first_only=True`); a scan with 401 points up to $\varepsilon = 2$ suffices. `band` cuts the 75 levels into the three lists of 25, in the order in which the problems were made.

```
band_gap = float(np.max(np.abs(band["A"] - band["B"])))
report("largest |eps_A - eps_B| on the band", f"{band_gap:.1e}")
check(len(rows) == 3 * len(k_plot) and band_gap < 1e-10
      and float(np.min(np.abs(band["A"] - band["C"]))) > 1e-3,
      "the band of B equals that of A at every momentum; the control differs")
```

The largest difference between the bands of A and B is exactly 0 (Out [18]): with σ_2 the shooting of B performs, step by step, the same arithmetic as that of A with a and b exchanged. The check also requires a level for every problem and a difference larger than 10^{-3} between A and C at every momentum.

In [19], the slopes at zero momentum.

```
small = 1e-4
problems = [(kk, *BAND[kind]) for kind in "ABC" for kk in (small, 2 * small)]
rows, found = find_levels(problems, top=0.01, points=401, first_only=True)
slopes = {kind: (4 * found[2 * i] / small - found[2 * i + 1] / (2 * small)) / 3
          for i, kind in enumerate("ABC")}
```

The band is an odd function of k, $\varepsilon(k) = ck + dk^3 + \ldots$, so $\varepsilon(k)/k = c + dk^2 + \ldots$. At k and $2k$: $\varepsilon(k)/k = c + dk^2$ and $\varepsilon(2k)/(2k) = c + 4dk^2$, so $4\varepsilon(k)/k - \varepsilon(2k)/(2k) = 3c$: the combination divided by 3 removes the k^2 term (**Richardson extrapolation**, Chapter 16). The cell shoots the three bands at $k = 10^{-4}$ and 2×10^{-4} (six problems; levels below 0.01) and forms this combination for each universe.

```
c_A = (2 * M_BARE / (2 * M_BARE - H) * (1 - math.exp(-(2 * M_BARE - H) * L_TIP))
       / (1 - math.exp(-2 * M_BARE * L_TIP)))
c_C = (2 * M_BARE / (2 * M_BARE + H) * (math.exp((2 * M_BARE + H) * L_TIP) - 1)
       / (math.exp(2 * M_BARE * L_TIP) - 1))
```

```
c_record = float(ks["checksNumeric"]["braneBandSlope_M1_H1_L3_a0"])
SLOPES = "Revision/kohn_sham/results/spectrum/brane-band-slope.csv"
with repository_file(SLOPES).open(encoding="utf-8", newline="") as handle:
    c_csv = float(next(csv.DictReader(handle))["c_theory"])  # the row a4,0 = 0
```

`c_A` and `c_C` are the closed forms c and c_{ctrl} of Section 19.12. `c_record` is the slope stored in `ks-theory.json` under checksNumeric, and `c_csv` the theoretical slope of the first row (the slice $a_{4,0} = 0$) of the record's slope table; `next(...)` takes the first row that the reader gives.

```
say(f"closed forms: c = {c_A:.13f}, c_ctrl = {c_C:.10f}; records: "
    f"{c_record:.13f} and {c_csv:.13f}")
for kind in "ABC":
    say(f"{kind}: shooting slope {slopes[kind]:.10f}")
check(len(rows) == 6 and abs(c_A - c_record) < 1e-13 and abs(c_A - c_csv) < 1e-13
      and abs(slopes["A"] / c_A - 1) < 1e-7 and abs(slopes["B"] / c_A - 1) < 1e-7,
      "the brane-band slope of A and of its partner B is c = 1.9051482536",
      record=f"{KS_THEORY}, checksNumeric braneBandSlope_M1_H1_L3_a0, and {SLOPES}")
check(abs(slopes["C"] / c_C - 1) < 1e-6 and c_C > 7 * c_A,
      "the control's band starts with the much larger slope c_ctrl = 13.42")
```

Out [19] shows $c = 1.9051482536449$ from the closed form and from both records, and $c_{\mathrm{ctrl}} = 13.4219751976$; the shooting slopes of A and B are 1.9051482536 and that of C 13.4219751976. The first check requires the closed form to equal both records to 10^{-13} and the shooting slopes of A and B to agree with it to a relative 10^{-7}; the second requires the control's slope to agree with its closed form and to be more than seven times larger.

In [20], the brane band (Figure 19b.6).

```
fig, ax = plt.subplots()
ax.plot(k_plot, band["A"], "o-", color=COLOUR["A"], lw=2.2, ms=6,
        label="A: $+m$, $\\theta = 0$")
ax.plot(k_plot, band["B"], "s--", color=COLOUR["B"], lw=1.4, ms=3,
        label="B: $-m$, $\\theta = \\pi$ (T3 partner)")
ax.plot(k_plot, band["C"], "^:", color=COLOUR["C"], lw=1.4, ms=5,
        label="C: $-m$, $\\theta = 0$ (control)")
k_line = np.linspace(0.0, 0.25, 51)
ax.plot(k_line, c_A * k_line, "k--", lw=0.8, label="$c\\,k$, $c = 1.905$")
short = k_line[k_line < 0.11]  # the control's line leaves the picture early
ax.plot(short, c_C * short, "k:", lw=0.8, label="$c_{ctrl}\\,k$, $c_{ctrl} = 13.42$")
ax.set_xlabel("3-space momentum $k$ (units of $|m|$)")
ax.set_ylabel("lowest positive level $\\varepsilon$ (units of $|m|$)")
ax.legend(fontsize=8, loc="lower right")
```

The three bands are drawn with markers joined by lines (`"o-"` circles and a solid line, an `s` followed by two hyphens squares and dashes, `"^:"` triangles and dots; `ms` is the marker size; B's small squares sit inside A's large circles). The straight lines ck and $c_{\mathrm{ctrl}}k$ are drawn in black (`k` for black, followed by two hyphens for the dashed line and by a colon, `"k:"`, for the dotted one); the control's line only up to $k = 0.11$, where it reaches about 1.5 (`k_line[k_line < 0.11]` keeps the momenta below 0.11).

```
save_figure(fig, "brane_band",
            "The lowest positive level of the band sector against the 3-space "
            "momentum $k$ (both in units of $|m|$) at the slice $a_{4,0} = 0$, for "
            "$m = 1$, $L = 3$, no interaction, found by shooting: A (blue circles), "
            "its T3 partner B (orange squares, on top of A) and the control C (aqua "
            "triangles). A and B start with the slope $c = 1.905$ of the brane band "
            "(dashed); the control's zero mode lives at the tip, where the redshift "
            "factor $e^{-Hy}$ is largest, so its level rises with the slope "
            "$c_{ctrl} = 13.42$ (dotted) and soon bends below the first level of "
            "the bulk.")
```

`save_figure` saves Figure 19b.6. What the student should see: the blue circles of A, each with the orange square of B inside, rising close to the dashed line $1.905k$ over the whole range; the control's triangles leaving zero along the steep dotted line and then bending over, staying below the first level of the bulk (the levels above the gap), as the caption says.

In [21], the last check.

```
paths = [output_file(f"{FIGURE_FOLDER}/{name}.png") for name in
         ["19b_1_eight_gammas", "19b_2_gamma_block_map", "19b_3_mean_field_map",
          "19b_4_characteristic_functions", "19b_5_k0_ladders", "19b_6_brane_band"]]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

The list holds the paths of the six figure files; the check requires that all exist, and `all_chec ks_passed` prints the last line, ALL 19 CHECKS PASSED (notebook 19b) (Out [21]).

19.17 Watching T3 in the Rust solver: four universes

What the solver does. The Revision Rust Kohn-Sham solver (Chapter 15; `Revision/kohn_sh am/solver`) solves one instantaneous Kohn-Sham problem at a time with its subcommand `single`: given the bare mass m, the coupling λ, the slice $a_{4,0}$, the particle number N, the tip angle θ and, for a thermal state, the temperature T, it finds every level of every sector by shooting with a counting angle, fills the levels, builds the mean field, mixes old and new densities (Anderson mixing) and repeats until the state reproduces itself; its canonical numerics are 900 Runge-Kutta steps, root tolerance 10^{-13} and self-consistency tolerance 10^{-11} (`Revision/kohn_sham/results/parameters.json`, entry numerics). A `single` run contains no map from one universe to another: each universe is solved on its own. (The solver does contain T3 code elsewhere: its self-test t3_block_map_solver_selftest, in `Revision/kohn_sham/solver/src/runs.rs`, solves two members and compares them; the runs of this chapter do not use it.)

What the agreement of two universes tests. The counting angle is the angle ϕ of the real form of Section 19.3, $a = r\cos\phi$, $b = r\sin\phi$, so that $\tan\phi = b/a$ (`Revision/kohn_sham/solver/src/sh oot.rs`, its header). Write $K = \kappa k$ and $E = \varepsilon - v$. Its equation follows from the real form, line by line:

$$\frac{d\phi}{dy} = \frac{a\,b' - b\,a'}{a^2 + b^2} = \frac{(jE - K)a^2 - Mab - Mab + (K + jE)b^2}{r^2} = jE - K\cos 2\phi - M\sin 2\phi.$$

Rule: the derivative of $\arctan(b/a)$ is $(ab' - ba')/(a^2 + b^2)$; insert $a' = Ma - (K + jE)b$ and $b' = (jE - K)a - Mb$; then $a^2 + b^2 = r^2$, $(a^2 - b^2)/r^2 = \cos^2\phi - \sin^2\phi = \cos 2\phi$ and $2ab/r^2 = 2\sin\phi\cos\phi = \sin 2\phi$. The map of T3 in the real form, $(a, b, j, M) \to (b, a, -j, -M)$, exchanges a and b, so the angle of the image is $\psi = \pi/2 - \phi$. Its derivative is $\psi' = -\phi' = -jE + K\cos 2\phi + M\sin 2\phi$. The equation of the partner problem, with $-j$ and $-M$, gives for ψ, line by line:

$$(-j)E - K\cos 2\psi - (-M)\sin 2\psi = -jE - K\cos(\pi - 2\phi) + M\sin(\pi - 2\phi) = -jE + K\cos 2\phi + M\sin 2\phi.$$

Rule: $\cos(\pi - x) = -\cos x$ and $\sin(\pi - x) = \sin x$. The two right sides are equal: the angle of B obeys the equation of A with ϕ replaced by $\pi/2 - \phi$. The tip condition of the solver starts the angle at $\theta/2$, that is at 0 for A and at $\pi/2 = \pi/2 - 0$ for B. So every Runge-Kutta step for B is the step for A, mirrored: the solver does the same arithmetic for both, and their agreement to the last digits is guaranteed **by construction**. The record says this explicitly (`Revision/pairing/kohn_sham/t3-completion.json`, entry numerical_demonstration; the README of `Revision/pairing/kohn_sham`): the shooting method is covariant under the map. What the agreement does test is the solver's handling of both members: the transformed boundary conditions, the labels of the levels, the filling convention, the self-consistency loop and the Mermin root. It does not test the discretisation. The test that does not depend on the discretisation is the record's demonstration with the independent reference solver of Chapter 16 (Section 19.10): its frames for the two members are not images of each other, so on one grid their levels differ by up to 2.23×10^{-4}, and only after Richardson extrapolation do they agree, to 2.043×10^{-14} (ground) and 3.038×10^{-14} (thermal). The controls, on the other hand, are real tests: nothing in the solver forces C, or D at the same coupling λ, to agree with A, and they do not (D agrees with A at the opposite coupling $-\lambda$, which is again the covariance of the shooting).

The four universes. Notebook 19a solves, always with $|m| = 1$:

name	bare mass	coupling	tip angle	role
A	$+1$	$+\lambda$	0	the canonical state of the record
B	-1	$+\lambda$	π	the T3 partner of A
C	-1	$+\lambda$	0	negative control: the tip is not transformed
D	-1	$-\lambda$	π	wrong partner: the coupling is reversed, as in T1

By T3, B must agree with A in every level, occupation, energy and energy-momentum profile, with the opposite S, Q and M_{eff}; C must differ (Section 19.12); and D, which is the T3 partner of the state of A with the coupling $-\lambda$, must agree with A at $-\lambda$, not at λ.

What is compared, and the tolerance. The solver writes for every state a record with the list of levels (each with its momentum shell n_2, block type j, brane parity, label, level ε, degeneracy g and occupation f), the energy E_{KS}, the chemical potential or Fermi level, the entropy, the four integrals

$2\,\mathrm{Vol}_7 \int e^{6Hy} X\,dy$ of $X = \rho, p_3, p_t, p_0$ and of n, the measured residuals of the conservation law along y, and a table of the profiles $n, S, Q, M_{\mathrm{eff}}, v_v, e_{\mathrm{int}}, \rho, p_3, p_t, p_0$ at the 151 points $y = -3, -2.98, \ldots, 0$. The notebook compares A and B in exactly the way the solver's self-test of the record does: the sorted lists of (ε, g, f), the energies, the integrals and the scalar densities, with the tolerance 10^{-9} that the record fixed before its comparison. It adds the orbital-by-orbital pairing of statement S1 and a point-by-point comparison of every profile column. The differences between A and B that the notebook measures are 6.88×10^{-14} ($N = 8$) and 9.86×10^{-14} ($N = 136$) for the levels at the slice $a_{4,0} = 1$ (Notebook 19a, Out [7]) and, over all the 19 pairs it shares with the record, at most 9.9×10^{-14} for the ground states, 1.0×10^{-13} along the history and 8.1×10^{-13} for the thermal states (Notebook 19a, Out [27]). The largest is the entropy of the thermal state at $T = 0.01$, a number near 42 that the solver adds up level by level from the occupations. The record's own demonstration finds at most 2.179×10^{-13} (ground states) and 3.877×10^{-12} (thermal states) in its 210 states (`Revision/pairing/kohn_sham/reports/t3-rust-demo.json`, printed again in Out [27]). All of these are rounding errors: a computer carries about sixteen significant digits, and the tiny rounding errors of the many arithmetic steps of the shooting and of the self-consistency loop add up. They lie far below the tolerance 10^{-9}, and the paragraph "What the agreement of two universes tests" above explains why the agreement stays at the rounding level by construction.

The margin. The solver keeps, above the highest occupied level, empty levels up to a **margin**; the record uses $0.25 + 2\sigma$ with $\sigma = 0$, 0.1 and 0.3 for $\lambda = 0$, $\pm\lambda_1$, $\pm\lambda_2$, that is the margins 0.25, 0.45, 0.85, for its ground states, and $0.2 + 2\sigma$ for its thermal states, that is 0.4 for the coupling λ_1 used here (`Revision/kohn_sham/solver/src/runs.rs`, functions margin_for and the thermal run). The notebook uses the same margins, so that its runs are the record's runs.

What the runs reproduce. Every run of A is a state of the committed canonical matrix (`Revision/kohn_sham/results/ground/summary.csv`, `emt-integrals.csv`, the profile files, and `Revision/kohn_sham/results/thermo/thermodynamics.csv` for the thermal states), and the notebook checks that it reproduces the recorded numbers. Its runs of A, B and C for $N = 8$ and 136 at λ_1, $a_{4,0} = 1$ repeat the solver's T3 self-test of `Revision/kohn_sham/reports/ks-rust-solver.json` (Section 19.10) number by number. And 19 of its states are states of the record's own demonstration `Revision/pairing/kohn_sham/reports/t3-rust-demo.json` (Section 19.10): there A, B and C equal the plus, image and control members of the table `Revision/pairing/kohn_sham/numerics/results/t3-rust-states.csv`, and D at the coupling λ equals its image member at $-\lambda$ (notebook section 17).

19.18 Example: the universes of mass $+M$ and $-M$ in the Rust solver (Notebook 19a)

Notebook 19a checks the eight gammas and the block map of Γ again, builds the Rust solver with cargo, and runs it 62 times: A, B and C for $N = 8$ and 136 at the slice $a_{4,0} = 1$ (the self-test of the record); A, B and C without interaction for $N = 8$ (the zero modes and the level in the gap); A, B and C at the five slices of the deflating history for $N = 136$ and 688; A, B, C and D for five couplings; and A, B and C at three temperatures. Then it compares the 19 states it shares with the record's demonstration of T3. It needs Rust (cargo 1.91.1 or newer), takes about two minutes (33.6 s in the recorded build on the computer that built this book), writes the solver's raw output (about 2 MB) into a folder that git ignores, draws ten figures and ends with the line ALL 37 CHECKS PASSED (notebook 19a).

19.19 How to run Notebook 19a

Step 1. What this notebook does and what it needs.

Notebook 19a (The Rust Kohn-Sham solver for the universes of mass +M and -M) is the file `Revision /textbook/notebooks/19a_t3_rust_pairs.ipynb` of the repository Dirac_claude. It checks the eight real 16 x 16 gamma matrices of the record, builds from them the chirality matrix Gamma and checks that Gamma is the block map of theorem T3. Then it builds the Rust Kohn-Sham solver of the repository with cargo (about a minute when the program is missing, a second when it is up to date) and solves the Kohn-Sham problem of dirac16complex in the deflating primordial field for the universe of mass +M (bare mass m, coupling lambda, tip angle 0), for its partner of theorem T3 (bare mass -m, the same coupling, tip angle pi), for the negative control with the untransformed tip (bare mass -m, tip angle 0) and for the wrong partner with the reversed coupling (-m, -lambda, tip angle pi). It does this for 8, 136 and 688 particles, at the five instants of the deflating history, for five couplings and at three temperatures (62 runs). It checks that the partners have the same levels, occupations, energies and energy-momentum profiles and the opposite scalar density, that the controls differ, that the +M runs reproduce the committed canonical states, that the solver's own T3 self-test is reproduced number by number, that the runs equal the record's own numerical demonstration of T3 in the 19 states that both solve, and that the free zero modes and the sub-gap level of the control agree with their closed forms, and it draws ten overlay figures. The solver writes its raw output (about 2 MB) into the folder `Revision/kohn_sham/solver/target/textbook_19a`, which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the Rust program revision_ks_solver, which is part of the repository and is built on your computer.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download `rustup-init.exe` from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run `rustup-init.exe` again). macOS and Linux: run the command below and

accept the default choice (on Linux first run `sudo apt install build-essential`, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file `Revision/kohn_sham/solver/target/release/revision_ks_solver` (on Windows with the ending `.exe`). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 19a_t3_rust_pairs.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 19a_t3_rust_pairs.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 19a_t3_rust_pairs.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/19a.captions.json`
- `Revision/textbook/figures/19a_1_level_ladder.png`
- `Revision/textbook/figures/19a_2_level_differences.png`
- `Revision/textbook/figures/19a_3_density_profiles.png`
- `Revision/textbook/figures/19a_4_mass_and_potential.png`
- `Revision/textbook/figures/19a_5_emt_profiles.png`
- `Revision/textbook/figures/19a_6_zero_modes_n8.png`
- `Revision/textbook/figures/19a_7_history_energy.png`
- `Revision/textbook/figures/19a_8_history_agreement.png`
- `Revision/textbook/figures/19a_9_coupling_scan.png`
- `Revision/textbook/figures/19a_10_thermal_states.png`

It changes no other file of the repository except the Rust build folder `target` next to each `Cargo.toml` it builds (inside it the notebook keeps the raw output of its program runs in `Revision/kohn_sham/solver/target/textbook_19a`, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 37 CHECKS PASSED (notebook 19a)
```

and the notebook must show 10 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/19a_t3_rust_pairs.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 19a_t3_rust_pairs.ipynb
python -m nbconvert --execute --inplace 19a_t3_rust_pairs.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this terminal.

- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- A cell shows the label with the star for a minute or more: this is normal for the cells that run the history and the coupling scan: they start the Rust program 17 to 27 times, each run takes about a second on a fast computer and up to five seconds on a laptop. Wait until the label shows a number.
- RuntimeError: the solver failed for a state: the error message ends with the last lines the solver printed. The solver and its input files must be the committed versions; get them back and run the notebook again.

```
git checkout -- Revision/kohn_sham Revision/algebra/gammas.json
```

- An AssertionError names a comparison with the record: the line above the error prints the largest difference. Differences in the last two or three digits are rounding effects of another computer and stay far below the tolerance; a larger difference means that the solver or its input files were changed.
- You want the disk space of the solver output back: the folder `Revision/kohn_sham/solver/target/textbook_19a` holds only the raw output of the last run (ignored by git); delete it at any time, the notebook writes it again.

19.20 Notebook 19a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/19a_t3_rust_pairs.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

Theorem **T3** of the Revision record says: in the Kohn-Sham model of the fermion
field dirac16complex in the author's deflating primordial field, every
self-consistent state of the universe with the bare mass $+m$, the coupling
$\lambda$ and the tip angle $\theta$ has an exact partner in the universe with the
bare mass $-m$, the **same** coupling $+\lambda$ and the tip angle $\pi - \theta$,
with the same levels, occupations, energies and energy-momentum profiles and the
opposite scalar density. T3 is PROVED by exact algebra (Wolfram 10 of 10 checks,
sympy 13 of 13). This notebook does not prove it again; it **watches it happen**
in the Rust Kohn-Sham solver of the repository, which knows nothing about T3 and
simply solves each universe on its own.

The notebook

- checks the eight real $16 \times 16$ gamma matrices of the record, builds from
```

```
  them the chirality matrix $\Gamma$ and checks that $\Gamma$ is the map of T3;
- builds the Rust program ‘revision_ks_solver‘ with cargo;
- solves four kinds of universes: A (mass $+m$, tip angle $0$; the canonical
  state of the record), B (mass $-m$, coupling $+\lambda$, tip angle $\pi$; the T3
  partner of A), C (mass $-m$, coupling $+\lambda$, tip angle $0$; the **negative
  control** with the untransformed tip) and D (mass $-m$, coupling $-\lambda$, tip
  angle $\pi$; the **wrong partner** with the reversed coupling);
- reproduces, number by number, the solver’s own T3 self-test stored in the
  record ‘Revision/kohn_sham/reports/ks-rust-solver.json‘;
- compares A and B level by level and point by point, for 8, 136 and 688
  particles, at the five instants of the deflating history, for five couplings and
  at three temperatures (62 runs of the solver), and shows that C and D differ;
- checks that every A run reproduces the committed canonical state of the record,
  and that the free zero modes and a level of the control agree with formulas
  derived here by hand;
- draws ten figures that put the universes on top of each other.

It takes about one to two minutes (longer on a laptop).
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 19a (The Rust Kohn-Sham solver for the universes of mass +M and -M) is the file
‘Revision/textbook/notebooks/19a_t3_rust_pairs.ipynb‘ of the repository Dirac_claude. It
checks the eight real 16 x 16 gamma matrices of the record, builds from them the
chirality matrix Gamma and checks that Gamma is the block map of theorem T3. Then it
builds the Rust Kohn-Sham solver of the repository with cargo (about a minute when the
program is missing, a second when it is up to date) and solves the Kohn-Sham problem of
dirac16complex in the deflating primordial field for the universe of mass +M (bare mass
m, coupling lambda, tip angle 0), for its partner of theorem T3 (bare mass -m, the same
coupling, tip angle pi), for the negative control with the untransformed tip (bare mass
-m, tip angle 0) and for the wrong partner with the reversed coupling (-m, -lambda, tip
angle pi). It does this for 8, 136 and 688 particles, at the five instants of the
deflating history, for five couplings and at three temperatures (62 runs). It checks that
the partners have the same levels, occupations, energies and energy-momentum profiles and
the opposite scalar density, that the controls differ, that the +M runs reproduce the
committed canonical states, that the solver’s own T3 self-test is reproduced number by
number, that the runs equal the record’s own numerical demonstration of T3 in the 19
states that both solve, and that the free zero modes and the sub-gap level of the control
agree with their closed forms, and it draws ten overlay figures. The solver writes its
raw output (about 2 MB) into the folder ‘Revision/kohn_sham/solver/target/textbook_19a‘,
which git ignores. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy
and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It also needs Rust (the program cargo, version 1.91.1 or newer), because it
runs the Rust program revision_ks_solver, which is part of the repository and is built on
your computer.
```

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home

folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Install Rust and build the Rust program (Rust once per computer, the build once per program).

Windows: download ‘rustup-init.exe‘ from https://rustup.rs, run it and accept the default choices; if it reports that the Visual Studio C++ Build Tools are missing, let it install them (or install the workload "Desktop development with C++" from https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run ‘rustup-init.exe‘ again). macOS and Linux: run the command below and accept the default choice (on Linux first run ‘sudo apt install build-essential‘, which provides the linker that Rust needs).

macOS (zsh) and Linux (bash):

```
curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Then close the terminal, open a new one, do Step 4 again, and check the version of cargo; it must print 1.91.1 or a newer version:

Windows, macOS and Linux:

```
cargo --version
```

Build the program with the following command, in the repository folder (the same command on all three systems):

Windows, macOS and Linux:

```
cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
```

The first build takes about 1 minute; later builds reuse the result and take a few seconds. The program is then the file ‘Revision/kohn_sham/solver/target/release/revision_ks_solver‘ (on Windows with the ending ‘.exe‘). The notebook runs the same build command itself before it uses the program (it takes a second when the program is up to date), so the notebook also works if you skip this build; building here first shows any problem with Rust before the notebook starts.

Step 6. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
```

```
    jupyter lab 19a_t3_rust_pairs.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 2 minutes on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 7. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 19a_t3_rust_pairs.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 19a_t3_rust_pairs.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 8. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/19a.captions.json‘
- ‘Revision/textbook/figures/19a_1_level_ladder.png‘
- ‘Revision/textbook/figures/19a_2_level_differences.png‘
- ‘Revision/textbook/figures/19a_3_density_profiles.png‘
- ‘Revision/textbook/figures/19a_4_mass_and_potential.png‘
- ‘Revision/textbook/figures/19a_5_emt_profiles.png‘
- ‘Revision/textbook/figures/19a_6_zero_modes_n8.png‘
- ‘Revision/textbook/figures/19a_7_history_energy.png‘
- ‘Revision/textbook/figures/19a_8_history_agreement.png‘
- ‘Revision/textbook/figures/19a_9_coupling_scan.png‘
- ‘Revision/textbook/figures/19a_10_thermal_states.png‘

It changes no other file of the repository except the Rust build folder ‘target‘ next to each ‘Cargo.toml‘ it builds (inside it the notebook keeps the raw output of its program runs in ‘Revision/kohn_sham/solver/target/textbook_19a‘, which git ignores); running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 37 CHECKS PASSED (notebook 19a)
```

and the notebook must show 10 figures below the cells that draw them.

Step 9. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/19a_t3_rust_pairs.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 19a_t3_rust_pairs.ipynb
  python -m nbconvert --execute --inplace 19a_t3_rust_pairs.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "cargo is not recognized" or "command not found: cargo" or "cargo was not found": open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from this

terminal.
- "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build again.
- A cell shows the label with the star for a minute or more: this is normal for the cells that run the history and the coupling scan: they start the Rust program 17 to 27 times, each run takes about a second on a fast computer and up to five seconds on a laptop. Wait until the label shows a number.
- RuntimeError: the solver failed for a state: the error message ends with the last lines the solver printed. The solver and its input files must be the committed versions; get them back and run the notebook again.

    git checkout -- Revision/kohn_sham Revision/algebra/gammas.json

- An AssertionError names a comparison with the record: the line above the error prints the largest difference. Differences in the last two or three digits are rounding effects of another computer and stay far below the tolerance; a larger difference means that the solver or its input files were changed.
- You want the disk space of the solver output back: the folder ‘Revision/kohn_sham/solver/target/textbook_19a‘ holds only the raw output of the last run (ignored by git); delete it at any time, the notebook writes it again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 19a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 19a (The Rust Kohn-Sham solver for the universes of mass +M and -M) is the
# file Revision/textbook/notebooks/19a_t3_rust_pairs.ipynb of the repository
# Dirac_claude. It checks the eight real 16 x 16 gamma matrices of the record, builds
# from them the chirality matrix Gamma and checks that Gamma is the block map of theorem
# T3. Then it builds the Rust Kohn-Sham solver of the repository with cargo (about a
# minute when the program is missing, a second when it is up to date) and solves the
# Kohn-Sham problem of dirac16complex in the deflating primordial field for the universe
# of mass +M (bare mass m, coupling lambda, tip angle 0), for its partner of theorem T3
# (bare mass -m, the same coupling, tip angle pi), for the negative control with the
# untransformed tip (bare mass -m, tip angle 0) and for the wrong partner with the
# reversed coupling (-m, -lambda, tip angle pi). It does this for 8, 136 and 688
# particles, at the five instants of the deflating history, for five couplings and at
# three temperatures (62 runs). It checks that the partners have the same levels,
# occupations, energies and energy-momentum profiles and the opposite scalar density,
# that the controls differ, that the +M runs reproduce the committed canonical states,
# that the solver's own T3 self-test is reproduced number by number, that the runs equal
# the record's own numerical demonstration of T3 in the 19 states that both solve, and
# that the free zero modes and the sub-gap level of the control agree with their closed
# forms, and it draws ten overlay figures. The solver writes its raw output (about 2 MB)
```

```
# into the folder Revision/kohn_sham/solver/target/textbook_19a, which git ignores. It
# needs a computer with Windows 11, macOS or Linux, an internet connection for the
# installation, the program Git, and Python 3.12 or newer (the notebooks were built with
# Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy
# 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient
# 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and
# matplotlib; the other packages run Jupyter, the program that shows and runs notebooks.
# It also needs Rust (the program cargo, version 1.91.1 or newer), because it runs the
# Rust program revision_ks_solver, which is part of the repository and is built on your
# computer.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
```

```
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
```

```
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. INSTALL RUST AND BUILD THE RUST PROGRAM (RUST ONCE PER COMPUTER, THE BUILD ONCE
# PER PROGRAM).
#
# Windows: download rustup-init.exe from https://rustup.rs, run it and accept the default
# choices; if it reports that the Visual Studio C++ Build Tools are missing, let it
# install them (or install the workload "Desktop development with C++" from
# https://visualstudio.microsoft.com/visual-cpp-build-tools/ and run rustup-init.exe
# again). macOS and Linux: run the command below and accept the default choice (on Linux
# first run sudo apt install build-essential, which provides the linker that Rust needs).
#
# macOS (zsh) and Linux (bash):
#     curl --proto "=https" --tlsv1.2 -sSf https://sh.rustup.rs | sh
#
# Then close the terminal, open a new one, do Step 4 again, and check the version of
# cargo; it must print 1.91.1 or a newer version:
#
# Windows, macOS and Linux:
#     cargo --version
#
# Build the program with the following command, in the repository folder (the same
# command on all three systems):
#
# Windows, macOS and Linux:
#     cargo build --release --manifest-path Revision/kohn_sham/solver/Cargo.toml
#
# The first build takes about 1 minute; later builds reuse the result and take a few
# seconds. The program is then the file
# Revision/kohn_sham/solver/target/release/revision_ks_solver (on Windows with the ending
# .exe). The notebook runs the same build command itself before it uses the program (it
# takes a second when the program is up to date), so the notebook also works if you skip
# this build; building here first shows any problem with Rust before the notebook starts.
#
# STEP 6. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 19a_t3_rust_pairs.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
```

```
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 2
# minutes on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 7. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 19a_t3_rust_pairs.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 19a_t3_rust_pairs.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 8. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/19a.captions.json
# - Revision/textbook/figures/19a_1_level_ladder.png
# - Revision/textbook/figures/19a_2_level_differences.png
# - Revision/textbook/figures/19a_3_density_profiles.png
# - Revision/textbook/figures/19a_4_mass_and_potential.png
# - Revision/textbook/figures/19a_5_emt_profiles.png
# - Revision/textbook/figures/19a_6_zero_modes_n8.png
# - Revision/textbook/figures/19a_7_history_energy.png
# - Revision/textbook/figures/19a_8_history_agreement.png
# - Revision/textbook/figures/19a_9_coupling_scan.png
# - Revision/textbook/figures/19a_10_thermal_states.png
#
# It changes no other file of the repository except the Rust build folder target next to
# each Cargo.toml it builds (inside it the notebook keeps the raw output of its program
# runs in Revision/kohn_sham/solver/target/textbook_19a, which git ignores); running it
# headless or saving it in JupyterLab also rewrites the notebook file itself. It does not
# use the internet while it runs. The files it writes are the same files that are stored
# in the repository (on another computer a figure may differ in a few bytes, which is
# harmless). To get the stored versions back, run this command in the repository folder
# (it also undoes every change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
```

```
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 37 CHECKS PASSED (notebook 19a)
#
# and the notebook must show 10 figures below the cells that draw them.
#
# STEP 9. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/19a_t3_rust_pairs.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 19a_t3_rust_pairs.ipynb
#     python -m nbconvert --execute --inplace 19a_t3_rust_pairs.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "cargo is not recognized" or "command not found: cargo" or "cargo was not found":
#   open a new terminal after installing Rust, do Step 4 again, and start JupyterLab from
#   this terminal.
# - "linker link.exe not found" (Windows) or "linker cc not found" (Linux): install the
#   C++ Build Tools (Windows) or build-essential (Linux) as described in Step 5 and build
#   again.
# - A cell shows the label with the star for a minute or more: this is normal for the
#   cells that run the history and the coupling scan: they start the Rust program 17 to
#   27 times, each run takes about a second on a fast computer and up to five seconds on
#   a laptop. Wait until the label shows a number.
```

```
# - RuntimeError: the solver failed for a state: the error message ends with the last
#   lines the solver printed. The solver and its input files must be the committed
#   versions; get them back and run the notebook again.
#     git checkout -- Revision/kohn_sham Revision/algebra/gammas.json
# - An AssertionError names a comparison with the record: the line above the error prints
#   the largest difference. Differences in the last two or three digits are rounding
#   effects of another computer and stay far below the tolerance; a larger difference
#   means that the solver or its input files were changed.
# - You want the disk space of the solver output back: the folder
#   Revision/kohn_sham/solver/target/textbook_19a holds only the raw output of the last
#   run (ignored by git); delete it at any time, the notebook writes it again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs

NOTEBOOK_ID = "19a"  # this notebook: chapter 19, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")

say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 19a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** (the author's names): $x_1, x_2, x_3$ = ordinary 3-space, which
  inflates (scale factor $e^{a_4}\sin^{1/6}z$); $x_4$ = the time; $x_5, x_6, x_7$ =
  the three **extra times**, which **deflate exponentially** (scale factor
  $e^{-a_4}\sin^{1/6}z$); $x_0$ = the hidden space direction, $z = 6Hx_0$ between
  $0$ and $\pi/2$.
- **Hidden coordinate** $y = \ln(\sin z)/(6H)$: it runs from the **tip** $y = -L$
  (the model cuts the hidden direction there, $L = 3$) to the **brane** $y = 0$.
- **Slice** $a_{4,0}$: one instant of the history $a_4 = AHx_4$ ($A = 1$), the value
  of $a_4$ at that instant. The slices are $a_{4,0} = 0, 0.5, 1, 1.5, 2$.
- **Gamma matrices** $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$: the author's eight
  real $16 \times 16$ matrices, one for each coordinate, with
  $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}$ times the unit matrix, where
  $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order
  $x_0, \dots, x_7$. **Chirality** $\Gamma$: their product
  $\gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$.
- **Pauli matrices** $\sigma_1, \sigma_2, \sigma_3$: the three $2 \times 2$ matrices
  $\begin{pmatrix}0&1\\1&0\end{pmatrix}$, $\begin{pmatrix}0&-i\\i&0\end{pmatrix}$,
  $\begin{pmatrix}1&0\\0&-1\end{pmatrix}$.
- **Kohn-Sham state**: an approximate state of $N$ identical fermions built from
  one-particle wave functions (**orbitals**) that each solve a one-particle equation
  in a common **effective potential**; the potential depends on the densities of the
  occupied orbitals, so the equations are solved **self-consistently** (repeat until
  nothing changes).
- **Level** $\varepsilon$: an allowed energy of the one-particle equation;
  **occupation** $f$: how many particles sit in an orbital of that level ($0$ to
  $1$); **degeneracy** $g$: how many orbitals share the level. **HOMO** and
  **LUMO**: the highest occupied and the lowest empty level; **Kohn-Sham gap**:
  LUMO minus HOMO.
- **Block type** $j = \pm 1$ and **brane parity** (even or odd): labels of the
  orbitals (explained in section 4).
- **Bare mass** $m$: the mass in the Lagrangian; **effective mass** $M_{eff}(y)$:
  the mass seen by an orbital, $m$ plus the mean-field term.
- **Coupling** $\lambda$: the strength of the self-interaction $U = (\lambda/2)S^2$.
  $\lambda_1$ and $\lambda_2$ are the two calibrated couplings of the record.
- **Densities**: the particle density $n(y)$, the scalar density $S(y)$ and the
  density $Q(y)$; **proper** means per unit of proper 7-volume; the **coordinate
  density** $e^{6Hy}n(y)$ is the density per unit of $y$.
- **Energy-momentum profiles**: the energy density $\rho(y)$ and the pressures
  $p_3(y)$ (3-space), $p_t(y)$ (extra times), $p_0(y)$ (hidden direction).
- **Tip angle** $\theta$: the angle of the boundary condition at the tip.
- **Zero mode**: an orbital with the level $\varepsilon = 0$ at zero momentum.
- **Partner**: the image of a state under the map of theorem T3. **Negative
  control**: a computation that must FAIL to agree; it shows that the agreement is
  not automatic.
- **Relative difference**: $|a - b|/\max(|a|, 1)$; numbers that agree to the last
  digits of a computer have relative differences near $10^{-15}$.
- Status labels: PROVED (exact), COMPUTED (numerical), ASSUMED (a choice),
  HYPOTHESIS, OPEN (not known).
- Units: $H = 1$ and $|m| = 1$; energies, momenta and temperatures in units of
  $|m|$, lengths in units of $1/H$.
```

Text cell:

```
## 4. The physical and mathematical situation

**The geometry.** The Kohn-Sham record works on the background $b_4 = 1$ of the author's
metric of 2026-10-14 (the second function $b_4(x_4)$ of the metric, which multiplies the
extra-time components, is put equal to 1). In the hidden coordinate $y$ this metric is
$ds^2 = e^{2Hy}[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 +
dx_7^2)] - dx_4^2 + dy^2$. While $a_4$ grows, 3-space inflates by $e^{a_4}$ and the
three extra times deflate by $e^{-a_4}$; the proper 7-volume factor
$e^{3a_4} e^{-3a_4} e^{6Hy} = e^{6Hy}$ does not change, because the deflation of the
extra times exactly compensates the inflation of 3-space. Along the history every
3-space momentum is redshifted: an orbital with momentum $k$ feels
$\kappa(y)\,k$ with $\kappa = e^{-Hy - a_{4,0}}$.

**The Kohn-Sham problem** (record `Revision/kohn_sham/ks-theory.json`). The 16
components of the field split exactly into eight blocks of two components
$\chi(y) = (\chi_1, \chi_2)$, labelled by $j = \pm 1$ and two more signs. In a block
of type $j$ an orbital with 3-momentum $k$ and level $\varepsilon$ solves

$$h_j\chi = \varepsilon\chi,\qquad h_j = j\Big[-i\sigma_1\frac{d}{dy}
+ M_{eff}(y)\,\sigma_2 + \kappa(y)\,k\,\sigma_3\Big] + v_v(y),$$

with the effective mass $M_{eff} = m + \frac{15}{16}\lambda S$ and the potential
$v_v = -\frac{1}{16}\lambda n$ (Hartree plus the exact exchange of the uniform gas;
no correlation).

**Boundary conditions.** At the brane $y = 0$ the $Z_2$ mirror, which is ASSUMED:
even parity $\chi_2(0) = 0$ or odd parity $\chi_1(0) = 0$; both parities are solved
and filled together. At the tip $y = -L$ a chosen condition
$(1 - Q(\theta))\chi(-L) = 0$ with $Q(\theta) = \cos\theta\,\sigma_3 +
\sin\theta\,\sigma_2$: for $\theta = 0$ it says $\chi_2(-L) = 0$, for $\theta = \pi$
it says $\chi_1(-L) = 0$.

**Theorem T3** (PROVED; records `Revision/pairing/kohn_sham/t3-theory.json`,
`wolfram-t3.json`, `python-t3.json`). The map $(\chi, j) \to (\sigma_2\chi, -j)$ at
the same momentum (in 16 components: the chirality matrix $\Gamma$), with the two
brane parities exchanged and $\theta \to \pi - \theta$, sends every self-consistent
Kohn-Sham state with $(m, \lambda, \theta)$ to a self-consistent state with
$(-m, +\lambda, \pi - \theta)$: equal levels with degeneracies, occupations, chemical
potential, entropy, Kohn-Sham energy $E_{KS}$, grand potential and free energy,
equal $n$, $v_v$, $\rho$, $p_3$, $p_t$, $p_0$; opposite $S$, $Q$ and $M_{eff}$. With
$(-m, -\lambda)$ the map fails, and with the untransformed tip it fails too.

**The four universes of this notebook** ($m = 1$):

| name | bare mass | coupling | tip angle | role |
| --- | --- | --- | --- | --- |
| A | $+1$ | $+\lambda$ | $0$ | the canonical state of the record |
| B | $-1$ | $+\lambda$ | $\pi$ | the T3 partner of A |
| C | $-1$ | $+\lambda$ | $0$ | negative control: tip not transformed |
| D | $-1$ | $-\lambda$ | $\pi$ | wrong partner: coupling reversed |

**What is assumed and what is not shown.** The $Z_2$ brane is ASSUMED; the tip is
```

```
a chosen cutoff; the states are instantaneous (adiabatic) mean-field states; the
history $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND (the Kohn-Sham states are not an
admissible source of the $a_4$ equations). T3 maps solutions onto solutions; it
does not create anything: no creation process, rate or amplitude of a pair of
universes follows from it, and nothing here says that the partner exists. Every
number in this notebook is COMPUTED by the solver; the agreement of A and B is a
numerical confirmation of T3, not its proof.
```

Text cell:

```
## 5. The eight real gamma matrices and the map of T3

The map of T3 is built from the author's eight real $16 \times 16$ gamma matrices.
The next cell reads them from the record `Revision/algebra/gammas.json` (a matrix is
a list of 16 rows of 16 numbers; a complex matrix would be stored as a pair of real
and imaginary parts, so a plain list means a real matrix) and checks three things:
(1) every entry is $-1$, $0$ or $+1$, so the matrices are real; (2) the
anticommutation rule $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\mathbf{1}$ for
all 64 pairs; (3) the product $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}
\cdots\gamma^{(x_7)}$ is the diagonal matrix with $-1$ eight times and then $+1$ eight
times, the matrix stored in the record, and anticommutes with every gamma.
```

In [2]:

```
import csv  # reads the tables (CSV files) of the record
import math  # pi, exp, tanh and cosh of single numbers
import re  # regular expressions: reads numbers out of the record's text

import numpy as np  # arrays of numbers

GAMMAS = "Revision/algebra/gammas.json"  # the record of the gamma matrices
ALGEBRA = "Revision/algebra/reports/wolfram-algebra.json"  # its check report
fixture = json.loads(repository_file(GAMMAS).read_text(encoding="utf-8"))
stored_real = all(isinstance(g, list) for g in fixture["gamma"])  # no complex part
gamma = [np.array(g, dtype=float) for g in fixture["gamma"]]  # gamma[k] is xk
eta = np.array(fixture["eta"], dtype=float)  # +1 +1 +1 +1 -1 -1 -1 -1 (x0, ..., x7)
entries = sorted({float(x) for g in gamma for x in g.flat})  # all values used
say(f"{len(gamma)} matrices of size {gamma[0].shape}; their entries: {entries}")
worst = 0.0  # largest violation of the anticommutation rule
for a in range(8):
    for b in range(8):
        anti = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
        rule = 2.0 * eta[a] * (a == b) * np.eye(16)  # 2 eta^ab times the unit
        worst = max(worst, float(np.max(np.abs(anti - rule))))
Gamma = gamma[0]  # gamma^(x0) first ...
for a in range(1, 8):
    Gamma = Gamma @ gamma[a]  # ... times gamma^(x1), gamma^(x2), ..., gamma^(x7)
diagonal = np.diag([-1.0] * 8 + [1.0] * 8)
anticommutes = all(np.array_equal(Gamma @ g, -(g @ Gamma)) for g in gamma)
check(stored_real and entries == [-1.0, 0.0, 1.0] and worst == 0.0,
      "the eight gamma matrices are real and obey g^a g^b + g^b g^a = 2 eta^ab",
      record=f"{ALGEBRA}, checks reality and Clifford_relation")
```

```
check(np.array_equal(Gamma, diagonal) and anticommutes
      and np.array_equal(Gamma, np.array(fixture["Gamma"], dtype=float)),
      "Gamma = g^(x0) g^(x1) ... g^(x7) = diag(-1 (8 times), +1 (8 times))",
      record=f"{ALGEBRA}, check Gamma_diag")
```

Out [2]:

```
8 matrices of size (16, 16); their entries: [-1.0, 0.0, 1.0]
PASS the eight gamma matrices are real and obey g^a g^b + g^b g^a = 2 eta^ab
     reproduces Revision/algebra/reports/wolfram-algebra.json, checks reality and
    Clifford_relation
PASS Gamma = g^(x0) g^(x1) ... g^(x7) = diag(-1 (8 times), +1 (8 times))
     reproduces Revision/algebra/reports/wolfram-algebra.json, check Gamma_diag
```

Text cell:

```
The record ‘Revision/kohn_sham/ks-theory.json‘ stores the unitary $16 \times 16$
matrix $V$ whose 16 columns are the basis of the eight blocks (two columns per
block, block $b$ in the columns $2b$ and $2b + 1$), and the label $(j, s_2, s_3)$ of
every block. The entries are written as text: ‘0‘, ‘1‘, ‘-1‘, ‘I‘, ‘-I‘ times
$1/(2\sqrt2)$. The next cell checks that $V$ is unitary, that in every block
$\gamma^{(x_0)}$ acts as $\sigma_3$ and $\gamma^{(x_0)}\gamma^{(x_4)}$ as
$j\sigma_1$, and that $\Gamma$ maps the two columns of block $(j, s_2, s_3)$ onto
the two columns of block $(-j, s_2, s_3)$ times $s_2\sigma_2$. That last statement
is the map $(\chi, j) \to (\sigma_2\chi, -j)$ of T3 (up to the sign $s_2$).
```

In [3]:

```
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
ENTRY = {"0": 0.0, "1": 1.0, "-1": -1.0, "I": 1j, "-I": -1j}  # the record's text
V = np.array([[ENTRY[x] for x in row]
              for row in ks["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
V = V / (2.0 * math.sqrt(2.0))  # the record stores 2 sqrt(2) V
labels = [tuple(int(x) for x in label) for label in ks["blockBasis"]["labels"]]
sig1 = np.array([[0, 1], [1, 0]], dtype=complex)  # Pauli matrix sigma1
sig2 = np.array([[0, -1j], [1j, 0]])  # Pauli matrix sigma2
sig3 = np.array([[1, 0], [0, -1]], dtype=complex)  # Pauli matrix sigma3
unitary = float(np.max(np.abs(V.conj().T @ V - np.eye(16))))
worst_map, worst_form = 0.0, 0.0
for b, (j, s2, s3) in enumerate(labels):
    p = labels.index((-j, s2, s3))  # the partner block
    Vb, Vp = V[:, 2 * b:2 * b + 2], V[:, 2 * p:2 * p + 2]  # their two columns
    worst_map = max(worst_map, float(np.max(np.abs(Gamma @ Vb - Vp @ (s2 * sig2)))))
    g0 = Vb.conj().T @ gamma[0] @ Vb  # gamma^(x0) inside block b
    g04 = Vb.conj().T @ gamma[0] @ gamma[4] @ Vb  # gamma^(x0) gamma^(x4) there
    worst_form = max(worst_form, float(np.max(np.abs(g0 - sig3))),
                     float(np.max(np.abs(g04 - j * sig1))))
    if b < 4:
        say(f"block {b} {labels[b]} <-> block {p} {labels[p]}: matrix "
            f"{s2:+d} sigma2")
```

```
say(f"largest deviations: unitarity {unitary:.1e}, block map {worst_map:.1e}, "
    f"block forms {worst_form:.1e}")
check(unitary < 1e-12 and worst_map < 1e-12 and worst_form < 1e-12,
      "Gamma maps block (j, s2, s3) onto block (-j, s2, s3) as s2 sigma2",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.Gamma_is_the_block_map")
```

Out [3]:

```
block 0 (1, 1, 1) <-> block 4 (-1, 1, 1): matrix +1 sigma2
block 1 (1, 1, -1) <-> block 5 (-1, 1, -1): matrix +1 sigma2
block 2 (1, -1, 1) <-> block 6 (-1, -1, 1): matrix -1 sigma2
block 3 (1, -1, -1) <-> block 7 (-1, -1, -1): matrix -1 sigma2
largest deviations: unitarity 1.1e-16, block map 0.0e+00, block forms 1.1e-16
PASS Gamma maps block (j, s2, s3) onto block (-j, s2, s3) as s2 sigma2
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.Gamma_is_the_block_map
```

Text cell:

```
## 6. Building the solver and a helper that runs it

The next cell builds the Rust program with cargo (the helper ‘rust_program‘ of the
set-up cell; the first build takes about a minute, later builds a second) and
defines the colours of the figures: blue for A, orange for B, aqua for C and yellow
for D, always in this order.
```

In [4]:

```
SOLVER_MANIFEST = "Revision/kohn_sham/solver/Cargo.toml"  # the crate of the solver
program = rust_program(SOLVER_MANIFEST, "revision_ks_solver")  # build it, get path
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_19a"  # git ignores it
RUN_FOLDER.mkdir(parents=True, exist_ok=True)

COLOUR = {"A": "#2a78d6", "B": "#eb6834", "C": "#1baf7a", "D": "#eda100"}
NAME = {"A": "A: $+m$, $+\\lambda$, $\\theta = 0$",
        "B": "B: $-m$, $+\\lambda$, $\\theta = \\pi$ (T3 partner)",
        "C": "C: $-m$, $+\\lambda$, $\\theta = 0$ (control)",
        "D": "D: $-m$, $-\\lambda$, $\\theta = \\pi$ (wrong partner)"}
say("Solver ready, colours defined.")
```

Out [4]:

```
Rust program revision_ks_solver is built and ready.
Solver ready, colours defined.
```

Text cell:

The next cell defines ‘solve‘, which runs the solver once. The subcommand ‘single‘ solves one Kohn-Sham state with the options ‘--m‘ (bare mass), ‘--lambda‘, ‘--a4‘ (the slice), ‘--N‘ (particle number), ‘--tip-theta‘ (tip angle), ‘--margin‘ (how far above the highest occupied level the solver keeps empty levels; the record uses $0.25 + 2\sigma$ with $\sigma = 0$, 0.1, 0.3 for $\lambda = 0$, $\pm\lambda_1$, $\pm\lambda_2$) and optionally ‘--T‘ (temperature). It writes a JSON file (levels, energies, integrals) and, with ‘--profiles‘, a table of the profiles at the 151 points $y = -3, -2.98, \dots, 0$. ‘--root‘ tells it where the repository is (it reads ‘ks-theory.json‘ and the gamma matrices from there). ‘solve‘ returns a dictionary with the JSON record, its levels as a list of tuples $(n_2, j, \text{parity}, \text{label}, \varepsilon, g, f)$ and the profiles as numpy arrays. The four universes are made by ‘universe‘.

In [5]:

```
TIP = {"A": 0.0, "B": math.pi, "C": 0.0, "D": math.pi}  # tip angle of each kind
MASS = {"A": 1.0, "B": -1.0, "C": -1.0, "D": -1.0}  # bare mass of each kind
SIGN = {"A": 1.0, "B": 1.0, "C": 1.0, "D": -1.0}  # D reverses the coupling


def read_profile(path):
    """A profile table as a dictionary: column name -> numpy array (151 values)."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}


def solve(label, m, lam, a4, N, theta, margin, T=None):
    """Run "revision_ks_solver single" for one state and return its results."""
    out = RUN_FOLDER / f"{label}.json"  # the solver's record of this state
    table = RUN_FOLDER / f"{label}.csv"  # its profiles
    command = [str(program), "single", "--root", str(REPO), "--m", repr(m),
               "--lambda", repr(lam), "--a4", repr(a4), "--N", repr(float(N)),
               "--tip-theta", repr(theta), "--margin", repr(margin),
               "--out", str(out), "--profiles", str(table)]
    if T is not None:
        command += ["--T", repr(T)]  # a thermal (Mermin) state
    done = subprocess.run(command, capture_output=True, text=True,
                          encoding="utf-8", errors="replace")
    last = (done.stdout.strip().split("\n") or [""])[-1]  # SUCCESS or FAILURE
    if done.returncode != 0 or last != "SUCCESS":
        raise RuntimeError(f"the solver failed for {label}: {done.stderr[-400:]}")
    state = json.loads(out.read_text(encoding="utf-8"))
    state["levels"] = [tuple(level) for level in
                       state["levels_n2_j_parity_label_eps_deg_f"]]
    state["profile"] = read_profile(table)
    return state


def universe(kind, lam, a4, N, margin, T=None):
    """Solve universe A, B, C or D with the coupling lam (D uses -lam)."""
    label = f"{kind}_N{N}_lam{lam:+.4g}_a{a4:g}" + ("" if T is None else f"_T{T:g}")
    return solve(label, MASS[kind], SIGN[kind] * lam, a4, N, TIP[kind], margin, T)
```

```
say("solve and universe defined; outputs go to the folder "
    "Revision/kohn_sham/solver/target/textbook_19a")
```

Out [5]:

```
solve and universe defined; outputs go to the folder
    Revision/kohn_sham/solver/target/textbook_19a
```

Text cell:

```
## 7. What the record says: the proof counts and the solver's own T3 self-test

The next cell reads the two proof reports of T3 and counts their passing checks
(Wolfram 10, sympy 13). Then it reads the couplings of the record
(‘parameters.json‘) and the solver's check named ‘t3_block_map_solver_selftest‘. That
check solved A, B and C at the slice $a_{4,0} = 1$ with the coupling $\lambda_1$ for
$N = 8$ and $N = 136$ and recorded the energies, the number of levels and the
control's energy in its text; the cell pulls these numbers out with a regular
expression (a pattern that matches text).
```

In [6]:

```
T3_WOLFRAM = "Revision/pairing/kohn_sham/reports/wolfram-t3.json"
T3_SYMPY = "Revision/pairing/kohn_sham/reports/python-t3.json"
wolfram_t3 = json.loads(repository_file(T3_WOLFRAM).read_text(encoding="utf-8"))
sympy_t3 = json.loads(repository_file(T3_SYMPY).read_text(encoding="utf-8"))
passed_w = sum(c["verdict"] == "PASS" for c in wolfram_t3["checks"])
passed_s = sum(c["verdict"] == "PASS" for c in sympy_t3["checks"])
total_w, total_s = len(wolfram_t3["checks"]), len(sympy_t3["checks"])
say(f"proof of T3: Wolfram {passed_w} of {total_w} checks pass, "
    f"sympy {passed_s} of {total_s}")
check(passed_w == total_w == 10 and passed_s == total_s == 13,
      "the proof records of T3 pass completely (10 and 13 checks)",
      record=f"{T3_WOLFRAM} and {T3_SYMPY}")

SOLVER_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
SELFTEST = "t3_block_map_solver_selftest"
solver_report = json.loads(repository_file(SOLVER_REPORT).read_text(encoding="utf-8"))
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
LAMBDA = {int(entry["N"]): (entry["lambda1"], entry["lambda2"])
          for entry in parameters["couplingCalibration"]["values"]}
for N, (l1, l2) in LAMBDA.items():
    say(f"N = {N:3d}: lambda_1 = {l1}, lambda_2 = {l2}")
selftest = next(c for c in solver_report["checks"] if c["name"] == SELFTEST)
PATTERN = re.compile(r"N = (\d+), lambda = (\S+), a4,0 = 1: E_KS (\S+) vs (\S+); "
                     r"(\d+) levels; .*?untransformed tip b\(-L\) = 0\): "
                     r"E_KS = (\S+) vs")
RECORD = {}  # N -> the recorded numbers of the self-test
for part in selftest["detail"].split(" | "):
```

```
    found = PATTERN.search(part)
    RECORD[int(found.group(1))] = {
        "lambda": float(found.group(2)), "E_A": float(found.group(3)),
        "E_B": float(found.group(4)), "levels": int(found.group(5)),
        "E_C": float(found.group(6))}
for N, entry in RECORD.items():
    E_A, E_B, E_C, count = (entry[key] for key in ("E_A", "E_B", "E_C", "levels"))
    say(f"record, N = {N}: E_KS(A) = {E_A:.12e}, E_KS(B) = {E_B:.12e}, "
        f"{count} levels, control E_KS(C) = {E_C:.10e}")
check(selftest["verdict"] == "PASS" and sorted(RECORD) == [8, 136]
      and all(RECORD[N]["lambda"] == LAMBDA[N][0] for N in RECORD),
      "the record holds the passing self-test for N = 8 and 136 at lambda_1",
      record=f"{SOLVER_REPORT}, check {SELFTEST}")
```

Out [6]:

```
proof of T3: Wolfram 10 of 10 checks pass, sympy 13 of 13
PASS the proof records of T3 pass completely (10 and 13 checks)
     reproduces Revision/pairing/kohn_sham/reports/wolfram-t3.json and
    Revision/pairing/kohn_sham/reports/python-t3.json
N =   8: lambda_1 = 0.01946, lambda_2 = 0.05838
N = 136: lambda_1 = 0.0009298, lambda_2 = 0.002789
N = 688: lambda_1 = 0.0001846, lambda_2 = 0.0005538
record, N = 8: E_KS(A) = -9.868426190876e-04, E_KS(B) = -9.868426190868e-04, 80 levels,
    control E_KS(C) = -2.0145243719e+00
record, N = 136: E_KS(A) = 3.239294915318e+01, E_KS(B) = 3.239294915318e+01, 160 levels,
    control E_KS(C) = 2.9283751318e+01
PASS the record holds the passing self-test for N = 8 and 136 at lambda_1
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    t3_block_map_solver_selftest
```

Text cell:

```
## 8. Reproducing the self-test: A, B and C for N = 8 and N = 136

The next cell solves the three universes A, B, C for $N = 8$ and $N = 136$ at the
slice $a_{4,0} = 1$ with the coupling $\lambda_1$ and the record's margin $0.45$
(six runs of the solver). Then it compares, as the self-test does: the sorted lists
of $(\varepsilon, g, f)$ of A and B, the energies $E_{KS}$, the integrals of
$\rho, p_3, p_t, p_0$ over the hidden direction and the scalar densities
($S_A + S_B$ must vanish). It prints each largest difference.
```

In [7]:

```
SELF = {}  # (N, kind) -> state
for N in (8, 136):
    for kind in "ABC":
        SELF[(N, kind)] = universe(kind, LAMBDA[N][0], 1.0, N, 0.45)


def sorted_levels(state):
```

```
    """The levels as a sorted list of (eps, g, f), as the self-test compares them."""
    return sorted((level[4], level[5], level[6]) for level in state["levels"])


def level_difference(first, second):
    """Largest difference of the sorted (eps, g, f) lists of two states."""
    pairs = zip(sorted_levels(first), sorted_levels(second))
    return max(max(abs(x - y) for x, y in zip(p, q)) for p, q in pairs)


def integral_difference(first, second):
    """Largest relative difference of the four integrated EMT components."""
    a, b = first["emtIntegrals_2Vol7_int_e6Hy"], second["emtIntegrals_2Vol7_int_e6Hy"]
    return max(abs(a[c] - b[c]) / max(abs(a[c]), 1.0) for c in ("rho", "p3", "p_t",
                                                                "p0"))


def scalar_sum(first, second):
    """max|S_A + S_B| / max|S_A| on the 151 profile points."""
    sa, sb = first["profile"]["S"], second["profile"]["S"]
    return float(np.max(np.abs(sa + sb)) / max(np.max(np.abs(sa)), 1e-300))


for N in (8, 136):
    A, B, C = SELF[(N, "A")], SELF[(N, "B")], SELF[(N, "C")]
    E_A, E_B, E_C = A["E_KS"], B["E_KS"], C["E_KS"]
    count_A, count_B = len(A["levels"]), len(B["levels"])
    say(f"N = {N}: E_KS(A) = {E_A:.12e}, E_KS(B) = {E_B:.12e}, "
        f"{count_A} and {count_B} levels")
    say(f"    levels (eps, g, f): {level_difference(A, B):.2e}; EMT integrals: "
        f"{integral_difference(A, B):.2e}; |S_A + S_B|/max|S|: "
        f"{scalar_sum(A, B):.2e}; control E_KS(C) = {E_C:.10e}")
```

Out [7]:

```
N = 8: E_KS(A) = -9.868426190876e-04, E_KS(B) = -9.868426190868e-04, 80 and 80 levels
    levels (eps, g, f): 6.88e-14; EMT integrals: 8.96e-16; |S_A + S_B|/max|S|: 1.84e-15;
    control E_KS(C) = -2.0145243719e+00
N = 136: E_KS(A) = 3.239294915318e+01, E_KS(B) = 3.239294915318e+01, 160 and 160 levels
    levels (eps, g, f): 9.86e-14; EMT integrals: 2.19e-16; |S_A + S_B|/max|S|: 8.38e-16;
    control E_KS(C) = 2.9283751318e+01
```

Text cell:

```
The next cell turns these numbers into checks. For each $N$: A and B have the
recorded number of levels; every difference between A and B is below the
self-test's tolerance $10^{-9}$; the energies agree with the energies printed in
the record within $10^{-9}$ (relative); the control C has the recorded energy and
differs from A by more than $1$. The tolerance $10^{-9}$ is the one fixed in the
record before its comparison; differences of a few $10^{-15}$ are the rounding of
the computer.
```

In [8]:

```
TOL = 1e-9  # the tolerance of the self-test, fixed in the record
REC = f"{SOLVER_REPORT}, check {SELFTEST}"


def close(a, b, tol=TOL):
    """True when a and b agree within tol relative to max(|a|, 1)."""
    return abs(a - b) <= tol * max(abs(a), 1.0)


for N in (8, 136):
    A, B, C = SELF[(N, "A")], SELF[(N, "B")], SELF[(N, "C")]
    rec = RECORD[N]
    check(len(A["levels"]) == len(B["levels"]) == rec["levels"]
          and level_difference(A, B) < TOL and integral_difference(A, B) < TOL
          and scalar_sum(A, B) < TOL and close(A["E_KS"], B["E_KS"]),
          f"N = {N}: A and B have equal levels, energies, EMT integrals, S_B = -S_A",
          record=REC)
    check(close(A["E_KS"], rec["E_A"]) and close(B["E_KS"], rec["E_B"])
          and close(C["E_KS"], rec["E_C"]) and abs(C["E_KS"] - A["E_KS"]) > 1.0,
          f"N = {N}: the energies of A, B and the control C are those of the record",
          record=REC)
```

Out [8]:

```
PASS N = 8: A and B have equal levels, energies, EMT integrals, S_B = -S_A
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    t3_block_map_solver_selftest
PASS N = 8: the energies of A, B and the control C are those of the record
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    t3_block_map_solver_selftest
PASS N = 136: A and B have equal levels, energies, EMT integrals, S_B = -S_A
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    t3_block_map_solver_selftest
PASS N = 136: the energies of A, B and the control C are those of the record
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, check
    t3_block_map_solver_selftest
```

Text cell:

```
## 9. A is the canonical state of the record

Universe A at $N = 136$, $\lambda_1$, $a_{4,0} = 1$ is the state ‘N136_lamp1_a10‘ of
the committed canonical matrix. The next cell reads its row of
‘ground/summary.csv‘, its row of ‘ground/emt-integrals.csv‘ and its profile file,
and compares them with the new run: the energy, the highest occupied level (HOMO),
the lowest empty level (LUMO), the Kohn-Sham gap, the four EMT integrals and the
particle number, and every column of the profile at every point (relative to the
column’s largest value). It also prints whether the new profile file is identical
byte for byte to the committed one (on the computer that built this book it is; on
another computer the last digit of a few numbers may differ).
```

In [9]:

```
GROUND = "Revision/kohn_sham/results/ground"


def read_rows(relative):
    """A CSV file of the record as {id: row}."""
    with repository_file(relative).open(encoding="utf-8", newline="") as handle:
        return {row["id"]: row for row in csv.DictReader(handle)}


def homo_lumo(state):
    """The highest occupied (f = 1) and the lowest empty (f = 0) level."""
    occupied = [lv[4] for lv in state["levels"] if lv[6] > 0.5]
    empty = [lv[4] for lv in state["levels"] if lv[6] < 0.5]
    return max(occupied), min(empty)


summary = read_rows(f"{GROUND}/summary.csv")
integrals = read_rows(f"{GROUND}/emt-integrals.csv")
A = SELF[(136, "A")]
homo, lumo = homo_lumo(A)
new = {"E_KS": A["E_KS"], "HOMO": homo, "LUMO": lumo, "KS_gap": lumo - homo}
row = summary["N136_lamp1_a10"]
worst_scalar = max(abs(new[key] - float(row[key])) for key in new)
emt_row = integrals["N136_lamp1_a10"]
new_int = A["emtIntegrals_2Vol7_int_e6Hy"]  # the new integrals
worst_integral = max(abs(new_int[c] - float(emt_row[f"int_{c}"]))
                     / max(abs(float(emt_row[f"int_{c}"])), 1.0)
                     for c in ("rho", "p3", "p_t", "p0", "n"))
committed = read_profile(repository_file(f"{GROUND}/profiles/N136_lamp1_a10.csv"))
worst_profile = max(float(np.max(np.abs(A["profile"][c] - committed[c]))
                          / max(np.max(np.abs(committed[c])), 1e-300))
                    for c in committed)
for key in new:
    say(f"{key:7}: new {new[key]: .15e}   record {float(row[key]): .15e}")
say(f"largest differences: scalars {worst_scalar:.1e}, integrals "
    f"{worst_integral:.1e}, profile columns {worst_profile:.1e}")
same_bytes = ((RUN_FOLDER / "A_N136_lam+0.0009298_a1.csv").read_bytes()
              == repository_file(f"{GROUND}/profiles/N136_lamp1_a10.csv").read_bytes())
report("new profile file byte-identical to the record", same_bytes)
check(worst_scalar < TOL and worst_integral < TOL and worst_profile < TOL,
      "A reproduces the canonical state N136_lamp1_a10 of the record",
      record=f"{GROUND}/summary.csv, emt-integrals.csv and profiles, N136_lamp1_a10")
```

Out [9]:

```
E_KS   : new  3.239294915318298e+01   record  3.239294915318298e+01
HOMO   : new  3.258848143646794e-01   record  3.258848143646794e-01
LUMO   : new  3.608587569459729e-01   record  3.608587569459729e-01
KS_gap : new  3.497394258129349e-02   record  3.497394258129349e-02
largest differences: scalars 0.0e+00, integrals 0.0e+00, profile columns 0.0e+00
```

```
RESULT new profile file byte-identical to the record = True
PASS A reproduces the canonical state N136_lamp1_a10 of the record
     reproduces Revision/kohn_sham/results/ground/summary.csv, emt-integrals.csv and
    profiles, N136_lamp1_a10
```

Text cell:

```
## 10. The two spectra, level by level

T3 does more than say that the sorted lists agree: it says WHICH level goes where.
The orbital of block type $j$ and brane parity $p$ goes to block type $-j$ and the
other parity, at the same momentum shell $n_2$ (the momentum $k = \Delta k
\sqrt{n_2}$) and with the same level. The next cell checks this for $N = 136$: for
every level of A it looks for a level of B in the shell $n_2$, with $-j$ and the
other parity, at the same $\varepsilon$ (within $10^{-9}$), and checks that the
pairing uses every level of B exactly once, with the same degeneracy and occupation.
It also prints, sector by sector, the label of the partner minus the label of the
level. The label is the solver's count of a level inside its sector, counted from
the boundary conditions; A and B have different boundary conditions, so the count
of a partner can differ by one. The map is fixed by the sector and the energy, not
by the label.
```

In [10]:

```
OTHER = {"even": "odd", "odd": "even"}  # the brane parity is exchanged


def level_map(first, second, tol=TOL):
    """Pair every level (n2, j, p, l, eps, g, f) of first with a level of second in
    (n2, -j, other p) at the same eps; return the pairs (None if one is missing)."""
    unused = list(second["levels"])
    pairs = []
    for lv in sorted(first["levels"]):
        match = [w for w in unused if w[0] == lv[0] and w[1] == -lv[1]
                 and w[2] == OTHER[lv[2]] and abs(w[4] - lv[4]) <= tol]
        if not match:
            return None
        best = min(match, key=lambda w: abs(w[4] - lv[4]))
        unused.remove(best)
        pairs.append((lv, best))
    return pairs


pairs136 = level_map(SELF[(136, "A")], SELF[(136, "B")])
shifts = {}
for lv, w in pairs136:
    key = f"j = {lv[1]:+d}, {lv[2]:4}"
    shifts.setdefault(key, set()).add(w[3] - lv[3])  # label of B minus label of A
for key in sorted(shifts):
    say(f"A levels with {key}: partner in j = opposite, other parity; label shift "
        f"{sorted(shifts[key])}")
worst = max(abs(lv[4] - w[4]) for lv, w in pairs136)
```

```
report("largest |eps_A - eps_B| of the paired levels, N = 136", f"{worst:.2e}")
check(pairs136 is not None and len(pairs136) == len(SELF[(136, "B")]["levels"])
      and all(lv[5] == w[5] and lv[6] == w[6] for lv, w in pairs136),
      "every level of A has its partner in B with -j and the other brane parity",
      record="Revision/pairing/kohn_sham/t3-theory.json, statement S1")
check(level_map(SELF[(136, "A")], SELF[(136, "C")]) is None,
      "the control C has no such partner levels (the pairing fails)")
```

Out [10]:

```
A levels with j = +1, even: partner in j = opposite, other parity; label shift [-1]
A levels with j = +1, odd : partner in j = opposite, other parity; label shift [0]
A levels with j = -1, even: partner in j = opposite, other parity; label shift [0]
A levels with j = -1, odd : partner in j = opposite, other parity; label shift [1]
RESULT largest |eps_A - eps_B| of the paired levels, N = 136 = 9.86e-14
PASS every level of A has its partner in B with -j and the other brane parity
     reproduces Revision/pairing/kohn_sham/t3-theory.json, statement S1
PASS the control C has no such partner levels (the pairing fails)
```

Text cell:

```
The next cell draws the three spectra side by side for $N = 136$ (a **level
ladder**): every level is a short horizontal line, coloured when it is occupied
($f = 1$) and grey when it is empty; the dashed line is the Fermi level (the
highest occupied level). Only the levels below $1.3$ are drawn. A and B look the
same, C does not.
```

In [11]:

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
for x, kind in enumerate("ABC"):
    state = SELF[(136, kind)]
    for lv in state["levels"]:
        if lv[4] > 1.3:
            continue  # only the lower part of the spectrum
        full = lv[6] > 0.5  # occupied?
        ax.plot([x - 0.32, x + 0.32], [lv[4], lv[4]],
                color=COLOUR[kind] if full else "0.7", lw=2.0 if full else 1.0)
    ax.plot([x - 0.42, x + 0.42], [state["mu_or_fermi_level"]] * 2, color="0.2",
            ls="--", lw=1.0)
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(["A: $+m$, $\\theta = 0$", "B: $-m$, $\\theta = \\pi$",
                    "C: $-m$, $\\theta = 0$ (control)"])
ax.set_xlim(-0.6, 2.6)
ax.set_ylabel("Kohn-Sham level $\\varepsilon$ (units of $|m|$)")
ax.set_title("$N = 136$, $\\lambda = \\lambda_1$, slice $a_{4,0} = 1$")
ax.grid(axis="x", visible=False)
save_figure(fig, "level_ladder",
            "Level ladders of the three universes A (bare mass $+m$, tip angle 0), B "
            "(bare mass $-m$, the same coupling, tip angle $\\pi$: the T3 partner) "
            "and C (bare mass $-m$, tip angle 0: the control) for $N = 136$ "
```

```
"particles, $\\lambda = \\lambda_1$, slice $a_{4,0} = 1$, all solved "
"independently by the Rust solver. Each short line is one Kohn-Sham "
"level (vertical axis, units of $|m|$), coloured when occupied and grey "
"when empty; the dashed line is the Fermi level. A and B are the same "
"ladder; C, with the untransformed tip, is a different one.")
```

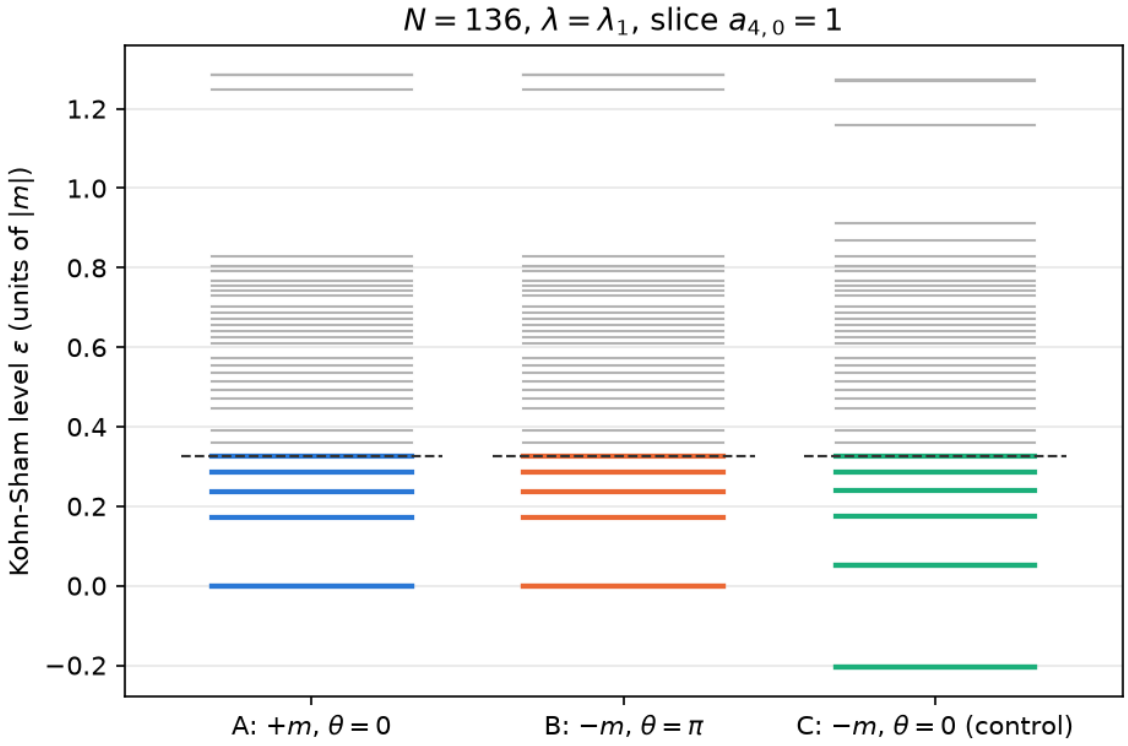

Figure 497: Level ladders of the three universes A (bare mass $+m$, tip angle 0), B (bare mass $-m$, the same coupling, tip angle π: the T3 partner) and C (bare mass $-m$, tip angle 0: the control) for $N = 136$ particles, $\lambda = \lambda_1$, slice $a_{4,0} = 1$, all solved independently by the Rust solver. Each short line is one Kohn-Sham level (vertical axis, units of $|m|$), coloured when occupied and grey when empty; the dashed line is the Fermi level. A and B are the same ladder; C, with the untransformed tip, is a different one. (Notebook 19a, figure 1.)

Out [11]:

```
Figure 19a.1 saved as Revision/textbook/figures/19a_1_level_ladder.png
```

Text cell:

```
The next cell draws the level-by-level differences on a logarithmic axis: for the
$i$-th level of the sorted lists, $|\varepsilon_{A,i} - \varepsilon_{B,i}|$ (A
against its partner) and $|\varepsilon_{A,i} - \varepsilon_{C,i}|$ (A against the
control). A difference that is exactly zero cannot be drawn on a logarithmic axis;
it is drawn at $10^{-17}$. The cell checks the two ranges that the caption states.
```

In [12]:

```
FLOOR = 1e-17  # where exact zeros are drawn on the logarithmic axis
eps = {kind: np.array([e for e, g, f in sorted_levels(SELF[(136, kind)])])
       for kind in "ABC"}
index = np.arange(1, len(eps["A"]) + 1)
fig, ax = plt.subplots()
ax.semilogy(index, np.maximum(np.abs(eps["A"] - eps["B"]), FLOOR), "o",
            color=COLOUR["B"], ms=4, label="A against B (T3 partner)")
ax.semilogy(index, np.maximum(np.abs(eps["A"] - eps["C"]), FLOOR), "^",
            color=COLOUR["C"], ms=4, label="A against C (control)")
ax.axhline(TOL, color="0.3", ls="--", lw=1.0, label="tolerance $10^{-9}$")
ax.set_xlabel("level number $i$ (levels sorted by energy)")
ax.set_ylabel("$|\\varepsilon_{A,i} - \\varepsilon_{X,i}|$ (units of $|m|$)")
ax.set_title("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$: 160 levels")
ax.legend(fontsize=8, loc="center right")
save_figure(fig, "level_differences",
            "Level-by-level differences between universe A and its T3 partner B "
            "(orange circles) and between A and the control C (aqua triangles), "
            "for $N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$; horizontal axis "
            "the number of the level in the sorted list, vertical axis the absolute "
            "difference in units of $|m|$ (logarithmic; exact zeros drawn at "
            "$10^{-17}$). The partner agrees to $10^{-13}$ or better, the rounding "
            "of the computer, far below the tolerance $10^{-9}$ (dashed); the "
            "control differs by $10^{-4}$ to $1$.")
max_partner = float(np.max(np.abs(eps["A"] - eps["B"])))
gaps_control = np.abs(eps["A"] - eps["C"])
report("largest level difference A - B", f"{max_partner:.2e}")
report("smallest and largest level difference A - C",
       f"{gaps_control.min():.2e} and {gaps_control.max():.3g}")
check(max_partner < 1e-12 and 1e-4 < gaps_control.min() and gaps_control.max() < 1,
      "levels: A - B below 1e-12, A - C between 1e-4 and 1 (the caption's ranges)")
```

Out [12]:

```
Figure 19a.2 saved as Revision/textbook/figures/19a_2_level_differences.png
RESULT largest level difference A - B = 9.86e-14
RESULT smallest and largest level difference A - C = 1.88e-04 and 0.456
PASS levels: A - B below 1e-12, A - C between 1e-4 and 1 (the caption's ranges)
```

Text cell:

```
## 11. The densities and the potentials, point by point

T3 says that the particle density $n(y)$ and the potential $v_v(y)$ of B are those
of A, while the scalar density $S(y)$, the density $Q(y)$ and the effective mass
$M_{eff}(y)$ change sign. The next cell checks this at the 151 points of the
profiles for $N = 8$ and $N = 136$ (relative to the largest value of each column)
and also checks the interaction energy density $e_{int}$, which contains $S$ only as
$S^2$.
```

In [13]:

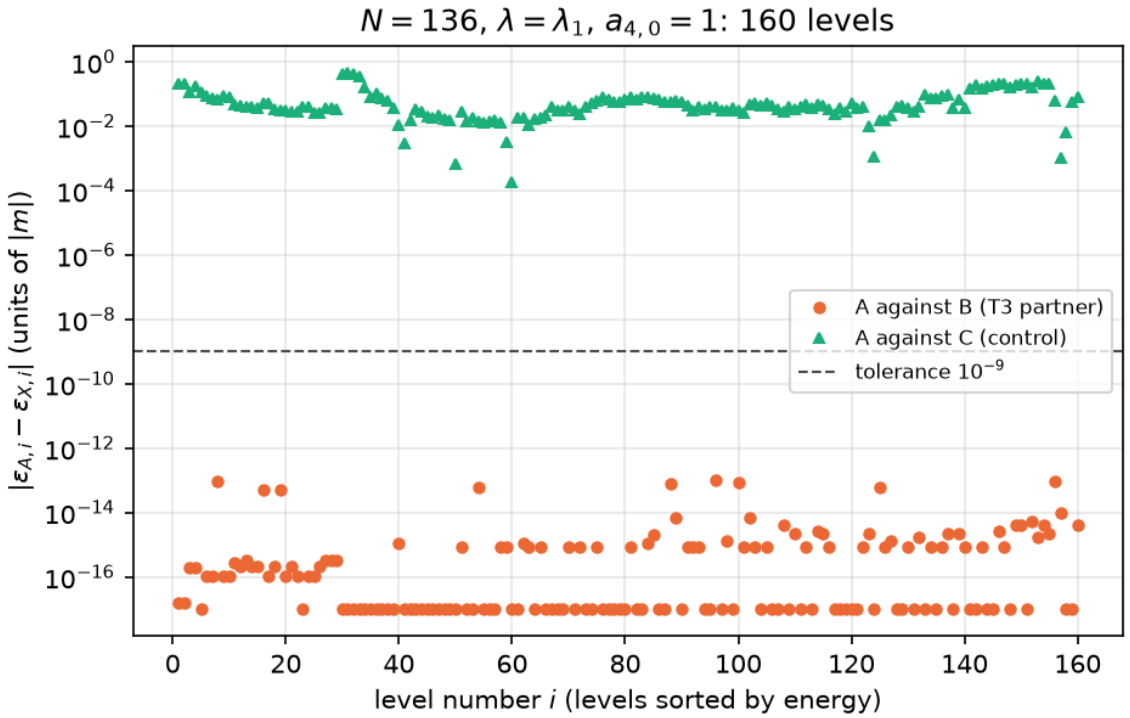

Figure 498: Level-by-level differences between universe A and its T3 partner B (orange circles) and between A and the control C (aqua triangles), for $N = 136$, $\lambda = \lambda_1$, $a_{4,0} = 1$; horizontal axis the number of the level in the sorted list, vertical axis the absolute difference in units of $|m|$ (logarithmic; exact zeros drawn at 10^{-17}). The partner agrees to 10^{-13} or better, the rounding of the computer, far below the tolerance 10^{-9} (dashed); the control differs by 10^{-4} to 1. (Notebook 19a, figure 2.)

```
EVEN = ("n", "v_v", "e_int", "rho", "p3", "p_t", "p0")  # unchanged by the map
ODD = ("S", "Q", "M_eff")  # change sign under the map


def profile_mismatch(first, second):
    """Largest relative mismatch of the T3 rule (even columns equal, odd opposite)."""
    worst = 0.0
    for column in EVEN + ODD:
        sign = -1.0 if column in ODD else 1.0
        a, b = first["profile"][column], second["profile"][column]
        scale = max(float(np.max(np.abs(a))), 1e-300)
        worst = max(worst, float(np.max(np.abs(b - sign * a))) / scale)
    return worst


for N in (8, 136):
    mismatch = profile_mismatch(SELF[(N, "A")], SELF[(N, "B")])
    control = profile_mismatch(SELF[(N, "A")], SELF[(N, "C")])
    say(f"N = {N}: T3 rule for B: {mismatch:.2e}; the same rule for C: {control:.3g}")
    check(mismatch < TOL and control > 0.1,
          f"N = {N}: n, v_v, e_int, rho, p3, p_t, p0 equal; S, Q, M_eff opposite",
          record="Revision/pairing/kohn_sham/t3-theory.json, statements S3 and S4")
```

Out [13]:

```
N = 8: T3 rule for B: 2.78e-15; the same rule for C: 298
PASS N = 8: n, v_v, e_int, rho, p3, p_t, p0 equal; S, Q, M_eff opposite
     reproduces Revision/pairing/kohn_sham/t3-theory.json, statements S3 and S4
N = 136: T3 rule for B: 5.80e-15; the same rule for C: 4.65e+03
PASS N = 136: n, v_v, e_int, rho, p3, p_t, p0 equal; S, Q, M_eff opposite
     reproduces Revision/pairing/kohn_sham/t3-theory.json, statements S3 and S4
```

Text cell:

```
The next cell draws the densities for $N = 136$. Left: the proper particle density
$n(y)$ (logarithmic axis) of A, B and C; A and B lie on top of each other (B is
dashed). Right: the scalar density $S(y)$ of A and B; B is the mirror image of A,
$S_B = -S_A$. The scalar density of the control C is not drawn on the right: it is
more than ten times larger (the cell prints and checks the factor) and would flatten
the two curves that matter.
```

In [14]:

```
y = SELF[(136, "A")]["profile"]["y"]  # the 151 points from the tip to the brane
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
STYLE = {"A": dict(lw=2.6), "B": dict(lw=1.6, ls="--"), "C": dict(lw=1.4, ls=":")}
for kind in "ABC":
    prof = SELF[(136, kind)]["profile"]
    left.semilogy(y, prof["n"], color=COLOUR[kind], label=NAME[kind], **STYLE[kind])
    if kind != "C":
        right.plot(y, prof["S"], color=COLOUR[kind], label=NAME[kind],
```

```
                    **STYLE[kind])
left.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
left.set_ylabel("proper particle density $n(y)$")
left.legend(fontsize=7, loc="upper right")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("proper scalar density $S(y)$")
right.axhline(0.0, color="0.3", lw=0.8)
right.legend(fontsize=7, loc="lower right")
fig.suptitle("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
save_figure(fig, "density_profiles",
            "Left: the proper particle density $n(y)$ of the universes A (blue), B "
            "(orange, dashed) and C (aqua, dotted) for $N = 136$, "
            "$\\lambda = \\lambda_1$, $a_{4,0} = 1$, against the hidden coordinate $y$ "
            "from the tip $y = -3$ to the brane $y = 0$ (logarithmic vertical axis, "
            "units $|m|^7$). Right: the proper scalar density $S(y)$ of A and B. "
            "The density of B is that of A, and its scalar density is the mirror "
            "image $S_B = -S_A$, as T3 states. The control C has its own densities; "
            "its scalar density, more than ten times larger, is not drawn.")
size = {kind: float(np.max(np.abs(SELF[(136, kind)]["profile"]["S"])))
        for kind in "AC"}
factor = size["C"] / size["A"]  # how much larger the control's scalar density is
report("largest |S| of C divided by largest |S| of A", f"{factor:.1f}")
check(factor > 10.0,
      "the scalar density of the control is more than ten times that of A")
```

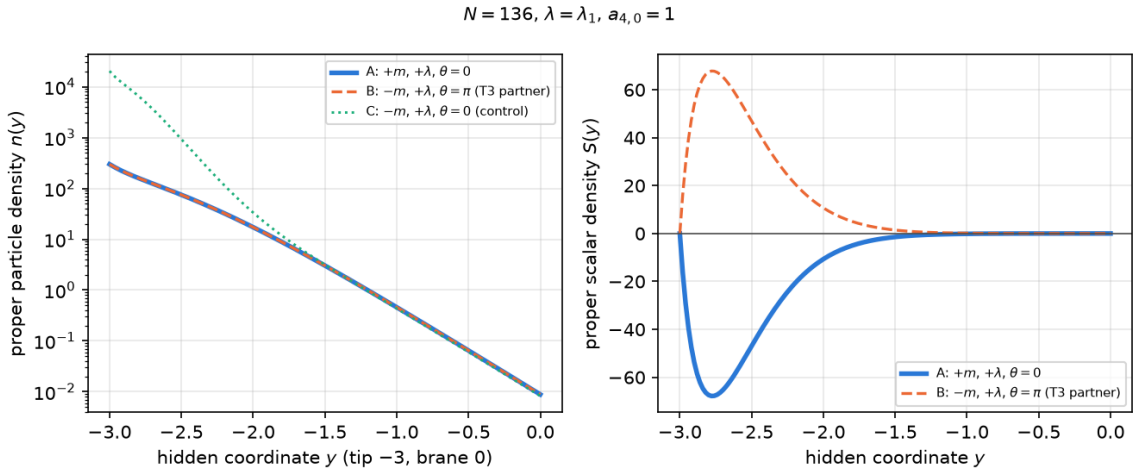

Figure 499: Left: the proper particle density $n(y)$ of the universes A (blue), B (orange, dashed) and C (aqua, dotted) for $N = 136$, $\lambda = \lambda_1$, $a_{4,0} = 1$, against the hidden coordinate y from the tip $y = -3$ to the brane $y = 0$ (logarithmic vertical axis, units $|m|^7$). Right: the proper scalar density $S(y)$ of A and B. The density of B is that of A, and its scalar density is the mirror image $S_B = -S_A$, as T3 states. The control C has its own densities; its scalar density, more than ten times larger, is not drawn. (Notebook 19a, figure 3.)

Out [14]:

```
Figure 19a.3 saved as Revision/textbook/figures/19a_3_density_profiles.png
```

```
RESULT largest |S| of C divided by largest |S| of A = 24.1
PASS the scalar density of the control is more than ten times that of A
```

Text cell:

```
The next cell draws the effective mass $M_{eff}(y) = \pm m + \frac{15}{16}\lambda S$
(left) and the potential $v_v(y) = -\frac{1}{16}\lambda n$ (right) of A, B and C.
For B the mass is $-M_{eff}$ of A at every point; the potential is the same.
```

In [15]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
for kind in "ABC":
    prof = SELF[(136, kind)]["profile"]
    left.plot(y, prof["M_eff"], color=COLOUR[kind], label=NAME[kind], **STYLE[kind])
    right.plot(y, prof["v_v"], color=COLOUR[kind], label=NAME[kind], **STYLE[kind])
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("effective mass $M_{eff}(y)$ (units of $|m|$)")
left.legend(fontsize=7, loc="center right")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("potential $v_v(y)$ (units of $|m|$)")
fig.suptitle("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
save_figure(fig, "mass_and_potential",
            "Left: the self-consistent effective mass $M_{eff}(y)$ of A (blue), B "
            "(orange, dashed) and C (aqua, dotted) for $N = 136$, "
            "$\\lambda = \\lambda_1$, $a_{4,0} = 1$ (units of $|m|$). Right: the "
            "self-consistent potential $v_v(y)$ (units of $|m|$). B has exactly the "
            "opposite mass of A at every point and the same potential: this is why "
            "the bare mass must change sign while the coupling keeps its sign. The "
            "control C has the mass $-m$ but its own, different mean field.")
```

Out [15]:

```
Figure 19a.4 saved as Revision/textbook/figures/19a_4_mass_and_potential.png
```

Text cell:

```
## 12. The energy-momentum profiles

The energy density and the three pressures are the source terms that the gravity
equations would need. The next cell draws $\rho$, $p_3$, $p_t$ and $p_0$ for
$N = 136$, each multiplied by the volume factor $e^{6Hy}$ (so the area under each
curve is proportional to the integral over the hidden direction). The cell also
checks the identity $p_0' + 6Hp_0 = 3H(p_3 + p_t)$ (the conservation law along
$y$), which the solver measures for every state, for B.
```

In [16]:

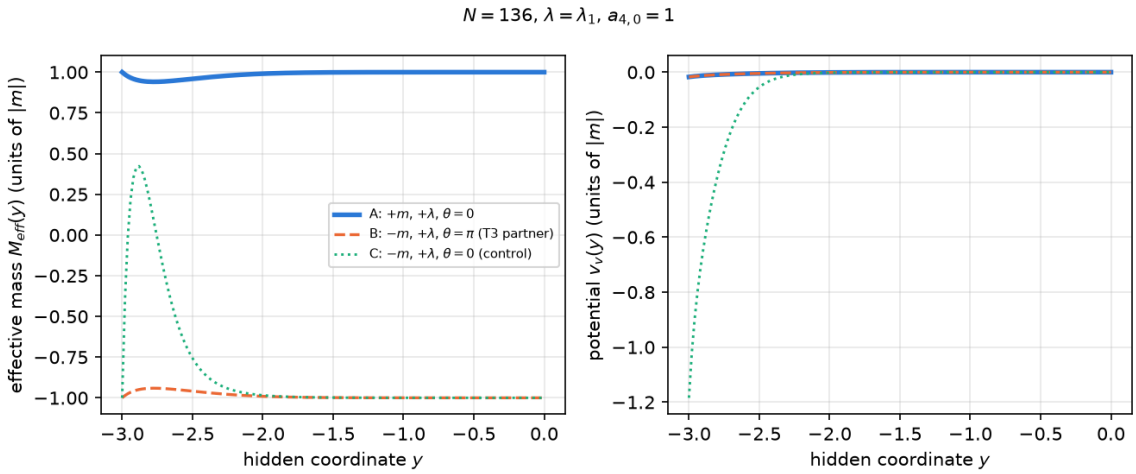

Figure 500: Left: the self-consistent effective mass $M_{eff}(y)$ of A (blue), B (orange, dashed) and C (aqua, dotted) for $N = 136$, $\lambda = \lambda_1$, $a_{4,0} = 1$ (units of $|m|$). Right: the self-consistent potential $v_v(y)$ (units of $|m|$). B has exactly the opposite mass of A at every point and the same potential: this is why the bare mass must change sign while the coupling keeps its sign. The control C has the mass $-m$ but its own, different mean field. (Notebook 19a, figure 4.)

```
weight = np.exp(6.0 * y)  # the proper-volume factor e^{6 H y}, H = 1
fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.0), sharex=True)
for ax, column, title in zip(axes.flat, ("rho", "p3", "p_t", "p0"),
                             ("energy density $\\rho$", "pressure $p_3$ (3-space)",
                              "pressure $p_t$ (extra times)",
                              "pressure $p_0$ (hidden direction)")):
    for kind in "ABC":
        prof = SELF[(136, kind)]["profile"]
        ax.plot(y, weight * prof[column], color=COLOUR[kind], label=NAME[kind],
                **STYLE[kind])
    ax.set_title(title, fontsize=9)
    ax.set_ylabel("$e^{6Hy}$ times the component")
for ax in axes[1]:
    ax.set_xlabel("hidden coordinate $y$")
axes[0, 0].legend(fontsize=7, loc="upper left")
fig.tight_layout()
save_figure(fig, "emt_profiles",
            "The energy-momentum profiles of A (blue), B (orange, dashed) and C "
            "(aqua, dotted) for $N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$: "
            "energy density $\\rho$, pressures $p_3$ (3-space), $p_t$ (the "
            "deflating extra times) and $p_0$ (hidden direction), each times the "
            "volume factor $e^{6Hy}$, against the hidden coordinate $y$ (units "
            "$|m|^8$ with $H = 1$). The four profiles of the partner B coincide "
            "with those of A at every point (T3, statement S4); the control C "
            "differs.")
for N in (8, 136):
    B = SELF[(N, "B")]
    integrated, pointwise = (B["yConservationIntegratedRel"],
                             B["yConservationPointwiseRel"])
    say(f"N = {N}, B: y-conservation, integrated {integrated:.1e}, pointwise "
```

```
        f"{pointwise:.1e} (relative)")
check(integrated < 1e-8 and pointwise < 1e-6,
      f"N = {N}: the partner B obeys p0' + 6H p0 = 3H (p3 + p_t)",
      record=f"{SOLVER_REPORT}, checks emt_y_conservation_integrated, pointwise")
```

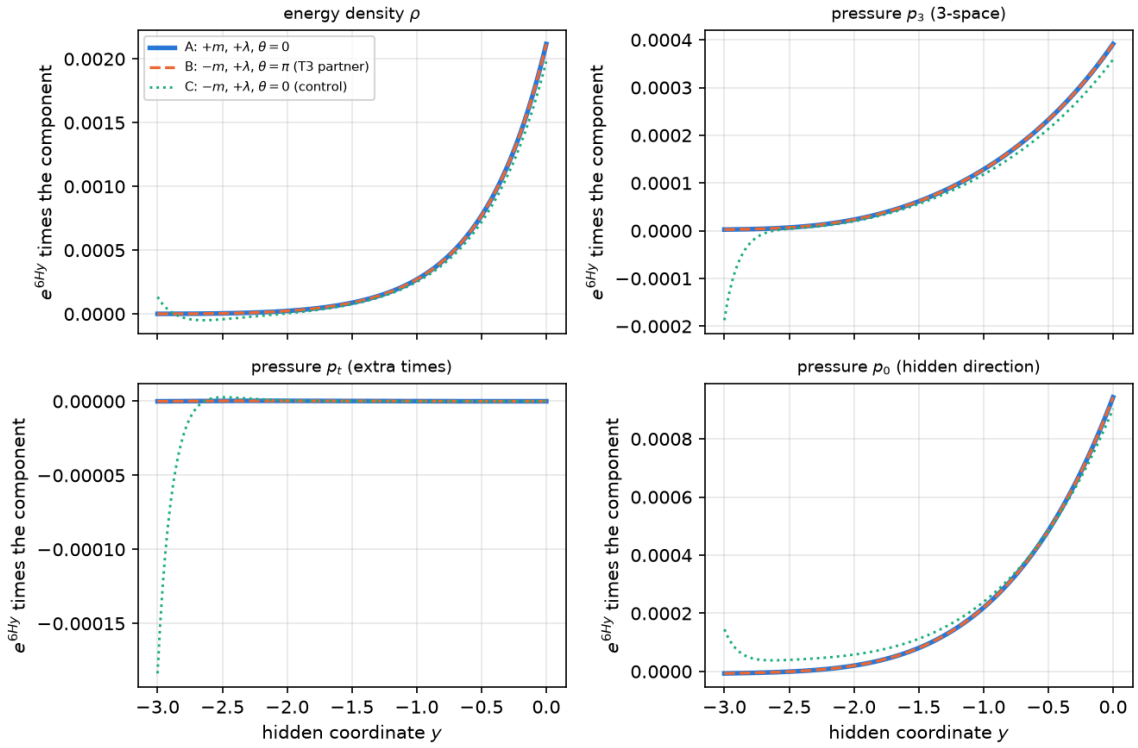

Figure 501: The energy-momentum profiles of A (blue), B (orange, dashed) and C (aqua, dotted) for $N = 136$, $\lambda = \lambda_1$, $a_{4,0} = 1$: energy density ρ, pressures p_3 (3-space), p_t (the deflating extra times) and p_0 (hidden direction), each times the volume factor e^{6Hy}, against the hidden coordinate y (units $|m|^8$ with $H = 1$). The four profiles of the partner B coincide with those of A at every point (T3, statement S4); the control C differs. (Notebook 19a, figure 5.)

Out [16]:

```
Figure 19a.5 saved as Revision/textbook/figures/19a_5_emt_profiles.png
N = 8, B: y-conservation, integrated 3.9e-13, pointwise 2.2e-09 (relative)
PASS N = 8: the partner B obeys p0' + 6H p0 = 3H (p3 + p_t)
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, checks
    emt_y_conservation_integrated, pointwise
N = 136, B: y-conservation, integrated 1.7e-12, pointwise 8.4e-11 (relative)
PASS N = 136: the partner B obeys p0' + 6H p0 = 3H (p3 + p_t)
     reproduces Revision/kohn_sham/reports/ks-rust-solver.json, checks
    emt_y_conservation_integrated, pointwise
```

Text cell:

```
## 13. N = 8: where the zero modes live, and a level of the control by hand
```

```
For $N = 8$ the particles sit in the eight zero modes of momentum $k = 0$. Without
interaction ($\lambda = 0$, so $M_{eff} = M = \pm m$ and $v_v = 0$) the block
equation at $k = 0$ is the first-order system $d\chi/dy = N\chi$ with
$N = M\sigma_3 + ij\varepsilon\sigma_1$ (record `ks-theory.json`), and at
$\varepsilon = 0$ it is $d\chi_1/dy = M\chi_1$, $d\chi_2/dy = -M\chi_2$. So:

- A ($M = +m$, even parity $\chi_2(0) = 0$, tip $\chi_2(-L) = 0$): $\chi_2 = 0$ and
  $\chi = (e^{my}, 0)$, largest at the brane;
- B ($M = -m$, odd parity $\chi_1(0) = 0$, tip angle $\pi$: $\chi_1(-L) = 0$):
  $\chi_1 = 0$ and $\chi = (0, e^{my})$, the T3 image $\sigma_2(e^{my}, 0) =
  (0, ie^{my})$ up to the factor $i$, again largest at the brane;
- C ($M = -m$, even parity, tip $\chi_2(-L) = 0$): $\chi = (e^{-my}, 0)$, largest
  at the TIP.

The coordinate density $e^{6Hy}n(y)$ is proportional to $|\chi|^2$, so its value at
the brane divided by its value at the tip is $e^{2mL} = e^6$ for A and B and
$e^{-2mL} = e^{-6}$ for C. The next cell solves the three $N = 8$ universes without
interaction at $a_{4,0} = 1$ (three runs) and checks these two ratios.
```

In [17]:

```
FREE8 = {kind: universe(kind, 0.0, 1.0, 8, 0.25) for kind in "ABC"}  # lambda = 0
L_TIP, M_BARE = 3.0, 1.0  # the tip distance L and the bare mass |m| of the record


def brane_over_tip(state):
    """Coordinate density e^{6Hy} n(y) at the brane divided by its value at the tip."""
    n = state["profile"]["n"]
    return float(n[-1] * weight[-1] / (n[0] * weight[0]))


expected = {"A": math.exp(2 * M_BARE * L_TIP), "B": math.exp(2 * M_BARE * L_TIP),
            "C": math.exp(-2 * M_BARE * L_TIP)}  # e^{+6}, e^{+6}, e^{-6}
ratio_error = 0.0
for kind in "ABC":
    ratio = brane_over_tip(FREE8[kind])
    ratio_error = max(ratio_error, abs(ratio / expected[kind] - 1.0))
    say(f"{kind}: brane / tip = {ratio:.6e}, closed form {expected[kind]:.6e}")
check(ratio_error < 1e-8,
      "free N = 8: the zero modes of A and B live at the brane, those of C at the tip")
```

Out [17]:

```
A: brane / tip = 4.034288e+02, closed form 4.034288e+02
B: brane / tip = 4.034288e+02, closed form 4.034288e+02
C: brane / tip = 2.478752e-03, closed form 2.478752e-03
PASS free N = 8: the zero modes of A and B live at the brane, those of C at the tip
```

Text cell:

```
The control C has one more surprise: a level inside the gap $|\varepsilon| < m$.
Take the odd parity ($\chi_1(0) = 0$) with the tip condition $\chi_2(-L) = 0$, at
```

```
$k = 0$ with $M = -m$ and $v_v = 0$. Write $D$ for the derivative $d/dy$. The block
equation $D\chi = N\chi$ reads

$$D\chi_1 = -m\chi_1 + ij\varepsilon\chi_2,\qquad
D\chi_2 = ij\varepsilon\chi_1 + m\chi_2 .$$

Differentiate the second equation: $D^2\chi_2 = ij\varepsilon D\chi_1 + mD\chi_2$.
Insert the first equation: $D^2\chi_2 = ij\varepsilon(-m\chi_1 +
ij\varepsilon\chi_2) + mD\chi_2 = -m(ij\varepsilon\chi_1) - \varepsilon^2\chi_2 +
mD\chi_2$ (because $i^2j^2 = -1$). The second equation says $ij\varepsilon\chi_1 =
D\chi_2 - m\chi_2$; insert it: $D^2\chi_2 = -mD\chi_2 + m^2\chi_2 -
\varepsilon^2\chi_2 + mD\chi_2 = (m^2 - \varepsilon^2)\chi_2$. For
$|\varepsilon| < m$ write $q = \sqrt{m^2 - \varepsilon^2}$; the solution with
$\chi_2(-L) = 0$ is $\chi_2 = \sinh(q(y + L))$, because $D^2\sinh(q(y + L)) =
q^2\sinh(q(y + L))$ and $\sinh 0 = 0$. The brane condition $\chi_1(0) = 0$ is
$D\chi_2(0) - m\chi_2(0) = 0$, that is $q\cosh(qL) = m\sinh(qL)$, or

$$\tanh(qL) = q/m .$$

The function $\tanh(qL) - q/m$ is $0$ at $q = 0$, positive just above $0$ (its slope
there is $L - 1/m = 2$) and negative at $q = m$, so it has a root in $(0, m)$; the
level is $\varepsilon_b = \sqrt{m^2 - q^2} = m/\cosh(qL)$ (since $q/m = \tanh(qL)$
and $1 - \tanh^2 = 1/\cosh^2$). For A ($M = +m$) the same steps give
$q\cosh(qL) = -m\sinh(qL)$, which has no root with $q > 0$: A has no level in the
gap in this sector. The next cell finds $q$ by **bisection** (halve an interval
that contains the root, keep the half where the function changes sign, 200 times)
and checks: the Kohn-Sham gap of the free control C is $\varepsilon_b$; the gaps of
A and B are equal to each other and to the gap of the committed state
‘N8_lam0_a10‘.
```

In [18]:

```
def bound_state_q(m, L):
    """The root q in (0, m) of tanh(q L) - q / m, by bisection."""
    low, high = 0.5 * m, m  # the function is positive at m/2 and negative at m
    for _ in range(200):
        middle = 0.5 * (low + high)
        if math.tanh(middle * L) - middle / m > 0.0:
            low = middle  # the root lies above the middle
        else:
            high = middle  # the root lies below the middle
    return 0.5 * (low + high)


q_root = bound_state_q(M_BARE, L_TIP)
eps_b = M_BARE / math.cosh(q_root * L_TIP)  # the level inside the gap
report("q of the control's sub-gap level", f"{q_root:.12f}")
report("sub-gap level eps_b = m / cosh(q L)", f"{eps_b:.12f}", "|m|")
gap = {}
for kind in "ABC":
    homo, lumo = homo_lumo(FREE8[kind])
    gap[kind] = lumo - homo
    say(f"{kind}: HOMO {homo:.3e}, LUMO {lumo:.12f}, Kohn-Sham gap {gap[kind]:.12f}")
```

```
recorded = float(summary["N8_lam0_a10"]["KS_gap"])
same_level = abs(math.sqrt(M_BARE ** 2 - q_root ** 2) - eps_b) < 1e-12  # two forms
check(abs(gap["C"] - eps_b) < 1e-9 and same_level,
      "the free control has the Kohn-Sham gap m/cosh(qL) of the sub-gap level")
check(close(gap["A"], recorded) and close(gap["B"], gap["A"]),
      "the free A and B have the gap of the committed state N8_lam0_a10",
      record=f"{GROUND}/summary.csv, N8_lam0_a10, column KS_gap")
```

Out [18]:

```
RESULT q of the control's sub-gap level = 0.994901528453
RESULT sub-gap level eps_b = m / cosh(q L) = 0.100851121375 |m|
A: HOMO 0.000e+00, LUMO 0.170349251323, Kohn-Sham gap 0.170349251323
B: HOMO 0.000e+00, LUMO 0.170349251323, Kohn-Sham gap 0.170349251323
C: HOMO 0.000e+00, LUMO 0.100851121375, Kohn-Sham gap 0.100851121375
PASS the free control has the Kohn-Sham gap m/cosh(qL) of the sub-gap level
PASS the free A and B have the gap of the committed state N8_lam0_a10
     reproduces Revision/kohn_sham/results/ground/summary.csv, N8_lam0_a10, column KS_gap
```

Text cell:

```
The next cell draws the coordinate density $e^{6Hy}n(y)$ of the three $N = 8$
universes: left without interaction (the thin black dotted lines are the closed
forms $e^{\pm 2my}$, scaled to the brane value), right with the coupling
$\lambda_1$ (the states of section 8). With the interaction, A and B keep their
shape, while the control's zero modes near the tip, where the proper density is
huge, feel a strong mean field and are pushed away from the very end.
```

In [19]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8), sharey=True)
for kind in "ABC":
    left.semilogy(y, weight * FREE8[kind]["profile"]["n"], color=COLOUR[kind],
                  label=NAME[kind], **STYLE[kind])
    right.semilogy(y, weight * SELF[(8, kind)]["profile"]["n"], color=COLOUR[kind],
                   label=NAME[kind], **STYLE[kind])
top = {kind: float(weight[-1] * FREE8[kind]["profile"]["n"][-1]) for kind in "AC"}
left.semilogy(y, top["A"] * np.exp(2 * M_BARE * y), "k:", lw=0.8)  # e^{2 m y}
left.semilogy(y, top["C"] * np.exp(-2 * M_BARE * y), "k:", lw=0.8)  # e^{-2 m y}
left.set_title("$N = 8$, $\\lambda = 0$, $a_{4,0} = 1$", fontsize=10)
right.set_title("$N = 8$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
left.set_ylabel("coordinate density $e^{6Hy}\\,n(y)$")
for ax in (left, right):
    ax.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
right.legend(fontsize=7, loc="lower right")
fig.tight_layout()
save_figure(fig, "zero_modes_n8",
            "The coordinate particle density $e^{6Hy}n(y)$ (particles per unit of "
            "$y$; logarithmic vertical axis) of the $N = 8$ universes A (blue), B "
            "(orange, dashed) and C (aqua, dotted) at $a_{4,0} = 1$, against the "
            "hidden coordinate $y$ from the tip $-3$ to the brane $0$. Left: without "
```

```
"interaction; the thin black dotted lines are the closed forms "
"$e^{2my}$ and $e^{-2my}$. A and B hold their particles in zero modes "
"at the brane; with the untransformed tip (C) the zero modes sit at the "
"tip. Right: with the coupling $\\lambda_1$; A and B are unchanged in "
"shape, the control is reshaped by its strong mean field near the tip.")
```

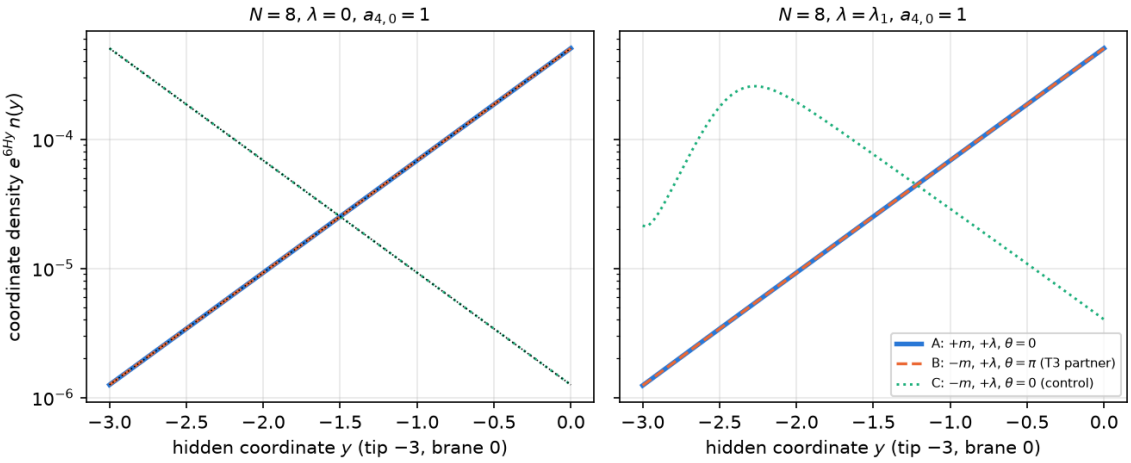

Figure 502: The coordinate particle density $e^{6Hy}n(y)$ (particles per unit of y; logarithmic vertical axis) of the $N = 8$ universes A (blue), B (orange, dashed) and C (aqua, dotted) at $a_{4,0} = 1$, against the hidden coordinate y from the tip -3 to the brane 0. Left: without interaction; the thin black dotted lines are the closed forms e^{2my} and e^{-2my}. A and B hold their particles in zero modes at the brane; with the untransformed tip (C) the zero modes sit at the tip. Right: with the coupling λ_1; A and B are unchanged in shape, the control is reshaped by its strong mean field near the tip. (Notebook 19a, figure 6.)

Out [19]:

```
Figure 19a.6 saved as Revision/textbook/figures/19a_6_zero_modes_n8.png
```

Text cell:

```
## 14. T3 along the deflating history

T3 holds at every slice of the history, because the slice $a_{4,0}$ enters only
through $\kappa = e^{-Hy - a_{4,0}}$, which the map does not touch. The next cell
solves A, B and C at the five slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$ for $N = 136$
and $N = 688$, each with its coupling $\lambda_1$ (27 new runs; the slice
$a_{4,0} = 1$ of $N = 136$ was solved above), and checks at every slice: A
reproduces the energy of the record; B has the levels, profiles and energy of A
(relative energy difference below $10^{-13}$); C differs from A by $0.5$ to $10$
percent.
```

In [20]:

```
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]
HIST = {}  # (N, kind, a4) -> state
for N in (136, 688):
    for a4 in SLICES:
        for kind in "ABC":
            if N == 136 and a4 == 1.0:
                HIST[(N, kind, a4)] = SELF[(N, kind)]  # already solved
            else:
                HIST[(N, kind, a4)] = universe(kind, LAMBDA[N][0], a4, N, 0.45)
ok_record, ok_partner, ok_control = True, True, True
partner_rel, control_pct = [], []  # relative differences A - B, percents A - C
for N in (136, 688):
    for a4 in SLICES:
        A, B, C = (HIST[(N, kind, a4)] for kind in "ABC")
        record_energy = float(summary[f"N{N}_lamp1_a{round(10 * a4):02d}"]["E_KS"])
        ok_record &= close(A["E_KS"], record_energy)
        partner_rel.append(abs(A["E_KS"] - B["E_KS"]) / abs(A["E_KS"]))
        control_pct.append(100.0 * abs(A["E_KS"] - C["E_KS"]) / abs(A["E_KS"]))
        ok_partner &= level_difference(A, B) < TOL and profile_mismatch(A, B) < TOL
        E_A, E_B, E_C = A["E_KS"], B["E_KS"], C["E_KS"]
        say(f"N = {N:3d}, a4,0 = {a4:3.1f}: E_KS A {E_A:.10f}, "
            f"B {E_B:.10f}, C {E_C:.6f}")
report("largest relative E_KS difference A - B", f"{max(partner_rel):.1e}")
report("relative E_KS difference A - C", f"{min(control_pct):.1f} to "
       f"{max(control_pct):.1f}", "percent")
check(ok_record, "A reproduces the recorded E_KS at all 10 states of the history",
      record=f"{GROUND}/summary.csv, N136_lamp1_a00 to a20, N688_lamp1_a00 to a20")
check(ok_partner and max(partner_rel) < 1e-13,
      "B has the levels, energy and profiles of A at every slice")
check(0.5 < min(control_pct) and max(control_pct) < 10.0,
      "the control C differs from A by 0.5 to 10 percent at every slice")
```

Out [20]:

```
N = 136, a4,0 = 0.0: E_KS A 80.2837009244, B 80.2837009244, C 73.179866
N = 136, a4,0 = 0.5: E_KS A 51.2491613321, B 51.2491613321, C 46.475561
N = 136, a4,0 = 1.0: E_KS A 32.3929491532, B 32.3929491532, C 29.283751
N = 136, a4,0 = 1.5: E_KS A 20.2130306754, B 20.2130306754, C 18.533932
N = 136, a4,0 = 2.0: E_KS A 12.4470595905, B 12.4470595905, C 12.175574
N = 688, a4,0 = 0.0: E_KS A 680.4447575708, B 680.4447575708, C 670.715818
N = 688, a4,0 = 0.5: E_KS A 437.5215029073, B 437.5215029073, C 431.359438
N = 688, a4,0 = 1.0: E_KS A 279.4468465636, B 279.4468465636, C 275.745397
N = 688, a4,0 = 1.5: E_KS A 176.7808903556, B 176.7808903556, C 175.297541
N = 688, a4,0 = 2.0: E_KS A 110.4456550192, B 110.4456550192, C 112.097612
RESULT largest relative E_KS difference A - B = 4.0e-14
RESULT relative E_KS difference A - C = 0.8 to 9.6 percent
PASS A reproduces the recorded E_KS at all 10 states of the history
     reproduces Revision/kohn_sham/results/ground/summary.csv, N136_lamp1_a00 to a20,
    N688_lamp1_a00 to a20
PASS B has the levels, energy and profiles of A at every slice
PASS the control C differs from A by 0.5 to 10 percent at every slice
```

Text cell:

The next cell draws the Kohn-Sham energy along the history. As the extra times deflate and 3-space inflates, every 3-momentum is redshifted and the energy of the gas falls; A and B fall together, the control C on its own curve.

In [21]:

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
MARK = {"A": dict(marker="o", ms=7, lw=2.2), "B": dict(marker="s", ms=4, lw=1.4,
                                                       ls="--"),
        "C": dict(marker="^", ms=6, lw=1.2, ls=":"),
        "D": dict(marker="D", ms=5, lw=1.4)}
for ax, N in zip(axes, (136, 688)):
    for kind in "ABC":
        ax.plot(SLICES, [HIST[(N, kind, a4)]["E_KS"] for a4 in SLICES],
                color=COLOUR[kind], label=NAME[kind], **MARK[kind])
    ax.set_xlabel("slice $a_{4,0}$ of the history $a_4 = Hx_4$")
    ax.set_ylabel("Kohn-Sham energy $E_{KS}$ (units of $|m|$)")
    ax.set_title(f"$N = {N}$, $\\lambda = \\lambda_1$", fontsize=10)
axes[0].legend(fontsize=7, loc="upper right")
fig.tight_layout()
save_figure(fig, "history_energy",
            "The Kohn-Sham energy $E_{KS}$ (units of $|m|$) of A (blue circles), B "
            "(orange squares, dashed) and C (aqua triangles, dotted) at the five "
            "slices $a_{4,0} = 0$ to $2$ of the deflating history, for $N = 136$ "
            "(left) and $N = 688$ (right) at the coupling $\\lambda_1$ of each $N$. "
            "The energy falls because the 3-momenta are redshifted as 3-space "
            "inflates and the extra times deflate; the T3 partner B follows A at "
            "every slice, the control C does not.")
```

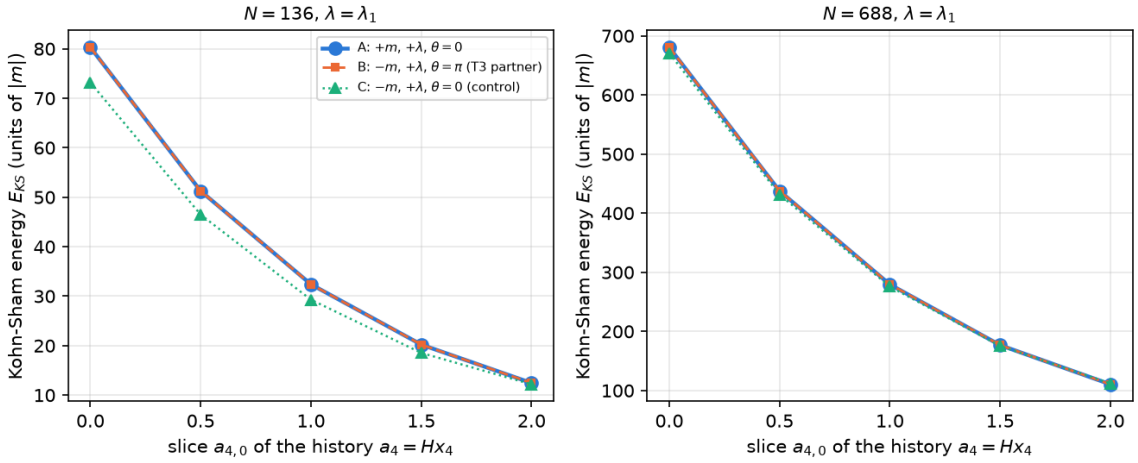

Figure 503: The Kohn-Sham energy E_{KS} (units of $|m|$) of A (blue circles), B (orange squares, dashed) and C (aqua triangles, dotted) at the five slices $a_{4,0} = 0$ to 2 of the deflating history, for $N = 136$ (left) and $N = 688$ (right) at the coupling λ_1 of each N. The energy falls because the 3-momenta are redshifted as 3-space inflates and the extra times deflate; the T3 partner B follows A at every slice, the control C does not. (Notebook 19a, figure 7.)

Out [21]:

```
Figure 19a.7 saved as Revision/textbook/figures/19a_7_history_energy.png
```

Text cell:

```
The next cell draws the same comparison as relative differences on a logarithmic
axis: $|E_A - E_B|/|E_A|$ (partner) and $|E_A - E_C|/|E_A|$ (control) at every
slice, for both particle numbers.
```

In [22]:

```
fig, ax = plt.subplots()
for N, marker in ((136, "o"), (688, "s")):
    for kind in "BC":
        values = [max(abs(HIST[(N, "A", a4)]["E_KS"] - HIST[(N, kind, a4)]["E_KS"])
                      / abs(HIST[(N, "A", a4)]["E_KS"]), FLOOR) for a4 in SLICES]
        ax.semilogy(SLICES, values, marker=marker, color=COLOUR[kind],
                    ls="--" if kind == "B" else ":",
                    label=f"$N = {N}$, A against {kind}")
ax.axhline(TOL, color="0.3", lw=1.0, label="tolerance $10^{-9}$")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("relative difference of $E_{KS}$")
ax.legend(fontsize=7, loc="center right")
save_figure(fig, "history_agreement",
            "Relative differences of the Kohn-Sham energy between A and its T3 "
            "partner B (orange) and between A and the control C (aqua) at the five "
            "slices of the history, for $N = 136$ (circles) and $N = 688$ (squares); "
            "logarithmic vertical axis, exact zeros drawn at $10^{-17}$. The "
            "partner agrees to better than $10^{-13}$ at every slice: T3 holds slice "
            "by slice along the deflating history. The control is off by $0.5$ to "
            "$10$ percent.")
```

Out [22]:

```
Figure 19a.8 saved as Revision/textbook/figures/19a_8_history_agreement.png
```

Text cell:

```
## 15. The coupling must keep its sign

T3 pairs $(m, \lambda)$ with $(-m, +\lambda)$. What about $(-m, -\lambda)$, the
parameter change of the classical theorem T1? The next cell solves, for $N = 136$ at
$a_{4,0} = 1$, the five couplings $\lambda = -\lambda_2, -\lambda_1, 0, +\lambda_1,
+\lambda_2$ for all four universes A, B, C, D (with the record's margins $0.85$,
$0.45$, $0.25$). It checks: A reproduces the record at every coupling; B equals A;
D at $\lambda$ equals A at $-\lambda$ (that is T3 applied to the state with
$-\lambda$), so D differs from A whenever $\lambda \ne 0$; C differs from A by $1$
to $3.5$ at every coupling.
```

In [23]:

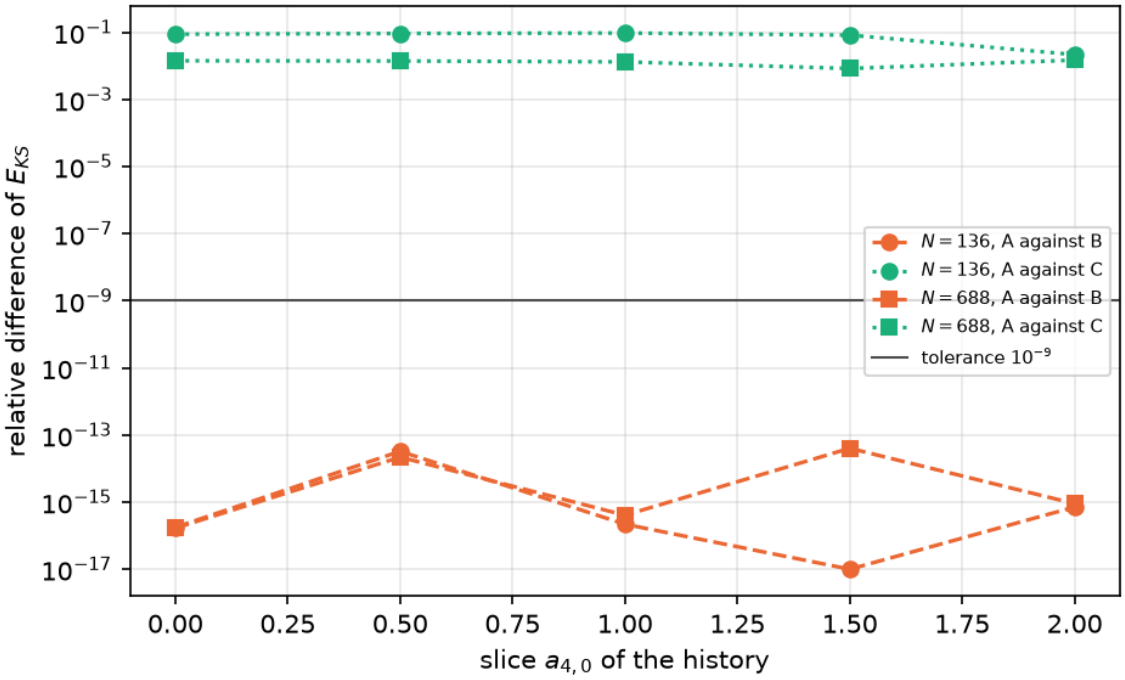

Figure 504: Relative differences of the Kohn-Sham energy between A and its T3 partner B (orange) and between A and the control C (aqua) at the five slices of the history, for $N = 136$ (circles) and $N = 688$ (squares); logarithmic vertical axis, exact zeros drawn at 10^{-17}. The partner agrees to better than 10^{-13} at every slice: T3 holds slice by slice along the deflating history. The control is off by 0.5 to 10 percent. (Notebook 19a, figure 8.)

```
l1, l2 = LAMBDA[136]
COUPLINGS = [("lamm2", -l2, 0.85), ("lamm1", -l1, 0.45), ("lam0", 0.0, 0.25),
             ("lamp1", l1, 0.45), ("lamp2", l2, 0.85)]  # tag, lambda, margin
SCAN = {}  # (tag, kind) -> state
for tag, lam, margin in COUPLINGS:
    for kind in "ABCD":
        if tag == "lamp1" and kind in "ABC":
            SCAN[(tag, kind)] = SELF[(136, kind)]  # already solved
        else:
            SCAN[(tag, kind)] = universe(kind, lam, 1.0, 136, margin)
MIRROR = {"lamm2": "lamp2", "lamm1": "lamp1", "lam0": "lam0", "lamp1": "lamm1",
          "lamp2": "lamm2"}  # the tag of -lambda
ok_record, ok_partner, ok_mirror, ok_wrong = True, True, True, True
control_gap = []  # |E_C - E_A| at each coupling
for tag, lam, margin in COUPLINGS:
    E = {kind: SCAN[(tag, kind)]["E_KS"] for kind in "ABCD"}
    ok_record &= close(E["A"], float(summary[f"N136_{tag}_a10"]["E_KS"]))
    ok_partner &= close(E["A"], E["B"]) and level_difference(
        SCAN[(tag, "A")], SCAN[(tag, "B")]) < TOL
    ok_mirror &= close(E["D"], SCAN[(MIRROR[tag], "A")]["E_KS"])
    ok_wrong &= (lam == 0.0) or abs(E["D"] - E["A"]) > 1e-4
    control_gap.append(abs(E["C"] - E["A"]))
    E_A, E_B, E_C, E_D = (E[kind] for kind in "ABCD")
    say(f"lambda = {lam:+.4e}: A {E_A:.10f}  B {E_B:.10f}  "
        f"C {E_C:.6f}  D {E_D:.10f}")
check(ok_record, "A reproduces the recorded E_KS for all five couplings",
      record=f"{GROUND}/summary.csv, N136_lamm2_a10 to N136_lamp2_a10")
check(ok_partner, "B = A for every coupling: the partner keeps +lambda")
check(ok_mirror and ok_wrong,
      "D(lambda) = A(-lambda), so (-m, -lambda) is not the partner for lambda != 0",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.mean_field_map (control)")
check(1.0 < min(control_gap) and max(control_gap) < 3.5,
      "the control C differs from A by 1 to 3.5 at every coupling")
```

Out [23]:

```
lambda = -2.7890e-03: A 32.3593734113  B 32.3593734113  C 34.540794  D 32.4116192324
lambda = -9.2980e-04: A 32.3755880337  B 32.3755880337  C 33.529490  D 32.3929491532
lambda = +0.0000e+00: A 32.3841156874  B 32.3841156874  C 30.731389  D 32.3841156874
lambda = +9.2980e-04: A 32.3929491532  B 32.3929491532  C 29.283751  D 32.3755880337
lambda = +2.7890e-03: A 32.4116192324  B 32.4116192324  C 28.980584  D 32.3593734113
PASS A reproduces the recorded E_KS for all five couplings
     reproduces Revision/kohn_sham/results/ground/summary.csv, N136_lamm2_a10 to
    N136_lamp2_a10
PASS B = A for every coupling: the partner keeps +lambda
PASS D(lambda) = A(-lambda), so (-m, -lambda) is not the partner for lambda != 0
     reproduces Revision/pairing/kohn_sham/reports/python-t3.json, check
    T3.mean_field_map (control)
PASS the control C differs from A by 1 to 3.5 at every coupling
```

Text cell:

```
The next cell draws the energies of the scan. Left: $E_{KS} - E_{KS}(\lambda = 0)$
of A, B and D against $\lambda/\lambda_1$; D is the mirror image of A. Right: the
energies of A and of the control C. C lies above A for the negative couplings and
below A for the others: its zero modes near the tip feel the coupling much more
strongly than those of A.
```

In [24]:

```
x = [lam / l1 for tag, lam, margin in COUPLINGS]
E0 = SCAN[("lam0", "A")]["E_KS"]
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
for kind in "ABD":
    left.plot(x, [SCAN[(tag, kind)]["E_KS"] - E0 for tag, lam, margin in COUPLINGS],
              color=COLOUR[kind], label=NAME[kind], **MARK[kind])
left.set_xlabel("coupling $\\lambda/\\lambda_1$")
left.set_ylabel("$E_{KS} - E_{KS}(\\lambda = 0)$ (units of $|m|$)")
left.axhline(0.0, color="0.3", lw=0.8)
low, high = left.get_ylim()
left.set_ylim(low, high + 0.6 * (high - low))  # room for the legend at the top
left.legend(fontsize=7, loc="upper center")
for kind in "AC":
    right.plot(x, [SCAN[(tag, kind)]["E_KS"] for tag, lam, margin in COUPLINGS],
               color=COLOUR[kind], label=NAME[kind], **MARK[kind])
right.set_xlabel("coupling $\\lambda/\\lambda_1$")
right.set_ylabel("$E_{KS}$ (units of $|m|$)")
right.legend(fontsize=7, loc="upper right")
fig.suptitle("$N = 136$, slice $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
save_figure(fig, "coupling_scan",
            "Left: the interaction part of the Kohn-Sham energy, $E_{KS}$ minus its "
            "value at $\\lambda = 0$ (units of $|m|$), against the coupling "
            "$\\lambda/\\lambda_1$ for $N = 136$, $a_{4,0} = 1$: universe A (blue), "
            "its T3 partner B with the same coupling (orange, dashed, on top of A) "
            "and D with the reversed coupling (yellow diamonds), which is the "
            "mirror image of A. Right: $E_{KS}$ of A and of the control C (aqua), "
            "which lies $1$ to $3.5$ away from A, above it for negative and below "
            "it for positive couplings. Only $(-m, +\\lambda)$ with the transformed "
            "tip is the partner of $(m, \\lambda)$.")
```

Out [24]:

```
Figure 19a.9 saved as Revision/textbook/figures/19a_9_coupling_scan.png
```

Text cell:

```
## 16. Thermal states: equal occupations, chemical potential and free energy

At a temperature $T > 0$ the occupations are the Fermi (Mermin) occupations
$f = 1/(1 + e^{(\varepsilon - \mu)/T})$, with the chemical potential $\mu$ fixed by
the particle number. Since the levels of B are those of A, so are $f$, $\mu$, the
entropy and the free energy $F = E - T S_{ent}$. The next cell solves A, B, C for
```

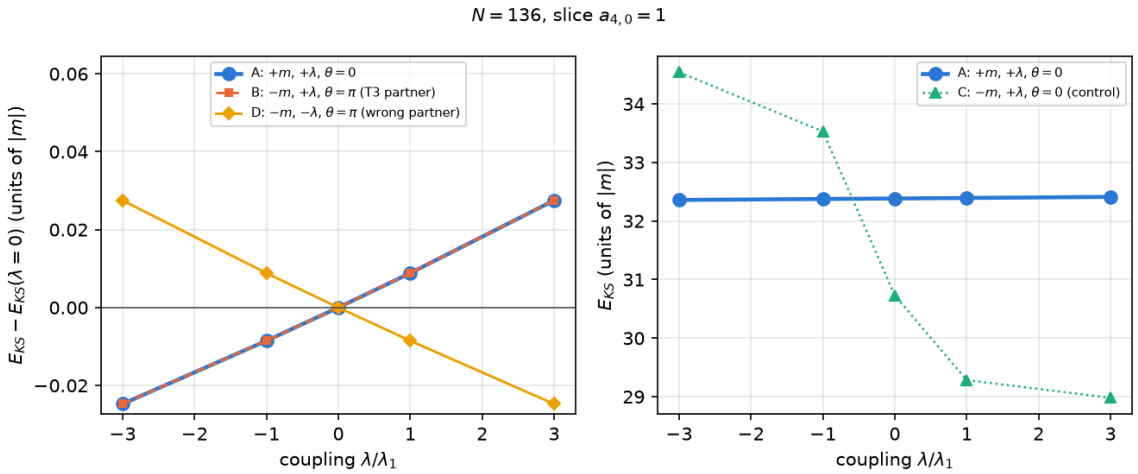

Figure 505: Left: the interaction part of the Kohn-Sham energy, E_{KS} minus its value at $\lambda = 0$ (units of $|m|$), against the coupling λ/λ_1 for $N = 136$, $a_{4,0} = 1$: universe A (blue), its T3 partner B with the same coupling (orange, dashed, on top of A) and D with the reversed coupling (yellow diamonds), which is the mirror image of A. Right: E_{KS} of A and of the control C (aqua), which lies 1 to 3.5 away from A, above it for negative and below it for positive couplings. Only $(-m, +\lambda)$ with the transformed tip is the partner of (m, λ). (Notebook 19a, figure 9.)

```
$N = 136$, $\lambda_1$, $a_{4,0} = 1$ at the three temperatures of the record
$T = 0.01, 0.02, 0.05$ (margin $0.4$, as the record's thermal states), checks that A
reproduces the record's $\mu$, $E$, entropy and $F$, that B equals A and that the
free energy of C lies $2.5$ to $3.5$ below that of A.
```

In [25]:

```
THERMO = read_rows("Revision/kohn_sham/results/thermo/thermodynamics.csv")
TEMPS = [0.01, 0.02, 0.05]
WARM = {}  # (T, kind) -> state
ok_record, ok_partner, ok_control = True, True, True
for T in TEMPS:
    for kind in "ABC":
        WARM[(T, kind)] = universe(kind, l1, 1.0, 136, 0.4, T=T)
    A, B, C = (WARM[(T, kind)] for kind in "ABC")
    F = {kind: WARM[(T, kind)]["E_KS"] - T * WARM[(T, kind)]["entropy"]
         for kind in "ABC"}  # free energy F = E - T S
    row = THERMO[f"N136_lamp1_a10_T{round(1000 * T)}"]
    ok_record &= (close(A["mu_or_fermi_level"], float(row["mu"]))
                  and close(A["E_KS"], float(row["E"]))
                  and close(A["entropy"], float(row["entropy"]))
                  and close(F["A"], float(row["F"])))
    ok_partner &= (close(A["mu_or_fermi_level"], B["mu_or_fermi_level"])
                   and close(A["entropy"], B["entropy"]) and close(F["A"], F["B"])
                   and level_difference(A, B) < TOL)
    ok_control &= 2.5 < F["A"] - F["C"] < 3.5
    mu_A, mu_B, mu_C = (s["mu_or_fermi_level"] for s in (A, B, C))
    F_A, F_B, F_C = (F[kind] for kind in "ABC")
```

```
    say(f"T = {T:.2f}: mu A {mu_A:.12f} B {mu_B:.12f} C {mu_C:.6f}; "
        f"F A {F_A:.9f} B {F_B:.9f} C {F_C:.5f}")
check(ok_record, "A reproduces mu, E, entropy and F of the three thermal states",
      record="Revision/kohn_sham/results/thermo/thermodynamics.csv, "
             "N136_lamp1_a10_T10, T20, T50")
check(ok_partner, "B has the occupations, mu, entropy and F of A at every T",
      record="Revision/pairing/kohn_sham/t3-theory.json, statement S4")
check(ok_control, "the control C has a free energy 2.5 to 3.5 below A at every T")
```

Out [25]:

```
T = 0.01: mu A 0.335053852472 B 0.335053852472 C 0.328004; F A 32.236662364 B
    32.236662364 C 29.08234
T = 0.02: mu A 0.324659275278 B 0.324659275278 C 0.318384; F A 31.588604294 B
    31.588604294 C 28.51026
T = 0.05: mu A 0.295073925379 B 0.295073925379 C 0.289068; F A 27.091045285 B
    27.091045285 C 24.21400
PASS A reproduces mu, E, entropy and F of the three thermal states
     reproduces Revision/kohn_sham/results/thermo/thermodynamics.csv, N136_lamp1_a10_T10,
    T20, T50
PASS B has the occupations, mu, entropy and F of A at every T
     reproduces Revision/pairing/kohn_sham/t3-theory.json, statement S4
PASS the control C has a free energy 2.5 to 3.5 below A at every T
```

Text cell:

```
The next cell draws, for $T = 0.05$, the occupation $f$ of every level of A, B and
C against its energy (left; the curve is the Fermi function of A), and the free
energy $F$ against the temperature (right).
```

In [26]:

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
T = 0.05
DOTS = {"A": dict(marker="o", ms=7, ls="none", mfc="none", mew=1.6),
        "B": dict(marker="x", ms=5, ls="none", mew=1.4),
        "C": dict(marker="^", ms=4, ls="none")}
for kind in "ABC":
    lv = WARM[(T, kind)]["levels"]
    left.plot([v[4] for v in lv], [v[6] for v in lv], color=COLOUR[kind],
              label=NAME[kind], **DOTS[kind])
mu = WARM[(T, "A")]["mu_or_fermi_level"]
grid = np.linspace(-0.3, 1.0, 400)
left.plot(grid, 1.0 / (1.0 + np.exp((grid - mu) / T)), color="0.4", lw=1.0,
          label="Fermi function of A")
left.set_xlim(-0.3, 1.0)
left.set_xlabel("level $\\varepsilon$ (units of $|m|$)")
left.set_ylabel("occupation $f$")
left.set_title(f"$T = {T}$", fontsize=10)
left.legend(fontsize=6, loc="lower left")
for kind in "ABC":
```

```
    right.plot(TEMPS, [WARM[(t, kind)]["E_KS"] - t * WARM[(t, kind)]["entropy"]
                       for t in TEMPS], color=COLOUR[kind], label=NAME[kind],
               **MARK[kind])
right.set_xlabel("temperature $T$ (units of $|m|$)")
right.set_ylabel("free energy $F = E - TS_{ent}$ (units of $|m|$)")
fig.suptitle("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
save_figure(fig, "thermal_states",
            "Thermal Kohn-Sham states for $N = 136$, $\\lambda = \\lambda_1$, "
            "$a_{4,0} = 1$. Left: the occupation $f$ of every level against its "
            "energy (units of $|m|$) at $T = 0.05$ for A (blue rings), B (orange "
            "crosses) and C (aqua triangles); the grey curve is the Fermi function "
            "of A. Every cross of B sits in a ring of A. Right: the free energy "
            "$F = E - TS_{ent}$ against the temperature $T$ (units of $|m|$); A and "
            "B coincide, the control C lies $2.5$ to $3.5$ lower.")
```

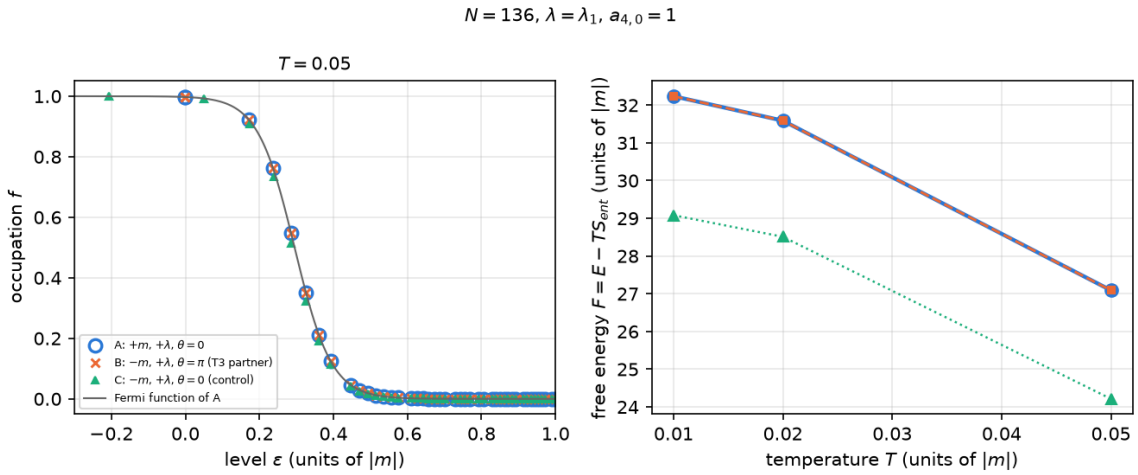

Figure 506: Thermal Kohn-Sham states for $N = 136$, $\lambda = \lambda_1$, $a_{4,0} = 1$. Left: the occupation f of every level against its energy (units of $|m|$) at $T = 0.05$ for A (blue rings), B (orange crosses) and C (aqua triangles); the grey curve is the Fermi function of A. Every cross of B sits in a ring of A. Right: the free energy $F = E - TS_{ent}$ against the temperature T (units of $|m|$); A and B coincide, the control C lies 2.5 to 3.5 lower. (Notebook 19a, figure 10.)

Out [26]:

```
Figure 19a.10 saved as Revision/textbook/figures/19a_10_thermal_states.png
```

Text cell:

```
## 17. The same runs in the record's demonstration of T3

The Revision record has its own numerical demonstration of T3
(‘Revision/pairing/kohn_sham/reports/t3-rust-demo.json‘, 7 checks). With the same
solver it solved the members "plus" (our A), "image" (our B) and "control" (our C)
for all 210 states of the canonical matrix, and it stored their numbers in the table
```

‘Revision/pairing/kohn_sham/numerics/results/t3-rust-states.csv‘, one row per state. The runs of this notebook cover 19 of these states: 7 ground states at the slice $a_{4,0} = 1$ (sections 8, 13 and 15), the other 9 states of the history (section 14) and 3 thermal states (section 16). The next cell reads the report and the table and prints the record’s own largest deviations. Then it checks, state by state: A equals the record’s plus member (E_{KS}); B equals its image member (E_{KS}, μ, the entropy and the four EMT integrals); C equals its control (E_{KS}); and D at the coupling λ equals the record’s image member at $-\lambda$. Last, it measures in each of the three groups the largest deviation between A and B: the levels, E_{KS}, μ, the entropy, the EMT integrals and the ten profile columns (even ones equal, odd ones opposite), each relative as in the cells above.

What this agreement tests. The solver follows the angle ϕ of the real form, $(a, b) = r(\cos\phi, \sin\phi)$. The map of T3, $(a, b, j, M) \to (b, a, -j, -M)$, turns ϕ into $\pi/2 - \phi$, and the equation of the angle of B is the equation of A with ϕ replaced by $\pi/2 - \phi$. So the solver does the same arithmetic for A and B, and their agreement to the last digits is guaranteed by construction: it tests the transformed boundary conditions, the labels, the filling, the self-consistency loop and the Mermin root, not the discretisation. The record’s second demonstration (‘t3-reference-demo.json‘), with the independent finite-difference reference solver, gives the test that does not depend on the discretisation.

In [27]:

```
DEMO = "Revision/pairing/kohn_sham/reports/t3-rust-demo.json"
DEMO_TABLE = "Revision/pairing/kohn_sham/numerics/results/t3-rust-states.csv"
demo = json.loads(repository_file(DEMO).read_text(encoding="utf-8"))
verdict = {c["name"]: c["verdict"] for c in demo["checks"]}
detail = {c["name"]: c["detail"] for c in demo["checks"]}
passed = sum(v == "PASS" for v in verdict.values())
say(f"record: {demo['states']} states, {passed} of {len(verdict)} checks pass")
for name in ("t3_equal_ground_states", "t3_equal_thermal_states"):
    worst = re.search(r"worst deviation (\S+),", detail[name]).group(1)
    say(f"record, {name}: worst deviation {worst}")
DEMO_ROWS = read_rows(DEMO_TABLE)
PAIRS, GROUP = {}, {}  # state id -> [A, B, C]; state id -> group of runs


def add(state_id, group, states):
    """Store the states A, B, C of one state of the record, once."""
    if state_id not in PAIRS:
        PAIRS[state_id], GROUP[state_id] = states, group


for N in (8, 136):
    add(f"N{N}_lamp1_a10", "ground", [SELF[(N, kind)] for kind in "ABC"])
add("N8_lam0_a10", "ground", [FREE8[kind] for kind in "ABC"])
for tag, lam, margin in COUPLINGS:
    add(f"N136_{tag}_a10", "ground", [SCAN[(tag, kind)] for kind in "ABC"])
for N in (136, 688):
    for a4 in SLICES:
        add(f"N{N}_lamp1_a{round(10 * a4):02d}", "history",
```

```
            [HIST[(N, kind, a4)] for kind in "ABC"])
for T in TEMPS:
    add(f"N136_lamp1_a10_T{round(1000 * T)}", "thermal",
        [WARM[(T, kind)] for kind in "ABC"])


def relative(a, b):
    """|a - b| relative to max(|a|, 1), the measure of close()."""
    return abs(a - b) / max(abs(a), 1.0)


worst_A, worst_B, worst_C = 0.0, 0.0, 0.0
for state_id, (A, B, C) in PAIRS.items():
    row = DEMO_ROWS[state_id]  # the record's row of this state
    worst_A = max(worst_A, relative(A["E_KS"], float(row["E_KS_plus"])))
    image = [(B["E_KS"], row["E_KS_image"]), (B["mu_or_fermi_level"],
             row["mu_image"]), (B["entropy"], row["entropy_image"])]
    image += [(B["emtIntegrals_2Vol7_int_e6Hy"][c], row[f"int_{c}_image"])
              for c in ("rho", "p3", "p_t", "p0")]
    worst_B = max([worst_B] + [relative(x, float(r)) for x, r in image])
    control = float(row["E_KS_control_untransformed_tip"])
    worst_C = max(worst_C, relative(C["E_KS"], control))
worst_D = max(relative(SCAN[(tag, "D")]["E_KS"],
                       float(DEMO_ROWS[f"N136_{MIRROR[tag]}_a10"]["E_KS_image"]))
              for tag, lam, margin in COUPLINGS)
say(f"{len(PAIRS)} states of the record solved here; largest relative differences "
    f"from the record: A {worst_A:.1e}, B {worst_B:.1e}, C {worst_C:.1e}, "
    f"D {worst_D:.1e}")


def t3_deviation(A, B):
    """Largest deviation of B from A under T3 (levels, E_KS, mu, entropy, EMT
    integrals, profiles with the odd columns reversed)."""
    return max(level_difference(A, B), relative(A["E_KS"], B["E_KS"]),
               relative(A["mu_or_fermi_level"], B["mu_or_fermi_level"]),
               relative(A["entropy"], B["entropy"]), integral_difference(A, B),
               profile_mismatch(A, B))


largest = {}
for group in ("ground", "history", "thermal"):
    ids = [state_id for state_id in PAIRS if GROUP[state_id] == group]
    largest[group] = max(t3_deviation(*PAIRS[state_id][:2]) for state_id in ids)
    say(f"{group}: {len(ids)} pairs A, B; largest T3 deviation "
        f"{largest[group]:.1e}")
check(passed == len(verdict) == 7 and demo["states"] == 210,
      "the record's T3 demonstration passes 7 of 7 checks in 210 states",
      record=DEMO)
check(len(PAIRS) == 19 and worst_A < TOL and worst_B < TOL,
      "A and B equal the record's plus and image members in 19 states",
      record=f"{DEMO}, checks t3_equal_ground_states, t3_equal_thermal_states")
check(worst_C < TOL, "C equals the record's control with the untransformed tip",
      record=f"{DEMO}, check negative_control_untransformed_tip")
check(worst_D < TOL, "D at lambda equals the record's image member at -lambda",
```

```
      record=f"{DEMO}, check negative_control_lambda_sign")
check(max(largest.values()) < TOL,
      "B equals A under T3 in all 19 pairs (ground, history, thermal)")
```

Out [27]:

```
record: 210 states, 7 of 7 checks pass
record, t3_equal_ground_states: worst deviation 2.179e-13
record, t3_equal_thermal_states: worst deviation 3.877e-12
19 states of the record solved here; largest relative differences from the record: A
    4.0e-14, B 2.8e-13, C 7.0e-14, D 0.0e+00
ground: 7 pairs A, B; largest T3 deviation 9.9e-14
history: 9 pairs A, B; largest T3 deviation 1.0e-13
thermal: 3 pairs A, B; largest T3 deviation 8.1e-13
PASS the record's T3 demonstration passes 7 of 7 checks in 210 states
     reproduces Revision/pairing/kohn_sham/reports/t3-rust-demo.json
PASS A and B equal the record's plus and image members in 19 states
     reproduces Revision/pairing/kohn_sham/reports/t3-rust-demo.json, checks
    t3_equal_ground_states, t3_equal_thermal_states
PASS C equals the record's control with the untransformed tip
     reproduces Revision/pairing/kohn_sham/reports/t3-rust-demo.json, check
    negative_control_untransformed_tip
PASS D at lambda equals the record's image member at -lambda
     reproduces Revision/pairing/kohn_sham/reports/t3-rust-demo.json, check
    negative_control_lambda_sign
PASS B equals A under T3 in all 19 pairs (ground, history, thermal)
```

Text cell:

```
## 18. The last check

The last cell checks that the ten figure files exist in the folder
Revision/textbook/figures, prints how many times the solver was run, and prints
the number of checks that passed.
```

In [28]:

```
runs = len({id(state) for state in [*SELF.values(), *FREE8.values(), *HIST.values(),
                                    *SCAN.values(), *WARM.values()]})
report("runs of the Rust solver in this notebook", runs)
names = ["level_ladder", "level_differences", "density_profiles",
         "mass_and_potential", "emt_profiles", "zero_modes_n8", "history_energy",
         "history_agreement", "coupling_scan", "thermal_states"]
paths = [output_file(f"{FIGURE_FOLDER}/19a_{k}_{name}.png")
         for k, name in enumerate(names, 1)]
check(runs == 62 and all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [28]:

```
RESULT runs of the Rust solver in this notebook = 62
PASS every figure file of this notebook exists
ALL 37 CHECKS PASSED (notebook 19a)
```

Text cell:

```
## 19. What this notebook showed

- PROVED in the records and re-checked here exactly: the author's eight gamma
  matrices are real $16 \times 16$ matrices with entries $-1$, $0$, $+1$ that obey
  $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\mathbf{1}$; their product
  $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$ is
  $\mathrm{diag}(-1, \dots, -1, +1, \dots, +1)$ and maps every Kohn-Sham block
  $(j, s_2, s_3)$ onto the block $(-j, s_2, s_3)$ as $s_2\sigma_2$: the map of T3.
- COMPUTED (62 runs of the Rust Kohn-Sham solver, which compare A and B in 19
  states: 7 ground states at $a_{4,0} = 1$, 9 more states of the history and 3
  thermal states; the other runs are the controls C and D): the universe of mass
  $-m$ with the same coupling $+\lambda$ and the transformed tip angle $\pi$ (B) has
  the same Kohn-Sham levels, level by level with the block type and the brane
  parity exchanged, the same occupations, chemical potential, entropy, energy and
  free energy, and the same particle density, potential and energy-momentum
  profiles $\rho$, $p_3$, $p_t$, $p_0$ as the universe of mass $+m$ (A), while its
  scalar density, density $Q$ and effective mass are exactly opposite. The largest
  deviations (section 17) are about $10^{-13}$ for the ground states and the
  history and below $10^{-12}$ for the thermal states: the rounding of the
  computer. This holds for 8, 136 and 688 particles, at every slice of the
  deflating history, for every coupling and at every temperature computed.
- What this agreement tests: the solver's shooting is covariant under the map (the
  angle equation of B is that of A with $\phi \to \pi/2 - \phi$), so the agreement
  to rounding is guaranteed by construction; it tests the solver's handling of the
  transformed boundary conditions, the labels, the filling, the self-consistency
  and the Mermin root. The test that does not depend on the discretisation is the
  record's reference demonstration (`t3-reference-demo.json`).
- COMPUTED: the runs equal the record's own demonstration (`t3-rust-demo.json`,
  table `t3-rust-states.csv`) in the 19 states that both solve: A its plus
  member, B its image member, C its control, D its image at $-\lambda$.
- COMPUTED: the solver's own T3 self-test of the record
  (`ks-rust-solver.json`, check `t3_block_map_solver_selftest`) is reproduced, and
  every A run reproduces the committed canonical state.
- COMPUTED (negative controls): with the untransformed tip (C) the zero modes move
  to the tip, a level appears inside the gap at $\varepsilon_b = m/\cosh(qL)$ with
  $\tanh(qL) = q/m$ (the closed form derived here), and every energy differs; with
  the reversed coupling (D) the energy is that of A at $-\lambda$, not at $\lambda$.
- PROVED elsewhere (records `wolfram-t3.json`, `python-t3.json`, and the
  completion `t3-completion.json`): theorem T3 itself. The runs here demonstrate it
  numerically on the states solved; they are not its proof.
- ASSUMED: the $Z_2$ brane; the tip is a chosen cutoff and T3 needs the transformed
  tip angle; the history is a PRESCRIBED BACKGROUND; mean field without correlation.
- NOT shown: that a universe is created, in pairs or otherwise. T3 maps solutions
  onto solutions; it gives no creation process, rate or amplitude, and it does not
  force the partner to exist.
```

19.21 Line-by-line walk-through of Notebook 19a

The notebook has 28 code cells, In [1] to In [28]; the numbers they print are in Section 19.20 under the labels Out [k].

In [1], the set-up cell. Its first 298 lines are the complete run instructions of Section 19.19 as comment lines, followed by a line of dashes. The code after the title THE SET-UP is the set-up code of Notebook 19b, explained line by line in Section 19.16, with three differences: the name is `NOTEBOOK_ID = "19a"`, and, because this notebook runs a Rust program, the imports contain two more lines and the cell defines one more function.

```
import shutil  # finds the program cargo
import subprocess  # runs cargo and the Rust programs
```

`shutil` can find a program on the computer, here cargo, the build tool of Rust; `subprocess` can run a program and collect what it prints.

```
def rust_program(manifest, binary):
    """Build the Rust program binary of the crate whose Cargo.toml is manifest (a
    repository path) with "cargo build --release" (about a second when it is up to date;
    minutes the first time) and return the path of the program."""
    if shutil.which("cargo") is None:
        raise FileNotFoundError(
            "cargo was not found: install Rust from https://rustup.rs, open a new "
            "terminal, activate the environment and start JupyterLab again")
    crate = (REPO / manifest).parent  # the folder that holds Cargo.toml
    completed = subprocess.run(
        ["cargo", "build", "--release", "--manifest-path", str(REPO / manifest),
         "--target-dir", str(crate / "target")],
        capture_output=True, text=True, encoding="utf-8", errors="replace")
    if completed.returncode != 0:  # show the end of cargo's error message
        print(completed.stderr[-3000:])
        raise RuntimeError(f"cargo build failed for {manifest}")
    for file_name in (binary, binary + ".exe"):  # Linux and macOS; Windows
        path = crate / "target" / "release" / file_name
        if path.is_file():
            say(f"Rust program {binary} is built and ready.")
            return path
    raise FileNotFoundError(f"cargo built {manifest} but {binary} is missing")
```

`rust_program` builds a Rust program and returns where it is. `shutil.which("cargo")` looks for cargo; if it is not found (`is None`), the function stops with a message that says how to install Rust. A **crate** is a Rust package; `crate` is the folder that holds its file `Cargo.toml` (`.parent` removes the last part of a path). `subprocess.run([...])` runs cargo with the arguments in the list, exactly as if the release build command for this manifest, with the build folder `target` next to `Cargo.toml`, were typed into a terminal; `capture_output=True` keeps cargo's messages instead of printing them, and `text=True` with `encoding="utf-8"` reads them as text (`errors="replace"` replaces a character that cannot be read instead of stopping). The **return code** of a program is 0 when it succeeded; otherwise the function prints the last 3000 characters of cargo's error messages (`[-3000:]` takes the end of a string) and stops. The `for` loop tries the program's name without and with the

ending `.exe` (Linux and macOS have no ending, Windows has `.exe`) and returns the first one that exists, after printing that the program is ready.

In [2], the eight gammas and Γ.

```
import csv  # reads the tables (CSV files) of the record
import math  # pi, exp, tanh and cosh of single numbers
import re  # regular expressions: reads numbers out of the record's text

import numpy as np  # arrays of numbers
```

`csv`, `math` and `numpy` as in Notebook 19b; `re` finds patterns in text (**regular expressions**), used in In [6] to read numbers out of a sentence of the record.

```
GAMMAS = "Revision/algebra/gammas.json"  # the record of the gamma matrices
ALGEBRA = "Revision/algebra/reports/wolfram-algebra.json"  # its check report
fixture = json.loads(repository_file(GAMMAS).read_text(encoding="utf-8"))
stored_real = all(isinstance(g, list) for g in fixture["gamma"])  # no complex part
gamma = [np.array(g, dtype=float) for g in fixture["gamma"]]  # gamma[k] is xk
eta = np.array(fixture["eta"], dtype=float)  # +1 +1 +1 +1 -1 -1 -1 -1 (x0, ..., x7)
entries = sorted({float(x) for g in gamma for x in g.flat})  # all values used
say(f"{len(gamma)} matrices of size {gamma[0].shape}; their entries: {entries}")
```

The same reading as In [2] of Notebook 19b, but with the matrices as arrays of decimal numbers (`dtype=float`); whole numbers up to 2^{53} are stored exactly in this format, so the products below are still exact. `entries` lists the values that occur; Out [2] prints −1.0, 0.0, 1.0.

```
worst = 0.0  # largest violation of the anticommutation rule
for a in range(8):
    for b in range(8):
        anti = gamma[a] @ gamma[b] + gamma[b] @ gamma[a]
        rule = 2.0 * eta[a] * (a == b) * np.eye(16)  # 2 eta^ab times the unit
        worst = max(worst, float(np.max(np.abs(anti - rule))))
Gamma = gamma[0]  # gamma^(x0) first ...
for a in range(1, 8):
    Gamma = Gamma @ gamma[a]  # ... times gamma^(x1), gamma^(x2), ..., gamma^(x7)
diagonal = np.diag([-1.0] * 8 + [1.0] * 8)
anticommutes = all(np.array_equal(Gamma @ g, -(g @ Gamma)) for g in gamma)
```

The anticommutation rule for all 64 pairs and the chirality Γ in the author's order, exactly as In [4] and In [5] of Notebook 19b (Section 19.16).

```
check(stored_real and entries == [-1.0, 0.0, 1.0] and worst == 0.0,
      "the eight gamma matrices are real and obey g^a g^b + g^b g^a = 2 eta^ab",
      record=f"{ALGEBRA}, checks reality and Clifford_relation")
check(np.array_equal(Gamma, diagonal) and anticommutes
      and np.array_equal(Gamma, np.array(fixture["Gamma"], dtype=float)),
      "Gamma = g^(x0) g^(x1) ... g^(x7) = diag(-1 (8 times), +1 (8 times))",
      record=f"{ALGEBRA}, check Gamma_diag")
```

Two checks (Out [2]), reproducing the checks reality, Clifford_relation and Gamma_diag of `wolfram-algebra.json`.

In [3], Γ is the block map of T3.

```
KS_THEORY = "Revision/kohn_sham/ks-theory.json"
ks = json.loads(repository_file(KS_THEORY).read_text(encoding="utf-8"))
ENTRY = {"0": 0.0, "1": 1.0, "-1": -1.0, "I": 1j, "-I": -1j}  # the record's text
V = np.array([[ENTRY[x] for x in row]
              for row in ks["blockBasis"]["unnormalisedColumns2Sqrt2V"]])
V = V / (2.0 * math.sqrt(2.0))  # the record stores 2 sqrt(2) V
labels = [tuple(int(x) for x in label) for label in ks["blockBasis"]["labels"]]
sig1 = np.array([[0, 1], [1, 0]], dtype=complex)  # Pauli matrix sigma1
sig2 = np.array([[0, -1j], [1j, 0]])  # Pauli matrix sigma2
sig3 = np.array([[1, 0], [0, -1]], dtype=complex)  # Pauli matrix sigma3
unitary = float(np.max(np.abs(V.conj().T @ V - np.eye(16))))
worst_map, worst_form = 0.0, 0.0
for b, (j, s2, s3) in enumerate(labels):
    p = labels.index((-j, s2, s3))  # the partner block
    Vb, Vp = V[:, 2 * b:2 * b + 2], V[:, 2 * p:2 * p + 2]  # their two columns
    worst_map = max(worst_map, float(np.max(np.abs(Gamma @ Vb - Vp @ (s2 * sig2)))))
    g0 = Vb.conj().T @ gamma[0] @ Vb  # gamma^(x0) inside block b
    g04 = Vb.conj().T @ gamma[0] @ gamma[4] @ Vb  # gamma^(x0) gamma^(x4) there
    worst_form = max(worst_form, float(np.max(np.abs(g0 - sig3))),
                     float(np.max(np.abs(g04 - j * sig1))))
```

These lines are In [5] of Notebook 19b (the second half), with the Pauli matrices called `sig1`, `sig2`, `sig3`: the block basis V read from the record, its unitarity, and for every block the deviation of ΓV_b from $V_p\, s_2\sigma_2$ and of the block forms of $\gamma^{(x_0)}$ and $\gamma^{(x_0)}\gamma^{(x_4)}$ from σ_3 and $j\sigma_1$.

```
    if b < 4:
        say(f"block {b} {labels[b]} <-> block {p} {labels[p]}: matrix "
            f"{s2:+d} sigma2")
say(f"largest deviations: unitarity {unitary:.1e}, block map {worst_map:.1e}, "
    f"block forms {worst_form:.1e}")
check(unitary < 1e-12 and worst_map < 1e-12 and worst_form < 1e-12,
      "Gamma maps block (j, s2, s3) onto block (-j, s2, s3) as s2 sigma2",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.Gamma_is_the_block_map")
```

For the four blocks of type $j = +1$ (`b < 4`) the loop also prints which block is the partner and the matrix $s_2\sigma_2$ between them. Out [3] shows block 0 $(1,1,1)$ with block 4 $(-1,1,1)$ and block 1 $(1,1,-1)$ with block 5, both with $+\sigma_2$; block 2 $(1,-1,1)$ with block 6 and block 3 $(1,-1,-1)$ with block 7, both with $-\sigma_2$; then the deviations (1.1×10^{-16}, exactly 0, 1.1×10^{-16}) and the PASS line.

In [4], the solver is built.

```
SOLVER_MANIFEST = "Revision/kohn_sham/solver/Cargo.toml"  # the crate of the solver
program = rust_program(SOLVER_MANIFEST, "revision_ks_solver")  # build it, get path
RUN_FOLDER = REPO / "Revision/kohn_sham/solver/target/textbook_19a"  # git ignores it
RUN_FOLDER.mkdir(parents=True, exist_ok=True)
```

`rust_program` (In [1]) builds the solver, which takes a second when it is up to date, and returns the path of the program, `program`. `RUN_FOLDER` is a folder inside the solver's build folder `target`, which git ignores, so the raw outputs of the 62 runs never enter the record; `mkdir` creates it.

```
COLOUR = {"A": "#2a78d6", "B": "#eb6834", "C": "#1baf7a", "D": "#eda100"}
NAME = {"A": "A: $+m$, $+\\lambda$, $\\theta = 0$",
        "B": "B: $-m$, $+\\lambda$, $\\theta = \\pi$ (T3 partner)",
        "C": "C: $-m$, $+\\lambda$, $\\theta = 0$ (control)",
        "D": "D: $-m$, $-\\lambda$, $\\theta = \\pi$ (wrong partner)"}
say("Solver ready, colours defined.")
```

Every universe keeps one colour in all ten figures, blue A, orange B, aqua C, yellow D, and one legend text, `NAME`. Out [4] shows the two printed lines.

In [5], the helpers `solve` and `universe`.

```
TIP = {"A": 0.0, "B": math.pi, "C": 0.0, "D": math.pi}  # tip angle of each kind
MASS = {"A": 1.0, "B": -1.0, "C": -1.0, "D": -1.0}  # bare mass of each kind
SIGN = {"A": 1.0, "B": 1.0, "C": 1.0, "D": -1.0}  # D reverses the coupling


def read_profile(path):
    """A profile table as a dictionary: column name -> numpy array (151 values)."""
    data = np.genfromtxt(path, delimiter=",", names=True)
    return {name: data[name] for name in data.dtype.names}
```

The three dictionaries are the table of Section 19.17: the tip angle, the bare mass and the sign of the coupling of each universe. `read_profile` reads a profile table: `np.genfromtxt` reads a CSV file of numbers whose first line holds the column names (`names=True`), and the dictionary comprehension makes one numpy array of 151 values per column (`data.dtype.names` is the list of the column names).

```
def solve(label, m, lam, a4, N, theta, margin, T=None):
    """Run "revision_ks_solver single" for one state and return its results."""
    out = RUN_FOLDER / f"{label}.json"  # the solver's record of this state
    table = RUN_FOLDER / f"{label}.csv"  # its profiles
    command = [str(program), "single", "--root", str(REPO), "--m", repr(m),
               "--lambda", repr(lam), "--a4", repr(a4), "--N", repr(float(N)),
               "--tip-theta", repr(theta), "--margin", repr(margin),
               "--out", str(out), "--profiles", str(table)]
    if T is not None:
        command += ["--T", repr(T)]  # a thermal (Mermin) state
```

`solve` runs the solver once. The two output files are named after the `label`. `command` is the command line as a list: the program, its subcommand `single` (solve one state), and pairs of an option and its value: the options named root (where the repository is: the solver reads `ks-theory.json` and the gammas from there), m, lambda, a4 (the slice), N, tip-theta, margin, out (the JSON record of the state) and profiles (the CSV table), each written with two hyphens in front, as the code shows. `repr(x)` writes a number with all its digits, so the solver receives exactly the value of the notebook. For a thermal state the option T with the temperature is appended (`+=` appends a list).

```
    done = subprocess.run(command, capture_output=True, text=True,
                          encoding="utf-8", errors="replace")
    last = (done.stdout.strip().split("\n") or [""])[-1]  # SUCCESS or FAILURE
    if done.returncode != 0 or last != "SUCCESS":
        raise RuntimeError(f"the solver failed for {label}: {done.stderr[-400:]}")
    state = json.loads(out.read_text(encoding="utf-8"))
    state["levels"] = [tuple(level) for level in
                       state["levels_n2_j_parity_label_eps_deg_f"]]
    state["profile"] = read_profile(table)
    return state
```

`subprocess.run` runs the solver and keeps its printed text. The solver's last printed line is SUCCESS or FAILURE; `last` takes it (`.strip()` removes blank space at the ends, `.split("\n")` cuts the text into lines, `[-1]` takes the last one). If the return code is not 0 or the last line is not SUCCESS, the notebook stops with the end of the solver's error text. Otherwise the solver's JSON record is read into the dictionary `state`; its list of levels, each a list $(n_2, j, \text{parity}, \text{label}, \varepsilon, g, f)$, is stored again as tuples under the shorter key `levels`, and the profile table under `profile`.

```
def universe(kind, lam, a4, N, margin, T=None):
    """Solve universe A, B, C or D with the coupling lam (D uses -lam)."""
    label = f"{kind}_N{N}_lam{lam:+.4g}_a{a4:g}" + ("" if T is None else f"_T{T:g}")
    return solve(label, MASS[kind], SIGN[kind] * lam, a4, N, TIP[kind], margin, T)


say("solve and universe defined; outputs go to the folder "
    "Revision/kohn_sham/solver/target/textbook_19a")
```

`universe` solves one of the four universes: it makes a label such as `A_N136_lam+0.0009298_a1` (`:+.4g` writes the coupling with its sign and four significant digits, `:g` a short number) and calls `solve` with the bare mass, the signed coupling and the tip angle of that kind. Out [5] prints the line about the output folder.

In [6], what the record says.

```
T3_WOLFRAM = "Revision/pairing/kohn_sham/reports/wolfram-t3.json"
T3_SYMPY = "Revision/pairing/kohn_sham/reports/python-t3.json"
wolfram_t3 = json.loads(repository_file(T3_WOLFRAM).read_text(encoding="utf-8"))
sympy_t3 = json.loads(repository_file(T3_SYMPY).read_text(encoding="utf-8"))
passed_w = sum(c["verdict"] == "PASS" for c in wolfram_t3["checks"])
passed_s = sum(c["verdict"] == "PASS" for c in sympy_t3["checks"])
total_w, total_s = len(wolfram_t3["checks"]), len(sympy_t3["checks"])
say(f"proof of T3: Wolfram {passed_w} of {total_w} checks pass, "
    f"sympy {passed_s} of {total_s}")
check(passed_w == total_w == 10 and passed_s == total_s == 13,
      "the proof records of T3 pass completely (10 and 13 checks)",
      record=f"{T3_WOLFRAM} and {T3_SYMPY}")
```

The two proof reports of T3 are read; `sum(c["verdict"] == "PASS" for c in ...)` counts the checks with the verdict PASS (each `True` counts as 1). The check requires 10 of 10 and 13 of 13 (Out [6]); `a == b == c` in Python means $a = b$ and $b = c$.

```
SOLVER_REPORT = "Revision/kohn_sham/reports/ks-rust-solver.json"
PARAMETERS = "Revision/kohn_sham/results/parameters.json"
SELFTEST = "t3_block_map_solver_selftest"
solver_report = json.loads(repository_file(SOLVER_REPORT).read_text(encoding="utf-8"))
parameters = json.loads(repository_file(PARAMETERS).read_text(encoding="utf-8"))
LAMBDA = {int(entry["N"]): (entry["lambda1"], entry["lambda2"])
          for entry in parameters["couplingCalibration"]["values"]}
for N, (l1, l2) in LAMBDA.items():
    say(f"N = {N:3d}: lambda_1 = {l1}, lambda_2 = {l2}")
selftest = next(c for c in solver_report["checks"] if c["name"] == SELFTEST)
```

The solver's report and the parameter record are read. `LAMBDA` maps each particle number to its pair of couplings (λ_1, λ_2); Out [6] prints the table of Section 19.1 (`:3d` writes a whole number three places wide). `next(c for c in ... if ...)` takes the first check whose name is t3_block_map_solver_selftest.

```
PATTERN = re.compile(r"N = (\d+), lambda = (\S+), a4,0 = 1: E_KS (\S+) vs (\S+); "
                     r"(\d+) levels; .*?untransformed tip b\(-L\) = 0\): "
                     r"E_KS = (\S+) vs")
RECORD = {}  # N -> the recorded numbers of the self-test
for part in selftest["detail"].split(" | "):
    found = PATTERN.search(part)
    RECORD[int(found.group(1))] = {
        "lambda": float(found.group(2)), "E_A": float(found.group(3)),
        "E_B": float(found.group(4)), "levels": int(found.group(5)),
        "E_C": float(found.group(6))}
```

The self-test stores its numbers inside a sentence, one part per particle number, separated by a vertical bar. `PATTERN` is a regular expression that matches such a part: `\d+` matches one or more digits, `\S+` one or more characters that are not blank, `.*?` as few arbitrary characters as possible, `\(` a literal bracket; every pattern in round brackets is a **group** whose matched text `found.group(k)` can be read out. A string written `r"..."` keeps its backslashes. For each part the notebook reads the particle number, the coupling, the energies of A and B, the number of levels and the energy of the control C, and stores them in `RECORD`.

```
for N, entry in RECORD.items():
    E_A, E_B, E_C, count = (entry[key] for key in ("E_A", "E_B", "E_C", "levels"))
    say(f"record, N = {N}: E_KS(A) = {E_A:.12e}, E_KS(B) = {E_B:.12e}, "
        f"{count} levels, control E_KS(C) = {E_C:.10e}")
check(selftest["verdict"] == "PASS" and sorted(RECORD) == [8, 136]
      and all(RECORD[N]["lambda"] == LAMBDA[N][0] for N in RECORD),
      "the record holds the passing self-test for N = 8 and 136 at lambda_1",
      record=f"{SOLVER_REPORT}, check {SELFTEST}")
```

The loop prints the recorded numbers (Out [6]: the numbers of Section 19.10). The check requires the self-test to have passed, to hold exactly the particle numbers 8 and 136, and to have used the coupling λ_1 of each.

In [7], the self-test repeated: six runs.

```
SELF = {}  # (N, kind) -> state
for N in (8, 136):
    for kind in "ABC":
        SELF[(N, kind)] = universe(kind, LAMBDA[N][0], 1.0, N, 0.45)
```

Six runs of the solver: A, B and C (`for kind in "ABC"` takes the three letters one after the other) for $N = 8$ and 136, at the slice $a_{4,0} = 1$, with λ_1 and the margin 0.45. `SELF` stores the six states under the key (N, kind).

```
def sorted_levels(state):
    """The levels as a sorted list of (eps, g, f), as the self-test compares them."""
    return sorted((level[4], level[5], level[6]) for level in state["levels"])


def level_difference(first, second):
    """Largest difference of the sorted (eps, g, f) lists of two states."""
    pairs = zip(sorted_levels(first), sorted_levels(second))
    return max(max(abs(x - y) for x, y in zip(p, q)) for p, q in pairs)
```

`sorted_levels` keeps of each level the triple (ε, g, f) (positions 4, 5, 6 of the tuple) and sorts the triples. `level_difference` pairs the sorted lists of two states entry by entry and returns the largest difference of any ε, g or f.

```
def integral_difference(first, second):
    """Largest relative difference of the four integrated EMT components."""
    a, b = first["emtIntegrals_2Vol7_int_e6Hy"], second["emtIntegrals_2Vol7_int_e6Hy"]
    return max(abs(a[c] - b[c]) / max(abs(a[c]), 1.0) for c in ("rho", "p3", "p_t",
                                                                "p0"))


def scalar_sum(first, second):
    """max|S_A + S_B| / max|S_A| on the 151 profile points."""
    sa, sb = first["profile"]["S"], second["profile"]["S"]
    return float(np.max(np.abs(sa + sb)) / max(np.max(np.abs(sa)), 1e-300))
```

`integral_difference` compares the four integrals $2\,\mathrm{Vol}_7 \int e^{6Hy} X\, dy$ of $X = \rho, p_3, p_t, p_0$, each relative to $\max(|a|, 1)$. `scalar_sum` measures how well $S_B = -S_A$ holds: the largest $|S_A + S_B|$ over the 151 points, divided by the largest $|S_A|$ (the `1e-300` avoids a division by zero).

```
for N in (8, 136):
    A, B, C = SELF[(N, "A")], SELF[(N, "B")], SELF[(N, "C")]
    E_A, E_B, E_C = A["E_KS"], B["E_KS"], C["E_KS"]
    count_A, count_B = len(A["levels"]), len(B["levels"])
    say(f"N = {N}: E_KS(A) = {E_A:.12e}, E_KS(B) = {E_B:.12e}, "
        f"{count_A} and {count_B} levels")
    say(f"    levels (eps, g, f): {level_difference(A, B):.2e}; EMT integrals: "
        f"{integral_difference(A, B):.2e}; |S_A + S_B|/max|S|: "
        f"{scalar_sum(A, B):.2e}; control E_KS(C) = {E_C:.10e}")
```

For each N the energies, the numbers of levels and the three differences are printed (Out [7]). For $N = 8$: $E_{KS}(A) = -9.868426190876 \times 10^{-4}$ and $E_{KS}(B) = -9.868426190868 \times 10^{-4}$, the record's numbers digit for digit; 80 levels each; level difference 6.88×10^{-14}, integrals 8.96×10^{-16}, $|S_A + S_B| / \max|S|$ 1.84×10^{-15}; the control -2.0145243719. For $N = 136$: 32.39294915318 for both, 160 levels, differences 9.86×10^{-14}, 2.19×10^{-16}, 8.38×10^{-16}; the control 29.283751318. The record's self-test reports 6.91×10^{-14} and 9.95×10^{-14} for the level differences of its own runs; differences of this size are rounding, and all of them are ten thousand times below the tolerance.

In [8], the self-test as checks.

```
TOL = 1e-9  # the tolerance of the self-test, fixed in the record
REC = f"{SOLVER_REPORT}, check {SELFTEST}"


def close(a, b, tol=TOL):
    """True when a and b agree within tol relative to max(|a|, 1)."""
    return abs(a - b) <= tol * max(abs(a), 1.0)
```

`TOL` is the record's tolerance 10^{-9}, and `close` compares two numbers relative to $\max(|a|, 1)$: relative for large numbers, absolute for numbers below 1.

```
for N in (8, 136):
    A, B, C = SELF[(N, "A")], SELF[(N, "B")], SELF[(N, "C")]
    rec = RECORD[N]
    check(len(A["levels"]) == len(B["levels"]) == rec["levels"]
          and level_difference(A, B) < TOL and integral_difference(A, B) < TOL
          and scalar_sum(A, B) < TOL and close(A["E_KS"], B["E_KS"]),
          f"N = {N}: A and B have equal levels, energies, EMT integrals, S_B = -S_A",
          record=REC)
    check(close(A["E_KS"], rec["E_A"]) and close(B["E_KS"], rec["E_B"])
          and close(C["E_KS"], rec["E_C"]) and abs(C["E_KS"] - A["E_KS"]) > 1.0,
          f"N = {N}: the energies of A, B and the control C are those of the record",
          record=REC)
```

Two checks per particle number (Out [8]): A and B have the recorded number of levels and agree within the tolerance in levels, integrals, scalar density and energy; and the three energies are the recorded ones, with the control more than 1 away from A.

In [9], A is the canonical state of the record.

```
GROUND = "Revision/kohn_sham/results/ground"


def read_rows(relative):
    """A CSV file of the record as {id: row}."""
    with repository_file(relative).open(encoding="utf-8", newline="") as handle:
        return {row["id"]: row for row in csv.DictReader(handle)}


def homo_lumo(state):
    """The highest occupied (f = 1) and the lowest empty (f = 0) level."""
```

```
    occupied = [lv[4] for lv in state["levels"] if lv[6] > 0.5]
    empty = [lv[4] for lv in state["levels"] if lv[6] < 0.5]
    return max(occupied), min(empty)
```

`read_rows` reads a CSV table of the record into a dictionary from the state name in the column `id` to its row. `homo_lumo` returns the highest occupied level (**HOMO**: occupation above one half) and the lowest empty one (**LUMO**).

```
summary = read_rows(f"{GROUND}/summary.csv")
integrals = read_rows(f"{GROUND}/emt-integrals.csv")
A = SELF[(136, "A")]
homo, lumo = homo_lumo(A)
new = {"E_KS": A["E_KS"], "HOMO": homo, "LUMO": lumo, "KS_gap": lumo - homo}
row = summary["N136_lamp1_a10"]
worst_scalar = max(abs(new[key] - float(row[key])) for key in new)
emt_row = integrals["N136_lamp1_a10"]
new_int = A["emtIntegrals_2Vol7_int_e6Hy"]  # the new integrals
worst_integral = max(abs(new_int[c] - float(emt_row[f"int_{c}"]))
                     / max(abs(float(emt_row[f"int_{c}"])), 1.0)
                     for c in ("rho", "p3", "p_t", "p0", "n"))
```

The record's summary table and integral table are read. For the run A at $N = 136$, λ_1, $a_{4,0} = 1$, the four numbers energy, HOMO, LUMO and the Kohn-Sham gap (LUMO minus HOMO) are compared with the row N136_lamp1_a10 of the summary, and the five integrals (ρ, p_3, p_t, p_0 and n) with the same row of the integral table (columns `int_rho` and so on).

```
committed = read_profile(repository_file(f"{GROUND}/profiles/N136_lamp1_a10.csv"))
worst_profile = max(float(np.max(np.abs(A["profile"][c] - committed[c]))
                          / max(np.max(np.abs(committed[c])), 1e-300))
                    for c in committed)
for key in new:
    say(f"{key:7}: new {new[key]: .15e}   record {float(row[key]): .15e}")
say(f"largest differences: scalars {worst_scalar:.1e}, integrals "
    f"{worst_integral:.1e}, profile columns {worst_profile:.1e}")
```

The committed profile file of the state is read, and every column of the new profile is compared with it at all 151 points, relative to the column's largest value. Out [9] prints the four numbers side by side (`{key:7}` pads the name to seven places, `:  .15e` writes sixteen digits with a blank for the sign): $E_{KS} = 32.39294915318298$, HOMO 0.3258848143646794, LUMO 0.3608587569459729, gap 0.03497394258129349, identical to the record; all three largest differences are exactly 0.

```
same_bytes = ((RUN_FOLDER / "A_N136_lam+0.0009298_a1.csv").read_bytes()
              == repository_file(f"{GROUND}/profiles/N136_lamp1_a10.csv").read_bytes())
report("new profile file byte-identical to the record", same_bytes)
check(worst_scalar < TOL and worst_integral < TOL and worst_profile < TOL,
      "A reproduces the canonical state N136_lamp1_a10 of the record",
      record=f"{GROUND}/summary.csv, emt-integrals.csv and profiles, N136_lamp1_a10")
```

`read_bytes` reads a file as raw bytes; `same_bytes` is true when the new profile file is identical byte for byte to the committed one. On the computer that built this book it is (`True` in Out [9]); on

another computer the last digit of a few numbers may differ, which is why the check itself uses the tolerance.

In [10], the pairing level by level (statement S1).

```
OTHER = {"even": "odd", "odd": "even"}  # the brane parity is exchanged


def level_map(first, second, tol=TOL):
    """Pair every level (n2, j, p, l, eps, g, f) of first with a level of second in
    (n2, -j, other p) at the same eps; return the pairs (None if one is missing)."""
    unused = list(second["levels"])
    pairs = []
    for lv in sorted(first["levels"]):
        match = [w for w in unused if w[0] == lv[0] and w[1] == -lv[1]
                 and w[2] == OTHER[lv[2]] and abs(w[4] - lv[4]) <= tol]
        if not match:
            return None
        best = min(match, key=lambda w: abs(w[4] - lv[4]))
        unused.remove(best)
        pairs.append((lv, best))
    return pairs
```

`level_map` tests statement S1 of T3: for every level of the first state it searches, among the levels of the second state not yet used, one in the same momentum shell (`w[0] == lv[0]`), with the opposite block type, the other brane parity and the same level within the tolerance. If there is none, the pairing fails and the function returns `None`; otherwise it takes the closest one (`min` with `key=` compares by the distance; `lambda w: ...` is a short unnamed function), removes it from the unused list, so that each level of the second state is used once, and records the pair.

```
pairs136 = level_map(SELF[(136, "A")], SELF[(136, "B")])
shifts = {}
for lv, w in pairs136:
    key = f"j = {lv[1]:+d}, {lv[2]:4}"
    shifts.setdefault(key, set()).add(w[3] - lv[3])  # label of B minus label of A
for key in sorted(shifts):
    say(f"A levels with {key}: partner in j = opposite, other parity; label shift "
        f"{sorted(shifts[key])}")
worst = max(abs(lv[4] - w[4]) for lv, w in pairs136)
report("largest |eps_A - eps_B| of the paired levels, N = 136", f"{worst:.2e}")
```

For $N = 136$ every level of A is paired with its level of B. For each sector of A (type and parity) the set of label differences, label of B minus label of A, is collected (`setdefault` starts an empty set for a new key). The **label** is the solver's count of a level inside its sector, counted from the boundary conditions; A and B have different boundary conditions, so the count of a partner can be shifted by one. Out [10] shows: the type $+1$ even levels of A are partnered by type -1 odd levels of B with labels one lower, the type -1 odd levels by labels one higher, the other two sectors with the same labels. The map of T3 is fixed by the sector and the energy, not by the label. The largest level difference of the pairs is 9.86×10^{-14}.

```
check(pairs136 is not None and len(pairs136) == len(SELF[(136, "B")]["levels"])
      and all(lv[5] == w[5] and lv[6] == w[6] for lv, w in pairs136),
      "every level of A has its partner in B with -j and the other brane parity",
      record="Revision/pairing/kohn_sham/t3-theory.json, statement S1")
check(level_map(SELF[(136, "A")], SELF[(136, "C")]) is None,
      "the control C has no such partner levels (the pairing fails)")
```

The first check requires the pairing to exist, to use every level of B, and to pair equal degeneracies and occupations; the second requires the same pairing to FAIL for the control.

In [11], the level ladders (Figure 19a.1).

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
for x, kind in enumerate("ABC"):
    state = SELF[(136, kind)]
    for lv in state["levels"]:
        if lv[4] > 1.3:
            continue  # only the lower part of the spectrum
        full = lv[6] > 0.5  # occupied?
        ax.plot([x - 0.32, x + 0.32], [lv[4], lv[4]],
                color=COLOUR[kind] if full else "0.7", lw=2.0 if full else 1.0)
    ax.plot([x - 0.42, x + 0.42], [state["mu_or_fermi_level"]] * 2, color="0.2",
            ls="--", lw=1.0)
```

At the horizontal positions 0, 1, 2 the levels of A, B and C below 1.3 are drawn as short horizontal lines (`continue` skips a level above 1.3), in the universe's colour and thicker when occupied, light grey and thinner when empty (`a if condition else b` chooses between two values). A dashed dark line marks the Fermi level that the solver reports (`[value] * 2` is a list holding the value twice, the heights of the two ends of the line).

```
ax.set_xticks([0, 1, 2])
ax.set_xticklabels(["A: $+m$, $\\theta = 0$", "B: $-m$, $\\theta = \\pi$",
                    "C: $-m$, $\\theta = 0$ (control)"])
ax.set_xlim(-0.6, 2.6)
ax.set_ylabel("Kohn-Sham level $\\varepsilon$ (units of $|m|$)")
ax.set_title("$N = 136$, $\\lambda = \\lambda_1$, slice $a_{4,0} = 1$")
ax.grid(axis="x", visible=False)
save_figure(fig, "level_ladder",
            "Level ladders of the three universes A (bare mass $+m$, tip angle 0), B "
            "(bare mass $-m$, the same coupling, tip angle $\\pi$: the T3 partner) "
            "and C (bare mass $-m$, tip angle 0: the control) for $N = 136$ "
            "particles, $\\lambda = \\lambda_1$, slice $a_{4,0} = 1$, all solved "
            "independently by the Rust solver. Each short line is one Kohn-Sham "
            "level (vertical axis, units of $|m|$), coloured when occupied and grey "
            "when empty; the dashed line is the Fermi level. A and B are the same "
            "ladder; C, with the untransformed tip, is a different one.")
```

Labels, limits and title; `save_figure` saves Figure 19a.1. What the student should see: two identical ladders for A and B, occupied up to the dashed Fermi level, which the solver reports at the highest occupied level 0.3259 (the HOMO of In [9]), with the first empty level at 0.3609; a different ladder for the control C, with its levels at other heights.

In [12], the level-by-level differences (Figure 19a.2).

```
FLOOR = 1e-17  # where exact zeros are drawn on the logarithmic axis
eps = {kind: np.array([e for e, g, f in sorted_levels(SELF[(136, kind)])])
       for kind in "ABC"}
index = np.arange(1, len(eps["A"]) + 1)
fig, ax = plt.subplots()
ax.semilogy(index, np.maximum(np.abs(eps["A"] - eps["B"]), FLOOR), "o",
            color=COLOUR["B"], ms=4, label="A against B (T3 partner)")
ax.semilogy(index, np.maximum(np.abs(eps["A"] - eps["C"]), FLOOR), "^",
            color=COLOUR["C"], ms=4, label="A against C (control)")
ax.axhline(TOL, color="0.3", ls="--", lw=1.0, label="tolerance $10^{-9}$")
```

`eps` holds the sorted levels of A, B and C (the tuple `(e, g, f)` is unpacked and only `e` kept); `index` numbers them $1, 2, \ldots, 160$. `semilogy` draws with a **logarithmic** vertical axis (equal steps for each factor of ten). A logarithm of zero does not exist, so `np.maximum(..., FLOOR)` raises every exact zero to 10^{-17}. A dashed horizontal line marks the tolerance.

```
ax.set_xlabel("level number $i$ (levels sorted by energy)")
ax.set_ylabel("$|\\varepsilon_{A,i} - \\varepsilon_{X,i}|$ (units of $|m|$)")
ax.set_title("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$: 160 levels")
ax.legend(fontsize=8, loc="center right")
save_figure(fig, "level_differences",
            "Level-by-level differences between universe A and its T3 partner B "
            "(orange circles) and between A and the control C (aqua triangles), "
            "for $N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$; horizontal axis "
            "the number of the level in the sorted list, vertical axis the absolute "
            "difference in units of $|m|$ (logarithmic; exact zeros drawn at "
            "$10^{-17}$). The partner agrees to $10^{-13}$ or better, the rounding "
            "of the computer, far below the tolerance $10^{-9}$ (dashed); the "
            "control differs by $10^{-4}$ to $1$.")
```

Axis labels, title and legend; `save_figure` saves Figure 19a.2.

```
max_partner = float(np.max(np.abs(eps["A"] - eps["B"])))
gaps_control = np.abs(eps["A"] - eps["C"])
report("largest level difference A - B", f"{max_partner:.2e}")
report("smallest and largest level difference A - C",
       f"{gaps_control.min():.2e} and {gaps_control.max():.3g}")
check(max_partner < 1e-12 and 1e-4 < gaps_control.min() and gaps_control.max() < 1,
      "levels: A - B below 1e-12, A - C between 1e-4 and 1 (the caption's ranges)")
```

The numbers behind the picture (Out [12]): A and B differ by at most 9.86×10^{-14}; A and C by between 1.88×10^{-4} and 0.456. The check guards the ranges that the caption states. What the student should see: a band of orange circles between the floor 10^{-17} and about 10^{-13}, far below the dashed tolerance line, and the aqua triangles eight to sixteen powers of ten higher.

In [13], the densities point by point (statements S3 and S4).

```
EVEN = ("n", "v_v", "e_int", "rho", "p3", "p_t", "p0")  # unchanged by the map
ODD = ("S", "Q", "M_eff")  # change sign under the map
```

```
def profile_mismatch(first, second):
    """Largest relative mismatch of the T3 rule (even columns equal, odd opposite)."""
    worst = 0.0
    for column in EVEN + ODD:
        sign = -1.0 if column in ODD else 1.0
        a, b = first["profile"][column], second["profile"][column]
        scale = max(float(np.max(np.abs(a))), 1e-300)
        worst = max(worst, float(np.max(np.abs(b - sign * a))) / scale)
    return worst
```

`EVEN` and `ODD` list the profile columns that T3 keeps and those it reverses (Section 19.9). `profile_mismatch` measures, for every column, the largest deviation of the second state's column from the first state's column times the expected sign, relative to the column's largest value, and returns the worst one.

```
for N in (8, 136):
    mismatch = profile_mismatch(SELF[(N, "A")], SELF[(N, "B")])
    control = profile_mismatch(SELF[(N, "A")], SELF[(N, "C")])
    say(f"N = {N}: T3 rule for B: {mismatch:.2e}; the same rule for C: {control:.3g}")
    check(mismatch < TOL and control > 0.1,
          f"N = {N}: n, v_v, e_int, rho, p3, p_t, p0 equal; S, Q, M_eff opposite",
          record="Revision/pairing/kohn_sham/t3-theory.json, statements S3 and S4")
```

Out [13]: the partner B obeys the rule to 2.78×10^{-15} ($N = 8$) and 5.80×10^{-15} ($N = 136$); the control C violates it by factors of 298 and 4650 (its profiles are many times larger than those of A in some columns). Both checks pass.

In [14], the densities (Figure 19a.3).

```
y = SELF[(136, "A")]["profile"]["y"]  # the 151 points from the tip to the brane
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
STYLE = {"A": dict(lw=2.6), "B": dict(lw=1.6, ls="--"), "C": dict(lw=1.4, ls=":")}
for kind in "ABC":
    prof = SELF[(136, kind)]["profile"]
    left.semilogy(y, prof["n"], color=COLOUR[kind], label=NAME[kind], **STYLE[kind])
    if kind != "C":
        right.plot(y, prof["S"], color=COLOUR[kind], label=NAME[kind],
                   **STYLE[kind])
```

`y` holds the 151 points. `STYLE` gives each universe a line width and style: A thick and solid, B thinner and dashed (so that A shows around it), C dotted; `dict(lw=2.6)` makes a dictionary, and `**STYLE[kind]` passes its entries as named arguments to `plot`. The left axes show the particle density $n(y)$ of the three universes on a logarithmic axis, the right axes the scalar density $S(y)$ of A and B only.

```
left.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
left.set_ylabel("proper particle density $n(y)$")
left.legend(fontsize=7, loc="upper right")
```

```
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("proper scalar density $S(y)$")
right.axhline(0.0, color="0.3", lw=0.8)
right.legend(fontsize=7, loc="lower right")
fig.suptitle("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
save_figure(fig, "density_profiles",
            "Left: the proper particle density $n(y)$ of the universes A (blue), B "
            "(orange, dashed) and C (aqua, dotted) for $N = 136$, "
            "$\\lambda = \\lambda_1$, $a_{4,0} = 1$, against the hidden coordinate $y$ "
            "from the tip $y = -3$ to the brane $y = 0$ (logarithmic vertical axis, "
            "units $|m|^7$). Right: the proper scalar density $S(y)$ of A and B. "
            "The density of B is that of A, and its scalar density is the mirror "
            "image $S_B = -S_A$, as T3 states. The control C has its own densities; "
            "its scalar density, more than ten times larger, is not drawn.")
```

Labels, a zero line on the right, legends and a common title (`suptitle`); `save_figure` saves Figure 19a.3. What the student should see: on the left the density of A falls from about 300 at the tip to about 0.009 at the brane (the record's profile file of N136_lamp1_a10: 298.7 and 0.008849), large at the tip because a proper density is a number per unit proper volume, and the proper volume factor e^{6Hy} is tiny there; the dashed orange line of B lies exactly on the blue line of A; the dotted line of C is different. On the right S_A is negative between the ends and zero at both ends (the tip condition $b(-L) = 0$ and either brane condition make $2ab$ vanish there), and S_B is its mirror image, positive.

```
size = {kind: float(np.max(np.abs(SELF[(136, kind)]["profile"]["S"])))
        for kind in "AC"}
factor = size["C"] / size["A"]  # how much larger the control's scalar density is
report("largest |S| of C divided by largest |S| of A", f"{factor:.1f}")
check(factor > 10.0,
      "the scalar density of the control is more than ten times that of A")
```

The caption says the control's scalar density is more than ten times larger; the cell measures the factor (24.1, Out [14]) and checks it.

In [15], the mass and the potential (Figure 19a.4).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
for kind in "ABC":
    prof = SELF[(136, kind)]["profile"]
    left.plot(y, prof["M_eff"], color=COLOUR[kind], label=NAME[kind], **STYLE[kind])
    right.plot(y, prof["v_v"], color=COLOUR[kind], label=NAME[kind], **STYLE[kind])
left.set_xlabel("hidden coordinate $y$")
left.set_ylabel("effective mass $M_{eff}(y)$ (units of $|m|$)")
left.legend(fontsize=7, loc="center right")
right.set_xlabel("hidden coordinate $y$")
right.set_ylabel("potential $v_v(y)$ (units of $|m|$)")
fig.suptitle("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
```

The self-consistent effective mass $M_{\text{eff}}(y)$ (left) and potential $v_v(y)$ (right) of the three universes, in the same styles.

```
save_figure(fig, "mass_and_potential",
            "Left: the self-consistent effective mass $M_{eff}(y)$ of A (blue), B "
            "(orange, dashed) and C (aqua, dotted) for $N = 136$, "
            "$\\lambda = \\lambda_1$, $a_{4,0} = 1$ (units of $|m|$). Right: the "
            "self-consistent potential $v_v(y)$ (units of $|m|$). B has exactly the "
            "opposite mass of A at every point and the same potential: this is why "
            "the bare mass must change sign while the coupling keeps its sign. The "
            "control C has the mass $-m$ but its own, different mean field.")
```

`save_figure` saves Figure 19a.4. What the student should see: on the left the mass of A near $+1$, with a dip to about 0.97 near $y = -2.4$ where S is most negative ($1 + \frac{15}{16}\lambda_1 S$ with $S \approx -37$ in the record's profile), and the mass of B its exact negative near -1; the control near -1 with its own shape. On the right the potentials of A and B on top of each other, small and negative (about -0.017 at the tip), and the control's different curve.

In [16], the energy-momentum profiles (Figure 19a.5).

```
weight = np.exp(6.0 * y)  # the proper-volume factor e^{6 H y}, H = 1
fig, axes = plt.subplots(2, 2, figsize=(9.0, 6.0), sharex=True)
for ax, column, title in zip(axes.flat, ("rho", "p3", "p_t", "p0"),
                             ("energy density $\\rho$", "pressure $p_3$ (3-space)",
                              "pressure $p_t$ (extra times)",
                              "pressure $p_0$ (hidden direction)")):
    for kind in "ABC":
        prof = SELF[(136, kind)]["profile"]
        ax.plot(y, weight * prof[column], color=COLOUR[kind], label=NAME[kind],
                **STYLE[kind])
    ax.set_title(title, fontsize=9)
    ax.set_ylabel("$e^{6Hy}$ times the component")
for ax in axes[1]:
    ax.set_xlabel("hidden coordinate $y$")
axes[0, 0].legend(fontsize=7, loc="upper left")
fig.tight_layout()
```

`weight` is the volume factor e^{6Hy} at the 151 points; multiplying a proper density by it gives a density per unit of y, so the area under each curve is proportional to the integral over the hidden direction. A grid of 2×2 axes with a common horizontal axis (`sharex=True`) shows ρ, p_3, p_t, p_0 times the weight, one per axes, for the three universes; `zip` runs through the axes, the column names and the titles together. Only the lower row gets the horizontal label (`axes[1]` is the second row), and only the first axes the legend.

```
save_figure(fig, "emt_profiles",
            "The energy-momentum profiles of A (blue), B (orange, dashed) and C "
            "(aqua, dotted) for $N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$: "
            "energy density $\\rho$, pressures $p_3$ (3-space), $p_t$ (the "
            "deflating extra times) and $p_0$ (hidden direction), each times the "
            "volume factor $e^{6Hy}$, against the hidden coordinate $y$ (units "
            "$|m|^8$ with $H = 1$). The four profiles of the partner B coincide "
            "with those of A at every point (T3, statement S4); the control C "
            "differs.")
```

`save_figure` saves Figure 19a.5. What the student should see: in all four panels the dashed orange curve of B exactly on the blue curve of A, and the dotted aqua curve of the control elsewhere; $p_t = e_{\text{int}}$ is much smaller than the other three, because the good sector has no kinetic pressure along the extra times.

```
for N in (8, 136):
    B = SELF[(N, "B")]
    integrated, pointwise = (B["yConservationIntegratedRel"],
                             B["yConservationPointwiseRel"])
    say(f"N = {N}, B: y-conservation, integrated {integrated:.1e}, pointwise "
        f"{pointwise:.1e} (relative)")
    check(integrated < 1e-8 and pointwise < 1e-6,
          f"N = {N}: the partner B obeys p0' + 6H p0 = 3H (p3 + p_t)",
          record=f"{SOLVER_REPORT}, checks emt_y_conservation_integrated, pointwise")
```

The solver measures for every state how well the conservation law $p_0' + 6Hp_0 = 3H(p_3 + p_t)$ of Section 19.4 holds, after integration over y and point by point. For the partner B the residuals are 3.9×10^{-13} and 2.2×10^{-9} ($N = 8$) and 1.7×10^{-12} and 8.4×10^{-11} ($N = 136$) (Out [16]), below the tolerances 10^{-8} and 10^{-6} used here; B is a genuine self-consistent state of its own problem.

In [17], where the zero modes live: three free runs.

```
FREE8 = {kind: universe(kind, 0.0, 1.0, 8, 0.25) for kind in "ABC"}  # lambda = 0
L_TIP, M_BARE = 3.0, 1.0  # the tip distance L and the bare mass |m| of the record


def brane_over_tip(state):
    """Coordinate density e^{6Hy} n(y) at the brane divided by its value at the tip."""
    n = state["profile"]["n"]
    return float(n[-1] * weight[-1] / (n[0] * weight[0]))
```

Three runs without interaction for $N = 8$ at $a_{4,0} = 1$ (margin 0.25): the eight quanta fill the eight zero modes. `brane_over_tip` divides the coordinate density $e^{6Hy}n$ at the last point (the brane, index -1) by its value at the first point (the tip, index 0).

```
expected = {"A": math.exp(2 * M_BARE * L_TIP), "B": math.exp(2 * M_BARE * L_TIP),
            "C": math.exp(-2 * M_BARE * L_TIP)}  # e^{+6}, e^{+6}, e^{-6}
ratio_error = 0.0
for kind in "ABC":
    ratio = brane_over_tip(FREE8[kind])
    ratio_error = max(ratio_error, abs(ratio / expected[kind] - 1.0))
    say(f"{kind}: brane / tip = {ratio:.6e}, closed form {expected[kind]:.6e}")
check(ratio_error < 1e-8,
      "free N = 8: the zero modes of A and B live at the brane, those of C at the tip")
```

The closed forms of Section 19.12: the ratio is $e^{2mL} = e^6$ for A and B and e^{-6} for C. Out [17] prints 403.4288 for A and B and 0.002478752 for C, equal to the closed forms; the check requires a relative agreement better than 10^{-8}.

In [18], the control's level in the gap, by hand.

```
def bound_state_q(m, L):
    """The root q in (0, m) of tanh(q L) - q / m, by bisection."""
    low, high = 0.5 * m, m  # the function is positive at m/2 and negative at m
    for _ in range(200):
        middle = 0.5 * (low + high)
        if math.tanh(middle * L) - middle / m > 0.0:
            low = middle  # the root lies above the middle
        else:
            high = middle  # the root lies below the middle
    return 0.5 * (low + high)
```

`bound_state_q` finds the root of $\tanh(qL) = q/m$ of Section 19.12 by bisection, starting from the interval $(m/2, m)$, where the function changes sign (at $m/2$: $\tanh(1.5) - 0.5 = 0.405 > 0$; at m: $\tanh(3) - 1 < 0$). Each of the 200 steps keeps the half that contains the root.

```
q_root = bound_state_q(M_BARE, L_TIP)
eps_b = M_BARE / math.cosh(q_root * L_TIP)  # the level inside the gap
report("q of the control's sub-gap level", f"{q_root:.12f}")
report("sub-gap level eps_b = m / cosh(q L)", f"{eps_b:.12f}", "|m|")
gap = {}
for kind in "ABC":
    homo, lumo = homo_lumo(FREE8[kind])
    gap[kind] = lumo - homo
    say(f"{kind}: HOMO {homo:.3e}, LUMO {lumo:.12f}, Kohn-Sham gap {gap[kind]:.12f}")
```

The root $q = 0.994901528453$ and the level $\varepsilon_b = m/\cosh(qL) = 0.100851121375$ (Out [18]). Then the Kohn-Sham gaps of the three free runs: in all three the HOMO is the zero modes at 0. For A and B the lowest empty level is 0.170349251323, a level of the brane band at the smallest nonzero momentum shell; for the control it is 0.100851121375: its level inside the gap, lower than its brane band.

```
recorded = float(summary["N8_lam0_a10"]["KS_gap"])
same_level = abs(math.sqrt(M_BARE ** 2 - q_root ** 2) - eps_b) < 1e-12  # two forms
check(abs(gap["C"] - eps_b) < 1e-9 and same_level,
      "the free control has the Kohn-Sham gap m/cosh(qL) of the sub-gap level")
check(close(gap["A"], recorded) and close(gap["B"], gap["A"]),
      "the free A and B have the gap of the committed state N8_lam0_a10",
      record=f"{GROUND}/summary.csv, N8_lam0_a10, column KS_gap")
```

The first check: the control's gap equals ε_b, a number derived by hand, to 10^{-9} (in fact to twelve digits), and the two forms $\sqrt{m^2 - q^2}$ and $m/\cosh(qL)$ agree. The second: A has the gap of the committed state N8_lam0_a10, and B the gap of A.

In [19], the zero modes (Figure 19a.6).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8), sharey=True)
for kind in "ABC":
    left.semilogy(y, weight * FREE8[kind]["profile"]["n"], color=COLOUR[kind],
                  label=NAME[kind], **STYLE[kind])
    right.semilogy(y, weight * SELF[(8, kind)]["profile"]["n"], color=COLOUR[kind],
```

```
                    label=NAME[kind], **STYLE[kind])
top = {kind: float(weight[-1] * FREE8[kind]["profile"]["n"][-1]) for kind in "AC"}
left.semilogy(y, top["A"] * np.exp(2 * M_BARE * y), "k:", lw=0.8)  # e^{2 m y}
left.semilogy(y, top["C"] * np.exp(-2 * M_BARE * y), "k:", lw=0.8)  # e^{-2 m y}
```

Two axes with a common vertical scale (`sharey=True`): on the left the coordinate densities $e^{6Hy}n$ of the free runs, on the right those of the interacting runs of In [7]. `top` holds the brane values of the free A and C, and the two thin black dotted lines are the closed forms e^{2my} and e^{-2my} scaled to these values.

```
left.set_title("$N = 8$, $\\lambda = 0$, $a_{4,0} = 1$", fontsize=10)
right.set_title("$N = 8$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
left.set_ylabel("coordinate density $e^{6Hy}\\,n(y)$")
for ax in (left, right):
    ax.set_xlabel("hidden coordinate $y$ (tip $-3$, brane $0$)")
right.legend(fontsize=7, loc="lower right")
fig.tight_layout()
save_figure(fig, "zero_modes_n8",
            "The coordinate particle density $e^{6Hy}n(y)$ (particles per unit of "
            "$y$; logarithmic vertical axis) of the $N = 8$ universes A (blue), B "
            "(orange, dashed) and C (aqua, dotted) at $a_{4,0} = 1$, against the "
            "hidden coordinate $y$ from the tip $-3$ to the brane $0$. Left: without "
            "interaction; the thin black dotted lines are the closed forms "
            "$e^{2my}$ and $e^{-2my}$. A and B hold their particles in zero modes "
            "at the brane; with the untransformed tip (C) the zero modes sit at the "
            "tip. Right: with the coupling $\\lambda_1$; A and B are unchanged in "
            "shape, the control is reshaped by its strong mean field near the tip.")
```

Titles, labels, legend; `save_figure` saves Figure 19a.6. What the student should see: on the left two straight lines on the logarithmic scale, the rising one of A and B (B dashed on top of A) along e^{2my}, and the falling one of C along e^{-2my}, each covering a factor $e^6 \approx 403$ between tip and brane; on the right A and B unchanged in shape, and the control's line bent near the tip, where its quanta, at a huge proper density, feel a strong mean field.

In [20], T3 along the deflating history: 27 runs.

```
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]
HIST = {}  # (N, kind, a4) -> state
for N in (136, 688):
    for a4 in SLICES:
        for kind in "ABC":
            if N == 136 and a4 == 1.0:
                HIST[(N, kind, a4)] = SELF[(N, kind)]  # already solved
            else:
                HIST[(N, kind, a4)] = universe(kind, LAMBDA[N][0], a4, N, 0.45)
```

A, B and C at the five slices of the history for $N = 136$ and 688, each with its coupling λ_1: 30 states, of which the three of $N = 136$ at $a_{4,0} = 1$ were solved in In [7] and are reused, so 27 new runs.

```
ok_record, ok_partner, ok_control = True, True, True
partner_rel, control_pct = [], []  # relative differences A - B, percents A - C
```

```
for N in (136, 688):
    for a4 in SLICES:
        A, B, C = (HIST[(N, kind, a4)] for kind in "ABC")
        record_energy = float(summary[f"N{N}_lamp1_a{round(10 * a4):02d}"]["E_KS"])
        ok_record &= close(A["E_KS"], record_energy)
        partner_rel.append(abs(A["E_KS"] - B["E_KS"]) / abs(A["E_KS"]))
        control_pct.append(100.0 * abs(A["E_KS"] - C["E_KS"]) / abs(A["E_KS"]))
        ok_partner &= level_difference(A, B) < TOL and profile_mismatch(A, B) < TOL
        E_A, E_B, E_C = A["E_KS"], B["E_KS"], C["E_KS"]
        say(f"N = {N:3d}, a4,0 = {a4:3.1f}: E_KS A {E_A:.10f}, "
            f"B {E_B:.10f}, C {E_C:.6f}")
```

For each of the ten pairs (particle number, slice): the state name of the record is made from the particle number and ten times the slice written with two digits (`:02d`, so 0.5 gives `a05`), and A must reproduce the recorded energy (`&=` keeps a truth value true only while every test is true); the relative energy difference A minus B and the percentage A minus C are collected; and B must agree with A in levels and in every profile column. The loop prints the three energies (Out [20]):

N	$a_{4,0}$	E_{KS} **of A and B**	E_{KS} **of C**
136	0.0	80.2837009244	73.179866
136	0.5	51.2491613321	46.475561
136	1.0	32.3929491532	29.283751
136	1.5	20.2130306754	18.533932
136	2.0	12.4470595905	12.175574
688	0.0	680.4447575708	670.715818
688	0.5	437.5215029073	431.359438
688	1.0	279.4468465636	275.745397
688	1.5	176.7808903556	175.297541
688	2.0	110.4456550192	112.097612

```
report("largest relative E_KS difference A - B", f"{max(partner_rel):.1e}")
report("relative E_KS difference A - C", f"{min(control_pct):.1f} to "
       f"{max(control_pct):.1f}", "percent")
check(ok_record, "A reproduces the recorded E_KS at all 10 states of the history",
      record=f"{GROUND}/summary.csv, N136_lamp1_a00 to a20, N688_lamp1_a00 to a20")
check(ok_partner and max(partner_rel) < 1e-13,
      "B has the levels, energy and profiles of A at every slice")
check(0.5 < min(control_pct) and max(control_pct) < 10.0,
      "the control C differs from A by 0.5 to 10 percent at every slice")
```

The largest relative difference between A and B along the whole history is 4.0×10^{-14}; the control differs by 0.8 to 9.6 percent. Three checks: A is the record at all ten states; B is A at every slice; C differs at every slice. This is T3 slice by slice, as Section 19.6 predicted.

In [21], the energy along the history (Figure 19a.7).

```
fig, axes = plt.subplots(1, 2, figsize=(9.0, 3.8))
MARK = {"A": dict(marker="o", ms=7, lw=2.2), "B": dict(marker="s", ms=4, lw=1.4,
                                                       ls="--"),
        "C": dict(marker="^", ms=6, lw=1.2, ls=":"),
```

```
        "D": dict(marker="D", ms=5, lw=1.4)}
for ax, N in zip(axes, (136, 688)):
    for kind in "ABC":
        ax.plot(SLICES, [HIST[(N, kind, a4)]["E_KS"] for a4 in SLICES],
                color=COLOUR[kind], label=NAME[kind], **MARK[kind])
    ax.set_xlabel("slice $a_{4,0}$ of the history $a_4 = Hx_4$")
    ax.set_ylabel("Kohn-Sham energy $E_{KS}$ (units of $|m|$)")
    ax.set_title(f"$N = {N}$, $\\lambda = \\lambda_1$", fontsize=10)
axes[0].legend(fontsize=7, loc="upper right")
fig.tight_layout()
```

`MARK` gives each universe a marker (circle, square, triangle, diamond), a marker size and a line style, used in this figure and in Figures 19a.9 and 19a.10. The two axes show E_{KS} against the slice for $N = 136$ and $N = 688$.

```
save_figure(fig, "history_energy",
            "The Kohn-Sham energy $E_{KS}$ (units of $|m|$) of A (blue circles), B "
            "(orange squares, dashed) and C (aqua triangles, dotted) at the five "
            "slices $a_{4,0} = 0$ to $2$ of the deflating history, for $N = 136$ "
            "(left) and $N = 688$ (right) at the coupling $\\lambda_1$ of each $N$. "
            "The energy falls because the 3-momenta are redshifted as 3-space "
            "inflates and the extra times deflate; the T3 partner B follows A at "
            "every slice, the control C does not.")
```

`save_figure` saves Figure 19a.7. What the student should see: falling curves, from 80.3 to 12.4 ($N = 136$) and from 680 to 110 ($N = 688$), with B's small squares inside A's circles at every slice and the control's triangles below them (above at the last slice of $N = 688$). The fall is the redshift of Chapter 14: a later slice is the first slice with every 3-momentum multiplied by $e^{-a_{4,0}}$.

In [22], the agreement along the history (Figure 19a.8).

```
fig, ax = plt.subplots()
for N, marker in ((136, "o"), (688, "s")):
    for kind in "BC":
        values = [max(abs(HIST[(N, "A", a4)]["E_KS"] - HIST[(N, kind, a4)]["E_KS"])
                      / abs(HIST[(N, "A", a4)]["E_KS"]), FLOOR) for a4 in SLICES]
        ax.semilogy(SLICES, values, marker=marker, color=COLOUR[kind],
                    ls="--" if kind == "B" else ":",
                    label=f"$N = {N}$, A against {kind}")
ax.axhline(TOL, color="0.3", lw=1.0, label="tolerance $10^{-9}$")
ax.set_xlabel("slice $a_{4,0}$ of the history")
ax.set_ylabel("relative difference of $E_{KS}$")
ax.legend(fontsize=7, loc="center right")
```

For both particle numbers and for B and C, the relative energy difference from A at the five slices, raised to the floor 10^{-17} where it is exactly zero, on a logarithmic axis, with the tolerance line.

```
save_figure(fig, "history_agreement",
            "Relative differences of the Kohn-Sham energy between A and its T3 "
            "partner B (orange) and between A and the control C (aqua) at the five "
```

```
            "slices of the history, for $N = 136$ (circles) and $N = 688$ (squares); "
            "logarithmic vertical axis, exact zeros drawn at $10^{-17}$. The "
            "partner agrees to better than $10^{-13}$ at every slice: T3 holds slice "
            "by slice along the deflating history. The control is off by $0.5$ to "
            "$10$ percent.")
```

`save_figure` saves Figure 19a.8. What the student should see: the orange curves between 10^{-17} and a few times 10^{-14}, far below the tolerance line; the aqua curves between about 8×10^{-3} and 0.1.

In [23], the coupling must keep its sign: 17 runs.

```
l1, l2 = LAMBDA[136]
COUPLINGS = [("lamm2", -l2, 0.85), ("lamm1", -l1, 0.45), ("lam0", 0.0, 0.25),
             ("lamp1", l1, 0.45), ("lamp2", l2, 0.85)]  # tag, lambda, margin
SCAN = {}  # (tag, kind) -> state
for tag, lam, margin in COUPLINGS:
    for kind in "ABCD":
        if tag == "lamp1" and kind in "ABC":
            SCAN[(tag, kind)] = SELF[(136, kind)]  # already solved
        else:
            SCAN[(tag, kind)] = universe(kind, lam, 1.0, 136, margin)
MIRROR = {"lamm2": "lamp2", "lamm1": "lamp1", "lam0": "lam0", "lamp1": "lamm1",
          "lamp2": "lamm2"}  # the tag of -lambda
```

The five couplings $-\lambda_2, -\lambda_1, 0, +\lambda_1, +\lambda_2$ of $N = 136$, each with the record's tag and margin, for all four universes at $a_{4,0} = 1$: 20 states, three reused from In [7], so 17 new runs (D at $+\lambda_1$ is new). `MIRROR` gives for each tag the tag of the opposite coupling.

```
ok_record, ok_partner, ok_mirror, ok_wrong = True, True, True, True
control_gap = []  # |E_C - E_A| at each coupling
for tag, lam, margin in COUPLINGS:
    E = {kind: SCAN[(tag, kind)]["E_KS"] for kind in "ABCD"}
    ok_record &= close(E["A"], float(summary[f"N136_{tag}_a10"]["E_KS"]))
    ok_partner &= close(E["A"], E["B"]) and level_difference(
        SCAN[(tag, "A")], SCAN[(tag, "B")]) < TOL
    ok_mirror &= close(E["D"], SCAN[(MIRROR[tag], "A")]["E_KS"])
    ok_wrong &= (lam == 0.0) or abs(E["D"] - E["A"]) > 1e-4
    control_gap.append(abs(E["C"] - E["A"]))
    E_A, E_B, E_C, E_D = (E[kind] for kind in "ABCD")
    say(f"lambda = {lam:+.4e}: A {E_A:.10f}  B {E_B:.10f}  "
        f"C {E_C:.6f}  D {E_D:.10f}")
```

For every coupling, four things are tested. The truth value `ok_record`: A is the record's state of that tag, one of N136_lamm2_a10 to N136_lamp2_a10. The truth value `ok_partner`: B equals A in energy and levels. The truth value `ok_mirror`: D at λ equals A at $-\lambda$, because D $= (-m, -\lambda, \pi)$ is the T3 partner of $(m, -\lambda, 0)$. The truth value `ok_wrong`: D differs from A by more than 10^{-4} unless $\lambda = 0$ (at $\lambda = 0$, D and B are the same problem). The loop prints the four energies (Out [23]):

λ	**A and B**	**C**	**D**
-0.002789	32.3593734113	34.540794	32.4116192324
-0.0009298	32.3755880337	33.529490	32.3929491532
0	32.3841156874	30.731389	32.3841156874
$+0.0009298$	32.3929491532	29.283751	32.3755880337
$+0.002789$	32.4116192324	28.980584	32.3593734113

Read the D column from the bottom up: it is the A column. The control differs even at $\lambda = 0$, because its tip is wrong.

```
check(ok_record, "A reproduces the recorded E_KS for all five couplings",
      record=f"{GROUND}/summary.csv, N136_lamm2_a10 to N136_lamp2_a10")
check(ok_partner, "B = A for every coupling: the partner keeps +lambda")
check(ok_mirror and ok_wrong,
      "D(lambda) = A(-lambda), so (-m, -lambda) is not the partner for lambda != 0",
      record="Revision/pairing/kohn_sham/reports/python-t3.json, check "
             "T3.mean_field_map (control)")
check(1.0 < min(control_gap) and max(control_gap) < 3.5,
      "the control C differs from A by 1 to 3.5 at every coupling")
```

Four checks (Out [23]); the third is the numerical face of the control of Step 3 (Section 19.8), and the fourth guards the range 1.15 to 3.43 that the caption of Figure 19a.9 states as "1 to 3.5".

In [24], the coupling scan (Figure 19a.9).

```
x = [lam / l1 for tag, lam, margin in COUPLINGS]
E0 = SCAN[("lam0", "A")]["E_KS"]
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
for kind in "ABD":
    left.plot(x, [SCAN[(tag, kind)]["E_KS"] - E0 for tag, lam, margin in COUPLINGS],
              color=COLOUR[kind], label=NAME[kind], **MARK[kind])
left.set_xlabel("coupling $\\lambda/\\lambda_1$")
left.set_ylabel("$E_{KS} - E_{KS}(\\lambda = 0)$ (units of $|m|$)")
left.axhline(0.0, color="0.3", lw=0.8)
low, high = left.get_ylim()
left.set_ylim(low, high + 0.6 * (high - low))  # room for the legend at the top
left.legend(fontsize=7, loc="upper center")
```

The horizontal axis is the coupling in units of λ_1: $-3, -1, 0, 1, 3$ (the tuples of `COUPLINGS` are unpacked into three names, of which only `lam` is used). On the left, the interaction part of the energy, E_{KS} minus the value of A at $\lambda = 0$, for A, B and D. `get_ylim` reads the vertical range that matplotlib chose, and `set_ylim` enlarges it upward by 60 percent so that the legend does not cover the curves.

```
for kind in "AC":
    right.plot(x, [SCAN[(tag, kind)]["E_KS"] for tag, lam, margin in COUPLINGS],
               color=COLOUR[kind], label=NAME[kind], **MARK[kind])
right.set_xlabel("coupling $\\lambda/\\lambda_1$")
right.set_ylabel("$E_{KS}$ (units of $|m|$)")
```

```
right.legend(fontsize=7, loc="upper right")
fig.suptitle("$N = 136$, slice $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
save_figure(fig, "coupling_scan",
            "Left: the interaction part of the Kohn-Sham energy, $E_{KS}$ minus its "
            "value at $\\lambda = 0$ (units of $|m|$), against the coupling "
            "$\\lambda/\\lambda_1$ for $N = 136$, $a_{4,0} = 1$: universe A (blue), "
            "its T3 partner B with the same coupling (orange, dashed, on top of A) "
            "and D with the reversed coupling (yellow diamonds), which is the "
            "mirror image of A. Right: $E_{KS}$ of A and of the control C (aqua), "
            "which lies $1$ to $3.5$ away from A, above it for negative and below "
            "it for positive couplings. Only $(-m, +\\lambda)$ with the transformed "
            "tip is the partner of $(m, \\lambda)$.")
```

On the right, the energies of A and C. `save_figure` saves Figure 19a.9. What the student should see: on the left A's curve rising from -0.025 at $-3\lambda_1$ through 0 to $+0.027$ at $+3\lambda_1$, B's squares on it, and D's diamonds on the curve reflected about the vertical axis; on the right A's nearly flat curve near 32.4 and the control's falling curve, above A for the negative couplings and below A from $\lambda = 0$ on.

In [25], thermal states: nine runs.

```
THERMO = read_rows("Revision/kohn_sham/results/thermo/thermodynamics.csv")
TEMPS = [0.01, 0.02, 0.05]
WARM = {}  # (T, kind) -> state
ok_record, ok_partner, ok_control = True, True, True
for T in TEMPS:
    for kind in "ABC":
        WARM[(T, kind)] = universe(kind, l1, 1.0, 136, 0.4, T=T)
    A, B, C = (WARM[(T, kind)] for kind in "ABC")
    F = {kind: WARM[(T, kind)]["E_KS"] - T * WARM[(T, kind)]["entropy"]
         for kind in "ABC"}  # free energy F = E - T S
    row = THERMO[f"N136_lamp1_a10_T{round(1000 * T)}"]
```

The record's table of thermal states is read. For the three temperatures $T = 0.01, 0.02, 0.05$ of the record, A, B and C are solved at $N = 136$, λ_1, $a_{4,0} = 1$, with the margin 0.4 and Mermin occupations: nine runs. For each the free energy $F = E_{KS} - TS_{\text{ent}}$ is formed (Exercise 19.7 shows that this equals $\Omega + \mu N$ of Section 19.4), and the record's row is found by its name, which ends in T10, T20 or T50 (a thousand times the temperature).

```
    ok_record &= (close(A["mu_or_fermi_level"], float(row["mu"]))
                  and close(A["E_KS"], float(row["E"]))
                  and close(A["entropy"], float(row["entropy"]))
                  and close(F["A"], float(row["F"])))
    ok_partner &= (close(A["mu_or_fermi_level"], B["mu_or_fermi_level"])
                   and close(A["entropy"], B["entropy"]) and close(F["A"], F["B"])
                   and level_difference(A, B) < TOL)
    ok_control &= 2.5 < F["A"] - F["C"] < 3.5
    mu_A, mu_B, mu_C = (s["mu_or_fermi_level"] for s in (A, B, C))
    F_A, F_B, F_C = (F[kind] for kind in "ABC")
    say(f"T = {T:.2f}: mu A {mu_A:.12f} B {mu_B:.12f} C {mu_C:.6f}; "
        f"F A {F_A:.9f} B {F_B:.9f} C {F_C:.5f}")
```

A must reproduce the record's chemical potential, energy, entropy and free energy; B must have the chemical potential, entropy, free energy and levels (with their occupations) of A; and the control's free energy must lie between 2.5 and 3.5 below that of A (`2.5 < x < 3.5` tests both bounds). Out [25] shows, for example at $T = 0.05$: $\mu = 0.295073925379$ and $F = 27.091045285$ for both A and B, and $\mu = 0.289068$, $F = 24.21400$ for the control.

```
check(ok_record, "A reproduces mu, E, entropy and F of the three thermal states",
      record="Revision/kohn_sham/results/thermo/thermodynamics.csv, "
             "N136_lamp1_a10_T10, T20, T50")
check(ok_partner, "B has the occupations, mu, entropy and F of A at every T",
      record="Revision/pairing/kohn_sham/t3-theory.json, statement S4")
check(ok_control, "the control C has a free energy 2.5 to 3.5 below A at every T")
```

Three checks (Out [25]).

In [26], the thermal states (Figure 19a.10).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
T = 0.05
DOTS = {"A": dict(marker="o", ms=7, ls="none", mfc="none", mew=1.6),
        "B": dict(marker="x", ms=5, ls="none", mew=1.4),
        "C": dict(marker="^", ms=4, ls="none")}
for kind in "ABC":
    lv = WARM[(T, kind)]["levels"]
    left.plot([v[4] for v in lv], [v[6] for v in lv], color=COLOUR[kind],
              label=NAME[kind], **DOTS[kind])
mu = WARM[(T, "A")]["mu_or_fermi_level"]
grid = np.linspace(-0.3, 1.0, 400)
left.plot(grid, 1.0 / (1.0 + np.exp((grid - mu) / T)), color="0.4", lw=1.0,
          label="Fermi function of A")
```

On the left, at $T = 0.05$, every level is a marker at the height of its occupation: A as open rings (`mfc="none"`: no fill; `mew`: the width of the marker's edge), B as crosses, C as small triangles, without connecting lines (`ls="none"`). The grey curve is the Fermi function $1/(1 + e^{(\varepsilon-\mu)/T})$ of A on 400 points.

```
left.set_xlim(-0.3, 1.0)
left.set_xlabel("level $\\varepsilon$ (units of $|m|$)")
left.set_ylabel("occupation $f$")
left.set_title(f"$T = {T}$", fontsize=10)
left.legend(fontsize=6, loc="lower left")
for kind in "ABC":
    right.plot(TEMPS, [WARM[(t, kind)]["E_KS"] - t * WARM[(t, kind)]["entropy"]
                       for t in TEMPS], color=COLOUR[kind], label=NAME[kind],
               **MARK[kind])
right.set_xlabel("temperature $T$ (units of $|m|$)")
right.set_ylabel("free energy $F = E - TS_{ent}$ (units of $|m|$)")
fig.suptitle("$N = 136$, $\\lambda = \\lambda_1$, $a_{4,0} = 1$", fontsize=10)
fig.tight_layout()
```

The left axes show the levels between -0.3 and 1.0. On the right, the free energy of the three universes against the temperature.

```
save_figure(fig, "thermal_states",
            "Thermal Kohn-Sham states for $N = 136$, $\\lambda = \\lambda_1$, "
            "$a_{4,0} = 1$. Left: the occupation $f$ of every level against its "
            "energy (units of $|m|$) at $T = 0.05$ for A (blue rings), B (orange "
            "crosses) and C (aqua triangles); the grey curve is the Fermi function "
            "of A. Every cross of B sits in a ring of A. Right: the free energy "
            "$F = E - TS_{ent}$ against the temperature $T$ (units of $|m|$); A and "
            "B coincide, the control C lies $2.5$ to $3.5$ lower.")
```

`save_figure` saves Figure 19a.10. What the student should see: on the left the rings of A on the grey Fermi curve, falling from 1 to 0 around $\mu = 0.295$, each with an orange cross inside it, and the control's triangles on its own, shifted curve; on the right the free energy falling with the temperature, A and B on top of each other (32.24, 31.59, 27.09) and the control about 3 lower.

In [27], the same runs in the record's demonstration of T3. The record's numerical demonstration (Section 19.10) solved the members plus, image and control for all 210 states; this cell checks that the runs of this notebook are the record's runs where the two overlap.

```
DEMO = "Revision/pairing/kohn_sham/reports/t3-rust-demo.json"
DEMO_TABLE = "Revision/pairing/kohn_sham/numerics/results/t3-rust-states.csv"
demo = json.loads(repository_file(DEMO).read_text(encoding="utf-8"))
verdict = {c["name"]: c["verdict"] for c in demo["checks"]}
detail = {c["name"]: c["detail"] for c in demo["checks"]}
passed = sum(v == "PASS" for v in verdict.values())
say(f"record: {demo['states']} states, {passed} of {len(verdict)} checks pass")
```

The two names are the report of the demonstration and its table. `json.loads` reads the report into a dictionary; its list `checks` holds one dictionary per check, with the keys `name`, `verdict` and `detail`. The two dictionary comprehensions make the look-ups "name to verdict" and "name to detail text". `passed` counts the verdicts equal to PASS (a comparison is `True` or `False`, and `sum` counts each `True` as 1). The line prints the number of states of the record (its key `states`) and the count: "record: 210 states, 7 of 7 checks pass" (Out [27]).

```
for name in ("t3_equal_ground_states", "t3_equal_thermal_states"):
    worst = re.search(r"worst deviation (\S+),", detail[name]).group(1)
    say(f"record, {name}: worst deviation {worst}")
DEMO_ROWS = read_rows(DEMO_TABLE)
PAIRS, GROUP = {}, {}  # state id -> [A, B, C]; state id -> group of runs
```

The detail text of each of the two equality checks contains the words "worst deviation" followed by a number and a comma; the regular expression finds them, and `group(1)` is the part matched by `(\S+)`, the number (`\S+` means one or more characters that are not spaces). Out [27] prints the record's own worst deviations, 2.179×10^{-13} for the 75 ground states and 3.877×10^{-12} for the 135 thermal states. `read_rows` (In [9]) reads the table into a dictionary from the state id, such as N136_lamp1_a10, to its row. `PAIRS` will hold, for every state of the record that this notebook solved, the three states A, B and C of this notebook, and `GROUP` the group of runs the state belongs to.

```
def add(state_id, group, states):
    """Store the states A, B, C of one state of the record, once."""
```

```
    if state_id not in PAIRS:
        PAIRS[state_id], GROUP[state_id] = states, group
```

`add` stores a state only the first time its id appears. This matters because some runs were reused: the state N136_lamp1_a10 is in `SELF`, in `HIST` (the slice $a_{4,0} = 1$) and in `SCAN` (the coupling $+\lambda_1$), and must be counted once.

```
for N in (8, 136):
    add(f"N{N}_lamp1_a10", "ground", [SELF[(N, kind)] for kind in "ABC"])
add("N8_lam0_a10", "ground", [FREE8[kind] for kind in "ABC"])
for tag, lam, margin in COUPLINGS:
    add(f"N136_{tag}_a10", "ground", [SCAN[(tag, kind)] for kind in "ABC"])
for N in (136, 688):
    for a4 in SLICES:
        add(f"N{N}_lamp1_a{round(10 * a4):02d}", "history",
            [HIST[(N, kind, a4)] for kind in "ABC"])
for T in TEMPS:
    add(f"N136_lamp1_a10_T{round(1000 * T)}", "thermal",
        [WARM[(T, kind)] for kind in "ABC"])
```

The ids are built in the naming scheme of the record (Section 19.1): `round(10 * a4):02d` writes ten times the slice with two digits (0.5 becomes 05), and `round(1000 * T)` writes the temperature in thousandths (0.05 becomes 50). The group "ground" receives the states at the slice $a_{4,0} = 1$ of In [7] (two), In [17] (one, without interaction) and In [23] (four more couplings; $+\lambda_1$ is already stored): 7 states. The group "history" receives the states of In [20] not yet stored: four slices for $N = 136$ and five for $N = 688$, 9 states. The group "thermal" receives the 3 states of In [25]. Together 19 states.

```
def relative(a, b):
    """|a - b| relative to max(|a|, 1), the measure of close()."""
    return abs(a - b) / max(abs(a), 1.0)
```

`relative` measures a difference in the same way as `close` of In [8]: relative to the size of the first number, but never relative to a number smaller than 1.

```
worst_A, worst_B, worst_C = 0.0, 0.0, 0.0
for state_id, (A, B, C) in PAIRS.items():
    row = DEMO_ROWS[state_id]  # the record's row of this state
    worst_A = max(worst_A, relative(A["E_KS"], float(row["E_KS_plus"])))
    image = [(B["E_KS"], row["E_KS_image"]), (B["mu_or_fermi_level"],
             row["mu_image"]), (B["entropy"], row["entropy_image"])]
    image += [(B["emtIntegrals_2Vol7_int_e6Hy"][c], row[f"int_{c}_image"])
              for c in ("rho", "p3", "p_t", "p0")]
    worst_B = max([worst_B] + [relative(x, float(r)) for x, r in image])
    control = float(row["E_KS_control_untransformed_tip"])
    worst_C = max(worst_C, relative(C["E_KS"], control))
```

For every one of the 19 states the loop takes the record's row and compares: the energy of A with the record's plus member (column `E_KS_plus`); seven numbers of B with the record's image member (the energy, the chemical potential or Fermi level, the entropy, which is 0 in a ground state, and the

four energy-momentum integrals, columns `int_rho_image` and so on); and the energy of C with the record's control (column `E_KS_control_untransformed_tip`). The table stores numbers as text, so `float` converts them. `max([worst_B] + [...])` takes the largest of the old worst value and the seven new differences.

```
worst_D = max(relative(SCAN[(tag, "D")]["E_KS"],
                       float(DEMO_ROWS[f"N136_{MIRROR[tag]}_a10"]["E_KS_image"]))
              for tag, lam, margin in COUPLINGS)
say(f"{len(PAIRS)} states of the record solved here; largest relative differences "
    f"from the record: A {worst_A:.1e}, B {worst_B:.1e}, C {worst_C:.1e}, "
    f"D {worst_D:.1e}")
```

D at the coupling λ is the universe $(-m, -\lambda)$ with the tip angle π, which is the image member of the record's state at the coupling $-\lambda$; `MIRROR` (In [23]) gives the tag of $-\lambda$. Out [27] prints the four largest differences: A 4.0×10^{-14}, B 2.8×10^{-13}, C 7.0×10^{-14}, D 0. The record's runs and these runs were made by the same program on the same computer, so most numbers agree to every digit and the others to the rounding of the last digits.

```
def t3_deviation(A, B):
    """Largest deviation of B from A under T3 (levels, E_KS, mu, entropy, EMT
    integrals, profiles with the odd columns reversed)."""
    return max(level_difference(A, B), relative(A["E_KS"], B["E_KS"]),
               relative(A["mu_or_fermi_level"], B["mu_or_fermi_level"]),
               relative(A["entropy"], B["entropy"]), integral_difference(A, B),
               profile_mismatch(A, B))
```

`t3_deviation` combines every comparison of A and B of this notebook into one number: the sorted levels with degeneracies and occupations (`level_difference`, In [7]), the energy, the chemical potential, the entropy, the four energy-momentum integrals (`integral_difference`, In [7]) and the ten profile columns with S, Q, M_{eff} reversed (`profile_mismatch`, In [13]).

```
largest = {}
for group in ("ground", "history", "thermal"):
    ids = [state_id for state_id in PAIRS if GROUP[state_id] == group]
    largest[group] = max(t3_deviation(*PAIRS[state_id][:2]) for state_id in ids)
    say(f"{group}: {len(ids)} pairs A, B; largest T3 deviation "
        f"{largest[group]:.1e}")
```

For each group, `ids` lists its states; `PAIRS[state_id][:2]` is the list `[A, B]` of a state, and the star `*` hands its two entries to `t3_deviation` as its two arguments. Out [27]: ground 7 pairs, largest deviation 9.9×10^{-14}; history 9 pairs, 1.0×10^{-13}; thermal 3 pairs, 8.1×10^{-13} (the entropy of the state N136_lamp1_a10_T10 at $T = 0.01$, a number near 42 that the solver adds up level by level from the occupations; the record's table `Revision/pairing/kohn_sham/numerics/results/t3-rust-states.csv` shows the same deviation, 8.117×10^{-13}, for this state in its column t3_scalars_dev). These are the rounding of the computer, and, as Section 19.17 showed, at the rounding level by construction, because the shooting is covariant under the map.

```
check(passed == len(verdict) == 7 and demo["states"] == 210,
      "the record's T3 demonstration passes 7 of 7 checks in 210 states",
```

```
      record=DEMO)
check(len(PAIRS) == 19 and worst_A < TOL and worst_B < TOL,
      "A and B equal the record's plus and image members in 19 states",
      record=f"{DEMO}, checks t3_equal_ground_states, t3_equal_thermal_states")
check(worst_C < TOL, "C equals the record's control with the untransformed tip",
      record=f"{DEMO}, check negative_control_untransformed_tip")
check(worst_D < TOL, "D at lambda equals the record's image member at -lambda",
      record=f"{DEMO}, check negative_control_lambda_sign")
check(max(largest.values()) < TOL,
      "B equals A under T3 in all 19 pairs (ground, history, thermal)")
```

Five checks, each with the tolerance 10^{-9} of the record: the record's demonstration passes completely; A and B are the record's plus and image members in all 19 states; C is the record's control; D is the record's image at the opposite coupling; and B equals A under T3 in every pair of this notebook. The PASS lines name the report and the check of the record that each comparison reproduces (Out [27]).

In [28], the last check.

```
runs = len({id(state) for state in [*SELF.values(), *FREE8.values(), *HIST.values(),
                                    *SCAN.values(), *WARM.values()]})
report("runs of the Rust solver in this notebook", runs)
names = ["level_ladder", "level_differences", "density_profiles",
         "mass_and_potential", "emt_profiles", "zero_modes_n8", "history_energy",
         "history_agreement", "coupling_scan", "thermal_states"]
paths = [output_file(f"{FIGURE_FOLDER}/19a_{k}_{name}.png")
         for k, name in enumerate(names, 1)]
check(runs == 62 and all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

`id(state)` is a number that identifies one object in the computer's memory; a state that was reused (stored under two keys) has one `id`, so the set of the ids of all stored states counts every run once: $6 + 3 + 27 + 17 + 9 = 62$. The check requires 62 runs and all ten figure files (`enumerate(names, 1)` counts from 1), and the last line is ALL 37 CHECKS PASSED (notebook 19a) (Out [28]). Of the 62 runs, 19 states compare A with its partner B (In [27]); the other runs are the controls C and D.

19.22 What T3 says about pairs of universes, and what it does not

What is proved. For the fermion field dirac16complex in the author's primordial universe, in the Kohn-Sham model of the Revision record: to every self-consistent instantaneous Kohn-Sham state of the universe of mass $+M$ (bare mass m, coupling λ, tip angle θ) the chirality Γ, acting on the blocks as σ_2, assigns a self-consistent Kohn-Sham state of the universe of mass $-M$ with the same coupling and the tip angle $\pi - \theta$, at the same slice of the deflating history, with the same levels, occupations, chemical potential, entropy, Kohn-Sham energy, grand potential, free energy and energy-momentum profiles, and the opposite scalar density; and conversely. Inside the ASSUMED Z2 orbifold, the mirror copy of every state carries $(-m, +\lambda)$. With the completion of the record (statement S6, Section 19.10), at the level of the 16-component expectation rule, every component of the energy-momentum tensor and of the current J^a is the same in the two members, while S

and Q change sign; and the filling convention of the $+M$ member, carried by the map, is the filling convention of the $-M$ member. Status: PROVED (Sections 19.5 to 19.11; `wolfram-t3.json` 10 of 10, `python-t3.json` 13 of 13; the completion `t3-completion.json` with `wolfram-t3-completion.json` 3 of 3 and `python-t3-completion.json` 7 of 7).

What is demonstrated numerically (COMPUTED; demonstrations, not proofs). The record's Rust demonstration (`t3-rust-demo.json`) finds the $-M$ partner equal to the $+M$ state in all 75 ground and 135 Mermin states of the canonical matrix, to 2.179×10^{-13} (ground) and 3.877×10^{-12} (thermal); this agreement is at the rounding level by construction, because the solver's shooting is covariant under the map (Section 19.17), so it tests the solver's handling of the two members, not the discretisation. The record's reference demonstration (`t3-reference-demo.json`) is the discretisation-independent test: on one grid the two members differ by up to 2.23×10^{-4}, after Richardson extrapolation by 2.043×10^{-14} (ground) and 3.038×10^{-14} (thermal), in 18 states; and the $-M$ universes of the two solvers agree to 1.255×10^{-11}. Notebook 19a compares A and B in 19 states (7 ground states at $a_{4,0} = 1$, 9 more states of the history, 3 thermal states; 62 runs with the controls), with the largest deviations 9.9×10^{-14}, 1.0×10^{-13} and 8.1×10^{-13} (Out [27]), and finds every run equal to the record's demonstration. The controls do differ: the untransformed tip changes the density profile by at least 4.750 times its maximum, and the T1 parameters $(-m, -\lambda)$ differ by at least 1.21×10^{-2}.

What is not established (the list of the theorem record `t3-theory.json`, entry not_established, in words):

- **No creation.** T3 maps self-consistent Kohn-Sham states onto self-consistent Kohn-Sham states of a second parameter set. It contains no process in which a universe, or a pair of universes, comes into being, no rate, no probability and no amplitude, and it does not force the partner to exist: a universe of mass $+M$ alone is an equally valid solution. The statement "the big bang creates universes in pairs" is the author's HYPOTHESIS; T3 is compatible with it, in the sense that a partner state with equal energy is always available, but compatibility is not a proof.

- **Instantaneous states only.** T3 holds slice by slice for the instantaneous (adiabatic) states along the PRESCRIBED BACKGROUND history $a_4 = AHx_4$; the time-dependent Kohn-Sham problem along the deflating history is OPEN.

- **An assumed brane and a chosen tip.** The Z2 mirror at the brane is ASSUMED; the tip condition is a choice of the model, and T3 needs its transformed form $\pi - \theta$ (Section 19.12 shows what fails without it).

- **Mean field only.** Hartree plus the exchange of the uniform gas, no correlation, quasi-free states, and the filling convention whose justification is OPEN.

- **Not two independently quantised universes.** The pairing is between two Kohn-Sham problems, (m, λ) and $(-m, +\lambda)$; it is not the T1 pairing $(m, \lambda) \to (-m, -\lambda)$, and it says nothing about two separately quantised universes (that question is the quantum reading Q of Chapter 18).

- **No back-reaction.** The Kohn-Sham energy-momentum tensor is not an admissible source of the a_4 field equations: it depends on x_0, and $p_3 + p_t \neq 2p_0$ (`Revision/field_equations_a4/reports/ks-source-conditions.json`, checks ks_profiles_depend_on_x0

and ks_profiles_violate_algebraic_condition). So T3 says nothing about how the geometry of either universe would respond.

- **Numbers only where they were computed.** The numerical demonstrations show the equalities only on the states they solved, to their stated tolerances. Control states for which a solver found no self-consistent solution (14 states in the Rust demonstration, 3 in the reference demonstration, all with $\lambda < 0$) are reported as such, not as a proof that no such state exists (the completion record `t3-completion.json`, entry not_established).

Matter and antimatter. T3 does not touch the matter-antimatter question. The partner has the same charge density $n(y)$ and the same particle number N, not the opposite ones, and by S6 every component of its current J^a is the same, not the opposite; it also has the same, not the opposite, energy: a T3 pair has twice the charge and twice the energy of one member; nothing cancels. (The pair-level statement with zero total charge belongs to T1 and the classical bilinears, Chapter 18.) Nothing in the Kohn-Sham model contains baryons, a violation of baryon number, a violation of CP, or a departure from equilibrium, the ingredients that an explanation of the observed excess of matter over antimatter would need. Chapter 21 teaches the observations and these conditions from zero and states what the theory of this book does and does not say about them; Chapter 20 collects what the pairing theorems T1, T2, Q and T3 together prove about pairs of universes and what they do not.

19.23 What we proved, what we computed, what we assumed

- PROVED, line by line in this chapter and in the records `Revision/pairing/kohn_sham/reports/wolfram-t3.json` (10 of 10 checks) and `Revision/pairing/kohn_sham/reports/python-t3.json` (13 of 13), theorem record `Revision/pairing/kohn_sham/t3-theory.json`: the chirality Γ maps every block (j, s_2, s_3) onto $(-j, s_2, s_3)$ and acts there as $s_2\sigma_2$ (Section 19.5; also blocks_relation_to_Gamma of `ks-theory-python.json`); $\sigma_2 h_j(M, v)\sigma_2 = h_{-j}(-M, v)$ and $\sigma_2 N_j(M)\sigma_2 = N_{-j}(-M)$ (Section 19.6); the tip angle goes to $\pi - \theta$, the brane parities are exchanged, norm and current are kept (Section 19.7); n, t are even and S, Q odd, and the mean field closes exactly for $(-m, +\lambda)$, not for $(-m, -\lambda)$ (Section 19.8); every self-consistent state has its partner with equal levels, occupations, energies and energy-momentum profiles (Section 19.9): theorem T3 (Section 19.10). Also PROVED, by the completion `Revision/pairing/kohn_sham/t3-completion.json` (`reports/wolfram-t3-completion.json` 3 of 3, `reports/python-t3-completion.json` 7 of 7): the mean-field coefficients read from `ks-theory.json` in both engines, with $M_{\text{eff}} - m = \partial e_{\text{int}}/\partial S$ and $v_v = \partial e_{\text{int}}/\partial n$ (Section 19.8); the filling convention carried onto the partner (Section 19.9); statement S6, $\rho' = -\Gamma\rho\Gamma$, so that every component of the energy-momentum tensor and of the current is unchanged and S, Q change sign (Section 19.10). Also PROVED: $\Gamma B \Gamma = -B$, the reason why T3 differs from T1 (Sections 19.5 and 19.11; check Q.image_krein_metric of `Revision/pairing/reports/python-pairing.json`); the mirror copy inside the assumed orbifold carries $(-m, +\lambda)$ (Section 19.11); the exact zero-momentum spectra, the zero modes, the control's level in the gap $\varepsilon_b = M/\cosh(qL)$ with $\tanh(qL) = q/M$, and the slopes c and c_{ctrl} (Section 19.12; checks T3_exact_k0_spectra, bc_exact_k0_spectra, brane_band_slope).
- COMPUTED: Notebook 19b (19 checks) reproduces the proof checks with sympy, the analytic levels of `Revision/kohn_sham/results/spectrum/free-k0-analytic.csv` to 8.9×10^{-16}, the same levels by shooting to 5.9×10^{-12}, and the slope $c = 1.9051482536$ of the record;

$c_{\mathrm{ctrl}} = 13.4219751976$. The record's demonstrations (Section 19.10): Rust, `Revision/pairing/kohn_sham/reports/t3-rust-demo.json` (210 states, 7 of 7), image equal to plus to 2.179×10^{-13} (ground) and 3.877×10^{-12} (thermal), at the rounding level by construction (the covariance of the shooting, Section 19.17); reference, `Revision/pairing/kohn_sham/reports/t3-reference-demo.json` (18 states, 7 of 7), equal to 2.043×10^{-14} and 3.038×10^{-14} after Richardson extrapolation against 2.23×10^{-4} on a single grid, the discretisation-independent test; controls off by at least 4.750 (untransformed tip) and 1.21×10^{-2} (T1 parameters). Notebook 19a (37 checks, 62 runs of the Rust solver, 19 states with A and B) reproduces the solver's T3 self-test of `Revision/kohn_sham/reports/ks-rust-solver.json` (check t3_block_map_solver_selftest), the committed canonical states and, in its 19 states, the record's demonstration `t3-rust-demo.json`; it finds the partner B equal to A to 9.9×10^{-14} (ground states), 1.0×10^{-13} (history) and 8.1×10^{-13} (thermal states) and 4.0×10^{-14} relative in the energy along the whole history; the control C off by 0.8 to 9.6 percent along the history and by 1.15 to 3.43 across the couplings; the wrong partner D equal to A at the opposite coupling; the control's Kohn-Sham gap equal to $\varepsilon_b = 0.100851121375$ to twelve digits.

- ASSUMED: the good sector; the Z2 mirror at the brane; the cut at $L = 3$ with a chosen tip condition (T3 needs the transformed one); the mean field of Hartree plus uniform-gas exchange without correlation. CONVENTION: the filling of the particle levels.
- PRESCRIBED BACKGROUND: the history $a_4 = AHx_4$, $A = 1$, on the background $b_4 = 1$ of the author's metric of 2026-10-14; the Kohn-Sham states are not an admissible source of the a_4 equations.
- OPEN: the time-dependent Kohn-Sham problem; the justification of the filling convention.
- HYPOTHESIS: that universes are created in pairs. Not proved here, and not provable from these equations, which contain no creation process.

19.24 Exercises

Exercise 19.1 (the conjugation rules). Multiply out the 2×2 matrices to show $\sigma_2\sigma_1\sigma_2 = -\sigma_1$ and $\sigma_2\sigma_3\sigma_2 = -\sigma_3$, without using the anticommutation rule.

Answer. First $\sigma_2\sigma_1 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} -i & 0 \\ 0 & i \end{pmatrix}$ (row times column: the first row $(0, -i)$ times the first column $(0, 1)$ gives $-i$, times the second column $(1, 0)$ gives 0; the second row $(i, 0)$ gives 0 and i). Then $(\sigma_2\sigma_1)\sigma_2 = \begin{pmatrix} -i & 0 \\ 0 & i \end{pmatrix}\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} 0 & (-i)(-i) \\ i \cdot i & 0 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ -1 & 0 \end{pmatrix} = -\sigma_1$. Likewise $\sigma_2\sigma_3 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & i \\ i & 0 \end{pmatrix}$ and $(\sigma_2\sigma_3)\sigma_2 = \begin{pmatrix} 0 & i \\ i & 0 \end{pmatrix}\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} i \cdot i & 0 \\ 0 & i(-i) \end{pmatrix} = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix} = -\sigma_3$.

Exercise 19.2 (the zero modes in the real form). The zero mode of A is $(a, b) = (e^{My}, 0)$ with $j = +1$, mass M, $k = 0$, $v = 0$, $\varepsilon = 0$. Write down its image under $(a, b, j, M) \to (b, a, -j, -M)$ and verify directly that the image solves the real form of B and obeys B's boundary conditions (odd parity at the brane, tip angle π).

Answer. The image is $(a, b) = (0, e^{My})$ with $j = -1$ and the mass $-M$. The real form with $k = 0$, $v = 0$, $\varepsilon = 0$ is $a' = (-M)a - 0 \cdot b = -Ma$ and $b' = 0 \cdot a - (-M)b = Mb$. For $a = 0$: $a' = 0 = -M \cdot 0$; for $b = e^{My}$: $b' = Me^{My} = Mb$. Both hold. The odd brane condition is $a(0) = 0$: true, since $a = 0$ everywhere. The tip angle π demands $a(-L) = 0$: true. So the image is the zero mode of B, largest at the brane, as Section 19.12 found.

Exercise 19.3 (a tip that needs no transformation). For which tip angles θ is the transformed condition, with $\pi - \theta$, the same condition as the original? What does the condition say in the real form, and why is it unchanged by $(a, b) \to (b, a)$?

Answer. The tip matrix $Q(\theta) = \cos\theta\,\sigma_3 + \sin\theta\,\sigma_2$ repeats with the period 2π, so the two conditions agree when $\pi - \theta = \theta$ up to a multiple of 2π: $\theta = \pi/2$ or $\theta = -\pi/2$. For $\theta = \pi/2$, $Q = \sigma_2$, and $(1 - \sigma_2)\chi = 0$ reads $\chi_1 + i\chi_2 = 0$ in the first row (and $-i\chi_1 + \chi_2 = 0$ in the second, the same equation times $-i$): $\chi_2 = i\chi_1$. With $\chi = (a, ib)$ this is $ib = ia$, that is $b(-L) = a(-L)$, which is unchanged when a and b are exchanged. For $\theta = -\pi/2$ the condition is $b(-L) = -a(-L)$, also unchanged. With such a tip the "control" C would be the partner B; the canonical tip $\theta = 0$ is not of this kind.

Exercise 19.4 (every tip condition kills the current). Let $(1 - Q(\theta))\phi = 0$ and $(1 - Q(\theta))\chi = 0$ at $y = -L$. Show that $\phi^\dagger \sigma_1 \chi = 0$ there, using only $Q^\dagger = Q$ and $\sigma_1 Q = -Q\sigma_1$; prove these two facts first.

Answer. $Q^\dagger = \cos\theta\,\sigma_3^\dagger + \sin\theta\,\sigma_2^\dagger = Q$, because the Pauli matrices are Hermitian and $\cos\theta$, $\sin\theta$ are real. $\sigma_1 Q = \cos\theta\,\sigma_1\sigma_3 + \sin\theta\,\sigma_1\sigma_2 = -\cos\theta\,\sigma_3\sigma_1 - \sin\theta\,\sigma_2\sigma_1 = -Q\sigma_1$, because σ_1 anticommutes with σ_3 and with σ_2. The conditions say $Q\chi = \chi$ and $Q\phi = \phi$. Then, line by line: $\phi^\dagger\sigma_1\chi = \phi^\dagger\sigma_1 Q\chi$ (insert $\chi = Q\chi$) $= -\phi^\dagger Q\sigma_1\chi$ (anticommute) $= -(Q^\dagger\phi)^\dagger\sigma_1\chi$ (since $\phi^\dagger Q = (Q^\dagger\phi)^\dagger$) $= -(Q\phi)^\dagger\sigma_1\chi = -\phi^\dagger\sigma_1\chi$. A number equal to its own negative is 0. So the boundary term of Section 19.3 vanishes at the tip for every θ.

Exercise 19.5 (how wrong is the wrong partner?). In the record's canonical state N136_lamp1_a10 ($\lambda_1 = 0.0009298$) the scalar density at $y = -2.4$ is $S = -37$ (the record's profile file `Revision/kohn_sham/results/ground/profiles/N136_lamp1_a10.csv`, rounded). How far is the mass that the wrong partner $(-m, -\lambda)$ gives to the image density from the mass $-M_{\text{eff}}$ that the image needs at that point?

Answer. By Section 19.8 the wrong partner gives $-M_{\text{eff}} + \frac{15}{8}\lambda S$. The mismatch is $\frac{15}{8}\lambda S = 1.875 \times 0.0009298 \times (-37) = -0.0645$ in units of $|m|$. The right partner gives exactly $-M_{\text{eff}}$; here $M_{\text{eff}} = 1 + \frac{15}{16}\lambda_1 S = 1 - 0.0323 = 0.968$, close to the value 0.9677 of the profile file. So the wrong partner's mass is off by twice the mean-field shift $\frac{15}{16}\lambda S = -0.0323$, which is why Notebook 19a finds D at $+\lambda$ equal to A at $-\lambda$ and different from A at $+\lambda$ (In [23]).

Exercise 19.6 (the two slopes for M = H). Show that for $M = H$ and $Y = e^{HL}$ the slopes of Section 19.12 are $c = 2Y/(Y+1)$ and $c_{\text{ctrl}} = \frac{2}{3}(Y^2+Y+1)/(Y+1)$, that $c_{\text{ctrl}} - c = \frac{2}{3}(Y-1)^2/(Y+1)$, and evaluate the difference for $L = 3$.

Answer. With $M = H$, $2M - H = H$ and $2M/(2M - H) = 2$: $c = 2(1 - e^{-HL})/(1 - e^{-2HL}) = 2(1-Y^{-1})/(1-Y^{-2})$. Multiply the numerator and the denominator by Y^2: $c = 2(Y^2-Y)/(Y^2-1) = 2Y(Y-1)/((Y-1)(Y+1)) = 2Y/(Y+1)$. With $2M + H = 3H$: $c_{\text{ctrl}} = \frac{2}{3}(e^{3HL} - 1)/(e^{2HL} - 1) = \frac{2}{3}(Y^3 - 1)/(Y^2 - 1) = \frac{2}{3}(Y-1)(Y^2 + Y + 1)/((Y-1)(Y+1)) = \frac{2}{3}(Y^2 + Y + 1)/(Y+1)$, using $Y^3 - 1 = (Y-1)(Y^2+Y+1)$. The difference over the common denominator: $\left[\frac{2}{3}(Y^2+Y+1) - 2Y\right]/(Y+1) = \frac{2}{3}(Y^2 - 2Y + 1)/(Y+1) = \frac{2}{3}(Y-1)^2/(Y+1)$, positive for every $L > 0$: the control's band is

always steeper. For $L=3$, $Y=e^3=20.0855$: $\frac{2}{3}\times 19.0855^2/21.0855=\frac{2}{3}\times 364.26/21.0855=11.517$, the difference $13.4220-1.9051$ of Notebook 19b.

Exercise 19.7 (the free energy). With the Fermi occupations $f=1/(1+e^x)$, $x=(\varepsilon-\mu)/T$, show that $F=\Omega+\mu N$ equals $E_{KS}-TS_{\text{ent}}$, with the definitions of Section 19.4.

Answer. Three facts about one level, line by line. (1) $1-f=e^x/(1+e^x)=1/(1+e^{-x})$, so $\ln(1+e^{-x})=-\ln(1-f)$. (2) $\ln f=-\ln(1+e^x)=-x-\ln(1+e^{-x})=-x+\ln(1-f)$, using $1+e^x=e^x(1+e^{-x})$ and fact (1). (3) Hence $f\ln f+(1-f)\ln(1-f)=-xf+f\ln(1-f)+(1-f)\ln(1-f)=-xf+\ln(1-f)$. Summed with the degeneracies: $TS_{\text{ent}}=-T\sum g[-xf+\ln(1-f)]=\sum gf(\varepsilon-\mu)-T\sum g\ln(1-f)=\sum gf\varepsilon-\mu N+T\sum g\ln(1+e^{-x})$, using $Tx=\varepsilon-\mu$, $\sum gf=N$ and fact (1). Write $I=2\,\text{Vol}_7\int e^{6Hy}e_{\text{int}}\,dy$. Then $\Omega=-T\sum g\ln(1+e^{-x})-I=(\sum gf\varepsilon-\mu N-TS_{\text{ent}})-I=E_{KS}-TS_{\text{ent}}-\mu N$, using the line before and $E_{KS}=\sum gf\varepsilon-I$. Adding μN gives $F=E_{KS}-TS_{\text{ent}}$. Notebook 19a computes F in this form (In [25]).

Exercise 19.8 (when does the control have a level in the gap?). Take $M=1$ and the tip at $L=\frac{1}{2}$ instead of $L=3$. Does the control's odd sector still have a level inside the gap?

Answer. The level in the gap is a root q in $(0,M)$ of $u(q)=\tanh(qL)-q/M$ (Section 19.12). Now $u(0)=0$ and $u'(0)=L-1/M=-\frac{1}{2}<0$. The function tanh is concave for $q>0$, so u is concave there, and a concave function that starts at 0 with a negative slope stays below the line $u'(0)q$, hence negative, for all $q>0$: there is no root, and no level inside the gap. At the edge $\varepsilon=M$ ($p=0$) the odd characteristic function of C is $\cosh 0-M\cdot L=1-\frac{1}{2}\neq 0$, so there is no level at the edge either. The control has a level in the gap exactly when $L>1/M$; the record's $L=3$ is such a case.

Exercise 19.9 (Krein signs of the eight blocks). Using the block form $B=js_2$, list the Krein signs of the eight blocks and show that Γ, which moves (j,s_2,s_3) to $(-j,s_2,s_3)$, reverses the Krein sign of every block. How many blocks have the Krein sign $+1$?

Answer. The products js_2 for $(j,s_2)=(1,1),(1,-1),(-1,1),(-1,-1)$ are $+1,-1,-1,+1$, each for $s_3=\pm 1$: the blocks $(1,1,\pm 1)$ and $(-1,-1,\pm 1)$ have the Krein sign $+1$, the blocks $(1,-1,\pm 1)$ and $(-1,1,\pm 1)$ have -1. Four blocks have $+1$ and four -1, eight components each way, in agreement with the eight eigenvalues $+1$ and eight -1 of B. The partner block of (j,s_2,s_3) has the sign $(-j)s_2=-js_2$: reversed, for every block. This is $\Gamma B\Gamma=-B$ written block by block (Section 19.5).

20 Do universes come in pairs? What the equations prove and what they do not

The author asked for a proof "that Universes of masses {+mass, -mass} are created in pairs", for the fermion field dirac16complex and for the commuting field dirac16complex00, and the request for this book speaks of proving that "the big bang creates universes in pairs". This chapter answers the question of its title exactly. It collects what the equations of this book PROVE about pairs of universes (the pairing theorems T1, T2, Q and T3 and the corollary C1), proves the central steps again line by line, follows them through one exact solution, and then states, just as exactly, what the equations do NOT prove: that any universe is created, in pairs or otherwise. No creation process, no rate, no probability and no amplitude follows from these equations. The author's statement that the big bang creates universes in pairs therefore remains what it was, a HYPOTHESIS, and this chapter says precisely what a proof of it would still need.

20.1 What this chapter does, and why

Why the question needs care. A sentence like "the equations prove that universes come in pairs" can mean several very different things. It can mean that for every solution of the equations there is a partner solution (a statement about the set of all solutions); it can mean that whenever one universe exists its partner exists too (a statement about what is realised); or it can mean that some process makes two universes at once out of something else (a statement about dynamics, with a rate or a probability). The first kind of statement can be proved from the field equations of this book, and it is proved. The second and the third cannot be proved from them, because these equations describe fields on a given gravitational background and contain no process that produces a universe. A student who confuses the three would believe that the theory has shown something it has not. This chapter therefore separates them carefully and gives every statement its status.

The answer in brief. Each item carries its status word (defined in Section 20.2).

1. PROVED (theorem T1, the chirality pairing): multiplying a field Ψ by the chirality matrix Γ turns every configuration of the theory with mass m and coupling λ into a configuration of the theory with $(-m, -\lambda)$, solutions into solutions, in every gravitational field, and reverses its energy-momentum tensor, its current and its charge. A T1 pair therefore has zero total energy, momentum, stress and charge at every point.

2. PROVED (theorem T2, the mirror pairing): the gamma matrix of the hidden direction together with the mirror $z \to \pi - z$ across the surface $z = \pi/2$ (an ASSUMED construction) turns a solution with $(-m, \lambda)$ into a solution with (m, λ) with EQUAL, not opposite, energy and charge.

3. PROVED (the quantum reading Q, dirac16complex only): the T1 image is the same quantum system relabelled; two independently quantised universes of masses $+m$ and $-m$ have identical one-particle spectra (the $\lambda = 0$ spectra, in flat 4+4 space or in a general field at a point with frozen coefficients) and energies that add, not cancel.

4. PROVED (theorem T3, the Kohn-Sham level, dirac16complex): every self-consistent instantaneous Kohn-Sham state with (m, λ) has a partner with $(-m, +\lambda)$ and equal energies (ASSUMED Z2 brane, transformed tip condition).

5. PROVED (corollary C1): a T1 pair taken as the only source of the author's metric is a zero source, and in Einstein gravity the author's metric then has no solution at all for $H > 0$.

6. COMPUTED in this chapter (Notebook 20b): one exact universe, a solution of the field equation and of all 64 Einstein equations together, that needs no partner; its T1 partner solves its own field equation but cannot be the source of the author's metric.

7. PROVED (the charge of a pair): a T1 partner carries the opposite charge, so a T1 pair has zero total charge; a T2 mirror copy carries the SAME charge. This does NOT solve the matter-antimatter problem: the theory as built has no baryons, no violation of baryon number, no violation of CP and no computed departure from equilibrium (Section 20.23).

8. NOT ESTABLISHED: that any universe is created; any creation process, rate, probability or amplitude; that a partner must exist.

9. HYPOTHESIS: "the big bang creates universes of masses $+M$ and $-M$ in pairs" (the author's statement).

T1, T2, Q, T3 and C1 are exact **maps between solution sets** and one consequence of them: they say which solutions exist if one solution exists. None of them is a statement that a universe comes into being.

The plan. Section 20.2 defines the words. Section 20.3 reads the request and the hypothesis word by word, and Section 20.4 separates the three questions hidden in it. Sections 20.5 and 20.6 state the pairing theorems and prove them line by line: this chapter gives the complete proofs of T1 and T2 (with the reflected-connection lemma) and the Krein-metric step of Q; the remaining steps of Q, which need the one-particle Hamiltonian of Chapter 10, are proved in Chapter 18, and T3, whose proof needs the Kohn-Sham blocks of Chapter 14, is proved in Chapter 19 (Section 20.6 outlines it). Section 20.7 proves the corollary C1, and Notebook 20a makes every step of it visible (Sections 20.8 to 20.11). Section 20.12 derives one exact universe of the theory, and Notebook 20b builds it and its partners (Sections 20.13 to 20.16). Section 20.17 shows, with the creation of an electron and a positron from light, what a proof of creation contains, and Notebook 20c computes it (Sections 20.18 to 20.21). Section 20.22 says what the conservation laws of this theory allow and forbid, Section 20.23 what pairs have to do with matter and antimatter, Section 20.24 lists what is not established, Section 20.25 what a calculation of creation would require, and Section 20.26 gives the answer to the question of the title. The chapter ends with the ledger "What we proved, what we computed, what we assumed" (Section 20.27) and with exercises and their complete answers (Section 20.28).

The three worked examples. Each is a complete Jupyter notebook; you may run them in any order. None of them needs Rust.

notebook	**what it computes**	**checks**	**figures**
20a	corollary C1 step by step: the spin connection on both patches, a configuration with its T1 partner and its T2 mirror copy, the vacuum equations of Einstein and Einstein-Lovelock gravity	37	7
20b	one exact universe of the coupled equations with the deflating history, its T1 partner, the T1 pair and its T2 mirror copy	25	4

notebook	what it computes	checks	figures
20c	allowed is not the same as happens: the thresholds of electron-positron pair creation, and what the pairing record does not establish	12	4

The records this chapter uses. Every formula and number comes from these Revision records or from the chapter's own notebooks, which reproduce the records where they overlap (each such check prints the record file and the check name).

record	what it holds
`Revision/docs/PAIR_CREATION_PROOFS.md`	the Revision document of the pairing theorems, the corollary C1 and the list of what is not established
`Revision/pairing/pairing-theory.json`	T1, T2 and Q with hypotheses, statements, proofs, and the list `not_established`
`Revision/pairing/reports/wolfram-pairing.json`	the WolframScript verification of T1, T2 and Q (101 checks, all PASS)
`Revision/pairing/reports/python-pairing.json`	the independent sympy verification (66 checks, all PASS)
`Revision/pairing/kohn_sham/t3-theory.json`	theorem T3, its hypotheses and its own list of what it does not establish
`Revision/pairing/kohn_sham/reports/`	`wolfram-t3.json` (10 checks) and `python-t3.json` (13 checks), all PASS
`Revision/field_equations_a4/a4-equations.json`	the Lovelock tensors of the author's metric, the null combination and the vacuum factor
`Revision/field_equations_a4/reports/`	`python-a4-report.json` and `wolfram-a4-report.json`: the field equations for a_4; `ks-source-conditions.json`: the Kohn-Sham states as a source
`Revision/lead_checks/reports/`	the lead's independent checks `einstein-gauss-bonnet-a4.json`, `emt-divergence-and-spin-connection.json` and `charge-conjugation-and-u1.json`
`Revision/algebra/gammas.json`	the author's eight real 16×16 gamma matrices

The counts in the table are those printed by Notebook 20a (In [2]) for the two pairing reports and those of the reports' own summaries for T3; a report is named below by its file name only, for example "`python-pairing.json`, check `gammas.Gamma`". Every check cited in this chapter has the verdict PASS.

Notation. The author's coordinates are $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates (scale factor $e^{a_4} \sin^{1/6} z$); x_4 is the time; x_5, x_6, x_7 are the three EXTRA TIMES, time-like, which DEFLATE EXPONENTIALLY (scale factor $e^{-a_4} \sin^{1/6} z$, with a_4 increasing); x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$ and $H > 0$ the author's constant. The metric is

$$ds^2 = \cot^2 z\, dx_0^2 + e^{2a_4} \sin^{1/3} z\, (dx_1^2 + dx_2^2 + dx_3^2) - dx_4^2$$
$$- e^{-2a_4} \sin^{1/3} z\, (dx_5^2 + dx_6^2 + dx_7^2),$$

with $a_4 = a_4(x_4)$ and $a_4' = da_4/dx_4$; its signature is (4,4) and $\sqrt{|g|} = \cos z$. This is the author's metric of 2026-10-14 on its background $b_4 = 1$: the metric has a second function $b_4(x_4) > 0$ that multiplies the three extra-time components (Chapter 3). The records of the pairing theorems and the notebooks of this chapter work at $b_4 = 1$; Section 20.7 shows that corollary C1 holds for every b_4. The frame metric is $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \ldots, x_7$. The gamma matrix of the direction x_a is $\gamma^{(x_a)}$, a real 16×16 matrix, with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)} = 2\eta^{ab} I_{16}$. The **deflating history** is the linear member $a_4 = AHx_4$ with $A = 1$: 3-space grows like e^{Hx_4} and the extra times shrink like e^{-Hx_4}.

20.2 The words of this chapter

- **Configuration**: a field Ψ given by its 16 components at every point, whether or not it obeys the field equation. A statement that holds for every configuration holds **off shell**; one that holds only for solutions holds **on shell**.
- **Solution**: a configuration that obeys the field equation $\gamma^\mu D_\mu \Psi = (m + \lambda S)\Psi$, where $S = \bar{\Psi}\Psi$ and $\bar{\Psi} = \Psi^\dagger C$.
- **Universe of mass** m (the meaning used by the Revision record): a solution of one of the two fields with mass parameter m and coupling λ in the author's primordial gravitational field. It is NOT the total gravitational mass or energy of that universe.
- **Parameter set** (m, λ): the mass and the coupling of the potential $U(S) = \frac{\lambda}{2} S^2$. The theory with (m, λ) and the theory with $(-m, -\lambda)$ are two DIFFERENT theories: their Lagrangians are different functions of the field.
- **Map between solution sets**: a rule that takes EVERY solution of one theory to a solution of another theory (here: multiply the field by a fixed matrix, possibly together with a change of the coordinate x_0).
- **Partner**: the image of a solution under such a map. **T1 partner**: $\Gamma\Psi$ with $(-m, -\lambda)$ at the same point. **T2 mirror copy**: $\gamma^{(x_0)}\Psi$ placed at the mirror point $\pi - z$, with $(-m, \lambda)$.
- **Pair**: a solution together with one partner. A **T1 pair** has opposite energy-momentum tensors; a **T2 pair** has equal ones.
- **Chirality**: the product of all eight gammas in the record's order, $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)} \ldots \gamma^{(x_7)}$, which equals the diagonal matrix $\mathrm{diag}(-I_8, I_8)$ (eight entries -1, then eight entries $+1$).
- **Charge matrix** $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, which defines the adjoint $\bar{\Psi} = \Psi^\dagger C$; **Krein matrix** $B = -iC\gamma^{(x_4)}$.
- **Energy-momentum tensor** $T_{\mu\nu}$: the source of gravity, 64 numbers at every point. **Energy density** $\rho = -T^{x_4}{}_{x_4}$; **pressure** along a direction μ: $p_\mu = T^\mu{}_\mu$ (no sum); p_3 for 3-space, p_t for the extra times, p_0 for the hidden direction.

- **Current** $J^\mu = -i\bar{\Psi}\gamma^\mu\Psi$; **charge density** $J^{x_4} = \Psi^\dagger B\Psi$; **charge** $Q = \int \cos z\, \Psi^\dagger B\Psi\, d^7x$, the integral over the seven directions other than x_4.
- **Patch, mirror patch, brane**: $0 < z < \pi/2$ is the **patch**; the map $z \to \pi - z$ takes it to the **mirror patch** $\pi/2 < z < \pi$; the surface $z = \pi/2$ between them is the **brane**. Gluing the two patches at the brane is the **Z2 construction**; it is ASSUMED.
- **Einstein-Lovelock equations**: $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu$, with the three Lovelock tensors $E_{(k)}$ (Chapter 12), couplings α_k, the **cosmological constant** Λ and the gravitational coupling κ. **Einstein gravity** is $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$; then $E_{(1)} = G$ is the Einstein tensor. **Einstein-Gauss-Bonnet gravity** is $\alpha_1 = 1$, $\alpha_3 = 0$.
- **Vacuum equations**: the field equations of gravity with zero source, $\sum_k \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda\delta^\mu_\nu = 0$. A **vacuum** is a solution of them.
- **Null combination** $\rho + p_0$: the energy density plus the pressure along x_0.
- **Corollary**: a statement that follows from a theorem in a few lines.
- **Process**: a solution of a dynamical equation that connects an initial state with a different final state. **Creation** of a universe: a process whose initial state has fewer universes than its final state.
- **Rate, probability, amplitude**: numbers that say how often a process happens; they come from a dynamical theory. An **amplitude** is the complex number whose squared modulus is a probability.
- **Cross section**: for a process in which a particle hits a target (for example a photon a nucleus), the effective area of the target that the particle must hit for the process to happen; the number of processes per unit time is the cross section times the number of incoming particles that cross a unit area per unit time. It is measured in units of area.
- **Bogoliubov coefficient**: in a quantum field on a gravitational field that changes in time, a number that says how much of a wave that oscillates with positive frequency at early times has turned into a wave of negative frequency at late times; its squared modulus is the mean number of particles created in that wave. It is the standard measure of particle creation by a changing background.
- **Vacuum decay**: the quantum transition of a state that has the lowest energy only among its neighbouring states (a **false vacuum**) into a state of still lower energy; its rate per unit volume is computed from the quantum theory.
- **Tunnelling**: a quantum transition through a region that the classical equations forbid (a **barrier**), with a small but nonzero probability; proposals in which a universe appears by tunnelling are of this kind (Section 20.25).
- **Wave function of the universe, path integral**: a wave function of the universe is a quantum state of the geometry together with the matter fields (Section 20.25); a path integral computes an amplitude by adding up one complex number for every history that connects the initial and the final state.
- **Allowed**: not forbidden by any conservation law. Allowed does NOT mean that the process happens, nor how often.

- **Conserved quantity**: a number whose value does not change in time for every solution.
- **Status words.** PROVED: an exact statement with a proof, verified by a named check of a Revision record or by an exact computation of this book's notebooks. COMPUTED: a floating-point result with its measured accuracy. ASSUMED: an assumption on which a result rests. HYPOTHESIS: a scientific conjecture that is not established. OPEN: a question that nobody has answered here.

20.3 The request and the hypothesis, read word by word

The request. The author's request, quoted from `Revision/README.md`: "PROVE that Universes of masses {+mass, -mass} are created in pairs for each case of the dirac16complex and the dirac16complex00 fields." The request for this book repeats it as "proving that the 'big bang' creates universes in pairs".

The hypothesis. We write it as one sentence and give it its status:

Hypothesis P (the author's). The big bang creates universes of masses $+M$ and $-M$ in pairs. Status: HYPOTHESIS.

The Revision record gives its answer to the request in its list of answers to the author of 2026-10-01 (`Revision/README.md`): the pairing theorems T1, T2, Q and T3 are proved, each under its stated hypotheses, and "that universes occur in pairs, or are created, is not proved: no creation process, rate or amplitude follows from these equations". This chapter explains why, word by word.

"A universe of mass M." In the record, M (or m) is the mass parameter of the field in its Lagrangian. A universe of mass m is a solution of the field equation with this parameter, in the author's gravitational field (Section 20.2). The sign of m alone does not say which of two physically different kinds of universe one has: theorem T2 (Section 20.6) shows that the image of a solution with $(-m, \lambda)$ under the reflection of one space-like frame direction is a solution with (m, λ) with the same energy-momentum tensor. So "$+M$" and "$-M$" are labels whose sign is tied to the orientation of a frame.

"Created." A creation is a process (Section 20.2): an equation whose solution starts with fewer universes and ends with more, or, in a quantum theory, a nonzero amplitude for the transition from a state with fewer universes to a state with more. The field equations of this book are equations for a field on a given gravitational background; they contain no term that turns "no universe" into "a universe". We return to this in Sections 20.22 and 20.25.

"In pairs." That the second universe is present together with the first one, and necessarily so. The pairing theorems say that the partner is a solution of the partner theory; they do not say that it is present (Section 20.4).

"The big bang." In cosmology the big bang is the hot, dense early phase from which the observed expansion started; in the classical solutions of Einstein's equations that describe it, the scale factor of space goes to zero at a finite time in the past, and the equations break down there (a **singularity**). What we can prove is narrower: the author's metric has no such moment wherever its history a_4 is regular. We prove it line by line, for any history a_4 that is finite with finite a_4' and a_4'' at the time considered, on the patch $0 < z < \pi/2$; the prescribed deflating history is such a history at every finite time. Whether the equations of this book, solved together, produce a big-bang moment is a different question, and it is not decided (see the end of this paragraph).

$$g_{x_1x_1} = g_{x_2x_2} = g_{x_3x_3} = e^{2a_4} \sin^{1/3} z, \qquad 0 < g_{x_1x_1} < \infty.$$

The exponential of a finite number is finite and positive, and $0 < \sin z < 1$ on the patch, so $\sin^{1/3} z$ is positive and finite.

$$g_{x_4x_4} = -1, \qquad g_{x_5x_5} = g_{x_6x_6} = g_{x_7x_7} = -e^{-2a_4} \sin^{1/3} z, \qquad -\infty < g_{x_5x_5} < 0.$$

The same rule with $-2a_4$ in the exponent.

$$g_{x_0x_0} = \cot^2 z, \qquad 0 < g_{x_0x_0} < \infty,$$

because $\cot z = \cos z / \sin z$ is finite and positive for $0 < z < \pi/2$.

$$\sqrt{|g|} = \cos z > 0.$$

This is the record's volume factor (`python-pairing.json`, check `geometry.brane_degenerate`, which also records that it and $g_{x_0x_0}$ vanish at the brane $z = \pi/2$, the edge of the patch). So at every finite time the metric is finite and non-degenerate at every point of the patch. Its curvature is finite as well: every component $R^{ab}{}_{cd}$ is a Laurent polynomial in $\cot z$ (a sum of powers $\cot^k z$ with whole, possibly negative, k) whose coefficients are polynomials in H, a_4' and a_4'' (`python-a4-report.json`, checks `riemann_entries_laurent` and `mixed_riemann_free_of_warp_and_a4`), and such an expression is finite where $\cot z$ is finite and not zero. For the prescribed deflating history $a_4 = Hx_4$ the three numbers $a_4 = Hx_4$, $a_4' = H$ and $a_4'' = 0$ are finite at every finite time x_4, so the metric and its curvature are finite at every finite time on the patch; and its 3-space scale factor $e^{Hx_4} \sin^{1/6} z$ tends to zero only as $x_4 \to -\infty$, never at a finite time. Status: PROVED for the prescribed deflating history $a_4 = AHx_4$, and for any history whose a_4, a_4' and a_4'' stay finite (derived here from the metric and the two record checks). The degenerate places of the metric are the edges of the patch, not a moment of time: the brane $z = \pi/2$ ($g_{x_0x_0} = 0$ and $\sqrt{|g|} = 0$) and the tip $z \to 0$.

What this proof does NOT show: that the equations of this book exclude a big-bang moment. It assumed that a_4, a_4' and a_4'' are finite, and it says nothing about a history in which one of them becomes infinite at a finite time. The history used in this book is prescribed, not computed: the Kohn-Sham history $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND, because the Kohn-Sham states violate the source conditions of the a_4 equations (`ks-source-conditions.json`, checks `ks_profiles_depend_on_x0` and `ks_history_is_a_prescribed_background`). And when the a_4 equations are solved with a given source, they can break down at a finite time: Chapter 12 integrates the evolution equation of Einstein-Gauss-Bonnet gravity with a given constant source and finds that the rate a_4' reaches a critical value at a finite time $x_4^\star$, where a_4'' becomes infinite and no solution with a smooth a_4' continues. The curvature contains a_4'' (for example the Einstein tensor component $G^{x_1}{}_{x_1} = -3a_4'^2 + a_4'' + 15H^2$, `a4-equations.json`, key `lovelockTensors`, entry `E1`, `x1x1`), so it becomes infinite there too: a singularity at a finite time. In Einstein gravity the record allows every history a_4 that has a continuous second derivative, each with the source that it requires (`a4-equations.json`, key `einstein`, entry `allowedA4`), so a history in which $a_4 \to -\infty$ at a finite time, that is in which the 3-space scale factor goes to zero, is not excluded either. So: the PRESCRIBED history contains no big-bang moment; whether the coupled equations of the fields and of a_4, solved together, contain one is not decided (OPEN).

20.4 Three questions: is it allowed, does it happen, how often?

The sentence "universes are created in pairs" packs three questions of very different kinds into one.

Q1, allowed? Do the conservation laws of the theory permit a transition from a state without universes (or with one) to a state with a pair? This question asks whether something FORBIDS the transition. It is answered by comparing the conserved quantities before and after.

Q2, does it happen? Is there a solution of the dynamical equations (the field equations of both members together with the gravitational field equations) that starts without the pair and ends with it? This is a question about dynamics.

Q3, how often? In a quantum theory: what is the amplitude of the transition, its probability, or the number of pairs made per unit time? Its answer is a number.

Symmetry is not realisation. A pairing theorem says: for every solution with mass $+m$ there is a solution with mass $-m$. It does not say that whenever the first is present, the second is present too. An everyday comparison makes the difference plain. The laws of mechanics do not change when everything is reflected in a mirror, so for every right hand that the laws allow, a left hand is allowed as well. That does not mean that every right hand comes with a left hand, still less that hands are created in right-left pairs. To go from "the partner is allowed" to "the partner is created together with the original" one needs an answer to Q2 or Q3.

What the Revision record answers. The pairing theorems T1, T2, Q and T3 answer none of the three questions directly: they are statements about the SETS of solutions of two theories. Q1 is answered in part in Section 20.22: the conservation laws of the theory forbid a T1 pair with nonzero member charges or momenta from appearing out of zero fields, provided that no charge or momentum flows through the brane $z = \pi/2$ (a no-flux condition that is ASSUMED, not derived). Q2 and Q3 are not answered by any equation of the record: no creation process, rate or amplitude is derived (`pairing-theory.json`, list `not_established`, items "No creation process" and "No rate and no amplitude"). Notebook 20c shows on the creation of electron-positron pairs from light what an answer to Q1, Q2 and Q3 looks like when physics does have it.

20.5 What is proved (1): theorem T1, line by line

Hypotheses (`pairing-theory.json`, theorem T1). (H1) The gravitational field is any vielbein with any spin connection, in particular the canonical one of the author's metric; it is a fixed (test) background, NOT varied, and both members of the pair live in the SAME field. (H2) Ψ has Grassmann components (dirac16complex) or commuting components (dirac16complex00); the map acts at the same point, with no change of coordinates. (H3) The potential is $U(S) = \frac{\lambda}{2}S^2$. (H4) The Lagrangian density, the field equation, the energy-momentum tensor and the current are

$$\mathcal{L}_{m,\lambda}[\Psi] = \sqrt{|g|}\,[K - mS - \tfrac{\lambda}{2}S^2], \qquad K = \tfrac{1}{2}(\bar{\Psi}\gamma^\mu D_\mu\Psi - (D_\mu\bar{\Psi})\gamma^\mu\Psi),$$

$$E_{m,\lambda}[\Psi] = \gamma^\mu D_\mu\Psi - (m + \lambda S)\Psi,$$

$$T_{\mu\nu} = \tfrac{1}{4}\big(\bar\Psi\gamma_\mu D_\nu\Psi + \bar\Psi\gamma_\nu D_\mu\Psi - (D_\mu\bar\Psi)\gamma_\nu\Psi - (D_\nu\bar\Psi)\gamma_\mu\Psi\big) - g_{\mu\nu}\,\frac{\mathcal{L}}{\sqrt{|g|}}, \qquad J^\mu = -i\bar\Psi\gamma^\mu\Psi,$$

with $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$, $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$, $\Omega_\mu = \frac{1}{2}\omega_{\mu ab}S^{ab}$, $S^{ab} = \frac{1}{4}[\gamma^{(a)},\gamma^{(b)}]$ and $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ for the diagonal vielbein with the lengths f_μ. (This $T_{\mu\nu}$ is the one of the pairing records; it is minus the tensor of the field-theory record. The pairing records write the current as $J^\mu = \bar\Psi\gamma^\mu\Psi$; this book uses the real current $J^\mu = -i\bar\Psi\gamma^\mu\Psi$ of the field-theory record and of `charge-conjugation-and-u 1.json`, whose x_4 component is $\Psi^\dagger B\Psi$ with the Hermitian matrix B. Every statement below is linear in T and in J, so neither the sign convention of T nor the factor $-i$ of J matters.)

Statement (theorem T1).

- (T1a) $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$ off shell; separately $S[\Gamma\Psi] = S[\Psi]$ and $K[\Gamma\Psi] = -K[\Psi]$.
- (T1b) $E_{-m,-\lambda}[\Gamma\Psi] = -\Gamma E_{m,\lambda}[\Psi]$: Ψ solves the (m,λ) equation exactly when $\Gamma\Psi$ solves the $(-m,-\lambda)$ equation.
- (T1c) $T_{\mu\nu}^{(-m,-\lambda)}[\Gamma\Psi] = -T_{\mu\nu}^{(m,\lambda)}[\Psi]$ and $J^\mu[\Gamma\Psi] = -J^\mu[\Psi]$ at every point, off and on shell.
- (T1d) The pair has $T + T' = 0$, $J + J' = 0$ and $Q + Q' = 0$, as classical bilinears.

Proof, step 1: the chirality anticommutes with every gamma. Take one direction b and move $\gamma^{(b)}$ from the left of Γ to its right:

$$\gamma^{(b)}\Gamma = \gamma^{(b)}\gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)} = (-1)^7\,\gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}\gamma^{(b)} = -\Gamma\gamma^{(b)}.$$

On its way $\gamma^{(b)}$ passes the eight factors of Γ: seven of them belong to other directions and each exchange gives a factor -1 (the Clifford relation $\gamma^{(a)}\gamma^{(b)} = -\gamma^{(b)}\gamma^{(a)}$ for $a \neq b$); one of them is $\gamma^{(b)}$ itself, which commutes with itself.

Step 2: the chirality commutes with C, with every S^{ab} and with every Ω_μ.

$$\Gamma C = \Gamma\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)} = (-1)^4\,\gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}\Gamma = C\Gamma.$$

Step 1 was applied four times, once for each factor of C. In the same way $\Gamma S^{ab} = S^{ab}\Gamma$ (two factors, $(-1)^2 = 1$), and therefore $\Gamma\Omega_\mu = \Omega_\mu\Gamma$, because Ω_μ is a sum of numbers times the S^{ab}.

Step 3: the covariant derivative commutes with Γ.

$$D_\mu(\Gamma\Psi) = \partial_\mu(\Gamma\Psi) + \Omega_\mu\Gamma\Psi = \Gamma\partial_\mu\Psi + \Gamma\Omega_\mu\Psi = \Gamma D_\mu\Psi.$$

Γ is a constant matrix, so it comes out of the derivative; step 2 moved it past Ω_μ.

Step 4: the adjoint of the partner. $\Gamma = \mathrm{diag}(-I_8, I_8)$ is real and symmetric, so $\Gamma^\dagger = \Gamma$ and $\Gamma^2 = I_{16}$ (`python-pairing.json`, check `gammas.Gamma`).

$$\overline{\Gamma\Psi} = (\Gamma\Psi)^\dagger C = \Psi^\dagger\Gamma^\dagger C = \Psi^\dagger\Gamma C = \Psi^\dagger C\Gamma = \bar\Psi\Gamma.$$

The rules used, in order: the definition of the adjoint; $(M\Psi)^\dagger = \Psi^\dagger M^\dagger$; $\Gamma^\dagger = \Gamma$; step 2. In the same way $D_\mu(\bar\Psi\Gamma) = \partial_\mu\bar\Psi\,\Gamma - \bar\Psi\Gamma\Omega_\mu = (D_\mu\bar\Psi)\Gamma$.

Step 5: the scalar is unchanged, the kinetic term changes sign.

$$S[\Gamma\Psi] = \bar\Psi\Gamma\,\Gamma\Psi = \bar\Psi\Psi = S[\Psi].$$

Step 4 and $\Gamma^2 = I_{16}$. For the kinetic term, every $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ is a number times one gamma, so by step 1 $\Gamma\gamma^\mu\Gamma = -\gamma^\mu\Gamma\Gamma = -\gamma^\mu$, and with steps 3 and 4

$$\overline{\Gamma\Psi}\,\gamma^\mu D_\mu(\Gamma\Psi) = \bar\Psi\,\Gamma\gamma^\mu\Gamma\,D_\mu\Psi = -\bar\Psi\gamma^\mu D_\mu\Psi, \qquad (D_\mu\overline{\Gamma\Psi})\gamma^\mu\Gamma\Psi = -(D_\mu\bar\Psi)\gamma^\mu\Psi.$$

Both terms of K change sign, so $K[\Gamma\Psi] = -K[\Psi]$.

Step 6: the Lagrangian (T1a).

$$\mathcal{L}_{m,\lambda}[\Gamma\Psi] = \sqrt{|g|}\,[\,-K - mS - \tfrac{\lambda}{2}S^2] = -\sqrt{|g|}\,[K - (-m)S - \tfrac{(-\lambda)}{2}S^2] = -\mathcal{L}_{-m,-\lambda}[\Psi].$$

The first equality inserts step 5; the second takes out the factor -1 and writes $+m = -(-m)$ and $+\lambda = -(-\lambda)$; the third is the definition of $\mathcal{L}$ with the parameters $(-m, -\lambda)$.

Step 7: the field equation (T1b).

$$E_{-m,-\lambda}[\Gamma\Psi] = \gamma^\mu D_\mu(\Gamma\Psi) - (-m - \lambda S)\Gamma\Psi = -\Gamma\gamma^\mu D_\mu\Psi + (m + \lambda S)\Gamma\Psi = -\Gamma E_{m,\lambda}[\Psi].$$

The first equality is the definition of E with the parameters $(-m, -\lambda)$ and $S[\Gamma\Psi] = S$; the second uses step 3 and then step 1 to move Γ to the left; the third takes out $-\Gamma$. Since Γ is invertible, the left side vanishes exactly when $E_{m,\lambda}[\Psi]$ does.

Step 8: the energy-momentum tensor and the current (T1c). Each of the four kinetic terms of $T_{\mu\nu}$ has one gamma between $\bar\Psi$ and Ψ and changes sign exactly as in step 5. The last term changes sign by step 6 with the parameters exchanged, $\mathcal{L}_{-m,-\lambda}[\Gamma\Psi] = -\mathcal{L}_{m,\lambda}[\Psi]$. Hence $T_{\mu\nu}^{(-m,-\lambda)}[\Gamma\Psi] = -T_{\mu\nu}^{(m,\lambda)}[\Psi]$. For the current,

$$J^\mu[\Gamma\Psi] = -i\bar\Psi\Gamma\gamma^\mu\Gamma\Psi = +i\bar\Psi\gamma^\mu\Psi = -J^\mu[\Psi],$$

by step 4 and step 1; integrating its x_4 component gives $Q[\Gamma\Psi] = -Q[\Psi]$.

Step 9: the pair and the statistics (T1d). Adding (T1c) to the quantities of Ψ gives $T + T' = 0$, $J + J' = 0$ and $Q + Q' = 0$. Γ multiplies each component by $+1$ or -1 and never reorders two Grassmann numbers, so steps 1 to 8 hold word for word for anticommuting components. Nothing in the proof used the form of the vielbein or of the connection, so it holds in every gravitational field. QED.

Records. The Wolfram pairing report has 51 checks of T1, in the author's field, in a general diagonal field and in a general field at one point, for both statistics. The sympy pairing report has 23 more, written independently. Examples, one per line:

statement	report	check
steps 1 and 2	`wolfram-pairing.json`	`Gamma_properties`
(T1a), commuting field	`wolfram-pairing.json`	`T1_Lagrangian_primordial_commuting`
(T1c), the tensor	`wolfram-pairing.json`	`T1_energy_momentum_primordial_commuting`
(T1c), the current	`wolfram-pairing.json`	`T1_current_primordial_commuting`
step 5, the scalar	`python-pairing.json`	`T1.metric.commuting.S_invariant`
(T1b)	`python-pairing.json`	`T1.metric.commuting.euler_lagrange_map`
(T1d), commuting field	`python-pairing.json`	`T1.metric.commuting.pair_total_emt_zero`
(T1d), Grassmann field	`python-pairing.json`	`T1.metric.grassmann.pair_total_emt_zero`

Notebook 20a reproduces T1 at a point of the deflating history (In [7]) and along the hidden direction (In [9]). The complete proof, with the four matrix lemmas behind steps 1 to 4 checked exactly, is in Chapter 18.

What T1 is and what it is not. It is not a symmetry of ONE theory: it changes the parameters and the sign of the action, and for $\lambda \neq 0$ it pairs (m, λ) with $(-m, -\lambda)$, not $+m$ with $-m$ at the same coupling (for $\lambda = 0$ it pairs $+m$ with $-m$ exactly). Both the mass and the coupling must change sign: the negative controls $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{-m,\lambda}[\Psi] \neq 0$ and $\mathcal{L}_{m,\lambda}[\Gamma\Psi] + \mathcal{L}_{m,\lambda}[\Psi] \neq 0$ are recorded (`python-pairing.json`, check `T1.metric.commuting.negative_controls`). The partner has $\rho' = -\rho$ and $p'_\mu = -p_\mu$, so every equation-of-state ratio $w = p/\rho$ is the same for both members.

20.6 What is proved (2): theorem T2, the quantum reading Q and theorem T3

Theorem T2 (the mirror pairing). Hypotheses (`pairing-theory.json`, theorem T2): n is a space-like frame direction (x_0, x_1, x_2 or x_3); in a general field the frame is reflected, $e' = R_n e$, where R_n is the 8×8 diagonal matrix with -1 in place n (this is the same metric, because $R_n \eta R_n = \eta$), and $\Psi' = \gamma^{(n)}\Psi$ at the same point; in the author's field the reflection of x_0 is the mirror $x_0 \to \pi/(6H) - x_0$, that is $z \to \pi - z$, across the brane, with the ASSUMED Z2 construction; both statistics; $U(S) = \frac{\lambda}{2}S^2$. Statement: $\mathcal{L}_{m,\lambda}[\gamma^{(n)}\Psi; R_n e] = +\mathcal{L}_{-m,\lambda}[\Psi; e]$; $S' = -S$; Ψ solves the $(-m, \lambda)$ equation exactly when its image solves the (m, λ) equation; in the author's field the image carries, at the mirror point, the pulled-back tensor $T' = R_0 T R_0$ and the current $J' = R_0 J$: the energy density and the charge are EQUAL, not opposite.

The proof, derived here line by line. First, the charge matrix obeys $C\gamma^{(n)}C^{-1} = -(\gamma^{(n)})^T$ (Chapter 5), that is $(\gamma^{(n)})^T C = -C\gamma^{(n)}$, so

$$\overline{\gamma^{(n)}\Psi} = \Psi^\dagger (\gamma^{(n)})^T C = -\Psi^\dagger C \gamma^{(n)} = -\bar{\Psi}\gamma^{(n)}.$$

The first equality uses that $\gamma^{(n)}$ is real, so $(\gamma^{(n)})^\dagger = (\gamma^{(n)})^T$; the second is the relation just quoted. Then

$$S[\gamma^{(n)}\Psi] = -\bar{\Psi}\gamma^{(n)}\gamma^{(n)}\Psi = -\eta_{nn}S[\Psi] = -S[\Psi],$$

because $(\gamma^{(n)})^2 = \eta_{nn} I_{16}$ and $\eta_{nn} = +1$ for a space-like n. Second, in the reflected frame the curved gammas are $\gamma'^{\mu} = \gamma^{(\mu)}/f_\mu$ for $\mu \neq n$ and $-\gamma^{(n)}/f_n$ for $\mu = n$. Moving $\gamma^{(n)}$ through them,

$$\gamma^{(n)}\gamma'^{\mu}\gamma^{(n)} = -\eta_{nn}\gamma^{\mu} \quad \text{for every } \mu,$$

because for $\mu \neq n$ one exchange gives -1 and $(\gamma^{(n)})^2 = \eta_{nn}$, and for $\mu = n$ the sign of the reflected frame gives the -1. Third, the covariant derivative of the image in the reflected frame is the image of the covariant derivative, $D'_\mu(\gamma^{(n)}\Psi) = \gamma^{(n)} D_\mu \Psi$ (the reflected-connection lemma). Its proof, line by line ($(\gamma^{(n)})^{-1} = \eta_{nn}\gamma^{(n)}$, because $(\gamma^{(n)})^2 = \eta_{nn} I_{16}$):

1. For $a \neq n$, $\gamma^{(n)}\gamma^{(a)}(\gamma^{(n)})^{-1} = -\gamma^{(a)}$ (one exchange); for $a = n$ it is $+\gamma^{(n)}$. Both cases together: $\gamma^{(n)}\gamma^{(a)}(\gamma^{(n)})^{-1} = -(R_n)^a{}_a\gamma^{(a)}$, since $(R_n)^a{}_a$ is -1 for $a = n$ and $+1$ otherwise.
2. $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$ contains two gammas, and inserting $(\gamma^{(n)})^{-1}\gamma^{(n)} = I_{16}$ between them gives $\gamma^{(n)}S^{ab}(\gamma^{(n)})^{-1} = (-1)^2(R_n)^a{}_a(R_n)^b{}_b S^{ab}$ (no sum), that is, S^{ab} for $a, b \neq n$ and $-S^{ab}$ when exactly one of a, b is n.
3. The reflected frame $e' = R_n e$ is the old frame with one leg reversed, by a constant matrix; its spin connection is therefore $\omega'_{\mu ab} = (R_n)^a{}_a(R_n)^b{}_b\,\omega_{\mu ab}$ (no sum): the components with exactly one index n change sign, the others do not.
4. So in $\Omega'_\mu = \frac{1}{2}\omega'_{\mu ab}S^{ab}$ each term carries the same sign as in line 2: $\Omega'_\mu = \gamma^{(n)}\Omega_\mu(\gamma^{(n)})^{-1}$.
5. Hence $D'_\mu(\gamma^{(n)}\Psi) = \partial_\mu(\gamma^{(n)}\Psi) + \gamma^{(n)}\Omega_\mu(\gamma^{(n)})^{-1}\gamma^{(n)}\Psi = \gamma^{(n)}(\partial_\mu\Psi + \Omega_\mu\Psi) = \gamma^{(n)}D_\mu\Psi$, because $\gamma^{(n)}$ is a constant matrix. In the same way $D'_\mu(\overline{\gamma^{(n)}\Psi}) = -(D_\mu\bar{\Psi})\gamma^{(n)}$.

Hence

$$\overline{\gamma^{(n)}\Psi}\,\gamma'^{\mu}D'_\mu(\gamma^{(n)}\Psi) = -\bar{\Psi}\,\gamma^{(n)}\gamma'^{\mu}\gamma^{(n)}\,D_\mu\Psi = \eta_{nn}\,\bar{\Psi}\gamma^{\mu}D_\mu\Psi,$$

so $K' = K$ for a space-like n, while $S' = -S$. Inserting both into the Lagrangian,

$$\mathcal{L}_{m,\lambda}[\Psi'; e'] = \sqrt{|g|}\,[K - m(-S) - \tfrac{\lambda}{2}(-S)^2] = \sqrt{|g|}\,[K - (-m)S - \tfrac{\lambda}{2}S^2] = \mathcal{L}_{-m,\lambda}[\Psi; e],$$

where the middle step uses $(-S)^2 = S^2$: the mass changes sign, the coupling does not, and the Lagrangian keeps its sign. The field equation follows in the same way. Multiplying $\gamma^{(n)}\gamma'^{\mu}\gamma^{(n)} = -\eta_{nn}\gamma^{\mu}$ on the left by $(\gamma^{(n)})^{-1} = \eta_{nn}\gamma^{(n)}$ gives $\gamma'^{\mu}\gamma^{(n)} = -\gamma^{(n)}\gamma^{\mu}$ (because $\eta_{nn}^2 = 1$), so, with $S' = -S$ and the lemma,

$$E'_{m,\lambda}[\gamma^{(n)}\Psi] = \gamma'^{\mu}\gamma^{(n)}D_\mu\Psi - (m - \lambda S)\gamma^{(n)}\Psi = -\gamma^{(n)}\big[\gamma^{\mu}D_\mu\Psi - (-m + \lambda S)\Psi\big] = -\gamma^{(n)}E_{-m,\lambda}[\Psi],$$

and since $\gamma^{(n)}$ is invertible, the image solves the (m,λ) equation exactly when Ψ solves the $(-m,\lambda)$ equation. For the current, with the adjoint $\overline{\gamma^{(n)}\Psi} = -\bar{\Psi}\gamma^{(n)}$ found first and $\gamma^{(n)}\gamma'^{\mu}\gamma^{(n)} = -\gamma^{\mu}$ for a space-like n,

$$J'^{\mu} = -i\,\overline{\gamma^{(n)}\Psi}\,\gamma'^{\mu}\gamma^{(n)}\Psi = +i\,\bar{\Psi}\,\gamma^{(n)}\gamma'^{\mu}\gamma^{(n)}\,\Psi = -i\bar{\Psi}\gamma^{\mu}\Psi = J^{\mu};$$

each kinetic term of $T_{\mu\nu}$ is unchanged in the same way (with the lemma for the derivatives), and so is $\mathcal{L}/\sqrt{|g|}$ by the Lagrangian line, so $T'_{\mu\nu} = T_{\mu\nu}$ in the reflected frame. In the author's field the mirror $z \to \pi - z$ leaves every metric component unchanged ($\sin(\pi - z) = \sin z$ and $\cot^2(\pi - z) = \cot^2 z$), and its pulled-back positive vielbein is $R_0 e$; this turns the frame statement into the mirror statement. Status: PROVED; the Z2 construction across the brane is ASSUMED, the metric is degenerate at the brane, and no junction condition there is derived. Records: the Wolfram pairing report has 28 checks of T2 and the sympy pairing report 16; among them:

statement	**report**	**check**
the mirror is an isometry	`wolfram-pairing.json`	`T2_mirror_is_isometry`
the same, independently	`python-pairing.json`	`geometry.mirror_isometry`
the connection of the mirror patch	`wolfram-pairing.json`	`connection_mirror_patch`
equal tensor and current at the mirror point	`wolfram-pairing.json`	`T2_mirror_energy_momentum_and_current_commuting`
the pulled-back tensor	`python-pairing.json`	`T2.metric.commuting.emt`
the current	`python-pairing.json`	`T2.metric.commuting.current`
$S' = -S$	`python-pairing.json`	`T2.metric.commuting.S_odd`
the field equations	`python-pairing.json`	`T2.metric.commuting.euler_lagrange_map`

Notebooks 20a (In [9] and In [11]) and 20b (In [9]) reproduce them.

The quantum reading Q (dirac16complex only). The quantised field obeys the canonical anticommutator $\{\Psi_A(x), \Psi^{\dagger}_B(y)\} = B_{AB}\,\delta^7(x-y)/\sqrt{|g|}$ on a slice $x_4 = \text{const}$ (Chapter 10). For the T1 image,

$$\{(\Gamma\Psi)_A, (\Gamma\Psi)^{\dagger}_B\} = (\Gamma B \Gamma^{\dagger})_{AB}\,\frac{\delta^7}{\sqrt{|g|}} = -B_{AB}\,\frac{\delta^7}{\sqrt{|g|}},$$

because the anticommutator is linear in each argument and

$$\Gamma B \Gamma = -i\,\Gamma C \gamma^{(x_4)} \Gamma = -i\,C\,\Gamma\gamma^{(x_4)}\Gamma = -i\,C(-\gamma^{(x_4)}) = -B,$$

by step 2 and then step 1 of Section 20.5. The statements (`pairing-theory.json`, theorem Q): (Q1) the T1 image carries the Krein metric $-B$, which is also what its own Lagrangian demands; (Q2) its generators coincide with those of Ψ: Ψ and $\Gamma\Psi$ are ONE quantum system

relabelled, and the T1 identity $T' = -T$ is an identity between operators of that one system; (Q3) an independently quantised $(-m, -\lambda)$ universe has the anticommutator $+B$, cannot be identified with $\Gamma\Psi$, and on the product of the two state spaces the generators ADD, so no cancellation follows; (Q4) the one-particle ($\lambda = 0$) spectra of $+m$ and $-m$ are IDENTICAL (similar matrices, $\Gamma h_m \Gamma = h_{-m}$), in flat 4+4 space or in a general field at a point with frozen coefficients (the coefficients of the equation taken constant, equal to their values at that point); (Q5) the T2 image keeps $+B$ ($\gamma^{(x_0)} B \gamma^{(x_0)\dagger} = +B$): the mirror universe is an ordinary, independently quantisable copy with equal energies. Records: `wolfram-pairing.json`, `Q_Krein_metric_of_images`, `Q_no_identification_of_independent_universes`, `Q_one_particle_maps` (12 Q checks); `python-pairing.json`, `Q.no_cancellation_independent_universes`, `Q.T2_image_keeps_B` (10 Q checks). Status: PROVED. The quantum reading therefore removes the one place where a zero total could have looked like "a pair that costs nothing": for two independent quantum universes the energies do not cancel.

Theorem T3 (the Kohn-Sham level, dirac16complex). At the level of the Kohn-Sham model of Chapters 14 and 15 (instantaneous mean-field states at a fixed slice $a_{4,0}$ of the deflating history, good sector, Hartree plus the exact uniform-gas exchange), the block map of Γ, with the two brane parities exchanged and the tip angle $\theta \to \pi - \theta$, maps every self-consistent Kohn-Sham state with (m, λ, θ) onto one with $(-m, +\lambda, \pi - \theta)$, with equal levels, occupations, Kohn-Sham energy, grand potential and energy-momentum profiles, and $S \to -S$ (`t3-theory.json`; `wolfram-t3.json` 10 of 10 and `python-t3.json` 13 of 13 checks PASS). The Z2 brane is ASSUMED; the tip condition is a choice and must be transformed; in its parameters the map is of the T2 type (equal energies), although its matrix is the chirality of T1. Status: PROVED under these hypotheses. The proof in outline (`t3-theory.json`, key `proof`; the check names below are those of `wolfram-t3.json`, and `python-t3.json` has the same checks with a dot after `T3`; Chapter 19 gives the proof line by line and runs the Rust solver for both members): the 2×2 block form of Γ maps each block Hamiltonian with the effective mass M_{eff} onto the block Hamiltonian of the other block with $-M_{\text{eff}}$ at the same level (check `T3_block_hamiltonian_map`); it turns the tip condition with angle θ into the one with $\pi - \theta$ and exchanges the brane parities (checks `T3_tip_condition_map`, `T3_brane_parities_exchanged`); the densities that enter the energies are even under the map and S is odd, so $M_{\text{eff}} \to -M_{\text{eff}}$ exactly when $(m, \lambda) \to (-m, +\lambda)$ (checks `T3_orbital_densities`, `T3_mean_field_map`); a one-to-one map between the orbitals with equal levels then gives equal occupations, energies and energy-momentum profiles (check `T3_energies_and_emt_profiles_equal`). T3's own list of what it does not establish begins with "No creation process, rate or amplitude" and adds: instantaneous states only (the time-dependent problem is OPEN), the brane ASSUMED, mean-field level only, and no back-reaction.

20.7 Corollary C1, line by line

Hypotheses (`Revision/docs/PAIR_CREATION_PROOFS.md`, section 7). (i) The gravitational field is the author's metric with the Einstein-Lovelock equations $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^{\mu}{}_{\nu} + \Lambda \delta^{\mu}_{\nu} = \kappa T^{\mu}{}_{\nu}$. (ii) The complete source is the sum of the classical energy-momentum tensors of the two members of a T1 pair at the same points of the same patch. (iii) Nothing else sources the metric.

Statement. The source vanishes, so a_4 must solve the vacuum equations. (a) In Einstein gravity they have no real solution for $H > 0$ and any Λ. (b) In Einstein-Lovelock gravity the linear member $a_4 = AHx_4 + a_0$ is a vacuum exactly when the vacuum factor V vanishes, where

$$V = \alpha_1 - 40\alpha_2 H^2 - 8A^2\alpha_2 H^2 + 360\alpha_3 H^4 + 144A^2\alpha_3 H^4 + 72A^4\alpha_3 H^4.$$

Proof of (a), line by line.

1. By hypothesis (ii) the source is $T^{\mu}{}_{\nu}[\Psi; m, \lambda] + T^{\mu}{}_{\nu}[\Gamma\Psi; -m, -\lambda]$.
2. By (T1c) the second term is minus the first, so the source is zero at every point.
3. With $\alpha_1 = 1$ and $\alpha_2 = \alpha_3 = 0$ the field equations become $G^{\mu}{}_{\nu} + \Lambda\delta^{\mu}_{\nu} = 0$.
4. The record gives $G^{x_4}{}_{x_4} = 3a_4'^2 + 21H^2$ and $G^{x_0}{}_{x_0} = -3a_4'^2 + 15H^2$ (`python-a4-report.json`, check `einstein_components`), so two of the equations read $3a_4'^2 + 21H^2 + \Lambda = 0$ and $-3a_4'^2 + 15H^2 + \Lambda = 0$.
5. Subtracting the second from the first removes Λ: $6a_4'^2 + 6H^2 = 0$.
6. For a real function a_4 the left side is at least $6H^2$, which is positive for $H > 0$; line 5 is false, so no real a_4 solves the equations. QED.

Adding the two equations of line 4 instead gives $36H^2 + 2\Lambda = 0$, so a solution would need $\Lambda = -18H^2$ and $a_4'^2 = -H^2$, that is $a_4' = \pm iH$: not a real rate. The other two independent equations, $15H^2 - 3a_4'^2 \pm a_4'' + \Lambda = 0$ for 3-space (+) and the extra times (−), then give $a_4'' = 0$. Records: `wolfram-a4-report.json`, check `einstein_no_vacuum_solution`; `python-a4-report.json`, check `einstein_no_vacuum`; `einstein-gauss-bonnet-a4.json`, check `no_vacuum_for_H_positive`. Notebook 20a, In [12], derives all of this with sympy from the record's components.

What the geometry requires of any source. With a source, the x_4 and x_0 equations read $\sum_k \alpha_k E_{(k)}{}^{x_4}{}_{x_4} + \Lambda = -\kappa\rho$ and $\sum_k \alpha_k E_{(k)}{}^{x_0}{}_{x_0} + \Lambda = \kappa p_0$, because $T^{x_4}{}_{x_4} = -\rho$ and $T^{x_0}{}_{x_0} = p_0$. Subtracting the first from the second:

$$\sum_k \alpha_k \left(E_{(k)}{}^{x_0}{}_{x_0} - E_{(k)}{}^{x_4}{}_{x_4}\right) = \kappa p_0 + \kappa\rho.$$

Λ cancels in the difference. So every source must have the null combination

$$\kappa(\rho + p_0) = -\sum_k \alpha_k \left(E_{(k)}{}^{x_4}{}_{x_4} - E_{(k)}{}^{x_0}{}_{x_0}\right).$$

In Einstein gravity, with the components of line 4,

$$\kappa(\rho + p_0) = -\left(3a_4'^2 + 21H^2 + 3a_4'^2 - 15H^2\right) = -6(a_4'^2 + H^2) \le -6H^2 < 0.$$

The first equality inserts the components; the second collects the terms; the inequality holds because $a_4'^2 \ge 0$ (`python-a4-report.json` and `wolfram-a4-report.json`, check `einstein_null_energy_x0`). A T1 pair supplies $\rho + p_0 = 0$, which is never $\le -6H^2$: this is C1(a) once more, now as a statement about the source.

Proof of (b), line by line. For the linear member $a_4' = AH$ the record writes the right-hand side as $-6(A^2 + 1)H^2 V$ with the vacuum factor V above (`wolfram-a4-report.json` and

`python-a4-report.json`, check `linear_member_vacuum_factor`). A vacuum has $\rho = p_0 = 0$, and $6(A^2+1)H^2 > 0$, so it needs $V = 0$. In Einstein-Gauss-Bonnet gravity ($\alpha_1 = 1$, $\alpha_3 = 0$):

$$V = 1 - 40\alpha_2 H^2 - 8A^2\alpha_2 H^2 = 1 - 8\alpha_2 H^2(A^2+5).$$

The terms with α_3 vanish, and $40 = 8 \cdot 5$ lets us take out $8\alpha_2 H^2$. Setting $V = 0$ and adding $8\alpha_2 H^2(A^2+5)$ to both sides:

$$8\alpha_2 H^2(A^2+5) = 1.$$

Dividing by $8\alpha_2 H^2$ (not zero) and subtracting 5:

$$A^2 = \frac{1}{8\alpha_2 H^2} - 5 = \frac{1 - 40\alpha_2 H^2}{8\alpha_2 H^2}.$$

The last step writes 5 as $40\alpha_2 H^2/(8\alpha_2 H^2)$. The right side is not negative exactly when $0 < \alpha_2 H^2 \leq 1/40$; at $1/40$ the vacuum has $A = 0$, so a_4 is constant and neither 3-space nor the extra times change; the author's deflating history needs $A > 0$. For the author's history $A = 1$: $8\alpha_2 H^2 \cdot 6 = 1$, so

$$\alpha_2 H^2 = \tfrac{1}{48}.$$

The cosmological constant follows from the x_4 equation with $\rho = 0$: $\Lambda = -\sum_k \alpha_k {E_{(k)}}^{x_4}{}_{x_4}$. We need the two components at $a_4' = AH$ with $A = 1$. The Einstein part is ${E_{(1)}}^{x_4}{}_{x_4} = {G}^{x_4}{}_{x_4} = 3a_4'^2 + 21H^2 = 3H^2 + 21H^2 = 24H^2$ (line 4 above with $a_4' = H$). For the Gauss-Bonnet part we use the x_4 equation once more: it says $\kappa\rho = -\sum_k \alpha_k {E_{(k)}}^{x_4}{}_{x_4} - \Lambda$, so the part of $\kappa\rho$ that multiplies α_2 is $-{E_{(2)}}^{x_4}{}_{x_4}$, and the record gives that part as $(36A^4 + 120A^2 + 420)H^4$ (`einstein-gauss-bonnet-a4.json`, check `gauss_bonnet_rho_alpha2`, which compares the record with the classical Gauss-Bonnet formula). Hence ${E_{(2)}}^{x_4}{}_{x_4} = -(36 + 120 + 420)H^4 = -576H^4$ at $A = 1$. Therefore

$$\Lambda = -\Big(24H^2 + \frac{1}{48H^2}(-576H^4)\Big) = -24H^2 + 12H^2 = -12H^2.$$

The first equality inserts $\alpha_2 = 1/(48H^2)$; the second uses $576/48 = 12$. Notebook 20a (In [16]) checks exactly that with these values all four independent vacuum equations hold for $a_4 = Hx_4$ (`python-a4-report.json` and `wolfram-a4-report.json`, check `einstein_gauss_bonnet_vacuum_linear`). With the third coupling, $V = 0$ is linear in $x = \alpha_2 H^2$ and $y = \alpha_3 H^4$ for a fixed slope A; collecting the terms with y on one side,

$$y\,(72A^4 + 144A^2 + 360) = (8A^2 + 40)\,x - 1, \qquad y = \frac{(8A^2+40)x - 1}{72A^4 + 144A^2 + 360},$$

a straight line in the plane of the two couplings; Einstein gravity ($x = y = 0$, $V = 1$) lies on none of these lines.

What C1 says, and what it does not say. PROVED: a T1 pair as the only source of the author's metric is a zero source; in Einstein gravity the metric then has no solution for $H > 0$, whatever

a_4 and Λ. PROVED: in Einstein-Gauss-Bonnet gravity with $\alpha_2 H^2 = 1/48$ and $\Lambda = -12H^2$ the author's deflating history IS a vacuum, so C1(a) depends on Einstein gravity. C1 does not apply to T2 pairs: the mirror copy carries the pulled-back, EQUAL tensor, so the sources add. C1 does not apply to two independently quantised universes (statement Q3). And C1 is not a creation statement: it compares two sources in ONE given geometry and derives no process, rate or amplitude. In Einstein gravity it says the opposite of "a pair makes the universe": a T1 pair alone cannot be the source of the author's metric at all.

With the second function b_4. The records state C1 on the background $b_4 = 1$; C1(a) holds for every $b_4(x_4)$. A zero source has $T^{x_4}{}_{x_0} = 0$, and in Einstein gravity the mixed component of the metric with b_4 is $G^{x_4}{}_{x_0} = \frac{3}{2} H \cot z \, b_4'/b_4$, so the mixed equation forces $b_4' = 0$ (`wolfram-a4-report.json`, check `b4_einstein_offdiagonal_forces_b4_constant`). A constant b_4 is the rescaling $x_{5,6,7} \to \sqrt{b_4}\, x_{5,6,7}$ of the extra times, which changes none of the components used in lines 3 to 6, so the proof of (a) applies unchanged. For the Einstein-Lovelock weights of the author's notebook, $w_1 = 1$, $w_2 = 1/32$, $w_3 = 1/1152$ and the author's $\Lambda = 10$ (in units $H = 1$: $\alpha_k = w_k$, and $\Lambda = -10$ in the convention of this chapter), the record proves that every vacuum of the metric with b_4 has a constant b_4 and $a_4 = \pm H x_4 + \text{const}$ (`Revision/metric_b4/results/vacuum-b4.json`; `wolfram-a4-report.json`, check `b4_author_weights_vacuum`). This is the case (b) of C1: with these weights $V = 1 - 48/32 + 576/1152 = 0$ at $A = 1$, so the author's deflating history is a vacuum of the author's own weights, and a T1 pair as the only source is compatible with it.

20.8 Example: corollary C1 made visible

Notebook 20a turns every line of Sections 20.5 and 20.7 into a computation. It reads the author's gamma matrices, builds C, Γ and the generators S^{ab}, and computes with sympy the canonical spin connection of the author's metric on the patch and on the mirror patch; it reproduces the record's twelve components and $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$, and finds $-3H\gamma^{(x_0)}$ on the mirror patch (a computation of this book, not a record). It then takes an arbitrary configuration of dirac16complex00 at one point of the deflating history, evaluates its 64-component energy-momentum tensor, its current and its field equation, and does the same for its T1 partner and its T2 mirror copy: the T1 pair cancels component by component, the T2 pair does not. Finally it derives the vacuum equations of Einstein and Einstein-Lovelock gravity from the record's Lovelock tensors and draws why Einstein gravity has no vacuum and which couplings do. It prints 37 PASS lines, 25 of which reproduce a named check of a Revision report, and draws seven figures. It needs numpy, sympy and matplotlib, no Rust, and runs in about 30 seconds (its recorded check run on the build computer took 16.1 seconds; the notebook's provenance file, which lies next to it, records every run).

20.9 How to run Notebook 20a

Step 1. What this notebook does and what it needs.

Notebook 20a (Corollary C1: a T1 pair as the only source of the author's metric) is the file `Revision/textbook/notebooks/20a_c1_zero_source.ipynb` of the repository Dirac_claude. It reads the author's gamma matrices and builds the canonical spin connection of the author's metric on the patch and on the mirror patch of the hidden direction; it evaluates the energy-momentum tensor, the current and the field equation of a configuration of dirac16complex00 and of its two partners, the T1 partner Gamma Psi with mass and coupling reversed and the T2 mirror copy across the

Z2 brane, and shows that the T1 pair has zero total source at every point while the T2 pair has twice the source of one member; it then derives with sympy, from the Revision record of the field equations for a4, that the zero source has no solution in Einstein gravity for H greater than 0 and any cosmological constant, and which Einstein-Lovelock couplings admit the linear member a4 = A H x4 as a vacuum. It reproduces the Revision records where they overlap and draws seven teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 20a_c1_zero_source.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 20a_c1_zero_source.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 20a_c1_zero_source.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/20a.captions.json`
- `Revision/textbook/figures/20a_1_t1_pair_tensor.png`
- `Revision/textbook/figures/20a_2_pair_profiles.png`
- `Revision/textbook/figures/20a_3_t2_mirror_tensor.png`
- `Revision/textbook/figures/20a_4_einstein_vacuum_parabolas.png`
- `Revision/textbook/figures/20a_5_null_combination.png`
- `Revision/textbook/figures/20a_6_gauss_bonnet_vacuum.png`
- `Revision/textbook/figures/20a_7_third_order_vacua.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 37 CHECKS PASSED (notebook 20a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/20a_c1_zero_source.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 20a_c1_zero_source.ipynb
python -m nbconvert --execute --inplace 20a_c1_zero_source.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads nine files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone (a notebook copied alone to another folder cannot find them).

20.10 Notebook 20a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/20a_c1_zero_source.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The pairing theorem T1 says that the field $\Gamma\Psi$ with the mass $-m$ and the
coupling $-\lambda$ carries exactly MINUS the energy-momentum tensor and MINUS the
current of the field $\Psi$ with $(m, \lambda)$, at every point of every
gravitational field. The Revision record draws one consequence for gravity, the
corollary **C1**: if such a pair is the ONLY source of the author's metric, the
source is zero, and in Einstein gravity the author's metric then has no solution
at all for $H > 0$. This notebook makes every step of C1 visible:

- it rebuilds the canonical spin connection of the author's metric (on the patch
  $0 < z < \pi/2$ of the hidden direction and on the mirror patch
  $\pi/2 < z < \pi$) and reproduces the record's twelve components and the term
  $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$;
- it evaluates, for a configuration of the commuting field dirac16complex00, the
  energy-momentum tensor (all 64 components), the current and the field equation,
  and the same for its T1 partner $\Gamma\Psi$: the totals vanish at every point
  (two figures);
- it does the same for the T2 mirror copy across the Z2 brane $z = \pi/2$: there
  the tensor is EQUAL (pulled back), so the T2 pair does NOT cancel (two figures);
- it derives with sympy, from the record's components of the Einstein tensor,
  that the vacuum equations (zero source) have no real solution, and draws why
  (two figures);
- it shows which Einstein-Lovelock couplings admit the deflating linear member
  $a_4 = AHx_4$ as a vacuum, so that a T1 pair could live in it (two figures);
- it ends with a precise list of what C1 says and what it does not say.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 20a (Corollary C1: a T1 pair as the only source of the author's metric) is the
file `Revision/textbook/notebooks/20a_c1_zero_source.ipynb` of the repository
Dirac_claude. It reads the author's gamma matrices and builds the canonical spin
connection of the author's metric on the patch and on the mirror patch of the hidden
direction; it evaluates the energy-momentum tensor, the current and the field equation of
a configuration of dirac16complex00 and of its two partners, the T1 partner Gamma Psi
with mass and coupling reversed and the T2 mirror copy across the Z2 brane, and shows
that the T1 pair has zero total source at every point while the T2 pair has twice the
```

source of one member; it then derives with sympy, from the Revision record of the field equations for a4, that the zero source has no solution in Einstein gravity for H greater than 0 and any cosmological constant, and which Einstein-Lovelock couplings admit the linear member a4 = A H x4 as a vacuum. It reproduces the Revision records where they overlap and draws seven teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 20a_c1_zero_source.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 20a_c1_zero_source.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the

name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 20a_c1_zero_source.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/20a.captions.json‘
- ‘Revision/textbook/figures/20a_1_t1_pair_tensor.png‘
- ‘Revision/textbook/figures/20a_2_pair_profiles.png‘
- ‘Revision/textbook/figures/20a_3_t2_mirror_tensor.png‘
- ‘Revision/textbook/figures/20a_4_einstein_vacuum_parabolas.png‘
- ‘Revision/textbook/figures/20a_5_null_combination.png‘
- ‘Revision/textbook/figures/20a_6_gauss_bonnet_vacuum.png‘
- ‘Revision/textbook/figures/20a_7_third_order_vacua.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 37 CHECKS PASSED (notebook 20a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/20a_c1_zero_source.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use

  ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 20a_c1_zero_source.ipynb
python -m nbconvert --execute --inplace 20a_c1_zero_source.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads nine files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone (a notebook copied alone to another folder cannot find them).

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 20a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 20a (Corollary C1: a T1 pair as the only source of the author's metric) is the
# file Revision/textbook/notebooks/20a_c1_zero_source.ipynb of the repository
# Dirac_claude. It reads the author's gamma matrices and builds the canonical spin
# connection of the author's metric on the patch and on the mirror patch of the hidden
# direction; it evaluates the energy-momentum tensor, the current and the field equation
# of a configuration of dirac16complex00 and of its two partners, the T1 partner Gamma
# Psi with mass and coupling reversed and the T2 mirror copy across the Z2 brane, and
# shows that the T1 pair has zero total source at every point while the T2 pair has twice
# the source of one member; it then derives with sympy, from the Revision record of the
# field equations for a4, that the zero source has no solution in Einstein gravity for H
# greater than 0 and any cosmological constant, and which Einstein-Lovelock couplings
# admit the linear member a4 = A H x4 as a vacuum. It reproduces the Revision records
# where they overlap and draws seven teaching figures. It needs a computer with Windows
```

```
# 11, macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
```

```
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
```

```
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 20a_c1_zero_source.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 20a_c1_zero_source.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 20a_c1_zero_source.ipynb (in the same folder, without running the cells again) to read
# the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/20a.captions.json
# - Revision/textbook/figures/20a_1_t1_pair_tensor.png
# - Revision/textbook/figures/20a_2_pair_profiles.png
# - Revision/textbook/figures/20a_3_t2_mirror_tensor.png
# - Revision/textbook/figures/20a_4_einstein_vacuum_parabolas.png
# - Revision/textbook/figures/20a_5_null_combination.png
# - Revision/textbook/figures/20a_6_gauss_bonnet_vacuum.png
```

```
# - Revision/textbook/figures/20a_7_third_order_vacua.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 37 CHECKS PASSED (notebook 20a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/20a_c1_zero_source.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 20a_c1_zero_source.ipynb
#     python -m nbconvert --execute --inplace 20a_c1_zero_source.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
```

```
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file below the folder Revision: the notebook reads nine
#   files of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone
#   (a notebook copied alone to another folder cannot find them).
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "20a"  # this notebook: chapter 20, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
```

```
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
```

```
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 20a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Coordinates** $x_0, \dots, x_7$ (the author's names): $x_1, x_2, x_3$ ordinary
  3-space (inflating), $x_4$ the time, $x_5, x_6, x_7$ the three EXTRA TIMES (they
  deflate exponentially: their scale factor is $e^{-a_4}\sin^{1/6}z$ with $a_4$
  increasing), $x_0$ the hidden space direction with $z = 6Hx_0$. In the code the
  positions 0 to 7 of a list stand for $x_0$ to $x_7$.
- **Patch and mirror patch**: the author's metric is used on $0 < z < \pi/2$ (the
  patch). The map $z \to \pi - z$ carries it onto $\pi/2 < z < \pi$ (the mirror
  patch). The surface $z = \pi/2$ between them is called the **brane**; extending
  the field across it (the Z2 construction) is an ASSUMPTION of the record.
- **Gamma matrices** $\gamma^{(x_0)}, \dots, \gamma^{(x_7)}$: the author's real
  $16 \times 16$ matrices with $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}
  = 2\eta^{ab}$, $\eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$.
- **$C$** $= \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$; the adjoint
  is $\bar\Psi = \Psi^\dagger C$. **Chirality** $\Gamma = \gamma^{(x_0)}
  \gamma^{(x_1)}\cdots\gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$.
- **Configuration**: a field $\Psi$ given by its 16 complex values and their 8
  first derivatives at a point (its first **jet**). A configuration need not solve
  the field equation; it is **on shell** when it does, **off shell** otherwise.
- **Pair** (in this notebook): a configuration together with a partner.
  **T1 partner**: $\Gamma\Psi$ with $(-m, -\lambda)$, at the same point.
  **T2 mirror copy**: $\gamma^{(x_0)}\Psi$ with $(-m, \lambda)$, placed at the
  mirror point $\pi - z$.
- **Energy-momentum tensor** $T_{\mu\nu}$: the source of gravity, 64 numbers at
```

  each point. **Energy density** ρ, **current** $J^\mu = -i\bar\Psi\gamma^\mu
  \Psi$, **charge density** $J^{x_4} = \Psi^\dagger B\Psi$.
- **Einstein tensor** $G^\mu{}_\nu$; **Lovelock tensors** $E_{(k)}{}^\mu{}_\nu$,
  $k = 1, 2, 3$ ($E_{(1)} = G$); couplings $\alpha_1, \alpha_2, \alpha_3$;
  cosmological constant Λ; gravitational coupling κ.
- **Vacuum equations**: the field equations of gravity with zero source,
  $\sum_k\alpha_kE_{(k)}{}^\mu{}_\nu + \Lambda\delta^\mu_\nu = 0$.
- **Linear member**: $a_4 = AHx_4 + a_0$; for $A > 0$ the extra times deflate as
  e^{-AHx_4} while 3-space inflates as e^{AHx_4}. The author's history is
  $A = 1$.
- **Null combination** $\rho + p_0$: the energy density plus the pressure along
  x_0.
- **Corollary**: a statement that follows from a theorem in a few lines.

Text cell:

```
## 4. The physical and mathematical situation

Both fields of the theory have the Lagrangian density
$\mathcal{L}_{m,\lambda} = \sqrt{|g|}\,[K - mS - \tfrac\lambda2S^2]$ with
$K = \tfrac12(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi)$ and
$S = \bar\Psi\Psi$. The pairing records use the energy-momentum tensor

$$T_{\mu\nu} = \tfrac14\big(\bar\Psi\gamma_\mu D_\nu\Psi + \bar\Psi\gamma_\nu D_\mu
\Psi - (D_\mu\bar\Psi)\gamma_\nu\Psi - (D_\nu\bar\Psi)\gamma_\mu\Psi\big)
- g_{\mu\nu}\,\mathcal{L}/\sqrt{|g|},$$

which is minus the tensor of the field-theory record; every statement below is
linear in $T$, so its sign convention does not matter. With it the energy
density is $\rho = -T_{x_4x_4}$.

**Theorem T1, part (T1c)** (PROVED in the Revision record):
$T^{(-m,-\lambda)}_{\mu\nu}[\Gamma\Psi] = -T^{(m,\lambda)}_{\mu\nu}[\Psi]$ and
$J^\mu[\Gamma\Psi] = -J^\mu[\Psi]$ at every point, off and on shell, in every
gravitational field, for both fields.

**Corollary C1** (PROVED in the record). *Hypotheses:* (i) the gravitational field
is the author's metric with the Einstein-Lovelock equations
$\sum_{k=1}^3\alpha_kE_{(k)}{}^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa
T^\mu{}_\nu$; (ii) the complete source is the sum of the classical tensors of the
two members of a T1 pair at the same points of the same patch; (iii) nothing else
sources the metric. *Statement:* the source vanishes, so $a_4$ must solve the
vacuum equations; (a) in Einstein gravity ($\alpha_1 = 1$, $\alpha_2 = \alpha_3 =
0$) these have no real solution for $H > 0$ and any $\Lambda$; (b) in
Einstein-Lovelock gravity the linear member needs $V = 0$ (the vacuum factor of
section 12).

**Proof of (a), line by line.**

1. By hypothesis (ii) the source is $T^\mu{}_\nu[\Psi; m, \lambda] +
   T^\mu{}_\nu[\Gamma\Psi; -m, -\lambda]$.
2. By (T1c) the second term is minus the first, so the source is $0$.
3. The field equations become $G^\mu{}_\nu + \Lambda\delta^\mu_\nu = 0$.
```

```
4. The record gives $G^{x_4}{}_{x_4} = 3a_4'^2 + 21H^2$ and
   $G^{x_0}{}_{x_0} = -3a_4'^2 + 15H^2$, so two of the equations read
   $3a_4'^2 + 21H^2 + \Lambda = 0$ and $-3a_4'^2 + 15H^2 + \Lambda = 0$.
5. Subtracting the second from the first removes $\Lambda$:
   $6a_4'^2 + 6H^2 = 0$.
6. For a real function $a_4$ the left side is at least $6H^2$, which is positive
   for $H > 0$. Line 5 is false, so no real $a_4$ solves the equations.

Sections 5 to 10 below check lines 1 and 2 numerically, point by point; sections
11 to 14 check lines 3 to 6 and part (b) with sympy.

**Status words.** PROVED: an exact statement with a proof (the record's, or this
notebook's exact computation). COMPUTED: a floating-point result with its error.
ASSUMED: a hypothesis. The Z2 brane and the mirror patch are ASSUMED.
```

Text cell:

```
## 5. The Revision records this notebook reproduces

The next cell defines three helpers: ‘read_json‘ reads a JSON file of the
repository; ‘record_verdict‘ finds a check by its name in a Revision report and
returns its verdict; ‘reproduces‘ is a check that passes only when this notebook's
own result holds AND the named record check has the verdict PASS (it prints the
two lines PASS and reproduces together). Then it reads the sentence of the
record's document that states C1, and the counts of the two pairing reports.
```

In [2]:

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory

GAMMAS = "Revision/algebra/gammas.json"  # the author's gamma matrices
PROOFS = "Revision/docs/PAIR_CREATION_PROOFS.md"  # the pairing document
PAIR_PY = "Revision/pairing/reports/python-pairing.json"  # sympy pairing record
PAIR_WL = "Revision/pairing/reports/wolfram-pairing.json"  # Wolfram pairing record
A4_RECORD = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
A4_PY = "Revision/field_equations_a4/reports/python-a4-report.json"
A4_WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
LEAD_EGB = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"
LEAD_EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict ("PASS") of the check called name in a report; None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
```

```
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name,
              record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # both lines with one print call


proofs = repository_file(PROOFS).read_text(encoding="utf-8")
C1_WORDS = ("a T1 pair taken as the complete classical source of the author's "
            "metric is a zero source, and the Einstein equations then have no "
            "solution for")
say(f"{PROOFS} says: \"{C1_WORDS} H > 0.\"")
check(C1_WORDS in proofs, "the record states the corollary C1")
wolfram_counts = read_json(PAIR_WL)["summary"]  # passed, failed, total
sympy_counts = read_json(PAIR_PY)["counts"]  # pass, fail, pending
passed, total = wolfram_counts["passed"], wolfram_counts["total"]
say(f"pairing reports: Wolfram {passed} of {total} PASS, sympy "
    f"{sympy_counts["pass"]} PASS and {sympy_counts["fail"]} FAIL")
check(wolfram_counts["passed"] == wolfram_counts["total"] == 101
      and sympy_counts["pass"] == 66 and sympy_counts["fail"] == 0,
      "both pairing reports pass every check (101 and 66)")
```

Out [2]:

```
Revision/docs/PAIR_CREATION_PROOFS.md says: "a T1 pair taken as the complete classical
    source of the author's metric is a zero source, and the Einstein equations then have
    no solution for H > 0."
PASS the record states the corollary C1
pairing reports: Wolfram 101 of 101 PASS, sympy 66 PASS and 0 FAIL
PASS both pairing reports pass every check (101 and 66)
```

Text cell:

```
## 6. The author's gamma matrices, $C$ and the chirality $\Gamma$

The next cell reads the eight gamma matrices from `gammas.json` (whole numbers,
which `sp.Rational` turns into exact numbers), checks the 64 Clifford relations,
and builds $C$, the chirality $\Gamma$ and the generators
$S^{ab} = \tfrac14[\gamma^{(a)}, \gamma^{(b)}]$. It checks the three facts T1
needs (Lemma 1 of the record): $\Gamma = \mathrm{diag}(-I_8, I_8)$, $\Gamma$
anticommutes with every gamma, and $\Gamma$ commutes with $C$. Finally it makes
floating-point copies (numpy arrays) for the numerical part.
```

In [3]:

```
import itertools  # loops over all combinations of indices

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the frame metric eta, in the order x0..x7
I16 = sp.eye(16)  # the 16 by 16 unit matrix
fixture = read_json(GAMMAS)
gamma = [sp.Matrix([[sp.Rational(x) for x in row] for row in matrix])
         for matrix in fixture["gamma"]]  # gamma[a] = gamma^(x_a), exact
clifford = all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
               == (2 * ETA[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g^(x0) g^(x1) g^(x2) g^(x3)
reproduces(clifford and C == C.T and C * C == I16
           and all((C * g).T == -(C * g) for g in gamma),
           "Clifford relation; C real symmetric, C^2 = 1, C gamma antisymmetric",
           PAIR_PY, "gammas.C")
chirality = gamma[0]  # the product g^(x0) g^(x1) ... g^(x7), built factor by factor
for a in range(1, 8):
    chirality = chirality * gamma[a]
reproduces(chirality == sp.diag(*([-1] * 8 + [1] * 8)),
           "Gamma = diag(-I8, I8)", PAIR_PY, "gammas.Gamma")
reproduces(all(chirality * g == -g * chirality for g in gamma)
           and chirality * C == C * chirality,
           "Gamma anticommutes with every gamma and commutes with C",
           PAIR_WL, "Gamma_properties")
S_exact = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
           for a in range(8)]  # the generators S^ab
to_numpy = lambda matrix: np.array(matrix.tolist(), dtype=float)  # noqa: E731
gamma_num = [to_numpy(g) for g in gamma]  # floating-point copies
C_num, Gamma_num = to_numpy(C), to_numpy(chirality)
S_num = [[to_numpy(S_exact[a][b]) for b in range(8)] for a in range(8)]
B_num = -1j * C_num @ gamma_num[4]  # B = -i C gamma^(x4): J^x4 = Psi^dagger B Psi
```

Out [3]:

```
PASS Clifford relation; C real symmetric, C^2 = 1, C gamma antisymmetric
     reproduces Revision/pairing/reports/python-pairing.json, check gammas.C
PASS Gamma = diag(-I8, I8)
     reproduces Revision/pairing/reports/python-pairing.json, check gammas.Gamma
PASS Gamma anticommutes with every gamma and commutes with C
     reproduces Revision/pairing/reports/wolfram-pairing.json, check Gamma_properties
```

Text cell:

```
## 7. The author's metric and its spin connection, on both patches

The vielbein of the author's diagonal metric (on the background $b_4 = 1$ of the
metric of 2026-10-14) is diagonal, $e^a{}_\mu = E_a\delta^a_\mu$, with the lengths
$E = (s_0\cot z, e^{a_4}s, e^{a_4}s, e^{a_4}s, 1, e^{-a_4}s, e^{-a_4}s, e^{-a_4}s)$
```

```
in the order $x_0, \dots, x_7$, $s = \sin^{1/6}z$. On the patch
$\cot z > 0$ and $s_0 = +1$; on the mirror patch $\cot z < 0$, and its own
positive vielbein has $s_0 = -1$ (the record's construction). The canonical spin
connection is $\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu e_b{}^\nu +
\Gamma^\nu{}_{\mu\lambda}e_b{}^\lambda)$ with the inverse vielbein $e_b{}^\nu =
\delta^\nu_b/E_b$. For $a = b$ the two terms cancel; for $a \neq b$ only the second
survives, and lowering $a$ with $\eta$:

$$\omega_{\mu\,ab} = \eta_{aa}\,\frac{E_a}{E_b}\,\Gamma^a{}_{\mu b}
\qquad (a \neq b).$$

The Christoffel symbols of a diagonal metric are
$\Gamma^a{}_{bc} = (\delta_{ac}\partial_bg_{aa} + \delta_{ab}\partial_cg_{aa} -
\delta_{bc}\partial_ag_{bb})/(2g_{aa})$ (no sum), and $\partial/\partial x_0 =
6H\,\partial/\partial z$ because $z = 6Hx_0$. The next cell computes all
$\omega_{\mu ab}$ with sympy for an arbitrary function $a_4(x_4)$, on both patches;
at a point $a_4$ and $a_4'$ become the numbers `a4_value` and `slope`.
```

In [4]:

```
H = sp.symbols("H", positive=True)  # the author's constant H > 0
z = sp.symbols("z", real=True)  # the hidden coordinate z = 6 H x0
x4 = sp.symbols("x4", real=True)  # the time
a4 = sp.Function("a4")(x4)  # the metric function a4(x4), arbitrary
a4_value, slope = sp.symbols("a4_value slope", real=True)  # a4 and a4' at a point


def lengths(s0):
    """The lengths E_a of the diagonal vielbein; s0 = +1 on the patch, -1 on the
    mirror patch (where cot z < 0, so -cot z is the positive length)."""
    s = sp.sin(z) ** sp.Rational(1, 6)  # the warp sin(z)^(1/6) of a length
    return ([s0 * sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
            + [sp.exp(-a4) * s] * 3)  # x0, x1, ..., x7


def d(expr, mu):
    """The partial derivative along the coordinate at position mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # d/dx0 = 6 H d/dz
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7


def spin_connection(s0):
    """The lengths E_a and omega[mu][a][b] = omega_mu ab at a point (a4 and a4'
    replaced by the symbols a4_value and slope)."""
    E = lengths(s0)
    g = [ETA[a] * E[a] ** 2 for a in range(8)]  # the metric g_aa
    christoffel = {}
    for a, b, c in itertools.product(range(8), repeat=3):
        value = 0  # the formula of the markdown cell, term by term
        if a == c:
```

```
            value += d(g[a], b)
        if a == b:
            value += d(g[a], c)
        if b == c:
            value -= d(g[b], a)
        christoffel[a, b, c] = value / (2 * g[a])
    omega = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
    for mu, a, b in itertools.product(range(8), repeat=3):
        if a != b:
            value = ETA[a] * E[a] * christoffel[a, mu, b] / E[b]
            value = value.subs(sp.Derivative(a4, x4), slope).subs(a4, a4_value)
            omega[mu][a][b] = sp.simplify(value)
    return [length.subs(a4, a4_value) for length in E], omega


E_patch, omega_patch = spin_connection(1)
E_mirror, omega_mirror = spin_connection(-1)
for label, omega in (("patch", omega_patch), ("mirror patch", omega_mirror)):
    count = sum(1 for mu in range(8) for a in range(8) for b in range(a + 1, 8)
                if omega[mu][a][b] != 0)
    say(f"{label}: {count} nonzero components omega_mu ab with a < b")
```

Out [4]:

```
patch: 12 nonzero components omega_mu ab with a < b
mirror patch: 12 nonzero components omega_mu ab with a < b
```

Text cell:

```
The record lists the twelve nonzero components on the patch ($a < b$; with
$u = e^{a_4}\sin^{1/6}z$ and $v = e^{-a_4}\sin^{1/6}z$): $\omega_{x_i,\,x_ix_4} =
a_4'u$ and $\omega_{x_i,\,x_0x_i} = -Hu$ for the three inflating directions
$i = 1, 2, 3$; $\omega_{x_t,\,x_4x_t} = -a_4'v$ and $\omega_{x_t,\,x_0x_t} = Hv$
for the three DEFLATING extra times $t = 5, 6, 7$. The next cell compares, builds
$\Omega_\mu = \tfrac12\sum_{a,b}\omega_{\mu ab}S^{ab}$, and checks the record's
$\sum_\mu\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ on the patch ($\gamma^\mu =
\gamma^{(\mu)}/E_\mu$). On the mirror patch the $x_0$ components change sign and
the sum is $-3H\gamma^{(x_0)}$ (computed here). It also checks that $z \to \pi - z$
leaves every metric component unchanged (the mirror is an isometry).
```

In [5]:

```
s = sp.sin(z) ** sp.Rational(1, 6)
u, v = sp.exp(a4_value) * s, sp.exp(-a4_value) * s
expected = {}  # (mu, a, b) -> the record's component, on the patch
for i in (1, 2, 3):  # the inflating 3-space directions
    expected[i, i, 4], expected[i, 0, i] = slope * u, -H * u
for t in (5, 6, 7):  # the deflating extra times
    expected[t, 4, t], expected[t, 0, t] = -slope * v, H * v
```

```
def components(omega):
    """The nonzero omega_mu ab with a < b, as a dictionary."""
    return {(mu, a, b): omega[mu][a][b] for mu in range(8) for a in range(8)
            for b in range(a + 1, 8) if omega[mu][a][b] != 0}


def same_components(found, wanted):
    return set(found) == set(wanted) and all(
        sp.simplify(found[key] - wanted[key]) == 0 for key in wanted)


reproduces(same_components(components(omega_patch), expected),
           "the twelve spin-connection components of the record (patch)",
           PAIR_PY, "geometry.spin_connection_components")
flipped = {key: (-value if 0 in key[1:] else value)
           for key, value in expected.items()}  # x0 components change sign
reproduces(same_components(components(omega_mirror), flipped),
           "mirror patch: the x0 components change sign, the others do not",
           PAIR_WL, "connection_mirror_patch")
Omega = {}  # s0 -> the eight matrices Omega_mu
for s0, omega in ((1, omega_patch), (-1, omega_mirror)):
    Omega[s0] = [sum((omega[mu][a][b] * S_exact[a][b] / 2 for a in range(8)
                      for b in range(8) if omega[mu][a][b] != 0), sp.zeros(16))
                 for mu in range(8)]
term = {}  # s0 -> sum over mu of gamma^mu Omega_mu
for s0, E in ((1, E_patch), (-1, E_mirror)):
    term[s0] = sum((gamma[mu] / E[mu] * Omega[s0][mu] for mu in range(8)),
                   sp.zeros(16)).applyfunc(sp.simplify)
reproduces(term[1] == 3 * H * gamma[0], "patch: gamma^mu Omega_mu = 3 H gamma^(x0)",
           LEAD_EMT, "gamma_Omega_equals_3H_gamma0")
check(term[-1] == -3 * H * gamma[0], "mirror patch: gamma^mu Omega_mu = -3 H gamma^(x0)")
g_patch = [ETA[a] * E_patch[a] ** 2 for a in range(8)]
reproduces(all(sp.simplify(entry.subs(z, sp.pi - z) - entry) == 0
               for entry in g_patch),
           "z -> pi - z leaves all eight metric components unchanged",
           PAIR_PY, "geometry.mirror_isometry")
```

Out [5]:

```
PASS the twelve spin-connection components of the record (patch)
     reproduces Revision/pairing/reports/python-pairing.json, check
    geometry.spin_connection_components
PASS mirror patch: the x0 components change sign, the others do not
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    connection_mirror_patch
PASS patch: gamma^mu Omega_mu = 3 H gamma^(x0)
     reproduces Revision/lead_checks/reports/emt-divergence-and-spin-connection.json,
    check gamma_Omega_equals_3H_gamma0
PASS mirror patch: gamma^mu Omega_mu = -3 H gamma^(x0)
PASS z -> pi - z leaves all eight metric components unchanged
     reproduces Revision/pairing/reports/python-pairing.json, check
    geometry.mirror_isometry
```

Text cell:

```
## 8. A configuration and its T1 partner at one point

From here on we work with floating-point numbers. The next cell turns the
lengths and the connection into numpy functions (‘sp.lambdify‘) and defines two
helpers. ‘geometry_at(s0, z, a4, a4’, H)‘ returns, at one point, the lengths, the
metric, the eight $\Omega_\mu$ and the curved gammas $\gamma^\mu = \gamma^{(\mu)}/
E_\mu$ and $\gamma_\mu = g_{\mu\mu}\gamma^\mu$. ‘bilinears(at, psi, dpsi, m, lam)‘
evaluates, for the first jet (the value ‘psi‘ and the eight first derivatives
‘dpsi[mu]‘), the quantities of section 4 term by term:
$\bar\Psi = \Psi^\dagger C$, $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$,
$D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$, $S$, $K$,
$\mathcal{L}/\sqrt{|g|}$, the 64 components $T_{\mu\nu}$, the current $J^\mu$ and
the field-equation residual $E_{m,\lambda} = \gamma^\mu D_\mu\Psi - (m + \lambda S)
\Psi$ (zero exactly when the configuration solves the field equation).
```

In [6]:

```
point_symbols = (z, a4_value, slope, H)
NUMERIC = {}  # s0 -> (the function of the lengths, the connection pieces)
for s0, E, omega in ((1, E_patch, omega_patch), (-1, E_mirror, omega_mirror)):
    lengths_f = sp.lambdify(point_symbols, E, "numpy")
    pieces = [(mu, a, b, sp.lambdify(point_symbols, omega[mu][a][b], "numpy"))
              for mu in range(8) for a in range(8) for b in range(8)
              if omega[mu][a][b] != 0]
    NUMERIC[s0] = (lengths_f, pieces)


def geometry_at(s0, z_value, a4_number, slope_number, H_number=1.0):
    """Lengths, metric, Omega_mu and the curved gammas at one point."""
    lengths_f, pieces = NUMERIC[s0]
    values = (z_value, a4_number, slope_number, H_number)
    E = np.array([float(x) for x in lengths_f(*values)])
    Om = [np.zeros((16, 16)) for _ in range(8)]
    for mu, a, b, f in pieces:  # Omega_mu = (1/2) sum omega_mu ab S^ab
        Om[mu] += float(f(*values)) * S_num[a][b] / 2
    return {"g": np.array(ETA) * E ** 2, "Omega": Om,
            "up": [gamma_num[mu] / E[mu] for mu in range(8)],
            "down": [ETA[mu] * E[mu] * gamma_num[mu] for mu in range(8)]}


def bilinears(at, psi, dpsi, m, lam):
    """S, K, L/sqrt|g|, T_mu nu, J^mu, the field-equation residual, and the
    largest imaginary part of a quantity that must be real."""
    bar = psi.conj() @ C_num  # Psibar = Psi^dagger C (a row)
    dbar = [row.conj() @ C_num for row in dpsi]  # d_mu Psibar
    D = [dpsi[mu] + at["Omega"][mu] @ psi for mu in range(8)]  # D_mu Psi
    Dbar = [dbar[mu] - bar @ at["Omega"][mu] for mu in range(8)]  # D_mu Psibar
    S = bar @ psi
    K = 0.5 * sum(bar @ at["up"][mu] @ D[mu] - Dbar[mu] @ at["up"][mu] @ psi
                  for mu in range(8))
```

```
    L = K - m * S - lam / 2 * S ** 2  # L / sqrt|g|
    T = np.zeros((8, 8), dtype=complex)
    for mu, nu in itertools.product(range(8), repeat=2):
        T[mu, nu] = 0.25 * (bar @ at["down"][mu] @ D[nu]
                            + bar @ at["down"][nu] @ D[mu]
                            - Dbar[mu] @ at["down"][nu] @ psi
                            - Dbar[nu] @ at["down"][mu] @ psi)
        if mu == nu:
            T[mu, nu] -= at["g"][mu] * L
    J = np.array([-1j * bar @ at["up"][mu] @ psi for mu in range(8)])
    residual = sum(at["up"][mu] @ D[mu] for mu in range(8)) - (m + lam * S) * psi
    imaginary = max(abs(S.imag), abs(K.imag), np.abs(T.imag).max(),
                    np.abs(J.imag).max())
    return {"S": S.real, "K": K.real, "L": L.real, "T": T.real, "J": J.real,
            "E": residual, "imag": imaginary}
```

Text cell:

```
The next cell chooses a point on the patch ($z = 0.6$, $a_4 = 0.5$ and
$a_4' = H$: the author's deflating history $A = 1$; units $H = 1$), the mass
$m = 2$ and the coupling $\lambda = 1/2$, and a configuration: 16 complex values
and $8 \times 16$ complex first derivatives drawn by a random-number generator
with a fixed seed (so that every run draws the same numbers). Such a configuration
is in general NOT a solution; T1 is an identity that holds off shell, so it must
hold for it. The partner is $\Gamma\Psi$ (value and derivatives multiplied by
$\Gamma$) with $(-m, -\lambda)$. The cell checks, at this point: the quantities
that must be real are real; $S' = S$, $K' = -K$; every one of the 64 components
of $T + T'$ and the 8 components of $J + J'$ vanish; the field-equation residuals
obey $E_{-m,-\lambda}[\Gamma\Psi] = -\Gamma E_{m,\lambda}[\Psi]$ (so one vanishes
exactly when the other does); and two negative controls (a wrong partner) do NOT
cancel.
```

In [7]:

```
rng = np.random.default_rng(20261007)  # a fixed seed: the same numbers every run
R0_SIGNS = [-1, 1, 1, 1, 1, 1, 1, 1]  # the mirror reverses the direction x0
psi = rng.normal(size=16) + 1j * rng.normal(size=16)  # the 16 complex values
dpsi = [rng.normal(size=16) + 1j * rng.normal(size=16) for _ in range(8)]
dpsi = dpsi[7:] + dpsi[:7]  # drawn hidden direction last, as before; now x0 first
Z0, A4_0, M0, LAM0 = 0.6, 0.5, 2.0, 0.5  # the point and the parameters (H = 1)
at = geometry_at(1, Z0, A4_0, 1.0)  # a4' = 1: the deflating history A = 1
one = bilinears(at, psi, dpsi, M0, LAM0)  # Psi with (m, lambda)
partner = bilinears(at, Gamma_num @ psi, [Gamma_num @ row for row in dpsi],
                    -M0, -LAM0)  # Gamma Psi with (-m, -lambda)
scale = np.abs(one["T"]).max()  # the size of the tensor
report("S of the configuration", f"{one["S"]:.6f}")
report("largest |T_mu nu| of the configuration", f"{scale:.4f}")
check(max(one["imag"], partner["imag"]) < 1e-12 * scale,
      "S, K, T and J are real numbers (to rounding)")
reproduces(abs(partner["S"] - one["S"]) < 1e-12 and abs(partner["K"] + one["K"])
           < 1e-12, "S' = S and K' = -K", PAIR_PY, "T1.metric.commuting.S_invariant")
reproduces(np.abs(one["T"] + partner["T"]).max() < 1e-12 * scale,
```

```
          "T1 pair: all 64 components of T + T' vanish",
          PAIR_PY, "T1.metric.commuting.pair_total_emt_zero")
reproduces(np.abs(one["J"] + partner["J"]).max() < 1e-12 * scale,
          "T1 pair: all 8 components of J + J' vanish",
          PAIR_WL, "T1_current_primordial_commuting")
reproduces(np.abs(partner["E"] + Gamma_num @ one["E"]).max() < 1e-12 * scale,
          "E_(-m,-lambda)[Gamma Psi] = -Gamma E_(m,lambda)[Psi]",
          PAIR_PY, "T1.metric.commuting.euler_lagrange_map")
wrong_1 = bilinears(at, Gamma_num @ psi, [Gamma_num @ row for row in dpsi],
                    -M0, LAM0)  # only the mass reversed
wrong_2 = bilinears(at, Gamma_num @ psi, [Gamma_num @ row for row in dpsi],
                    M0, LAM0)  # nothing reversed
reproduces(min(np.abs(one["T"] + wrong_1["T"]).max(),
               np.abs(one["T"] + wrong_2["T"]).max()) > 1e-3 * scale,
          "negative controls: with (-m, +lambda) or (m, lambda) no cancellation",
          PAIR_PY, "T1.metric.commuting.negative_controls")
```

Out [7]:

```
RESULT S of the configuration = 1.354934
RESULT largest |T_mu nu| of the configuration = 26.7625
PASS S, K, T and J are real numbers (to rounding)
PASS S' = S and K' = -K
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.S_invariant
PASS T1 pair: all 64 components of T + T' vanish
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.pair_total_emt_zero
PASS T1 pair: all 8 components of J + J' vanish
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    T1_current_primordial_commuting
PASS E_(-m,-lambda)[Gamma Psi] = -Gamma E_(m,lambda)[Psi]
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.euler_lagrange_map
PASS negative controls: with (-m, +lambda) or (m, lambda) no cancellation
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.negative_controls
```

Text cell:

```
The next cell draws the three $8 \times 8$ tables: $T_{\mu\nu}$ of the
configuration, $T'_{\mu\nu}$ of its T1 partner, and their sum.
```

In [8]:

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.3))
panels = ((one["T"], "$T_{\\mu\\nu}[\\Psi;\\,m,\\lambda]$"),
          (partner["T"], "$T_{\\mu\\nu}[\\Gamma\\Psi;\\,-m,-\\lambda]$"),
          (one["T"] + partner["T"], "sum: the source of the pair"))
for ax, (values, title) in zip(axes, panels):
    image = ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
```

```
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)  # no grid lines across the coloured squares
    ax.set_title(title, fontsize=10)
fig.colorbar(image, ax=list(axes), shrink=0.8, label="value (units $H = 1$)")
save_figure(fig, "t1_pair_tensor",
            "The energy-momentum tensor $T_{\\mu\\nu}$ (rows $\\mu$, columns $\\nu$, "
            "$x_0$ to $x_7$) of an arbitrary configuration $\\Psi$ of "
            "dirac16complex00 with $m = 2$, $\\lambda = 1/2$ at one point of the "
            "author's metric ($z = 0.6$, $a_4 = 0.5$, $a_4' = H$, units $H = 1$; "
            "colour scale red positive, blue negative), of its T1 partner "
            "$\\Gamma\\Psi$ with $(-m, -\\lambda)$, and of their sum. The middle "
            "table is the left one with every sign reversed, so the sum is white: "
            "all 64 components of the source of the pair are zero, as theorem T1 "
            "says, although the configuration is not even a solution.")
```

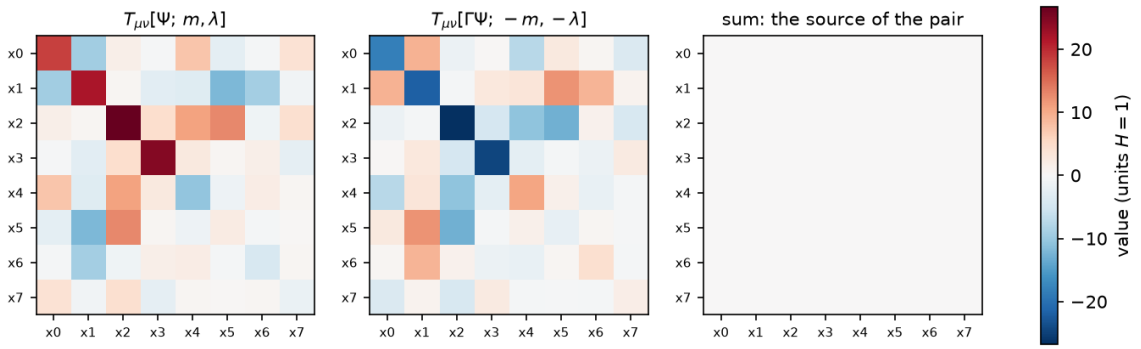

Figure 507: The energy-momentum tensor $T_{\mu\nu}$ (rows μ, columns ν, x_0 to x_7) of an arbitrary configuration Ψ of dirac16complex00 with $m = 2$, $\lambda = 1/2$ at one point of the author's metric ($z = 0.6$, $a_4 = 0.5$, $a_4' = H$, units $H = 1$; colour scale red positive, blue negative), of its T1 partner $\Gamma\Psi$ with $(-m, -\lambda)$, and of their sum. The middle table is the left one with every sign reversed, so the sum is white: all 64 components of the source of the pair are zero, as theorem T1 says, although the configuration is not even a solution. (Notebook 20a, figure 1.)

Out [8]:

```
Figure 20a.1 saved as Revision/textbook/figures/20a_1_t1_pair_tensor.png
```

Text cell:

```
## 9. Along the hidden direction: the T1 partner and the T2 mirror copy

T1 holds at EVERY point. To see it along a line, the next cell uses a field that
varies along the hidden direction, $\Psi(z) = \Psi_0 + \tfrac{3}{10}(\pi/2 - z)^2
\Psi_1$, with the values $\Psi_0$ and the derivatives along $x_1, \dots, x_7$ of
section 8 and a second random spinor $\Psi_1$. Its derivative along $x_0$ is
$\partial_0\Psi = 6H\,d\Psi/dz = -\tfrac{18}{5}H(\pi/2 - z)\Psi_1$; it vanishes at
```

```
the brane like $\pi/2 - z$, which keeps the term
$\tan z\,\gamma^{(x_0)}\partial_0\Psi$ of the field equation finite there (the
factor $\tan z = 1/E_0$ grows without bound at $z = \pi/2$, where
$g_{00} = \cot^2z$ vanishes). The field is not a solution (T1 and T2 hold off
shell). The geometry changes with $z$ through $\sin^{1/6}z$ and $\cot z$. The cell
evaluates three objects:

- the field $\Psi$ with $(m, \lambda)$ on the patch, $0 < z < \pi/2$;
- its T1 partner $\Gamma\Psi$ with $(-m, -\lambda)$ at the same points;
- its T2 mirror copy: by theorem T2 (with $m$ replaced by $-m$), the field
  $\Psi'(\pi - z) = \gamma^{(x_0)}\Psi(z)$ on the mirror patch solves the
  $(-m, \lambda)$ equations if and only if $\Psi$ solves the $(m, \lambda)$
  equations. Its derivatives are $\gamma^{(x_0)}$ times those of $\Psi$, with the
  $x_0$ derivative reversed (the mirror reverses the direction $x_0$).

It computes the energy density $\rho = -T_{x_4x_4}$ and the charge density
$J^{x_4}$ of each, and checks: the T1 partner has $-\rho$ and $-J^{x_4}$; the
mirror copy has, at the mirror point $\pi - z$, the SAME $\rho$ and the SAME
$J^{x_4}$.
```

In [9]:

```
psi_1 = rng.normal(size=16) + 1j * rng.normal(size=16)  # the spinor Psi_1
z_patch = np.linspace(0.05, np.pi / 2 - 0.02, 160)  # points of the patch
rho, charge = {"one": [], "T1": [], "T2": []}, {"one": [], "T1": [], "T2": []}
for z_value in z_patch:
    field = psi + 0.3 * (np.pi / 2 - z_value) ** 2 * psi_1  # Psi(z)
    slopes_here = [row.copy() for row in dpsi]  # the derivatives along x1 .. x7
    slopes_here[0] = -3.6 * (np.pi / 2 - z_value) * psi_1  # 6 H dPsi/dz, H = 1
    mirror_slopes = [R0_SIGNS[mu] * (gamma_num[0] @ slopes_here[mu])
                     for mu in range(8)]  # the mirror reverses d/dx0
    here = geometry_at(1, z_value, A4_0, 1.0)
    there = geometry_at(-1, np.pi - z_value, A4_0, 1.0)  # the mirror point
    results = {
        "one": bilinears(here, field, slopes_here, M0, LAM0),
        "T1": bilinears(here, Gamma_num @ field,
                        [Gamma_num @ row for row in slopes_here], -M0, -LAM0),
        "T2": bilinears(there, gamma_num[0] @ field, mirror_slopes, -M0, LAM0),
    }
    for key, result in results.items():
        rho[key].append(-result["T"][4, 4])  # rho = -T_x4x4
        charge[key].append(result["J"][4])  # J^x4
rho = {key: np.array(values) for key, values in rho.items()}
charge = {key: np.array(values) for key, values in charge.items()}
size = max(np.abs(rho["one"]).max(), np.abs(charge["one"]).max())
check(np.abs(rho["T1"] + rho["one"]).max() < 1e-12 * size
      and np.abs(charge["T1"] + charge["one"]).max() < 1e-12 * size,
      "along z: the T1 partner has -rho and -J^x4 at every point")
check(np.abs(rho["T2"] - rho["one"]).max() < 1e-10 * size
      and np.abs(charge["T2"] - charge["one"]).max() < 1e-10 * size,
      "along z: the T2 copy has the same rho and J^x4 at the mirror point")
```

Out [9]:

```
PASS along z: the T1 partner has -rho and -J^x4 at every point
PASS along z: the T2 copy has the same rho and J^x4 at the mirror point
```

Text cell:

```
The next cell draws the two densities across the whole interval $0 < z < \pi$:
on the left of the brane the configuration and its T1 partner, on the right the
T2 mirror copy (plotted at its own points $\pi - z$).
```

In [10]:

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for ax, data, label in ((axes[0], rho, "energy density $\\rho$"),
                        (axes[1], charge, "charge density $J^{x_4}$")):
    ax.plot(z_patch, data["one"], color="tab:blue", label="$\\Psi$, $(m, \\lambda)$")
    ax.plot(z_patch, data["T1"], "--", color="tab:red",
            label="T1 partner $\\Gamma\\Psi$, $(-m, -\\lambda)$")
    ax.plot(z_patch, data["one"] + data["T1"], color="black", linewidth=2.0,
            label="T1 pair: sum")
    ax.plot(np.pi - z_patch, data["T2"], color="tab:green",
            label="T2 copy $\\gamma^{(x_0)}\\Psi$, $(-m, \\lambda)$")
    ax.axvline(np.pi / 2, color="grey", linestyle=":", linewidth=1.5)
    ax.text(np.pi / 2, ax.get_ylim()[1], " brane", va="top", fontsize=8)
    ax.set_xlabel("hidden coordinate $z = 6Hx_0$")
    ax.set_ylabel(label + " (units $H = 1$)")
handles, names = axes[0].get_legend_handles_labels()
fig.legend(handles, names, loc="lower center", ncol=4, fontsize=8.5,
           bbox_to_anchor=(0.5, -0.06))  # one legend below both panels
axes[0].set_title("T1: reversed at the same point; T2: equal at the mirror point",
                  fontsize=9)
axes[1].set_title("the same for the charge density", fontsize=9)
save_figure(fig, "pair_profiles",
            "Left: the energy density $\\rho = -T_{x_4x_4}$, right: the charge "
            "density $J^{x_4} = \\Psi^\\dagger B\\Psi$, as functions of the hidden "
            "coordinate $z$ (horizontal, from 0 to $\\pi$; the brane $z = \\pi/2$ "
            "dotted; units $H = 1$), for an arbitrary field $\\Psi(z)$ of "
            "dirac16complex00 (not a solution) with $(m, \\lambda) = (2, 1/2)$ on "
            "the patch (blue), "
            "its T1 partner $\\Gamma\\Psi$ with $(-m, -\\lambda)$ (red dashed) and "
            "their sum (black, zero everywhere), and the T2 mirror copy "
            "$\\gamma^{(x_0)}\\Psi$ with $(-m, \\lambda)$ on the mirror patch "
            "(green). The T1 partner is the blue curve reflected in the horizontal "
            "axis; the T2 copy is the blue curve reflected in the vertical line of "
            "the brane, with the same sign: T1 pairs cancel, T2 pairs add.")
```

Out [10]:

```
Figure 20a.2 saved as Revision/textbook/figures/20a_2_pair_profiles.png
```

Text cell:

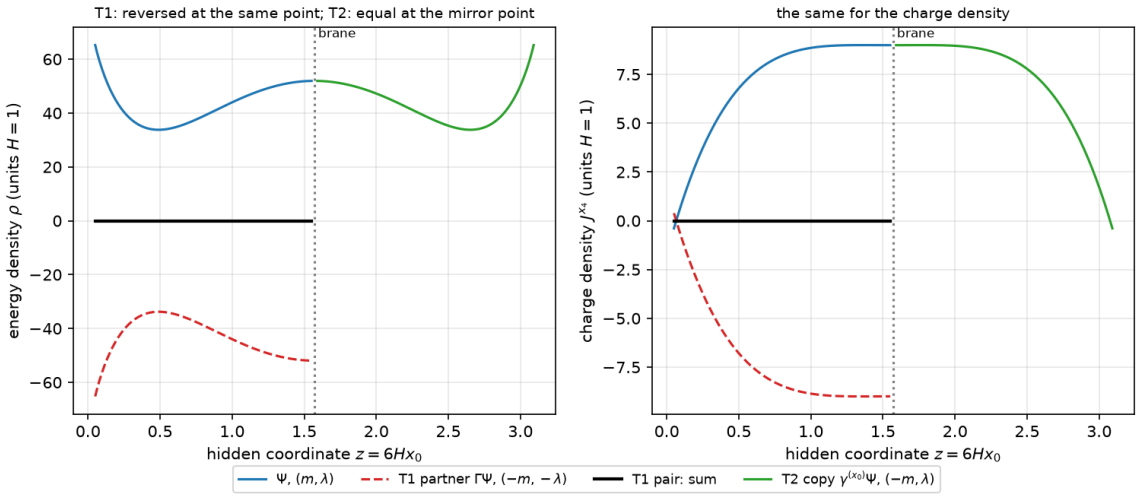

Figure 508: Left: the energy density $\rho = -T_{x_4x_4}$, right: the charge density $J^{x_4} = \Psi^\dagger B\Psi$, as functions of the hidden coordinate z (horizontal, from 0 to π; the brane $z = \pi/2$ dotted; units $H = 1$), for an arbitrary field $\Psi(z)$ of dirac16complex00 (not a solution) with $(m, \lambda) = (2, 1/2)$ on the patch (blue), its T1 partner $\Gamma\Psi$ with $(-m, -\lambda)$ (red dashed) and their sum (black, zero everywhere), and the T2 mirror copy $\gamma^{(x_0)}\Psi$ with $(-m, \lambda)$ on the mirror patch (green). The T1 partner is the blue curve reflected in the horizontal axis; the T2 copy is the blue curve reflected in the vertical line of the brane, with the same sign: T1 pairs cancel, T2 pairs add. (Notebook 20a, figure 2.)

```
## 10. The T2 mirror pair does not cancel

Theorem T2 in the author's field says more than equal energy densities: at the
mirror point the whole tensor of the copy is the pulled-back tensor,
$T_{\mu\nu}[\Psi'](\pi - z) = (R_0\,T[\Psi](z)\,R_0)_{\mu\nu}$ with $R_0 =
\mathrm{diag}(-1,1,1,1,1,1,1,1)$: the components with exactly one index $x_0$
change sign (the direction $x_0$ is reversed by the mirror), all others are EQUAL.
Likewise $J'^\mu = (R_0)^\mu{}_\nu J^\nu$, $S' = -S$, and the field-equation
residuals obey $E_{-m,\lambda}[\Psi'] = -\gamma^{(x_0)}E_{m,\lambda}[\Psi]$. The
next cell checks all of this at the point $z = 0.6$ with the full configuration
(including its derivative along $x_0$, which the mirror reverses) and draws the
three tables.
```

In [11]:

```
R0 = np.array(R0_SIGNS, dtype=float)  # the reflection R0 of the x0 direction
mirror_dpsi = [R0[mu] * (gamma_num[0] @ dpsi[mu]) for mu in range(8)]
copy = bilinears(geometry_at(-1, np.pi - Z0, A4_0, 1.0), gamma_num[0] @ psi,
                 mirror_dpsi, -M0, LAM0)  # the T2 copy at the mirror point
pulled_back = np.outer(R0, R0) * one["T"]  # R0 T R0
reproduces(np.abs(copy["T"] - pulled_back).max() < 1e-10 * scale,
           "T2 copy: T' = R0 T R0 at the mirror point (64 components)",
           PAIR_PY, "T2.metric.commuting.emt")
reproduces(np.abs(copy["J"] - R0 * one["J"]).max() < 1e-10 * scale,
           "T2 copy: J' = R0 J (the charge density is unchanged)",
           PAIR_PY, "T2.metric.commuting.current")
reproduces(abs(copy["S"] + one["S"]) < 1e-10 * scale, "T2 copy: S' = -S",
```

```
           PAIR_PY, "T2.metric.commuting.S_odd")
reproduces(np.abs(copy["E"] + gamma_num[0] @ one["E"]).max() < 1e-10 * scale,
           "T2 copy: E_(-m,lambda)[Psi'] = -gamma^(x0) E_(m,lambda)[Psi]",
           PAIR_PY, "T2.metric.commuting.euler_lagrange_map")
reproduces(np.abs(copy["T"] + one["T"]).max() > 0.1 * scale,
           "T2 pair: the tensors do NOT cancel",
           PAIR_WL, "T2_mirror_energy_momentum_and_current_commuting")
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.3))
panels = ((one["T"], "$T_{\\mu\\nu}[\\Psi]$ at $z$"),
          (copy["T"], "$T_{\\mu\\nu}$ of the T2 copy at $\\pi - z$"),
          (copy["T"] - pulled_back, "copy minus $R_0TR_0$"))
for ax, (values, title) in zip(axes, panels):
    image = ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)
    ax.set_title(title, fontsize=10)
fig.colorbar(image, ax=list(axes), shrink=0.8, label="value (units $H = 1$)")
save_figure(fig, "t2_mirror_tensor",
            "Left: the energy-momentum tensor $T_{\\mu\\nu}$ of the configuration "
            "$\\Psi$ with $(m, \\lambda) = (2, 1/2)$ at $z = 0.6$ on the patch "
            "(rows and columns $x_0$ to $x_7$, units $H = 1$). Middle: the tensor "
            "of its T2 mirror copy $\\gamma^{(x_0)}\\Psi$ with $(-m, \\lambda)$ at "
            "the mirror point $z = \\pi - 0.6$. The two tables are equal except "
            "for the first row and column, whose off-diagonal entries (one index "
            "$x_0$) change sign. Right: the copy minus $R_0TR_0$, zero everywhere. "
            "Unlike the T1 partner, the mirror copy carries the SAME energy, so "
            "the T2 pair has twice the source of one member.")
```

Out [11]:

```
PASS T2 copy: T' = R0 T R0 at the mirror point (64 components)
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.emt
PASS T2 copy: J' = R0 J (the charge density is unchanged)
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.current
PASS T2 copy: S' = -S
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.S_odd
PASS T2 copy: E_(-m,lambda)[Psi'] = -gamma^(x0) E_(m,lambda)[Psi]
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.euler_lagrange_map
PASS T2 pair: the tensors do NOT cancel
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    T2_mirror_energy_momentum_and_current_commuting
```

Out [11]:

```
Figure 20a.3 saved as Revision/textbook/figures/20a_3_t2_mirror_tensor.png
```

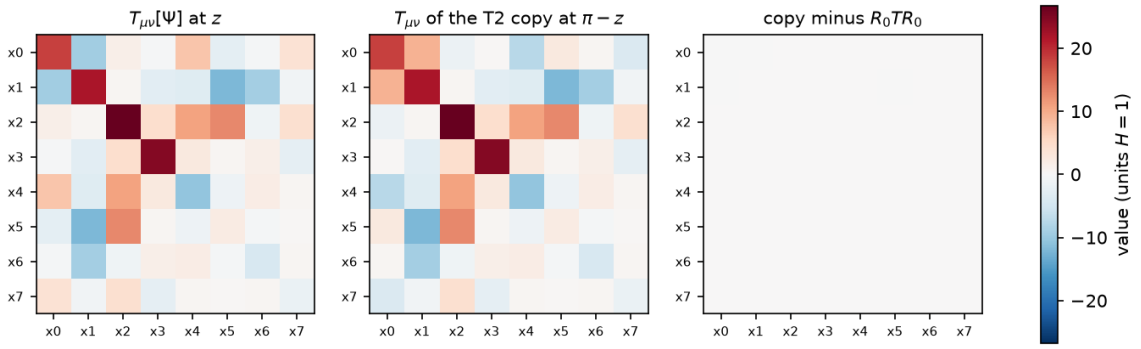

Figure 509: Left: the energy-momentum tensor $T_{\mu\nu}$ of the configuration Ψ with $(m, \lambda) = (2, 1/2)$ at $z = 0.6$ on the patch (rows and columns x_0 to x_7, units $H = 1$). Middle: the tensor of its T2 mirror copy $\gamma^{(x_0)}\Psi$ with $(-m, \lambda)$ at the mirror point $z = \pi - 0.6$. The two tables are equal except for the first row and column, whose off-diagonal entries (one index x_0) change sign. Right: the copy minus $R_0 T R_0$, zero everywhere. Unlike the T1 partner, the mirror copy carries the SAME energy, so the T2 pair has twice the source of one member. (Notebook 20a, figure 3.)

Text cell:

```
## 11. The pair as the only source: Einstein's vacuum equations

By lines 1 and 2 of the proof (checked in sections 8 and 9) a T1 pair as the only
source gives $T = 0$, and the field equations of gravity become the vacuum
equations. The next cell reads the four independent components of the Einstein
tensor $G = E_{(1)}$ from the record `a4-equations.json` (the symbols `ad1` and
`ad2` there are $a_4'$ and $a_4''$), forms the vacuum equations
$G^\mu{}_\mu + \Lambda = 0$ for $\mu = x_0, x_1, x_4, x_5$ (the other diagonal
components repeat these, the off-diagonal ones are identically zero), and lets
sympy solve them for $a_4'$, $a_4''$ and $\Lambda$ WITHOUT assuming that $a_4'$
is real. It also forms the two combinations of the proof: the $x_4$ equation minus
the $x_0$ equation, and their sum.
```

In [12]:

```
record = read_json(A4_RECORD)
ad1, ad2 = sp.symbols("ad1 ad2")  # a4' and a4'' (complex allowed)
Lam = sp.symbols("Lambda")  # the cosmological constant
alpha2, alpha3, A = sp.symbols("alpha2 alpha3 A", real=True)
KEYS = ["x0x0", "x1x1", "x4x4", "x5x5"]  # the independent diagonal components


def lovelock(order, key):
    """The component key of E_(order) from the record, as a sympy expression."""
    text = record["lovelockTensors"][f"E{order}"][key]["input"].replace("^", "**")
    return sp.sympify(text, locals={"ad1": ad1, "ad2": ad2, "H": H})


G = {key: lovelock(1, key) for key in KEYS}  # the Einstein tensor
for key in KEYS:
```

```
    say(f"G^{key[:2]}_{key[2:]} = {G[key]}")
vacuum = [G[key] + Lam for key in KEYS]  # zero source
solutions = sp.solve(vacuum, [ad1, ad2, Lam], dict=True)
say(f"all solutions of the vacuum equations: {solutions}")
found = {(s[ad1], s[ad2], s[Lam]) for s in solutions}
reproduces(found == {(sp.I * H, 0, -18 * H ** 2), (-sp.I * H, 0, -18 * H ** 2)},
           "the only solutions have a4' = +i H or -i H: none is real",
           LEAD_EGB, "no_vacuum_for_H_positive")
difference = sp.expand(vacuum[2] - vacuum[0])  # x4 equation minus x0 equation
total = sp.expand(vacuum[2] + vacuum[0])  # x4 equation plus x0 equation
report("x4 equation minus x0 equation", difference)
report("x4 equation plus x0 equation", total)
reproduces(difference == 6 * ad1 ** 2 + 6 * H ** 2
           and total == 36 * H ** 2 + 2 * Lam,
           "6 a4'^2 + 6 H^2 = 0 and 36 H^2 + 2 Lambda = 0 (record's lines)",
           A4_WL, "einstein_no_vacuum_solution")
a_real = sp.symbols("a_real", real=True)  # a real value of a4'
reproduces(sp.solve(difference.subs(ad1, a_real), a_real) == [],
           "for real a4' the difference 6 a4'^2 + 6 H^2 never vanishes",
           A4_PY, "einstein_no_vacuum")
```

Out [12]:

```
G^x0_x0 = 15*H**2 - 3*ad1**2
G^x1_x1 = 15*H**2 - 3*ad1**2 + ad2
G^x4_x4 = 21*H**2 + 3*ad1**2
G^x5_x5 = 15*H**2 - 3*ad1**2 - ad2
all solutions of the vacuum equations: [{Lambda: -18*H**2, ad1: -I*H, ad2: 0}, {Lambda:
    -18*H**2, ad1: I*H, ad2: 0}]
PASS the only solutions have a4' = +i H or -i H: none is real
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    no_vacuum_for_H_positive
RESULT x4 equation minus x0 equation = 6*H**2 + 6*ad1**2
RESULT x4 equation plus x0 equation = 36*H**2 + 2*Lambda
PASS 6 a4'^2 + 6 H^2 = 0 and 36 H^2 + 2 Lambda = 0 (record's lines)
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    einstein_no_vacuum_solution
PASS for real a4' the difference 6 a4'^2 + 6 H^2 never vanishes
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_no_vacuum
```

Text cell:

```
The next cell draws the two vacuum conditions as curves in the plane of $a_4'/H$
(horizontal) and $\Lambda/H^2$ (vertical): the $x_4$ equation holds on the
parabola $\Lambda = -(3a_4'^2 + 21H^2)$, which opens downwards, the $x_0$ equation
on $\Lambda = 3a_4'^2 - 15H^2$, which opens upwards. A vacuum would be a point on
both. The vertical distance between them is $6a_4'^2 + 6H^2 \geq 6H^2$: they
never meet. At the author's history $a_4' = H$ the gap is $12H^2$.
```

In [13]:

```
slopes = np.linspace(-3.0, 3.0, 601)  # a4'/H
lower = -(3 * slopes ** 2 + 21)  # Lambda/H^2 where the x4 equation holds
upper = 3 * slopes ** 2 - 15  # Lambda/H^2 where the x0 equation holds
check(np.min(upper - lower) >= 6.0 - 1e-12, "the two curves stay 6 H^2 apart or more")
fig, ax = plt.subplots(figsize=(7.0, 4.6))
ax.plot(slopes, lower, color="tab:red", label="$x_4$: $\\Lambda = -(3a_4'^2 + 21H^2)$")
ax.plot(slopes, upper, color="tab:blue", label="$x_0$: $\\Lambda = 3a_4'^2 - 15H^2$")
ax.fill_between(slopes, lower, upper, color="grey", alpha=0.2,
                label="gap $6a_4'^2 + 6H^2$")
ax.annotate("", xy=(1.0, -12.0), xytext=(1.0, -24.0),
            arrowprops={"arrowstyle": "<->", "color": "black"})
ax.text(1.12, -19.3, "$12H^2$ at $a_4' = H$\n(the author's history)", fontsize=8,
        va="top")  # va="top": the text hangs below the given height
ax.axhline(-18.0, color="black", linestyle=":", linewidth=1.0)
ax.text(-2.95, -17.3, "$\\Lambda = -18H^2$: both hold only for $a_4' = \\pm iH$",
        fontsize=8)
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("$\\Lambda/H^2$")
ax.set_title("Einstein gravity with zero source: the two conditions never meet")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "einstein_vacuum_parabolas",
            "The vacuum equations of Einstein gravity for the author's metric, the "
            "situation of a T1 pair as the only source: horizontal axis the rate "
            "$a_4'/H$, vertical axis the cosmological constant $\\Lambda/H^2$. The "
            "$x_4$ component holds on the red parabola, the $x_0$ component on the "
            "blue one; a vacuum solution would lie on both. They are separated by "
            "$6a_4'^2 + 6H^2 \\geq 6H^2$ (grey band), $12H^2$ at the author's "
            "deflating history $a_4' = H$, so no real rate and no $\\Lambda$ solve "
            "both; the only solutions, $a_4' = \\pm iH$ with $\\Lambda = -18H^2$ "
            "(dotted line), are not real.")
```

Out [13]:

```
PASS the two curves stay 6 H^2 apart or more
```

Out [13]:

```
Figure 20a.4 saved as Revision/textbook/figures/20a_4_einstein_vacuum_parabolas.png
```

Text cell:

```
## 12. What the geometry requires and what a pair supplies

Line 5 of the proof is a statement about the source. Write the $x_4$ and $x_0$
equations with a source: $\sum_k\alpha_kE_{(k)}{}^{x_4}{}_{x_4} + \Lambda =
-\kappa\rho$ and $\sum_k\alpha_kE_{(k)}{}^{x_0}{}_{x_0} + \Lambda = \kappa p_0$
(the record's sign convention). Subtracting the first from the second removes
$\Lambda$ and gives the **null combination** the geometry REQUIRES of any source:

$$\kappa(\rho + p_0) = -\sum_k\alpha_k\big(E_{(k)}{}^{x_4}{}_{x_4} -
E_{(k)}{}^{x_0}{}_{x_0}\big).$$
```

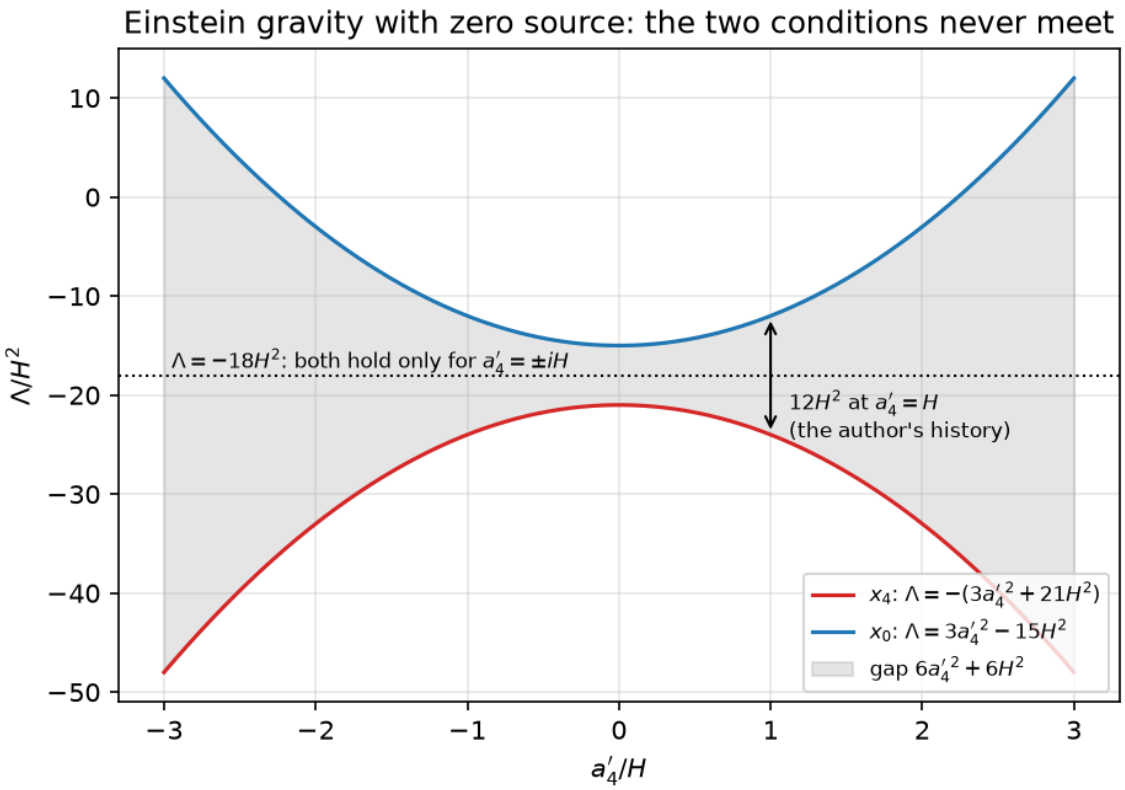

Figure 510: The vacuum equations of Einstein gravity for the author's metric, the situation of a T1 pair as the only source: horizontal axis the rate a_4'/H, vertical axis the cosmological constant Λ/H^2. The x_4 component holds on the red parabola, the x_0 component on the blue one; a vacuum solution would lie on both. They are separated by $6a_4'^2 + 6H^2 \geq 6H^2$ (grey band), $12H^2$ at the author's deflating history $a_4' = H$, so no real rate and no Λ solve both; the only solutions, $a_4' = \pm iH$ with $\Lambda = -18H^2$ (dotted line), are not real. (Notebook 20a, figure 4.)

```
In Einstein gravity this is $-6(a_4'^2 + H^2) < 0$ (the record's
'einstein_null_energy_x0'). For the linear member $a_4' = AH$ the record writes
the right-hand side as $-6(A^2 + 1)H^2V$ with the **vacuum factor**

$$V = \alpha_1 - 40\alpha_2H^2 - 8A^2\alpha_2H^2 + 360\alpha_3H^4 +
144A^2\alpha_3H^4 + 72A^4\alpha_3H^4 .$$

A T1 pair SUPPLIES $\rho + p_0 = 0$; so it can be the only source only where the
requirement is zero, that is where $V = 0$. The next cell reads the record's null
combination and vacuum factor, checks both against the Lovelock components, and
checks the Einstein case.
```

In [14]:

```
couplings = {"alpha1": 1, "alpha2": alpha2, "alpha3": alpha3, "H": H, "ad1": ad1,
             "ad2": ad2, "AA": A}  # alpha1 = 1 throughout
null_text = record["generalSource"]["nullCombinations"]["kappa(rho+p0)"]["input"]
null_required = sp.sympify(null_text.replace("^", "**"), locals=couplings)
V_text = record["linearMember"]["vacuumFactor"]["input"]
V = sp.sympify(V_text.replace("^", "**"), locals=couplings)
lovelock_sum = {key: lovelock(1, key) + alpha2 * lovelock(2, key)
                + alpha3 * lovelock(3, key) for key in KEYS}
own_null = -(lovelock_sum["x4x4"] - lovelock_sum["x0x0"])
check(sp.expand(own_null - null_required) == 0,
      "the null combination of the record equals -(E^x4_x4 - E^x0_x0)")
linear = {ad1: A * H, ad2: 0}  # the linear member a4 = A H x4 + a0
reproduces(sp.expand(own_null.subs(linear) + 6 * (A ** 2 + 1) * H ** 2 * V) == 0,
           "linear member: kappa (rho + p0) = -6 (A^2 + 1) H^2 V",
           A4_WL, "linear_member_vacuum_factor")
reproduces(sp.expand(V.subs({alpha2: 0, alpha3: 0})) == 1
           and sp.expand(null_required.subs({alpha2: 0, alpha3: 0})
                         + 6 * ad1 ** 2 + 6 * H ** 2) == 0,
           "Einstein: V = 1 and kappa (rho + p0) = -6 (a4'^2 + H^2)",
           A4_PY, "einstein_null_energy_x0")
reproduces(sp.expand(null_required.subs({alpha2: 0, alpha3: 0})
                     + 6 * ad1 ** 2 + 6 * H ** 2) == 0,
           "the same null combination from the lead's independent Einstein tensor",
           LEAD_EGB, "einstein_null_energy")
```

Out [14]:

```
PASS the null combination of the record equals -(E^x4_x4 - E^x0_x0)
PASS linear member: kappa (rho + p0) = -6 (A^2 + 1) H^2 V
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    linear_member_vacuum_factor
PASS Einstein: V = 1 and kappa (rho + p0) = -6 (a4'^2 + H^2)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_null_energy_x0
PASS the same null combination from the lead's independent Einstein tensor
     reproduces Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json, check
    einstein_null_energy
```

Text cell:

```
The next cell draws the required null combination $\kappa(\rho + p_0)/H^2$ for the
linear member as a function of the slope $A$, in Einstein gravity and in two
Einstein-Gauss-Bonnet theories ($\alpha_3 = 0$; $\alpha_2H^2 = 1/48$ and $1/40$).
The black line is what a T1 pair supplies: zero. In Einstein gravity the curve
stays at or below $-6H^2$; in the Gauss-Bonnet theories it crosses zero once.
```

In [15]:

```
A_axis = np.linspace(0.0, 2.0, 401)
V_numeric = sp.lambdify((A, alpha2, alpha3), V.subs(H, 1), "numpy")
fig, ax = plt.subplots(figsize=(7.0, 4.4))
for x2, style, label in ((0.0, "-", "Einstein ($\\alpha_2 = 0$)"),
                         (1 / 48, "--", "Gauss-Bonnet, $\\alpha_2H^2 = 1/48$"),
                         (1 / 40, "-.", "Gauss-Bonnet, $\\alpha_2H^2 = 1/40$")):
    required = -6 * (A_axis ** 2 + 1) * V_numeric(A_axis, x2, 0.0)
    ax.plot(A_axis, required, style, label=label)
ax.axhline(0.0, color="black", linewidth=2.0, label="what a T1 pair supplies: 0")
ax.plot([1.0, 0.0], [0.0, 0.0], "o", color="tab:red", markersize=7,
        label="a T1 pair fits: $V = 0$")
ax.set_xlabel("slope $A$ of the linear member $a_4 = AHx_4$")
ax.set_ylabel("required $\\kappa(\\rho + p_0)/H^2$")
ax.set_title("The source the geometry requires, and the zero of a T1 pair")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "null_combination",
            "The null combination $\\kappa(\\rho + p_0)$ (energy density plus the "
            "pressure along $x_0$, times the gravitational coupling; vertical, "
            "units $H^2$) that the field equations require of ANY source of the "
            "linear member $a_4 = AHx_4$, against its slope $A$ (horizontal), for "
            "Einstein gravity (solid) and two Einstein-Gauss-Bonnet theories with "
            "$\\alpha_2H^2 = 1/48$ (dashed) and $1/40$ (dash-dotted). A T1 pair "
            "supplies zero (black line). In Einstein gravity the requirement is "
            "$-6(A^2 + 1)H^2 < 0$ for every $A$, so a T1 pair can never be the "
            "only source; with the Gauss-Bonnet term the requirement vanishes at "
            "one slope (red dots: $A = 1$, the author's history, for "
            "$\\alpha_2H^2 = 1/48$, and $A = 0$ for $1/40$).")
```

Out [15]:

```
Figure 20a.5 saved as Revision/textbook/figures/20a_5_null_combination.png
```

Text cell:

```
## 13. Einstein-Gauss-Bonnet gravity: when a zero source is allowed

Part (b) of C1. With $\alpha_1 = 1$ and $\alpha_3 = 0$ the vacuum factor is
$V = 1 - 40\alpha_2H^2 - 8A^2\alpha_2H^2 = 1 - 8\alpha_2H^2(A^2 + 5)$. Setting
$V = 0$ and solving for $A^2$, line by line: add $8\alpha_2H^2(A^2 + 5)$ to both
sides, $8\alpha_2H^2(A^2 + 5) = 1$; divide by $8\alpha_2H^2$ (not zero),
```

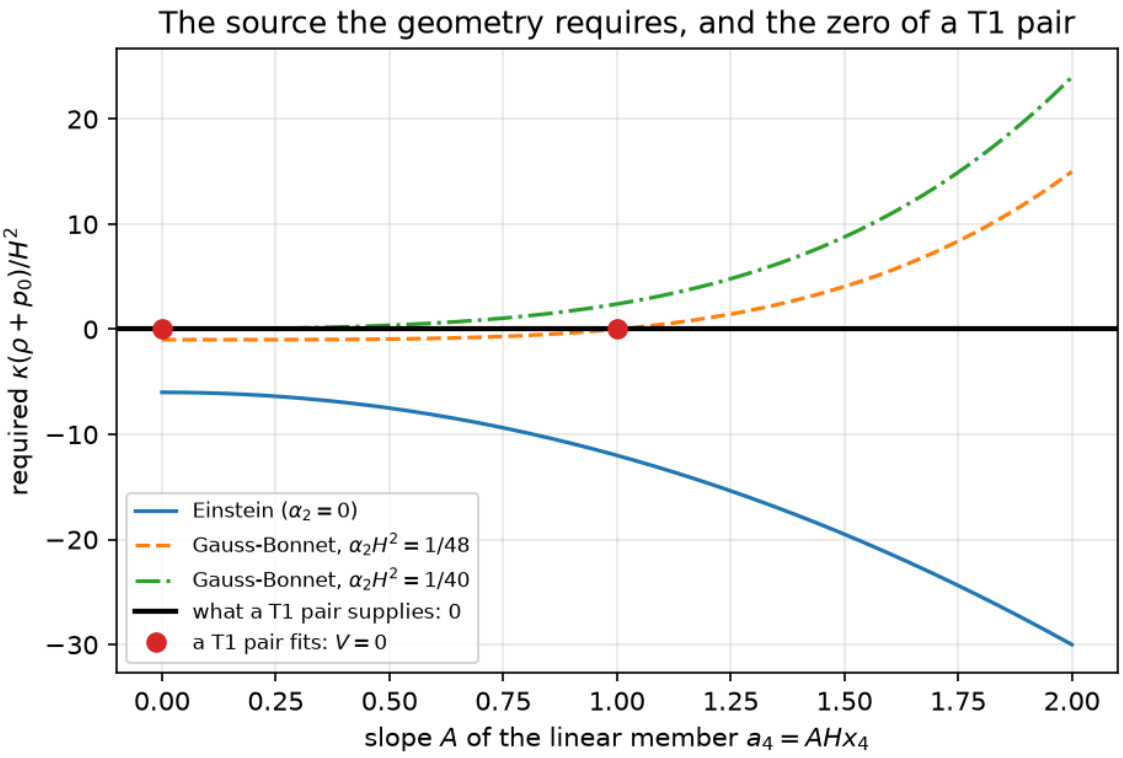

Figure 511: The null combination $\kappa(\rho + p_0)$ (energy density plus the pressure along x_0, times the gravitational coupling; vertical, units H^2) that the field equations require of ANY source of the linear member $a_4 = AHx_4$, against its slope A (horizontal), for Einstein gravity (solid) and two Einstein-Gauss-Bonnet theories with $\alpha_2 H^2 = 1/48$ (dashed) and $1/40$ (dash-dotted). A T1 pair supplies zero (black line). In Einstein gravity the requirement is $-6(A^2 + 1)H^2 < 0$ for every A, so a T1 pair can never be the only source; with the Gauss-Bonnet term the requirement vanishes at one slope (red dots: $A = 1$, the author's history, for $\alpha_2 H^2 = 1/48$, and $A = 0$ for $1/40$). (Notebook 20a, figure 5.)

```
$A^2 + 5 = 1/(8\alpha_2H^2)$; subtract 5,

$$A^2 = \frac{1 - 40\alpha_2H^2}{8\alpha_2H^2},$$

real and not negative exactly when $0 < \alpha_2H^2 \leq 1/40$. Then the $x_4$
equation fixes $\Lambda = -\sum_k\alpha_kE_{(k)}{}^{x_4}{}_{x_4}$. For the author's
history $A = 1$: $8\alpha_2H^2 \cdot 6 = 1$, so $\alpha_2H^2 = 1/48$. The next cell
checks the formula and then checks EXACTLY that with $\alpha_2 = 1/(48H^2)$ and
the resulting $\Lambda$ all four independent components of the vacuum equations
vanish for $a_4 = Hx_4$. In that theory a T1 pair CAN be the only source of the
author's deflating history; C1(a) is a statement about Einstein gravity.
```

In [16]:

```
V_gb = sp.expand(V.subs(alpha3, 0))
A_squared = sp.solve(V_gb, A ** 2)[0]
reproduces(sp.simplify(A_squared - (1 - 40 * alpha2 * H ** 2)
                       / (8 * alpha2 * H ** 2)) == 0,
           "Gauss-Bonnet vacuum: A^2 = (1 - 40 alpha2 H^2)/(8 alpha2 H^2)",
           A4_WL, "einstein_gauss_bonnet_vacuum_linear")
x2_author = sp.solve(A_squared.subs(H, 1) - 1, alpha2)[0]  # A = 1, H = 1
report("alpha2 H^2 for a vacuum with A = 1", x2_author)
gauss_bonnet = {key: (lovelock(1, key) + alpha2 * lovelock(2, key))
                .subs({ad1: H, ad2: 0, alpha2: x2_author / H ** 2})
                for key in KEYS}
Lam_author = sp.simplify(-gauss_bonnet["x4x4"])  # Lambda from the x4 equation
report("Lambda of that vacuum", Lam_author)
reproduces(x2_author == sp.Rational(1, 48) and Lam_author == -12 * H ** 2
           and all(sp.simplify(gauss_bonnet[key] + Lam_author) == 0 for key in KEYS),
           "a4 = H x4 solves the Gauss-Bonnet vacuum equations exactly",
           A4_PY, "einstein_gauss_bonnet_vacuum_linear")
zero_off = all(record["lovelockTensors"][f"E{k}"][key]["input"] == "0"
               for k in (1, 2, 3) for key in ("x4x0", "x0x4"))
check(zero_off, "the mixed components x4x0 and x0x4 of E_(1..3) are 0 in the record")
Lam_curve = sp.simplify(-(lovelock(1, "x4x4") + alpha2 * lovelock(2, "x4x4"))
                        .subs({ad1: A * H, ad2: 0}).subs(A ** 2, A_squared))
say(f"Lambda along the vacuum curve: {sp.factor(Lam_curve)}")
```

Out [16]:

```
PASS Gauss-Bonnet vacuum: A^2 = (1 - 40 alpha2 H^2)/(8 alpha2 H^2)
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    einstein_gauss_bonnet_vacuum_linear
RESULT alpha2 H^2 for a vacuum with A = 1 = 1/48
RESULT Lambda of that vacuum = -12*H**2
PASS a4 = H x4 solves the Gauss-Bonnet vacuum equations exactly
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_gauss_bonnet_vacuum_linear
PASS the mixed components x4x0 and x0x4 of E_(1..3) are 0 in the record
Lambda along the vacuum curve: 3*(3840*H**4*alpha2**2 - 192*H**2*alpha2 + 1)/(16*alpha2)
```

Text cell:

```
The next cell draws the Gauss-Bonnet vacua: on the left the slope
$A = \sqrt{(1 - 40\alpha_2H^2)/(8\alpha_2H^2)}$ against $\alpha_2H^2$, on the right
the cosmological constant $\Lambda$ that each vacuum needs. As $\alpha_2 \to 0$
(Einstein gravity) the slope grows without bound: Einstein gravity has no vacuum.
```

In [17]:

```
x_axis = np.linspace(0.004, 1 / 40, 400)  # alpha2 H^2
A_curve = np.sqrt((1 - 40 * x_axis) / (8 * x_axis))
Lam_numeric = sp.lambdify(alpha2, Lam_curve.subs(H, 1), "numpy")
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.3))
axes[0].plot(x_axis, A_curve, color="tab:purple")
axes[0].plot([1 / 48], [1.0], "o", color="tab:red", markersize=8,
             label="$A = 1$ at $\\alpha_2H^2 = 1/48$")
axes[0].set_ylabel("slope $A$ of the vacuum $a_4 = AHx_4$")
axes[1].plot(x_axis, Lam_numeric(x_axis), color="tab:purple")
axes[1].plot([1 / 48], [-12.0], "o", color="tab:red", markersize=8,
             label="$\\Lambda = -12H^2$ at $\\alpha_2H^2 = 1/48$")
axes[1].set_ylabel("$\\Lambda/H^2$ of that vacuum")
for ax in axes:
    ax.set_xlabel("Gauss-Bonnet coupling $\\alpha_2H^2$")
    ax.axvline(1 / 40, color="grey", linestyle=":")
    ax.legend(fontsize=8)
axes[0].set_title("no vacuum for $\\alpha_2 \\to 0$ (Einstein), $A = 0$ at $1/40$",
                  fontsize=9)
axes[1].set_title("the cosmological constant each vacuum needs", fontsize=9)
save_figure(fig, "gauss_bonnet_vacuum",
            "The vacua of Einstein-Gauss-Bonnet gravity ($\\alpha_1 = 1$, "
            "$\\alpha_3 = 0$) among the linear members $a_4 = AHx_4$ of the author's "
            "metric: the geometries in which a T1 pair could be the only source. "
            "Left: the slope $A = \\sqrt{(1 - 40\\alpha_2H^2)/(8\\alpha_2H^2)}$ "
            "against the coupling $\\alpha_2H^2$ (horizontal, up to the dotted "
            "limit $1/40$ where $A = 0$); it grows without bound as "
            "$\\alpha_2 \\to 0$, the Einstein limit, which has no vacuum. Right: "
            "the cosmological constant $\\Lambda/H^2$ that the same vacuum needs. "
            "Red dots: the author's deflating history $A = 1$ is a vacuum for "
            "$\\alpha_2H^2 = 1/48$ and $\\Lambda = -12H^2$ (checked exactly).")
```

Out [17]:

```
Figure 20a.6 saved as Revision/textbook/figures/20a_6_gauss_bonnet_vacuum.png
```

Text cell:

```
## 14. Third order: one straight line of vacua per slope

With the third Lovelock coupling as well, $V = 0$ reads
$1 - 40x - 8A^2x + 360y + 144A^2y + 72A^4y = 0$ in the variables
$x = \alpha_2H^2$ and $y = \alpha_3H^4$. For a FIXED slope $A$ this is linear in
$x$ and $y$: solving for $y$,
```

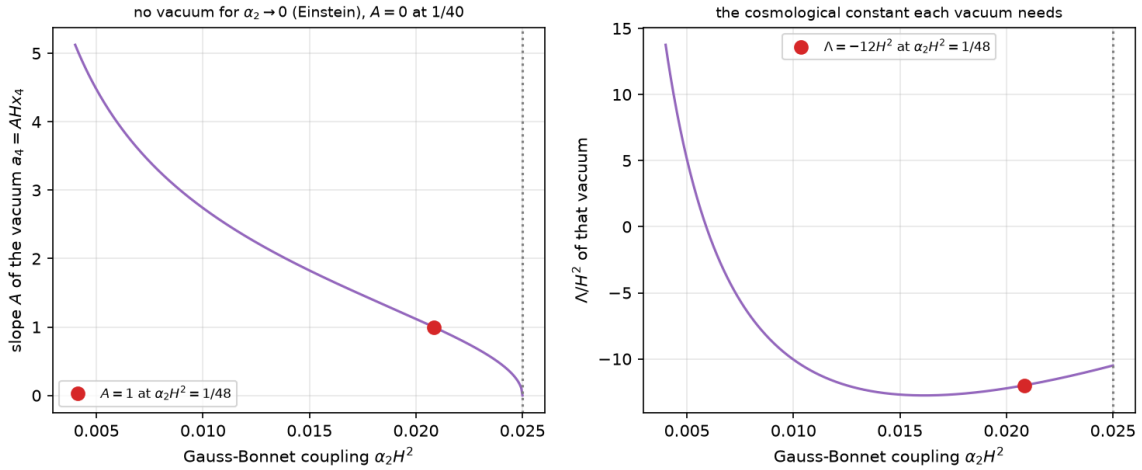

Figure 512: The vacua of Einstein-Gauss-Bonnet gravity ($\alpha_1 = 1$, $\alpha_3 = 0$) among the linear members $a_4 = AHx_4$ of the author's metric: the geometries in which a T1 pair could be the only source. Left: the slope $A = \sqrt{(1 - 40\alpha_2 H^2)/(8\alpha_2 H^2)}$ against the coupling $\alpha_2 H^2$ (horizontal, up to the dotted limit 1/40 where $A = 0$); it grows without bound as $\alpha_2 \to 0$, the Einstein limit, which has no vacuum. Right: the cosmological constant Λ/H^2 that the same vacuum needs. Red dots: the author's deflating history $A = 1$ is a vacuum for $\alpha_2 H^2 = 1/48$ and $\Lambda = -12H^2$ (checked exactly). (Notebook 20a, figure 6.)

```
$$y = \frac{(8A^2 + 40)x - 1}{72A^4 + 144A^2 + 360},$$

a straight line in the plane of the two couplings. Einstein gravity is the origin
$x = y = 0$, where $V = 1$: it lies on none of the lines. The next cell checks the
line formula with sympy and draws the lines for several slopes; the point
$(1/48, 0)$ of section 13 lies on the line $A = 1$.
```

In [18]:

```
y_line = sp.solve(V.subs({alpha2: sp.Symbol("x"), alpha3: sp.Symbol("y"), H: 1}),
                  sp.Symbol("y"))[0]
wanted = ((8 * A ** 2 + 40) * sp.Symbol("x") - 1) / (72 * A ** 4 + 144 * A ** 2 + 360)
check(sp.simplify(y_line - wanted) == 0, "for fixed A the vacua form a straight line")
check(V.subs({alpha2: 0, alpha3: 0}) == 1, "Einstein gravity (origin) has V = 1")
x_plot = np.linspace(-0.01, 0.05, 200)
fig, ax = plt.subplots(figsize=(7.0, 4.6))
for slope_A in (0.0, 0.5, 1.0, 2.0, 3.0):
    line = ((8 * slope_A ** 2 + 40) * x_plot - 1) / (
        72 * slope_A ** 4 + 144 * slope_A ** 2 + 360)
    ax.plot(x_plot, 1000 * line, label=f"$A = {slope_A:g}$")
ax.plot([0.0], [0.0], "s", color="black", markersize=8,
        label="Einstein gravity: no vacuum")
ax.plot([1 / 48], [0.0], "o", color="tab:red", markersize=8,
        label="section 13: $A = 1$, $\\alpha_3 = 0$")
ax.axhline(0.0, color="grey", linewidth=0.8)
ax.axvline(0.0, color="grey", linewidth=0.8)
ax.set_xlabel("$\\alpha_2H^2$")
```

```
ax.set_ylabel("$1000\\,\\alpha_3H^4$")
ax.set_title("Couplings for which $a_4 = AHx_4$ is a vacuum (third order)")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "third_order_vacua",
            "The Einstein-Lovelock couplings for which the linear member "
            "$a_4 = AHx_4$ of the author's metric is a vacuum (so that a T1 pair "
            "could be its only source): horizontal axis the second-order coupling "
            "$\\alpha_2H^2$, vertical axis the third-order coupling times 1000, "
            "$1000\\,\\alpha_3H^4$ ($\\alpha_1 = 1$). For each slope $A$ the "
            "condition $V = 0$ is a straight line (coloured, $A = 0$ to $3$). "
            "Einstein gravity is the origin (black square), where $V = 1$: it "
            "lies on no line. The red dot is the Gauss-Bonnet vacuum of the "
            "author's history $A = 1$ at $\\alpha_2H^2 = 1/48$.")
```

Out [18]:

```
PASS for fixed A the vacua form a straight line
PASS Einstein gravity (origin) has V = 1
```

Out [18]:

```
Figure 20a.7 saved as Revision/textbook/figures/20a_7_third_order_vacua.png
```

Text cell:

```
## 15. What C1 says, and what it does not say

- **PROVED (record, reproduced here):** the T1 partner $\Gamma\Psi$ with
  $(-m, -\lambda)$ carries minus the energy-momentum tensor and minus the current
  of $\Psi$ at every point, off and on shell; a T1 pair as the ONLY source is a
  zero source; in Einstein gravity the author's metric then has no solution for
  $H > 0$, whatever $a_4$ and $\Lambda$ (the two vacuum conditions never meet).
- **PROVED (record, reproduced here):** in Einstein-Lovelock gravity the linear
  member is a vacuum exactly when $V = 0$; with the Gauss-Bonnet coupling
  $\alpha_2H^2 = 1/48$ and $\Lambda = -12H^2$ the author's history $a_4 = Hx_4$ is
  a vacuum (checked exactly in section 13). So C1(a) depends on Einstein gravity.
- **C1 does not apply to T2 pairs:** the mirror copy carries the pulled-back,
  EQUAL tensor $R_0TR_0$, so the sources add (sections 9 and 10).
- **C1 does not apply to two independently quantised universes** (the record's
  statement Q3): their generators add without cancelling.
- **C1 is not a creation statement.** It concerns the sum of two classical sources
  in ONE given geometry. It derives no process that produces a pair, no rate, no
  amplitude, and it does not say that such a pair, or such a geometry, exists in
  nature. It says the opposite of "a pair creates the universe": in Einstein
  gravity a T1 pair alone cannot be the source of the author's metric at all.
- **ASSUMED:** the classical commuting field dirac16complex00 (for dirac16complex
  the record states the same as identities between Grassmann-algebra elements);
  the Z2 brane and the mirror patch (sections 9 and 10); the units $H = 1$ of the
  numerical examples.
```

Text cell:

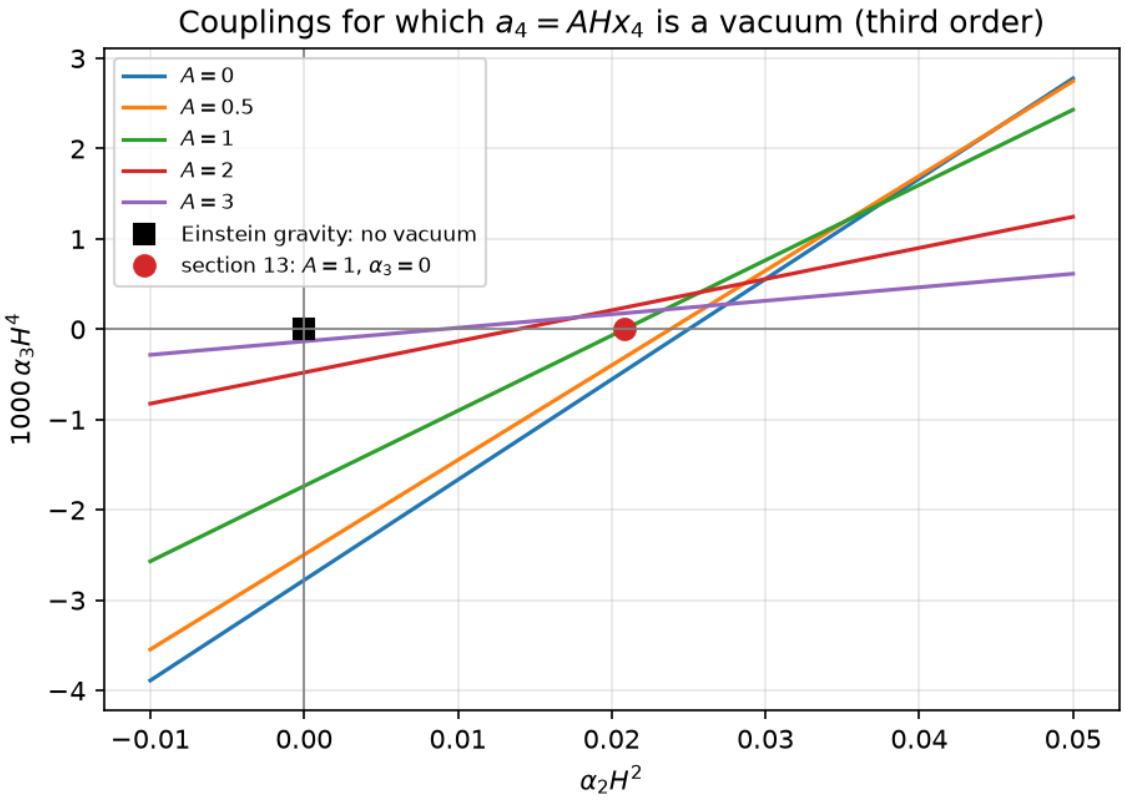

Figure 513: The Einstein-Lovelock couplings for which the linear member $a_4 = AHx_4$ of the author's metric is a vacuum (so that a T1 pair could be its only source): horizontal axis the second-order coupling $\alpha_2 H^2$, vertical axis the third-order coupling times 1000, $1000\,\alpha_3 H^4$ ($\alpha_1 = 1$). For each slope A the condition $V = 0$ is a straight line (coloured, $A = 0$ to 3). Einstein gravity is the origin (black square), where $V = 1$: it lies on no line. The red dot is the Gauss-Bonnet vacuum of the author's history $A = 1$ at $\alpha_2 H^2 = 1/48$. (Notebook 20a, figure 7.)

```
## 16. The last check

The last cell checks that the seven figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [19]:

```
figure_names = ["t1_pair_tensor", "pair_profiles", "t2_mirror_tensor",
                "einstein_vacuum_parabolas", "null_combination",
                "gauss_bonnet_vacuum", "third_order_vacua"]
paths = [output_file(f"{FIGURE_FOLDER}/20a_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [19]:

```
PASS every figure file of this notebook exists
ALL 37 CHECKS PASSED (notebook 20a)
```

Text cell:

```
## 17. What this notebook showed

- The canonical spin connection of the author's metric has the twelve components
  of the record, and $\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ on the patch
  ($-3H\gamma^{(x_0)}$ on the mirror patch, with its own positive vielbein).
- For an arbitrary configuration of dirac16complex00 at a point of the deflating
  history, its T1 partner $\Gamma\Psi$ with $(-m, -\lambda)$ has exactly the
  opposite energy-momentum tensor (64 components) and current: the pair's source
  is zero at every point (figures 1 and 2). The wrong partners do not cancel.
- The T2 mirror copy across the brane has the SAME energy density and charge
  density at the mirror point; its tensor is $R_0TR_0$ (figures 2 and 3): mirror
  pairs add, they do not cancel.
- With the pair as the only source, Einstein's equations for the author's metric
  reduce to $6a_4'^2 + 6H^2 = 0$ and $36H^2 + 2\Lambda = 0$; the only solutions
  are $a_4' = \pm iH$, not real (figure 4). The geometry requires
  $\kappa(\rho + p_0) = -6(a_4'^2 + H^2) < 0$ of every source; a T1 pair supplies
  zero (figure 5).
- In Einstein-Gauss-Bonnet gravity the linear member is a vacuum for
  $A^2 = (1 - 40\alpha_2H^2)/(8\alpha_2H^2)$; the author's history $A = 1$ for
  $\alpha_2H^2 = 1/48$, $\Lambda = -12H^2$ (figure 6); with the third coupling the
  vacua of each slope form a straight line that misses Einstein gravity
  (figure 7).
- None of this is a creation process: C1 compares sources in one given geometry.
```

20.11 Line-by-line walk-through of Notebook 20a

The notebook has 19 code cells, In [1] to In [19]. This section explains every line or small group of lines of every one of them, in order: a group of lines is quoted, then explained. Where a later cell

repeats a pattern already explained, we say so and explain only what is new. The markdown cells between the code cells are printed in the complete text above and need no explanation.

In [1], the set-up cell. Every line that starts with `#` is a **comment**, which Python skips. The comment lines at the top of the cell are the complete run instructions of Section 20.9, so that the notebook file carries its own instructions. The code starts after the line that announces THE SET-UP; it is the same in every notebook of this book except for the notebook's name, and it computes no physics.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module** (a part of Python or of an installed package) so that the code can use it. `json`, `os`, `textwrap` and `pathlib` come with Python itself. `from pathlib import Path` takes only the name `Path` out of `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs the Python code), which show a picture file below a cell.

```
NOTEBOOK_ID = "20a"  # this notebook: chapter 20, example a
```

A **variable** is a name for a value. This line gives the name `NOTEBOOK_ID` to the **string** (a piece of text in quotes) `"20a"`; the figure files and the last printed line are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs each time it is called. The text in triple quotes under the `def` line is its **docstring**: it says what the function does and is not executed.

`Path.cwd()` is the folder in which Jupyter runs the notebook, and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it; `[here, *here.parents]` is a list that starts with `here` and continues with them (the star unpacks one list into another). The `for` loop visits these folders one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that contains `Revision /textbook/requirements.txt` is the repository, and `return` hands it back. If no folder does, `raise` stops the notebook with a `FileNotFoundError` whose message says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The first code line calls the function and names its result `REPO`. The second chooses where files are written: `os.environ` holds the **environment variables** of the running program (named texts that it receives from the computer), and `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and `str(REPO)`, the repository folder as text, otherwise. When you run the notebook the variable is not set, so the figures go into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions. `repository_file` gives the full path of a file of the repository, for reading a Revision record. `output_file` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates the folder of that file and every missing folder above it, and does nothing if the folder exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters, the width of a line of the book; `textwrap.fill` breaks the text at blanks and starts every line after the first with four blanks. This is why some long PASS lines of the notebook continue on an indented second line.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to matplotlib's built-in settings, so that the figures are the same on every computer. `plt.rcParams.update({...})` then sets the default size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid (`grid.alpha` 0.3: 30 per cent opaque). The braces make a **dictionary**, a collection of pairs `key:  value`.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: a name in braces is replaced by its value, so `CAPTION_FILE` is the text `Revision/textbook/figures/20a.captions.json`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty JSON dictionary and a line end into the captions file; `encoding="utf-8"` fixes how the letters are stored and `newline="\n"` stores the same line end on every system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far: the figures are numbered 1, 2, 3, and a cell that is run twice keeps its number. The file name joins the notebook id, the number and the name, for example `20a_1_t1_pair_tensor.png`. `fig.savefig` writes a PNG file with 150 dots per inch, cuts away the empty margin, and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory; otherwise Jupyter would draw it a second time. The caption is stored, the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys), `display(Image(...))` shows the saved picture below the cell, and the last line prints where it was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (an ordered collection in square brackets). `check` is the function behind every check of the book: if `condition` is false, `raise AssertionError(...)` stops the notebook with an error that names the check; otherwise the name is appended to `PASSED` and the line PASS followed by the name is printed, and, when `record` is given, a second line that names the Revision record. `record=None` makes that argument optional.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds the unit only when one is given. `all_checks_passed` prints the last line of the notebook, with `len(PASSED)`, the number of checks that passed. The last statement prints the one output line of In [1], Set-up of notebook 20a complete.

In [2], the records and the helper `reproduces`.

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory
```

Two more modules of Python: `contextlib` provides `redirect_stdout`, which sends whatever `print` writes into another place, and `io` provides `StringIO`, a text buffer held in memory.

```
GAMMAS = "Revision/algebra/gammas.json"  # the author's gamma matrices
PROOFS = "Revision/docs/PAIR_CREATION_PROOFS.md"  # the pairing document
PAIR_PY = "Revision/pairing/reports/python-pairing.json"  # sympy pairing record
```

```
PAIR_WL = "Revision/pairing/reports/wolfram-pairing.json"  # Wolfram pairing record
A4_RECORD = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
A4_PY = "Revision/field_equations_a4/reports/python-a4-report.json"
A4_WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
LEAD_EGB = "Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json"
LEAD_EMT = "Revision/lead_checks/reports/emt-divergence-and-spin-connection.json"
```

Nine short names for the nine files of the repository that the notebook reads: the author's gammas, the Revision document of the pairing proofs, the two pairing reports, the record of the field equations for a_4 with its two reports, and the lead's two independent reports. Names written in capital letters are, by custom, values that the program does not change.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict ("PASS") of the check called name in a report; None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None
```

`read_json` reads a file of the repository as text and turns it into Python dictionaries and lists with `json.loads`. Every report holds, under the key `"checks"`, a list whose entries are dictionaries with a `"name"` and a `"verdict"`. `record_verdict` walks through that list and returns the verdict of the check with the given name, or `None` (Python's word for "nothing") if the report has no such check.

```
def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name,
              record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # both lines with one print call
```

`reproduces` is the check used whenever a result of the notebook reproduces a Revision record. `found` is true only if the named check of the named report has the verdict PASS. The `with` block sends the output of `check` into the buffer `lines`; `check` passes only if the notebook's own `conditio n` holds AND `found` is true, and it prints the PASS line and the line "reproduces" followed by the report and the check. The last line prints both lines at once (`end=""` adds no extra line end). If either condition fails, `check` raises its error inside the `with` block and the notebook stops.

```
proofs = repository_file(PROOFS).read_text(encoding="utf-8")
C1_WORDS = ("a T1 pair taken as the complete classical source of the author's "
            "metric is a zero source, and the Einstein equations then have no "
            "solution for")
say(f"{PROOFS} says: \"{C1_WORDS} H > 0.\"")
check(C1_WORDS in proofs, "the record states the corollary C1")
```

The first line reads the whole Revision document as one text. `C1_WORDS` is the record's sentence of the corollary C1 (two strings written next to each other inside parentheses are joined into one). `say` prints it with the name of the document; `\"` puts a double quote inside a string written with double quotes. The check `C1_WORDS in proofs` is true when the sentence occurs in the document, word for word: the first printed lines of the cell.

```
wolfram_counts = read_json(PAIR_WL)["summary"]  # passed, failed, total
sympy_counts = read_json(PAIR_PY)["counts"]  # pass, fail, pending
passed, total = wolfram_counts["passed"], wolfram_counts["total"]
say(f"pairing reports: Wolfram {passed} of {total} PASS, sympy "
    f"{sympy_counts["pass"]} PASS and {sympy_counts["fail"]} FAIL")
check(wolfram_counts["passed"] == wolfram_counts["total"] == 101
      and sympy_counts["pass"] == 66 and sympy_counts["fail"] == 0,
      "both pairing reports pass every check (101 and 66)")
```

The two pairing reports store their counts under different keys: `"summary"` with `"passed"` and `"total"` (Wolfram) and `"counts"` with `"pass"` and `"fail"` (sympy). `passed, total = a, b` gives two names at once. Inside the braces of an f-string Python 3.12 and newer allow the same kind of quotes as around the string, as in `sympy_counts["pass"]`. The check compares the counts with the numbers of the record (101 Wolfram checks, 66 sympy checks, no failure); `a == b == 101` is true only if both comparisons hold. If a later revision of the record changed these counts, this check would fail and show the change. Output: pairing reports: Wolfram 101 of 101 PASS, sympy 66 PASS and 0 FAIL.

In [3], the gammas, C and the chirality.

```
import itertools  # loops over all combinations of indices

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra
```

`itertools.product` (used below) runs over all combinations of indices. **numpy**, short `np`, computes with arrays of floating-point numbers (numbers with about 16 significant digits). **sympy**, short `sp`, computes exactly with whole numbers, fractions and symbols.

```
NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the frame metric eta, in the order x0..x7
I16 = sp.eye(16)  # the 16 by 16 unit matrix
fixture = read_json(GAMMAS)
gamma = [sp.Matrix([[sp.Rational(x) for x in row] for row in matrix])
         for matrix in fixture["gamma"]]  # gamma[a] = gamma^(x_a), exact
```

Python counts list positions from 0, and position k stands for x_k: position 0 for the hidden direction x_0, position 4 for the time x_4; `NAMES` holds the labels for the figures and `ETA` the diagonal of η in the same order. `sp.eye(16)` is the exact unit matrix I_{16}. The file `gammas.json` stores, under the key `"gamma"`, the eight matrices as lists of rows of whole numbers. The double **list comprehension** `[[... for x in row] for row in matrix]` builds, row by row, a list of lists in which every entry is turned into an exact sympy number by `sp.Rational`; `sp.Matrix` makes a sympy matrix of it; the outer comprehension does this for each of the eight matrices. The record lists the matrices in the order $x_0, \ldots, x_7$, so `gamma[0]` is $\gamma^{(x_0)}$ and `gamma[7]` is $\gamma^{(x_7)}$.

```
clifford = all(gamma[a] * gamma[b] + gamma[b] * gamma[a]
               == (2 * ETA[a] if a == b else 0) * I16
               for a in range(8) for b in range(8))
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g^(x0) g^(x1) g^(x2) g^(x3)
reproduces(clifford and C == C.T and C * C == I16
           and all((C * g).T == -(C * g) for g in gamma),
           "Clifford relation; C real symmetric, C^2 = 1, C gamma antisymmetric",
           PAIR_PY, "gammas.C")
```

`*` between sympy matrices is the matrix product. `all(...)` is true when the comparison holds for all 64 pairs (a, b): the anticommutator $\gamma^{(a)}\gamma^{(b)} + \gamma^{(b)}\gamma^{(a)}$ must equal $2\eta_{aa}I_{16}$ when $a = b$ and the zero matrix otherwise (`x if condition else y` chooses one of two values). The next line builds C from the record's definition. The check requires the Clifford relation, $C^T = C$ (`.T` is the transpose; the entries are real, so C is real symmetric), $C^2 = I_{16}$, and that every $C\gamma^{(a)}$ is antisymmetric; it reproduces `python-pairing.json`, check `gammas.C`.

```
chirality = gamma[0]  # the product g^(x0) g^(x1) ... g^(x7), built factor by factor
for a in range(1, 8):
    chirality = chirality * gamma[a]
reproduces(chirality == sp.diag(*([-1] * 8 + [1] * 8)),
           "Gamma = diag(-I8, I8)", PAIR_PY, "gammas.Gamma")
reproduces(all(chirality * g == -g * chirality for g in gamma)
           and chirality * C == C * chirality,
           "Gamma anticommutes with every gamma and commutes with C",
           PAIR_WL, "Gamma_properties")
```

The loop multiplies $\gamma^{(x_0)}$ from the right by $\gamma^{(x_1)}, \ldots, \gamma^{(x_7)}$, in the record's order. `[-1] * 8 + [1] * 8` is the list of eight entries -1 followed by eight entries $+1$, and `sp.diag(*list)` is the diagonal matrix with these entries: the check is $\Gamma = \mathrm{diag}(-I_8, I_8)$. The second check is steps 1 and 2 of Section 20.5: $\Gamma\gamma^{(a)} = -\gamma^{(a)}\Gamma$ for all eight gammas and $\Gamma C = C\Gamma$.

```
S_exact = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
           for a in range(8)]  # the generators S^ab
to_numpy = lambda matrix: np.array(matrix.tolist(), dtype=float)  # noqa: E731
gamma_num = [to_numpy(g) for g in gamma]  # floating-point copies
C_num, Gamma_num = to_numpy(C), to_numpy(chirality)
S_num = [[to_numpy(S_exact[a][b]) for b in range(8)] for a in range(8)]
B_num = -1j * C_num @ gamma_num[4]  # B = -i C gamma^(x4): J^x4 = Psi^dagger B Psi
```

`S_exact[a][b]` is the generator $S^{ab} = \frac{1}{4}[\gamma^{(a)}, \gamma^{(b)}]$, exactly, for all 64 pairs. `lambda matrix: ...` is a one-line function: it turns a sympy matrix into a numpy array of floating-point numbers (`tolist()` gives the list of rows); the comment `noqa: E731` tells a style checker that the one-line form is intended. The next lines make floating-point copies of the gammas, of C, of Γ and of the S^{ab}. In numpy, `@` is the matrix product and `1j` is the imaginary unit i; `B_num` is $B = -iC\gamma^{(x_4)}$ (position 4 is x_4). The cell prints the three PASS lines with their records.

In [4], the spin connection on both patches.

```
H = sp.symbols("H", positive=True)  # the author's constant H > 0
z = sp.symbols("z", real=True)  # the hidden coordinate z = 6 H x0
```

```
x4 = sp.symbols("x4", real=True)  # the time
a4 = sp.Function("a4")(x4)  # the metric function a4(x4), arbitrary
a4_value, slope = sp.symbols("a4_value slope", real=True)  # a4 and a4' at a point
```

`sp.symbols` makes sympy symbols, letters that sympy computes with; `positive=True` and `real=Tru e` tell sympy what kind of number they stand for, which lets it simplify correctly (for example $\sqrt{H^2} = H$). `sp.Function("a4")(x4)` is an unknown function $a_4(x_4)$: sympy can differentiate it without knowing it. The last two symbols will replace a_4 and a_4' at a point.

```
def lengths(s0):
    """The lengths E_a of the diagonal vielbein; s0 = +1 on the patch, -1 on the
    mirror patch (where cot z < 0, so -cot z is the positive length)."""
    s = sp.sin(z) ** sp.Rational(1, 6)  # the warp sin(z)^(1/6) of a length
    return ([s0 * sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
            + [sp.exp(-a4) * s] * 3)  # x0, x1, ..., x7
```

The lengths E_a (written f_a in Sections 20.5 and 20.6) of the diagonal vielbein, in the order $x_0, \ldots, x_7$: $s_0 \cot z$ for the hidden direction, three times $e^{a_4} \sin^{1/6} z$ for the inflating 3-space, 1 for the time, and three times $e^{-a_4} \sin^{1/6} z$ for the DEFLATING extra times (on the background $b_4 = 1$). `**` is the power and `sp.Rational(1, 6)` the exact fraction 1/6. `[x] * 3` is a list of three copies of `x`, and `+` joins lists. On the mirror patch $\cot z$ is negative, so the positive length there is $-\cot z$: the argument `s0` is $+1$ on the patch and -1 on the mirror patch.

```
def d(expr, mu):
    """The partial derivative along the coordinate at position mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # d/dx0 = 6 H d/dz
    return sp.Integer(0)  # nothing depends on x1, x2, x3, x5, x6, x7
```

The partial derivative ∂_μ of an expression: along x_4 (position 4) it is `sp.diff(expr, x4)`; along x_0 (position 0) the chain rule gives $\partial/\partial x_0 = (dz/dx_0)\, \partial/\partial z = 6H\, \partial/\partial z$; along the other six directions it is zero, because the metric does not depend on them.

```
def spin_connection(s0):
    """The lengths E_a and omega[mu][a][b] = omega_mu ab at a point (a4 and a4'
    replaced by the symbols a4_value and slope)."""
    E = lengths(s0)
    g = [ETA[a] * E[a] ** 2 for a in range(8)]  # the metric g_aa
    christoffel = {}
    for a, b, c in itertools.product(range(8), repeat=3):
        value = 0  # the formula of the markdown cell, term by term
        if a == c:
            value += d(g[a], b)
        if a == b:
            value += d(g[a], c)
        if b == c:
            value -= d(g[b], a)
        christoffel[a, b, c] = value / (2 * g[a])
```

The diagonal metric is $g_{aa} = \eta_{aa} E_a^2$. `itertools.product(range(8), repeat=3)` runs over all $8^3 = 512$ triples (a, b, c). For each, the loop builds the Christoffel symbol of a diagonal metric term by term, $\Gamma^a{}_{bc} = (\delta_{ac}\partial_b g_{aa} + \delta_{ab}\partial_c g_{aa} - \delta_{bc}\partial_a g_{bb})/(2g_{aa})$: each `if` adds one term exactly when its Kronecker delta is 1, and `+=` and `-=` add to or subtract from `value`. The dictionary `christoffel` stores the result under the key `(a, b, c)`.

```
    omega = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
    for mu, a, b in itertools.product(range(8), repeat=3):
        if a != b:
            value = ETA[a] * E[a] * christoffel[a, mu, b] / E[b]
            value = value.subs(sp.Derivative(a4, x4), slope).subs(a4, a4_value)
            omega[mu][a][b] = sp.simplify(value)
    return [length.subs(a4, a4_value) for length in E], omega
```

`omega` starts as an $8 \times 8 \times 8$ nest of zeros (`_` is a loop name whose value is not used). For $a \neq b$ the component is $\omega_{\mu ab} = \eta_{aa}(E_a/E_b)\Gamma^a{}_{\mu b}$, the formula derived in the markdown cell above the code; for $a = b$ it is zero. `.subs(old, new)` substitutes: first a_4' (sympy's `Derivative(a4, x4)`) by the symbol `slope`, then a_4 by `a4_value`, so that the components become expressions in numbers that can be chosen at a point. `sp.simplify` writes each one in its simplest form. The function returns the lengths (with the same substitution) and the components.

```
E_patch, omega_patch = spin_connection(1)
E_mirror, omega_mirror = spin_connection(-1)
for label, omega in (("patch", omega_patch), ("mirror patch", omega_mirror)):
    count = sum(1 for mu in range(8) for a in range(8) for b in range(a + 1, 8)
                if omega[mu][a][b] != 0)
    say(f"{label}: {count} nonzero components omega_mu ab with a < b")
```

The connection is computed on both patches. For each, `sum(1 for ... if ...)` counts the nonzero components with $a < b$ (the others follow from the antisymmetry $\omega_{\mu ba} = -\omega_{\mu ab}$). Output: 12 on the patch and 12 on the mirror patch.

In [5], the twelve components, $\gamma^\mu \Omega_\mu$ and the mirror isometry.

```
s = sp.sin(z) ** sp.Rational(1, 6)
u, v = sp.exp(a4_value) * s, sp.exp(-a4_value) * s
expected = {}  # (mu, a, b) -> the record's component, on the patch
for i in (1, 2, 3):  # the inflating 3-space directions
    expected[i, i, 4], expected[i, 0, i] = slope * u, -H * u
for t in (5, 6, 7):  # the deflating extra times
    expected[t, 4, t], expected[t, 0, t] = -slope * v, H * v
```

`u` and `v` are the two lengths $e^{a_4} \sin^{1/6} z$ and $e^{-a_4} \sin^{1/6} z$ at a point. The dictionary `expected` holds the record's twelve components: for each inflating direction i, $\omega_{x_i, x_i x_4} = a_4' u$ and $\omega_{x_i, x_0 x_i} = -Hu$; for each deflating extra time t, $\omega_{x_t, x_4 x_t} = -a_4' v$ and $\omega_{x_t, x_0 x_t} = Hv$ (with $a < b$ in the order $x_0, \ldots, x_7$, so the hidden index comes first, and $\omega_{\mu, ba} = -\omega_{\mu, ab}$).

```
def components(omega):
    """The nonzero omega_mu ab with a < b, as a dictionary."""
```

```
    return {(mu, a, b): omega[mu][a][b] for mu in range(8) for a in range(8)
            for b in range(a + 1, 8) if omega[mu][a][b] != 0}


def same_components(found, wanted):
    return set(found) == set(wanted) and all(
        sp.simplify(found[key] - wanted[key]) == 0 for key in wanted)
```

`components` collects the nonzero components with $a < b$ into a dictionary (a **dictionary comprehension**). `same_components` is true when two such dictionaries have the same keys (`set(...)` is the collection of keys without order) and every difference simplifies to zero.

```
reproduces(same_components(components(omega_patch), expected),
           "the twelve spin-connection components of the record (patch)",
           PAIR_PY, "geometry.spin_connection_components")
flipped = {key: (-value if 0 in key[1:] else value)
           for key, value in expected.items()}  # x0 components change sign
reproduces(same_components(components(omega_mirror), flipped),
           "mirror patch: the x0 components change sign, the others do not",
           PAIR_WL, "connection_mirror_patch")
```

The first check compares the patch with the record. On the mirror patch the length $E_0 = -\cot z$ has the opposite sign, so every component with a frame index x_0 changes sign: `flipped` is `expected` with the sign reversed wherever the position 0 occurs among the two frame indices `key[1:]`. The second check compares the mirror patch with it.

```
Omega = {}  # s0 -> the eight matrices Omega_mu
for s0, omega in ((1, omega_patch), (-1, omega_mirror)):
    Omega[s0] = [sum((omega[mu][a][b] * S_exact[a][b] / 2 for a in range(8)
                      for b in range(8) if omega[mu][a][b] != 0), sp.zeros(16))
                 for mu in range(8)]
term = {}  # s0 -> sum over mu of gamma^mu Omega_mu
for s0, E in ((1, E_patch), (-1, E_mirror)):
    term[s0] = sum((gamma[mu] / E[mu] * Omega[s0][mu] for mu in range(8)),
                   sp.zeros(16)).applyfunc(sp.simplify)
```

For each patch, `Omega[s0][mu]` is the matrix $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$; `sum(..., sp.zeros(16))` adds matrices starting from the 16×16 zero matrix. `term[s0]` is $\sum_\mu \gamma^\mu\Omega_\mu$ with $\gamma^\mu = \gamma^{(\mu)}/E_\mu$; `.applyfunc(sp.simplify)` simplifies each of its 256 entries.

```
reproduces(term[1] == 3 * H * gamma[0], "patch: gamma^mu Omega_mu = 3 H gamma^(x0)",
           LEAD_EMT, "gamma_Omega_equals_3H_gamma0")
check(term[-1] == -3 * H * gamma[0], "mirror patch: gamma^mu Omega_mu = -3 H gamma^(x0)")
g_patch = [ETA[a] * E_patch[a] ** 2 for a in range(8)]
reproduces(all(sp.simplify(entry.subs(z, sp.pi - z) - entry) == 0
               for entry in g_patch),
           "z -> pi - z leaves all eight metric components unchanged",
           PAIR_PY, "geometry.mirror_isometry")
```

On the patch the sum is $3H\gamma^{(x_0)}$, the record's spin-connection term, with no a_4 or a_4' left: the terms of the three inflating and the three deflating directions along $\gamma^{(x_4)}$ cancel. On the mirror patch it is $-3H\gamma^{(x_0)}$; this check has no record (a computation of this notebook). The last check substitutes $z \to \pi - z$ into the eight metric components and finds each unchanged: the mirror is an isometry.

In [6], the geometry at a point and the bilinears.

```
point_symbols = (z, a4_value, slope, H)
NUMERIC = {}  # s0 -> (the function of the lengths, the connection pieces)
for s0, E, omega in ((1, E_patch, omega_patch), (-1, E_mirror, omega_mirror)):
    lengths_f = sp.lambdify(point_symbols, E, "numpy")
    pieces = [(mu, a, b, sp.lambdify(point_symbols, omega[mu][a][b], "numpy"))
              for mu in range(8) for a in range(8) for b in range(8)
              if omega[mu][a][b] != 0]
    NUMERIC[s0] = (lengths_f, pieces)
```

`sp.lambdify(symbols, expression, "numpy")` turns a sympy expression into an ordinary Python function of the listed symbols that computes with numpy. For each patch, `lengths_f` computes the eight lengths and `pieces` is a list of the nonzero components (μ, a, b), each with its own function; they are stored under the key $+1$ or -1.

```
def geometry_at(s0, z_value, a4_number, slope_number, H_number=1.0):
    """Lengths, metric, Omega_mu and the curved gammas at one point."""
    lengths_f, pieces = NUMERIC[s0]
    values = (z_value, a4_number, slope_number, H_number)
    E = np.array([float(x) for x in lengths_f(*values)])
    Om = [np.zeros((16, 16)) for _ in range(8)]
    for mu, a, b, f in pieces:  # Omega_mu = (1/2) sum omega_mu ab S^ab
        Om[mu] += float(f(*values)) * S_num[a][b] / 2
    return {"g": np.array(ETA) * E ** 2, "Omega": Om,
            "up": [gamma_num[mu] / E[mu] for mu in range(8)],
            "down": [ETA[mu] * E[mu] * gamma_num[mu] for mu in range(8)]}
```

`geometry_at` evaluates the geometry at one point: the patch (`s0`), z, a_4, a_4' and H (1 unless given). `f(*values)` calls a function with the four numbers as its arguments. It returns a dictionary with the metric $g_{\mu\mu}$, the eight matrices Ω_μ (built from the pieces), the curved gammas with an upper index $\gamma^\mu = \gamma^{(\mu)}/E_\mu$ and with a lower index $\gamma_\mu = g_{\mu\mu}\gamma^\mu = \eta_{\mu\mu}E_\mu\gamma^{(\mu)}$.

```
def bilinears(at, psi, dpsi, m, lam):
    """S, K, L/sqrt|g|, T_mu nu, J^mu, the field-equation residual, and the
    largest imaginary part of a quantity that must be real."""
    bar = psi.conj() @ C_num  # Psibar = Psi^dagger C (a row)
    dbar = [row.conj() @ C_num for row in dpsi]  # d_mu Psibar
    D = [dpsi[mu] + at["Omega"][mu] @ psi for mu in range(8)]  # D_mu Psi
    Dbar = [dbar[mu] - bar @ at["Omega"][mu] for mu in range(8)]  # D_mu Psibar
    S = bar @ psi
    K = 0.5 * sum(bar @ at["up"][mu] @ D[mu] - Dbar[mu] @ at["up"][mu] @ psi
                  for mu in range(8))
    L = K - m * S - lam / 2 * S ** 2  # L / sqrt|g|
```

`bilinears` evaluates, from the first jet of a configuration (the 16 values `psi` and the eight derivative columns `dpsi[mu]`), every quantity of Section 20.5. `psi.conj()` is the complex conjugate of each entry; for a one-dimensional numpy array the product `psi.conj() @ C_num` is the row $\Psi^\dagger C = \bar\Psi$. Then $\partial_\mu\bar\Psi$, $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$, the scalar S, the kinetic term K and $\mathcal{L}/\sqrt{|g|} = K - mS - \frac{\lambda}{2}S^2$.

```
    T = np.zeros((8, 8), dtype=complex)
    for mu, nu in itertools.product(range(8), repeat=2):
        T[mu, nu] = 0.25 * (bar @ at["down"][mu] @ D[nu]
                            + bar @ at["down"][nu] @ D[mu]
                            - Dbar[mu] @ at["down"][nu] @ psi
                            - Dbar[nu] @ at["down"][mu] @ psi)
        if mu == nu:
            T[mu, nu] -= at["g"][mu] * L
    J = np.array([-1j * bar @ at["up"][mu] @ psi for mu in range(8)])
    residual = sum(at["up"][mu] @ D[mu] for mu in range(8)) - (m + lam * S) * psi
```

The 64 components of $T_{\mu\nu}$, term by term as in the formula of Section 20.5 (the metric is diagonal, so the term $-g_{\mu\nu}\mathcal{L}/\sqrt{|g|}$ appears only for $\mu = \nu$); the eight components of the current $J^\mu = -i\bar\Psi\gamma^\mu\Psi$; and the field-equation residual $E_{m,\lambda} = \gamma^\mu D_\mu\Psi - (m + \lambda S)\Psi$, a column of 16 numbers that is zero exactly when the configuration solves the field equation.

```
    imaginary = max(abs(S.imag), abs(K.imag), np.abs(T.imag).max(),
                    np.abs(J.imag).max())
    return {"S": S.real, "K": K.real, "L": L.real, "T": T.real, "J": J.real,
            "E": residual, "imag": imaginary}
```

S, K, $T_{\mu\nu}$ and J^μ must be real numbers (Chapter 7 proves it); in floating point they come out with tiny imaginary parts of rounding size. `imaginary` is the largest of them, so that the next cell can check it; the function returns the real parts and the residual. The cell prints nothing.

In [7], a configuration and its T1 partner at one point.

```
rng = np.random.default_rng(20261007)  # a fixed seed: the same numbers every run
R0_SIGNS = [-1, 1, 1, 1, 1, 1, 1, 1]  # the mirror reverses the direction x0
psi = rng.normal(size=16) + 1j * rng.normal(size=16)  # the 16 complex values
dpsi = [rng.normal(size=16) + 1j * rng.normal(size=16) for _ in range(8)]
dpsi = dpsi[7:] + dpsi[:7]  # drawn hidden direction last, as before; now x0 first
Z0, A4_0, M0, LAM0 = 0.6, 0.5, 2.0, 0.5  # the point and the parameters (H = 1)
```

`np.random.default_rng(20261007)` is a random-number generator started from a fixed **seed**, so that every run draws the same numbers and the notebook is reproducible. `rng.normal(size=16)` draws 16 numbers from the bell-shaped normal distribution; real part plus i times another 16 such numbers gives 16 random complex values Ψ, and the comprehension draws eight more such columns, the derivatives $\partial_\mu\Psi$. They are drawn in the order in which this notebook drew them before it moved to the author's coordinates, with the hidden direction last; `dpsi[7:] + dpsi[:7]` moves that last column to the front, so that `dpsi[mu]` is $\partial_\mu\Psi$ for x_μ and every number of the notebook stays the same. `R0_SIGNS` is the diagonal of R_0, used later. The point is $z = 0.6$, $a_4 = 0.5$, and the parameters are $m = 2$, $\lambda = 1/2$, in units $H = 1$.

```
at = geometry_at(1, Z0, A4_0, 1.0)  # a4' = 1: the deflating history A = 1
one = bilinears(at, psi, dpsi, M0, LAM0)  # Psi with (m, lambda)
partner = bilinears(at, Gamma_num @ psi, [Gamma_num @ row for row in dpsi],
                    -M0, -LAM0)  # Gamma Psi with (-m, -lambda)
scale = np.abs(one["T"]).max()  # the size of the tensor
report("S of the configuration", f"{one["S"]:.6f}")
report("largest |T_mu nu| of the configuration", f"{scale:.4f}")
```

The geometry at the point on the patch with $a_4' = 1$, the rate of the deflating history. `one` holds the quantities of the configuration with (m, λ); `partner` those of $\Gamma\Psi$ (value and derivatives multiplied by Γ, because $\partial_\mu(\Gamma\Psi) = \Gamma\partial_\mu\Psi$) with $(-m, -\lambda)$. `scale` is the largest modulus of a component of T; every tolerance below is measured relative to it. `:.6f` prints a number with six digits after the point. Output: RESULT S of the configuration = 1.354934 and RESULT largest modulus of $T_{\mu\nu}$ = 26.7625. The configuration is random: it is NOT a solution, and T1 must hold anyway, because it is an off-shell identity.

```
check(max(one["imag"], partner["imag"]) < 1e-12 * scale,
      "S, K, T and J are real numbers (to rounding)")
reproduces(abs(partner["S"] - one["S"]) < 1e-12 and abs(partner["K"] + one["K"])
           < 1e-12, "S' = S and K' = -K", PAIR_PY, "T1.metric.commuting.S_invariant")
reproduces(np.abs(one["T"] + partner["T"]).max() < 1e-12 * scale,
           "T1 pair: all 64 components of T + T' vanish",
           PAIR_PY, "T1.metric.commuting.pair_total_emt_zero")
reproduces(np.abs(one["J"] + partner["J"]).max() < 1e-12 * scale,
           "T1 pair: all 8 components of J + J' vanish",
           PAIR_WL, "T1_current_primordial_commuting")
reproduces(np.abs(partner["E"] + Gamma_num @ one["E"]).max() < 1e-12 * scale,
           "E_(-m,-lambda)[Gamma Psi] = -Gamma E_(m,lambda)[Psi]",
           PAIR_PY, "T1.metric.commuting.euler_lagrange_map")
```

Five checks of theorem T1 at this point, each to 10^{-12} times the size of the tensor, which allows only rounding errors: the imaginary parts are rounding; $S' = S$ and $K' = -K$ (step 5 of Section 20.5); all 64 components of $T + T'$ vanish (T1c); all eight components of $J + J'$ vanish; and the residuals obey $E_{-m,-\lambda}[\Gamma\Psi] = -\Gamma E_{m,\lambda}[\Psi]$ (step 7), so one vanishes exactly when the other does.

```
wrong_1 = bilinears(at, Gamma_num @ psi, [Gamma_num @ row for row in dpsi],
                    -M0, LAM0)  # only the mass reversed
wrong_2 = bilinears(at, Gamma_num @ psi, [Gamma_num @ row for row in dpsi],
                    M0, LAM0)  # nothing reversed
reproduces(min(np.abs(one["T"] + wrong_1["T"]).max(),
               np.abs(one["T"] + wrong_2["T"]).max()) > 1e-3 * scale,
           "negative controls: with (-m, +lambda) or (m, lambda) no cancellation",
           PAIR_PY, "T1.metric.commuting.negative_controls")
```

Two **negative controls**, computations that must FAIL to cancel: the partner $\Gamma\Psi$ with only the mass reversed, and with nothing reversed. In both cases the sum $T + T'$ is far from zero (more than 10^{-3} times the size of the tensor). This shows that the cancellation of the previous check is not an accident of the test: the mass AND the coupling must change sign.

In [8], figure 1: the T1 pair cancels.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.3))
panels = ((one["T"], "$T_{\\mu\\nu}[\\Psi;\\,m,\\lambda]$"),
          (partner["T"], "$T_{\\mu\\nu}[\\Gamma\\Psi;\\,-m,-\\lambda]$"),
          (one["T"] + partner["T"], "sum: the source of the pair"))
```

`plt.subplots(1, 3, ...)` makes a figure with one row of three panels (**axes**), 12 by 4.3 inches. `panels` pairs each table with its title; the titles are written in LaTeX, and inside a Python string a backslash is written twice, so `\\mu` reaches matplotlib as `\mu`.

```
for ax, (values, title) in zip(axes, panels):
    image = ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)  # no grid lines across the coloured squares
    ax.set_title(title, fontsize=10)
fig.colorbar(image, ax=list(axes), shrink=0.8, label="value (units $H = 1$)")
```

`zip` walks through the panels and the tables together. `ax.imshow` draws an 8×8 table as coloured squares (a **heat map**): the colour map `RdBu_r` runs from blue (negative) through white (zero) to red (positive), and `vmin=-scale, vmax=scale` uses the same scale in all three panels, so that equal colours mean equal numbers. The tick lines put the labels x_0 to x_7 on both axes; `ax.grid(False)` switches off the grid lines. `fig.colorbar` draws the colour scale beside the panels.

```
save_figure(fig, "t1_pair_tensor",
            "The energy-momentum tensor $T_{\\mu\\nu}$ (rows $\\mu$, columns $\\nu$, "
            ...
            "says, although the configuration is not even a solution.")
```

(The caption string, nine lines, is shown here by its first and last line with `...` between them; it is printed in full in the complete text of the notebook and under the figure. The same is done for every caption below.) `save_figure` saves the figure as `20a_1_t1_pair_tensor.png` and shows it.
What figure 1 shows. Three heat maps of the 64 components $T_{\mu\nu}$ (rows μ, columns ν, from x_0 to x_7; colour scale in units $H = 1$): left the random configuration, middle its T1 partner, right their sum. The student should see that the middle map is the left one with every colour reversed (red becomes blue of the same depth), and that the right map is white everywhere: the source of the T1 pair is zero, component by component, even though the configuration does not solve the field equation. This is theorem T1 (T1c and T1d) at one point of the deflating history.

In [9], along the hidden direction: the T1 partner and the T2 mirror copy.

```
psi_1 = rng.normal(size=16) + 1j * rng.normal(size=16)  # the spinor Psi_1
z_patch = np.linspace(0.05, np.pi / 2 - 0.02, 160)  # points of the patch
rho, charge = {"one": [], "T1": [], "T2": []}, {"one": [], "T1": [], "T2": []}
```

A second random spinor Ψ_1 (the generator continues where it stopped, so the numbers are new but fixed). `np.linspace(a, b, n)` gives n equally spaced numbers from a to b: 160 points of the patch from $z = 0.05$ to just below the brane. `rho` and `charge` are dictionaries of empty lists, one list for each of the three objects.

```
for z_value in z_patch:
    field = psi + 0.3 * (np.pi / 2 - z_value) ** 2 * psi_1  # Psi(z)
    slopes_here = [row.copy() for row in dpsi]  # the derivatives along x1 .. x7
    slopes_here[0] = -3.6 * (np.pi / 2 - z_value) * psi_1  # 6 H dPsi/dz, H = 1
    mirror_slopes = [R0_SIGNS[mu] * (gamma_num[0] @ slopes_here[mu])
                     for mu in range(8)]  # the mirror reverses d/dx0
```

At each point the field is $\Psi(z) = \Psi_0 + \frac{3}{10}(\pi/2 - z)^2\Psi_1$. Its derivatives along $x_1, \ldots, x_7$ are those of In [7] (`row.copy()` makes independent copies); its derivative along x_0 is $6H\, d\Psi/dz = 6 \cdot \frac{3}{10} \cdot 2(\pi/2 - z)(-1)\Psi_1 = -3.6(\pi/2 - z)\Psi_1$ (chain rule, $H = 1$), which vanishes at the brane and keeps the term $\tan z\, \gamma^{(x_0)}\partial_0\Psi$ finite there. The mirror copy has the derivatives $\gamma^{(x_0)}\partial_\mu\Psi$, with the sign of the x_0 derivative reversed, because the mirror $x_0 \to \pi/(6H) - x_0$ reverses that direction.

```
    here = geometry_at(1, z_value, A4_0, 1.0)
    there = geometry_at(-1, np.pi - z_value, A4_0, 1.0)  # the mirror point
    results = {
        "one": bilinears(here, field, slopes_here, M0, LAM0),
        "T1": bilinears(here, Gamma_num @ field,
                        [Gamma_num @ row for row in slopes_here], -M0, -LAM0),
        "T2": bilinears(there, gamma_num[0] @ field, mirror_slopes, -M0, LAM0),
    }
    for key, result in results.items():
        rho[key].append(-result["T"][4, 4])  # rho = -T_x4x4
        charge[key].append(result["J"][4])  # J^x4
```

The geometry at the point z of the patch and at the mirror point $\pi - z$ of the mirror patch. The three objects: the field with (m, λ), its T1 partner with $(-m, -\lambda)$ at the same point, and its T2 mirror copy $\gamma^{(x_0)}\Psi$ with $(-m, \lambda)$ at the mirror point. For each, the energy density $\rho = -T_{x_4x_4}$ (position 4, 4; the notebook computes the tensor $T_{\mu\nu}$ of the pairing records; because $g^{x_4x_4} = -1$, $T^{x_4}{}_{x_4} = g^{x_4x_4}T_{x_4x_4} = -T_{x_4x_4}$, so $-T_{x_4x_4} = +T^{x_4}{}_{x_4}$ in the pairing convention, which is $-T^{x_4}{}_{x_4}$ in the field-theory convention of Section 20.2, that is ρ) and the charge density J^{x_4} are appended to their lists.

```
rho = {key: np.array(values) for key, values in rho.items()}
charge = {key: np.array(values) for key, values in charge.items()}
size = max(np.abs(rho["one"]).max(), np.abs(charge["one"]).max())
check(np.abs(rho["T1"] + rho["one"]).max() < 1e-12 * size
      and np.abs(charge["T1"] + charge["one"]).max() < 1e-12 * size,
      "along z: the T1 partner has -rho and -J^x4 at every point")
check(np.abs(rho["T2"] - rho["one"]).max() < 1e-10 * size
      and np.abs(charge["T2"] - charge["one"]).max() < 1e-10 * size,
      "along z: the T2 copy has the same rho and J^x4 at the mirror point")
```

The dictionary comprehension `{key: np.array(values) for ...}` turns each list into a numpy array, so that whole arrays can be added and compared. `size` is the largest modulus of the field's energy density or charge density along the line; the tolerances are measured relative to it. Two checks at all 160 points: the T1 partner has exactly $-\rho$ and $-J^{x_4}$; the T2 copy has, at the mirror point, the SAME ρ and the SAME J^{x_4}. The second tolerance is 10^{-10} rather than 10^{-12} because the copy is evaluated with a different (mirror-patch) geometry, whose numbers $\sin^{1/6}(\pi - z)$ and

$-\cot(\pi - z)$ agree with those of the patch only up to rounding, and near the brane the factor $\tan z = 1/E_0$ is large and amplifies these rounding differences. The cell prints the two PASS lines.

In [10], figure 2: the profiles across the brane.

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for ax, data, label in ((axes[0], rho, "energy density $\\rho$"),
                        (axes[1], charge, "charge density $J^{x_4}$")):
    ax.plot(z_patch, data["one"], color="tab:blue", label="$\\Psi$, $(m, \\lambda)$")
    ax.plot(z_patch, data["T1"], "--", color="tab:red",
            label="T1 partner $\\Gamma\\Psi$, $(-m, -\\lambda)$")
    ax.plot(z_patch, data["one"] + data["T1"], color="black", linewidth=2.0,
            label="T1 pair: sum")
    ax.plot(np.pi - z_patch, data["T2"], color="tab:green",
            label="T2 copy $\\gamma^{(x_0)}\\Psi$, $(-m, \\lambda)$")
```

Two panels, the energy density on the left and the charge density on the right; the loop draws the same four curves in each. `ax.plot(x, y, ...)` draws a curve through the points; the style string of two minus signs makes it dashed, `color` sets its colour and `label` its name in the legend. The first three curves live on the patch (the field, its T1 partner and their sum); the fourth, the T2 copy, is drawn at its own points $\pi - z$ on the mirror patch.

```
    ax.axvline(np.pi / 2, color="grey", linestyle=":", linewidth=1.5)
    ax.text(np.pi / 2, ax.get_ylim()[1], " brane", va="top", fontsize=8)
    ax.set_xlabel("hidden coordinate $z = 6Hx_0$")
    ax.set_ylabel(label + " (units $H = 1$)")
handles, names = axes[0].get_legend_handles_labels()
fig.legend(handles, names, loc="lower center", ncol=4, fontsize=8.5,
           bbox_to_anchor=(0.5, -0.06))  # one legend below both panels
axes[0].set_title("T1: reversed at the same point; T2: equal at the mirror point",
                  fontsize=9)
axes[1].set_title("the same for the charge density", fontsize=9)
save_figure(fig, "pair_profiles",
            "Left: the energy density $\\rho = -T_{x_4x_4}$, right: the charge "
            ...
            "the brane, with the same sign: T1 pairs cancel, T2 pairs add.")
```

`ax.axvline` draws the dotted vertical line of the brane $z = \pi/2$, and `ax.text` writes the word brane at its top (`ax.get_ylim()[1]` is the upper end of the vertical axis; `va="top"` hangs the text below that height). The axis labels name the quantities and the units. `get_legend_handles_labels` collects the curves and their names from the left panel, and `fig.legend` draws one legend for both panels, in four columns (`ncol=4`), below them (`bbox_to_anchor=(0.5, -0.06)` places its lower centre slightly below the figure). The two titles follow. The call of `save_figure` saves the figure as `20a_2_pair_profiles.png`; its caption string (twelve lines, shown here by its first and last line with `...` between them) is printed in full in the complete text of the notebook and under the figure.
What figure 2 shows. Horizontal axis: the hidden coordinate z from 0 to π, with the brane dotted at $\pi/2$; vertical axis: the energy density (left) and the charge density (right), units $H = 1$. Left of the brane, the blue curve of the field and the red dashed curve of its T1 partner are mirror images of each other in the horizontal axis, and their black sum lies on zero everywhere. Right of the brane, the green curve of the T2 copy is the blue curve reflected in the vertical line of the

brane, with the SAME sign. The student should see the essential difference of the two theorems: T1 partners cancel, T2 partners add.

In [11], the T2 mirror copy does not cancel; figure 3.

```
R0 = np.array(R0_SIGNS, dtype=float)  # the reflection R0 of the x0 direction
mirror_dpsi = [R0[mu] * (gamma_num[0] @ dpsi[mu]) for mu in range(8)]
copy = bilinears(geometry_at(-1, np.pi - Z0, A4_0, 1.0), gamma_num[0] @ psi,
                 mirror_dpsi, -M0, LAM0)  # the T2 copy at the mirror point
pulled_back = np.outer(R0, R0) * one["T"]  # R0 T R0
```

`R0` is the diagonal of $R_0 = \mathrm{diag}(-1, 1, 1, 1, 1, 1, 1, 1)$. The copy of the configuration of In [7] (with its full jet, including the x_0 derivative, which the mirror reverses) is evaluated at the mirror point $\pi - 0.6$. `np.outer(R0, R0)` is the 8×8 table of products $R_{0,\mu} R_{0,\nu}$; multiplying it entry by entry with T gives $(R_0 T R_0)_{\mu\nu}$: the entries with exactly one index x_0 change sign, all others stay.

```
reproduces(np.abs(copy["T"] - pulled_back).max() < 1e-10 * scale,
           "T2 copy: T' = R0 T R0 at the mirror point (64 components)",
           PAIR_PY, "T2.metric.commuting.emt")
reproduces(np.abs(copy["J"] - R0 * one["J"]).max() < 1e-10 * scale,
           "T2 copy: J' = R0 J (the charge density is unchanged)",
           PAIR_PY, "T2.metric.commuting.current")
reproduces(abs(copy["S"] + one["S"]) < 1e-10 * scale, "T2 copy: S' = -S",
           PAIR_PY, "T2.metric.commuting.S_odd")
reproduces(np.abs(copy["E"] + gamma_num[0] @ one["E"]).max() < 1e-10 * scale,
           "T2 copy: E_(-m,lambda)[Psi'] = -gamma^(x0) E_(m,lambda)[Psi]",
           PAIR_PY, "T2.metric.commuting.euler_lagrange_map")
reproduces(np.abs(copy["T"] + one["T"]).max() > 0.1 * scale,
           "T2 pair: the tensors do NOT cancel",
           PAIR_WL, "T2_mirror_energy_momentum_and_current_commuting")
```

Five checks of theorem T2 at the point: the copy's tensor is the pulled-back one, $T' = R_0 T R_0$; its current is $J' = R_0 J$ (so the charge density J^{x_4} is unchanged); its scalar is $S' = -S$; its residual is $-\gamma^{(x_0)}$ times that of the configuration (so it solves the $(-m, \lambda)$ equation on the mirror patch exactly when the configuration solves the (m, λ) equation on the patch); and the sum $T + T'$ is NOT small (more than a tenth of the size of the tensor): a T2 pair has no zero source.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.3))
panels = ((one["T"], "$T_{\\mu\\nu}[\\Psi]$ at $z$"),
          (copy["T"], "$T_{\\mu\\nu}$ of the T2 copy at $\\pi - z$"),
          (copy["T"] - pulled_back, "copy minus $R_0TR_0$"))
for ax, (values, title) in zip(axes, panels):
    image = ax.imshow(values, cmap="RdBu_r", vmin=-scale, vmax=scale)
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=7)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=7)
    ax.grid(False)
    ax.set_title(title, fontsize=10)
fig.colorbar(image, ax=list(axes), shrink=0.8, label="value (units $H = 1$)")
```

```
save_figure(fig, "t2_mirror_tensor",
            "Left: the energy-momentum tensor $T_{\\mu\\nu}$ of the configuration "
            ...
            "the T2 pair has twice the source of one member.")
```

Three new panels: the tensor of the configuration, the tensor of its T2 copy, and the copy minus the pulled-back tensor. The drawing loop is the one of In [8], line for line: a heat map with the same colour scale in each panel, the labels x_0 to x_7 on both axes, no grid, a title; then the common colour bar, and `save_figure` saves `20a_3_t2_mirror_tensor.png`. **What figure 3 shows.** Three heat maps of $T_{\mu\nu}$ (rows and columns x_0 to x_7, units $H = 1$, red positive, blue negative): left the configuration at $z = 0.6$, middle its T2 copy at $\pi - 0.6$, right their difference after the pull-back. The student should see that the left and middle maps are equal except in the last row and the last column, whose off-diagonal entries (one index x_0) have the opposite colour, and that the right map is white. Unlike the T1 partner, the mirror copy carries the SAME energy: a T2 pair has twice the source of one member.

In [12], the T1 pair as the only source: Einstein's vacuum equations.

```
record = read_json(A4_RECORD)
ad1, ad2 = sp.symbols("ad1 ad2")  # a4' and a4'' (complex allowed)
Lam = sp.symbols("Lambda")  # the cosmological constant
alpha2, alpha3, A = sp.symbols("alpha2 alpha3 A", real=True)
KEYS = ["x0x0", "x1x1", "x4x4", "x5x5"]  # the independent diagonal components
```

The record of the field equations for a_4 is read. Its formulas use the names `ad1` and `ad2` for a_4' and a_4''; the notebook makes symbols with the same names and, on purpose, does NOT declare them real, so that sympy may find complex solutions if there are any. `Lam` is Λ; `alpha2`, `alpha3` and `A` are the two higher Lovelock couplings and the slope of the linear member. `KEYS` names the four independent diagonal components (the others repeat these).

```
def lovelock(order, key):
    """The component key of E_(order) from the record, as a sympy expression."""
    text = record["lovelockTensors"][f"E{order}"][key]["input"].replace("^", "**")
    return sp.sympify(text, locals={"ad1": ad1, "ad2": ad2, "H": H})
```

The record stores each component of $E_{(1)}$, $E_{(2)}$ and $E_{(3)}$ as a text in Wolfram's input form, in which a power is written `^`. `.replace("^", "**")` turns it into Python's form, and `sp.sympify` reads the text as a sympy expression; `locals` makes the names in the text mean the notebook's symbols.

```
G = {key: lovelock(1, key) for key in KEYS}  # the Einstein tensor
for key in KEYS:
    say(f"G^{key[:2]}_{key[2:]} = {G[key]}")
vacuum = [G[key] + Lam for key in KEYS]  # zero source
solutions = sp.solve(vacuum, [ad1, ad2, Lam], dict=True)
say(f"all solutions of the vacuum equations: {solutions}")
```

`G` holds the four components of the Einstein tensor $E_{(1)} = G$; the loop prints them (`key[:2]` and `key[2:]` are the first two and the last two letters of the key). Output: $G^{x_0}{}_{x_0} = 15H^2 - 3a_4'^2$,

$G^{x_1}{}_{x_1} = 15H^2 - 3a_4'^2 + a_4''$, $G^{x_4}{}_{x_4} = 21H^2 + 3a_4'^2$, $G^{x_5}{}_{x_5} = 15H^2 - 3a_4'^2 - a_4''$. `vacuum` is the list of the four vacuum equations $G^{\mu}{}_{\mu} + \Lambda = 0$, and `sp.solve` finds ALL solutions for the three unknowns a_4', a_4'', Λ, each as a dictionary. Output: two solutions, $\Lambda = -18H^2$, $a_4'' = 0$ and $a_4' = -iH$ or $+iH$.

```
found = {(s[ad1], s[ad2], s[Lam]) for s in solutions}
reproduces(found == {(sp.I * H, 0, -18 * H ** 2), (-sp.I * H, 0, -18 * H ** 2)},
           "the only solutions have a4' = +i H or -i H: none is real",
           LEAD_EGB, "no_vacuum_for_H_positive")
difference = sp.expand(vacuum[2] - vacuum[0])  # x4 equation minus x0 equation
total = sp.expand(vacuum[2] + vacuum[0])  # x4 equation plus x0 equation
report("x4 equation minus x0 equation", difference)
report("x4 equation plus x0 equation", total)
```

`found` is the set of the solutions as triples; the check requires exactly the two triples $(\pm iH, 0, -18H^2)$ (`sp.I` is sympy's i): no real rate. Then the two combinations of the proof of C1(a): the x_4 equation minus the x_0 equation and their sum, multiplied out by `sp.expand`. Output: $6H^2 + 6a_4'^2$ and $36H^2 + 2\Lambda$, lines 5 and the remark after line 6 of Section 20.7.

```
reproduces(difference == 6 * ad1 ** 2 + 6 * H ** 2
           and total == 36 * H ** 2 + 2 * Lam,
           "6 a4'^2 + 6 H^2 = 0 and 36 H^2 + 2 Lambda = 0 (record's lines)",
           A4_WL, "einstein_no_vacuum_solution")
a_real = sp.symbols("a_real", real=True)  # a real value of a4'
reproduces(sp.solve(difference.subs(ad1, a_real), a_real) == [],
           "for real a4' the difference 6 a4'^2 + 6 H^2 never vanishes",
           A4_PY, "einstein_no_vacuum")
```

The first check compares the two combinations with the record's lines. The second substitutes for a_4' a symbol declared real and asks sympy for the real solutions of $6a_4'^2 + 6H^2 = 0$: the answer is the empty list `[]`, line 6 of the proof.

In [13], figure 4: the two vacuum conditions never meet.

```
slopes = np.linspace(-3.0, 3.0, 601)  # a4'/H
lower = -(3 * slopes ** 2 + 21)  # Lambda/H^2 where the x4 equation holds
upper = 3 * slopes ** 2 - 15  # Lambda/H^2 where the x0 equation holds
check(np.min(upper - lower) >= 6.0 - 1e-12, "the two curves stay 6 H^2 apart or more")
```

601 values of a_4'/H from -3 to 3. Solving the x_4 equation for Λ gives $\Lambda = -(3a_4'^2 + 21H^2)$, a parabola that opens downwards; solving the x_0 equation gives $\Lambda = 3a_4'^2 - 15H^2$, a parabola that opens upwards (in units of H^2). Their vertical distance is $6a_4'^2 + 6H^2 \geq 6H^2$; the check confirms it at all 601 points.

```
fig, ax = plt.subplots(figsize=(7.0, 4.6))
ax.plot(slopes, lower, color="tab:red", label="$x_4$: $\\Lambda = -(3a_4'^2 + 21H^2)$")
ax.plot(slopes, upper, color="tab:blue", label="$x_0$: $\\Lambda = 3a_4'^2 - 15H^2$")
ax.fill_between(slopes, lower, upper, color="grey", alpha=0.2,
                label="gap $6a_4'^2 + 6H^2$")
ax.annotate("", xy=(1.0, -12.0), xytext=(1.0, -24.0),
```

```
            arrowprops={"arrowstyle": "<->", "color": "black"})
ax.text(1.12, -19.3, "$12H^2$ at $a_4' = H$\n(the author's history)", fontsize=8,
        va="top")  # va="top": the text hangs below the given height
ax.axhline(-18.0, color="black", linestyle=":", linewidth=1.0)
ax.text(-2.95, -17.3, "$\\Lambda = -18H^2$: both hold only for $a_4' = \\pm iH$",
        fontsize=8)
ax.set_xlabel("$a_4'/H$")
ax.set_ylabel("$\\Lambda/H^2$")
ax.set_title("Einstein gravity with zero source: the two conditions never meet")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "einstein_vacuum_parabolas",
            "The vacuum equations of Einstein gravity for the author's metric, the "
            ...
            "(dotted line), are not real.")
```

The two parabolas are drawn in red and blue; `fill_between` shades the band between them in grey (`alpha=0.2`: 20 per cent opaque). `ax.annotate` with an empty text draws a double arrow at $a_4' = H$ from $\Lambda = -24H^2$ to $-12H^2$, the gap $12H^2$ of the deflating history, and `ax.text` labels it (`\n` starts a new line). `ax.axhline` draws the dotted horizontal line $\Lambda = -18H^2$, the value at which the two equations would hold together if a_4' could be $\pm iH$. **What figure 4 shows.** Horizontal axis: the rate a_4'/H; vertical axis: the cosmological constant Λ/H^2. A vacuum would be a point on both parabolas, and there is none: the grey band between them is never thinner than $6H^2$ and is $12H^2$ thick at the author's history. The student should see at a glance why no choice of the rate and of Λ removes the need for a source in Einstein gravity, and therefore why a T1 pair, whose source is zero, cannot be the only source of the author's metric.

In [14], what the geometry requires and what a pair supplies.

```
couplings = {"alpha1": 1, "alpha2": alpha2, "alpha3": alpha3, "H": H, "ad1": ad1,
             "ad2": ad2, "AA": A}  # alpha1 = 1 throughout
null_text = record["generalSource"]["nullCombinations"]["kappa(rho+p0)"]["input"]
null_required = sp.sympify(null_text.replace("^", "**"), locals=couplings)
V_text = record["linearMember"]["vacuumFactor"]["input"]
V = sp.sympify(V_text.replace("^", "**"), locals=couplings)
```

`couplings` tells `sympify` what the names in the record's texts mean; $\alpha_1 = 1$ throughout (the record writes the slope as `AA`). The record's null combination $\kappa(\rho + p_0)$ for a general source and its vacuum factor V of the linear member are read as sympy expressions.

```
lovelock_sum = {key: lovelock(1, key) + alpha2 * lovelock(2, key)
                + alpha3 * lovelock(3, key) for key in KEYS}
own_null = -(lovelock_sum["x4x4"] - lovelock_sum["x0x0"])
check(sp.expand(own_null - null_required) == 0,
      "the null combination of the record equals -(E^x4_x4 - E^x0_x0)")
```

`lovelock_sum` is $\sum_k \alpha_k E_{(k)}$ for the four components. `own_null` is the notebook's own null combination, $-\sum_k \alpha_k ({E_{(k)}}^{x_4}{}_{x_4} - {E_{(k)}}^{x_0}{}_{x_0})$, derived in Section 20.7; the check finds it equal to the record's.

```
linear = {ad1: A * H, ad2: 0}  # the linear member a4 = A H x4 + a0
reproduces(sp.expand(own_null.subs(linear) + 6 * (A ** 2 + 1) * H ** 2 * V) == 0,
           "linear member: kappa (rho + p0) = -6 (A^2 + 1) H^2 V",
           A4_WL, "linear_member_vacuum_factor")
reproduces(sp.expand(V.subs({alpha2: 0, alpha3: 0})) == 1
           and sp.expand(null_required.subs({alpha2: 0, alpha3: 0})
                         + 6 * ad1 ** 2 + 6 * H ** 2) == 0,
           "Einstein: V = 1 and kappa (rho + p0) = -6 (a4'^2 + H^2)",
           A4_PY, "einstein_null_energy_x0")
reproduces(sp.expand(null_required.subs({alpha2: 0, alpha3: 0})
                     + 6 * ad1 ** 2 + 6 * H ** 2) == 0,
           "the same null combination from the lead's independent Einstein tensor",
           LEAD_EGB, "einstein_null_energy")
```

For the linear member ($a_4' = AH$, $a_4'' = 0$) the null combination equals $-6(A^2+1)H^2V$. In Einstein gravity ($\alpha_2 = \alpha_3 = 0$) $V = 1$ and the null combination is $-6(a_4'^2 + H^2)$, which the record's sympy report and the lead's independent report both state.

In [15], figure 5: the required null combination against the slope.

```
A_axis = np.linspace(0.0, 2.0, 401)
V_numeric = sp.lambdify((A, alpha2, alpha3), V.subs(H, 1), "numpy")
fig, ax = plt.subplots(figsize=(7.0, 4.4))
for x2, style, label in ((0.0, "-", "Einstein ($\\alpha_2 = 0$)"),
                         (1 / 48, "--", "Gauss-Bonnet, $\\alpha_2H^2 = 1/48$"),
                         (1 / 40, "-.", "Gauss-Bonnet, $\\alpha_2H^2 = 1/40$")):
    required = -6 * (A_axis ** 2 + 1) * V_numeric(A_axis, x2, 0.0)
    ax.plot(A_axis, required, style, label=label)
```

401 slopes A from 0 to 2. `V_numeric` is V as a numpy function of A, α_2 and α_3 in units $H = 1$. The loop draws the required $\kappa(\rho + p_0)/H^2 = -6(A^2+1)V$ for three theories: Einstein gravity (solid), Gauss-Bonnet with $\alpha_2 H^2 = 1/48$ (dashed) and with $1/40$ (dash-dotted).

```
ax.axhline(0.0, color="black", linewidth=2.0, label="what a T1 pair supplies: 0")
ax.plot([1.0, 0.0], [0.0, 0.0], "o", color="tab:red", markersize=7,
        label="a T1 pair fits: $V = 0$")
ax.set_xlabel("slope $A$ of the linear member $a_4 = AHx_4$")
ax.set_ylabel("required $\\kappa(\\rho + p_0)/H^2$")
ax.set_title("The source the geometry requires, and the zero of a T1 pair")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "null_combination",
            "The null combination $\\kappa(\\rho + p_0)$ (energy density plus the "
            "pressure along $x_0$, times the gravitational coupling; vertical, "
            ...
            "$\\alpha_2H^2 = 1/48$, and $A = 0$ for $1/40$).")
```

The thick black line at zero is what a T1 pair supplies; the two red dots mark where a requirement curve reaches it. **What figure 5 shows.** Horizontal axis: the slope A; vertical axis: the null combination the field equations require of any source, in units H^2. The Einstein curve, $-6(A^2+1)$, stays at or below -6 for every slope, so it never meets the black line: in Einstein gravity a T1 pair

is never the only source. Each Gauss-Bonnet curve reaches zero at exactly one slope: the dashed one crosses it at $A = 1$ (the author's history) for $\alpha_2 H^2 = 1/48$, and the dash-dotted one touches it at $A = 0$ for $1/40$ and is positive at every other slope (there $V = -A^2/5$, so the requirement is $6A^2(A^2+1)/5$). The student should see that C1(a) is a property of Einstein gravity, not of every theory of gravity.

In [16], Einstein-Gauss-Bonnet gravity: when a zero source is allowed.

```
V_gb = sp.expand(V.subs(alpha3, 0))
A_squared = sp.solve(V_gb, A ** 2)[0]
reproduces(sp.simplify(A_squared - (1 - 40 * alpha2 * H ** 2)
                       / (8 * alpha2 * H ** 2)) == 0,
           "Gauss-Bonnet vacuum: A^2 = (1 - 40 alpha2 H^2)/(8 alpha2 H^2)",
           A4_WL, "einstein_gauss_bonnet_vacuum_linear")
```

V with $\alpha_3 = 0$; `sp.solve(V_gb, A ** 2)` solves $V = 0$ for A^2 (sympy treats A^2 as the unknown) and `[0]` takes the one solution. The check compares it with the formula derived line by line in Section 20.7.

```
x2_author = sp.solve(A_squared.subs(H, 1) - 1, alpha2)[0]  # A = 1, H = 1
report("alpha2 H^2 for a vacuum with A = 1", x2_author)
gauss_bonnet = {key: (lovelock(1, key) + alpha2 * lovelock(2, key))
                .subs({ad1: H, ad2: 0, alpha2: x2_author / H ** 2})
                for key in KEYS}
Lam_author = sp.simplify(-gauss_bonnet["x4x4"])  # Lambda from the x4 equation
report("Lambda of that vacuum", Lam_author)
```

Setting $A^2 = 1$ (with $H = 1$) and solving for α_2 gives $1/48$ (Output: RESULT $\alpha_2 H^2$ for a vacuum with $A = 1 = 1/48$). The four components of $E_{(1)} + \alpha_2 E_{(2)}$ are evaluated at $a_4' = H$, $a_4'' = 0$, $\alpha_2 = 1/(48H^2)$; the vacuum x_4 equation $E^{x_4}{}_{x_4} + \Lambda = 0$ gives $\Lambda = -E^{x_4}{}_{x_4}$. Output: RESULT Lambda of that vacuum $= -12H^2$, the value derived by hand in Section 20.7.

```
reproduces(x2_author == sp.Rational(1, 48) and Lam_author == -12 * H ** 2
           and all(sp.simplify(gauss_bonnet[key] + Lam_author) == 0 for key in KEYS),
           "a4 = H x4 solves the Gauss-Bonnet vacuum equations exactly",
           A4_PY, "einstein_gauss_bonnet_vacuum_linear")
zero_off = all(record["lovelockTensors"][f"E{k}"][key]["input"] == "0"
               for k in (1, 2, 3) for key in ("x4x0", "x0x4"))
check(zero_off, "the mixed components x4x0 and x0x4 of E_(1..3) are 0 in the record")
```

The check requires $\alpha_2 H^2 = 1/48$, $\Lambda = -12H^2$, and that ALL FOUR independent vacuum equations vanish exactly for the deflating history. The second check confirms that the record has no mixed component $x_4 x_0$ in any of the three Lovelock tensors, so no further equation remains: in this theory the author's deflating history is a vacuum, and a T1 pair CAN be its only source.

```
Lam_curve = sp.simplify(-(lovelock(1, "x4x4") + alpha2 * lovelock(2, "x4x4"))
                        .subs({ad1: A * H, ad2: 0}).subs(A ** 2, A_squared))
say(f"Lambda along the vacuum curve: {sp.factor(Lam_curve)}")
```

Along the whole family of Gauss-Bonnet vacua the cosmological constant is $\Lambda = -E^{x_4}{}_{x_4}$ at $a_4' = AH$ with A^2 replaced by the vacuum formula; `sp.factor` writes it as a fraction. Output: $3(3840H^4\alpha_2^2 - 192H^2\alpha_2 + 1)/(16\alpha_2)$, that is $720\alpha_2 H^4 - 36H^2 + 3/(16\alpha_2)$; at $\alpha_2 = 1/(48H^2)$ this is $15H^2 - 36H^2 + 9H^2 = -12H^2$.

In [17], figure 6: the Gauss-Bonnet vacua.

```
x_axis = np.linspace(0.004, 1 / 40, 400)  # alpha2 H^2
A_curve = np.sqrt((1 - 40 * x_axis) / (8 * x_axis))
Lam_numeric = sp.lambdify(alpha2, Lam_curve.subs(H, 1), "numpy")
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.3))
axes[0].plot(x_axis, A_curve, color="tab:purple")
axes[0].plot([1 / 48], [1.0], "o", color="tab:red", markersize=8,
             label="$A = 1$ at $\\alpha_2H^2 = 1/48$")
axes[0].set_ylabel("slope $A$ of the vacuum $a_4 = AHx_4$")
axes[1].plot(x_axis, Lam_numeric(x_axis), color="tab:purple")
axes[1].plot([1 / 48], [-12.0], "o", color="tab:red", markersize=8,
             label="$\\Lambda = -12H^2$ at $\\alpha_2H^2 = 1/48$")
axes[1].set_ylabel("$\\Lambda/H^2$ of that vacuum")
```

The 400 values of the coupling $\alpha_2 H^2$ run from 0.004 to the limit $1/40$. For each of them the array `A_curve` holds the slope of the vacuum, $\sqrt{(1 - 40\alpha_2 H^2)/(8\alpha_2 H^2)}$, computed with the square root `np.sqrt`; it is drawn in the left panel. The right panel shows the cosmological constant of each vacuum, the expression `Lam_curve` of In [16] turned into a numpy function. The red dots mark the author's history.

```
for ax in axes:
    ax.set_xlabel("Gauss-Bonnet coupling $\\alpha_2H^2$")
    ax.axvline(1 / 40, color="grey", linestyle=":")
    ax.legend(fontsize=8)
axes[0].set_title("no vacuum for $\\alpha_2 \\to 0$ (Einstein), $A = 0$ at $1/40$",
                  fontsize=9)
axes[1].set_title("the cosmological constant each vacuum needs", fontsize=9)
save_figure(fig, "gauss_bonnet_vacuum",
            "The vacua of Einstein-Gauss-Bonnet gravity ($\\alpha_1 = 1$, "
            ...
            "$\\alpha_2H^2 = 1/48$ and $\\Lambda = -12H^2$ (checked exactly).")
```

Both panels get the horizontal label, the dotted limit $\alpha_2 H^2 = 1/40$ and a legend. **What figure 6 shows.** Horizontal axis: the Gauss-Bonnet coupling $\alpha_2 H^2$. Left: the slope A of the vacuum; it grows without bound as $\alpha_2 \to 0$, the Einstein limit, which has no vacuum, and falls to $A = 0$ at $1/40$. Right: the cosmological constant Λ/H^2 that the same vacuum needs. The student should see that each coupling between 0 and $1/40$ has exactly one vacuum among the linear members, and that the author's deflating history $A = 1$ is the one at $\alpha_2 H^2 = 1/48$, with $\Lambda = -12H^2$.

In [18], third order: one straight line of vacua per slope; figure 7.

```
y_line = sp.solve(V.subs({alpha2: sp.Symbol("x"), alpha3: sp.Symbol("y"), H: 1}),
                  sp.Symbol("y"))[0]
wanted = ((8 * A ** 2 + 40) * sp.Symbol("x") - 1) / (72 * A ** 4 + 144 * A ** 2 + 360)
check(sp.simplify(y_line - wanted) == 0, "for fixed A the vacua form a straight line")
check(V.subs({alpha2: 0, alpha3: 0}) == 1, "Einstein gravity (origin) has V = 1")
```

$V = 0$ is solved for $y = \alpha_3 H^4$ with $x = \alpha_2 H^2$ (`sp.Symbol("x")` makes one symbol). The check compares the solution with the straight line derived in Section 20.7; the second check confirms that the origin $x = y = 0$, Einstein gravity, has $V = 1$ and is no vacuum.

```
x_plot = np.linspace(-0.01, 0.05, 200)
fig, ax = plt.subplots(figsize=(7.0, 4.6))
for slope_A in (0.0, 0.5, 1.0, 2.0, 3.0):
    line = ((8 * slope_A ** 2 + 40) * x_plot - 1) / (
        72 * slope_A ** 4 + 144 * slope_A ** 2 + 360)
    ax.plot(x_plot, 1000 * line, label=f"$A = {slope_A:g}$")
ax.plot([0.0], [0.0], "s", color="black", markersize=8,
        label="Einstein gravity: no vacuum")
ax.plot([1 / 48], [0.0], "o", color="tab:red", markersize=8,
        label="section 13: $A = 1$, $\\alpha_3 = 0$")
ax.axhline(0.0, color="grey", linewidth=0.8)
ax.axvline(0.0, color="grey", linewidth=0.8)
ax.set_xlabel("$\\alpha_2H^2$")
ax.set_ylabel("$1000\\,\\alpha_3H^4$")
ax.set_title("Couplings for which $a_4 = AHx_4$ is a vacuum (third order)")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "third_order_vacua",
            "The Einstein-Lovelock couplings for which the linear member "
            ...
            "author's history $A = 1$ at $\\alpha_2H^2 = 1/48$.")
```

For five slopes the line is drawn, its height multiplied by 1000 so that the small values of $\alpha_3 H^4$ are readable (`{slope_A:g}` prints a number without needless zeros). The black square is the origin, Einstein gravity; the red dot is the Gauss-Bonnet vacuum of In [16] (the legend's "section 13" is the notebook's own section). **What figure 7 shows.** Horizontal axis: $\alpha_2 H^2$; vertical axis: $1000\,\alpha_3 H^4$. For each slope A the couplings for which $a_4 = AHx_4$ is a vacuum form a straight line; the five coloured lines all pass below the origin. The student should see that Einstein gravity, the black square, lies on no line, while every line contains infinitely many theories with a vacuum, among them the red dot of the author's history.

In [19], the last check.

```
figure_names = ["t1_pair_tensor", "pair_profiles", "t2_mirror_tensor",
                "einstein_vacuum_parabolas", "null_combination",
                "gauss_bonnet_vacuum", "third_order_vacua"]
paths = [output_file(f"{FIGURE_FOLDER}/20a_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

`enumerate(list, 1)` pairs each name with its number, starting at 1, so that `paths` lists the seven figure files `20a_1_t1_pair_tensor.png` to `20a_7_third_order_vacua.png`. The check confirms that all exist, and `all_checks_passed()` prints the last line. Output: PASS every figure file of this notebook exists, and ALL 37 CHECKS PASSED (notebook 20a).

20.12 One universe without a partner: an exact solution and its two partners

Why an example. The pairing theorems speak about ALL solutions at once, and so they are easy to misread. One exact solution makes every statement a number that can be checked. We build one **universe** of the commuting field dirac16complex00: a solution of its field equation that is, by itself, the COMPLETE source of the author's metric with the exponentially deflating history $a_4 = Hx_4$ in Einstein gravity. Then we build its T1 partner, the T1 pair and its T2 mirror copy, and ask of each whether it solves its own field equation and whether it can be the source of the author's metric. The answer teaches the lesson of the whole chapter in one example: the universe needs no partner, and the partners exist as solutions of OTHER equations, which nothing in the theory forces anyone to realise.

Status. The universe below and the statements about its partners are COMPUTED by Notebook 20b and derived by hand in this section (exact arithmetic for the field equations and for the two Einstein conditions; floating point, to 10^{-10}, for the 64 components of the Einstein equations at nine points). They are a result of this book, not a Revision record. They rest on PROVED record statements, which are named at each step, and on these ASSUMED inputs: Einstein gravity ($\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$); the background $b_4 = 1$ of the author's metric (the condensate below has $T^{x_4}{}_{x_0} = 0$, and in Einstein gravity such a source forces a constant b_4, which is $b_4 = 1$ after a rescaling of the extra times; Section 20.7); the classical commuting field dirac16complex00 (not the quantised dirac16complex); the sign convention $\sigma_T = +1$ of the record of the field equations for a_4 (it fixes the overall sign of the energy-momentum tensor; Chapter 12); the chosen numbers $A = 1$, $\Lambda = -36H^2$ and $M = -5H$; and, for the mirror copy, the Z2 construction across the brane. Units: $H = \kappa = 1$.

Step 1: the field equation of a condensate. A **condensate** is a field that depends on the time x_4 only, $\Phi = \Phi(x_4)$. Its derivatives along the seven other directions vanish, $\gamma^{x_4} = \gamma^{(x_4)}$ because $g_{x_4x_4} = -1$, and the spin connection enters the field equation only through $\sum_\mu \gamma^\mu \Omega_\mu = 3H\gamma^{(x_0)}$ (`python-a4-report.json`, check `authorT16_gravity_term`; `emt-divergence-and-spin-connection.json`, check `gamma_Omega_equals_3H_gamma0`). So the field equation $\gamma^\mu D_\mu \Phi = (m + \lambda S)\Phi$ becomes

$$\gamma^{(x_4)}\Phi' + 3H\gamma^{(x_0)}\Phi = M\Phi, \qquad M = m + \lambda S,$$

where $\Phi' = d\Phi/dx_4$ and M is the **effective mass**. Subtracting $3H\gamma^{(x_0)}\Phi$ from both sides,

$$\gamma^{(x_4)}\Phi' = \big(M - 3H\gamma^{(x_0)}\big)\Phi.$$

Multiplying from the left by $-\gamma^{(x_4)}$, and using $(-\gamma^{(x_4)})\gamma^{(x_4)} = -\eta^{x_4x_4} I_{16} = I_{16}$ (the Clifford relation with $\eta^{x_4x_4} = -1$),

$$\Phi' = \mathcal{A}\Phi, \qquad \mathcal{A} = -\gamma^{(x_4)}\big(M - 3H\gamma^{(x_0)}\big)$$

(`wolfram-a4-report.json`, check `condensate_equation_x0_consistent`). Chapter 12 derives line by line two facts about $\mathcal{A}$, which we use here: $\mathcal{A}^2 = -(M^2 - 9H^2)I_{16}$, so for $M^2 > 9H^2$ every eigenvalue of $\mathcal{A}$ is $+iw$ or $-iw$ with the **frequency** $w = \sqrt{M^2 - 9H^2}$; and $C\mathcal{A} + \mathcal{A}^T C = 0$, so the density $S = \bar{\Phi}\Phi$ does not change in time (`python-a4-report.json`, check `authorT16_condensate_S_constant`). An eigenvector Φ_0 with $\mathcal{A}\Phi_0 = -iw\Phi_0$ gives the solution $\Phi(x_4) = e^{-iwx_4}\Phi_0$, because its derivative is $-iw\Phi = \mathcal{A}\Phi$.

Step 2: its source. The record proves for a condensate that the diagonal of the kinetic tensor is $K^{x_4}{}_{x_4} = MS$ and zero in the seven other directions (`python-a4-report.json`, check `authorT16_condensate_kinetic_diagonal`), and that the off-diagonal components of its energy-momentum tensor are multiples of 15 **three-gamma bilinears** $\bar{\Phi}\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$ (`wolfram-a4-report.json`, check `condensate_offdiagonal_are_three_gamma_bilinears`). When these 15 numbers vanish, the tensor is diagonal with

$$\rho = mS + \tfrac{\lambda}{2}S^2, \qquad p_3 = p_t = p_0 = p = \tfrac{\lambda}{2}S^2$$

(Chapter 12 derives both lines from $L = MS - mS - \frac{\lambda}{2}S^2 = \frac{\lambda}{2}S^2$). Two combinations will be needed. Adding,

$$\rho + p = mS + \lambda S^2 = (m + \lambda S)S = MS,$$

because $\frac{\lambda}{2}S^2 + \frac{\lambda}{2}S^2 = \lambda S^2$, and then S is taken out. Subtracting,

$$\rho - p = mS,$$

because the two terms $\frac{\lambda}{2}S^2$ cancel.

Step 3: the two Einstein conditions. For the linear member $a_4 = AHx_4$ we have $a_4' = AH$ and $a_4'' = 0$. The record's Einstein tensor (`python-a4-report.json`, check `einstein_components`) then gives $G^{x_4}{}_{x_4} = (3A^2 + 21)H^2$ and $G^{x_1}{}_{x_1} = G^{x_5}{}_{x_5} = G^{x_0}{}_{x_0} = (15 - 3A^2)H^2$. With $T^{x_4}{}_{x_4} = -\rho$ and $T^{x_0}{}_{x_0} = p$, the time equation and the hidden equation are

$$(3A^2 + 21)H^2 + \Lambda = -\kappa\rho, \qquad (15 - 3A^2)H^2 + \Lambda = \kappa p;$$

the equations for x_1, x_2, x_3 and for x_5, x_6, x_7 are the hidden equation again, because their components of G and of T are the same, and every off-diagonal equation reads $0 = 0$ once the 15 three-gamma bilinears vanish. Subtracting the second equation from the first removes Λ:

$$(3A^2 + 21 - 15 + 3A^2)H^2 = -\kappa\rho - \kappa p \quad \Longrightarrow \quad 6(A^2 + 1)H^2 = -\kappa MS,$$

where the last step collects $3A^2 + 3A^2 = 6A^2$ and $21 - 15 = 6$ and uses $\rho + p = MS$. Adding the two equations removes A:

$$(3A^2 + 21 + 15 - 3A^2)H^2 + 2\Lambda = -\kappa\rho + \kappa p \quad \Longrightarrow \quad 36H^2 + 2\Lambda = -\kappa mS,$$

using $-\kappa(\rho - p) = -\kappa mS$. These are the record's two conditions (`python-a4-report.json` and `wolfram-a4-report.json`, check `condensate_einstein_quadratic_U`):

$$\kappa MS = -6(A^2 + 1)H^2, \qquad \kappa mS = -(36H^2 + 2\Lambda).$$

Step 4: the numbers of the universe. We choose $A = 1$ (the author's deflating history: 3-space grows like e^{x_4}, the three extra times shrink like e^{-x_4}), $\Lambda = -36$ and $M = -5$. The choice

$|M| = 5 > 3$ makes the condensate oscillate, with the frequency $w = \sqrt{25 - 9} = 4$. The choice of Λ gives the universe a positive energy density: the time equation says $\kappa\rho = -(24 + \Lambda)$ at $A = 1$, which is positive exactly when $\Lambda < -24$. Line by line:

1. The first condition with $\kappa = 1$: $MS = -6 \cdot 2 = -12$, so $S = -12/M = -12/(-5) = 12/5$.
2. The second condition: $mS = -(36+2\cdot(-36)) = -(36-72) = 36$, so $m = 36/S = 36\cdot 5/12 = 15$.
3. The definition $M = m + \lambda S$ solved for λ: $\lambda = (M - m)/S = (-5 - 15) \cdot 5/12 = -100/12 = -25/3$.
4. The energy density: $\rho = mS + \frac{\lambda}{2}S^2 = 36 + \left(-\frac{25}{6}\right) \cdot \frac{144}{25} = 36 - 24 = 12$.
5. The pressure: $p = \frac{\lambda}{2}S^2 = -24$; the equation of state is $w_{\mathrm{EoS}} = p/\rho = -2$.

Two checks of the arithmetic: $\rho + p = -12 = MS = (-5)(12/5)$, and $\rho - p = 36 = mS = 15 \cdot 12/5$. So the universe has mass $m = +15$ and coupling $\lambda = -25/3$.

Step 5: the condensate itself. It remains to find a spinor Φ_0 with $\mathcal{A}\Phi_0 = -4i\Phi_0$, all 15 three-gamma bilinears zero, and $S = 12/5$. The record's recipe (`wolfram-a4-report.json`, check `condensate_diagonal_witness_exact`): take the eigenvector v_1 of $\mathcal{A}$ with the eigenvalue $-iw$ on which the three products $\gamma^{(x_1)}\gamma^{(x_5)}$, $\gamma^{(x_2)}\gamma^{(x_6)}$ and $\gamma^{(x_3)}\gamma^{(x_7)}$ all have the value -1, the eigenvector v_2 with the same eigenvalue on which they all have the value $+1$, the number $c = \overline{v_1^\dagger C v_2}$ (the bar over a number is its complex conjugate), and the family $\Phi_0(t) = v_1 + t\,c\,v_2$ with a real parameter t. Chapter 12 shows that every member of the family has all 15 bilinears zero and the density $S(t) = 2t\,|v_1^\dagger C v_2|^2$, which is linear in t. Notebook 20b solves $S(t) = 12/5$ exactly and finds $t = 15/512$; by the formula for $S(t)$ this means $|v_1^\dagger C v_2|^2 = (12/5)/(2t) = (12/5)(256/15) = 1024/25$, the value Chapter 12 finds at $M = -5$. The universe is

$$\Phi(x_4) = e^{-4ix_4}\,\Phi_0(15/512),$$

and its **charge density** is $J^{x_4} = \Phi^\dagger B \Phi = -3$, the same at every time because $|e^{-4ix_4}|^2 = 1$ (COMPUTED exactly, Notebook 20b, In [5]). The universe solves its field equation exactly at every time, and its energy-momentum tensor solves all 64 Einstein equations of the author's metric with $a_4 = Hx_4$ and $\Lambda = -36H^2$, at every point checked (Notebook 20b, In [5] and In [6]). It is a complete universe of this theory, and nothing in it refers to a partner. This is the record's sentence made concrete: "a single universe with mass $+m$ is an equally valid solution without its partner" (`Revision/docs/PAIR_CREATION_PROOFS.md`, section 11.3).

Step 6: the T1 partner $\Gamma\Phi$. By theorem T1 the field $\Gamma\Phi$ solves the field equation with $(-m, -\lambda) = (-15, 25/3)$. We check the pieces by hand. Its density is $S' = S = 12/5$ (step 5 of Section 20.5), so its effective mass is

$$M' = -m + (-\lambda)S' = -(m + \lambda S) = -M = 5.$$

Its matrix is $\mathcal{A}' = -\gamma^{(x_4)}(M' - 3H\gamma^{(x_0)}) = -\gamma^{(x_4)}(-M - 3H\gamma^{(x_0)})$. We show that it equals $\Gamma\mathcal{A}\Gamma$, line by line:

$$\Gamma\mathcal{A}\Gamma = -\Gamma\gamma^{(x_4)}(M - 3H\gamma^{(x_0)})\Gamma = -\Gamma\gamma^{(x_4)}\Gamma\,\Gamma(M - 3H\gamma^{(x_0)})\Gamma,$$

by inserting $\Gamma\Gamma = I_{16}$ in the middle;

$$= -(-\gamma^{(x_4)})(M + 3H\gamma^{(x_0)}) = \gamma^{(x_4)}(M + 3H\gamma^{(x_0)}),$$

because $\Gamma\gamma^{(a)}\Gamma = -\gamma^{(a)}\Gamma\Gamma = -\gamma^{(a)}$ for every gamma (step 1 of Section 20.5) and $\Gamma M\Gamma = M$ for the number M;

$$= -\gamma^{(x_4)}(-M - 3H\gamma^{(x_0)}) = \mathcal{A}',$$

by taking out the factor -1. Two matrices related by $\mathcal{A}' = \Gamma\mathcal{A}\Gamma^{-1}$ are called **similar**; they have the same eigenvalues, because $\mathcal{A}v = \mu v$ gives $\mathcal{A}'(\Gamma v) = \Gamma\mathcal{A}v = \mu\,\Gamma v$. So the partner oscillates with the SAME frequency $w = 4$: the opposite sign of its energy is not an opposite frequency. (The record proves the same kind of statement for the one-particle Hamiltonians, $\Gamma h_m\Gamma = h_{-m}$: `wolfram-pairing.json`, check `Q_one_particle_maps`.) By (T1c) its tensor is $T' = -T$: $\rho' = -12$, $p' = +24$, and its charge density is $+3$.

Step 7: the partner alone is not a source of the author's metric. In Section 20.7 we derived what Einstein gravity requires of every source of the author's metric, for every a_4:

$$\kappa(\rho + p_0) = -6(a_4'^2 + H^2) \le -6H^2.$$

The universe supplies $\kappa(\rho + p_0) = 12 - 24 = -12$, which meets the requirement exactly when $a_4'^2 + 1 = 2$, that is $a_4' = \pm H$: the deflating history $+H$ (and its mirror choice $-H$; the equations do not select the sign, Chapter 12). The partner supplies $\kappa(\rho' + p_0') = -12 + 24 = +12$, which is larger than $-6H^2$, the largest value the requirement can take. So no real rate a_4' and no Λ make the partner alone a source of the author's metric. The partner's tensor does not depend on a_4 (the condensate does not feel the deflation, step 1), so this holds for every member of the author's family $a_4(x_4)$, not only for the linear one. The partner is a solution of ITS field equation in the author's geometry, as T1 says, but it cannot produce that geometry. (Status: PROVED here by hand from the record's null combination, `python-a4-report.json`, check `einstein_null_energy_x0`; checked by Notebook 20b, In [7].)

Step 8: the T1 pair. With the universe and its partner as the only source, the source is $T + T' = 0$ (T1d), and the Einstein residual is $G^\mu{}_\nu + \Lambda\delta^\mu_\nu$. At $a_4' = H$ and $\Lambda = -36H^2$ it is

$$\begin{aligned} G^\mu{}_\nu + \Lambda\delta^\mu_\nu &= \mathrm{diag}(12, 12, 12, 24, 12, 12, 12, 12)H^2 - 36H^2\,\delta^\mu_\nu \\ &= \mathrm{diag}(-24, -24, -24, -12, -24, -24, -24, -24)H^2 \neq 0, \end{aligned}$$

where the first matrix is G (step 3 with $A = 1$: $15 - 3 = 12$ and $3 + 21 = 24$). This is corollary C1 on the example: the pair does not solve the equations that the universe alone solves.

Step 9: the T2 mirror copy. By theorem T2 (Section 20.6, with m replaced by $-m$) the field $\Phi''(\pi - z) = \gamma^{(x_0)}\Phi(z)$ on the mirror patch solves the field equation with $(-m, \lambda) = (-15, -25/3)$

there. For the condensate, which does not depend on z, the copy is simply $\gamma^{(x_0)}\Phi(x_4)$. Its density is $S'' = -S$ (Section 20.6), so its effective mass is

$$M'' = -m + \lambda S'' = -m - \lambda S = -M = 5.$$

On the mirror patch the spin-connection term is $-3H\gamma^{(x_0)}$ (computed by Notebook 20a, In [5]; there the positive length of the hidden direction is $-\cot z$). We check the copy's field equation by hand. Multiply the universe's equation $\gamma^{(x_4)}\Phi' + 3H\gamma^{(x_0)}\Phi = M\Phi$ from the left by $-\gamma^{(x_0)}$:

$$-\gamma^{(x_0)}\gamma^{(x_4)}\Phi' - 3H\gamma^{(x_0)}\gamma^{(x_0)}\Phi = -M\gamma^{(x_0)}\Phi.$$

In the first term use $-\gamma^{(x_0)}\gamma^{(x_4)} = \gamma^{(x_4)}\gamma^{(x_0)}$ (the Clifford relation for the two different directions x_4 and x_0) and $\gamma^{(x_0)}\Phi' = (\gamma^{(x_0)}\Phi)'$ (a constant matrix comes out of the derivative); in the second term group the factors as $(-3H\gamma^{(x_0)})(\gamma^{(x_0)}\Phi)$; on the right write $-M = M''$:

$$\gamma^{(x_4)}\big(\gamma^{(x_0)}\Phi\big)' + \big(-3H\gamma^{(x_0)}\big)\big(\gamma^{(x_0)}\Phi\big) = M''\big(\gamma^{(x_0)}\Phi\big),$$

which is exactly the condensate equation on the mirror patch for the field $\gamma^{(x_0)}\Phi$ with the effective mass $M'' = -M$. (Status: PROVED here by hand; checked exactly by Notebook 20b, In [9].) Its tensor is the pulled-back one, $R_0 T R_0$, which for the diagonal tensor of a condensate is T itself: the SAME energy density 12 and pressure -24. Its charge density is $(\gamma^{(x_0)}\Phi)^\dagger B(\gamma^{(x_0)}\Phi) = \Phi^\dagger\gamma^{(x_0)}B\gamma^{(x_0)}\Phi = \Phi^\dagger B\Phi = -3$, the SAME as the universe's, because $\gamma^{(x_0)}$ is real and symmetric and $\gamma^{(x_0)}B\gamma^{(x_0)\dagger} = +B$ (`python-pairing.json`, check `Q.T2_image_keeps_B`). The mirror is an isometry, so the Einstein tensor is the same on both patches, and the copy alone solves all 64 Einstein equations on the mirror patch. The universe on the patch and its copy on the mirror patch together fill the Z2-doubled geometry with twice the energy and twice the charge, not zero. Whether the two halves can be joined at the brane (a junction condition, a brane tension) is not derived anywhere in the record: OPEN; the Z2 construction is ASSUMED.

The bookkeeping. In units $H = \kappa = 1$:

object	m	λ	ρ	p	J^{x_4}	S	**alone a source?**
universe	15	$-25/3$	12	-24	-3	12/5	yes
T1 partner	-15	25/3	-12	24	3	12/5	no
T1 pair	both	both	0	0	0	24/5	no (C1)
T2 copy	-15	$-25/3$	12	-24	-3	$-12/5$	yes (mirror)
T2 pair	both	both	24	-48	-6	0	each half

The objects are the universe Φ, its T1 partner $\Gamma\Phi$ and its T2 copy $\gamma^{(x_0)}\Phi$. The last column says whether the object alone is a source of the author's metric in Einstein gravity: the universe with $a_4 = \pm Hx_4$ and $\Lambda = -36H^2$ on the patch; its T1 partner for no a_4 and no Λ; the T2 copy on the mirror patch. The universe and its T1 partner live on the patch, the T2 copy on the mirror patch. The row of the T1 pair adds the two members at the same point; the row of the T2 pair adds the two members each at its own point (so its ρ is a total of two halves, not a density at one point).

What the example shows, and what it does not. It shows: (1) a universe of this theory can be a complete solution of the coupled equations with no partner at all; (2) its T1 partner exists as a solution of a DIFFERENT field equation (mass -15, coupling $25/3$), with the same frequency and the opposite energy and charge, and it cannot be the source of the author's metric in Einstein gravity; (3) the T1 pair has zero energy and charge and therefore cannot be that source either (C1); (4) the T2 copy is an ordinary solution with the same energy and charge on the mirror patch. It does NOT show that the partner or the copy is created, or must exist; it derives no process, rate or amplitude; it says nothing about the quantised field dirac16complex; and it does not address the stability of the condensate (OPEN).

20.13 Example: one universe and its two partners, computed

Notebook 20b builds the universe of Section 20.12 with exact arithmetic and the author's eight gamma matrices, checks its field equation exactly at every time, evaluates its complete energy-momentum tensor at nine points of the author's metric (three values of the hidden coordinate times three times x_4) and checks all 64 Einstein equations there; then it does the same for the T1 partner, the T1 pair and the T2 mirror copy. It prints 25 PASS lines, 14 of which reproduce a named check of a Revision report, and draws four figures. It needs numpy, sympy and matplotlib, no Rust, and runs in about 30 seconds (its recorded check run on the build computer took 18.7 seconds).

20.14 How to run Notebook 20b

Step 1. What this notebook does and what it needs.

Notebook 20b (One universe and its two partners: an exact case study) is the file `Revision/textbook/notebooks/20b_one_universe_partners.ipynb` of the repository Dirac_claude. It builds, with exact arithmetic and the author's gamma matrices, one exact solution of the coupled equations: a homogeneous condensate of dirac16complex00 that is by itself the complete source of the author's metric with the exponentially deflating history a4 = H x4 in Einstein gravity. It then builds its T1 partner Gamma Phi with mass and coupling reversed, the T1 pair, and its T2 mirror copy across the Z2 brane, checks the field equation of each exactly and the Einstein equations of each numerically at many points, and shows: the universe needs no partner; the T1 partner solves its own field equation but cannot be the source of any member of the author's family in Einstein gravity; the T1 pair cannot either (corollary C1); the T2 copy is a solution with equal energy on the mirror patch. It reproduces the Revision records where they overlap and draws four teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at

the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 20b_one_universe_partners.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 20b_one_universe_partners.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 20b_one_universe_partners.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/20b.captions.json`

- `Revision/textbook/figures/20b_1_einstein_residuals.png`
- `Revision/textbook/figures/20b_2_null_requirement.png`
- `Revision/textbook/figures/20b_3_bookkeeping.png`
- `Revision/textbook/figures/20b_4_partner_in_time.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 25 CHECKS PASSED (notebook 20b)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/20b_one_universe_partners.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 20b_one_universe_partners.ipynb
python -m nbconvert --execute --inplace 20b_one_universe_partners.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads seven files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone (a notebook copied alone to another folder cannot find them).

20.15 Notebook 20b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/20b_one_universe_partners.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The pairing theorems are statements about ALL solutions at once. This notebook
follows them through ONE exact solution, so that every statement becomes a
number. The solution (the **universe** of this notebook) is a homogeneous
condensate $\Phi(x_4)$ of the commuting field dirac16complex00 that is the
COMPLETE source of the author's metric with the exponentially deflating history
$a_4 = Hx_4$ in Einstein gravity: the field equation (16 components) and the
Einstein equations (64 components) hold together. Then the notebook builds

- the **T1 partner** $\Gamma\Phi$ with the mass and the coupling reversed: it
  solves its own field equation, has the same frequency, the opposite energy
```

```
  density and charge density, and is NOT a source of the author's metric;
- the **T1 pair** (universe plus T1 partner) as the only source: its source is
  zero, and the Einstein equations fail (corollary C1);
- the **T2 mirror copy** $\gamma^{(x_0)}\Phi$ with the mass reversed and the same
  coupling, placed on the mirror patch across the Z2 brane: it solves the coupled
  equations there, with the SAME energy density and charge density.

It checks every field equation exactly (sympy), every energy-momentum tensor and
every Einstein equation numerically at many points (rounding $10^{-10}$), and
draws four teaching figures. The lesson is in section 13: a universe is a complete
solution without any partner; the theorems pair solutions, they do not create
them.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 20b (One universe and its two partners: an exact case study) is the file
‘Revision/textbook/notebooks/20b_one_universe_partners.ipynb‘ of the repository
Dirac_claude. It builds, with exact arithmetic and the author's gamma matrices, one exact
solution of the coupled equations: a homogeneous condensate of dirac16complex00 that is
by itself the complete source of the author's metric with the exponentially deflating
history a4 = H x4 in Einstein gravity. It then builds its T1 partner Gamma Phi with mass
and coupling reversed, the T1 pair, and its T2 mirror copy across the Z2 brane, checks
the field equation of each exactly and the Einstein equations of each numerically at many
points, and shows: the universe needs no partner; the T1 partner solves its own field
equation but cannot be the source of any member of the author's family in Einstein
gravity; the T1 pair cannot either (corollary C1); the T2 copy is a solution with equal
energy on the mirror patch. It reproduces the Revision records where they overlap and
draws four teaching figures. It needs a computer with Windows 11, macOS or Linux, an
internet connection for the installation, the program Git, and Python 3.12 or newer (the
notebooks were built with Python 3.14.5) with these packages at exactly these versions:
numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat
5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself
imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that
shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:
```

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
```

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 20b_one_universe_partners.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 20b_one_universe_partners.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 20b_one_universe_partners.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/20b.captions.json‘
- ‘Revision/textbook/figures/20b_1_einstein_residuals.png‘
- ‘Revision/textbook/figures/20b_2_null_requirement.png‘
- ‘Revision/textbook/figures/20b_3_bookkeeping.png‘
- ‘Revision/textbook/figures/20b_4_partner_in_time.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 25 CHECKS PASSED (notebook 20b)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/20b_one_universe_partners.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 20b_one_universe_partners.ipynb
  python -m nbconvert --execute --inplace 20b_one_universe_partners.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads seven files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone (a notebook copied alone to another folder cannot find them).

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 20b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 20b (One universe and its two partners: an exact case study) is the file
# Revision/textbook/notebooks/20b_one_universe_partners.ipynb of the repository
# Dirac_claude. It builds, with exact arithmetic and the author's gamma matrices, one
# exact solution of the coupled equations: a homogeneous condensate of dirac16complex00
# that is by itself the complete source of the author's metric with the exponentially
# deflating history a4 = H x4 in Einstein gravity. It then builds its T1 partner Gamma
# Phi with mass and coupling reversed, the T1 pair, and its T2 mirror copy across the Z2
# brane, checks the field equation of each exactly and the Einstein equations of each
# numerically at many points, and shows: the universe needs no partner; the T1 partner
# solves its own field equation but cannot be the source of any member of the author's
# family in Einstein gravity; the T1 pair cannot either (corollary C1); the T2 copy is a
# solution with equal energy on the mirror patch. It reproduces the Revision records
# where they overlap and draws four teaching figures. It needs a computer with Windows
# 11, macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
```

```
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
```

```
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 20b_one_universe_partners.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
```

```
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 20b_one_universe_partners.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 20b_one_universe_partners.ipynb (in the same folder, without running the cells again)
# to read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/20b.captions.json
# - Revision/textbook/figures/20b_1_einstein_residuals.png
# - Revision/textbook/figures/20b_2_null_requirement.png
# - Revision/textbook/figures/20b_3_bookkeeping.png
# - Revision/textbook/figures/20b_4_partner_in_time.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 25 CHECKS PASSED (notebook 20b)
#
# and the notebook must show 4 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/20b_one_universe_partners.ipynb inside the folder
```

```
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 20b_one_universe_partners.ipynb
#     python -m nbconvert --execute --inplace 20b_one_universe_partners.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file below the folder Revision: the notebook reads seven
#   files of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone
#   (a notebook copied alone to another folder cannot find them).
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "20b"  # this notebook: chapter 20, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
```

```
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
```

```
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 20b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook
```

- **Coordinates** $x_0, \dots, x_7$ (the author's names): x_1, x_2, x_3 ordinary 3-space (inflating), x_4 the time, x_5, x_6, x_7 the three EXTRA TIMES (deflating exponentially: scale factor $e^{-a_4}\sin^{1/6}z$), x_0 the hidden direction with $z = 6Hx_0$; the **patch** $0 < z < \pi/2$, the **mirror patch** $\pi/2 < z < \pi$, the **brane** $z = \pi/2$ between them (the Z2 construction across it is ASSUMED). In the code positions 0 to 7 stand for x_0 to x_7.
- **Gamma matrices** $\gamma^{(a)}$, $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$, **chirality** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$, $B = -iC\gamma^{(x_4)}$.
- **Spinor** Φ: 16 complex numbers (here functions of x_4); $\bar\Phi = \Phi^\dagger C$; **density** $S = \bar\Phi\Phi$; **charge density** $J^{x_4} = \Phi^\dagger B\Phi$.
- **Mass** m, **coupling** λ, potential $U = \tfrac\lambda2S^2$, **effective mass** $M = m + \lambda S$.
- **Condensate**: a solution that depends on the time x_4 only. **Frequency** w: $\Phi \propto e^{-iwx_4}$.
- **Energy-momentum tensor** $T^\mu{}_\nu$ in the sign convention of the record of the field equations ($\sigma_T = +1$): energy density $\rho = -T^{x_4}{}_{x_4}$, pressures $p_\mu = T^\mu{}_\mu$ (no sum).
- **Einstein equations** $G^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu$; the **residual** $R^\mu{}_\nu = G^\mu{}_\nu + \Lambda\delta^\mu_\nu - \kappa T^\mu{}_\nu$ is zero exactly when they hold.
- **Linear member** $a_4 = AHx_4$: $A = 1$ is the author's deflating history.
- **Null combination** $\rho + p_0$; **units** $H = \kappa = 1$.

Text cell:

```
## 4. The physical and mathematical situation

**The coupled equations.** The field: $\gamma^\mu D_\mu\Phi = (m + \lambda S)\Phi$.
For a condensate only the $x_4$ derivative survives, and with the spin-connection
term $\sum_\mu\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$ of the record the equation
becomes $\Phi' = \mathcal{A}\Phi$ with $\mathcal{A} = -\gamma^{(x_4)}(M -
3H\gamma^{(x_0)})$ (the record's 'condensate_equation_x0_consistent'). Gravity: the
record gives, for the linear member and a condensate with all 15 three-gamma
bilinears zero, $\rho = mS + \tfrac\lambda2S^2$, $p_3 = p_t = p_0 = \tfrac\lambda2
S^2$, and the Einstein equations reduce to two conditions:

$$\kappa MS = -6(A^2 + 1)H^2,\qquad \kappa mS = -(36H^2 + 2\Lambda).$$

**The theorems used (PROVED in the Revision record).** T1: $\Gamma\Phi$ solves the
$(-m, -\lambda)$ field equation, $T \to -T$, $J \to -J$. T2: $\gamma^{(x_0)}\Phi$
placed at the mirror point solves the $(-m, \lambda)$ field equation on the mirror
patch, with the pulled-back (equal) $T$ and $J$. C1: a T1 pair as the only source
is a zero source and has no Einstein solution. The record's null combination:
in Einstein gravity every source of the author's metric must have
$\kappa(\rho + p_0) = -6(a_4'^2 + H^2) < 0$.

**What is new here (COMPUTED in this notebook, not a Revision record).** The
explicit universe and its partners, and the conclusions of sections 10 to 12.

**ASSUMED:** Einstein gravity; the sign convention $\sigma_T = +1$; the classical
commuting field dirac16complex00 (not the quantised dirac16complex); the chosen
```

```
numbers $A = 1$, $\Lambda = -36H^2$, $M = -5H$; the Z2 construction across the
brane (section 12).
```

Text cell:

```
## 5. The Revision records this notebook reproduces

The next cell defines the helpers ‘read_json‘ (reads a JSON file of the
repository), ‘record_verdict‘ (the verdict of a named check in a report) and
‘reproduces‘ (a check that passes only when this notebook’s own result holds AND
the named record check is PASS). It then reads one sentence of the record’s list of
what the pairing theorems do NOT establish; section 13 shows it on the example.
```

In [2]:

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory

GAMMAS = "Revision/algebra/gammas.json"  # the author’s gamma matrices
A4_RECORD = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
A4_PY = "Revision/field_equations_a4/reports/python-a4-report.json"
A4_WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
PAIR_PY = "Revision/pairing/reports/python-pairing.json"
PAIR_WL = "Revision/pairing/reports/wolfram-pairing.json"
PROOFS = "Revision/docs/PAIR_CREATION_PROOFS.md"


def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict ("PASS") of the check called name in a report; None if absent."""
    for entry in read_json(report_file)["checks"]:
        if entry["name"] == name:
            return entry["verdict"]
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name,
              record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # both lines with one print call


NO_NECESSITY = ("a single universe with mass $+m$ is an equally valid solution "
                "without its partner")
```

```
proofs = repository_file(PROOFS).read_text(encoding="utf-8")
say(f"{PROOFS} says: \"... a single universe with mass +m is an equally valid "
    "solution without its partner.\"")
check(NO_NECESSITY in proofs, "the record states that no partner is required")
```

Out [2]:

```
Revision/docs/PAIR_CREATION_PROOFS.md says: "... a single universe with mass +m is an
    equally valid solution without its partner."
PASS the record states that no partner is required
```

Text cell:

```
## 6. The gamma matrices and the spin connection on both patches

The next cell reads the author's gammas (exact whole numbers), builds $C$,
$\Gamma$, $B$ and $S^{ab}$, and computes the canonical spin connection of the
author's metric exactly as Notebook 20a does: lengths $E = (s_0\cot z, e^{a_4}s,
e^{a_4}s, e^{a_4}s, 1, e^{-a_4}s, e^{-a_4}s, e^{-a_4}s)$ with $s = \sin^{1/6}z$,
$s_0 = +1$ on the patch and $-1$ on the mirror patch, Christoffel symbols of the
diagonal metric, and $\omega_{\mu ab} = \eta_{aa}(E_a/E_b)\Gamma^a{}_{\mu b}$ for
$a \neq b$. It checks the record's $\sum_\mu\gamma^\mu\Omega_\mu = 3H\gamma^{(x_0)}$
on the patch and finds $-3H\gamma^{(x_0)}$ on the mirror patch.
```

In [3]:

```
import itertools  # loops over all combinations of indices

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the frame metric eta, x0..x7
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
gamma = [sp.Matrix([[sp.Rational(x) for x in row] for row in matrix])
         for matrix in read_json(GAMMAS)["gamma"]]  # gamma[a] = gamma^(x_a)
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g^(x0) g^(x1) g^(x2) g^(x3)
chirality = gamma[0]  # Gamma = g^(x0) g^(x1) ... g^(x7), factor by factor
for a in range(1, 8):
    chirality = chirality * gamma[a]
B = -sp.I * C * gamma[4]  # the Krein matrix B = -i C gamma^(x4)
S_exact = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
           for a in range(8)]
H = sp.symbols("H", positive=True)
z = sp.symbols("z", real=True)
x4 = sp.symbols("x4", real=True)
a4 = sp.Function("a4")(x4)
a4_value, slope = sp.symbols("a4_value slope", real=True)


def d(expr, mu):
```

```
    """The partial derivative along the coordinate at position mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # d/dx0 = 6 H d/dz
    return sp.Integer(0)


def spin_connection(s0):
    """The lengths E_a and omega[mu][a][b] = omega_mu ab (s0 = +1 patch, -1 mirror)."""
    s = sp.sin(z) ** sp.Rational(1, 6)
    E = [s0 * sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
    E = E + [sp.exp(-a4) * s] * 3  # x0, x1, ..., x7
    g = [ETA[a] * E[a] ** 2 for a in range(8)]
    christoffel = {}
    for a, b, c in itertools.product(range(8), repeat=3):
        value = 0
        if a == c:
            value += d(g[a], b)
        if a == b:
            value += d(g[a], c)
        if b == c:
            value -= d(g[b], a)
        christoffel[a, b, c] = value / (2 * g[a])
    omega = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
    for mu, a, b in itertools.product(range(8), repeat=3):
        if a != b:
            value = ETA[a] * E[a] * christoffel[a, mu, b] / E[b]
            value = value.subs(sp.Derivative(a4, x4), slope).subs(a4, a4_value)
            omega[mu][a][b] = sp.simplify(value)
    return [length.subs(a4, a4_value) for length in E], omega


CONNECTION = {s0: spin_connection(s0) for s0 in (1, -1)}
gravity_term = {}  # s0 -> sum over mu of gamma^mu Omega_mu
for s0, (E, omega) in CONNECTION.items():
    Omega = [sum((omega[mu][a][b] * S_exact[a][b] / 2 for a in range(8)
                  for b in range(8) if omega[mu][a][b] != 0), Z16)
             for mu in range(8)]
    gravity_term[s0] = sum((gamma[mu] / E[mu] * Omega[mu] for mu in range(8)),
                           Z16).applyfunc(sp.simplify)
reproduces(gravity_term[1] == 3 * H * gamma[0],
           "patch: gamma^mu Omega_mu = 3 H gamma^(x0)", A4_PY, "authorT16_gravity_term")
check(gravity_term[-1] == -3 * H * gamma[0],
      "mirror patch: gamma^mu Omega_mu = -3 H gamma^(x0)")
```

Out [3]:

```
PASS patch: gamma^mu Omega_mu = 3 H gamma^(x0)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_gravity_term
PASS mirror patch: gamma^mu Omega_mu = -3 H gamma^(x0)
```

Text cell:

```
## 7. Choosing the universe: the two Einstein conditions

We fix the geometry first: $H = \kappa = 1$, the author's deflating history
$A = 1$ ($a_4 = Hx_4$; the extra times shrink as $e^{-x_4}$), and the cosmological
constant $\Lambda = -36H^2$, chosen so that the universe has a positive energy
density. We choose the effective mass $M = m + \lambda S = -5H$ (then the
condensate oscillates, because $|M| > 3H$, with $w = \sqrt{M^2 - 9H^2} = 4H$). The
two conditions then fix everything, line by line:

1. $\kappa MS = -6(A^2 + 1)H^2 = -12$, so $S = -12/M = 12/5$;
2. $\kappa mS = -(36 + 2\Lambda) = 36$, so $m = 36/S = 15$;
3. $M = m + \lambda S$, so $\lambda = (M - m)/S = -20/(12/5) = -25/3$;
4. $\rho = mS + \tfrac\lambda2S^2 = 36 - 24 = 12$ and $p = \tfrac\lambda2S^2 = -24$.

The next cell checks the two conditions against the record (it rebuilds them from
the record's Einstein tensor) and solves them with sympy.
```

In [4]:

```
record = read_json(A4_RECORD)
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
A, Lam, kappa = sp.symbols("A Lambda kappa", real=True)
m, lam, S_sym = sp.symbols("m lambda S", real=True)


def einstein(key):
    """The component key of the Einstein tensor E_(1) from the record."""
    text = record["lovelockTensors"]["E1"][key]["input"].replace("^", "**")
    return sp.sympify(text, locals={"ad1": ad1, "ad2": ad2, "H": H})


G = {key: einstein(key) for key in ("x1x1", "x4x4", "x5x5", "x0x0")}
LINEAR = {ad1: A * H, ad2: 0}  # the linear member a4 = A H x4
rho_c, p_c = m * S_sym + lam * S_sym ** 2 / 2, lam * S_sym ** 2 / 2  # condensate
time_eq = G["x4x4"].subs(LINEAR) + Lam + kappa * rho_c  # = 0
hidden_eq = G["x0x0"].subs(LINEAR) + Lam - kappa * p_c  # = 0
first = sp.expand(time_eq - hidden_eq)  # kappa (m + lambda S) S + 6 (A^2 + 1) H^2
second = sp.expand(time_eq + hidden_eq)  # kappa m S + 36 H^2 + 2 Lambda
reproduces(sp.expand(first - (kappa * (m + lam * S_sym) * S_sym
                              + 6 * (A ** 2 + 1) * H ** 2)) == 0
           and sp.expand(second - (kappa * m * S_sym + 36 * H ** 2 + 2 * Lam)) == 0,
           "Einstein: kappa M S = -6 (A^2 + 1) H^2, kappa m S = -(36 H^2 + 2 Lambda)",
           A4_PY, "condensate_einstein_quadratic_U")
CHOICE = {H: 1, kappa: 1, A: 1, Lam: -36}  # the geometry of this notebook
M_VALUE = -5  # the effective mass M = m + lambda S
S_value = sp.solve(first.subs(CHOICE).subs(lam, (M_VALUE - m) / S_sym), S_sym)[0]
m_value = sp.solve(second.subs(CHOICE).subs(S_sym, S_value), m)[0]
lam_value = (M_VALUE - m_value) / S_value
rho_value = rho_c.subs({m: m_value, lam: lam_value, S_sym: S_value})
p_value = p_c.subs({lam: lam_value, S_sym: S_value})
```

```
for label, value in (("S", S_value), ("m", m_value), ("lambda", lam_value),
                     ("rho", rho_value), ("p", p_value)):
    report(label, value)
check((S_value, m_value, lam_value, rho_value, p_value)
      == (sp.Rational(12, 5), 15, sp.Rational(-25, 3), 12, -24),
      "the universe: S = 12/5, m = 15, lambda = -25/3, rho = 12, p = -24")
```

Out [4]:

```
PASS Einstein: kappa M S = -6 (A^2 + 1) H^2, kappa m S = -(36 H^2 + 2 Lambda)
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    condensate_einstein_quadratic_U
RESULT S = 12/5
RESULT m = 15
RESULT lambda = -25/3
RESULT rho = 12
RESULT p = -24
PASS the universe: S = 12/5, m = 15, lambda = -25/3, rho = 12, p = -24
```

Text cell:

```
## 8. Building the universe: the condensate $\Phi(x_4)$

The record's recipe for a condensate whose 15 three-gamma bilinears vanish (they
must, for the off-diagonal Einstein equations), at $M = -5$, $H = 1$, $w = 4$: the
spinor $v_1$ with $\mathcal{A}v_1 = -iwv_1$ and $\gamma^{(x_1)}\gamma^{(x_5)} =
\gamma^{(x_2)}\gamma^{(x_6)} = \gamma^{(x_3)}\gamma^{(x_7)} = -1$ on it, the
spinor $v_2$ with the same frequency and the value $+1$, and $c =
\overline{v_1^\dagger Cv_2}$; each is the null space of a stacked matrix. The
family $\Phi_0(t) = v_1 + t\,c\,v_2$ keeps all 15 bilinears zero for every real
$t$ and has $S(t) = 2t|v_1^\dagger Cv_2|^2$, so $t$ is fixed by $S = 12/5$. The
universe is $\Phi(x_4) = e^{-iwx_4}\Phi_0$. The next cell builds it, checks the 15
bilinears and $S$, checks that $S$ is constant ($C\mathcal{A} + \mathcal{A}^TC =
0$), checks the 16 components of the field equation $\gamma^{(x_4)}\Phi' +
3H\gamma^{(x_0)}\Phi = (m + \lambda S)\Phi$ EXACTLY at every time $x_4$, and
computes the charge density $J^{x_4} = \Phi^\dagger B\Phi$.
```

In [5]:

```
w = sp.sqrt(M_VALUE ** 2 - 9)  # the frequency, H = 1
A_matrix = -gamma[4] * (M_VALUE * I16 - 3 * gamma[0])  # Phi' = A Phi
spaces = []
for sign in (-1, 1):  # the value of g^(x1)g^(x5) = g^(x2)g^(x6) = g^(x3)g^(x7)
    stack = sp.Matrix.vstack(A_matrix + sp.I * w * I16,
                             gamma[1] * gamma[5] - sign * I16,
                             gamma[2] * gamma[6] - sign * I16,
                             gamma[3] * gamma[7] - sign * I16)
    spaces.append(stack.nullspace())  # every solution of stack * v = 0
v1, v2 = spaces[0][0], spaces[1][0]
c = sp.conjugate(sp.expand((v1.H * C * v2)[0]))  # c = conj(v1^dagger C v2)
t = sp.symbols("t", real=True)
```

```
family = (v1 + t * c * v2).applyfunc(sp.expand)  # Phi_0(t)


def bilinear(phi, matrix):
    """phibar matrix phi = phi^dagger C matrix phi (exact)."""
    return sp.expand((phi.H * C * matrix * phi)[0])


t_value = sp.solve(bilinear(family, I16) - S_value, t)[0]
phi0 = family.subs(t, t_value).applyfunc(sp.expand)  # the universe at x4 = 0
report("t", t_value)
NEEDED = sorted({tuple(sorted((i, 4, 0))) for i in (1, 2, 3, 5, 6, 7)}
                | {tuple(sorted((i, j, 4))) for i in (1, 2, 3) for j in (5, 6, 7)})
zero_15 = all(bilinear(family, gamma[a] * gamma[b] * gamma[e]) == 0
              for a, b, e in NEEDED)  # for every real t, so also for t_value
reproduces([len(space) for space in spaces] == [1, 1] and zero_15
           and bilinear(phi0, I16) == S_value,
           "the universe: 15 three-gamma bilinears 0 and S = 12/5",
           A4_WL, "condensate_diagonal_witness_exact")
reproduces((C * A_matrix + A_matrix.T * C).applyfunc(sp.expand) == Z16,
           "C A + A^T C = 0: S stays 12/5 at every time", A4_PY,
           "authorT16_condensate_S_constant")
Phi = phi0 * sp.exp(-sp.I * w * x4)  # the universe Phi(x4)


def field_residual(field, term, mass, coupling, density):
    """gamma^(x4) field' + term field - (mass + coupling density) field."""
    return (gamma[4] * sp.diff(field, x4) + term * field
            - (mass + coupling * density) * field).applyfunc(sp.simplify)


universe_ok = field_residual(Phi, gravity_term[1].subs(H, 1), m_value, lam_value,
                             S_value) == sp.zeros(16, 1)
reproduces(universe_ok, "the universe solves its field equation (16 components)",
           A4_WL, "condensate_equation_x0_consistent")
charge_value = sp.simplify((phi0.H * B * phi0)[0])  # J^x4, constant in time
report("charge density J^x4 of the universe", charge_value)
```

Out [5]:

```
RESULT t = 15/512
PASS the universe: 15 three-gamma bilinears 0 and S = 12/5
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_diagonal_witness_exact
PASS C A + A^T C = 0: S stays 12/5 at every time
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_S_constant
PASS the universe solves its field equation (16 components)
     reproduces Revision/field_equations_a4/reports/wolfram-a4-report.json, check
    condensate_equation_x0_consistent
RESULT charge density J^x4 of the universe = -3
```

Text cell:

9. The universe alone solves the Einstein equations

The next cell turns the geometry into numpy functions and evaluates the complete energy-momentum tensor of a configuration from its first jet, as in Notebook 20a: the value Φ and the derivatives (here only $\partial_4\Phi = -iw\Phi$; the condensate does not depend on the other coordinates). From the pairing records' tensor $T_{\mu\nu}$ it forms the mixed tensor of this notebook's convention, $T^\mu{}_\nu = -g^{\mu\mu}T_{\mu\nu}$ (the two conventions differ by a sign), and the Einstein residual $R^\mu{}_\nu = G^\mu{}_\nu + \Lambda\delta^\mu_\nu - \kappa T^\mu{}_\nu$ with G from the record at $a_4' = H$. It does this at nine points: $z = 0.3, 0.8, 1.3$ on the patch and the times $x_4 = 0, 0.7, 2$ (where $a_4 = x_4$). All 64 components of the residual must vanish at every point.

In [6]:

```
point_symbols = (z, a4_value, slope, H)
S_num = [[np.array(S_exact[a][b].tolist(), dtype=float) for b in range(8)]
         for a in range(8)]
gamma_num = [np.array(g.tolist(), dtype=float) for g in gamma]
C_num = np.array(C.tolist(), dtype=float)
Gamma_num = np.array(chirality.tolist(), dtype=float)
B_num = np.array(B.tolist(), dtype=complex)
NUMERIC = {}
for s0, (E, omega) in CONNECTION.items():
    pieces = [(mu, a, b, sp.lambdify(point_symbols, omega[mu][a][b], "numpy"))
              for mu in range(8) for a in range(8) for b in range(8)
              if omega[mu][a][b] != 0]
    NUMERIC[s0] = (sp.lambdify(point_symbols, E, "numpy"), pieces)


def geometry_at(s0, z_value, a4_number, slope_number):
    """Metric, Omega_mu and curved gammas at one point (H = 1)."""
    lengths_f, pieces = NUMERIC[s0]
    values = (z_value, a4_number, slope_number, 1.0)
    E = np.array([float(x) for x in lengths_f(*values)])
    Om = [np.zeros((16, 16)) for _ in range(8)]
    for mu, a, b, f in pieces:
        Om[mu] += float(f(*values)) * S_num[a][b] / 2
    return {"g": np.array(ETA) * E ** 2, "Omega": Om,
            "up": [gamma_num[mu] / E[mu] for mu in range(8)],
            "down": [ETA[mu] * E[mu] * gamma_num[mu] for mu in range(8)]}


def mixed_tensor(at, psi, dpsi, mass, coupling):
    """T^mu_nu (convention sigma_T = +1) and K^mu_mu of a first jet."""
    bar = psi.conj() @ C_num
    dbar = [row.conj() @ C_num for row in dpsi]
    D = [dpsi[mu] + at["Omega"][mu] @ psi for mu in range(8)]
    Dbar = [dbar[mu] - bar @ at["Omega"][mu] for mu in range(8)]
    S = (bar @ psi).real
    kinetic = [0.5 * (bar @ at["up"][mu] @ D[mu] - Dbar[mu] @ at["up"][mu] @ psi)
               for mu in range(8)]  # K^mu_mu, no sum
    L = sum(kinetic).real - mass * S - coupling * S ** 2 / 2
```

```
    T = np.zeros((8, 8))
    for mu, nu in itertools.product(range(8), repeat=2):
        low = 0.25 * (bar @ at["down"][mu] @ D[nu] + bar @ at["down"][nu] @ D[mu]
                      - Dbar[mu] @ at["down"][nu] @ psi
                      - Dbar[nu] @ at["down"][mu] @ psi).real
        if mu == nu:
            low -= at["g"][mu] * L  # the pairing records' T_mu nu
        T[mu, nu] = -low / at["g"][mu]  # T^mu_nu = -g^(mu mu) T_mu nu
    return T, np.array([k.real for k in kinetic])


G_numeric = np.diag([float(G[k].subs({ad1: 1, ad2: 0, H: 1})) for k in
                     ("x0x0", "x1x1", "x1x1", "x1x1", "x4x4", "x5x5", "x5x5",
                      "x5x5")])  # G^mu_nu at a4' = H, H = 1, in the order x0..x7
LAMBDA = -36.0
phi0_num = np.array([complex(entry) for entry in phi0])
W = float(w)
POINTS = [(zv, xv) for zv in (0.3, 0.8, 1.3) for xv in (0.0, 0.7, 2.0)]


def universe_jet(x_value, matrix=None):
    """Value and derivatives of matrix Phi at the time x_value."""
    value = phi0_num * np.exp(-1j * W * x_value)
    if matrix is not None:
        value = matrix @ value
    jet = [np.zeros(16, dtype=complex) for _ in range(8)]
    jet[4] = -1j * W * value  # d Phi / d x4 = -i w Phi
    return value, jet


worst_residual, worst_kinetic = 0.0, 0.0
for z_value, x_value in POINTS:
    at = geometry_at(1, z_value, x_value, 1.0)  # a4 = x4, a4' = 1
    T, kinetic = mixed_tensor(at, *universe_jet(x_value), float(m_value),
                              float(lam_value))
    residual = G_numeric + LAMBDA * np.eye(8) - T  # kappa = 1
    worst_residual = max(worst_residual, np.abs(residual).max())
    wanted = np.zeros(8)
    wanted[4] = M_VALUE * float(S_value)  # K^x4_x4 = M S, all others 0
    worst_kinetic = max(worst_kinetic, np.abs(kinetic - wanted).max())
T_universe = T  # the tensor at the last point (it is the same at every point)
report("rho of the universe", f"{-T_universe[4, 4]:.10f}")
report("p of the universe", f"{T_universe[1, 1]:.10f}")
reproduces(worst_kinetic < 1e-10, "K^x4_x4 = M S and K^mu_mu = 0 otherwise (9 points)",
           A4_PY, "authorT16_condensate_kinetic_diagonal")
check(worst_residual < 1e-10,
      "the universe ALONE solves all 64 Einstein equations (9 points)")
```

Out [6]:

```
RESULT rho of the universe = 12.0000000000
RESULT p of the universe = -24.0000000000
PASS K^x4_x4 = M S and K^mu_mu = 0 otherwise (9 points)
```

```
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    authorT16_condensate_kinetic_diagonal
PASS the universe ALONE solves all 64 Einstein equations (9 points)
```

Text cell:

```
## 10. The T1 partner $\Gamma\Phi$

By T1 the partner $\Gamma\Phi$ solves the field equation with $(-m, -\lambda) =
(-15, 25/3)$. Its density is $S' = S$ (the scalar is unchanged), so its effective
mass is $M' = -m - \lambda S = -M = 5$. Its matrix $\mathcal{A}' =
-\gamma^{(x_4)}(M' - 3H\gamma^{(x_0)})$ equals $\Gamma\mathcal{A}\Gamma$, so it
has the SAME eigenvalues $\pm iw$: the partner oscillates with the same frequency
(similar matrices have equal spectra; the record proves the same for the
one-particle Hamiltonians, $\Gamma h_m\Gamma = h_{-m}$). Its tensor is $T' = -T$
and its charge density is $-J^{x_4}$. The next cell checks all of this exactly
where possible, and numerically for the tensor.

Can the partner ALONE be the source of the author's metric? The record's null
combination says that every source must have $\kappa(\rho + p_0) =
-6(a_4'^2 + H^2)$, which is at most $-6H^2$. The universe has $\kappa(\rho + p_0)
= 12 - 24 = -12$, which fits $a_4' = \pm H$. The partner has $\rho' + p_0' =
-(\rho + p_0) = +12$: positive, so it fits NO rate $a_4'$ and no $\Lambda$, in
Einstein gravity. (Its tensor does not depend on $a_4$, so this holds for every
member of the author's family.)
```

In [7]:

```
partner = chirality * Phi  # Gamma Phi
partner_ok = field_residual(partner, gravity_term[1].subs(H, 1), -m_value,
                            -lam_value, S_value) == sp.zeros(16, 1)
reproduces(partner_ok and bilinear(chirality * phi0, I16) == S_value,
           "Gamma Phi solves the (-m, -lambda) field equation; S' = S",
           PAIR_PY, "T1.metric.commuting.euler_lagrange_map")
A_partner = -gamma[4] * (-M_VALUE * I16 - 3 * gamma[0])  # M' = -M
same_spectrum = (chirality * A_matrix * chirality == A_partner
                 and (A_partner * A_partner + w ** 2 * I16) == Z16)
reproduces(same_spectrum, "A' = Gamma A Gamma: the partner has the same frequency",
           PAIR_WL, "Q_one_particle_maps")
partner_charge = sp.simplify(((chirality * phi0).H * B * (chirality * phi0))[0])
reproduces(partner_charge == -charge_value, "the partner's charge density is -J^x4",
           PAIR_PY, "T1.metric.commuting.current")
worst_sum, worst_partner = 0.0, 0.0
for z_value, x_value in POINTS:
    at = geometry_at(1, z_value, x_value, 1.0)
    T_one, _ = mixed_tensor(at, *universe_jet(x_value), float(m_value),
                            float(lam_value))
    T_two, _ = mixed_tensor(at, *universe_jet(x_value, Gamma_num),
                            -float(m_value), -float(lam_value))
    worst_sum = max(worst_sum, np.abs(T_one + T_two).max())
    residual = G_numeric + LAMBDA * np.eye(8) - T_two
    worst_partner = max(worst_partner, np.abs(residual - 2 * T_one).max())
```

```
T_partner = T_two
reproduces(worst_sum < 1e-10, "T' = -T at all 9 points (64 components each)",
           PAIR_PY, "T1.metric.commuting.emt")
check(worst_partner < 1e-10 and np.abs(2 * T_one).max() > 1.0,
      "partner alone: the Einstein residual is 2 kappa T, not zero")
null_universe = -T_universe[4, 4] + T_universe[0, 0]  # kappa (rho + p0)
null_partner = -T_partner[4, 4] + T_partner[0, 0]
report("kappa (rho + p0) of the universe", f"{null_universe:.10f}")
report("kappa (rho + p0) of the partner", f"{null_partner:.10f}")
rate = sp.symbols("rate", real=True)  # a real value of a4'/H
required = sp.expand(-(einstein("x4x4") - einstein("x0x0")).subs(
    {ad1: rate, ad2: 0, H: 1}))  # the requirement -6 (a4'^2 + 1)
universe_null = rho_value + p_value  # exact: 12 - 24 = -12
partner_null = -(rho_value + p_value)  # exact: T' = -T gives +12
check(abs(null_universe - float(universe_null)) < 1e-10
      and abs(null_partner - float(partner_null)) < 1e-10,
      "kappa (rho + p0): universe -12, partner +12 (numbers = exact values)")
reproduces(required == -6 * rate ** 2 - 6
           and sorted(sp.solve(required - universe_null, rate)) == [-1, 1]
           and sp.solve(required - partner_null, rate) == [],
           "the universe fits a4' = +-H; the partner fits no real rate",
           A4_PY, "einstein_null_energy_x0")
```

Out [7]:

```
PASS Gamma Phi solves the (-m, -lambda) field equation; S' = S
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.euler_lagrange_map
PASS A' = Gamma A Gamma: the partner has the same frequency
     reproduces Revision/pairing/reports/wolfram-pairing.json, check Q_one_particle_maps
PASS the partner's charge density is -J^x4
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.current
PASS T' = -T at all 9 points (64 components each)
     reproduces Revision/pairing/reports/python-pairing.json, check
    T1.metric.commuting.emt
PASS partner alone: the Einstein residual is 2 kappa T, not zero
RESULT kappa (rho + p0) of the universe = -12.0000000000
RESULT kappa (rho + p0) of the partner = 12.0000000000
PASS kappa (rho + p0): universe -12, partner +12 (numbers = exact values)
PASS the universe fits a4' = +-H; the partner fits no real rate
     reproduces Revision/field_equations_a4/reports/python-a4-report.json, check
    einstein_null_energy_x0
```

Text cell:

```
## 11. The T1 pair as the only source (corollary C1)

With the universe AND its T1 partner as the only source, the source is
$T + T' = 0$ and the residual is $G^\mu{}_\nu + \Lambda\delta^\mu_\nu$, which is
not zero for any $\Lambda$ (Notebook 20a). At $a_4' = H$, $\Lambda = -36H^2$ it is
$\mathrm{diag}(-24, -24, -24, -12, -24, -24, -24, -24)H^2$. The next cell checks
this at the nine points.
```

In [8]:

```
worst_pair = 0.0
for z_value, x_value in POINTS:
    at = geometry_at(1, z_value, x_value, 1.0)
    T_one, _ = mixed_tensor(at, *universe_jet(x_value), float(m_value),
                            float(lam_value))
    T_two, _ = mixed_tensor(at, *universe_jet(x_value, Gamma_num),
                            -float(m_value), -float(lam_value))
    residual = G_numeric + LAMBDA * np.eye(8) - (T_one + T_two)
    expected = np.diag([-24.0, -24, -24, -24, -12, -24, -24, -24])
    worst_pair = max(worst_pair, np.abs(residual - expected).max())
check(worst_pair < 1e-10,
      "T1 pair alone: residual = G + Lambda = diag(-24, ..., -12, ...), not zero")
```

Out [8]:

```
PASS T1 pair alone: residual = G + Lambda = diag(-24, ..., -12, ...), not zero
```

Text cell:

```
## 12. The T2 mirror copy across the brane

By T2 (with $m \to -m$) the field $\Phi'(\pi - z) = \gamma^{(x_0)}\Phi(z)$ on the
mirror patch solves the $(-m, \lambda) = (-15, -25/3)$ field equation there. For
the condensate $\Phi$ does not depend on $z$, so the copy is simply
$\gamma^{(x_0)}\Phi(x_4)$. Its density is $S' = -S$, so its effective mass is
$M' = -m + \lambda S' = -M$. On the mirror patch the spin-connection term is
$-3H\gamma^{(x_0)}$ (section 6). Its tensor is the pulled-back one, $R_0TR_0$,
which for the diagonal $T$ of the condensate is $T$ itself: the SAME energy
density $12$ and pressure $-24$. Its charge density is $+J^{x_4}$ (the record:
$\gamma^{(x_0)}B\gamma^{(x_0)\dagger} = +B$). Because the mirror is an isometry, the
Einstein tensor is the same on both patches. So the copy is a complete solution
of the coupled equations on the mirror patch: together with the universe it
fills the Z2-doubled geometry, with twice the energy, NOT zero. Whether the two
halves can be joined at the brane (a junction condition, a brane tension) is not
derived in the record (OPEN); the Z2 construction is ASSUMED.
```

In [9]:

```
copy = gamma[0] * Phi  # gamma^(x0) Phi on the mirror patch
copy_ok = field_residual(copy, gravity_term[-1].subs(H, 1), -m_value, lam_value,
                         -S_value) == sp.zeros(16, 1)
reproduces(copy_ok and bilinear(gamma[0] * phi0, I16) == -S_value,
           "the mirror copy solves the (-m, lambda) equation there; S' = -S",
           PAIR_PY, "T2.metric.commuting.euler_lagrange_map")
copy_charge = sp.simplify(((gamma[0] * phi0).H * B * (gamma[0] * phi0))[0])
reproduces(copy_charge == charge_value, "the copy's charge density is +J^x4",
           PAIR_PY, "Q.T2_image_keeps_B")
worst_copy, worst_pull = 0.0, 0.0
```

```
R0 = np.diag([-1.0, 1, 1, 1, 1, 1, 1, 1])  # the reflection of x0
for z_value, x_value in POINTS:
    there = geometry_at(-1, np.pi - z_value, x_value, 1.0)  # the mirror point
    T_copy, _ = mixed_tensor(there, *universe_jet(x_value, gamma_num[0]),
                             -float(m_value), float(lam_value))
    T_one, _ = mixed_tensor(geometry_at(1, z_value, x_value, 1.0),
                            *universe_jet(x_value), float(m_value), float(lam_value))
    worst_pull = max(worst_pull, np.abs(T_copy - R0 @ T_one @ R0).max())
    residual = G_numeric + LAMBDA * np.eye(8) - T_copy
    worst_copy = max(worst_copy, np.abs(residual).max())
T_mirror = T_copy
reproduces(worst_pull < 1e-10, "the copy's tensor is R0 T R0 = T at 9 mirror points",
           PAIR_PY, "T2.metric.commuting.emt")
check(worst_copy < 1e-10,
      "the mirror copy ALONE solves the 64 Einstein equations on the mirror patch")
```

Out [9]:

```
PASS the mirror copy solves the (-m, lambda) equation there; S' = -S
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.euler_lagrange_map
PASS the copy's charge density is +J^x4
     reproduces Revision/pairing/reports/python-pairing.json, check Q.T2_image_keeps_B
PASS the copy's tensor is R0 T R0 = T at 9 mirror points
     reproduces Revision/pairing/reports/python-pairing.json, check
    T2.metric.commuting.emt
PASS the mirror copy ALONE solves the 64 Einstein equations on the mirror patch
```

Text cell:

```
The next cell draws the Einstein residual $R^\mu{}_\nu$ (an $8 \times 8$ table)
for the four situations: the universe alone on the patch, the T1 partner alone,
the T1 pair, and the T2 mirror copy alone on the mirror patch. A white table means
that the Einstein equations hold.
```

In [10]:

```
tables = [
    ("universe $\\Phi$ alone", G_numeric + LAMBDA * np.eye(8) - T_universe),
    ("T1 partner $\\Gamma\\Phi$ alone", G_numeric + LAMBDA * np.eye(8) - T_partner),
    ("T1 pair (C1)", G_numeric + LAMBDA * np.eye(8) - T_universe - T_partner),
    ("T2 copy alone (mirror patch)", G_numeric + LAMBDA * np.eye(8) - T_mirror),
]
fig, axes = plt.subplots(1, 4, figsize=(14.0, 4.0))
for ax, (title, values) in zip(axes, tables):
    image = ax.imshow(values, cmap="RdBu_r", vmin=-48, vmax=48)
    for mu in range(8):
        if abs(values[mu, mu]) > 1e-6:
            ink = "white" if abs(values[mu, mu]) > 30 else "black"  # readable
            ax.text(mu, mu, f"{values[mu, mu]:.0f}", ha="center", va="center",
                    fontsize=6.5, color=ink)
```

```
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=6.5)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=6.5)
    ax.grid(False)
    ax.set_title(title, fontsize=9)
fig.colorbar(image, ax=list(axes), shrink=0.75, label="residual (units $H^2$)")
save_figure(fig, "einstein_residuals",
            "The residual $G^\\mu{}_\\nu + \\Lambda\\delta^\\mu_\\nu - \\kappa "
            "T^\\mu{}_\\nu$ of the Einstein equations (rows $\\mu$, columns $\\nu$, "
            "$x_0$ to $x_7$; units $H = \\kappa = 1$; white means zero, the numbers "
            "are the nonzero diagonal entries) for the author's metric with the "
            "deflating history $a_4 = Hx_4$ and $\\Lambda = -36H^2$, with four "
            "different sources: the universe $\\Phi$ (a condensate with "
            "$m = 15$, $\\lambda = -25/3$) alone, which solves all 64 equations; "
            "its T1 partner $\\Gamma\\Phi$ alone, residual $2\\kappa T$; the T1 pair, "
            "residual $G + \\Lambda$ (corollary C1); and the T2 mirror copy "
            "$\\gamma^{(x_0)}\\Phi$ alone on the mirror patch, which solves all 64 "
            "equations there.")
```

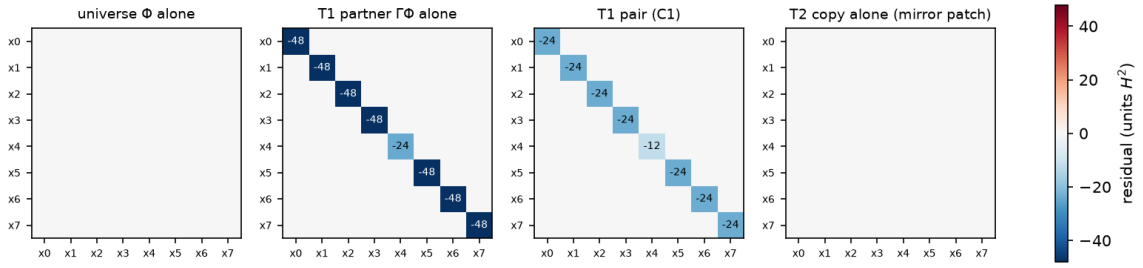

Figure 514: The residual $G^\mu{}_\nu + \Lambda\delta^\mu_\nu - \kappa T^\mu{}_\nu$ of the Einstein equations (rows μ, columns ν, x_0 to x_7; units $H = \kappa = 1$; white means zero, the numbers are the nonzero diagonal entries) for the author's metric with the deflating history $a_4 = Hx_4$ and $\Lambda = -36H^2$, with four different sources: the universe Φ (a condensate with $m = 15$, $\lambda = -25/3$) alone, which solves all 64 equations; its T1 partner $\Gamma\Phi$ alone, residual $2\kappa T$; the T1 pair, residual $G + \Lambda$ (corollary C1); and the T2 mirror copy $\gamma^{(x_0)}\Phi$ alone on the mirror patch, which solves all 64 equations there. (Notebook 20b, figure 1.)

Out [10]:

```
Figure 20b.1 saved as Revision/textbook/figures/20b_1_einstein_residuals.png
```

Text cell:

```
The next cell draws the null combination: the curve $\kappa(\rho + p_0) =
-6(a_4'^2 + H^2)$ that Einstein gravity requires of any source, against the rate
$a_4'/H$, and the values that the universe, the T1 partner, the T1 pair and the
T2 copy supply (each a constant, a horizontal line).
```

In [11]:

```
rates = np.linspace(-2.5, 2.5, 501)
fig, ax = plt.subplots(figsize=(7.0, 4.6))
ax.plot(rates, -6 * (rates ** 2 + 1), color="black", linewidth=2.0,
        label="required: $-6(a_4'^2 + H^2)$")
supplied = (("universe $\\Phi$", null_universe, "tab:blue", "-"),
            ("T1 partner $\\Gamma\\Phi$", null_partner, "tab:red", "-"),
            ("T1 pair", null_universe + null_partner, "tab:purple", "--"),
            ("T2 copy", -T_mirror[4, 4] + T_mirror[0, 0], "tab:green", ":"))
for label, value, colour, style in supplied:
    ax.axhline(value, color=colour, linestyle=style, linewidth=1.6,
               label=f"{label} supplies {value:+.0f}")
ax.plot([-1.0, 1.0], [-12.0, -12.0], "o", color="tab:blue", markersize=8,
        label="solutions: $a_4' = \\pm H$")
ax.set_xlabel("rate $a_4'/H$ of the metric function")
ax.set_ylabel("$\\kappa(\\rho + p_0)/H^2$")
ax.set_ylim(-45, 18)
ax.set_title("What Einstein gravity requires, and what each source supplies")
ax.legend(fontsize=7.5, loc="lower center", ncol=2)
save_figure(fig, "null_requirement",
            "The null combination $\\kappa(\\rho + p_0)$ (energy density plus the "
            "pressure along $x_0$; vertical, units $H^2$) that Einstein gravity "
            "requires of every source of the author's metric, $-6(a_4'^2 + H^2)$ "
            "(black curve), against the rate $a_4'/H$ (horizontal), and the "
            "constant values supplied by the universe ($-12$, blue), its T1 "
            "partner ($+12$, red), the T1 pair (0, purple dashed) and the T2 "
            "mirror copy ($-12$, green dotted, on top of the blue line). Only the "
            "universe and its mirror copy meet the curve, at $a_4' = \\pm H$ (blue "
            "dots; the sign is a choice, $+H$ is the deflating history); the "
            "partner and the pair lie above the curve's maximum $-6$ and meet it "
            "nowhere.")
```

Out [11]:

```
Figure 20b.2 saved as Revision/textbook/figures/20b_2_null_requirement.png
```

Text cell:

```
## 13. The bookkeeping, and what this example shows

The next cell collects, for the universe, the T1 partner, the T1 pair, the T2 copy
and the T2 pair (universe on the patch plus copy on the mirror patch, each
counted at its own point), the energy density $\rho$, the pressure $p$, the charge
density $J^{x_4}$ and the density $S$, and draws them as bars.
```

In [12]:

```
rho_u, p_u = -T_universe[4, 4], T_universe[1, 1]
rho_m, p_m = -T_mirror[4, 4], T_mirror[1, 1]
J_u = float(charge_value)
rows = {
    "universe": (rho_u, p_u, J_u, float(S_value)),
```

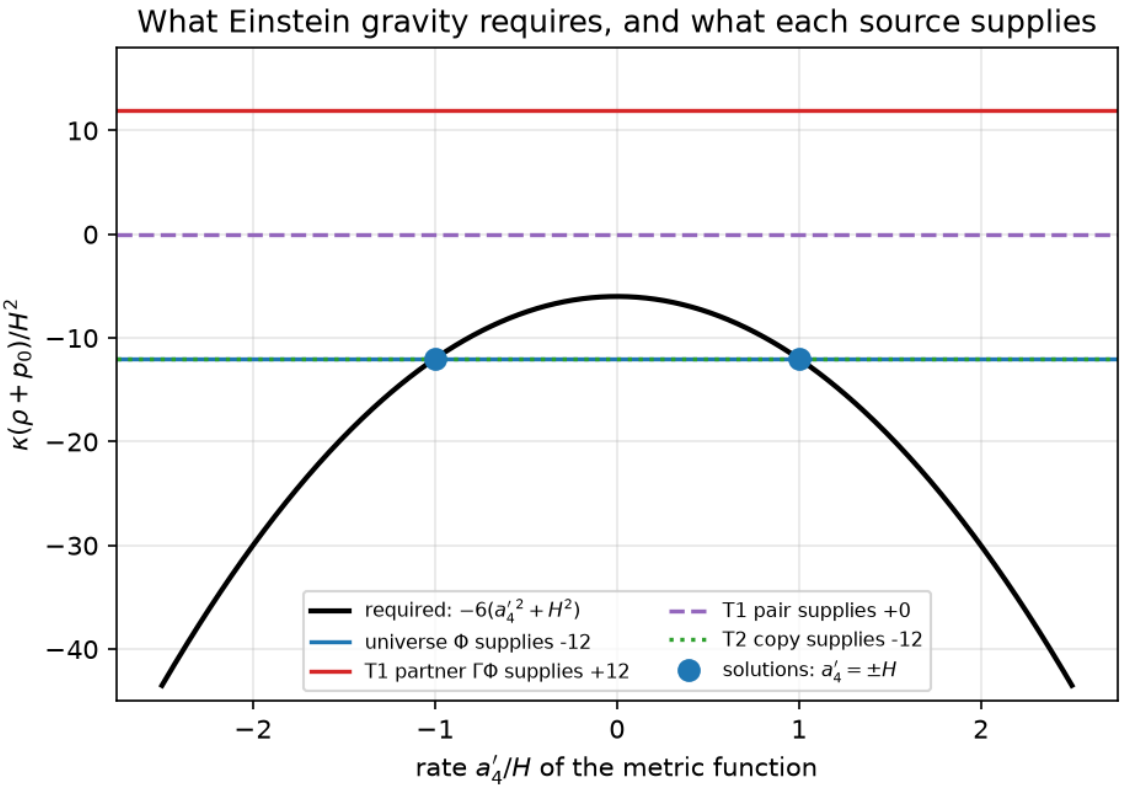

Figure 515: The null combination $\kappa(\rho + p_0)$ (energy density plus the pressure along x_0; vertical, units H^2) that Einstein gravity requires of every source of the author's metric, $-6(a_4'^2 + H^2)$ (black curve), against the rate a_4'/H (horizontal), and the constant values supplied by the universe (-12, blue), its T1 partner ($+12$, red), the T1 pair (0, purple dashed) and the T2 mirror copy (-12, green dotted, on top of the blue line). Only the universe and its mirror copy meet the curve, at $a_4' = \pm H$ (blue dots; the sign is a choice, $+H$ is the deflating history); the partner and the pair lie above the curve's maximum -6 and meet it nowhere. (Notebook 20b, figure 2.)

```
    "T1 partner": (-T_partner[4, 4], T_partner[1, 1], float(partner_charge),
                   float(S_value)),
    "T1 pair": (rho_u - T_partner[4, 4], p_u + T_partner[1, 1],
                J_u + float(partner_charge), 2 * float(S_value)),
    "T2 copy": (rho_m, p_m, float(copy_charge), -float(S_value)),
    "T2 pair": (rho_u + rho_m, p_u + p_m, J_u + float(copy_charge), 0.0),
}
for name, (r, p, j, s) in rows.items():
    say(f"{name:11s}: rho = {r:+7.2f}, p = {p:+7.2f}, J^x4 = {j:+6.2f}, "
        f"S = {s:+5.2f}")
check(abs(rows["T1 pair"][0]) < 1e-10 and abs(rows["T1 pair"][2]) < 1e-12
      and abs(rows["T2 pair"][0] - 24) < 1e-9 and abs(rows["T2 pair"][2] + 6) < 1e-12,
      "T1 pair: rho = 0 and J = 0; T2 pair: rho = 24 and J = -6")
quantities = ["$\\rho$", "$p$", "$J^{x_4}$", "$S$"]
fig, ax = plt.subplots(figsize=(9.0, 5.0))
width = 0.16
colours = ["tab:blue", "tab:red", "tab:purple", "tab:green", "tab:olive"]
for k, (name, values) in enumerate(rows.items()):
    positions = np.arange(4) + (k - 2) * width
    ax.bar(positions, values, width=width, color=colours[k], label=name)
    for x_value, value in zip(positions, values):
        if abs(value) < 1e-9:  # a bar of height zero: write 0 so it is seen
            ax.text(x_value, 1.0, "0", ha="center", fontsize=8,
                    color=colours[k], fontweight="bold")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(range(4))
ax.set_xticklabels(quantities)
ax.set_ylabel("value (units $H = \\kappa = 1$)")
ax.set_title("The bookkeeping of one universe and its partners")
ax.legend(fontsize=8, ncol=5, loc="upper center", bbox_to_anchor=(0.5, -0.08))
ax.set_ylim(-52, 30)
save_figure(fig, "bookkeeping",
            "Energy density $\\rho$, pressure $p$, charge density $J^{x_4}$ and "
            "density $S$ (groups of bars; units $H = \\kappa = 1$) of the universe "
            "$\\Phi$ (blue), its T1 partner $\\Gamma\\Phi$ (red), the T1 pair "
            "(purple), the T2 mirror copy $\\gamma^{(x_0)}\\Phi$ (green) and the "
            "T2 pair (olive). The T1 partner has $\\rho$, $p$ and $J^{x_4}$ "
            "reversed and the same $S$, so the T1 pair has zero energy, pressure "
            "and charge; the T2 copy has the same $\\rho$, $p$ and $J^{x_4}$ and "
            "the reversed $S$, so the T2 pair has twice the energy and the "
            "charge of the universe.")
```

Out [12]:

```
universe   : rho =  +12.00, p =  -24.00, J^x4 =  -3.00, S = +2.40
T1 partner : rho =  -12.00, p =  +24.00, J^x4 =  +3.00, S = +2.40
T1 pair    : rho =   +0.00, p =   +0.00, J^x4 =  +0.00, S = +4.80
T2 copy    : rho =  +12.00, p =  -24.00, J^x4 =  -3.00, S = -2.40
T2 pair    : rho =  +24.00, p =  -48.00, J^x4 =  -6.00, S = +0.00
PASS T1 pair: rho = 0 and J = 0; T2 pair: rho = 24 and J = -6
```

Out [12]:

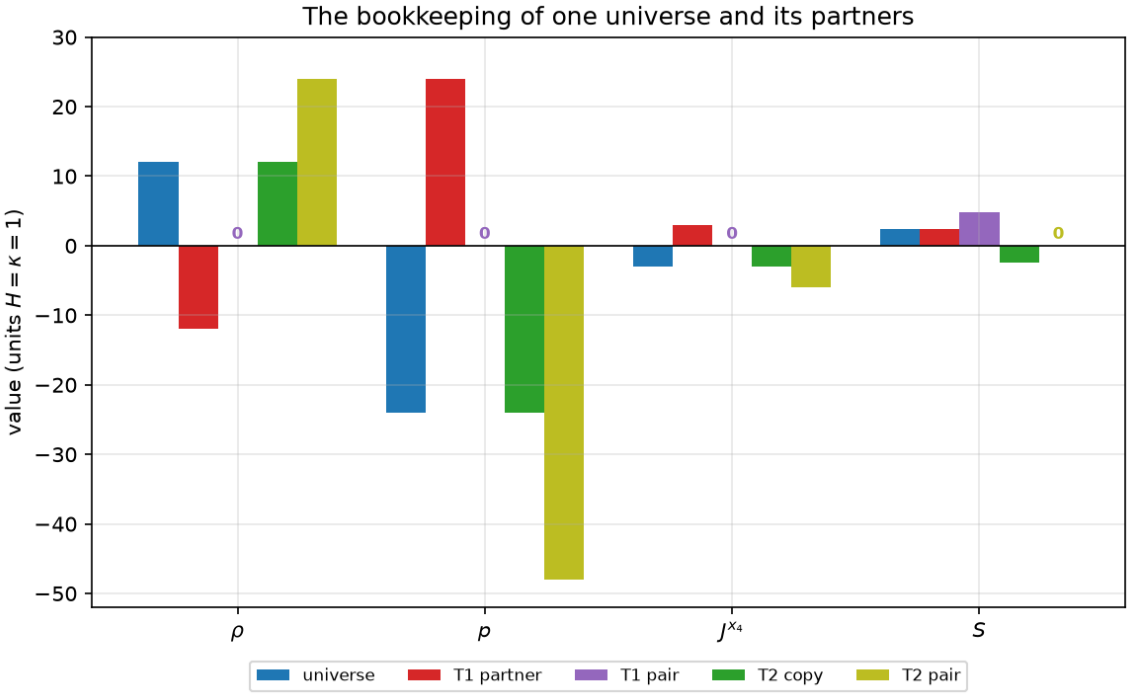

Figure 516: Energy density ρ, pressure p, charge density J^{x_4} and density S (groups of bars; units $H = \kappa = 1$) of the universe Φ (blue), its T1 partner $\Gamma\Phi$ (red), the T1 pair (purple), the T2 mirror copy $\gamma^{(x_0)}\Phi$ (green) and the T2 pair (olive). The T1 partner has ρ, p and J^{x_4} reversed and the same S, so the T1 pair has zero energy, pressure and charge; the T2 copy has the same ρ, p and J^{x_4} and the reversed S, so the T2 pair has twice the energy and the charge of the universe. (Notebook 20b, figure 3.)

```
Figure 20b.3 saved as Revision/textbook/figures/20b_3_bookkeeping.png
```

Text cell:

```
The last picture shows the universe and its T1 partner in time. On the left, the
real parts of two components of $\Phi(x_4)$ and of $\Gamma\Phi(x_4)$: component 1
lies in the half where $\Gamma = -1$ (it changes sign), component 9 in the half
where $\Gamma = +1$ (it does not); both oscillate with the same period
$2\pi/w$. On the right, the 16 eigenvalues of $\mathcal{A}$ (universe) and
$\mathcal{A}'$ (partner) in the complex plane: both are $\pm 4i$, each eight
times. Same frequency, opposite energy density: the opposite sign of the energy
is not an opposite frequency.
```

In [13]:

```
times = np.linspace(0.0, 3.0, 601)
phi_t = np.outer(np.exp(-1j * W * times), phi0_num)  # rows: times, columns: A
partner_t = phi_t @ Gamma_num.T  # Gamma Phi at every time
fig, (left, right) = plt.subplots(1, 2, figsize=(12.0, 4.4))
for index, colour in ((0, "tab:blue"), (8, "tab:orange")):
    left.plot(times, phi_t[:, index].real, color=colour, linewidth=4.0,
              alpha=0.35, label=f"universe, component {index + 1}")
    left.plot(times, partner_t[:, index].real, "--", color=colour,
              label=f"partner, component {index + 1}")  # drawn on top
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("real part")
left.set_title("same oscillation, the first half of the components reversed",
               fontsize=9)
left.legend(fontsize=7.5, loc="lower left", ncol=2)
left.set_ylim(-2.2, 1.7)
eig_u = np.linalg.eigvals(np.array(A_matrix.tolist(), dtype=float))
eig_p = np.linalg.eigvals(np.array(A_partner.tolist(), dtype=float))
right.plot(eig_u.real, eig_u.imag, "o", markersize=11, markerfacecolor="none",
           color="tab:blue", label="universe: eigenvalues of $\\mathcal{A}$")
right.plot(eig_p.real, eig_p.imag, "x", markersize=9, color="tab:red",
           label="partner: eigenvalues of $\\mathcal{A}'$")
right.set_xlim(-1.5, 1.5)
right.set_ylim(-5.5, 5.5)
right.set_xlabel("real part")
right.set_ylabel("imaginary part (units $H$)")
right.set_title("equal spectra: $\\pm 4i$, eight times each", fontsize=9)
right.legend(fontsize=8, loc="center right")
check(np.allclose(np.sort(eig_u.imag), np.sort(eig_p.imag), atol=1e-12)
      and np.abs(eig_u.real).max() < 1e-12,
      "both spectra are +-4i, eight times each (numerically)")
save_figure(fig, "partner_in_time",
            "Left: the real parts of components 1 and 9 of the universe "
            "$\\Phi(x_4)$ (thick pale lines) and of its T1 partner "
            "$\\Gamma\\Phi(x_4)$ (dashed) against the time $x_4$ (units $1/H$). "
            "Component 1 belongs to "
            "the half where $\\Gamma = -1$ and is reversed, component 9 to the half "
            "where $\\Gamma = +1$ and is unchanged; both oscillate with the period "
```

```
"$2\\pi/4$. Right: the 16 eigenvalues of the matrix $\\mathcal{A}$ of "
"the universe (circles) and of $\\mathcal{A}' = \\Gamma\\mathcal{A}"
"\\Gamma$ of the partner (crosses) in the complex plane: both are "
"$\\pm 4i$, each eight times. The partner has the same frequency and "
"the opposite energy density.")
```

Out [13]:

```
PASS both spectra are +-4i, eight times each (numerically)
```

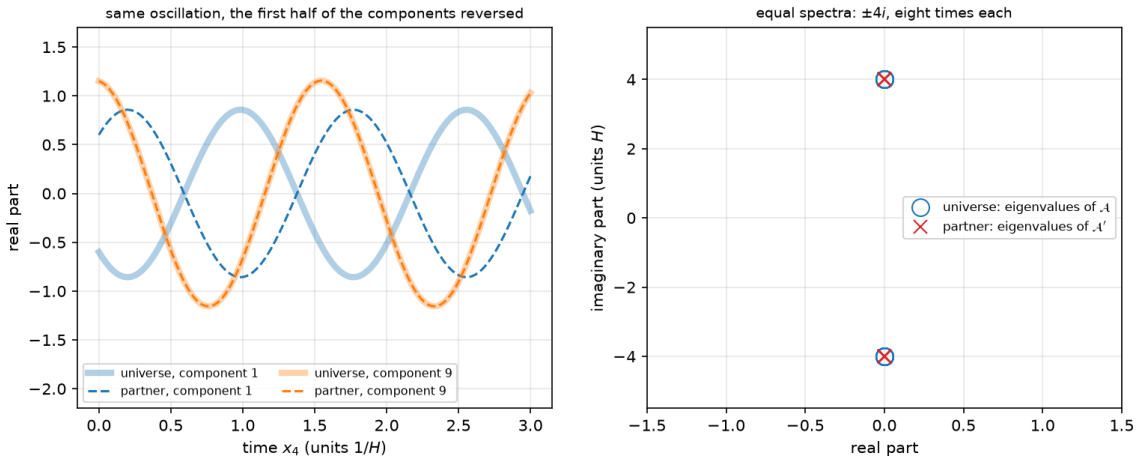

Figure 517: Left: the real parts of components 1 and 9 of the universe $\Phi(x_4)$ (thick pale lines) and of its T1 partner $\Gamma\Phi(x_4)$ (dashed) against the time x_4 (units $1/H$). Component 1 belongs to the half where $\Gamma = -1$ and is reversed, component 9 to the half where $\Gamma = +1$ and is unchanged; both oscillate with the period $2\pi/4$. Right: the 16 eigenvalues of the matrix $\mathcal{A}$ of the universe (circles) and of $\mathcal{A}' = \Gamma\mathcal{A}\Gamma$ of the partner (crosses) in the complex plane: both are $\pm 4i$, each eight times. The partner has the same frequency and the opposite energy density. (Notebook 20b, figure 4.)

Out [13]:

```
Figure 20b.4 saved as Revision/textbook/figures/20b_4_partner_in_time.png
```

Text cell:

```
**What this example shows (COMPUTED here, exact for the field equations, to
$10^{-10}$ for the Einstein equations):**

1. A universe needs no partner. The condensate $\Phi$ with $m = 15$,
   $\lambda = -25/3$ is by itself a complete solution of the coupled equations
   with the deflating history $a_4 = Hx_4$. The record's sentence "a single
   universe with mass $+m$ is an equally valid solution without its partner" is
   here a solution one can write down.
2. The T1 partner exists as a solution of ITS field equation (mass $-15$,
   coupling $25/3$), with the same frequency and opposite energy and charge. It is
```

```
   not a source of the author's metric in Einstein gravity, for any rate and any
   $\Lambda$, because its $\rho + p_0$ is positive.
3. The T1 pair has zero energy and zero charge, and therefore (C1) it cannot be the
   source of the author's metric in Einstein gravity.
4. The T2 mirror copy (mass $-15$, coupling $-25/3$) is a solution on the mirror
   patch with the SAME energy density and charge density. The Z2-doubled geometry
   with the universe on one side and its copy on the other solves the bulk
   equations on both patches; the brane itself is ASSUMED and its junction is not
   derived (OPEN).

**What it does not show:** that the partner or the copy is created, or that it
must exist; any process, rate or amplitude; anything about the quantised field
dirac16complex (the condensate is a classical solution of dirac16complex00); the
stability of the condensate.
```

Text cell:

```
## 14. The last check

The last cell checks that the four figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [14]:

```
figure_names = ["einstein_residuals", "null_requirement", "bookkeeping",
                "partner_in_time"]
paths = [output_file(f"{FIGURE_FOLDER}/20b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [14]:

```
PASS every figure file of this notebook exists
ALL 25 CHECKS PASSED (notebook 20b)
```

Text cell:

```
## 15. What this notebook showed

- With $A = 1$, $\Lambda = -36H^2$ and $M = -5H$ the record's two Einstein
  conditions give $S = 12/5$, $m = 15$, $\lambda = -25/3$, $\rho = 12$,
  $p = -24$; the record's recipe gives an exact condensate with these numbers
  and the charge density $J^{x_4} = -3$, which solves its field equation exactly
  and, alone, all 64 Einstein equations (figure 1).
- Its T1 partner $\Gamma\Phi$ solves the $(-15, 25/3)$ field equation with the
  same frequency $w = 4H$ (figure 4) and has $\rho' = -12$, $p' = 24$,
  $J' = +3$; with $\kappa(\rho' + p_0') = +12 > -6$ it is the source of no member
```

```
  of the author's family in Einstein gravity (figure 2).
- The T1 pair has zero energy, pressure and charge, and its Einstein residual is
  $G + \Lambda \neq 0$: corollary C1 on an example (figures 1 and 3).
- The T2 mirror copy $\gamma^{(x_0)}\Phi$ with $(-15, -25/3)$ solves the coupled
  equations on the mirror patch with the same $\rho = 12$ and $J^{x_4} = -3$; the
  T2 pair has $\rho = 24$ and $J^{x_4} = -6$ (figure 3).
- The equations pair solutions; they neither require nor create the partner.
```

20.16 Line-by-line walk-through of Notebook 20b

The notebook has 14 code cells, In [1] to In [14]. As in Section 20.11, each group of lines is quoted and then explained; lines that repeat a pattern of Notebook 20a are quoted as well, with a short reminder.

In [1], the set-up cell. It is word for word the set-up cell of Notebook 20a, explained line by line in Section 20.11, with two differences: its comment lines at the top hold the run instructions of Notebook 20b (Section 20.14), and one line names the notebook:

```
NOTEBOOK_ID = "20b"  # this notebook: chapter 20, example b
```

So the figures are saved as `20b_1_...` to `20b_4_...`, the captions in `20b.captions.json`, and the last line will read ALL 25 CHECKS PASSED (notebook 20b). Output: Set-up of notebook 20b complete.

In [2], the records and the helper `reproduces`.

```
import contextlib  # redirect_stdout: send printed lines into a buffer
import io  # StringIO: a text buffer in memory

GAMMAS = "Revision/algebra/gammas.json"  # the author's gamma matrices
A4_RECORD = "Revision/field_equations_a4/a4-equations.json"  # the a4 equations
A4_PY = "Revision/field_equations_a4/reports/python-a4-report.json"
A4_WL = "Revision/field_equations_a4/reports/wolfram-a4-report.json"
PAIR_PY = "Revision/pairing/reports/python-pairing.json"
PAIR_WL = "Revision/pairing/reports/wolfram-pairing.json"
PROOFS = "Revision/docs/PAIR_CREATION_PROOFS.md"
```

The two modules `contextlib` and `io` and the names of the seven files of the repository that the notebook reads: the author's gammas, the record of the field equations for a_4 with its two reports, the two pairing reports and the Revision document of the pairing proofs.

```
def read_json(relative):
    """Read a JSON file of the repository (a Revision record)."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


def record_verdict(report_file, name):
    """The verdict ("PASS") of the check called name in a report; None if absent."""
    for entry in read_json(report_file)["checks"]:
```

```
        if entry["name"] == name:
            return entry["verdict"]
    return None


def reproduces(condition, name, report_file, record_name):
    """A check that also requires the record check record_name to be PASS."""
    found = record_verdict(report_file, record_name) == "PASS"
    lines = io.StringIO()  # a text buffer
    with contextlib.redirect_stdout(lines):  # print() now writes into the buffer
        check(condition and found, name,
              record=f"{report_file}, check {record_name}")
    print(lines.getvalue(), end="")  # both lines with one print call
```

The three helpers of Notebook 20a, In [2], line for line: `read_json` reads a JSON record; `record_verdict` returns the verdict of a named check of a report (or `None`); `reproduces` passes only when the notebook's own condition holds AND the named record check has the verdict PASS, and then prints the PASS line together with the line that names the record (Section 20.11 explains every line).

```
NO_NECESSITY = ("a single universe with mass $+m$ is an equally valid solution "
                "without its partner")
proofs = repository_file(PROOFS).read_text(encoding="utf-8")
say(f"{PROOFS} says: \"... a single universe with mass +m is an equally valid "
    "solution without its partner.\"")
check(NO_NECESSITY in proofs, "the record states that no partner is required")
```

`NO_NECESSITY` is a sentence of the record's answer to the request (two strings in parentheses are joined into one). The document is read as one text; `say` prints the sentence; the check confirms that it occurs in the document word for word. Output: the quoted sentence and PASS the record states that no partner is required.

In [3], the gammas, C, Γ, B and the spin connection on both patches.

```
import itertools  # loops over all combinations of indices

import numpy as np  # floating-point arrays
import sympy as sp  # exact algebra

NAMES = ["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"]  # positions 0..7
ETA = [1, 1, 1, 1, -1, -1, -1, -1]  # the frame metric eta, x0..x7
I16, Z16 = sp.eye(16), sp.zeros(16, 16)
gamma = [sp.Matrix([[sp.Rational(x) for x in row] for row in matrix])
         for matrix in read_json(GAMMAS)["gamma"]]  # gamma[a] = gamma^(x_a)
C = gamma[0] * gamma[1] * gamma[2] * gamma[3]  # C = g^(x0) g^(x1) g^(x2) g^(x3)
chirality = gamma[0]  # Gamma = g^(x0) g^(x1) ... g^(x7), factor by factor
for a in range(1, 8):
    chirality = chirality * gamma[a]
B = -sp.I * C * gamma[4]  # the Krein matrix B = -i C gamma^(x4)
S_exact = [[(gamma[a] * gamma[b] - gamma[b] * gamma[a]) / 4 for b in range(8)]
           for a in range(8)]
```

The same objects as in Notebook 20a, In [3], built the same way: the labels, the diagonal of η, the exact unit matrix I_{16} and the exact 16×16 zero matrix `Z16` (`sp.zeros(16, 16)`; two names are assigned at once), the eight exact gammas read from `gammas.json` (here the reading and the conversion are written in one comprehension), C, the chirality Γ (called `chirality`) multiplied factor by factor in the record's order, the Krein matrix $B = -iC\gamma^{(x_4)}$ (`sp.I` is the exact imaginary unit; position 4 is x_4), and the 64 generators S^{ab}.

```
H = sp.symbols("H", positive=True)
z = sp.symbols("z", real=True)
x4 = sp.symbols("x4", real=True)
a4 = sp.Function("a4")(x4)
a4_value, slope = sp.symbols("a4_value slope", real=True)


def d(expr, mu):
    """The partial derivative along the coordinate at position mu."""
    if mu == 4:
        return sp.diff(expr, x4)
    if mu == 0:
        return 6 * H * sp.diff(expr, z)  # d/dx0 = 6 H d/dz
    return sp.Integer(0)
```

The symbols $H > 0$, z, x_4, the unknown function $a_4(x_4)$, the two symbols that replace a_4 and a_4' at a point, and the partial derivative `d(expr, mu)` with $\partial/\partial x_0 = 6H\,\partial/\partial z$ (Notebook 20a, In [4]).

```
def spin_connection(s0):
    """The lengths E_a and omega[mu][a][b] = omega_mu ab (s0 = +1 patch, -1 mirror)."""
    s = sp.sin(z) ** sp.Rational(1, 6)
    E = [s0 * sp.cot(z)] + [sp.exp(a4) * s] * 3 + [sp.Integer(1)]
    E = E + [sp.exp(-a4) * s] * 3  # x0, x1, ..., x7
    g = [ETA[a] * E[a] ** 2 for a in range(8)]
    christoffel = {}
    for a, b, c in itertools.product(range(8), repeat=3):
        value = 0
        if a == c:
            value += d(g[a], b)
        if a == b:
            value += d(g[a], c)
        if b == c:
            value -= d(g[b], a)
        christoffel[a, b, c] = value / (2 * g[a])
    omega = [[[sp.Integer(0)] * 8 for _ in range(8)] for _ in range(8)]
    for mu, a, b in itertools.product(range(8), repeat=3):
        if a != b:
            value = ETA[a] * E[a] * christoffel[a, mu, b] / E[b]
            value = value.subs(sp.Derivative(a4, x4), slope).subs(a4, a4_value)
            omega[mu][a][b] = sp.simplify(value)
    return [length.subs(a4, a4_value) for length in E], omega
```

The canonical spin connection on the patch (`s0 = 1`) or on the mirror patch (`s0 = -1`), exactly as in Notebook 20a, In [4]: the lengths E_a ($s_0 \cot z$, three inflating $e^{a_4} \sin^{1/6} z$, the time 1, three

DEFLATING $e^{-a_4}\sin^{1/6} z$), the diagonal metric $g_{aa} = \eta_{aa}E_a^2$, the Christoffel symbols term by term, and $\omega_{\mu ab} = \eta_{aa}(E_a/E_b)\Gamma^a{}_{\mu b}$ for $a \neq b$ with a_4 and a_4' replaced by the point symbols. The only difference from 20a is that the list of lengths is built inside the function in two lines (the first five lengths, then `E + [sp.exp(-a4) * s] * 3` appends the three of the extra times).

```
CONNECTION = {s0: spin_connection(s0) for s0 in (1, -1)}
gravity_term = {}  # s0 -> sum over mu of gamma^mu Omega_mu
for s0, (E, omega) in CONNECTION.items():
    Omega = [sum((omega[mu][a][b] * S_exact[a][b] / 2 for a in range(8)
                  for b in range(8) if omega[mu][a][b] != 0), Z16)
             for mu in range(8)]
    gravity_term[s0] = sum((gamma[mu] / E[mu] * Omega[mu] for mu in range(8)),
                           Z16).applyfunc(sp.simplify)
reproduces(gravity_term[1] == 3 * H * gamma[0],
           "patch: gamma^mu Omega_mu = 3 H gamma^(x0)", A4_PY, "authorT16_gravity_term")
check(gravity_term[-1] == -3 * H * gamma[0],
      "mirror patch: gamma^mu Omega_mu = -3 H gamma^(x0)")
```

`{s0: spin_connection(s0) for s0 in (1, -1)}` computes the connection on both patches and stores the pair (lengths, components) under the key $+1$ or -1; `.items()` walks through the keys and values of a dictionary. For each patch, `Omega` holds the eight matrices $\Omega_\mu = \frac{1}{2}\sum_{a,b}\omega_{\mu ab}S^{ab}$, and `gravity_term[s0]` is $\sum_\mu \gamma^\mu\Omega_\mu$ with $\gamma^\mu = \gamma^{(\mu)}/E_\mu$, simplified entry by entry. The checks: $3H\gamma^{(x_0)}$ on the patch (the record's term, `python-a4-report.json`, check `authorT16_gravity_term`) and $-3H\gamma^{(x_0)}$ on the mirror patch (a computation of this book). Output: two PASS lines.

In [4], choosing the universe: the two Einstein conditions.

```
record = read_json(A4_RECORD)
ad1, ad2 = sp.symbols("ad1 ad2", real=True)  # a4' and a4''
A, Lam, kappa = sp.symbols("A Lambda kappa", real=True)
m, lam, S_sym = sp.symbols("m lambda S", real=True)
```

The record of the field equations for a_4 is read. The symbols: a_4' and a_4'' under the record's names `ad1` and `ad2` (here declared real), the slope A, Λ (`Lam`), κ, the mass m, the coupling λ (`lam`; the word `lambda` is reserved in Python) and the density S (`S_sym`, a symbol, to keep it apart from numbers).

```
def einstein(key):
    """The component key of the Einstein tensor E_(1) from the record."""
    text = record["lovelockTensors"]["E1"][key]["input"].replace("^", "**")
    return sp.sympify(text, locals={"ad1": ad1, "ad2": ad2, "H": H})
```

`einstein(key)` reads one component of the Einstein tensor $E_{(1)} = G$ from the record's text in Wolfram's input form and turns it into a sympy expression (`^` becomes `**`; `locals` binds the names `ad1`, `ad2` and `H` to the notebook's symbols), as `lovelock` did in Notebook 20a, In [12].

```
G = {key: einstein(key) for key in ("x1x1", "x4x4", "x5x5", "x0x0")}
LINEAR = {ad1: A * H, ad2: 0}  # the linear member a4 = A H x4
```

```
rho_c, p_c = m * S_sym + lam * S_sym ** 2 / 2, lam * S_sym ** 2 / 2  # condensate
time_eq = G["x4x4"].subs(LINEAR) + Lam + kappa * rho_c  # = 0
hidden_eq = G["x0x0"].subs(LINEAR) + Lam - kappa * p_c  # = 0
first = sp.expand(time_eq - hidden_eq)  # kappa (m + lambda S) S + 6 (A^2 + 1) H^2
second = sp.expand(time_eq + hidden_eq)  # kappa m S + 36 H^2 + 2 Lambda
```

`G` holds the four independent diagonal components. `LINEAR` is the substitution of the linear member, $a_4' = AH$ and $a_4'' = 0$. `rho_c` and `p_c` are the condensate's $\rho = mS + \frac{\lambda}{2}S^2$ and $p = \frac{\lambda}{2}S^2$ (step 2 of Section 20.12). `time_eq` is the time equation written as "left side minus right side", ${G^{x_4}}_{x_4} + \Lambda + \kappa\rho = 0$, and `hidden_eq` the hidden equation, ${G^{x_0}}_{x_0} + \Lambda - \kappa p = 0$. `first` is their difference and `second` their sum, multiplied out by `sp.expand`: exactly the two combinations of step 3.

```
reproduces(sp.expand(first - (kappa * (m + lam * S_sym) * S_sym
                              + 6 * (A ** 2 + 1) * H ** 2)) == 0
           and sp.expand(second - (kappa * m * S_sym + 36 * H ** 2 + 2 * Lam)) == 0,
           "Einstein: kappa M S = -6 (A^2 + 1) H^2, kappa m S = -(36 H^2 + 2 Lambda)",
           A4_PY, "condensate_einstein_quadratic_U")
```

The check compares the two combinations with $\kappa(m + \lambda S)S + 6(A^2 + 1)H^2$ and $\kappa m S + 36H^2 + 2\Lambda$; each difference must expand to zero. It reproduces the record's two conditions (`python-a4-report .json`, check `condensate_einstein_quadratic_U`).

```
CHOICE = {H: 1, kappa: 1, A: 1, Lam: -36}  # the geometry of this notebook
M_VALUE = -5  # the effective mass M = m + lambda S
S_value = sp.solve(first.subs(CHOICE).subs(lam, (M_VALUE - m) / S_sym), S_sym)[0]
m_value = sp.solve(second.subs(CHOICE).subs(S_sym, S_value), m)[0]
lam_value = (M_VALUE - m_value) / S_value
rho_value = rho_c.subs({m: m_value, lam: lam_value, S_sym: S_value})
p_value = p_c.subs({lam: lam_value, S_sym: S_value})
```

`CHOICE` fixes the geometry of the notebook: $H = \kappa = 1$, $A = 1$, $\Lambda = -36$; `M_VALUE` is the effective mass $M = -5$. Then the four lines of step 4, done by sympy: in `first`, λ is replaced by $(M - m)/S$ (this is $M = m + \lambda S$ solved for λ), and `sp.solve(..., S_sym)` solves for S (`[0]` takes the one solution); in `second`, S is replaced by its value and the equation is solved for m; then λ, ρ and p follow by substitution.

```
for label, value in (("S", S_value), ("m", m_value), ("lambda", lam_value),
                     ("rho", rho_value), ("p", p_value)):
    report(label, value)
check((S_value, m_value, lam_value, rho_value, p_value)
      == (sp.Rational(12, 5), 15, sp.Rational(-25, 3), 12, -24),
      "the universe: S = 12/5, m = 15, lambda = -25/3, rho = 12, p = -24")
```

The loop prints the five numbers as RESULT lines, and the check compares them with the exact values of step 4. Output: RESULT S = 12/5, m = 15, lambda = -25/3, rho = 12, p = -24, and the PASS line.

In [5], building the universe.

```
w = sp.sqrt(M_VALUE ** 2 - 9)  # the frequency, H = 1
A_matrix = -gamma[4] * (M_VALUE * I16 - 3 * gamma[0])  # Phi' = A Phi
spaces = []
for sign in (-1, 1):  # the value of g^(x1)g^(x5) = g^(x2)g^(x6) = g^(x3)g^(x7)
    stack = sp.Matrix.vstack(A_matrix + sp.I * w * I16,
                             gamma[1] * gamma[5] - sign * I16,
                             gamma[2] * gamma[6] - sign * I16,
                             gamma[3] * gamma[7] - sign * I16)
    spaces.append(stack.nullspace())  # every solution of stack * v = 0
v1, v2 = spaces[0][0], spaces[1][0]
```

`w` is the frequency $\sqrt{M^2-9} = 4$ (exactly; `sp.sqrt` of 16). `A_matrix` is $\mathcal{A} = -\gamma^{(x_4)}(M - 3\gamma^{(x_0)})$ with $H = 1$. For each of the two values ∓ 1 (`sign`), `sp.Matrix.vstack` stacks four 16×16 matrices on top of each other into one 64×16 matrix: $\mathcal{A} + iwI_{16}$ and the three matrices $\gamma^{(x_1)}\gamma^{(x_5)} - \text{sign}\, I_{16}$, $\gamma^{(x_2)}\gamma^{(x_6)} - \text{sign}\, I_{16}$, $\gamma^{(x_3)}\gamma^{(x_7)} - \text{sign}\, I_{16}$. A column v with `stack * v = 0` satisfies all four conditions at once: $\mathcal{A}v = -iwv$ and each of the three products has the value `sign` on v. `stack.nullspace()` returns a list of columns that span all such v (the **null space**). `v1` is the first column for the value -1, `v2` the first for $+1$.

```
c = sp.conjugate(sp.expand((v1.H * C * v2)[0]))  # c = conj(v1^dagger C v2)
t = sp.symbols("t", real=True)
family = (v1 + t * c * v2).applyfunc(sp.expand)  # Phi_0(t)
```

`v1.H` is the **conjugate transpose** $v_1^\dagger$ (a row), so `(v1.H * C * v2)[0]` is the number $v_1^\dagger C v_2$, and `sp.conjugate` gives $c = \overline{v_1^\dagger C v_2}$. `t` is a real symbol and `family` the column $\Phi_0(t) = v_1 + t\, c\, v_2$, multiplied out entry by entry.

```
def bilinear(phi, matrix):
    """phibar matrix phi = phi^dagger C matrix phi (exact)."""
    return sp.expand((phi.H * C * matrix * phi)[0])


t_value = sp.solve(bilinear(family, I16) - S_value, t)[0]
phi0 = family.subs(t, t_value).applyfunc(sp.expand)  # the universe at x4 = 0
report("t", t_value)
```

`bilinear(phi, matrix)` is $\bar{\phi}\, M \phi = \phi^\dagger C M \phi$, exact. The equation $S(t) = 12/5$ is solved for t, the universe at $x_4 = 0$, `phi0`, is the family at that t, and the value is printed. Output: RESULT t = 15/512.

```
NEEDED = sorted({tuple(sorted((i, 4, 0))) for i in (1, 2, 3, 5, 6, 7)}
                | {tuple(sorted((i, j, 4))) for i in (1, 2, 3) for j in (5, 6, 7)})
zero_15 = all(bilinear(family, gamma[a] * gamma[b] * gamma[e]) == 0
              for a, b, e in NEEDED)  # for every real t, so also for t_value
reproduces([len(space) for space in spaces] == [1, 1] and zero_15
           and bilinear(phi0, I16) == S_value,
           "the universe: 15 three-gamma bilinears 0 and S = 12/5",
           A4_WL, "condensate_diagonal_witness_exact")
```

NEEDED lists the 15 index triples of the three-gamma bilinears that must vanish: the six triples $\{i, x_4, x_0\}$ with i one of $x_1, x_2, x_3, x_5, x_6, x_7$ (positions 1, 2, 3, 5, 6, 7 together with 4 and 0), and the nine triples $\{i, j, x_4\}$ with i in 3-space and j an extra time. Each triple is sorted, the braces `{...}` make a **set** (a collection without repetitions), `|` joins two sets, and `sorted` puts the triples in a fixed order. `zero_15` is true when all 15 bilinears of the family vanish for the SYMBOL t, so for every real t, in particular for 15/512. The check requires the two null spaces to be one-dimensional, the 15 zeros, and $S = 12/5$ for `phi0`; it reproduces the record's witness (`wolfram-a4-report.json`, check `condensate_diagonal_witness_exact`).

```
reproduces((C * A_matrix + A_matrix.T * C).applyfunc(sp.expand) == Z16,
           "C A + A^T C = 0: S stays 12/5 at every time", A4_PY,
           "authorT16_condensate_S_constant")
Phi = phi0 * sp.exp(-sp.I * w * x4)  # the universe Phi(x4)
```

The check $C\mathcal{A} + \mathcal{A}^T C = 0$ (`.T` is the transpose) shows that S stays 12/5 at every time (step 1). `Phi` is the universe $\Phi(x_4) = \Phi_0 e^{-iwx_4}$, a column of 16 exact functions of x_4.

```
def field_residual(field, term, mass, coupling, density):
    """gamma^(x4) field' + term field - (mass + coupling density) field."""
    return (gamma[4] * sp.diff(field, x4) + term * field
            - (mass + coupling * density) * field).applyfunc(sp.simplify)
```

`field_residual` returns $\gamma^{(x_4)}\Phi' + (\text{term})\Phi - (\text{mass} + \text{coupling} \cdot \text{density})\Phi$ for a field that depends on x_4 only: the left side minus the right side of the condensate equation, with the spin-connection term `term` given as an argument (so that the same function serves on the mirror patch). `sp.diff( field, x4)` differentiates each entry, and `.applyfunc(sp.simplify)` simplifies each entry of the result.

```
universe_ok = field_residual(Phi, gravity_term[1].subs(H, 1), m_value, lam_value,
                             S_value) == sp.zeros(16, 1)
reproduces(universe_ok, "the universe solves its field equation (16 components)",
           A4_WL, "condensate_equation_x0_consistent")
charge_value = sp.simplify((phi0.H * B * phi0)[0])  # J^x4, constant in time
report("charge density J^x4 of the universe", charge_value)
```

The universe's residual with the patch term $3\gamma^{(x_0)}$ (`gravity_term[1].subs(H, 1)`), $m = 15$, $\lambda = -25/3$ and $S = 12/5$ must be the zero column, at every time x_4 (the time is a symbol). The last lines compute the charge density $\Phi_0^\dagger B \Phi_0$ exactly; the factor e^{-iwx_4} drops out of it. Output: RESULT t = 15/512, three PASS lines with their records, and RESULT charge density J^x4 of the universe = -3.

In [6], the universe alone solves the Einstein equations.

```
point_symbols = (z, a4_value, slope, H)
S_num = [[np.array(S_exact[a][b].tolist(), dtype=float) for b in range(8)]
         for a in range(8)]
gamma_num = [np.array(g.tolist(), dtype=float) for g in gamma]
C_num = np.array(C.tolist(), dtype=float)
```

```
Gamma_num = np.array(chirality.tolist(), dtype=float)
B_num = np.array(B.tolist(), dtype=complex)
NUMERIC = {}
for s0, (E, omega) in CONNECTION.items():
    pieces = [(mu, a, b, sp.lambdify(point_symbols, omega[mu][a][b], "numpy"))
              for mu in range(8) for a in range(8) for b in range(8)
              if omega[mu][a][b] != 0]
    NUMERIC[s0] = (sp.lambdify(point_symbols, E, "numpy"), pieces)
```

The floating-point copies of the generators, the gammas, C, Γ and B (`dtype=complex` because B is imaginary), and, for both patches, the numpy functions of the lengths and of the nonzero connection components (`sp.lambdify`, as in Notebook 20a, In [6]).

```
def geometry_at(s0, z_value, a4_number, slope_number):
    """Metric, Omega_mu and curved gammas at one point (H = 1)."""
    lengths_f, pieces = NUMERIC[s0]
    values = (z_value, a4_number, slope_number, 1.0)
    E = np.array([float(x) for x in lengths_f(*values)])
    Om = [np.zeros((16, 16)) for _ in range(8)]
    for mu, a, b, f in pieces:
        Om[mu] += float(f(*values)) * S_num[a][b] / 2
    return {"g": np.array(ETA) * E ** 2, "Omega": Om,
            "up": [gamma_num[mu] / E[mu] for mu in range(8)],
            "down": [ETA[mu] * E[mu] * gamma_num[mu] for mu in range(8)]}
```

`geometry_at` is the function of Notebook 20a, In [6], with $H = 1$ fixed: it returns the metric, the eight Ω_μ and the curved gammas with upper and lower index at one point.

```
def mixed_tensor(at, psi, dpsi, mass, coupling):
    """T^mu_nu (convention sigma_T = +1) and K^mu_mu of a first jet."""
    bar = psi.conj() @ C_num
    dbar = [row.conj() @ C_num for row in dpsi]
    D = [dpsi[mu] + at["Omega"][mu] @ psi for mu in range(8)]
    Dbar = [dbar[mu] - bar @ at["Omega"][mu] for mu in range(8)]
    S = (bar @ psi).real
    kinetic = [0.5 * (bar @ at["up"][mu] @ D[mu] - Dbar[mu] @ at["up"][mu] @ psi)
               for mu in range(8)]  # K^mu_mu, no sum
    L = sum(kinetic).real - mass * S - coupling * S ** 2 / 2
```

`mixed_tensor` evaluates the energy-momentum tensor of a first jet, as `bilinears` did in Notebook 20a: $\bar\Phi = \Phi^\dagger C$, $\partial_\mu\bar\Phi$, the covariant derivatives, the real density S, and the eight diagonal kinetic terms $K^\mu{}_\mu = \frac{1}{2}(\bar\Phi\gamma^\mu D_\mu\Phi - (D_\mu\bar\Phi)\gamma^\mu\Phi)$, one for each μ with no sum, kept in the list `kinetic`. Their sum is the kinetic term K, and `L` is $\mathcal{L}/\sqrt{|g|} = K - mS - \frac{\lambda}{2}S^2$ (`.real` drops an imaginary part of rounding size).

```
    T = np.zeros((8, 8))
    for mu, nu in itertools.product(range(8), repeat=2):
        low = 0.25 * (bar @ at["down"][mu] @ D[nu] + bar @ at["down"][nu] @ D[mu]
                      - Dbar[mu] @ at["down"][nu] @ psi
```

```
                    - Dbar[nu] @ at["down"][mu] @ psi).real
        if mu == nu:
            low -= at["g"][mu] * L  # the pairing records' T_mu nu
        T[mu, nu] = -low / at["g"][mu]  # T^mu_nu = -g^(mu mu) T_mu nu
    return T, np.array([k.real for k in kinetic])
```

For each of the 64 pairs (μ, ν), `low` is the tensor $T_{\mu\nu}$ of the pairing records (four kinetic terms, and $-g_{\mu\mu}\mathcal{L}/\sqrt{|g|}$ on the diagonal). The notebook needs the mixed tensor $T^{\mu}{}_{\nu}$ in the convention of the record of the field equations, which is minus the pairing records' tensor (Section 20.5): raising the first index with the diagonal metric, $T^{\mu}{}_{\nu} = g^{\mu\mu}T_{\mu\nu}$ with $g^{\mu\mu} = 1/g_{\mu\mu}$, and changing the sign gives `T[mu, nu] = -low / at["g"][mu]`. The function returns $T^{\mu}{}_{\nu}$ and the eight diagonal kinetic terms.

```
G_numeric = np.diag([float(G[k].subs({ad1: 1, ad2: 0, H: 1})) for k in
                     ("x0x0", "x1x1", "x1x1", "x1x1", "x4x4", "x5x5", "x5x5",
                      "x5x5")])  # G^mu_nu at a4' = H, H = 1, in the order x0..x7
LAMBDA = -36.0
phi0_num = np.array([complex(entry) for entry in phi0])
W = float(w)
POINTS = [(zv, xv) for zv in (0.3, 0.8, 1.3) for xv in (0.0, 0.7, 2.0)]
```

`G_numeric` is the diagonal matrix of $G^{\mu}{}_{\nu}$ at $a_4' = H = 1$, $a_4'' = 0$, in the order $x_0, \ldots, x_7$ (components 1 to 3 equal $G^{x_1}{}_{x_1}$, 5 to 7 equal $G^{x_5}{}_{x_5}$): $\mathrm{diag}(12, 12, 12, 12, 24, 12, 12, 12)$. `LAMBDA` is $\Lambda = -36$, `phi0_num` the universe at $x_4 = 0$ as 16 complex floating-point numbers, `W` the frequency 4.0, and `POINTS` the nine points: $z = 0.3, 0.8, 1.3$ combined with $x_4 = 0, 0.7, 2$.

```
def universe_jet(x_value, matrix=None):
    """Value and derivatives of matrix Phi at the time x_value."""
    value = phi0_num * np.exp(-1j * W * x_value)
    if matrix is not None:
        value = matrix @ value
    jet = [np.zeros(16, dtype=complex) for _ in range(8)]
    jet[4] = -1j * W * value  # d Phi / d x4 = -i w Phi
    return value, jet
```

`universe_jet(x, matrix)` returns the first jet of the universe (or of `matrix` times the universe) at the time x: the value $\Phi_0 e^{-iwx}$ and eight derivative columns, all zero except the one along x_4, $-iw\Phi$. `matrix=None` makes the argument optional; `is not None` tests whether it was given.

```
worst_residual, worst_kinetic = 0.0, 0.0
for z_value, x_value in POINTS:
    at = geometry_at(1, z_value, x_value, 1.0)  # a4 = x4, a4' = 1
    T, kinetic = mixed_tensor(at, *universe_jet(x_value), float(m_value),
                              float(lam_value))
    residual = G_numeric + LAMBDA * np.eye(8) - T  # kappa = 1
    worst_residual = max(worst_residual, np.abs(residual).max())
    wanted = np.zeros(8)
    wanted[4] = M_VALUE * float(S_value)  # K^x4_x4 = M S, all others 0
    worst_kinetic = max(worst_kinetic, np.abs(kinetic - wanted).max())
```

At each of the nine points: the geometry on the patch with $a_4 = x_4$ (the history $a_4 = Hx_4$ with $H = 1$) and $a_4' = 1$; the tensor and the kinetic terms of the universe (`*universe_jet(x_value)` unpacks the pair (value, jet) into two arguments); the Einstein residual $G + \Lambda - \kappa T$ with $\kappa = 1$; and the largest deviation of the kinetic terms from the record's values ${K^{x_4}}_{x_4} = MS = -12$ and zero otherwise. `worst_residual` and `worst_kinetic` keep the largest values over all points.

```
T_universe = T  # the tensor at the last point (it is the same at every point)
report("rho of the universe", f"{-T_universe[4, 4]:.10f}")
report("p of the universe", f"{T_universe[1, 1]:.10f}")
reproduces(worst_kinetic < 1e-10, "K^x4_x4 = M S and K^mu_mu = 0 otherwise (9 points)",
           A4_PY, "authorT16_condensate_kinetic_diagonal")
check(worst_residual < 1e-10,
      "the universe ALONE solves all 64 Einstein equations (9 points)")
```

The tensor at the last point is kept as `T_universe` (it is the same at every point: the condensate has constant ρ and p). The RESULT lines print $\rho = -{T^{x_4}}_{x_4}$ and $p = {T^{x_1}}_{x_1}$ with ten decimals (`:.10f`). Output: RESULT rho of the universe = 12.0000000000, RESULT p of the universe = -24.0000000000, PASS for the kinetic terms (reproducing `python-a4-report.json`, check `authorT16_condensate_kinetic_diagonal`), and PASS the universe ALONE solves all 64 Einstein equations (9 points): every one of the 9×64 residuals is below 10^{-10}.

In [7], the T1 partner $\Gamma\Phi$.

```
partner = chirality * Phi  # Gamma Phi
partner_ok = field_residual(partner, gravity_term[1].subs(H, 1), -m_value,
                            -lam_value, S_value) == sp.zeros(16, 1)
reproduces(partner_ok and bilinear(chirality * phi0, I16) == S_value,
           "Gamma Phi solves the (-m, -lambda) field equation; S' = S",
           PAIR_PY, "T1.metric.commuting.euler_lagrange_map")
```

`partner` is $\Gamma\Phi$, exact. Its residual with the patch term, the parameters $(-m, -\lambda)$ and the density S must vanish at every time, and its density must equal S: step 6 of Section 20.12, reproducing `python-pairing.json`, check `T1.metric.commuting.euler_lagrange_map`.

```
A_partner = -gamma[4] * (-M_VALUE * I16 - 3 * gamma[0])  # M' = -M
same_spectrum = (chirality * A_matrix * chirality == A_partner
                 and (A_partner * A_partner + w ** 2 * I16) == Z16)
reproduces(same_spectrum, "A' = Gamma A Gamma: the partner has the same frequency",
           PAIR_WL, "Q_one_particle_maps")
```

`A_partner` is $\mathcal{A}' = -\gamma^{(x_4)}(-M - 3\gamma^{(x_0)})$. The check requires $\Gamma\mathcal{A}\Gamma = \mathcal{A}'$, the similarity derived by hand in step 6, and $\mathcal{A}'^2 + w^2 I_{16} = 0$, so that every eigenvalue of the partner's matrix is $\pm iw$: the same frequency.

```
partner_charge = sp.simplify(((chirality * phi0).H * B * (chirality * phi0))[0])
reproduces(partner_charge == -charge_value, "the partner's charge density is -J^x4",
           PAIR_PY, "T1.metric.commuting.current")
```

The partner's charge density $(\Gamma\Phi_0)^\dagger B(\Gamma\Phi_0)$, exact, must be $-(-3) = +3$ (`python-pairing.json`, check `T1.metric.commuting.current`).

```
worst_sum, worst_partner = 0.0, 0.0
for z_value, x_value in POINTS:
    at = geometry_at(1, z_value, x_value, 1.0)
    T_one, _ = mixed_tensor(at, *universe_jet(x_value), float(m_value),
                            float(lam_value))
    T_two, _ = mixed_tensor(at, *universe_jet(x_value, Gamma_num),
                            -float(m_value), -float(lam_value))
    worst_sum = max(worst_sum, np.abs(T_one + T_two).max())
    residual = G_numeric + LAMBDA * np.eye(8) - T_two
    worst_partner = max(worst_partner, np.abs(residual - 2 * T_one).max())
T_partner = T_two
reproduces(worst_sum < 1e-10, "T' = -T at all 9 points (64 components each)",
           PAIR_PY, "T1.metric.commuting.emt")
check(worst_partner < 1e-10 and np.abs(2 * T_one).max() > 1.0,
      "partner alone: the Einstein residual is 2 kappa T, not zero")
```

At the nine points: the tensor of the universe (`T_one`) and of the partner (`T_two`, the jet of $\Gamma\Phi$ with $(-m, -\lambda)$; `_` receives the kinetic terms, which are not needed). `worst_sum` is the largest component of $T + T'$, and `worst_partner` the largest deviation of the partner's Einstein residual from $2T$: with the partner as the only source the residual is $G + \Lambda - T' = (G + \Lambda - T) + 2T = 2T$, because the universe makes $G + \Lambda - T$ vanish and $T' = -T$. The checks: $T' = -T$ to 10^{-10} at all points (`python-pairing.json`, check `T1.metric.commuting.emt`), and the partner's residual is $2T$, whose largest entry is larger than 1, so NOT zero.

```
null_universe = -T_universe[4, 4] + T_universe[0, 0]  # kappa (rho + p0)
null_partner = -T_partner[4, 4] + T_partner[0, 0]
report("kappa (rho + p0) of the universe", f"{null_universe:.10f}")
report("kappa (rho + p0) of the partner", f"{null_partner:.10f}")
```

The null combination $\kappa(\rho + p_0) = -T^{x_4}{}_{x_4} + T^{x_0}{}_{x_0}$ of the universe and of the partner, printed with ten decimals. Output: -12.0000000000 and 12.0000000000.

```
rate = sp.symbols("rate", real=True)  # a real value of a4'/H
required = sp.expand(-(einstein("x4x4") - einstein("x0x0")).subs(
    {ad1: rate, ad2: 0, H: 1}))  # the requirement -6 (a4'^2 + 1)
universe_null = rho_value + p_value  # exact: 12 - 24 = -12
partner_null = -(rho_value + p_value)  # exact: T' = -T gives +12
check(abs(null_universe - float(universe_null)) < 1e-10
      and abs(null_partner - float(partner_null)) < 1e-10,
      "kappa (rho + p0): universe -12, partner +12 (numbers = exact values)")
reproduces(required == -6 * rate ** 2 - 6
           and sorted(sp.solve(required - universe_null, rate)) == [-1, 1]
           and sp.solve(required - partner_null, rate) == [],
           "the universe fits a4' = +-H; the partner fits no real rate",
           A4_PY, "einstein_null_energy_x0")
```

`rate` is a real symbol for a_4'/H, and `required` is the requirement $-(G^{x_4}{}_{x_4} - G^{x_0}{}_{x_0}) = -6(a_4'^2 + 1)$ with $H = 1$, computed from the record's components. `universe_null` and `partner_null` are the

exact values -12 and $+12$. The first check compares the floating-point values with the exact ones. The second requires the requirement to be $-6\,\mathrm{rate}^2 - 6$, the universe's value to be met exactly at the two real rates -1 and $+1$, and the partner's value at no real rate (`sp.solve` returns the empty list `[]`): step 7 of Section 20.12, reproducing `python-a4-report.json`, check `einstein_null_energy_x0`. Output of the cell: six PASS lines (five of them with their records) and the two RESULT lines.

In [8], the T1 pair as the only source.

```
worst_pair = 0.0
for z_value, x_value in POINTS:
    at = geometry_at(1, z_value, x_value, 1.0)
    T_one, _ = mixed_tensor(at, *universe_jet(x_value), float(m_value),
                            float(lam_value))
    T_two, _ = mixed_tensor(at, *universe_jet(x_value, Gamma_num),
                            -float(m_value), -float(lam_value))
    residual = G_numeric + LAMBDA * np.eye(8) - (T_one + T_two)
    expected = np.diag([-24.0, -24, -24, -24, -12, -24, -24, -24])
    worst_pair = max(worst_pair, np.abs(residual - expected).max())
check(worst_pair < 1e-10,
      "T1 pair alone: residual = G + Lambda = diag(-24, ..., -12, ...), not zero")
```

At the nine points the residual with the T1 pair as the only source, $G+\Lambda-(T+T')$, must equal the matrix $\mathrm{diag}(-24,-24,-24,-24,-12,-24,-24,-24)$ of step 8 (`np.diag` makes a diagonal matrix from a list). Output: PASS T1 pair alone: residual = G + Lambda = diag(-24, ..., -12, ...), not zero. This is corollary C1 on the example.

In [9], the T2 mirror copy across the brane.

```
copy = gamma[0] * Phi  # gamma^(x0) Phi on the mirror patch
copy_ok = field_residual(copy, gravity_term[-1].subs(H, 1), -m_value, lam_value,
                         -S_value) == sp.zeros(16, 1)
reproduces(copy_ok and bilinear(gamma[0] * phi0, I16) == -S_value,
           "the mirror copy solves the (-m, lambda) equation there; S' = -S",
           PAIR_PY, "T2.metric.commuting.euler_lagrange_map")
copy_charge = sp.simplify(((gamma[0] * phi0).H * B * (gamma[0] * phi0))[0])
reproduces(copy_charge == charge_value, "the copy's charge density is +J^x4",
           PAIR_PY, "Q.T2_image_keeps_B")
```

`copy` is $\gamma^{(x_0)}\Phi$ (position 0 is x_0). Its residual is computed with the MIRROR-patch term $-3\gamma^{(x_0)}$ (`gravity_term[-1]`), the parameters $(-m,\lambda)$ and the density $-S$; it must vanish at every time, and its density must be $-S$: step 9 of Section 20.12, reproducing `python-pairing.json`, check `T2.metric.commuting.euler_lagrange_map`. The copy's charge density, exact, must equal the universe's -3 (`python-pairing.json`, check `Q.T2_image_keeps_B`).

```
worst_copy, worst_pull = 0.0, 0.0
R0 = np.diag([-1.0, 1, 1, 1, 1, 1, 1, 1])  # the reflection of x0
for z_value, x_value in POINTS:
    there = geometry_at(-1, np.pi - z_value, x_value, 1.0)  # the mirror point
    T_copy, _ = mixed_tensor(there, *universe_jet(x_value, gamma_num[0]),
```

```
                            -float(m_value), float(lam_value))
    T_one, _ = mixed_tensor(geometry_at(1, z_value, x_value, 1.0),
                            *universe_jet(x_value), float(m_value), float(lam_value))
    worst_pull = max(worst_pull, np.abs(T_copy - R0 @ T_one @ R0).max())
    residual = G_numeric + LAMBDA * np.eye(8) - T_copy
    worst_copy = max(worst_copy, np.abs(residual).max())
T_mirror = T_copy
```

`R0` is the diagonal matrix $R_0 = \mathrm{diag}(-1, 1, 1, 1, 1, 1, 1, 1)$. At each of the nine points, `there` is the geometry at the mirror point $\pi - z$ of the mirror patch (`s0 = -1`), and `T_copy` the tensor of the copy there, with $(-m, \lambda)$. `T_one` is the universe's tensor at the original point. `worst_pull` is the largest deviation of the copy's tensor from the pulled-back tensor $R_0 T R_0$ (`@` is the matrix product), and `worst_copy` the largest Einstein residual of the copy alone on the mirror patch. The last value of `T_copy` is kept as `T_mirror`.

```
reproduces(worst_pull < 1e-10, "the copy's tensor is R0 T R0 = T at 9 mirror points",
           PAIR_PY, "T2.metric.commuting.emt")
check(worst_copy < 1e-10,
      "the mirror copy ALONE solves the 64 Einstein equations on the mirror patch")
```

The checks: the copy's tensor is $R_0 T R_0$, which equals T because T is diagonal (`python-pairing.json`, check `T2.metric.commuting.emt`), and the copy ALONE solves all 64 Einstein equations on the mirror patch. Output: four PASS lines, three with their records.

In [10], figure 1: the Einstein residuals of the four sources.

```
tables = [
    ("universe $\\Phi$ alone", G_numeric + LAMBDA * np.eye(8) - T_universe),
    ("T1 partner $\\Gamma\\Phi$ alone", G_numeric + LAMBDA * np.eye(8) - T_partner),
    ("T1 pair (C1)", G_numeric + LAMBDA * np.eye(8) - T_universe - T_partner),
    ("T2 copy alone (mirror patch)", G_numeric + LAMBDA * np.eye(8) - T_mirror),
]
```

`tables` is a list of four pairs (title, residual): the universe alone, the T1 partner alone, the T1 pair, and the T2 copy alone (on the mirror patch). Each residual is $G + \Lambda - T$ for the source named.

```
fig, axes = plt.subplots(1, 4, figsize=(14.0, 4.0))
for ax, (title, values) in zip(axes, tables):
    image = ax.imshow(values, cmap="RdBu_r", vmin=-48, vmax=48)
    for mu in range(8):
        if abs(values[mu, mu]) > 1e-6:
            ink = "white" if abs(values[mu, mu]) > 30 else "black"  # readable
            ax.text(mu, mu, f"{values[mu, mu]:.0f}", ha="center", va="center",
                    fontsize=6.5, color=ink)
    ax.set_xticks(range(8))
    ax.set_xticklabels(NAMES, fontsize=6.5)
    ax.set_yticks(range(8))
    ax.set_yticklabels(NAMES, fontsize=6.5)
    ax.grid(False)
    ax.set_title(title, fontsize=9)
fig.colorbar(image, ax=list(axes), shrink=0.75, label="residual (units $H^2$)")
```

One row of four panels, 14 by 4 inches. Each residual is drawn as a heat map with the colour scale from -48 to 48 (`vmin`, `vmax`), the same in all panels. The inner loop writes the value of every nonzero diagonal entry into its square (`f"{...:.0f}"` prints it without decimals; `ha` and `va` centre the text), in white on dark squares (modulus above 30) and in black otherwise, so that it can be read. The labels, the switched-off grid, the titles and one common colour bar follow, as in Notebook 20a, In [8].

```
save_figure(fig, "einstein_residuals",
            "The residual $G^\\mu{}_\\nu + \\Lambda\\delta^\\mu_\\nu - \\kappa "
...
            "equations there.")
```

`save_figure` saves `20b_1_einstein_residuals.png` and shows it (the eleven-line caption is shown here by its first and last lines). **What figure 1 shows.** Four 8×8 tables of the residual $G^\mu{}_\nu + \Lambda\delta^\mu_\nu - \kappa T^\mu{}_\nu$ (rows μ, columns ν, x_0 to x_7; colour scale in units H^2, white means zero), for the author's metric with $a_4 = Hx_4$ and $\Lambda = -36H^2$. The universe alone gives a white table: it solves all 64 equations. The T1 partner alone gives $2\kappa T$, with -48 on the diagonal except -24 at x_4. The T1 pair gives $G + \Lambda$, with -24 on the diagonal except -12 at x_4. The T2 copy alone on the mirror patch gives a white table again. The student should see that only the universe and its mirror copy solve the equations; the T1 partner and the T1 pair do not.

In [11], figure 2: what Einstein gravity requires and what each source supplies.

```
rates = np.linspace(-2.5, 2.5, 501)
fig, ax = plt.subplots(figsize=(7.0, 4.6))
ax.plot(rates, -6 * (rates ** 2 + 1), color="black", linewidth=2.0,
        label="required: $-6(a_4'^2 + H^2)$")
```

501 rates a_4'/H from -2.5 to 2.5, and the requirement $-6(a_4'^2 + H^2)$ drawn as a thick black curve.

```
supplied = (("universe $\\Phi$", null_universe, "tab:blue", "-"),
            ("T1 partner $\\Gamma\\Phi$", null_partner, "tab:red", "-"),
            ("T1 pair", null_universe + null_partner, "tab:purple", "--"),
            ("T2 copy", -T_mirror[4, 4] + T_mirror[0, 0], "tab:green", ":"))
for label, value, colour, style in supplied:
    ax.axhline(value, color=colour, linestyle=style, linewidth=1.6,
               label=f"{label} supplies {value:+.0f}")
ax.plot([-1.0, 1.0], [-12.0, -12.0], "o", color="tab:blue", markersize=8,
        label="solutions: $a_4' = \\pm H$")
```

`supplied` lists four (label, value, colour, line style) entries: the universe's -12, the partner's $+12$, the pair's 0 and the copy's $-T^{x_4}{}_{x_4} + T^{x_0}{}_{x_0} = -12$. Each is drawn as a horizontal line (`ax.axhline`); the label prints the value with its sign (`:+.0f`). Two blue dots mark the two places where the universe's line meets the curve, $a_4' = \pm H$.

```
ax.set_xlabel("rate $a_4'/H$ of the metric function")
ax.set_ylabel("$\\kappa(\\rho + p_0)/H^2$")
ax.set_ylim(-45, 18)
ax.set_title("What Einstein gravity requires, and what each source supplies")
ax.legend(fontsize=7.5, loc="lower center", ncol=2)
```

The axis labels, the vertical range from -45 to 18 (`set_ylim`), the title and a legend in two columns below the curve.

```
save_figure(fig, "null_requirement",
            "The null combination $\\kappa(\\rho + p_0)$ (energy density plus the "
            "pressure along $x_0$; vertical, units $H^2$) that Einstein gravity "
...
            "nowhere.")
```

What figure 2 shows. Horizontal axis: the rate a_4'/H; vertical axis: the null combination $\kappa(\rho+p_0)$ in units H^2. The black parabola is what Einstein gravity requires of every source; its highest point is -6, at $a_4'=0$. The blue line of the universe (-12) meets it at $a_4'=\pm H$; the green dotted line of the T2 copy lies on top of the blue one. The red line of the T1 partner ($+12$) and the purple dashed line of the T1 pair (0) lie above the parabola's highest point and meet it nowhere. The student should see in one picture why the partner and the pair cannot be the source of the author's metric for any rate, while the universe and its mirror copy can.

In [12], figure 3: the bookkeeping.

```
rho_u, p_u = -T_universe[4, 4], T_universe[1, 1]
rho_m, p_m = -T_mirror[4, 4], T_mirror[1, 1]
J_u = float(charge_value)
rows = {
    "universe": (rho_u, p_u, J_u, float(S_value)),
    "T1 partner": (-T_partner[4, 4], T_partner[1, 1], float(partner_charge),
                   float(S_value)),
    "T1 pair": (rho_u - T_partner[4, 4], p_u + T_partner[1, 1],
                J_u + float(partner_charge), 2 * float(S_value)),
    "T2 copy": (rho_m, p_m, float(copy_charge), -float(S_value)),
    "T2 pair": (rho_u + rho_m, p_u + p_m, J_u + float(copy_charge), 0.0),
}
```

The numbers of the table of Section 20.12: ρ and p of the universe and of the copy, the universe's charge density as a floating-point number, and the dictionary `rows`, which holds for each of the five objects the four numbers (ρ, p, J^{x_4}, S). The T1 pair adds the universe and the partner at the same point; the T2 pair adds the universe and the copy, each at its own point; the T2 pair's S is $S+(-S)=0$.

```
for name, (r, p, j, s) in rows.items():
    say(f"{name:11s}: rho = {r:+7.2f}, p = {p:+7.2f}, J^x4 = {j:+6.2f}, "
        f"S = {s:+5.2f}")
check(abs(rows["T1 pair"][0]) < 1e-10 and abs(rows["T1 pair"][2]) < 1e-12
      and abs(rows["T2 pair"][0] - 24) < 1e-9 and abs(rows["T2 pair"][2] + 6) < 1e-12,
      "T1 pair: rho = 0 and J = 0; T2 pair: rho = 24 and J = -6")
```

The loop prints one line per object; in the f-string, `{name:11s}` pads the name to 11 characters and `{r:+7.2f}` prints a number with its sign, two decimals, in a field of 7 characters, so that the columns line up. Output: the five lines of the table (universe $+12$, -24, -3, $+2.40$; T1 partner -12, $+24$, $+3$, $+2.40$; T1 pair 0, 0, 0, $+4.80$; T2 copy $+12$, -24, -3, -2.40; T2 pair $+24$, -48, -6, 0). The check: the T1 pair has $\rho=0$ and $J^{x_4}=0$, the T2 pair $\rho=24$ and $J^{x_4}=-6$.

```
quantities = ["$\\rho$", "$p$", "$J^{x_4}$", "$S$"]
fig, ax = plt.subplots(figsize=(9.0, 5.0))
width = 0.16
colours = ["tab:blue", "tab:red", "tab:purple", "tab:green", "tab:olive"]
for k, (name, values) in enumerate(rows.items()):
    positions = np.arange(4) + (k - 2) * width
    ax.bar(positions, values, width=width, color=colours[k], label=name)
    for x_value, value in zip(positions, values):
        if abs(value) < 1e-9:  # a bar of height zero: write 0 so it is seen
            ax.text(x_value, 1.0, "0", ha="center", fontsize=8,
                    color=colours[k], fontweight="bold")
```

A grouped bar chart: four groups (the quantities ρ, p, J^{x_4}, S), five bars in each (the objects). `np.arange(4)` is $0, 1, 2, 3$, the centres of the groups; the bar of object k is shifted by $(k-2)$ times the width 0.16, so the five bars stand side by side. `enumerate` numbers the objects $k = 0, \ldots, 4$. A bar of height zero cannot be seen, so the inner loop writes a bold 0 in its place.

```
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(range(4))
ax.set_xticklabels(quantities)
ax.set_ylabel("value (units $H = \\kappa = 1$)")
ax.set_title("The bookkeeping of one universe and its partners")
ax.legend(fontsize=8, ncol=5, loc="upper center", bbox_to_anchor=(0.5, -0.08))
ax.set_ylim(-52, 30)
save_figure(fig, "bookkeeping",
...
            "charge of the universe.")
```

A thin line at zero, the names of the quantities under the groups, the label, the title, a legend in five columns below the chart, and the vertical range. **What figure 3 shows.** Groups of bars for ρ, p, J^{x_4} and S (units $H = \kappa = 1$). The red bars of the T1 partner are the blue bars of the universe reversed in ρ, p and J^{x_4} and equal in S, so the T1 pair (purple) is zero in the first three groups. The green bars of the T2 copy equal the blue ones in ρ, p and J^{x_4} and are reversed in S, so the T2 pair (olive) has twice the energy, twice the pressure and twice the charge. The student should see the difference between the two theorems as two different patterns of bars: T1 cancels, T2 adds.

In [13], figure 4: the partner in time.

```
times = np.linspace(0.0, 3.0, 601)
phi_t = np.outer(np.exp(-1j * W * times), phi0_num)  # rows: times, columns: A
partner_t = phi_t @ Gamma_num.T  # Gamma Phi at every time
```

601 times from 0 to 3. `np.outer(a, b)` is the table of all products $a_i b_j$: here row i is the universe $\Phi_0 e^{-iwx_4}$ at the i-th time. `partner_t` is $\Gamma\Phi$ at every time: multiplying each row by Γ^T from the right is the same as multiplying each column vector by Γ from the left.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(12.0, 4.4))
for index, colour in ((0, "tab:blue"), (8, "tab:orange")):
    left.plot(times, phi_t[:, index].real, color=colour, linewidth=4.0,
```

```
            alpha=0.35, label=f"universe, component {index + 1}")
    left.plot(times, partner_t[:, index].real, "--", color=colour,
            label=f"partner, component {index + 1}")  # drawn on top
left.set_xlabel("time $x_4$ (units $1/H$)")
left.set_ylabel("real part")
left.set_title("same oscillation, the first half of the components reversed",
               fontsize=9)
left.legend(fontsize=7.5, loc="lower left", ncol=2)
left.set_ylim(-2.2, 1.7)
```

The left panel: the real parts of components 1 and 9 (positions 0 and 8) of the universe as thick pale lines (`linewidth=4.0`, `alpha=0.35`), and of the partner as dashed lines drawn on top. Component 1 lies in the half where $\Gamma = -1$, component 9 in the half where $\Gamma = +1$. Labels, title, legend and the vertical range follow.

```
eig_u = np.linalg.eigvals(np.array(A_matrix.tolist(), dtype=float))
eig_p = np.linalg.eigvals(np.array(A_partner.tolist(), dtype=float))
right.plot(eig_u.real, eig_u.imag, "o", markersize=11, markerfacecolor="none",
           color="tab:blue", label="universe: eigenvalues of $\\mathcal{A}$")
right.plot(eig_p.real, eig_p.imag, "x", markersize=9, color="tab:red",
           label="partner: eigenvalues of $\\mathcal{A}'$")
right.set_xlim(-1.5, 1.5)
right.set_ylim(-5.5, 5.5)
right.set_xlabel("real part")
right.set_ylabel("imaginary part (units $H$)")
right.set_title("equal spectra: $\\pm 4i$, eight times each", fontsize=9)
right.legend(fontsize=8, loc="center right")
```

The right panel: `np.linalg.eigvals` computes the 16 eigenvalues of $\mathcal{A}$ and of $\mathcal{A}'$ in floating point; they are drawn in the complex plane (real part horizontal, imaginary part vertical), the universe's as open circles (`markerfacecolor="none"`), the partner's as crosses.

```
check(np.allclose(np.sort(eig_u.imag), np.sort(eig_p.imag), atol=1e-12)
      and np.abs(eig_u.real).max() < 1e-12,
      "both spectra are +-4i, eight times each (numerically)")
```

The check: sorted by their imaginary parts, the two lists of eigenvalues agree to 10^{-12} (`np.allclose`), and their real parts are zero to 10^{-12}.

```
save_figure(fig, "partner_in_time",
            "Left: the real parts of components 1 and 9 of the universe "
...
            "the opposite energy density.")
```

What figure 4 shows. Left: the real parts of components 1 and 9 of the universe (thick pale) and of its T1 partner (dashed) against the time x_4 in units $1/H$. Component 1 of the partner is the universe's component 1 with the opposite sign; component 9 is the same; both oscillate with the period $2\pi/4 \approx 1.57$. Right: the eigenvalues of $\mathcal{A}$ (circles) and of $\mathcal{A}' = \Gamma\mathcal{A}\Gamma$ (crosses) in the complex plane: all lie at $+4i$ and $-4i$, eight times each, and the crosses sit inside the circles. The student

should see that the partner has the same frequency as the universe, although its energy density is the opposite: a negative energy density here is not a negative frequency.

In [14], the last check.

```
figure_names = ["einstein_residuals", "null_requirement", "bookkeeping",
                "partner_in_time"]
paths = [output_file(f"{FIGURE_FOLDER}/20b_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

As in Notebook 20a, In [19]: the four figure files `20b_1_einstein_residuals.png` to `20b_4_partner_in_time.png` must exist, and `all_checks_passed()` prints the last line. Output: PASS every figure file of this notebook exists, and ALL 25 CHECKS PASSED (notebook 20b).

20.17 Allowed is not the same as happens: an electron and a positron made from light

Why a lesson from ordinary physics. To judge whether the equations of this book prove that universes are created, we must know what a proof of creation contains when physics does have one. The creation of an electron and a positron (its **antiparticle**: the same mass, the opposite electric charge) from light is understood completely. It shows the three questions of Section 20.4 answered one after the other, and it shows that the answer to the first (allowed?) is computed from conservation laws alone, while the answers to the second and the third (does it happen? how often?) need a dynamical theory.

Words and the standard physics we use. We use ordinary space with three directions and one time, units in which the speed of light is 1, and energies, momenta and masses measured in units of the electron mass m. A body of mass μ with the **momentum** $\vec{p} = (p_1, p_2, p_3)$, a vector of length $|\vec{p}| = \sqrt{p_1^2 + p_2^2 + p_3^2}$, has the energy $E = \sqrt{\mu^2 + |\vec{p}|^2}$ (the energy-momentum relation of special relativity). A **photon**, a particle of light, has $\mu = 0$, so $E = |\vec{p}|$. For a collection of bodies the total energy E_{tot} and the total momentum $\vec{p}_{\text{tot}}$ are sums over the bodies, and the **invariant mass** μ of the collection is defined by

$$\mu^2 = E_{\text{tot}}^2 - |\vec{p}_{\text{tot}}|^2.$$

For a single body it is its mass. ASSUMED (standard physics, not derived in this book): the energy-momentum relation, and the conservation of the total energy, the total momentum and the total electric charge of an isolated system in every process. Since μ^2 is built from conserved totals, it is conserved too. A **threshold** is the smallest energy at which a process is allowed.

The rule of every threshold. A process from a state A to a state B is forbidden if some conserved quantity has different values in A and in B. So $\mu_A = \mu_B$ is NECESSARY. The lemma below says that a final state of bodies with masses $\mu_1, \mu_2, \ldots$ has $\mu_B \geq \mu_1 + \mu_2 + \cdots$, so the initial state must bring at least the sum of the final masses as invariant mass. This is a necessary condition only.

Lemma (two bodies have at least the sum of their masses). Two bodies with masses $\mu_1, \mu_2 \geq 0$ and momenta $\vec{p}_1, \vec{p}_2$ have together $(E_1 + E_2)^2 - |\vec{p}_1 + \vec{p}_2|^2 \geq (\mu_1 + \mu_2)^2$.

Proof, line by line. Write $a = |\vec{p}_1|$ and $b = |\vec{p}_2|$.

1. Multiplying out, $(E_1 + E_2)^2 - |\vec{p}_1 + \vec{p}_2|^2 = (E_1^2 - a^2) + (E_2^2 - b^2) + 2(E_1E_2 - \vec{p}_1 \cdot \vec{p}_2)$, because $|\vec{p}_1 + \vec{p}_2|^2 = a^2 + b^2 + 2\vec{p}_1 \cdot \vec{p}_2$ (the **dot product** $\vec{p}_1 \cdot \vec{p}_2 = p_{1,1}p_{2,1} + p_{1,2}p_{2,2} + p_{1,3}p_{2,3}$).
2. The energy-momentum relation $E_i^2 - |\vec{p}_i|^2 = \mu_i^2$ turns this into $\mu_1^2 + \mu_2^2 + 2(E_1E_2 - \vec{p}_1 \cdot \vec{p}_2)$.
3. The dot product is at most the product of the lengths, $\vec{p}_1 \cdot \vec{p}_2 = ab\cos\theta \leq ab$ (θ is the angle between the vectors). So it is enough to show $E_1E_2 \geq \mu_1\mu_2 + ab$.
4. Both sides of that inequality are not negative, so we may compare their squares: $E_1^2E_2^2 - (\mu_1\mu_2 + ab)^2 = (\mu_1^2 + a^2)(\mu_2^2 + b^2) - (\mu_1\mu_2 + ab)^2$.
5. Multiplying out, $(\mu_1^2 + a^2)(\mu_2^2 + b^2) = \mu_1^2\mu_2^2 + \mu_1^2b^2 + \mu_2^2a^2 + a^2b^2$ and $(\mu_1\mu_2 + ab)^2 = \mu_1^2\mu_2^2 + 2\mu_1\mu_2ab + a^2b^2$; the terms $\mu_1^2\mu_2^2$ and a^2b^2 cancel in the difference, and what remains is $\mu_1^2b^2 - 2\mu_1\mu_2ab + \mu_2^2a^2 = (\mu_1b - \mu_2a)^2 \geq 0$, a square.
6. Hence $E_1E_2 - \vec{p}_1 \cdot \vec{p}_2 \geq \mu_1\mu_2$, and line 2 is at least $\mu_1^2 + \mu_2^2 + 2\mu_1\mu_2 = (\mu_1 + \mu_2)^2$. QED.

Equality needs equality in lines 3 and 5: parallel momenta and $\mu_1 b = \mu_2 a$, that is $b/\mu_2 = a/\mu_1$: the two bodies move with the same velocity. Applied twice, the lemma gives the same for three bodies, $\mu \geq \mu_1 + \mu_2 + \mu_3$ (first combine two of them into one collection of invariant mass at least $\mu_1 + \mu_2$, then add the third).

Proposition 1 (one photon can never make a pair). A single photon in empty space cannot turn into an electron and a positron. *Proof.* Before: one photon, $\mu_A^2 = E^2 - |\vec{p}|^2 = 0$. After: two bodies of mass m, so $\mu_B \geq 2m > 0$ by the lemma. Conservation needs $\mu_A = \mu_B$, so $0 \geq 2m$, which is false. QED. Charge does not forbid the process ($-1 + 1 = 0$, the photon's charge), and energy alone does not either (the photon may have any energy): the COMBINATION of energy and momentum forbids it.

Proposition 2 (a photon on a nucleus). A photon of energy E_γ that hits a **nucleus** (the heavy centre of an atom) of mass M at rest can make an electron-positron pair, the nucleus remaining in the final state, only if $E_\gamma \geq 2m(1 + m/M)$. *Proof, line by line.*

1. Before: the total energy is $E_\gamma + M$ and the total momentum is the photon's, of length E_γ, so $\mu_A^2 = (E_\gamma + M)^2 - E_\gamma^2 = M^2 + 2E_\gamma M$ (multiply out; E_γ^2 cancels).
2. After: by the lemma applied twice, $\mu_B \geq m + m + M = 2m + M$.
3. Conservation, $\mu_A = \mu_B$, therefore needs $M^2 + 2E_\gamma M \geq (2m + M)^2 = M^2 + 4mM + 4m^2$.
4. Subtract M^2 from both sides and divide by $2M > 0$: $E_\gamma \geq 2m + 2m^2/M = 2m(1 + m/M)$. QED.

Conversely, at and above this energy the conservation laws allow the process: at exactly the threshold the three final bodies move together with the common velocity $v = |\vec{p}_{\text{tot}}|/E_{\text{tot}}$ (the equality case of the lemma), and each carries the share μ_i/μ of the total momentum. For $M = 4m$:

$E_\gamma = 2m(1+1/4) = 5m/2$; the totals are $|\vec{p}_{\rm tot}| = 5m/2$ and $E_{\rm tot} = 5m/2 + 4m = 13m/2$; $\mu = 6m$; the electron and the positron carry the momentum $(1/6)(5m/2) = 5m/12$ each and the energy $\sqrt{m^2 + 25m^2/144} = 13m/12$ each; the nucleus carries $(4/6)(5m/2) = 5m/3$ and $\sqrt{16m^2 + 25m^2/9} = 13m/3$. The momenta add up to $5m/12 + 5m/12 + 5m/3 = 30m/12 = 5m/2$ and the energies to $13m/12 + 13m/12 + 52m/12 = 78m/12 = 13m/2$: exactly the initial totals (COMPUTED exactly, Notebook 20c, In [5]). For a heavy nucleus the threshold approaches the two **rest energies** $2m$: for $M = 1000m$ it is $1001m/500 = 2.002m$, and for $M = 1836m$ (about the mass of a proton) $1837m/918 \approx 2.001089m$. The nucleus takes up the momentum but almost no energy.

Proposition 3 (two photons). Two photons of energies E_a and E_b that meet at the angle θ between their directions can make a pair only if $E_a E_b \geq 2m^2/(1-\cos\theta)$. *Proof.* $\mu^2 = (E_a+E_b)^2 - |\vec{p}_a+\vec{p}_b|^2 = E_a^2 + 2E_aE_b + E_b^2 - (E_a^2 + E_b^2 + 2E_aE_b\cos\theta) = 2E_aE_b(1-\cos\theta)$, using $|\vec{p}_a| = E_a$, $|\vec{p}_b| = E_b$ and $\vec{p}_a \cdot \vec{p}_b = E_aE_b\cos\theta$; the lemma requires $\mu \geq 2m$, that is $\mu^2 \geq 4m^2$, and dividing by $2(1-\cos\theta) > 0$ gives the claim. QED. Head on ($\theta = 180$ degrees, $\cos\theta = -1$) this is $E_aE_b \geq m^2$; photons moving in the same direction ($\theta = 0$, $\mu = 0$) can never make a pair.

Status. The lemma and Propositions 1 to 3 are PROVED here from the ASSUMED standard physics above, and Notebook 20c checks them with sympy and on random samples. They answer Q1 completely: below the threshold the process is forbidden, at and above it allowed. They say NOTHING about Q2 and Q3. That pairs are really made above the threshold, and how often, was computed from **quantum electrodynamics**, the quantum theory of electrons and light, by Bethe and Heitler (H. Bethe and W. Heitler, Proc. R. Soc. Lond. A 146, 83 (1934)) for a photon on a nucleus, and by Breit and Wheeler (G. Breit and J. A. Wheeler, Phys. Rev. 46, 1087 (1934)) for two photons. This book quotes these works; it does not compute their results.

Three lessons for universes.

1. Conservation laws are NECESSARY conditions. They can rule a process out (Proposition 1); they never rule a process in.

2. Whether an allowed process happens, and how often, is computed from the DYNAMICS, here an interaction between light and the electron field. Without it the photon could not hand its energy to the pair.

3. The electron and the positron are two quanta of ONE field, the electron field, whose total charge is $-1 + 1 = 0$, and that field interacts with light. The two universes of a T1 pair are configurations of two DIFFERENT theories, with parameters (m, λ) and $(-m, -\lambda)$, and no term of any Lagrangian of the Revision record couples them. Section 20.22 draws the consequence for their charges.

The comparison, item by item (the right column is the subject of Sections 20.22 to 20.25):

question	electron and positron from light	universes of masses $+m$ and $-m$
Q1: conserved quantities	energy, momentum, charge	the U(1) charge of each universe (local law PROVED; its total is constant only under the ASSUMED no-flux condition at the brane $z = \pi/2$); the momenta of each universe along $x_1, x_2, x_3, x_5, x_6, x_7$ and its rotation quantities (derived, Section 20.22 (d')); the energy of a universe is not conserved in the deflating metric
Q1: an interaction that lets energy or charge pass	light couples to electrons	none in the theory
Q2: the dynamics of creation	quantum electrodynamics	none derived
Q3: rate, probability, amplitude	computed in 1934 (quoted)	none

20.18 Example: the threshold of pair creation, computed

Notebook 20c proves the lemma's key line with sympy and tests the lemma on 20000 random pairs of bodies, shows on 5000 random electron-positron states that one photon can never make a pair, derives the threshold $2m(1 + m/M)$ on a nucleus with an exact final state at the threshold, and the allowed region for two photons. At the end it reads, from the Revision record of the pairing theorems, which of the three questions those theorems answer for universes, and the U(1) record of one universe. It prints 12 PASS lines, one of which reproduces a named check of a Revision report, and draws four figures. It needs numpy, sympy and matplotlib, no Rust, and runs in about 15 seconds (its recorded check run on the build computer, which was busy with other work at the time, took 28.1 seconds).

20.19 How to run Notebook 20c

Step 1. What this notebook does and what it needs.

Notebook 20c (Allowed is not the same as happens: the threshold of pair creation) is the file `Revision/textbook/notebooks/20c_allowed_not_happens.ipynb` of the repository Dirac_claude. It computes, from the relativistic relation between energy, momentum and mass and the conservation of energy and momentum, when the creation of an electron and a positron from light is allowed: it proves with sympy that the invariant mass of two bodies is at least the sum of their masses and tests it on 20000 random pairs, shows that a single photon can never make a pair, derives the threshold photon energy 2m(1 + m/M) on a nucleus of mass M with an exact final state at the threshold, and the allowed region for two photons at any angle. It then reads from the Revision record what the pairing theorems do not establish (no creation process, no rate, no amplitude), and draws four teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python

3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 20c_allowed_not_happens.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 20c_allowed_not_happens.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error,

every check passed; open the notebook with `jupyter lab 20c_allowed_not_happens.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/20c.captions.json`
- `Revision/textbook/figures/20c_1_invariant_mass_inequality.png`
- `Revision/textbook/figures/20c_2_nucleus_threshold.png`
- `Revision/textbook/figures/20c_3_allowed_photon_energies.png`
- `Revision/textbook/figures/20c_4_two_photon_regions.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 12 CHECKS PASSED (notebook 20c)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/20c_allowed_not_happens.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.

- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 20c_allowed_not_happens.ipynb
python -m nbconvert --execute --inplace 20c_allowed_not_happens.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads two files of the repository; it must be opened inside the folder Revision/textbook/notebooks of a complete copy of the repository made with git clone (a notebook copied alone to another folder cannot find them).

20.20 Notebook 20c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/20c_allowed_not_happens.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The request behind this chapter speaks of universes "created in pairs". Before we
can say what the equations of this book do and do not prove about that, we must be
clear what a proof of creation would contain. This notebook works it out on the
```

```
one example of pair creation that physics understands completely: an electron and
a positron (its antiparticle) made from light. It separates three questions:

- **Q1, allowed?** Do the conservation laws (energy, momentum, charge) permit the
  process? This is answered here completely, by algebra and by computation.
- **Q2, does it happen?** Is there a dynamical law that turns the initial state
  into the final one? For light this is quantum electrodynamics, which this
  notebook does NOT compute.
- **Q3, how often?** A number: a rate, a probability, a cross section. Again a
  result of the dynamics, not of the conservation laws.

It computes: the invariant mass and the inequality behind every threshold (proved
with sympy, tested on 20000 random pairs); why one photon can never make a pair;
the threshold photon energy on a nucleus, with an exact final state at the
threshold; and the allowed region for two photons. At the end it reads, from the
Revision record of the pairing theorems, which of the three questions those
theorems answer for universes: none of Q2 and Q3.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 20c (Allowed is not the same as happens: the threshold of pair creation) is the
file ‘Revision/textbook/notebooks/20c_allowed_not_happens.ipynb‘ of the repository
Dirac_claude. It computes, from the relativistic relation between energy, momentum and
mass and the conservation of energy and momentum, when the creation of an electron and a
positron from light is allowed: it proves with sympy that the invariant mass of two
bodies is at least the sum of their masses and tests it on 20000 random pairs, shows that
a single photon can never make a pair, derives the threshold photon energy 2m(1 + m/M) on
a nucleus of mass M with an exact final state at the threshold, and the allowed region
for two photons at any angle. It then reads from the Revision record what the pairing
theorems do not establish (no creation process, no rate, no amplitude), and draws four
teaching figures. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy,
sympy and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".
```

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 20c_allowed_not_happens.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 20c_allowed_not_happens.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 20c_allowed_not_happens.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/20c.captions.json‘
- ‘Revision/textbook/figures/20c_1_invariant_mass_inequality.png‘
- ‘Revision/textbook/figures/20c_2_nucleus_threshold.png‘
- ‘Revision/textbook/figures/20c_3_allowed_photon_energies.png‘
- ‘Revision/textbook/figures/20c_4_two_photon_regions.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS every figure file of this notebook exists
    ALL 12 CHECKS PASSED (notebook 20c)
```

and the notebook must show 4 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/20c_allowed_not_happens.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 20c_allowed_not_happens.ipynb
      python -m nbconvert --execute --inplace 20c_allowed_not_happens.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file below the folder Revision: the notebook reads two files of the repository; it must be opened inside the folder

Revision/textbook/notebooks of a complete copy of the repository made with git clone (a notebook copied alone to another folder cannot find them).

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 20c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 20c (Allowed is not the same as happens: the threshold of pair creation) is
# the file Revision/textbook/notebooks/20c_allowed_not_happens.ipynb of the repository
# Dirac_claude. It computes, from the relativistic relation between energy, momentum and
# mass and the conservation of energy and momentum, when the creation of an electron and
# a positron from light is allowed: it proves with sympy that the invariant mass of two
# bodies is at least the sum of their masses and tests it on 20000 random pairs, shows
# that a single photon can never make a pair, derives the threshold photon energy 2m(1 +
# m/M) on a nucleus of mass M with an exact final state at the threshold, and the allowed
# region for two photons at any angle. It then reads from the Revision record what the
# pairing theorems do not establish (no creation process, no rate, no amplitude), and
# draws four teaching figures. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
```

```
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
```

```
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 20c_allowed_not_happens.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
```

```
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 20c_allowed_not_happens.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 20c_allowed_not_happens.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/20c.captions.json
# - Revision/textbook/figures/20c_1_invariant_mass_inequality.png
# - Revision/textbook/figures/20c_2_nucleus_threshold.png
# - Revision/textbook/figures/20c_3_allowed_photon_energies.png
# - Revision/textbook/figures/20c_4_two_photon_regions.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 12 CHECKS PASSED (notebook 20c)
#
# and the notebook must show 4 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
```

```
#   Revision/textbook/notebooks/20c_allowed_not_happens.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 20c_allowed_not_happens.ipynb
#     python -m nbconvert --execute --inplace 20c_allowed_not_happens.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file below the folder Revision: the notebook reads two
#   files of the repository; it must be opened inside the folder
#   Revision/textbook/notebooks of a complete copy of the repository made with git clone
#   (a notebook copied alone to another folder cannot find them).
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "20c"  # this notebook: chapter 20, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
```

```
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
```

```
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 20c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Units**: the speed of light is 1, and energies, momenta and masses are
  measured in units of the electron mass $m$ (so $m = 1$ in the code).
- **Momentum** $\vec p = (p_1, p_2, p_3)$, a vector; its length is $|\vec p| =
  \sqrt{p_1^2 + p_2^2 + p_3^2}$. **Energy** $E$ of a body of mass $\mu$:
  $E = \sqrt{\mu^2 + |\vec p|^2}$ (the energy-momentum relation of special
  relativity). A **photon** (light) has $\mu = 0$, so $E = |\vec p|$.
- **Electron, positron**: two particles of the same mass $m$ and opposite
  electric charge $-1$ and $+1$. **Nucleus**: the heavy centre of an atom, mass
  $M$.
- **Conservation law**: a quantity whose total is the same before and after any
  process (here the total energy, the total momentum and the total charge).
- **Invariant mass** $\mu$ of a collection of bodies: $\mu^2 = E_{\rm tot}^2 -
  |\vec p_{\rm tot}|^2$, computed from the totals. Because the totals are
  conserved, so is $\mu$. For one body $\mu$ is its mass.
- **Threshold**: the smallest energy at which a process is allowed.
- **Allowed** (in this notebook): not forbidden by any conservation law. Allowed
  does not mean that the process happens, nor how often.
- **Rate, cross section, amplitude**: numbers that say how often a process
  happens; they come from a dynamical theory.
- **Lemma, proposition**: a proved statement (a lemma is a step towards others).
```

Text cell:

```
## 4. The physical and mathematical situation

**ASSUMED (standard physics of special relativity):** each body has $E^2 =
\mu^2 + |\vec p|^2$; the total energy and the total momentum of an isolated system
do not change; nor does its total electric charge. Everything else is derived and
computed here.

**The rule of every threshold.** A process A $\to$ B is forbidden if any conserved
quantity has different values in A and in B. Since $\mu^2$ is built from conserved
totals, $\mu_A = \mu_B$ is NECESSARY. The lemma of section 5 says that a final
state made of bodies with masses $\mu_1, \mu_2, \dots$ always has $\mu_B \geq \mu_1
+ \mu_2 + \cdots$. So the initial state must bring at least that much invariant
mass:

$$\mu_A \;\geq\; \text{(sum of the final masses)}.$$

This is a necessary condition only. Whether the process then happens, and how
often, is decided by the dynamics.
```

Text cell:

```
## 5. The lemma: two bodies have at least the sum of their masses

**Lemma.** Two bodies with masses $\mu_1, \mu_2 \geq 0$, momenta $\vec p_1,
\vec p_2$ and energies $E_i = \sqrt{\mu_i^2 + |\vec p_i|^2}$ have together
$(E_1 + E_2)^2 - |\vec p_1 + \vec p_2|^2 \geq (\mu_1 + \mu_2)^2$.
```

```
*Proof, line by line.* Write $a = |\vec p_1|$, $b = |\vec p_2|$.

1. Multiply out the squares: $(E_1 + E_2)^2 - |\vec p_1 + \vec p_2|^2 =
   (E_1^2 - a^2) + (E_2^2 - b^2) + 2(E_1E_2 - \vec p_1\cdot\vec p_2)$.
2. Use $E_i^2 - |\vec p_i|^2 = \mu_i^2$: this is $\mu_1^2 + \mu_2^2 +
   2(E_1E_2 - \vec p_1\cdot\vec p_2)$.
3. The dot product is at most the product of the lengths, $\vec p_1\cdot\vec p_2
   \leq ab$, so it suffices to show $E_1E_2 \geq \mu_1\mu_2 + ab$.
4. Both sides are not negative, so compare their squares:
   $E_1^2E_2^2 - (\mu_1\mu_2 + ab)^2 = (\mu_1^2 + a^2)(\mu_2^2 + b^2) -
   (\mu_1\mu_2 + ab)^2$.
5. Multiplying out, the terms $\mu_1^2\mu_2^2$ and $a^2b^2$ cancel and what remains
   is $\mu_1^2b^2 + \mu_2^2a^2 - 2\mu_1\mu_2ab = (\mu_1b - \mu_2a)^2 \geq 0$.
6. Hence $E_1E_2 - \vec p_1\cdot\vec p_2 \geq \mu_1\mu_2$, and line 2 is at least
   $\mu_1^2 + \mu_2^2 + 2\mu_1\mu_2 = (\mu_1 + \mu_2)^2$. QED.

The next cell checks line 5 with sympy, then tests the lemma on 20000 random
pairs of bodies (random masses between 0 and 3, random momenta, fixed seed).
```

In [2]:

```
import numpy as np  # arrays and random numbers
import sympy as sp  # exact algebra

mu1, mu2, a, b = sp.symbols("mu1 mu2 a b", nonnegative=True)
E1_sq, E2_sq = mu1 ** 2 + a ** 2, mu2 ** 2 + b ** 2  # E_i^2 = mu_i^2 + |p_i|^2
line5 = sp.expand(E1_sq * E2_sq - (mu1 * mu2 + a * b) ** 2)
check(sp.expand(line5 - (mu1 * b - mu2 * a) ** 2) == 0,
      "line 5: E1^2 E2^2 - (mu1 mu2 + a b)^2 = (mu1 b - mu2 a)^2")


def invariant_mass(energies, momenta):
    """mu = sqrt(E_tot^2 - |p_tot|^2) of a collection of bodies."""
    E_total = np.sum(energies, axis=0)
    p_total = np.sum(momenta, axis=0)
    return np.sqrt(np.maximum(E_total ** 2 - np.sum(p_total ** 2, axis=-1), 0.0))


rng = np.random.default_rng(20261007)  # fixed seed: the same numbers every run
count = 20000
masses = rng.uniform(0.0, 3.0, size=(2, count))  # mu1, mu2
momenta = rng.normal(size=(2, count, 3)) * rng.uniform(0.0, 4.0, size=(2, count, 1))
energies = np.sqrt(masses ** 2 + np.sum(momenta ** 2, axis=-1))
mu_total = invariant_mass(energies, momenta)
margin = mu_total - masses.sum(axis=0)  # must never be negative
report("smallest margin mu_total - (mu1 + mu2) in 20000 pairs",
       f"{margin.min():.2e}")
check(margin.min() > -1e-12, "the lemma holds for all 20000 random pairs")
```

Out [2]:

```
PASS line 5: E1^2 E2^2 - (mu1 mu2 + a b)^2 = (mu1 b - mu2 a)^2
```

```
RESULT smallest margin mu_total - (mu1 + mu2) in 20000 pairs = 2.34e-04
PASS the lemma holds for all 20000 random pairs
```

Text cell:

```
The next cell draws the 20000 pairs: the invariant mass of the pair against the
sum of the two masses. Every point lies on or above the diagonal; points on the
diagonal are pairs whose bodies move together (equal velocities, the equality case
$\mu_1b = \mu_2a$ of line 5 with parallel momenta).
```

In [3]:

```
fig, ax = plt.subplots(figsize=(6.4, 5.0))
ax.plot(masses.sum(axis=0), mu_total, ".", markersize=1.5, alpha=0.4,
        color="tab:blue", label="20000 random pairs")
ax.plot([0, 6], [0, 6], color="black", linewidth=1.5,
        label="$\\mu = \\mu_1 + \\mu_2$")
ax.set_xlabel("sum of the masses $\\mu_1 + \\mu_2$ (units $m$)")
ax.set_ylabel("invariant mass of the pair $\\mu$ (units $m$)")
ax.set_xlim(0, 6)
ax.set_ylim(0, 16)
ax.set_title("Two bodies have at least the sum of their masses")
ax.legend(fontsize=8, loc="upper left", markerscale=6)
save_figure(fig, "invariant_mass_inequality",
            "The invariant mass "
            "$\\mu = \\sqrt{E_{\\rm tot}^2 - |\\vec p_{\\rm tot}|^2}$ of 20000 random "
            "pairs of bodies (vertical) against the sum of their "
            "masses $\\mu_1 + \\mu_2$ (horizontal), both in units of the electron "
            "mass $m$; masses between 0 and 3, random momenta, fixed seed. Every "
            "point lies on or above the black diagonal $\\mu = \\mu_1 + \\mu_2$, as "
            "the lemma proves: a final state of bodies with given masses needs at "
            "least the sum of those masses as invariant mass, which is the origin "
            "of every threshold.")
```

Out [3]:

```
Figure 20c.1 saved as Revision/textbook/figures/20c_1_invariant_mass_inequality.png
```

Text cell:

```
## 6. One photon can never make a pair

**Proposition.** A single photon in empty space cannot turn into an electron and
a positron. *Proof.* Before: one photon, $\mu_A^2 = E^2 - |\vec p|^2 = 0$. After:
two bodies of mass $m$, so by the lemma $\mu_B \geq 2m > 0$. The invariant mass is
conserved, so $0 \geq 2m$, which is false. QED.

Charge does not forbid it ($-1 + 1 = 0$, the photon's charge), and energy alone
does not either (the photon may have any energy): the COMBINATION of energy and
momentum forbids it. The next cell checks the statement on 5000 random final
states: none has invariant mass zero; the smallest is $2m$.
```

Figure 518: The invariant mass $\mu = \sqrt{E_{\mathrm{tot}}^2 - |\vec{p}_{\mathrm{tot}}|^2}$ of 20000 random pairs of bodies (vertical) against the sum of their masses $\mu_1 + \mu_2$ (horizontal), both in units of the electron mass m; masses between 0 and 3, random momenta, fixed seed. Every point lies on or above the black diagonal $\mu = \mu_1 + \mu_2$, as the lemma proves: a final state of bodies with given masses needs at least the sum of those masses as invariant mass, which is the origin of every threshold. (Notebook 20c, figure 1.)

In [4]:

```
pair_momenta = rng.normal(size=(2, 5000, 3)) * 3.0  # electron and positron
pair_energies = np.sqrt(1.0 + np.sum(pair_momenta ** 2, axis=-1))  # m = 1
pair_mu = invariant_mass(pair_energies, pair_momenta)
report("smallest invariant mass of 5000 electron-positron states",
       f"{pair_mu.min():.4f}")
check(pair_mu.min() >= 2.0 - 1e-12, "every electron-positron state has mu >= 2 m > 0")
```

Out [4]:

```
RESULT smallest invariant mass of 5000 electron-positron states = 2.0003
PASS every electron-positron state has mu >= 2 m > 0
```

Text cell:

```
## 7. A photon on a nucleus: the threshold

A photon of energy $E_\gamma$ hits a nucleus of mass $M$ at rest; the final state
is an electron, a positron and the nucleus. Line by line:

1. Before: total energy $E_\gamma + M$, total momentum of length $E_\gamma$ (the
   photon's), so $\mu_A^2 = (E_\gamma + M)^2 - E_\gamma^2 = M^2 + 2E_\gamma M$.
2. After: by the lemma (applied twice) $\mu_B \geq m + m + M$.
3. Conservation, $\mu_A = \mu_B$, therefore needs $M^2 + 2E_\gamma M \geq
   (2m + M)^2 = M^2 + 4mM + 4m^2$.
4. Subtract $M^2$ and divide by $2M$: $E_\gamma \geq 2m + 2m^2/M =
   2m(1 + m/M)$.

At exactly the threshold the three final bodies move together with the common
velocity $v = p_{\rm tot}/E_{\rm tot}$ (the equality case of the lemma). For
$M = 4m$: $E_\gamma = 5m/2$, $p_{\rm tot} = 5m/2$, $E_{\rm tot} = 13m/2$,
$\mu = 6m$, and each body carries the share $\mu_i/\mu$ of the momentum: $5m/12$
for the electron and for the positron, $5m/3$ for the nucleus. The next cell
solves line 3 with sympy, checks this final state EXACTLY (energies and momenta
add up to the initial ones), and computes the threshold for three nuclei: $M =
4m$ (helium-like), $1000m$ and $1836m$ (about a proton).
```

In [5]:

```
E_gamma, M_nuc, m_e = sp.symbols("E_gamma M m", positive=True)
mu_initial_sq = (E_gamma + M_nuc) ** 2 - E_gamma ** 2  # line 1
threshold = sp.solve(sp.Eq(mu_initial_sq, (2 * m_e + M_nuc) ** 2), E_gamma)[0]
say(f"threshold photon energy: {sp.factor(threshold)}")
check(sp.simplify(threshold - 2 * m_e * (1 + m_e / M_nuc)) == 0,
      "E_threshold = 2 m (1 + m/M)")
M4 = 4  # the nucleus of mass 4 m, with m = 1
E_th = threshold.subs({m_e: 1, M_nuc: M4})  # 5/2
p_total, E_total = E_th, E_th + M4  # the initial totals
mu_final = sp.Integer(2 + M4)  # 6 m
```

```
shares = [1, 1, M4]  # electron, positron, nucleus masses
p_parts = [sp.Rational(mass) * p_total / mu_final for mass in shares]
E_parts = [sp.sqrt(mass ** 2 + p ** 2) for mass, p in zip(shares, p_parts)]
say(f"final state at threshold: momenta {p_parts}, energies {E_parts}")
check(sum(p_parts) == p_total and sp.simplify(sum(E_parts) - E_total) == 0
      and E_th == sp.Rational(5, 2),
      "M = 4 m: the threshold state conserves energy and momentum exactly")
for M_value in (4, 1000, 1836):
    value = threshold.subs({m_e: 1, M_nuc: M_value})
    report(f"threshold for M = {M_value} m", f"{sp.nsimplify(value)} = "
           f"{float(value):.6f} m")
check(threshold.subs({m_e: 1, M_nuc: 1836}) == 2 + sp.Rational(2, 1836),
      "M = 1836 m: the threshold is 2 m + 2 m/1836, just above 2 m")
```

Out [5]:

```
threshold photon energy: 2*m*(M + m)/M
PASS E_threshold = 2 m (1 + m/M)
final state at threshold: momenta [5/12, 5/12, 5/3], energies [13/12, 13/12, 13/3]
PASS M = 4 m: the threshold state conserves energy and momentum exactly
RESULT threshold for M = 4 m = 5/2 = 2.500000 m
RESULT threshold for M = 1000 m = 1001/500 = 2.002000 m
RESULT threshold for M = 1836 m = 1837/918 = 2.001089 m
PASS M = 1836 m: the threshold is 2 m + 2 m/1836, just above 2 m
```

Text cell:

```
The next cell draws the threshold $E_\gamma/m = 2(1 + m/M)$ against the nucleus
mass $M/m$ on a logarithmic axis: a light nucleus needs much more than the two rest
energies $2m$, a heavy one takes up the momentum and almost no energy.
```

In [6]:

```
M_axis = np.logspace(-0.5, 4, 400)  # M/m from about 0.3 to 10000
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.semilogx(M_axis, 2 * (1 + 1 / M_axis), color="tab:blue",
            label="threshold $2m(1 + m/M)$")
ax.axhline(2.0, color="black", linestyle=":", label="the two rest energies $2m$")
for M_value, marker in ((4, "o"), (1000, "s"), (1836, "^")):
    ax.plot([M_value], [2 * (1 + 1 / M_value)], marker, color="tab:red",
            markersize=7, label=f"$M = {M_value}m$: {2 * (1 + 1 / M_value):.5f}$m$")
ax.fill_between(M_axis, 2 * (1 + 1 / M_axis), 9.0, color="tab:green", alpha=0.12,
                label="allowed (not forbidden)")
ax.set_ylim(1.5, 9.0)
ax.set_xlabel("mass of the nucleus $M/m$")
ax.set_ylabel("photon energy $E_\\gamma/m$")
ax.set_title("Pair creation on a nucleus: the threshold")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "nucleus_threshold",
            "The smallest photon energy $E_\\gamma$ (vertical, units of the "
            "electron mass $m$) at which energy and momentum conservation allow a "
```

```
"photon hitting a nucleus of mass $M$ (horizontal, logarithmic, units "
"$m$) to make an electron-positron pair: $E_\\gamma = 2m(1 + m/M)$ "
"(blue). Above it (green) the process is allowed, below it forbidden. "
"Red markers: $M = 4m$ ($2.5m$), $1000m$ ($2.002m$) and $1836m$, about "
"a proton ($2.00109m$). For heavy nuclei the threshold approaches the "
"two rest energies $2m$ (dotted). The curve says nothing about how "
"often the process happens above the threshold.")
```

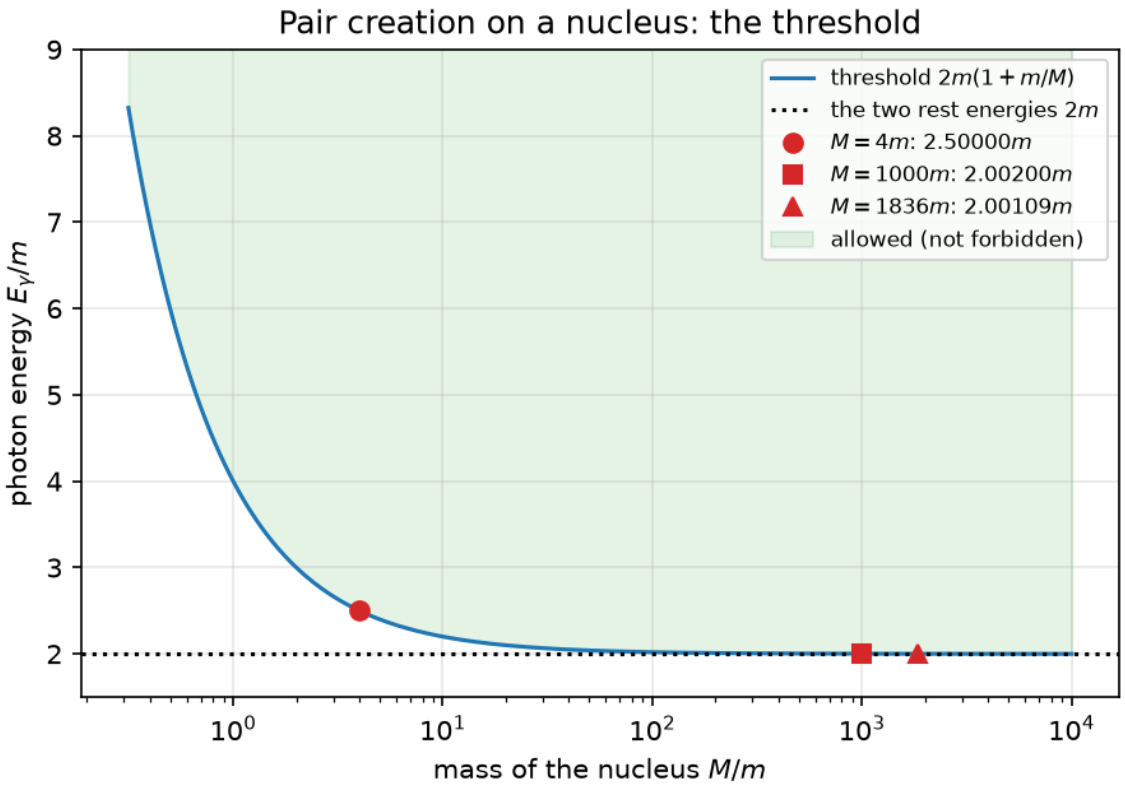

Figure 519: The smallest photon energy E_γ (vertical, units of the electron mass m) at which energy and momentum conservation allow a photon hitting a nucleus of mass M (horizontal, logarithmic, units m) to make an electron-positron pair: $E_\gamma = 2m(1+m/M)$ (blue). Above it (green) the process is allowed, below it forbidden. Red markers: $M = 4m$ ($2.5m$), $1000m$ ($2.002m$) and $1836m$, about a proton ($2.00109m$). For heavy nuclei the threshold approaches the two rest energies $2m$ (dotted). The curve says nothing about how often the process happens above the threshold. (Notebook 20c, figure 2.)

Out [6]:

```
Figure 20c.2 saved as Revision/textbook/figures/20c_2_nucleus_threshold.png
```

Text cell:

```
The next cell shows the same threshold as a comparison of two curves for $M =
4m$: the invariant mass the initial state brings, $\mu_A = \sqrt{M^2 +
```

```
2E_\gamma M}$, rises with the photon energy; the invariant mass the final state
needs at least is the constant $2m + M = 6m$. They cross at $E_\gamma = 5m/2$.
```

In [7]:

```
E_axis = np.linspace(0.0, 6.0, 601)
brought = np.sqrt(M4 ** 2 + 2 * E_axis * M4)  # mu_A for M = 4 m
crossing = E_axis[np.argmin(np.abs(brought - (2 + M4)))]
report("crossing of the two curves (M = 4 m)", f"{crossing:.2f} m")
check(abs(crossing - 2.5) < 0.01, "the curves cross at E_gamma = 2.5 m")
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.plot(E_axis, brought, color="tab:blue", label="brought: $\\sqrt{M^2 + 2E_\\gamma M}$")
ax.axhline(2 + M4, color="tab:red", label="needed at least: $2m + M = 6m$")
ax.axvline(2.5, color="black", linestyle=":", label="threshold $E_\\gamma = 5m/2$")
ax.fill_between(E_axis, 2 + M4, brought, where=brought >= 2 + M4,
                color="tab:green", alpha=0.15, label="allowed")
ax.set_xlabel("photon energy $E_\\gamma/m$")
ax.set_ylabel("invariant mass (units $m$)")
ax.set_title("Photon on a nucleus of mass $M = 4m$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "allowed_photon_energies",
            "For a photon hitting a nucleus of mass $M = 4m$ at rest: the invariant "
            "mass the initial state brings, $\\sqrt{M^2 + 2E_\\gamma M}$ (blue), "
            "and the smallest invariant mass the final state of electron, positron "
            "and nucleus needs, $2m + M = 6m$ (red), against the photon energy "
            "$E_\\gamma$ (horizontal; all in units of the electron mass $m$). "
            "Conservation of energy and momentum allows the pair only where the "
            "blue curve is at or above the red line (green), from the threshold "
            "$E_\\gamma = 5m/2$ (dotted) on.")
```

Out [7]:

```
RESULT crossing of the two curves (M = 4 m) = 2.50 m
PASS the curves cross at E_gamma = 2.5 m
```

Out [7]:

```
Figure 20c.3 saved as Revision/textbook/figures/20c_3_allowed_photon_energies.png
```

Text cell:

```
## 8. Two photons

Two photons of energies $E_a$ and $E_b$ meet at the angle $\theta$ between their
directions. Line by line: $\mu^2 = (E_a + E_b)^2 - |\vec p_a + \vec p_b|^2$;
multiplying out with $|\vec p_a| = E_a$, $|\vec p_b| = E_b$ and $\vec p_a\cdot\vec
p_b = E_aE_b\cos\theta$ gives $\mu^2 = 2E_aE_b(1 - \cos\theta)$. The lemma requires
$\mu \geq 2m$, so the pair is allowed exactly when
```

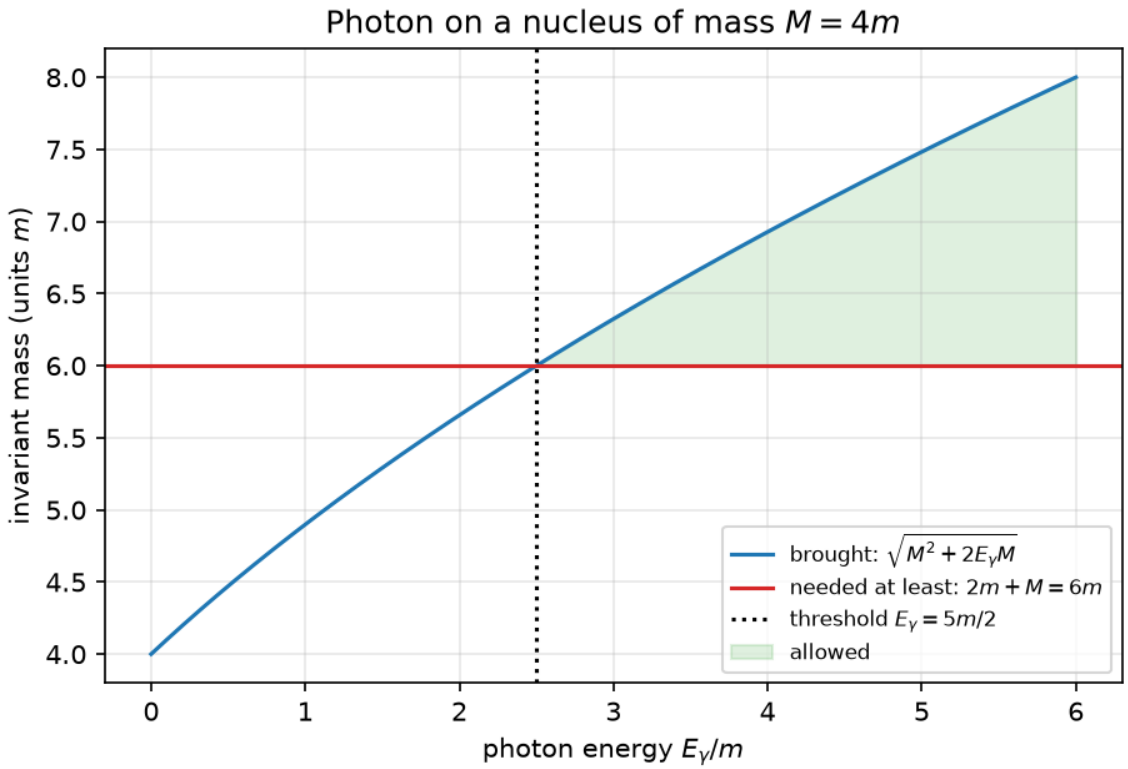

Figure 520: For a photon hitting a nucleus of mass $M = 4m$ at rest: the invariant mass the initial state brings, $\sqrt{M^2 + 2E_\gamma M}$ (blue), and the smallest invariant mass the final state of electron, positron and nucleus needs, $2m + M = 6m$ (red), against the photon energy E_γ (horizontal; all in units of the electron mass m). Conservation of energy and momentum allows the pair only where the blue curve is at or above the red line (green), from the threshold $E_\gamma = 5m/2$ (dotted) on. (Notebook 20c, figure 3.)

```
$$E_aE_b \;\geq\; \frac{2m^2}{1 - \cos\theta}.$$

Head on ($\theta = 180$ degrees, $\cos\theta = -1$) this is $E_aE_b \geq m^2$; for
photons moving in the same direction ($\theta = 0$) never. The next cell checks
the formula with sympy and draws the allowed regions for three angles.
```

In [8]:

```
Ea, Eb, theta = sp.symbols("E_a E_b theta", positive=True)
pa = sp.Matrix([Ea, 0, 0])  # photon a along the first axis
pb = sp.Matrix([Eb * sp.cos(theta), Eb * sp.sin(theta), 0])  # photon b at angle
mu_sq = sp.simplify((Ea + Eb) ** 2 - (pa + pb).dot(pa + pb))
check(sp.simplify(mu_sq - 2 * Ea * Eb * (1 - sp.cos(theta))) == 0,
      "two photons: mu^2 = 2 E_a E_b (1 - cos theta)")
check(sp.simplify(mu_sq.subs(theta, sp.pi) - 4 * Ea * Eb) == 0
      and mu_sq.subs(theta, 0) == 0,
      "head on mu^2 = 4 E_a E_b; same direction mu^2 = 0 (never allowed)")
grid = np.linspace(0.05, 6.0, 300)
EA, EB = np.meshgrid(grid, grid)
fig, ax = plt.subplots(figsize=(6.4, 5.4))
for degrees, colour in ((180, "tab:blue"), (90, "tab:orange"), (45, "tab:red")):
    bound = 2.0 / (1 - np.cos(np.radians(degrees)))  # E_a E_b >= bound (m = 1)
    ax.contourf(EA, EB, (EA * EB >= bound).astype(float), levels=[0.5, 1.5],
                colors=[colour], alpha=0.12)
    ax.contour(EA, EB, EA * EB, levels=[bound], colors=[colour])
    ax.plot([], [], color=colour,
            label=f"$\\theta = {degrees}$ deg: $E_aE_b \\geq {bound:.2f}m^2$")
ax.set_xlabel("energy of photon a, $E_a/m$")
ax.set_ylabel("energy of photon b, $E_b/m$")
ax.set_title("Two photons: allowed above each curve")
ax.legend(fontsize=8, loc="upper right")
save_figure(fig, "two_photon_regions",
            "Two photons of energies $E_a$ and $E_b$ (horizontal and vertical, "
            "units of the electron mass $m$) meeting at the angle $\\theta$: an "
            "electron-positron pair is allowed above the curve $E_aE_b = "
            "2m^2/(1 - \\cos\\theta)$ (shaded), drawn for head-on photons, "
            "$\\theta = 180$ degrees (blue, $E_aE_b \\geq m^2$), for $90$ degrees "
            "(orange, $2m^2$) and $45$ degrees (red, about $6.83m^2$). The smaller "
            "the angle, the more energy is needed; photons moving in the same "
            "direction can never make a pair.")
```

Out [8]:

```
PASS two photons: mu^2 = 2 E_a E_b (1 - cos theta)
PASS head on mu^2 = 4 E_a E_b; same direction mu^2 = 0 (never allowed)
```

Out [8]:

```
Figure 20c.4 saved as Revision/textbook/figures/20c_4_two_photon_regions.png
```

Text cell:

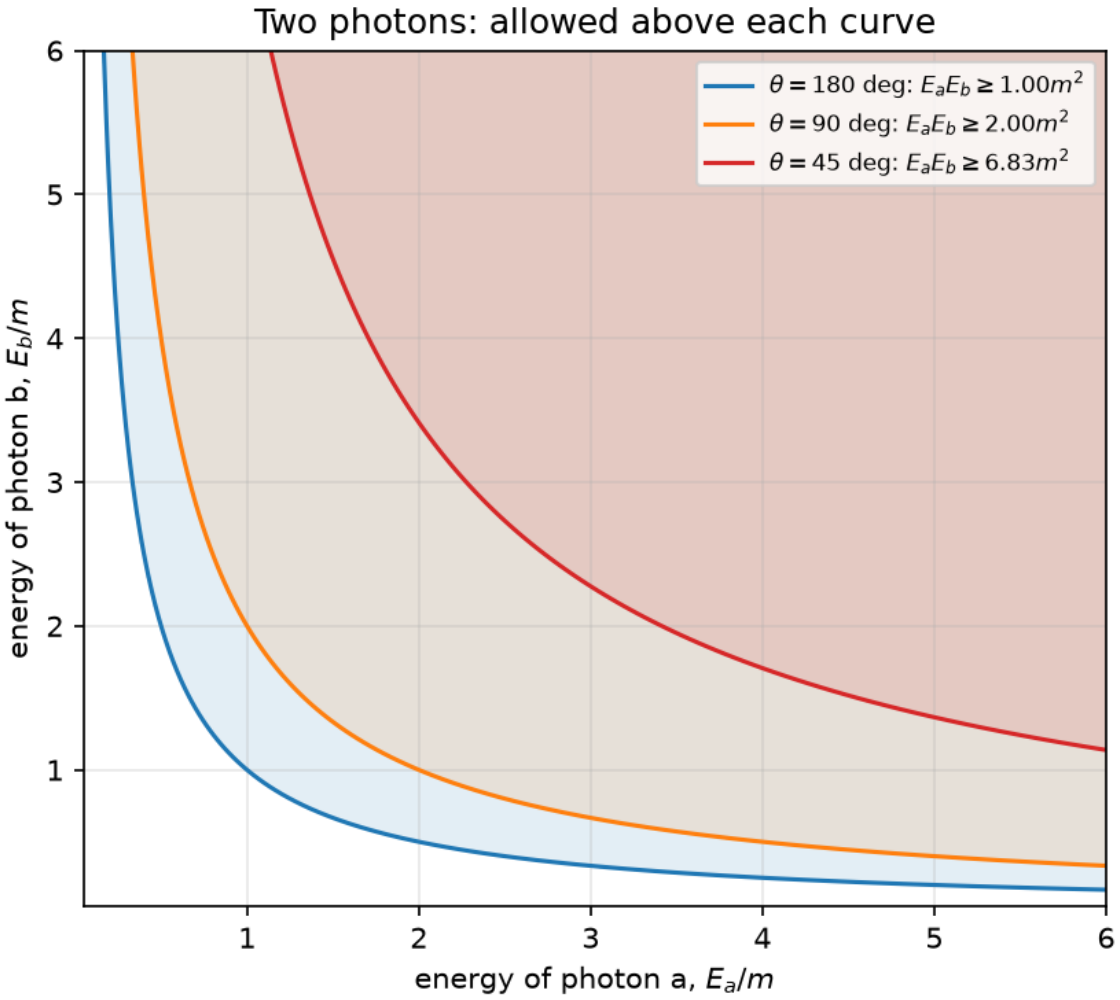

Figure 521: Two photons of energies E_a and E_b (horizontal and vertical, units of the electron mass m) meeting at the angle θ: an electron-positron pair is allowed above the curve $E_aE_b = 2m^2/(1-\cos\theta)$ (shaded), drawn for head-on photons, $\theta = 180$ degrees (blue, $E_aE_b \geq m^2$), for 90 degrees (orange, $2m^2$) and 45 degrees (red, about $6.83m^2$). The smaller the angle, the more energy is needed; photons moving in the same direction can never make a pair. (Notebook 20c, figure 4.)

9. What the conservation laws cannot say, and what this means for universes

Sections 5 to 8 answered Q1 completely for light: below a threshold the process is forbidden, above it allowed. They say NOTHING about Q2 and Q3. That pairs are really made above the threshold, and how often, was computed from quantum electrodynamics, the dynamical theory of electrons and light, by Bethe and Heitler (H. Bethe and W. Heitler, Proc. R. Soc. Lond. A 146, 83 (1934)) for a photon on a nucleus and by Breit and Wheeler (G. Breit and J. A. Wheeler, Phys. Rev. 46, 1087 (1934)) for two photons. This notebook quotes these works; it does not compute their results.

For universes of masses $+m$ and $-m$ the theory of this book supplies:

- **Q1, conserved quantities.** Light: energy, momentum and charge. Universes: the U(1) charge of each universe obeys an exact local conservation law (PROVED in the record); its total is constant only if no charge flows through the brane $z = \pi/2$ (the no-flux condition, ASSUMED: the record does not derive it, and on the homogeneous solutions of chapter 18 charge does flow through the brane). The momenta along the six directions on which the metric does not depend (x_1, x_2, x_3, x_5, x_6, x_7) obey local laws too, separately for each universe (derived from the record's on-shell law $\nabla_\mu T^\mu{}_\nu = 0$ and the symmetry of T; ASSUMED: the boundary terms vanish); the energy of a universe is NOT a conserved quantity, because the deflating metric depends on the time x_4 (for a homogeneous source the record's identity is $d\rho/dx_4 = -3a_4'(p_3 - p_t)$).
- **Q1, interaction.** Light couples to electrons. No term of any Lagrangian of the record couples two universes.
- **Q2, the dynamics of creation.** Light: quantum electrodynamics. Universes: none; no creation process is derived.
- **Q3, rate and amplitude.** Light: computed in 1934. Universes: none; no rate, probability or amplitude.

One consequence can be drawn in two lines from the record's conservation law (status: derived here; ASSUMED: the boundary terms vanish, e.g. periodic coordinates and no flux through the brane $z = \pi/2$, a condition that the record does not derive). Each universe obeys its own field equation and nothing couples the two, so under these conditions the charge Q_+ of the first universe is conserved on its own. If both fields were zero at some time, Q_+ would be zero at every time. A T1 pair whose members carry the charges Q and $-Q$ with $Q \neq 0$ can therefore not evolve out of zero fields in the theory as built: the zero TOTAL charge of a T1 pair does not make its appearance allowed, because the separate charges are conserved too (under the no-flux condition).

The next cell reads the record's own list of what the pairing theorems do not establish and checks that it names the missing answers to Q2 and Q3.

In [9]:

```
theory = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                    .read_text(encoding="utf-8"))
missing = theory["not_established"]  # the record's list
say(f"the record lists {len(missing)} things the pairing theorems do not establish;")
```

```
for item in missing[:3]:
    say("- " + item.split(":")[0] + ": " + item.split(":")[1].split(";")[0].strip())
starts = [item.split(":")[0] for item in missing]
check({"No creation process", "No rate and no amplitude",
       "No dynamical necessity"} <= set(starts),
      "the record states: no creation process, no rate or amplitude, no necessity")
u1 = json.loads(repository_file(
    "Revision/lead_checks/reports/charge-conjugation-and-u1.json")
    .read_text(encoding="utf-8"))
verdicts = {entry["name"]: entry["verdict"] for entry in u1["checks"]}
check(verdicts.get("u1_noether_matrix_identity") == "PASS",
      "the record proves the local U(1) conservation law of one universe",
      record="Revision/lead_checks/reports/charge-conjugation-and-u1.json, check "
             "u1_noether_matrix_identity")
```

Out [9]:

```
the record lists 9 things the pairing theorems do not establish;
- No creation process: the theorems map solutions to solutions and quantities to
    quantities
- No rate and no amplitude: no transition amplitude, probability, cross-section, rate or
    Bogoliubov coefficient for creating universes of masses {+m, -m} is computed or
    implied.
- No dynamical necessity: nothing forces the partner configuration to exist or to be
    realised
PASS the record states: no creation process, no rate or amplitude, no necessity
PASS the record proves the local U(1) conservation law of one universe
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    u1_noether_matrix_identity
```

Text cell:

```
## 10. The last check

The last cell checks that the four figure files exist in the folder
Revision/textbook/figures and prints the number of checks that passed.
```

In [10]:

```
figure_names = ["invariant_mass_inequality", "nucleus_threshold",
                "allowed_photon_energies", "two_photon_regions"]
paths = [output_file(f"{FIGURE_FOLDER}/20c_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [10]:

```
PASS every figure file of this notebook exists
ALL 12 CHECKS PASSED (notebook 20c)
```

Text cell:

```
## 11. What this notebook showed

- The invariant mass of two bodies is at least the sum of their masses (PROVED
  with sympy, and true for 20000 random pairs); this is the origin of every
  threshold (figure 1).
- One photon can never make an electron-positron pair; on a nucleus of mass $M$
  the photon needs at least $2m(1 + m/M)$: $2.5m$ for $M = 4m$, $2.00109m$ for
  $M = 1836m$, with an exact final state at the threshold (figures 2 and 3); two
  photons need $E_aE_b \geq 2m^2/(1 - \cos\theta)$ (figure 4).
- These are answers to Q1 only (allowed or forbidden). That the process happens,
  and how often, needs a dynamical theory (quantum electrodynamics, quoted, not
  computed).
- For universes the record has local conservation laws but no interaction
  between universes, no creation process, no rate and no amplitude: Q2 and Q3 are
  not answered, and with separately conserved charges (ASSUMED: no flux through
  the brane) a T1 pair with nonzero charges cannot evolve out of zero fields in
  the theory as built.
```

20.21 Line-by-line walk-through of Notebook 20c

The notebook has 10 code cells, In [1] to In [10].

In [1], the set-up cell. It is word for word the set-up cell of Notebook 20a, explained line by line in Section 20.11, except that its comment lines hold the run instructions of Notebook 20c (Section 20.19) and one line names the notebook:

```
NOTEBOOK_ID = "20c"  # this notebook: chapter 20, example c
```

Output: Set-up of notebook 20c complete.

In [2], the lemma.

```
import numpy as np  # arrays and random numbers
import sympy as sp  # exact algebra

mu1, mu2, a, b = sp.symbols("mu1 mu2 a b", nonnegative=True)
E1_sq, E2_sq = mu1 ** 2 + a ** 2, mu2 ** 2 + b ** 2  # E_i^2 = mu_i^2 + |p_i|^2
line5 = sp.expand(E1_sq * E2_sq - (mu1 * mu2 + a * b) ** 2)
check(sp.expand(line5 - (mu1 * b - mu2 * a) ** 2) == 0,
      "line 5: E1^2 E2^2 - (mu1 mu2 + a b)^2 = (mu1 b - mu2 a)^2")
```

numpy and sympy are loaded. The four symbols μ_1, μ_2, a, b are declared not negative (`nonnegative=True`), as in the lemma. `E1_sq` and `E2_sq` are $E_1^2 = \mu_1^2 + a^2$ and $E_2^2 = \mu_2^2 + b^2$. `line5` is $E_1^2E_2^2 - (\mu_1\mu_2 + ab)^2$, multiplied out; the check confirms that it equals $(\mu_1 b - \mu_2 a)^2$, line 5 of the proof, exactly.

```
def invariant_mass(energies, momenta):
    """mu = sqrt(E_tot^2 - |p_tot|^2) of a collection of bodies."""
```

```
    E_total = np.sum(energies, axis=0)
    p_total = np.sum(momenta, axis=0)
    return np.sqrt(np.maximum(E_total ** 2 - np.sum(p_total ** 2, axis=-1), 0.0))
```

`invariant_mass` computes $\mu = \sqrt{E_{\text{tot}}^2 - |\vec{p}_{\text{tot}}|^2}$ for many collections at once. The energies form an array with one row per body; `np.sum(..., axis=0)` adds the rows, so `E_total` holds the total energy of each collection. The momenta form an array with one row per body and three numbers per momentum; `axis=0` again adds over the bodies. `np.sum(p_total ** 2, axis=-1)` adds the squares of the three components (the last axis) and gives $|\vec{p}_{\text{tot}}|^2$. `np.maximum(..., 0.0)` replaces a tiny negative rounding error by 0 before the square root.

```
rng = np.random.default_rng(20261007)  # fixed seed: the same numbers every run
count = 20000
masses = rng.uniform(0.0, 3.0, size=(2, count))  # mu1, mu2
momenta = rng.normal(size=(2, count, 3)) * rng.uniform(0.0, 4.0, size=(2, count, 1))
energies = np.sqrt(masses ** 2 + np.sum(momenta ** 2, axis=-1))
mu_total = invariant_mass(energies, momenta)
margin = mu_total - masses.sum(axis=0)  # must never be negative
report("smallest margin mu_total - (mu1 + mu2) in 20000 pairs",
       f"{margin.min():.2e}")
check(margin.min() > -1e-12, "the lemma holds for all 20000 random pairs")
```

A random-number generator with a fixed seed. `masses` is a 2×20000 array of masses drawn uniformly between 0 and 3. `momenta` is a $2 \times 20000 \times 3$ array: normal random vectors, each multiplied by a random length factor between 0 and 4 (the last axis of size 1 multiplies all three components of a vector by the same number). `energies` is $\sqrt{\mu^2 + |\vec{p}|^2}$ for each body. `margin` is the invariant mass of each pair minus the sum of its two masses; the lemma says it is never negative. Output: RESULT smallest margin mu_total - (mu1 + mu2) in 20000 pairs = 2.34e-04, and PASS the lemma holds for all 20000 random pairs (the check allows -10^{-12} for rounding).

In [3], figure 1: the lemma on 20000 pairs.

```
fig, ax = plt.subplots(figsize=(6.4, 5.0))
ax.plot(masses.sum(axis=0), mu_total, ".", markersize=1.5, alpha=0.4,
        color="tab:blue", label="20000 random pairs")
ax.plot([0, 6], [0, 6], color="black", linewidth=1.5,
        label="$\\mu = \\mu_1 + \\mu_2$")
ax.set_xlabel("sum of the masses $\\mu_1 + \\mu_2$ (units $m$)")
ax.set_ylabel("invariant mass of the pair $\\mu$ (units $m$)")
ax.set_xlim(0, 6)
ax.set_ylim(0, 16)
ax.set_title("Two bodies have at least the sum of their masses")
ax.legend(fontsize=8, loc="upper left", markerscale=6)
```

A scatter plot: each pair is a dot (marker `"."`, size 1.5, 40 per cent opaque) at the sum of its masses (horizontal) and its invariant mass (vertical); the black line is the diagonal $\mu = \mu_1 + \mu_2$. The axis ranges, the title and a legend (`markerscale=6` enlarges the dot in the legend) follow.

```
save_figure(fig, "invariant_mass_inequality",
            "The invariant mass "
```

```
...
            "of every threshold.")
```

What figure 1 shows. Horizontal axis: the sum of the two masses $\mu_1 + \mu_2$; vertical axis: the invariant mass μ of the pair; both in units of the electron mass m. Every one of the 20000 dots lies on or above the black diagonal, and the dots crowd towards it from above. The student should see the lemma as a picture: a collection of bodies never has less invariant mass than the sum of its masses, which is why every creation process has a threshold.

In [4], one photon can never make a pair.

```
pair_momenta = rng.normal(size=(2, 5000, 3)) * 3.0  # electron and positron
pair_energies = np.sqrt(1.0 + np.sum(pair_momenta ** 2, axis=-1))  # m = 1
pair_mu = invariant_mass(pair_energies, pair_momenta)
report("smallest invariant mass of 5000 electron-positron states",
       f"{pair_mu.min():.4f}")
check(pair_mu.min() >= 2.0 - 1e-12, "every electron-positron state has mu >= 2 m > 0")
```

5000 random electron-positron states: the two momenta are normal random vectors times 3, and the energies are $\sqrt{1 + |\vec{p}|^2}$ because $m = 1$. The smallest invariant mass is printed with four decimals and must be at least $2m$; a single photon has invariant mass 0, so no state of the list can be reached from one photon. Output: RESULT smallest invariant mass of 5000 electron-positron states = 2.0003, and the PASS line.

In [5], a photon on a nucleus: the threshold.

```
E_gamma, M_nuc, m_e = sp.symbols("E_gamma M m", positive=True)
mu_initial_sq = (E_gamma + M_nuc) ** 2 - E_gamma ** 2  # line 1
threshold = sp.solve(sp.Eq(mu_initial_sq, (2 * m_e + M_nuc) ** 2), E_gamma)[0]
say(f"threshold photon energy: {sp.factor(threshold)}")
check(sp.simplify(threshold - 2 * m_e * (1 + m_e / M_nuc)) == 0,
      "E_threshold = 2 m (1 + m/M)")
```

The positive symbols E_γ, M and m. `mu_initial_sq` is line 1 of the proof of Proposition 2, $\mu_A^2 = (E_\gamma + M)^2 - E_\gamma^2$. `sp.Eq(a, b)` is the equation $a = b$; solving $\mu_A^2 = (2m + M)^2$ for E_γ gives the threshold, and `sp.factor` prints it as a product. Output: threshold photon energy: $2m$(M + m)/M. The check compares it with $2m(1 + m/M)$.

```
M4 = 4  # the nucleus of mass 4 m, with m = 1
E_th = threshold.subs({m_e: 1, M_nuc: M4})  # 5/2
p_total, E_total = E_th, E_th + M4  # the initial totals
mu_final = sp.Integer(2 + M4)  # 6 m
shares = [1, 1, M4]  # electron, positron, nucleus masses
p_parts = [sp.Rational(mass) * p_total / mu_final for mass in shares]
E_parts = [sp.sqrt(mass ** 2 + p ** 2) for mass, p in zip(shares, p_parts)]
say(f"final state at threshold: momenta {p_parts}, energies {E_parts}")
check(sum(p_parts) == p_total and sp.simplify(sum(E_parts) - E_total) == 0
      and E_th == sp.Rational(5, 2),
      "M = 4 m: the threshold state conserves energy and momentum exactly")
```

The exact final state at the threshold for $M = 4m$ (with $m = 1$): the threshold $E_\gamma = 5/2$; the initial totals $|\vec{p}_{\text{tot}}| = 5/2$ and $E_{\text{tot}} = 13/2$; the final invariant mass $\mu = 6$; the masses of the electron, the positron and the nucleus; each momentum the share μ_i/μ of the total; each energy $\sqrt{\mu_i^2 + p_i^2}$ (`zip` pairs each mass with its momentum). Output: momenta [5/12, 5/12, 5/3], energies [13/12, 13/12, 13/3]. The check requires the momenta to add up to 5/2 and the energies to 13/2 exactly, and the threshold to be 5/2.

```
for M_value in (4, 1000, 1836):
    value = threshold.subs({m_e: 1, M_nuc: M_value})
    report(f"threshold for M = {M_value} m", f"{sp.nsimplify(value)} = "
           f"{float(value):.6f} m")
check(threshold.subs({m_e: 1, M_nuc: 1836}) == 2 + sp.Rational(2, 1836),
      "M = 1836 m: the threshold is 2 m + 2 m/1836, just above 2 m")
```

The threshold for $M = 4m$, $1000m$ and $1836m$, printed as an exact fraction (`sp.nsimplify`) and with six decimals. Output: $5/2 = 2.500000$ m, $1001/500 = 2.002000$ m, $1837/918 = 2.001089$ m. The last check confirms exactly that for $M = 1836m$ the threshold is $2m + 2m/1836$, just above the two rest energies.

In [6], figure 2: the threshold against the mass of the nucleus.

```
M_axis = np.logspace(-0.5, 4, 400)  # M/m from about 0.3 to 10000
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.semilogx(M_axis, 2 * (1 + 1 / M_axis), color="tab:blue",
            label="threshold $2m(1 + m/M)$")
ax.axhline(2.0, color="black", linestyle=":", label="the two rest energies $2m$")
for M_value, marker in ((4, "o"), (1000, "s"), (1836, "^")):
    ax.plot([M_value], [2 * (1 + 1 / M_value)], marker, color="tab:red",
            markersize=7, label=f"$M = {M_value}m$: {2 * (1 + 1 / M_value):.5f}$m$")
ax.fill_between(M_axis, 2 * (1 + 1 / M_axis), 9.0, color="tab:green", alpha=0.12,
                label="allowed (not forbidden)")
ax.set_ylim(1.5, 9.0)
ax.set_xlabel("mass of the nucleus $M/m$")
ax.set_ylabel("photon energy $E_\\gamma/m$")
ax.set_title("Pair creation on a nucleus: the threshold")
ax.legend(fontsize=8, loc="upper right")
```

400 masses M/m spaced evenly on a logarithmic scale from $10^{-0.5} \approx 0.3$ to 10^4 (`np.logspace`). `ax.semilogx` draws the threshold $2(1 + 1/M)$ with a logarithmic horizontal axis. The dotted line is the two rest energies $2m$; the three red markers are the thresholds for $M = 4m$, $1000m$ and $1836m$, labelled with five decimals; `fill_between` shades the allowed region above the curve in light green. The vertical range, the labels, the title and the legend follow.

```
save_figure(fig, "nucleus_threshold",
            "The smallest photon energy $E_\\gamma$ (vertical, units of the "
...
            "often the process happens above the threshold.")
```

What figure 2 shows. Horizontal axis: the mass of the nucleus M/m (logarithmic); vertical axis: the photon energy E_γ/m. The blue curve is the threshold $2m(1 + m/M)$; above it (green) the

process is allowed by the conservation laws, below it forbidden. A light nucleus needs much more than the two rest energies; for a heavy nucleus the curve approaches the dotted line $2m$. The student should also see what the picture does NOT say: nothing about how often the process happens in the green region.

In [7], figure 3: what the initial state brings and what the final state needs.

```
E_axis = np.linspace(0.0, 6.0, 601)
brought = np.sqrt(M4 ** 2 + 2 * E_axis * M4)  # mu_A for M = 4 m
crossing = E_axis[np.argmin(np.abs(brought - (2 + M4)))]
report("crossing of the two curves (M = 4 m)", f"{crossing:.2f} m")
check(abs(crossing - 2.5) < 0.01, "the curves cross at E_gamma = 2.5 m")
```

601 photon energies from 0 to 6. `brought` is the invariant mass of the initial state, $\sqrt{M^2 + 2E_\gamma M}$ with $M = 4$. `np.argmin(np.abs(brought - 6))` is the position of the photon energy at which `brought` is closest to the needed $2m + M = 6$; `crossing` is that energy. Output: RESULT crossing of the two curves (M = 4 m) = 2.50 m, and the PASS line (to 0.01, the spacing of the grid).

```
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.plot(E_axis, brought, color="tab:blue", label="brought: $\\sqrt{M^2 + 2E_\\gamma M}$")
ax.axhline(2 + M4, color="tab:red", label="needed at least: $2m + M = 6m$")
ax.axvline(2.5, color="black", linestyle=":", label="threshold $E_\\gamma = 5m/2$")
ax.fill_between(E_axis, 2 + M4, brought, where=brought >= 2 + M4,
                color="tab:green", alpha=0.15, label="allowed")
ax.set_xlabel("photon energy $E_\\gamma/m$")
ax.set_ylabel("invariant mass (units $m$)")
ax.set_title("Photon on a nucleus of mass $M = 4m$")
ax.legend(fontsize=8, loc="lower right")
```

The rising blue curve of what the initial state brings, the red horizontal line of what the final state needs at least, the dotted vertical line of the threshold, and the green shading where the first is at or above the second (`where=` restricts the shading). Labels, title and legend follow.

```
save_figure(fig, "allowed_photon_energies",
            "For a photon hitting a nucleus of mass $M = 4m$ at rest: the invariant "
...
            "$E_\\gamma = 5m/2$ (dotted) on.")
```

What figure 3 shows. Horizontal axis: the photon energy E_γ/m; vertical axis: invariant mass in units m; nucleus $M = 4m$. The blue curve (brought) rises with the photon energy and crosses the red line (needed, $6m$) at $E_\gamma = 5m/2$, the dotted threshold. The student should see the threshold as the meeting point of two curves: to the left the process is forbidden, to the right allowed.

In [8], figure 4: two photons.

```
Ea, Eb, theta = sp.symbols("E_a E_b theta", positive=True)
pa = sp.Matrix([Ea, 0, 0])  # photon a along the first axis
pb = sp.Matrix([Eb * sp.cos(theta), Eb * sp.sin(theta), 0])  # photon b at angle
mu_sq = sp.simplify((Ea + Eb) ** 2 - (pa + pb).dot(pa + pb))
```

```
check(sp.simplify(mu_sq - 2 * Ea * Eb * (1 - sp.cos(theta))) == 0,
      "two photons: mu^2 = 2 E_a E_b (1 - cos theta)")
check(sp.simplify(mu_sq.subs(theta, sp.pi) - 4 * Ea * Eb) == 0
      and mu_sq.subs(theta, 0) == 0,
      "head on mu^2 = 4 E_a E_b; same direction mu^2 = 0 (never allowed)")
```

The positive symbols E_a, E_b, θ. Photon a moves along the first axis, $\vec{p}_a = (E_a, 0, 0)$; photon b at the angle θ, $\vec{p}_b = (E_b \cos\theta, E_b \sin\theta, 0)$ (its length is E_b). `mu_sq` is $\mu^2 = (E_a + E_b)^2 - |\vec{p}_a + \vec{p}_b|^2$ (`.dot` is the dot product). The first check confirms $\mu^2 = 2E_aE_b(1 - \cos\theta)$, the proof of Proposition 3; the second the two special cases: head on ($\theta = \pi$, 180 degrees) $\mu^2 = 4E_aE_b$, and in the same direction ($\theta = 0$) $\mu^2 = 0$.

```
grid = np.linspace(0.05, 6.0, 300)
EA, EB = np.meshgrid(grid, grid)
fig, ax = plt.subplots(figsize=(6.4, 5.4))
for degrees, colour in ((180, "tab:blue"), (90, "tab:orange"), (45, "tab:red")):
    bound = 2.0 / (1 - np.cos(np.radians(degrees)))  # E_a E_b >= bound (m = 1)
    ax.contourf(EA, EB, (EA * EB >= bound).astype(float), levels=[0.5, 1.5],
                colors=[colour], alpha=0.12)
    ax.contour(EA, EB, EA * EB, levels=[bound], colors=[colour])
    ax.plot([], [], color=colour,
            label=f"$\\theta = {degrees}$ deg: $E_aE_b \\geq {bound:.2f}m^2$")
ax.set_xlabel("energy of photon a, $E_a/m$")
ax.set_ylabel("energy of photon b, $E_b/m$")
ax.set_title("Two photons: allowed above each curve")
ax.legend(fontsize=8, loc="upper right")
```

A grid of 300 by 300 energy pairs from 0.05 to 6 (`np.meshgrid` makes the two tables `EA` and `EB` of all combinations). For three angles, 180, 90 and 45 degrees (`np.radians` turns degrees into radians), `bound` is $2/(1 - \cos\theta)$, the threshold of E_aE_b with $m = 1$. `contourf` shades, in the colour of the angle, the region where $E_aE_b \geq$ `bound` (the comparison gives `True` or `False`, `.astype(float)` turns it into 1 or 0, and the level band from 0.5 to 1.5 selects the ones); `contour` draws its boundary curve $E_aE_b =$ `bound`. `ax.plot([], [], ...)` draws nothing but adds an entry with the colour and the bound (two decimals) to the legend. Labels, title and legend follow.

```
save_figure(fig, "two_photon_regions",
            "Two photons of energies $E_a$ and $E_b$ (horizontal and vertical, "
...
            "direction can never make a pair.")
```

What figure 4 shows. Horizontal axis: the energy of photon a; vertical axis: the energy of photon b; both in units m. Above each curve the pair is allowed: for head-on photons above $E_aE_b = m^2$ (blue), for 90 degrees above $2m^2$ (orange), for 45 degrees above about $6.83m^2$ (red). The student should see that the smaller the angle between the photons, the more energy is needed, and that photons moving in the same direction can never make a pair.

In [9], what the record says about universes.

```
theory = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                    .read_text(encoding="utf-8"))
```

```
missing = theory["not_established"]  # the record's list
say(f"the record lists {len(missing)} things the pairing theorems do not establish;")
for item in missing[:3]:
    say("- " + item.split(":")[0] + ": " + item.split(":")[1].split(";")[0].strip())
```

The record `pairing-theory.json` is read, and its list `not_established` is taken (`missing`). The first line prints how many items the list has; the loop prints the first three items shortened: `item.split(":")[0]` is the text before the first colon (the item's title), and `item.split(":")[1].split(";")[0].strip()` the text after it up to the first semicolon, without blanks at the ends. Output: the record lists 9 things the pairing theorems do not establish, followed by the three items No creation process, No rate and no amplitude, and No dynamical necessity, each with its first clause.

```
starts = [item.split(":")[0] for item in missing]
check({"No creation process", "No rate and no amplitude",
       "No dynamical necessity"} <= set(starts),
      "the record states: no creation process, no rate or amplitude, no necessity")
```

`starts` is the list of the titles of all nine items. `set(starts)` makes a set of them, and `a <= b` for two sets is true when every element of `a` is in `b`. The check requires the three titles to be in the record's list.

```
u1 = json.loads(repository_file(
    "Revision/lead_checks/reports/charge-conjugation-and-u1.json")
    .read_text(encoding="utf-8"))
verdicts = {entry["name"]: entry["verdict"] for entry in u1["checks"]}
check(verdicts.get("u1_noether_matrix_identity") == "PASS",
      "the record proves the local U(1) conservation law of one universe",
      record="Revision/lead_checks/reports/charge-conjugation-and-u1.json, check "
             "u1_noether_matrix_identity")
```

The lead's report on charge conjugation and the U(1) charge is read, `verdicts` maps each check name to its verdict (`.get(name)` returns `None` if the name is absent), and the check requires `u1_noether_matrix_identity` to have the verdict PASS: the record's proof of the LOCAL conservation law $\partial_\mu(\cos z\, J^\mu) = 0$ of the charge of one universe on shell (the total charge is constant only when, in addition, no charge flows through the brane, Section 20.22 (a)). Output: two PASS lines, the second with the line naming the record.

In [10], the last check.

```
figure_names = ["invariant_mass_inequality", "nucleus_threshold",
                "allowed_photon_energies", "two_photon_regions"]
paths = [output_file(f"{FIGURE_FOLDER}/20c_{k}_{name}.png")
         for k, name in enumerate(figure_names, 1)]
check(all(path.is_file() for path in paths),
      "every figure file of this notebook exists")
all_checks_passed()
```

As in Notebook 20a, In [19]: the four figure files must exist, and `all_checks_passed()` prints the last line. Output: PASS every figure file of this notebook exists, and ALL 12 CHECKS PASSED (notebook 20c).

20.22 What the conservation laws of this theory allow and forbid

Section 20.17 answered question Q1 (allowed?) for light from conservation laws alone. This section does the same for universes, with the conservation laws that the Revision record proves.

When a conservation law forbids. A **conserved quantity** is a number computed from the state of a system that does not change in time for every solution. It forbids a transition from a state A to a state B exactly when its values in A and in B differ. A system may have several conserved quantities, and each forbids on its own: the conservation laws allow a transition only when EVERY conserved quantity has the same value in A and in B. The **empty state** of the classical theory is the configuration in which every field is zero everywhere; all its bilinears, and hence all its conserved quantities, are zero.

(a) The charge of one universe: an exact local law, and a total that is constant only without a flow through the brane. The Lagrangian of Section 20.5 does not change when the field is multiplied by a constant phase, $\Psi \to e^{i\alpha}\Psi$ with a real number α: then $\Psi^\dagger \to e^{-i\alpha}\Psi^\dagger$, so $\bar\Psi \to e^{-i\alpha}\bar\Psi$; every term of $\mathcal{L}$ contains as many factors $\bar\Psi$ as factors Ψ (one of each in K and in mS, two of each in $\frac{\lambda}{2}S^2$; the constant phase comes out of every derivative), so the phases cancel in pairs. Such a symmetry is called a **U(1) symmetry**, and by Noether's theorem it has a conserved current, here $J^\mu = -i\bar\Psi\gamma^\mu\Psi$ (Chapter 21 derives it). The record proves the conservation law directly, for both statistics and in the author's metric: the divergence $\partial_\mu(\sqrt{|g|}\,J^\mu)$ equals, for every configuration, $\sqrt{|g|}$ times a sum of two bilinears, each of which contains the left side E of the field equation or the left side $\bar E$ of its adjoint; so on shell ($E = 0$, $\bar E = 0$)

$$\partial_\mu(\cos z\, J^\mu) = 0.$$

The records of this law:

field	report	check
dirac16complex (Grassmann)	`wolfram-field-theory.json`	`current_conservation_identity_G`
dirac16complex00 (commuting)	`wolfram-field-theory.json`	`current_conservation_identity_C`
dirac16complex (Grassmann)	`python-field-theory.json`	`grassmann_current_conservation`
dirac16complex00 (commuting)	`python-field-theory.json`	`commuting_current_conservation`
both, reduced to a 16×16 matrix identity	`charge-conjugation-and-u1.json`	`u1_noether_matrix_identity`

From it we find WHEN the charge $Q = \int \cos z\, J^{x_4}\, d^7x$ is constant in time, line by line:

1. Write the sum over μ as the x_4 term plus the seven others: $\partial_4(\cos z\, J^{x_4}) = -\sum_{i\neq 4} \partial_i(\cos z\, J^{x_i})$.
2. Integrate both sides over the seven coordinates other than x_4, on a slice $x_4 = \text{const}$; on the left the derivative with respect to x_4 may be taken out of the integral over the other coordinates: $\frac{dQ}{dx_4} = -\sum_{i\neq 4} \int \partial_i(\cos z\, J^{x_i})\, d^7x$.

3. Each term on the right is, by the fundamental theorem of calculus in the variable x_i, the difference of the values of $\cos z\, J^{x_i}$ at the two ends of the range of x_i, integrated over the six remaining coordinates.

4. ASSUMED: these **boundary terms** vanish. This is so, for example, when each of the coordinates $x_1, x_2, x_3, x_5, x_6, x_7$ is periodic, so that its two ends are the same place, and when no current flows through the two ends of the hidden direction. Then $dQ/dx_4 = 0$. At the tip $z = 0$ the flux density $\cos z\, J^{x_0}$ vanishes for the exact solutions of Sections 18.21 and 21.18 (it contains the factor $\sin z$). At the brane $z = \pi/2$ it does NOT vanish in general: for the homogeneous solutions of Section 18.21, $\cos z\, J^{x_0} = \sin z\, q_0$ with the frame current $q_0 = -i\bar{\Psi}\gamma^{(x_0)}\Psi$, which on the solution of Notebook 18c runs from -0.3454 to 0.3454, so the charge of a patch of coordinate volume V_6 in the six directions changes at the rate $dQ/dx_4 = -V_6\, q_0$; Section 21.18 shows another exact solution whose charge changes by what flows out through the brane. The **no-flux condition** at the brane is therefore a real restriction, and no equation of the Revision record derives it (it would be part of the junction condition at the brane, which is OPEN).

Status: the local law is PROVED (records above): no process creates or destroys charge at any point inside the patch; charge can only flow from one place to another. The constancy of the integrated charge holds only under the boundary conditions of line 4, which are ASSUMED; at the brane this no-flux condition is not derived and fails on the exact solutions just cited, so whether the total charge of a universe is conserved is OPEN. Under the no-flux condition no net charge can be generated inside one universe.

(b) Two universes do not interact. A T1 pair consists of two fields: Ψ_+ of the theory (m, λ) and Ψ_- of the theory $(-m, -\lambda)$. Their joint Lagrangian is the sum $\mathcal{L}_{m,\lambda}[\Psi_+] + \mathcal{L}_{-m,-\lambda}[\Psi_-]$: no term of any Lagrangian of the Revision record contains both fields. The field equation of Ψ_+ is obtained by varying Ψ_+ alone; the second term does not contain Ψ_+ and contributes nothing, so Ψ_+ obeys the equation of a single universe whatever Ψ_- does, and the same holds for Ψ_-. Applying (a) to each field separately, under the no-flux condition of (a) (ASSUMED): the charge Q_+ of Ψ_+ and the charge Q_- of Ψ_- are EACH conserved, not only their sum.

(c) What the separate conservation forbids. Line by line:

1. Suppose that at some time $x_4 = t_0$ both fields are zero everywhere (the empty state).

2. Then $J_+^{x_4} = 0$ everywhere at t_0, so $Q_+(t_0) = 0$.

3. By (b), Q_+ is conserved (under the ASSUMED no-flux condition at the brane), so $Q_+(x_4) = 0$ at every time.

4. The members of a T1 pair have opposite charges, $Q_- = -Q_+$ (T1c).

5. Hence a T1 pair whose members carry the charges Q and $-Q$ with $Q \neq 0$ cannot evolve out of zero fields in the theory as built.

The zero TOTAL charge of a T1 pair does not make its appearance allowed, because the separate charges are conserved too (under the no-flux condition). Example: the universe of Section 20.12 has the constant charge density $J^{x_4} = -3$ and $\cos z > 0$ on the patch, so its charge over any region

of the patch is negative, not zero; neither it nor its T1 pair can evolve out of zero fields. (This universe is homogeneous and its J^{x_4} does not change, so by the law $dJ^{x_4}/dx_4 = -6Hq_0$ of Section 18.21 its frame current q_0 is zero: nothing flows through its brane. And for every homogeneous field (f) below gives the same conclusion without any boundary condition.) Status: derived here in five lines from the PROVED local law (a), with the ASSUMED boundary conditions of (a), including the no-flux condition at the brane; for fields that are not homogeneous the argument needs that condition; it is a consequence of the record's identity, not a record of its own (Notebook 20c, section 9, states it in the same words).

(d) The energy of a universe is not conserved in the deflating metric. The metric depends on the time x_4 through a_4, so the energy of one universe is not a conserved quantity. The local law $\nabla_\mu T^\mu{}_\nu = 0$ holds on shell (Chapter 9; `python-field-theory.json`, check `commuting_emt_conservation_on_shell`), and for a homogeneous diagonal source its x_4 component reads $-d\rho/dx_4 - 3a_4'(p_3 - p_t) = 0$ (`emt-divergence-and-spin-connection.json`, check `divergence_x4_component`). Adding $d\rho/dx_4$ to both sides,

$$\frac{d\rho}{dx_4} = -3a_4' \, (p_3 - p_t) :$$

the energy density changes whenever the pressure of 3-space and that of the extra times differ, and energy is exchanged between the inflating 3-space and the deflating extra times. (For the universe of Section 20.12 $p_3 = p_t$, and its ρ is constant.) So the energy is not conserved.

(d') The momenta of a universe: exact local laws, and totals that are constant only without a flow through the brane. The metric does not depend on the six coordinates $x_1, x_2, x_3, x_5, x_6, x_7$: shifting one of them by a constant, $x_i \to x_i + c$, leaves every metric component unchanged. Such a change of coordinates that leaves the metric unchanged is a **symmetry of the metric** (an **isometry**). For each of these six directions the x_i component of the local law gives a conserved **momentum**, line by line:

1. The divergence of a tensor $T^\mu{}_\nu$ is $\nabla_\mu T^\mu{}_\nu = \partial_\mu T^\mu{}_\nu + \Gamma^\mu{}_{\mu\lambda} T^\lambda{}_\nu - \Gamma^\lambda{}_{\mu\nu} T^\mu{}_\lambda$, with the Christoffel symbols $\Gamma^\lambda{}_{\mu\nu} = \frac{1}{2} g^{\lambda\sigma}(\partial_\mu g_{\sigma\nu} + \partial_\nu g_{\sigma\mu} - \partial_\sigma g_{\mu\nu})$ of Chapter 3 (they are not the chirality matrix): the covariant derivative of Chapter 3 adds one Christoffel term for the upper index and subtracts one for the lower index.

2. For the diagonal metric of this book the contracted symbol is $\Gamma^\mu{}_{\mu\lambda} = \frac{1}{2} \sum_\mu \partial_\lambda g_{\mu\mu}/g_{\mu\mu}$ (in the definition with λ and μ contracted, the first and third terms cancel), which is $\partial_\lambda \ln \sqrt{|g|}$, because $|g|$ is the absolute value of the product of the eight $g_{\mu\mu}$ and the logarithm of a product is the sum of the logarithms. So the first two terms of line 1 together are $\frac{1}{\sqrt{|g|}} \partial_\mu (\sqrt{|g|}\, T^\mu{}_\nu)$ (the product rule, read backwards).

3. In the last term, lowering the index gives $\Gamma^\lambda{}_{\mu\nu} T^\mu{}_\lambda = T^{\mu\sigma} \, \frac{1}{2}(\partial_\mu g_{\sigma\nu} + \partial_\nu g_{\sigma\mu} - \partial_\sigma g_{\mu\nu})$. The tensor is symmetric, $T^{\mu\sigma} = T^{\sigma\mu}$ (`python-field-theory.json`, checks `commuting_emt_symmetric` and `grassmann_emt_symmetric`), so exchanging the names μ and σ in the first term shows that it cancels the third; what remains is $\frac{1}{2} T^{\mu\sigma} \partial_\nu g_{\sigma\mu}$.

4. For $\nu = x_i$ with i one of $1, 2, 3, 5, 6, 7$ every $\partial_{x_i} g_{\sigma\mu}$ is zero. On shell the divergence vanishes, $\nabla_\mu T^\mu{}_\nu = 0$, so we are left with $\partial_\mu(\cos z \, T^\mu{}_{x_i}) = 0$, because $\sqrt{|g|} = \cos z$. The records of the on-shell law are the two checks `commuting_emt_conservation_on_shell` and `grassmann_emt_conservation_on_shell` of the report `python-field-theory.json`.

5. This has the form of the charge law of (a) with J^μ replaced by $T^\mu{}_{x_i}$, so the four lines of (a) give that $P_i = \int \cos z \, T^{x_4}{}_{x_i} \, d^7x$ is constant in time, with the same ASSUMED boundary conditions.

The same holds for the rotations of 3-space (in the planes of x_1, x_2, x_3, where the three metric components are equal) and for the rotations of the three extra times among themselves: they too leave the metric unchanged and give local laws of the same form, whose totals are constant under the same ASSUMED boundary conditions. Because the two universes do not couple (b), each of these is conserved SEPARATELY for each universe under these conditions, exactly like the charges; and the T1 partner has $T' = -T$, so its momenta are $-P_i$. Like the charges, the separately conserved momenta therefore forbid a T1 pair whose members have nonzero momenta from appearing out of zero fields (the five lines of (c) with Q replaced by P_i). Status: derived here from the PROVED local law and the PROVED symmetry of T, with the boundary terms ASSUMED to vanish (at the brane this is a no-flux condition like that of (a), not derived). Energy is the one quantity that the deflating metric removes from the list: because the metric depends on x_4, it has no local law of this form. The charges and the momenta (and the rotation quantities) keep exact local laws, and their totals are constant under the ASSUMED no-flux condition at the brane.

(e) The totals of the two kinds of pair. A T1 pair has $T + T' = 0$ and $Q_+ + Q_- = 0$: its totals are those of the empty state, so the conservation of the TOTALS does not forbid its appearance; (c) shows that the separate charges do (under the ASSUMED no-flux condition), unless $Q_+ = 0$. A T2 pair has the total charge $2Q_+$ (the copy carries the SAME charge): under the same condition already its total forbids its appearance from zero fields unless $Q_+ = 0$. (Without that condition the charge of the glued pair of patches changes at twice the rate of one patch: Section 18.21.)

(f) A classical condensate cannot grow out of zero. The zero field solves every field equation of this book, because every term of the field equation contains Ψ. For a condensate the field equation is the ordinary differential equation $\Phi' = -\gamma^{(x_4)}(m + \lambda S - 3H\gamma^{(x_0)})\Phi$ (Section 20.12, step 1), whose right side is a polynomial in the real and imaginary parts of the 16 components that vanishes at $\Phi = 0$. The uniqueness theorem for ordinary differential equations with such a smooth right side (the theorem of Picard and Lindelöf, quoted without proof in Chapter 2) says that near every time only one solution has a given value at that time. The zero function is a solution with the value zero. So a condensate that is zero at one time t_0 is zero on a small interval around t_0. Suppose it were not zero at some later time t_2. Let t^* be the least upper bound of the times s (the smallest number that none of them exceeds) such that the condensate is zero at every time from t_0 to s. Because the solution is continuous, it is zero at t^* itself, so $t^* \neq t_2$ and hence $t^* < t_2$; by the theorem, applied at t^*, it is zero on a small interval beyond t^*, which contradicts the choice of t^*. The same argument runs backwards in time. So the condensate is zero at every time. For the full field equation in 4+4 dimensions, whose initial-value problem is not well posed in the extra-time directions (Chapter 8), no such uniqueness theorem is proved in this book: OPEN.

Summary of Q1 for universes. The totals of a T1 pair are those of the empty state; the separate conservation of each member's charge and momenta, which holds because nothing couples the two universes and under the ASSUMED no-flux condition at the brane, forbids the appearance of a T1 pair with charged members, or with members of nonzero momentum, out of zero fields; under the same condition a T2 pair is forbidden already by its total charge unless it is neutral. Two things could change this, and the record contains neither: an interaction between the two universes, which could leave the totals as the only conserved charges, and a flow through the brane, which the record neither excludes nor describes (both OPEN). And even where the conservation laws allow

something, that is not a creation (Section 20.17, lesson 1).

20.23 Pairs of universes, matter and antimatter

The request for this book also speaks of matter-antimatter mysteries. Chapter 21 teaches that subject from zero; this section states exactly what the pairing theorems contribute to it, and what they do not.

Words. Matter is what stars, planets and we are made of: protons, neutrons and electrons. Their **antiparticles** (antiprotons, antineutrons, positrons) have the same masses and opposite charges and make up **antimatter**. The **baryon number** counts protons and neutrons minus antiprotons and antineutrons. Observations show that the universe we see contains matter and almost no antimatter; Chapter 21 gives the observations and the numbers. To produce such an excess from a state without one, three conditions must hold, as Sakharov stated in 1967 (A. D. Sakharov, JETP Lett. 5, 24 (1967)): (1) some process violates the baryon number; (2) it violates the symmetries C (charge conjugation) and CP (charge conjugation combined with a reflection of space); (3) it happens out of thermal equilibrium.

What is PROVED in this theory.

1. The U(1) charge of one universe obeys an exact LOCAL conservation law (Section 20.22 (a)): no process creates or destroys it at any point inside the patch. Its total is constant only under the no-flux condition at the brane $z = \pi/2$, which is ASSUMED, not proved (OPEN; the homogeneous solutions of Section 18.21 have a patch charge that changes at the rate $-V_6 q_0$); under that condition no net charge of this kind can be generated inside one universe.

2. The pair-level statement: by (T1c) a T1 partner carries the OPPOSITE charge, $J^\mu[\Gamma\Psi] = -J^\mu[\Psi]$, so a T1 pair of universes of masses $+m$ and $-m$ has zero total charge; a T2 mirror copy carries the SAME charge (Section 20.6). Example: the universe of Section 20.12 has $J^{x_4} = -3$, its T1 partner $+3$, its mirror copy -3.

3. Charge conjugation is a MATRIX (rule of this book, from the lead's record `charge-conjugation-and-u1.json`, 12 checks, all PASS; Chapters 5 and 21 derive it step by step). The author's gammas are real, so for a REAL field plain complex conjugation is the identity and is NOT charge conjugation. Solving exactly for every matrix that maps solutions to solutions gives two charge-conjugation matrices: $\mathcal{C}_+ = C$, with $\mathcal{C}_+^{-1}\gamma^{(a)}\mathcal{C}_+ = -(\gamma^{(a)})^T$, which keeps the mass; and $\mathcal{C}_- = \Gamma C$, with $\mathcal{C}_-^{-1}\gamma^{(a)}\mathcal{C}_- = +(\gamma^{(a)})^T$, which reverses the mass (checks `charge_conjugation_matrix_plus` and `charge_conjugation_matrix_minus`). For real commuting fields the current vanishes identically and $\mathcal{C}_+$ acts as the identity; the nontrivial real matrix map is Γ with $(m, \lambda) \to (-m, -\lambda)$, which reverses the current and the kinetic term: theorem T1 (check `real_fields_charge_conjugation`). For the quantised Grassmann field the conjugation that preserves the canonical anticommutator is $\Psi \to \Gamma\Psi^{\dagger T}$, and it reverses the mass (check `quantum_charge_conjugation_unitary_type`).

The universe/anti-universe class of ideas. That our universe might have a partner universe in which the roles of matter and antimatter are exchanged, so that the pair as a whole is neutral, is an old class of ideas. A published example of the class, built differently from anything in this book, is the CPT-symmetric universe of L. Boyle, K. Finn and N. Turok, Phys. Rev. Lett. 121, 251301

(2018); nothing in this chapter is taken from that work, and nothing in this chapter is attributed to it. Item 2 above shows that the T1 pairs of this theory have the bookkeeping of such a pair: opposite charges, zero total. Every scenario that would use this to explain the observed excess of matter in our universe is a HYPOTHESIS, and Section 20.22 (c) shows a first obstacle inside the theory as built: since each universe's charge is conserved on its own under the ASSUMED no-flux condition at the brane, a T1 pair with charged members cannot evolve out of zero fields under that condition; its charges can only be assumed as initial data.

What the theory as built does NOT do. It does NOT solve the matter-antimatter problem. Precisely:

- it contains no baryons (no protons, neutrons or other particles of the Standard Model of particle physics) and no baryon number;
- its only charge is the U(1) charge, which obeys an exact local conservation law, so no process at any point of the patch violates a baryon-like number (Sakharov's condition 1 fails there: PROVED); the total charge of a universe could change only by a flow through the brane $z = \pi/2$, which the ASSUMED no-flux condition excludes and which the record does not describe as a process (OPEN);
- condition 2 (C and CP violation) fails exactly for the commuting field dirac16complex00: its same-mass charge conjugation $\mathcal{C}_+ = C$ is an exact symmetry of its Lagrangian that reverses the charge, so by Proposition 2 of Section 21.5 a C-symmetric start can develop no average charge asymmetry (PROVED; Chapter 21); for the quantised field dirac16complex the only conjugation that keeps the canonical anticommutator, Γ, reverses the mass, and no rates in which C or CP violation could act are computed (NOT COMPUTED);
- no departure from thermal equilibrium of any baryon-number-violating process is computed (condition 3 is not addressed; the deflating history is time-dependent, but no rate is computed in it);
- no asymmetry, no number for it, and no comparison with observation is derived.

What would be needed (OPEN; every item a task, not a result): a charge that can play the role of the baryon number; an interaction that violates it; C and CP violation in that interaction; a departure from equilibrium along a computed history; a calculation of the resulting asymmetry and its comparison with observation; and, for the pair scenario, either an interaction between the universes that leaves only the total charge conserved or a theory of the initial data. Chapter 21 develops the conditions and the scorecard in detail.

20.24 What is not established

The following are NOT established by any equation of this book or of the Revision record. The items 1 to 9 are the record's own list: the nine items of `pairing-theory.json`, key `not_established`, in their order. The list of `PAIR_CREATION_PROOFS.md`, section 11.2, has ten items: its items 1 to 8 are items 1 to 8 here, its item 10 is item 9 here, and its item 9 adds that dirac16complex00 is a classical field, for which no quantum statement is made (the quantum reading Q of Section 20.6 is for dirac16complex only). The sympy checker of the record keeps a third list, which states the points

of items 1 to 9 here in twelve sentences (`python-pairing.json`, key `not_established`; Notebook 00c prints it). The items 10 to 13 are added by this chapter.

1. **No creation process.** The theorems map solutions to solutions and quantities to quantities; nothing produces a universe, a pair of universes or a change of the number of universes; no initial state, vacuum decay or tunnelling process is derived.

2. **No rate and no amplitude.** No transition amplitude, probability, cross section, rate or Bogoliubov coefficient for creating universes of masses $\{+m, -m\}$ is computed or implied; no wave function of the universe and no path integral is part of the theorems.

3. **No dynamical necessity.** Nothing forces the partner to exist or to be realised; a single universe with mass $+m$ is an equally valid solution without its partner (Section 20.12 builds one).

4. **T1 is not a symmetry of one theory.** It changes the parameters and the sign of the action; for $\lambda \neq 0$ it pairs (m, λ) with $(-m, -\lambda)$, not $+m$ with $-m$ at the same coupling. The pure pairing $(m, \lambda) \to (-m, \lambda)$ is T2, at EQUAL energy-momentum.

5. **The zero total holds for classical bilinears only.** The vanishing total energy-momentum and charge of a T1 pair holds for classical bilinears and as an operator identity within ONE quantum system; two independently quantised universes have generators that add without cancelling (statement Q3).

6. **Test fields.** The gravitational field is fixed and the same for both members; corollary C1 is the only statement about back-reaction, it concerns the sum of two classical sources in one geometry, and its part (a) depends on Einstein gravity (Einstein-Gauss-Bonnet gravity admits the deflating history as a vacuum, Section 20.7).

7. **The Z2 brane.** The mirror across $z = \pi/2$ uses the ASSUMED Z2 construction; the metric is degenerate there; no junction condition, brane tension or matching of the field across the brane is derived.

8. **Quantum positivity.** A positive-norm Fock space for either universe is not established by the pairing theorems.

9. **The Kohn-Sham level.** T3 holds for instantaneous mean-field Kohn-Sham states with the ASSUMED brane and the transformed tip condition; the time-dependent problem is OPEN, correlation is not included, and the Kohn-Sham history $a_4 = AHx_4$ is a PRESCRIBED BACKGROUND (the Kohn-Sham states violate the source conditions of the a_4 equations: `ks-source-conditions.json`, check `ks_profiles_violate_algebraic_condition`).

10. **No big bang in the equations.** For the PRESCRIBED deflating history $a_4 = AHx_4$ (and for any history whose a_4, a_4' and a_4'' stay finite) the author's metric and its curvature are finite at every finite time on the patch (Section 20.3), and T1 holds at every time, so no moment is singled out. The history is prescribed, not computed; whether the coupled equations of the fields and of a_4 contain a big-bang moment is not decided (OPEN), and with a given source the a_4 equations of Einstein-Gauss-Bonnet gravity can break down at a finite time (Chapter 12; Section 20.3).

11. **No value of the mass.** m, λ, H and κ are free parameters; no relation between H and m is derived.

12. **No solution of the matter-antimatter problem** (Section 20.23).

13. **No uniqueness for the full 4+4 field equation.** Whether a classical field that vanishes at one time vanishes at all times is proved here only for condensates (Section 20.22 (f)).

20.25 What a calculation of creation would require

Questions Q2 and Q3 can be attacked in two ways. Both are OPEN; this section lists what each would need, so that a student can see where the missing work lies (Chapter 22 collects the open problems).

The classical route (Q2). One would need a solution of the coupled system, the field equations of both members together with the Einstein-Lovelock equations whose source is the total energy-momentum tensor, so that the metric is COMPUTED rather than prescribed, which starts without the pair and ends with it. Section 20.22 shows the obstacles: under the ASSUMED no-flux condition at the brane $z = \pi/2$ (not derived; it fails on the homogeneous solutions of Section 18.21) the separate conservation of the two charges forbids a charged pair from zero fields, and a condensate cannot grow out of zero at all (this needs no boundary condition). So a classical solution can at most describe how a pair that is already present evolves: the pair would have to be in the initial data (and then it is not created), or the solution would have to start at a singular moment with a rule for what emerges from it (and such a rule is an initial condition, not a derivation), or the equations would have to be changed.

The quantum route (Q3). By analogy with Section 20.17, one level up:

1. **A space of states with different numbers of universes**, containing a state with no universe and states with one universe or with a pair. In quantum cosmology the corresponding object is a wave function of the geometry together with the matter fields, which obeys the Wheeler-DeWitt equation (B. S. DeWitt, Phys. Rev. 160, 1113 (1967)); proposals of this kind include those of Hartle and Hawking (J. B. Hartle and S. W. Hawking, Phys. Rev. D 28, 2960 (1983)) and of Vilenkin (A. Vilenkin, Phys. Lett. B 117, 25 (1982)). Nothing of this kind exists in this book, which quantises only the matter field in a prescribed geometry (Chapter 10).

2. **A probability interpretation.** Probabilities need a positive inner product. The canonical state space of dirac16complex is a Krein space with an indefinite inner product, and a positive Fock space is known only in the good sector without extra-time momentum (Chapter 10).

3. **A dynamics that connects the states:** an interaction, or a time-dependent background, whose matrix element between the empty state and the pair state is not zero. Nothing in the theory couples two universes. For a pair whose members carry the charges $\pm Q$ with $Q \neq 0$, the interaction would also have to break the separate conservation of the two charges while keeping their sum (Section 20.22; a flow through the brane, which the record does not describe, is the other open possibility).

4. **The amplitude and what follows from it:** the amplitude $\langle \text{pair}|\, U\, |\text{empty}\rangle$ of the evolution operator U, the probability $|\langle \text{pair}|U|\text{empty}\rangle|^2$, and a rate.

5. **The stability question.** A theory with quanta of negative energy that interact with quanta of positive energy can make the empty state unstable, because pairs of zero total energy can

then be produced without limit (J. M. Cline, S. Jeon and G. D. Moore, Phys. Rev. D 70, 043543 (2004)). A calculation of the creation of T1 pairs, whose members have opposite energies, would have to face this question.

6. **Numbers.** The mass m, the constant H, the gravitational coupling κ and the coupling λ would have to be fixed; the theory as built fixes none of them.

That a universe of zero total energy might appear without violating energy conservation was proposed long ago (E. P. Tryon, Nature 246, 396 (1973)); a zero total has never, by itself, produced a rate: that is always a separate calculation.

The photon analogy, completed.

ingredient	**photon on a nucleus**	**a T1 pair of universes**
the totals allow it?	yes, above the threshold (Proposition 2)	yes: all totals of the pair are zero
an interaction that lets energy or charge pass	light couples to the electron field	none: each universe's charge obeys its own local law and, under the ASSUMED no-flux condition at the brane, is conserved on its own, which forbids a charged pair from zero fields
dynamics connecting the initial and the final state	quantum electrodynamics	none in the theory
a computed probability	the Bethe-Heitler cross section (quoted)	none (OPEN)

The first line is where the pairing theorems stand. The last three lines are the missing steps, and the second shows that a coupling between universes would be needed before any rate could be computed.

20.26 The answer to the question of the title

Do universes come in pairs? For each of the two fields, dirac16complex and dirac16complex00, the equations of this book PROVE: to every solution with mass m the explicit map Γ assigns a solution with mass $-m$ and coupling $-\lambda$, with the opposite energy-momentum tensor and charge, in every gravitational field (T1); the map $\gamma^{(x_0)}$ with the ASSUMED Z2 mirror assigns a solution with mass $-m$ and the same coupling, with the SAME energy-momentum and charge, in the author's primordial field (T2); for dirac16complex at the Kohn-Sham level the block map of Γ with the transformed boundary conditions assigns to every self-consistent instantaneous state with (m, λ) one with $(-m, +\lambda)$ at equal energies (T3); at the quantum level the T1 partner is the same quantum system relabelled, and two independently quantised universes have identical one-particle spectra (the $\lambda = 0$ spectra, in flat 4+4 space or in a general field at a point with frozen coefficients) and energies that add (Q); and a T1 pair as the only source of the author's metric is a zero source, which in Einstein gravity admits no solution for $H > 0$ (C1). These are exact maps between solution sets, with their stated hypotheses.

The equations do NOT prove that universes come in pairs: a single universe with mass $+m$ is an equally valid solution without its partner, and Section 20.12 builds one, exactly. They do NOT prove that any universe is created, in pairs or otherwise: they contain no creation process, no rate, no probability and no amplitude, and in the theory as built, under the ASSUMED no-flux condition at the brane $z = \pi/2$ (not derived; it fails on the homogeneous solutions of Section 18.21), the separate conservation of each universe's charge forbids a charged T1 pair from appearing out of zero fields; without that condition this is OPEN for fields that are not homogeneous (Section 20.22 (c) and (f)). They do NOT solve the matter-antimatter problem.

Hypothesis P ("the big bang creates universes of masses $+M$ and $-M$ in pairs") therefore remains what it was: a HYPOTHESIS. Sections 20.22 to 20.25 say precisely what a proof of it would still need.

20.27 What we proved, what we computed, what we assumed

statement	status	where verified
T1: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$; solutions to solutions; $T \to -T$, $J \to -J$; the pair has zero total source and charge (classical bilinears; every gravitational field; both fields)	PROVED	`wolfram-pairing.json` (51 T1 checks), `python-pairing.json` (23 T1 checks); Section 20.5; Notebook 20a, In [7] and In [9]; Notebook 20b, In [7]
T2: $\gamma^{(n)}$ with a frame reflection of character -1: $(m, \lambda) \to (-m, \lambda)$, $\mathcal{L} \to +\mathcal{L}$; in the author's field the mirror copy has the pulled-back, EQUAL tensor and charge	PROVED; the Z2 construction ASSUMED	`wolfram-pairing.json` (28 T2 checks), `python-pairing.json` (16 T2 checks); Section 20.6; Notebook 20a, In [9] and In [11]; Notebook 20b, In [9]
Q: the T1 image carries $-B$ and is the same quantum system; independent universes do not cancel; equal one-particle spectra ($\lambda = 0$; flat 4+4 space, or a general field at a point with frozen coefficients); the T2 image keeps $+B$	PROVED	`wolfram-pairing.json` (12 Q checks), `python-pairing.json` (10 Q checks); Section 20.6
T3: Kohn-Sham states with (m, λ, θ) and $(-m, +\lambda, \pi - \theta)$ have equal levels, energies and energy-momentum profiles	PROVED; brane ASSUMED, tip transformed, instantaneous mean-field states	`wolfram-t3.json` (10 checks), `python-t3.json` (13 checks); Section 20.6; Chapter 19

statement	status	where verified
C1(a): a T1 pair as the only source; Einstein gravity has no real solution for $H > 0$; the only solutions are $a_4' = \pm iH$, $\Lambda = -18H^2$	PROVED	`wolfram-a4-report.json`, check `einstein_no_vacuum_solution`; `python-a4-report.json`, check `einstein_no_vacuum`; `einstein-gauss-bonnet-a4.json`, check `no_vacuum_for_H_positive`; Section 20.7; Notebook 20a, In [12]
every source of the author's metric needs $\kappa(\rho + p_0) = -6(a_4'^2 + H^2)$ in Einstein gravity	PROVED	both a4 reports, check `einstein_null_energy_x0`; Notebook 20a, In [14]
C1(b): the linear member is a vacuum exactly when $V = 0$; with $\alpha_2 H^2 = 1/48$ and $\Lambda = -12H^2$ the deflating history is a Gauss-Bonnet vacuum	PROVED	both a4 reports, checks `linear_member_vacuum_factor` and `einstein_gauss_bonnet_vacuum_linear`; exact check in Notebook 20a, In [16]
$\sum_\mu \gamma^\mu \Omega_\mu = -3H\gamma^{(x_0)}$ on the mirror patch	COMPUTED exactly (a result of this book, not a record)	Notebook 20a, In [5]; Notebook 20b, In [3]
the author's metric and its curvature are finite and non-degenerate at every finite time on the patch for the prescribed history $a_4 = AHx_4$ (and any history with finite a_4, a_4', a_4''): no big-bang moment in the prescribed history	PROVED for the prescribed history; for computed histories OPEN (with a given source the a_4 equations can break down at a finite time, Chapter 12)	Section 20.3, from `python-pairing.json`, check `geometry.brane_degenerate`, and `python-a4-report.json`, check `riemann_entries_laurent`
the universe $m = 15$, $\lambda = -25/3$, $A = 1$, $\Lambda = -36H^2$: $S = 12/5$, $\rho = 12$, $p = -24$, $J^{x_4} = -3$, frequency $4H$, a complete solution without a partner	COMPUTED (exact for the field equation; Einstein equations to 10^{-10} at nine points); a result of this book	Section 20.12; Notebook 20b, In [4] to In [6]
its T1 partner solves its own equation with the same frequency but is the source of no member of the author's family in Einstein gravity	PROVED here by hand (record's null combination); checked numerically	Section 20.12, steps 6 and 7; Notebook 20b, In [7]
its T2 mirror copy solves the coupled equations on the mirror patch with equal ρ and J^{x_4}	PROVED here by hand; checked	Section 20.12, step 9; Notebook 20b, In [9]

statement	status	where verified
the charge of one universe obeys an exact local law; its total is constant under the no-flux condition at the brane; then a T1 pair with charged members cannot evolve out of zero fields	the local law PROVED; the no-flux condition ASSUMED (not derived; it fails on the homogeneous solutions of Section 18.21, OPEN); the consequence derived here	`charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity`; both field-theory reports; Section 20.22
$d\rho/dx_4 = -3a_4'(p_3 - p_t)$: the energy of one universe is not conserved	PROVED	`emt-divergence-and-spin-connection.json`, check `divergence_x4_component`; Section 20.22 (d)
the momenta $P_i = \int \cos z\, T^{x_4}{}_{x_i}\, d^7x$ $(i = 1, 2, 3, 5, 6, 7)$ of each universe are conserved separately; a T1 pair whose members have nonzero momenta cannot evolve out of zero fields	derived here from the PROVED local law and the PROVED symmetry of T; boundary terms ASSUMED to vanish	`python-field-theory.json`, checks `commuting_emt_conservation_on_shell`, `grassmann_emt_conservation_on_shell`, `commuting_emt_symmetric` and `grassmann_emt_symmetric`; Section 20.22 (d')
a condensate that vanishes at one time vanishes always	PROVED here with the quoted uniqueness theorem for ODEs	Section 20.22 (f)
the threshold lemma; one photon makes no pair; $E_\gamma \geq 2m(1 + m/M)$; $E_a E_b \geq 2m^2/(1 - \cos\theta)$	PROVED here from ASSUMED special relativity	Section 20.17; Notebook 20c, In [2] to In [8]
the rates of pair creation by light	quoted from the literature (Bethe-Heitler, Breit-Wheeler), not computed	Section 20.17
a T1 pair has zero total charge; a T2 copy has the same charge	PROVED	(T1c) in Section 20.5 and the statement of T2 in Section 20.6 ($J' = R_0 J$)
the two charge-conjugation matrices $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$; for real fields the nontrivial real map is Γ (T1)	PROVED	`charge-conjugation-and-u1.json` (12 checks); Chapters 5 and 21
a partner universe explains the matter excess of our universe	HYPOTHESIS	Section 20.23
the theory solves the matter-antimatter problem	NOT ESTABLISHED: no baryons, no baryon-number violation, no CP violation, no departure from equilibrium computed	Section 20.23
a creation process, rate, probability or amplitude for universes	NOT ESTABLISHED (OPEN)	`pairing-theory.json`, key `not_established`; Sections 20.24 and 20.25
Hypothesis P: "the big bang creates universes of masses $+M$ and $-M$ in pairs"	HYPOTHESIS	Sections 20.3 and 20.26

statement	status	where verified
C1(a) for every second function $b_4(x_4)$ of the metric of 2026-10-14 (a zero source forces b_4 constant); for the author's Einstein-Lovelock weights every vacuum has b_4 constant and $a_4 = \pm Hx_4 + \text{const}$	PROVED	`wolfram-a4-report.json`, checks `b4_einstein_offdiagonal_forces_b4_constant` and `b4_author_weights_vacuum`; `Revision/metric_b4/results/vacuum-b4.json`; Section 20.7
Einstein gravity in the example; the background $b_4 = 1$ of the pairing records and of the notebooks; the sign convention $\sigma_T = +1$; the classical field dirac16complex00 in the examples; the units $H = \kappa = 1$; the Z2 brane and the mirror patch	ASSUMED	Sections 20.12 and 20.6
the junction at the brane; the stability of the condensate; uniqueness for the full 4+4 field equation; a coupling between universes; whether the coupled equations of the fields and of a_4 contain a big-bang moment	OPEN	Sections 20.3, 20.12, 20.22 and 20.25

20.28 Exercises

Exercise 20.1 (the chirality commutes with the generators). Show, using only step 1 of Section 20.5 ($\Gamma\gamma^{(b)} = -\gamma^{(b)}\Gamma$ for every b), that $\Gamma S^{ab} = S^{ab}\Gamma$ for $a \neq b$, where $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$.

Answer. Take the first product: $\Gamma\gamma^{(a)}\gamma^{(b)} = -\gamma^{(a)}\Gamma\gamma^{(b)}$ (step 1 for the direction a) $= +\gamma^{(a)}\gamma^{(b)}\Gamma$ (step 1 for the direction b). In the same way $\Gamma\gamma^{(b)}\gamma^{(a)} = \gamma^{(b)}\gamma^{(a)}\Gamma$. Subtracting the second line from the first and multiplying by $\frac{1}{4}$: $\Gamma S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})\Gamma = S^{ab}\Gamma$. The two signs $(-1)(-1) = +1$ are the reason: S^{ab} contains two gammas.

Exercise 20.2 (a time-like reflection gives a T1-type map). In Section 20.6 the reflection of a space-like direction n gave $S' = -\eta_{nn}S$ and $K' = \eta_{nn}K$. Take the time direction $n = x_4$ ($\eta_{nn} = -1$) and compute $\mathcal{L}_{m,\lambda}[\gamma^{(x_4)}\Psi; R_4 e]$ in terms of $\mathcal{L}$ of Ψ.

Answer. With $\eta_{nn} = -1$: $S' = -(-1)S = S$ and $K' = (-1)K = -K$. Insert into the Lagrangian: $\mathcal{L}_{m,\lambda}[\Psi'; e'] = \sqrt{|g|}\,[K' - mS' - \frac{\lambda}{2}S'^2] = \sqrt{|g|}\,[-K - mS - \frac{\lambda}{2}S^2]$. Take out -1 and write $m = -(-m)$, $\lambda = -(-\lambda)$: $= -\sqrt{|g|}\,[K - (-m)S - \frac{(-\lambda)}{2}S^2] = -\mathcal{L}_{-m,-\lambda}[\Psi; e]$. This is a T1-type map, $(m, \lambda) \to (-m, -\lambda)$ with $\mathcal{L} \to -\mathcal{L}$, as the table of the eight reflections in `PAIR_CREATION_PROOFS.md` (section 5.5) records for x_4, x_5, x_6, x_7.

Exercise 20.3 (another universe and its partner). Repeat steps 4 and 7 of Section 20.12 with

$A = 1$, $\Lambda = -30H^2$ and $M = -5H$ (units $H = \kappa = 1$). Find S, m, λ, ρ, p and the equation of state p/ρ, and decide whether the T1 partner can be the source of the author's metric.

Answer. First condition: $MS = -12$, so $S = 12/5$ (as before; it does not involve Λ). Second: $mS = -(36 + 2(-30)) = -(36 - 60) = 24$, so $m = 24 \cdot 5/12 = 10$. Then $\lambda = (M - m)/S = (-5 - 10) \cdot 5/12 = -75/12 = -25/4$. $\rho = mS + \frac{\lambda}{2}S^2 = 24 + (-\frac{25}{8}) \cdot \frac{144}{25} = 24 - 18 = 6$, and $p = \frac{\lambda}{2}S^2 = -18$, so $p/\rho = -3$. (These are the numbers of the third example of Chapter 12.) Check: $\rho + p = -12 = MS$ and $\rho - p = 24 = mS$. The partner has $\rho' = -6$, $p' = 18$, so $\kappa(\rho' + p'_0) = -6 + 18 = +12 > -6$: it meets the requirement $-6(a_4'^2 + 1)$ at no real rate, and it cannot be the source of any member of the author's family in Einstein gravity. (The partner's null combination is minus the universe's, $-(\rho + p) = -MS = +12$, whatever Λ is: the result does not depend on the choice of Λ.)

Exercise 20.4 (a Gauss-Bonnet vacuum with a faster deflation). In Einstein-Gauss-Bonnet gravity ($\alpha_1 = 1$, $\alpha_3 = 0$), find the coupling $\alpha_2 H^2$ for which the linear member with $A = 2$ (3-space grows like e^{2Hx_4}, the extra times shrink like e^{-2Hx_4}) is a vacuum, and the cosmological constant it needs.

Answer. From Section 20.7, $V = 1 - 8\alpha_2 H^2(A^2 + 5) = 1 - 8\alpha_2 H^2 \cdot 9 = 1 - 72\alpha_2 H^2$, which vanishes for $\alpha_2 H^2 = 1/72$; this lies in the allowed range $0 < \alpha_2 H^2 \le 1/40$. The cosmological constant is $\Lambda = -(E_{(1)}{}^{x_4}{}_{x_4} + \alpha_2 E_{(2)}{}^{x_4}{}_{x_4})$. At $a'_4 = 2H$: $E_{(1)}{}^{x_4}{}_{x_4} = 3a_4'^2 + 21H^2 = 12H^2 + 21H^2 = 33H^2$, and $E_{(2)}{}^{x_4}{}_{x_4} = -(36A^4 + 120A^2 + 420)H^4 = -(576 + 480 + 420)H^4 = -1476H^4$ (Section 20.7). So $\Lambda = -(33H^2 - 1476H^2/72) = -(33 - 20.5)H^2 = -12.5H^2 = -\frac{25}{2}H^2$. Check with the curve printed by Notebook 20a, In [16], $\Lambda = 720\alpha_2 H^4 - 36H^2 + 3/(16\alpha_2)$: with $\alpha_2 = 1/(72H^2)$ this is $10H^2 - 36H^2 + 13.5H^2 = -12.5H^2$.

Exercise 20.5 (a threshold with an exact final state). A photon hits a nucleus of mass $M = 2m$ at rest and makes an electron-positron pair. Find the threshold energy and the momenta and energies of the three final bodies at the threshold, and check that the totals are conserved.

Answer. $E_\gamma = 2m(1 + m/M) = 2m(1 + 1/2) = 3m$. The initial totals: $|\vec{p}_{\text{tot}}| = 3m$, $E_{\text{tot}} = 3m + 2m = 5m$, so $\mu^2 = 25m^2 - 9m^2 = 16m^2$ and $\mu = 4m = m + m + 2m$, the sum of the final masses, as at a threshold. Each body carries the share μ_i/μ of the momentum: the electron and the positron $(1/4)(3m) = 3m/4$ each, with the energy $\sqrt{m^2 + 9m^2/16} = 5m/4$ each; the nucleus $(2/4)(3m) = 3m/2$, with the energy $\sqrt{4m^2 + 9m^2/4} = 5m/2$. Totals: momenta $3m/4 + 3m/4 + 3m/2 = 3m$, energies $5m/4 + 5m/4 + 5m/2 = 5m$, the initial ones. All three move with the velocity $(3m/4)/(5m/4) = (3m/2)/(5m/2) = 3/5$, the equality case of the lemma.

Exercise 20.6 (two photons at a right angle). One photon has the energy $E_a = 4m$. What is the smallest energy of a second photon that meets it at 90 degrees and can make an electron-positron pair with it? And if the second photon moves head on?

Answer. At $\theta = 90$ degrees, $\cos\theta = 0$, and Proposition 3 needs $E_a E_b \ge 2m^2$, so $E_b \ge 2m^2/(4m) = m/2$. Head on, $\cos\theta = -1$, and the condition is $E_a E_b \ge 2m^2/2 = m^2$, so $E_b \ge m/4$.

Exercise 20.7 (the charge density of the partner). Using $J^\mu = -i\bar{\Psi}\gamma^\mu\Psi$, $\bar{\Psi} = \Psi^\dagger C$ and $\gamma^{x_4} = \gamma^{(x_4)}$, show that $J^{x_4} = \Psi^\dagger B \Psi$ with $B = -iC\gamma^{(x_4)}$, and that $J^{x_4}[\Gamma\Psi] = -J^{x_4}[\Psi]$. Then explain in two sentences why the zero total charge of a T1 pair does not allow it to appear out of zero fields.

Answer. $J^{x_4} = -i\Psi^\dagger C\gamma^{(x_4)}\Psi = \Psi^\dagger(-iC\gamma^{(x_4)})\Psi = \Psi^\dagger B\Psi$, by inserting the definitions and moving the number $-i$ next to the matrices. For the partner, $(\Gamma\Psi)^\dagger B(\Gamma\Psi) = \Psi^\dagger \Gamma B \Gamma \Psi$ because $\Gamma^\dagger = \Gamma$,

and $\Gamma B\Gamma = -i\,\Gamma C\gamma^{(x_4)}\Gamma = -i\,C\,\Gamma\gamma^{(x_4)}\Gamma = -i\,C(-\gamma^{(x_4)}) = -B$ (step 2, $\Gamma C = C\Gamma$, then step 1 of Section 20.5); so $J^{x_4}[\Gamma\Psi] = -\Psi^\dagger B\Psi = -J^{x_4}[\Psi]$. The two universes of the pair are two different fields with no coupling, so under the ASSUMED no-flux condition at the brane the charge of each is conserved on its own (Section 20.22 (b)); starting from zero fields each charge then stays zero, so a pair whose members carry the charges Q and $-Q$ with $Q \neq 0$ can never be reached, although the total is zero.

21 Matter and antimatter from zero: what this theory explains and what it does not

The world we see is made of matter, and almost no antimatter. Why that is so is one of the open questions of physics. This chapter explains the question from zero, with the observations, the three conditions that Andrei Sakharov found for any answer, and small worked models of each condition. Then it asks, condition by condition, what the theory of this book says. On the way it derives, line by line, the facts of the theory that matter for the question: that charge conjugation in this theory is made by a **matrix** (with the intertwiner condition of the Revision record as the definition, there are exactly two such matrices), that the charge of the field obeys an **exact local conservation law** in the author's metric while the three extra times deflate (its total is constant only if no charge flows through the edge $z = \pi/2$ of the hidden direction, a condition that is assumed, not derived), how the **quantised** field can be conjugated, and what a pair of solutions of masses $+m$ and $-m$ carries. The chapter ends with a scorecard and with a precise list of what the theory as built does not do.

21.1 What this chapter does, and the answer first

The request that this book answers ends with the words "this theory solves matter anti-matter mysteries". The honest answer comes first, because every later section supports it.

The answer. The theory as built does **not** solve the matter-antimatter problem. The reasons, each of which this chapter proves or documents:

- The theory contains no baryons (no protons, neutrons or quarks), so it cannot say anything about the baryon number of the observed universe.
- The only number of the theory that could play the role of "matter minus antimatter" is the **U(1) charge** Q of its field. It obeys an **exact local conservation law**: in the author's metric, for every history $a_4(x_4)$, in particular the one in which ordinary space inflates and the three extra times x_5, x_6, x_7 deflate exponentially, the charge of a region changes only by what flows through its boundary (PROVED, Sections 21.16 and 21.17). So no process described by these equations makes a net charge at any point inside the patch $0 < z < \pi/2$ of one universe, that is, off the brane: Sakharov's first condition fails there. The total charge of a universe is constant only if no charge flows through the brane $z = \pi/2$, the edge of the hidden direction; this **no-flux condition** is ASSUMED, not derived (OPEN), and Section 21.18 shows an exact solution whose charge changes by exactly what flows out through the brane.
- For the commuting field dirac16complex00 the same-mass charge conjugation is an exact symmetry that reverses the charge (PROVED, Sections 21.10 and 21.31), so Sakharov's second

condition fails for it as well (PROVED, Section 21.31). For the quantised field dirac16complex no violation of C or CP in reaction rates is built into the theory or computed (NOT COMPUTED). No departure from thermal equilibrium is computed (NOT COMPUTED); the Kohn-Sham history of this book is a prescribed background (Section 21.31).

What the theory does provide, exactly.

- Charge conjugation is a matrix. The author's gamma matrices are real, so for a real field plain complex conjugation changes nothing at all. Solving exactly for every matrix that obeys the intertwiner condition $M(\gamma^a)^* = s\,\gamma^a M$ (the record's definition, Section 21.8; other matrices, e.g. $P_- C$ for $V = 0$, can also map solutions to solutions) gives exactly two, up to a factor: $\mathcal{C}_+ = C$, which keeps the mass, and $\mathcal{C}_- = \Gamma C$, which reverses it (PROVED, Sections 21.8 to 21.10; Notebook 21a).
- For a real field the current vanishes, $\mathcal{C}_+$ acts as the identity, and, among the matrices that obey this condition, the only nontrivial real matrix map is Γ, which reverses the mass (PROVED, Section 21.10).
- For the quantised anticommuting field dirac16complex the only charge-conjugation matrix that keeps the canonical anticommutator gives $\Psi \to \Gamma\Psi^{\dagger T}$, and it reverses the mass (PROVED, Sections 21.25 and 21.26; Notebook 21c).
- **Pair level.** The chirality partner $\Gamma\Psi$ of a solution with the parameters (m, λ) is a solution with $(-m, -\lambda)$ and carries exactly the opposite current, so the pair has total charge zero as classical bilinears (PROVED, theorem T1 of the Revision record; Section 21.19; Notebook 21b). This is a **map between two sets of solutions**. It does not create anything: a single universe of mass $+m$ is an equally valid solution without its partner, and no creation process, rate or amplitude follows from these equations (Chapter 20). The idea that a universe and an anti-universe together carry zero charge belongs to a class of ideas in the literature; any scenario in which our universe is one member of such a pair is a HYPOTHESIS (Section 21.37).

The plan. Sections 21.3 to 21.6 explain the problem from zero: particles and antiparticles, the conserved numbers, the observed excess of matter, and Sakharov's three conditions, each derived in plain words with a small model. Section 21.7 sets out the theory of this book. Sections 21.8 to 21.15 derive the charge-conjugation matrices and place Notebook 21a, which solves for them exactly from the Revision gammas. Sections 21.16 to 21.23 derive the exact local conservation law of the charge, the charge balance through the brane and the pair-level bookkeeping, with Notebook 21b. Sections 21.24 to 21.30 treat the reflections, the chirality and the quantised field, with Notebook 21c. Sections 21.31 to 21.36 apply Sakharov's conditions to the theory and place Notebook 21d, which computes the toy models and draws the scorecard. Section 21.37 says what would be needed; Section 21.38 lists every statement with its status; Sections 21.39 and 21.40 hold the exercises and their answers.

Labels. As everywhere in this book: PROVED means exact, with the Revision verifier file and check name, or with a complete derivation given here and an exact check in a notebook; COMPUTED means numerical, with the cell that computes it and its accuracy; ASSUMED marks an input, a convention or a quoted fact of physics; HYPOTHESIS marks a claim that nothing in the Revision record derives; OPEN marks a question that has not been answered; NOT COMPUTED marks something that no computation of the record addresses.

21.2 The words and symbols of this chapter

Each word is defined in plain terms here; the later sections make the definitions precise with formulas.

- **Matter, antimatter.** In cosmology "matter" means the protons and neutrons of atoms, which carry almost all of their mass. **Antimatter** is made of antiparticles.
- **Antiparticle.** For every known particle there is a particle with the same mass and the opposite values of all its charges: the positron e^+ for the electron e^-, the antiproton $\bar{p}$ for the proton p. A bar over a letter marks an antiparticle.
- **Quark.** A constituent of protons and neutrons. The up quark u has electric charge $+2/3$ and the down quark d has $-1/3$, in units of the proton charge.
- **Baryon, baryon number $\mathcal{B}$.** A baryon is made of three quarks, for example the proton $p = uud$ and the neutron $n = udd$. Every quark has $\mathcal{B} = 1/3$ and every antiquark $-1/3$. (Notebook 21d writes the baryon number with the plain letter B; in this chapter's prose the calligraphic $\mathcal{B}$ is used, because the plain B is the Krein matrix below.)
- **Meson.** A quark and an antiquark, such as the pions $\pi^+ = u\bar{d}$, $\pi^- = d\bar{u}$ and π^0; $\mathcal{B} = 0$.
- **Lepton number** L: $+1$ for the electron and its neutrino ν_e, -1 for the positron and the antineutrino $\bar{\nu}_e$, 0 for quarks and photons. It is not the Lagrangian $\mathcal{L}$.
- **Electric charge** Q_{el}, in units of the proton charge. (Notebook 21d prints it as Q in its bookkeeping; in this chapter Q alone is the U(1) charge of the field, defined in Section 21.7.)
- **Conserved number.** A number whose total before any process equals its total after it.
- **Baryon-to-photon ratio** η_B: the number of baryons minus antibaryons per photon of the microwave background (Section 21.4). The letter η with two indices, η_{ab} or η^{ab}, is the frame metric; $\eta(K)$ without a subscript is the efficiency of the rate model of Section 21.6.
- **C, P, T, CP.** Charge conjugation C exchanges particles and antiparticles; parity P reflects space; time reversal T reverses the time; CP is C followed by P. A theory **violates** C (or CP) when a process and its image do not happen at the same rate.
- **Thermal equilibrium, temperature** T, **chemical potential** μ, **Fermi-Dirac occupation** f: Section 21.5. (The letter T also names the energy-momentum tensor in Chapter 9; in this chapter it is a temperature or the name of time reversal.)
- **Rate** K: a number of events per unit time.
- **Coordinates** $x_0, \ldots, x_7$, in the author's order: x_1, x_2, x_3 ordinary 3-space (inflating); x_4 the time; x_5, x_6, x_7 the three **extra times**, time-like, which **deflate exponentially**; x_0 the **hidden** space direction, with the angle $z = 6Hx_0$ between 0 and $\pi/2$ (the **patch**). The end $z = \pi/2$ is called the **brane**.
- **Gamma matrices** $\gamma^{(x0)}, \ldots, \gamma^{(x7)}$: the author's eight real 16×16 matrices; **charge matrix** C, **chirality** Γ, **Krein matrix** B, **generators** S^{ab}: Section 21.7.

- **Spinor field** Ψ: at every point a column of 16 numbers, the **components** $\Psi_1, \ldots, \Psi_{16}$. **Commuting** components are ordinary complex numbers (the field dirac16complex00); **anticommuting** (Grassmann) components obey $\Psi_r \Psi_c = -\Psi_c \Psi_r$ (the field dirac16complex). In Sections 21.10 and 21.15 the number $\epsilon = +1$ or -1 marks the two cases; in Sections 21.5, 21.6 and 21.36 the same letter ϵ is the baryon number made per decay, as in Notebook 21d.

- **Complex conjugate** z^*, **transpose** M^T, **conjugate transpose** $M^\dagger = (M^*)^T$; **symmetric** $M^T = M$; **antisymmetric** $M^T = -M$; **Hermitian** $M^\dagger = M$. A matrix or column is **real** when all its entries are real.

- **Dirac adjoint** $\bar\Psi = \Psi^\dagger C$; **scalar** $S = \bar\Psi\Psi$; **current** $J^\mu = -i\bar\Psi\gamma^\mu\Psi$; **charge density** $J^{(x4)} = \Psi^\dagger B \Psi$; **charge** $Q = \int \cos z \, \Psi^\dagger B \Psi \, d^7x$ (Section 21.7).

- **Charge-conjugation matrix** $\mathcal{C}$: a constant matrix for which $M = \mathcal{C}C$ obeys the **intertwiner condition** $M(\gamma^a)^* = s\,\gamma^a M$ with one sign s for all eight a (the definition of the Revision record, Section 21.8). Then $\Psi^c = \mathcal{C}\bar\Psi^T$ solves the field equation of the same form whenever Ψ does; the converse is not true in general (Section 21.8). The two that exist, up to a factor, are $\mathcal{C}_+$ and $\mathcal{C}_-$.

- **Majorana (reality) condition**: the requirement $\Psi^c = \Psi$.

- **Bilinear**: a number $\Psi^\dagger K \Psi = \sum_{r,c} \Psi_r^* K_{rc} \Psi_c$ with a fixed matrix K.

- **Noether's theorem**: a continuous symmetry of the Lagrangian gives a current whose divergence vanishes on every solution (Section 21.16).

- **Chirality partner**: $\Gamma\Psi$. **Theorem T1** of the Revision record: it solves the field equations with $(m, \lambda) \to (-m, -\lambda)$ and carries the opposite current (Section 21.19).

- **Canonical anticommutator, Krein form, Krein inertia, positive representation**: Section 21.25.

- **Majorana-type term**: a term $\Psi^T M \Psi$ built from Ψ twice, without $\Psi^\dagger$ (Section 21.32).

- **Status labels**: PROVED, COMPUTED, ASSUMED, HYPOTHESIS, OPEN, NOT COMPUTED (Section 21.1).

21.3 Particles, antiparticles and conserved numbers

Antiparticles. In 1928 Dirac wrote down the equation of the electron. Its solutions come with negative as well as positive energies, and Dirac concluded that a particle with the mass of the electron and the opposite electric charge must exist. That particle, the positron, was found in cosmic rays in 1932 (C. D. Anderson, Phys. Rev. 43, 491 (1933)). Since then every known particle has been found to have an antiparticle with the same mass and opposite charges. A few neutral particles, such as the photon, are their own antiparticles.

Annihilation and particle pair creation. An electron and a positron that meet can annihilate into photons. Conversely, a photon of enough energy that passes near a nucleus (which takes up momentum) can turn into an electron and a positron. This is the **pair creation of particles**, an observed process of ordinary physics, in which the total electric charge stays the same because the

two members carry opposite charges. It must not be confused with the question whether universes are created in pairs, which Chapter 20 treats and which no equation of this book answers.

The conserved numbers. Three numbers are counted in every process: the electric charge Q_{el}, the baryon number $\mathcal{B}$ and the lepton number L (Section 21.2). The values for a particle made of quarks follow from its quark content by addition, with an antiquark counted with the opposite sign. Line by line for the particles of the next example:

- proton $p = uud$: $\mathcal{B} = \frac{1}{3} + \frac{1}{3} + \frac{1}{3} = 1$ and $Q_{\text{el}} = \frac{2}{3} + \frac{2}{3} - \frac{1}{3} = 1$ (add the numbers of the three quarks);
- neutron $n = udd$: $\mathcal{B} = 1$ and $Q_{\text{el}} = \frac{2}{3} - \frac{1}{3} - \frac{1}{3} = 0$ (the same rule);
- antiproton $\bar{p} = \bar{u}\bar{u}\bar{d}$: $\mathcal{B} = -1$ and $Q_{\text{el}} = -1$ (every sign reversed);
- pion $\pi^+ = u\bar{d}$: $\mathcal{B} = \frac{1}{3} - \frac{1}{3} = 0$ and $Q_{\text{el}} = \frac{2}{3} - (-\frac{1}{3}) = 1$ (the antiquark $\bar{d}$ counts with the opposite sign of d);
- pion $\pi^- = d\bar{u}$: $\mathcal{B} = 0$ and $Q_{\text{el}} = -\frac{1}{3} - \frac{2}{3} = -1$;
- neutral pion π^0: $\mathcal{B} = 0$ and $Q_{\text{el}} = 0$ (Notebook 21d writes it as $u\bar{u}$; the physical π^0 is a mixture of $u\bar{u}$ and $d\bar{d}$, which have the same numbers).

The electron has $(\mathcal{B}, L, Q_{\text{el}}) = (0, 1, -1)$, the positron $(0, -1, 1)$, the antineutrino $(0, -1, 0)$ and the photon $(0, 0, 0)$.

Worked example: bookkeeping. A number is conserved in a process if its total before equals its total after. For four processes (the totals are sums over the particles on each side):

process	$\mathcal{B}$ **before, after**	L **before, after**	Q_{el} **before, after**	**observed?**
neutron decay $n \to p\, e^- \bar{\nu}_e$	1, $1 + 0 + 0 = 1$	0, $0 + 1 - 1 = 0$	0, $1 - 1 + 0 = 0$	yes
annihilation $p\,\bar{p} \to \pi^+\pi^-\pi^0$	$1 - 1 = 0$, 0	0, 0	$1 - 1 = 0$, $1 - 1 + 0 = 0$	yes
pair creation $\gamma\, p \to p\, e^- e^+$	1, 1	0, $1 - 1 = 0$	1, $1 - 1 + 1 = 1$	yes
proton decay $p \to e^+\pi^0$	1, 0	0, -1	1, $1 + 0 = 1$	never seen

The three observed processes conserve all three numbers. Proton decay would change $\mathcal{B}$ by $0-1 = -1$ and L by $-1-0 = -1$; it would keep Q_{el} and the difference $\mathcal{B}-L$ (before $1-0 = 1$, after $0-(-1) = 1$). It has been searched for and never observed. Notebook 21d computes this table with exact fractions (In [3]) and draws it (figure 21d.1).

The matter-antimatter problem is the question why the universe contains baryons but essentially no antibaryons.

21.4 What is observed: the baryon-to-photon ratio, and no antimatter domains

This section quotes observations. Nothing in it is computed by this book, and no notebook of this chapter uses a measured number; every number here is ASSUMED (quoted) from the literature named with it.

The definition. The universe is filled with the photons of the **cosmic microwave background**, the thermal radiation left over from the hot early universe, today at the temperature 2.7255 K (D. J. Fixsen, Astrophys. J. 707, 916 (2009)). After the first seconds of the universe the number of baryons and the number of these photons in a volume that expands with the universe both stay constant, so their ratio is a fixed number of our universe, the **baryon-to-photon ratio**

$$\eta_B = \frac{n_b - n_{\bar{b}}}{n_\gamma},$$

where n_b, $n_{\bar{b}}$ and n_γ are the numbers of baryons, antibaryons and photons per unit volume. Today $n_{\bar{b}}$ is negligible.

Its value. It is measured in two independent ways. The pattern of hot and cold spots in the microwave background depends on the density of baryons; the Planck satellite's 2018 analysis gives the baryon density parameter $\Omega_b h^2 = 0.0224 \pm 0.0001$ (Planck Collaboration, "Planck 2018 results. VI. Cosmological parameters", Astron. Astrophys. 641, A6 (2020)). The abundances of the light nuclei made in the first minutes (big-bang nucleosynthesis) depend on η_B as well, and the two determinations agree (R. H. Cyburt, B. D. Fields, K. A. Olive and T.-H. Yeh, Rev. Mod. Phys. 88, 015004 (2016)). Converted with the photon density of a black body at 2.7255 K and the mass of the proton (a standard conversion, quoted and not derived here), the Planck value corresponds to

$$\eta_B \approx 6.1 \times 10^{-10} :$$

about six baryons for every ten billion photons, or $1/\eta_B \approx 1.6 \times 10^9$ photons per baryon (the reciprocal of the quoted value, rounded).

Why this is a problem. Run the expansion backwards. When the thermal energy was far above the rest energy of a proton, baryons, antibaryons and photons were all abundant and turned into each other all the time. Had the early universe held exactly as many baryons as antibaryons, almost all of them would have annihilated as it cooled, leaving far less matter than we see (L. Canetti, M. Drewes and M. Shaposhnikov, New J. Phys. 14, 095012 (2012)). The baryons of today are the remnant of a tiny excess of baryons over antibaryons, about one part in 10^9. One could put this excess into the initial state by hand; the standard view, reviewed in the article just cited, is that physical processes should **generate** it from a symmetric start. Explaining how is the matter-antimatter problem, also called **baryogenesis**.

No antimatter domains. Could the universe be symmetric as a whole, with some regions of matter and others of antimatter? Wherever such regions touch, particles and antiparticles annihilate and make gamma rays. Cohen, De Rujula and Glashow showed that the gamma rays from the boundaries would exceed the observed diffuse gamma-ray background unless our region of matter is essentially the whole visible universe (A. G. Cohen, A. De Rujula and S. L. Glashow, Astrophys. J. 495, 539 (1998)). A patchwork universe is therefore excluded.

statement	status	source
$\Omega_b h^2 = 0.0224 \pm 0.0001$	ASSUMED (quoted observation)	Planck Collaboration, Astron. Astrophys. 641, A6 (2020)
$\eta_B \approx 6.1 \times 10^{-10}$, agreement with nucleosynthesis	ASSUMED (quoted)	the same; Cyburt et al., Rev. Mod. Phys. 88, 015004 (2016)
no domains of antimatter in the visible universe	ASSUMED (quoted)	Cohen, De Rujula and Glashow, Astrophys. J. 495, 539 (1998)
a value of η_B from the theory of this book	NOT COMPUTED	nothing in the Revision record computes it

21.5 Sakharov's three conditions, derived in plain words

In 1967 Andrei Sakharov named three ingredients that any process which makes the baryon excess from a symmetric start must contain (A. D. Sakharov, JETP Lett. 5, 24 (1967)). Each follows from a short argument. Throughout, the system is the whole universe; it starts with $\mathcal{B} = 0$ and without any asymmetry, and we ask what is needed for $\mathcal{B} \neq 0$ later. The arguments are general; the small models that illustrate them are ASSUMED toy models, not part of the theory of this book, and Notebook 21d computes each of them.

Condition 1: the baryon number must change.

Proposition 1. If every process conserves $\mathcal{B}$, then $\mathcal{B}(t) = \mathcal{B}(0)$ at every time t.

Proof. Conservation means $d\mathcal{B}/dt = 0$ at every time. A function of one variable whose derivative is zero everywhere on an interval is constant there (the mean value theorem of calculus: $\mathcal{B}(t) - \mathcal{B}(0) = t\,\mathcal{B}'(\tau)$ for some τ between 0 and t, and $\mathcal{B}'(\tau) = 0$). □

Starting from $\mathcal{B} = 0$, the universe would keep $\mathcal{B} = 0$ for ever. So some process must change $\mathcal{B}$.

Condition 2: C and CP must be violated. A transformation is a **symmetry of the dynamics** if it turns every possible history of the system into another possible history that runs in the same direction of time.

Proposition 2. Let $\mathcal{S}$ be a symmetry of the dynamics that reverses $\mathcal{B}$: if the history h has the baryon number $\mathcal{B}[h](t)$ at the time t, its image $\mathcal{S}h$ has $-\mathcal{B}[h](t)$. Let the initial state be an ensemble, a list of possible starting configurations c with probabilities $p(c)$, in which every configuration and its image are equally likely, $p(\mathcal{S}c) = p(c)$. Then the average baryon number is zero at every time.

Proof, line by line. Let h_c be the history that starts from c.

$$\langle \mathcal{B}(t) \rangle = \sum_c p(c)\, \mathcal{B}[h_c](t)$$

(the definition of the average over the ensemble)

$$= \sum_c p(\mathcal{S}c)\, \mathcal{B}[h_{\mathcal{S}c}](t)$$

(the map $c \to \mathcal{S}c$ is one to one, so summing over all c is the same as summing over all $\mathcal{S}c$)

$$= \sum_c p(c)\, \mathcal{B}[\mathcal{S}h_c](t)$$

(the probabilities are equal by assumption, and the history that starts from $\mathcal{S}c$ is $\mathcal{S}h_c$ because $\mathcal{S}$ is a symmetry of the dynamics)

$$= \sum_c p(c)\, \big(- \mathcal{B}[h_c](t)\big) = -\langle \mathcal{B}(t) \rangle$$

($\mathcal{S}$ reverses $\mathcal{B}$). A number equal to minus itself is zero. □

Apply this with $\mathcal{S} = \mathrm{C}$: a start without asymmetry holds particles and antiparticles in equal numbers, so it is C-symmetric, and if C were a symmetry of the dynamics no average asymmetry could develop. Apply it with $\mathcal{S} = \mathrm{CP}$: a start that is the same in every direction is also symmetric under the reflection P, which does not change $\mathcal{B}$; so CP reverses $\mathcal{B}$ as well, and if CP were a symmetry no asymmetry could develop. Hence **both C and CP must be violated**.

A decay model for conditions 1 and 2. A heavy toy particle X decays in two ways: with probability r into a channel of baryon number $\mathcal{B}_1$, with probability $1 - r$ into a channel of baryon number $\mathcal{B}_2$. Its antiparticle $\bar{X}$ decays into the antichannels, of baryon numbers $-\mathcal{B}_1$ and $-\mathcal{B}_2$, with probabilities $\bar{r}$ and $1 - \bar{r}$. (In ordinary quantum field theory the total decay rates of X and $\bar{X}$ are equal; the probabilities r and $\bar{r}$ can differ only if C and CP are violated. This is quoted, ASSUMED.) Start with N particles and N antiparticles. Line by line:

$$\text{one } X \text{ makes on average } r\mathcal{B}_1 + (1-r)\mathcal{B}_2$$

(each channel's baryon number times its probability, summed: the average of a quantity that takes two values);

$$\text{one } \bar{X} \text{ makes on average } \; -\bar{r}\mathcal{B}_1 - (1-\bar{r})\mathcal{B}_2$$

(the same rule with the antichannels);

$$N\big[r\mathcal{B}_1 + (1-r)\mathcal{B}_2 - \bar{r}\mathcal{B}_1 - (1-\bar{r})\mathcal{B}_2\big]$$

(the total of N particles and N antiparticles is N times the sum of the two averages);

$$= N\big[(r-\bar{r})\mathcal{B}_1 + (-r+\bar{r})\mathcal{B}_2\big] = N\,(r-\bar{r})(\mathcal{B}_1 - \mathcal{B}_2)$$

(collect the terms with $\mathcal{B}_1$ and with $\mathcal{B}_2$: in the second, $\mathcal{B}_2 - r\mathcal{B}_2 - \mathcal{B}_2 + \bar{r}\mathcal{B}_2 = -(r-\bar{r})\mathcal{B}_2$; then factor out $r - \bar{r}$). The net baryon number is a **product**. It vanishes if $\mathcal{B}_1 = \mathcal{B}_2$ (then X carries a conserved baryon number: condition 1 fails) or if $r = \bar{r}$ (particle and antiparticle behave alike: condition 2 fails). Example: $X \to qq$ with $\mathcal{B}_1 = \frac{2}{3}$ and $X \to \bar{q}\bar{\ell}$ (an antiquark and an antilepton) with $\mathcal{B}_2 = -\frac{1}{3}$, and $r = 0.6$, $\bar{r} = 0.5$: per pair ($N = 1$) the net baryon number is $(0.6 - 0.5)(\frac{2}{3} + \frac{1}{3}) = 0.1 \times 1 = \frac{1}{10}$ (Notebook 21d, In [5], exact; figure 21d.2 shows it for all r and $\bar{r}$).

Condition 3: a departure from thermal equilibrium. In **thermal equilibrium** at temperature T, a system's reactions run forwards and backwards equally often, and each quantum state of energy E holds a fermion with the **Fermi-Dirac** probability $f(E) = 1/(e^{(E-\mu)/T}+1)$ (Boltzmann's constant set to 1). Here μ, the **chemical potential**, belongs to a conserved number; it is quoted from statistical physics (ASSUMED) that the equilibrium state depends only on the energy and on the conserved numbers. A particle of baryon number $+1$ sees $+\mu$ and its antiparticle, which has the same mass and hence the same energy, sees $-\mu$:

$$f_+ = \frac{1}{e^{(E-\mu)/T}+1}, \qquad f_- = \frac{1}{e^{(E+\mu)/T}+1}.$$

The surplus of particles per state, line by line:

$$f_+ - f_- = \frac{\left(e^{(E+\mu)/T}+1\right) - \left(e^{(E-\mu)/T}+1\right)}{\left(e^{(E-\mu)/T}+1\right)\left(e^{(E+\mu)/T}+1\right)}$$

(the two fractions brought to their common denominator, the product of the two denominators);

$$= \frac{e^{(E+\mu)/T} - e^{(E-\mu)/T}}{e^{2E/T} + e^{(E-\mu)/T} + e^{(E+\mu)/T} + 1}$$

(the two 1s of the numerator cancel; the denominator is multiplied out, with $e^{(E-\mu)/T}e^{(E+\mu)/T} = e^{2E/T}$ because exponents add);

$$= \frac{e^{\mu/T} - e^{-\mu/T}}{e^{E/T} + e^{-\mu/T} + e^{\mu/T} + e^{-E/T}}$$

(numerator and denominator divided by $e^{E/T}$, using $e^u/e^v = e^{u-v}$);

$$= \frac{2\sinh(\mu/T)}{2\cosh(E/T) + 2\cosh(\mu/T)} = \frac{\sinh(\mu/T)}{\cosh(E/T) + \cosh(\mu/T)}$$

(the definitions $\sinh u = (e^u - e^{-u})/2$ and $\cosh u = (e^u + e^{-u})/2$, then the factor 2 cancelled). If reactions that change $\mathcal{B}$ run back and forth in equilibrium, $\mathcal{B}$ is not a conserved number and has no chemical potential: $\mu = 0$, so $\sinh 0 = 0$ and $f_+ = f_-$ at **every** energy. No asymmetry. An asymmetry can only be made while the universe is **out of equilibrium**, for example in decays that happen too late for the inverse processes to keep up.

Worked example. For a state with $E = 2T$ and $\mu = 0$, both occupations are $1/(e^2+1) = 0.119203$. A chemical potential $\mu = 0.1T$ would give $f_+ = 1/(e^{1.9}+1) = 0.130108$ and $f_- = 1/(e^{2.1}+1) = 0.109097$, a surplus of 0.021012 particles per state (Notebook 21d, In [7], where sympy also checks the formula exactly; figure 21d.3 draws the surplus against E/T).

21.6 Condition 3 in a rate model: decays out of equilibrium

How far from equilibrium must the decays be? A small ASSUMED model answers this in closed form. A toy universe cools; the time t is measured in units of its cooling time. Let $n(t)$ be the

number of heavy pairs X, $\bar{X}$ (in units of their starting number) and $n_{eq}(t) = e^{-t}$ the number that equilibrium would hold (heavy particles become rare as the temperature falls). Decays and inverse decays pull n towards n_{eq} at the rate K; every net decay of a pair makes ϵ baryons (the number of Section 21.5); inverse decays, which need X particles in equilibrium, erase the asymmetry a at the rate Kn_{eq}:

$$\frac{dn}{dt} = -K(n - n_{eq}), \qquad \frac{da}{dt} = \epsilon K(n - n_{eq}) - Kn_{eq}\, a, \qquad n(0) = 1, \quad a(0) = 0.$$

Solving it exactly, line by line (for $K \neq 1$).

Line 1. Write $n = n_{eq} + \Delta$. Then $dn/dt = dn_{eq}/dt + d\Delta/dt = -e^{-t} + d\Delta/dt$ (the derivative of a sum, and $de^{-t}/dt = -e^{-t}$). The first equation says $dn/dt = -K\Delta$, so $d\Delta/dt = -K\Delta + e^{-t}$, with $\Delta(0) = n(0) - n_{eq}(0) = 1 - 1 = 0$.

Line 2. The function $\Delta(t) = (e^{-t} - e^{-Kt})/(K-1)$ solves it. Check: its derivative is $(-e^{-t} + Ke^{-Kt})/(K-1)$ (chain rule for each exponential), and $-K\Delta + e^{-t} = \big(-Ke^{-t} + Ke^{-Kt} + (K-1)e^{-t}\big)/(K-1) = (-e^{-t} + Ke^{-Kt})/(K-1)$ (common denominator, then $-K + K - 1 = -1$): the two agree, and $\Delta(0) = (1-1)/(K-1) = 0$.

Line 3. Let $W(t) = K(1 - e^{-t})$, so that $dW/dt = Ke^{-t} = Kn_{eq}$ and $W(0) = 0$. By the product rule, $d(a\, e^{W})/dt = (da/dt + Kn_{eq}\, a)\, e^{W}$, and the second equation turns this into $\epsilon K\Delta\, e^{W}$. Integrating from 0 to t (fundamental theorem of calculus, with $a(0) = 0$) gives $a(t)e^{W(t)} = \epsilon \int_0^t K\Delta(s) e^{W(s)} ds$, that is

$$a(t) = \epsilon \int_0^t K\Delta(s)\, e^{W(s) - W(t)}\, ds.$$

Line 4. As $t \to \infty$, $W(t) \to K$, and $W(s) - K = -Ke^{-s}$. The final asymmetry is $a(\infty) = \epsilon\, \eta(K)$ with the **efficiency**

$$\eta(K) = \int_0^\infty K\Delta(s)\, e^{-Ke^{-s}}\, ds.$$

Line 5. Substitute $y = e^{-s}$: then $ds = -dy/y$, the limits $s = 0$ and $s = \infty$ become $y = 1$ and $y = 0$, and $\Delta = (y - y^K)/(K-1)$. Turning the limits round removes the minus sign:

$$\eta(K) = \frac{K}{K-1} \int_0^1 \frac{y - y^K}{y}\, e^{-Ky}\, dy = \frac{K}{K-1} \int_0^1 (1 - y^{K-1}) e^{-Ky}\, dy.$$

Line 6. The first part is $\int_0^1 e^{-Ky} dy = (1 - e^{-K})/K$. In the second substitute $u = Ky$ ($dy = du/K$, limits 0 and K): $\int_0^1 y^{K-1} e^{-Ky} dy = \int_0^K (u/K)^{K-1} e^{-u} du/K = K^{-K}\gamma(K, K)$, where $\gamma(s, x) = \int_0^x u^{s-1} e^{-u} du$ is the **lower incomplete gamma function**. Hence

$$\eta(K) = \frac{K}{K-1}\Big(\frac{1 - e^{-K}}{K} - K^{-K}\gamma(K, K)\Big).$$

What it says. For slow decays (small K) the decays happen late, when n_{eq} is already tiny, so the erasing term $Kn_{eq}a$ is ineffective and almost the whole ϵ survives. For fast decays (large K)

n stays close to n_{eq} and the inverse decays erase almost everything: $K^{-K}\gamma(K,K)$ and e^{-K} are exponentially small, so $\eta(K) = 1/(K-1)$ up to exponentially small terms. Two controls: with $\epsilon = 0$ (no C and CP violation) the second equation keeps $a = 0$ for ever; without the erasing term $da/dt = \epsilon K(n - n_{eq}) = -\epsilon\, dn/dt$, so $a(\infty) = \epsilon\big(n(0) - n(\infty)\big) = \epsilon$. Notebook 21d checks lines 1 to 3 with sympy, evaluates $\eta(K)$ with mpmath, integrates the two equations with the Runge-Kutta method of Chapter 2 and finds $\eta = 0.9955325865$, 0.4110164748 and 0.0344827586 for $K = 0.1$, 3 and 30, with the integration agreeing to 10^{-7} (In [9] and In [10]; figures 21d.4 and 21d.5). For $K = 30$ the value is $1/29$, the fast-decay limit.

statement	**status**	**where**
Propositions 1 and 2 (conditions 1 and 2)	PROVED (above)	Section 21.5
decay model: net baryon number $N(r - \bar{r})(\mathcal{B}_1 - \mathcal{B}_2)$; example 1/10	PROVED for the ASSUMED model; COMPUTED exactly	Notebook 21d, In [5]
equilibrium surplus $\sinh(\mu/T)/(\cosh(E/T) + \cosh(\mu/T))$; zero for $\mu = 0$	PROVED (above; sympy)	Notebook 21d, In [7]
rate model: the closed form of $\eta(K)$ and its two limits	PROVED for the ASSUMED model; COMPUTED (RK4 to 10^{-7})	Notebook 21d, In [9] to In [12]

21.7 The theory of this book: two fields, one phase symmetry

This section collects what the rest of the chapter needs. Every formula is from the Revision record; the chapters named derive them in full.

The metric. The author's metric is diagonal. With the abbreviations $s = \sin^{1/3} z$ and $z = 6Hx_0$, its entries in the order $x_0, \ldots, x_7$ are

$$g = \mathrm{diag}\big(\cot^2 z,\ e^{2a_4}s,\ e^{2a_4}s,\ e^{2a_4}s,\ -1,\ -e^{-2a_4}s,\ -e^{-2a_4}s,\ -e^{-2a_4}s\big).$$

Its **scale factors** $f_a = \sqrt{|g_{aa}|}$ are $f_0 = \cot z$ (the hidden direction), $f_1 = f_2 = f_3 = e^{a_4}\sin^{1/6} z$ (ordinary space), $f_4 = 1$ (the time) and $f_5 = f_6 = f_7 = e^{-a_4}\sin^{1/6} z$ (the extra times). This is the author's metric of 2026-10-14 on its background $b_4 = 1$: the metric has a second function $b_4(x_4) > 0$, and with it $f_5 = f_6 = f_7 = e^{-a_4} b_4^{1/2} \sin^{1/6} z$ (Chapter 3). The records and the notebooks of this chapter work at $b_4 = 1$; Section 21.7 says what a changing b_4 changes in the charge. When a_4 grows with the time, ordinary space inflates and the three extra times deflate exponentially. The volume factor is $\sqrt{|\det g|} = f_0 f_1 \cdots f_7 = e^{3a_4}e^{-3a_4}\sin z \cot z = \cos z$: the inflation of space and the deflation of the extra times cancel, and a_4 drops out (Chapter 12; record check `sqrt_abs_det_g_is_cos_z` of `Revision/field_equations_a4/reports/wolfram-a4-report.json`). This one fact will be the reason why the local conservation law of the charge has no time-derivative terms (Section 21.17).

The gammas. The Revision record `Revision/algebra/gammas.json` holds the author's eight real 16×16 gamma matrices $\gamma^{(x0)}, \ldots, \gamma^{(x7)}$, rebuilt in Revision code from the author's formulas. Every entry is -1, 0 or $+1$, and each row and column holds one nonzero entry (a **signed permutation matrix**). They obey the **Clifford relation**

$$\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}\,1, \qquad \eta = \mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1),$$

and the **symmetry pattern** $(\gamma^a)^T = \eta_{aa}\gamma^a$: the four space-like gammas (x_0, x_1, x_2, x_3) are symmetric, the four time-like ones antisymmetric. Two consequences used again and again: different gammas **anticommute**, $\gamma^a\gamma^b = -\gamma^b\gamma^a$ for $a \neq b$, and $\gamma^a\gamma^a = \eta^{aa}1$. Notebook 21a checks all of this exactly (In [3]).

The four matrices built from them (Chapters 4 and 5):

- the **charge matrix** $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$, the product of the four space-like gammas (the author's sigma16);
- the **chirality** $\Gamma = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\cdots\gamma^{(x7)}$, the product of all eight; it is diagonal, with the entries -1 in the first eight places and $+1$ in the last eight, written $\Gamma = \mathrm{diag}(-1_8, 1_8)$;
- the **Krein matrix** $B = -iC\gamma^{(x4)}$;
- the 28 **generators** $S^{ab} = \frac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$ of the rotations and boosts, which for $a \neq b$ equal $\frac{1}{2}\gamma^a\gamma^b$.

Three rules for moving gammas, each derived from the Clifford relation.

(R1) Γ anticommutes with every gamma. Moving γ^b from the left of Γ to its right passes the seven factors γ^a with $a \neq b$, each of which costs a sign, and the factor γ^b itself, which costs none: the total sign is $(-1)^7 = -1$.

(R2) Γ commutes with every product of an even number of gammas, in particular with C and with every S^{ab} (each factor costs a sign by (R1), and an even number of signs multiply to $+1$). Also $\Gamma\Gamma = 1$ and $\Gamma^T = \Gamma$, because Γ is diagonal with entries ± 1.

(R3) $C^T = C$, $CC = 1$ and $C\gamma^a C = -(\gamma^a)^T$ for every a. Derivation, line by line. First

$$C^T = (\gamma^{(x3)})^T(\gamma^{(x2)})^T(\gamma^{(x1)})^T(\gamma^{(x0)})^T = \gamma^{(x3)}\gamma^{(x2)}\gamma^{(x1)}\gamma^{(x0)}$$

(the transpose of a product is the product of the transposes in reverse order; space-like gammas are symmetric). Reversing the order of four different anticommuting factors takes $3 + 2 + 1 = 6$ exchanges, so $C^T = (-1)^6 C = C$. Then

$$CC = CC^T = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}\gamma^{(x3)}\gamma^{(x2)}\gamma^{(x1)}\gamma^{(x0)} = 1$$

(each neighbouring pair $\gamma\gamma$ of a space-like gamma is $+1$; remove them from the inside out). Next, $\gamma^a C = -C\gamma^a$ when a is space-like (passing through C costs three signs for the other three factors and none for γ^a itself) and $\gamma^a C = +C\gamma^a$ when a is time-like (four signs). With the symmetry pattern, $(C\gamma^a)^T = (\gamma^a)^T C^T = \eta_{aa}\gamma^a C = -C\gamma^a$ in both cases: every $C\gamma^a$ is **antisymmetric**. Writing this as $(\gamma^a)^T C = -C\gamma^a$ and multiplying from the right by C (with $CC = 1$) gives $(\gamma^a)^T = -C\gamma^a C$.

The record checks these facts exactly (`Revision/algebra/reports/python-algebra.json` and `Revision/lead_checks/reports/charge-conjugation-and-u1.json`, check `representation_real`; Notebook 21a, In [5]).

The two fields. dirac16complex is a field Ψ with 16 complex **anticommuting** (Grassmann) components, quantised canonically (Chapter 10). dirac16complex00 is a field with 16 complex **commuting** components, a classical field. Both transform as Pin(4,4) spinors and are coupled to gravity through the vielbein (the scale factors) and the canonical spin connection Ω_μ (Chapter 6). The Revision record (`Revision/theory/field-theory.json`, formula `Lagrangian`) gives the same Lagrangian density for both:

$$\mathcal{L} = \cos z \left[\tfrac{1}{2}\sum_\mu \left(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi\right) - mS - U(S)\right], \qquad \bar\Psi = \Psi^\dagger C, \quad S = \bar\Psi\Psi,$$

with the curved gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ (no sum), $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$, $D_\mu\bar\Psi = \partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$, the mass m and the self-interaction $U(S) = \frac{\lambda}{2}S^2$ with the coupling λ. Its Euler-Lagrange equation (Chapter 7) is the **field equation**

$$\gamma^\mu D_\mu\Psi = V\Psi, \qquad V = m + U'(S),$$

which in the author's metric reads (record formula `field_equation`)

$$e^{-a_4}\sin^{-1/6} z\sum_{i=1}^{3}\gamma^{(xi)}\partial_i\Psi + \gamma^{(x4)}\partial_4\Psi + e^{a_4}\sin^{-1/6} z\sum_{t=5}^{7}\gamma^{(xt)}\partial_t\Psi + \tan z\,\gamma^{(x0)}\partial_0\Psi + 3H\gamma^{(x0)}\Psi = V\Psi.$$

The term $3H\gamma^{(x0)}\Psi$ is $\gamma^\mu\Omega_\mu\Psi$, the spin connection contracted with the gammas (record formula `gammaOmega_total`; Chapter 8). **Every matrix in this equation is real**: the gammas, the scale factors, $\tan z$, $3H$, and every entry of every Ω_μ (lead check `spinor_connection_real`); V is real because $S = \Psi^\dagger C\Psi$ is real (Chapter 7).

The phase symmetry and its current. Multiplying every component by one phase, $\Psi \to e^{i\alpha}\Psi$, multiplies $\Psi^\dagger$ by $e^{-i\alpha}$, so every bilinear $\Psi^\dagger X\Psi$, hence $\mathcal{L}$, is unchanged. The group of these phases is called **U(1)**. Its current and charge (record formula `current`) are

$$J^\mu = -i\bar\Psi\gamma^\mu\Psi, \qquad J^{(x4)} = \Psi^\dagger B\Psi, \qquad Q = \int \cos z\,\Psi^\dagger B\Psi\,dx_0\,dx_1\,dx_2\,dx_3\,dx_5\,dx_6\,dx_7.$$

(The time component: $J^{x4} = -i\Psi^\dagger C\gamma^{(x4)}\Psi = \Psi^\dagger B\Psi$, because $f_4 = 1$.) Section 21.17 proves the local conservation law behind Q: Q does not change in time when no charge flows through the boundary of the slice, and Section 21.18 shows that at the brane $z = \pi/2$ this is a real condition.

With a changing b_4. In the author's metric with a second function $b_4(x_4)$ the volume factor is $\sqrt{|g|} = b_4^{3/2}\cos z$ (`Revision/theory/reports/wolfram-field-theory-b4.json`, check `sqrt_det_g`), so the charge becomes $Q = \int b_4^{3/2}\cos z\,\Psi^\dagger B\Psi\,dx_0\,dx_1\,dx_2\,dx_3\,dx_5\,dx_6\,dx_7$; the extra-time derivatives carry the factor $e^{a_4}b_4^{-1/2}\sin^{-1/6} z$, and $\gamma^\mu\Omega_\mu$ gains the term $\frac{3}{4}(b_4'/b_4)\gamma^{(x4)}$ (Chapter 7). Every matrix of the field equation stays real (`Revision/theory/reports/python-field-theory-b4.json`, check `spinor_connection_real_b4`), and the matrix identity behind the conservation law holds for every b_4 (same report, check `u1_noether_matrix_identity_b4`). So the statements of this chapter about the conjugation matrices and the local conservation law do not depend on b_4; the notebooks and the exact solutions of this chapter are computed at $b_4 = 1$.

21.8 Charge conjugation is a matrix: the condition on the matrix

The question. An antiparticle has the same mass and the opposite charge. In a field theory the step from a solution to its "antiparticle solution" is made by a map called **charge conjugation**: it turns every solution of the field equation into a solution whose charge density has the opposite sign. In Dirac's theory of the electron, whose gamma matrices are complex, this map combines the complex conjugate Ψ^* with a fixed matrix. In the author's theory the gammas are real. Take a **real** field, one whose components are real numbers, so that $\Psi^* = \Psi$. On such a field, complex conjugation does nothing: it is the identity map, and a map that changes nothing cannot exchange matter and antimatter. Whatever charge conjugation is in this theory, it must be made by a **matrix**. This section derives a condition on the matrix that is enough for it to do its job and makes that condition the definition, as the Revision record does; the next section finds every matrix that obeys this condition.

The wish. We look for a constant 16×16 matrix $\mathcal{C}$ such that, whenever Ψ solves the field equation, the column

$$\Psi^c = \mathcal{C}\,\bar{\Psi}^T$$

solves the field equation of the same form, either with the same V (**same mass**) or with $-V$ (**mass reversed**).

From the wish to an equation for a matrix, line by line.

$$\bar{\Psi}^T = (\Psi^\dagger C)^T = C^T(\Psi^\dagger)^T = C\Psi^*$$

(the definition of $\bar{\Psi}$; the rule $(XY)^T = Y^T X^T$; the transpose of the row $\Psi^\dagger$ is the column Ψ^*, and $C^T = C$ by (R3)). So $\Psi^c = \mathcal{C}C\Psi^*$. Write $M = \mathcal{C}C$; then $\Psi^c = M\Psi^*$, and because $CC = 1$, $\mathcal{C} = MC$.

Step 1 (conjugate the equation). The complex conjugate of a product is the product of the conjugates, a real factor is its own conjugate, and the derivative of the conjugate is the conjugate of the derivative (the coordinates are real). All matrices of $\gamma^\mu D_\mu$ and the number V are real (Section 21.7). So $\gamma^\mu D_\mu \Psi^* = V\Psi^*$: **the column Ψ^* solves the same equation.**

Step 2 (multiply by M from the left). $M\gamma^\mu D_\mu \Psi^* = V\,M\Psi^*$ (a number V may stand on either side of a matrix).

Step 3 (the condition). Suppose that, with one sign $s = +1$ or $s = -1$ for all eight directions,

$$M\gamma^a = s\,\gamma^a M \qquad (a = x0, \ldots, x7).$$

Then $M\gamma^\mu = s\,\gamma^\mu M$ (the scale factor $1/f_\mu$ is a number). M commutes with every product of two gammas: $M\gamma^a\gamma^b = s\,\gamma^a M\gamma^b = s^2\gamma^a\gamma^b M = \gamma^a\gamma^b M$ (the condition used twice, then $s^2 = 1$). The spin connection is a combination of such products, so $M\Omega_\mu = \Omega_\mu M$; a constant matrix commutes with a derivative; hence $MD_\mu = D_\mu M$.

Step 4 (the new field). With step 3, $M\gamma^\mu D_\mu \Psi^* = s\,\gamma^\mu M D_\mu \Psi^* = s\,\gamma^\mu D_\mu (M\Psi^*)$, and step 2 becomes $s\,\gamma^\mu D_\mu (M\Psi^*) = V\,(M\Psi^*)$. Multiplying by s and using $s^2 = 1$:

$$\gamma^\mu D_\mu(M\Psi^*) = s\,V\,(M\Psi^*).$$

The new field $M\Psi^*$ solves the equation with V when $s=+1$ and with $-V$ when $s=-1$. Since the gammas are real, $(\gamma^a)^* = \gamma^a$, and the condition can be written $M(\gamma^a)^* = s\,\gamma^a M$, the form in which the Revision record solves it. (For $V = m + \lambda S$, which depends on the field, Section 21.10 shows that for commuting components both matrices found below keep S; then $sV = sm + s\lambda S$, so the new field solves the equation with (m, λ) when $s=+1$ and with $(-m, -\lambda)$ when $s=-1$. For a free field, $\lambda = 0$, only the mass matters.)

Definition. A **charge-conjugation matrix** is a constant 16×16 matrix $\mathcal{C}$ for which the matrix $M = \mathcal{C}C$ obeys the **intertwiner condition**

$$M(\gamma^a)^* = s\,\gamma^a M \qquad (a = x0, \ldots, x7),$$

with one sign s ($+1$ or -1) for all eight directions; it is called **same mass** when $s=+1$ and **mass reversed** when $s=-1$. This is the definition of the Revision record: its checks `intertwiners_same_mass` and `intertwiners_reversed_mass` solve exactly this condition. Steps 1 to 4 prove that every charge-conjugation matrix fulfils the wish: for every solution Ψ, the column $\Psi^c = M\Psi^*$ solves the field equation with sV.

Why the definition is the condition and not the wish. The condition is enough for the wish, but the converse is not true in general: a matrix can turn every solution into a solution without obeying the condition. An example, line by line, for $V=0$ (mass 0 and no self-interaction, a case that the field equation allows), with the chiral projectors $P_\pm = \frac{1}{2}(1 \pm \Gamma)$ and $M = P_-$, that is $\mathcal{C} = P_- C$:

1. P_- is a combination of 1 and of Γ, so by (R2) it commutes with every product of two gammas, hence with every Ω_μ; as a constant matrix it commutes with every derivative. So $P_- D_\mu = D_\mu P_-$.
2. By (R1), $\gamma^a P_- = \frac{1}{2}(\gamma^a - \gamma^a\Gamma) = \frac{1}{2}(\gamma^a + \Gamma\gamma^a) = P_+\gamma^a$, and in the same way $P_-\gamma^a = \gamma^a P_+$. Multiplying by the scale factors gives $\gamma^\mu P_- = P_+\gamma^\mu$.
3. For every solution Ψ: $\gamma^\mu D_\mu(P_-\Psi^*) = P_+\gamma^\mu D_\mu\Psi^* = P_+(V\Psi^*) = 0$ (lines 1 and 2, then Step 1, then $V=0$).

So $P_-\Psi^*$ solves the equation with $V=0$, which is also $-V$: the matrix P_-C fulfils the wish. It does not obey the condition, for either sign: if $P_-\gamma^a = s\,\gamma^a P_-$ held, then with line 2 $\gamma^a P_+ = s\,\gamma^a P_-$; multiplying from the left by $\eta_{aa}\gamma^a$ (and $\eta_{aa}\gamma^a\gamma^a = 1$) gives $P_+ = s\,P_-$, and multiplying from the right by P_+ gives $P_+ = 0$, because $P_+P_+ = P_+$ and $P_-P_+ = \frac{1}{4}(1 - \Gamma\Gamma) = 0$. But $P_+ = \mathrm{diag}(0, 1_8)$ is not zero. (Section 5.28 gives the same example.) So with the wish as the definition, the count "exactly two" of Theorem CC would be false for $V=0$; with the condition as the definition, it is proved below. For $V \neq 0$ this book does not study whether some matrix outside the condition fulfils the wish in the author's metric (OPEN); nothing in this chapter depends on that question, because the Revision record and this chapter use the condition.

21.9 The two charge-conjugation matrices

Theorem CC.

- (a) The matrices M with $M(\gamma^a)^* = +\gamma^a M$ for all eight a are exactly the multiples of 1; the matrices with $M(\gamma^a)^* = -\gamma^a M$ for all a are exactly the multiples of Γ.
- (b) Hence, with the intertwiner condition as the definition (Section 21.8), there are exactly two charge-conjugation matrices, up to a factor: $\mathcal{C}_+ = C$ (same mass), with $\Psi^c = \mathcal{C}_+ \bar{\Psi}^T = \Psi^*$, and $\mathcal{C}_- = \Gamma C$ (mass reversed), with $\Psi^c = \mathcal{C}_- \bar{\Psi}^T = \Gamma\Psi^*$.
- (c) $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$ and $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = +(\gamma^a)^T$ for every a; both are real and symmetric. Conversely, the only matrices X with $\gamma^a X = \zeta X(\gamma^a)^T$ for all a are the multiples of C ($\zeta = -1$) and of ΓC ($\zeta = +1$).

Proof of (a). The gammas are real, so the conditions read $M\gamma^a = \pm\gamma^a M$. For the sign $+$, M commutes with all eight gammas. The 16 components carry an **irreducible** representation of the group Pin(4,4), which the gammas generate (Chapter 5); by Schur's lemma, a matrix that commutes with every matrix of an irreducible representation is a multiple of 1 (record check `pin_commutant_dimension_1` of `Revision/algebra/reports/python-algebra.json`, which solves these equations exactly and finds a space of dimension 1). For the sign $-$, M anticommutes with every gamma; then ΓM commutes with every gamma:

$$\gamma^a(\Gamma M) = -\Gamma\gamma^a M = -\Gamma(-M\gamma^a) = (\Gamma M)\gamma^a$$

(first (R1), then the condition read as $\gamma^a M = -M\gamma^a$, then the two signs multiplied). So $\Gamma M = c\,1$ for a number c, and multiplying from the left by Γ (with $\Gamma\Gamma = 1$) gives $M = c\,\Gamma$. Conversely 1 commutes and Γ anticommutes with every gamma (R1), so both are solutions. □

Why each equation halves the freedom. A 16×16 matrix has 256 entries. Take only the first condition, $M\gamma^{(x1)} = \gamma^{(x1)}M$. The matrix $\gamma^{(x1)}$ is real and symmetric with $\gamma^{(x1)}\gamma^{(x1)} = 1$, so its eigenvalues are $+1$ and -1 (if $\gamma u = cu$ then $u = \gamma\gamma u = c^2 u$, so $c^2 = 1$); its trace is zero (its nonzero entries lie outside the diagonal blocks, figure 21a.1), so each eigenvalue occurs 8 times. If M commutes with $\gamma^{(x1)}$ and $\gamma^{(x1)}u = cu$, then $\gamma^{(x1)}(Mu) = M\gamma^{(x1)}u = c\,Mu$: M maps each eigenspace into itself. In a basis of eigenvectors, M is then made of two free 8×8 blocks: $64 + 64 = 128$ free entries, half of 256. (For $M\gamma^{(x1)} = -\gamma^{(x1)}M$, M maps each eigenspace into the other one, again $64 + 64 = 128$.) Notebook 21a computes the whole sequence exactly: imposing the conditions for the first k gammas leaves $256/2^k$ free numbers, $256, 128, 64, \ldots, 2, 1$ (In [7], figure 21a.2). With all eight only one direction is left: the multiples of 1 or of Γ, as part (a) says.

Proof of (b). $\mathcal{C} = MC$ (Section 21.8). $M = 1$ gives $\mathcal{C}_+ = C$ and $\Psi^c = \Psi^*$; $M = \Gamma$ gives $\mathcal{C}_- = \Gamma C$ and $\Psi^c = \Gamma\Psi^*$. □

Proof of (c). For $\mathcal{C}_+ = C$: $C^{-1} = C$, so $C^{-1}\gamma^a C = C\gamma^a C = -(\gamma^a)^T$ by (R3). For $\mathcal{C}_-$: the inverse of a product is the product of the inverses in reverse order, $\mathcal{C}_-^{-1} = C^{-1}\Gamma^{-1} = C\Gamma$, and

$$\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = C\Gamma\gamma^a\Gamma C = C(-\gamma^a\Gamma)\Gamma C = -C\gamma^a C = +(\gamma^a)^T$$

(first (R1), then $\Gamma\Gamma = 1$, then (R3)). Both are products of real matrices, hence real. $C^T = C$ is (R3); $(\Gamma C)^T = C^T\Gamma^T = C\Gamma = \Gamma C$ by the transpose rule, (R3), (R2). For the converse: by (R3) the equation $\gamma^a X = \zeta X(\gamma^a)^T$ reads $\gamma^a X = -\zeta XC\gamma^a C$; multiplying from the right by C gives $\gamma^a(XC) = -\zeta(XC)\gamma^a$, so by part (a) XC is a multiple of 1 when $\zeta = -1$ and of Γ when $\zeta = +1$; multiplying from the right by C gives $X = c\,C$ or $X = c\,\Gamma C$. □

The shape of the two matrices. $\Gamma = \mathrm{diag}(-1_8, 1_8)$, so ΓC is C with the signs of its first eight rows reversed; both have their entries only in the two diagonal 8×8 blocks (figure 21a.3).

statement	status	where it is verified
the gammas, C and the S^{ab} are real; B is purely imaginary, Hermitian, $B^2 = 1$	PROVED	`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, checks `representation_real`, `B_imaginary_hermitian`; Notebook 21a, In [5]
Theorem CC (a): only multiples of 1 and of Γ	PROVED; the 2048 equations of each sign solved exactly	the same report, checks `intertwiners_same_mass`, `intertwiners_reversed_mass`; Notebook 21a, In [6]
each gamma equation halves the solution space, $256/2^k$	COMPUTED exactly (rank over the integers)	Notebook 21a, In [7] (its own computation)
Theorem CC (b), (c): with the intertwiner condition as the definition, $\mathcal{C}_+ = C$, $\mathcal{C}_- = \Gamma C$, their transposition rules	PROVED	the same report, checks `charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus`; Notebook 21a, In [8]
the intertwiner condition is enough, not necessary: for $V = 0$ the matrix $P_- C$ maps every solution to a solution without obeying it	PROVED	Section 21.8 (derivation); Section 5.28

21.10 The reality conditions, the bilinears and real fields

The reality (Majorana) conditions. A field equal to its own conjugate obeys $\Psi = M\Psi^*$. Is that a sensible requirement? Conjugating it gives $\Psi^* = M^*\Psi$, and inserting this back gives $\Psi = MM^*\Psi$. If $MM^* = 1$, this is automatically true and the condition is **consistent**; otherwise it forces further restrictions. For $M = 1$: $MM^* = 1$, and the condition $\Psi = \Psi^*$ says the field is real. For $M = \Gamma$: $\Gamma\Gamma^* = \Gamma\Gamma = 1$, and the condition $\Psi = \Gamma\Psi^*$ says, component by component, $\Psi_r = -\Psi_r^*$ for $r = 1, \ldots, 8$ (purely imaginary components, where $\Gamma = -1$) and $\Psi_r = \Psi_r^*$ for $r = 9, \ldots, 16$ (real components). Both conditions are consistent (lead check `majorana_conditions_consistent`; Notebook 21a, In [10], figure 21a.4).

How a bilinear changes, line by line. For a fixed matrix K the bilinear $\Psi^\dagger K\Psi = \sum_{r,c} \Psi_r^* K_{rc} \Psi_c$ is one number. Replace Ψ by $\Psi' = M\Psi^*$ with a real matrix M. Exchanging two components costs the factor $\epsilon = +1$ for commuting and $\epsilon = -1$ for anticommuting components (the conjugated components Ψ_r^* count as further anticommuting numbers).

Line 1. $(\Psi')^\dagger = (M\Psi^*)^\dagger = (\Psi^*)^\dagger M^\dagger = \Psi^T M^\dagger$ (the rule $(XY)^\dagger = Y^\dagger X^\dagger$; conjugating twice gives the number back). So $\Psi'^\dagger K \Psi' = \Psi^T M^\dagger K M \Psi^* = \sum_{r,c} \Psi_r \, (M^\dagger K M)_{rc} \, \Psi_c^*$.

Line 2. Exchange the factors Ψ_r and Ψ_c^*, which costs ϵ: $\sum_{r,c} \epsilon \, \Psi_c^* (M^\dagger K M)_{rc} \Psi_r$.

Line 3. Rename the summation letters, $r \leftrightarrow c$: $\sum_{r,c} \Psi_r^* \, \epsilon (M^\dagger K M)_{cr} \, \Psi_c = \Psi^\dagger K' \Psi$ with

$$K' = \epsilon \, (M^\dagger K M)^T .$$

If $K' = +K$ the bilinear is kept; if $K' = -K$ it is reversed.

The scalar and the currents. S has $K = C$, the current J^a has $K = -iC\gamma^a$. Both maps are real and symmetric, $M^\dagger = M$.

- $\mathcal{C}_+$ $(M = 1)$, scalar: $K' = \epsilon C^T = \epsilon C$ (R3). So $S \to \epsilon S$.
- $\mathcal{C}_+$, currents: $K' = \epsilon(-iC\gamma^a)^T = \epsilon(-i)(C\gamma^a)^T = \epsilon(-i)(-C\gamma^a) = -\epsilon K$ (the number $-i$ is not changed by a transpose; $C\gamma^a$ is antisymmetric, (R3)). So $J^a \to -\epsilon J^a$.
- $\mathcal{C}_-$ $(M = \Gamma)$, scalar: $\Gamma C \Gamma = C \Gamma \Gamma = C$ (R2), so $K' = \epsilon C$ and $S \to \epsilon S$.
- $\mathcal{C}_-$, currents: $\Gamma(-iC\gamma^a)\Gamma = -iC\Gamma\gamma^a\Gamma = -iC(-\gamma^a) = iC\gamma^a$ (R2, R1, $\Gamma\Gamma = 1$), and its transpose is $i(C\gamma^a)^T = -iC\gamma^a = K$. So $K' = \epsilon K$ and $J^a \to \epsilon J^a$.

map	**components**	S **becomes**	**every** J^a **becomes**
$\mathcal{C}_+$ $(M = 1)$	commuting $(\epsilon = +1)$	$+S$	$-J^a$
$\mathcal{C}_+$ $(M = 1)$	anticommuting $(\epsilon = -1)$	$-S$	$+J^a$
$\mathcal{C}_-$ $(M = \Gamma)$	commuting $(\epsilon = +1)$	$+S$	$+J^a$
$\mathcal{C}_-$ $(M = \Gamma)$	anticommuting $(\epsilon = -1)$	$-S$	$-J^a$

This is the table that the Revision record measured (lead check `bilinears_under_charge_conjugation`; Notebook 21a, In [14], compares with the record's own table). For the commuting field dirac16complex00, $\mathcal{C}_+$ keeps the mass and the scalar and reverses every current, in particular the charge density: it is the antiparticle map of this field. $\mathcal{C}_-$ reverses the mass and keeps the charge. The anticommuting rows are statements about classical Grassmann components; the quantised field is decided by the operator computation of Section 21.26. (The record's detail text adds in parentheses that normal ordering adds no sign: it subtracts the vacuum value, a number, so a bilinear that changes as $X \to sX + c$ has the normal-ordered form changing as $:X: \to s \; :X:$; the record makes no Fock-space computation for this. Its earlier remark that normal ordering supplies one more sign was withdrawn on 2026-10-08. Section 21.26 confirms the corrected text for the charge with explicit operators: the same-mass candidate counts the charge $Q - 16$, the charge shifted by a constant and not reversed, in agreement with the operator computation of Section 5.34.)

A check with numbers (Notebook 21a, In [17]): for one fixed complex column Ψ the notebook finds $S = +6.0205$ and $J^{(x4)} = +6.4970$; for Ψ^*, $S = +6.0205$ and $J^{(x4)} = -6.4970$; for $\Gamma\Psi^*$, $S = +6.0205$ and $J^{(x4)} = +6.4970$.

All 256 bilinears. Every 16×16 matrix is a combination of the 256 products $\Gamma_A = \gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$ ($a_1 < \cdots < a_k$ in the order $x_0, \ldots, x_7$, $k = 0, \ldots, 8$, with $\Gamma_\emptyset = 1$), so the bilinears $\bar{\Psi}\Gamma_A\Psi = \Psi^\dagger C\Gamma_A\Psi$ are a complete list. The number k of gammas is the **degree**. For $\mathcal{C}_+$ with commuting components, $K' = (C\Gamma_A)^T$, and line by line:

$$(C\Gamma_A)^T = \Gamma_A^T C^T = (\gamma^{a_k})^T \cdots (\gamma^{a_1})^T\, C$$

(the transpose of a product reverses the order; $C^T = C$);

$$= (-C\gamma^{a_k}C)\cdots(-C\gamma^{a_1}C)\, C = (-1)^k C\gamma^{a_k}\cdots\gamma^{a_1} C\, C$$

(R3 for each factor; the inner pairs $CC = 1$ cancel);

$$= (-1)^k(-1)^{k(k-1)/2}\, C\gamma^{a_1}\cdots\gamma^{a_k} = (-1)^{k(k+1)/2}\, C\Gamma_A$$

($CC = 1$ at the end; reversing the order of k different anticommuting factors takes $k(k-1)/2$ exchanges, each costing a sign; and $k + k(k-1)/2 = k(k+1)/2$). So under $\mathcal{C}_+$ a commuting bilinear of degree k gets the sign $(-1)^{k(k+1)/2}$: $+, -, -, +, +, -, -, +, +$ for $k = 0, \ldots, 8$. For $\mathcal{C}_-$ the extra factor is $\Gamma C\Gamma_A\Gamma = C\,\Gamma\Gamma_A\Gamma = (-1)^k C\Gamma_A$ (R2, then R1 once for each of the k factors), so the sign is $(-1)^k(-1)^{k(k+1)/2} = (-1)^{k(k-1)/2}$ (the exponents differ by $2k$, an even number). Anticommuting components multiply every sign by $\epsilon = -1$. Notebook 21a computes the sign of all 256 bilinears in the four cases and finds exactly these rules (In [15], figure 21a.6).

Real fields. For a field with real commuting components three facts hold.

- (F1) **The currents vanish.** $J^a = -i\Psi^T A\Psi$ with the antisymmetric $A = C\gamma^a$ (R3). For every column u of real numbers and every antisymmetric A, the number $u^T Au$ equals its own transpose, $u^T Au = (u^T Au)^T = u^T A^T u = -u^T Au$ (a number is its own transpose; the transpose of a product reverses the order; $A^T = -A$), and a number equal to minus itself is zero. A real field carries no current and no U(1) charge.
- (F2) $\mathcal{C}_+$ **acts as the identity.** $\Psi^c = \Psi^* = \Psi$: a real field is its own same-mass conjugate.
- (F3) **The real matrix Γ reverses the mass.** The map $\Psi \to \Gamma\Psi$ (no conjugation) takes real fields to real fields. It keeps the scalar, $(\Gamma\Psi)^T C(\Gamma\Psi) = \Psi^T\Gamma^T C\Gamma\Psi = \Psi^T C\Psi$ (R2), and it reverses every kinetic matrix, $\Gamma^T C\gamma^a\Gamma = C\Gamma\gamma^a\Gamma = -C\gamma^a$ (R2, R1). In the field equation, Γ commutes with Ω_μ (an even matrix, R2) and with every derivative, and anticommutes with every γ^μ, so $\gamma^\mu D_\mu(\Gamma\Psi) = -\Gamma\gamma^\mu D_\mu\Psi = -V\,\Gamma\Psi$: with S unchanged, $\Gamma\Psi$ solves the equation with $(m, \lambda) \to (-m, -\lambda)$. This is theorem T1 of the Revision record (Section 21.19).

By Theorem CC, every real matrix that obeys the intertwiner condition of Section 21.8 is a multiple of 1 or of Γ. So for real fields, among the matrices that obey this condition, the only nontrivial matrix map is Γ, and it reverses the mass (lead check `real_fields_charge_conjugation`; Notebook 21a, In [18] and In [19], figure 21a.7). A real field has no charge to reverse (F1); what Γ exchanges is the sign of the mass. Plain complex conjugation, which is the identity on a real field, is never a matter-antimatter map for it.

21.11 The conjugations acting on an exact solution in the author's metric

Theorem CC is general. Here it is watched at work on an **exact solution of the field equation in the author's curved metric**, valid for every history $a_4(x_4)$, in particular the one in which the three extra times deflate exponentially.

The solution. The Revision record (`Revision/theory/field-theory.json`, formula `exact_solutions`, item (i)) states: for $U = 0$,

$$\Psi = \sin^{\alpha} z \,\Big(\cosh(kx_4)\, 1 + \frac{\sinh(kx_4)}{k} M_m \Big) \chi,$$

$$M_m = -m\gamma^{(x4)} + b\,\gamma^{(x4)}\gamma^{(x0)}, \qquad b = 3H(2\alpha+1), \qquad k^2 = b^2 - m^2,$$

with a constant column χ and any number α. It depends only on x_4 and z. **Why it solves the equation**, line by line:

Step 1. $M_m^2 = m^2\gamma^{(x4)}\gamma^{(x4)} - mb\big(\gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)} + \gamma^{(x4)}\gamma^{(x0)}\gamma^{(x4)}\big) + b^2\gamma^{(x4)}\gamma^{(x0)}\gamma^{(x4)}\gamma^{(x0)}$ (the square multiplied out, keeping the order of the matrices).

Step 2. $\gamma^{(x4)}\gamma^{(x4)} = -1$ (x_4 is time-like) and $\gamma^{(x0)}\gamma^{(x0)} = +1$; $\gamma^{(x4)}\gamma^{(x0)}\gamma^{(x4)} = -\gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)} = \gamma^{(x0)}$ (anticommute the last two, then $\gamma^{(x4)}\gamma^{(x4)} = -1$). So the bracket is $-\gamma^{(x0)} + \gamma^{(x0)} = 0$, and the last term is $b^2\gamma^{(x4)}(\gamma^{(x0)}\gamma^{(x4)})\gamma^{(x0)} = -b^2\gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)}\gamma^{(x0)} = b^2$. Hence $M_m^2 = -m^2 + b^2 = k^2$ (times 1).

Step 3. Let $U(x_4) = \cosh(kx_4) + \sinh(kx_4)M_m/k$. Its derivative is $k\sinh(kx_4) + \cosh(kx_4)M_m$ (chain rule), and $M_m U = \cosh(kx_4)M_m + \sinh(kx_4)M_m^2/k = \cosh(kx_4)M_m + k\sinh(kx_4)$ (step 2): so $dU/dx_4 = M_m U$, and $\partial_4\Psi = M_m\Psi$ ($\sin^{\alpha} z$ and χ do not depend on x_4).

Step 4. The derivatives along $x_1, x_2, x_3, x_5, x_6, x_7$ vanish, so the factors $e^{\mp a_4}$ of the field equation multiply zero: **the solution holds for every history** a_4. With $\partial_0 = 6H\,\partial_z$ (chain rule, $z = 6Hx_0$) the equation becomes

$$E_m[\Psi] \equiv \gamma^{(x4)}\partial_4\Psi + 6H\tan z\,\gamma^{(x0)}\partial_z\Psi + 3H\gamma^{(x0)}\Psi - m\Psi = 0.$$

Step 5. $\gamma^{(x4)}\partial_4\Psi = \gamma^{(x4)}M_m\Psi = \big(-m\gamma^{(x4)}\gamma^{(x4)} + b\gamma^{(x4)}\gamma^{(x4)}\gamma^{(x0)}\big)\Psi = (m - b\gamma^{(x0)})\Psi$ (step 3, then $\gamma^{(x4)}\gamma^{(x4)} = -1$).

Step 6. $\partial_z \sin^{\alpha} z = \alpha \sin^{\alpha-1} z \cos z$, so $\partial_z\Psi = \alpha\cot z\,\Psi$ and $6H\tan z\,\gamma^{(x0)}\partial_z\Psi = 6H\alpha\,\gamma^{(x0)}\Psi$ ($\tan z \cot z = 1$).

Step 7. Adding: $E_m[\Psi] = \big(m - b\gamma^{(x0)} + 6H\alpha\gamma^{(x0)} + 3H\gamma^{(x0)} - m\big)\Psi = \big(3H(2\alpha+1) - b\big)\gamma^{(x0)}\Psi = 0$, because $b = 3H(2\alpha+1)$.

The record checks this exactly (check `exact_solution_family_x4_x0` of `Revision/theory/reports/python-field-theory.json`), and Notebook 21a checks it again with sympy (In [11]).

The numbers chosen. $H = 1/6$ (so $z = x_0$), $\alpha = 1$ and $m = 2$. Then $b = 3 \cdot \frac{1}{6} \cdot 3 = \frac{3}{2}$ and $k^2 = \frac{9}{4} - 4 = -\frac{7}{4} < 0$. So $k = iw$ with $w = \sqrt{7}/2$ is imaginary; with $\cosh(iwx_4) = \cos(wx_4)$ and $\sinh(iwx_4)/(iw) = \sin(wx_4)/w$ the solution oscillates in time, and $U = \cos(wx_4) + \sin(wx_4)M_m/w$ is a **real** matrix.

The two conjugates. Since U and $\sin z$ are real, $\Psi^* = \sin z\, U\chi^*$: the same formula with the column χ^*, so Ψ^* solves the equation with the **same** mass $+2$. For the other, $\Gamma M_m \Gamma = -m\Gamma\gamma^{(x4)}\Gamma + b\Gamma\gamma^{(x4)}\gamma^{(x0)}\Gamma = m\gamma^{(x4)} + b\gamma^{(x4)}\gamma^{(x0)} = M_{-m}$ (R1 for the single gamma, R2 for the pair), and k^2 is the same for $\pm m$; so $\Gamma U_m \Gamma = U_{-m}$ and $\Gamma\Psi^* = \sin z\, U_{-m}(\Gamma\chi^*)$: the solution formula with the mass -2. $\Gamma\Psi^*$ **solves the equation with the reversed mass** -2**.** The negative controls fail: $E_{-2}[\Psi^*] = E_{+2}[\Psi^*] + 4\Psi^* = 4\Psi^*$ (the last term of E_m changes by $-(-2)\Psi^* + 2\Psi^*$), so its size is four times the size of Ψ^*, about 10; the notebook measures at least 9.67 along the time (In [13]). By the table of Section 21.10, Ψ^* carries the charge density $-J^{(x4)}$ and $\Gamma\Psi^*$ carries $+J^{(x4)}$ (figure 21a.5).

statement	**status**	**where it is verified**
the record states the field equation and the exact solution (i) as used here	checked word for word	`Revision/theory/field-theory.json`, formulas `field_equation`, `exact_solutions`; Notebook 21a, In [11]
the solution solves the field equation for every a_4	PROVED (above; sympy)	`Revision/theory/reports/python-field-theory.json`, check `exact_solution_family_x4_x0`; Notebook 21a, In [11]
Ψ^* solves with $+2$, $\Gamma\Psi^*$ with -2; the exchanged masses fail	PROVED (sympy, exact); COMPUTED along x_4 (size below 10^{-12} and exactly four times the field)	Notebook 21a, In [12] and In [13]
Ψ^* has $-J^{(x4)}$, $\Gamma\Psi^*$ has $+J^{(x4)}$ at every time	COMPUTED (161 times, to rounding)	Notebook 21a, In [13]

21.12 Example: Notebook 21a solves for every charge-conjugation matrix

Notebook 21a turns Sections 21.7 to 21.11 into exact computations on the author's gammas. It reads the gammas from `Revision/algebra/gammas.json` and the lead report `Revision/lead_checks/reports/charge-conjugation-and-u1.json`; it checks the eight real gammas and draws them; it builds C, Γ, B and the S^{ab}; it writes the conditions $M(\gamma^a)^* = s\gamma^a M$ as two systems of 2048 linear equations for the 256 entries of M and solves them exactly over the rational numbers; it watches each equation halve the solution space; it builds $\mathcal{C}_+$ and $\mathcal{C}_-$, checks their transposition rules and solves the transposition equations; it checks the two reality conditions; it reads the field equation and the exact solution from the field-theory record word for word and applies both conjugations to the solution; it computes the signs of all 256 bilinears; and it treats real fields. Nine of its checks reproduce checks of the lead report, two reproduce the field-theory record. It runs in about 30 seconds; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/21a_conjugation_matrices.PROVENANCE.md`. It needs no Rust, prints 27 PASS lines and draws seven figures.

21.13 How to run Notebook 21a

Step 1. What this notebook does and what it needs.

Notebook 21a (Charge conjugation is a matrix: calC_+ = C and calC_- = Gamma C) is the file `Revision/textbook/notebooks/21a_conjugation_matrices.ipynb` of the repository Dirac_claude. It checks that the eight gamma matrices of the Revision record are real 16 by 16 matrices that obey the Clifford relation of signature (4,4), solves exactly the linear equations M conj(gamma^a) = s gamma^a M for all 16 by 16 matrices M (conj is the complex conjugate; s = +1 keeps the mass, s = -1 reverses it) and finds two one-dimensional solution spaces spanned by 1 and by the chirality Gamma, watches each of the eight equations halve the solution space, builds the charge-conjugation matrices calC_+ = C and calC_- = Gamma C with their transposition rules and reality conditions, applies both conjugations to an exact solution of the field equation in the author's curved metric (valid for every history a4, the deflating one included), tabulates how all 256 bilinears change sign for commuting and for anticommuting components, and shows that for a REAL field calC_+ is the identity while the real matrix Gamma reverses the mass. Seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 21a_conjugation_matrices.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21a_conjugation_matrices.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 21a_conjugation_matrices.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/21a.captions.json`
- `Revision/textbook/figures/21a_1_eight_gammas.png`
- `Revision/textbook/figures/21a_2_halving_solutions.png`
- `Revision/textbook/figures/21a_3_conjugation_matrices.png`
- `Revision/textbook/figures/21a_4_reality_conditions.png`
- `Revision/textbook/figures/21a_5_curved_solution_images.png`
- `Revision/textbook/figures/21a_6_bilinear_signs.png`
- `Revision/textbook/figures/21a_7_real_fields.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the seven figure files of notebook 21a exist
ALL 27 CHECKS PASSED (notebook 21a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/21a_conjugation_matrices.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 21a_conjugation_matrices.ipynb
python -m nbconvert --execute --inplace 21a_conjugation_matrices.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra`, `Revision/theory` or `Revision/lead_checks`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

21.14 Notebook 21a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/21a_conjugation_matrices.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

**Charge conjugation is a matrix.** In this theory every particle field $\Psi$ is a
column of 16 numbers, and the field equation contains eight $16 \times 16$ gamma
matrices. The author's gamma matrices are *real*: every entry is $-1$, $0$ or
$+1$. For a *real* field ($\Psi^* = \Psi$) plain complex conjugation changes
nothing at all: it is the identity map, and it cannot turn matter into
antimatter. Charge conjugation must therefore be built with a **matrix**
$\mathcal{C}$:

$$\Psi^c = \mathcal{C}\,\bar\Psi^T = \mathcal{C}\,C\,\Psi^* ,$$

where $\bar\Psi = \Psi^\dagger C$ is the Dirac adjoint and $C$ the charge matrix
(the author's sigma16). This notebook finds, exactly, every matrix that obeys the
intertwiner condition of Section 4 (the definition of a charge-conjugation matrix
used by the Revision record). It

1. checks that the Revision record holds **eight real $16 \times 16$ gamma
   matrices** that obey the Clifford relation of signature (4,4), and draws them;
2. solves the linear equations $M(\gamma^a)^* = s\,\gamma^a M$
   ($a = x0, \dots, x7$) for all $16 \times 16$ matrices $M = \mathcal{C}C$: for
   $s = +1$ (same mass) the solutions are the multiples of $1$, for $s = -1$ (mass
   reversed) the multiples of the chirality $\Gamma$; it also shows how each of
   the eight equations halves the space of solutions;
3. builds the two charge-conjugation matrices $\mathcal{C}_+ = C$ and
   $\mathcal{C}_- = \Gamma C$, checks their transposition rules and their reality
   (Majorana) conditions;
4. applies both conjugations to an **exact solution of the field equation in the
   author's curved metric** and shows which mass each image solves the equation
   with;
5. computes how all 256 bilinears $\bar\Psi\,\Gamma_A\Psi$ change sign, for
   commuting and for anticommuting components;
6. treats **real fields**: the current vanishes, $\mathcal{C}_+$ is the identity,
   and, among the matrices that obey the intertwiner condition, the nontrivial
   real map is the matrix $\Gamma$ with the mass reversed.

Nine checks reproduce checks of the Revision record
`Revision/lead_checks/reports/charge-conjugation-and-u1.json`; two more read the
field equation and its exact solution from the Revision field-theory record
`Revision/theory/field-theory.json` and reproduce the check
exact_solution_family_x4_x0 of `Revision/theory/reports/python-field-theory.json`.
Seven teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 21a (Charge conjugation is a matrix: calC_+ = C and calC_- = Gamma C) is the
file ‘Revision/textbook/notebooks/21a_conjugation_matrices.ipynb‘ of the repository
Dirac_claude. It checks that the eight gamma matrices of the Revision record are real 16
by 16 matrices that obey the Clifford relation of signature (4,4), solves exactly the
linear equations M conj(gamma^a) = s gamma^a M for all 16 by 16 matrices M (conj is the
complex conjugate; s = +1 keeps the mass, s = -1 reverses it) and finds two
one-dimensional solution spaces spanned by 1 and by the chirality Gamma, watches each of
the eight equations halve the solution space, builds the charge-conjugation matrices
calC_+ = C and calC_- = Gamma C with their transposition rules and reality conditions,
applies both conjugations to an exact solution of the field equation in the author’s
curved metric (valid for every history a4, the deflating one included), tabulates how all
256 bilinears change sign for commuting and for anticommuting components, and shows that
for a REAL field calC_+ is the identity while the real matrix Gamma reverses the mass.
Seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet
connection for the installation, the program Git, and Python 3.12 or newer (the notebooks
were built with Python 3.14.5) with these packages at exactly these versions: numpy
2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4,
nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy,
sympy and matplotlib; the other packages run Jupyter, the program that shows and runs
notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:

Linux (bash):

    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
```

```
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 21a_conjugation_matrices.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step

"What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21a_conjugation_matrices.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 21a_conjugation_matrices.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/21a.captions.json‘
- ‘Revision/textbook/figures/21a_1_eight_gammas.png‘
- ‘Revision/textbook/figures/21a_2_halving_solutions.png‘
- ‘Revision/textbook/figures/21a_3_conjugation_matrices.png‘
- ‘Revision/textbook/figures/21a_4_reality_conditions.png‘
- ‘Revision/textbook/figures/21a_5_curved_solution_images.png‘
- ‘Revision/textbook/figures/21a_6_bilinear_signs.png‘
- ‘Revision/textbook/figures/21a_7_real_fields.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the seven figure files of notebook 21a exist
ALL 27 CHECKS PASSED (notebook 21a)
```

and the notebook must show 7 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/21a_conjugation_matrices.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 21a_conjugation_matrices.ipynb
      python -m nbconvert --execute --inplace 21a_conjugation_matrices.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘, ‘Revision/theory‘ or ‘Revision/lead_checks‘: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 21a (the complete instructions)
```

```
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 21a (Charge conjugation is a matrix: calC_+ = C and calC_- = Gamma C) is the
# file Revision/textbook/notebooks/21a_conjugation_matrices.ipynb of the repository
# Dirac_claude. It checks that the eight gamma matrices of the Revision record are real
# 16 by 16 matrices that obey the Clifford relation of signature (4,4), solves exactly
# the linear equations M conj(gamma^a) = s gamma^a M for all 16 by 16 matrices M (conj is
# the complex conjugate; s = +1 keeps the mass, s = -1 reverses it) and finds two
# one-dimensional solution spaces spanned by 1 and by the chirality Gamma, watches each
# of the eight equations halve the solution space, builds the charge-conjugation matrices
# calC_+ = C and calC_- = Gamma C with their transposition rules and reality conditions,
# applies both conjugations to an exact solution of the field equation in the author's
# curved metric (valid for every history a4, the deflating one included), tabulates how
# all 256 bilinears change sign for commuting and for anticommuting components, and shows
# that for a REAL field calC_+ is the identity while the real matrix Gamma reverses the
# mass. Seven teaching plots. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter,
# the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
```

```
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
```

```
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 21a_conjugation_matrices.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 21a_conjugation_matrices.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
```

```
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 21a_conjugation_matrices.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/21a.captions.json
# - Revision/textbook/figures/21a_1_eight_gammas.png
# - Revision/textbook/figures/21a_2_halving_solutions.png
# - Revision/textbook/figures/21a_3_conjugation_matrices.png
# - Revision/textbook/figures/21a_4_reality_conditions.png
# - Revision/textbook/figures/21a_5_curved_solution_images.png
# - Revision/textbook/figures/21a_6_bilinear_signs.png
# - Revision/textbook/figures/21a_7_real_fields.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the seven figure files of notebook 21a exist
#     ALL 27 CHECKS PASSED (notebook 21a)
#
# and the notebook must show 7 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/21a_conjugation_matrices.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
```

```
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 21a_conjugation_matrices.ipynb
#     python -m nbconvert --execute --inplace 21a_conjugation_matrices.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra, Revision/theory or
#   Revision/lead_checks: the notebook was opened outside the repository, or the
#   repository is incomplete; clone the repository again and open the notebook from its
#   folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "21a"  # this notebook: chapter 21, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
```

```
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
```

```
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 21a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Matrix**: a rectangular table of numbers; a $16 \times 16$ matrix has 16 rows
  and 16 columns. The product $AB$ of two matrices has the entry $\sum_k A_{rk}
  B_{kc}$ in row $r$ and column $c$. The **identity** $1$ has ones on the diagonal
  and zeros elsewhere: $1A = A1 = A$.
- **Real, complex conjugate**: $z^* = a - ib$ for $z = a + ib$; a matrix or a
```

```
  column is *real* when all its entries are real, so that $M^* = M$.
- **Transpose** $M^T$: rows and columns exchanged, $(M^T)_{rc} = M_{cr}$;
  **symmetric**: $M^T = M$; **antisymmetric**: $M^T = -M$; **conjugate
  transpose** $M^\dagger = (M^*)^T$; **Hermitian**: $M^\dagger = M$.
- **Spinor field** $\Psi$: at every point a column of 16 numbers (components).
  The author's coordinates are $x1, x2, x3$ (ordinary space), $x4$ (the time),
  $x5, x6, x7$ (the three extra times, which deflate exponentially) and $x0$ (the
  hidden direction).
- **Gamma matrices** $\gamma^{(x0)}, \dots, \gamma^{(x7)}$: eight fixed
  $16 \times 16$ matrices with the **Clifford relation** $\gamma^a\gamma^b +
  \gamma^b\gamma^a = 2\eta^{ab}\,1$, where $\eta = \mathrm{diag}(+1, +1, +1, +1, -1,
  -1, -1, -1)$ in the order $x0, \dots, x7$ (**signature (4,4)**: four plus signs,
  four minus signs).
- **Charge matrix** $C = \gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$
  (the product of the four space-like gammas); **Dirac adjoint**
  $\bar\Psi = \Psi^\dagger C$; **chirality** $\Gamma = \gamma^{(x0)}\gamma^{(x1)}
  \cdots\gamma^{(x7)}$ (all eight); **Krein matrix** $B = -iC\gamma^{(x4)}$.
- **Field equation**: $(\gamma^\mu D_\mu - V)\Psi = 0$ with $V = m + U'(S)$ real,
  $m$ the **mass**, $S = \bar\Psi\Psi$ the **scalar**; $D_\mu$ is the derivative
  with the gravitational spin connection.
- **Charge-conjugation matrix** $\mathcal{C}$: a constant matrix for which
  $M = \mathcal{C}C$ obeys the **intertwiner condition**
  $M(\gamma^a)^* = s\,\gamma^a M$ for all eight $a$, with one sign $s$ ($s = +1$
  same mass, $s = -1$ mass reversed); this is the definition of the Revision
  record. Then $\Psi^c = \mathcal{C}\bar\Psi^T$ solves a field equation of the
  same form whenever $\Psi$ does (Section 4); the converse is not true in
  general (Section 4).
- **Solution space, dimension**: all solutions of a set of linear equations form a
  space; its *dimension* is the number of free numbers in the general solution.
- **Bilinear**: $\Psi^\dagger K\Psi = \sum_{r,c}\Psi_r^* K_{rc}\Psi_c$ for a fixed
  matrix $K$; the scalar $S = \Psi^\dagger C\Psi$ and the **current**
  $J^a = -i\bar\Psi\gamma^a\Psi$ are bilinears.
- **Commuting, anticommuting components**: the components of dirac16complex00 are
  ordinary numbers ($\Psi_r\Psi_c = \Psi_c\Psi_r$); those of dirac16complex are
  anticommuting (Grassmann) numbers ($\Psi_r\Psi_c = -\Psi_c\Psi_r$). We write
  $\epsilon = +1$ and $\epsilon = -1$ for the two cases.
- **Majorana (reality) condition**: the requirement $\Psi^c = \Psi$.
```

Text cell:

```
## 4. The physical and mathematical situation

**Why a matrix is needed.** Antimatter is described by the *charge conjugate*
$\Psi^c$ of a field: a field built from $\Psi$ that solves the field equation
again and carries the opposite charge. In Dirac's theory of the electron, whose
gamma matrices are complex, $\Psi^c$ involves the complex conjugate $\Psi^*$
together with a matrix. The author's gammas are real; then for a real field
$\Psi^* = \Psi$, and complex conjugation alone does nothing. The matrix is
essential. We derive a condition on the matrix that is enough for $\Psi^c$ to be
a solution, and then we find all matrices that obey the condition below.

**Deriving the condition on the matrix**, line by line. Let $\Psi$ solve
$\gamma^\mu D_\mu\Psi = V\Psi$ with real $V$. In the author's metric the gammas
```

$\gamma^\mu = e^\mu{}_a\gamma^a$ and the spin connection Ω_μ in
$D_\mu = \partial_\mu + \Omega_\mu$ are real (Revision lead check
spinor_connection_real), so the operator $\gamma^\mu D_\mu$ is real.

1. Take the complex conjugate of the equation. Every real factor stays as it is,
   so $\gamma^\mu D_\mu\Psi^* = V\Psi^*$.
2. Multiply from the left by a constant matrix M:
   $M\gamma^\mu D_\mu\Psi^* = V\,M\Psi^*$.
3. Suppose $M\gamma^a = s\,\gamma^a M$ for every a, with one sign $s = \pm1$.
   The spin connection is a sum of products $\gamma^a\gamma^b$, and
   $M\gamma^a\gamma^b = s\gamma^a M\gamma^b = s^2\gamma^a\gamma^b M =
   \gamma^a\gamma^b M$, so M commutes with Ω_μ and $MD_\mu = D_\mu M$.
4. Hence $M\gamma^\mu D_\mu\Psi^* = s\,\gamma^\mu D_\mu(M\Psi^*)$, and step 2
   becomes $s\,\gamma^\mu D_\mu(M\Psi^*) = V\,(M\Psi^*)$. Multiplying by s (and
   using $s^2 = 1$):
   $$\gamma^\mu D_\mu(M\Psi^*) = s\,V\,(M\Psi^*) .$$
   So $M\Psi^*$ solves the equation with the same V when $s = +1$ and with
   $-V$ (the mass reversed) when $s = -1$.

Because the gammas are real, $(\gamma^a)^* = \gamma^a$ and the condition is
$M(\gamma^a)^* = s\,\gamma^a M$: it is enough that M commutes ($s = +1$) or
anticommutes ($s = -1$) with all eight gammas (the converse fails for $V = 0$,
see below). The textbook form of charge conjugation is
$\Psi^c = \mathcal{C}\bar\Psi^T$; since $\bar\Psi^T = (\Psi^\dagger C)^T =
C^T\Psi^* = C\Psi^*$ (C is symmetric), $\Psi^c = \mathcal{C}C\Psi^*$, so
$M = \mathcal{C}C$ and, because $CC = 1$, $\mathcal{C} = MC$.

The definition. A **charge-conjugation matrix** is a constant matrix
$\mathcal{C}$ for which $M = \mathcal{C}C$ obeys this **intertwiner condition**
$M(\gamma^a)^* = s\,\gamma^a M$ with one sign s for all eight a. This is the
definition of the Revision record: its checks intertwiners_same_mass and
intertwiners_reversed_mass solve exactly this condition. Steps 1 to 4 show that
the condition is enough for $M\Psi^*$ to be a solution. The converse is not true
in general: for $V = 0$ (mass 0, no self-interaction) the chiral projector
$P_- = \frac12(1 - \Gamma)$ also maps every solution to a solution without
obeying the condition for either sign (Notebook 05c). The count below therefore
concerns the matrices that obey the condition.

What to expect. The matrices that commute with all eight gammas are the
multiples of 1, because the 16 components carry an *irreducible*
representation of Pin(4,4). If M anticommutes with every gamma, then ΓM
commutes with every gamma (since Γ anticommutes with every gamma), so
$\Gamma M = c\,1$ and $M = c\,\Gamma$. The notebook solves the 4096 equations
exactly and finds exactly these two families.

Text cell:

5. The eight real gamma matrices of the Revision record

The next cell reads the Revision record ‘Revision/algebra/gammas.json‘, which holds
the author’s eight gamma matrices (rebuilt in Revision code from the author’s
formulas) in the order $x0, \dots, x7$, and the frame metric η. It stores
each gamma as a numpy array of whole numbers, defines a helper for products of

```
gammas, and reads the Revision record of the lead checks, whose verdicts the
notebook will reproduce.
```

In [2]:

```
import itertools  # all subsets of a list (for the 256 products of gammas)

import numpy as np  # arrays of numbers, matrices and linear algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # the list "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
# gamma["x4"] is gamma^(x4), a 16 x 16 array of whole numbers (int64)
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1


def product(directions):
    """The matrix product gamma^(d1) gamma^(d2) ... in the order of the list."""
    result = I16
    for d in directions:
        result = result @ gamma[d]  # @ is the matrix product
    return result


RECORD = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
lead = json.loads(repository_file(RECORD).read_text(encoding="utf-8"))
LEAD = {c["name"]: c for c in lead["checks"]}  # check name -> its entry


def lead_passed(name):
    """True when the Revision record has the check name with the verdict PASS."""
    return LEAD.get(name, {}).get("verdict") == "PASS"


say(f"coordinates: {' '.join(COORDS)}")
say("eta: " + " ".join(f"{ETA[x]:+d}" for x in COORDS))
say(f"lead record: {lead['summary']['passed']} of {lead['summary']['total']} "
    "checks passed")
```

Out [2]:

```
coordinates: x0 x1 x2 x3 x4 x5 x6 x7
eta: +1 +1 +1 +1 -1 -1 -1 -1
lead record: 12 of 12 checks passed
```

Text cell:

```
The next cell checks the user's central requirement: that the calculation uses
**eight real-valued $16 \times 16$ Dirac matrices**. It checks that there are
```

```
eight of them, that each has 16 rows and 16 columns, that every entry is $-1$,
$0$ or $+1$ (so each is real), that each row and each column holds exactly one
nonzero entry (a *signed permutation matrix*), and that they obey the Clifford
relation $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab}1$ for all 64 pairs
$(a, b)$. It also checks the symmetry pattern $(\gamma^a)^T = \eta_{aa}\gamma^a$:
the four space-like gammas are symmetric, the four time-like ones antisymmetric.
```

In [3]:

```
shapes_ok = len(gamma) == 8 and all(g.shape == (16, 16) for g in gamma.values())
entries_ok = all(set(np.unique(g)) <= {-1, 0, 1} for g in gamma.values())
# a signed permutation matrix: exactly one nonzero entry in every row and column
permutation_ok = all((np.abs(g).sum(axis=0) == 1).all()
                     and (np.abs(g).sum(axis=1) == 1).all() for g in gamma.values())
check(shapes_ok and entries_ok and permutation_ok,
      "eight real 16 x 16 gamma matrices, entries -1, 0, +1, signed permutations")
clifford_ok = all(
    np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                   2 * (ETA[a] if a == b else 0) * I16)
    for a in COORDS for b in COORDS)  # all 64 ordered pairs (a, b)
check(clifford_ok, "Clifford relation gamma^a gamma^b + gamma^b gamma^a = "
      "2 eta^ab 1 for all 64 pairs, signature (4,4)")
check(all(np.array_equal(gamma[x].T, ETA[x] * gamma[x]) for x in COORDS),
      "symmetry pattern (gamma^a)^T = eta_aa gamma^a")
```

Out [3]:

```
PASS eight real 16 x 16 gamma matrices, entries -1, 0, +1, signed permutations
PASS Clifford relation gamma^a gamma^b + gamma^b gamma^a = 2 eta^ab 1 for all 64 pairs,
    signature (4,4)
PASS symmetry pattern (gamma^a)^T = eta_aa gamma^a
```

Text cell:

```
The next cell draws the eight matrices as heat maps: a red square is an entry
$+1$, a blue square $-1$, a pale square $0$. The black lines split each matrix
into four $8 \times 8$ blocks: every gamma has its nonzero entries only in the two
off-diagonal blocks (it exchanges the two halves of the spinor, which will turn out
to be the two chiral halves).
```

In [4]:

```
from matplotlib.colors import LinearSegmentedColormap

# blue for -1, almost white for 0, red for +1
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])


def heat_map(ax, matrix, title, row_label=True):
```

```
    """Draw a 16 x 16 matrix with entries from -1 to +1 (blue -1, pale 0, red +1).
    Rows and columns are numbered from 1; black lines separate the two halves."""
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)
    return image


fig, axes = plt.subplots(2, 4, figsize=(12.0, 6.6))
for k, (ax, x) in enumerate(zip(axes.flat, COORDS)):
    kind = "space-like" if ETA[x] == 1 else "time-like"
    image = heat_map(ax, gamma[x], f"$\\gamma^{{({x})}}$, {kind}",
                     row_label=(k % 4 == 0))
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.7, label="matrix entry")
save_figure(fig, "eight_gammas",
            "The eight real $16 \\times 16$ gamma matrices of the Revision record, "
            "in the author's coordinate order $x1, x2, x3$ (ordinary space), $x4$ "
            "(the time), $x5, x6, x7$ (the deflating extra times), $x0$ (the hidden "
            "direction); horizontal axis the column, vertical axis the row, colour "
            "the entry (red $+1$, blue $-1$, pale $0$). Every matrix has exactly one "
            "nonzero entry in each row and column, and only in the two off-diagonal "
            "$8 \\times 8$ blocks: each gamma exchanges the two halves of the "
            "spinor.")
```

Out [4]:

```
Figure 21a.1 saved as Revision/textbook/figures/21a_1_eight_gammas.png
```

Text cell:

```
## 6. C, the chirality Gamma, B and the generators S^ab

The next cell builds, from the gammas, the charge matrix $C = \gamma^{(x0)}
\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$, the chirality $\Gamma = \gamma^{(x0)}
\gamma^{(x1)}\cdots\gamma^{(x7)}$, the Krein matrix $B = -iC\gamma^{(x4)}$ (`1j` is
Python's imaginary unit $i$) and the 28 generators $S^{ab} = \tfrac14[\gamma^a,
\gamma^b]$ of the rotations and boosts. It compares $C$ and $\Gamma$ with the
matrices stored in the record and checks: $C$ is real and symmetric with $C^2 =
1$; $\Gamma = \mathrm{diag}(-1, \dots, -1, +1, \dots, +1)$ (eight of each); all
$S^{ab}$ are real (record check representation_real); $B$ is purely imaginary,
Hermitian and $B^2 = 1$ (record check B_imaginary_hermitian).
```

In [5]:

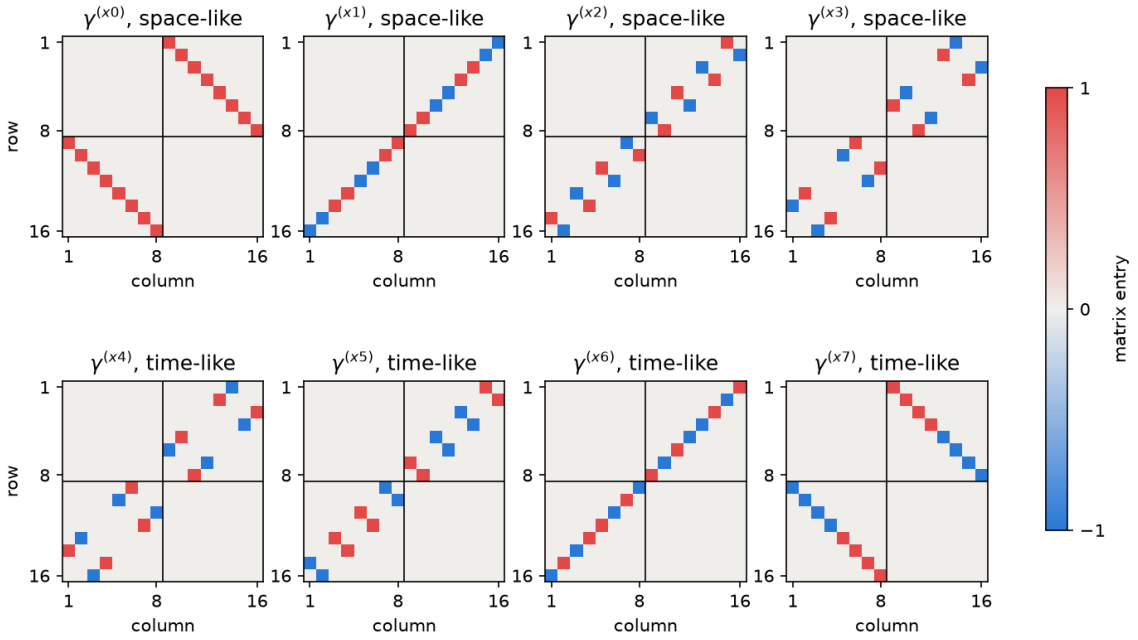

Figure 522: The eight real 16×16 gamma matrices of the Revision record, in the author's coordinate order $x1, x2, x3$ (ordinary space), $x4$ (the time), $x5, x6, x7$ (the deflating extra times), $x0$ (the hidden direction); horizontal axis the column, vertical axis the row, colour the entry (red $+1$, blue -1, pale 0). Every matrix has exactly one nonzero entry in each row and column, and only in the two off-diagonal 8×8 blocks: each gamma exchanges the two halves of the spinor. (Notebook 21a, figure 1.)

```
C = product(["x0", "x1", "x2", "x3"])  # the charge matrix (the author's sigma16)
Gamma = product(["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"])  # the chirality
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4), a complex matrix
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 pairs
S_gen = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}

same_as_record = (np.array_equal(C, np.array(fixture["C"]))
                  and np.array_equal(Gamma, np.array(fixture["Gamma"])))
gamma_diag = np.diag(np.concatenate([-np.ones(8), np.ones(8)])).astype(np.int64)
check(same_as_record and np.array_equal(Gamma, gamma_diag),
      "C and Gamma equal the record; Gamma = diag(-1 (8 times), +1 (8 times))")
check(np.array_equal(C.T, C) and np.array_equal(C @ C, I16)
      and all(np.isrealobj(s) for s in S_gen.values()) and lead_passed(
          "representation_real"),
      "gamma^a, C (symmetric, C^2 = 1) and all 28 S^ab are real",
      record=f"{RECORD}, check representation_real")
check(np.array_equal(B.real, np.zeros((16, 16))) and np.allclose(B, B.conj().T)
      and np.allclose(B @ B, np.eye(16)) and lead_passed("B_imaginary_hermitian"),
      "B = -i C gamma^(x4) is purely imaginary, Hermitian, B^2 = 1",
      record=f"{RECORD}, check B_imaginary_hermitian")
```

Out [5]:

```
PASS C and Gamma equal the record; Gamma = diag(-1 (8 times), +1 (8 times))
PASS gamma^a, C (symmetric, C^2 = 1) and all 28 S^ab are real
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    representation_real
PASS B = -i C gamma^(x4) is purely imaginary, Hermitian, B^2 = 1
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    B_imaginary_hermitian
```

Text cell:

```
## 7. Every matrix that obeys the condition, solved exactly

Write the unknown matrix $M$ as its 256 entries $M_{rc}$. For one gamma the
condition $M(\gamma^a)^* - s\,\gamma^a M = 0$ is a $16 \times 16$ matrix of
zeros, that is, 256 linear equations in the 256 unknowns; for all eight gammas
2048 equations. (For both signs together: 4096.) Reading the entries of $M$ row
by row as one long column $y$, the product $MG$ becomes $(1 \otimes G^T)\,y$ and
$GM$ becomes $(G \otimes 1)\,y$, where $\otimes$ is the Kronecker product
(`np.kron`): the block matrix whose block $(r, c)$ is the entry $A_{rc}$ times the
whole matrix $B$. The next cell builds the coefficient matrix of these equations
and solves them **exactly**, with rational numbers (sympy's `DomainMatrix` over
the rationals `QQ`), so no rounding can hide or invent a solution.
```

In [6]:

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over the rationals
```

```
def conjugation_system(s, directions=COORDS):
    """Coefficient matrix of M conj(gamma^a) - s gamma^a M = 0 for the 256
    unknowns M_rc (read row by row), one block of 256 equations per direction."""
    blocks = []
    for x in directions:
        G_conj = np.conj(gamma[x]).astype(np.int64)  # (gamma^a)*, equal to gamma^a
        blocks.append(np.kron(I16, G_conj.T) - s * np.kron(gamma[x], I16))
    return np.vstack(blocks)


def exact_solutions(A):
    """A basis of all solutions y of A y = 0, each returned as a 16 x 16 matrix."""
    null = DomainMatrix.from_list(A.tolist(), sp.QQ).nullspace().to_Matrix()
    return [np.array(null.row(k).tolist()[0], dtype=object).reshape(16, 16)
            for k in range(null.rows)]


def proportional(X, Y):
    """True when X and Y are nonzero and X is a number times Y."""
    X = np.array(X, dtype=float).reshape(-1)
    Y = np.array(Y, dtype=float).reshape(-1)
    return bool(X.any() and Y.any() and np.linalg.matrix_rank(np.array([X, Y])) == 1)


A_same, A_reversed = conjugation_system(+1), conjugation_system(-1)
M_same, M_reversed = exact_solutions(A_same), exact_solutions(A_reversed)
say(f"s = +1: {A_same.shape[0]} equations, {len(M_same)} independent solution(s)")
say(f"s = -1: {A_reversed.shape[0]} equations, {len(M_reversed)} independent "
    "solution(s)")
check(len(M_same) == 1 and proportional(M_same[0], I16)
      and lead_passed("intertwiners_same_mass"),
      "s = +1 (same mass): the solutions are the multiples of the identity",
      record=f"{RECORD}, check intertwiners_same_mass")
check(len(M_reversed) == 1 and proportional(M_reversed[0], Gamma)
      and lead_passed("intertwiners_reversed_mass"),
      "s = -1 (mass reversed): the solutions are the multiples of Gamma",
      record=f"{RECORD}, check intertwiners_reversed_mass")
```

Out [6]:

```
s = +1: 2048 equations, 1 independent solution(s)
s = -1: 2048 equations, 1 independent solution(s)
PASS s = +1 (same mass): the solutions are the multiples of the identity
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    intertwiners_same_mass
PASS s = -1 (mass reversed): the solutions are the multiples of Gamma
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    intertwiners_reversed_mass
```

Text cell:

```
**Each equation halves the solution space.** Imposing the condition for only the
first $k$ gammas ($k = 0, 1, \dots, 8$) leaves more solutions. The next cell
computes the dimension of the solution space for each $k$ exactly, as $256$ minus
the *rank* of the coefficient matrix (the number of independent equations among
them), and plots it. Why halving: a matrix $M$ that commutes with $\gamma^{(x1)}$
maps each of its two eigenspaces (dimension 8 each, eigenvalues $\pm1$) into
itself, which leaves $8 \cdot 8 + 8 \cdot 8 = 128$ free entries of the 256; each
further gamma cuts the freedom in half again, down to $256/2^8 = 1$.
```

In [7]:

```
dimensions = {}
for s in (+1, -1):
    dims = [256]  # k = 0: no equation, every matrix is a solution
    for k in range(1, 9):
        A = conjugation_system(s, COORDS[:k])  # the first k directions only
        rank = DomainMatrix.from_list(A.tolist(), sp.ZZ).rank()  # exact rank
        dims.append(256 - rank)
    dimensions[s] = dims
    say(f"s = {s:+d}: dimensions for k = 0..8: {dims}")
check(dimensions[+1] == dimensions[-1] == [256 // 2 ** k for k in range(9)],
      "each gamma equation halves the solution space: 256, 128, ..., 2, 1")

fig, ax = plt.subplots(figsize=(7.5, 4.4))
k_values = np.arange(9)
ax.semilogy(k_values, dimensions[+1], "o-", markersize=9,
            label="$s = +1$: $M$ commutes with the first $k$ gammas")
ax.semilogy(k_values, dimensions[-1], "s--", markersize=5,
            label="$s = -1$: $M$ anticommutes with the first $k$ gammas")
ax.set_xticks(k_values, ["0"] + [f"{k}\n(+{x})" for k, x in zip(range(1, 9), COORDS)])
ax.set_yticks([1, 2, 4, 8, 16, 32, 64, 128, 256],
              ["1", "2", "4", "8", "16", "32", "64", "128", "256"])
ax.set_xlabel("number $k$ of gamma equations imposed (and the direction added)")
ax.set_ylabel("dimension of the solution space")
ax.set_title("Every equation halves the space of conjugation matrices")
ax.legend()
save_figure(fig, "halving_solutions",
            "The dimension of the space of $16 \\times 16$ matrices $M$ that obey "
            "$M(\\gamma^a)^\\ast = s\\,\\gamma^a M$ for the first $k$ directions only "
            "(computed exactly as $256$ minus the rank of the equations), for "
            "$s = +1$ (circles) and $s = -1$ (squares); horizontal axis $k$ with "
            "the direction added, vertical axis the dimension on a logarithmic "
            "scale. Both sequences are $256/2^k$: every new gamma halves the "
            "freedom, and with all eight only one direction is left, the multiples "
            "of $1$ ($s = +1$) or of $\\Gamma$ ($s = -1$).")
```

Out [7]:

```
s = +1: dimensions for k = 0..8: [256, 128, 64, 32, 16, 8, 4, 2, 1]
s = -1: dimensions for k = 0..8: [256, 128, 64, 32, 16, 8, 4, 2, 1]
PASS each gamma equation halves the solution space: 256, 128, ..., 2, 1
```

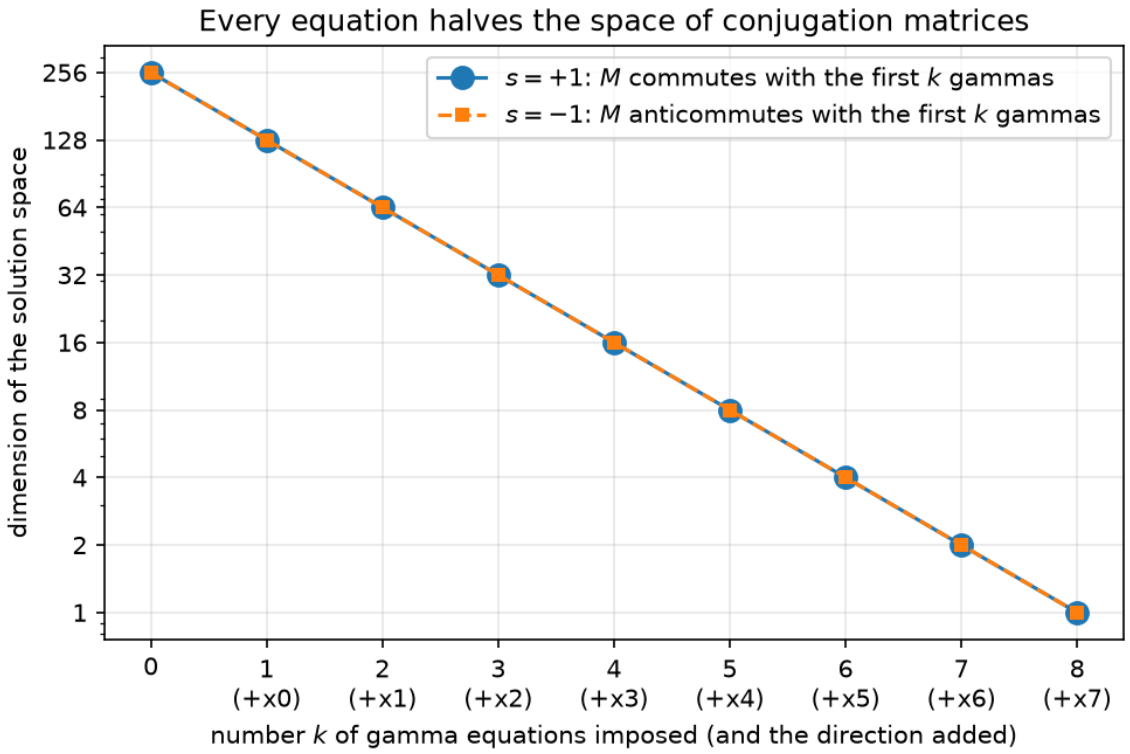

Figure 523: The dimension of the space of 16×16 matrices M that obey $M(\gamma^a)^* = s\,\gamma^a M$ for the first k directions only (computed exactly as 256 minus the rank of the equations), for $s = +1$ (circles) and $s = -1$ (squares); horizontal axis k with the direction added, vertical axis the dimension on a logarithmic scale. Both sequences are $256/2^k$: every new gamma halves the freedom, and with all eight only one direction is left, the multiples of 1 ($s = +1$) or of Γ ($s = -1$). (Notebook 21a, figure 2.)

Out [7]:

```
Figure 21a.2 saved as Revision/textbook/figures/21a_2_halving_solutions.png
```

Text cell:

```
## 8. The two charge-conjugation matrices

From $M = 1$ and $M = \Gamma$ we get $\mathcal{C} = MC$:
$$\mathcal{C}_+ = C, \qquad \mathcal{C}_- = \Gamma C .$$
The next cell checks the **transposition rules** that characterise them,
$\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$ and
$\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- = +(\gamma^a)^T$ for every $a$, and that
both are real (and symmetric). Then it solves, again exactly, the equations
$\gamma^a X = \zeta X(\gamma^a)^T$ for all $a$ with $\zeta = -1$ and $\zeta = +1$:
their only solutions are the multiples of $C$ and of $\Gamma C$, so these two
matrices are the only ones with such transposition rules.
```

In [8]:

```
calC_plus = C  # M = 1
calC_minus = Gamma @ C  # M = Gamma
inv_plus = np.linalg.inv(calC_plus)  # the inverse matrix
inv_minus = np.linalg.inv(calC_minus)
check(all(np.allclose(inv_plus @ gamma[x] @ calC_plus, -gamma[x].T) for x in COORDS)
      and np.array_equal(calC_plus.T, calC_plus)
      and lead_passed("charge_conjugation_matrix_plus"),
      "calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real, symmetric",
      record=f"{RECORD}, check charge_conjugation_matrix_plus")
check(all(np.allclose(inv_minus @ gamma[x] @ calC_minus, gamma[x].T) for x in COORDS)
      and np.array_equal(calC_minus.T, calC_minus)
      and lead_passed("charge_conjugation_matrix_minus"),
      "calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real, symmetric",
      record=f"{RECORD}, check charge_conjugation_matrix_minus")


def transposition_system(zeta):
    """Coefficient matrix of gamma^a X - zeta X (gamma^a)^T = 0 for all a."""
    return np.vstack([np.kron(gamma[x], I16) - zeta * np.kron(I16, gamma[x])
                      for x in COORDS])


X_minus = exact_solutions(transposition_system(-1))  # expected: multiples of C
X_plus = exact_solutions(transposition_system(+1))  # expected: multiples of Gamma C
check(len(X_minus) == 1 and proportional(X_minus[0], calC_plus)
      and len(X_plus) == 1 and proportional(X_plus[0], calC_minus),
      "gamma^a X = zeta X (gamma^a)^T: zeta = -1 only C, zeta = +1 only Gamma C")
```

Out [8]:

```
PASS calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real, symmetric
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    charge_conjugation_matrix_plus
PASS calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real, symmetric
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    charge_conjugation_matrix_minus
PASS gamma^a X = zeta X (gamma^a)^T: zeta = -1 only C, zeta = +1 only Gamma C
```

Text cell:

```
The next cell draws the two charge-conjugation matrices and the chirality that
relates them. Since $\Gamma$ is $-1$ on the first eight components and $+1$ on
the last eight, $\Gamma C$ is $C$ with the signs of its first eight rows reversed.
```

In [9]:

```
fig, axes = plt.subplots(1, 3, figsize=(11.5, 4.2))
heat_map(axes[0], calC_plus, r"$\mathcal{C}_+ = C$ (same mass)")
heat_map(axes[1], Gamma, r"chirality $\Gamma$", row_label=False)
image = heat_map(axes[2], calC_minus, r"$\mathcal{C}_- = \Gamma C$ (mass reversed)",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
save_figure(fig, "conjugation_matrices",
            "Heat maps of the two charge-conjugation matrices and of the chirality "
            "that relates them: $\\mathcal{C}_+ = C$ (left, the conjugation that "
            "keeps the mass), $\\Gamma$ (middle) and $\\mathcal{C}_- = \\Gamma C$ "
            "(right, the conjugation that reverses the mass); horizontal axis the "
            "column, vertical axis the row, colour the entry (red $+1$, blue $-1$, "
            "pale $0$). $C$ and $\\Gamma C$ have their entries only in the two "
            "diagonal blocks, and they differ only by the sign of the upper half, "
            "where $\\Gamma = -1$.")
```

Out [9]:

```
Figure 21a.3 saved as Revision/textbook/figures/21a_3_conjugation_matrices.png
```

Text cell:

```
## 9. The two reality (Majorana) conditions

A field that equals its own conjugate, $\Psi^c = \Psi$, satisfies
$\Psi = M\Psi^*$. Applying the condition twice gives $\Psi = M(M\Psi^*)^* =
MM^*\Psi$, so the condition can hold for nonzero fields without contradiction only
if $MM^* = 1$. For $M = 1$ the condition is $\Psi = \Psi^*$: the field is real.
For $M = \Gamma$ it is $\Psi = \Gamma\Psi^*$: the last eight components (where
$\Gamma = +1$) are real and the first eight (where $\Gamma = -1$) are purely
imaginary. The next cell checks $MM^* = 1$ for both (record check
majorana_conditions_consistent), makes one example of each kind from a fixed
complex column $v$ by $\Psi = (v + Mv^*)/2$, checks that each example satisfies
its condition, and draws them.
```

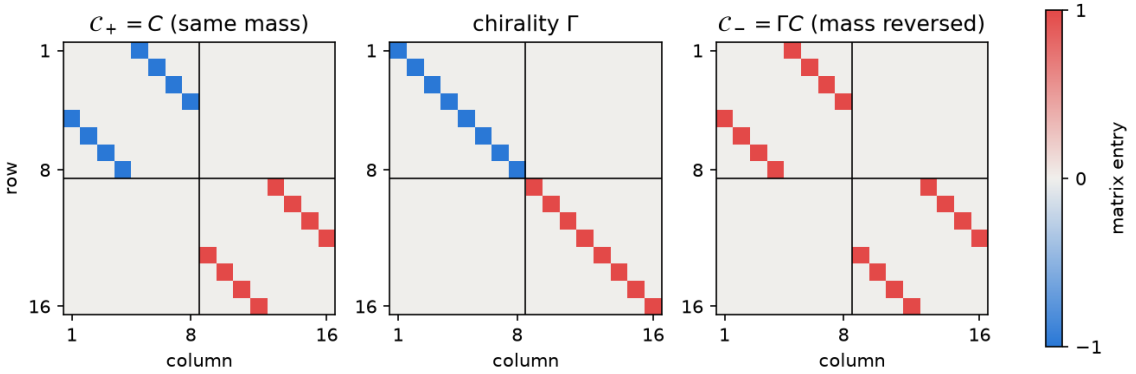

Figure 524: Heat maps of the two charge-conjugation matrices and of the chirality that relates them: $\mathcal{C}_+ = C$ (left, the conjugation that keeps the mass), Γ (middle) and $\mathcal{C}_- = \Gamma C$ (right, the conjugation that reverses the mass); horizontal axis the column, vertical axis the row, colour the entry (red $+1$, blue -1, pale 0). C and ΓC have their entries only in the two diagonal blocks, and they differ only by the sign of the upper half, where $\Gamma = -1$. (Notebook 21a, figure 3.)

In [10]:

```
check(np.array_equal(I16 @ np.conj(I16), I16)
      and np.array_equal(Gamma @ np.conj(Gamma), I16)
      and lead_passed("majorana_conditions_consistent"),
      "M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent",
      record=f"{RECORD}, check majorana_conditions_consistent")

rows = np.arange(1, 17)  # the component numbers 1, ..., 16
v = (np.cos(rows) + 0.5) + 1j * np.sin(2.0 * rows)  # a fixed complex column
psi_real = (v + np.conj(v)) / 2  # satisfies Psi = Psi*
psi_gamma = (v + Gamma @ np.conj(v)) / 2  # satisfies Psi = Gamma Psi*
check(np.allclose(psi_real, np.conj(psi_real))
      and np.allclose(psi_gamma, Gamma @ np.conj(psi_gamma)),
      "the two example fields satisfy Psi = Psi* and Psi = Gamma Psi*")

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0), sharey=True)
for ax, psi, title in ((axes[0], psi_real, r"$\Psi = \Psi^*$ (condition of "
                        r"$\mathcal{C}_+$)"),
                       (axes[1], psi_gamma, r"$\Psi = \Gamma\Psi^*$ (condition of "
                        r"$\mathcal{C}_-$)")):
    ax.bar(rows - 0.2, psi.real, width=0.4, label="real part")
    ax.bar(rows + 0.2, psi.imag, width=0.4, label="imaginary part")
    ax.axvline(8.5, color="black", linewidth=0.8)
    ax.set_xticks([1, 4, 8, 12, 16])
    ax.set_xlabel("component number")
    ax.set_title(title)
    ax.legend(loc="lower left")
axes[0].set_ylabel("value of the component")
save_figure(fig, "reality_conditions",
            "One field satisfying each reality (Majorana) condition, made from the "
            "same complex column: left $\\Psi = \\Psi^\\ast$ (every imaginary part is "
            "zero), right $\\Psi = \\Gamma\\Psi^\\ast$ (components 1 to 8, where "
```

```
            "$\\Gamma = -1$, are purely imaginary; components 9 to 16, where "
            "$\\Gamma = +1$, are real); horizontal axis the component number, "
            "vertical axis the real part (left bar of each pair) and the imaginary "
            "part (right bar). Both conditions can be imposed, because "
            "$MM^\\ast = 1$ for $M = 1$ and for $M = \\Gamma$.")
```

Out [10]:

```
PASS M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    majorana_conditions_consistent
PASS the two example fields satisfy Psi = Psi* and Psi = Gamma Psi*
```

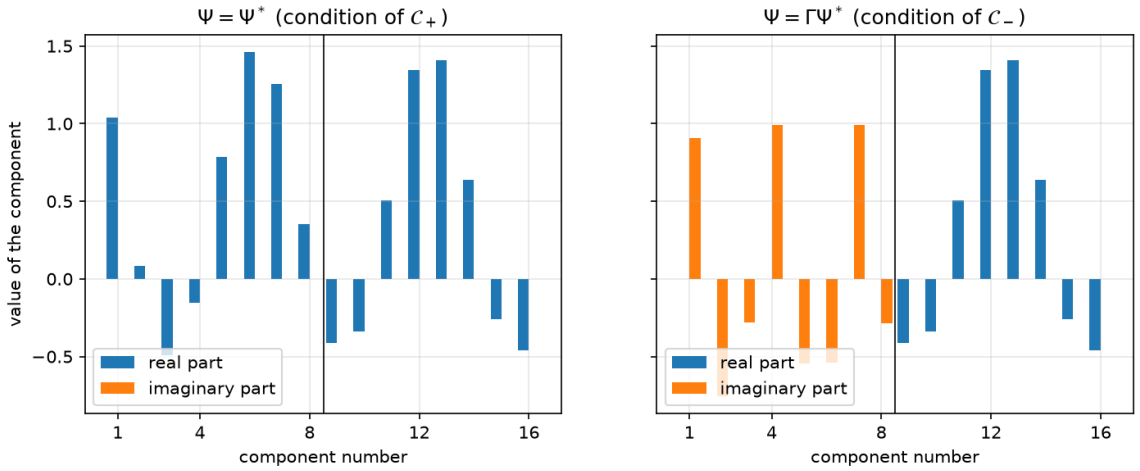

Figure 525: One field satisfying each reality (Majorana) condition, made from the same complex column: left $\Psi = \Psi^*$ (every imaginary part is zero), right $\Psi = \Gamma\Psi^*$ (components 1 to 8, where $\Gamma = -1$, are purely imaginary; components 9 to 16, where $\Gamma = +1$, are real); horizontal axis the component number, vertical axis the real part (left bar of each pair) and the imaginary part (right bar). Both conditions can be imposed, because $MM^* = 1$ for $M = 1$ and for $M = \Gamma$. (Notebook 21a, figure 4.)

Out [10]:

```
Figure 21a.4 saved as Revision/textbook/figures/21a_4_reality_conditions.png
```

Text cell:

```
## 10. The conjugations acting on an exact solution in the author's metric

The Revision field-theory record (‘Revision/theory/field-theory.json‘, formula
exact_solutions, item (i)) gives an exact solution of the field equation with
$U = 0$ in the author's curved metric:
$$\Psi = \sin^\alpha z\,\Big(\cosh(kx_4)\,1 + \frac{\sinh(kx_4)}{k}\,M_m\Big)\chi,
\quad M_m = -m\gamma^{(x4)} + b\,\gamma^{(x4)}\gamma^{(x0)},\quad b = 3H(2\alpha+1),
```

```
\quad k^2 = b^2 - m^2,$$
with $z = 6Hx_0$ and a constant column $\chi$. It depends only on $x_4$ and $x_0$.
The field equation of the record reads
$$e^{-a_4}\sin^{-1/6}z\,(\gamma^{(x1)}\partial_1 + \gamma^{(x2)}\partial_2 +
\gamma^{(x3)}\partial_3)\Psi + \gamma^{(x4)}\partial_4\Psi + e^{a_4}\sin^{-1/6}z
\,(\gamma^{(x5)}\partial_5 + \gamma^{(x6)}\partial_6 + \gamma^{(x7)}\partial_7)\Psi
+ \tan z\,\gamma^{(x0)}\partial_0\Psi + 3H\gamma^{(x0)}\Psi = m\Psi .$$
The derivatives along $x1, x2, x3, x5, x6, x7$ vanish for this $\Psi$, so the
factors $e^{\mp a_4}$ drop out: the solution holds for **every** history
$a_4(x_4)$, in particular for the one in which the three extra times deflate
exponentially. What remains is
$$E_m[\Psi] = \gamma^{(x4)}\partial_4\Psi + 6H\tan z\,\gamma^{(x0)}\partial_z\Psi
+ 3H\gamma^{(x0)}\Psi - m\Psi = 0 ,$$
where $\partial_0 = 6H\,\partial_z$ by the chain rule ($z = 6Hx_0$).

We take $H = 1/6$ (so that $z = x_0$), $\alpha = 1$ and $m = 2$. Then $b = 3/2$
and $k^2 = 9/4 - 4 = -7/4 < 0$, so $k = i\sqrt7/2$ is imaginary; with
$\cosh(i\theta) = \cos\theta$ and $\sinh(i\theta)/i = \sin\theta$ the solution
oscillates: $\cosh(kx_4) = \cos(wx_4)$ and $\sinh(kx_4)/k = \sin(wx_4)/w$ with
$w = \sqrt7/2$. Both are real functions, so conjugation acts only on $\chi$.

The next cell first reads the two formulas from the Revision record (written there
in Wolfram notation: ‘g[x4]‘ is $\gamma^{(x4)}$, ‘d4‘ is $\partial_4$, ‘al‘ is
$\alpha$, ‘E^a4[x4]‘ is $e^{a_4}$) and checks that they are, word for word, the
formulas written above, so that any later change of the record is caught. Then it
builds $\Psi$ with sympy for a fixed complex $\chi$ and checks, exactly, that
$E_m[\Psi] = 0$ (Revision check exact_solution_family_x4_x0).
```

In [11]:

```
THEORY = {f["key"]: f["wl"] for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
PY_THEORY = {c["name"]: c["verdict"].upper() for c in json.loads(repository_file(
    "Revision/theory/reports/python-field-theory.json").read_text(
    encoding="utf-8"))["checks"]}
# the left side of the record's field equation, word for word (right side m + U')
FIELD_EQUATION_LEFT = (
    "Tan[z] g[x0] d0 Psi"
    " + E^-a4[x4] Sin[z]^(-1/6) (g[x1] d1 + g[x2] d2 + g[x3] d3) Psi + g[x4] d4 Psi"
    " + E^a4[x4] Sin[z]^(-1/6) (g[x5] d5 + g[x6] d6 + g[x7] d7) Psi"
    " + 3 H g[x0] Psi")
# item (i) of the record's formula exact_solutions, word for word
SOLUTION_I = (
    "(i) U = 0: Psi = Sin[z]^al (Cosh[k x4] + Sinh[k x4]/k M) chi, M = -m g[x4]"
    " + 3 H (2 al + 1) g[x4].g[x0], k^2 = 9 H^2 (2 al + 1)^2 - m^2 (any a4)")
left, right = THEORY["field_equation"].split(" = ")  # the two sides
say("record, exact solution " + SOLUTION_I)
check(left == FIELD_EQUATION_LEFT and right.startswith("(m + U")
      and THEORY["exact_solutions"].startswith(SOLUTION_I + "; "),
      "the record states the field equation and the exact solution (i) used here",
      record="Revision/theory/field-theory.json, formulas field_equation and "
      "exact_solutions")
```

```
x4, z = sp.symbols("x4 z", real=True)  # the time x4 and the angle z = 6 H x0
H, alpha, m = sp.Rational(1, 6), 1, 2  # the parameters chosen above
b = 3 * H * (2 * alpha + 1)  # b = 3/2
w = sp.sqrt(m ** 2 - b ** 2)  # k = i w with w = sqrt(7)/2
G = {x: sp.Matrix(gamma[x].tolist()) for x in COORDS}  # exact sympy matrices
Gamma_s = sp.Matrix(Gamma.tolist())


def M_of(mass):
    """M_m = -m gamma^(x4) + b gamma^(x4) gamma^(x0)."""
    return -mass * G["x4"] + b * G["x4"] * G["x0"]


def E(mass, field):
    """E_m[Psi] = gamma^(x4) d4 Psi + 6H tan z gamma^(x0) dz Psi + 3H gamma^(x0) Psi
    - m Psi (the field equation of the record for fields of x4 and z only)."""
    tan_z = sp.sin(z) / sp.cos(z)  # tan z written as sin z / cos z
    return (G["x4"] * field.diff(x4) + 6 * H * tan_z * G["x0"] * field.diff(z)
            + 3 * H * G["x0"] * field - mass * field)


# a fixed complex column chi with exact entries (j + i (17 - j)) / 16, j = 1..16
chi = sp.Matrix([sp.Rational(j, 16) + sp.I * sp.Rational(17 - j, 16)
                 for j in range(1, 17)])
U = sp.cos(w * x4) * sp.eye(16) + sp.sin(w * x4) / w * M_of(m)  # real 16 x 16
Psi = sp.sin(z) ** alpha * U * chi
check(E(m, Psi).expand() == sp.zeros(16, 1)
      and sp.simplify(M_of(m) ** 2 + w ** 2 * sp.eye(16)) == sp.zeros(16, 16)
      and PY_THEORY["exact_solution_family_x4_x0"] == "PASS",
      "the record's exact solution solves the field equation (mass m = 2)",
      record="Revision/theory/reports/python-field-theory.json, check "
      "exact_solution_family_x4_x0")
```

Out [11]:

```
record, exact solution (i) U = 0: Psi = Sin[z]^al (Cosh[k x4] + Sinh[k x4]/k M) chi, M =
    -m g[x4] + 3 H (2 al + 1) g[x4].g[x0], k^2 = 9 H^2 (2 al + 1)^2 - m^2 (any a4)
PASS the record states the field equation and the exact solution (i) used here
     reproduces Revision/theory/field-theory.json, formulas field_equation and
    exact_solutions
PASS the record's exact solution solves the field equation (mass m = 2)
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_solution_family_x4_x0
```

Text cell:

```
The next cell forms the two conjugates. Because the matrix $U$ in $\Psi =
\sin z\,U\chi$ is real, $\Psi^* = \sin z\,U\chi^*$ and $\Gamma\Psi^* = \sin z\,
\Gamma U\chi^*$. The general argument of section 4 predicts: $\Psi^*$ solves the
equation with the **same** mass $m = 2$, and $\Gamma\Psi^*$ with the **reversed**
mass $-2$. The cell checks both exactly, and checks the two negative controls:
$\Psi^*$ does not solve the equation with $-2$, and $\Gamma\Psi^*$ does not solve
it with $+2$.
```

In [12]:

```
Psi_conj = Psi.conjugate()  # Psi*: every entry complex-conjugated (x4, z are real)
Psi_gamma_conj = Gamma_s * Psi_conj  # Gamma Psi*


def vanishes(vector):
    """True when every component expands to exactly zero (sympy multiplies out
    every product; cos z / cos z cancels automatically)."""
    return vector.expand() == sp.zeros(16, 1)


check(vanishes(E(m, Psi_conj)) and vanishes(E(-m, Psi_gamma_conj)),
      "Psi* solves the equation with mass +2, Gamma Psi* with mass -2 (exact)")
check(not vanishes(E(-m, Psi_conj)) and not vanishes(E(m, Psi_gamma_conj)),
      "negative controls: Psi* fails with -2, Gamma Psi* fails with +2")
```

Out [12]:

```
PASS Psi* solves the equation with mass +2, Gamma Psi* with mass -2 (exact)
PASS negative controls: Psi* fails with -2, Gamma Psi* fails with +2
```

Text cell:

```
The next cell turns the exact expressions into fast numerical functions
(‘sp.lambdify‘) and evaluates them along the time $x_4$ at the fixed angle
$z = \pi/4$. It computes the charge density $J^{(x4)} = \Psi^\dagger B\Psi$ of
$\Psi$, $\Psi^*$ and $\Gamma\Psi^*$ (‘np.einsum("tr,rc,tc->t", ...)‘ forms
$\sum_{r,c}\Psi_r^* B_{rc}\Psi_c$ at every time $t$) and checks the signs of the
table of section 11 below: $\Psi^*$ carries $-J^{(x4)}$, $\Gamma\Psi^*$ carries
$+J^{(x4)}$. It also computes the size of the left-hand side of the field
equation, $|E_{\pm2}[\cdot]|$ (the square root of the sum of the squared moduli of
its 16 components). Where a field solves the equation, $|E|$ is zero up to
rounding (about $10^{-15}$). Where it does not, $|E|$ is far from zero: since
$E_{-2}[\Psi^*] = E_{+2}[\Psi^*] + 4\Psi^*$ and the first term vanishes,
$|E_{-2}[\Psi^*]| = 4|\Psi^*|$, about 10 here (and in the same way
$|E_{+2}[\Gamma\Psi^*]| = 4|\Gamma\Psi^*|$). The cell checks this too. The figure
shows the charge densities on the left and $|E|$ on a logarithmic scale on the
right.
```

In [13]:

```
times = np.linspace(0.0, 8.0, 161)  # 161 times x4 from 0 to 8
z_fixed = np.pi / 4


def numeric(expression):
    """Evaluate a sympy 16 x 1 expression of x4 at z = pi/4 for all times."""
    f = sp.lambdify((x4, z), expression, "numpy")
    return np.array([np.array(f(t, z_fixed), dtype=complex).reshape(16)
```

```
                    for t in times])


def size(expression):
    """|E| at every time: the length of the 16-component column."""
    return np.linalg.norm(numeric(expression), axis=1)


curves = {
    r"$|E_{+2}[\Psi^*]|$ (same mass)": size(E(m, Psi_conj)),
    r"$|E_{-2}[\Gamma\Psi^*]|$ (mass reversed)": size(E(-m, Psi_gamma_conj)),
    r"$|E_{-2}[\Psi^*]|$ (control)": size(E(-m, Psi_conj)),
    r"$|E_{+2}[\Gamma\Psi^*]|$ (control)": size(E(m, Psi_gamma_conj)),
}
floor = 1e-17  # added before taking the logarithm, so that 0 can be drawn
values = list(curves.values())
say(f"largest |E| of the two solutions below 1e-12: "
    f"{max(values[0].max(), values[1].max()) < 1e-12}")
say(f"smallest |E| of the two controls: {min(values[2].min(), values[3].min()):.2f}")
field_size = size(Psi_conj)  # |Psi*| = |Gamma Psi*| at every time (Gamma permutes)
check(max(values[0].max(), values[1].max()) < 1e-12
      and np.allclose(values[2], 4 * field_size)
      and np.allclose(values[3], 4 * field_size),
      "numerically: the solutions give |E| below 1e-12, the controls |E| = 4 |field|")

B_c = B.astype(complex)  # the charge density is J^(x4) = Psi^dagger B Psi
density = {}
for field, label in ((Psi, r"$\Psi$"), (Psi_conj, r"$\Psi^*$"),
                     (Psi_gamma_conj, r"$\Gamma\Psi^*$")):
    values_f = numeric(field)  # shape (161, 16): the field at every time
    density[label] = np.einsum("tr,rc,tc->t", np.conj(values_f), B_c, values_f).real
d_psi, d_conj, d_gconj = density.values()
check(np.allclose(d_conj, -d_psi) and np.allclose(d_gconj, d_psi)
      and np.ptp(d_psi) > 0.1,
      "charge density: Psi* has -J^(x4), Gamma Psi* has +J^(x4) at every time")

fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for (label, values_d), style in zip(density.items(), ["-", "--", ":"]):
    axes[0].plot(times, values_d, style, linewidth=2, label=f"{label}")
axes[0].set_xlabel("time $x_4$")
axes[0].set_ylabel("charge density $\\Psi^\\dagger B\\Psi$ at $z = \\pi/4$")
axes[0].set_title("A solution and its two conjugates")
axes[0].legend(fontsize=8)
for (label, values_e), style in zip(curves.items(), ["-", "--", "-.", ":"]):
    axes[1].semilogy(times, values_e + floor, style, label=label)
axes[1].set_xlabel("time $x_4$")
axes[1].set_ylabel("$|E|$ at $z = \\pi/4$ (log scale)")
axes[1].set_title("Which mass does each conjugate solve?")
axes[1].legend(fontsize=8)
save_figure(fig, "curved_solution_images",
            "Charge conjugation acting on an exact solution of the field equation "
            "in the author's metric ($H = 1/6$, $\\alpha = 1$, mass $m = 2$, valid "
            "for every history $a_4$). Left: the charge density "
            "$\\Psi^\\dagger B\\Psi$ of $\\Psi$ (solid), $\\Psi^\\ast$ (dashed) and "
```

```
"$\\Gamma\\Psi^\\ast$ (dotted, on top of the solid curve) versus the "
"time $x_4$ at $z = \\pi/4$ (pure numbers); $\\Psi^\\ast$ carries the "
"opposite charge density. Right: the size $|E|$ of the left-hand "
"side of the field equation versus $x_4$, logarithmic scale. $\\Psi^\\ast$ "
"solves the equation with the same mass $+2$ and $\\Gamma\\Psi^\\ast$ the "
"one with the reversed mass $-2$ ($|E|$ at rounding level, about "
"$10^{-16}$), while the exchanged masses fail ($|E|$ equals four "
"times the size of the field, about 10).")
```

Out [13]:

```
largest |E| of the two solutions below 1e-12: True
smallest |E| of the two controls: 9.67
PASS numerically: the solutions give |E| below 1e-12, the controls |E| = 4 |field|
PASS charge density: Psi* has -J^(x4), Gamma Psi* has +J^(x4) at every time
```

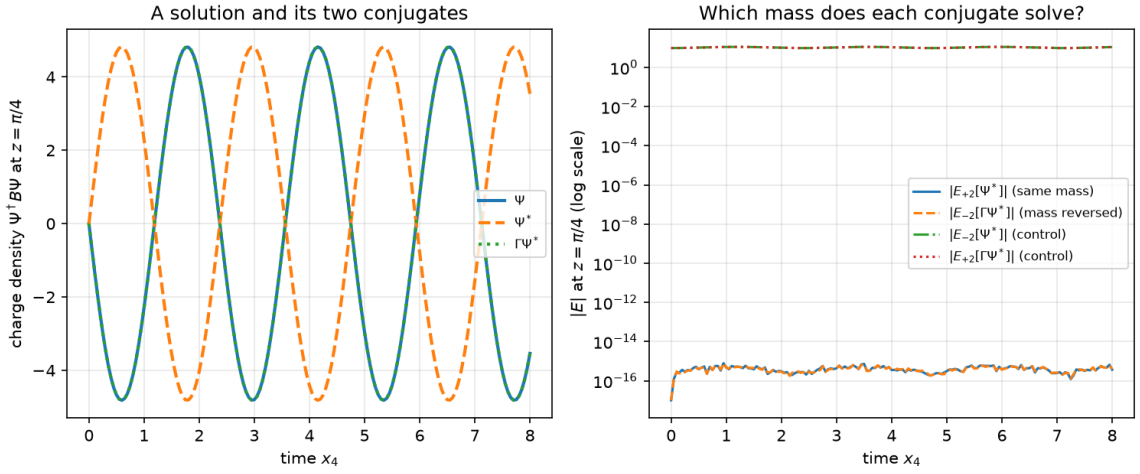

Figure 526: Charge conjugation acting on an exact solution of the field equation in the author's metric ($H = 1/6$, $\alpha = 1$, mass $m = 2$, valid for every history a_4). Left: the charge density $\Psi^\dagger B \Psi$ of Ψ (solid), Ψ^* (dashed) and $\Gamma\Psi^*$ (dotted, on top of the solid curve) versus the time x_4 at $z = \pi/4$ (pure numbers); Ψ^* carries the opposite charge density. Right: the size $|E|$ of the left-hand side of the field equation versus x_4, logarithmic scale. Ψ^* solves the equation with the same mass $+2$ and $\Gamma\Psi^*$ the one with the reversed mass -2 ($|E|$ at rounding level, about 10^{-16}), while the exchanged masses fail ($|E|$ equals four times the size of the field, about 10). (Notebook 21a, figure 5.)

Out [13]:

```
Figure 21a.5 saved as Revision/textbook/figures/21a_5_curved_solution_images.png
```

Text cell:

```
## 11. How the bilinears change: the sign table

Under $\Psi \to M\Psi^*$ a bilinear $\Psi^\dagger K\Psi = \sum_{r,c}\Psi_r^*K_{rc}
```

```
\Psi_c$ becomes $\sum_{r,c}(M\Psi^*)_r^*K_{rc}(M\Psi^*)_c = \sum\Psi_r(M^\dagger
KM)_{rc}\Psi_c^*$ ($M$ is real). To bring this back to the form $\sum\Psi^*_c
(\dots)\Psi_r$ we exchange the two factors $\Psi_r$ and $\Psi_c^*$: for commuting
components this costs nothing, for anticommuting components a sign. Hence the new
bilinear is $\Psi^\dagger K'\Psi$ with
$$K' = \epsilon\,(M^\dagger KM)^T ,\qquad \epsilon = +1 \text{ (commuting)},\
\epsilon = -1 \text{ (anticommuting)} .$$
If $K' = \pm K$, the bilinear keeps or reverses its sign. The next cell computes
this for the scalar $S$ ($K = C$) and the eight currents $J^a$ ($K = -iC\gamma^a$),
for both matrices and both kinds of components, and compares with the signs
measured in the Revision record (check bilinears_under_charge_conjugation).
```

In [14]:

```
def effective(Mmat, K, eps):
    """The matrix K' of the bilinear after Psi -> M Psi*, with the reorder sign eps."""
    return eps * (Mmat.conj().T @ K @ Mmat).T


def sign_of(Kp, K):
    """+1 if K' = K, -1 if K' = -K, 0 otherwise."""
    return 1 if np.allclose(Kp, K) else (-1 if np.allclose(Kp, -K) else 0)


K_S = C.astype(complex)  # the scalar S = Psi^dagger C Psi
K_J = [-1j * (C @ gamma[x]) for x in COORDS]  # the currents J^a
measured = {}
for label, Mmat in (("plus", I16), ("minus", Gamma)):
    for eps in (1, -1):
        sS = sign_of(effective(Mmat, K_S, eps), K_S)
        sJ = [sign_of(effective(Mmat, K, eps), K) for K in K_J]
        measured[f"{label},eps={eps}"] = [sS, sJ]
        kind = "commuting" if eps == 1 else "anticommuting"
        say(f"calC_{'+' if label == 'plus' else '-'}, {kind:13}: S -> {sS:+d} S, "
            f"J -> {sJ[0]:+d} J (all eight components alike: {len(set(sJ)) == 1})")
detail = LEAD["bilinears_under_charge_conjugation"]["detail"]
recorded = json.loads(detail.split("measured: ", 1)[1])  # the record's own table
check(measured == recorded and lead_passed("bilinears_under_charge_conjugation"),
      "signs of S and J under calC_+ and calC_-, both statistics, equal the record",
      record=f"{RECORD}, check bilinears_under_charge_conjugation")
```

Out [14]:

```
calC_+, commuting    : S -> +1 S, J -> -1 J (all eight components alike: True)
calC_+, anticommuting: S -> -1 S, J -> +1 J (all eight components alike: True)
calC_-, commuting    : S -> +1 S, J -> +1 J (all eight components alike: True)
calC_-, anticommuting: S -> -1 S, J -> -1 J (all eight components alike: True)
PASS signs of S and J under calC_+ and calC_-, both statistics, equal the record
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    bilinears_under_charge_conjugation
```

Text cell:

```
**All 256 bilinears.** Every $16 \times 16$ matrix is a combination of the 256
products $\Gamma_A = \gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$ ($a_1 < \dots <
a_k$ in the order $x0, \dots, x7$, $k = 0, \dots, 8$; $\Gamma_\emptyset = 1$), so
the bilinears $\bar\Psi\Gamma_A\Psi = \Psi^\dagger C\Gamma_A\Psi$ are a complete
list. The next cell computes the sign of each of them for the four cases. The
result depends only on the number $k$ of gammas (the *degree*): for commuting
components $\mathcal{C}_+$ gives $(-1)^{k(k+1)/2}$ and $\mathcal{C}_-$ gives
$(-1)^{k(k-1)/2}$; anticommuting components add one more factor $-1$.
```

In [15]:

```
CASES = [("plus", I16, 1), ("plus", I16, -1), ("minus", Gamma, 1), ("minus", Gamma, -1)]
degree_signs = np.zeros((4, 9), dtype=int)  # rows: the four cases, columns: k
for row, (label, Mmat, eps) in enumerate(CASES):
    for k in range(9):
        signs = {sign_of(effective(Mmat, C @ product(A), eps), C @ product(A))
                 for A in itertools.combinations(COORDS, k)}
        # one common sign for all products of degree k, else 0
        degree_signs[row, k] = signs.pop() if len(signs) == 1 else 0
k_all = np.arange(9)
formula = np.array([(-1) ** (k_all * (k_all + 1) // 2),
                    -(-1) ** (k_all * (k_all + 1) // 2),
                    (-1) ** (k_all * (k_all - 1) // 2),
                    -(-1) ** (k_all * (k_all - 1) // 2)])
for row, (label, _, eps) in enumerate(CASES):
    say(f"calC_{'+' if label == 'plus' else '-'}, eps = {eps:+d}: signs for k = 0..8: "
        + " ".join(f"{s:+d}" for s in degree_signs[row]))
check(np.array_equal(degree_signs, formula),
      "all 256 bilinears: the sign depends only on the degree k, as the formulas say")
```

Out [15]:

```
calC_+, eps = +1: signs for k = 0..8: +1 -1 -1 +1 +1 -1 -1 +1 +1
calC_+, eps = -1: signs for k = 0..8: -1 +1 +1 -1 -1 +1 +1 -1 -1
calC_-, eps = +1: signs for k = 0..8: +1 +1 -1 -1 +1 +1 -1 -1 +1
calC_-, eps = -1: signs for k = 0..8: -1 -1 +1 +1 -1 -1 +1 +1 -1
PASS all 256 bilinears: the sign depends only on the degree k, as the formulas say
```

Text cell:

```
The next cell draws the table: rows are the four cases, columns the degree $k$;
the column $k = 0$ is the scalar $S$ and the column $k = 1$ the current $J$.
```

In [16]:

```
fig, ax = plt.subplots(figsize=(9.0, 3.8))
ax.imshow(degree_signs, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
for row in range(4):
    for k in range(9):
```

```
        ax.text(k, row, f"{degree_signs[row, k]:+d}", ha="center", va="center")
ax.set_xticks(range(9), ["0\n($S$)", "1\n($J$)"] + [str(k) for k in range(2, 9)])
ax.set_yticks(range(4), [r"$\mathcal{C}_+$, commuting",
                         r"$\mathcal{C}_+$, anticommuting",
                         r"$\mathcal{C}_-$, commuting",
                         r"$\mathcal{C}_-$, anticommuting"])
ax.set_xlabel("degree $k$ (number of gamma factors in the bilinear)")
ax.set_title("Sign of every bilinear under the two charge conjugations")
ax.grid(False)
save_figure(fig, "bilinear_signs",
            "The sign that each of the 256 bilinears $\\bar\\Psi\\gamma^{a_1}"
            "\\cdots\\gamma^{a_k}\\Psi$ acquires under $\\Psi \\to M\\Psi^\\ast$ "
            "($M = 1$ for $\\mathcal{C}_+$, $M = \\Gamma$ for $\\mathcal{C}_-$), "
            "for commuting and anticommuting components; horizontal axis the degree "
            "$k$, rows the four cases, colour and number the sign (red $+1$, blue "
            "$-1$). All products of the same degree share one sign. For commuting "
            "components $\\mathcal{C}_+$ keeps the scalar $S$ ($k = 0$) and "
            "reverses the current $J$ ($k = 1$); anticommuting components reverse "
            "every sign.")
```

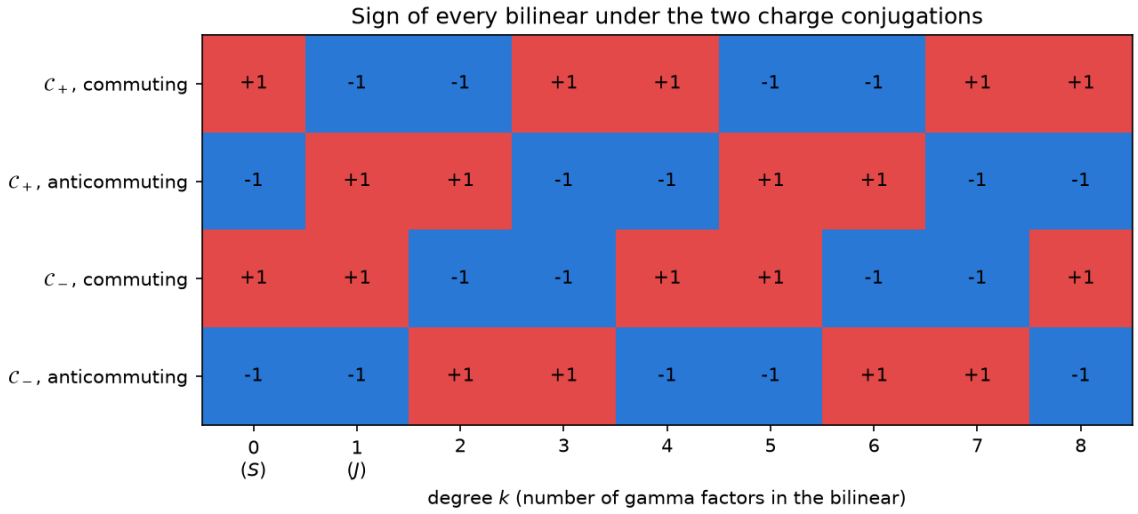

Figure 527: The sign that each of the 256 bilinears $\bar\Psi\gamma^{a_1}\cdots\gamma^{a_k}\Psi$ acquires under $\Psi \to M\Psi^*$ ($M = 1$ for $\mathcal{C}_+$, $M = \Gamma$ for $\mathcal{C}_-$), for commuting and anticommuting components; horizontal axis the degree k, rows the four cases, colour and number the sign (red $+1$, blue -1). All products of the same degree share one sign. For commuting components $\mathcal{C}_+$ keeps the scalar S ($k = 0$) and reverses the current J ($k = 1$); anticommuting components reverse every sign. (Notebook 21a, figure 6.)

Out [16]:

```
Figure 21a.6 saved as Revision/textbook/figures/21a_6_bilinear_signs.png
```

Text cell:

```
**A numerical confirmation with numbers.** For commuting components the bilinears
are ordinary complex numbers, and we can simply evaluate them. The next cell takes
```

```
a fixed complex field value $\Psi$ (from a random-number generator with the fixed
seed 2101, so every run gives the same numbers), computes $S$ and the eight $J^a$
for $\Psi$, $\Psi^*$ and $\Gamma\Psi^*$, and checks the signs of the table:
$\Psi^*$ keeps $S$ and reverses $J$; $\Gamma\Psi^*$ keeps both.
```

In [17]:

```
rng = np.random.default_rng(2101)  # a fixed seed: the same numbers in every run
psi = rng.normal(size=16) + 1j * rng.normal(size=16)  # a complex field value


def S_of(p):
    """S = Psi^dagger C Psi."""
    return np.conj(p) @ C @ p


def J_of(p):
    """The eight currents J^a = Psi^dagger (-i C gamma^a) Psi."""
    return np.array([np.conj(p) @ K @ p for K in K_J])


for label, p in (("Psi", psi), ("Psi*", np.conj(psi)),
                 ("Gamma Psi*", Gamma @ np.conj(psi))):
    say(f"{label:10}: S = {S_of(p).real:+.4f}, J^(x4) = {J_of(p)[4].real:+.4f}")
check(np.isclose(S_of(np.conj(psi)), S_of(psi))
      and np.allclose(J_of(np.conj(psi)), -J_of(psi))
      and np.isclose(S_of(Gamma @ np.conj(psi)), S_of(psi))
      and np.allclose(J_of(Gamma @ np.conj(psi)), J_of(psi)),
      "numbers: Psi* keeps S and reverses J; Gamma Psi* keeps S and J")
```

Out [17]:

```
Psi       : S = +6.0205, J^(x4) = +6.4970
Psi*      : S = +6.0205, J^(x4) = -6.4970
Gamma Psi*: S = +6.0205, J^(x4) = +6.4970
PASS numbers: Psi* keeps S and reverses J; Gamma Psi* keeps S and J
```

Text cell:

```
## 12. Real fields

For a **real** field ($\Psi^* = \Psi$) with commuting components:

1. $\mathcal{C}_+$ maps $\Psi$ to $\Psi^c = \Psi^* = \Psi$: it is the identity. A
   real field is its own charge conjugate.
2. Every current vanishes: $J^a = -i\Psi^TC\gamma^a\Psi = 0$, because the matrix
   $C\gamma^a$ is antisymmetric, and $\Psi^TA\Psi = \sum_{r,c}\Psi_rA_{rc}\Psi_c$
   is zero for an antisymmetric $A$ (the terms $(r, c)$ and $(c, r)$ cancel). A
   real field carries no U(1) charge.
3. The real matrix $\Gamma$ maps real fields to real fields, keeps
   $S = \Psi^TC\Psi$ (since $\Gamma^TC\Gamma = C$) and reverses every kinetic
```

```
   matrix ($\Gamma^TC\gamma^a\Gamma = -C\gamma^a$). With $m \to -m$ and
   $\lambda \to -\lambda$ it maps solutions to solutions (theorem T1 of the
   Revision pairing record). For real fields, the matter-antimatter map is the
   **matrix** $\Gamma$ together with the reversed mass.

The next cell checks these statements exactly with sympy, using a real field with
16 symbolic real components $r_1, \dots, r_{16}$ (record check
real_fields_charge_conjugation).
```

In [18]:

```
r = sp.Matrix(sp.symbols("r1:17", real=True))  # a real field with symbolic entries
C_s = sp.Matrix(C.tolist())
antisymmetric = all(np.array_equal((C @ gamma[x]).T, -(C @ gamma[x])) for x in COORDS)
currents_zero = all(sp.expand((-sp.I * r.T * C_s * G[x] * r)[0, 0]) == 0
                    for x in COORDS)
S_kept = sp.expand((Gamma_s * r).T * C_s * (Gamma_s * r) - r.T * C_s * r) == \
    sp.zeros(1, 1)
kinetic_reversed = all(np.array_equal(Gamma.T @ C @ gamma[x] @ Gamma, -(C @ gamma[x]))
                       for x in COORDS)
check(antisymmetric and currents_zero and S_kept and kinetic_reversed
      and lead_passed("real_fields_charge_conjugation"),
      "real fields: J^a = 0, calC_+ is the identity, Gamma keeps S and reverses "
      "every kinetic matrix C gamma^a",
      record=f"{RECORD}, check real_fields_charge_conjugation")
```

Out [18]:

```
PASS real fields: J^a = 0, calC_+ is the identity, Gamma keeps S and reverses every
    kinetic matrix C gamma^a
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    real_fields_charge_conjugation
```

Text cell:

```
The next cell draws two pictures of these facts. Left: the matrix
$C\gamma^{(x4)}$, which is antisymmetric (the entry in row $r$, column $c$ is minus
the entry in row $c$, column $r$: the picture is its own mirror image across the
diagonal with the colours exchanged). Right: for two fixed real columns $\Psi$ and
$\Phi$ (think of $\Phi$ as the derivative of $\Psi$ in the kinetic term), the eight
kinetic numbers $\Psi^TC\gamma^a\Phi$ before and after the map $\Gamma$: all eight
change sign, while the scalar $S = \Psi^TC\Psi$ is unchanged.
```

In [19]:

```
psi_r = rng.normal(size=16)  # a real column
phi_r = rng.normal(size=16)  # a second real column
kinetic_before = np.array([psi_r @ C @ gamma[x] @ phi_r for x in COORDS])
kinetic_after = np.array([(Gamma @ psi_r) @ C @ gamma[x] @ (Gamma @ phi_r)
```

```
                          for x in COORDS])
S_before, S_after = psi_r @ C @ psi_r, (Gamma @ psi_r) @ C @ (Gamma @ psi_r)
say(f"S before = {S_before:+.4f}, S after = {S_after:+.4f}")
check(np.allclose(kinetic_after, -kinetic_before) and np.isclose(S_after, S_before),
      "numbers: Gamma reverses the eight kinetic numbers and keeps S")

fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.3), width_ratios=[1, 1.4])
heat_map(axes[0], C @ gamma["x4"], r"$C\gamma^{(x4)}$ is antisymmetric")
positions = np.arange(8)
axes[1].bar(positions - 0.2, kinetic_before, width=0.4,
            label=r"$\Psi^T C\gamma^a\Phi$")
axes[1].bar(positions + 0.2, kinetic_after, width=0.4,
            label=r"$(\Gamma\Psi)^T C\gamma^a(\Gamma\Phi)$")
axes[1].axhline(0.0, color="black", linewidth=0.8)
axes[1].set_xticks(positions, COORDS)
axes[1].set_xlabel("direction $a$")
axes[1].set_ylabel("kinetic number")
axes[1].set_title(f"$\\Gamma$ reverses the kinetic numbers; $S$ = {S_before:.3f} "
                  "stays")
axes[1].legend()
save_figure(fig, "real_fields",
            "Real fields. Left: heat map of the real matrix $C\\gamma^{(x4)}$ "
            "(horizontal axis the column, vertical axis the row, red $+1$, blue "
            "$-1$); it is antisymmetric, so $\\Psi^T C\\gamma^{(x4)}\\Psi = 0$ for "
            "every real $\\Psi$ and a real field carries no current. Right: the "
            "eight kinetic numbers $\\Psi^T C\\gamma^a\\Phi$ of two fixed real "
            "columns (left bars) and of their images under the real matrix "
            "$\\Gamma$ (right bars); horizontal axis the direction $a$, vertical "
            "axis the value (pure numbers). Every kinetic number changes sign "
            "while $S = \\Psi^T C\\Psi$ stays: with the mass reversed, $\\Gamma$ "
            "maps real solutions to real solutions.")
```

Out [19]:

```
S before = -6.5629, S after = -6.5629
PASS numbers: Gamma reverses the eight kinetic numbers and keeps S
```

Out [19]:

```
Figure 21a.7 saved as Revision/textbook/figures/21a_7_real_fields.png
```

Text cell:

```
## 13. The Revision record and the figure files

The next cell checks that the Revision record of the lead checks reports all its
checks as passed, and that the nine checks reproduced in this notebook are among
them. (The remaining three are reproduced elsewhere in this chapter's notebooks:
the reality of the spin connection and the U(1) identity concern the local
conservation law of the charge (notebook 21b), and the last check concerns the
quantised field (notebook 21c).) Then it checks
that the seven figure files exist and prints the number of checks that passed.
```

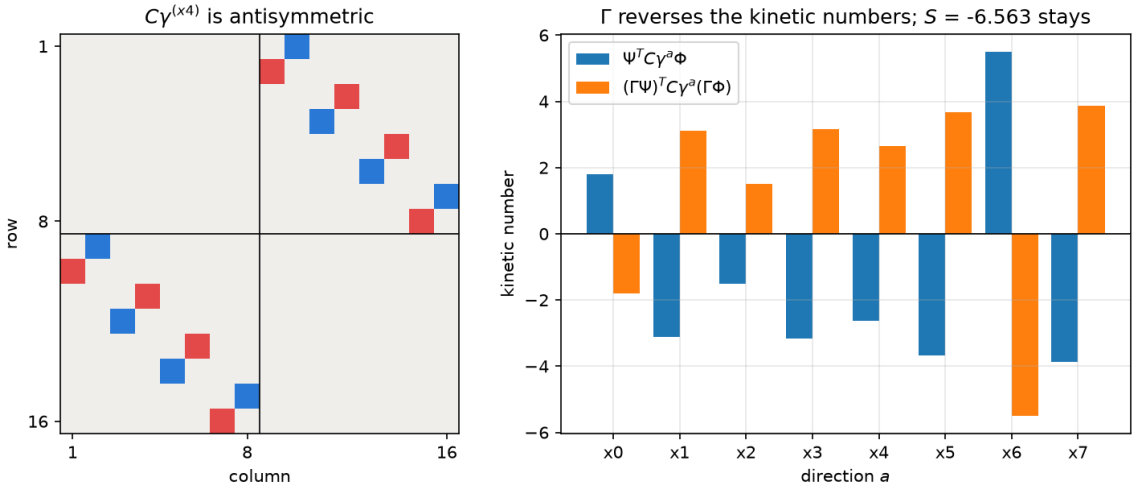

Figure 528: Real fields. Left: heat map of the real matrix $C\gamma^{(x4)}$ (horizontal axis the column, vertical axis the row, red $+1$, blue -1); it is antisymmetric, so $\Psi^T C\gamma^{(x4)}\Psi = 0$ for every real Ψ and a real field carries no current. Right: the eight kinetic numbers $\Psi^T C\gamma^a\Phi$ of two fixed real columns (left bars) and of their images under the real matrix Γ (right bars); horizontal axis the direction a, vertical axis the value (pure numbers). Every kinetic number changes sign while $S = \Psi^T C\Psi$ stays: with the mass reversed, Γ maps real solutions to real solutions. (Notebook 21a, figure 7.)

In [20]:

```
REPRODUCED_HERE = ["representation_real", "B_imaginary_hermitian",
                   "intertwiners_same_mass", "intertwiners_reversed_mass",
                   "charge_conjugation_matrix_plus", "charge_conjugation_matrix_minus",
                   "majorana_conditions_consistent",
                   "bilinears_under_charge_conjugation",
                   "real_fields_charge_conjugation"]
summary = lead["summary"]
check(summary["passed"] == summary["total"] == 12
      and all(lead_passed(name) for name in REPRODUCED_HERE),
      "the lead record passes 12 of 12; the nine checks reproduced here are in it")
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)  # in the order of saving
check(len(names) == 7 and all(
    output_file(f"{FIGURE_FOLDER}/21a_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the seven figure files of notebook 21a exist")
all_checks_passed()
```

Out [20]:

```
PASS the lead record passes 12 of 12; the nine checks reproduced here are in it
PASS the seven figure files of notebook 21a exist
ALL 27 CHECKS PASSED (notebook 21a)
```

Text cell:

```
## 14. What this notebook showed

- PROVED (exact, whole-number arithmetic): the calculation uses **eight real
  $16 \times 16$ gamma matrices** (signed permutation matrices) that obey the
  Clifford relation of signature (4,4), $\eta = \mathrm{diag}(+1, +1, +1, +1, -1,
  -1, -1, -1)$ in the order $x0, \dots, x7$.
- PROVED (exact rational linear algebra): the only matrices $M$ with
  $M(\gamma^a)^* = s\,\gamma^a M$ for all eight gammas are the multiples of $1$
  ($s = +1$, same mass) and of $\Gamma$ ($s = -1$, mass reversed); each gamma
  equation halves the solution space, $256 \to 128 \to \dots \to 1$.
- PROVED: with the intertwiner condition as the definition (that of the Revision
  record), the two charge-conjugation matrices are $\mathcal{C}_+ = C$ (with
  $\mathcal{C}_+^{-1}\gamma^a\mathcal{C}_+ = -(\gamma^a)^T$; $\Psi^c = \Psi^*$) and
  $\mathcal{C}_- = \Gamma C$ (with $\mathcal{C}_-^{-1}\gamma^a\mathcal{C}_- =
  +(\gamma^a)^T$; $\Psi^c = \Gamma\Psi^*$); both are real, and both reality
  (Majorana) conditions are consistent. The condition is sufficient, not
  necessary: for $V = 0$ the projector $P_-$ also maps every solution to a
  solution (Notebook 05c).
- PROVED (exact, sympy): on the Revision record's exact solution in the author's
  curved metric (valid for every history $a_4$, the deflating one included),
  $\Psi^*$ solves the equation with the same mass and $\Gamma\Psi^*$ the one with
  the reversed mass.
- PROVED: under $\mathcal{C}_+$ commuting components keep $S$ and reverse $J$;
  under $\mathcal{C}_-$ they keep both; anticommuting components reverse every
  sign classically; all 256 bilinears follow the degree rules $(-1)^{k(k+1)/2}$
  and $(-1)^{k(k-1)/2}$.
- PROVED: for a real field $\mathcal{C}_+$ is the identity and $J = 0$ (a real
  field is its own conjugate and has no U(1) charge); among the matrices that obey
  the intertwiner condition, the nontrivial real map is the matrix $\Gamma$ with
  the mass reversed. Plain complex conjugation is never
  the charge conjugation of a real field: charge conjugation is a matrix.
- Reproduced: nine checks of 'Revision/lead_checks/reports/
  charge-conjugation-and-u1.json', the formulas field_equation and exact_solutions
  of 'Revision/theory/field-theory.json' (compared word for word) and the check
  exact_solution_family_x4_x0 of 'Revision/theory/reports/python-field-theory.json'.
```

21.15 Line-by-line walk-through of Notebook 21a

The notebook has 20 code cells, In [1] to In [20]. This section explains every line of every one of them, in order: a line or a small group of lines is quoted, then explained. The printed lines of each cell are shown in Section 21.14 after the label Out; the figures are shown there with their captions.

In [1], the set-up cell. Every line that starts with # is a **comment**, which Python skips. The comment lines at the top of the cell are the complete run instructions of Section 21.13, so that the notebook file carries its own instructions. The code starts after the line that announces THE SET-UP; it is the same in every notebook of this book and computes no physics.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux
```

`import` loads a **module**, a part of Python or of an installed package, so that the code can use it; the text after `#` on the same line is a comment. `json`, `os`, `textwrap` and `pathlib` come with Python. `from pathlib import Path` takes the single name `Path` out of `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system.

```
import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell
```

These load the plotting package matplotlib, its drawing functions under the short name `plt`, and the two functions `Image` and `display` of IPython (the part of Jupyter that runs Python), which show a picture file below a cell.

```
NOTEBOOK_ID = "21a"  # this notebook: chapter 21, example a
```

A **variable** is a name for a value; this line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"21a"`. The figure files are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**, a named piece of code that runs when it is called. The text in triple quotes under the `def` line is its **docstring**, a description that Python stores but does not execute. `Path.cwd()` is the folder in which Jupyter runs the notebook, and `.resolve()` writes it as a complete address. `here.parents` is the list of the folders above it, and `[here, *here.parents]` is the list that starts with `here` and continues with them (the star unpacks one list into another). The `for` loop visits these folders one after the other; the indented lines below it run once for each. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back. If none does, `raise` stops the notebook with a `FileNotFoundError` whose message (two strings written next to each other, which Python joins) says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))
```

The function is called and its result named `REPO`. The last line chooses where files are written: `os.environ` holds the **environment variables** of the program (named texts it receives from the computer), and `.get(name, default)` returns the value of `TEXTBOOK_OUTPUT_ROOT` if it is set and `str(REPO)` (the path written as a string) otherwise. When you run the notebook it is not set, so files go into the repository; the book's checking tool sets it to a scratch folder.

```
def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path
```

Two small functions. `repository_file` gives the full path of a repository file, for reading a Revision record. `output_file` gives the full path at which to write a file; `path.parent.mkdir(parents=True, exist_ok=True)` first creates its folder and any missing folder above it, and does nothing if the folder exists.

```
def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`say` prints a text in lines of at most 89 characters; `textwrap.fill` breaks the text at blanks and starts every line after the first with four blanks. That is why some printed lines of Section 21.14 continue on an indented second line.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})
```

`matplotlib.rcdefaults()` returns to the built-in settings, so that the figures are the same on every computer. `plt.rcParams.update({...})` sets the size of a figure (7.0 by 4.2 inches), the size of its letters (10 points) and a faint grid (`grid.alpha` 0.3 means 30 per cent opaque). The braces make a **dictionary**: pairs `key: value`, separated by commas. A line that ends inside an open bracket continues on the next line.

```
FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

A string that starts with `f` is an **f-string**: a name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/21a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty JSON dictionary and a line end into the captions file; `encoding="utf-8"` fixes how letters are stored, and `"\n"` is the line-end character.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`setdefault(name, value)` returns the number already stored for this figure name, or stores and returns one more than the number of figures so far (`len` counts the entries of a dictionary): the figures are numbered 1, 2, 3, and a cell run twice keeps its number. The file name joins the notebook id, the number and the name, for example `21a_1_eight_gammas.png`. `fig.savefig` writes the PNG file with 150 dots per inch, cuts away the empty margin and stores no program name, so that two runs write the same bytes. `plt.close(fig)` removes the figure from memory. The caption is stored, the whole dictionary is written to the captions file (`json.dumps` turns it into JSON text with sorted keys), `display(Image(...))` shows the saved picture below the cell (the output `<IPython.core.display.Image object>` in Section 21.14 is the text label of that picture), and the last line prints where the figure was saved.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")
```

`PASSED` is an empty **list** (square brackets). `check` is the notebook's test: `if not condition` is true when the condition is false, and then `raise AssertionError(...)` stops the notebook with an error that names the check. Otherwise the name is appended to the list and the line PASS with the name is printed. `record=None` gives the third argument a default value; when a cell passes a record, a second line says which Revision record and check the result reproduces.

```
def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`report` prints a key number as a line that starts with RESULT; `(f" {unit}" if unit else "")` adds the unit only when one is given (an empty string counts as false). `all_checks_passed` prints the last line of the notebook with the number of passed checks. The last statement prints the one line of Out [1].

In [2]: reading the gammas and the lead record.

```
import itertools  # all subsets of a list (for the 256 products of gammas)

import numpy as np  # arrays of numbers, matrices and linear algebra
```

`itertools` comes with Python; its function `combinations` (In [15]) lists all subsets of a given size. `numpy`, imported under the short name `np`, stores numbers in **arrays** (tables of numbers) and does matrix algebra.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # the list "x0", ..., "x7"
ETA = dict(zip(COORDS, fixture["eta"]))  # +1 space-like, -1 time-like
```

`read_text` reads the whole Revision record `Revision/algebra/gammas.json` as text, and `json.loads` turns the text into Python objects: here a dictionary, called `fixture`. `fixture["coordinates"]` is the entry stored under the key `"coordinates"`, the list of the eight names `"x0"` to `"x7"`. `zip` pairs each name with the matching entry of `fixture["eta"]`, and `dict` turns the pairs into a dictionary, so that `ETA["x4"]` is -1 and `ETA["x0"]` is $+1$.

```
# gamma["x4"] is gamma^(x4), a 16 x 16 array of whole numbers (int64)
gamma = {x: np.array(m, dtype=np.int64) for x, m in zip(COORDS, fixture["gamma"])}
I16 = np.eye(16, dtype=np.int64)  # the identity matrix 1
```

The braces with `for` inside build a dictionary in one line (a **dictionary comprehension**): for each name `x` and stored matrix `m` (a list of 16 lists of 16 numbers), the entry `gamma[x]` is the numpy array of `m` with whole-number entries (`int64`, 64-bit integers). Whole numbers make every product below exact. `np.eye(16)` is the 16×16 identity matrix.

```
def product(directions):
    """The matrix product gamma^(d1) gamma^(d2) ... in the order of the list."""
    result = I16
    for d in directions:
        result = result @ gamma[d]  # @ is the matrix product
    return result
```

`product(["x0", "x1"])` starts with the identity and multiplies from the right by each listed gamma in turn; `@` is numpy's matrix product.

```
RECORD = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
lead = json.loads(repository_file(RECORD).read_text(encoding="utf-8"))
LEAD = {c["name"]: c for c in lead["checks"]}  # check name -> its entry
```

The lead report of the Revision record is read in the same way. Its entry `"checks"` is a list of checks, each a dictionary with a name, a verdict and a detail text; `LEAD` makes them findable by name.

```
def lead_passed(name):
    """True when the Revision record has the check name with the verdict PASS."""
    return LEAD.get(name, {}).get("verdict") == "PASS"
```

`LEAD.get(name, {})` returns the check's entry, or an empty dictionary if the record has no such check; `.get("verdict")` then returns its verdict or nothing; `==` compares with `"PASS"`. The function is therefore true only when the record holds the named check with the verdict PASS.

```
say(f"coordinates: {' '.join(COORDS)}")
say("eta: " + " ".join(f"{ETA[x]:+d}" for x in COORDS))
say(f"lead record: {lead['summary']['passed']} of {lead['summary']['total']} "
    "checks passed")
```

A string followed by `.join(list)` joins the strings of the list with that string between them; here the string is one blank. In `f"{ETA[x]:+d}"` the format `+d` prints a whole number with its sign. The third line reads the record's summary. Out [2] shows the eight coordinates in the author's order, the signs $+1, +1, +1, +1, -1, -1, -1, -1$, and "lead record: 12 of 12 checks passed".

In [3]: the eight real gammas and the Clifford relation.

```
shapes_ok = len(gamma) == 8 and all(g.shape == (16, 16) for g in gamma.values())
entries_ok = all(set(np.unique(g)) <= {-1, 0, 1} for g in gamma.values())
```

`len(gamma)` counts the entries of the dictionary; `g.shape` is the pair (rows, columns) of an array; `all(...)` is true when the test inside is true for every item. `np.unique(g)` lists the different entries of `g`, `set(...)` makes a set of them, and `<=` asks whether that set lies inside the set $\{-1, 0, 1\}$: every entry is -1, 0 or $+1$, so every gamma is real.

```
# a signed permutation matrix: exactly one nonzero entry in every row and column
permutation_ok = all((np.abs(g).sum(axis=0) == 1).all()
                     and (np.abs(g).sum(axis=1) == 1).all() for g in gamma.values())
check(shapes_ok and entries_ok and permutation_ok,
      "eight real 16 x 16 gamma matrices, entries -1, 0, +1, signed permutations")
```

`np.abs(g)` replaces every entry by its size; `.sum(axis=0)` adds each column and `.sum(axis=1)` each row; `== 1` compares every sum with 1 and `.all()` asks that all comparisons hold. Since the entries are 0 or ± 1, a sum of sizes equal to 1 means exactly one nonzero entry. The check combines the three tests with `and`.

```
clifford_ok = all(
    np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                   2 * (ETA[a] if a == b else 0) * I16)
    for a in COORDS for b in COORDS)  # all 64 ordered pairs (a, b)
check(clifford_ok, "Clifford relation gamma^a gamma^b + gamma^b gamma^a = "
      "2 eta^ab 1 for all 64 pairs, signature (4,4)")
```

For each of the $8 \times 8 = 64$ ordered pairs, `np.array_equal` compares the anticommutator with $2\eta^{ab}1$ exactly; `(ETA[a] if a == b else 0)` is η_{aa} on the diagonal and 0 otherwise. Whole-number arithmetic makes this an exact proof of the Clifford relation for the stored matrices.

```
check(all(np.array_equal(gamma[x].T, ETA[x] * gamma[x]) for x in COORDS),
      "symmetry pattern (gamma^a)^T = eta_aa gamma^a")
```

`.T` is the transpose. The check proves the symmetry pattern of Section 21.7. Out [3] shows the three PASS lines.

In [4]: the heat maps of the eight gammas.

```
from matplotlib.colors import LinearSegmentedColormap

# blue for -1, almost white for 0, red for +1
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
```

A **colour map** turns numbers into colours. `from_list` makes one that runs through three colours given as hexadecimal codes (red, green and blue amounts): blue at the lowest value, almost white in the middle, red at the highest.

```
def heat_map(ax, matrix, title, row_label=True):
    """Draw a 16 x 16 matrix with entries from -1 to +1 (blue -1, pale 0, red +1).
    Rows and columns are numbered from 1; black lines separate the two halves."""
    image = ax.imshow(matrix, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
```

```
    ax.set_xlabel("column")
    if row_label:
        ax.set_ylabel("row")
    ax.grid(False)
    return image
```

`ax` is one panel of a figure. `ax.imshow` draws the matrix as a grid of coloured squares, with -1 (`vmin`) blue and $+1$ (`vmax`) red. Python counts rows and columns from 0, so the tick positions 0, 7 and 15 get the labels 1, 8 and 16. `axhline` and `axvline` draw a horizontal and a vertical line at 7.5, between rows (columns) 8 and 9, which splits the matrix into its four 8×8 blocks. The axis labels are set (the row label only when asked), the grid is switched off, and the drawn image is returned for the colour bar.

```
fig, axes = plt.subplots(2, 4, figsize=(12.0, 6.6))
for k, (ax, x) in enumerate(zip(axes.flat, COORDS)):
    kind = "space-like" if ETA[x] == 1 else "time-like"
    image = heat_map(ax, gamma[x], f"$\\gamma^{{({x})}}$, {kind}",
                     row_label=(k % 4 == 0))
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.7, label="matrix entry")
```

`plt.subplots(2, 4, ...)` makes a figure with two rows of four panels, 12 by 6.6 inches. `axes.fla t` runs through the panels row by row, `zip` pairs them with the coordinates, and `enumerate` adds the counter `k` = 0, 1, 2, ... The title is LaTeX: in an f-string a doubled brace prints one brace and a doubled backslash prints one backslash, so for `x = "x4"` the title is $\gamma^{(x4)}$ followed by "time-like". `k % 4` is the remainder of k divided by 4, which is 0 for the first panel of each row: only those get a row label. `fig.colorbar` adds one colour bar for all panels.

```
save_figure(fig, "eight_gammas",
            "The eight real $16 \\times 16$ gamma matrices of the Revision record, "
            "in the author's coordinate order $x1, x2, x3$ (ordinary space), $x4$ "
            "(the time), $x5, x6, x7$ (the deflating extra times), $x0$ (the hidden "
            "direction); horizontal axis the column, vertical axis the row, colour "
            "the entry (red $+1$, blue $-1$, pale $0$). Every matrix has exactly one "
            "nonzero entry in each row and column, and only in the two off-diagonal "
            "$8 \\times 8$ blocks: each gamma exchanges the two halves of the "
            "spinor.")
```

Figure 21a.1, saved with the caption written in the following string lines, which Python joins into one string. **What the figure shows.** Eight panels, one per gamma. Each panel has exactly one coloured square in every row and column, and all of them lie in the upper-right and lower-left blocks: every gamma maps the first eight components into the last eight and back. These two halves are the chiral halves, where $\Gamma = -1$ and $\Gamma = +1$; that every gamma exchanges them is the reason why Γ anticommutes with every gamma.

In [5]: C, Γ, B and the generators.

```
C = product(["x0", "x1", "x2", "x3"])  # the charge matrix (the author's sigma16)
Gamma = product(["x0", "x1", "x2", "x3", "x4", "x5", "x6", "x7"])  # the chirality
B = -1j * (C @ gamma["x4"])  # B = -i C gamma^(x4), a complex matrix
```

The three matrices of Section 21.7. Python writes the imaginary unit i as `1j`, so `-1j * (...)` multiplies every entry by $-i$; `B` is therefore an array of complex numbers.

```
pairs = [(a, b) for i, a in enumerate(COORDS) for b in COORDS[i + 1:]]  # 28 pairs
S_gen = {(a, b): (gamma[a] @ gamma[b] - gamma[b] @ gamma[a]) / 4 for a, b in pairs}
```

`COORDS[i + 1:]` is the part of the list after position `i`, so each pair (a, b) with a before b appears once: $8 \cdot 7/2 = 28$ pairs. `S_gen` stores the generator $S^{ab} = \frac{1}{4}(\gamma^a\gamma^b - \gamma^b\gamma^a)$ for each pair; division by 4 makes the entries decimal numbers (they are 0 and $\pm\frac{1}{2}$).

```
same_as_record = (np.array_equal(C, np.array(fixture["C"]))
                  and np.array_equal(Gamma, np.array(fixture["Gamma"])))
gamma_diag = np.diag(np.concatenate([-np.ones(8), np.ones(8)])).astype(np.int64)
check(same_as_record and np.array_equal(Gamma, gamma_diag),
      "C and Gamma equal the record; Gamma = diag(-1 (8 times), +1 (8 times))")
```

The record also stores C and Γ; the first line compares them with the products just computed. `np.concatenate` joins eight -1 and eight $+1$ into one list, `np.diag` makes the diagonal matrix with that list on its diagonal, and `.astype(np.int64)` turns its entries into whole numbers. The check proves $\Gamma = \mathrm{diag}(-1_8, 1_8)$.

```
check(np.array_equal(C.T, C) and np.array_equal(C @ C, I16)
      and all(np.isrealobj(s) for s in S_gen.values()) and lead_passed(
          "representation_real"),
      "gamma^a, C (symmetric, C^2 = 1) and all 28 S^ab are real",
      record=f"{RECORD}, check representation_real")
```

This proves (R3)'s $C^T = C$ and $CC = 1$, checks that every generator is stored as real numbers (`np.isrealobj` is true for an array without complex entries), and asks that the lead record holds the check `representation_real` with the verdict PASS; the second printed line names that record check.

```
check(np.array_equal(B.real, np.zeros((16, 16))) and np.allclose(B, B.conj().T)
      and np.allclose(B @ B, np.eye(16)) and lead_passed("B_imaginary_hermitian"),
      "B = -i C gamma^(x4) is purely imaginary, Hermitian, B^2 = 1",
      record=f"{RECORD}, check B_imaginary_hermitian")
```

`B.real` is the array of the real parts, which must all be zero; `B.conj().T` is the conjugate transpose $B^\dagger$, and `np.allclose` compares two arrays entry by entry and accepts a difference up to 10^{-8} plus 10^{-5} times the size of the corresponding entry of the second array (numpy's default tolerances), far above rounding errors and far below the differences that matter here; `B @ B` must be the identity. Out [5] shows the three PASS lines, two of them with the record checks they reproduce.

In [6]: every matrix that obeys the condition, solved exactly.

```
import sympy as sp  # exact algebra
from sympy.polys.matrices import DomainMatrix  # exact matrices over the rationals
```

`sympy` computes with exact numbers and symbols. A `DomainMatrix` is sympy's fast matrix of exact numbers from a chosen **domain**: the rational numbers `QQ` (fractions) or the integers `ZZ`.

```
def conjugation_system(s, directions=COORDS):
    """Coefficient matrix of M conj(gamma^a) - s gamma^a M = 0 for the 256
    unknowns M_rc (read row by row), one block of 256 equations per direction."""
    blocks = []
    for x in directions:
        G_conj = np.conj(gamma[x]).astype(np.int64)  # (gamma^a)*, equal to gamma^a
        blocks.append(np.kron(I16, G_conj.T) - s * np.kron(gamma[x], I16))
    return np.vstack(blocks)
```

The unknown matrix M is read row by row as one column y of 256 numbers, $y_{16r+c} = M_{rc}$ (rows and columns counted from 0). The entry (r, c) of MG is $\sum_k M_{rk} G_{kc}$; as a linear function of y it is row $16r + c$ of the **Kronecker product** $1 \otimes G^T$ (`np.kron(I16, G_conj.T)`, the 256×256 block matrix whose block (r, r') is $\delta_{rr'} G^T$). The entry (r, c) of GM is $\sum_k G_{rk} M_{kc}$, row $16r + c$ of $G \otimes 1$ (`np.kron(gamma[x], I16)`). So each direction contributes 256 equations $(1 \otimes G^{*T} - s\, G \otimes 1)\, y = 0$. `np.conj` takes the complex conjugate, which changes nothing for the real gammas but keeps the code faithful to the condition $M(\gamma^a)^* = s\gamma^a M$. `np.vstack` stacks the blocks into one tall coefficient matrix with $8 \cdot 256 = 2048$ rows.

```
def exact_solutions(A):
    """A basis of all solutions y of A y = 0, each returned as a 16 x 16 matrix."""
    null = DomainMatrix.from_list(A.tolist(), sp.QQ).nullspace().to_Matrix()
    return [np.array(null.row(k).tolist()[0], dtype=object).reshape(16, 16)
            for k in range(null.rows)]
```

`A.tolist()` turns the numpy array into nested Python lists; `DomainMatrix.from_list(..., sp.QQ)` makes an exact matrix of rational numbers; `.nullspace()` computes, by exact Gaussian elimination, a basis of all solutions of $Ay = 0$ (one solution per row), and `.to_Matrix()` turns it into an ordinary sympy matrix. The list comprehension takes each basis row (`null.row(k)`), turns it into a numpy array of exact sympy numbers (`dtype=object`) and folds the 256 numbers back into a 16×16 matrix with `.reshape(16, 16)`. No rounding is involved anywhere: a solution can be neither hidden nor invented.

```
def proportional(X, Y):
    """True when X and Y are nonzero and X is a number times Y."""
    X = np.array(X, dtype=float).reshape(-1)
    Y = np.array(Y, dtype=float).reshape(-1)
    return bool(X.any() and Y.any() and np.linalg.matrix_rank(np.array([X, Y])) == 1)
```

Both matrices are written as rows of 256 decimal numbers (`reshape(-1)` makes one long row). Two nonzero rows are multiples of each other exactly when the 2×256 matrix made of them has **rank** 1 (only one independent row); `X.any()` is true when some entry is nonzero.

```
A_same, A_reversed = conjugation_system(+1), conjugation_system(-1)
M_same, M_reversed = exact_solutions(A_same), exact_solutions(A_reversed)
say(f"s = +1: {A_same.shape[0]} equations, {len(M_same)} independent solution(s)")
say(f"s = -1: {A_reversed.shape[0]} equations, {len(M_reversed)} independent "
    "solution(s)")
```

The two systems are built and solved; a comma on both sides assigns two values at once. `A_same.shape[0]` is the number of rows (equations). Out [6] prints 2048 equations and 1 independent solution for each sign.

```
check(len(M_same) == 1 and proportional(M_same[0], I16)
      and lead_passed("intertwiners_same_mass"),
      "s = +1 (same mass): the solutions are the multiples of the identity",
      record=f"{RECORD}, check intertwiners_same_mass")
check(len(M_reversed) == 1 and proportional(M_reversed[0], Gamma)
      and lead_passed("intertwiners_reversed_mass"),
      "s = -1 (mass reversed): the solutions are the multiples of Gamma",
      record=f"{RECORD}, check intertwiners_reversed_mass")
```

The solution space of each sign has dimension 1, and its basis matrix is a multiple of 1 (for $s = +1$) or of Γ (for $s = -1$): Theorem CC (a), reproducing the two record checks.

In [7]: each equation halves the solution space.

```
dimensions = {}
for s in (+1, -1):
    dims = [256]  # k = 0: no equation, every matrix is a solution
    for k in range(1, 9):
        A = conjugation_system(s, COORDS[:k])  # the first k directions only
        rank = DomainMatrix.from_list(A.tolist(), sp.ZZ).rank()  # exact rank
        dims.append(256 - rank)
    dimensions[s] = dims
    say(f"s = {s:+d}: dimensions for k = 0..8: {dims}")
```

For each sign and each $k = 1, \ldots, 8$ (`range(1, 9)` stops before 9), the system of the first k directions (`COORDS[:k]`, the first k names) is built. Its **rank**, the number of independent equations, is computed exactly over the integers; the number of free unknowns, the dimension of the solution space, is 256 minus the rank. `dims.append` adds it to the list. Out [7] prints $[256, 128, 64, 32, 16, 8, 4, 2, 1]$ for both signs.

```
check(dimensions[+1] == dimensions[-1] == [256 // 2 ** k for k in range(9)],
      "each gamma equation halves the solution space: 256, 128, ..., 2, 1")
```

`256 // 2 ** k` is the whole-number quotient $256/2^k$ (`**` is a power, `//` division without remainder). Python reads `a == b == c` as "a equals b and b equals c".

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
k_values = np.arange(9)
ax.semilogy(k_values, dimensions[+1], "o-", markersize=9,
            label="$s = +1$: $M$ commutes with the first $k$ gammas")
ax.semilogy(k_values, dimensions[-1], "s--", markersize=5,
            label="$s = -1$: $M$ anticommutes with the first $k$ gammas")
```

`np.arange(9)` is the array 0, 1, ..., 8. `semilogy` draws a curve with a logarithmic vertical axis, on which halving is a constant step. The style string `"o-"` means circles joined by a line, and the

style string made of the letter s and two hyphens means small squares joined by a dashed line; the squares are drawn smaller so that the circles of the first curve remain visible around them. `label` is the text of the legend.

```
ax.set_xticks(k_values, ["0"] + [f"{k}\n(+{x})" for k, x in zip(range(1, 9), COORDS)])
ax.set_yticks([1, 2, 4, 8, 16, 32, 64, 128, 256],
              ["1", "2", "4", "8", "16", "32", "64", "128", "256"])
ax.set_xlabel("number $k$ of gamma equations imposed (and the direction added)")
ax.set_ylabel("dimension of the solution space")
ax.set_title("Every equation halves the space of conjugation matrices")
ax.legend()
```

The tick labels of the horizontal axis show k and, below it (`\n` is a line break), the direction added. The vertical ticks are the powers of 2. Axis labels, a title and the legend follow.

```
save_figure(fig, "halving_solutions",
            "The dimension of the space of $16 \\times 16$ matrices $M$ that obey "
            "$M(\\gamma^a)^\\ast = s\\,\\gamma^a M$ for the first $k$ directions only "
            "(computed exactly as $256$ minus the rank of the equations), for "
            "$s = +1$ (circles) and $s = -1$ (squares); horizontal axis $k$ with "
            "the direction added, vertical axis the dimension on a logarithmic "
            "scale. Both sequences are $256/2^k$: every new gamma halves the "
            "freedom, and with all eight only one direction is left, the multiples "
            "of $1$ ($s = +1$) or of $\\Gamma$ ($s = -1$).")
```

Figure 21a.2. **What the figure shows.** Two identical straight staircases on the logarithmic axis, from 256 down to 1: every gamma equation removes exactly half of the remaining freedom, as Section 21.9 explains for the first step, until only the multiples of 1 or of Γ are left.

In [8]: the two charge-conjugation matrices and their transposition rules.

```
calC_plus = C  # M = 1
calC_minus = Gamma @ C  # M = Gamma
inv_plus = np.linalg.inv(calC_plus)  # the inverse matrix
inv_minus = np.linalg.inv(calC_minus)
```

$\mathcal{C} = MC$ with $M = 1$ and $M = \Gamma$ (Theorem CC (b)). `np.linalg.inv` computes the inverse matrix in decimal numbers.

```
check(all(np.allclose(inv_plus @ gamma[x] @ calC_plus, -gamma[x].T) for x in COORDS)
      and np.array_equal(calC_plus.T, calC_plus)
      and lead_passed("charge_conjugation_matrix_plus"),
      "calC_+ = C: calC_+^-1 gamma^a calC_+ = -(gamma^a)^T, real, symmetric",
      record=f"{RECORD}, check charge_conjugation_matrix_plus")
check(all(np.allclose(inv_minus @ gamma[x] @ calC_minus, gamma[x].T) for x in COORDS)
      and np.array_equal(calC_minus.T, calC_minus)
      and lead_passed("charge_conjugation_matrix_minus"),
      "calC_- = Gamma C: calC_-^-1 gamma^a calC_- = +(gamma^a)^T, real, symmetric",
      record=f"{RECORD}, check charge_conjugation_matrix_minus")
```

The transposition rules of Theorem CC (c) for all eight gammas, and the symmetry of both matrices (they are whole-number arrays, hence real).

```
def transposition_system(zeta):
    """Coefficient matrix of gamma^a X - zeta X (gamma^a)^T = 0 for all a."""
    return np.vstack([np.kron(gamma[x], I16) - zeta * np.kron(I16, gamma[x])
                      for x in COORDS])
```

The converse part of (c). With X read row by row as before, γX is `np.kron(gamma[x], I16)` applied to it, and the entry (r, c) of $X\gamma^T$ is $\sum_k X_{rk}\gamma_{ck}$, which is `np.kron(I16, gamma[x])` applied to it (the transpose of γ^T is γ).

```
X_minus = exact_solutions(transposition_system(-1))  # expected: multiples of C
X_plus = exact_solutions(transposition_system(+1))  # expected: multiples of Gamma C
check(len(X_minus) == 1 and proportional(X_minus[0], calC_plus)
      and len(X_plus) == 1 and proportional(X_plus[0], calC_minus),
      "gamma^a X = zeta X (gamma^a)^T: zeta = -1 only C, zeta = +1 only Gamma C")
```

Both systems are solved exactly: each has a one-dimensional solution space, spanned by C for $\zeta = -1$ and by ΓC for $\zeta = +1$. So the usual textbook definition by transposition rules gives the same two matrices. Out [8] shows the three PASS lines.

In [9]: the heat maps of the two matrices.

```
fig, axes = plt.subplots(1, 3, figsize=(11.5, 4.2))
heat_map(axes[0], calC_plus, r"$\mathcal{C}_+ = C$ (same mass)")
heat_map(axes[1], Gamma, r"chirality $\Gamma$", row_label=False)
image = heat_map(axes[2], calC_minus, r"$\mathcal{C}_- = \Gamma C$ (mass reversed)",
                 row_label=False)
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8, label="matrix entry")
```

One row of three panels. A string that starts with `r` is a **raw string**, in which a backslash is an ordinary character, convenient for LaTeX titles. The three matrices are drawn with the helper of In [4].

```
save_figure(fig, "conjugation_matrices",
            "Heat maps of the two charge-conjugation matrices and of the chirality "
            "that relates them: $\\mathcal{C}_+ = C$ (left, the conjugation that "
            "keeps the mass), $\\Gamma$ (middle) and $\\mathcal{C}_- = \\Gamma C$ "
            "(right, the conjugation that reverses the mass); horizontal axis the "
            "column, vertical axis the row, colour the entry (red $+1$, blue $-1$, "
            "pale $0$). $C$ and $\\Gamma C$ have their entries only in the two "
            "diagonal blocks, and they differ only by the sign of the upper half, "
            "where $\\Gamma = -1$.")
```

Figure 21a.3. **What the figure shows.** C and ΓC have one entry per row and column, all inside the two diagonal blocks (they are products of an even number of gammas, which keep the halves apart). The middle panel is the diagonal Γ: blue in the upper half, red in the lower. The right panel is the left one with the colours of the upper eight rows exchanged.

In [10]: the two reality conditions.

```
check(np.array_equal(I16 @ np.conj(I16), I16)
      and np.array_equal(Gamma @ np.conj(Gamma), I16)
      and lead_passed("majorana_conditions_consistent"),
      "M M* = 1 for M = 1 and M = Gamma: both reality conditions are consistent",
      record=f"{RECORD}, check majorana_conditions_consistent")
```

The consistency condition $MM^* = 1$ of Section 21.10 for both matrices, reproducing the record check.

```
rows = np.arange(1, 17)  # the component numbers 1, ..., 16
v = (np.cos(rows) + 0.5) + 1j * np.sin(2.0 * rows)  # a fixed complex column
psi_real = (v + np.conj(v)) / 2  # satisfies Psi = Psi*
psi_gamma = (v + Gamma @ np.conj(v)) / 2  # satisfies Psi = Gamma Psi*
```

`v` is a fixed complex column whose r-th entry is $\cos r + 0.5 + i \sin 2r$: an arbitrary but reproducible choice. $(v+v^*)/2$ is the real part of v, so it is real. For the second: $\Gamma\big((v+\Gamma v^*)/2\big)^* = (\Gamma v^* + \Gamma\Gamma v)/2 = (v + \Gamma v^*)/2$ (Γ is real and $\Gamma\Gamma = 1$), so `psi_gamma` obeys $\Psi = \Gamma\Psi^*$.

```
check(np.allclose(psi_real, np.conj(psi_real))
      and np.allclose(psi_gamma, Gamma @ np.conj(psi_gamma)),
      "the two example fields satisfy Psi = Psi* and Psi = Gamma Psi*")
```

The check confirms both conditions with numbers.

```
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.0), sharey=True)
for ax, psi, title in ((axes[0], psi_real, r"$\Psi = \Psi^*$ (condition of "
                        r"$\mathcal{C}_+$)"),
                       (axes[1], psi_gamma, r"$\Psi = \Gamma\Psi^*$ (condition of "
                        r"$\mathcal{C}_-$)")):
    ax.bar(rows - 0.2, psi.real, width=0.4, label="real part")
    ax.bar(rows + 0.2, psi.imag, width=0.4, label="imaginary part")
    ax.axvline(8.5, color="black", linewidth=0.8)
    ax.set_xticks([1, 4, 8, 12, 16])
    ax.set_xlabel("component number")
    ax.set_title(title)
    ax.legend(loc="lower left")
axes[0].set_ylabel("value of the component")
```

Two panels with a common vertical axis (`sharey=True`). The loop runs over two triples (panel, field, title). `ax.bar` draws bars: the real parts shifted 0.2 to the left of each component number, the imaginary parts 0.2 to the right, each 0.4 wide. A vertical line at 8.5 separates the two halves. Ticks, labels, title and legend (in the lower left corner) follow.

```
save_figure(fig, "reality_conditions",
            "One field satisfying each reality (Majorana) condition, made from the "
            "same complex column: left $\\Psi = \\Psi^\\ast$ (every imaginary part is "
            "zero), right $\\Psi = \\Gamma\\Psi^\\ast$ (components 1 to 8, where "
            "$\\Gamma = -1$, are purely imaginary; components 9 to 16, where "
```

```
"$\\Gamma = +1$, are real); horizontal axis the component number, "
"vertical axis the real part (left bar of each pair) and the imaginary "
"part (right bar). Both conditions can be imposed, because "
"$MM^\\ast = 1$ for $M = 1$ and for $M = \\Gamma$.")
```

Figure 21a.4. **What the figure shows.** On the left every second bar of a pair is missing (zero imaginary parts). On the right components 1 to 8 have only imaginary parts and components 9 to 16 only real parts, exactly as the condition $\Psi = \Gamma\Psi^*$ demands component by component.

In [11]: the exact solution of the record in the author's metric.

```
THEORY = {f["key"]: f["wl"] for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
PY_THEORY = {c["name"]: c["verdict"].upper() for c in json.loads(repository_file(
    "Revision/theory/reports/python-field-theory.json").read_text(
    encoding="utf-8"))["checks"]}
```

The field-theory record holds a list of formulas, each with a key and its text in Wolfram notation (`"wl"`); `THEORY` maps each key to its text. The independent sympy report of the same record holds checks whose verdicts are written in small letters; `.upper()` turns them into capitals, so that `PY_THEORY[name]` is `"PASS"` for a passed check.

```
# the left side of the record's field equation, word for word (right side m + U')
FIELD_EQUATION_LEFT = (
    "Tan[z] g[x0] d0 Psi"
    " + E^-a4[x4] Sin[z]^(-1/6) (g[x1] d1 + g[x2] d2 + g[x3] d3) Psi + g[x4] d4 Psi"
    " + E^a4[x4] Sin[z]^(-1/6) (g[x5] d5 + g[x6] d6 + g[x7] d7) Psi"
    " + 3 H g[x0] Psi")
# item (i) of the record's formula exact_solutions, word for word
SOLUTION_I = (
    "(i) U = 0: Psi = Sin[z]^al (Cosh[k x4] + Sinh[k x4]/k M) chi, M = -m g[x4]"
    " + 3 H (2 al + 1) g[x4].g[x0], k^2 = 9 H^2 (2 al + 1)^2 - m^2 (any a4)")
```

Two strings, each written as several pieces in parentheses that Python joins: the left side of the field equation and item (i) of the exact solutions, exactly as this book writes them in Section 21.11, but in the record's notation (`g[x4]` is $\gamma^{(x4)}$, `d4` is ∂_4, `al` is α, `E^a4[x4]` is e^{a_4}, the dot is the matrix product).

```
left, right = THEORY["field_equation"].split(" = ")  # the two sides
say("record, exact solution " + SOLUTION_I)
check(left == FIELD_EQUATION_LEFT and right.startswith("(m + U")
      and THEORY["exact_solutions"].startswith(SOLUTION_I + "; "),
      "the record states the field equation and the exact solution (i) used here",
      record="Revision/theory/field-theory.json, formulas field_equation and "
      "exact_solutions")
```

`split(" = ")` cuts the record's equation at the equals sign into its two sides. The check compares the left side word for word, asks that the right side starts with the factor $m + U'$, and asks that the record's text of the exact solutions starts with item (i) followed by a semicolon. If the record is ever changed, this check fails and the notebook stops.

```
x4, z = sp.symbols("x4 z", real=True)  # the time x4 and the angle z = 6 H x0
H, alpha, m = sp.Rational(1, 6), 1, 2  # the parameters chosen above
b = 3 * H * (2 * alpha + 1)  # b = 3/2
w = sp.sqrt(m ** 2 - b ** 2)  # k = i w with w = sqrt(7)/2
```

`sp.symbols` makes two symbols that sympy treats as unknown real numbers. `sp.Rational(1, 6)` is the exact fraction 1/6. Then $b = 3 \cdot \frac{1}{6} \cdot 3 = \frac{3}{2}$ and $w = \sqrt{4 - 9/4} = \sqrt{7}/2$, both exact.

```
G = {x: sp.Matrix(gamma[x].tolist()) for x in COORDS}  # exact sympy matrices
Gamma_s = sp.Matrix(Gamma.tolist())
```

Exact sympy copies of the gammas and of Γ.

```
def M_of(mass):
    """M_m = -m gamma^(x4) + b gamma^(x4) gamma^(x0)."""
    return -mass * G["x4"] + b * G["x4"] * G["x0"]
```

The matrix M_m of the solution; for sympy matrices `*` is the matrix product.

```
def E(mass, field):
    """E_m[Psi] = gamma^(x4) d4 Psi + 6H tan z gamma^(x0) dz Psi + 3H gamma^(x0) Psi
    - m Psi (the field equation of the record for fields of x4 and z only)."""
    tan_z = sp.sin(z) / sp.cos(z)  # tan z written as sin z / cos z
    return (G["x4"] * field.diff(x4) + 6 * H * tan_z * G["x0"] * field.diff(z)
            + 3 * H * G["x0"] * field - mass * field)
```

The left side of the field equation for a field of x_4 and z only (step 4 of Section 21.11); `field.diff(x4)` is the exact partial derivative of every component.

```
# a fixed complex column chi with exact entries (j + i (17 - j)) / 16, j = 1..16
chi = sp.Matrix([sp.Rational(j, 16) + sp.I * sp.Rational(17 - j, 16)
                 for j in range(1, 17)])
U = sp.cos(w * x4) * sp.eye(16) + sp.sin(w * x4) / w * M_of(m)  # real 16 x 16
Psi = sp.sin(z) ** alpha * U * chi
```

`chi` is a column of 16 exact complex numbers, $(j + i(17 - j))/16$; `sp.I` is i. `U` is $\cos(wx_4)\,1 + \sin(wx_4)M_m/w$, the form the solution takes for imaginary k; it is real. `Psi` is the solution $\sin z\, U\chi$.

```
check(E(m, Psi).expand() == sp.zeros(16, 1)
      and sp.simplify(M_of(m) ** 2 + w ** 2 * sp.eye(16)) == sp.zeros(16, 16)
      and PY_THEORY["exact_solution_family_x4_x0"] == "PASS",
      "the record's exact solution solves the field equation (mass m = 2)",
      record="Revision/theory/reports/python-field-theory.json, check "
      "exact_solution_family_x4_x0")
```

`.expand()` multiplies out every product, and the result must be the zero column: the field equation holds exactly. The second test is $M_m^2 = -w^2 = k^2$ (step 2 of Section 21.11). Out [11] prints the record's text of the solution and the two PASS lines.

In [12]: the two conjugates, exactly.

```
Psi_conj = Psi.conjugate()  # Psi*: every entry complex-conjugated (x4, z are real)
Psi_gamma_conj = Gamma_s * Psi_conj  # Gamma Psi*
```

`.conjugate()` takes the complex conjugate of every component; since `x4` and `z` were declared real, sympy conjugates only the numbers i in χ.

```
def vanishes(vector):
    """True when every component expands to exactly zero (sympy multiplies out
    every product; cos z / cos z cancels automatically)."""
    return vector.expand() == sp.zeros(16, 1)


check(vanishes(E(m, Psi_conj)) and vanishes(E(-m, Psi_gamma_conj)),
      "Psi* solves the equation with mass +2, Gamma Psi* with mass -2 (exact)")
check(not vanishes(E(-m, Psi_conj)) and not vanishes(E(m, Psi_gamma_conj)),
      "negative controls: Psi* fails with -2, Gamma Psi* fails with +2")
```

The first check proves what Section 21.11 derived: Ψ^* solves the equation with the same mass $+2$, $\Gamma\Psi^*$ the one with the reversed mass -2. The second is a **negative control**, a test that must fail if the computation is meaningful: with the masses exchanged the equations do not hold.

In [13]: the conjugates along the time, numerically.

```
times = np.linspace(0.0, 8.0, 161)  # 161 times x4 from 0 to 8
z_fixed = np.pi / 4
```

`np.linspace(0, 8, 161)` gives 161 equally spaced times from 0 to 8 (steps of 0.05). The angle is fixed at $z = \pi/4$.

```
def numeric(expression):
    """Evaluate a sympy 16 x 1 expression of x4 at z = pi/4 for all times."""
    f = sp.lambdify((x4, z), expression, "numpy")
    return np.array([np.array(f(t, z_fixed), dtype=complex).reshape(16)
                     for t in times])


def size(expression):
    """|E| at every time: the length of the 16-component column."""
    return np.linalg.norm(numeric(expression), axis=1)
```

`sp.lambdify` turns an exact expression into a fast numerical function of x_4 and z. `numeric` evaluates it at every time and stacks the 161 columns into an array of shape 161×16. `size` computes for each time the length of the column, the square root of the sum of the squared moduli of its 16 entries (`np.linalg.norm` along `axis=1`, that is along each row of the array).

```
curves = {
    r"$|E_{+2}[\Psi^*]|$ (same mass)": size(E(m, Psi_conj)),
    r"$|E_{-2}[\Gamma\Psi^*]|$ (mass reversed)": size(E(-m, Psi_gamma_conj)),
    r"$|E_{-2}[\Psi^*]|$ (control)": size(E(-m, Psi_conj)),
```

```
    r"$|E_{+2}[\Gamma\Psi^*]|$ (control)": size(E(m, Psi_gamma_conj)),
}
floor = 1e-17  # added before taking the logarithm, so that 0 can be drawn
values = list(curves.values())
```

The four sizes, stored under their legend texts: the two solutions and the two controls. On a logarithmic axis zero cannot be drawn, so `floor` (10^{-17}) will be added. `values` is the list of the four arrays, in the order written.

```
say(f"largest |E| of the two solutions below 1e-12: "
    f"{max(values[0].max(), values[1].max()) < 1e-12}")
say(f"smallest |E| of the two controls: {min(values[2].min(), values[3].min()):.2f}")
field_size = size(Psi_conj)  # |Psi*| = |Gamma Psi*| at every time (Gamma permutes)
```

The first line prints True: both solutions give sizes below 10^{-12} at every time (rounding only). The second prints the smallest size of the controls, 9.67 (`:.2f` prints two decimals). `field_size` is the length of Ψ^* at every time; Γ only changes signs of components, so $\Gamma\Psi^*$ has the same length.

```
check(max(values[0].max(), values[1].max()) < 1e-12
      and np.allclose(values[2], 4 * field_size)
      and np.allclose(values[3], 4 * field_size),
      "numerically: the solutions give |E| below 1e-12, the controls |E| = 4 |field|")
```

The controls are exactly four times the length of the field, as Section 21.11 predicted ($E_{-2}[\Psi^*] = 4\Psi^*$).

```
B_c = B.astype(complex)  # the charge density is J^(x4) = Psi^dagger B Psi
density = {}
for field, label in ((Psi, r"$\Psi$"), (Psi_conj, r"$\Psi^*$"),
                     (Psi_gamma_conj, r"$\Gamma\Psi^*$")):
    values_f = numeric(field)  # shape (161, 16): the field at every time
    density[label] = np.einsum("tr,rc,tc->t", np.conj(values_f), B_c, values_f).real
d_psi, d_conj, d_gconj = density.values()
```

For each of the three fields the values at all times are computed, and `np.einsum("tr,rc,tc->t", ...)` forms, for each time t, the sum $\sum_{r,c} \Psi^*_r B_{rc} \Psi_c$: the letters name the indices of the three arrays, and the index after the arrow is the one kept. `.real` keeps the real part (the imaginary part is zero up to rounding, since B is Hermitian). The last line unpacks the three density arrays.

```
check(np.allclose(d_conj, -d_psi) and np.allclose(d_gconj, d_psi)
      and np.ptp(d_psi) > 0.1,
      "charge density: Psi* has -J^(x4), Gamma Psi* has +J^(x4) at every time")
```

Ψ^* carries the opposite charge density at every time and $\Gamma\Psi^*$ the same, as the table of Section 21.10 says for commuting components. `np.ptp` (peak to peak) is the largest minus the smallest value; requiring it above 0.1 makes sure the density is not trivially zero or constant.

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for (label, values_d), style in zip(density.items(), ["-", "--", ":"]):
    axes[0].plot(times, values_d, style, linewidth=2, label=f"{label}")
axes[0].set_xlabel("time $x_4$")
axes[0].set_ylabel("charge density $\\Psi^\\dagger B\\Psi$ at $z = \\pi/4$")
axes[0].set_title("A solution and its two conjugates")
axes[0].legend(fontsize=8)
```

The left panel draws the three densities against the time, with a solid, a dashed and a dotted line (`density.items()` gives the pairs label and values).

```
for (label, values_e), style in zip(curves.items(), ["-", "--", "-.", ":"]):
    axes[1].semilogy(times, values_e + floor, style, label=label)
axes[1].set_xlabel("time $x_4$")
axes[1].set_ylabel("$|E|$ at $z = \\pi/4$ (log scale)")
axes[1].set_title("Which mass does each conjugate solve?")
axes[1].legend(fontsize=8)
```

The right panel draws the four sizes on a logarithmic axis, each raised by the floor.

```
save_figure(fig, "curved_solution_images",
            "Charge conjugation acting on an exact solution of the field equation "
            "in the author's metric ($H = 1/6$, $\\alpha = 1$, mass $m = 2$, valid "
            "for every history $a_4$). Left: the charge density "
            "$\\Psi^\\dagger B\\Psi$ of $\\Psi$ (solid), $\\Psi^\\ast$ (dashed) and "
            "$\\Gamma\\Psi^\\ast$ (dotted, on top of the solid curve) versus the "
            "time $x_4$ at $z = \\pi/4$ (pure numbers); $\\Psi^\\ast$ carries the "
            "opposite charge density. Right: the size $|E|$ of the left-hand "
            "side of the field equation versus $x_4$, logarithmic scale. $\\Psi^\\ast$ "
            "solves the equation with the same mass $+2$ and $\\Gamma\\Psi^\\ast$ the "
            "one with the reversed mass $-2$ ($|E|$ at rounding level, about "
            "$10^{-16}$), while the exchanged masses fail ($|E|$ equals four "
            "times the size of the field, about 10).")
```

Figure 21a.5. **What the figure shows.** Left: the dashed curve is the mirror image of the solid one in the horizontal axis (opposite charge density), and the dotted curve lies on the solid one. The densities are quadratic in $\cos(wx_4)$ and $\sin(wx_4)$ with $w = \sqrt{7}/2$, so they oscillate with the angular frequency $2w = \sqrt{7}$, a period of about 2.37. Right: the two solutions stay at the rounding level near 10^{-16} at the bottom, the two controls near 10 at the top: about seventeen orders of magnitude separate "solves" from "does not solve".

In [14]: the sign table of S and J.

```
def effective(Mmat, K, eps):
    """The matrix K' of the bilinear after Psi -> M Psi*, with the reorder sign eps."""
    return eps * (Mmat.conj().T @ K @ Mmat).T


def sign_of(Kp, K):
    """+1 if K' = K, -1 if K' = -K, 0 otherwise."""
    return 1 if np.allclose(Kp, K) else (-1 if np.allclose(Kp, -K) else 0)
```

`effective` is the rule $K' = \epsilon(M^\dagger K M)^T$ of Section 21.10. `sign_of` compares K' with K and $-K$ and returns the sign, or 0 when neither matches (which never happens here).

```
K_S = C.astype(complex)  # the scalar S = Psi^dagger C Psi
K_J = [-1j * (C @ gamma[x]) for x in COORDS]  # the currents J^a
measured = {}
```

The matrices of the scalar and of the eight currents; `measured` will collect the signs.

```
for label, Mmat in (("plus", I16), ("minus", Gamma)):
    for eps in (1, -1):
        sS = sign_of(effective(Mmat, K_S, eps), K_S)
        sJ = [sign_of(effective(Mmat, K, eps), K) for K in K_J]
        measured[f"{label},eps={eps}"] = [sS, sJ]
        kind = "commuting" if eps == 1 else "anticommuting"
        say(f"calC_{'+' if label == 'plus' else '-'}, {kind:13}: S -> {sS:+d} S, "
            f"J -> {sJ[0]:+d} J (all eight components alike: {len(set(sJ)) == 1})")
```

Two nested loops over the two matrices and the two kinds of components. For each case the sign of S and the list of the eight signs of the currents are stored under a key such as `"plus,eps=1"`, the spelling the record uses. The printed line uses a small conditional inside the f-string to write + or −, pads the kind to 13 characters (`{kind:13}`) so that the columns line up, and reports whether the eight current signs are all equal (`set(sJ)` has one element then). Out [14] prints the four rows of the table of Section 21.10.

```
detail = LEAD["bilinears_under_charge_conjugation"]["detail"]
recorded = json.loads(detail.split("measured: ", 1)[1])  # the record's own table
check(measured == recorded and lead_passed("bilinears_under_charge_conjugation"),
      "signs of S and J under calC_+ and calC_-, both statistics, equal the record",
      record=f"{RECORD}, check bilinears_under_charge_conjugation")
```

The record's check stores its own measured table in its detail text after the words "measured: ". `split("measured:  ", 1)[1]` takes the text after them, `json.loads` reads it as a dictionary, and the check demands that the notebook's table is identical to the record's.

In [15]: the signs of all 256 bilinears.

```
CASES = [("plus", I16, 1), ("plus", I16, -1), ("minus", Gamma, 1), ("minus", Gamma, -1)]
degree_signs = np.zeros((4, 9), dtype=int)  # rows: the four cases, columns: k
```

The four cases (matrix, kind of components) and an array of zeros with four rows and nine columns for the signs of the degrees $k = 0, \ldots, 8$.

```
for row, (label, Mmat, eps) in enumerate(CASES):
    for k in range(9):
        signs = {sign_of(effective(Mmat, C @ product(A), eps), C @ product(A))
                 for A in itertools.combinations(COORDS, k)}
        # one common sign for all products of degree k, else 0
        degree_signs[row, k] = signs.pop() if len(signs) == 1 else 0
```

`itertools.combinations(COORDS, k)` lists every choice of k different directions in the order $x_0, \ldots, x_7$; together over $k = 0, \ldots, 8$ these are the 256 products Γ_A (`product(A)`; the empty choice gives the identity). For each one the sign of the bilinear with the matrix $C\Gamma_A$ is computed, and the braces collect the signs into a **set**, which keeps each different value once. If all products of degree k share one sign, the set has one element and `pop()` takes it out; otherwise 0 is stored.

```
k_all = np.arange(9)
formula = np.array([(-1) ** (k_all * (k_all + 1) // 2),
                    -(-1) ** (k_all * (k_all + 1) // 2),
                    (-1) ** (k_all * (k_all - 1) // 2),
                    -(-1) ** (k_all * (k_all - 1) // 2)])
```

The four rows of the degree rules of Section 21.10: $(-1)^{k(k+1)/2}$ for $\mathcal{C}_+$, $(-1)^{k(k-1)/2}$ for $\mathcal{C}_-$, each with an extra minus sign for anticommuting components. numpy computes the power for all nine values of k at once.

```
for row, (label, _, eps) in enumerate(CASES):
    say(f"calC_{'+' if label == 'plus' else '-'}, eps = {eps:+d}: signs for k = 0..8: "
        + " ".join(f"{s:+d}" for s in degree_signs[row]))
check(np.array_equal(degree_signs, formula),
      "all 256 bilinears: the sign depends only on the degree k, as the formulas say")
```

The underscore `_` is a name for a value that is not used. Each row of measured signs is printed (Out [15]); the check demands that every sign was common to its degree (no 0) and equals the formula.

In [16]: the sign table as a picture.

```
fig, ax = plt.subplots(figsize=(9.0, 3.8))
ax.imshow(degree_signs, cmap=SIGNS, vmin=-1, vmax=1, aspect="auto")
for row in range(4):
    for k in range(9):
        ax.text(k, row, f"{degree_signs[row, k]:+d}", ha="center", va="center")
```

The 4×9 array is drawn as coloured squares (`aspect="auto"` lets the squares stretch to fill the panel), and `ax.text` writes each sign in the middle of its square (`ha` and `va` are the horizontal and vertical alignment).

```
ax.set_xticks(range(9), ["0\n($S$)", "1\n($J$)"] + [str(k) for k in range(2, 9)])
ax.set_yticks(range(4), [r"$\mathcal{C}_+$, commuting",
                         r"$\mathcal{C}_+$, anticommuting",
                         r"$\mathcal{C}_-$, commuting",
                         r"$\mathcal{C}_-$, anticommuting"])
ax.set_xlabel("degree $k$ (number of gamma factors in the bilinear)")
ax.set_title("Sign of every bilinear under the two charge conjugations")
ax.grid(False)
```

The column labels mark the scalar ($k = 0$) and the current ($k = 1$); `str(k)` writes a number as text. The row labels name the four cases.

```
save_figure(fig, "bilinear_signs",
            "The sign that each of the 256 bilinears $\\bar\\Psi\\gamma^{a_1}"
            "\\cdots\\gamma^{a_k}\\Psi$ acquires under $\\Psi \\to M\\Psi^\\ast$ "
            "($M = 1$ for $\\mathcal{C}_+$, $M = \\Gamma$ for $\\mathcal{C}_-$), "
            "for commuting and anticommuting components; horizontal axis the degree "
            "$k$, rows the four cases, colour and number the sign (red $+1$, blue "
            "$-1$). All products of the same degree share one sign. For commuting "
            "components $\\mathcal{C}_+$ keeps the scalar $S$ ($k = 0$) and "
            "reverses the current $J$ ($k = 1$); anticommuting components reverse "
            "every sign.")
```

Figure 21a.6. **What the figure shows.** Each row repeats a pattern of two equal signs followed by two opposite ones: $+, -, -, +, +, -, -, +, +$ for $\mathcal{C}_+$ with commuting components, shifted by one place for $\mathcal{C}_-$, and the colours exchanged for anticommuting components. The pattern has period 4 in k because $k(k+1)/2$ changes its parity in pairs.

In [17]: a numerical confirmation with numbers.

```
rng = np.random.default_rng(2101)  # a fixed seed: the same numbers in every run
psi = rng.normal(size=16) + 1j * rng.normal(size=16)  # a complex field value
```

`np.random.default_rng(2101)` makes a random-number generator started from the fixed **seed** 2101, so every run draws the same numbers. `rng.normal(size=16)` draws 16 numbers from the bell-shaped normal distribution; two such draws make a complex column.

```
def S_of(p):
    """S = Psi^dagger C Psi."""
    return np.conj(p) @ C @ p


def J_of(p):
    """The eight currents J^a = Psi^dagger (-i C gamma^a) Psi."""
    return np.array([np.conj(p) @ K @ p for K in K_J])
```

For a numpy column `p`, `np.conj(p) @ C @ p` is the number $\sum_{r,c} p_r^* C_{rc} p_c$.

```
for label, p in (("Psi", psi), ("Psi*", np.conj(psi)),
                 ("Gamma Psi*", Gamma @ np.conj(psi))):
    say(f"{label:10}: S = {S_of(p).real:+.4f}, J^(x4) = {J_of(p)[4].real:+.4f}")
```

For the three columns the scalar and the time current (`J_of(p)[4]`, position 4 is x_4) are printed with a sign and four decimals: Out [17] shows $S = +6.0205$ for all three and $J^{(x4)} = +6.4970$, -6.4970, $+6.4970$.

```
check(np.isclose(S_of(np.conj(psi)), S_of(psi))
      and np.allclose(J_of(np.conj(psi)), -J_of(psi))
      and np.isclose(S_of(Gamma @ np.conj(psi)), S_of(psi))
      and np.allclose(J_of(Gamma @ np.conj(psi)), J_of(psi)),
      "numbers: Psi* keeps S and reverses J; Gamma Psi* keeps S and J")
```

The commuting rows of the table, confirmed with numbers for all eight currents (`np.isclose` compares two numbers up to rounding).

In [18]: real fields, exactly.

```
r = sp.Matrix(sp.symbols("r1:17", real=True))  # a real field with symbolic entries
C_s = sp.Matrix(C.tolist())
antisymmetric = all(np.array_equal((C @ gamma[x]).T, -(C @ gamma[x])) for x in COORDS)
```

`sp.symbols("r1:17", real=True)` makes the 16 real symbols $r_1, \ldots, r_{16}$ (the range stops before 17); `sp.Matrix` puts them into a column, a real field with completely general components. `C_s` is the exact copy of C. The third line checks that all eight $C\gamma^a$ are antisymmetric.

```
currents_zero = all(sp.expand((-sp.I * r.T * C_s * G[x] * r)[0, 0]) == 0
                    for x in COORDS)
S_kept = sp.expand((Gamma_s * r).T * C_s * (Gamma_s * r) - r.T * C_s * r) == \
    sp.zeros(1, 1)
kinetic_reversed = all(np.array_equal(Gamma.T @ C @ gamma[x] @ Gamma, -(C @ gamma[x]))
                       for x in COORDS)
```

`(-sp.I * r.T * C_s * G[x] * r)[0, 0]` is the single entry of the 1×1 matrix $-ir^T C\gamma^a r$; expanded, it is exactly zero for every a: (F1). The second statement is broken over two lines by a backslash at the end of the first (a **line continuation**); it tests that $(\Gamma r)^T C(\Gamma r) - r^T C r$ is exactly zero, so Γ keeps S. The third tests $\Gamma^T C\gamma^a \Gamma = -C\gamma^a$ for all eight directions: (F3).

```
check(antisymmetric and currents_zero and S_kept and kinetic_reversed
      and lead_passed("real_fields_charge_conjugation"),
      "real fields: J^a = 0, calC_+ is the identity, Gamma keeps S and reverses "
      "every kinetic matrix C gamma^a",
      record=f"{RECORD}, check real_fields_charge_conjugation")
```

The four facts together reproduce the record check. (That $\mathcal{C}_+$ is the identity on a real field needs no computation: $\Psi^* = \Psi$.)

In [19]: real fields, in a picture.

```
psi_r = rng.normal(size=16)  # a real column
phi_r = rng.normal(size=16)  # a second real column
kinetic_before = np.array([psi_r @ C @ gamma[x] @ phi_r for x in COORDS])
kinetic_after = np.array([(Gamma @ psi_r) @ C @ gamma[x] @ (Gamma @ phi_r)
                          for x in COORDS])
S_before, S_after = psi_r @ C @ psi_r, (Gamma @ psi_r) @ C @ (Gamma @ psi_r)
```

Two real columns from the same generator (the next numbers it draws). Think of `phi_r` as the derivative of the field in the kinetic term. The eight **kinetic numbers** $\Psi^T C\gamma^a \Phi$ are computed before and after the map Γ, and so is the scalar: $S = \Psi^T C \Psi$ before and $(\Gamma\Psi)^T C(\Gamma\Psi)$ after. For a real column `psi_r @ C @ psi_r` is exactly $\Psi^T C \Psi$.

```
say(f"S before = {S_before:+.4f}, S after = {S_after:+.4f}")
check(np.allclose(kinetic_after, -kinetic_before) and np.isclose(S_after, S_before),
      "numbers: Gamma reverses the eight kinetic numbers and keeps S")
```

Out [19] prints $S = -6.5629$ before and after, and the check confirms that all eight kinetic numbers change sign.

```
fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.3), width_ratios=[1, 1.4])
heat_map(axes[0], C @ gamma["x4"], r"$C\gamma^{(x4)}$ is antisymmetric")
positions = np.arange(8)
axes[1].bar(positions - 0.2, kinetic_before, width=0.4,
            label=r"$\Psi^T C\gamma^a\Phi$")
axes[1].bar(positions + 0.2, kinetic_after, width=0.4,
            label=r"$(\Gamma\Psi)^T C\gamma^a(\Gamma\Phi)$")
axes[1].axhline(0.0, color="black", linewidth=0.8)
axes[1].set_xticks(positions, COORDS)
axes[1].set_xlabel("direction $a$")
axes[1].set_ylabel("kinetic number")
axes[1].set_title(f"$\\Gamma$ reverses the kinetic numbers; $S$ = {S_before:.3f} "
                  "stays")
axes[1].legend()
```

Two panels, the right one 1.4 times as wide (`width_ratios`). Left: the heat map of $C\gamma^{(x4)}$. Right: pairs of bars for the eight kinetic numbers before and after the map, a black line at zero, the coordinate names as tick labels, and a title that includes the value of S with three decimals.

```
save_figure(fig, "real_fields",
            "Real fields. Left: heat map of the real matrix $C\\gamma^{(x4)}$ "
            "(horizontal axis the column, vertical axis the row, red $+1$, blue "
            "$-1$); it is antisymmetric, so $\\Psi^T C\\gamma^{(x4)}\\Psi = 0$ for "
            "every real $\\Psi$ and a real field carries no current. Right: the "
            "eight kinetic numbers $\\Psi^T C\\gamma^a\\Phi$ of two fixed real "
            "columns (left bars) and of their images under the real matrix "
            "$\\Gamma$ (right bars); horizontal axis the direction $a$, vertical "
            "axis the value (pure numbers). Every kinetic number changes sign "
            "while $S = \\Psi^T C\\Psi$ stays: with the mass reversed, $\\Gamma$ "
            "maps real solutions to real solutions.")
```

Figure 21a.7. **What the figure shows.** Left: every red square has a blue partner at the mirror position across the diagonal, the picture of antisymmetry, which is why a real field carries no current. Right: each pair of bars has equal height and opposite sign: Γ reverses the kinetic term while keeping the mass term, which is the same as keeping the kinetic term and reversing the mass, up to the overall sign of the Lagrangian.

In [20]: the record and the figure files.

```
REPRODUCED_HERE = ["representation_real", "B_imaginary_hermitian",
                   "intertwiners_same_mass", "intertwiners_reversed_mass",
                   "charge_conjugation_matrix_plus", "charge_conjugation_matrix_minus",
                   "majorana_conditions_consistent",
```

```
                    "bilinears_under_charge_conjugation",
                    "real_fields_charge_conjugation"]
summary = lead["summary"]
check(summary["passed"] == summary["total"] == 12
      and all(lead_passed(name) for name in REPRODUCED_HERE),
      "the lead record passes 12 of 12; the nine checks reproduced here are in it")
```

The list of the nine record checks this notebook reproduced; the check asks that the lead report passes all 12 of its checks and holds each of the nine. (The other three concern the spin connection and the U(1) identity, reproduced in Notebook 21b, and the quantised field, reproduced in Notebook 21c.)

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)  # in the order of saving
check(len(names) == 7 and all(
    output_file(f"{FIGURE_FOLDER}/21a_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the seven figure files of notebook 21a exist")
all_checks_passed()
```

`sorted(..., key=FIGURE_NUMBERS.get)` lists the figure names in the order of their numbers. The check asks that there are seven and that each file exists where it was written. The last line prints ALL 27 CHECKS PASSED (notebook 21a), the line you must see at the end.

21.16 The phase symmetry and Noether's current

Sakharov's first condition asks whether some process can change the number that counts matter minus antimatter. The theory of this book has no baryons; the only number of this kind it has is the U(1) charge Q of Section 21.7. This section finds the current that belongs to the phase symmetry, and Section 21.17 proves its exact local conservation law: the charge can change only by a flow through the boundary.

The symmetry. Under $\Psi \to e^{i\alpha}\Psi$ with a constant α, $\Psi^\dagger \to e^{-i\alpha}\Psi^\dagger$ (the conjugate of $e^{i\alpha}$ is $e^{-i\alpha}$), so every bilinear $\Psi^\dagger X \Psi$ is multiplied by $e^{-i\alpha}e^{i\alpha} = 1$, and $\mathcal{L}$, built from bilinears, does not change.

A phase that changes from point to point, line by line. Let $\alpha(x)$ be a function of the coordinates and write $\alpha_\mu = \partial_\mu \alpha$. For $\Psi' = e^{i\alpha}\Psi$:

$$D_\mu \Psi' = \partial_\mu(e^{i\alpha}\Psi) + \Omega_\mu e^{i\alpha}\Psi = e^{i\alpha}\big(D_\mu\Psi + i\alpha_\mu\Psi\big)$$

(product rule and chain rule, $\partial_\mu e^{i\alpha} = i\alpha_\mu e^{i\alpha}$; the number $e^{i\alpha}$ passes through the matrix Ω_μ). In the same way $\bar\Psi' = (e^{i\alpha}\Psi)^\dagger C = e^{-i\alpha}\bar\Psi$ and

$$D_\mu \bar\Psi' = \partial_\mu(e^{-i\alpha}\bar\Psi) - e^{-i\alpha}\bar\Psi\Omega_\mu = e^{-i\alpha}\big(D_\mu\bar\Psi - i\alpha_\mu\bar\Psi\big).$$

Inserting both into the kinetic term, the phases $e^{-i\alpha}e^{i\alpha} = 1$ cancel and

$$\tfrac{1}{2}\big(\bar\Psi'\gamma^\mu D_\mu\Psi' - (D_\mu\bar\Psi')\gamma^\mu\Psi'\big) = \tfrac{1}{2}\big(\bar\Psi\gamma^\mu(D_\mu\Psi + i\alpha_\mu\Psi) - (D_\mu\bar\Psi - i\alpha_\mu\bar\Psi)\gamma^\mu\Psi\big)$$

$$= \tfrac{1}{2}(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi) + i\alpha_\mu\bar\Psi\gamma^\mu\Psi$$

(the two terms with α_μ are equal and add: $\tfrac{1}{2}(i+i)=i$). Since $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, we have $\bar\Psi\gamma^\mu\Psi = iJ^\mu$ and $i\alpha_\mu\bar\Psi\gamma^\mu\Psi = i\cdot i\,\alpha_\mu J^\mu = -\alpha_\mu J^\mu$. The scalar S does not change. Hence, with the sum over μ,

$$\mathcal{L}' - \mathcal{L} = -\cos z\,\alpha_\mu J^\mu.$$

The current is the coefficient of $\partial_\mu\alpha$. This is **Noether's construction**. On a solution of the field equations the action $\int \mathcal{L}\,d^8x$ does not change to first order under any small change of the field that vanishes at the boundary (the principle of stationary action of Chapter 7). For a small α that vanishes at the boundary, $e^{i\alpha}\Psi$ is such a change, so

$$0 = \int (\mathcal{L}' - \mathcal{L})\,d^8x = -\int \cos z\,(\partial_\mu\alpha)J^\mu\,d^8x = \int \alpha\,\partial_\mu(\cos z\,J^\mu)\,d^8x$$

(the last step is integration by parts in each coordinate; the boundary term vanishes because α does). This holds for every such α, which is only possible if $\partial_\mu(\cos z\,J^\mu) = 0$ at every point inside the region: **the current is conserved on every solution**, as a **local** law (whether the total charge stays constant depends on what flows through the boundary, Sections 21.17 and 21.18). Notebook 21b checks the formula $\mathcal{L}' - \mathcal{L} = -\cos z\,\alpha_\mu J^\mu$ with numbers at one point of the author's metric, with the full spin connection, $m = 0.7$, $\lambda = 0.3$ and random field values: it finds $\mathcal{L} = -16.780795$, the same value after a constant phase, and $\mathcal{L}' - \mathcal{L} = +2.348835 = -\cos z\,\alpha_\mu J^\mu$ for a local phase (In [9]).

21.17 Exact local charge conservation in the author's metric

Noether's argument rests on the principle of stationary action. The Revision record also proves the conservation law directly, as an exact identity, and this section derives it line by line. Write the current as $J^\mu = \Psi^\dagger K^\mu\Psi$ with $K^\mu = -iC\gamma^\mu$ and the field equation as $E = \gamma^\mu D_\mu\Psi - V\Psi = 0$, with real $V = m + U'(S)$; the components are commuting here (the matrix identity found at the end contains no field, and the record checks the conservation law for both kinds of components: checks `commuting_current_conservation` and `grassmann_current_conservation` of `Revision /theory/reports/python-field-theory.json`).

Line 1 (product rule).

$$\partial_\mu(\cos z\,J^\mu) = \Psi^\dagger\big[\partial_\mu(\cos z\,K^\mu)\big]\Psi + \cos z\big[(\partial_\mu\Psi)^\dagger K^\mu\Psi + \Psi^\dagger K^\mu\partial_\mu\Psi\big].$$

Line 2 (the combination of the field equation). Consider $-i\cos z\,(\Psi^\dagger CE - E^\dagger C\Psi)$. Insert E; since the gammas and the Ω_μ are real, $E^\dagger = (\partial_\mu\Psi)^\dagger(\gamma^\mu)^T + \Psi^\dagger\Omega_\mu^T(\gamma^\mu)^T - V\Psi^\dagger$. The two terms with V give $-i\cos z\,(V\Psi^\dagger C\Psi - V\Psi^\dagger C\Psi) = 0$, because V is real.

Line 3 (the derivative terms). What remains of the derivative terms is $-i\cos z\,\big[\Psi^\dagger C\gamma^\mu\partial_\mu\Psi - (\partial_\mu\Psi)^\dagger(\gamma^\mu)^T C\Psi\big]$. By (R3), $(\gamma^\mu)^T C = -C\gamma^\mu$ (the scale factor $1/f_\mu$ is a number), so this is $-i\cos z\,\big[\Psi^\dagger C\gamma^\mu\partial_\mu\Psi + (\partial_\mu\Psi)^\dagger C\gamma^\mu\Psi\big] = \cos z\big[\Psi^\dagger K^\mu\partial_\mu\Psi + (\partial_\mu\Psi)^\dagger K^\mu\Psi\big]$: exactly the second part of line 1.

Line 4 (the spin-connection terms). They give $-i\cos z\,\Psi^\dagger\big[C\gamma^\mu\Omega_\mu - \Omega_\mu^T(\gamma^\mu)^TC\big]\Psi$.

Line 5 (the matrix identity). So $\partial_\mu(\cos z\,J^\mu) = -i\cos z\,(\Psi^\dagger CE - E^\dagger C\Psi)$ holds for every field as soon as the first part of line 1 equals line 4, that is (dividing by $-i$) as soon as the 16×16 **matrix identity**

$$\sum_\mu \partial_\mu\big(\cos z\,C\gamma^\mu\big) = \cos z\sum_\mu\big(C\gamma^\mu\Omega_\mu - \Omega_\mu^T(\gamma^\mu)^TC\big)$$

holds (and only then: two matrices that give the same bilinear for every column are equal when, as here, i times each is Hermitian).

Line 6 (conservation). On a solution $E=0$, so $\partial_\mu(\cos z\,J^\mu)=0$. Integrate over a region of the seven coordinates other than x_4 and over a time interval: by the fundamental theorem of calculus the charge $Q(x_4)=\int\cos z\,J^{(x4)}d^7x$ changes only by the **flux** of $\cos z\,J^\mu$ through the boundary of the region. With no flux, Q is constant. Whether the flux through the brane $z=\pi/2$ vanishes is not decided by the field equation: the record derives no boundary condition there, Section 21.18 shows an exact solution through whose brane charge flows, and Section 18.21 shows the same for the homogeneous solutions (there the charge of a patch of coordinate volume V_6 changes at the rate $-V_6q_0$, with the frame current q_0 running from -0.3454 to 0.3454 on the solution of Notebook 18c). The **no-flux condition** at the brane is therefore ASSUMED wherever it is used, and whether a junction condition at the brane implies it is OPEN.

The matrix identity in the author's metric, by hand. The left side is a sum over the eight directions of $\partial_\mu(\cos z/f_\mu)\,C\gamma^{(\mu)}$ (the curved gamma is $\gamma^{(\mu)}/f_\mu$).

- For $\mu = x_1, x_2, x_3, x_5, x_6, x_7$ the factor $\cos z/f_\mu$ depends only on x_4 and x_0, and it is differentiated along x_μ itself: zero.
- For $\mu = x_4$: $f_4 = 1$ and $\cos z$ does not depend on x_4: zero. (This is where the volume factor matters.)
- For $\mu = x_0$: $\cos z/f_0 = \cos z/\cot z = \sin z$, and $\partial_0\sin z = 6H\cos z$ (chain rule, $z = 6Hx_0$).

So the left side is $6H\cos z\,C\gamma^{(x0)}$. For the right side the record gives the spin connection (formula `Omega_components` of `Revision/theory/field-theory.json`; Chapters 6 and 8 derive it):

$$\Omega_{x_i} = \tfrac{1}{2}f_i\big(a_4'\gamma^{(xi)}\gamma^{(x4)} + H\gamma^{(xi)}\gamma^{(x0)}\big)\qquad (i=1,2,3),$$

$$\Omega_{x_t} = -\tfrac{1}{2}f_t\big(a_4'\gamma^{(x4)}\gamma^{(xt)} + H\gamma^{(xt)}\gamma^{(x0)}\big)\qquad (t=5,6,7),$$

and $\Omega_{x_4}=\Omega_{x_0}=0$, with $f_i = e^{a_4}\sin^{1/6}z$ and $f_t = e^{-a_4}\sin^{1/6}z$. Write $Y_\mu = \gamma^\mu\Omega_\mu$ (no sum). Since $\Omega_\mu^T(\gamma^\mu)^T = Y_\mu^T$ and, by (R3), $Y^TC = -CY$ for any combination Y of single gammas, each direction contributes $\cos z\,(CY_\mu - Y_\mu^TC) = 2\cos z\,CY_\mu$.

- Ordinary space, $i=1,2,3$: $Y_{x_i} = (\gamma^{(xi)}/f_i)\tfrac{1}{2}f_i\big(a_4'\gamma^{(xi)}\gamma^{(x4)} + H\gamma^{(xi)}\gamma^{(x0)}\big) = \tfrac{1}{2}\big(a_4'\gamma^{(x4)} + H\gamma^{(x0)}\big)$, because $\gamma^{(xi)}\gamma^{(xi)} = +1$. Contribution: $\cos z\,C\big(+a_4'\gamma^{(x4)} + H\gamma^{(x0)}\big)$.

- Extra times, $t = 5, 6, 7$: $Y_{x_t} = -\frac{1}{2}(a_4'\gamma^{(xt)}\gamma^{(x4)}\gamma^{(xt)} + H\gamma^{(xt)}\gamma^{(xt)}\gamma^{(x0)})$. Here $\gamma^{(xt)}\gamma^{(x4)}\gamma^{(xt)} = -\gamma^{(x4)}\gamma^{(xt)}\gamma^{(xt)} = \gamma^{(x4)}$ (anticommute, then $\gamma^{(xt)}\gamma^{(xt)} = -1$) and $\gamma^{(xt)}\gamma^{(xt)} = -1$, so $Y_{x_t} = \frac{1}{2}(-a_4'\gamma^{(x4)} + H\gamma^{(x0)})$. Contribution: $\cos z\, C(-a_4'\gamma^{(x4)} + H\gamma^{(x0)})$.

Adding the six contributions: the coefficient of $C\gamma^{(x4)}$ is $(3-3)a_4'\cos z = 0$ and that of $C\gamma^{(x0)}$ is $6H\cos z$. **Both sides equal $6H\cos z\, C\gamma^{(x0)}$, for every history a_4.** (The same six Ys add to $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)}$, the record's formula `gammaOmega_total`.) The identity is PROVED (lead check `u1_noether_matrix_identity` of `Revision/lead_checks/reports/charge-conjugation-and-u1.json`, which builds the connection from the metric with sympy and verifies the identity for a general $a_4(x_4)$; Notebook 21b repeats this, In [3] to In [5]).

Why the time-derivative terms cancel. Each inflating direction of space contributes $+a_4'$ to the coefficient of $C\gamma^{(x4)}$, each deflating extra time $-a_4'$. Suppose instead the extra times also inflated, with the scale factor $e^{+a_4}\sin^{1/6} z$ (a **control metric**, not the author's). Then each of the six contributes $+a_4'$, the right side has $6a_4'$ times the volume factor in front of $C\gamma^{(x4)}$, and the identity still holds, because now the volume factor is $\sqrt{|g|} = e^{6a_4}\cos z$ and the left side has the term $\partial_4(e^{6a_4}\cos z)\, C\gamma^{(x4)} = 6a_4'\, e^{6a_4}\cos z\, C\gamma^{(x4)}$. The identity holds in every metric; what is special about the author's metric is that $\sqrt{|g|} = e^{3a_4}e^{-3a_4}\cos z = \cos z$ does not depend on the time: **the deflation of the three extra times exactly compensates the inflation of the three space directions**, the volume of a slice stays constant, and no time-derivative term appears at all. Notebook 21b computes both metrics and draws the eight contributions (In [6] and In [7], figure 21b.1). A **negative control**: with the spin connection set to zero the right side vanishes but the left side does not, so without the spin connection the current would not be conserved (In [5], figure 21b.2).

statement	status	where it is verified
the current of the phase symmetry is $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, real; a constant phase leaves $\mathcal{L}$ unchanged	PROVED (Section 21.16); COMPUTED at a point	record formula `current`; Notebook 21b, In [2] and In [9]
the spin connection of the author's metric is real; it equals the record's formula; $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)}$	PROVED (sympy, general a_4)	lead check `spinor_connection_real`; formulas `Omega_components`, `gammaOmega_total`; Notebook 21b, In [4]
the U(1) matrix identity; both sides $6H\cos z\, C\gamma^{(x0)}$; fails without the connection	PROVED (sympy, general a_4)	lead check `u1_noether_matrix_identity`; Notebook 21b, In [5]
the a_4' terms cancel because the deflation keeps $\sqrt{\lvert g\rvert} = \cos z$; the control metric balances $6a_4'$ by its growing volume	PROVED (sympy)	Notebook 21b, In [6] (its own computation)

21.18 The charge of an exact solution: the flux through the brane, a constant charge, an indefinite density

The solution and its current. Take again the exact solution of Section 21.11 with $H = 1/6$, $\alpha = 1$, $m = 2$: $\Psi = \sin z\, U(x_4)\chi$ with the real matrix $U = \cos(wx_4) + \sin(wx_4)M/w$, $M =$

$-2\gamma^{(x4)} + \frac{3}{2}\gamma^{(x4)}\gamma^{(x0)}$, $w = \sqrt{7}/2$, and a fixed complex column χ (Notebook 21b uses one with exact rational entries). All eight components of its current are in general nonzero, but Ψ depends only on x_4 and z, so in the conservation law $\sum_\mu \partial_\mu(\cos z\, J^\mu) = 0$ only the terms with $\mu = x_4$ and $\mu = x_0$ survive. Their two densities are

$$\cos z\, J^{(x4)} = \cos z\, \Psi^\dagger B \Psi, \qquad \cos z\, J^{x0} = \cos z \tan z\, \Psi^\dagger K_0 \Psi = \sin z\, \Psi^\dagger K_0 \Psi, \quad K_0 = -iC\gamma^{(x0)},$$

because the curved gamma of x_0 is $\gamma^{(x0)}/f_0 = \tan z\, \gamma^{(x0)}$. With $H = 1/6$, $\partial_0 = \partial_z$, and the law reads $\partial_4(\cos z\, J^{(x4)}) + \partial_z(\cos z\, J^{x0}) = 0$; Notebook 21b checks it exactly (In [10]) and with finite differences (In [14]).

The charge balance, line by line. Since U is real, $\Psi^\dagger B \Psi = \sin^2 z\, \chi^\dagger U^T B U \chi = \sin^2 z\, f(x_4)$ with $f(x_4) = \chi^\dagger U^T B U \chi$, and in the same way $\Psi^\dagger K_0 \Psi = \sin^2 z\, g(x_4)$ with $g = \chi^\dagger U^T K_0 U \chi$.

Line 1. The charge of the patch (per unit of the other six coordinates) is $Q(x_4) = \int_0^{\pi/2} \cos z \sin^2 z\, dz\; f(x_4) = \frac{1}{3} f(x_4)$ (substitute $u = \sin z$, $du = \cos z\, dz$: $\int_0^1 u^2 du = \frac{1}{3}$).

Line 2. The flux density is $\sin z \cdot \sin^2 z\, g = \sin^3 z\, g(x_4)$: it is 0 at the tip $z = 0$ and $g(x_4)$ at the brane $z = \pi/2$.

Line 3. Integrate the conservation law over $0 < z < \pi/2$: $\frac{d}{dx_4} \int_0^{\pi/2} \cos z\, J^{(x4)} dz + \big[\sin^3 z\, g\big]_0^{\pi/2} = 0$ (the integral of a derivative is the difference of the end values), so $dQ/dx_4 = -g(x_4)$.

Line 4. Integrating from 0 to x_4:

$$Q(x_4) - Q(0) = -\int_0^{x_4} g(t)\, dt :$$

the charge of the patch changes exactly by what flows out through the brane. Notebook 21b checks this exactly with sympy (In [12]) and draws it (figure 21b.4); for its column χ the charge at $x_4 = 0$ is 0.178467.

A solution with zero flux. Since $M^2 = -w^2$ (Section 21.11, step 2, with $k^2 = -w^2$), define $P = \frac{1}{2}(1 - iM/w)$. Line by line:

- $P^2 = \frac{1}{4}(1 - 2iM/w + i^2M^2/w^2) = \frac{1}{4}(1 - 2iM/w + 1) = P$ (the square multiplied out; $i^2M^2/w^2 = (-1)(-w^2)/w^2 = 1$): P is a **projector**.
- $MP = \frac{1}{2}(M - iM^2/w) = \frac{1}{2}(M + iw) = iw\, \frac{1}{2}(1 - iM/w) = iwP$ (the same rule; $-iM^2/w = iw$; factor out iw, using $1/i = -i$).
- For $\chi_0 = P\chi$: $M\chi_0 = iw\chi_0$, hence $U\chi_0 = \big(\cos(wx_4) + i \sin(wx_4)\big)\chi_0 = e^{iwx_4}\chi_0$ (Euler's formula).

So $\Psi_0 = \sin z\, e^{iwx_4}\chi_0$ has a single frequency: a **stationary state**. Its charge density $\cos z \sin^2 z\, \chi_0^\dagger B \chi_0$ does not depend on the time (the phase e^{iwx_4} cancels against its conjugate), so by line 3 its flux through the brane is zero, and its charge is **constant**. For the column of Notebook 21b the constant is

$$Q_0 = -\frac{1424}{1875} - \frac{13\sqrt{7}}{1500} = -0.782397$$

(In [12], exact). It stays the same while the extra times deflate (figure 21b.6), and any other history gives the same constant: Ψ_0 does not depend on $x_1, x_2, x_3, x_5, x_6, x_7$, so it solves the field equation for every history a_4 (Section 21.11, step 4; record check `exact_solution_family_x4_x0`), and its charge density $\cos z \sin^2 z\, \chi_0^\dagger B \chi_0$ contains no a_4 (not even in the volume factor $\cos z$, because the deflation of the extra times compensates the inflation of space, Section 21.17). (The conservation law alone would not give this: a law that holds for every a_4 does not by itself make its constant independent of a_4.)

The charge density can be negative. $J^{(x4)} = \Psi^\dagger B \Psi$ is a quadratic expression with the matrix B, which has eight eigenvalues $+1$ and eight -1 (B is Hermitian, $B^2 = 1$ and its trace is 0; lead check `B_imaginary_hermitian` and record check `B_properties`). Writing $\Psi = \sum_j c_j v_j$ in orthonormal eigenvectors v_j of B gives $\Psi^\dagger B \Psi = \sum_{B=+1} |c_j|^2 - \sum_{B=-1} |c_j|^2$, which can have either sign; divided by $\Psi^\dagger \Psi = \sum |c_j|^2$ it lies between -1 and $+1$. *A worked example in flat space* ($H = 0$, a_4 constant). The field equation along the time is $\gamma^{(x4)} \partial_4 \Psi = m\Psi$. A **rest state** $\Psi = u\, e^{-imx_4}$ with $-i\gamma^{(x4)} u = u$ solves it: $\gamma^{(x4)} \partial_4 \Psi = \gamma^{(x4)} (-im) u\, e^{-imx_4} = m(-i\gamma^{(x4)} u) e^{-imx_4} = m\Psi$. It has the positive frequency m. Since C commutes with $\gamma^{(x4)}$ ((R3): x_4 is time-like) and $B = C(-i\gamma^{(x4)})$, B commutes with $-i\gamma^{(x4)}$ and maps the 8-dimensional space of such u into itself; there it has four eigenvalues $+1$ (eigenvector u_+) and four -1 (eigenvector u_-), as the notebook computes. For $u = \frac{3}{5} u_+ + \frac{4}{5} u_-$ (length 1, since $\frac{9}{25} + \frac{16}{25} = 1$) the charge density is $\frac{9}{25}(+1) + \frac{16}{25}(-1) = -\frac{7}{25}$: a positive-frequency field with a **negative** charge density (Notebook 21b, In [16]; figure 21b.7 shows that for 20000 random columns the normalised density takes both signs about equally often, a fraction 0.504 negative). The sign of the charge of a single field is therefore not fixed by the sign of its frequency; Chapter 10 explains what this means after quantisation (the Krein space).

statement	**status**	**where it is verified**
the exact solution solves the field equation (any a_4); local conservation for it	PROVED (sympy)	`Revision/theory/reports/python-field-theory.json`, checks `exact_solution_family_x4_x0`, `commuting_current_conservation`; Notebook 21b, In [10]
charge balance $Q(x_4) - Q(0) = -\int_0^{x_4} g$	PROVED (sympy); COMPUTED to 10^{-12} on 161 times	Notebook 21b, In [12] and In [13] (its own computation)
the stationary solution has zero flux and the constant charge $-1424/1875 - 13\sqrt{7}/1500 = -0.782397$	PROVED (sympy, exact)	Notebook 21b, In [12]
finite differences converge to the exact law with order 2	COMPUTED (ratios 3.966 to 4.000)	Notebook 21b, In [14]
B has signature (8,8); a positive-frequency rest state with charge density $-7/25$	PROVED (above); COMPUTED	Notebook 21b, In [16]

21.19 The pair-level bookkeeping: theorem T1 and the zero total charge

Theorem T1 of the Revision record (`Revision/docs/PAIR_CREATION_PROOFS.md`, its section 4; Chapter 18 gives the complete proof). For both fields, in every gravitational field taken as a fixed background, in particular the author's metric with the deflating extra times: for every configuration Ψ,

$$\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi], \qquad J^{\mu}[\Gamma\Psi] = -J^{\mu}[\Psi],$$

$\Gamma\Psi$ solves the field equations with (m, λ) if and only if Ψ solves them with $(-m, -\lambda)$, and the energy-momentum tensor is reversed as well.

The current part, line by line. $\overline{\Gamma\Psi} = (\Gamma\Psi)^{\dagger}C = \Psi^{\dagger}\Gamma^{\dagger}C = \Psi^{\dagger}\Gamma C = \Psi^{\dagger}C\Gamma = \bar{\Psi}\Gamma$ (the rule $(XY)^{\dagger} = Y^{\dagger}X^{\dagger}$; Γ is real and symmetric; Γ commutes with C by (R2)). Then $\overline{\Gamma\Psi}\gamma^{\mu}\Gamma\Psi = \bar{\Psi}\Gamma\gamma^{\mu}\Gamma\Psi = -\bar{\Psi}\gamma^{\mu}\Gamma\Gamma\Psi = -\bar{\Psi}\gamma^{\mu}\Psi$ (R1, then $\Gamma\Gamma = 1$). Multiplying by $-i$: $J^{\mu}[\Gamma\Psi] = -J^{\mu}[\Psi]$ at every point, for both kinds of components (Γ only multiplies each component by ± 1, so no components are reordered).

On the exact solution. For $\lambda = 0$, $\Gamma\Psi$ solves the equation with the mass -2: every term of E_m in Section 21.11 except the mass term contains one gamma, which anticommutes with Γ, so $E_{-m}[\Gamma\Psi] = -\Gamma E_m[\Psi] = 0$. Its charge density is the opposite of that of Ψ at every point, and the pair carries none (Notebook 21b, In [11], figure 21b.3). For the stationary solution the partner has the constant charge $+0.782397$ and the pair 0 (figure 21b.4, right). For five random columns at three times, $Q + Q' = 0$ to 10^{-12} while Q itself is nonzero (In [17], figure 21b.8).

What this says, exactly. A pair made of a solution Ψ with (m, λ) and its partner $\Gamma\Psi$ with $(-m, -\lambda)$, both in the same gravitational field, has total current zero and total charge zero, as **classical bilinears** (ordinary numbers for dirac16complex00, elements of the Grassmann algebra for dirac16complex). Record checks: `T1_current_primordial_commuting` and `T1_current_primordial_grassmann` of `Revision/pairing/reports/wolfram-pairing.json`, `T1.metric.commuting.current` of `Revision/pairing/reports/python-pairing.json`. Status: PROVED.

What it does not say (rule R3 of this book; the record's own list is in section 11.2 of `Revision/docs/PAIR_CREATION_PROOFS.md`):

- It does not create anything. T1 is a map between the solutions of two parameter sets. No equation of this theory produces a universe, or a pair of universes, from anything; no creation process, rate, probability or amplitude follows from these equations (Chapter 20).
- No equation forces the partner to exist: a single universe with the charge $Q \neq 0$ is an equally valid solution. If no charge flows through the brane (the ASSUMED no-flux condition, Sections 21.17 and 21.18), Q is constant, so its value is fixed by the initial data and is not explained.
- For $\lambda \neq 0$ the partner has the coupling $-\lambda$ as well; T1 is not a pairing of $+m$ with $-m$ at a fixed coupling.
- At the quantum level, the chirality image $\Gamma\Psi$ of the quantised field carries the Krein metric $-B$: it is the same quantum system, relabelled, and the identity $Q + Q' = 0$ is of the form $X + (-X) = 0$ within that one system. An independently quantised universe of mass $-m$ carries $+B$, cannot be identified with $\Gamma\Psi$, and its charge does not cancel ours (record checks

`Q_Krein_metric_of_images` and `Q_no_identification_of_independent_universes` of the Wolfram pairing report; Chapter 10).

- A T1 pair taken as the complete classical source of the author's metric is a zero source, and Einstein's equations then have no solution for $H > 0$ (corollary C1 of the record; check `einstein_no_vacuum_solution` of `Revision/field_equations_a4/reports/wolfram-a4-report.json`). So a T1 pair alone cannot even be the source of the author's universe.
- The member of mass $-m$ is not antimatter of our world: its negative mass is a parameter of a second field configuration. Nothing in this book says that antimatter has negative mass.

The idea that a universe and an anti-universe together are symmetric belongs to a class of ideas in the published literature; a published example is L. Boyle, K. Finn and N. Turok, "CPT-Symmetric Universe", Phys. Rev. Lett. 121, 251301 (2018), who propose that the universe after the big bang is the CPT image of the universe before it, so that the two epochs form a universe/anti-universe pair. Their proposal is cited only as an example of the class; this book neither uses nor tests it, and the pairing of this theory is of a different kind (both members exist at the same time x_4, related by the chirality matrix and $m \to -m$). Any scenario in which our universe is one member of such a pair is a **HYPOTHESIS** (Section 21.37).

21.20 Example: Notebook 21b follows the charge in the author's metric

Notebook 21b turns Sections 21.16 to 21.19 into exact computations. It reads the gammas, the lead report, the field-theory record and both pairing reports; it builds the canonical spin connection of the author's metric with sympy for an unspecified history $a_4(x_4)$ and checks it against the record; it verifies the U(1) matrix identity, its negative control and the control metric with inflating extra times; it checks the Noether current with a local phase at one point; it follows the charge of the record's exact solution (local conservation, the balance through the brane, a stationary solution of constant charge, a finite-difference test); it shows that the charge density is indefinite; and it does the pair-level bookkeeping of theorem T1. It runs in about 40 seconds; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/21b_charge_conservation.PROVENANCE.md`. It needs no Rust, prints 21 PASS lines and draws eight figures.

21.21 How to run Notebook 21b

Step 1. What this notebook does and what it needs.

Notebook 21b (Local U(1) charge conservation, the brane flux and the pair-level charge bookkeeping) is the file `Revision/textbook/notebooks/21b_charge_conservation.ipynb` of the repository Dirac_claude. It builds the canonical spin connection of the author's metric for a general history a4(x4) with sympy, checks that it is real, reduces the U(1) Noether identity to a 16 by 16 matrix identity and verifies it exactly (with a negative control without the spin connection and a control metric with inflating extra times, which shows that the a4 terms cancel because the deflating extra times keep the volume of a slice constant), checks the Noether current by a local phase change, follows the charge of an exact solution of the field equation in the author's metric (exact local conservation, second-order convergence of a finite-difference test, the charge balance with the flux

through the brane, a solution with zero flux and constant charge), shows that the charge density is indefinite, and does the pair-level bookkeeping of theorem T1: the chirality partner carries the opposite current, so the total charge of a pair is zero. Eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 21b_charge_conservation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 40 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21b_charge_conservation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 21b_charge_conservation.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/21b.captions.json`
- `Revision/textbook/figures/21b_1_volume_balance.png`
- `Revision/textbook/figures/21b_2_noether_matrices.png`
- `Revision/textbook/figures/21b_3_charge_density_maps.png`
- `Revision/textbook/figures/21b_4_charge_balance.png`
- `Revision/textbook/figures/21b_5_local_conservation.png`
- `Revision/textbook/figures/21b_6_charge_through_deflation.png`
- `Revision/textbook/figures/21b_7_indefinite_charge.png`
- `Revision/textbook/figures/21b_8_pair_bookkeeping.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the eight figure files of notebook 21b exist
ALL 21 CHECKS PASSED (notebook 21b)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/21b_charge_conservation.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 21b_charge_conservation.ipynb
python -m nbconvert --execute --inplace 21b_charge_conservation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra`, `Revision/theory`, `Revision/pairing` or `Revision/lead_checks`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

21.22 Notebook 21b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/21b_charge_conservation.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the

cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The first of Sakharov's three conditions for making more matter than antimatter
is that some process must **change** the number that counts matter minus
antimatter. The theory of this book contains no baryons; the only number of this
kind it has is the **U(1) charge** $Q$ of the field: the Lagrangian does not
change when the field is
multiplied by a constant phase $e^{i\alpha}$, and by Noether's theorem the charge
of this symmetry obeys a **local** conservation law: no process creates or
destroys it at any point of the patch $0 < z < \pi/2$ (off the brane), and the
charge of a region changes only by what flows through its boundary. This
notebook checks that law exactly in
the author's primordial metric, for **every** history $a_4(x_4)$, in particular
the one in which ordinary space inflates and the three extra times deflate
exponentially. It

1. builds the canonical spin connection of the author's metric with sympy and
   checks that it is real and gives $\gamma^\mu\Omega_\mu = 3H\gamma^{(x0)}$;
2. reduces the conservation law $\partial_\mu(\cos z\,J^\mu) = 0$ (on solutions)
   to one $16 \times 16$ matrix identity and verifies it exactly, with a negative
   control (no spin connection: the identity fails) and a control metric (the
   extra times inflating too), which shows **why** the time-derivative terms
   cancel: the deflation of the extra times keeps the volume of a slice constant;
3. checks the Noether current by changing the phase of a field from point to
   point;
4. follows the charge of an **exact solution** of the field equation in the
   author's metric: local conservation (exact, and a finite-difference test), the
   charge balance with the flux through the brane $z = \pi/2$, and a solution
   whose flux vanishes, so that its charge stays constant while the extra times
   deflate;
5. shows that the charge density $\Psi^\dagger B\Psi$ can be positive or negative;
6. does the **pair-level bookkeeping** of theorem T1: the chirality partner
   $\Gamma\Psi$ (a solution with the mass reversed) carries exactly the opposite
   current, so a pair has total charge zero.

Two checks reproduce `Revision/lead_checks/reports/charge-conjugation-and-u1.json`;
others reproduce the Revision field-theory and pairing records. Eight plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 21b (Local U(1) charge conservation, the brane flux and the pair-level charge
bookkeeping) is the file `Revision/textbook/notebooks/21b_charge_conservation.ipynb` of
the repository Dirac_claude. It builds the canonical spin connection of the author's
metric for a general history a4(x4) with sympy, checks that it is real, reduces the U(1)
```

Noether identity to a 16 by 16 matrix identity and verifies it exactly (with a negative control without the spin connection and a control metric with inflating extra times, which shows that the a4 terms cancel because the deflating extra times keep the volume of a slice constant), checks the Noether current by a local phase change, follows the charge of an exact solution of the field equation in the author's metric (exact local conservation, second-order convergence of a finite-difference test, the charge balance with the flux through the brane, a solution with zero flux and constant charge), shows that the charge density is indefinite, and does the pair-level bookkeeping of theorem T1: the chirality partner carries the opposite current, so the total charge of a pair is zero. Eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 21b_charge_conservation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 40 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter nbconvert --to notebook --execute --inplace 21b_charge_conservation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 21b_charge_conservation.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/21b.captions.json‘
- ‘Revision/textbook/figures/21b_1_volume_balance.png‘
- ‘Revision/textbook/figures/21b_2_noether_matrices.png‘
- ‘Revision/textbook/figures/21b_3_charge_density_maps.png‘
- ‘Revision/textbook/figures/21b_4_charge_balance.png‘
- ‘Revision/textbook/figures/21b_5_local_conservation.png‘
- ‘Revision/textbook/figures/21b_6_charge_through_deflation.png‘
- ‘Revision/textbook/figures/21b_7_indefinite_charge.png‘
- ‘Revision/textbook/figures/21b_8_pair_bookkeeping.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
    git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS the eight figure files of notebook 21b exist
    ALL 21 CHECKS PASSED (notebook 21b)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from

a copy outside the repository. Open the file
‘Revision/textbook/notebooks/21b_charge_conservation.ipynb‘ inside the folder
Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was
downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with
‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use
‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick
"Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt
install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not
active; do Step 4. If the environment is active and the message stays, the programs
jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
programs (‘python -m jupyter‘ does not help then: it has to find the same programs).
Start them through Python itself, in the folder of the notebook: the first command
below does what ‘jupyter lab‘ does, the second what the headless command does:

```
python -m jupyterlab 21b_charge_conservation.ipynb
python -m nbconvert --execute --inplace 21b_charge_conservation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
loop" and zmq: this is a message of the package pyzmq, not an error; the run continues
normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the
first run after the installation: this is a message, not an error; the run continues
and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
and Run All Cells; if it fails again, install the packages again with the pip commands
of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘, ‘Revision/theory‘,
‘Revision/pairing‘ or ‘Revision/lead_checks‘: the notebook was opened outside the
repository, or the repository is incomplete; clone the repository again and open the
notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then
it imports the packages, finds the repository folder and defines the helpers
‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a
check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a
RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 21b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 21b (Local U(1) charge conservation, the brane flux and the pair-level charge
# bookkeeping) is the file Revision/textbook/notebooks/21b_charge_conservation.ipynb of
# the repository Dirac_claude. It builds the canonical spin connection of the author’s
# metric for a general history a4(x4) with sympy, checks that it is real, reduces the
# U(1) Noether identity to a 16 by 16 matrix identity and verifies it exactly (with a
# negative control without the spin connection and a control metric with inflating extra
```

```
# times, which shows that the a4 terms cancel because the deflating extra times keep the
# volume of a slice constant), checks the Noether current by a local phase change,
# follows the charge of an exact solution of the field equation in the author's metric
# (exact local conservation, second-order convergence of a finite-difference test, the
# charge balance with the flux through the brane, a solution with zero flux and constant
# charge), shows that the charge density is indefinite, and does the pair-level
# bookkeeping of theorem T1: the chirality partner carries the opposite current, so the
# total charge of a pair is zero. Eight teaching plots. It needs a computer with Windows
# 11, macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
```

```
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
```

```
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 21b_charge_conservation.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 40
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 21b_charge_conservation.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 21b_charge_conservation.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
```

```
#
# - Revision/textbook/figures/21b.captions.json
# - Revision/textbook/figures/21b_1_volume_balance.png
# - Revision/textbook/figures/21b_2_noether_matrices.png
# - Revision/textbook/figures/21b_3_charge_density_maps.png
# - Revision/textbook/figures/21b_4_charge_balance.png
# - Revision/textbook/figures/21b_5_local_conservation.png
# - Revision/textbook/figures/21b_6_charge_through_deflation.png
# - Revision/textbook/figures/21b_7_indefinite_charge.png
# - Revision/textbook/figures/21b_8_pair_bookkeeping.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the eight figure files of notebook 21b exist
#     ALL 21 CHECKS PASSED (notebook 21b)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/21b_charge_conservation.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
```

```
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 21b_charge_conservation.ipynb
#     python -m nbconvert --execute --inplace 21b_charge_conservation.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra, Revision/theory,
#   Revision/pairing or Revision/lead_checks: the notebook was opened outside the
#   repository, or the repository is incomplete; clone the repository again and open the
#   notebook from its folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "21b"  # this notebook: chapter 21, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
```

```
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 21b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Phase**: a complex number $e^{i\alpha} = \cos\alpha + i\sin\alpha$ of modulus
  1. Multiplying every component of $\Psi$ by the same $e^{i\alpha}$ is a
  **global U(1) transformation** (U(1) is the name of the group of these phases).
- **Symmetry, Noether's theorem**: when the Lagrangian does not change under a
  continuous transformation, there is a **current** $J^\mu$ whose divergence
  vanishes on every solution (a **local** conservation law); its time component
  integrated over space is the **charge** $Q$, which changes only by what flows
  through the boundary of the region (it is constant when nothing flows
  through it).
- **Current** $J^\mu = -i\bar\Psi\gamma^\mu\Psi$ (eight components, $\mu = x0,
  \dots, x7$), with $\bar\Psi = \Psi^\dagger C$ and the curved gammas
  $\gamma^\mu = e^\mu{}_a\gamma^a$; its time component is the **charge density**
  $J^{(x4)} = \Psi^\dagger B\Psi$, $B = -iC\gamma^{(x4)}$.
```

- **Divergence** $\partial_\mu(\cos z\,J^\mu) = \sum_\mu \partial(\cos z\,J^\mu) /\partial x_\mu$; $\cos z = \sqrt{|g|}$ is the volume factor of the author's metric.
- **Flux**: the amount of charge that flows through a surface per unit time; **charge balance**: the charge inside a region changes exactly by the flux through its boundary.
- **Vielbein** $e^a{}_\mu$: the eight scale factors f_a of the author's metric, $f_{1,2,3} = e^{a_4}\sin^{1/6}z$ (ordinary space), $f_4 = 1$ (the time), $f_{5,6,7} = e^{-a_4}\sin^{1/6}z$ (the deflating extra times), $f_0 = \cot z$ (the hidden direction), with $z = 6Hx_0$.
- **Spin connection** $\Omega_\mu = \tfrac12\omega_{\mu ab}S^{ab}$: the matrices that make the derivative $D_\mu = \partial_\mu + \Omega_\mu$ of a spinor covariant; $\omega_{\mu ab}$ are built from the vielbein and the Christoffel symbols, and $S^{ab} = \tfrac14[\gamma^a, \gamma^b]$.
- **Brane**: the end $z = \pi/2$ of the patch $0 < z < \pi/2$ of the hidden direction.
- **Chirality partner**: $\Gamma\Psi$ with $\Gamma = \gamma^{(x0)}\gamma^{(x1)} \cdots\gamma^{(x7)}$; by theorem T1 of the Revision pairing record it solves the field equation with the mass reversed.
- **Indefinite**: a quadratic expression that can take both signs.

Text cell:

```
## 4. The physical and mathematical situation

**The symmetry.** The Lagrangian of both fields of the Revision record is
$\mathcal{L} = \cos z\,[\tfrac12(\bar\Psi\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)
\gamma^\mu\Psi) - mS - U(S)]$ with $S = \bar\Psi\Psi$. Under $\Psi \to
e^{i\alpha}\Psi$ with a constant $\alpha$, $\Psi^\dagger \to e^{-i\alpha}
\Psi^\dagger$, so every bilinear $\Psi^\dagger X\Psi$ is multiplied by
$e^{-i\alpha}e^{i\alpha} = 1$: $\mathcal{L}$ does not change.

**The identity behind conservation**, line by line. Write the current as
$J^\mu = \Psi^\dagger K^\mu\Psi$ with $K^\mu = -iC\gamma^\mu$, and the field
equation as $E = \gamma^\mu D_\mu\Psi - V\Psi = 0$ with real $V = m + U'(S)$.

1. Product rule: $\partial_\mu(\cos z\,J^\mu) = \Psi^\dagger[\partial_\mu(\cos z
   \,K^\mu)]\Psi + \cos z\,[(\partial_\mu\Psi)^\dagger K^\mu\Psi + \Psi^\dagger
   K^\mu\partial_\mu\Psi]$.
2. The Revision record states the identity $\partial_\mu(\cos z\,J^\mu) = -i\cos
   z\,(\Psi^\dagger CE - E^\dagger C\Psi)$. Insert $E$: the $V$ terms give
   $-i\cos z\,(V\Psi^\dagger C\Psi - V\Psi^\dagger C\Psi) = 0$ because $V$ is real.
3. The derivative terms give $-i\cos z\,[\Psi^\dagger C\gamma^\mu\partial_\mu\Psi
   - (\partial_\mu\Psi)^\dagger(\gamma^\mu)^TC\Psi]$. With $(\gamma^\mu)^TC =
   -C\gamma^\mu$ (the gammas are real) this is $\cos z\,[\Psi^\dagger K^\mu
   \partial_\mu\Psi + (\partial_\mu\Psi)^\dagger K^\mu\Psi]$: exactly the second
   part of line 1.
4. The terms with the spin connection give $-i\cos z\,\Psi^\dagger[C\gamma^\mu
   \Omega_\mu - \Omega_\mu^T(\gamma^\mu)^TC]\Psi$.
5. So the identity holds for every field if and only if the first part of line 1
   equals line 4 for every $\Psi$, that is (dividing by $-i$) if and only if the
   $16 \times 16$ **matrix identity**
   $$\sum_\mu\partial_\mu(\cos z\,C\gamma^\mu) = \cos z\sum_\mu\big(C\gamma^\mu
```

```
   \Omega_\mu - \Omega_\mu^T(\gamma^\mu)^TC\big)$$
   holds. (Two Hermitian matrices that give the same value $\Psi^\dagger X\Psi$
   for every column $\Psi$ are equal.)
6. On a solution $E = 0$, so $\partial_\mu(\cos z\,J^\mu) = 0$. Integrating over
   a slab between two times and using the fundamental theorem of calculus, the
   charge $Q(x_4) = \int\cos z\,J^{(x4)}\,d^7x$ changes only by the flux of
   $\cos z\,J^\mu$ through the boundary of the region; with no flux, $Q$ is
   constant.

This notebook verifies the matrix identity of line 5 exactly, for a general
history $a_4(x_4)$ (so for the deflating one), and then watches lines 1 to 6 at
work on an exact solution.
```

Text cell:

```
## 5. The gammas and the Revision records

The next cell reads the eight gamma matrices of the Revision record (both as
whole-number numpy arrays and as exact sympy matrices), checks that they are eight
real $16 \times 16$ matrices with the Clifford relation of signature (4,4), builds
$C$, $\Gamma$, $B$ and the generators $S^{ab}$, and reads the Revision records
whose checks this notebook reproduces. It also checks that the record's formula
current defines the current and the charge density exactly as this notebook does.
```

In [2]:

```
import numpy as np  # numbers, arrays, matrices
import sympy as sp  # exact algebra and calculus

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = fixture["eta"]  # +1 space-like, -1 time-like, in the order x0..x7
gamma = [np.array(mat, dtype=np.int64) for mat in fixture["gamma"]]  # index 0..7
I16 = np.eye(16, dtype=np.int64)
real_ok = all(g.shape == (16, 16) and set(np.unique(g)) <= {-1, 0, 1} for g in gamma)
clifford_ok = all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                                 2 * (ETA[a] if a == b else 0) * I16)
                  for a in range(8) for b in range(8))
check(len(gamma) == 8 and real_ok and clifford_ok,
      "eight real 16 x 16 gamma matrices with the Clifford relation, signature (4,4)")

G = [sp.Matrix(g.tolist()) for g in gamma]  # exact sympy copies, G[k] = gamma^(xk)
C_s = G[0] * G[1] * G[2] * G[3]  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
C = np.array(C_s.tolist(), dtype=np.int64)
Gamma = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5] \
    @ gamma[6] @ gamma[7]  # the chirality (index 0 is x0)
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4); index 4 is x4
K0 = -1j * (C @ gamma[0])  # the matrix of the x0 current: -i C gamma^(x0)
S_gen = [[(G[a] * G[b] - G[b] * G[a]) / 4 for b in range(8)] for a in range(8)]

LEAD_FILE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
```

```
LEAD = {c["name"]: c["verdict"] for c in json.loads(
    repository_file(LEAD_FILE).read_text(encoding="utf-8"))["checks"]}
THEORY = {f["key"]: f for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
PY_THEORY = {c["name"]: c["verdict"].upper() for c in json.loads(repository_file(
    "Revision/theory/reports/python-field-theory.json").read_text(
    encoding="utf-8"))["checks"]}
WL_PAIR = {c["name"]: c["verdict"] for c in json.loads(repository_file(
    "Revision/pairing/reports/wolfram-pairing.json").read_text(
    encoding="utf-8"))["checks"]}
PY_PAIR = {c["name"]: c["verdict"] for c in json.loads(repository_file(
    "Revision/pairing/reports/python-pairing.json").read_text(
    encoding="utf-8"))["checks"]}
clauses = THEORY["current"]["wl"].split("; ")  # the statements of the formula
say("record formula current: " + clauses[0] + "; " + clauses[2])
check(clauses[0] == "J^mu = -i Psibar gamma^mu Psi"
      and clauses[2] == "J^x4 = Psi^dagger B Psi",
      "the record defines J^mu = -i Psibar gamma^mu Psi and J^x4 = Psi^dagger B Psi",
      record="Revision/theory/field-theory.json, formula current")
```

Out [2]:

```
PASS eight real 16 x 16 gamma matrices with the Clifford relation, signature (4,4)
record formula current: J^mu = -i Psibar gamma^mu Psi; J^x4 = Psi^dagger B Psi
PASS the record defines J^mu = -i Psibar gamma^mu Psi and J^x4 = Psi^dagger B Psi
     reproduces Revision/theory/field-theory.json, formula current
```

Text cell:

```
## 6. The canonical spin connection of the author's metric

The next cell builds, with sympy, everything the spin connection needs, for an
unspecified function $a_4(x_4)$ (so the result holds for every history):

- the scale factors $f_a$ and the metric $g_{\mu\mu} = \eta_{aa}f_a^2$ (diagonal);
- the Christoffel symbols $\Gamma^l{}_{ij} = \tfrac12 g^{lm}(\partial_j g_{mi} +
  \partial_i g_{mj} - \partial_m g_{ij})$;
- the connection coefficients $\omega_\mu{}^a{}_b = f_a\,\big(\partial_\mu
  (\delta_{ab}/f_b) + \Gamma^a{}_{\mu b}/f_b\big)$ (the vielbein postulate
  $\omega_\mu{}^a{}_b = e^a{}_\nu(\partial_\mu e_b{}^\nu + \Gamma^\nu{}_{\mu
  \lambda}e_b{}^\lambda)$ for the diagonal vielbein $e^a{}_\nu = f_a\delta^a_\nu$),
  lowered with $\eta$;
- $\Omega_\mu = \tfrac12\sum_{a,b}\omega_{\mu ab}S^{ab}$.

It is written as a function of the three scale factors of the extra times, so
that the same code can later build a *control* metric in which the extra times
inflate. (About 7 seconds.)
```

In [3]:

```
x = sp.symbols("x0:8", real=True)  # the coordinates x0, ..., x7
H = sp.symbols("H", positive=True)  # the author's constant H > 0
```

```
a4 = sp.Function("a4", real=True)(x[4])  # an unspecified history a4(x4)
z = 6 * H * x[0]  # z = 6 H x0
eta = sp.diag(*ETA)


def geometry(extra_sign):
    """Scale factors, volume factor and the eight matrices Omega_mu of the metric
    whose extra-time factors are exp(extra_sign a4) sin^(1/6) z (extra_sign = -1:
    the author's deflating extra times)."""
    f = ([sp.cot(z)] + [sp.exp(a4) * sp.sin(z) ** sp.Rational(1, 6)] * 3
         + [sp.Integer(1)]
         + [sp.exp(extra_sign * a4) * sp.sin(z) ** sp.Rational(1, 6)] * 3)
    g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # the diagonal metric
    g_inv = g.inv()
    chris = [[[sp.simplify(sum(g_inv[l, m] * (sp.diff(g[m, i], x[j])
                                              + sp.diff(g[m, j], x[i])
                                              - sp.diff(g[i, j], x[m]))
                               for m in range(8)) / 2)
               for j in range(8)] for i in range(8)] for l in range(8)]
    e_inv = [1 / v for v in f]  # e_a^mu = 1 / f_a on the diagonal
    Omega = []
    for mu in range(8):
        om = sp.zeros(8, 8)  # omega_mu^a_b
        for a in range(8):
            for b_ in range(8):
                om[a, b_] = sp.simplify(f[a] * (sp.diff(e_inv[b_] if a == b_ else 0,
                                                       x[mu])
                                                + chris[a][mu][b_] * e_inv[b_]))
        om_low = eta * om  # omega_mu ab = eta_ac omega_mu^c_b
        Om = sp.zeros(16, 16)
        for a in range(8):
            for b_ in range(8):
                if om_low[a, b_] != 0:
                    Om += om_low[a, b_] * S_gen[a][b_] / 2
        Omega.append(Om.applyfunc(sp.simplify))
    volume = sp.simplify(sp.prod(f))  # sqrt|g| = product of the scale factors
    return f, volume, Omega


f_dfl, vol_dfl, Omega = geometry(-1)  # the author's metric (deflating extra times)
say(f"sqrt|g| of the author's metric = {vol_dfl}")
```

Out [3]:

```
sqrt|g| of the author's metric = cos(6*H*x0)
```

Text cell:

```
The next cell checks three facts of the Revision record:

1. every entry of every $\Omega_\mu$ is a **real** expression (no imaginary unit;
   record check spinor_connection_real), and not all of them vanish;
```

```
2. $\Omega_\mu$ agrees with the record's formula Omega_components:
   $\Omega_{x_i} = \tfrac12 e^{a_4}\sin^{1/6}z\,(a_4'\gamma^{(xi)}\gamma^{(x4)} +
   H\gamma^{(xi)}\gamma^{(x0)})$ for $i = 1, 2, 3$, $\Omega_{x_t} = -\tfrac12
   e^{-a_4}\sin^{1/6}z\,(a_4'\gamma^{(x4)}\gamma^{(xt)} + H\gamma^{(xt)}
   \gamma^{(x0)})$ for $t = 5, 6, 7$, and $\Omega_{x_4} = \Omega_{x_0} = 0$ (the
   cell also checks that the record's text, in Wolfram notation with `g[x4]` for
   $\gamma^{(x4)}$, states exactly these pieces);
3. the contraction $\gamma^\mu\Omega_\mu = \sum_\mu f_\mu^{-1}\gamma^\mu
   \Omega_\mu = 3H\gamma^{(x0)}$ (record formula gammaOmega_total).
```

In [4]:

```
a4p = sp.Derivative(a4, x[4])  # a4' = d a4 / d x4
real_ok = all(not entry.has(sp.I) for Om in Omega for entry in Om)
nonzero = any(Om != sp.zeros(16, 16) for Om in Omega)
check(real_ok and nonzero and LEAD["spinor_connection_real"] == "PASS",
      "every entry of Omega_mu is real and not all vanish",
      record=f"{LEAD_FILE}, check spinor_connection_real")

s16 = sp.sin(z) ** sp.Rational(1, 6)
expected = []
for mu in range(8):
    if mu in (1, 2, 3):  # x1, x2, x3: inflating ordinary space
        expected.append(sp.exp(a4) * s16 / 2 * (a4p * G[mu] * G[4] + H * G[mu] * G[0]))
    elif mu in (5, 6, 7):  # x5, x6, x7: the deflating extra times
        expected.append(-sp.exp(-a4) * s16 / 2 * (a4p * G[4] * G[mu]
                                                  + H * G[mu] * G[0]))
    else:  # x4 and x0
        expected.append(sp.zeros(16, 16))
record_text = THEORY["Omega_components"]["wl"]  # the record's statement
pieces_of_record = [  # the formula above, piece by piece, in the record's notation
    "Omega_xi = (1/2) E^a4[x4] Sin[6 H x0]^(1/6) (a4",
    "g[xi].g[x4] + H g[xi].g[x0]) (i = 1, 2, 3)",
    "Omega_xt = -(1/2) E^-a4[x4] Sin[6 H x0]^(1/6) (a4",
    "g[x4].g[xt] + H g[xt].g[x0]) (t = 5, 6, 7)",
    "Omega_x0 = Omega_x4 = 0"]
check(all((Omega[mu] - expected[mu]).applyfunc(sp.simplify) == sp.zeros(16, 16)
          for mu in range(8))
      and all(piece in record_text for piece in pieces_of_record),
      "Omega_mu equals the record formula Omega_components",
      record="Revision/theory/field-theory.json, formula Omega_components")
gamma_up = [G[mu] / f_dfl[mu] for mu in range(8)]  # gamma^mu = gamma^a / f_a
contraction = sum((gamma_up[mu] * Omega[mu] for mu in range(8)), sp.zeros(16, 16))
check((contraction - 3 * H * G[0]).applyfunc(sp.simplify) == sp.zeros(16, 16)
      and THEORY["gammaOmega_total"]["wl"] == '3*H*gamma["x0"]',
      "gamma^mu Omega_mu = 3 H gamma^(x0) for every history a4",
      record="Revision/theory/field-theory.json, formula gammaOmega_total")
```

Out [4]:

```
PASS every entry of Omega_mu is real and not all vanish
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
```

```
    spinor_connection_real
PASS Omega_mu equals the record formula Omega_components
     reproduces Revision/theory/field-theory.json, formula Omega_components
PASS gamma^mu Omega_mu = 3 H gamma^(x0) for every history a4
     reproduces Revision/theory/field-theory.json, formula gammaOmega_total
```

Text cell:

```
## 7. The U(1) identity as a matrix identity

The next cell computes both sides of the matrix identity of section 4, line 5:
$$\mathrm{LHS} = \sum_\mu\partial_\mu(\cos z\,C\gamma^\mu),\qquad
\mathrm{RHS} = \cos z\sum_\mu\big(C\gamma^\mu\Omega_\mu - \Omega_\mu^T
(\gamma^\mu)^TC\big),$$
checks that their difference simplifies to the zero matrix (record check
u1_noether_matrix_identity), and checks that the left side is exactly $6H\cos z\,
C\gamma^{(x0)}$. As a **negative control** it sets the spin connection to zero:
then the right side vanishes while the left side does not, so without the spin
connection the current would not be conserved.
```

In [5]:

```
cos_z = vol_dfl  # sqrt|g| = cos z
lhs = sum((sp.diff(cos_z / f_dfl[mu], x[mu]) * C_s * G[mu] for mu in range(8)),
          sp.zeros(16, 16))
pieces = [cos_z * (C_s * gamma_up[mu] * Omega[mu]
                   - Omega[mu].T * gamma_up[mu].T * C_s) for mu in range(8)]
rhs = sum(pieces, sp.zeros(16, 16))
check((lhs - rhs).applyfunc(sp.simplify) == sp.zeros(16, 16)
      and LEAD["u1_noether_matrix_identity"] == "PASS",
      "the U(1) Noether matrix identity holds exactly for a general history a4",
      record=f"{LEAD_FILE}, check u1_noether_matrix_identity")
check((lhs - 6 * H * sp.cos(z) * C_s * G[0]).applyfunc(sp.simplify)
      == sp.zeros(16, 16), "the left side is exactly 6 H cos z C gamma^(x0)")
check(lhs.applyfunc(sp.simplify) != sp.zeros(16, 16),
      "negative control: with Omega = 0 the right side is 0 but the left side is "
      "not")
```

Out [5]:

```
PASS the U(1) Noether matrix identity holds exactly for a general history a4
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    u1_noether_matrix_identity
PASS the left side is exactly 6 H cos z C gamma^(x0)
PASS negative control: with Omega = 0 the right side is 0 but the left side is not
```

Text cell:

```
**Why the $a_4'$ terms cancel.** The next cell splits the right side into the
contributions of the eight directions. Each is a combination of $C\gamma^{(x4)}$
```

```
and $C\gamma^{(x0)}$; the cell extracts the two coefficients (divided by the
volume factor $\sqrt{|g|}$) by the trace formula $c = \mathrm{tr}(P\,Y^T)/16$ for
$P = c\,Y + \dots$ (the products of gammas are orthogonal under this trace). Each
direction of ordinary space contributes $+a_4'$ to the $C\gamma^{(x4)}$
coefficient, each deflating extra time $-a_4'$: they cancel. Then it repeats the
whole computation for a **control metric** in which the extra times *inflate*
($e^{+a_4}$): the identity still holds (it holds in every metric), but now the six
contributions add up to $6a_4'$, and the left side contains the same $6a_4'$,
because the volume factor $\sqrt{|g|} = e^{6a_4}\cos z$ grows. In the author's
metric $\sqrt{|g|} = e^{3a_4}e^{-3a_4}\cos z = \cos z$ does not depend on $x_4$:
**the deflation of the extra times exactly compensates the inflation of space**,
and the time-derivative terms drop out. (About 7 seconds.)
```

In [6]:

```
CG4, CG0 = C_s * G[4], C_s * G[0]


def coefficients(matrix, volume):
    """The coefficients of C gamma^(x4) and C gamma^(x0) in matrix, divided by the
    volume factor (trace formula; checked to leave no remainder)."""
    c4 = sp.simplify((matrix * CG4.T).trace() / 16 / volume)
    c0 = sp.simplify((matrix * CG0.T).trace() / 16 / volume)
    remainder = (matrix - volume * (c4 * CG4 + c0 * CG0)).applyfunc(sp.simplify)
    return c4, c0, remainder == sp.zeros(16, 16)


dfl = [coefficients(p, cos_z) for p in pieces]  # the author's metric
f_ctl, vol_ctl, Omega_ctl = geometry(+1)  # control: inflating extra times
gamma_up_ctl = [G[mu] / f_ctl[mu] for mu in range(8)]
pieces_ctl = [vol_ctl * (C_s * gamma_up_ctl[mu] * Omega_ctl[mu]
                         - Omega_ctl[mu].T * gamma_up_ctl[mu].T * C_s)
              for mu in range(8)]
lhs_ctl = sum((sp.diff(vol_ctl / f_ctl[mu], x[mu]) * C_s * G[mu] for mu in range(8)),
              sp.zeros(16, 16))
ctl = [coefficients(p, vol_ctl) for p in pieces_ctl]
lhs_c4, _, _ = coefficients(lhs_ctl, vol_ctl)  # control: the volume-growth term
lhs_c4_dfl, _, _ = coefficients(lhs, cos_z)  # author's metric: zero
say(f"control metric: sqrt|g| = {vol_ctl}")
for mu in range(8):  # the coefficients in units of a4' (sympy divides exactly)
    say(f"{COORDS[mu]}: author's metric {str(sp.simplify(dfl[mu][0] / a4p)):>2} a4'"
        f" | control metric {str(sp.simplify(ctl[mu][0] / a4p)):>2} a4'")
expected_dfl = [0] + [a4p] * 3 + [0] + [-a4p] * 3  # +a4' space, -a4' extra times
expected_ctl = [0] + [a4p] * 3 + [0] + [a4p] * 3  # all six +a4'
check(all(c[2] for c in dfl + ctl)
      and all(sp.simplify(c[0] - e) == 0 for c, e in zip(dfl, expected_dfl))
      and all(sp.simplify(c[0] - e) == 0 for c, e in zip(ctl, expected_ctl))
      and sp.simplify(lhs_c4 - 6 * a4p) == 0 and sp.simplify(lhs_c4_dfl) == 0
      and (lhs_ctl - sum(pieces_ctl, sp.zeros(16, 16))).applyfunc(sp.simplify)
      == sp.zeros(16, 16),
      "a4' terms: +a4' (space) and -a4' (deflating extra times) cancel; the "
      "control metric balances 6 a4' by its growing volume")
```

Out [6]:

```
control metric: sqrt|g| = exp(6*a4(x4))*cos(6*H*x0)
x0: author's metric  0 a4' | control metric  0 a4'
x1: author's metric  1 a4' | control metric  1 a4'
x2: author's metric  1 a4' | control metric  1 a4'
x3: author's metric  1 a4' | control metric  1 a4'
x4: author's metric  0 a4' | control metric  0 a4'
x5: author's metric -1 a4' | control metric  1 a4'
x6: author's metric -1 a4' | control metric  1 a4'
x7: author's metric -1 a4' | control metric  1 a4'
PASS a4' terms: +a4' (space) and -a4' (deflating extra times) cancel; the control metric
    balances 6 a4' by its growing volume
```

Text cell:

```
The next cell draws the $C\gamma^{(x4)}$ coefficients of the eight directions
(in units of $a_4'$) for the author's metric and for the control metric, together
with the coefficient of the left side (the volume-growth term).
```

In [7]:

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.3), sharey=True)
labels = COORDS + ["left side"]
for ax, data, lhs_value, title in (
        (axes[0], dfl, lhs_c4_dfl, "author's metric: extra times deflate"),
        (axes[1], ctl, lhs_c4, "control: extra times inflate")):
    heights = [float(sp.simplify(c[0] / a4p)) for c in data + [(lhs_value,)]]
    colours = ["#e34948" if h > 0 else ("#2a78d6" if h < 0 else "#999999")
               for h in heights[:8]] + ["#555555"]
    ax.bar(range(9), heights, color=colours)
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(range(9), labels, rotation=30)
    ax.set_title(title + f"; sum of the eight = {round(sum(heights[:8]))}")
    ax.set_xlabel("direction $\\mu$ of the term")
axes[0].set_ylabel("coefficient of $C\\gamma^{(x4)}$ in units of $a_4'$")
save_figure(fig, "volume_balance",
            "Why the time-derivative terms of the U(1) identity cancel. Each bar is "
            "the coefficient of $C\\gamma^{(x4)}$, in units of $a_4'$ and divided by "
            "the volume factor, contributed by one direction $\\mu$ to the right "
            "side $\\cos z\\,\\sum_\\mu(C\\gamma^\\mu\\Omega_\\mu - \\Omega_\\mu^T"
            "(\\gamma^\\mu)^TC)$; the last bar is the coefficient of the left "
            "side. Left: the author's metric, in which ordinary space contributes "
            "$+1$ per direction and the deflating extra times $-1$ each; the sum "
            "and the left side are $0$. Right: a control metric with inflating "
            "extra times; the six contributions add to $6$, matched by the growing "
            "volume factor $e^{6a_4}\\cos z$ on the left side.")
```

Out [7]:

```
Figure 21b.1 saved as Revision/textbook/figures/21b_1_volume_balance.png
```

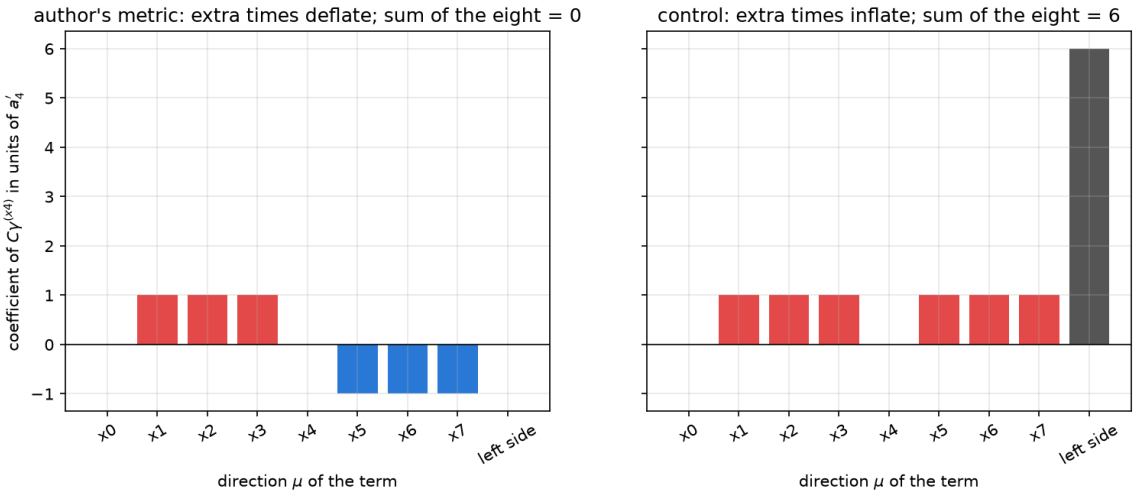

Figure 529: Why the time-derivative terms of the U(1) identity cancel. Each bar is the coefficient of $C\gamma^{(x4)}$, in units of a_4' and divided by the volume factor, contributed by one direction μ to the right side $\cos z \sum_\mu (C\gamma^\mu \Omega_\mu - \Omega_\mu^T (\gamma^\mu)^T C)$; the last bar is the coefficient of the left side. Left: the author's metric, in which ordinary space contributes $+1$ per direction and the deflating extra times -1 each; the sum and the left side are 0. Right: a control metric with inflating extra times; the six contributions add to 6, matched by the growing volume factor $e^{6a_4} \cos z$ on the left side. (Notebook 21b, figure 1.)

Text cell:

```
The next cell draws the three $16 \times 16$ matrices of the identity, divided by
$H\cos z$: the left side ($6C\gamma^{(x0)}$), the right side (the same matrix),
and the right side of the negative control without spin connection (zero).
```

In [8]:

```
from matplotlib.colors import TwoSlopeNorm

sample = {a4p: 0.7, a4: 0.3, H: sp.Rational(1, 6), x[0]: 0.5}  # any point


def sampled(matrix):
    """matrix / (H cos z) evaluated at the sample point, as numbers."""
    return np.array(sp.N((matrix / (H * sp.cos(z))).subs(a4p, 0.7).subs(sample))
                    .tolist(), dtype=float)


lhs_num, rhs_num = sampled(lhs), sampled(rhs)
say(f"LHS = RHS at the sample point to 1e-12: "
    f"{np.abs(lhs_num - rhs_num).max() < 1e-12}")
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
norm = TwoSlopeNorm(vmin=-6, vcenter=0, vmax=6)  # blue negative, red positive
for ax, matrix, title in ((axes[0], lhs_num, "left side / $(H\\cos z)$"),
                          (axes[1], rhs_num, "right side / $(H\\cos z)$"),
```

```
                          (axes[2], 0 * rhs_num, "right side, $\\Omega = 0$")):
    image = ax.imshow(matrix, cmap="RdBu_r", norm=norm)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("column")
    ax.grid(False)
axes[0].set_ylabel("row")
fig.colorbar(image, ax=axes, shrink=0.8, label="matrix entry")
save_figure(fig, "noether_matrices",
            "The $16 \\times 16$ matrices of the U(1) identity in the author's "
            "metric, divided by $H\\cos z$: left side $\\sum_\\mu\\partial_\\mu"
            "(\\cos z\\,C\\gamma^\\mu)$ (left panel), right side $\\cos z\\sum_\\mu"
            "(C\\gamma^\\mu\\Omega_\\mu - \\Omega_\\mu^T(\\gamma^\\mu)^TC)$ (middle) "
            "and the right side of the negative control with the spin connection "
            "removed (right); horizontal axis the column, vertical axis the row, "
            "colour the entry (red positive, blue negative). The first two are the "
            "same matrix $6C\\gamma^{(x0)}$ for every history $a_4$; without the "
            "spin connection the right side would be zero and the current would "
            "not be conserved.")
```

Out [8]:

```
LHS = RHS at the sample point to 1e-12: True
```

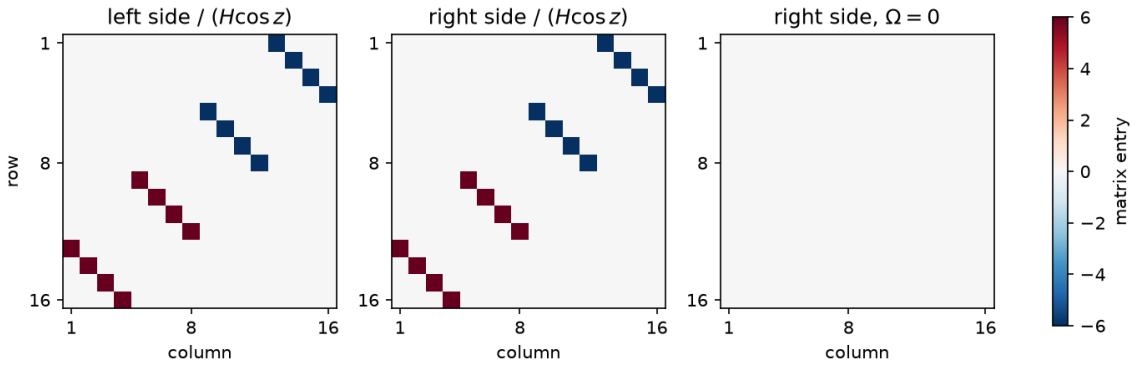

Figure 530: The 16×16 matrices of the U(1) identity in the author's metric, divided by $H\cos z$: left side $\sum_\mu \partial_\mu(\cos z\, C\gamma^\mu)$ (left panel), right side $\cos z \sum_\mu (C\gamma^\mu\Omega_\mu - \Omega_\mu^T(\gamma^\mu)^T C)$ (middle) and the right side of the negative control with the spin connection removed (right); horizontal axis the column, vertical axis the row, colour the entry (red positive, blue negative). The first two are the same matrix $6C\gamma^{(x0)}$ for every history a_4; without the spin connection the right side would be zero and the current would not be conserved. (Notebook 21b, figure 2.)

Out [8]:

```
Figure 21b.2 saved as Revision/textbook/figures/21b_2_noether_matrices.png
```

Text cell:

8. The Noether current from a local phase change

Noether's construction finds the current by making the phase depend on the point: $\Psi \to e^{i\alpha(x)}\Psi$. Then $D_\mu(e^{i\alpha}\Psi) = e^{i\alpha}(D_\mu\Psi + i(\partial_\mu\alpha)\Psi)$, and inserting this into the kinetic term gives, line by line,
$$\tfrac12\big(\bar\Psi\gamma^\mu(D_\mu\Psi + i\alpha_\mu\Psi) - (D_\mu\bar\Psi - i\alpha_\mu\bar\Psi)\gamma^\mu\Psi\big) = K + i\alpha_\mu\bar\Psi\gamma^\mu\Psi = K - \alpha_\mu J^\mu ,$$
with $\alpha_\mu = \partial_\mu\alpha$ and $\bar\Psi\gamma^\mu\Psi = iJ^\mu$. So $\mathcal{L}' - \mathcal{L} = -\cos z\,\alpha_\mu J^\mu$: the current is the coefficient of $\partial_\mu\alpha$. The next cell checks this with numbers at one point of the author's metric: it evaluates the full Lagrangian (with the spin connection, $m = 0.7$, $\lambda = 0.3$, $U = \tfrac\lambda2S^2$) for a fixed complex field value and fixed complex first derivatives, once as they are and once after the local phase change, and compares the difference with $-\cos z\, \alpha_\mu J^\mu$. With all $\alpha_\mu = 0$ (a constant phase) the Lagrangian does not change at all.

In [9]:

```
point = {x[0]: 0.7, H: sp.Rational(1, 6)}  # z = 0.7 (H = 1/6, so z = x0)
A4_value, A4_slope = 0.4, 0.25  # a4 and a4' at the point (any values)


def at_point(expression):
    """A sympy expression evaluated at the point, as a numpy array of complex."""
    e = expression.subs(a4p, A4_slope).subs(a4, A4_value).subs(point)
    return np.array(sp.N(e).tolist(), dtype=complex) if hasattr(e, "tolist") \
        else complex(sp.N(e))


Om_num = [at_point(Om) for Om in Omega]  # Omega_mu at the point
f_num = [at_point(v).real for v in f_dfl]  # the scale factors at the point
g_up = [gamma[mu] / f_num[mu] for mu in range(8)]  # gamma^mu at the point
cos_num = np.cos(0.7)
m_val, lam = 0.7, 0.3
rng = np.random.default_rng(2102)  # fixed seed: the same numbers in every run
psi = rng.normal(size=16) + 1j * rng.normal(size=16)  # Psi at the point
dpsi = rng.normal(size=(8, 16)) + 1j * rng.normal(size=(8, 16))  # d_mu Psi
alpha_mu = rng.normal(size=8)  # the slopes d_mu alpha of the phase
# drawn with x0 last, as before the move to x0..x7; the roll puts x0 first
dpsi, alpha_mu = np.roll(dpsi, 1, axis=0), np.roll(alpha_mu, 1)


def lagrangian(p, dp):
    """cos z [ (1/2)(Psibar gamma^mu D_mu Psi - (D_mu Psibar) gamma^mu Psi)
    - m S - (lambda/2) S^2 ] at the point, for the value p and derivatives dp."""
    bar = np.conj(p) @ C  # Psibar = Psi^dagger C
    kinetic = 0
    for mu in range(8):
        D = dp[mu] + Om_num[mu] @ p  # D_mu Psi
        Dbar = np.conj(dp[mu]) @ C - bar @ Om_num[mu]  # D_mu Psibar
```

```
        kinetic += (bar @ g_up[mu] @ D - Dbar @ g_up[mu] @ p) / 2
    S = bar @ p
    return cos_num * (kinetic - m_val * S - lam / 2 * S ** 2)


phase = np.exp(1j * 0.9)  # e^(i alpha) at the point, alpha = 0.9
L0 = lagrangian(psi, dpsi)
L_const = lagrangian(phase * psi, phase * dpsi)  # constant phase: alpha_mu = 0
L_local = lagrangian(phase * psi, phase * (dpsi + 1j * alpha_mu[:, None] * psi))
J = np.array([-1j * (np.conj(psi) @ C @ g_up[mu] @ psi) for mu in range(8)])
say(f"L = {L0.real:+.6f} (imaginary part below 1e-12: {abs(L0.imag) < 1e-12}); "
    f"after a constant phase {L_const.real:+.6f}")
say(f"L' - L = {(L_local - L0).real:+.6f};  -cos z alpha_mu J^mu = "
    f"{(-cos_num * alpha_mu @ J).real:+.6f}")
check(abs(L0.imag) < 1e-12 and abs(L_const - L0) < 1e-12
      and abs((L_local - L0) - (-cos_num * alpha_mu @ J)) < 1e-12
      and np.allclose(J.imag, 0),
      "L is real and phase invariant; a local phase changes it by -cos z "
      "(d_mu alpha) J^mu with J^mu = -i Psibar gamma^mu Psi real")
```

Out [9]:

```
L = -16.780795 (imaginary part below 1e-12: True); after a constant phase -16.780795
L' - L = +2.348835;  -cos z alpha_mu J^mu = +2.348835
PASS L is real and phase invariant; a local phase changes it by -cos z (d_mu alpha) J^mu
    with J^mu = -i Psibar gamma^mu Psi real
```

Text cell:

```
## 9. The charge of an exact solution in the author's metric

The Revision field-theory record (formula exact_solutions, item (i)) gives the
exact solution with $U = 0$
$$\Psi = \sin^\alpha z\,\Big(\cos(wx_4)\,1 + \frac{\sin(wx_4)}{w}M\Big)\chi,\quad
M = -m\gamma^{(x4)} + b\,\gamma^{(x4)}\gamma^{(x0)},\quad b = 3H(2\alpha + 1),$$
written here for $k^2 = b^2 - m^2 = -w^2 < 0$ (then $\cosh(kx_4) = \cos(wx_4)$
and $\sinh(kx_4)/k = \sin(wx_4)/w$). It does not depend on $x1, x2, x3, x5, x6,
x7$, so the factors $e^{\pm a_4}$ of the field equation multiply derivatives that
vanish: it solves the field equation for **every** history $a_4$, the deflating
one included. We take $H = 1/6$ (so $z = x_0$), $\alpha = 1$, $m = 2$, so $b =
3/2$ and $w = \sqrt7/2$, and a fixed complex column $\chi$ with exact rational
entries. The current has two nonzero components:
$$\cos z\,J^{(x4)} = \cos z\,\Psi^\dagger B\Psi,\qquad
\cos z\,J^{(x0)} = \cos z\tan z\,\Psi^\dagger K_0\Psi = \sin z\,\Psi^\dagger K_0
\Psi,\quad K_0 = -iC\gamma^{(x0)},$$
because $\gamma^{x0} = \gamma^{(x0)}/f_0 = \tan z\,\gamma^{(x0)}$. The next cell
checks that the record states item (i) word for word as used here (in Wolfram
notation, `al` is $\alpha$ and $k^2 = 9H^2(2\alpha + 1)^2 - m^2 = b^2 - m^2$),
builds $\Psi$, checks the field equation exactly, and checks the local
conservation law $\partial_4(\cos z\,J^{(x4)}) + \partial_z(\cos z\,J^{(x0)}) = 0$
exactly ($\partial_0 = \partial_z$ for $H = 1/6$).
```

In [10]:

```
x4, zz = sp.symbols("x4 z", real=True)  # the time and the angle z = x0
Hn, al, m_sol = sp.Rational(1, 6), 1, 2
b_sol = 3 * Hn * (2 * al + 1)  # 3/2
w = sp.sqrt(m_sol ** 2 - b_sol ** 2)  # sqrt(7)/2
M_sol = -m_sol * G[4] + b_sol * G[4] * G[0]
U_sol = sp.cos(w * x4) * sp.eye(16) + sp.sin(w * x4) / w * M_sol  # a real matrix
chi_values = [(0.62, -0.31), (0.15, 0.88), (-0.47, 0.26), (0.93, -0.05),
              (-0.21, -0.64), (0.38, 0.12), (0.07, -0.93), (-0.85, 0.44),
              (0.51, 0.69), (-0.12, -0.27), (0.29, 0.58), (-0.66, 0.03),
              (0.44, -0.72), (0.81, 0.35), (-0.39, -0.18), (0.24, 0.97)]
chi = sp.Matrix([sp.Rational(round(100 * re), 100) + sp.I * sp.Rational(
    round(100 * im), 100) for re, im in chi_values])  # exact rational entries
Psi = sp.sin(zz) ** al * U_sol * chi
tan_z = sp.sin(zz) / sp.cos(zz)
E = (G[4] * Psi.diff(x4) + 6 * Hn * tan_z * G[0] * Psi.diff(zz)
     + 3 * Hn * G[0] * Psi - m_sol * Psi)  # the field equation, U = 0
SOLUTION_I = (  # item (i) of the record's formula exact_solutions, word for word
    "(i) U = 0: Psi = Sin[z]^al (Cosh[k x4] + Sinh[k x4]/k M) chi, M = -m g[x4]"
    " + 3 H (2 al + 1) g[x4].g[x0], k^2 = 9 H^2 (2 al + 1)^2 - m^2 (any a4)")
check(E.expand() == sp.zeros(16, 1)
      and THEORY["exact_solutions"]["wl"].startswith(SOLUTION_I + "; ")
      and PY_THEORY["exact_solution_family_x4_x0"] == "PASS",
      "the record's exact solution solves the field equation (m = 2)",
      record="Revision/theory/reports/python-field-theory.json, check "
      "exact_solution_family_x4_x0")
B_s = -sp.I * C_s * G[4]
K0_s = -sp.I * C_s * G[0]
density = sp.cos(zz) * (Psi.H * B_s * Psi)[0, 0]  # cos z J^(x4)
flux = sp.sin(zz) * (Psi.H * K0_s * Psi)[0, 0]  # cos z J^(x0)
divergence = sp.expand(sp.diff(density, x4) + sp.diff(flux, zz))
check(sp.simplify(divergence) == 0
      and PY_THEORY["commuting_current_conservation"] == "PASS",
      "d4(cos z J^(x4)) + dz(cos z J^(x0)) = 0 exactly for the exact solution",
      record="Revision/theory/reports/python-field-theory.json, check "
      "commuting_current_conservation")
```

Out [10]:

```
PASS the record's exact solution solves the field equation (m = 2)
     reproduces Revision/theory/reports/python-field-theory.json, check
    exact_solution_family_x4_x0
PASS d4(cos z J^(x4)) + dz(cos z J^(x0)) = 0 exactly for the exact solution
     reproduces Revision/theory/reports/python-field-theory.json, check
    commuting_current_conservation
```

Text cell:

```
The next cell draws the charge density $\cos z\,J^{(x4)}$ over the time $x_4$ and
the angle $z$, for $\Psi$ and for its chirality partner $\Gamma\Psi$, and their
```

```
sum. The partner is a solution of the field equation with the mass reversed
(theorem T1 of the Revision pairing record, here with $\lambda = 0$); the cell
checks this exactly and checks that its whole current is the opposite of that of
$\Psi$, so the sum is zero everywhere (record checks T1_current_primordial_commuting
and T1.metric.commuting.current).
```

In [11]:

```
Gamma_s = sp.Matrix(Gamma.tolist())
partner = Gamma_s * Psi
E_partner = (G[4] * partner.diff(x4) + 6 * Hn * tan_z * G[0] * partner.diff(zz)
             + 3 * Hn * G[0] * partner + m_sol * partner)  # mass -2
density_p = sp.cos(zz) * (partner.H * B_s * partner)[0, 0]
flux_p = sp.sin(zz) * (partner.H * K0_s * partner)[0, 0]
check(E_partner.expand() == sp.zeros(16, 1)
      and sp.expand(density_p + density) == 0 and sp.expand(flux_p + flux) == 0
      and WL_PAIR["T1_current_primordial_commuting"] == "PASS"
      and PY_PAIR["T1.metric.commuting.current"] == "PASS",
      "Gamma Psi solves the equation with mass -2 and carries the opposite "
      "current: the pair's total current is zero",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "T1_current_primordial_commuting")

density_f = sp.lambdify((x4, zz), density, "numpy")
times = np.linspace(0.0, 8.0, 161)
angles = np.linspace(0.0, np.pi / 2, 91)
T, Z = np.meshgrid(times, angles)  # every pair (time, angle)
rho = np.real(density_f(T, Z))  # the charge density of Psi
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0), sharey=True)
limit = np.abs(rho).max()
for ax, values, title in ((axes[0], rho, "$\\Psi$ (mass $+2$)"),
                          (axes[1], -rho, "$\\Gamma\\Psi$ (mass $-2$)"),
                          (axes[2], rho - rho, "sum of the pair")):
    image = ax.pcolormesh(T, Z, values, cmap="RdBu_r", vmin=-limit, vmax=limit,
                          shading="auto")
    ax.set_title(title)
    ax.set_xlabel("time $x_4$")
axes[0].set_ylabel("angle $z = 6Hx_0$ (brane at $\\pi/2$)")
fig.colorbar(image, ax=axes, shrink=0.85, label="charge density $\\cos z\\,J^{(x4)}$")
save_figure(fig, "charge_density_maps",
            "The charge density $\\cos z\\,J^{(x4)}$ of an exact solution of the "
            "field equation in the author's metric ($H = 1/6$, $\\alpha = 1$, mass "
            "$+2$; left), of its chirality partner $\\Gamma\\Psi$ (a solution with "
            "mass $-2$; middle) and of the pair (right); horizontal axis the time "
            "$x_4$, vertical axis the angle $z = 6Hx_0$ from the tip $0$ to the "
            "brane $\\pi/2$, colour the density (red positive, blue negative, pure "
            "numbers). The partner carries exactly the opposite density at every "
            "point, so the pair carries none.")
```

Out [11]:

```
PASS Gamma Psi solves the equation with mass -2 and carries the opposite current: the
    pair's total current is zero
```

```
 reproduces Revision/pairing/reports/wolfram-pairing.json, check
T1_current_primordial_commuting
```

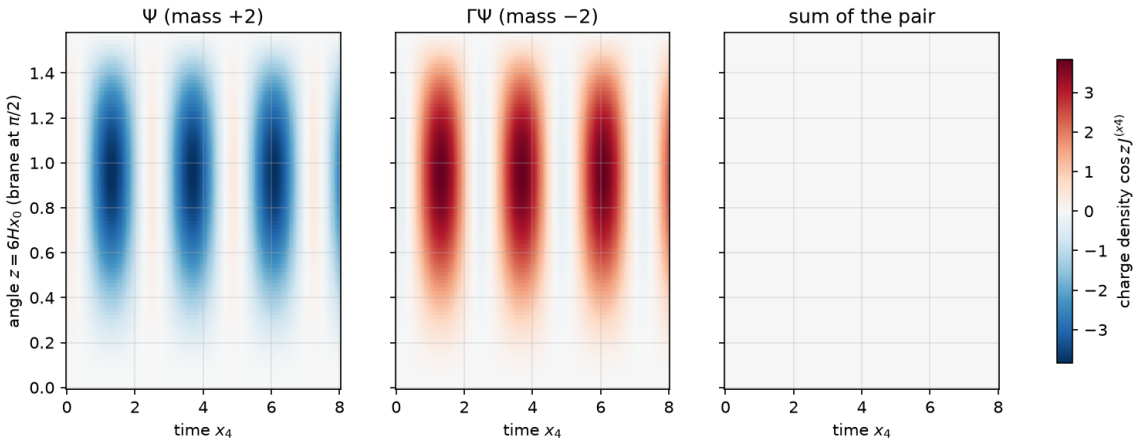

Figure 531: The charge density $\cos z\, J^{(x4)}$ of an exact solution of the field equation in the author's metric ($H = 1/6$, $\alpha = 1$, mass $+2$; left), of its chirality partner $\Gamma\Psi$ (a solution with mass -2; middle) and of the pair (right); horizontal axis the time x_4, vertical axis the angle $z = 6Hx_0$ from the tip 0 to the brane $\pi/2$, colour the density (red positive, blue negative, pure numbers). The partner carries exactly the opposite density at every point, so the pair carries none. (Notebook 21b, figure 3.)

Out [11]:

```
Figure 21b.3 saved as Revision/textbook/figures/21b_3_charge_density_maps.png
```

Text cell:

```
## 10. The charge balance and a solution with zero flux

Integrate the conservation law over the patch $0 < z < \pi/2$ (for each unit of
the other six coordinates). Since $\Psi$ contains $\sin z$, the density is
$\cos z\,\sin^2z\,f(x_4)$ with $f = \chi^\dagger U^TBU\chi$, and the charge is
$$Q(x_4) = \int_0^{\pi/2}\cos z\,\sin^2 z\,dz\;f(x_4) = \tfrac13 f(x_4)$$
(substitute $u = \sin z$: $\int_0^1 u^2\,du = 1/3$). The flux $\sin^3z\,g(x_4)$,
$g = \chi^\dagger U^TK_0U\chi$, vanishes at the tip $z = 0$ and equals $g(x_4)$ at
the brane $z = \pi/2$. The fundamental theorem of calculus gives the **charge
balance**
$$Q(x_4) - Q(0) = -\int_0^{x_4} g(t)\,dt :$$
the charge of the patch changes exactly by what flows out through the brane. The
next cell checks this exactly (sympy integrates $g$ exactly).

Can the flux vanish? Since $M^2 = -w^2\,1$, the matrix $P = \tfrac12(1 - iM/w)$
satisfies $P^2 = P$ (a projector) and $MP = iwP$. For $\chi_0 = P\chi$ we get
$M\chi_0 = iw\chi_0$, hence $U\chi_0 = (\cos(wx_4) + i\sin(wx_4))\chi_0 =
e^{iwx_4}\chi_0$, and the solution $\Psi_0 = \sin z\,e^{iwx_4}\chi_0$ has a
single frequency: it is a **stationary state**. Its charge density
$\cos z\sin^2z\,\chi_0^\dagger B\chi_0$ does not depend on the time, so by the
conservation law its flux is zero, and its charge $Q$ is **constant**. The cell
checks this too, and that this constant is not zero.
```

In [12]:

```
t = sp.symbols("t", real=True)
f_of = sp.expand((U_sol * chi).H * B_s * (U_sol * chi))[0, 0]  # f(x4)
g_of = sp.expand((U_sol * chi).H * K0_s * (U_sol * chi))[0, 0]  # g(x4)
Q_of = f_of / 3
balance = sp.simplify(Q_of - Q_of.subs(x4, 0)
                      + sp.integrate(g_of.subs(x4, t), (t, 0, x4)))
check(balance == 0, "charge balance: Q(x4) - Q(0) = - (flux through the brane), "
      "exactly")

P_stat = (sp.eye(16) - sp.I * M_sol / w) / 2  # projector onto M chi = i w chi
chi0 = P_stat * chi  # the single-frequency part of chi
g0 = sp.simplify(sp.expand((U_sol * chi0).H * K0_s * (U_sol * chi0))[0, 0])
Q0 = sp.simplify(sp.expand((U_sol * chi0).H * B_s * (U_sol * chi0))[0, 0] / 3)
check((P_stat * P_stat - P_stat).applyfunc(sp.simplify) == sp.zeros(16, 16)
      and (M_sol * chi0 - sp.I * w * chi0).applyfunc(sp.simplify) == sp.zeros(16, 1)
      and g0 == 0 and Q0.free_symbols == set() and Q0 != 0,
      "the stationary solution sin z exp(i w x4) chi0: zero flux, constant nonzero Q")
report("charge of the general solution at x4 = 0",
       f"{float(sp.re(Q_of.subs(x4, 0))):.6f}")
report("constant charge of the stationary solution", f"{Q0} = {float(Q0):.6f}")
```

Out [12]:

```
PASS charge balance: Q(x4) - Q(0) = - (flux through the brane), exactly
PASS the stationary solution sin z exp(i w x4) chi0: zero flux, constant nonzero Q
RESULT charge of the general solution at x4 = 0 = 0.178467
RESULT constant charge of the stationary solution = -1424/1875 - 13*sqrt(7)/1500 =
    -0.782397
```

Text cell:

```
The next cell draws the balance: on the left, for the general $\chi$, the charge
$Q(x_4)$ and the right side $Q(0) - \int_0^{x_4}g$ (they lie on top of each
other) together with the flux $g$ through the brane; on the right, for $\chi_0$,
the constant charge of $\Psi$, of its partner and of the pair.
```

In [13]:

```
Q_f = sp.lambdify(x4, Q_of, "numpy")
g_f = sp.lambdify(x4, g_of, "numpy")
integral_f = sp.lambdify(x4, sp.integrate(g_of.subs(x4, t), (t, 0, x4)), "numpy")
Q_vals = np.real(Q_f(times))
rhs_vals = np.real(Q_f(0.0) - integral_f(times))
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.3))
axes[0].plot(times, Q_vals, linewidth=3, label="charge $Q(x_4)$ of the patch")
axes[0].plot(times, rhs_vals, "--", color="black",
             label="$Q(0) - \\int_0^{x_4}$ flux")
axes[0].plot(times, np.real(g_f(times)), ":", label="flux $g(x_4)$ through the brane")
```

```
axes[0].set_xlabel("time $x_4$")
axes[0].set_ylabel("charge, flux (pure numbers)")
axes[0].set_title("General solution: charge flows through the brane")
top = 1.9 * np.abs(np.real(g_f(times))).max()  # room for the legend above
axes[0].set_ylim(-1.2 * np.abs(np.real(g_f(times))).max(), top)
axes[0].legend(fontsize=8, loc="upper right")
Q0_value = float(Q0)
axes[1].plot(times, np.full_like(times, Q0_value), linewidth=2,
             label="$Q$ of $\\Psi$ (mass $+2$)")
axes[1].plot(times, np.full_like(times, -Q0_value), "--", linewidth=2,
             label="$Q$ of $\\Gamma\\Psi$ (mass $-2$)")
axes[1].plot(times, np.zeros_like(times), ":", color="black", linewidth=2,
             label="total charge of the pair")
axes[1].set_xlabel("time $x_4$")
axes[1].set_title("Zero flux: the charge is constant")
axes[1].set_ylim(-1.3 * abs(Q0_value), 2.2 * abs(Q0_value))  # legend fits on top
axes[1].legend(fontsize=8, loc="upper right")
check(np.max(np.abs(Q_vals - rhs_vals)) < 1e-12,
      "numerically: Q(x4) and Q(0) minus the integrated flux agree to 1e-12")
save_figure(fig, "charge_balance",
            "The charge balance of exact solutions in the author's metric ($H = "
            "1/6$, $\\alpha = 1$, mass $2$). Left: for a general column $\\chi$, "
            "the charge $Q(x_4)$ of the patch $0 < z < \\pi/2$ (thick line), the "
            "prediction $Q(0) - \\int_0^{x_4} g$ of the conservation law (dashed, "
            "on top of it) and the flux $g$ through the brane (dotted), versus the "
            "time $x_4$. Right: for the stationary solution $\\sin z\\,e^{iwx_4}"
            "\\chi_0$ the flux vanishes, its charge is constant, its chirality "
            "partner "
            "carries the opposite constant charge and the pair carries none. All "
            "quantities per unit of the other six coordinates (pure numbers).")
```

Out [13]:

```
PASS numerically: Q(x4) and Q(0) minus the integrated flux agree to 1e-12
```

Out [13]:

```
Figure 21b.4 saved as Revision/textbook/figures/21b_4_charge_balance.png
```

Text cell:

```
**A finite-difference test.** A computer usually checks a conservation law with
numbers on a grid, replacing each derivative by a *central difference*
$\partial_x F \approx (F(x + h) - F(x - h))/(2h)$, whose error is proportional to
$h^2$. The next cell evaluates the divergence $\partial_4(\cos z\,J^{(x4)}) +
\partial_z(\cos z\,J^{(x0)})$ this way at the point $(x_4, z) = (1.3, 0.7)$ for
steps $h = 0.2, 0.1, \dots$: the result is not zero, but it shrinks by a factor 4
each time $h$ is halved (second-order convergence) towards the exact value 0.
```

In [14]:

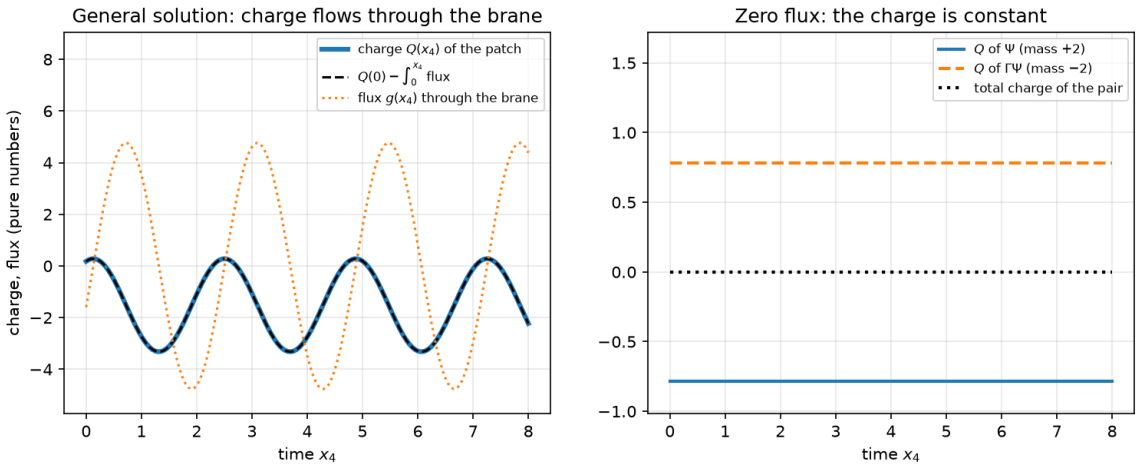

Figure 532: The charge balance of exact solutions in the author's metric ($H = 1/6$, $\alpha = 1$, mass 2). Left: for a general column χ, the charge $Q(x_4)$ of the patch $0 < z < \pi/2$ (thick line), the prediction $Q(0) - \int_0^{x_4} g$ of the conservation law (dashed, on top of it) and the flux g through the brane (dotted), versus the time x_4. Right: for the stationary solution $\sin z\, e^{iwx_4}\chi_0$ the flux vanishes, its charge is constant, its chirality partner carries the opposite constant charge and the pair carries none. All quantities per unit of the other six coordinates (pure numbers). (Notebook 21b, figure 4.)

```
flux_f = sp.lambdify((x4, zz), flux, "numpy")
steps = 0.2 / 2.0 ** np.arange(7)  # 0.2, 0.1, ..., 0.003125
t0, z0 = 1.3, 0.7
residuals = np.array([abs(np.real(
    (density_f(t0 + h, z0) - density_f(t0 - h, z0)) / (2 * h)
    + (flux_f(t0, z0 + h) - flux_f(t0, z0 - h)) / (2 * h))) for h in steps])
ratios = residuals[:-1] / residuals[1:]
say("residuals: " + ", ".join(f"{r:.2e}" for r in residuals))
say("ratios when h is halved: " + ", ".join(f"{q:.3f}" for q in ratios))
check(np.all(np.abs(ratios - 4.0) < 0.05),
      "finite differences: the residual falls by 4 when h is halved (order 2)")

fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.loglog(steps, residuals, "o-", label="|central-difference divergence|")
ax.loglog(steps, residuals[0] * (steps / steps[0]) ** 2, "--",
          label="slope 2: proportional to $h^2$")
ax.set_xlabel("step $h$")
ax.set_ylabel("residual of the conservation law")
ax.set_title("A numerical test converges to the exact law")
ax.legend()
save_figure(fig, "local_conservation",
            "The local conservation law $\\partial_4(\\cos z\\,J^{(x4)}) + "
            "\\partial_z(\\cos z\\,J^{(x0)}) = 0$ tested with central differences "
            "of step $h$ at the point $x_4 = 1.3$, $z = 0.7$ for the exact solution "
            "of the author's metric; horizontal axis the step $h$, vertical axis "
            "the size of the computed divergence, both on logarithmic scales "
            "(pure numbers). The points follow the dashed line of slope 2: the "
            "error of the difference formula, proportional to $h^2$, is all there "
            "is; the exact divergence is zero.")
```

Out [14]:

```
residuals: 3.68e-03, 9.29e-04, 2.33e-04, 5.82e-05, 1.46e-05, 3.64e-06, 9.10e-07
ratios when h is halved: 3.966, 3.991, 3.998, 3.999, 4.000, 4.000
PASS finite differences: the residual falls by 4 when h is halved (order 2)
```

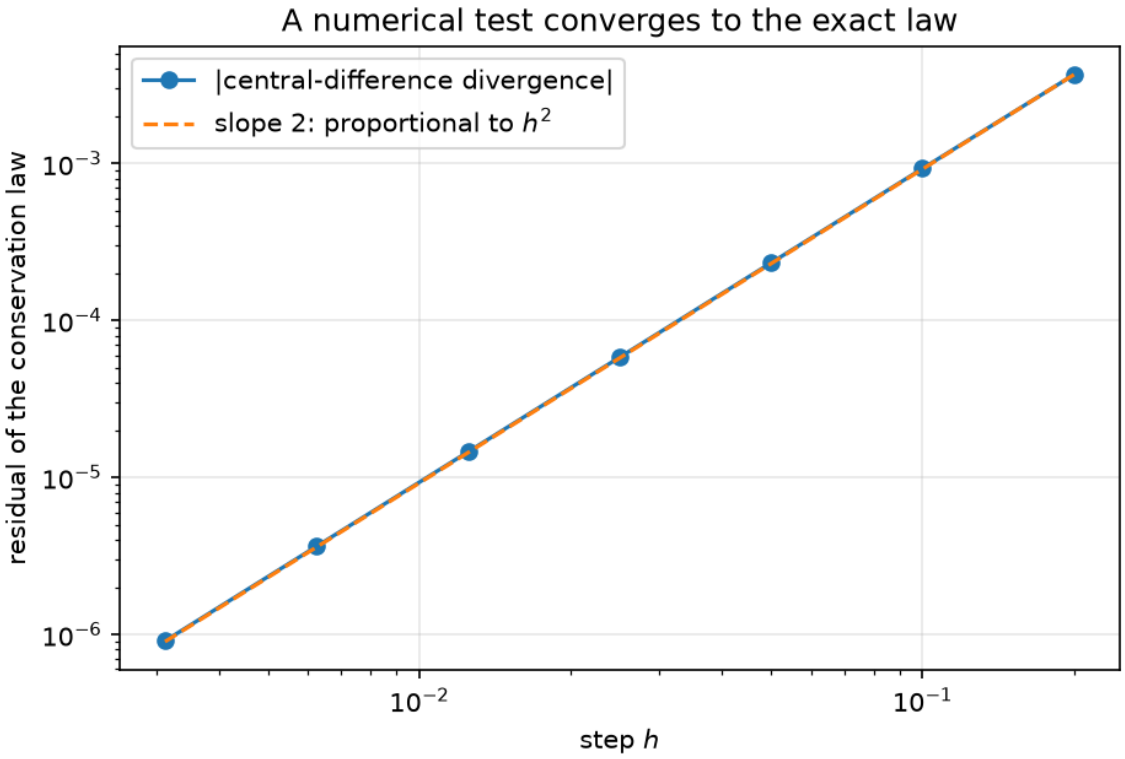

Figure 533: The local conservation law $\partial_4(\cos z\, J^{(x4)}) + \partial_z(\cos z\, J^{(x0)}) = 0$ tested with central differences of step h at the point $x_4 = 1.3$, $z = 0.7$ for the exact solution of the author's metric; horizontal axis the step h, vertical axis the size of the computed divergence, both on logarithmic scales (pure numbers). The points follow the dashed line of slope 2: the error of the difference formula, proportional to h^2, is all there is; the exact divergence is zero. (Notebook 21b, figure 5.)

Out [14]:

```
Figure 21b.5 saved as Revision/textbook/figures/21b_5_local_conservation.png
```

Text cell:

```
**The charge while the extra times deflate.** The next cell draws, on a common
time axis, the scale factors of the author's metric for the linear history
$a_4 = AHx_4$ (here $A = 1$, $H = 1/6$, chosen only for the picture): ordinary
space grows as $e^{a_4}$, the extra times shrink as $e^{-a_4}$, the volume factor
$e^{3a_4}e^{-3a_4} = 1$ stays; and below, the charge of the stationary (zero-flux)
solution, which stays constant. Any other history gives the same constant charge:
the stationary solution does not depend on $x1, x2, x3, x5, x6, x7$, so it solves
```

```
the field equation for every history $a_4$ (Section 9; record check
exact_solution_family_x4_x0), and its charge density
$\cos z\sin^2z\,\chi_0^\dagger B\chi_0$ contains no $a_4$.
```

In [15]:

```
a4_vals = times / 6.0  # a4 = A H x4 with A = 1, H = 1/6
fig, axes = plt.subplots(2, 1, figsize=(8.0, 6.4), sharex=True)
axes[0].semilogy(times, np.exp(a4_vals), label="ordinary space: $e^{a_4}$")
axes[0].semilogy(times, np.exp(-a4_vals), "--",
                 label="extra times $x5, x6, x7$: $e^{-a_4}$ (deflating)")
axes[0].semilogy(times, np.exp(3 * a4_vals) * np.exp(-3 * a4_vals), ":",
                 color="black", label="slice volume $e^{3a_4}e^{-3a_4} = 1$")
axes[0].set_ylabel("scale factor (log scale)")
axes[0].set_title("The history $a_4 = AHx_4$ ($A = 1$, $H = 1/6$)")
axes[0].legend(fontsize=8)
axes[1].plot(times, np.full_like(times, Q0_value), linewidth=2,
             label="charge $Q$ of the stationary solution")
axes[1].set_ylim(-1.5 * abs(Q0_value), 1.5 * abs(Q0_value))
axes[1].axhline(0.0, color="black", linewidth=0.8)
axes[1].set_xlabel("time $x_4$")
axes[1].set_ylabel("charge $Q$")
axes[1].legend(fontsize=8)
save_figure(fig, "charge_through_deflation",
            "Top: the scale factors of the author's metric along the time $x_4$ "
            "for the linear history $a_4 = AHx_4$ with $A = 1$, $H = 1/6$ "
            "(chosen for the picture): ordinary space $e^{a_4}$ grows, the three "
            "extra times $e^{-a_4}$ deflate exponentially, and their product, the "
            "volume factor of a slice, stays $1$ (logarithmic vertical scale). "
            "Bottom: the charge $Q$ of the stationary exact solution versus $x_4$; "
            "it does not change while space inflates and the extra times deflate. "
            "The stationary solution solves the field equation for every "
            "history $a_4$ and its charge density contains no $a_4$, so the "
            "constant does not depend on this choice.")
```

Out [15]:

```
Figure 21b.6 saved as Revision/textbook/figures/21b_6_charge_through_deflation.png
```

Text cell:

```
## 11. The charge density can be negative

The charge density $J^{(x4)} = \Psi^\dagger B\Psi$ is a *quadratic form* with the
matrix $B$, which has eight eigenvalues $+1$ and eight eigenvalues $-1$
(signature (8,8)). Writing $\Psi$ in the eigenvectors of $B$ as $\Psi = \sum_j c_j
v_j$ gives $\Psi^\dagger B\Psi = \sum_{B = +1}|c_j|^2 - \sum_{B = -1}|c_j|^2$, which
can have either sign; divided by $\Psi^\dagger\Psi = \sum|c_j|^2$ it lies between
$-1$ and $+1$.

**A worked example in flat space** ($H = 0$, $a_4$ constant). The field equation
```

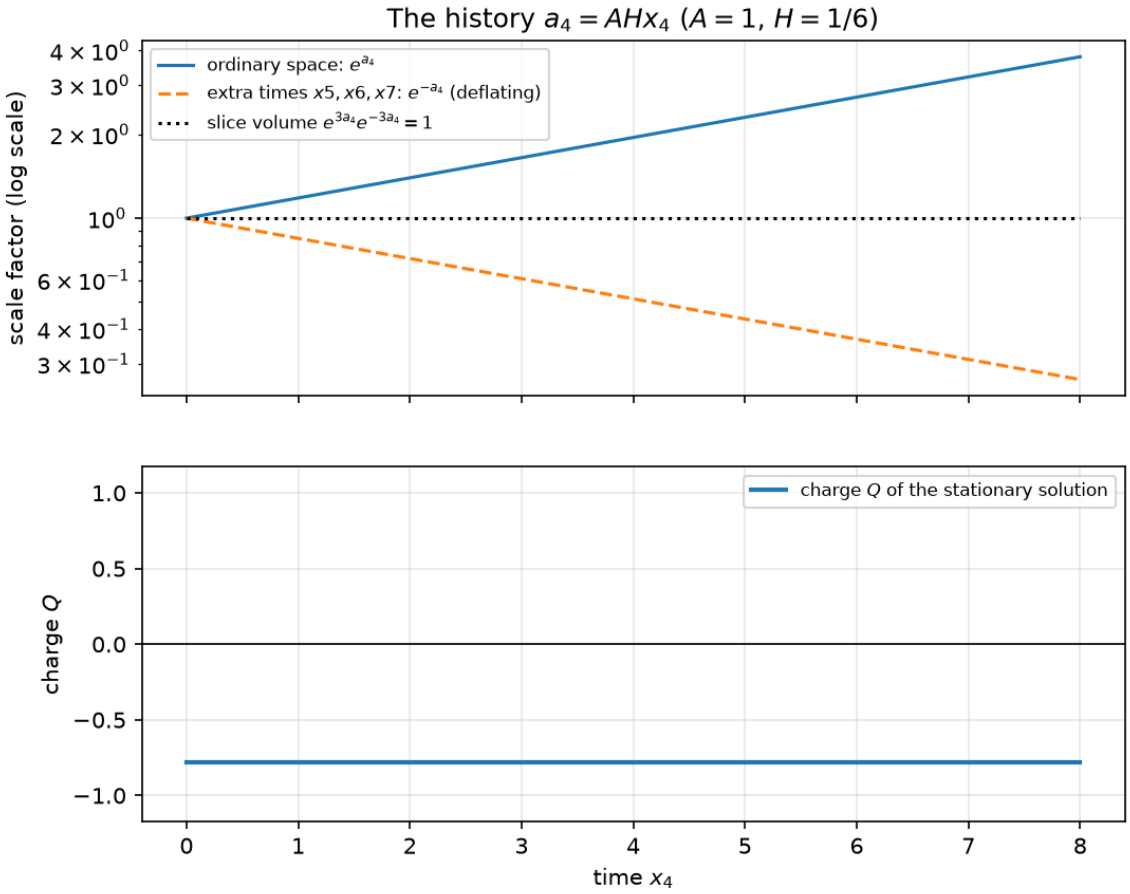

Figure 534: Top: the scale factors of the author's metric along the time x_4 for the linear history $a_4 = AHx_4$ with $A = 1$, $H = 1/6$ (chosen for the picture): ordinary space e^{a_4} grows, the three extra times e^{-a_4} deflate exponentially, and their product, the volume factor of a slice, stays 1 (logarithmic vertical scale). Bottom: the charge Q of the stationary exact solution versus x_4; it does not change while space inflates and the extra times deflate. The stationary solution solves the field equation for every history a_4 and its charge density contains no a_4, so the constant does not depend on this choice. (Notebook 21b, figure 6.)

```
along the time is $\gamma^{(x4)}\partial_4\Psi = m\Psi$. A *rest state* $\Psi =
u\,e^{-imx_4}$ with $-i\gamma^{(x4)}u = u$ solves it: $\gamma^{(x4)}(-im)u =
m(-i\gamma^{(x4)})u = mu$; it has the positive frequency $m$. Since $B =
C(-i\gamma^{(x4)})$ and $C$ commutes with $\gamma^{(x4)}$, $B$ maps the
8-dimensional space of such $u$ into itself and has there four eigenvalues $+1$
(vector $u_+$) and four $-1$ (vector $u_-$). For $u = \tfrac35u_+ + \tfrac45u_-$
(normalised) the charge density is $\tfrac{9}{25} - \tfrac{16}{25} =
-\tfrac{7}{25}$: a positive-frequency field with negative charge density. The
next cell computes this exactly from the gammas and draws the distribution of
$\Psi^\dagger B\Psi/\Psi^\dagger\Psi$ for 20000 random columns.
```

In [16]:

```
from fractions import Fraction

eig_B = np.linalg.eigvalsh(B)
plus_space = (I16 - 1j * gamma[4]) / 2  # projector onto -i gamma^(x4) = +1
# B restricted to that space: an 8-dimensional block, eigenvalues +-1
values, vectors = np.linalg.eigh(plus_space @ B @ plus_space)
u_plus = vectors[:, np.argmax(values)]  # a vector with -i g4 u = u, B u = +u
u_minus = vectors[:, np.argmin(values)]  # a vector with -i g4 u = u, B u = -u
u = 0.6 * u_plus + 0.8 * u_minus  # 3/5 and 4/5
rest_density = np.conj(u) @ B @ u
restricted = np.linalg.eigvalsh(plus_space @ B @ plus_space)
check(np.allclose(sorted(eig_B), [-1] * 8 + [1] * 8)
      and np.allclose(-1j * gamma[4] @ u, u)
      and np.sum(restricted > 0.5) == 4 and np.sum(restricted < -0.5) == 4
      and abs(rest_density - (-7 / 25)) < 1e-12,
      "B has signature (8,8); a positive-frequency rest state has charge density "
      "-7/25")
report("charge density of the rest state 3/5 u+ + 4/5 u-",
       Fraction(round(rest_density.real * 25), 25))

samples = rng.normal(size=(20000, 16)) + 1j * rng.normal(size=(20000, 16))
ratio = (np.einsum("nr,rc,nc->n", np.conj(samples), B, samples).real
         / np.einsum("nr,nr->n", np.conj(samples), samples).real)
say(f"fraction of random columns with negative charge density: "
    f"{np.mean(ratio < 0):.3f}")
check(ratio.min() >= -1 - 1e-12 and ratio.max() <= 1 + 1e-12
      and 0.4 < np.mean(ratio < 0) < 0.6,
      "random columns: the ratio lies in [-1, 1] and takes both signs")
fig, ax = plt.subplots(figsize=(7.5, 4.3))
ax.hist(ratio, bins=60, range=(-1, 1), color="#7a8fb3")
ax.axvline(-7 / 25, color="#e34948", linewidth=2,
           label="rest state with positive frequency: $-7/25$")
ax.set_xlabel("$\\Psi^\\dagger B\\Psi / \\Psi^\\dagger\\Psi$")
ax.set_ylabel("number of random columns")
ax.set_title("The charge density is indefinite")
ax.legend(fontsize=8)
save_figure(fig, "indefinite_charge",
            "Histogram of the normalised charge density $\\Psi^\\dagger B\\Psi / "
            "\\Psi^\\dagger\\Psi$ for 20000 random complex columns $\\Psi$ (fixed "
            "seed); horizontal axis the ratio, from $-1$ to $+1$, vertical axis the "
```

```
"number of columns in each of 60 bins. Because $B$ has eight "
"eigenvalues $+1$ and eight $-1$, the ratio takes both signs equally "
"often. The red line marks the worked example: a flat-space rest "
"state of positive frequency with charge density $-7/25$.")
```

Out [16]:

```
PASS B has signature (8,8); a positive-frequency rest state has charge density -7/25
RESULT charge density of the rest state 3/5 u+ + 4/5 u- = -7/25
fraction of random columns with negative charge density: 0.504
PASS random columns: the ratio lies in [-1, 1] and takes both signs
```

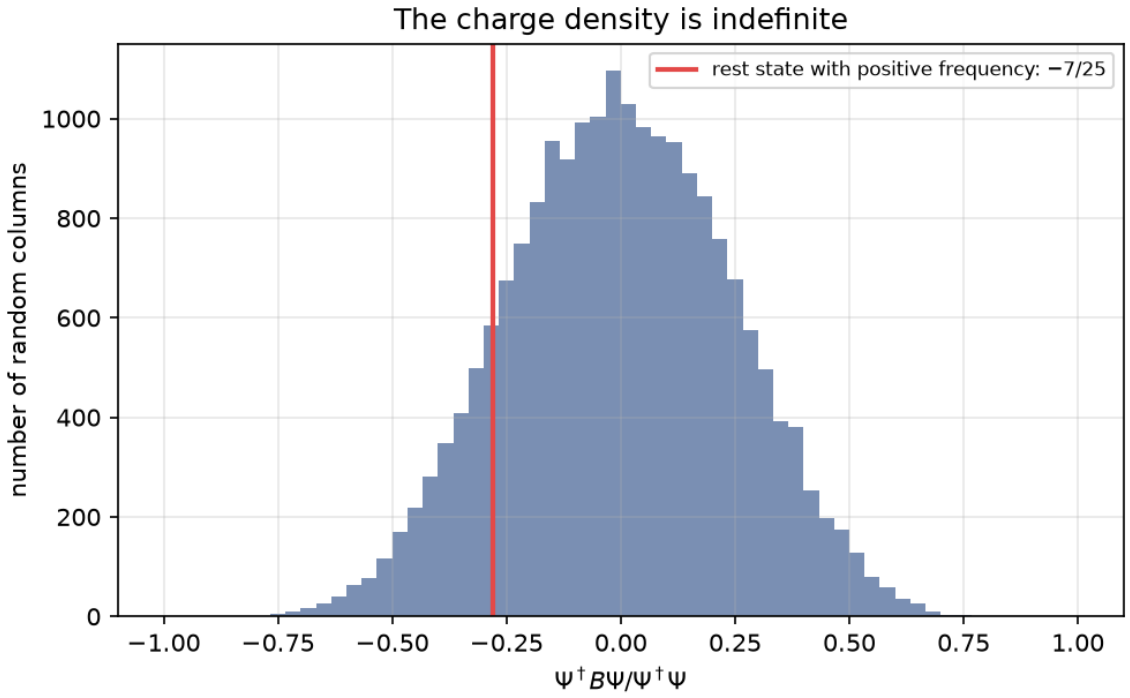

Figure 535: Histogram of the normalised charge density $\Psi^\dagger B\Psi/\Psi^\dagger\Psi$ for 20000 random complex columns Ψ (fixed seed); horizontal axis the ratio, from -1 to $+1$, vertical axis the number of columns in each of 60 bins. Because B has eight eigenvalues $+1$ and eight -1, the ratio takes both signs equally often. The red line marks the worked example: a flat-space rest state of positive frequency with charge density $-7/25$. (Notebook 21b, figure 7.)

Out [16]:

```
Figure 21b.7 saved as Revision/textbook/figures/21b_7_indefinite_charge.png
```

Text cell:

```
## 12. The pair-level charge bookkeeping
```

```
Theorem T1 of the Revision pairing record says: for every configuration $\Psi$ of
the theory with $(m, \lambda)$, the chirality partner $\Gamma\Psi$ is a
configuration of the theory with $(-m, -\lambda)$, and $J^\mu[\Gamma\Psi] =
-J^\mu[\Psi]$ at every point. Hence a pair $\{\Psi, \Gamma\Psi\}$ has total
charge $Q + Q' = 0$ **as classical bilinears**. The next cell checks the
pointwise statement with numbers for five random columns at three times, using
the exact solution's matrix $U(x_4)$: it computes $Q$ of $\Psi$, of $\Gamma\Psi$
and their sum, and draws them. (What this does *not* say: it does not make two
independently quantised universes cancel each other, and it does not create
anything; a single universe with charge $Q$ is an equally valid solution.)
```

In [17]:

```
U_f = sp.lambdify(x4, U_sol, "numpy")
columns = rng.normal(size=(5, 16)) + 1j * rng.normal(size=(5, 16))
check_times = [0.0, 2.5, 5.0]
totals, charges = [], []
for col in columns:
    u_t = [np.array(U_f(tt), dtype=float) @ col for tt in check_times]
    q_psi = [np.conj(v) @ B @ v / 3 for v in u_t]  # Q = f / 3 for alpha = 1
    q_par = [np.conj(Gamma @ v) @ B @ (Gamma @ v) / 3 for v in u_t]
    charges.append((q_psi[0].real, q_par[0].real))
    totals += [abs(a + b_) for a, b_ in zip(q_psi, q_par)]
check(max(totals) < 1e-12 and min(abs(c[0]) for c in charges) > 1e-3,
      "five random columns at three times: Q + Q(partner) = 0, Q itself nonzero")

fig, ax = plt.subplots(figsize=(8.0, 4.3))
idx = np.arange(5)
ax.bar(idx - 0.25, [c[0] for c in charges], width=0.25, label="$Q$ of $\\Psi$")
ax.bar(idx, [c[1] for c in charges], width=0.25, label="$Q$ of $\\Gamma\\Psi$")
ax.plot(idx + 0.25, [c[0] + c[1] for c in charges], "D", color="black",
        label="total of the pair (zero)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(idx, [f"column {k + 1}" for k in idx])
ax.set_ylabel("charge $Q$ at $x_4 = 0$")
ax.set_title("Theorem T1: the partner carries the opposite charge")
ax.legend(fontsize=8)
save_figure(fig, "pair_bookkeeping",
            "Pair-level charge bookkeeping for five random columns $\\chi$ in the "
            "exact solution of the author's metric: the charge $Q$ of $\\Psi$ "
            "(left bar of each group), of its chirality partner $\\Gamma\\Psi$, a "
            "solution with the mass reversed (right bar), and their total (black "
            "diamond, zero); horizontal axis the column, vertical axis the charge at "
            "$x_4 = 0$ per unit of the other six coordinates (pure numbers). The "
            "charges of single universes can have either sign and any size; only "
            "the pair adds to zero.")
```

Out [17]:

```
PASS five random columns at three times: Q + Q(partner) = 0, Q itself nonzero
```

Out [17]:

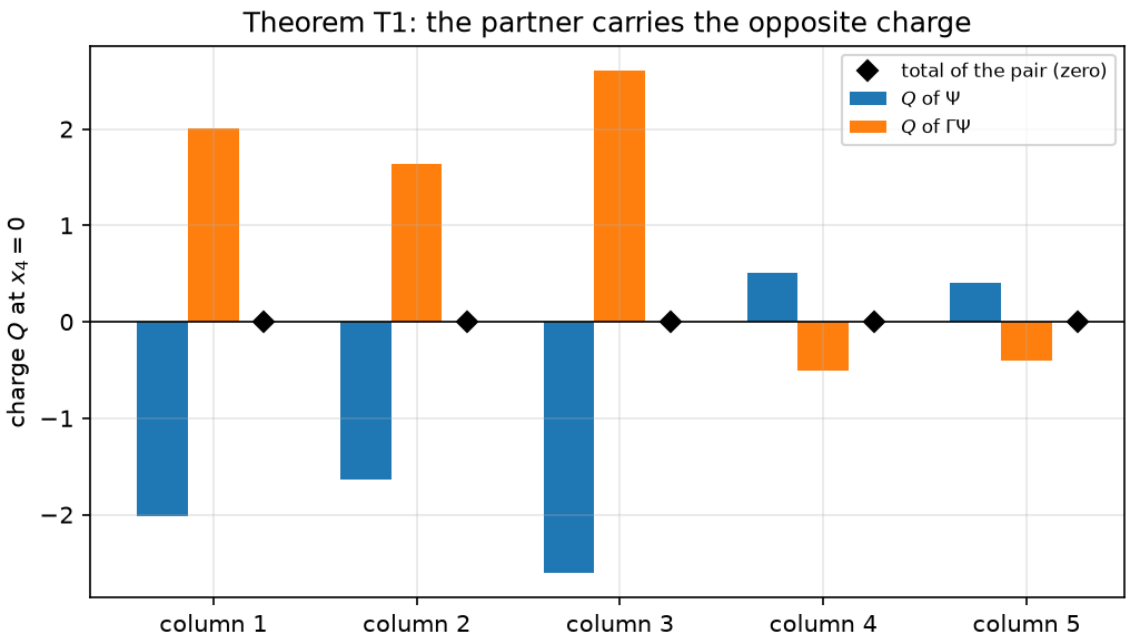

Figure 536: Pair-level charge bookkeeping for five random columns χ in the exact solution of the author's metric: the charge Q of Ψ (left bar of each group), of its chirality partner $\Gamma\Psi$, a solution with the mass reversed (right bar), and their total (black diamond, zero); horizontal axis the column, vertical axis the charge at $x_4 = 0$ per unit of the other six coordinates (pure numbers). The charges of single universes can have either sign and any size; only the pair adds to zero. (Notebook 21b, figure 8.)

```
Figure 21b.8 saved as Revision/textbook/figures/21b_8_pair_bookkeeping.png
```

Text cell:

```
## 13. The figure files

The last cell checks that the eight figure files exist and prints the number of
checks that passed.
```

In [18]:

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)
check(len(names) == 8 and all(
    output_file(f"{FIGURE_FOLDER}/21b_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the eight figure files of notebook 21b exist")
all_checks_passed()
```

Out [18]:

```
PASS the eight figure files of notebook 21b exist
ALL 21 CHECKS PASSED (notebook 21b)
```

Text cell:

```
## 14. What this notebook showed

- PROVED (exact, sympy, for a general history $a_4(x_4)$): the canonical spin
  connection of the author's metric is real and gives $\gamma^\mu\Omega_\mu =
  3H\gamma^{(x0)}$; the U(1) Noether identity reduces to the matrix identity
  $\sum_\mu\partial_\mu(\cos z\,C\gamma^\mu) = \cos z\sum_\mu(C\gamma^\mu
  \Omega_\mu - \Omega_\mu^T(\gamma^\mu)^TC)$, which holds; both sides equal
  $6H\cos z\,C\gamma^{(x0)}$. Without the spin connection it fails.
- PROVED: the $a_4'$ terms cancel because the three deflating extra times
  compensate the three inflating space directions, so that the volume factor
  $\sqrt{|g|} = \cos z$ does not depend on the time; in a control metric with
  inflating extra times they add to $6a_4'$ and are balanced by the growing
  volume.
- COMPUTED (numbers at a point): the Noether current of the phase symmetry is
  $J^\mu = -i\bar\Psi\gamma^\mu\Psi$, real; a constant phase leaves the
  Lagrangian unchanged.
- PROVED for an exact solution of the Revision record in the author's metric:
  the local conservation law holds exactly; the charge of the patch changes only
  by the flux through the brane $z = \pi/2$; for a stationary (single-frequency)
  solution the flux vanishes and the charge is a nonzero constant, for every
  history $a_4$. A finite-difference test converges to the exact law with order 2.
- PROVED: the charge density $\Psi^\dagger B\Psi$ is indefinite (signature
  (8,8)); a positive-frequency rest state can carry the charge density $-7/25$.
- PROVED (theorem T1, reproduced): the chirality partner $\Gamma\Psi$ solves the
  equation with the mass reversed and carries exactly the opposite current, so
```

```
  the total charge of a pair is zero as classical bilinears.
- Consequence for Sakharov's first condition: no process of this theory creates
  or destroys U(1) charge at any point of the patch $0 < z < \pi/2$, off the
  brane (the local law). The charge of a universe
  changes only by what flows through its boundary, and the first solution of
  this notebook shows that charge does flow through the brane $z = \pi/2$. Only
  under a no-flux condition at the brane, which is ASSUMED and not derived
  (OPEN), is the charge of a universe constant and fixed by the initial data.
  This is a statement about the classical field equations; it is not a creation
  mechanism for anything.
```

21.23 Line-by-line walk-through of Notebook 21b

The notebook has 18 code cells, In [1] to In [18]. Each is explained below, line by line or in small groups of lines.

In [1], the set-up cell. It is the set-up cell of Notebook 21a, explained line by line in Section 21.15 under In [1], word for word except for two things: the comment lines at the top hold the run instructions of Notebook 21b (Section 21.21), and the line

```
NOTEBOOK_ID = "21b"  # this notebook: chapter 21, example b
```

names this notebook, so that its figures are called `21b_<k>_<name>.png`. The cell prints its one line, Set-up of notebook 21b complete.

In [2]: the gammas, C, Γ, B and the records.

```
import numpy as np  # numbers, arrays, matrices
import sympy as sp  # exact algebra and calculus
```

numpy for numbers and arrays, sympy for exact algebra and calculus (Section 21.15 explains both).

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = fixture["eta"]  # +1 space-like, -1 time-like, in the order x0..x7
gamma = [np.array(mat, dtype=np.int64) for mat in fixture["gamma"]]  # index 0..7
I16 = np.eye(16, dtype=np.int64)
```

The Revision record of the gammas is read as in Notebook 21a, but here the gammas are kept in a **list**, numbered from 0 in the order of the record, $x_0, \ldots, x_7$: `gamma[k]` is $\gamma^{(xk)}$, so `gamma[0]` is $\gamma^{(x0)}$ of the hidden direction and `gamma[4]` is $\gamma^{(x4)}$ of the time. `ETA` is the list of the eight signs in the same order.

```
real_ok = all(g.shape == (16, 16) and set(np.unique(g)) <= {-1, 0, 1} for g in gamma)
clifford_ok = all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                                 2 * (ETA[a] if a == b else 0) * I16)
                  for a in range(8) for b in range(8))
check(len(gamma) == 8 and real_ok and clifford_ok,
      "eight real 16 x 16 gamma matrices with the Clifford relation, signature (4,4)")
```

The same tests as In [3] of Notebook 21a, written with the numbers `a`, `b` from 0 to 7 (`range(8)`): eight real 16×16 matrices with entries -1, 0, $+1$ that obey the Clifford relation for all 64 pairs.

```
G = [sp.Matrix(g.tolist()) for g in gamma]  # exact sympy copies, G[k] = gamma^(xk)
C_s = G[0] * G[1] * G[2] * G[3]  # C = gamma^(x0) gamma^(x1) gamma^(x2) gamma^(x3)
C = np.array(C_s.tolist(), dtype=np.int64)
```

Exact sympy copies of the gammas; C is built exactly (`C_s`) and also kept as a whole-number numpy array (`C`).

```
Gamma = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3] @ gamma[4] @ gamma[5] \
    @ gamma[6] @ gamma[7]  # the chirality (index 0 is x0)
B = -1j * (C @ gamma[4])  # B = -i C gamma^(x4); index 4 is x4
K0 = -1j * (C @ gamma[0])  # the matrix of the x0 current: -i C gamma^(x0)
S_gen = [[(G[a] * G[b] - G[b] * G[a]) / 4 for b in range(8)] for a in range(8)]
```

The chirality is the product of all eight in the order $x_0, x_1, \ldots, x_7$; the backslash at the end of the first line continues the statement on the next. $B = -iC\gamma^{(x4)}$ and $K_0 = -iC\gamma^{(x0)}$ are the matrices of the charge density and of the x_0 current (Section 21.18). `S_gen[a][b]` is the exact generator S^{ab} for all 64 ordered pairs (zero when $a = b$).

```
LEAD_FILE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
LEAD = {c["name"]: c["verdict"] for c in json.loads(
    repository_file(LEAD_FILE).read_text(encoding="utf-8"))["checks"]}
THEORY = {f["key"]: f for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
PY_THEORY = {c["name"]: c["verdict"].upper() for c in json.loads(repository_file(
    "Revision/theory/reports/python-field-theory.json").read_text(
    encoding="utf-8"))["checks"]}
```

Four Revision records are read into dictionaries: the lead report (check name to verdict), the formulas of the field-theory record (key to the whole formula entry, so that `THEORY[key]["wl"]` is its text), and the verdicts of the sympy field-theory report, in capitals.

```
WL_PAIR = {c["name"]: c["verdict"] for c in json.loads(repository_file(
    "Revision/pairing/reports/wolfram-pairing.json").read_text(
    encoding="utf-8"))["checks"]}
PY_PAIR = {c["name"]: c["verdict"] for c in json.loads(repository_file(
    "Revision/pairing/reports/python-pairing.json").read_text(
    encoding="utf-8"))["checks"]}
```

The two pairing reports of the Revision record: the WolframScript verifier and the independent sympy checker. Their verdicts are already written as PASS.

```
clauses = THEORY["current"]["wl"].split("; ")  # the statements of the formula
say("record formula current: " + clauses[0] + "; " + clauses[2])
check(clauses[0] == "J^mu = -i Psibar gamma^mu Psi"
      and clauses[2] == "J^x4 = Psi^dagger B Psi",
      "the record defines J^mu = -i Psibar gamma^mu Psi and J^x4 = Psi^dagger B Psi",
      record="Revision/theory/field-theory.json, formula current")
```

The record's formula `current` is a list of statements separated by semicolons; `split("; ")` cuts it into them. The first defines the current and the third the charge density; the check demands that they are exactly the definitions this notebook uses. Out [2] prints the two statements and two PASS lines.

In [3]: the canonical spin connection for every history.

```
x = sp.symbols("x0:8", real=True)  # the coordinates x0, ..., x7
H = sp.symbols("H", positive=True)  # the author's constant H > 0
a4 = sp.Function("a4", real=True)(x[4])  # an unspecified history a4(x4)
z = 6 * H * x[0]  # z = 6 H x0
eta = sp.diag(*ETA)
```

Eight real symbols for the coordinates (`x[k]` is x_k: `x[0]` is x_0, `x[4]` is x_4), a positive symbol H, and an **unspecified function** $a_4(x_4)$: sympy knows nothing about it except that it depends on x_4, so every result below holds for every history, the deflating one included. `sp.diag(*ETA)` is the diagonal matrix η (the star passes the eight list entries as eight separate arguments).

```
def geometry(extra_sign):
    """Scale factors, volume factor and the eight matrices Omega_mu of the metric
    whose extra-time factors are exp(extra_sign a4) sin^(1/6) z (extra_sign = -1:
    the author's deflating extra times)."""
    f = ([sp.cot(z)] + [sp.exp(a4) * sp.sin(z) ** sp.Rational(1, 6)] * 3
         + [sp.Integer(1)]
         + [sp.exp(extra_sign * a4) * sp.sin(z) ** sp.Rational(1, 6)] * 3)
    g = sp.diag(*[ETA[a] * f[a] ** 2 for a in range(8)])  # the diagonal metric
    g_inv = g.inv()
```

The function builds the geometry of a metric whose three extra-time scale factors are $e^{\pm a_4}\sin^{1/6} z$: with `extra_sign = -1` the author's metric, with `+1` the control metric of Section 21.17. A list times 3 repeats the list three times, and `+` joins lists: `f` is the list of the eight scale factors. The metric is diagonal with $g_{aa} = \eta_{aa} f_a^2$, and `g.inv()` is its inverse.

```
    chris = [[[sp.simplify(sum(g_inv[l, m] * (sp.diff(g[m, i], x[j])
                                              + sp.diff(g[m, j], x[i])
                                              - sp.diff(g[i, j], x[m]))
                               for m in range(8)) / 2)
               for j in range(8)] for i in range(8)] for l in range(8)]
```

The Christoffel symbols $\Gamma^l{}_{ij} = \frac{1}{2}\sum_m g^{lm}(\partial_j g_{mi} + \partial_i g_{mj} - \partial_m g_{ij})$ (Chapter 3), stored so that `chris[l][i][j]` is $\Gamma^l{}_{ij}$: three nested list comprehensions, the innermost index written last. `sp.diff(expr, x[j])` is the exact partial derivative and `sp.simplify` tidies each result.

```
    e_inv = [1 / v for v in f]  # e_a^mu = 1 / f_a on the diagonal
    Omega = []
    for mu in range(8):
        om = sp.zeros(8, 8)  # omega_mu^a_b
        for a in range(8):
            for b_ in range(8):
```

```
            om[a, b_] = sp.simplify(f[a] * (sp.diff(e_inv[b_] if a == b_ else 0,
                                                   x[mu])
                                            + chris[a][mu][b_] * e_inv[b_]))
```

The inverse vielbein is diagonal with entries $1/f_a$. For each direction μ the 8×8 table ${\omega_\mu}^a{}_b$ of the canonical connection is computed from the **vielbein postulate** ${\omega_\mu}^a{}_b = e^a{}_\nu(\partial_\mu {e_b}^\nu + {\Gamma^\nu}_{\mu\lambda} {e_b}^\lambda)$; with the diagonal vielbein $e^a{}_\nu = f_a \delta^a_\nu$ and ${e_b}^\nu = \delta^\nu_b / f_b$ the sums collapse to ${\omega_\mu}^a{}_b = f_a(\partial_\mu(\delta_{ab}/f_b) + {\Gamma^a}_{\mu b}/f_b)$, which is the line in the loop (`e_inv[b_] if a == b_ else 0` is δ_{ab}/f_b; the letter `b_` avoids a clash with other names).

```
        om_low = eta * om  # omega_mu ab = eta_ac omega_mu^c_b
        Om = sp.zeros(16, 16)
        for a in range(8):
            for b_ in range(8):
                if om_low[a, b_] != 0:
                    Om += om_low[a, b_] * S_gen[a][b_] / 2
        Omega.append(Om.applyfunc(sp.simplify))
```

The first index is lowered with η (a matrix product), and the spinor connection $\Omega_\mu = \frac{1}{2}\sum_{a,b} \omega_{\mu ab} S^{ab}$ is summed (only the nonzero coefficients). `applyfunc(sp.simplify)` simplifies every entry, and the matrix is added to the list `Omega`.

```
    volume = sp.simplify(sp.prod(f))  # sqrt|g| = product of the scale factors
    return f, volume, Omega


f_dfl, vol_dfl, Omega = geometry(-1)  # the author's metric (deflating extra times)
say(f"sqrt|g| of the author's metric = {vol_dfl}")
```

`sp.prod(f)` multiplies the eight scale factors, which for a diagonal metric is $\sqrt{|\det g|}$. The function returns the three results, and it is called for the author's metric. Out [3] prints `cos(6*H*x0)`: the volume factor is $\cos z$, with no a_4 in it (Section 21.7). This cell takes about 7 seconds.

In [4]: the connection agrees with the record.

```
a4p = sp.Derivative(a4, x[4])  # a4' = d a4 / d x4
real_ok = all(not entry.has(sp.I) for Om in Omega for entry in Om)
nonzero = any(Om != sp.zeros(16, 16) for Om in Omega)
check(real_ok and nonzero and LEAD["spinor_connection_real"] == "PASS",
      "every entry of Omega_mu is real and not all vanish",
      record=f"{LEAD_FILE}, check spinor_connection_real")
```

`a4p` is the symbol of the derivative a_4'. `entry.has(sp.I)` asks whether an entry contains the imaginary unit; none does, so every Ω_μ is real. `any` asks that at least one Ω_μ is not the zero matrix. This reproduces the lead check that makes $\gamma^\mu D_\mu$ a real operator (Section 21.8, step 1).

```
s16 = sp.sin(z) ** sp.Rational(1, 6)
expected = []
```

```
for mu in range(8):
    if mu in (1, 2, 3):  # x1, x2, x3: inflating ordinary space
        expected.append(sp.exp(a4) * s16 / 2 * (a4p * G[mu] * G[4] + H * G[mu] * G[0]))
    elif mu in (5, 6, 7):  # x5, x6, x7: the deflating extra times
        expected.append(-sp.exp(-a4) * s16 / 2 * (a4p * G[4] * G[mu]
                                                  + H * G[mu] * G[0]))
    else:  # x4 and x0
        expected.append(sp.zeros(16, 16))
```

The record's formula `Omega_components` (Section 21.17), written out for each direction: `if`, `elif` ("else if") and `else` choose the right case.

```
record_text = THEORY["Omega_components"]["wl"]  # the record's statement
pieces_of_record = [  # the formula above, piece by piece, in the record's notation
    "Omega_xi = (1/2) E^a4[x4] Sin[6 H x0]^(1/6) (a4",
    "g[xi].g[x4] + H g[xi].g[x0]) (i = 1, 2, 3)",
    "Omega_xt = -(1/2) E^-a4[x4] Sin[6 H x0]^(1/6) (a4",
    "g[x4].g[xt] + H g[xt].g[x0]) (t = 5, 6, 7)",
    "Omega_x0 = Omega_x4 = 0"]
```

The record's text of the formula and five pieces of it in the record's notation. The pieces stop just before the derivative of a_4, which the record writes with a straight single quote; the notebook avoids writing that character.

```
check(all((Omega[mu] - expected[mu]).applyfunc(sp.simplify) == sp.zeros(16, 16)
          for mu in range(8))
      and all(piece in record_text for piece in pieces_of_record),
      "Omega_mu equals the record formula Omega_components",
      record="Revision/theory/field-theory.json, formula Omega_components")
```

Every computed Ω_μ minus the expected one simplifies to zero, and every piece occurs in the record's text (`in` asks whether one string occurs inside another).

```
gamma_up = [G[mu] / f_dfl[mu] for mu in range(8)]  # gamma^mu = gamma^a / f_a
contraction = sum((gamma_up[mu] * Omega[mu] for mu in range(8)), sp.zeros(16, 16))
check((contraction - 3 * H * G[0]).applyfunc(sp.simplify) == sp.zeros(16, 16)
      and THEORY["gammaOmega_total"]["wl"] == '3*H*gamma["x0"]',
      "gamma^mu Omega_mu = 3 H gamma^(x0) for every history a4",
      record="Revision/theory/field-theory.json, formula gammaOmega_total")
```

The curved gammas $\gamma^\mu = \gamma^{(\mu)}/f_\mu$; `sum(..., sp.zeros(16, 16))` adds the eight products starting from the zero matrix. The contraction equals $3H\gamma^{(x0)}$, and the record's formula `gammaOmega_total` says the same. Out [4] shows the three PASS lines.

In [5]: the U(1) matrix identity.

```
cos_z = vol_dfl  # sqrt|g| = cos z
lhs = sum((sp.diff(cos_z / f_dfl[mu], x[mu]) * C_s * G[mu] for mu in range(8)),
          sp.zeros(16, 16))
```

```
pieces = [cos_z * (C_s * gamma_up[mu] * Omega[mu]
                   - Omega[mu].T * gamma_up[mu].T * C_s) for mu in range(8)]
rhs = sum(pieces, sp.zeros(16, 16))
```

The two sides of the identity of Section 21.17: the left side $\sum_\mu \partial_\mu(\cos z\, C\gamma^\mu) = \sum_\mu \partial_\mu(\cos z/f_\mu)\, C\gamma^{(\mu)}$, and the right side as the list of its eight contributions (kept for In [6]) and their sum.

```
check((lhs - rhs).applyfunc(sp.simplify) == sp.zeros(16, 16)
      and LEAD["u1_noether_matrix_identity"] == "PASS",
      "the U(1) Noether matrix identity holds exactly for a general history a4",
      record=f"{LEAD_FILE}, check u1_noether_matrix_identity")
check((lhs - 6 * H * sp.cos(z) * C_s * G[0]).applyfunc(sp.simplify)
      == sp.zeros(16, 16), "the left side is exactly 6 H cos z C gamma^(x0)")
check(lhs.applyfunc(sp.simplify) != sp.zeros(16, 16),
      "negative control: with Omega = 0 the right side is 0 but the left side is "
      "not")
```

The difference of the two sides simplifies to the zero matrix for the unspecified a_4: the identity holds for every history (reproducing the lead check). The left side equals $6H\cos z\, C\gamma^{(x0)}$, as computed by hand in Section 21.17. The negative control: without the spin connection the right side would be the zero matrix, and the left side is not zero, so the identity, and with it the conservation of the current, would fail.

In [6]: why the a_4' terms cancel.

```
CG4, CG0 = C_s * G[4], C_s * G[0]


def coefficients(matrix, volume):
    """The coefficients of C gamma^(x4) and C gamma^(x0) in matrix, divided by the
    volume factor (trace formula; checked to leave no remainder)."""
    c4 = sp.simplify((matrix * CG4.T).trace() / 16 / volume)
    c0 = sp.simplify((matrix * CG0.T).trace() / 16 / volume)
    remainder = (matrix - volume * (c4 * CG4 + c0 * CG0)).applyfunc(sp.simplify)
    return c4, c0, remainder == sp.zeros(16, 16)
```

The matrices $C\gamma^{(x4)}$ and $C\gamma^{(x0)}$ are signed permutation matrices, so each times its own transpose is 1, with trace 16, while the trace of one times the transpose of the other is 0 (they are orthogonal). So if a matrix is $P = c_4 C\gamma^{(x4)} + c_0 C\gamma^{(x0)}$, then $c_4 = \mathrm{tr}(P(C\gamma^{(x4)})^T)/16$, and likewise c_0. The function divides by the volume factor and checks that nothing else is left over (`remainder` is the zero matrix).

```
dfl = [coefficients(p, cos_z) for p in pieces]  # the author's metric
f_ctl, vol_ctl, Omega_ctl = geometry(+1)  # control: inflating extra times
gamma_up_ctl = [G[mu] / f_ctl[mu] for mu in range(8)]
pieces_ctl = [vol_ctl * (C_s * gamma_up_ctl[mu] * Omega_ctl[mu]
                         - Omega_ctl[mu].T * gamma_up_ctl[mu].T * C_s)
              for mu in range(8)]
lhs_ctl = sum((sp.diff(vol_ctl / f_ctl[mu], x[mu]) * C_s * G[mu] for mu in range(8)),
              sp.zeros(16, 16))
```

The coefficients of the eight contributions in the author's metric; then the whole computation again for the control metric (`geometry(+1)`): its curved gammas, its eight contributions and its left side. This takes about 7 seconds.

```
ctl = [coefficients(p, vol_ctl) for p in pieces_ctl]
lhs_c4, _, _ = coefficients(lhs_ctl, vol_ctl)  # control: the volume-growth term
lhs_c4_dfl, _, _ = coefficients(lhs, cos_z)  # author's metric: zero
say(f"control metric: sqrt|g| = {vol_ctl}")
```

The coefficients of the control contributions, and the $C\gamma^{(x4)}$ coefficient of each left side (the underscores discard the other two returned values). Out [6] first prints the control volume factor `exp(6*a4(x4))*cos(6*H*x0)`, which grows with a_4.

```
for mu in range(8):  # the coefficients in units of a4' (sympy divides exactly)
    say(f"{COORDS[mu]}: author's metric {str(sp.simplify(dfl[mu][0] / a4p)):>2} a4'"
        f" | control metric {str(sp.simplify(ctl[mu][0] / a4p)):>2} a4'")
```

For each direction the $C\gamma^{(x4)}$ coefficient divided by a_4' is printed for both metrics (`:>2` right-aligns the text in two characters). Out [6] shows $0, 1, 1, 1, 0, -1, -1, -1$ for the author's metric and $0, 1, 1, 1, 0, 1, 1, 1$ for the control (in the order $x_0, \ldots, x_7$).

```
expected_dfl = [0] + [a4p] * 3 + [0] + [-a4p] * 3  # +a4' space, -a4' extra times
expected_ctl = [0] + [a4p] * 3 + [0] + [a4p] * 3  # all six +a4'
check(all(c[2] for c in dfl + ctl)
      and all(sp.simplify(c[0] - e) == 0 for c, e in zip(dfl, expected_dfl))
      and all(sp.simplify(c[0] - e) == 0 for c, e in zip(ctl, expected_ctl))
      and sp.simplify(lhs_c4 - 6 * a4p) == 0 and sp.simplify(lhs_c4_dfl) == 0
      and (lhs_ctl - sum(pieces_ctl, sp.zeros(16, 16))).applyfunc(sp.simplify)
      == sp.zeros(16, 16),
      "a4' terms: +a4' (space) and -a4' (deflating extra times) cancel; the "
      "control metric balances 6 a4' by its growing volume")
```

One check for everything of the paragraph "Why the time-derivative terms cancel" of Section 21.17: no remainders; the expected coefficients in both metrics; the left side of the control has $6a_4'$ and that of the author's metric 0; and the identity holds in the control metric too.

In [7]: the volume balance as bars.

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.3), sharey=True)
labels = COORDS + ["left side"]
for ax, data, lhs_value, title in (
        (axes[0], dfl, lhs_c4_dfl, "author's metric: extra times deflate"),
        (axes[1], ctl, lhs_c4, "control: extra times inflate")):
    heights = [float(sp.simplify(c[0] / a4p)) for c in data + [(lhs_value,)]]
```

Two panels, one per metric. The loop runs over two groups of four values. `data + [(lhs_value,)]` appends a one-element tuple to the eight coefficient triples, so that `c[0]` is also the left-side coefficient; each is divided by a_4' and turned into a decimal number with `float`.

```
    colours = ["#e34948" if h > 0 else ("#2a78d6" if h < 0 else "#999999")
               for h in heights[:8]] + ["#555555"]
    ax.bar(range(9), heights, color=colours)
    ax.axhline(0.0, color="black", linewidth=0.8)
    ax.set_xticks(range(9), labels, rotation=30)
    ax.set_title(title + f"; sum of the eight = {round(sum(heights[:8]))}")
    ax.set_xlabel("direction $\\mu$ of the term")
axes[0].set_ylabel("coefficient of $C\\gamma^{(x4)}$ in units of $a_4'$")
```

Red bars for positive, blue for negative, grey for zero coefficients, and a dark bar for the left side. The tick labels are turned by 30 degrees; the title states the sum of the eight contributions (`heights[:8]`, rounded to a whole number).

```
save_figure(fig, "volume_balance",
            "Why the time-derivative terms of the U(1) identity cancel. Each bar is "
            "the coefficient of $C\\gamma^{(x4)}$, in units of $a_4'$ and divided by "
            "the volume factor, contributed by one direction $\\mu$ to the right "
            "side $\\cos z\\,\\sum_\\mu(C\\gamma^\\mu\\Omega_\\mu - \\Omega_\\mu^T"
            "(\\gamma^\\mu)^TC)$; the last bar is the coefficient of the left "
            "side. Left: the author's metric, in which ordinary space contributes "
            "$+1$ per direction and the deflating extra times $-1$ each; the sum "
            "and the left side are $0$. Right: a control metric with inflating "
            "extra times; the six contributions add to $6$, matched by the growing "
            "volume factor $e^{6a_4}\\cos z$ on the left side.")
```

Figure 21b.1. **What the figure shows.** Left: three red bars of height $+1$ (space) and three blue bars of height -1 (extra times), sum 0, and an empty left-side bar: the deflation cancels the inflation. Right: six red bars, sum 6, and a dark left-side bar of height 6: in a metric whose volume grows, the growth itself supplies the balancing term.

In [8]: the matrices of the identity as heat maps.

```
from matplotlib.colors import TwoSlopeNorm

sample = {a4p: 0.7, a4: 0.3, H: sp.Rational(1, 6), x[0]: 0.5}  # any point
```

`TwoSlopeNorm` maps numbers to colours with the middle colour at a chosen centre value. `sample` is a dictionary of values for a sample point: $a_4' = 0.7$, $a_4 = 0.3$, $H = 1/6$, $x_0 = 0.5$.

```
def sampled(matrix):
    """matrix / (H cos z) evaluated at the sample point, as numbers."""
    return np.array(sp.N((matrix / (H * sp.cos(z))).subs(a4p, 0.7).subs(sample))
                    .tolist(), dtype=float)
```

The matrix is divided by $H\cos z$; `.subs` replaces symbols by values. The derivative a_4' is replaced first, on its own, because replacing a_4 by a number inside the derivative would destroy it. `sp.N` turns the exact result into decimal numbers, and the result is a numpy array.

```
lhs_num, rhs_num = sampled(lhs), sampled(rhs)
say(f"LHS = RHS at the sample point to 1e-12: "
    f"{np.abs(lhs_num - rhs_num).max() < 1e-12}")
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.0))
norm = TwoSlopeNorm(vmin=-6, vcenter=0, vmax=6)  # blue negative, red positive
```

Both sides at the sample point; the printed line (True) confirms that they agree to 10^{-12}. Three panels; the colour scale runs from -6 (blue) through 0 to $+6$ (red).

```
for ax, matrix, title in ((axes[0], lhs_num, "left side / $(H\\cos z)$"),
                          (axes[1], rhs_num, "right side / $(H\\cos z)$"),
                          (axes[2], 0 * rhs_num, "right side, $\\Omega = 0$")):
    image = ax.imshow(matrix, cmap="RdBu_r", norm=norm)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("column")
    ax.grid(False)
axes[0].set_ylabel("row")
fig.colorbar(image, ax=axes, shrink=0.8, label="matrix entry")
```

The three matrices: the left side, the right side, and the right side without the spin connection (the zero matrix, `0 * rhs_num`). `"RdBu_r"` is matplotlib's red-blue colour map reversed, so that positive numbers are red.

```
save_figure(fig, "noether_matrices",
            "The $16 \\times 16$ matrices of the U(1) identity in the author's "
            "metric, divided by $H\\cos z$: left side $\\sum_\\mu\\partial_\\mu"
            "(\\cos z\\,C\\gamma^\\mu)$ (left panel), right side $\\cos z\\sum_\\mu"
            "(C\\gamma^\\mu\\Omega_\\mu - \\Omega_\\mu^T(\\gamma^\\mu)^TC)$ (middle) "
            "and the right side of the negative control with the spin connection "
            "removed (right); horizontal axis the column, vertical axis the row, "
            "colour the entry (red positive, blue negative). The first two are the "
            "same matrix $6C\\gamma^{(x0)}$ for every history $a_4$; without the "
            "spin connection the right side would be zero and the current would "
            "not be conserved.")
```

Figure 21b.2. **What the figure shows.** The left and middle panels are identical: sixteen squares of value ± 6 arranged like the signed permutation matrix $C\gamma^{(x0)}$. The right panel is empty: without the connection nothing balances the left side.

In [9]: the Noether current from a local phase change.

```
point = {x[0]: 0.7, H: sp.Rational(1, 6)}  # z = 0.7 (H = 1/6, so z = x0)
A4_value, A4_slope = 0.4, 0.25  # a4 and a4' at the point (any values)
```

A point of the author's metric: $x_0 = 0.7$ with $H = 1/6$, so $z = 0.7$; and arbitrary values $a_4 = 0.4$, $a_4' = 0.25$ of the history there.

```
def at_point(expression):
    """A sympy expression evaluated at the point, as a numpy array of complex."""
    e = expression.subs(a4p, A4_slope).subs(a4, A4_value).subs(point)
    return np.array(sp.N(e).tolist(), dtype=complex) if hasattr(e, "tolist") \
        else complex(sp.N(e))
```

The values are substituted (the derivative first, as in In [8]). If the result is a matrix (it has the method `tolist`; `hasattr` asks this), it becomes a complex numpy array, otherwise a complex number; the backslash continues the line.

```
Om_num = [at_point(Om) for Om in Omega]  # Omega_mu at the point
f_num = [at_point(v).real for v in f_dfl]  # the scale factors at the point
g_up = [gamma[mu] / f_num[mu] for mu in range(8)]  # gamma^mu at the point
cos_num = np.cos(0.7)
m_val, lam = 0.7, 0.3
```

The spin connection, the scale factors and the curved gammas at the point, the volume factor $\cos 0.7$, and the mass and coupling $m = 0.7$, $\lambda = 0.3$.

```
rng = np.random.default_rng(2102)  # fixed seed: the same numbers in every run
psi = rng.normal(size=16) + 1j * rng.normal(size=16)  # Psi at the point
dpsi = rng.normal(size=(8, 16)) + 1j * rng.normal(size=(8, 16))  # d_mu Psi
alpha_mu = rng.normal(size=8)  # the slopes d_mu alpha of the phase
# drawn with x0 last, as before the move to x0..x7; the roll puts x0 first
dpsi, alpha_mu = np.roll(dpsi, 1, axis=0), np.roll(alpha_mu, 1)
```

A random-number generator with the fixed seed 2102 provides a complex field value, eight complex first derivatives (an 8×16 array, one row per direction) and eight slopes $\alpha_\mu = \partial_\mu \alpha$ of the phase. They are drawn in the order in which this notebook drew them before it moved to the author's coordinates, with the hidden direction last; `np.roll(..., 1, axis=0)` moves the last row to the front, so that row μ belongs to x_μ and every printed number stays the same. A Lagrangian density at one point depends only on these numbers.

```
def lagrangian(p, dp):
    """cos z [ (1/2)(Psibar gamma^mu D_mu Psi - (D_mu Psibar) gamma^mu Psi)
    - m S - (lambda/2) S^2 ] at the point, for the value p and derivatives dp."""
    bar = np.conj(p) @ C  # Psibar = Psi^dagger C
    kinetic = 0
    for mu in range(8):
        D = dp[mu] + Om_num[mu] @ p  # D_mu Psi
        Dbar = np.conj(dp[mu]) @ C - bar @ Om_num[mu]  # D_mu Psibar
        kinetic += (bar @ g_up[mu] @ D - Dbar @ g_up[mu] @ p) / 2
    S = bar @ p
    return cos_num * (kinetic - m_val * S - lam / 2 * S ** 2)
```

The Lagrangian density of the record, term by term: the Dirac adjoint $\bar\Psi = \Psi^\dagger C$ (a row); for each direction $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$ and $D_\mu\bar\Psi = (\partial_\mu\Psi)^\dagger C - \bar\Psi\Omega_\mu$; the kinetic term summed over the eight directions (`+=` adds to the running total); the scalar S; and $\cos z\,[\,\text{kinetic} - mS - \frac{\lambda}{2}S^2\,]$.

```
phase = np.exp(1j * 0.9)  # e^(i alpha) at the point, alpha = 0.9
L0 = lagrangian(psi, dpsi)
L_const = lagrangian(phase * psi, phase * dpsi)  # constant phase: alpha_mu = 0
L_local = lagrangian(phase * psi, phase * (dpsi + 1j * alpha_mu[:, None] * psi))
J = np.array([-1j * (np.conj(psi) @ C @ g_up[mu] @ psi) for mu in range(8)])
```

Three evaluations: the field as it is; after a constant phase $e^{0.9i}$ (value and derivatives multiplied by it); and after a local phase, whose derivatives are $e^{i\alpha}(\partial_\mu\Psi + i\alpha_\mu\Psi)$ (Section 21.16). `alpha_mu[:, None]` turns the eight slopes into a column, so that the product with the 16 components gives an 8×16 array, one row per direction. `J` holds the eight components of the current at the point.

```
say(f"L = {L0.real:+.6f} (imaginary part below 1e-12: {abs(L0.imag) < 1e-12}); "
    f"after a constant phase {L_const.real:+.6f}")
say(f"L' - L = {(L_local - L0).real:+.6f};  -cos z alpha_mu J^mu = "
    f"{(-cos_num * alpha_mu @ J).real:+.6f}")
```

Out [9] prints $\mathcal{L} = -16.780795$ with an imaginary part below 10^{-12} (the Lagrangian is real), the same value after a constant phase, and $\mathcal{L}' - \mathcal{L} = +2.348835 = -\cos z\,\alpha_\mu J^\mu$ (`alpha_mu @ J` is the sum $\sum_\mu \alpha_\mu J^\mu$).

```
check(abs(L0.imag) < 1e-12 and abs(L_const - L0) < 1e-12
      and abs((L_local - L0) - (-cos_num * alpha_mu @ J)) < 1e-12
      and np.allclose(J.imag, 0),
      "L is real and phase invariant; a local phase changes it by -cos z "
      "(d_mu alpha) J^mu with J^mu = -i Psibar gamma^mu Psi real")
```

The four statements to 10^{-12}, including that all eight current components are real.

In [10]: the exact solution and local conservation.

```
x4, zz = sp.symbols("x4 z", real=True)  # the time and the angle z = x0
Hn, al, m_sol = sp.Rational(1, 6), 1, 2
b_sol = 3 * Hn * (2 * al + 1)  # 3/2
w = sp.sqrt(m_sol ** 2 - b_sol ** 2)  # sqrt(7)/2
M_sol = -m_sol * G[4] + b_sol * G[4] * G[0]
U_sol = sp.cos(w * x4) * sp.eye(16) + sp.sin(w * x4) / w * M_sol  # a real matrix
```

New symbols for the time and the angle (the name `zz` keeps `z` of In [3] intact), the parameters $H = 1/6$, $\alpha = 1$, $m = 2$, and $b = 3/2$, $w = \sqrt{7}/2$, M and U of Section 21.11, exactly.

```
chi_values = [(0.62, -0.31), (0.15, 0.88), (-0.47, 0.26), (0.93, -0.05),
              (-0.21, -0.64), (0.38, 0.12), (0.07, -0.93), (-0.85, 0.44),
              (0.51, 0.69), (-0.12, -0.27), (0.29, 0.58), (-0.66, 0.03),
              (0.44, -0.72), (0.81, 0.35), (-0.39, -0.18), (0.24, 0.97)]
chi = sp.Matrix([sp.Rational(round(100 * re), 100) + sp.I * sp.Rational(
    round(100 * im), 100) for re, im in chi_values])  # exact rational entries
Psi = sp.sin(zz) ** al * U_sol * chi
```

Sixteen pairs (real part, imaginary part) with two decimals; `round(100 * re)` makes the whole number of hundredths, and `sp.Rational(..., 100)` the exact fraction, so χ has exact entries such as $31/50 - 31i/100$. `Psi` is the exact solution $\sin z\, U\chi$.

```
tan_z = sp.sin(zz) / sp.cos(zz)
E = (G[4] * Psi.diff(x4) + 6 * Hn * tan_z * G[0] * Psi.diff(zz)
     + 3 * Hn * G[0] * Psi - m_sol * Psi)  # the field equation, U = 0
SOLUTION_I = (  # item (i) of the record's formula exact_solutions, word for word
    "(i) U = 0: Psi = Sin[z]^al (Cosh[k x4] + Sinh[k x4]/k M) chi, M = -m g[x4]"
    " + 3 H (2 al + 1) g[x4].g[x0], k^2 = 9 H^2 (2 al + 1)^2 - m^2 (any a4)")
```

The left side of the field equation $E_m[\Psi]$ of Section 21.11 (step 4) with $m = 2$, and the record's text of the solution, word for word, as in Notebook 21a.

```
check(E.expand() == sp.zeros(16, 1)
      and THEORY["exact_solutions"]["wl"].startswith(SOLUTION_I + "; ")
      and PY_THEORY["exact_solution_family_x4_x0"] == "PASS",
      "the record's exact solution solves the field equation (m = 2)",
      record="Revision/theory/reports/python-field-theory.json, check "
      "exact_solution_family_x4_x0")
```

The solution solves the equation exactly; the record states it as used here; the record's own check passed.

```
B_s = -sp.I * C_s * G[4]
K0_s = -sp.I * C_s * G[0]
density = sp.cos(zz) * (Psi.H * B_s * Psi)[0, 0]  # cos z J^(x4)
flux = sp.sin(zz) * (Psi.H * K0_s * Psi)[0, 0]  # cos z J^(x0)
divergence = sp.expand(sp.diff(density, x4) + sp.diff(flux, zz))
```

Exact B and K_0; `Psi.H` is sympy's conjugate transpose $\Psi^\dagger$. The two densities of Section 21.18, $\cos z\, \Psi^\dagger B \Psi$ and $\sin z\, \Psi^\dagger K_0 \Psi$, and the divergence $\partial_4(\ldots) + \partial_z(\ldots)$, the only two terms of the conservation law that do not vanish for this solution.

```
check(sp.simplify(divergence) == 0
      and PY_THEORY["commuting_current_conservation"] == "PASS",
      "d4(cos z J^(x4)) + dz(cos z J^(x0)) = 0 exactly for the exact solution",
      record="Revision/theory/reports/python-field-theory.json, check "
      "commuting_current_conservation")
```

The divergence simplifies to exactly zero, as Section 21.17 proved in general. (The text cell before this code cell says that "the current has two nonzero components"; precisely, all eight components of this solution's current are nonzero, but only these two enter the conservation law, because the others do not depend on their own coordinate. The computation is unaffected.)

In [11]: the chirality partner and the density maps.

```
Gamma_s = sp.Matrix(Gamma.tolist())
partner = Gamma_s * Psi
E_partner = (G[4] * partner.diff(x4) + 6 * Hn * tan_z * G[0] * partner.diff(zz)
             + 3 * Hn * G[0] * partner + m_sol * partner)  # mass -2
density_p = sp.cos(zz) * (partner.H * B_s * partner)[0, 0]
flux_p = sp.sin(zz) * (partner.H * K0_s * partner)[0, 0]
```

The partner $\Gamma\Psi$, the left side of the field equation with the mass -2 (the mass term $-(-2)\Gamma\Psi = +2\Gamma\Psi$), and the partner's two densities.

```
check(E_partner.expand() == sp.zeros(16, 1)
      and sp.expand(density_p + density) == 0 and sp.expand(flux_p + flux) == 0
      and WL_PAIR["T1_current_primordial_commuting"] == "PASS"
      and PY_PAIR["T1.metric.commuting.current"] == "PASS",
      "Gamma Psi solves the equation with mass -2 and carries the opposite "
      "current: the pair's total current is zero",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "T1_current_primordial_commuting")
```

Theorem T1 on this solution (Section 21.19): the partner solves the equation with the reversed mass, and its densities are exactly the negatives of those of Ψ; both pairing reports hold the corresponding checks with the verdict PASS.

```
density_f = sp.lambdify((x4, zz), density, "numpy")
times = np.linspace(0.0, 8.0, 161)
angles = np.linspace(0.0, np.pi / 2, 91)
T, Z = np.meshgrid(times, angles)  # every pair (time, angle)
rho = np.real(density_f(T, Z))  # the charge density of Psi
```

The density becomes a fast function; 161 times from 0 to 8 and 91 angles from 0 to $\pi/2$; `np.meshgrid` makes two arrays that together list every pair (time, angle); `rho` is the density at all of them.

```
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.0), sharey=True)
limit = np.abs(rho).max()
for ax, values, title in ((axes[0], rho, "$\\Psi$ (mass $+2$)"),
                          (axes[1], -rho, "$\\Gamma\\Psi$ (mass $-2$)"),
                          (axes[2], rho - rho, "sum of the pair")):
    image = ax.pcolormesh(T, Z, values, cmap="RdBu_r", vmin=-limit, vmax=limit,
                          shading="auto")
    ax.set_title(title)
    ax.set_xlabel("time $x_4$")
axes[0].set_ylabel("angle $z = 6Hx_0$ (brane at $\\pi/2$)")
fig.colorbar(image, ax=axes, shrink=0.85, label="charge density $\\cos z\\,J^{(x4)}$")
```

Three colour maps over the plane of time and angle (`pcolormesh` colours each small cell by its value; the colour scale is symmetric about zero). The partner's map is drawn as `-rho`, which the check above proved equal to its density; the third map is the sum, zero.

```
save_figure(fig, "charge_density_maps",
            "The charge density $\\cos z\\,J^{(x4)}$ of an exact solution of the "
            "field equation in the author's metric ($H = 1/6$, $\\alpha = 1$, mass "
            "$+2$; left), of its chirality partner $\\Gamma\\Psi$ (a solution with "
            "mass $-2$; middle) and of the pair (right); horizontal axis the time "
            "$x_4$, vertical axis the angle $z = 6Hx_0$ from the tip $0$ to the "
            "brane $\\pi/2$, colour the density (red positive, blue negative, pure "
            "numbers). The partner carries exactly the opposite density at every "
            "point, so the pair carries none.")
```

Figure 21b.3. **What the figure shows.** Left: for this column the density is mostly negative: dark blue bands, separated by faint red ones, repeat in time with the oscillation of the solution (period about 2.37), and every band fades out towards the tip $z = 0$ (the factor $\sin^2 z$) and towards the brane $z = \pi/2$ (the factor $\cos z$). Middle: the same picture with red and blue exchanged. Right: a uniform pale panel: the pair's density is zero everywhere.

In [12]: the charge balance and the stationary solution.

```
t = sp.symbols("t", real=True)
f_of = sp.expand((U_sol * chi).H * B_s * (U_sol * chi))[0, 0]  # f(x4)
g_of = sp.expand((U_sol * chi).H * K0_s * (U_sol * chi))[0, 0]  # g(x4)
Q_of = f_of / 3
```

The functions $f(x_4) = \chi^\dagger U^T B U \chi$ and $g(x_4) = \chi^\dagger U^T K_0 U \chi$ of Section 21.18 (the conjugate transpose of the real U times χ is $\chi^\dagger U^T$), and the charge $Q = f/3$.

```
balance = sp.simplify(Q_of - Q_of.subs(x4, 0)
                      + sp.integrate(g_of.subs(x4, t), (t, 0, x4)))
check(balance == 0, "charge balance: Q(x4) - Q(0) = - (flux through the brane), "
      "exactly")
```

`sp.integrate(g_of.subs(x4, t), (t, 0, x4))` is the exact integral $\int_0^{x_4} g(t)\,dt$ (the variable renamed to t). The balance $Q(x_4) - Q(0) + \int_0^{x_4} g = 0$ holds exactly.

```
P_stat = (sp.eye(16) - sp.I * M_sol / w) / 2  # projector onto M chi = i w chi
chi0 = P_stat * chi  # the single-frequency part of chi
g0 = sp.simplify(sp.expand((U_sol * chi0).H * K0_s * (U_sol * chi0))[0, 0])
Q0 = sp.simplify(sp.expand((U_sol * chi0).H * B_s * (U_sol * chi0))[0, 0] / 3)
```

The projector $P = \frac{1}{2}(1 - iM/w)$, the stationary column $\chi_0 = P\chi$, its flux function g_0 and its charge Q_0.

```
check((P_stat * P_stat - P_stat).applyfunc(sp.simplify) == sp.zeros(16, 16)
      and (M_sol * chi0 - sp.I * w * chi0).applyfunc(sp.simplify) == sp.zeros(16, 1)
      and g0 == 0 and Q0.free_symbols == set() and Q0 != 0,
      "the stationary solution sin z exp(i w x4) chi0: zero flux, constant nonzero Q")
```

$P^2 = P$; $M\chi_0 = iw\chi_0$; the flux is exactly zero; `Q0.free_symbols == set()` means that Q_0 contains no symbol at all, so it does not depend on the time; and it is not zero.

```
report("charge of the general solution at x4 = 0",
       f"{float(sp.re(Q_of.subs(x4, 0))):.6f}")
report("constant charge of the stationary solution", f"{Q0} = {float(Q0):.6f}")
```

Out [12] prints RESULT lines: $Q(0) = 0.178467$ for the general column and $Q_0 = -1424/1875 - 13\sqrt{7}/1500 = -0.782397$ (sympy writes the square root as `sqrt(7)`).

In [13]: the balance in a figure.

```
Q_f = sp.lambdify(x4, Q_of, "numpy")
g_f = sp.lambdify(x4, g_of, "numpy")
integral_f = sp.lambdify(x4, sp.integrate(g_of.subs(x4, t), (t, 0, x4)), "numpy")
Q_vals = np.real(Q_f(times))
rhs_vals = np.real(Q_f(0.0) - integral_f(times))
```

Fast functions for the charge, the flux and its integral; the charge at the 161 times and the prediction $Q(0) - \int_0^{x_4} g$.

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.3))
axes[0].plot(times, Q_vals, linewidth=3, label="charge $Q(x_4)$ of the patch")
axes[0].plot(times, rhs_vals, "--", color="black",
             label="$Q(0) - \\int_0^{x_4}$ flux")
axes[0].plot(times, np.real(g_f(times)), ":", label="flux $g(x_4)$ through the brane")
axes[0].set_xlabel("time $x_4$")
axes[0].set_ylabel("charge, flux (pure numbers)")
axes[0].set_title("General solution: charge flows through the brane")
```

The left panel: the charge as a thick line, the prediction as a thin black dashed line on top of it, and the flux as a dotted line.

```
top = 1.9 * np.abs(np.real(g_f(times))).max()  # room for the legend above
axes[0].set_ylim(-1.2 * np.abs(np.real(g_f(times))).max(), top)
axes[0].legend(fontsize=8, loc="upper right")
```

The vertical range is set from -1.2 to 1.9 times the largest flux, which leaves room for the legend at the top.

```
Q0_value = float(Q0)
axes[1].plot(times, np.full_like(times, Q0_value), linewidth=2,
             label="$Q$ of $\\Psi$ (mass $+2$)")
axes[1].plot(times, np.full_like(times, -Q0_value), "--", linewidth=2,
             label="$Q$ of $\\Gamma\\Psi$ (mass $-2$)")
axes[1].plot(times, np.zeros_like(times), ":", color="black", linewidth=2,
             label="total charge of the pair")
axes[1].set_xlabel("time $x_4$")
axes[1].set_title("Zero flux: the charge is constant")
axes[1].set_ylim(-1.3 * abs(Q0_value), 2.2 * abs(Q0_value))  # legend fits on top
axes[1].legend(fontsize=8, loc="upper right")
```

The right panel: three constant lines (`np.full_like(times, value)` is an array of the same length filled with one value): the stationary charge, the partner's opposite charge (Section 21.19) and the pair's zero total.

```
check(np.max(np.abs(Q_vals - rhs_vals)) < 1e-12,
      "numerically: Q(x4) and Q(0) minus the integrated flux agree to 1e-12")
```

The balance holds to 10^{-12} at all 161 times.

```
save_figure(fig, "charge_balance",
            "The charge balance of exact solutions in the author's metric ($H = "
            "1/6$, $\\alpha = 1$, mass $2$). Left: for a general column $\\chi$, "
            "the charge $Q(x_4)$ of the patch $0 < z < \\pi/2$ (thick line), the "
            "prediction $Q(0) - \\int_0^{x_4} g$ of the conservation law (dashed, "
            "on top of it) and the flux $g$ through the brane (dotted), versus the "
            "time $x_4$. Right: for the stationary solution $\\sin z\\,e^{iwx_4}"
            "\\chi_0$ the flux vanishes, its charge is constant, its chirality "
            "partner "
            "carries the opposite constant charge and the pair carries none. All "
            "quantities per unit of the other six coordinates (pure numbers).")
```

Figure 21b.4. **What the figure shows.** Left: the charge rises and falls; wherever the flux (dotted) is positive the charge decreases and wherever it is negative the charge increases, and the dashed prediction lies exactly on the thick line. Right: two horizontal lines at ∓ 0.782397 and the zero line between them.

In [14]: a finite-difference test.

```
flux_f = sp.lambdify((x4, zz), flux, "numpy")
steps = 0.2 / 2.0 ** np.arange(7)  # 0.2, 0.1, ..., 0.003125
t0, z0 = 1.3, 0.7
```

The flux as a fast function; seven steps h that halve each time; the test point $(x_4, z) = (1.3, 0.7)$.

```
residuals = np.array([abs(np.real(
    (density_f(t0 + h, z0) - density_f(t0 - h, z0)) / (2 * h)
    + (flux_f(t0, z0 + h) - flux_f(t0, z0 - h)) / (2 * h))) for h in steps])
ratios = residuals[:-1] / residuals[1:]
```

For each step the divergence is computed with **central differences**, $\partial F \approx (F(x+h) - F(x-h))/(2h)$, whose error is proportional to h^2 (Chapter 2). `residuals[:-1] / residuals[1:]` divides each residual by the next one (`[:-1]` is all but the last, `[1:]` all but the first).

```
say("residuals: " + ", ".join(f"{r:.2e}" for r in residuals))
say("ratios when h is halved: " + ", ".join(f"{q:.3f}" for q in ratios))
check(np.all(np.abs(ratios - 4.0) < 0.05),
      "finite differences: the residual falls by 4 when h is halved (order 2)")
```

Out [14] prints the residuals from 3.68×10^{-3} down to 9.10×10^{-7} (`.2e` is scientific notation with two decimals) and the ratios 3.966, 3.991, 3.998, 3.999, 4.000, 4.000: halving h divides the residual by 4, the signature of order 2 converging to the exact value zero.

```
fig, ax = plt.subplots(figsize=(7.0, 4.4))
ax.loglog(steps, residuals, "o-", label="|central-difference divergence|")
ax.loglog(steps, residuals[0] * (steps / steps[0]) ** 2, "--",
          label="slope 2: proportional to $h^2$")
ax.set_xlabel("step $h$")
ax.set_ylabel("residual of the conservation law")
ax.set_title("A numerical test converges to the exact law")
ax.legend()
```

A plot with both axes logarithmic (`loglog`), the residuals and a reference line proportional to h^2 through the first point.

```
save_figure(fig, "local_conservation",
            "The local conservation law $\\partial_4(\\cos z\\,J^{(x4)}) + "
            "\\partial_z(\\cos z\\,J^{(x0)}) = 0$ tested with central differences "
            "of step $h$ at the point $x_4 = 1.3$, $z = 0.7$ for the exact solution "
            "of the author's metric; horizontal axis the step $h$, vertical axis "
            "the size of the computed divergence, both on logarithmic scales "
            "(pure numbers). The points follow the dashed line of slope 2: the "
            "error of the difference formula, proportional to $h^2$, is all there "
            "is; the exact divergence is zero.")
```

Figure 21b.5. **What the figure shows.** Seven points on a straight line of slope 2, lying on the dashed reference line: what a computer measures as a nonzero divergence is only the error of the difference formula.

In [15]: the charge while the extra times deflate.

```
a4_vals = times / 6.0  # a4 = A H x4 with A = 1, H = 1/6
fig, axes = plt.subplots(2, 1, figsize=(8.0, 6.4), sharex=True)
axes[0].semilogy(times, np.exp(a4_vals), label="ordinary space: $e^{a_4}$")
axes[0].semilogy(times, np.exp(-a4_vals), "--",
                 label="extra times $x5, x6, x7$: $e^{-a_4}$ (deflating)")
axes[0].semilogy(times, np.exp(3 * a4_vals) * np.exp(-3 * a4_vals), ":",
                 color="black", label="slice volume $e^{3a_4}e^{-3a_4} = 1$")
```

The linear history $a_4 = AHx_4$ with $A = 1$ and $H = 1/6$, chosen only for the picture. Two panels stacked vertically with a common time axis; the upper one draws, on a logarithmic axis, the growth factor of space, the shrinking factor of the extra times and the product of the cubes, which is exactly 1.

```
axes[0].set_ylabel("scale factor (log scale)")
axes[0].set_title("The history $a_4 = AHx_4$ ($A = 1$, $H = 1/6$)")
axes[0].legend(fontsize=8)
axes[1].plot(times, np.full_like(times, Q0_value), linewidth=2,
```

```
             label="charge $Q$ of the stationary solution")
axes[1].set_ylim(-1.5 * abs(Q0_value), 1.5 * abs(Q0_value))
axes[1].axhline(0.0, color="black", linewidth=0.8)
axes[1].set_xlabel("time $x_4$")
axes[1].set_ylabel("charge $Q$")
axes[1].legend(fontsize=8)
```

Labels and legend of the upper panel; the lower panel draws the constant charge of the stationary solution and the zero line.

```
save_figure(fig, "charge_through_deflation",
            "Top: the scale factors of the author's metric along the time $x_4$ "
            "for the linear history $a_4 = AHx_4$ with $A = 1$, $H = 1/6$ "
            "(chosen for the picture): ordinary space $e^{a_4}$ grows, the three "
            "extra times $e^{-a_4}$ deflate exponentially, and their product, the "
            "volume factor of a slice, stays $1$ (logarithmic vertical scale). "
            "Bottom: the charge $Q$ of the stationary exact solution versus $x_4$; "
            "it does not change while space inflates and the extra times deflate. "
            "The stationary solution solves the field equation for every "
            "history $a_4$ and its charge density contains no $a_4$, so the "
            "constant does not depend on this choice.")
```

Figure 21b.6. **What the figure shows.** Top: two straight lines on the logarithmic scale, one rising and one falling at the same rate, and a flat dotted line at 1 between them. Bottom: a flat line at -0.782: the charge is untouched by the deflation.

In [16]: the charge density is indefinite.

```
from fractions import Fraction

eig_B = np.linalg.eigvalsh(B)
plus_space = (I16 - 1j * gamma[4]) / 2  # projector onto -i gamma^(x4) = +1
```

`Fraction` (a Python module for exact fractions) is used to print the result. `np.linalg.eigvalsh` computes the eigenvalues of a Hermitian matrix in increasing order. `plus_space` is $\frac{1}{2}(1 + (-i\gamma^{(x4)}))$, the projector onto the eigenvectors of $-i\gamma^{(x4)}$ with eigenvalue $+1$ (the rest states of Section 21.18).

```
# B restricted to that space: an 8-dimensional block, eigenvalues +-1
values, vectors = np.linalg.eigh(plus_space @ B @ plus_space)
u_plus = vectors[:, np.argmax(values)]  # a vector with -i g4 u = u, B u = +u
u_minus = vectors[:, np.argmin(values)]  # a vector with -i g4 u = u, B u = -u
```

`np.linalg.eigh` returns the eigenvalues and an orthonormal set of eigenvectors (the columns of `vectors`). The matrix PBP has the eigenvalues of B restricted to the rest-state space and zeros elsewhere; `np.argmax` and `np.argmin` give the positions of the largest ($+1$) and smallest (-1) eigenvalue, and `vectors[:, k]` is column `k`.

```
u = 0.6 * u_plus + 0.8 * u_minus  # 3/5 and 4/5
rest_density = np.conj(u) @ B @ u
restricted = np.linalg.eigvalsh(plus_space @ B @ plus_space)
```

The worked example $u = \frac{3}{5}u_+ + \frac{4}{5}u_-$, its charge density $u^\dagger B u$, and the eigenvalues of PBP once more for the check.

```
check(np.allclose(sorted(eig_B), [-1] * 8 + [1] * 8)
      and np.allclose(-1j * gamma[4] @ u, u)
      and np.sum(restricted > 0.5) == 4 and np.sum(restricted < -0.5) == 4
      and abs(rest_density - (-7 / 25)) < 1e-12,
      "B has signature (8,8); a positive-frequency rest state has charge density "
      "-7/25")
report("charge density of the rest state 3/5 u+ + 4/5 u-",
       Fraction(round(rest_density.real * 25), 25))
```

B has eight eigenvalues -1 and eight $+1$; u is a rest state; the restricted block has four eigenvalues $+1$ and four -1 (`np.sum` of a true-false array counts the trues); the density is $-7/25$. The RESULT line prints the density as the exact fraction $-7/25$ (25 times the density, rounded to a whole number, over 25).

```
samples = rng.normal(size=(20000, 16)) + 1j * rng.normal(size=(20000, 16))
ratio = (np.einsum("nr,rc,nc->n", np.conj(samples), B, samples).real
         / np.einsum("nr,nr->n", np.conj(samples), samples).real)
say(f"fraction of random columns with negative charge density: "
    f"{np.mean(ratio < 0):.3f}")
```

20000 random complex columns (from the generator of In [9], which continues its sequence). For each, `einsum` computes $\Psi^\dagger B \Psi$ and $\Psi^\dagger \Psi$, and `ratio` is their quotient. `np.mean(ratio < 0)` is the fraction of negative ones: Out [16] prints 0.504.

```
check(ratio.min() >= -1 - 1e-12 and ratio.max() <= 1 + 1e-12
      and 0.4 < np.mean(ratio < 0) < 0.6,
      "random columns: the ratio lies in [-1, 1] and takes both signs")
fig, ax = plt.subplots(figsize=(7.5, 4.3))
ax.hist(ratio, bins=60, range=(-1, 1), color="#7a8fb3")
ax.axvline(-7 / 25, color="#e34948", linewidth=2,
           label="rest state with positive frequency: $-7/25$")
ax.set_xlabel("$\\Psi^\\dagger B\\Psi / \\Psi^\\dagger\\Psi$")
ax.set_ylabel("number of random columns")
ax.set_title("The charge density is indefinite")
ax.legend(fontsize=8)
```

The ratio lies between -1 and $+1$ (Section 21.18) and is negative for between 40 and 60 per cent of the columns. `ax.hist` draws a **histogram**: the interval from -1 to 1 is cut into 60 bins and each bar counts the columns whose ratio falls into it. A red vertical line marks $-7/25$.

```
save_figure(fig, "indefinite_charge",
            "Histogram of the normalised charge density $\\Psi^\\dagger B\\Psi / "
            "\\Psi^\\dagger\\Psi$ for 20000 random complex columns $\\Psi$ (fixed "
            "seed); horizontal axis the ratio, from $-1$ to $+1$, vertical axis the "
            "number of columns in each of 60 bins. Because $B$ has eight "
            "eigenvalues $+1$ and eight $-1$, the ratio takes both signs equally "
            "often. The red line marks the worked example: a flat-space rest "
            "state of positive frequency with charge density $-7/25$.")
```

Figure 21b.7. **What the figure shows.** A bell-shaped histogram centred at zero and symmetric: the charge density of a random field is positive or negative with equal probability, and the red line shows that a field of positive frequency can lie on the negative side.

In [17]: the pair-level bookkeeping.

```
U_f = sp.lambdify(x4, U_sol, "numpy")
columns = rng.normal(size=(5, 16)) + 1j * rng.normal(size=(5, 16))
check_times = [0.0, 2.5, 5.0]
totals, charges = [], []
```

The matrix $U(x_4)$ as a fast function, five random complex columns χ, three test times, and two empty lists.

```
for col in columns:
    u_t = [np.array(U_f(tt), dtype=float) @ col for tt in check_times]
    q_psi = [np.conj(v) @ B @ v / 3 for v in u_t]  # Q = f / 3 for alpha = 1
    q_par = [np.conj(Gamma @ v) @ B @ (Gamma @ v) / 3 for v in u_t]
    charges.append((q_psi[0].real, q_par[0].real))
    totals += [abs(a + b_) for a, b_ in zip(q_psi, q_par)]
```

For each column and each time, $v = U(x_4)\chi$ and the charge $Q = v^\dagger B v/3$ of the solution $\sin z\, U\chi$ (Section 21.18, line 1), and the charge of the partner Γv. The charges at the first time are kept for the figure; the sizes of the totals $Q + Q'$ are collected.

```
check(max(totals) < 1e-12 and min(abs(c[0]) for c in charges) > 1e-3,
      "five random columns at three times: Q + Q(partner) = 0, Q itself nonzero")
```

All 15 totals vanish to 10^{-12}, while every single charge is far from zero.

```
fig, ax = plt.subplots(figsize=(8.0, 4.3))
idx = np.arange(5)
ax.bar(idx - 0.25, [c[0] for c in charges], width=0.25, label="$Q$ of $\\Psi$")
ax.bar(idx, [c[1] for c in charges], width=0.25, label="$Q$ of $\\Gamma\\Psi$")
ax.plot(idx + 0.25, [c[0] + c[1] for c in charges], "D", color="black",
        label="total of the pair (zero)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(idx, [f"column {k + 1}" for k in idx])
ax.set_ylabel("charge $Q$ at $x_4 = 0$")
ax.set_title("Theorem T1: the partner carries the opposite charge")
ax.legend(fontsize=8)
```

For each column two bars (the charges of Ψ and of $\Gamma\Psi$) and a black diamond (`"D"`) at their total.

```
save_figure(fig, "pair_bookkeeping",
            "Pair-level charge bookkeeping for five random columns $\\chi$ in the "
            "exact solution of the author's metric: the charge $Q$ of $\\Psi$ "
            "(left bar of each group), of its chirality partner $\\Gamma\\Psi$, a "
```

```
        "solution with the mass reversed (right bar), and their total (black "
        "diamond, zero); horizontal axis the column, vertical axis the charge at "
        "$x_4 = 0$ per unit of the other six coordinates (pure numbers). The "
        "charges of single universes can have either sign and any size; only "
        "the pair adds to zero.")
```

Figure 21b.8. **What the figure shows.** Pairs of bars of equal height and opposite sign, some positive first and some negative first, and five diamonds on the zero line: the pair-level statement of theorem T1. The figure shows a property of pairs of solutions; it does not show that such pairs are made (Section 21.19).

In [18]: the figure files.

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)
check(len(names) == 8 and all(
    output_file(f"{FIGURE_FOLDER}/21b_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the eight figure files of notebook 21b exist")
all_checks_passed()
```

As in the last cell of Notebook 21a: the eight figure files exist, and the last line prints ALL 21 CHECKS PASSED (notebook 21b).

21.24 Reflections and the chirality: P, T and Γ

Sakharov's second condition speaks of C, P and T. Sections 21.8 to 21.10 found the matrices of charge conjugation. This section collects what the Revision record proves about the reflections, which play the roles of P and T, and about the chirality Γ. All statements are from the pairing record (`Revision/docs/PAIR_CREATION_PROOFS.md`, its sections 3 and 5; Chapter 18 gives the complete proofs); the one-line derivations are repeated here.

Reflections. The reflection R_n of a frame direction n reverses that direction and keeps the others: it is the diagonal matrix with -1 in place n and $+1$ elsewhere. Reflecting a space-like direction (x_0, x_1, x_2 or x_3) is a **parity**-type operation (P); reflecting the time x_4 is **time reversal** (T); reflecting an extra time x_5, x_6, x_7 is a time-reversal-type operation of an extra time. On the spinor, the reflection of the direction n is carried by the matrix $P_n = \Gamma\gamma^{(n)}$, which is plus or minus the product of the seven other gammas, hence an element of Pin(4,4) (record check `T2_Pn_in_Pin44`), and which acts on the gammas exactly as R_n (check `T2_Pn_covers_the_reflection`).

What a reflection does to the scalar, in one line. With the real matrix $\gamma^{(n)}$ (so $(\gamma^{(n)})^\dagger = (\gamma^{(n)})^T$) and (R3) in the form $(\gamma^a)^T C = -C\gamma^a$:

$$S[\gamma^{(n)}\Psi] = \Psi^\dagger (\gamma^{(n)})^T C \gamma^{(n)} \Psi = -\Psi^\dagger C \gamma^{(n)}\gamma^{(n)}\Psi = -\eta_{nn}\, S[\Psi],$$

because $\gamma^{(n)}\gamma^{(n)} = \eta_{nn}$. So a space-like reflection reverses the mass term and a time-like one keeps it. The record also proves (checks `T2_kernels_gamma_n_with_frame_reflection` of the Wolfram pairing report and `T2.general_field.reflection_table` of the sympy pairing report) that the kinetic term is multiplied by $+\eta_{nn}$ when the frame is reflected along. The result for the eight directions:

direction	η_{nn}	S under $\gamma^{(n)}$	kinetic term	parameters and Lagrangian
x_1, x_2, x_3 (P type)	$+1$	$-S$	kept	$(-m, \lambda)$, $+\mathcal{L}$ (T2 type)
x_4 (T)	-1	$+S$	reversed	$(-m, -\lambda)$, $-\mathcal{L}$ (T1 type)
x_5, x_6, x_7	-1	$+S$	reversed	$(-m, -\lambda)$, $-\mathcal{L}$ (T1 type)
x_0 (P type)	$+1$	$-S$	kept	$(-m, \lambda)$, $+\mathcal{L}$ (T2 type)

Every reflection maps the theory with mass m to a theory with mass $-m$. In the author's metric only the reflection of x_0 is realised by a map of the space onto itself, the mirror across the brane $z = \pi/2$, and that uses the ASSUMED Z2 construction (theorem T2 of the record; Chapter 18); the reflections of x_1, x_2, x_3 are statements about frames. For a space-like reflection the charge density is unchanged (record, statement T2c), so the T2 partner has equal, not opposite, charge.

The chirality. Γ is the product of all eight gammas, the spinor image of reflecting all eight directions at once. It keeps S and reverses every kinetic term and every current (Sections 21.10 and 21.19): theorem T1.

The Krein signs. After quantisation the matrix B fixes the canonical anticommutator (Section 21.25). A constant map $\Psi \to M\Psi$ turns it into $MBM^\dagger$, and the record proves $MBM^\dagger = \sigma_M B$ with a sign σ_M (check `Q_Krein_metric_of_images`). For the chirality, line by line: $\Gamma B \Gamma^\dagger = \Gamma B \Gamma = -iC\Gamma\gamma^{(x4)}\Gamma = -iC(-\gamma^{(x4)}) = -B$ (Γ is real and symmetric; it commutes with C, (R2), and anticommutes with $\gamma^{(x4)}$, (R1)). So $\sigma_\Gamma = -1$: the chirality image of a quantised field carries the Krein metric $-B$. For the single gammas the record finds $\sigma = +1$ for x_0, x_1, x_2, x_3, x_4 and -1 for x_5, x_6, x_7.

statement	status	where it is verified
$P_n = \Gamma\gamma^{(n)}$ is in Pin(4,4) and covers the reflection R_n	PROVED	`Revision/pairing/reports/wolfram-pairing.json`, checks `T2_Pn_in_Pin44`, `T2_Pn_covers_the_reflection`
$S[\gamma^{(n)}\Psi] = -\eta_{nn}S$; the table of the eight reflections	PROVED	the same report, `T2_character_of_Pn`, `T2_kernels_gamma_n_with_frame_reflection`; `Revision/pairing/reports/python-pairing.json`, `T2.general_field.reflection_table`
$\Gamma B \Gamma = -B$ and the Krein signs of the reflections	PROVED	`Revision/pairing/reports/wolfram-pairing.json`, check `Q_Krein_metric_of_images`

21.25 The quantised field: which conjugation keeps the canonical anticommutator

From numbers to operators. After **canonical quantisation** (Chapter 10) the 16 components of the field dirac16complex are no longer numbers but **operators** Ψ_A, $A = 1, \ldots, 16$: rules that turn one state of the quantum system into another. Two operators need not commute. The **anticommutator** of two operators is $\{X, Y\} = XY + YX$. The Revision record (`Revision /theory/field-theory.json`, formula `quantisation`) fixes, on a slice of constant time x_4,

$$\{\Psi_A(x), \Psi_C^\dagger(y)\} = B_{AC}\, \delta^7(x-y)/\cos z, \qquad \{\Psi_A(x), \Psi_C(y)\} = 0,$$

the **canonical anticommutator**, which the Lagrangian dictates. Because B has eight negative eigenvalues, $\Psi^\dagger$ cannot be the ordinary (Hilbert) adjoint of Ψ: the state space carries an indefinite form, a **Krein space**. The record realises the rule in the **positive representation** $\Psi_A = b_A$, $\Psi_C^\dagger = \sum_D b_D^* B_{DC}$, where b_A and b_A^* are ordinary fermion operators (Section 21.26). **Its scope.** This representation realises the anticommutation rule of the 16 components at one point on an ordinary (positive) space of 2^{16} states; it says nothing about the energy. A positive Fock space on which the normal-ordered Hamiltonian is non-negative is constructed in the record only for single good-sector momenta (no dependence on the extra times) with frozen coefficients: the Wolfram check `Fock_space_good_sector_example` and the sympy check `good_sector_positive_fock_realisation` of the field-theory record, whose reports are in the folder `Revision/theory/reports/`. In the author's metric the curved good-sector mode operator is Hermitian only up to a boundary term at the brane $z = \pi/2$, and without a boundary condition there the good sector has complex frequencies (for $U = 0$) whenever $m^2 < 9H^2$ (checks `good_sector_hermiticity_up_to_the_brane_flux` and `good_sector_x0_independent_modes_without_boundary_condition` of `Revision/theory/reports/wolfram-scope.json`). A positive-norm Hilbert space for the full field in the deflating background is not established (OPEN). In this section the delta function plays no role and is left out: everything is said for one point.

The question. Charge conjugation of the quantised field must turn Ψ into a new field built from the adjoints, $\Psi' = M\Psi^{\dagger T}$, that is $\Psi'_A = \sum_B M_{AB}\Psi_B^\dagger$ with a constant matrix M. For Ψ' to be a field of the **same** quantum theory it must obey the **same** canonical rule.

The condition, line by line. The adjoint of Ψ'_C is $\Psi_C'^\dagger = \sum_D M_{CD}^* \Psi_D$ (the adjoint of a number times an operator conjugates the number; the adjoint of $\Psi^\dagger$ is Ψ).

Line 1. $\{\Psi'_A, \Psi_C'^\dagger\} = \sum_{B,D} M_{AB} M_{CD}^* \{\Psi_B^\dagger, \Psi_D\}$ (the anticommutator is linear in each of its two slots).

Line 2. $\{\Psi_B^\dagger, \Psi_D\} = \{\Psi_D, \Psi_B^\dagger\} = B_{DB} = (B^T)_{BD}$ (the anticommutator is symmetric, $XY + YX = YX + XY$; then the canonical rule).

Line 3. So $\{\Psi'_A, \Psi_C'^\dagger\} = \sum_{B,D} M_{AB}(B^T)_{BD}(M^\dagger)_{DC} = (MB^TM^\dagger)_{AC}$ (the definition of the matrix product, with $M_{CD}^* = (M^\dagger)_{DC}$).

Line 4. Ψ' obeys the canonical rule if and only if

$$MB^TM^\dagger = B.$$

Which M satisfy it. B is Hermitian and purely imaginary, so $B^T = (B^\dagger)^* = B^* = -B$. For $M = \Gamma$: $\Gamma B^T \Gamma^\dagger = -\Gamma B \Gamma = -(-B) = B$ (Section 21.24): **the condition holds**. For $M = 1$: $B^T = -B \neq B$: **it fails**. More generally, for the family $M(t) = \cos t\, 1 + \sin t\, \Gamma$, which is real and symmetric:

$$M(t)\, B^T M(t)^\dagger = -M(t) B M(t) = -\big(\cos^2 t\, B + \cos t \sin t\, (\Gamma B + B\Gamma) + \sin^2 t\, \Gamma B \Gamma\big)$$

(the product multiplied out). Γ anticommutes with B (Γ commutes with C and anticommutes with $\gamma^{(x4)}$), so the middle term vanishes, and $\Gamma B \Gamma = -B$. Hence

$$M(t)\, B^T M(t)^\dagger = -(\cos^2 t - \sin^2 t)\, B = -\cos(2t)\, B,$$

which equals B only when $\cos 2t = -1$, that is $t = \pi/2$ or $3\pi/2$: $M = \pm\Gamma$. The size of the failure is $|1 + \cos 2t|$ times the size of B, which is 4 (the square root of the sum of the squares of its 16 entries of modulus 1); at $t = 0$ it is 8. A multiple $c\,\Gamma$ gives $|c|^2 B$, so it works exactly when $|c| = 1$, a phase. Notebook 21c computes all of this (In [4], figure 21c.2), reproducing the lead check `quantum_charge_conjugation_unitary_type`.

Which mass? A conjugation must also map solutions to solutions. The intertwiner condition of Section 21.8, the definition of a charge-conjugation matrix, is enough for this (though not necessary: for $V = 0$ the projector P_- also does it, Section 21.8), and by Theorem CC the only matrices that obey it are the multiples of 1 (same mass) and of Γ (mass reversed). So for the quantised field **the only charge-conjugation matrix that keeps the canonical anticommutator gives $\Psi \to \Gamma \Psi^{\dagger T}$ (times a phase), and it reverses the mass.** The same-mass candidate $\Psi \to \Psi^{\dagger T}$ would turn B into $-B$.

21.26 Particles and holes; the mass reversed, the spectra equal

Fermion modes from zero. A **fermion mode** is a place that holds at most one particle: it is either empty (0) or occupied (1). Sixteen modes have $2^{16} = 65536$ states, one for each list of 16 zeros and ones. The **annihilation operator** b_j empties mode j (and gives zero if it is already empty); the **creation operator** b_j^* fills it (and gives zero if it is already full). To make operators of different modes anticommute, each action on mode j also multiplies by (-1) raised to the number of occupied modes before j (the **Jordan-Wigner** sign rule). With it, $\{b_j, b_k^*\} = \delta_{jk}$ and $\{b_j, b_k\} = 0$. The number operator $b_j^* b_j$ is 1 on states where mode j is occupied and 0 otherwise, and $N = \sum_j b_j^* b_j$ counts the occupied modes. Notebook 21c builds these operators explicitly on all 65536 states (In [5]).

The charge counts the occupied modes. In the positive representation the charge operator is

$$Q = \sum_{A,C} \Psi^\dagger_A B_{AC} \Psi_C = \sum_{A,C,D} b^*_D B_{DA} B_{AC} b_C = \sum_{D,C} b^*_D (B^2)_{DC} b_C = \sum_D b^*_D b_D = N$$

(insert $\Psi^\dagger_A = \sum_D b^*_D B_{DA}$ and $\Psi_C = b_C$; the sum over A is a matrix product; $B^2 = 1$).

The charge of the conjugated field, line by line, for $\Psi' = \Gamma \Psi^{\dagger T}$. Since Γ is diagonal and real, $\Psi'_C = \Gamma_{CC} \Psi^\dagger_C$ and $\Psi'^\dagger_A = \Gamma_{AA} \Psi_A$.

Line 1. $Q' = \sum_{A,C} \Psi'^{\dagger}_A B_{AC} \Psi'_C = \sum_{A,C} \Gamma_{AA} B_{AC} \Gamma_{CC}\, \Psi_A \Psi^{\dagger}_C = \sum_{A,C} (\Gamma B \Gamma)_{AC} \Psi_A \Psi^{\dagger}_C$ (insert; the numbers Γ_{AA} and Γ_{CC} commute with the operators; for a diagonal Γ, $\Gamma_{AA} B_{AC} \Gamma_{CC} = (\Gamma B \Gamma)_{AC}$).

Line 2. $\Gamma B \Gamma = -B$ (Section 21.24), so $Q' = -\sum_{A,C} B_{AC} \Psi_A \Psi^{\dagger}_C$.

Line 3. The canonical rule gives $\Psi_A \Psi^{\dagger}_C = B_{AC} - \Psi^{\dagger}_C \Psi_A$, so $Q' = -\sum_{A,C} B^2_{AC} + \sum_{A,C} \Psi^{\dagger}_C (B^T)_{CA} \Psi_A$ (with $B_{AC} = (B^T)_{CA}$).

Line 4. $\sum_{A,C} B^2_{AC} = \mathrm{tr}(BB^T) = \mathrm{tr}(B(-B)) = -\mathrm{tr}(B^2) = -16$, and $B^T = -B$ turns the second sum into $-Q$. Hence

$$Q' = 16 - Q.$$

The valid conjugate counts the **empty** modes. Up to the constant 16, which does not change any difference of charges, it **reverses the charge**: $Q' - 8 = -(Q - 8)$. Particles and holes are exchanged. The same four lines with $M = 1$ (no factors Γ, so no sign in line 2) give $Q'' = \sum_{A,C} B_{AC} \Psi_A \Psi^{\dagger}_C = -16 + Q$: the invalid same-mass candidate would not even reverse the charge, it would shift it. Notebook 21c checks these three operator identities on a random state of the 16 modes (In [8], figure 21c.4). This operator result is the quantum counterpart of the anticommuting rows of the table of Section 21.10: the valid conjugation reverses the charge and the mass; the same-mass candidate keeps the charge (up to a shift). Normal ordering subtracts the vacuum value ($Q = N = 0$ in the empty state, so Q' has the vacuum value 16 and Q'' the value -16) and gives $:Q':= -Q$ and $:Q'':= Q$: it adds no sign, as the corrected detail text of the record's check `bilinears_under_charge_conjugation` states (Section 21.10).

The mass is reversed, the spectra are equal. In flat 4+4 space ($H = 0$, a_4 constant) a plane wave $u\, e^{ik\cdot x}$, with the momenta k_a along the seven directions other than the time, evolves by $i\, \partial_4 u = h_m(k) u$ with the **one-particle Hamiltonian** of the record

$$h_m(k) = -im\gamma^{(x4)} - \gamma^{(x4)} \sum_{a \neq x4} k_a \gamma^{(a)}.$$

Four facts, each derived in a few lines (write $K = \sum_{a \neq x4} k_a \gamma^{(a)}$, so $h = -\gamma^{(x4)}(im + K)$):

- *The square.* $h^2 = \gamma^{(x4)}(im+K)\gamma^{(x4)}(im+K) = \gamma^{(x4)}\gamma^{(x4)}(im-K)(im+K) = -(-m^2-K^2) = m^2 + K^2$ ($\gamma^{(x4)}$ anticommutes with every gamma in K; $\gamma^{(x4)}\gamma^{(x4)} = -1$; the cross terms $imK - Kim$ cancel). By the Clifford relation $K^2 = \sum_a k_a^2 \eta^{aa}$ (the products of different gammas cancel in pairs). Hence $h^2 = w^2\, 1$ with $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 - k_6^2 - k_7^2$: the frequencies are $\pm w$ (record check `Q_one_particle_flat_dispersion`).
- *The chirality.* $\Gamma h_m \Gamma = -im\Gamma\gamma^{(x4)}\Gamma - \Gamma\gamma^{(x4)}K\Gamma = im\gamma^{(x4)} - \gamma^{(x4)}K = h_{-m}(k)$ (R1 for the single gamma, R2 for the pair): the Hamiltonians of $+m$ and $-m$ are **similar**, so their spectra are **identical**, not opposite (record check `Q_one_particle_maps`).
- *The conjugated mode.* The conjugated field contains Γu^*. Conjugating the evolution gives $-i\, \partial_4 u^* = h^* u^*$, that is $i\, \partial_4 u^* = -h^* u^*$, and multiplying by Γ (with $\Gamma\Gamma = 1$): $i\, \partial_4 (\Gamma u^*) = -\Gamma h^* \Gamma\, (\Gamma u^*)$. Now $h^*_m = im\gamma^{(x4)} - \gamma^{(x4)}K$ (only the i changes sign), so $-\Gamma h^*_m \Gamma = -(-im\gamma^{(x4)} - \gamma^{(x4)}K) = h_{-m}(-k)$: **the conjugated mode is a mode of the theory with the reversed mass** (and the reversed momentum, as $e^{-ik\cdot x}$ says).

- *Krein self-adjointness.* $BhB = h^\dagger$, hence $Bh = h^\dagger B$ ($B^2 = 1$). Derivation: with $(-i)^2 = -1$ and $C\gamma^{(x4)} = \gamma^{(x4)}C$, $BhB = -(C\gamma^{(x4)})h(C\gamma^{(x4)})$; the mass part gives $-(C\gamma^{(x4)})(-im\gamma^{(x4)})(C\gamma^{(x4)}) = imC\gamma^{(x4)}\gamma^{(x4)}C\gamma^{(x4)} = -im\gamma^{(x4)}$ (using $\gamma^{(x4)}\gamma^{(x4)} = -1$ and $CC = 1$) and the momentum part gives $K^T\gamma^{(x4)}$ (using $CKC = -K^T$); and $h^\dagger = im(\gamma^{(x4)})^T - K^T(\gamma^{(x4)})^T = -im\gamma^{(x4)} + K^T\gamma^{(x4)}$, the same (record check `mode_hamiltonian_B_selfadjoint_dispersion` of the sympy field-theory report).

Krein inertia. For a real frequency w each eigenspace of h has dimension 8, and the form $u^\dagger Bu$ restricted to it has four positive and four negative directions: **Krein inertia** (4,4). For an imaginary frequency the eigenspace is **Krein-neutral**: if $hu = wu$ and $hv = wv$, then $w\,u^\dagger Bv = u^\dagger Bhv = u^\dagger h^\dagger Bv = (hu)^\dagger Bv = w^* u^\dagger Bv$, so $(w - w^*)u^\dagger Bv = 0$, and the form vanishes when w is not real (record check `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`). Imaginary frequencies occur when the extra-time momenta dominate, $k_5^2 + k_6^2 + k_7^2 > m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2$; these modes grow exponentially in time (the ill-posedness of the extra times, Chapter 8). The record's eight samples (`Revision/pairing/pairing-theory.json`, data `one_particle_flat`), which Notebook 21c reproduces (In [10]):

m	**momenta** $(k_0, \ldots, k_7)$	w	**dimensions of the $\pm w$ eigenspaces**	**Krein inertia on $+w$ and $-w$**
± 2	(4, 1, 2, 0, 0, 0, 0, 0)	5	8 and 8	(4,4) and (4,4)
± 3	(0, 0, 0, 0, 0, 0, 0, 0)	3	8 and 8	(4,4) and (4,4)
± 1	(0, 1, 0, 0, 0, 0, 0, 0)	$\sqrt{2} = 1.414214$	8 and 8	(4,4) and (4,4)
± 2	(0, 0, 0, 0, 0, 1, 0, 0)	$\sqrt{3} = 1.732051$	8 and 8	(4,4) and (4,4)

(Each row stands for two samples of the record, $+m$ and $-m$, with identical results. The entry k_4 is not used. Check: for the first row $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 = 4 + 16 + 1 + 4 = 25$.)

statement	**status**	**where it is verified**
the canonical anticommutator $B_{AC}\delta/\cos z$ and the positive representation	record statement, checked word for word	`Revision/theory/field-theory.json`, formula `quantisation`; Notebook 21c, In [2]
$MB^TM^\dagger = B$ holds for $M = \Gamma$, gives $-B$ for $M = 1$; in the family $\cos t + \sin t\,\Gamma$ only $t = \pi/2, 3\pi/2$	PROVED (above); COMPUTED	lead check `quantum_charge_conjugation_unitary_type`; Notebook 21c, In [4]
with explicit operators on 65536 states: $\{\Psi_A, \Psi_C^\dagger\} = B_{AC}$; $\Gamma\Psi^{\dagger T}$ gives B, $\Psi^{\dagger T}$ gives $-B$	COMPUTED (all 256 pairs, to rounding)	`Revision/theory/reports/python-field-theory.json`, check `canonical_anticommutator_B`; Notebook 21c, In [6]
$Q = N$, $Q' = 16 - Q$, $Q'' = Q - 16$	PROVED (above); COMPUTED on a random state	Notebook 21c, In [8] (its own computation)

statement	status	where it is verified
$-\Gamma h_m^* \Gamma = h_{-m}(-k)$; $\Gamma h_m \Gamma = h_{-m}$; $h^2 = w^2$; $Bh = h^\dagger B$	PROVED (sympy, symbolic m, k)	Wolfram pairing report, checks `Q_one_particle_maps`, `Q_one_particle_flat_dispersion`; Notebook 21c, In [9]
the eight samples; inertia (4,4) at real, Krein-neutral at imaginary frequencies	PROVED in the record; COMPUTED	checks `Q_one_particle_Krein_signatures`, `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`; Notebook 21c, In [10] to In [12]

What this does **not** do: no regularised quantum field theory in signature (4,4) is constructed here, no positive-norm Hilbert space for the full field is established (Section 21.25), the statements concern the anticommutator and the one-particle problem, and nothing in them creates particles, antiparticles or universes.

21.27 Example: Notebook 21c conjugates the quantised field

Notebook 21c turns Sections 21.25 and 21.26 into computations. It reads the gammas, the lead report, the field-theory record and its sympy report, and the pairing record; it checks the properties of B; it computes $MB^TM^\dagger$ for $M = \Gamma$, $M = 1$ and the family $\cos t\, 1 + \sin t\, \Gamma$; it builds 16 fermion modes with the Jordan-Wigner rule and measures all 256 anticommutators of three fields with explicit operators on 65536 states; it checks $Q' = 16 - Q$; it proves the one-particle facts with sympy, reproduces the eight samples of the pairing record and scans the momenta into the region of growing modes. It runs in about 30 seconds; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/21c_quantum_conjugation.PROVENANCE.md`. It needs no Rust, prints 17 PASS lines and draws six figures.

21.28 How to run Notebook 21c

Step 1. What this notebook does and what it needs.

Notebook 21c (The conjugation of the quantised field: M B^T M^dagger = B) is the file `Revision/textbook/notebooks/21c_quantum_conjugation.ipynb` of the repository Dirac_claude. It derives the condition M B^T M^dagger = B that a conjugation Psi to M Psi^dagger T must satisfy to preserve the canonical anticommutator of the quantised field dirac16complex (B = -i C gamma^(x4), the Krein matrix), finds that it holds for M = Gamma and fails for M = 1 (and, among the matrices cos t + sin t Gamma, only for Gamma), verifies this with explicit fermion operators on the 65536 states of 16 modes (Jordan-Wigner construction in the positive representation of the Revision record), shows that the valid conjugation turns the charge Q into 16 - Q (particles into holes), proves with sympy that it maps the one-particle Hamiltonian of mass m to that of mass -m, and reproduces the one-particle samples of the Revision pairing record: equal spectra for +m and -m, Krein inertia (4,4) at real frequencies, Krein-neutral growing modes at large extra-time momentum.

Six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
```

```
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 21c_quantum_conjugation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21c_quantum_conjugation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 21c_quantum_conjugation.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/21c.captions.json`
- `Revision/textbook/figures/21c_1_krein_matrix.png`
- `Revision/textbook/figures/21c_2_which_conjugation.png`
- `Revision/textbook/figures/21c_3_operator_anticommutators.png`
- `Revision/textbook/figures/21c_4_charge_of_states.png`
- `Revision/textbook/figures/21c_5_dispersion_pm_mass.png`
- `Revision/textbook/figures/21c_6_krein_inertia_scan.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 21c exist
ALL 17 CHECKS PASSED (notebook 21c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/21c_quantum_conjugation.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 21c_quantum_conjugation.ipynb
python -m nbconvert --execute --inplace 21c_quantum_conjugation.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra`, `Revision/theory`, `Revision/pairing` or `Revision/lead_checks`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.
- "MemoryError" in the cells of section 7: the operators act on columns of 65536 complex numbers (1 MB each); close other programs and run the notebook again.

21.29 Notebook 21c: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/21c_quantum_conjugation.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the

cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

After *quantisation* the 16 components of the field dirac16complex are no longer
numbers but **operators** $\Psi_A$ ($A = 1, \dots, 16$), and the Revision record
fixes how they combine: on a slice of constant time $x_4$,
$$\{\Psi_A(x), \Psi^\dagger_C(y)\} = B_{AC}\,\delta^7(x - y)/\cos z,\qquad
B = -iC\gamma^{(x4)} ,$$
the **canonical anticommutator**. A charge conjugation of the quantised field must
turn $\Psi$ into a new field $\Psi' = M\Psi^{\dagger T}$ (built from the adjoint
operators, arranged as a column, times a constant matrix $M$) that obeys **the
same** rule; otherwise $\Psi'$ is not a field of the same quantum theory. This
notebook

1. derives the condition $MB^TM^\dagger = B$ and shows: it holds for
   $M = \Gamma$, it fails for $M = 1$ (that choice gives $-B$), and among all
   matrices $\cos t\,1 + \sin t\,\Gamma$ it holds only for $\pm\Gamma$
   (record check quantum_charge_conjugation_unitary_type);
2. builds the 16 field operators explicitly, as $65536 \times 65536$ operators on
   the states of 16 fermion modes, and measures all 256 anticommutators of
   $\Psi$, of $\Gamma\Psi^{\dagger T}$ and of $\Psi^{\dagger T}$;
3. shows that the valid conjugation counts the *empty* modes: its charge is
   $16 - Q$, the charge reversed up to a constant (particles become holes);
4. proves with sympy that the conjugated field obeys the field equation with the
   **mass reversed**, and that nevertheless the one-particle spectra of the masses
   $+m$ and $-m$ are identical; it reproduces the eight samples of the Revision
   pairing record (frequencies, dimensions, Krein inertia) and scans the
   extra-time momentum into the region of growing modes.

Six teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 21c (The conjugation of the quantised field: M B^T M^dagger = B) is the file
‘Revision/textbook/notebooks/21c_quantum_conjugation.ipynb‘ of the repository
Dirac_claude. It derives the condition M B^T M^dagger = B that a conjugation Psi to M
Psi^dagger T must satisfy to preserve the canonical anticommutator of the quantised field
dirac16complex (B = -i C gamma^(x4), the Krein matrix), finds that it holds for M = Gamma
and fails for M = 1 (and, among the matrices cos t + sin t Gamma, only for Gamma),
verifies this with explicit fermion operators on the 65536 states of 16 modes
(Jordan-Wigner construction in the positive representation of the Revision record), shows
that the valid conjugation turns the charge Q into 16 - Q (particles into holes), proves
with sympy that it maps the one-particle Hamiltonian of mass m to that of mass -m, and
reproduces the one-particle samples of the Revision pairing record: equal spectra for +m
and -m, Krein inertia (4,4) at real frequencies, Krein-neutral growing modes at large
```

extra-time momentum. Six teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
```

```
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 21c_quantum_conjugation.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 30 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21c_quantum_conjugation.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 21c_quantum_conjugation.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/21c.captions.json‘
- ‘Revision/textbook/figures/21c_1_krein_matrix.png‘
- ‘Revision/textbook/figures/21c_2_which_conjugation.png‘
- ‘Revision/textbook/figures/21c_3_operator_anticommutators.png‘
- ‘Revision/textbook/figures/21c_4_charge_of_states.png‘
- ‘Revision/textbook/figures/21c_5_dispersion_pm_mass.png‘
- ‘Revision/textbook/figures/21c_6_krein_inertia_scan.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the six figure files of notebook 21c exist
ALL 17 CHECKS PASSED (notebook 21c)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/21c_quantum_conjugation.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 21c_quantum_conjugation.ipynb
      python -m nbconvert --execute --inplace 21c_quantum_conjugation.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘, ‘Revision/theory‘, ‘Revision/pairing‘ or ‘Revision/lead_checks‘: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder ‘Revision/textbook/notebooks‘.
- "MemoryError" in the cells of section 7: the operators act on columns of 65536 complex numbers (1 MB each); close other programs and run the notebook again.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 21c (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 21c (The conjugation of the quantised field: M B^T M^dagger = B) is the file
# Revision/textbook/notebooks/21c_quantum_conjugation.ipynb of the repository
# Dirac_claude. It derives the condition M B^T M^dagger = B that a conjugation Psi to M
# Psi^dagger T must satisfy to preserve the canonical anticommutator of the quantised
# field dirac16complex (B = -i C gamma^(x4), the Krein matrix), finds that it holds for M
# = Gamma and fails for M = 1 (and, among the matrices cos t + sin t Gamma, only for
# Gamma), verifies this with explicit fermion operators on the 65536 states of 16 modes
# (Jordan-Wigner construction in the positive representation of the Revision record),
# shows that the valid conjugation turns the charge Q into 16 - Q (particles into holes),
# proves with sympy that it maps the one-particle Hamiltonian of mass m to that of mass
# -m, and reproduces the one-particle samples of the Revision pairing record: equal
# spectra for +m and -m, Krein inertia (4,4) at real frequencies, Krein-neutral growing
# modes at large extra-time momentum. Six teaching plots. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
```

```
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, sympy and matplotlib; the other
# packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
```

```
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
```

```
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 21c_quantum_conjugation.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 30
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 21c_quantum_conjugation.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 21c_quantum_conjugation.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/21c.captions.json
# - Revision/textbook/figures/21c_1_krein_matrix.png
# - Revision/textbook/figures/21c_2_which_conjugation.png
# - Revision/textbook/figures/21c_3_operator_anticommutators.png
# - Revision/textbook/figures/21c_4_charge_of_states.png
# - Revision/textbook/figures/21c_5_dispersion_pm_mass.png
# - Revision/textbook/figures/21c_6_krein_inertia_scan.png
#
# It changes no other file of the repository; running it headless or saving it in
```

```
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the six figure files of notebook 21c exist
#     ALL 17 CHECKS PASSED (notebook 21c)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/21c_quantum_conjugation.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 21c_quantum_conjugation.ipynb
#     python -m nbconvert --execute --inplace 21c_quantum_conjugation.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
```

```
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra, Revision/theory,
#   Revision/pairing or Revision/lead_checks: the notebook was opened outside the
#   repository, or the repository is incomplete; clone the repository again and open the
#   notebook from its folder Revision/textbook/notebooks.
# - "MemoryError" in the cells of section 7: the operators act on columns of 65536
#   complex numbers (1 MB each); close other programs and run the notebook again.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "21c"  # this notebook: chapter 21, example c


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
```

```
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
```

```
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 21c complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Operator**: a rule that turns one state of a quantum system into another; with
  a basis of the states it is a (large) matrix. Operators need not commute:
  $XY \ne YX$ in general.
- **Anticommutator** $\{X, Y\} = XY + YX$.
- **Fermion modes, occupation**: a fermion mode is either empty (0) or occupied
  (1); 16 modes have $2^{16} = 65536$ states, each a list of 16 zeros and ones.
  The **annihilation operator** $b_j$ empties mode $j$ (and gives zero if it is
  already empty), the **creation operator** $b_j^*$ fills it; they obey
  $\{b_j, b_k^*\} = \delta_{jk}$ and $\{b_j, b_k\} = 0$. The sign rule that makes
  them anticommute is the **Jordan-Wigner** rule: acting on mode $j$ gives the
  factor $(-1)$ to the power of the number of occupied modes before $j$.
- **Hilbert adjoint** $X^*$: the conjugate transpose of the matrix of $X$ in an
  ordinary (positive) inner product.
- **Canonical anticommutator**: the rule $\{\Psi_A, \Psi^\dagger_C\} = B_{AC}$
  (times the delta function) that canonical quantisation of the Lagrangian gives.
  Because $B$ has eight eigenvalues $+1$ and eight $-1$, $\Psi^\dagger$ cannot be
  the Hilbert adjoint of $\Psi$; the Revision record realises it as $\Psi^\dagger
  = b^*B$ (the **positive representation**: $\Psi_A = b_A$, $\Psi^\dagger_C =
  \sum_D b^*_D B_{DC}$).
- **Krein matrix, Krein form**: $B$, and the indefinite form $u^\dagger Bv$;
  **Krein inertia** $(p, n)$ of a space: the numbers of positive and negative
  eigenvalues of the form restricted to it; **Krein-neutral**: the form vanishes
  on the space.
```

```
- **Charge operator** $Q = \sum_{A,C}\Psi^\dagger_A B_{AC}\Psi_C$ (the integral
  of the charge density $\Psi^\dagger B\Psi$ for one point).
- **One-particle Hamiltonian** $h_m(k)$: the $16 \times 16$ matrix that gives the
  time evolution $i\,\partial_4 u = h_m(k)u$ of a plane wave $u\,e^{ik\cdot x}$ in
  flat 4+4 space; its eigenvalues are the **frequencies** $\pm w$.
- **Extra-time momentum** $k_5, k_6, k_7$: the momentum along the extra times;
  the **good sector** has $k_5 = k_6 = k_7 = 0$.
```

Text cell:

```
## 4. The physical and mathematical situation

**The condition, line by line.** Let $\Psi' = M\Psi^{\dagger T}$, that is
$\Psi'_A = \sum_B M_{AB}\Psi^\dagger_B$, with a constant matrix $M$. Its adjoint
is $\Psi'^\dagger_C = \sum_D M^*_{CD}\Psi_D$ (the adjoint of a product of a number
and an operator conjugates the number, and the adjoint of $\Psi^\dagger$ is
$\Psi$).

1. $\{\Psi'_A, \Psi'^\dagger_C\} = \sum_{B,D}M_{AB}M^*_{CD}\{\Psi^\dagger_B,
   \Psi_D\}$, because the anticommutator is linear in each slot.
2. $\{\Psi^\dagger_B, \Psi_D\} = \{\Psi_D, \Psi^\dagger_B\} = B_{DB} =
   (B^T)_{BD}$.
3. Hence $\{\Psi'_A, \Psi'^\dagger_C\} = \sum_{B,D}M_{AB}(B^T)_{BD}(M^\dagger)_{DC}
   = (MB^TM^\dagger)_{AC}$.
4. So $\Psi'$ obeys the canonical rule if and only if
   $$MB^TM^\dagger = B .$$

$B$ is Hermitian and purely imaginary, so $B^T = (B^\dagger)^* = B^* = -B$, and the
condition reads $MBM^\dagger = -B$. For $M = \Gamma$: $\Gamma B\Gamma =
-iC\Gamma\gamma^{(x4)}\Gamma = +iC\gamma^{(x4)} = -B$ (because $\Gamma$ commutes
with $C$ and anticommutes with $\gamma^{(x4)}$), so the condition **holds**. For
$M = 1$: $B = -B$ is **false**.

**Which mass?** A conjugation must also map solutions of the field equation to
solutions. The classical analysis of the charge-conjugation matrices (Notebook
21a) defines them by the intertwiner condition $M(\gamma^a)^* = s\,\gamma^a M$
(the definition of the Revision record), which is enough for this, and shows
that the only matrices obeying it are the multiples of $1$ (same mass) and of
$\Gamma$ (mass reversed). (Other matrices can also map solutions to solutions:
for $V = 0$ the projector $P_-$ does, Notebook 05c.) So for the quantised field
the only charge-conjugation matrix that keeps the canonical anticommutator gives
$\Psi \to \Gamma\Psi^{\dagger T}$ (times a phase), and it **reverses the mass**.
Section 9 checks this directly on the one-particle Hamiltonian.
```

Text cell:

```
## 5. The gammas and the Krein matrix B

The next cell reads the eight gamma matrices of the Revision record, checks that
they are eight real $16 \times 16$ matrices with the Clifford relation of
signature (4,4), builds $C$, the chirality $\Gamma$ and $B = -iC\gamma^{(x4)}$,
and reads the Revision records used below. It also checks that the record's
```

formula quantisation states the canonical anticommutator and the positive
representation exactly as this notebook uses them.

In [2]:

```
import numpy as np  # numbers, arrays, matrices
import sympy as sp  # exact algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = fixture["eta"]
gamma = [np.array(mat, dtype=np.int64) for mat in fixture["gamma"]]  # x0 .. x7
I16 = np.eye(16, dtype=np.int64)
check(len(gamma) == 8
      and all(g.shape == (16, 16) and set(np.unique(g)) <= {-1, 0, 1} for g in gamma)
      and all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                             2 * (ETA[a] if a == b else 0) * I16)
              for a in range(8) for b in range(8)),
      "eight real 16 x 16 gamma matrices with the Clifford relation, signature (4,4)")
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # C = g(x0) g(x1) g(x2) g(x3)
Gamma = C @ gamma[4] @ gamma[5] @ gamma[6] @ gamma[7]  # the chirality
B = -1j * (C @ gamma[4])  # the Krein matrix B = -i C gamma^(x4)


def load_checks(path):
    """name -> verdict (upper case) of a Revision report."""
    entries = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
    return {c["name"]: c["verdict"].upper() for c in entries}


LEAD_FILE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
LEAD = load_checks(LEAD_FILE)
PY_THEORY = load_checks("Revision/theory/reports/python-field-theory.json")
WL_PAIR = load_checks("Revision/pairing/reports/wolfram-pairing.json")
THEORY = {f["key"]: f for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
PAIRING = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
clauses = THEORY["quantisation"]["wl"].split("; ")  # the statements of the formula
rule = next(c for c in clauses if c.startswith("{Psi_A(x), Psi^dagger_C(y)}"))
positive = next(c for c in clauses if c.startswith("positive representation"))
say("record: " + rule)
say("record: " + positive)
check(rule.endswith("= B_AC delta^7(x - y)/Cos[z]")
      and positive == "positive representation chi = Psi^dagger B, "
      "{Psi_A, chi_C} = delta_AC",
      "the record's anticommutator B_AC delta / cos z and positive representation",
      record="Revision/theory/field-theory.json, formula quantisation")
```

Out [2]:

```
PASS eight real 16 x 16 gamma matrices with the Clifford relation, signature (4,4)
record: {Psi_A(x), Psi^dagger_C(y)}_(x4 = y4) = B_AC delta^7(x - y)/Cos[z]
record: positive representation chi = Psi^dagger B, {Psi_A, chi_C} = delta_AC
PASS the record's anticommutator B_AC delta / cos z and positive representation
     reproduces Revision/theory/field-theory.json, formula quantisation
```

Text cell:

```
The next cell checks the properties of $B$ that the derivation used: purely
imaginary, Hermitian, $B^2 = 1$, trace 0, so eight eigenvalues $+1$ and eight
$-1$; and $B^T = -B$. It draws $B$ (its imaginary part, since the real part is
zero) and its eigenvalues.
```

In [3]:

```
eigenvalues = np.linalg.eigvalsh(B)  # eigenvalues of a Hermitian matrix, sorted
check(np.array_equal(B.real, np.zeros((16, 16))) and np.allclose(B, B.conj().T)
      and np.allclose(B @ B, np.eye(16)) and abs(np.trace(B)) < 1e-12
      and np.allclose(eigenvalues, [-1] * 8 + [1] * 8)
      and LEAD["B_imaginary_hermitian"] == "PASS"
      and PY_THEORY["B_properties"] == "PASS",
      "B is purely imaginary, Hermitian, B^2 = 1, trace 0: signature (8,8)",
      record=f"{LEAD_FILE}, check B_imaginary_hermitian")
check(np.allclose(B.T, -B), "B^T = -B")

from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.3), width_ratios=[1, 1.3])
image = axes[0].imshow(B.imag, cmap=SIGNS, vmin=-1, vmax=1)
axes[0].set_title("imaginary part of $B = -iC\\gamma^{(x4)}$")
axes[0].set_xticks([0, 7, 15], ["1", "8", "16"])
axes[0].set_yticks([0, 7, 15], ["1", "8", "16"])
axes[0].set_xlabel("column")
axes[0].set_ylabel("row")
axes[0].grid(False)
fig.colorbar(image, ax=axes[0], ticks=[-1, 0, 1], shrink=0.8)
axes[1].bar(np.arange(1, 17), eigenvalues,
            color=["#2a78d6"] * 8 + ["#e34948"] * 8)
axes[1].set_xlabel("eigenvalue number (sorted)")
axes[1].set_ylabel("eigenvalue of $B$")
axes[1].set_title("eight eigenvalues $-1$, eight $+1$")
save_figure(fig, "krein_matrix",
            "The Krein matrix $B = -iC\\gamma^{(x4)}$ of the canonical "
            "anticommutator. Left: its imaginary part as a heat map (its real part "
            "is zero; horizontal axis the column, vertical axis the row, red $+1$, "
            "blue $-1$); every row and column has one entry, and the nonzero "
            "entries join the two chiral halves. Right: its sixteen eigenvalues in "
            "increasing order, eight $-1$ and eight $+1$ (signature (8,8)); "
            "because of the eight negative eigenvalues the field adjoint cannot be "
            "an ordinary Hilbert adjoint.")
```

Out [3]:

```
PASS B is purely imaginary, Hermitian, B^2 = 1, trace 0: signature (8,8)
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    B_imaginary_hermitian
PASS B^T = -B
```

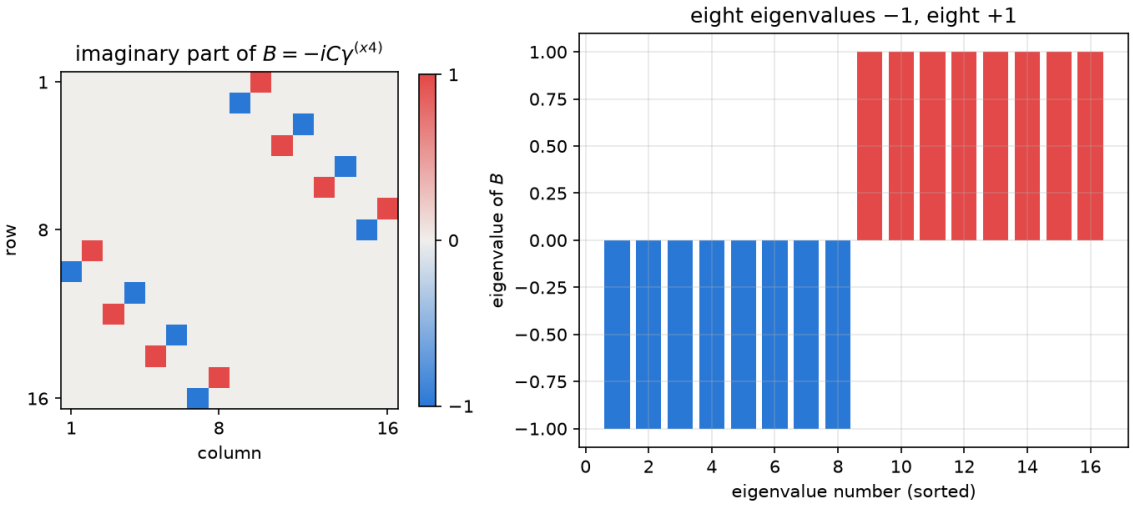

Figure 537: The Krein matrix $B = -iC\gamma^{(x4)}$ of the canonical anticommutator. Left: its imaginary part as a heat map (its real part is zero; horizontal axis the column, vertical axis the row, red $+1$, blue -1); every row and column has one entry, and the nonzero entries join the two chiral halves. Right: its sixteen eigenvalues in increasing order, eight -1 and eight $+1$ (signature (8,8)); because of the eight negative eigenvalues the field adjoint cannot be an ordinary Hilbert adjoint. (Notebook 21c, figure 1.)

Out [3]:

```
Figure 21c.1 saved as Revision/textbook/figures/21c_1_krein_matrix.png
```

Text cell:

```
## 6. Which conjugation keeps the anticommutator?

The next cell computes $MB^TM^\dagger$ for $M = \Gamma$ and $M = 1$ and compares
with the Revision record (check quantum_charge_conjugation_unitary_type). Then it
looks at the whole family $M(t) = \cos t\,1 + \sin t\,\Gamma$ for $t$ from $0$ to
$2\pi$ and computes the size of the mismatch $\|M(t)B^TM(t)^\dagger - B\|$ (the
square root of the sum of the squared moduli of the 256 entries): it vanishes
only at $t = \pi/2$ and $t = 3\pi/2$, where $M = \pm\Gamma$. Finally it checks
that a multiple $c\,\Gamma$ works exactly when $|c| = 1$ (a phase).
```

In [4]:

```
def mismatch(M):
    """|| M B^T M^dagger - B ||: zero when M keeps the canonical anticommutator."""
    return np.linalg.norm(M @ B.T @ M.conj().T - B)


q_gamma = Gamma @ B.T @ Gamma.T  # M = Gamma (real, so M^dagger = M^T)
q_one = B.T  # M = 1
check(np.allclose(q_gamma, B) and np.allclose(q_one, -B)
      and LEAD["quantum_charge_conjugation_unitary_type"] == "PASS",
      "M B^T M^dagger = +B for M = Gamma and = -B for M = 1",
      record=f"{LEAD_FILE}, check quantum_charge_conjugation_unitary_type")

t_values = np.linspace(0.0, 2 * np.pi, 721)  # steps of half a degree
family = [np.cos(t) * np.eye(16) + np.sin(t) * Gamma for t in t_values]
mismatches = np.array([mismatch(M) for M in family])
zeros = t_values[mismatches < 1e-12]
say("t where the mismatch vanishes (in units of pi): "
    + ", ".join(f"{t / np.pi:.3f}" for t in zeros))
phases_ok = all(abs(mismatch(c * Gamma)) < 1e-12 for c in (1j, np.exp(0.3j), -1))
scales_bad = all(mismatch(c * Gamma) > 1 for c in (0.5, 2.0))
check(np.allclose(zeros / np.pi, [0.5, 1.5]) and phases_ok and scales_bad,
      "in the family cos t + sin t Gamma only t = pi/2, 3pi/2 work; c Gamma works "
      "exactly for |c| = 1")

fig, ax = plt.subplots(figsize=(8.0, 4.3))
ax.plot(t_values / np.pi, mismatches)
ax.plot([0.5, 1.5], [0, 0], "o", color="#e34948", markersize=9,
        label="$M = \\pm\\Gamma$: mismatch zero")
ax.plot([0.0, 1.0, 2.0], [mismatch(np.eye(16))] * 3, "s", color="#2a78d6",
        label="$M = \\pm 1$: mismatch $\\|{-B} - B\\| = 8$")
ax.set_xlabel("$t / \\pi$ in $M(t) = \\cos t\\,1 + \\sin t\\,\\Gamma$")
ax.set_ylabel("$\\|M B^T M^\\dagger - B\\|$")
ax.set_title("Only $\\pm\\Gamma$ keeps the canonical anticommutator")
ax.legend(fontsize=8)
save_figure(fig, "which_conjugation",
            "The mismatch $\\|M B^T M^\\dagger - B\\|$ (the size of the failure of "
            "the canonical anticommutator) for the conjugations $\\Psi \\to "
            "M\\Psi^{\\dagger T}$ with $M = \\cos t\\,1 + \\sin t\\,\\Gamma$; "
            "horizontal axis $t/\\pi$ from $0$ to $2$, vertical axis the mismatch "
            "(pure number). It vanishes only at $t = \\pi/2$ and $3\\pi/2$, that is "
            "for $M = \\pm\\Gamma$ (red dots); the same-mass choice $M = \\pm 1$ "
            "(blue squares) gives $-B$ instead of $B$, a mismatch of $\\|2B\\| = 8$.")
```

Out [4]:

```
PASS M B^T M^dagger = +B for M = Gamma and = -B for M = 1
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    quantum_charge_conjugation_unitary_type
t where the mismatch vanishes (in units of pi): 0.500, 1.500
PASS in the family cos t + sin t Gamma only t = pi/2, 3pi/2 work; c Gamma works exactly
    for |c| = 1
```

Out [4]:

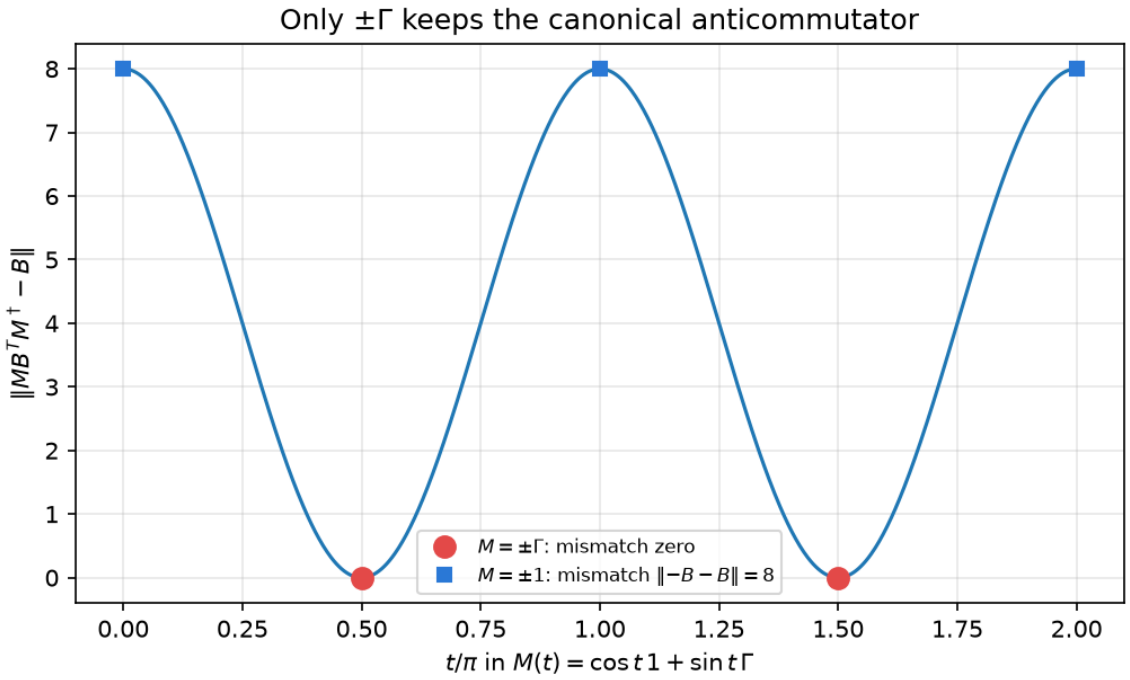

Figure 538: The mismatch $\|MB^TM^\dagger - B\|$ (the size of the failure of the canonical anticommutator) for the conjugations $\Psi \to M\Psi^{\dagger T}$ with $M = \cos t\, 1 + \sin t\, \Gamma$; horizontal axis t/π from 0 to 2, vertical axis the mismatch (pure number). It vanishes only at $t = \pi/2$ and $3\pi/2$, that is for $M = \pm\Gamma$ (red dots); the same-mass choice $M = \pm 1$ (blue squares) gives $-B$ instead of B, a mismatch of $\|2B\| = 8$. (Notebook 21c, figure 2.)

```
Figure 21c.2 saved as Revision/textbook/figures/21c_2_which_conjugation.png
```

Text cell:

```
## 7. The anticommutators with explicit operators

The next cell builds 16 fermion modes on a computer. A state of the 16 modes is a
column of $2^{16} = 65536$ complex numbers, one for each occupation pattern; the
pattern of basis state number $n$ is the binary digits of $n$ (bit $j$ is the
occupation of mode $j$). The annihilation operator $b_j$ moves the amplitude of
each pattern with bit $j$ set to the pattern with bit $j$ cleared, multiplied by
$(-1)^{(\text{number of occupied modes below } j)}$; the creation operator $b_j^*$
does the reverse. Instead of storing $65536 \times 65536$ matrices, the cell stores
these index lists and applies the operators to columns.

Then the field operators of the positive representation of the Revision record:
$\Psi_A = b_A$ and $\Psi^\dagger_C = \sum_D b_D^*B_{DC}$. Every column of $B$ has
exactly one nonzero entry, so $\Psi^\dagger_C$ is a single creation operator times
$\pm i$. The cell first checks the basic rules $\{b_j, b_k^*\} = \delta_{jk}$ for a
few pairs.
```

In [5]:

```
MODES, STATES = 16, 2 ** 16
index = np.arange(STATES)  # basis state numbers 0 .. 65535
occupied_count = np.zeros(STATES, dtype=np.int64)  # number of occupied modes
for j in range(MODES):
    occupied_count += (index >> j) & 1  # >> shifts the bits; & 1 reads bit j
OPS = []  # for each mode j: the states where j is occupied, and the sign factors
for j in range(MODES):
    filled = ((index >> j) & 1).astype(bool)
    below = index & ((1 << j) - 1)  # the bits of the modes below j
    parity = np.zeros(STATES, dtype=np.int64)
    for i in range(j):
        parity += (below >> i) & 1
    OPS.append((index[filled], index[~filled], np.where(parity % 2 == 0, 1.0, -1.0)))


def annihilate(j, v):
    """b_j v: empty mode j (with the Jordan-Wigner sign)."""
    filled, _, sign = OPS[j]
    out = np.zeros_like(v)
    out[filled ^ (1 << j)] = sign[filled] * v[filled]
    return out


def create(j, v):
    """b_j^* v: fill mode j (with the Jordan-Wigner sign)."""
    _, empty, sign = OPS[j]
    out = np.zeros_like(v)
    out[empty | (1 << j)] = sign[empty] * v[empty]
    return out
```

```
rng = np.random.default_rng(2103)  # fixed seed: the same state in every run
v = rng.normal(size=STATES) + 1j * rng.normal(size=STATES)
v /= np.linalg.norm(v)  # a random normalised state of the 16 modes
basic = all(np.allclose(annihilate(j, create(k, v)) + create(k, annihilate(j, v)),
                        (1.0 if j == k else 0.0) * v)
            for j, k in [(0, 0), (3, 3), (15, 15), (0, 1), (4, 9), (15, 2)])
check(basic, "Jordan-Wigner operators: {b_j, b_k^*} = delta_jk on a random state")
```

Out [5]:

```
PASS Jordan-Wigner operators: {b_j, b_k^*} = delta_jk on a random state
```

Text cell:

```
The next cell defines the three fields whose anticommutators we compare: the
field $\Psi$ itself, the valid conjugate $\Psi' = \Gamma\Psi^{\dagger T}$ and the
same-mass candidate $\Psi'' = \Psi^{\dagger T}$ ($M = 1$). For each pair $(A, C)$
it applies $\{X, Y\} = XY + YX$ to the random state $v$, reads off the number
$N_{AC} = v^\dagger\{X, Y\}v$, and checks that $\{X, Y\}v = N_{AC}v$ (the
anticommutator acts as a number). It also checks $\{\Psi_A, \Psi_C\} = 0$. (About
10 seconds: 1024 anticommutators of operators on 65536 states.)
```

In [6]:

```
B_column = [int(np.flatnonzero(B[:, c])[0]) for c in range(16)]  # row of the entry


def psi(A, u):
    """Psi_A = b_A."""
    return annihilate(A, u)


def psi_dagger(Cc, u):
    """Psi^dagger_C = sum_D b_D^* B_DC (a single term)."""
    D = B_column[Cc]
    return B[D, Cc] * create(D, u)


def field_pair(M):
    """For the conjugate M Psi^{dagger T} with diagonal M: the maps u -> Psi'_A u
    and u -> Psi'^dagger_C u (Psi'_A = M_AA Psi^dagger_A, Psi'^dagger_C =
    conj(M_CC) Psi_C)."""
    d = np.diag(M)
    return (lambda A, u: d[A] * psi_dagger(A, u),
            lambda Cc, u: np.conj(d[Cc]) * psi(Cc, u))


def anticommutator_matrix(first, second):
    """N_AC with {first_A, second_C} v = N_AC v, and whether it acts as a number."""
```

```
    N = np.zeros((16, 16), dtype=complex)
    number_like = True
    for A in range(16):
        for Cc in range(16):
            w = first(A, second(Cc, v)) + second(Cc, first(A, v))
            N[A, Cc] = np.vdot(v, w)  # v^dagger w
            number_like &= np.allclose(w, N[A, Cc] * v)
    return N, number_like


N_psi, ok_psi = anticommutator_matrix(psi, psi_dagger)
N_same, ok_same = anticommutator_matrix(psi, psi)  # {Psi_A, Psi_C}
N_gamma, ok_gamma = anticommutator_matrix(*field_pair(Gamma))
N_one, ok_one = anticommutator_matrix(*field_pair(np.eye(16)))
check(ok_psi and ok_same and np.allclose(N_psi, B) and np.allclose(N_same, 0)
      and PY_THEORY["canonical_anticommutator_B"] == "PASS",
      "operators: {Psi_A, Psi^dagger_C} = B_AC and {Psi_A, Psi_C} = 0 (256 pairs)",
      record="Revision/theory/reports/python-field-theory.json, check "
      "canonical_anticommutator_B")
check(ok_gamma and ok_one and np.allclose(N_gamma, B) and np.allclose(N_one, -B),
      "operators: Gamma Psi^{dagger T} has +B, Psi^{dagger T} has -B")
```

Out [6]:

```
PASS operators: {Psi_A, Psi^dagger_C} = B_AC and {Psi_A, Psi_C} = 0 (256 pairs)
     reproduces Revision/theory/reports/python-field-theory.json, check
    canonical_anticommutator_B
PASS operators: Gamma Psi^{dagger T} has +B, Psi^{dagger T} has -B
```

Text cell:

```
The next cell draws the three measured $16 \times 16$ matrices $N_{AC}$ (their
imaginary parts; the real parts are zero).
```

In [7]:

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2))
for ax, N, title in ((axes[0], N_psi, "$\\{\\Psi_A, \\Psi^\\dagger_C\\}$"),
                     (axes[1], N_gamma, "$\\Psi' = \\Gamma\\Psi^{\\dagger T}$"),
                     (axes[2], N_one, "$\\Psi'' = \\Psi^{\\dagger T}$")):
    image = ax.imshow(N.imag, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("$C$")
    ax.grid(False)
axes[0].set_ylabel("$A$")
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8,
             label="imaginary part of the anticommutator")
save_figure(fig, "operator_anticommutators",
            "The anticommutators $\\{X_A, Y^\\dagger_C\\}$ measured with explicit "
```

```
"operators on the 65536 states of 16 fermion modes (imaginary parts; "
"the real parts are zero; horizontal axis $C$, vertical axis $A$, red "
"$+1$, blue $-1$). Left: the field $\\Psi$ of the positive "
"representation, which gives the Krein matrix $B$. Middle: the "
"conjugate $\\Gamma\\Psi^{\\dagger T}$, which gives $B$ again, so it is "
"a field of the same quantum theory. Right: the same-mass candidate "
"$\\Psi^{\\dagger T}$, which gives $-B$ (every colour reversed), so it is "
"not.")
```

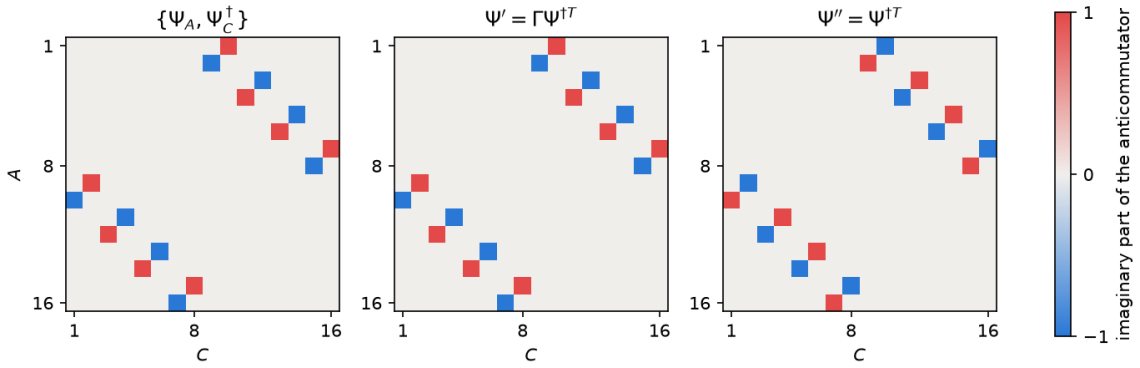

Figure 539: The anticommutators $\{X_A, Y_C^\dagger\}$ measured with explicit operators on the 65536 states of 16 fermion modes (imaginary parts; the real parts are zero; horizontal axis C, vertical axis A, red $+1$, blue -1). Left: the field Ψ of the positive representation, which gives the Krein matrix B. Middle: the conjugate $\Gamma\Psi^{\dagger T}$, which gives B again, so it is a field of the same quantum theory. Right: the same-mass candidate $\Psi^{\dagger T}$, which gives $-B$ (every colour reversed), so it is not. (Notebook 21c, figure 3.)

Out [7]:

```
Figure 21c.3 saved as Revision/textbook/figures/21c_3_operator_anticommutators.png
```

Text cell:

```
## 8. The valid conjugation counts the empty modes

The charge operator of the field is $Q = \sum_{A,C}\Psi^\dagger_AB_{AC}\Psi_C$. In
the positive representation $\Psi^\dagger B\Psi = b^*B\,B\,b = b^*b$ (since
$B^2 = 1$): $Q$ is the **number of occupied modes**. For the conjugated field
$\Psi' = \Gamma\Psi^{\dagger T}$, line by line:

1. $Q' = \sum_{A,C}\Psi'^\dagger_AB_{AC}\Psi'_C = \sum_{A,C}\Gamma_{AA}\Gamma_{CC}
   B_{AC}\,\Psi_A\Psi^\dagger_C = \sum_{A,C}(\Gamma B\Gamma)_{AC}\Psi_A
   \Psi^\dagger_C$ ($\Gamma$ is diagonal and real).
2. $\Gamma B\Gamma = -B$, so $Q' = -\sum B_{AC}\Psi_A\Psi^\dagger_C$.
3. The anticommutator gives $\Psi_A\Psi^\dagger_C = B_{AC} - \Psi^\dagger_C
   \Psi_A$, so $Q' = -\sum_{A,C}B_{AC}^2 + \sum_{A,C}\Psi^\dagger_C(B^T)_{CA}
   \Psi_A$.
4. $\sum B_{AC}^2 = \mathrm{tr}(BB^T) = -\mathrm{tr}(B^2) = -16$ and $B^T = -B$,
```

```
   so $Q' = 16 - Q$.

The conjugate counts the **empty** modes: up to the constant 16 (which does not
change any difference of charges), it **reverses the charge**, $Q' - 8 = -(Q -
8)$, as particles and holes are exchanged. The same steps with $M = 1$ give
$Q'' = Q - 16$: the invalid candidate would not even reverse the charge. The next
cell checks the three operator identities on the random state and draws the
charges of the 17 kinds of basis states (0 to 16 occupied modes).
```

In [8]:

```
def charge(first_dagger, second, u):
    """sum_AC first_dagger_A B_AC second_C u (only the 16 nonzero B_AC)."""
    out = np.zeros_like(u)
    for Cc in range(16):
        A = B_column[Cc]  # the row of the nonzero entry of column C
        out += first_dagger(A, B[A, Cc] * second(Cc, u))
    return out


conj_g, conj_g_dagger = field_pair(Gamma)
conj_1, conj_1_dagger = field_pair(np.eye(16))
Q_v = charge(psi_dagger, psi, v)
Qp_v = charge(conj_g_dagger, conj_g, v)
Qpp_v = charge(conj_1_dagger, conj_1, v)
check(np.allclose(Q_v, occupied_count * v)
      and np.allclose(Qp_v, (16 - occupied_count) * v)
      and np.allclose(Qpp_v, (occupied_count - 16) * v),
      "Q = number of occupied modes, Q' = 16 - Q (Gamma), Q'' = Q - 16 (M = 1)")

n = np.arange(17)
fig, ax = plt.subplots(figsize=(8.0, 4.4))
ax.plot(n, n - 8, "o-", label="$Q - 8$ of the field $\\Psi$")
ax.plot(n, (16 - n) - 8, "s-", label="$Q' - 8$ of $\\Gamma\\Psi^{\\dagger T}$ "
        "(valid)")
ax.plot(n, (n - 16) + 8, "^--", color="#999999",
        label="$Q'' + 8$ of $\\Psi^{\\dagger T}$ (invalid)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("number of occupied modes in the basis state")
ax.set_ylabel("charge (shifted by the constant 8)")
ax.set_title("The valid conjugation reverses the charge")
ax.legend(fontsize=8)
save_figure(fig, "charge_of_states",
            "The charge of the basis states of 16 fermion modes, measured by three "
            "fields; horizontal axis the number of occupied modes (0 to 16), "
            "vertical axis the charge shifted by a constant (minus 8 for $\\Psi$ "
            "and $\\Gamma\\Psi^{\\dagger T}$, plus 8 for $\\Psi^{\\dagger T}$; "
            "pure numbers). "
            "The field $\\Psi$ counts the occupied modes; the valid conjugate "
            "$\\Gamma\\Psi^{\\dagger T}$ counts the empty ones, $Q' = 16 - Q$, so "
            "the shifted charge is exactly reversed (particles and holes "
            "exchanged); the invalid candidate $\\Psi^{\\dagger T}$ would give "
            "$Q - 16$, not a reversal.")
```

Out [8]:

```
PASS Q = number of occupied modes, Q' = 16 - Q (Gamma), Q'' = Q - 16 (M = 1)
```

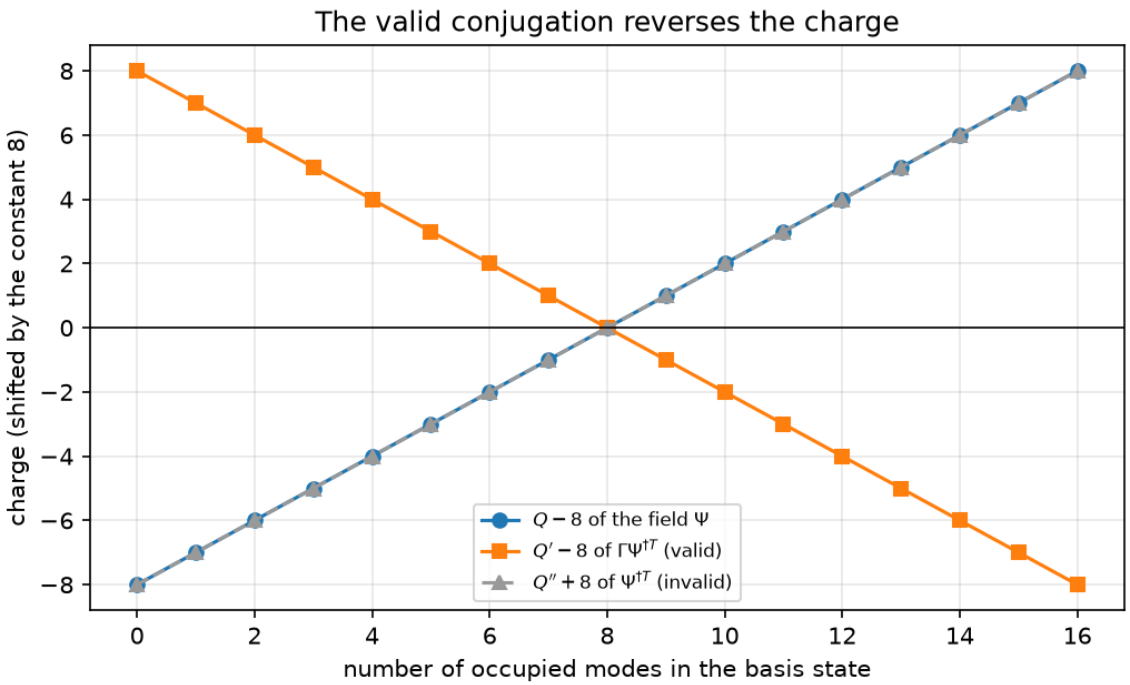

Figure 540: The charge of the basis states of 16 fermion modes, measured by three fields; horizontal axis the number of occupied modes (0 to 16), vertical axis the charge shifted by a constant (minus 8 for Ψ and $\Gamma\Psi^{\dagger T}$, plus 8 for $\Psi^{\dagger T}$; pure numbers). The field Ψ counts the occupied modes; the valid conjugate $\Gamma\Psi^{\dagger T}$ counts the empty ones, $Q' = 16 - Q$, so the shifted charge is exactly reversed (particles and holes exchanged); the invalid candidate $\Psi^{\dagger T}$ would give $Q - 16$, not a reversal. (Notebook 21c, figure 4.)

Out [8]:

```
Figure 21c.4 saved as Revision/textbook/figures/21c_4_charge_of_states.png
```

Text cell:

```
## 9. The mass is reversed, the spectra are equal

In flat 4+4 space a plane wave $u\,e^{ik\cdot x}$ ($k$ the momenta $k_0, k_1, k_2,
k_3, k_5, k_6, k_7$ along the directions other than the time) evolves by
$i\,\partial_4u = h_m(k)u$ with the one-particle Hamiltonian of the Revision
pairing record
$$h_m(k) = -im\gamma^{(x4)} - \gamma^{(x4)}\sum_{a \ne x4}k_a\gamma^a .$$
The conjugated field contains $\Gamma u^*$: taking the complex conjugate of the
evolution equation gives $i\,\partial_4u^* = -h_m(k)^*u^*$, and multiplying by
$\Gamma$ gives $i\,\partial_4(\Gamma u^*) = -\Gamma h_m(k)^*\Gamma\,(\Gamma u^*)$
```

```
(since $\Gamma^2 = 1$). The next cell proves with sympy, for symbolic $m$ and $k$:

1. $-\Gamma h_m(k)^*\Gamma = h_{-m}(-k)$: the conjugated mode is a mode of the
   theory with the **reversed mass** (and the reversed momentum, as $e^{-ik\cdot
   x}$ says);
2. $\Gamma h_m(k)\Gamma = h_{-m}(k)$: the two Hamiltonians are *similar*, so the
   one-particle spectra of the masses $+m$ and $-m$ are **identical**, not opposite
   (record check Q_one_particle_maps);
3. $h_m(k)^2 = w^2\,1$ with $w^2 = m^2 + k_0^2 + k_1^2 + k_2^2 + k_3^2 - k_5^2 -
   k_6^2 - k_7^2$ (record check Q_one_particle_flat_dispersion): the frequencies
   are $\pm w$;
4. $Bh_m = h_m^\dagger B$ for real $m$ and $k$: $h_m$ is self-adjoint for the Krein
   form (Revision check mode_hamiltonian_B_selfadjoint_dispersion).
```

In [9]:

```
m = sp.symbols("m", real=True)
k = sp.symbols("k0:8", real=True)  # k[a] along xa; k[4] (the x4 entry) is not used
G = [sp.Matrix(g.tolist()) for g in gamma]
Gamma_s, B_s = sp.Matrix(Gamma.tolist()), -sp.I * sp.Matrix(C.tolist()) * G[4]


def h(mass, momenta):
    """h_m(k) = -i m gamma^(x4) - gamma^(x4) sum_(a != x4) k_a gamma^a."""
    total = -sp.I * mass * G[4]
    for a in range(8):
        if a != 4:
            total -= momenta[a] * G[4] * G[a]
    return total


zero = sp.zeros(16, 16)
h_m = h(m, k)
minus_k = [-q for q in k]
w2 = m ** 2 + k[0] ** 2 + k[1] ** 2 + k[2] ** 2 + k[3] ** 2 - k[5] ** 2 - k[6] ** 2 \
    - k[7] ** 2
check((-Gamma_s * h_m.conjugate() * Gamma_s - h(-m, minus_k)).expand() == zero,
      "-Gamma conj(h_m(k)) Gamma = h_(-m)(-k): the conjugated mode has the reversed "
      "mass")
check((Gamma_s * h_m * Gamma_s - h(-m, k)).expand() == zero
      and WL_PAIR["Q_one_particle_maps"] == "PASS",
      "Gamma h_m(k) Gamma = h_(-m)(k): equal spectra for +m and -m",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_maps")
check((h_m * h_m - w2 * sp.eye(16)).expand() == zero
      and (B_s * h_m - h_m.H * B_s).expand() == zero
      and WL_PAIR["Q_one_particle_flat_dispersion"] == "PASS"
      and PY_THEORY["mode_hamiltonian_B_selfadjoint_dispersion"] == "PASS",
      "h_m(k)^2 = w^2 1 and B h_m = h_m^dagger B",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_flat_dispersion")
```

Out [9]:

```
PASS -Gamma conj(h_m(k)) Gamma = h_(-m)(-k): the conjugated mode has the reversed mass
PASS Gamma h_m(k) Gamma = h_(-m)(k): equal spectra for +m and -m
     reproduces Revision/pairing/reports/wolfram-pairing.json, check Q_one_particle_maps
PASS h_m(k)^2 = w^2 1 and B h_m = h_m^dagger B
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_one_particle_flat_dispersion
```

Text cell:

```
**The eight samples of the pairing record.** For each sample $(m, k)$ of the record
(‘Revision/pairing/pairing-theory.json‘, data one_particle_flat) the next cell
computes numerically the frequency $w$, the dimensions of the eigenspaces of $+w$
and $-w$, and the **Krein inertia** of each: the numbers of positive and negative
eigenvalues of the matrix $V^\dagger BV$, where the columns of $V$ are an
orthonormal basis of the eigenspace (the eigenspace of $+w$ is the range of the
projector $P_+ = \tfrac12(1 + h_m/w)$). It compares every number with the record
(check Q_one_particle_Krein_signatures).
```

In [10]:

```
def numeric_h(mass, momenta):
    """h_m(k) as a numpy array."""
    total = -1j * mass * gamma[4].astype(complex)
    for a in range(8):
        if a != 4:
            total = total - momenta[a] * (gamma[4] @ gamma[a])
    return total


def eigenspace_facts(H_mat, w):
    """Dimension and Krein inertia (positive, negative) of the eigenspace of w."""
    P = (np.eye(16) + H_mat / w) / 2  # projector onto the eigenvalue +w (w != 0)
    U_, s_, _ = np.linalg.svd(P)
    V = U_[:, s_ > 1e-9]  # orthonormal basis of the range of P
    form = np.linalg.eigvalsh(V.conj().T @ B @ V)
    return V.shape[1], (int(np.sum(form > 1e-9)), int(np.sum(form < -1e-9)))


rows_ok = True
for sample in PAIRING["data"]["one_particle_flat"]["samples"]:
    H_mat = numeric_h(sample["m"], sample["k"])
    w = float(sp.sympify(sample["w"].replace("Sqrt[", "sqrt(").replace("]", ")")))
    w_here = np.sqrt(np.max(np.linalg.eigvals(H_mat @ H_mat).real))
    dim_p, inertia_p = eigenspace_facts(H_mat, w_here)
    dim_m, inertia_m = eigenspace_facts(H_mat, -w_here)
    same = (abs(w_here - w) < 1e-12 and dim_p == sample["dim_plus_w"]
            and dim_m == sample["dim_minus_w"]
            and list(inertia_p) == sample["B_inertia_plus_w"]
            and list(inertia_m) == sample["B_inertia_minus_w"])
    rows_ok &= same
    k_text = ",".join(str(q) for q in sample["k"])
    say(f"m = {sample['m']:+d}, k = ({k_text}): w = {w_here:.6f}, dims {dim_p} "
```

```
        f"{dim_m}, inertia ({inertia_p[0]},{inertia_p[1]}) "
        f"({inertia_m[0]},{inertia_m[1]}), same: {same}")
check(rows_ok and WL_PAIR["Q_one_particle_Krein_signatures"] == "PASS",
      "all eight one-particle samples of the pairing record reproduced",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_Krein_signatures")
```

Out [10]:

```
m = +2, k = (4,1,2,0,0,0,0,0): w = 5.000000, dims 8 8, inertia (4,4) (4,4), same: True
m = -2, k = (4,1,2,0,0,0,0,0): w = 5.000000, dims 8 8, inertia (4,4) (4,4), same: True
m = +3, k = (0,0,0,0,0,0,0,0): w = 3.000000, dims 8 8, inertia (4,4) (4,4), same: True
m = -3, k = (0,0,0,0,0,0,0,0): w = 3.000000, dims 8 8, inertia (4,4) (4,4), same: True
m = +1, k = (0,1,0,0,0,0,0,0): w = 1.414214, dims 8 8, inertia (4,4) (4,4), same: True
m = -1, k = (0,1,0,0,0,0,0,0): w = 1.414214, dims 8 8, inertia (4,4) (4,4), same: True
m = +2, k = (0,0,0,0,0,1,0,0): w = 1.732051, dims 8 8, inertia (4,4) (4,4), same: True
m = -2, k = (0,0,0,0,0,1,0,0): w = 1.732051, dims 8 8, inertia (4,4) (4,4), same: True
PASS all eight one-particle samples of the pairing record reproduced
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_one_particle_Krein_signatures
```

Text cell:

```
**Scans.** The next cell computes the sixteen eigenvalues of $h_{+m}(k)$ and of
$h_{-m}(k)$ for $m = 1$ along two lines of momenta: along ordinary space ($k_1$
from $-3$ to $3$, all other momenta zero), where $w = \sqrt{1 + k_1^2}$ is real;
and along an extra time ($k_5$ from $-3$ to $3$), where $w^2 = 1 - k_5^2$ becomes
negative for $|k_5| > 1$: the frequencies become imaginary, $\pm i\sqrt{k_5^2 -
1}$, and the modes grow exponentially in time (the ill-posedness of the extra
times). It checks that the spectra of $+m$ and $-m$ agree at every point.

One warning about the two points $k_5 = \pm1$ of the scan. There $w = 0$ and
$h_m^2 = 0$: every eigenvalue is exactly 0, but $h_m$ itself is not the zero
matrix, and such a matrix cannot be brought to diagonal form. For a matrix of this
kind the computer's eigenvalues are accurate only to about the square root of the
rounding unit, about $10^{-8}$, instead of about $10^{-15}$. The function
`spectrum` rounds the eigenvalues to 9 decimals (so that equal values sort in the
same order); the cell compares the two spectra to $10^{-8}$ at all other points
and checks separately that at $k_5 = \pm1$ all 32 computed eigenvalues are
smaller than $10^{-6}$.
```

In [11]:

```
def spectrum(mass, k1=0.0, k5=0.0):
    """The 16 eigenvalues of h_m with momenta k1 (x1) and k5 (x5), sorted."""
    momenta = [0, k1, 0, 0, 0, k5, 0, 0]  # k0, ..., k7
    values = np.linalg.eigvals(numeric_h(mass, momenta))
    return np.sort_complex(np.round(values, 9))


scan = np.linspace(-3.0, 3.0, 241)  # steps of 0.025; contains k5 = -1 and +1
```

```
spec_k1 = {s: np.array([spectrum(s, k1=q) for q in scan]) for s in (1, -1)}
spec_k5 = {s: np.array([spectrum(s, k5=q) for q in scan]) for s in (1, -1)}
exceptional = np.abs(np.abs(scan) - 1.0) < 1e-9  # the two points k5 = -1, +1
check(np.allclose(spec_k1[1], spec_k1[-1], rtol=0, atol=1e-8)
      and np.allclose(spec_k5[1][~exceptional], spec_k5[-1][~exceptional],
                      rtol=0, atol=1e-8)
      and np.abs(spec_k5[1][exceptional]).max() < 1e-6
      and np.abs(spec_k5[-1][exceptional]).max() < 1e-6
      and np.allclose(np.abs(spec_k1[1].real).max(axis=1), np.sqrt(1 + scan ** 2)),
      "scans: identical spectra for m = +1 and m = -1; w = sqrt(1 + k1^2)")

fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for s, style, label in ((1, "-", "$m = +1$"), (-1, "--", "$m = -1$")):
    axes[0].plot(scan, spec_k1[s].real.max(axis=1), style, color="#e34948",
                 label=f"$+w$, {label}")
    axes[0].plot(scan, spec_k1[s].real.min(axis=1), style, color="#2a78d6",
                 label=f"$-w$, {label}")
    axes[1].plot(scan, spec_k5[s].real.max(axis=1), style, color="#e34948",
                 label=f"largest real part, {label}")
    axes[1].plot(scan, spec_k5[s].imag.max(axis=1), style, color="#2ca02c",
                 label=f"largest imaginary part, {label}")
axes[0].set_xlabel("momentum $k_1$ along ordinary space")
axes[0].set_ylabel("frequency (eigenvalue of $h_m$)")
axes[0].set_title("Good sector: real frequencies $\\pm\\sqrt{m^2 + k_1^2}$")
axes[0].legend(fontsize=7)
axes[1].set_xlabel("momentum $k_5$ along the extra time $x5$")
axes[1].set_ylabel("largest real or imaginary part of the eigenvalues")
axes[1].set_title("Extra-time momentum: imaginary beyond $|k_5| = |m|$")
axes[1].legend(fontsize=7)
save_figure(fig, "dispersion_pm_mass",
            "One-particle frequencies of the flat 4+4 Hamiltonian $h_m(k)$ for "
            "$m = +1$ (solid) and $m = -1$ (dashed, on top of the solid lines). "
            "Left: versus the momentum $k_1$ along ordinary space; the frequencies "
            "$\\pm\\sqrt{1 + k_1^2}$ are real and the same for both masses. Right: "
            "versus the momentum $k_5$ along an extra time; for $|k_5| < 1$ the "
            "largest real part is $\\sqrt{1 - k_5^2}$, for $|k_5| > 1$ the "
            "frequencies are imaginary, $\\pm i\\sqrt{k_5^2 - 1}$ (green), and the "
            "modes grow in time. Horizontal axes the momentum, vertical axes the "
            "frequency (pure numbers, units with $m = 1$).")
```

Out [11]:

```
PASS scans: identical spectra for m = +1 and m = -1; w = sqrt(1 + k1^2)
```

Out [11]:

```
Figure 21c.5 saved as Revision/textbook/figures/21c_5_dispersion_pm_mass.png
```

Text cell:

```
**Krein inertia along the extra-time scan.** For every real frequency the
eigenspace of $+w$ has dimension 8 and Krein inertia (4,4): the form
```

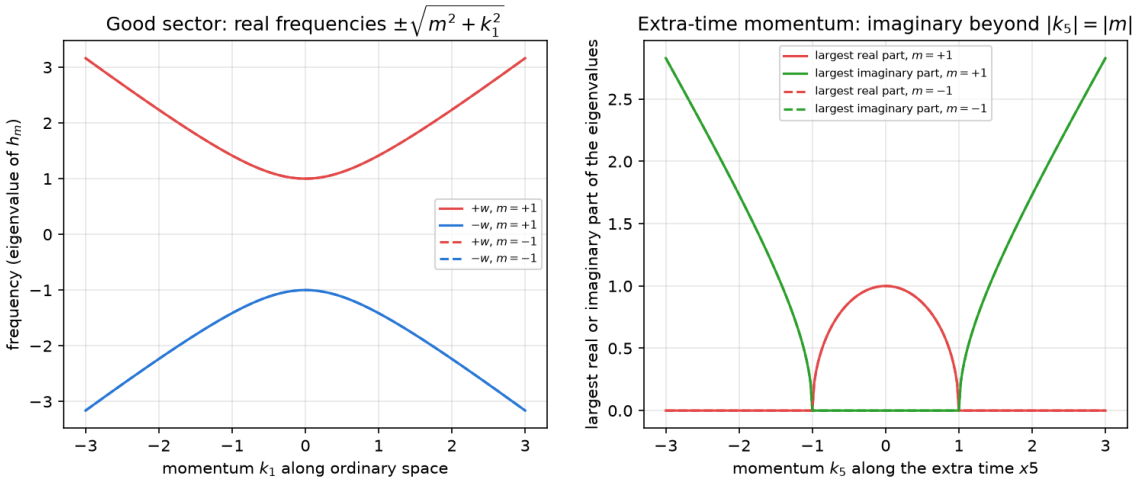

Figure 541: One-particle frequencies of the flat 4+4 Hamiltonian $h_m(k)$ for $m = +1$ (solid) and $m = -1$ (dashed, on top of the solid lines). Left: versus the momentum k_1 along ordinary space; the frequencies $\pm\sqrt{1+k_1^2}$ are real and the same for both masses. Right: versus the momentum k_5 along an extra time; for $|k_5| < 1$ the largest real part is $\sqrt{1-k_5^2}$, for $|k_5| > 1$ the frequencies are imaginary, $\pm i\sqrt{k_5^2-1}$ (green), and the modes grow in time. Horizontal axes the momentum, vertical axes the frequency (pure numbers, units with $m = 1$). (Notebook 21c, figure 5.)

```
$u^\dagger Bu$ takes both signs on it. For an imaginary frequency the record
proves that the eigenspace is **Krein-neutral**: from $Bh = h^\dagger B$,
$w\,u^\dagger Bv = u^\dagger Bhv = (hu)^\dagger Bv = w^*u^\dagger Bv$, so
$(w - w^*)u^\dagger Bv = 0$ and the form vanishes when $w$ is not real. The next
cell computes the inertia of the eigenspace of the eigenvalue with the largest
real part (or, beyond $|k_5| = 1$, the largest imaginary part) along the $k_5$
scan and checks both statements (record check
Q_one_particle_complex_and_zero_frequencies_Krein_neutral).
```

In [12]:

```
def leading_inertia(k5):
    """Dimension and Krein inertia of the eigenspace of the leading eigenvalue."""
    H_mat = numeric_h(1, [0, 0, 0, 0, 0, k5, 0, 0])  # k5 along x5
    w2_here = 1 - k5 ** 2
    w = np.sqrt(w2_here) if w2_here > 0 else 1j * np.sqrt(-w2_here)
    return eigenspace_facts(H_mat, w)


k5_scan = scan[np.abs(np.abs(scan) - 1.0) > 1e-6]  # leave out w = 0 at |k5| = 1
facts = [leading_inertia(q) for q in k5_scan]
positive = np.array([f[1][0] for f in facts])
negative = np.array([f[1][1] for f in facts])
dims = np.array([f[0] for f in facts])
real_region = np.abs(k5_scan) < 1
check(np.all(dims == 8)
      and np.all(positive[real_region] == 4) and np.all(negative[real_region] == 4)
```

```
      and np.all(positive[~real_region] == 0) and np.all(negative[~real_region] == 0)
      and WL_PAIR["Q_one_particle_complex_and_zero_frequencies_Krein_neutral"]
      == "PASS",
      "inertia (4,4) at real frequencies, Krein-neutral at imaginary frequencies",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_complex_and_zero_frequencies_Krein_neutral")

fig, ax = plt.subplots(figsize=(8.0, 4.3))
ax.plot(k5_scan, positive, "o", markersize=6, markerfacecolor="none",
        label="positive directions of the form")  # open circles
ax.plot(k5_scan, negative, "x", markersize=4,
        label="negative directions of the form")  # crosses inside the circles
ax.axvspan(-3, -1, color="#f2d0a9", alpha=0.5, label="imaginary frequency")
ax.axvspan(1, 3, color="#f2d0a9", alpha=0.5)
ax.set_xlim(-3, 3)
ax.set_ylim(-0.5, 8.5)
ax.set_xlabel("momentum $k_5$ along the extra time $x5$ ($m = 1$)")
ax.set_ylabel("Krein inertia of the 8-dimensional eigenspace")
ax.set_title("Inertia (4,4) for real frequencies, neutral for growing modes")
ax.legend(fontsize=8, loc="upper center")
save_figure(fig, "krein_inertia_scan",
            "The Krein inertia of the 8-dimensional eigenspace of the leading "
            "eigenvalue of $h_m(k)$, $m = 1$, along the momentum $k_5$ of an extra "
            "time: the numbers of positive (circles) and negative (crosses) "
            "eigenvalues of the form $u^\\dagger Bu$ restricted to it; horizontal "
            "axis $k_5$, vertical axis the counts. For $|k_5| < 1$ (real "
            "frequency) the inertia is (4,4); in the shaded regions $|k_5| > 1$ the "
            "frequency is imaginary, the mode grows, and the form vanishes on the "
            "whole eigenspace (Krein-neutral).")
```

Out [12]:

```
PASS inertia (4,4) at real frequencies, Krein-neutral at imaginary frequencies
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    Q_one_particle_complex_and_zero_frequencies_Krein_neutral
```

Out [12]:

```
Figure 21c.6 saved as Revision/textbook/figures/21c_6_krein_inertia_scan.png
```

Text cell:

```
## 10. The figure files

The last cell checks that the six figure files exist and prints the number of
checks that passed.
```

In [13]:

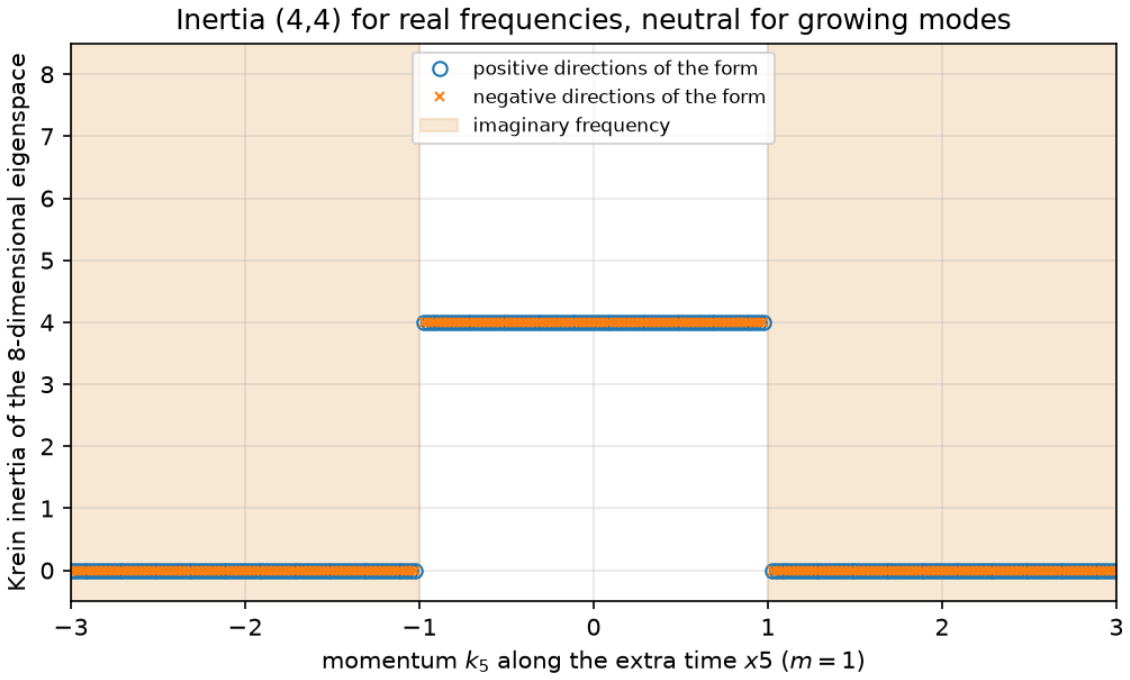

Figure 542: The Krein inertia of the 8-dimensional eigenspace of the leading eigenvalue of $h_m(k)$, $m = 1$, along the momentum k_5 of an extra time: the numbers of positive (circles) and negative (crosses) eigenvalues of the form $u^\dagger Bu$ restricted to it; horizontal axis k_5, vertical axis the counts. For $|k_5| < 1$ (real frequency) the inertia is (4,4); in the shaded regions $|k_5| > 1$ the frequency is imaginary, the mode grows, and the form vanishes on the whole eigenspace (Krein-neutral). (Notebook 21c, figure 6.)

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)
check(len(names) == 6 and all(
    output_file(f"{FIGURE_FOLDER}/21c_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the six figure files of notebook 21c exist")
all_checks_passed()
```

Out [13]:

```
PASS the six figure files of notebook 21c exist
ALL 17 CHECKS PASSED (notebook 21c)
```

Text cell:

```
## 11. What this notebook showed

- PROVED (exact matrix algebra, reproducing the Revision record): a conjugation
  $\Psi \to M\Psi^{\dagger T}$ of the quantised field keeps the canonical
  anticommutator $\{\Psi, \Psi^\dagger\} = B\,\delta$ if and only if
  $MB^TM^\dagger = B$; this holds for $M = \Gamma$ (times a phase) and fails for
  $M = 1$, which gives $-B$. Among $\cos t\,1 + \sin t\,\Gamma$ only $\pm\Gamma$
  works.
- COMPUTED (explicit operators on the 65536 states of 16 fermion modes, in the
  positive representation of the Revision record): all 256 anticommutators of
  $\Psi$, of $\Gamma\Psi^{\dagger T}$ and of $\Psi^{\dagger T}$ act as the numbers
  $B$, $B$ and $-B$.
- PROVED: the valid conjugation counts the empty modes, $Q' = 16 - Q$: up to a
  constant it reverses the charge (particles and holes exchanged).
- PROVED (sympy, symbolic $m$ and $k$): the conjugated field evolves with the
  **reversed mass**, $-\Gamma h_m(k)^*\Gamma = h_{-m}(-k)$; at the same time
  $\Gamma h_m\Gamma = h_{-m}$, so the one-particle spectra of $+m$ and $-m$ are
  identical, not opposite.
- Reproduced: the eight one-particle samples of the pairing record; Krein inertia
  (4,4) at every real frequency of the scan; Krein-neutral eigenspaces for the
  growing extra-time modes.
- Consequence: for the quantised field dirac16complex the conjugation that is
  compatible with canonical quantisation is $\Psi \to \Gamma\Psi^{\dagger T}$,
  which maps the theory with mass $m$ to the theory with mass $-m$; the same-mass
  candidate is not compatible. This is a statement about the anticommutator and
  the field equation; no regularised quantum field theory in signature (4,4) is
  constructed here, and nothing in it creates particles, antiparticles or
  universes.
```

21.30 Line-by-line walk-through of Notebook 21c

The notebook has 13 code cells, In [1] to In [13].

In [1], the set-up cell. It is the set-up cell of Notebook 21a, explained line by line in Section 21.15 under In [1], except that the comment lines at the top hold the run instructions of Notebook 21c (Section 21.28) and that the line

```
NOTEBOOK_ID = "21c"  # this notebook: chapter 21, example c
```

names this notebook. The cell prints Set-up of notebook 21c complete.

In [2]: the gammas, B and the records.

```
import numpy as np  # numbers, arrays, matrices
import sympy as sp  # exact algebra

fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
COORDS = fixture["coordinates"]  # "x0", ..., "x7"
ETA = fixture["eta"]
gamma = [np.array(mat, dtype=np.int64) for mat in fixture["gamma"]]  # x0 .. x7
I16 = np.eye(16, dtype=np.int64)
```

The packages and the Revision gammas, kept in a list numbered from 0 as in Notebook 21b (`gamma[4]` is $\gamma^{(x4)}$, `gamma[0]` is $\gamma^{(x0)}$).

```
check(len(gamma) == 8
      and all(g.shape == (16, 16) and set(np.unique(g)) <= {-1, 0, 1} for g in gamma)
      and all(np.array_equal(gamma[a] @ gamma[b] + gamma[b] @ gamma[a],
                             2 * (ETA[a] if a == b else 0) * I16)
              for a in range(8) for b in range(8)),
      "eight real 16 x 16 gamma matrices with the Clifford relation, signature (4,4)")
```

The same test as in Notebooks 21a and 21b, written as one check: eight real signed-entry matrices that obey the Clifford relation.

```
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # C = g(x0) g(x1) g(x2) g(x3)
Gamma = C @ gamma[4] @ gamma[5] @ gamma[6] @ gamma[7]  # the chirality
B = -1j * (C @ gamma[4])  # the Krein matrix B = -i C gamma^(x4)
```

C, then Γ as C times the four time-like gammas (the same product $\gamma^{(x0)}\gamma^{(x1)}\cdots\gamma^{(x7)}$), and B.

```
def load_checks(path):
    """name -> verdict (upper case) of a Revision report."""
    entries = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
    return {c["name"]: c["verdict"].upper() for c in entries}
```

A helper that reads a Revision report and returns the dictionary from check name to verdict, in capitals.

```
LEAD_FILE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
LEAD = load_checks(LEAD_FILE)
PY_THEORY = load_checks("Revision/theory/reports/python-field-theory.json")
WL_PAIR = load_checks("Revision/pairing/reports/wolfram-pairing.json")
THEORY = {f["key"]: f for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
PAIRING = json.loads(repository_file("Revision/pairing/pairing-theory.json")
                     .read_text(encoding="utf-8"))
```

The lead report, the sympy field-theory report, the Wolfram pairing report, the formulas of the field-theory record, and the whole pairing record `Revision/pairing/pairing-theory.json`, whose one-particle samples are used in In [10].

```
clauses = THEORY["quantisation"]["wl"].split("; ")  # the statements of the formula
rule = next(c for c in clauses if c.startswith("{Psi_A(x), Psi^dagger_C(y)}"))
positive = next(c for c in clauses if c.startswith("positive representation"))
say("record: " + rule)
say("record: " + positive)
```

The record's formula `quantisation` is cut at its semicolons. `next(c for c in clauses if ...)` returns the first statement that satisfies the condition: the one that starts with the anticommutator and the one that starts with the words positive representation. Out [2] prints both.

```
check(rule.endswith("= B_AC delta^7(x - y)/Cos[z]")
      and positive == "positive representation chi = Psi^dagger B, "
      "{Psi_A, chi_C} = delta_AC",
      "the record's anticommutator B_AC delta / cos z and positive representation",
      record="Revision/theory/field-theory.json, formula quantisation")
```

The check demands that the record states the canonical rule with $B_{AC}\delta^7/\cos z$, and the positive representation with $\chi = \Psi^\dagger B$ and $\{\Psi_A, \chi_C\} = \delta_{AC}$. (Since $B^2 = 1$, $\chi = \Psi^\dagger B$ means $\Psi^\dagger = \chi B$: with $\Psi_A = b_A$ and $\chi_C = b_C^*$ this is the representation of Section 21.25.)

In [3]: the properties of B.

```
eigenvalues = np.linalg.eigvalsh(B)  # eigenvalues of a Hermitian matrix, sorted
check(np.array_equal(B.real, np.zeros((16, 16))) and np.allclose(B, B.conj().T)
      and np.allclose(B @ B, np.eye(16)) and abs(np.trace(B)) < 1e-12
      and np.allclose(eigenvalues, [-1] * 8 + [1] * 8)
      and LEAD["B_imaginary_hermitian"] == "PASS"
      and PY_THEORY["B_properties"] == "PASS",
      "B is purely imaginary, Hermitian, B^2 = 1, trace 0: signature (8,8)",
      record=f"{LEAD_FILE}, check B_imaginary_hermitian")
check(np.allclose(B.T, -B), "B^T = -B")
```

The eigenvalues in increasing order; B is purely imaginary, Hermitian, squares to 1, has trace 0 (`np.trace` adds the diagonal), so eight eigenvalues -1 and eight $+1$; two record checks say the same. The second check is $B^T = -B$, used in Section 21.25.

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.3), width_ratios=[1, 1.3])
```

The blue-white-red colour map of Notebook 21a, and a figure with two panels, the right one wider.

```
image = axes[0].imshow(B.imag, cmap=SIGNS, vmin=-1, vmax=1)
axes[0].set_title("imaginary part of $B = -iC\\gamma^{(x4)}$")
```

```
axes[0].set_xticks([0, 7, 15], ["1", "8", "16"])
axes[0].set_yticks([0, 7, 15], ["1", "8", "16"])
axes[0].set_xlabel("column")
axes[0].set_ylabel("row")
axes[0].grid(False)
fig.colorbar(image, ax=axes[0], ticks=[-1, 0, 1], shrink=0.8)
```

The left panel draws the imaginary part of B (`B.imag`) as a heat map, with a colour bar.

```
axes[1].bar(np.arange(1, 17), eigenvalues,
            color=["#2a78d6"] * 8 + ["#e34948"] * 8)
axes[1].set_xlabel("eigenvalue number (sorted)")
axes[1].set_ylabel("eigenvalue of $B$")
axes[1].set_title("eight eigenvalues $-1$, eight $+1$")
```

The right panel draws the sixteen eigenvalues as bars, the first eight blue and the last eight red.

```
save_figure(fig, "krein_matrix",
            "The Krein matrix $B = -iC\\gamma^{(x4)}$ of the canonical "
            "anticommutator. Left: its imaginary part as a heat map (its real part "
            "is zero; horizontal axis the column, vertical axis the row, red $+1$, "
            "blue $-1$); every row and column has one entry, and the nonzero "
            "entries join the two chiral halves. Right: its sixteen eigenvalues in "
            "increasing order, eight $-1$ and eight $+1$ (signature (8,8)); "
            "because of the eight negative eigenvalues the field adjoint cannot be "
            "an ordinary Hilbert adjoint.")
```

Figure 21c.1. **What the figure shows.** Left: sixteen coloured squares, one per row and column, all in the off-diagonal blocks (B contains one gamma, $\gamma^{(x4)}$, times the block-diagonal C). Right: a staircase of eight bars at -1 and eight at $+1$.

In [4]: which conjugation keeps the anticommutator.

```
def mismatch(M):
    """|| M B^T M^dagger - B ||: zero when M keeps the canonical anticommutator."""
    return np.linalg.norm(M @ B.T @ M.conj().T - B)
```

The size of the failure of the condition of Section 21.25; `np.linalg.norm` of a matrix is the square root of the sum of the squared moduli of all its entries.

```
q_gamma = Gamma @ B.T @ Gamma.T  # M = Gamma (real, so M^dagger = M^T)
q_one = B.T  # M = 1
check(np.allclose(q_gamma, B) and np.allclose(q_one, -B)
      and LEAD["quantum_charge_conjugation_unitary_type"] == "PASS",
      "M B^T M^dagger = +B for M = Gamma and = -B for M = 1",
      record=f"{LEAD_FILE}, check quantum_charge_conjugation_unitary_type")
```

$MB^TM^\dagger$ for $M = \Gamma$ (real, so its conjugate transpose is its transpose) and for $M = 1$: $+B$ and $-B$, as the lead check records.

```
t_values = np.linspace(0.0, 2 * np.pi, 721)  # steps of half a degree
family = [np.cos(t) * np.eye(16) + np.sin(t) * Gamma for t in t_values]
mismatches = np.array([mismatch(M) for M in family])
zeros = t_values[mismatches < 1e-12]
say("t where the mismatch vanishes (in units of pi): "
    + ", ".join(f"{t / np.pi:.3f}" for t in zeros))
```

721 values of t from 0 to 2π (steps of $2\pi/720$, half a degree), the matrices $\cos t\,1 + \sin t\,\Gamma$ and their mismatches. `t_values[mismatches < 1e-12]` keeps only the values of t where the mismatch is below 10^{-12} (an array indexed by a true-false array keeps the entries marked true). Out [4] prints 0.500 and 1.500, that is $t = \pi/2$ and $3\pi/2$.

```
phases_ok = all(abs(mismatch(c * Gamma)) < 1e-12 for c in (1j, np.exp(0.3j), -1))
scales_bad = all(mismatch(c * Gamma) > 1 for c in (0.5, 2.0))
check(np.allclose(zeros / np.pi, [0.5, 1.5]) and phases_ok and scales_bad,
      "in the family cos t + sin t Gamma only t = pi/2, 3pi/2 work; c Gamma works "
      "exactly for |c| = 1")
```

Multiples $c\,\Gamma$ with $|c| = 1$ ($c = i$, $e^{0.3i}$, -1) work; $c = 0.5$ and $c = 2$ fail (they give $|c|^2 B$). The check combines the three findings.

```
fig, ax = plt.subplots(figsize=(8.0, 4.3))
ax.plot(t_values / np.pi, mismatches)
ax.plot([0.5, 1.5], [0, 0], "o", color="#e34948", markersize=9,
        label="$M = \\pm\\Gamma$: mismatch zero")
ax.plot([0.0, 1.0, 2.0], [mismatch(np.eye(16))] * 3, "s", color="#2a78d6",
        label="$M = \\pm 1$: mismatch $\\|{-B} - B\\| = 8$")
ax.set_xlabel("$t / \\pi$ in $M(t) = \\cos t\\,1 + \\sin t\\,\\Gamma$")
ax.set_ylabel("$\\|M B^T M^\\dagger - B\\|$")
ax.set_title("Only $\\pm\\Gamma$ keeps the canonical anticommutator")
ax.legend(fontsize=8)
```

The mismatch against t/π, red dots at the two zeros, and blue squares at $t = 0, \pi, 2\pi$, where $M = \pm 1$ and the mismatch is $\|-B - B\| = 2 \cdot 4 = 8$.

```
save_figure(fig, "which_conjugation",
            "The mismatch $\\|M B^T M^\\dagger - B\\|$ (the size of the failure of "
            "the canonical anticommutator) for the conjugations $\\Psi \\to "
            "M\\Psi^{\\dagger T}$ with $M = \\cos t\\,1 + \\sin t\\,\\Gamma$; "
            "horizontal axis $t/\\pi$ from $0$ to $2$, vertical axis the mismatch "
            "(pure number). It vanishes only at $t = \\pi/2$ and $3\\pi/2$, that is "
            "for $M = \\pm\\Gamma$ (red dots); the same-mass choice $M = \\pm 1$ "
            "(blue squares) gives $-B$ instead of $B$, a mismatch of $\\|2B\\| = 8$.")
```

Figure 21c.2. **What the figure shows.** The curve $4|1 + \cos 2t|$ of Section 21.25: it starts at 8, falls to zero at $t = \pi/2$, rises to 8 at $t = \pi$, falls to zero at $3\pi/2$ and returns to 8. Only the two isolated points $M = \pm\Gamma$ are allowed.

In [5]: sixteen fermion modes on a computer.

```
MODES, STATES = 16, 2 ** 16
index = np.arange(STATES)  # basis state numbers 0 .. 65535
occupied_count = np.zeros(STATES, dtype=np.int64)  # number of occupied modes
for j in range(MODES):
    occupied_count += (index >> j) & 1  # >> shifts the bits; & 1 reads bit j
```

A state of the 16 modes is a column of 65536 complex numbers, one **amplitude** per occupation pattern; basis state number n has the pattern given by the binary digits (**bits**) of n: bit j is the occupation of mode j (counted from 0). The operator written as two greater-than signs shifts the binary digits of every number j places to the right, and `& 1` (bitwise and with 1) keeps the last digit: together they read bit j. Adding these bits over all modes gives, for each basis state, the number of occupied modes.

```
OPS = []  # for each mode j: the states where j is occupied, and the sign factors
for j in range(MODES):
    filled = ((index >> j) & 1).astype(bool)
    below = index & ((1 << j) - 1)  # the bits of the modes below j
    parity = np.zeros(STATES, dtype=np.int64)
    for i in range(j):
        parity += (below >> i) & 1
    OPS.append((index[filled], index[~filled], np.where(parity % 2 == 0, 1.0, -1.0)))
```

For each mode j: `filled` marks the states in which mode j is occupied. The operator written as two less-than signs shifts digits to the left, so `1` shifted by `j` places is 2^j, and $2^j - 1$ has ones exactly in the bits $0, \ldots, j-1$; the bitwise and with it keeps the bits of the modes below j. `parity` counts the occupied modes below j, and `np.where(parity % 2 == 0, 1.0, -1.0)` turns an even count into the sign $+1$ and an odd one into -1: the Jordan-Wigner sign of Section 21.26. The list `OPS` stores, for each mode, the states where it is occupied (`index[filled]`), those where it is empty (`~` negates the true-false array) and the signs.

```
def annihilate(j, v):
    """b_j v: empty mode j (with the Jordan-Wigner sign)."""
    filled, _, sign = OPS[j]
    out = np.zeros_like(v)
    out[filled ^ (1 << j)] = sign[filled] * v[filled]
    return out
```

The annihilation operator applied to a state `v`: the amplitude of every pattern with mode j occupied is moved, with its sign, to the pattern with bit j cleared (`^` is the bitwise exclusive or, which flips bit j); every other amplitude of the result is zero. Instead of storing a 65536×65536 matrix, the notebook stores these index lists.

```
def create(j, v):
    """b_j^* v: fill mode j (with the Jordan-Wigner sign)."""
    _, empty, sign = OPS[j]
    out = np.zeros_like(v)
    out[empty | (1 << j)] = sign[empty] * v[empty]
    return out
```

The creation operator does the reverse: amplitudes of patterns with mode j empty move to the pattern with bit j set (`|` is the bitwise or). The sign depends only on the modes below j, which the move does not change.

```
rng = np.random.default_rng(2103)  # fixed seed: the same state in every run
v = rng.normal(size=STATES) + 1j * rng.normal(size=STATES)
v /= np.linalg.norm(v)  # a random normalised state of the 16 modes
```

A random state of the 16 modes, scaled to length 1 (`/=` divides in place).

```
basic = all(np.allclose(annihilate(j, create(k, v)) + create(k, annihilate(j, v)),
                        (1.0 if j == k else 0.0) * v)
            for j, k in [(0, 0), (3, 3), (15, 15), (0, 1), (4, 9), (15, 2)])
check(basic, "Jordan-Wigner operators: {b_j, b_k^*} = delta_jk on a random state")
```

The rule $\{b_j, b_k^*\} = \delta_{jk}$, tested on the random state for six pairs: $b_j b_k^* v + b_k^* b_j v$ must be v for $j = k$ and zero otherwise.

In [6]: the anticommutators of three fields.

```
B_column = [int(np.flatnonzero(B[:, c])[0]) for c in range(16)]  # row of the entry
```

Every column of B has exactly one nonzero entry; `np.flatnonzero` lists the positions of the nonzero entries of column `c` (`B[:, c]`), and the first one is kept. So $\Psi_C^\dagger = \sum_D b_D^* B_{DC}$ is a single creation operator times $B_{DC} = \pm i$.

```
def psi(A, u):
    """Psi_A = b_A."""
    return annihilate(A, u)


def psi_dagger(Cc, u):
    """Psi^dagger_C = sum_D b_D^* B_DC (a single term)."""
    D = B_column[Cc]
    return B[D, Cc] * create(D, u)
```

The field operators of the positive representation applied to a state. (The letter `Cc` avoids a clash with the matrix `C`.)

```
def field_pair(M):
    """For the conjugate M Psi^{dagger T} with diagonal M: the maps u -> Psi'_A u
    and u -> Psi'^dagger_C u (Psi'_A = M_AA Psi^dagger_A, Psi'^dagger_C =
    conj(M_CC) Psi_C)."""
    d = np.diag(M)
    return (lambda A, u: d[A] * psi_dagger(A, u),
            lambda Cc, u: np.conj(d[Cc]) * psi(Cc, u))
```

For a diagonal matrix M (both Γ and 1 are diagonal), the conjugate field is $\Psi'_A = M_{AA}\Psi_A^\dagger$ and its adjoint $\Psi_C'^\dagger = M_{CC}^*\Psi_C$. `np.diag(M)` takes the diagonal, and `lambda` defines a small unnamed function; the function returns the pair of them.

```
def anticommutator_matrix(first, second):
    """N_AC with {first_A, second_C} v = N_AC v, and whether it acts as a number."""
    N = np.zeros((16, 16), dtype=complex)
    number_like = True
    for A in range(16):
        for Cc in range(16):
            w = first(A, second(Cc, v)) + second(Cc, first(A, v))
            N[A, Cc] = np.vdot(v, w)  # v^dagger w
            number_like &= np.allclose(w, N[A, Cc] * v)
    return N, number_like
```

For each of the 256 pairs (A, C) the anticommutator is applied to the random state, $w = XYv + YXv$; `np.vdot(v, w)` is $v^\dagger w$, the number N_{AC}; and `number_like` stays true only if $w = N_{AC}v$ for every pair, that is, if the anticommutator acts on the state as a plain number (`&=` combines with "and").

```
N_psi, ok_psi = anticommutator_matrix(psi, psi_dagger)
N_same, ok_same = anticommutator_matrix(psi, psi)  # {Psi_A, Psi_C}
N_gamma, ok_gamma = anticommutator_matrix(*field_pair(Gamma))
N_one, ok_one = anticommutator_matrix(*field_pair(np.eye(16)))
```

Four measurements: $\{\Psi_A, \Psi_C^\dagger\}$, $\{\Psi_A, \Psi_C\}$, and the anticommutators of the conjugate fields with $M = \Gamma$ and $M = 1$ (the star unpacks the returned pair into the two arguments). In all, 1024 anticommutators of operators on 65536 states; this takes about 10 seconds.

```
check(ok_psi and ok_same and np.allclose(N_psi, B) and np.allclose(N_same, 0)
      and PY_THEORY["canonical_anticommutator_B"] == "PASS",
      "operators: {Psi_A, Psi^dagger_C} = B_AC and {Psi_A, Psi_C} = 0 (256 pairs)",
      record="Revision/theory/reports/python-field-theory.json, check "
      "canonical_anticommutator_B")
check(ok_gamma and ok_one and np.allclose(N_gamma, B) and np.allclose(N_one, -B),
      "operators: Gamma Psi^{dagger T} has +B, Psi^{dagger T} has -B")
```

The positive representation obeys the canonical rule; the valid conjugate $\Gamma\Psi^{\dagger T}$ obeys it too; the same-mass candidate $\Psi^{\dagger T}$ obeys it with $-B$. This is Section 21.25 confirmed with explicit operators.

In [7]: the measured anticommutators as heat maps.

```
fig, axes = plt.subplots(1, 3, figsize=(12.0, 4.2))
for ax, N, title in ((axes[0], N_psi, "$\\{\\Psi_A, \\Psi^\\dagger_C\\}$"),
                     (axes[1], N_gamma, "$\\Psi' = \\Gamma\\Psi^{\\dagger T}$"),
                     (axes[2], N_one, "$\\Psi'' = \\Psi^{\\dagger T}$")):
    image = ax.imshow(N.imag, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("$C$")
    ax.grid(False)
axes[0].set_ylabel("$A$")
fig.colorbar(image, ax=axes, ticks=[-1, 0, 1], shrink=0.8,
             label="imaginary part of the anticommutator")
```

Three heat maps of the imaginary parts of the measured 16×16 matrices (their real parts are zero), with C along the horizontal and A along the vertical axis.

```
save_figure(fig, "operator_anticommutators",
            "The anticommutators $\\{X_A, Y^\\dagger_C\\}$ measured with explicit "
            "operators on the 65536 states of 16 fermion modes (imaginary parts; "
            "the real parts are zero; horizontal axis $C$, vertical axis $A$, red "
            "$+1$, blue $-1$). Left: the field $\\Psi$ of the positive "
            "representation, which gives the Krein matrix $B$. Middle: the "
            "conjugate $\\Gamma\\Psi^{\\dagger T}$, which gives $B$ again, so it is "
            "a field of the same quantum theory. Right: the same-mass candidate "
            "$\\Psi^{\\dagger T}$, which gives $-B$ (every colour reversed), so it is "
            "not.")
```

Figure 21c.3. **What the figure shows.** The left and middle panels are the same picture, the heat map of B of figure 21c.1; the right panel is that picture with red and blue exchanged.

In [8]: the valid conjugation counts the empty modes.

```
def charge(first_dagger, second, u):
    """sum_AC first_dagger_A B_AC second_C u (only the 16 nonzero B_AC)."""
    out = np.zeros_like(u)
    for Cc in range(16):
        A = B_column[Cc]  # the row of the nonzero entry of column C
        out += first_dagger(A, B[A, Cc] * second(Cc, u))
    return out
```

The charge operator $\sum_{A,C} X^\dagger_A B_{AC} Y_C$ applied to a state; since each column of B has one nonzero entry, the double sum has only 16 terms.

```
conj_g, conj_g_dagger = field_pair(Gamma)
conj_1, conj_1_dagger = field_pair(np.eye(16))
Q_v = charge(psi_dagger, psi, v)
Qp_v = charge(conj_g_dagger, conj_g, v)
Qpp_v = charge(conj_1_dagger, conj_1, v)
```

The two conjugate fields and their adjoints, and the three charges Q, Q' (valid conjugate) and Q'' (same-mass candidate) applied to the random state.

```
check(np.allclose(Q_v, occupied_count * v)
      and np.allclose(Qp_v, (16 - occupied_count) * v)
      and np.allclose(Qpp_v, (occupied_count - 16) * v),
      "Q = number of occupied modes, Q' = 16 - Q (Gamma), Q'' = Q - 16 (M = 1)")
```

`occupied_count * v` multiplies each amplitude by the number of occupied modes of its basis state: that is how the number operator N acts. The check proves, on this state, $Q = N$, $Q' = 16 - N$ and $Q'' = N - 16$ (Section 21.26).

```
n = np.arange(17)
fig, ax = plt.subplots(figsize=(8.0, 4.4))
ax.plot(n, n - 8, "o-", label="$Q - 8$ of the field $\\Psi$")
ax.plot(n, (16 - n) - 8, "s-", label="$Q' - 8$ of $\\Gamma\\Psi^{\\dagger T}$ "
        "(valid)")
ax.plot(n, (n - 16) + 8, "^--", color="#999999",
        label="$Q'' + 8$ of $\\Psi^{\\dagger T}$ (invalid)")
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xlabel("number of occupied modes in the basis state")
ax.set_ylabel("charge (shifted by the constant 8)")
ax.set_title("The valid conjugation reverses the charge")
ax.legend(fontsize=8)
```

For a basis state with $n = 0, \dots, 16$ occupied modes the three charges are n, $16 - n$ and $n - 16$; they are drawn shifted by the constant 8 (minus 8 for the first two, plus 8 for the third), so that a reversal shows as a mirror image in the horizontal axis.

```
save_figure(fig, "charge_of_states",
            "The charge of the basis states of 16 fermion modes, measured by three "
            "fields; horizontal axis the number of occupied modes (0 to 16), "
            "vertical axis the charge shifted by a constant (minus 8 for $\\Psi$ "
            "and $\\Gamma\\Psi^{\\dagger T}$, plus 8 for $\\Psi^{\\dagger T}$; "
            "pure numbers). "
            "The field $\\Psi$ counts the occupied modes; the valid conjugate "
            "$\\Gamma\\Psi^{\\dagger T}$ counts the empty ones, $Q' = 16 - Q$, so "
            "the shifted charge is exactly reversed (particles and holes "
            "exchanged); the invalid candidate $\\Psi^{\\dagger T}$ would give "
            "$Q - 16$, not a reversal.")
```

Figure 21c.4. **What the figure shows.** A rising line (Ψ) and a falling line (the valid conjugate) that cross at $n = 8$: the charge reversed about its middle. The grey dashed line of the invalid candidate lies exactly on the rising line: shifted back by the constant, it measures the same charge as Ψ, not the opposite.

In [9]: the mass is reversed, the spectra are equal.

```
m = sp.symbols("m", real=True)
k = sp.symbols("k0:8", real=True)  # k[a] along xa; k[4] (the x4 entry) is not used
G = [sp.Matrix(g.tolist()) for g in gamma]
Gamma_s, B_s = sp.Matrix(Gamma.tolist()), -sp.I * sp.Matrix(C.tolist()) * G[4]
```

A real symbol for the mass, eight real symbols $k_0, \dots, k_7$ for the momenta (`k[a]` along x_a; `k[4]`, along the time, is not used), exact gammas, Γ and B.

```
def h(mass, momenta):
    """h_m(k) = -i m gamma^(x4) - gamma^(x4) sum_(a != x4) k_a gamma^a."""
    total = -sp.I * mass * G[4]
    for a in range(8):
        if a != 4:
            total -= momenta[a] * G[4] * G[a]
    return total
```

The one-particle Hamiltonian of Section 21.26, built term by term (`-=` subtracts from the running total; `!=` means "not equal").

```
zero = sp.zeros(16, 16)
h_m = h(m, k)
minus_k = [-q for q in k]
w2 = m ** 2 + k[0] ** 2 + k[1] ** 2 + k[2] ** 2 + k[3] ** 2 - k[5] ** 2 - k[6] ** 2 \
    - k[7] ** 2
```

The zero matrix, $h_m(k)$ with symbolic mass and momenta, the reversed momenta, and w^2 (the backslash continues the line).

```
check((-Gamma_s * h_m.conjugate() * Gamma_s - h(-m, minus_k)).expand() == zero,
      "-Gamma conj(h_m(k)) Gamma = h_(-m)(-k): the conjugated mode has the reversed "
      "mass")
check((Gamma_s * h_m * Gamma_s - h(-m, k)).expand() == zero
      and WL_PAIR["Q_one_particle_maps"] == "PASS",
      "Gamma h_m(k) Gamma = h_(-m)(k): equal spectra for +m and -m",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_maps")
```

The first two facts of Section 21.26, proved exactly for every real mass and momentum: the conjugated mode evolves with the reversed mass, and the Hamiltonians of $\pm m$ are similar.

```
check((h_m * h_m - w2 * sp.eye(16)).expand() == zero
      and (B_s * h_m - h_m.H * B_s).expand() == zero
      and WL_PAIR["Q_one_particle_flat_dispersion"] == "PASS"
      and PY_THEORY["mode_hamiltonian_B_selfadjoint_dispersion"] == "PASS",
      "h_m(k)^2 = w^2 1 and B h_m = h_m^dagger B",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_flat_dispersion")
```

The square and the Krein self-adjointness (`h_m.H` is $h^\dagger$), with the two record checks.

In [10]: the eight samples of the pairing record.

```
def numeric_h(mass, momenta):
    """h_m(k) as a numpy array."""
    total = -1j * mass * gamma[4].astype(complex)
    for a in range(8):
        if a != 4:
            total = total - momenta[a] * (gamma[4] @ gamma[a])
    return total
```

The same Hamiltonian with numbers instead of symbols.

```
def eigenspace_facts(H_mat, w):
    """Dimension and Krein inertia (positive, negative) of the eigenspace of w."""
    P = (np.eye(16) + H_mat / w) / 2  # projector onto the eigenvalue +w (w != 0)
```

```
    U_, s_, _ = np.linalg.svd(P)
    V = U_[:, s_ > 1e-9]  # orthonormal basis of the range of P
    form = np.linalg.eigvalsh(V.conj().T @ B @ V)
    return V.shape[1], (int(np.sum(form > 1e-9)), int(np.sum(form < -1e-9)))
```

Since $h^2 = w^2$, the matrix $P = \frac{1}{2}(1 + h/w)$ is the projector onto the eigenvectors of h with eigenvalue w ($P^2 = \frac{1}{4}(1 + 2h/w + h^2/w^2) = \frac{1}{4}(2 + 2h/w) = P$). The **singular value decomposition** `np.linalg.svd` writes P as a product of three matrices; the columns of the first that belong to nonzero singular values form an orthonormal basis V of the eigenspace. The form $u^\dagger B u$ restricted to the eigenspace is the matrix $V^\dagger B V$; its eigenvalues are counted by sign. The function returns the dimension (the number of columns of V) and the Krein inertia.

```
rows_ok = True
for sample in PAIRING["data"]["one_particle_flat"]["samples"]:
    H_mat = numeric_h(sample["m"], sample["k"])
    w = float(sp.sympify(sample["w"].replace("Sqrt[", "sqrt(").replace("]", ")")))
    w_here = np.sqrt(np.max(np.linalg.eigvals(H_mat @ H_mat).real))
    dim_p, inertia_p = eigenspace_facts(H_mat, w_here)
    dim_m, inertia_m = eigenspace_facts(H_mat, -w_here)
```

For each sample of the record: the Hamiltonian; the record's frequency, written in Wolfram notation such as `Sqrt[2]`, which `.replace` turns into `sqrt(2)` and `sp.sympify` reads as a number; the frequency computed here as the square root of the largest eigenvalue of h^2; and the dimensions and inertias of both eigenspaces.

```
    same = (abs(w_here - w) < 1e-12 and dim_p == sample["dim_plus_w"]
            and dim_m == sample["dim_minus_w"]
            and list(inertia_p) == sample["B_inertia_plus_w"]
            and list(inertia_m) == sample["B_inertia_minus_w"])
    rows_ok &= same
```

Every number is compared with the record (the inertia, a pair, is turned into a list for the comparison).

```
    k_text = ",".join(str(q) for q in sample["k"])
    say(f"m = {sample['m']:+d}, k = ({k_text}): w = {w_here:.6f}, dims {dim_p} "
        f"{dim_m}, inertia ({inertia_p[0]},{inertia_p[1]}) "
        f"({inertia_m[0]},{inertia_m[1]}), same: {same}")
```

One printed line per sample: Out [10] shows the eight rows of the table of Section 21.26, each ending with "same: True".

```
check(rows_ok and WL_PAIR["Q_one_particle_Krein_signatures"] == "PASS",
      "all eight one-particle samples of the pairing record reproduced",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_Krein_signatures")
```

All eight samples agree with the record.

In [11]: the spectra along two lines of momenta.

```
def spectrum(mass, k1=0.0, k5=0.0):
    """The 16 eigenvalues of h_m with momenta k1 (x1) and k5 (x5), sorted."""
    momenta = [0, k1, 0, 0, 0, k5, 0, 0]  # k0, ..., k7
    values = np.linalg.eigvals(numeric_h(mass, momenta))
    return np.sort_complex(np.round(values, 9))
```

The sixteen eigenvalues of h_m with a momentum k_1 along ordinary space and k_5 along an extra time (the list `momenta` holds $k_0, \ldots, k_7$, so k_1 stands at position 1 and k_5 at position 5), rounded to 9 decimals (so that equal values sort in the same order) and sorted (`np.sort_complex` sorts by real part, then by imaginary part).

```
scan = np.linspace(-3.0, 3.0, 241)  # steps of 0.025; contains k5 = -1 and +1
spec_k1 = {s: np.array([spectrum(s, k1=q) for q in scan]) for s in (1, -1)}
spec_k5 = {s: np.array([spectrum(s, k5=q) for q in scan]) for s in (1, -1)}
exceptional = np.abs(np.abs(scan) - 1.0) < 1e-9  # the two points k5 = -1, +1
```

241 momenta from -3 to 3; the spectra for $m = +1$ and $m = -1$ along k_1 and along k_5; and a true-false array that marks the two points $k_5 = \pm 1$, where $w = 0$.

```
check(np.allclose(spec_k1[1], spec_k1[-1], rtol=0, atol=1e-8)
      and np.allclose(spec_k5[1][~exceptional], spec_k5[-1][~exceptional],
                      rtol=0, atol=1e-8)
      and np.abs(spec_k5[1][exceptional]).max() < 1e-6
      and np.abs(spec_k5[-1][exceptional]).max() < 1e-6
      and np.allclose(np.abs(spec_k1[1].real).max(axis=1), np.sqrt(1 + scan ** 2)),
      "scans: identical spectra for m = +1 and m = -1; w = sqrt(1 + k1^2)")
```

The spectra of $m = +1$ and $m = -1$ agree to 10^{-8} at every point (`rtol=0, atol=1e-8` means an absolute tolerance only), except at the two points $k_5 = \pm 1$. There $h^2 = 0$ although $h \neq 0$: such a matrix cannot be brought to diagonal form, and a computer finds its eigenvalues only to about the square root of the rounding unit, about 10^{-8}; the check therefore asks only that all 32 computed eigenvalues there are below 10^{-6}. Finally the largest real part along k_1 is $\sqrt{1 + k_1^2}$.

```
fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for s, style, label in ((1, "-", "$m = +1$"), (-1, "--", "$m = -1$")):
    axes[0].plot(scan, spec_k1[s].real.max(axis=1), style, color="#e34948",
                 label=f"$+w$, {label}")
    axes[0].plot(scan, spec_k1[s].real.min(axis=1), style, color="#2a78d6",
                 label=f"$-w$, {label}")
    axes[1].plot(scan, spec_k5[s].real.max(axis=1), style, color="#e34948",
                 label=f"largest real part, {label}")
    axes[1].plot(scan, spec_k5[s].imag.max(axis=1), style, color="#2ca02c",
                 label=f"largest imaginary part, {label}")
```

For both masses (solid and dashed lines): along k_1 the largest and smallest eigenvalue ($\pm w$); along k_5 the largest real part and the largest imaginary part.

```
axes[0].set_xlabel("momentum $k_1$ along ordinary space")
axes[0].set_ylabel("frequency (eigenvalue of $h_m$)")
```

```
axes[0].set_title("Good sector: real frequencies $\\pm\\sqrt{m^2 + k_1^2}$")
axes[0].legend(fontsize=7)
axes[1].set_xlabel("momentum $k_5$ along the extra time $x5$")
axes[1].set_ylabel("largest real or imaginary part of the eigenvalues")
axes[1].set_title("Extra-time momentum: imaginary beyond $|k_5| = |m|$")
axes[1].legend(fontsize=7)
```

Labels, titles and legends of the two panels.

```
save_figure(fig, "dispersion_pm_mass",
            "One-particle frequencies of the flat 4+4 Hamiltonian $h_m(k)$ for "
            "$m = +1$ (solid) and $m = -1$ (dashed, on top of the solid lines). "
            "Left: versus the momentum $k_1$ along ordinary space; the frequencies "
            "$\\pm\\sqrt{1 + k_1^2}$ are real and the same for both masses. Right: "
            "versus the momentum $k_5$ along an extra time; for $|k_5| < 1$ the "
            "largest real part is $\\sqrt{1 - k_5^2}$, for $|k_5| > 1$ the "
            "frequencies are imaginary, $\\pm i\\sqrt{k_5^2 - 1}$ (green), and the "
            "modes grow in time. Horizontal axes the momentum, vertical axes the "
            "frequency (pure numbers, units with $m = 1$).")
```

Figure 21c.5. **What the figure shows.** Left: two hyperbola-like curves $\pm\sqrt{1+k_1^2}$, the dashed ones exactly on the solid ones. Right: a red half-circle $\sqrt{1-k_5^2}$ between $k_5 = -1$ and 1, and outside it green curves $\sqrt{k_5^2-1}$ rising from zero: imaginary frequencies, growing modes, the same for both masses.

In [12]: the Krein inertia along the extra-time scan.

```
def leading_inertia(k5):
    """Dimension and Krein inertia of the eigenspace of the leading eigenvalue."""
    H_mat = numeric_h(1, [0, 0, 0, 0, 0, k5, 0, 0])  # k5 along x5
    w2_here = 1 - k5 ** 2
    w = np.sqrt(w2_here) if w2_here > 0 else 1j * np.sqrt(-w2_here)
    return eigenspace_facts(H_mat, w)
```

For $m = 1$ and momentum k_5: $w^2 = 1 - k_5^2$, and the leading eigenvalue is $w = \sqrt{w^2}$ when $w^2 > 0$ and $w = i\sqrt{-w^2}$ otherwise; the projector of In [10] works for an imaginary w as well, since still $h^2 = w^2$.

```
k5_scan = scan[np.abs(np.abs(scan) - 1.0) > 1e-6]  # leave out w = 0 at |k5| = 1
facts = [leading_inertia(q) for q in k5_scan]
positive = np.array([f[1][0] for f in facts])
negative = np.array([f[1][1] for f in facts])
dims = np.array([f[0] for f in facts])
real_region = np.abs(k5_scan) < 1
```

The scan without the two points of zero frequency (where the projector would divide by zero); the dimension and the counts of positive and negative directions for each momentum; and a mark for the region of real frequencies.

```
check(np.all(dims == 8)
      and np.all(positive[real_region] == 4) and np.all(negative[real_region] == 4)
      and np.all(positive[~real_region] == 0) and np.all(negative[~real_region] == 0)
      and WL_PAIR["Q_one_particle_complex_and_zero_frequencies_Krein_neutral"]
      == "PASS",
      "inertia (4,4) at real frequencies, Krein-neutral at imaginary frequencies",
      record="Revision/pairing/reports/wolfram-pairing.json, check "
      "Q_one_particle_complex_and_zero_frequencies_Krein_neutral")
```

Every eigenspace has dimension 8; for real frequencies the inertia is (4,4); for imaginary frequencies there are no positive and no negative directions: the form vanishes on the eigenspace (Section 21.26).

```
fig, ax = plt.subplots(figsize=(8.0, 4.3))
ax.plot(k5_scan, positive, "o", markersize=6, markerfacecolor="none",
        label="positive directions of the form")  # open circles
ax.plot(k5_scan, negative, "x", markersize=4,
        label="negative directions of the form")  # crosses inside the circles
ax.axvspan(-3, -1, color="#f2d0a9", alpha=0.5, label="imaginary frequency")
ax.axvspan(1, 3, color="#f2d0a9", alpha=0.5)
ax.set_xlim(-3, 3)
ax.set_ylim(-0.5, 8.5)
ax.set_xlabel("momentum $k_5$ along the extra time $x5$ ($m = 1$)")
ax.set_ylabel("Krein inertia of the 8-dimensional eigenspace")
ax.set_title("Inertia (4,4) for real frequencies, neutral for growing modes")
ax.legend(fontsize=8, loc="upper center")
```

The two counts against k_5: open circles for the positive and small crosses for the negative directions (they coincide, so the crosses sit inside the circles); `axvspan` shades the two regions of imaginary frequency.

```
save_figure(fig, "krein_inertia_scan",
            "The Krein inertia of the 8-dimensional eigenspace of the leading "
            "eigenvalue of $h_m(k)$, $m = 1$, along the momentum $k_5$ of an extra "
            "time: the numbers of positive (circles) and negative (crosses) "
            "eigenvalues of the form $u^\\dagger Bu$ restricted to it; horizontal "
            "axis $k_5$, vertical axis the counts. For $|k_5| < 1$ (real "
            "frequency) the inertia is (4,4); in the shaded regions $|k_5| > 1$ the "
            "frequency is imaginary, the mode grows, and the form vanishes on the "
            "whole eigenspace (Krein-neutral).")
```

Figure 21c.6. **What the figure shows.** In the middle, between $k_5 = -1$ and 1, a row of circles with crosses at height 4; in the two shaded regions the same marks at height 0. A growing mode carries no Krein charge of either sign.

In [13]: the figure files.

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)
check(len(names) == 6 and all(
    output_file(f"{FIGURE_FOLDER}/21c_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the six figure files of notebook 21c exist")
all_checks_passed()
```

The six figure files exist, and the last line prints ALL 17 CHECKS PASSED (notebook 21c).

21.31 Sakharov's conditions applied to this theory

Sections 21.3 to 21.6 taught the three conditions on toy models; Sections 21.7 to 21.30 derived the facts of the theory. This section puts them together, condition by condition. Each status is set by a check of the Revision record; Notebook 21d reads every one of these verdicts before it draws the scorecard (Section 21.37).

Condition 1: a process must change the number. The theory has no baryons; its only number of this kind is the U(1) charge Q. Section 21.17 proved that $\partial_\mu(\cos z\, J^\mu) = 0$ on every solution, in the author's metric with an arbitrary history $a_4(x_4)$, the deflating one included (lead check `u1_noether_matrix_identity`). So the charge of a region changes only by the flux through its boundary: no process creates or destroys charge at any point inside the patch $0 < z < \pi/2$, that is, off the brane, where the law is proved. A flux through the brane $z = \pi/2$ does change the charge of the patch (Section 21.18; on the homogeneous solutions of Section 18.21 at the rate $-V_6 q_0$), and the record does not describe where that charge goes: with the Z2 mirror glued on, the mirror patch changes its charge at the same rate and with the same sign (Section 18.21), so gluing does not stop the change. Only with no flux (the no-flux condition, ASSUMED, not derived) is the charge of one universe constant. In the language of the decay model of Section 21.5: every process of the theory has channels of the **same** charge, $\mathcal{B}_1 = \mathcal{B}_2$, so the net charge made by N pairs, $N(r - \bar{r})(\mathcal{B}_1 - \mathcal{B}_2)$, is zero; the rate model of Section 21.6 then starts with $\epsilon = 0$ and ends with $a = 0$, however slow the decays and however far from equilibrium (Notebook 21d, In [13]). **Condition 1 FAILS for every process inside the patch (PROVED: the local law); for the total charge of a universe it fails under the no-flux condition at the brane, which is ASSUMED (whether a junction condition at the brane implies it is OPEN).** For the quantised field this holds at the level of the canonical field equations; no regularised quantum field theory in signature (4,4) is constructed in the record, and no anomaly (a quantum breaking of a classical conservation law) is computed: OPEN.

Condition 2: C and CP must be violated. For the commuting field dirac16complex00 the same-mass charge conjugation $\Psi \to \Psi^c = \Psi^*$ is an exact symmetry. Proof, line by line:

- Every term of $\mathcal{L}$ is a sum of products $\Psi_r^* X_{rc} \Phi_c$ with a real matrix X (built from C, the gammas, the scale factors and the Ω_μ) and with Φ the field or one of its derivatives, or a product of two such sums (the term $\frac{\lambda}{2} S^2$), times the real $\cos z$ (Section 21.7).
- Replacing Ψ by Ψ^* turns $\Psi_r^* X_{rc} \Phi_c$ into $\Psi_r X_{rc} \Phi_c^* = (\Psi_r^* X_{rc} \Phi_c)^*$ (commuting numbers may be reordered; X_{rc} is real; the conjugate of a product is the product of the conjugates).
- So $\mathcal{L}[\Psi^*] = (\mathcal{L}[\Psi])^*$, and $\mathcal{L}$ is real (record checks `L_real_C` of `Revision/theory/reports/wolfram-field-theory.json` and `commuting_lagrangian_real` of the sympy report): $\mathcal{L}[\Psi^*] = \mathcal{L}[\Psi]$.

The map keeps the Lagrangian, hence maps solutions to solutions of the same theory, and it reverses every current (Section 21.10). By Proposition 2 of Section 21.5 a C-symmetric start can then develop no average charge asymmetry, even if condition 1 were met. Notebook 21d checks $\mathcal{L}[\Psi^*] = \mathcal{L}[\Psi] = -11.2851368126$ at one point of the author's metric with the record's spin connection (In [14]). For

the quantised field dirac16complex no same-mass charge-conjugation matrix is compatible with canonical quantisation: the only one that is, giving $\Psi \to \Gamma\Psi^{\dagger T}$, reverses the mass (Section 21.25). The maps Γ and $\Gamma\Psi^*$ also reverse the mass: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = \mathcal{L}_{m,\lambda}[\Gamma\Psi^*] = -\mathcal{L}_{-m,-\lambda}[\Psi] = 8.4767165144$ at the same point (In [14]; theorem T1, record check `T1_Lagrangian_primordial_commuting`). Whether particle and antiparticle processes of this theory happen at different rates (C and CP violation in rates) is **NOT COMPUTED**: the record computes no reaction rates at all. The status of condition 2 is therefore different for the two fields. For the commuting field dirac16complex00 condition 2 **FAILS (PROVED)**: $\mathcal{C}_+$ is an exact symmetry that reverses the charge, so Proposition 2 applies (the reality of the gammas, of C and of the Ω_μ: lead checks `representation_real` and `spinor_connection_real`; the reversal of the current: lead check `bilinears_under_charge_conjugation`). For the quantised field dirac16complex it is **NOT COMPUTED**: no rates are computed, and the only charge-conjugation matrix compatible with its quantisation reverses the mass (lead check `quantum_charge_conjugation_unitary_type`).

Condition 3: a departure from thermal equilibrium. The Revision record contains no computation of reaction rates or of a departure from equilibrium. Its Kohn-Sham states (Chapters 14 and 15) are instantaneous states along the history $a_4 = AHx_4$, and that history is a **prescribed background**: the Kohn-Sham states violate the source conditions of the a_4 equations, so the history is not a dynamical consequence of the field (record check `ks_history_is_a_prescribed_background` of `Revision/field_equations_a4/reports/ks-source-conditions.json`; Chapter 17). **NOT COMPUTED.**

The conclusion. Since condition 1 fails exactly inside the patch, conditions 2 and 3 cannot help there: whatever the rates and however far from equilibrium, no process of the theory as built makes a net charge at any point inside the patch $0 < z < \pi/2$ of one universe, and under the ASSUMED no-flux condition the total charge of the universe cannot change either. The flow through the brane is the one open door: the record neither excludes it nor describes it as a process (OPEN).

Sakharov condition	**this theory**	**status**	**record check**
1. a process changes the number	the local law $\partial_\mu(\cos z\, J^\mu) = 0$ holds for every a_4; the total Q is constant only under the no-flux condition at the brane	inside the patch: FAILS (PROVED); total charge: FAILS under the no-flux condition (ASSUMED; OPEN)	`u1_noether_matrix_identity`
2. C and CP violated	the commuting field: $\mathcal{C}_+$ is exact and reverses the charge, so a C-symmetric start keeps zero charge; the quantised field has only the mass-reversing conjugation; no rates computed	dirac16complex00: FAILS (PROVED: $\mathcal{C}_+$ exact, Proposition 2); dirac16complex: NOT COMPUTED (no rates; only the mass-reversing conjugation)	`representation_real`, `spinor_connection_real`, `bilinears_under_charge_conjugation`, `quantum_charge_conjugation_unitary_type`

Sakharov condition	this theory	status	record check
3. out of equilibrium	no rate computed; the Kohn-Sham history is a prescribed background	NOT COMPUTED	`ks_history_is_a_prescribed_background`

21.32 What a charge-violating term would look like

Condition 1 could be met only by adding to the theory a term that is **not** invariant under $\Psi \to e^{i\alpha}\Psi$. The simplest candidates are **Majorana-type mass terms** $\Psi^T M \Psi$ with a constant 16×16 matrix M, built from Ψ twice and without $\Psi^\dagger$. Under $\Psi \to e^{i\alpha}\Psi$ such a term is multiplied by $e^{i\alpha}e^{i\alpha} = e^{2i\alpha}$: it carries U(1) charge 2. This section finds every such term that respects the rotations and boosts of the theory. It is a statement about possible additions to the theory, not about the theory as built.

The condition, line by line. A rotation or boost acts as $\Psi \to R\Psi$ with $R = e^{\theta S^{ab}}$. The term is invariant when $(R\Psi)^T M (R\Psi) = \Psi^T R^T M R \Psi$ equals $\Psi^T M \Psi$ for every Ψ, that is $R^T M R = M$. For a small angle θ, $R = 1 + \theta S + \dots$ (the dots are terms with θ^2 and higher powers), and

$$R^T M R = (1 + \theta S^T) M (1 + \theta S) + \dots = M + \theta\,(S^T M + M S) + \dots ,$$

so the term of first order must vanish:

$$(S^{ab})^T M + M S^{ab} = 0 \qquad \text{for all 28 generators } S^{ab}.$$

For the 256 entries of M these are $28 \times 256 = 7168$ linear equations.

Their solutions. First, $C S^{ab} C = -(S^{ab})^T$ for $a \neq b$:

$$C S^{ab} C = \tfrac{1}{2} C\gamma^a C C \gamma^b C = \tfrac{1}{2}\big(-(\gamma^a)^T\big)\big(-(\gamma^b)^T\big) = \tfrac{1}{2}(\gamma^b\gamma^a)^T = -\tfrac{1}{2}(\gamma^a\gamma^b)^T = -(S^{ab})^T$$

(insert $CC = 1$ in the middle; (R3) twice; the transpose of a product reverses the order; $\gamma^b\gamma^a = -\gamma^a\gamma^b$ for $a \neq b$). Then the condition $S^T M + M S = 0$ with $S^T = -CSC$ reads $-CSCM + MS = 0$; multiplying from the left by C gives $-SCM + CMS = 0$, that is

$$(CM)\, S^{ab} = S^{ab}\, (CM) \qquad \text{for all } S^{ab}:$$

M solves the equations exactly when CM commutes with every generator. The matrices that commute with all 28 generators form a space of dimension 2, spanned by 1 and Γ (equivalently by the projectors onto the two chiral halves; record check `spin_commutant_dimension_2` of `Revision/algebra/reports/python-algebra.json`: the 16 components split under Spin(4,4) into two inequivalent halves). Hence $CM = c_1 1 + c_2 \Gamma$, and multiplying by C: $M = c_1 C + c_2 C\Gamma$. **The invariant Majorana-type matrices are exactly the combinations of C and $C\Gamma$.** Notebook 21d solves the 7168 equations exactly and finds a space of dimension 2 spanned by them (In [16]); it also checks a finite rotation-boost R, built with $e^{\theta S} = \cos(\theta/2) + 2\sin(\theta/2)S$ when $S^2 = -\frac{1}{4}$

(a rotation) and $e^{\theta S} = \cosh(\theta/2) + 2\sinh(\theta/2)S$ when $S^2 = +\frac{1}{4}$ (a boost). (These follow from the power series of the exponential, as Euler's formula does, because $(2S)^2 = -1$ or $+1$. And $S^2 = \frac{1}{4}\gamma^a\gamma^b\gamma^a\gamma^b = -\frac{1}{4}\eta^{aa}\eta^{bb}$: a rotation when the two directions are both space-like or both time-like, a boost otherwise.)

For anticommuting components these terms vanish, line by line. With $\Psi_r\Psi_c = -\Psi_c\Psi_r$ and hence $\Psi_r\Psi_r = 0$:

$$\Psi^T M \Psi = \sum_{r,c} \Psi_r M_{rc} \Psi_c = \sum_{r<c} (M_{rc} - M_{cr}) \Psi_r \Psi_c$$

(the diagonal terms vanish; a term with $r > c$ is rewritten as $-M_{rc}\Psi_c\Psi_r$ and the summation letters renamed). Only the antisymmetric part of M survives. C is symmetric (R3) and so is $C\Gamma$ ($(C\Gamma)^T = \Gamma^T C^T = \Gamma C = C\Gamma$ by (R2)). So **the field dirac16complex has no Majorana-type mass term at all.** For the commuting field dirac16complex00 the terms $\Psi^T C\Psi$ and $\Psi^T C\Gamma\Psi$ are nonzero and carry charge 2 (In [16] and In [17], figure 21d.7). Adding one of them to the Lagrangian (together with its complex conjugate, to keep $\mathcal{L}$ real) would break the U(1) symmetry and let the charge change in steps of 2. That this would be consistent, or that nature contains such a term, is a HYPOTHESIS that nothing in the Revision record examines; for the anticommuting field a charge-violating term would need derivatives, more fields or new structures, which are not classified here (OPEN).

statement	status	where it is verified
invariance needs $S^T M + MS = 0$; solutions exactly the span of C and $C\Gamma$	PROVED (above); the 7168 equations solved exactly	`Revision/algebra/reports/python-algebra.json`, check `spin_com mutant_dimension_2`; Notebook 21d, In [16]
a finite rotation-boost keeps C and $C\Gamma$	COMPUTED (to rounding)	Notebook 21d, In [16]
C, $C\Gamma$ symmetric: no Majorana-type mass term for anticommuting components; charge 2 for commuting ones	PROVED (above); COMPUTED	Notebook 21d, In [16] and In [17]

21.33 Example: Notebook 21d computes the three conditions and the scorecard

Notebook 21d computes the worked examples of Sections 21.3, 21.5, 21.6, 21.31 and 21.32 and draws the scorecard. It counts $\mathcal{B}$, L and Q_{el} in four processes with exact fractions; it proves the net baryon number of the decay model with sympy; it checks the equilibrium surplus and evaluates the worked example; it solves the rate model in closed form with the incomplete gamma function and integrates it with RK4; then it turns to the theory: it applies condition 1 with the record's verdict, evaluates the record's Lagrangian at a point of the author's metric for Ψ, Ψ^*, $\Gamma\Psi$ and $\Gamma\Psi^*$, solves for every invariant Majorana-type matrix, and builds the scorecard from the PASS verdicts of the record. It uses no measured number. It runs in about 20 seconds; the times measured on the build computer, which depend on how busy that computer is, are recorded in the notebook's provenance file `Revision/textbook/notebooks/21d_sakharov_scorecard.PROVENANCE.md`. It needs no Rust, prints 19 PASS lines and draws eight figures.

21.34 How to run Notebook 21d

Step 1. What this notebook does and what it needs.

Notebook 21d (Sakharov's three conditions and the scorecard of this theory) is the file `Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb` of the repository Dirac_claude. It teaches the three conditions that a process making more matter than antimatter must meet (Sakharov 1967) with exact toy computations of its own: the bookkeeping of baryon number, lepton number and electric charge in four processes; a decay model whose net baryon number is the product of the baryon-number violation and the particle-antiparticle difference of the decay probabilities (derived with sympy); the Fermi-Dirac occupations of particles and antiparticles, which agree when the chemical potential vanishes; and a two-equation rate model of decays out of equilibrium whose final asymmetry is found in closed form and by Runge-Kutta integration. No measured number is used. It then applies the three conditions to this theory with the verdicts of the Revision record: the exact local conservation law of the U(1) charge, the same-mass conjugation of the commuting field as an exact symmetry of the Lagrangian of the record at a point of the author's metric, the mass-reversing conjugation of the quantised field, the two invariant Majorana-type mass matrices C and C Gamma (charge 2, absent for anticommuting components), and draws the scorecard: the theory as built does not solve the matter-antimatter problem. Eight teaching plots. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 21d_sakharov_scorecard.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar

of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21d_sakharov_scorecard.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 21d_sakharov_scorecard.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/21d.captions.json`
- `Revision/textbook/figures/21d_1_bookkeeping.png`
- `Revision/textbook/figures/21d_2_decay_asymmetry.png`
- `Revision/textbook/figures/21d_3_equilibrium_occupations.png`
- `Revision/textbook/figures/21d_4_rate_model_histories.png`
- `Revision/textbook/figures/21d_5_washout_efficiency.png`
- `Revision/textbook/figures/21d_6_lagrangian_maps.png`
- `Revision/textbook/figures/21d_7_majorana_terms.png`
- `Revision/textbook/figures/21d_8_scorecard.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS the eight figure files of notebook 21d exist
ALL 19 CHECKS PASSED (notebook 21d)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 21d_sakharov_scorecard.ipynb
python -m nbconvert --execute --inplace 21d_sakharov_scorecard.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.

- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder `Revision/algebra`, `Revision/theory`, `Revision/pairing`, `Revision/lead_checks` or `Revision/field_equations_a4`: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder `Revision/textbook/notebooks`.

21.35 Notebook 21d: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The part of the universe we can observe is made of **matter** (protons, neutrons,
electrons); **antimatter** (antiprotons, positrons, ...) appears only for short
times, made in collisions. Why there is more matter than antimatter is an open
question of physics. In 1967 Andrei Sakharov showed that any process that makes
more matter than antimatter out of a symmetric start must meet **three
conditions**. This notebook

1. counts the conserved numbers (baryon number $B$, lepton number $L$, electric
   charge $Q$) in four processes, and shows what a violation of $B$ looks like;
2. computes, with sympy, the net baryon number made by the decays of a toy
   particle and its antiparticle: it is the product of the **violation of $B$**
   (condition 1) and of the **difference between particle and antiparticle**
   (condition 2, C and CP violation);
3. shows with the Fermi-Dirac occupations that in **thermal equilibrium** without
   a conserved $B$ particles and antiparticles are equally common, and solves a
   two-equation rate model, in closed form and with the Runge-Kutta method, in
   which an asymmetry survives only when the decays happen **out of equilibrium**
   (condition 3);
4. applies the three conditions to the theory of this book, using the verdicts of
   the Revision record: the U(1) charge obeys an exact local conservation law
   (condition 1 fails inside the patch $0 < z < \pi/2$, off the brane; the total
   charge is constant only under the ASSUMED no-flux condition at the brane
   $z = \pi/2$),
   the same-mass conjugation is an exact symmetry of the commuting field, the
   quantised field has only the mass-reversing conjugation, and no departure from
   equilibrium has been computed;
5. finds every Majorana-type mass matrix that a charge-violating term could use
   (exactly two, $C$ and $C\Gamma$; both vanish for anticommuting components);
6. draws the **scorecard**: the theory as built does not solve the
```

```
   matter-antimatter problem.

The toy computations use no measured number. Eight teaching plots.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 21d (Sakharov's three conditions and the scorecard of this theory) is the file
‘Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb‘ of the repository
Dirac_claude. It teaches the three conditions that a process making more matter than
antimatter must meet (Sakharov 1967) with exact toy computations of its own: the
bookkeeping of baryon number, lepton number and electric charge in four processes; a
decay model whose net baryon number is the product of the baryon-number violation and the
particle-antiparticle difference of the decay probabilities (derived with sympy); the
Fermi-Dirac occupations of particles and antiparticles, which agree when the chemical
potential vanishes; and a two-equation rate model of decays out of equilibrium whose
final asymmetry is found in closed form and by Runge-Kutta integration. No measured
number is used. It then applies the three conditions to this theory with the verdicts of
the Revision record: the exact local conservation law of the U(1) charge, the same-mass
conjugation of the commuting field as an exact symmetry of the Lagrangian of the record
at a point of the author's metric, the mass-reversing conjugation of the quantised field,
the two invariant Majorana-type mass matrices C and C Gamma (charge 2, absent for
anticommuting components), and draws the scorecard: the theory as built does not solve
the matter-antimatter problem. Eight teaching plots. It needs a computer with Windows 11,
macOS or Linux, an internet connection for the installation, the program Git, and Python
3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at
exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the other
packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
```

```
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
    cd Revision/textbook/notebooks
    jupyter lab 21d_sakharov_scorecard.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 20 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 21d_sakharov_scorecard.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 21d_sakharov_scorecard.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/21d.captions.json‘
- ‘Revision/textbook/figures/21d_1_bookkeeping.png‘
- ‘Revision/textbook/figures/21d_2_decay_asymmetry.png‘
- ‘Revision/textbook/figures/21d_3_equilibrium_occupations.png‘
- ‘Revision/textbook/figures/21d_4_rate_model_histories.png‘
- ‘Revision/textbook/figures/21d_5_washout_efficiency.png‘
- ‘Revision/textbook/figures/21d_6_lagrangian_maps.png‘
- ‘Revision/textbook/figures/21d_7_majorana_terms.png‘
- ‘Revision/textbook/figures/21d_8_scorecard.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the

notebook (the output of its last code cell) you must see exactly these lines:

```
    PASS the eight figure files of notebook 21d exist
    ALL 19 CHECKS PASSED (notebook 21d)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
      python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
      python -m jupyterlab 21d_sakharov_scorecard.ipynb
      python -m nbconvert --execute --inplace 21d_sakharov_scorecard.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- "FileNotFoundError" naming a file in the folder ‘Revision/algebra‘, ‘Revision/theory‘, ‘Revision/pairing‘, ‘Revision/lead_checks‘ or ‘Revision/field_equations_a4‘: the notebook was opened outside the repository, or the repository is incomplete; clone the repository again and open the notebook from its folder ‘Revision/textbook/notebooks‘.

The first code cell below (the set-up cell) repeats these instructions as comments; then

it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 21d (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 21d (Sakharov's three conditions and the scorecard of this theory) is the file
# Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb of the repository
# Dirac_claude. It teaches the three conditions that a process making more matter than
# antimatter must meet (Sakharov 1967) with exact toy computations of its own: the
# bookkeeping of baryon number, lepton number and electric charge in four processes; a
# decay model whose net baryon number is the product of the baryon-number violation and
# the particle-antiparticle difference of the decay probabilities (derived with sympy);
# the Fermi-Dirac occupations of particles and antiparticles, which agree when the
# chemical potential vanishes; and a two-equation rate model of decays out of equilibrium
# whose final asymmetry is found in closed form and by Runge-Kutta integration. No
# measured number is used. It then applies the three conditions to this theory with the
# verdicts of the Revision record: the exact local conservation law of the U(1) charge,
# the same-mass conjugation of the commuting field as an exact symmetry of the Lagrangian
# of the record at a point of the author's metric, the mass-reversing conjugation of the
# quantised field, the two invariant Majorana-type mass matrices C and C Gamma (charge 2,
# absent for anticommuting components), and draws the scorecard: the theory as built does
# not solve the matter-antimatter problem. Eight teaching plots. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports numpy, sympy, mpmath and matplotlib; the
# other packages run Jupyter, the program that shows and runs notebooks. It does not need
# Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
```

```
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
```

```
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 21d_sakharov_scorecard.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 20
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
```

```
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 21d_sakharov_scorecard.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 21d_sakharov_scorecard.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/21d.captions.json
# - Revision/textbook/figures/21d_1_bookkeeping.png
# - Revision/textbook/figures/21d_2_decay_asymmetry.png
# - Revision/textbook/figures/21d_3_equilibrium_occupations.png
# - Revision/textbook/figures/21d_4_rate_model_histories.png
# - Revision/textbook/figures/21d_5_washout_efficiency.png
# - Revision/textbook/figures/21d_6_lagrangian_maps.png
# - Revision/textbook/figures/21d_7_majorana_terms.png
# - Revision/textbook/figures/21d_8_scorecard.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS the eight figure files of notebook 21d exist
#     ALL 19 CHECKS PASSED (notebook 21d)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
```

```
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/21d_sakharov_scorecard.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 21d_sakharov_scorecard.ipynb
#     python -m nbconvert --execute --inplace 21d_sakharov_scorecard.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - "FileNotFoundError" naming a file in the folder Revision/algebra, Revision/theory,
#   Revision/pairing, Revision/lead_checks or Revision/field_equations_a4: the notebook
#   was opened outside the repository, or the repository is incomplete; clone the
#   repository again and open the notebook from its folder Revision/textbook/notebooks.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "21d"  # this notebook: chapter 21, example d
```

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 21d complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Quark**: a constituent of protons and neutrons. The up quark $u$ has electric
  charge $+2/3$, the down quark $d$ has $-1/3$ (in units of the proton charge);
  every quark has **baryon number** $B = 1/3$. An **antiquark** ($\bar u$, $\bar d$)
  has the opposite numbers.
- **Baryon**: a particle made of three quarks, such as the proton $p = uud$ and the
  neutron $n = udd$ ($B = 1$); a **meson** (pion $\pi$) is a quark and an antiquark
  ($B = 0$).
- **Lepton number** $L$: $+1$ for the electron $e^-$ and the neutrino $\nu_e$,
  $-1$ for the positron $e^+$ and the antineutrino $\bar\nu_e$, $0$ for quarks and
  the photon $\gamma$.
- **Conserved number**: a number whose total before a process equals its total
  after it, in every process that happens.
- **Branching probability**: the probability that a decaying particle decays in one
  particular way (one **channel**).
- **C** (charge conjugation): the exchange of particles and antiparticles; **CP**:
  C combined with a mirror reflection of space. A theory **violates** C (or CP)
  when the exchanged process does not happen with the same probability.
- **Thermal equilibrium** at temperature $T$: the state that a system reaches when
  its reactions run forwards and backwards equally often. **Fermi-Dirac
  occupation**: the probability $f(E) = 1/(e^{(E - \mu)/T} + 1)$ that a state of
  energy $E$ holds a fermion; $\mu$ is the **chemical potential** of a conserved
  number (Boltzmann's constant is 1).
- **Rate**: a number per unit time; here the time is measured in units of the
  cooling time of the toy universe.
- **U(1) charge** $Q$ of the field of this theory: the number that belongs to the phase
  symmetry $\Psi \to e^{i\alpha}\Psi$, $Q = \int\cos z\,\Psi^\dagger B\Psi\,d^7x$.
  Its local conservation law is PROVED (chapter 21); the total $Q$ is constant only
  if no charge flows through the brane $z = \pi/2$, and this no-flux condition is
  ASSUMED.
- **Majorana-type term**: a term $\Psi^TM\Psi$ built from $\Psi$ twice (no
  $\Psi^\dagger$); under $\Psi \to e^{i\alpha}\Psi$ it is multiplied by
  $e^{2i\alpha}$, so it carries U(1) charge 2.
- **Status labels**: PROVED (exact), COMPUTED (numerical), ASSUMED (an input),
  HYPOTHESIS (not derived), OPEN (not answered), NOT COMPUTED (no computation
  exists).
```

Text cell:

```
## 4. The physical and mathematical situation

**Sakharov's three conditions** (A. D. Sakharov, 1967; quoted here as known
physics, and illustrated by the toy computations of this notebook). Suppose the
universe starts with equal amounts of matter and antimatter. To end with more
matter,

1. some process must **change the baryon number** $B$ (otherwise the total $B$,
```

```
   zero at the start, stays zero);
2. **C and CP must be violated** (otherwise every process that makes baryons has a
   mirror process that makes the same number of antibaryons at the same rate);
3. these processes must happen **out of thermal equilibrium** (in equilibrium a
   number that is not conserved has zero chemical potential, and particles and
   antiparticles are equally common).

**The theory of this book** has no quarks and no baryons. The only number of this
kind it has is the U(1) charge $Q$ of its field; the Revision record proves that
$Q$ obeys an exact local conservation law at every point of the patch
$0 < z < \pi/2$ (off the brane $z = \pi/2$) in the author's metric, for every
history $a_4(x_4)$, in particular the one in which the three extra times
$x5, x6, x7$ deflate exponentially: inside the patch, charge can only flow from
one place to another. The total $Q$ of a universe is constant only if no charge
flows through the brane $z = \pi/2$; this no-flux condition is ASSUMED, not
derived. The notebook first teaches the three conditions on toy models and then
asks, condition by condition, what this theory does.

**Honesty.** The toy models (a decay model and a rate model) are illustrations
with ASSUMED equations; they are not part of this theory. Every statement about
this theory comes from a check of the Revision record that the notebook reads, or
from a computation of the notebook that reproduces one.
```

Text cell:

```
## 5. The Revision records

The next cell reads the verdicts of the Revision reports that this notebook uses:
the lead checks of charge conjugation and U(1) conservation, the pairing record,
the algebra record and the record of the Kohn-Sham source conditions. It prints
the verdicts of the checks the scorecard will rest on.
```

In [2]:

```
from fractions import Fraction  # exact fractions such as 1/3

import mpmath  # numbers with many digits and special functions
import numpy as np  # numbers, arrays, matrices
import sympy as sp  # exact algebra


def load_checks(path):
    """name -> verdict (upper case) of every check of a Revision report."""
    entries = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
    return {c["name"]: c["verdict"].upper() for c in entries}


LEAD_FILE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
PAIR_FILE = "Revision/pairing/reports/wolfram-pairing.json"
ALGEBRA_FILE = "Revision/algebra/reports/python-algebra.json"
KS_FILE = "Revision/field_equations_a4/reports/ks-source-conditions.json"
LEAD, PAIR = load_checks(LEAD_FILE), load_checks(PAIR_FILE)
```

```
ALGEBRA, KS = load_checks(ALGEBRA_FILE), load_checks(KS_FILE)
USED = [(LEAD_FILE, LEAD, "u1_noether_matrix_identity"),
        (LEAD_FILE, LEAD, "representation_real"),
        (LEAD_FILE, LEAD, "spinor_connection_real"),
        (LEAD_FILE, LEAD, "bilinears_under_charge_conjugation"),
        (LEAD_FILE, LEAD, "quantum_charge_conjugation_unitary_type"),
        (PAIR_FILE, PAIR, "T1_current_primordial_commuting"),
        (PAIR_FILE, PAIR, "T1_current_primordial_grassmann"),
        (PAIR_FILE, PAIR, "T1_Lagrangian_primordial_commuting"),
        (ALGEBRA_FILE, ALGEBRA, "spin_commutant_dimension_2"),
        (KS_FILE, KS, "ks_history_is_a_prescribed_background")]
for path, verdicts, name in USED:
    say(f"record {path.split('/')[-1]}: {name} = {verdicts[name]}")
```

Out [2]:

```
record charge-conjugation-and-u1.json: u1_noether_matrix_identity = PASS
record charge-conjugation-and-u1.json: representation_real = PASS
record charge-conjugation-and-u1.json: spinor_connection_real = PASS
record charge-conjugation-and-u1.json: bilinears_under_charge_conjugation = PASS
record charge-conjugation-and-u1.json: quantum_charge_conjugation_unitary_type = PASS
record wolfram-pairing.json: T1_current_primordial_commuting = PASS
record wolfram-pairing.json: T1_current_primordial_grassmann = PASS
record wolfram-pairing.json: T1_Lagrangian_primordial_commuting = PASS
record python-algebra.json: spin_commutant_dimension_2 = PASS
record ks-source-conditions.json: ks_history_is_a_prescribed_background = PASS
```

Text cell:

```
## 6. Counting conserved numbers

The next cell computes $B$ and $Q$ of the proton, the neutron, the antiproton and
the three pions from their quark content (exact fractions), adds the leptons and
the photon, and counts $B$, $L$ and $Q$ before and after four processes:

- neutron decay $n \to p + e^- + \bar\nu_e$ (observed);
- proton-antiproton annihilation $p + \bar p \to \pi^+ + \pi^- + \pi^0$
  (observed);
- pair creation near a proton, $\gamma + p \to p + e^- + e^+$ (observed);
- proton decay $p \to e^+ + \pi^0$ (searched for and never observed; it is the
  kind of process that condition 1 asks for).

It checks that the first three conserve all three numbers and that proton decay
would change $B$ and $L$ by $-1$ each, while $Q$ and the difference $B - L$ stay.
```

In [3]:

```
QUARK = {"u": (Fraction(1, 3), Fraction(2, 3)),  # (B, Q) of the up quark
         "d": (Fraction(1, 3), Fraction(-1, 3))}  # (B, Q) of the down quark
```

```
def hadron(quarks="", antiquarks=""):
    """(B, L, Q) of a particle made of the given quarks and antiquarks."""
    B = sum(QUARK[x][0] for x in quarks) - sum(QUARK[x][0] for x in antiquarks)
    Q = sum(QUARK[x][1] for x in quarks) - sum(QUARK[x][1] for x in antiquarks)
    return (B, Fraction(0), Q)


PARTICLE = {"p": hadron("uud"), "n": hadron("udd"), "pbar": hadron("", "uud"),
            "pi+": hadron("u", "d"), "pi-": hadron("d", "u"), "pi0": hadron("u", "u"),
            "e-": (0, 1, -1), "e+": (0, -1, 1), "nubar": (0, -1, 0), "gamma": (0, 0, 0)}
PROCESSES = [("neutron decay", ["n"], ["p", "e-", "nubar"]),
             ("annihilation", ["p", "pbar"], ["pi+", "pi-", "pi0"]),
             ("pair creation", ["gamma", "p"], ["p", "e-", "e+"]),
             ("proton decay", ["p"], ["e+", "pi0"])]
changes, totals = {}, {}  # process -> changes of (B, L, Q); totals before, after
for label, before, after in PROCESSES:
    total_before = [sum(PARTICLE[x][k] for x in before) for k in range(3)]
    total_after = [sum(PARTICLE[x][k] for x in after) for k in range(3)]
    totals[label] = (total_before, total_after)
    changes[label] = [a - b for a, b in zip(total_after, total_before)]
    say(f"{label:14}: {' + '.join(before):10} -> {' + '.join(after):15} "
        f"change of (B, L, Q) = ({', '.join(str(c) for c in changes[label])})")
check(PARTICLE["p"] == (1, 0, 1) and PARTICLE["n"] == (1, 0, 0)
      and PARTICLE["pi+"][2] == 1 and PARTICLE["pi0"] == (0, 0, 0),
      "quark content: p has (B, L, Q) = (1, 0, 1), n (1, 0, 0), pi+ charge 1")
check(all(changes[p[0]] == [0, 0, 0] for p in PROCESSES[:3])
      and changes["proton decay"] == [-1, -1, 0],
      "three observed processes conserve B, L, Q; proton decay changes B and L by -1")
```

Out [3]:

```
neutron decay : n          -> p + e- + nubar  change of (B, L, Q) = (0, 0, 0)
annihilation  : p + pbar   -> pi+ + pi- + pi0 change of (B, L, Q) = (0, 0, 0)
pair creation : gamma + p  -> p + e- + e+     change of (B, L, Q) = (0, 0, 0)
proton decay  : p          -> e+ + pi0        change of (B, L, Q) = (-1, -1, 0)
PASS quark content: p has (B, L, Q) = (1, 0, 1), n (1, 0, 0), pi+ charge 1
PASS three observed processes conserve B, L, Q; proton decay changes B and L by -1
```

Text cell:

```
The next cell draws the bookkeeping as a table: for each process the totals of
$B$, $L$ and $Q$ before and after; a black frame marks every number that changes.
```

In [4]:

```
REACTIONS = ["$n \\to p\\,e^-\\bar\\nu_e$", "$p\\,\\bar p \\to \\pi^+\\pi^-\\pi^0$",
             "$\\gamma\\,p \\to p\\,e^-e^+$", "$p \\to e^+\\pi^0$ (never seen)"]
table = np.array([[float(totals[p[0]][side][k]) for k in range(3)
                   for side in (0, 1)] for p in PROCESSES])  # rows: processes
fig, ax = plt.subplots(figsize=(10.0, 4.2))
ax.imshow(table, cmap="RdBu_r", vmin=-2.5, vmax=2.5, aspect="auto")
```

```
for row in range(len(PROCESSES)):
    for col in range(6):
        value = table[row, col]
        ax.text(col, row, "0" if value == 0 else f"{value:+.0f}", ha="center",
                va="center", fontsize=11)
        if col % 2 == 1 and table[row, col] != table[row, col - 1]:  # a change
            for c in (col - 1, col):
                ax.add_patch(plt.Rectangle((c - 0.46, row - 0.44), 0.92, 0.88,
                                           fill=False, linewidth=2.5))
ax.set_xticks(range(6), ["$B$ before", "$B$ after", "$L$ before", "$L$ after",
                         "$Q$ before", "$Q$ after"])
ax.set_yticks(range(len(PROCESSES)), [f"{p[0]}\n{r}" for p, r in
                                      zip(PROCESSES, REACTIONS)], fontsize=9)
for x in (1.5, 3.5):
    ax.axvline(x, color="black", linewidth=1)
ax.grid(False)
ax.set_title("Bookkeeping: only proton decay would change $B$ (and $L$)")
save_figure(fig, "bookkeeping",
            "The totals of the baryon number $B$, the lepton number $L$ and the "
            "electric charge $Q$ before and after four processes (rows): neutron "
            "decay, proton-antiproton annihilation into three pions and pair "
            "creation near a proton (all observed), and proton decay into a "
            "positron and a pion (never observed); columns the number and the side "
            "of the process, colour and figure the total (red positive, blue "
            "negative, pure numbers). Black frames mark the numbers that change: "
            "only proton decay would change $B$ and $L$, the kind of process that "
            "Sakharov's first condition requires.")
```

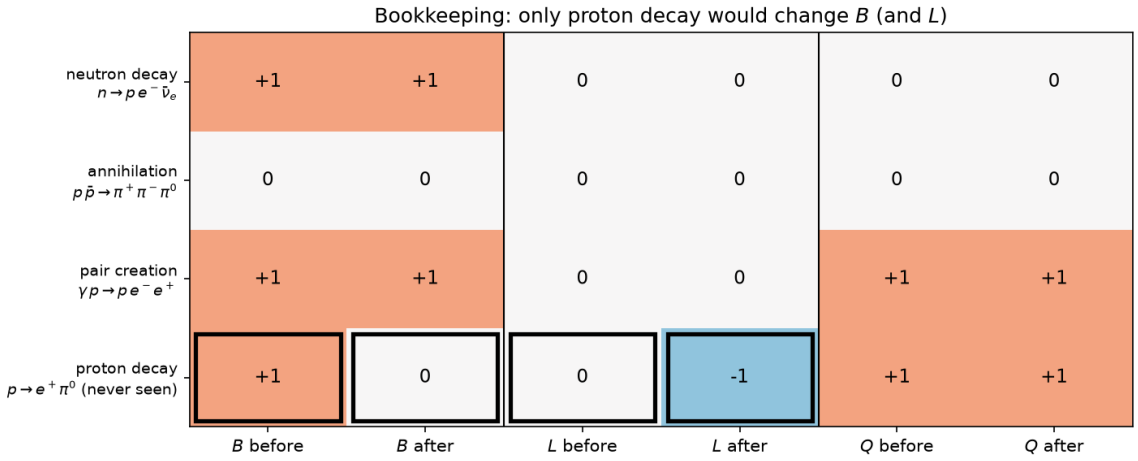

Figure 543: The totals of the baryon number B, the lepton number L and the electric charge Q before and after four processes (rows): neutron decay, proton-antiproton annihilation into three pions and pair creation near a proton (all observed), and proton decay into a positron and a pion (never observed); columns the number and the side of the process, colour and figure the total (red positive, blue negative, pure numbers). Black frames mark the numbers that change: only proton decay would change B and L, the kind of process that Sakharov's first condition requires. (Notebook 21d, figure 1.)

Out [4]:

```
Figure 21d.1 saved as Revision/textbook/figures/21d_1_bookkeeping.png
```

Text cell:

```
## 7. Conditions 1 and 2: a decay model

A heavy toy particle $X$ decays in two ways: with probability $r$ into a channel of
baryon number $B_1$, with probability $1 - r$ into a channel of baryon number
$B_2$. Its antiparticle $\bar X$ decays into the antichannels (baryon numbers
$-B_1$, $-B_2$) with probabilities $\bar r$ and $1 - \bar r$. (Ordinary quantum
field theory makes the *total* decay rates of $X$ and $\bar X$ equal; the
branching probabilities $r$ and $\bar r$ can differ only if C and CP are
violated.) Start with $N$ particles $X$ and $N$ antiparticles $\bar X$; line by
line:

1. One $X$ makes on average $rB_1 + (1 - r)B_2$ baryons.
2. One $\bar X$ makes on average $-\bar rB_1 - (1 - \bar r)B_2$.
3. The total is $N[rB_1 + (1 - r)B_2 - \bar rB_1 - (1 - \bar r)B_2] =
   N[(r - \bar r)B_1 - (r - \bar r)B_2] = N(r - \bar r)(B_1 - B_2)$.

The net baryon number is a **product**: it vanishes if $B_1 = B_2$ (then $X$
carries a conserved baryon number: **condition 1** fails) or if $r = \bar r$
(particle and antiparticle behave alike: **condition 2** fails). The next cell
checks this with sympy and evaluates one example: $X \to qq$ ($B_1 = 2/3$) and
$X \to \bar q\bar\ell$ ($B_2 = -1/3$) with $r = 0.6$, $\bar r = 0.5$.
```

In [5]:

```
r, rbar, B1, B2, N = sp.symbols("r rbar B1 B2 N", real=True)
from_X = r * B1 + (1 - r) * B2  # average baryon number made by one X
from_Xbar = -rbar * B1 - (1 - rbar) * B2  # by one antiparticle
net = sp.expand(N * (from_X + from_Xbar))  # N particles and N antiparticles
check(sp.expand(net - N * (r - rbar) * (B1 - B2)) == 0,
      "net baryon number = N (r - rbar) (B1 - B2) exactly")
example = net.subs({N: 1, B1: sp.Rational(2, 3), B2: sp.Rational(-1, 3),
                    r: sp.Rational(3, 5), rbar: sp.Rational(1, 2)})
report("net baryon number per pair X, Xbar for r = 0.6, rbar = 0.5", example)
check(example == sp.Rational(1, 10)
      and net.subs(B2, B1) == 0 and net.subs(rbar, r) == 0,
      "example 1/10 per pair; zero when B1 = B2 or when r = rbar")
```

Out [5]:

```
PASS net baryon number = N (r - rbar) (B1 - B2) exactly
RESULT net baryon number per pair X, Xbar for r = 0.6, rbar = 0.5 = 1/10
PASS example 1/10 per pair; zero when B1 = B2 or when r = rbar
```

Text cell:

```
The next cell draws the net baryon number per pair over all probabilities $r$
and $\bar r$, once for the baryon-number violating particle of the example
($B_1 = 2/3$, $B_2 = -1/3$) and once for a particle whose two channels have the
same baryon number ($B_1 = B_2 = 1/3$).
```

In [6]:

```
grid = np.linspace(0.0, 1.0, 201)  # values of r and rbar from 0 to 1
R, RBAR = np.meshgrid(grid, grid)
net_f = sp.lambdify((r, rbar, B1, B2), net.subs(N, 1), "numpy")
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.4), sharey=True)
for ax, (b1, b2), title in ((axes[0], (2 / 3, -1 / 3), "$B$ violated: $B_1 = 2/3$, "
                            "$B_2 = -1/3$"),
                           (axes[1], (1 / 3, 1 / 3), "$B$ conserved: "
                            "$B_1 = B_2 = 1/3$")):
    values = net_f(R, RBAR, b1, b2) + 0 * R  # + 0 * R: an array even when constant
    image = ax.pcolormesh(R, RBAR, values, cmap="RdBu_r", vmin=-1, vmax=1,
                          shading="auto")
    ax.plot([0, 1], [0, 1], "--", color="black", linewidth=1,
            label="$r = \\bar r$ (no C, CP violation)")
    ax.set_xlabel("branching probability $r$ of $X$")
    ax.set_title(title)
    ax.legend(fontsize=8, loc="upper left")
axes[0].set_ylabel("branching probability $\\bar r$ of $\\bar X$")
fig.colorbar(image, ax=axes, shrink=0.85, label="net baryon number per pair")
save_figure(fig, "decay_asymmetry",
            "The net baryon number $(r - \\bar r)(B_1 - B_2)$ made by the decays of "
            "one toy particle $X$ and its antiparticle, versus the branching "
            "probability $r$ of $X$ (horizontal) and $\\bar r$ of $\\bar X$ "
            "(vertical), colour the net number (red positive, blue negative). Left: "
            "channels with $B_1 = 2/3$ and $B_2 = -1/3$ (baryon number violated); "
            "the net number vanishes only on the diagonal $r = \\bar r$. Right: "
            "channels with equal baryon number; the net number is zero everywhere. "
            "An asymmetry needs both conditions 1 and 2.")
```

Out [6]:

```
Figure 21d.2 saved as Revision/textbook/figures/21d_2_decay_asymmetry.png
```

Text cell:

```
## 8. Condition 3: equilibrium

In thermal equilibrium at temperature $T$ a state of energy $E$ holds a particle of
baryon number $+1$ with probability $f_+ = 1/(e^{(E - \mu)/T} + 1)$ and the
antiparticle (baryon number $-1$, the same mass, hence the same energy) with
probability $f_- = 1/(e^{(E + \mu)/T} + 1)$; $\mu$ is the chemical potential of
the baryon number. Line by line:

1. $f_+ - f_- = \dfrac{e^{(E + \mu)/T} - e^{(E - \mu)/T}}{(e^{(E - \mu)/T} + 1)
   (e^{(E + \mu)/T} + 1)}$ (common denominator).
```

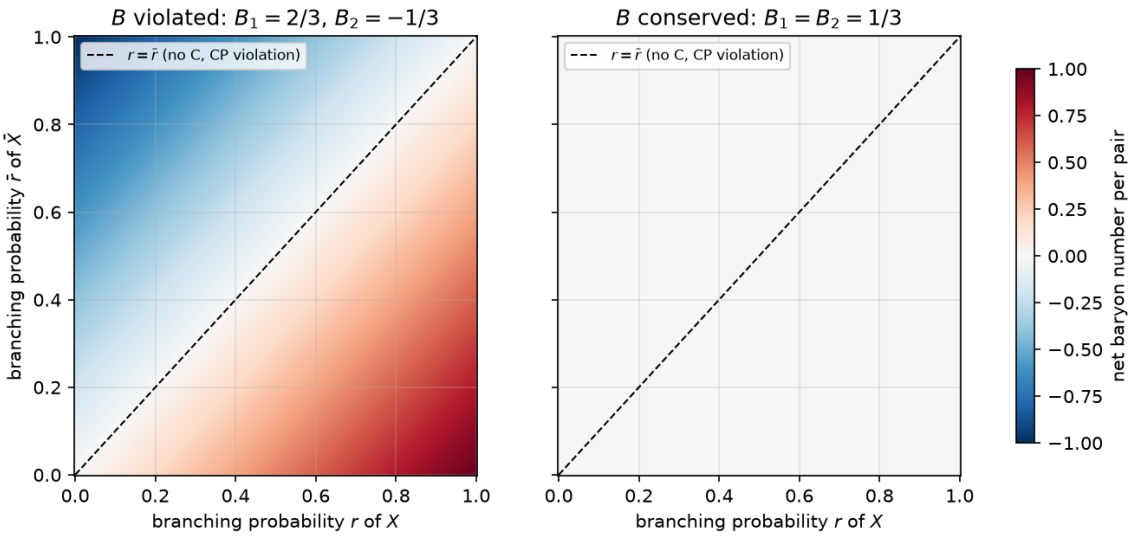

Figure 544: The net baryon number $(r - \bar{r})(B_1 - B_2)$ made by the decays of one toy particle X and its antiparticle, versus the branching probability r of X (horizontal) and $\bar{r}$ of $\bar{X}$ (vertical), colour the net number (red positive, blue negative). Left: channels with $B_1 = 2/3$ and $B_2 = -1/3$ (baryon number violated); the net number vanishes only on the diagonal $r = \bar{r}$. Right: channels with equal baryon number; the net number is zero everywhere. An asymmetry needs both conditions 1 and 2. (Notebook 21d, figure 2.)

```
2. Multiply out the denominator: $e^{2E/T} + e^{(E - \mu)/T} + e^{(E + \mu)/T} +
   1$; divide numerator and denominator by $e^{E/T}$:
   $f_+ - f_- = \dfrac{2\sinh(\mu/T)}{2\cosh(E/T) + 2\cosh(\mu/T)} =
   \dfrac{\sinh(\mu/T)}{\cosh(E/T) + \cosh(\mu/T)}$.
3. If reactions that change $B$ run back and forth in equilibrium, $B$ is not
   conserved and has no chemical potential: $\mu = 0$, so $\sinh 0 = 0$ and
   $f_+ = f_-$ for **every** energy. No asymmetry.

The next cell checks the formula of line 2 with sympy and evaluates the worked
example $E = 2T$: at $\mu = 0$ both occupations are $1/(e^2 + 1)$; a chemical
potential $\mu = 0.1T$ would give a surplus of particles.
```

In [7]:

```
E_T, mu_T = sp.symbols("E_T mu_T", real=True)  # E / T and mu / T
f_plus = 1 / (sp.exp(E_T - mu_T) + 1)
f_minus = 1 / (sp.exp(E_T + mu_T) + 1)
difference = sp.sinh(mu_T) / (sp.cosh(E_T) + sp.cosh(mu_T))
check(sp.simplify((f_plus - f_minus - difference).rewrite(sp.exp)) == 0,
      "f+ - f- = sinh(mu/T) / (cosh(E/T) + cosh(mu/T)) exactly")
at_zero = [float(f.subs({E_T: 2, mu_T: 0})) for f in (f_plus, f_minus)]
at_tenth = [float(f.subs({E_T: 2, mu_T: sp.Rational(1, 10)}))
            for f in (f_plus, f_minus)]
report("f+ = f- at E = 2T, mu = 0", f"{at_zero[0]:.6f}")
report("f+, f- at E = 2T, mu = 0.1 T", f"{at_tenth[0]:.6f}, {at_tenth[1]:.6f}")
report("surplus of particles per state", f"{at_tenth[0] - at_tenth[1]:.6f}")
```

```
check(at_zero[0] == at_zero[1] and abs(at_zero[0] - 1 / (np.exp(2) + 1)) < 1e-15
      and at_tenth[0] > at_tenth[1],
      "mu = 0: equal occupations 1/(e^2 + 1); mu > 0: more particles")
```

Out [7]:

```
PASS f+ - f- = sinh(mu/T) / (cosh(E/T) + cosh(mu/T)) exactly
RESULT f+ = f- at E = 2T, mu = 0 = 0.119203
RESULT f+, f- at E = 2T, mu = 0.1 T = 0.130108, 0.109097
RESULT surplus of particles per state = 0.021012
PASS mu = 0: equal occupations 1/(e^2 + 1); mu > 0: more particles
```

Text cell:

```
The next cell draws $f_+ - f_-$ against $E/T$ for four values of $\mu/T$.
```

In [8]:

```
energies = np.linspace(0.0, 8.0, 321)  # E / T from 0 to 8
diff_f = sp.lambdify((E_T, mu_T), difference, "numpy")
fig, ax = plt.subplots(figsize=(7.5, 4.3))
for mu_value, style in ((0.2, "-"), (0.1, "--"), (0.05, "-."), (0.0, ":")):
    ax.plot(energies, diff_f(energies, mu_value) + 0 * energies, style, linewidth=2,
            label=f"$\\mu/T = {mu_value}$")
ax.plot([2.0], [at_tenth[0] - at_tenth[1]], "o", color="black",
        label="worked example $E = 2T$, $\\mu = 0.1T$")
ax.set_xlabel("energy $E/T$")
ax.set_ylabel("$f_+ - f_-$ (particles minus antiparticles per state)")
ax.set_title("In equilibrium with $\\mu = 0$ there is no surplus")
ax.legend(fontsize=8)
save_figure(fig, "equilibrium_occupations",
            "The surplus $f_+ - f_- = \\sinh(\\mu/T)/(\\cosh(E/T) + \\cosh(\\mu/T))$ "
            "of particles over antiparticles per quantum state in thermal "
            "equilibrium, versus the energy $E/T$ of the state (horizontal) for the "
            "chemical potentials $\\mu/T = 0.2, 0.1, 0.05, 0$ (pure numbers). The "
            "dot is the worked example $E = 2T$, $\\mu = 0.1T$. When the reactions "
            "that change the baryon number are in equilibrium, $\\mu = 0$ and the "
            "surplus vanishes at every energy (dotted line).")
```

Out [8]:

```
Figure 21d.3 saved as Revision/textbook/figures/21d_3_equilibrium_occupations.png
```

Text cell:

```
## 9. Condition 3: decays out of equilibrium (a rate model)

A toy universe cools; the time $t$ is measured in units of its cooling time. Let
```

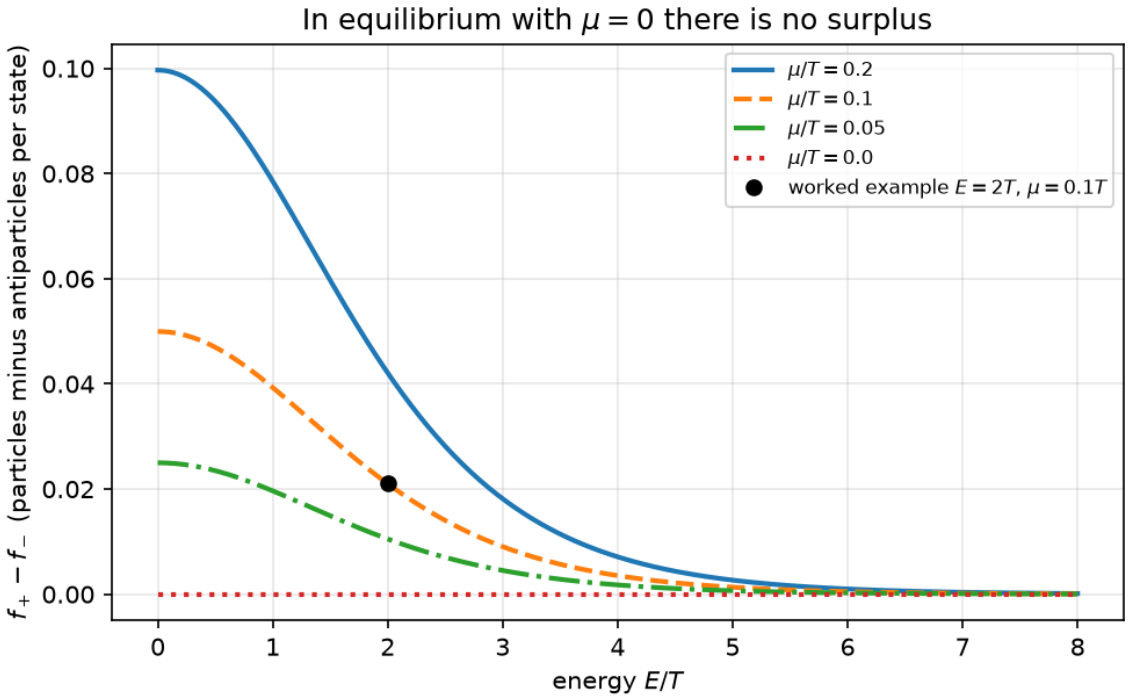

Figure 545: The surplus $f_+ - f_- = \sinh(\mu/T)/(\cosh(E/T)+\cosh(\mu/T))$ of particles over antiparticles per quantum state in thermal equilibrium, versus the energy E/T of the state (horizontal) for the chemical potentials $\mu/T = 0.2, 0.1, 0.05, 0$ (pure numbers). The dot is the worked example $E = 2T$, $\mu = 0.1T$. When the reactions that change the baryon number are in equilibrium, $\mu = 0$ and the surplus vanishes at every energy (dotted line). (Notebook 21d, figure 3.)

```
$n(t)$ be the number of heavy pairs $X$, $\bar X$ (per unit volume, in units of
their starting number) and $n_{eq}(t) = e^{-t}$ the number that equilibrium would
have (heavy particles become rare as the temperature falls). Decays and inverse
decays pull $n$ towards $n_{eq}$ at the rate $K$; each net decay of a pair makes
$\epsilon$ baryons (the number of section 7); inverse decays, which need $X$
particles in equilibrium, erase the asymmetry $a$ at the rate $Kn_{eq}$. The
ASSUMED model is
$$\frac{dn}{dt} = -K(n - n_{eq}),\qquad \frac{da}{dt} = \epsilon K(n - n_{eq})
- Kn_{eq}\,a,\qquad n(0) = 1,\ a(0) = 0 .$$

**Solving it exactly**, line by line.

1. Write $n = n_{eq} + \Delta$. Since $dn_{eq}/dt = -e^{-t}$, the first equation
   becomes $d\Delta/dt = -K\Delta + e^{-t}$ with $\Delta(0) = 0$.
2. Its solution is $\Delta(t) = (e^{-t} - e^{-Kt})/(K - 1)$ (for $K \ne 1$;
   differentiate to check: $(-e^{-t} + Ke^{-Kt})/(K - 1) = -K\Delta + e^{-t}$).
3. With $W(t) = K(1 - e^{-t})$, so $dW/dt = Kn_{eq}$, the second equation is
   $d(a\,e^{W})/dt = \epsilon K\Delta\,e^{W}$ (product rule), hence
   $a(t) = \epsilon\int_0^t K\Delta(s)\,e^{W(s) - W(t)}\,ds$.
4. As $t \to \infty$, $W(t) \to K$ and $W(s) - K = -Ke^{-s}$, so the final
   asymmetry is $a(\infty) = \epsilon\,\eta(K)$ with the **efficiency**
   $\eta(K) = \int_0^\infty K\Delta(s)\,e^{-Ke^{-s}}\,ds$.
5. Substitute $y = e^{-s}$ ($ds = -dy/y$):
   $\eta(K) = \frac{K}{K - 1}\int_0^1(1 - y^{K-1})\,e^{-Ky}\,dy$.
6. $\int_0^1 e^{-Ky}dy = (1 - e^{-K})/K$, and with $u = Ky$,
   $\int_0^1 y^{K-1}e^{-Ky}dy = K^{-K}\gamma(K, K)$, where $\gamma(s, x) =
   \int_0^x u^{s-1}e^{-u}du$ is the *lower incomplete gamma function* (mpmath's
   `gammainc(s, 0, x)`):
   $$\eta(K) = \frac{K}{K - 1}\Big(\frac{1 - e^{-K}}{K} - K^{-K}\gamma(K, K)\Big).$$

The next cell checks lines 1 to 3 with sympy and defines $\eta(K)$ with mpmath
(30 digits).
```

In [9]:

```
t, s_, K_s, eps_s = sp.symbols("t s K epsilon", positive=True)
Delta = (sp.exp(-t) - sp.exp(-K_s * t)) / (K_s - 1)  # line 2
W = K_s * (1 - sp.exp(-t))  # line 3
line2_ok = (sp.simplify(sp.diff(Delta, t) - (-K_s * Delta + sp.exp(-t))) == 0
            and Delta.subs(t, 0) == 0)
line3_ok = sp.simplify(sp.diff(W, t) - K_s * sp.exp(-t)) == 0
check(line2_ok and line3_ok, "rate model: Delta(t) and W(t) solve lines 1 to 3")
mpmath.mp.dps = 30  # work with 30 significant digits


def efficiency(K):
    """eta(K) = K/(K - 1) ((1 - e^-K)/K - K^-K gamma(K, K)), K != 1."""
    K = mpmath.mpf(K)
    incomplete = mpmath.gammainc(K, 0, K)  # the lower incomplete gamma(K, K)
    return float(K / (K - 1) * ((1 - mpmath.exp(-K)) / K - K ** (-K) * incomplete))


for K in (0.1, 3.0, 30.0):
```

```
    say(f"K = {K:5}: eta = {efficiency(K):.10f}")
```

Out [9]:

```
PASS rate model: Delta(t) and W(t) solve lines 1 to 3
K =   0.1: eta = 0.9955325865
K =   3.0: eta = 0.4110164748
K =  30.0: eta = 0.0344827586
```

Text cell:

```
The next cell integrates the two equations numerically with the classical
fourth-order Runge-Kutta method (RK4) for three rates $K = 0.1$ (slow decays),
$K = 3$ and $K = 30$ (fast decays, close to equilibrium), with $\epsilon = 1$, and
compares $a(\infty)$ with the closed form $\eta(K)$. It also runs two controls: with
$\epsilon = 0$ (no C, CP violation) the asymmetry stays exactly zero, and without
the erasing term the final asymmetry is exactly $\epsilon$ for every $K$ (because
then $da/dt = -\epsilon\,dn/dt$, so $a(\infty) = \epsilon(n(0) - n(\infty)) =
\epsilon$).
```

In [10]:

```
def integrate(K, eps=1.0, erase=True, keep_every=50):
    """RK4 for (n, a) from t = 0 to 60 + 40/K; returns the final state and the
    history (every keep_every-th step)."""
    t_end = 60.0 + 40.0 / K  # long enough for n and n_eq to be negligible
    steps = int(np.ceil(t_end / min(0.1 / K, 0.02)))  # step below 0.1/K
    dt = t_end / steps

    def rates(time, y):
        n_eq = np.exp(-time)
        return np.array([-K * (y[0] - n_eq),
                         eps * K * (y[0] - n_eq) - (K * n_eq * y[1] if erase else 0)])

    y, time, history = np.array([1.0, 0.0]), 0.0, [(0.0, 1.0, 0.0)]
    for step in range(steps):
        k1 = rates(time, y)
        k2 = rates(time + dt / 2, y + dt / 2 * k1)
        k3 = rates(time + dt / 2, y + dt / 2 * k2)
        k4 = rates(time + dt, y + dt * k3)
        y = y + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
        time += dt
        if (step + 1) % keep_every == 0:
            history.append((time, y[0], y[1]))
    return y, np.array(history)


RATES = (0.1, 3.0, 30.0)
runs = {K: integrate(K) for K in RATES}
for K in RATES:
    say(f"K = {K:5}: RK4 a(infinity) = {runs[K][0][1]:.10f}, "
```

```
        f"closed form {efficiency(K):.10f}")
check(all(abs(runs[K][0][1] - efficiency(K)) < 1e-7 for K in RATES),
      "RK4 agrees with the closed form eta(K) to 1e-7 for K = 0.1, 3, 30")
no_cp, _ = integrate(3.0, eps=0.0)
no_erase, _ = integrate(3.0, erase=False)
check(no_cp[1] == 0.0 and abs(no_erase[1] - 1.0) < 1e-9,
      "controls: eps = 0 gives a = 0 exactly; without erasing a(infinity) = eps")
```

Out [10]:

```
K =   0.1: RK4 a(infinity) = 0.9955325865, closed form 0.9955325865
K =   3.0: RK4 a(infinity) = 0.4110164753, closed form 0.4110164748
K =  30.0: RK4 a(infinity) = 0.0344827586, closed form 0.0344827586
PASS RK4 agrees with the closed form eta(K) to 1e-7 for K = 0.1, 3, 30
PASS controls: eps = 0 gives a = 0 exactly; without erasing a(infinity) = eps
```

Text cell:

```
The next cell draws the three histories: the number of heavy pairs $n$, its
equilibrium value $n_{eq}$ and the asymmetry $a/\epsilon$.
```

In [11]:

```
fig, axes = plt.subplots(2, 1, figsize=(8.0, 6.6), sharex=True)
times = np.linspace(0.0, 60.0, 601)
axes[0].semilogy(times, np.exp(-times), color="black", linewidth=1,
                 label="equilibrium $n_{eq} = e^{-t}$")
for K, style in zip(RATES, ["-", "--", ":"]):
    history = runs[K][1]
    shown = history[history[:, 0] <= 60.0]
    axes[0].semilogy(shown[:, 0], shown[:, 1], style, linewidth=2,
                     label=f"$n$, $K = {K}$")
    axes[1].plot(shown[:, 0], shown[:, 2], style, linewidth=2, label=f"$K = {K}$")
axes[0].set_ylim(1e-12, 2.0)
axes[0].set_ylabel("heavy pairs (log scale)")
axes[0].legend(fontsize=8, loc="lower left")
axes[1].set_xlabel("time $t$ (units of the cooling time)")
axes[1].set_ylabel("asymmetry $a/\\epsilon$")
axes[1].set_ylim(0.0, 1.05)
axes[1].legend(fontsize=8, loc="center right", bbox_to_anchor=(1.0, 0.68))
save_figure(fig, "rate_model_histories",
            "The rate model of decays out of equilibrium for three decay rates $K$. "
            "Top: the number $n$ of heavy pairs (coloured) and the equilibrium "
            "value $n_{eq} = e^{-t}$ (thin black), logarithmic scale. Bottom: the "
            "asymmetry $a/\\epsilon$. Horizontal axis the time in units of the "
            "cooling time (pure numbers). Slow decays ($K = 0.1$) happen late, "
            "far from equilibrium, when nothing erases their product: almost the "
            "whole $\\epsilon$ survives. Fast decays ($K = 30$) keep $n$ close to "
            "$n_{eq}$, and the inverse decays erase almost everything.")
```

Out [11]:

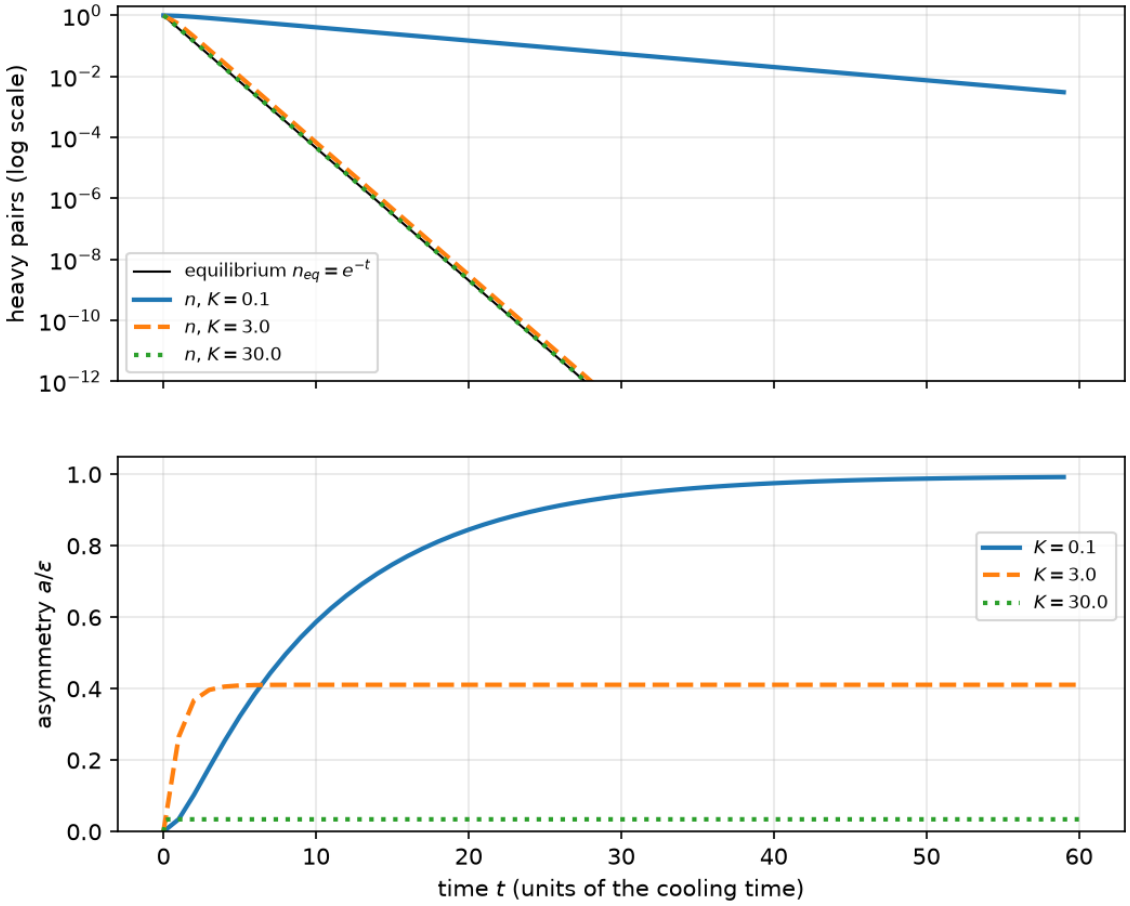

Figure 546: The rate model of decays out of equilibrium for three decay rates K. Top: the number n of heavy pairs (coloured) and the equilibrium value $n_{eq} = e^{-t}$ (thin black), logarithmic scale. Bottom: the asymmetry a/ϵ. Horizontal axis the time in units of the cooling time (pure numbers). Slow decays ($K = 0.1$) happen late, far from equilibrium, when nothing erases their product: almost the whole ϵ survives. Fast decays ($K = 30$) keep n close to n_{eq}, and the inverse decays erase almost everything. (Notebook 21d, figure 4.)

```
Figure 21d.4 saved as Revision/textbook/figures/21d_4_rate_model_histories.png
```

Text cell:

```
The next cell computes the efficiency $\eta(K) = a(\infty)/\epsilon$ for 24 rates
from $K = 0.01$ to $K = 1000$ with the closed form and draws it with the three RK4
results. It checks the two limits: $\eta \to 1$ for slow decays, and $\eta =
1/(K - 1)$ up to the exponentially small term $K^{-K}\gamma(K, K)$ for fast ones,
so the asymmetry disappears as the decays approach equilibrium.
```

In [12]:

```
K_grid = np.logspace(-2.0, 3.0, 24)  # 24 rates; K = 1 is not among them
eta = np.array([efficiency(K) for K in K_grid])
check(eta[0] > 0.99 and abs(efficiency(1000.0) * 999.0 - 1.0) < 1e-12
      and np.all(np.diff(eta) < 0),
      "efficiency: near 1 for K = 0.01, exactly 1/(K - 1) at K = 1000, decreasing")
fig, ax = plt.subplots(figsize=(7.5, 4.4))
ax.loglog(K_grid, eta, "-", linewidth=2, label="closed form $\\eta(K)$")
ax.loglog(K_grid[K_grid > 2], 1 / (K_grid[K_grid > 2] - 1), ":", color="black",
          label="$1/(K - 1)$: near equilibrium")
ax.loglog(RATES, [runs[K][0][1] for K in RATES], "o", color="#e34948",
          label="RK4 integration")
ax.set_xlabel("decay rate $K$ (units of the cooling rate)")
ax.set_ylabel("efficiency $a(\\infty)/\\epsilon$")
ax.set_title("Equilibrium erases the asymmetry")
ax.legend(fontsize=8, loc="lower left")
save_figure(fig, "washout_efficiency",
            "The efficiency $\\eta(K) = a(\\infty)/\\epsilon$ of the rate model: the "
            "fraction of the asymmetry made by the decays that survives, versus "
            "the decay rate $K$, both on logarithmic scales (pure numbers). Solid: "
            "the closed form with the incomplete gamma function; dots: the RK4 "
            "integrations; dotted: the limit $1/(K - 1)$. Slow decays (out of "
            "equilibrium) keep almost all of it; the closer the decays are to "
            "equilibrium (large $K$), the less survives: condition 3.")
```

Out [12]:

```
PASS efficiency: near 1 for K = 0.01, exactly 1/(K - 1) at K = 1000, decreasing
```

Out [12]:

```
Figure 21d.5 saved as Revision/textbook/figures/21d_5_washout_efficiency.png
```

Text cell:

```
## 10. This theory, condition 1: no process changes the U(1) charge locally
```

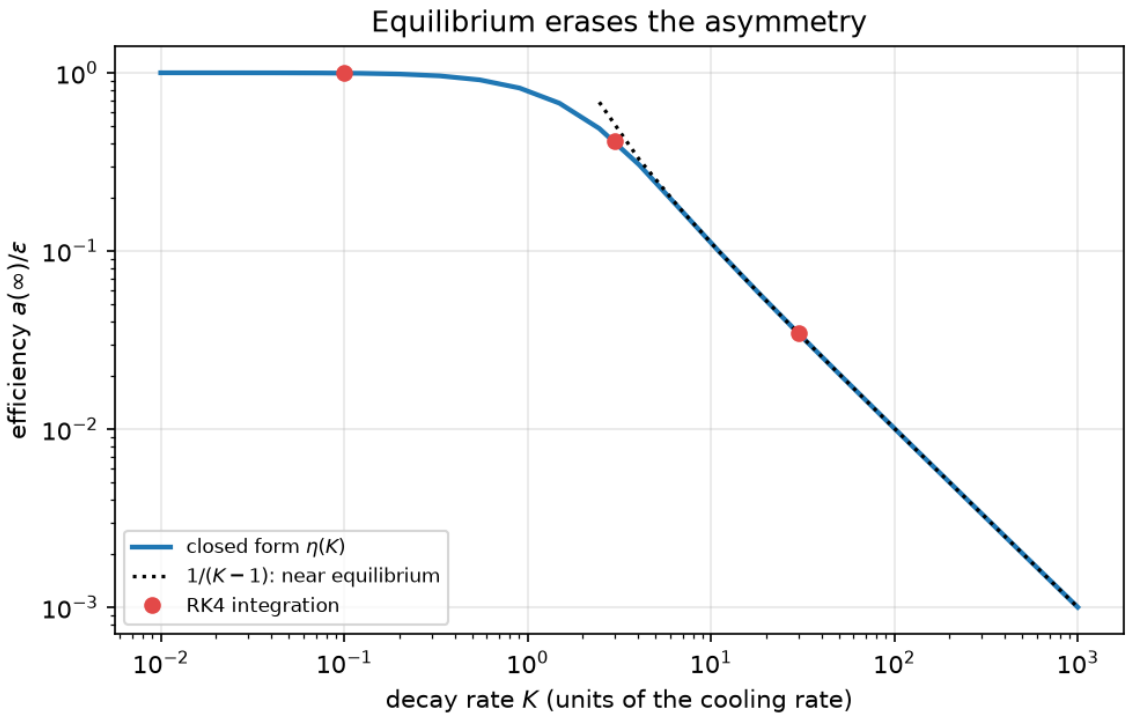

Figure 547: The efficiency $\eta(K) = a(\infty)/\epsilon$ of the rate model: the fraction of the asymmetry made by the decays that survives, versus the decay rate K, both on logarithmic scales (pure numbers). Solid: the closed form with the incomplete gamma function; dots: the RK4 integrations; dotted: the limit $1/(K-1)$. Slow decays (out of equilibrium) keep almost all of it; the closer the decays are to equilibrium (large K), the less survives: condition 3. (Notebook 21d, figure 5.)

```
The Revision record proves (check u1_noether_matrix_identity) the exact local
conservation law $\partial_\mu(\cos z\,J^\mu) = 0$ on every solution, at every
point of the patch $0 < z < \pi/2$ (off the brane), in the author's metric with an
arbitrary history $a_4(x_4)$: charge can only flow from one place to another, and
the U(1) charge $Q = \int\cos z\,\Psi^\dagger B\Psi\,d^7x$ of a universe changes
only by what flows through its boundary. At the brane $z = \pi/2$ the record does
not exclude such a flow (notebook 21b follows an exact solution whose charge
changes by the flux through the brane); the no-flux condition that would make $Q$
constant is ASSUMED, not derived. In the language of section 7: every process of
this theory, at any point of the patch $0 < z < \pi/2$, has channels of the
**same** charge, $B_1 = B_2$, so the factor $(B_1 - B_2)$ is zero, and the rate
model then starts with $\epsilon = 0$ and ends with $a = 0$, whatever the rates
and however far from equilibrium. The next cell checks that the record holds this
verdict and evaluates the decay formula and the rate model with this input.
```

In [13]:

```
Q_channel = sp.Symbol("Q_channel")  # the common charge of every channel
eps_theory = net.subs({N: 1, B1: Q_channel, B2: Q_channel})  # B1 = B2
a_theory, _ = integrate(0.1, eps=float(eps_theory))  # slow decays, far from equil.
say(f"net charge per pair with equal channels: {eps_theory}; rate model: "
    f"a(infinity) = {a_theory[1]}")
check(LEAD["u1_noether_matrix_identity"] == "PASS" and eps_theory == 0
      and a_theory[1] == 0.0,
      "local U(1) law: no process makes a net charge at any point of the patch "
      "0 < z < pi/2, condition 1 fails there (total charge: no flux through the "
      "brane ASSUMED)",
      record=f"{LEAD_FILE}, check u1_noether_matrix_identity")
```

Out [13]:

```
net charge per pair with equal channels: 0; rate model: a(infinity) = 0.0
PASS local U(1) law: no process makes a net charge at any point of the patch 0 < z <
    pi/2, condition 1 fails there (total charge: no flux through the brane ASSUMED)
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    u1_noether_matrix_identity
```

Text cell:

```
## 11. This theory, condition 2: the conjugations

Condition 2 asks whether particle and antiparticle behave differently. The next
cell evaluates the Lagrangian of the Revision record,
$$\mathcal{L}_{m,\lambda}[\Psi] = \cos z\,\Big[\tfrac12\sum_\mu\big(\bar\Psi
\gamma^\mu D_\mu\Psi - (D_\mu\bar\Psi)\gamma^\mu\Psi\big) - mS -
\tfrac\lambda2S^2\Big],\quad S = \bar\Psi\Psi,\quad \bar\Psi = \Psi^\dagger C,$$
with $D_\mu\Psi = \partial_\mu\Psi + \Omega_\mu\Psi$, $D_\mu\bar\Psi =
\partial_\mu\bar\Psi - \bar\Psi\Omega_\mu$, the record's spin connection
$\Omega_\mu$ (formula Omega_components) and the curved gammas $\gamma^\mu =
\gamma^{(\mu)}/f_\mu$, at one point of the author's metric ($H = 1/6$,
$z = 0.7$, $a_4 = 0.4$, $a_4' = 0.25$), for a fixed commuting field value
```

```
$\Psi$ and fixed first derivatives $\partial_\mu\Psi$ (random numbers with a fixed
seed), $m = 0.7$, $\lambda = 0.3$. It compares four numbers:

1. $\mathcal{L}_{m,\lambda}[\Psi]$ and $\mathcal{L}_{m,\lambda}[\Psi^*]$: they are
   **equal**, so the same-mass conjugation $\mathcal{C}_+$ ($\Psi^c = \Psi^*$) is
   an exact symmetry of the commuting field dirac16complex00. (Why: every matrix
   in $\mathcal{L}$, that is $C$, the gammas and $\Omega_\mu$, is real, so
   replacing $\Psi$ by $\Psi^*$ turns $\mathcal{L}$ into its complex conjugate;
   and $\mathcal{L}$ is real.) The map also reverses every current, so it turns
   every solution into a solution of the same theory with the opposite charge. A
   start in which every field configuration and its conjugate are equally likely
   therefore keeps the average charge zero at all times: for the commuting field
   **condition 2 fails** (PROVED).
2. $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$
   (theorem T1, record check T1_Lagrangian_primordial_commuting) and the same for
   $\Gamma\Psi^*$ (the conjugation $\mathcal{C}_-$): these maps reverse the mass.

For the **quantised** field dirac16complex the record proves (check
quantum_charge_conjugation_unitary_type) that the only conjugation compatible with
the canonical anticommutator is $\Psi \to \Gamma\Psi^{\dagger T}$, which reverses
the mass: within one universe of mass $m$ there is no same-mass conjugation of the
quantised field. Whether its particles and antiparticles react at different rates
is NOT COMPUTED: the record computes no reaction rates.
```

In [14]:

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
gamma = [np.array(mat, dtype=np.int64) for mat in fixture["gamma"]]  # x0 .. x7
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # C = g(x0) g(x1) g(x2) g(x3)
Gamma = C @ gamma[4] @ gamma[5] @ gamma[6] @ gamma[7]  # the chirality
THEORY = {f["key"]: f["wl"] for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
lagrangian_text = ("L = Cos[z] [ (1/2) sum_mu (Psibar gamma^mu D_mu Psi - (D_mu "
                   "Psibar) gamma^mu Psi) - m S - U(S) ]")
omega_text = ["(1/2) E^a4[x4] Sin[6 H x0]^(1/6) (a4",
              "g[xi].g[x4] + H g[xi].g[x0]) (i = 1, 2, 3)",
              "-(1/2) E^-a4[x4] Sin[6 H x0]^(1/6) (a4",
              "g[x4].g[xt] + H g[xt].g[x0]) (t = 5, 6, 7)", "Omega_x0 = Omega_x4 = 0"]
record_ok = (THEORY["Lagrangian"].startswith(lagrangian_text)
             and all(piece in THEORY["Omega_components"] for piece in omega_text))

H, z, a4, a4p = 1 / 6, 0.7, 0.4, 0.25  # the point of the author's metric
s16 = np.sin(z) ** (1 / 6)
f = [1 / np.tan(z)] + [np.exp(a4) * s16] * 3 + [1.0] + [np.exp(-a4) * s16] * 3
Omega = []  # the record's formula Omega_components at the point
for mu in range(8):
    if mu in (1, 2, 3):  # x1, x2, x3
        Omega.append(np.exp(a4) * s16 / 2 * (a4p * gamma[mu] @ gamma[4]
                                             + H * gamma[mu] @ gamma[0]))
    elif mu in (5, 6, 7):  # x5, x6, x7: the deflating extra times
        Omega.append(-np.exp(-a4) * s16 / 2 * (a4p * gamma[4] @ gamma[mu]
                                               + H * gamma[mu] @ gamma[0]))
```

```
    else:  # x4 and x0
        Omega.append(np.zeros((16, 16)))
gamma_up = [gamma[mu] / f[mu] for mu in range(8)]  # curved gammas at the point


def lagrangian(p, dp, m, lam):
    """The record's Lagrangian at the point for the field value p and the eight
    first derivatives dp[mu] (commuting components)."""
    bar = np.conj(p) @ C  # Psibar = Psi^dagger C
    kinetic = 0.0
    for mu in range(8):
        D = dp[mu] + Omega[mu] @ p  # D_mu Psi
        Dbar = np.conj(dp[mu]) @ C - bar @ Omega[mu]  # D_mu Psibar
        kinetic += (bar @ gamma_up[mu] @ D - Dbar @ gamma_up[mu] @ p) / 2
    S = bar @ p
    return np.cos(z) * (kinetic - m * S - lam / 2 * S ** 2)


rng = np.random.default_rng(2104)  # fixed seed: the same numbers in every run
psi = rng.normal(size=16) + 1j * rng.normal(size=16)
dpsi = rng.normal(size=(8, 16)) + 1j * rng.normal(size=(8, 16))
dpsi = np.roll(dpsi, 1, axis=0)  # drawn with x0 last, as before; now x0 first
m_value, lam_value = 0.7, 0.3
values = {
    "L[Psi]": lagrangian(psi, dpsi, m_value, lam_value),
    "L[Psi*]": lagrangian(np.conj(psi), np.conj(dpsi), m_value, lam_value),
    "L[Gamma Psi]": lagrangian(Gamma @ psi, dpsi @ Gamma.T, m_value, lam_value),
    "L[Gamma Psi*]": lagrangian(Gamma @ np.conj(psi), np.conj(dpsi) @ Gamma.T,
                                m_value, lam_value),
    "-L_(-m,-lambda)[Psi]": -lagrangian(psi, dpsi, -m_value, -lam_value)}
for label, value in values.items():
    say(f"{label:21}: {value.real:+.10f} (imaginary part below 1e-12: "
        f"{abs(value.imag) < 1e-12})")
check(record_ok and all(abs(v.imag) < 1e-12 for v in values.values())
      and abs(values["L[Psi*]"] - values["L[Psi]"]) < 1e-12,
      "commuting field: L[Psi*] = L[Psi], the same-mass conjugation is exact")
check(abs(values["L[Gamma Psi]"] - values["-L_(-m,-lambda)[Psi]"]) < 1e-12
      and abs(values["L[Gamma Psi*]"] - values["-L_(-m,-lambda)[Psi]"]) < 1e-12
      and PAIR["T1_Lagrangian_primordial_commuting"] == "PASS",
      "L[Gamma Psi] = L[Gamma Psi*] = -L_(-m,-lambda)[Psi]: these maps reverse m",
      record=f"{PAIR_FILE}, check T1_Lagrangian_primordial_commuting")
check(LEAD["bilinears_under_charge_conjugation"] == "PASS"
      and LEAD["quantum_charge_conjugation_unitary_type"] == "PASS",
      "record: calC_+ keeps S and reverses J (commuting); the quantised field has "
      "only the mass-reversing conjugation",
      record=f"{LEAD_FILE}, check quantum_charge_conjugation_unitary_type")
```

Out [14]:

```
L[Psi]               : -11.2851368126 (imaginary part below 1e-12: True)
L[Psi*]              : -11.2851368126 (imaginary part below 1e-12: True)
L[Gamma Psi]         : +8.4767165144 (imaginary part below 1e-12: True)
L[Gamma Psi*]        : +8.4767165144 (imaginary part below 1e-12: True)
```

```
-L_(-m,-lambda)[Psi] : +8.4767165144 (imaginary part below 1e-12: True)
PASS commuting field: L[Psi*] = L[Psi], the same-mass conjugation is exact
PASS L[Gamma Psi] = L[Gamma Psi*] = -L_(-m,-lambda)[Psi]: these maps reverse m
     reproduces Revision/pairing/reports/wolfram-pairing.json, check
    T1_Lagrangian_primordial_commuting
PASS record: calC_+ keeps S and reverses J (commuting); the quantised field has only the
    mass-reversing conjugation
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, check
    quantum_charge_conjugation_unitary_type
```

Text cell:

```
The next cell draws the five numbers of the previous cell.
```

In [15]:

```
fig, ax = plt.subplots(figsize=(8.5, 4.3))
labels = ["$\\mathcal{L}_{m,\\lambda}[\\Psi]$", "$\\mathcal{L}_{m,\\lambda}[\\Psi^*]$",
          "$\\mathcal{L}_{m,\\lambda}[\\Gamma\\Psi]$",
          "$\\mathcal{L}_{m,\\lambda}[\\Gamma\\Psi^*]$",
          "$-\\mathcal{L}_{-m,-\\lambda}[\\Psi]$"]
colours = ["#2a78d6", "#2a78d6", "#e34948", "#e34948", "#999999"]
ax.bar(range(5), [v.real for v in values.values()], color=colours)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(range(5), labels, fontsize=9)
ax.set_ylabel("value of the Lagrangian density at the point")
ax.set_title("Same-mass conjugation keeps $\\mathcal{L}$; $\\Gamma$ reverses the mass")
save_figure(fig, "lagrangian_maps",
            "The Lagrangian density of the Revision record for the commuting field "
            "at one point of the author's metric ($H = 1/6$, $z = 0.7$, $a_4 = 0.4$, "
            "$a_4' = 0.25$, $m = 0.7$, $\\lambda = 0.3$, a fixed field value and "
            "fixed first derivatives): for $\\Psi$ and its same-mass conjugate "
            "$\\Psi^\\ast$ (blue, equal), for the images $\\Gamma\\Psi$ and "
            "$\\Gamma\\Psi^\\ast$ (red) and for minus the Lagrangian of the reversed "
            "mass and coupling (grey); vertical axis the value (pure numbers). The "
            "red bars equal the grey one: $\\Gamma$ maps the theory with $(m, "
            "\\lambda)$ to the theory with $(-m, -\\lambda)$.")
```

Out [15]:

```
Figure 21d.6 saved as Revision/textbook/figures/21d_6_lagrangian_maps.png
```

Text cell:

```
## 12. What a charge-violating term would look like

Condition 1 could be met only by adding to the theory a term that is not
invariant under $\Psi \to e^{i\alpha}\Psi$. The simplest candidates are
**Majorana-type mass terms** $\Psi^TM\Psi$ with a constant $16 \times 16$ matrix
```

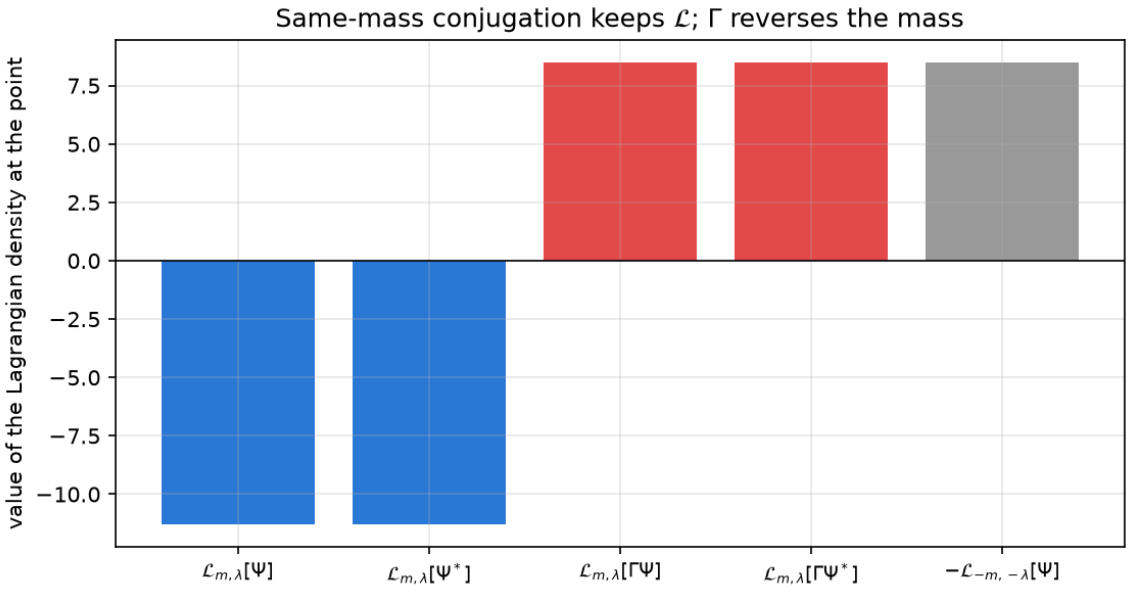

Figure 548: The Lagrangian density of the Revision record for the commuting field at one point of the author's metric ($H = 1/6$, $z = 0.7$, $a_4 = 0.4$, $a_4' = 0.25$, $m = 0.7$, $\lambda = 0.3$, a fixed field value and fixed first derivatives): for Ψ and its same-mass conjugate Ψ^* (blue, equal), for the images $\Gamma\Psi$ and $\Gamma\Psi^*$ (red) and for minus the Lagrangian of the reversed mass and coupling (grey); vertical axis the value (pure numbers). The red bars equal the grey one: Γ maps the theory with (m, λ) to the theory with $(-m, -\lambda)$. (Notebook 21d, figure 6.)

```
$M$. Such a term must still be invariant under the rotations and boosts of
Spin(4,4), $\Psi \to R\Psi$ with $R = e^{\theta S^{ab}}$, that is $R^TMR = M$; for
small $\theta$ this is
$$(S^{ab})^TM + MS^{ab} = 0\quad\text{for all 28 generators } S^{ab} .$$
The next cell solves these $28 \cdot 256 = 7168$ linear equations exactly (as in
the search for the charge-conjugation matrices) and finds a two-dimensional space
spanned by $C$ and $C\Gamma$. (This reproduces the Revision algebra check
spin_commutant_dimension_2: $M$ solves the equations exactly when $CM$ commutes
with every $S^{ab}$, because $CS^{ab}C = -(S^{ab})^T$.) It also checks a finite
transformation $R$ (a product of three rotations and boosts, built from
$e^{\theta S} = \cos(\theta/2) + 2\sin(\theta/2)S$ when $S^2 = -\tfrac14$ and
$\cosh(\theta/2) + 2\sinh(\theta/2)S$ when $S^2 = +\tfrac14$).

**For anticommuting components these terms vanish.** With $\Psi_r\Psi_c =
-\Psi_c\Psi_r$ and $\Psi_r\Psi_r = 0$, $\Psi^TM\Psi = \sum_{r<c}(M_{rc} -
M_{cr})\Psi_r\Psi_c$ keeps only the antisymmetric part of $M$; $C$ and $C\Gamma$
are symmetric, so the field dirac16complex has **no** Majorana-type mass term. For
the commuting field dirac16complex00 they survive and carry charge 2.
```

In [16]:

```
from sympy.polys.matrices import DomainMatrix  # exact matrices over the rationals

I16 = np.eye(16, dtype=np.int64)
pairs = [(a, b) for a in range(8) for b in range(a + 1, 8)]  # the 28 generators
```

```
S2 = {(a, b): gamma[a] @ gamma[b] for a, b in pairs}  # 2 S^ab = gamma^a gamma^b
# unknown M read row by row: S^T M -> kron(S^T, 1), M S -> kron(1, S^T)
system = np.vstack([np.kron(S.T, I16) + np.kron(I16, S.T) for S in S2.values()])
null = DomainMatrix.from_list(system.tolist(), sp.QQ).nullspace().to_Matrix()
basis = np.array([np.array(null.row(k).tolist()[0], dtype=float)
                  for k in range(null.rows)])  # each row: one solution, 256 numbers
CG = C @ Gamma
span_ok = all(np.linalg.matrix_rank(np.vstack([basis, M.reshape(1, -1)])) == 2
              for M in (C, CG))
say(f"{system.shape[0]} equations; dimension of the solution space: {null.rows}")
check(null.rows == 2 and span_ok
      and all(np.array_equal(C @ S @ C, -S.T) for S in S2.values())
      and ALGEBRA["spin_commutant_dimension_2"] == "PASS",
      "invariant Majorana-type matrices: a 2-dimensional space spanned by C, C Gamma",
      record=f"{ALGEBRA_FILE}, check spin_commutant_dimension_2")

ETA = [1, 1, 1, 1, -1, -1, -1, -1]


def exp_generator(a, b, theta):
    """exp(theta S^ab) for S^ab = gamma^a gamma^b / 2 (a rotation or a boost)."""
    S = S2[(a, b)] / 2
    if ETA[a] * ETA[b] == 1:  # S^2 = -1/4: a rotation
        return np.cos(theta / 2) * np.eye(16) + 2 * np.sin(theta / 2) * S
    return np.cosh(theta / 2) * np.eye(16) + 2 * np.sinh(theta / 2) * S  # a boost


R = exp_generator(1, 2, 0.7) @ exp_generator(1, 4, 0.4) @ exp_generator(0, 5, 1.1)
check(np.allclose(R.T @ C @ R, C) and np.allclose(R.T @ CG @ R, CG)
      and not np.allclose(R.T @ R, np.eye(16)),
      "a finite rotation-boost R keeps C and C Gamma (R^T M R = M), not the identity")
symmetric = np.array_equal(C, C.T) and np.array_equal(CG, CG.T)
psi_c = rng.normal(size=16) + 1j * rng.normal(size=16)  # a commuting field value
alpha = 0.9
phase_ok = all(np.isclose((np.exp(1j * alpha) * psi_c) @ M @ (np.exp(1j * alpha)
                                                              * psi_c),
                          np.exp(2j * alpha) * (psi_c @ M @ psi_c)) for M in (C, CG))
check(symmetric and phase_ok and abs(psi_c @ C @ psi_c) > 0.1,
      "C, C Gamma symmetric: zero for anticommuting components; for commuting "
      "ones nonzero, charge 2")
```

Out [16]:

```
7168 equations; dimension of the solution space: 2
PASS invariant Majorana-type matrices: a 2-dimensional space spanned by C, C Gamma
     reproduces Revision/algebra/reports/python-algebra.json, check
    spin_commutant_dimension_2
PASS a finite rotation-boost R keeps C and C Gamma (R^T M R = M), not the identity
PASS C, C Gamma symmetric: zero for anticommuting components; for commuting ones nonzero,
    charge 2
```

Text cell:

```
The next cell draws the two invariant matrices and the phase that the
Majorana-type term $\Psi^TC\Psi$ and the U(1)-invariant scalar $S = \Psi^\dagger
C\Psi$ acquire under $\Psi \to e^{i\alpha}\Psi$.
```

In [17]:

```
from matplotlib.colors import LinearSegmentedColormap

SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.2), width_ratios=[1, 1, 1.3])
for ax, M, title in ((axes[0], C, "$C$ (symmetric)"),
                     (axes[1], CG, "$C\\Gamma$ (symmetric)")):
    image = ax.imshow(M, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("column")
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.grid(False)
axes[0].set_ylabel("row")
alphas = np.linspace(0.0, np.pi, 91)
majorana = np.array([(np.exp(1j * x) * psi_c) @ C @ (np.exp(1j * x) * psi_c)
                     for x in alphas])
scalar = np.array([np.conj(np.exp(1j * x) * psi_c) @ C @ (np.exp(1j * x) * psi_c)
                   for x in alphas])
base = np.angle(majorana[0])
axes[2].plot(alphas, np.unwrap(np.angle(majorana)) - base, linewidth=2,
             label="$\\Psi^T C\\Psi$: phase $2\\alpha$ (charge 2)")
axes[2].plot(alphas, np.angle(scalar) - np.angle(scalar[0]), "--", linewidth=2,
             label="$\\Psi^\\dagger C\\Psi$: unchanged (charge 0)")
axes[2].set_xlabel("phase $\\alpha$ of $\\Psi \\to e^{i\\alpha}\\Psi$")
axes[2].set_ylabel("change of the phase of the term")
axes[2].legend(fontsize=8)
axes[2].set_title("A Majorana-type term carries charge 2")
save_figure(fig, "majorana_terms",
            "Left and middle: heat maps of the only two matrices $M$ for which a "
            "Majorana-type term $\\Psi^T M\\Psi$ is invariant under the rotations and "
            "boosts of Spin(4,4), $C$ and $C\\Gamma$ (horizontal axis the column, "
            "vertical axis the row, red $+1$, blue $-1$); both are symmetric, so "
            "the term vanishes for anticommuting components. Right: the change of "
            "the phase of $\\Psi^T C\\Psi$ (solid) and of $\\Psi^\\dagger C\\Psi$ "
            "(dashed) for a commuting field under $\\Psi \\to e^{i\\alpha}\\Psi$, "
            "versus $\\alpha$ (radians): the Majorana-type term turns twice as "
            "fast, it carries U(1) charge 2 and would break the U(1) symmetry "
            "and with it the local conservation law of the charge.")
```

Out [17]:

```
Figure 21d.7 saved as Revision/textbook/figures/21d_7_majorana_terms.png
```

Text cell:

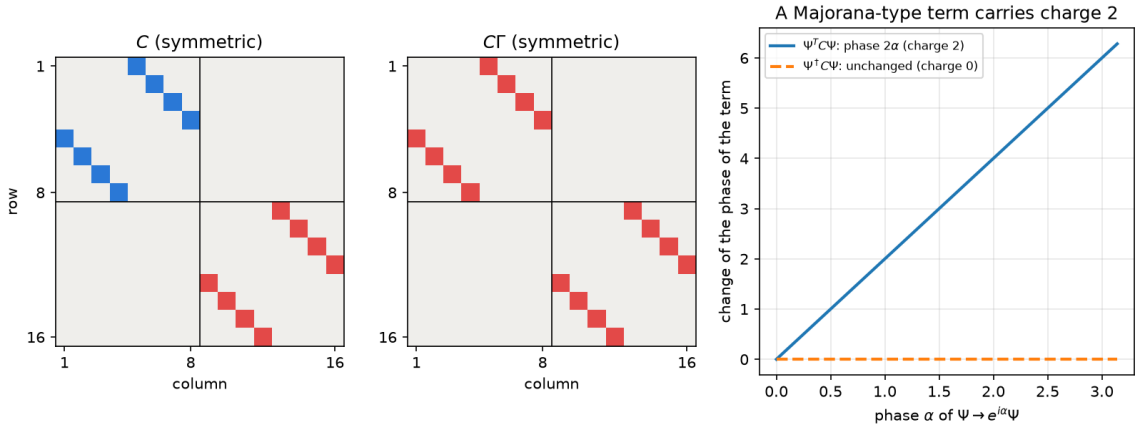

Figure 549: Left and middle: heat maps of the only two matrices M for which a Majorana-type term $\Psi^T M \Psi$ is invariant under the rotations and boosts of Spin(4,4), C and $C\Gamma$ (horizontal axis the column, vertical axis the row, red $+1$, blue -1); both are symmetric, so the term vanishes for anticommuting components. Right: the change of the phase of $\Psi^T C \Psi$ (solid) and of $\Psi^\dagger C \Psi$ (dashed) for a commuting field under $\Psi \to e^{i\alpha}\Psi$, versus α (radians): the Majorana-type term turns twice as fast, it carries U(1) charge 2 and would break the U(1) symmetry and with it the local conservation law of the charge. (Notebook 21d, figure 7.)

```
## 13. The scorecard

The next cell builds the scorecard of this theory from the verdicts of the
Revision record (each status is set only if the record holds the named check
with the verdict PASS) and draws it as a table. Condition 1 has two parts: it
fails for every process inside the patch (the local law, PROVED), and for the
total charge of a universe only under the no-flux condition at the brane
(ASSUMED). Condition 2 has two statuses: it
fails for the commuting field (its same-mass conjugation is exact, section 11;
the record checks that the gammas, $C$ and $\Omega_\mu$ are real and that the
conjugation reverses the current), and it is not computed for the quantised field
(no rates; its only conjugation reverses the mass). Condition 3: the Revision
record contains no computation of reaction rates or of a departure from
equilibrium; its Kohn-Sham history is a prescribed background (record check
ks_history_is_a_prescribed_background), not a dynamical history.
```

In [18]:

```
def status(passed, text):
    """The status text when the record check passed, otherwise UNSUPPORTED."""
    return text if passed else "UNSUPPORTED"


ROWS = [
    ("1. a process changes the number",
     "local U(1) law d_mu(cos z J^mu) = 0 for every history a4: no process inside "
     "the patch changes Q; the total Q is constant only if no charge flows "
     "through the brane z = pi/2 (ASSUMED, not derived)",
     status(LEAD["u1_noether_matrix_identity"] == "PASS",
```

```
           "inside the patch: FAILS (PROVED); total: no flux ASSUMED"),
     "u1_noether_matrix_identity"),
    ("2. C and CP violated",
     "commuting field: the same-mass conjugation is an exact symmetry that "
     "reverses the charge, so a C-symmetric start keeps zero charge; quantised "
     "field: the only conjugation, Gamma, reverses the mass; no rates computed",
     status(all(LEAD[name] == "PASS" for name in (
         "representation_real", "spinor_connection_real",
         "bilinears_under_charge_conjugation",
         "quantum_charge_conjugation_unitary_type")),
         "commuting: FAILS (PROVED); quantised: NOT COMPUTED"),
     "representation_real, spinor_connection_real, "
     "bilinears_under_charge_conjugation, quantum_charge_conjugation_unitary_type"),
    ("3. out of equilibrium",
     "no rate computed; the Kohn-Sham history is a prescribed background",
     status(KS["ks_history_is_a_prescribed_background"] == "PASS", "NOT COMPUTED"),
     "ks_history_is_a_prescribed_background"),
    ("pair level (theorem T1)",
     "the partner Gamma Psi of mass -m carries the opposite current: total charge 0",
     status(PAIR["T1_current_primordial_commuting"] == "PASS"
            and PAIR["T1_current_primordial_grassmann"] == "PASS",
            "PROVED (classical bilinears)"),
     "T1_current_primordial_commuting, T1_current_primordial_grassmann"),
    ("verdict",
     "no process makes a net charge at any point of the patch 0 < z < pi/2 of one "
     "universe (nor a total charge, under the ASSUMED no-flux condition at the "
     "brane); no baryons in the theory",
     status(LEAD["u1_noether_matrix_identity"] == "PASS", "PROBLEM NOT SOLVED"),
     "the checks above")]
for row in ROWS:
    say(f"{row[0]:34} | {row[2]}")
check([row[2] for row in ROWS] == [
          "inside the patch: FAILS (PROVED); total: no flux ASSUMED",
          "commuting: FAILS (PROVED); quantised: NOT COMPUTED",
          "NOT COMPUTED", "PROVED (classical bilinears)", "PROBLEM NOT SOLVED"],
      "scorecard: every status is backed by a PASS verdict of the Revision record")
```

Out [18]:

```
1. a process changes the number    | inside the patch: FAILS (PROVED); total: no flux
    ASSUMED
2. C and CP violated               | commuting: FAILS (PROVED); quantised: NOT COMPUTED
3. out of equilibrium              | NOT COMPUTED
pair level (theorem T1)            | PROVED (classical bilinears)
verdict                            | PROBLEM NOT SOLVED
PASS scorecard: every status is backed by a PASS verdict of the Revision record
```

Text cell:

```
The next cell draws the scorecard.
```

In [19]:

```
COLOURS = {"inside the patch: FAILS (PROVED); total: no flux ASSUMED": "#f4c7c3",
           "commuting: FAILS (PROVED); quantised: NOT COMPUTED": "#f2e2b8",
           "NOT COMPUTED": "#e3e3e3", "PROVED (classical bilinears)": "#cfe8c4",
           "PROBLEM NOT SOLVED": "#f4c7c3"}
fig, ax = plt.subplots(figsize=(13.0, 6.2))
ax.set_xlim(0, 13.0)
ax.set_ylim(0, len(ROWS) + 1)
ax.axis("off")
columns = [(0.1, 2.4, "Sakharov condition"), (2.6, 4.3, "this theory"),
           (7.1, 2.3, "status"), (9.5, 3.5, "Revision record check")]
for x0, width, heading in columns:
    ax.text(x0, len(ROWS) + 0.5, heading, fontsize=11, fontweight="bold",
            va="center")
for k, row in enumerate(ROWS):
    y = len(ROWS) - k - 0.5  # the first row at the top
    ax.add_patch(plt.Rectangle((0.0, y - 0.48), 13.0, 0.96, color=COLOURS[row[2]],
                               zorder=0))
    for (x0, width, _), text in zip(columns, row):
        chars = int(width * 11.5)  # about 11 characters per unit of width
        ax.text(x0, y, textwrap.fill(text, chars, break_long_words=False),
                fontsize=8.5, va="center")  # long check names stay whole
ax.set_title("Sakharov's conditions applied to the theory as built", fontsize=12)
save_figure(fig, "scorecard",
            "The scorecard of the theory as built against Sakharov's three "
            "conditions, with the status of each row and the Revision record "
            "check it rests on (each status is set by the notebook only when the "
            "record holds that check with the verdict PASS). Condition 1 fails "
            "for every process inside the patch (the local U(1) law holds for "
            "every history $a_4$), so there conditions 2 and 3 cannot help; the "
            "total charge of a universe is constant only under the no-flux "
            "condition at the brane $z = \\pi/2$, which is ASSUMED; condition 2 "
            "also fails for the "
            "commuting field (its same-mass conjugation is exact) and is not "
            "computed for the quantised field; the pair-level statement of theorem "
            "T1 is exact but creates nothing. The theory does not solve the "
            "matter-antimatter problem.")
```

Out [19]:

```
Figure 21d.8 saved as Revision/textbook/figures/21d_8_scorecard.png
```

Text cell:

```
## 14. The figure files

The last cell checks that the eight figure files exist and prints the number of
checks that passed.
```

In [20]:

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)
check(len(names) == 8 and all(
```

Sakharov's conditions applied to the theory as built

Sakharov condition	this theory	status	Revision record check
1. a process changes the number	local U(1) law d_mu(cos z J^mu) = 0 for every history a4: no process inside the patch changes Q; the total Q is constant only if no charge flows through the brane z = pi/2 (ASSUMED, not derived)	inside the patch: FAILS (PROVED); total: no flux ASSUMED	u1_noether_matrix_identity
2. C and CP violated	commuting field: the same-mass conjugation is an exact symmetry that reverses the charge, so a C-symmetric start keeps zero charge; quantised field: the only conjugation, Gamma, reverses the mass; no rates computed	commuting: FAILS (PROVED); quantised: NOT COMPUTED	representation_real, spinor_connection_real, bilinears_under_charge_conjugation, quantum_charge_conjugation_unitary_type
3. out of equilibrium	no rate computed; the Kohn-Sham history is a prescribed background	NOT COMPUTED	ks_history_is_a_prescribed_background
pair level (theorem T1)	the partner Gamma Psi of mass -m carries the opposite current: total charge 0	PROVED (classical bilinears)	T1_current_primordial_commuting, T1_current_primordial_grassmann
verdict	no process makes a net charge at any point of the patch 0 < z < pi/2 of one universe (nor a total charge, under the ASSUMED no-flux condition at the brane); no baryons in the theory	PROBLEM NOT SOLVED	the checks above

Figure 550: The scorecard of the theory as built against Sakharov's three conditions, with the status of each row and the Revision record check it rests on (each status is set by the notebook only when the record holds that check with the verdict PASS). Condition 1 fails for every process inside the patch (the local U(1) law holds for every history a_4), so there conditions 2 and 3 cannot help; the total charge of a universe is constant only under the no-flux condition at the brane $z = \pi/2$, which is ASSUMED; condition 2 also fails for the commuting field (its same-mass conjugation is exact) and is not computed for the quantised field; the pair-level statement of theorem T1 is exact but creates nothing. The theory does not solve the matter-antimatter problem. (Notebook 21d, figure 8.)

```
    output_file(f"{FIGURE_FOLDER}/21d_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the eight figure files of notebook 21d exist")
all_checks_passed()
```

Out [20]:

```
PASS the eight figure files of notebook 21d exist
ALL 19 CHECKS PASSED (notebook 21d)
```

Text cell:

```
## 15. What this notebook showed

- ASSUMED (known physics, quoted): Sakharov's three conditions; the quark content
  and the numbers $B$, $L$, $Q$ of the particles.
- COMPUTED (exact fractions): the three observed processes conserve $B$, $L$ and
  $Q$; proton decay $p \to e^+\pi^0$ would change $B$ and $L$ by $-1$ each and
  keep $B - L$.
- PROVED (sympy) for the toy decay model: the net baryon number is $N(r - \bar r)
  (B_1 - B_2)$, zero unless both condition 1 and condition 2 hold; example $1/10$
  per pair.
- PROVED: in equilibrium $f_+ - f_- = \sinh(\mu/T)/(\cosh(E/T) + \cosh(\mu/T))$,
  zero for $\mu = 0$.
- PROVED (closed form, checked by RK4 to $10^{-7}$) for the ASSUMED rate model:
  the surviving fraction of the asymmetry is $\eta(K)$, near 1 for slow decays and
  $1/(K - 1)$ for fast ones: condition 3.
- This theory (Revision record): the U(1) charge obeys an exact local
  conservation law (u1_noether_matrix_identity), so no process of the theory can
  make a net charge at any point of the patch $0 < z < \pi/2$ (off the brane) of
  one universe, whatever the rates and however far from equilibrium: condition 1
  FAILS there (PROVED). The total charge of a universe is constant only under the
  no-flux condition at the brane $z = \pi/2$, which is ASSUMED, not derived
  (OPEN). The same-mass conjugation is an exact symmetry of the commuting field
  (COMPUTED here at a point of the author's metric) that reverses the charge, so
  condition 2 FAILS for the commuting field (PROVED); the quantised field has
  only the mass-reversing conjugation (quantum_charge_conjugation_unitary_type),
  and C and CP violation in its rates is NOT COMPUTED; no departure from
  equilibrium is computed (the Kohn-Sham history is a prescribed background).
- PROVED (exact linear algebra): the only Spin(4,4)-invariant Majorana-type mass
  matrices are $C$ and $C\Gamma$; both are symmetric, so the anticommuting field
  has no such term; for the commuting field they carry U(1) charge 2.
- The pair-level statement (theorem T1, PROVED for classical bilinears): the
  partner of mass $-m$ carries the opposite current, so a pair has total charge
  zero. That pairs of universes exist, or were created, is a HYPOTHESIS (the
  universe/anti-universe class of ideas; a published example is L. Boyle, K. Finn
  and N. Turok, Phys. Rev. Lett. 121, 251301 (2018)); this theory does not derive
  it.
- **The theory as built does not solve the matter-antimatter problem.** What would
  be needed (OPEN): baryons in the theory; a charge-violating interaction (for the
  commuting field a Majorana-type term like the ones found here; the
  anticommuting field has no invariant mass-type term, so it would need terms
  with derivatives, more fields or new background structures, which are not
```

```
  classified here); CP violation in reaction rates; and a computed departure from
  equilibrium along a dynamical history of the deflating extra times.
```

21.36 Line-by-line walk-through of Notebook 21d

The notebook has 20 code cells, In [1] to In [20].

In [1], the set-up cell. It is the set-up cell of Notebook 21a, explained line by line in Section 21.15 under In [1], except that the comment lines at the top hold the run instructions of Notebook 21d (Section 21.34) and that the line

```
NOTEBOOK_ID = "21d"  # this notebook: chapter 21, example d
```

names this notebook. The cell prints Set-up of notebook 21d complete.

In [2]: the verdicts of the Revision record.

```
from fractions import Fraction  # exact fractions such as 1/3

import mpmath  # numbers with many digits and special functions
import numpy as np  # numbers, arrays, matrices
import sympy as sp  # exact algebra
```

`Fraction` computes with exact fractions such as 1/3; `mpmath` computes with as many digits as asked and knows special functions such as the incomplete gamma function of Section 21.6.

```
def load_checks(path):
    """name -> verdict (upper case) of every check of a Revision report."""
    entries = json.loads(repository_file(path).read_text(encoding="utf-8"))["checks"]
    return {c["name"]: c["verdict"].upper() for c in entries}
```

The helper of Notebook 21c: a Revision report becomes a dictionary from check name to verdict in capitals.

```
LEAD_FILE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
PAIR_FILE = "Revision/pairing/reports/wolfram-pairing.json"
ALGEBRA_FILE = "Revision/algebra/reports/python-algebra.json"
KS_FILE = "Revision/field_equations_a4/reports/ks-source-conditions.json"
LEAD, PAIR = load_checks(LEAD_FILE), load_checks(PAIR_FILE)
ALGEBRA, KS = load_checks(ALGEBRA_FILE), load_checks(KS_FILE)
```

The four reports on which the scorecard rests: the lead checks of charge conjugation and U(1) conservation, the Wolfram pairing report, the sympy algebra report, and the record of the Kohn-Sham source conditions of the a_4 equations.

```
USED = [(LEAD_FILE, LEAD, "u1_noether_matrix_identity"),
        (LEAD_FILE, LEAD, "representation_real"),
```

```
        (LEAD_FILE, LEAD, "spinor_connection_real"),
        (LEAD_FILE, LEAD, "bilinears_under_charge_conjugation"),
        (LEAD_FILE, LEAD, "quantum_charge_conjugation_unitary_type"),
        (PAIR_FILE, PAIR, "T1_current_primordial_commuting"),
        (PAIR_FILE, PAIR, "T1_current_primordial_grassmann"),
        (PAIR_FILE, PAIR, "T1_Lagrangian_primordial_commuting"),
        (ALGEBRA_FILE, ALGEBRA, "spin_commutant_dimension_2"),
        (KS_FILE, KS, "ks_history_is_a_prescribed_background")]
for path, verdicts, name in USED:
    say(f"record {path.split('/')[-1]}: {name} = {verdicts[name]}")
```

The ten record checks the notebook will use, each as a triple (file, its verdicts, check name). Two of them, `representation_real` (the gammas, C and the S^{ab} are real) and `spinor_connection_real` (every entry of every Ω_μ is real), are the facts on which the exact same-mass conjugation of the commuting field rests (Section 21.31); the scorecard of In [18] uses them. The loop prints the file name (`split` cuts the path at every slash, and the index `[-1]` takes the last part), the check name and its verdict: Out [2] shows ten lines, all PASS.

In [3]: counting conserved numbers.

```
QUARK = {"u": (Fraction(1, 3), Fraction(2, 3)),  # (B, Q) of the up quark
         "d": (Fraction(1, 3), Fraction(-1, 3))}  # (B, Q) of the down quark
```

The baryon number and electric charge of the up and down quarks, as exact fractions.

```
def hadron(quarks="", antiquarks=""):
    """(B, L, Q) of a particle made of the given quarks and antiquarks."""
    B = sum(QUARK[x][0] for x in quarks) - sum(QUARK[x][0] for x in antiquarks)
    Q = sum(QUARK[x][1] for x in quarks) - sum(QUARK[x][1] for x in antiquarks)
    return (B, Fraction(0), Q)
```

A particle made of quarks: a string such as `"uud"` is read letter by letter, the numbers of its quarks are added and those of its antiquarks subtracted (the addition rule of Section 21.3); the lepton number is 0. The function returns the triple $(\mathcal{B}, L, Q_{\mathrm{el}})$.

```
PARTICLE = {"p": hadron("uud"), "n": hadron("udd"), "pbar": hadron("", "uud"),
            "pi+": hadron("u", "d"), "pi-": hadron("d", "u"), "pi0": hadron("u", "u"),
            "e-": (0, 1, -1), "e+": (0, -1, 1), "nubar": (0, -1, 0), "gamma": (0, 0, 0)}
```

The particles of the four processes with their triples: the proton, neutron, antiproton and pions from their quark content (the neutral pion as $u\bar{u}$), the leptons and the photon directly.

```
PROCESSES = [("neutron decay", ["n"], ["p", "e-", "nubar"]),
             ("annihilation", ["p", "pbar"], ["pi+", "pi-", "pi0"]),
             ("pair creation", ["gamma", "p"], ["p", "e-", "e+"]),
             ("proton decay", ["p"], ["e+", "pi0"])]
changes, totals = {}, {}  # process -> changes of (B, L, Q); totals before, after
```

The four processes, each with its name, the particles before and the particles after; two empty dictionaries for the results.

```
for label, before, after in PROCESSES:
    total_before = [sum(PARTICLE[x][k] for x in before) for k in range(3)]
    total_after = [sum(PARTICLE[x][k] for x in after) for k in range(3)]
    totals[label] = (total_before, total_after)
    changes[label] = [a - b for a, b in zip(total_after, total_before)]
    say(f"{label:14}: {' + ' '.join(before):10} -> {' + ' '.join(after):15} "
        f"change of (B, L, Q) = ({', '.join(str(c) for c in changes[label])})")
```

For each process the three totals before and after (for each $k = 0, 1, 2$ the sum of the k-th number over the particles), and the changes, after minus before. The printed line pads the name to 14 and the two lists to 10 and 15 characters. Out [3] shows the changes $(0, 0, 0)$ for the three observed processes and $(-1, -1, 0)$ for proton decay.

```
check(PARTICLE["p"] == (1, 0, 1) and PARTICLE["n"] == (1, 0, 0)
      and PARTICLE["pi+"][2] == 1 and PARTICLE["pi0"] == (0, 0, 0),
      "quark content: p has (B, L, Q) = (1, 0, 1), n (1, 0, 0), pi+ charge 1")
check(all(changes[p[0]] == [0, 0, 0] for p in PROCESSES[:3])
      and changes["proton decay"] == [-1, -1, 0],
      "three observed processes conserve B, L, Q; proton decay changes B and L by -1")
```

The quark-content values of Section 21.3, and the bookkeeping: the first three processes (`PROCESSES[:3]`) conserve all three numbers, proton decay would change $\mathcal{B}$ and L by -1.

In [4]: the bookkeeping as a picture.

```
REACTIONS = ["$n \\to p\\,e^-\\bar\\nu_e$", "$p\\,\\bar p \\to \\pi^+\\pi^-\\pi^0$",
             "$\\gamma\\,p \\to p\\,e^-e^+$", "$p \\to e^+\\pi^0$ (never seen)"]
table = np.array([[float(totals[p[0]][side][k]) for k in range(3)
                   for side in (0, 1)] for p in PROCESSES])  # rows: processes
```

The reactions written in LaTeX for the row labels. `table` has one row per process and six columns: for each of the three numbers its total before (`side` 0) and after (`side` 1), as decimal numbers.

```
fig, ax = plt.subplots(figsize=(10.0, 4.2))
ax.imshow(table, cmap="RdBu_r", vmin=-2.5, vmax=2.5, aspect="auto")
for row in range(len(PROCESSES)):
    for col in range(6):
        value = table[row, col]
        ax.text(col, row, "0" if value == 0 else f"{value:+.0f}", ha="center",
                va="center", fontsize=11)
```

The table as coloured squares (red positive, blue negative), with each total written in its square with its sign (`+.0f`: a sign and no decimals).

```
        if col % 2 == 1 and table[row, col] != table[row, col - 1]:  # a change
            for c in (col - 1, col):
                ax.add_patch(plt.Rectangle((c - 0.46, row - 0.44), 0.92, 0.88,
                                           fill=False, linewidth=2.5))
```

In every "after" column (the odd column numbers) the total is compared with the "before" column to its left; if they differ, a black frame (`plt.Rectangle` with lower-left corner, width and height, not filled) is drawn around both squares.

```
ax.set_xticks(range(6), ["$B$ before", "$B$ after", "$L$ before", "$L$ after",
                         "$Q$ before", "$Q$ after"])
ax.set_yticks(range(len(PROCESSES)), [f"{p[0]}\n{r}" for p, r in
                                      zip(PROCESSES, REACTIONS)], fontsize=9)
for x in (1.5, 3.5):
    ax.axvline(x, color="black", linewidth=1)
ax.grid(False)
ax.set_title("Bookkeeping: only proton decay would change $B$ (and $L$)")
```

Column labels, row labels (the process name and, below it, the reaction), two vertical lines that separate the three numbers, and a title.

```
save_figure(fig, "bookkeeping",
            "The totals of the baryon number $B$, the lepton number $L$ and the "
            "electric charge $Q$ before and after four processes (rows): neutron "
            "decay, proton-antiproton annihilation into three pions and pair "
            "creation near a proton (all observed), and proton decay into a "
            "positron and a pion (never observed); columns the number and the side "
            "of the process, colour and figure the total (red positive, blue "
            "negative, pure numbers). Black frames mark the numbers that change: "
            "only proton decay would change $B$ and $L$, the kind of process that "
            "Sakharov's first condition requires.")
```

Figure 21d.1. **What the figure shows.** Three rows in which each "after" square has the colour of its "before" square, and one row, proton decay, with two framed pairs: $\mathcal{B}$ from $+1$ to 0 and L from 0 to -1.

In [5]: the decay model.

```
r, rbar, B1, B2, N = sp.symbols("r rbar B1 B2 N", real=True)
from_X = r * B1 + (1 - r) * B2  # average baryon number made by one X
from_Xbar = -rbar * B1 - (1 - rbar) * B2  # by one antiparticle
net = sp.expand(N * (from_X + from_Xbar))  # N particles and N antiparticles
```

Five real symbols (r, $\bar{r}$, $\mathcal{B}_1$, $\mathcal{B}_2$, N), the two averages of Section 21.5, and the multiplied-out net baryon number.

```
check(sp.expand(net - N * (r - rbar) * (B1 - B2)) == 0,
      "net baryon number = N (r - rbar) (B1 - B2) exactly")
```

The factorised form of Section 21.5 holds exactly.

```
example = net.subs({N: 1, B1: sp.Rational(2, 3), B2: sp.Rational(-1, 3),
                    r: sp.Rational(3, 5), rbar: sp.Rational(1, 2)})
```

```
report("net baryon number per pair X, Xbar for r = 0.6, rbar = 0.5", example)
check(example == sp.Rational(1, 10)
      and net.subs(B2, B1) == 0 and net.subs(rbar, r) == 0,
      "example 1/10 per pair; zero when B1 = B2 or when r = rbar")
```

The worked example with exact fractions: $1/10$ per pair (printed as a RESULT line). Replacing $\mathcal{B}_2$ by $\mathcal{B}_1$, or $\bar{r}$ by r, gives exactly zero: the two conditions.

In [6]: the decay model as a map.

```
grid = np.linspace(0.0, 1.0, 201)  # values of r and rbar from 0 to 1
R, RBAR = np.meshgrid(grid, grid)
net_f = sp.lambdify((r, rbar, B1, B2), net.subs(N, 1), "numpy")
fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.4), sharey=True)
```

201 values from 0 to 1 for each probability, all their pairs, and the net number per pair ($N = 1$) as a fast function.

```
for ax, (b1, b2), title in ((axes[0], (2 / 3, -1 / 3), "$B$ violated: $B_1 = 2/3$, "
                               "$B_2 = -1/3$"),
                              (axes[1], (1 / 3, 1 / 3), "$B$ conserved: "
                               "$B_1 = B_2 = 1/3$")):
    values = net_f(R, RBAR, b1, b2) + 0 * R  # + 0 * R: an array even when constant
    image = ax.pcolormesh(R, RBAR, values, cmap="RdBu_r", vmin=-1, vmax=1,
                          shading="auto")
```

Two cases: channels of baryon number $2/3$ and $-1/3$, and two channels of the same baryon number $1/3$. In the second case the expression is the constant zero, and a function that returns a constant returns a single number; adding `0 * R` turns it into an array of the right shape. `pcolormesh` colours the plane of $(r, \bar{r})$.

```
    ax.plot([0, 1], [0, 1], "--", color="black", linewidth=1,
            label="$r = \\bar r$ (no C, CP violation)")
    ax.set_xlabel("branching probability $r$ of $X$")
    ax.set_title(title)
    ax.legend(fontsize=8, loc="upper left")
axes[0].set_ylabel("branching probability $\\bar r$ of $\\bar X$")
fig.colorbar(image, ax=axes, shrink=0.85, label="net baryon number per pair")
```

A dashed diagonal marks $r = \bar{r}$; labels, legend and colour bar follow.

```
save_figure(fig, "decay_asymmetry",
            "The net baryon number $(r - \\bar r)(B_1 - B_2)$ made by the decays of "
            "one toy particle $X$ and its antiparticle, versus the branching "
            "probability $r$ of $X$ (horizontal) and $\\bar r$ of $\\bar X$ "
            "(vertical), colour the net number (red positive, blue negative). Left: "
            "channels with $B_1 = 2/3$ and $B_2 = -1/3$ (baryon number violated); "
            "the net number vanishes only on the diagonal $r = \\bar r$. Right: "
            "channels with equal baryon number; the net number is zero everywhere. "
            "An asymmetry needs both conditions 1 and 2.")
```

Figure 21d.2. **What the figure shows.** Left: red below the diagonal ($r > \bar{r}$), blue above it, white on it; the colour is strongest in the corners, where $r - \bar{r} = \pm 1$. Right: uniformly white: without a change of baryon number no asymmetry, whatever the probabilities.

In [7]: the equilibrium occupations.

```
E_T, mu_T = sp.symbols("E_T mu_T", real=True)  # E / T and mu / T
f_plus = 1 / (sp.exp(E_T - mu_T) + 1)
f_minus = 1 / (sp.exp(E_T + mu_T) + 1)
difference = sp.sinh(mu_T) / (sp.cosh(E_T) + sp.cosh(mu_T))
```

Symbols for E/T and μ/T, the two Fermi-Dirac occupations and the formula of Section 21.5 for their difference.

```
check(sp.simplify((f_plus - f_minus - difference).rewrite(sp.exp)) == 0,
      "f+ - f- = sinh(mu/T) / (cosh(E/T) + cosh(mu/T)) exactly")
```

`.rewrite(sp.exp)` writes sinh and cosh with exponentials; the difference then simplifies to zero: the line-by-line derivation of Section 21.5 is confirmed.

```
at_zero = [float(f.subs({E_T: 2, mu_T: 0})) for f in (f_plus, f_minus)]
at_tenth = [float(f.subs({E_T: 2, mu_T: sp.Rational(1, 10)}))
            for f in (f_plus, f_minus)]
report("f+ = f- at E = 2T, mu = 0", f"{at_zero[0]:.6f}")
report("f+, f- at E = 2T, mu = 0.1 T", f"{at_tenth[0]:.6f}, {at_tenth[1]:.6f}")
report("surplus of particles per state", f"{at_tenth[0] - at_tenth[1]:.6f}")
```

The worked example: both occupations at $E = 2T$, $\mu = 0$, and at $\mu = 0.1T$. Out [7] prints 0.119203; 0.130108 and 0.109097; and the surplus 0.021012.

```
check(at_zero[0] == at_zero[1] and abs(at_zero[0] - 1 / (np.exp(2) + 1)) < 1e-15
      and at_tenth[0] > at_tenth[1],
      "mu = 0: equal occupations 1/(e^2 + 1); mu > 0: more particles")
```

At $\mu = 0$ the two occupations are equal to $1/(e^2 + 1)$; at $\mu > 0$ particles are more common.

In [8]: the surplus against the energy.

```
energies = np.linspace(0.0, 8.0, 321)  # E / T from 0 to 8
diff_f = sp.lambdify((E_T, mu_T), difference, "numpy")
fig, ax = plt.subplots(figsize=(7.5, 4.3))
for mu_value, style in ((0.2, "-"), (0.1, "--"), (0.05, "-."), (0.0, ":")):
    ax.plot(energies, diff_f(energies, mu_value) + 0 * energies, style, linewidth=2,
            label=f"$\\mu/T = {mu_value}$")
```

321 energies from 0 to $8T$, the surplus as a fast function, and one curve for each of four chemical potentials (with `+ 0 * energies` for the same reason as in In [6]).

```
ax.plot([2.0], [at_tenth[0] - at_tenth[1]], "o", color="black",
        label="worked example $E = 2T$, $\\mu = 0.1T$")
ax.set_xlabel("energy $E/T$")
ax.set_ylabel("$f_+ - f_-$ (particles minus antiparticles per state)")
ax.set_title("In equilibrium with $\\mu = 0$ there is no surplus")
ax.legend(fontsize=8)
```

A black dot at the worked example, axis labels, title and legend.

```
save_figure(fig, "equilibrium_occupations",
            "The surplus $f_+ - f_- = \\sinh(\\mu/T)/(\\cosh(E/T) + \\cosh(\\mu/T))$ "
            "of particles over antiparticles per quantum state in thermal "
            "equilibrium, versus the energy $E/T$ of the state (horizontal) for the "
            "chemical potentials $\\mu/T = 0.2, 0.1, 0.05, 0$ (pure numbers). The "
            "dot is the worked example $E = 2T$, $\\mu = 0.1T$. When the reactions "
            "that change the baryon number are in equilibrium, $\\mu = 0$ and the "
            "surplus vanishes at every energy (dotted line).")
```

Figure 21d.3. **What the figure shows.** Three curves that start near $\tanh(\mu/2T)$ at $E = 0$ and fall towards zero for large energies, the higher the larger μ; the curve for $\mu = 0$ is the horizontal axis itself.

In [9]: the rate model in closed form.

```
t, s_, K_s, eps_s = sp.symbols("t s K epsilon", positive=True)
Delta = (sp.exp(-t) - sp.exp(-K_s * t)) / (K_s - 1)  # line 2
W = K_s * (1 - sp.exp(-t))  # line 3
```

Positive symbols for the time and the rate (two further symbols, for s and ϵ, are created but not used below), and the two functions $\Delta(t)$ and $W(t)$ of Section 21.6.

```
line2_ok = (sp.simplify(sp.diff(Delta, t) - (-K_s * Delta + sp.exp(-t))) == 0
            and Delta.subs(t, 0) == 0)
line3_ok = sp.simplify(sp.diff(W, t) - K_s * sp.exp(-t)) == 0
check(line2_ok and line3_ok, "rate model: Delta(t) and W(t) solve lines 1 to 3")
mpmath.mp.dps = 30  # work with 30 significant digits
```

Δ solves $d\Delta/dt = -K\Delta + e^{-t}$ with $\Delta(0) = 0$, and $dW/dt = Ke^{-t}$: lines 1 to 3. mpmath is told to work with 30 significant digits.

```
def efficiency(K):
    """eta(K) = K/(K - 1) ((1 - e^-K)/K - K^-K gamma(K, K)), K != 1."""
    K = mpmath.mpf(K)
    incomplete = mpmath.gammainc(K, 0, K)  # the lower incomplete gamma(K, K)
    return float(K / (K - 1) * ((1 - mpmath.exp(-K)) / K - K ** (-K) * incomplete))


for K in (0.1, 3.0, 30.0):
    say(f"K = {K:5}: eta = {efficiency(K):.10f}")
```

The closed form of line 6. `mpmath.mpf` makes a 30-digit number; `mpmath.gammainc(K, 0, K)` is $\int_0^K u^{K-1}e^{-u}du = \gamma(K, K)$. The result is returned as an ordinary decimal number. Out [9] prints $\eta = 0.9955325865$, 0.4110164748 and 0.0344827586 for $K = 0.1$, 3 and 30.

In [10]: the rate model integrated with RK4.

```
def integrate(K, eps=1.0, erase=True, keep_every=50):
    """RK4 for (n, a) from t = 0 to 60 + 40/K; returns the final state and the
    history (every keep_every-th step)."""
    t_end = 60.0 + 40.0 / K  # long enough for n and n_eq to be negligible
    steps = int(np.ceil(t_end / min(0.1 / K, 0.02)))  # step below 0.1/K
    dt = t_end / steps
```

The integration runs to $t = 60 + 40/K$, by which time n and n_{eq} are negligible (for slow decays n approaches n_{eq} only on the time scale $1/K$). The step is at most $0.1/K$ and at most 0.02; `np.ceil` rounds the number of steps up to a whole number, and `dt` is the exact step.

```
    def rates(time, y):
        n_eq = np.exp(-time)
        return np.array([-K * (y[0] - n_eq),
                         eps * K * (y[0] - n_eq) - (K * n_eq * y[1] if erase else 0)])
```

The right sides of the two equations of Section 21.6 for the state $y = (n, a)$; with `erase=False` the erasing term is left out.

```
    y, time, history = np.array([1.0, 0.0]), 0.0, [(0.0, 1.0, 0.0)]
    for step in range(steps):
        k1 = rates(time, y)
        k2 = rates(time + dt / 2, y + dt / 2 * k1)
        k3 = rates(time + dt / 2, y + dt / 2 * k2)
        k4 = rates(time + dt, y + dt * k3)
        y = y + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
        time += dt
        if (step + 1) % keep_every == 0:
            history.append((time, y[0], y[1]))
    return y, np.array(history)
```

The classical fourth-order **Runge-Kutta method** (Chapter 2): from the start $n = 1$, $a = 0$, each step evaluates the slopes at the beginning, twice in the middle and at the end of the step, and advances with their weighted average; the error over a fixed time falls like dt^4. Every 50th state is kept for the figure.

```
RATES = (0.1, 3.0, 30.0)
runs = {K: integrate(K) for K in RATES}
for K in RATES:
    say(f"K = {K:5}: RK4 a(infinity) = {runs[K][0][1]:.10f}, "
        f"closed form {efficiency(K):.10f}")
check(all(abs(runs[K][0][1] - efficiency(K)) < 1e-7 for K in RATES),
      "RK4 agrees with the closed form eta(K) to 1e-7 for K = 0.1, 3, 30")
```

The three integrations with $\epsilon = 1$; the final asymmetry (`runs[K][0][1]`, the second entry of the final state) is printed next to the closed form. Out [10] shows agreement to 10 decimals for $K = 0.1$ and 30 and to 5×10^{-10} for $K = 3$ (0.4110164753 against 0.4110164748); the check demands 10^{-7}.

```
no_cp, _ = integrate(3.0, eps=0.0)
no_erase, _ = integrate(3.0, erase=False)
check(no_cp[1] == 0.0 and abs(no_erase[1] - 1.0) < 1e-9,
      "controls: eps = 0 gives a = 0 exactly; without erasing a(infinity) = eps")
```

The two controls of Section 21.6: with $\epsilon = 0$ the asymmetry stays exactly zero; without the erasing term the final asymmetry is $\epsilon = 1$.

In [11]: the histories.

```
fig, axes = plt.subplots(2, 1, figsize=(8.0, 6.6), sharex=True)
times = np.linspace(0.0, 60.0, 601)
axes[0].semilogy(times, np.exp(-times), color="black", linewidth=1,
                 label="equilibrium $n_{eq} = e^{-t}$")
```

Two panels stacked vertically; the top one draws $n_{eq} = e^{-t}$ on a logarithmic axis from $t = 0$ to 60.

```
for K, style in zip(RATES, ["-", "--", ":"]):
    history = runs[K][1]
    shown = history[history[:, 0] <= 60.0]
    axes[0].semilogy(shown[:, 0], shown[:, 1], style, linewidth=2,
                     label=f"$n$, $K = {K}$")
    axes[1].plot(shown[:, 0], shown[:, 2], style, linewidth=2, label=f"$K = {K}$")
```

For each rate the stored history (columns: time, n, a), restricted to $t \leq 60$ (`history[:, 0] <= 60.0` marks the rows to keep); n in the top panel, a in the bottom one.

```
axes[0].set_ylim(1e-12, 2.0)
axes[0].set_ylabel("heavy pairs (log scale)")
axes[0].legend(fontsize=8, loc="lower left")
axes[1].set_xlabel("time $t$ (units of the cooling time)")
axes[1].set_ylabel("asymmetry $a/\\epsilon$")
axes[1].set_ylim(0.0, 1.05)
axes[1].legend(fontsize=8, loc="center right", bbox_to_anchor=(1.0, 0.68))
```

Axis ranges, labels and legends; `bbox_to_anchor` places the lower legend at the right edge, 68 per cent of the way up.

```
save_figure(fig, "rate_model_histories",
            "The rate model of decays out of equilibrium for three decay rates $K$. "
            "Top: the number $n$ of heavy pairs (coloured) and the equilibrium "
            "value $n_{eq} = e^{-t}$ (thin black), logarithmic scale. Bottom: the "
            "asymmetry $a/\\epsilon$. Horizontal axis the time in units of the "
            "cooling time (pure numbers). Slow decays ($K = 0.1$) happen late, "
            "far from equilibrium, when nothing erases their product: almost the "
            "whole $\\epsilon$ survives. Fast decays ($K = 30$) keep $n$ close to "
            "$n_{eq}$, and the inverse decays erase almost everything.")
```

Figure 21d.4. **What the figure shows.** Top: for $K = 3$ and $K = 30$ the curves of n lie on or just above the black line of equilibrium; for $K = 0.1$ the curve stays far above it and falls only slowly, like e^{-Kt}. Bottom: all three asymmetries rise steadily to their final values (they never decrease in this model): nearly 1 for $K = 0.1$, 0.41 for $K = 3$ and only $1/29$ for $K = 30$, because the faster the decays, the more of their product the inverse decays erase while it is being made.

In [12]: the efficiency for all rates.

```
K_grid = np.logspace(-2.0, 3.0, 24)  # 24 rates; K = 1 is not among them
eta = np.array([efficiency(K) for K in K_grid])
check(eta[0] > 0.99 and abs(efficiency(1000.0) * 999.0 - 1.0) < 1e-12
      and np.all(np.diff(eta) < 0),
      "efficiency: near 1 for K = 0.01, exactly 1/(K - 1) at K = 1000, decreasing")
```

`np.logspace(-2, 3, 24)` gives 24 rates from 10^{-2} to 10^{3}, equally spaced on a logarithmic scale (the formula has $K - 1$ in a denominator, and $K = 1$ is not among them). The efficiency is above 0.99 for $K = 0.01$, equals $1/999$ at $K = 1000$ to 10^{-12} relative accuracy (the exponentially small terms are invisible), and decreases from each rate to the next (`np.diff` gives the differences of neighbours).

```
fig, ax = plt.subplots(figsize=(7.5, 4.4))
ax.loglog(K_grid, eta, "-", linewidth=2, label="closed form $\\eta(K)$")
ax.loglog(K_grid[K_grid > 2], 1 / (K_grid[K_grid > 2] - 1), ":", color="black",
          label="$1/(K - 1)$: near equilibrium")
ax.loglog(RATES, [runs[K][0][1] for K in RATES], "o", color="#e34948",
          label="RK4 integration")
ax.set_xlabel("decay rate $K$ (units of the cooling rate)")
ax.set_ylabel("efficiency $a(\\infty)/\\epsilon$")
ax.set_title("Equilibrium erases the asymmetry")
ax.legend(fontsize=8, loc="lower left")
```

Both axes logarithmic: the closed form, the fast-decay limit $1/(K - 1)$ for $K > 2$, and the three RK4 results as red dots.

```
save_figure(fig, "washout_efficiency",
            "The efficiency $\\eta(K) = a(\\infty)/\\epsilon$ of the rate model: the "
            "fraction of the asymmetry made by the decays that survives, versus "
            "the decay rate $K$, both on logarithmic scales (pure numbers). Solid: "
            "the closed form with the incomplete gamma function; dots: the RK4 "
            "integrations; dotted: the limit $1/(K - 1)$. Slow decays (out of "
            "equilibrium) keep almost all of it; the closer the decays are to "
            "equilibrium (large $K$), the less survives: condition 3.")
```

Figure 21d.5. **What the figure shows.** A flat plateau near 1 for small rates that bends down into a straight line of slope -1 for large rates, on which the dotted limit lies; the three red dots sit on the curve.

In [13]: this theory, condition 1.

```
Q_channel = sp.Symbol("Q_channel")  # the common charge of every channel
eps_theory = net.subs({N: 1, B1: Q_channel, B2: Q_channel})  # B1 = B2
a_theory, _ = integrate(0.1, eps=float(eps_theory))  # slow decays, far from equil.
```

In this theory every channel of every process carries the same charge (Section 21.31): the decay formula with $\mathcal{B}_1 = \mathcal{B}_2$ gives the net charge per pair, and the rate model is run with that ϵ and the slowest rate, as far from equilibrium as in In [10].

```
say(f"net charge per pair with equal channels: {eps_theory}; rate model: "
    f"a(infinity) = {a_theory[1]}")
check(LEAD["u1_noether_matrix_identity"] == "PASS" and eps_theory == 0
      and a_theory[1] == 0.0,
      "local U(1) law: no process makes a net charge at any point of the patch "
      "0 < z < pi/2, condition 1 fails there (total charge: no flux through the "
      "brane ASSUMED)",
      record=f"{LEAD_FILE}, check u1_noether_matrix_identity")
```

Out [13] prints a net charge 0 and $a(\infty) = 0.0$; the check also demands the record's verdict PASS for the local conservation law $\partial_\mu(\cos z\, J^\mu) = 0$. Its label says what that verdict gives: no process makes a net charge at any point of the patch $0 < z < \pi/2$ (the local law is proved there, off the brane $z = \pi/2$), so condition 1 fails there; for the total charge of a universe the no-flux condition at the brane is needed in addition, and it is ASSUMED (Section 21.31). The label is one text made of three string pieces that stand next to each other, and `record=` names the record file and check on the PASS line.

In [14]: this theory, condition 2: the Lagrangian at a point.

```
fixture = json.loads(repository_file("Revision/algebra/gammas.json")
                     .read_text(encoding="utf-8"))
gamma = [np.array(mat, dtype=np.int64) for mat in fixture["gamma"]]  # x0 .. x7
C = gamma[0] @ gamma[1] @ gamma[2] @ gamma[3]  # C = g(x0) g(x1) g(x2) g(x3)
Gamma = C @ gamma[4] @ gamma[5] @ gamma[6] @ gamma[7]  # the chirality
```

The Revision gammas, C and Γ, as in Notebook 21c.

```
THEORY = {f["key"]: f["wl"] for f in json.loads(repository_file(
    "Revision/theory/field-theory.json").read_text(encoding="utf-8"))["formulas"]}
lagrangian_text = ("L = Cos[z] [ (1/2) sum_mu (Psibar gammaˆmu D_mu Psi - (D_mu "
                   "Psibar) gammaˆmu Psi) - m S - U(S) ]")
omega_text = ["(1/2) Eˆa4[x4] Sin[6 H x0]ˆ(1/6) (a4",
              "g[xi].g[x4] + H g[xi].g[x0]) (i = 1, 2, 3)",
              "-(1/2) Eˆ-a4[x4] Sin[6 H x0]ˆ(1/6) (a4",
              "g[x4].g[xt] + H g[xt].g[x0]) (t = 5, 6, 7)", "Omega_x0 = Omega_x4 = 0"]
record_ok = (THEORY["Lagrangian"].startswith(lagrangian_text)
             and all(piece in THEORY["Omega_components"] for piece in omega_text))
```

The record's formulas; the beginning of its Lagrangian and pieces of its spin connection, in the record's notation, which the notebook uses below; `record_ok` is true when the record states them so.

```
H, z, a4, a4p = 1 / 6, 0.7, 0.4, 0.25  # the point of the author's metric
s16 = np.sin(z) ** (1 / 6)
f = [1 / np.tan(z)] + [np.exp(a4) * s16] * 3 + [1.0] + [np.exp(-a4) * s16] * 3
```

A point of the author's metric, now with plain numbers: $H = 1/6$, $z = 0.7$, $a_4 = 0.4$, $a_4' = 0.25$; and the eight scale factors there ($1/\tan z = \cot z$).

```
Omega = []  # the record's formula Omega_components at the point
for mu in range(8):
    if mu in (1, 2, 3):  # x1, x2, x3
        Omega.append(np.exp(a4) * s16 / 2 * (a4p * gamma[mu] @ gamma[4]
                                             + H * gamma[mu] @ gamma[0]))
    elif mu in (5, 6, 7):  # x5, x6, x7: the deflating extra times
        Omega.append(-np.exp(-a4) * s16 / 2 * (a4p * gamma[4] @ gamma[mu]
                                               + H * gamma[mu] @ gamma[0]))
    else:  # x4 and x0
        Omega.append(np.zeros((16, 16)))
gamma_up = [gamma[mu] / f[mu] for mu in range(8)]  # curved gammas at the point
```

The record's spin connection (Section 21.17) evaluated at the point, and the curved gammas.

```
def lagrangian(p, dp, m, lam):
    """The record's Lagrangian at the point for the field value p and the eight
    first derivatives dp[mu] (commuting components)."""
    bar = np.conj(p) @ C  # Psibar = Psi^dagger C
    kinetic = 0.0
    for mu in range(8):
        D = dp[mu] + Omega[mu] @ p  # D_mu Psi
        Dbar = np.conj(dp[mu]) @ C - bar @ Omega[mu]  # D_mu Psibar
        kinetic += (bar @ gamma_up[mu] @ D - Dbar @ gamma_up[mu] @ p) / 2
    S = bar @ p
    return np.cos(z) * (kinetic - m * S - lam / 2 * S ** 2)
```

The Lagrangian density of the record at the point, as in In [9] of Notebook 21b, with the mass and the coupling as arguments.

```
rng = np.random.default_rng(2104)  # fixed seed: the same numbers in every run
psi = rng.normal(size=16) + 1j * rng.normal(size=16)
dpsi = rng.normal(size=(8, 16)) + 1j * rng.normal(size=(8, 16))
dpsi = np.roll(dpsi, 1, axis=0)  # drawn with x0 last, as before; now x0 first
m_value, lam_value = 0.7, 0.3
```

A fixed random field value and eight derivatives (seed 2104), $m = 0.7$, $\lambda = 0.3$. The derivatives are drawn with the hidden direction last, as before the move to the author's coordinates, and `np.roll` moves that row to the front (row μ belongs to x_μ), so every printed number stays the same.

```
values = {
    "L[Psi]": lagrangian(psi, dpsi, m_value, lam_value),
    "L[Psi*]": lagrangian(np.conj(psi), np.conj(dpsi), m_value, lam_value),
```

```
    "L[Gamma Psi]": lagrangian(Gamma @ psi, dpsi @ Gamma.T, m_value, lam_value),
    "L[Gamma Psi*]": lagrangian(Gamma @ np.conj(psi), np.conj(dpsi) @ Gamma.T,
                                m_value, lam_value),
    "-L_(-m,-lambda)[Psi]": -lagrangian(psi, dpsi, -m_value, -lam_value)}
```

Five numbers: the Lagrangian of Ψ; of Ψ^* (value and derivatives conjugated); of $\Gamma\Psi$, whose derivatives are $\Gamma\partial_\mu\Psi$ (each row of `dpsi` times Γ^T from the right is Γ times that row); of $\Gamma\Psi^*$; and minus the Lagrangian of Ψ with the reversed parameters.

```
for label, value in values.items():
    say(f"{label:21}: {value.real:+.10f} (imaginary part below 1e-12: "
        f"{abs(value.imag) < 1e-12})")
```

Out [14] prints -11.2851368126 for $\mathcal{L}[\Psi]$ and $\mathcal{L}[\Psi^*]$ and $+8.4767165144$ for the other three, all real.

```
check(record_ok and all(abs(v.imag) < 1e-12 for v in values.values())
      and abs(values["L[Psi*]"] - values["L[Psi]"]) < 1e-12,
      "commuting field: L[Psi*] = L[Psi], the same-mass conjugation is exact")
check(abs(values["L[Gamma Psi]"] - values["-L_(-m,-lambda)[Psi]"]) < 1e-12
      and abs(values["L[Gamma Psi*]"] - values["-L_(-m,-lambda)[Psi]"]) < 1e-12
      and PAIR["T1_Lagrangian_primordial_commuting"] == "PASS",
      "L[Gamma Psi] = L[Gamma Psi*] = -L_(-m,-lambda)[Psi]: these maps reverse m",
      record=f"{PAIR_FILE}, check T1_Lagrangian_primordial_commuting")
```

The same-mass conjugation keeps the Lagrangian (Section 21.31); Γ and $\Gamma\Psi^*$ map it to minus the Lagrangian with $(-m, -\lambda)$ (theorem T1).

```
check(LEAD["bilinears_under_charge_conjugation"] == "PASS"
      and LEAD["quantum_charge_conjugation_unitary_type"] == "PASS",
      "record: calC_+ keeps S and reverses J (commuting); the quantised field has "
      "only the mass-reversing conjugation",
      record=f"{LEAD_FILE}, check quantum_charge_conjugation_unitary_type")
```

The two record verdicts on which row 2 of the scorecard rests.

In [15]: the five Lagrangian values as bars.

```
fig, ax = plt.subplots(figsize=(8.5, 4.3))
labels = ["$\\mathcal{L}_{m,\\lambda}[\\Psi]$", "$\\mathcal{L}_{m,\\lambda}[\\Psi^*]$",
          "$\\mathcal{L}_{m,\\lambda}[\\Gamma\\Psi]$",
          "$\\mathcal{L}_{m,\\lambda}[\\Gamma\\Psi^*]$",
          "$-\\mathcal{L}_{-m,-\\lambda}[\\Psi]$"]
colours = ["#2a78d6", "#2a78d6", "#e34948", "#e34948", "#999999"]
ax.bar(range(5), [v.real for v in values.values()], color=colours)
ax.axhline(0.0, color="black", linewidth=0.8)
ax.set_xticks(range(5), labels, fontsize=9)
ax.set_ylabel("value of the Lagrangian density at the point")
ax.set_title("Same-mass conjugation keeps $\\mathcal{L}$; $\\Gamma$ reverses the mass")
```

Five bars with LaTeX labels: blue for Ψ and Ψ^*, red for the two images, grey for the comparison value.

```
save_figure(fig, "lagrangian_maps",
            "The Lagrangian density of the Revision record for the commuting field "
            "at one point of the author's metric ($H = 1/6$, $z = 0.7$, $a_4 = 0.4$, "
            "$a_4' = 0.25$, $m = 0.7$, $\\lambda = 0.3$, a fixed field value and "
            "fixed first derivatives): for $\\Psi$ and its same-mass conjugate "
            "$\\Psi^\\ast$ (blue, equal), for the images $\\Gamma\\Psi$ and "
            "$\\Gamma\\Psi^\\ast$ (red) and for minus the Lagrangian of the reversed "
            "mass and coupling (grey); vertical axis the value (pure numbers). The "
            "red bars equal the grey one: $\\Gamma$ maps the theory with $(m, "
            "\\lambda)$ to the theory with $(-m, -\\lambda)$.")
```

Figure 21d.6. **What the figure shows.** Two equal blue bars below zero (-11.29) and three equal bars above zero ($+8.48$).

In [16]: the invariant Majorana-type matrices.

```
from sympy.polys.matrices import DomainMatrix  # exact matrices over the rationals

I16 = np.eye(16, dtype=np.int64)
pairs = [(a, b) for a in range(8) for b in range(a + 1, 8)]  # the 28 generators
S2 = {(a, b): gamma[a] @ gamma[b] for a, b in pairs}  # 2 S^ab = gamma^a gamma^b
```

The exact solver of Notebook 21a, the identity, the 28 pairs of directions and, for each, $2S^{ab} = \gamma^a\gamma^b$ (twice the generator: whole numbers, and the factor 2 does not change the solutions of the homogeneous equations).

```
# unknown M read row by row: S^T M -> kron(S^T, 1), M S -> kron(1, S^T)
system = np.vstack([np.kron(S.T, I16) + np.kron(I16, S.T) for S in S2.values()])
null = DomainMatrix.from_list(system.tolist(), sp.QQ).nullspace().to_Matrix()
basis = np.array([np.array(null.row(k).tolist()[0], dtype=float)
                  for k in range(null.rows)])  # each row: one solution, 256 numbers
```

With M read row by row as in Notebook 21a, S^TM is `np.kron(S.T, I16)` applied to it and MS is `np.kron(I16, S.T)` applied to it; the 28 blocks of 256 equations are stacked and solved exactly. Each row of `basis` is one solution, 256 decimal numbers.

```
CG = C @ Gamma
span_ok = all(np.linalg.matrix_rank(np.vstack([basis, M.reshape(1, -1)])) == 2
              for M in (C, CG))
say(f"{system.shape[0]} equations; dimension of the solution space: {null.rows}")
```

$C\Gamma$; and `span_ok` is true when adding C (or $C\Gamma$) as a third row to the two basis rows leaves the rank at 2, that is, when each lies in the space of solutions. Out [16] prints 7168 equations and dimension 2.

```
check(null.rows == 2 and span_ok
      and all(np.array_equal(C @ S @ C, -S.T) for S in S2.values())
      and ALGEBRA["spin_commutant_dimension_2"] == "PASS",
      "invariant Majorana-type matrices: a 2-dimensional space spanned by C, C Gamma",
      record=f"{ALGEBRA_FILE}, check spin_commutant_dimension_2")
```

The space has dimension 2 and is spanned by C and $C\Gamma$; the identity $CSC = -S^T$ of Section 21.32 holds for all 28 generators; the algebra record's commutant has dimension 2.

```
ETA = [1, 1, 1, 1, -1, -1, -1, -1]


def exp_generator(a, b, theta):
    """exp(theta S^ab) for S^ab = gamma^a gamma^b / 2 (a rotation or a boost)."""
    S = S2[(a, b)] / 2
    if ETA[a] * ETA[b] == 1:  # S^2 = -1/4: a rotation
        return np.cos(theta / 2) * np.eye(16) + 2 * np.sin(theta / 2) * S
    return np.cosh(theta / 2) * np.eye(16) + 2 * np.sinh(theta / 2) * S  # a boost
```

The signs of the frame metric, and the finite transformation $e^{\theta S^{ab}}$ by the formulas of Section 21.32: a rotation when the two directions have the same sign of η, a boost otherwise.

```
R = exp_generator(1, 2, 0.7) @ exp_generator(1, 4, 0.4) @ exp_generator(0, 5, 1.1)
check(np.allclose(R.T @ C @ R, C) and np.allclose(R.T @ CG @ R, CG)
      and not np.allclose(R.T @ R, np.eye(16)),
      "a finite rotation-boost R keeps C and C Gamma (R^T M R = M), not the identity")
```

A product of a rotation in the plane of x_1, x_2 (angle 0.7), a boost in the plane of x_1, x_4 (0.4) and a boost in the plane of x_0, x_5 (1.1; the same boost as -1.1 in the plane of x_5, x_0, because $S^{x_0 x_5} = -S^{x_5 x_0}$). It keeps C and $C\Gamma$ ($R^T M R = M$), while $R^T R$ is not the identity: the invariance is not an accident of an orthogonal R.

```
symmetric = np.array_equal(C, C.T) and np.array_equal(CG, CG.T)
psi_c = rng.normal(size=16) + 1j * rng.normal(size=16)  # a commuting field value
alpha = 0.9
phase_ok = all(np.isclose((np.exp(1j * alpha) * psi_c) @ M @ (np.exp(1j * alpha)
                                                              * psi_c),
                          np.exp(2j * alpha) * (psi_c @ M @ psi_c)) for M in (C, CG))
check(symmetric and phase_ok and abs(psi_c @ C @ psi_c) > 0.1,
      "C, C Gamma symmetric: zero for anticommuting components; for commuting "
      "ones nonzero, charge 2")
```

Both matrices are symmetric, so the terms vanish for anticommuting components (Section 21.32). For a commuting field value (the generator of In [14] continues), the term $\Psi^T M \Psi$ is multiplied by $e^{2i\alpha}$ when Ψ is multiplied by $e^{i\alpha}$ ($\alpha = 0.9$): charge 2; and $\Psi^T C \Psi$ is not zero.

In [17]: the two matrices and the phase of the term.

```
from matplotlib.colors import LinearSegmentedColormap
```

```
SIGNS = LinearSegmentedColormap.from_list("signs", ["#2a78d6", "#f0efec", "#e34948"])
fig, axes = plt.subplots(1, 3, figsize=(13.0, 4.2), width_ratios=[1, 1, 1.3])
```

The blue-white-red colour map and three panels, the last one wider.

```
for ax, M, title in ((axes[0], C, "$C$ (symmetric)"),
                     (axes[1], CG, "$C\\Gamma$ (symmetric)")):
    image = ax.imshow(M, cmap=SIGNS, vmin=-1, vmax=1)
    ax.set_title(title)
    ax.set_xticks([0, 7, 15], ["1", "8", "16"])
    ax.set_yticks([0, 7, 15], ["1", "8", "16"])
    ax.set_xlabel("column")
    ax.axhline(7.5, color="black", linewidth=0.8)
    ax.axvline(7.5, color="black", linewidth=0.8)
    ax.grid(False)
axes[0].set_ylabel("row")
```

Heat maps of C and $C\Gamma$ with the lines that separate the two halves.

```
alphas = np.linspace(0.0, np.pi, 91)
majorana = np.array([(np.exp(1j * x) * psi_c) @ C @ (np.exp(1j * x) * psi_c)
                     for x in alphas])
scalar = np.array([np.conj(np.exp(1j * x) * psi_c) @ C @ (np.exp(1j * x) * psi_c)
                   for x in alphas])
base = np.angle(majorana[0])
```

91 phases α from 0 to π; for each, the Majorana-type term $\Psi^T C \Psi$ and the scalar $\Psi^\dagger C \Psi$ of the rotated field. `np.angle` gives the phase of a complex number; `base` is the phase of the term at $\alpha = 0$.

```
axes[2].plot(alphas, np.unwrap(np.angle(majorana)) - base, linewidth=2,
             label="$\\Psi^T C\\Psi$: phase $2\\alpha$ (charge 2)")
axes[2].plot(alphas, np.angle(scalar) - np.angle(scalar[0]), "--", linewidth=2,
             label="$\\Psi^\\dagger C\\Psi$: unchanged (charge 0)")
axes[2].set_xlabel("phase $\\alpha$ of $\\Psi \\to e^{i\\alpha}\\Psi$")
axes[2].set_ylabel("change of the phase of the term")
axes[2].legend(fontsize=8)
axes[2].set_title("A Majorana-type term carries charge 2")
```

The change of the phase of each term against α. `np.angle` returns values between $-\pi$ and π, so the phase of the Majorana-type term jumps by 2π when it passes π; `np.unwrap` removes these jumps.

```
save_figure(fig, "majorana_terms",
            "Left and middle: heat maps of the only two matrices $M$ for which a "
            "Majorana-type term $\\Psi^T M\\Psi$ is invariant under the rotations and "
            "boosts of Spin(4,4), $C$ and $C\\Gamma$ (horizontal axis the column, "
            "vertical axis the row, red $+1$, blue $-1$); both are symmetric, so "
            "the term vanishes for anticommuting components. Right: the change of "
            "the phase of $\\Psi^T C\\Psi$ (solid) and of $\\Psi^\\dagger C\\Psi$ "
```

```
            "(dashed) for a commuting field under $\\Psi \\to e^{i\\alpha}\\Psi$, "
            "versus $\\alpha$ (radians): the Majorana-type term turns twice as "
            "fast, it carries U(1) charge 2 and would break the U(1) symmetry "
            "and with it the local conservation law of the charge.")
```

Figure 21d.7. **What the figure shows.** Left and middle: two block-diagonal patterns, each its own mirror image across the diagonal (symmetric). Right: a straight line of slope 2, reaching 2π at $\alpha = \pi$, and a flat dashed line at zero.

In [18]: the scorecard from the record's verdicts.

```
def status(passed, text):
    """The status text when the record check passed, otherwise UNSUPPORTED."""
    return text if passed else "UNSUPPORTED"
```

A status is written only if the record check behind it passed; otherwise the row would read UNSUPPORTED (and the check below would fail).

```
ROWS = [
    ("1. a process changes the number",
     "local U(1) law d_mu(cos z J^mu) = 0 for every history a4: no process inside "
     "the patch changes Q; the total Q is constant only if no charge flows "
     "through the brane z = pi/2 (ASSUMED, not derived)",
     status(LEAD["u1_noether_matrix_identity"] == "PASS",
            "inside the patch: FAILS (PROVED); total: no flux ASSUMED"),
     "u1_noether_matrix_identity"),
```

Each row has four entries: the condition, what this theory has, the status, and the record check. Row 1 has two parts (Section 21.31): condition 1 fails for every process inside the patch, proved by the U(1) identity (the local law, written in plain letters: `d_mu(cos z J^mu) = 0` is $\partial_\mu(\cos z\, J^\mu) = 0$); for the total charge of a universe the no-flux condition at the brane is needed, and the status says that it is ASSUMED. The status text is written only if the lead check `u1_noether_matrix_identity` has the verdict PASS.

```
    ("2. C and CP violated",
     "commuting field: the same-mass conjugation is an exact symmetry that "
     "reverses the charge, so a C-symmetric start keeps zero charge; quantised "
     "field: the only conjugation, Gamma, reverses the mass; no rates computed",
     status(all(LEAD[name] == "PASS" for name in (
         "representation_real", "spinor_connection_real",
         "bilinears_under_charge_conjugation",
         "quantum_charge_conjugation_unitary_type")),
         "commuting: FAILS (PROVED); quantised: NOT COMPUTED"),
     "representation_real, spinor_connection_real, "
     "bilinears_under_charge_conjugation, quantum_charge_conjugation_unitary_type"),
```

Row 2 has two statuses, one for each field (Section 21.31). For the commuting field dirac16complex00 condition 2 fails: the same-mass conjugation is exact (all matrices of $\mathcal{L}$ are real) and reverses the charge, so by Proposition 2 of Section 21.5 a start in which every configuration and its conjugate are

equally likely keeps zero average charge. For the quantised field dirac16complex it is not computed. `all(...)` is `True` only when every one of the four named lead checks has the verdict PASS: `representation_real` and `spinor_connection_real` (the reality of the gammas, of C and of the Ω_μ), `bilinears_under_charge_conjugation` (the conjugation reverses the current) and `quantum_charge_conjugation_unitary_type` (the quantised field has only the mass-reversing conjugation). Python joins two string pieces that stand next to each other into one text: the last entry is the list of the four checks.

```
    ("3. out of equilibrium",
     "no rate computed; the Kohn-Sham history is a prescribed background",
     status(KS["ks_history_is_a_prescribed_background"] == "PASS", "NOT COMPUTED"),
     "ks_history_is_a_prescribed_background"),
```

Row 3: not computed; the only history of the record is a prescribed background.

```
    ("pair level (theorem T1)",
     "the partner Gamma Psi of mass -m carries the opposite current: total charge 0",
     status(PAIR["T1_current_primordial_commuting"] == "PASS"
            and PAIR["T1_current_primordial_grassmann"] == "PASS",
            "PROVED (classical bilinears)"),
     "T1_current_primordial_commuting, T1_current_primordial_grassmann"),
```

The pair-level row: proved for classical bilinears, for both fields.

```
    ("verdict",
     "no process makes a net charge at any point of the patch 0 < z < pi/2 of one "
     "universe (nor a total charge, under the ASSUMED no-flux condition at the "
     "brane); no baryons in the theory",
     status(LEAD["u1_noether_matrix_identity"] == "PASS", "PROBLEM NOT SOLVED"),
     "the checks above")]
for row in ROWS:
    say(f"{row[0]:34} | {row[2]}")
```

The verdict row (its second entry is again one text, made of three string pieces that stand next to each other), and a loop that prints the condition (padded to 34 characters) and the status of each row: Out [18].

```
check([row[2] for row in ROWS] == [
          "inside the patch: FAILS (PROVED); total: no flux ASSUMED",
          "commuting: FAILS (PROVED); quantised: NOT COMPUTED",
          "NOT COMPUTED", "PROVED (classical bilinears)", "PROBLEM NOT SOLVED"],
      "scorecard: every status is backed by a PASS verdict of the Revision record")
```

The five statuses must be exactly these, which is possible only if every record check behind them passed. The list of the five expected texts opens with `[` at the end of the first line and is written one or more texts per line.

In [19]: the scorecard as a figure.

```
COLOURS = {"inside the patch: FAILS (PROVED); total: no flux ASSUMED": "#f4c7c3",
           "commuting: FAILS (PROVED); quantised: NOT COMPUTED": "#f2e2b8",
           "NOT COMPUTED": "#e3e3e3", "PROVED (classical bilinears)": "#cfe8c4",
           "PROBLEM NOT SOLVED": "#f4c7c3"}
fig, ax = plt.subplots(figsize=(13.0, 6.2))
ax.set_xlim(0, 13.0)
ax.set_ylim(0, len(ROWS) + 1)
ax.axis("off")
```

A pale background colour for each status (red for the failure of condition 1, whose status names its two parts, and for the verdict, yellow for the two statuses of condition 2, fails for the commuting field and not computed for the quantised one, grey for not computed, green for proved). The figure is used as a drawing area with coordinates from 0 to 13 across and 0 to 6 up; `ax.axis("off")` hides the axes.

```
columns = [(0.1, 2.4, "Sakharov condition"), (2.6, 4.3, "this theory"),
           (7.1, 2.3, "status"), (9.5, 3.5, "Revision record check")]
for x0, width, heading in columns:
    ax.text(x0, len(ROWS) + 0.5, heading, fontsize=11, fontweight="bold",
            va="center")
```

The four columns, each with its left edge, its width and its heading; the headings are written in bold in the top line.

```
for k, row in enumerate(ROWS):
    y = len(ROWS) - k - 0.5  # the first row at the top
    ax.add_patch(plt.Rectangle((0.0, y - 0.48), 13.0, 0.96, color=COLOURS[row[2]],
                               zorder=0))
    for (x0, width, _), text in zip(columns, row):
        chars = int(width * 11.5)  # about 11 characters per unit of width
        ax.text(x0, y, textwrap.fill(text, chars, break_long_words=False),
                fontsize=8.5, va="center")  # long check names stay whole
ax.set_title("Sakharov's conditions applied to the theory as built", fontsize=12)
```

For each row, from the top down: a coloured band across the whole width (`zorder=0` puts it behind the text), and the four entries, each broken into lines that fit its column (`textwrap.fill` of the set-up cell; `break_long_words=False` keeps long check names whole).

```
save_figure(fig, "scorecard",
            "The scorecard of the theory as built against Sakharov's three "
            "conditions, with the status of each row and the Revision record "
            "check it rests on (each status is set by the notebook only when the "
            "record holds that check with the verdict PASS). Condition 1 fails "
            "for every process inside the patch (the local U(1) law holds for "
            "every history $a_4$), so there conditions 2 and 3 cannot help; the "
            "total charge of a universe is constant only under the no-flux "
            "condition at the brane $z = \\pi/2$, which is ASSUMED; condition 2 "
            "also fails for the "
            "commuting field (its same-mass conjugation is exact) and is not "
            "computed for the quantised field; the pair-level statement of theorem "
```

```
        "T1 is exact but creates nothing. The theory does not solve the "
        "matter-antimatter problem.")
```

Figure 21d.8. **What the figure shows.** A table of five coloured rows: red for condition 1 (fails inside the patch; for the total charge the no-flux condition is ASSUMED) and for the verdict (problem not solved), yellow for condition 2 (fails for the commuting field, not computed for the quantised field), grey for condition 3 (not computed), green for the pair level (proved for classical bilinears), each with the record checks it rests on.

In [20]: the figure files.

```
names = sorted(FIGURE_NUMBERS, key=FIGURE_NUMBERS.get)
check(len(names) == 8 and all(
    output_file(f"{FIGURE_FOLDER}/21d_{FIGURE_NUMBERS[n]}_{n}.png").is_file()
    for n in names), "the eight figure files of notebook 21d exist")
all_checks_passed()
```

The eight figure files exist, and the last line prints ALL 19 CHECKS PASSED (notebook 21d).

21.37 The scorecard, pairs of universes as a hypothesis, and what would be needed

The scorecard. Figure 21d.8 and the table of Section 21.31 put the three conditions side by side. Condition 1 fails exactly for every process inside the patch, and for the total charge of a universe under the ASSUMED no-flux condition at the brane; condition 2 fails exactly for the commuting field and is not computed for the quantised field; condition 3 is not computed; and conditions 2 and 3 could not help while condition 1 fails; the pair-level statement is exact but creates nothing. **The theory as built does not solve the matter-antimatter problem.** It contains no baryons, no process of it changes the charge at any point inside the patch $0 < z < \pi/2$ of one universe (the total charge could change only by a flow through the brane, which the record neither excludes nor describes), and it predicts no value of the baryon-to-photon ratio η_B.

Pairs of universes, stated as a hypothesis. Theorem T1 suggests a picture of the kind described at the end of Section 21.19: our universe carries some charge, and a partner universe of mass $-m$ carries the opposite charge, so that the pair as a whole is symmetric. The picture rests on three statements, each of which is a **HYPOTHESIS**: nothing in the Revision record derives it.

- **H1 (HYPOTHESIS).** Our universe is one member of a T1 pair: two configurations in the same gravitational field, Ψ with the parameters (m, λ) and its partner $\Gamma\Psi$ with $(-m, -\lambda)$, in this correlated configuration.
- **H2 (HYPOTHESIS).** The configuration has a nonzero charge, $Q \neq 0$. Given H1, the partner has $-Q$ by theorem T1; the content of H2 is $Q \neq 0$, and nothing computes its value or its sign.
- **H3 (HYPOTHESIS, not derivable within the theory).** The U(1) charge of the field is the baryon number (or the difference of baryon and lepton numbers). The theory contains no quarks, baryons or leptons, so this cannot be derived from it.

What would follow, and only for classical fields. If H1, H2 and H3 held, the two charges would add to zero at every time (theorem T1), each would be separately constant under the ASSUMED no-flux condition at the brane (Sections 21.17 and 21.18), and the baryon excess of one member would be balanced by the opposite excess of the other.

What the picture does not do.

- It does not produce an asymmetry. Under the no-flux condition each charge is constant, so a nonzero charge today is the same as a nonzero charge at the start: the asymmetry is an initial condition, put in by H2. Without that condition a charge could change only by a flow through the brane, which the record does not describe; no asymmetry is computed either way.
- It does not predict η_B: neither the size nor the sign of the charge is computed, nor the photons to which η_B refers.
- It does not meet Sakharov's conditions; it replaces them by an assumed correlated configuration.
- At the quantum level there is no cancellation between two independently quantised universes: the chirality image is the same quantum system with the Krein metric $-B$, and an independent universe of mass $-m$ carries $+B$ and its own charge (Section 21.19).
- It does not show that pairs are created (Chapter 20), and it does not say that antimatter has negative mass.
- It is a statement about a model in signature (4,4), with four time-like directions, while observed spacetime has one time; applying it to "our universe" is a further assumption.

A published example of the class of universe/anti-universe ideas is the CPT-symmetric universe of Boyle, Finn and Turok, Phys. Rev. Lett. 121, 251301 (2018) (Section 21.19); it is cited as an example of the class only.

What would have to be added (all OPEN; none of these ingredients is present in the theory, none is claimed to be natural, and none has been shown to be sufficient):

- *Baryons.* Couplings of the field to the quarks and leptons of the Standard Model of particle physics, or another link between the U(1) charge and the baryon number (H3).
- *A charge-violating interaction* (condition 1). For the commuting field the candidates found in Section 21.32 are the Majorana-type terms $\Psi^T C\Psi$ and $\Psi^T C\Gamma\Psi$ (each with its complex conjugate), of charge 2. For the anticommuting field no Majorana-type mass term exists; a charge-violating term would need derivatives, more fields or new structures, which are not classified here. A remark derived here but not checked by the record: if one such term $g\,\Psi^T M\Psi$ (plus its conjugate) is added, the rotation $\Psi \to e^{i\alpha}\Psi$ changes its coefficient to $g\,e^{2i\alpha}$, so the phase of a single coefficient can be removed by renaming the field; only relative phases between several such terms could be a source of CP violation.
- *C and CP violation in rates* (condition 2): rates of processes of the extended theory, computed and shown to differ between particles and antiparticles.

- *A departure from equilibrium* (condition 3): a dynamical history in which the charge-violating processes run out of equilibrium, for example during the deflation of the extra times, with their rates. The record has only instantaneous Kohn-Sham states along a prescribed history; the time-dependent problem and the back-reaction on a_4 are OPEN (Chapters 15, 17 and 22).
- *A computed* η_B: only with all of the above could a value be computed and compared with the observed $\eta_B \approx 6.1 \times 10^{-10}$ of Section 21.4.

21.38 What we proved, what we computed, what we assumed

statement	**status**	**where it is verified**
the observed excess of matter: $\Omega_b h^2 = 0.0224 \pm 0.0001$, $\eta_B \approx 6.1 \times 10^{-10}$; no antimatter domains	ASSUMED (quoted observations)	the literature cited in Section 21.4
Sakharov's three conditions (Propositions 1 and 2; the equilibrium surplus)	PROVED (Section 21.5) for the stated assumptions; the Fermi-Dirac law and equal particle and antiparticle masses quoted (ASSUMED)	Notebook 21d, In [7]
decay model $N(r - \bar{r})(\mathcal{B}_1 - \mathcal{B}_2)$; rate model $\eta(K)$ in closed form	PROVED for the ASSUMED toy models; COMPUTED (RK4 to 10^{-7})	Notebook 21d, In [5], In [9] to In [12]
the gammas are eight real 16×16 signed permutation matrices with signature (4,4); C, Γ, B, S^{ab} as stated	PROVED	`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, `representation_real`, `B_imaginary_hermitian`; Notebook 21a, In [3], In [5]
charge conjugation is a matrix: with the intertwiner condition as the definition (the record's), exactly $\mathcal{C}_+ = C$ (same mass) and $\mathcal{C}_- = \Gamma C$ (mass reversed); their transposition rules; both reality conditions consistent	PROVED	the same report, `intertwiners_same_mass`, `intertwiners_reversed_mass`, `charge_conjugation_matrix_plus`, `charge_conjugation_matrix_minus`, `majorana_conditions_consistent`; Notebook 21a, In [6] to In [10]
the intertwiner condition is enough, not necessary: for $V = 0$ the matrix $P_- C$ maps every solution to a solution without obeying it	PROVED	Section 21.8 (derivation); Section 5.28
each gamma equation halves the solution space	COMPUTED exactly	Notebook 21a, In [7]
signs of S, J and of all 256 bilinears for both statistics	PROVED	the same report, `bilinears_under_charge_conjugation`; Notebook 21a, In [14], In [15]

statement	status	where it is verified
real fields: $J=0$, $\mathcal{C}_+$ the identity, Γ the only nontrivial real map among the matrices that obey the intertwiner condition, with the mass reversed	PROVED	the same report, `real_fields_charge_conjugation`; Notebook 21a, In [18]
on the record's exact solution (any a_4): Ψ^* solves with $+m$, $\Gamma\Psi^*$ with $-m$	PROVED	`Revision/theory/reports/python-field-theory.json`, `exact_solution_family_x4_x0`; Notebook 21a, In [11] to In [13]
the Noether current $J^\mu=-i\bar\Psi\gamma^\mu\Psi$	PROVED (Section 21.16); COMPUTED at a point	record formula `current`; Notebook 21b, In [9]
exact local U(1) conservation law $\partial_\mu(\cos z\, J^\mu)=0$ in the author's metric for every a_4; the a_4' terms cancel because $\sqrt{\lvert g\rvert}=\cos z$	PROVED	lead check `u1_noether_matrix_identity`; Notebook 21b, In [5], In [6]
charge balance through the brane; a stationary solution with the constant charge $-1424/1875-13\sqrt{7}/1500=-0.782397$	PROVED (sympy)	Notebook 21b, In [10] to In [13]
the charge density is indefinite; a rest state of positive frequency with density $-7/25$	PROVED	Notebook 21b, In [16]
theorem T1: the partner $\Gamma\Psi$ has $(-m,-\lambda)$ and the opposite current; a pair has total charge 0 as classical bilinears	PROVED	`Revision/pairing/reports/wolfram-pairing.json`, `T1_current_primordial_commuting`, `T1_current_primordial_grassmann`, `T1_Lagrangian_primordial_commuting`; Notebook 21b, In [11], In [17]; Notebook 21d, In [14]
the reflections (P, T types) and their Krein signs	PROVED in the record	the Wolfram pairing report, `T2_Pn_in_Pin44`, `T2_character_of_Pn`, `Q_Krein_metric_of_images`
quantised field: of the charge-conjugation matrices only the one giving $\Psi\to\Gamma\Psi^{\dagger T}$ keeps the canonical anticommutator; it reverses the mass	PROVED	lead check `quantum_charge_conjugation_unitary_type`; Notebook 21c, In [4], In [6]
$Q'=16-Q$ (charge reversed up to a constant); $Q''=Q-16$ for the same-mass candidate	PROVED (Section 21.26); COMPUTED with explicit operators	Notebook 21c, In [8]

statement	status	where it is verified
one-particle spectra of $\pm m$ identical; inertia (4,4) at real, Krein-neutral at imaginary frequencies	PROVED	Wolfram pairing report, `Q_one_particle_maps`, `Q_one_particle_Krein_signatures`, `Q_one_particle_complex_and_zero_frequencies_Krein_neutral`; Notebook 21c, In [9] to In [12]
the same-mass conjugation is an exact symmetry of the commuting field	PROVED (Section 21.31); COMPUTED at a point	`Revision/theory/reports/wolfram-field-theory.json`, `L_real_C`; Notebook 21d, In [14]
invariant Majorana-type matrices: exactly C and $C\Gamma$; none for anticommuting components	PROVED	`Revision/algebra/reports/python-algebra.json`, `spin_commutant_dimension_2`; Notebook 21d, In [16]
the total charge of a universe is constant (no flux through the brane $z = \pi/2$)	ASSUMED (not derived; it fails on the exact solution of Section 21.18 with the general column χ and on the solution of Notebook 18c in Section 18.21, although the stationary solution of Section 21.18 has zero flux; OPEN)	Notebook 21b, In [12]; Notebook 18c
Sakharov condition 1	FAILS inside the patch (PROVED); for the total charge FAILS under the ASSUMED no-flux condition	lead check `u1_noether_matrix_identity`; Notebook 21d, In [13]
Sakharov condition 2 (C and CP violation)	dirac16complex00: FAILS (PROVED: $\mathcal{C}_+$ exact, Proposition 2); dirac16complex: NOT COMPUTED (no rates; only the mass-reversing conjugation)	lead checks `representation_real`, `spinor_connection_real`, `bilinears_under_charge_conjugation`, `quantum_charge_conjugation_unitary_type`; Notebook 21d, In [14], In [18]
Sakharov condition 3 (departure from equilibrium)	NOT COMPUTED	`Revision/field_equations_a4/reports/ks-source-conditions.json`, `ks_history_is_a_prescribed_background`
our universe is one member of a pair; the U(1) charge is the baryon number	HYPOTHESIS	Section 21.37
a creation process, rate or amplitude for universes, in pairs or otherwise	not derived by any equation of the record	Chapter 20; `Revision/docs/PAIR_CREATION_PROOFS.md`, its section 11.2

statement	status	where it is verified
a regularised quantum theory in signature (4,4); anomalies of the U(1) symmetry	OPEN	not constructed in the record
normal ordering adds no sign: $:Q':=-Q$, $:Q'':=Q$ (the same-mass candidate keeps the charge)	PROVED (Section 21.26); COMPUTED with explicit operators; the record's detail text states the same since its correction of 2026-10-08 (no Fock-space computation there)	lead check `bilinears_under_charge_conjugation`, its detail text; Notebook 21c, In [8]

In one sentence. In this theory charge conjugation is a matrix, the charge obeys an exact local conservation law while the extra times deflate (its total is constant only under the ASSUMED no-flux condition at the brane), a solution and its chirality partner carry opposite charges, and nothing in the theory as built can make more matter than antimatter.

21.39 Exercises

1. **Bookkeeping.** Count Q_{el}, $\mathcal{B}$ and L before and after (a) the annihilation $p+\bar{p}\to\pi^+ +\pi^-$ and (b) the hypothetical decay $n\to p+e^-$ without the antineutrino. Which numbers are conserved?

2. **The decay model.** For $N=1000$ pairs, $r=0.51$, $\bar{r}=0.49$, $\mathcal{B}_1=\frac{2}{3}$, $\mathcal{B}_2=-\frac{1}{3}$, compute the net baryon number with the formula of Section 21.5. Then repeat with $\mathcal{B}_1=\mathcal{B}_2=\frac{1}{3}$ and with $r=\bar{r}=0.5$.

3. **The equilibrium surplus.** Show that at $E=0$ the surplus f_+-f_- of Section 21.5 equals $\tanh(\mu/2T)$, and evaluate it for $\mu=0.1T$. Then show that for E much larger than T and μ it is approximately $2\sinh(\mu/T)\,e^{-E/T}$.

4. **The rate model at $K=2$.** Use $\gamma(2,x)=1-(1+x)e^{-x}$ to compute the efficiency $\eta(2)$ exactly, and check it against the values $\eta(0.1)$ and $\eta(3)$ of Notebook 21d.

5. **Why C is not one of the matrices M.** Show that C commutes with the four time-like gammas and anticommutes with the four space-like ones. Explain why this does not contradict $\mathcal{C}_+=C$.

6. **Real fields carry no current.** For the 2×2 antisymmetric matrix A with $A_{12}=1$, $A_{21}=-1$ and $A_{11}=A_{22}=0$, compute u^TAu for a real column $u=(u_1,u_2)$ and $u^\dagger Au$ for a complex one. What does this say about the current of a real and of a complex field?

7. **The charge of the general exact solution.** For the exact solution $\Psi=\sin^\alpha z\,U(x_4)\chi$ of Section 21.11 with any $\alpha>-\frac{1}{2}$, show that the charge of the patch is $Q(x_4)=f(x_4)/(2\alpha+1)$ with $f=\chi^\dagger U^T BU\chi$.

8. **A half-way conjugation.** Compute $MB^TM^\dagger$ for $M=(1+\Gamma)/\sqrt{2}$, and explain with the formula of Section 21.25 why the result is the zero matrix.

9. **Anticommuting components and a 2×2 Majorana term.** For two anticommuting components Ψ_1, Ψ_2 and the matrix M with entries $M_{11}=a$, $M_{12}=b$, $M_{21}=c$, $M_{22}=d$, compute $\Psi^T M \Psi$. When does it vanish?

21.40 Answers to the exercises

1. (a) Before: $Q_{\text{el}} = 1-1=0$, $\mathcal{B}=1-1=0$, $L=0$. After: $Q_{\text{el}}=1-1=0$, $\mathcal{B}=0+0=0$, $L=0$. All three are conserved (such annihilations into two pions are observed). (b) Before: $Q_{\text{el}}=0$, $\mathcal{B}=1$, $L=0$. After: $Q_{\text{el}}=1-1=0$, $\mathcal{B}=1+0=1$, $L=0+1=1$. The charge and the baryon number are conserved, the lepton number would change by $+1$: this is why the observed decay contains the antineutrino ($L=-1$), as the table of Section 21.3 shows.

2. With the formula $N(r-\bar{r})(\mathcal{B}_1-\mathcal{B}_2)$: $1000\times(0.51-0.49)\times(\frac{2}{3}+\frac{1}{3}) = 1000\times 0.02\times 1 = 20$ baryons. With $\mathcal{B}_1=\mathcal{B}_2$ the last factor is 0, and with $r=\bar{r}$ the middle factor is 0: no net baryon number in either case (conditions 1 and 2 of Section 21.5).

3. At $E=0$, $\cosh 0=1$, so $f_+ - f_- = \sinh(\mu/T)/(1+\cosh(\mu/T))$. With $u=\mu/2T$ and the double-angle formulas $\sinh 2u = 2\sinh u\cosh u$ and $1+\cosh 2u = 2\cosh^2 u$ (both follow from the definitions with exponentials), this is $2\sinh u\cosh u/(2\cosh^2 u) = \tanh u = \tanh(\mu/2T)$. For $\mu=0.1T$: $\tanh 0.05 = 0.049958$ (to six decimals). For large E, $\cosh(E/T) = (e^{E/T}+e^{-E/T})/2 \approx e^{E/T}/2$, which is much larger than $\cosh(\mu/T)$; so $f_+ - f_- \approx \sinh(\mu/T)/(e^{E/T}/2) = 2\sinh(\mu/T)\,e^{-E/T}$: the surplus dies out exponentially with the energy, as figure 21d.3 shows.

4. With $K=2$: $K/(K-1)=2$, $(1-e^{-2})/2$, $K^{-K}=\frac{1}{4}$ and $\gamma(2,2)=1-3e^{-2}$. So

$$\eta(2) = 2\Big(\frac{1-e^{-2}}{2} - \frac{1-3e^{-2}}{4}\Big) = (1-e^{-2}) - \frac{1-3e^{-2}}{2} = \frac{1}{2} + \frac{e^{-2}}{2} = 0.567668$$

(multiply out; collect the constants $1-\frac{1}{2}$ and the terms with e^{-2}, $-1+\frac{3}{2}=\frac{1}{2}$; $e^{-2}=0.135335$). It lies between $\eta(0.1)=0.9955$ and $\eta(3)=0.4110$, as it must, since η decreases with K (Notebook 21d, In [12]). (The formula $\gamma(2,x)=\int_0^x ue^{-u}du = 1-(1+x)e^{-x}$ follows by integration by parts.)

5. $C=\gamma^{(x0)}\gamma^{(x1)}\gamma^{(x2)}\gamma^{(x3)}$. Moving a time-like $\gamma^{(t)}$ through C passes four different gammas, each costing a sign: $(-1)^4=+1$, so they commute. Moving a space-like gamma that occurs in C passes the three other factors (three signs) and itself (no sign): $(-1)^3=-1$, so they anticommute. Hence C satisfies neither $C\gamma^a = +\gamma^a C$ for all a nor $C\gamma^a=-\gamma^a C$ for all a: it is not one of the matrices M of Theorem CC. There is no contradiction: the charge-conjugation matrix is $\mathcal{C}=MC$, and $\mathcal{C}_+ = C$ belongs to $M=1$. The map is $\Psi^c = \mathcal{C}_+\bar{\Psi}^T = C\,C\Psi^* = \Psi^*$; the matrix C appears twice, once in $\mathcal{C}_+$ and once in the Dirac adjoint, and $CC=1$.

6. $u^T A u = u_1A_{12}u_2 + u_2A_{21}u_1 = u_1u_2 - u_2u_1 = 0$ for real (indeed for any commuting) numbers. For a complex column, $u^\dagger A u = u_1^*u_2 - u_2^*u_1$, and since $u_2^*u_1$ is the conjugate of $u_1^*u_2$, this is $2i\,\text{Im}(u_1^*u_2)$, which is nonzero in general (for $u=(1,i)$ it is $2i$). The current is $J^a = -i\Psi^\dagger(C\gamma^a)\Psi$ with the antisymmetric $C\gamma^a$: it vanishes for every real field (fact F1 of Section 21.10) and is in general nonzero, and real ($-i\cdot 2i\,\text{Im} = 2\,\text{Im}$), for a complex field. A real field carries no U(1) charge.

7. Since U is real, $\Psi^\dagger B\Psi = \sin^{2\alpha}z\,\chi^\dagger U^T B U\chi = \sin^{2\alpha}z\,f(x_4)$. The charge of the patch is $\int_0^{\pi/2}\cos z\,\sin^{2\alpha}z\,dz\,f(x_4)$. Substitute $u=\sin z$, $du=\cos z\,dz$, limits 0 and 1: $\int_0^1 u^{2\alpha}du = 1/(2\alpha+1)$ (the power rule of integration, valid for $2\alpha>-1$). So $Q=f/(2\alpha+1)$; for $\alpha=1$ this is the $f/3$ of Section 21.18.

8. $M = (1+\Gamma)/\sqrt{2}$ is $\cos t\, 1 + \sin t\, \Gamma$ with $t = \pi/4$. By Section 21.25, $M(t) B^T M(t)^\dagger = -\cos(2t) B$, and $\cos(\pi/2) = 0$: the result is the zero matrix. Directly: M is real and symmetric, so $M B^T M^\dagger = -MBM = -\frac{1}{2}(B + \Gamma B + B\Gamma + \Gamma B \Gamma) = -\frac{1}{2}(B + 0 - B) = 0$, because $\Gamma B = -B\Gamma$ and $\Gamma B \Gamma = -B$. In fact $(1+\Gamma)/2$ is the projector onto the components 9 to 16, and B joins the two halves (figure 21c.1), so $\frac{1+\Gamma}{2} B \frac{1+\Gamma}{2} = 0$: such an M would give a field whose anticommutator vanishes, which is not a field of the theory at all.

9. $\Psi^T M \Psi = a\Psi_1\Psi_1 + b\Psi_1\Psi_2 + c\Psi_2\Psi_1 + d\Psi_2\Psi_2$ (the sum over the four entries). For anticommuting components $\Psi_1\Psi_1 = \Psi_2\Psi_2 = 0$ and $\Psi_2\Psi_1 = -\Psi_1\Psi_2$, so $\Psi^T M \Psi = (b-c)\Psi_1\Psi_2$. It vanishes exactly when $b = c$, that is, when M is symmetric. This is the 2×2 case of Section 21.32: since C and $C\Gamma$ are symmetric, the anticommuting field dirac16complex has no Majorana-type mass term.

22 Open problems and how a student could attack them

Every earlier chapter of this book marked the questions it could not answer with the label OPEN, and every starting point it could not derive with the label ASSUMED. This last chapter of Part V collects the most important of them and treats each one as a research problem for a student who has worked through Chapters 0 to 21. For each problem it says exactly what is asked, what the Revision record already proves or computes about it, why it is hard, what a first step could be, and in which files of the repository the work would start. For one of the problems, the question of how fast the three extra times x_5, x_6, x_7 may deflate before the instantaneous Kohn-Sham states of Chapters 14 and 15 stop describing the gas, the chapter carries out the first step completely, with Notebook 22a, as an example of what such a step looks like in practice. Nothing in this chapter is claimed to be solved.

22.1 What this chapter does

Why a chapter of open problems. A theory is finished when every question it raises has an answer. The theory of this book is not finished, and the honesty rule of Chapter 0 forbids pretending that it is. The Revision record proves some statements exactly (the pairing theorems, the field equations for a_4, the exact reduction of the Kohn-Sham problem), computes others numerically with measured errors (the Kohn-Sham states along the deflating history), and leaves the rest open. An open problem is not a failure of the record: it is the place where new work can begin, and pushing against the edges of a theory is one of the best ways to understand it. This chapter lists the edges.

The five parts of each problem. For each problem the chapter gives:

1. **the question**, put as precisely as possible, together with what would count as an answer;
2. **what is known**, every statement with its label and with the record file and check that verify it;
3. **why it is hard**: which obstacle is known, and which step nobody has taken;
4. **a first step** that a student could take with what this book teaches;
5. **where to start**: the files of the repository that already exist.

The problems.

problem	sections	status	where it arose
1. The creation question: are universes created, in pairs or otherwise?	22.4	OPEN; the pairing maps T1, T2, Q, T3 and the corollary C1 are PROVED	Chapters 18 to 20
2. Beyond the instantaneous Kohn-Sham states: the time-dependent problem	22.5 to 22.13	OPEN; a first estimate for the free field is COMPUTED in Notebook 22a	Chapters 14 and 15
3. The back-reaction of the gas on a_4	22.14	OPEN; no recorded Kohn-Sham state is an admissible source	Chapters 12 and 17
4. The Z2 brane and its junction conditions	22.15	the Z2 mirror is ASSUMED; its junction conditions are OPEN	Chapters 14, 18 and 19
5. The dark-sector hypotheses	22.16 to 22.24	HYPOTHESIS, INVESTIGATED by the record and established in neither form; the observer's density ASSUMED; the remaining questions OPEN; Notebook 22b reproduces the record's numbers	Chapters 9, 12 and 15
6. Matter and antimatter	22.25	not solved by the theory as built; every scenario is a HYPOTHESIS	Chapter 21
7. Smaller open items	22.26	OPEN, CHOSEN or CONVENTION, as listed there	several chapters

The problems are linked. The time-dependent problem (Problem 2) needs the history of a_4, which the back-reaction (Problem 3) would determine; both T2 and T3 of Chapters 18 and 19 rest on the Z2 brane (Problem 4); the dark-sector results of the record (Problem 5) are read on the prescribed history and rest on an ASSUMED observer density, and an equation of state that is a consequence of the coupled equations needs Problem 3 again; and a creation process (Problem 1) would need a dynamical geometry, which none of the equations of the record provides. A student is well advised to start with a problem whose first step is a finite calculation: Problem 2, whose first step this chapter carries out, or the first steps of Problems 3 and 4.

The notebooks of this chapter. Notebook 22a is the worked example of Problem 2. It reads the adiabaticity measure Q of the 75 recorded ground states, shows that Q grows exactly in proportion to the deflation rate, reproduces the record's Q with the solver's shooting method written again in Python, collapses every slice and every momentum shell onto one curve with the exact rescaling

identity, measures the jumps into the negative-energy branch, and solves the exact time evolution of one sector of the free field for deflation rates from 0.25 to 251. It draws eight figures, needs no Rust, and ends with the line ALL 20 CHECKS PASSED (notebook 22a). Notebook 22b is the worked example of Problem 5: it reads the committed outputs of the Revision dark-sector record, reproduces its key numbers (the observer identities on the computed history, the radiation-like Kohn-Sham gas, the dark-matter-like bulk band, the condensate, the mixtures and the models of dirac16complex00 against the observed values), each asserted against its record file and check, draws five figures, needs no Rust, and ends with the line ALL 28 CHECKS PASSED (notebook 22b).

The status of every statement. Every statement carries one of the five labels of Chapter 0: PROVED (exact, with the record file and check where the Revision record verifies it), COMPUTED (a number of a numerical computation, with its measured uncertainty and the file or notebook cell that holds it), ASSUMED (a starting point that is not derived), HYPOTHESIS (an idea that is stated and examined but not established) and OPEN (a question nobody has answered). Three special cases of ASSUMED, used as in Chapters 14 to 17, appear here as well: PRESCRIBED BACKGROUND (the history $a_4 = AHx_4$, given and not solved for), CHOSEN (a numerical or boundary choice, such as the tip of the hidden direction) and CONVENTION (a rule fixed by agreement, such as which levels count as particles). A few derivations of this chapter are the book's own and are not checked by a Revision verifier; each of them is written out line by line and says so.

What this chapter does not do. It does not solve any of the problems it lists. In particular, and in the words of the honesty rule: that the big bang creates universes in pairs is NOT proved, and no creation process, rate, amplitude or big-bang dynamics follows from the equations of the record (Section 22.4); the theory as built does NOT solve the matter-antimatter problem (Section 22.25); and the record's investigation of the dark-sector hypotheses establishes neither of them (Sections 22.16 to 22.24).

Notation and units. The author's coordinates are $x_0, \ldots, x_7$: x_1, x_2, x_3 are ordinary 3-space, which inflates with the scale factor $e^{a_4} \sin^{1/6} z$; x_4 is the time; x_5, x_6, x_7 are the three extra times, which deflate exponentially with the scale factor $e^{-a_4} \sin^{1/6} z$; x_0 is the hidden space direction, with $z = 6Hx_0$ between 0 and $\pi/2$. The signature is (4,4): $\eta = \mathrm{diag}(+1, +1, +1, +1, -1, -1, -1, -1)$ in the order $x_0, \ldots, x_7$. The hidden coordinate of the Kohn-Sham chapters is $y = \ln(\sin z)/(6H)$; it runs from the **tip** $y = -L$ (with $L = 3$) to the **brane** $y = 0$. A **slice** is one instant of the history, and $a_{4,0}$ is the value of a_4 there; the record's history is $a_4 = AHx_4$ with $A = 1$, on the background $b_4 = 1$ of the second function $b_4(x_4)$ of the author's metric of 2026-10-14 (Chapter 3), on which the Kohn-Sham record works. In all numbers $H = 1$ and $m = 1$, so energies and momenta are in units of m, lengths and times in units of $1/H$. A state of the record is named like N688_lamm2_a00: $N = 688$ quanta, the coupling $-\lambda_2$ (the tags lam0, lamp1, lamm1, lamp2, lamm2 stand for $\lambda = 0, +\lambda_1, -\lambda_1, +\lambda_2, -\lambda_2$), the slice $a_{4,0} = 0.0$ (the two digits are ten times the slice).

22.2 The words of this chapter

Most words of this chapter were defined in earlier chapters; they are repeated here in plain words so that the chapter can be read on its own.

- **Open problem**: a question that neither the Revision record nor this book answers, put precisely enough that one can recognise an answer.

- **Test field** and **back-reaction**: a field that is computed in a gravitational field given in advance, which the field itself does not change, is a test field. The change of the gravitational field caused by the energy and momentum of the field is its back-reaction. All Kohn-Sham results of the record are test-field results.

- **Prescribed background**: the history $a_4 = AHx_4$ used by the Kohn-Sham chapters. It is given, not solved for: the computed Kohn-Sham states cannot be its source in the field equations for a_4 (Chapter 17).

- **Rate** A and **e-fold**: on the history $a_4 = AHx_4$ the extra-time scale factor $e^{-a_4}\sin^{1/6} z$ shrinks by the factor $e = 2.718\ldots$ (one e-fold) in the time $1/(AH)$; the rate A is the number of e-folds per unit of time $1/H$.

- **Instantaneous** (or **adiabatic**) **state**: the ground state of the Kohn-Sham Hamiltonian of one slice, computed as if the slice lasted forever. The word **adiabatic** also describes an evolution that is so slow that every particle stays in its instantaneous level.

- **Time-dependent Kohn-Sham problem**: the problem in which the orbitals evolve in the time x_4 under the Hamiltonian of each moment, with the potentials recomputed at every moment from the evolving densities.

- **Sector**: one momentum shell n_2 (3-momentum of length $k = 0.25\sqrt{n_2}$), one block type $j = \pm 1$ and one brane parity (even or odd). The exact evolution never moves a particle from one sector to another. The **Fermi shell** of a state is its highest occupied shell.

- **Level**, **orbital** and **label**: a level ε and its orbital φ solve $h\varphi = \varepsilon\varphi$ for the Hamiltonian h of one slice. The label n is the integer that numbers the levels of one sector in order: $n = 0$ is the **band level** (bound to the brane; the occupied level of the Fermi shell), $n = 1, 2, \ldots$ are the **bulk levels** above it, and $n = -1, -2, \ldots$ are the levels of negative energy, the **negative branch**, which the record's filling convention treats as the normal-ordered sea that holds no particles.

- **Amplitude** and **probability**: when an orbital is written as a sum $\sum_n c_n\varphi_n$ of instantaneous orbitals, the complex number c_n is the amplitude of level n and $|c_n|^2$ the probability to find the particle in level n.

- **Basis**: a set of orbitals in which every orbital is written as a sum. A **truncated** basis keeps only some of them; a computation **converges** when its result stops changing as more are kept.

- **Dressing**: the small admixture of other levels that a particle carries while the background keeps moving. **Adiabaticity measure** Q: the size of this admixture to first order (Section 22.7). $G = Q/A$ is Q per unit rate.

- **Redshifted momentum** $q = ke^{-a_{4,0}}$: the 3-momentum as the deflating history sees it (Section 22.7).

- **Sudden limit**: the limit of an infinitely fast change, in which the orbital has no time to change at all (Section 22.8). **Breakdown**: the instantaneous picture breaks down when the probability that a particle is NOT in its instantaneous level stops being small.

- **Brane**: the end $y = 0$ (that is $z = \pi/2$) of the hidden direction. **Z2 mirror** (or **orbifold**): the construction that continues the universe beyond the brane by its mirror image. **Kink**:

a point where a function is continuous but its slope jumps. **Junction condition**: the rule that ties the jump of the slopes of the metric (or of a field) across a surface to the energy and momentum concentrated on that surface. **Tension**: the energy per unit area of a surface, which acts like a negative pressure along it.

- **Equation of state** w: the ratio p/ρ of a pressure to the energy density. **CPL parametrisation**: the formula $w(a) = w_0 + w_a(1 - a)$ for an equation of state that changes with the scale factor a of the observed universe. **Phantom**: $w < -1$.
- **Null energy condition** along a direction: $\rho + p \geq 0$, with p the pressure of that direction.
- **Einstein-Hilbert Lagrangian**: the Lagrangian density $\sqrt{|g|}R$ of the geometry, with R the Ricci scalar; varying its action with respect to the metric gives Einstein's equations without a source.
- **Quantum cosmology** and **wave function of the universe**: a quantum theory in which a few numbers that describe the whole geometry, such as a scale factor, are quantum variables; its wave function is a function of these numbers. The **Wheeler-DeWitt equation** is the equation this wave function must obey. The **tunnelling proposal** and the **no-boundary proposal** are two proposed boundary conditions that pick one solution of it; they are named in this chapter, not used.
- **Bogoliubov method**: a way to count the quanta of a field that a changing background creates: one compares the modes of the field before and after the change, and the part of a positive-energy mode of the start that has turned into negative-energy modes of the end measures the creation. It is quoted in this chapter, not derived or applied.
- **Hypothesis**: an idea that is stated and examined but not established; here, every scenario that goes beyond what the record proves or computes.

22.3 How to work on an open problem

The Revision record answered every question it did answer by one procedure, and a student who attacks an open problem should follow it, because it is what makes an answer trustworthy.

1. **Write the question down precisely**, with what would count as an answer and what would not. The specification `Revision/SPEC.md` is an example: section 7 asks for instantaneous Kohn-Sham states and says in the same sentence that the time-dependent problem is OPEN.
2. **Derive the result by hand**, line by line, as this book does.
3. **Verify every exact statement with two programs that share no code.** The record uses a Wolfram Language verifier and an independent Python checker built on sympy for each exact statement, each writing a report with a name, a verdict and a detail for every check. Include **negative controls**: inputs that must fail, so that a check that passes everything is caught.
4. **Compute every number with two independent methods** and compare them with tolerances fixed before the comparison, as the cross-check of Chapter 16 does. Repeat every run to show that it gives the same bytes.

5. **Never weaken a check to make it pass.** A failing check is information; the task is to find out why it fails (Chapter 16 tells how the first cross-check failed and what it found).

6. **Write down what is proved, what is computed, what is assumed and what is not claimed**, and label every statement.

The tools that already exist, and that the first steps below use, are these:

tool	what it does	where
exact verifiers (Wolfram Language)	prove identities with exact numbers and symbols and write a report	the files ending in `.wls` in the folders `Revision/algebra/wolfram`, `Revision/theory/wolfram`, `Revision/pairing/wolfram`, `Revision/field_equations_a4/wolfram`
independent exact checkers (Python, sympy)	the same statements without Wolfram code	the files whose names start with `check_` in the folders `Revision/algebra/python`, `Revision/theory/python`, `Revision/pairing/python`, `Revision/field_equations_a4/python`
the lead's checks	three further independent checks (the energy-momentum divergence, the Einstein and Gauss-Bonnet equations for a_4, charge conjugation and the U(1) charge)	the folder `Revision/lead_checks`
the Kohn-Sham theory	the exact reduction, the functional, the boundary conditions, the adiabaticity measure	`Revision/kohn_sham/ks-theory.json` with the verifiers in `Revision/kohn_sham/theory`
the Rust Kohn-Sham solver	shooting with RK4 and a Pruefer count, self-consistency, the adiabaticity table	the crate `Revision/kohn_sham/solver`
the reference solver and the cross-check	a second method and the comparison with tolerances fixed in advance	`Revision/kohn_sham/reference/run_reference.py`, `Revision/kohn_sham/checker/crosscheck_ks.py`
the source-condition checker	the Kohn-Sham states as sources of the equations for a_4	`Revision/field_equations_a4/python/check_ks_source_conditions.py`
the notebooks of this book	every example, built and checked byte for byte	`Revision/textbook/notebooks` and the tool `Revision/textbook/tools/nbkit.py`

22.4 Problem 1: the creation question

The question. The author asked for a proof "that Universes of masses {+mass, -mass} are created in pairs" (`Revision/README.md`), and the request for this book repeats it as "the big bang creates universes in pairs". As an open problem the question is: is there a theory in which the creation of a universe, or of a pair of universes of masses $+m$ and $-m$, is a process with a computable amplitude, probability or rate; and if so, what is it, and does it create pairs? An answer would be such a theory together with the computed number; a correspondence between solutions, however exact, is not an answer, because it says nothing about whether either solution comes into being.

What is known.

- PROVED (Chapter 18; `Revision/pairing/reports/wolfram-pairing.json` and `Revision /pairing/reports/python-pairing.json`): theorem T1, the chirality map Γ: $\mathcal{L}_{m,\lambda}[\Gamma\Psi] = -\mathcal{L}_{-m,-\lambda}[\Psi]$, so Γ maps every solution with (m, λ) to a solution with $(-m, -\lambda)$, with the energy-momentum tensor $T \to -T$ and the current $J \to -J$, in every gravitational field taken as a fixed background, for both fields.
- PROVED (same reports): theorem T2, Γ combined with a Pin(4,4) reflection of character -1, maps (m, λ) to $(-m, \lambda)$ with $\mathcal{L} \to +\mathcal{L}$; in the author's field it is the mirror across the Z2 brane (an ASSUMED construction), and the partner has EQUAL, not opposite, energy-momentum.
- PROVED (same reports): the quantum reading Q for dirac16complex: the chirality image carries the Krein metric $-B$ and is the same quantum system written in other variables; two universes quantised independently of each other carry $+B$ each, and their generators add without cancelling.
- PROVED (Chapter 19): theorem T3, every self-consistent instantaneous Kohn-Sham state with (m, λ, θ) has a partner with $(-m, +\lambda, \pi - \theta)$ with equal levels, occupations, energies and energy-momentum profiles (ASSUMED Z2 brane). The two reports are `wolfram-t3.json` and `python-t3.json` in the folder `Revision/pairing/kohn_sham/reports`.
- PROVED (Chapter 20): corollary C1, a T1 pair taken as the complete classical source of the author's metric is a zero source, and in Einstein gravity the author's metric then has no solution for $H > 0$. The derivation is short and is given below.
- The record itself lists what these theorems do not establish: the report `Revision/pairing /reports/python-pairing.json` holds, under its key `not_established`, twelve items. Among them: no creation process (nothing in these equations produces a universe, a pair of universes or a change of the number of universes); no rate, probability or amplitude (no wave function of the universe, path integral or tunnelling computation is part of the theorems); no dynamical necessity (a single universe of mass $+m$ is an equally valid solution without its partner); the vanishing total energy-momentum of a T1 pair is not a cancellation between two independently quantised universes; and the gravitational back-reaction is not part of the theorems.
- OPEN: whether any universe is CREATED, in pairs or otherwise. The equations of the record are field equations on a gravitational field that is given, and their canonical quantisation; no creation process, rate, amplitude or big-bang dynamics follows from them.

Corollary C1 in Einstein gravity, line by line. The field equations for a_4 in Einstein gravity (Chapter 12; `Revision/field_equations_a4/a4-equations.json`, key `einstein`) contain the x_4 component and the x_0 component

$$3(a_4')^2 + 21H^2 + \Lambda = -\kappa\rho, \qquad -3(a_4')^2 + 15H^2 + \Lambda = \kappa p_0,$$

where $a_4' = da_4/dx_4$, Λ is the cosmological constant, κ the gravitational coupling, ρ the energy density and p_0 the pressure of the source along x_0. By T1 the two members of a pair have $T' = -T$, so the total source $T + T'$ vanishes: $\rho = 0$ and $p_0 = 0$.

$$3(a_4')^2 + 21H^2 + \Lambda = 0, \qquad -3(a_4')^2 + 15H^2 + \Lambda = 0.$$

Rule: insert $\rho = 0$ and $p_0 = 0$ into the two components.

$$36H^2 + 2\Lambda = 0.$$

Rule: add the two equations; the terms $3(a_4')^2$ and $-3(a_4')^2$ cancel.

$$6(a_4')^2 + 6H^2 = 0.$$

Rule: subtract the second equation from the first; Λ cancels.

$$(a_4')^2 = -H^2.$$

Rule: divide by 6 and subtract H^2 from both sides.

The square of a real number is never negative, and $-H^2 < 0$ for $H > 0$. So no real history $a_4(x_4)$ exists: the only solutions of the two equations are the complex slopes $a_4' = \pm iH$ with $\Lambda = -18H^2$. PROVED: `Revision/field_equations_a4/reports/wolfram-a4-report.json`, check `einstein_no_vacuum_solution`; `Revision/field_equations_a4/reports/python-a4-report.json`, check `einstein_no_vacuum`; and independently `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, check `no_vacuum_for_H_positive`, which also lists the complex solutions. The consequence: a T1 pair cannot by itself be the source of the author's universe in Einstein gravity. This is a statement about sources in one common geometry; it is not a derivation that a geometry, or a pair, is created.

Why it is hard.

- *The geometry has no dynamics of its own in the record.* Every theorem of Chapters 18 to 20 is a statement about fields in a gravitational field that is given (hypothesis H1 of T1). Without a dynamical geometry there is no process "a universe appears", and no state "no universe" from which it could appear.
- *A universe is not a particle in a fixed background.* To speak of the creation of universes one needs a quantum theory in which the geometry itself is a quantum variable, such as the quantum cosmology of a few variables of the metric (the Wheeler-DeWitt equation, the

tunnelling and the no-boundary proposals). In plain words: **quantum cosmology** treats a few numbers that describe the whole geometry, such as a scale factor, as quantum variables; its **wave function of the universe** is a function of these numbers, and the **Wheeler-DeWitt equation** is the equation that this wave function must obey, the analogue for a geometry of the Schroedinger equation of a particle. That equation alone does not fix the wave function; it needs a boundary condition, and the **tunnelling proposal** (A. Vilenkin) and the **no-boundary proposal** (J. B. Hartle and S. W. Hawking) are two proposed boundary conditions of this kind. They are named here, not used. None of these is formulated for this 8-dimensional theory, and in signature (4,4) both the classical and the quantum side bring their own difficulties: modes with momentum along the extra times grow instead of oscillating (Chapter 8), and the quantised field has an indefinite (Krein) inner product (Chapter 10).

- *Nothing forces the partner to exist.* T1, T2 and T3 relate the solutions of two parameter sets. A single universe of mass $+m$ is an equally valid solution.
- *The cancelling pair has no home in Einstein gravity.* By C1 a T1 pair as the complete source of one common geometry leaves no solution for $H > 0$. A T2 pair does not cancel at all: its two members carry equal energy-momentum.

A first step. (a) Redo the derivation of C1 above, and its version for Einstein-Gauss-Bonnet gravity, in which vacuum solutions of the linear form $a_4 = AHx_4 + a_0$ exist exactly for $0 < \alpha_2 H^2 \leq 1/40$ (Chapter 12; checks `linear_member_vacuum_factor` and `einstein_gauss_bonnet_vacuum_linear` of both reports in `Revision/field_equations_a4/reports`). (b) Prepare the classical side of a reduced quantum model. From the four independent components of the Einstein tensor in the record (`Revision/field_equations_a4/a4-equations.json`, key `lovelockTensors.E1`) the student can compute the Ricci scalar of the author's metric; this is a derivation of this book, from the record's components. The record's field-theory verifier computes the same R from the metric directly: the detail of its check `curvature_nonzero_flat_only_formally` in `Revision/theory/reports/python-field-theory.json` states $R = 6(a_4'^2 - 7H^2)$, and its comparison `ricci_scalar` with the Wolfram verifier has the verdict agree. The trace of the Einstein tensor adds three 3-space components, the time component, three extra-time components and the hidden component:

$$G^{\mu}{}_{\mu} = 3\big(-3(a_4')^2 + a_4'' + 15H^2\big) + \big(3(a_4')^2 + 21H^2\big) + 3\big(-3(a_4')^2 - a_4'' + 15H^2\big) + \big(-3(a_4')^2 + 15H^2\big).$$

Rule: the trace is the sum of the eight diagonal components $G^{x_0}{}_{x_0} + \cdots + G^{x_7}{}_{x_7}$, and the record gives $G^{x_1}{}_{x_1} = G^{x_2}{}_{x_2} = G^{x_3}{}_{x_3}$ and $G^{x_5}{}_{x_5} = G^{x_6}{}_{x_6} = G^{x_7}{}_{x_7}$.

$$G^{\mu}{}_{\mu} = -18(a_4')^2 + 126H^2.$$

Rule: collect terms: $(a_4')^2$ appears with $-9 + 3 - 9 - 3 = -18$, a_4'' with $3 - 3 = 0$, and H^2 with $45 + 21 + 45 + 15 = 126$.

$$G^{\mu}{}_{\mu} = R - \tfrac{8}{2}R = -3R.$$

Rule: $G^{\mu}{}_{\nu} = R^{\mu}{}_{\nu} - \frac{1}{2}R\,\delta^{\mu}_{\nu}$; the trace of $R^{\mu}{}_{\nu}$ is R, and the trace of δ^{μ}_{ν} in eight dimensions is 8.

$$R = 6(a_4')^2 - 42H^2.$$

Rule: divide $-18(a_4')^2 + 126H^2 = -3R$ by -3.

The second derivative a_4'' has dropped out: the inflation of 3-space and the deflation of the extra times enter with opposite signs. The **Einstein-Hilbert Lagrangian** is the Lagrangian density $\sqrt{|g|}R$ of the geometry itself: when its action is varied with respect to the components of the metric, the Euler-Lagrange equations (Chapter 7) are Einstein's equations without a source (with a source, the Lagrangian of the matter is added). With $\sqrt{|g|} = \cos z$ independent of a_4 (Chapter 12), the Einstein-Hilbert Lagrangian $\sqrt{|g|}R$ of the author's metric contains a_4 only through $(a_4')^2$, which is the starting point of a reduced quantum model with the one variable a_4. Quantising such a model is a HYPOTHESIS-level programme of quantum cosmology; even if it is carried out, it gives a wave function of a_4, not a process that changes the number of universes, and both the boundary condition of that wave function and the meaning of "two universes" in it would have to be supplied. (c) Study the one kind of creation that could occur within the record's own equations, the creation of quanta of the field inside ONE universe by the changing background (OPEN: the record computes no such process), which in ordinary Dirac theory is the jump of a particle from a filled negative level to an empty positive one (the Bogoliubov method, introduced for expanding universes by L. Parker, Phys. Rev. 183, 1057 (1969); quoted, not derived here). Notebook 22a measures how strongly the deflating background couples the negative branch to the positive one in the free field (Section 22.13); whether such jumps mean the creation of pairs of quanta in the 4+4 quantisation with its Krein metric is OPEN, and they have nothing to do with the creation of universes.

Where to start. The proofs and their limits: `Revision/docs/PAIR_CREATION_PROOFS.md` (its section 11 lists what is proved and what is not), `Revision/pairing/pairing-theory.json` (key `not_established`), the verifiers `Revision/pairing/wolfram/verify_pairing.wls` and `Revision/pairing/python/check_pairing.py`; the equations for a_4: `Revision/field_equations_a4/a4-equations.json` with its two verifiers; the lead's check `Revision/lead_checks/einstein_gauss_bonnet_a4.py`, which has its own curvature code.

22.5 Problem 2: beyond the instantaneous Kohn-Sham states

The question. Chapters 14 and 15 computed instantaneous Kohn-Sham states: at each slice $a_{4,0}$ of the history $a_4 = AHx_4$ the solver finds the ground state of the Hamiltonian of that one instant, as if the background stood still. But the background does not stand still: the three extra times deflate, 3-space inflates, and the Hamiltonian changes with x_4. The open problem has two levels. First: how fast would the extra times have to deflate before the instantaneous states stop describing the gas, and what happens then? Second: solve the self-consistent time-dependent Kohn-Sham problem, in which the orbitals evolve in x_4 and the potentials follow the evolving densities. An answer to the first is an estimate of the breakdown rate with a measured error; an answer to the second is a solver for the time-dependent problem, checked like the solvers of Chapters 15 and 16.

What is known.

- PROVED: the reduction of the field equation to the hidden coordinate is exact also when a_4 depends on x_4; in each sector the orbital obeys $i\,\partial\chi/\partial x_4 = h(x_4)\chi$ with a Hermitian h, and the momentum, the block and the brane parity are conserved, so particles can only jump between

levels of the same sector. The record states this in `Revision/kohn_sham/ks-theory.json` under the keys `sectorAndAnsatz.exactReduction` and `adiabaticity.exactEvolution`, and both theory reports in the folder `Revision/kohn_sham/reports` verify it with their check `hamiltonian_16_hermitian`.

- PROVED: the identity behind the adiabaticity measure Q (Sections 22.6 and 22.7); check `adiabatic_offdiagonal_identity` of the report `ks-theory-python.json` in the same folder.
- COMPUTED: at the record's rate $A = 1$ the largest Q of each of the 75 ground states is at most 0.0935 (largest value 0.0934532, state N688_lamm2_a00), and $Q = 0$ exactly for $N = 8$; the column `transition_probability_estimate_Q2` gives Q^2, at most 0.0087335 (the record's name for Q^2; Section 22.7 explains how this book reads it). The table is `adiabaticity.csv` in the folder `Revision/kohn_sham/results/adiabatic`.
- COMPUTED: for the 15 series of the record (three particle numbers, five couplings) the set of occupied levels does not change between neighbouring slices (the tables `fermi-level-crossings.csv` and `history.json` of the same folder). But the record also shows that this is not always so: for $N = 696$ at $\lambda = 0$ the brane-band levels redshift below the $k = 0$ bulk level along the history, the instantaneous ground state changes its occupied levels, and the state continued adiabatically from $a_{4,0} = 0$ lies above the instantaneous ground state by 3.657, 6.053, 7.612 and 8.623 (units of m) at the slices 0.5, 1, 1.5 and 2. This is the table `crossing-demo.csv` of the same folder, checked by `adiabatic_crossing_flag_demonstration` of the solver's report `Revision/kohn_sham/reports/ks-rust-solver.json`.
- PRESCRIBED BACKGROUND: the history itself (`Revision/kohn_sham/results/adiabatic/history.json`, key `status`).
- OPEN: the non-adiabatic (time-dependent) problem (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.caveat`).

Why it is hard. With interaction the effective mass $M = m + \frac{15}{16}\lambda S$ and the potential $v = -\lambda n/16$ depend on the densities of all occupied orbitals of all sectors (Chapter 14), so the evolution of one sector depends on all the others at every moment, and a gas of 688 quanta occupies many sectors. The functional itself is a question: the simplest choice, the ground-state functional evaluated with the densities of each moment, is the adiabatic approximation of time-dependent density functional theory, whose foundation in ordinary quantum mechanics is the theorem of E. Runge and E. K. U. Gross (Phys. Rev. Lett. 52, 997 (1984); quoted, not derived here); nobody has established its analogue for the 4+4 field with its Krein metric. When levels of different sectors cross the Fermi level, the instantaneous ground state and the evolved state part ways, as the case $N = 696$ shows. The negative branch is coupled to the positive one by the moving background, and how such jumps are to be read in the quantised theory is OPEN. And the history is prescribed: a full answer would also need Problem 3.

A first step: an estimate in the free field. In the free field ($\lambda = 0$) the mass is $M = m$ and $v = 0$ in every sector, so the sectors do not influence each other and the evolution equation of one sector is the complete, exact time evolution of the free field in that sector. That makes an exact first step possible. Sections 22.6 to 22.8 derive the equations it needs, Notebook 22a solves them (Sections 22.9 to 22.12), and Section 22.13 collects what it found and how a student could go on.

22.6 The amplitude equations, line by line

The setting. Take one sector. Its orbital is a pair of complex functions $\chi(y, x_4) = (\chi_1, \chi_2)$ on $-L \le y \le 0$. Its evolution equation is (`Revision/kohn_sham/ks-theory.json`, keys `adiabaticity.exactEvolution` and `blockEquation.hamiltonian`)

$$i\,\frac{\partial \chi}{\partial x_4} = h\,\chi, \qquad h = j\Big[-i\sigma_1\frac{d}{dy} + M\sigma_2 + \kappa k\sigma_3\Big] + v, \qquad \kappa = e^{-Hy-a_4(x_4)},$$

where $j = \pm 1$ is the block type, k the length of the 3-momentum, and σ_1, σ_2, σ_3 the Pauli matrices

$$\sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The factor κk is the 3-momentum as the orbital feels it: 3-space has the scale factor e^{Hy+a_4} in the hidden coordinate (Chapter 14), and a fixed coordinate momentum k corresponds to the physical momentum k divided by that scale factor. As the history goes on, a_4 grows and κk shrinks: the momenta redshift.

The inner product and Hermiticity. For two orbitals φ and ψ write $\langle\varphi|\psi\rangle = \int_{-L}^{0} \varphi^\dagger \psi\, dy$, where $\varphi^\dagger$ is the row of the complex conjugates of the components of φ. An operator h is **Hermitian** when $\langle\varphi|h\psi\rangle = \langle h\varphi|\psi\rangle$ for all orbitals that obey the boundary conditions; the Kohn-Sham h is (checks `hamiltonian_16_hermitian` and `bc_self_adjoint_boundary_term` of `Revision/kohn_sham/reports/ks-theory-python.json`). Its first consequence is that the total probability does not change. Write $\dot\chi = \partial\chi/\partial x_4$:

$$\frac{d}{dx_4}\langle\chi|\chi\rangle = \langle\dot\chi|\chi\rangle + \langle\chi|\dot\chi\rangle.$$

Rule: the product rule, applied inside the integral to $\chi^\dagger\chi$.

$$\frac{d}{dx_4}\langle\chi|\chi\rangle = \langle -ih\chi|\chi\rangle + \langle\chi| -ih\chi\rangle = i\langle h\chi|\chi\rangle - i\langle\chi|h\chi\rangle.$$

Rule: the evolution equation gives $\dot\chi = -ih\chi$; a factor $-i$ in the left slot comes out complex-conjugated as $+i$ (the left slot carries the dagger), in the right slot as $-i$.

$$\frac{d}{dx_4}\langle\chi|\chi\rangle = 0.$$

Rule: h is Hermitian, so the two terms are equal and cancel.

The instantaneous orbitals. At each slice the levels ε_n and orbitals φ_n solve $h\varphi_n = \varepsilon_n\varphi_n$ with the boundary conditions of the record (the regular tip, $b(-L) = 0$; the even brane parity, $b(0) = 0$, or the odd one). They are **orthonormal**: $\langle\varphi_m|\varphi_n\rangle = 1$ for $m = n$ and 0 otherwise. In the free field they depend on x_4 only through a_4, so we write $\varphi_n(y; a_4)$ and $\varepsilon_n(a_4)$. In the **real form** of the record (`Revision/kohn_sham/ks-theory.json`, key `blockEquation.realForm`) every orbital can be written $\varphi = (a, ib)$ with two real functions $a(y)$ and $b(y)$, which obey for $j = +1$

$$a' = Ma - (\kappa k + \varepsilon - v)\, b, \qquad b' = (\varepsilon - v - \kappa k)\, a - Mb,$$

where the prime is d/dy; in the free field $M = 1$ and $v = 0$. Then $\varphi^\dagger \varphi = a^2 + b^2$, and for two orbitals $\varphi_m^\dagger \sigma_3 \varphi_n = a_m a_n - b_m b_n$.

The equations for the amplitudes. Write ∂_a for the derivative with respect to a_4 at fixed y, and a dot for d/dx_4.

Line 1. Assume that the instantaneous orbitals form a basis, and write the evolving orbital as

$$\chi(y, x_4) = \sum_n c_n(x_4)\, \varphi_n(y; a_4(x_4)).$$

Rule: the expansion in a basis; the amplitudes c_n carry the time dependence that the orbitals of the slice do not. (A computer keeps finitely many levels; Notebook 22a keeps 20 and measures what the others would change.)

Line 2.

$$i\dot\chi = i \sum_n \Big[\dot c_n\, \varphi_n + c_n\, \dot a_4\, \partial_a \varphi_n \Big].$$

Rule: the product rule for each term $c_n \varphi_n$, and the chain rule for φ_n, which depends on x_4 through $a_4(x_4)$: $d\varphi_n/dx_4 = \dot a_4\, \partial_a \varphi_n$.

Line 3.

$$h\chi = \sum_n c_n\, \varepsilon_n\, \varphi_n.$$

Rule: h acts on each term (it is linear), and $h\varphi_n = \varepsilon_n \varphi_n$.

Line 4. Set line 2 equal to line 3, multiply from the left by $\varphi_m^\dagger$ and integrate over y:

$$i\dot c_m + i\dot a_4 \sum_n K_{mn} c_n = \varepsilon_m c_m, \qquad K_{mn} = \langle \varphi_m | \partial_a \varphi_n \rangle.$$

Rule: by orthonormality $\langle \varphi_m | \varphi_n \rangle$ is 1 for $n = m$ and 0 otherwise, so of the first and the last sum only the term $n = m$ survives; the middle sum keeps all terms and defines the numbers K_{mn}.

Line 5.

$$\dot c_m = -i\varepsilon_m c_m - \dot a_4 \sum_n K_{mn} c_n.$$

Rule: subtract $i\dot a_4 \sum_n K_{mn} c_n$ from both sides and multiply by $-i$ (because $1/i = -i$). This is the equation that Notebook 22a integrates. It says that the amplitudes turn with the frequencies ε_m and are mixed by the motion of the background, with the strength $\dot a_4 K_{mn}$.

Line 6. To compute K_{mn} for $m \neq n$, differentiate the eigenvalue equation $h\varphi_n = \varepsilon_n \varphi_n$ with respect to a_4:

$$(\partial_a h)\varphi_n + h\,\partial_a \varphi_n = (\partial_a \varepsilon_n)\varphi_n + \varepsilon_n\,\partial_a \varphi_n.$$

Rule: the product rule on both sides.

$$\langle \varphi_m | \partial_a h | \varphi_n \rangle + \varepsilon_m K_{mn} = 0 + \varepsilon_n K_{mn}.$$

Rule: multiply from the left by $\varphi_m^\dagger$ and integrate; h is Hermitian and its levels are real, so $\langle \varphi_m | h\psi \rangle = \langle h\varphi_m | \psi \rangle = \varepsilon_m \langle \varphi_m | \psi \rangle$ with $\psi = \partial_a \varphi_n$; and $\langle \varphi_m | \varphi_n \rangle = 0$ for $m \neq n$.

$$K_{mn} = \frac{\langle m | \partial_a h | n \rangle}{\varepsilon_n - \varepsilon_m} \qquad (m \neq n).$$

Rule: subtract $\varepsilon_m K_{mn}$ and divide by $\varepsilon_n - \varepsilon_m$, which is not zero for different levels of one sector; $\langle m | X | n \rangle$ is short for $\langle \varphi_m | X \varphi_n \rangle$. PROVED: check `adiabatic_offdiagonal_identity` of `Revision/kohn_sham/reports/ks-theory-python.json`.

Line 7. $K_{nn} = 0$:

$$1 = \langle \varphi_n | \varphi_n \rangle = \int (a_n^2 + b_n^2)\, dy \quad \Longrightarrow \quad 0 = 2 \int (a_n \partial_a a_n + b_n \partial_a b_n)\, dy = 2K_{nn}.$$

Rule: differentiate the normalisation with respect to a_4 (the derivative of the constant 1 is 0); with $\varphi_n = (a_n, i b_n)$ and real a_n, b_n the integrand of K_{nn} is $a_n \partial_a a_n + (-i b_n)(i \partial_a b_n) = a_n \partial_a a_n + b_n \partial_a b_n$.

Line 8. In the free field only κ depends on a_4, and

$$\partial_a \kappa = \partial_a e^{-Hy - a_4} = -\kappa, \qquad \partial_a h = -j\kappa k\, \sigma_3.$$

Rule: the derivative of e^{-a_4} with respect to a_4 is $-e^{-a_4}$; $M = 1$ and $v = 0$ do not depend on a_4. PROVED: the record states $\partial_a h_j = -j\kappa k \sigma_3$ for frozen potentials (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.derivative`) and verifies it with the check `adiabatic_hellmann_feynman` of `Revision/kohn_sham/reports/ks-theory-python.json`. Lines 1 to 5 and line 7 are derivations of this book, which no Revision check covers.

$$\langle m | \partial_a h | n \rangle = -k\, e^{-a_4} \int_{-L}^{0} e^{-Hy}\, (a_m a_n - b_m b_n)\, dy \qquad (j = +1).$$

Rule: $\kappa = e^{-Hy} e^{-a_4}$ (the rule $e^{s+t} = e^s e^t$), and $\varphi_m^\dagger \sigma_3 \varphi_n = a_m a_n - b_m b_n$ in the real form.

The total probability in the amplitudes. By orthonormality $\langle \chi | \chi \rangle = \sum_n |c_n|^2$, and it is 1 at all times by the conservation law above. Notebook 22a uses this as a test of its numerics: the computed $\sum_n |c_n|^2$ must stay 1.

22.7 What the adiabaticity measure Q measures, and two exact rules

The dressing. Let the history run at the constant rate $\dot{a}_4 = AH$, and let the particle be in level 0. If the other amplitudes are small, only the term $n = 0$ matters on the right of line 5 ("first order"), and over a short time the levels and the numbers K_{m0} hardly change. The amplitude of level 0 then turns as $c_0 = e^{-i\varepsilon_0 x_4}$. Try, for every $m \neq 0$, an amplitude that turns together with it, $c_m = \alpha_m e^{-i\varepsilon_0 x_4}$ with a constant α_m:

$$-i\varepsilon_0\,\alpha_m e^{-i\varepsilon_0 x_4} = -i\varepsilon_m\,\alpha_m e^{-i\varepsilon_0 x_4} - AH\,K_{m0}\,e^{-i\varepsilon_0 x_4}.$$

Rule: the left side is $\dot{c}_m$ (the derivative of $e^{-i\varepsilon_0 x_4}$ is $-i\varepsilon_0 e^{-i\varepsilon_0 x_4}$); the right side is line 5 with only $n = 0$ kept and $\dot{a}_4 = AH$.

$$i(\varepsilon_m - \varepsilon_0)\,\alpha_m = -AH\,K_{m0}.$$

Rule: divide by $e^{-i\varepsilon_0 x_4}$, which is never zero, and add $i\varepsilon_m\alpha_m$ to both sides.

$$\alpha_m = \frac{iAH\,K_{m0}}{\varepsilon_m - \varepsilon_0}.$$

Rule: divide by $i(\varepsilon_m - \varepsilon_0)$ and use $-1/i = i$.

$$|\alpha_m| = AH\,\frac{|\langle m|\partial_a h|0\rangle|}{(\varepsilon_0 - \varepsilon_m)^2}.$$

Rule: insert line 6, $K_{m0} = \langle m|\partial_a h|0\rangle/(\varepsilon_0 - \varepsilon_m)$, and take the absolute value ($|i| = 1$).

This is exactly the record's adiabaticity measure Q_{0m} (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.measure`: $Q_{nm} = AH|\langle n|\partial_a h|m\rangle|/(\varepsilon_n - \varepsilon_m)^2$ for n occupied and m empty in the same sector). So Q has a plain meaning: it is the amplitude, and Q^2 the probability, of the admixture of level m that a particle carries while the background moves, as long as Q is much smaller than 1. This admixture is the dressing. It is not a transition: when the motion stops slowly, the dressing disappears again. The record itself calls Q^2 the leading-order transition probability (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.measure`). For a constant rate this book reads it instead as the probability of the dressing; this reading is a derivation of this book, not of the record, and Notebook 22a tests it numerically in In [14]: at $A = 0.3$ the exact evolution, started in the dressed state, agrees with $\sum_m Q_{0m}^2$ within 15 percent. Only when the motion is started or stopped abruptly is Q^2 of the size of a real transition: the abrupt start at $A = 1$ in Figure 22a.6 (right) leaves up to about three times the dressing.

Exact rule 1: Q grows in proportion to the rate. The levels, orbitals and matrix elements of a slice depend only on $a_{4,0}$, not on how fast a_4 changes. The rate enters Q only through the factor AH. Hence $Q(A) = A\,Q(A = 1)$ exactly, and the naive criterion "breakdown when Q reaches 1" puts the breakdown at the rate $A = 1/Q_{max}$, where Q_{max} is the record's largest Q at $A = 1$.

Exact rule 2: one variable is enough. In the free field a_4 enters h only through the product

$$\kappa k = e^{-Hy - a_{4,0}}\,k = e^{-Hy}\left(k\,e^{-a_{4,0}}\right) = e^{-Hy}\,q, \qquad q = k\,e^{-a_{4,0}}.$$

Rule: $e^{s+t} = e^s e^t$, and the factors are regrouped. So the Hamiltonian of momentum k at the slice $a_{4,0}$ is the Hamiltonian of momentum q at the slice 0: same levels, same orbitals. This is the rescaling identity of the record (`Revision/kohn_sham/ks-theory.json`, key `rescalingIdentity`; PROVED: check `rescaling_identity` of `Revision/kohn_sham/reports/ks-theory-python.json`; COMPUTED by the solver: check `free_rescaling_relation_and_band_monotone` of `Revision/kohn_sham/reports/ks-rust-solver.json`, which finds the two sides equal to 0.0). By line 8 the matrix element is $-q \int e^{-Hy}(a_m a_n - b_m b_n)\, dy$ with the orbitals of momentum q at slice 0. Therefore, for $\lambda = 0$,

$$\frac{Q}{A} = G(q),$$

one function of one variable for every shell and every slice. We call q the **redshifted momentum**. As the history goes on, the q of every shell slides from k towards 0.

22.8 The sudden limit, two histories and three estimates of the breakdown

The adiabatic and the sudden limit. At a very small rate the particle stays in its instantaneous level, up to the dressing (the **adiabatic limit**). At a very large rate the opposite happens. Suppose that a_4 jumps from the value s_1 to the value s_2 in a time interval of length τ, and let τ go to zero. Integrating $\dot{\chi} = -ih\chi$ over the interval changes χ by at most τ times the largest size of $h\chi$ on it, which goes to zero with τ. So just after the jump the orbital is still the orbital $\varphi_0(s_1)$ of the start. Expanded in the orbitals of the new slice,

$$\varphi_0(s_1) = \sum_n \langle \varphi_n(s_2) | \varphi_0(s_1) \rangle \, \varphi_n(s_2),$$

Rule: the expansion of line 1 at the slice s_2; the amplitudes follow by multiplying with $\varphi_n(s_2)^\dagger$ and integrating (orthonormality).

and the probability that the particle is no longer in level 0 is

$$P_{sudden} = 1 - \left| \langle \varphi_0(s_2) | \varphi_0(s_1) \rangle \right|^2 = 1 - \Big(\int_{-L}^{0} \left(a_0(s_2) a_0(s_1) + b_0(s_2) b_0(s_1) \right) dy \Big)^2 .$$

Rule: the probability of level 0 is the square of the absolute value of its amplitude, and the probabilities of all levels add up to 1; in the real form the overlap is a real integral. The sudden limit depends only on how much the band orbital itself changes between the two slices. In the language of line 5, an infinitely fast passage is line 5 without the term $-i\varepsilon_m c_m$, which has no time to act; Notebook 22a computes the sudden limit both ways.

Time measured in units of the passage. Let a passage from $a_4 = 0$ to $a_4 = 2$ (the span of the record's slices) last the time T, and write $x_4 = Tu$ with u running from 0 to 1. Then

$$\frac{dc_m}{du} = T \, \frac{dc_m}{dx_4} = -iT\varepsilon_m c_m - \frac{da_4}{du} \sum_n K_{mn} c_n .$$

Rule: the chain rule, $dc/du = (dx_4/du)(dc/dx_4) = T\,dc/dx_4$, applied to line 5, and $T\dot{a}_4 = da_4/du$ by the same rule.

The smooth passage. The first history of Notebook 22a is

$$a_4(u) = 2\Big[u - \frac{\sin(2\pi u)}{2\pi}\Big].$$

At $u = 0$ it is 0 and at $u = 1$ it is 2, because $\sin 0 = \sin 2\pi = 0$. Its rate:

$$\frac{da_4}{du} = 2\big[1 - \cos(2\pi u)\big] = 4\sin^2(\pi u).$$

Rule: the derivative of $\sin(2\pi u)/(2\pi)$ is $\cos(2\pi u)$; then the identity $1 - \cos 2\theta = 2\sin^2\theta$ with $\theta = \pi u$.

$$\frac{da_4}{dx_4} = \frac{1}{T}\,\frac{da_4}{du} = \frac{4}{T}\sin^2(\pi u).$$

Rule: the chain rule with $du/dx_4 = 1/T$. The rate starts at 0, rises to its peak $4/T$ at $u = 1/2$ (where $\sin^2(\pi/2) = 1$) and falls back to 0. With $H = 1$ the peak rate is $A = 4/T$, so a passage with peak rate A lasts $T = 4/A$. Because the rate vanishes at both ends, the particle carries no dressing at the start and at the end: whatever probability has left level 0 at the end is a real transition.

The constant rate. The second history is the record's kind, $a_4 = 2u$, with the constant rate $da_4/dx_4 = 2/T = A$, so $T = 2/A$. Here the particle carries the dressing all the time, and the test is whether the exact evolution reproduces the first-order dressing $\sum_m Q_{0m}^2$ of Section 22.7.

First order for the smooth passage. Write $c_m = d_m\, e^{-iT\int_0^u \varepsilon_m\,du'}$, which removes the fast turning of each amplitude, and keep on the right of the u-equation only $c_0 = e^{-iT\int_0^u \varepsilon_0\,du'}$:

$$\frac{dd_m}{du} = -\frac{da_4}{du}\,K_{m0}\,e^{iT\int_0^u(\varepsilon_m-\varepsilon_0)\,du'}.$$

Rule: the product rule gives $dc_m/du = (dd_m/du - iT\varepsilon_m d_m)\,e^{-iT\int\varepsilon_m}$; the term $-iT\varepsilon_m c_m$ of the equation cancels the second part; dividing by $e^{-iT\int\varepsilon_m}$ and inserting c_0 gives the right side.

$$d_m(1) = -\int_0^1 \frac{da_4}{du}\,K_{m0}\,e^{iT\int_0^u(\varepsilon_m-\varepsilon_0)\,du'}\,du, \qquad P^{(1)} = \sum_{m\neq 0}|d_m(1)|^2.$$

Rule: integrate from $u = 0$, where $d_m = 0$ (the particle starts in level 0), to $u = 1$; $|c_m| = |d_m|$ because the exponential has absolute value 1. For a slow passage (large T) the fast-turning exponential makes the integral small: the transitions die out. This is a derivation of this book; Notebook 22a tests it against the exact evolution.

Three estimates of the breakdown. The instantaneous picture breaks down when the probability P that a particle is not in its instantaneous level stops being small. Notebook 22a compares three estimates of the rate at which that happens:

1. the naive estimate: Q reaches 1, that is $A = 1/Q_{max}$ (exact rule 1);

2. the first-order estimate: the dressing probability $(AQ_0)^2$ reaches the sudden value P_{sudden} of the passage from $a_{4,0} = 0$ to 2, that is $A = \sqrt{P_{sudden}}/Q_0$, where Q_0 is the record's largest Q of the free state $N = 688$ at the slice 0;

3. the exact evolution: the rate $A_{1/2}$ at which the exact P of the smooth passage reaches half of P_{sudden}.

22.9 Example: when does the instantaneous picture break down (Notebook 22a)

Notebook 22a carries out the first step of Problem 2. It reads the table `Revision/kohn_sham/results/adiabatic/adiabaticity.csv` (the largest Q of each of the 75 ground states), `Revision/kohn_sham/results/adiabatic/history.json` (the status of the history), `Revision/kohn_sham/results/ground/levels` (the level tables), `Revision/kohn_sham/results/parameters.json` (the units and numerical parameters) and `Revision/kohn_sham/ks-theory.json`. It writes the solver's shooting method again in Python for the free block, reproduces the record's levels and Q, draws the one curve $G(q)$ of exact rule 2, measures the jumps into the negative branch, and integrates the amplitude equation of Section 22.6 in a basis of 20 instantaneous levels for the smooth passage and the constant rate of Section 22.8. Everything that is computed beyond the record is computed for the free field ($\lambda = 0$) and for one sector: $j = +1$, even brane parity, the Fermi shell $n_2 = 11$ of $N = 688$. It needs only numpy and matplotlib, takes about a minute (the build and check runs of 2026-10-08 recorded in its provenance file took 18.5 and 15.6 seconds; on a computer busy with other work it takes longer), draws eight figures and ends with the line ALL 20 CHECKS PASSED (notebook 22a).

22.10 How to run Notebook 22a

Step 1. What this notebook does and what it needs.

Notebook 22a (When does the instantaneous Kohn-Sham picture break down) is the file `Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb` of the repository Dirac_claude. It estimates how fast the three extra times would have to deflate before the instantaneous Kohn-Sham states of dirac16complex stop describing the gas: it reads the adiabaticity measure Q of the 75 recorded ground states and shows that Q grows exactly in proportion to the rate, reproduces Q of the free Fermi-shell sectors with the solver's shooting method, collapses every slice and shell onto one curve with the exact rescaling identity, measures the jumps into the negative-energy branch, and solves the exact time evolution of the free Fermi-shell sector of N = 688 in a basis of 20 instantaneous levels for rates from 0.25 to 251, compared with the first-order estimate and the sudden limit. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell

(or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is

a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 22a_adiabatic_breakdown.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 22a_adiabatic_breakdown.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 22a_adiabatic_breakdown.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/22a.captions.json`

- `Revision/textbook/figures/22a_1_naive_rates.png`
- `Revision/textbook/figures/22a_2_one_curve.png`
- `Revision/textbook/figures/22a_3_across_the_gap.png`
- `Revision/textbook/figures/22a_4_sector_levels.png`
- `Revision/textbook/figures/22a_5_smooth_passage.png`
- `Revision/textbook/figures/22a_6_constant_rate.png`
- `Revision/textbook/figures/22a_7_sudden_ceiling.png`
- `Revision/textbook/figures/22a_8_convergence.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 22a)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.

- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 22a_adiabatic_breakdown.ipynb
python -m nbconvert --execute --inplace 22a_adiabatic_breakdown.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

22.11 Notebook 22a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The Kohn-Sham states of dirac16complex in the Revision record are **instantaneous**
states: at each slice $a_{4,0}$ of the history $a_4 = A H x_4$ the solver finds the
ground state of the Hamiltonian of that one instant. The record measures how good
this picture is with the adiabaticity measure $Q$ and finds $Q_{max} \le 0.0935$ at
the record's rate $A = 1$. Behind this number lies one of the OPEN problems of this
chapter, the time-dependent problem: **how fast would the three extra times $x_5$,
$x_6$, $x_7$ have to deflate before the instantaneous picture breaks down, and what
happens then?** This notebook makes that estimate in six steps:
```

- it reads Q_{max} of all 75 ground states of the record, shows why Q grows exactly in proportion to the rate A, and computes the naive breakdown rate $1/Q_{max}$ of every state;
- it solves again, with the Rust solver's shooting method written in vectorised Python, the Fermi-shell sectors of the free states $N = 136$ and $N = 688$ at the five slices and reproduces the record's Q_{max}, energy difference and matrix element;
- with the exact rescaling identity it collapses every slice and every shell onto ONE curve: Q/A against the redshifted momentum $q = k e^{-a_{4,0}}$;
- it measures the jumps across the gap into the negative-energy branch, which the record's filling convention leaves out;
- it solves the exact time evolution of one sector of the free field (where the evolution equation is exact) in a basis of 20 instantaneous levels, for rates from $A = 0.25$ to $A = 251$, and compares it with the first-order estimate and with the sudden limit;
- it collects the breakdown estimates in one table.

It draws eight figures and needs no Rust. Units: $H = m = 1$, as in the record.

Text cell:

2. How to run this notebook

Step 1. What this notebook does and what it needs.

Notebook 22a (When does the instantaneous Kohn-Sham picture break down) is the file ‘Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb‘ of the repository Dirac_claude. It estimates how fast the three extra times would have to deflate before the instantaneous Kohn-Sham states of dirac16complex stop describing the gas: it reads the adiabaticity measure Q of the 75 recorded ground states and shows that Q grows exactly in proportion to the rate, reproduces Q of the free Fermi-shell sectors with the solver's shooting method, collapses every slice and shell onto one curve with the exact rescaling identity, measures the jumps into the negative-energy branch, and solves the exact time evolution of the free Fermi-shell sector of N = 688 in a basis of 20 instantaneous levels for rates from 0.25 to 251, compared with the first-order estimate and the sudden limit. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the

installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
    xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
    sudo apt update
    sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
    py -3 --version
```

macOS (zsh) and Linux (bash):

```
    python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
    sudo add-apt-repository ppa:deadsnakes/ppa
    sudo apt update
    sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
    cd $HOME
    git clone https://github.com/once-ere/Dirac_claude.git
    py -3 -m venv "$HOME\dirac-book-env"
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
    cd ~
    git clone https://github.com/once-ere/Dirac_claude.git
    python3 -m venv ~/dirac-book-env
    source ~/dirac-book-env/bin/activate
    python -m pip install --upgrade pip
    python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
    python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
    python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
    Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
    & "$HOME\dirac-book-env\Scripts\Activate.ps1"
    cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
    source ~/dirac-book-env/bin/activate
    cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 22a_adiabatic_breakdown.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 1 minute on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 22a_adiabatic_breakdown.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 22a_adiabatic_breakdown.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/22a.captions.json‘
- ‘Revision/textbook/figures/22a_1_naive_rates.png‘
- ‘Revision/textbook/figures/22a_2_one_curve.png‘
- ‘Revision/textbook/figures/22a_3_across_the_gap.png‘
- ‘Revision/textbook/figures/22a_4_sector_levels.png‘
- ‘Revision/textbook/figures/22a_5_smooth_passage.png‘
- ‘Revision/textbook/figures/22a_6_constant_rate.png‘
- ‘Revision/textbook/figures/22a_7_sudden_ceiling.png‘
- ‘Revision/textbook/figures/22a_8_convergence.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it

runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 22a)
```

and the notebook must show 8 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

  ```
  python -m jupyterlab 22a_adiabatic_breakdown.ipynb
  python -m nbconvert --execute --inplace 22a_adiabatic_breakdown.ipynb
  ```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues

  and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
  and Run All Cells; if it fails again, install the packages again with the pip commands
  of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 22a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 22a (When does the instantaneous Kohn-Sham picture break down) is the file
# Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb of the repository
# Dirac_claude. It estimates how fast the three extra times would have to deflate before
# the instantaneous Kohn-Sham states of dirac16complex stop describing the gas: it reads
# the adiabaticity measure Q of the 75 recorded ground states and shows that Q grows
# exactly in proportion to the rate, reproduces Q of the free Fermi-shell sectors with
# the solver’s shooting method, collapses every slice and shell onto one curve with the
# exact rescaling identity, measures the jumps into the negative-energy branch, and
# solves the exact time evolution of the free Fermi-shell sector of N = 688 in a basis of
# 20 instantaneous levels for rates from 0.25 to 251, compared with the first-order
# estimate and the sudden limit. It needs a computer with Windows 11, macOS or Linux, an
# internet connection for the installation, the program Git, and Python 3.12 or newer
# (the notebooks were built with Python 3.14.5) with these packages at exactly these
# versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab
# 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The
# notebook itself imports numpy and matplotlib; the other packages run Jupyter, the
# program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
```

```
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
```

```
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 22a_adiabatic_breakdown.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 1 minute
# on a typical laptop. Scroll to the end and compare the last printed lines with the
# lines in the step "What the notebook writes and what you must see" below. To keep the
```

```
# results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the
# menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 22a_adiabatic_breakdown.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 22a_adiabatic_breakdown.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/22a.captions.json
# - Revision/textbook/figures/22a_1_naive_rates.png
# - Revision/textbook/figures/22a_2_one_curve.png
# - Revision/textbook/figures/22a_3_across_the_gap.png
# - Revision/textbook/figures/22a_4_sector_levels.png
# - Revision/textbook/figures/22a_5_smooth_passage.png
# - Revision/textbook/figures/22a_6_constant_rate.png
# - Revision/textbook/figures/22a_7_sudden_ceiling.png
# - Revision/textbook/figures/22a_8_convergence.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 20 CHECKS PASSED (notebook 22a)
#
# and the notebook must show 8 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
```

```
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/22a_adiabatic_breakdown.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 22a_adiabatic_breakdown.ipynb
#     python -m nbconvert --execute --inplace 22a_adiabatic_breakdown.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "22a"  # this notebook: chapter 22, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).
```

```
    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
```

```
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 22a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Slice** $a_{4,0}$: one instant of the history, named by its value of $a_4$.
  **History**: $a_4 = A H x_4$; the extra times shrink by the factor $e = 2.718...$
  (one **e-fold**) in the time $1/(AH)$. **Rate** $A$: how many e-folds per unit of
  time $1/H$; the record uses $A = 1$.
- **Sector**: one 3-momentum shell $n_2$ ($k = 0.25\sqrt{n_2}$), one block type
  $j$ and one brane parity. The exact evolution never moves a particle out of its
  sector. **Fermi shell**: the highest occupied shell of a state.
- **Instantaneous level and orbital**: a solution $h\varphi = \varepsilon\varphi$
  of the Hamiltonian $h$ of one slice. **Label** $n$: the integer that numbers the
  levels of a sector: $n = 0$ is the **band level** (bound to the brane, occupied in
  the Fermi shell), $n = 1, 2, ...$ the **bulk levels** above it (empty), $n = -1,
  -2, ...$ the levels of negative energy, the **negative branch** (in the record's
  convention the normal-ordered sea, not populated).
- **Amplitude** $c_n$: a complex number with $|c_n|^2$ = the probability to find
  the particle in level $n$. **Matrix element** $\langle m|X|n\rangle$: the integral
  $\int \varphi_m^\dagger X \varphi_n\,dy$.
- **Adiabatic**: so slow that the particle stays in its instantaneous level.
  **Sudden**: so fast that the orbital has no time to change at all.
- **Dressing**: the small admixture of other levels that a particle carries while
  the background keeps moving; **adiabaticity measure** $Q$: the amplitude of that
  admixture to first order (derived in section 4). $G(q) = Q/A$: $Q$ per unit rate.
- **Redshifted momentum** $q = k e^{-a_{4,0}}$: the 3-momentum as the deflating
  history sees it (section 4, exact rule 2).
- **Breakdown rate**: a rate at which the probability that a particle is NOT in its
  instantaneous level stops being small.
- **First order**: an approximation that keeps only terms linear in the small
  amplitudes. **Basis**: the set of instantaneous levels in which the evolving
  orbital is expanded; a **truncated** basis keeps only some of them, and
  **convergence** means the result stops changing when more are kept.
- **Unitary**: an evolution that keeps the total probability equal to 1.
- **RK4**: the classical Runge-Kutta method of order 4 for differential equations.
  **Bisection**: finding a root by halving an interval that contains it.
  **Vectorised**: one numpy operation acts on whole arrays of cases at once.
- **Chebyshev interpolation**: the polynomial through the values of a function at
  the Chebyshev nodes $s_j = 1 + \cos\theta_j$, $\theta_j = \pi(j + 1/2)/N$; for
  smooth functions it is accurate to many digits.
```

Text cell:

```
## 4. The physical and mathematical situation

**The background.** In the author's metric the three space directions $x_1, x_2,
x_3$ grow with the factor $e^{a_4}$ and the three extra times $x_5, x_6, x_7$ shrink
with $e^{-a_4}$ (both times $\sin^{1/6} z$); $x_4$ is the time and $x_0$ the hidden
direction, written with $y = \ln(\sin z)/(6H)$ from the tip $y = -L$ to the brane
$y = 0$. The record's history is $a_4 = A H x_4$ with $A = 1$, labelled a PRESCRIBED
BACKGROUND (it is not a solution of the $a_4$ equations with the Kohn-Sham source),
on the background $b_4 = 1$ of the second function $b_4(x_4)$ of the author's metric
of 2026-10-14, on which the Kohn-Sham record works.
Here $H = m = 1$ as in the record, and $A$ is the only thing we change.
```

The exact evolution. In each sector the Kohn-Sham orbital $\chi(y, x_4)$, with two real functions a and b in $\chi = (a, i b)$, obeys exactly (ks-theory.json, adiabaticity.exactEvolution)

$$i\,\frac{\partial\chi}{\partial x_4} = h\,\chi, \qquad
h = j\left[-i\sigma_1\frac{d}{dy} + M\sigma_2 + \kappa k\sigma_3\right] + v,
\qquad \kappa = e^{-Hy - a_4(x_4)}.$$

Without interaction ($\lambda = 0$) $M = m = 1$ and $v = 0$, and this equation is the complete time evolution of the free field in the sector; with interaction M and v depend on the state itself (self-consistent time-dependent Kohn-Sham theory, OPEN). This notebook solves the free case exactly, in the sector $j = +1$, even brane parity, of the Fermi shell $n_2 = 11$ of $N = 688$.

The equations for the amplitudes, line by line.

1. Write the orbital as a sum over the instantaneous orbitals, $\chi = \sum_n c_n(x_4)\,\varphi_n(a_4(x_4))$, with $\int\varphi_m^\dagger \varphi_n\,dy = 1$ for $m = n$ and 0 otherwise.
2. Left side: by the product rule and the chain rule, $i\partial_{x_4}\chi = i\sum_n[\dot c_n\varphi_n + c_n\dot a_4\,\partial_a \varphi_n]$; the dot is d/dx_4 and $\partial_a = \partial/\partial a_4$.
3. Right side: $h\chi = \sum_n c_n\varepsilon_n\varphi_n$, because every φ_n is an instantaneous orbital.
4. Multiply both sides by $\varphi_m^\dagger$ and integrate over y; the orthonormality of line 1 keeps $n = m$ in the first and last sums: $i\dot c_m + i\dot a_4\sum_n K_{mn}c_n = \varepsilon_m c_m$, with $K_{mn} = \int\varphi_m^\dagger\,\partial_a\varphi_n\,dy$.
5. Solve for the derivative: $\dot c_m = -i\varepsilon_m c_m - \dot a_4\sum_n K_{mn}c_n$. This is the equation the notebook integrates.
6. K_{mn} for $m \ne n$: differentiate $h\varphi_n = \varepsilon_n\varphi_n$ with respect to a_4: $(\partial_a h)\varphi_n + h\,\partial_a\varphi_n = (\partial_a\varepsilon_n)\varphi_n + \varepsilon_n\partial_a\varphi_n$. Multiply by $\varphi_m^\dagger$ and integrate; h is Hermitian, so $\int\varphi_m^\dagger h\,\psi\,dy = \varepsilon_m\int\varphi_m^\dagger\psi\,dy$: $\langle m|\partial_a h|n\rangle + \varepsilon_m K_{mn} = \varepsilon_n K_{mn}$, hence $K_{mn} = \langle m|\partial_a h|n\rangle/(\varepsilon_n - \varepsilon_m)$.
7. $K_{nn} = 0$: a and b are real, so $\int\varphi_n^\dagger\varphi_n\,dy = \int(a^2 + b^2)\,dy = 1$; its derivative is $2\int(a\,\partial_a a + b\,\partial_a b)\,dy = 2K_{nn} = 0$.
8. For $\lambda = 0$ only κ depends on a_4, and $\partial_a\kappa = -\kappa$, so $\partial_a h = -j\kappa k\sigma_3$ and, for $j = +1$ and $H = 1$, $\langle m|\partial_a h|n\rangle = -k e^{-a_4}\int e^{-y}(a_m a_n - b_m b_n)\,dy$ (with $\chi = (a, ib)$, $\chi_m^\dagger\sigma_3\chi_n = a_m a_n - b_m b_n$).

What Q measures. Start in level 0 at the constant rate $\dot a_4 = AH$. To first order (the other amplitudes small) line 5 reads $\dot c_m = -i\varepsilon_m c_m - AH K_{m0}c_0$ with $c_0 = e^{-i\varepsilon_0 x_4}$. Try $c_m = \alpha_m e^{-i\varepsilon_0 x_4}$ with a constant α_m: then $-i\varepsilon_0\alpha_m = -i\varepsilon_m\alpha_m - AHK_{m0}$, so $\alpha_m = iAHK_{m0}/(\varepsilon_m - \varepsilon_0)$: an admixture of level m that travels with the particle. By line 6 its size is

$$|\alpha_m| = AH\,\frac{|\langle m|\partial_a h|0\rangle|}{(\varepsilon_0 -

```
\varepsilon_m)^2} = Q_{0m},$$

the record's adiabaticity measure. So $Q$ is the amplitude, and $Q^2$ the
probability, with which the moving particle is NOT in its instantaneous level, as
long as $Q \ll 1$. This admixture is the **dressing**. (The record calls $Q^2$
the leading-order transition probability; reading it for a constant rate as the
probability of the dressing is this notebook's derivation, tested in section 14.)

**Exact rule 1: Q grows in proportion to the rate.** The levels, orbitals and
matrix elements of a slice depend on $a_{4,0}$, not on how fast $a_4$ changes; only
the factor $AH$ of $Q$ does. So $Q(A) = A\,Q(A = 1)$ exactly.

**Exact rule 2: one variable is enough.** $a_4$ enters $h$ only through
$\kappa k = e^{-Hy}(k e^{-a_4})$ (the rescaling identity of ks-theory.json, checked
by the solver). With the redshifted momentum $q = k e^{-a_{4,0}}$ the levels and
orbitals of momentum $k$ at the slice $a_{4,0}$ are those of momentum $q$ at the
slice $0$, and $\langle m|\partial_a h|n\rangle = -q\int e^{-y}(a_m a_n - b_m b_n)
\,dy$. Hence for $\lambda = 0$: $Q/A = G(q)$, one function of one variable for
every shell and every slice.

**The two limits.** At a small rate the particle stays in its instantaneous level,
up to the dressing (the adiabatic limit). At a very large rate the orbital has no
time to change: at the end it is still the orbital $\varphi_0$ of the start (the
sudden limit), and the probability that it has left level 0 is
$P_{sudden} = 1 - |\int\varphi_0(\mathrm{end})^\dagger\varphi_0(\mathrm{start})
\,dy|^2$.

**What we call breakdown.** The instantaneous picture breaks down when the
probability $P$ that a particle is not in its instantaneous level stops being
small. Three estimates are compared: the naive one, $Q = 1$, that is
$A = 1/Q_{max}$; the rate at which $Q^2$ reaches the sudden value $P_{sudden}$; and
the exact evolution.

**Status.** The record's $Q$ values are COMPUTED (Revision record). The curves,
the time evolution and the breakdown rates of this notebook are COMPUTED here for
the free field ($\lambda = 0$) in one sector, with measured numerical errors. The
self-consistent time-dependent problem ($\lambda \ne 0$) is OPEN, the history is a
PRESCRIBED BACKGROUND, the Z2 brane is ASSUMED, and the meaning of jumps into the
negative branch is OPEN.
```

Text cell:

```
## 5. The record: Q of the 75 ground states

The next cell reads three Revision records: the table `adiabaticity.csv` (one row
per ground state: $N = 8, 136, 688$; the couplings $\lambda = 0, \pm\lambda_1,
\pm\lambda_2$, tags lam0, lamp1, lamm1, lamp2, lamm2; the five slices), the file
`history.json` with the status of the history, and `ks-theory.json`. It prints the
largest $Q_{max}$ and the record's formula for $Q$, and checks the label of the
history and the two facts the record states: $Q_{max} \le 0.0935$, and $Q_{max} = 0$
exactly for $N = 8$ (only the $k = 0$ zero modes are occupied, and there
$\partial_a h = 0$).
```

In [2]:

```
import csv  # reads the tables (CSV files) of the Revision record
import math  # exp, sqrt and log of single numbers

import numpy as np  # arrays of numbers and their arithmetic
from numpy.polynomial import chebyshev  # Chebyshev polynomials (section 11)

ADIABATIC = "Revision/kohn_sham/results/adiabatic/adiabaticity.csv"
HISTORY = "Revision/kohn_sham/results/adiabatic/history.json"
THEORY = "Revision/kohn_sham/ks-theory.json"
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]  # the slices a4,0 of the record
TAGS = ["lam0", "lamp1", "lamm1", "lamp2", "lamm2"]  # 0, +-lambda_1, +-lambda_2
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order


def state_id(n, a4, tag="lam0"):
    """The record's name of a ground state, for example N688_lam0_a05."""
    return f"N{n}_{tag}_a{round(10 * a4):02d}"


with open(repository_file(ADIABATIC), newline="", encoding="utf-8") as handle:
    record = {row["id"]: row for row in csv.DictReader(handle)}
history = json.loads(repository_file(HISTORY).read_text(encoding="utf-8"))
theory = json.loads(repository_file(THEORY).read_text(encoding="utf-8"))
q_max = {(n, tag, a4): float(record[state_id(n, a4, tag)]["Q_max"])
         for n in (8, 136, 688) for tag in TAGS for a4 in SLICES}
largest = max(q_max.values())
n8_zero = all(q_max[(8, tag, a4)] == 0.0 for tag in TAGS for a4 in SLICES)
report("ground states in the record", len(record))
report("largest Q_max at the rate A = 1", f"{largest:.10f}")
say("The record's measure: " + theory["adiabaticity"]["measure"])
check(history["status"].startswith("PRESCRIBED BACKGROUND")
      and theory["adiabaticity"]["historyStatus"].startswith("PRESCRIBED BACKGROUND"),
      "the history a4 = A H x4 is labelled a PRESCRIBED BACKGROUND",
      record=f"{HISTORY}, key status, and {THEORY}, adiabaticity.historyStatus")
check(len(record) == 75 and largest <= 0.0935 and n8_zero,
      "75 states, Q_max <= 0.0935, and Q_max = 0 exactly for N = 8",
      record=f"{ADIABATIC}, column Q_max")
```

Out [2]:

```
RESULT ground states in the record = 75
RESULT largest Q_max at the rate A = 1 = 0.0934532439
The record's measure: Q_nm = A H |<n| d_a h |m>| / (eps_n - eps_m)^2 = A H |<n| d_a m>| /
    |eps_n - eps_m| for n occupied, m empty in the same sector; adiabatic iff max Q_nm <<
    1; leading-order transition probability ~ Q_nm^2
PASS the history a4 = A H x4 is labelled a PRESCRIBED BACKGROUND
     reproduces Revision/kohn_sham/results/adiabatic/history.json, key status, and
    Revision/kohn_sham/ks-theory.json, adiabaticity.historyStatus
PASS 75 states, Q_max <= 0.0935, and Q_max = 0 exactly for N = 8
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max
```

Text cell:

```
## 6. Q grows in proportion to the rate: the naive breakdown rates

By exact rule 1 of section 4, a history $A$ times faster than the record's has
$Q = A\,Q_{max}$. The naive criterion "breakdown when $Q$ reaches 1" therefore puts
the breakdown at the rate $A = 1/Q_{max}$. The next cell computes this rate for the
50 states with $Q_{max} > 0$ and draws it against the slice (left; the five
couplings differ so little that they are drawn as a thin band around the free
state $\lambda = 0$), and draws $Q = A\,Q_{max}$ against $A$ for the free state
$N = 688$ at the five slices (right): straight lines of slope 1 on logarithmic axes.
```

In [3]:

```
naive = {key: 1.0 / value for key, value in q_max.items() if value > 0.0}
weakest = min(naive, key=naive.get)  # the state that reaches Q = 1 first
rates = np.geomspace(0.1, 100.0, 200)  # rates A for the right panel
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), layout="constrained")
for colour, marker, n in ((PALETTE[0], "o", 136), (PALETTE[1], "s", 688)):
    band = np.array([[naive[(n, tag, a4)] for a4 in SLICES] for tag in TAGS])
    left.fill_between(SLICES, band.min(axis=0), band.max(axis=0), color=colour,
                      alpha=0.25, lw=0)  # the spread over the five couplings
    left.plot(SLICES, band[0], "-", color=colour, marker=marker, ms=8, lw=2.0,
              label=f"$N = {n}$")
left.axhline(1.0, color="0.3", lw=1.2, ls=":")
left.text(0.05, 1.12, "the record's rate $A = 1$", fontsize=8, color="0.3")
left.set_yscale("log")
left.set_ylim(0.7, 50.0)
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("naive breakdown rate $1/Q_{max}$")
left.set_title("Rate at which $Q$ would reach 1")
left.legend(fontsize=8)
for colour, a4 in zip(PALETTE, SLICES):
    right.plot(rates, rates * q_max[(688, "lam0", a4)], color=colour, lw=2.0,
               label=f"$a_{{4,0}} = {a4}$")
right.axhline(1.0, color="0.3", lw=1.2, ls=":")
right.axvline(1.0, color="0.3", lw=1.2, ls="--")
right.text(1.08, 0.012, "record: $A = 1$", fontsize=8, color="0.3")
right.set_xscale("log")
right.set_yscale("log")
right.set_xlabel("rate $A$ (e-folds of the extra times per unit time $1/H$)")
right.set_ylabel("$Q = A\\,Q_{max}$")
right.set_title("$N = 688$, $\\lambda = 0$: $Q$ grows like $A$")
right.legend(fontsize=8)
save_figure(fig, "naive_rates",
            "Left: the naive breakdown rate $1/Q_{max}$ (vertical axis, logarithmic) "
            "of the ground states $N = 136$ (blue circles) and $N = 688$ (orange "
            "squares) against the slice $a_{4,0}$ (horizontal axis); the lines are "
            "the free states, the shaded bands the spread over the five couplings, "
            "the dotted line the record's rate $A = 1$. Right: $Q = A\\,Q_{max}$ "
            "against the rate $A$ for the free state $N = 688$ at the five slices "
            "(both axes logarithmic); $Q$ reaches 1 (dotted) only above $A = 10$. "
```

```
            "The naive estimate says the record's history is at least ten times "
            "too slow to break the instantaneous picture.")
report(f"naive breakdown rate 1/Q_max of {state_id(weakest[0], weakest[2], weakest[1])}",
       f"{naive[weakest]:.4f}")
check(naive[weakest] > 10.0,
      "every recorded state keeps Q below 1 up to the rate A = 10",
      record=f"{ADIABATIC}, column Q_max")
```

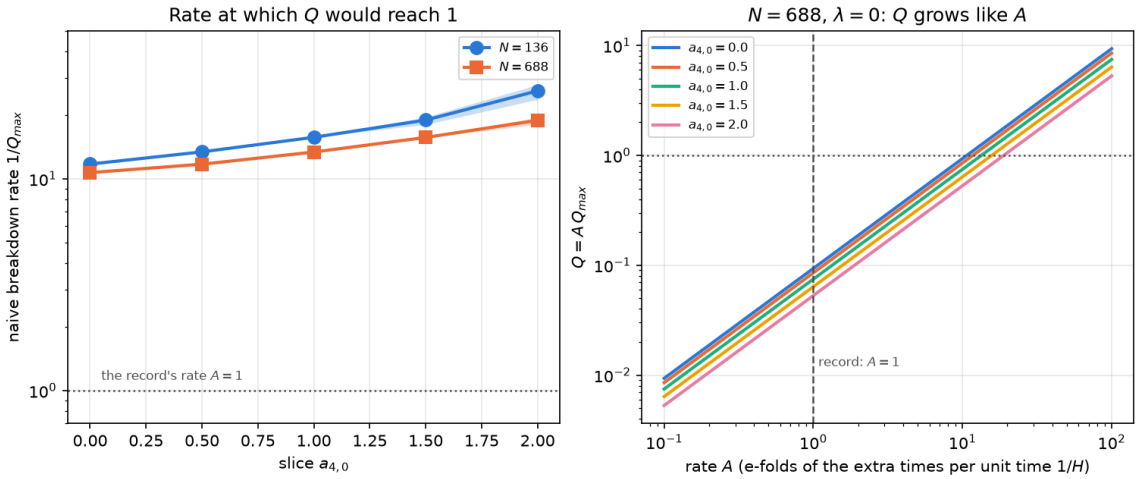

Figure 551: Left: the naive breakdown rate $1/Q_{max}$ (vertical axis, logarithmic) of the ground states $N = 136$ (blue circles) and $N = 688$ (orange squares) against the slice $a_{4,0}$ (horizontal axis); the lines are the free states, the shaded bands the spread over the five couplings, the dotted line the record's rate $A = 1$. Right: $Q = A\,Q_{max}$ against the rate A for the free state $N = 688$ at the five slices (both axes logarithmic); Q reaches 1 (dotted) only above $A = 10$. The naive estimate says the record's history is at least ten times too slow to break the instantaneous picture. (Notebook 22a, figure 1.)

Out [3]:

```
Figure 22a.1 saved as Revision/textbook/figures/22a_1_naive_rates.png
RESULT naive breakdown rate 1/Q_max of N688_lamm2_a00 = 10.7005
PASS every recorded state keeps Q below 1 up to the rate A = 10
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max
```

Text cell:

```
## 7. The solver's shooting method, vectorised

To go beyond the record we need the levels and orbitals ourselves. The next cell
reads the record's numerical parameters from `parameters.json` (the units, the tip
cutoff $L = 3$, the 900 RK4 steps) and defines five functions for the free block
($j = +1$, $M = 1$, $v = 0$, even brane parity):

- `shoot(eps, q)` integrates the block equation $a' = a - (q e^{-y} + \varepsilon)
  b$, $b' = (\varepsilon - q e^{-y})a - b$ (the real form of $h\varphi =
```

```
  \varepsilon\varphi$, ks-theory.json blockEquation.realForm) from the tip, where
  $b = 0$, to the brane with RK4, for whole arrays of energies and momenta at once,
  and follows the **Pruefer angle** $\theta = \mathrm{atan2}(b, a)$ continuously;
  the phase $\Phi = \theta(0)$ grows with $\varepsilon$, and the level with label
  $n$ is where $\Phi = n\pi$ (then $b(0) = 0$, the even brane condition);
- `levels(labels, q)` finds these levels by bisection to $10^{-13}$, the solver's
  root tolerance;
- `orbitals(eps, q)` returns the normalised orbitals on the fine grid of step ends
  and midpoints (midpoints by cubic Hermite interpolation, norm by Simpson's rule,
  as in the solver);
- `couplings(q, a, b)` returns the matrix of $\langle m|\partial_a h|n\rangle =
  -q\int e^{-y}(a_m a_n - b_m b_n)\,dy$ (line 8 of section 4);
- `per_unit_rate(eps, matrix)` returns $G_{nm} = |\langle n|\partial_a h|m\rangle|
  /(\varepsilon_n - \varepsilon_m)^2$, that is $Q/A$.
```

In [4]:

```
PARAMETERS = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                        .read_text(encoding="utf-8"))
L = PARAMETERS["physics"]["L_tipCutoff"]  # the tip cutoff y = -L
STEPS = PARAMETERS["numerics"]["rk4Steps"]  # RK4 steps from the tip to the brane
POINTS = 2 * STEPS + 1  # the fine grid: step ends and step midpoints
STEP = L / STEPS  # the length of one RK4 step in y
Y = -L * ((POINTS - 1 - np.arange(POINTS)) / (POINTS - 1))  # y from -L to 0
EW = np.exp(-Y)  # e^{-Hy} with H = 1, so kappa k = q e^{-y} with q = k e^{-a4,0}
SIMPSON = np.full(POINTS, 2.0 * STEP / 6.0)  # Simpson weights on the fine grid:
SIMPSON[1::2] = 4.0 * STEP / 6.0  # midpoints carry 4/6 of a step,
SIMPSON[0] = SIMPSON[-1] = STEP / 6.0  # the two ends 1/6
check(PARAMETERS["physics"]["H"] == 1.0 and PARAMETERS["physics"]["m"] == 1.0
      and L == 3.0 and STEPS == 900 and PARAMETERS["physics"]["dk"] == 0.25,
      "the record's units H = m = 1, tip cutoff L = 3, lattice step 0.25, 900 steps",
      record="Revision/kohn_sham/results/parameters.json, physics and numerics")


def shoot(eps, q, store=False):
    """Integrate the free block equation from the tip to the brane for the arrays
    eps (energies) and q (redshifted momenta) at once.  Returns the phase
    Phi = theta(0) and, if store is True, the list of (a, b) at the step ends."""
    a, b = np.ones_like(eps), np.zeros_like(eps)  # tip: b(-L) = 0, a(-L) = 1
    theta, raw = np.zeros_like(eps), np.zeros_like(eps)  # the angle, last atan2
    ends = [(a, b)] if store else None
    for i in range(STEPS):
        k0, k1, k2 = q * EW[2 * i], q * EW[2 * i + 1], q * EW[2 * i + 2]
        p1a, p1b = a - (k0 + eps) * b, (eps - k0) * a - b  # (a', b') at the start
        aa, bb = a + 0.5 * STEP * p1a, b + 0.5 * STEP * p1b
        p2a, p2b = aa - (k1 + eps) * bb, (eps - k1) * aa - bb  # at the midpoint
        aa, bb = a + 0.5 * STEP * p2a, b + 0.5 * STEP * p2b
        p3a, p3b = aa - (k1 + eps) * bb, (eps - k1) * aa - bb  # midpoint again
        aa, bb = a + STEP * p3a, b + STEP * p3b
        p4a, p4b = aa - (k2 + eps) * bb, (eps - k2) * aa - bb  # at the end
        a = a + STEP / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)  # the RK4 step
        b = b + STEP / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
        new = np.arctan2(b, a)  # the angle of the point (a, b), in (-pi, pi]
```

```
        turn = new - raw  # how far the angle turned in this step, brought back
        turn = np.where(turn > np.pi, turn - 2.0 * np.pi, turn)  # into (-pi, pi]
        turn = np.where(turn <= -np.pi, turn + 2.0 * np.pi, turn)
        theta, raw = theta + turn, new  # the angle, followed continuously
        if store:
            ends.append((a, b))
    return theta, ends


def levels(labels, q):
    """The level with label n (Phi = n pi) for every pair (label, q): bisection on
    [-40, 40], halving the bracket until it is shorter than 1e-13."""
    target = np.pi * np.asarray(labels, dtype=float)
    q = np.asarray(q, dtype=float)
    low, high = np.full(q.shape, -40.0), np.full(q.shape, 40.0)
    while np.max(high - low) > 1e-13:  # 50 halvings of the length 80
        middle = 0.5 * (low + high)
        above = shoot(middle, q)[0] > target  # Phi grows with eps
        low, high = np.where(above, low, middle), np.where(above, middle, high)
    return 0.5 * (low + high)


def orbitals(eps, q):
    """The normalised orbitals (a, b) on the fine grid, one row per (eps, q)."""
    ends = shoot(eps, q, store=True)[1]
    a_e = np.array([pair[0] for pair in ends]).T  # rows: orbitals; columns: ends
    b_e = np.array([pair[1] for pair in ends]).T
    kap = q[:, None] * EW[None, 0::2]  # kappa k at the step ends
    da = a_e - (kap + eps[:, None]) * b_e  # the derivatives at the step ends
    db = (eps[:, None] - kap) * a_e - b_e
    a, b = np.zeros((len(eps), POINTS)), np.zeros((len(eps), POINTS))
    a[:, 0::2], b[:, 0::2] = a_e, b_e
    # cubic Hermite midpoint: (left + right)/2 + STEP (left' - right')/8
    a[:, 1::2] = 0.5 * (a_e[:, :-1] + a_e[:, 1:]) + STEP / 8.0 * (da[:, :-1] - da[:, 1:])
    b[:, 1::2] = 0.5 * (b_e[:, :-1] + b_e[:, 1:]) + STEP / 8.0 * (db[:, :-1] - db[:, 1:])
    norm = np.sqrt(np.sum(SIMPSON * (a * a + b * b), axis=1))  # Simpson's rule
    return a / norm[:, None], b / norm[:, None]


def couplings(q, a, b):
    """The matrix <m| d_a h |n> = -q int e^{-y} (a_m a_n - b_m b_n) dy of the
    orbitals (rows of a and b) of one sector at the redshifted momentum q."""
    weight = SIMPSON * EW  # Simpson weights times e^{-y}
    return -q * ((a * weight) @ a.T - (b * weight) @ b.T)


def per_unit_rate(eps, matrix):
    """G[n, m] = |<n| d_a h |m>| / (eps_n - eps_m)^2 = Q / A (0 for n = m)."""
    gap = eps[:, None] - eps[None, :]
    safe = np.where(gap == 0.0, 1.0, gap)  # 1 on the diagonal: no division by 0
    return np.where(gap == 0.0, 0.0, np.abs(matrix) / safe ** 2)
```

Out [4]:

```
PASS the record's units H = m = 1, tip cutoff L = 3, lattice step 0.25, 900 steps
     reproduces Revision/kohn_sham/results/parameters.json, physics and numerics
```

Text cell:

```
## 8. The Fermi shells of the record, solved again

The record's largest $Q$ of the free states is always the jump from the band level
(label 0) to the first bulk level (label 1) in the Fermi shell: $n_2 = 4$
($k = 0.5$) for $N = 136$ and $n_2 = 11$ ($k = 0.829$) for $N = 688$. The next cell
solves the labels 0, 1, 2 of these sectors for the ten free states, **at the slice
0 with the redshifted momentum** $q = k e^{-a_{4,0}}$ (exact rule 2 of section 4),
compares the 30 levels with the record's level tables, and compares $Q$, the energy
difference and the matrix element with the record's columns `Q_max`,
`Q_max_delta_eps` and `Q_max_matrix_element`. Agreement at every slice is at the
same time a test of the rescaling identity.
```

In [5]:

```
LEVELS = "Revision/kohn_sham/results/ground/levels"
SHELL = {136: 4, 688: 11}  # the Fermi shell n2 of each N
fermi = [(n, a4) for n in (136, 688) for a4 in SLICES]  # the ten free states
q_fermi = np.array([0.25 * math.sqrt(SHELL[n]) * math.exp(-a4) for n, a4 in fermi])
eps_fermi = levels(np.tile([0, 1, 2], 10), np.repeat(q_fermi, 3)).reshape(10, 3)
worst_level, compared = 0.0, 0
for (n, a4), row in zip(fermi, eps_fermi):
    path = repository_file(f"{LEVELS}/{state_id(n, a4)}.csv")
    with open(path, newline="", encoding="utf-8") as handle:
        for entry in csv.DictReader(handle):
            sector = (int(entry["n2"]), int(entry["j"]), entry["parity"])
            if sector == (SHELL[n], 1, "even") and int(entry["label"]) <= 2:
                difference = float(entry["eps"]) - row[int(entry["label"])]
                worst_level = max(worst_level, abs(difference))
                compared += 1
a_f, b_f = orbitals(eps_fermi[:, :2].ravel(), np.repeat(q_fermi, 2))
worst_q, same_pair, q_free = 0.0, True, {}
say("   N  a4,0        q      eps_0      eps_1   Q (here)   Q_max (record)")
for i, (n, a4) in enumerate(fermi):
    matrix = couplings(q_fermi[i], a_f[2 * i:2 * i + 2], b_f[2 * i:2 * i + 2])
    gap = eps_fermi[i, 1] - eps_fermi[i, 0]
    q_free[(n, a4)] = abs(matrix[0, 1]) / gap ** 2  # Q at the rate A = 1
    row = record[state_id(n, a4)]
    for value, column in ((q_free[(n, a4)], "Q_max"), (gap, "Q_max_delta_eps"),
                          (abs(matrix[0, 1]), "Q_max_matrix_element")):
        worst_q = max(worst_q, abs(value / float(row[column]) - 1.0))
    pair = f"{SHELL[n]}:+1:even:0 -> {SHELL[n]}:+1:even:1"
    same_pair = same_pair and row["Q_max_pair"] == pair
    recorded = float(row["Q_max"])
    say(f"{n:4d}  {a4:4.1f}  {q_fermi[i]:7.4f}  {eps_fermi[i, 0]:9.6f}"
        f"  {eps_fermi[i, 1]:9.6f}  {q_free[(n, a4)]:.7f}  {recorded:.7f}")
report("levels compared with the record", compared)
report("largest |eps(here) - eps(record)|", f"{worst_level:.1e}")
```

```
report("largest relative difference of Q, delta eps, matrix element", f"{worst_q:.1e}")
check(compared == 30 and worst_level < 1e-12,
      "the 30 levels of the Fermi-shell sectors equal the record's within 1e-12",
      record=f"{LEVELS}/N136_lam0_a*.csv and N688_lam0_a*.csv, column eps")
check(worst_q < 1e-9,
      "Q_max, its energy difference and matrix element reproduced within 1e-9",
      record=f"{ADIABATIC}, columns Q_max, Q_max_delta_eps, Q_max_matrix_element; "
             "computed at the slice 0 with q = k e^(-a4,0), the rescaling identity "
             "of Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_rescaling_relation_and_band_monotone")
check(same_pair,
      "the recorded pair is band level -> first bulk level of the Fermi shell",
      record=f"{ADIABATIC}, column Q_max_pair")
```

Out [5]:

```
   N  a4,0        q      eps_0      eps_1   Q (here)   Q_max (record)
 136   0.0   0.5000   0.799646   2.852575  0.0851488  0.0851488
 136   0.5   0.3033   0.512555   2.400924  0.0745600  0.0745600
 136   1.0   0.1839   0.325768   2.071505  0.0636184  0.0636184
 136   1.5   0.1116   0.204636   1.820977  0.0527487  0.0527487
 136   2.0   0.0677   0.126766   1.651181  0.0383947  0.0383947
 688   0.0   0.8292   1.247113   3.500518  0.0934506  0.0934506
 688   0.5   0.5029   0.803752   2.858769  0.0852600  0.0852600
 688   1.0   0.3050   0.515229   2.405351  0.0746905  0.0746905
 688   1.5   0.1850   0.327505   2.074832  0.0637382  0.0637382
 688   2.0   0.1122   0.205760   1.823448  0.0528933  0.0528933
RESULT levels compared with the record = 30
RESULT largest |eps(here) - eps(record)| = 8.8e-14
RESULT largest relative difference of Q, delta eps, matrix element = 1.3e-13
PASS the 30 levels of the Fermi-shell sectors equal the record's within 1e-12
     reproduces Revision/kohn_sham/results/ground/levels/N136_lam0_a*.csv and
    N688_lam0_a*.csv, column eps
PASS Q_max, its energy difference and matrix element reproduced within 1e-9
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, columns Q_max,
    Q_max_delta_eps, Q_max_matrix_element; computed at the slice 0 with q = k e^(-a4,0),
    the rescaling identity of Revision/kohn_sham/reports/ks-rust-solver.json, check
    free_rescaling_relation_and_band_monotone
PASS the recorded pair is band level -> first bulk level of the Fermi shell
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max_pair
```

Text cell:

```
## 9. One curve for every slice and every shell

By exact rule 2 the free $Q/A$ of a sector depends on one number only, the
redshifted momentum $q$. The next cell solves the labels $-3$ to $4$ of the sector
at 61 values of $q$ from $0.02$ to $6$ (equally spaced on a logarithmic scale),
computes the table $G_{nm}(q)$ of all jumps, and locates the largest value of the
band jump $G_{01}$ on a finer grid around the best grid point.
```

In [6]:

```
Q_GRID = np.geomspace(0.02, 6.0, 61)  # redshifted momenta q (units of m)
CURVE = np.arange(-3, 5)  # the labels -3 ... 4 of the sector
BAND = 3  # the position of label 0 in CURVE
q_rows = np.repeat(Q_GRID, len(CURVE))  # one row per (q, label)
curve_eps = levels(np.tile(CURVE, len(Q_GRID)), q_rows)
curve_a, curve_b = orbitals(curve_eps, q_rows)
curve_eps = curve_eps.reshape(len(Q_GRID), len(CURVE))
G = np.array([per_unit_rate(curve_eps[i], couplings(
    Q_GRID[i], curve_a[8 * i:8 * i + 8], curve_b[8 * i:8 * i + 8]))
    for i in range(len(Q_GRID))])  # G[i, n, m]: Q per unit rate at Q_GRID[i]
del curve_a, curve_b  # the orbitals are no longer needed
g01 = G[:, BAND, BAND + 1]  # band level -> first bulk level
top = int(np.argmax(g01))
fine_q = np.linspace(Q_GRID[top - 1], Q_GRID[top + 1], 41)  # around the top
fine_eps = levels(np.tile([0, 1], 41), np.repeat(fine_q, 2)).reshape(41, 2)
fine_a, fine_b = orbitals(fine_eps.ravel(), np.repeat(fine_q, 2))
fine_g = np.array([per_unit_rate(fine_eps[i], couplings(
    fine_q[i], fine_a[2 * i:2 * i + 2], fine_b[2 * i:2 * i + 2]))[0, 1]
    for i in range(41)])
g_star, q_star = float(fine_g.max()), float(fine_q[int(np.argmax(fine_g))])
report("largest Q per unit rate of the band jump, G*", f"{g_star:.5f}")
report("at the redshifted momentum q*", f"{q_star:.3f}")
report("naive breakdown rate of the band level, 0.02 <= q <= 6, 1/G*",
       f"{1.0 / g_star:.3f}")
report("largest G of the jumps band -> bulk 2, 3, 4 on the grid",
       ", ".join(f"{G[:, BAND, BAND + m].max():.5f}" for m in (2, 3, 4)))
```

Out [6]:

```
RESULT largest Q per unit rate of the band jump, G* = 0.09979
RESULT at the redshifted momentum q* = 2.128
RESULT naive breakdown rate of the band level, 0.02 <= q <= 6, 1/G* = 10.021
RESULT largest G of the jumps band -> bulk 2, 3, 4 on the grid = 0.00640, 0.00249,
    0.00087
```

Text cell:

```
The next cell draws $G_{01}(q)$ and $G_{02}(q)$ and puts the record's $Q_{max}$ of
all 50 states with $Q_{max} > 0$ on the same axes, each at the $q = k e^{-a_{4,0}}$
of its Fermi shell. The free states fall exactly on the curve; the interacting ones
lie close to it. The grey band is the range of $q$ that the record covers. It also
checks that the top of the curve lies above every recorded value and below $0.1$,
and how much the interaction changes $Q_{max}$.
```

In [7]:

```
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(Q_GRID, g01, color="black", lw=2.0, label="$G_{01}$: band -> 1st bulk")
ax.plot(Q_GRID, G[:, BAND, BAND + 2], color="0.45", lw=1.5, ls="--",
        label="$G_{02}$: band -> 2nd bulk")
q_low, q_high = float(q_fermi.min()), float(q_fermi.max())
```

```
ax.axvspan(q_low, q_high, color="0.85", alpha=0.6, lw=0)  # the recorded range
for colour, marker, n in ((PALETTE[0], "o", 136), (PALETTE[1], "s", 688)):
    for tag in TAGS:
        xs = [0.25 * math.sqrt(SHELL[n]) * math.exp(-a4) for a4 in SLICES]
        ys = [q_max[(n, tag, a4)] for a4 in SLICES]
        ax.plot(xs, ys, marker, color=colour, ms=8, mfc="none", mew=1.5,
                label=f"record, $N = {n}$" if tag == "lam0" else None)
ax.plot([q_star], [g_star], "*", color=PALETTE[3], ms=16, mec="black",
        label=f"top: $G^* = {g_star:.4f}$ at $q = {q_star:.2f}$")
ax.set_xscale("log")
ax.set_xlabel("redshifted momentum $q = k\\,e^{-a_{4,0}}$ (units of $m$)")
ax.set_ylabel("$G = Q/A$ (pure number)")
ax.set_title("Every slice and every shell on one curve ($\\lambda = 0$)")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "one_curve",
            "The adiabaticity measure per unit rate, $G = Q/A$ (vertical axis, pure "
            "number), of the jump from the band level to the first (black) and "
            "second (grey, dashed) bulk level of a free sector, against the "
            "redshifted momentum $q = k e^{-a_{4,0}}$ (horizontal axis, "
            "logarithmic, units of $m$). Open symbols: the recorded $Q_{max}$ of "
            "the 50 states with $N = 136$ (circles) and $N = 688$ (squares), each "
            "at the $q$ of its Fermi shell; the five couplings overlap. The grey "
            "band is the range of the record; the star is the top of the curve, "
            "the largest $Q/A$ of the jump to the first bulk level for every "
            "shell and slice with $0.02 \\le q \\le 6$ (all states of the record).")
spread = max(abs(q_max[(n, tag, a4)] / q_max[(n, "lam0", a4)] - 1.0)
             for n in (136, 688) for tag in TAGS for a4 in SLICES)
report("largest relative change of Q_max by the interaction", f"{spread:.4f}")
check(largest < g_star < 0.1,
      "the top of the curve lies above every recorded Q_max and below 0.1")
check(spread < 0.15, "the interaction changes Q_max by less than 15 percent",
      record=f"{ADIABATIC}, column Q_max, all couplings")
```

Out [7]:

```
Figure 22a.2 saved as Revision/textbook/figures/22a_2_one_curve.png
RESULT largest relative change of Q_max by the interaction = 0.0980
PASS the top of the curve lies above every recorded Q_max and below 0.1
PASS the interaction changes Q_max by less than 15 percent
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max, all
    couplings
```

Text cell:

```
## 10. Jumps across the gap into the negative branch

The record's $Q$ counts only jumps between levels of positive energy: its filling
convention treats the negative branch as the normal-ordered sea, which is not
populated. But the moving background couples the two branches too, through the
same matrix elements. In ordinary Dirac theory a jump from a filled negative level
to an empty positive one is the creation of a particle-antiparticle pair OF THE
FIELD, inside one universe; whether that reading holds in the 4+4 quantisation with
```

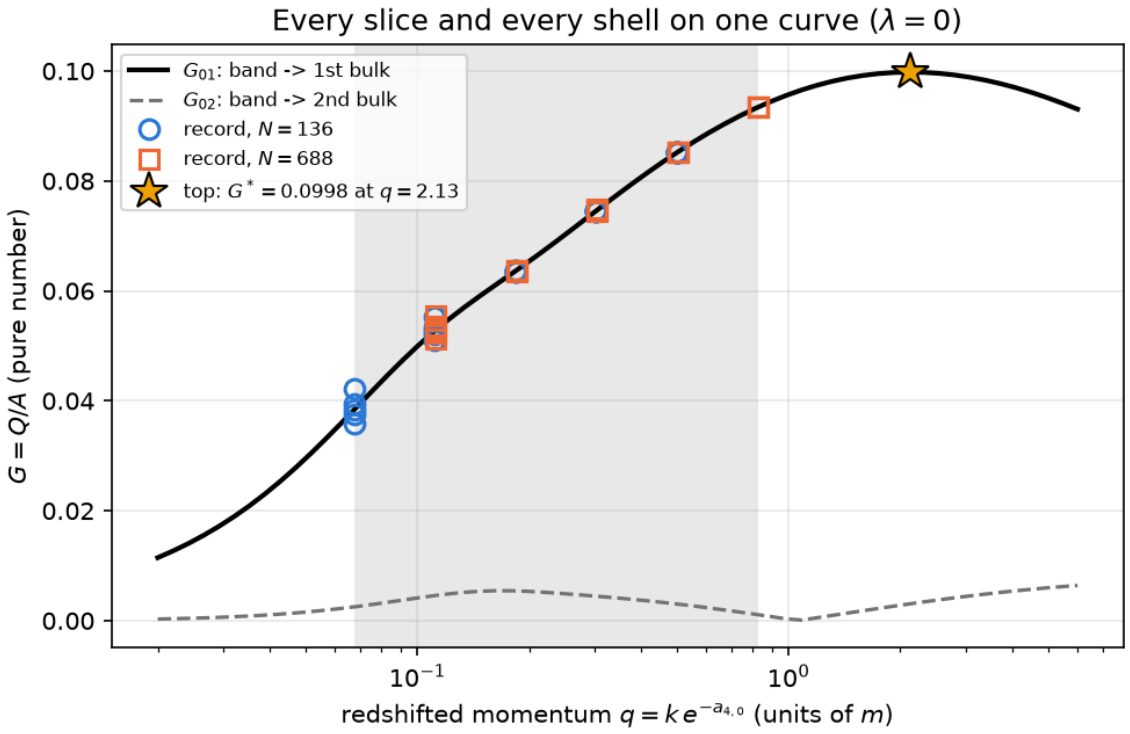

Figure 552: The adiabaticity measure per unit rate, $G = Q/A$ (vertical axis, pure number), of the jump from the band level to the first (black) and second (grey, dashed) bulk level of a free sector, against the redshifted momentum $q = ke^{-a_{4,0}}$ (horizontal axis, logarithmic, units of m). Open symbols: the recorded Q_{max} of the 50 states with $N = 136$ (circles) and $N = 688$ (squares), each at the q of its Fermi shell; the five couplings overlap. The grey band is the range of the record; the star is the top of the curve, the largest Q/A of the jump to the first bulk level for every shell and slice with $0.02 \leq q \leq 6$ (all states of the record). (Notebook 22a, figure 2.)

its Krein metric is OPEN, and it has nothing to do with the creation of universes. The next cell draws, from the same table $G_{nm}(q)$, the jumps from the negative levels -1 and -2 to the bulk levels and to the band level, next to the band jump of section 9, and finds the largest jump from a negative to a positive bulk level.

In [8]:

```
def g_pair(n, m):
    """G of the jump from label n to label m along Q_GRID."""
    return G[:, BAND + n, BAND + m]


sea = np.max([g_pair(n, m) for n in (-3, -2, -1) for m in (1, 2, 3, 4)], axis=0)
sea_top = int(np.argmax(sea))
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(Q_GRID, g01, color="black", lw=2.0, label="band 0 -> bulk 1 (record's Q)")
for colour, style, (n, m) in zip(PALETTE, ("-", "--", "-.", ":"),
                                 ((-1, 1), (-1, 2), (-2, 1), (-1, 0))):
    kind = "band" if m == 0 else "bulk"
    ax.plot(Q_GRID, g_pair(n, m), color=colour, lw=2.0, ls=style,
            label=f"negative {n} -> {kind} {m}")
ax.set_xscale("log")
ax.set_xlabel("redshifted momentum $q = k\\,e^{-a_{4,0}}$ (units of $m$)")
ax.set_ylabel("$G = Q/A$ (pure number)")
ax.set_title("Jumps across the gap between the two branches ($\\lambda = 0$)")
ax.legend(fontsize=8)
save_figure(fig, "across_the_gap",
            "The adiabaticity measure per unit rate $G = Q/A$ (vertical axis) of "
            "jumps from the negative levels $-1$ and $-2$ of a free sector to the "
            "bulk levels 1 and 2 and to the band level 0 (coloured), compared "
            "with the band jump of the record (black), against the redshifted "
            "momentum $q$ (horizontal axis, logarithmic, units of $m$). The jumps "
            "across the gap stay below 0.05 per unit rate; the strongest of them "
            "($-1 \\to$ bulk 1) peaks near $q = 0.11$ at 0.049 and exceeds the "
            "band jump only below $q$ of about 0.1; their reading as pair "
            "creation of the field is OPEN.")
report("largest G of a jump negative level -> bulk level", f"{sea[sea_top]:.5f}")
report("at the redshifted momentum", f"{Q_GRID[sea_top]:.4f}")
check(sea[sea_top] < 0.5 * g_star,
      "every jump across the gap is weaker than half the top of the band jump")
```

Out [8]:

```
Figure 22a.3 saved as Revision/textbook/figures/22a_3_across_the_gap.png
RESULT largest G of a jump negative level -> bulk level = 0.04887
RESULT at the redshifted momentum = 0.1107
PASS every jump across the gap is weaker than half the top of the band jump
```

Text cell:

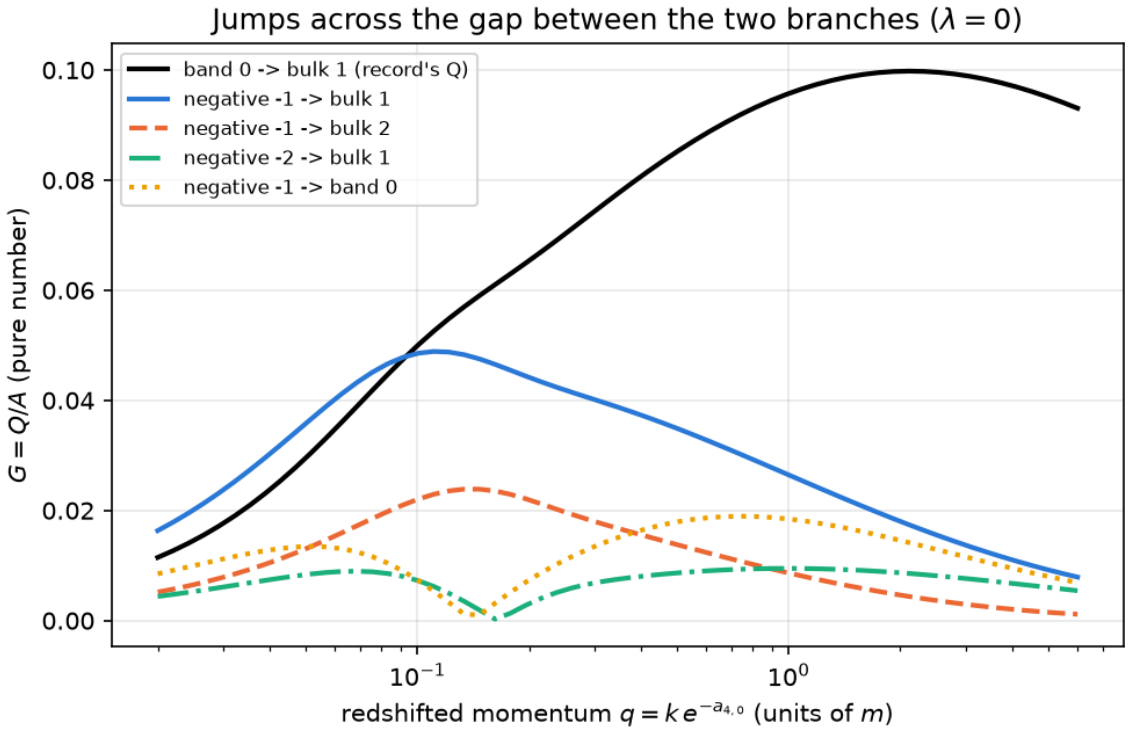

Figure 553: The adiabaticity measure per unit rate $G = Q/A$ (vertical axis) of jumps from the negative levels -1 and -2 of a free sector to the bulk levels 1 and 2 and to the band level 0 (coloured), compared with the band jump of the record (black), against the redshifted momentum q (horizontal axis, logarithmic, units of m). The jumps across the gap stay below 0.05 per unit rate; the strongest of them ($-1 \to$ bulk 1) peaks near $q = 0.11$ at 0.049 and exceeds the band jump only below q of about 0.1; their reading as pair creation of the field is OPEN. (Notebook 22a, figure 3.)

11. The Fermi-shell sector along the recorded span

For the time evolution we need the levels and the matrix K_{mn} not at a few slices but at every instant. The next cell takes the Fermi-shell sector of $N = 688$ ($n_2 = 11$, $k = 0.829$, 96 particles in its band level), solves 20 labels (-9 to 10) at 56 Chebyshev nodes of the span $a_{4,0} = 0$ to 2, and builds the interpolating Chebyshev polynomials of every level and every matrix element: the coefficients are $c_k = (2/N)\sum_j f(s_j)\cos(k\theta_j)$, with c_0 halved. It then checks the interpolation against a direct solution at five other points, and checks that the interpolated Q at the five slices equals the record's.

In [9]:

```
K11 = 0.25 * math.sqrt(11.0)  # the Fermi-shell momentum of N = 688 (units of m)
END = 2.0  # the recorded span of slices, a4,0 = 0 ... 2
NODES = 56  # Chebyshev nodes on the span
BASIS = np.arange(-9, 11)  # 20 labels: nine negative, the band level, ten bulk
SIZE, ZERO = len(BASIS), 9  # ZERO: the position of label 0 (the band level)
angles = np.pi * (np.arange(NODES) + 0.5) / NODES  # theta_j of the nodes
nodes_s = 0.5 * END * (1.0 + np.cos(angles))  # the nodes as slices a4,0
q_nodes = np.repeat(K11 * np.exp(-nodes_s), SIZE)  # one row per (node, label)
node_eps = levels(np.tile(BASIS, NODES), q_nodes)
node_a, node_b = orbitals(node_eps, q_nodes)
node_eps = node_eps.reshape(NODES, SIZE)
node_m = np.array([couplings(K11 * math.exp(-s), node_a[SIZE * i:SIZE * (i + 1)],
                             node_b[SIZE * i:SIZE * (i + 1)])
                   for i, s in enumerate(nodes_s)])
del node_a, node_b  # the orbitals are no longer needed


def chebyshev_coefficients(values):
    """c_k = (2/NODES) sum_j values_j cos(k theta_j), c_0 halved: the polynomial
    through the values at the nodes (one column per function)."""
    table = np.cos(np.outer(np.arange(NODES), angles))  # cos(k theta_j)
    coefficients = 2.0 / NODES * table @ values.reshape(NODES, -1)
    coefficients[0] *= 0.5
    return coefficients


FIT_EPS = chebyshev_coefficients(node_eps)  # 20 levels
FIT_M = chebyshev_coefficients(node_m)  # 400 matrix elements


def interpolated(s):
    """Levels (one row per point s) and coupling matrices from the fits."""
    x = 2.0 * np.asarray(s, dtype=float) / END - 1.0  # a4,0 = 0 ... 2 -> -1 ... 1
    eps = chebyshev.chebval(x, FIT_EPS).T
    matrix = chebyshev.chebval(x, FIT_M).T.reshape(len(x), SIZE, SIZE)
    return eps, matrix


test_s = np.array([0.03, 0.25, 0.77, 1.31, 1.97])  # five points between the nodes
```

```
test_q = np.repeat(K11 * np.exp(-test_s), SIZE)
test_eps = levels(np.tile(BASIS, 5), test_q)
test_a, test_b = orbitals(test_eps, test_q)
fit_eps, fit_m = interpolated(test_s)
error_eps = float(np.max(np.abs(fit_eps - test_eps.reshape(5, SIZE))))
error_m = max(float(np.max(np.abs(fit_m[i] - couplings(
    K11 * math.exp(-s), test_a[SIZE * i:SIZE * (i + 1)],
    test_b[SIZE * i:SIZE * (i + 1)])))) for i, s in enumerate(test_s))
slice_eps, slice_m = interpolated(SLICES)
slice_q = [abs(slice_m[i, ZERO, ZERO + 1]) / (slice_eps[i, ZERO + 1]
           - slice_eps[i, ZERO]) ** 2 for i in range(5)]
worst_slice = max(abs(slice_q[i] / q_max[(688, "lam0", a4)] - 1.0)
                  for i, a4 in enumerate(SLICES))
report("largest interpolation error of a level", f"{error_eps:.1e}")
report("largest interpolation error of a matrix element", f"{error_m:.1e}")
report("largest relative difference of the interpolated Q to the record",
       f"{worst_slice:.1e}")
check(error_eps < 1e-9 and error_m < 1e-7,
      "the Chebyshev fits reproduce levels within 1e-9 and couplings within 1e-7")
check(worst_slice < 1e-8, "the interpolated Q at the five slices equals the record's",
      record=f"{ADIABATIC}, column Q_max of N688_lam0_a00 ... a20")
```

Out [9]:

```
RESULT largest interpolation error of a level = 7.3e-11
RESULT largest interpolation error of a matrix element = 6.4e-09
RESULT largest relative difference of the interpolated Q to the record = 1.4e-13
PASS the Chebyshev fits reproduce levels within 1e-9 and couplings within 1e-7
PASS the interpolated Q at the five slices equals the record's
     reproduces Revision/kohn_sham/results/adiabatic/adiabaticity.csv, column Q_max of
    N688_lam0_a00 ... a20
```

Text cell:

```
The next cell draws the 20 levels of the sector along the span (left) and the
numbers $|K_{m0}|$ that drive the jumps out of the band level (right): $|K_{m0}|$ is
how far the band orbital turns towards level $m$ per e-fold of the extra times.
```

In [10]:

```
s_plot = np.linspace(0.0, END, 201)
eps_plot, m_plot = interpolated(s_plot)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.4), layout="constrained")
for i, label in enumerate(BASIS):
    if label == 0:
        left.plot(s_plot, eps_plot[:, i], color=PALETTE[3], lw=3.0,
                  label="band level 0 (occupied)")
    else:
        colour = PALETTE[0] if label > 0 else PALETTE[1]
        left.plot(s_plot, eps_plot[:, i], color=colour, lw=1.0)
left.plot([], [], color=PALETTE[0], lw=1.0, label="bulk levels 1 ... 10")
```

```
left.plot([], [], color=PALETTE[1], lw=1.0, label="negative levels -1 ... -9")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("level $\\varepsilon$ (units of $m$)")
left.set_title("Sector $n_2 = 11$, $j = +1$, even, of $N = 688$")
left.legend(fontsize=8, loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=3)
for colour, style, label in zip(PALETTE, ("-", "--", "-.", ":"), (1, 2, -1, -2)):
    m = ZERO + label
    turn = np.abs(m_plot[:, m, ZERO]) / np.abs(eps_plot[:, ZERO] - eps_plot[:, m])
    right.plot(s_plot, turn, color=colour, ls=style, lw=2.0, label=f"$m = {label}$")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$|K_{m0}|$ (per e-fold)")
right.set_title("How fast the band orbital turns towards level $m$")
right.legend(fontsize=8)
save_figure(fig, "sector_levels",
            "Left: the 20 instantaneous levels (vertical axis, units of $m$) of the "
            "free Fermi-shell sector of $N = 688$ against the slice $a_{4,0}$ "
            "(horizontal axis): the occupied band level (thick) falls as the 3-"
            "momentum redshifts, the bulk levels above it and the negative levels "
            "below it move towards their values at $k = 0$. Right: $|K_{m0}|$, how "
            "far the band orbital turns towards level $m$ per e-fold (vertical "
            "axis), for $m = 1, 2, -1, -2$; all are below 0.25, so the band orbital "
            "changes slowly.")
```

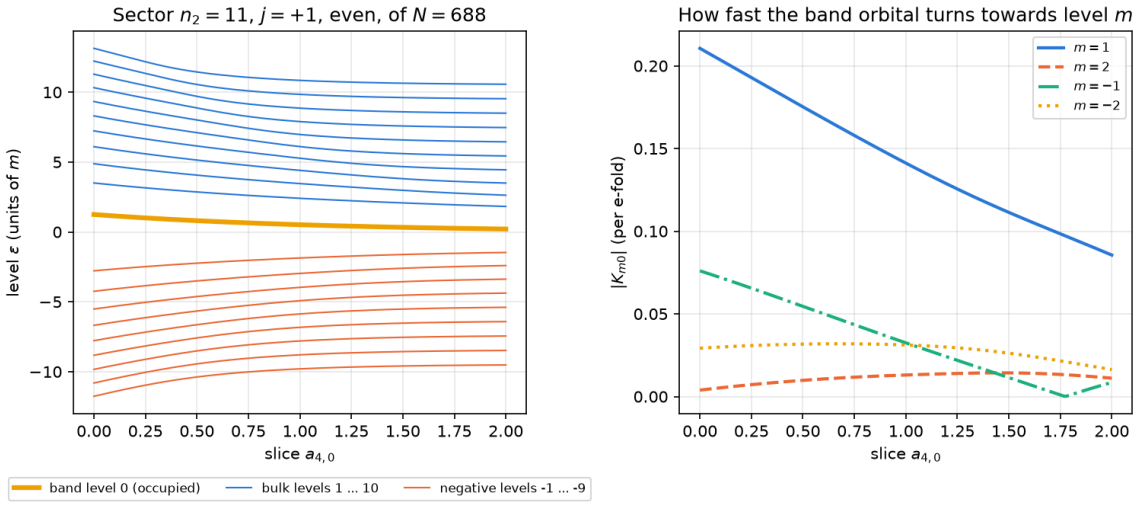

Figure 554: Left: the 20 instantaneous levels (vertical axis, units of m) of the free Fermi-shell sector of $N = 688$ against the slice $a_{4,0}$ (horizontal axis): the occupied band level (thick) falls as the 3-momentum redshifts, the bulk levels above it and the negative levels below it move towards their values at $k = 0$. Right: $|K_{m0}|$, how far the band orbital turns towards level m per e-fold (vertical axis), for $m = 1, 2, -1, -2$; all are below 0.25, so the band orbital changes slowly. (Notebook 22a, figure 4.)

Out [10]:

```
Figure 22a.4 saved as Revision/textbook/figures/22a_4_sector_levels.png
```

Text cell:

```
## 12. The exact time evolution in the basis of instantaneous levels

Now we integrate line 5 of section 4, $\dot c_m = -i\varepsilon_m c_m - \dot a_4
\sum_n K_{mn}c_n$, in the basis of the 20 levels. We write the time as $x_4 = T u$
with $u$ running from 0 to 1, so that $dc/du = -iT\varepsilon c - (da_4/du)\,K c$;
$T$ is the duration of the passage. The next cell defines:

- `frame(s)`: the levels and the matrix $K_{mn} = \langle m|\partial_a h|n\rangle
  /(\varepsilon_n - \varepsilon_m)$ ($K_{nn} = 0$) at the slices $s$, from the
  Chebyshev fits;
- `passage(...)`: RK4 for $dc/du$ on a fine grid of 4000 steps of $u$ (it may use
  every second, fourth, ... point for fewer steps);
- `steps_needed(T)`: the smallest number of steps for which the fastest phase
  turns by at most 0.2 radian per step;
- the two histories: a **smooth passage** $a_4 = 2[u - \sin(2\pi u)/(2\pi)]$, whose
  rate $da_4/dx_4 = (4/T)\sin^2(\pi u)$ starts at zero, rises to its peak $A = 4/T$
  in the middle and falls back to zero; and the **constant rate** $a_4 = 2u$ of the
  record's kind, with $A = 2/T$.
```

In [11]:

```
U_STEPS = 4000  # the finest number of RK4 steps of one passage, u = 0 ... 1
U = np.linspace(0.0, 1.0, 2 * U_STEPS + 1)  # the fine grid: step ends, midpoints
CHOICES = [200, 400, 500, 1000, 2000, 4000]  # step numbers that divide U_STEPS
OFF = ~np.eye(SIZE, dtype=bool)  # every pair n != m
SPAN = float(np.ptp(node_eps))  # the largest energy difference of the basis


def frame(s):
    """Levels e[i, n] and the matrix k[i, m, n] = <m| d_a n> at the slices s[i]."""
    e, matrix = interpolated(s)
    gap = e[:, None, :] - e[:, :, None]  # gap[i, m, n] = eps_n - eps_m
    k = np.zeros_like(matrix)
    k[:, OFF] = matrix[:, OFF] / gap[:, OFF]  # line 6 of section 4; K_nn = 0
    return e, k


def steps_needed(duration):
    """The smallest step number of CHOICES with duration * SPAN / steps <= 0.2."""
    for steps in CHOICES:
        if duration * SPAN / steps <= 0.2:
            return steps
    return CHOICES[-1]


def passage(duration, e, k, rate, start, steps, keep=0):
    """RK4 for dc/du = -i duration e(u) c - rate(u) k(u) c from u = 0 to 1.
    e, k and rate are given on the fine grid U. Returns the final amplitudes and,
    if keep > 0, the list of the amplitudes after every keep steps."""
    stride = U_STEPS // steps  # fine-grid points per half step
    h = 1.0 / steps
    c = start.astype(complex)
    kept = [c]
```

```
    def slope(i, c):
        return -1j * duration * e[i] * c - rate[i] * (k[i] @ c)

    for step in range(steps):
        i = 2 * stride * step  # the start of this step on the fine grid
        s1 = slope(i, c)
        s2 = slope(i + stride, c + 0.5 * h * s1)
        s3 = slope(i + stride, c + 0.5 * h * s2)
        s4 = slope(i + 2 * stride, c + h * s3)
        c = c + h / 6.0 * (s1 + 2.0 * s2 + 2.0 * s3 + s4)
        if keep and (step + 1) % keep == 0:
            kept.append(c)
    return c, kept


S_SMOOTH = END * (U - np.sin(2.0 * np.pi * U) / (2.0 * np.pi))  # a4 along it
RATE_SMOOTH = 2.0 * END * np.sin(np.pi * U) ** 2  # its derivative d a4 / d u
E_SMOOTH, K_SMOOTH = frame(S_SMOOTH)
S_LINEAR = END * U  # the constant rate: a4 = END u
RATE_LINEAR = np.full_like(U, END)
E_LINEAR, K_LINEAR = frame(S_LINEAR)
START = np.zeros(SIZE, dtype=complex)
START[ZERO] = 1.0  # the particle starts in the band level
report("largest energy difference of the basis", f"{SPAN:.4f}", "m")
```

Out [11]:

```
RESULT largest energy difference of the basis = 24.8912 m
```

Text cell:

```
## 13. A smooth passage at every rate

The smooth passage starts and ends with the rate zero, so there is no dressing at
the ends: whatever probability $P = 1 - |c_0|^2$ has left the band level at the end
is a real transition. The next cell runs the passage for 31 peak rates $A$ from
$0.25$ to $251$ (ten per factor 10), checks that the total probability stays 1,
computes the sudden limit twice (in the basis, with $T = 0$, and directly from the
overlap of the two band orbitals at $a_{4,0} = 0$ and $2$), and computes the
first-order estimate $P^{(1)} = \sum_m |d_m|^2$ with $d_m = -\int_0^1 (da_4/du)
K_{m0}\,e^{iT\int_0^u(\varepsilon_m - \varepsilon_0)\,du'}\,du$ (line 5 with $c_0 =
1$ on the right side, written for $d_m = c_m e^{iT\int\varepsilon_m du}$). Finally
it finds the rate $A_{1/2}$ at which $P$ reaches half the sudden value.
```

In [12]:

```
RATES = 10.0 ** np.linspace(-0.6, 2.4, 31)  # peak rates A from 0.25 to 251
excited, norm_error = [], 0.0
for rate in RATES:
    duration = 2.0 * END / rate  # T = 4/A: the peak rate is A
```

```
    c, _ = passage(duration, E_SMOOTH, K_SMOOTH, RATE_SMOOTH, START,
                   steps_needed(duration))
    excited.append(1.0 - abs(c[ZERO]) ** 2)  # probability to have left level 0
    norm_error = max(norm_error, abs(float(np.vdot(c, c).real) - 1.0))
excited = np.array(excited)
c, _ = passage(0.0, E_SMOOTH, K_SMOOTH, RATE_SMOOTH, START, 400)  # infinitely fast
sudden_basis = 1.0 - abs(c[ZERO]) ** 2
q_ends = np.array([K11, K11 * math.exp(-END)])  # the band level at a4,0 = 0 and 2
ends_a, ends_b = orbitals(levels([0, 0], q_ends), q_ends)
overlap = float(np.sum(SIMPSON * (ends_a[0] * ends_a[1] + ends_b[0] * ends_b[1])))
sudden = 1.0 - overlap ** 2  # the sudden limit from the two orbitals directly
du = 1.0 / (2 * U_STEPS)  # the spacing of the fine grid U
gaps = E_SMOOTH - E_SMOOTH[:, [ZERO]]  # eps_m - eps_0 along the passage
phase = np.vstack([np.zeros((1, SIZE)), np.cumsum(0.5 * (gaps[1:] + gaps[:-1]) * du,
                                                  axis=0)])  # trapezoid rule
weights = np.full(len(U), 2.0 * du / 3.0)  # Simpson weights on U
weights[1::2] = 4.0 * du / 3.0
weights[0] = weights[-1] = du / 3.0
drive = (weights * RATE_SMOOTH)[:, None] * K_SMOOTH[:, :, ZERO]  # da4/du K_m0
first = np.array([float(np.sum(np.abs(np.sum(drive * np.exp(
    1j * (2.0 * END / rate) * phase), axis=0)) ** 2)) for rate in RATES])
above = int(np.argmax(excited >= 0.5 * sudden))  # first rate past half the limit
x0, x1 = math.log(RATES[above - 1]), math.log(RATES[above])
y0, y1 = excited[above - 1] - 0.5 * sudden, excited[above] - 0.5 * sudden
rate_half = math.exp(x0 - y0 * (x1 - x0) / (y1 - y0))  # straight line in log A
report("largest |total probability - 1|", f"{norm_error:.1e}")
report("sudden limit: overlap / basis", f"{sudden:.6f} / {sudden_basis:.6f}")
report("P at A = 0.25, 1, 2.51, 251",
       ", ".join(f"{excited[i]:.3e}" for i in (0, 6, 10, 30)))
report("rate at which P reaches half the sudden limit, A_1/2", f"{rate_half:.3f}")
check(norm_error < 1e-9, "the evolution keeps the total probability 1 within 1e-9")
check(abs(sudden_basis / sudden - 1.0) < 1e-3,
      "the sudden limit of the 20-level basis equals the direct overlap within 0.1%")
check(abs(excited[-1] / sudden - 1.0) < 1e-3 and excited[0] < 1e-6
      and bool(np.all(excited <= sudden * (1.0 + 1e-3))),
      "P rises from below 1e-6 at A = 0.25 to the sudden limit at A = 251, never above")
```

Out [12]:

```
RESULT largest |total probability - 1| = 1.2e-11
RESULT sudden limit: overlap / basis = 0.074906 / 0.074902
RESULT P at A = 0.25, 1, 2.51, 251 = 1.036e-07, 1.380e-02, 5.895e-02, 7.490e-02
RESULT rate at which P reaches half the sudden limit, A_1/2 = 1.508
PASS the evolution keeps the total probability 1 within 1e-9
PASS the sudden limit of the 20-level basis equals the direct overlap within 0.1%
PASS P rises from below 1e-6 at A = 0.25 to the sudden limit at A = 251, never above
```

Text cell:

```
The next cell draws $P$ against the peak rate $A$, with the first-order estimate,
the sudden limit and three rates: the record's $A = 1$, the exact $A_{1/2}$ and the
naive $1/Q_{max}$ of section 6.
```

In [13]:

```
fig, ax = plt.subplots(figsize=(7.5, 4.6))
ax.plot(RATES, excited, "o-", color=PALETTE[0], ms=5, lw=2.0,
        label="exact evolution (20 levels)")
ax.plot(RATES, first, "--", color=PALETTE[1], lw=2.0, label="first order")
ax.axhline(sudden, color="0.3", ls=":", lw=1.5, label="sudden limit")
for value, style, text, height in ((1.0, "--", "record", 2e-7),
                                   (rate_half, "-", "$A_{1/2}$", 2e-5),
                                   (naive[weakest], "-.", "$1/Q_{max}$", 2e-7)):
    ax.axvline(value, color="0.45", ls=style, lw=1.2)
    ax.text(value * 1.06, height, text, fontsize=9, color="0.3")
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_ylim(1e-8, 0.3)
ax.set_xlabel("peak rate $A$ (e-folds per unit time $1/H$)")
ax.set_ylabel("$P$: probability to have left the band level")
ax.set_title("Smooth passage $a_{4,0} = 0 \\to 2$, free Fermi shell of $N = 688$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "smooth_passage",
            "The probability $P$ that the particle of the band level has left it "
            "at the end of a smooth passage from $a_{4,0} = 0$ to $2$ (vertical "
            "axis, logarithmic), against the peak rate $A$ (horizontal axis, "
            "logarithmic): exact evolution in the basis of 20 instantaneous levels "
            "(blue), the first-order estimate (orange, dashed) and the sudden limit "
            "(dotted). Vertical lines: the record's rate $A = 1$, the rate "
            "$A_{1/2}$ at which $P$ reaches half the sudden limit, and the naive "
            "breakdown rate $1/Q_{max}$. The gas stops following its level near "
            "$A = 1$, about seven times below the naive estimate ($A_{1/2} = 1.5$ "
            "against $1/Q_{max} = 10.7$), but $P$ never exceeds the sudden limit "
            "of about 7.5 percent.")
```

Out [13]:

```
Figure 22a.5 saved as Revision/textbook/figures/22a_5_smooth_passage.png
```

Text cell:

```
## 14. A constant rate: the dressing that Q measures

Section 4 showed that at a constant rate the particle carries the dressing
$\alpha_m = iAK_{m0}/(\varepsilon_m - \varepsilon_0)$, of total probability
$\sum_m Q_{0m}^2$. The next cell tests this with the exact evolution at the
constant rates $A = 0.3$, $1$ and $3$ over the span $a_{4,0} = 0$ to $2$: it starts
in the dressed state (level 0 plus the admixtures $\alpha_m$, normalised) and
compares the exact $P(a_4)$ with $\sum_m Q_{0m}(a_4)^2$ at 101 points. At $A = 1$
it also starts in the bare level 0, to show the extra transition caused by
switching the motion on abruptly.
```

In [14]:

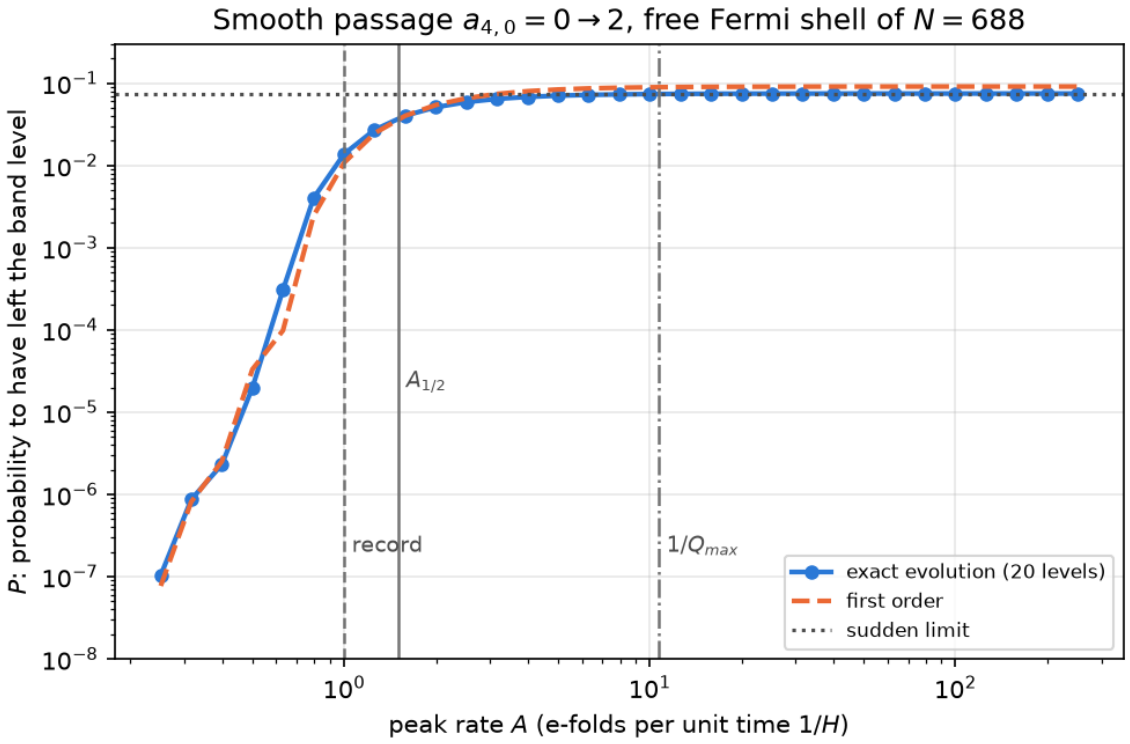

Figure 555: The probability P that the particle of the band level has left it at the end of a smooth passage from $a_{4,0} = 0$ to 2 (vertical axis, logarithmic), against the peak rate A (horizontal axis, logarithmic): exact evolution in the basis of 20 instantaneous levels (blue), the first-order estimate (orange, dashed) and the sudden limit (dotted). Vertical lines: the record's rate $A = 1$, the rate $A_{1/2}$ at which P reaches half the sudden limit, and the naive breakdown rate $1/Q_{max}$. The gas stops following its level near $A = 1$, about seven times below the naive estimate ($A_{1/2} = 1.5$ against $1/Q_{max} = 10.7$), but P never exceeds the sudden limit of about 7.5 percent. (Notebook 22a, figure 5.)

```
MARKS = np.arange(0, 2 * U_STEPS + 1, 2 * U_STEPS // 100)  # 101 points of U


def dressed(rate):
    """Level 0 with the first-order admixtures i A K_m0 / (eps_m - eps_0)."""
    e, k = E_LINEAR[0], K_LINEAR[0]
    c = START.copy()
    for m in range(SIZE):
        if m != ZERO:
            c[m] = 1j * rate * k[m, ZERO] / (e[m] - e[ZERO])
    return c / np.linalg.norm(c)


def dressing(rate):
    """The first-order estimate sum_m Q_0m^2 at the 101 marks."""
    e, k = E_LINEAR[MARKS], K_LINEAR[MARKS]
    total = np.zeros(len(MARKS))
    for m in range(SIZE):
        if m != ZERO:  # Q_0m = A |K_m0| / |eps_m - eps_0|
            total += (rate * k[:, m, ZERO] / (e[:, m] - e[:, ZERO])) ** 2
    return total


runs = {}
for rate in (0.3, 1.0, 3.0):
    duration = END / rate  # T = 2/A for the constant rate A
    steps = steps_needed(duration)
    _, kept = passage(duration, E_LINEAR, K_LINEAR, RATE_LINEAR, dressed(rate),
                      steps, keep=steps // 100)
    runs[rate] = np.array([1.0 - abs(c[ZERO]) ** 2 for c in kept])
steps = steps_needed(END)
_, kept = passage(END, E_LINEAR, K_LINEAR, RATE_LINEAR, START, steps, keep=steps // 100)
bare = np.array([1.0 - abs(c[ZERO]) ** 2 for c in kept])
s_marks = S_LINEAR[MARKS]
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), layout="constrained")
for colour, rate in zip(PALETTE, (0.3, 1.0, 3.0)):
    left.plot(s_marks, runs[rate], color=colour, lw=2.0, label=f"exact, $A = {rate}$")
    left.plot(s_marks, dressing(rate), color=colour, lw=1.5, ls="--")
left.plot([], [], color="0.4", ls="--", label="first order $\\sum_m Q_{0m}^2$")
left.set_yscale("log")
left.set_ylim(3e-5, 0.3)
left.set_xlabel("$a_4$ along the constant-rate history")
left.set_ylabel("$P$: probability outside the band level")
left.set_title("Dressed start: exact against first order")
left.legend(fontsize=8, loc="lower left")
right.plot(s_marks, runs[1.0], color=PALETTE[1], lw=2.0, label="dressed start")
right.plot(s_marks, bare, color=PALETTE[0], lw=2.0, ls="-.", label="bare start")
right.plot(s_marks, dressing(1.0), color="0.4", lw=1.5, ls="--",
           label="first order $\\sum_m Q_{0m}^2$")
right.set_xlabel("$a_4$ along the constant-rate history")
right.set_ylabel("$P$")
right.set_title("$A = 1$: dressed and bare start")
right.legend(fontsize=8)
save_figure(fig, "constant_rate",
```

```
            "Left: the probability $P$ that the particle is not in its "
            "instantaneous band level (vertical axis, logarithmic) along a history "
            "of constant rate $A = 0.3$, $1$, $3$ from $a_4 = 0$ to $2$ "
            "(horizontal axis), exact (solid) and the first-order dressing "
            "$\\sum_m Q_{0m}^2$ (dashed), starting in the dressed state. At $A = "
            "0.3$ they agree; at $A = 1$ the exact $P$ stays near 0.009 while the "
            "estimate falls; at $A = 3$ first order fails. Right: $A = 1$ with the "
            "dressed start and with the bare start (an abrupt start of the motion), "
            "which leaves about three times more.")
deviation = float(np.max(np.abs(runs[0.3] / dressing(0.3) - 1.0)))
report("A = 0.3: largest relative difference exact / first order", f"{deviation:.3f}")
report("A = 1: exact P at a4 = 0 and 2", f"{runs[1.0][0]:.5f}, {runs[1.0][-1]:.5f}")
report("A = 1: first-order dressing at a4 = 0 and 2",
       f"{dressing(1.0)[0]:.5f}, {dressing(1.0)[-1]:.5f}")
report("A = 1, bare start: largest P", f"{bare.max():.5f}")
check(deviation < 0.15, "at A = 0.3 the exact P agrees with the first-order dressing "
      "within 15 percent at all 101 points")
```

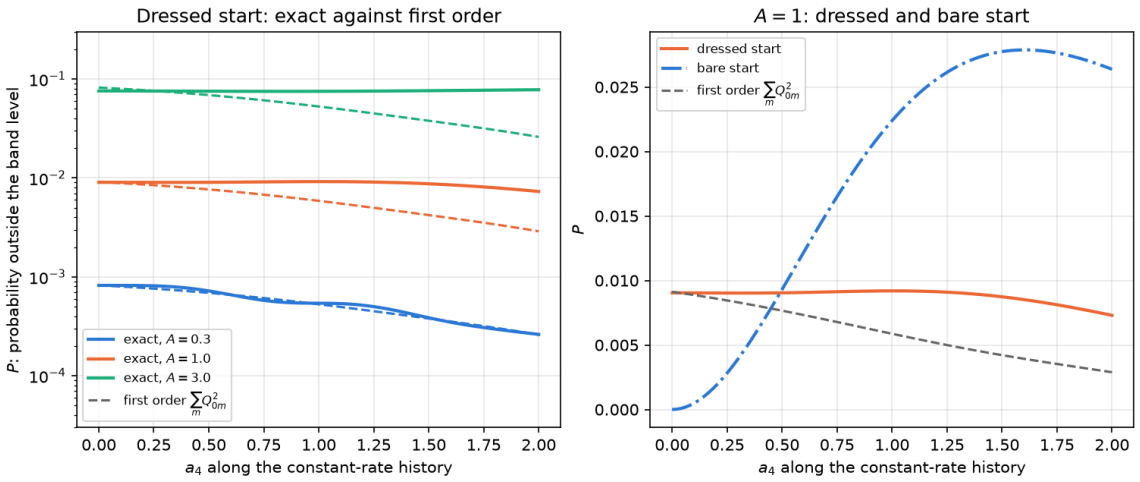

Figure 556: Left: the probability P that the particle is not in its instantaneous band level (vertical axis, logarithmic) along a history of constant rate $A = 0.3$, 1, 3 from $a_4 = 0$ to 2 (horizontal axis), exact (solid) and the first-order dressing $\sum_m Q_{0m}^2$ (dashed), starting in the dressed state. At $A = 0.3$ they agree; at $A = 1$ the exact P stays near 0.009 while the estimate falls; at $A = 3$ first order fails. Right: $A = 1$ with the dressed start and with the bare start (an abrupt start of the motion), which leaves about three times more. (Notebook 22a, figure 6.)

Out [14]:

```
Figure 22a.6 saved as Revision/textbook/figures/22a_6_constant_rate.png
RESULT A = 0.3: largest relative difference exact / first order = 0.113
RESULT A = 1: exact P at a4 = 0 and 2 = 0.00904, 0.00730
RESULT A = 1: first-order dressing at a4 = 0 and 2 = 0.00912, 0.00289
RESULT A = 1, bare start: largest P = 0.02787
PASS at A = 0.3 the exact P agrees with the first-order dressing within 15 percent at all
    101 points
```

Text cell:

```
## 15. How far the sudden limit reaches, and how many levels are needed

The sudden limit caps the damage, and it depends on how much the band orbital
itself changes. The next cell computes the band orbital at the end slices $a_{4,0}
= 0, 0.1, ..., 6$ (beyond the record's span; free field, computed here) and the
sudden limit $1 - |\int\varphi_0(a_{4,0})^\dagger\varphi_0(0)\,dy|^2$ against the end
slice, and draws a few of the orbitals.
```

In [15]:

```
ENDS = np.linspace(0.0, 6.0, 61)  # end slices of a sudden jump from a4,0 = 0
ends_q = K11 * np.exp(-ENDS)
band_a, band_b = orbitals(levels(np.zeros(61), ends_q), ends_q)
ceiling = 1.0 - (band_a @ (SIMPSON * band_a[0]) + band_b @ (SIMPSON * band_b[0])) ** 2
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), layout="constrained")
for colour, index in zip(PALETTE, (0, 10, 20, 60)):
    left.plot(Y, band_a[index], color=colour, lw=2.0,
              label=f"$a$, $a_{{4,0}} = {ENDS[index]:.0f}$")
    left.plot(Y, band_b[index], color=colour, lw=1.2, ls="--")
left.plot([], [], color="0.4", ls="--", label="$b$ (dashed)")
left.set_xlabel("hidden coordinate $y$ (tip at $-3$, brane at 0)")
left.set_ylabel("band orbital (normalised)")
left.set_title("The band orbital of the Fermi shell")
left.legend(fontsize=8)
right.plot(ENDS, ceiling, color=PALETTE[0], lw=2.0)
right.plot([END], [sudden], "o", color=PALETTE[1], ms=9,
           label="end of the recorded span")
right.set_xlabel("end slice $a_{4,0}$ of a sudden jump from 0")
right.set_ylabel("$P_{sudden}$")
right.set_title("The ceiling of the damage")
right.legend(fontsize=8)
save_figure(fig, "sudden_ceiling",
            "Left: the band orbital of the Fermi shell of $N = 688$, components "
            "$a$ (solid) and $b$ (dashed), against the hidden coordinate $y$ "
            "(horizontal axis) at the slices $a_{4,0} = 0$, $1$, $2$, $6$: as the "
            "momentum redshifts, $b$ shrinks and the orbital approaches the brane "
            "zero mode $e^{y}$. Right: the sudden limit $P_{sudden}$ (vertical "
            "axis) of a jump from $a_{4,0} = 0$ to the end slice (horizontal "
            "axis); it grows and levels off near 0.105: even an instant jump to "
            "the far future leaves almost 90 percent of the particles in the band "
            "level.")
report("sudden limit for the end slices 2, 4, 6",
       ", ".join(f"{ceiling[i]:.4f}" for i in (20, 40, 60)))
check(np.all(np.diff(ceiling) > -1e-12) and ceiling[-1] < 0.11
      and abs(ceiling[20] - sudden) < 1e-12,
      "the sudden limit grows with the end slice and stays below 0.11 up to 6")
```

Out [15]:

```
Figure 22a.7 saved as Revision/textbook/figures/22a_7_sudden_ceiling.png
RESULT sudden limit for the end slices 2, 4, 6 = 0.0749, 0.1021, 0.1051
PASS the sudden limit grows with the end slice and stays below 0.11 up to 6
```

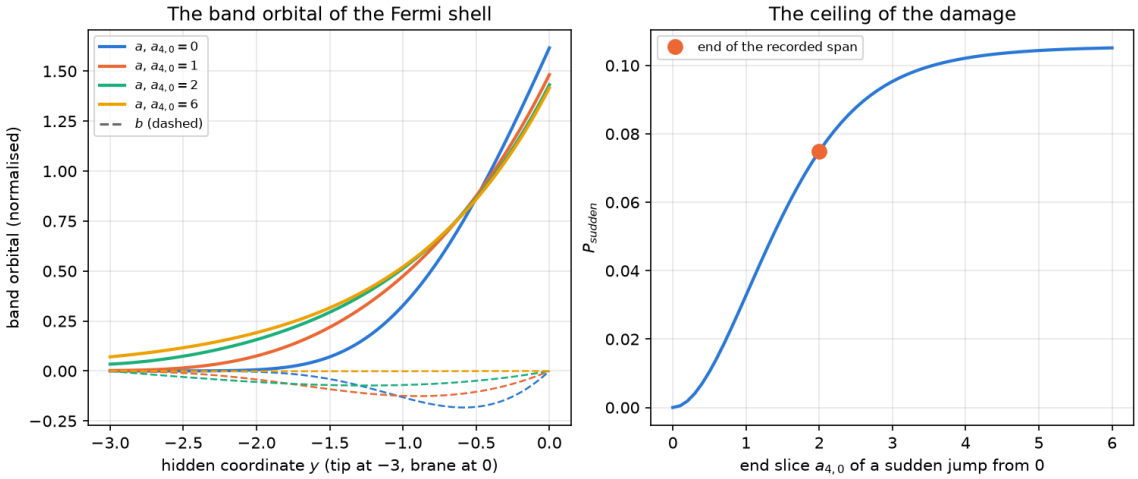

Figure 557: Left: the band orbital of the Fermi shell of $N = 688$, components a (solid) and b (dashed), against the hidden coordinate y (horizontal axis) at the slices $a_{4,0} = 0$, 1, 2, 6: as the momentum redshifts, b shrinks and the orbital approaches the brane zero mode e^{y}. Right: the sudden limit P_{sudden} (vertical axis) of a jump from $a_{4,0} = 0$ to the end slice (horizontal axis); it grows and levels off near 0.105: even an instant jump to the far future leaves almost 90 percent of the particles in the band level. (Notebook 22a, figure 7.)

Text cell:

```
The last numerical question is how good the truncated basis and the time steps
are. The next cell repeats the sudden limit and the smooth passages at $A = 1$ and
$A = 3$ with smaller bases (the labels $-l ... l$ and $-l ... l + 1$, from 3 to 20
levels), draws the differences to the reference (the direct overlap for the sudden
limit, the 20-level result for $A = 1$ and $3$), and repeats three passages with
twice or half the number of steps.
```

In [16]:

```
sizes, errors = [], {"sudden": [], 1.0: [], 3.0: []}
results = {}
for low in range(1, 10):
    for high in (low, low + 1):
        chosen = np.where((BASIS >= -low) & (BASIS <= high))[0]
        zero = int(np.where(BASIS[chosen] == 0)[0][0])
        e, k = E_SMOOTH[:, chosen], K_SMOOTH[:, chosen][:, :, chosen]
        start = np.zeros(len(chosen), dtype=complex)
        start[zero] = 1.0
        sizes.append(len(chosen))
        c, _ = passage(0.0, e, k, RATE_SMOOTH, start, 400)
        errors["sudden"].append(abs(1.0 - abs(c[zero]) ** 2 - sudden))
        for rate in (1.0, 3.0):
            duration = 2.0 * END / rate
            c, _ = passage(duration, e, k, RATE_SMOOTH, start, steps_needed(duration))
            results[(len(chosen), rate)] = 1.0 - abs(c[zero]) ** 2
for rate in (1.0, 3.0):
```

```
    errors[rate] = [abs(results[(size, rate)] - results[(SIZE, rate)])
                    for size in sizes]
halving = 0.0
for rate in (RATES[0], 1.0, 10.0):
    duration = 2.0 * END / rate
    steps = steps_needed(duration)
    other = 2 * steps if steps < U_STEPS else steps // 2
    p_one, p_two = (1.0 - abs(passage(duration, E_SMOOTH, K_SMOOTH, RATE_SMOOTH, START,
                                      n)[0][ZERO]) ** 2 for n in (steps, other))
    halving = max(halving, abs(p_one - p_two))
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(sizes, errors["sudden"], "o-", color=PALETTE[0], lw=2.0, ms=6,
        label="sudden limit (against the direct overlap)")
for colour, marker, rate in ((PALETTE[1], "s", 1.0), (PALETTE[2], "^", 3.0)):
    ax.plot(sizes[:-1], errors[rate][:-1], marker + "--", color=colour, lw=1.5, ms=6,
            label=f"$A = {rate:.0f}$ (against 20 levels)")
ax.set_yscale("log")
ax.set_xticks(range(3, 21))
ax.set_xlabel("number of instantaneous levels in the basis")
ax.set_ylabel("$|P - P_{reference}|$")
ax.set_title("Convergence with the size of the basis")
ax.legend(fontsize=8)
save_figure(fig, "convergence",
            "The error of the probability $P$ (vertical axis, logarithmic) against "
            "the number of instantaneous levels kept in the basis (horizontal "
            "axis), for the sudden limit (blue, against the direct overlap of the "
            "two orbitals) and for the smooth passages at $A = 1$ and $A = 3$ "
            "(against the 20-level result). The errors fall steadily; with 20 "
            "levels the sudden limit is right to a few millionths.")
report("sudden limit: error with 20 levels", f"{errors["sudden"][-1]:.1e}")
report("A = 1: change from 18 to 20 levels", f"{errors[1.0][-3]:.1e}")
report("largest change of P when the steps are doubled or halved", f"{halving:.1e}")
check(errors["sudden"][-1] < 1e-5 and errors[1.0][-3] < 1e-6,
      "the 20-level basis is converged: sudden limit within 1e-5, A = 1 within 1e-6")
check(halving < 1e-9, "doubling or halving the time steps changes P by less than 1e-9")
```

Out [16]:

```
Figure 22a.8 saved as Revision/textbook/figures/22a_8_convergence.png
RESULT sudden limit: error with 20 levels = 4.2e-06
RESULT A = 1: change from 18 to 20 levels = 2.4e-08
RESULT largest change of P when the steps are doubled or halved = 3.0e-11
PASS the 20-level basis is converged: sudden limit within 1e-5, A = 1 within 1e-6
PASS doubling or halving the time steps changes P by less than 1e-9
```

Text cell:

```
## 16. The breakdown estimate

The last cell collects the four estimates of the rate at which the instantaneous
picture breaks down, for the free Fermi-shell sector of $N = 688$ over the recorded
span, prints the probability that a particle is not in its instantaneous level at
```

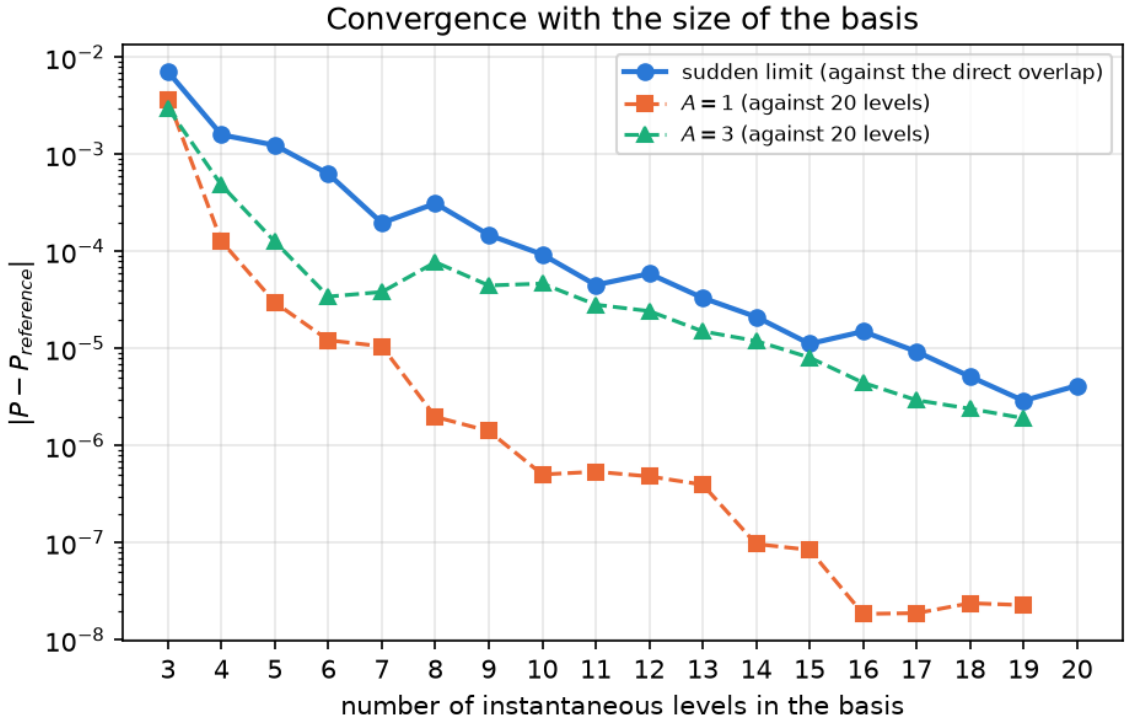

Figure 558: The error of the probability P (vertical axis, logarithmic) against the number of instantaneous levels kept in the basis (horizontal axis), for the sudden limit (blue, against the direct overlap of the two orbitals) and for the smooth passages at $A = 1$ and $A = 3$ (against the 20-level result). The errors fall steadily; with 20 levels the sudden limit is right to a few millionths. (Notebook 22a, figure 8.)

the record's rate, checks that every figure file exists, and prints the number of checks that passed.

In [17]:

```
q_first = q_max[(688, "lam0", 0.0)]  # the largest Q of the free Fermi shell, A = 1
estimates = [
    ("naive: Q = 1 for the weakest recorded state", naive[weakest]),
    ("naive: Q = 1 for the band level, 0.02 <= q <= 6", 1.0 / g_star),
    ("first order: Q^2 reaches the sudden limit", math.sqrt(sudden) / q_first),
    ("exact: P reaches half the sudden limit", rate_half),
]
say("breakdown estimate                                    rate A")
for label, value in estimates:
    say(f"{label:50s} {value:9.3f}")
report("P at the record's rate A = 1, constant rate (dressed start)",
       f"{runs[1.0].min():.4f} ... {runs[1.0].max():.4f}")
report("P at the peak rate A = 1, smooth passage", f"{excited[6]:.4f}")
report("sudden limit over the span 0 ... 2, the most P reaches in the scan",
       f"{sudden:.4f}")
NAMES = ["naive_rates", "one_curve", "across_the_gap", "sector_levels",
         "smooth_passage", "constant_rate", "sudden_ceiling", "convergence"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/22a_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

Out [17]:

```
breakdown estimate                                    rate A
naive: Q = 1 for the weakest recorded state           10.701
naive: Q = 1 for the band level, 0.02 <= q <= 6       10.021
first order: Q^2 reaches the sudden limit              2.929
exact: P reaches half the sudden limit                 1.508
RESULT P at the record's rate A = 1, constant rate (dressed start) = 0.0073 ... 0.0092
RESULT P at the peak rate A = 1, smooth passage = 0.0138
RESULT sudden limit over the span 0 ... 2, the most P reaches in the scan = 0.0749
PASS every figure file of this notebook exists
ALL 20 CHECKS PASSED (notebook 22a)
```

Text cell:

```
## 17. What this notebook showed

- The record's adiabaticity measure $Q$ is, in this notebook's reading (the record
  calls $Q^2$ the leading-order transition probability), the first-order amplitude
  of the dressing that a moving particle carries; by an exact argument it grows in
  proportion to the rate $A$ of the history. At the record's rate $A = 1$,
  $Q_{max} \le 0.0935$ for all 75 states (COMPUTED, Revision record), so the naive
  breakdown "$Q = 1$" would need $A \ge 10.7$.
- The solver's shooting method, written again in vectorised Python, reproduces the
```

```
  record's levels and $Q$ of the free Fermi shells; with the exact rescaling
  identity every slice and every shell falls on ONE curve $Q/A = G(q)$, whose top
  is $G^* = 0.0998$ at $q = 2.13$ (COMPUTED here): for a particle in the band level
  of every shell and slice with $0.02 \le q \le 6$ (all states of the record) the
  naive breakdown rate is at least $A = 10.0$.
- Jumps across the gap into the negative branch, which the record leaves out, are
  weaker than the top of the band jump (below 0.05 per unit rate; the strongest,
  $-1 \to$ bulk 1, peaks near $q = 0.11$) but stronger than the band jump itself
  below $q$ of about 0.1; their reading as pair creation of the field in 4+4
  dimensions is OPEN, and they say nothing about the creation of universes.
- The exact evolution of the free Fermi-shell sector of $N = 688$ (20-level basis,
  converged to a few millionths) tells a sharper story: the gas stops following its
  instantaneous level already near $A = 1$ ($A_{1/2} \approx 1.5$ for a smooth
  passage), about seven times below the naive estimate ($1/Q_{max} = 10.7$); but
  the damage is capped by the sudden limit, 7.5 percent over the recorded span and
  about 10.5 percent even for an instant jump to $a_{4,0} = 6$, because the band
  orbital itself changes slowly.
  At the record's rate the probability to find a Fermi-shell particle outside its
  instantaneous level is about 1 percent (COMPUTED here).
- What remains OPEN: the self-consistent time-dependent Kohn-Sham problem with
  interaction, the meaning of the negative branch, the history itself (a PRESCRIBED
  BACKGROUND, without back-reaction) and the ASSUMED Z2 brane. A student could
  attack the first by starting from the record's self-consistent states and
  recomputing the potentials $M$ and $v$ at every time step from the evolving
  orbitals of all occupied sectors (time-dependent Kohn-Sham theory), then
  repeating sections 13 and 14 with them.
```

22.12 Line-by-line walk-through of Notebook 22a

The notebook has 17 code cells, In [1] to In [17]. This section explains every line of every one of them, in order. In the notebook each code cell is preceded by a text cell that says what the cell does; the printed output of each cell is in Section 22.11 under the label Out [k], and the eight figures are shown there with their captions. For each figure this section adds what the student should see and why.

In [1], the set-up cell. It is the same in every notebook of the book except for the notebook's name. Its first 231 lines repeat the complete run instructions of Section 22.10 as **comment lines**: every line that starts with `#` is skipped by Python and is there only so that the notebook file carries its own instructions. A line of dashes and the title THE SET-UP between two lines of `=` signs mark where the code begins.

```
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "22a"  # this notebook: chapter 22, example a
```

`import` loads a **module** (a part of Python or of an installed package) so that the cell can use it; the text after `#` on each line is a comment that says what the module is for. `json` reads the text format JSON in which most Revision records are stored. `os` gives access to the operating system, here to read an environment variable. `textwrap` breaks long printed text into lines. `from pathlib import Path` takes the single name `Path` out of the module `pathlib`; a `Path` is the address of a file or folder, written the same way on every operating system. `matplotlib` is the plotting package and `matplotlib.pyplot` its drawing functions, called `plt` by convention. `Image` and `display` show a saved picture below a cell. The last line gives the name `NOTEBOOK_ID` to the **string** (a text in quotes) `"22a"`; the figure files are named after it.

```
def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")
```

`def` defines a **function**: a named piece of code that runs when it is called. The text in triple quotes under the `def` line is its **docstring**, a description that Python keeps but does not run. `Path.cwd()` is the folder in which the notebook runs, and `.resolve()` writes it as a complete address. The list `[here, *here.parents]` holds this folder followed by its parent, the parent of that, and so on (the star unpacks the parents into the list), and the `for` loop takes them one after the other. The operator `/` joins a folder and a name into a longer path, and `.is_file()` is true when that file exists. The first folder that holds `Revision/textbook/requirements.txt` is the repository, and `return` hands it back; if no folder does, `raise` stops the notebook with an error message that says what to do.

```
# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
```

```
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))
```

`REPO` is the repository folder; it is never printed, because it differs from computer to computer while the printed output must be the same everywhere. `os.environ.get(name, default)` reads an **environment variable** (a named text that a program receives from the computer) or returns the default; when you run the notebook the variable TEXTBOOK_OUTPUT_ROOT is not set, so `OUTPUT_ROOT` is the repository, while the book's checking tool sets it to a scratch folder so that a check never changes the repository. `repository_file` gives the full path of a repository file for reading; `output_file` gives the full path at which to write one, after creating its folder (`mkdir` with `parents=True` creates missing parent folders as well, and `exist_ok=True` makes it do nothing if the folder exists). `say` prints a text in lines of at most 89 characters, continuation lines indented by four blanks.

```
matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")
```

`matplotlib.rcdefaults()` returns to the built-in plotting settings, so that the figures are the same on every computer, and `plt.rcParams.update` sets the default figure size (7.0 by 4.2 inches), the letter size (10 points) and a faint grid. The braces `{...}` make a **dictionary**: pairs of a key and a value, written `key: value`. A string that starts with `f` is an **f-string**: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/22a.captions.json"`. `FIGURE_NUMBERS` and `CAPTIONS` start as empty dictionaries. The last line writes an empty dictionary `{}` into the captions file; `newline="\n"` stores the same line end on every operating system.

```
def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
```

```
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")
```

`save_figure` numbers the figures 1, 2, 3, ... in the order in which they are saved (`setdefault` returns the number already stored for this name, or stores one more than the count so far), saves the figure as a PNG file with 150 dots per inch, cut to its content and without the program's name inside (so that two runs write the same bytes), closes it, records its caption in the captions file (`json.dumps` turns the dictionary into JSON text, one entry per line, keys sorted), shows the saved picture below the cell, and prints where it was saved. The caption given to it is the caption that the book prints under the figure.

```
PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

`PASSED` is an empty **list** (an ordered collection, in square brackets). `check` is the function behind every check of the notebook: if the statement `condition` is false, `raise AssertionError(...)` stops the notebook with an error that names the check; if it is true, the name is appended to `PASSED`

and the line PASS followed by the name is printed, and a second line names the Revision record when the argument `record` is given (`None` is Python's word for "nothing"). Python's own `assert` statement is not used, because Python started with the option `-O` skips it. `report` prints a key number as a line that starts with RESULT. `all_checks_passed` prints the last line of the notebook with the number of checks that passed (`len` is the length of a list). The last statement prints the one output line of In [1].

In [2], the record: Q of the 75 ground states.

```
import csv  # reads the tables (CSV files) of the Revision record
import math  # exp, sqrt and log of single numbers

import numpy as np  # arrays of numbers and their arithmetic
from numpy.polynomial import chebyshev  # Chebyshev polynomials (section 11)
```

Four more modules: `csv` reads tables stored as CSV files (comma-separated values: one line per row, the entries separated by commas); `math` computes the exponential, square root and logarithm of single numbers; `numpy`, called `np`, holds **arrays** (tables of numbers of any shape) and computes with whole arrays at once; and `chebyshev` evaluates sums of Chebyshev polynomials, which In [9] uses.

```
ADIABATIC = "Revision/kohn_sham/results/adiabatic/adiabaticity.csv"
HISTORY = "Revision/kohn_sham/results/adiabatic/history.json"
THEORY = "Revision/kohn_sham/ks-theory.json"
SLICES = [0.0, 0.5, 1.0, 1.5, 2.0]  # the slices a4,0 of the record
TAGS = ["lam0", "lamp1", "lamm1", "lamp2", "lamm2"]  # 0, +-lambda_1, +-lambda_2
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

Three names for the repository paths of the records the cell reads; the five slices of the record; the five coupling tags; and eight colours, written as hexadecimal codes, which every figure uses in this fixed order (blue, orange, green, yellow, pink, dark green, violet, red).

```
def state_id(n, a4, tag="lam0"):
    """The record's name of a ground state, for example N688_lam0_a05."""
    return f"N{n}_{tag}_a{round(10 * a4):02d}"
```

`state_id` builds the record's name of a state: for $n = 688$, $a_4 = 0.5$ and the default tag lam0 it returns `N688_lam0_a05`. Inside the braces of the f-string, `round(10 * a4)` turns the slice into the whole number 5, and the format `:02d` writes it with two digits, padded by a zero.

```
with open(repository_file(ADIABATIC), newline="", encoding="utf-8") as handle:
    record = {row["id"]: row for row in csv.DictReader(handle)}
history = json.loads(repository_file(HISTORY).read_text(encoding="utf-8"))
theory = json.loads(repository_file(THEORY).read_text(encoding="utf-8"))
```

`with open(...) as handle:` opens the table and closes it again when the indented line has run. `csv.DictReader` reads it row by row, each row a dictionary from the column names of the first line (`id`, `N`, `lambda_tag`, `Q_max`, `Q_max_pair`, ...) to the texts of that row. The braces with `for` inside make a **dictionary comprehension**: `record` maps each state's name (column `id`) to its row. `json.loads` turns the text of the two JSON files into Python dictionaries.

```
q_max = {(n, tag, a4): float(record[state_id(n, a4, tag)]["Q_max"])
         for n in (8, 136, 688) for tag in TAGS for a4 in SLICES}
largest = max(q_max.values())
n8_zero = all(q_max[(8, tag, a4)] == 0.0 for tag in TAGS for a4 in SLICES)
```

`q_max` maps every triple (particle number, tag, slice), a **tuple** written in round brackets, to the number in the column `Q_max` of that state; `float` turns the text into a number. The two lines of the comprehension run over the 3 particle numbers, the 5 tags and the 5 slices: 75 states. `largest` is the largest of all 75 values. `all(...)` is true when every one of the 25 values for $N = 8$ is exactly zero.

```
report("ground states in the record", len(record))
report("largest Q_max at the rate A = 1", f"{largest:.10f}")
say("The record's measure: " + theory["adiabaticity"]["measure"])
```

Out [2] shows 75 states and the largest value 0.0934532439, printed with ten digits after the point (format `.10f`), and then the record's own definition of Q from the key `adiabaticity.measure` of `ks-theory.json`: the formula derived in Section 22.7, with the remark that Q^2 is the leading-order transition probability.

```
check(history["status"].startswith("PRESCRIBED BACKGROUND")
      and theory["adiabaticity"]["historyStatus"].startswith("PRESCRIBED BACKGROUND"),
      "the history a4 = A H x4 is labelled a PRESCRIBED BACKGROUND",
      record=f"{HISTORY}, key status, and {THEORY}, adiabaticity.historyStatus")
check(len(record) == 75 and largest <= 0.0935 and n8_zero,
      "75 states, Q_max <= 0.0935, and Q_max = 0 exactly for N = 8",
      record=f"{ADIABATIC}, column Q_max")
```

The first check requires that both records begin their statement of the history with the words PRESCRIBED BACKGROUND (`startswith` tests the beginning of a string): the notebook works on a background that is given, not solved for, and says so. The second check requires the 75 rows, the bound 0.0935 and the exact zeros for $N = 8$. Those zeros have a reason: the eight quanta of $N = 8$ occupy only the zero modes at $k = 0$, and at $k = 0$ the derivative $\partial_a h = -j\kappa k\sigma_3$ of Section 22.6 vanishes.

In [3], Q grows in proportion to the rate (Figure 22a.1).

```
naive = {key: 1.0 / value for key, value in q_max.items() if value > 0.0}
weakest = min(naive, key=naive.get)  # the state that reaches Q = 1 first
rates = np.geomspace(0.1, 100.0, 200)  # rates A for the right panel
```

By exact rule 1 of Section 22.7, Q reaches 1 at the rate $A = 1/Q_{max}$. `naive` maps every state with $Q_{max} > 0$ (the 50 states with $N = 136$ and 688) to this rate. `min(naive, key=naive.get)` returns the key whose value is smallest: the state that would reach $Q = 1$ first. `np.geomspace(0.1 , 100.0, 200)` makes 200 rates from 0.1 to 100, equally spaced on a logarithmic scale (each one the same factor larger than the one before).

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), layout="constrained")
for colour, marker, n in ((PALETTE[0], "o", 136), (PALETTE[1], "s", 688)):
    band = np.array([[naive[(n, tag, a4)] for a4 in SLICES] for tag in TAGS])
    left.fill_between(SLICES, band.min(axis=0), band.max(axis=0), color=colour,
                      alpha=0.25, lw=0)  # the spread over the five couplings
    left.plot(SLICES, band[0], "-", color=colour, marker=marker, ms=8, lw=2.0,
              label=f"$N = {n}$")
```

`plt.subplots(1, 2, ...)` makes a figure with two panels side by side, `left` and `right`, 10 by 4.2 inches, with the layout adjusted automatically. For $N = 136$ (blue circles) and $N = 688$ (orange squares) the loop builds `band`, an array with one row per tag and one column per slice. `fill_between` shades the area between the smallest and the largest value of each column (`axis=0` takes the minimum down the columns), that is, the spread over the five couplings, a quarter opaque (`alpha=0.25`) and without border (`lw=0`). `left.plot` draws the first row, the free state lam0, as a line with markers of size 8.

```
left.axhline(1.0, color="0.3", lw=1.2, ls=":")
left.text(0.05, 1.12, "the record's rate $A = 1$", fontsize=8, color="0.3")
left.set_yscale("log")
left.set_ylim(0.7, 50.0)
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("naive breakdown rate $1/Q_{max}$")
left.set_title("Rate at which $Q$ would reach 1")
left.legend(fontsize=8)
```

A dotted horizontal line at $A = 1$, the record's rate, with a small label; a logarithmic vertical axis from 0.7 to 50; the axis labels, the title and the legend. Text between dollar signs is set as mathematics by matplotlib.

```
for colour, a4 in zip(PALETTE, SLICES):
    right.plot(rates, rates * q_max[(688, "lam0", a4)], color=colour, lw=2.0,
               label=f"$a_{{4,0}} = {a4}$")
right.axhline(1.0, color="0.3", lw=1.2, ls=":")
right.axvline(1.0, color="0.3", lw=1.2, ls="--")
right.text(1.08, 0.012, "record: $A = 1$", fontsize=8, color="0.3")
right.set_xscale("log")
right.set_yscale("log")
right.set_xlabel("rate $A$ (e-folds of the extra times per unit time $1/H$)")
right.set_ylabel("$Q = A\\,Q_{max}$")
right.set_title("$N = 688$, $\\lambda = 0$: $Q$ grows like $A$")
right.legend(fontsize=8)
```

`zip` pairs the first five colours with the five slices. For each slice the right panel draws $Q = A\,Q_{max}$ of the free state $N = 688$ against the 200 rates. In the f-string the doubled braces `{{4,0}}` print single braces, so the label reads $a_{4,0} = 0.5$ and so on. A dotted line marks $Q = 1$, a dashed vertical line the record's rate; both axes are logarithmic. In a normal Python string a backslash starts an escape, so `\\,` and `\\lambda` are written with two backslashes to give matplotlib the single backslash of `\,` (a thin space) and of `\lambda`.

```
save_figure(fig, "naive_rates",
            "Left: the naive breakdown rate $1/Q_{max}$ (vertical axis, logarithmic) "
            "of the ground states $N = 136$ (blue circles) and $N = 688$ (orange "
            "squares) against the slice $a_{4,0}$ (horizontal axis); the lines are "
            "the free states, the shaded bands the spread over the five couplings, "
            "the dotted line the record's rate $A = 1$. Right: $Q = A\\,Q_{max}$ "
            "against the rate $A$ for the free state $N = 688$ at the five slices "
            "(both axes logarithmic); $Q$ reaches 1 (dotted) only above $A = 10$. "
            "The naive estimate says the record's history is at least ten times "
            "too slow to break the instantaneous picture.")
report(f"naive breakdown rate 1/Q_max of {state_id(weakest[0], weakest[2], weakest[1])}",
       f"{naive[weakest]:.4f}")
check(naive[weakest] > 10.0,
      "every recorded state keeps Q below 1 up to the rate A = 10",
      record=f"{ADIABATIC}, column Q_max")
```

`save_figure` saves Figure 22a.1 with its caption (Python joins strings written next to each other into one). The report names the weakest state, N688_lamm2_a00 (the tuple `weakest` is (688, "lamm2", 0.0), so `weakest[0]` is the particle number, `weakest[2]` the slice and `weakest[1]` the tag), and its naive breakdown rate 10.7005 (Out [3]). The check requires that every recorded state keeps Q below 1 up to the rate $A = 10$.

What the student should see in Figure 22a.1, and why. On the left every point lies far above the dotted line: the naive breakdown rate is between 10.7 (N688_lamm2_a00) and 27.8 (N136_lamm2_a20; 26.05 for the free state N136 at the slice 2), so by this measure the record's history is at least ten times too slow to disturb the instantaneous picture. Both curves rise with the slice: as the history goes on, the momentum of the Fermi shell redshifts, $q = ke^{-a_{4,0}}$ falls, and Q/A falls with it (Figure 22a.2 shows why). The shaded bands are thin: the five couplings change Q_{max} by at most about ten percent. On the right the five lines are straight with slope 1 on the logarithmic axes, which is exact rule 1: ten times the rate gives ten times Q. They cross the dotted line $Q = 1$ between $A = 10.7$ and $A = 18.9$.

In [4], the solver's shooting method, vectorised.

```
PARAMETERS = json.loads(repository_file("Revision/kohn_sham/results/parameters.json")
                        .read_text(encoding="utf-8"))
L = PARAMETERS["physics"]["L_tipCutoff"]  # the tip cutoff y = -L
STEPS = PARAMETERS["numerics"]["rk4Steps"]  # RK4 steps from the tip to the brane
POINTS = 2 * STEPS + 1  # the fine grid: step ends and step midpoints
STEP = L / STEPS  # the length of one RK4 step in y
```

The cell reads the record's parameter file and takes from it the tip cutoff $L = 3$ and the number of RK4 steps, 900, with which the Rust solver integrates from the tip $y = -L$ to the brane $y = 0$. RK4 evaluates the equation at the start, the middle and the end of every step, so the notebook keeps a **fine grid** of $2 \cdot 900 + 1 = 1801$ points: the 901 step ends and the 900 midpoints. One step has the length $3/900 = 1/300$.

```
Y = -L * ((POINTS - 1 - np.arange(POINTS)) / (POINTS - 1))  # y from -L to 0
EW = np.exp(-Y)  # e^{-Hy} with H = 1, so kappa k = q e^{-y} with q = k e^{-a4,0}
SIMPSON = np.full(POINTS, 2.0 * STEP / 6.0)  # Simpson weights on the fine grid:
```

```
SIMPSON[1::2] = 4.0 * STEP / 6.0  # midpoints carry 4/6 of a step,
SIMPSON[0] = SIMPSON[-1] = STEP / 6.0  # the two ends 1/6
```

`np.arange(POINTS)` is the array $0, 1, \ldots, 1800$; the formula turns it into the 1801 values of y, from -3 at index 0 to 0 at the last index, equally spaced. `EW` holds e^{-y} at every point: with $H = 1$ the momentum term of Section 22.6 is $\kappa k = qe^{-y}$ (exact rule 2). `SIMPSON` holds the weights of **Simpson's rule**: the integral of a function over one step is $\frac{h}{6}(f_{start} + 4f_{middle} + f_{end})$ with h the step length, and adding these over all steps gives every midpoint the weight $4h/6$, every inner step end $2h/6$ (it belongs to two steps), and the two outer ends $h/6$. `np.full` fills an array with one value, `SIMPSON[1::2]` selects every second entry starting at index 1 (the midpoints), and the index `-1` means the last entry. An integral is then `np.sum(SIMPSON * f)`.

```
check(PARAMETERS["physics"]["H"] == 1.0 and PARAMETERS["physics"]["m"] == 1.0
      and L == 3.0 and STEPS == 900 and PARAMETERS["physics"]["dk"] == 0.25,
      "the record's units H = m = 1, tip cutoff L = 3, lattice step 0.25, 900 steps",
      record="Revision/kohn_sham/results/parameters.json, physics and numerics")
```

The check requires the record's units $H = m = 1$, $L = 3$, the lattice step $\Delta k = 0.25$ of the 3-momenta and 900 steps, the numbers on which everything below relies (Out [4]).

```
def shoot(eps, q, store=False):
    """Integrate the free block equation from the tip to the brane for the arrays
    eps (energies) and q (redshifted momenta) at once.  Returns the phase
    Phi = theta(0) and, if store is True, the list of (a, b) at the step ends."""
    a, b = np.ones_like(eps), np.zeros_like(eps)  # tip: b(-L) = 0, a(-L) = 1
    theta, raw = np.zeros_like(eps), np.zeros_like(eps)  # the angle, last atan2
    ends = [(a, b)] if store else None
```

`shoot` solves the real form of the free block equation of Section 22.6 ($j = +1$, $M = 1$, $v = 0$), $a' = a - (qe^{-y} + \varepsilon)b$, $b' = (\varepsilon - qe^{-y})a - b$, for a trial energy ε, from the tip to the brane. It is **vectorised**: `eps` and `q` are arrays, and every line acts on all their entries at once, so one call solves hundreds of cases. At the tip it starts with $a = 1$, $b = 0$ (`np.ones_like` and `np.zeros_like` make arrays of ones and zeros of the same shape as `eps`); $b(-L) = 0$ is the record's regular tip condition, and the size of a is irrelevant because the equation is linear. `theta` will hold the **Pruefer angle**, the angle of the point (a, b) in the plane, followed continuously; `raw` holds the angle of the previous step; `ends` collects the values (a, b) at the step ends when `store` is true.

```
    for i in range(STEPS):
        k0, k1, k2 = q * EW[2 * i], q * EW[2 * i + 1], q * EW[2 * i + 2]
        p1a, p1b = a - (k0 + eps) * b, (eps - k0) * a - b  # (a', b') at the start
        aa, bb = a + 0.5 * STEP * p1a, b + 0.5 * STEP * p1b
        p2a, p2b = aa - (k1 + eps) * bb, (eps - k1) * aa - bb  # at the midpoint
        aa, bb = a + 0.5 * STEP * p2a, b + 0.5 * STEP * p2b
        p3a, p3b = aa - (k1 + eps) * bb, (eps - k1) * aa - bb  # midpoint again
        aa, bb = a + STEP * p3a, b + STEP * p3b
        p4a, p4b = aa - (k2 + eps) * bb, (eps - k2) * aa - bb  # at the end
        a = a + STEP / 6.0 * (p1a + 2.0 * p2a + 2.0 * p3a + p4a)  # the RK4 step
        b = b + STEP / 6.0 * (p1b + 2.0 * p2b + 2.0 * p3b + p4b)
```

The loop takes the 900 steps. `k0`, `k1`, `k2` are qe^{-y} at the start, the middle and the end of step `i` (fine-grid indices $2i$, $2i+1$, $2i+2$). Then comes the classical **Runge-Kutta rule of order 4** (RK4, Chapter 2): the first slope `p1` from the equation at the start; a half step with it to the midpoint and the second slope `p2` there; a half step with `p2` and the third slope `p3`, again at the midpoint; a full step with `p3` and the fourth slope `p4` at the end; and the new values as the start plus one step times the weighted mean $(p_1 + 2p_2 + 2p_3 + p_4)/6$. Each line computes the two components a and b side by side.

```
        new = np.arctan2(b, a)  # the angle of the point (a, b), in (-pi, pi]
        turn = new - raw  # how far the angle turned in this step, brought back
        turn = np.where(turn > np.pi, turn - 2.0 * np.pi, turn)  # into (-pi, pi]
        turn = np.where(turn <= -np.pi, turn + 2.0 * np.pi, turn)
        theta, raw = theta + turn, new  # the angle, followed continuously
        if store:
            ends.append((a, b))
    return theta, ends
```

`np.arctan2(b, a)` is the angle of the point (a, b), between $-\pi$ and π. When the point circles around the origin this angle jumps by 2π as it passes π; the next three lines remove such jumps: the turn of one step is brought back into the range $(-\pi, \pi]$ (`np.where(condition, x, y)` takes `x` where the condition holds and `y` elsewhere), and adding the turns gives the angle followed continuously. The steps are short, so the true turn of one step is far below π and this is safe. At the brane, $\Phi = \theta(0)$ is the phase of the orbital. Chapter 15 showed for the Rust solver why this counts levels: Φ grows with ε, and $b(0) = 0$, the even brane condition, holds exactly when Φ is a whole multiple of π; the level with label n is the energy with $\Phi = n\pi$. The function returns the phases and, if asked, the stored values.

```
def levels(labels, q):
    """The level with label n (Phi = n pi) for every pair (label, q): bisection on
    [-40, 40], halving the bracket until it is shorter than 1e-13."""
    target = np.pi * np.asarray(labels, dtype=float)
    q = np.asarray(q, dtype=float)
    low, high = np.full(q.shape, -40.0), np.full(q.shape, 40.0)
    while np.max(high - low) > 1e-13:  # 50 halvings of the length 80
        middle = 0.5 * (low + high)
        above = shoot(middle, q)[0] > target  # Phi grows with eps
        low, high = np.where(above, low, middle), np.where(above, middle, high)
    return 0.5 * (low + high)
```

`levels` finds, for every pair of a label n and a momentum q, the energy at which $\Phi = n\pi$, by **bisection** (Chapter 2): every level lies in the bracket $[-40, 40]$; the phase is computed in the middle of the bracket; if it is above the target, the level lies in the lower half, otherwise in the upper half; the bracket is halved until it is shorter than 10^{-13}, the root tolerance of the Rust solver. Starting from the length 80 this takes 50 halvings, because $80/2^{50} \approx 7 \times 10^{-14}$. All pairs are bisected together, one vectorised call of `shoot` per halving. The answer is the middle of the last bracket.

```
def orbitals(eps, q):
    """The normalised orbitals (a, b) on the fine grid, one row per (eps, q)."""
    ends = shoot(eps, q, store=True)[1]
```

```
    a_e = np.array([pair[0] for pair in ends]).T  # rows: orbitals; columns: ends
    b_e = np.array([pair[1] for pair in ends]).T
    kap = q[:, None] * EW[None, 0::2]  # kappa k at the step ends
    da = a_e - (kap + eps[:, None]) * b_e  # the derivatives at the step ends
    db = (eps[:, None] - kap) * a_e - b_e
```

`orbitals` integrates again at the found levels, this time storing a and b at the 901 step ends, and arranges them as arrays with one row per orbital and one column per step end (`.T` transposes, that is, exchanges rows and columns). `q[:, None] * EW[None, 0::2]` multiplies a column of momenta with a row of the values e^{-y} at the step ends (`0::2` takes every second fine-grid point from index 0), which gives the table qe^{-y} for every orbital and every step end; `None` adds a dimension of length 1 so that numpy repeats the column and the row (**broadcasting**). `da` and `db` are the derivatives a' and b' from the block equation at every step end.

```
    a, b = np.zeros((len(eps), POINTS)), np.zeros((len(eps), POINTS))
    a[:, 0::2], b[:, 0::2] = a_e, b_e
    # cubic Hermite midpoint: (left + right)/2 + STEP (left' - right')/8
    a[:, 1::2] = 0.5 * (a_e[:, :-1] + a_e[:, 1:]) + STEP / 8.0 * (da[:, :-1] - da[:, 1:])
    b[:, 1::2] = 0.5 * (b_e[:, :-1] + b_e[:, 1:]) + STEP / 8.0 * (db[:, :-1] - db[:, 1:])
    norm = np.sqrt(np.sum(SIMPSON * (a * a + b * b), axis=1))  # Simpson's rule
    return a / norm[:, None], b / norm[:, None]
```

The orbitals are needed on the whole fine grid. The step ends are copied into the even columns. The midpoints are filled by **cubic Hermite interpolation**: the cubic polynomial with the values f_0, f_1 and the slopes f_0', f_1' at the two ends of a step of length h has, in the middle, the value $(f_0 + f_1)/2 + h(f_0' - f_1')/8$ (Exercise 3 of Section 22.28 derives it); `a_e[:, :-1]` are the left ends of all steps and `a_e[:, 1:]` the right ends. This is what the Rust solver does. Finally each orbital is **normalised**: its norm $\sqrt{\int (a^2 + b^2)\, dy}$ is computed with Simpson's rule (`axis=1` sums along each row), and both components are divided by it, so that $\int (a^2 + b^2)\, dy = 1$.

```
def couplings(q, a, b):
    """The matrix <m| d_a h |n> = -q int e^{-y} (a_m a_n - b_m b_n) dy of the
    orbitals (rows of a and b) of one sector at the redshifted momentum q."""
    weight = SIMPSON * EW  # Simpson weights times e^{-y}
    return -q * ((a * weight) @ a.T - (b * weight) @ b.T)
```

`couplings` computes the matrix of line 8 of Section 22.6 in its rescaled form (exact rule 2), $\langle m|\partial_a h|n\rangle = -q \int e^{-y}(a_m a_n - b_m b_n)\, dy$, for all pairs of orbitals of one sector at once. `@` is the matrix product: `(a * weight) @ a.T` has in row m and column n the sum over the grid of $a_m\, w\, a_n$, which is the Simpson integral of $e^{-y} a_m a_n$.

```
def per_unit_rate(eps, matrix):
    """G[n, m] = |<n| d_a h |m>| / (eps_n - eps_m)^2 = Q / A (0 for n = m)."""
    gap = eps[:, None] - eps[None, :]
    safe = np.where(gap == 0.0, 1.0, gap)  # 1 on the diagonal: no division by 0
    return np.where(gap == 0.0, 0.0, np.abs(matrix) / safe ** 2)
```

`per_unit_rate` turns the levels and the coupling matrix into the table $G_{nm} = |\langle n|\partial_a h|m\rangle|/(\varepsilon_n - \varepsilon_m)^2$, which is Q/A by Section 22.7. `gap` is the table of all differences of levels; on the diagonal it is zero, so `safe` puts 1 there before dividing, and the result is set to 0 there.

In [5], the Fermi shells of the record, solved again.

```
LEVELS = "Revision/kohn_sham/results/ground/levels"
SHELL = {136: 4, 688: 11}  # the Fermi shell n2 of each N
fermi = [(n, a4) for n in (136, 688) for a4 in SLICES]  # the ten free states
q_fermi = np.array([0.25 * math.sqrt(SHELL[n]) * math.exp(-a4) for n, a4 in fermi])
eps_fermi = levels(np.tile([0, 1, 2], 10), np.repeat(q_fermi, 3)).reshape(10, 3)
```

In the record the largest Q of every free state is the jump from the band level to the first bulk level of its Fermi shell: $n_2 = 4$, $k = 0.25\sqrt{4} = 0.5$, for $N = 136$, and $n_2 = 11$, $k = 0.25\sqrt{11} = 0.829$, for $N = 688$. `fermi` lists the ten free states (two particle numbers, five slices) and `q_fermi` their redshifted momenta $q = ke^{-a_{4,0}}$. `np.tile([0, 1, 2], 10)` repeats the labels ten times and `np.repeat(q_fermi, 3)` repeats each momentum three times, so the two arrays list the 30 pairs (label, q); one call of `levels` solves all 30, and `.reshape(10, 3)` arranges them as ten rows of three levels. Every level is solved at the slice 0 with the redshifted momentum, as exact rule 2 allows.

```
worst_level, compared = 0.0, 0
for (n, a4), row in zip(fermi, eps_fermi):
    path = repository_file(f"{LEVELS}/{state_id(n, a4)}.csv")
    with open(path, newline="", encoding="utf-8") as handle:
        for entry in csv.DictReader(handle):
            sector = (int(entry["n2"]), int(entry["j"]), entry["parity"])
            if sector == (SHELL[n], 1, "even") and int(entry["label"]) <= 2:
                difference = float(entry["eps"]) - row[int(entry["label"])]
                worst_level = max(worst_level, abs(difference))
                compared += 1
```

For each of the ten states the loop opens the record's level table (for example `N688_lam0_a00.csv`, one row per level with the columns `n2`, `j`, `parity`, `label`, `eps`, ...), picks the rows of the Fermi-shell sector with $j = +1$ and even parity and a label of at most 2, and compares the recorded level with the one just computed; `worst_level` keeps the largest difference and `compared` counts the comparisons.

```
a_f, b_f = orbitals(eps_fermi[:, :2].ravel(), np.repeat(q_fermi, 2))
worst_q, same_pair, q_free = 0.0, True, {}
say("   N  a4,0        q      eps_0      eps_1   Q (here)   Q_max (record)")
```

`eps_fermi[:, :2]` takes the labels 0 and 1 of every row and `.ravel()` lines them up into one array of 20 levels; `orbitals` returns their 20 orbitals, two consecutive rows per state. The last line prints the header of the table of Out [5].

```
for i, (n, a4) in enumerate(fermi):
    matrix = couplings(q_fermi[i], a_f[2 * i:2 * i + 2], b_f[2 * i:2 * i + 2])
    gap = eps_fermi[i, 1] - eps_fermi[i, 0]
```

```
    q_free[(n, a4)] = abs(matrix[0, 1]) / gap ** 2  # Q at the rate A = 1
    row = record[state_id(n, a4)]
    for value, column in ((q_free[(n, a4)], "Q_max"), (gap, "Q_max_delta_eps"),
                          (abs(matrix[0, 1]), "Q_max_matrix_element")):
        worst_q = max(worst_q, abs(value / float(row[column]) - 1.0))
```

`enumerate` numbers the states $i = 0, \dots, 9$. For each one, `couplings` gives the 2×2 matrix of the band orbital and the first bulk orbital (rows $2i$ and $2i+1$), `gap` is $\varepsilon_1 - \varepsilon_0$, and $Q = |\langle 0|\partial_a h|1\rangle|/(\varepsilon_1 - \varepsilon_0)^2$ at the rate $A = 1$ (with $H = 1$). The inner loop compares three numbers with the record's three columns, Q, the energy difference and the matrix element, as relative differences, and keeps the largest.

```
    pair = f"{SHELL[n]}:+1:even:0 -> {SHELL[n]}:+1:even:1"
    same_pair = same_pair and row["Q_max_pair"] == pair
    recorded = float(row["Q_max"])
    say(f"{n:4d}  {a4:4.1f}  {q_fermi[i]:7.4f}  {eps_fermi[i, 0]:9.6f}"
        f"  {eps_fermi[i, 1]:9.6f}  {q_free[(n, a4)]:.7f}  {recorded:.7f}")
```

The record names the pair of levels of its largest Q in the column `Q_max_pair`, written as shell, block type, parity and label of both levels; `same_pair` stays true only if every state names the band level and the first bulk level of its Fermi shell. The last statement prints one row of the table: the formats `4d`, `4.1f`, `7.4f`, `9.6f` and `.7f` give whole numbers of width 4 and numbers with 1, 4, 6 and 7 digits after the point.

```
report("levels compared with the record", compared)
report("largest |eps(here) - eps(record)|", f"{worst_level:.1e}")
report("largest relative difference of Q, delta eps, matrix element", f"{worst_q:.1e}")
check(compared == 30 and worst_level < 1e-12,
      "the 30 levels of the Fermi-shell sectors equal the record's within 1e-12",
      record=f"{LEVELS}/N136_lam0_a*.csv and N688_lam0_a*.csv, column eps")
check(worst_q < 1e-9,
      "Q_max, its energy difference and matrix element reproduced within 1e-9",
      record=f"{ADIABATIC}, columns Q_max, Q_max_delta_eps, Q_max_matrix_element; "
             "computed at the slice 0 with q = k e^(-a4,0), the rescaling identity "
             "of Revision/kohn_sham/reports/ks-rust-solver.json, check "
             "free_rescaling_relation_and_band_monotone")
check(same_pair,
      "the recorded pair is band level -> first bulk level of the Fermi shell",
      record=f"{ADIABATIC}, column Q_max_pair")
```

Out [5] shows the table and the results: 30 levels compared, the largest difference 8.8×10^{-14}, and the largest relative difference of Q, of the energy difference and of the matrix element 1.3×10^{-13}. For example, for $N = 688$ at the slice 0 the band level is $\varepsilon_0 = 1.247113$, the first bulk level $\varepsilon_1 = 3.500518$, and $Q = 0.0934506$ both here and in the record. The three checks require the 30 levels within 10^{-12}, the three columns within 10^{-9} (relative), and the recorded pair. Because every level was computed at the slice 0 with the momentum $ke^{-a_{4,0}}$, the agreement at all five slices is at the same time a test of the rescaling identity, which the second check names.

In [6], one curve for every slice and every shell.

```
Q_GRID = np.geomspace(0.02, 6.0, 61)  # redshifted momenta q (units of m)
CURVE = np.arange(-3, 5)  # the labels -3 ... 4 of the sector
BAND = 3  # the position of label 0 in CURVE
q_rows = np.repeat(Q_GRID, len(CURVE))  # one row per (q, label)
curve_eps = levels(np.tile(CURVE, len(Q_GRID)), q_rows)
curve_a, curve_b = orbitals(curve_eps, q_rows)
curve_eps = curve_eps.reshape(len(Q_GRID), len(CURVE))
```

By exact rule 2 the free Q/A of a sector is one function $G(q)$. The cell computes it at 61 momenta from 0.02 to 6, equally spaced on a logarithmic scale. `CURVE` holds the eight labels $-3, -2, \ldots, 4$, and label 0 sits at position 3 of this array (`BAND`). The $61 \times 8 = 488$ levels and orbitals are computed in one call each and the levels are arranged as 61 rows of 8.

```
G = np.array([per_unit_rate(curve_eps[i], couplings(
    Q_GRID[i], curve_a[8 * i:8 * i + 8], curve_b[8 * i:8 * i + 8]))
    for i in range(len(Q_GRID))])  # G[i, n, m]: Q per unit rate at Q_GRID[i]
del curve_a, curve_b  # the orbitals are no longer needed
```

For each momentum the eight orbitals give an 8×8 coupling matrix and from it the table G_{nm}; `G` stacks the 61 tables into one array with three indices: the momentum, n and m. `del` deletes the orbitals (488 rows of 1801 numbers each) to free memory.

```
g01 = G[:, BAND, BAND + 1]  # band level -> first bulk level
top = int(np.argmax(g01))
fine_q = np.linspace(Q_GRID[top - 1], Q_GRID[top + 1], 41)  # around the top
fine_eps = levels(np.tile([0, 1], 41), np.repeat(fine_q, 2)).reshape(41, 2)
fine_a, fine_b = orbitals(fine_eps.ravel(), np.repeat(fine_q, 2))
fine_g = np.array([per_unit_rate(fine_eps[i], couplings(
    fine_q[i], fine_a[2 * i:2 * i + 2], fine_b[2 * i:2 * i + 2]))[0, 1]
    for i in range(41)])
g_star, q_star = float(fine_g.max()), float(fine_q[int(np.argmax(fine_g))])
```

`g01` is the curve of the band jump, $G_{01}(q)$. `np.argmax` gives the position of its largest value on the coarse grid. Around it, between the two neighbouring grid points, `np.linspace` makes 41 equally spaced momenta, and the labels 0 and 1 are solved again there; `fine_g` is G_{01} on this fine grid, and its largest value G^* and the momentum q^* where it is reached are kept.

```
report("largest Q per unit rate of the band jump, G*", f"{g_star:.5f}")
report("at the redshifted momentum q*", f"{q_star:.3f}")
report("naive breakdown rate of the band level, 0.02 <= q <= 6, 1/G*",
       f"{1.0 / g_star:.3f}")
report("largest G of the jumps band -> bulk 2, 3, 4 on the grid",
       ", ".join(f"{G[:, BAND, BAND + m].max():.5f}" for m in (2, 3, 4)))
```

Out [6]: $G^* = 0.09979$ at $q^* = 2.128$, and $1/G^* = 10.021$. The last line prints, for the jumps from the band level to the second, third and fourth bulk levels, the largest value on the 61 momenta of the grid: `G[:, BAND, BAND + m]` is the column of the jump to label m, `.max()` its largest entry, and `", ".join(...)` writes the three numbers separated by commas; Out [6] shows 0.00640, 0.00249 and

0.00087. What this establishes, and only this: every shell at every slice whose redshifted momentum lies between 0.02 and 6 sits somewhere on the one curve, and that includes every state of the record (their q lie between $0.5e^{-2} = 0.068$ and 0.829). For all of them the jump from the band level to the first bulk level never has Q/A above G^*, the jumps to the second, third and fourth bulk levels stay below 0.0065, 0.0025 and 0.0009, and so the naive breakdown rate of the band level is at least $1/G^* = 10.021$ (COMPUTED here, free field, for the labels up to 4). Beyond $q = 6$ the curve was not computed, and nothing is claimed there.

In [7], drawing the one curve (Figure 22a.2).

```
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(Q_GRID, g01, color="black", lw=2.0, label="$G_{01}$: band -> 1st bulk")
ax.plot(Q_GRID, G[:, BAND, BAND + 2], color="0.45", lw=1.5, ls="--",
        label="$G_{02}$: band -> 2nd bulk")
q_low, q_high = float(q_fermi.min()), float(q_fermi.max())
ax.axvspan(q_low, q_high, color="0.85", alpha=0.6, lw=0)  # the recorded range
```

One panel, 7.5 by 4.5 inches. The black line is G_{01}, the grey dashed line G_{02} (band level to second bulk level). `axvspan` shades the range of momenta that the record's Fermi shells cover, from the smallest to the largest of the ten values of `q_fermi`.

```
for colour, marker, n in ((PALETTE[0], "o", 136), (PALETTE[1], "s", 688)):
    for tag in TAGS:
        xs = [0.25 * math.sqrt(SHELL[n]) * math.exp(-a4) for a4 in SLICES]
        ys = [q_max[(n, tag, a4)] for a4 in SLICES]
        ax.plot(xs, ys, marker, color=colour, ms=8, mfc="none", mew=1.5,
                label=f"record, $N = {n}$" if tag == "lam0" else None)
ax.plot([q_star], [g_star], "*", color=PALETTE[3], ms=16, mec="black",
        label=f"top: $G^* = {g_star:.4f}$ at $q = {q_star:.2f}$")
```

For both particle numbers and all five couplings the record's Q_{max} (at $A = 1$, so equal to Q/A) is drawn as open markers (`mfc="none"`: no fill; `mew`: the width of the marker's edge) at the redshifted momentum of its Fermi shell; only the free states get a legend entry (the expression `... if tag == "lam0" else None` gives the label or nothing). A yellow star with a black edge marks the top of the curve.

```
ax.set_xscale("log")
ax.set_xlabel("redshifted momentum $q = k\\,e^{-a_{4,0}}$ (units of $m$)")
ax.set_ylabel("$G = Q/A$ (pure number)")
ax.set_title("Every slice and every shell on one curve ($\\lambda = 0$)")
ax.legend(fontsize=8, loc="upper left")
save_figure(fig, "one_curve",
            "The adiabaticity measure per unit rate, $G = Q/A$ (vertical axis, pure "
            "number), of the jump from the band level to the first (black) and "
            "second (grey, dashed) bulk level of a free sector, against the "
            "redshifted momentum $q = k e^{-a_{4,0}}$ (horizontal axis, "
            "logarithmic, units of $m$). Open symbols: the recorded $Q_{max}$ of "
            "the 50 states with $N = 136$ (circles) and $N = 688$ (squares), each "
            "at the $q$ of its Fermi shell; the five couplings overlap. The grey "
            "band is the range of the record; the star is the top of the curve, "
```

```
            "the largest $Q/A$ of the jump to the first bulk level for every "
            "shell and slice with $0.02 \\le q \\le 6$ (all states of the record).")
```

A logarithmic horizontal axis, the labels, the title, the legend in the upper left corner, and `save_figure` saves Figure 22a.2. In the caption, `\\le` gives matplotlib's and the book's sign $\leq$ (less than or equal); the doubled backslash is needed in a normal Python string, as explained for In [3].

```
spread = max(abs(q_max[(n, tag, a4)] / q_max[(n, "lam0", a4)] - 1.0)
             for n in (136, 688) for tag in TAGS for a4 in SLICES)
report("largest relative change of Q_max by the interaction", f"{spread:.4f}")
check(largest < g_star < 0.1,
      "the top of the curve lies above every recorded Q_max and below 0.1")
check(spread < 0.15, "the interaction changes Q_max by less than 15 percent",
      record=f"{ADIABATIC}, column Q_max, all couplings")
```

`spread` is the largest relative difference between the Q_{max} of an interacting state and that of the free state with the same particle number and slice: 0.0980, that is, 9.8 percent (Out [7]). The first check requires $Q_{max} < G^* < 0.1$ for all recorded states (Python allows the chain `a < b < c`); the second that the interaction changes Q_{max} by less than 15 percent.

What the student should see in Figure 22a.2, and why. The black curve rises from about 0.011 at $q = 0.02$ to its top 0.0998 at $q = 2.13$ and falls slowly beyond. All ten free states of the record lie exactly on it, at five different slices and two different shells: this is exact rule 2 at work, the deflating history moves each shell along the one curve towards smaller q. The interacting states lie close to the curve; the largest scatter is at the smallest q (the shell of $N = 136$ at the slice 2), where the interaction changes Q by up to ten percent. The whole recorded range (grey) lies left of the top, which is why Q falls along the history in Figure 22a.1. The dashed curve of the second bulk level stays below 0.007 and touches zero near $q = 1.1$, where its matrix element passes through zero: the band level couples mainly to its nearest neighbour.

In [8], jumps across the gap into the negative branch (Figure 22a.3).

```
def g_pair(n, m):
    """G of the jump from label n to label m along Q_GRID."""
    return G[:, BAND + n, BAND + m]


sea = np.max([g_pair(n, m) for n in (-3, -2, -1) for m in (1, 2, 3, 4)], axis=0)
sea_top = int(np.argmax(sea))
```

The record's Q counts only jumps between levels of positive energy, because its filling convention treats the negative branch as the normal-ordered sea, which holds no particles. But the moving background couples the two branches through the same matrix elements, and the table `G` already contains them. `g_pair(n, m)` returns the curve of the jump from label n to label m (the label is shifted by `BAND` to its position in the array). `sea` is, at each momentum, the largest of the twelve jumps from a negative level (-3, -2, -1) to a bulk level (1 to 4), and `sea_top` the position of its maximum.

```
fig, ax = plt.subplots(figsize=(7.5, 4.5))
ax.plot(Q_GRID, g01, color="black", lw=2.0, label="band 0 -> bulk 1 (record's Q)")
```

```
for colour, style, (n, m) in zip(PALETTE, ("-", "--", "-.", ":"),
                                 ((-1, 1), (-1, 2), (-2, 1), (-1, 0))):
    kind = "band" if m == 0 else "bulk"
    ax.plot(Q_GRID, g_pair(n, m), color=colour, lw=2.0, ls=style,
            label=f"negative {n} -> {kind} {m}")
```

The black band jump of In [7] for comparison, and four jumps across the gap, each with its own colour and line style (solid, dashed, dash-dotted, dotted): from -1 to the bulk levels 1 and 2, from -2 to bulk 1, and from -1 to the band level 0.

```
ax.set_xscale("log")
ax.set_xlabel("redshifted momentum $q = k\\,e^{-a_{4,0}}$ (units of $m$)")
ax.set_ylabel("$G = Q/A$ (pure number)")
ax.set_title("Jumps across the gap between the two branches ($\\lambda = 0$)")
ax.legend(fontsize=8)
save_figure(fig, "across_the_gap",
            "The adiabaticity measure per unit rate $G = Q/A$ (vertical axis) of "
            "jumps from the negative levels $-1$ and $-2$ of a free sector to the "
            "bulk levels 1 and 2 and to the band level 0 (coloured), compared "
            "with the band jump of the record (black), against the redshifted "
            "momentum $q$ (horizontal axis, logarithmic, units of $m$). The jumps "
            "across the gap stay below 0.05 per unit rate; the strongest of them "
            "($-1 \\to$ bulk 1) peaks near $q = 0.11$ at 0.049 and exceeds the "
            "band jump only below $q$ of about 0.1; their reading as pair "
            "creation of the field is OPEN.")
report("largest G of a jump negative level -> bulk level", f"{sea[sea_top]:.5f}")
report("at the redshifted momentum", f"{Q_GRID[sea_top]:.4f}")
check(sea[sea_top] < 0.5 * g_star,
      "every jump across the gap is weaker than half the top of the band jump")
```

The axes, the title, the legend and Figure 22a.3; in its caption `\\to` gives the arrow $\to$. Out [8] gives the strongest jump from a negative to a bulk level, $G = 0.04887$ at $q = 0.1107$, and the check requires it to be below half of G^*.

What the student should see in Figure 22a.3, and why. The strongest jump across the gap, from -1 to bulk 1 (blue), rises to its peak 0.0489 near $q = 0.11$ and falls on both sides; below about $q = 0.1$ it is stronger than the band jump of the record, above it is weaker, and at large q it is far weaker. The other jumps across the gap stay below 0.025; the jumps from -1 to the band level and from -2 to bulk 1 pass through zero near $q = 0.15$. So in the free field the moving background does couple the negative branch to the positive one, at a strength comparable to the band jump at small momenta. In ordinary Dirac theory a jump from a filled negative level to an empty positive one is the creation of a particle-antiparticle pair OF THE FIELD, inside one universe. Whether that reading holds in the 4+4 quantisation with its Krein metric is OPEN, and it has nothing to do with the creation of universes.

In [9], the Fermi-shell sector along the recorded span.

```
K11 = 0.25 * math.sqrt(11.0)  # the Fermi-shell momentum of N = 688 (units of m)
END = 2.0  # the recorded span of slices, a4,0 = 0 ... 2
NODES = 56  # Chebyshev nodes on the span
BASIS = np.arange(-9, 11)  # 20 labels: nine negative, the band level, ten bulk
SIZE, ZERO = len(BASIS), 9  # ZERO: the position of label 0 (the band level)
```

For the time evolution the levels and the matrix K_{mn} are needed at every instant, not only at five slices. The cell takes the Fermi-shell sector of $N = 688$ ($n_2 = 11$, $k = 0.829$, $j = +1$, even parity; its band level holds 96 quanta, the degeneracy $4 \cdot 24$ of the shell in the record's level table), the span $a_{4,0} = 0$ to 2, 56 interpolation nodes, and a basis of 20 labels, -9 to 10; label 0 is at position 9.

```
angles = np.pi * (np.arange(NODES) + 0.5) / NODES  # theta_j of the nodes
nodes_s = 0.5 * END * (1.0 + np.cos(angles))  # the nodes as slices a4,0
q_nodes = np.repeat(K11 * np.exp(-nodes_s), SIZE)  # one row per (node, label)
node_eps = levels(np.tile(BASIS, NODES), q_nodes)
node_a, node_b = orbitals(node_eps, q_nodes)
node_eps = node_eps.reshape(NODES, SIZE)
```

The **Chebyshev nodes** are the slices $s_j = 1 + \cos\theta_j$ with $\theta_j = \pi(j + \frac{1}{2})/56$, $j = 0, \ldots, 55$ (`0.5 * END` is 1). They crowd towards the two ends of the span, which is what makes polynomial interpolation through them accurate. At each node the 20 levels and orbitals are solved: $56 \cdot 20 = 1120$ levels in one call, each at the slice 0 with the momentum ke^{-s_j}.

```
node_m = np.array([couplings(K11 * math.exp(-s), node_a[SIZE * i:SIZE * (i + 1)],
                             node_b[SIZE * i:SIZE * (i + 1)])
                   for i, s in enumerate(nodes_s)])
del node_a, node_b  # the orbitals are no longer needed
```

At each node the 20×20 matrix $\langle m|\partial_a h|n\rangle$ of the 20 orbitals of that node; `node_m` stacks the 56 matrices. The orbitals are then deleted.

```
def chebyshev_coefficients(values):
    """c_k = (2/NODES) sum_j values_j cos(k theta_j), c_0 halved: the polynomial
    through the values at the nodes (one column per function)."""
    table = np.cos(np.outer(np.arange(NODES), angles))  # cos(k theta_j)
    coefficients = 2.0 / NODES * table @ values.reshape(NODES, -1)
    coefficients[0] *= 0.5
    return coefficients
```

A function f on the span is approximated by the polynomial $\sum_{k=0}^{55} c_k T_k(x)$, where $x = s - 1$ runs from -1 to 1 and T_k is the Chebyshev polynomial of degree k, defined by $T_k(\cos\theta) = \cos(k\theta)$. The coefficients that make the polynomial pass exactly through the values at the 56 nodes are $c_k = \frac{2}{56}\sum_j f(s_j)\cos(k\theta_j)$, with c_0 halved. `np.outer` makes the table of all products $k\theta_j$, and the matrix product with the values computes all sums at once; `values.reshape(NODES, -1)` arranges any array of values as 56 rows (one per node) and as many columns as there are functions (`-1` lets numpy count them).

```
FIT_EPS = chebyshev_coefficients(node_eps)  # 20 levels
FIT_M = chebyshev_coefficients(node_m)  # 400 matrix elements


def interpolated(s):
    """Levels (one row per point s) and coupling matrices from the fits."""
    x = 2.0 * np.asarray(s, dtype=float) / END - 1.0  # a4,0 = 0 ... 2 -> -1 ... 1
```

```
    eps = chebyshev.chebval(x, FIT_EPS).T
    matrix = chebyshev.chebval(x, FIT_M).T.reshape(len(x), SIZE, SIZE)
    return eps, matrix
```

Coefficients for the 20 levels and for the $20 \cdot 20 = 400$ matrix elements. `interpolated` maps the slices to $x = 2s/2 - 1$, evaluates all the polynomials there with `chebval`, which returns one row per function and one column per point, transposes, and arranges the matrix elements again as 20×20 matrices, one per point.

```
test_s = np.array([0.03, 0.25, 0.77, 1.31, 1.97])  # five points between the nodes
test_q = np.repeat(K11 * np.exp(-test_s), SIZE)
test_eps = levels(np.tile(BASIS, 5), test_q)
test_a, test_b = orbitals(test_eps, test_q)
fit_eps, fit_m = interpolated(test_s)
error_eps = float(np.max(np.abs(fit_eps - test_eps.reshape(5, SIZE))))
error_m = max(float(np.max(np.abs(fit_m[i] - couplings(
    K11 * math.exp(-s), test_a[SIZE * i:SIZE * (i + 1)],
    test_b[SIZE * i:SIZE * (i + 1)])))) for i, s in enumerate(test_s))
```

The test of the interpolation: at five slices that are not nodes, the levels and matrices are solved directly and compared with the interpolated ones; `error_eps` and `error_m` are the largest differences.

```
slice_eps, slice_m = interpolated(SLICES)
slice_q = [abs(slice_m[i, ZERO, ZERO + 1]) / (slice_eps[i, ZERO + 1]
           - slice_eps[i, ZERO]) ** 2 for i in range(5)]
worst_slice = max(abs(slice_q[i] / q_max[(688, "lam0", a4)] - 1.0)
                  for i, a4 in enumerate(SLICES))
```

A second test: Q of the band jump computed from the interpolated levels and matrices at the five slices of the record, compared with the record's Q_{max} of the free state $N = 688$.

```
report("largest interpolation error of a level", f"{error_eps:.1e}")
report("largest interpolation error of a matrix element", f"{error_m:.1e}")
report("largest relative difference of the interpolated Q to the record",
       f"{worst_slice:.1e}")
check(error_eps < 1e-9 and error_m < 1e-7,
      "the Chebyshev fits reproduce levels within 1e-9 and couplings within 1e-7")
check(worst_slice < 1e-8, "the interpolated Q at the five slices equals the record's",
      record=f"{ADIABATIC}, column Q_max of N688_lam0_a00 ... a20")
```

Out [9]: the interpolation reproduces the levels to 7.3×10^{-11} and the matrix elements to 6.4×10^{-9}, and the interpolated Q at the five slices agrees with the record to 1.4×10^{-13} (relative). The two checks require 10^{-9}, 10^{-7} and 10^{-8}.

In [10], the levels of the sector and how fast the band orbital turns (Figure 22a.4).

```
s_plot = np.linspace(0.0, END, 201)
eps_plot, m_plot = interpolated(s_plot)
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.4), layout="constrained")
```

```
for i, label in enumerate(BASIS):
    if label == 0:
        left.plot(s_plot, eps_plot[:, i], color=PALETTE[3], lw=3.0,
                  label="band level 0 (occupied)")
    else:
        colour = PALETTE[0] if label > 0 else PALETTE[1]
        left.plot(s_plot, eps_plot[:, i], color=colour, lw=1.0)
```

201 slices across the span, the interpolated levels and matrices there, and two panels. On the left every one of the 20 levels is drawn against the slice: the band level thick and yellow, the bulk levels blue and the negative levels orange.

```
left.plot([], [], color=PALETTE[0], lw=1.0, label="bulk levels 1 ... 10")
left.plot([], [], color=PALETTE[1], lw=1.0, label="negative levels -1 ... -9")
left.set_xlabel("slice $a_{4,0}$")
left.set_ylabel("level $\\varepsilon$ (units of $m$)")
left.set_title("Sector $n_2 = 11$, $j = +1$, even, of $N = 688$")
left.legend(fontsize=8, loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=3)
```

Two empty plots (`[]`, no points) create one legend entry each for the nineteen thin lines; the legend is placed below the panel in three columns (`bbox_to_anchor` gives its position relative to the panel).

```
for colour, style, label in zip(PALETTE, ("-", "--", "-.", ":"), (1, 2, -1, -2)):
    m = ZERO + label
    turn = np.abs(m_plot[:, m, ZERO]) / np.abs(eps_plot[:, ZERO] - eps_plot[:, m])
    right.plot(s_plot, turn, color=colour, ls=style, lw=2.0, label=f"$m = {label}$")
right.set_xlabel("slice $a_{4,0}$")
right.set_ylabel("$|K_{m0}|$ (per e-fold)")
right.set_title("How fast the band orbital turns towards level $m$")
right.legend(fontsize=8)
```

On the right, for the levels $m = 1, 2, -1, -2$, the number $|K_{m0}| = |\langle m|\partial_a h|0\rangle|/|\varepsilon_0 - \varepsilon_m|$ of line 6 of Section 22.6. It says how far the band orbital turns towards level m per unit change of a_4, that is, per e-fold of the extra times.

```
save_figure(fig, "sector_levels",
            "Left: the 20 instantaneous levels (vertical axis, units of $m$) of the "
            "free Fermi-shell sector of $N = 688$ against the slice $a_{4,0}$ "
            "(horizontal axis): the occupied band level (thick) falls as the 3-"
            "momentum redshifts, the bulk levels above it and the negative levels "
            "below it move towards their values at $k = 0$. Right: $|K_{m0}|$, how "
            "far the band orbital turns towards level $m$ per e-fold (vertical "
            "axis), for $m = 1, 2, -1, -2$; all are below 0.25, so the band orbital "
            "changes slowly.")
```

`save_figure` saves Figure 22a.4. This cell prints nothing else.

What the student should see in Figure 22a.4, and why. On the left the band level falls from 1.247 at the slice 0 to 0.206 at the slice 2 (the values of Out [5]): as q redshifts towards 0 it approaches

the zero mode, the level $\varepsilon = 0$ of the brane-bound orbital at $k = 0$. The bulk levels fall and the negative levels rise, all towards their values at $k = 0$; the gap between the band level and the first bulk level stays above 1.6 (at the slice 2 it is $1.823448 - 0.205760 = 1.617688$, Out [5]). On the right $|K_{10}|$ falls from 0.21 to about 0.09 along the span; the turning towards the negative level -1 starts at about 0.08 and passes through zero near the slice 1.8; the others stay below about 0.035. All are well below 1: the band orbital changes slowly per e-fold, which is why the instantaneous picture works as well as it does.

In [11], the exact time evolution in the basis of instantaneous levels.

```
U_STEPS = 4000  # the finest number of RK4 steps of one passage, u = 0 ... 1
U = np.linspace(0.0, 1.0, 2 * U_STEPS + 1)  # the fine grid: step ends, midpoints
CHOICES = [200, 400, 500, 1000, 2000, 4000]  # step numbers that divide U_STEPS
OFF = ~np.eye(SIZE, dtype=bool)  # every pair n != m
SPAN = float(np.ptp(node_eps))  # the largest energy difference of the basis
```

The passage is integrated in the variable $u = x_4/T$ of Section 22.8, from 0 to 1, with RK4. The finest integration uses 4000 steps; `U` is its fine grid of 8001 points (step ends and midpoints), on which the levels and matrices are tabulated once. A coarser integration may use 200, 400, 500, 1000 or 2000 steps; these numbers divide 4000, so their step ends and midpoints are points of the same fine grid. `np.eye(SIZE, dtype=bool)` is the 20×20 table that is true on the diagonal, and `~` turns true into false and back, so `OFF` selects the pairs $m \neq n$. `np.ptp` ("peak to peak") is the largest value minus the smallest: `SPAN` is the largest difference between any two levels of the basis at any node, 24.89 (Out [11]).

```
def frame(s):
    """Levels e[i, n] and the matrix k[i, m, n] = <m| d_a n> at the slices s[i]."""
    e, matrix = interpolated(s)
    gap = e[:, None, :] - e[:, :, None]  # gap[i, m, n] = eps_n - eps_m
    k = np.zeros_like(matrix)
    k[:, OFF] = matrix[:, OFF] / gap[:, OFF]  # line 6 of section 4; K_nn = 0
    return e, k
```

`frame` gives, at a list of slices, the levels and the matrices $K_{mn} = \langle m|\partial_a h|n\rangle/(\varepsilon_n - \varepsilon_m)$ (line 6 of Section 22.6; the comment refers to section 4 of the notebook, which derives it), with $K_{nn} = 0$ (line 7). The array `gap` holds $\varepsilon_n - \varepsilon_m$ in position $[i, m, n]$, which the two `None` arrange by broadcasting; only the off-diagonal entries are divided.

```
def steps_needed(duration):
    """The smallest step number of CHOICES with duration * SPAN / steps <= 0.2."""
    for steps in CHOICES:
        if duration * SPAN / steps <= 0.2:
            return steps
    return CHOICES[-1]
```

In the variable u the amplitudes turn with the frequencies $T\varepsilon_m$, so in one step of length $1/N$ a phase changes by up to about $T \cdot \mathrm{SPAN}/N$. `steps_needed` returns the smallest step number of `CHOICES` that keeps this below 0.2 radian, and the finest one when none does.

```
def passage(duration, e, k, rate, start, steps, keep=0):
    """RK4 for dc/du = -i duration e(u) c - rate(u) k(u) c from u = 0 to 1.
    e, k and rate are given on the fine grid U. Returns the final amplitudes and,
    if keep > 0, the list of the amplitudes after every keep steps."""
    stride = U_STEPS // steps  # fine-grid points per half step
    h = 1.0 / steps
    c = start.astype(complex)
    kept = [c]
```

`passage` integrates the equation $dc/du = -iT\varepsilon(u)c - (da_4/du)K(u)c$ of Section 22.8 for the vector c of the 20 amplitudes. The arguments are the duration T, the levels `e` and matrices `k` on the fine grid, the rate da_4/du on the fine grid, the starting amplitudes, the number of steps, and how often to keep the amplitudes. A step of length $h = 1/N$ spans $2 \cdot 4000/N$ intervals of the fine grid, so half a step spans `stride` $= 4000/N$ of them (`//` is division of whole numbers). `astype(complex)` makes the amplitudes complex numbers.

```
    def slope(i, c):
        return -1j * duration * e[i] * c - rate[i] * (k[i] @ c)

    for step in range(steps):
        i = 2 * stride * step  # the start of this step on the fine grid
        s1 = slope(i, c)
        s2 = slope(i + stride, c + 0.5 * h * s1)
        s3 = slope(i + stride, c + 0.5 * h * s2)
        s4 = slope(i + 2 * stride, c + h * s3)
        c = c + h / 6.0 * (s1 + 2.0 * s2 + 2.0 * s3 + s4)
        if keep and (step + 1) % keep == 0:
            kept.append(c)
    return c, kept
```

`slope(i, c)` is the right side of the equation at the fine-grid point i; `1j` is Python's i, and `e[i] * c` multiplies each amplitude by its own level. Each step is the RK4 rule of In [4], with the slopes at the start (fine point i), twice at the middle ($i+$ `stride`) and at the end ($i + 2$ `stride`). After every `keep` steps the amplitudes are stored (`%` is the remainder of a division). The function returns the final amplitudes and the stored ones.

```
S_SMOOTH = END * (U - np.sin(2.0 * np.pi * U) / (2.0 * np.pi))  # a4 along it
RATE_SMOOTH = 2.0 * END * np.sin(np.pi * U) ** 2  # its derivative d a4 / d u
E_SMOOTH, K_SMOOTH = frame(S_SMOOTH)
S_LINEAR = END * U  # the constant rate: a4 = END u
RATE_LINEAR = np.full_like(U, END)
E_LINEAR, K_LINEAR = frame(S_LINEAR)
```

The two histories of Section 22.8 on the fine grid: the smooth passage $a_4 = 2[u - \sin(2\pi u)/(2\pi)]$ with $da_4/du = 4\sin^2(\pi u)$, and the constant rate $a_4 = 2u$ with $da_4/du = 2$; for each, `frame` tabulates the levels and the matrices K at all 8001 points.

```
START = np.zeros(SIZE, dtype=complex)
START[ZERO] = 1.0  # the particle starts in the band level
report("largest energy difference of the basis", f"{SPAN:.4f}", "m")
```

The starting amplitudes: 1 for the band level, 0 for the 19 others. The report prints `SPAN` with its unit m.

In [12], a smooth passage at every rate.

```
RATES = 10.0 ** np.linspace(-0.6, 2.4, 31)  # peak rates A from 0.25 to 251
excited, norm_error = [], 0.0
for rate in RATES:
    duration = 2.0 * END / rate  # T = 4/A: the peak rate is A
    c, _ = passage(duration, E_SMOOTH, K_SMOOTH, RATE_SMOOTH, START,
                   steps_needed(duration))
    excited.append(1.0 - abs(c[ZERO]) ** 2)  # probability to have left level 0
    norm_error = max(norm_error, abs(float(np.vdot(c, c).real) - 1.0))
excited = np.array(excited)
```

31 peak rates $A = 10^{-0.6}, \ldots, 10^{2.4}$, that is from 0.251 to 251, ten per factor 10; the seventh is 1 (up to rounding in the last digit: the computer stores 1.0000000000000002) and the eleventh $10^{0.4} = 2.51$. For each the passage lasts $T = 4/A$ (Section 22.8), and the probability that the particle has left the band level at the end is $P = 1 - |c_0|^2$. It counts every other level of the basis, the negative ones included. `np.vdot(c, c)` is $\sum_n |c_n|^2$, the total probability, and `norm_error` keeps its largest distance from 1. The underscore `_` receives the stored amplitudes, which are not needed here.

```
c, _ = passage(0.0, E_SMOOTH, K_SMOOTH, RATE_SMOOTH, START, 400)  # infinitely fast
sudden_basis = 1.0 - abs(c[ZERO]) ** 2
q_ends = np.array([K11, K11 * math.exp(-END)])  # the band level at a4,0 = 0 and 2
ends_a, ends_b = orbitals(levels([0, 0], q_ends), q_ends)
overlap = float(np.sum(SIMPSON * (ends_a[0] * ends_a[1] + ends_b[0] * ends_b[1])))
sudden = 1.0 - overlap ** 2  # the sudden limit from the two orbitals directly
```

The sudden limit, twice. First in the basis: with the duration $T = 0$ the term $-iT\varepsilon c$ vanishes and only the transport by K remains, which is the infinitely fast passage of Section 22.8. Second directly: the band orbitals at the slices 0 and 2 (momenta k and ke^{-2}), their overlap $\int (a_0 a_2 + b_0 b_2)\, dy$ by Simpson's rule, and $P_{sudden} = 1 - \text{overlap}^2$.

```
du = 1.0 / (2 * U_STEPS)  # the spacing of the fine grid U
gaps = E_SMOOTH - E_SMOOTH[:, [ZERO]]  # eps_m - eps_0 along the passage
phase = np.vstack([np.zeros((1, SIZE)), np.cumsum(0.5 * (gaps[1:] + gaps[:-1]) * du,
                                                  axis=0)])  # trapezoid rule
weights = np.full(len(U), 2.0 * du / 3.0)  # Simpson weights on U
weights[1::2] = 4.0 * du / 3.0
weights[0] = weights[-1] = du / 3.0
drive = (weights * RATE_SMOOTH)[:, None] * K_SMOOTH[:, :, ZERO]  # da4/du K_m0
first = np.array([float(np.sum(np.abs(np.sum(drive * np.exp(
    1j * (2.0 * END / rate) * phase), axis=0)) ** 2)) for rate in RATES])
```

The first-order estimate $P^{(1)} = \sum_m |d_m(1)|^2$ of Section 22.8. `gaps` holds $\varepsilon_m - \varepsilon_0$ at every fine point (`[:, [ZERO]]` keeps the column as a column, so the subtraction works row by row). `phase` is $\int_0^u (\varepsilon_m - \varepsilon_0)\, du'$ at every fine point, built by the **trapezoid rule**: each interval adds its length times the mean of the values at its two ends, and `np.cumsum` adds these up one after the other

(`np.vstack` puts the starting row of zeros on top). `weights` are Simpson's weights for the 8000 intervals of length `du` of the grid `U` (here written for pairs of intervals: $du/3$, $4du/3$, $2du/3$). `drive` holds $(da_4/du)K_{m0}$ times the weights, one column per level m. For each rate the integral $d_m(1)$ is the weighted sum of `drive` times $e^{iT\cdot\text{phase}}$ with $T = 4/A$; the absolute squares are added over m. The column $m = 0$ contributes nothing, because $K_{00} = 0$.

```
above = int(np.argmax(excited >= 0.5 * sudden))  # first rate past half the limit
x0, x1 = math.log(RATES[above - 1]), math.log(RATES[above])
y0, y1 = excited[above - 1] - 0.5 * sudden, excited[above] - 0.5 * sudden
rate_half = math.exp(x0 - y0 * (x1 - x0) / (y1 - y0))  # straight line in log A
```

`excited >= 0.5 * sudden` is an array of true and false, and `np.argmax` of it is the position of the first true: the first rate at which P has reached half the sudden limit. Between that rate and the one before, the difference $P - P_{sudden}/2$ is replaced by a straight line in $\ln A$, and its zero is $A_{1/2}$: the line through (x_0, y_0) and (x_1, y_1) crosses zero at $x = x_0 - y_0(x_1 - x_0)/(y_1 - y_0)$, and $A_{1/2} = e^x$.

```
report("largest |total probability - 1|", f"{norm_error:.1e}")
report("sudden limit: overlap / basis", f"{sudden:.6f} / {sudden_basis:.6f}")
report("P at A = 0.25, 1, 2.51, 251",
       ", ".join(f"{excited[i]:.3e}" for i in (0, 6, 10, 30)))
report("rate at which P reaches half the sudden limit, A_1/2", f"{rate_half:.3f}")
check(norm_error < 1e-9, "the evolution keeps the total probability 1 within 1e-9")
check(abs(sudden_basis / sudden - 1.0) < 1e-3,
      "the sudden limit of the 20-level basis equals the direct overlap within 0.1%")
check(abs(excited[-1] / sudden - 1.0) < 1e-3 and excited[0] < 1e-6
      and bool(np.all(excited <= sudden * (1.0 + 1e-3))),
      "P rises from below 1e-6 at A = 0.25 to the sudden limit at A = 251, never above")
```

Out [12]: the total probability stays 1 to 1.2×10^{-11}; the sudden limit is 0.074906 from the overlap and 0.074902 in the basis; P is 1.04×10^{-7} at $A = 0.25$, 0.0138 at $A = 1$, 0.0590 at $A = 2.51$ and 0.0749 at $A = 251$; and $A_{1/2} = 1.508$. (`", ".join(...)` writes the four numbers separated by commas; the format `.3e` writes a number with three digits after the point and a power of ten.) The three checks require the conservation of probability, the agreement of the two sudden limits within 0.1 percent, and that P rises from below 10^{-6} to the sudden limit without ever exceeding it by more than 0.1 percent. That P never exceeds the sudden limit is a COMPUTED finding for this sector and this passage, not a theorem.

In [13], drawing the smooth passage (Figure 22a.5).

```
fig, ax = plt.subplots(figsize=(7.5, 4.6))
ax.plot(RATES, excited, "o-", color=PALETTE[0], ms=5, lw=2.0,
        label="exact evolution (20 levels)")
ax.plot(RATES, first, "--", color=PALETTE[1], lw=2.0, label="first order")
ax.axhline(sudden, color="0.3", ls=":", lw=1.5, label="sudden limit")
```

The exact P (blue, with markers), the first-order estimate (orange, dashed) and the sudden limit (dotted horizontal line).

```
for value, style, text, height in ((1.0, "--", "record", 2e-7),
                                   (rate_half, "-", "$A_{1/2}$", 2e-5),
                                   (naive[weakest], "-.", "$1/Q_{max}$", 2e-7)):
    ax.axvline(value, color="0.45", ls=style, lw=1.2)
    ax.text(value * 1.06, height, text, fontsize=9, color="0.3")
```

Three vertical lines with labels: the record's rate $A = 1$ (dashed), $A_{1/2}$ (solid) and the naive breakdown rate $1/Q_{max} = 10.70$ of In [3] (dash-dotted); each label is placed a little to the right of its line, at the given height.

```
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_ylim(1e-8, 0.3)
ax.set_xlabel("peak rate $A$ (e-folds per unit time $1/H$)")
ax.set_ylabel("$P$: probability to have left the band level")
ax.set_title("Smooth passage $a_{4,0} = 0 \\to 2$, free Fermi shell of $N = 688$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "smooth_passage",
            "The probability $P$ that the particle of the band level has left it "
            "at the end of a smooth passage from $a_{4,0} = 0$ to $2$ (vertical "
            "axis, logarithmic), against the peak rate $A$ (horizontal axis, "
            "logarithmic): exact evolution in the basis of 20 instantaneous levels "
            "(blue), the first-order estimate (orange, dashed) and the sudden limit "
            "(dotted). Vertical lines: the record's rate $A = 1$, the rate "
            "$A_{1/2}$ at which $P$ reaches half the sudden limit, and the naive "
            "breakdown rate $1/Q_{max}$. The gas stops following its level near "
            "$A = 1$, about seven times below the naive estimate ($A_{1/2} = 1.5$ "
            "against $1/Q_{max} = 10.7$), but $P$ never exceeds the sudden limit "
            "of about 7.5 percent.")
```

Both axes logarithmic, the vertical one from 10^{-8} to 0.3; labels, title and legend; Figure 22a.5.

What the student should see in Figure 22a.5, and why. At small rates P is tiny (10^{-7} at $A = 0.25$) and rises very steeply, by about five powers of ten up to $A = 1$: a slow passage leaves the particle in its level, and the fast-turning phases of Section 22.8 suppress the transitions the more strongly the slower the passage. Near $A = 1$, where $P = 0.0138$, the rise slows down, and from about $A = 5$ on the curve is nearly flat at the sudden limit 0.0749: there the passage is so fast that the orbital has no time to adapt at all. The first-order estimate follows the exact curve up to about $A = 1$ (with the same small wiggles, which come from the oscillating phases) and then overshoots the sudden limit, because at large rates the amplitudes are no longer small. The three vertical lines tell the story of the breakdown: the gas stops following its level near $A_{1/2} = 1.51$, about seven times below the naive estimate 10.7; but however fast the passage, at most about 7.5 percent of the particles leave the band level over the recorded span.

In [14], a constant rate: the dressing that Q measures (Figure 22a.6).

```
MARKS = np.arange(0, 2 * U_STEPS + 1, 2 * U_STEPS // 100)  # 101 points of U


def dressed(rate):
    """Level 0 with the first-order admixtures i A K_m0 / (eps_m - eps_0)."""
```

```
    e, k = E_LINEAR[0], K_LINEAR[0]
    c = START.copy()
    for m in range(SIZE):
        if m != ZERO:
            c[m] = 1j * rate * k[m, ZERO] / (e[m] - e[ZERO])
    return c / np.linalg.norm(c)
```

`MARKS` are 101 points of the fine grid, every 80th, at $u = 0, 0.01, \ldots, 1$. `dressed(rate)` builds the dressed state of Section 22.7 at the start of the constant-rate history: amplitude 1 for the band level and $\alpha_m = iAK_{m0}/(\varepsilon_m - \varepsilon_0)$ for every other level (with $H = 1$), divided by its length `np.linalg.norm(c)` $= \sqrt{\sum |c_n|^2}$ so that the total probability is 1.

```
def dressing(rate):
    """The first-order estimate sum_m Q_0m^2 at the 101 marks."""
    e, k = E_LINEAR[MARKS], K_LINEAR[MARKS]
    total = np.zeros(len(MARKS))
    for m in range(SIZE):
        if m != ZERO:  # Q_0m = A |K_m0| / |eps_m - eps_0|
            total += (rate * k[:, m, ZERO] / (e[:, m] - e[:, ZERO])) ** 2
    return total
```

`dressing(rate)` is the first-order dressing probability $\sum_{m \neq 0} Q_{0m}^2$ at the 101 marks, with $Q_{0m} = A|K_{m0}|/|\varepsilon_m - \varepsilon_0|$, which equals the formula of Section 22.7 by line 6. The sum runs over all 19 other levels of the basis, the negative ones included; the matrix K is real, so its square needs no absolute value.

```
runs = {}
for rate in (0.3, 1.0, 3.0):
    duration = END / rate  # T = 2/A for the constant rate A
    steps = steps_needed(duration)
    _, kept = passage(duration, E_LINEAR, K_LINEAR, RATE_LINEAR, dressed(rate),
                      steps, keep=steps // 100)
    runs[rate] = np.array([1.0 - abs(c[ZERO]) ** 2 for c in kept])
steps = steps_needed(END)
_, kept = passage(END, E_LINEAR, K_LINEAR, RATE_LINEAR, START, steps, keep=steps // 100)
bare = np.array([1.0 - abs(c[ZERO]) ** 2 for c in kept])
s_marks = S_LINEAR[MARKS]
```

For the constant rates $A = 0.3$, 1 and 3 the history lasts $T = 2/A$; the evolution starts in the dressed state, and the amplitudes are kept after every hundredth of the steps, which gives the same 101 points as `MARKS`. `runs[rate]` holds $P = 1 - |c_0|^2$ at these points. At $A = 1$ ($T = 2$) a second run starts in the bare level 0 instead: that is an abrupt start of the motion. `s_marks` are the values of a_4 at the marks.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), layout="constrained")
for colour, rate in zip(PALETTE, (0.3, 1.0, 3.0)):
    left.plot(s_marks, runs[rate], color=colour, lw=2.0, label=f"exact, $A = {rate}$")
    left.plot(s_marks, dressing(rate), color=colour, lw=1.5, ls="--")
left.plot([], [], color="0.4", ls="--", label="first order $\\sum_m Q_{0m}^2$")
```

```
left.set_yscale("log")
left.set_ylim(3e-5, 0.3)
left.set_xlabel("$a_4$ along the constant-rate history")
left.set_ylabel("$P$: probability outside the band level")
left.set_title("Dressed start: exact against first order")
left.legend(fontsize=8, loc="lower left")
```

The left panel: for each rate the exact P (solid) and the first-order dressing (dashed, same colour) against a_4; one empty plot gives the legend entry for the dashed lines; a logarithmic vertical axis from 3×10^{-5} to 0.3.

```
right.plot(s_marks, runs[1.0], color=PALETTE[1], lw=2.0, label="dressed start")
right.plot(s_marks, bare, color=PALETTE[0], lw=2.0, ls="-.", label="bare start")
right.plot(s_marks, dressing(1.0), color="0.4", lw=1.5, ls="--",
           label="first order $\\sum_m Q_{0m}^2$")
right.set_xlabel("$a_4$ along the constant-rate history")
right.set_ylabel("$P$")
right.set_title("$A = 1$: dressed and bare start")
right.legend(fontsize=8)
```

The right panel, with a linear vertical axis: at $A = 1$ the dressed start, the bare start and the first-order dressing.

```
save_figure(fig, "constant_rate",
            "Left: the probability $P$ that the particle is not in its "
            "instantaneous band level (vertical axis, logarithmic) along a history "
            "of constant rate $A = 0.3$, $1$, $3$ from $a_4 = 0$ to $2$ "
            "(horizontal axis), exact (solid) and the first-order dressing "
            "$\\sum_m Q_{0m}^2$ (dashed), starting in the dressed state. At $A = "
            "0.3$ they agree; at $A = 1$ the exact $P$ stays near 0.009 while the "
            "estimate falls; at $A = 3$ first order fails. Right: $A = 1$ with the "
            "dressed start and with the bare start (an abrupt start of the motion), "
            "which leaves about three times more.")
deviation = float(np.max(np.abs(runs[0.3] / dressing(0.3) - 1.0)))
report("A = 0.3: largest relative difference exact / first order", f"{deviation:.3f}")
report("A = 1: exact P at a4 = 0 and 2", f"{runs[1.0][0]:.5f}, {runs[1.0][-1]:.5f}")
report("A = 1: first-order dressing at a4 = 0 and 2",
       f"{dressing(1.0)[0]:.5f}, {dressing(1.0)[-1]:.5f}")
report("A = 1, bare start: largest P", f"{bare.max():.5f}")
check(deviation < 0.15, "at A = 0.3 the exact P agrees with the first-order dressing "
      "within 15 percent at all 101 points")
```

Figure 22a.6 and the numbers of Out [14]: at $A = 0.3$ the exact P and the first-order dressing differ by at most 11.3 percent; at $A = 1$ the exact P is 0.00904 at the start and 0.00730 at the end, while the first-order dressing falls from 0.00912 to 0.00289; the bare start at $A = 1$ reaches 0.02787. The check requires agreement within 15 percent at $A = 0.3$.

What the student should see in Figure 22a.6, and why. On the left the blue pair ($A = 0.3$) lies almost on top of each other and falls from about 8×10^{-4} to 2.6×10^{-4}: at a small rate the particle carries exactly the dressing that Q measures, and the dressing shrinks along the history because Q

falls as the momentum redshifts. At $A = 1$ the exact curve stays near 0.009 while the dressing falls to 0.003; the difference is, in the reading suggested by Section 22.7, probability that has really left the band level, not dressing that would disappear if the motion stopped slowly (the notebook does not separate the two by a computation of its own). At $A = 3$ the exact P stays near 0.075, the size of the sudden limit, and the first-order dressing is no guide. On the right, the abrupt (bare) start at $A = 1$ shakes the particle: P rises from 0 to 0.028 near $a_4 = 1.6$, about three times the dressing, because starting the motion suddenly is itself a sudden change.

In [15], how far the sudden limit reaches (Figure 22a.7).

```
ENDS = np.linspace(0.0, 6.0, 61)  # end slices of a sudden jump from a4,0 = 0
ends_q = K11 * np.exp(-ENDS)
band_a, band_b = orbitals(levels(np.zeros(61), ends_q), ends_q)
ceiling = 1.0 - (band_a @ (SIMPSON * band_a[0]) + band_b @ (SIMPSON * band_b[0])) ** 2
```

The sudden limit caps the damage, and it depends on how much the band orbital itself changes. The cell computes the band orbital of the Fermi shell at 61 end slices from 0 to 6, three times beyond the record's span (free field, computed here), and the sudden limit $1 - (\int \varphi_0(s)^\dagger \varphi_0(0)\, dy)^2$ of a jump from 0 to each of them. `band_a @ (SIMPSON * band_a[0])` computes in one matrix product the Simpson integrals of $a_0(s)a_0(0)$ for all 61 end slices.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(10.0, 4.2), layout="constrained")
for colour, index in zip(PALETTE, (0, 10, 20, 60)):
    left.plot(Y, band_a[index], color=colour, lw=2.0,
              label=f"$a$, $a_{{4,0}} = {ENDS[index]:.0f}$")
    left.plot(Y, band_b[index], color=colour, lw=1.2, ls="--")
left.plot([], [], color="0.4", ls="--", label="$b$ (dashed)")
left.set_xlabel("hidden coordinate $y$ (tip at $-3$, brane at 0)")
left.set_ylabel("band orbital (normalised)")
left.set_title("The band orbital of the Fermi shell")
left.legend(fontsize=8)
```

The left panel draws the two components of the band orbital, a solid and b dashed, against y at the end slices 0, 1, 2 and 6 (the indices 0, 10, 20, 60 of `ENDS`).

```
right.plot(ENDS, ceiling, color=PALETTE[0], lw=2.0)
right.plot([END], [sudden], "o", color=PALETTE[1], ms=9,
           label="end of the recorded span")
right.set_xlabel("end slice $a_{4,0}$ of a sudden jump from 0")
right.set_ylabel("$P_{sudden}$")
right.set_title("The ceiling of the damage")
right.legend(fontsize=8)
```

The right panel: the sudden limit against the end slice, with an orange dot at the end of the recorded span, the value of In [12].

```
save_figure(fig, "sudden_ceiling",
            "Left: the band orbital of the Fermi shell of $N = 688$, components "
            "$a$ (solid) and $b$ (dashed), against the hidden coordinate $y$ "
```

```
            "(horizontal axis) at the slices $a_{4,0} = 0$, $1$, $2$, $6$: as the "
            "momentum redshifts, $b$ shrinks and the orbital approaches the brane "
            "zero mode $e^{y}$. Right: the sudden limit $P_{sudden}$ (vertical "
            "axis) of a jump from $a_{4,0} = 0$ to the end slice (horizontal "
            "axis); it grows and levels off near 0.105: even an instant jump to "
            "the far future leaves almost 90 percent of the particles in the band "
            "level.")
report("sudden limit for the end slices 2, 4, 6",
       ", ".join(f"{ceiling[i]:.4f}" for i in (20, 40, 60)))
check(np.all(np.diff(ceiling) > -1e-12) and ceiling[-1] < 0.11
      and abs(ceiling[20] - sudden) < 1e-12,
      "the sudden limit grows with the end slice and stays below 0.11 up to 6")
```

Figure 22a.7 and Out [15]: the sudden limit is 0.0749, 0.1021 and 0.1051 for the end slices 2, 4 and 6. The check requires that it never decreases (`np.diff` gives the differences of neighbours), that it stays below 0.11, and that its value at the slice 2 equals the sudden limit of In [12].

What the student should see in Figure 22a.7, and why. On the left the band orbital is concentrated near the brane: a grows towards $y = 0$ and b is small and negative. As the slice grows, b shrinks towards zero and a spreads a little towards the tip, approaching the zero mode $(e^y, 0)$, the exact brane-bound orbital of momentum 0 (Chapter 14). The orbitals of the slices 2 and 6 are already close to each other. On the right the sudden limit rises steeply at first and then levels off near 0.105: the orbital cannot move further than to the zero mode, so even an instantaneous jump to the far future leaves almost 90 percent of the particles in the band level.

In [16], how many levels are needed (Figure 22a.8).

```
sizes, errors = [], {"sudden": [], 1.0: [], 3.0: []}
results = {}
for low in range(1, 10):
    for high in (low, low + 1):
        chosen = np.where((BASIS >= -low) & (BASIS <= high))[0]
        zero = int(np.where(BASIS[chosen] == 0)[0][0])
        e, k = E_SMOOTH[:, chosen], K_SMOOTH[:, chosen][:, :, chosen]
        start = np.zeros(len(chosen), dtype=complex)
        start[zero] = 1.0
        sizes.append(len(chosen))
```

A truncated basis can only be trusted if the results stop changing as it grows. The loop builds smaller bases from the labels $-l, \ldots, l$ and $-l, \ldots, l+1$ for $l = 1, \ldots, 9$, that is, of $3, 4, 5, \ldots, 20$ levels. `np.where(condition)[0]` gives the positions where the condition holds (`&` is "and" for arrays); `zero` is the position of label 0 within the chosen labels; `e` and `k` keep the columns, and for K also the rows, of the chosen levels; `start` puts the particle into the band level.

```
        c, _ = passage(0.0, e, k, RATE_SMOOTH, start, 400)
        errors["sudden"].append(abs(1.0 - abs(c[zero]) ** 2 - sudden))
        for rate in (1.0, 3.0):
            duration = 2.0 * END / rate
            c, _ = passage(duration, e, k, RATE_SMOOTH, start, steps_needed(duration))
            results[(len(chosen), rate)] = 1.0 - abs(c[zero]) ** 2
for rate in (1.0, 3.0):
```

```
    errors[rate] = [abs(results[(size, rate)] - results[(SIZE, rate)])
                    for size in sizes]
```

For each basis: the sudden limit, compared with the direct overlap of In [12], and the smooth passages at $A = 1$ and $A = 3$. Their errors are measured against the result of the full 20-level basis.

```
halving = 0.0
for rate in (RATES[0], 1.0, 10.0):
    duration = 2.0 * END / rate
    steps = steps_needed(duration)
    other = 2 * steps if steps < U_STEPS else steps // 2
    p_one, p_two = (1.0 - abs(passage(duration, E_SMOOTH, K_SMOOTH, RATE_SMOOTH, START,
                                      n)[0][ZERO]) ** 2 for n in (steps, other))
    halving = max(halving, abs(p_one - p_two))
```

The second numerical question, the time steps: three passages ($A = 0.25$, 1, 10) are repeated with twice the number of steps (or half of it, when the finest number is already in use), and `halving` keeps the largest change of P. The parentheses around the expression with `for` make a **generator**, whose two values are assigned to `p_one` and `p_two`.

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(sizes, errors["sudden"], "o-", color=PALETTE[0], lw=2.0, ms=6,
        label="sudden limit (against the direct overlap)")
for colour, marker, rate in ((PALETTE[1], "s", 1.0), (PALETTE[2], "^", 3.0)):
    ax.plot(sizes[:-1], errors[rate][:-1], marker + "--", color=colour, lw=1.5, ms=6,
            label=f"$A = {rate:.0f}$ (against 20 levels)")
ax.set_yscale("log")
ax.set_xticks(range(3, 21))
ax.set_xlabel("number of instantaneous levels in the basis")
ax.set_ylabel("$|P - P_{reference}|$")
ax.set_title("Convergence with the size of the basis")
ax.legend(fontsize=8)
```

The errors against the size of the basis on a logarithmic axis, with a tick at every size from 3 to 20. For $A = 1$ and 3 the last point (the 20-level basis itself, whose error is zero by definition) is left out (`[:-1]`), because zero has no place on a logarithmic axis.

```
save_figure(fig, "convergence",
            "The error of the probability $P$ (vertical axis, logarithmic) against "
            "the number of instantaneous levels kept in the basis (horizontal "
            "axis), for the sudden limit (blue, against the direct overlap of the "
            "two orbitals) and for the smooth passages at $A = 1$ and $A = 3$ "
            "(against the 20-level result). The errors fall steadily; with 20 "
            "levels the sudden limit is right to a few millionths.")
report("sudden limit: error with 20 levels", f"{errors["sudden"][-1]:.1e}")
report("A = 1: change from 18 to 20 levels", f"{errors[1.0][-3]:.1e}")
report("largest change of P when the steps are doubled or halved", f"{halving:.1e}")
check(errors["sudden"][-1] < 1e-5 and errors[1.0][-3] < 1e-6,
      "the 20-level basis is converged: sudden limit within 1e-5, A = 1 within 1e-6")
check(halving < 1e-9, "doubling or halving the time steps changes P by less than 1e-9")
```

Figure 22a.8 and Out [16]: with 20 levels the sudden limit is right to 4.2×10^{-6}; at $A = 1$ going from 18 to 20 levels changes P by 2.4×10^{-8} (`errors[1.0][-3]` is the entry of the 18-level basis, the third from the end of the list of sizes); and doubling or halving the time steps changes P by at most 3.0×10^{-11}. The two checks require 10^{-5}, 10^{-6} and 10^{-9}.

What the student should see in Figure 22a.8, and why. All three error curves fall by several powers of ten as levels are added, with small ups and downs where a newly added level happens to matter little. The sudden limit converges most slowly (from 7×10^{-3} with 3 levels to a few millionths with 20), because a sudden jump excites all levels at once, while a passage at a finite rate mostly reaches the nearby levels; the passage at $A = 1$ converges fastest. These measured errors are the uncertainties of the COMPUTED probabilities of In [12] and In [14]: they are thousands of times smaller than the effects the figures show.

In [17], the breakdown estimate.

```
q_first = q_max[(688, "lam0", 0.0)]  # the largest Q of the free Fermi shell, A = 1
estimates = [
    ("naive: Q = 1 for the weakest recorded state", naive[weakest]),
    ("naive: Q = 1 for the band level, 0.02 <= q <= 6", 1.0 / g_star),
    ("first order: Q^2 reaches the sudden limit", math.sqrt(sudden) / q_first),
    ("exact: P reaches half the sudden limit", rate_half),
]
```

The four estimates of the breakdown rate of Section 22.8: the naive one for the weakest recorded state ($1/Q_{max}$, In [3]); the naive one for the band level of every shell and slice with redshifted momentum $0.02 \le q \le 6$ ($1/G^*$, In [6]); the first-order one, the rate at which $(AQ_0)^2$ reaches the sudden limit, $A = \sqrt{P_{sudden}}/Q_0$ with $Q_0 = 0.0934506$, the record's Q_{max} of the free state $N = 688$ at the slice 0; and the exact $A_{1/2}$ (In [12]). `estimates` is a list of pairs (text, value).

```
say("breakdown estimate                                    rate A")
for label, value in estimates:
    say(f"{label:50s} {value:9.3f}")
report("P at the record's rate A = 1, constant rate (dressed start)",
       f"{runs[1.0].min():.4f} ... {runs[1.0].max():.4f}")
report("P at the peak rate A = 1, smooth passage", f"{excited[6]:.4f}")
report("sudden limit over the span 0 ... 2, the most P reaches in the scan",
       f"{sudden:.4f}")
```

The table of Out [17], each text padded to 50 characters (`:50s`) and each rate printed with three digits after the point in a field of width 9: 10.701, 10.021, 2.929 and 1.508. Then the probability that a particle of the Fermi shell is outside its instantaneous level at the record's rate: between 0.0073 and 0.0092 along the constant-rate history with the dressed start, 0.0138 at the end of the smooth passage with peak rate 1; and the sudden limit 0.0749, the most that P reached in the scan of In [12].

```
NAMES = ["naive_rates", "one_curve", "across_the_gap", "sector_levels",
         "smooth_passage", "constant_rate", "sudden_ceiling", "convergence"]
missing = [name for number, name in enumerate(NAMES, start=1)
           if not output_file(f"{FIGURE_FOLDER}/22a_{number}_{name}.png").is_file()]
check(missing == [], "every figure file of this notebook exists")
all_checks_passed()
```

The last check requires that all eight figure files exist (`enumerate(NAMES, start=1)` numbers the names from 1, and the file of figure k is `22a_k_name.png`), and the last line prints ALL 20 CHECKS PASSED (notebook 22a): two checks in In [2], one each in In [3] and In [4], three in In [5], two in In [7], one in In [8], two in In [9], three in In [12], one each in In [14] and In [15], two in In [16] and one in In [17].

22.13 What Notebook 22a found, and how a student could continue

The results. Every number below is printed by the notebook in the cell named; the uncertainties are the ones the notebook measures.

result	value	status	where
largest Q of the record at $A = 1$	0.0934532 (N688_lamm2_a00)	COMPUTED (Revision record)	Out [2]; `adiabaticity.csv`
naive breakdown rate $1/Q_{max}$ of the weakest recorded state	10.70	COMPUTED from the record	Out [3]
record's levels and Q of the free Fermi shells, reproduced	to 8.8×10^{-14} and 1.3×10^{-13} (relative)	COMPUTED here	Out [5]
top of the curve $G(q) = Q/A$ of the band jump	$G^* = 0.09979$ at $q^* = 2.128$	COMPUTED here ($\lambda = 0$)	Out [6]
naive breakdown rate of the band level of every shell with $0.02 \le q \le 6$ (all states of the record), $1/G^*$	10.02	COMPUTED here ($\lambda = 0$)	Out [6]
largest change of Q_{max} by the interaction	9.8 percent (N136_lamp2_a20)	COMPUTED from the record	Out [7]
strongest jump from a negative level to a bulk level, per unit rate	0.0489 at $q = 0.111$	COMPUTED here; its reading OPEN	Out [8]
P at the end of the smooth passage with peak rate $A = 1$	0.0138	COMPUTED here	Out [12]
rate $A_{1/2}$ at which P reaches half the sudden limit	1.508, between the scan rates 1.259 and 1.585	COMPUTED here	Out [12]
sudden limit of the span $a_{4,0} = 0$ to 2	0.074906 (overlap), 0.074902 (20-level basis)	COMPUTED here	Out [12]
P along the constant rate $A = 1$, dressed start	0.0073 to 0.0092	COMPUTED here	Out [14], Out [17]

result	value	status	where
sudden limit for the end slices 4 and 6	0.1021 and 0.1051	COMPUTED here ($\lambda = 0$, beyond the record's span)	Out [15]
numerical errors	total probability 1.2×10^{-11}; basis: sudden limit 4.2×10^{-6}, $A = 1$ 2.4×10^{-8}; time steps 3.0×10^{-11}	measured	Out [12], Out [16]

The rate $A_{1/2}$ comes from a straight-line interpolation in $\ln A$ between two rates of the scan (ten per factor 10); that P crosses half the sudden limit between the scan rates $10^{0.1} = 1.259$ and $10^{0.2} = 1.585$ is certain, and the interpolated value 1.508 lies between them; the interpolation error itself is not measured.

What the results mean. Four statements, each for the free field ($\lambda = 0$) in the sector computed, follow from the table.

- In this book's reading (Section 22.7; the record calls Q^2 the leading-order transition probability), the record's Q measures the dressing, and the numbers confirm this reading: at a small rate ($A = 0.3$) a particle carries the dressing $\sum_m Q_{0m}^2$, and the exact evolution agrees with it within 11.3 percent along the whole span (Out [14]).
- The naive criterion "Q reaches 1" overestimates the breakdown rate about sevenfold (10.7 against 1.508). The reason is visible in the derivation of Section 22.7: what can leave the level is a probability, Q^2 to first order, and what it should be compared with is not 1 but the largest probability that can leave at all, the sudden limit 0.0749. The first-order estimate built on that comparison, $A = 2.929$, is within a factor of two of the exact $A_{1/2}$.
- The damage is capped: however fast the passage, at most about 7.5 percent of the particles of the Fermi shell leave the band level over the record's span, and at most about 10.5 percent even for a jump to the slice 6, because the band orbital itself changes little (Figure 22a.7).
- At the record's rate $A = 1$, about one percent of the Fermi-shell particles are outside their instantaneous level (0.7 to 0.9 percent along the constant-rate history, 1.4 percent at the end of the smooth passage). The instantaneous picture is therefore a fair first approximation at $A = 1$ for this sector, but the record's history is not deep inside the adiabatic regime: the gas stops following its levels at rates only about 1.5 times larger.

What the notebook does not show. It treats the free field only, one sector only ($j = +1$, even parity, the Fermi shell of $N = 688$), and one particle at a time; it does not include the interaction, the other sectors, the many-body state or the thermal states. The probability P counts the negative levels of the basis among the levels a particle can leave to; how that part is to be read in the quantised theory is part of the OPEN question of the negative branch. The history is a PRESCRIBED BACKGROUND, and the Z2 brane is ASSUMED.

A consequence for Fermi-level crossings, from the record's own statement. The exact evolution conserves the momentum, the block and the brane parity, so particles jump only between

levels of the same sector (`Revision/kohn_sham/ks-theory.json`, key `adiabaticity.exactEvolution`). In the free field, therefore, the number of particles in every sector stays what it was at the start. When levels of different sectors cross the Fermi level, as in the record's example $N = 696$ (Section 22.5), the evolving gas cannot move its particles into the sector that the instantaneous ground state would fill: at best, in the adiabatic limit, it becomes the adiabatically continued state of the record, which lies above the instantaneous ground state by up to 8.623 (units of m) at the slice 2 (`Revision/kohn_sham/results/adiabatic/crossing-demo.csv`). Whether interactions beyond the mean field (collisions, which the functional of the record does not contain) let the gas relax towards the instantaneous ground state is OPEN.

How a student could continue.

1. *Other shells and sectors.* By exact rule 2 the free evolution of a shell of momentum k from the slice s_1 to the slice s_2 depends only on how $q = ke^{-a_4}$ moves from ke^{-s_1} to ke^{-s_2}. Repeat In [9] to In [16] for the Fermi shell of $N = 136$ ($k = 0.5$), for the odd brane parity and for the block type $j = -1$, and draw the breakdown rate as a function of k.

2. *The interacting problem.* Write a time-dependent Kohn-Sham solver: evolve every occupied orbital of every sector with a method that conserves probability exactly, and at every time step recompute the densities $n(y)$ and $S(y)$ and from them $M = m + \frac{15}{16}\lambda S$ and $v = -\lambda n/16$ (`Revision/kohn_sham/ks-theory.json`, key `exchange.kohnShamPotentials`). Tests fixed in advance: for $\lambda = 0$ it must reproduce Notebook 22a; the particle number must be conserved; at a very small rate the gas must stay close to the record's instantaneous states; and the energy balance must follow the identity $dE/da_4 = -3\,(2\mathrm{Vol}_7)\int e^{6Hy}(p_3 - p_t)\,dy$ of the record (key `emt.energyChange`) up to the dressing.

3. *Crossings.* Follow the record's example $N = 696$ through its Fermi-level crossing with the evolution of item 2, and measure how far the evolved gas stays from the instantaneous ground state.

4. *The negative branch.* Formulate the jumps of Figure 22a.3 in the quantised theory: compare the modes of the field before and after a passage (the Bogoliubov method), and find out whether, with the Krein metric of Chapter 10, they describe the creation of pairs of quanta of positive norm. This is OPEN, and it concerns quanta inside one universe, not universes.

Where to start. The builder of the notebook, `Revision/textbook/notebooks/src/22a_adiabatic_breakdown.py`; the theory, `Revision/kohn_sham/ks-theory.json` (keys `adiabaticity`, `exchange`, `emt`); the Rust solver in `Revision/kohn_sham/solver` with its README; the folder `Revision/kohn_sham/results/adiabatic` (`adiabaticity.csv`, `crossing-demo.csv`, `fermi-level-crossings.csv`, `history.json`); the reference solver `Revision/kohn_sham/reference/run_reference.py` for a second method.

22.14 Problem 3: the back-reaction of the gas on a_4

The question. Every Kohn-Sham result of the record is a test-field result: the gas lives on the history $a_4 = AHx_4$, which is prescribed. The open problem is to solve the gas and the geometry together: a history $a_4(x_4)$ (or a more general metric) and gas states such that the gas is a solution in that geometry and its energy-momentum tensor is the source of that geometry in the field equations.

An answer is such a solution, with its numbers checked as in Chapter 16, or a proof, under stated assumptions, that none exists.

What is known.

- PROVED (Chapter 12; `Revision/field_equations_a4/a4-equations.json`, key `generalSource`; both reports in `Revision/field_equations_a4/reports`): for the author's metric the Einstein-Lovelock equations reduce to the constraint (the x_4 component), the evolution equation $a_4'' F(a_4') = \kappa(p_3 - p_t)$, the hidden ($x_0$) equation and conditions on the source: every component of the source must be independent of x_0, the source must obey $p_3 + p_t = 2p_0$, and its mixed x_4-x_0 components must vanish. The left-hand sides of the equations are free of x_0 (key `generalSource.x0_dependence`; independently `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, check `einstein_x0_independent`), and that is why the source must be free of x_0 too.

- PROVED: the linear member $a_4 = AHx_4 + a_0$ needs $p_3 = p_t = p_0$ and a constant ρ (check `linear_member_equal_pressures` of both reports).

- PROVED (Chapter 12; key `b4` of `a4-equations.json`; `wolfram-a4-report.json`, with the checks `b4_offdiagonal_factorises`, `b4_einstein_offdiagonal_forces_b4_constant` and `b4_identity_x1_plus_x5_minus_2x0`): with the second function $b_4(x_4)$ of the author's metric the coupled problem has one more unknown. The mixed equation reads $\kappa q_{40} = \cot z\,(b_4'/b_4)\,Q$ with $Q = \frac{3}{2}H$ in Einstein gravity, so there a source with $T^{x_4}{}_{x_0} = 0$ forces a constant b_4; and the algebraic condition becomes $\kappa(p_3 + p_t - 2p_0) = \frac{1}{4}(b_4'/b_4)^2 + b_4''/(2b_4)$ in Einstein gravity. A coupled solution may therefore use b_4 to meet the source conditions; this too is OPEN for the gas.

- PROVED: in Einstein gravity every source obeys $\kappa(\rho + p_0) = -6\big((a_4')^2 + H^2\big)$ (check `einstein_null_energy_x0` of both reports; `Revision/lead_checks/reports/einstein-gauss-bonnet-a4.json`, check `einstein_null_energy`). The derivation is given below.

- COMPUTED: no recorded Kohn-Sham state is an admissible source. The report is `ks-source-conditions.json` in the folder `Revision/field_equations_a4/reports`, and its checks say why. Check `ks_profiles_depend_on_x0`: for every state with a nonzero energy-momentum tensor the energy density varies along the hidden coordinate, by at least 0.0497 of the largest component (smallest case N136_lamm2_a20). Check `ks_profiles_violate_algebraic_condition`: $\max_y |p_3 + p_t - 2p_0| / \max |T|$ lies between 2.09 and 3.99. Check `ks_integrals_violate_algebraic_condition`: even the source integrated over the hidden direction violates $p_3 + p_t = 2p_0$; the ratio $(\int p_3 + \int p_t)/(2\int p_0)$ is never 1 (closest 0.414, N688_lamm2_a00). Check `ks_history_is_a_prescribed_background`: along the history the integrated energy changes, for example $\int \rho = 80.28$, 32.38, 12.45 for N136_lam0 at the slices 0, 1 and 2, and $p_3 \neq p_t$, while the linear member needs a constant ρ and equal pressures.

- PROVED and COMPUTED, the energy exchange: along the history the energy of the gas changes as $dE/da_4 = -3\,(2\mathrm{Vol}_7)\int_{-L}^{0} e^{6Hy}(p_3 - p_t)\,dy$, where Vol_7 is the coordinate volume of 3-space and the extra times. The identity is the key `emt.energyChange` of `Revision/kohn_sham/ks-theory.json`, verified by the check `adiabatic_hellmann_feynman` of the theory report `ks-theory-python.json`. The record computes both sides: for N688_lam0_a00 the finite difference of the energies gives -597.9157012757526 and the energy-momentum integral -597.9157012806555 (the columns `dE_da4_finite_difference` and `dE_da4_emt` of

`adiabaticity.csv`). In a coupled solution this energy would have to be exchanged with the geometry.

- Chapter 12 constructs three exact solutions of the coupled equations of Einstein gravity and a homogeneous condensate of the classical field dirac16complex00, two of them with the extra times deflating as e^{-Hx_4} (Notebook 12d; a computation of this book under the assumptions stated there, not a Revision record). For that classical, homogeneous source the back-reaction is solved. For the quantised gas it is OPEN.

The null combination in Einstein gravity, line by line. The constraint and the hidden equation of Einstein gravity (`Revision/field_equations_a4/a4-equations.json`, key `einstein`) are

$$3(a_4')^2 + 21H^2 + \Lambda = -\kappa\rho, \qquad -3(a_4')^2 + 15H^2 + \Lambda = \kappa p_0.$$

$$\kappa\rho = -3(a_4')^2 - 21H^2 - \Lambda.$$

Rule: multiply the constraint by -1.

$$\kappa(\rho + p_0) = -3(a_4')^2 - 21H^2 - \Lambda - 3(a_4')^2 + 15H^2 + \Lambda = -6(a_4')^2 - 6H^2.$$

Rule: add the hidden equation; Λ cancels, $-3-3=-6$ and $-21+15=-6$.

The right-hand side is negative for every real history when $H > 0$. So in Einstein gravity, with a positive coupling κ, every source of the author's metric must have $\rho + p_0 < 0$ everywhere: it must violate the null energy condition along the hidden direction. This holds for every a_4 and every Λ; any candidate source, a time-dependent Kohn-Sham gas included, must meet it, or the gravity theory must contain more than the Einstein term.

Why it is hard. The metric of the record fixes the dependence on x_0 completely ($\sin^{1/3} z$ and $\cot^2 z$), and leaves only one free function, $a_4(x_4)$. A gas bound to the brane, whose density falls off towards the tip, cannot be the source of such a metric: its energy-momentum depends on y. A coupled solution therefore needs a more general metric, whose factors depend on y and on x_4; the field equations then become partial differential equations in two variables, and the Kohn-Sham reduction of Chapter 14, which used the exact warp e^{Hy}, must be redone for the new metric at every step. In addition, the brane enters (Problem 4), the couplings α_2, α_3, Λ and κ are unknown, and in Einstein gravity the source must violate the null energy condition along x_0, as just shown.

A first step. (a) Redo the derivation above, and its Einstein-Gauss-Bonnet version from the general null combination of the record (`Revision/field_equations_a4/a4-equations.json`, key `generalSource.nullCombinations`). (b) Check the energy-exchange identity on the record's numbers (Exercise 7 of Section 22.28). (c) Write down a more general metric that keeps the symmetries of the author's metric (3-space isotropic, the three extra times isotropic) but lets the factors depend on y and x_4, for example $ds^2 = e^{2\mathcal{A}(y,x_4)}(dx_1^2 + dx_2^2 + dx_3^2) - e^{2\mathcal{B}(y,x_4)}(dx_5^2 + dx_6^2 + dx_7^2) - e^{2\mathcal{C}(y,x_4)}dx_4^2 + dy^2$, which contains the author's metric as $\mathcal{A} = Hy + a_4$, $\mathcal{B} = Hy - a_4$, $\mathcal{C} = 0$ (this ansatz is a suggestion of this book, a HYPOTHESIS about a useful next step, not a result). Compute its Einstein tensor with two independent programs, as the record does for the author's

metric, and find which conditions it puts on a source. (d) Only then ask whether a Kohn-Sham gas can satisfy them, starting with the free gas.

Where to start. `Revision/field_equations_a4/README.md` and its verifiers; `Revision/field_equations_a4/python/check_ks_source_conditions.py` and its report; the lead's independent curvature code `Revision/lead_checks/einstein_gauss_bonnet_a4.py`; the profiles `Revision/kohn_sham/results/ground/profiles` and the integrals `Revision/kohn_sham/results/ground/emt-integrals.csv`; the GKD and Lovelock programs of Chapter 11 in `Revision/gkd_lovelock`.

22.15 Problem 4: the Z2 brane and its junction conditions

The question. The hidden direction ends at $z = \pi/2$, the brane. The record continues the universe beyond it by its mirror image, the Z2 construction: the field on the mirror patch $\pi/2 < z < \pi$ is the reflected field. This is ASSUMED. The open problem is to derive it, or to replace it, from field equations: which conditions must the metric and the field satisfy at the brane (the **junction conditions**), what energy and momentum must sit on the brane itself, and is the mirror the right continuation? An answer is a set of junction conditions derived from the field equations of gravity and of the field, with the brane's energy-momentum, checked like every other exact statement of the record.

What is known.

- PROVED: the map $z \to \pi - z$ (that is, $x_0 \to \pi/(6H) - x_0$) is an isometry of the author's metric: every component takes the same value at z and at $\pi - z$ (`Revision/pairing/reports/python-pairing.json`, check `geometry.mirror_isometry`; `Revision/pairing/reports/wolfram-pairing.json`, check `T2_mirror_is_isometry`).
- PROVED: in the coordinate x_0 the metric is degenerate at the brane: $g_{00} = \cot^2 z$ and $\sqrt{|g|} = \cos z$ vanish at $z = \pi/2$ (`Revision/pairing/reports/python-pairing.json`, check `geometry.brane_degenerate`).
- ASSUMED (`Revision/kohn_sham/ks-theory.json`, key `boundaryConditions.brane`): the Z2 mirror (orbifold) at $y = 0$, with the mirror warp $e^{-H|y|}$; continuity of the mirrored field gives the two brane parities, $b(0) = 0$ (even) and $a(0) = 0$ (odd), which both stop the current along y (checks `bc_brane_parity_conditions`, `bc_mirror_map_PA` and `bc_mirror_parities_of_densities` of `Revision/kohn_sham/reports/ks-theory-python.json`). PROVED under this assumption: the doubled problem is symmetric under the mirror exactly when the mirror copy carries $(-m, +\lambda)$ (`Revision/pairing/kohn_sham/reports/python-t3.json`, check `T3.z2_mirror_copy_carries_minus_m_plus_lambda`).
- What rests on it: theorem T2 in the author's field (the mirror pairing), theorem T3, and every Kohn-Sham number of the record (the densities are those of the doubled system, each patch with the weight 1/2; key `densities.total`).
- NOT established, as the record says itself: "no junction condition, brane tension or matching of the field across it is derived" (`Revision/pairing/reports/python-pairing.json`, key `not_established`). OPEN.

The brane in the coordinate y, line by line. The two derivations of this paragraph and the next are written out by the book from statements of the record. The first, the line element in the coordinate y, is verified by the record's checks `geometry_hidden_coordinate` and `geometry_warped_form` of `Revision/kohn_sham/reports/ks-theory-python.json`; the second, the kink of the mirror warp, is the book's own, and no Revision verifier checks it as such. The record's hidden coordinate is $y = \ln(\sin z)/(6H)$, with $dy = \cot z\, dx_0$ (`Revision/kohn_sham/ks-theory.json`, key `geometry.hiddenCoordinate`).

$$\cot^2 z\, dx_0^2 = dy^2.$$

Rule: square both sides of $dy = \cot z\, dx_0$.

$$\sin z = e^{6Hy}, \qquad \sin^{1/3} z = e^{2Hy}.$$

Rule: solve $y = \ln(\sin z)/(6H)$ for $\sin z$ (multiply by $6H$, then apply the exponential), and raise both sides to the power $1/3$.

$$ds^2 = e^{2Hy}\big[e^{2a_4}(dx_1^2 + dx_2^2 + dx_3^2) - e^{-2a_4}(dx_5^2 + dx_6^2 + dx_7^2)\big] - dx_4^2 + dy^2.$$

Rule: insert both lines into the author's metric (the record's key `geometry.lineElement`; check `geometry_warped_form` of `Revision/kohn_sham/reports/ks-theory-python.json`).

At the brane $y = 0$ every coefficient of this line element is finite and nonzero ($e^0 = 1$). The zero of g_{00} in the coordinate x_0 belongs to that coordinate: $dx_0 = \tan z\, dy$, and $\tan z$ grows without bound as $z \to \pi/2$, so near the brane one unit of y corresponds to ever more units of x_0. In the same way $\sqrt{|g|}$ is $e^{6Hy} = \sin z$ in the coordinate y, and $\sin z \cdot \cot z = \cos z$ in the coordinate x_0 (rule: a volume element changes by the factor $|dy/dx_0| = \cot z$). What makes the brane special is therefore not a singularity of the geometry but the end of the coordinate range: y cannot exceed 0, because $\sin z$ cannot exceed 1. To go on, a continuation must be chosen, and the Z2 mirror is one choice.

The mirror has a kink, line by line. On the patch the warp factor of the six directions $x_1, x_2, x_3, x_5, x_6, x_7$ is $W(y) = e^{Hy}$ (key `geometry.warp`). The mirror continues it to $W(y) = e^{-H|y|}$ on $-L \le y \le L$ (key `boundaryConditions.brane`).

$$W'(y) = He^{Hy} \ \ (y < 0), \qquad W'(y) = -He^{-Hy} \ \ (y > 0).$$

Rule: for $y < 0$, $|y| = -y$ and $W = e^{Hy}$; for $y > 0$, $|y| = y$ and $W = e^{-Hy}$; differentiate each.

$$W'(0^-) = H, \qquad W'(0^+) = -H.$$

Rule: let y go to 0 from below and from above; $e^0 = 1$. The function W is continuous at $y = 0$ (both sides give 1) but its slope jumps by $-2H$: W has a kink.

$$\int_{-\epsilon}^{\epsilon} W''(y)\, dy = W'(\epsilon) - W'(-\epsilon) = -He^{-H\epsilon} - He^{-H\epsilon} = -2He^{-H\epsilon} \ \to \ -2H \quad (\epsilon \to 0).$$

Rule: the fundamental theorem of calculus (the integral of a derivative is the difference of the values at the ends), then the two slopes above, then $e^{-H\epsilon} \to 1$.

The integral of W'' over an interval around the brane does not shrink to zero with the interval: the second derivative of W has a part concentrated on the brane. The curvature of a metric contains the second derivatives of its components, so the curvature of the doubled metric has a part concentrated on the brane, and the field equations can then only hold if the source has a part concentrated there too: energy and momentum that live on the brane. Which energy density and which pressures, in which directions, the field equations demand is exactly what the junction conditions would say. Nobody has derived them for this metric: OPEN.

Why it is hard. The brane is a 7-dimensional surface whose own signature is (3,4): three space directions x_1, x_2, x_3 and four times x_4, x_5, x_6, x_7. The junction conditions of general relativity (W. Israel, Nuovo Cimento B 44, 1 (1966); quoted, not derived here) are formulated for surfaces in ordinary spacetime; the Lovelock terms of Chapter 11 change them; and the warp multiplies only six of the seven brane directions, because x_4 is not warped. For the field, the record's parities come from continuity of the mirrored field; other continuations are possible, and T2 and T3 depend on this one.

A first step. (a) Redo the two derivations above. (b) Compute the curvature of the doubled metric with $W = e^{-H|y|}$, keeping the concentrated parts, with two independent programs (the lead's curvature code in `Revision/lead_checks/einstein_gauss_bonnet_a4.py` is one starting point); write down the brane energy-momentum that Einstein's equations then require, and check whether it is a pure tension (the same negative pressure in every brane direction) or whether the unwarped time x_4 makes it different in different directions. (c) Integrate the field equation of the spinor across the brane in the same way and compare the resulting matching conditions with the record's parities $b(0) = 0$ and $a(0) = 0$. (d) Solve the Kohn-Sham problem of Chapter 15 with another brane condition and measure how much the levels and energies change.

Where to start. The Kohn-Sham theory record `Revision/kohn_sham/ks-theory.json`, with its keys `geometry` and `boundaryConditions`, and the two verifiers of this record in the folder `Revision/kohn_sham/theory`. For the mirror pairing, the pairing record `Revision/pairing/pairing-theory.json`, the T3 record `t3-theory.json` in the folder `Revision/pairing/kohn_sham`, and section 5.4 of the document `Revision/docs/PAIR_CREATION_PROOFS.md`, which proves T2 with the mirror.

22.16 Problem 5: the dark-sector hypotheses

The question. The author's request contains two hypotheses, quoted from `Revision/README.md`: "Hypothesis: dirac16complex provides a possible physical mechanism for a time-varying dark energy equation of state and/or a possible physical mechanism for a time-varying dark matter equation of state, both of which you will investigate." and "Hypothesis00: dirac16complex00 provides [the same], both of which you will investigate." (the brackets are those of the README, which abridges the repeated sentence). The problem is to investigate them: compute the equation of state that an observer in 3-space would see as the extra times deflate and 3-space inflates, its change in time, and compare it with the observations. An answer is such a computation with its uncertainties, and an honest statement of what each field can and cannot produce.

What is known now. Status: both hypotheses have been INVESTIGATED by the Revision

record, and neither is established. The investigation lives in two folders, `Revision/dark_sector/dirac16complex` (four scripts whose reports hold 30, 5, 13 and 9 checks, all PASS) and `Revision/dark_sector/dirac16complex00` (two independent implementations with 49 and 28 checks, all PASS), and it is written up in the registered document `Revision/docs/DARK_SECTOR_HYPOTHESES.md`. Sections 22.17 to 22.19 teach what the record finds, Notebook 22b (Section 22.20) reproduces its key numbers from the committed outputs, and Section 22.24 lists what remains open. The short answer, every item with the assumption it rests on:

1. The energy density ρ_4 that a 3-space observer assigns to 3-space is not fixed by the field equations; it is an ASSUMPTION, and the record states three definitions, called A, B and C (Section 22.17). Every verdict moves by exactly -1 between A and B on one side and C on the other.

2. dirac16complex, dark matter: a time-varying equation of state that falls from $1/3$ toward 0 (dark-matter-like) is present in the theory for quanta of the massive bulk band, but NOT in the computed Kohn-Sham ground states, whose gas is radiation-like under A and B (Section 22.18).

3. dirac16complex, dark energy: only under C, with w_{eff} between -0.707107 and -0.671895 and a slope of thawing sign far smaller than the observed one; nothing computed comes near the observed pair $(w_0, w_a) = (-0.861, -0.60)$, and no computed state crosses $w = -1$.

4. dirac16complex00, dark matter: a gas of positive-energy modes without extra-time momentum has the dark-matter-like law from $1/3$ to 0 under B (Section 22.19).

5. dirac16complex00, dark energy: only under C; the observed numbers are reproduced only by models whose parameters were CHOSEN to reproduce them, and a crossing of $w = -1$ only with a ghost-like component of negative energy.

6. On the prescribed history $a_4 = AHx_4$ the observer's expansion itself reads $w = -1$ exactly, with no time variation (Section 22.17).

In the words of the record: it "establishes neither Hypothesis nor Hypothesis00". Both remain HYPOTHESES.

The observed numbers. They come from a private document of the author, and the book quotes them only as `Revision/README.md` states them: the parametrisation $w(a) = w_0 + w_a(1 - a)$ (CPL, after Chevallier, Polarski and Linder), the constant-w fit $w = -0.764$, and $(w_0, w_a) = (-0.861, -0.60)$; in this convention thawing means $w_a < 0$. The record calls them the **Unite values**. In that document H is the Hubble rate and a the scale factor of the observed universe; they are not the H and a_4 of the author's metric.

What the CPL numbers say, line by line.

$$w(1) = w_0 + w_a \cdot 0 = w_0 = -0.861.$$

Rule: today the scale factor is $a = 1$, so $1 - a = 0$.

$$w(0) = w_0 + w_a = -0.861 - 0.60 = -1.461.$$

Rule: in the far past $a \to 0$, so $1 - a \to 1$.

$$w_0 + w_a(1-a) = -1 \quad \Longleftrightarrow \quad 1 - a = \frac{-1 - w_0}{w_a} = \frac{-0.139}{-0.60} = 0.231667 \quad \Longleftrightarrow \quad a = 0.768333 = \frac{461}{600}.$$

Rule: subtract w_0 from both sides and divide by w_a; then subtract from 1 ($0.231667 = 139/600$). The record checks this crossing exactly (`Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json`, check `unite_crossing_point`).

So the fitted w falls below -1 for $a < 461/600$. What that requires:

$$w < -1,\ \rho > 0 \quad \Longleftrightarrow \quad p < -\rho \quad \Longleftrightarrow \quad \rho + p < 0.$$

Rule: multiply $w = p/\rho < -1$ by the positive number ρ (the direction of an inequality is kept), then add ρ to both sides.

A fluid of positive energy density with $w < -1$ (called **phantom**) violates the null energy condition. Section 22.19 shows what in this theory can produce such a reading, and at what price.

22.17 The observer's density and the equation of state, line by line

Why an assumption is needed. The observations are made in four dimensions, three of space and one of time; the theory lives in eight. An observer who sees only x_1, x_2, x_3 and the time x_4 must turn the energy of a state, which is spread over all seven other directions, into an energy per unit volume of 3-space. Over the hidden direction x_0 the record averages with the weight $\sqrt{|g|} = \cos z$, which does not depend on x_4 (check `observer_hidden_average_commutes` of `python-derive-eos.json`). Over the three extra times there is no unique choice, because they are time-like: an observer does not move through them as through a space direction. The record therefore states three definitions and decides between none of them.

The proper 7-volume does not change, line by line. The proper length of a step dx along a direction is $\sqrt{|g_{xx}|}\,dx$; the proper volume of a small box is the product of the proper lengths of its sides.

$$\left(e^{a_4} \sin^{1/6} z\right)^3 = e^{3a_4} \sin^{1/2} z.$$

Rule: the three directions x_1, x_2, x_3 each have the factor $e^{a_4} \sin^{1/6} z$; a product of three equal factors is the third power, and $3 \cdot \frac{1}{6} = \frac{1}{2}$. This is the proper 3-volume per unit coordinate volume: it inflates like e^{3a_4}.

$$\left(e^{-a_4} \sin^{1/6} z\right)^3 = e^{-3a_4} \sin^{1/2} z.$$

Rule: the same for the three extra times, whose factor is $e^{-a_4} \sin^{1/6} z$. The proper extra-time volume deflates like e^{-3a_4}.

$$e^{3a_4} \sin^{1/2} z \cdot e^{-3a_4} \sin^{1/2} z \cdot \cot z = \sin z \cot z = \cos z.$$

Rule: multiply by the factor $\sqrt{g_{00}} = \cot z$ of the hidden direction; $e^{3a_4}e^{-3a_4} = e^0 = 1$, $\sin^{1/2} z \sin^{1/2} z = \sin z$, and $\sin z \cdot \cos z / \sin z = \cos z$. The proper 7-volume element of a slice $x_4 =$ const is $\cos z$: the inflation of 3-space is compensated exactly by the deflation of the extra times (`Revision/dark_sector/dirac16complex/reports/derivation-checks.json`, checks `proper_3_and_extra_time_volume_scalings` and `proper_7_volume_element_independent_of_a4`).

The energy balance, line by line. Let E be the energy of a state, the proper 7-volume integral of the energy density ρ, and P_3, P_t, P_0 the integrals of the pressures of 3-space, of the extra times and of the hidden direction. The record re-derives the conservation law $\nabla_\mu T^\mu{}_\nu = 0$ from the author's metric for a diagonal energy-momentum tensor that depends on x_4 and x_0; its x_4 component reads (checks `conservation_x4_identity` and `conservation_x4_author_metric`)

$$\frac{\partial \rho}{\partial x_4} = -3a_4' \, (p_3 - p_t).$$

$$\frac{dE}{dx_4} = \int \cos z \, \frac{\partial \rho}{\partial x_4} = -3a_4' \int \cos z \, (p_3 - p_t) = -3a_4' \, (P_3 - P_t).$$

Rule: integrate over a slice with the weight $\cos z$; because the weight does not depend on x_4, the derivative can be taken outside the integral; a_4' does not depend on the position and comes out of the integral.

$$\frac{dE}{da_4} = \frac{dE/dx_4}{da_4/dx_4} = -3 \, (P_3 - P_t) = -3X, \qquad X = P_3 - P_t.$$

Rule: the chain rule, $dE/dx_4 = (dE/da_4)(da_4/dx_4)$, divided by $a_4' = da_4/dx_4$ (check `integrated_identity_dE_da4`). So the energy of a state changes as 3-space inflates exactly by the difference of the two pressures: positive pressure in 3-space takes energy away, as for a gas in an expanding box, and negative pressure along the extra times does the same.

The three definitions. They differ in how the extra times are counted:

- **(A)** the extra times are compact, with a fixed coordinate period (closed time-like directions): $\rho_4 = E/(\text{proper 3-volume}) \propto E \, e^{-3a_4}$;
- **(B)** the extra times are not compact, and ρ_4 is taken per unit extra-time COORDINATE volume: again $\rho_4 \propto E \, e^{-3a_4}$;
- **(C)** ρ_4 is taken per unit PROPER 7-volume (equivalently per unit proper extra-time volume): $\rho_4 \propto E$.

All three are written as $\rho_4 \propto E \, e^{-3(1-s)a_4}$, with $s = 0$ for A and B and $s = 1$ for C. Status: ASSUMPTION. Which of them, if any, describes a physical observer is OPEN (Section 22.24). The record for dirac16complex00 calls the same two scalings N1 (the scaling of A and B) and N2 (that of C).

The equation of state read from a dilution. For a 4-dimensional observer a fluid with a constant equation of state w thins out as $\rho \propto a^{-3(1+w)}$ (radiation, $w = 1/3$, as a^{-4}; dust, $w = 0$, as

a^{-3}; a cosmological constant, $w=-1$, not at all). Turned around, this defines the equation of state that the observer infers from any dilution:

$$\ln\rho = -3(1+w)\ln a + \text{const} \quad\Longrightarrow\quad \frac{d\ln\rho}{d\ln a} = -3(1+w) \quad\Longrightarrow\quad w = -1 - \frac{1}{3}\frac{d\ln\rho}{d\ln a}.$$

Rule: take the logarithm of $\rho = c\,a^{-3(1+w)}$ (the logarithm of a product is the sum of the logarithms, and $\ln a^n = n\ln a$), differentiate with respect to $\ln a$, then divide by -3 and subtract 1. Applied to ρ_4 it defines $w_{\text{eff}} = -1 - \frac{1}{3}\,d\ln\rho_4/d\ln a$.

The observer identities, line by line. The observer's scale factor is $a = e^{a_4 - a_{4,\text{today}}}$ (normalised to $a=1$ at a chosen today, a free parameter).

$$\ln a = a_4 - a_{4,\text{today}} \quad\Longrightarrow\quad d\ln a = da_4.$$

Rule: the logarithm undoes the exponential; $a_{4,\text{today}}$ is a constant.

$$\ln\rho_4 = \text{const} + \ln E - 3(1-s)a_4.$$

Rule: the logarithm of $c\,E\,e^{-3(1-s)a_4}$.

$$\frac{d\ln\rho_4}{d\ln a} = \frac{1}{E}\frac{dE}{da_4} - 3(1-s) = -\frac{3X}{E} - 3 + 3s.$$

Rule: differentiate with respect to a_4 (which is $\ln a$ up to a constant); $d\ln E/da_4 = (dE/da_4)/E$, and the energy balance $dE/da_4 = -3X$.

$$w_{\text{eff}} = -1 - \frac{1}{3}\Big(-\frac{3X}{E} - 3 + 3s\Big) = -1 + \frac{X}{E} + 1 - s = \frac{X}{E} - s.$$

Rule: insert into the definition of w_{eff} and multiply out.

$$w_{\text{eff}}(A) = w_{\text{eff}}(B) = \frac{X}{E}, \qquad w_{\text{eff}}(C) = \frac{X}{E} - 1.$$

Rule: $s=0$ for A and B, $s=1$ for C. These are exact (checks `w_eff_A_equals_X_over_E`, `w_eff_B_equals_w_eff_A`, `w_eff_C_equals_X_over_E_minus_1` and `w_eff_general_normaliser` of `derivation-checks.json`; for dirac16complex00, `observer_N1_weff_identity` and `observer_N2_weff_identity` of `python-derive-eos.json`). Notebook 22b tests them on the computed Kohn-Sham history (In [3]).

How every verdict depends on the assumption. The same state reads differently under each definition (the rows are records of `Revision/docs/DARK_SECTOR_HYPOTHESES.md`, section 3.2, derived in Sections 22.18 and 22.19):

state	w_{eff} **under A, B**	w_{eff} **under C**	**ratio** p_3/ρ
homogeneous condensate	0, dust-like	-1, cosmological-constant-like	$\lambda S/(2m+\lambda S)$, constant
massless quanta	1/3, radiation	$-2/3$	1/3
massive quanta, flat limit	$1/3 \to 0$	$-2/3 \to -1$	as under A, B
Kohn-Sham gas, $N = 688$, $\lambda = 0$	0.2929 to 0.3183	-0.7071 to -0.6817	0.2929 to 0.3183

The ratio p_3/ρ, the 8-dimensional equation of state, does not depend on the normalisation at all. INTERPRETATION (labelled, as in the record): supernova distances measure the dilution-inferred w_{eff} of a 4-dimensional observer, which agrees with the ratio only when no energy is exchanged with the extra times.

The expansion seen by the observer. An observer who reads the scale factor $a = e^{a_4}$ and the time $t = x_4$ and uses the 4-dimensional Friedmann equations infers $w_{\text{exp}} = -1 - \frac{2}{3}\, a_4''/(a_4')^2$ (check `expansion_inferred_w`). On the history $a_4 = AHx_4$ of the Kohn-Sham record, $a_4' = AH$ is constant and $a_4'' = 0$, so $w_{\text{exp}} = -1$ exactly: the expansion itself has no time-varying equation of state. This is also the only history that a condensate allows as a source: a condensate has $p_3 = p_t$, so the evolution equation $a_4'' F(a_4') = \kappa(p_3 - p_t)$ of Section 22.14 gives $a_4'' F(a_4') = 0$ at every time, and because F is a polynomial in a_4' that is not identically zero this forces $a_4'' = 0$ (the theorem of Chapter 12, key `theoremLinear` of `Revision/field_equations_a4/a4-equations.json`).

When is a reading phantom?

$$w_{\text{eff}}(C) < -1 \quad\Longleftrightarrow\quad \frac{X}{E} - 1 < -1 \quad\Longleftrightarrow\quad \frac{X}{E} < 0.$$

Rule: add 1 to both sides. With $E > 0$ this needs $X < 0$, that is $P_t > P_3$: a pressure along the extra times larger than the pressure of 3-space; otherwise it needs $E < 0$ together with $X > 0$, a state of negative energy. Under A and B the same condition gives $w_{\text{eff}} < 0$ (check `phantom_conditio n`, which verifies the identity $w_{\text{eff}}(C) + 1 = X/E$, not the origin of a negative E). For a single plane-wave mode of real frequency the record proves $X \geq 0$ (check `flat_mode_X_nonnegative`): its 3-space pressure is ≥ 0 and its extra-time pressure ≤ 0, so both make X larger, not smaller.

22.18 dirac16complex: the Kohn-Sham gas, the bulk band, the condensate and the mixtures

The Kohn-Sham history. The record runs the Revision Rust Kohn-Sham solver of Chapter 15 at the 41 slices $a_{4,0} = 0, 0.05, \ldots, 2$ for $N = 8$, 136 and 688 quanta and the five couplings $\lambda = 0, \pm\lambda_1, \pm\lambda_2$: 615 runs, with $H = m = 1$, $L = 3$, the tip angle $\theta = 0$ and temperature $T = 0$ (`Revision/dark_sector/dirac16complex/reports/ks-history-run.json`, 5 checks). It reproduces the 75 committed states of Chapter 15 with relative deviation 0 (check `committe d_slices_reproduced`), and the occupied levels are the same at all 41 slices of every series (check `occupied_labels_fixed_along_history`), so the instantaneous ground states are the adiabatically continued states. The history is a PRESCRIBED BACKGROUND: the gas is a test field without back-reaction (Section 22.14). The record's script `compute_eos.py` evaluates w_{eff} for the three definitions, the tangents, the fits, the condensate and the mixtures (13 checks), and an independent

second computation, `independent_free_gas.py` (Chebyshev collocation, no solver output used except for the final comparison), reproduces the solver's energies over 82 states to 1.897×10^{-12} and $w_{\text{eff}} = X/E$ to 1.492×10^{-12} (9 checks).

The gas is radiation-like. The occupied levels lie on the brane band, which is massless at zero 3-momentum (the brane zero mode, slope $c\,e^{-a_4}$ with $c = 1.9051482536$ at small k; check `brane_band_slope`). So the gas redshifts like radiation: $d\ln E/da_4$ goes from -0.8787 at $a_4 = 0$ to -0.9549 at $a_4 = 2$ for $N = 688$, $\lambda = 0$, toward the radiation value -1. Over all series with $N = 136$ and 688, X/E lies between 0.292893 and 0.328105 and rises monotonically toward 1/3 (checks `gas_radiation_like_band` and `gas_X_over_E_rises_toward_one_third` of `eos-checks.json`; Notebook 22b, In [3] and Figure 22b.1). For $N = 688$, $\lambda = 0$: $w_{\text{eff}}(A) = w_{\text{eff}}(B)$ goes from 0.2929 to 0.3183 and $w_{\text{eff}}(C)$ from -0.7071 to -0.6817. The pressure of the hidden direction gives P_0/E from 0.3535 to 0.6482: a pressure, but not one that the 3-space observer sees as pressure.

The CPL tangents of the gas. With $a = e^{a_4 - a_{4,\text{today}}}$, $da = a\,da_4$, so at $a = 1$ the slope dw/da equals dw/da_4 and the tangent is $w_0 = w_{\text{eff}}$ at $a_{4,\text{today}}$, $w_a = -dw_{\text{eff}}/da_4$ there. For $N = 688$, $\lambda = 0$ (`Revision/dark_sector/dirac16complex/outputs/eos-summary.json`; Notebook 22b, Out [4]):

$a_{4,\text{today}}$	**0.5**	**1**	**1.5**	**2**
w_0 under C	-0.7037	-0.6982	-0.6907	-0.6817
w_a (every definition)	-0.00905	-0.01292	-0.01701	-0.01775

(w_0 under A and B is the value under C plus 1; w_a is the same in every definition, because the definitions differ by the constant s.) Over all series with $N = 136$ and 688 the tangent w_a lies between -0.020523 and -0.008894, the least-squares fits give w_a between -0.030371 and -0.018132, and $w_{\text{eff}}(C)$ lies between -0.707107 and -0.671895 (check `gas_cpl_thawing_sign_small`). The sign is that of thawing; the size is smaller than the observed 0.60 by a factor of about 20 or more. For $N = 8$ with $\lambda \neq 0$ only the brane zero modes are occupied: E is constant and $P_3 = P_t$, so $w_{\text{eff}}(A) = 0$ and $w_{\text{eff}}(C) = -1$, constant, with $E < 0$ for $\lambda > 0$ (check `n8_interacting_zero_modes_constant`); for $N = 8$ with $\lambda = 0$, $E = 0$ and there is no equation of state. The negative E at $\lambda > 0$ comes from the interaction energy of the record's functional, Hartree plus the uniform-gas exchange of Chapter 13: with the exact Fock exchange in its place, solved self-consistently (the exact-Fock variant of the Kohn-Sham record, `Revision/kohn_sham/results/exx/exact-fock-variant.csv`), the same $N = 8$ states have $|E| \leq 2.04 \times 10^{-13}$. So this negative E is a property of the approximate functional, and good-sector states with $E < 0$ are not established beyond it (`Revision/docs/DARK_SECTOR_HYPOTHESES.md`, section 4.7).

The dark-matter-like law, line by line. In the flat limit (no warp) a quantum of mass M and fixed coordinate momentum k has the physical momentum k/a and the energy $\omega = \sqrt{M^2 + k^2/a^2}$, with $a = e^{a_4}$. With the number of quanta fixed, $E \propto \omega$, and the energy balance gives $X/E = -\frac{1}{3}\,d\ln E/da_4$.

$$\ln \omega = \tfrac{1}{2} \ln \left(M^2 + k^2 e^{-2a_4}\right).$$

Rule: $\ln \sqrt{x} = \frac{1}{2} \ln x$, and $1/a^2 = e^{-2a_4}$.

$$\frac{d\ln\omega}{da_4} = \frac{1}{2}\cdot\frac{-2k^2e^{-2a_4}}{M^2+k^2e^{-2a_4}} = -\frac{k^2}{M^2a^2+k^2}.$$

Rule: the chain rule, $d\ln f/da_4 = f'/f$, with $d(e^{-2a_4})/da_4 = -2e^{-2a_4}$; then multiply numerator and denominator by $a^2 = e^{2a_4}$.

$$w_{\text{eff}}(A) = \frac{X}{E} = \frac{k^2}{3\,(M^2a^2+k^2)}.$$

Rule: multiply by $-\frac{1}{3}$. For $a \to 0$ the momentum term dominates and $w_{\text{eff}} \to 1/3$ (radiation); for $a \to \infty$ the mass dominates and $w_{\text{eff}} \to 0$ (dust): a time-varying, dark-matter-like equation of state (check `massive_mode_w_eff_law`; for $M = 0$ it is $1/3$ at every time, check `massless_mode_w_eff`). In the author's metric the record finds this law for the massive **bulk band** (odd brane parity; its edge at zero momentum is $\varepsilon(0) = 1.292292828069$): the lowest odd-parity level of the shell $n_2 = 1$ has $w_{\text{eff}}(A)$ falling monotonically from 0.239626 at $a_4 = -3$ to 2.268×10^{-3} at $a_4 = 4$, while the brane-band level of the same shell stays between 0.291594 and 0.333275, tending to $1/3$ (`Revision/dark_sector/dirac16complex/reports/independent-checks.json`, checks `bulk_band_dark_matter_law` and `brane_band_radiation_law`; Notebook 22b, In [5] and Figure 22b.2). At late times the bulk level falls like $1/a$ rather than like $1/a^2$: the ratio $w(4)/w(3.75) = 0.765177$ is close to $e^{-0.25} = 0.778801$, which the record reads as a term of $\varepsilon(k)$ linear in small k. The computed ground states ($N \le 688$, $T = 0$) are filled below the bulk edge and do not populate the bulk band. COMPUTED; that such quanta are present in the universe is not established.

The condensate, line by line. A homogeneous condensate is an exact solution of both fields, with $S = \bar{\Psi}\Psi$ constant (Chapter 9), and

$$\rho = mS + \tfrac{\lambda}{2}S^2, \qquad p_3 = p_t = p_0 = \tfrac{\lambda}{2}S^2$$

(checks `condensate_rho_p` and `condensate_satisfies_both_identities` of `derivation-checks.json`).

$$X = P_3 - P_t = 0 \quad\Longrightarrow\quad w_{\text{eff}}(A) = w_{\text{eff}}(B) = 0, \qquad w_{\text{eff}}(C) = -1.$$

Rule: equal pressures give $X = 0$; insert into the observer identities. The condensate is dust-like under A and B and cosmological-constant-like under C, constant in time in both (check `condensate_w_eff`).

$$\frac{p}{\rho} = \frac{\frac{\lambda}{2}S^2}{mS+\frac{\lambda}{2}S^2} = \frac{\lambda S/m}{2+\lambda S/m} = \frac{u}{2+u}, \qquad u = \frac{\lambda S}{m}.$$

Rule: divide numerator and denominator by $mS/2$. The ratio is constant (check `condensate_ratio_w`).

$$\frac{u}{2+u} = -0.764 \implies u = -0.764\,(2+u) \implies 1.764\,u = -1.528,$$

$$u = -\frac{1.528}{1.764} = -\frac{382}{441} = -0.866213151927.$$

Rule: multiply by $2+u$, bring $-0.764u$ to the left, divide by 1.764; 1528/1764 reduces by 4 to 382/441. Then $\rho/(mS) = 1 + u/2 = 250/441 = 0.566893424036$, so $\rho > 0$ exactly when $mS > 0$ (check `condensate_ratio_equal_unite_constant_w`; Notebook 22b, In [6]). This value of u is CHOSEN to give -0.764: one parameter tuned to one number, and it is the 8-dimensional ratio, not the observer's w_{eff}, which is 0 or -1.

$$\frac{u}{2+u} < -1 \quad \Longleftrightarrow \quad -2 < u < -1.$$

Rule: if $2+u > 0$, multiply by it: $u < -2-u$, so $u < -1$, and with $u > -2$ the window $-2 < u < -1$; if $2+u < 0$, multiplying reverses the inequality: $u > -2-u$, so $u > -1$, which contradicts $u < -2$. The ratio is phantom exactly in this window (`python-derive-eos.json`, check `condensate_phantom_interval`; Figure 22b.3), and the CHOSEN $u = -0.866$ lies outside it.

Mixtures with a condensate, line by line. Take a radiation-like gas, $X_g = E_g/3$, so that $dE_g/da_4 = -E_g$ and $E_g \propto e^{-a_4} \propto 1/a$, and add a condensate with $X_c = 0$ and E_c constant. Let $r = E_g/E_c = r_0/a$, with r_0 its value today.

$$w_{\text{eff}}(A) = \frac{X_g + X_c}{E_g + E_c} = \frac{E_g/3}{E_g + E_c} = \frac{1}{3}\,\frac{r}{1+r}.$$

Rule: the observer identity for the whole mixture; divide numerator and denominator by E_c.

$$\frac{dw}{da} = \frac{1}{3}\,\frac{1}{(1+r)^2}\,\frac{dr}{da} = -\frac{1}{3}\,\frac{r_0}{a^2(1+r)^2} \quad \Longrightarrow \quad w_0 = \frac{r_0}{3(1+r_0)}, \qquad w_a = \frac{r_0}{3(1+r_0)^2} > 0.$$

Rule: the quotient rule gives $d(r/(1+r))/dr = 1/(1+r)^2$, and $dr/da = -r_0/a^2$; at $a = 1$, $r = r_0$, and $w_a = -dw/da$. The mixture is **freezing**: as the gas redshifts, w moves toward the condensate's value (check `mixture_radiation_condensate_cpl`). The same holds under C, with w_0 lowered by 1.

$$\frac{r_0}{3(1+r_0)} - 1 = -0.861 \implies r_0 = 0.417\,(1+r_0) \implies r_0 = \frac{417}{583} = 0.715266,$$

$$w_a = \frac{r_0}{3(1+r_0)^2} = \frac{0.139}{1+r_0} = 0.081037.$$

Rule: add 1 and multiply by 3 ($3 \cdot 0.139 = 0.417$); collect r_0; for w_a use $r_0/(3(1+r_0)) = 0.139$, and $0.139 \cdot 583/1000 = 0.081037$. So r_0 is CHOSEN for $w_0 = -0.861$, and the slope then has the wrong sign: $+0.081037$, not -0.60 (check `mixture_C_matching_w0_unite`). With the computed gas $N = 688$, $\lambda = 0$ and today at $a_{4,\text{today}} = 2$, under C (`eos-checks.json`; Notebook 22b, In [7] and Figure 22b.4):

- the gas share 0.436703 is CHOSEN so that $w_{\text{eff}}(C) = -0.861$ today; then $w_a = 0.067014 > 0$, freezing (check `mixture_C_w0_unite_has_positive_wa`);

- the gas share 0.6807148417136 is CHOSEN so that the constant-w proxy over $1/3 \le a \le 1$ equals -0.764; the CPL fit of that mixture is $(w_0, w_a) = (-0.7841, 0.0603)$, opposite in the sign of w_a to the observed fit (check `mixture_C_constant_w_unite_reachable_only_with_freezing_cpl`);
- in the ratio definition, with a condensate of any constant ratio between -3 and 1, any gas share and $a_{4,\text{today}} \in \{0.5, 1, 1.5, 2\}$, no mixture comes closer to the observed pair than 0.600001 (check `ratio_mixture_scan_cannot_reach_unite_wa`).

Phantom and the crossing of -1. No computed state of dirac16complex crosses $w = -1$: every computed state with $E \neq 0$ has $X \ge 0$, and all of them have $E > 0$ except the $N = 8$, $\lambda > 0$ states, which have $E < 0$ (from the uniform-gas exchange functional; with the exact Fock exchange $|E| \le 2.04 \times 10^{-13}$, as stated above) and $X = 0$ exactly; the 41 $N = 8$, $\lambda = 0$ states have $E = 0$ and no equation of state (`Revision/docs/DARK_SECTOR_HYPOTHESES.md`, section 5.5). Modes of dirac16complex with extra-time momentum lie outside the good sector and are OPEN.

22.19 dirac16complex00: modes of both energy signs, the populations and the models M1 to M5

Modes and their energies. dirac16complex00 is a classical field of 16 commuting complex components. A plane wave $\Phi = u\, e^{i(kx_1 + qx_5 - \omega x_4)}$ with 3-momentum k and extra-time momentum q (physical momenta in a frozen local frame) obeys $\omega u = hu$ with $h^2 = (m^2 + k^2 - q^2)\, I_{16}$, so

$$\omega^2 = m^2 + k^2 - q^2$$

(check `mode_dispersion_h_squared`): the extra times are time-like, and q enters with the opposite sign to k. The charge density is the Krein form $Q = \Phi^\dagger B \Phi$ of Chapter 10, conserved by the frozen-frame evolution (check `mode_generator_B_selfadjoint`); in the field theory the U(1) charge obeys a PROVED local conservation law, and its total is constant only under the ASSUMED no-flux condition at $z = \pi/2$ (Section 22.25). For the three test cases $(m, k, q) = (3, 4, 0)$, $(5, 0, 3)$ and $(4, 4, 4)$, with $\omega = \pm 5, \pm 4, \pm 4$, every real-frequency eigenspace has dimension 8 and a Krein form of signature (4,4), and on it $\rho = \omega Q$ exactly (checks `mode_krein_inertia_*` and `mode_emt_*`; the second implementation finds $\rho = +1.25$ and -1.25 at $\omega = 1.25$ for $Q = +1$ and -1, check `B_krein_signed_energy`). The three test cases are illustrations, not a proof of a general statement. The general statement is Theorem 10.1 of Chapter 10, the one-particle Krein-inertia theorem of the pairing record, which holds for the plane waves of flat 4+4 space and of the frozen local frame used here (PROVED: `Revision/pairing/reports/wolfram-pairing.json`, check `Q_one_particle_Krein_inertia_real_frequencies`; `Revision/pairing/reports/python-pairing.json`, check `Q.one_particle_Krein_inertia_proof`): at EVERY real frequency each eigenspace has dimension 8 and a Krein form of signature (4,4), so it holds waves with $Q > 0$ and waves with $Q < 0$. With $\rho = \omega Q$, the energy of a single wave, which the record checks exactly on the three test cases, the two signs of Q are two signs of the energy. So at every real frequency there are modes of BOTH energy signs: the classical energy of this field is unbounded below. A component of negative classical energy is called **ghost-like**; it is not an established physical state.

Growing modes. For $q^2 > m^2 + k^2$ the frequency is imaginary and the mode grows (Chapter 8). In the deflating field the physical extra-time momentum is qa, which grows with a, so every mode

with $q \neq 0$ reaches a **turning point** $a_{\rm turn}$, where $\omega = 0$, and grows after it: the deflation drives the growth. The field-equation implementation B measures, past the turning point $a_{\rm turn} = 1.8433886$, the growth rate 0.54337468 against the adiabatic estimate 0.55114035 (check `B_growth_rate`).

The adiabatic gas, line by line. In the deflating local model (the warp frozen at one hidden position: an APPROXIMATION) a mode has $\omega^2 = m^2 + k^2/a^2 - q^2a^2$, its Krein charge is an adiabatic invariant, and its energy density is $\rho_i = \omega_i Q_i \propto \omega_i$. Under B the observer identity gives $w_{\rm eff}(B) = -\frac{1}{3}\, d\ln\rho_i/d\ln a = -\frac{1}{3}\, d\ln\omega_i/d\ln a$.

$$\ln\omega = \tfrac{1}{2}\ln\left(m^2 + k^2a^{-2} - q^2a^2\right), \qquad \frac{d\ln\omega}{d\ln a} = \frac{-k^2a^{-2} - q^2a^2}{\omega^2}.$$

Rule: as in Section 22.18, with $d(a^{-2})/d\ln a = -2a^{-2}$ and $d(a^2)/d\ln a = 2a^2$; the factor $\frac{1}{2}$ cancels the factors 2.

$$\varepsilon_i = w_{\rm eff}(B)_i = \frac{k^2/a^2 + q^2a^2}{3\,\omega_i^2} \;\geq\; 0.$$

Rule: multiply by $-\frac{1}{3}$; both terms of the numerator are ≥ 0, and $\omega^2 > 0$ for a real frequency. The extra-time momentum RAISES ε: its pressure is negative, which increases X (checks `wkb_mode_gas_conservation`, `wkb_epsilon_sign_free`). For a mixture of components (check `mixture_weighted_average`):

$$w_{\rm eff}(B) = \frac{\sum_i \varepsilon_i\rho_i}{\sum_i \rho_i}, \qquad w_{\rm eff}(C) = \frac{\sum_i \varepsilon_i\rho_i}{\sum_i \rho_i} - 1.$$

No crossing without a ghost. If every $\rho_i \geq 0$, the first fraction is an average of numbers $\varepsilon_i \geq 0$ with weights $\rho_i \geq 0$, so it is ≥ 0: $w_{\rm eff}(B) \geq 0$ and $w_{\rm eff}(C) \geq -1$ at every a at which every component has a real frequency. A mode with $q \neq 0$ loses its real frequency past its turning point $a_{\rm turn}$, where the theorem does not apply. A crossing of -1 needs the numerator to change sign, that is a component with $\rho_i < 0$: negative classical energy, a ghost-like component. PROVED within the adiabatic model (`Revision/dark_sector/dirac16complex00/eos-theory.json`, key `wkb.theorem`). The only phantom value without negative energy is the constant ratio of a condensate with $-2 < u < -1$ (Section 22.18), which is not the observer's $w_{\rm eff}$.

The models. Each model is normalised to total energy density 1 at $a = 1$ and read under C unless stated. The parameters marked CHOSEN were fixed so that the model reproduces an observed number; that number is then an input, NOT an output (`eos-theory.json`, key `models`):

model	content	CHOSEN parameters
M1	condensate, an exact solution	$u = \lambda S/m$; the ratio is -0.764 at $u = -382/441$ (CHOSEN)
M2	positive-energy gas with $q = 0$	$k^2/(k^2+m^2) = 417/1000$ at $a = 1$, CHOSEN for $w_0 = -0.861$
M3	one positive-energy extra-time mode, $k = 0$	$s = q^2/m^2 = 417/1417$, CHOSEN for $w_0 = -0.861$

model	content	CHOSEN parameters
M4	condensate plus a positive extra-time mode	$s = 264037/403037$ and the mode share $\Omega_q = 57963/264037$, both CHOSEN so that the tangent is $(-0.861, -0.60)$
M5	as M4 plus a GHOST-LIKE part: massless modes of negative energy $-G/a$	$G = 3/10$ (a stated choice); s and Ω_q CHOSEN so that the fit over $1/2 \le a \le 1$ is $(-0.861, -0.60)$

M2 line by line, with $r = k^2/m^2$ and $q = 0$: $\varepsilon = (r/a^2)/(3(1+r/a^2)) = r/(3(r+a^2))$; at $a = 1$ this is $\frac{1}{3}\, r/(1+r) = \frac{1}{3}\, k^2/(k^2+m^2)$, and setting it to 0.139 (so that $w_{\text{eff}}(C) = -0.861$) gives $k^2/(k^2+m^2) = 0.417$. Its derivative is $dw/da = -\frac{2}{3}\, ra/(r+a^2)^2$, so $w_a = \frac{2}{3}\, r/(1+r)^2 = \frac{2}{3} \cdot 0.417 \cdot 0.583 = 0.162074$ (using $r/(1+r) = 0.417$ and $1/(1+r) = 0.583$): freezing (check `M2_tangent_exact`). Under B the same model has $w_{\text{eff}} = \frac{1}{3}\, k^2/(k^2+m^2a^2)$, from 1/3 to 0: the dark-matter-like law of dirac16complex00. M3 with $k = 0$: $w_{\text{eff}}(C) = -1 + \frac{1}{3}\, sa^2/(1-sa^2)$, which is ≥ -1 for $a < a_{\text{turn}}$, where $1 - sa^2 > 0$; at $a = 1$, $s/(1-s) = 0.417$ gives $s = 417/1417$, and $w_a = -\frac{2}{3}\, s/(1-s)^2 = -0.393926$: thawing; the turning point is at $a_{\text{turn}} = 1/\sqrt{s} = 1.8434$ (check `M3_tangent_exact`). Past it the frequency of the mode is imaginary, the mode grows (check `B_growth_rate`), and its energy density becomes large and negative (implementation B: $\rho/Q = -2.9228317 \times 10^{61}$ at $a = 2.2$, `Revision/dark_sector/dirac16complex00/results/independent-numerics.json`), so the bound $w_{\text{eff}}(C) \ge -1$ does not extend past a_{turn}.

The resulting CPL numbers under C (tangent; least-squares fit over $1/2 \le a \le 1$; the constant-w proxy, the mean of $w(a)$ over that range; records `M2_tangent_exact` to `M5_without_ghost_no_crossing` of `python-derive-eos.json`; Notebook 22b, In [8] and Figure 22b.5):

model	tangent w_0	tangent w_a	fit w_0	fit w_a	proxy	crosses -1 on $1/3 \le a \le 1$?
M1	-1	0	-1	0	-1	no
M2	-0.861	$+0.162074$	-0.8655	0.2173	-0.8112	no
M3	-0.861	-0.393926	-0.8734	-0.2196	-0.9283	no
M4	-0.861	-0.600	-0.8832	-0.2203	-0.9383	no
M5	-0.8396	-1.0399	-0.861	-0.600	-1.011	at $a = 0.7791$

The last column is the record's search for a crossing on $1/3 \le a \le 1$; it says nothing about $a > 1$. Past the turning point a_{turn} of an extra-time mode (M3: 1.8434, M4: 1.2355, M5: 1.3271) the frequency of that mode is imaginary and the theorem above does not apply.

- M4: $s = 0.6551$, $\Omega_q = 0.2195$. Its w does not cross -1 on $1/300 \le a \le 1$: the smallest $w_{\text{eff}}(C)$ over $a = 1/300, \ldots, 1$ is -0.999999214228 (check `M4_never_phantom`); by the theorem above, $w_{\text{eff}}(C) \ge -1$ holds up to the turning point of its extra-time mode. The phantom past $w_0 + w_a = -1.461$ belongs to the straight CPL line, not to the model. Its extra-time mode reaches its turning point at $a_{\text{turn}} = 1.2355$ and then grows without bound; past a_{turn}

its frequency is imaginary, the theorem above does not apply, and $w \geq -1$ is not established there. Under B or in the ratio, M4 is not dark energy ($w \geq 0$).

- M5: $s = 0.5678$, $\Omega_q = 0.5947$, condensate share 0.7053; its w crosses -1 at $a = 0.7791$ (the observed line at 0.7683), and without its ghost-like part it does not cross on $1/3 \leq a \leq 1$, nor, by the theorem above, before the turning point $a_{\text{turn}} = 1.3271$ of its extra-time mode; past that point this is not established (checks `M5_crosses_minus_1`, `M5_without_ghost_no_crossing`). INTERPRETATION, as in the record: because s and Ω_q were chosen so that the fit equals the observed line, a crossing close to the line's crossing is expected and is not an independent agreement.
- Implementation B rebuilds M2 to M5 from solutions of the full 16-component field equation with the author's gammas and agrees with A to within 0.00083082726 in every tangent, fit and proxy; it finds the M5 crossing at $a = 0.77905405$ (check `B_vs_A_M5_crossing` of `python-independent-numerics.json`).

The constant-w proxy is not the supernova fit. Applied to the observed CPL line itself, the mean of $w(a)$ is -1.011 over $1/2 \leq a \leq 1$ and -1.061 over $1/3 \leq a \leq 1$ (check `unite_line_fit_proxy`), not -0.764: the supernova constant-w fit weights the data, which no part of the record computes.

The comparison with the observed values. Every match is labelled with the number of parameters CHOSEN for it:

observed value	**dirac16complex**	**dirac16complex00**	**by construction?**
constant $w = -0.764$	condensate ratio at $u = -382/441$; mixture under C with gas share 0.6807 (proxy; slope $+0.0603$)	condensate ratio at $u = -382/441$, $\lambda < 0$	yes: one parameter for one number
$w_0 = -0.861$	mixture under C with gas share 0.436703: $w_a = 0.067014$, freezing	M2 ($w_a = +0.162074$), M3 ($w_a = -0.393926$)	yes: one parameter for one number
$(w_0, w_a) = (-0.861, -0.60)$	not reached: gas $\lvert w_a \rvert \leq 0.030371$; ratio mixtures never closer than 0.600001	M4 tangent exactly; M5 fit exactly	yes: two parameters for two numbers
crossing of -1 at $a = 0.7683$	no crossing in any computed state	only M5, with a ghost-like part: $a = 0.7791$	the fit by construction; the ghost share a stated choice

A model with as many tuned parameters as matched numbers reproduces them by construction and is NOT a prediction. Nothing in the record selects the populations of M2 to M5, the mixture shares or the condensate's u. What is NOT tuned are the signs and the ranges: for positive-energy content of real frequency $w_{\text{eff}}(C) \geq -1$ in the adiabatic model (for an extra-time mode only before its turning point); every computed state of dirac16complex has $X \geq 0$; the computed gas has the

thawing sign with $|w_a| \leq 0.030371$; gas-condensate mixtures are freezing; M4, tuned to the observed tangent, does not cross -1 before its turning point $a_{\text{turn}} = 1.2355$ (checked on $1/300 \leq a \leq 1$).

22.20 Example: the dark-sector numbers, reproduced from the record (Notebook 22b)

Notebook 22b reads the committed outputs of the dark-sector record: the table `Revision/dark_sector/dirac16complex/outputs/eos-history.csv` (energy, pressures and dw_{eff}/da_4 of the 14 Kohn-Sham series with nonzero energy, 41 slices each), the summary `eos-summary.json` and the independent free-gas output `independent-free-gas.json` in the same folder, and the model record `Revision/dark_sector/dirac16complex00/eos-theory.json` with the field-equation results `results/independent-numerics.json` of the same folder; and it checks that every check of the six check reports of the dark-sector record is PASS (the four of `Revision/dark_sector/dirac16complex`, among them the report `ks-history-run.json` of the 615 runs of the Kohn-Sham solver, and the two of `Revision/dark_sector/dirac16complex00`). It then tests the energy balance and the observer identities of Section 22.17 on the computed history, reproduces the band and the tangents of the gas, the bulk and brane bands, the condensate ratio and its phantom window, the mixture laws, and the models M2 to M5 with their crossings, each number asserted against its record file and check, and draws five figures. It needs only numpy and matplotlib, takes a few seconds (the build and check runs of 2026-10-08 recorded in its provenance file took 7.5 and 7.4 seconds; on a computer busy with other work it takes longer) and ends with the line ALL 28 CHECKS PASSED (notebook 22b).

22.21 How to run Notebook 22b

Step 1. What this notebook does and what it needs.

Notebook 22b (The dark-sector hypotheses: what the Revision record finds) is the file `Revision/textbook/notebooks/22b_dark_sector.ipynb` of the repository Dirac_claude. It reads the committed outputs and reports of the Revision dark-sector record for dirac16complex and dirac16complex00 and reproduces their key numbers: the observer identities w_eff(A) = w_eff(B) = X/E and w_eff(C) = X/E - 1 on the Kohn-Sham history, the radiation-like band of the Kohn-Sham gas and its CPL tangents, the dark-matter-like law of the massive bulk band, the condensate ratio, the gas and condensate mixtures, and the models M2 to M5 of dirac16complex00 against the Unite CPL values, every tuned parameter labelled CHOSEN. Every reproduced number is checked against its record file and check, and five teaching figures are drawn. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at

a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are

those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 22b_dark_sector.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 22b_dark_sector.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 22b_dark_sector.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- `Revision/textbook/figures/22b.captions.json`

- `Revision/textbook/figures/22b_1_gas_three_definitions.png`
- `Revision/textbook/figures/22b_2_bulk_and_brane_bands.png`
- `Revision/textbook/figures/22b_3_condensate_ratio.png`
- `Revision/textbook/figures/22b_4_mixtures_against_unite.png`
- `Revision/textbook/figures/22b_5_models_against_unite.png`

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 28 CHECKS PASSED (notebook 22b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file `Revision/textbook/notebooks/22b_dark_sector.ipynb` inside the folder Dirac_claude.
- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 22b_dark_sector.ipynb
python -m nbconvert --execute --inplace 22b_dark_sector.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

22.22 Notebook 22b: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/22b_dark_sector.ipynb`: every cell in order, with the output that the stored, executed notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

The author asked that two hypotheses be investigated: that dirac16complex
(Hypothesis) and dirac16complex00 (Hypothesis00) provide a possible physical
mechanism for a time-varying dark-energy and/or dark-matter equation of state. The
Revision record has investigated them, in the folders
‘Revision/dark_sector/dirac16complex‘ and ‘Revision/dark_sector/dirac16complex00‘.
This notebook reads the committed outputs and reports of that record and reproduces
its key numbers, each one checked against its record file and check:

- the observer identities $w_{\rm eff}(A) = w_{\rm eff}(B) = X/E$ and
  $w_{\rm eff}(C) = X/E - 1$, tested on the computed Kohn-Sham history, together
  with the energy balance $dE/da_4 = -3X$;
- the Kohn-Sham gas of dirac16complex: radiation-like, $X/E$ rising toward $1/3$,
  and its CPL tangents (small, thawing sign);
- the massive bulk band, which shows the dark-matter-like fall from about $1/3$
  toward 0, and the brane band, which does not;
- the condensate: its constant ratio $p/\rho = u/(2 + u)$ with $u = \lambda S/m$, and
```

```
  the value of $u$ CHOSEN to give the Unite constant $w = -0.764$;
- mixtures of the gas with a condensate: freezing, never the Unite slope;
- the models M2 to M5 of dirac16complex00: their tangents and fits against the
  Unite CPL line, every tuned parameter labelled CHOSEN, and the crossing of
  $w = -1$, which happens only with a ghost-like component.

It draws five figures and needs no Rust. Units: $H = m = 1$, as in the record.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 22b (The dark-sector hypotheses: what the Revision record finds) is the file
‘Revision/textbook/notebooks/22b_dark_sector.ipynb‘ of the repository Dirac_claude. It
reads the committed outputs and reports of the Revision dark-sector record for
dirac16complex and dirac16complex00 and reproduces their key numbers: the observer
identities w_eff(A) = w_eff(B) = X/E and w_eff(C) = X/E - 1 on the Kohn-Sham history, the
radiation-like band of the Kohn-Sham gas and its CPL tangents, the dark-matter-like law
of the massive bulk band, the condensate ratio, the gas and condensate mixtures, and the
models M2 to M5 of dirac16complex00 against the Unite CPL values, every tuned parameter
labelled CHOSEN. Every reproduced number is checked against its record file and check,
and five teaching figures are drawn. It needs a computer with Windows 11, macOS or Linux,
an internet connection for the installation, the program Git, and Python 3.12 or newer
(the notebooks were built with Python 3.14.5) with these packages at exactly these
versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10,
nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook
itself imports numpy and matplotlib; the other packages run Jupyter, the program that
shows and runs notebooks. It does not need Rust.

**Step 2. Install Git and Python (once per computer).**

A terminal is the window in which you type commands: on Windows the app Windows
PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any
terminal (it runs the shell bash). Type every command of this section into a terminal
exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the
default choices. Download Python from https://www.python.org/downloads/ and run the
installer; in its first window tick the box "Add python.exe to PATH" before you click
"Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then
open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

    xcode-select --install

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these
two commands:
```

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in

this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 22b_dark_sector.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the

address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 15 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 22b_dark_sector.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 22b_dark_sector.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/22b.captions.json‘
- ‘Revision/textbook/figures/22b_1_gas_three_definitions.png‘
- ‘Revision/textbook/figures/22b_2_bulk_and_brane_bands.png‘
- ‘Revision/textbook/figures/22b_3_condensate_ratio.png‘
- ‘Revision/textbook/figures/22b_4_mixtures_against_unite.png‘
- ‘Revision/textbook/figures/22b_5_models_against_unite.png‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every figure file of this notebook exists
ALL 28 CHECKS PASSED (notebook 22b)
```

and the notebook must show 5 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

      python -m ipykernel install --user --name python3

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/22b_dark_sector.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 22b_dark_sector.ipynb
      python -m nbconvert --execute --inplace 22b_dark_sector.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 22b (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
```

```
#
# Notebook 22b (The dark-sector hypotheses: what the Revision record finds) is the file
# Revision/textbook/notebooks/22b_dark_sector.ipynb of the repository Dirac_claude. It
# reads the committed outputs and reports of the Revision dark-sector record for
# dirac16complex and dirac16complex00 and reproduces their key numbers: the observer
# identities w_eff(A) = w_eff(B) = X/E and w_eff(C) = X/E - 1 on the Kohn-Sham history,
# the radiation-like band of the Kohn-Sham gas and its CPL tangents, the dark-matter-like
# law of the massive bulk band, the condensate ratio, the gas and condensate mixtures,
# and the models M2 to M5 of dirac16complex00 against the Unite CPL values, every tuned
# parameter labelled CHOSEN. Every reproduced number is checked against its record file
# and check, and five teaching figures are drawn. It needs a computer with Windows 11,
# macOS or Linux, an internet connection for the installation, the program Git, and
# Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages
# at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0,
# jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert
# 7.16.6. The notebook itself imports numpy and matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
```

```
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
```

```
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 22b_dark_sector.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 15
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 22b_dark_sector.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 22b_dark_sector.ipynb (in the same folder, without running the cells again) to read the
# printed lines and see the figures.
#
```

```
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/22b.captions.json
# - Revision/textbook/figures/22b_1_gas_three_definitions.png
# - Revision/textbook/figures/22b_2_bulk_and_brane_bands.png
# - Revision/textbook/figures/22b_3_condensate_ratio.png
# - Revision/textbook/figures/22b_4_mixtures_against_unite.png
# - Revision/textbook/figures/22b_5_models_against_unite.png
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every figure file of this notebook exists
#     ALL 28 CHECKS PASSED (notebook 22b)
#
# and the notebook must show 5 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/22b_dark_sector.ipynb inside the folder Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
```

```
#     python -m jupyterlab 22b_dark_sector.ipynb
#     python -m nbconvert --execute --inplace 22b_dark_sector.ipynb
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
#
# ------------------------------------------------------------------------------------------
# ==========================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ==========================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "22b"  # this notebook: chapter 22, example b


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
```

```
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order
```

```
def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 22b complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Equation of state** $w$: a pressure divided by an energy density. **Observer**:
  someone who, like us, measures only the three space directions $x_1, x_2, x_3$
  and the time $x_4$.
- **$\rho_4$**: the energy density that this observer assigns to 3-space. It is NOT
  fixed by the field equations; the record states three definitions: (A) the extra
  times are closed with a fixed coordinate period, $\rho_4 \propto E e^{-3a_4}$;
  (B) $\rho_4$ per unit extra-time coordinate volume, $\rho_4 \propto E e^{-3a_4}$;
  (C) $\rho_4$ per unit proper 7-volume, $\rho_4 \propto E$. This is an
  ASSUMPTION about the observer.
- **$E, P_3, P_t$**: the energy and the integrated pressures of 3-space and of the
  extra times of one state; **$X = P_3 - P_t$**.
- **$w_{\rm eff}$**: the equation of state the observer infers from how fast
  $\rho_4$ thins out, $w_{\rm eff} = -1 - \frac{1}{3}\,d\ln\rho_4/d\ln a$, with the
  observer's scale factor $a = e^{a_4 - a_{4,\rm today}}$.
- **Radiation-like**: $w_{\rm eff} = 1/3$; **dust-like** (dark-matter-like):
  $w_{\rm eff} = 0$; **cosmological-constant-like**: $w = -1$; **phantom**:
  $w < -1$.
- **CPL**: $w(a) = w_0 + w_a(1 - a)$. **Tangent**: $w_0 = w(1)$ and
```

```
  $w_a = -dw/da$ at $a = 1$. **Fit**: the least-squares straight line in $1 - a$
  over a range of $a$. **Thawing**: $w_a < 0$; **freezing**: $w_a > 0$.
- **Unite values**: the supernova fits quoted by the record, $w = -0.764$ (constant
  $w$) and $(w_0, w_a) = (-0.861, -0.60)$.
- **CHOSEN**: a parameter fixed so that a model reproduces a Unite number; that
  number is then an input, NOT a prediction.
- **Ghost-like component**: a component of negative classical energy.
```

Text cell:

```
## 4. The physical and mathematical situation

The author's metric has the 3-space scale factor $e^{a_4}\sin^{1/6}z$ and the
extra-time scale factor $e^{-a_4}\sin^{1/6}z$: as $a_4$ grows, 3-space inflates and
the three extra times $x_5, x_6, x_7$ deflate exponentially. The proper 7-volume
element is $\cos z$, independent of $a_4$, and for a homogeneous source the
conservation law gives exactly (record `derivation-checks.json`)
$$\frac{dE}{da_4} = -3X, \qquad X = P_3 - P_t .$$
With $\rho_4 \propto E e^{-3(1-s)a_4}$ ($s = 0$ for A and B, $s = 1$ for C) and
$d\ln a = da_4$:
$$w_{\rm eff} = -1 - \frac{1}{3}\left(\frac{1}{E}\frac{dE}{da_4} - 3(1 - s)\right)
= -1 + \frac{X}{E} + 1 - s = \frac{X}{E} - s .$$
The Kohn-Sham states are instantaneous ground states along the PRESCRIBED
BACKGROUND $a_4 = AHx_4$ (a test field without back-reaction), with the ASSUMED Z2
brane. The models of dirac16complex00 are adiabatic (WKB) gases of plane-wave modes,
an APPROXIMATION that the record measures with a second implementation.
```

Text cell:

```
## 5. The records

The next cell loads the modules, reads the six check reports of the dark-sector
record and checks that every one of their checks is PASS, then reads the table of
the Kohn-Sham history and the four output files whose numbers the later cells
reproduce.
```

In [2]:

```
import csv  # reads the CSV tables of the Revision record
from fractions import Fraction  # exact fractions such as -382/441

import numpy as np  # arrays of numbers and their arithmetic

D16 = "Revision/dark_sector/dirac16complex"  # the record of dirac16complex
D00 = "Revision/dark_sector/dirac16complex00"  # the record of dirac16complex00
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order


def read_json(relative):
    """Read a JSON record of the repository into a Python dictionary."""
```

```
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))


REPORTS = {"derive": f"{D16}/reports/derivation-checks.json",
           "history": f"{D16}/reports/ks-history-run.json",
           "eos": f"{D16}/reports/eos-checks.json",
           "independent": f"{D16}/reports/independent-checks.json",
           "derive00": f"{D00}/reports/python-derive-eos.json",
           "numerics00": f"{D00}/reports/python-independent-numerics.json"}
EXPECTED = {"derive": 30, "history": 5, "eos": 13, "independent": 9,
            "derive00": 49, "numerics00": 28}  # the checks each report holds
for key, relative in REPORTS.items():
    entries = read_json(relative)["checks"]
    passed = [entry for entry in entries if entry["verdict"] == "PASS"]
    say(f"{relative}: {len(passed)} of {len(entries)} checks PASS")
    check(len(passed) == len(entries) == EXPECTED[key],
          f"all {EXPECTED[key]} checks of the report {key} are PASS",
          record=f"{relative}, all checks")

with open(repository_file(f"{D16}/outputs/eos-history.csv"), newline="",
          encoding="utf-8") as handle:
    rows = list(csv.DictReader(handle))
COLUMNS = ["N", "a4", "E", "P3", "Pt", "dw_eff_da4"]
SERIES = {}  # series name -> {column name: numpy array over the slices}
for name in sorted({row["series"] for row in rows}):
    mine = [row for row in rows if row["series"] == name]
    SERIES[name] = {column: np.array([float(row[column]) for row in mine])
                    for column in COLUMNS}
GAS = [name for name in SERIES if SERIES[name]["N"][0] in (136, 688)]
summary = read_json(f"{D16}/outputs/eos-summary.json")
free_gas = read_json(f"{D16}/outputs/independent-free-gas.json")
theory00 = read_json(f"{D00}/eos-theory.json")
numerics00 = read_json(f"{D00}/results/independent-numerics.json")
say(f"{len(rows)} rows read: {len(SERIES)} series, {len(GAS)} of them with "
    "N = 136 or 688")
check(all(len(SERIES[name]["a4"]) == 41 for name in SERIES)
      and all(np.all(np.diff(SERIES[name]["a4"]) > 0) for name in SERIES),
      "every series has 41 slices in increasing order of a4")
```

Out [2]:

```
Revision/dark_sector/dirac16complex/reports/derivation-checks.json: 30 of 30 checks PASS
PASS all 30 checks of the report derive are PASS
     reproduces Revision/dark_sector/dirac16complex/reports/derivation-checks.json, all
    checks
Revision/dark_sector/dirac16complex/reports/ks-history-run.json: 5 of 5 checks PASS
PASS all 5 checks of the report history are PASS
     reproduces Revision/dark_sector/dirac16complex/reports/ks-history-run.json, all
    checks
Revision/dark_sector/dirac16complex/reports/eos-checks.json: 13 of 13 checks PASS
PASS all 13 checks of the report eos are PASS
     reproduces Revision/dark_sector/dirac16complex/reports/eos-checks.json, all checks
Revision/dark_sector/dirac16complex/reports/independent-checks.json: 9 of 9 checks PASS
```

```
PASS all 9 checks of the report independent are PASS
     reproduces Revision/dark_sector/dirac16complex/reports/independent-checks.json, all
    checks
Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json: 49 of 49 checks
    PASS
PASS all 49 checks of the report derive00 are PASS
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json, all
    checks
Revision/dark_sector/dirac16complex00/reports/python-independent-numerics.json: 28 of 28
    checks PASS
PASS all 28 checks of the report numerics00 are PASS
     reproduces Revision/dark_sector/dirac16complex00/reports/python-independent-
    numerics.json, all checks
574 rows read: 14 series, 10 of them with N = 136 or 688
PASS every series has 41 slices in increasing order of a4
```

Text cell:

```
## 6. The observer's equation of state on the Kohn-Sham gas

The next cell tests the energy balance $dE/da_4 = -3X$ on the computed history
(fourth-order differences of $E$ against $X$ from the solver's integrals), then
computes $w_{\rm eff}$ directly from its definition, by differentiating
$\ln\rho_4$ for the definitions A, B and C, and compares with $X/E$ and $X/E - 1$.
It checks the band of $X/E$ over all gas series and that $X/E$ rises with $a_4$,
and draws the histories (Figure 1).
```

In [3]:

```
def x_over_e(name):
    """X/E = (P3 - Pt)/E of one series at its 41 slices."""
    return (SERIES[name]["P3"] - SERIES[name]["Pt"]) / SERIES[name]["E"]


gas = SERIES["N688_lam0"]  # the gas of N = 688 quanta with lambda = 0
a4, E = gas["a4"], gas["E"]
X = gas["P3"] - gas["Pt"]
h = a4[1] - a4[0]  # the step 0.05 between two slices


def derivative(f):
    """df/da4 at the slices 2 to 38: fourth-order central differences."""
    return (f[:-4] - 8 * f[1:-3] + 8 * f[3:-1] - f[4:]) / (12 * h)


worst = 0.0  # the largest relative violation of dE/da4 = -3X
for name in GAS:
    E_n = SERIES[name]["E"]
    X_n = SERIES[name]["P3"] - SERIES[name]["Pt"]
    violation = np.max(np.abs(derivative(E_n) + 3 * X_n[2:-2])) / np.max(np.abs(E_n))
    worst = max(worst, violation)
report("largest |dE/da4 + 3X|/max|E| over the 10 gas series", f"{worst:.3e}")
```

```
check(f"{worst:.3e}" == "1.235e-07", "the energy balance dE/da4 = -3X on the data",
      record=f"{D16}/reports/eos-checks.json, "
             "check conservation_dE_da4_equals_minus_3X")

rho4_AB = E * np.exp(-3 * a4)  # definitions A and B: rho4 ~ E e^(-3 a4)
rho4_C = E  # definition C: rho4 ~ E
w_AB = -1 - derivative(np.log(rho4_AB)) / 3  # -1 - (1/3) d ln rho4/d ln a
w_C = -1 - derivative(np.log(rho4_C)) / 3
error_AB = np.max(np.abs(w_AB - (X / E)[2:-2]))
error_C = np.max(np.abs(w_C - (X / E - 1)[2:-2]))
report("largest |w_eff(A, B) - X/E| from the dilution", f"{error_AB:.1e}")
report("largest |w_eff(C) - (X/E - 1)| from the dilution", f"{error_C:.1e}")
check(max(error_AB, error_C) < 1e-7,
      "w_eff(A) = w_eff(B) = X/E and w_eff(C) = X/E - 1 on the data",
      record=f"{D16}/reports/derivation-checks.json, checks "
             "w_eff_A_equals_X_over_E and w_eff_C_equals_X_over_E_minus_1")

ratios = np.concatenate([x_over_e(name) for name in GAS])
report("X/E of N688_lam0 at a4 = 0 and a4 = 2", f"{X[0] / E[0]:.4f}, "
       f"{X[-1] / E[-1]:.4f}")
report("X/E over all gas series, smallest and largest",
       f"{ratios.min():.6f}, {ratios.max():.6f}")
report("w_eff(C) over all gas series, smallest and largest",
       f"{ratios.min() - 1:.6f}, {ratios.max() - 1:.6f}")
check(f"{ratios.min():.6f}" == "0.292893" and f"{ratios.max():.6f}" == "0.328105",
      "the gas is radiation-like: X/E in [0.292893, 0.328105]",
      record=f"{D16}/reports/eos-checks.json, check gas_radiation_like_band")
check(all(np.all(np.diff(x_over_e(name)) > 0) for name in GAS),
      "X/E rises with a4 in every gas series (toward 1/3, not toward 0)",
      record=f"{D16}/reports/eos-checks.json, "
             "check gas_X_over_E_rises_toward_one_third")

fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
for colour, name in zip(PALETTE, ["N136_lam0", "N688_lam0", "N688_lamp2"]):
    left.plot(SERIES[name]["a4"], x_over_e(name), color=colour, label=name)
    right.plot(SERIES[name]["a4"], x_over_e(name) - 1, color=colour, label=name)
left.axhline(1 / 3, color="black", linestyle="--", linewidth=0.8,
             label="radiation, 1/3")
right.axhline(-2 / 3, color="black", linestyle="--", linewidth=0.8,
              label="1/3 - 1")
left.set_title("definitions A and B: $w_{eff} = X/E$")
right.set_title("definition C: $w_{eff} = X/E - 1$")
for ax in (left, right):
    ax.set_xlabel("$a_4$ along the prescribed history")
    ax.legend(fontsize=8)
left.set_ylabel("$w_{eff}$")
save_figure(fig, "gas_three_definitions",
            "The equation of state $w_{eff}$ that a 3-space observer infers from "
            "the thinning of the Kohn-Sham gas of dirac16complex along the "
            "prescribed history, for three recorded series; horizontal axis $a_4$ "
            "from 0 to 2 (one unit is one e-fold of inflation of 3-space and of "
            "deflation of the extra times), vertical axis $w_{eff}$ (a pure "
            "number). Left, definitions A and B of the observer density: "
            "$w_{eff} = X/E$ rises from about 0.29 toward the radiation value 1/3 "
```

```
            "(dashed). Right, definition C: the same curves moved down by exactly "
            "1, between about $-0.71$ and $-0.67$, never near $-1$. The observer "
            "assumption alone decides whether the same gas reads as radiation or "
            "as dark energy.")
```

Out [3]:

```
RESULT largest |dE/da4 + 3X|/max|E| over the 10 gas series = 1.235e-07
PASS the energy balance dE/da4 = -3X on the data
     reproduces Revision/dark_sector/dirac16complex/reports/eos-checks.json, check
    conservation_dE_da4_equals_minus_3X
RESULT largest |w_eff(A, B) - X/E| from the dilution = 1.7e-08
RESULT largest |w_eff(C) - (X/E - 1)| from the dilution = 1.7e-08
PASS w_eff(A) = w_eff(B) = X/E and w_eff(C) = X/E - 1 on the data
     reproduces Revision/dark_sector/dirac16complex/reports/derivation-checks.json,
    checks w_eff_A_equals_X_over_E and w_eff_C_equals_X_over_E_minus_1
RESULT X/E of N688_lam0 at a4 = 0 and a4 = 2 = 0.2929, 0.3183
RESULT X/E over all gas series, smallest and largest = 0.292893, 0.328105
RESULT w_eff(C) over all gas series, smallest and largest = -0.707107, -0.671895
PASS the gas is radiation-like: X/E in [0.292893, 0.328105]
     reproduces Revision/dark_sector/dirac16complex/reports/eos-checks.json, check
    gas_radiation_like_band
PASS X/E rises with a4 in every gas series (toward 1/3, not toward 0)
     reproduces Revision/dark_sector/dirac16complex/reports/eos-checks.json, check
    gas_X_over_E_rises_toward_one_third
```

Out [3]:

```
Figure 22b.1 saved as Revision/textbook/figures/22b_1_gas_three_definitions.png
```

Text cell:

```
## 7. The CPL tangents of the gas

The observer's scale factor is $a = e^{a_4 - a_{4,\rm today}}$, so at $a = 1$ the
slope $dw/da$ equals $dw/da_4$ and the tangent is $w_0 = w(a_{4,\rm today})$,
$w_a = -dw_{\rm eff}/da_4$. The next cell reads $dw_{\rm eff}/da_4$ from the record's
table at the four values of $a_{4,\rm today}$ that the record uses, prints the
tangents of the gas N688_lam0 under definition C, and checks them and the range of
the tangents of all gas series.
```

In [4]:

```
TODAY = [0.5, 1.0, 1.5, 2.0]  # the four choices of a4,today of the record
index = [int(round(t / h)) for t in TODAY]  # their positions in the slice list
recorded = {entry["series"]: entry for entry in summary["series"]}
say("a4,today   w0 under C   wa (every definition)")
differences = []
for t, i, entry in zip(TODAY, index, recorded["N688_lam0"]["cplTangent"]):
```

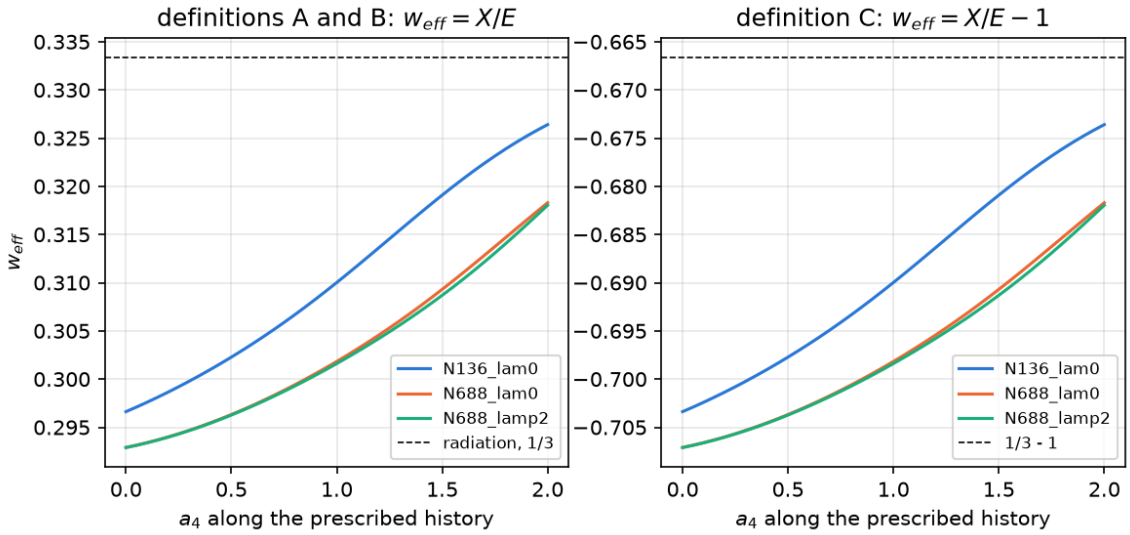

Figure 559: The equation of state w_{eff} that a 3-space observer infers from the thinning of the Kohn-Sham gas of dirac16complex along the prescribed history, for three recorded series; horizontal axis a_4 from 0 to 2 (one unit is one e-fold of inflation of 3-space and of deflation of the extra times), vertical axis w_{eff} (a pure number). Left, definitions A and B of the observer density: $w_{eff} = X/E$ rises from about 0.29 toward the radiation value 1/3 (dashed). Right, definition C: the same curves moved down by exactly 1, between about -0.71 and -0.67, never near -1. The observer assumption alone decides whether the same gas reads as radiation or as dark energy. (Notebook 22b, figure 1.)

```
    w0 = X[i] / E[i] - 1  # w_eff(C) today
    wa = -gas["dw_eff_da4"][i]  # w_a = -dw/da at a = 1, and da = da4 there
    say(f"{t:8.1f}   {w0:10.4f}   {wa:12.5f}")
    differences += [abs(w0 - entry["w_eff_C"]["w0"]), abs(wa - entry["w_eff_C"]["wa"])]
check(max(differences) < 1e-12, "the CPL tangents of N688_lam0",
      record=f"{D16}/outputs/eos-summary.json, key cplTangent of N688_lam0")

tangents = [-SERIES[name]["dw_eff_da4"][i] for name in GAS for i in index]
report("tangent wa of every gas series, smallest and largest",
       f"{min(tangents):.6f}, {max(tangents):.6f}")
check(f"{min(tangents):.6f}" == "-0.020523" and f"{max(tangents):.6f}" == "-0.008894"
      and f"{ratios.min() - 1:.6f}" == "-0.707107"
      and f"{ratios.max() - 1:.6f}" == "-0.671895",
      "every gas tangent wa is negative (thawing sign) and far below 0.60",
      record=f"{D16}/reports/eos-checks.json, check gas_cpl_thawing_sign_small")
```

Out [4]:

```
a4,today   w0 under C   wa (every definition)
     0.5      -0.7037       -0.00905
     1.0      -0.6982       -0.01292
     1.5      -0.6907       -0.01701
     2.0      -0.6817       -0.01775
PASS the CPL tangents of N688_lam0
     reproduces Revision/dark_sector/dirac16complex/outputs/eos-summary.json, key
```

```
    cplTangent of N688_lam0
RESULT tangent wa of every gas series, smallest and largest = -0.020523, -0.008894
PASS every gas tangent wa is negative (thawing sign) and far below 0.60
     reproduces Revision/dark_sector/dirac16complex/reports/eos-checks.json, check
    gas_cpl_thawing_sign_small
```

Text cell:

```
## 8. The bulk band and the brane band

The occupied levels of the computed ground states lie on the brane band, which is
massless at zero 3-momentum. The record's independent free-gas computation also
follows one level of the massive bulk band (odd brane parity) and one brane-band
level of the same 3-momentum shell over $a_4$ from $-3$ to $4$. The next cell reads
their $w_{\rm eff}(A)$, checks the dark-matter-like fall of the bulk level and the
radiation-like value of the brane level, and draws both with the gas (Figure 2).
```

In [5]:

```
bulk = free_gas["bulkBand_n2_1"]  # the lowest massive (odd-parity) level, shell 1
brane = free_gas["braneBand_n2_1"]  # the brane-band level of the same shell
a4_band = np.array([point["a4"] for point in bulk])
w_bulk = np.array([point["w_eff_A_B"] for point in bulk])
w_brane = np.array([point["w_eff_A_B"] for point in brane])
report("w_eff(A) of the bulk level at a4 = -3 and a4 = 4",
       f"{w_bulk[0]:.6f}, {w_bulk[-1]:.3e}")
report("ratio w(4)/w(3.75) of the bulk level", f"{w_bulk[-1] / w_bulk[-2]:.6f}")
report("w_eff(A) of the brane level, smallest and largest",
       f"{w_brane.min():.6f}, {w_brane.max():.6f}")
check(a4_band[0] == -3 and a4_band[-1] == 4 and np.all(np.diff(w_bulk) < 0)
      and f"{w_bulk[0]:.6f}" == "0.239626" and f"{w_bulk[-1]:.3e}" == "2.268e-03",
      "the bulk level falls from 0.239626 to 2.268e-03 (dark-matter-like)",
      record=f"{D16}/reports/independent-checks.json, check bulk_band_dark_matter_law")
check(f"{w_brane.min():.6f}" == "0.291594" and f"{w_brane.max():.6f}" == "0.333275",
      "the brane level stays between 0.291594 and 0.333275 (radiation-like)",
      record=f"{D16}/reports/independent-checks.json, check brane_band_radiation_law")

fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(a4_band, w_bulk, "o-", color=PALETTE[1], markersize=3,
        label="bulk band (massive), one level")
ax.plot(a4_band, w_brane, "s-", color=PALETTE[0], markersize=3,
        label="brane band (massless at k = 0), one level")
ax.plot(a4, X / E, color=PALETTE[2], linewidth=2.5, label="the gas N688_lam0")
ax.axhline(1 / 3, color="black", linestyle="--", linewidth=0.8)
ax.axhline(0.0, color="black", linestyle=":", linewidth=0.8)
ax.set_xlabel("$a_4$")
ax.set_ylabel("$w_{eff}$ under definitions A and B")
ax.legend(fontsize=8)
save_figure(fig, "bulk_and_brane_bands",
            "Which quanta of dirac16complex behave like dark matter. Horizontal "
            "axis $a_4$ from $-3$ to 4, vertical axis $w_{eff}$ under definitions "
            "A and B (a pure number). Orange: one level of the massive bulk band "
```

```
        "falls from 0.24 toward the dust value 0 (dotted), the "
        "dark-matter-like law. Blue: a brane-band level of the same shell "
        "tends to the radiation value 1/3 (dashed). Green, thick: the computed "
        "Kohn-Sham gas, which fills only the brane band and is radiation-like. "
        "The computed ground states do not populate the bulk band.")
```

Out [5]:

```
RESULT w_eff(A) of the bulk level at a4 = -3 and a4 = 4 = 0.239626, 2.268e-03
RESULT ratio w(4)/w(3.75) of the bulk level = 0.765177
RESULT w_eff(A) of the brane level, smallest and largest = 0.291594, 0.333275
PASS the bulk level falls from 0.239626 to 2.268e-03 (dark-matter-like)
     reproduces Revision/dark_sector/dirac16complex/reports/independent-checks.json,
    check bulk_band_dark_matter_law
PASS the brane level stays between 0.291594 and 0.333275 (radiation-like)
     reproduces Revision/dark_sector/dirac16complex/reports/independent-checks.json,
    check brane_band_radiation_law
```

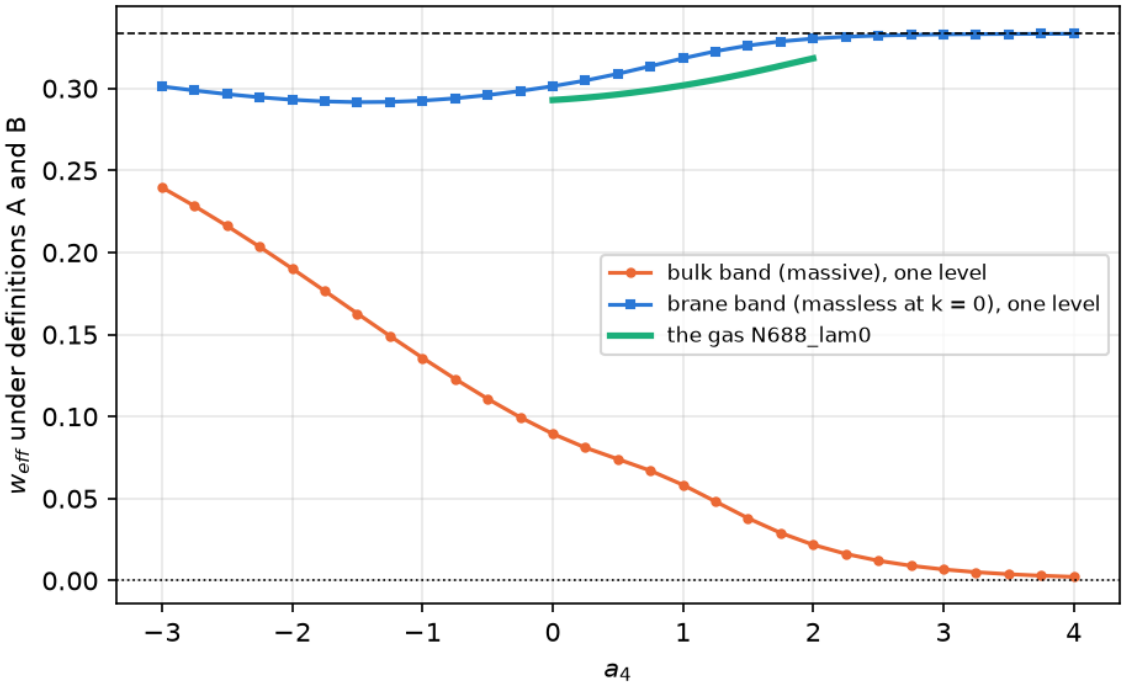

Figure 560: Which quanta of dirac16complex behave like dark matter. Horizontal axis a_4 from -3 to 4, vertical axis w_{eff} under definitions A and B (a pure number). Orange: one level of the massive bulk band falls from 0.24 toward the dust value 0 (dotted), the dark-matter-like law. Blue: a brane-band level of the same shell tends to the radiation value 1/3 (dashed). Green, thick: the computed Kohn-Sham gas, which fills only the brane band and is radiation-like. The computed ground states do not populate the bulk band. (Notebook 22b, figure 2.)

Out [5]:

```
Figure 22b.2 saved as Revision/textbook/figures/22b_2_bulk_and_brane_bands.png
```

Text cell:

```
## 9. The condensate

A homogeneous condensate is an exact solution of both fields, with
$\rho = mS + \lambda S^2/2$ and $p_3 = p_t = p_0 = \lambda S^2/2$, all constant.
So $X = 0$: $w_{\rm eff} = 0$ under A and B (dust-like), $-1$ under C. Its ratio is
$p/\rho = u/(2 + u)$ with $u = \lambda S/m$. The next cell computes, with exact
fractions, the value $u = -382/441$ that the record CHOSE so that the ratio equals
$-0.764$, checks the phantom window, and draws the ratio (Figure 3).
```

In [6]:

```
u = Fraction(-382, 441)  # lambda S/m, CHOSEN so that the ratio is -0.764
ratio = u / (2 + u)  # p/rho = (lambda S^2/2)/(m S + lambda S^2/2) = u/(2 + u)
density = 1 + u / 2  # rho/(m S)
report("the ratio u/(2 + u) at u = -382/441", f"{ratio} = {float(ratio)}")
report("rho/(m S) = 1 + u/2", f"{density} = {float(density):.12f}")
check(ratio == Fraction(-764, 1000)
      and abs(float(u) - summary["condensate"]["u_for_ratio_minus_0p764"]) < 1e-12,
      "the condensate ratio is -0.764 at u = -382/441 (one CHOSEN parameter)",
      record=f"{D16}/reports/derivation-checks.json, "
             "check condensate_ratio_equal_unite_constant_w")

grid = np.linspace(-3.0, 1.0, 4001)
grid = grid[np.abs(grid + 2) > 1e-9]  # leave out the pole u = -2
below = grid / (2 + grid) < -1  # where the ratio is phantom
check(np.array_equal(below, (grid > -2) & (grid < -1)),
      "the ratio is below -1 exactly for -2 < u < -1",
      record=f"{D00}/reports/python-derive-eos.json, "
             "check condensate_phantom_interval")

fig, ax = plt.subplots(figsize=(7.0, 4.2))
for part in (grid[grid < -2], grid[grid > -2]):
    ax.plot(part, part / (2 + part), color=PALETTE[0])
ax.axvspan(-2, -1, color=PALETTE[7], alpha=0.12, label="phantom ratio, -2 < u < -1")
ax.axhline(0.0, color="black", linestyle="--", linewidth=0.8,
           label="$w_{eff}$, definitions A and B")
ax.axhline(-1.0, color="black", linestyle=":", linewidth=0.8,
           label="$w_{eff}$, definition C")
ax.plot([float(u)], [float(ratio)], "o", color=PALETTE[1],
        label="CHOSEN: u = -382/441 gives -0.764")
ax.set_ylim(-4.0, 4.0)
ax.set_xlabel("$u = \\lambda S/m$")
ax.set_ylabel("ratio $p/\\rho$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "condensate_ratio",
            "The constant ratio $p/\\rho = u/(2 + u)$ of a homogeneous condensate "
            "against $u = \\lambda S/m$ (horizontal axis, a pure number; the "
            "ratio has a pole at $u = -2$). The orange point is the value "
            "$u = -382/441$ CHOSEN so that the ratio equals the Unite constant "
            "$w = -0.764$: one parameter for one number, not a prediction. The red "
```

```
        "band is the phantom window. The observer does not see this ratio: "
        "the dilution gives $w_{eff} = 0$ under A and B (dashed) and $-1$ "
        "under C (dotted), constant in time.")
```

Out [6]:

```
RESULT the ratio u/(2 + u) at u = -382/441 = -191/250 = -0.764
RESULT rho/(m S) = 1 + u/2 = 250/441 = 0.566893424036
PASS the condensate ratio is -0.764 at u = -382/441 (one CHOSEN parameter)
     reproduces Revision/dark_sector/dirac16complex/reports/derivation-checks.json, check
    condensate_ratio_equal_unite_constant_w
PASS the ratio is below -1 exactly for -2 < u < -1
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check condensate_phantom_interval
```

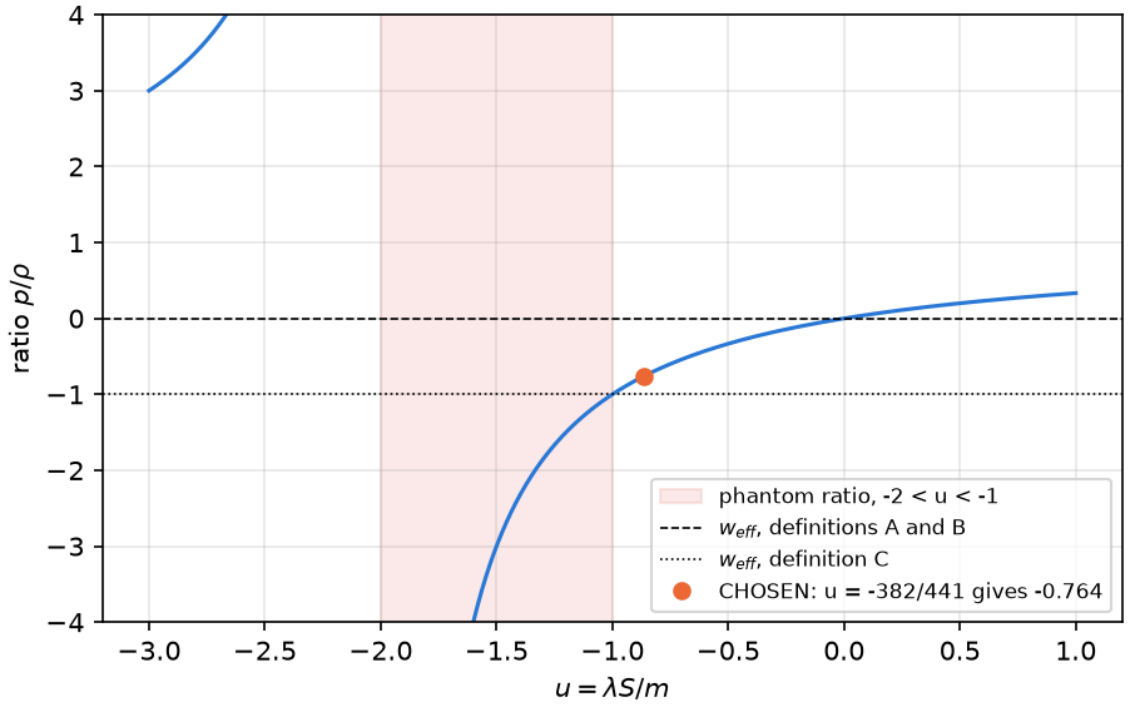

Figure 561: The constant ratio $p/\rho = u/(2+u)$ of a homogeneous condensate against $u = \lambda S/m$ (horizontal axis, a pure number; the ratio has a pole at $u = -2$). The orange point is the value $u = -382/441$ CHOSEN so that the ratio equals the Unite constant $w = -0.764$: one parameter for one number, not a prediction. The red band is the phantom window. The observer does not see this ratio: the dilution gives $w_{eff} = 0$ under A and B (dashed) and -1 under C (dotted), constant in time. (Notebook 22b, figure 3.)

Out [6]:

```
Figure 22b.3 saved as Revision/textbook/figures/22b_3_condensate_ratio.png
```

Text cell:

```
## 10. Mixtures of the gas with a condensate

For a radiation-like gas ($X = E/3$, $E \propto 1/a$) plus a condensate ($X = 0$,
$E$ constant) with energy ratio $r = r_0/a$, the record derives exactly
$w_{\rm eff}(A) = \frac{1}{3}r/(1 + r)$, so $w_0 = r_0/(3(r_0 + 1))$ and
$w_a = r_0/(3(r_0 + 1)^2) > 0$: freezing. The next cell evaluates this law at the
$r_0$ CHOSEN for $w_{\rm eff}(C) = -0.861$ today, then mixes the computed gas
N688_lam0 with a condensate (today $a_{4,\rm today} = 2$), with the gas share CHOSEN
for $w_{\rm eff}(C) = -0.861$, and draws mixtures against the Unite line (Figure 4).
```

In [7]:

```
r0 = Fraction(417, 583)  # gas/condensate energy today, CHOSEN for w0(C) = -0.861
w0_exact = r0 / (3 * (r0 + 1)) - 1  # w_eff(C) today
wa_exact = r0 / (3 * (r0 + 1) ** 2)  # w_a of the exact law: positive (freezing)
report("exact law: w0 under C and wa", f"{w0_exact}, {float(wa_exact):.6f}")
check(w0_exact == Fraction(-861, 1000) and f"{float(wa_exact):.6f}" == "0.081037",
      "radiation plus condensate with w0(C) = -0.861 has wa = 0.081037 > 0",
      record=f"{D16}/reports/derivation-checks.json, "
             "check mixture_C_matching_w0_unite")


def mixture_w_C(share):
    """w_eff(C) of the gas N688_lam0 plus a condensate along the history; the gas
    has the energy share "share" today (a4 = 2)."""
    condensate = E[-1] * (1 - share) / share  # constant condensate energy
    return X / (E + condensate) - 1


share = 0.139 / (X[-1] / E[-1])  # CHOSEN: gives w_eff(C) = -0.861 today
w_mix = mixture_w_C(share)
# w_a = -dw/da4 at a4 = 2, one-sided fourth-order differences (as the record)
wa_mix = -(3 * w_mix[-5] - 16 * w_mix[-4] + 36 * w_mix[-3] - 48 * w_mix[-2]
           + 25 * w_mix[-1]) / (12 * h)
recorded_mix = summary["mixture_C_w0_minus_0p861"]
report("gas share today for w_eff(C) = -0.861 (CHOSEN)", f"{share:.6f}")
report("w0 and wa of that mixture under C", f"{w_mix[-1]:.3f}, {wa_mix:.6f}")
check(abs(share - recorded_mix["gas_fraction_today"]) < 1e-12
      and abs(wa_mix - recorded_mix["wa"]) < 1e-10 and wa_mix > 0,
      "the computed gas mixture with w0(C) = -0.861 is freezing, wa = 0.067014",
      record=f"{D16}/reports/eos-checks.json, check mixture_C_w0_unite_has_positive_wa")

a_obs = np.exp(a4 - 2.0)  # the observer scale factor, a = 1 today (a4 = 2)
fig, ax = plt.subplots(figsize=(7.0, 4.2))
for colour, part in zip(PALETTE, [0.25, share, 0.75, 1.0]):
    ax.plot(a_obs, mixture_w_C(part), color=colour,
            label=f"gas share today {part:.3f}")
ax.plot(a_obs, -0.861 - 0.60 * (1 - a_obs), "k--", label="Unite CPL line")
ax.axhline(-1.0, color="black", linestyle=":", linewidth=0.8)
ax.set_xlabel("observer scale factor $a = e^{a_4 - 2}$")
ax.set_ylabel("$w_{eff}$ under definition C")
ax.legend(fontsize=8)
save_figure(fig, "mixtures_against_unite",
```

```
            "Mixtures of the computed Kohn-Sham gas N688_lam0 with a condensate, "
            "under definition C, against the observer scale factor $a$ "
            "(horizontal axis, $a = 1$ today, taken at $a_4 = 2$); vertical axis "
            "$w_{eff}$ (a pure number). Each coloured curve has another gas "
            "share today; the share 0.437 is CHOSEN so that $w_{eff} = -0.861$ "
            "today. Every curve that contains condensate falls toward the "
            "condensate value $-1$ (dotted) as $a$ grows: freezing, $w_a > 0$; "
            "the pure gas (share 1) rises only slightly. The Unite CPL line "
            "(dashed) rises steeply instead: $w_a = -0.60$. No share gives the "
            "Unite slope.")
```

Out [7]:

```
RESULT exact law: w0 under C and wa = -861/1000, 0.081037
PASS radiation plus condensate with w0(C) = -0.861 has wa = 0.081037 > 0
     reproduces Revision/dark_sector/dirac16complex/reports/derivation-checks.json, check
    mixture_C_matching_w0_unite
RESULT gas share today for w_eff(C) = -0.861 (CHOSEN) = 0.436703
RESULT w0 and wa of that mixture under C = -0.861, 0.067014
PASS the computed gas mixture with w0(C) = -0.861 is freezing, wa = 0.067014
     reproduces Revision/dark_sector/dirac16complex/reports/eos-checks.json, check
    mixture_C_w0_unite_has_positive_wa
```

Out [7]:

```
Figure 22b.4 saved as Revision/textbook/figures/22b_4_mixtures_against_unite.png
```

Text cell:

```
## 11. The models M2 to M5 of dirac16complex00

Each component of an adiabatic (WKB) gas of dirac16complex00 has an energy
density $\rho_i$ and $\varepsilon_i = (k^2/a^2 + q^2a^2)/(3\omega_i^2) \ge 0$ with
$\omega^2 = m^2 + k^2/a^2 - q^2a^2$; a mixture has
$w_{\rm eff}(C) = \sum_i\varepsilon_i\rho_i/\sum_i\rho_i - 1$. The record's models
(normalised to total energy 1 at $a = 1$): M2 a good-sector gas ($q = 0$), M3 one
extra-time mode ($k = 0$), M4 a condensate plus an extra-time mode, M5 the same plus
a GHOST-LIKE component of negative energy $-G/a$. The next cell rebuilds them,
checks their tangents, the M5 fit, the crossings of $-1$ and the agreement of the
field-equation implementation B, and draws them against the Unite line (Figure 5).
```

In [8]:

```
def model(a, components):
    """The total energy density and the sum of eps_i rho_i of a WKB model at a
    (a number or an array).  Each component is (kind, weight at a = 1, s)."""
    a = np.asarray(a, dtype=float)
    rho_total, eps_rho = np.zeros_like(a), np.zeros_like(a)
    for kind, weight, s in components:
```

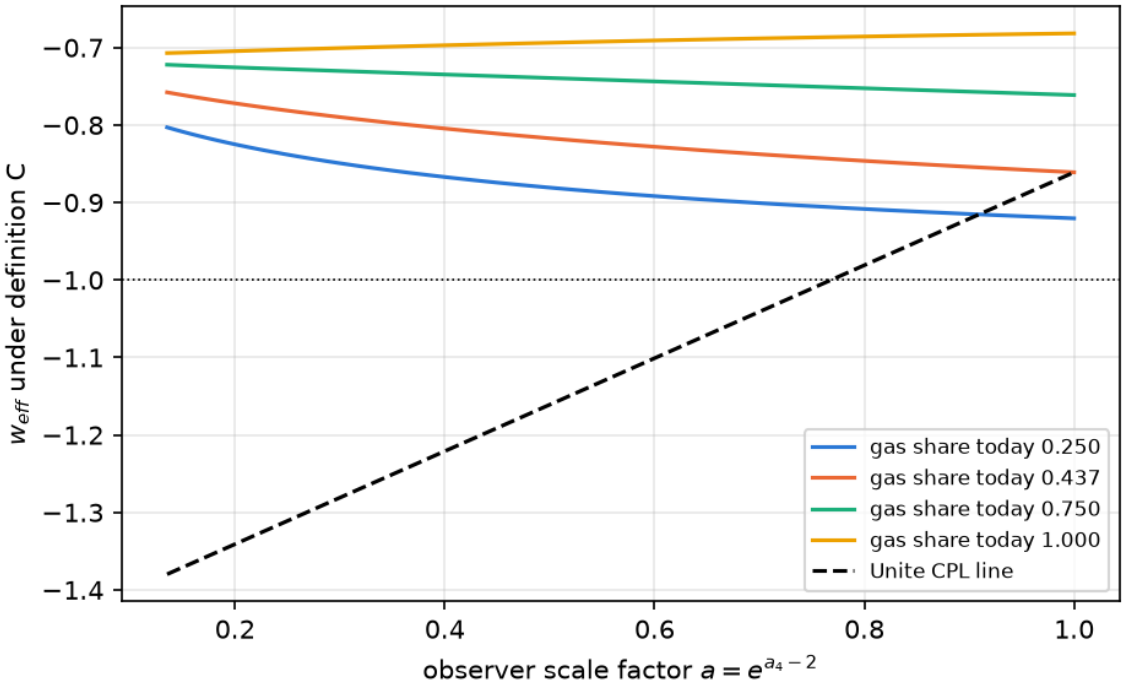

Figure 562: Mixtures of the computed Kohn-Sham gas N688_lam0 with a condensate, under definition C, against the observer scale factor a (horizontal axis, $a = 1$ today, taken at $a_4 = 2$); vertical axis w_{eff} (a pure number). Each coloured curve has another gas share today; the share 0.437 is CHOSEN so that $w_{eff} = -0.861$ today. Every curve that contains condensate falls toward the condensate value -1 (dotted) as a grows: freezing, $w_a > 0$; the pure gas (share 1) rises only slightly. The Unite CPL line (dashed) rises steeply instead: $w_a = -0.60$. No share gives the Unite slope. (Notebook 22b, figure 4.)

```
        if kind == "condensate":  # rho constant, eps = 0
            rho, eps = weight + 0 * a, 0 * a
        elif kind == "kmode":  # q = 0 and s = k^2/(k^2 + m^2) at a = 1
            r = s / (1 - s)  # k^2/m^2
            rho = weight * np.sqrt((r / a**2 + 1) / (r + 1))  # omega(a)/omega(1)
            eps = (r / a**2) / (3 * (r / a**2 + 1))
        elif kind == "qmode":  # k = 0 and s = q^2/m^2 at a = 1
            rho = weight * np.sqrt((1 - s * a**2) / (1 - s))
            eps = s * a**2 / (3 * (1 - s * a**2))
        else:  # "ghost": massless modes of NEGATIVE classical energy
            rho, eps = -weight / a, 1 / 3 + 0 * a
        rho_total = rho_total + rho
        eps_rho = eps_rho + eps * rho
    return rho_total, eps_rho


def w_model(a, components):
    """w_eff under definition C: sum eps_i rho_i / sum rho_i - 1."""
    rho_total, eps_rho = model(a, components)
    return eps_rho / rho_total - 1


def tangent(components, d=1e-3):
    """CPL tangent w0 = w(1), wa = -dw/da at a = 1 (fourth-order differences)."""
    f = [float(w_model(1 + k * d, components)) for k in (-2, -1, 1, 2)]
    slope = (f[0] - 8 * f[1] + 8 * f[2] - f[3]) / (12 * d)  # dw/da at a = 1
    return float(w_model(1.0, components)), -slope


M5_RECORD = theory00["models"]["M5_with_ghost_component"]
G = float(Fraction(M5_RECORD["parameters"]["G"]))  # ghost share 3/10, a choice
s5 = float(M5_RECORD["parameters"]["s"])  # CHOSEN with q5 to fit the Unite line
q5 = float(M5_RECORD["parameters"]["Omega_q"])
c5 = float(M5_RECORD["parameters"]["Omega_c"])
MODELS = {"M2": [("kmode", 1.0, 417 / 1000)],  # s CHOSEN: w0 = -0.861
          "M3": [("qmode", 1.0, 417 / 1417)],  # s CHOSEN: w0 = -0.861
          "M4": [("condensate", 1 - 57963 / 264037, 0.0),  # both CHOSEN:
                 ("qmode", 57963 / 264037, 264037 / 403037)],  # tangent = Unite
          "M5": [("condensate", c5, 0.0), ("qmode", q5, s5), ("ghost", G, 0.0)]}
EXPECTED_WA = {"M2": 81037 / 500000, "M3": -196963 / 500000, "M4": -0.6}
RECORD_CHECK = {"M2": "M2_tangent_exact", "M3": "M3_tangent_exact",
                "M4": "M4_tangent_equals_unite"}
for name in ("M2", "M3", "M4"):
    w0, wa = tangent(MODELS[name])
    report(f"{name} tangent (w0, wa) under C", f"({w0:.6f}, {wa:.6f})")
    check(abs(w0 + 0.861) < 1e-12 and abs(wa - EXPECTED_WA[name]) < 1e-8,
          f"the tangent of {name}",
          record=f"{D00}/reports/python-derive-eos.json, check {RECORD_CHECK[name]}")

a_fit = np.linspace(0.5, 1.0, 101)  # 101 points of a in [1/2, 1], as the record
wa_fit, w0_fit = np.polyfit(1 - a_fit, w_model(a_fit, MODELS["M5"]), 1)
report("M5 least-squares fit over a in [1/2, 1]", f"({w0_fit:.6f}, {wa_fit:.6f})")
check(abs(w0_fit + 0.861) < 1e-9 and abs(wa_fit + 0.6) < 1e-9,
      "the M5 fit equals the Unite pair (by construction)",
```

```
      record=f"{D00}/reports/python-derive-eos.json, check M5_fit_equals_unite")


def crossings(components):
    """The values of a in [1/3, 1] where w = -1 (400 intervals, then bisection)."""
    points = np.linspace(1 / 3, 1, 401)
    values = w_model(points, components) + 1
    found = []
    for low, high, v_low, v_high in zip(points, points[1:], values, values[1:]):
        if v_low * v_high < 0:
            for _ in range(60):  # halve the interval 60 times
                middle = (low + high) / 2
                v_middle = float(w_model(middle, components) + 1)
                if v_low * v_middle <= 0:
                    high = middle
                else:
                    low, v_low = middle, v_middle
            found.append((low + high) / 2)
    return found


cross_M5 = crossings(MODELS["M5"])
no_ghost = [("condensate", 1 - q5, 0.0), ("qmode", q5, s5)]  # M5 without ghost
lowest_M4 = float(np.min(w_model(np.arange(1, 301) / 300, MODELS["M4"])))
crossing_A = float(M5_RECORD["N2"]["crossings_of_minus_1_in_[1/3,1]"][0])
crossing_B = numerics00["models"]["M5_with_ghost_component"]["crossing_N2"]
M4_RECORD = theory00["models"]["M4_condensate_plus_extra_time_mode"]
report("M5 crosses w = -1 at a", f"{cross_M5[0]:.11f}")
report("the same crossing from the field equation (implementation B)",
       f"{crossing_B}")
report("lowest w_eff(C) of M4 for a = 1/300 to 1", f"{lowest_M4:.12f}")
check(len(cross_M5) == 1 and abs(cross_M5[0] - crossing_A) < 1e-10,
      "M5 crosses -1 once, at a = 0.7791",
      record=f"{D00}/reports/python-derive-eos.json, check M5_crosses_minus_1")
check(crossings(no_ghost) == [] and crossings(MODELS["M4"]) == []
      and abs(lowest_M4 - float(M4_RECORD["min_w_N2_on_(0,1]"])) < 1e-11,
      "no crossing on 1/3 <= a <= 1 without the ghost: M5 without it, and M4",
      record=f"{D00}/reports/python-derive-eos.json, checks "
             "M5_without_ghost_no_crossing and M4_never_phantom")
check(abs(crossing_B - cross_M5[0]) < 1e-4,
      "implementation B finds the M5 crossing within 1e-4",
      record=f"{D00}/reports/python-independent-numerics.json, "
             "check B_vs_A_M5_crossing")

a_plot = np.linspace(1 / 3, 1.0, 300)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
LABELS = {"M2": "M2, gas (1 CHOSEN)", "M3": "M3, extra-time mode (1 CHOSEN)",
          "M4": "M4 (2 CHOSEN)", "M5": "M5 (2 CHOSEN, ghost-like part)"}
for colour, name in zip(PALETTE, MODELS):
    left.plot(a_plot, w_model(a_plot, MODELS[name]), color=colour,
              label=LABELS[name])
left.plot(a_plot, -0.861 - 0.60 * (1 - a_plot), "k--", label="Unite CPL line")
left.axhline(-1.0, color="black", linestyle=":", linewidth=0.8)
left.set_xlabel("observer scale factor $a$")
```

```
left.set_ylabel("$w_{eff}$ under definition C")
left.legend(fontsize=7)
PARTS = [("condensate", MODELS["M5"][0]), ("extra-time mode", MODELS["M5"][1]),
         ("ghost-like part", MODELS["M5"][2])]
for colour, (label, component) in zip(PALETTE[4:], PARTS):
    right.plot(a_plot, model(a_plot, [component])[0], color=colour, label=label)
right.plot(a_plot, model(a_plot, MODELS["M5"])[0], "k-", label="total")
right.axhline(0.0, color="black", linewidth=0.6)
right.set_xlabel("observer scale factor $a$")
right.set_ylabel("energy density, M5")
right.legend(fontsize=7)
fig.subplots_adjust(wspace=0.3)  # room between the two panels
save_figure(fig, "models_against_unite",
            "The models of dirac16complex00 under definition C against the "
            "observer scale factor $a$ from 1/3 to 1 (horizontal axes). Left: "
            "$w_{eff}$ (a pure number) of M2 to M5 and the Unite CPL line "
            "(dashed). As $a$ grows, M2 falls toward $-1$ (freezing, $w_a > 0$); "
            "M3 and M4 rise away from $-1$ (thawing, $w_a < 0$) and stay above "
            "$-1$ (dotted) on this range. M2, M3 and M4 pass through $-0.861$ at "
            "$a = 1$; the tangent of M4 equals the Unite line at $a = 1$ because "
            "its two parameters were CHOSEN for that. M5 ends at $-0.8396$: its "
            "two parameters were CHOSEN so that its least-squares fit over "
            "$a$ from 1/2 to 1, not its value today, equals the Unite pair. "
            "Only M5 crosses $-1$, at $a = 0.779$, and only because of its "
            "ghost-like part. Right: the energy densities of the three parts of "
            "M5 (total 1 at $a = 1$); the ghost-like part is negative, $-0.3/a$. "
            "Every match is by "
            "construction, NOT a prediction.")
```

Out [8]:

```
RESULT M2 tangent (w0, wa) under C = (-0.861000, 0.162074)
PASS the tangent of M2
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check M2_tangent_exact
RESULT M3 tangent (w0, wa) under C = (-0.861000, -0.393926)
PASS the tangent of M3
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check M3_tangent_exact
RESULT M4 tangent (w0, wa) under C = (-0.861000, -0.600000)
PASS the tangent of M4
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check M4_tangent_equals_unite
RESULT M5 least-squares fit over a in [1/2, 1] = (-0.861000, -0.600000)
PASS the M5 fit equals the Unite pair (by construction)
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check M5_fit_equals_unite
RESULT M5 crosses w = -1 at a = 0.77909966367
RESULT the same crossing from the field equation (implementation B) = 0.77905405
RESULT lowest w_eff(C) of M4 for a = 1/300 to 1 = -0.999999214228
PASS M5 crosses -1 once, at a = 0.7791
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check M5_crosses_minus_1
```

```
PASS no crossing on 1/3 <= a <= 1 without the ghost: M5 without it, and M4
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    checks M5_without_ghost_no_crossing and M4_never_phantom
PASS implementation B finds the M5 crossing within 1e-4
     reproduces Revision/dark_sector/dirac16complex00/reports/python-independent-
    numerics.json, check B_vs_A_M5_crossing
```

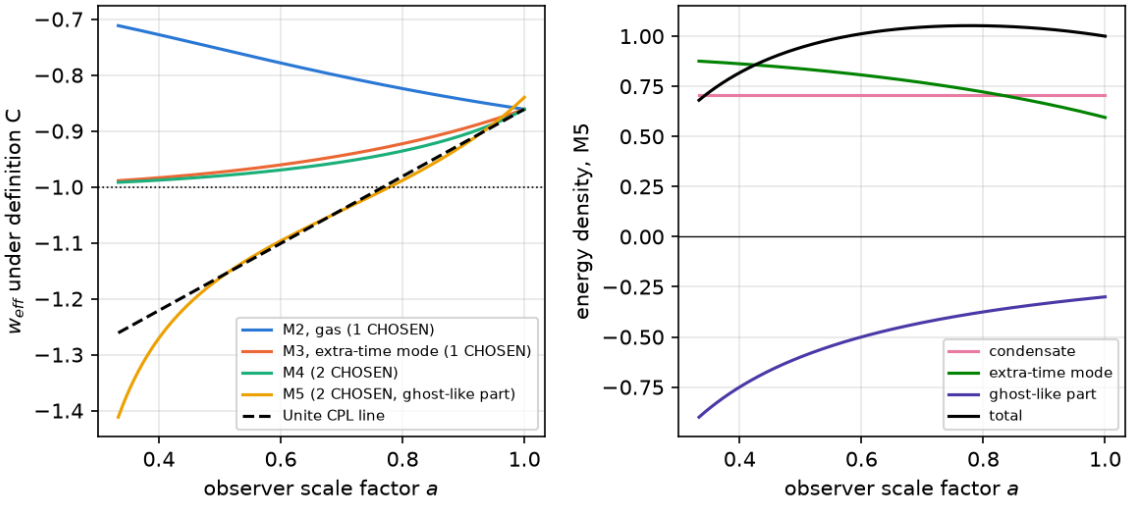

Figure 563: The models of dirac16complex00 under definition C against the observer scale factor a from 1/3 to 1 (horizontal axes). Left: w_{eff} (a pure number) of M2 to M5 and the Unite CPL line (dashed). As a grows, M2 falls toward -1 (freezing, $w_a > 0$); M3 and M4 rise away from -1 (thawing, $w_a < 0$) and stay above -1 (dotted) on this range. M2, M3 and M4 pass through -0.861 at $a = 1$; the tangent of M4 equals the Unite line at $a = 1$ because its two parameters were CHOSEN for that. M5 ends at -0.8396: its two parameters were CHOSEN so that its least-squares fit over a from 1/2 to 1, not its value today, equals the Unite pair. Only M5 crosses -1, at $a = 0.779$, and only because of its ghost-like part. Right: the energy densities of the three parts of M5 (total 1 at $a = 1$); the ghost-like part is negative, $-0.3/a$. Every match is by construction, NOT a prediction. (Notebook 22b, figure 5.)

Out [8]:

```
Figure 22b.5 saved as Revision/textbook/figures/22b_5_models_against_unite.png
```

Text cell:

```
## 12. The comparison with the Unite values

The next cell prints which content reproduces each Unite value and how many
parameters were CHOSEN for it, checks the Unite values and the scale factor at
which their CPL line crosses $-1$, and checks that every figure file exists.
```

In [9]:

```
unite = theory00["models"]["unite"]
w0_u, wa_u = Fraction(unite["w0"]), Fraction(unite["wa"])
crossing_u = 1 + (1 + w0_u) / wa_u  # w0 + wa (1 - a) = -1 solved for a
TABLE = [("w = -0.764", "condensate ratio, u = -382/441", "1 for 1"),
         ("w0 = -0.861", "M2; M3; gas plus condensate", "1 for 1"),
         ("(w0, wa) = (-0.861, -0.60)", "M4 tangent; M5 fit", "2 for 2"),
         ("crossing of -1, a = 0.7683", "only M5", "ghost-like part")]
say("Unite value | reproduced by | CHOSEN parameters")
for row in TABLE:
    say(" | ".join(row))
report("the Unite CPL line crosses w = -1 at a", f"{crossing_u}")
check(w0_u == Fraction(-861, 1000) and wa_u == Fraction(-3, 5)
      and crossing_u == Fraction(unite["crossing_of_minus_1"]["a"])
      and crossing_u == Fraction(461, 600),
      "the Unite values and their crossing a = 461/600",
      record=f"{D00}/reports/python-derive-eos.json, check unite_crossing_point")
files = [f"{FIGURE_FOLDER}/{NOTEBOOK_ID}_{number}_{name}.png"
         for name, number in FIGURE_NUMBERS.items()]
check(len(files) == 5 and all(output_file(path).is_file() for path in files),
      "every figure file of this notebook exists")
all_checks_passed()
```

Out [9]:

```
Unite value | reproduced by | CHOSEN parameters
w = -0.764 | condensate ratio, u = -382/441 | 1 for 1
w0 = -0.861 | M2; M3; gas plus condensate | 1 for 1
(w0, wa) = (-0.861, -0.60) | M4 tangent; M5 fit | 2 for 2
crossing of -1, a = 0.7683 | only M5 | ghost-like part
RESULT the Unite CPL line crosses w = -1 at a = 461/600
PASS the Unite values and their crossing a = 461/600
     reproduces Revision/dark_sector/dirac16complex00/reports/python-derive-eos.json,
    check unite_crossing_point
PASS every figure file of this notebook exists
ALL 28 CHECKS PASSED (notebook 22b)
```

Text cell:

```
## 13. What this notebook showed

- The observer's density $\rho_4$ is an ASSUMPTION. On the computed history the
  dilution gives exactly $w_{\rm eff} = X/E$ under A and B and $X/E - 1$ under C, so
  every verdict moves by exactly $-1$ between the definitions.
- The Kohn-Sham gas of dirac16complex is radiation-like: $X/E$ between 0.292893
  and 0.328105, rising toward $1/3$; under C it lies between $-0.707107$ and
  $-0.671895$ with small tangent slopes of thawing sign ($w_a$ between $-0.020523$
  and $-0.008894$), far from the Unite $w_a = -0.60$.
- The dark-matter-like fall from about $1/3$ toward 0 belongs to the massive bulk
  band (0.239626 to 0.002268), which the computed ground states do not populate.
- A condensate is dust-like (A, B) or cosmological-constant-like (C); its ratio is
  $-0.764$ only at the CHOSEN $u = -382/441$. Gas and condensate mixtures are
  freezing ($w_a = 0.067014$ for the CHOSEN share 0.436703).
```

```
- The dirac16complex00 models reproduce the Unite tangent (M4) and fit (M5) because
  their parameters were CHOSEN to; a crossing of $-1$ needs the ghost-like
  component (M5). None of these matches is a prediction.
- OPEN: which $\rho_4$ describes a real observer; a self-consistent history with an
  admissible source; populated bulk-band or thermal states; what would select the
  populations; a fit to supernova data.
```

22.23 Line-by-line walk-through of Notebook 22b

The notebook has 9 code cells, In [1] to In [9]. This section explains every line of In [2] to In [9], in order; the printed output of each cell is in Section 22.22 under the label Out [k], and the five figures are shown there with their captions. For each figure this section adds what the student should see and why.

In [1], the set-up cell. It is the set-up cell of every notebook of this book, with the name `"22b"`; Section 22.12 explains each of its lines for Notebook 22a, and only the name differs. Its comment lines repeat the run instructions of Section 22.21. Its code finds the repository folder (`REPO`), decides where files are written (`OUTPUT_ROOT`), and defines the helpers the later cells use: `repository_file(relative)` gives the path of a repository file for reading; `output_file(relative)` the path at which to write one; `say(text)` prints in lines of at most 89 characters; `save_figure(fig, name, caption)` saves a figure as `Revision/textbook/figures/22b_<k>_<name>.png`, records its caption, shows it and prints where it was saved, numbering the figures in the dictionary `FIGURE_NUMBERS`; `check(condition, name, record)` stops the notebook if the condition is false and otherwise prints PASS with the name, and a second line naming the Revision record when `record` is given; `report(label, value)` prints a line RESULT label = value; `all_checks_passed()` prints the last line. The folder name `FIGURE_FOLDER` is `"Revision/textbook/figures"`, and `NOTEBOOK_ID` is `"22b"`. It prints one line.

In [2], the records.

```
import csv  # reads the CSV tables of the Revision record
from fractions import Fraction  # exact fractions such as -382/441

import numpy as np  # arrays of numbers and their arithmetic
```

Three modules: `csv` reads tables stored as CSV files (comma-separated values: one line per row, the entries separated by commas); `Fraction` computes with exact fractions such as $-382/441$, without any rounding; `numpy`, called `np`, holds **arrays** (tables of numbers) and computes with whole arrays at once.

```
D16 = "Revision/dark_sector/dirac16complex"  # the record of dirac16complex
D00 = "Revision/dark_sector/dirac16complex00"  # the record of dirac16complex00
PALETTE = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300",
           "#4a3aa7", "#e34948"]  # the colours of the figures, in a fixed order
```

Two names for the two folders of the dark-sector record, used in every path below, and eight colours written as hexadecimal codes (blue, orange, green, yellow, pink, dark green, violet, red), which the figures use in this order.

```
def read_json(relative):
    """Read a JSON record of the repository into a Python dictionary."""
    return json.loads(repository_file(relative).read_text(encoding="utf-8"))
```

`read_json` reads a JSON file of the repository: `repository_file` gives its full path, `.read_text` reads it as text, and `json.loads` turns the text into a Python **dictionary** (pairs of a key and a value).

```
REPORTS = {"derive": f"{D16}/reports/derivation-checks.json",
           "history": f"{D16}/reports/ks-history-run.json",
           "eos": f"{D16}/reports/eos-checks.json",
           "independent": f"{D16}/reports/independent-checks.json",
           "derive00": f"{D00}/reports/python-derive-eos.json",
           "numerics00": f"{D00}/reports/python-independent-numerics.json"}
EXPECTED = {"derive": 30, "history": 5, "eos": 13, "independent": 9,
            "derive00": 49, "numerics00": 28}  # the checks each report holds
```

The six check reports of the dark-sector record, each under a short key, and the number of checks each one holds according to the record's own document (`Revision/docs/DARK_SECTOR_HYPOTHESES.md`, section 11.1): the derivation (`derive`, 30 checks), the 615 runs of the Kohn-Sham solver along the prescribed history (`history`, 5 checks: every run succeeded, the committed states are reproduced, the occupied levels stay the same at all 41 slices, and the conservation law along y of Chapter 15 and the particle number hold in every state), the equation of state of the Kohn-Sham gas (`eos`, 13), the independent free-gas computation (`independent`, 9), and the two implementations for dirac16complex00 (`derive00`, 49, and `numerics00`, 28). A string that starts with `f` is an **f-string**: each name in braces is replaced by its value, so `f"{D16}/reports/eos-checks.json"` is the full repository path.

```
for key, relative in REPORTS.items():
    entries = read_json(relative)["checks"]
    passed = [entry for entry in entries if entry["verdict"] == "PASS"]
    say(f"{relative}: {len(passed)} of {len(entries)} checks PASS")
    check(len(passed) == len(entries) == EXPECTED[key],
          f"all {EXPECTED[key]} checks of the report {key} are PASS",
          record=f"{relative}, all checks")
```

The loop takes the reports one after the other. `entries` is the list of checks of one report, each a dictionary with the keys `name`, `verdict` and `detail`; the **list comprehension** in square brackets keeps those whose verdict is PASS. The cell prints the count, and the check requires that every check passed and that the count is the expected one (the chained comparison `a == b == c` is true when all three are equal). These are the lines of Out [2] that name the six reports.

```
with open(repository_file(f"{D16}/outputs/eos-history.csv"), newline="",
          encoding="utf-8") as handle:
    rows = list(csv.DictReader(handle))
COLUMNS = ["N", "a4", "E", "P3", "Pt", "dw_eff_da4"]
SERIES = {}  # series name -> {column name: numpy array over the slices}
for name in sorted({row["series"] for row in rows}):
    mine = [row for row in rows if row["series"] == name]
```

```
    SERIES[name] = {column: np.array([float(row[column]) for row in mine])
                    for column in COLUMNS}
```

`with open(...) as handle:` opens the table of the Kohn-Sham history and closes it when the indented line has run; `csv.DictReader` reads it row by row, each row a dictionary from the column names of the first line to the texts of that row, and `list` collects all rows. `COLUMNS` names the six columns the notebook uses: the number of quanta N, the slice a_4, the energy E, the integrated pressures P_3 and P_t, and the record's derivative dw_{eff}/da_4. The braces with `for` inside, `{row["series"] for row in rows}`, make a **set** of the series names (each name once), which `sorted` puts in alphabetical order so that every run treats them in the same order. For each series, `mine` keeps its rows, and the inner dictionary comprehension makes, for each column, a numpy array of the numbers of that column (`float` turns a text into a number).

```
GAS = [name for name in SERIES if SERIES[name]["N"][0] in (136, 688)]
summary = read_json(f"{D16}/outputs/eos-summary.json")
free_gas = read_json(f"{D16}/outputs/independent-free-gas.json")
theory00 = read_json(f"{D00}/eos-theory.json")
numerics00 = read_json(f"{D00}/results/independent-numerics.json")
```

`GAS` lists the ten series of the gas proper, those with $N = 136$ or 688 (the four series with $N = 8$ hold only brane zero modes). The other four lines read the summary of the equation of state of dirac16complex, the independent free-gas computation, the model record of dirac16complex00 (implementation A) and the results of its field-equation implementation B.

```
say(f"{len(rows)} rows read: {len(SERIES)} series, {len(GAS)} of them with "
    "N = 136 or 688")
check(all(len(SERIES[name]["a4"]) == 41 for name in SERIES)
      and all(np.all(np.diff(SERIES[name]["a4"]) > 0) for name in SERIES),
      "every series has 41 slices in increasing order of a4")
```

The cell prints the counts (574 rows, 14 series, 10 gas series) and checks that every series has 41 slices and that a_4 increases along each (`np.diff` gives the differences of neighbouring entries; `np.all(... > 0)` is true when all are positive). The finite differences of the next cells rely on this order.

In [3], the observer's equation of state on the gas.

```
def x_over_e(name):
    """X/E = (P3 - Pt)/E of one series at its 41 slices."""
    return (SERIES[name]["P3"] - SERIES[name]["Pt"]) / SERIES[name]["E"]
```

`x_over_e` returns the array of $X/E = (P_3 - P_t)/E$ of one series; numpy subtracts and divides the arrays entry by entry.

```
gas = SERIES["N688_lam0"]  # the gas of N = 688 quanta with lambda = 0
a4, E = gas["a4"], gas["E"]
X = gas["P3"] - gas["Pt"]
h = a4[1] - a4[0]  # the step 0.05 between two slices
```

The series N688_lam0 is the main example. `a4` and `E` are its slices and energies, `X` its $X = P_3 - P_t$, and `h` the distance between two neighbouring slices, 0.05.

```
def derivative(f):
    """df/da4 at the slices 2 to 38: fourth-order central differences."""
    return (f[:-4] - 8 * f[1:-3] + 8 * f[3:-1] - f[4:]) / (12 * h)
```

`derivative` computes df/da_4 from the values of f at the slices by the **fourth-order central difference** $f'(x) \approx [f(x-2h) - 8f(x-h) + 8f(x+h) - f(x+2h)]/(12h)$, whose error shrinks like h^4 (Chapter 2 treats the order of a numerical method). The four **slices** of the array do the shifting: `f[:-4]` holds the entries 0 to 36, that is $f(x-2h)$ for the slices 2 to 38; `f[1:-3]` the entries 1 to 37, $f(x-h)$; `f[3:-1]` the entries 3 to 39, $f(x+h)$; and `f[4:]` the entries 4 to 40, $f(x+2h)$. So the result holds the derivative at the interior slices 2 to 38; the two slices at each end have no two neighbours on both sides.

```
worst = 0.0  # the largest relative violation of dE/da4 = -3X
for name in GAS:
    E_n = SERIES[name]["E"]
    X_n = SERIES[name]["P3"] - SERIES[name]["Pt"]
    violation = np.max(np.abs(derivative(E_n) + 3 * X_n[2:-2])) / np.max(np.abs(E_n))
    worst = max(worst, violation)
report("largest |dE/da4 + 3X|/max|E| over the 10 gas series", f"{worst:.3e}")
check(f"{worst:.3e}" == "1.235e-07", "the energy balance dE/da4 = -3X on the data",
      record=f"{D16}/reports/eos-checks.json, "
             "check conservation_dE_da4_equals_minus_3X")
```

The energy balance of Section 22.17, $dE/da_4 = -3X$, tested on the data. For each gas series the loop differentiates the solver's energies E and compares with $-3X$ from the solver's pressure integrals, at the interior slices (`X_n[2:-2]` drops the two end entries on each side, to match). The violation is measured relative to the largest energy of the series, and `worst` keeps the largest violation over the ten series. Printed with three digits it is 1.235×10^{-7}, the number of the record's check (Out [3]): the two sides agree to about seven digits, the accuracy of the solver and of the finite differences. The format `:.3e` writes a number in scientific notation with three digits after the point.

```
rho4_AB = E * np.exp(-3 * a4)  # definitions A and B: rho4 ~ E e^(-3 a4)
rho4_C = E  # definition C: rho4 ~ E
w_AB = -1 - derivative(np.log(rho4_AB)) / 3  # -1 - (1/3) d ln rho4/d ln a
w_C = -1 - derivative(np.log(rho4_C)) / 3
```

The observer's density under the definitions A and B ($E\,e^{-3a_4}$, up to a constant factor, which does not change a logarithmic derivative) and under C (E). The next two lines compute $w_{\text{eff}} = -1 - \frac{1}{3}\,d\ln\rho_4/d\ln a$ directly from its definition, with $d\ln a = da_4$: `np.log` takes the natural logarithm of every entry, and `derivative` differentiates it.

```
error_AB = np.max(np.abs(w_AB - (X / E)[2:-2]))
error_C = np.max(np.abs(w_C - (X / E - 1)[2:-2]))
report("largest |w_eff(A, B) - X/E| from the dilution", f"{error_AB:.1e}")
report("largest |w_eff(C) - (X/E - 1)| from the dilution", f"{error_C:.1e}")
```

```
check(max(error_AB, error_C) < 1e-7,
      "w_eff(A) = w_eff(B) = X/E and w_eff(C) = X/E - 1 on the data",
      record=f"{D16}/reports/derivation-checks.json, checks "
             "w_eff_A_equals_X_over_E and w_eff_C_equals_X_over_E_minus_1")
```

The dilution-inferred values are compared with the identities X/E and $X/E - 1$ of Section 22.17. Both differences are at most 1.7×10^{-8} (Out [3]): on the computed history the identities hold to the accuracy of the differences. The record proves them exactly with sympy; this is the numerical test.

```
ratios = np.concatenate([x_over_e(name) for name in GAS])
report("X/E of N688_lam0 at a4 = 0 and a4 = 2", f"{X[0] / E[0]:.4f}, "
       f"{X[-1] / E[-1]:.4f}")
report("X/E over all gas series, smallest and largest",
       f"{ratios.min():.6f}, {ratios.max():.6f}")
report("w_eff(C) over all gas series, smallest and largest",
       f"{ratios.min() - 1:.6f}, {ratios.max() - 1:.6f}")
```

`np.concatenate` joins the ten arrays of X/E into one array of 410 numbers. The cell prints X/E of N688_lam0 at the first and the last slice (0.2929 and 0.3183; `X[-1]` is the last entry), the smallest and largest X/E of all gas series (0.292893 and 0.328105), and the same moved down by 1, the range under C (-0.707107 to -0.671895).

```
check(f"{ratios.min():.6f}" == "0.292893" and f"{ratios.max():.6f}" == "0.328105",
      "the gas is radiation-like: X/E in [0.292893, 0.328105]",
      record=f"{D16}/reports/eos-checks.json, check gas_radiation_like_band")
check(all(np.all(np.diff(x_over_e(name)) > 0) for name in GAS),
      "X/E rises with a4 in every gas series (toward 1/3, not toward 0)",
      record=f"{D16}/reports/eos-checks.json, "
             "check gas_X_over_E_rises_toward_one_third")
```

The two checks compare the band with the six digits of the record's check, and require that X/E increases from each slice to the next in every gas series: the gas approaches radiation, it does not fall toward dust.

```
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
for colour, name in zip(PALETTE, ["N136_lam0", "N688_lam0", "N688_lamp2"]):
    left.plot(SERIES[name]["a4"], x_over_e(name), color=colour, label=name)
    right.plot(SERIES[name]["a4"], x_over_e(name) - 1, color=colour, label=name)
```

`plt.subplots(1, 2, ...)` makes a figure with two panels side by side, 9.0 by 3.8 inches, and names them `left` and `right`. `zip` pairs the first three colours with three series; each series is drawn in the left panel as X/E (definitions A and B) and in the right panel as $X/E - 1$ (definition C), with its name as the label of the legend.

```
left.axhline(1 / 3, color="black", linestyle="--", linewidth=0.8,
             label="radiation, 1/3")
right.axhline(-2 / 3, color="black", linestyle="--", linewidth=0.8,
              label="1/3 - 1")
left.set_title("definitions A and B: $w_{eff} = X/E$")
right.set_title("definition C: $w_{eff} = X/E - 1$")
```

`axhline` draws a horizontal line across a panel: the radiation value 1/3 on the left (dashed: the line style written as two minus signs) and the same value moved down by 1, $-2/3$, on the right. The titles name the definitions; text between dollar signs is set as mathematics.

```
for ax in (left, right):
    ax.set_xlabel("$a_4$ along the prescribed history")
    ax.legend(fontsize=8)
left.set_ylabel("$w_{eff}$")
save_figure(fig, "gas_three_definitions",
            "The equation of state $w_{eff}$ that a 3-space observer infers from "
...)
```

Both panels get the label of the horizontal axis and a legend; the left one the label of the vertical axis. `save_figure` saves the figure as Figure 22b.1 with its caption (shortened here; the full caption is printed under the figure in Section 22.22). **What to see:** the two panels have the same shape, moved by exactly 1. On the left, every curve rises toward the dashed radiation line, the larger gas ($N = 688$) a little lower than the smaller one, and the interaction (λ_2, green) changes almost nothing. On the right, the same curves lie between -0.71 and -0.67: far above -1, and rising, which is the thawing sign. The physics of the gas is the same in both panels; only the ASSUMED definition of the observer's density differs.

In [4], the CPL tangents of the gas.

```
TODAY = [0.5, 1.0, 1.5, 2.0]  # the four choices of a4,today of the record
index = [int(round(t / h)) for t in TODAY]  # their positions in the slice list
recorded = {entry["series"]: entry for entry in summary["series"]}
say("a4,today   w0 under C   wa (every definition)")
differences = []
```

The four values of $a_{4,\mathrm{today}}$ that the record uses, and their positions in the list of slices: $0.5/0.05 = 10$, then 20, 30 and 40 (`round` removes the tiny error of the division, and `int` makes a whole number). `recorded` maps each series name to its entry of the summary. The cell prints a header line and starts an empty list `differences`.

```
for t, i, entry in zip(TODAY, index, recorded["N688_lam0"]["cplTangent"]):
    w0 = X[i] / E[i] - 1  # w_eff(C) today
    wa = -gas["dw_eff_da4"][i]  # w_a = -dw/da at a = 1, and da = da4 there
    say(f"{t:8.1f}   {w0:10.4f}   {wa:12.5f}")
    differences += [abs(w0 - entry["w_eff_C"]["w0"]), abs(wa - entry["w_eff_C"]["wa"])]
check(max(differences) < 1e-12, "the CPL tangents of N688_lam0",
      record=f"{D16}/outputs/eos-summary.json, key cplTangent of N688_lam0")
```

For each choice of today, the tangent of Section 22.18: w_0 is $w_{\mathrm{eff}}(C) = X/E - 1$ at that slice, and $w_a = -dw_{\mathrm{eff}}/da_4$ there, read from the record's column `dw_eff_da4` (the derivative of X/E; under C the constant -1 does not change it). The format `8.1f` writes a number in 8 places with one digit after the point. The two numbers are compared with the summary's tangent of the same series; `+=` appends both differences to the list. The table of Out [4] is the table of Section 22.18.

```
tangents = [-SERIES[name]["dw_eff_da4"][i] for name in GAS for i in index]
report("tangent wa of every gas series, smallest and largest",
       f"{min(tangents):.6f}, {max(tangents):.6f}")
check(f"{min(tangents):.6f}" == "-0.020523" and f"{max(tangents):.6f}" == "-0.008894"
      and f"{ratios.min() - 1:.6f}" == "-0.707107"
      and f"{ratios.max() - 1:.6f}" == "-0.671895",
      "every gas tangent wa is negative (thawing sign) and far below 0.60",
      record=f"{D16}/reports/eos-checks.json, check gas_cpl_thawing_sign_small")
```

The tangent w_a of every gas series at every choice of today (40 numbers; the comprehension with two `for` runs over both). The smallest is -0.020523 and the largest -0.008894: all negative, the thawing sign, and at most about one thirtieth of the observed 0.60. The check compares them, and the range of $w_{\text{eff}}(C)$ from In [3], with the six digits of the record's check.

In [5], the bulk band and the brane band.

```
bulk = free_gas["bulkBand_n2_1"]  # the lowest massive (odd-parity) level, shell 1
brane = free_gas["braneBand_n2_1"]  # the brane-band level of the same shell
a4_band = np.array([point["a4"] for point in bulk])
w_bulk = np.array([point["w_eff_A_B"] for point in bulk])
w_brane = np.array([point["w_eff_A_B"] for point in brane])
```

The independent free-gas output holds two lists of points, one for the lowest level of the massive bulk band of the shell $n_2 = 1$ and one for the brane-band level of the same shell, each point a dictionary with the slice `a4` and the value `w_eff_A_B` of w_{eff} under A and B. The three arrays collect the slices (-3 to 4 in steps of 0.25) and the two curves.

```
report("w_eff(A) of the bulk level at a4 = -3 and a4 = 4",
       f"{w_bulk[0]:.6f}, {w_bulk[-1]:.3e}")
report("ratio w(4)/w(3.75) of the bulk level", f"{w_bulk[-1] / w_bulk[-2]:.6f}")
report("w_eff(A) of the brane level, smallest and largest",
       f"{w_brane.min():.6f}, {w_brane.max():.6f}")
```

The first and last value of the bulk level (0.239626 and 2.268×10^{-3}), the ratio of its last two values (0.765177, which Section 22.18 compares with $e^{-0.25}$), and the range of the brane level (0.291594 to 0.333275).

```
check(a4_band[0] == -3 and a4_band[-1] == 4 and np.all(np.diff(w_bulk) < 0)
      and f"{w_bulk[0]:.6f}" == "0.239626" and f"{w_bulk[-1]:.3e}" == "2.268e-03",
      "the bulk level falls from 0.239626 to 2.268e-03 (dark-matter-like)",
      record=f"{D16}/reports/independent-checks.json, check bulk_band_dark_matter_law")
check(f"{w_brane.min():.6f}" == "0.291594" and f"{w_brane.max():.6f}" == "0.333275",
      "the brane level stays between 0.291594 and 0.333275 (radiation-like)",
      record=f"{D16}/reports/independent-checks.json, check brane_band_radiation_law")
```

The first check requires the range of slices, that the bulk level falls at every step (all differences negative), and its two end values as the record's check states them; the second the range of the brane level.

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
ax.plot(a4_band, w_bulk, "o-", color=PALETTE[1], markersize=3,
        label="bulk band (massive), one level")
ax.plot(a4_band, w_brane, "s-", color=PALETTE[0], markersize=3,
        label="brane band (massless at k = 0), one level")
ax.plot(a4, X / E, color=PALETTE[2], linewidth=2.5, label="the gas N688_lam0")
```

One panel. The bulk level is drawn in orange with small circles (`"o-"`: points joined by lines), the brane level in blue with small squares, and the gas N688_lam0 of In [3] as a thick green line over its range $0 \le a_4 \le 2$.

```
ax.axhline(1 / 3, color="black", linestyle="--", linewidth=0.8)
ax.axhline(0.0, color="black", linestyle=":", linewidth=0.8)
ax.set_xlabel("$a_4$")
ax.set_ylabel("$w_{eff}$ under definitions A and B")
ax.legend(fontsize=8)
save_figure(fig, "bulk_and_brane_bands",
            "Which quanta of dirac16complex behave like dark matter. Horizontal "
...)
```

The radiation value 1/3 (dashed) and the dust value 0 (dotted), the axis labels, the legend, and Figure 22b.2. **What to see:** only the orange curve goes from near radiation toward dust, the dark-matter-like behaviour of the flat-limit law of Section 22.18. The blue curve dips slightly and then climbs to 1/3, and the green gas, which consists of brane-band quanta, follows the blue behaviour. The time-varying dark-matter-like equation of state exists in the theory, but in a band that the computed states leave empty.

In [6], the condensate.

```
u = Fraction(-382, 441)  # lambda S/m, CHOSEN so that the ratio is -0.764
ratio = u / (2 + u)  # p/rho = (lambda S^2/2)/(m S + lambda S^2/2) = u/(2 + u)
density = 1 + u / 2  # rho/(m S)
report("the ratio u/(2 + u) at u = -382/441", f"{ratio} = {float(ratio)}")
report("rho/(m S) = 1 + u/2", f"{density} = {float(density):.12f}")
```

`Fraction(-382, 441)` is the exact fraction $-382/441$; arithmetic with it stays exact. The ratio $u/(2+u)$ of Section 22.18 comes out as the fraction $-191/250 = -0.764$, and $\rho/(mS) = 1 + u/2 = 250/441 = 0.566893424036$ (Out [6]; `float` turns a fraction into a decimal number).

```
check(ratio == Fraction(-764, 1000)
      and abs(float(u) - summary["condensate"]["u_for_ratio_minus_0p764"]) < 1e-12,
      "the condensate ratio is -0.764 at u = -382/441 (one CHOSEN parameter)",
      record=f"{D16}/reports/derivation-checks.json, "
             "check condensate_ratio_equal_unite_constant_w")
```

The ratio must equal $-764/1000$ exactly (`Fraction` reduces it to $-191/250$ and compares exactly), and u must agree with the value the summary records.

```
grid = np.linspace(-3.0, 1.0, 4001)
grid = grid[np.abs(grid + 2) > 1e-9]  # leave out the pole u = -2
below = grid / (2 + grid) < -1  # where the ratio is phantom
check(np.array_equal(below, (grid > -2) & (grid < -1)),
      "the ratio is below -1 exactly for -2 < u < -1",
      record=f"{D00}/reports/python-derive-eos.json, "
             "check condensate_phantom_interval")
```

A test of the phantom window derived in Section 22.18 on 4001 values of u from -3 to 1 (step 0.001). The second line removes the value $u = -2$, where the ratio has a zero denominator: the condition in square brackets is an array of true and false, and indexing with it keeps the entries where it is true. `below` marks where the ratio is below -1; `(grid > -2) & (grid < -1)` marks the window (the operator between the two conditions means "and", applied entry by entry); `np.array_equal` requires that the two markings agree at every point.

```
fig, ax = plt.subplots(figsize=(7.0, 4.2))
for part in (grid[grid < -2], grid[grid > -2]):
    ax.plot(part, part / (2 + part), color=PALETTE[0])
ax.axvspan(-2, -1, color=PALETTE[7], alpha=0.12, label="phantom ratio, -2 < u < -1")
```

The ratio is drawn in two pieces, left and right of the pole, so that no line jumps across it. `axvspan` shades the vertical band $-2 < u < -1$ in pale red (`alpha=0.12` makes it nearly transparent).

```
ax.axhline(0.0, color="black", linestyle="--", linewidth=0.8,
           label="$w_{eff}$, definitions A and B")
ax.axhline(-1.0, color="black", linestyle=":", linewidth=0.8,
           label="$w_{eff}$, definition C")
ax.plot([float(u)], [float(ratio)], "o", color=PALETTE[1],
        label="CHOSEN: u = -382/441 gives -0.764")
ax.set_ylim(-4.0, 4.0)
```

The two values that the observer infers for any condensate, 0 (A, B; dashed) and -1 (C; dotted), and the CHOSEN point in orange; `set_ylim` shows the vertical range -4 to 4, so that the steep branches near the pole are cut off.

```
ax.set_xlabel("$u = \\lambda S/m$")
ax.set_ylabel("ratio $p/\\rho$")
ax.legend(fontsize=8, loc="lower right")
save_figure(fig, "condensate_ratio",
            "The constant ratio $p/\\rho = u/(2 + u)$ of a homogeneous condensate "
...)
```

The axis labels (the doubled backslash `\\` puts one backslash into the text, which the mathematics needs before `lambda` and `rho`), the legend in the lower right corner, and Figure 22b.3. **What to see:** the ratio can take any value, and the value -0.764 is one point on the curve, reached by choosing u; it says nothing about the observer, whose two readings are the horizontal lines, the same for every u. Left of the pole the ratio is above 1, right of it the curve climbs from far below through the phantom band to -1 at $u = -1$, to 0 at $u = 0$ and on toward 1.

In [7], the mixtures.

```
r0 = Fraction(417, 583)  # gas/condensate energy today, CHOSEN for w0(C) = -0.861
w0_exact = r0 / (3 * (r0 + 1)) - 1  # w_eff(C) today
wa_exact = r0 / (3 * (r0 + 1) ** 2)  # w_a of the exact law: positive (freezing)
report("exact law: w0 under C and wa", f"{w0_exact}, {float(wa_exact):.6f}")
check(w0_exact == Fraction(-861, 1000) and f"{float(wa_exact):.6f}" == "0.081037",
      "radiation plus condensate with w0(C) = -0.861 has wa = 0.081037 > 0",
      record=f"{D16}/reports/derivation-checks.json, "
             "check mixture_C_matching_w0_unite")
```

The exact law of Section 22.18 for a radiation-like gas plus a condensate, with exact fractions: at the CHOSEN $r_0 = 417/583$, w_0 under C is exactly $-861/1000$ and $w_a = 0.081037$, positive (`**` is the power).

```
def mixture_w_C(share):
    """w_eff(C) of the gas N688_lam0 plus a condensate along the history; the gas
    has the energy share "share" today (a4 = 2)."""
    condensate = E[-1] * (1 - share) / share  # constant condensate energy
    return X / (E + condensate) - 1
```

`mixture_w_C` mixes the computed gas with a condensate and returns $w_{\text{eff}}(C)$ at all 41 slices. Today is the last slice, $a_4 = 2$. If the gas has the share `share` of the energy today, the condensate energy is $E_c = E(2)(1 - \text{share})/\text{share}$ (from $E(2)/(E(2) + E_c) = \text{share}$), constant in time. The condensate adds energy but no X, so the mixture has $w_{\text{eff}}(C) = X/(E + E_c) - 1$.

```
share = 0.139 / (X[-1] / E[-1])  # CHOSEN: gives w_eff(C) = -0.861 today
w_mix = mixture_w_C(share)
```

Today $w_{\text{eff}}(C) = \text{share} \cdot X/E - 1$, so $w_{\text{eff}}(C) = -0.861$ needs share $= 0.139/(X/E)$ at $a_4 = 2$: $0.139/0.3183 = 0.436703$. This share is CHOSEN; `w_mix` is the mixture's history.

```
# w_a = -dw/da4 at a4 = 2, one-sided fourth-order differences (as the record)
wa_mix = -(3 * w_mix[-5] - 16 * w_mix[-4] + 36 * w_mix[-3] - 48 * w_mix[-2]
           + 25 * w_mix[-1]) / (12 * h)
recorded_mix = summary["mixture_C_w0_minus_0p861"]
```

At the last slice there are no neighbours on the right, so the derivative uses the **one-sided fourth-order difference** $f'(x) \approx [3f(x-4h) - 16f(x-3h) + 36f(x-2h) - 48f(x-h) + 25f(x)]/(12h)$, the same stencil as the record; `w_mix[-5]` is the fifth entry from the end. The minus sign makes it w_a. `recorded_mix` is the record's entry for this mixture.

```
report("gas share today for w_eff(C) = -0.861 (CHOSEN)", f"{share:.6f}")
report("w0 and wa of that mixture under C", f"{w_mix[-1]:.3f}, {wa_mix:.6f}")
check(abs(share - recorded_mix["gas_fraction_today"]) < 1e-12
      and abs(wa_mix - recorded_mix["wa"]) < 1e-10 and wa_mix > 0,
      "the computed gas mixture with w0(C) = -0.861 is freezing, wa = 0.067014",
      record=f"{D16}/reports/eos-checks.json, check mixture_C_w0_unite_has_positive_wa")
```

The share 0.436703 and $(w_0, w_a) = (-0.861, 0.067014)$ are printed and compared with the record: w_a is positive, freezing, the opposite sign to the observed -0.60.

```
a_obs = np.exp(a4 - 2.0)  # the observer scale factor, a = 1 today (a4 = 2)
fig, ax = plt.subplots(figsize=(7.0, 4.2))
for colour, part in zip(PALETTE, [0.25, share, 0.75, 1.0]):
    ax.plot(a_obs, mixture_w_C(part), color=colour,
            label=f"gas share today {part:.3f}")
ax.plot(a_obs, -0.861 - 0.60 * (1 - a_obs), "k--", label="Unite CPL line")
ax.axhline(-1.0, color="black", linestyle=":", linewidth=0.8)
```

The observer's scale factor $a = e^{a_4 - 2}$ runs from $e^{-2} = 0.135$ to 1 over the history. Four mixtures are drawn against it, with the gas shares 0.25, the CHOSEN 0.437, 0.75 and 1 (pure gas), together with the observed CPL line (the style `"k"` followed by two minus signs: black and dashed) and the value -1 (dotted).

```
ax.set_xlabel("observer scale factor $a = e^{a_4 - 2}$")
ax.set_ylabel("$w_{eff}$ under definition C")
ax.legend(fontsize=8)
save_figure(fig, "mixtures_against_unite",
            "Mixtures of the computed Kohn-Sham gas N688_lam0 with a condensate, "
...)
```

Labels, legend and Figure 22b.4. **What to see:** every curve that contains condensate slopes DOWN toward -1 as a grows, because the gas redshifts and the condensate's -1 takes over; the observed line slopes UP. The orange curve meets the observed line at $a = 1$ (that is how its share was chosen) and leaves it at once in the opposite direction. No choice of the share turns one slope into the other.

In [8], the models of dirac16complex00.

```
def model(a, components):
    """The total energy density and the sum of eps_i rho_i of a WKB model at a
    (a number or an array).  Each component is (kind, weight at a = 1, s)."""
    a = np.asarray(a, dtype=float)
    rho_total, eps_rho = np.zeros_like(a), np.zeros_like(a)
    for kind, weight, s in components:
```

`model` evaluates the adiabatic gas of Section 22.19 at the scale factor `a`, which may be one number or an array (`np.asarray` makes an array of it either way). A model is a list of components, each a **tuple** `(kind, weight, s)`: what it is, its energy density at $a = 1$, and its parameter. `rho_tota l` will hold $\sum_i \rho_i$ and `eps_rho` the sum $\sum_i \varepsilon_i \rho_i$; `np.zeros_like(a)` makes arrays of zeros of the shape of `a`.

```
        if kind == "condensate":  # rho constant, eps = 0
            rho, eps = weight + 0 * a, 0 * a
        elif kind == "kmode":  # q = 0 and s = k^2/(k^2 + m^2) at a = 1
            r = s / (1 - s)  # k^2/m^2
            rho = weight * np.sqrt((r / a**2 + 1) / (r + 1))  # omega(a)/omega(1)
            eps = (r / a**2) / (3 * (r / a**2 + 1))
```

A condensate (with $\lambda = 0$) has constant energy density and $\varepsilon = 0$; adding `0 * a` gives the numbers the shape of `a`. A gas of modes with $q = 0$ (`"kmode"`) is described by $s = k^2/(k^2 + m^2)$ at $a = 1$;

$r = s/(1-s)$ is then k^2/m^2. Its energy density is $\rho_i \propto \omega = m\sqrt{1 + r/a^2}$, scaled to the weight at $a = 1$, and $\varepsilon = (k^2/a^2)/(3\omega^2) = (r/a^2)/(3(r/a^2 + 1))$, the formula of Section 22.19 with $q = 0$ and $m = 1$.

```
        elif kind == "qmode":  # k = 0 and s = q^2/m^2 at a = 1
            rho = weight * np.sqrt((1 - s * a**2) / (1 - s))
            eps = s * a**2 / (3 * (1 - s * a**2))
        else:  # "ghost": massless modes of NEGATIVE classical energy
            rho, eps = -weight / a, 1 / 3 + 0 * a
```

An extra-time mode with $k = 0$ (`"qmode"`), $s = q^2/m^2$ at $a = 1$: $\omega = m\sqrt{1 - sa^2}$, so ρ_i is the weight times $\sqrt{(1 - sa^2)/(1 - s)}$, and $\varepsilon = sa^2/(3(1 - sa^2))$. The ghost-like component: massless modes ($\varepsilon = 1/3$) of NEGATIVE energy density $-G/a$, with G the weight.

```
        rho_total = rho_total + rho
        eps_rho = eps_rho + eps * rho
    return rho_total, eps_rho
```

Each component adds its ρ_i and its $\varepsilon_i\rho_i$ to the two sums, which the function returns.

```
def w_model(a, components):
    """w_eff under definition C: sum eps_i rho_i / sum rho_i - 1."""
    rho_total, eps_rho = model(a, components)
    return eps_rho / rho_total - 1
```

`w_model` is the mixture formula of Section 22.19 under C.

```
def tangent(components, d=1e-3):
    """CPL tangent w0 = w(1), wa = -dw/da at a = 1 (fourth-order differences)."""
    f = [float(w_model(1 + k * d, components)) for k in (-2, -1, 1, 2)]
    slope = (f[0] - 8 * f[1] + 8 * f[2] - f[3]) / (12 * d)  # dw/da at a = 1
    return float(w_model(1.0, components)), -slope
```

`tangent` returns $w_0 = w(1)$ and $w_a = -dw/da$ at $a = 1$; the slope is the fourth-order central difference of In [3], with the step $d = 10^{-3}$ in a.

```
M5_RECORD = theory00["models"]["M5_with_ghost_component"]
G = float(Fraction(M5_RECORD["parameters"]["G"]))  # ghost share 3/10, a choice
s5 = float(M5_RECORD["parameters"]["s"])  # CHOSEN with q5 to fit the Unite line
q5 = float(M5_RECORD["parameters"]["Omega_q"])
c5 = float(M5_RECORD["parameters"]["Omega_c"])
```

The parameters of M5 are read from the record: the ghost share $G = 3/10$ (stored as the text `"3/10"`, which `Fraction` reads), and s, the mode share Ω_q and the condensate share Ω_c, which the record found by solving the two fit conditions (stored with 12 digits).

```
MODELS = {"M2": [("kmode", 1.0, 417 / 1000)],  # s CHOSEN: w0 = -0.861
          "M3": [("qmode", 1.0, 417 / 1417)],  # s CHOSEN: w0 = -0.861
          "M4": [("condensate", 1 - 57963 / 264037, 0.0),  # both CHOSEN:
                 ("qmode", 57963 / 264037, 264037 / 403037)],  # tangent = Unite
          "M5": [("condensate", c5, 0.0), ("qmode", q5, s5), ("ghost", G, 0.0)]}
```

The four models of the table of Section 22.19, each as a list of components with its CHOSEN parameters. The weights of M4 and M5 add up to 1 at $a = 1$: $(1 - \Omega_q) + \Omega_q = 1$, and $\Omega_c + \Omega_q - G = 0.705325 + 0.594675 - 0.3 = 1$.

```
EXPECTED_WA = {"M2": 81037 / 500000, "M3": -196963 / 500000, "M4": -0.6}
RECORD_CHECK = {"M2": "M2_tangent_exact", "M3": "M3_tangent_exact",
                "M4": "M4_tangent_equals_unite"}
for name in ("M2", "M3", "M4"):
    w0, wa = tangent(MODELS[name])
    report(f"{name} tangent (w0, wa) under C", f"({w0:.6f}, {wa:.6f})")
    check(abs(w0 + 0.861) < 1e-12 and abs(wa - EXPECTED_WA[name]) < 1e-8,
          f"the tangent of {name}",
          record=f"{D00}/reports/python-derive-eos.json, check {RECORD_CHECK[name]}")
```

The exact tangent slopes of the record ($81037/500000 = 0.162074$, $-196963/500000 = -0.393926$ and -0.6) and the names of the record's checks. For each model the cell computes the tangent and checks $w_0 = -0.861$ to 10^{-12} and w_a to 10^{-8}, the accuracy of the finite difference. Out [8] prints $(-0.861, 0.162074)$, $(-0.861, -0.393926)$ and $(-0.861, -0.600000)$.

```
a_fit = np.linspace(0.5, 1.0, 101)  # 101 points of a in [1/2, 1], as the record
wa_fit, w0_fit = np.polyfit(1 - a_fit, w_model(a_fit, MODELS["M5"]), 1)
report("M5 least-squares fit over a in [1/2, 1]", f"({w0_fit:.6f}, {wa_fit:.6f})")
check(abs(w0_fit + 0.861) < 1e-9 and abs(wa_fit + 0.6) < 1e-9,
      "the M5 fit equals the Unite pair (by construction)",
      record=f"{D00}/reports/python-derive-eos.json, check M5_fit_equals_unite")
```

The least-squares fit of M5, as in the record: 101 equally spaced values of a from $1/2$ to 1, and `np.polyfit(x, y, 1)` finds the straight line $y = c_1 x + c_0$ closest to the points (the sum of the squared vertical distances is smallest) and returns $[c_1, c_0]$. With $x = 1 - a$ the line is $w_0 + w_a(1 - a)$, so $c_1 = w_a$ and $c_0 = w_0$. The fit is the observed pair to 10^{-9}: by construction, since s and Ω_q were chosen for exactly this.

```
def crossings(components):
    """The values of a in [1/3, 1] where w = -1 (400 intervals, then bisection)."""
    points = np.linspace(1 / 3, 1, 401)
    values = w_model(points, components) + 1
    found = []
```

`crossings` finds where $w = -1$ between $a = 1/3$ and 1: it divides the range into 400 intervals (401 points) and evaluates $w + 1$, which changes sign at a crossing.

```
    for low, high, v_low, v_high in zip(points, points[1:], values, values[1:]):
        if v_low * v_high < 0:
```

```
            for _ in range(60):  # halve the interval 60 times
                middle = (low + high) / 2
                v_middle = float(w_model(middle, components) + 1)
                if v_low * v_middle <= 0:
                    high = middle
                else:
                    low, v_low = middle, v_middle
            found.append((low + high) / 2)
    return found
```

`zip(points, points[1:], ...)` runs over neighbouring pairs of points with their values. Where the product of the two values is negative, the sign changes inside the interval, and **bisection** narrows it: the middle is evaluated, and the half in which the sign still changes is kept (if the left value and the middle value have opposite signs or the middle is zero, the crossing is in the left half, so `high` moves to the middle; otherwise in the right half). After 60 halvings the interval is 2^{-60} times its original width 1/600, far below the precision of the numbers, and its middle is recorded. The underscore `_` is the name of a loop variable that is not used.

```
cross_M5 = crossings(MODELS["M5"])
no_ghost = [("condensate", 1 - q5, 0.0), ("qmode", q5, s5)]  # M5 without ghost
lowest_M4 = float(np.min(w_model(np.arange(1, 301) / 300, MODELS["M4"])))
crossing_A = float(M5_RECORD["N2"]["crossings_of_minus_1_in_[1/3,1]"][0])
crossing_B = numerics00["models"]["M5_with_ghost_component"]["crossing_N2"]
M4_RECORD = theory00["models"]["M4_condensate_plus_extra_time_mode"]
```

The crossings of M5; the same model with its ghost-like part removed (the condensate share becomes $1-\Omega_q$, so that the total is 1 again); the smallest w of M4 at $a = 1/300, 2/300, \ldots, 1$ (`np.arange(1, 301)` gives the whole numbers 1 to 300); the record's crossing from implementation A (stored as a text in a list) and from the field-equation implementation B; and the record's entry for M4.

```
report("M5 crosses w = -1 at a", f"{cross_M5[0]:.11f}")
report("the same crossing from the field equation (implementation B)",
       f"{crossing_B}")
report("lowest w_eff(C) of M4 for a = 1/300 to 1", f"{lowest_M4:.12f}")
```

Out [8] prints the crossing $a = 0.77909966367$, implementation B's 0.77905405, and the lowest w of M4, -0.999999214228: above -1.

```
check(len(cross_M5) == 1 and abs(cross_M5[0] - crossing_A) < 1e-10,
      "M5 crosses -1 once, at a = 0.7791",
      record=f"{D00}/reports/python-derive-eos.json, check M5_crosses_minus_1")
check(crossings(no_ghost) == [] and crossings(MODELS["M4"]) == []
      and abs(lowest_M4 - float(M4_RECORD["min_w_N2_on_(0,1]"])) < 1e-11,
      "no crossing on 1/3 <= a <= 1 without the ghost: M5 without it, and M4",
      record=f"{D00}/reports/python-derive-eos.json, checks "
             "M5_without_ghost_no_crossing and M4_never_phantom")
check(abs(crossing_B - cross_M5[0]) < 1e-4,
      "implementation B finds the M5 crossing within 1e-4",
      record=f"{D00}/reports/python-independent-numerics.json, "
             "check B_vs_A_M5_crossing")
```

Three checks: M5 crosses -1 exactly once, where the record says; neither M5 without its ghost-like part nor M4 crosses on $1/3 \le a \le 1$, the range that `crossings` searches (an empty list `[]`), and the lowest value of M4 on $1/300 \le a \le 1$ is the record's; and the field-equation implementation B agrees on the crossing within 10^{-4} (the difference is 4.6×10^{-5}, the accuracy of B's numerical solution).

```
a_plot = np.linspace(1 / 3, 1.0, 300)
fig, (left, right) = plt.subplots(1, 2, figsize=(9.0, 3.8))
LABELS = {"M2": "M2, gas (1 CHOSEN)", "M3": "M3, extra-time mode (1 CHOSEN)",
          "M4": "M4 (2 CHOSEN)", "M5": "M5 (2 CHOSEN, ghost-like part)"}
for colour, name in zip(PALETTE, MODELS):
    left.plot(a_plot, w_model(a_plot, MODELS[name]), color=colour,
              label=LABELS[name])
```

300 values of a from $1/3$ to 1 and a figure with two panels. The labels state for each model how many parameters were CHOSEN. The loop draws $w_{\mathrm{eff}}(C)$ of the four models in the left panel (looping over a dictionary gives its keys, here in the order M2, M3, M4, M5).

```
left.plot(a_plot, -0.861 - 0.60 * (1 - a_plot), "k--", label="Unite CPL line")
left.axhline(-1.0, color="black", linestyle=":", linewidth=0.8)
left.set_xlabel("observer scale factor $a$")
left.set_ylabel("$w_{eff}$ under definition C")
left.legend(fontsize=7)
```

The observed CPL line (dashed), the value -1 (dotted), the labels and the legend of the left panel.

```
PARTS = [("condensate", MODELS["M5"][0]), ("extra-time mode", MODELS["M5"][1]),
         ("ghost-like part", MODELS["M5"][2])]
for colour, (label, component) in zip(PALETTE[4:], PARTS):
    right.plot(a_plot, model(a_plot, [component])[0], color=colour, label=label)
right.plot(a_plot, model(a_plot, MODELS["M5"])[0], "k-", label="total")
right.axhline(0.0, color="black", linewidth=0.6)
```

The right panel shows the **populations** of M5: the energy density of each of its three parts, computed by calling `model` with a list that holds only that part (`[0]` takes the first of the two returned sums, the energy density), in the colours from the fifth on (`PALETTE[4:]`), and their total in black, with the zero line.

```
right.set_xlabel("observer scale factor $a$")
right.set_ylabel("energy density, M5")
right.legend(fontsize=7)
fig.subplots_adjust(wspace=0.3)  # room between the two panels
save_figure(fig, "models_against_unite",
            "The models of dirac16complex00 under definition C against the "
...)
```

Labels and legend of the right panel; `subplots_adjust(wspace=0.3)` widens the gap between the panels so that the labels do not overlap; Figure 22b.5. **What to see:** on the left, M2, M3 and M4 pass through $w = -0.861$ at $a = 1$, because their parameters were CHOSEN to give that value

today. M5 ends a little higher, at $w = -0.8396$ (the record `eos-theory.json`, model M5, key `w(a=1)`: -0.839622680899), because its two parameters were CHOSEN so that its least-squares fit over a from 1/2 to 1, not its value today, equals the Unite pair $(-0.861, -0.60)$. Read the curves from left to right, in the direction in which a grows. M2 falls toward -1: it slopes the wrong way (freezing, $w_a > 0$), while the dashed Unite line rises. M3 and M4 rise away from -1: they slope the right way (thawing, $w_a < 0$), but toward the past they flatten above the dotted line -1: with positive energies and real frequencies they cannot go below it, and the whole plotted range lies before the turning points of their extra-time modes ($a_{\rm turn} = 1.8434$ for M3, 1.2355 for M4); past those points the bound is not established. M4 even has the right slope at $a = 1$, by construction. Only M5 follows the dashed line through -1, and it does so only because its ghost-like part, the violet curve on the right, carries a negative energy that grows like $1/a$ toward the past. Nothing in the field equations selects any of these populations.

In [9], the comparison with the observed values.

```
unite = theory00["models"]["unite"]
w0_u, wa_u = Fraction(unite["w0"]), Fraction(unite["wa"])
crossing_u = 1 + (1 + w0_u) / wa_u  # w0 + wa (1 - a) = -1 solved for a
```

The record stores the observed values as exact fractions in text form (`"-861/1000"`, `"-3/5"`), which `Fraction` reads. The crossing of the CPL line with -1 is solved as in Section 22.16: $a = 1 + (1 + w_0)/w_a$.

```
TABLE = [("w = -0.764", "condensate ratio, u = -382/441", "1 for 1"),
         ("w0 = -0.861", "M2; M3; gas plus condensate", "1 for 1"),
         ("(w0, wa) = (-0.861, -0.60)", "M4 tangent; M5 fit", "2 for 2"),
         ("crossing of -1, a = 0.7683", "only M5", "ghost-like part")]
say("Unite value | reproduced by | CHOSEN parameters")
for row in TABLE:
    say(" | ".join(row))
report("the Unite CPL line crosses w = -1 at a", f"{crossing_u}")
```

A short form of the comparison table of Section 22.19: each observed value, what reproduces it, and how many parameters were CHOSEN for it. `" | ".join(row)` writes the three texts of a row with a vertical bar between them. The crossing is printed as the fraction 461/600.

```
check(w0_u == Fraction(-861, 1000) and wa_u == Fraction(-3, 5)
      and crossing_u == Fraction(unite["crossing_of_minus_1"]["a"])
      and crossing_u == Fraction(461, 600),
      "the Unite values and their crossing a = 461/600",
      record=f"{D00}/reports/python-derive-eos.json, check unite_crossing_point")
files = [f"{FIGURE_FOLDER}/{NOTEBOOK_ID}_{number}_{name}.png"
         for name, number in FIGURE_NUMBERS.items()]
check(len(files) == 5 and all(output_file(path).is_file() for path in files),
      "every figure file of this notebook exists")
all_checks_passed()
```

The observed values are the ones `Revision/README.md` quotes, and their crossing is exactly the record's 461/600. `files` rebuilds the names of the saved figures from the dictionary `FIGURE_NUMBERS` of the set-up cell, and the last check requires five of them, all present. The last line prints ALL 28 CHECKS PASSED (notebook 22b).

22.24 What Notebook 22b found, and what remains open for the dark sector

The results. Every number below is printed by the notebook in the cell named, and every one reproduces the Revision record named in the same cell.

result	value	status	where
energy balance $dE/da_4 = -3X$ on the history	relative violation 1.235×10^{-7}	PROVED exactly in the record; COMPUTED here	Out [3]
$w_{\text{eff}}(A,B) = X/E$, $w_{\text{eff}}(C) = X/E - 1$ from the dilution	to 1.7×10^{-8}	PROVED in the record; COMPUTED here	Out [3]
X/E of the gas, all series	0.292893 to 0.328105, rising	COMPUTED (record)	Out [3]
$w_{\text{eff}}(C)$ of the gas	-0.707107 to -0.671895	COMPUTED; rests on the ASSUMED definition C	Out [3]
tangent w_a of the gas	-0.020523 to -0.008894	COMPUTED (record)	Out [4]
bulk-band level, $w_{\text{eff}}(A)$	0.239626 to 2.268×10^{-3}	COMPUTED (record)	Out [5]
condensate ratio at $u = -382/441$	-0.764 exactly	PROVED; u CHOSEN	Out [6]
gas plus condensate, $w_0(C) = -0.861$	share 0.436703, $w_a = 0.067014$	COMPUTED; share CHOSEN	Out [7]
M2, M3, M4 tangents	$w_a = 0.162074$, -0.393926, -0.600	PROVED (record); parameters CHOSEN	Out [8]
M5 fit over $1/2 \le a \le 1$	$(-0.861, -0.600)$	by construction	Out [8]
M5 crossing of -1	$a = 0.77909966367$ (B: 0.77905405)	COMPUTED; needs the ghost-like part	Out [8]

What the results mean. The equations of state that the record finds follow from two things: which quanta are present, and which definition of ρ_4 the observer uses. Neither is fixed by the field equations of the record. Under A and B the theory contains radiation-like content (the computed gas) and dark-matter-like content (the bulk band of dirac16complex, the good-sector gas of dirac16complex00), but the computed states of dirac16complex are only of the first kind. Under C the same content reads like dark energy: the gas between -0.71 and -0.67, a condensate at -1. The observed values are reached only by tuning as many parameters as numbers, and the crossing of -1 only with a component of negative energy. Neither hypothesis is established, and neither is refuted: what the record rules out, under its stated assumptions, is that the computed states of dirac16complex give the observed pair, and that any positive-energy content of dirac16complex00 with real frequencies crosses -1 (within the adiabatic model; for an extra-time mode only before its turning point, past which this is not established).

Why it is hard. The observer's ρ_4 needs a physical argument, not a choice: how a 4-dimensional observer measures energy when three time-like directions shrink. The populations need a dynamical origin: which quanta the early universe made, which is the time-dependent problem (Problem 2) and, beyond it, the creation question (Problem 1). The equation of state must come from the coupled equations, which brings back the back-reaction (Problem 3): no computed Kohn-Sham

state is an admissible source of the author's metric. And the ghost-like and growing modes of dirac16complex00 make its classical energy unbounded below and its initial-value problem ill-posed (Chapter 8).

What remains OPEN (the record's own list, `Revision/docs/DARK_SECTOR_HYPOTHESES.md`, section 10):

1. a self-consistent history $a_4(x_4)$ with an admissible source (no recorded Kohn-Sham state is one; the condensate is an exact source only on the linear member);
2. populated bulk-band or thermal states along the history; only $T = 0$ ground states and $0 \leq a_4 \leq 2$ are computed;
3. modes of dirac16complex with extra-time momentum (outside the good sector), and the non-adiabatic Kohn-Sham problem;
4. for dirac16complex00: sources that depend on x_0, the frozen warp of the adiabatic model, the growing modes, a quantum treatment;
5. which normalisation of ρ_4, if any, describes a physical 3-space observer, and the value of $a_{4,\mathrm{today}}$;
6. what would select the populations of M2 to M5, the mixture shares or the condensate's u;
7. a supernova likelihood (distances, covariances): no fit to data is part of the record.

A first step. (a) Redo the derivations of Sections 22.17 to 22.19 by hand, then change one assumption at a time in Notebook 22b: take today at $a_{4,\mathrm{today}} = 1$ instead of 2 in In [7], or remove the ghost-like part of M5 in In [8] and watch its crossing of -1 disappear (Exercise 14 shows why). (b) Write down a physical model of the observer: for example a 4-dimensional brane-world observer whose energy is the integral over the extra times with a weight derived from the metric, and find which s of Section 22.17 it gives. (c) Populate the bulk band thermally along the history (fixed entropy) with the Kohn-Sham solver of Chapter 15 and compute X/E; this is the record's open item 2.

Where to start. These files of the repository:

- the folder `Revision/dark_sector/dirac16complex` with its README, its four scripts in the subfolders `derive`, `compute` and `independent`, and its outputs and reports;
- the folder `Revision/dark_sector/dirac16complex00` with its README, its two implementations in the subfolder `python`, the model record `eos-theory.json`, and its results and reports;
- the document `Revision/docs/DARK_SECTOR_HYPOTHESES.md`, which states every result with its record;
- the builder of Notebook 22b, `Revision/textbook/notebooks/src/22b_dark_sector.py`.

22.25 Problem 6: matter and antimatter

The question. The universe we observe contains matter and almost no antimatter (Chapter 21 teaches the observations and the measured baryon-to-photon ratio from zero). The author asked that this theory solve the matter-antimatter mysteries. The open problem is whether an extension of the theory could produce the observed excess of matter from a start without excess, and with the measured size.

What is known.

- PROVED: the LOCAL conservation law $\partial_\mu(\cos z\, J^\mu) = 0$ of the U(1) current holds on every solution in the author's metric, for every history $a_4(x_4)$ (`Revision/lead_checks/reports/charge-conjugation-and-u1.json`, check `u1_noether_matrix_identity`). It says that charge is neither made nor destroyed at any point: the charge $Q = \int \cos z\, \Psi^\dagger B \Psi\, d^7x$ of a slice changes only by the flux of $\cos z\, J^\mu$ through the boundary of the slice. NOT proved: that the total charge of one universe is constant. That needs no flux through the brane $z = \pi/2$, a boundary condition that the record does not impose: wherever it is used it is ASSUMED, and whether it holds is OPEN, part of the junction conditions of Problem 4 (Sections 10.2, 18.21 and 21.18 show exact solutions whose charge changes by exactly this flux). Hence no net charge can be generated at any point inside one universe, and the total charge of one universe is constant if no charge flows through the brane.
- PROVED: the charge conjugations are matrices: $\mathcal{C}_+ = C$ (same mass) and $\mathcal{C}_- = \Gamma C$ (mass reversed) (checks `charge_conjugation_matrix_plus` and `charge_conjugation_matrix_minus` of the same report); for a real commuting field the current vanishes identically and $\mathcal{C}_+$ acts as the identity, so the nontrivial real map between matter and antimatter is the matrix Γ with the mass reversed, which is T1 (check `real_fields_charge_conjugation`); for the quantised Grassmann field the conjugation that keeps the canonical anticommutator is $\Psi \to \Gamma \Psi^{\dagger T}$, which reverses the mass (check `quantum_charge_conjugation_unitary_type`). Chapters 5 and 21 derive all of this.
- PROVED: a T1 partner carries the opposite charge, so a T1 pair $\{+m, -m\}$ has total charge zero as classical bilinears (T1d, Chapter 18); for two universes that are quantised independently of each other there is no such cancellation (Q). The idea that our universe has a partner of opposite charge, an anti-universe, belongs to a class of ideas of which one published example is L. Boyle, K. Finn and N. Turok, Phys. Rev. Lett. 121, 251301 (2018). That our universe has such a partner is a HYPOTHESIS.
- The theory as built does NOT solve the matter-antimatter problem: it has no baryons, no process that violates baryon number, no violation of CP, and no computation of a departure from thermal equilibrium.

What would be needed. Sakharov's three conditions (Chapter 21) in an extended theory: processes that violate baryon number; violation of C and of CP; and a departure from thermal equilibrium; and then a computation of the excess that agrees with the measured baryon-to-photon ratio. Every scenario that adds these is a HYPOTHESIS until it is computed and checked.

Why it is hard. The U(1) symmetry of the theory is exact, so inside the patch $0 < z < \pi/2$ no process of the Lagrangian of the record makes a net charge (the local law above); an asymmetry

therefore needs either new terms in the Lagrangian, and with every new term the pairing theorems must be derived again (they hold for the Lagrangian of the record, not for an arbitrary extension), or a flow of charge through the brane $z = \pi/2$, which the record neither excludes nor describes (OPEN; Sections 18.21 and 21.18). For the commuting field dirac16complex00 even such a flow could not produce an average asymmetry from a C-symmetric start unless the condition at the brane itself broke the symmetry $\mathcal{C}_+$ (Proposition 2 of Section 21.5): no such condition is derived. The field dirac16complex is not the field of the baryons of the Standard Model, so even a charge asymmetry of this field would still have to be connected to baryons. A departure from equilibrium needs the time-dependent dynamics of Problem 2, and a quantum computation of rates needs the Krein quantisation of Chapter 10 beyond the good sector.

A first step. (a) List the simplest terms that could be added to the Lagrangian and decide, with the exact matrix methods of the record (the solution of $M(\gamma^a)^* = \pm\gamma^a M$ in `Revision/lead_checks/charge_conjugation_and_u1.py` is the model), which of them break the U(1) and which of the discrete maps ($\mathcal{C}_+$, $\mathcal{C}_-$, Γ, the reflections) they respect. (b) For each such term, derive T1 again and state exactly how it changes. (c) Only then add a departure from equilibrium, with the time-dependent tools of Problem 2, and compute the charge produced from a symmetric start. Each of these steps is labelled HYPOTHESIS until it is done and checked.

Where to start. `Revision/lead_checks/charge_conjugation_and_u1.py` and its report; Chapters 5 and 21 and their notebooks; the pairing record `Revision/pairing/pairing-theory.json`.

22.26 Smaller open items

The table lists further open items that earlier chapters met, with their status and the record that states them.

item	status	record
modes with momentum along the extra times grow, at rates without bound	the growth PROVED; a mechanism that removes it, or a well-posed formulation, OPEN	`Revision/theory/reports/python-scope.json`, check `extra_time_growth_rates_unbounded`; Chapter 8
every quantum and Kohn-Sham result is restricted to the good sector (no extra-time momentum)	ASSUMED	`Revision/kohn_sham/ks-theory.json`, key `sectorAndAnsatz.sector`; Chapter 10
a positive-norm Fock space for a universe beyond the good sector	not established; OPEN	`Revision/pairing/reports/python-pairing.json`, key `not_established`
correlation in the Kohn-Sham functional	none (Hartree and exchange only); ASSUMED	`Revision/kohn_sham/ks-theory.json`, key `exchange.kohnShamPotentials.correlation`
which levels count as particles (the positive branch and the zero modes at $k = 0$)	CONVENTION; its justification OPEN	`Revision/kohn_sham/ks-theory.json`, key `thermodynamics.fillingConvention`

item	status	record
the tip at $y = -3$ with the tip angle $\theta = 0$	CHOSEN	`Revision/kohn_sham/ks-theory.json`, key `boundaryConditions.tip`
the meaning of jumps into the negative branch	OPEN	Notebook 22a, In [8]; Section 22.13
relaxation of the gas through Fermi-level crossings	OPEN	`Revision/kohn_sham/results/adiabatic/crossing-demo.csv`; Section 22.13
a constant total U(1) charge: no flux of the current through the brane $z = \pi/2$	ASSUMED where used; OPEN	`Revision/lead_checks/reports/charge-conjugation-and-u1.json` (the local law only); Sections 10.2, 18.21, 21.18 and 22.25

22.27 What we proved, what we computed, what we assumed

PROVED (exact; each derivation is written out line by line in this chapter; the record checks are named where the Revision record verifies the statement):

statement	where it is verified
a T1 pair as the complete source leaves no real history a_4 in Einstein gravity for $H > 0$ (corollary C1): $(a_4')^2 = -H^2$	`wolfram-a4-report.json`, `einstein_no_vacuum_solution`; `python-a4-report.json`, `einstein_no_vacuum`; lead's report `einstein-gauss-bonnet-a4.json`, `no_vacuum_for_H_positive`; Section 22.4
the Ricci scalar of the author's metric is $R = 6(a_4')^2 - 42H^2$	derived in Section 22.4 from the record's Einstein components; the same R in `python-field-theory.json` (detail of `curvature_nonzero_flat_only_formally`; Wolfram comparison `ricci_scalar`: agree)
the total probability $\langle\chi\|\chi\rangle$ is conserved	`ks-theory-python.json` and `ks-theory-wolfram.json`, `hamiltonian_16_hermitian`; Section 22.6
$K_{mn} = \langle m\|\partial_a h\|n\rangle/(\varepsilon_n - \varepsilon_m)$ for $m \neq n$	`ks-theory-python.json`, `adiabatic_offdiagonal_identity`; Section 22.6, line 6
$\partial_a h = -j\kappa k\sigma_3$ in the free field	`ks-theory-python.json`, `adiabatic_hellmann_feynman`; Section 22.6, line 8
the amplitude equations (lines 1 to 5) and $K_{nn} = 0$ (line 7)	derivations of this book, Section 22.6; no Revision check covers them
the dressing amplitude $\|\alpha_m\| = Q_{0m}$; $Q(A) = A\,Q(1)$	derivations of this book, Section 22.7 ($Q(A) = A\,Q(1)$ read off the record's formula, `ks-theory.json`, key `adiabaticity.measure`); the dressing tested numerically in Notebook 22a, In [14]

statement	where it is verified
in the free field $Q/A = G(q)$ with $q = ke^{-a_{4,0}}$	ks-theory-python.json, rescaling_identity; Section 22.7
the sudden limit $P_{sudden} = 1 - \|\langle\varphi_0(s_2)\|\varphi_0(s_1)\rangle\|^2$, the rate of the smooth passage, the first-order formula for $P^{(1)}$	derivations of this book, Section 22.8; tested numerically in Notebook 22a, In [12] and In [14]
in Einstein gravity every source has $\kappa(\rho + p_0) = -6((a_4')^2 + H^2)$	both a4 reports, einstein_null_energy_x0; lead's report, einstein_null_energy; Section 22.14
the line element in the coordinate y is regular at $y = 0$; the mirror warp $e^{-H\|y\|}$ has a kink, and $\int_{-\epsilon}^{\epsilon} W''\,dy \to -2H$	derived in Section 22.15 from the record's statements (ks-theory-python.json, geometry_warped_form)
$w(1) = w_0$, $w(0) = w_0 + w_a$; the CPL line crosses -1 at $a = 461/600$; $w < -1$ with $\rho > 0$ if and only if $\rho + p < 0$	algebra, Section 22.16; python-derive-eos.json, unite_crossing_point
the proper 7-volume element is $\cos z$, independent of a_4; $dE/da_4 = -3X$	derivation-checks.json, proper_7_volume_element_independent_of_a4, integrated_identity_dE_da4; Section 22.17
$w_{\text{eff}}(A) = w_{\text{eff}}(B) = X/E$ and $w_{\text{eff}}(C) = X/E - 1$ (in general $X/E - s$), for each stated definition of ρ_4	derivation-checks.json, w_eff_A_equals_X_over_E, w_eff_B_equals_w_eff_A, w_eff_C_equals_X_over_E_minus_1, w_eff_general_normaliser; Section 22.17
on the linear member $w_{\text{exp}} = -1$; $w_{\text{eff}}(C) < -1$ if and only if $X/E < 0$	derivation-checks.json, expansion_inferred_w, phantom_condition; Section 22.17
the flat-limit law $w_{\text{eff}} = k^2/(3(M^2a^2 + k^2))$ of a massive quantum	derivation-checks.json, massive_mode_w_eff_law; Section 22.18
the condensate: $X = 0$, $w_{\text{eff}} = 0$ (A, B) and -1 (C); ratio $u/(2 + u)$, equal to -0.764 at $u = -382/441$, phantom exactly for $-2 < u < -1$	derivation-checks.json, condensate_w_eff, condensate_ratio_equal_unite_constant_w; python-derive-eos.json, condensate_phantom_interval; Section 22.18
radiation plus condensate: $w_a = r_0/(3(1 + r_0)^2) > 0$ (freezing)	derivation-checks.json, mixture_radiation_condensate_cpl, mixture_C_matching_w0_unite; Section 22.18
dirac16complex00: modes of both energy signs at every real frequency; in the adiabatic model $\varepsilon_i \geq 0$, and with positive energies of real frequency $w_{\text{eff}}(C) \geq -1$ (for an extra-time mode only before its turning point); the tangents of M2, M3, M4	wolfram-pairing.json, Q_one_particle_Krein_inertia_real_frequencies, and python-pairing.json, Q.one_particle_Krein_inertia_proof (Theorem 10.1, the general statement); python-derive-eos.json, mode_krein_inertia_* (the three test cases), wkb_epsilon_sign_free, M2_tangent_exact, M3_tangent_exact, M4_tangent_equals_unite; Section 22.19
the LOCAL U(1) law $\partial_\mu(\cos z\, J^\mu) = 0$ (a constant total charge needs no flux through the brane: ASSUMED where used, OPEN), the charge-conjugation matrices, T1, T2, Q, T3	Chapters 5, 10 and 18 to 21; charge-conjugation-and-u1.json, pairing and T3 reports; Section 22.25

COMPUTED (numerical, with measured accuracy): from the Revision record, $Q_{max} \leq 0.0935$ for the 75 ground states at $A = 1$, the adiabatically continued state of $N = 696$ above the instantaneous ground state by up to 8.623, the violations of the source conditions (ratios 2.09 to 3.99, closest integrated ratio 0.414), and the agreement of dE/da_4 from finite differences and from the energy-momentum integral (for N688_lam0_a00, -597.9157012757526 against -597.9157012806555). In Notebook 22a (free field, the Fermi-shell sector of $N = 688$): the record's levels and Q reproduced to 10^{-13}; $G^* = 0.09979$ at $q^* = 2.128$; the strongest jump across the gap 0.0489 per unit rate; $P = 0.0138$ at the peak rate 1; $A_{1/2} = 1.508$; the sudden limit 0.0749 (span 0 to 2) and 0.1051 (0 to 6); the breakdown estimates 10.701, 10.021, 2.929 and 1.508; with the errors of Section 22.13. From the dark-sector record (reproduced in Notebook 22b): the Kohn-Sham gas along the prescribed history, X/E from 0.292893 to 0.328105 rising toward 1/3, $w_{\text{eff}}(C)$ from -0.707107 to -0.671895, tangent w_a from -0.020523 to -0.008894, the energy balance on the data to 1.235×10^{-7}; the bulk-band level from 0.239626 to 2.268×10^{-3}; the computed mixture with $w_a = 0.067014$; the M5 fit, its crossing at $a = 0.7791$ and the field-equation implementation B (crossing 0.77905405, agreement with A within 0.00083082726).

ASSUMED: the Z2 brane and the parities it gives; the good sector; the tip (CHOSEN); which levels count as particles (CONVENTION); the history $a_4 = AHx_4$ (PRESCRIBED BACKGROUND); in Notebook 22a the free field ($\lambda = 0$) and one sector; the observer's density ρ_4 (definitions A, B, C) and the value of $a_{4,\text{today}}$; the adiabatic (WKB) model of dirac16complex00, an APPROXIMATION measured by implementation B; every parameter labelled CHOSEN in Sections 22.18 and 22.19 (several of them chosen to reproduce the observed values, so that those matches are NOT predictions); the no-flux condition at the brane wherever a constant total U(1) charge is used; the expansion in instantaneous orbitals, truncated to 20 levels with its convergence measured; the first-order approximations of Sections 22.7 and 22.8, whose range of validity the notebook measures.

HYPOTHESIS: the author's dark-sector hypotheses (investigated by the record, established in neither form; the ghost-like component of M5 is not an established physical state); that our universe has a partner of opposite charge; every scenario of creation or of matter-antimatter asymmetry; the generalised metric of Section 22.14 and the reduced quantum model of Section 22.4 as useful next steps.

OPEN: whether any universe is created, in pairs or otherwise (no creation process, rate, amplitude or big-bang dynamics follows from the equations); the time-dependent Kohn-Sham problem with interaction; the reading of jumps into the negative branch; relaxation through Fermi-level crossings; the back-reaction of the gas on a_4; the junction conditions of the brane; which definition of ρ_4 describes a physical 3-space observer; populated bulk-band or thermal states; what selects the populations of the dark-sector models; a fit to supernova data; whether the total U(1) charge of one universe is constant (no flux through the brane); an asymmetry between matter and antimatter in an extended theory; the extra-time modes beyond the good sector.

22.28 Exercises

Exercise 1. The record's largest Q of the free state N136_lam0_a00 is 0.0851488 at $A = 1$ (Out [5]). (a) What is its naive breakdown rate? (b) What is Q at $A = 5$, and the first-order dressing probability Q^2 there? (c) Can the first-order estimate be trusted at $A = 5$?

Answer. (a) By exact rule 1, $Q(A) = 0.0851488\,A$ reaches 1 at $A = 1/0.0851488 = 11.744$. (b) $Q(5) = 5 \cdot 0.0851488 = 0.425744$ and $Q^2 = 0.181258$. (c) No: first order needs $Q \ll 1$, and 0.43 is

not small. Notebook 22a shows for the free Fermi shell of $N = 688$ what happens at such rates: the exact probability levels off at the sudden limit of about 0.075 over the record's span, while the first-order estimate overshoots (Figure 22a.5). A first-order number of 0.18 is therefore not to be trusted.

Exercise 2. From the row of Out [5] for $N = 688$ at the slice 0 ($\varepsilon_0 = 1.247113$, $\varepsilon_1 = 3.500518$, $Q = 0.0934506$) compute the energy difference, the matrix element $|\langle 0|\partial_a h|1\rangle|$ and $|K_{10}|$. Compare with the record's column `Q_max_matrix_element` (0.4745266881) and with the right panel of Figure 22a.4.

Answer. $\varepsilon_1 - \varepsilon_0 = 2.253405$. From $Q = |\langle 0|\partial_a h|1\rangle|/(\varepsilon_1 - \varepsilon_0)^2$ (Section 22.7, with $A = H = 1$): $|\langle 0|\partial_a h|1\rangle| = Q(\varepsilon_1 - \varepsilon_0)^2 = 0.0934506 \cdot 5.077834 = 0.474527$, which agrees with the record to the seven digits of the inputs. By line 6 of Section 22.6, $|K_{10}| = |\langle 1|\partial_a h|0\rangle|/|\varepsilon_0 - \varepsilon_1| = Q(\varepsilon_1 - \varepsilon_0) = 0.0934506 \cdot 2.253405 = 0.210582$, the starting value of the blue curve on the right of Figure 22a.4.

Exercise 3. Show that the cubic polynomial $p(t)$ with $p(0) = f_0$, $p(h) = f_1$, $p'(0) = f'_0$, $p'(h) = f'_1$ has the value $p(h/2) = (f_0 + f_1)/2 + h(f'_0 - f'_1)/8$ in the middle (the rule of the function `orbitals` in In [4]). Test it on $f(t) = t^3$ with $h = 1$.

Answer. Write $p(t) = f_0 + f'_0 t + ct^2 + dt^3$; this already gives $p(0) = f_0$ and $p'(0) = f'_0$. Put $\Delta = f_1 - f_0 - f'_0 h$ and $\delta = f'_1 - f'_0$. The two conditions at $t = h$ read $ch^2 + dh^3 = \Delta$ and $2ch + 3dh^2 = \delta$. Multiply the first by $3/h$ and subtract the second: $ch = 3\Delta/h - \delta$, so $ch^2 = 3\Delta - \delta h$, and from the first condition $dh^3 = \Delta - ch^2$. Then $p(h/2) = f_0 + f'_0 h/2 + ch^2/4 + dh^3/8 = f_0 + f'_0 h/2 + ch^2/4 + (\Delta - ch^2)/8 = f_0 + f'_0 h/2 + ch^2/8 + \Delta/8$. Insert $ch^2 = 3\Delta - \delta h$: $ch^2/8 + \Delta/8 = \Delta/2 - \delta h/8$. Hence $p(h/2) = f_0 + f'_0 h/2 + (f_1 - f_0 - f'_0 h)/2 - (f'_1 - f'_0)h/8 = (f_0 + f_1)/2 + h(f'_0 - f'_1)/8$. Test: $f_0 = 0$, $f_1 = 1$, $f'_0 = 0$, $f'_1 = 3$: $(0 + 1)/2 + (0 - 3)/8 = 0.5 - 0.375 = 0.125 = (1/2)^3$. Exact, as it must be, because t^3 is itself a cubic.

Exercise 4. Compute the redshifted momenta of the Fermi shell of $N = 136$ at the slice 1.5 and of the Fermi shell of $N = 688$ at the slice 2. Why are their Q in Out [5] (0.0527487 and 0.0528933) nearly equal, and why is the second slightly larger?

Answer. $q = 0.5\, e^{-1.5} = 0.5 \cdot 0.223130 = 0.111565$ and $q = 0.829156\, e^{-2} = 0.829156 \cdot 0.135335 = 0.112214$ (Out [5] prints 0.1116 and 0.1122). By exact rule 2 the free Q/A depends only on q, and the two values of q differ by only 0.6 percent, so the two Q are nearly equal although the shells and the slices are different. In this range $G(q)$ rises with q (the whole recorded range lies left of the top in Figure 22a.2), so the larger q has the slightly larger Q. The band levels behave the same way: 0.204636 and 0.205760.

Exercise 5. The sudden limit of the span $a_{4,0} = 0$ to 2 is $P_{sudden} = 0.074906$ (Out [12]). Compute the overlap $|\langle\varphi_0(2)|\varphi_0(0)\rangle|$ and the angle θ with $\cos\theta$ equal to it. Show that for a particle whose orbital is turned by the angle θ in the plane of two orthonormal orbitals, $P_{sudden} = \sin^2\theta$.

Answer. $|\langle\varphi_0(2)|\varphi_0(0)\rangle| = \sqrt{1 - 0.074906} = \sqrt{0.925094} = 0.961818$, and $\theta = \arccos 0.961818 = 0.27723$ radian, about 15.9 degrees. If the new band orbital is $\varphi_0(2) = \cos\theta\, \varphi_0(0) + \sin\theta\, \varphi_1$ with φ_1 orthonormal to $\varphi_0(0)$, the overlap is $\cos\theta$ and $P_{sudden} = 1 - \cos^2\theta = \sin^2\theta$; here $\sin^2(0.27723) = 0.074906$. The band orbital turns by only about 16 degrees over two e-folds of the extra times.

Exercise 6. Show that $a'_4 = iH$ with $\Lambda = -18H^2$ solves both vacuum equations of Einstein gravity of Section 22.4. Why is it not a history of the author's universe?

Answer. $(iH)^2 = -H^2$. The x_4 equation: $3(-H^2) + 21H^2 - 18H^2 = 0$. The x_0 equation:

$-3(-H^2) + 15H^2 - 18H^2 = 3H^2 + 15H^2 - 18H^2 = 0$. Both hold. But $a_4 = iHx_4 + a_0$ is complex, so the metric components $e^{2a_4}\sin^{1/3} z$ would be complex numbers, and a metric must be real. So there is no real history: this is corollary C1 again.

Exercise 7. The source-condition report gives for N136_lam0_a00 the integrals $\int \rho = 80.2822$, $\int p_3 = 23.8133$, $\int p_t = 0$ (each $2\mathrm{Vol}_7 \int e^{6Hy}(\dots)\,dy$). Use the energy-exchange identity of Section 22.14 to compute dE/da_4, and compare with the record's columns `dE_da4_emt` (-71.43975618) and `dE_da4_finite_difference` (-71.43975618). Do the same for N688_lam0_a00 ($\int p_3 = 199.305$, $\int p_t = 0$). What does the sign mean?

Answer. $dE/da_4 = -3(\int p_3 - \int p_t) = -3 \cdot 23.8133 = -71.4399$, in agreement with both columns to the six digits that the source-condition report prints. For $N = 688$: $-3 \cdot 199.305 = -597.915$, against -597.9157 in the record. The sign: as a_4 grows (the extra times deflate and 3-space inflates), the energy of the gas falls, because its 3-momenta redshift; $p_t = 0$ here because the gas has no kinetic pressure along the extra times. In a coupled solution this energy would have to go into the geometry, which is why the history cannot be the linear member with a constant ρ (Section 22.14).

Exercise 8. For the mirror warp $W(y) = e^{-H|y|}$ with $H = 1$: compute $W(0)$, the slopes $W'(0^-)$ and $W'(0^+)$, and $\int_{-0.01}^{0.01} W''\,dy$. Then show that $\sqrt{|g|} = e^{6Hy}$ in the coordinate y becomes $\cos z$ in the coordinate x_0.

Answer. $W(0) = 1$; $W'(0^-) = 1$, $W'(0^+) = -1$. $\int_{-0.01}^{0.01} W''\,dy = W'(0.01) - W'(-0.01) = -e^{-0.01} - e^{-0.01} = -2e^{-0.01} = -1.980100$, close to $-2H = -2$, and closer still for a shorter interval. For the volume: $\sqrt{|g|}\,dy$ must equal $\sqrt{|g|_{x_0}}\,dx_0$, and $dy = \cot z\,dx_0$, so $\sqrt{|g|_{x_0}} = e^{6Hy}\cot z = \sin z \cot z = \cos z$, using $\sin z = e^{6Hy}$.

Exercise 9. For the CPL values $(w_0, w_a) = (-0.861, -0.60)$ compute w at $a = 1$, $a = 1/2$ and $a \to 0$, and the scale factor at which $w = -1$. Is this thawing or freezing in the convention of Section 22.16? What does $w < -1$ require of a fluid with $\rho > 0$?

Answer. $w(1) = -0.861$; $w(1/2) = -0.861 - 0.60 \cdot 0.5 = -1.161$; $w(0) = -0.861 - 0.60 = -1.461$. $w = -1$ when $0.60(1 - a) = 0.139$, that is $1 - a = 0.231667$ and $a = 0.768$. Since $w_a < 0$ it is thawing in this convention (w rises towards today from below). For $\rho > 0$, $w < -1$ means $\rho + p < 0$ (Section 22.16): the fluid violates the null energy condition. These are statements about the quoted fit; this book claims nothing about whether either field can produce them.

Exercise 10. From Out [17] check the first-order estimate 2.929 of the breakdown rate, and compute how much larger the naive estimate is than the exact $A_{1/2}$ and than the first-order estimate.

Answer. The first-order estimate is the rate at which $(AQ_0)^2 = P_{sudden}$ with $Q_0 = 0.0934506$ and $P_{sudden} = 0.074906$: $A = \sqrt{0.074906}/0.0934506 = 0.273690/0.0934506 = 2.929$. The naive estimate 10.701 is $10.701/1.508 = 7.10$ times the exact $A_{1/2}$ and $10.701/2.929 = 3.65$ times the first-order estimate. Comparing the dressing probability with the largest probability that can leave the level at all is what brings the estimate within a factor of two of the exact evolution.

Exercise 11. (a) The gas N688_lam0 has $X/E = 0.3183$ at $a_4 = 2$ (Out [3]). Give its w_{eff} under the definitions A, B and C, and under the general normalisation of Section 22.17 with $s = 1/2$. (b) Show from $dE/da_4 = -3X$ that a state with $X = E/3$ at every time has $E \propto e^{-a_4}$, and that under A its density thins out like a^{-4}. (c) What does a condensate give under C, and why?

Answer. (a) Under A and B, $w_{\text{eff}} = X/E = 0.3183$; under C, $0.3183 - 1 = -0.6817$; with $s = 1/2$, $0.3183 - 0.5 = -0.1817$. The same state gives three different readings: only the ASSUMED

normalisation changed. (b) With $X = E/3$ the balance reads $dE/da_4 = -E$; the function whose derivative is minus itself is $E = E_0 e^{-a_4}$ (check: $d(E_0 e^{-a_4})/da_4 = -E_0 e^{-a_4}$). Under A, $\rho_4 \propto E\, e^{-3a_4} = E_0 e^{-4a_4}$, and since $a = e^{a_4 - a_{4,\text{today}}}$, $e^{-4a_4} \propto a^{-4}$: radiation, $w = 1/3$. (c) A condensate has $X = 0$, so $dE/da_4 = 0$ and E is constant; under C, $\rho_4 \propto E$ is constant, and $w_{\text{eff}} = -1 - \frac{1}{3} \cdot 0 = -1$: it reads like a cosmological constant (under A and B, $\rho_4 \propto e^{-3a_4} \propto a^{-3}$, like dust).

Exercise 12. For a massive quantum in the flat limit with $M = 1$ and $k = 1$, compute $w_{\text{eff}}(A) = k^2/(3(M^2a^2 + k^2))$ at $a = 0.1$, 1 and 10, the same values under C, and the CPL tangent at $a = 1$ under C. Is it thawing or freezing?

Answer. $w_{\text{eff}}(A) = 1/(3(a^2 + 1))$: at $a = 0.1$, $1/(3 \cdot 1.01) = 0.330033$; at $a = 1$, $1/6 = 0.166667$; at $a = 10$, $1/(3 \cdot 101) = 0.003300$. It falls from radiation toward dust: the dark-matter-like law. Under C: -0.669967, -0.833333, -0.996700. The derivative is $dw/da = -2a/(3(a^2 + 1)^2)$, at $a = 1$ equal to $-2/12 = -1/6$, so $w_0 = -0.833333$ under C and $w_a = -dw/da = +0.166667$ in every definition: freezing (it moves toward -1 as a grows). This is why the bulk band, even if it were populated, would not give the observed thawing slope under C.

Exercise 13. The record mixes the computed gas ($X/E = 0.3182941676741$ at $a_4 = 2$) with a condensate so that $w_{\text{eff}}(C) = -0.861$ today. (a) Compute the gas share of the energy today. (b) For an exactly radiation-like gas ($X/E = 1/3$) the same condition gives $r_0 = 417/583$; what is the gas share then? (c) Why does the computed gas need the larger share?

Answer. (a) The condensate adds energy but no X, so $w_{\text{eff}}(C) = \text{share} \cdot X/E - 1$; setting this to -0.861 gives share $= 0.139/0.3182941676741 = 0.436703$ (Out [7]). (b) With $X/E = 1/3$: share $= 3 \cdot 0.139 = 0.417$, and indeed $r_0/(1 + r_0) = (417/583)/(1000/583) = 417/1000$. (c) The computed gas has $X/E = 0.3183$, a little below $1/3$, so it contributes less X per unit energy, and a larger share of it is needed to reach the same 0.139. In both cases the share is CHOSEN, and the slope comes out positive (0.067014 and 0.081037): freezing, not the observed -0.60.

Exercise 14. Two components of an adiabatic gas of dirac16complex00 have the energy densities $\rho_1 = 0.7$, $\rho_2 = 0.3$ and the values $\varepsilon_1 = 0$ (a condensate), $\varepsilon_2 = 0.5$ (an extra-time mode). (a) Compute $w_{\text{eff}}(C) = \sum_i \varepsilon_i \rho_i / \sum_i \rho_i - 1$. (b) Replace them by $\rho_1 = 1.3$, $\varepsilon_1 = 0$ and a ghost-like component $\rho_2 = -0.3$, $\varepsilon_2 = 1/3$ (the total is again 1); compute $w_{\text{eff}}(C)$. (c) Explain with these numbers why a crossing of -1 needs a component of negative energy.

Answer. (a) $(0 \cdot 0.7 + 0.5 \cdot 0.3)/(0.7 + 0.3) - 1 = 0.15 - 1 = -0.85$. (b) $(0 \cdot 1.3 + \frac{1}{3} \cdot (-0.3))/(1.3 - 0.3) - 1 = -0.1 - 1 = -1.1$: below -1, phantom. (c) With positive weights ρ_i the fraction is an average of the numbers $\varepsilon_i \geq 0$, so it lies between their smallest and largest value and cannot be negative: $w_{\text{eff}}(C) \geq -1$. A negative weight breaks this rule: the "average" -0.1 lies outside the range of the ε_i (0 to 1/3). This is the theorem of Section 22.19 (for components of real frequency), and it is why M5 crosses -1 only because of its ghost-like part, and why the record's M5 without that part does not cross on $1/3 \leq a \leq 1$ (Out [8]). The size of the ghost share is a choice: the record notes that $G = 0.28$ would need $s = 0.726$ and $G = 0.24$ would need $s = 0.921$ (an exploration of the record, not a check), so nothing fixes it.

23 Reproducing everything

23.1 What this chapter does

Every result of this book comes from a file that a program wrote, and every one of those programs can be run again. This chapter gives every command that reproduces the book and the Revision record behind it, in the order in which they depend on each other, for Windows 11 (PowerShell), macOS (zsh) and Linux (bash). For each command it says what the command runs, what it prints when it works, how long it took on the development machine (Windows 11, 24 processor threads), and what to do when it fails. Every time quoted here is a MEASURED time read from a provenance file, from the step table of the Revision gate, or from a run made for this chapter on 2026-10-08, and it is labelled as measured or expected; on another computer the times differ, the results do not.

The chapter's example, Notebook 23a, is the bookkeeping of the whole book. By default it READS the recorded checks of the other 90 notebooks from their provenance files; it does NOT re-run them. It prints a table of every notebook, draws the run times, reads the step table of the Revision gate, and builds the index of every check of every report of the Revision record together with the chapters that cite it. With the environment variable `BOOK_RERUN_ALL` set to `1` it also re-runs every other notebook (Section 23.9 gives its measured time and the commands). The chapter ends with the two indexes of the book (Sections 23.13 and 23.14) and with the glossary.

23.2 The words of this chapter

- **Command**: one line typed into a terminal and run by pressing Enter. A **terminal** is the window in which commands are typed: on Windows the app Windows PowerShell or Terminal, on macOS the app Terminal (its shell is zsh), on Linux any terminal (its shell is bash).
- **Repository**: the folder Dirac_claude with every file of this project, downloaded with the program Git. The **repository root** is that folder itself; every command of this chapter is typed there.
- **Verifier** and **checker**: a program of the Revision record that recomputes a result (exactly, with Wolfram or with sympy, or numerically, with Rust or Python) and writes a **report**: a JSON file that lists named **checks**, each with a **verdict** such as PASS.
- **Gate**: the script `Revision/verify_revision.sh` (for bash) and its twin `Revision/verify_revision.ps1` (for PowerShell 7). It re-runs every verifier and checker in **dependency order** (a program that reads the output of another runs after it), then demands that every committed output is unchanged byte for byte and that every report passes.
- **Step**: one command of the gate, with a name such as `algebra-wolfram`. A **long step** is one documented as longer than 5 minutes, or one that needs such a step (`ks-rust-determinism`, 30 s, needs the refined run).
- **Builder**: the Python file `Revision/textbook/notebooks/src/NNx_name.py` from which the tool `nbkit` builds Notebook NNx.
- **nbkit check**: re-executing a notebook into a scratch folder and comparing the result byte for byte with the stored notebook, its files and its provenance file. A **scratch folder** is a folder for throw-away files.

- **Provenance file**: `NNx_name.PROVENANCE.md`, next to each notebook. Its last line records the measured facts of the notebook's last verified run, among them the date, result and seconds of its last nbkit check.
- **Assembly**: joining the 24 chapter files, with every notebook expanded, into the one file `Revision/textbook/UNIVERSES_IN_PAIRS_TEXTBOOK.md`.
- **Wall time**: the time a clock on the wall shows between the start and the end of a command.
- **Environment variable**: a named piece of text that a terminal hands to every program it starts.

23.3 What you need, and the set-up in every new terminal

The notebooks need Git and Python 3.12 or newer with the nine pinned packages; the run-instruction section before every notebook of this book gives the complete installation for the three systems, and the installation commands are repeated here so that this chapter can be followed on its own. Install Git and Python as described there, then download the repository, create the private environment `dirac-book-env` and install the packages:

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

macOS (zsh) and Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

In every new terminal, activate the environment and go to the repository root before any command of this chapter:

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

The Revision gate needs four more programs, which the notebooks do not need.

- **Rust** (the command `cargo`), for the Rust programs of the GKD and Kohn-Sham records and for the notebooks that run them: install it from https://rustup.rs and open a new terminal.
- **WolframScript** (the command `wolframscript`), for the exact Wolfram verifiers. It must be installed AND activated by you, with your own Wolfram licence; the gate never activates it.
- **pdflatex**, for the six documents of the Revision record and for the book's PDF: on Windows MiKTeX (https://miktex.org), on macOS MacTeX (https://tug.org/mactex), on Linux the TeX Live packages of your distribution.
- On Windows, **PowerShell 7** (the command `pwsh`), from https://aka.ms/powershell, for the PowerShell twin of the gate; Git for Windows also brings bash, so the bash form of the gate works there too.

23.4 The order of everything

The five stages below must be run in this order, because each reads what the stages before it wrote and checked.

1. The Revision record: the gate re-runs every verifier and checker (Section 23.5).
2. The notebooks of the book: each nbkit check re-executes one notebook (Section 23.6). The notebooks read the committed Revision record, so they come after it.
3. Notebook 23a: it reads the provenance files of all other notebooks and the reports of the Revision record, so it comes after both (Sections 23.9 to 23.12).
4. The assembly of the book from its chapters and notebooks (Section 23.7).
5. The PDF of the book, built from the assembled file (Section 23.7).

Stages 1 and 2 only VERIFY: they change no committed file when everything agrees. Stages 4 and 5 rewrite the book's files; run them only when you have changed a chapter or a notebook.

23.5 Stage 1: the Revision gate, step by step

First look at what the gate would do, without running anything. The option dry-run prints every selected step with its expected wall time and its command, and the last line `revision_verific ation=NOT-RUN`:

Windows (PowerShell 7):

```
pwsh -NoProfile -File Revision/verify_revision.ps1 --dry-run
```

macOS (zsh), Linux (bash), and Git Bash on Windows:

```
bash Revision/verify_revision.sh --dry-run
```

The full gate is the same command without any option; the fast option skips the long steps; the option steps runs only the named steps (the two audits `committed-unchanged` and `reports-pass` always run):

```
bash Revision/verify_revision.sh
bash Revision/verify_revision.sh --fast
bash Revision/verify_revision.sh --steps algebra-wolfram,algebra-sympy
pwsh -NoProfile -File Revision/verify_revision.ps1
pwsh -NoProfile -File Revision/verify_revision.ps1 --fast
pwsh -NoProfile -File Revision/verify_revision.ps1 --steps algebra-wolfram,algebra-sympy
```

What it runs. Notebook 23a writes the gate's complete step table to the data file `Revision/textbook/data/23a_gate_steps.csv`, one line per step with its expected wall time and the program it runs, prints its long steps and audits in Out [9] of Section 23.11, and draws the whole table as Figure 23a.6. In order, the 85 steps are these groups.

- **The algebra** (2 steps, expected 10 s and 5 s): the author's gamma matrices, C, Gamma and B, exactly with Wolfram (`verify_algebra.wls`) and with sympy (`check_algebra.py`); Chapters 4 and 5.
- **GKD and the Lovelock tensors** (10 steps): the Rust program `lovelock_gkd` is built (60 s), computes the curvature and the three Lovelock tensors of the metric with b_4 (15 s) and runs its exhaustive self-test (900 s, long, the longest step); Wolfram (110 s) and sympy (30 s) verify the result; four steps read the author's notebook and one compares it with the record (10 s, 5 s, 10 s, 120 s, 60 s); Chapter 11.
- **The metric with b_4** (5 steps): the author's notebook `Dirac_Einstein-Lovelock_4+4_Nash.nb` must be the pinned version (5 s); its stored outputs for the metric with b_4 are extracted (10 s); Wolfram (90 s) and sympy (40 s) recompute the curvature and the Lovelock tensors with b_4 and compare them with those outputs; and the vacuum theorem for the author's weights is proved (10 s); Chapters 3 and 12.
- **The field theory** (10 steps): the Lagrangians, field equations, energy-momentum tensor and quantisation with Wolfram (310 s, long) and sympy (240 s), the scope checks with Wolfram (60 s) and sympy (5 s), a comparison of the two (2 s), and the energy-momentum tensor operator for $\lambda \neq 0$ in a finite Fock model (10 s); then the field theory in the metric with b_4 with Wolfram (15 s), its energy-momentum tensor for the commuting and the Grassmann field (190 s and 110 s) and sympy (5 s); Chapters 6 to 10.
- **The pairing theorems** (2 steps): Wolfram (280 s) and sympy (240 s); Chapter 18.
- **The a4 equations** (6 steps): Wolfram (70 s) and sympy (30 s), the block of the equations with b_4 (35 s), the Kohn-Sham source conditions (5 s), the a4 equations with the Kohn-Sham source (20 s) and its unit tests (30 s); Chapters 12 and 17.

- **The Kohn-Sham record** (24 steps): the theory with Wolfram and sympy (60 s each), its statements for the metric with b_4 (5 s) and the consequences of b_4 (5 s); the Rust solver built and tested (60 s each); the canonical matrix (250 s), the Mermin roots (30 s), a repeat run (250 s) and its byte comparison (5 s), a refined run (330 s, long) and the determinism report (30 s, long because it needs the refined run); the Python reference solver in four parts (240 s, 235 s, 265 s, 270 s) and the step that joins them (5 s), the refinement measurement (90 s), the repeat run of the reference solver for the cross-check, again in four parts (240 s, 235 s, 265 s, 270 s), the cross-check (25 s) and the study of the tip cutoff L (200 s); Chapters 14 to 16.
- **T3 and its completion** (6 steps): Wolfram (15 s, 20 s) and sympy (10 s, 10 s), and the two numerical demonstrations with the Rust solver (285 s) and the reference solver (170 s); Chapter 19.
- **The dark sector** (7 steps): four for dirac16complex (10 s, 150 s, 5 s, 180 s) and three for dirac16complex00 (30 s each); Chapter 22.
- **The lead's checks** (3 steps): the energy-momentum divergence (10 s), Einstein-Gauss-Bonnet (20 s) and the charge-conjugation matrices with the local U(1) law (15 s); Chapters 9, 12 and 21.
- **The Revision notebooks** (1 step, 90 s): every notebook of `Revision/notebooks` is audited and checked.
- **The six documents** (6 steps, 30 s each): each PDF is rebuilt in verify mode.
- **The audits** (3 steps): `committed-unchanged` (10 s) demands that no committed output changed; `reports-pass` (5 s) reads every report of the step table and demands that every check passes; `unit-tests` (150 s) runs the Revision unit tests, except the textbook test, which has its own build.

How long it takes. Out [9] adds up the step table: the expected wall time of the full gate is 8057 s, that is 2.24 hours, of which the 4 long steps take the difference to the time of the fast option, which skips them (Figure 23a.6); with the fast option the expected times add up to 6487 s (108.1 minutes). The README of the Revision record states the MEASURED time of the fast option on 2026-10-08, when it had 55 steps, as about 20 minutes (most steps ran faster than the expected times of the table) and expects about 3.1 hours for the full gate.

What it prints when it works. First the tools it found (`revision_tool_python=...` and, when needed, cargo, wolframscript, git and pdflatex), then `revision_selected_steps=85 expected_seconds=8057 skipped_steps=0` (with the fast option: 81 steps, 6487 s, 4 skipped, each skipped step named on a line `revision_skipped_step=...`), then the precheck line `revision_output_paths=... preexisting_changes=0`. For every step it prints `revision_step=NAME expected_seconds=N`, the step's own output, `revision_step_seconds=NAME n (expected N)` and `revision_step_ok=NAME`. The step `reports-pass` prints one line `revision_report=PATH checks=k/k passed` per report. The last two lines are `revision_gate_seconds=...` and

```
revision_verification=OK
```

Every step's complete output is also written to a log file `build/logs/revision/STEP-bash.log` (or `STEP-pwsh.log`); the folder `build/` is ignored by Git.

When it fails. The gate stops at the first failing step and prints `revision_failed_step=NAME`, `revision_failed_log=PATH` and `revision_verification=FAILED`. Open the log file it names and read its last lines. The most likely causes:

- `revision_failed_step=tools` with "wolframscript was not found", "cargo was not found" or "pdflatex was not found": install the program (Section 23.3), open a new terminal and run the gate again; for WolframScript, also activate it with your licence.
- A Wolfram step whose log shows a licence or kernel-limit message: the gate retries it after 30 s, at most 3 times; if it still fails, close other Wolfram sessions and run that step alone with the option steps.
- `revision_failed_step=precheck`: an output file of a selected step already differs from the committed version, and the gate refuses to overwrite your work. Run it in a fresh clone of the repository, or leave those steps out with the option steps.
- `revision_failed_step=committed-unchanged`: a verifier wrote an output that differs from the committed one; the lines `revision_changed_output=PATH` name the files. Run `git diff` on them to see the difference; this is a real disagreement with the record and must be reported, not hidden.

23.6 Stage 2: every notebook of the book

One command checks all notebooks of one chapter. The first line of each pair below names the folder of the builders; the second runs nbkit check on the builders of chapter 00 into the scratch folder `book-check` in your home folder. For another chapter, replace `00` by its two-digit number, `01` to `23`.

Windows (PowerShell):

```
$builders = (Get-ChildItem Revision/textbook/notebooks/src/00*.py).FullName
python Revision/textbook/tools/nbkit.py check $builders --scratch "$HOME\book-check"
```

macOS (zsh) and Linux (bash):

```
src=Revision/textbook/notebooks/src
python Revision/textbook/tools/nbkit.py check $src/00*.py --scratch ~/book-check
```

To check every notebook of the book, one after the other, give the folder itself:

Windows (PowerShell):

```
$src = "Revision/textbook/notebooks/src"
python Revision/textbook/tools/nbkit.py check $src --scratch "$HOME\book-check"
```

macOS (zsh) and Linux (bash):

```
src=Revision/textbook/notebooks/src
python Revision/textbook/tools/nbkit.py check $src --scratch ~/book-check
```

What it prints when it works. For each notebook NNx four lines, for example for the first notebook

```
check_00a_compared_files=4
check_00a_seconds=3.6
check_00a_peak_mb=174.0
check_00a=PASSED
```

(the seconds and megabytes differ from run to run), and at the end `nbkit_seconds=...` and `nbkit=OK`. The number of compared files counts the notebook itself together with the files it writes: Notebook 00a writes three files, so nbkit compares $3 + 1 = 4$ files with the stored ones. A check changes no file of the repository.

How long it takes. The recorded check times of the notebooks of each chapter, added up, are printed by Notebook 23a in Out [6] of Section 23.11 and drawn in Figure 23a.5. On the development machine the Kohn-Sham chapters 14, 15 and 16 take the most, 88.1 s, 240.4 s and 121.2 s; chapter 11 takes 48.0 s (Notebook 11a only 7.6 s: it reads the record of the Rust GKD self-test, whose run takes 7 to 13 minutes, instead of running it); every other chapter takes between 9.8 s (chapter 17) and 87.2 s (chapter 6). All 90 notebooks of chapters 0 to 22 take 1097.9 s, about 18 minutes, one after the other (Out [6]); Notebook 23a adds about 6.5 s (its recorded check). The seven notebooks that run a Rust program (11a, 11b, 15a, 15b, 15d, 16a and 19a) build it with cargo the first time, which takes a few minutes more.

When it fails. A failed check prints `check_NNx=FAILED` and, before it, lines that start with `problem=`. The most likely causes:

- A problem line saying that the notebook differs from the stored one: the new execution printed something else than the stored one. The new notebook is in the scratch folder, in the subfolder named after the notebook; compare it with the stored one (the line after the problem shows the first difference). A different package version is the usual cause: install exactly the pinned versions (Section 23.3).
- `cargo was not found`: install Rust (Section 23.3) and open a new terminal.
- A problem line saying that the provenance file differs from the provenance regenerated from its own record: someone edited the provenance file by hand or rebuilt the notebook without its provenance; rebuild the notebook with `nbkit.py build` and the date of the verified execution.

23.7 Stages 4 and 5: assembling the book and building its PDF

The assembler checks the whole book and compares it with the stored file, without writing anything, with the first command below; the second command writes the book's Markdown file `Revision/textbook/UNIVERSES_IN_PAIRS_TEXTBOOK.md`. The commands are the same in PowerShell, zsh and bash:

```
python Revision/textbook/tools/assemble_textbook.py --check
python Revision/textbook/tools/assemble_textbook.py
```

It checks every rule of the book's format: every reference "Chapter N" and "Section N.M" resolves, no section number repeats, every notebook marker resolves to an executed notebook and is preceded by its run instructions, every figure file exists, every code line fits. It prints one line `check_NAME=true` per check, then the measurements (`measurement_chapters=24`, the numbers of sections, notebooks, figures and lines, `measurement_pages=...`), `failed_check_count=0`, `output=...` and finally `textbook_assembly=OK`. A failed check prints `problem=...` lines that name the chapter, the reference or the file, and the last line `textbook_assembly=FAILED`. On the development machine the check took about 4 seconds (measured 2026-10-08).

The PDF is built from the assembled file with one command. In PowerShell, the first three lines put the parts of the command into variables (a list of texts after `=` is passed to the program as separate words), and the fourth runs it; in zsh and bash, the backslash at the end of a line continues the command on the next line:

Windows (PowerShell):

```
$book = "Revision/textbook/UNIVERSES_IN_PAIRS_TEXTBOOK.md"
$layout = "--developer-layout", "--number-sections-from-zero", "--wide-page-numbers"
$more = "--specifications", "Revision/pdf-specifications.json", "--date", "October 2026"
python scripts/build_provenance_pdf.py $book $layout $more
```

macOS (zsh) and Linux (bash):

```
python scripts/build_provenance_pdf.py Revision/textbook/UNIVERSES_IN_PAIRS_TEXTBOOK.md \
    --developer-layout --number-sections-from-zero --wide-page-numbers \
    --specifications Revision/pdf-specifications.json --date "October 2026"
```

Without further options it builds in VERIFY mode: it builds the PDF twice, demands that the two builds are byte for byte identical and that pdflatex printed no warning, and compares the result with the edition `universes-in-pairs-textbook` registered in `Revision/pdf-specifications.json` (the registry entry holds the page count of the PDF and its sha256, a fingerprint of its bytes). The option date sets the date printed under the title, so it is part of the PDF: the comparison succeeds only with the date with which the registered edition was built. The book's edition is built with the date "October 2026", as in the commands above; without the option the builder prints its default date, "September 2026", and the PDF then differs from the edition built with October. With the option register added at the end it records a changed book as the new edition. It needs pdflatex. When it works it prints one line `check_NAME=true` per check, the measurements (among them the page count), `failed_check_count=0` and the last line `provenance_pdf=OK`; any LaTeX warning (an overfull line, a figure too large for a page) is printed and fails the build, and the message names the line of the `.tex` file, which belongs to the chapter text at the same place.

To try the PDF build without touching the registered book, `check_chapter.py` builds one chapter, or with the option book all chapters, into a scratch folder with a throw-away copy of the registry:

Windows (PowerShell):

```
$chapter = "Revision/textbook/chapters/23-reproducing-everything.md"
python Revision/textbook/tools/check_chapter.py $chapter --scratch "$HOME\book-pdf"
python Revision/textbook/tools/check_chapter.py --book --scratch "$HOME\book-pdf"
```

macOS (zsh) and Linux (bash):

```
chapter=Revision/textbook/chapters/23-reproducing-everything.md
python Revision/textbook/tools/check_chapter.py $chapter --scratch ~/book-pdf
python Revision/textbook/tools/check_chapter.py --book --scratch ~/book-pdf
```

It prints `chapter_check=OK` when the chapter builds without any problem and warning. A test build of the whole book made in this way on 2026-10-08, with the chapters as they stood that day, took 287 s for its first pass (two complete pdflatex builds of a book of 6289 pages, compared byte for byte), and that pass ended FAILED: pdflatex printed warnings for the text of other chapters as it stood then, so the second, verify pass, which repeats the builds, did not run. A warning-free build of the whole book had not yet been measured when this chapter was written.

23.8 What to do when something fails: the general rules

1. Read the LAST lines first: every tool of this chapter ends with one line that says OK or FAILED, and the lines just before it say why.
2. Make sure the environment is active (the prompt starts with `(dirac-book-env)`) and that you are in the repository root; most "file not found" messages come from a terminal in another folder.
3. Install exactly the pinned versions; a newer matplotlib draws a figure with different bytes, and a byte comparison then fails although the physics is the same.
4. Run the failing piece alone: one gate step with the option steps, one notebook with one nbkit check.
5. Never "fix" a failure by editing a report, a provenance file or a stored notebook by hand. A difference between a fresh run and the record is a finding; report it.

23.9 Example: the bookkeeping of the whole book (Notebook 23a)

Notebook 23a collects in one place what the other notebooks and the Revision record have recorded. By default it re-runs nothing: every table and figure it shows is read from files. It reads the provenance file of every other notebook and prints a table of all 90 with their checks, figures, last recorded check and section of the book; it adds them up per chapter and draws five figures of the counts and times; it reads the step table of the gate and draws its expected times; and it builds the index of the 1357 checks of the 51 reports of the Revision record, with the chapters that cite each, and writes it to a data file. With `BOOK_RERUN_ALL=1` its thirteenth part, "Re-running every notebook", re-runs the nbkit check of every other notebook, at most 8 at a time and the slowest first, into a scratch folder in the system's temporary folder, and prints the fresh results per chapter. On the development machine (24 threads, so 8 notebooks at a time) that re-run took 406 s of wall

time on 2026-10-08 (the whole notebook, run headless, 414 s), and all 90 notebooks passed. To run it, set the variable in the terminal and run the notebook headless, writing the executed copy into the folder `book-rerun` of your home folder (so that the stored notebook is not overwritten); afterwards remove the variable again:

Windows (PowerShell):

```
$nb = "Revision/textbook/notebooks/23a_reproduce_everything.ipynb"
$options = "--to", "notebook", "--execute", "--ExecutePreprocessor.timeout=3600"
$env:BOOK_RERUN_ALL = "1"
python -m nbconvert $options --output-dir "$HOME\book-rerun" $nb
Remove-Item Env:BOOK_RERUN_ALL
```

macOS (zsh) and Linux (bash):

```
nb=Revision/textbook/notebooks/23a_reproduce_everything.ipynb
BOOK_RERUN_ALL=1 python -m nbconvert --to notebook --execute \
    --ExecutePreprocessor.timeout=3600 --output-dir ~/book-rerun $nb
```

(In zsh and bash, a variable written before a command on the same line holds only for that command.) It prints `[NbConvertApp] Writing ... bytes to ...`; the fresh table is in the executed copy, which you open in JupyterLab, and in the file `fresh_results.csv` whose path the notebook prints. To use JupyterLab directly instead, set the variable in the terminal first and then start JupyterLab from that same terminal.

23.10 How to run Notebook 23a

Step 1. What this notebook does and what it needs.

Notebook 23a (Reproducing everything: every notebook, every check, the gate) is the file `Revision/textbook/notebooks/23a_reproduce_everything.ipynb` of the repository Dirac_claude. It reads the recorded nbkit check of every other notebook of the book from its provenance file (by default it does NOT re-run the other notebooks), prints a table of every notebook with its title, number of checks, number of figures, the date, result and seconds of its last recorded check and the section of the book where it is printed, and draws the run times; it reads the step table of the Revision gate with the expected wall time of every step; and it builds the index of every check of every report of the Revision record, with its verdict and the chapters that cite it, written to a data file. With the environment variable BOOK_RERUN_ALL set to 1 it also runs the nbkit check of every other notebook, several at a time, into a scratch folder in the system temporary folder, and tabulates the fresh results. It needs a computer with Windows 11, macOS or Linux, an internet connection for the installation, the program Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type `python3.12` instead of `python3` in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private

Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands `python` and `jupyter` typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run `git pull` in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with `Set-ExecutionPolicy` allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 23a_reproduce_everything.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with `http://localhost:8888/lab` from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows `[*]`; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 23a_reproduce_everything.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with `[NbConvertApp] Writing` and shows no error, every check passed; open the notebook with `jupyter lab 23a_reproduce_everything.ipynb` (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- Revision/textbook/figures/23a.captions.json
- Revision/textbook/figures/23a_1_check_time_per_notebook.png
- Revision/textbook/figures/23a_2_cumulative_check_time.png
- Revision/textbook/figures/23a_3_checks_per_chapter.png
- Revision/textbook/figures/23a_4_figures_per_chapter.png
- Revision/textbook/figures/23a_5_check_time_per_chapter.png
- Revision/textbook/figures/23a_6_gate_step_times.png
- Revision/textbook/data/23a_notebooks.csv
- Revision/textbook/data/23a_check_index.csv
- Revision/textbook/data/23a_gate_steps.csv

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every file this notebook writes exists
ALL 10 CHECKS PASSED (notebook 23a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

```
python -m ipykernel install --user --name python3
```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file Revision/textbook/notebooks/23a_reproduce_everything.ipynb inside the folder Dirac_claude.

- "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was downloaded before; skip the line with `git clone`.
- Windows: "running scripts is disabled on this system": run the line with `Set-ExecutionPolicy` and then the activation line again. "py is not recognized": use `python` instead of `py -3`; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run `sudo apt install python3-venv` and repeat Step 3.
- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (`python -m jupyter` does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what `jupyter lab` does, the second what the headless command does:

```
python -m jupyterlab 23a_reproduce_everything.ipynb
python -m nbconvert --execute --inplace 23a_reproduce_everything.ipynb
```

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The last section prints that no other notebook was re-run: that is the default. To re-run every other notebook, set the environment variable BOOK_RERUN_ALL to 1 in the terminal, start JupyterLab or the headless run from that same terminal, and run the notebook again. Unset it afterwards.

```
$env:BOOK_RERUN_ALL = "1"      (Windows PowerShell)
export BOOK_RERUN_ALL=1          (macOS zsh, Linux bash)
```

- With BOOK_RERUN_ALL set to 1, the notebooks that run a Rust program fail: install Rust from https://rustup.rs, open a new terminal, activate the environment and run the notebook again; the first build of each Rust program takes a few minutes. On the development machine the re-run of all other notebooks took 406 seconds of wall time, 8 notebooks at a time.

23.11 Notebook 23a: complete text

This section prints the complete text of the notebook file `Revision/textbook/notebooks/23a_reproduce_everything.ipynb`: every cell in order, with the output that the stored, executed

notebook holds. A text cell (Jupyter calls it a markdown cell) is printed in a frame after the label Text cell. A code cell is printed after the label In [k], where k is the number that Jupyter gives the cell when it runs it; the text that the cell prints follows after the label Out [k], and every figure that the cell draws follows as a figure with its caption.

Text cell:

```
## 1. What this notebook computes

This book has one executed notebook for every worked example. Each notebook has a
provenance file next to it, and the last line of that file records the most recent
verified check of the notebook: the tool ‘nbkit‘ executed the notebook a second time
into a scratch folder and compared everything it produced, byte for byte, with the
stored copy. This notebook collects those records and the check reports of the
Revision record into the tables of the last chapter:

- a table of every other notebook of the book: its id, title, number of checks,
  number of figures, the date, result and seconds of its last RECORDED check, and
  the section of the book where its run instructions are printed;
- the totals per chapter, and five figures of the counts and the run times;
- the step table of the Revision gate ‘Revision/verify_revision.sh‘ (the script that
  re-runs every verifier of the Revision record), with the expected wall time of
  every step, and a sixth figure;
- the index of the checks: every check of every report of the Revision record, its
  verdict and the chapters that cite it, written to a data file.

By DEFAULT this notebook does NOT re-run the other notebooks: it reads what their
provenance files record. Only when the environment variable ‘BOOK_RERUN_ALL‘ is set
to ‘1‘ does its last computing section run the check of every other notebook
again, several at a time, and print the fresh results. Notebook 23a does not list
itself: its own record changes every time it is built.
```

Text cell:

```
## 2. How to run this notebook

**Step 1. What this notebook does and what it needs.**

Notebook 23a (Reproducing everything: every notebook, every check, the gate) is the file
‘Revision/textbook/notebooks/23a_reproduce_everything.ipynb‘ of the repository
Dirac_claude. It reads the recorded nbkit check of every other notebook of the book from
its provenance file (by default it does NOT re-run the other notebooks), prints a table
of every notebook with its title, number of checks, number of figures, the date, result
and seconds of its last recorded check and the section of the book where it is printed,
and draws the run times; it reads the step table of the Revision gate with the expected
wall time of every step; and it builds the index of every check of every report of the
Revision record, with its verdict and the chapters that cite it, written to a data file.
With the environment variable BOOK_RERUN_ALL set to 1 it also runs the nbkit check of
every other notebook, several at a time, into a scratch folder in the system temporary
folder, and tabulates the fresh results. It needs a computer with Windows 11, macOS or
Linux, an internet connection for the installation, the program Git, and Python 3.12 or
newer (the notebooks were built with Python 3.14.5) with these packages at exactly these
```

versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6. The notebook itself imports matplotlib; the other packages run Jupyter, the program that shows and runs notebooks. It does not need Rust.

Step 2. Install Git and Python (once per computer).

A terminal is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). Type every command of this section into a terminal exactly as printed, one line at a time, and press Enter after each line.

Windows: download Git from https://git-scm.com/download/win and install it with the default choices. Download Python from https://www.python.org/downloads/ and run the installer; in its first window tick the box "Add python.exe to PATH" before you click "Install Now".

macOS: download Python from https://www.python.org/downloads/ and run the installer. Then open the app Terminal and run this command, which installs Git and the compiler tools:

macOS (zsh):

```
xcode-select --install
```

Linux (Debian or Ubuntu; other distributions have packages of the same names): run these two commands:

Linux (bash):

```
sudo apt update
sudo apt install git python3 python3-venv
```

Then close the terminal and open a NEW one (a terminal opened before the installation does not know the new programs), and check the version of Python; it must print 3.12 or a higher number:

Windows (PowerShell):

```
py -3 --version
```

macOS (zsh) and Linux (bash):

```
python3 --version
```

If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python 3.12 with the following three commands, and in Step 3 type ‘python3.12‘ instead of ‘python3‘ in the command that creates the environment:

Linux (bash), only when the version is too low:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

Step 3. Get the repository and install the packages (once per computer).

The commands below download the repository (a few hundred megabytes; the folder then needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home folder, create a private Python environment in the folder dirac-book-env inside your home folder (a private environment is a folder with its own copy of Python and of the packages, so that nothing else on your computer is changed), activate it (from then on the commands ‘python‘ and ‘jupyter‘ typed in this terminal are those of the environment, and the prompt starts with (dirac-book-env)), and install the packages at the pinned versions (this downloads about 90 MB and takes a few minutes; the environment takes about 400 MB of disk space). If you have done this step before, for this or for another notebook, skip it and go to Step 4; to get the newest version of the repository, run ‘git pull‘ in the repository folder after Step 4.

Windows (PowerShell):

```
cd $HOME
git clone https://github.com/once-ere/Dirac_claude.git
py -3 -m venv "$HOME\dirac-book-env"
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

The line with ‘Set-ExecutionPolicy‘ allows PowerShell to run the activation script in this one window only; it changes nothing permanently.

macOS (zsh):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Linux (bash):

```
cd ~
git clone https://github.com/once-ere/Dirac_claude.git
python3 -m venv ~/dirac-book-env
source ~/dirac-book-env/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
```

Step 4. In every new terminal: activate the environment and go into the repository folder.

Every later step starts in the repository folder with the environment active. Run these lines whenever you open a new terminal (they do no harm if the environment is already active):

Windows (PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
& "$HOME\dirac-book-env\Scripts\Activate.ps1"
cd "$HOME\Dirac_claude"
```

macOS (zsh) and Linux (bash):

```
source ~/dirac-book-env/bin/activate
cd ~/Dirac_claude
```

Step 5. Run the notebook in JupyterLab.

In the repository folder, with the environment active, run these two lines (the first one goes into the folder of the notebooks):

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter lab 23a_reproduce_everything.ipynb
```

JupyterLab opens in your web browser and shows the notebook (if no browser window opens, copy the address that starts with ‘http://localhost:8888/lab‘ from the terminal into the address bar of your browser). The name at the top right of the notebook must read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the label to its left shows ‘[*]‘; when it has finished, the label shows a number. The notebook has finished when the last code cell shows a number; this takes about 10 seconds on a typical laptop. Scroll to the end and compare the last printed lines with the lines in the step "What the notebook writes and what you must see" below. To keep the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose the menu File > Shut Down, or press Ctrl+C twice in the terminal.

Step 6. Or run the notebook without a browser (headless).

In the repository folder, with the environment active, run:

Windows, macOS and Linux:

```
cd Revision/textbook/notebooks
jupyter nbconvert --to notebook --execute --inplace 23a_reproduce_everything.ipynb
```

This runs every cell from the top to the bottom and saves the results into the notebook file. A failed check stops it with an error message that contains AssertionError and the name of the check. If the command ends with a line that starts with ‘[NbConvertApp] Writing‘ and shows no error, every check passed; open the notebook with ‘jupyter lab 23a_reproduce_everything.ipynb‘ (in the same folder, without running the cells again) to read the printed lines and see the figures.

Step 7. What the notebook writes and what you must see.

The notebook writes (or overwrites) these files:

- ‘Revision/textbook/figures/23a.captions.json‘
- ‘Revision/textbook/figures/23a_1_check_time_per_notebook.png‘
- ‘Revision/textbook/figures/23a_2_cumulative_check_time.png‘
- ‘Revision/textbook/figures/23a_3_checks_per_chapter.png‘
- ‘Revision/textbook/figures/23a_4_figures_per_chapter.png‘
- ‘Revision/textbook/figures/23a_5_check_time_per_chapter.png‘
- ‘Revision/textbook/figures/23a_6_gate_step_times.png‘
- ‘Revision/textbook/data/23a_notebooks.csv‘
- ‘Revision/textbook/data/23a_check_index.csv‘
- ‘Revision/textbook/data/23a_gate_steps.csv‘

It changes no other file of the repository; running it headless or saving it in JupyterLab also rewrites the notebook file itself. It does not use the internet while it runs. The files it writes are the same files that are stored in the repository (on another computer a figure may differ in a few bytes, which is harmless). To get the stored versions back, run this command in the repository folder (it also undoes every change you made yourself in the folder Revision/textbook):

Windows, macOS and Linux:

```
git checkout -- Revision/textbook
```

Every check of the notebook prints a line that starts with PASS. At the end of the notebook (the output of its last code cell) you must see exactly these lines:

```
PASS every file this notebook writes exists
ALL 10 CHECKS PASSED (notebook 23a)
```

and the notebook must show 6 figures below the cells that draw them.

Step 8. If something goes wrong.

- "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started without the environment. Stop it (menu File > Shut Down), do Step 4, and start it again. If the error remains, another Python installation has registered its own kernel called python3; with the environment active, run the following command and start JupyterLab again:

  ```
  python -m ipykernel install --user --name python3
  ```

- "FileNotFoundError: The repository folder was not found": the notebook was opened from a copy outside the repository. Open the file ‘Revision/textbook/notebooks/23a_reproduce_everything.ipynb‘ inside the folder Dirac_claude.
- "fatal: destination path ’Dirac_claude’ already exists" in Step 3: the repository was downloaded before; skip the line with ‘git clone‘.
- Windows: "running scripts is disabled on this system": run the line with ‘Set-ExecutionPolicy‘ and then the activation line again. "py is not recognized": use ‘python‘ instead of ‘py -3‘; "python is not recognized": install Python again and tick "Add python.exe to PATH".
- Linux: "ensurepip is not available" when the environment is created: run ‘sudo apt install python3-venv‘ and repeat Step 3.

- "jupyter is not recognized" or "command not found: jupyter": the environment is not active; do Step 4. If the environment is active and the message stays, the programs jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for programs (‘python -m jupyter‘ does not help then: it has to find the same programs). Start them through Python itself, in the folder of the notebook: the first command below does what ‘jupyter lab‘ does, the second what the headless command does:

      python -m jupyterlab 23a_reproduce_everything.ipynb
      python -m nbconvert --execute --inplace 23a_reproduce_everything.ipynb

- Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event loop" and zmq: this is a message of the package pyzmq, not an error; the run continues normally.
- A red box with "Matplotlib is building the font cache; this may take a moment." in the first run after the installation: this is a message, not an error; the run continues and the message does not come again.
- An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel and Run All Cells; if it fails again, install the packages again with the pip commands of Step 3, because a different package version can change the last digits of a result.
- The last section prints that no other notebook was re-run: that is the default. To re-run every other notebook, set the environment variable BOOK_RERUN_ALL to 1 in the terminal, start JupyterLab or the headless run from that same terminal, and run the notebook again. Unset it afterwards.

      $env:BOOK_RERUN_ALL = "1"   (Windows PowerShell)
      export BOOK_RERUN_ALL=1   (macOS zsh, Linux bash)

- With BOOK_RERUN_ALL set to 1, the notebooks that run a Rust program fail: install Rust from https://rustup.rs, open a new terminal, activate the environment and run the notebook again; the first build of each Rust program takes a few minutes. On the development machine the re-run of all other notebooks took 406 seconds of wall time, 8 notebooks at a time.

The first code cell below (the set-up cell) repeats these instructions as comments; then it imports the packages, finds the repository folder and defines the helpers ‘save_figure‘ (saves a figure and shows it), ‘check‘ (stops with an AssertionError when a check fails, prints a PASS line when it passes), ‘report‘ (prints a key number as a RESULT line) and ‘all_checks_passed‘ (prints the last line).

In [1]:

```
# HOW TO RUN NOTEBOOK 23a (the complete instructions)
#
# STEP 1. WHAT THIS NOTEBOOK DOES AND WHAT IT NEEDS.
#
# Notebook 23a (Reproducing everything: every notebook, every check, the gate) is the
# file Revision/textbook/notebooks/23a_reproduce_everything.ipynb of the repository
# Dirac_claude. It reads the recorded nbkit check of every other notebook of the book
# from its provenance file (by default it does NOT re-run the other notebooks), prints a
# table of every notebook with its title, number of checks, number of figures, the date,
# result and seconds of its last recorded check and the section of the book where it is
# printed, and draws the run times; it reads the step table of the Revision gate with the
# expected wall time of every step; and it builds the index of every check of every
```

```
# report of the Revision record, with its verdict and the chapters that cite it, written
# to a data file. With the environment variable BOOK_RERUN_ALL set to 1 it also runs the
# nbkit check of every other notebook, several at a time, into a scratch folder in the
# system temporary folder, and tabulates the fresh results. It needs a computer with
# Windows 11, macOS or Linux, an internet connection for the installation, the program
# Git, and Python 3.12 or newer (the notebooks were built with Python 3.14.5) with these
# packages at exactly these versions: numpy 2.4.6, sympy 1.14.0, mpmath 1.3.0, matplotlib
# 3.11.0, jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and
# nbconvert 7.16.6. The notebook itself imports matplotlib; the other packages run
# Jupyter, the program that shows and runs notebooks. It does not need Rust.
#
# STEP 2. INSTALL GIT AND PYTHON (ONCE PER COMPUTER).
#
# A terminal is the window in which you type commands: on Windows the app Windows
# PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux
# any terminal (it runs the shell bash). Type every command of this section into a
# terminal exactly as printed, one line at a time, and press Enter after each line.
#
# Windows: download Git from https://git-scm.com/download/win and install it with the
# default choices. Download Python from https://www.python.org/downloads/ and run the
# installer; in its first window tick the box "Add python.exe to PATH" before you click
# "Install Now".
#
# macOS: download Python from https://www.python.org/downloads/ and run the installer.
# Then open the app Terminal and run this command, which installs Git and the compiler
# tools:
#
# macOS (zsh):
#     xcode-select --install
#
# Linux (Debian or Ubuntu; other distributions have packages of the same names): run
# these two commands:
#
# Linux (bash):
#     sudo apt update
#     sudo apt install git python3 python3-venv
#
# Then close the terminal and open a NEW one (a terminal opened before the installation
# does not know the new programs), and check the version of Python; it must print 3.12 or
# a higher number:
#
# Windows (PowerShell):
#     py -3 --version
#
# macOS (zsh) and Linux (bash):
#     python3 --version
#
# If an older Linux prints a lower number (Ubuntu 22.04 has Python 3.10), install Python
# 3.12 with the following three commands, and in Step 3 type python3.12 instead of
# python3 in the command that creates the environment:
#
# Linux (bash), only when the version is too low:
#     sudo add-apt-repository ppa:deadsnakes/ppa
#     sudo apt update
```

```
#     sudo apt install python3.12 python3.12-venv
#
# STEP 3. GET THE REPOSITORY AND INSTALL THE PACKAGES (ONCE PER COMPUTER).
#
# The commands below download the repository (a few hundred megabytes; the folder then
# needs up to about 1 GB of disk space) into the folder Dirac_claude inside your home
# folder, create a private Python environment in the folder dirac-book-env inside your
# home folder (a private environment is a folder with its own copy of Python and of the
# packages, so that nothing else on your computer is changed), activate it (from then on
# the commands python and jupyter typed in this terminal are those of the environment,
# and the prompt starts with (dirac-book-env)), and install the packages at the pinned
# versions (this downloads about 90 MB and takes a few minutes; the environment takes
# about 400 MB of disk space). If you have done this step before, for this or for another
# notebook, skip it and go to Step 4; to get the newest version of the repository, run
# git pull in the repository folder after Step 4.
#
# Windows (PowerShell):
#     cd $HOME
#     git clone https://github.com/once-ere/Dirac_claude.git
#     py -3 -m venv "$HOME\dirac-book-env"
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# The line with Set-ExecutionPolicy allows PowerShell to run the activation script in
# this one window only; it changes nothing permanently.
#
# macOS (zsh):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# Linux (bash):
#     cd ~
#     git clone https://github.com/once-ere/Dirac_claude.git
#     python3 -m venv ~/dirac-book-env
#     source ~/dirac-book-env/bin/activate
#     python -m pip install --upgrade pip
#     python -m pip install numpy==2.4.6 sympy==1.14.0 mpmath==1.3.0 matplotlib==3.11.0
#     python -m pip install jupyterlab==4.4.10 nbformat==5.10.4 nbclient==0.10.2
#     python -m pip install ipykernel==7.1.0 nbconvert==7.16.6
#
# STEP 4. IN EVERY NEW TERMINAL: ACTIVATE THE ENVIRONMENT AND GO INTO THE REPOSITORY
# FOLDER.
#
# Every later step starts in the repository folder with the environment active. Run these
```

```
# lines whenever you open a new terminal (they do no harm if the environment is already
# active):
#
# Windows (PowerShell):
#     Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process -Force
#     & "$HOME\dirac-book-env\Scripts\Activate.ps1"
#     cd "$HOME\Dirac_claude"
#
# macOS (zsh) and Linux (bash):
#     source ~/dirac-book-env/bin/activate
#     cd ~/Dirac_claude
#
# STEP 5. RUN THE NOTEBOOK IN JUPYTERLAB.
#
# In the repository folder, with the environment active, run these two lines (the first
# one goes into the folder of the notebooks):
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter lab 23a_reproduce_everything.ipynb
#
# JupyterLab opens in your web browser and shows the notebook (if no browser window
# opens, copy the address that starts with http://localhost:8888/lab from the terminal
# into the address bar of your browser). The name at the top right of the notebook must
# read Python 3 (ipykernel). Choose the menu Run > Run All Cells. While a cell runs, the
# label to its left shows [*]; when it has finished, the label shows a number. The
# notebook has finished when the last code cell shows a number; this takes about 10
# seconds on a typical laptop. Scroll to the end and compare the last printed lines with
# the lines in the step "What the notebook writes and what you must see" below. To keep
# the results, save the notebook (menu File > Save Notebook). To stop JupyterLab, choose
# the menu File > Shut Down, or press Ctrl+C twice in the terminal.
#
# STEP 6. OR RUN THE NOTEBOOK WITHOUT A BROWSER (HEADLESS).
#
# In the repository folder, with the environment active, run:
#
# Windows, macOS and Linux:
#     cd Revision/textbook/notebooks
#     jupyter nbconvert --to notebook --execute --inplace 23a_reproduce_everything.ipynb
#
# This runs every cell from the top to the bottom and saves the results into the notebook
# file. A failed check stops it with an error message that contains AssertionError and
# the name of the check. If the command ends with a line that starts with [NbConvertApp]
# Writing and shows no error, every check passed; open the notebook with jupyter lab
# 23a_reproduce_everything.ipynb (in the same folder, without running the cells again) to
# read the printed lines and see the figures.
#
# STEP 7. WHAT THE NOTEBOOK WRITES AND WHAT YOU MUST SEE.
#
# The notebook writes (or overwrites) these files:
#
# - Revision/textbook/figures/23a.captions.json
# - Revision/textbook/figures/23a_1_check_time_per_notebook.png
# - Revision/textbook/figures/23a_2_cumulative_check_time.png
```

```
# - Revision/textbook/figures/23a_3_checks_per_chapter.png
# - Revision/textbook/figures/23a_4_figures_per_chapter.png
# - Revision/textbook/figures/23a_5_check_time_per_chapter.png
# - Revision/textbook/figures/23a_6_gate_step_times.png
# - Revision/textbook/data/23a_notebooks.csv
# - Revision/textbook/data/23a_check_index.csv
# - Revision/textbook/data/23a_gate_steps.csv
#
# It changes no other file of the repository; running it headless or saving it in
# JupyterLab also rewrites the notebook file itself. It does not use the internet while
# it runs. The files it writes are the same files that are stored in the repository (on
# another computer a figure may differ in a few bytes, which is harmless). To get the
# stored versions back, run this command in the repository folder (it also undoes every
# change you made yourself in the folder Revision/textbook):
#
# Windows, macOS and Linux:
#     git checkout -- Revision/textbook
#
# Every check of the notebook prints a line that starts with PASS. At the end of the
# notebook (the output of its last code cell) you must see exactly these lines:
#
#     PASS every file this notebook writes exists
#     ALL 10 CHECKS PASSED (notebook 23a)
#
# and the notebook must show 6 figures below the cells that draw them.
#
# STEP 8. IF SOMETHING GOES WRONG.
#
# - "ModuleNotFoundError: No module named ..." in the notebook: JupyterLab was started
#   without the environment. Stop it (menu File > Shut Down), do Step 4, and start it
#   again. If the error remains, another Python installation has registered its own
#   kernel called python3; with the environment active, run the following command and
#   start JupyterLab again:
#     python -m ipykernel install --user --name python3
# - "FileNotFoundError: The repository folder was not found": the notebook was opened
#   from a copy outside the repository. Open the file
#   Revision/textbook/notebooks/23a_reproduce_everything.ipynb inside the folder
#   Dirac_claude.
# - "fatal: destination path 'Dirac_claude' already exists" in Step 3: the repository was
#   downloaded before; skip the line with git clone.
# - Windows: "running scripts is disabled on this system": run the line with
#   Set-ExecutionPolicy and then the activation line again. "py is not recognized": use
#   python instead of py -3; "python is not recognized": install Python again and tick
#   "Add python.exe to PATH".
# - Linux: "ensurepip is not available" when the environment is created: run sudo apt
#   install python3-venv and repeat Step 3.
# - "jupyter is not recognized" or "command not found: jupyter": the environment is not
#   active; do Step 4. If the environment is active and the message stays, the programs
#   jupyter-lab and jupyter-nbconvert are not in the folders where the terminal looks for
#   programs (python -m jupyter does not help then: it has to find the same programs).
#   Start them through Python itself, in the folder of the notebook: the first command
#   below does what jupyter lab does, the second what the headless command does:
#     python -m jupyterlab 23a_reproduce_everything.ipynb
#     python -m nbconvert --execute --inplace 23a_reproduce_everything.ipynb
```

```
# - Windows: the headless run prints a RuntimeWarning that mentions the "Proactor event
#   loop" and zmq: this is a message of the package pyzmq, not an error; the run
#   continues normally.
# - A red box with "Matplotlib is building the font cache; this may take a moment." in
#   the first run after the installation: this is a message, not an error; the run
#   continues and the message does not come again.
# - An AssertionError names a check that failed: choose the menu Kernel > Restart Kernel
#   and Run All Cells; if it fails again, install the packages again with the pip
#   commands of Step 3, because a different package version can change the last digits of
#   a result.
# - The last section prints that no other notebook was re-run: that is the default. To
#   re-run every other notebook, set the environment variable BOOK_RERUN_ALL to 1 in the
#   terminal, start JupyterLab or the headless run from that same terminal, and run the
#   notebook again. Unset it afterwards.
#     $env:BOOK_RERUN_ALL = "1"      (Windows PowerShell)
#     export BOOK_RERUN_ALL=1          (macOS zsh, Linux bash)
# - With BOOK_RERUN_ALL set to 1, the notebooks that run a Rust program fail: install
#   Rust from https://rustup.rs, open a new terminal, activate the environment and run
#   the notebook again; the first build of each Rust program takes a few minutes. On the
#   development machine the re-run of all other notebooks took 406 seconds of wall time,
#   8 notebooks at a time.
#
# ------------------------------------------------------------------------------------
# ====================================================================================
# THE SET-UP (the same in every notebook of this book; it computes no physics)
# ====================================================================================
import json  # reads and writes JSON files (text files that hold names and numbers)
import os  # reads the environment variable TEXTBOOK_OUTPUT_ROOT (explained below)
import textwrap  # breaks long printed text into lines of at most 89 characters
from pathlib import Path  # file and folder paths that work on Windows, macOS and Linux

import matplotlib  # the plotting package
import matplotlib.pyplot as plt  # its drawing functions, called plt by convention
from IPython.display import Image, display  # shows a saved picture below a cell

NOTEBOOK_ID = "23a"  # this notebook: chapter 23, example a


def find_repository_root():
    """Return the repository folder (Dirac_claude).

    Jupyter runs a notebook in the folder that holds it.  Starting there, go up one
    folder at a time until a folder contains Revision/textbook/requirements.txt (the
    list of the book's packages); that folder is the repository."""
    here = Path.cwd().resolve()  # the folder in which this notebook runs
    for folder in [here, *here.parents]:  # this folder, its parent, its grandparent ...
        if (folder / "Revision" / "textbook" / "requirements.txt").is_file():
            return folder
    raise FileNotFoundError(
        "The repository folder was not found: open this notebook inside the folder "
        "Revision/textbook/notebooks of the repository Dirac_claude")


# The repository folder.  It is never printed: it differs from computer to computer,
```

```
# and the printed output of a notebook must not.
REPO = find_repository_root()
# Every file is WRITTEN below OUTPUT_ROOT.  OUTPUT_ROOT is the repository folder unless
# the environment variable TEXTBOOK_OUTPUT_ROOT names another folder; the book's
# checking tool sets it, so that a check run writes into a scratch folder instead.
OUTPUT_ROOT = Path(os.environ.get("TEXTBOOK_OUTPUT_ROOT", str(REPO)))


def repository_file(relative):
    """The path of the repository file relative, for READING (a Revision record)."""
    return REPO / relative


def output_file(relative):
    """The path at which to WRITE the repository file relative (its folder is made)."""
    path = OUTPUT_ROOT / relative
    path.parent.mkdir(parents=True, exist_ok=True)
    return path


def say(text):
    """Print text in lines of at most 89 characters (the width of a page of the book)."""
    print(textwrap.fill(str(text), width=89, subsequent_indent="    "))


matplotlib.rcdefaults()  # ignore personal matplotlib settings: same figures everywhere
plt.rcParams.update({"figure.figsize": (7.0, 4.2), "font.size": 10.0,
                     "axes.grid": True, "grid.alpha": 0.3})

FIGURE_FOLDER = "Revision/textbook/figures"  # where the figures are saved
CAPTION_FILE = f"{FIGURE_FOLDER}/{NOTEBOOK_ID}.captions.json"  # their captions
FIGURE_NUMBERS = {}  # figure name -> its number k (file name <id>_<k>_<name>.png)
CAPTIONS = {}  # figure file name -> caption, written to CAPTION_FILE after every figure
# Start with an empty captions file ({} is an empty JSON dictionary); save_figure fills
# it.  newline="\n" writes the same line ends on Windows, macOS and Linux.
output_file(CAPTION_FILE).write_text("{}\n", encoding="utf-8", newline="\n")


def save_figure(fig, name, caption):
    """Save the figure fig as Revision/textbook/figures/<id>_<k>_<name>.png, record its
    caption in CAPTION_FILE, show the saved picture below the cell and close the figure.
    k counts the figures of the notebook 1, 2, 3, ...; a cell run again keeps its k."""
    number = FIGURE_NUMBERS.setdefault(name, len(FIGURE_NUMBERS) + 1)
    file_name = f"{NOTEBOOK_ID}_{number}_{name}.png"
    relative = f"{FIGURE_FOLDER}/{file_name}"
    # dpi=150: 150 dots per inch.  bbox_inches="tight": cut away the empty margin.
    # metadata={"Software": None}: no program name is stored in the PNG file, so that
    # every run writes exactly the same bytes.
    fig.savefig(output_file(relative), dpi=150, bbox_inches="tight",
                metadata={"Software": None})
    plt.close(fig)  # forget the figure, so that Jupyter does not draw it a second time
    CAPTIONS[file_name] = caption
    output_file(CAPTION_FILE).write_text(
        json.dumps(CAPTIONS, indent=1, sort_keys=True) + "\n", encoding="utf-8",
```

```
        newline="\n")
    display(Image(filename=str(output_file(relative))),
            metadata={"textbook_figure": file_name})  # the saved picture itself
    say(f"Figure {NOTEBOOK_ID}.{number} saved as {relative}")


PASSED = []  # the names of the checks that passed, in order


def check(condition, name, record=None):
    """A check.  If condition is False, stop with an AssertionError that names the
    check (an if statement is used instead of assert, because python -O would skip an
    assert).  Otherwise print "PASS <name>" and, when the check reproduces a Revision
    record, a second line naming the record file and its check."""
    if not condition:
        raise AssertionError(f"check failed: {name}")
    PASSED.append(name)
    say(f"PASS {name}")
    if record is not None:
        say(f"     reproduces {record}")


def report(label, value, unit=""):
    """Print a key number as a line "RESULT <label> = <value> <unit>"."""
    say(f"RESULT {label} = {value}" + (f" {unit}" if unit else ""))


def all_checks_passed():
    """Print the last line of the notebook: how many checks passed."""
    print(f"ALL {len(PASSED)} CHECKS PASSED (notebook {NOTEBOOK_ID})")


say(f"Set-up of notebook {NOTEBOOK_ID} complete: repository folder found, helpers "
    "defined.")
```

Out [1]:

```
Set-up of notebook 23a complete: repository folder found, helpers defined.
```

Text cell:

```
## 3. The words used in this notebook

- **Builder**: the Python file ‘Revision/textbook/notebooks/src/NNx_name.py‘ that
  defines the cells of notebook NNx; the tool ‘nbkit‘ builds the notebook from it.
- **nbkit check**: the command ‘python Revision/textbook/tools/nbkit.py check
  BUILDER‘. It executes the notebook again into a scratch folder and compares the
  notebook, every file it writes and its provenance file byte for byte with the
  stored ones; it prints ‘check_NNx=PASSED‘ or ‘check_NNx=FAILED‘.
- **Provenance file**: ‘NNx_name.PROVENANCE.md‘, next to each notebook; its last line,
  an HTML comment that starts with the word ‘nbkit-record‘, holds the measured facts
  of the last verified run, written in JSON.
```

- **JSON**: a text format for numbers, strings, lists and tables of named values.
- **CSV**: a text table, one row per line, the cells separated by commas.
- **Wall time**: the time on a clock on the wall, from start to finish, in seconds.
- **Report**: a JSON file of the Revision record that holds a list (or table) of named checks, each with a verdict such as PASS.
- **Gate**: the script `Revision/verify_revision.sh` (and its twin `Revision/verify_revision.ps1`) that re-runs every verifier and checker of the Revision record in dependency order.
- **Cite**: a chapter cites a check when one paragraph of the chapter, or the output of one code cell of one of the chapter's notebooks, names both the check and the file name of its report.
- **Environment variable**: a named text value that a terminal hands to every program it starts; `BOOK_RERUN_ALL` is one.

Text cell:

```
## 4. The physical and mathematical situation

Nothing in this notebook is physics: it is bookkeeping, and every number it prints is
COUNTED from files of the repository. The seconds are those MEASURED by nbkit on the
development machine (Windows 11, 24 threads) and recorded in the provenance files;
the gate's times are the EXPECTED wall times written in its step table. On another
computer the times differ, but the counts, the verdicts and the results do not.
```

Text cell:

```
## 5. The provenance record of every notebook

The next cell reads every provenance file of the folder of notebooks, except the
one of this notebook. From each it takes the title, the number of checks (from the
line `ALL n CHECKS PASSED`) and the recorded check; from the notebook's caption file
it counts the figures. It then checks that builders and provenance files match one
to one, that every recorded check passed, and that every figure file exists.
```

In [2]:

```
import csv  # reads and writes comma-separated tables
import re  # regular expressions: patterns that find pieces of text

NOTEBOOKS = repository_file("Revision/textbook/notebooks")  # notebooks and records
RECORD = re.compile(r"nbkit-record (\{.*\}) -->")  # the record line
ALL_LINE = re.compile(r"ALL (\d+) CHECKS PASSED")  # the last line of a notebook
TITLE = re.compile(r"^# Provenance of Notebook \w+: (.+)$", re.MULTILINE)


def by_name(paths):
    """Sort paths by their repository-relative name, the same on every system."""
    return sorted(paths, key=lambda path: path.relative_to(REPO).as_posix())


ROWS = []  # one dictionary per notebook
```

```
for path in by_name(NOTEBOOKS.glob("*.PROVENANCE.md")):
    name = path.name.removesuffix(".PROVENANCE.md")  # e.g. 00a_check_installation
    if name.startswith(NOTEBOOK_ID):
        continue  # this notebook's own record changes whenever it is built
    text = path.read_text(encoding="utf-8")
    record = json.loads(RECORD.search(text).group(1))
    captions = json.loads(repository_file(
        f"{FIGURE_FOLDER}/{name[:3]}.captions.json").read_text(encoding="utf-8"))
    ROWS.append({"id": name[:3], "chapter": int(name[:2]), "name": name,
                 "title": TITLE.search(text).group(1),
                 "checks": int(ALL_LINE.search(text).group(1)),
                 "figures": len(captions), "figure_files": sorted(captions),
                 "date": record["check"]["date"],
                 "result": record["check"]["result"],
                 "seconds": record["check"]["seconds"],
                 "builder": f"Revision/textbook/notebooks/src/{name}.py"})

builders = [path.stem for path in by_name(NOTEBOOKS.glob("src/*.py"))
            if not path.stem.startswith(NOTEBOOK_ID)]
say(f"{len(ROWS)} provenance files read, {len(builders)} builders found.")
check([row["name"] for row in ROWS] == builders,
      "every builder has a provenance file and every provenance file a builder")
check(all(row["result"] == "passed" for row in ROWS),
      "the last recorded nbkit check of every notebook passed")
check(all(repository_file(f"{FIGURE_FOLDER}/{figure}").is_file()
          for row in ROWS for figure in row["figure_files"]),
      "every figure named in a caption file exists")
```

Out [2]:

```
90 provenance files read, 90 builders found.
PASS every builder has a provenance file and every provenance file a builder
PASS the last recorded nbkit check of every notebook passed
PASS every figure named in a caption file exists
```

Text cell:

```
## 6. Where each notebook is printed in the book

A notebook is placed in its chapter by a marker line: an HTML comment that holds the
word ‘NOTEBOOK‘ and the notebook's id, such as ‘NOTEBOOK 03b‘. The book's assembler
replaces the marker by two new sections: the run instructions,
numbered one more than the last section heading ‘### N.M‘ before the marker, and
then the complete text of the notebook. The next cell reads chapters 0 to 22 (this
chapter 23 is left out, so that its own text cannot change the results), finds
every marker and the heading before it, and checks that every notebook is placed
exactly once, in the chapter of its own number.
```

In [3]:

```
CHAPTERS = [path for path in by_name(repository_file("Revision/textbook/chapters")
                                     .glob("[0-9][0-9]-*.md"))
```

```
            if int(path.name[:2]) <= 22]  # the chapter files 00 to 22
HEADING = re.compile(r"^### (\d+)\.(\d+) ")  # a section heading such as ### 3.4
MARKER = re.compile(r"^<!-{2} NOTEBOOK (\w+) -->$")  # a notebook marker line
PLACED = {}  # notebook id -> list of (chapter file name, section of its instructions)
for chapter in CHAPTERS:
    section = None
    for line in chapter.read_text(encoding="utf-8").splitlines():
        if HEADING.match(line):
            section = HEADING.match(line).groups()  # e.g. ("3", "4")
        elif MARKER.match(line):
            number = f"{section[0]}.{int(section[1]) + 1}"  # the next section
            PLACED.setdefault(MARKER.match(line).group(1), []).append(
                (chapter.name, number))

for row in ROWS:
    places = PLACED.get(row["id"], [])
    row["section"] = places[0][1] if len(places) == 1 else "?"
say(f"{len(CHAPTERS)} chapter files read, {len(PLACED)} notebook markers found.")
check(all(len(PLACED.get(row["id"], [])) == 1
          and PLACED[row["id"]][0][0].startswith(row["id"][:2] + "-")
          for row in ROWS),
      "every notebook is placed exactly once, in the chapter of its number")
```

Out [3]:

```
23 chapter files read, 90 notebook markers found.
PASS every notebook is placed exactly once, in the chapter of its number
```

Text cell:

```
## 7. The table of every notebook

The next cell defines a function that prints the rows of a range of chapters as a
table, and prints chapters 0 to 10. The columns: the id; the title (cut to 34
characters, with ‘...‘ when it is longer); the number of checks; the number of
figures; the date of the last recorded check; its result; its wall time in seconds;
and the section of the book that holds the notebook’s run instructions (the
notebook’s complete text is the section after it).
```

In [4]:

```
def show_table(first, last):
    """Print the notebooks of the chapters first to last as a table."""
    print("id  title                              checks figs checked    result "
          "   sec section")
    for row in ROWS:
        if first <= row["chapter"] <= last:
            title = row["title"]
            if len(title) > 34:
                title = title[:31] + "..."
            print(f"{row["id"]:3} {title:34} {row["checks"]:6d} {row["figures"]:4d} "
```

```
                f"{row["date"]:10} {row["result"]:6} {row["seconds"]:6.1f} "
                f"{row["section"]:>7}")


show_table(0, 10)
```

Out [4]:

```
id  title                                checks figs checked    result    sec section
00a Checking the installation                 9    2 2026-10-14 passed    5.5    0.11
00b The eight directions and the au...       17    4 2026-10-14 passed    3.4    0.15
00c The honesty ledger: reading and...       18    4 2026-10-14 passed    3.3    0.19
00d Why every run gives the same by...       22    6 2026-10-14 passed    3.6    0.23
01a Matrices, permutations and dete...       30    6 2026-10-14 passed    4.1    1.23
01b Complex numbers, rotations and ...       34    6 2026-10-14 passed    4.0    1.15
01c The generalized Kronecker delta...       28    6 2026-10-14 passed   10.4    1.46
01d Reproducing the GKD self-tests ...       19    6 2026-10-14 passed   10.2    1.52
01e Index notation, the summation c...       25    6 2026-10-14 passed    9.7    1.31
01f Numbers: fractions, real number...       26    6 2026-10-14 passed    4.0     1.7
01g Eigenvalues, eigenvectors and t...       30    9 2026-10-14 passed    4.9    1.39
02a Euler, midpoint and RK4: the or...       29    8 2026-10-14 passed    4.2     2.9
02b The shooting method: a vibratin...       16    8 2026-10-14 passed    4.4    2.15
02c Partial derivatives, finite dif...       17    6 2026-10-14 passed    4.9    2.21
02d Shooting two first-order equati...       19    8 2026-10-14 passed    5.1    2.26
03a The author's metric: signature,...       29    7 2026-10-14 passed    4.6    3.11
03b The curvature of the author's m...       40    7 2026-10-14 passed   24.7    3.28
03c Curvature from zero: the plane,...       16    5 2026-10-08 passed    4.6    3.20
03d Free fall in the author's metri...       23    5 2026-10-14 passed    6.2    3.33
04a The author's gamma matrices T16...       28    6 2026-10-14 passed    3.8     4.6
04b Square roots of quadratic forms...       19    6 2026-10-14 passed    3.6    4.10
04c The 256 products of the author'...       19    8 2026-10-14 passed    5.2    4.14
04d A second set of gammas from ten...       18    6 2026-10-14 passed    4.8    4.18
05a The matrices C, Gamma and B bui...       26    6 2026-10-14 passed    3.4     5.8
05b Commutants: Pin(4,4) acts irred...       17    5 2026-10-14 passed    8.7    5.15
05c The charge-conjugation matrices...       20    9 2026-10-14 passed    5.3    5.31
05d Spin(4,4) transformations: rota...       20    6 2026-10-14 passed    3.8    5.20
05e The quantised field: reordering...       18    8 2026-10-14 passed    3.5    5.36
05f What the scaled commutators gen...       24    6 2026-10-14 passed    6.3    5.25
06a Vielbein, spin connection and t...       36   10 2026-10-14 passed   25.9    6.12
06b What depends on the frame: boos...       26    6 2026-10-14 passed   49.0    6.20
06c One missing metric factor: the ...       27    6 2026-10-14 passed   12.3    6.26
07a A Grassmann algebra by hand              38    7 2026-10-14 passed    6.6    7.15
07b The Euler-Lagrange equations in...       47    6 2026-10-14 passed    5.5    7.25
07c Real fields: the Majorana-type ...       23    4 2026-10-14 passed    3.8    7.31
07d The action principle and the Eu...       30    8 2026-10-14 passed    6.4     7.7
08a Growth rates of the extra-time ...       30    6 2026-10-14 passed    3.5    8.21
08b The deflating extra times drive...       25    5 2026-10-14 passed    4.9    8.27
08c The spin-connection term depend...       32    5 2026-10-14 passed   43.6    8.13
08d The rescaling that removes the ...       23    5 2026-10-14 passed   27.0    8.34
09a The energy-momentum tensor and ...       27    5 2026-10-14 passed    7.5    9.15
09b Homogeneous condensates: energy...       33    7 2026-10-14 passed    8.3    9.23
09c The full energy-momentum tensor...       25    5 2026-10-14 passed    7.8    9.28
10a One-particle spectra and Krein ...       28    6 2026-10-14 passed    3.8    10.8
```

```
10b The canonical anticommutator fo...     25    5 2026-10-14 passed    4.0   10.17
10c The good sector: a positive Foc...     14    5 2026-10-14 passed    3.1   10.24
10d The curved good sector and the ...     16    5 2026-10-15 passed    5.5   10.30
10e Invariant forms and Krein-unita...     15    5 2026-10-14 passed    2.7   10.36
10f The Krein structure along the d...     19    5 2026-10-14 passed    3.7   10.41
10g The quantum reading of the pair...     30    5 2026-10-14 passed    4.3   10.47
```

Text cell:

```
The next cell prints chapters 11 to 22 and writes the complete table, with the full
titles and the builders, to the data file
‘Revision/textbook/data/23a_notebooks.csv‘.
```

In [5]:

```
show_table(11, 22)
COLUMNS = ["id", "chapter", "title", "checks", "figures", "date", "result",
           "seconds", "section", "builder"]
NOTEBOOK_TABLE = "Revision/textbook/data/23a_notebooks.csv"
with output_file(NOTEBOOK_TABLE).open("w", encoding="utf-8", newline="") as handle:
    writer = csv.writer(handle, lineterminator="\n")
    writer.writerow(COLUMNS)
    writer.writerows([row[column] for column in COLUMNS] for row in ROWS)
say(f"The table of {len(ROWS)} notebooks was written to {NOTEBOOK_TABLE}.")
```

Out [5]:

```
id  title                              checks figs checked    result    sec section
11a The generalized Kronecker delta...     24    6 2026-10-15 passed    7.6    11.7
11b The three Lovelock tensors of t...     38    5 2026-10-15 passed   36.1   11.17
11c The Lovelock tensors along the ...     20    5 2026-10-15 passed    4.3   11.25
12a The field equations for a4 deri...     59    5 2026-10-15 passed    5.9   12.14
12b The source that the linear memb...     24    8 2026-10-15 passed    4.0   12.18
12c Integrating the evolution equat...     30    6 2026-10-15 passed    4.1   12.23
12d An exact solution: a condensate...     40    6 2026-10-15 passed    6.2   12.28
13a A one-dimensional Kohn-Sham toy...     36   10 2026-10-14 passed    9.2   13.34
13b Exchange of a contact interacti...     44    9 2026-10-15 passed    4.6   13.18
13c Slater determinants, second qua...     33    8 2026-10-08 passed    6.2   13.12
13d Charge sloshing and mixing in t...     17    6 2026-10-08 passed    4.1   13.23
13e Finite temperature (Mermin) and...     24    7 2026-10-08 passed    4.6   13.29
14a The reduction of the Kohn-Sham ...     28    5 2026-10-15 passed   11.1    14.9
14b The free Kohn-Sham spectrum at ...     20    6 2026-10-14 passed   19.0   14.16
14c The brane band: free Kohn-Sham ...     15    6 2026-10-14 passed   44.5   14.23
14d The exact local exchange of the...     16    4 2026-10-14 passed   13.5   14.29
15a Running the Rust Kohn-Sham solv...     25    8 2026-10-14 passed   74.0   15.12
15b Energy-momentum profiles and th...     18    7 2026-10-14 passed   10.3   15.18
15c Adiabaticity along the deflatin...     12    6 2026-10-08 passed   11.1   15.23
15d Thermodynamics of the Kohn-Sham...     15    7 2026-10-14 passed   71.2   15.28
15e The solver’s method by hand: sh...     18    6 2026-10-14 passed   73.8    15.7
16a Running the reference solver on...     51    7 2026-10-14 passed   95.7   16.10
16b The reference method by hand: s...     31    7 2026-10-14 passed   13.5   16.17
```

```
16c How the cross-check caught a ro...     26    6 2026-10-14 passed   12.0   16.22
17a The source conditions of the a4...     16    7 2026-10-15 passed    4.5    17.9
17b The conservation laws and the s...     20    6 2026-10-15 passed    5.3   17.19
18a The matrix facts behind the pai...     16    7 2026-10-15 passed    3.5    18.7
18b T1 and T2 proved symbolically i...     20    5 2026-10-15 passed   17.3   18.18
18c T1 and T2 numerically: solution...     21    7 2026-10-15 passed    4.8   18.25
19a The Rust Kohn-Sham solver for t...     37   10 2026-10-15 passed   32.3   19.19
19b Theorem T3 by hand: the block m...     19    6 2026-10-15 passed    9.4   19.14
20a Corollary C1: a T1 pair as the ...     37    7 2026-10-15 passed    4.8    20.9
20b One universe and its two partne...     25    4 2026-10-15 passed    4.4   20.14
20c Allowed is not the same as happ...     12    4 2026-10-08 passed    4.2   20.19
21a Charge conjugation is a matrix:...     27    7 2026-10-15 passed    6.8   21.13
21b Local U(1) charge conservation,...     21    8 2026-10-15 passed   12.9   21.21
21c The conjugation of the quantise...     17    6 2026-10-15 passed    9.1   21.28
21d Sakharov's three conditions and...     19    8 2026-10-15 passed    5.4   21.34
22a When does the instantaneous Koh...     20    8 2026-10-15 passed   14.6   22.10
22b The dark-sector hypotheses: wha...     28    5 2026-10-15 passed    2.6   22.21
The table of 90 notebooks was written to Revision/textbook/data/23a_notebooks.csv.
```

Text cell:

```
## 8. The totals per chapter

The next cell adds up, for each chapter, the number of notebooks, checks and figures
and the recorded check seconds, prints them, and reports the totals of the book.
```

In [6]:

```
PER_CHAPTER = {}  # chapter number -> its totals
for row in ROWS:
    total = PER_CHAPTER.setdefault(row["chapter"], {"notebooks": 0, "checks": 0,
                                                    "figures": 0, "seconds": 0.0})
    total["notebooks"] += 1
    total["checks"] += row["checks"]
    total["figures"] += row["figures"]
    total["seconds"] += row["seconds"]

print("chapter notebooks checks figures  seconds")
for chapter, total in sorted(PER_CHAPTER.items()):
    print(f"{chapter:7d} {total["notebooks"]:9d} {total["checks"]:6d} "
          f"{total["figures"]:7d} {total["seconds"]:8.1f}")
ALL_SECONDS = sum(row["seconds"] for row in ROWS)
report("notebooks (without 23a)", len(ROWS))
report("checks in these notebooks", sum(row["checks"] for row in ROWS))
report("figures of these notebooks", sum(row["figures"] for row in ROWS))
report("recorded check time, one after the other", f"{ALL_SECONDS:.1f}", "s")
check(sorted(PER_CHAPTER) == list(range(23)),
      "every chapter 0 to 22 has at least one notebook")
```

Out [6]:

```
chapter notebooks checks figures  seconds
      0         4     66      16     15.8
      1         7    192      45     47.3
      2         4     81      30     18.6
      3         4    108      24     40.1
      4         4     84      26     17.4
      5         6    125      40     31.0
      6         3     89      22     87.2
      7         4    138      25     22.3
      8         4    110      21     79.0
      9         3     85      17     23.6
     10         7    147      36     27.1
     11         3     82      16     48.0
     12         4    153      25     20.2
     13         5    154      40     28.7
     14         4     79      21     88.1
     15         5     88      34    240.4
     16         3    108      20    121.2
     17         2     36      13      9.8
     18         3     57      19     25.6
     19         2     56      16     41.7
     20         3     74      15     13.4
     21         4     84      29     34.2
     22         2     48      13     17.2
RESULT notebooks (without 23a) = 90
RESULT checks in these notebooks = 2244
RESULT figures of these notebooks = 563
RESULT recorded check time, one after the other = 1097.9 s
PASS every chapter 0 to 22 has at least one notebook
```

Text cell:

```
## 9. Two figures of the run times

The next cell draws the recorded check time of every notebook as a bar (on a
logarithmic axis, because the times range from about a second to minutes), and the
share of the total time taken by the slowest notebooks, adding them up from the
slowest down.
```

In [7]:

```
ids = [row["id"] for row in ROWS]
seconds = [row["seconds"] for row in ROWS]
fig, ax = plt.subplots(figsize=(12, 4.5))
ax.bar(range(len(ROWS)), seconds, color="#2a78d6")
ax.set_yscale("log")  # equal steps on the axis are equal factors
ax.set_xticks(range(len(ROWS)), ids, rotation=90, fontsize=6)
ax.set_xlim(-1, len(ROWS))
ax.set_xlabel("notebook")
ax.set_ylabel("recorded nbkit check time (s)")
ax.grid(axis="y", which="both", alpha=0.3)
save_figure(fig, "check_time_per_notebook",
```

```
            "The wall time in seconds of the last recorded nbkit check of every "
            "notebook of chapters 0 to 22 (one bar each, in book order), on a "
            "logarithmic vertical axis. Most notebooks take a few seconds; a few, "
            "which run the Rust programs or long numerical scans, take minutes.")

slowest = sorted(seconds, reverse=True)
shares = [100 * sum(slowest[:k]) / ALL_SECONDS for k in range(len(slowest) + 1)]
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(range(len(shares)), shares, color="#eb6834", marker="o", markersize=3)
ax.set_xlabel("number of notebooks, slowest first")
ax.set_ylabel("share of the total check time (%)")
ax.grid(alpha=0.3)
save_figure(fig, "cumulative_check_time",
            "The share in percent of the total recorded check time taken by the "
            "slowest notebooks: the curve adds the check times up from the slowest "
            "notebook down. It rises steeply at first, because a few slow notebooks "
            "take a large part of the time, and then flattens.")
report("share of the three slowest notebooks", f"{shares[3]:.1f}", "%")
```

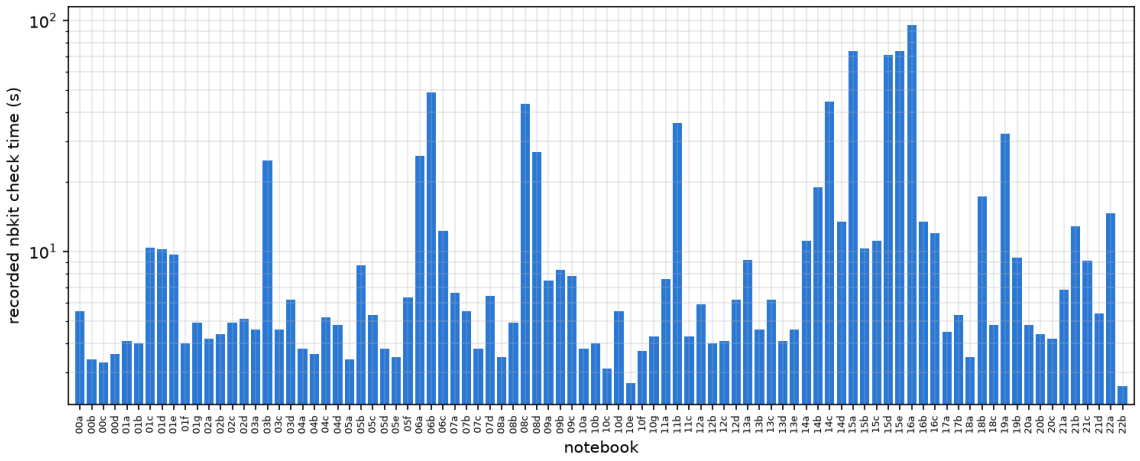

Figure 564: The wall time in seconds of the last recorded nbkit check of every notebook of chapters 0 to 22 (one bar each, in book order), on a logarithmic vertical axis. Most notebooks take a few seconds; a few, which run the Rust programs or long numerical scans, take minutes. (Notebook 23a, figure 1.)

Out [7]:

```
Figure 23a.1 saved as Revision/textbook/figures/23a_1_check_time_per_notebook.png
```

Out [7]:

```
Figure 23a.2 saved as Revision/textbook/figures/23a_2_cumulative_check_time.png
RESULT share of the three slowest notebooks = 22.2 %
```

Text cell:

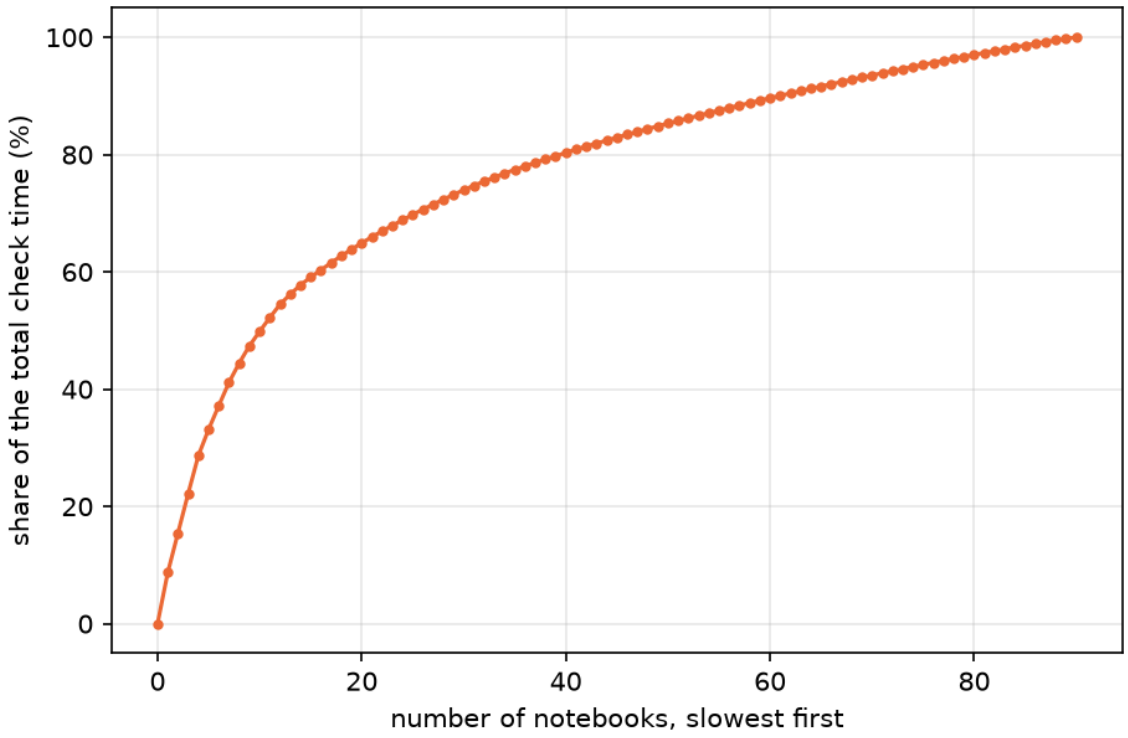

Figure 565: The share in percent of the total recorded check time taken by the slowest notebooks: the curve adds the check times up from the slowest notebook down. It rises steeply at first, because a few slow notebooks take a large part of the time, and then flattens. (Notebook 23a, figure 2.)

10. Checks, figures and run time per chapter

The next cell draws three bar charts with one bar per chapter: the number of checks,
the number of figures and the recorded check time of its notebooks.

In [8]:

```
chapters = sorted(PER_CHAPTER)
for key, name, label, colour, caption in [
        ("checks", "checks_per_chapter", "checks", "#1baf7a",
         "The number of checks of the notebooks of each chapter (the sum of the "
         "numbers n in their last lines ALL n CHECKS PASSED); horizontal axis the "
         "chapter, vertical axis the number of checks."),
        ("figures", "figures_per_chapter", "figures", "#eda100",
         "The number of figures drawn by the notebooks of each chapter (the "
         "entries of their caption files); horizontal axis the chapter, vertical "
         "axis the number of figures."),
        ("seconds", "check_time_per_chapter", "recorded check time (s)", "#4a3aa7",
         "The recorded nbkit check time in seconds of the notebooks of each "
         "chapter, added up; horizontal axis the chapter, vertical axis the "
         "seconds. This is the time one command per chapter needs on the "
         "development machine.")]:
    fig, ax = plt.subplots(figsize=(8, 3.6))
    ax.bar(chapters, [PER_CHAPTER[chapter][key] for chapter in chapters],
           color=colour)
    ax.set_xticks(chapters)
    ax.set_xlabel("chapter")
    ax.set_ylabel(label)
    ax.grid(axis="y", alpha=0.3)
    save_figure(fig, name, caption)
```

Out [8]:

```
Figure 23a.3 saved as Revision/textbook/figures/23a_3_checks_per_chapter.png
```

Out [8]:

```
Figure 23a.4 saved as Revision/textbook/figures/23a_4_figures_per_chapter.png
```

Out [8]:

```
Figure 23a.5 saved as Revision/textbook/figures/23a_5_check_time_per_chapter.png
```

Text cell:

```
## 11. The steps of the Revision gate

The gate ‘Revision/verify_revision.sh‘ holds its steps as a table between the first
```

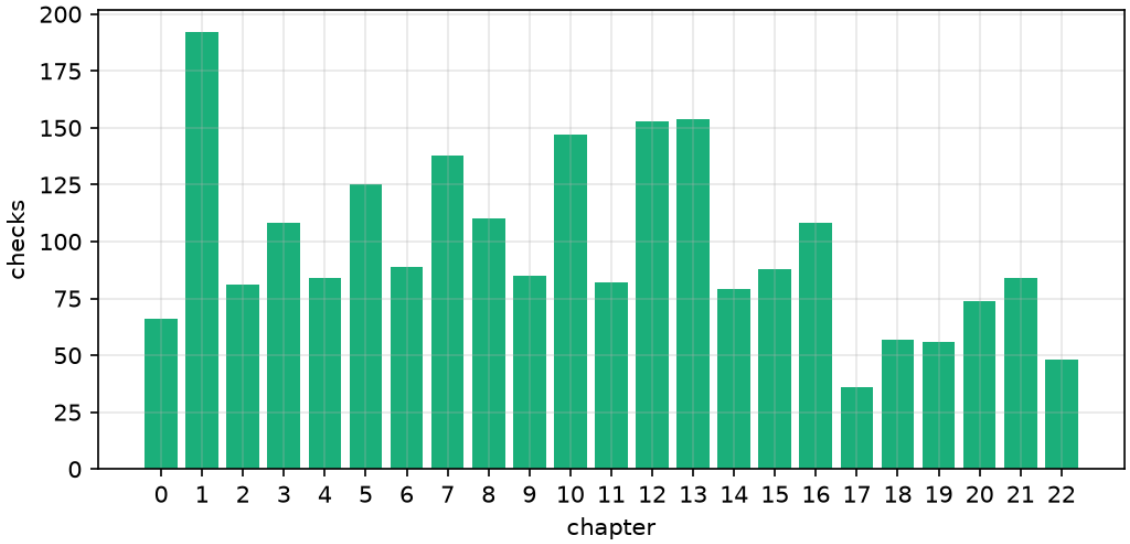

Figure 566: The number of checks of the notebooks of each chapter (the sum of the numbers n in their last lines ALL n CHECKS PASSED); horizontal axis the chapter, vertical axis the number of checks. (Notebook 23a, figure 3.)

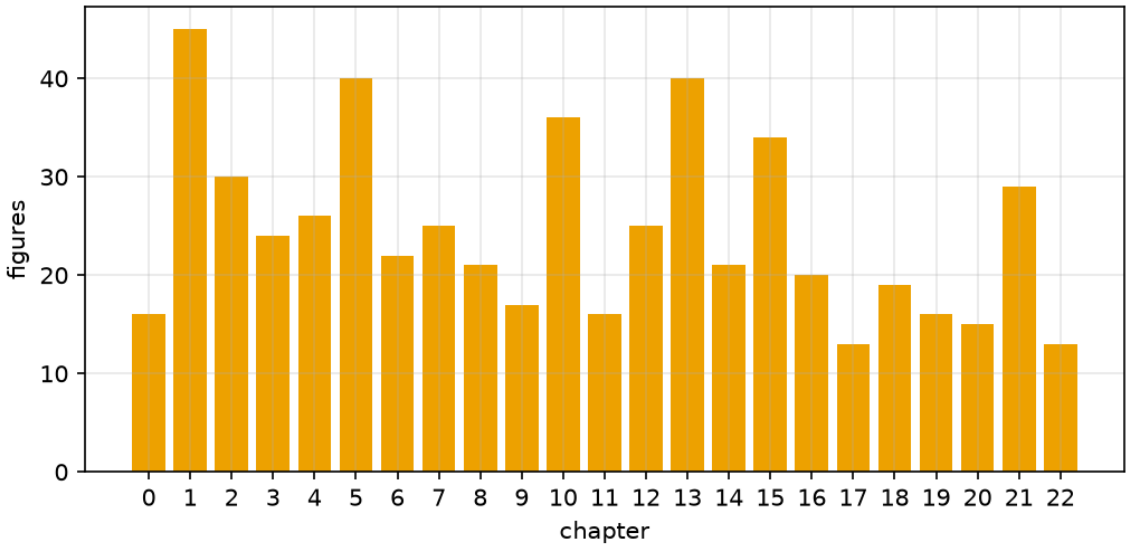

Figure 567: The number of figures drawn by the notebooks of each chapter (the entries of their caption files); horizontal axis the chapter, vertical axis the number of figures. (Notebook 23a, figure 4.)

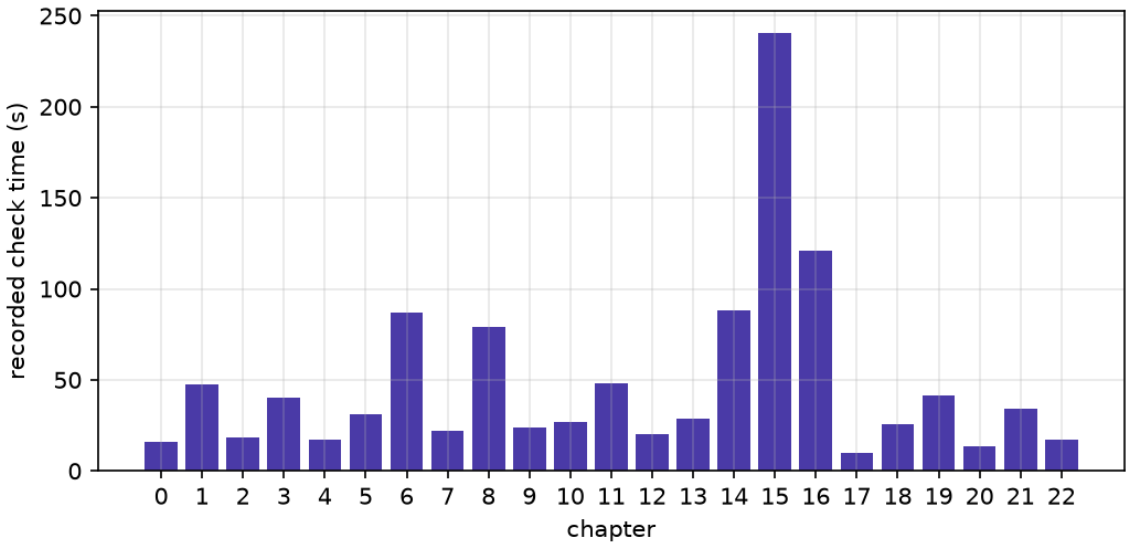

Figure 568: The recorded nbkit check time in seconds of the notebooks of each chapter, added up; horizontal axis the chapter, vertical axis the seconds. This is the time one command per chapter needs on the development machine. (Notebook 23a, figure 5.)

and the second appearance of the word ‘REVISION_GATE_STEPS‘. Each line of it has seven fields separated by ‘|‘: the step name, the expected wall time in seconds, the kind (‘1‘ = long, skipped by the option ‘--fast‘; ‘A‘ = an audit that always runs; ‘0‘ = any other step), a work folder, the output files, the reports and the command. The next cell reads the table, writes every step with its time and a short form of its command to the data file ‘Revision/textbook/data/23a_gate_steps.csv‘, prints the long steps and the audits, and adds up the expected time of the full gate and of ‘--fast‘.

In [9]:

```
gate = repository_file("Revision/verify_revision.sh").read_text(encoding="utf-8")
table = gate.split("REVISION_GATE_STEPS")[1]  # the text between the two marks
STEPS = [line.split("|") for line in table.splitlines() if line.count("|") == 6]


def short_command(command):
    """What a step runs: the script file for python and wolframscript, else the
    first two words of the command that are not options."""
    words = [word.strip("{}").rsplit("/", 1)[-1] for word in command.split()
             if not word.startswith("-")]
    if words[0] in ("python", "wolframscript"):
        return next(word for word in words[1:] if "." in word)
    return " ".join(words[:2])


GATE_TABLE = "Revision/textbook/data/23a_gate_steps.csv"
with output_file(GATE_TABLE).open("w", encoding="utf-8", newline="") as handle:
    writer = csv.writer(handle, lineterminator="\n")
```

```
    writer.writerow(["step", "expected_seconds", "kind", "program"])
    for name, expected, kind, _, _, _, command in STEPS:
        mark = {"1": "long", "A": "all"}.get(kind, "")
        writer.writerow([name, int(expected), mark, short_command(command)])
say(f"all {len(STEPS)} steps: {GATE_TABLE}; the long steps and the audits:")
print(f"{"step":33} {"sec":>5} {"kind":4} command")
for name, expected, kind, _, _, _, command in STEPS:
    if kind in ("1", "A"):  # the long steps and the audits that always run
        mark = {"1": "long", "A": "all"}.get(kind, "")
        print(f"{name:33} {int(expected):5d} {mark:4} {short_command(command)}")
FULL = sum(int(step[1]) for step in STEPS)
FAST = sum(int(step[1]) for step in STEPS if step[2] != "1")
report("gate steps", len(STEPS))
report("long steps (skipped by --fast)", sum(step[2] == "1" for step in STEPS))
report("expected wall time of the full gate", f"{FULL} s = {FULL / 3600:.2f} h")
report("expected wall time with --fast", f"{FAST} s = {FAST / 60:.1f} min")
```

Out [9]:

```
all 85 steps: Revision/textbook/data/23a_gate_steps.csv; the long steps and the audits:
step                              sec kind command
gkd-rust-selftest                 900 long gkd_exe gkd-selftest
theory-wolfram                    310 long verify_field_theory.wls
ks-rust-refined                   330 long ks_solver all
ks-rust-determinism                30 long compare_runs.py
committed-unchanged                10 all  audit unchanged
reports-pass                        5 all  audit reports
RESULT gate steps = 85
RESULT long steps (skipped by --fast) = 4
RESULT expected wall time of the full gate = 8057 s = 2.24 h
RESULT expected wall time with --fast = 6487 s = 108.1 min
```

Text cell:

```
The next cell draws the expected wall time of every step of the gate, in the order
in which the gate runs them; the long steps are drawn in a second colour.
```

In [10]:

```
fig, ax = plt.subplots(figsize=(12, 5))
colours = ["#e34948" if step[2] == "1" else "#2a78d6" for step in STEPS]
ax.bar(range(len(STEPS)), [int(step[1]) for step in STEPS], color=colours)
ax.set_yscale("log")
ax.set_xticks(range(len(STEPS)), [step[0] for step in STEPS], rotation=90,
              fontsize=6)
ax.set_xlim(-1, len(STEPS))
ax.set_ylabel("expected wall time (s)")
ax.grid(axis="y", which="both", alpha=0.3)
save_figure(fig, "gate_step_times",
            "The expected wall time in seconds of every step of the Revision gate, "
            "in the order the gate runs them, on a logarithmic vertical axis; red "
```

```
    "bars are the long steps that the option fast skips, blue bars all "
    "others. A few long Wolfram and Kohn-Sham steps take most of the time.")
```

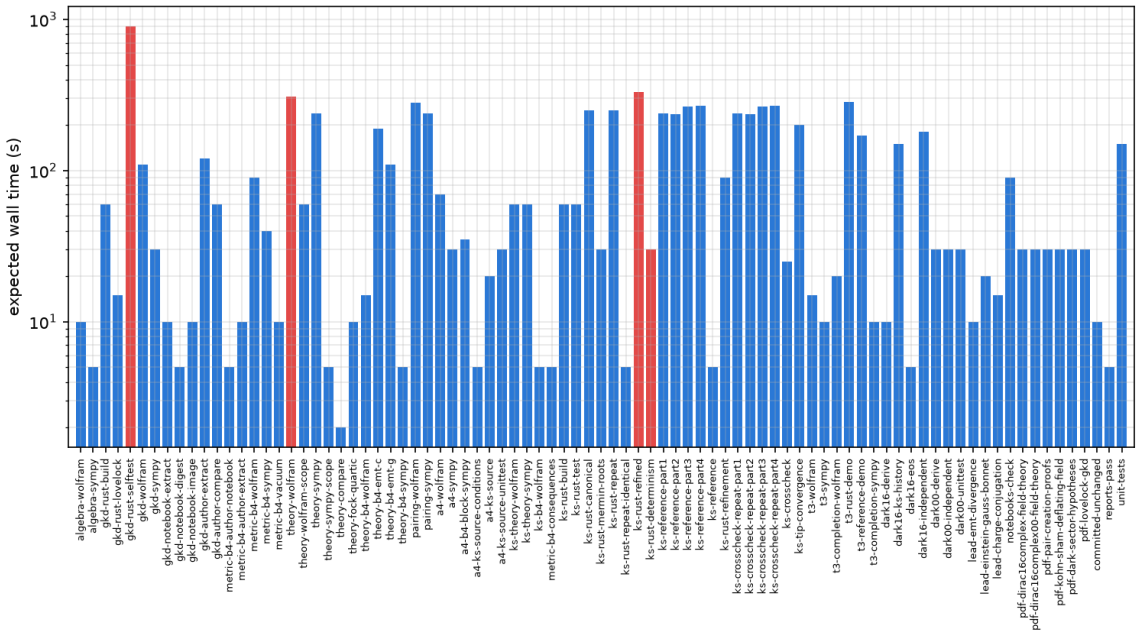

Figure 569: The expected wall time in seconds of every step of the Revision gate, in the order the gate runs them, on a logarithmic vertical axis; red bars are the long steps that the option fast skips, blue bars all others. A few long Wolfram and Kohn-Sham steps take most of the time. (Notebook 23a, figure 6.)

Out [10]:

```
Figure 23a.6 saved as Revision/textbook/figures/23a_6_gate_step_times.png
```

Text cell:

```
## 12. The index of the checks of the Revision record

A report is any JSON file of the Revision record (outside the textbook folder and
the Rust build folders) whose top level holds a list ‘checks‘ of named checks with
verdicts, a table ‘checks‘ of named checks (each ‘true‘ or a table with ‘passed‘ or
a verdict), or, for the GKD self-test, a list ‘results‘ of cases each with a number
of ‘mismatches‘. This is the rule of the gate’s step ‘reports-pass‘. The next cell
defines a function that turns one such file into a list of (check name, verdict),
reads every JSON file of the record, and keeps the reports.
```

In [11]:

```
def check_list(document):
    """The (name, verdict) pairs of a report, or [] if it is not a report."""
```

```
    if not isinstance(document, dict):
        return []
    checks, results = document.get("checks"), document.get("results")
    if isinstance(checks, list):
        return [(item["name"], str(item["verdict"]).upper()) for item in checks]
    if isinstance(checks, dict) and all(isinstance(item, (bool, dict))
                                        for item in checks.values()):
        return [(name, "PASS" if item is True or (isinstance(item, dict) and (
            item.get("passed") is True or
            str(item.get("verdict", "")).upper() == "PASS")) else "FAIL")
            for name, item in checks.items()]
    if isinstance(results, list) and results and all(
            isinstance(item, dict) and "mismatches" in item for item in results):
        return [(", ".join(f"{key}={value}" for key, value in item.items()
                           if key != "mismatches"),
                 "PASS" if item["mismatches"] == 0 else "FAIL") for item in results]
    return []


REPORTS = {}  # report path -> list of (check name, verdict)
for path in by_name(repository_file("Revision").rglob("*.json")):
    relative = path.relative_to(REPO).as_posix()
    if relative.startswith("Revision/textbook/") or "/target/" in relative:
        continue
    pairs = check_list(json.loads(path.read_text(encoding="utf-8")))
    if pairs:
        REPORTS[relative] = pairs
gate_reports = {path for step in STEPS for path in step[5].split(",") if path != "-"}
say(f"{len(REPORTS)} reports found with {sum(map(len, REPORTS.values()))} checks; "
    f"the gate's step table names {len(gate_reports)} reports.")
check(gate_reports <= set(REPORTS),
      "every report named in the gate's step table is in the index")
check(all(verdict in ("PASS", "NOT-AVAILABLE")
          for pairs in REPORTS.values() for _, verdict in pairs),
      "every check of every report is PASS or NOT-AVAILABLE")
```

Out [11]:

```
51 reports found with 1357 checks; the gate's step table names 51 reports.
PASS every report named in the gate's step table is in the index
PASS every check of every report is PASS or NOT-AVAILABLE
```

Text cell:

```
The next cell finds the chapters that cite each check. It cuts every chapter of
chapters 0 to 22 into paragraphs (blocks of lines between empty lines), adds the
output of every code cell of the chapter's notebooks as further blocks, and marks a
check as cited by the chapter when one block contains both the file name of the
check's report and the check's name as a whole word.
```

In [12]:

```
BLOCKS = {}  # chapter number -> its paragraphs and the outputs of its notebooks
for chapter in CHAPTERS:
    number = int(chapter.name[:2])  # the chapter number, e.g. 5
    text = chapter.read_text(encoding="utf-8")
    blocks = [block for block in re.split(r"\n\s*\n", text) if block.strip()]
    for row in ROWS:
        if row["chapter"] == number:
            notebook = json.loads((NOTEBOOKS / f"{row["name"]}.ipynb").read_text(
                encoding="utf-8"))
            for cell in notebook["cells"]:
                outputs = [output.get("text", "") for output in cell.get("outputs", [])]
                blocks.append("".join("".join(item) for item in outputs))
    BLOCKS[number] = blocks

CITED = {}  # (report, check name) -> set of chapter numbers
for number, blocks in BLOCKS.items():
    for block in blocks:
        for report_path, pairs in REPORTS.items():
            if report_path.rsplit("/", 1)[1] not in block:
                continue
            for check_name, _ in pairs:
                word = r"(?<![\w.-])" + re.escape(check_name) + r"(?![\w-])"
                if re.search(word, block):
                    CITED.setdefault((report_path, check_name), set()).add(number)
say(f"{len(CITED)} (report, check) pairs are cited by at least one chapter.")
```

Out [12]:

```
760 (report, check) pairs are cited by at least one chapter.
```

Text cell:

```
The next cell prints one line per report: its path (under ‘Revision/‘, without
‘.json‘), its number of checks, how many are PASS, how many NOT-AVAILABLE, and how
many are cited by at least one chapter. It then writes the complete index, one row
per check, to the data file ‘Revision/textbook/data/23a_check_index.csv‘, reads the
file back and checks that it has one row per check.
```

In [13]:

```
print(f"{"report (under Revision/, without .json)":63} {"checks":>6} {"PASS":>4} "
      f"{"N/A":>3} {"cited":>5}")
INDEX = []  # one row per check: report, check, verdict, citing chapters
for report_path, pairs in REPORTS.items():
    for check_name, verdict in pairs:
        chapters = sorted(CITED.get((report_path, check_name), set()))
        INDEX.append([report_path, check_name, verdict,
                      " ".join(str(chapter) for chapter in chapters)])
    short = report_path.removeprefix("Revision/").removesuffix(".json")
    print(f"{short:63} {len(pairs):6d} "
          f"{sum(verdict == "PASS" for _, verdict in pairs):4d} "
```

```
          f"{sum(verdict == "NOT-AVAILABLE" for _, verdict in pairs):3d} "
          f"{sum((report_path, name) in CITED for name, _ in pairs):5d}")

CHECK_INDEX = "Revision/textbook/data/23a_check_index.csv"
with output_file(CHECK_INDEX).open("w", encoding="utf-8", newline="") as handle:
    writer = csv.writer(handle, lineterminator="\n")
    writer.writerow(["report", "check", "verdict", "cited_by_chapters"])
    writer.writerows(INDEX)
with output_file(CHECK_INDEX).open(encoding="utf-8", newline="") as handle:
    read_back = list(csv.reader(handle))
report("reports", len(REPORTS))
report("checks in the index", len(INDEX))
report("checks cited by at least one chapter", sum(bool(row[3]) for row in INDEX))
check(read_back[1:] == INDEX, "the index file holds exactly one row per check")
```

Out [13]:

```
report (under Revision/, without .json)                          checks PASS N/A cited
algebra/reports/python-algebra                                       35   35   0    35
algebra/reports/wolfram-algebra                                      45   45   0    44
dark_sector/dirac16complex/reports/derivation-checks                 30   30   0    16
dark_sector/dirac16complex/reports/eos-checks                        13   13   0     5
dark_sector/dirac16complex/reports/independent-checks                 9    9   0     2
dark_sector/dirac16complex/reports/ks-history-run                     5    5   0     2
dark_sector/dirac16complex00/reports/python-derive-eos               49   49   0    13
dark_sector/dirac16complex00/reports/python-independent-numerics     28   28   0     1
field_equations_a4/ks_source/reports/ks-source-a4                    23   23   0     7
field_equations_a4/reports/ks-source-conditions                       6    6   0     6
field_equations_a4/reports/python-a4-report                          63   63   0    49
field_equations_a4/reports/python-b4-block                           11   11   0     0
field_equations_a4/reports/wolfram-a4-report                         64   64   0    32
gkd_lovelock/comparison/author-comparison-report                     78   73   5     0
gkd_lovelock/results/gkd-selftest                                     9    9   0     0
gkd_lovelock/results/lovelock-report                                 19   19   0    19
gkd_lovelock/results/python-lovelock-report                          49   49   0    34
gkd_lovelock/results/wolfram-gkd-report                              29   29   0    14
kohn_sham/reports/ks-b4-wolfram                                       4    4   0     0
kohn_sham/reports/ks-crosscheck                                      31   31   0    20
kohn_sham/reports/ks-reference                                       37   37   0    10
kohn_sham/reports/ks-rust-determinism                                14   14   0    12
kohn_sham/reports/ks-rust-mermin-roots                                5    5   0     1
kohn_sham/reports/ks-rust-solver                                     42   42   0    25
kohn_sham/reports/ks-theory-python                                   58   58   0    48
kohn_sham/reports/ks-theory-wolfram                                  46   46   0    10
kohn_sham/tip_convergence/tip-convergence                             7    7   0     1
lead_checks/reports/charge-conjugation-and-u1                        12   12   0    12
lead_checks/reports/einstein-gauss-bonnet-a4                         15   15   0     8
lead_checks/reports/emt-divergence-and-spin-connection               10   10   0     8
metric_b4/results/consequences-b4                                     4    4   0     0
metric_b4/results/python-metric-b4-report                            21   21   0     0
metric_b4/results/vacuum-b4                                           6    6   0     0
metric_b4/results/wolfram-metric-b4-report                           27   27   0     4
pairing/kohn_sham/reports/python-t3-completion                        7    7   0     1
```

```
pairing/kohn_sham/reports/python-t3                              13   13   0    13
pairing/kohn_sham/reports/t3-reference-demo                       7    7   0     0
pairing/kohn_sham/reports/t3-rust-demo                            7    7   0     4
pairing/kohn_sham/reports/wolfram-t3-completion                   3    3   0     1
pairing/kohn_sham/reports/wolfram-t3                             10   10   0    10
pairing/reports/python-pairing                                   66   66   0    61
pairing/reports/wolfram-pairing                                 101  101   0    49
theory/fock_quartic/reports/fock-quartic                         21   21   0     4
theory/reports/python-field-theory-b4                             8    8   0     7
theory/reports/python-field-theory                               70   70   0    61
theory/reports/python-scope                                      14   14   0    14
theory/reports/wolfram-field-theory-b4-emt-C                      4    4   0     0
theory/reports/wolfram-field-theory-b4-emt-G                      4    4   0     0
theory/reports/wolfram-field-theory-b4                           19   19   0    17
theory/reports/wolfram-field-theory                              84   84   0    66
theory/reports/wolfram-scope                                     15   15   0    14
RESULT reports = 51
RESULT checks in the index = 1357
RESULT checks cited by at least one chapter = 760
PASS the index file holds exactly one row per check
```

Text cell:

```
As an example of the index, the next cell prints the rows of one report: the twelve
checks of the charge-conjugation matrices and the local U(1) law,
‘Revision/lead_checks/reports/charge-conjugation-and-u1.json‘, with the chapters
that cite each check.
```

In [14]:

```
EXAMPLE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
print(f"{"check":44} {"verdict":7} cited by chapters")
for report_path, check_name, verdict, chapters in INDEX:
    if report_path == EXAMPLE:
        print(f"{check_name:44} {verdict:7} {chapters or "-"}")
check(sum(row[0] == EXAMPLE for row in INDEX) == 12,
      "the example report holds 12 checks",
      record=f"{EXAMPLE}, all checks")
```

Out [14]:

```
check                                       verdict cited by chapters
representation_real                         PASS    0 5 7 18 21
B_imaginary_hermitian                       PASS    0 5 7 21
intertwiners_same_mass                      PASS    0 5 7 18 21
intertwiners_reversed_mass                  PASS    0 5 7 18 21
charge_conjugation_matrix_plus              PASS    0 5 7 18 20 21 22
charge_conjugation_matrix_minus             PASS    0 5 7 18 20 21 22
majorana_conditions_consistent              PASS    0 5 7 21
bilinears_under_charge_conjugation          PASS    0 5 7 18 21
quantum_charge_conjugation_unitary_type     PASS    0 5 7 10 18 20 21 22
```

```
real_fields_charge_conjugation               PASS    0 5 7 18 20 21 22
spinor_connection_real                       PASS    0 5 7 21
u1_noether_matrix_identity                   PASS    0 5 10 18 20 21 22
PASS the example report holds 12 checks
     reproduces Revision/lead_checks/reports/charge-conjugation-and-u1.json, all checks
```

Text cell:

```
## 13. Re-running every notebook (only with BOOK_RERUN_ALL=1)

The next cell does nothing but print one sentence, unless the environment variable
‘BOOK_RERUN_ALL‘ is ‘1‘. Then it runs ‘nbkit check‘ on the builder of every other
notebook, at most 8 at a time (half the processor threads, when that is fewer),
the slowest first, into a new scratch folder in the system’s temporary folder. It
never changes a file of the repository. It prints, per chapter, how many notebooks
passed and their fresh and recorded seconds, every notebook that failed, the wall
time of the whole re-run, and writes the fresh results to ‘fresh_results.csv‘ in the
scratch folder; it stops with an error if any notebook failed. The figures above
keep showing the RECORDED times. On the development machine (24 threads, so 8 at a
time) this re-run of the 90 notebooks took 406 s of wall time on 2026-10-08, and
all 90 passed; each notebook took longer than its recorded time, because 8 ran at
once and shared the processor.
```

In [15]:

```
RERUN = os.environ.get("BOOK_RERUN_ALL") == "1"  # the switch of this section
if not RERUN:
    say("BOOK_RERUN_ALL is not set to 1, so no other notebook was re-run: every "
        "table and figure above shows the RECORDED results of the provenance files.")
else:
    import subprocess  # starts other programs
    import sys  # sys.executable is the Python program of this notebook
    import tempfile  # makes a new folder in the system’s temporary folder
    import time  # a clock, for the wall time of the re-run
    from concurrent.futures import ThreadPoolExecutor  # several at a time

    scratch = Path(tempfile.mkdtemp(prefix="book_rerun_"))
    environment = {key: value for key, value in os.environ.items()
                   if key not in ("BOOK_RERUN_ALL", "TEXTBOOK_OUTPUT_ROOT")}
    nbkit = str(repository_file("Revision/textbook/tools/nbkit.py"))

    def rerun(row):
        """Run nbkit check on the builder of one notebook; keep its key lines."""
        done = subprocess.run(
            [sys.executable, nbkit, "check", str(repository_file(row["builder"])),
             "--scratch", str(scratch)], capture_output=True, text=True,
            encoding="utf-8", errors="replace", cwd=REPO, env=environment)
        values = dict(line.split("=", 1) for line in done.stdout.splitlines()
                      if "=" in line)
        fresh = values.get(f"check_{row["id"]}_seconds", "")
        return {"id": row["id"], "chapter": row["chapter"],
                "result": values.get(f"check_{row["id"]}", "FAILED"),
```

```
                "seconds": float(fresh) if fresh[:1].isdigit() else 0.0,
                "recorded": row["seconds"]}

    workers = max(1, min(8, (os.cpu_count() or 2) // 2))
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=workers) as pool:
        FRESH = list(pool.map(rerun, sorted(ROWS, key=lambda row: -row["seconds"])))
    wall = time.perf_counter() - start
    FRESH.sort(key=lambda item: item["id"])
    print("chapter notebooks passed  fresh s recorded s")
    for chapter in sorted(PER_CHAPTER):
        items = [item for item in FRESH if item["chapter"] == chapter]
        print(f"{chapter:7d} {len(items):9d} "
              f"{sum(item["result"] == "PASSED" for item in items):6d} "
              f"{sum(item["seconds"] for item in items):8.1f} "
              f"{sum(item["recorded"] for item in items):10.1f}")
    failed = [item["id"] for item in FRESH if item["result"] != "PASSED"]
    for item_id in failed:
        say(f"FAILED {item_id}: run its nbkit check alone to see the problem lines")
    with (scratch / "fresh_results.csv").open("w", encoding="utf-8",
                                              newline="") as handle:
        writer = csv.writer(handle, lineterminator="\n")
        writer.writerow(["id", "chapter", "result", "seconds", "recorded"])
        writer.writerows([item[key] for key in ("id", "chapter", "result",
                                                "seconds", "recorded")]
                         for item in FRESH)
    say(f"Re-run of {len(FRESH)} notebooks, {workers} at a time: {wall:.0f} s.")
    print(f"Fresh results: {scratch / "fresh_results.csv"}")
    assert not failed, f"nbkit check failed for {failed}"
    say(f"RERUN all {len(FRESH)} other notebooks passed their nbkit check.")
```

Out [15]:

```
BOOK_RERUN_ALL is not set to 1, so no other notebook was re-run: every table and figure
    above shows the RECORDED results of the provenance files.
```

Text cell:

```
## 14. The last check

The last cell checks that every file this notebook writes exists and prints the
number of checks that passed.
```

In [16]:

```
check(all(output_file(path).is_file() for path in
          [f"{FIGURE_FOLDER}/{name}" for name in CAPTIONS]
          + [NOTEBOOK_TABLE, CHECK_INDEX, GATE_TABLE, CAPTION_FILE]),
      "every file this notebook writes exists")
all_checks_passed()
```

Out [16]:

```
PASS every file this notebook writes exists
ALL 10 CHECKS PASSED (notebook 23a)
```

Text cell:

```
## 15. What this notebook showed

- Every other notebook of the book has a builder and a provenance file, is placed
  exactly once in its own chapter, and its last RECORDED nbkit check passed; the
  table gives, for each, its checks, figures, check date, result, seconds and
  section, and the data file ‘Revision/textbook/data/23a_notebooks.csv‘ holds it.
- The totals per chapter and the six figures show where the checks, the figures
  and the run time of the book are.
- The gate of the Revision record has the steps printed in Section 11, with their
  expected wall times; the full gate and the option ‘--fast‘ are added up.
- Every check of every report of the Revision record is PASS or NOT-AVAILABLE; the
  index of all of them, with the chapters that cite each, is in
  ‘Revision/textbook/data/23a_check_index.csv‘.
- By default nothing was re-run: the results are the RECORDED ones. With
  ‘BOOK_RERUN_ALL=1‘ the last computing section re-runs every other notebook.
```

23.12 Line-by-line walk-through of Notebook 23a

In [1], the set-up cell. It is the set-up cell of every notebook of this book, with the name `"23a"`. Its comment lines repeat the run instructions of Section 23.10. Its code finds the repository folder (`REPO`), decides where files are written (`OUTPUT_ROOT`: the repository, or a scratch folder during an nbkit check), and defines the helpers the later cells use: `repository_file(relative)` gives the path of a repository file for reading; `output_file(relative)` the path at which to write one; `say(text)` prints in lines of at most 89 characters; `save_figure(fig, name, caption)` saves a figure as `Revision/textbook/figures/23a_<k>_<name>.png`, records its caption in the dictionary `CAPTIONS` and the file `CAPTION_FILE`, shows it and prints where it was saved; `check(condition, name, record)` stops the notebook if the condition is false and otherwise prints PASS with the name (and a second line naming the Revision record when `record` is given); `report(label, value, unit)` prints a line RESULT label = value unit; `all_checks_passed()` prints the last line. `FIGURE_FOLDER` is `"Revision/textbook/figures"` and `NOTEBOOK_ID` is `"23a"`. The modules `json`, `os` and `Path` and the plotting module `plt` are imported there.

In [2], the provenance records.

```
import csv  # reads and writes comma-separated tables
import re  # regular expressions: patterns that find pieces of text
```

Two modules of Python's standard library: `csv` reads and writes tables in which each line is a row and the cells are separated by commas; `re` provides **regular expressions**, patterns that describe pieces of text to be found.

```
NOTEBOOKS = repository_file("Revision/textbook/notebooks")  # notebooks and records
RECORD = re.compile(r"nbkit-record (\{.*\}) -->")  # the record line
ALL_LINE = re.compile(r"ALL (\d+) CHECKS PASSED")  # the last line of a notebook
TITLE = re.compile(r"^# Provenance of Notebook \w+: (.+)$", re.MULTILINE)
```

`NOTEBOOKS` is the folder of the notebooks and their provenance files. `re.compile` turns a pattern into an object that can search text. In a pattern, `\{` and `\}` are the braces themselves, `.*` means "any characters", and the round brackets mark the piece to be returned: `RECORD` finds the record line and returns its JSON text; `\d+` means "one or more digits", so `ALL_LINE` finds the line `ALL n CHECKS PASSED` and returns n; `\w+` means "one or more letters or digits", `^` and `$` mean the start and the end of a line (with `re.MULTILINE`, of any line), so `TITLE` returns the title from the first line of a provenance file. The `r` before a string makes Python keep every backslash as it is.

```
def by_name(paths):
    """Sort paths by their repository-relative name, the same on every system."""
    return sorted(paths, key=lambda path: path.relative_to(REPO).as_posix())
```

`by_name` sorts a list of paths by their names relative to the repository, written with forward slashes. `lambda path: ...` is a small function without a name that gives each path its sorting key. Sorting by this text gives the same order on Windows, macOS and Linux, so that the printed tables never depend on the computer.

```
ROWS = []  # one dictionary per notebook
for path in by_name(NOTEBOOKS.glob("*.PROVENANCE.md")):
    name = path.name.removesuffix(".PROVENANCE.md")  # e.g. 00a_check_installation
    if name.startswith(NOTEBOOK_ID):
        continue  # this notebook's own record changes whenever it is built
```

`ROWS` starts as an empty list. The loop visits every file whose name ends in `.PROVENANCE.md`, in name order. `removesuffix` cuts the ending off the name, leaving for example `00a_check_installation`. `continue` jumps to the next file when the name starts with `"23a"`: this notebook's own record changes every time it is built, so it cannot list itself.

```
    text = path.read_text(encoding="utf-8")
    record = json.loads(RECORD.search(text).group(1))
    captions = json.loads(repository_file(
        f"{FIGURE_FOLDER}/{name[:3]}.captions.json").read_text(encoding="utf-8"))
    ROWS.append({"id": name[:3], "chapter": int(name[:2]), "name": name,
                 "title": TITLE.search(text).group(1),
                 "checks": int(ALL_LINE.search(text).group(1)),
                 "figures": len(captions), "figure_files": sorted(captions),
                 "date": record["check"]["date"],
                 "result": record["check"]["result"],
                 "seconds": record["check"]["seconds"],
                 "builder": f"Revision/textbook/notebooks/src/{name}.py"})
```

The file is read as text; `RECORD.search(text).group(1)` is the JSON text of the record line, and `json.loads` turns it into a dictionary. The caption file of the notebook, `Revision/textboo`

k/figures/NNx.captions.json, holds one entry per figure; name[:3] is the id (the first three characters). ROWS.append adds one dictionary per notebook: the id, the chapter number (int turns the text "05" into the number 5), the name, the title, the number of checks, the number of figures and the sorted list of figure files, the date, result and seconds of the last recorded check (record["check"]), and the path of the builder.

```
builders = [path.stem for path in by_name(NOTEBOOKS.glob("src/*.py"))
            if not path.stem.startswith(NOTEBOOK_ID)]
say(f"{len(ROWS)} provenance files read, {len(builders)} builders found.")
```

builders lists the names of the builder files (path.stem is a file name without its ending .py), again without this notebook. The line printed by say is the first line of Out [2]: 90 provenance files and 90 builders.

```
check([row["name"] for row in ROWS] == builders,
      "every builder has a provenance file and every provenance file a builder")
check(all(row["result"] == "passed" for row in ROWS),
      "the last recorded nbkit check of every notebook passed")
check(all(repository_file(f"{FIGURE_FOLDER}/{figure}").is_file()
          for row in ROWS for figure in row["figure_files"]),
      "every figure named in a caption file exists")
```

Three checks. The first compares the two lists of names: every builder has a provenance file and every provenance file a builder, in the same order. The second demands that every recorded check passed (all is true when the condition holds for every row). The third demands that every figure named in a caption file exists. Each prints a PASS line in Out [2].

In [3], where each notebook is printed.

```
CHAPTERS = [path for path in by_name(repository_file("Revision/textbook/chapters")
                                     .glob("[0-9][0-9]-*.md"))
            if int(path.name[:2]) <= 22]  # the chapter files 00 to 22
HEADING = re.compile(r"^### (\d+)\.(\d+) ")  # a section heading such as ### 3.4
MARKER = re.compile(r"^<!-{2} NOTEBOOK (\w+) -->$")  # a notebook marker line
PLACED = {}  # notebook id -> list of (chapter file name, section of its instructions)
```

CHAPTERS is the list of the chapter files of chapters 0 to 22 ([0-9][0-9]-*.md matches every file name that starts with two digits and a hyphen and ends in .md; the condition int(path.name[:2]) <= 22 leaves out this chapter 23, so that its own text cannot change the results). HEADING finds a section heading such as ### 3.4 and returns its two numbers. MARKER finds a notebook marker line and returns the id; -{2} means "the character - two times", which lets the cell describe the marker without containing one. PLACED will hold, for each notebook id, the list of places where it is placed.

```
for chapter in CHAPTERS:
    section = None
    for line in chapter.read_text(encoding="utf-8").splitlines():
        if HEADING.match(line):
            section = HEADING.match(line).groups()  # e.g. ("3", "4")
```

```
        elif MARKER.match(line):
            number = f"{section[0]}.{int(section[1]) + 1}"  # the next section
            PLACED.setdefault(MARKER.match(line).group(1), []).append(
                (chapter.name, number))
```

For each chapter, `section` remembers the last heading seen. Each line is tested: a heading updates `section` (`.groups()` returns the two numbers as texts); a marker records the chapter's file name and the section number one higher than the last heading, `N.(M+1)`, which is the number the assembler gives the notebook's run-instruction section. `setdefault` creates an empty list for an id seen for the first time.

```
for row in ROWS:
    places = PLACED.get(row["id"], [])
    row["section"] = places[0][1] if len(places) == 1 else "?"
say(f"{len(CHAPTERS)} chapter files read, {len(PLACED)} notebook markers found.")
check(all(len(PLACED.get(row["id"], [])) == 1
          and PLACED[row["id"]][0][0].startswith(row["id"][:2] + "-")
          for row in ROWS),
      "every notebook is placed exactly once, in the chapter of its number")
```

Each row receives its section, or `"?"` when the notebook is not placed exactly once. The printed line is Out [3]: 23 chapter files and 90 markers. The check demands that each notebook has exactly one marker and that the marker is in the chapter file whose name starts with the notebook's own two digits.

In [4], the table, chapters 0 to 10.

```
def show_table(first, last):
    print("id  title                                checks figs checked    result "
          "   sec section")
    for row in ROWS:
        if first <= row["chapter"] <= last:
            title = row["title"]
            if len(title) > 34:
                title = title[:31] + "..."
            print(f"{row["id"]:3} {title:34} {row["checks"]:6d} {row["figures"]:4d} "
                  f"{row["date"]:10} {row["result"]:6} {row["seconds"]:6.1f} "
                  f"{row["section"]:>7}")
```

`show_table(first, last)` prints a header line and then one line per notebook of the chapters `first` to `last`. A title longer than 34 characters is cut to 31 characters and three dots. In an **f-string** (a string with `f` before it) every `{...}` is replaced by the value inside; after the colon, `:3` and `:34` fill the value to that width, `:6d` writes a whole number in 6 places, `:6.1f` a number with one decimal in 6 places, and `:>7` puts the text at the right end of 7 places.

```
show_table(0, 10)
```

prints the first part of the table, Out [4]: the 50 notebooks of chapters 0 to 10. The last column is the section of the run instructions; for example Notebook 05c is placed in Section 5.31, and its complete text is Section 5.32.

In [5], the table, chapters 11 to 22, and the data file.

```
show_table(11, 22)
COLUMNS = ["id", "chapter", "title", "checks", "figures", "date", "result",
           "seconds", "section", "builder"]
NOTEBOOK_TABLE = "Revision/textbook/data/23a_notebooks.csv"
```

The second part of the table is printed (Out [5], 40 notebooks). `COLUMNS` names the columns of the data file, and `NOTEBOOK_TABLE` its path.

```
with output_file(NOTEBOOK_TABLE).open("w", encoding="utf-8", newline="") as handle:
    writer = csv.writer(handle, lineterminator="\n")
    writer.writerow(COLUMNS)
    writer.writerows([row[column] for column in COLUMNS] for row in ROWS)
say(f"The table of {len(ROWS)} notebooks was written to {NOTEBOOK_TABLE}.")
```

`with ... as handle` opens the file for writing (`"w"`) and closes it at the end of the indented block; `newline=""` and `lineterminator="\n"` make every line end with a single line-feed character on every system. `writer.writerow` writes the header and `writer.writerows` one row per notebook, each the list of the values of the columns. The last line prints where the table was written.

In [6], the totals per chapter.

```
PER_CHAPTER = {}  # chapter number -> its totals
for row in ROWS:
    total = PER_CHAPTER.setdefault(row["chapter"], {"notebooks": 0, "checks": 0,
                                                    "figures": 0, "seconds": 0.0})
    total["notebooks"] += 1
    total["checks"] += row["checks"]
    total["figures"] += row["figures"]
    total["seconds"] += row["seconds"]
```

`PER_CHAPTER` gets, for each chapter, a dictionary of four totals, created with zeros the first time the chapter appears (`setdefault`). `+=` adds the row's value to the total.

```
print("chapter notebooks checks figures  seconds")
for chapter, total in sorted(PER_CHAPTER.items()):
    print(f"{chapter:7d} {total["notebooks"]:9d} {total["checks"]:6d} "
          f"{total["figures"]:7d} {total["seconds"]:8.1f}")
```

The header and one line per chapter, in chapter order (`sorted` sorts the pairs of the dictionary by the chapter number). This is the table of Out [6].

```
ALL_SECONDS = sum(row["seconds"] for row in ROWS)
report("notebooks (without 23a)", len(ROWS))
report("checks in these notebooks", sum(row["checks"] for row in ROWS))
report("figures of these notebooks", sum(row["figures"] for row in ROWS))
report("recorded check time, one after the other", f"{ALL_SECONDS:.1f}", "s")
check(sorted(PER_CHAPTER) == list(range(23)),
      "every chapter 0 to 22 has at least one notebook")
```

`ALL_SECONDS` is the sum of all recorded check times. The four RESULT lines of Out [6]: 90 notebooks, 2244 checks, 563 figures and 1097.9 s, the time of all checks one after the other. The check demands that the chapter numbers are exactly 0 to 22 (`list(range(23))` is the list 0, 1, ..., 22).

In [7], two figures of the run times.

```
ids = [row["id"] for row in ROWS]
seconds = [row["seconds"] for row in ROWS]
fig, ax = plt.subplots(figsize=(12, 4.5))
ax.bar(range(len(ROWS)), seconds, color="#2a78d6")
ax.set_yscale("log")  # equal steps on the axis are equal factors
ax.set_xticks(range(len(ROWS)), ids, rotation=90, fontsize=6)
ax.set_xlim(-1, len(ROWS))
ax.set_xlabel("notebook")
ax.set_ylabel("recorded nbkit check time (s)")
ax.grid(axis="y", which="both", alpha=0.3)
```

`ids` and `seconds` are the ids and recorded times in book order. `plt.subplots(figsize=(12, 4.5))` makes a figure 12 inches wide and 4.5 inches high with one drawing area `ax`. `ax.bar` draws one bar per notebook at the positions 0, 1, 2, ...; `set_yscale("log")` makes the vertical axis logarithmic, so that 1 s, 10 s and 100 s are equally far apart; `set_xticks` writes the ids under the bars, turned by 90 degrees; `set_xlim` leaves half a bar of space at both ends; the two labels name the axes, and `grid` draws light horizontal lines.

```
save_figure(fig, "check_time_per_notebook",
            "The wall time in seconds of the last recorded nbkit check of every "
            "notebook of chapters 0 to 22 (one bar each, in book order), on a "
            "logarithmic vertical axis. Most notebooks take a few seconds; a few, "
            "which run the Rust programs or long numerical scans, take minutes.")
```

`save_figure` saves the figure as Figure 23a.1 with the caption given. It shows that most notebooks take a few seconds and a few take minutes; the tallest bar is Notebook 11a.

```
slowest = sorted(seconds, reverse=True)
shares = [100 * sum(slowest[:k]) / ALL_SECONDS for k in range(len(slowest) + 1)]
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(range(len(shares)), shares, color="#eb6834", marker="o", markersize=3)
ax.set_xlabel("number of notebooks, slowest first")
ax.set_ylabel("share of the total check time (%)")
ax.grid(alpha=0.3)
```

`slowest` is the list of times sorted from the largest down (`reverse=True`). `shares[k]` is the percentage of the total time taken by the k slowest notebooks: `slowest[:k]` is the list of the first k entries, `sum` adds them, and the result is divided by `ALL_SECONDS` and multiplied by 100; `k` runs from 0 to 90, so `shares[0]` is 0 and `shares[90]` is 100. `ax.plot` draws the points joined by lines, with small circles (`marker="o"`).

```
save_figure(fig, "cumulative_check_time",
...)
report("share of the three slowest notebooks", f"{shares[3]:.1f}", "%")
```

The figure is saved as Figure 23a.2, and the RESULT line of Out [7] gives `shares[3]`: the three slowest notebooks take 22.2 % of the whole time.

In [8], checks, figures and time per chapter.

```
chapters = sorted(PER_CHAPTER)
for key, name, label, colour, caption in [
        ("checks", "checks_per_chapter", "checks", "#1baf7a",
...),
        ("figures", "figures_per_chapter", "figures", "#eda100",
...),
        ("seconds", "check_time_per_chapter", "recorded check time (s)", "#4a3aa7",
...)
    fig, ax = plt.subplots(figsize=(8, 3.6))
    ax.bar(chapters, [PER_CHAPTER[chapter][key] for chapter in chapters],
           color=colour)
    ax.set_xticks(chapters)
    ax.set_xlabel("chapter")
    ax.set_ylabel(label)
    ax.grid(axis="y", alpha=0.3)
    save_figure(fig, name, caption)
```

`chapters` is the sorted list of chapter numbers. The loop goes through a list of three groups of five values; in each pass `key` is the total to draw (`"checks"`, `"figures"` or `"seconds"`), `name` the figure's file name, `label` the text of the vertical axis, `colour` the colour of the bars and `caption` the caption (the three captions are printed with the figures). Each pass draws one bar per chapter of the chosen total, writes every chapter number under the axis, labels the axes and saves the figure: Figures 23a.3, 23a.4 and 23a.5.

In [9], the steps of the gate.

```
gate = repository_file("Revision/verify_revision.sh").read_text(encoding="utf-8")
table = gate.split("REVISION_GATE_STEPS")[1]  # the text between the two marks
STEPS = [line.split("|") for line in table.splitlines() if line.count("|") == 6]
```

The gate script is read as text. `gate.split("REVISION_GATE_STEPS")` cuts it at every appearance of that word; the piece with index 1 is the text between the first and the second appearance, which is the step table. `STEPS` keeps every line of it that has exactly six `|` characters, that is seven fields, split into a list of the seven fields.

```
def short_command(command):
    """What a step runs: the script file for python and wolframscript, else the
    first two words of the command that are not options."""
    words = [word.strip("{}").rsplit("/", 1)[-1] for word in command.split()
             if not word.startswith("-")]
    if words[0] in ("python", "wolframscript"):
        return next(word for word in words[1:] if "." in word)
    return " ".join(words[:2])
```

`short_command` makes a short form of a step's command for the table. It splits the command into words (`split()`), leaves out the options (words that start with `-`), removes the braces of

the placeholders such as {python}, and keeps of every path only the part after the last /. For a command run by python or wolframscript it returns the first later word with a . in it, the script file; next returns the first item of such a sequence. For any other command it returns the first two words, such as cargo build.

```
GATE_TABLE = "Revision/textbook/data/23a_gate_steps.csv"
with output_file(GATE_TABLE).open("w", encoding="utf-8", newline="") as handle:
    writer = csv.writer(handle, lineterminator="\n")
    writer.writerow(["step", "expected_seconds", "kind", "program"])
    for name, expected, kind, _, _, _, command in STEPS:
        mark = {"1": "long", "A": "all"}.get(kind, "")
        writer.writerow([name, int(expected), mark, short_command(command)])
```

The complete step table is written to the data file Revision/textbook/data/23a_gate_steps.csv, one row per step, with the header step, expected_seconds, kind, program: the seven fields are unpacked into seven names (_ marks fields not used here); mark is long for kind 1, all for kind A (the audits that always run) and empty otherwise. csv.writer writes the rows, as in In [5].

```
say(f"all {len(STEPS)} steps: {GATE_TABLE}; the long steps and the audits:")
print(f"{"step":33} {"sec":>5} {"kind":4} command")
for name, expected, kind, _, _, _, command in STEPS:
    if kind in ("1", "A"):  # the long steps and the audits that always run
        mark = {"1": "long", "A": "all"}.get(kind, "")
        print(f"{name:33} {int(expected):5d} {mark:4} {short_command(command)}")
```

One printed line names the data file. Then the header and one line for each long step and each audit: printing all 85 steps would make the output of the cell longer than the 80 lines a cell of this book may print. This is the table of Out [9].

```
FULL = sum(int(step[1]) for step in STEPS)
FAST = sum(int(step[1]) for step in STEPS if step[2] != "1")
report("gate steps", len(STEPS))
report("long steps (skipped by --fast)", sum(step[2] == "1" for step in STEPS))
report("expected wall time of the full gate", f"{FULL} s = {FULL / 3600:.2f} h")
report("expected wall time with --fast", f"{FAST} s = {FAST / 60:.1f} min")
```

FULL adds up the expected seconds of all steps, FAST those of the steps that are not long. The RESULT lines of Out [9]: 85 steps, 4 of them long, 8057 s (2.24 h) for the full gate and 6487 s (108.1 min) with the fast option.

In [10], the figure of the gate.

```
fig, ax = plt.subplots(figsize=(12, 5))
colours = ["#e34948" if step[2] == "1" else "#2a78d6" for step in STEPS]
ax.bar(range(len(STEPS)), [int(step[1]) for step in STEPS], color=colours)
ax.set_yscale("log")
ax.set_xticks(range(len(STEPS)), [step[0] for step in STEPS], rotation=90,
              fontsize=6)
ax.set_xlim(-1, len(STEPS))
ax.set_ylabel("expected wall time (s)")
ax.grid(axis="y", which="both", alpha=0.3)
```

`colours` gives each bar a colour: red (`"#e34948"`) for a long step, blue otherwise; the expression `A if condition else B` chooses between two values. The bars are the expected seconds on a logarithmic axis, with the step names written under them.

```
save_figure(fig, "gate_step_times",
...)
```

saves Figure 23a.6.

In [11], the reports of the Revision record.

```
def check_list(document):
    """The (name, verdict) pairs of a report, or [] if it is not a report."""
    if not isinstance(document, dict):
        return []
    checks, results = document.get("checks"), document.get("results")
    if isinstance(checks, list):
        return [(item["name"], str(item["verdict"]).upper()) for item in checks]
```

`check_list` turns a JSON document into a list of pairs (check name, verdict). `isinstance(document, dict)` asks whether the document is a dictionary; if not, it is not a report and the empty list is returned. `document.get("checks")` is the value stored under `checks`, or `None` if there is none. When `checks` is a list, each item is a dictionary with a `name` and a `verdict`; `str(...).upper()` writes the verdict in capital letters, so that `pass` and `PASS` count alike.

```
    if isinstance(checks, dict) and all(isinstance(item, (bool, dict))
                                        for item in checks.values()):
        return [(name, "PASS" if item is True or (isinstance(item, dict) and (
            item.get("passed") is True or
            str(item.get("verdict", "")).upper() == "PASS")) else "FAIL")
            for name, item in checks.items()]
```

When `checks` is a dictionary whose values are all `True`/`False` or dictionaries, each entry is a named check: it is PASS when its value is `True`, or a dictionary with `passed` equal to `True` or with the verdict PASS; otherwise FAIL. (A dictionary of plain numbers, such as a summary `{"total": 13, "pass": 13}`, is not a list of checks and is skipped.)

```
    if isinstance(results, list) and results and all(
            isinstance(item, dict) and "mismatches" in item for item in results):
        return [(", ".join(f"{key}={value}" for key, value in item.items()
                           if key != "mismatches"),
                 "PASS" if item["mismatches"] == 0 else "FAIL") for item in results]
    return []
```

The GKD self-test stores a list `results` of cases, each with a number of `mismatches`. Each case becomes a check named by its other entries (for example `p=1, mode=exhaustive, pairs=64`), PASS when it has no mismatch. Any other document gives the empty list.

```
REPORTS = {}  # report path -> list of (check name, verdict)
for path in by_name(repository_file("Revision").rglob("*.json")):
    relative = path.relative_to(REPO).as_posix()
    if relative.startswith("Revision/textbook/") or "/target/" in relative:
        continue
    pairs = check_list(json.loads(path.read_text(encoding="utf-8")))
    if pairs:
        REPORTS[relative] = pairs
```

`rglob("*.json")` finds every JSON file in the folder `Revision` and all folders below it. Files of the textbook and of the Rust build folders (`target`) are skipped. Each file is read and turned into its list of checks; a file with at least one check is kept in `REPORTS`, under its repository path.

```
gate_reports = {path for step in STEPS for path in step[5].split(",") if path != "-"}
say(f"{len(REPORTS)} reports found with {sum(map(len, REPORTS.values()))} checks; "
    f"the gate's step table names {len(gate_reports)} reports.")
check(gate_reports <= set(REPORTS),
      "every report named in the gate's step table is in the index")
check(all(verdict in ("PASS", "NOT-AVAILABLE")
          for pairs in REPORTS.values() for _, verdict in pairs),
      "every check of every report is PASS or NOT-AVAILABLE")
```

`gate_reports` is the set of the report paths named in the sixth field of the gate's steps (`"-"` means none). The printed line of Out [11]: 51 reports with 1357 checks, and the gate names 51 reports. The first check demands that every report of the gate is in the index (`<=` between two sets means "is contained in"); the second, that every verdict is PASS or NOT-AVAILABLE. NOT-AVAILABLE occurs only in the comparison with the author's notebook, for 5 outputs that the author's notebook does not store; this is the same rule as the gate's step `reports-pass`.

In [12], the chapters that cite each check.

```
BLOCKS = {}  # chapter number -> its paragraphs and the outputs of its notebooks
for chapter in CHAPTERS:
    number = int(chapter.name[:2])  # the chapter number, e.g. 5
    text = chapter.read_text(encoding="utf-8")
    blocks = [block for block in re.split(r"\n\s*\n", text) if block.strip()]
```

For each chapter of 0 to 22, `BLOCKS` will hold a list of text blocks. `re.split(r"\n\s*\n", text)` cuts the chapter at every empty line (a line break, possibly blanks, another line break), giving its paragraphs; empty pieces are dropped.

```
    for row in ROWS:
        if row["chapter"] == number:
            notebook = json.loads((NOTEBOOKS / f"{row["name"]}.ipynb").read_text(
                encoding="utf-8"))
            for cell in notebook["cells"]:
                outputs = [output.get("text", "") for output in cell.get("outputs", [])]
                blocks.append("".join("".join(item) for item in outputs))
    BLOCKS[number] = blocks
```

For each notebook of the chapter, the stored notebook file is read as JSON. A notebook is a dictionary whose entry `cells` is the list of its cells; a code cell has a list `outputs`, and a printed output has its text under `text` (a list of lines). The printed text of all outputs of one cell is joined into one block and added. `BLOCKS[number]` stores the chapter's blocks.

```
CITED = {}  # (report, check name) -> set of chapter numbers
for number, blocks in BLOCKS.items():
    for block in blocks:
        for report_path, pairs in REPORTS.items():
            if report_path.rsplit("/", 1)[1] not in block:
                continue
            for check_name, _ in pairs:
                word = r"(?<![\w.-])" + re.escape(check_name) + r"(?![\w-])"
                if re.search(word, block):
                    CITED.setdefault((report_path, check_name), set()).add(number)
say(f"{len(CITED)} (report, check) pairs are cited by at least one chapter.")
```

For every block, every report whose file name (the part of its path after the last `/`) occurs in the block is examined, and each of its check names is searched for as a whole word: `re.escape` makes any special character of the name an ordinary character, and the two conditions `(?<![\w.-])` and `(?![\w-])` demand that the name is not part of a longer word (no letter, digit, `_`, `.` or `-` just before it, no letter, digit, `_` or `-` just after it). Each hit adds the chapter number to the set of the pair (report, check) in `CITED`. Out [12]: 760 pairs are cited at least once.

In [13], the index of the checks.

```
print(f"{"report (under Revision/, without .json)":63} {"checks":>6} {"PASS":>4} "
      f"{"N/A":>3} {"cited":>5}")
INDEX = []  # one row per check: report, check, verdict, citing chapters
for report_path, pairs in REPORTS.items():
    for check_name, verdict in pairs:
        chapters = sorted(CITED.get((report_path, check_name), set()))
        INDEX.append([report_path, check_name, verdict,
                      " ".join(str(chapter) for chapter in chapters)])
    short = report_path.removeprefix("Revision/").removesuffix(".json")
    print(f"{short:63} {len(pairs):6d} "
          f"{sum(verdict == "PASS" for _, verdict in pairs):4d} "
          f"{sum(verdict == "NOT-AVAILABLE" for _, verdict in pairs):3d} "
          f"{sum((report_path, name) in CITED for name, _ in pairs):5d}")
```

The header, then for each report: one row per check is added to `INDEX`, with the citing chapters written as numbers separated by blanks; `short` is the report path without `Revision/` at the start (`removeprefix`) and without `.json` at the end; the printed line gives the numbers of checks, of PASS, of NOT-AVAILABLE and of cited checks (`sum` of true conditions counts them). This is the table of Out [13].

```
CHECK_INDEX = "Revision/textbook/data/23a_check_index.csv"
with output_file(CHECK_INDEX).open("w", encoding="utf-8", newline="") as handle:
    writer = csv.writer(handle, lineterminator="\n")
    writer.writerow(["report", "check", "verdict", "cited_by_chapters"])
```

```
    writer.writerows(INDEX)
with output_file(CHECK_INDEX).open(encoding="utf-8", newline="") as handle:
    read_back = list(csv.reader(handle))
report("reports", len(REPORTS))
report("checks in the index", len(INDEX))
report("checks cited by at least one chapter", sum(bool(row[3]) for row in INDEX))
check(read_back[1:] == INDEX, "the index file holds exactly one row per check")
```

The index is written to the data file `Revision/textbook/data/23a_check_index.csv` with the header `report, check, verdict, cited_by_chapters`, then read back with `csv.reader`. The RESULT lines of Out [13]: 51 reports, 1357 checks, 760 of them cited by at least one chapter. The check demands that the rows read back, after the header (`read_back[1:]`), are exactly the rows written.

In [14], one report as an example.

```
EXAMPLE = "Revision/lead_checks/reports/charge-conjugation-and-u1.json"
print(f"{"check":44} {"verdict":7} cited by chapters")
for report_path, check_name, verdict, chapters in INDEX:
    if report_path == EXAMPLE:
        print(f"{check_name:44} {verdict:7} {chapters or "-"}")
check(sum(row[0] == EXAMPLE for row in INDEX) == 12,
      "the example report holds 12 checks",
      record=f"{EXAMPLE}, all checks")
```

The rows of the report of the charge-conjugation matrices and the local U(1) law are printed with their citing chapters (`chapters or "-"` prints a dash when no chapter cites the check). Out [14] shows its 12 checks, all PASS, each cited by at least four chapters; the check `u1_noether_matrix_identity`, the exact local conservation law of Chapter 21, is cited by chapters 0, 5, 10, 18, 20, 21 and 22. The check demands that the report holds 12 checks, and its second line names the record it reproduces.

In [15], the re-run of every notebook.

```
RERUN = os.environ.get("BOOK_RERUN_ALL") == "1"  # the switch of this section
if not RERUN:
    say("BOOK_RERUN_ALL is not set to 1, so no other notebook was re-run: every "
        "table and figure above shows the RECORDED results of the provenance files.")
```

`os.environ.get("BOOK_RERUN_ALL")` is the value of the environment variable, or `None` when it is not set; `RERUN` is true only when the value is the text `"1"`. When it is not, the cell prints the sentence of Out [15] and does nothing else; this is what happened when the stored notebook was built.

```
else:
    import subprocess  # starts other programs
    import sys  # sys.executable is the Python program of this notebook
    import tempfile  # makes a new folder in the system's temporary folder
    import time  # a clock, for the wall time of the re-run
```

```
from concurrent.futures import ThreadPoolExecutor  # several at a time

scratch = Path(tempfile.mkdtemp(prefix="book_rerun_"))
environment = {key: value for key, value in os.environ.items()
               if key not in ("BOOK_RERUN_ALL", "TEXTBOOK_OUTPUT_ROOT")}
nbkit = str(repository_file("Revision/textbook/tools/nbkit.py"))
```

With `BOOK_RERUN_ALL=1` the cell imports four more modules: `subprocess` starts other programs, `sys.executable` is the Python program that runs this notebook, `tempfile.mkdtemp` makes a new, empty folder in the system's temporary folder (its name starts with `book_rerun_`), `time.perf_counter` is a clock. `ThreadPoolExecutor` runs a function on several items at the same time. `environment` is a copy of the environment variables without `BOOK_RERUN_ALL` (so that the notebooks started now do not start the re-run themselves) and without the output folder of an nbkit check. `nbkit` is the path of the tool.

```
def rerun(row):
    done = subprocess.run(
        [sys.executable, nbkit, "check", str(repository_file(row["builder"])),
         "--scratch", str(scratch)], capture_output=True, text=True,
        encoding="utf-8", errors="replace", cwd=REPO, env=environment)
    values = dict(line.split("=", 1) for line in done.stdout.splitlines()
                  if "=" in line)
    fresh = values.get(f"check_{row["id"]}_seconds", "")
    return {"id": row["id"], "chapter": row["chapter"],
            "result": values.get(f"check_{row["id"]}", "FAILED"),
            "seconds": float(fresh) if fresh[:1].isdigit() else 0.0,
            "recorded": row["seconds"]}
```

`rerun(row)` runs the nbkit check of one notebook's builder, with the new scratch folder as its scratch folder (the command of Section 23.6), from the repository root (`cwd=REPO`), keeping its printed output (`capture_output=True`, as text). Every printed line of nbkit has the form `key=value`; `line.split("=", 1)` cuts it at the first `=`, and `dict(...)` collects the pairs. The function returns the id, the chapter, the result (`FAILED` if nbkit printed none), the fresh seconds (0 when nbkit printed no number) and the recorded seconds.

```
workers = max(1, min(8, (os.cpu_count() or 2) // 2))
start = time.perf_counter()
with ThreadPoolExecutor(max_workers=workers) as pool:
    FRESH = list(pool.map(rerun, sorted(ROWS, key=lambda row: -row["seconds"])))
wall = time.perf_counter() - start
FRESH.sort(key=lambda item: item["id"])
```

`workers` is half the number of processor threads, at least 1 and at most 8. The notebooks are handed to the pool slowest first (sorted by minus their recorded time), so that the longest ones start at once and the short ones fill the gaps. `pool.map` returns the results in the order of the input; afterwards they are sorted by id. `wall` is the wall time of the whole re-run.

```
print("chapter notebooks passed  fresh s recorded s")
for chapter in sorted(PER_CHAPTER):
```

```
        items = [item for item in FRESH if item["chapter"] == chapter]
        print(f"{chapter:7d} {len(items):9d} "
              f"{sum(item["result"] == "PASSED" for item in items):6d} "
              f"{sum(item["seconds"] for item in items):8.1f} "
              f"{sum(item["recorded"] for item in items):10.1f}")
```

One line per chapter: the number of notebooks, how many passed, and their fresh and recorded seconds added up.

```
    failed = [item["id"] for item in FRESH if item["result"] != "PASSED"]
    for item_id in failed:
        say(f"FAILED {item_id}: run its nbkit check alone to see the problem lines")
    with (scratch / "fresh_results.csv").open("w", encoding="utf-8",
                                              newline="") as handle:
        writer = csv.writer(handle, lineterminator="\n")
        writer.writerow(["id", "chapter", "result", "seconds", "recorded"])
        writer.writerows([item[key] for key in ("id", "chapter", "result",
                                                "seconds", "recorded")]
                         for item in FRESH)
    say(f"Re-run of {len(FRESH)} notebooks, {workers} at a time: {wall:.0f} s.")
    print(f"Fresh results: {scratch / "fresh_results.csv"}")
    assert not failed, f"nbkit check failed for {failed}"
    say(f"RERUN all {len(FRESH)} other notebooks passed their nbkit check.")
```

`failed` lists the ids of the notebooks that did not pass; each is named on a line. The fresh results are written to `fresh_results.csv` in the scratch folder (never into the repository). The cell prints the number of notebooks, the number run at a time and the wall time, and on a line of its own (with `print`, so that the path is never broken) where the CSV file is; then it stops with an error message (`assert`) if any notebook failed; otherwise it prints that all passed. The figures of the notebook keep showing the RECORDED times. On the development machine this cell, run once with `BOOK_RERUN_ALL=1` on 2026-10-08, printed all 90 notebooks as passed and a wall time of 406 s.

In [16], the last check.

```
check(all(output_file(path).is_file() for path in
          [f"{FIGURE_FOLDER}/{name}" for name in CAPTIONS]
          + [NOTEBOOK_TABLE, CHECK_INDEX, GATE_TABLE, CAPTION_FILE]),
      "every file this notebook writes exists")
all_checks_passed()
```

The check demands that every file this notebook writes exists: the six figures (`CAPTIONS` holds their names), the two data files and the caption file. `all_checks_passed()` prints the last line, `ALL 10 CHECKS PASSED (notebook 23a)`.

23.13 The index of the checks of the Revision record

The index of the checks is the file `Revision/textbook/data/23a_check_index.csv`, written by In [13] of Notebook 23a: one row for each of the 1357 checks of the 51 reports of the Revision record, with the report's path, the check's name, its verdict and the chapters (of 0 to 22) that cite it. Out

[13] in Section 23.11 is its summary, one line per report, and Out [14] shows the rows of one report in full. To find a check by name, open the file in any spreadsheet program, or search it in the terminal:

Windows (PowerShell):

```
Select-String -Path Revision/textbook/data/23a_check_index.csv -Pattern u1_noether
```

macOS (zsh) and Linux (bash):

```
grep u1_noether Revision/textbook/data/23a_check_index.csv
```

Both find the row `Revision/lead_checks/reports/charge-conjugation-and-u1.json,u1_noether_matrix_identity,PASS,0 5 10 18 20 21 22`. grep prints the row alone; Select-String prints in front of it the path of the file and the number of the line in the file, each followed by a colon. A check that no chapter cites (597 of the 1357) is still part of the record and still verified by the gate; the book teaches the results it needs and cites the checks that establish them, and the other checks guard the computations from which those results come (inputs read correctly, intermediate identities, the Rust programs' own tests).

23.14 The index of the notebooks

The index of the notebooks is the table of Out [4] and Out [5] in Section 23.11, also written to `Revision/textbook/data/23a_notebooks.csv`: for each notebook its id, title, number of checks, number of figures, last recorded check, and in the last column the section that holds its run instructions; its complete text is the section after it, and its line-by-line walk-through the section after that. Notebook 23a itself is printed in Sections 23.10 and 23.11 and explained in Section 23.12. All 90 recorded checks of the other notebooks are dated 2026-10-08 and passed.

23.15 What we proved, what we computed, what we assumed

- PROVED: nothing new about physics; this chapter is bookkeeping. Notebook 23a checks facts about files: every builder has a provenance file, every notebook is placed exactly once in its own chapter, every recorded nbkit check passed, every report named by the gate is in the index, and every one of the 1357 checks of the 51 reports is PASS or NOT-AVAILABLE (the 5 NOT-AVAILABLE checks are outputs that the author's notebook does not store).
- COMPUTED (counted from files on 2026-10-15): 90 notebooks in chapters 0 to 22 with 2244 checks and 563 figures; their recorded checks take 1097.9 s one after the other, and the three slowest take 22.2 % of it; the gate has 85 steps, 4 of them long, with expected wall times of 8057 s in full and 6487 s with the fast option; 760 checks of the record are cited by at least one chapter. MEASURED once with `BOOK_RERUN_ALL=1` on 2026-10-08: all 90 other notebooks passed a fresh nbkit check, 8 at a time, in 406 s of wall time; the fresh time of each chapter was between 0.89 and 1.50 times its recorded time (chapter 8: 129.9 s against 86.5 s), because 8 notebooks shared the processor.

- MEASURED elsewhere and quoted: the recorded check times (provenance files), the gate's expected times (its step table, from the folder READMEs and provenance files), the fast option's measured 20 minutes (Revision README), the assembler's 4 seconds and the 287 s of the first pass of the whole-book test build, a pass that ended FAILED on LaTeX warnings in the text of other chapters as it stood then (runs made for this chapter on 2026-10-08).
- ASSUMED: that the files read are the committed ones. The notebook reads whatever is in the working folder; after any other notebook is rebuilt or any chapter changes a citation, Notebook 23a must be rebuilt, or its nbkit check fails because its outputs changed.
- What "cited" means is a DEFINITION of this chapter (one paragraph, or one cell's output, names both the report's file name and the check), not a reading of the author's intention; a chapter may discuss a check in other words without being counted.

23.16 Exercises with worked answers

Exercise 1. You have a fresh clone and want to know, before waiting hours, how long the gate will take on the development machine with and without the fast option. Which command answers this without running anything, and what numbers do you expect?

Answer. The dry run, which runs nothing:

```
bash Revision/verify_revision.sh --dry-run
bash Revision/verify_revision.sh --dry-run --fast
```

(or the PowerShell twin with the same options). The first prints every step with its expected wall time and the line `revision_selected_steps=85 expected_seconds=8057 skipped_steps=0`; the second prints `revision_selected_steps=81 expected_seconds=6487 skipped_steps=4`. These are the sums of the step table that Notebook 23a prints in Out [9]: 8057 s = 2.24 h and 6487 s = 108.1 min. The measured time of the fast option was about 20 minutes, less than the expected sum, because the table's times are generous upper estimates.

Exercise 2. Using Out [6], how long does the nbkit check of all notebooks of chapters 14, 15 and 16 take, one after the other, and what fraction of the whole book's check time is that?

Answer. The three chapter totals are 88.1 s, 240.4 s and 121.2 s; their sum is $88.1 + 240.4 + 121.2 = 449.7$ s. The whole book takes 1097.9 s, so the fraction is $449.7/1097.9 = 0.4096$, about 41 %: three chapters of the Kohn-Sham part take almost a third of the time, because their notebooks run the Rust solver and the reference solver.

Exercise 3. Why does Notebook 23a leave itself out of its table, and why does it leave chapter 23 out of the citation search? What would go wrong otherwise?

Answer. Its own provenance file is rewritten every time it is built, with the new check result and seconds; if the notebook printed its own record, the printed table would differ from the record it was built from, and the second (check) execution would print yet another time, so the byte comparison of nbkit check would fail. Chapter 23 quotes the notebook's own outputs and check names; if the notebook counted citations in chapter 23, every edit of this chapter would change the notebook's output, which the chapter quotes again: a loop with no fixed point. Leaving both out breaks the loop.

Exercise 4. In In [12] a check is cited by a block only when the block contains both the report's file name and the check name as a whole word. Give an example of why the file name is required, and one of why "whole word" is required.

Answer. The same check name can occur in two reports: `fixture_input` is a check of both `ks-theory-python.json` and `ks-theory-wolfram.json`; without the file name, a paragraph about the Wolfram report would also count as citing the Python one. Whole words: a check named `T3` (if one existed) would otherwise be "found" inside `T3C`, inside the word `T3_block_hamiltonian_map` or inside a path; the conditions `(?<![\w.-])` and `(?![\w-])` refuse a name that is part of a longer word.

Exercise 5. You run Notebook 23a with `BOOK_RERUN_ALL=1` on a computer with 4 processor threads. How many notebooks run at the same time, and can the re-run take less time than the slowest notebook, 11a, alone?

Answer. `workers = max(1, min(8, 4 // 2)) = max(1, min(8, 2)) = 2`: two at a time (`//` divides and drops the remainder). No: every notebook runs on one worker from start to end, so the re-run lasts at least as long as the slowest notebook, 7.6 s on the development machine. That is why the slowest are started first: with two workers, 11a runs on one while the other works through the rest, and the total is close to the larger of 7.6 s and half of the total time, $1097.9/2 \approx 549$ s, plus the time the two workers slow each other down.

Exercise 6. The gate prints `revision_failed_step=precheck`. What happened, what must you NOT do, and what are two correct ways forward?

Answer. An output file of a selected step already differs from the committed version: the gate verifies the committed record and refuses to overwrite uncommitted work. Do not delete or reset the changed files without knowing what they are; they may be your own work. Either run the gate in a fresh clone of the repository (`git clone` into a new folder), or leave the steps whose outputs differ out with the option steps; after you have committed or set aside your changes, the full gate runs.

23.17 Glossary

This glossary lists, in alphabetical order, the technical words that the book defines: the words printed in bold where a chapter first explains them, the words of section 3, "The words used in this notebook", of every notebook, the words of the lists "The words of this chapter", and a few words that a notebook defines elsewhere, such as Sakharov's three conditions. Each entry gives the meaning in plain words, taken from a place that defines the word, and then, in brackets, every section where the word is defined or explained. A section titled "Notebook NNx: complete text" means section 3 of that notebook; "the opening of Chapter N" means the paragraphs before its first section. Where a word is defined in several places, the entry quotes the first of them that has the form of a definition (a list entry "word: meaning", or a sentence such as "a word is ..."); the other sections listed explain the same word again, often for a particular case. For a few central words (among them the five status labels, the extra times, the U(1) charge and the word universe) the entry is written from all its defining sections. A word that is defined together with others in one bold phrase (for example "Eigenvalue, eigenvector") is listed under that phrase when it has no entry of its own. The words in capitals PROVED, COMPUTED, ASSUMED, HYPOTHESIS and OPEN are the five status labels of the book; each has its own entry.

0-9

- **0/1 matrix**: A 0/1 matrix has only the entries 0 and 1. (Section 11.8.)
- **2 × 2 matrix**: A 2 × 2 matrix is a table of four numbers in two rows and two columns. (Section 1.12.)
- **3-momentum** k: the momentum along ordinary 3-space; on the coordinate torus of size ℓ the allowed vectors are $\Delta k\,(n_1, n_2, n_3)$ with integers n_i and $\Delta k = 2\pi/\ell = 0.25$ (in units of H). (Section 14.24.)
- **3-space**: x_1, x_2, x_3 are ordinary 3-space; it inflates, with the scale factor $e^{a_4} \sin^{1/6} z$. (Section 3.1.)
- **A 3-space direction**: A 3-space direction, $\mu = x1$ (the same for $x2$ and $x3$). (Section 6.8.)
- **40 digits**: Double precision: the computer's usual numbers, with about 16 significant digits; 40 digits: mpmath's numbers with as many digits as asked for. (Section 15.29.)
- **40-digit arithmetic**: numbers with 40 significant decimal digits, computed by the package mpmath (slow, but exact far beyond what is needed here). (Section 16.23.)

A

- **Absolute value** $|x|$: x without its sign; $\sqrt{x^2} = |x|$. (Sections 1.5 and 1.8.)
- **Acceleration**: Its velocity is $\dot{q} = dq/dt$ and its acceleration $\ddot{q} = d^2q/dt^2$ (a dot over a letter is the derivative with respect to the time t). (Sections 7.2 and 7.8.)
- **Action** $S[q] = \int_0^T L(q(t), \dot{q}(t))\,dt$: one number for a whole path. (Sections 7.2, 7.8 and 9.4.)
- **Activated regime**: When a gap is many times T the state is in the activated regime: only very few quanta are thermally lifted across it. (Sections 16.20 and 16.23.)
- **Adiabatic**: a change of the Hamiltonian slow enough that a system in an eigenstate stays in the eigenstate that continues it (the adiabatic theorem). (Sections 14.3, 15.24, 22.2 and 22.11.)
- **Adiabatic limit**: At a very small rate the particle stays in its instantaneous level, up to the dressing (the adiabatic limit). (Section 22.8.)
- **Adiabatic theorem**: A system whose Hamiltonian changes slowly enough stays in the eigenstate that continues its initial one (the adiabatic theorem of quantum mechanics; Chapter 14 introduced this adiabatic picture); a system whose Hamiltonian changes too fast is kicked into higher levels. (Section 15.21.)
- **Adiabatically continued state**: Adiabatically continued state: the state that keeps the occupations of the first slice. (Section 15.24.)
- **Adiabaticity**: how well an evolution is adiabatic, that is so slow that every particle stays in its instantaneous level. It is measured by the adiabaticity measure Q_{max} (see that entry): a small Q_{max} means that the slow-change approximation holds. (Sections 16.11 and 22.2.)

- **Adiabaticity measure** $Q_{nm} = AH|\langle n|\partial_a h|m\rangle|/(\varepsilon_n - \varepsilon_m)^2$: the first-order amplitude of the jump $n \to m$; Q_{max} is the largest over all allowed pairs. (Sections 15.21, 15.24, 22.2 and 22.11.)

- **Adjoint**: Adjoint $\bar{\Phi} = \Phi^\dagger C$ with $C = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\gamma^{(x_3)}$; $\Phi^\dagger$ is the row of complex conjugates. (Section 12.29.)

- **Admissible source**: a source for which the field equations of the author's metric can hold. (Sections 17.2, 17.10 and 17.20.)

- **Algebraic condition**: The evolution equation is the 3-space equation minus the extra-time equation; the algebraic condition is $p_3 + p_t = 2p_0$. (Sections 12.2 and 12.9.)

- **Amplification factor** $R(z)$: for the test equation $y' = \lambda y$, one step of a method multiplies y by a number $R(z)$ that depends only on $z = \lambda h$. (Sections 2.5 and 2.10.)

- **Amplitude** and **probability**: when an orbital is written as a sum $\sum_n c_n\varphi_n$ of instantaneous orbitals, the complex number c_n is the amplitude of level n and $|c_n|^2$ the probability to find the particle in level n. (Sections 15.24, 20.2, 20.20, 21.30, 22.2 and 22.11.)

- **Anderson mixing**: a mixing that uses several earlier steps to estimate the slope of G (for one variable it is close to the secant method). (Sections 13.21, 13.24, 15.5, 15.8 and 15.13.)

- **Angle**: The number θ is called the angle of a rotation and the rapidity of a boost. (Section 5.18.)

- **Angular frequency**: where $\omega > 0$ is a fixed number, the angular frequency of the spring. (Section 7.2.)

- **Anisotropic stress**: the difference $p_3 - p_t$ between the 3-space pressure and the extra-time pressure. (Sections 12.2 and 12.24.)

- **Annihilates**: give $bb^* = \mathrm{diag}(1, 0)$ and $b^*b = \mathrm{diag}(0, 1)$, whose sum is the identity; $b|1\rangle = |0\rangle$ (it annihilates the fermion), $b^*|0\rangle = |1\rangle$ (it creates one), and $b^*|1\rangle = 0$: a second fermion cannot be created. (Section 10.11.)

- **Annihilation and creation operators** f_p, f_p^*: f_p empties mode p (and gives zero if it is empty), f_p^* fills it (zero if it is full), each with a sign $(-1)^{(\text{number of occupied modes below } p)}$. (Section 5.37.)

- **Annihilation operator**: Creation operator $a_p^\dagger$, annihilation operator a_p: add or remove a fermion in orbital p, with the sign $(-1)^{\nu_p}$, where ν_p is the number of occupied orbitals before p; on the 16 occupation-number states they are 16×16 matrices. (Sections 5.34, 13.4, 13.13, 21.26 and 21.29.)

- **Annotation**: Tick marks, labels, the vertical range, and an annotation: a text in a white box with an arrow pointing at the place $(2.5\pi, 0)$, where two curves lie on the axis and would otherwise be hard to see. (Section 5.33.)

- **Ansatz**: An ansatz is a guessed form of the solution with unknown parts to be determined. (Section 14.3.)

- **Anti-Hermitian**: A matrix is Hermitian if it equals its conjugate transpose, $M^\dagger = M$, and anti-Hermitian if $M^\dagger = -M$. (Sections 8.17, 8.35 and 10.4.)
- **Anticommutator**: $\{M, N\} = MN + NM$. (Sections 1.28, 1.32, 4.7, 5.9, 5.34, 5.37, 6.13, 8.3, 8.14, 10.11, 10.18, 13.4, 13.13, 17.21, 18.4, 21.25 and 21.29.)
- **Anticommutator matrix**: for two lists of operators $X_1, \ldots, X_n$ and $Y_1, \ldots, Y_n$ whose anticommutators are numbers, the matrix with entries $\{X_i, Y_j\}$. (Section 10.48.)
- **Anticommute**: $AB = -BA$, that is $\{A, B\} = AB + BA = 0$. (Sections 1.28, 4.1, 4.2, 4.9, 4.11, 4.19, 5.2, 5.9, 5.34, 7.1, 7.10, 7.16, 7.32, 8.2, 14.10, 18.8 and 21.7.)
- **Anticommuting**: The components of dirac16complex are anticommuting numbers, also called Grassmann numbers after the mathematician who introduced them: exchanging two of them in a product costs a sign, $\Psi_r \Psi_c = -\Psi_c \Psi_r$, and in particular $\Psi_r \Psi_r = -\Psi_r \Psi_r$, so $\Psi_r \Psi_r = 0$. (Sections 1.10, 5.29, 8.2, 21.2 and 21.7.)
- **Antimatter**: matter made of antiparticles; the antiparticle of a particle has the same mass and the opposite charge, as the positron has for the electron (Chapter 21). (Sections 0.2, 20.23 and 21.2.)
- **Antiparticle**: A particle $b_s^*|0\rangle$ and an antiparticle $d_s^*|0\rangle$ (a hole in the filled sea) both have the energy $+E > 0$. (Sections 5.28, 9.13, 10.20, 10.25, 20.17 and 20.23.)
- **Antiquark**: An antiquark ($\bar{u}$, $\bar{d}$) has the opposite numbers. (Section 21.35.)
- **Antisymmetric**: changing sign when two particles are exchanged. (Sections 1.28, 1.32, 4.2, 4.7, 5.2, 5.9, 6.7, 6.13, 6.27, 7.16, 13.13, 18.8, 21.2, 21.7 and 21.14.)
- **Antisymmetric part**: Write M as the sum of its symmetric part $\frac{1}{2}(M + M^T)$ and its antisymmetric part $\frac{1}{2}(M - M^T)$. (Section 7.12.)
- **Antisymmetry**: (i) Antisymmetry: exchanging the particles i and j exchanges the rows i and j of the table, which changes the sign of the determinant (rule 3). (Section 13.3.)
- **Argument** (or **phase angle**): the angle between the arrow and the positive real axis, measured counterclockwise in radians (π radians = 180 degrees). (Sections 1.11, 1.16 and 2.11.)
- **Array**: `np.linspace(-1.0, 3.0, 401)` makes an array (numpy's list of numbers) of 401 numbers from -1 to 3 with equal steps of $4/400 = 0.01$. (Sections 0.13, 1.9, 2.11, 3.13, 13.14, 15.9, 17.11, 21.15, 22.12 and 22.23.)
- **Assembly**: joining the 24 chapter files, with every notebook expanded, into the one file `Revision/textbook/UNIVERSES_IN_PAIRS_TEXTBOOK.md`. (Section 23.2.)
- **Associative**: Products are associative, $(AB)D = A(BD)$ (both sides are $\sum_{k,l} A_{ik} B_{kl} D_{lj}$), so brackets may be left out. (Sections 1.18 and 4.2.)
- **ASSUMED**: a status label: the statement is used but not derived; it is a starting point of the book (a convention, a physical input or an approximation), and everything that depends on it says so. Example: the Z2 mirror at the edge of the hidden direction. (Sections 0.3, 0.20, 0.26, 1.55, 2.29, 3.1, 3.36, 4.21, 5.40, 6.1, 6.30, 7.34, 8.1, 11.28, 12.31, 14.32, 15.31 and 22.27.)

- **Asymptotic regime**: A ratio close to 4 shows that the grids are fine enough for the h^2 term to dominate (the asymptotic regime), which is what Richardson extrapolation needs. (Section 16.4.)
- **Atan2(b, a)**: the angle of the point (a, b) between $-\pi$ and π. (Section 2.27.)
- **Aufbau**: filling the lowest levels with the N particles. (Sections 13.6, 14.21, 14.24 and 15.24.)
- **The author's second route** (Section 11.5): a determinant identity for the Levi-Civita symbol; raising all labels of the Levi-Civita tensor of a metric gives the factor $\sqrt{|\det g|}/\det g$, so the product of two Levi-Civita tensors is the sign of $\det g$ times the product of two symbols; for the author's metric $\det g = \cos^2 z > 0$ (record `python-lovelock-report.json`, check `sqrt_abs_det_g`), the sign is $+1$, and the author's second route gives the generalized delta without an extra sign. (Section 11.28.)
- **Average**: The ratio $\int E\xi\, dt / \int \xi\, dt$ is an average of E over the bump. (Section 7.3.)
- **Axes**: `plt.subplots(1, 3, ...)` makes a figure `fig` with one row of three panels, the axes (drawing areas) `axes[0]`, `axes[1]`, `axes[2]`, 10.5 by 3.9 inches in all. (Sections 5.10, 11.9, 14.11, 17.11, 19.16 and 20.11.)

B

- **Back-reaction**: Back-reaction: the effect of the matter's own gravity on the metric; a test field has none. (Sections 17.2, 17.10, 17.20, 17.22 and 22.2.)
- **Band level**: The label n is the integer that numbers the levels of one sector in order: $n = 0$ is the band level (bound to the brane; the occupied level of the Fermi shell), $n = 1, 2, \ldots$ are the bulk levels above it, and $n = -1, -2, \ldots$ are the levels of negative energy, the negative branch, which the record's filling convention treats as the normal-ordered sea that holds no particles. (Sections 22.2 and 22.11.)
- **Bare mass** m: the mass in the Lagrangian; effective mass $M_{eff}(y)$: the mass seen by an orbital, m plus the mean-field term. (Section 19.20.)
- **Barrier**: a region that the classical equations forbid, for example a region of too high energy; tunnelling is a quantum transition through it, with a small but nonzero probability. (Section 20.2.)
- **Baryogenesis**: Explaining how is the matter-antimatter problem, also called baryogenesis. (Section 21.4.)
- **Baryon**: a particle made of three quarks, such as the proton $p = uud$ and the neutron $n = udd$ ($B = 1$); a meson (pion π) is a quark and an antiquark ($B = 0$). (Sections 21.2 and 21.35.)
- **Baryon number**: The baryon number counts protons and neutrons minus antiprotons and antineutrons. (Sections 20.23, 21.2 and 21.35.)
- **Baryon-to-photon ratio** η_B: the number of baryons minus antibaryons per photon of the microwave background (Section 21.4). The letter η with two indices, η_{ab} or η^{ab}, is the frame metric; $\eta(K)$ without a subscript is the efficiency of the rate model of Section 21.6. (Sections 21.2 and 21.4.)

- **Basic operators**: Every operator used here is a combination of 32 basic operators O_i: $O_p = F_p$ and $O_{16+p} = F_p^*$ for $p = 0, \dots, 15$, where $F_p = b_{p+1} = f_p$ for $p < 8$ (particles) and $F_p = d_{p-7}^* = f_p^*$ for $p \geq 8$ (antiparticles appear in Ψ through their creation operators), and F_p^* is the Hilbert adjoint of F_p. (Section 5.38.)

- **Basis**: a set of orbitals in which every orbital is written as a sum. (Sections 4.13, 4.15, 5.16, 5.23, 7.29, 22.2 and 22.11.)

- **Bianchi identities**: $R^a{}_{bcd} + R^a{}_{cdb} + R^a{}_{dbc} = 0$ (first) and $\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$ (contracted): identities that every metric satisfies; checking them checks the computation. (Sections 3.29, 11.26, 12.2, 12.15, 17.2 and 17.20.)

- **Bilinear**: an expression $\Psi^\dagger K \Psi = \sum_{r,c} \Psi_r^* K_{rc} \Psi_c$ with a fixed matrix K. (Sections 5.1, 5.29, 5.32, 8.35, 9.2, 12.29, 18.2, 18.4, 18.8, 21.2 and 21.14.)

- **Bilinear form**: an expression $\sum_{A,B} x_A M_{AB} y_B$ that contains one factor from the column x and one from the column y in every term. (Sections 7.12, 7.16, 10.33 and 10.37.)

- **Bilinear operator**: $\Psi^\dagger K \Psi = \sum_{A,C} \Psi_A^\dagger K_{AC} \Psi_C$ for a fixed 16×16 matrix K. (Section 5.37.)

- **Binary**: the way of writing numbers with the two digits 0 and 1, in powers of 2 instead of powers of 10: binary 101 is $4 + 0 + 1 = 5$, and binary 0.11 is $1/2 + 1/4 = 3/4$. (Sections 0.22, 0.24, 1.49 and 1.53.)

- **Binary digits**: The computer stores a pattern of occupations as a whole number n whose binary digits are the occupations: digit q (counted from 0, from the right) is the occupation of mode q. (Section 5.38.)

- **Binomial coefficient** $\binom{n}{k}$ ("n choose k"): the number of subsets with k elements of a set with n elements; $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ with $n! = 1 \cdot 2 \cdots n$ and $0! = 1$. For example $\binom{8}{2} = 28$. (Sections 4.13, 4.15, 7.10 and 10.20.)

- **Birthday problem**: This is the birthday problem: the chance that p people have p different birthdays, with 8 "days" instead of 365. (Sections 1.50 and 11.4.)

- **Bisection**: halving an interval that contains the number again and again, keeping the half that still contains it. (Sections 1.3, 1.8, 2.16, 10.43, 13.14, 13.24, 13.26, 13.30, 14.14, 14.17, 14.18, 15.3, 15.8, 15.9, 16.3, 16.18, 16.23, 17.21, 19.16, 22.11, 22.12 and 22.23.)

- **Bisection method**: `bisection` finds the same time independently, by the bisection method: start with an interval $[-30, 30]$ in which E^2 changes sign; look at the middle; if E^2 is still positive there, the onset is later and the middle becomes the new left end, otherwise the new right end; after 200 halvings the interval is far smaller than the rounding of the numbers, and its middle is returned. (Section 8.29.)

- **Bit mask**: To test this quickly it stores a set of labels as a bit mask, the whole number $\sum 2^{\text{label}}$: the set $\{x_1, x_3\}$ (numbers 0 and 2) is $2^0 + 2^2 = 5$. (Section 11.12.)

- **Bits**: the 16 components are numbered $j = 8b_1 + 4b_2 + 2b_3 + b_4$ with four binary digits (bits) b_1, b_2, b_3, b_4, each 0 or 1, one per slot; for example $j = 5$ has the bits 0101. (Sections 0.22, 0.24, 1.4, 1.49, 1.53, 4.17, 4.19 and 21.30.)

- **Block**: a 16×16 matrix cut into four 8×8 pieces; rows and columns 1 to 8 form the first half, 9 to 16 the second half. $\mathrm{diag}(X, Y)$ is the matrix with the blocks X (top left) and Y (bottom right) and zeros elsewhere. (Sections 4.2, 5.9, 14.10, 19.2 and 19.15.)

- **Block diagonal**: a 16×16 matrix whose upper-right and lower-left 8×8 blocks are zero; block off-diagonal: one whose upper-left and lower-right blocks are zero. (Sections 4.2, 4.15 and 5.5.)

- **Block equation**: in each of eight 2×2 blocks the Kohn-Sham equation is $h_j \chi = \varepsilon \chi$ with $h_j = j[-i\sigma_1 \, d/dy + M\sigma_2 + \kappa k \, \sigma_3] + v$, where $j = \pm 1$ is the block type, M the (effective) mass, κk the weighted 3-momentum and v the vector potential. (Section 14.17.)

- **Block equation in real form**: Its level ε and the two functions obey the block equation in real form (`Revision/kohn_sham/ks-theory.json`, blockEquation.realForm; checks block_hamiltonian and block_ode_equivalent): $a' = Ma - \big(\kappa k + j(\varepsilon - v)\big)\, b, \qquad b' = \big(j(\varepsilon - v) - \kappa k\big)\, a - Mb$. (Section 15.2.)

- **Block matrix**: a big matrix cut into smaller matrices (blocks). For example $\begin{pmatrix} 0 & B \\ D & 0 \end{pmatrix}$ has the zero matrix as its upper-left and lower-right blocks. (Section 4.7.)

- **Block off-diagonal**: A matrix whose two off-diagonal blocks are zero is block diagonal; one whose two diagonal blocks are zero is block off-diagonal. (Sections 4.2, 4.15 and 5.5.)

- **Block type** $j = \pm 1$ and **brane parity** (even or odd): labels of the orbitals (explained in section 4 of Notebook 19a). (Sections 16.18, 19.2 and 19.20.)

- **Blueshifted**: The frame velocity along an extra time grows like e^{a_4}: motion along the extra times is blueshifted, because they deflate (in general it grows like $1/h_j$, that is like $e^{-\int \alpha_t \, dx_4}$). (Section 3.31.)

- **Bogoliubov coefficient**: in a quantum field on a gravitational field that changes in time, a number that says how much of a wave that oscillates with positive frequency at early times has turned into a wave of negative frequency at late times; its squared modulus is the mean number of particles created in that wave. It is the standard measure of particle creation by a changing background. (Sections 10.45, 18.2 and 20.2.)

- **Bogoliubov method**: a way to count the quanta of a field that a changing background creates: one compares the modes of the field before and after the change, and the part of a positive-energy mode of the start that has turned into negative-energy modes of the end measures the creation. It is quoted in Chapter 22, not derived or applied. (Section 22.2.)

- **Bonding**: For $\epsilon = -t$ the first row of $(h + t)\phi = 0$ reads $t\phi_L - t\phi_R = 0$, so $\phi_L = \phi_R$, and normalised $\phi = (1, 1)/\sqrt{2}$ (the bonding orbital, lower energy); for $\epsilon = +t$ we get $(1, -1)/\sqrt{2}$ (antibonding). (Section 13.2.)

- **Boolean mask**: `energies[dirac_svals < 1e-9]` keeps the energies at which it is below 10^{-9} (zero up to rounding): a boolean mask, an array of True/False that selects entries. (Section 4.12.)

- **Boost**: the matrix with rows $(\cosh\varphi, \sinh\varphi)$ and $(\sinh\varphi, \cosh\varphi)$ acting on a pair (t, x) of a time-like and a space-like coordinate; it keeps $x^2 - t^2$. (Sections 1.13, 1.16, 4.13, 5.12, 5.18, 5.21, 5.26, 6.21, 8.3, 8.14, 10.34 and 10.37.)

- **Boost pair**: Kinds of pairs $\{a, b\}$ of two different frame directions: space-space (both space-like), time-time (both time-like) and boost pair (one space-like and one time-like). (Section 6.27.)

- **Bosons**: Nature realises this in two ways: for bosons Ψ itself is unchanged, for fermions it changes sign: $\Psi(\dots, x_i\sigma_i, \dots, x_j\sigma_j, \dots) = -\Psi(\dots, x_j\sigma_j, \dots, x_i\sigma_i, \dots)$ (fermions). (Section 13.3.)

- **Bound**: If all of them had their largest size and the same sign, their total would be about $N\epsilon = \epsilon/h$ times the size of the solution; this is a pessimistic bound, the scale of the worst case. (Section 2.6.)

- **Bound state**: A bound state is a solution with $-V_0 < E < 0$ that decays to zero far away on both sides; its energy is an eigenvalue. (Section 2.13.)

- **Boundary conditions**: at the tip $b(-L) = 0$ (the *regular tip*, a choice of the model); at the brane $b(0) = 0$ (*even* orbitals) or $a(0) = 0$ (*odd* orbitals), which follow from the ASSUMED mirror symmetry of the model (the record labels it ASSUMED). (Sections 0.2, 2.27, 10.27, 10.31 and 14.17.)

- **Boundary terms**: An operator A is called antisymmetric for the weight $w(x_0)$ when $\int w\, p\, (Aq)\, dx_0 = -\int w\, (Ap)\, q\, dx_0$ for all functions p, q, up to boundary terms (values at the ends of the interval). (Sections 6.24, 7.3, 7.8, 8.32, 8.35, 10.27 and 20.22.)

- **Boundary value**: We call the numerator, divided by $1 - e^{-6HL}$, the boundary value of p_0. (Section 17.15.)

- **Boundary-value problem**: a differential equation with conditions at two different points (here at both ends of an interval), instead of all conditions at the start. (Sections 2.12 and 2.16.)

- **Bracket**: two guesses at which F has opposite signs; a continuous F has a zero between them. (Sections 2.12, 2.16, 2.27, 15.3, 15.8 and 19.16.)

- **Branch point**: Branch point: a value of a_4' where $F(a_4') = 0$, so that the evolution equation cannot be solved for a_4''. (Sections 17.2 and 17.17.)

- **Branching probability**: the probability that a decaying particle decays in one particular way (one channel). (Section 21.35.)

- **Brane**: a boundary surface of spacetime; in this book the end $z = \pi/2$ of the hidden direction (in the hidden coordinate $y = \ln(\sin z)/(6H)$ the end $y = 0$), where the author's coordinate patch ends and $g_{00} = \cot^2 z = 0$. In the ASSUMED Z2 construction the patch $0 < z < \pi/2$ is glued there to its mirror patch $\pi/2 < z < \pi$. (Sections 2.24, 5.6, 8.31, 8.35, 14.2, 15.1, 15.13, 16.1, 17.2, 17.10, 17.20, 18.2, 18.19, 18.26, 19.2, 19.15, 19.20, 20.2, 20.10, 20.15, 21.2, 21.22, 22.1 and 22.2.)

- **Brane band**: the even-parity level of label 0 in the blocks $j = +1$, which is the zero mode at $k = 0$ and rises with k. (Sections 14.19, 14.24, 15.2, 15.4, 16.15, 16.18, 19.2 and 19.12.)

- **Brane parity**: even means $\chi_2(0) = 0$, odd means $\chi_1(0) = 0$; both kinds of orbitals belong to every state. (Sections 19.2, 19.15 and 19.20.)

- **Brane tension**: Brane tension: an energy per unit area carried by such a surface itself (for the brane, per unit of its seven-dimensional area). (Section 18.2.)

- **Brane value**: A level is a value of ε at which this brane value is zero. (Sections 16.11 and 19.16.)

- **Brane zero mode**: So $\chi = (Ae^{My}, 0)$ is an orbital with $\varepsilon = 0$, for every L: the brane zero mode, largest at the brane. (Section 14.13.)

- **Breakdown**: a point where $F(a_4') = 0$, so that the evolution equation can no longer be solved for a_4''. (Sections 12.24 and 22.2.)

- **Breakdown rate**: a rate at which the probability that a particle is NOT in its instantaneous level stops being small. (Section 22.11.)

- **Broadcasting**: `lowers[:, :, None] == uppers[:, None, :]` compares every lower label with every upper label of the same pair at once (numpy broadcasting: the inserted axes of length 1 are stretched to match), giving one matrix `Outer` of True and False per pair. (Sections 1.48, 5.33, 18.27 and 22.12.)

- **Brute force**, **literal sum**: a computation that tries every case without any shortcut. (Section 11.18.)

- **Bubble sort**: `sort_with_sign(generators)` sorts a list of generator numbers by the method called bubble sort and counts the exchanges. (Section 7.17.)

- **Builder**: the Python file `Revision/textbook/notebooks/src/NNx_name.py` from which the tool `nbkit` builds Notebook NNx. (Sections 0.9, 23.2 and 23.11.)

- **Bulk band**: In the author's metric the record finds this law for the massive bulk band (odd brane parity; its edge at zero momentum is $\varepsilon(0) = 1.292292828069$): the lowest odd-parity level of the shell $n_2 = 1$ has $w_{\text{eff}}(A)$ falling monotonically from 0.239626 at $a_4 = -3$ to 2.268×10^{-3} at $a_4 = 4$, while the brane-band level of the same shell stays between 0.291594 and 0.333275, tending to 1/3 (`Revision/dark_sector/dirac16complex/reports/independent-checks.json`, checks `bulk_band_dark_matter_law` and `brane_band_radiation_law`; Notebook 22b, In [5] and Figure 22b.2). (Section 22.18.)

- **Bulk edge**: the lowest level that is not on the brane band (here a $k = 0$ level of the odd parity). (Sections 14.19, 14.24, 15.2 and 16.13.)

- **Bulk levels**: The label n is the integer that numbers the levels of one sector in order: $n = 0$ is the band level (bound to the brane; the occupied level of the Fermi shell), $n = 1, 2, \ldots$ are the bulk levels above it, and $n = -1, -2, \ldots$ are the levels of negative energy, the negative branch, which the record's filling convention treats as the normal-ordered sea that holds no particles. (Sections 22.2 and 22.11.)

- **Bump**: a smooth shape that is positive on a small interval and zero outside. (Sections 7.3 and 7.8.)

- **Byte**: 8 bits, a whole number from 0 to 255. (Sections 0.24, 0.25, 1.49 and 1.53.)

- **Byte for byte**: Byte for byte: two files equal in every byte. (Sections 11.2, 11.8 and 11.18.)
- **Byte for byte identical**: Byte for byte identical: two files that agree in every character. (Section 16.11.)
- **Byte-identical**: two files with exactly the same bytes. (Section 0.24.)

C

- **C** (charge conjugation): the exchange of particles and antiparticles; CP: C combined with a mirror reflection of space. (Sections 21.35 and 22.17.)
- **C1** (a consequence of T1): a T1 pair taken as the complete classical source of the author's metric is a zero source, and Einstein's equations then have no solution for $H > 0$. (Sections 0.2 and 17.20.)
- **C2**: Source conditions (from the a_4 equations): C1 no component of the source depends on x_0 and $q_{40} = q_{04} = 0$; C2 $p_3 + p_t = 2p_0$; C3 for the linear member $a_4 = AHx_4 + a_0$: $p_3 = p_t = p_0$ and a constant ρ. (Section 17.20.)
- **C3**: Source conditions (from the a_4 equations): C1 no component of the source depends on x_0 and $q_{40} = q_{04} = 0$; C2 $p_3 + p_t = 2p_0$; C3 for the linear member $a_4 = AHx_4 + a_0$: $p_3 = p_t = p_0$ and a constant ρ. (Section 17.20.)
- **Cancellation**: the loss of digits when two nearly equal numbers are subtracted. (Sections 1.4 and 1.8.)
- **Canonical**: The canonical one is fixed by the vielbein postulate (below). (Section 6.13.)
- **Canonical anticommutation relations**: Because of this sign, operators of different modes anticommute; together these rules give the canonical anticommutation relations. (Sections 5.34 and 5.37.)
- **Canonical anticommutator**: the rule $\{\Psi_A, \Psi_C^\dagger\} = B_{AC}$ (times the delta function) that canonical quantisation of the Lagrangian gives. (Sections 10.12, 21.25 and 21.29.)
- **Canonical conjugate**: the operator $\Psi_A^\dagger$ that the Lagrangian pairs with Ψ_A; the question of the notebook is whether it equals the Hilbert adjoint Ψ_A^*. (Sections 5.37 and 10.18.)
- **Canonical Hartree-Fock equations**: It depends on the occupied orbitals only through ρ, which does not change when the occupied orbitals are mixed among themselves by a unitary matrix; such a mixing can make the Hermitian matrix Λ diagonal, which gives the canonical Hartree-Fock equations $\hat{F}\phi_a = \epsilon_a \phi_a$. (Section 13.6.)
- **Canonical history**: the author's metric leaves the function $a_4(x_4)$ free; the author requires only that the extra times deflate ($a_4' > 0$). The linear member with the constant rate $a_4' = AH$ and $A = 1$ is the canonical history of the Revision record (ASSUMED, a prescribed choice); Notebook 12c also uses the faster member $A = 2$. (Sections 12.2, 12.12 and 12.24.)
- **Canonical matrix**: the fixed list of states the solver computes: particle numbers $N = 8, 136, 688$; couplings $\lambda = 0, \pm\lambda_1, \pm\lambda_2$; slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$; and temperatures $T = 0.01, 0.02, 0.05$ (in units of the mass m) for $\lambda = 0, \pm\lambda_1$. (Sections 15.1, 15.13 and 16.3.)

- **Canonical numerics** of the Rust solver: $G = 900$ RK4 steps and its standard tolerances. (Sections 16.6 and 16.11.)
- **Canonical quantisation**: After canonical quantisation (Chapter 10) the 16 components of the field dirac16complex are no longer numbers but operators Ψ_A, $A = 1, \ldots, 16$: rules that turn one state of the quantum system into another. (Section 21.25.)
- **Canonical spin connection** $\omega_{\mu ab}$: the connection that makes the gammas $\gamma^\mu = e^\mu{}_a \gamma^{(a)}$ covariantly constant (the *vielbein postulate*); it is antisymmetric in a, b. $\Omega_\mu = \frac{1}{2}\sum_{a,b} \omega_{\mu ab} S^{ab}$ enters the covariant derivative $D_\mu \Psi = \partial_\mu \Psi + \Omega_\mu \Psi$. (Sections 6.4, 7.18, 7.26, 8.4, 8.14 and 18.19.)
- **Capture**: `re.search` looks for a pattern in that text: `\d+` matches one or more digits, round brackets around a part of the pattern capture what that part matches, `.*?` matches any text up to the next part of the pattern, and `\(` matches a round bracket itself. (Section 12.25.)
- **cargo**: cargo is Rust's program that turns the Rust code of a program into a program that the computer can run (it builds it). (Sections 0.8 and 0.12.)
- **Carry a representation**: One says that the 16 components carry a representation of the group. (Section 5.11.)
- **Cauchy problem**: This is the initial-value problem, also called the Cauchy problem. (Section 8.16.)
- **Cells**: Every worked example of the book is a Jupyter notebook: a file (with the ending `.ipynb`) that holds text and Python code in cells, together with everything the code printed and drew when it ran. (Sections 0.7, 16.14 and 16.18.)
- **Central difference**: The central difference: $\frac{f(x+h)-f(x-h)}{2h} = f'(x) + \frac{h^2}{6} f'''(x) + \ldots$. (Sections 2.18, 3.30, 6.14, 7.9 and 21.23.)
- **Central finite differences**: They are replaced by central finite differences, $\partial F \approx (F(x + h) - F(x - h))/(2h)$, whose error falls like h^2 as the step h shrinks (the Taylor series: the terms of first and third order of $F(x + h)$ and $F(x - h)$ cancel in the difference, up to the error term $\frac{h^2}{6} F'''$). (Section 9.30.)
- **Centred average**: So the centred difference and the centred average of the two neighbours half a cell away are second-order approximations of f' and f at the midpoint, and their errors contain only even powers of h (the next terms are h^4, h^6, ...). (Section 16.14.)
- **Centred difference**: The growth rate at the end of the run is computed by a centred difference, the change of $\ln(u^\dagger u)$ over the last two steps divided by their length, which approximates the derivative at the middle point; the first-order prediction is evaluated at that middle point. (Sections 8.29 and 16.14.)
- **Chain matrix**: The 3×3 chain matrix T_3 has 2 on the diagonal, -1 just above and just below it, and 0 elsewhere. (Section 1.34.)
- **Chain rule**: if f depends on z and z on x_0, then $\partial f/\partial x_0 = (df/dz)(\partial z/\partial x_0)$. (Sections 2.22 and 3.2.)

- **Change of basis**: two sets of matrices are related by a change of basis when an invertible matrix Q gives $\gamma^a = Q\,\hat{\gamma}^a Q^{-1}$ for every a. Writing the column $\Psi = Q\chi$ then turns every equation written with one set into the same equation written with the other. (Section 4.19.)

- **Channel**: Branching probability: the probability that a decaying particle decays in one particular way (one channel). (Section 21.35.)

- **Character** of a map M: the sign χ in $M^\dagger C M = \chi C$. It says whether S keeps ($\chi = +1$) or reverses ($\chi = -1$) its sign under $\Psi \to M\Psi$. (Sections 18.2 and 18.8.)

- **Characteristic function**: a function of ε whose zeros are exactly the levels. (Sections 19.2, 19.12 and 19.15.)

- **Characteristic polynomial**: $p(\lambda) = \det(M - \lambda I)$, a polynomial in λ whose roots are the eigenvalues. (Sections 1.34, 1.40 and 4.9.)

- **Charge** Q and **charge current** J: a number carried by a field whose density obeys an exact local conservation law, so that the charge of a region changes only by what flows through the boundary of the region (the law is proved at every point with $0 < z < \pi/2$; at the edge $z = \pi/2$ the no-flux condition, that no charge flows through the edge, is ASSUMED, not derived, and only under it is the total charge of a universe constant), and the flow of this number through space (Chapters 10, 18 and 21). (Sections 0.2, 10.25, 20.2, 21.2 and 21.22.)

- **Charge balance**: Flux: the amount of charge that flows through a surface per unit time; charge balance: the charge inside a region changes exactly by the flux through its boundary. (Sections 21.18 and 21.22.)

- **Charge conjugation**: a map that turns a field into a field of the opposite charge. (Sections 5.28, 7.29, 7.32 and 21.8.)

- **Charge current**: charge Q and charge current J: a number carried by a field whose density obeys an exact local conservation law, so that the charge of a region changes only by what flows through the boundary of the region (the law is proved at every point with $0 < z < \pi/2$; at the edge $z = \pi/2$ the no-flux condition, that no charge flows through the edge, is ASSUMED, not derived, and only under it is the total charge of a universe constant), and the flow of this number through space (Chapters 10, 18 and 21). (Section 0.2.)

- **Charge density**: What is the charge density of a spinor field, the quantity whose total stays constant in time when no charge flows in or out through the edges of space? (Sections 5.1, 5.6, 5.32, 7.28, 18.26, 20.2, 20.10, 20.12, 20.15, 21.2 and 21.22.)

- **Charge matrix**: The charge matrix is the product of the four space-like gammas. (Sections 5.4, 20.2, 21.2, 21.7 and 21.14.)

- **The charge of the conjugated field**: The charge of the conjugated field, line by line, for $\Psi' = \Gamma \Psi^{\dagger T}$. (Section 21.26.)

- **Charge operator**: Charge operator $Q = \sum_{A,C} \Psi^\dagger_A B_{AC} \Psi_C$ (the integral of the charge density $\Psi^\dagger B \Psi$ for one point). (Section 21.29.)

- **Charge sloshing**: the loop moves the charge back and forth between regions instead of settling. (Sections 13.21 and 13.24.)

- **Charge-conjugation matrix** $\mathcal{C}$: a constant matrix for which $M = \mathcal{C}C$ obeys the intertwiner condition (the definition of the Revision record); same mass for $s = +1$, mass reversed for $s = -1$. In Notebook 05c, section 4 shows that then $\Psi^c = \mathcal{C}\bar{\Psi}^T$ solves the field equation with V (same mass) or with $-V$ (mass reversed) whenever Ψ solves it. (Sections 5.1, 5.28, 5.32, 7.29, 7.32, 21.2, 21.8 and 21.14.)

- **Chart**: A chart is a choice of coordinates. (Section 17.20.)

- **Chebyshev interpolation**: the polynomial through the values of a function at the Chebyshev nodes $s_j = 1 + \cos\theta_j$, $\theta_j = \pi(j + 1/2)/N$; for smooth functions it is accurate to many digits. (Section 22.11.)

- **Chebyshev nodes**: The Chebyshev nodes are the slices $s_j = 1 + \cos\theta_j$ with $\theta_j = \pi(j + \frac{1}{2})/56$, $j = 0, \ldots, 55$ (`0.5 * END` is 1). (Section 22.12.)

- **Check**: a statement that must be true. The helper `check(condition, name)` of the set-up cell stops the notebook with an *AssertionError* when the condition is false, and prints `PASS name` when it is true. (Sections 0.4, 0.7, 0.12, 0.20, 5.9 and 23.2.)

- **Checker**: Verifier and checker: a program of the Revision record that recomputes a result (exactly, with Wolfram or with sympy, or numerically, with Rust or Python) and writes a report: a JSON file that lists named checks, each with a verdict such as PASS. (Section 23.2.)

- **Chemical potential** μ: the energy that fixes the average particle number. (Sections 13.9, 13.26, 13.30, 15.26, 15.29, 16.23, 21.2, 21.5 and 21.35.)

- **Chiral halves**: the components 1 to 8 (chirality $\Gamma = -1$) and 9 to 16 ($\Gamma = +1$); $P_- = \mathrm{diag}(1_8, 0)$ and $P_+ = \mathrm{diag}(0, 1_8)$. (Sections 5.1, 5.5, 5.16, 18.2 and 18.8.)

- **Chiral projectors**: $P_\mp = \frac{1}{2}(I \mp \Gamma)$ with the chirality $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)} = \mathrm{diag}(-I_8, I_8)$; P_- keeps components 1 to 8, P_+ components 9 to 16. (Sections 10.37 and 18.2.)

- **Chirality** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$: the product of all eight gammas in the author's order. (Sections 0.2, 4.5, 5.5, 10.9, 18.2, 18.8, 19.15, 19.20, 20.2, 20.10, 20.15, 21.2, 21.7 and 21.14.)

- **Chirality image**: for a field Ψ, the field $\chi = \Gamma\Psi$. Because $\Gamma^2 = I_{16}$, also $\Psi = \Gamma\chi$: the image carries exactly the same information, written in other variables. (Section 10.48.)

- **Chirality matrix** $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\cdots\gamma^{(x_7)}$: the product of all eight gammas; it anticommutes with every $\gamma^{(a)}$ and $\Gamma^2 = 1$. (Sections 7.32 and 14.10.)

- **Chirality partner**: $\Gamma\Psi$. (Sections 21.2 and 21.22.)

- **Chirality product**: the product $\Gamma = \gamma^{(x_0)}\gamma^{(x_1)}\gamma^{(x_2)}\cdots\gamma^{(x_7)}$ of all eight gammas, in this order (the author's order); the Revision record calls it the chirality. Here it is only a product of eight matrices. (Section 4.19.)

- **Christoffel symbols** $\Gamma^a{}_{bc}$: numbers built from first partial derivatives of the metric; the geometry of the book explains their meaning. Here they are only an exercise in partial derivatives. (Sections 2.19, 2.22, 3.15, 3.21, 3.29, 6.3, 6.13, 7.18, 7.26, 8.4, 8.14, 9.9, 9.16, 11.2, 11.18, 12.2, 12.15, 17.2, 17.20 and 18.19.)

- **Circle of latitude**: A circle of latitude $\theta = \theta_0$ with $0 < \theta_0 < \pi/2$, run through at unit speed, is NOT a geodesic: along it $d^2\theta/ds^2 = 0$, while the right side is $\sin\theta_0 \cos\theta_0 \, (d\varphi/ds)^2 = \sin\theta_0 \cos\theta_0/(a^2 \sin^2\theta_0) = \cot\theta_0/a^2 \neq 0$ (unit speed means $a \sin\theta_0 \, d\varphi/ds = 1$). (Section 3.16.)

- **Class**: A class is a recipe for objects that carry their own data: `XorShift64Star(seed)` makes a generator object, and the function `__init__` (run when the object is made) stores the seed as its state `self.state`. (Sections 1.54, 7.17, 15.9 and 18.20.)

- **Classical bilinears**: A pair made of a solution Ψ with (m, λ) and its partner $\Gamma\Psi$ with $(-m, -\lambda)$, both in the same gravitational field, has total current zero and total charge zero, as classical bilinears (ordinary numbers for dirac16complex00, elements of the Grassmann algebra for dirac16complex). (Sections 19.11 and 21.19.)

- **Classical field**: a field whose values are numbers, as everywhere in Chapter 0, and not the operators of quantum theory (Chapter 7). (Section 0.2.)

- **Classical fourth-order Runge-Kutta method (RK4)**: The Rust program of the Revision record solves them by the shooting method: it guesses an energy, integrates the equations from one end to the other with the classical fourth-order Runge-Kutta method (RK4) in 900 steps, measures by how much the condition at the far end fails, and corrects the energy with Newton's method. (Section 2.1.)

- **Classical Runge-Kutta method of fourth order**: `rk4` is the classical Runge-Kutta method of fourth order (Chapter 2). (Section 18.27.)

- **Clifford algebra** Cl(4,4): the set of all real combinations of products of eight matrices that satisfy the Clifford relation with four signs $+1$ and four signs -1. (Sections 4.3 and 4.15; the opening of Chapter 4.)

- **Clifford product**: a product $\gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$ of k *different* gammas, taken in the order $x0, \ldots, x7$; k is its degree; it is even or odd with k. The degree-0 product is 1. (Section 5.16.)

- **Clifford relation**: $\gamma^a\gamma^b + \gamma^b\gamma^a = 2\eta^{ab} I_{16}$ for all a, b: each gamma matrix squares to $+I_{16}$ (space-like direction) or $-I_{16}$ (time-like direction), and two different gamma matrices anticommute. (Sections 1.28, 1.32, 4.1, 4.3, 4.7, 4.11, 4.19, 5.2, 6.13, 8.2, 17.20, 21.7 and 21.14.)

- **Closed shell**: A particle number N is a closed shell when the filling ends exactly at the end of a group of degenerate levels (levels equal within 10^{-9}), so that the state is unique. (Sections 14.21, 14.24, 15.2 and 17.2.)

- **Closed-shell slab**: the Kohn-Sham states of the model, uniform along 3-space but not along the hidden coordinate y, with complete lattice shells filled. (Section 14.30.)

- **Cobweb diagram**: a picture of an iteration: go vertically to the curve G, horizontally to the diagonal, vertically again, and so on. (Sections 13.24 and 13.25.)

- **Code**: We describe such a matrix by the code of each row: the code $+5$ in row 2 means that the only nonzero entry of row 2 is $+1$ in column 5, so $(Mu)_2 = +u_5$; the code -0 means $-u_0$. (Section 4.2.)

- **Code cell**: A markdown cell holds text; a code cell holds Python code. (Section 0.7.)

- **Coefficient**: A linear expression in some variables $p, q, \ldots$ is a sum of the variables, each multiplied by a fixed number (or, below, a fixed matrix), its coefficient: $\alpha p + \beta q$ is linear, while p^2, pq and $\sqrt{p^2+q^2}$ are not. (Sections 4.1, 7.10 and 7.16.)

- **Coefficient functions**: This form shows the structure that every proof below uses: K is a sum of coefficient functions of the gravitational field ($e^{\mu}{}_{a}$, $\omega_{\mu bc}$; and $\sqrt{|g|}$ in front of $\mathcal{L}$) times bilinears whose kernels are $C\gamma^{(a)}$ and $C\gamma^{(a)}S^{bc}$, $CS^{bc}\gamma^{(a)}$, products of one or three gammas after C. (Section 18.4.)

- **Coefficient matrix**: A *system* is a list of such equations; its coefficients form the coefficient matrix A (one row per equation), and the system reads $Ay = 0$. (Section 5.16.)

- **Coefficient ring**: the symbols $E = e^{a_4}$, $s = \sin^{1/6} z$, $c = \cos z$, $A_1 = a_4'$ and the second and third derivatives $A_2 = a_4''$, $A_3 = a_4'''$, together with H, m, λ, in which every coefficient is written; with the rule $c^2 = 1 - s^{12}$ it allows an exact test whether an expression is zero. (Sections 7.26 and 7.32.)

- **Cofactor**: C_{rc}, the cofactor, contains no entry of row r. (Section 11.13.)

- **Cofactor expansion**: (The record's sympy verifier ran a different exhaustive comparison on the same 266,304 pairs: the literal determinant against a determinant computed by an independent method, the cofactor expansion, which writes a determinant as a sum over the entries of its first row, each entry times the determinant of the smaller matrix left when its row and column are removed, with the signs $+, -, +, \ldots$ in turn; its check is `gkd_literal_equals_cofactor_expansion` in `python-lovelock-report.json`. (Section 1.48.)

- **Colour bar**: The tick labels are the names $x0$ to $x7$, the axes are the frame indices a (rows) and b (columns), and each panel gets its own colour bar (`fig.colorbar`), the scale that translates colours into numbers. (Sections 6.14 and 17.11.)

- **Colour map**: Two tools of matplotlib: a colour map made of a short list of colours, and a coloured square for a legend. (Sections 0.17, 1.25, 5.10 and 21.15.)

- **Colour scale**: A colour scale turns a number into a colour: `DIVERGING` runs from dark blue for -1 through light grey for 0 to red for $+1$, and `SEQUENTIAL` from almost white for 0 to dark blue for large values. (Sections 10.10 and 18.9.)

- **Column**: A column u is a list of n numbers $u_0, \ldots, u_{n-1}$ written one below the other; the matrix acts on it by $(Mu)_i = \sum_j M_{ij} u_j$. (Section 4.2.)

- **Combination**: A combination of matrices $M_1, \ldots, M_N$ is $c_1 M_1 + \cdots + c_N M_N$ with numbers c_j; the span is the set of all combinations. (Section 5.11.)

- **Command**: one line typed into a terminal and run by pressing Enter. (Section 23.2.)

- **Comment**: A line that starts with `#` is a comment: Python skips it; it is there for the reader. (Sections 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 14.11, 15.9, 17.11, 20.11 and 21.15.)

- **Comment lines**: Its first part repeats the complete run instructions of Section 0.11 as comment lines: every line that starts with `#` is a comment, which Python skips. (Sections 0.13, 4.8, 13.14, 16.12, 18.9, 19.16 and 22.12.)

- **Commutant**: all matrices X that commute with every matrix of a list. (Sections 5.11, 5.16 and 14.30.)
- **Commutative**: They are not commutative: in general $AB \neq BA$. (Section 4.2.)
- **Commutator**: where $[A, B] = AB - BA$ is the commutator. (Sections 1.28, 1.32, 4.7, 5.18, 5.21, 5.26, 6.7, 6.13, 7.28, 8.4, 10.11 and 14.11.)
- **Commute**: The components of the field dirac16complex00 are ordinary (complex) numbers, which commute: $\Psi_r \Psi_c = \Psi_c \Psi_r$. (Sections 1.10, 1.28, 4.2, 4.9, 5.9, 5.29, 7.32, 14.10 and 18.8.)
- **Commute with all gammas**: $X\gamma^a = \gamma^a X$ for all eight a. (Section 4.19.)
- **Commuting** and **complex conjugation**: ordinary numbers are commuting, the order of a product does not matter ($ab = ba$), while the anticommuting numbers of Chapter 7 have $ab = -ba$; complex conjugation changes the sign of the imaginary part of every complex number, $a + ib \to a - ib$ (Chapter 1). (Sections 0.2, 1.28, 21.2 and 21.7.)
- **Commuting, anticommuting components**: the components of dirac16complex00 are ordinary numbers ($\Psi_r \Psi_c = \Psi_c \Psi_r$); those of dirac16complex are *anticommuting* (Grassmann) numbers, $\Psi_r \Psi_c = -\Psi_c \Psi_r$, so that exchanging two factors in a product costs a sign. We write $\epsilon = +1$ for commuting and $\epsilon = -1$ for anticommuting components. (Sections 5.32 and 21.14.)
- **Completeness**: Then W is invertible with $W^{-1} = W^\dagger$, and so also $WW^\dagger = I_{16}$, which written out is the completeness relation $\sum_s u_s u_s^\dagger + \sum_s v_s v_s^\dagger = I_{16}$. (Sections 10.20 and 10.25.)
- **Completion** (of theorem T3): the completion record of T3 (2026-10-08). An adversarial verification found no error in the statement or the proof of T3, but gaps in its verification; the completion record closes the three exact ones: the Kohn-Sham coefficients read from the record in both engines (Section 19.8), the filling convention carried onto the partner (Section 19.9), and statement S6, the 16-component expectation rule (Section 19.10). Its Wolfram report passes 3 of 3 checks and its Python report 7 of 7. (Section 19.1.)
- **Complex conjugate**: for a complex number $z = a + ib$ (a, b real) it is $z^* = a - ib$. (Sections 1.10, 1.40, 2.7, 4.9, 5.9, 5.32, 7.4, 10.9, 21.2 and 21.14.)
- **Complex conjugation**: replacing every number by its complex conjugate ($i \to -i$). On a real column it changes nothing: it is the identity. (Sections 0.2 and 7.32.)
- **Complex number**: $z = a + ib$ with real numbers a (the real part, $\mathrm{Re}\, z$) and b (the imaginary part, $\mathrm{Im}\, z$). (Sections 1.10, 1.16, 2.7, 4.9 and 4.11.)
- **Complex plane**: the plane in which $z = a + ib$ is the point (or the arrow from the origin to the point) (a, b); horizontal axis Re, vertical axis Im. (Section 1.16.)
- **Component**: one number of a tensor, such as $E^{x_1}{}_{x_1}$; a diagonal component has $h = j$. (Sections 1.18, 1.24, 11.2, 11.26 and 21.2.)
- **The components along the deflating history**: The components along the deflating history (Sections 11.20 to 11.27). (Section 11.1.)
- **COMPUTED**: a status label: the statement is a number from a numerical (floating-point) computation, given with its measured uncertainty and the file that holds it. (Sections 0.3, 0.20, 3.1, 4.21, 5.40, 6.1, 6.30, 8.1, 11.28, 12.31, 13.38, 14.32, 15.31, 16.26, 17.23 and 22.27.)

- **Condensate**: a configuration that depends only on the time x_4. (Sections 9.18, 9.24, 9.29, 12.2, 12.19, 12.29, 20.12 and 20.15.)
- **The condition**: The condition, line by line. (Sections 21.25 and 21.32.)
- **Conditional expression**: `tiny == 1` is True when exactly one is that small; `x if condition else y` is a conditional expression, which gives `x` when the condition is True and `y` otherwise. (Sections 0.17 and 1.41.)
- **Conditioning**: how strongly a result reacts to small errors of the input. (Sections 15.29 and 16.23.)
- **Configuration**: any field Ψ, that is, 16 components $\Psi_1(x), \ldots, \Psi_{16}(x)$ given at every point x, whether or not it obeys the field equations. (Sections 18.2, 20.2 and 20.10.)
- **Conjugate** $z^* = a - ib$: the mirror image of z in the real axis. (Sections 1.16 and 7.11.)
- **Conjugate transpose** $M^\dagger$: transpose M and replace every entry $x + iy$ by $x - iy$. (Sections 1.36, 1.40, 4.9, 4.11, 5.9, 5.32, 8.22, 20.16, 21.2 and 21.14.)
- **Conjugation**: the rule that turns an element F into F^*. For complex Grassmann numbers the generators come in pairs θ and $\bar{\theta} = \theta^*$; the conjugation exchanges the two members of every pair, replaces every coefficient by its complex conjugate and REVERSES the order of every product: $(FG)^* = G^*F^*$. (Sections 7.11 and 7.16.)
- **Connection coefficients**: The n^3 numbers $\Gamma^a{}_{bc}$ at each point are the connection coefficients; they are chosen so that $\nabla_b V^a$ is a tensor, which fixes how they change between systems of coordinates (we do not need that rule). (Section 3.15.)
- **Conservation**: Conservation means $\nabla_\mu T^\mu{}_\nu = 0$ for all eight. (Sections 9.9, 9.16, 11.2, 12.2, 12.15, 12.24 and 17.2.)
- **Conservation law**: the statement $\nabla_\mu T^\mu{}_\nu = 0$ (the covariant divergence vanishes). In flat spacetime it says that energy and momentum are neither created nor destroyed, they only flow. In the author's metric, which depends on the time x_4 through a_4, it is a local balance law: the energy is not conserved, it changes along the history (sections 4 of Notebook 15b and 9). The momenta along $x_1, x_2, x_3, x_5, x_6, x_7$, coordinates on which the metric does not depend, have totals that are constant, but only under ASSUMED boundary conditions (Section 20.22 of the book). (Sections 9.9, 15.19, 17.20 and 20.20.)
- **Conservation law along** y: This is the conservation law along y (ks-theory.json, emt.conservationY). (Section 15.16.)
- **Conserved**: Conserved: the same number at every point of the path. (Section 3.34.)
- **Conserved number**: a number whose total before a process equals its total after it, in every process that happens. (Section 21.35.)
- **Conserved quantity**: a number whose value does not change in time for every solution. (Sections 20.2 and 20.22.)
- **Consistent**: If $MM^* = 1$ this is automatically true, and the condition is called consistent; otherwise it forces further conditions on Ψ (possibly $\Psi = 0$). (Sections 5.29 and 21.10.)

- **Constant**: They are constant: the same matrices at every point and at every time. (Sections 4.4 and 21.18.)
- **Constraint**: an equation with no second derivative a_4''. (Sections 11.21, 12.2, 12.15, 12.24, 17.2 and 17.20.)
- **Constraint propagation**: The derivative of the constraint is $3a_4'$ times the evolution factor: this is the constraint propagation of the record (`a4-equations.json`, key `generalSource`, entry `constraint_propagation`; `python-a4-report.json`, check `bianchi_x4`). (Section 11.22.)
- **Contact interaction**: $w(\mathbf{r}, \mathbf{r}') = g_c\,\delta(\mathbf{r} - \mathbf{r}')$; it acts only when two particles are at the same point. (Sections 13.19, 13.35 and 14.30.)
- **Contour map**: `ax.contourf(A, B, E_map, levels=30, ...)` colours the plane of the angles (α, β) in 30 bands of equal energy (a contour map: each band joins the points with nearly the same value), and the colour scale is labelled with the energy and its unit t. (Section 13.14.)
- **Contracted Bianchi identity**: For every metric the Einstein tensor has zero divergence, $\sum_\mu \nabla_\mu G^\mu{}_\nu = 0$ for each ν (the divergence is the covariant derivative with its index μ set equal to the upper index of G and summed; a standard theorem, the contracted Bianchi identity; quoted, ASSUMED in general). (Section 3.25.)
- **Contraction**: setting an upper and a lower index equal and summing over it, as in δ^a_a or $v^a w_a$. (Sections 1.26, 1.32, 1.47 and 3.14.)
- **Control**: a comparison case computed with the same method as the case under test; a negative control (see that entry) is one in which the effect must NOT occur. In Chapter 19 the control C is the universe of mass $-M$ with the UNtransformed tip, and the wrong partner D the one with the reversed coupling. (Sections 19.2, 19.10 and 19.12.)
- **Control metric**: Suppose instead the extra times also inflated, with the scale factor $e^{+a_4}\sin^{1/6} z$ (a control metric, not the author's). (Section 21.17.)
- **The conventions of the book**: The conventions of the book (Revision record, `Revision/SPEC.md` section 4; record formula `equation_of_state_definitions`): the energy density and the pressures are (the formula is displayed in that section). (Section 9.4.)
- **Convergence**: Basis: the set of instantaneous levels in which the evolving orbital is expanded; a truncated basis keeps only some of them, and convergence means the result stops changing when more are kept. (Section 22.11.)
- **Convergence factor**: Error: the distance $x - x^*$ from the fixed point; convergence factor: the number by which one step multiplies the error near the fixed point. (Sections 13.21 and 13.24.)
- **Convergence order**: the power q in "error $\approx$ constant $\times h^q$"; halving h divides the error by 2^q. (Section 12.24.)
- **Convergence ratio**: So the convergence ratio $(x(h) - x(h/2))/(x(h/2) - x(h/4))$ is 4 up to a correction of order h^2. (Section 16.4.)

- **Coordinate**: a number that says where a point is along one direction. The author's universe has eight coordinates, named $x_0, x_1, \ldots, x_7$ (the author's own names, in the author's order; the hidden direction x_0 comes first). (Sections 0.16, 3.2, 3.12, 3.21, 3.34, 4.9, 6.13, 6.27, 7.26, 8.14, 8.22, 8.28, 8.35, 9.16, 11.2, 11.18, 11.26, 12.2, 12.15, 12.29, 14.10, 17.2, 17.10, 17.20, 19.15, 19.20, 20.10, 20.15 and 21.2.)

- **Coordinate covariant derivative**: They enter the coordinate covariant derivative of a vector field, $\nabla_\mu V^\nu = \partial_\mu V^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda} V^\lambda$, the derivative that compares the vector at two points correctly although the coordinate directions change between them. (Section 6.3.)

- **Coordinate density**: Coordinate density: the same per unit of the coordinate y (it contains the volume factor e^{6Hy}). (Sections 15.13, 15.19 and 19.20.)

- **Coordinate gammas**: The coordinate gammas are $\gamma^\mu = \gamma^{(\mu)}/f_\mu$ (the frame gamma of the same direction divided by its frame factor); they obey $\gamma^\mu\gamma^\nu + \gamma^\nu\gamma^\mu = 2g^{\mu\nu}$ with the inverse metric $g^{\mu\mu} = \eta_{\mu\mu}/f_\mu^2$. (Sections 7.18, 7.26, 8.2 and 9.2.)

- **Coordinate index**: The lower index μ is a coordinate index: it numbers the coordinates. (Sections 6.2 and 6.13.)

- **Coordinate label**: the name of one of the eight coordinates $x_0, \ldots, x_7$ of the author's metric (x_1, x_2, x_3 ordinary space, x_4 the time, x_5, x_6, x_7 the three extra times, which deflate exponentially, x_0 the hidden direction). Programs number them $0, 1, \ldots, 7$: the number 0 stands for x_1, the number 7 for x_0. (Section 11.8.)

- **Coordinate momenta**: Take a wave $\Psi = u(x_4)\, e^{i(q_1 x_1 + q_5 x_5)}$ with the coordinate momenta q_1 (along 3-space) and q_5 (along an extra time). (Section 10.39.)

- **Coordinate momentum** q_a and **frame momentum** k_a: a wave $e^{iq_a x_a}$ has the coordinate momentum q_a; measured with proper lengths its momentum is $k_a = q_a/f_a$, the frame momentum. (Sections 10.3 and 10.42.)

- **Coordinate plane**: With a before b in the order $x0, \ldots, x7$ there are $8 \cdot 7/2 = 28$ generators, one for each coordinate plane (a, b). (Section 5.12.)

- **Coordinate wave number** q_a and **frame momentum** $k_{(a)} = q_a/f_a$, where f_a is the scale factor: the wave $e^{iq_a x_a}$ has q_a radians of phase per unit of coordinate length and $k_{(a)}$ radians per unit of proper length. (Sections 8.24 and 8.28.)

- **Coordinates on the sphere** of radius a: θ (from 0 at the north pole to π at the south pole) and φ (around the axis); the point is $(a\sin\theta\cos\varphi,\ a\sin\theta\sin\varphi,\ a\cos\theta)$. (Section 3.21.)

- **Corollary**: a statement that follows from a theorem in a few lines; corollary C1 of Chapter 20 is an example. (Sections 20.2 and 20.10.)

- **Correlation energy**: the exact ground-state energy minus the Hartree-Fock energy (never positive). (Sections 13.7 and 13.13.)

- **Cosmic microwave background**: The universe is filled with the photons of the cosmic microwave background, the thermal radiation left over from the hot early universe, today at the temperature 2.7255 K (D. J. Fixsen, Astrophys. (Section 21.4.)

- **Cosmological constant**: Couplings: $\alpha_1, \alpha_2, \alpha_3$ weigh the three Lovelock tensors; Λ is the cosmological constant; $\kappa > 0$ measures the strength of gravity. (Sections 3.26, 3.29, 12.2, 17.2, 20.2 and 20.10.)

- **Cosmological-constant-like**: Radiation-like: $w_{\text{eff}} = 1/3$; dust-like (dark-matter-like): $w_{\text{eff}} = 0$; cosmological-constant-like: $w = -1$; phantom: $w < -1$. (Section 22.22.)

- **Coulomb interaction**: $w = 1/|\mathbf{r} - \mathbf{r}'|$, the repulsion of two electrons (atomic units). (Section 13.19.)

- **Counting**: Counting: for each of the six directions $i \in \{x_1, x_2, x_3, x_5, x_6, x_7\}$ there are ${\Gamma^i}_{ix_4}$, ${\Gamma^i}_{x_4 i}$, ${\Gamma^i}_{ix_0}$, ${\Gamma^i}_{x_0 i}$, ${\Gamma^{x_4}}_{ii}$ and ${\Gamma^{x_0}}_{ii}$: $6 \times 6 = 36$ symbols, plus ${\Gamma^{x_0}}_{x_0 x_0}$: 37 nonzero symbols of the 512, of which 25 are different. (Section 12.4.)

- **Couplings**: $\alpha_1, \alpha_2, \alpha_3$ weigh the three Lovelock tensors; Λ is the cosmological constant; $\kappa > 0$ measures the strength of gravity. (Sections 12.2, 12.15, 12.19, 17.2, 19.20 and 20.15.)

- **Covariant constancy**: Defining property of a spinor connection: $[\Omega_\mu, \gamma^a] = -\sum_b {\omega_\mu}^a{}_b \gamma^b$; covariant constancy of the curved gammas $\gamma^\mu = \sum_a {e_a}^\mu \gamma^a$: $D_\mu \gamma^\nu = \partial_\mu \gamma^\nu + \sum_\lambda {\Gamma^\nu}_{\mu\lambda} \gamma^\lambda + [\Omega_\mu, \gamma^\nu] = 0$. (Section 6.27.)

- **Covariant derivative along the curve**: The right side needs V only on the curve, so it defines the covariant derivative along the curve for any $V(\lambda)$. (Section 3.16.)

- **Covariant derivative** of a spinor: $D_\mu \Psi = \partial_\mu \Psi + \Omega_\mu \Psi$ (∂_μ is the partial derivative with respect to x^μ). (Sections 3.21, 5.28, 6.1, 6.13, 6.21, 7.18, 7.26 and 8.35.)

- **Covariant divergence** $\nabla_\mu {T^\mu}_\nu$: the derivative of the tensor summed over its upper index, corrected by the Christoffel symbols. (Sections 9.9, 9.16, 9.29, 11.26, 17.2 and 17.20.)

- **Covariantly constant**: It makes the gammas covariantly constant: $\partial_\mu \gamma^\nu + \sum_\lambda {\Gamma^\nu}_{\mu\lambda} \gamma^\lambda + [\Omega_\mu, \gamma^\nu] = 0$ for all 64 pairs (μ, ν) (PROVED; `python-field-theory.json`, check `covariant_constancy_D_mu_gamma_nu`). (Sections 8.4 and 14.11.)

- **Covector**: A covector is a set of n numbers W_a for every system of coordinates that changes by this rule. (Section 3.14.)

- **CP**: C (charge conjugation): the exchange of particles and antiparticles; CP: C combined with a mirror reflection of space. (Section 21.35.)

- **CPL**: $w(a) = w_0 + w_a(1 - a)$. (Section 22.22.)

- **CPL parametrisation**: CPL parametrisation: the formula $w(a) = w_0 + w_a(1 - a)$ for an equation of state that changes with the scale factor a of the observed universe. (Section 22.2.)

- **Crate**: Rust, cargo, program: Rust is a programming language; cargo builds a Rust crate (a folder with a file `Cargo.toml` and source files) into a program file that the computer runs. (Sections 11.8, 15.14 and 19.21.)

- **Creation**: Creation of a universe: a process whose initial state has fewer universes than its final state. (Section 20.2.)

- **Creation operator** $a_p^\dagger$, **annihilation operator** a_p: add or remove a fermion in orbital p, with the sign $(-1)^{\nu_p}$, where ν_p is the number of occupied orbitals before p; on the 16 occupation-number states they are 16×16 matrices. (Sections 5.34, 13.4, 13.13, 21.26 and 21.29.)

- **Critical rate**: The right-hand side grows without limit, but G has a largest value at the critical rate $v_c = \sqrt{(2 - 80\alpha_2)/(48\alpha_2)}$, where $G' = F = 0$. (Section 12.21.)

- **Cross-check**: The Revision record answers it with a second program that shares no code with the first and solves the same equations by a different method (the reference solver), and with a third program that compares the two, number by number, with a rule for the allowed difference that was written down before any comparison was made (the cross-check). (the opening of Chapter 16.)

- **Cross-section**: a number, with the units of an area, that measures how likely a collision or a reaction is. No cross-section for the creation of universes is computed or implied in this book. (Sections 18.2, 20.2 and 20.20.)

- **CSV**: a text table, one row per line, the cells separated by commas. (Sections 16.11 and 23.11.)

- **CSV file**: The Rust solver's record of the partner problems, a CSV file (comma-separated values: a table as text, one row per line, the cells separated by commas, the first line the column names). (Section 14.11.)

- **Cubic Hermite interpolation**: the value in the middle of a step from the values and the derivatives at its two ends. (Sections 2.24, 14.18, 15.8, 17.2, 17.20 and 22.12.)

- **Cubic Lovelock density** $L_{(3)}$: the third Lovelock scalar, of third order in the curvature: $L_{(3)} = 8(2T_1 + 8T_2 + 24T_3 + 3T_4 + 24T_5 + 16T_6 - 12T_7 + T_8)$, where $T_1, \ldots, T_8$ are the eight cubic invariants made of three factors of the Riemann tensor, the Ricci tensor and the Ricci scalar (each written out in that section). (Section 11.15.)

- **Current** $J^a = -i\bar{\Psi}\gamma^{(a)}\Psi$: the conserved current of the phase symmetry $\Psi \to e^{i\alpha}\Psi$; its time component gives the charge. (Sections 5.4, 5.32, 7.23, 7.28, 7.32, 14.12, 14.17, 18.26, 19.3, 20.2, 20.10, 21.2, 21.14 and 21.22.)

- **The current part**: The current part, line by line. (Section 21.19.)

- **Curvature**: What does not depend on the frame is curvature. (Sections 1.43, 6.17 and 8.4.)

- **Curvature of a plane** $K(a, b) = R^{ab}{}_{ab}$ (no sum): the curvature of the surface spanned by the directions x_a and x_b. (Section 11.18.)

- **Curvature of that plane**: For the coordinate plane of x_a and x_b, $\sigma(a, b) = R^{ab}{}_{ab}$ (no sum) is the curvature of that plane (the sectional curvature). (Section 3.29.)

- **Curvature of the coordinate plane**: is the curvature of the coordinate plane of x_a and x_b (its sectional curvature; "plane" means here the plane of the two directions x_a and x_b at one point). (Section 3.17.)

- **Curvature of the spinor connection** $F_{\mu\nu} = \partial_\mu\Omega_\nu - \partial_\nu\Omega_\mu + [\Omega_\mu, \Omega_\nu]$: the 16 x 16 matrix with $D_\mu D_\nu\Psi - D_\nu D_\mu\Psi = F_{\mu\nu}\Psi$. (Sections 6.5 and 6.21.)

- **The curvature that the sums use** (Section 11.10): the mixed components $R^{x_4 k}{}_{x_0 k} = \sigma_k H a_4' \cot z$ carry the sign $\sigma_k = \pm 1$ of inflation or deflation, not of the space-like or time-like character, and $R^{x_4}{}_{x_0} = 0$ because three inflating and three deflating directions cancel, on the background $b_4 = 1$ (with a changing b_4, $R^{x_4}{}_{x_0} = \frac{3}{2} H \cot z \, b_4'/b_4$, record `curvature.json`); every nonzero $R^{ab}{}_{cd}$ has the weight 2. (Section 11.28.)

- **Curve**: A curve is a point that moves with a parameter λ (a time, or a length along the curve): $x^a(\lambda)$. (Section 3.14.)

- **Curved Clifford relation**: They obey the curved Clifford relation. (Section 6.2.)

- **Curved gamma**: The constant gamma matrix of a direction carries the direction in parentheses, $\gamma^{(x4)}$; a gamma without parentheses, γ^{x4} or γ^{μ}, is a curved gamma (Section 6.2). (Sections 6.1, 6.13, 6.21 and 14.11.)

- **Cyclic property**: It has the cyclic property $\mathrm{tr}(AB) = \mathrm{tr}(BA)$. (Section 4.2.)

D

- **Dagger**: Take a column Ψ of n complex Grassmann components with generators $\psi_A = \Psi_A$ and the conjugate row $\Psi^\dagger$ with generators $\chi_A = \psi_A^*$ (the dagger $\dagger$ means transpose and conjugate). (Sections 7.12, 10.9 and 18.8.)

- **Dark energy**: Dark energy is the name for the unknown cause of the observed speeding-up of the expansion of the universe, and dark matter for the unseen matter that is detected only through its gravity. (Section 9.1.)

- **Dark matter**: Dark energy is the name for the unknown cause of the observed speeding-up of the expansion of the universe, and dark matter for the unseen matter that is detected only through its gravity. (Section 9.1.)

- **Data**: A field equation that contains the time derivative $\partial_4 \Psi$ is used like this: one gives the field on the slice $x_4 = 0$ (a seven-dimensional set with the coordinates $x_1, x_2, x_3, x_5, x_6, x_7, x_0$), the data, and asks for the field at later times. (Section 8.16.)

- **Decorator**: The line `@lru_cache(maxsize=None)` above the function is a decorator: it wraps the function so that a result, once computed for some n_2, is remembered and returned at once the next time. (Section 16.24.)

- **Deep dive**: the way this book is written: every notion is defined, in plain words, before it is used; every derivation is written out line by line, each line followed by the rule that produced it from the line before; every worked example is a complete Jupyter notebook, printed in full, with the complete instructions for running it just before it and the explanation of every line of its code just after it; every number in the text names the notebook cell that computes it and, where it reproduces the Revision record, the record file and the check. (Section 0.1.)

- **Default value**: `diagonal_values` evaluates the eight functions at an array of values of z for one value of a_4 and one of b_4 (`b4_value=1.0` is a default value: a call that leaves out the third argument gets $b_4 = 1$) and returns a table with eight rows, row k for g_{kk}. (Sections 3.13, 5.33, 6.14, 9.30, 10.43, 13.25, 14.18 and 15.9.)

- **Defining property** of a spinor connection: $[\Omega_\mu, \gamma^a] = -\sum_b \omega_\mu{}^a{}_b \gamma^b$; covariant constancy of the curved gammas $\gamma^\mu = \sum_a e_a{}^\mu \gamma^a$: $D_\mu \gamma^\nu = \partial_\mu \gamma^\nu + \sum_\lambda \Gamma^\nu{}_{\mu\lambda} \gamma^\lambda + [\Omega_\mu, \gamma^\nu] = 0$. (Sections 6.7 and 6.27.)

- **Deflate**: The author's metric (the rule that gives the length of a small step in each direction; Section 0.14 introduces it from zero) makes ordinary space grow (inflate) and the three extra times shrink (deflate) exponentially as the time x_4 runs. (Sections 0.1, 0.14, 0.16, 3.12, 4.4, 8.2, 11.26 and 14.2.)

- **Deflate exponentially**: A step dx_5 along an extra time has the proper duration $b_4^{1/2} e^{-a_4} \sin^{1/6} z \, dx_5$: when a_4 grows and b_4 stays constant, the three extra times deflate exponentially (in general their rate of change is $-a_4' + b_4'/(2b_4)$, Section 3.7). (Sections 1.1, 2.1, 3.1, 3.5, 7.1, 11.2, 12.1, 17.2, 17.10, 17.20, 19.15, 19.20 and 21.2.)

- **Deflating**: shrinking. When a_4 grows, e^{-a_4} shrinks: the extra times deflate exponentially while 3-space inflates. (Sections 8.28, 12.15 and 14.10.)

- **Deflating history**: the time dependence $a_4 = AHx_4$ with $A = 1$; as x_4 grows, 3-space inflates and the extra times deflate, both exponentially. (Sections 10.42, 18.26 and 20.1.)

- **Deflation rate**: the rate a_4'. While $a_4' > 0$ the scale factor e^{-a_4} of the extra times shrinks exponentially, at the momentary rate a_4'. (Section 12.24.)

- **Degeneracy** g: the number of different orbitals that share one level. (Sections 15.2, 15.13, 16.20, 16.23, 19.2 and 19.20.)

- **Degree**: The number k of factors is the degree of γ_A; the product is even if k is even and odd if k is odd. (Sections 4.13, 4.15, 5.3, 5.16, 7.10, 7.16 and 21.10.)

- **Delta function** $\delta^7(x - y)$: an idealised object that is zero whenever the point x differs from y and whose integral over the seven coordinates other than the time is 1. (Sections 5.34, 5.37 and 10.12.)

- **Delta-SCF**: the excitation energy as the difference of two self-consistent energies, with one particle moved from the highest occupied orbital (H) to the lowest empty one (L). (Sections 13.27, 13.30, 13.35, 15.13 and 16.11.)

- **Density** $S = \bar{\Phi}\Phi$, a real number. **Bilinear**: a number $\bar{\Phi} X \Phi$ with a 16 by 16 matrix X; a three-gamma bilinear has $X = \gamma^{(a)}\gamma^{(b)}\gamma^{(c)}$ with three different directions. (Sections 11.2, 11.11, 12.29, 13.1, 13.24, 13.35, 14.30, 19.20 and 20.15.)

- **Density functional theory**: This chapter builds, from zero, the tool with which Chapters 14 to 16 compute the states of many quanta of the field dirac16complex in the author's primordial field: density functional theory, DFT for short, in the form of Kohn and Sham. (the opening of Chapter 13.)

- **Density matrix** $\rho(x, x')$: $\sum_a \phi_a(x) \, \phi_a^*(x')$ over the occupied orbitals; its diagonal $\rho(x, x)$ is the density. (Sections 13.13 and 13.19.)

- **Density operator** $\hat{\rho}$: a Hermitian matrix with eigenvalues between 0 and 1 that add up to 1; it describes a mixture of states with these probabilities. (Sections 13.26 and 13.30.)

- **Dependency order**: It re-runs every verifier and checker in dependency order (a program that reads the output of another runs after it), then demands that every committed output is unchanged byte for byte and that every report passes. (Section 23.2.)
- **Derivative**: $y'(t) = dy/dt$ is the rate of change of y at time t, the slope of the graph of y. (Section 2.10.)
- **Determinant**: one number computed from a square matrix. For a diagonal matrix it is the product of the diagonal entries. (Sections 0.14, 0.16, 1.12, 1.20, 1.24, 1.40, 2.19, 3.6, 3.12, 5.3, 11.2, 11.8 and 12.3.)
- **Determinant of a diagonal matrix**: the product of its diagonal entries. (Section 2.22.)
- **Diagonal**: The rules of this book are diagonal: each squared step is multiplied by its own number g_{ii}, called the entry of the metric for the direction x_i, and the results are added: $ds^2 = g_{00}\,dx_0^2 + g_{11}\,dx_1^2 + \cdots + g_{77}\,dx_7^2$. (Sections 0.14, 1.5, 1.20, 2.19, 3.3, 11.2, 12.2 and 12.15.)
- **Diagonal / off-diagonal entry**: the entry $T^{\nu}{}_{\mu}$ with $\nu = \mu$ / with $\nu \neq \mu$. Off-diagonal entries describe flows of energy and momentum (for example $T^{x_4}{}_{x_1}$ is a flow of momentum along x_1) and stresses that shear. (Section 9.29.)
- **Diagonal frame**, scale factors f_a: $f_{1,2,3} = e^{a_4}\sin^{1/6} z$, $f_4 = 1$, $f_{5,6,7} = e^{-a_4}\sin^{1/6} z$, $f_0 = \cot z$; $\gamma^{\mu} = \gamma^{(\mu)}/f_{\mu}$. (Sections 7.18, 8.2 and 8.35.)
- **Diagonal matrix**: a square table of numbers that is zero everywhere except on the line from the top left to the bottom right (the *diagonal*). The author's metric is an 8×8 diagonal matrix with the entries $g_{00}, \ldots, g_{77}$. (Sections 0.16 and 4.19.)
- **Diagonal metric**: $g_{\mu\nu} = 0$ whenever $\mu \neq \nu$; then $ds^2 = \sum_{\mu} g_{\mu\mu}(dx^{\mu})^2$. (Section 3.12.)
- **Diagonal vielbein**: The diagonal vielbein is $e^{a}{}_{\mu} = f_{\mu}\delta^{a}_{\mu}$, its inverse $e^{\mu}{}_{a} = \delta^{\mu}_{a}/f_a$, and the curved gammas are $\gamma^{\mu} = e^{\mu}{}_{a}\gamma^{(a)} = \gamma^{(\mu)}/f_{\mu}$ (no sum) and $\gamma_{\mu} = g_{\mu\mu}\gamma^{\mu} = \eta_{\mu\mu}f_{\mu}\gamma^{(\mu)}$. (Section 18.3.)
- **Dictionary**: The braces `{...}` make a dictionary: pairs of a key and a value, written `key: value`. (Sections 0.13, 0.20, 1.9, 2.11, 3.13, 3.22, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15, 22.12 and 22.23.)
- **Dictionary comprehension**: A dictionary comprehension (a dictionary written with a `for` inside its braces) stores the volume factor at $b_4 = 1$ under each of $a_4 = 0$, 1 and 2; `volume_b4_2` is the volume factor at $a_4 = 1$, $b_4 = 2$. (Sections 0.17, 0.21, 0.25, 1.9, 2.11, 3.22, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 12.25, 13.14, 14.18, 15.9, 16.12, 17.11, 18.9, 20.11, 21.15 and 22.12.)
- **Difference formula**: On the grid the second derivative is replaced by the difference formula $(\phi_{k+1} - 2\phi_k + \phi_{k-1})/h^2$, which follows from the Taylor expansions $\phi_{k\pm1} = \phi_k \pm h\,\phi'_k + \frac{1}{2}h^2\phi''_k \pm \frac{1}{6}h^3\phi'''_k + O(h^4)$: adding them, the odd powers cancel, and the error is of order h^2. (Section 13.2.)
- **Difference quotient**: The heat capacity is also computed as dE/dT, a difference quotient: energies at the temperatures $T \pm dT$ (and $T \pm 2dT$) are subtracted and divided by multiples of $dT = 0.01\,T$. (Section 16.7.)

- **Dimension**: The span of a list of matrices is the set of all their combinations; a basis of a space is an independent list that spans it, and the number of its elements is the dimension of the space. (Sections 4.13, 4.15, 4.19, 5.11, 5.16, 7.10, 8.17 and 21.14.)
- **Dirac adjoint** $\bar{\Psi} = \Psi^\dagger C$: a row of 16 numbers. (Sections 5.4, 5.32, 7.19, 8.2, 9.2, 18.2, 21.2 and 21.14.)
- **Dirac conjugate**: It is not a new equation: it is the Dirac conjugate of the field equation. (Section 7.21.)
- **Dirac operator**: Dirac operator: $\gamma^\mu D_\mu = \sum_\mu \gamma^\mu D_\mu$. (Section 6.13.)
- **Dirac sea**: the picture in which every negative-energy level is filled in the vacuum. Its energy, $-E$ for each of the 8 negative levels, is $-8E$. (Sections 10.20 and 10.25.)
- **Dirac's exchange energy**: With $n_\sigma = n/2$ and $k_F = (3\pi^2 n)^{1/3}$ this is Dirac's exchange energy (1930). (Section 13.15.)
- **dirac16complex** has components that **anticommute**: for two of its components $\Psi_A \Psi_B = -\Psi_B \Psi_A$. (Sections 0.1 and 7.1.)
- **dirac16complex00**: the second field of the theory, with 16 COMMUTING complex components; it is a classical field (it is not quantised). (Sections 0.1, 7.1 and 10.25.)
- **Discrete symmetry**: a symmetry that is a single operation, such as a reflection, and not a continuous family of operations such as the multiplications of the U(1) symmetry (Chapter 21). (Section 0.2.)
- **Discretisation error**: The discretisation error: the program replaces the continuous interval $-L \le y \le 0$ by finitely many points, so it solves a nearby problem, not the exact one. (Section 16.2.)
- **Dispersion relation**: the dispersion relation: the wave oscillates in time for every k (PROVED; Notebook 07d, In [14]). (Sections 7.5 and 7.8.)
- **Distributive**: The product is, however, associative, $(AB)C = A(BC)$, because both sides have the entries $\sum_j \sum_k A_{ij} B_{jk} C_{kl}$, and distributive, $A(B + C) = AB + AC$ (multiply out inside the sum). (Section 1.18.)
- **Divergence** $\nabla_h P^h{}_j$: the curved-space form of "change plus outflow"; zero divergence means conservation. (Sections 3.25, 11.1, 11.2, 12.2 and 21.22.)
- **Divergence form**: the theory record writes the term $\gamma^\mu \Omega_\mu$ as a sum of derivatives: it equals $1/(2\sqrt{|g|})$ times the sum over μ of $\partial_\mu(\sqrt{|g|}\,\gamma^{(\mu)}/f_\mu)$, computed on both patches; the coefficient of $\gamma^{(\mu)}$ is $\partial_\mu(\sqrt{|g|}/f_\mu)/(2\sqrt{|g|})$. (Section 18.27.)
- **Docstring**: Two kinds of lines are text for the reader and are not executed: a comment is everything after a `#` sign on a line, and a docstring is a text in triple quotes `"""..."""` directly below a `def` line, which says in words what the function does (Python stores it with the function and skips it when the function runs). (Sections 0.13, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15 and 22.12.)
- **Domain**: A `DomainMatrix` is sympy's fast matrix of exact numbers from a chosen domain: the rational numbers `QQ` (fractions) or the integers `ZZ`. (Section 21.15.)

- **Dot product**: The dot product of two vectors with n components is the number. (Sections 1.18, 2.18 and 20.17.)
- **Double**: the ordinary number of a computer, with 53 binary digits (about 16 decimal digits). (Sections 16.20 and 16.23.)
- **Double cover**: two spinor matrices, R and $-R$, move the vectors in the same way. (Sections 5.12, 5.18 and 5.21.)
- **Double occupancy**: the probability that both electrons sit on the same site. (Sections 13.7 and 13.13.)
- **Double precision**: the computer's usual numbers, with about 16 significant digits; 40 digits: mpmath's numbers with as many digits as asked for. (Section 15.29.)
- **Double-counting formula**: This double-counting formula gives the same energy as the direct sum of the four parts, which the notebook checks. (Section 13.32.)
- **Doubled system**: The weight $w = \frac{1}{2}$ belongs to the doubled system: the Z2 mirror makes a copy of the patch, every orbital is normalised on the patch, $\int_{-L}^{0}(a^2 + b^2)\,dy = 1$, and the particle number $N = \sum gf$ counts the quanta of the patch and of its mirror copy together; one patch holds $N/2$. (Section 15.2.)
- **Dressing**: the small admixture of other levels that a particle carries while the background keeps moving. (Sections 22.2 and 22.11.)
- **Drift**: The drift is the largest change of the Krein form, divided by $\max(u^\dagger u, 1)$: rounding errors grow with the size of the numbers, so the change is measured relative to it. (Section 8.23.)
- **Dual**: The dual of an antisymmetric 4×4 matrix M is $(\star M)_{pq} = \frac{1}{2}\sum_{r,s}\epsilon_{pqrs}M_{rs}$, where the Levi-Civita symbol ϵ_{pqrs} is the permutation sign of (p, q, r, s). (Section 4.5.)
- **Dummy**: A summed index is called a dummy index (its name does not matter: $A_{ij}B_{jk} = A_{il}B_{lk}$), an index that is not summed a free index (the formula holds for each of its values). numpy's function `einsum` reads this convention literally: `np.einsum("ij,jk->ik", A, B)` says "the first factor has the indices i, j, the second j, k; j appears twice and is summed; the result has the indices i, k". (Sections 1.18, 1.24 and 1.26.)
- **Dummy index**: an index that appears twice in one term, once up and once down; it is summed over all its values. Its name does not matter. (Section 1.32.)
- **Duration**: A step dt alone has $ds^2 = -dt^2 < 0$: it is not a length but a duration, and the time that a clock carried along the step shows, its proper time, is $d\tau = \sqrt{-ds^2} = dt$. (Section 3.4.)
- **Dust-like**: Radiation-like: $w_{\text{eff}} = 1/3$; dust-like (dark-matter-like): $w_{\text{eff}} = 0$; cosmological-constant-like: $w = -1$; phantom: $w < -1$. (Section 22.22.)

E

- **E-fold**: a growth by the factor e; a growth of a_4 by $\ln 2 \approx 0.693$ doubles e^{a_4} and halves e^{-a_4}. (Sections 1.5, 1.8, 22.2 and 22.11.)
- **Effective mass** $V = m + U'(S) = m + \lambda S$: the number that multiplies Φ on the right side of the field equation. (Sections 7.19, 9.2, 9.24, 12.26, 12.29, 14.3, 15.2, 19.20, 20.12 and 20.15.)
- **Effective potential**: Kohn-Sham state: an approximate state of N identical fermions built from one-particle wave functions (orbitals) that each solve a one-particle equation in a common effective potential; the potential depends on the densities of the occupied orbitals, so the equations are solved self-consistently (repeat until nothing changes). (Section 19.20.)
- **Efficiency**: The final asymmetry is $a(\infty) = \epsilon\,\eta(K)$ with the efficiency. (Section 21.6.)
- **Eigen-orbital**: Words of the Kohn-Sham method used again here (Chapters 13 to 15 define them in full): an orbital is the wave of one quantum, and an eigen-orbital one that solves the Kohn-Sham equation with a definite energy; the Fermi level is the energy that separates the occupied orbitals from the empty ones; a state is self-consistent when the potentials computed from its own density are the potentials its orbitals were computed with; the good sector is the set of orbitals that do not depend on the extra times x_5, x_6, x_7; the Z2 mirror is the ASSUMED rule that the hidden direction continues beyond the brane as a mirror image of the computed patch, with the field there fixed by the field on the computed side (Chapter 14 states the rule; record `Revision/kohn_sham/ks-theory.json`, key `boundaryConditions.brane`). (Section 17.2.)
- **Eigenfunctions**: Often it contains a number that is not known in advance, and it has a solution that is not zero everywhere only for special values of that number: these values are the eigenvalues and the solutions the eigenfunctions. (Section 2.12.)
- **Eigenspace**: The set of all eigenvectors of one eigenvalue (with the zero column) is its eigenspace, and the number of independent ones is its dimension. (Sections 8.17, 10.5 and 10.9.)
- **Eigenstate**: A state with $h\phi = \epsilon\,\phi$ is an eigenstate (a stationary state) with the energy ϵ, an eigenvalue of h. (Section 13.2.)
- **Eigenvalue and eigenvector**: a number λ and a nonzero column u with $Mu = \lambda u$. (Section 4.11.)
- **Eigenvalue** and **eigenvector**: a number ε and a column $z \neq 0$ with $Tz = \varepsilon z$. A symmetric matrix of size n has n real eigenvalues. (Sections 1.34, 1.40, 2.12, 4.9, 4.19, 5.3, 5.9, 8.17, 8.22, 9.25, 10.9, 13.2, 13.35, 16.18 and 18.26.)
- **Eigenvalue, eigenfunction**: a value of the number λ (or the energy E) for which the boundary-value problem has a solution that is not zero everywhere; that solution is an eigenfunction. (Section 2.16.)
- **Eigenvector, joint eigenvector**: a column v with $Mv = \mu v$ for a number μ (the eigenvalue); a joint eigenvector of several matrices is an eigenvector of each of them. (Section 9.29.)
- **Eigenvectors**: So the energies of the fields of one momentum k are the eigenvalues of the 16×16 matrix h, and the columns u are its eigenvectors (the columns $v \neq 0$ with $hv = Ev$, Rule 6 of Section 5.3). (Sections 1.34, 1.40, 4.9, 4.19, 5.9, 5.34, 8.17, 8.22, 9.25, 10.9, 13.35, 16.18 and 18.26.)

- **Einstein equations** with cosmological constant Λ and coupling κ: $G^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu$. (Sections 12.29 and 20.15.)
- **Einstein gravity**: Einstein gravity: $\alpha_1 = 1$, $\alpha_2 = \alpha_3 = 0$. (Sections 12.2, 12.15, 12.19, 17.2 and 20.2.)
- **Einstein tensor**: Ricci tensor $R^a{}_b = \sum_c R^{ac}{}_{bc}$, Ricci scalar $R = \sum_a R^a{}_a$, Einstein tensor $G^a{}_b = R^a{}_b - \frac{1}{2}\delta^a{}_b R$: the averages of the curvature that enter Einstein's field equations. (Sections 3.17, 3.29, 11.2, 11.18, 12.2, 12.15 and 20.10.)
- **Einstein's equations**: the field equations of gravity, which say how the source shapes the metric (Chapter 12). (Section 0.2.)
- **Einstein's field equations** $G^\mu{}_\nu + \Lambda\,\delta^\mu{}_\nu = \kappa\,T^\mu{}_\nu$: the Einstein tensor plus a constant Λ (the cosmological constant) times $\delta^\mu{}_\nu$ equals a positive constant κ times the energy-momentum tensor $T^\mu{}_\nu$ of the matter. (Section 3.29.)
- **Einstein-Gauss-Bonnet gravity**: Einstein-Gauss-Bonnet gravity: $\alpha_1 = 1$, $\alpha_3 = 0$. (Sections 12.2 and 20.2.)
- **Einstein-Hilbert Lagrangian**: the Lagrangian density $\sqrt{|g|}R$ of the geometry, with R the Ricci scalar; varying its action with respect to the metric gives Einstein's equations without a source. (Sections 22.2 and 22.4.)
- **Einstein-Lovelock equations**: $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda\delta^\mu_\nu = \kappa T^\mu{}_\nu$, with the three Lovelock tensors $E_{(k)}$ (Chapter 12), couplings α_k, the cosmological constant Λ and the gravitational coupling κ. (Section 20.2.)
- **Einstein-Lovelock gravity**: We use the most general gravity theory that keeps the field equations of second order in eight dimensions, Einstein-Lovelock gravity, and its special case, Einstein gravity. (Section 12.1.)
- **einsum**: numpy's function that evaluates a product written with the summation convention, for example `np.einsum("ij,jk->ik", A, B)`. (Sections 1.24 and 1.32.)
- **Electric charge**: Electric charge Q_{el}, in units of the proton charge. (Section 21.2.)
- **Electron, positron**: two particles of the same mass m and opposite electric charge -1 and $+1$. (Section 20.20.)
- **Elimination**: Computers therefore use elimination: by rule 6 one may subtract multiples of rows from other rows without changing the determinant, and doing so step by step makes every entry below the diagonal zero, which needs about $n^3/3$ multiplications. (Section 1.21.)
- **Ellipse**: So the unit circle becomes an ellipse whose longest and shortest half-axes have the lengths $\sqrt{9} = 3$ and $\sqrt{1} = 1$, the two eigenvalues, and lie along the two eigenvectors (Figure 01g.1). (Section 1.34.)
- **Empty set**: A set is a collection of different objects, written in braces, for example $A = \{x_2, x_5\}$; the empty set $\{\}$ has no element. (Section 4.13.)

- **Empty state**: A system may have several conserved quantities, and each forbids on its own: the conservation laws allow a transition only when EVERY conserved quantity has the same value in A and in B. The empty state of the classical theory is the configuration in which every field is zero everywhere; all its bilinears, and hence all its conserved quantities, are zero. (Section 20.22.)

- **End angle** $\Phi(\varepsilon) = \theta(0)$: the angle reached at the brane when we shoot with the energy ε. (Section 2.27.)

- **Energy**: a column that depends on the time as $e^{-iE\,x4}$ (E real) has the energy E. (Sections 2.2, 4.9, 5.34, 5.37, 7.4, 7.8 and 20.20.)

- **Energy density** $\rho = -T^{x_4}{}_{x_4}$: energy per unit (proper) volume. (Sections 3.26, 3.29, 5.34, 5.37, 9.1, 9.16, 9.24, 10.25, 12.2, 15.16, 15.19, 17.2, 17.10, 20.2 and 20.10.)

- **Energy exchange**: the change of the energy density in time caused by work done by the inflating 3-space and the deflating extra times. (Section 9.24.)

- **Energy kernel**: at the one-particle level the energy of a plane wave is $\Psi^\dagger \mathcal{E}_m(k)\Psi$ with the energy kernel $\mathcal{E}_m(k) = Bh_m(k)$; the evolution is $i\partial_4\Psi =$ (anticommutator matrix) times (energy kernel) times Ψ, that is $i\partial_4\Psi = B\mathcal{E}_m\Psi = h_m\Psi$. (Section 18.23.)

- **Energy level** ε: a value of the energy for which the equations have a solution with all boundary conditions; an eigenvalue. Unit: the mass m (in units with $m = H = 1$). (Sections 2.1 and 2.27.)

- **Energy-change law**: This is the energy-change law (ks-theory.json, emt.energyChange; PROVED, check emt_x4_component; COMPUTED for all 75 states against differences of the self-consistent energies at neighbouring slices with fixed occupations, check emt_energy_change_dE_da4, worst 1.5×10^{-10}). (Section 15.16.)

- **Energy-momentum profiles**: the energy density $\rho(y)$ and the pressures $p_3(y)$ (3-space), $p_t(y)$ (extra times), $p_0(y)$ (hidden direction). (Section 19.20.)

- **Energy-momentum tensor** T: the table of the energy and the momentum that a field carries and of the pressures it exerts (Chapter 9). (Sections 0.2, 3.26, 3.29, 9.16, 12.2, 12.15, 12.29, 15.16, 15.19, 17.2, 17.10, 17.20, 18.26, 20.2, 20.10 and 20.15.)

- **Engine**: the program that did the computation: Wolfram Language (the language of Mathematica, run with wolframscript), Python (with the packages sympy, numpy and mpmath), or Rust (a fast compiled language). The *lead's independent checks* are short Python programs that share no code with the other verifiers. (Sections 0.20 and 0.21.)

- **Ensemble**: The Revision solver uses an ensemble form: one particle is moved from the highest occupied group of equal levels to the lowest empty group, spread evenly over each group, which keeps the symmetries of its reduction (`Revision/kohn_sham/results/parameters.json`, conventions deltaScf). (Sections 13.27 and 15.5.)

- **Entropy** $S = -\sum g[f \ln f + (1-f)\ln(1-f)]$: a measure of how many microscopic arrangements the gas can take. (Sections 13.26, 13.30, 15.26 and 15.29.)

- **Entry**: A_{ij} is the entry in row i and column j. (Sections 1.24, 4.2, 10.9 and 18.8.)

- **Envelope theorem**: (i) $dF/dT = -S_s$ (the envelope theorem). (Section 13.26.)
- **Environment variable**: a named setting that a program receives from the computer when it starts. (Sections 0.13, 0.22, 0.24, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15, 22.12, 23.2 and 23.11.)
- **Equation of state** $w = p/\rho$: pressure divided by energy density. With different pressures along different directions there is one such ratio for each: $w_3 = p_3/\rho$ (3-space), $w_t = p_t/\rho$ (the extra times) and $w_0 = p_0/\rho$ (the hidden direction); these are the equations of state of Section 9.4. (Sections 9.1, 9.4, 9.16, 9.24, 12.2, 12.19, 22.2 and 22.22.)
- **Equator**: The equator $\theta = \pi/2$, $\varphi = s/a$ is a geodesic: $d^2\theta/ds^2 = 0$ and $\sin(\pi/2)\cos(\pi/2) = 0$, and $d^2\varphi/ds^2 = 0$ with $d\theta/ds = 0$. (Section 3.16.)
- **Equilibrium density**: For an external potential v write $\hat{H}_v = \hat{T} + \hat{W} + \hat{V}$, its grand potential $\Omega_v[\hat{\rho}] = \mathrm{Tr}[\hat{\rho}\,(\hat{H}_v - \mu\hat{N})] - T\,S[\hat{\rho}]$, its Gibbs state $\hat{\rho}_v$, and n_v for the density of $\hat{\rho}_v$, the equilibrium density. (Section 13.26.)
- **Error**: the computed value minus the exact value (we print its size, the *absolute value*). (Sections 2.6, 2.10, 13.24 and 16.2.)
- **Error function**: (the error function $\mathrm{erf}(u) = \frac{2}{\sqrt{\pi}}\int_0^u e^{-t^2}dt$ gives $\int_{u_0}^{u_1} e^{-t^2}dt = \frac{\sqrt{\pi}}{2}(\mathrm{erf}\,u_1 - \mathrm{erf}\,u_0)$, and erf is odd, $\mathrm{erf}(-u) = -\,\mathrm{erf}\,u$). (Section 12.21.)
- **Euler characteristic**: The name comes from a classical theorem that this book quotes and does not prove (ASSUMED): in a space of even dimension $n = 2m$ the Lovelock scalar of order m is called the Euler density, because for a closed space (finite and without an edge, like the surface of a ball) with a positive-definite metric (every squared length positive, unlike the author's metric) its integral over the whole space, with the volume factor $\sqrt{|\det g|}$, is a constant times the Euler characteristic: a whole number that depends only on the shape of the space (2 for the surface of a ball, 0 for the surface of a ring) and does not change when the metric is deformed (the Chern-Gauss-Bonnet theorem). (Sections 11.2 and 11.11.)
- **Euler density**: in a space of even dimension $n = 2m$, the Lovelock scalar of order m (in eight dimensions $L_{(4)}$). (Sections 11.2 and 11.11.)
- **Euler's formula**: $e^{i\theta} = \cos\theta + i\sin\theta$ for real θ. (Sections 1.11 and 1.16.)
- **Euler, midpoint, RK4**: three rules that compute the value after one step from the value before it (the formulas are in section 4 of Notebook 02a). (Section 2.10.)
- **Euler-Lagrange equation**: the field equation obtained from a Lagrangian L by requiring that the action does not change, to first order, when the field is changed a little. (Sections 7.8, 7.16 and 7.26; the opening of Chapter 7.)
- **Euler-Lagrange expression**: $\partial\mathcal{L}/\partial\Psi_A^* - \sum_\mu \partial_\mu(\partial\mathcal{L}/\partial(\partial_\mu\Psi_A^*))$; the field equations say it is zero. (Sections 7.3, 7.8 and 18.19.)
- **Even**: So $u(-x) = c\,u(x)$, and applying this twice gives $c^2 = 1$: every bound state is even, $u(-x) = u(x)$, or odd, $u(-x) = -u(x)$. (Sections 1.19, 1.24, 2.13, 2.16, 3.30, 4.13, 4.15, 5.5, 5.12, 5.16, 7.11, 7.16, 11.2, 11.8 and 16.18.)
- **Even function**: $f(-u) = f(u)$. (Section 11.26.)

- **Evolution equation**: This difference is the only combination in which a_4'' survives; in the field equations it is the evolution equation, which decides how a_4 changes (Chapter 12). (Sections 3.26, 11.21, 12.2, 12.9, 12.15, 17.2, 17.3 and 17.20.)
- **Evolution form**: and because $(\gamma^{(4)})^2 = -1$ and $f_4 = 1$ it can be solved for the time derivative (the evolution form; multiply by $-\gamma^{(4)}$): $\partial_4\Phi = -\gamma^{(4)}\Big[V\Phi - \sum_{a\neq 4}\frac{1}{f_a}\gamma^{(a)}\partial_a\Phi - 3H\gamma^{(0)}\Phi\Big]$. (Section 9.2.)
- **Exact arithmetic**: computing with fractions, never rounding. The package sympy does this (`DomainMatrix` over `QQ`, the rational numbers). (Section 5.16.)
- **Exact** computation: with symbols and fractions (sympy), not with rounded decimal numbers; an exact check that an expression is zero proves it for ALL values of the symbols. (Sections 1.24, 6.13, 6.21, 6.27, 11.2, 11.18, 16.18 and 18.8.)
- **Exact local conservation law**: It obeys an exact local conservation law: in the author's metric, for every history $a_4(x_4)$, in particular the one in which ordinary space inflates and the three extra times x_5, x_6, x_7 deflate exponentially, the charge of a region changes only by what flows through its boundary (PROVED, Sections 21.16 and 21.17). (Section 21.1; the opening of Chapter 21.)
- **Exactly rounded**: The module `math` of Python provides `math.fsum`, which adds stored numbers exactly and rounds only once, at the end (the exactly rounded sum), and π. (Section 0.25.)
- **Exchange**: (The index C' carries a prime so that it is not confused with the matrix C.) The first product is the direct (Hartree) term, the second the exchange (Fock) term. (Sections 13.4 and 14.26.)
- **Exchange energy**: E_x is the exchange energy (the Fock term); it has no classical analogue, it comes from the antisymmetry of Φ, and for a repulsive w it lowers the energy. (Sections 13.5 and 13.19.)
- **Exchange hole**: the dip in the probability of finding a second fermion near a first one, caused by the Pauli principle. (Sections 13.15 and 13.19.)
- **Exchange potential**: Hartree potential: the potential that one particle feels from the average density of all particles; exchange potential: the correction that comes from the Pauli principle (equal-label particles avoid each other). (Section 13.35.)
- **Exchange-correlation energy**: The price is one quantity, the exchange-correlation energy, that is not known exactly and must be approximated. (Sections 13.1 and 13.10.)
- **Exchange-only**: So this is an exchange-only Kohn-Sham scheme, which for a contact interaction is the same as the Hartree-Fock approximation: an approximation to the true ground state, whose error is the correlation energy (Section 13.7 showed how large it can be on two sites). (Section 13.32.)
- **Exclusive or** (XOR, `x ^ y`): the two numbers are compared bit by bit, and a result bit is 1 when exactly one of the two bits is 1. Applying the same exclusive or twice gives back the number: for single bits, $(a \oplus b) \oplus b = a$ in all four cases ($b = 0$ changes nothing twice; $b = 1$ flips the bit twice), so it holds bit by bit. That is why an XOR step loses nothing. (Sections 1.49 and 1.53.)

- **Exhaustive check**: a check of *every* possible case, not of examples. (Section 1.47.)
- **Exhaustive test**: a test of every possible case. (Section 11.8.)
- **Expansion rate** (Hubble rate) of a direction: $\partial_{x_4} \ln h_\mu$, the fraction by which its lengths grow per unit of time x_4. (Section 3.12.)
- **Expansion rate of the extra times**: We call α_t the expansion rate of the extra times. (Section 3.7.)
- **Expectation value**: $\langle\phi|X|\phi\rangle$ in a normalised state ϕ. (Sections 5.37, 9.13, 10.25, 13.2 and 19.11.)
- **Expectation-value rule**: for one quantum in the mode u, $\langle\Psi^\dagger X\Psi\rangle = u^\dagger BXu$; for many, $\langle\Psi^\dagger X\Psi\rangle = \mathrm{Tr}(X\rho)$ with the one-body matrix $\rho = \sum_{\text{occupied}} f\, uu^\dagger B$ (f the occupation). (Sections 10.21 and 14.30.)
- **Expected number**: The expected number of events in a list is the average number of them that happen, taken over very many repetitions; it equals the sum of the probabilities of the single events, whether or not they are independent (a rule of the theory of chance that we use without proof). (Section 0.18.)
- **Expected value**: For N tries that each succeed with probability q, the number of successes is about Nq, its expected value, typically within one standard deviation $\sqrt{Nq(1-q)}$ of it (ASSUMED from probability theory). (Sections 1.50 and 1.53.)
- **Exponent**: The caption quotes the count for $n = 16$ as computed, not typed: `f"{x:.1e}"` writes 16! as the text 2.1e+13, and `.split("e")` cuts it at the letter e into the mantissa 2.1 and the exponent +13 (both are still texts). (Section 1.25.)
- **Exponential**: the function e^t; $e^{-t} = 1/e^t$. (Sections 0.16, 5.18 and 5.26.)
- **Exponential function** e^x and **natural logarithm** $\ln x$, which undoes it: $\ln(e^x) = x$. The number $e = 2.71828\ldots$. (Sections 1.5 and 1.8.)
- **Exponential of a matrix**: $\exp(M) = 1 + M + \frac{1}{2}M^2 + \frac{1}{6}M^3 + \cdots = \sum_{k\geq 0} M^k/k!$, the same power series as for e^x. (Sections 5.21 and 10.31.)
- **Exponential wall**: This growth is called the exponential wall of the many-body problem. (Section 13.1.)
- **Exponentially**: A quantity whose rate of change is a fixed negative multiple of itself decreases exponentially: in every interval of time of length $1/(AH)$ it is multiplied by the same factor $e^{-1} = 0.3679$. (Section 0.14.)
- **Exporter**: the small Rust program, held as text inside the Wolfram check Revision/gkd_lovelock/verification/verify_lovelock_gkd.wls, that wrote the GKD values into a file for the Wolfram check. (Sections 1.49 and 1.53.)
- **Extra times** x_5, x_6, x_7: the three time-like directions of the author's space-time besides the time x_4. They are time-like like x_4, and they deflate exponentially: lengths along them carry the scale factor $e^{-a_4}\sin^{1/6} z$, which shrinks while a_4 grows, whereas ordinary space, with the scale factor $e^{a_4}\sin^{1/6} z$, inflates. (Sections 0.1, 1.1, 2.1, 3.1, 3.12, 3.34, 4.1, 6.8, 6.13, 6.27, 7.1, 7.26, 8.2, 9.2, 11.2, 12.1, 12.15, 17.2, 17.10, 17.20, 19.15, 19.20 and 21.2.)

- **Extra-time momentum** k_5, k_6, k_7: the momentum along the extra times; the good sector has $k_5 = k_6 = k_7 = 0$. (Section 21.29.)

F

- **F-string**: A string that starts with `f` is an f-string: every name in braces is replaced by its value, so `CAPTION_FILE` is `"Revision/textbook/figures/00a.captions.json"`. (Sections 0.13, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15, 22.12, 22.23 and 23.12.)
- **Factorial**: Here $n! = 1 \cdot 2 \cdot 3 \cdots n$ (with $0! = 1$) is the factorial. (Sections 1.16, 2.3, 4.13 and 7.10.)
- **Falling factorial** $n!/(n-p)! = n(n-1)\cdots(n-p+1)$: the number of ways to choose p different labels from n in order (`math.perm(n, p)`). (Sections 1.43 and 1.47.)
- **False vacuum**: a state that has the lowest energy only among its neighbouring states; vacuum decay is its quantum transition into a state of still lower energy. (Section 20.2.)
- **Fermi level**: Words of the Kohn-Sham method used again here (Chapters 13 to 15 define them in full): an orbital is the wave of one quantum, and an eigen-orbital one that solves the Kohn-Sham equation with a definite energy; the Fermi level is the energy that separates the occupied orbitals from the empty ones; a state is self-consistent when the potentials computed from its own density are the potentials its orbitals were computed with; the good sector is the set of orbitals that do not depend on the extra times x_5, x_6, x_7; the Z2 mirror is the ASSUMED rule that the hidden direction continues beyond the brane as a mirror image of the computed patch, with the field there fixed by the field on the computed side (Chapter 14 states the rule; record `Revision/kohn_sham/ks-theory.json`, key `boundaryConditions.brane`). (Section 17.2.)
- **Fermi shell**: Fermi shell: the highest occupied shell of a state. (Sections 22.2 and 22.11.)
- **Fermi sphere, Fermi wave number** k_F: the ground state of the free gas fills every plane wave with $|\mathbf{k}| < k_F$, once per label. (Section 13.19.)
- **Fermi wave number**: The non-interacting ground state fills all $\mathbf{k}$ with $|\mathbf{k}| < k_F$ (the Fermi wave number), once for each of the g labels. (Section 13.15.)
- **Fermi-Dirac**: each orbital is an independent two-state system, occupied with the Fermi-Dirac probability f_a (the second step multiplies numerator and denominator by $e^{(\epsilon_a-\mu)/T}$). (Sections 13.26, 15.26 and 21.5.)
- **Fermi-Dirac occupation** $f(\epsilon) = 1/(e^{(\epsilon-\mu)/T}+1)$: the probability that an orbital of energy ϵ is occupied. (Sections 13.30, 15.29, 16.23, 21.2 and 21.35.)
- **Fermi-level crossing**: Fermi-level crossing: an empty level comes down below an occupied one along the history, so the aufbau filling changes. (Sections 15.21 and 15.24.)
- **Fermion**: a particle of a kind that obeys the Pauli principle: two identical fermions with the same label never occupy the same one-particle state. (Sections 7.10, 13.3 and 13.35.)

- **Fermion mode**: A fermion mode is a place that holds zero or one particle. (Sections 5.34, 10.18 and 21.26.)
- **Fermion modes, occupation**: a fermion mode is either empty (0) or occupied (1); 16 modes have $2^{16} = 65536$ states, each a list of 16 zeros and ones. (Section 21.29.)
- **Field**: a function of several coordinates, here $\phi(x_1, x_4)$ or $\phi(x_5, x_4)$, with the author's names: x_1 a direction of 3-space, x_4 the time, x_5 one of the three extra times. (Sections 0.1, 7.5 and 7.8.)
- **Field equation** (the Euler-Lagrange equation of the Revision record): $(\gamma^\mu D_\mu - V)\Psi = 0$ with $V = m + U'(S)$ a real function, m the mass, $D_\mu = \partial_\mu + \Omega_\mu$ the covariant derivative with the spin connection Ω_μ. (Sections 5.32, 7.21, 9.2, 12.2, 21.7 and 21.14.)
- **The field equations of Einstein-Lovelock gravity**: The field equations of Einstein-Lovelock gravity are $\sum_{k=1}^{3} \alpha_k \, E_{(k)}{}^\mu{}_\nu + \Lambda \, \delta^\mu_\nu = \kappa \, T^\mu{}_\nu$. (Section 12.8.)
- **Field equations of gravity**: $\sum_{k=1}^{3} \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda \, \delta^\mu_\nu = \kappa \, T^\mu{}_\nu$. (Sections 12.1 and 17.2.)
- **Field operators**: $\Psi_A = \sum_s \left((u_s)_A b_s + (v_s)_A d_s^* \right)$ for $A = 1, \ldots, 16$, with χ_A its Hilbert adjoint and the canonical conjugate $\Psi_A^\dagger = \sum_C \chi_C B_{CA}$. (Section 5.37.)
- **Field-equation expression**: Define the field-equation expression $E = \gamma^\mu D_\mu \Phi - V\Phi$ and its adjoint $\bar{E} = (D_\mu \bar{\Phi})\gamma^\mu + V\bar{\Phi}$ (sums over μ); a configuration is on shell exactly when $E = 0$, and then $\bar{E} = 0$ too. (Section 9.8.)
- **Figure, plot, axis**: a figure is a picture; a plot draws numbers as points or lines; the horizontal axis and the vertical axis are the two number lines of the plot. (Section 0.12.)
- **Fine grid**: One RK4 step needs the coefficients M, v and κ at the start, the middle and the end of the step, so the potentials are stored on the fine grid of the $2G + 1 = 1801$ step ends and midpoints, $y_i = -L + i\,h/2$. (Sections 15.3 and 22.12.)
- **Fingerprint**: To tie the two together, a report writes down a fingerprint of each file it read. (Sections 0.18, 0.20 and 0.24.)
- **Finite difference**: an approximation of a derivative from values of the function at nearby points, with a small spacing h. (Sections 2.22, 3.29, 9.17, 9.29, 11.26, 15.19, 17.2 and 17.20.)
- **First Bianchi identity**: (S2) The first Bianchi identity $R^a{}_{bcd} + R^a{}_{cdb} + R^a{}_{dbc} = 0$. (Section 3.17.)
- **First integral**: an equation with only first derivatives that every solution of a second-order equation obeys; here $a_4'^2$ as a function of a_4. (Sections 17.2 and 17.20.)
- **First jet**: At one point x, every quantity of the theory (the Lagrangian, the 16 components of the field operator, the 36 components of $T_{\mu\nu}$, the 8 of J^μ) is a polynomial in the 288 numbers of the first jet: the 16 values Ψ_A, the 16 values Ψ_A^* and the $2 \times 8 \times 16 = 256$ first derivatives $\partial_\mu \Psi_A$, $\partial_\mu \Psi_A^*$. (Section 18.20.)
- **First order**: an approximation that keeps only terms linear in the small amplitudes. (Section 22.11.)

- **First variation**: The first variation is its slope at $\epsilon = 0$: $S_1 = \frac{d}{d\epsilon} S[q + \epsilon\xi]\Big|_{\epsilon=0}$. (Sections 7.2 and 7.8.)
- **First-order Lagrangian**: a Lagrangian that contains the velocities only to the first power, like the Lagrangians of the fields of this book. (Section 7.8.)
- **First-order mean-field potential**: More words of the Kohn-Sham computations: a shell is a group of orbitals with the same energy, and a closed shell is a particle number N that fills every orbital up to some energy completely and none above it (so the ground state is unique); the first-order mean-field potential is the potential that one quantum feels from the others when the interaction is counted once, to first order in the coupling λ, with the density of the free gas; fixed occupations means that the same orbitals stay filled when a_4 is changed a little, even if their energies move. (Section 17.2.)
- **First-order system**: the second-order equation written as two first-order ones, $\frac{d}{dx_4} a_4 = v$ and $\frac{d}{dx_4} v = \kappa(p_3 - p_t)/F(v)$, with $v = a_4'$. (Section 12.24.)
- **Fisher-Yates shuffle**: a way to put a list into a random order by swapping entries one after the other. (Sections 1.49 and 1.53.)
- **Fit**: Fit: the least-squares straight line in $1 - a$ over a range of a. (Section 22.22.)
- **The five labels**: PROVED (exact; the book gives the complete proof and, where the Revision record contains an exact computer check of the statement, names the report file and the check), COMPUTED (a number from a numerical computation, with its measured uncertainty and the file that holds it), ASSUMED (a starting point that the book does not derive: a convention, a physical input or an approximation), HYPOTHESIS (an idea that is stated and examined but not established), OPEN (a question that neither the Revision record nor the book answers). (Section 0.20.)
- **Fixed occupations**: More words of the Kohn-Sham computations: a shell is a group of orbitals with the same energy, and a closed shell is a particle number N that fills every orbital up to some energy completely and none above it (so the ground state is unique); the first-order mean-field potential is the potential that one quantum feels from the others when the interaction is counted once, to first order in the coupling λ, with the density of the free gas; fixed occupations means that the same orbitals stay filled when a_4 is changed a little, even if their energies move. (Section 17.2.)
- **Fixed point** of a map G: a number x^* with $G(x^*) = x^*$. (Sections 13.21, 13.24 and 19.4.)
- **Flat**: A space whose Riemann tensor is zero everywhere is flat; conversely, near each point of a space with zero Riemann tensor there are coordinates in which the metric is constant (a standard theorem, quoted, ASSUMED). (Section 3.17.)
- **Flat 4+4 space**: space-time with the constant metric η, whose diagonal entries are $+1$, $+1$, $+1$, $+1$, -1, -1, -1, -1 in the order $x_0, \ldots, x_7$: no gravity, no inflation, no deflation. (Sections 4.9 and 4.11.)
- **Floating-point number**: the way the computer stores a number with a fraction part: a whole number of at most 53 binary digits times a power of 2. Python's `float` and numpy's `float64` are such numbers. They have about 16 significant decimal digits. (Sections 0.22, 0.24, 1.4, 1.8, 1.24, 4.11, 14.11 and 18.27.)

- **Floor**: The second term, $2 \cdot 10^{-12}$ times the size of R (at least 1), is a floor for the rounding errors of the computer, which do not shrink with h. (Section 16.5.)

- **Flux**: the value of the bracket at an end; a nonzero flux at $z = \pi/2$ means that something flows through the patch end (in or out). (Sections 5.6, 8.32, 10.2, 10.27, 10.31, 21.17 and 21.22.)

- **Fock operator**: $\hat{F}$ is the Fock operator. (Section 13.6.)

- **Fock space**: all combinations of the 65536 basis states. (Sections 5.34, 5.37, 9.13, 10.11, 10.18 and 10.25.)

- **Format string**: The three curves are W, W^2 and W^6 with $H = 1$; the third argument of `semilogy`, a short text called a format string, sets the line style: two hyphens draw a dashed line and a colon a dotted one (the plain `plot` lines of this book use the same format strings); `legend` shows the `label` texts in a box. (Section 14.11.)

- **Former measure**: `\w+` is one or more letters, digits or underscores (the name of a state, such as N8_lam0_a00_T10), `\S+` a number, and `\(` and `\)` are round brackets themselves; the four bracketed parts are the name of the state, the error of the old method, the former measure (the difference between the two runs when both used the old method) and the present measure (the difference seen by the present refined run). (Section 0.25.)

- **Forward difference**: The forward difference: $\frac{f(x+h)-f(x)}{h} = f'(x) + \frac{h}{2} f''(x) + \dots$. (Section 2.18.)

- **Fourth-order central difference**: `slope` estimates the derivative of a tabulated function with the fourth-order central difference of step $\delta = sh$: $f'(y) \approx \frac{-f(y+2\delta)+8f(y+\delta)-8f(y-\delta)+f(y-2\delta)}{12\,\delta}$. (Sections 17.21 and 22.23.)

- **Fraction** (rational number): p/q with whole numbers p and $q \neq 0$. (Sections 1.2 and 1.8.)

- **Frame**: Such a set is a frame, and the matrix $e^a{}_\mu$ that converts coordinate steps into frame components is the vielbein (German for "many legs"): the metric is $g_{\mu\nu} = \sum_{a,b} e^a{}_\mu \, \eta_{ab} \, e^b{}_\nu$. (Sections 3.7, 4.4, 4.9, 6.1, 8.2 and 8.14.)

- **Frame current** q_8: for a homogeneous field, the number $q_0 = -i\bar{\Psi}\gamma^{(x_0)}\Psi$, which depends on x_4 only; the hidden-direction charge current is $J^{x_0} = \tan z \, q_0$, so the flux through the brane is $\sqrt{|g|}J^{x_0} = \sin z \, q_0$. It is NOT the charge Q. (Section 18.21.)

- **Frame factors**: These factors are the frame factors: they carry the metric into every derivative term. (Sections 4.4, 7.18 and 8.2.)

- **Frame gamma matrices** $\gamma^{(a)}$: the author's eight real 16×16 matrices with $\gamma^{(a)}\gamma^{(b)}+\gamma^{(b)}\gamma^{(a)} = 2\eta^{ab}$, $\eta = \mathrm{diag}(1,1,1,1,-1,-1,-1,-1)$ (in the order $x_0, \dots, x_7$). (Section 7.26.)

- **Frame index**: The upper index a is a frame index: it numbers the eight frame directions. (Sections 6.2 and 6.13.)

- **Frame metric** η_{ab}: the diagonal table $\mathrm{diag}(+1,+1,+1,+1,-1,-1,-1,-1)$ in the order $x_0, \dots, x_7$; η^{ab} (upper indices) is its inverse. (Sections 1.9, 1.13, 1.27, 1.32, 4.7, 6.13, 6.27 and 8.2.)

- **Frame momenta**: What carries over is the algebra of the frame: in a region so small that the metric functions change little across it, waves with frame momenta k_a obey the same matrix algebra; the Revision record uses this local plane-wave reading with the coefficients of the equation held fixed. (Sections 4.9 and 8.17.)

- **Frame momentum**: Coordinate momentum q_a and frame momentum k_a: a wave $e^{iq_a x_a}$ has the coordinate momentum q_a; measured with proper lengths its momentum is $k_a = q_a/f_a$, the frame momentum. (Sections 8.28, 10.3, 10.39 and 10.42.)

- **Frame rotation** (local Lorentz transformation): a new vielbein $e'^a{}_\mu = \sum_b \Lambda^a{}_b(x)\, e^b{}_\mu$ with a matrix $\Lambda(x)$ that keeps η: $\Lambda^T \eta \Lambda = \eta$. It describes the same metric. (Section 6.21.)

- **Frame velocity**: Scale factor $h_a = \sqrt{|g_{aa}|}$ and frame velocity $\hat{u}^a = h_a u^a$: the proper length (along an extra time: the proper duration) that the body covers along x_a per unit of its own proper time, measured with the rulers and clocks of an observer at rest at its place; divided by u^4 it is the velocity that this observer measures. (Sections 3.31 and 3.34.)

- **Frame-dependent**: a quantity whose value changes when the vielbein is changed (such as $\gamma^\mu \Omega_\mu$). (Sections 6.15 and 6.21.)

- **Frame-independent**: Frame-independent: a statement that holds in every vielbein (such as "$F_{\mu\nu}$ is not zero"). (Section 6.21.)

- **Free**: A summed index is a dummy index (its name does not matter); an index that is not summed is a free index. (Sections 1.18, 1.24 and 1.26.)

- **Free energy**: Free energy $F = E - TS$; heat capacity $C_V = dE/dT$ at a fixed particle number. (Sections 13.30, 15.26 and 15.29.)

- **Free fall**: motion under no force other than the geometry. (Sections 3.16 and 3.34.)

- **Free index**: an index that appears once in every term of a formula; the formula holds for each of its values, so a formula with k free indices over 8 coordinates stands for 8^k equations. (Section 1.32.)

- **Freezing**: The mixture is freezing: as the gas redshifts, w moves toward the condensate's value (check `mixture_radiation_condensate_cpl`). (Sections 22.18 and 22.22.)

- **Frequency**: the number ε in $e^{-i\varepsilon t}$; for real ε the real and imaginary parts oscillate like cos and sin. (Sections 1.13, 1.16, 8.17, 9.29, 10.3, 10.9, 12.29, 20.12, 20.15 and 21.29.)

- **Frozen coefficients**: in a curved space the factors in front of the derivatives change from point to point and in time; near one point and for a short time one replaces them by their values at that point. The momenta measured with these factors are the *frame momenta* k_a. This is a device of the analysis near one point (labelled MODEL); it does NOT say that the extra times stop deflating: in the author's metric they always deflate exponentially, and Notebook 08b follows a wave along the deflating history. (Sections 8.17 and 8.22.)

- **Frozen-coefficient model**: The end of this section defines the frozen-coefficient model of the author's universe, which has the same equation with the derivatives divided by scale factors; it is an ASSUMPTION, because it also leaves out two terms of the author's field equation. (Section 10.3.)

- **Function**: `def` defines a function: a named piece of code that runs when it is called. (Sections 0.13, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15 and 22.12.)
- **Function of a Hermitian matrix**: A function of a Hermitian matrix $A = \sum_i a_i\, u_i u_i^\dagger$ (eigenvalues a_i, orthonormal eigenvectors u_i) is defined through its eigenvalues, $f(A) = \sum_i f(a_i)\, u_i u_i^\dagger$; for $f = \exp$ this agrees with the power series, because $A^m = \sum_i a_i^m u_i u_i^\dagger$. (Section 13.26.)
- **Function of several variables**: a rule $f(x, y)$ that gives a number for every pair of numbers (x, y) (or for every list $x_0, \ldots, x_7$). (Section 2.22.)
- **Functional**: The square brackets in $S[q]$ say that S depends on the whole function q, not on one number; a rule that takes a function and gives a number is called a functional. (Sections 7.2, 7.8 and 13.9.)
- **Functional derivative**: for every η, the function $\delta F/\delta n(x)$ is the functional derivative of F at n. (Sections 13.6 and 13.9.)
- **Fundamental symmetry**: A Krein space is a space with a Hermitian form $[u, v]$ that is *nondegenerate* (no nonzero u has $[u, v] = 0$ for all v) but *indefinite* (it takes both signs), together with a matrix J, the fundamental symmetry, with $J^\dagger = J = J^{-1}$, such that $[u, Jv]$ is a positive inner product. (Sections 10.13 and 10.18.)
- **Fundamental theorem of algebra**: The fundamental theorem of algebra (ASSUMED) says that a polynomial of degree n has exactly n roots $\lambda_1, \ldots, \lambda_n$ when complex numbers are allowed and a repeated root is counted as often as it appears, its multiplicity. (Section 1.34.)

G

- **Gamma matrices**: eight fixed 16×16 matrices of the author, one for each direction, from which the equations of both fields are built (Chapter 4). (Sections 0.2, 1.1, 1.32, 6.13, 8.22, 12.29, 14.10, 17.20, 18.8, 19.15, 19.20, 20.10, 20.15, 21.2 and 21.14; the opening of Chapter 4.)
- **Gap**: an energy interval without levels. (Sections 14.20, 16.20 and 16.23.)
- **Gate**: the script `Revision/verify_revision.sh` (for bash) and its twin `Revision/verify_revision.ps1` (for PowerShell 7). (Sections 23.2 and 23.11.)
- **Gauss-Bonnet**: Lovelock tensors $E_{(1)}, E_{(2)}, E_{(3)}$: the three curvature tensors built with the GKD; $E_{(1)}$ is the Einstein tensor and $E_{(2)}$ the Gauss-Bonnet tensor. (Section 12.2.)
- **Gauss-Bonnet scalar**: GB $= R^2 - 4R^a{}_b R^b{}_a + R^{ab}{}_{cd} R^{cd}{}_{ab}$ is the classical Gauss-Bonnet scalar. (Section 11.15.)
- **Gauss-Bonnet tensor**: Gauss-Bonnet tensor: the classical name of the order-2 Lovelock tensor. (Sections 11.2 and 11.18.)
- **Gaussian curvature**: A space of two dimensions has a single coordinate plane, and its curvature $\sigma(1, 2)$ is the Gaussian curvature. (Sections 3.17 and 3.21.)

- **Gaussian integers**: `random_matrix` is a 4×4 sympy matrix with random Gaussian integers (whole numbers $a + bi$ with a, b from -3 to 3; the `lambda i, j: ...` is a small unnamed function that gives the entry in row `i` and column `j`). (Sections 4.16 and 7.17.)

- **Generalized Kronecker delta**: $\delta^{u_1 \dots u_p}_{l_1 \dots l_p} = \det[\delta(l_i, u_j)]$, the author's `kδ[lower, upper]`. (Sections 1.1, 1.42, 1.47, 11.1, 11.2, 11.8, 11.18, 11.28, 12.2 and 12.15; the opening of Chapter 11.)

- **Generator expression**: The generator expression counts 1 for every nonzero symbol with $\mu \leq \nu$ (the symbols are symmetric in the two lower indices, so the pairs with $\nu < \mu$ are not new); the count is printed as a RESULT line: 25. (Sections 7.9, 8.15 and 18.9.)

- **Generator, Spin(4,4)**: the 28 matrices S^{ab} $(a < b)$ generate the spin transformations $R = \exp(\theta S^{ab})$, which turn the frame of the eight directions; their products form Spin(4,4). Spin(4,3) is the part generated by the 21 S^{ab} with $a, b \neq x_4$ (the transformations that leave the time x_4 alone). (Section 10.37.)

- **Generators** $S^{ab} = \frac{1}{4}[\gamma^a, \gamma^b] = \frac{1}{4}(\gamma^a \gamma^b - \gamma^b \gamma^a)$: the 28 matrices $(a < b)$ from which every element of the part of Spin(4,4) connected to 1 is built as a product of exponentials $\exp(\theta S^{ab})$. (Sections 4.13, 5.12, 5.16, 5.21, 7.10, 7.16, 8.4, 10.33, 10.48, 16.24, 17.11, 17.21, 18.8, 18.19, 21.2, 21.7 and 22.12.)

- **Geodesic**: a curve whose velocity is parallel transported along itself: the straightest possible curve. Its equation is $d^2x^a/ds^2 + \sum_{b,c} \Gamma^a{}_{bc}\,(dx^b/ds)(dx^c/ds) = 0$. (Sections 3.16, 3.21 and 3.29.)

- **Geodesic deviation equation**: In general the distance $\xi(s)$ between two neighbouring geodesics of a surface obeys the geodesic deviation equation $d^2\xi/ds^2 = -\sigma\,\xi$ with the Gaussian curvature $\sigma = \sigma(1,2)$ of Section 3.17 (a standard theorem, quoted, ASSUMED; on the sphere we have just verified it, with $\sigma = 1/a^2$). (Section 3.18.)

- **Geodesic equation**: $du^a/d\tau = -\sum_{b,c} \Gamma^a{}_{bc}\,u^b u^c$, with the Christoffel symbols $\Gamma^a{}_{bc}$ of the metric; its solutions are the paths of free fall. (Section 3.34.)

- **Geometric-tail rule**: If the last two ratios both lie between 0 and 0.5, the study takes the differences to go on shrinking like a geometric series, each one the fraction r (the last ratio) of the one before; this is the geometric-tail rule, and the limit, which the record calls the extrapolated value and Chapter 15 also the large-L value, adds the differences still missing: $x_\infty = x(L_{max}) + d_{last}\,(r + r^2 + r^3 + \dots) = x(L_{max}) + d_{last}\,\frac{r}{1-r}$. (Section 15.2.)

- **Ghost-like**: A component of negative classical energy is called ghost-like; it is not an established physical state. (Section 22.19.)

- **Ghost-like component**: a component of negative classical energy. (Section 22.22.)

- **Gibbs state**: In a finite-dimensional state space Ω has exactly one minimiser, the Gibbs state $\hat{\rho}_0 = e^{-(\hat{H}-\mu\hat{N})/T}/Z$ with $Z = \mathrm{Tr}\, e^{-(\hat{H}-\mu\hat{N})/T}$, and $\Omega[\hat{\rho}_0] = -T \ln Z$. (Sections 13.26 and 13.30.)

- **Git**: Git is the program that downloads the repository and keeps track of the versions of its files. (Section 0.8.)

- **GKD**: the Revision record's program (written in the language Rust) that computes the generalized delta without a determinant. (Sections 1.47, 11.2, 11.8 and 11.18; the opening of Chapter 11.)
- **Global U(1) transformation**: Multiplying every component of Ψ by the same $e^{i\alpha}$ is a global U(1) transformation (U(1) is the name of the group of these phases). (Section 21.22.)
- **Good sector**: the fields that do not depend on x_5, x_6, x_7 ($k_5 = k_6 = k_7 = 0$). (Sections 4.9, 5.34, 5.37, 8.17, 8.22, 8.35, 9.13, 10.4, 10.9, 10.20, 10.25, 10.42, 15.16, 17.2 and 21.29.)
- **Gradient**: the arrow $(\partial f/\partial x, \partial f/\partial y)$. It points in the direction in which f grows fastest and is perpendicular to the level curve through its point. (Sections 2.18, 2.22 and 3.14.)
- **Gram matrix**: The 256 numbers $\mathrm{tr}(\gamma_A^T \gamma_B)$ therefore form the table $16\,\delta_{AB}$; this table is called the Gram matrix. (Sections 4.13, 10.5, 14.31 and 15.5.)
- **Gram-Schmidt method**: This is the Gram-Schmidt method. (Section 10.10.)
- **Gram-Schmidt procedure**: The Gram-Schmidt procedure: it goes through the columns of a matrix P (the rows of `P.T`), subtracts from each column its parts along the columns already kept ($e^\dagger v$ is the size of the part of v along the unit column e), and keeps it, divided by its length, when the length that remains is not zero. (Section 5.38.)
- **Grand potential**: When the particle number is fixed only on average by a chemical potential μ, the equilibrium state at the temperature T minimises the grand potential. (Sections 13.26, 13.30, 15.26 and 15.29.)
- **Grassmann**: Statistics: dirac16complex has complex ANTICOMMUTING components, called Grassmann numbers ($\theta_1\theta_2 = -\theta_2\theta_1$, so $\theta\theta = 0$); dirac16complex00 has complex COMMUTING components (ordinary complex numbers). (Section 18.2.)
- **Grassmann algebra**: all sums of ordinary numbers times monomials, with the rules above. (Sections 7.10 and 7.16.)
- **Grassmann generators**: symbols that anticommute, $\theta_1\theta_2 = -\theta_2\theta_1$, so $\theta_1\theta_1 = 0$. They model the anticommuting components of dirac16complex. (Section 18.19.)
- **Grassmann numbers**: The components of dirac16complex are anticommuting numbers, also called Grassmann numbers after the mathematician who introduced them: exchanging two of them in a product costs a sign, $\Psi_r\Psi_c = -\Psi_c\Psi_r$, and in particular $\Psi_r\Psi_r = -\Psi_r\Psi_r$, so $\Psi_r\Psi_r = 0$. (Sections 5.29, 7.1, 7.10 and 7.16.)
- **Gravitational field**: in this book, the author's metric of Section 0.1, the rule for the lengths of small steps; since Einstein, gravity is described by such a rule (Chapter 3). (Section 0.2.)
- **Gravitational term**: We call $M = 3H\gamma^{(x_0)} + \frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ the gravitational term. (Section 7.18.)
- **Great circle**: Great circle: a circle on the sphere whose centre is the centre of the sphere (the equator, the meridians, and all their rotations). (Sections 3.16 and 3.21.)
- **Greatest common divisor** $\gcd(p, q)$: the largest whole number that divides both p and q (`math.gcd`). (Sections 1.2 and 1.8.)

- **Grid**: the equally spaced times $t_n = nh$, $n = 0, 1, \ldots, N$, with step $h = T/N$. (Sections 7.3, 7.8, 16.11 and 16.18.)
- **Grid error**: the difference between a number computed on a grid and the exact number. For the reference it shrinks like h^2. (Section 16.11.)
- **Grid, finite differences**: the line is replaced by equally spaced points; the second derivative is replaced by the difference formula $(f_{k+1} - 2f_k + f_{k-1})/h^2$. (Section 13.35.)
- **Ground states**: Ground states ($T = 0$): three particle numbers $N = 8, 136, 688$, five couplings and five slices $a_{4,0} = 0, 0.5, 1, 1.5, 2$, that is $3 \cdot 5 \cdot 5 = 75$ states. (Sections 13.1 and 15.2.)
- **The ground states without interaction** (Notebook 15a, Out [7]; `Revision/kohn_sham/results/ground/summary.csv`): (Section 15.15.)
- **Group**: a set of invertible matrices that contains the products and the inverses of its members. (Sections 5.11, 5.16 and 19.21.)
- **Group, Pin(4,4)**: a group is a collection of transformations that can be combined and undone; Pin(4,4) is the group of the transformations of the sixteen components of a field that go with the rotations and reflections of the eight directions, rotations in a wide sense that includes the mixing of space-like and time-like directions (Chapter 5). (Section 0.2.)
- **Growth rate**: κ is the growth rate. (Sections 7.5, 8.18 and 8.22.)
- **Guide line**: For each ground state (`zip` pairs the colours with the ids) the left axes draw the distances $|E_{KS}(G) - R|$ against h with logarithmic axes (`loglog`: equal distances on the axis are equal factors), as circles joined by lines (`"o-"`, line width 1.5, marker size 6), and a dashed guide line proportional to h^2 that starts a factor 3 below the first point. (Section 16.12.)

H

- **Half nodes**: The cell ends, the nodes, are $y_i = -L + i\,h$ for $i = 0, \ldots, G$; the cell centres, the half nodes, are $y_{p+1/2} = -L + (p + \frac{1}{2})\,h$ for $p = 0, \ldots, G - 1$. (Sections 16.14 and 16.18.)
- **Half-density term**: The record calls it the half-density term (volume and vielbein divergence). (Sections 6.29 and 8.5.)
- **Hamiltonian**: The energy operator of one particle in an external potential $v(x)$ is the Hamiltonian. (Sections 10.11 and 13.2.)
- **Hamiltonian density**: The rest of the Lagrangian has no x_4-derivative; the Revision record writes it as $-\mathcal{H}$ with the Hamiltonian density $\mathcal{H} = \cos z\,[\Psi^\dagger(mC - \sum_{\mu \neq x_4} C\gamma^\mu D_\mu)\Psi + U(S)]$, so that $\mathcal{L} = \Psi^\dagger K \partial_4 \Psi - \mathcal{H}$ exactly, up to the total derivative (`python-field-theory.json`, check `hamiltonian_form_and_heisenberg_equation`). (Section 10.12.)
- **Harmonic trap**: the external potential $v(x) = x^2/2$, a parabola; a particle in it has the levels $1/2, 3/2, 5/2, \ldots$ (in the units below). (Sections 13.32 and 13.35.)
- **Hartree**: The first term is a Hartree (direct) term, the second an exchange term. (Sections 13.4 and 14.26.)

- **Hartree approximation**: The older Hartree approximation keeps v_H and drops the exchange integral, and with it the cancellation of the self-interaction. (Section 13.6.)
- **Hartree energy**: E_H is the Hartree energy: the classical energy of the cloud n with itself. (Sections 13.5 and 13.19.)
- **Hartree only**: (b) Hartree only: drop the exchange, $v_s = v + g_c n$; every fermion is then also repelled by its own density (the self-interaction of Section 13.5). (Section 13.32.)
- **Hartree potential**: the potential that one particle feels from the average density of all particles; exchange potential: the correction that comes from the Pauli principle (equal-label particles avoid each other). (Sections 13.6 and 13.35.)
- **Hartree term, exchange term**: the direct and the exchange part of the interaction energy. (Section 14.30.)
- **Hartree-Fock**: Hartree-Fock: keeping both, for a quasi-free state. (Sections 13.13 and 14.30.)
- **Hartree-Fock approximation**: The Hartree-Fock approximation takes the best single determinant: it minimises $\langle\Phi|\hat{H}|\Phi\rangle$ over all choices of N orthonormal orbitals. (Section 13.6.)
- **Hash values**: When Python prints a set, the order of its members depends on their hash values, numbers computed from the names with a secret key that Python chooses anew every time it starts, unless the environment variable `PYTHONHASHSEED` (a named setting that a program receives when it starts) fixes the key. (Section 0.22.)
- **Headless**: A notebook can also be run without a browser (headless) with `jupyter nbconver t`; the run instructions print the exact command. (Section 0.8.)
- **Heat capacity** $C_V = dE/dT = T\,dS/dT$ at fixed N: how much energy one unit of temperature costs. (Sections 13.26, 13.30, 15.26 and 15.29.)
- **Heat map**: a picture of a matrix in which every entry is a coloured square. (Sections 0.16, 1.18, 1.24, 4.7, 5.9, 6.13, 6.21, 6.27, 7.17, 9.16, 9.17, 10.10, 13.14, 14.11, 15.24, 16.19, 17.2, 18.9, 19.15 and 20.11.)
- **Heisenberg equation**: Differentiating with the product rule, and using that H commutes with $e^{\pm iHx_4}$ (both are power series in H), gives the Heisenberg equation. (Section 10.11.)
- **Hellmann-Feynman rule**: for a normalised eigenvector, the derivative of the level is the expectation value of the derivative of the Hamiltonian: $d\varepsilon/dk = \langle\chi|\,\partial h/\partial k\,|\chi\rangle$. (Sections 14.24 and 19.2.)
- **Hellmann-Feynman theorem**: the derivative of a level with respect to a parameter equals the matrix element of the derivative of the Hamiltonian. (Sections 15.21, 15.24 and 16.15.)
- **Hermitian**: $M^\dagger = M$. A Hermitian matrix has real eigenvalues. (Sections 1.36, 4.9, 4.11, 5.4, 5.9, 7.12, 7.16, 8.17, 8.22, 8.35, 9.25, 10.4, 10.9, 10.11, 13.2, 13.6, 14.4, 21.2, 21.14 and 22.6.)
- **Hermitian form, charge density**: a form $\Psi^\dagger X \Psi$ with a Hermitian matrix X; it is real for every Ψ. A charge density is the time component of a current. (Section 10.37.)
- **Hermitian matrix**: $H^\dagger = H$; a real Hermitian matrix is a symmetric matrix. (Sections 1.40 and 14.10.)

- **Hermitian part, anti-Hermitian part**: every matrix is $h = h_H + h_A$ with $h_H = \frac{1}{2}(h + h^\dagger)$ Hermitian and $h_A = \frac{1}{2}(h - h^\dagger)$ anti-Hermitian ($h_A^\dagger = -h_A$). (Section 10.42.)
- **Hex code**: Each colour is written as a hex code: `#` and three pairs of hexadecimal digits for the amounts of red, green and blue. (Section 3.13.)
- **Hexadecimal**: writing a number with the 16 digits `0-9` and `A-F` (worth 10 to 15); Python writes such a number with the prefix `0x`. One hexadecimal digit is exactly 4 bits. (Sections 0.18, 0.20, 0.24, 1.49 and 1.53.)
- **Hexadecimal colour codes**: The colours are written as hexadecimal colour codes: after `#`, three pairs of hexadecimal digits give the amounts of red, green and blue (from 00, none, to ff, 255). (Sections 0.17 and 17.11.)
- **Hidden**: The first, x_0, is a hidden direction of space. (Sections 0.1, 3.1, 3.12, 3.34, 7.26, 8.2, 9.2, 12.1, 12.15 and 21.2.)
- **Hidden angle** $z = 6Hx_0$: the metric depends on x_0 only through z, which runs from 0 to $\pi/2$ (the patch). $H > 0$ is a constant of the author's metric with the unit of an inverse length. (Section 12.2.)
- **Hidden coordinate** y: the book's coordinate along the hidden direction x_0, $y = \ln(\sin z)/(6H)$ with $z = 6Hx_0$; $y = 0$ is the end of the patch $z = \pi/2$ (the *brane*), and the interval is cut off at $y = -L$ (the *tip*). (Sections 2.24, 2.27, 3.12, 3.34, 8.28, 8.35, 14.10, 14.17, 17.2, 17.10, 17.20, 19.2, 19.15 and 19.20.)
- **Hidden direction**: Coordinates $x0, \ldots, x7$: the eight numbers that name a point, with the author's names: $x0$ the hidden direction; $x1, x2, x3$ ordinary 3-space; $x4$ the time; $x5, x6, x7$ the three extra times. (Sections 6.13, 6.27 and 10.31.)
- **Hidden equation**: Their four independent parts are the constraint (time, x_4), the 3-space equation, the extra-time equation and the hidden equation (x_0). (Section 12.2.)
- **Hidden position** y: the coordinate $y = \ln(\sin z)/(6H)$ by which the Kohn-Sham record measures the hidden direction, so that $\sin^{1/6} z = e^{Hy}$. The patch end $z = \pi/2$ is $y = 0$, and $y < 0$ lies towards the tip. (Section 10.42.)
- **Hilbert adjoint** X^* of an operator: the conjugate transpose of its matrix (with the ordinary, positive inner product of the Fock space). (Sections 5.34, 5.37, 10.11, 10.18 and 21.29.)
- **Hilbert norm**: The ordinary size of a column u is the Hilbert norm $u^\dagger u = |u_1|^2 + \cdots + |u_{16}|^2$. (Sections 8.19, 8.22 and 8.28.)
- **Hilbert space**: A space of states with a positive inner product is a Hilbert space. (Sections 10.11 and 10.18.)
- **Histogram**: bars that show how many numbers fall into each interval. (Sections 0.21, 1.32, 9.24, 9.25, 10.19 and 21.23.)
- **History** $a_4 = AHx_4$, $b_4 = 1$: the time dependence of the metric used for the numbers, the one of the Revision Kohn-Sham work ($A = 1$, $H = 1$); with $A > 0$ the extra times deflate. (Sections 3.34, 8.28, 11.2, 11.26, 14.24, 15.13, 17.10, 17.20 and 22.11.)

- **Hohenberg-Kohn theorem**: the ground-state density determines the external potential (up to a constant); Kohn-Sham inversion: finding the potential of NON-interacting electrons that gives the same density. (Section 13.13.)
- **Hole**: an empty place in a level that is normally occupied. (Sections 13.27 and 15.29.)
- **Holonomy** (turning angle): the angle by which a parallel-transported vector is turned after going once around a closed curve. (Section 3.21.)
- **HOMO**: `homo_lumo` returns the highest occupied level (HOMO: occupation above one half) and the lowest empty one (LUMO). (Sections 13.35, 15.13, 16.11, 19.20 and 19.21.)
- **Homogeneous**: Take a homogeneous configuration: one that depends on the time x_4 only, so that $\partial_\mu \Phi = 0$ for every $\mu \neq x_4$. (Sections 9.12 and 18.21.)
- **Homogeneous configuration**: A condensate (also called a homogeneous configuration) is a field that is the same at every place: it depends on the time x_4 only, not on $x_1, x_2, x_3, x_5, x_6, x_7, x_0$. (Section 9.18.)
- **Homogeneous field**: a field that does not depend on $x_1, x_2, x_3, x_5, x_6, x_7, x_0$, only on the time x_4. (Section 18.26.)
- **Homogeneous of degree 2**: e_{int} is homogeneous of degree 2: doubling both densities multiplies it by 4. (Section 14.27.)
- **Honesty ledger**: The honesty ledger is the table of the main statements of the book, each with its label (Section 0.3), a short note where one is needed, and the reports of the Revision record that verify it. (Section 0.18.)
- **Honesty rule**: The book therefore follows one rule above every other, the honesty rule: it teaches exactly what the Revision record proves and computes, with every assumption, and it never writes "proved" for a statement that is not proved. (Section 0.2.)
- **Hyperbola**: Since $\cosh^2 t - \sinh^2 t = 1$, the points $(\pm\cosh t, \sinh t)$ lie on the hyperbola $a^2 - b^2 = 1$ (blue, the columns of Krein norm $+1$) and the points $(\sinh t, \pm\cosh t)$ on $a^2 - b^2 = -1$ (orange). (Sections 1.13, 6.16 and 10.19.)
- **Hyperbolic cosine**: the function $\cosh\varphi = (e^\varphi + e^{-\varphi})/2$. (Section 1.13.)
- **Hyperbolic functions**: $\cosh\varphi = (e^\varphi + e^{-\varphi})/2$ and $\sinh\varphi = (e^\varphi - e^{-\varphi})/2$; they satisfy $\cosh^2\varphi - \sinh^2\varphi = 1$. (Sections 1.16, 5.21, 6.21 and 8.3.)
- **Hyperbolic sine**: the function $\sinh\varphi = (e^\varphi - e^{-\varphi})/2$. (Section 1.13.)
- **Hyperbolic sine and cosine**: We use the hyperbolic sine and cosine, $\sinh t = (e^t - e^{-t})/2$ and $\cosh t = (e^t + e^{-t})/2$: each is the derivative of the other, $\sinh 0 = 0$, and both are positive for $t > 0$. (Section 16.13.)
- **Hypotheses** (`Revision/pairing/kohn_sham/t3-theory.json`, hypotheses H1 to H6, in words): (Sections 19.10, 20.5 and 20.7.)
- **HYPOTHESIS**: a status label: an idea, such as a proposed mechanism or scenario, that is stated and examined but that this book does not establish. Example: the author's Hypothesis P, that the big bang creates universes of masses $+M$ and $-M$ in pairs (Section 20.3). (Sections 0.3, 0.20, 5.39, 12.31, 15.31, 20.3, 21.19, 21.37, 22.2 and 22.27.)

- **Hypothesis P**: Hypothesis P ("the big bang creates universes of masses $+M$ and $-M$ in pairs") therefore remains what it was: a HYPOTHESIS. (Section 20.26.)

I

- **Identity**: The identity 1 has ones on the diagonal and zeros elsewhere: $1A = A1 = A$. (Sections 4.19, 10.9 and 21.14.)
- **Identity in the jets**: a polynomial whose every coefficient is zero. It is zero for EVERY value of the jet, that is, for every field at every point, whether the field solves the field equations or not (*off shell*). An identity off shell holds in particular for solutions (*on shell*). (Section 18.19.)
- **Identity matrix** I: the square matrix with 1 on the diagonal and 0 elsewhere. (Sections 1.12, 1.18, 1.24, 4.2, 4.7 and 5.9.)
- **If**: Hence the total charge $Q = \int \cos z \, \Psi^\dagger B \Psi \, d^7x$ of one solution (at $b_4 = 1$; for a general b_4 it carries the factor $b_4^{3/2}$) is constant in time if its current flux through the boundary of the seven other directions vanishes (Section 5.6); this is a conditional statement, and its condition is not established (next list). (Section 5.39.)
- **Ill-conditioned**: The problem is ill-conditioned: its answer reacts strongly to small errors of its input. (Section 16.20.)
- **Illinois method**: All brackets are refined together, 16 rounds, by the Illinois method. (Section 19.16.)
- **Image**: the orbital, state or problem obtained by applying the map of T3. (Sections 5.11, 19.2 and 19.10.)
- **Imaginary part**: The number $a = \operatorname{Re} z$ is its real part and $b = \operatorname{Im} z$ its imaginary part. (Sections 1.10 and 1.16.)
- **Imaginary unit** i: a new number with $i^2 = -1$. Python writes it `1j`, sympy writes it `I`. (Sections 1.10 and 1.16.)
- **Indefinite**: a quadratic expression that can take both signs. (Sections 1.37, 5.34 and 21.22.)
- **Indefinite (Krein) metric**: a rule for the squared length of a quantum state that can also give negative numbers (Chapter 10). (Section 0.2.)
- **Independent universes**: two quantum fields Ψ_1, Ψ_2 on one state space with $\{\Psi_{1A}, \Psi_{2C}^\dagger\} = 0$ and $\{\Psi_{1A}, \Psi_{2C}\} = 0$: nothing done to one changes the anticommutation rules of the other. (Section 10.48.)
- **Index**: a letter such as i, j or k that numbers the components of a vector or the rows and columns of a matrix. In mathematics rows and columns are numbered $1, 2, 3, \ldots$; Python numbers them $0, 1, 2, \ldots$, so the entry A_{11} (row 1, column 1) is `A[0, 0]` in Python. (Sections 1.18, 1.24, 1.26, 1.32 and 3.2.)
- **Index list**: an ordered list of labels, such as (x_1, x_3, x_3); its length is the number of entries (here 3). (Sections 1.42, 1.47, 11.2 and 11.8.)

- **Index notation**: $a, b, c, \ldots$ each run over the eight coordinates $x_0, \ldots, x_7$; an upper and a lower index are positions in a table of numbers; ∂_c means the partial derivative $\partial/\partial x_c$. (Sections 1.26 and 3.29.)
- **Index up, index down**: a tensor component carries upper and lower coordinate labels, such as $R^{ab}{}_{cd}$; a label is raised with the inverse metric. (Sections 11.18 and 12.2.)
- **Inertia**: On an eigenspace of a real frequency $E \neq 0$, by contrast, the Krein form has four positive and four negative directions: its inertia is (4,4) (PROVED in the pairing record for every real frequency; `python-pairing.json`, checks `Q.one_particle_Krein_inertia_proof` and, for an exact sample, `Q.one_particle_Krein_inertia`). (Sections 8.19 and 10.9.)
- **Inflate**: The author's metric (the rule that gives the length of a small step in each direction; Section 0.14 introduces it from zero) makes ordinary space grow (inflate) and the three extra times shrink (deflate) exponentially as the time x_4 runs. (Sections 0.1, 0.14, 0.16, 3.5, 3.12, 4.4, 8.2, 11.26 and 14.2.)
- **Inflating**: Inflating means growing, deflating shrinking. (Sections 8.28 and 14.10.)
- **Initial values**: Given the values at one point (the initial values) it fixes the functions everywhere. (Section 15.8.)
- **Initial-value (Cauchy) problem**: find the field at later times x_4 from its values at $x_4 = 0$ (the *data*). (Section 8.22.)
- **Initial-value problem**: an ODE together with the starting value $y(0)$. Exactly one solution belongs to it (for the smooth f of Notebook 02a). (Sections 2.2, 2.10, 8.16 and 12.24.)
- **Inner product**: The inner product of two states ϕ and ψ is the complex number. (Sections 10.11, 10.18 and 13.2.)
- **Instability**: An imaginary part of a frequency is the mathematical sign of an instability; Chapter 8 meets it in the waves along the extra times. (Section 1.13.)
- **Instantaneous**: computed with the coefficients of one instant, as if they did not change. (Sections 10.39, 10.42, 15.21, 19.2 and 22.2.)
- **Instantaneous (adiabatic) state**: the ground state of the Hamiltonian $h(a_{4,0})$ of one instant of the history (one slice $a_{4,0}$); the history itself keeps moving, with 3-space inflating and the extra times deflating. (Section 15.24.)
- **Instantaneous level and orbital**: a solution $h\varphi = \varepsilon\varphi$ of the Hamiltonian h of one slice. (Section 22.11.)
- **Integers**: The whole numbers (or integers) are $\ldots, -2, -1, 0, 1, 2, \ldots$. (Section 1.2.)
- **Integral over the patch** $\int X$: the total amount of a density X in the computed region, $2\,\mathrm{Vol}_7 \int_{-3}^{0} e^{6Hy} X \, dy$ (Section 17.7). (Sections 17.2 and 17.10.)
- **Integration by parts**: the product rule $\frac{d}{dt}(fg) = \dot{f}g + f\dot{g}$ integrated from 0 to T: $\int_0^T f\dot{g}\,dt = [fg]_0^T - \int_0^T \dot{f}g\,dt$, where $[fg]_0^T = f(T)g(T) - f(0)g(0)$ is the boundary term. (Sections 7.2 and 7.8.)

- **Interaction**: The interaction $-U(S) = -\frac{\lambda}{2}S^2$ with a real coupling λ is quartic in the field. (Section 7.19.)
- **Interaction energy density** $e_{int} = \lambda(\frac{15}{32}S^2 - \frac{1}{32}n^2)$: the energy of the contact interaction per proper 7-volume. (Sections 15.19 and 19.4.)
- **Intermediate value theorem**: If F is continuous, it has a zero between them: this is the intermediate value theorem of calculus (ASSUMED: quoted without proof). (Section 2.12.)
- **Intertwiner**: a matrix X with $\gamma^a X = X\hat{\gamma}^a$ for every a. (Sections 4.17, 4.19, 5.11 and 5.16.)
- **Intertwiner condition**: A charge-conjugation matrix is a constant 16×16 matrix $\mathcal{C}$ for which the matrix $M = \mathcal{C}C$ obeys the intertwiner condition. (Sections 5.28, 5.32, 21.2, 21.8 and 21.14.)
- **Invariant**: Three facts follow from the definition, by the computation just made: a contraction of one upper with one lower index of a tensor is again a tensor (with two fewer indices); a product of tensors is a tensor; and a quantity in which every index is contracted, an invariant (or scalar), has the same value in every system of coordinates. (Sections 3.14, 3.29, 5.11, 10.33 and 10.37.)
- **Invariant mass** μ of a collection of bodies: $\mu^2 = E_{\text{tot}}^2 - |\vec{p}_{\text{tot}}|^2$, computed from the totals. Because the totals are conserved, so is μ. For one body μ is its mass. (Sections 20.17 and 20.20.)
- **Inverse** M^{-1}: the matrix with $MM^{-1} = M^{-1}M = 1$. When $MM = 1$, the matrix is its own inverse. (Sections 1.19, 4.2, 5.9 and 6.13.)
- **Inverse metric**: Metric $g_{\mu\nu}$, inverse metric $g^{\mu\nu}$ (the inverse matrix: $\sum_\lambda g^{\mu\lambda}g_{\lambda\nu} = \delta^\mu{}_\nu$, which is 1 when $\mu = \nu$ and 0 otherwise). (Sections 3.29, 11.2 and 11.18.)
- **Inverse vielbein**: The inverse vielbein $e_a{}^\mu$ is the inverse matrix, $\sum_\mu e^a{}_\mu e_b{}^\mu = \delta^a_b$ (1 when $a = b$, 0 otherwise). (Section 6.2.)
- **Inversion**: a pair of places in a permutation whose two entries stand in the wrong order (the larger one first). (Sections 1.19, 1.24, 1.47, 4.2, 11.2, 11.8 and 18.9.)
- **Invertible**: A square matrix M is invertible if there is a matrix M^{-1} with $MM^{-1} = M^{-1}M = I$; then the equation $Mv = w$ can be undone, $v = M^{-1}w$. (Section 1.20.)
- **Involution**: A map is an involution when applying it twice gives back the start. (Sections 19.2 and 19.9.)
- **Irrational**: A number that is not a fraction is called irrational; its decimal digits never end and never repeat (by the formula of Section 1.2, a number whose digits end or repeat is a fraction). (Sections 1.3 and 1.8.)
- **Irreducible**: The representation is irreducible when there is no other invariant subspace: the 16 components cannot be cut into smaller pieces that the group keeps apart. (Sections 5.11, 5.16 and 21.9.)
- **Isometry**: a change of coordinates that does not change the metric. (Sections 18.2, 18.19 and 20.22.)

- **Isotropic**: For a configuration that looks the same in the three directions of 3-space (such a configuration is called isotropic in 3-space), $T^{x_1}{}_{x_1} = T^{x_2}{}_{x_2} = T^{x_3}{}_{x_3} = p_3$, and likewise p_t for a configuration isotropic in the extra times. (Section 9.4.)

- **Iteration**: One iteration: (1) from the input x, solve every level of the label set (Section 15.3); (2) fill them (aufbau at $T = 0$, Fermi-Dirac at $T > 0$); (3) build the densities n and S and from them the output potentials $x_{out} = (\frac{15}{16}\lambda S,\ -\frac{1}{16}\lambda n)$; (4) the residual is $r = x_{out} - x$, and its size is its largest entry in absolute value. (Sections 13.35 and 15.5.)

J

- **Jacobi identity**: Multiplying out shows the Jacobi identity $[[X, Y], Z] = [X, [Y, Z]] - [Y, [X, Z]]$ for any three matrices (both sides are $XYZ - YXZ - ZXY + ZYX$). (Section 5.23.)

- **Jacobi's formula** for the derivative of the volume factor of a diagonal metric: (Section 2.19.)

- **Jacobian matrix**: The $n \times n$ table of partial derivatives $\partial x'^a/\partial x^b$ is the Jacobian matrix of the change. (Section 3.14.)

- **Janak's theorem**: the derivative of the self-consistent energy with respect to the occupation f_a of an orbital is its level ϵ_a. (Section 13.30.)

- **Jet**: the value of a field component and its derivatives at one point, treated as independent symbols (ψ_A, $\partial_\mu \psi_A$, ...). (Sections 7.13, 7.26, 7.32, 9.17, 18.2, 18.19 and 20.10.)

- **Jet at a point**: the values of the field and of its eight first derivatives at one point. The energy-momentum tensor needs nothing else. (Section 9.16.)

- **Jitter**: `rng.uniform(-0.28, 0.28, size=...)` gives one random number between -0.28 and 0.28 per ratio; adding it to the class's position spreads the dots a little sideways so that they do not hide one another (jitter). (Section 16.12.)

- **Job**: A ground-state job (one unit of work of the program) solves the state, its Delta-SCF excited state (one quantum moved from the highest occupied to the lowest empty group of levels), four neighbouring slices $a_{4,0} \pm 0.002$ and $a_{4,0} \pm 0.004$ at fixed occupations (for the derivative dE/da_4 and the adiabaticity measure of Chapter 15), and the list of the lowest particle-hole excitations, each on the three grids. (Section 16.3.)

- **Jordan-Wigner**: The sign rule that makes them anticommute is the Jordan-Wigner rule: acting on mode j gives the factor (-1) to the power of the number of occupied modes before j. (Sections 21.26 and 21.29.)

- **JSON** and **CSV**: two text formats for stored numbers (JSON: named entries in braces; CSV: a table, one line per row, entries separated by commas). (Sections 0.16, 16.11 and 23.11.)

- **Junction condition**: an equation that says how the fields and the geometry must match across a surface, here across the brane $z = \pi/2$. The junction conditions at the brane are not derived in this book: the Z2 mirror continuation is ASSUMED, and deriving the conditions is an open problem (OPEN, Section 22.15). (Sections 18.2, 22.2 and 22.15.)

- **Jupyter notebook**: Every worked example of the book is a Jupyter notebook: a file (with the ending `.ipynb`) that holds text and Python code in cells, together with everything the code printed and drew when it ran. (Section 0.7.)
- **JupyterLab** shows a notebook in your web browser and runs it: with the environment active, in the folder `Revision/textbook/notebooks`, the command `jupyter lab` followed by the name of the notebook file opens it, and the menu Run > Run All Cells runs every cell from the top. (Section 0.8.)

K

- **Kernel**: The kernel is the running Python program that executes the code cells one after the other. (Sections 0.7, 5.11 and 18.2.)
- **Key**: `read_levels` reads the level table of one state and returns a dictionary whose key is the key of a level, the four numbers (shell n_2, block type j, parity, Pruefer label l), and whose value is the pair (level, occupation). (Section 15.14.)
- **Key rule**: With line 10 this gives the key rule for all 64 pairs $A, B = 0, \ldots, 7$: $\tau_A \bar{\tau}_B + \tau_B \bar{\tau}_A = \bar{\tau}_A \tau_B + \bar{\tau}_B \tau_A = 2\,\mathrm{eta4488}_{AB}\, I_8$. (Section 4.5.)
- **Kinetic numbers**: The eight kinetic numbers $\Psi^T C \gamma^a \Psi$ are computed before and after the map Γ, and so is the scalar: $S = \Psi^T C \Psi$ before and $(\Gamma\Psi)^T C (\Gamma\Psi)$ after. (Section 21.15.)
- **Kinetic part**: The terms with derivatives (the K's) form the kinetic part; the terms without derivatives, $mS + U(S)$, form the potential part. (Section 9.7.)
- **Kinetic term** K_μ: the part of the Lagrangian that holds the derivative along the direction μ; potential energy: $mS + U(S)$, the part without derivatives. (Sections 7.19, 9.16 and 18.4.)
- **Kinetic term of the direction**: The number K_μ (no sum) is called the kinetic term of the direction x_μ: the part of the Lagrangian that holds the derivative along x_μ. (Section 9.3.)
- **Kinetic-sum identity**: This kinetic-sum identity holds off shell (PROVED; check `kinetic_sum_on_shell_C`). (Section 9.8.)
- **Kink**: Kink: a point where a function is continuous but its slope jumps. (Section 22.2.)
- **Klein's inequality**: $D = \mathrm{Tr}[\hat{\rho}(\ln\hat{\rho} - \ln\hat{\sigma})] \geq 0$ for two density operators, with equality only when they are equal. (Sections 13.26 and 13.30.)
- **Klein-Gordon equation**: So $\partial_4^2\phi = \partial_1^2\phi - m^2\phi$: the wave equation with a mass, called the Klein-Gordon equation. (Section 7.5.)
- **Kohn-Sham**: The spinor fields of Chapters 4 to 10 need its vielbein (the scale factors of Section 3.7); the energy-momentum tensor of Chapter 9 needs its Christoffel symbols (Section 3.23); the field equations of the metric functions a_4 and b_4 in Chapter 12 need its Einstein tensor (Section 3.25); the Kohn-Sham model of Chapters 14 to 17 (Kohn-Sham: an approximation of density functional theory that replaces many interacting particles by independent particles, each moving in one common effective potential; Chapter 13 teaches it from zero) is written in the hidden coordinate y of Section 3.9. (Section 3.1.)

- **Kohn-Sham approximation**: Rows 8, 9, 12 and 13 rest on the Kohn-Sham approximation, which is ASSUMED, not derived. (Section 0.18.)
- **Kohn-Sham energy**: and the Kohn-Sham energy of the doubled system is $E_{KS} = \sum g\, f\, \varepsilon - 2\,\mathrm{Vol}_7 \int_{-L}^{0} e^{6Hy} e_{int}\, dy$. (Section 15.2.)
- **Kohn-Sham equations**: one-particle Schroedinger equations in an effective potential $v_s(x)$ chosen so that their orbitals give the density of the interacting system. (Sections 13.10 and 13.35.)
- **Kohn-Sham gap** Δ_{KS}: the smallest cost of moving one particle from an occupied Kohn-Sham orbital to an empty one, $\Delta_{KS} = \epsilon_{LUMO} - \epsilon_{HOMO}$ (lowest unoccupied minus highest occupied level). In an interacting system the levels move when the occupations change, so Δ_{KS} is only a first estimate of the lowest excitation energy. (Sections 13.27, 15.13 and 19.20.)
- **Kohn-Sham inversion**: Hohenberg-Kohn theorem: the ground-state density determines the external potential (up to a constant); Kohn-Sham inversion: finding the potential of NON-interacting electrons that gives the same density. (Section 13.13.)
- **Kohn-Sham potential**: the derivative of the interaction energy density with respect to a density; it enters the one-quantum equation. (Sections 13.10 and 14.30.)
- **Kohn-Sham state**: an approximate state of many identical fermions, built from one-particle wave functions (*orbitals*) that each solve a one-particle equation in a common *effective potential*; the potential depends on the densities of the occupied orbitals, so the equations must be solved *self-consistently*. (Sections 15.13, 17.2, 17.10, 17.20, 19.2 and 19.20; the opening of Chapter 19.)
- **Kohn-Sham universe**: a solution of the equations of the density-functional approximation, in which the field moves in a potential computed from its own density; self-consistent means that this potential and this density fit each other; the mass of the field in this model is written M (Chapters 13 to 15). (Section 0.2.)
- **Krein**: Because B has eight eigenvalues $+1$ and eight -1, the state space carries an indefinite Krein form when $\Psi^\dagger$ is read as the ordinary adjoint (Chapter 10). (Section 18.22.)
- **Krein (indefinite) form**: the form $\Phi^\dagger B \Phi$ built with the matrix B, which has eight eigenvalues $+1$ and eight eigenvalues -1; it can be positive or negative. (Section 9.24.)
- **Krein boost**: A Krein boost of a pair, $u_1' = \cosh t\, u_1 + \sinh t\, u_9$ and $u_9' = \sinh t\, u_1 + \cosh t\, u_9$ (with $Bu_1 = u_1$, $Bu_9 = -u_9$, both of length 1 and orthogonal), keeps the Krein norms, because $\cosh^2 t - \sinh^2 t = 1$, and keeps the two modes Krein-orthogonal, because $\cosh t \sinh t - \sinh t \cosh t = 0$; but it changes the ordinary squared length to $\cosh^2 t + \sinh^2 t$. (Section 10.14.)
- **Krein form**: the number $u^\dagger B v$ for two columns u, v. Because B is Hermitian, $u^\dagger B u$ is real, but it can be positive, negative or zero. In the theory it is the charge of the wave u. (Sections 8.19, 8.22, 8.28, 10.2, 10.9, 10.18 and 21.29.)
- **Krein form, Krein charge**: $\int \cos z\, u^\dagger B v\, dx_0$; for $u = v = \Psi$ it is the charge of the wave. (Section 10.31.)

- **Krein inertia** of a set of columns: how many independent directions have a positive and how many a negative Krein norm. (Sections 9.21, 10.5, 10.42, 18.26, 21.26 and 21.29.)
- **Krein matrix** $B = -iC\gamma^{(x_4)}$: the charge density is $\Psi^\dagger B\Psi$, and B is the matrix of the canonical anticommutator of the quantised field (Chapter 10). (Sections 18.2, 20.2, 21.2, 21.7, 21.14 and 21.29.)
- **Krein norm** of a column u: the real number $u^\dagger Bu$, which can be positive or negative because B has eight eigenvalues $+1$ and eight -1. (Sections 10.2, 18.2 and 18.26.)
- **Krein self-adjoint**: In words, h is Krein self-adjoint: $Bh = h^\dagger B$. (Section 10.4.)
- **Krein sign** σ of a map M: the sign in $MBM^\dagger = \sigma B$. (Sections 14.5, 18.2, 18.8 and 19.2.)
- **Krein space**: a space with an *indefinite* inner product (some vectors have a negative norm). (Sections 1.37, 5.34, 5.37, 10.13, 10.18 and 21.25.)
- **Krein-neutral**: So the Krein form vanishes identically on the eigenspace of a growing frequency: the eigenspace is Krein-neutral (PROVED by the argument just given; the pairing record proves the same and checks it on exact samples, `Revision/pairing/reports/python-pairing.json`, check `Q.one_particle_complex_frequency_Krein_neutral`). (Sections 8.19, 10.5, 10.42, 18.23, 18.27, 21.26 and 21.29.)
- **Krein-orthonormal modes**: columns u_n with $u_n^\dagger Bu_m = \epsilon_n\delta_{nm}$, $\epsilon_n = +1$ or -1. A *Krein boost* mixes a mode of Krein norm $+1$ with one of -1 by the numbers $\cosh t$ and $\sinh t$; it keeps the Krein form but changes the ordinary length. (Section 10.18.)
- **Krein-unitary**: We call such an R Krein-unitary. (Sections 10.34 and 10.37.)
- **Kretschmann scalar** $K = \sum_{a,b,c,d} R^{ab}{}_{cd}R^{cd}{}_{ab}$: one number per point, the same in every coordinate system (an invariant). (Sections 3.17, 3.21 and 3.29.)
- **Kronecker delta** δ^a_b: 1 if $a = b$ and 0 otherwise; as a table it is the identity matrix I. (Sections 1.18, 1.24, 1.32, 1.42, 1.47, 4.2, 4.7, 11.2 and 11.8.)
- **Kronecker product** `np.kron(L, R)`: the big matrix whose block in block row r and block column k is L_{rk} times the whole matrix R. (Sections 4.17, 4.19, 5.13, 5.16, 10.38 and 21.15.)
- **KS gap**: HOMO, LUMO: the highest occupied and lowest unoccupied level; the KS gap is their difference. (Section 16.11.)

L

- **L1 measure**: The record measures the remaining part in two ways: $\mu_1 = \int e^{6Hy}|X - \bar{X}|\,dy / \int e^{6Hy}|X|\,dy$ (the L1 measure) and $\mu = \sqrt{D(\bar{X})/D(0)}$ (the L2 measure); both are 0 for a flat profile. (Section 17.17.)
- **L2 measure**: The record measures the remaining part in two ways: $\mu_1 = \int e^{6Hy}|X - \bar{X}|\,dy / \int e^{6Hy}|X|\,dy$ (the L1 measure) and $\mu = \sqrt{D(\bar{X})/D(0)}$ (the L2 measure); both are 0 for a flat profile. (Section 17.17.)

- **Label**: a name of a coordinate, here $x_0, \ldots, x_7$. The computer often numbers them $0, \ldots, 7$; only the question "are two labels equal?" matters below, so the names do not change any result. (Sections 1.42, 1.47, 2.24, 2.27, 11.2, 13.19, 13.35, 14.13, 14.14, 14.24, 15.8, 16.11, 19.21, 22.2 and 22.11.)

- **Label set**: For a state of the gas the solver chooses the sectors and labels it will solve, its label set. (Section 15.4.)

- **Lagrange multiplier**: The number λ is a Lagrange multiplier; with several conditions one takes one multiplier for each. (Section 13.6.)

- **Lagrangian**: the function from which the equations of a field follow (Chapter 7). (Sections 0.2, 7.2, 7.8 and 10.48; the opening of Chapter 7.)

- **Lagrangian density** L: the function whose integral (the action) must not change to first order when the field is varied; that requirement is the Euler-Lagrange equation, the field equation. (Sections 7.5, 7.8 and 7.26.)

- **Lambda**: A lambda is a function written in one line without a name: `lambda e: ...` takes an entry `e`, multiplies it out, replaces every w^2 by $m^2 + k_s^2 - k_t^2$ (the expression `w2_sym` of In [6]) and multiplies out again. (Sections 10.10, 16.12 and 16.19.)

- **Laplace's rule**: Laplace's rule for determinants of every size, and the expansion of every Lovelock tensor along the column of its free upper label, $P_{(k)} = \delta\, L_{(k)} - 2k\, Y_{(k)}$ (Section 11.13; Notebook 11b, In [21]). (Section 11.28.)

- **Laurent polynomial**: The program represents every quantity as an exact Laurent polynomial (a sum of monomials that may contain negative powers, such as $\cot^{-1} z$) in H, a_4', a_4'' (and the higher derivatives), e^{a_4}, $\sin^{1/3} z$ and $\cot z$, with coefficients that are exact fractions of whole numbers (the files `poly.rs` and `rational.rs` of the crate; every operation that would overflow stops the program instead of rounding). (Sections 11.12, 11.18 and 12.16.)

- **Laws of powers**: Power a^p and root $a^{1/n}$; the laws of powers $a^p a^q = a^{p+q}$, $(a^p)^q = a^{pq}$, $(ab)^p = a^p b^p$, $a^{-p} = 1/a^p$ for positive a, b. (Section 1.8.)

- **Lax-Mizohata theorem**: a standard theorem of the theory of partial differential equations (P. D. Lax, 1957; S. Mizohata, 1961), quoted in this book without proof and therefore ASSUMED: if the initial-value problem of a linear system of first-order equations with smooth coefficients is well posed near a point, then at that point the frequencies of its principal part are real for every real momentum. (Section 8.18.)

- **Lead checks**: short independent Python programs of the Revision record (folder `Revision/lead_checks`), written from scratch by the coordinator of the Revision work (the "lead") without importing any other Revision code; each writes a report with a list of named checks and their verdicts. (Sections 3.7, 3.12 and 3.29.)

- **Lead report**: The lead report is the file `emt-divergence-and-spin-connection.json` in the folder `Revision/lead_checks/reports`, written by an independent program of the lead of the Revision work for the metric at $b_4 = 1$. (Section 6.1.)

- **Leaf**, **pruning** (skipping): a complete combination of k curvature entries in the computer's search; pruning means leaving out combinations known to give zero. (Sections 11.2 and 11.12.)

- **Least squares**: the straight line through a set of points that makes the sum of the squared vertical distances of the points from the line as small as possible. (Sections 2.6, 13.21, 17.2 and 17.20.)

- **Ledger**: the table of the statements, their labels and the reports that verify them. (Section 0.20.)

- **Left (right) derivative** $\partial_L F/\partial\theta_k$ ($\partial_R F/\partial\theta_k$): move θ_k to the far left (right) of every monomial that contains it, collecting a -1 for every generator it passes, and then delete it. (Section 7.16.)

- **Left derivative**: The left derivative $\partial_L F/\partial\theta_k$: in every monomial that contains θ_k, move θ_k to the far left, collecting a -1 for every generator it passes, and then delete it; monomials without θ_k give nothing. (Sections 7.11 and 18.20.)

- **Left multiplication**: For a fixed quaternion u the right multiplication R_u maps q to $q\,u$ and the left multiplication L_u maps q to $u\,q$. (Section 4.5.)

- **Legend**: `plot` draws a curve through the points; `lw` is the line width, `ls` the line style (two hyphens in quotes mean dashed, a colon `":"` dotted), and `label` the text for the legend (the box that names the curves), which `legend` draws in the upper right corner. (Section 19.16.)

- **Legendre transform**: the rule that turns a Lagrangian into the energy (Hamiltonian) density: for each field, its time derivative times its canonical momentum, summed over the fields, minus the Lagrangian. (Section 18.2.)

- **Leibniz formula**: the sum over all $n!$ permutations σ of $1,\dots,n$ (the Leibniz formula). (Section 1.20.)

- **Lemma**: a proved statement that is a step towards others (Notebook 20c). Compare Proposition and Schur's lemma. (Section 20.20.)

- **Length**: Length (norm) of a vector, $|v| = \sqrt{v_1^2 + \cdots + v_n^2}$; a unit vector has length 1. (Sections 1.36, 1.40, 1.42, 1.47, 2.18, 10.11, 11.2 and 11.8.)

- **Length factor**: the number $f_i = \sqrt{|g_{ii}|}$; a small step dx_i along x_i has the size $f_i\,|dx_i|$. (Sections 0.14, 0.16 and 1.5.)

- **Lepton number** L: $+1$ for the electron and its neutrino ν_e, -1 for the positron and the antineutrino $\bar{\nu}_e$, 0 for quarks and photons. It is not the Lagrangian $\mathcal{L}$. (Sections 21.2 and 21.35.)

- **Level**: an allowed energy ε of the one-particle equation. (Sections 14.3, 14.17, 14.24, 15.13, 16.18, 16.23, 19.2, 19.15, 19.20 and 22.2.)

- **Level curve**: A level curve (contour) is a curve on which f has one fixed value. (Sections 2.18 and 2.22.)

- **Level key** `n2:j:parity:rank`: the shell $n_2 = |\vec{n}|^2$ of the 3-space momentum, the block type $j = \pm 1$, the brane parity (even or odd), and the rank, the position of the level among the particle levels of its sector (0 = the lowest). (Section 16.11.)

- **Level ladder**: For each case a level ladder at the horizontal place $x_0 = 0, 3, 6$: every exact level is a short horizontal line (from $x_0 + \text{shift} - 0.4$ to $x_0 + \text{shift} + 0.4$), even parity at the left and odd parity 1.2 further right, and the numerical level is a marker in its middle. (Section 14.18.)

- **Levi-Civita symbol** $\varepsilon_{i_1 \ldots i_n}$: the sign of the list $(i_1, \ldots, i_n)$ if it is a re-ordering of all n labels, and 0 otherwise. (Sections 1.44, 1.47, 4.5 and 11.2.)

- **Levi-Civita tensors**: The author's notebook builds the generalized delta also in a second way, from two Levi-Civita tensors, in the cell In[32] (record `Revision/gkd_lovelock/results/notebook-input-cells.txt`). (Section 11.5.)

- **Lichnerowicz identity**: The Revision record proves the Lichnerowicz identity for the square of the Dirac operator in the author's metric, for every Ψ, solution or not (the formula is displayed in that section). (Section 7.23.)

- **Lie algebra so(4,4)**: the commutation rules $[S^{ab}, S^{cd}] = \eta^{bc} S^{ad} - \eta^{ac} S^{bd} - \eta^{bd} S^{ac} + \eta^{ad} S^{bc}$ of the 28 generators. (Sections 5.18 and 5.21.)

- **Light line**: One square panel. 200 rapidities from -1.4 to 1.4 trace the two unit hyperbolas: $(\cosh b, \sinh b)$, where $t^2 - y^2 = 1$ (blue), and $(\sinh b, \cosh b)$, where $y^2 - t^2 = 1$ (orange); the dotted diagonal is the light line $t = y$. (Section 6.22.)

- **Light-like** vector: $v \neq 0$ with $Q(v) = 0$ (notebook 01e). (Sections 1.13, 1.27, 1.32 and 1.40.)

- **Line**: Two wrong paths with the same end values are the line $q = t$ and the parabola $q = t^2$. (Section 7.2.)

- **Line continuation**: The second statement is broken over two lines by a backslash at the end of the first (a line continuation); it tests that $(\Gamma r)^T C (\Gamma r) - r^T C r$ is exactly zero, so Γ keeps S. (Section 21.15.)

- **Line element**: the formula for ds^2. (Sections 1.29, 1.32, 3.3 and 3.12.)

- **Linear**: The map $v \mapsto Av$ is linear: $A(v + w) = Av + Aw$ and $A(cv) = c\,Av$, because $\sum_j A_{ij}(v_j + w_j) = \sum_j A_{ij} v_j + \sum_j A_{ij} w_j$ and $\sum_j A_{ij}(c v_j) = c \sum_j A_{ij} v_j$ (multiply out inside the sum; a common factor comes out of a sum). (Sections 1.18 and 4.2.)

- **Linear combination**: $c_1 M_1 + c_2 M_2 + \ldots$ with numbers c_j. (Sections 4.2 and 4.15.)

- **Linear equation, system**: an equation of the form $c_1 y_1 + c_2 y_2 + \cdots + c_n y_n = 0$ for unknown numbers $y_1, \ldots, y_n$ with given coefficients c_j. (Section 5.16.)

- **Linear equations**: So $LX - XR = 0$ is a list of nm linear equations for the nm entries of X: in the equation (r, c) the unknown X_{kc} has the coefficient L_{rk} and the unknown X_{rk} the coefficient $-R_{kc}$. (Section 5.13.)

- **Linear expression**: A linear expression in some variables $p, q, \ldots$ is a sum of the variables, each multiplied by a fixed number (or, below, a fixed matrix), its coefficient: $\alpha p + \beta q$ is linear, while p^2, pq and $\sqrt{p^2 + q^2}$ are not. (Section 4.1.)

- **Linear history** $a_4 = AHx_4$ with a constant number A: then $a_4' = AH$ and $a_4'' = 0$, and the extra times shrink like e^{-AHx_4}: exponential deflation for $A > 0$. (Sections 3.8, 11.2 and 11.26.)

- **Linear member**: the history $a_4 = AHx_4 + a_0$ with a constant slope A. For $A > 0$ the extra times deflate as e^{-AHx_4}. (Sections 12.2, 12.19, 12.29, 17.2, 17.10, 20.10 and 20.15.)

- **Linear mixing**: Plain iteration: $x \leftarrow G(x)$; linear mixing: $x \leftarrow (1-\beta)\,x + \beta\,G(x)$ with the mixing parameter β. (Sections 13.21, 13.24, 15.5 and 15.8.)

- **Linearly dependent**: We use one fact of linear algebra without proof (ASSUMED): more than n vectors with n components are linearly dependent, that is, some combination $c_1u_1+c_2u_2+\cdots$ with numbers c_i not all 0 gives the zero vector. (Section 1.36.)

- **Linearly independent**: Multiplying from the left by γ_A^T and taking the trace gives $\sum_B c_B \,\mathrm{tr}(\gamma_A^T\gamma_B) = 16\,c_A = 0$, so every coefficient is 0: the 256 products are linearly independent (no combination of them with coefficients not all zero gives the zero matrix). (Sections 4.13, 4.15, 5.11 and 8.6.)

- **Linearly independent matrices**: no one of them is a combination of the others. Writing each 16×16 matrix as one row of 256 numbers, N matrices are independent when these N rows have rank N. (Section 5.16.)

- **List**: `here.parents` lists the parent folder, its parent, and so on up to the top of the disk; `[here, *here.parents]` is the list (an ordered collection, written in square brackets) that starts with `here` and continues with all of them. (Sections 0.13, 0.20, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15, 21.23 and 22.12.)

- **List comprehension**: The line `different = [...]` is a list comprehension: it collects every package `p` whose installed version differs from its pinned version `v` (`!=` means "is not equal to"). (Sections 0.13, 1.9, 3.13, 4.8, 4.12, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 19.16, 20.11 and 22.23.)

- **Literal sum**: Brute force, literal sum: a computation that tries every case without any shortcut. (Section 11.18.)

- **Local**: What this theory proves is: the charge of each field obeys an exact local conservation law (it follows from an exact U(1) symmetry), so no process creates or destroys charge at any point inside a universe off its edge (at every point with $0 < z < \pi/2$, where the law is proved): charge can only flow from one place to another; the total charge Q of a universe is constant only if no charge flows out through the edge $z = \pi/2$ of the hidden direction, and this no-flux condition is ASSUMED, not derived (whether a boundary condition at the edge supplies it is OPEN; Chapters 10, 18 and 21 show exact solutions through whose edge charge does flow, so that their total charge changes); the discrete symmetries of the theory are derived exactly (Chapter 21), among them its two charge conjugations, which are matrices, $\mathcal{C}_+ = C$ and $\mathcal{C}_- = \Gamma C$; and a T1 partner (mass $-m$, self-coupling $-\lambda$) carries the opposite charge, so a T1 pair $\{+m, -m\}$ has total charge zero as classical fields (for two universes that are quantised independently of each other there is no such cancellation; this is part of Q). (Sections 0.2, 14.26, 21.16 and 21.22.)

- **Local conservation law**: The Revision record proves a local conservation law: for every solution of the field equation. (Sections 0.2, 5.6 and 10.2.)

- **Local density approximation**: The local density approximation (LDA) treats each small volume as a piece of uniform gas with the local density: $E_{xc}^{LDA}[n] = \int e_{xc}(n(\mathbf{r}))\, d^3r, \qquad v_{xc}^{LDA}(\mathbf{r}) = \frac{de_{xc}}{dn}\Big|_{n(\mathbf{r})}$. (Section 13.15.)

- **Local frame rotation**: Such a point-dependent turning of the frame is a local frame rotation (a rotation, or a boost when it mixes a space-like with a time-like direction; Chapter 5). (Section 6.2.)

- **Local-frame model**: the equation $i\,du/dx_4 = h(x_4)u$ in which the frame momenta change with x_4 as the history prescribes, while the hidden position is held fixed (section 4 of Notebook 08b). (Sections 8.25, 8.28 and 10.42.)

- **Log-log plot**: A log-log plot marks both axes in powers of ten, so the errors of a method of order p lie on a straight line of slope p, and the order can be read off as a slope. (Sections 2.6 and 2.10.)

- **Log-sum-exp**: The solver solves $\ln(P + d_-) - \ln(H_l + d_+) = 0$ with $d_\pm = \max(\pm d, 0)$, adding each sum in the form $\ln \sum e^{t_i} = t_{max} + \ln \sum e^{t_i - t_{max}}$ (the log-sum-exp, which never overflows or underflows), by Newton's method in a bracket and then bisection down to neighbouring double-precision numbers (`Revision/kohn_sham/solver/src/mermin.rs`). (Section 15.26.)

- **Logarithm**: The logarithm $\log_{10} x$ is the power to which 10 must be raised to give x; for example $\log_{10} 1000 = 3$ and $\log_{10} 0.001 = -3$. (Sections 2.6 and 2.10.)

- **Logarithmic**: `ax.set_yscale("log")` makes the vertical axis logarithmic: equal distances on it mean equal factors (0.1, 1, 10, 100). (Sections 0.17, 4.12 and 19.21.)

- **Logarithmic axes**: The right panel draws the size of the errors (`np.abs`) against the width on logarithmic axes (`loglog`), one marker shape per centre, and a grey dashed reference line proportional to w^2, which on these axes is a straight line of slope 2. (Sections 7.9 and 16.12.)

- **Logarithmic axis**: an axis on which equal steps mean equal *factors* (1, 10, 100, ...) instead of equal differences; on it e^t and e^{-t} are straight lines. (Sections 0.16, 1.5, 1.8, 9.17 and 11.9.)

- **Logarithmic derivative** $\partial \ln s/\partial x = (\partial s/\partial x)/s$: the rate of growth of s per unit of x, as a fraction of s itself. (Sections 2.19 and 2.22.)

- **Logarithmic vertical axis**: `semilogy` draws with a logarithmic vertical axis: equal distances mean equal factors (each tick is ten times the one below). (Section 14.11.)

- **Long division**: the school method that produces the decimal digits of p/q one after the other from the remainders. (Section 1.8.)

- **Long step**: A long step is one documented as longer than 5 minutes, or one that needs such a step (`ks-rust-determinism`, 30 s, needs the refined run). (Section 23.2.)

- **Lovelock equations**: In eight dimensions the field equations of gravity that the author's theory uses are the Lovelock equations (Chapters 11 and 12). (Section 1.42.)

- **Lovelock scalar**: $L_{(1)}$ is the first Lovelock scalar: the Revision record builds the field equations from three curvature scalars $L_{(1)}, L_{(2)}, L_{(3)}$, made from products of one, two or three Riemann tensors with the generalized Kronecker delta of Chapter 1, and the first of them is

exactly twice the Ricci scalar, $L_{(1)} = 2R$ (Chapter 11 treats all three). (Sections 3.30, 11.2, 11.11, 11.18 and 11.26.)

- **Lovelock scalars and tensors**: curvature quantities built from products of one, two or three Riemann tensors with the generalized Kronecker delta; the first Lovelock scalar is $L_{(1)} = 2R$ and the first Lovelock tensor is $P_{(1)} = -4G$. A later chapter treats them in full. (Section 3.29.)
- **Lovelock tensors**: combinations of the curvature of spacetime that enter the field equations of gravity in more than four dimensions; the Revision record computes three of them for the author's metric, with GKD as the weight of every term. (Sections 1.43, 1.47, 3.26, 11.2, 11.11, 11.18, 11.26, 12.2, 12.7, 12.15, 17.2 and 20.10; the opening of Chapter 11.)
- **Lower**: A pair of index lists is a lower list $(l_1, \ldots, l_p)$ and an upper list $(u_1, \ldots, u_p)$ of the same length. (Sections 11.2 and 11.8.)
- **Lower incomplete gamma function**: In the second substitute $u = Ky$ ($dy = du/K$, limits 0 and K): $\int_0^1 y^{K-1} e^{-Ky} dy = \int_0^K (u/K)^{K-1} e^{-u} du/K = K^{-K}\gamma(K, K)$, where $\gamma(s, x) = \int_0^x u^{s-1} e^{-u} du$ is the lower incomplete gamma function. (Section 21.6.)
- **Lower index**: A second list of eight numbers that belongs to the same vector, $v_0, \ldots, v_7$, carries a lower index; Section 1.27 says how one list is made from the other. (Section 1.26.)
- **Lowering**: So lowering an index keeps the four space-like components and changes the sign of the four time-like ones: for $v^a = (1, 2, \ldots, 8)$ (that is $v^0 = 1, v^1 = 2, \ldots, v^7 = 8$), $v_a = (1, 2, 3, 4, -5, -6, -7, -8)$. (Sections 1.27 and 6.27.)
- **Lowering and raising**: $v_a = \eta_{ab} v^b$ and $v^a = \eta^{ab} v_b$. (Section 1.32.)
- **Lowest terms**: The spelling is in lowest terms when p and q have no common factor larger than 1. (Sections 1.2 and 1.8.)
- **LUMO**: HOMO, LUMO: the highest occupied and lowest unoccupied level; the KS gap is their difference. (Sections 13.35, 15.13, 16.11, 19.20 and 19.21.)

M

- **Machine epsilon** ϵ: the gap between 1 and the next larger floating-point number, $\epsilon = 2^{-52} \approx 2.22 \times 10^{-16}$. (Sections 0.22, 0.24, 1.4, 1.8, 2.6, 2.10, 16.18, 16.20 and 16.23.)
- **Majorana**: A field equal to its own conjugate obeys $\Psi = M\Psi^*$; such a requirement is called a Majorana or reality condition, after the physicist who first used one. (Section 5.29.)
- **Majorana (reality) condition**: the requirement $\Psi^c = \Psi$. (Sections 5.32, 7.29, 7.32, 21.2 and 21.14.)
- **Majorana-type**: built with the transpose Θ^T instead of the Dirac adjoint (a Lagrangian of this kind is called Majorana-type, after the physicist Ettore Majorana, who studied real spinor fields). (Section 7.28.)
- **Majorana-type Lagrangian** L_g: the Lagrangian of the author's notebook, built with the transpose Θ^T of a real column instead of the Dirac adjoint $\bar{\Psi} = \Psi^\dagger C$. (Section 7.32.)

- **Majorana-type mass terms**: The simplest candidates are Majorana-type mass terms $\Psi^T M \Psi$ with a constant 16×16 matrix M, built from Ψ twice and without $\Psi^\dagger$. (Section 21.32.)

- **Majorana-type term**: a term $\Psi^T M \Psi$ built from Ψ twice, without $\Psi^\dagger$ (Section 21.32). (Sections 21.2 and 21.35.)

- **Manifest**: a file that lists other files with their fingerprints. (Sections 0.22, 0.24 and 15.13.)

- **Mantissa**: The caption quotes the count for $n = 16$ as computed, not typed: `f"{x:.1e}"` writes 16! as the text 2.1e+13, and `.split("e")` cuts it at the letter e into the mantissa 2.1 and the exponent +13 (both are still texts). (Section 1.25.)

- **Map**: a rule that turns a field Ψ into a new field Ψ'. All maps of Chapter 18 are of the form $\Psi'(x) = M\Psi(x)$ with a constant 16×16 matrix M, possibly combined with a change of the coordinate x_0. (Sections 4.2 and 18.2.)

- **Map between solution sets**: a rule that takes EVERY solution of one theory to a solution of another theory (here: multiply the field by a fixed matrix, possibly together with a change of the coordinate x_0). (Section 20.2.)

- **Map between two sets of solutions**: This is a map between two sets of solutions. (Section 21.1.)

- **Maps between sets of solutions**: All of these are exact statements about maps between sets of solutions of the field equations. (Section 5.39.)

- **Maps between solution sets**: They are exact maps between solution sets: if a configuration of the field is a solution of the equations, its image under the map is a solution of the equations with the mapped parameters (for T1, of the equations with $(-m, -\lambda)$ in place of (m, λ)). (Sections 0.2 and 20.1.)

- **Margins**: The eight measured largest differences are below their tolerances by factors (margins) between 4.7 and 7692 (COMPUTED: `Revision/kohn_sham/reports/ks-rust-determinism.json`; the measured values are quoted by `Revision/kohn_sham/reports/ks-crosscheck.json`, key `rust_matrix_wide_uncertainties`). (Sections 0.22, 0.25, 16.12 and 19.17.)

- **Markdown cell**: A markdown cell holds text; a code cell holds Python code. (Section 0.7.)

- **Mask**: The same selection by hand, to draw it: `d1` and `d2` are the two differences of every level (numpy subtracts arrays entry by entry), `keep` is a mask, an array of truth values that is true where both differences exceed the floor (`&` combines two masks with "and"), and `energies` are the Richardson values of the kept levels (`array[keep]` keeps the entries where the mask is true). (Section 16.12.)

- **Mass** m: a number in the equations of a field; for a field that describes particles it is their mass (Chapter 7). (Sections 0.2, 2.27, 12.29, 20.15 and 21.14.)

- **Mass gap**: the energies $-m < \varepsilon < m$, where a free particle of mass m has no level. (Sections 19.2 and 19.15.)

- **Mass reversed**: solves the field equation of the same form, either with the same V (same mass) or with $-V$ (mass reversed). (Sections 5.28, 5.32 and 21.8.)
- **Mass shell** (or *dispersion relation*): the relation between E, the k_a and the mass m that a plane wave must obey. (Sections 4.3, 4.9 and 4.11.)
- **Mass term**: The mass term $-mS = -m\bar{\Psi}\Psi$ is linear in the mass m (a real number) and quadratic in the field: every term contains one component of $\Psi^\dagger$ and one of Ψ. (Sections 7.19 and 18.4.)
- **matplotlib**: sympy: the Python package for exact algebra with symbols; numpy: arrays of numbers; matplotlib: plots. (Section 3.12.)
- **The matrices C, Γ and B**: three fixed 16×16 matrices built from gamma matrices: C is the product of the four space-like ones, Γ (the chirality) the product of all eight, and B the product of C and the gamma matrix of the time x_4, times $-i$; B enters the quantum theory (Chapter 5). (Sections 0.2, 0.14, 1.18, 1.24, 2.19, 4.2, 4.7, 4.19, 5.9, 5.28, 7.16, 10.9, 16.18, 18.8, 19.15, 21.8 and 21.14; the opening of Chapter 21.)
- **Matrix element** $\langle n|X|m\rangle$: for orbitals $\chi_n = (a_n, ib_n)$ the integral $\int \chi_n^\dagger X \chi_m \, dy$; for $X = \sigma_3$ it is $\int (a_n a_m - b_n b_m)\, dy$. (Sections 15.24 and 22.11.)
- **Matrix exponential**: for a square matrix M the matrix $e^{Mx_4} = 1 + Mx_4 + \frac{1}{2}M^2x_4^2 + \ldots$; the column $e^{Mx_4}\chi$ solves $\frac{d}{dx_4}\Phi = M\Phi$ with $\Phi(0) = \chi$. When $M^2 = k^2$ times the unit matrix, the series sums to $\cosh(kx_4) + \frac{\sinh(kx_4)}{k}M$. (Sections 8.18, 9.18 and 9.24.)
- **Matrix identity**: *Line 5 (the matrix identity).* So $\partial_\mu(\cos z\, J^\mu) = -i\cos z\,(\Psi^\dagger C E - E^\dagger C \Psi)$ holds for every field as soon as the first part of line 1 equals line 4, that is (dividing by $-i$) as soon as the 16×16 matrix identity. (Section 21.17.)
- **Matrix product**: $(AB)_{ik} = \sum_j A_{ij}B_{jk}$: entry (i,k) of AB is row i of A times column k of B, entry by entry, added up. (Sections 1.24 and 5.9.)
- **Matrix, entry, row, column**: a matrix is a rectangle of numbers; the number in row i and column j is the entry M_{ij}. (Section 11.8.)
- **Matter**: Matter is what stars, planets and we are made of: protons, neutrons and electrons. (Section 20.23.)
- **The matter-antimatter problem**: The matter-antimatter problem is the question why the universe contains baryons but essentially no antibaryons. (Section 21.3.)
- **Max|T|**: the largest absolute value of ρ, p_3, p_t and p_0 of a state over the whole patch. We divide by it to compare states of very different size. (Sections 17.2 and 17.10.)
- **Mean field**: each electron feels the other one only through its average density; here an electron on a site feels U times the density of the other label on that site, i.e. $Un_L/2$ on L and $Un_R/2$ on R. (Sections 13.24 and 19.2.)
- **Mean value theorem**: (by the mean value theorem of calculus, $g(\phi) - g(\omega) = (\phi - \omega)\, g'(\xi)$ for some ξ between ω and ϕ; applied to $g = \tan$, whose derivative is $1 + \tan^2$, it gives $\tan\phi - \tan\omega = (\phi - \omega)(1 + \tan^2\xi)$, and $1 + \tan^2\xi = 1 + \ldots$ for small angles). (Section 2.24.)

- **Mean-field potential** $w(x)$: here the Hartree plus exchange potential, so that $v_s = v + w$. (Sections 13.32 and 13.35.)

- **Median**: The median is the middle value of the sorted list: `len(ratios) // 2` is the position of the middle (`//` divides and drops the remainder; with 39 ratios it is the 20th value, position 19 counting from 0). (Sections 14.18, 15.9 and 16.12.)

- **Mermin condition**: $N(\mu) = \sum_i g_i\, f(x_i) = N$, the equation that fixes μ. (Sections 16.20 and 16.23.)

- **Meson**: Baryon: a particle made of three quarks, such as the proton $p = uud$ and the neutron $n = udd$ ($B = 1$); a meson (pion π) is a quark and an antiquark ($B = 0$). (Section 21.35.)

- **Metric**: the rule that turns a small step in the coordinates into its size. For the author's metric the rule is $ds^2 = g_{00}dx_0^2 + g_{11}dx_1^2 + \cdots + g_{77}dx_7^2$, where dx_i is the small step along x_i and g_{ii} is the i-th entry of the metric. (Sections 0.1, 0.14, 0.16, 1.1, 1.24, 1.32, 3.3, 3.12, 3.21, 3.29, 6.13, 7.26, 8.14, 11.2, 11.18, 12.2, 12.15, 14.10, 17.2 and 18.19.)

- **Metric functions**: where $z = 6Hx_0$, $H > 0$ is a constant of the author, and a_4 and b_4 are functions of the time x_4 alone, the metric functions (their field equations are the subject of Chapter 12); b_4 is positive. (Sections 0.14, 3.1 and 8.2.)

- **Metric product**: The metric product is $\eta(u, v) = \sum_a \eta_{aa} u_a v_a$; u is a unit vector when $\eta(u, u) = +1$ or -1. (Sections 5.12 and 5.21.)

- **Metric, diagonal entries** g_{aa}: the eight numbers (functions of x_4 and x_0) that turn coordinate distances into true distances in the author's 8-dimensional world; positive for the space directions x_0, x_1, x_2, x_3 and negative for the time directions x_4, x_5, x_6, x_7. (Section 2.22.)

- **Milne wedge**: the part $t > |y|$ of the flat plane with one time t and one space direction y, $ds^2 = -dt^2 + dy^2$, described by $\tau = \sqrt{t^2 - y^2}$ and θ with $t = \tau \cosh\theta$, $y = \tau \sinh\theta$; its metric is $ds^2 = -d\tau^2 + \tau^2 d\theta^2$. (Sections 6.23 and 6.27.)

- **Minimum**: For $T < \pi/\omega$ the bracket is positive, so $S_2 > 0$ and the action of $q + \epsilon\xi_1$ is $S_0 + S_2\epsilon^2$ with a positive S_2: the true path is a minimum along this shape. (Section 7.2.)

- **Minor**: Minor $M^{(rc)}$: the matrix left when row r and column c of M are removed. (Sections 11.2 and 11.18.)

- **Mirror patch**: The map $z \to \pi - z$ takes it to the mirror patch $\pi/2 < z < \pi$. (Sections 18.2, 18.19, 18.26, 20.2 and 20.15.)

- **Mismatch**: 3. see where the "shot" lands at the right end: the mismatch $F(\lambda) = u(1; \lambda)$. (Section 2.12.)

- **Mixed**: $T^{\mu}{}_{\nu}$ has one of each and is called mixed. (Sections 2.18, 12.2 and 17.2.)

- **Mixed components**: Its components $\omega_{\mu}{}^{a}{}_{b}$, with the first frame index up, are called the mixed components. (Sections 6.4, 6.27 and 12.2.)

- **Mixed partial derivative**: $\partial^2 f/\partial x \partial y$, first with respect to y, then x (or the other way round). Schwarz's theorem: for a function with continuous second derivatives the order does not matter. (Section 2.22.)

- **Mixed-product rule**: The mixed-product rule $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$ holds. (Section 4.17.)

- **Mixing**: feeding only part of the change back (linear mixing) or a combination of several earlier passes (Anderson mixing) to make the loop converge. (Sections 13.35 and 15.8.)

- **Mode Hamiltonian** h: the 16×16 matrix with $E\,u = h\,u$ for the field $u\,e^{i(k\,x1 - E\,x4)}$ of one momentum; its eigenvalues are the energies of the fields of that momentum (section 8 of Notebook 05e derives it). (Sections 5.34, 5.37, 10.3, 10.25 and 10.42.)

- **Mode matrix** h_k: the 16×16 complex matrix with $Eu = h_k u$. (Sections 8.17, 8.22 and 8.28.)

- **Mode operator** h: the field equation written as $i\,\partial_4 \Psi = h\Psi$; for fields that depend only on x_4 and x_0 (and $U = 0$), $h = -im\gamma^{(4)} + i\gamma^{(4)}\gamma^{(0)}(\tan z\,\partial_0 + 3H)$. (Sections 8.32 and 8.35.)

- **Mode, occupation, pattern**: a fermion *mode* is a place that holds zero or one particle. With 16 modes a basis state is a *pattern* of 16 occupations (0 or 1); the computer stores it as a whole number n whose binary digit p is the occupation of mode p ($p = 0, \ldots, 15$). There are $2^{16} = 65536$ patterns. (Section 5.37.)

- **Module**: `import` loads a module (a part of Python or of a package) so that the code can use it; the text after `#` on each line is a comment. (Sections 0.13, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15 and 22.12.)

- **Modulus** $|z| = \sqrt{a^2 + b^2}$: the length of the arrow. (Sections 1.10 and 1.16.)

- **Moment equation**: A moment equation is a field equation multiplied by e^{6Hy} and integrated over the patch. (Section 17.20.)

- **Moment** of a field equation: the equation multiplied by the volume weight e^{6Hy} and integrated over the patch; it involves the source only through its weighted means $\bar{\rho}$, $\bar{p}_3$, $\bar{p}_t$, $\bar{p}_0$. (Sections 17.2, 17.17 and 17.20.)

- **Momenta**: The metric does not depend on the six transverse coordinates, so by Section 3.16 the six momenta $p_k = g_{kk}u^k$ (no sum) do not change during free fall: $p_i = h_i^2\,u^i, \qquad p_j = -h_j^2\,u^j \qquad$ (both constant). (Sections 3.31, 4.9 and 10.3.)

- **Momentum** $p_a = g_{aa}u^a$ (no sum) of a direction x_a. **Conserved**: the same number at every point of the path. (Sections 3.16, 3.34, 5.34, 5.37, 8.17, 10.9, 20.17, 20.20 and 20.22.)

- **Momentum weight**: The factor e^{-Hy-a_4} is exactly the momentum weight $\kappa(y, x_4) = e^{-Hy-a_4(x_4)}$ (the Revision record's name for it; this $\kappa(y, x_4)$, always written in Chapter 3 with its two arguments, is a function and has nothing to do with the constant κ of Einstein's equations in Section 3.26) that the Revision Kohn-Sham record uses to turn a conserved 3-space momentum into the momentum seen at height y and time x_4 (`Revision/kohn_sham/ks-theory.json`, key `geometry.kappa`; PROVED here; Notebook 03d, In [5]). (Sections 3.31, 14.2 and 19.3.)

- **Monomial**: a product of different generators written in increasing order, such as $\theta_0\theta_2\theta_5$; the empty product is the number 1. (Sections 7.10, 7.16, 7.26, 11.18, 12.16, 18.19 and 18.20.)

- **Monotone (increasing)**: a function that grows whenever its argument grows. (Section 2.27.)

- **mpmath**: a Python package that computes with as many decimal digits as requested (here 50). (Sections 1.8 and 1.16.)
- **Multiple of the identity**: A number c times a matrix multiplies every entry by c; cI_n is called a multiple of the identity. (Section 4.2.)
- **Multiplicity**: The fundamental theorem of algebra (ASSUMED) says that a polynomial of degree n has exactly n roots $\lambda_1, \ldots, \lambda_n$ when complex numbers are allowed and a repeated root is counted as often as it appears, its multiplicity. (Sections 1.34, 1.40, 4.9, 4.11, 4.19 and 5.3.)

N

- **Natural logarithm**: The exponential function e^t is defined for every real t, is always positive and grows; its inverse is the natural logarithm $\ln x$, defined for $x > 0$: $\ln(e^t) = t$ and $e^{\ln x} = x$. (Sections 1.5 and 1.8.)
- **nbkit**: The book's tool nbkit (the file `Revision/textbook/tools/nbkit.py`) turns a builder into a notebook in two steps. (Section 0.9.)
- **nbkit check**: re-executing a notebook into a scratch folder and comparing the result byte for byte with the stored notebook, its files and its provenance file. (Sections 23.2 and 23.11.)
- **Negative branch**: Label n: the integer that numbers the levels of a sector: $n = 0$ is the band level (bound to the brane, occupied in the Fermi shell), $n = 1, 2, ...$ the bulk levels above it (empty), $n = -1, -2, ...$ the levels of negative energy, the negative branch (in the record's convention the normal-ordered sea, not populated). (Sections 22.2 and 22.11.)
- **Negative control**: a deliberately changed case in which the effect must NOT occur (or must occur differently); it shows that a check can fail and that the result depends on the feature that was changed. (Sections 0.22, 1.50, 6.1, 6.13, 7.28, 7.32, 9.26, 9.29, 18.2, 18.13, 18.19, 19.2, 19.20, 20.11, 21.15, 21.17 and 22.3.)
- **Neutral subspace**: a subspace on which $u^\dagger B v = 0$ for all u, v in it. (Section 10.9.)
- **Newton's method**: replace a function by its tangent line at the current guess and take the zero of that line as the next guess. (Sections 2.27, 15.8, 16.20 and 16.23.)
- **Nilpotent**: For a light-like $v \neq 0$, $\gamma(v)$ is not the zero matrix, because $\mathrm{tr}(\gamma(v)\gamma^b) = v_a \,\mathrm{tr}(\gamma^a\gamma^b) = 16\, v_a \eta^{ab} = 16\, v^b$ (the third contraction of Section 1.28; raising the index) is not 0 for a component $v^b \neq 0$; but $\gamma(v)^2 = 0$: a matrix with a power equal to 0 is called nilpotent, and all its eigenvalues are 0 ($\lambda^2 v = \gamma(v)^2 v = 0$ gives $\lambda = 0$); its characteristic polynomial is λ^{16}. (Sections 1.37 and 8.10.)
- **Nilpotent matrix**: a matrix $N \neq 0$ with $N^k = 0$ for some power k; all its eigenvalues are 0. (Section 1.40.)
- **No-boundary proposal**: a proposed boundary condition (J. B. Hartle and S. W. Hawking) that picks one solution of the Wheeler-DeWitt equation. It is named in Chapter 22, not used, and is not formulated for this 8-dimensional theory. (Sections 22.2 and 22.4.)

- **No-flux condition**: the condition that no charge flows through the edge $z = \pi/2$ of the hidden direction (the brane). The local conservation law is proved at every point with $0 < z < \pi/2$; the no-flux condition is ASSUMED, not derived, and only under it is the total charge of a universe constant. Exact solutions exist through whose edge charge does flow, so that their total charge changes. (Sections 0.2, 5.6, 5.39, 18.21, 20.22, 21.1 and 21.17.)
- **No-flux condition at the brane**: the condition $\sqrt{|g|}J^{x_0} = 0$ at $z = \pi/2$ (for the homogeneous solutions, $q_0 = 0$): no charge flows out through the brane. Only under it is the total charge of a universe constant. It is ASSUMED wherever it is used, not derived; it holds for some exact solutions and fails for others (Notebook 18c), and whether the junction condition at the brane implies it is OPEN. (Section 18.28.)
- **Node**: a point where the eigenfunction changes sign. (Sections 2.16, 2.17, 2.24, 16.14 and 16.18.)
- **Noether identity**: On every solution of the field equation the tensor is conserved: this is a theorem, proved in the record from the invariance of the action under a change of coordinates (the Noether identity; PROVED, see the table below). (Section 9.9.)
- **Noether's construction**: This is Noether's construction. (Section 21.16.)
- **Noether's theorem**: a continuous symmetry of the Lagrangian gives a current whose divergence vanishes on every solution (Section 21.16). (Sections 21.2 and 21.22.)
- **Non-characteristic**: So the time derivative of every component is fixed by the field and its derivatives along the other seven directions at the same time x_4: the slices $x_4 =$ const are non-characteristic (PROVED; Notebook 07b, In [19]; `wolfram-field-theory-b4.json`, checks `evolution_form_G` and `evolution_form_C`; the part $\frac{3}{4}(b_4'/b_4)\gamma^{(x_4)}$ of M gives $-\frac{3}{4}(b_4'/b_4)\Psi$ on the right, formula record key `evolution_form`). (Sections 7.22 and 8.16.)
- **Non-interacting v-representability**: The Kohn-Sham scheme ASSUMES that the density of interest is the ground-state density of some non-interacting system in some potential v_s (non-interacting v-representability). (Section 13.10.)
- **Non-trivial**: The author asked that the Lagrangians of both fields be non-trivial: their Euler-Lagrange equations must always contain nonzero contributions from gravity through the canonical spin connection, unless space-time is flat 4+4 space ([1] for dirac16complex, [2] for dirac16complex00), and must be self-consistent. (Section 7.23.)
- **Nondegenerate**: Such a form is called nondegenerate on the eigenspace: its Gram matrix has no zero eigenvalue. (Section 10.5.)
- **Nonlinear**: They look like one-particle equations, but $\hat{F}$ contains the unknown orbitals: they are nonlinear and are solved by iteration, starting from a guess and repeating until nothing changes any more (self-consistency, Section 13.21). (Section 13.6.)
- **Norm** of a field in the hidden direction: $\int \cos z \, \Psi^\dagger \Psi \, dx_0$ ($\cos z = \sqrt{|g|}$ is the volume factor). It is *finite* when the integral does not diverge at the ends. (Sections 8.16 and 8.35.)
- **Normal ordering**: writing every product with the annihilators to the right, by definition dropping the constant that the reordering produces (here $-8E$ for the energy). The normal-ordered value of an operator X in a state is its value minus the vacuum value. (Sections 5.34, 5.37, 7.29, 10.20, 10.25 and 14.30.)

- **Normal random numbers**: random numbers whose histogram is the bell curve $e^{-x^2/2}/\sqrt{2\pi}$; numpy's `normal` draws them. (Sections 1.25, 1.27 and 1.32.)
- **Normal-ordered**: After canonical quantisation (Chapter 10; the field becomes an operator, a rule that acts on the quantum states) the record defines the tensor as an operator too: every bilinear is replaced by the normal-ordered product of the quantised field (the product rearranged so that the operators that remove a particle stand to the right of those that create one, which gives the empty state the energy zero; Chapter 10 defines it exactly), $\hat{T}^{\nu}{}_{\mu} = :T^{\nu}{}_{\mu}[\hat{\Psi}, \hat{\bar{\Psi}}]:$, normal ordered with respect to the waves of the good sector (the waves that do not depend on the extra times x_5, x_6, x_7), and the numbers are expectation values (the average value that measurements of the operator give in a given quantum state) (record `Revision/docs/DIRAC16COMPLEX_FIELD_THEORY.md`, section 12). (Section 9.13.)
- **Normal-ordered energy**: the energy of a state minus the energy of the vacuum (the filled sea), as in the third notebook of Chapter 10. (Section 10.48.)
- **Normal-ordered square**: Its normal-ordered square puts all creation operators to the left: $:S^2: = \sum V_{pq} V_{rs}\, a_p^\dagger a_r^\dagger a_s a_q$. (Section 13.4.)
- **Normalisation**: multiplying an orbital by a number so that $\int_{-L}^{0} (a^2 + b^2)\, dy = 1$. (Section 15.8.)
- **Normalised**: The function u is the wave function: $u(x)^2$ is the probability density of finding the particle at x, so u is normalised, $\int u^2\, dx = 1$. (Sections 2.13, 10.11, 10.14, 13.2 and 22.12.)
- **Normalised Lovelock tensor**: Normalised Lovelock tensor $E_{(k)} = -P_{(k)}/2^{k+1}$; $E_{(1)}$ is the Einstein tensor. (Sections 11.2 and 11.26.)
- **NOT COMPUTED**: a label used next to the five status labels in Chapters 20 and 21: no computation of the quantity exists in the Revision record or in this book (for example the rate of a process that violates C or CP). (Sections 20.23, 21.1, 21.2, 21.4, 21.31, 21.36 and 21.38.)
- **Nucleus**: A photon of energy E_γ that hits a nucleus (the heavy centre of an atom) of mass M at rest can make an electron-positron pair, the nucleus remaining in the final state, only if $E_\gamma \geq 2m(1 + m/M)$. (Sections 20.17 and 20.20.)
- **Null**: A vector $v \neq 0$ is space-like if $Q(v) > 0$, time-like if $Q(v) < 0$ and light-like (or null) if $Q(v) = 0$. (Sections 1.27, 1.32, 3.4 and 5.12.)
- **Null combination** $\rho + p_0$: the energy density plus the pressure along x_0. (Sections 20.2, 20.10 and 20.15.)
- **Null energy condition** (NEC) along x_0: $\rho + p_0 \geq 0$. Ordinary matter satisfies it; a source with $w < -1$ and $\rho > 0$ (called *phantom*) violates it. (Sections 3.26, 12.2, 12.15, 12.19 and 22.2.)
- **Null lines**: The right picture draws the two hyperbola branches and the two dotted null lines $v_4 = \pm v_1$, on which $v_1^2 - v_4^2 = 0$. (Section 5.22.)
- **Null space**: The columns v with $Av = -i\omega v$ are the solutions of $(A + i\omega)v = 0$, the null space of $A + i\omega$. (Sections 9.30 and 20.16.)

- **Null vector**: a direction of zero length, such as the frame direction $x_4 + x_0$ (time-like plus space-like). (Sections 12.2 and 12.11.)
- **Number density**: (Check gas_densities.) The three densities that matter are the number density (the U(1) charge density, whose integral counts the quanta), the scalar density and a third density Q (the formula is displayed in that section). (Section 19.4.)
- **Number operator**: The number operator $\hat{n}_p = a_p^\dagger a_p$ gives n_p, and $\hat{N} = \sum_p \hat{n}_p$ counts the particles of a state. (Sections 10.11 and 13.4.)
- **The numerical parameters** (parameters.json, numerics): $G = 900$ steps of the Runge-Kutta method on $[-L, 0]$, a root tolerance of $10^{-13}\,m$ for each level, a self-consistency tolerance of $10^{-11}\,m$ with at most 400 iterations, Anderson mixing with depth 6 and $\beta = 0.4$, the step $\delta = 0.002$ in a_4 for derivatives along the history, and levels closer than $10^{-9}\,m$ count as equal (one degenerate group). (Section 15.2.)
- **Numerics**: The cross-check is a test of the numerics: of the discretisation, the rounding and the programming of the Rust solver, against a second program that differs in all three. (Section 16.25.)
- **numpy**: sympy: the Python package for exact algebra with symbols; numpy: arrays of numbers; matplotlib: plots. (Sections 3.12, 10.9 and 20.11.)

O

- **O(4,4)**: the real 8×8 matrices Λ with $\Lambda^T \eta \Lambda = \eta$ (they keep the metric product); SO(4,4): those with determinant $+1$. (Sections 5.12 and 5.26.)
- **O(4,4), SO(4,4)**: the real 8×8 matrices Λ with $\Lambda^T \eta \Lambda = \eta$ (they keep the metric product); SO(4,4): those with determinant $+1$. (Section 5.21.)
- **Observer**: Observer: someone who, like us, measures only the three space directions x_1, x_2, x_3 and the time x_4. (Section 22.22.)
- **Occupation**: Level ε: an allowed energy of the one-particle equation; occupation f: how many particles sit in an orbital of that level (0 to 1); degeneracy g: how many orbitals share the level. (Sections 10.18, 19.2 and 19.20.)
- **Occupation numbers**: A determinant built from some of them is fixed, up to its sign, by the occupation numbers $n_p \in \{0, 1\}$, $n_p = 1$ when ϕ_p is used. (Section 13.4.)
- **Occupation states**: The finite model of limit 2 gives exact values on the occupation states of its one mode set at rest (momentum $k = 0$). (Section 9.13.)
- **Occupation-number state** $|n_0 n_1 n_2 n_3\rangle$: the determinant that uses the orbitals p with $n_p = 1$; there are $2^4 = 16$ of them for four orbitals. (Section 13.13.)
- **Occupied**: A level is occupied if particles sit in it ($f = 1$) and empty if not ($f = 0$). (Section 15.13.)
- **Odd**: The doubled problem is therefore mirror-symmetric only if the mass is odd across the brane. (Sections 1.19, 2.13, 2.16, 4.13, 4.15, 5.5, 5.12, 5.16, 7.11, 7.16, 11.2, 11.8, 14.12, 16.18 and 19.11.)

- **Off shell**: A configuration that satisfies the field equation is said to be on shell; one that need not satisfy it is off shell. (Sections 9.2, 9.16, 18.2, 20.2 and 20.10.)

- **Off-diagonal**: Besides these 27 nonzero diagonal components there are 12 nonzero off-diagonal ones. (Section 12.5.)

- **On shell**: on solutions of the field equation. (Sections 7.26, 9.2, 9.16, 18.2, 20.2 and 20.10.)

- **One momentum**: A field of one momentum k along $x1$ is a field of the form $\Psi = \psi\, e^{ik\,x1}$: a column ψ (which may still depend on the time) times the number $e^{ik\,x1} = \cos(k\,x1) + i\sin(k\,x1)$, which goes once around the circle of complex numbers of size 1 each time $x1$ grows by $2\pi/k$. (Section 5.34.)

- **One-body density matrix**: Both formulas are summarised by the one-body density matrix $\rho = \sum_{a\in O} |\phi_a\rangle\langle\phi_a|$, whose matrix elements are $\rho_{qp} = \langle a_p^\dagger a_q\rangle$: $\langle a_p^\dagger a_q\rangle = \rho_{qp}$, $\qquad \langle a_p^\dagger a_q^\dagger a_s a_r\rangle = \rho_{rp}\,\rho_{sq} - \rho_{sp}\,\rho_{rq}$. (Section 13.4.)

- **One-body matrix**: The 16×16 matrix ρ is the one-body matrix of the state. (Section 14.26.)

- **One-particle Hamiltonian** $h_m(k)$: the 16×16 matrix that gives the time evolution $i\,\partial_4 u = h_m(k)u$ of a plane wave $u\,e^{ik\cdot x}$ in flat 4+4 space; its eigenvalues are the frequencies $\pm w$. (Sections 21.26 and 21.29.)

- **One-particle matrix**: The frequencies w are the eigenvalues of the one-particle matrix $h_m(k)$. (Section 18.22.)

- **One-sided fourth-order difference**: At the last slice there are no neighbours on the right, so the derivative uses the one-sided fourth-order difference $f'(x) \approx [3f(x-4h) - 16f(x-3h) + 36f(x-2h) - 48f(x-h) + 25f(x)]/(12h)$, the same stencil as the record; `w_mix[-5]` is the fifth entry from the end. (Section 22.23.)

- **One-step value**: The one-step value $r(h)$ has no h^2 error left; its error starts with h^4. (Section 16.5.)

- **Onset**: So along the history (where X grows with the time) E^2 crosses zero exactly once and stays negative afterwards; the moment of the crossing is the onset. (Sections 8.24 and 8.28.)

- **Onset time** x_4^*: the time at which w^2 changes from positive (the wave oscillates) to negative (the wave grows). (Sections 10.39 and 10.42.)

- **OPEN**: a status label: a question that neither the Revision record nor this book answers; it is not settled. Example: whether the big bang creates universes in pairs is, as a question of this book, OPEN: it is not proved. (Sections 0.3, 0.20, 3.1, 3.31, 8.1, 12.31, 15.31 and 22.27.)

- **Open problem**: a question that neither the Revision record nor this book answers, put precisely enough that one can recognise an answer. (Section 22.2.)

- **Open shell**: a degenerate group of levels only partly filled. (Sections 15.4 and 15.24.)

- **Operator**: a rule that turns a state into another state, here a big matrix that acts on columns of numbers. Operators are multiplied by doing one after the other; the order matters. (Sections 5.34, 5.37, 9.13, 10.11, 10.18, 13.2, 21.25 and 21.29.)

- **Orbifold**: The Revision theory ASSUMES a Z2 mirror (also called an orbifold): the patch $y \le 0$ is glued at $y = 0$ to a mirror copy of itself, with the warp $e^{-H|y|}$ on both sides, and a field on the doubled interval obeys $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$. (Sections 14.12 and 22.2.)

- **Orbital**: a single-particle state of the Kohn-Sham model. Here it is described by two real functions $a(y)$ and $b(y)$, its *components*. (Sections 2.1, 2.27, 13.3, 13.13, 13.35, 14.1, 14.12, 14.17, 14.24, 15.8, 16.18, 17.2, 19.2, 19.15, 19.20 and 22.2.)

- **Orbital relaxation**: At $\tau = 0$ the integrand is Δ_{KS}; the difference $\Delta_{SCF} - \Delta_{KS}$ measures how much the levels move when the particle is transferred (the orbital relaxation). (Sections 13.27 and 15.5.)

- **Order** p: a method has order p when its error at a fixed end time is close to Ch^p for small h, with a number C that does not depend on h. Halving h then divides the error by 2^p. (Sections 2.6, 2.10, 11.2, 11.11 and 18.27.)

- **Order of accuracy**: Its order of accuracy p: its error falls like h^p when the step h shrinks. (Sections 17.2 and 17.20.)

- **Order of convergence**: the power q in an error $\approx Ch^q$; the reference has $q = 2$. (Sections 7.8, 16.4 and 16.18.)

- **An ordinary differential equation**: An ordinary differential equation (ODE) is an equation of the form. (Sections 2.2, 2.10, 12.21, 12.24, 15.8 and 18.26.)

- **Orthogonal**: two functions with $\int u_m u_n \, dx = 0$. (Sections 1.28, 2.13, 2.16, 4.2, 4.19, 7.29 and 13.2.)

- **Orthogonal matrix**: a real matrix O with $O^T O = I$, so $O^T = O^{-1}$. (Sections 1.32, 1.40 and 4.15.)

- **Orthogonal projection**: So the closest flat profile is the weighted mean $\bar{X}$ of Section 17.7; it is called the orthogonal projection of X on the flat functions. (Section 17.17.)

- **Orthonormal basis**: `eigenspace` returns, as columns, all right singular vectors of $h_k - \lambda I$ whose singular value is below 10^{-9}: an orthonormal basis (columns of length 1, perpendicular to each other) of the eigenspace of λ. (Section 8.23.)

- **Orthonormal columns**: columns of length 1 with $u^\dagger v = 0$ for any two different ones. (Section 4.11.)

- **Orthonormal** columns: each has length 1 ($u^\dagger u = 1$) and every two are perpendicular ($u^\dagger w = 0$). (Sections 4.9, 5.34, 5.37, 10.10, 10.25, 13.2 and 22.6.)

- **Oscillation and growth**: e^{-iEx_4} oscillates (its size stays 1) when E is real; when $E = \pm i\kappa$ with $\kappa > 0$ it is $e^{\pm\kappa x_4}$, which grows or decays exponentially. (Section 8.22.)

- **Oscillator**: For the oscillator, a particle of mass 1 on a spring, it is the kinetic energy minus the potential energy: $L(q, \dot{q}) = \frac{1}{2}\dot{q}^2 - \frac{1}{2}\omega^2 q^2$. (Section 7.2.)

- **Out of equilibrium**: An asymmetry can only be made while the universe is out of equilibrium, for example in decays that happen too late for the inverse processes to keep up. (Section 21.5.)

- **Outer**: Mathematica's name for the operation that builds that matrix, `Outer[delta, lower, upper]`. The author's function is called `k\[Delta]` (Mathematica's plain-text spelling of "k" followed by the Greek letter delta). (Section 1.47.)
- **Outer[delta, lower, upper]**: the author's Mathematica name for the $p \times p$ matrix with the entries $M_{ij} = \delta(l_i, u_j)$: row i belongs to the lower label l_i, column j to the upper label u_j. (Section 11.8.)

P

- **Package**: a collection of Python code that someone else wrote and that we *import* (load) to use it: numpy (arrays of numbers and matrices), sympy (exact algebra and calculus with symbols), mpmath (numbers with as many digits as we want), matplotlib (plots). (Section 0.12.)
- **Pair**: a solution together with one partner. (Sections 7.12, 11.2, 11.8, 20.2 and 20.10.)
- **Pair creation of particles**: This is the pair creation of particles, an observed process of ordinary physics, in which the total electric charge stays the same because the two members carry opposite charges. (Section 21.3.)
- **Pair curvature**: Pair curvature $K_{ab} = R^{ab}{}_{ab}$ (no sum): the curvature of the plane spanned by the directions a and b (a name used in Chapter 12). (Sections 12.2 and 12.5.)
- **Paired**: Two universes are paired when an explicit invertible rule takes every solution of the theory with mass m to a solution of the theory with mass $-m$, and takes its Lagrangian, its energy-momentum tensor and its current to stated multiples of those of the partner. (Section 18.1.)
- **Pairwise summation**: `np.sum` adds in pairs, then pairs of pairs, and so on (pairwise summation). (Section 0.25.)
- **Panel**: `draw_signs` draws one matrix into one panel `ax` (a pair of axes) of a figure. (Section 4.8.)
- **Parabola**: Two wrong paths with the same end values are the line $q = t$ and the parabola $q = t^2$. (Section 7.2.)
- **Parallel transport** along a curve $x^a(t)$: carrying a vector so that its covariant derivative along the curve is zero, $dV^a/dt + \sum_{b,c} \Gamma^a{}_{bc}\,(dx^b/dt)\,V^c = 0$ ("not turning"). (Section 3.21.)
- **Parallel transported**: A vector is parallel transported along the curve when $DV^a/d\lambda = 0$: it is carried along without turning. (Section 3.16.)
- **Parameter set** (m, λ): the mass and the coupling of the potential $U(S) = \frac{\lambda}{2}S^2$. The theory with (m, λ) and the theory with $(-m, -\lambda)$ are two DIFFERENT theories: their Lagrangians are different functions of the field. (Section 20.2.)
- **Parameters**: the mass m and the coupling λ of the potential $U(S) = \frac{\lambda}{2}S^2$. The theory with the parameters (m, λ) and the theory with $(-m, -\lambda)$ are two DIFFERENT theories: their Lagrangians are different functions of the field. (Sections 3.13 and 18.2.)

- **Parity**: the condition at the brane $y = 0$: even $b(0) = 0$, odd $a(0) = 0$ (the ASSUMED Z_2 brane). (Sections 14.12, 15.2, 16.18 and 21.24.)
- **Partial derivative** $\partial f/\partial x$: the ordinary derivative with respect to x while all the other variables are held fixed. We also write $\partial_x f$, and for the author's coordinates $\partial_4 = \partial/\partial x_4$ and $\partial_0 = \partial/\partial x_0$. (Sections 2.1, 2.18, 2.22, 3.2 and 7.2.)
- **Partial sum**: A series is an endless sum $t_0 + t_1 + t_2 + \cdots$; its partial sum $S_N = t_0 + \cdots + t_N$ adds the terms up to number N, and the series converges to a number S when the error $|S_N - S|$ becomes as small as we like for large N. (Sections 1.11 and 1.16.)
- **Particle**: A particle $b_s^*|0\rangle$ and an antiparticle $d_s^*|0\rangle$ (a hole in the filled sea) both have the energy $+E > 0$. (Sections 10.20, 10.25, 13.27, 14.21 and 16.13.)
- **Particle and antiparticle operators**: $b_s = f_{s-1}$ for $s = 1, \ldots, 8$ (modes 0 to 7) and $d_s = f_{s+7}$ (modes 8 to 15). (Section 5.37.)
- **Particle branch, sea**: by a CONVENTION of the Revision theory, the levels that are positive in the free problem (and the $k = 0$ zero modes) are particle levels; the negative branch is the filled, normal-ordered sea. (Section 14.24.)
- **Partner**: the image of a state under the map of theorem T3. (Sections 19.2, 19.20 and 20.2.)
- **Partner block**: So Γv lies in the block $(-j, s_2, s_3)$: the partner block, with the opposite type and the same s_2, s_3 (check blocks_relation_to_Gamma). (Section 19.5.)
- **PASS line**: a printed line that says a check passed; "reproduces" names the Revision record (report file and check name) that found the same result. (Section 12.15.)
- **Patch**: the range $0 < z < \pi/2$ on which the metric is used. (Sections 0.14, 3.5, 3.12, 12.2, 17.2, 18.2, 18.19, 18.26, 20.2, 20.15 and 21.2.)
- **Patch and mirror patch**: the author's metric is used on $0 < z < \pi/2$ (the patch). The map $z \to \pi - z$ carries it onto $\pi/2 < z < \pi$ (the mirror patch). (Section 20.10.)
- **Patch end**: Its end $z = \pi/2$ is the patch end; the end $z \to 0$ is the tip. (Sections 3.5, 3.12, 3.34, 8.2, 8.31, 10.27 and 10.31.)
- **Path** (or motion): a function $q(t)$ that gives the position q of a particle on a line at every time t between a start time and an end time. Here the times run from 0 to T. (Sections 7.2 and 7.8.)
- **Path integral**: a way to compute a quantum amplitude by adding one contribution for every possible history of the fields. It is named in the list of what is NOT established (Section 18.28) and not used. (Section 18.2.)
- **Pattern**: With n modes a basic state is a pattern of n occupations, each 0 or 1, so there are 2^n patterns; all combinations of them form the Fock space. (Sections 1.48, 5.34, 10.11, 10.25 and 12.25.)
- **Pattern as a whole number**: the program stores a pattern as one whole number n whose binary digit number p (counted from 0) is the occupation of mode p; the program reads that digit by shifting n to the right by p binary places and keeping the last binary digit. (Section 10.18.)

- **Pauli**: (ii) Pauli: if two orbitals are equal, $\phi_a = \phi_b$ with $a \neq b$, the columns a and b of the table are equal; the transposed table then has two equal rows, so its determinant is 0 (rule 4), and the determinant of the table is the same number (rule 2): $\Phi = 0$. (Section 13.3.)
- **Pauli matrices** $\sigma_1, \sigma_2, \sigma_3$: the three 2×2 matrices $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$. (Sections 4.3, 14.1, 14.10, 19.15 and 19.20.)
- **Pauli principle**: This is the Pauli principle, and it follows from the anticommutator alone. (Section 10.11.)
- **pdflatex**, for the six documents of the Revision record and for the book's PDF: on Windows MiKTeX (https://miktex.org), on macOS MacTeX (https://tug.org/mactex), on Linux the TeX Live packages of your distribution. (Section 23.3.)
- **Perfect fluid (at rest)**: matter whose energy-momentum tensor is diagonal, with the energy density and the pressures on the diagonal. (Section 9.29.)
- **Perfect fluid at rest**: That does not yet make it a perfect fluid at rest, which has a diagonal energy-momentum tensor: Section 9.12 showed that the spin connection puts entries off the diagonal. (Section 9.26.)
- **Period**: The length of the repeating block is the period; for 1/7 it is 6. (Sections 1.2 and 1.8.)
- **Period-doubling bifurcation**: This splitting of one fixed point into a cycle of period 2 is called a period-doubling bifurcation. (Section 13.25.)
- **Periodic box**: Put the gas in a cube of side ℓ and volume $V = \ell^3$ whose opposite faces are glued together (a periodic box). (Section 13.15.)
- **Permutation**: a re-ordering of the numbers $0, 1, \ldots, n-1$ (or of any n different objects). There are $n! = 1 \cdot 2 \cdots n$ of them. (Sections 1.19, 1.24, 1.47, 11.2 and 11.8.)
- **Permutation matrix**: the matrix with exactly one 1 in every row and every column and 0 elsewhere; row i has its 1 in column $\sigma(i)$ for a permutation σ. (Sections 1.19 and 1.24.)
- **Permutation sign** (Mathematica's *Signature*): for a list of different numbers, $+1$ if the number of *inversions* (pairs of positions in which the larger number stands first) is even, -1 if it is odd; for a list with a repeated number, 0. (Sections 4.2, 4.7 and 18.9.)
- **Perpendicular** (orthogonal) vectors: $u^T w = \sum_i u_i w_i = 0$. (Sections 1.36 and 1.40.)
- **Phantom**: $| -2 < x < -1 \mid w < -1$ (called phantom) $\mid 2 + x > 0$, and $x < -(2 + x)$ means $x < -1 \mid$. (Sections 9.19, 12.2, 22.2, 22.16 and 22.22.)
- **Phase**: a complex number $e^{i\alpha} = \cos\alpha + i \sin\alpha$ of modulus 1. (Section 21.22.)
- **Phase angle**: Every complex number $z \neq 0$ can be written as $z = re^{i\varphi}$ with $r = |z|$ and a real angle φ, its argument (or phase angle): the point $(a/r, b/r)$ lies on the unit circle, so it is $(\cos\varphi, \sin\varphi)$ for some angle φ (school trigonometry, ASSUMED), and then $z = r(\cos\varphi + i \sin\varphi) = re^{i\varphi}$. (Sections 1.11 and 1.16.)

- **Phase factor**: So $\lambda 1$ gives $-|\lambda|^2 B$, which is never B (that would need $(1+|\lambda|^2)B = 0$), and $\lambda\Gamma$ gives $|\lambda|^2 B$, which is B exactly when $|\lambda| = 1$, that is when $\lambda = e^{i\alpha}$ is a phase factor (a complex number of size 1). (Section 5.34.)
- **Phase function** $\Phi(\varepsilon)$: j times the Pruefer angle at the brane. It increases strictly with ε. (Section 15.8.)
- **Phase portrait**: the curve traced by the point (x, v) (position, velocity) as time goes on. (Section 2.10.)
- **Photon**: the particle of light (Chapter 21). (Sections 0.2, 20.17 and 20.20.)
- **Pigeonhole principle**: if more than n objects are put into n boxes, some box gets at least two of them. (Sections 1.43, 1.47, 11.2 and 11.8.)
- **Pin(4,4)**: all products $\gamma(u_1)\cdots\gamma(u_k)$ of unit vectors; Spin(4,4): those with an even number k of factors. (Sections 5.12, 5.21, 5.26, 18.2 and 18.8.)
- **Pinned**: The nine packages are installed with Python's installer pip at fixed (pinned) versions, the versions with which the book was built: numpy 2.4.6 (arrays of numbers, matrices), sympy 1.14.0 (exact algebra and calculus with symbols), mpmath 1.3.0 (numbers with as many digits as we ask for), matplotlib 3.11.0 (plots), and jupyterlab 4.4.10, nbformat 5.10.4, nbclient 0.10.2, ipykernel 7.1.0 and nbconvert 7.16.6 (the programs that show, store and run notebooks). (Section 0.8.)
- **Pinned version**: the exact version of a package with which the book was built. Another version may print slightly different numbers or pictures. (Section 0.12.)
- **Pivots**: Write $T - x\mathbb{1}$ (where $\mathbb{1}$ is the unit matrix) as a product $\mathcal{L}D\mathcal{L}^T$, with $\mathcal{L}$ (a script letter, because L is the length 3 of the interval) having ones on its diagonal and one band below it, and D diagonal with entries $q_1, \ldots, q_n$, the pivots. (Section 16.15.)
- **Plain iteration**: $x \leftarrow G(x)$; linear mixing: $x \leftarrow (1-\beta)\,x + \beta\,G(x)$ with the mixing parameter β. (Sections 13.21 and 13.24.)
- **Plane curvature**: Christoffel symbol $\Gamma^a{}_{bc}$, Riemann tensor $R^a{}_{bcd}$ and $R^{ab}{}_{cd} = g^{bb}R^a{}_{bcd}$, plane curvature $K(a,b) = R^{ab}{}_{ab}$ (no sum), Ricci tensor $R^h{}_j = \sum_c R^{hc}{}_{jc}$, Ricci scalar $R = \sum_h R^h{}_h$, Einstein tensor $G^h{}_j = R^h{}_j - \frac{1}{2}\delta^h_j R$: Chapter 3 (Sections 3.15 and 3.17) defines them. (Section 11.2.)
- **Plane wave**: a solution of the form $u\,e^{i(k\cdot x - Ex_4)}$, with a constant column u; E is its *energy* (frequency in the time x_4) and the k_a are its *momenta* (wave numbers in the other directions). (Sections 4.9, 4.11, 7.5, 7.8, 8.17, 8.22, 9.21, 10.3, 10.9, 13.15 and 13.19.)
- **PNG**: PNG is the file format in which the figures are saved. (Section 0.12.)
- **Polar coordinates**: Polar coordinates of the plane: the distance r from the origin and the angle φ from the horizontal axis; $x = r\cos\varphi$, $y = r\sin\varphi$. (Section 3.21.)
- **Polar form**: $z = re^{i\varphi}$ with $r = |z|$ and φ the argument. (Section 1.16.)
- **Polarisation**: With the polarisation $\zeta = (n_\uparrow - n_\downarrow)/n$, so $n_\uparrow = \frac{n}{2}(1+\zeta)$ and $n_\downarrow = \frac{n}{2}(1-\zeta)$: one label alone ($g = 1$) has the kinetic energy $\frac{3}{10}(6\pi^2)^{2/3}n_\sigma^{5/3}$ per volume (the formula above with $g = 1$), and the contact interaction gives $g_c\,n_\uparrow n_\downarrow$ (Section 13.5). (Section 13.15.)

- **Polarization** $\zeta = (n_{up} - n_{down})/n$: how unequally two labels are occupied. (Section 13.19.)
- **Polynomial**: By the Leibniz formula every term of $\det(M - \lambda I)$ is a product of n entries, of which at most n (the diagonal ones) contain λ; so p is a polynomial (a sum of powers of λ with number coefficients) of degree n. (Sections 1.34, 2.5, 11.18 and 12.16.)
- **Populations**: The right panel shows the populations of M5: the energy density of each of its three parts, computed by calling `model` with a list that holds only that part (`[0]` takes the first of the two returned sums, the energy density), in the colours from the fifth on (`PALETTE[ 4:]`), and their total in black, with the zero line. (Section 22.23.)
- **Positive branch**: the levels whose value without interaction ($\lambda = 0$) at the same slice is positive. By the filling convention of the Kohn-Sham record, particles occupy the positive branch together with the zero modes at $k = 0$ (the levels $\varepsilon = 0$ of the free problem); the levels of the negative branch form the normal-ordered sea and are never occupied, not even at a temperature. (Section 15.2.)
- **Positive representation**: Because B has eight eigenvalues $+1$ and eight -1, $\Psi^\dagger$ cannot be the Hilbert adjoint of Ψ; the Revision record realises it as $\Psi^\dagger = b^* B$ (the positive representation: $\Psi_A = b_A$, $\Psi^\dagger_C = \sum_D b^*_D B_{DC}$). (Sections 21.25 and 21.29.)
- **Potential**: In quantum mechanics the allowed energies E of a particle in one dimension, moving in a potential $V(x)$ (its potential energy at the position x), are the eigenvalues of the Schrödinger equation; in units in which Planck's constant divided by 2π (written $\hbar$) and the mass m are both 1, it reads $-\frac{1}{2}\,u''(x) + V(x)\,u(x) = E\,u(x)$. (Sections 2.13, 12.29, 15.2 and 18.4.)
- **Potential energy**: Kinetic term K_μ: the part of the Lagrangian that holds the derivative along the direction μ; potential energy: $mS + U(S)$, the part without derivatives. (Section 9.16.)
- **Potential part**: The terms with derivatives (the K's) form the kinetic part; the terms without derivatives, $mS + U(S)$, form the potential part. (Section 9.7.)
- **Potential well, bound state**: a region where V is lower than outside; a bound state is a solution with E below the outside value of V that decays to zero far away. Its energy is an eigenvalue. (Section 2.16.)
- **Power**: Power a^p and root $a^{1/n}$; the laws of powers $a^p a^q = a^{p+q}$, $(a^p)^q = a^{pq}$, $(ab)^p = a^p b^p$, $a^{-p} = 1/a^p$ for positive a, b. (Section 1.8.)
- **Power iteration**: multiply a vector by M again and again and rescale it to length 1; it turns towards the eigenvector of the eigenvalue of largest size. (Sections 1.37 and 1.40.)
- **PowerShell 7**: on Windows, the program PowerShell 7 (the command `pwsh`), downloaded from the address aka.ms/powershell, which runs the PowerShell twin of the gate; Git for Windows also brings bash, so the bash form of the gate works there too. (Section 23.3.)
- **Prescribed background**: a metric that is given and not solved for from field equations. (Sections 0.2, 0.14, 3.12, 3.34, 14.1, 14.32, 15.16, 17.1, 17.2, 17.10, 17.20, 21.31 and 22.2.)
- **Prescribed source**: a source chosen by hand to see what the equations do. It is ASSUMED, not derived from a field. (Sections 12.2 and 12.24.)

- **Present measure**: `\w+` is one or more letters, digits or underscores (the name of a state, such as N8_lam0_a00_T10), `\S+` a number, and `\(` and `\)` are round brackets themselves; the four bracketed parts are the name of the state, the error of the old method, the former measure (the difference between the two runs when both used the old method) and the present measure (the difference seen by the present refined run). (Section 0.25.)

- **Pressure**: Cosmologists describe the matter of the universe, at each moment, by two numbers: the energy density ρ (energy per unit volume) and the pressure p (the push per unit area, which is also energy per unit volume). (Sections 3.26, 3.29, 9.1, 9.16, 9.24, 12.2, 15.16, 15.19, 17.2, 17.10 and 20.2.)

- **Prime**: $a_4' = da_4/dx_4$ and $a_4'' = d^2a_4/dx_4^2$. The code writes them as the symbols `ad1` and `ad2`. (Sections 12.2 and 17.2.)

- **Prime number**: a whole number larger than 1 that only 1 and itself divide. (Sections 1.2 and 1.8.)

- **Principal part**: of a system of first-order partial differential equations at a point: the derivative terms alone, with their coefficients taken at that point. For the field equation with $U = 0$ it is the mode matrix without the mass term, with the frame momenta of that point. (Section 8.18.)

- **The principle of stationary action** (ASSUMED: it is the starting principle of classical mechanics and of classical field theory, not derived from anything simpler). Among all paths with the same end values $q(0) = q_A$ and $q(T) = q_B$, the particle follows one whose action does not change to first order when the path is changed a little. (Sections 7.2 and 7.8; the opening of Chapter 7.)

- **Private environment**: a folder with its own copy of Python and of the packages, so that installing them changes nothing else on the computer. (Section 0.12.)

- **A private Python environment**: A private Python environment is a folder with its own copy of Python and of the packages, so that installing the packages changes nothing else on your computer. (Section 0.8.)

- **Probability**: Amplitude and probability: when an orbital is written as a sum $\sum_n c_n\varphi_n$ of instantaneous orbitals, the complex number c_n is the amplitude of level n and $|c_n|^2$ the probability to find the particle in level n. (Sections 0.18, 20.2 and 22.2.)

- **Probability amplitude**: the number from which quantum theory computes how likely a process is (Chapter 10). (Section 0.2.)

- **Process**: a solution of a dynamical equation that connects an initial state with a different final state. (Section 20.2.)

- **Product** γ_A: for a set A of directions, the product of their gamma matrices in increasing order, for example $\gamma_{\{x_2,x_5\}} = \gamma^{(x_2)}\gamma^{(x_5)}$; for the empty set, $\gamma_{\{\}} = I_{16}$. (Sections 1.12, 1.34, 4.15, 4.19, 18.8 and 21.5.)

- **Product Fock space**: the Fock space of the modes of both universes together, here 16+16 = 32 fermion modes (2^{32} occupation patterns). (Section 10.48.)

- **Product of a set of directions** γ_A: for a set A of directions, the product of their gammas in increasing order, for example $\gamma_{\{x_2,x_5\}} = \gamma^{(x_2)}\gamma^{(x_5)}$; $\gamma_{\{\}} = I_{16}$. There are $2^8 = 256$ sets, so 256 products; they are a *basis* of all real 16×16 matrices when every such matrix is exactly one combination $\sum_A c_A \gamma_A$. (Section 4.19.)

- **Product of different gammas**: a product $P = \gamma^{a_1}\gamma^{a_2}\cdots\gamma^{a_k}$ of k gammas of k different directions $a_1, \ldots, a_k$; the number k is its degree. (Section 5.3.)

- **Product of matrices**: $(AB)_{ij} = \sum_k A_{ik}B_{kj}$ (row i of A times column j of B, entry by entry, added up). In Python it is written `A @ B`. The order matters: in general $AB \neq BA$. (Section 4.7.)

- **Product rule**: For commutators the product rule $[A, BC] = [A, B]C + B[A, C]$ holds, because both sides are $ABC - BCA$ (on the right, $ABC - BAC + BAC - BCA$). (Sections 5.18 and 7.3.)

- **Profile**: a density as a function of y, stored at the 151 points $y = -3 + 0.02\,i$. (Sections 16.11, 17.2, 17.7 and 17.10.)

- **Projector**: a matrix P with $PP = P$. It maps every column into one subspace, its range, and leaves the columns of that range unchanged; its eigenvalues are 0 or 1, so its trace is the number of eigenvalues 1. (Sections 5.5, 5.9, 8.25, 9.26, 9.29, 9.30, 10.5, 10.9, 14.5, 14.10, 14.26, 18.2, 18.8 and 21.18.)

- **Proper**: Densities: the particle density $n(y)$, the scalar density $S(y)$ and the density $Q(y)$; proper means per unit of proper 7-volume; the coordinate density $e^{6Hy}n(y)$ is the density per unit of y. (Sections 15.2 and 19.20.)

- **Proper density**: a number of particles (or an energy) per unit of proper 7-volume. (Sections 15.13, 15.19, 17.2 and 17.10.)

- **Proper time** τ: the time shown by a clock carried by the body. (Sections 3.4, 3.16 and 3.34.)

- **Proposition**: a proved statement (Notebook 20c); a lemma is a proposition that serves as a step towards others. (Section 20.20.)

- **PROVED**: a status label: the statement is exact. The book gives the complete proof and, where the Revision record contains an exact computer check of the statement, names it: the report file that the verifier wrote and the name of the check in it. A statement is proved by an exact computation of every case, by an exact symbolic computation that holds for every value of its symbols, or by a general derivation in the book (Section 0.3). (Sections 0.3, 0.20, 3.1, 4.21, 5.40, 6.1, 6.30, 8.1, 12.31, 13.38, 14.32, 15.31, 16.26, 17.23 and 22.27.)

- **Provenance file**: `NNx_name.PROVENANCE.md`, next to each notebook. Its last line records the measured facts of the notebook's last verified run, among them the date, result and seconds of its last nbkit check. (Sections 0.7, 23.2 and 23.11.)

- **Pruefer angle** $\theta(y)$: the angle of the point (a, b) seen from the origin, $a = r\cos\theta$, $b = r\sin\theta$ with $r > 0$; we count every full turn, so θ can grow beyond 2π (*winding*). r is the distance of the point from the origin, $r^2 = a^2 + b^2$. (Sections 2.1, 2.24, 2.27, 14.14, 14.17, 15.8 and 22.12.)

- **Pruning**: Leaf, pruning (skipping): a complete combination of k curvature entries in the computer's search; pruning means leaving out combinations known to give zero. (Section 11.2.)
- **Pseudo-random generator**: a formula that turns a number (its state) into a new state and an output, again and again; the outputs look random but are completely fixed by the first state, the seed. (Sections 1.49 and 1.53.)
- **Pseudo-random numbers**: Pseudo-random numbers: numbers made by a fixed recipe that look random; the recipe starts from a number called the seed, and the same seed gives the same numbers on every computer. (Section 11.8.)
- **Purely imaginary**: A real number a is the complex number $a + i0$; a number ib with real part 0 is called purely imaginary. (Section 1.10.)
- **Python**: the programming language of the notebooks. A *version number* such as 3.14.5 names one release of a program: the first number (3) changes rarely, the second (14) for new features, the third (5) for corrections. (Sections 0.8, 0.12 and 0.20.)
- **Python dictionary**: a table that maps keys to values; here monomials (tuples of generator numbers) to coefficients. (Section 7.16.)

Q

- **Q** (dirac16complex): the quantum reading of T1; the image field carries the indefinite (Krein) metric $-B$ and is the same quantum system written in other variables, and there is no cancellation between two universes that are quantised independently of each other. (Section 0.2; the opening of Chapter 18.)
- **QR factorisation**: `rng.normal(size=(M, M))` is a 4×4 table of random numbers, `1j` is the imaginary unit i, so the argument is a random complex matrix; `np.linalg.qr(...)[0]` is the first factor of its QR factorisation, a unitary matrix U (its columns are orthonormal). (Section 13.14.)
- **Quadratic convergence**: The new error is proportional to the square of the old one (quadratic convergence): once the error is small, the number of correct digits roughly doubles at every step. (Section 16.20.)
- **Quadratic form** of a symmetric matrix S: the number $Q(v) = v^T S v = S_{ab} v^a v^b$. (Sections 1.36, 1.40, 4.1, 4.11 and 5.4.)
- **Quantise**: To quantise dirac16complex means to replace its 16 components Ψ_A by operators and to fix their anticommutators. (Section 10.12.)
- **Quantised**: turned into a quantum theory, in which the field becomes an operator, a rule that turns one state of the system into another (Chapter 10). (Sections 0.2 and 5.34; the opening of Chapter 21.)
- **Quantised field, anticommutator**: after quantisation the components are operators with $\{\Psi_r, \Psi_c^\dagger\} = B_{rc}\,\delta$ (δ the delta function of the positions); a conjugation of the quantised field must preserve this rule. (Section 5.32.)

- **Quantum**: An occupation state has each of the 16 modes either empty or holding one quantum (one excitation of the quantised field); Chapter 10 splits the 16 modes into 8 particle modes and 8 antiparticle modes (an antiparticle has the opposite charge of a particle, and both have positive energy). (Section 9.13.)
- **Quantum cosmology**: a quantum theory in which a few numbers that describe the whole geometry, such as a scale factor, are quantum variables; its wave function (the wave function of the universe) is a function of these numbers and obeys the Wheeler-DeWitt equation. It is named in Chapter 22, not used. (Sections 22.2 and 22.4.)
- **Quantum electrodynamics**: That pairs are really made above the threshold, and how often, was computed from quantum electrodynamics, the quantum theory of electrons and light, by Bethe and Heitler (H. Bethe and W. Heitler, Proc. (Section 20.17.)
- **Quantum reading Q**: The Revision pairing record answers with five statements, the quantum reading Q (record `Revision/pairing/pairing-theory.json`, theorem entry `Q`; Revision proof document, section 6). (Section 10.44.)
- **Quantum state**: In the examples of Chapter 10 a quantum state is a column $\psi = (\psi_1, \ldots, \psi_n)$ of n complex numbers. (Section 10.11.)
- **Quark**: a constituent of protons and neutrons. (Section 21.35.)
- **Quasi-free state, Wick's rule**: a Slater determinant or a thermal state of independent quanta; the expectation of four field operators is a sum of products of two-operator expectations (a "direct" and an "exchange" term). (Section 14.30.)
- **Quaternion**: *The blocks are quaternion multiplications.* A quaternion is $q = q_4 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k}$ with four real numbers; we store it as the list (q_1, q_2, q_3, q_4), the real part last, to match the blocks. (Section 4.5.)

R

- **Radians**: with θ in radians (the angle measured as the length of the arc on the circle of radius 1; π radians are 180 degrees). (Section 1.11.)
- **Radiation-like**: $w_{\text{eff}} = 1/3$; dust-like (dark-matter-like): $w_{\text{eff}} = 0$; cosmological-constant-like: $w = -1$; phantom: $w < -1$. (Section 22.22.)
- **Radius**: So every eigenvalue lies within the radius ρ_r of some diagonal entry (Gershgorin's theorem), and the interval from $\min(d - \rho) - 1$ to $\max(d + \rho) + 1$ contains them all. (Section 16.15.)
- **Raising**: Raising with η^{ab} undoes it: $\eta^{ab} v_b = \eta^{ab}\eta_{bc} v^c = \delta^a{}_c v^c = v^a$ (insert the lowered list; the inverse rule; the delta renames). (Section 1.27.)
- **Random numbers with a seed**: Random numbers with a seed: numbers that look random but are the same in every run, because the generator starts from a fixed number. (Section 4.11.)
- **Random walk**: a path that takes a step of $+1$ or -1 at random, again and again. (Sections 0.24 and 0.25.)

- **Random-number generator**: `rng` is a random-number generator with a fixed starting value (the seed 12345), so that the "random" numbers it gives are the same in every run and the figure is the same byte for byte. (Sections 0.24, 1.25, 4.12 and 16.12.)
- **Rank** of a list of matrices: the largest number of independent matrices in it, which is the dimension of its span. sympy computes it exactly with fractions. (Sections 4.13, 4.15, 4.16, 4.19, 5.11, 5.16, 5.26, 8.15, 9.29, 9.30, 10.10, 10.48, 14.5, 16.11, 16.12, 16.13 and 21.15.)
- **Rapidity** b: the "angle" of a boost. (Sections 1.13, 1.16, 5.18, 6.16, 6.21 and 8.3.)
- **Rapidity rate**: The rapidity may change from point to point; we take $b = \beta x_4 + b_0$ with two constants β (the rapidity rate) and b_0, a different boost at every time. (Section 8.3.)
- **Rate** K: a number of events per unit time. (Sections 20.2, 20.20, 21.2, 21.35, 22.2 and 22.11.)
- **Ratio**: The right side is the tolerance, and the left side divided by the right side is the ratio. (Sections 16.2, 16.7, 16.11 and 16.23.)
- **Rational number**: A fraction (or rational number) is a quotient p/q of two whole numbers with $q \neq 0$. (Section 1.2.)
- **Raw string**: A string that starts with `r` is a raw string: its backslashes are kept as they are, which the mathematical titles (typeset by matplotlib between dollar signs) need. (Sections 0.17, 5.10 and 21.15.)
- **Real**: The author's gamma matrices are real: every entry is -1, 0 or $+1$. (Sections 4.1, 5.28, 5.32, 7.11, 7.16, 8.2, 9.2, 19.15, 21.2, 21.8, 21.11 and 21.14.)
- **Real field**: a column Φ whose 16 components are real ($\Phi^* = \Phi$); for Grassmann components, a column of real generators ($\theta^* = \theta$). (Sections 7.28 and 7.32.)
- **Real form**: writing $\chi = (a, ib)$ with real a, b makes all coefficients real. (Sections 14.17, 19.3 and 22.6.)
- **Real matrix**: a matrix whose entries are real numbers (no i). The author's gammas are even simpler: every entry is -1, 0 or $+1$. (Section 18.8.)
- **Real number**: a number on the number line; it may need infinitely many digits that do not repeat, like $\sqrt{2}$ or π. (Sections 1.3 and 1.8.)
- **Real part**: The number $a = \mathrm{Re}\, z$ is its real part and $b = \mathrm{Im}\, z$ its imaginary part. (Sections 1.10 and 1.16.)
- **Reality**: (ii) Reality: every entry of every γ^a is -1, 0 or $+1$, by (i). (Section 4.5.)
- **Reality condition**: A field equal to its own conjugate obeys $\Psi = M\Psi^*$; such a requirement is called a Majorana or reality condition, after the physicist who first used one. (Section 5.29.)
- **Record**: short for the Revision record (see that entry): the committed files of the folder Revision, from which every formula and number of the book comes. A record file is one of them, for example a report whose numbers a notebook reproduces. (Sections 0.16, 5.9 and 16.11.)

- **Recursive**: A function that calls itself is recursive: it goes down into the nested structure until it reaches single values. (Section 16.12.)
- **Redshift**: a 3-momentum k acts like $k\,e^{-a_{4,0}}$ at the slice $a_{4,0}$: momenta shrink as 3-space inflates. (Section 14.24.)
- **Redshift, blueshift**: the decrease, increase of a frame velocity (or momentum) caused by the expansion, contraction of the space it points along. (Section 3.34.)
- **Redshifted**: At a fixed height y, along a deflating history (a_4 increasing, b_4 constant), the frame velocity in 3-space falls like e^{-a_4}: motion in 3-space is redshifted, because 3-space inflates and stretches it out. (Sections 3.31, 14.2 and 15.4.)
- **Redshifted momentum** $q = ke^{-a_{4,0}}$: the 3-momentum as the deflating history sees it (Section 22.7). (Sections 22.2, 22.7 and 22.11.)
- **The reference solver**: The reference solver (`Revision/kohn_sham/reference/run_reference.py` with the module `ks_fd.py` in the same folder). (Sections 16.3 and 16.11; the opening of Chapter 16.)
- **Refined numerics**: Refined numerics: $G = 1800$ steps and ten times smaller tolerances. (Sections 16.6 and 16.11.)
- **Refined run**: `Revision/kohn_sham/reports/ks-rust-determinism.json` (14 checks, all PASS) compares a second run with the canonical one (all 244 files, the 243 result files and the manifest, byte-identical; check repeat_byte_identical) and a refined run with twice the RK4 steps and ten times smaller tolerances, against tolerances fixed before the comparison (10^{-8} for levels, energies and thermodynamics, 10^{-6} for profiles and derived quantities): for example E_{KS} agrees to 1.9×10^{-12} relative (check refined_ground_energies) and 23724 levels agree label by label to $2.1 \times 10^{-9}\,m$ (check refined_eigenvalues). (Section 15.10.)
- **Reflection** R_u along a unit vector u: $R_u v = v - 2\,\eta(u,v)\,u/\eta(u,u)$; it reverses u and keeps every direction orthogonal to u. (Sections 5.12, 5.21, 5.26, 18.2 and 18.8.)
- **Regular**: The decaying branch, $\sigma_2\chi = -\chi$ for $k > 0$, is the regular one. (Section 14.12.)
- **Regular expression**: A regular expression is a pattern that describes a family of texts: in `(\d+)/(\d+)`, `\d` means one digit, `+` means one or more of what stands before it, and the round brackets mark the parts to return. (Sections 0.21, 1.48, 2.28, 7.9, 9.30, 11.9, 14.18, 15.20, 16.12, 19.21 and 23.12.)
- **Regular tip**: the chosen condition $b(-L) = 0$ at the cutoff. (Sections 14.12 and 14.17.)
- **Relative difference**: $|a - b|/\max(|a|, 1)$; numbers that agree to the last digits of a computer have relative differences near 10^{-15}. (Section 19.20.)
- **Relative error**: |computed − exact| divided by |exact|. (Section 1.8.)
- **Remainder**: The last term is the remainder: it shows that the error of keeping only the terms up to h^n is proportional to h^{n+1} when h is small. (Sections 1.2 and 2.3.)
- **Repeating decimal**: a decimal whose digits repeat a block for ever, written with the block in brackets: $1/7 = 0.(142857)$. (Section 1.8.)

- **Report**: a JSON file of the Revision record that holds a list (or table) of named checks, each with a verdict such as PASS. (Sections 0.4, 0.16, 0.20, 3.13, 4.7, 13.20, 23.2 and 23.11.)
- **Repository**: the folder Dirac_claude with every file of this project, downloaded with the program Git. (Sections 0.1, 0.8 and 23.2.)
- **Repository root**: The repository root is that folder itself; every command of Chapter 23 is typed there. (Section 23.2.)
- **Representation, invariant subspace, irreducible**: a group of 16×16 matrices *represents* the group on columns of 16 numbers. A set of columns that every group matrix maps into itself is *invariant*. (Section 5.16.)
- **Required source**: the ρ and p that the field equations demand for a given a_4; we read them off the equations instead of assuming a kind of matter. (Sections 12.1 and 12.19.)
- **Rescaling**: writing the field as a known function times a new field, $\Psi = w(x)\,\chi$, and deriving the equation that χ obeys. (Sections 6.21 and 8.35.)
- **Residual**: in the self-consistent loop, the largest change of the potential from one iteration to the next; the loop stops when it is below 10^{-11}. (Sections 5.33, 9.25, 13.32, 13.35, 15.5, 15.8, 15.13, 16.23 and 20.15.)
- **Rest energies**: For a heavy nucleus the threshold approaches the two rest energies $2m$: for $M = 1000m$ it is $1001m/500 = 2.002m$, and for $M = 1836m$ (about the mass of a proton) $1837m/918 \approx 2.001089m$. (Section 20.17.)
- **Rest state**: A rest state $\Psi = u\,e^{-imx_4}$ with $-i\gamma^{(x4)}u = u$ solves it: $\gamma^{(x4)}\partial_4\Psi = \gamma^{(x4)}(-im)u\,e^{-imx_4} = m(-i\gamma^{(x4)}u)e^{-imx_4} = m\Psi$. (Section 21.18.)
- **Restricted**: Hartree-Fock (HF): the best single determinant; restricted (both labels in the same orbital) or unrestricted (each label its own orbital). (Section 13.13.)
- **Return code**: Every program ends with a return code, 0 for success. (Sections 11.9, 15.14, 16.12 and 19.21.)
- **Revision record**: the computations stored in the folder Revision of the repository, done anew for the author's metric. Every number of the book comes from it or from a notebook of the book. (Sections 0.1, 0.20, 1.1, 1.24, 4.7, 11.8, 11.18 and 14.10.)
- **Ricci scalar**: Riemann tensor $R^a{}_{bcd}$ (the MTW convention of the Revision records), Gaussian curvature $\sigma = R^{\theta\varphi}{}_{\theta\varphi}$ of a surface, Ricci scalar R, Kretschmann scalar K: the measures of curvature. (Sections 3.17, 3.21, 3.29, 6.21, 11.2, 11.18, 12.2 and 12.15.)
- **Ricci tensor** $R^h{}_j$, **Ricci scalar** R, **Einstein tensor** $G^h{}_j$: sums of Riemann components defined in Section 12.6. (Sections 3.17, 3.29, 6.21, 11.2, 11.18, 12.2 and 12.15.)
- **Richardson extrapolation**: combining two finite-difference estimates so that their leading errors cancel. (Sections 14.19, 14.24, 15.9, 16.3, 16.5, 16.11, 16.18 and 19.16.)
- **Richardson's rule**: an estimate of the error made from two computations with the step sizes h and $h/2$. (Section 2.10.)

- **Riemann tensor** $R^{\rho}{}_{\sigma\mu\nu}$: the measure of curvature, built from the Christoffel symbols and their derivatives; it is zero everywhere only for a flat space. (Sections 3.21, 3.29, 6.21, 8.14, 11.2, 11.18, 12.2 and 12.15.)

- **Right derivative**: The right derivative $\partial_R F/\partial\theta_k$ moves θ_k to the far right instead. (Section 7.11.)

- **Right multiplication**: For a fixed quaternion u the right multiplication R_u maps q to $q\,u$ and the left multiplication L_u maps q to $u\,q$. (Section 4.5.)

- **Right-hand side**: for an unknown function $y(t)$, where f is a known rule that produces a number from t and y, the right-hand side. (Section 2.2.)

- **RK4**: the classical Runge-Kutta method of order 4 for solving $dy/dt = f(t, y)$ step by step. (Sections 3.21, 3.34, 8.22, 8.28, 12.24, 14.17, 15.8 and 22.11.)

- **RK4 step count** G: the number of RK4 steps on $-L \le y \le 0$; the step is $h = L/G$. (Section 2.27.)

- **Root** (or zero) of a polynomial: a number where it is 0. (Sections 1.8, 1.34, 1.40, 15.8 and 16.23.)

- **Root of unity**: a complex number z with $z^n = 1$ for a whole number $n \ge 1$. (Section 1.16.)

- **Root-mean-square width**: The root-mean-square width $\sqrt{\int x^2 n\,dx/N}$ measures how far the cloud spreads; `.format(*widths)` puts the three numbers into the three braces of the string. (Section 13.36.)

- **Roots of unity**: They are the roots of unity of order n, the corners of a regular polygon with n sides on the unit circle. (Section 1.12.)

- **Rotation**: turning the plane about the origin by an angle; it keeps every length. (Sections 1.12, 1.16, 4.13, 5.12, 5.18, 5.21, 5.26, 10.34 and 10.37.)

- **Rotation matrix**: The matrix of $e^{i\alpha} = \cos\alpha + i\sin\alpha$ is the rotation matrix. (Sections 1.12 and 1.16.)

- **Round half to even**: If it lies exactly halfway between two neighbours, the rule is to take the neighbour whose m is even (round half to even). (Section 0.22.)

- **Rounding**: replacing a number by the nearest floating-point number. It happens after every addition, multiplication and division. (Sections 0.22 and 0.24.)

- **Rounding bound**: Rounding bound: an upper limit for the error of a computed root. (Section 16.23.)

- **Rounding error**: the difference between the exact result of an operation and the double that the computer stores. (Sections 2.10, 16.2 and 16.23.)

- **Rounding floor**: For RK4 on Problem A this rounding floor is near 10^{-16}, and with more steps the error grows again but stays 524 to 3709 times below the bound $N\epsilon e^{-1}$ (COMPUTED, Notebook 02a, In [10]). (Section 2.6.)

- **Runge-Kutta method**: The classical fourth-order Runge-Kutta method (Chapter 2): from the start $n = 1$, $a = 0$, each step evaluates the slopes at the beginning, twice in the middle and at the end of the step, and advances with their weighted average; the error over a fixed time falls like dt^4. (Sections 9.24, 9.25, 18.26, 19.16 and 21.36.)
- **Runge-Kutta rule of order 4**: Then comes the classical Runge-Kutta rule of order 4 (RK4, Chapter 2): the first slope `p1` from the equation at the start; a half step with it to the midpoint and the second slope `p2` there; a half step with `p2` and the third slope `p3`, again at the midpoint; a full step with `p3` and the fourth slope `p4` at the end; and the new values as the start plus one step times the weighted mean $(p_1 + 2p_2 + 2p_3 + p_4)/6$. (Section 22.12.)
- **Running sum**: Two panels with a shared vertical axis (`sharey=True`); the loop draws in each a bar chart with grey bars for x_0 and x_4, blue for 3-space and red for the extra times, and the running sum (`np.cumsum`, the sum of the first one, two, three, ... values) as black dots joined by lines. (Sections 1.33 and 7.27.)
- **Rust**: Rust is a second programming language. (Sections 0.8, 0.12, 0.20 and 23.3.)
- **The Rust solver**: The Rust solver (Chapter 15). (Sections 15.10, 16.3 and 16.11.)
- **Rust uncertainty**: Rule: multiply by $\frac{16}{15}$; the size of the canonical error is the Rust uncertainty. (Section 16.6.)
- **Rust, cargo, program**: Rust is a programming language; cargo builds a Rust crate (a folder with a file `Cargo.toml` and source files) into a program file that the computer runs. (Section 11.8.)

S

- **Saddle**: For $T > \pi/\omega$ the coefficient is negative: the true path is still stationary ($S_1 = 0$), but along ξ_1 the action is a maximum, while along faster shapes (ξ_n with large n) it is still a minimum; such a point is called a saddle. (Sections 7.2 and 13.32.)
- **Safeguarded Newton-bisection**: The Revision Rust solver finds the root of the same function Φ with fewer evaluations, by the safeguarded Newton-bisection of Section 2.24 (the function find_level of `Revision/kohn_sham/solver/src/shoot.rs`): it widens a bracket around a first guess until $\Phi-$ target changes sign, then takes Newton steps with the derivative $d\Phi/d\varepsilon = \int r^2 dy/r(0)^2$ derived above, takes the midpoint of the bracket instead whenever a Newton step would leave the bracket or the last step has not at least halved the mismatch $|\Phi - \text{target}|$, and stops when the bracket, or its last Newton step, is shorter than the tolerance 10^{-13} (rootTolerance of `Revision/kohn_sham/results/parameters.json`). (Section 14.14.)
- **Sakharov's three conditions**: the three ingredients that, as A. D. Sakharov showed in 1967, any process must contain that makes more matter than antimatter from a start with equal amounts: (1) some process changes the baryon number; (2) C and CP are violated; (3) these processes happen out of thermal equilibrium. In the theory of this book the only number of this kind is the U(1) charge; Section 21.31 applies the three conditions to the theory as built. (Sections 21.5, 21.31 and 21.35.)
- **Same mass**: solves the field equation of the same form, either with the same V (same mass) or with $-V$ (mass reversed). (Sections 5.28, 5.32 and 21.8.)

- **Sample point**: `at_sample(expr)` gives the decimal value of an expression at the sample point $H = 1$, $z = \pi/4$, $a_4 = \frac{1}{2}$, $a'_4 = \frac{1}{2}$, $b_4 = 1$, $b'_4 = \frac{1}{2}$, $a''_4 = b''_4 = 0$. (Section 6.14.)

- **Scalar**: Its Dirac adjoint is the row $\bar{\Phi} = \Phi^\dagger C$ (the dagger means: transpose and take the complex conjugate of every entry), and the scalar is the number $S = \bar{\Phi}\Phi = \Phi^\dagger C \Phi$. (Sections 5.4, 5.32, 9.2, 18.2, 18.26, 21.2 and 21.14.)

- **Scalar density** $S = \bar{\Psi}\Psi = \Psi^\dagger C \Psi$: the bilinear that the mass term and the potential of the Lagrangian contain. (Sections 7.12, 7.16, 7.19 and 19.4.)

- **Scale**: where U_{ref} is the Richardson uncertainty of the reference, U_{Rust} the measured uncertainty of the Rust solver, and the scale s is $\max(1, |x_{ref}|)$ for a single number and the largest value of the profile for a point of a profile. (Section 16.7.)

- **Scale factor** $s_a = \sqrt{|g_{aa}|}$: the factor by which a coordinate length along x_a is multiplied to give a true length. (Sections 2.1, 2.22, 3.7, 3.12, 3.34, 7.26, 8.2, 8.28, 10.42, 12.2, 12.15, 12.24, 14.2, 14.10, 17.2 and 21.7.)

- **Scaled commutator**: For two directions a and b the scaled commutator. (Section 5.12.)

- **Schroedinger equation**: the equation of quantum mechanics that gives the allowed energies E of a particle in a potential $V(x)$; here, for one dimension and in units with $\hbar = m = 1$ ($\hbar$ is Planck's constant divided by 2π, m the mass), $-\frac{1}{2}u''(x) + V(x)u(x) = Eu(x)$. We take it as given. (Sections 2.13 and 2.16.)

- **Schur's lemma** (used here, proved in the text): when the matrices of a representation span all matrices, the representation is irreducible and its commutant is the multiples of 1; an intertwiner between two irreducible representations is zero or invertible. (Section 5.16.)

- **Schwarz's theorem**: This is Schwarz's theorem: for a function whose second partial derivatives are continuous, the order of differentiation does not matter (ASSUMED: a standard theorem of calculus, quoted without proof; Notebook 02c confirms it for this example, and Section 2.24 uses it). (Section 2.18.)

- **Scientific notation**: The `say` line prints the errors in scientific notation (`{e:+.3e}` gives three decimals and a power of ten) and the ratios with four decimals; `", ".join(...)` joins the texts with commas. (Section 7.9.)

- **Scratch folder**: A scratch folder is a folder for throw-away files. (Section 23.2.)

- **Sea**: the levels of the negative branch, which the filling CONVENTION of the record leaves out (normal ordering); sea holes would be thermal antiparticles. (Sections 5.34, 14.21, 15.2 and 15.29.)

- **Sea holes**: Sea: the levels of the negative branch, which the filling CONVENTION of the record leaves out (normal ordering); sea holes would be thermal antiparticles. (Section 15.29.)

- **Secant method**: For one variable and two remembered passes the combined residual can be made exactly zero, and the next input is the point where the straight line through the two last (input, residual) pairs crosses zero: the secant method, which estimates the slope of G from the history. (Section 13.21.)

- **Secant rule**: draw the straight line through the last two points $(\lambda, F(\lambda))$ and take its zero as the next guess. (Section 2.16.)
- **Second difference quotient**: The bracket is $E = -\ddot{q} - \omega^2 q$ with $\ddot{q}$ replaced by the second difference quotient $(q_{n+1} - 2q_n + q_{n-1})/h^2$. (Section 7.3.)
- **Second order**: Such a method is said to be of second order (or to have order of convergence 2), because for small h the first term $c\,h^2$ dominates: halving h divides the error by about $2^2 = 4$. (Section 16.4.)
- **Second quantisation**: Second quantisation labels a determinant only by which orbitals it uses. (Section 13.4.)
- **Sectional curvature**: is the curvature of the coordinate plane of x_a and x_b (its sectional curvature; "plane" means here the plane of the two directions x_a and x_b at one point). (Section 3.17.)
- **Sector** of a Hermitian matrix B with $B^2 = I$: the columns u with $Bu = u$ (the sector $B = +1$), or those with $Bu = -u$ (the sector $B = -1$). (Sections 4.9, 4.11, 9.26, 9.29, 15.2, 15.24, 22.2 and 22.11.)
- **Seed**: It starts from a number called the seed, and the same seed always gives the same sequence. (Sections 0.22, 0.24, 1.25, 1.49, 1.53, 4.12, 5.17, 7.9, 8.23, 9.17, 11.8, 13.14, 14.11, 16.12, 18.9, 20.11 and 21.15.)
- **Self-adjoint**: the Hamiltonian satisfies $\int \varphi^\dagger h\chi\, dy = \int (h\varphi)^\dagger \chi\, dy$; then the levels are real. (Section 14.17.)
- **Self-adjoint for the Krein form**: The mode matrix satisfies $Bh_k = h_k^\dagger B$ for every momentum (PROVED; `python-field-theory.json`, check `mode_hamiltonian_B_selfadjoint_dispersion`; a matrix with this property is called self-adjoint for the Krein form). (Section 8.19.)
- **Self-consistency**: the potential makes the orbitals, the orbitals make the density, the density makes the potential; a solution is self-consistent when the potential that comes out equals the potential that went in. (Sections 13.1, 13.6, 13.35 and 15.8.)
- **Self-consistent**: the orbitals reproduce the mean field that made them. A computer finds such a state by repeating "densities, then mean field, then orbitals, then densities" until nothing changes. (Sections 0.2, 7.23, 17.2 and 19.2.)
- **Self-consistent state**: (v) So the densities that the partner problem computes from its own orbitals and occupations are $(n, -S)$, the densities we started from in (ii): B is a fixed point of the partner's loop, a self-consistent state. (Section 19.9.)
- **Self-consistently**: Kohn-Sham state: an approximate state of N identical fermions built from one-particle wave functions (orbitals) that each solve a one-particle equation in a common effective potential; the potential depends on the densities of the occupied orbitals, so the equations are solved self-consistently (repeat until nothing changes). (Section 19.20.)
- **Self-coupling** λ: the strength with which a field acts on itself (Chapter 7). (Sections 0.2 and 12.29.)

- **Series**: A series is an endless sum $t_0 + t_1 + t_2 + \cdots$; its partial sum $S_N = t_0 + \cdots + t_N$ adds the terms up to number N, and the series converges to a number S when the error $|S_N - S|$ becomes as small as we like for large N. (Sections 1.11 and 1.16.)

- **Set**: an unordered collection of different things, written `{"x1", "x2"}` in Python. Python keeps the names of a set in an order decided by their *hash values*: numbers computed from the names with a secret key that Python chooses anew each time it starts, unless the *environment variable* PYTHONHASHSEED fixes the key. (Sections 0.22, 0.24, 1.41, 2.28, 4.8, 4.13, 4.15, 5.10, 6.14, 8.23, 9.30, 10.43, 11.9, 14.11, 15.14, 17.11, 20.16, 21.15 and 22.23.)

- **Set comprehension**: A set comprehension (braces) keeps each bilinear once, `sorted` puts them in order: 42 components, 15 distinct three-gamma bilinears. (Sections 12.20 and 19.16.)

- **Set-up cell**: The first code cell, the set-up cell, repeats the instructions as comment lines and prepares the helpers that every notebook uses. (Sections 0.7 and 1.1.)

- **sha256**: The fingerprint used here is sha256: a fixed recipe that turns the bytes of a file, however many there are, into a number of 256 binary digits, written as 64 hexadecimal characters. (Sections 0.18 and 15.13.)

- **sha256 fingerprint**: a 64-digit hexadecimal number computed from a file by the method SHA-256; two different files have different fingerprints for all practical purposes, so equal fingerprints mean equal files, byte for byte. (Sections 1.50, 1.53, 17.2 and 17.20.)

- **Shape**: To make "changed a little" precise, take a fixed shape $\xi(t)$ with $\xi(0) = \xi(T) = 0$ (so that the changed path keeps the end values) and a small number ϵ, and form the varied path $q + \epsilon\xi$. (Section 7.2.)

- **Shell**: all lattice vectors with the same $n^2 = n_1^2 + n_2^2 + n_3^2$; $r_3(n^2)$ is their number (for example $r_3(1) = 6$). (Sections 14.21, 14.24, 15.2, 15.13, 17.2 and 19.3.)

- **Shell structure**: Thomas-Fermi uses no orbitals, so it cannot show the four bumps of the Kohn-Sham density (its shell structure). (Section 13.32.)

- **Shift left** by k places (two less-than signs): k zero bits are appended, which multiplies by 2^k (every bit moves to a place worth 2^k times more). (Section 1.49.)

- **Shift right** by k places (Python writes it with the operator made of two greater-than signs): the last k bits are dropped. This is whole-number division by 2^k: writing $x = 2^k q + r$ with $0 \le r < 2^k$, the last k bits of x are the bits of r, and the bits before them are those of q. (Section 1.49.)

- **Shoelace formula**: The signed area of a polygon with the corners $(x_1, y_1), \ldots, (x_K, y_K)$, listed in order, is given by the shoelace formula. (Section 1.21.)

- **Shooting**: solve from one end with the conditions there and a guessed eigenvalue (like aiming a cannon), measure the *mismatch* at the other end (where the shot lands), and change the guess until the mismatch is zero. (Sections 2.16, 14.14, 14.17, 15.8, 19.15 and 19.16.)

- **Shooting function** F: the mismatch as a function of the guess; the eigenvalues are its zeros. (Sections 2.12, 2.16 and 14.14.)

- **Shooting method**: The Rust program of the Revision record solves them by the shooting method: it guesses an energy, integrates the equations from one end to the other with the classical fourth-order Runge-Kutta method (RK4) in 900 steps, measures by how much the condition at the far end fails, and corrects the energy with Newton's method. (Section 2.1.)
- **Sign convention**: Energy-momentum tensor $T^{\mu}{}_{\nu}$: the energy density $\rho = -T^{x_4}{}_{x_4}$, the pressures $p_\mu = T^{\mu}{}_{\mu}$ (no sum), and the mixed components; sign convention $\sigma_T = +1$ of the Revision record. (Section 12.29.)
- **Sign of a permutation**: $\operatorname{sign}(\sigma) = (-1)^N$, where N is the number of inversions: $+1$ for an even permutation (even N), -1 for an odd one. (Section 11.8.)
- **Sign** of a permutation: $+1$ if it has an even number of inversions (an even permutation), -1 if odd. (Sections 1.19, 1.24, 1.47 and 11.2.)
- **Sign of a reordering**: putting the factors of a product into increasing order by exchanging neighbours; every exchange of two generators gives a factor -1. The total factor is $+1$ for an even and -1 for an odd number of exchanges. (Section 7.16.)
- **Sign pattern** of g: the pair $(\operatorname{sign}\det A, \operatorname{sign}\det D)$. (Sections 5.23 and 5.26.)
- **Signature**: the numbers of positive and of negative entries of the metric. Here it is (4,4): four space-like and four time-like directions. (Sections 0.5, 0.14, 0.16, 1.27, 1.32, 1.36, 1.40, 2.19, 3.4, 3.12, 4.4, 5.4, 5.9, 6.13, 7.26, 10.2, 10.9, 12.2 and 21.14.)
- **Signed area**: The signed area of a polygon with the corners $(x_1, y_1), \ldots, (x_K, y_K)$, listed in order, is given by the shoelace formula. (Section 1.21.)
- **Signed byte**: A signed byte uses the same 8 bits for the numbers -128 to 127: a negative number $-m$ is stored as $256 - m$ (the two's complement). (Sections 1.49 and 1.53.)
- **Signed permutation matrix**: a square matrix with exactly one nonzero entry in every row and in every column, that entry being $+1$ or -1. Acting on a column it puts the components into a new order and changes some of their signs. Every gamma matrix of the author, and the matrix C, is one. (Sections 1.19, 1.24, 4.2, 4.5, 4.7, 4.17, 4.19, 5.2, 5.9, 7.12, 17.20, 17.21, 18.2, 18.8, 19.15, 19.16 and 21.7.)
- **The signs of the directions**: the table η whose diagonal entries are $+1$, $+1$, $+1$, $+1$, -1, -1, -1, -1 in the order $x_0, \ldots, x_7$: the directions x_0, x_1, x_2, x_3 are space-like and x_4, x_5, x_6, x_7 time-like; the signature is (4,4). (Section 0.5.)
- **Similar**: Matrices related in this way ($N = SMS^{-1}$) are called similar, and similar matrices have the same eigenvalues. (Sections 10.6, 20.12 and 21.26.)
- **Similar matrices**: $N = PMP^{-1}$; they have the same spectrum. (Section 18.26.)
- **Simpson's rule**: an integration rule with the weights $1, 4, 2, 4, \ldots, 4, 1$ times (step/3). (Sections 13.15, 14.17, 14.18, 15.8, 17.2, 17.20, 17.21 and 22.12.)
- **Singular value decomposition**: The singular value decomposition (`np.linalg.svd`) writes a matrix A as $U\Sigma V^\dagger$ with a list of numbers Σ (the singular values, sorted from large to small) and two matrices U and V whose columns have length 1 and are perpendicular; a column of V whose singular value is zero solves $Au = 0$. numpy returns $V^\dagger$ as `vh`, so the last row of `vh`, conjugated, is the last column of V. (Sections 8.23, 9.30, 18.27 and 21.30.)

- **Singular values**: for a matrix M the square roots of the eigenvalues of $M^T M$. The number of zero singular values of M is the number of independent solutions of $Mx = 0$. (Sections 5.27 and 10.37.)

- **Singularity**: "The big bang." In cosmology the big bang is the hot, dense early phase from which the observed expansion started; in the classical solutions of Einstein's equations that describe it, the scale factor of space goes to zero at a finite time in the past, and the equations break down there (a singularity). (Section 20.3.)

- **Size**: The size of a matrix (the largest factor by which it stretches the length of a column) is therefore exactly k_5 for h_A: $\|h_A u\|^2 = u^\dagger h_A^\dagger h_A u = k_5^2\, u^\dagger u$. (Section 10.39.)

- **Size of a matrix (spectral norm)**: the largest factor by which the matrix stretches the length of a column; in code `np.linalg.norm(M, 2)`. (Section 10.42.)

- **Slater determinant**: the antisymmetric state of N fermions built from N orthonormal orbitals, $\Phi = \det[\phi_a(x_i)]/\sqrt{N!}$. (Sections 13.3 and 13.13.)

- **Slice**: the function of ONE variable obtained by holding the others fixed; the partial derivative is the slope of its graph. (Sections 2.18, 2.22, 3.8, 4.8, 5.10, 14.3, 14.10, 14.24, 15.1, 15.13, 16.1, 17.2, 17.10, 17.20, 18.27, 19.2, 19.16, 19.20, 22.1, 22.11 and 22.23.)

- **Slope** c: $d\varepsilon/dk$ of the brane band at $k = 0$. (Sections 2.6, 3.8, 11.2, 12.2, 12.19, 14.24, 16.18 and 17.2.)

- **Smallest singular value** of a square matrix M: the smallest length of Mu over all columns u of length 1. It is 0 exactly when some nonzero u has $Mu = 0$. numpy computes it with `np.linalg.svd`. (Sections 4.9 and 4.11.)

- **So(4,4)**: the real 8×8 matrices X with $X^T\eta + \eta X = 0$ (the *Lie algebra* of O(4,4)). (Sections 5.12, 5.23 and 5.26.)

- **Sobolev norm of order** s: a way to measure the size of data that also counts how fast they wiggle; for one wave of momentum K and amplitude ϵ it is $\epsilon\,(1 + K^2)^{s/2}$. Larger s punishes wiggly data more. (Sections 8.18 and 8.22.)

- **Solution space**: the set of all solutions X of a list of linear equations without constant terms; its dimension is the number of free parameters. (Sections 4.19, 5.16 and 21.14.)

- **Solution**, **boundary condition**: a solution of a field's equations is a field that satisfies them at every point; a boundary condition is a further condition that a solution must satisfy at the edge of the region in which it is defined (Chapters 2 and 14). (Sections 0.2, 18.1 and 20.2.)

- **Solver**: a program that computes the solution of equations. (Section 16.11.)

- **Source**: the energy-momentum tensor of the fields as it enters the equations of gravity (Chapter 12). (Sections 0.2, 12.2, 12.15, 12.19, 17.2, 17.10 and 17.20.)

- **Source conditions** (from the a_4 equations): C1 no component of the source depends on x_0 and $q_{40} = q_{04} = 0$; C2 $p_3 + p_t = 2p_0$; C3 for the linear member $a_4 = AHx_4 + a_0$: $p_3 = p_t = p_0$ and a constant ρ. (Sections 17.1, 17.2 and 17.20.)

- **Source strength**: In Einstein gravity, with the initial rate $a_4'(0) = H$ (the rate of the history on which the states were computed) and the source strength $\sigma_0 = \kappa\bar{\rho}(0)/H^2$, the constraint at $a_4 = 0$ fixes Λ: $3H^2 + 21H^2 + \Lambda = -\sigma_0 H^2 \quad \Longrightarrow \quad \Lambda = -24H^2 - \sigma_0 H^2$. (Section 17.17.)

- **Space block**: A is the space block and D the time block (the primes distinguish the other two blocks from the matrices B and C). (Sections 5.23 and 5.26.)

- **Space-like and time-like**: two kinds of direction. Along a space-like direction a step is measured like a distance; along a time-like direction it is measured like a duration. In the metric a space-like direction has a positive entry and a time-like direction a negative entry. (Section 0.16.)

- **Space-like** and **time-like**: a direction along which a step is measured like a distance, or like a duration; x_0, x_1, x_2 and x_3 are space-like, x_4 to x_7 time-like (Section 0.14). (Sections 0.2, 0.14, 1.5, 1.13, 1.27, 1.32, 3.4, 3.12, 4.3, 4.4, 5.1, 6.13 and 6.27.)

- **Space-like, time-like direction**: $\eta_{nn} = +1$ (x_1, x_2, x_3 and the hidden direction x_0), respectively $\eta_{nn} = -1$ (the time x_4 and the three EXTRA TIMES x_5, x_6, x_7, which deflate exponentially in the author's metric). (Section 18.8.)

- **Space-space**: The factor $\frac{1}{2}(\eta_{aa} + \eta_{bb})$ is $+1$ for a pair of two space-like directions (space-space), -1 for a pair of two time-like directions (time-time) and 0 for a boost pair. (Sections 6.23 and 6.27.)

- **Span**: all combinations of a list of matrices. (Sections 4.15, 5.11, 5.16 and 5.26.)

- **Spectral theorem**: Symmetry matters: a real symmetric matrix has only real eigenvalues and an orthonormal set of eigenvectors (the spectral theorem, quoted from linear algebra), as the levels of a self-adjoint problem must; and the counting method of Section 16.15 works only for symmetric matrices. (Section 16.14.)

- **Spectrum**: the list of all eigenvalues of a matrix. (Sections 1.34, 1.40 and 5.9.)

- **Spin**: The spin-statistics rule of ordinary four-dimensional quantum field theory says that particles of half-integer spin (the spin is the intrinsic angular momentum of a particle, measured in units of Planck's constant divided by 2π; the electron has spin $\frac{1}{2}$), which are the fermions of Chapter 7, must be described by anticommuting fields; described by commuting fields, as dirac16complex00 is, the energy of a spinor field has no lower bound, and this is exactly what the plane waves above show. (Section 9.21.)

- **Spin connection** $\omega_\mu{}^a{}_b$: the numbers that say how the vielbein turns when one moves along x^μ. (Sections 5.28, 6.1, 6.4, 6.13, 6.21, 8.35, 9.16, 12.29, 14.10, 18.3 and 21.22.)

- **Spin density**: The last term comes from the change of the spin connection; the record calls the bilinear in it the totally antisymmetric spin density (it changes sign when two of its three indices are exchanged). (Section 9.6.)

- **Spin generators**: With the spin generators $S^{ab} = \frac{1}{4}(\gamma^{(a)}\gamma^{(b)} - \gamma^{(b)}\gamma^{(a)})$, which equal $\frac{1}{2}\gamma^{(a)}\gamma^{(b)}$ for $a \neq b$ (the two products differ only in sign) and 0 for $a = b$ (Chapter 5), the spinor connection is $\Omega_\mu = \frac{1}{2}\,\omega_{\mu ab} S^{ab} = \sum_{a<b} \omega_{\mu ab} S^{ab}$. (Section 7.18.)

- **Spin transformation**: A spin transformation is a 16×16 matrix R that acts on the components, $\Psi \to R\Psi$, and at the same time turns the eight directions into each other. (Sections 5.18 and 7.19.)

- **Spin(4,3)**: the spin group of the seven slice directions $x_0, x_1, x_2, x_3, x_5, x_6, x_7$ (every direction except the time x_4: four space-like, three time-like). In Chapter 10 it is generated by the 21 generators S^{ab} whose pair of directions does not contain x_4; exactly these 21 generators are Krein-unitary. (Sections 10.34 and 10.37.)

- **Spin(4,4)**: the subgroup of Pin(4,4) (see that entry) made of the products $\gamma(u_1) \cdots \gamma(u_k)$ of an EVEN number k of unit vectors. Under it the 16 components split into two inequivalent halves of 8 (PROVED, Section 5.13), while under Pin(4,4) they are irreducible. (Sections 5.12, 5.13, 5.21 and 5.26.)

- **Spin-statistics**: This is the classical form of the spin-statistics problem of a commuting field with a first-order Lagrangian. (Section 9.21.)

- **Spin-statistics rule**: The spin-statistics rule of ordinary four-dimensional quantum field theory says that particles of half-integer spin (the spin is the intrinsic angular momentum of a particle, measured in units of Planck's constant divided by 2π; the electron has spin $\frac{1}{2}$), which are the fermions of Chapter 7, must be described by anticommuting fields; described by commuting fields, as dirac16complex00 is, the energy of a spinor field has no lower bound, and this is exactly what the plane waves above show. (Section 9.21.)

- **Spin-statistics theorem**: In ordinary 3+1 dimensional physics this choice is not free: the spin-statistics theorem of quantum field theory shows that, if the energy is to be bounded below and measurements at two points that no signal can connect are not to disturb each other, fields of half-integer spin (such as the electron's) must be quantised with anticommutators and fields of whole-number spin with commutators. (Section 10.12.)

- **Spinor** Φ: a column of 16 complex numbers (here: functions of x_4). dirac16complex00 is a field of such spinors whose components are ordinary (commuting) complex numbers. (Sections 7.1, 12.29 and 20.15.)

- **Spinor boost** $R(x)$: a 16 x 16 matrix that turns spinors when the frame is boosted; it covers Λ when $R^{-1}\gamma^a R = \sum_b \Lambda^a{}_b \gamma^b$ for every a. (Sections 6.21 and 6.27.)

- **Spinor connection** $\Omega_\mu = \frac{1}{2}\sum_{a,b} \omega_{\mu ab} S^{ab}$: a 16 x 16 matrix for every μ. (Sections 6.7, 6.13, 6.21, 6.27, 7.18, 7.26, 8.2 and 8.14.)

- **Spinor curvature**: Spinor curvature $F_{\mu\nu} = \partial_\mu \Omega_\nu - \partial_\nu \Omega_\mu + [\Omega_\mu, \Omega_\nu]$, where $[A, B] = AB - BA$ (the *commutator*); the record proves $F_{\mu\nu} = \frac{1}{4}\sum R_{\rho\sigma\mu\nu}\gamma^\rho\gamma^\sigma$. (Section 8.14.)

- **Spinor field** Ψ: at every point of spacetime a column of 16 numbers $\Psi_1, \ldots, \Psi_{16}$ (its *components*). (Sections 5.32, 21.2 and 21.14.)

- **Spinor norm**: Spinor norm $N(g) = \eta(u_1, u_1) \cdots \eta(u_k, u_k)$ of such a product. (Section 5.21.)

- **Spread**: For a state with a nonzero source the record computes the spread of the energy density. (Section 17.4.)

- **Square**: It is square if $m = n$. (Sections 1.18 and 1.24.)

- **Square root of a quadratic form**: an expression L, linear in the variables, whose square is the quadratic form: $L^2 = p^2 + q^2$ (times the identity matrix when L is a matrix). (Section 4.11.)

- **Squared length**: The squared length of a vector is $Q(v) = \eta_{ab} v^a v^b = v^a v_a = (v^0)^2 + (v^1)^2 + (v^2)^2 + (v^3)^2 - (v^4)^2 - (v^5)^2 - (v^6)^2 - (v^7)^2$. (Sections 1.27, 1.32 and 3.4.)

- **Squared length of the velocity**: $g(u,u) = \sum_a g_{aa}(u^a)^2$ (the metric is diagonal). For a body with mass it equals -1 when τ is its proper time. (Section 3.34.)

- **Stack**: The stack is a list of partial combinations still to be extended, each a tuple of the chosen entry numbers with the two masks; it starts with the empty combination. (Section 11.19.)

- **Stacked bar chart**: A stacked bar chart: for each field the bars of the three types are drawn on top of each other (`bottom=bottoms` starts each new bar where the previous one ended), the counts are written into the bars and the totals above them (`"\n"` in a label is a line break). (Section 7.27.)

- **Staggered**: Such a grid, on which the two components live on alternating points, is called staggered. (Section 16.14.)

- **Staggered grid**: a is stored at the half nodes, b at the nodes. (Sections 16.3 and 16.18.)

- **Standard deviation**: A measured fraction p from N tries is expected to scatter around the true value by about one standard deviation $\sqrt{p(1-p)/N}$ (ASSUMED from probability), here $\sqrt{0.25/200000} \approx 0.0011$ for the first; both measurements lie within 4 standard deviations of $1/2$ and of $1/2 + 1/\pi$ (Figure 01e.2). (Sections 1.27, 1.50, 1.53 and 11.4.)

- **State**: In quantum theory a state is a column of numbers, and an operator is a rule that turns a state into another state; here every operator is a (big) matrix acting on columns. (Sections 1.49, 1.53, 5.34, 5.37, 10.18 and 22.2.)

- **State id**: a name such as `N136_lamp2_a20`: $N = 136$ particles, coupling $+\lambda_2$ (`lam0`: $\lambda = 0$; `lamp1`, `lamm1`: $\pm\lambda_1$; `lamp2`, `lamm2`: $\pm\lambda_2$), slice $a_{4,0} = 2.0$ (the digits are ten times $a_{4,0}$). A thermal state ends with `_T10`, `_T20` or `_T50`: the temperature $T = 0.01$, 0.02 or 0.05 (in units of the mass m). (Sections 16.11 and 16.23.)

- **Stationary**: A path is stationary if $S_1 = 0$ for every shape ξ; the principle says that the true motion is stationary. (Sections 5.6 and 7.2.)

- **Stationary path**: a path whose first variation is zero for every variation. (Section 7.8.)

- **Stationary state**: So $\Psi_0 = \sin z\, e^{iwx_4}\chi_0$ has a single frequency: a stationary state. (Section 21.18.)

- **Statistics**: whether the components of a field commute (the field dirac16complex00) or anticommute (the field dirac16complex, whose components are Grassmann numbers). (Sections 7.26 and 18.2.)

- **Status labels**: PROVED (exact, by a derivation or by exact computation, with the record file and check), COMPUTED (numerical, with its accuracy), ASSUMED, HYPOTHESIS, OPEN. (Sections 11.2, 12.2, 12.19, 17.10, 17.20, 21.2 and 21.35.)

- **Status words**: PROVED (exact, for all values), CHECKED (exact or to rounding, for the values shown), REPRODUCED (agrees with a Revision record), NOT ESTABLISHED (not shown by these equations). (Section 10.48.)
- **Step**: one command of the gate, with a name such as `algebra-wolfram`. (Sections 15.8 and 23.2.)
- **Step rule**: A step rule says how to compute y_{n+1} from y_n. (Section 2.4.)
- **Step size**: It chooses a step size h, the times $t_n = nh$, and computes numbers y_n that approximate $y(t_n)$, one step after the other. (Section 2.4.)
- **Storage rule**: We call this the storage rule: a number between 2^k and 2^{k+1} is stored as $m \cdot 2^{k-52}$ with a 53-digit m. (Section 0.22.)
- **Strictly increasing**: So $\Phi(\varepsilon)$ is strictly increasing: each target value $l\pi$ or $\pi/2 + l\pi$ is reached for exactly one energy. (Section 2.24.)
- **String**: A variable is a name for a value; this line gives the name `NOTEBOOK_ID` to the text `"00a"` (a text in quotes is called a string). (Sections 0.13, 0.21, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 13.14, 14.11, 15.9, 16.12, 17.11, 18.9, 19.16, 20.11, 21.15 and 22.12.)
- **Sturm count** $c(x)$: the number of eigenvalues of T below a number x. (Sections 16.3, 16.15 and 16.18.)
- **Subgroup**: A subgroup is a subset that is itself a group. (Section 5.11.)
- **Subprocess**: a program that another program starts and whose printed output it reads. (Section 0.24.)
- **Subset**: A subset of a set S is a set whose elements all belong to S. (Sections 4.13, 4.15 and 16.9.)
- **Subspace**: A subspace is a set of columns that contains every sum and every multiple of its members (for example all columns whose components 9 to 16 are zero). (Section 5.11.)
- **Successive differences**: For each computed number x, for example E_{KS}, the study forms the successive differences $d = x(L + 0.5) - x(L)$ over the unbroken run $L = 3, 3.5, 4, \ldots$ of values at which the self-consistent iteration converged (the run ends just before the first L at which the iteration fails, and values at larger L after a failure are not used), and the ratio r of each difference to the one before it. (Section 15.2.)
- **Sudden**: Sudden: so fast that the orbital has no time to change at all. (Section 22.11.)
- **Sudden limit**: the limit of an infinitely fast change, in which the orbital has no time to change at all (Section 22.8). (Section 22.2.)
- **Sum**: Comparing the two: the sum of the eigenvalues is the trace. (Section 1.34.)
- **Sum convention**: the summation convention (see that entry). Chapter 12 writes every sum with $\sum$ or in words, except in the definitions of Section 12.7 and in the formulas quoted from the record, where an index that appears once up and once down is summed over its eight values; "no sum" marks the exceptions. (Sections 5.28 and 12.2.)

- **Summation convention**: an index that appears twice in one product is summed over all its values without writing $\sum$; so $A_{ij}B_{jk}$ means $\sum_j A_{ij}B_{jk}$. (Sections 1.24, 1.26 and 1.32.)
- **Summed label**: in Chapter 11 a label that appears once up and once down inside one term is summed over its eight values, unless "no sum" is said; the free labels h and j are never summed. (Section 11.2.)
- **Sylvester's law of inertia**: the matrices S and $P^T S P$ (P invertible) have the same signature. (Sections 1.40, 3.4 and 16.15.)
- **Symbol** (sympy): a letter that stands for any number, so that sympy can compute exactly with formulas instead of with decimal numbers. (Sections 0.16, 4.11, 4.12, 11.5 and 17.11.)
- **Symmetric (Belinfante) energy-momentum tensor**: the symmetric part $\frac{1}{2}(T^{\nu\rho}_{\text{var}} + T^{\rho\nu}_{\text{var}})$ of the variation tensor T_{var} (the energy-momentum tensor obtained by changing the metric in the action), named after the physicist F. J. Belinfante, who showed how to make the energy-momentum tensor of a spinor field symmetric. The record proves its formula for every configuration, solution or not (PROVED; check `T_symmetric_part_Belinfante_C`). (Section 9.6.)
- **Symmetric** array: $S_{ab} = S_{ba}$; antisymmetric array: $A_{ab} = -A_{ba}$ (its diagonal is 0). (Sections 1.28, 1.32, 3.3, 4.2, 4.7, 5.2, 5.9, 6.4, 6.7, 6.23, 7.16, 16.14, 16.18, 18.8, 21.2 and 21.14.)
- **Symmetric difference** $A \triangle B$: the directions that lie in exactly one of the sets A and B. (Sections 4.13 and 4.15.)
- **Symmetric logarithmic**: The vertical axis is symmetric logarithmic (`"symlog"`): linear between -0.001 and 0.001 and logarithmic outside, so that large and small values of both signs fit. (Sections 1.41, 5.27, 8.15 and 15.14.)
- **Symmetric logarithmic axis**: logarithmic for large positive and negative values and linear near zero, so that both signs fit on one axis. (Sections 15.19 and 17.2.)
- **Symmetric matrix**: $S^T = S$. (Section 1.40.)
- **Symmetric operator (formally Hermitian)**: an operator h with $\int \cos z\, u^\dagger (hv)\, dx_0 = \int \cos z\, (hu)^\dagger v\, dx_0$ for all wave functions u, v. A *boundary term* is a part of the integrand that is a derivative, $\partial_0(\ldots)$; its integral is the difference of the bracket at the two ends, so the operator is symmetric when that difference vanishes. (Section 10.31.)
- **Symmetric part**: Write M as the sum of its symmetric part $\frac{1}{2}(M+M^T)$ and its antisymmetric part $\frac{1}{2}(M - M^T)$. (Sections 7.12 and 9.4.)
- **The symmetries C and CP**: C exchanges every particle with its antiparticle, P reflects space as in a mirror, and CP does both (Chapter 21). (Section 0.2.)
- **Symmetrised**: It is symmetrised: the derivative acts once on Ψ and once on $\bar{\Psi}$, with opposite signs (Section 7.20 shows why). (Section 7.19.)
- **Symmetry**: A symmetry maps the solutions of one theory to solutions of the SAME theory. (Sections 18.13 and 21.22.)

- **Symmetry of the dynamics**: A transformation is a symmetry of the dynamics if it turns every possible history of the system into another possible history that runs in the same direction of time. (Section 21.5.)
- **Symmetry of the metric**: Such a change of coordinates that leaves the metric unchanged is a symmetry of the metric (an isometry). (Section 20.22.)
- **Symmetry pattern**: (iii) The symmetry pattern $(\gamma^a)^T = \eta_{aa}\gamma^a$: symmetric for the space-like x_0, x_1, x_2, x_3, antisymmetric for the time-like $x_4, \ldots, x_7$. (Sections 4.5 and 21.7.)
- **sympy**: the Python package for exact algebra with symbols; numpy: arrays of numbers; matplotlib: plots. (Sections 3.12, 4.11, 10.9, 14.10, 18.20, 19.15 and 20.11.)
- **sympy symbol**: a letter that stands for any number; `ad1` and `ad2` stand for $a_4' = da_4/dx_4$ and $a_4'' = d^2a_4/dx_4^2$. (Section 12.15.)
- **Sympy, Rational**: sympy computes with exact fractions such as $-1/32$, so its checks are exact, not rounded. (Section 13.19.)
- **System**: Several unknown functions that change together, say a position $x(t)$ and a velocity $v(t)$, form a system. (Section 2.2.)
- **System of ODEs**: several unknown functions changing together, for example a position x and a velocity v; y is then a list (a *vector*) of numbers. (Section 2.10.)

T

- **T1** (both fields, every gravitational field): a fixed 16×16 matrix Γ, the chirality, turns every solution with (m, λ) into a solution with $(-m, -\lambda)$; the Lagrangian (the function from which the equations follow) changes as $L_{m,\lambda}[\Gamma\Psi] = -L_{-m,-\lambda}[\Psi]$, the energy-momentum tensor T (the table of the energy and momentum that the field carries) changes into $-T$, and the charge current J into $-J$. (Section 0.2; the opening of Chapter 18.)
- **T1 pair**: A T1 pair has opposite energy-momentum tensors; a T2 pair has equal ones. (Section 20.2.)
- **T1 partner**: T1 partner: $\Gamma\Psi$ with $(-m, -\lambda)$ at the same point. (Sections 20.2 and 20.10.)
- **T2** (both fields): Γ combined with the reflection of one space-like direction (an element of the group Pin(4,4)) turns (m, λ) into $(-m, \lambda)$, with the same self-coupling; in the author's universe the reflection is the Z2 mirror across the edge $z = \pi/2$ of the hidden direction (a choice of boundary condition, which is ASSUMED, not derived), and the energy-momentum tensor of the image is equal to that of the original (reflected across the mirror), not opposite to it. (Section 0.2; the opening of Chapter 18.)
- **T2 mirror copy**: T2 mirror copy: $\gamma^{(x_0)}\Psi$ placed at the mirror point $\pi - z$, with $(-m, \lambda)$. (Sections 20.2 and 20.10.)
- **T2 pair**: A T1 pair has opposite energy-momentum tensors; a T2 pair has equal ones. (Section 20.2.)

- **T3** (dirac16complex, the Kohn-Sham level: within the Kohn-Sham approximation, which is ASSUMED (Section 0.18), with the ASSUMED Z2 mirror and the correspondingly transformed boundary conditions): every self-consistent Kohn-Sham universe with (M, λ) has a partner with $(-M, \lambda)$, and the two have equal energies and equal energy-momentum tensors. (Section 0.2; the opening of Chapter 19.)

- **The table of the plane curvatures**: The table of the plane curvatures (PROVED; Notebook 03b computes all 28 and finds these seven formulas, In [16]), with $\alpha = a_4'$ and $\alpha_t = -a_4' + b_4'/(2b_4)$ (the formula is displayed in that section). (Section 3.24.)

- **Tangent**: the straight line that touches a curve at one point; its slope is the derivative there. (Sections 15.19 and 22.22.)

- **Taylor's theorem** (ASSUMED: a standard theorem of one-variable calculus, quoted without proof). If y has $n+1$ continuous derivatives, then for every step h. (Section 2.3.)

- **Temperature** T: measured as an energy (Boltzmann's constant is set to 1). (Sections 13.30, 15.29 and 21.2.)

- **Temporary folder**: `tempfile.mkdtemp` creates a new, empty temporary folder in the place the operating system keeps for temporary files (outside the repository; its name starts with `textbook_16a_` and ends with random characters) and returns its name, which `Path` turns into a path: `RUN_FOLDER`. (Section 16.12.)

- **Tension**: Tension: the energy per unit area of a surface, which acts like a negative pressure along it. (Section 22.2.)

- **Tensor** $T^{\nu}{}_{\mu}$: a table of 8×8 numbers at every point, one for each pair of directions (ν, μ). The upper index ν is the row, the lower index μ the column. Lowering the upper index with the metric gives $T_{\nu\mu} = g_{\nu\nu} T^{\nu}{}_{\mu}$ (the metric here is diagonal). (Sections 1.43, 3.14, 9.16 and 11.5.)

- **A terminal** is the window in which you type commands: on Windows the app Windows PowerShell (or Terminal), on macOS the app Terminal (it runs the shell zsh), on Linux any terminal (it runs the shell bash). A command is typed exactly as printed and sent with the Enter key. The commands are printed in framed blocks, one command per line; where they differ between the systems, the block is labelled with the system. (Sections 0.8 and 23.2.)

- **Test equation**: To understand the accuracy of a method we apply it to the simplest equation of all, the test equation. (Section 2.5.)

- **Test field** and **back-reaction**: a field that is computed in a gravitational field given in advance, which the field itself does not change, is a test field. The change of the gravitational field caused by the energy and momentum of the field is its back-reaction. All Kohn-Sham results of the record are test-field results. (Sections 14.3, 15.16, 17.1, 17.2, 17.10, 17.20, 17.22 and 22.2.)

- **Test particle**: A test particle is a body so small that it does not change the metric (ASSUMED: the particles of this section are test particles). (Sections 3.31 and 3.34.)

- **Text buffer**: `io.StringIO()` is a text buffer, a piece of memory that collects printed text. (Section 10.10.)

- **Thawing**: Thawing: $w_a < 0$; freezing: $w_a > 0$. (Section 22.22.)
- **The theorem of the linear history** (PROVED; record entry `theoremLinear`, see the table). Its hypotheses are four: (1) Φ is a homogeneous condensate of the commuting field that solves its field equation; (2) it meets the off-diagonal conditions above; (3) a_4 is twice continuously differentiable (a_4' and a_4'' exist and are continuous functions of x_4); (4) the three Lovelock couplings $\alpha_1, \alpha_2, \alpha_3$ of Chapter 12 are not all zero. Its conclusion: if the author's metric with this a_4 solves the field equations of gravity with the condensate as the source, then $a_4 = AHx_4 + a_0$ with real constants A and a_0. The argument uses, besides (3) and (4), the equality $p_3 = p_t$ of a condensate and one equation of the record (the formula is displayed in that section). (Section 9.26.)
- **Thermal ensemble**: Wick's theorem also holds for a thermal ensemble of non-interacting fermions, the statistical mixture of states that describes them at a temperature $T > 0$ (Section 13.26), with occupations f_a between 0 and 1 in place of 0 and 1. (Section 13.4.)
- **Thermal equilibrium**: a state in which every process runs forwards as often as backwards, so that on average nothing changes (Chapter 21). (Sections 0.2, 21.2, 21.5 and 21.35.)
- **Thermal hole**: If the sea were treated thermally, each of its orbitals would be empty with the probability $1 - f\big((-\varepsilon_{band} - \mu)/T\big) = f\big((\varepsilon_{band} + \mu)/T\big) = 1/(1 + e^{(\mu+\varepsilon_{band})/T})$; such an empty sea orbital is a thermal hole, which a full quantum treatment would count as an antiparticle of the same universe. (Sections 15.26, 16.20 and 16.23.)
- **Thermal particles**: Thermal holes $H_{th} = \sum_{\varepsilon_i<\mu} g_i\,(1 - f(x_i))$ (missing particles below μ) and thermal particles $P = \sum_{\varepsilon_i>\mu} g_i\, f(x_i)$ (particles above μ). (Sections 16.20 and 16.23.)
- **Thermal states**: Thermal states: the same three N, the three couplings $0, \pm\lambda_1$, the five slices and three temperatures $T = 0.01, 0.02, 0.05$, that is $3 \cdot 3 \cdot 5 \cdot 3 = 135$ states. (Section 15.2.)
- **Thomas-Fermi**: (c) Thomas-Fermi: the kinetic energy of each small piece of the line is that of a uniform gas with the local density. (Sections 13.15 and 13.32.)
- **Thomas-Fermi approximation**: the kinetic energy of each small piece of the line is taken from a uniform gas of the same density. (Section 13.35.)
- **The three Lovelock tensors**: The three Lovelock tensors (Sections 11.10 to 11.19). (Section 11.1.)
- **Three rules for moving gammas**: Three rules for moving gammas, each derived from the Clifford relation. (Section 21.7.)
- **Three-gamma bilinear**: the number $\bar{\Phi}\gamma^{(a)}\gamma^{(b)}\gamma^{(c)}\Phi$ for three different directions a, b, c. (Sections 9.26, 9.29, 12.26, 12.29 and 20.12.)
- **Threshold**: the smallest energy at which a process is allowed. (Sections 20.17 and 20.20.)
- **The time**: x_4 is the time. (Section 3.1.)
- **Time block**: A is the space block and D the time block (the primes distinguish the other two blocks from the matrices B and C). (Sections 5.23 and 5.26.)

- **The time direction contributes only through** b_4: The time direction contributes only through b_4, and nothing when b_4 is constant. (Section 6.9.)

- **Time reversal**: Reflecting a space-like direction (x_0, x_1, x_2 or x_3) is a parity-type operation (P); reflecting the time x_4 is time reversal (T); reflecting an extra time x_5, x_6, x_7 is a time-reversal-type operation of an extra time. (Section 21.24.)

- **Time-dependent Kohn-Sham problem**: the problem in which the orbitals evolve in the time x_4 under the Hamiltonian of each moment, with the potentials recomputed at every moment from the evolving densities. (Section 22.2.)

- **Time-derivative kernel** N: the matrix in the terms of a Lagrangian that contain ∂_4, written as $\frac{i}{2}(\Psi^\dagger N \partial_4 \Psi - \partial_4 \Psi^\dagger N \Psi)$. Canonical quantisation turns it into the anticommutator $\{\Psi_A, \Psi_C^\dagger\} = (N^{-1})_{AC}$ (at one point; derived in the second notebook of Chapter 10). (Section 10.48.)

- **Time-like**: space-like and time-like: a direction along which a step is measured like a distance, or like a duration; x_0, x_1, x_2 and x_3 are space-like, x_4 to x_7 time-like (Section 0.14). (Sections 0.2, 0.14, 1.5, 1.13, 1.27, 1.32, 3.4, 3.12, 4.3, 4.4, 5.1, 6.13 and 6.27.)

- **Time-time**: The factor $\frac{1}{2}(\eta_{aa} + \eta_{bb})$ is $+1$ for a pair of two space-like directions (space-space), -1 for a pair of two time-like directions (time-time) and 0 for a boost pair. (Sections 6.23 and 6.27.)

- **Tip**: Its end $z \to 0$ is the tip and its end $z = \pi/2$ the patch end. (Sections 2.24, 3.5, 3.12, 3.34, 5.6, 8.2, 8.31, 10.27, 10.31, 14.2, 15.1, 15.13, 16.1, 17.2, 17.10, 19.2, 19.15, 19.20 and 22.1.)

- **Tip angle** θ: the angle of the boundary condition at the tip. (Sections 19.2, 19.15 and 19.20.)

- **Tip condition**: the condition $(1 - Q(\theta))\chi(-L) = 0$ at the cutoff $y = -L$; $\theta = 0$ ($b(-L) = 0$) is the canonical choice. (Section 14.24.)

- **Tip cutoff**: The Kohn-Sham computations stop at $y = -L = -3$, the tip cutoff; the region $-3 \leq y \leq 0$ is the patch. (Sections 17.2, 17.10 and 17.20.)

- **Tip value**: Brane value and tip value: its values at $y = 0$ and $y = -3$. (Section 16.11.)

- **Tolerance**: the largest difference between two numbers that a check accepts as agreement. (Sections 0.22, 0.24, 15.13, 16.2, 16.7, 16.11, 16.23 and 18.27.)

- **Torus**: The three directions of 3-space are taken as a torus of coordinate size ℓ: a quantum leaving the box on one side comes back on the other, so the allowed coordinate momenta are $\mathbf{k} = \Delta k\,(n_1, n_2, n_3)$ with whole numbers n_1, n_2, n_3 and $\Delta k = 2\pi/\ell$. (Section 14.3.)

- **Total charge**: The total charge is the integral of the charge density over the seven directions other than the time. (Section 5.6.)

- **Total derivative**: $\frac{d}{dt}F(q(t), t)$ for some function F. (Sections 7.4, 7.8, 7.13, 7.26 and 7.32.)

- **Total derivative on jets**: Jet, total derivative on jets, coefficient ring: as in Notebook 07b: the field components and their derivatives at one point are symbols, and every coefficient is written with $E = e^{a_4}$, $s = \sin^{1/6} z$, $c = \cos z$, $A_1 = a_4'$, A_2, A_3 (higher derivatives of a_4), H, m, λ. (Section 7.32.)

- **Toy fluid (ILLUSTRATION)**: a source with assumed pressures $p_3 = w_3\rho$ and $p_t = w_t\rho$, used only to show what the conservation identity does; it is not a solution of the field equations of this book. (Section 9.24.)

- **Trace** $\operatorname{tr} M$: the sum of the diagonal entries of a matrix; it obeys $\operatorname{tr}(PR) = \operatorname{tr}(RP)$. (Sections 1.18, 1.26, 1.32, 1.40, 1.44, 2.19, 4.11, 4.15, 5.3, 5.9, 6.8, 6.13, 7.27, 8.14, 8.15, 8.17, 9.8, 9.16, 9.26, 9.29, 10.9, 12.32, 14.11, 14.30 and 18.20.)

- **Transcendental equation**: an equation such as $\tan(pL) = -p/M$ that has no solution formula and is solved numerically. (Sections 2.13 and 2.27.)

- **Transition**: a jump of a particle from an occupied level n to an empty level m; its amplitude is a number whose square is the probability of the jump. (Section 15.24.)

- **Transition state**: The midpoint value $\epsilon_{LUMO}(\frac{1}{2}) - \epsilon_{HOMO}(\frac{1}{2})$ (Slater's transition state) is a good estimate of the integral when the integrand is nearly a straight line in τ. (Section 13.27.)

- **Transpose** A^T: the matrix with rows and columns exchanged, $(A^T)_{ij} = A_{ji}$. (Sections 1.12, 1.18, 1.24, 4.7, 4.19, 5.9, 5.32, 7.12, 7.16, 10.9, 18.8, 21.2 and 21.14.)

- **Transposition**: A transposition exchanges two objects. (Section 11.8.)

- **Transverse**: The six transverse scale factors (those of $x_1, x_2, x_3, x_5, x_6, x_7$) multiply to. (Sections 3.7, 3.12 and 3.23.)

- **Trapezoid rule**: the integral over one step approximated by the step length times the average of the two end values. (Sections 2.24, 2.27, 15.3 and 22.12.)

- **Trapezoidal rule**: The flux is added up over time by the trapezoidal rule: on each step the area under the curve is the step times the mean of the two end values, `(fluxes[1:] + fluxes[:-1]) / 2` (`fluxes[1:]` drops the first value and `fluxes[:-1]` the last); `np.cumsum` forms the running sums, and a 0 is put in front for the time 0. (Section 10.32.)

- **Triangle inequality**: Rule: the triangle inequality $|p - q| \le |p| + |q|$ for any two numbers p, q. (Section 16.7.)

- **Triangular**: For a matrix with zeros below the diagonal (a triangular matrix) only the term of the identity survives in the Leibniz formula, because every other permutation σ has some row i with $\sigma(i) < i$ (the numbers $\sigma(i) - i$ add up to 0 and are not all 0, so one of them is negative), and the entry $A_{i\sigma(i)}$ below the diagonal is 0. (Section 1.21.)

- **Tridiagonal**: In the order of the unknowns each row couples only to its two neighbours in the list (a u to the w on either side, a w to the u on either side), so the matrix T of the problem $Tz = \varepsilon z$ is tridiagonal; at $k = 0$ its diagonal is zero. (Sections 16.14 and 16.18.)

- **Tridiagonal matrix**: The differential equations then become the eigenvalue problem of a symmetric tridiagonal matrix with $2G - 1$ rows (a matrix whose only nonzero entries are on the main diagonal and its two neighbours). (Section 16.3.)

- **True path**: The true path is $q(t) = \sin t / \sin 1$: it has $\ddot{q} = -\sin t / \sin 1 = -q$ (the second derivative of $\sin t$ is $-\sin t$), which is Newton's law $\ddot{q} = -\omega^2 q$ for the spring, and $q(0) = 0$, $q(1) = \sin 1 / \sin 1 = 1$. (Section 7.2.)

- **Truncated**: A truncated basis keeps only some of them; a computation converges when its result stops changing as more are kept. (Sections 22.2 and 22.11.)
- **Tunnelling**: a quantum transition through a region that the classical equations forbid (a barrier), with a small but nonzero probability; proposals in which a universe appears by tunnelling are of this kind (Section 20.25). (Sections 18.2 and 20.2.)
- **Tunnelling proposal**: a proposed boundary condition (A. Vilenkin) that picks one solution of the Wheeler-DeWitt equation, in which a universe appears by tunnelling. It is named in Chapter 22, not used, and is not formulated for this 8-dimensional theory. (Sections 22.2 and 22.4.)
- **Tuple**: The loop runs over a tuple (a list in round brackets that cannot be changed) of the two figure names and checks that each file exists. (Sections 0.13, 0.21, 0.25, 2.17, 7.9, 11.9, 12.25, 13.25, 14.11, 14.18, 16.12, 17.11, 19.16, 22.12 and 22.23.)
- **Turning point**: the instant at which the x_4 velocity u^4 passes through zero, so that x_4 stops growing. (Sections 3.31, 3.34, 17.2, 17.20 and 22.19.)
- **Twisted**: For g in Pin(4,4) put $\alpha(g) = g$ if g is even and $\alpha(g) = -g$ if it is odd (the twisted action). (Section 5.12.)
- **The two conservation identities** (Section 11.22), from zero divergence: $\frac{d}{dx_4} E^{x_4}{}_{x_4} = 3a_4'(E^{x_1}{}_{x_1} - E^{x_5}{}_{x_5})$ (I) and $E^{x_0}{}_{x_0} = \frac{1}{2}(E^{x_1}{}_{x_1} + E^{x_5}{}_{x_5})$ (II), and the constraint propagation $\partial E^{x_4}{}_{x_4}/\partial a_4' = 3a_4' F_k$ (record `python-a4-report.json`, check `bianchi_x4`; Notebook 11c, In [4] and In [5]). (Section 11.28.)
- **Two's complement**: A signed byte uses the same 8 bits for the numbers −128 to 127: a negative number $-m$ is stored as $256 - m$ (the two's complement). (Sections 1.49 and 1.53.)
- **Two-site model**: two places L and R, one orbital each, two labels (up, down); hopping t between the sites, repulsion U when both electrons sit on the same site. (Sections 13.13 and 13.24.)

U

- **U(1)**: The group of these phases is called U(1). (Section 21.7.)
- **U(1) charge** Q: the number that belongs to the phase symmetry $\Psi \to e^{i\alpha}\Psi$ of the field, $Q = \int \cos z \, \Psi^\dagger B \Psi \, d^7x$ (the integral over the seven directions other than x_4); it is the only number of the theory that could play the role of matter minus antimatter. It obeys an exact LOCAL conservation law, $\sum_\mu \partial_\mu(\cos z \, J^\mu) = 0$ for every solution of the field equation: the charge of a region changes only by what flows through its boundary (PROVED; lead check `u1_noether_matrix_identity`). The TOTAL charge of a universe is constant only if no charge flows through the brane $z = \pi/2$; this no-flux condition is ASSUMED, not derived (OPEN), and Section 21.18 shows an exact solution whose charge changes by exactly what flows out through the brane. (Sections 21.1, 21.7, 21.16, 21.17, 21.18 and 21.35.)
- **U(1) symmetry**: the multiplication of every component of a field by one and the same complex number of absolute value 1, which changes no equation; it is the reason why the charge obeys its local conservation law: at no point with $0 < z < \pi/2$ is charge created or destroyed, it can only flow from one place to another (Chapter 21). (Sections 0.2 and 20.22.)

- **Ulp**: A difference of one gap is called one unit in the last place, one ulp. (Section 0.22.)
- **Uncertainty**: What each can do is measure an uncertainty: a number U that, for reasons given in Sections 16.5 and 16.6, should be at least as large as the size $|x - X|$ of its error. (Sections 16.2 and 16.11.)
- **Undetermined function**: `sp.Function(name)(y)` makes an undetermined function of y: sympy knows nothing about it except that it depends on y, so an identity proved with it holds for every function. (Section 19.16.)
- **Uniform gas**: infinitely many fermions spread with the same density everywhere; it is studied in a large periodic box (a box whose opposite faces are glued together), whose volume is then made infinite. (Sections 13.15, 13.19 and 14.30.)
- **Unit circle**: First, $|e^{i\theta}|^2 = \cos^2\theta + \sin^2\theta = 1$ (the modulus of $a + ib$ with $a = \cos\theta$, $b = \sin\theta$; then $\cos^2 + \sin^2 = 1$): the number $e^{i\theta}$ lies on the circle of radius 1, the unit circle, at the angle θ. (Section 1.11.)
- **Unit columns**: Proof, line by line, with the unit columns $e_1, \dots, e_n$ (e_j has 1 in place j and 0 elsewhere) (the formula is displayed in that section). (Section 4.2.)
- **Unit in the last place**: A difference of one gap is called one unit in the last place, one ulp. (Section 0.22.)
- **Unit vector**: eight numbers $v = (v_{x0}, \dots, v_{x7})$ with metric product $\eta(v, v) = \sum_a \eta_{aa} v_a^2 = +1$ (*space-like*) or -1 (*time-like*); $\eta_{aa} = +1$ for $x0, x1, x2, x3$ and -1 for $x4, x5, x6, x7$. The basic unit vectors are $e_{x0}, \dots, e_{x7}$ (one entry 1, the others 0). (Sections 1.40, 5.12, 5.21 and 5.26.)
- **Unitary**: an evolution that keeps the total probability equal to 1. (Sections 10.34, 10.37, 14.5, 14.10 and 22.11.)
- **Unite values**: the supernova fits quoted by the record, $w = -0.764$ (constant w) and $(w_0, w_a) = (-0.861, -0.60)$. (Sections 22.16 and 22.22.)
- **Units**: the plots use $H = 1$ and $\kappa = 1$, so $\kappa\rho$, κp and Λ are in units of H^2; $\alpha_2 H^2$ and $\alpha_3 H^4$ are pure numbers. (Sections 12.19, 12.24, 12.29, 13.35, 17.2, 20.15 and 20.20.)
- **Universal**: F is universal: it does not depend on v. (Section 13.8.)
- **Universe** (of mass m): a solution of the field equation with the mass parameter m of the field's Lagrangian, in the author's gravitational field. The sign of m alone does not say which of two physically different kinds of universe one has: theorem T2 shows that the image of a solution with $(-m, \lambda)$ under the reflection of one space-like frame direction is a solution with (m, λ) with the same energy-momentum tensor. Section 20.12 builds one example, a solution of the commuting field dirac16complex00 that is by itself the complete source of the author's metric. (Sections 20.3 and 20.12.)
- **Unpack**: The first two lines unpack the four components of each. (Section 4.8.)
- **Unrestricted**: Hartree-Fock (HF): the best single determinant; restricted (both labels in the same orbital) or unrestricted (each label its own orbital). (Sections 13.7 and 13.13.)

- **Unspecified function**: Eight real symbols for the coordinates (`x[k]` is x_k: `x[0]` is x_0, `x[4]` is x_4), a positive symbol H, and an unspecified function $a_4(x_4)$: sympy knows nothing about it except that it depends on x_4, so every result below holds for every history, the deflating one included. (Section 21.23.)
- **Unsymmetrised**: Call $K_u = \bar{\Psi}\gamma^\mu D_\mu \Psi = \Psi^\dagger (C\gamma^\mu)(D_\mu \Psi)$ the unsymmetrised kinetic term. (Section 7.20.)
- **Upper**: A pair of index lists is a lower list $(l_1, \ldots, l_p)$ and an upper list $(u_1, \ldots, u_p)$ of the same length. (Sections 11.2 and 11.8.)
- **Upper and lower index**: v^a and v_a are two lists of eight numbers that belong to the same vector (section 6 of Notebook 01e says how one is made from the other). An upper index is a label, not a power: v^3 is a component, and its square is written $(v^3)^2$. (Section 1.32.)
- **Upper index**: A vector v has the components $v^0, \ldots, v^7$, written with an upper index: the component of v along x_3 is v^3, which Python stores as `v[3]` (the author and Python both count from 0). (Section 1.26.)

V

- **Vacuum**: in quantum theory, the state with no particles: every mode empty, written $|0\rangle$. In gravity (Section 20.2), a solution of the vacuum equations, the field equations of gravity with zero source. (Sections 5.34, 5.37, 10.11, 10.18, 10.20, 10.25, 12.2, 12.15, 12.19, 13.4, 18.2 and 20.2.)
- **Vacuum decay**: the quantum transition of a state that has the lowest energy only among its neighbouring states (a false vacuum) into a state of still lower energy; its rate per unit volume is computed from the quantum theory. (Sections 18.2 and 20.2.)
- **Vacuum equations**: the field equations of gravity with zero source, $\sum_k \alpha_k E_{(k)}{}^\mu{}_\nu + \Lambda \delta^\mu_\nu = 0$. (Sections 20.2 and 20.10.)
- **Vacuum value**: Vacuum value $\langle 0|X|0\rangle$ of an operator X and normal ordering $:X:= X - \langle 0|X|0\rangle$ (for an operator built from two field operators; for products of b, b^*, d, d^* this is the same as moving every creation operator to the left of every annihilation operator, with a sign -1 for each exchange). (Section 5.37.)
- **Validation**: a test of a stated uncertainty against a more accurate number. (Section 16.11.)
- **Variable**: A variable is a name for a value; this line gives the name `NOTEBOOK_ID` to the text `"00a"` (a text in quotes is called a string). (Sections 0.13, 1.9, 2.11, 3.13, 4.8, 5.10, 6.14, 7.9, 8.15, 9.17, 10.10, 11.9, 12.16, 14.11, 15.9, 17.11, 18.9, 20.11 and 21.15.)
- **Variation**: a shape $\xi(t)$ with $\xi(0) = \xi(T) = 0$; the varied path is $q + \epsilon\xi$ with a small number ϵ. (Section 7.8.)
- **Variation tensor**: Its result, which we call the variation tensor $T_{\rm var}$, is (the formula is displayed in that section). (Section 9.6.)

- **Variational form**: The solver also computes the variational form, $\sum g f \varepsilon - \langle (M_{in} - m) S_{out} + v_{in} n_{out} \rangle + \langle e_{int}(S_{out}, n_{out}) \rangle$, with the input potentials and output densities of the last iteration; the two agree to 2.2×10^{-13} relative in all 75 ground states (check ground_energy_two_forms). (Section 15.5.)

- **The variational principle for the density**: (We ASSUME that the minimum is attained.) The variational principle for the density then reads $E_0 = \min_n \left(F[n] + \int v\, n\, dx \right)$. (Section 13.8.)

- **Varied path**: To make "changed a little" precise, take a fixed shape $\xi(t)$ with $\xi(0) = \xi(T) = 0$ (so that the changed path keeps the end values) and a small number ϵ, and form the varied path $q + \epsilon\xi$. (Section 7.2.)

- **Vector**: an ordered list of numbers, such as $v = (1, 2, 0)$. (Sections 1.18, 1.24, 2.2, 3.14, 5.12 and 5.21.)

- **Vector components**: a vector $V = V^r e_r + V^\varphi e_\varphi$ is written with the coordinate basis vectors e_r (length 1) and e_φ (length r). (Section 3.21.)

- **Vector matrix** $\Lambda(g)$: the 8×8 matrix with $\alpha(g)\gamma^c g^{-1} = \sum_d \Lambda_{dc} \gamma^d$, where $\alpha(g) = g$ for an even and $-g$ for an odd g (the *twisted* action). (Sections 5.12 and 5.26.)

- **Vector potential**: where the effective mass $M_{\text{eff}} = m + \frac{15}{16}\lambda S$ and the vector potential $v_v = -\frac{1}{16}\lambda n$ are made by all the quanta through their scalar density S and number density n. (Section 14.3.)

- **Vector rule**: Chapter 5 proved the vector rule $[S^{ab}, \gamma^c] = \eta^{bc}\gamma^a - \eta^{ac}\gamma^b$. (Section 6.7.)

- **Vectorised**: Vectorised: one numpy operation acts on whole arrays of cases at once. (Sections 22.11 and 22.12.)

- **Vectorized computation**: numpy applies one operation to millions of numbers in an array at once, much faster than a Python loop. (Section 1.47.)

- **Velocity** $u^a = dx^a/d\tau$: how fast each coordinate changes per unit of proper time. (Sections 3.14, 3.34, 7.2 and 7.8.)

- **Velocity kernel**: so the Lagrangian contains $\frac{i}{2}\sqrt{|g|}\,\Psi^\dagger N \partial_4 \Psi$ (plus its partner term) with the velocity kernel $N = B$. (Section 18.22.)

- **Verdict**: A report lists its checks; each check has a name, a verdict (PASS or FAIL) and a detail text that says exactly what was verified. (Sections 0.4, 0.20 and 23.2.)

- **Verifier** and **checker**: a program of the Revision record that recomputes a result (exactly, with Wolfram or with sympy, or numerically, with Rust or Python) and writes a report: a JSON file that lists named checks, each with a verdict such as PASS. (Sections 0.20 and 23.2.)

- **Version number**: A version number such as 3.14.5 names one release of a program: the first number changes rarely, the second for new features, the third for corrections. (Section 0.8.)

- **Vertex**: the matrix that sits between the field components in an interaction; for dirac16complex the contact interaction is $(\lambda/2)\, S^2$ with $S = \bar{\Psi}\Psi = \Psi^\dagger C\, \Psi$, so the vertex is the 16×16 matrix C. (Sections 13.4 and 13.19.)

- **Vielbein** $e^a{}_\mu$: a matrix with $g_{\mu\nu} = \sum_{a,b} e^a{}_\mu \eta_{ab} e^b{}_\nu$; for a diagonal metric the diagonal matrix of the scale factors. (Sections 3.7, 3.12, 5.28, 6.2, 6.13, 6.21, 7.18, 7.26, 8.2, 12.30, 14.2, 18.19 and 21.22.)
- **Vielbein factors**: The author's metric is diagonal, $g_{\mu\mu} = \eta_{\mu\mu} f_\mu^2$ with the positive vielbein factors (Chapter 3 calls them scale factors h_a; they are the same numbers). (Sections 6.2, 9.2 and 18.3.)
- **Vielbein postulate**: The canonical one is fixed by the vielbein postulate (below). (Sections 6.4, 6.13, 7.18, 8.4, 18.3 and 21.23.)
- **Violation**: the violation of C2. (Section 17.5.)
- **Volume element**: $\sqrt{|g|} = \cos z$; an integral over the hidden direction is $\int \cos z \, (\ldots)\, dx_0$. (Section 10.31.)
- **Volume factor**: This number is the volume factor: a small box with the coordinate sides $dx_0, \ldots, dx_7$ has the sides $f_0\, dx_0, \ldots, f_7\, dx_7$ in length, so its volume is $f_0 f_1 \cdots f_7\, dx_0 \cdots dx_7$, and the product of the length factors is $f_0 \cdots f_7 = \sqrt{|g_{00}| \cdots |g_{77}|} = \sqrt{|\det g|}$ (a product of square roots is the square root of the product). (Sections 0.14, 2.19, 3.6 and 7.18.)

W

- **Wall time**: the time a clock on the wall shows between the start and the end of a command. (Sections 23.2 and 23.11.)
- **Warp factor** $W = \sin^{1/6} z$: the factor that the scale factors of the six transverse directions $x_1, x_2, x_3, x_5, x_6, x_7$ share; the six transverse entries $g_{11}, g_{22}, g_{33}, g_{55}, g_{66}, g_{77}$ of the metric contain its square $W^2 = \sin^{1/3} z$. (Sections 3.7, 3.12, 14.2 and 14.10.)
- **Warped form**: Inserting dy^2 and W^2 into the line element of Section 3.5 gives the warped form. (Sections 3.9 and 14.2.)
- **Wave function** $u(x)$: the eigenfunction of the Schroedinger equation; $u(x)^2$ is the probability density of finding the particle at x, so we *normalise* it: $\int u^2\, dx = 1$. (Sections 2.13, 2.16, 13.1 and 13.2.)
- **Wave function of the universe**: in quantum cosmology, a quantum state assigned to a whole universe, its geometry included: a function of the few numbers that describe the whole geometry, such as a scale factor. It must obey the Wheeler-DeWitt equation. None is formulated for this 8-dimensional theory. (Sections 18.2, 22.2 and 22.4.)
- **Wave number**: A plane wave $\phi = \cos(kx_1)\cos(\omega x_4)$, with wave number k and angular frequency ω, has $\partial_1^2 \phi = -k^2 \phi$ and $\partial_4^2 \phi = -\omega^2 \phi$, so $E = (-k^2 + \omega^2 - m^2)\phi$, which vanishes exactly when. (Sections 7.5 and 7.8.)
- **Weighted average over the hidden direction** $\bar{X}$ (also called the **moment** of X): the integral of $e^{6Hy} X$ over the patch divided by the integral of e^{6Hy}. (Section 17.20.)
- **Weighted mean**: $\int e^{6Hy} p_0\, dy / \int e^{6Hy} dy$, the average of p_0 over the proper volume. (Sections 17.2, 17.7 and 17.20.)

- **Weighted variance**: The bracket is $\sum_a w_a$ times the weighted variance of the numbers $\epsilon_a - \mu$ with the weights w_a, which is never negative (it equals $\sum_a w_a(\epsilon_a - \bar{\epsilon})^2$ with the weighted mean $\bar{\epsilon}$, a sum of non-negative terms). (Section 13.26.)

- **Well posed**: A problem of the kind "given the field at one time, find it at later times" is called well posed (in the sense of the mathematician Jacques Hadamard) when a solution exists, is unique, and changes only a little when the given data change a little. (Sections 4.9 and 8.16.)

- **Wheeler-DeWitt equation**: the equation that the wave function of the universe must obey in quantum cosmology: the analogue, for a geometry, of the Schroedinger equation of a particle. It needs a boundary condition to fix its solution. It is named in this book, not used. (Sections 22.2 and 22.4.)

- **Whole numbers** (integers): $\ldots, -2, -1, 0, 1, 2, \ldots$. Python computes with them exactly, however large they are. (Sections 1.2 and 1.8.)

- **Wick ordered**: There it holds as an exact operator identity, for the solution of the interacting operator field equation, if and only if the potential and the tensor are both Wick ordered (the whole product of field operators is normal ordered at once: the potential as $\frac{\lambda}{2}{:}SS{:}$, not as $\frac{\lambda}{2}{:}S{:}{:}S{:}$, and the tensor as a Wick product, not by subtracting its value in the empty state); with every other ordering tested it fails, already in expectation values (PROVED in that model; its report, named in the table below, passes 21 of 21 checks). (Section 9.13.)

- **Wick's theorem**: $\langle a_p^\dagger a_q^\dagger a_s a_r \rangle = \rho_{rp}\rho_{sq} - \rho_{sp}\rho_{rq}$ for determinants and thermal ensembles of non-interacting fermions. (Sections 13.4 and 13.13.)

- **Winding**: So θ is defined everywhere, and we let it grow continuously, counting every full turn (winding), so that it can exceed 2π. (Section 2.24.)

- **Witness**: an explicit example that proves that something exists. (Sections 9.26, 9.29 and 12.29.)

- **WKB approximation** (after Wentzel, Kramers and Brillouin): when the growth rate κ changes slowly, the size grows like e^{2W} with $W = \int \kappa \, dx_4$. (Sections 8.25 and 8.28.)

- **Wolfram Language**: Engine: the program that did the computation: Wolfram Language (the language of Mathematica, run with wolframscript), Python (with the packages sympy, numpy and mpmath), or Rust (a fast compiled language). (Section 0.20.)

- **WolframScript**: WolframScript (the command `wolframscript`), for the exact Wolfram verifiers. (Section 23.3.)

- **Word**: A word is a list of items in the order of the matrix product: an exponential `("exp", a, b, theta)` or a gamma `("gamma", e)`. (Section 5.27.)

- **Wrapper**: Such a function, which calls another and adds something around it, is called a wrapper. (Section 16.12.)

Z

- **Z2 brane** (Z2 mirror brane): the brane $z = \pi/2$ together with the ASSUMED Z2 construction: the patch is glued there to a mirror copy of itself (the map $z \to \pi - z$; Z2 is the group of two elements, doing nothing and reflecting), and the mirror fixes a boundary condition for the field at that surface (the even or the odd brane parity). It is ASSUMED, not derived; the junction condition at the brane is OPEN. (Sections 8.31, 14.17, 18.2 and 20.2.)

- **Z2 construction**: Gluing the mirror patch to the patch at the brane is the Z2 construction of the Revision record; it is ASSUMED. (Sections 18.2 and 20.2.)

- **Z2 mirror**: a reflection across the edge $z = \pi/2$ of the hidden direction, imposed as a boundary condition; Z2 is the group of two elements, doing nothing and reflecting (Chapter 14). (Sections 0.2, 5.6, 14.12, 17.2 and 22.2.)

- **Z2 mirror brane**: the ASSUMED picture that the patch $y \leq 0$ is glued at $y = 0$ to a mirror copy and the field obeys the parity relation $\Psi(-y) = \pm\gamma^{(x_0)}\Psi(y)$; the relation (not the gluing alone) gives the even parity ($b(0) = 0$) and the odd parity ($a(0) = 0$). (Section 14.17.)

- **Zero modes,** $N = 8$: the eight brane-bound orbitals at zero 3-momentum, $\chi = (a, 0)$ with $a \propto e^{My}$; filling them gives the state $N = 8$. (Sections 2.24, 14.30, 15.2, 15.3, 16.13, 16.18, 19.2, 19.12 and 19.20.)